===== .\app.py ===== import base64 import os import tempfile

import anthropic import cv2 import streamlit as st

====== PENGATURAN ====== # Cek nama model terbaru di: https://docs.anthropic.com/en/docs/about-claude/models MODELS = { "Claude Opus 4.1": "claude-opus-4-1-20250805", "Claude Sonnet 4": "claude-sonnet-4-20250514", # lebih murah }

st.set_page_config(page_title="Claude Pribadi", page_icon=" ", layout="wide")

api_key = os.environ.get("ANTHROPIC_API_KEY") if not api_key: st.error("API key belum diatur. Set variabel ANTHROPIC_API_KEY dulu.") st.stop() client = anthropic.Anthropic(api_key=api_key)

====== STATE ====== if "messages" not in st.session_state: st.session_state.messages = [] # format untuk API if "display" not in st.session_state: st.session_state.display = [] # untuk ditampilkan di layar if "uploader_key" not in st.session_state: st.session_state.uploader_key = 0

====== FUNGSI BANTU ====== def b64(data: bytes) -> str: return base64.b64encode(data).decode("utf-8")

def video_to_frames(file_bytes, suffix, n_frames=10, max_side=1024): """Ambil beberapa frame merata dari video, kembalikan list (detik, base64_jpg)."""" with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as f: f.write(file_bytes) path = f.name cap = cv2.VideoCapture(path) total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) or 1 fps = cap.get(cv2.CAP_PROP_FPS) or 25 frames = [] for i in range(n_frames): idx = int(total * i / n_frames) cap.set(cv2.CAP_PROP_POS_FRAMES, idx) ok, frame = cap.read() if not ok: continue h, w = frame.shape[:2] scale = max_side / max(h, w) if scale < 1: frame = cv2.resize(frame, (int(w * scale), int(h * scale))) ok, buf = cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, 80]) if ok: frames.append((idx / fps, b64(buf.tobytes()))) cap.release() os.remove(path) return frames

def build_content(files, text, n_frames): """Ubah file upload + teks menjadi format konten API Claude."""" content = [] for f in files: data = f.getvalue() mime = f.type or "" if mime in ("image/jpeg", "image/png", "image/gif", "image/webp"): content.append({"type": "image", "source": {"type": "base64", "media_type": mime, "data": b64(data)}}) elif mime == "application/pdf": content.append({"type": "document", "source": {"type": "base64", "media_type": "application/pdf", "data": b64(data)}}) elif mime.startswith("video/"): suffix = os.path.splitext(f.name)[1] or ".mp4" frames = video_to_frames(data, suffix, n_frames) content.append({"type": "text", "text": f"Berikut {len(frames)} frame dari video '{f.name}' secara berurutan " f"(video tanpa audio):"}) for sec, img in frames: content.append({"type": "text", "text": f"Frame pada detik {sec:.1f}:"})

content.append({”type”: ”image”, ”source”: {”type”: ”base64”, ”media_type”: ”image/jpeg”, ”data”: img}}) elif mime.startswith(”text/”) or f.name.endswith((”.py”, ”.md”, ”.csv”, ”.json”, ”.txt”)): content.append({”type”: ”text”, ”text”: f”Isi file '{f.name}':\n\n{data.decode('utf-8', errors='ignore')}”}) content.append({”type”: ”text”, ”text”: text}) return content

====== SIDEBAR ====== with st.sidebar: st.title(” Pengaturan”) model_name = st.selectbox(”Model”, list(MODELS.keys())) system_prompt = st.text_area(”System prompt”, ”Kamu adalah asisten yang membantu. Jawab dalam Bahasa Indonesia.”) max_tokens = st.slider(”Panjang jawaban maks (token)”, 512, 8192, 4096, 512) n_frames = st.slider(”Jumlah frame per video”, 4, 30, 10)

st.divider() files = st.file_uploader(” Lampiran (gambar, PDF, video, teks)”, type=[”jpg”, ”jpeg”, ”png”, ”gif”, ”webp”, ”pdf”, ”mp4”, ”mov”, ”avi”, ”mkv”, ”webm”, ”txt”, ”md”, ”csv”, ”json”, ”py”], accept_multiple_files=True, key=f”uploader_{st.session_state.uploader_key}”,)

if st.button(” Chat baru”, use_container_width=True): st.session_state.messages = [] st.session_state.display = [] st.rerun()

====== TAMPILKAN RIWAYAT ====== st.title(” Claude Pribadi”) for role, text, attachments in st.session_state.display: with st.chat_message(role): if attachments: st.caption(” ” + ”, ”.join(attachments)) st.markdown(text)

====== INPUT CHAT ====== if prompt := st.chat_input(”Ketik pesan...”): files = files or [] names = [f.name for f in files]

with st.chat_message(”user”): if names: st.caption(” ” + ”, ”.join(names)) for f in files: if f.type and f.type.startswith(”image/”): st.image(f, width=250) st.markdown(prompt)

with st.spinner(”Memproses lampiran...”): content = build_content(files, prompt, n_frames) st.session_state.messages.append({”role”: ”user”, ”content”: content}) st.session_state.display.append((”user”, prompt, names))

with st.chat_message(”assistant”): try: def stream_reply(): with client.messages.stream(model=MODELS[model_name], max_tokens=max_tokens, system=system_prompt, messages=st.session_state.messages,) as stream: for chunk in stream.text_stream: yield chunk

reply = st.write_stream(stream_reply()) st.session_state.messages.append({”role”: ”assistant”, ”content”: reply}) st.session_state.display.append((”assistant”, reply, [])) except anthropic.APIError as e: st.error(f”Error dari API: {e}”) st.session_state.messages.pop() # buang pesan gagal st.session_state.display.pop()

kosongkan uploader setelah terkirim st.session_state.uploader_key += 1 st.rerun()

===== .\gabung.py ===== # gabung.py → jalankan: python gabung.py

import os EXT = {".py", ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".json", ".env.example"} SKIP = {"node_modules", ".git", "venv", "__pycache__", "dist", "build"}

with open("semua_kode.txt", "w", encoding="utf-8") as out: for root, dirs, files in os.walk("."): dirs[:] = [d for d in dirs if d not in SKIP] for f in files: if os.path.splitext(f)[1] in EXT and f != "package-lock.json": path = os.path.join(root, f) out.write(f"\n\n===== {path} =====\n") out.write(open(path, encoding="utf-8", errors="ignore").read()) print("Selesai: semua_kode.txt")

===== .\main.py ===== import os, sys, sqlite3, tempfile, subprocess, uuid, shutil, threading, webbrowser from pathlib import Path from typing import Optional

BASE_DIR = Path(__file__).resolve().parent

def berhenti(pesan): print("\n" + "=" * 60) print(pesan) print("=" * 60) input("\nTekan Enter untuk keluar...") sys.exit(1)

---------- Cek versi Python ---------- if sys.version_info < (3, 9): berhenti(" Versi Python terlalu lama. Install Python 3.10 atau lebih baru dari python.org")

---------- Cek paket ---------- try: from dotenv import load_dotenv from fastapi import FastAPI, UploadFile, File, HTTPException from fastapi.staticfiles import StaticFiles from pydantic import BaseModel import anthropic import uvicorn import multipart # dibutuhkan untuk upload file except ImportError as e: berhenti(f" Paket belum terinstall: {e.name}\n\n" f"Jalankan perintah ini di terminal (di folder proyek):\n\n" f" pip install -r requirements.txt")

from pricing import hitung_biaya

---------- Lokasi folder data ---------- # Pakai D:\claude-platform kalau drive D: ada, kalau tidak pakai folder 'data' di sini if os.getenv("DATA_DIR"): DATA_DIR = Path(os.getenv("DATA_DIR")) elif Path("D:/").exists(): DATA_DIR = Path(r"D:\claude-platform") else: DATA_DIR = BASE_DIR / "data"

MODEL_DIR = DATA_DIR / "models" / "whisper" TMP_DIR = DATA_DIR / "tmp" try: for d in (MODEL_DIR, TMP_DIR, DATA_DIR / ".cache"): d.mkdir(parents=True, exist_ok=True) except Exception as e: berhenti(f" Gagal membuat folder {DATA_DIR}: {e}")

os.environ.setdefault("HF_HOME", str(DATA_DIR / ".cache" / "huggingface"))

---------- Cari ffmpeg ---------- def cari_ffmpeg(): # 1. Dari variabel FFMPEG_PATH (opsional, bisa diisi di .env) p = os.getenv("FFMPEG_PATH") if p and Path(p).exists(): return p # 2. Dari PATH sistem p = shutil.which("ffmpeg") if p: return p # 3. Lokasi umum di Windows (kalau PATH belum ter-refresh) local = os.getenv("LOCALAPPDATA", "")

user = os.getenv(”USERPROFILE”, ””) kandidat = [DATA_DIR / ”ffmpeg” / ”bin” / ”ffmpeg.exe”, Path(r”C:\ffmpeg\bin\ffmpeg.exe”), Path(r”C:\Program Files\ffmpeg\bin\ffmpeg.exe”), Path(r”C:\ProgramData\chocolatey\bin\ffmpeg.exe”), Path(local) / ”Microsoft” / ”WinGet” / ”Links” / ”ffmpeg.exe”, Path(user) / ”scoop” / ”shims” / ”ffmpeg.exe”,] for k in kandidat: if k.exists(): return str(k) # 4. Folder paket winget (nama foldernya berisi nomor versi) winget = Path(local) / ”Microsoft” / ”WinGet” / ”Packages” if winget.exists(): for k in winget.glob(”*FFmpeg*/**/bin/ffmpeg.exe”): return str(k) return None

def whisper_terpasang(): try: import faster_whisper # noqa: F401 return True except ImportError: return False

---------- API key ---------- ENV_FILE = BASE_DIR / ”.env” if not ENV_FILE.exists(): ENV_FILE.write_text(”ANTHROPIC_API_KEY=\n”, encoding=”utf-8”) berhenti(f” File .env belum ada, sudah saya buatkan di:\n {ENV_FILE}\n\n” f”Buka dengan Notepad, isi menjadi:\n ANTHROPIC_API_KEY=sk-ant-xxxxxxxx\n” f”lalu simpan dan jalankan lagi.”)

load_dotenv(ENV_FILE) if not os.getenv(”ANTHROPIC_API_KEY”, ””).strip(): berhenti(f” ANTHROPIC_API_KEY masih kosong di file:\n {ENV_FILE}\n\n” f”Isi menjadi:\n ANTHROPIC_API_KEY=sk-ant-xxxxxxxx”)

---------- Cek folder static ---------- STATIC_DIR = BASE_DIR / ”static” if not (STATIC_DIR / ”index.html”).exists(): berhenti(f” File tidak ditemukan:\n {STATIC_DIR / 'index.html'}\n\n” f”Buat folder 'static' dan taruh index.html di dalamnya.”)

client = anthropic.Anthropic() DEFAULT_MODEL = os.getenv(”CLAUDE_MODEL”, ”claude-opus-4-1”) app = FastAPI()

---------- Database riwayat ---------- db = sqlite3.connect(BASE_DIR / ”chat.db”, check_same_thread=False) db.executescript(””” CREATE TABLE IF NOT EXISTS conversations(id TEXT PRIMARY KEY, title TEXT, created_at DATETIME DEFAULT CURRENT_TIMESTAMP); CREATE TABLE IF NOT EXISTS messages(id INTEGER PRIMARY KEY AUTOINCREMENT, conversation_id TEXT, role TEXT, content TEXT, model TEXT, input_tokens INT, output_tokens INT, cost_usd REAL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP); ”””) db_lock = threading.Lock()

def buat_system_prompt(): ff = cari_ffmpeg() wh = whisper_terpasang() status_ff = f”terpasang ({ff})” if ff else ”BELUM terpasang” status_wh = ”terpasang” if wh else ”BELUM terpasang” teks = (”Kamu asisten dalam aplikasi chat pribadi. Jawab dalam Bahasa Indonesia. ” ”Kamu tidak bisa menjalankan perintah di komputer pengguna. ” ”Aplikasi ini punya tombol untuk transkripsi audio/video (Whisper + ffmpeg). ” f”Status saat ini: ffmpeg {status_ff}, faster-whisper {status_wh}. ”) if ff and wh: teks += (”Semua alat sudah siap,

jadi jangan menyuruh pengguna menginstal ulang. " "Jika pengguna ingin transkripsi, arahkan untuk menekan tombol . ") else: teks += "Untuk alat yang belum terpasang, sarankan pengguna menjalankan: python setup_tools.py " teks += ("Jika pengguna meminta perubahan kode, berikan file LENGKAP yang siap ditimpa " "(replace all), bukan potongan per bagian.") return teks

class ChatIn(BaseModel): conversation_id: Optional[str] = None message: str model: Optional[str] = None

@app.get("/models") def models(): try: return [m.id for m in client.models.list()] except anthropic.APIError as e: raise HTTPException(502, f"Gagal ambil daftar model: {e}")

@app.get("/status") def status(): return {"ffmpeg": cari_ffmpeg(), "faster_whisper": whisper_terpasang(), "data_dir": str(DATA_DIR)}

@app.post("/chat") def chat(body: ChatIn): model = body.model or DEFAULT_MODEL cid = body.conversation_id or str(uuid.uuid4())

with db_lock: rows = db.execute("SELECT role,content FROM messages WHERE conversation_id=? ORDER BY id", (cid,)).fetchall() history = [{"role": r, "content": c} for r, c in rows] history.append({"role": "user", "content": body.message})

try: resp = client.messages.create(model=model, max_tokens=8192, system=buat_system_prompt(), messages=history) except anthropic.APIError as e: raise HTTPException(502, f"Error dari Claude API: {e}")

reply = "".join(b.text for b in resp.content if b.type == "text") u = resp.usage biaya = hitung_biaya(model, u.input_tokens, u.output_tokens)

with db_lock: if not body.conversation_id: db.execute("INSERT INTO conversations(id,title) VALUES(?,?)", (cid, body.message[:50])) db.execute("INSERT INTO messages(conversation_id,role,content) VALUES(?,?,?)", (cid, "user", body.message)) db.execute("""INSERT INTO messages(conversation_id,role,content,model, input_tokens,output_tokens,cost_usd) VALUES(?,?,?,?,?,?,?)""", (cid, "assistant", reply, model, u.input_tokens, u.output_tokens, biaya["usd"])) db.commit()

return {"conversation_id": cid, "reply": reply, "usage": {"input_tokens": u.input_tokens, "output_tokens": u.output_tokens}, "cost": biaya}

@app.get("/conversations") def list_conversations(): with db_lock: rows = db.execute("""SELECT c.id, c.title, c.created_at, COALESCE(SUM(m.cost_usd),0) FROM conversations c LEFT JOIN messages m ON m.conversation_id=c.id GROUP BY c.id ORDER BY c.created_at DESC""").fetchall() return [{"id": r[0], "title": r[1], "created_at": r[2], "total_usd": round(r[3], 6)} for r in rows]

@app.get("/conversations/{cid}") def get_conversation(cid: str): with db_lock: rows = db.execute("""SELECT role,content,model,input_tokens,output_tokens,cost_usd,created_a FROM messages WHERE conversation_id=? ORDER BY id""", (cid,)).fetchall()

keys = [”role”, ”content”, ”model”, ”input_tokens”, ”output_tokens”, ”cost_usd”, ”created_at”] return [dict(zip(keys, r)) for r in rows]

@app.delete(”/conversations/{cid}”) def delete_conversation(cid: str): with db_lock: db.execute(”DELETE FROM messages WHERE conversation_id=?”, (cid,)) db.execute(”DELETE FROM conversations WHERE id=?”, (cid,)) db.commit() return {”ok”: True}

---------- Transkripsi audio/video ---------- _whisper = None _whisper_lock = threading.Lock()

def get_whisper(): global _whisper with _whisper_lock: if _whisper is None: try: from faster_whisper import WhisperModel except ImportError: raise HTTPException(500, ”Paket faster-whisper belum terinstall. ” ”Jalankan: pip install faster-whisper”) _whisper = WhisperModel(”small”, device=”auto”, compute_type=”int8”, download_root=str(MODEL_DIR)) return _whisper

@app.post(”/transcribe”) def transcribe(file: UploadFile = File(...)): ffmpeg = cari_ffmpeg() if not ffmpeg: raise HTTPException(500, ”ffmpeg tidak ditemukan. Tutup server, buka terminal BARU, ” ”lalu cek dengan perintah: ffmpeg -version. ” ”Atau isi FFMPEG_PATH=C:\\lokasi\\ffmpeg.exe di file .env”) with tempfile.TemporaryDirectory(dir=TMP_DIR) as tmp: src = os.path.join(tmp, ”input” + Path(file.filename or ””).suffix) with open(src, ”wb”) as f: shutil.copyfileobj(file.file, f) wav = os.path.join(tmp, ”audio.wav”) try: subprocess.run([ffmpeg, ”-y”, ”-i”, src, ”-vn”, ”-ar”, ”16000”, ”-ac”, ”1”, wav], check=True, capture_output=True) except subprocess.CalledProcessError as e: raise HTTPException(400, ”ffmpeg gagal: ” + e.stderr.decode(errors=”ignore”)[-500:]) segments, info = get_whisper().transcribe(wav, language=None) segs = [{”start”: round(s.start, 1), ”end”: round(s.end, 1), ”text”: s.text.strip()} for s in segments] return {”language”: info.language, ”text”: ” ”.join(s[”text”] for s in segs), ”segments”: segs}

app.mount(”/”, StaticFiles(directory=STATIC_DIR, html=True), name=”static”)

if __name__ == ”__main__”: PORT = 8000 ff = cari_ffmpeg() print(”=” * 60) print(f” Server berjalan. Buka di browser: http://localhost:{PORT}”) print(f” Folder data : {DATA_DIR}”) print(f” ffmpeg : {ff or ' TIDAK DITEMUKAN'}”) print(f” faster-whisper: {' terpasang' if whisper_terpasang() else ' belum (pip install faster-whisper)'}”) print(” (Jangan tutup jendela ini selama aplikasi dipakai)”) print(”=” * 60) threading.Timer(2, lambda: webbrowser.open(f”http://localhost:{PORT}”)).start() try: uvicorn.run(app, host=”127.0.0.1”, port=PORT) except Exception as e: berhenti(f” Server gagal jalan: {e}\n\n” f”Kemungkinan port {PORT} sedang dipakai program lain.”)

===== .\pricing.py ===== # Harga USD per 1 juta token. CEK ULANG di https://www.anthropic.com/pricing # karena harga bisa berubah. PRICING = { ”claude-opus-4-5”: {”input”: 5.00, ”output”: 25.00}, ”claude-opus-4-1”:

{”input”: 15.00, ”output”: 75.00}, ”claude-opus-4”: {”input”: 15.00, ”output”: 75.00}, ”claude-sonnet-4-5”: {”input”: 3.00, ”output”: 15.00}, ”claude-sonnet-4”: {”input”: 3.00, ”output”: 15.00}, ”claude-haiku-4-5”: {”input”: 1.00, ”output”: 5.00}, ”claude-3-5-haiku”: {”input”: 0.80, ”output”: 4.00}, ”claude-3-haiku”: {”input”: 0.25, ”output”: 1.25}, } USD_TO_IDR = 16000 # sesuaikan kurs

def hitung_biaya(model: str, input_tokens: int, output_tokens: int) -> dict: # cek nama terpanjang dulu supaya ”claude-opus-4-1” tidak tertukar dengan ”claude-opus-4” p = None for k, v in sorted(PRICING.items(), key=lambda kv: -len(kv[0])): if model.startswith(k): p = v break if not p: return {”usd”: None, ”idr”: None} usd = (input_tokens * p[”input”] + output_tokens * p[”output”]) / 1_000_000 return {”usd”: round(usd, 6), ”idr”: round(usd * USD_TO_IDR, 2)}

===== .\setup_tools.py ===== # setup_tools.py → jalankan sekali: python setup_tools.py import os, sys, subprocess, zipfile, urllib.request, shutil from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent

Lokasi data harus sama dengan logika di main.py if os.getenv(”DATA_DIR”): DATA_DIR = Path(os.getenv(”DATA_DIR”)) elif Path(”D:/”).exists(): DATA_DIR = Path(r”D:\claude-platform”) else: DATA_DIR = BASE_DIR / ”data”

MODEL_DIR = DATA_DIR / ”models” / ”whisper” FFMPEG_BIN = DATA_DIR / ”ffmpeg” / ”bin” TMP_DIR = DATA_DIR / ”tmp” for d in (MODEL_DIR, FFMPEG_BIN, TMP_DIR): d.mkdir(parents=True, exist_ok=True) os.environ.setdefault(”HF_HOME”, str(DATA_DIR / ”.cache” / ”huggingface”))

FFMPEG_URL = ”https://www.gyan.dev/ffmpeg/builds/ffmpeg-release-essentials.zip”

def langkah(t): print(”\n” + ”=” * 60 + f”\n{t}\n” + ”=” * 60)

def progress(blok, ukuran_blok, total): if total > 0: persen = min(100, blok * ukuran_blok * 100 // total) print(f”\r Mengunduh... {persen}%”, end=””, flush=True)

1. Paket Python langkah(”1/3 Menginstal paket Python (faster-whisper)...”) subprocess.check_call([sys.executable, ”-m”, ”pip”, ”install”, ”faster-whisper”])

2. ffmpeg langkah(”2/3 Menyiapkan ffmpeg...”) if (FFMPEG_BIN / ”ffmpeg.exe”).exists() or shutil.which(”ffmpeg”): print(” ffmpeg sudah ada, dilewati.”) else: zip_path = TMP_DIR / ”ffmpeg.zip” urllib.request.urlretrieve(FFMPEG_URL, zip_path, progress) print(”\n Mengekstrak...”) with zipfile.ZipFile(zip_path) as z: for nama in z.namelist(): if nama.endswith((”bin/ffmpeg.exe”, ”bin/ffprobe.exe”)): tujuan = FFMPEG_BIN / Path(nama).name with z.open(nama) as src, open(tujuan, ”wb”)

as dst: shutil.copyfileobj(src, dst) zip_path.unlink() print(f" ffmpeg terpasang di {FFMPEG_BIN}")

3. Model Whisper (unduh sekarang supaya transkripsi pertama tidak lama) langkah("3/3 Mengunduh model Whisper 'small' (±460MB)...") from faster_whisper import WhisperModel WhisperModel("small", device="cpu", compute_type="int8", download_root=str(MODEL_DIR)) print(" Model siap.")

langkah(" Selesai! Sekarang jalankan: python main.py") input("\nTekan Enter untuk keluar...")

===== .\ffmpeg\doc\bootstrap.min.css =====

/*!

* Bootstrap v3.2.0 (http://getbootstrap.com)

* Copyright 2011-2014 Twitter, Inc.

* Licensed under MIT (https://github.com/twbs/bootstrap/blob/master/LICENSE)

//! normalize.css v3.0.1 | MIT License | git.io/normalize */html{font-family:sans-serif;-webkit-text-size-adjust:100%;-ms-text-size-adjust:100%}body{margin:0}article,aside,details,f block;vertical-align:baseline}audio:not([controls]){display:none;height:0}[hidden],template{display:none}a{bac 0}a:active,a:hover{outline:0}abbr[title]{border-bottom:1px dotted}b,strong{font-weight:700}dfn{font-style:italic}h1{margin:.67em 0;font-size:2em}mark{color:#000;background:#ff0}small{fo size:80%}sub,sup{position:relative;font-size:75%;line-height:0;vertical-align:baseline}sup{top:-.5em}sub{bottom:-.25em}img{border:0}svg:not(:root){overflow:hidden}figure{margin:1em 40px}hr{height:0;-webkit-box-sizing:content-box;-moz-box-sizing:content-box;box-sizing:content-box}pre{overflow:auto}code,kbd,pre,samp{font-family:monospace,monospace;font-size:1em}button,input,optgroup,select,textarea{margin:0;font:inherit;color:i transform:none}button,html input[type=button],input[type=reset],input[type=submit]{-webkit-appearance:button;cursor:pointer}button[disabled],html input[disabled]{cursor:default}button::-moz-focus-inner,input::-moz-focus-inner{padding:0;border:0}input{line-height:normal}input[type=checkbox],input[type=radio]{-webkit-box-sizing:border-box;-moz-box-sizing:border-box;box-sizing:border-box;padding:0}input[type=number]::-webkit-inner-spin-button,input[type=number]::-webkit-outer-spin-button{height:auto}input[type=search]{-webkit-box-sizing:content-box;-moz-box-sizing:content-box;box-sizing:content-box;-webkit-appearance:textfield}input[type=search]::-webkit-search-cancel-button,input[type=search]::-webkit-search-decoration{-webkit-appearance:none}fieldset{padding:.35em .625em .75em;margin:0 2px;border:1px solid silver}legend{padding:0;border:0}textarea{overflow:auto}optgroup weight:700}table{border-spacing:0;border-collapse:collapse}td,th{padding:0}@media print{*{color:#000!important;text-shadow:none!important;background:transparent!important;-webkit-box-shadow:none!important;box-shadow:none!important}a,a:visited{text-decoration:underline}a[href]:after{content:" (" attr(href) ")"}abbr[title]:after{content:" (" attr(title) ")"}a[href^="javascript:"]:after,a[href^="#"]:after{content:""}pre,blockquote{border:1px solid #999;page-break-inside:avoid}thead{display:table-header-group}tr,img{page-break-inside:avoid}img{max-width:100%!important}p,h2,h3{orphans:3;widows:3}h2,h3{page-break-after:avoid}select{background:#fff!important}.navbar{display:none}.table

td,.table th{background-color:#fff!important}.btn>.caret,.dropup>.btn>.caret{border-top-color:#000!important}.label{border:1px solid #000}.table{border-collapse:collapse!important}.table-bordered th,.table-bordered td{border:1px solid #ddd!important}}@font-face{font-family:'Glyphicons Halflings';src:url(../fonts/glyphicons-halflings-regular.eot);src:url(../fonts/glyphicons-halflings-regular.eot?#iefix) format('embedded-opentype'),url(../fonts/glyphicons-halflings-regular.woff) format('woff'),url(../fonts/glyphicons-halflings-regular.ttf) format('truetype'),url(../fonts/glyphicons-halflings-regular.svg#glyphicons_halflingsregular) format('svg')}.glyphicon{position:relative;top:1px;display:inline-block;font-family:'Glyphicons Halflings';font-style:normal;font-weight:400;line-height:1;-webkit-font-smoothing:antialiased;-moz-osx-font-smoothing:grayscale}.glyphicon-asterisk:before{content:"\2a"}.glyphicon-plus:before{content:"\2b"}.glyphicon-euro:before{content:"\20ac"}.glyphicon-minus:before{content:"\2212"}.glyphicon-cloud:before{content:"\2601"}.glyphicon-envelope:before{content:"\2709"}.glyphicon-pencil:before{content:"\270f"}.glyphicon-glass:before{content:"\e001"}.glyphicon-music:before{content:"\e002"}.glyphicon-search:before{content:"\e003"}.glyphicon-heart:before{content:"\e005"}.glyphicon-star:before{content:"\e006"}.glyphicon-star-empty:before{content:"\e007"}.glyphicon-user:before{content:"\e008"}.glyphicon-film:before{content:"\e009"}.glyphicon-th-large:before{content:"\e010"}.glyphicon-th:before{content:"\e011"}.glyphicon-th-list:before{content:"\e012"}.glyphicon-ok:before{content:"\e013"}.glyphicon-remove:before{content:"\e014"}.glyphicon-zoom-in:before{content:"\e015"}.glyphicon-zoom-out:before{content:"\e016"}.glyphicon-off:before{content:"\e017"}.glyphicon-signal:before{content:"\e018"}.glyphicon-cog:before{content:"\e019"}.glyphicon-trash:before{content:"\e020"}.glyphicon-home:before{content:"\e021"}.glyphicon-file:before{content:"\e022"}.glyphicon-time:before{content:"\e023"}.glyphicon-road:before{content:"\e024"}.glyphicon-download-alt:before{content:"\e025"}.glyphicon-download:before{content:"\e026"}.glyphicon-upload:before{content:"\e027"}.glyphicon-inbox:before{content:"\e028"}.glyphicon-play-circle:before{content:"\e029"}.glyphicon-repeat:before{content:"\e030"}.glyphicon-refresh:before{content:"\e031"}.glyphicon-list-alt:before{content:"\e032"}.glyphicon-lock:before{content:"\e033"}.glyphicon-flag:before{content:"\e034"}.glyphicon-headphones:before{content:"\e035"}.glyphicon-volume-off:before{content:"\e036"}.glyphicon-volume-down:before{content:"\e037"}.glyphicon-volume-up:before{content:"\e038"}.glyphicon-qrcode:before{content:"\e039"}.glyphicon-barcode:before{content:"\e040"}.glyphicon-tag:before{content:"\e041"}.glyphicon-tags:before{content:"\e042"}.glyphicon-book:before{content:"\e043"}.glyphicon-bookmark:before{content:"\e044"}.glyphicon-print:before{content:"\e045"}.glyphicon-camera:before{content:"\e046"}.glyphicon-font:before{content:"\e047"}.glyphicon-bold:before{content:"\e048"}.glyphicon-italic:before{content:"\e049"}.glyphicon-text-height:before{content:"\e050"}.glyphicon-text-width:before{content:"\e051"}.glyphicon-align-left:before{content:"\e052"}.glyphicon-align-center:before{content:"\e053"}.glyphicon-align-right:before{content:"\e054"}.glyphicon-align-justify:before{content:"\e055"}.glyphicon-list:before{content:"\e056"}.glyphicon-indent-left:before{content:"\e057"}.glyphicon-indent-right:before{content:"\e058"}.glyphicon-facetime-video:before{content:"\e059"}.glyphicon-picture:before{content:"\e060"}.glyphicon-map-marker:before{content:"\e062"}.glyphicon-adjust:before{content:"\e063"}.glyphicon-tint:before{content:"\e064"}.glyphicon-edit:before{content:"\e065"}.glyphicon-share:before{content:"\e066"}.glyphicon-check:before{content:"\e067"}.glyphicon-

move:before{content:"\e068"}.glyphicon-step-backward:before{content:"\e069"}.glyphicon-fast-backward:before{content:"\e070"}.glyphicon-backward:before{content:"\e071"}.glyphicon-play:before{content:"\e072"}.glyphicon-pause:before{content:"\e073"}.glyphicon-stop:before{content:"\e074"}.glyphicon-forward:before{content:"\e075"}.glyphicon-fast-forward:before{content:"\e076"}.glyphicon-step-forward:before{content:"\e077"}.glyphicon-eject:before{content:"\e078"}.glyphicon-chevron-left:before{content:"\e079"}.glyphicon-chevron-right:before{content:"\e080"}.glyphicon-plus-sign:before{content:"\e081"}.glyphicon-minus-sign:before{content:"\e082"}.glyphicon-remove-sign:before{content:"\e083"}.glyphicon-ok-sign:before{content:"\e084"}.glyphicon-question-sign:before{content:"\e085"}.glyphicon-info-sign:before{content:"\e086"}.glyphicon-screenshot:before{content:"\e087"}.glyphicon-remove-circle:before{content:"\e088"}.glyphicon-ok-circle:before{content:"\e089"}.glyphicon-ban-circle:before{content:"\e090"}.glyphicon-arrow-left:before{content:"\e091"}.glyphicon-arrow-right:before{content:"\e092"}.glyphicon-arrow-up:before{content:"\e093"}.glyphicon-arrow-down:before{content:"\e094"}.glyphicon-share-alt:before{content:"\e095"}.glyphicon-resize-full:before{content:"\e096"}.glyphicon-resize-small:before{content:"\e097"}.glyphicon-exclamation-sign:before{content:"\e101"}.glyphicon-gift:before{content:"\e102"}.glyphicon-leaf:before{content:"\e103"}.glyphicon-fire:before{content:"\e104"}.glyphicon-eye-open:before{content:"\e105"}.glyphicon-eye-close:before{content:"\e106"}.glyphicon-warning-sign:before{content:"\e107"}.glyphicon-plane:before{content:"\e108"}.glyphicon-calendar:before{content:"\e109"}.glyphicon-random:before{content:"\e110"}.glyphicon-comment:before{content:"\e111"}.glyphicon-magnet:before{content:"\e112"}.glyphicon-chevron-up:before{content:"\e113"}.glyphicon-chevron-down:before{content:"\e114"}.glyphicon-retweet:before{content:"\e115"}.glyphicon-shopping-cart:before{content:"\e116"}.glyphicon-folder-close:before{content:"\e117"}.glyphicon-folder-open:before{content:"\e118"}.glyphicon-resize-vertical:before{content:"\e119"}.glyphicon-resize-horizontal:before{content:"\e120"}.glyphicon-hdd:before{content:"\e121"}.glyphicon-bullhorn:before{content:"\e122"}.glyphicon-bell:before{content:"\e123"}.glyphicon-certificate:before{content:"\e124"}.glyphicon-thumbs-up:before{content:"\e125"}.glyphicon-thumbs-down:before{content:"\e126"}.glyphicon-hand-right:before{content:"\e127"}.glyphicon-hand-left:before{content:"\e128"}.glyphicon-hand-up:before{content:"\e129"}.glyphicon-hand-down:before{content:"\e130"}.glyphicon-circle-arrow-right:before{content:"\e131"}.glyphicon-circle-arrow-left:before{content:"\e132"}.glyphicon-circle-arrow-up:before{content:"\e133"}.glyphicon-circle-arrow-down:before{content:"\e134"}.glyphicon-globe:before{content:"\e135"}.glyphicon-wrench:before{content:"\e136"}.glyphicon-tasks:before{content:"\e137"}.glyphicon-filter:before{content:"\e138"}.glyphicon-briefcase:before{content:"\e139"}.glyphicon-fullscreen:before{content:"\e140"}.glyphicon-dashboard:before{content:"\e141"}.glyphicon-paperclip:before{content:"\e142"}.glyphicon-heart-empty:before{content:"\e143"}.glyphicon-link:before{content:"\e144"}.glyphicon-phone:before{content:"\e145"}.glyphicon-pushpin:before{content:"\e146"}.glyphicon-usd:before{content:"\e148"}.glyphicon-gbp:before{content:"\e149"}.glyphicon-sort:before{content:"\e150"}.glyphicon-sort-by-alphabet:before{content:"\e151"}.glyphicon-sort-by-alphabet-alt:before{content:"\e152"}.glyphicon-sort-by-order:before{content:"\e153"}.glyphicon-sort-by-order-alt:before{content:"\e154"}.glyphicon-sort-by-attributes:before{content:"\e155"}.glyphicon-sort-by-attributes-alt:before{content:"\e156"}.glyphicon-unchecked:before{content:"\e157"}.glyphicon-expand:before{content:"\e158"}.glyphicon-collapse-down:before{content:"\e159"}.glyphicon-collapse-up:before{content:"\e160"}.glyphicon-log-in:before{content:"\e161"}.glyphicon-flash:before{content:"\e162"}.glyphicon-log-out:before{content:"\e163"}.glyphicon-

new-window:before{content:"\e164"}.glyphicon-record:before{content:"\e165"}.glyphicon-
save:before{content:"\e166"}.glyphicon-open:before{content:"\e167"}.glyphicon-
saved:before{content:"\e168"}.glyphicon-import:before{content:"\e169"}.glyphicon-
export:before{content:"\e170"}.glyphicon-send:before{content:"\e171"}.glyphicon-
floppy-disk:before{content:"\e172"}.glyphicon-floppy-saved:before{content:"\e173"}.glyphicon-
floppy-remove:before{content:"\e174"}.glyphicon-floppy-save:before{content:"\e175"}.glyphicon-
floppy-open:before{content:"\e176"}.glyphicon-credit-card:before{content:"\e177"}.glyphicon-
transfer:before{content:"\e178"}.glyphicon-cutlery:before{content:"\e179"}.glyphicon-
header:before{content:"\e180"}.glyphicon-compressed:before{content:"\e181"}.glyphicon-
earphone:before{content:"\e182"}.glyphicon-phone-alt:before{content:"\e183"}.glyphicon-
tower:before{content:"\e184"}.glyphicon-stats:before{content:"\e185"}.glyphicon-
sd-video:before{content:"\e186"}.glyphicon-hd-video:before{content:"\e187"}.glyphicon-
subtitles:before{content:"\e188"}.glyphicon-sound-stereo:before{content:"\e189"}.glyphicon-
sound-dolby:before{content:"\e190"}.glyphicon-sound-5-1:before{content:"\e191"}.glyphicon-
sound-6-1:before{content:"\e192"}.glyphicon-sound-7-1:before{content:"\e193"}.glyphicon-
copyright-mark:before{content:"\e194"}.glyphicon-registration-mark:before{content:"\e195"}.glyphicon-
cloud-download:before{content:"\e197"}.glyphicon-cloud-upload:before{content:"\e198"}.glyphicon-
tree-conifer:before{content:"\e199"}.glyphicon-tree-deciduous:before{content:"\e200"}*{-
webkit-box-sizing:border-box;-moz-box-sizing:border-box;box-sizing:border-
box}:before,:after{-webkit-box-sizing:border-box;-moz-box-sizing:border-
box;box-sizing:border-box}html{font-size:10px;-webkit-tap-highlight-color:rgba(0,0,0,0)}body{font-
family:"Helvetica Neue",Helvetica,Arial,sans-serif;font-size:14px;line-height:1.42857143;color:#333;background-
color:#fff}input,button,select,textarea{font-family:inherit;font-size:inherit;line-
height:inherit}a{color:#428bca;text-decoration:none}a:hover,a:focus{color:#2a6496;text-
decoration:underline}a:focus{outline:thin dotted;outline:5px auto -webkit-focus-
ring-color;outline-offset:-2px}figure{margin:0}img{vertical-align:middle}.img-
responsive,.thumbnail>img,.thumbnail a>img,.carousel-inner>.item>img,.carousel-
inner>.item>a>img{display:block;width:100% \9;max-width:100%;height:auto}.img-
rounded{border-radius:6px}.img-thumbnail{display:inline-block;width:100%
\9;max-width:100%;height:auto;padding:4px;line-height:1.42857143;background-
color:#fff;border:1px solid #ddd;border-radius:4px;-webkit-transition:all .2s
ease-in-out;-o-transition:all .2s ease-in-out;transition:all .2s ease-in-out}.img-
circle{border-radius:50%}hr{margin-top:20px;margin-bottom:20px;border:0;border-
top:1px solid #eee}.sr-only{position:absolute;width:1px;height:1px;padding:0;margin:-
1px;overflow:hidden;clip:rect(0,0,0,0);border:0}.sr-only-focusable:active,.sr-
only-focusable:focus{position:static;width:auto;height:auto;margin:0;overflow:visible;clip:auto}h1,h2,h3,h4,h5,h
family:inherit;font-weight:500;line-height:1.1;color:inherit}h1 small,h2 small,h3
small,h4 small,h5 small,h6 small,.h1 small,.h2 small,.h3 small,.h4 small,.h5
small,.h6 small,h1 .small,h2 .small,h3 .small,h4 .small,h5 .small,h6 .small,.h1
.small,.h2 .small,.h3 .small,.h4 .small,.h5 .small,.h6 .small{font-weight:400;line-
height:1;color:#777}h1,.h1,h2,.h2,h3,.h3{margin-top:20px;margin-bottom:10px}h1
small,.h1 small,h2 small,.h2 small,h3 small,.h3 small,h1 .small,.h1 .small,h2
.small,.h2 .small,h3 .small,.h3 .small{font-size:65%}h4,.h4,h5,.h5,h6,.h6{margin-
top:10px;margin-bottom:10px}h4 small,.h4 small,h5 small,.h5 small,h6
small,.h6 small,h4 .small,.h4 .small,h5 .small,.h5 .small,h6 .small,.h6
.small{font-size:75%}h1,.h1{font-size:36px}h2,.h2{font-size:30px}h3,.h3{font-

size:24px}h4,.h4{font-size:18px}h5,.h5{font-size:14px}h6,.h6{font-size:12px}p{margin:0
0 10px}.lead{margin-bottom:20px;font-size:16px;font-weight:300;line-height:1.4}@media
(min-width:768px){.lead{font-size:21px}}small,.small{font-size:85%}cite{font-
style:normal}mark,.mark{padding:.2em;background-color:#fcf8e3}.text-
left{text-align:left}.text-right{text-align:right}.text-center{text-align:center}.text-
justify{text-align:justify}.text-nowrap{white-space:nowrap}.text-lowercase{text-
transform:lowercase}.text-uppercase{text-transform:uppercase}.text-capitalize{text-
transform:capitalize}.text-muted{color:#777}.text-primary{color:#428bca}a.text-
primary:hover{color:#3071a9}.text-success{color:#3c763d}a.text-success:hover{color:#2b542c}.text-
info{color:#31708f}a.text-info:hover{color:#245269}.text-warning{color:#8a6d3b}a.text-
warning:hover{color:#66512c}.text-danger{color:#a94442}a.text-danger:hover{color:#843534}.bg-
primary{color:#fff;background-color:#428bca}a.bg-primary:hover{background-
color:#3071a9}.bg-success{background-color:#dff0d8}a.bg-success:hover{background-
color:#c1e2b3}.bg-info{background-color:#d9edf7}a.bg-info:hover{background-
color:#afd9ee}.bg-warning{background-color:#fcf8e3}a.bg-warning:hover{background-
color:#f7ecb5}.bg-danger{background-color:#f2dede}a.bg-danger:hover{background-
color:#e4b9b9}.page-header{padding-bottom:9px;margin:40px 0 20px;border-
bottom:1px solid #eee}ul,ol{margin-top:0;margin-bottom:10px}ul ul,ol
ul,ul ol,ol ol{margin-bottom:0}.list-unstyled{padding-left:0;list-style:none}.list-
inline{padding-left:0;margin-left:-5px;list-style:none}.list-inline>li{display:inline-
block;padding-right:5px;padding-left:5px}dl{margin-top:0;margin-bottom:20px}dt,dd{line-
height:1.42857143}dt{font-weight:700}dd{margin-left:0}@media (min-
width:768px){.dl-horizontal dt{float:left;width:160px;overflow:hidden;clear:left;text-
align:right;text-overflow:ellipsis;white-space:nowrap}.dl-horizontal dd{margin-
left:180px}}abbr[title],abbr[data-original-title]{cursor:help;border-bottom:1px
dotted #777}.initialism{font-size:90%;text-transform:uppercase}blockquote{padding:10px
20px;margin:0 0 20px;font-size:17.5px;border-left:5px solid #eee}blockquote
p:last-child,blockquote ul:last-child,blockquote ol:last-child{margin-bottom:0}blockquote
footer,blockquote small,blockquote .small{display:block;font-size:80%;line-
height:1.42857143;color:#777}blockquote footer:before,blockquote small:before,blockquote
.small:before{content:'\2014 \00A0'}.blockquote-reverse,blockquote.pull-
right{padding-right:15px;padding-left:0;text-align:right;border-right:5px
solid #eee;border-left:0}.blockquote-reverse footer:before,blockquote.pull-
right footer:before,.blockquote-reverse small:before,blockquote.pull-right
small:before,.blockquote-reverse .small:before,blockquote.pull-right .small:before{content:''}.blockquote-
reverse footer:after,blockquote.pull-right footer:after,.blockquote-reverse
small:after,blockquote.pull-right small:after,.blockquote-reverse .small:after,blockquote.pull-
right .small:after{content:'\00A0 \2014'}blockquote:before,blockquote:after{content:""}address{margin-
bottom:20px;font-style:normal;line-height:1.42857143}code,kbd,pre,samp{font-
family:Menlo,Monaco,Consolas,"Courier New",monospace}code{padding:2px
4px;font-size:90%;color:#c7254e;background-color:#f9f2f4;border-radius:4px}kbd{padding:2px
4px;font-size:90%;color:#fff;background-color:#333;border-radius:3px;-
webkit-box-shadow:inset 0 -1px 0 rgba(0,0,0,.25);box-shadow:inset 0
-1px 0 rgba(0,0,0,.25)}kbd kbd{padding:0;font-size:100%;-webkit-box-
shadow:none;box-shadow:none}pre{display:block;padding:9.5px;margin:0
0 10px;font-size:13px;line-height:1.42857143;color:#333;word-break:break-

all;word-wrap:break-word;background-color:#f5f5f5;border:1px solid #ccc;border-radius:4px}pre code{padding:0;font-size:inherit;color:inherit;white-space:pre-wrap;background-color:transparent;border-radius:0}.pre-scrollable{max-height:340px;overflow-y:scroll}.container{padding-right:15px;padding-left:15px;margin-right:auto;margin-left:auto}@media (min-width:768px){.container{width:750px}}@media (min-width:992px){.container{width:970px}}@media (min-width:1200px){.container{width:1170px}}.container-fluid{padding-right:15px;padding-left:15px;margin-right:auto;margin-left:auto}.row{margin-right:-15px;margin-left:-15px}.col-xs-1,.col-sm-1,.col-md-1,.col-lg-1,.col-xs-2,.col-sm-2,.col-md-2,.col-lg-2,.col-xs-3,.col-sm-3,.col-md-3,.col-lg-3,.col-xs-4,.col-sm-4,.col-md-4,.col-lg-4,.col-xs-5,.col-sm-5,.col-md-5,.col-lg-5,.col-xs-6,.col-sm-6,.col-md-6,.col-lg-6,.col-xs-7,.col-sm-7,.col-md-7,.col-lg-7,.col-xs-8,.col-sm-8,.col-md-8,.col-lg-8,.col-xs-9,.col-sm-9,.col-md-9,.col-lg-9,.col-xs-10,.col-sm-10,.col-md-10,.col-lg-10,.col-xs-11,.col-sm-11,.col-md-11,.col-lg-11,.col-xs-12,.col-sm-12,.col-md-12,.col-lg-12{position:relative;min-height:1px;padding-right:15px;padding-left:15px}.col-xs-1,.col-xs-2,.col-xs-3,.col-xs-4,.col-xs-5,.col-xs-6,.col-xs-7,.col-xs-8,.col-xs-9,.col-xs-10,.col-xs-11,.col-xs-12{float:left}.col-xs-12{width:100%}.col-xs-11{width:91.66666667%}.col-xs-10{width:83.33333333%}.col-xs-9{width:75%}.col-xs-8{width:66.66666667%}.col-xs-7{width:58.33333333%}.col-xs-6{width:50%}.col-xs-5{width:41.66666667%}.col-xs-4{width:33.33333333%}.col-xs-3{width:25%}.col-xs-2{width:16.66666667%}.col-xs-1{width:8.33333333%}.col-xs-pull-12{right:100%}.col-xs-pull-11{right:91.66666667%}.col-xs-pull-10{right:83.33333333%}.col-xs-pull-9{right:75%}.col-xs-pull-8{right:66.66666667%}.col-xs-pull-7{right:58.33333333%}.col-xs-pull-6{right:50%}.col-xs-pull-5{right:41.66666667%}.col-xs-pull-4{right:33.33333333%}.col-xs-pull-3{right:25%}.col-xs-pull-2{right:16.66666667%}.col-xs-pull-1{right:8.33333333%}.col-xs-pull-0{right:auto}.col-xs-push-12{left:100%}.col-xs-push-11{left:91.66666667%}.col-xs-push-10{left:83.33333333%}.col-xs-push-9{left:75%}.col-xs-push-8{left:66.66666667%}.col-xs-push-7{left:58.33333333%}.col-xs-push-6{left:50%}.col-xs-push-5{left:41.66666667%}.col-xs-push-4{left:33.33333333%}.col-xs-push-3{left:25%}.col-xs-push-2{left:16.66666667%}.col-xs-push-1{left:8.33333333%}.col-xs-push-0{left:auto}.col-xs-offset-12{margin-left:100%}.col-xs-offset-11{margin-left:91.66666667%}.col-xs-offset-10{margin-left:83.33333333%}.col-xs-offset-9{margin-left:75%}.col-xs-offset-8{margin-left:66.66666667%}.col-xs-offset-7{margin-left:58.33333333%}.col-xs-offset-6{margin-left:50%}.col-xs-offset-5{margin-left:41.66666667%}.col-xs-offset-4{margin-left:33.33333333%}.col-xs-offset-3{margin-left:25%}.col-xs-offset-2{margin-left:16.66666667%}.col-xs-offset-1{margin-left:8.33333333%}.col-xs-offset-0{margin-left:0}@media (min-width:768px){.col-sm-1,.col-sm-2,.col-sm-3,.col-sm-4,.col-sm-5,.col-sm-6,.col-sm-7,.col-sm-8,.col-sm-9,.col-sm-10,.col-sm-11,.col-sm-12{float:left}.col-sm-12{width:100%}.col-sm-11{width:91.66666667%}.col-sm-10{width:83.33333333%}.col-sm-9{width:75%}.col-sm-8{width:66.66666667%}.col-sm-7{width:58.33333333%}.col-sm-6{width:50%}.col-sm-5{width:41.66666667%}.col-sm-4{width:33.33333333%}.col-sm-3{width:25%}.col-sm-2{width:16.66666667%}.col-sm-1{width:8.33333333%}.col-sm-pull-12{right:100%}.col-sm-pull-11{right:91.66666667%}.col-sm-pull-10{right:83.33333333%}.col-sm-pull-9{right:75%}.col-sm-pull-8{right:66.66666667%}.col-sm-pull-7{right:58.33333333%}.col-sm-pull-6{right:50%}.col-sm-pull-5{right:41.66666667%}.col-sm-pull-4{right:33.33333333%}.col-sm-pull-3{right:25%}.col-sm-pull-2{right:16.66666667%}.col-sm-pull-1{right:8.33333333%}.col-sm-pull-0{right:auto}.col-sm-push-12{left:100%}.col-

sm-push-11{left:91.66666667%}.col-sm-push-10{left:83.33333333%}.col-sm-push-9{left:75%}.col-sm-push-8{left:66.66666667%}.col-sm-push-7{left:58.33333333%}.col-sm-push-6{left:50%}.col-sm-push-5{left:41.66666667%}.col-sm-push-4{left:33.33333333%}.col-sm-push-3{left:25%}.col-sm-push-2{left:16.66666667%}.col-sm-push-1{left:8.33333333%}.col-sm-push-0{left:auto}.col-sm-offset-12{margin-left:100%}.col-sm-offset-11{margin-left:91.66666667%}.col-sm-offset-10{margin-left:83.33333333%}.col-sm-offset-9{margin-left:75%}.col-sm-offset-8{margin-left:66.66666667%}.col-sm-offset-7{margin-left:58.33333333%}.col-sm-offset-6{margin-left:50%}.col-sm-offset-5{margin-left:41.66666667%}.col-sm-offset-4{margin-left:33.33333333%}.col-sm-offset-3{margin-left:25%}.col-sm-offset-2{margin-left:16.66666667%}.col-sm-offset-1{margin-left:8.33333333%}.col-sm-offset-0{margin-left:0}}@media (min-width:992px){.col-md-1,.col-md-2,.col-md-3,.col-md-4,.col-md-5,.col-md-6,.col-md-7,.col-md-8,.col-md-9,.col-md-10,.col-md-11,.col-md-12{float:left}.col-md-12{width:100%}.col-md-11{width:91.66666667%}.col-md-10{width:83.33333333%}.col-md-9{width:75%}.col-md-8{width:66.66666667%}.col-md-7{width:58.33333333%}.col-md-6{width:50%}.col-md-5{width:41.66666667%}.col-md-4{width:33.33333333%}.col-md-3{width:25%}.col-md-2{width:16.66666667%}.col-md-1{width:8.33333333%}.col-md-pull-12{right:100%}.col-md-pull-11{right:91.66666667%}.col-md-pull-10{right:83.33333333%}.col-md-pull-9{right:75%}.col-md-pull-8{right:66.66666667%}.col-md-pull-7{right:58.33333333%}.col-md-pull-6{right:50%}.col-md-pull-5{right:41.66666667%}.col-md-pull-4{right:33.33333333%}.col-md-pull-3{right:25%}.col-md-pull-2{right:16.66666667%}.col-md-pull-1{right:8.33333333%}.col-md-pull-0{right:auto}.col-md-push-12{left:100%}.col-md-push-11{left:91.66666667%}.col-md-push-10{left:83.33333333%}.col-md-push-9{left:75%}.col-md-push-8{left:66.66666667%}.col-md-push-7{left:58.33333333%}.col-md-push-6{left:50%}.col-md-push-5{left:41.66666667%}.col-md-push-4{left:33.33333333%}.col-md-push-3{left:25%}.col-md-push-2{left:16.66666667%}.col-md-push-1{left:8.33333333%}.col-md-push-0{left:auto}.col-md-offset-12{margin-left:100%}.col-md-offset-11{margin-left:91.66666667%}.col-md-offset-10{margin-left:83.33333333%}.col-md-offset-9{margin-left:75%}.col-md-offset-8{margin-left:66.66666667%}.col-md-offset-7{margin-left:58.33333333%}.col-md-offset-6{margin-left:50%}.col-md-offset-5{margin-left:41.66666667%}.col-md-offset-4{margin-left:33.33333333%}.col-md-offset-3{margin-left:25%}.col-md-offset-2{margin-left:16.66666667%}.col-md-offset-1{margin-left:8.33333333%}.col-md-offset-0{margin-left:0}}@media (min-width:1200px){.col-lg-1,.col-lg-2,.col-lg-3,.col-lg-4,.col-lg-5,.col-lg-6,.col-lg-7,.col-lg-8,.col-lg-9,.col-lg-10,.col-lg-11,.col-lg-12{float:left}.col-lg-12{width:100%}.col-lg-11{width:91.66666667%}.col-lg-10{width:83.33333333%}.col-lg-9{width:75%}.col-lg-8{width:66.66666667%}.col-lg-7{width:58.33333333%}.col-lg-6{width:50%}.col-lg-5{width:41.66666667%}.col-lg-4{width:33.33333333%}.col-lg-3{width:25%}.col-lg-2{width:16.66666667%}.col-lg-1{width:8.33333333%}.col-lg-pull-12{right:100%}.col-lg-pull-11{right:91.66666667%}.col-lg-pull-10{right:83.33333333%}.col-lg-pull-9{right:75%}.col-lg-pull-8{right:66.66666667%}.col-lg-pull-7{right:58.33333333%}.col-lg-pull-6{right:50%}.col-lg-pull-5{right:41.66666667%}.col-lg-pull-4{right:33.33333333%}.col-lg-pull-3{right:25%}.col-lg-pull-2{right:16.66666667%}.col-lg-pull-1{right:8.33333333%}.col-lg-pull-0{right:auto}.col-lg-push-12{left:100%}.col-lg-push-11{left:91.66666667%}.col-lg-push-10{left:83.33333333%}.col-lg-push-9{left:75%}.col-lg-push-8{left:66.66666667%}.col-lg-push-7{left:58.33333333%}.col-lg-push-6{left:50%}.col-lg-push-5{left:41.66666667%}.col-

lg-push-4{left:33.33333333%}.col-lg-push-3{left:25%}.col-lg-push-2{left:16.66666667%}.col-lg-push-1{left:8.33333333%}.col-lg-push-0{left:auto}.col-lg-offset-12{margin-left:100%}.col-lg-offset-11{margin-left:91.66666667%}.col-lg-offset-10{margin-left:83.33333333%}.col-lg-offset-9{margin-left:75%}.col-lg-offset-8{margin-left:66.66666667%}.col-lg-offset-7{margin-left:58.33333333%}.col-lg-offset-6{margin-left:50%}.col-lg-offset-5{margin-left:41.66666667%}.col-lg-offset-4{margin-left:33.33333333%}.col-lg-offset-3{margin-left:25%}.col-lg-offset-2{margin-left:16.66666667%}.col-lg-offset-1{margin-left:8.33333333%}.col-lg-offset-0{margin-left:0}}table{background-color:transparent}th{text-align:left}.table{width:100%;max-width:100%;margin-bottom:20px}.table>thead>tr>th,.table>tbody>tr>th,.table>tfoot>tr>th,.table>thead height:1.42857143;vertical-align:top;border-top:1px solid #ddd}.table>thead>tr>th{vertical-align:bottom;border-bottom:2px solid #ddd}.table>caption+thead>tr:first-child>th,.table>colgroup+thead>tr:first-child>th,.table>thead:first-child>tr:first-child>th,.table>caption+thead>tr:first-child>td,.table>colgroup+thead>tr:first-child>td,.table>thead:first-child>tr:first-child>td{border-top:0}.table>tbody+tbody{border-top:2px solid #ddd}.table-condensed>thead>tr>th,.table-condensed>tbody>tr>th,.table-condensed>tfoot>tr>th,.table-condensed>thead>tr>td,.table-condensed>tbody>tr>td,.table-condensed>tfoot>tr>td{padding:5px}.table-bordered{border:1px solid #ddd}.table-bordered>thead>tr>th,.table-bordered>tbody>tr>th,.table-bordered>tfoot>tr>th,.table-bordered>thead>tr>td,.table-bordered>tbody>tr>td,.table-bordered>tfoot>tr>td{border:1px solid #ddd}.table-bordered>thead>tr>th,.table-bordered>thead>tr>td{border-bottom-width:2px}.table-striped>tbody>tr:nth-child(odd)>td,.table-striped>tbody>tr:nth-child(odd)>th{background-color:#f9f9f9}.table-hover>tbody>tr:hover>td,.table-hover>tbody>tr:hover>th{background-color:#f5f5f5}table col[class*=col-]{position:static;display:table-column;float:none}table td[class*=col-],table th[class*=col-]{position:static;display:table-cell;float:none}.table>thead>tr>td.active,.tal color:#f5f5f5}.table-hover>tbody>tr>td.active:hover,.table-hover>tbody>tr>th.active:hover,.table-hover>tbody>tr.active:hover>td,.table-hover>tbody>tr:hover>.active,.table-hover>tbody>tr.active:hover>th{background-color:#e8e8e8}.table>thead>tr>td.success,.table>tbody>tr>t color:#dff0d8}.table-hover>tbody>tr>td.success:hover,.table-hover>tbody>tr>th.success:hover,.table-hover>tbody>tr.success:hover>td,.table-hover>tbody>tr:hover>.success,.table-hover>tbody>tr.success:hover>th{background-color:#d0e9c6}.table>thead>tr>td.info,.table>tbody>tr>td. color:#d9edf7}.table-hover>tbody>tr>td.info:hover,.table-hover>tbody>tr>th.info:hover,.table-hover>tbody>tr.info:hover>td,.table-hover>tbody>tr:hover>.info,.table-hover>tbody>tr.info:hover>th{background-color:#c4e3f3}.table>thead>tr>td.warning,.table>tbody>tr>td color:#fcf8e3}.table-hover>tbody>tr>td.warning:hover,.table-hover>tbody>tr>th.warning:hover,.table-hover>tbody>tr.warning:hover>td,.table-hover>tbody>tr:hover>.warning,.table-hover>tbody>tr.warning:hover>th{background-color:#faf2cc}.table>thead>tr>td.danger,.table>tbody>tr> color:#f2dede}.table-hover>tbody>tr>td.danger:hover,.table-hover>tbody>tr>th.danger:hover,.table-hover>tbody>tr.danger:hover>td,.table-hover>tbody>tr:hover>.danger,.table-hover>tbody>tr.danger:hover>th{background-color:#ebcccc}@media screen and (max-width:767px){.table-responsive{width:100%;margin-bottom:15px;overflow-x:auto;overflow-y:hidden;-webkit-overflow-scrolling:touch;-ms-overflow-style:-ms-autohiding-scrollbar;border:1px solid

#ddd}.table-responsive>.table{margin-bottom:0}.table-responsive>.table>thead>tr>th,.table-

responsive>.table>tbody>tr>th,.table-responsive>.table>tfoot>tr>th,.table-responsive>.table>thead>tr>td,.table-responsive>.table>tbody>tr>td,.table-responsive>.table>tfoot>tr>td{white-space:nowrap}.table-responsive>.table-bordered{border:0}.table-responsive>.table-bordered>thead>tr>th:first-child,.table-responsive>.table-bordered>tbody>tr>th:first-child,.table-responsive>.table-bordered>tfoot>tr>th:first-child,.table-responsive>.table-bordered>thead>tr>td:first-child,.table-responsive>.table-bordered>tbody>tr>td:first-child,.table-responsive>.table-bordered>tfoot>tr>td:first-child{border-left:0}.table-responsive>.table-bordered>thead>tr>th:last-child,.table-responsive>.table-bordered>tbody>tr>th:last-child,.table-responsive>.table-bordered>tfoot>tr>th:last-child,.table-responsive>.table-bordered>thead>tr>td:last-child,.table-responsive>.table-bordered>tbody>tr>td:last-child,.table-responsive>.table-bordered>tfoot>tr>td:last-child{border-right:0}.table-responsive>.table-bordered>tbody>tr:last-child>th,.table-responsive>.table-bordered>tfoot>tr:last-child>th,.table-responsive>.table-bordered>tbody>tr:last-child>td,.table-responsive>.table-bordered>tfoot>tr:last-child>td{border-bottom:0}}fieldset{min-width:0;padding:0;margin:0;border:0}legend{display:block;width:100%;padding:0;marg
bottom:20px;font-size:21px;line-height:inherit;color:#333;border:0;border-bottom:1px solid #e5e5e5}label{display:inline-block;max-width:100%;margin-bottom:5px;font-weight:700}input[type=search]{-webkit-box-sizing:border-box;-moz-box-sizing:border-box;box-sizing:border-box}input[type=radio],input[type=checkbox]{margin:4px
0 0;margin-top:1px \9;line-height:normal}input[type=file]{display:block}input[type=range]{display:block;widt
dotted;outline:5px auto -webkit-focus-ring-color;outline-offset:-2px}output{display:block;padding-top:7px;font-size:14px;line-height:1.42857143;color:#555}.form-control{display:block;width:100%;height:34px;p
12px;font-size:14px;line-height:1.42857143;color:#555;background-color:#fff;background-image:none;border:1px solid #ccc;border-radius:4px;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075);box-shadow:inset 0 1px 1px rgba(0,0,0,.075);-webkit-transition:border-color ease-in-out .15s,-webkit-box-shadow ease-in-out .15s;-o-transition:border-color ease-in-out .15s,box-shadow ease-in-out .15s;transition:border-color ease-in-out .15s,box-shadow ease-in-out .15s}.form-control:focus{border-color:#66afe9;outline:0;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 8px rgba(102,175,233,.6);box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 8px rgba(102,175,233,.6)}.form-control::-moz-placeholder{color:#777;opacity:1}.form-control:-ms-input-placeholder{color:#777}.form-control::-webkit-input-placeholder{color:#777}.form-control[disabled],.form-control[readonly],fieldset[disabled] .form-control{cursor:not-allowed;background-color:#eee;opacity:1}textarea.form-control{height:auto}input[type=search]{-webkit-appearance:none}input[type=date],input[type=time],input[type=datetime-local],input[type=month]{line-height:34px;line-height:1.42857143 \0}input[type=date].input-sm,input[type=time].input-sm,input[type=datetime-local].input-sm,input[type=month].input-sm{line-height:30px}input[type=date].input-lg,input[type=time].input-lg,input[type=datetime-local].input-lg,input[type=month].input-lg{line-height:46px}.form-group{margin-bottom:15px}.radio,.checkbox{position:relative;display:block;min-height:20px;margin-top:10px;margin-bottom:10px}.radio label,.checkbox label{padding-left:20px;margin-bottom:0;font-weight:400;cursor:pointer}.radio input[type=radio],.radio-inline input[type=radio],.checkbox input[type=checkbox],.checkbox-

inline input[type=checkbox]{position:absolute;margin-top:4px \9;margin-left:-20px}.radio+.radio,.checkbox+.checkbox{margin-top:-5px}.radio-inline,.checkbox-inline{display:inline-block;padding-left:20px;margin-bottom:0;font-weight:400;vertical-align:middle;cursor:pointer}.radio-inline+.radio-inline,.checkbox-inline+.checkbox-inline{margin-top:0;margin-left:10px}input[type=radio][disabled],input[type=checkbox][disabled],input[type=
input[type=radio],fieldset[disabled] input[type=checkbox]{cursor:not-allowed}.radio-inline.disabled,.checkbox-inline.disabled,fieldset[disabled] .radio-inline,fieldset[disabled] .checkbox-inline{cursor:not-allowed}.radio.disabled label,.checkbox.disabled label,fieldset[disabled] .radio label,fieldset[disabled] .checkbox label{cursor:not-allowed}.form-control-static{padding-top:7px;padding-bottom:7px;margin-bottom:0}.form-control-static.input-lg,.form-control-static.input-sm{padding-right:0;padding-left:0}.input-sm,.form-horizontal .form-group-sm .form-control{height:30px;padding:5px 10px;font-size:12px;line-height:1.5;border-radius:3px}select.input-sm{height:30px;line-height:30px}textarea.input-sm,select[multiple].input-sm{height:auto}.input-lg,.form-horizontal .form-group-lg .form-control{height:46px;padding:10px 16px;font-size:18px;line-height:1.33;border-radius:6px}select.input-lg{height:46px;line-height:46px}textarea.input-lg,select[multiple].input-lg{height:auto}.has-feedback{position:relative}.has-feedback .form-control{padding-right:42.5px}.form-control-feedback{position:absolute;top:25px;right:0;z-index:2;display:block;width:34px;height:34px;line-height:34px;text-align:center}.input-lg+.form-control-feedback{width:46px;height:46px;line-height:46px}.input-sm+.form-control-feedback{width:30px;height:30px;line-height:30px}.has-success .help-block,.has-success .control-label,.has-success .radio,.has-success .checkbox,.has-success .radio-inline,.has-success .checkbox-inline{color:#3c763d}.has-success .form-control{border-color:#3c763d;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075);box-shadow:inset 0 1px 1px rgba(0,0,0,.075)}.has-success .form-control:focus{border-color:#2b542c;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #67b168;box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #67b168}.has-success .input-group-addon{color:#3c763d;background-color:#dff0d8;border-color:#3c763d}.has-success .form-control-feedback{color:#3c763d}.has-warning .help-block,.has-warning .control-label,.has-warning .radio,.has-warning .checkbox,.has-warning .radio-inline,.has-warning .checkbox-inline{color:#8a6d3b}.has-warning .form-control{border-color:#8a6d3b;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075);box-shadow:inset 0 1px 1px rgba(0,0,0,.075)}.has-warning .form-control:focus{border-color:#66512c;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #c0a16b;box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #c0a16b}.has-warning .input-group-addon{color:#8a6d3b;background-color:#fcf8e3;border-color:#8a6d3b}.has-warning .form-control-feedback{color:#8a6d3b}.has-error .help-block,.has-error .control-label,.has-error .radio,.has-error .checkbox,.has-error .radio-inline,.has-error .checkbox-inline{color:#a94442}.has-error .form-control{border-color:#a94442;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075);box-shadow:inset 0 1px 1px rgba(0,0,0,.075)}.has-error .form-control:focus{border-color:#843534;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #ce8483;box-shadow:inset 0 1px 1px rgba(0,0,0,.075),0 0 6px #ce8483}.has-error .input-group-addon{color:#a94442;background-color:#f2dede;border-color:#a94442}.has-error .form-control-feedback{color:#a94442}.has-

feedback label.sr-only~.form-control-feedback{top:0}.help-block{display:block;margin-top:5px;margin-bottom:10px;color:#737373}@media (min-width:768px){.form-inline .form-group{display:inline-block;margin-bottom:0;vertical-align:middle}.form-inline .form-control{display:inline-block;width:auto;vertical-align:middle}.form-inline .input-group{display:inline-table;vertical-align:middle}.form-inline .input-group .input-group-addon,.form-inline .input-group .input-group-btn,.form-inline .input-group .form-control{width:auto}.form-inline .input-group>.form-control{width:100%}.form-inline .control-label{margin-bottom:0;vertical-align:middle}.form-inline .radio,.form-inline .checkbox{display:inline-block;margin-top:0;margin-bottom:0;vertical-align:middle}.form-inline .radio label,.form-inline .checkbox label{padding-left:0}.form-inline .radio input[type=radio],.form-inline .checkbox input[type=checkbox]{position:relative;margin-left:0}.form-inline .has-feedback .form-control-feedback{top:0}}.form-horizontal .radio,.form-horizontal .checkbox,.form-horizontal .radio-inline,.form-horizontal .checkbox-inline{padding-top:7px;margin-top:0;margin-bottom:0}.form-horizontal .radio,.form-horizontal .checkbox{min-height:27px}.form-horizontal .form-group{margin-right:-15px;margin-left:-15px}@media (min-width:768px){.form-horizontal .control-label{padding-top:7px;margin-bottom:0;text-align:right}}.form-horizontal .has-feedback .form-control-feedback{top:0;right:15px}@media (min-width:768px){.form-horizontal .form-group-lg .control-label{padding-top:14.3px}}@media (min-width:768px){.form-horizontal .form-group-sm .control-label{padding-top:6px}}.btn{display:inline-block;padding:6px 12px;margin-bottom:0;font-size:14px;font-weight:400;line-height:1.42857143;text-align:center;white-space:nowrap;vertical-align:middle;cursor:pointer;-webkit-user-select:none;-moz-user-select:none;-ms-user-select:none;user-select:none;background-image:none;border:1px solid transparent;border-radius:4px}.btn:focus,.btn:active:focus,.btn.active:focus{outlin dotted;outline:5px auto -webkit-focus-ring-color;outline-offset:-2px}.btn:hover,.btn:focus{color:#333;text-decoration:none}.btn:active,.btn.active{background-image:none;outline:0;-webkit-box-shadow:inset 0 3px 5px rgba(0,0,0,.125);box-shadow:inset 0 3px 5px rgba(0,0,0,.125)}.btn.disabled,.btn[disabled],fieldset[disabled] .btn{pointer-events:none;cursor:not-allowed;filter:alpha(opacity=65);-webkit-box-shadow:none;box-shadow:none;opacity:.65}.btn-default{color:#333;background-color:#fff;border-color:#ccc}.btn-default:hover,.btn-default:focus,.btn-default:active,.btn-default.active,.open>.dropdown-toggle.btn-default{color:#333;background-color:#e6e6e6;border-color:#adadad}.btn-default:active,.btn-default.active,.open>.dropdown-toggle.btn-default{background-image:none}.btn-default.disabled,.btn-default[disabled],fieldset[disabled] .btn-default,.btn-default.disabled:hover,.btn-default[disabled]:hover,fieldset[disabled] .btn-default:hover,.btn-default.disabled:focus,.btn-default[disabled]:focus,fieldset[disabled] .btn-default:focus,.btn-default.disabled:active,.btn-default[disabled]:active,fieldset[disabled] .btn-default:active,.btn-default.disabled.active,.btn-default[disabled].active,fieldset[disabled] .btn-default.active{background-color:#fff;border-color:#ccc}.btn-default .badge{color:#fff;background-color:#333}.btn-primary{color:#fff;background-color:#428bca;border-color:#357ebd}.btn-primary:hover,.btn-primary:focus,.btn-primary:active,.btn-primary.active,.open>.dropdown-toggle.btn-primary{color:#fff;background-color:#3071a9;border-color:#285e8e}.btn-primary:active,.btn-primary.active,.open>.dropdown-toggle.btn-primary{background-image:none}.btn-primary.disabled,.btn-

primary[disabled],fieldset[disabled] .btn-primary,.btn-primary.disabled:hover,.btn-primary[disabled]:hover,fieldset[disabled] .btn-primary:hover,.btn-primary.disabled:focus,.btn-primary[disabled]:focus,fieldset[disabled] .btn-primary:focus,.btn-primary.disabled:active,.btn-primary[disabled]:active,fieldset[disabled] .btn-primary:active,.btn-primary.disabled.active,.btn-primary[disabled].active,fieldset[disabled] .btn-primary.active{background-color:#428bca;border-color:#357ebd}.btn-primary .badge{color:#428bca;background-color:#fff}.btn-success{color:#fff;background-color:#5cb85c;border-color:#4cae4c}.btn-success:hover,.btn-success:focus,.btn-success:active,.btn-success.active,.open>.dropdown-toggle.btn-success{color:#fff;background-color:#449d44;border-color:#398439}.btn-success:active,.btn-success.active,.open>.dropdown-toggle.btn-success{background-image:none}.btn-success.disabled,.btn-success[disabled],fieldset[disabled] .btn-success,.btn-success.disabled:hover,.btn-success[disabled]:hover,fieldset[disabled] .btn-success:hover,.btn-success.disabled:focus,.btn-success[disabled]:focus,fieldset[disabled] .btn-success:focus,.btn-success.disabled:active,.btn-success[disabled]:active,fieldset[disabled] .btn-success:active,.btn-success.disabled.active,.btn-success[disabled].active,fieldset[disabled] .btn-success.active{background-color:#5cb85c;border-color:#4cae4c}.btn-success .badge{color:#5cb85c;background-color:#fff}.btn-info{color:#fff;background-color:#5bc0de;border-color:#46b8da}.btn-info:hover,.btn-info:focus,.btn-info:active,.btn-info.active,.open>.dropdown-toggle.btn-info{color:#fff;background-color:#31b0d5;border-color:#269abc}.btn-info:active,.btn-info.active,.open>.dropdown-toggle.btn-info{background-image:none}.btn-info.disabled,.btn-info[disabled],fieldset[disabled] .btn-info,.btn-info.disabled:hover,.btn-info[disabled]:hover,fieldset[disabled] .btn-info:hover,.btn-info.disabled:focus,.btn-info[disabled]:focus,fieldset[disabled] .btn-info:focus,.btn-info.disabled:active,.btn-info[disabled]:active,fieldset[disabled] .btn-info:active,.btn-info.disabled.active,.btn-info[disabled].active,fieldset[disabled] .btn-info.active{background-color:#5bc0de;border-color:#46b8da}.btn-info .badge{color:#5bc0de;background-color:#fff}.btn-warning{color:#fff;background-color:#f0ad4e;border-color:#eea236}.btn-warning:hover,.btn-warning:focus,.btn-warning:active,.btn-warning.active,.open>.dropdown-toggle.btn-warning{color:#fff;background-color:#ec971f;border-color:#d58512}.btn-warning:active,.btn-warning.active,.open>.dropdown-toggle.btn-warning{background-image:none}.btn-warning.disabled,.btn-warning[disabled],fieldset[disabled] .btn-warning,.btn-warning.disabled:hover,.btn-warning[disabled]:hover,fieldset[disabled] .btn-warning:hover,.btn-warning.disabled:focus,.btn-warning[disabled]:focus,fieldset[disabled] .btn-warning:focus,.btn-warning.disabled:active,.btn-warning[disabled]:active,fieldset[disabled] .btn-warning:active,.btn-warning.disabled.active,.btn-warning[disabled].active,fieldset[disabled] .btn-warning.active{background-color:#f0ad4e;border-color:#eea236}.btn-warning .badge{color:#f0ad4e;background-color:#fff}.btn-danger{color:#fff;background-color:#d9534f;border-color:#d43f3a}.btn-danger:hover,.btn-danger:focus,.btn-danger:active,.btn-danger.active,.open>.dropdown-toggle.btn-danger{color:#fff;background-color:#c9302c;border-color:#ac2925}.btn-danger:active,.btn-danger.active,.open>.dropdown-toggle.btn-danger{background-image:none}.btn-danger.disabled,.btn-danger[disabled],fieldset[disabled] .btn-danger,.btn-danger.disabled:hover,.btn-danger[disabled]:hover,fieldset[disabled] .btn-danger:hover,.btn-danger.disabled:focus,.btn-danger[disabled]:focus,fieldset[disabled] .btn-danger:focus,.btn-danger.disabled:active,.btn-danger[disabled]:active,fieldset[disabled] .btn-danger:active,.btn-danger.disabled.active,.btn-danger[disabled].active,fieldset[disabled]

.btn-danger.active{background-color:#d9534f;border-color:#d43f3a}.btn-
danger .badge{color:#d9534f;background-color:#fff}.btn-link{font-weight:400;color:#428bca;cursor:pointer;bo
radius:0}.btn-link,.btn-link:active,.btn-link[disabled],fieldset[disabled] .btn-
link{background-color:transparent;-webkit-box-shadow:none;box-shadow:none}.btn-
link,.btn-link:hover,.btn-link:focus,.btn-link:active{border-color:transparent}.btn-
link:hover,.btn-link:focus{color:#2a6496;text-decoration:underline;background-
color:transparent}.btn-link[disabled]:hover,fieldset[disabled] .btn-link:hover,.btn-
link[disabled]:focus,fieldset[disabled] .btn-link:focus{color:#777;text-decoration:none}.btn-
lg,.btn-group-lg>.btn{padding:10px 16px;font-size:18px;line-height:1.33;border-
radius:6px}.btn-sm,.btn-group-sm>.btn{padding:5px 10px;font-size:12px;line-
height:1.5;border-radius:3px}.btn-xs,.btn-group-xs>.btn{padding:1px 5px;font-
size:12px;line-height:1.5;border-radius:3px}.btn-block{display:block;width:100%}.btn-
block+.btn-block{margin-top:5px}input[type=submit].btn-block,input[type=reset].btn-
block,input[type=button].btn-block{width:100%}.fade{opacity:0;-webkit-
transition:opacity .15s linear;-o-transition:opacity .15s linear;transition:opacity
.15s linear}.fade.in{opacity:1}.collapse{display:none}.collapse.in{display:block}tr.collapse.in{display:table-
row}tbody.collapse.in{display:table-row-group}.collapsing{position:relative;height:0;overflow:hidden;-
webkit-transition:height .35s ease;-o-transition:height .35s ease;transition:height
.35s ease}.caret{display:inline-block;width:0;height:0;margin-left:2px;vertical-
align:middle;border-top:4px solid;border-right:4px solid transparent;border-
left:4px solid transparent}.dropdown{position:relative}.dropdown-toggle:focus{outline:0}.dropdown-
menu{position:absolute;top:100%;left:0;z-index:1000;display:none;float:left;min-
width:160px;padding:5px 0;margin:2px 0 0;font-size:14px;text-align:left;list-
style:none;background-color:#fff;-webkit-background-clip:padding-box;background-
clip:padding-box;border:1px solid #ccc;border:1px solid rgba(0,0,0,.15);border-
radius:4px;-webkit-box-shadow:0 6px 12px rgba(0,0,0,.175);box-shadow:0 6px
12px rgba(0,0,0,.175)}.dropdown-menu.pull-right{right:0;left:auto}.dropdown-
menu .divider{height:1px;margin:9px 0;overflow:hidden;background-color:#e5e5e5}.dropdown-
menu>li>a{display:block;padding:3px 20px;clear:both;font-weight:400;line-
height:1.42857143;color:#333;white-space:nowrap}.dropdown-menu>li>a:hover,.dropdown-
menu>li>a:focus{color:#262626;text-decoration:none;background-color:#f5f5f5}.dropdown-
menu>.active>a,.dropdown-menu>.active>a:hover,.dropdown-menu>.active>a:focus{color:#fff;text-
decoration:none;background-color:#428bca;outline:0}.dropdown-menu>.disabled>a,.dropdown-
menu>.disabled>a:hover,.dropdown-menu>.disabled>a:focus{color:#777}.dropdown-
menu>.disabled>a:hover,.dropdown-menu>.disabled>a:focus{text-decoration:none;cursor:not-
allowed;background-color:transparent;background-image:none;filter:progid:DXImageTransform.Microsoft.gradi
menu{display:block}.open>a{outline:0}.dropdown-menu-right{right:0;left:auto}.dropdown-
menu-left{right:auto;left:0}.dropdown-header{display:block;padding:3px
20px;font-size:12px;line-height:1.42857143;color:#777;white-space:nowrap}.dropdown-
backdrop{position:fixed;top:0;right:0;bottom:0;left:0;z-index:990}.pull-right>.dropdown-
menu{right:0;left:auto}.dropup .caret,.navbar-fixed-bottom .dropdown
.caret{content:"";border-top:0;border-bottom:4px solid}.dropup .dropdown-
menu,.navbar-fixed-bottom .dropdown .dropdown-menu{top:auto;bottom:100%;margin-
bottom:1px}@media (min-width:768px){.navbar-right .dropdown-menu{right:0;left:auto}.navbar-
right .dropdown-menu-left{right:auto;left:0}}.btn-group,.btn-group-vertical{position:relative;display:inline-
block;vertical-align:middle}.btn-group>.btn,.btn-group-vertical>.btn{position:relative;float:left}.btn-

group>.btn:hover,.btn-group-vertical>.btn:hover,.btn-group>.btn:focus,.btn-group-vertical>.btn:focus,.btn-group>.btn:active,.btn-group-vertical>.btn:active,.btn-group>.btn.active,.btn-group-vertical>.btn.active{z-index:2}.btn-group>.btn:focus,.btn-group-vertical>.btn:focus{outline:0}.btn-group .btn+.btn,.btn-group .btn+.btn-group,.btn-group .btn-group+.btn,.btn-group .btn-group+.btn-group{margin-left:-1px}.btn-toolbar{margin-left:-5px}.btn-toolbar .btn-group,.btn-toolbar .input-group{float:left}.btn-toolbar>.btn,.btn-toolbar>.btn-group,.btn-toolbar>.input-group{margin-left:5px}.btn-group>.btn:not(:first-child):not(:last-child):not(.dropdown-toggle){border-radius:0}.btn-group>.btn:first-child{margin-left:0}.btn-group>.btn:first-child:not(:last-child):not(.dropdown-toggle){border-top-right-radius:0;border-bottom-right-radius:0}.btn-group>.btn:last-child:not(:first-child),.btn-group>.dropdown-toggle:not(:first-child){border-top-left-radius:0;border-bottom-left-radius:0}.btn-group>.btn-group{float:left}.btn-group>.btn-group:not(:first-child):not(:last-child)>.btn{border-radius:0}.btn-group>.btn-group:first-child>.btn:last-child,.btn-group>.btn-group:first-child>.dropdown-toggle{border-top-right-radius:0;border-bottom-right-radius:0}.btn-group>.btn-group:last-child>.btn:first-child{border-top-left-radius:0;border-bottom-left-radius:0}.btn-group .dropdown-toggle:active,.btn-group.open .dropdown-toggle{outline:0}.btn-group>.btn+.dropdown-toggle{padding-right:8px;padding-left:8px}.btn-group>.btn-lg+.dropdown-toggle{padding-right:12px;padding-left:12px}.btn-group.open .dropdown-toggle{-webkit-box-shadow:inset 0 3px 5px rgba(0,0,0,.125);box-shadow:inset 0 3px 5px rgba(0,0,0,.125)}.btn-group.open .dropdown-toggle.btn-link{-webkit-box-shadow:none;box-shadow:none}.btn .caret{margin-left:0}.btn-lg .caret{border-width:5px 5px 0;border-bottom-width:0}.dropup .btn-lg .caret{border-width:0 5px 5px}.btn-group-vertical>.btn,.btn-group-vertical>.btn-group,.btn-group-vertical>.btn-group>.btn{display:block;float:none;width:100%;max-width:100%}.btn-group-vertical>.btn-group>.btn{float:none}.btn-group-vertical>.btn+.btn,.btn-group-vertical>.btn+.btn-group,.btn-group-vertical>.btn-group+.btn,.btn-group-vertical>.btn-group+.btn-group{margin-top:-1px;margin-left:0}.btn-group-vertical>.btn:not(:first-child):not(:last-child){border-radius:0}.btn-group-vertical>.btn:first-child:not(:last-child){border-top-right-radius:4px;border-bottom-right-radius:0;border-bottom-left-radius:0}.btn-group-vertical>.btn:last-child:not(:first-child){border-top-left-radius:0;border-top-right-radius:0;border-bottom-left-radius:4px}.btn-group-vertical>.btn-group:not(:first-child):not(:last-child)>.btn{border-radius:0}.btn-group-vertical>.btn-group:first-child:not(:last-child)>.btn:last-child,.btn-group-vertical>.btn-group:first-child:not(:last-child)>.dropdown-toggle{border-bottom-right-radius:0;border-bottom-left-radius:0}.btn-group-vertical>.btn-group:last-child:not(:first-child)>.btn:first-child{border-top-left-radius:0;border-top-right-radius:0}.btn-group-justified{display:table;width:100%;table-layout:fixed;border-collapse:separate}.btn-group-justified>.btn,.btn-group-justified>.btn-group{display:table-cell;float:none;width:1%}.btn-group-justified>.btn-group .btn{width:100%}.btn-group-justified>.btn-group .dropdown-menu{left:auto}[data-toggle=buttons]>.btn>input[type=radio],[data-toggle=buttons]>.btn>input[type=checkbox]{position:absolute;z-index:-1;filter:alpha(opacity=0);opacity:0}.input-group{position:relative;display:table;border-collapse:separate}.input-group[class*=col-]{float:none;padding-right:0;padding-

left:0}.input-group .form-control{position:relative;z-index:2;float:left;width:100%;margin-bottom:0}.input-group-lg>.form-control,.input-group-lg>.input-group-addon,.input-group-lg>.input-group-btn>.btn{height:46px;padding:10px 16px;font-size:18px;line-height:1.33;border-radius:6px}select.input-group-lg>.form-control,select.input-group-lg>.input-group-addon,select.input-group-lg>.input-group-btn>.btn{height:46px;line-height:46px}textarea.input-group-lg>.form-control,textarea.input-group-lg>.input-group-addon,textarea.input-group-lg>.input-group-btn>.btn,select[multiple].input-group-lg>.form-control,select[multiple].input-group-lg>.input-group-addon,select[multiple].input-group-lg>.input-group-btn>.btn{height:auto}.input-group-sm>.form-control,.input-group-sm>.input-group-addon,.input-group-sm>.input-group-btn>.btn{height:30px;padding:5px 10px;font-size:12px;line-height:1.5;border-radius:3px}select.input-group-sm>.form-control,select.input-group-sm>.input-group-addon,select.input-group-sm>.input-group-btn>.btn{height:30px;line-height:30px}textarea.input-group-sm>.form-control,textarea.input-group-sm>.input-group-addon,textarea.input-group-sm>.input-group-btn>.btn,select[multiple].input-group-sm>.form-control,select[multiple].input-group-sm>.input-group-addon,select[multiple].input-group-sm>.input-group-btn>.btn{height:auto}.input-group-addon,.input-group-btn,.input-group .form-control{display:table-cell}.input-group-addon:not(:first-child):not(:last-child),.input-group-btn:not(:first-child):not(:last-child),.input-group .form-control:not(:first-child):not(:last-child){border-radius:0}.input-group-addon,.input-group-btn{width:1%;white-space:nowrap;vertical-align:middle}.input-group-addon{padding:6px 12px;font-size:14px;font-weight:400;line-height:1;color:#555;text-align:center;background-color:#eee;border:1px solid #ccc;border-radius:4px}.input-group-addon.input-sm{padding:5px 10px;font-size:12px;border-radius:3px}.input-group-addon.input-lg{padding:10px 16px;font-size:18px;border-radius:6px}.input-group-addon input[type=radio],.input-group-addon input[type=checkbox]{margin-top:0}.input-group .form-control:first-child,.input-group-addon:first-child,.input-group-btn:first-child>.btn,.input-group-btn:first-child>.btn-group>.btn,.input-group-btn:first-child>.dropdown-toggle,.input-group-btn:last-child>.btn:not(:last-child):not(.dropdown-toggle),.input-group-btn:last-child>.btn-group:not(:last-child)>.btn{border-top-right-radius:0;border-bottom-right-radius:0}.input-group-addon:first-child{border-right:0}.input-group .form-control:last-child,.input-group-addon:last-child,.input-group-btn:last-child>.btn,.input-group-btn:last-child>.btn-group>.btn,.input-group-btn:last-child>.dropdown-toggle,.input-group-btn:first-child>.btn:not(:first-child),.input-group-btn:first-child>.btn-group:not(:first-child)>.btn{border-top-left-radius:0;border-bottom-left-radius:0}.input-group-addon:last-child{border-left:0}.input-group-btn{position:relative;font-size:0;white-space:nowrap}.input-group-btn>.btn{position:relative}.input-group-btn>.btn+.btn{margin-left:-1px}.input-group-btn>.btn:hover,.input-group-btn>.btn:focus,.input-group-btn>.btn:active{z-index:2}.input-group-btn:first-child>.btn,.input-group-btn:first-child>.btn-group{margin-right:-1px}.input-group-btn:last-child>.btn,.input-group-btn:last-child>.btn-group{margin-left:-1px}.nav{padding-left:0;margin-bottom:0;list-style:none}.nav>li{position:relative;display:block}.nav>li>a 15px}.nav>li>a:hover,.nav>li>a:focus{text-decoration:none;background-color:#eee}.nav>li.disabled>a{color:#777}.nav>li.disabled>a:hover,.nav>li.disabled>a:focus{color:#777;tex

decoration:none;cursor:not-allowed;background-color:transparent}.nav .open>a,.nav
.open>a:hover,.nav .open>a:focus{background-color:#eee;border-color:#428bca}.nav
.nav-divider{height:1px;margin:9px 0;overflow:hidden;background-color:#e5e5e5}.nav>li>a>img{max-
width:none}.nav-tabs{border-bottom:1px solid #ddd}.nav-tabs>li{float:left;margin-
bottom:-1px}.nav-tabs>li>a{margin-right:2px;line-height:1.42857143;border:1px
solid transparent;border-radius:4px 4px 0 0}.nav-tabs>li>a:hover{border-
color:#eee #eee #ddd}.nav-tabs>li.active>a,.nav-tabs>li.active>a:hover,.nav-
tabs>li.active>a:focus{color:#555;cursor:default;background-color:#fff;border:1px
solid #ddd;border-bottom-color:transparent}.nav-tabs.nav-justified{width:100%;border-
bottom:0}.nav-tabs.nav-justified>li{float:none}.nav-tabs.nav-justified>li>a{margin-
bottom:5px;text-align:center}.nav-tabs.nav-justified>.dropdown .dropdown-
menu{top:auto;left:auto}@media (min-width:768px){.nav-tabs.nav-justified>li{display:table-
cell;width:1%}.nav-tabs.nav-justified>li>a{margin-bottom:0}}.nav-tabs.nav-
justified>li>a{margin-right:0;border-radius:4px}.nav-tabs.nav-justified>.active>a,.nav-
tabs.nav-justified>.active>a:hover,.nav-tabs.nav-justified>.active>a:focus{border:1px
solid #ddd}@media (min-width:768px){.nav-tabs.nav-justified>li>a{border-
bottom:1px solid #ddd;border-radius:4px 4px 0 0}.nav-tabs.nav-justified>.active>a,.nav-
tabs.nav-justified>.active>a:hover,.nav-tabs.nav-justified>.active>a:focus{border-
bottom-color:#fff}}.nav-pills>li{float:left}.nav-pills>li>a{border-radius:4px}.nav-
pills>li+li{margin-left:2px}.nav-pills>li.active>a,.nav-pills>li.active>a:hover,.nav-
pills>li.active>a:focus{color:#fff;background-color:#428bca}.nav-stacked>li{float:none}.nav-
stacked>li+li{margin-top:2px;margin-left:0}.nav-justified{width:100%}.nav-
justified>li{float:none}.nav-justified>li>a{margin-bottom:5px;text-align:center}.nav-
justified>.dropdown .dropdown-menu{top:auto;left:auto}@media (min-
width:768px){.nav-justified>li{display:table-cell;width:1%}.nav-justified>li>a{margin-
bottom:0}}.nav-tabs-justified{border-bottom:0}.nav-tabs-justified>li>a{margin-
right:0;border-radius:4px}.nav-tabs-justified>.active>a,.nav-tabs-justified>.active>a:hover,.nav-
tabs-justified>.active>a:focus{border:1px solid #ddd}@media (min-width:768px){.nav-
tabs-justified>li>a{border-bottom:1px solid #ddd;border-radius:4px 4px 0
0}.nav-tabs-justified>.active>a,.nav-tabs-justified>.active>a:hover,.nav-
tabs-justified>.active>a:focus{border-bottom-color:#fff}}.tab-content>.tab-
pane{display:none}.tab-content>.active{display:block}.nav-tabs .dropdown-
menu{margin-top:-1px;border-top-left-radius:0;border-top-right-radius:0}.navbar{position:relative;min-
height:50px;margin-bottom:20px;border:1px solid transparent}@media (min-
width:768px){.navbar{border-radius:4px}}@media (min-width:768px){.navbar-
header{float:left}}.navbar-collapse{padding-right:15px;padding-left:15px;overflow-
x:visible;-webkit-overflow-scrolling:touch;border-top:1px solid transparent;-
webkit-box-shadow:inset 0 1px 0 rgba(255,255,255,.1);box-shadow:inset
0 1px 0 rgba(255,255,255,.1)}.navbar-collapse.in{overflow-y:auto}@media
(min-width:768px){.navbar-collapse{width:auto;border-top:0;-webkit-box-
shadow:none;box-shadow:none}.navbar-collapse.collapse{display:block!important;height:auto!important;paddi
bottom:0;overflow:visible!important}.navbar-collapse.in{overflow-y:visible}.navbar-
fixed-top .navbar-collapse,.navbar-static-top .navbar-collapse,.navbar-
fixed-bottom .navbar-collapse{padding-right:0;padding-left:0}}.navbar-
fixed-top .navbar-collapse,.navbar-fixed-bottom .navbar-collapse{max-
height:340px}@media (max-width:480px) and (orientation:landscape){.navbar-

fixed-top .navbar-collapse,.navbar-fixed-bottom .navbar-collapse{max-height:200px}}.container>.navbar-header,.container-fluid>.navbar-header,.container>.navbar-collapse,.container-fluid>.navbar-collapse{margin-right:-15px;margin-left:-15px}@media (min-width:768px){.container>.navbar-header,.container-fluid>.navbar-header,.container>.navbar-collapse,.container-fluid>.navbar-collapse{margin-right:0;margin-left:0}}.navbar-static-top{z-index:1000;border-width:0 0 1px}@media (min-width:768px){.navbar-static-top{border-radius:0}}.navbar-fixed-top,.navbar-fixed-bottom{position:fixed;right:0;left:0;z-index:1030;-webkit-transform:translate3d(0,0,0);-o-transform:translate3d(0,0,0);transform:translate3d(0,0,0)}@media (min-width:768px){.navbar-fixed-top,.navbar-fixed-bottom{border-radius:0}}.navbar-fixed-top{top:0;border-width:0 0 1px}.navbar-fixed-bottom{bottom:0;margin-bottom:0;border-width:1px 0 0}.navbar-brand{float:left;height:50px;padding:15px 15px;font-size:18px;line-height:20px}.navbar-brand:hover,.navbar-brand:focus{text-decoration:none}@media (min-width:768px){.navbar>.container .navbar-brand,.navbar>.container-fluid .navbar-brand{margin-left:-15px}}.navbar-toggle{position:relative;float:right;padding:9px 10px;margin-top:8px;margin-right:15px;margin-bottom:8px;background-color:transparent;background-image:none;border:1px solid transparent;border-radius:4px}.navbar-toggle:focus{outline:0}.navbar-toggle .icon-bar{display:block;width:22px;height:2px;border-radius:1px}.navbar-toggle .icon-bar+.icon-bar{margin-top:4px}@media (min-width:768px){.navbar-toggle{display:none}}.navbar-nav{margin:7.5px -15px}.navbar-nav>li>a{padding-top:10px;padding-bottom:10px;line-height:20px}@media (max-width:767px){.navbar-nav .open .dropdown-menu{position:static;float:none;width:auto;margin-top:0;background-color:transparent;border:0;-webkit-box-shadow:none;box-shadow:none}.navbar-nav .open .dropdown-menu>li>a,.navbar-nav .open .dropdown-menu .dropdown-header{padding:5px 15px 5px 25px}.navbar-nav .open .dropdown-menu>li>a{line-height:20px}.navbar-nav .open .dropdown-menu>li>a:hover,.navbar-nav .open .dropdown-menu>li>a:focus{background-image:none}}@media

(min-width:768px){.navbar-nav{float:left;margin:0}.navbar-nav>li{float:left}.navbar-nav>li>a{padding-top:15px;padding-bottom:15px}.navbar-nav.navbar-right:last-child{margin-right:-15px}}@media (min-width:768px){.navbar-left{float:left!important}.navbar-right{float:right!important}}.navbar-form{padding:10px 15px;margin-top:8px;margin-right:-15px;margin-bottom:8px;margin-left:-15px;border-top:1px solid transparent;border-bottom:1px solid transparent;-webkit-box-shadow:inset 0 1px 0 rgba(255,255,255,.1),0 1px 0 rgba(255,255,255,.1);box-shadow:inset 0 1px 0 rgba(255,255,255,.1),0 1px 0 rgba(255,255,255,.1)}@media (min-width:768px){.navbar-form .form-group{display:inline-block;margin-bottom:0;vertical-align:middle}.navbar-form .form-control{display:inline-block;width:auto;vertical-align:middle}.navbar-form .input-group{display:inline-table;vertical-align:middle}.navbar-form .input-group .input-group-addon,.navbar-form .input-group .input-group-btn,.navbar-form .input-group .form-control{width:auto}.navbar-form .input-group>.form-control{width:100%}.navbar-form .control-label{margin-bottom:0;vertical-align:middle}.navbar-form .radio,.navbar-form .checkbox{display:inline-block;margin-top:0;margin-

bottom:0;vertical-align:middle}.navbar-form .radio label,.navbar-form .checkbox label{padding-left:0}.navbar-form .radio input[type=radio],.navbar-form .checkbox input[type=checkbox]{position:relative;margin-left:0}.navbar-form .has-feedback .form-control-feedback{top:0}}@media (max-width:767px){.navbar-form .form-group{margin-bottom:5px}}@media (min-width:768px){.navbar-form{width:auto;padding-top:0;padding-bottom:0;margin-right:0;margin-left:0;border:0;-webkit-box-shadow:none;box-shadow:none}.navbar-form.navbar-right:last-child{margin-right:-15px}}.navbar-nav>li>.dropdown-menu{margin-top:0;border-top-left-radius:0;border-top-right-radius:0}.navbar-fixed-bottom .navbar-nav>li>.dropdown-menu{border-bottom-right-radius:0;border-bottom-left-radius:0}.navbar-btn{margin-top:8px;margin-bottom:8px}.navbar-btn.btn-sm{margin-top:10px;margin-bottom:10px}.navbar-btn.btn-xs{margin-top:14px;margin-bottom:14px}.navbar-text{margin-top:15px;margin-bottom:15px}@media (min-width:768px){.navbar-text{float:left;margin-right:15px;margin-left:15px}.navbar-text.navbar-right:last-child{margin-right:0}}.navbar-default{background-color:#f8f8f8;border-color:#e7e7e7}.navbar-default .navbar-brand{color:#777}.navbar-default .navbar-brand:hover,.navbar-default .navbar-brand:focus{color:#5e5e5e;background-color:transparent}.navbar-default .navbar-text{color:#777}.navbar-default .navbar-nav>li>a{color:#777}.navbar-default .navbar-nav>li>a:hover,.navbar-default .navbar-nav>li>a:focus{color:#333;background-color:transparent}.navbar-default .navbar-nav>.active>a,.navbar-default .navbar-nav>.active>a:hover,.navbar-default .navbar-nav>.active>a:focus{color:#555;background-color:#e7e7e7}.navbar-default .navbar-nav>.disabled>a,.navbar-default .navbar-nav>.disabled>a:hover,.navbar-default .navbar-nav>.disabled>a:focus{color:#ccc;background-color:transparent}.navbar-default .navbar-toggle{border-color:#ddd}.navbar-default .navbar-toggle:hover,.navbar-default .navbar-toggle:focus{background-color:#ddd}.navbar-default .navbar-toggle .icon-bar{background-color:#888}.navbar-default .navbar-collapse,.navbar-default .navbar-form{border-color:#e7e7e7}.navbar-default .navbar-nav>.open>a,.navbar-default .navbar-nav>.open>a:hover,.navbar-default .navbar-nav>.open>a:focus{color:#555;background-color:#e7e7e7}@media (max-width:767px){.navbar-default .navbar-nav .open .dropdown-menu>li>a{color:#777}.navbar-default .navbar-nav .open .dropdown-menu>li>a:hover,.navbar-default .navbar-nav .open .dropdown-menu>li>a:focus{color:#333;background-color:transparent}.navbar-default .navbar-nav .open .dropdown-menu>.active>a,.navbar-default .navbar-nav .open .dropdown-menu>.active>a:hover,.navbar-default .navbar-nav .open .dropdown-menu>.active>a:focus{color:#555;background-color:#e7e7e7}.navbar-default .navbar-nav .open .dropdown-menu>.disabled>a,.navbar-default .navbar-nav .open .dropdown-menu>.disabled>a:hover,.navbar-default .navbar-nav .open .dropdown-menu>.disabled>a:focus{color:#ccc;background-color:transparent}}.navbar-default .navbar-link{color:#777}.navbar-default .navbar-link:hover{color:#333}.navbar-default .btn-link{color:#777}.navbar-default .btn-link:hover,.navbar-default .btn-link:focus{color:#333}.navbar-default .btn-link[disabled]:hover,fieldset[disabled] .navbar-default .btn-link:hover,.navbar-default .btn-link[disabled]:focus,fieldset[disabled] .navbar-default .btn-link:focus{color:#ccc}.navbar-inverse{background-color:#222;border-color:#080808}.navbar-inverse .navbar-brand{color:#777}.navbar-inverse

.navbar-brand:hover,.navbar-inverse .navbar-brand:focus{color:#fff;background-color:transparent}.navbar-inverse .navbar-text{color:#777}.navbar-inverse .navbar-nav>li>a{color:#777}.navbar-inverse .navbar-nav>li>a:hover,.navbar-inverse .navbar-nav>li>a:focus{color:#fff;background-color:transparent}.navbar-inverse .navbar-nav>.active>a,.navbar-inverse .navbar-nav>.active>a:hover,.navbar-inverse .navbar-nav>.active>a:focus{color:#fff;background-color:#080808}.navbar-inverse .navbar-nav>.disabled>a,.navbar-inverse .navbar-nav>.disabled>a:hover,.navbar-inverse .navbar-nav>.disabled>a:focus{color:#444;background-color:transparent}.navbar-inverse .navbar-toggle{border-color:#333}.navbar-inverse .navbar-toggle:hover,.navbar-inverse .navbar-toggle:focus{background-color:#333}.navbar-inverse .navbar-toggle .icon-bar{background-color:#fff}.navbar-inverse .navbar-collapse,.navbar-inverse .navbar-form{border-color:#101010}.navbar-inverse .navbar-nav>.open>a,.navbar-inverse .navbar-nav>.open>a:hover,.navbar-inverse .navbar-nav>.open>a:focus{color:#fff;background-color:#080808}@media (max-width:767px){.navbar-inverse .navbar-nav .open .dropdown-menu>.dropdown-header{border-color:#080808}.navbar-inverse .navbar-nav .open .dropdown-menu .divider{background-color:#080808}.navbar-inverse .navbar-nav .open .dropdown-menu>li>a{color:#777}.navbar-inverse .navbar-nav .open .dropdown-menu>li>a:hover,.navbar-inverse .navbar-nav .open .dropdown-menu>li>a:focus{color:#fff;background-color:transparent}.navbar-inverse .navbar-nav .open .dropdown-menu>.active>a,.navbar-inverse .navbar-nav .open .dropdown-menu>.active>a:hover,.navbar-inverse .navbar-nav .open .dropdown-menu>.active>a:focus{color:#fff;background-color:#080808}.navbar-inverse .navbar-nav .open .dropdown-menu>.disabled>a,.navbar-inverse .navbar-nav .open .dropdown-menu>.disabled>a:hover,.navbar-inverse .navbar-nav .open .dropdown-menu>.disabled>a:focus{color:#444;background-color:transparent}}.navbar-inverse .navbar-link{color:#777}.navbar-inverse .navbar-link:hover{color:#fff}.navbar-inverse .btn-link{color:#777}.navbar-inverse .btn-link:hover,.navbar-inverse .btn-link:focus{color:#fff}.navbar-inverse .btn-link[disabled]:hover,fieldset[disabled] .navbar-inverse .btn-link:hover,.navbar-inverse .btn-link[disabled]:focus,fieldset[disabled] .navbar-inverse .btn-link:focus{color:#444}.breadcrumb{padding:8px 15px;margin-bottom:20px;list-style:none;background-color:#f5f5f5;border-radius:4px}.breadcrumb>li{display:inline-block}.breadcrumb>li+li:before{padding:0 5px;color:#ccc;content:"/\00a0"}.breadcrumb>.active{color:#777
block;padding-left:0;margin:20px 0;border-radius:4px}.pagination>li{display:inline}.pagination>li>a,.paginati
12px;margin-left:-1px;line-height:1.42857143;color:#428bca;text-decoration:none;background-color:#fff;border:1px solid #ddd}.pagination>li:first-child>a,.pagination>li:first-child>span{margin-left:0;border-top-left-radius:4px;border-bottom-left-radius:4px}.pagination>li:last-child>a,.pagination>li:last-child>span{border-top-right-radius:4px;border-bottom-right-radius:4px}.pagination>li>a:hover,.pagination>li>span:hover,.pagin
color:#eee;border-color:#ddd}.pagination>.active>a,.pagination>.active>span,.pagination>.active>a:hover,.
index:2;color:#fff;cursor:default;background-color:#428bca;border-color:#428bca}.pagination>.disabled>span
allowed;background-color:#fff;border-color:#ddd}.pagination-lg>li>a,.pagination-lg>li>span{padding:10px 16px;font-size:18px}.pagination-lg>li:first-child>a,.pagination-lg>li:first-child>span{border-top-left-radius:6px;border-bottom-left-radius:6px}.pagination-lg>li:last-child>a,.pagination-lg>li:last-child>span{border-top-right-radius:6px;border-bottom-right-radius:6px}.pagination-sm>li>a,.pagination-sm>li>span{padding:5px

10px;font-size:12px}.pagination-sm>li:first-child>a,.pagination-sm>li:first-child>span{border-top-left-radius:3px;border-bottom-left-radius:3px}.pagination-sm>li:last-child>a,.pagination-sm>li:last-child>span{border-top-right-radius:3px;border-bottom-right-radius:3px}.pager{padding-left:0;margin:20px 0;text-align:center;list-style:none}.pager li{display:inline}.pager li>a,.pager li>span{display:inline-block;padding:5px 14px;background-color:#fff;border:1px solid #ddd;border-radius:15px}.pager li>a:hover,.pager li>a:focus{text-decoration:none;background-color:#eee}.pager .next>a,.pager .next>span{float:right}.pager .previous>a,.pager .previous>span{float:left}.pager .disabled>a,.pager .disabled>a:hover,.pager .disabled>a:focus,.pager .disabled>span{color:#777;cursor:not-allowed;background-color:#fff}.label{display:inline;padding:.2em .6em .3em;font-size:75%;font-weight:700;line-height:1;color:#fff;text-align:center;white-space:nowrap;vertical-align:baseline;border-radius:.25em}a.label:hover,a.label:focus{color:#fff;text-decoration:none;cursor:pointer}.label:empty{display:none}.btn .label{position:relative;top:-1px}.label-default{background-color:#777}.label-default[href]:hover,.label-default[href]:focus{background-color:#5e5e5e}.label-primary{background-color:#428bca}.label-primary[href]:hover,.label-primary[href]:focus{background-color:#3071a9}.label-success{background-color:#5cb85c}.label-success[href]:hover,.label-success[href]:focus{background-color:#449d44}.label-info{background-color:#5bc0de}.label-info[href]:hover,.label-info[href]:focus{background-color:#31b0d5}.label-warning{background-color:#f0ad4e}.label-warning[href]:hover,.label-warning[href]:focus{background-color:#ec971f}.label-danger{background-color:#d9534f}.label-danger[href]:hover,.label-danger[href]:focus{background-color:#c9302c}.badge{display:inline-block;min-width:10px;padding:3px 7px;font-size:12px;font-weight:700;line-height:1;color:#fff;text-align:center;white-space:nowrap;vertical-align:baseline;background-color:#777;border-radius:10px}.badge:empty{display:none}.btn .badge{position:relative;top:-1px}.btn-xs .badge{top:0;padding:1px 5px}a.badge:hover,a.badge:focus{color:#fff;text-decoration:none;cursor:pointer}a.list-group-item.active>.badge,.nav-pills>.active>a>.badge{color:#428bca;ba
color:#fff}.nav-pills>li>a>.badge{margin-left:3px}.jumbotron{padding:30px;margin-bottom:30px;color:inherit;background-color:#eee}.jumbotron h1,.jumbotron .h1{color:inherit}.jumbotron p{margin-bottom:15px;font-size:21px;font-weight:200}.jumbotron>hr{border-top-color:#d5d5d5}.container .jumbotron{border-radius:6px}.jumbotron .container{max-width:100%}@media screen and (min-width:768px){.jumbotron{padding-top:48px;padding-bottom:48px}.container .jumbotron{padding-right:60px;padding-left:60px}.jumbotron h1,.jumbotron .h1{font-size:63px}}.thumbnail{display:block;padding:4px;margin-bottom:20px;line-height:1.42857143;background-color:#fff;border:1px solid #ddd;border-radius:4px;-webkit-transition:all .2s ease-in-out;-o-transition:all .2s ease-in-out;transition:all .2s ease-in-out}.thumbnail>img,.thumbnail a>img{margin-right:auto;margin-left:auto}a.thumbnail:hover,a.thumbnail:focus,a.thumbnail.active{border-color:#428bca}.thumbnail .caption{padding:9px;color:#333}.alert{padding:15px;margin-bottom:20px;border:1px solid transparent;border-radius:4px}.alert h4{margin-top:0;color:inherit}.alert .alert-link{font-weight:700}.alert>p,.alert>ul{margin-bottom:0}.alert>p+p{margin-top:5px}.alert-dismissable,.alert-dismissible{padding-right:35px}.alert-dismissable .close,.alert-dismissible .close{position:relative;top:-2px;right:-21px;color:inherit}.alert-success{color:#3c763d;background-color:#dff0d8;border-

color:#d6e9c6}.alert-success hr{border-top-color:#c9e2b3}.alert-success .alert-link{color:#2b542c}.alert-info{color:#31708f;background-color:#d9edf7;border-color:#bce8f1}.alert-info hr{border-top-color:#a6e1ec}.alert-info .alert-link{color:#245269}.alert-warning{color:#8a6d3b;background-color:#fcf8e3;border-color:#faebcc}.alert-warning hr{border-top-color:#f7e1b5}.alert-warning .alert-link{color:#66512c}.alert-danger{color:#a94442;background-color:#f2dede;border-color:#ebccd1}.alert-danger hr{border-top-color:#e4b9c0}.alert-danger .alert-link{color:#843534}@-webkit-keyframes progress-bar-stripes{from{background-position:40px 0}to{background-position:0 0}}@-o-keyframes progress-bar-stripes{from{background-position:40px 0}to{background-position:0 0}}@keyframes progress-bar-stripes{from{background-position:40px 0}to{background-position:0 0}}.progress{height:20px;margin-bottom:20px;overflow:hidden;background-color:#f5f5f5;border-radius:4px;-webkit-box-shadow:inset 0 1px 2px rgba(0,0,0,.1);box-shadow:inset 0 1px 2px rgba(0,0,0,.1)}.progress-bar{float:left;width:0;height:100%;font-size:12px;line-height:20px;color:#fff;text-align:center;background-color:#428bca;-webkit-box-shadow:inset 0 -1px 0 rgba(0,0,0,.15);box-shadow:inset 0 -1px 0 rgba(0,0,0,.15);-webkit-transition:width .6s ease;-o-transition:width .6s ease;transition:width .6s ease}.progress-striped .progress-bar,.progress-bar-striped{background-image:-webkit-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:-o-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);-webkit-background-size:40px 40px;background-size:40px 40px}.progress.active .progress-bar,.progress-bar.active{-webkit-animation:progress-bar-stripes 2s linear infinite;-o-animation:progress-bar-stripes 2s linear infinite;animation:progress-bar-stripes 2s linear infinite}.progress-bar[aria-valuenow="1"],.progress-bar[aria-valuenow="2"]{min-width:30px}.progress-bar[aria-valuenow="0"]{min-width:30px;color:#777;background-color:transparent;background-image:none;-webkit-box-shadow:none;box-shadow:none}.progress-bar-success{background-color:#5cb85c}.progress-striped .progress-bar-success{background-image:-webkit-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:-o-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent)}.progress-bar-info{background-color:#5bc0de}.progress-striped .progress-bar-info{background-image:-webkit-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:-o-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent)}.progress-bar-warning{background-

color:#f0ad4e}.progress-striped .progress-bar-warning{background-image:-webkit-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:-o-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent)}.progress-bar-danger{background-color:#d9534f}.progress-striped .progress-bar-danger{background-image:-webkit-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:-o-linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent);background-image:linear-gradient(45deg,rgba(255,255,255,.15) 25%,transparent 25%,transparent 50%,rgba(255,255,255,.15) 50%,rgba(255,255,255,.15) 75%,transparent 75%,transparent)}.media,.media-body{overflow:hidden;zoom:1}.media,.media .media{margin-top:15px}.media:first-child{margin-top:0}.media-object{display:block}.media-heading{margin:0 0 5px}.media>.pull-left{margin-right:10px}.media>.pull-right{margin-left:10px}.media-list{padding-left:0;list-style:none}.list-group{padding-left:0;margin-bottom:20px}.list-group-item{position:relative;display:block;padding:10px 15px;margin-bottom:-1px;background-color:#fff;border:1px solid #ddd}.list-group-item:first-child{border-top-left-radius:4px;border-top-right-radius:4px}.list-group-item:last-child{margin-bottom:0;border-bottom-right-radius:4px;border-bottom-left-radius:4px}.list-group-item>.badge{float:right}.list-group-item>.badge+.badge{margin-right:5px}a.list-group-item{color:#555}a.list-group-item .list-group-item-heading{color:#333}a.list-group-item:hover,a.list-group-item:focus{color:#555;text-decoration:none;background-color:#f5f5f5}.list-group-item.disabled,.list-group-item.disabled:hover,.list-group-item.disabled:focus{color:#777;background-color:#eee}.list-group-item.disabled .list-group-item-heading,.list-group-item.disabled:hover .list-group-item-heading,.list-group-item.disabled:focus .list-group-item-heading{color:inherit}.list-group-item.disabled .list-group-item-text,.list-group-item.disabled:hover .list-group-item-text,.list-group-item.disabled:focus .list-group-item-text{color:#777}.list-group-item.active,.list-group-item.active:hover,.list-group-item.active:focus{z-index:2;color:#fff;background-color:#428bca;border-color:#428bca}.list-group-item.active .list-group-item-heading,.list-group-item.active:hover .list-group-item-heading,.list-group-item.active:focus .list-group-item-heading,.list-group-item.active .list-group-item-heading>small,.list-group-item.active:hover .list-group-item-heading>small,.list-group-item.active:focus .list-group-item-heading>small,.list-group-item.active .list-group-item-heading>.small,.list-group-item.active:hover .list-group-item-heading>.small,.list-group-item.active:focus .list-group-item-heading>.small{color:inherit}.list-group-item.active .list-group-item-text,.list-group-item.active:hover .list-group-item-text,.list-group-item.active:focus .list-group-item-text{color:#e1edf7}.list-group-item-success{color:#3c763d;background-color:#dff0d8}a.list-group-item-success{color:#3c763d}a.list-group-item-success .list-group-item-heading{color:inherit}a.list-group-item-success:hover,a.list-group-item-success:focus{color:#3c763d;background-color:#d0e9c6}a.list-

group-item-success.active,a.list-group-item-success.active:hover,a.list-group-item-success.active:focus{color:#fff;background-color:#3c763d;border-color:#3c763d}.list-group-item-info{color:#31708f;background-color:#d9edf7}a.list-group-item-info{color:#31708f}a.list-group-item-info .list-group-item-heading{color:inherit}a.list-group-item-info:hover,a.list-group-item-info:focus{color:#31708f;background-color:#c4e3f3}a.list-group-item-info.active,a.list-group-item-info.active:hover,a.list-group-item-info.active:focus{color:#fff;background-color:#31708f;border-color:#31708f}.list-group-item-warning{color:#8a6d3b;background-color:#fcf8e3}a.list-group-item-warning{color:#8a6d3b}a.list-group-item-warning .list-group-item-heading{color:inherit}a.list-group-item-warning:hover,a.list-group-item-warning:focus{color:#8a6d3b;background-color:#faf2cc}a.list-group-item-warning.active,a.list-group-item-warning.active:hover,a.list-group-item-warning.active:focus{color:#fff;background-color:#8a6d3b;border-color:#8a6d3b}.list-group-item-danger{color:#a94442;background-color:#f2dede}a.list-group-item-danger{color:#a94442}a.list-group-item-danger .list-group-item-heading{color:inherit}a.list-group-item-danger:hover,a.list-group-item-danger:focus{color:#a94442;background-color:#ebcccc}a.list-group-item-danger.active,a.list-group-item-danger.active:hover,a.list-group-item-danger.active:focus{color:#fff;background-color:#a94442;border-color:#a94442}.list-group-item-heading{margin-top:0;margin-bottom:5px}.list-group-item-text{margin-bottom:0;line-height:1.3}.panel{margin-bottom:20px;background-color:#fff;border:1px solid transparent;border-radius:4px;-webkit-box-shadow:0 1px 1px rgba(0,0,0,.05);box-shadow:0 1px 1px rgba(0,0,0,.05)}.panel-body{padding:15px}.panel-heading{padding:10px 15px;border-bottom:1px solid transparent;border-top-left-radius:3px;border-top-right-radius:3px}.panel-heading>.dropdown .dropdown-toggle{color:inherit}.panel-title{margin-top:0;margin-bottom:0;font-size:16px;color:inherit}.panel-title>a{color:inherit}.panel-footer{padding:10px 15px;background-color:#f5f5f5;border-top:1px solid #ddd;border-bottom-right-radius:3px;border-bottom-left-radius:3px}.panel>.list-group{margin-bottom:0}.panel>.list-group .list-group-item{border-width:1px 0;border-radius:0}.panel>.list-group:first-child .list-group-item:first-child{border-top:0;border-top-left-radius:3px;border-top-right-radius:3px}.panel>.list-group:last-child .list-group-item:last-child{border-bottom:0;border-bottom-right-radius:3px;border-bottom-left-radius:3px}.panel-heading+.list-group .list-group-item:first-child{border-top-width:0}.list-group+.panel-footer{border-top-width:0}.panel>.table,.panel>.table-responsive>.table,.panel>.panel-collapse>.table{margin-bottom:0}.panel>.table:first-child,.panel>.table-responsive:first-child>.table:first-child{border-top-left-radius:3px;border-top-right-radius:3px}.panel>.table:first-child>thead:first-child>tr:first-child td:first-child,.panel>.table-responsive:first-child>.table:first-child>thead:first-child>tr:first-child td:first-child,.panel>.table:first-child>tbody:first-child>tr:first-child td:first-child,.panel>.table-responsive:first-child>.table:first-child>tbody:first-child>tr:first-child td:first-child,.panel>.table:first-child>thead:first-child>tr:first-child th:first-child,.panel>.table-responsive:first-child>.table:first-child>thead:first-child>tr:first-child th:first-child,.panel>.table:first-child>tbody:first-child>tr:first-child th:first-child,.panel>.table-responsive:first-child>.table:first-child>tbody:first-child>tr:first-child th:first-child{border-top-left-radius:3px}.panel>.table:first-

child>thead:first-child>tr:first-child td:last-child,.panel>.table-responsive:first-child>.table:first-child>thead:first-child>tr:first-child td:last-child,.panel>.table:first-child>tbody:first-child>tr:first-child td:last-child,.panel>.table-responsive:first-child>.table:first-child>tbody:first-child>tr:first-child td:last-child,.panel>.table:first-child>thead:first-child>tr:first-child th:last-child,.panel>.table-responsive:first-child>.table:first-child>thead:first-child>tr:first-child th:last-child,.panel>.table:first-child>tbody:first-child>tr:first-child th:last-child,.panel>.table-responsive:first-child>.table:first-child>tbody:first-child>tr:first-child th:last-child{border-top-right-radius:3px}.panel>.table:last-child,.panel>.table-responsive:last-child>.table:last-child{border-bottom-right-radius:3px;border-bottom-left-radius:3px}.panel>.table:last-child>tbody:last-child>tr:last-child td:first-child,.panel>.table-responsive:last-child>.table:last-child>tbody:last-child>tr:last-child td:first-child,.panel>.table:last-child>tfoot:last-child>tr:last-child td:first-child,.panel>.table-responsive:last-child>.table:last-child>tfoot:last-child>tr:last-child td:first-child,.panel>.table:last-child>tbody:last-child>tr:last-child th:first-child,.panel>.table-responsive:last-child>.table:last-child>tbody:last-child>tr:last-child th:first-child,.panel>.table:last-child>tfoot:last-child>tr:last-child th:first-child,.panel>.table-responsive:last-child>.table:last-child>tfoot:last-child>tr:last-child th:first-child{border-bottom-left-radius:3px}.panel>.table:last-child>tbody:last-child>tr:last-child td:last-child,.panel>.table-responsive:last-child>.table:last-child>tbody:last-child>tr:last-child td:last-child,.panel>.table:last-child>tfoot:last-child>tr:last-child td:last-child,.panel>.table-responsive:last-child>.table:last-child>tfoot:last-child>tr:last-child td:last-child,.panel>.table:last-child>tbody:last-child>tr:last-child th:last-child,.panel>.table-responsive:last-child>.table:last-child>tbody:last-child>tr:last-child th:last-child,.panel>.table:last-child>tfoot:last-child>tr:last-child th:last-child,.panel>.table-responsive:last-child>.table:last-child>tfoot:last-child>tr:last-child th:last-child{border-bottom-right-radius:3px}.panel>.panel-body+.table,.panel>.panel-body+.table-responsive{border-top:1px solid #ddd}.panel>.table>tbody:first-child>tr:first-child th,.panel>.table>tbody:first-child>tr:first-child td{border-top:0}.panel>.table-bordered,.panel>.table-responsive>.table-bordered{border:0}.panel>.table-bordered>thead>tr>th:first-child,.panel>.table-responsive>.table-bordered>thead>tr>th:first-child,.panel>.table-bordered>tbody>tr>th:first-child,.panel>.table-responsive>.table-bordered>tbody>tr>th:first-child,.panel>.table-bordered>tfoot>tr>th:first-child,.panel>.table-responsive>.table-bordered>tfoot>tr>th:first-child,.panel>.table-bordered>thead>tr>td:first-child,.panel>.table-responsive>.table-bordered>thead>tr>td:first-child,.panel>.table-bordered>tbody>tr>td:first-child,.panel>.table-responsive>.table-bordered>tbody>tr>td:first-child,.panel>.table-bordered>tfoot>tr>td:first-child,.panel>.table-responsive>.table-bordered>tfoot>tr>td:first-child{border-left:0}.panel>.table-bordered>thead>tr>th:last-child,.panel>.table-responsive>.table-bordered>thead>tr>th:last-child,.panel>.table-bordered>tbody>tr>th:last-child,.panel>.table-responsive>.table-bordered>tbody>tr>th:last-child,.panel>.table-bordered>tfoot>tr>th:last-child,.panel>.table-responsive>.table-bordered>tfoot>tr>th:last-child,.panel>.table-bordered>thead>tr>td:last-child,.panel>.table-responsive>.table-bordered>thead>tr>td:last-child,.panel>.table-bordered>tbody>tr>td:last-child,.panel>.table-responsive>.table-bordered>tbody>tr>td:last-child,.panel>.table-

bordered>tfoot>tr>td:last-child,.panel>.table-responsive>.table-bordered>tfoot>tr>td:last-child{border-right:0}.panel>.table-bordered>thead>tr:first-child>td,.panel>.table-responsive>.table-bordered>thead>tr:first-child>td,.panel>.table-bordered>tbody>tr:first-child>td,.panel>.table-responsive>.table-bordered>tbody>tr:first-child>td,.panel>.table-bordered>thead>tr:first-child>th,.panel>.table-responsive>.table-bordered>thead>tr:first-child>th,.panel>.table-bordered>tbody>tr:first-child>th,.panel>.table-responsive>.table-bordered>tbody>tr:first-child>th{border-bottom:0}.panel>.table-bordered>tbody>tr:last-child>td,.panel>.table-responsive>.table-bordered>tbody>tr:last-child>td,.panel>.table-bordered>tfoot>tr:last-child>td,.panel>.table-responsive>.table-bordered>tfoot>tr:last-child>td,.panel>.table-bordered>tbody>tr:last-child>th,.panel>.table-responsive>.table-bordered>tbody>tr:last-child>th,.panel>.table-bordered>tfoot>tr:last-child>th,.panel>.table-responsive>.table-bordered>tfoot>tr:last-child>th{border-bottom:0}.panel>.table-responsive{margin-bottom:0;border:0}.panel-group{margin-bottom:20px}.panel-group .panel{margin-bottom:0;border-radius:4px}.panel-group .panel+.panel{margin-top:5px}.panel-group .panel-heading{border-bottom:0}.panel-group .panel-heading+.panel-collapse>.panel-body{border-top:1px solid #ddd}.panel-group .panel-footer{border-top:0}.panel-group .panel-footer+.panel-collapse .panel-body{border-bottom:1px solid #ddd}.panel-default{border-color:#ddd}.panel-default>.panel-heading{color:#333;background-color:#f5f5f5;border-color:#ddd}.panel-default>.panel-heading+.panel-collapse>.panel-body{border-top-color:#ddd}.panel-default>.panel-heading .badge{color:#f5f5f5;background-color:#333}.panel-default>.panel-footer+.panel-collapse>.panel-body{border-bottom-color:#ddd}.panel-primary{border-color:#428bca}.panel-primary>.panel-heading{color:#fff;background-color:#428bca;border-color:#428bca}.panel-primary>.panel-heading+.panel-collapse>.panel-body{border-top-color:#428bca}.panel-primary>.panel-heading .badge{color:#428bca;background-color:#fff}.panel-primary>.panel-footer+.panel-collapse>.panel-body{border-bottom-color:#428bca}.panel-success{border-color:#d6e9c6}.panel-success>.panel-heading{color:#3c763d;background-color:#dff0d8;border-color:#d6e9c6}.panel-success>.panel-heading+.panel-collapse>.panel-body{border-top-color:#d6e9c6}.panel-success>.panel-heading .badge{color:#dff0d8;background-color:#3c763d}.panel-success>.panel-footer+.panel-collapse>.panel-body{border-bottom-color:#d6e9c6}.panel-info{border-color:#bce8f1}.panel-info>.panel-heading{color:#31708f;background-color:#d9edf7;border-color:#bce8f1}.panel-info>.panel-heading+.panel-collapse>.panel-body{border-top-color:#bce8f1}.panel-info>.panel-heading .badge{color:#d9edf7;background-color:#31708f}.panel-info>.panel-footer+.panel-collapse>.panel-body{border-bottom-color:#bce8f1}.panel-warning{border-color:#faebcc}.panel-warning>.panel-heading{color:#8a6d3b;background-color:#fcf8e3;border-color:#faebcc}.panel-warning>.panel-heading+.panel-collapse>.panel-body{border-top-color:#faebcc}.panel-warning>.panel-heading .badge{color:#fcf8e3;background-color:#8a6d3b}.panel-warning>.panel-footer+.panel-collapse>.panel-body{border-bottom-color:#faebcc}.panel-danger{border-color:#ebccd1}.panel-danger>.panel-heading{color:#a94442;background-color:#f2dede;border-color:#ebccd1}.panel-danger>.panel-heading+.panel-collapse>.panel-body{border-top-color:#ebccd1}.panel-danger>.panel-heading .badge{color:#f2dede;background-color:#a94442}.panel-danger>.panel-

footer+.panel-collapse>.panel-body{border-bottom-color:#ebccd1}.embed-responsive{position:relative;display:block;height:0;padding:0;overflow:hidden}.embed-responsive .embed-responsive-item,.embed-responsive iframe,.embed-responsive embed,.embed-responsive object{position:absolute;top:0;bottom:0;left:0;width:100%;height:100%;border:0}.em responsive.embed-responsive-16by9{padding-bottom:56.25%}.embed-responsive.embed-responsive-4by3{padding-bottom:75%}.well{min-height:20px;padding:19px;margin-bottom:20px;background-color:#f5f5f5;border:1px solid #e3e3e3;border-radius:4px;-webkit-box-shadow:inset 0 1px 1px rgba(0,0,0,.05);box-shadow:inset 0 1px 1px rgba(0,0,0,.05)}.well blockquote{border-color:#ddd;border-color:rgba(0,0,0,.15)}.well-lg{padding:24px;border-radius:6px}.well-sm{padding:9px;border-radius:3px}.close{float:right;font-size:21px;font-weight:700;line-height:1;color:#000;text-shadow:0 1px 0 #fff;filter:alpha(opacity=20);opacity:.2}.close:hover,.close:focus{color:#000;text-decoration:none;cursor:pointer;filter:alpha(opacity=50);opacity:.5}button.close{-webkit-appearance:none;padding:0;cursor:pointer;background:0 0;border:0}.modal-open{overflow:hidden}.modal{position:fixed;top:0;right:0;bottom:0;left:0;z-index:1050;display:none;overflow:hidden;-webkit-overflow-scrolling:touch;outline:0}.modal.fade

.modal-dialog{-webkit-transition:-webkit-transform .3s ease-out;-o-transition:-o-transform .3s ease-out;transition:transform .3s ease-out;-webkit-transform:translate3d(0,-25%,0);-o-transform:translate3d(0,-25%,0);transform:translate3d(0,-25%,0)}.modal.in .modal-dialog{-webkit-transform:translate3d(0,0,0);-o-transform:translate3d(0,0,0);transform:translate3d(0,0,0 open .modal{overflow-x:hidden;overflow-y:auto}.modal-dialog{position:relative;width:auto;margin:10px}.moda content{position:relative;background-color:#fff;-webkit-background-clip:padding-box;background-clip:padding-box;border:1px solid #999;border:1px solid rgba(0,0,0,.2);border-radius:6px;outline:0;-webkit-box-shadow:0 3px 9px rgba(0,0,0,.5);box-shadow:0 3px 9px rgba(0,0,0,.5)}.modal-backdrop{position:fixed;top:0;right:0;bottom:0;left:0 index:1040;background-color:#000}.modal-backdrop.fade{filter:alpha(opacity=0);opacity:0}.modal-backdrop.in{filter:alpha(opacity=50);opacity:.5}.modal-header{min-height:16.43px;padding:15px;border-bottom:1px solid #e5e5e5}.modal-header .close{margin-top:-2px}.modal-title{margin:0;line-height:1.42857143}.modal-body{position:relative;padding:15px}.modal-footer{padding:15px;text-align:right;border-top:1px solid #e5e5e5}.modal-footer .btn+.btn{margin-bottom:0;margin-left:5px}.modal-footer .btn-group .btn+.btn{margin-left:-1px}.modal-footer .btn-block+.btn-block{margin-left:0}.modal-scrollbar-measure{position:absolute;top:-9999px;width:50px;height:50px;overflow:scroll}@media (min-width:768px){.modal-dialog{width:600px;margin:30px auto}.modal-content{-webkit-box-shadow:0 5px 15px rgba(0,0,0,.5);box-shadow:0 5px 15px rgba(0,0,0,.5)}.modal-sm{width:300px}}@media (min-width:992px){.modal-lg{width:900px}}.tooltip{position:absolute;z-index:1070;display:block;font-size:12px;line-height:1.4;visibility:visible;filter:alpha(opacity=0);opacity:0}.tooltip.in{filter:alpha(opacity=90);o 0;margin-top:-3px}.tooltip.right{padding:0 5px;margin-left:3px}.tooltip.bottom{padding:5px 0;margin-top:3px}.tooltip.left{padding:0 5px;margin-left:-3px}.tooltip-inner{max-width:200px;padding:3px 8px;color:#fff;text-align:center;text-decoration:none;background-color:#000;border-radius:4px}.tooltip-arrow{position:absolute;width:0;height:0;bo color:transparent;border-style:solid}.tooltip.top .tooltip-arrow{bottom:0;left:50%;margin-left:-5px;border-width:5px 5px 0;border-top-color:#000}.tooltip.top-left .tooltip-arrow{bottom:0;left:5px;border-width:5px 5px 0;border-top-color:#000}.tooltip.top-

right .tooltip-arrow{right:5px;bottom:0;border-width:5px 5px 0;border-top-color:#000}.tooltip.right .tooltip-arrow{top:50%;left:0;margin-top:-5px;border-width:5px 5px 5px 0;border-right-color:#000}.tooltip.left .tooltip-arrow{top:50%;right:0;margin-top:-5px;border-width:5px 0 5px 5px;border-left-color:#000}.tooltip.bottom .tooltip-arrow{top:0;left:50%;margin-left:-5px;border-width:0 5px 5px;border-bottom-color:#000}.tooltip.bottom-left .tooltip-arrow{top:0;left:5px;border-width:0 5px 5px;border-bottom-color:#000}.tooltip.bottom-right .tooltip-arrow{top:0;right:5px;border-width:0 5px 5px;border-bottom-color:#000}.popover{position:absolute;top:0;left:0;z-index:1060;display:none;max-width:276px;padding:1px;text-align:left;white-space:normal;background-color:#fff;-webkit-background-clip:padding-box;background-clip:padding-box;border:1px solid #ccc;border:1px solid rgba(0,0,0,.2);border-radius:6px;-webkit-box-shadow:0 5px 10px rgba(0,0,0,.2);box-shadow:0 5px 10px rgba(0,0,0,.2)}.popover.top{margin-top:-10px}.popover.right{margin-left:10px}.popover.bottom{margin-top:10px}.popover.left{margin-left:-10px}.popover-title{padding:8px 14px;margin:0;font-size:14px;font-weight:400;line-height:18px;background-color:#f7f7f7;border-bottom:1px solid #ebebeb;border-radius:5px 5px 0 0}.popover-content{padding:9px 14px}.popover>.arrow,.popover>.arrow:after{position:absolu
color:transparent;border-style:solid}.popover>.arrow{border-width:11px}.popover>.arrow:after{content:"";bo
width:10px}.popover.top>.arrow{bottom:-11px;left:50%;margin-left:-11px;border-top-color:#999;border-top-color:rgba(0,0,0,.25);border-bottom-width:0}.popover.top>.arrow:after{bottom:1px
left:-10px;content:" ";border-top-color:#fff;border-bottom-width:0}.popover.right>.arrow{top:50%;left:-11px;margin-top:-11px;border-right-color:#999;border-right-color:rgba(0,0,0,.25);border-left-width:0}.popover.right>.arrow:after{bottom:-10px;left:1px;content:" ";border-right-color:#fff;border-left-width:0}.popover.bottom>.arrow{top:-11px;left:50%;margin-left:-11px;border-top-width:0;border-bottom-color:#999;border-bottom-color:rgba(0,0,0,.25)}.popover.bottom>.arrow:after{top:1px;margin-left:-10px;content:" ";border-top-width:0;border-bottom-color:#fff}.popover.left>.arrow{top:50%;right:-11px;margin-top:-11px;border-right-width:0;border-left-color:#999;border-left-color:rgba(0,0,0,.25)}.popover.left>.arrow:after{right:1px;bottom:-10px;content:" ";border-right-width:0;border-left-color:#fff}.carousel{position:relative}.carousel-inner{position:relative;width:100%;overflow:hidden}.carousel-inner>.item{position:relative;display:none;-webkit-transition:.6s ease-in-out left;-o-transition:.6s ease-in-out left;transition:.6s ease-in-out left}.carousel-inner>.item>img,.carousel-inner>.item>a>img{line-height:1}.carousel-inner>.active,.carousel-inner>.next,.carousel-inner>.prev{display:block}.carousel-inner>.active{left:0}.carousel-inner>.next,.carousel-inner>.prev{position:absolute;top:0;width:100%}.carousel
inner>.next{left:100%}.carousel-inner>.prev{left:-100%}.carousel-inner>.next.left,.carousel-inner>.prev.right{left:0}.carousel-inner>.active.left{left:-100%}.carousel-inner>.active.right{left:100%}.carousel-control{position:absolute;top:0;bottom:0;left:0;width:15%;font-size:20px;color:#fff;text-align:center;text-shadow:0 1px 2px rgba(0,0,0,.6);filter:alpha(opacity=50);opacity:.5}.c
control.left{background-image:-webkit-linear-gradient(left,rgba(0,0,0,.5) 0,rgba(0,0,0,.0001) 100%);background-image:-o-linear-gradient(left,rgba(0,0,0,.5) 0,rgba(0,0,0,.0001) 100%);background-image:-webkit-gradient(linear,left top,right top,from(rgba(0,0,0,.5)),to(rgba(0,0,0,.0001)));background-image:linear-gradient(to right,rgba(0,0,0,.5) 0,rgba(0,0,0,.0001) 100%);filter:progid:DXImageTransform.Microsoft.gradient(s
endColorstr='#00000000', GradientType=1);background-repeat:repeat-

x}.carousel-control.right{right:0;left:auto;background-image:-webkit-linear-
gradient(left,rgba(0,0,0,.0001) 0,rgba(0,0,0,.5) 100%);background-image:-o-
linear-gradient(left,rgba(0,0,0,.0001) 0,rgba(0,0,0,.5) 100%);background-image:-
webkit-gradient(linear,left top,right top,from(rgba(0,0,0,.0001)),to(rgba(0,0,0,.5)));background-
image:linear-gradient(to right,rgba(0,0,0,.0001) 0,rgba(0,0,0,.5) 100%);fil-
ter:progid:DXImageTransform.Microsoft.gradient(startColorstr='#00000000',
endColorstr='#80000000', GradientType=1);background-repeat:repeat-
x}.carousel-control:hover,.carousel-control:focus{color:#fff;text-decoration:none;filter:alpha(opacity=90);outlin
control .icon-prev,.carousel-control .icon-next,.carousel-control .glyphicon-
chevron-left,.carousel-control .glyphicon-chevron-right{position:absolute;top:50%;z-
index:5;display:inline-block}.carousel-control .icon-prev,.carousel-control
.glyphicon-chevron-left{left:50%;margin-left:-10px}.carousel-control .icon-
next,.carousel-control .glyphicon-chevron-right{right:50%;margin-right:-
10px}.carousel-control .icon-prev,.carousel-control .icon-next{width:20px;height:20px;margin-
top:-10px;font-family:serif}.carousel-control .icon-prev:before{content:'\2039'}.carousel-
control .icon-next:before{content:'\203a'}.carousel-indicators{position:absolute;bottom:10px;left:50%;z-
index:15;width:60%;padding-left:0;margin-left:-30%;text-align:center;list-
style:none}.carousel-indicators li{display:inline-block;width:10px;height:10px;margin:1px;text-
indent:-999px;cursor:pointer;background-color:#000 \9;background-color:rgba(0,0,0,0);border:1px
solid #fff;border-radius:10px}.carousel-indicators .active{width:12px;height:12px;margin:0;background-
color:#fff}.carousel-caption{position:absolute;right:15%;bottom:20px;left:15%;z-
index:10;padding-top:20px;padding-bottom:20px;color:#fff;text-align:center;text-
shadow:0 1px 2px rgba(0,0,0,.6)}.carousel-caption .btn{text-shadow:none}@media
screen and (min-width:768px){.carousel-control .glyphicon-chevron-left,.carousel-
control .glyphicon-chevron-right,.carousel-control .icon-prev,.carousel-control
.icon-next{width:30px;height:30px;margin-top:-15px;font-size:30px}.carousel-
control .glyphicon-chevron-left,.carousel-control .icon-prev{margin-left:-
15px}.carousel-control .glyphicon-chevron-right,.carousel-control .icon-
next{margin-right:-15px}.carousel-caption{right:20%;left:20%;padding-
bottom:30px}.carousel-indicators{bottom:20px}}.clearfix:before,.clearfix:after,.dl-
horizontal dd:before,.dl-horizontal dd:after,.container:before,.container:after,.container-
fluid:before,.container-fluid:after,.row:before,.row:after,.form-horizontal
.form-group:before,.form-horizontal .form-group:after,.btn-toolbar:before,.btn-
toolbar:after,.btn-group-vertical>.btn-group:before,.btn-group-vertical>.btn-
group:after,.nav:before,.nav:after,.navbar:before,.navbar:after,.navbar-header:before,.navbar-
header:after,.navbar-collapse:before,.navbar-collapse:after,.pager:before,.pager:after,.panel-
body:before,.panel-body:after,.modal-footer:before,.modal-footer:after{display:table;content:"
"}.clearfix:after,.dl-horizontal dd:after,.container:after,.container-fluid:after,.row:after,.form-
horizontal .form-group:after,.btn-toolbar:after,.btn-group-vertical>.btn-
group:after,.nav:after,.navbar:after,.navbar-header:after,.navbar-collapse:after,.pager:after,.panel-
body:after,.modal-footer:after{clear:both}.center-block{display:block;margin-
right:auto;margin-left:auto}.pull-right{float:right!important}.pull-left{float:left!important}.hide{display:none!i
hide{font:0/0 a;color:transparent;text-shadow:none;background-color:transparent;border:0}.hidden{display:no
webkit-transform:translate3d(0,0,0);-o-transform:translate3d(0,0,0);transform:translate3d(0,0,0)}@-
ms-viewport{width:device-width}.visible-xs,.visible-sm,.visible-md,.visible-
lg{display:none!important}.visible-xs-block,.visible-xs-inline,.visible-xs-inline-

block,.visible-sm-block,.visible-sm-inline,.visible-sm-inline-block,.visible-md-block,.visible-md-inline,.visible-md-inline-block,.visible-lg-block,.visible-lg-inline,.visible-lg-inline-block{display:none!important}@media (max-width:767px){.visible-xs{display:block!important}table.visible-xs{display:table}tr.visible-xs{display:table-row!important}th.visible-xs,td.visible-xs{display:table-cell!important}}@media (max-width:767px){.visible-xs-block{display:block!important}}@media (max-width:767px){.visible-xs-inline{display:inline!important}}@media (max-width:767px){.visible-xs-inline-block{display:inline-block!important}}@media (min-width:768px) and (max-width:991px){.visible-sm{display:block!important}table.visible-sm{display:table}tr.visible-sm{display:table-row!important}th.visible-sm,td.visible-sm{display:table-cell!important}}@media (min-width:768px) and (max-width:991px){.visible-sm-block{display:block!important}}@media (min-width:768px) and (max-width:991px){.visible-sm-inline{display:inline!important}}@media (min-width:768px) and (max-width:991px){.visible-sm-inline-block{display:inline-block!important}}@media (min-width:992px) and (max-width:1199px){.visible-md{display:block!important}table.visible-md{display:table}tr.visible-md{display:table-row!important}th.visible-md,td.visible-md{display:table-cell!important}}@media (min-width:992px) and (max-width:1199px){.visible-md-block{display:block!important}}@media (min-width:992px) and (max-width:1199px){.visible-md-inline{display:inline!important}}@media (min-width:992px) and (max-width:1199px){.visible-md-inline-block{display:inline-block!important}}@media (min-width:1200px){.visible-lg{display:block!important}table.visible-lg{display:table}tr.visible-lg{display:table-row!important}th.visible-lg,td.visible-lg{display:table-cell!important}}@media (min-width:1200px){.visible-lg-block{display:block!important}}@media (min-width:1200px){.visible-lg-inline{display:inline!important}}@media (min-width:1200px){.visible-lg-inline-block{display:inline-block!important}}@media (max-width:767px){.hidden-xs{display:none!important}}@media (min-width:768px) and (max-width:991px){.hidden-sm{display:none!important}}@media (min-width:992px) and (max-width:1199px){.hidden-md{display:none!important}}@media (min-width:1200px){.hidden-lg{display:none!important}}.visible-print{display:none!important}@media print{.visible-print{display:block!important}table.visible-print{display:table}tr.visible-print{display:table-row!important}th.visible-print,td.visible-print{display:table-cell!important}}.visible-print-block{display:none!important}@media print{.visible-print-block{display:block!important}}.visible-print-inline{display:none!important}@media print{.visible-print-inline{display:inline!important}}.visible-print-inline-block{display:none!important}@media print{.visible-print-inline-block{display:inline-block!important}}@media print{.hidden-print{display:none!important}}

===== .\ffmpeg\doc\community.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> Community </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Community </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Organisation 2 General Assembly 3 Voting 4 Technical Committee <ul class="toc-numbered-mark"> 4.1 Resolution Process <ul class="toc-numbered-mark"> 4.1.1 Seizing 4.1.2 Announcement 4.1.3 RFC call 4.1.4 Within TC 4.1.5 Decisions   5 Community Committee 6 Code of Conduct

 <h2 class="chapter">1 Organisation# TOC</h2>

The FFmpeg project is organized through a community working on global consensus. Decisions are taken by the ensemble of active members, through voting and are aided by two committees. <h2 class="chapter">2 General Assembly# TOC</h2>

The ensemble of active members is called the General Assembly (GA). The General Assembly is sovereign and legitimate for all its decisions regarding the FFmpeg project. The General Assembly is made up of active contributors. Contributors are considered "active contributors" if they have authored more than 20 patches in the last 36 months in the main FFmpeg repository, or if they have been voted in by the GA. The list of active contributors is updated twice each year, on 1st January and 1st July, 0:00 UTC. Additional members are added to the General Assembly through a vote after proposal by a member of the General Assembly. They are part of the GA for two years, after which they need a confirmation by the GA. A script to generate the current members of the general assembly (minus members voted in) can be

found in ‘tools/general_assembly.pl‘. <h2 class="chapter">3 Voting# TOC</h2>

Voting is done using a ranked voting system, currently running on https://vote.ffmpeg.org/ . Majority vote means more than 50% of the expressed ballots. <h2 class="chapter">4 Technical Committee# TOC</h2>

The Technical Committee (TC) is here to arbitrate and make decisions when technical conflicts occur in the project. They will consider the merits of all the positions, judge them and make a decision. The TC resolves technical conflicts but is not a technical steering committee. Decisions by the TC are binding for all the contributors. Decisions made by the TC can be re-opened after 1 year or by a majority vote of the General Assembly, requested by one of the member of the GA. The TC is elected by the General Assembly for a duration of 1 year, and is composed of 5 members. Members can be re-elected if they wish. A majority vote in the General Assembly can trigger a new election of the TC. The members of the TC can be elected from outside of the GA. Candidates for election can either be suggested or self-nominated. The conflict resolution process is detailed in the resolution process document. The TC can be contacted at <tc@ffmpeg>. <h3 class="section">4.1 Resolution Process# TOC</h3>

The Technical Committee (TC) is here to arbitrate and make decisions when technical conflicts occur in the project. The TC main role is to resolve technical conflicts. It is therefore not a technical steering committee, but it is understood that some decisions might impact the future of the project. <h4 class="subsection">4.1.1 Seizing# TOC</h4>

The TC can take possession of any technical matter that it sees fit. To involve the TC in a matter, email tc or CC them on an ongoing discussion. As members of TC are developers, they also can email tc to raise an issue. <h4 class="subsection">4.1.2 Announcement<a

class=”anchor hidden-xs” href=”#Announcement” aria-hidden=”true”>#
TOC</h4>

The TC, once seized, must announce itself on the main mailing list, with a [TC] tag. The TC has 2 modes of operation: a RFC one and an internal one. If the TC thinks it needs the input from the larger community, the TC can call for a RFC. Else, it can decide by itself. The decision to use a RFC process or an internal discussion is a discretionary decision of the TC. The TC can also reject a seizure for a few reasons such as: the matter was not discussed enough previously; it lacks expertise to reach a beneficial decision on the matter; or the matter is too trivial. <h4 class=”subsection”>4.1.3 RFC call# TOC</h4>

In the RFC mode, one person from the TC posts on the mailing list the technical question and will request input from the community. The mail will have the following specification: a precise title a specific tag [TC RFC] a top-level email contain a precise question that does not exceed 100 words and that is answerable by developers may have an extra description, or a link to a previous discussion, if deemed necessary, contain a precise end date for the answers. The answers from the community must be on the main mailing list and must have the following specification: keep the tag and the title unchanged limited to 400 words a first-level, answering directly to the main email answering to the question. Further replies to answers are permitted, as long as they conform to the community standards of politeness, they are limited to 100 words, and are not nested more than once. (max-depth=2) After the end-date, mails on the thread will be ignored. Violations of those rules will be escalated through the Community Committee. After all the emails are in, the TC has 96 hours to give its final decision. Exceptionally, the TC can request an extra delay, that will be notified on the mailing list. <h4 class=”subsection”>4.1.4 Within TC# TOC</h4>

In the internal case, the TC has 96 hours to give its final decision. Exceptionally, the TC can request an extra delay. <h4 class=”subsection”>4.1.5 Decisions# TOC</h4>

The decisions from the TC will be sent on the mailing list, with the [TC] tag. Internally, the TC should take decisions with a majority, or using ranked-choice voting. Each TC member must

vote on such decision according to what is, in their view, best for the project. If a TC member feels they are affected by a conflict of interest with regards to the case, they should announce it and recuse themselves from the TC discussion and vote. A conflict of interest is presumed to occur when a TC member has a personal interest (e.g. financial) in a specific outcome of the case. The decision from the TC should be published with a summary of the reasons that lead to this decision. The decisions from the TC are final, until the matters are reopened after no less than one year. <h2 class="chapter">5 Community Committee# TOC</h2>

The Community Committee (CC) is here to arbitrage and make decisions when inter-personal conflicts occur in the project. It will decide quickly and take actions, for the sake of the project. The CC can remove privileges of offending members, including removal of commit access and temporary ban from the community. Decisions made by the CC can be re-opened after 1 year or by a majority vote of the General Assembly. Indefinite bans from the community must be confirmed by the General Assembly, in a majority vote. The CC is elected by the General Assembly for a duration of 1 year, and is composed of 5 members. Members can be re-elected if they wish. A majority vote in the General Assembly can trigger a new election of the CC. The members of the CC can be elected from outside of the GA. Candidates for election can either be suggested or self-nominated. The CC is governed by and responsible for enforcing the Code of Conduct. The CC can be contacted at <cc@ffmpeg>. <h2 class="chapter">6 Code of Conduct# TOC</h2>

Be friendly and respectful towards others and third parties. Treat others the way you yourself want to be treated. Be considerate. Not everyone shares the same viewpoint and priorities as you do. Different opinions and interpretations help the project. Looking at issues from a different perspective assists development. Do not assume malice for things that can be attributed to incompetence. Even if it is malice, it’s rarely good to start with that as initial assumption. Stay friendly even if someone acts contrarily. Everyone has a bad day once in a while. If you yourself have a bad day or are angry then try to take a break and reply once you are calm and without anger if you have to. Try to help other team members and cooperate if you can. The goal of software development is to create technical excellence, not for any individual to be

better and "win" against the others. Large software projects are only possible and successful through teamwork. If someone struggles do not put them down. Give them a helping hand instead and point them in the right direction. Finally, keep in mind the immortal words of Bill and Ted, "Be excellent to each other." <p style=”font-size: small;”> This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\default.css ===== a.summary-letter { text-decoration: none; }

a { color: #2D6198; }

a:visited { color: #884488; }

#banner { background-color: white; position: relative; text-align: center; }

#banner img { margin-bottom: 1px; margin-top: 5px; }

#body { margin-left: 1em; margin-right: 1em; }

body { background-color: #313131; margin: 0; text-align: justify; }

.center { margin-left: auto; margin-right: auto; text-align: center; }

#container { background-color: white; color: #202020; margin-left: 1em; margin-right: 1em; }

#footer { text-align: center; }

h1 a, h2 a, h3 a, h4 a { text-decoration: inherit; color: inherit; }

h1, h2, h3, h4 { padding-left: 0.4em; border-radius: 4px; padding-bottom: 0.25em; padding-top: 0.25em; border: 1px solid #6A996A; }

h1 { background-color: #7BB37B; color: #151515; font-size: 1.2em; padding-bottom: 0.3em; padding-top: 0.3em; }

h2 { color: #313131; font-size: 1.0em; background-color: #ABE3AB; }

h3 { color: #313131; font-size: 0.9em; margin-bottom: -6px; background-color: #BBF3BB; }

h4 { color: #313131; font-size: 0.8em; margin-bottom: -8px; background-color: #D1FDD1; }

img { border: 0; }

#navbar { background-color: #738073; border-bottom: 1px solid #5C665C; border-top: 1px solid #5C665C; margin-top: 12px; padding: 0.3em; position: relative; text-align: center; }

#navbar a, #navbar_secondary a { color: white; padding: 0.3em; text-decoration: none; }

#navbar a:hover, #navbar_secondary a:hover { background-color: #313131; color: white; text-decoration: none; }

#navbar_secondary { background-color: #738073; border-bottom: 1px solid #5C665C; border-left: 1px solid #5C665C; border-right: 1px solid #5C665C; padding: 0.3em; position: relative; text-align: center; }

p { margin-left: 1em; margin-right: 1em; }

pre { margin-left: 3em; margin-right: 3em; padding: 0.3em; border: 1px solid #bbb; background-color: #f7f7f7; }

dl dt { font-weight: bold; }

#proj_desc { font-size: 1.2em; }

#repos { margin-left: 1em; margin-right: 1em; border-collapse: collapse; border: solid 1px #6A996A; }

#repos th { background-color: #7BB37B; border: solid 1px #6A996A; }

#repos td { padding: 0.2em; border: solid 1px #6A996A; }

===== .\ffmpeg\doc\developer.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> Developer Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Developer Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Introduction 2 Coding Rules <ul class="toc-numbered-mark"> 2.1 Language <ul class="toc-numbered-mark"> 2.1.1 SIMD/DSP 2.1.2 Other languages 2.2 Code formatting conventions <ul class="toc-numbered-mark"> 2.2.1 Examples 2.2.2 Vim configuration 2.2.3 Emacs configuration <a

id=”toc-Comments” href=”#Comments”>2.3 Comments 2.4 Naming conventions 2.5 Miscellaneous conventions 3 Development Policy <ul class=”toc-numbered-mark”> 3.1 Code behaviour 3.2 Patches/Committing 3.3 Code 3.4 Library public interfaces <ul class=”toc-numbered-mark”> 3.4.1 Adding new interfaces 3.4.2 Removing interfaces 3.4.3 Major version bumps 3.5 Documentation/Other 4 Submitting patches 5 New codecs or formats checklist 6 Patch submission checklist 7 Patch review process 8 Regression tests <ul class=”toc-numbered-mark”> 8.1 Adding files to the fate-suite dataset 8.2 Visualizing Test Coverage 8.3 Using Valgrind 9 Maintenance process <ul class=”toc-numbered-mark”> 9.1 MAINTAINERS 9.2 Becoming a maintainer 10 Release process <ul class=”toc-numbered-mark”> 10.1 Criteria for Point Releases 10.2 Release Checklist

 <h2 class=”chapter”>1 Introduction# <a class=”anchor hidden-xs”href=”#toc-

Introduction" aria-hidden="true">TOC</h2>

This text is concerned with the development of FFmpeg itself. Information on using the FFmpeg libraries in other programs can be found elsewhere, e.g. in: <ul class="itemize mark-bullet"> the installed header files the Doxygen documentation generated from the headers the examples under <samp class="file">doc/examples</samp>

For more detailed legal information about the use of FFmpeg in external programs read the <samp class="file">LICENSE</samp> file in the source tree and consult https://ffmpeg.org/legal.html. If you modify FFmpeg code for your own use case, you are highly encouraged to submit your changes back to us, using this document as a guide. There are both pragmatic and ideological reasons to do so: <ul class="itemize mark-bullet"> Maintaining external changes to keep up with upstream development is time-consuming and error-prone. With your code in the main tree, it will be maintained by FFmpeg developers. FFmpeg developers include leading experts in the field who can find bugs or design flaws in your code. By supporting the project you find useful you ensure it continues to be maintained and developed.

All proposed code changes should be submitted for review to Forgejo or the development mailing list, as described in more detail in the Submitting patches chapter. The code should comply with the Development Policy and follow the Coding Rules. The developer making the commit and the author are responsible for their changes and should try to fix issues their commit causes. <h2 class="chapter">2 Coding Rules# TOC</h2>

 <h3 class="section">2.1 Language# TOC</h3>

FFmpeg is mainly programmed in the ISO C11 language, except for the public headers which must stay C99 compatible. Compiler-specific extensions may be used with good reason, but must not be depended on, i.e. the code must still compile and work with compilers lacking the extension.

The following C99 features must not be used anywhere in the codebase: <ul class="itemize mark-bullet"> variable-length arrays;

complex numbers;

 <h4 class="subsection">2.1.1 SIMD/DSP# TOC</h4> As modern compilers are unable to generate efficient SIMD or other performance-critical DSP code from plain C, handwritten assembly is used. Usually such code is isolated in a separate function. Then the standard approach is writing multiple versions of this function – a plain C one that works everywhere and may also be useful for debugging, and potentially multiple architecture-specific optimized implementations. Initialization code then chooses the best available version at runtime and loads it into a function pointer; the function in question is then always called through this pointer. The specific syntax used for writing assembly is: <ul class="itemize mark-bullet"> NASM on x86;

GAS on ARM and RISC-V.

A unit testing framework for assembly called <code class="code">checkasm</code> lives under <samp class="file">tests/checkasm</samp>. All new assembly should come with <code class="code">checkasm</code> tests; adding tests for existing assembly that lacks them is also strongly encouraged. <h4 class="subsection">2.1.2 Other languages# TOC</h4>

Other languages than C may be used in special cases: <ul class="itemize mark-bullet"> Compiler intrinsics or inline assembly when the code in question cannot be written in the standard way described in the SIMD/DSP section. This typically applies to code that needs to be inlined.

Objective-C where required for interacting with macOS-specific interfaces.

 <h3 class="section">2.2 Code formatting conventions# TOC</h3>

There are the following guidelines regarding the code style in files: <ul class="itemize mark-bullet"> Indent size is 4.

The TAB character is forbidden outside of Makefiles as is any form of trailing whitespace. Commits containing either will be rejected by the git repository.

You should try to limit your code lines to 80 characters; however, do so if and only if this improves readability.

K&R coding style is used. The presentation is one inspired by ’indent -i4 -kr -nut’. <h4 class="subsection">2.2.1 Examples# TOC</h4> Some notable examples to illustrate common code style in FFmpeg: <ul class="itemize mark-bullet"> Space around assignments and after <code class="code">if</code>/<code class="code">do</code>/<code class="code">while</code>/<code class="code">for</code> keywords:

 <pre class="example-preformatted">// Good if (condition) av_foo(); </pre>

 <pre class="example-preformatted">// Good for (size_t i = 0; i < len; i++) av_bar(i); </pre>

 <pre class="example-preformatted">// Good size_t size = 0; </pre>

However no spaces between the parentheses and condition, unless it helps readability of complex conditions, so the following should not be done: <pre class="example-preformatted">// Bad style if (condition) av_foo(); </pre>

No unnecessary parentheses, unless it helps readability:

 <pre class="example-preformatted">// Good int fields = ilace ? 2 : 1; </pre>

Don’t wrap single-line blocks in braces. Use braces only if there is an accompanying else statement. This keeps future code changes easier to keep track of.

 <pre class="example-preformatted">// Good if (bits_pixel == 24) { avctx->pix_fmt = AV_PIX_FMT_BGR24; } else if (bits_pixel == 8) { avctx->pix_fmt = AV_PIX_FMT_GRAY8; } else return AVERROR_INVALIDDATA;

</pre>

Avoid assignments in conditions where it makes sense:

 <pre class="example-preformatted">// Good video_enc->chroma_intra_matrix = av_mallocz(sizeof(*video_enc-

>chroma_intra_matrix) * 64) if (!video_enc->chroma_intra_matrix) return AVERROR(ENOMEM); </pre>

 <pre class="example-preformatted">// Bad style if (!(video_enc->chroma_intra_matrix = av_mallocz(sizeof(*video_enc->chroma_intra_matrix) * 64))) return AVERROR(ENOMEM); </pre>

 <pre class="example-preformatted">// Ok while ((entry = av_dict_iterate(options, entry))) av_log(ctx, AV_LOG_INFO, "Item '%s': '%s'\n", entry->key, entry->value); </pre>

When declaring a pointer variable, the <code class="code">*</code> goes with the variable not the type:

 <pre class="example-preformatted">// Good AVStream *stream; </pre>

 <pre class="example-preformatted">// Bad style AVStream* stream; </pre>

When sensible, prefer a narrow variable scope, especially in for loops:

 <pre class="example-preformatted">// Good for (unsigned i = 0; i < submix->nb_elements; i++) { // Do something... } </pre>

 <pre class="example-preformatted">// Bad style unsigned i; //... for (i = 0; i < submix->nb_elements; i++) { // Do something... } </pre>

If you work on a file that does not follow these guidelines consistently, change the parts that you are editing to follow these guidelines but do not make unrelated changes in the file to make it conform to these. <h4 class="subsection">2.2.2 Vim configuration# TOC</h4> In order to configure Vim to follow FFmpeg formatting conventions, paste the following snippet into your <samp class="file">.vimrc</samp>: <pre class="example-preformatted">" indentation rules for FFmpeg: 4 spaces, no tabs set expandtab set shiftwidth=4 set softtabstop=4 set cindent set cinoptions=(0 " Allow tabs in Makefiles. autocmd FileType make,automake set noexpandtab shiftwidth=8 softtabstop=8 " Trailing whitespace and tabs are forbidden, so highlight them. highlight ForbiddenWhitespace ctermbg=red guibg=red match ForbiddenWhitespace /\s\+$\|\t/ " Do not highlight spaces at the end of line while typing on that line. autocmd InsertEnter * match ForbiddenWhitespace /\t\|\s\+\%#\@<!$/ </pre>

 <h4 class="subsection">2.2.3 Emacs configuration# TOC</h4> For Emacs, add these roughly equivalent lines to your <samp class="file">.emacs.d/init.el</samp>: <pre class="lisp-preformatted">(c-add-style "ffmpeg" '("k&r" (c-basic-offset . 4) (indent-tabs-mode . nil) (show-trailing-whitespace . t) (c-offsets-alist (statement-cont . (c-lineup-assignments +))))) (setq c-default-style "ffmpeg") </pre>

 <h3 class="section">2.3 Comments# TOC</h3> Use the JavaDoc/Doxygen format (see examples below) so that code documentation can be generated automatically. All nontrivial functions should have a comment above them explaining what the function does, even if it is just one sentence. All structures and their member variables should be documented, too. Avoid Qt-style and similar Doxygen syntax with <code class="code">!</code> in it, i.e. replace <code class="code">//!</code> with <code class="code">///</code> and similar. Also @ syntax should be employed for markup commands, i.e. use <code class="code">@param</code> and not <code class="code">\param</code>.   <pre class="example-preformatted">/** * @file * MPEG codec. * @author ... */

/** * Summary sentence. * more text ... * ... */ typedef struct Foobar { int var1; /**< var1 description */ int var2; ///< var2 description /** var3 description */ int var3; } Foobar;

/** * Summary sentence. * more text ... * ... * @param my_parameter description of my_parameter * @return return value description */ int myfunc(int my_parameter) ... </pre>

 <h3 class="section">2.4 Naming conventions# TOC</h3>

Names of functions, variables, and struct members must be lowercase, using underscores (_) to separate words. For example, ‘<samp class="samp">avfilter_get_video_buffer</samp>’ is an acceptable function name and ‘<samp class="samp">AVFilterGetVideo</samp>’ is not. Struct, union, enum, and typedeffed type names must use CamelCase. All structs and unions should be typedeffed to the same name as the struct/union tag, e.g. <code class="code">typedef struct AVFoo { ... }

AVFoo;</code>. Enums are typically not typedeffed. Enumeration constants and macros must be UPPERCASE, except for macros masquerading as functions, which should use the function naming convention. All identifiers in the libraries should be namespaced as follows: <ul class="itemize mark-bullet"> No namespacing for identifiers with file and lower scope (e.g. local variables, static functions), and struct and union members,

The <code class="code">ff_</code> prefix must be used for variables and functions visible outside of file scope, but only used internally within a single library, e.g. ‘<samp class="samp">ff_w64_demuxer</samp>’. This prevents name collisions when FFmpeg is statically linked.

For variables and functions visible outside of file scope, used internally across multiple libraries, use <code class="code">avpriv_</code> as prefix, for example, ‘<samp class="samp">avpriv_report_missing_feature</samp>’.

All other internal identifiers, like private type or macro names, should be namespaced only to avoid possible internal conflicts. E.g. <code class="code">H264_NAL_SPS</code> vs. <code class="code">HEVC_NAL_SPS</code>.

Each library has its own prefix for public symbols, in addition to the commonly used <code class="code">av_</code> (<code class="code">avformat_</code> for libavformat, <code class="code">avcodec_</code> for libavcodec, <code class="code">swr_</code> for libswresample, etc). Check the existing code and choose names accordingly.

Other public identifiers (struct, union, enum, macro, type names) must use their library’s public prefix (<code class="code">AV</code>, <code class="code">Sws</code>, or <code class="code">Swr</code>).

Furthermore, name space reserved for the system should not be invaded. Identifiers ending in <code class="code">_t</code> are reserved by <a class="url" href="http://pubs.opengroup.org/onlinepubs/007904975/functions/xsh_chap02_02.html#tag_02
Also avoid names starting with <code class="code">___</code> or <code class="code">_</code> followed by an uppercase letter as they are reserved by the C standard. Names starting with <code class="code">_</code> are reserved at the file level and may not be used for externally visible symbols. If in doubt, just avoid names starting with <code class="code">_</code> altogether. <h3 class="section">2.5 Miscellaneous conventions# TOC</h3>

<ul class="itemize mark-bullet"> Casts should be used only when necessary. Unneeded parentheses should also be avoided if they don’t

make the code easier to understand. Where applicable, SI units shall be used. For example timeouts should use seconds as the fundamental unit not microseconds. That means a bare value like ‘<samp class="samp">1.0</samp>’ must mean 1 second, ‘<samp class="samp">50m</samp>’ means 50 milliseconds. For weight, gram shall be used.

 <h2 class="chapter">3 Development Policy# TOC</h2>

 <h3 class="section">3.1 Code behaviour# TOC</h3>

 The code must be valid. It must not crash, abort, access invalid pointers, leak memory, cause data races or signed integer overflow, or otherwise cause undefined behaviour. Error codes should be checked and, when applicable, forwarded to the caller. Our libraries may be called by multiple independent callers in the same process. These calls may happen from any number of threads and the different call sites may not be aware of each other - e.g. a user program may be calling our libraries directly, and use one or more libraries that also call our libraries. The code must behave correctly under such conditions. The code must treat as untrusted any bytestream received from a caller or read from a file, network, etc. Arbitrary input must never cause unsafe behavior or a security issue. Common real-world errors of common formats and codecs should be detected where practical, reported where appropriate, and handled to produce the best output possible, for example through error concealment. All errors must still be handled safely, but if they are detected, reporting or how they are handled is left to the implementation. The code must use the <code class="code">av_malloc()</code> family of functions from <samp class="file">libavutil/mem.h</samp> to perform all memory allocation, except in special cases (e.g. when interacting with an external library that requires a specific allocator to be used). All allocations should be checked and <code class="code">AVERROR(ENOMEM)</code> returned on failure. A common mistake is that error paths leak memory - make sure that does not happen. Our libraries must not access the stdio streams stdin/stdout/stderr directly (e.g. via <code class="code">printf()</code> family of functions), as that is not library-safe. For logging, use <code class="code">av_log()</code>. <h3 class="section">3.2

Patches/Committing# TOC</h3> Contributions should be licensed under the LGPL 2.1, including an "or any later version" clause, or, if you prefer a gift-style license, the ISC or MIT license. GPL 2 including an "or any later version" clause is also acceptable, but LGPL is preferred. If you add a new file, give it a proper license header. Do not copy and paste it from a random place, use an existing file as template. This means unfinished code which is enabled and breaks compilation, or compiles but does not work/breaks the regression tests. Code which is unfinished but disabled may be permitted under-circumstances, like missing samples or an implementation with a small subset of features. Always check the mailing list for any reviewers with issues and test FATE before you push. Commit messages are highly important tools for informing other developers on what a given change does and why. Every commit must always have a properly filled out commit message with the following format: <pre class="example-preformatted">area changed: short 1 line description

details describing what and why and giving references. </pre>

If the commit addresses a known bug on our bug tracker or other external issue

(e.g. CVE), the commit message should include the relevant bug ID(s) or other

external identifiers. Note that this should be done in addition to a proper

explanation and not instead of it. Comments such as "fixed!" or "Changed it."

are not acceptable.

When applying patches that have been discussed at length on the mailing list,

reference the thread in the commit message.

If it works for you, others, and passes FATE then it should be OK to commit

it, provided it fits the other committing criteria. You should not worry about over-testing things. If your code has problems (portability, triggers compiler bugs, unusual environment etc) they will be reported and eventually fixed.

They should be split them into self-contained pieces. Also do not forget that if part B depends on part A, but A does not depend on B, then A can and should be committed first and separate from B. Keeping changes well split into self-contained parts makes reviewing and understanding them on the commit log mailing list easier. This also helps in case of debugging later on.

Also if you have doubts about splitting or not splitting, do not hesitate to ask/discuss it on the developer mailing list.

Keep bug fixes as small as possible while fully fixing the reported issue. Changes not needed for correctness or robustness should be submitted separately,

especially if they increase complexity or affect backporting.

We refuse source indentation and other cosmetic changes if they are mixed with functional changes, such commits will be rejected and removed. However, indentation changes that can be ignored by <code class="code">git diff --ignore-all-space</code>

(e.g. changes in whitespace amount, leading/trailing spaces) may be mixed with functional changes, since reviewers can use <code class="code">git diff -w</code> or

<code class="code">git log -p --ignore-all-space</code> to review only the functional parts of

the change. Forgejo’s pull request interface also provides a

“Hide whitespace changes” option for this purpose. Every
developer has his own indentation style, you should not change it. Of course
if you (re)write something, you can use your own style, even though we would
prefer if the indentation throughout FFmpeg was consistent (Many projects
force a given indentation style - we do not.). If you really need to make
non-whitespace cosmetic changes, separate them strictly from real changes.

NOTE: If you had to put if(){ .. } over a large (> 5 lines) chunk of code,
then either do NOT change the indentation of the inner part within (do not
move it to the right)! or do so in a separate commit

Make sure the author of the commit is set correctly. (see git commit –author)
If you apply a patch, send an
answer to ffmpeg-devel (or wherever you got the patch from) saying that
you applied the patch.

If a commit/patch fixes an issues found by some researcher, always credit the
researcher in the commit message for finding/reporting the issue.

Do NOT commit to code actively maintained by others without permission.
Send a patch to ffmpeg-devel. If no one answers within a reasonable
time-frame (12h for build failures and security fixes, 3 days small changes,
1 week for big patches) then commit your patch if you think it is OK.
Also note, the maintainer can simply ask for more time to review!

<h3 class=”section”>3.3 Code# TOC</h3>

Compiler warnings indicate potential bugs or code with bad style. If a type of

warning always points to correct and clean code, that warning should

be disabled, not the code changed.

Thus the remaining warnings can either be bugs or correct code.

If it is a bug, the bug has to be fixed. If it is not, the code should

be changed to not generate a warning unless that causes a slowdown

or obfuscates the code.

<h3 class=”section”>3.4 Library public interfaces# TOC</h3>

Every library in FFmpeg provides a set of public APIs in its installed headers,

which are those listed in the variable <code class=”code”>HEADERS</code> in that library’s

<samp class=”file”>Makefile</samp>. All identifiers defined in those headers (except for those

explicitly documented otherwise), and corresponding symbols exported from

compiled shared or static libraries are considered public interfaces and must

comply with the API and ABI compatibility rules described in this section.

Public APIs must be backward compatible within a given major version. I.e. any

valid user code that compiles and works with a given library version must still

compile and work with any later version, as long as the major version number is

unchanged. "Valid user code" here means code that is calling our APIs in a

documented and/or intended manner and is not relying on any undefined behavior.

Incrementing the major version may break backward compatibility, but only to the

extent described in Major version bumps.

We also guarantee backward ABI compatibility for shared and static libraries.

I.e. it should be possible to replace a shared or static build of our library

with a build of any later version (re-linking the user binary in the static

case) without breaking any valid user binaries, as long as the major version

number remains unchanged.

<h4 class=”subsection”>3.4.1 Adding new interfaces# TOC</h4>

Any new public identifiers in installed headers are considered new API - this

includes new functions, structs, macros, enum values, typedefs, new fields in

existing structs, new installed headers, etc. Consider the following

guidelines when adding new APIs.

While new APIs can be added relatively easily, changing or removing them is much

harder due to abovementioned compatibility requirements. You should then

consider carefully whether the functionality you are adding really needs to be

exposed to our callers as new public API.

Your new API should have at least one well-established use case outside of the

library that cannot be easily achieved with existing APIs. Every library in

FFmpeg also has a defined scope - your new API must fit within it.

If your new API is replacing an existing one, it should be strictly superior to

it, so that the advantages of using the new API outweigh the cost to the

callers of changing their code. After adding the new API you should then

deprecate the old one and schedule it for removal, as described in

Removing interfaces.

If you deem an existing API deficient and want to fix it, the preferred approach

in most cases is to add a differently-named replacement and deprecate the

existing API rather than modify it. It is important to make the changes visible

to our callers (e.g. through compile- or run-time deprecation warnings) and make

it clear how to transition to the new API (e.g. in the Doxygen documentation or

on the wiki).

The FFmpeg libraries are used by a variety of callers to perform a wide range of

multimedia-related processing tasks. You should therefore - within reason - try

to design your new API for the broadest feasible set of use cases and avoid

unnecessarily limiting it to a specific type of callers (e.g. just media

playback or just transcoding).

Check whether similar APIs already exist in FFmpeg. If they do, try to model

your new addition on them to achieve better overall consistency.

The naming of your new identifiers should follow the Naming conventions

and be aligned with other similar APIs, if applicable.

You should also consider how your API might be extended in the future in a

backward-compatible way. If you are adding a new struct <code class="code">AVFoo</code>, the

standard approach is requiring the caller to always allocate it through a

constructor function, typically named <code class="code">av_foo_alloc()</code>. This way new fields

may be added to the end of the struct without breaking ABI compatibility.

Typically you will also want a destructor - <code class="code">av_foo_free(AVFoo**)</code> that

frees the indirectly supplied object (and its contents, if applicable) and

writes <code class="code">NULL</code> to the supplied pointer, thus eliminating the potential

dangling pointer in the caller’s memory.

If you are adding new functions, consider whether it might be desirable to tweak

their behavior in the future - you may want to add a flags argument, even though

it would be unused initially.

All new APIs must be documented as Doxygen-formatted comments above the

identifiers you add to the public headers. You should also briefly mention the

change in <samp class="file">doc/APIchanges</samp>.

Backward-incompatible API or ABI changes require incrementing (bumping) the

major version number, as described in Major version bumps. Major

bumps are significant events that happen on a schedule - so if your change

strictly requires one you should add it under <code class="code">#if</code> preprocessor guards that

disable it until the next major bump happens.

New APIs that can be added without breaking API or ABI compatibility require

bumping the minor version number.

Incrementing the third (micro) version component means a noteworthy binary

compatible change (e.g. encoder bug fix that matters for the decoder). The third

component always starts at 100 to distinguish FFmpeg from Libav.

<h4 class="subsection">3.4.2 Removing interfaces# TOC</h4>

Due to abovementioned compatibility guarantees, removing APIs is an involved

process that should only be undertaken with good reason. Typically a deficient,

restrictive, or otherwise inadequate API is replaced by a superior one, though

it does at times happen that we remove an API without any replacement (e.g. when

the feature it provides is deemed not worth the maintenance effort, out of scope

of the project, fundamentally flawed, etc.).

The removal has two steps - first the API is deprecated and scheduled for

removal, but remains present and functional. The second step is actually

removing the API - this is described in Major version bumps.

To deprecate an API you should signal to our users that they should stop using

it. E.g. if you intend to remove struct members or functions, you should mark

them with <code class="code">attribute_deprecated</code>. When this cannot be done, it may be

possible to detect the use of the deprecated API at runtime and print a warning

(though take care not to print it too often). You should also document the

deprecation (and the replacement, if applicable) in the relevant Doxygen

documentation block.

Finally, you should define a deprecation guard along the lines of

<code class="code">#define FF_API_<FOO> (LIBAVBAR_VERSION_MAJOR < XX)</code> (where XX is the major

version in which the API will be removed) in <samp class="file">libavbar/version_major.h</samp>

(<samp class="file">version.h</samp> in case of <code class="code">libavutil</code>). Then wrap all uses of the

deprecated API in <code class="code">#if FF_API_<FOO> .... #endif</code>, so that the code will

automatically get disabled once the major version reaches XX. You can also use

<code class="code">FF_DISABLE_DEPRECATION_WARNINGS</code> and <code class="code">FF_ENABLE_DEPRECATION_WARNINGS</code>

to suppress compiler deprecation warnings inside these guards. You should test

that the code compiles and works with the guard macro evaluating to both true

and false.

<h4 class="subsection">3.4.3 Major version bumps# TOC</h4>

A major version bump signifies an API and/or ABI compatibility break. To reduce

the negative effects on our callers, who are required to adapt their code,

backward-incompatible changes during a major bump should be limited to:

<ul class="itemize mark-bullet">

Removing previously deprecated APIs.

Performing ABI- but not API-breaking changes, like reordering struct contents.

 <h3 class="section">3.5 Documentation/Other# TOC</h3> It is important to be subscribed to the ffmpeg-devel mailing list. Almost any non-trivial patch is to be sent there for review. Other developers may have comments about your contribution. We expect you see those comments, and to improve it if requested. (N.B. Experienced committers have other channels, and may sometimes skip review for trivial fixes.) Also, discussion here about bug fixes and FFmpeg improvements by other developers may be helpful information for you. Finally, by being a list subscriber, your contribution will be posted immediately to the list, without the moderation hold which messages from non-subscribers experience. However, it is more important to the project that we receive your patch than that you be subscribed to the ffmpeg-devel list. If you have a patch, and don’t want to subscribe and discuss the patch, then please do send it to the list anyway. Diffs of all commits are sent to the ffmpeg-cvslog mailing list. Some developers read this list to review all code base changes from all sources. Subscribing to this list is not mandatory. Update the documentation if you change behavior or add features. If you are unsure how best to do this, send a patch to ffmpeg-devel, the documentation maintainer(s) will review and commit your stuff. Try to keep important discussions and requests (also) on the public developer mailing list, so that all developers can benefit from

them. Make sure that no parts of the codebase that you maintain are missing from the <samp class="file">MAINTAINERS</samp> file. If something that you want to maintain is missing add it with your name after it. If at some point you no longer want to maintain some code, then please help in finding a new maintainer and also don’t forget to update the <samp class="file">MAINTAINERS</samp> file. We think our rules are not too hard. If you have comments, contact us. <h2 class="chapter">4 Submitting patches# TOC</h2>

First, read the Coding Rules above if you did not yet, in particular the rules regarding patch submission. When you submit your patch, please use <code class="code">git format-patch</code> or <code class="code">git send-email</code>. We cannot read other diffs :-). Also please do not submit a patch which contains several unrelated changes. Split it into separate, self-contained pieces. This does not mean splitting file by file. Instead, make the patch as small as possible while still keeping it as a logical unit that contains an individual change, even if it spans multiple files. This makes reviewing your patches much easier for us and greatly increases your chances of getting your patch applied. Use the patcheck tool of FFmpeg to check your patch. The tool is located in the tools directory. Run the Regression tests before submitting a patch in order to verify it does not cause unexpected problems. It also helps quite a bit if you tell us what the patch does (for example ’replaces lrint by lrintf’), and why (for example ’*BSD isn’t C99 compliant and has no lrint()’) Also please if you send several patches, send each patch as a separate mail, do not attach several unrelated patches to the same mail. Patches should be posted to Forgejo or the ffmpeg-devel mailing list. Use <code class="code">git send-email</code> when possible since it will properly send patches without requiring extra care. If you cannot, then send patches as base64-encoded attachments, so your patch is not trashed during transmission. Also ensure the correct mime type is used (text/x-diff or text/x-patch or at least text/plain) and that only one patch is inline or attached per mail. You can check https://patchwork.ffmpeg.org, if your patch does not show up, its mime type likely was wrong.

Please see https://git-send-email.io/. For gmail additionally see <a class="url" href="https://shallowsky.com/blog/tech/email/

app-passwds.html”>https://shallowsky.com/blog/tech/email/gmail-app-passwds.html. Using <code class=”code”>git send-email</code> might not be desirable for everyone. The following trick allows to send patches via email clients in a safe way. It has been tested with Outlook and Thunderbird (with X-Unsent extension) and might work with other applications. Create your patch like this: <pre class=”verbatim”>git format-patch -s -o "outputfolder" --add-header "X-Unsent: 1" --suffix .eml --to ffmpeg-devel@ffmpeg.org -1 1a2b3c4d </pre> Now you’ll just need to open the eml file with the email application and execute ’Send’. Your patch or pull request will be reviewed on the mailing list or on code.ffmpeg.org. You will likely be asked to make some changes and are expected to send in an improved version that incorporates the requests from the review. This process may go through several iterations. Once your patch is deemed good enough, some developer will pick it up and commit it to the official FFmpeg tree. If you want an LLM-based review, add Forgejo Fairy as a reviewer to your pull request. Issues with Fairy herself can be reported at her repository. Give us a few days to react. But if some time passes without reaction, send a reminder by email. Your patch should eventually be dealt with.

 <h2 class=”chapter”>5 New codecs or formats checklist# TOC</h2>

<ol class=”enumerate”> Did you use av_cold for codec initialization and close functions?

 Did you add a long_name under NULL_IF_CONFIG_SMALL to the AVCodec or AVInputFormat/AVOutputFormat struct?

 Did you bump the minor version number (and reset the micro version number) in <samp class=”file”>libavcodec/version.h</samp> or <samp class=”file”>libavformat/version.h</samp>?

 Did you register it in <samp class=”file”>allcodecs.c</samp> or <samp class=”file”>allformats.c</samp>?

 Did you add the AVCodecID to <samp class=”file”>codec_id.h</samp>? When adding new codec IDs, also add an entry to the codec descriptor list in <samp class=”file”>libavcodec/codec_desc.c</samp>.

 If it has a FourCC, did you add it to <samp class=”file”>libavformat/riff.c</samp>, even if it is only a decoder?

 Did you add a rule to compile the appropriate files in the Makefile? Remember to do this even if you’re just adding a format to a file that is already being compiled by some other rule, like a raw demuxer.

 Did you add an entry to the table of supported formats or codecs in <samp class="file">doc/general_contents.texi</samp>?

 Did you add an entry in the Changelog?

 If it depends on a parser or a library, did you add that dependency in configure?

 Did you <code class="code">git add</code> the appropriate files before committing?

 Did you make sure it compiles standalone, i.e. with <code class="code">configure --disable-everything --enable-decoder=foo</code> (or <code class="code">--enable-demuxer</code> or whatever your component is)?

 <h2 class="chapter">6 Patch submission checklist# TOC</h2>

<ol class="enumerate"> Does <code class="code">make fate</code> pass with the patch applied?

 Was the patch generated with git format-patch or send-email?

 Did you sign-off your patch? (<code class="code">git commit -s</code>) See <a class="uref" href="https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/plai
patches.rst">Sign your work for the meaning of sign-off.

 Did you provide a clear git commit log message?

 Is the patch against latest FFmpeg git master branch?

 Are you subscribed to ffmpeg-devel? (the list is subscribers only due to spam)

 Have you checked that the changes are minimal, so that the same cannot be achieved with a smaller patch and/or simpler final code?

 If the change is to speed critical code, did you benchmark it?

 If you did any benchmarks, did you provide them in the mail?

 Have you checked that the patch does not introduce buffer overflows or other security issues?

 Did you test your decoder or demuxer against damaged data? If no, see tools/trasher, the noise bitstream filter, and zzuf. Your decoder or demuxer should not crash, end in a (near) infinite loop, or allocate ridiculous amounts of memory when fed damaged data.

 Did you test your decoder or demuxer against sample files? Samples may be obtained at https://samples.ffmpeg.org.

 Does the patch not mix functional and cosmetic changes?

 Did you add tabs or trailing whitespace to the code? Both are forbidden.

 Is the patch attached to the email you send?

 Is the mime type of the patch correct? It should be text/x-diff or text/x-patch or at least text/plain and not application/octet-stream.

 If the patch fixes a bug, did you provide a verbose analysis of the bug?

 If the patch fixes a bug, did you provide enough information, including a sample, so the bug can be reproduced and the fix can be verified? Note please do not attach samples >100k to mails but rather provide a URL, you can upload to https://streams.videolan.org/upload/.

 Did you provide a verbose summary about what the patch does change?

 Did you provide a verbose explanation why it changes things like it does?

 Did you provide a verbose summary of the user visible advantages and disadvantages if the patch is applied?

 Did you provide an example so we can verify the new feature added by the patch easily?

 If you added a new file, did you insert a license header? It should be taken from FFmpeg, not randomly copied and pasted from somewhere else.

 You should maintain alphabetical order in alphabetically ordered lists as long as doing so does not break API/ABI compatibility.

 Lines with similar content should be aligned vertically when doing so improves readability.

 Consider adding a regression test for your code. All new modules should be covered by tests. That includes demuxers, muxers, decoders, encoders filters, bitstream filters, parsers. If its not possible to do that, add an explanation why to your patchset, its ok to not test if there’s a reason.

 If you added NASM code please check that things still work with –disable-x86asm.

 Test your code with valgrind and or Address Sanitizer to ensure it’s free of leaks, out of array accesses, etc.

 <h2 class="chapter">7 Patch review process# TOC</h2>

All patches posted to ffmpeg-devel will be reviewed, unless they contain a clear note that the patch is not for the git master branch. Reviews and comments will be posted as replies to the patch on the mailing list. The patch submitter then has to take care of every comment, that can be by resubmitting a changed patch or by discussion. Resubmitted patches will themselves be reviewed like any other patch. If at some point a patch passes review with no comments then it is approved, that can for simple and small patches happen immediately while large patches will generally have to be changed and reviewed many times before they are approved. After a patch is approved it will be committed to the repository. We will review all submitted patches, but sometimes we are quite busy so especially for large patches this can take several weeks. If you feel that the review process is too slow and you are willing to try to take over maintainership of the area of code you change then just clone git master and maintain the area of code there. We will merge each area from where its best maintained. When resubmitting patches, please do not make any significant changes not related to the comments received during review. Such patches will be rejected. Instead, submit significant changes or new features as separate patches. Everyone is welcome to review patches. Also if you are waiting for your patch to be reviewed, please consider helping to review other patches, that is a great way to get everyone’s patches reviewed sooner. <h2 class="chapter">8 Regression tests# TOC</h2>

Before submitting a patch (or committing to the repository), you should at least test that you did not break anything. Running ’make fate’ accomplishes this, please see fate.html for details. [Of course, some patches may change the results of the regression tests. In this case, the reference results of the regression tests shall be modified accordingly]. <h3 class="section">8.1 Adding files to the fate-suite dataset# <a class="anchor

hidden-xs”href=”#toc-Adding-files-to-the-fate_002dsuite-dataset” aria-hidden=”true”>TOC</h3>

If you need a sample uploaded send a mail to samples-request. When there is no muxer or encoder available to generate test media for a specific test then the media has to be included in the fate-suite. First please make sure that the sample file is as small as possible to test the respective decoder or demuxer sufficiently. Large files increase network bandwidth and disk space requirements. Once you have a working fate test and fate sample, provide in the commit message or introductory message for the patch series that you post to the ffmpeg-devel mailing list, a direct link to download the sample media. <h3 class=”section”>8.2 Visualizing Test Coverage# TOC</h3>

The FFmpeg build system allows visualizing the test coverage in an easy manner with the coverage tools <code class=”code”>gcov</code>/<code class=”code”>lcov</code>. This involves the following steps: <ol class=”enumerate”> Configure to compile with instrumentation enabled: <code class=”code”>configure --toolchain=gcov</code>.

 Run your test case, either manually or via FATE. This can be either the full FATE regression suite, or any arbitrary invocation of any front-end tool provided by FFmpeg, in any combination.

 Run <code class=”code”>make lcov</code> to generate coverage data in HTML format.

 View <code class=”code”>lcov/index.html</code> in your preferred HTML viewer.

You can use the command <code class=”code”>make lcov-reset</code> to reset the coverage measurements. You will need to rerun <code class=”code”>make lcov</code> after running a new test. <h3 class=”section”>8.3 Using Valgrind# TOC</h3>

The configure script provides a shortcut for using valgrind to spot bugs related to memory handling. Just add the option <code class=”code”>--toolchain=valgrind-memcheck</code> or <code class=”code”>--toolchain=valgrind-massif</code> to your configure line, and reasonable defaults will be set for running FATE under the supervision of either the memcheck or the massif tool of the valgrind suite. In case you need finer control over how valgrind is invoked, use the <code class=”code”>--target-exec='valgrind

<your_custom_valgrind_options></code> option in your configure line instead. <h2 class="chapter">9 Maintenance process# TOC</h2>

 <h3 class="section">9.1 MAINTAINERS# TOC</h3>

The developers maintaining each part of the codebase are listed in <samp class="file">MAINTAINERS</samp>. Being listed in <samp class="file">MAINTAINERS</samp>, gives one the right to have git write access to the specific repository. <h3 class="section">9.2 Becoming a maintainer# TOC</h3>

People add themselves to <samp class="file">MAINTAINERS</samp> by sending a patch like any other code change. These get reviewed by the community like any other patch. It is expected that, if someone has an objection to a new maintainer, she is willing to object in public with her full name and is willing to take over maintainership for the area.

 <h2 class="chapter">10 Release process# TOC</h2>

FFmpeg maintains a set of release branches, which are the recommended deliverable for system integrators and distributors (such as Linux distributions, etc.). At regular times, a release manager prepares, tests and publishes tarballs on the https://ffmpeg.org website. There are two kinds of releases: <ol class="enumerate"> Major releases always include the latest and greatest features and functionality.

 Point releases are cut from release branches, which are named <code class="code">release/X</code>, with <code class="code">X</code> being the release version number.

Note that we promise to our users that shared libraries from any FFmpeg re-

lease never break programs that have been compiled against previous versions of the same release series in any case! However, from time to time, we do make API changes that require adaptations in applications. Such changes are only allowed in (new) major releases and require further steps such as bumping library version numbers and/or adjustments to the symbol versioning file. Please discuss such changes on the ffmpeg-devel mailing list in time to allow forward planning. <h3 class=”section”>10.1 Criteria for Point Releases# TOC</h3>

Changes that match the following criteria are valid candidates for inclusion into a point release: <ol class=”enumerate”> Fixes a security issue, preferably identified by a CVE number issued by http://cve.mitre.org/.

 Fixes a documented bug in <a class=”url” href=”https://trac.ffmpeg.org”>https://trac.ffmpeg.org<

 Improves the included documentation.

 Retains both source code and binary compatibility with previous point releases of the same release branch.

The order for checking the rules is (1 OR 2 OR 3) AND 4.

 <h3 class=”section”>10.2 Release Checklist# TOC</h3>

The release process involves the following steps: <ol class=”enumerate”> Ensure that the <samp class=”file”>RELEASE</samp> file contains the version number for the upcoming release.

 Add the release at <a class=”url” href=”https://trac.ffmpeg.org/admin/ticket/versions”>https://tr

 Announce the intent to do a release to the mailing list.

 Make sure all relevant security fixes have been backported. See https://ffmpeg.org/security.html.

 Ensure that the FATE regression suite still passes in the release branch on at least i386 and amd64 (cf. Regression tests).

 Prepare the release tarballs in <code class=”code”>bz2</code>

and <code class=”code”>gz</code> formats, and supplementing files that contain <code class=”code”>gpg</code> signatures

 Publish the tarballs at <a class=”url” href=”https://ffmpeg.org/releases”>https://ffmpeg.org/relea
Create and push an annotated tag in the form <code class=”code”>nX</code>,
with <code class=”code”>X</code> containing the version number.

 Propose and send a patch to the ffmpeg-devel mailing list with a news entry for the website.

 Publish the news entry.

 Send an announcement to the mailing list.

<p style=”font-size: small;”> This document was generated using makeinfo.
</p> </body> </html>

===== .\ffmpeg\doc\drawvg-reference.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> drawvg - Language Reference
</title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> drawvg - Language Reference </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”>

1 Introduction

2 Syntax

<ul class=”toc-numbered-mark”>

2.1 Structure

2.2 Comments

2.3 Commands

<ul class=”toc-numbered-mark”>

2.3.1 Single-Letter Aliases

2.3.2 Implicit Commands

2.4 Arguments

<ul class="toc-numbered-mark">

2.4.1 Number Literals

2.4.2 Expressions

2.4.3 Variable Names

2.4.4 Colors

2.4.5 Constants

3 Guide

<ul class="toc-numbered-mark">

3.1 Paths

<ul class="toc-numbered-mark">

3.1.1 Current Point

3.1.2 Defining a Shape

3.1.3 Fill

3.1.4 Stroke

3.1.5 Clip

3.1.6 Preserving Paths

3.2 Variables

<ul class="toc-numbered-mark">

3.2.1 User Variables

3.3 Patterns

<ul class="toc-numbered-mark">

3.3.1 Gradients

3.3.2 Variables

3.4 Transformations

3.5 State Stack

3.6 Frame Metadata

3.7 <code class="code">if</code> / <code class="code">repeat</code> Statements

<ul class="toc-numbered-mark">

3.7.1 Comparison and Logical Operators

3.7.2 Early Exit

3.8 Procedures

3.9 Functions in Expressions

<ul class="toc-numbered-mark">

3.9.1 Function <code class="code">p</code>

3.9.2 Function <code class="code">pathlen</code>

3.9.3 Function <code class="code">randomg</code>

3.10 Tracing with <code class="code">print</code>

<ul class="toc-numbered-mark">

3.10.1 Function print

3.10.2 Command <code class="code">print</code>

4 Commands

<ul class="toc-numbered-mark">

4.1 <code class="code">arc</code>

4.2 <code class="code">arcn</code>

4.3 <code class="code">break</code>

4.4 <code class="code">call</code>

4.5 <code class="code">circle</code>

4.6 <code class="code">clip</code>, <code class="code">eoclip</code>

4.7 <code class="code">Z</code>, <code class="code">z</code>, <code class="code">closepath</code>

4.8 <code class="code">colorstop</code>

4.9 <code class="code">C</code>, <code class="code">curveto</code>

4.10 <code class="code">c</code>, <code class="code">rcurveto</code>

4.11 <code class="code">defhsla</code>

4.12 <code class="code">defrgba</code>

4.13 <code class="code">ellipse</code>

4.14 <code class="code">fill</code>, <code class="code">eofill</code>

4.15 <code class="code">getmetadata</code><

4.16 <code class="code">H</code>, <code class="code">h</code>

4.17 <code class="code">if</code>

4.18 <code class="code">lineargrad</code>

4.19 <code class="code">L</code>, <code class="code">lineto</code>

4.20 <code class="code">l</code>, <code class="code">rlineto</code>

4.21 <code class="code">M</code>, <code class="code">moveto</code>

4.22 <code class="code">m</code>, <code class="code">rmoveto</code>

4.23 <code class="code">newpath</code>

4.24 <code class="code">preserve</code>

4.25 <code class="code">print</code>

4.26 <code class="code">proc</code>

4.27 <code class="code">Q</code>

4.28 <code class="code">q</code>

4.29 <code class="code">radialgrad</code>

4.30 <code class="code">rect</code>

4.31 <code class="code">repeat</code>

4.32 <code class="code">resetclip</code>

4.33 <code class="code">resetdash</code>

4.34 <code class="code">resetmatrix</code></li

4.35 <code class="code">restore</code>

4.36 <code class="code">rotate</code>

4.37 <code class="code">roundedrect</code></

4.38 <code class="code">save</code>

4.39 <code class="code">scale</code>

4.40 <code class="code">scalexy</code>

4.41 <code class="code">setcolor</code>

4.42 <code class="code">setdash</code>

4.43 <code class="code">setdashoffset</code><

4.44 <code class="code">sethsla</code>

4.45 <code class="code">setlinecap</code>

4.46 <code class="code">setlinejoin</code>

4.47 <code class="code">setlinewidth</code></

4.48 <code class="code">setrgba</code>

4.49 <code class="code">setvar</code>

4.50 <code class="code">stroke</code>

4.51 <code class="code">S</code>, <code class="code">s</code>

4.52 <code class="code">translate</code>

4.53 <code class="code">T</code>, <code class="code">t</code>

4.54 <code class="code">V</code>, <code class="code">v</code>

 <h2 class="chapter">1 Introduction# TOC</h2>

drawvg (draw vector graphics) is a language to draw two-dimensional graphics on top of video frames. It is not intended to be used as a general-purpose language. Since its scope is limited, it prioritizes being concise and easy to use. For example, using the Canvas API we can render a triangle running this code in a Web browser: <pre class="example-preformatted">const canvas = document.getElementById("canvas");
const ctx = canvas.getContext("2d");

ctx.beginPath(); ctx.moveTo(125, 50); ctx.lineTo(100, 100); ctx.lineTo(150, 100); ctx.closePath(); ctx.stroke(); </pre>

The same triangle can be written with this drawvg script: <pre class="example-preformatted">moveto 125 50 lineto 100 100 150 100 closepath stroke </pre>

It can be shortened using the aliases for <code class="code">moveto</code>, <code class="code">lineto</code>, and <code class="code">closepath</code>:   <pre class="example-preformatted">M 125 50 L 100 100 150 100 Z stroke </pre>

Both newlines (<code class="code">U+000A</code>) and spaces (<code class="code">U+0020</code>) can be used interchangeably as delimiters, so multiple commands can appear on the same line:

<pre class="example-preformatted">M 125 50 L 100 100 150 100 Z stroke
</pre>

Finally, drawvg can use <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#Expression-Evaluation">FFmpeg expressions and frame metadata in command arguments. In this example, we are using the variables <var class="var">w</var> (frame width) and <var class="var">h</var> (frame height) to create a circle in the middle of the frame.   <pre class="example-preformatted">circle (w / 2) (h / 2) (w / 3) stroke </pre>

Many commands are a direct equivalent to a function in the Cairo graphics library. For such commands, the reference below provides a link to the related Cairo documentation. <h2 class="chapter">2 Syntax# TOC</h2>

The syntax is heavily inspired by languages like Magick Vector Graphics, or SVG’s <code class="code"><path></code></a
Many command names are taken from PostS
  <h3 class="section">2.1 Structure# TOC</h3>

A drawvg script consists of a series of commands to describe 2D graphics. A command is an identifier (like <code class="code">setcolor</code> or <code class="code">lineto</code>) followed by its arguments. Each item in the code (command name, arguments, etc.) is separated by any of the following characters: <ul class="itemize mark-bullet"> Space (<code class="code">' '</code>) Comma (<code class="code">','</code>) Newline (<code class="code">'\n'</code>) Tabs (<code class="code">'\t'</code>) Return (<code class="code">'\r'</code>)

The beginning of the item indicates how it will be interpreted: <dl class="table"> <dt><code class="code">//</code></dt> <dd> Comment </dd> <dt><code class="code">0</code>, …, <code class="code">9</code>, <code class="code">+</code>, <code class="code">-</code></dt> <dd> Number literal </dd> <dt><code class="code">(</code></dt> <dd> Expression </dd> <dt><code class="code">{</code>, <code class="code">}</code></dt> <dd> Block delim-

iters </dd> <dt>Anything else</dt> <dd> Name of a command, a color, etc. </dd> </dl>

 <h3 class=”section”>2.2 Comments# TOC</h3>

Comments start with two slashes (<code class=”code”>//</code>), and stop at the end of the line (either a <code class=”code”>\n</code>, or the end of the script).   <pre class=”example-preformatted”>circle 100 100 50 // this is ignored fill

// this is also ignored </pre>

<code class=”code”>//</code> must appear after a space, or at the beginning of the line. If <code class=”code”>//</code> is preceded by any non-blank character, the parser will consider <code class=”code”>//</code> as part of the previous item. For example, in this script: <pre class=”example-preformatted”>circle 10 10 50// something </pre>

The parser throws an error because it tries to parse <code class=”code”>50//</code> as a number literal. <h3 class=”section”>2.3 Commands# TOC</h3>

The way commands are parsed is inspired by SVG’s <code class=”code”><path></code>: <ul class=”itemize mark-bullet”> Every command in the script starts with its name, and it is followed by zero or more arguments.

There are no explicit delimiters between commands or arguments.

Most programming languages expect characters like parenthesis, commas, or semicolons, to separate items. For example: <pre class=”example-preformatted”>moveto(10, 10); lineto(20, 30); </pre>

The equivalent in drawvg is: <pre class=”example-preformatted”>moveto 10 10 lineto 20 30 </pre>

If the command has no arguments (like <code class=”code”>closepath</code> or <code class=”code”>stroke</code>), the next command starts at the next item.

<table class="cartouche"><tr><th> Example</th></tr><tr><td>
In the next script there are 4 different commands: <pre class="example-preformatted">newpath rect 10 20 30 40 setcolor teal fill </pre>

<ol class="enumerate"> <code class="code">newpath</code> requires no arguments.

 <code class="code">rect</code> requires 4 arguments, so it takes the next 4 numbers.

 <code class="code">setcolor</code> requires 1 argument, so it takes the word <code class="code">teal</code>.

 <code class="code">fill</code> requires no arguments.

</td></tr></table>

 <h4 class="subsection">2.3.1 Single-Letter Aliases# TOC</h4>

Most commands in SVG’s <code class="code"><path></code> are also present in drawvg. For some of them, there is an alias to a longer name: <ul class="itemize mark-bullet"> <code class="code">curveto</code> for <code class="code">C</code>.
<code class="code">rcurveto</code> for <code class="code">c</code>.
<code class="code">lineto</code> for <code class="code">L</code>.
<code class="code">rlineto</code> for <code class="code">l</code>.
<code class="code">moveto</code> for <code class="code">M</code>.
<code class="code">rmoveto</code> for <code class="code">m</code>.
<code class="code">closepath</code> for <code class="code">Z</code>, <code class="code">z</code>.

Other commands only exist in a single-letter form: <ul class="itemize mark-bullet"> <code class="code">H</code>, <code class="code">h</code> <code

class=”code”>Q</code>, <code class=”code”>q</code> <code class=”code”>S</code>, <code class=”code”>s</code> <code class=”code”>V</code>, <code class=”code”>v</code> <code class=”code”>T</code>, <code class=”code”>t</code>

This makes it possible to use a path in SVG to create the same shape in a drawvg script. <h4 class=”subsection”>2.3.2 Implicit Commands# TOC</h4>

For many commands, the name can be omitted when it is used multiple times in successive calls. In the reference below, these commands has a Can be Implicit note in their signature. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> For example, in this script: <pre class=”example-preformatted”>M 50 50 l 10 10 l 10 -10 l 10 10 l 10 -10 l 10 10 stroke </pre>

After the first call to <code class=”code”>l</code> (alias to <code class=”code”>rlineto</code>), the command can be executed without the name, so it can be written as: <pre class=”example-preformatted”>M 50 50 l 10 10 10 -10 10 10 10 -10 10 10 stroke </pre>

</td></tr></table>

To reuse the same command (<code class=”code”>l</code>, in the previous example), the parser checks if the item after the last argument is a numeric value, like a number literal or a FFmpeg expression. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example: <pre class=”example-preformatted”>l 10 20 30 40 stroke </pre>

<code class=”code”>l</code> requires 2 arguments, and can be implicit, so the parser performs this operation: <ol class=”enumerate”> Takes the two next items (<code class=”code”>10</code> and <code class=”code”>20</code>) and emits the first instruction.

 Checks if the item after <code class=”code”>20</code> is a numeric value. Since it is <code class=”code”>30</code>, it takes <code

class=”code”>30</code> and <code class=”code”>40</code> and emits the second instruction (<code class=”code”>l 30 40</code>).

 Checks if the next item after <code class=”code”>40</code> is a numeric value, but it is a command (<code class=”code”>stroke</code>), so it stops reusing <code class=”code”>l</code>.

</td></tr></table>

This is another feature taken from SVG’s <code class=”code”><path></code> An important difference with SVG is that the separator between items is always required. In SVG, it can be omitted in some cases. For example, the expression <code class=”code”>m1-2</code> is equivalent to <code class=”code”>m 1 -2</code> in SVG, but a syntax error in drawvg. <h3 class=”section”>2.4 Arguments# TOC</h3>

Most commands expect numeric arguments, like number literals, variable names, or expressions. <code class=”code”>setcolor</code> and <code class=”code”>colorstop</code> expect a color. <code class=”code”>setlinecap</code> and <code class=”code”>setlinejoin</code> expect a constant value. <h4 class=”subsection”>2.4.1 Number Literals# TOC</h4>

A number literal is an item in the script that represents a constant value. Any item that starts with a decimal digit (between <code class=”code”>0</code> and <code class=”code”>9</code>), a <code class=”code”>-</code> or a <code class=”code”>+</code>, is interpreted as a number literal. The value is parsed with <code class=”code”>av_strtod</code>. It supports the prefix <code class=”code”>0x</code> to write a value with hexadecimal digits, and <a class=”uref” href=”https://ffmpeg.org/ffmpeg-utils.html#:~:text=The%20evaluator%20also%20recognizes%20 units (like <code class=”code”>K</code> or <code class=”code”>GiB</code>). In the next example, all literals represent the same value: <pre class=”example-preformatted”>10000 1e4 10K 0x2710 </pre>

 <h4 class=”subsection”>2.4.2 Expres-

sions# TOC</h4>

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#Expression-Evaluation”>FFmpeg expressions can be used as arguments for any command that expects a numeric argument. The expression must be enclosed in parenthesis. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> The variables <var class=”var”>w</var> and <var class=”var”>h</var> represent the width and height of the frame. We can compute the center of the frame by dividing them by <code class=”code”>2</code>:   <pre class=”example-preformatted”>M (w / 2) (h / 2) </pre>

They can also contain parenthesis (to group operations, to call functions, etc):   <pre class=”example-preformatted”>moveto ((w + 10) / 2) // x (h / (2 * cos(t))) // y </pre>

</td></tr></table>

The variables <var class=”var”>n</var> and <var class=”var”>t</var> can be used to compute a value that changes over time. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> To draw a circle oscillating from left to right, we can use an expression based on <code class=”code”>sin(t)</code> for the <code class=”code”>x</code> coordinate: <pre class=”example-preformatted”>circle (w / 2 + sin(2 * t) * w / 4) // x (h / 2) // y (w / 5) // radius

stroke </pre>

</td></tr></table>

Expressions can be split in multiple lines, but they can’t contain comments within them.   <pre class=”example-preformatted”>moveto // This is a comment. (w // This is part of the expression, not a comment. + h) </pre>

 <h4 class=”subsection”>2.4.3 Variable Names# TOC</h4>

When an expression is only a reference to a variable, the parenthesis can be omitted, and the item is just the variable name. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> The next 3 expressions are equivalent: in all cases, they create a rectangle covering the whole frame.   <pre class=”example-preformatted”>rect (0) (0) (w) (h)

rect 0 0 w h

rect (0) 0 (w) h </pre>

</td></tr></table>

It is possible to create a variable with the same name of a command, and then use it as an argument. In the previous example, the item <var class="var">h</var> is a reference to a variable (frame height), but in other contexts it may be a command (<code class="code">h</code>). For implicit commands, the parser prioritizes commands over variable names when it has to determine if the command is reused. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, the variable <var class="var">c</var> is used as the first argument in two calls to <code class="code">l</code>. However, only the first one is valid, because in the second call the parser recognizes <code class="code">c</code> as a command.   <pre class="example-preformatted">setvar c 5 l c 10 c 15 </pre>

This issue can be fixed by surrounding the start of the second call with parenthesis: <pre class="example-preformatted">setvar c 5 l c 10 (c) 15 </pre>

</td></tr></table>

 <h4 class="subsection">2.4.4 Colors# TOC</h4>

The color to stroke and to fill paths can be set with <code class="code">setcolor</code>. Its argument has the same syntax for colors in FFmpeg: <ul class="itemize mark-bullet"> A <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#Color">predefined color name.

In <code class="code">#RRGGBB</code> format.

Optionally, an <code class="code">@a</code> suffix can be added to set the alpha value, where <code class="code">a</code> is a number between <code class="code">0</code> and <code class="code">1</code>.

 <pre class="example-preformatted">circle 70 70 60 setcolor #FF0000 fill

circle 170 170 60 setcolor blue@0.5 fill </pre>

The color can be a variable name. In that case, it must be assigned with <code class="code">defrgba</code>,

<code class=”code”>defhsla</code>, or <code class=”code”>setvar</code> and a color.   <pre class=”example-preformatted”>circle 70 70 60 setvar CustomGreen #22FF44 setcolor CustomGreen fill

circle 170 170 60 defhsla CustomBlue 200 0.7 0.5 1 setcolor CustomBlue fill </pre>

The commands <code class=”code”>setrgba</code> and <code class=”code”>sethsla</code> allow setting colors using expressions. Similar to <code class=”code”>defrgba</code> and <code class=”code”>defhsla</code>, but with no intermediate variable. <h4 class=”subsection”>2.4.5 Constants# TOC</h4>

The argument for <code class=”code”>setlinecap</code> and <code class=”code”>setlinejoin</code> is an identifier referring to a constant value.   <pre class=”example-preformatted”>setlinecap round </pre>

 <h2 class=”chapter”>3 Guide# TOC</h2>

 <h3 class=”section”>3.1 Paths# TOC</h3>

A path is a complex shape, composed by lines and curves, that can be used to fill a region, to stroke an outline, or to establish a clip region. In order to draw anything on top of a video frame, first we have to define a path, and then use <code class=”code”>stroke</code> or <code class=”code”>fill</code>. The tutorial on paths in MDN is a good introduction to the topic. It is focused on SVG&rs
<code class=”code”><path></code>, but the same concepts can be applied in drawvg. <h4 class=”subsection”>3.1.1 Current Point<a class=”anchor hidden-xs” href=”#Current-

Point" aria-hidden="true"># TOC</h4>

Some commands require a current point. Initially, the current point is set to <code class="code">NaN</code>. It is initialized with <code class="code">M</code> or <code class="code">moveto</code>. Other commands, like <code class="code">lineto</code> or <code class="code">curveto</code>, updates the current point to the new end of the shape. The current point can be cleared with <code class="code">newpath</code>. Commands that clear the path, like <code class="code">stroke</code> or <code class="code">fill</code>, also clear the current point. <table class="cartouche"><tr><th> Example</th></tr><tr><td> <code class="code">rlineto</code> uses coordinates relative to the current point. Given this script: <pre class="example-preformatted">moveto 20 100 rlineto 150 -90 rlineto -50 200 closepath stroke </pre>

These are the coordinates of the current point after executing each command: <table class="multitable"> <thead><tr><th width="50%">Command</th><th width="50%">Current Point</th></tr></thead> <tbody><tr><td width="50%"><code class="code">moveto 20 100</code></td><td width="50%"><code class="code">20, 100</code></td></tr> <tr><td width="50%"><code class="code">rlineto 150 -90</code></td><td width="50%"><code class="code">170, 10</code></td></tr> <tr><td width="50%"><code class="code">rlineto -10 50</code></td><td width="50%"><code class="code">140, 210</code></td></tr> <tr><td width="50%"><code class="code">closepath</code></td><td width="50%"><code class="code">20, 100</code></td></tr> </tbody> </table>

The same script can be written with single-letter aliases: <pre class="example-preformatted">M 20 100 l 150 -90 -50 200 z stroke </pre>

</td></tr></table>

 <h4 class="subsection">3.1.2 Defining a Shape# TOC</h4>

A path is defined by adding lines, curves, or basic shapes. <ul class="itemize mark-bullet"> Basic shapes

<ul class="itemize mark-bullet"> <code class="code">circle</code> <code class="code">ellipse</code> <code class="code">rect</code> <code class="code">roundedrect</code>

Lines <ul class="itemize mark-bullet"> <code class="code">M</code>, <code class="code">moveto</code> <code class="code">m</code>, <code class="code">rmoveto</code> <code class="code">H</code>, <code class="code">h</code> <code class="code">V</code>, <code class="code">v</code> <code class="code">L</code>, <code class="code">lineto</code> <code class="code">l</code>, <code class="code">rlineto</code> <code class="code">Z</code>, <code class="code">z</code>, <code class="code">closepath</code>

Curves <ul class="itemize mark-bullet"> <code class="code">arc</code>, <code class="code">arcn</code> <code class="code">C</code>, <code class="code">curveto</code>, <code class="code">c</code>, <code class="code">rcurveto</code> <code class="code">Q</code>, <code class="code">q</code> <code class="code">S</code>, <code class="code">s</code> <code class="code">T</code>, <code class="code">t</code>

Single-letter commands are taken from SVG’s <code class="code"><path></code> <h4 class="subsection">3.1.3 Fill# TOC</h4>

The region within the shape defined by a path can be filled with <code class=”code”>fill</code> or <code class=”code”>eofill</code>. Each command uses a different fill rule: <ul class=”itemize mark-bullet”> <code class=”code”>fill</code> uses the winding rule, also known as nonzero rule. <code class=”code”>eofill</code> uses the even–odd rule.

<table class=”cartouche”><tr><th> Example</th></tr><tr><td> This script shows the difference between the winding and even–odd rules: <pre class=”example-preformatted”>rect 50 10 100 60 circle 150 70 40 setcolor seagreen fill

rect 50 130 100 60 circle 150 190 40 setcolor skyblue eofill </pre>

</td></tr></table>

 <h4 class=”subsection”>3.1.4 Stroke# TOC</h4>

<code class=”code”>stroke</code> draws a line around the shape defined by the path. The stroke can be configured with different commands: <ul class=”itemize mark-bullet”> <code class=”code”>setdash</code> <code class=”code”>setdashoffset</code> <code class=”code”>setlinecap</code> <code class=”code”>setlinejoin</code> <code class=”code”>setlinewidth</code> <code class=”code”>resetdash</code>

<table class=”cartouche”><tr><th> Example</th></tr><tr><td> This example use <code class=”code”>setdashoffset</code> to animate the stroke: <pre class=”example-preformatted”>moveto 0 0 lineto w h

setlinecap round setdash 50 50 setlinewidth 20 setdashoffset (hypot(w, h) * t / -3) setcolor seagreen

stroke </pre>

</td></tr></table>

 <h4 class="subsection">3.1.5 Clip# TOC</h4>

A clip region can be established with <code class="code">clip</code> and <code class="code">eoclip</code>. If there is an active clip region, the new clip region will be the intersection between the existing one and the path. <code class="code">resetclip</code> reset the clip region to the whole frame. <code class="code">eoclip</code> uses the even–odd rule to compute the clip region. <table class="cartouche"><tr><th> Example</th></tr><tr><td> <pre class="example-preformatted">rect 50 50 100 200 clip

circle 30 30 150 setcolor seagreen fill

// Draw outside the clip region. resetclip circle 30 30 150 setlinewidth 3 setcolor skyblue stroke </pre>

</td></tr></table>

 <h4 class="subsection">3.1.6 Preserving Paths# TOC</h4>

The path is cleared after any operation on it, like <code class="code">fill</code> or <code class="code">stroke</code>. To reuse the same path in multiple operations, <code class="code">preserve</code> must be called before them. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, each path is used twice.   <pre class="example-preformatted">circle 120 120 50 setcolor seagreen preserve stroke clip

circle 100 100 50 setcolor skyblue preserve fill setcolor tomato stroke </pre>

</td></tr></table>

 <h3 class="section">3.2 Variables<a class="anchor hidden-xs" href="#Variables" aria-

hidden=”true”># TOC</h3>

A drawvg can use some variables, provided by the interpreter, to compute values in <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#Expression-Evaluation”>FFmpeg expressions: <dl class=”table”> <dt><var class=”var”>cx</var></dt> <dd> X coordinate of the current point. </dd> <dt><var class=”var”>cy</var></dt> <dd> Y coordinate of the current point. </dd> <dt><var class=”var”>w</var></dt> <dd> Width, in pixels, of the frame. </dd> <dt><var class=”var”>h</var></dt> <dd> Height, in pixels, of the frame. </dd> <dt><var class=”var”>i</var></dt> <dd> The loop counter in repeat blocks. </dd> <dt><var class=”var”>n</var></dt> <dd> Frame number. </dd> <dt><var class=”var”>t</var></dt> <dd> Timestamp, in seconds. </dd> <dt><var class=”var”>ts</var></dt> <dd> Timestamp, in seconds, of the first frame. </dd> <dt><var class=”var”>duration</var></dt> <dd> Duration, in seconds, of the frame. </dd> </dl>

 <h4 class=”subsection”>3.2.1 User Variables#
TOC</h4>

New variables can be created with the <code class=”code”>setvar</code> command. It associates a name with a numeric value. The name must follow these rules: <ul class=”itemize mark-bullet”> It must start with an ASCII letter or an underscore (<code class=”code”>_</code>).

It can contain only ASCII letters, underscores, and digits.

It must not match the name of a variable provided by the interpreter (like <var class=”var”>w</var> or <var class=”var”>t</var>).

The same variable can be assigned multiple times. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, the result of an expression is stored in a variable with the name <var class=”var”>progress</var>. Then, it is used for the <var class=”var”>x</var> and <var class=”var”>width</var> arguments of <code class=”code”>rect</code>.   <pre class=”example-preformatted”>setvar progress (w * (pow(mod(t / 2 + 0.5, 1), 2.5)))

rect ((w - progress) / 2) 0 progress h

setcolor darkblue fill </pre>

</td></tr></table>

Currently, a script can contain only 20 different variable names, but this limit can be modified in the future. <h3 class=”section”>3.3 Patterns# TOC</h3>

The pattern for fill and stroke operations can be either a solid color, or a gradient. <ul class=”itemize mark-bullet”> Solid colors.

<ul class=”itemize mark-bullet”> <code class=”code”>setcolor</code> <code class=”code”>sethsla</code> <code class=”code”>setrgba</code>

Gradients.

<ul class=”itemize mark-bullet”> <code class=”code”>lineargrad</code> <code class=”code”>radialgrad</code>

The pattern is not cleared after being used in a fill or stroke operation, but it is replaced by any command that sets a new pattern. <h4 class=”subsection”>3.3.1 Gradients# TOC</h4>

To configure a gradient, first call to <code class=”code”>lineargrad</code> or <code class=”code”>radialgrad</code>, and then add color stops by calling <code class=”code”>colorstop</code> for each stop. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, the whole frame is filled with a linear gradient: <pre class=”example-preformatted”>lineargrad 0 0 w h colorstop 0 skyblue colorstop 1 darkblue

rect 0 0 w h fill </pre>

In this example, a radial gradient is used to simulate a sphere: <pre class=”example-preformatted”>radialgrad 90 90 5 120 120 100 colorstop 0.0 #90DDFF colorstop 0.9 #000030 colorstop 1.0 #000000

rect 0 0 w h fill </pre>

</td></tr></table>

 <h4 class="subsection">3.3.2 Variables# TOC</h4>

<code class="code">setcolor</code> and <code class="code">colorstop</code> accept a variable name as the argument. The variable must be assigned with <code class="code">defrgba</code>, <code class="code">defhsla</code>, or <code class="code">setvar</code> and a color. <table class="cartouche"><tr><th> Example</th></tr><tr><td> <pre class="example-preformatted">// Use color #1020FF, alpha = 50% setvar someblue #1020FF@0.5

setcolor someblue

rect 30 30 120 120 fill

rect 90 90 120 120 fill </pre>

</td></tr></table>

If a variable has the same name of a <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#Color">known color, the variable has preference, and will be used instead of the predefined color. <table class="cartouche"><tr><th> Example</th></tr><tr><td> <pre class="example-preformatted">setcolor teal rect 30 30 120 120 fill

setvar teal #70AAAA setcolor teal // Use the new color for 'teal'. rect 90 90 120 120 fill </pre>

</td></tr></table>

<code class="code">defrgba</code> and <code class="code">defhsla</code> assign a color to a variable, by providing an expression for each color component: <ul class="itemize mark-bullet"> For <code class="code">defrgba</code>: red, green, blue, and alpha.

For <code class="code">defhsla</code>: hue, saturation, lightness, and alpha.

Each color component must be in range <code class="code">0</code> to <code class="code">1</code>, except hue, which is <code class="code">0</code> to <code class="code">360</code>.

<table class="cartouche"><tr><th> Example</th></tr><tr><td> <pre class="example-preformatted">defrgba colorA 1 0.5 0.25 1 // colorA = RGB(255, 127, 63) defhsla colorB 200 0.75 0.25 1 // colorB = HSL(200, 75%, 25%)

rect 0 0 (w / 2) h setcolor colorA fill

rect (w / 2) 0 (w / 2) h setcolor colorB fill </pre>

</td></tr></table>

 <h3 class="section">3.4 Transformations# TOC</h3>

The coordinates for each command can be scaled, rotated, and translated, by using the following commands: <ul class="itemize mark-bullet"> <code class="code">rotate</code> <code class="code">scale</code> <code class="code">scalexy</code> <code class="code">translate</code>

The transformations are applied when the command is executed. They have no effect on the existing path, only on the new segments added to it. They are done by updating the current transformation matrix in the Cairo context. To reset the matrix to its original state, before any transformation, use <code class="code">resetmatrix</code>. The transform origin for scale and rotation is initially at <code class="code">0, 0</code>, but it can be adjusted with <code class="code">translate</code>. <table class="cartouche"><tr><th> Example</th></tr><tr><td> <pre class="example-preformatted">// Map (0, 0) as the center of the frame. translate (w / 2) (h / 2)

// Scale the space as if the frame is 1x1 pixel. scalexy w h

// Draw multiple lines with the same arguments, // but each one on a different rotation. repeat 10 { rotate (PI / 10) M -0.25 0 H 0.25 }

// Reset transformations, so the scale does not // affect stroke. resetmatrix

stroke </pre>

</td></tr></table>

 <h3 class="section">3.5 State Stack<a class="anchor

hidden-xs" href="#State-Stack" aria-hidden="true"># TOC</h3>

The state of a drawvg script contains all parameters used for drawing operations, like the current color, the transformation matrix, the stroke configuration, etc. The <code class="code">save</code> command pushes a snapshot of the state to an internal stack. Later, <code class="code">restore</code> pops the latest snapshot from the stack, and uses it as the new state. The parameters that can be saved and restored are: <ul class="itemize mark-bullet"> Pattern for stroke and fill operations.

<ul class="itemize mark-bullet"> <code class="code">lineargrad</code> <code class="code">radialgrad</code> <code class="code">setrgba</code> <code class="code">setcolor</code> <code class="code">sethsla</code>

Transformation matrix.

<ul class="itemize mark-bullet"> <code class="code">resetmatrix</code> <code class="code">rotate</code> <code class="code">scale</code> <code class="code">scalexy</code> <code class="code">translate</code>

Stroke configuration.

<ul class="itemize mark-bullet"> <code class="code">setdash</code> <code class="code">setdashoffset</code> <code class="code">setlinecap</code> <code class="code">setlinejoin</code> <code class="code">setlinewidth</code>

Clip region

<ul class="itemize mark-bullet"> <code class="code">clip</code> <code class="code">resetclip</code>

 <h3 class="section">3.6 Frame Metadata# TOC</h3>

Some FFmpeg filters add metadata to frames. The command <code class="code">getmetadata</code> can read metadata items containing a numeric value, and store it in a variable that can be used for command arguments. <table class="cartouche"><tr><th> Example</th></tr><tr><td> The <code class="code">cropdetect</code> filter computes the parameters to remove empty regions around the video. These parameters are accessible in the <code class="code">lavfi.cropdetect</code> keys of the frame metadata.   <pre class="example-preformatted">// Get metadata from cropdetect filter and store it // in 'cd*' variables. getmetadata cdx lavfi.cropdetect.x getmetadata cdy lavfi.cropdetect.y getmetadata cdw lavfi.cropdetect.w getmetadata cdh lavfi.cropdetect.h

rect cdx cdy cdw cdh setcolor yellow@0.5 setlinewidth 10 stroke </pre>

To test the script, copy it to a <code class="code">drawcropdetect.vgs</code> file, and then execute a command like this: <pre class="example-preformatted">ffplay -i example-video.webm -vf 'cropdetect, drawvg=file=drawcropdetect.vgs' </pre>

</td></tr></table>

 <h3 class="section">3.7 <code class="code">if</code> / <code class="code">repeat</code> Statements# TOC</h3>

There is limited support for control flow statements: only <code class="code">if</code> and <code class="code">repeat</code>. Both commands receive two arguments: an expression and a block.   <pre class="example-preformatted">if (condition) { // commands }

repeat (count) { // commands } </pre>

<code class="code">if</code> executes its block if the result of <code class="code">(condition)</code> is not zero. <code class="code">repeat</code> executes its block the number of times specified by <code class="code">(count)</code>. In each iteration, the variable <var class="var">i</var> is used as a loop counter. If the result of the expression is not a finite number (like <code class="code">NaN</code>) the block is not executed. <h4 class="subsection">3.7.1 Comparison

and Logical Operators# TOC</h4>

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#Expression-Evaluation">FFmpeg expressions only supports arithmetic operators (like <code class="code">+</code> for addition). Comparison operators (like <code class="code">!=</code>) are supported via functions, while logical operators (like <code class="code">&</code> for <code class="code">AND</code>) can be emulated with arithmetic operations. <table class="multitable"> <thead><tr><th width="50%">Expression</th><th width="50%">FFmpeg Equivalent</th></tr></thead> <tbody><tr><td width="50%"><code class="code">x = y</code></td><td width="50%"><code class="code">eq(x, y)</code></td></tr> <tr><td width="50%"><code class="code">x < y</code></td><td width="50%"><code class="code">lt(x, y)</code></td></tr> <tr><td width="50%"><code class="code">x > y</code></td><td width="50%"><code class="code">gt(x, y)</code></td></tr> <tr><td width="50%"><code class="code">x y</code></td><td width="50%"><code class="code">lte(x, y)</code></td></tr> <tr><td width="50%"><code class="code">x y</code></td><td width="50%"><code class="code">gte(x, y)</code></td></tr> <tr><td width="50%"><code class="code">a x b</code></td><td width="50%"><code class="code">between(x, a, b)</code></td></tr> </tbody> </table>

Logical operators can be emulated with multiplication (for <code class="code">AND</code>), or addition (for <code class="code">OR</code>): <table class="multitable"> <thead><tr><th width="50%">Expression</th><th width="50%">FFmpeg Equivalent</th></tr></thead> <tbody><tr><td width="50%"><code class="code">x OR y</code></td><td width="50%"><code class="code">x + y</code></td></tr> <tr><td width="50%"><code class="code">x AND y</code></td><td width="50%"><code class="code">x * y</code></td></tr> </tbody> </table>

<table class="cartouche"><tr><th> Example</th></tr><tr><td> In other programming languages, a code like this: <pre class="example-preformatted">if (x > y & z != 1) { // … } </pre>

Can be written for drawvg like this: <pre class="example-preformatted">if (gt(x, y) * not(eq(z, 1))) { // … } </pre>

</td></tr></table>

 <h4 class="subsection">3.7.2 Early Exit# <a class="anchor hidden-xs"href="#toc-Early-

Exit" aria-hidden="true">TOC</h4> <code class="code">break</code> causes a <code class="code">repeat</code> loop to be terminated immediately. If it is executed outside a <code class="code">repeat</code> block, it terminates the whole script, or the current procedure. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, we are using the <code class="code">randomg</code> function to draw a line with random segments. The loop can be executed <code class="code">500</code> times, but it is interrupted if the X coordinate of the current point (<var class="var">cx</var>) exceeds the frame width (<var class="var">w</var>). The current point is updated after each call to <code class="code">rlineto</code>.   <pre class="example-preformatted">moveto 0 0

repeat 500 { rlineto (randomg(0) * 15) (randomg(0) * 20)

if (gt(cx, w)) { break } }

stroke </pre>

</td></tr></table>

 <h3 class="section">3.8 Procedures# TOC</h3>

A procedure is a name associated with a block that can be executed multiple times. It can take between 0 and 6 parameters. <code class="code">proc</code> is used to set the parameter names and the block for a procedure: <pre class="example-preformatted">proc p0 { // … }

proc p1 param1 param2 { // … } </pre>

Inside the block, the arguments can be accessed as regular variables: <pre class="example-preformatted">proc square center_x center_y side { rect (center_x - side / 2) (center_y - side / 2) side side } </pre>

<code class="code">call</code> executes the block assigned to the procedure name. It requires the name of the procedure, and the value for each parameter defined in the call to <code class="code">proc</code>.   <pre class="example-preformatted">call p0

call p1 1 2

call square (w / 2) (h / 2) (w / t) </pre>

<table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, the procedure <code class="code">zigzag</code> draws multiple lines from the current point.   <pre class="example-preformatted">setvar len (w / 10) setlinewidth 5

proc zigzag { repeat 10 { l len len len (-len) }

stroke }

setcolor #40C0FF M 0 60 call zigzag

setcolor #00AABB M 0 120 call zigzag

setcolor #20F0B7 M 0 180 call zigzag </pre>

The color and the Y coordinate of the starting point can be sent as procedure arguments: <pre class="example-preformatted">setvar len (w / 10) setlinewidth 5

proc zigzag color y { setcolor color

M 0 y repeat 10 { l len len len (-len) }

stroke }

call zigzag #40C0FF 60 call zigzag #00AABB 120 call zigzag #20F0B7 180 </pre>

</td></tr></table>

When the procedure returns, the value of the variable for each argument is restored to the value it had before calling the procedure. Changes in other variables (with <code class="code">setvar</code>, <code class="code">getmetadata</code>, <code class="code">defhsla</code>, and <code class="code">defrgba</code>) are preserved. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In the next example, the variable <var class="var">A</var> has the value <code class="code">0</code> before calling the procedure <var class="var">P</var>. During the execution of <var class="var">P</var>, <code class="code">A</code> is <code class="code">1</code>, but after it, <var class="var">A</var> is <code class="code">0</code> again.   <pre class="example-preformatted">setvar A 0

proc P A { print A }

print A call P 1 print A </pre>

It writes the following messages: <pre class="verbatim">[7:7] A = 0.000 [4:8] A = 1.000 [9:7] A = 0.000 </pre> </td></tr></table>

<code class=”code”>break</code> causes the script to leave the current procedure, similar to the <code class=”code”>return</code> statement in other programming languages, unless it is called within a <code class=”code”>repeat</code> loop. The body of the procedure must be defined with <code class=”code”>proc</code> before using <code class=”code”>call</code>. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, when the procedure <code class=”code”>notyet</code> is called, its body has not yet defined, so the execution fails with the error <code class=”code”>Missing body for procedure 'notyet'</code>.   <pre class=”example-preformatted”>call notyet

proc notyet { // ... } </pre>

</td></tr></table>

A procedure can be redefined by other calls to <code class=”code”>proc</code> with the same name. In such case, <code class=”code”>call</code> invokes the last assigned block. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, the procedure <code class=”code”>example</code> has two different blocks.   <pre class=”example-preformatted”>proc example { // block1 }

call example // executes block1

proc example { // block2 }

call example // executes block2 </pre>

</td></tr></table>

 <h3 class=”section”>3.9 Functions in Expressions# TOC</h3>

There are some functions specific to drawvg available in <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#Expression-Evaluation”>FFmpeg expressions. <h4 class=”subsection”>3.9.1 Function <code class=”code”>p</code># TOC</h4>

<code class=”code”>p(x, y)</code> returns the color of the pixel at coordinates <code class=”code”>x, y</code>, as a <code class=”code”>0xRRGGBBAA</code>

value. It can be assigned to a variable, so the color can be available for <code class=”code”>setcolor</code> and <code class=”code”>colorstop</code> commands. If a single expression contains multiple calls to the function, it must return the value of the last call in order to use it as a color. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, the first call to <code class=”code”>p(0, 0)</code> is stored in the variable <var class=”var”>0</var> of the expression. Then, the same expression makes a second call to <code class=”code”>p(1, 1)</code>, and finally it returns the value in the variable <var class=”var”>0</var>.   <pre class=”example-preformatted”>setvar pixel (st(0, p(0, 0)); p(1, 1); ld(0)) </pre>

Since the result of the expression is not the last call to <code class=”code”>p</code>, the variable <var class=”var”>pixel</var> can not be used as a color, but it still can be used as a numeric <code class=”code”>0xRRGGBBAA</code> value. </td></tr></table>

If the coordinates are outside the frame, or any of the arguments is not a finite number (like <code class=”code”>NaN</code>), the function returns <code class=”code”>NaN</code>. The transformation matrix is applied to the arguments. To use the original frame coordinates, call <code class=”code”>resetmatrix</code> between <code class=”code”>save</code> and <code class=”code”>restore</code>:   <pre class=”example-preformatted”>save resetmatrix setvar pixel (p(0, 0)) // top-left pixel of the frame. restore

setcolor pixel </pre>

Bitwise operations can be used to extract individual color components: <pre class=”example-preformatted”>setvar pixel (p(x, y))

if (not(isnan(pixel))) { setvar px_red (pixel / 0x1000000) setvar px_green (bitand(pixel / 0x10000, 0xFF)) setvar px_blue (bitand(pixel / 0x100, 0xFF)) setvar px_alpha (bitand(pixel, 0xFF)) } </pre>

 <h4 class=”subsection”>3.9.2 Function <code class=”code”>pathlen</code># TOC</h4>

<code class=”code”>pathlen(n)</code> computes the length of the current path, by adding the length of each line segment returned by <code class=”code”>cairo_copy_path_flat</code>.

The function expects an argument <var class="var">n</var>, as the maximum number of line segments to add to the length, or <code class="code">0</code> to add all segments. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, <code class="code">pathlen</code> is used to animate the stroke of a spiral, in a 5 seconds loop.   <pre class="example-preformatted">M (w / 2) (h / 2)

setvar a -1 repeat 16 { rcurveto (a * 2 / 3) 0 (a * 2 / 3) (a) 0 (a)

setvar a (-sgn(a) * (abs(a) + 10)) }

setlinewidth 3 setdash (pathlen(0) * (1 - mod(t / 5, 1))) 1e6

setcolor teal stroke </pre>

</td></tr></table>

 <h4 class="subsection">3.9.3 Function <code class="code">randomg</code># TOC</h4>

<code class="code">randomg(idx)</code> is similar to the <code class="code">random(idx)</code> function, available in <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#Expression-Evaluation">FFmpeg expressions, but its state is global to the frame, instead of specific to each expression. To understand the difference, we need to dive into how <code class="code">random(idx)</code> works inside a drawvg script. First, each expression in FFmpeg has a set of 10 internal variables, which can be written with <code class="code">st(idx, value)</code>, and can be read with <code class="code">ld(idx)</code>. <var class="var">idx</var> is a value between <code class="code">0</code> and <code class="code">9</code>. These variables are initialized to <code class="code">0</code>. When a drawvg script is parsed, each expression is compiled with <c
class="code">av_expr_parse</code>, from <a class="uref" href="https://ffmpeg.org/libavutil.html">
and these compiled expressions are reused for every frame. The changes in the internal variables (with <code class="code">st(idx, value)</code>) are visible between frames, but they are not shared between expressions. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, the expression for the X coordinate updates its internal variable <code class="code">0</code> in every frame: <pre class="example-preformatted">circle (st(0, mod(ld(0) + 15, w))) // X 120 // Y (ld(0) + 20) // radius

fill </pre>

<code class="code">st(idx, value)</code> returns the updated value, so it can be used as the result of the expression. The radius is not affected because its internal variable (from <code class="code">ld(0)</code>) is not updated by the other expression. Also, note that this example is just to show how internal variables are kept between frames. A better approach to create this animation is to use the variables n or t: <pre class="example-preformatted">circle (mod(n * 15, w)) 120 20 fill </pre>

</td></tr></table>

The function <code class="code">random(idx)</code> returns a pseudorandom value between <code class="code">0</code> and <code class="code">1</code>. <var class="var">idx</var> is the internal variable that is used both as the seed and to keep the state of the number generator. <table class="cartouche"><tr><th> Example</th></tr><tr><td> The next example uses <code class="code">random(0)</code> to generate a random value for the center of a circle: <pre class="example-preformatted">circle (random(0) * w) (random(0) * h) 10

fill </pre>

The circle in every frame is at a different position, but always on the diagonal line of the frame. This happens because the values for the coordinates X and Y are identical, since both number generators use the same seed. To distribute the circles over the whole frame we need different seeds for each expression. This can be achieved by writing a non-zero value (like <code class="code">0xF0F0</code>) to the internal variable of one of expressions, but only when its value is <code class="code">0</code>:   <pre class="example-preformatted">circle (random(0) * w) (st(0, if(ld(0), ld(0), 0xF0F0)); random(0) * h) 10

fill </pre>

This approach is only useful if we need completely different positions in each frame. In the next example, random values are used to distribute many circles over the frame, but the position is fixed. The only change over time is the fill color: <pre class="example-preformatted">repeat 20 { circle (st(0, i + 1e5); random(0) * w) (st(0, i + 1e10); random(0) * h) 10 }

sethsla (t * 60) 0.5 0.5 1 preserve fill

setcolor black@0.5 setlinewidth 1 stroke </pre>

This is achieved by using a precomputed state before calling <code class="code">random(0)</code>. The variable <var class="var">i</var>, updated by <code class="code">repeat</code>, is needed to compute different states in each iteration. </td></tr></table>

The <code class=”code”>randomg(idx)</code> function, which is specific to drawvg scripts, is similar to <code class=”code”>random(idx)</code>, but intended to solve the previous problems: <ul class=”itemize mark-bullet”> All frames have the same seed. The state is shared between expressions.

The parameter <var class=”var”>idx</var> has two uses: <ul class=”itemize mark-bullet”> The last two bits are the index of an internal state, so it is possible to have 4 different number generators.

The first call to <code class=”code”>randomg</code> with a specific index will use the argument as the seed for the number generator in that index.

In a script like this: <pre class=”example-preformatted”>M (randomg(0xFF1)) (randomg(0xFF0)) l (randomg(0xAA1)) (randomg(0xFF0)) </pre>

There are 4 calls to <code class=”code”>randomg</code>: <ol class=”enumerate”> The first call, with the argument <code class=”code”>0xFF1</code>, uses the internal state at index <code class=”code”>1</code> (because <code class=”code”>0xFF1</code> modulo <code class=”code”>4</code> is <code class=”code”>1</code>).

Since this is the first use of that index, the number generator is initialized with the seed <code class=”code”>0xFF1</code>. The second call has the same behaviour: it initializes the state at index <code class=”code”>0</code> with the value <code class=”code”>0xFF0</code>.

 The third call has the argument <code class=”code”>0xAA1</code>, and it uses index <code class=”code”>1</code>. Since that state is already initialized (with the seed <code class=”code”>0xFF1</code>), the value <code class=”code”>0xAA1</code> is ignored, and it returns the next number.

<table class=”cartouche”><tr><th> Example</th></tr><tr><td> This example renders a simple rain animation, moving lines from top to bottom. <code class=”code”>randomg</code> is used to distribute the lines over the frame, and to apply different speeds to each one.   <pre class=”example-preformatted”>rect 0 0 w h setcolor midnightblue fill

setcolor white

repeat 50 { setvar offset (t * (randomg(0) + 1))

moveto (mod(randomg(0) + offset / 6, 1) * w) (mod(randomg(0) + offset, 1) * h)

rlineto 6 36

setlinewidth (randomg(1) / 2 + 0.2) stroke } </pre>

</td></tr></table>

 <h3 class="section">3.10 Tracing with <code class="code">print</code>#
TOC</h3>

It is possible to trace the execution of a drawvg script by printing the value of an expression, either with the <code class="code">print</code> command, or with the print function. In both cases, the values are written to the FFmpeg log. Printing expressions may have a noticeable impact on the performance, so it is preferable to use it only when necessary. <h4 class="subsection">3.10.1 Function print# TOC</h4>

The function <code class="code">print(t)</code> writes the value of t, and returns its argument. <table class="cartouche"><tr><th> Example</th></tr><tr><td> Given a line line this: <pre class="example-preformatted">M (sin(2 * PI * t) * w) 0 </pre>

We can see the values of <code class="code">sin(2 * PI * t)</code> by surrounding it with a call to <code class="code">print()</code>:   <pre class="example-preformatted">M (print(sin(2 * PI * t)) * w) 0 </pre>

Executing this script with a 1 second / 8 FPS video shows the expected values for the sine function. <pre class="verbatim">$ ffmpeg \ -f lavfi \ -i 'color=r=8:d=1, drawvg=M (print(sin(2 * PI * t)) * w) 0' \ -f null /dev/null \ |& grep 'Eval @'

[Eval @ 0x7f500f502d20] 0.000000 [Eval @ 0x7f4ff784b420] 0.707107 [Eval @ 0x7f4ff784ba20] 1.000000 [Eval @ 0x7f4ff784c020] 0.707107 [Eval @ 0x7f4ff784c620] 0.000000 [Eval @ 0x7f4ff784cc20] -0.707107 [Eval @ 0x7f4ff784d220] -1.000000 [Eval @ 0x7f4ff784d820] -0.707107 </pre> </td></tr></table>

 <h4 class="subsection">3.10.2 Command <code class="code">print</code># TOC</h4>

The command <code class="code">print</code> accepts an arbitrary number of arguments, and for each one it writes:

<ul class=”itemize mark-bullet”> The source location (line and column). The source code of the expression. The result of evaluating the expression.

When there are multiple expressions, they are separated by the <code class=”code”>|</code> character. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> The next script prints the position of the current point after the <code class=”code”>l</code> command: <pre class=”example-preformatted”>M 10 20 l 100 100 print cx cy stroke </pre>

For each frame, it produces this output: <pre class=”verbatim”>[3:7] cx = 110.000000 | [3:10] cy = 120.000000 </pre> The next example prints the values of <code class=”code”>random(0)</code>: <pre class=”verbatim”>$ ffmpeg \ -f lavfi \ -i 'color=r=8:d=1, drawvg=print (random(0))' \ -f null /dev/null \ |& grep 'drawvg @'

[drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.229731 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.959813 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.071676 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.044600 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.134127 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.320513 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.857675 [drawvg @ 0x50a000000180] [1:7] (random(0)) = 0.562456 </pre> </td></tr></table>

 <h2 class=”chapter”>4 Commands# TOC</h2>

 <h3 class=”section”>4.1 <code class=”code”>arc</code># TOC</h3>

<code class=”code”>arc <var class=”var”>xc</var> <var class=”var”>yc</var> <var class=”var”>radius</var> <var class=”var”>angle1</var> <var class=”var”>angle2</var></code>   — Can be implicit Adds a circular arc of the given <var class=”var”>radius</var> to the current path. The arc is centered at <var class=”var”>xc, yc</var>, begins at <var class=”var”>angle1</var> and proceeds in the direction of increasing angles to end at <var class=”var”>angle2</var>. If there is a current point, a line is added from it to the beginning of the arc. If this is not desired, use <code class=”code”>newpath</code> before <code class=”code”>arc</code> to clear the current point.

See the documentation of the <code class=”code”>cairo_arc</code> function for more details. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”example-preformatted”>arc 120 120 60 0 (3 * PI / 2) stroke </pre>

</td></tr></table>

 <h3 class=”section”>4.2 <code class=”code”>arcn</code># TOC</h3>

<code class=”code”>arcn <var class=”var”>xc</var> <var class=”var”>yc</var> <var class=”var”>radius</var> <var class=”var”>angle1</var> <var class=”var”>angle2</var></code>   — Can be implicit Similar to <code class=”code”>arc</code>, but it differs in the direction of the arc between the two angles. See the documentation of the <code class=”code”>cairo_arc_negative</code> function for more details. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, both <code class=”code”>arc</code> and <code class=”code”>arcn</code> have the same angles, but they render different arcs: <pre class=”example-preformatted”>arc 120 90 60 (PI / 2) 0

newpath arcn 120 150 60 (PI / 2) 0

stroke </pre>

</td></tr></table>

<code class=”code”>newpath</code> is needed to prevent a line between the two arcs. <h3 class=”section”>4.3 <code class=”code”>break</code># TOC</h3>

<code class=”code”>break</code> <code class=”code”>break</code> terminates the execution of the innermost block, either a <code class=”code”>repeat</code> loop or a procedure. If it is used outside of a <code class=”code”>repeat</code> / <code class=”code”>proc</code>

block, it terminates the script for the current frame. <h3 class=”section”>4.4 <code class=”code”>call</code># TOC</h3>

<code class=”code”>call <var class=”var”>name</var> <var class=”var”>args</var>*</code> Invokes a procedure defined by <code class=”code”>proc</code>. See the Procedures section above for more details. <h3 class=”section”>4.5 <code class=”code”>circle</code># TOC</h3>

<code class=”code”>circle <var class=”var”>xc</var> <var class=”var”>yc</var> <var class=”var”>radius</var></code>   — Can be implicit Adds a circle of the given <var class=”var”>radius</var> to the current path. The circle is centered at <var class=”var”>xc, yc</var>. The current point is cleared before and after adding the circle. This is a convenience wrapper for <code class=”code”>arc</code>. A call to <code class=”code”>circle</code> is equivalent to: <pre class=”example-preformatted”>newpath arc xc yc radius (0) (2 * PI) newpath </pre>

 <h3 class=”section”>4.6 <code class=”code”>clip</code>, <code class=”code”>eoclip</code># TOC</h3>

<code class=”code”>clip, eoclip</code> Establishes a new clip region by intersecting the current clip region with the current path as it would be filled by <code class=”code”>fill</code> or <code class=”code”>eofill</code>. <code class=”code”>eoclip</code> uses the even–odd rule. See fill rules for more details. The path is cleared after updating the clip region, unless the <code class=”code”>preserve</code> command is used before <code class=”code”>clip</code> or <a

class=”ref” href=”#cmd_005feoclip”><code class=”code”>eoclip</code>.
 See the documentation of the <code class=”code”>cairo_clip</code> function for more details. <h3 class=”section”>4.7 <code class=”code”>Z</code>, <code class=”code”>z</code>, <code class=”code”>closepath</code># TOC</h3>

<code class=”code”>Z, z, closepath</code>
 Adds a line segment to the path from the current point to the beginning of the current sub-path, and closes this sub-path. The beginning is set by any of the move commands (<code class=”code”>M</code>, <code class=”code”>m</code>, <code class=”code”>moveto</code>, <code class=”code”>rmoveto</code>).
 See the documentation of the <code class=”code”>cairo_close_path</code> function for more details. <h3 class=”section”>4.8 <code class=”code”>colorstop</code># TOC</h3>

<code class=”code”>colorstop <var class=”var”>offset</var> <var class=”var”>color</var></code>   — Can be implicit Adds a color stop to a gradient pattern.
 <var class=”var”>offset</var> is a value between <code class=”code”>0</code> and <code class=”code”>1</code>, and it specifies the location along the gradient’s control vector. This command must be executed after <code class=”code”>lineargrad</code> or <code class=”code”>radialgrad</code>. Color stops can be added in any number of calls to <code class=”code”>colorstop</code>. In the next example, the 3 blocks define the same gradient: <pre class=”example-preformatted”>// 1 colorstop 0.0 red colorstop 0.5 green colorstop 1.0 blue

// 2 colorstop 0 red 0.5 green colorstop 1 blue

// 3 colorstop 0 red 0.5 green 1 blue </pre>

See the documentation of the <code class=”code”>cairo_pattern_add_color_sto function for more details. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example, color stops are added in a <code class=”code”>repeat</code> loop.   <pre class=”example-preformatted”>lineargrad 0 0 w h

repeat 6 { defhsla s (i * 60) 0.8 0.5 1 colorstop (i / 5) s }

rect 0 0 w h fill </pre>

It is possible to avoid transitions between color stops by repeating the same color in two stops: <pre class=”example-preformatted”>lineargrad 0 0 w h

repeat 6 { defhsla s (i * 60) 0.8 0.5 1 colorstop (i / 5) s colorstop ((i + 1) / 5) s }

rect 0 0 w h fill </pre>

</td></tr></table>

 <h3 class=”section”>4.9 <code class=”code”>C</code>, <code class=”code”>curveto</code># TOC</h3>

<code class=”code”>C, curveto <var class=”var”>x1</var> <var class=”var”>y1</var> <var class=”var”>x2</var> <var class=”var”>y2</var> <var class=”var”>x</var> <var class=”var”>y</var></code>   — Can be implicit Draw a cubic Bézier curve from the current point to the end point specified by <var class=”var”>x, y</var>. The start control point is specified by <var class=”var”>x1, y1</var> and the end control point is specified by <var class=”var”>x2, y2</var>.

The behaviour is identical to the <code class=”code”>C</code> command in SVG’s <code class=”code”><path></code></a For more details, see Cubic Bézier Curve on MDN, and the Curve Commands section of the Paths tutorial on MDN. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”example-preformatted”>moveto 20 20

curveto 0 (h / 2) // start control point w (h / 2) // end control point (w - 20)
(h - 20) // end point

stroke </pre>

</td></tr></table>

<a
name="c_002c-rcurveto"> <h3 class="section">4.10 <code class="code">c</code>,
<code class="code">rcurveto</code><a class="anchor
hidden-xs" href="#c_002c-rcurveto" aria-hidden="true"># <a
class="anchor hidden-xs"href="#toc-c_002c-rcurveto" aria-hidden="true">TOC</h3>

<code class="code">c, rcurveto <var class="var">dx1</var>
<var class="var">dy1</var> <var class="var">dx2</var> <var class="var">dy2</var>
<var class="var">dx</var> <var class="var">dy</var></code>
   — <i
class="i">Can be implicit</i> Like <a class="ref"
href="#cmd_005fcurveto"><code class="code">curveto</code>,
but the coordinates are relative to the <a class="ref" href="#current-
point">current point. <a
name="defhsla"> <h3 class="section">4.11 <code class="code">defhsla</code><span
class="pull-right"><a class="anchor hidden-xs" href="#defhsla" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-defhsla"
aria-hidden="true">TOC</h3>

<code class="code">defhsla varname <var class="var">h</var>
<var class="var">s</var> <var class="var">l</var> <var class="var">a</var></code>
 Similar to <code
class="code">sethsla</code>, but instead of establishing the
color for stroke and fill operations, the computed color is assigned
to the variable <var class="var">varname</var>. <var
class="var">varname</var> can then be used as a color for <a class="ref"
href="#cmd_005fsetcolor"><code class="code">setcolor</code> and
<code class="code">colorstop</code>.
 See <code class="code">sethsla</code>
for more details on how the color is computed. <a class="anchor"
id="cmd_005fdefrgba"> <h3 class="section">4.12
<code class="code">defrgba</code><a class="anchor
hidden-xs" href="#defrgba" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-defrgba" aria-hidden="true">TOC</h3>

<code class="code">defrgba varname <var class="var">r</var>
<var class="var">g</var> <var class="var">b</var> <var class="var">a</var></code>
 Computes a color from the red,
green, blue, and
alpha components, and assigns it to the
variable <var class="var">varname</var>. All compo-
nents are values between <code class="code">0</code> and <code

class=”code”>1</code>. Values outside that range are clamped to it. <h3 class=”section”>4.13 <code class=”code”>ellipse</code># TOC</h3>

<code class=”code”>ellipse <var class=”var”>cx</var> <var class=”var”>cy</var> <var class=”var”>rx</var> <var class=”var”>ry</var></code>   — Can be implicit Adds an ellipse to the current path. Similar to <code class=”code”>circle</code>, but it is possible to use different radius for both axes. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”verbatim”>ellipse 120 120 75 50 stroke </pre> </td></tr></table>

 <h3 class=”section”>4.14 <code class=”code”>fill</code>, <code class=”code”>eofill</code># TOC</h3>

<code class=”code”>fill, eofill</code> Fill the current path, using the current pattern (either a solid color or a gradient). <code class=”code”>eofill</code> uses the even–odd rule. See fill rules for more details. The path is cleared after the operation, unless the <code class=”code”>preserve</code> command is used before <code class=”code”>fill</code> or <code class=”code”>eofill</code>. See the documentation of the <code class=”code”>cairo_fill</code> function for more details. <h3 class=”section”>4.15 <code class=”code”>getmetadata</code># TOC</h3>

<code class=”code”>getmetadata <var class=”var”>varname</var> <var class=”var”>key</var></code> Get the value of a metadata entry created by another filter, and assign it to the variable <var class=”var”>varname</var>. If there is no metadata entry for <var class=”var”>key</var>, or its value is not a

number, <var class="var">varname</var> is set to <code class="code">NaN</code>. See the Frame Metadata section above for an example. <h3 class="section">4.16 <code class="code">H</code>, <code class="code">h</code># TOC</h3>

<code class="code">H, h <var class="var">x</var></code>   — Can be implicit Draw a horizontal line from the current point to x. The coordinate for <code class="code">H</code> is absolute, and for <code class="code">h</code> it is relative to the current point. <h3 class="section">4.17 <code class="code">if</code># TOC</h3>

<code class="code">if <var class="var">condition</var> { <var class="var">block</var> }</code> Executes a block if the value of <var class="var">condition</var> is not zero, and a finite number (unlike <code class="code">NaN</code>). See the Comparison and Logical Operators section above for more details on how to write conditional expressions. <h3 class="section">4.18 <code class="code">lineargrad</code># TOC</h3>

<code class="code">lineargrad <var class="var">x0</var> <var class="var">y0</var> <var class="var">x1</var> <var class="var">y1</var></code> Set the current pattern to a new linear gradient, along the line from the coordinates <var class="var">x0, y0</var> to <var class="var">x1, y1</var>. This gradient can be used for stroke and fill operations. Use <code class="code">colorstop</code> to set the color for each position in the gradient. <h3 class="section">4.19 <code class="code">L</code>, <code class="code">lineto</code><a class="anchor hidden-xs" href="#L_002c-lineto" aria-

hidden=”true”># TOC</h3>

<code class=”code”>L, lineto <var class=”var”>x</var> <var class=”var”>y</var></code>   — Can be implicit Draw a line from the current point to the coordinates at <var class=”var”>x, y</var>. See the documentation of the <code class=”code”>cairo_line_to</code> function for more details. <h3 class=”section”>4.20 <code class=”code”>l</code>, <code class=”code”>rlineto</code># TOC</h3>

<code class=”code”>l, rlineto <var class=”var”>dx</var> <var class=”var”>dy</var></code>   — Can be implicit Like <code class=”code”>lineto</code>, but the coordinates are relative to the current point. <h3 class=”section”>4.21 <code class=”code”>M</code>, <code class=”code”>moveto</code># TOC</h3>

<code class=”code”>M, moveto <var class=”var”>x</var> <var class=”var”>y</var></code>   — Can be implicit Begin a new sub-path, and set the current point to <var class=”var”>x, y</var>. <h3 class=”section”>4.22 <code class=”code”>m</code>, <code class=”code”>rmoveto</code># TOC</h3>

<code class=”code”>m, rmoveto <var class=”var”>dx</var> <var class=”var”>dy</var></code>   — Can be implicit Like <code class=”code”>moveto</code>, but the coordinates are relative to the current point. <a

class=”anchor” id=”cmd_005fnewpath”>
<h3 class=”section”>4.23 <code class=”code”>newpath</code><span
class=”pull-right”><a class=”anchor hidden-xs” href=”#newpath” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-newpath”
aria-hidden=”true”>TOC</h3>

<code class=”code”>newpath</code>
Begin a new sub-path. Like <code
class=”code”>moveto</code>, but there is no <a class=”ref”
href=”#current-point”>current point after it. <table class=”cartouche”><tr><th>
Example</th></tr><tr><td> In the next example, <a class=”ref”
href=”#cmd_005fnewpath”><code class=”code”>newpath</code> is
used in the path on the right to prevent the line connecting both arcs.
<pre class=”verbatim”>setlinewidth 3

setcolor skyblue arcn 70 90 20 0 (PI) arc 70 150 20 0 (PI) stroke

setcolor seagreen arcn 170 90 20 0 (PI) newpath arc 170 150 20 0 (PI) stroke
</pre> </td></tr></table>

<h3 class=”section”>4.24 <code class=”code”>preserve</code><span
class=”pull-right”><a class=”anchor hidden-xs” href=”#preserve” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-preserve”
aria-hidden=”true”>TOC</h3>

<code class=”code”>preserve</code>
Indicates that the next operation to fill, stroke, or clip, must preserve the
path, so the same path can be used in multiple operations. It has
effect on these commands: <ul class=”itemize mark-bullet”> <a
class=”ref” href=”#cmd_005fclip”><code class=”code”>clip</code>
<code class=”code”>eoclip</code>
<code class=”code”>eofill</code>
<code class=”code”>fill</code>
<code class=”code”>stroke</code>

The script can contain any command between <code
class=”code”>preserve</code> and the associated operation. This al-
lows modifying other properties, like the current color. <table
class=”cartouche”><tr><th> Example</th></tr><tr><td> In this
example, the same path is used for both <code
class=”code”>fill</code> and <code
class=”code”>stroke</code>, but with different colors. <pre
class=”verbatim”>circle (w / 2) (h / 2) (w / 3)

setcolor skyblue preserve fill

setlinewidth 10 setcolor seagreen stroke </pre> </td></tr></table>

<code class="code">preserve</code> can be called multiple times, if the same path has to be used in 3 or more operations. <table class="cartouche"><tr><th> Example</th></tr><tr><td> In this example, the path created by <code class="code">circle</code> is used by <code class="code">fill</code>, <code class="code">stroke</code>, and <code class="code">clip</code>. After <code class="code">clip</code>, the path is cleared. <pre class="verbatim">circle 100 100 50

preserve fill preserve stroke clip </pre> </td></tr></table>

 <h3 class="section">4.25 <code class="code">print</code># TOC</h3>

<code class="code">print <var class="var">expr</var></code>   — Can be implicit Print its arguments to the FFmpeg log. See the Command print section above for more details. <h3 class="section">4.26 <code class="code">proc</code># TOC</h3>

<code class="code">proc <var class="var">name</var> <var class="var">params</var>* { <var class="var">block</var> }</code> Assign the block and the parameters for the procedure <var class="var">name</var>. The procedure can be called multiple times with the <code class="code">call</code> command. See the Procedures section above for more details. <h3 class="section">4.27 <code class="code">Q</code># TOC</h3>

<code class="code">Q x1 y1 <var class="var">x</var> <var class="var">y</var></code> Draw a quadratic Bézier curve from the current point to the end point specified by <var class="var">x, y</var>. The control point is specified by <var class="var">x1, y1</var>.

The behaviour is identical to the <code class="code">Q</code>

command in SVG’s <code class=”code”><path></code>
For more details, see Quadratic Bézier curve on MDN, and the Curve Commands section of the Paths tutorial on MDN. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”verbatim”>moveto 20 20

Q 0 h // control point (w - 20) (h - 20) // end point

stroke </pre> </td></tr></table>

 <h3 class=”section”>4.28 <code class=”code”>q</code># TOC</h3>

<code class=”code”>q <var class=”var”>dx1</var> <var class=”var”>dy1</var> <var class=”var”>dx</var> <var class=”var”>dy</var></code> Like <code class=”code”>Q</code>, but the coordinates are relative to the current point. <h3 class=”section”>4.29 <code class=”code”>radialgrad</code># TOC</h3>

<code class=”code”>radialgrad <var class=”var”>cx0</var> <var class=”var”>cy0</var> <var class=”var”>radius0</var> <var class=”var”>cx1</var> <var class=”var”>cy1</var> <var class=”var”>radius1</var></code> Creates a new radial gradient between the two circles defined by <var class=”var”>cx0 cy0 radius0</var> and <var class=”var”>cx1 cy1 radius1</var>. Each set of arguments is the coordinates of the center and the radius. This gradient can be used for stroke and fill operations. Use <code class=”code”>colorstop</code> to set the color for each position in the gradient. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> The animation in the next example shows how the two circles defined in the <code class=”code”>radialgrad</code> arguments interact with each other. The red circle represent the circle for the <var class=”var”>cx0 cy0 radius0</var> arguments, and the yellow circle is the one for the <var class=”var”>cx1 cy1 radius1</var> arguments. <pre class=”verbatim”>setvar cx0 (mod(t * 30, w)) setvar cy0 120 setvar radius0 20

setvar cx1 120 setvar cy1 120 setvar radius1 70

radialgrad cx0 cy0 radius0 cx1 cy1 radius1

colorstop 0 lightblue 1 darkblue

// Fill the frame with the gradient. rect 0 0 w h fill

// Draw inner circle. circle cx0 cy0 radius0 setcolor red stroke

// Draw outer circle. circle cx1 cy1 radius1 setcolor yellow stroke </pre>
</td></tr></table>

 <h3
class="section">4.30 <code class="code">rect</code><span class="pull-
right">#
TOC</h3>

<code class="code">rect <var class="var">x</var> <var
class="var">y</var> <var class="var">width</var> <var class="var">height</var></code>
 Adds a rectangle of the given size (<var class="var">width</var> ×
<var class="var">height</var>), at position <var class="var">x, y</var>,
to the current path. The current
point is cleared before and after adding the rectangle. See the
documentation of the <a class="uref" href="https://www.cairographics.org/manual/cairo-
Paths.html#cairo-rectangle"><code class="code">cairo_rectangle</code>
function for more details. <a
name="repeat"> <h3 class="section">4.31 <code class="code">repeat</code><span
class="pull-right"><a class="anchor hidden-xs" href="#repeat" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-repeat"
aria-hidden="true">TOC</h3>

<code class="code">repeat <var class="var">count</var>
{ <var class="var">block</var> }</code> Executes a block
the number of times indicated by <var class="var">count</var>.
In each iteration, the variable <var class="var">i</var> is used as a <a
class="uref" href="https://en.wikipedia.org/wiki/For_loop#Loop_counters">loop
counter. It takes the values from <code class="code">0</code> to
<code class="code">count - 1</code>. When the loop is terminated, the
variable is restored to the value before starting the loop. If <var
class="var">count</var> is less than <code class="code">1</code>, or it is
not a finite number (like <code
class="code">NaN</code>), the block is not executed. <a
class="anchor" id="cmd_005fresetclip">
<h3 class="section">4.32 <code class="code">resetclip</code><span
class="pull-right"><a class="anchor hidden-xs" href="#resetclip" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-resetclip"
aria-hidden="true">TOC</h3>

<code class="code">resetclip</code>
Reset the current clip region to its original state, covering the
whole frame. See the documentation of the <a class="uref"

href=”https://www.cairographics.org/manual/cairo-cairo-t.html#cairo-reset-clip”><code class=”code”>cairo_reset_clip</code> function for more details. <h3 class=”section”>4.33 <code class=”code”>resetdash</code># TOC</h3>

<code class=”code”>resetdash</code> Disable the dash pattern to be used by <code class=”code”>stroke</code>. This reverts any change made by <code class=”code”>setdash</code> and <code class=”code”>setdashoffset</code>. It calls <code class=”code”>cairo_set_dash</code> with <code class=”code”>num_dashes</code> set to <code class=”code”>0</code>. <h3 class=”section”>4.34 <code class=”code”>resetmatrix</code># TOC</h3>

<code class=”code”>resetmatrix</code> Resets the current transformation matrix. <h3 class=”section”>4.35 <code class=”code”>restore</code># TOC</h3>

<code class=”code”>restore</code> Restores the state saved by a preceding call to <code class=”code”>save</code>. For more details, see the State Stack section above, and the <code class=”code”>cairo_restore</code> function. <h3 class=”section”>4.36 <code class=”code”>rotate</code># TOC</h3>

<code class=”code”>rotate <var class=”var”>angle</var></code> Modifies the current transformation matrix by rotating the user-space axes by <var class=”var”>angle</var> radians. See the documentation of the <code class=”code”>cairo_rotate</code>

function for more details. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> In this example: <ul class=”itemize mark-bullet”> <code class=”code”>scalexy</code> maps the coordinates to a 1x1 frame. <code class=”code”>translate</code> put <code class=”code”>0, 0</code> at the center of the frame. <code class=”code”>rotate</code> rotates 45°. <code class=”code”>resetmatrix</code> reverts the transformations before <code class=”code”>stroke</code>, so the line width is not affected by the scale.

<pre class=”verbatim”>scalexy w h translate 0.5 0.5 rotate (PI / 4) rect -0.25 -0.25 0.5 0.5 resetmatrix stroke </pre> </td></tr></table>

 <h3 class=”section”>4.37 <code class=”code”>roundedrect</code># TOC</h3>

<code class=”code”>roundedrect <var class=”var”>x</var> <var class=”var”>y</var> <var class=”var”>width</var> <var class=”var”>height</var> <var class=”var”>radius</var></code>   — Can be implicit Like <code class=”code”>rect</code>, but a circular arc is used for the corners. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> The next example shows the same rectangle, with different values for the corner radius. The radius is computed by multiplying <var class=”var”>i</var> (the counter) by <code class=”code”>4.5</code>. This number is chosen to make the last shape a perfect circle.   <pre class=”example-preformatted”>repeat 9 { roundedrect (mod(i, 3) * 80 + 5) // x (floor(i / 3) * 80 + 5) // y 70 70 // size (i * 4.5) // radius }

stroke </pre>

</td></tr></table>

 <h3 class=”section”>4.38 <code class=”code”>save</code># TOC</h3>

<code class=”code”>save</code> Saves a copy of the current state on an internal stack. This copy can be restored later with <code

class=”code”>restore</code>. For more details, see the State Stack section above, and the <code class=”code”>cairo_save</code> function. <h3 class=”section”>4.39 <code class=”code”>scale</code># TOC</h3>

<code class=”code”>scale <var class=”var”>sxy</var></code> Similar to <code class=”code”>scalexy</code>, but the same value is used for both axes. It is equivalent to: <code class=”code”>scalexy <var class=”var”>sxy</var> <var class=”var”>sxy</var></code> <h3 class=”section”>4.40 <code class=”code”>scalexy</code># TOC</h3>

<code class=”code”>scalexy <var class=”var”>sx</var> <var class=”var”>sy</var></code> Modifies the current transformation matrix by scaling the X and Y user-space axes by <var class=”var”>sx</var> and <var class=”var”>sy</var> respectively. See the documentation of the <code class=”code”>cairo_scale</code> function for more details. See <code class=”code”>rotate</code> for an example on combining multiple transformations. <h3 class=”section”>4.41 <code class=”code”>setcolor</code># TOC</h3>

<code class=”code”>setcolor <var class=”var”>color</var></code> Set a solid color as the current pattern for stroke and fill operations See the Colors section above for more details. <h3 class=”section”>4.42 <code class=”code”>setdash</code># TOC</h3>

<code class=”code”>setdash <var class=”var”>length</var></code>   — Can be implicit Sets the dash pat-

tern to be used by <code class="code">stroke</code>. Each call to <code class="code">setdash</code> adds a length to the pattern, alternating between on and off portions of the stroke. After a call to <code class="code">setdash</code>, <code class="code">resetdash</code> is needed either to create a new pattern, or to discard the current one. See the documentation of the <code class="code">cairo_set_dash</code> function for more details. <h3 class="section">4.43 <code class="code">setdashoffset</code># TOC</h3>

<code class="code">setdashoffset <var class="var">offset</var></code> Set the offset into the dash pattern at which the stroke should start. <code class="code">setdash</code> must be called before <code class="code">setdashoffset</code>. See the documentation of the <code class="code">cairo_set_dash</code> function for more details. <table class="cartouche"><tr><th> Example</th></tr><tr><td> The next animation shows the effect of <code class="code">setdashoffset</code> when its argument changes over time. <pre class="verbatim">scalexy w h M 0.5 1 curveto 0 0.5, 1 0.5, 0.5 0 resetmatrix

setdash 20 5 // 20 on, 5 off setdashoffset (t * 100)

setlinewidth 20 stroke </pre> </td></tr></table>

 <h3 class="section">4.44 <code class="code">sethsla</code># TOC</h3>

<code class="code">sethsla <var class="var">h</var> <var class="var">s</var> <var class="var">l</var> <var class="var">a</var></code> Set the current pattern to a solid color, given the hue, saturation, and lightness, and alpha components. h is the hue, a value between <code class="code">0</code> and <code class="code">359</code>. Negative values are clamped to <code class="code">0</code>, and values greater than <code

class=”code”>359</code> are interpreted as modulo 360. s (saturation), l (lightness), and a (alpha), are values between <code class=”code”>0</code> and <code class=”code”>1</code>. The conversion to RGB is implemented according to the formulae from Wikipedia. <h3 class=”section”>4.45 <code class=”code”>setlinecap</code># TOC</h3>

<code class=”code”>setlinecap <var class=”var”>cap</var></code> Set the current line cap style, which determines the shape used to draw the end points of lines. <var class=”var”>cap</var> must be one of the following names: <ul class=”itemize mark-bullet”> <code class=”code”>butt</code> <code class=”code”>round</code> <code class=”code”>square</code>

It calls to <code class=”code”>cairo_set_line_cap</code> to set the line cap style. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> This example draws 3 lines with the same length, each one with a different line cap style: <pre class=”verbatim”>setlinewidth 40

setlinecap butt setcolor tomato M 60 40 v 100 stroke

setlinecap round setcolor seagreen M 120 40 v 100 stroke

setlinecap square setcolor skyblue M 180 40 v 100 stroke

M 20 40 H 220 m 0 100 H 20 setcolor black@0.5 setlinewidth 2 stroke </pre> </td></tr></table>

 <h3 class=”section”>4.46 <code class=”code”>setlinejoin</code># TOC</h3>

<code class=”code”>setlinejoin <var class=”var”>join</var></code> Sets the current line join style, which determines the shape used to join two line segments. <var class=”var”>join</var> must be one of the following names: <ul class=”itemize mark-bullet”> <code class=”code”>bevel</code> <code class=”code”>miter</code> <code class=”code”>round</code>

It calls to <code class=”code”>cairo_set_line_join</code>

to set the line join style. <table class="cartouche"><tr><th> Example</th></tr><tr><td> This example draws 3 lines with the same length, each one with a different line join style: <pre class="verbatim">setlinewidth 30

setlinejoin bevel setcolor tomato M 70 20 l 50 50 50 -50 stroke

setlinejoin miter setcolor seagreen M 70 90 l 50 50 50 -50 stroke

setlinejoin round setcolor skyblue M 70 160 l 50 50 50 -50 stroke </pre> </td></tr></table>

 <h3 class="section">4.47 <code class="code">setlinewidth</code># TOC</h3>

<code class="code">setlinewidth <var class="var">width</var></code> Set the line width for <code class="code">stroke</code>. <var class="var">width</var> is affected by the transformation matrix. To specify a width that is not affected by other transformations, <code class="code">resetmatrix</code> can be used between <code class="code">save</code> / <code class="code">restore</code>: <pre class="verbatim">save

resetmatrix setlinewidth 1 stroke

// Restore matrix after stroke. restore </pre> See the documentation of the <code class="code">cairo_set_line_width</code> function for more details. <h3 class="section">4.48 <code class="code">setrgba</code># TOC</h3>

<code class="code">setrgba <var class="var">r</var> <var class="var">g</var> <var class="var">b</var> <var class="var">a</var></code> Set the current pattern to a solid color, given the red, green, blue, and alpha components. All components are values between <code class="code">0</code> and <code class="code">1</code>. Values outside that range are clamped to it. <h3 class="section">4.49 <code class="code">setvar</code><span

class=”pull-right”># TOC</h3>

<code class=”code”>setvar <var class=”var”>varname</var> <var class=”var”>value</var></code> Set the variable <var class=”var”>varname</var> to <var class=”var”>value</var>. See the User Variables section above for more details. <h3 class=”section”>4.50 <code class=”code”>stroke</code># TOC</h3>

<code class=”code”>stroke</code> Strokes the current path according to the current line width, line join, line cap, and dash settings. The path is cleared after the operation, unless the <code class=”code”>preserve</code> command is used before <code class=”code”>stroke</code>. See the documentation of the <code class=”code”>cairo_stroke</code> function for more details. <h3 class=”section”>4.51 <code class=”code”>S</code>, <code class=”code”>s</code># TOC</h3>

<code class=”code”>S, s <var class=”var”>x2</var> <var class=”var”>y2</var> <var class=”var”>x</var> <var class=”var”>y</var></code>   — Can be implicit Draw a smooth cubic Bézier curve from the current point to the end point specified by <var class=”var”>x, y</var>. The end control point is specified by <var class=”var”>x2, y2</var>. The start control point is the reflection of the end control point of the previous curve command about the current point. The behaviour is identical to the <code class=”code”>S</code> command in SVG’s <code class=”code”><path></code> For more details, see Cubic Bézier Curve on MDN, and the Curve

Commands section of the Paths tutorial on MDN. <code class=”code”>s</code> is like <code class=”code”>S</code>, but the coordinates are relative to the current point. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”example-preformatted”>M 20 120

c 25 -50, 25 50, 50 0

repeat 3 { s 20 50, 50 0 }

stroke </pre>

</td></tr></table>

 <h3 class=”section”>4.52 <code class=”code”>translate</code># TOC</h3>

<code class=”code”>translate <var class=”var”>tx</var> <var class=”var”>ty</var></code> Modifies the current transformation matrix by translating the user-space origin by <var class=”var”>tx, ty</var>. See the documentation of the <code class=”code”>cairo_translate</code> function for more details. <h3 class=”section”>4.53 <code class=”code”>T</code>, <code class=”code”>t</code># TOC</h3>

<code class=”code”>T, t <var class=”var”>x</var> <var class=”var”>y</var></code>   — Can be implicit Draw a smooth quadratic Bézier curve from the current point to the end point specified by <var class=”var”>x, y</var>. The control point is the reflection of the control point of the previous curve command about the current point. The behaviour is identical to the <code class=”code”>T</code> command in SVG’s <code class=”code”><path></code> For more details, see <a class=”uref” href=”https://developer.mozilla.org/en-

US/docs/Web/SVG/Reference/Attribute/d#quadratic_b%C3%A9zier_curve”>Quadratic Bézier curve on MDN, and the Curve Commands section of the Paths tutorial on MDN. <code class=”code”>t</code> is like <code class=”code”>T</code>, but the coordinates are relative to the current point. <table class=”cartouche”><tr><th> Example</th></tr><tr><td> <pre class=”example-preformatted”>M 20 120

q 10 -20, 20 0

repeat 9 { t 20 0 }

stroke </pre>

</td></tr></table>

 <h3 class=”section”>4.54 <code class=”code”>V</code>, <code class=”code”>v</code># TOC</h3>

<code class=”code”>V, v <var class=”var”>y</var></code>   — Can be implicit Draw a vertical line from the current point to y. The coordinate for <code class=”code”>V</code> is absolute, and for <code class=”code”>v</code> it is relative to the current point. <p style=”font-size: small;”> This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\faq.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset=”utf-8”> <title> FFmpeg FAQ </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”> <link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body> <h1> FFmpeg FAQ </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1 General Questions <ul class=”toc-numbered-mark”> 1.1 Why doesn’t FFmpeg support feature [xyz]? 1.2 FFmpeg does not support codec XXX. Can you include a Windows DLL loader to support it? 1.3 I cannot read this file although this format seems to be supported by ffmpeg. 1.4 Which codecs are supported by Windows? 2 Compilation <ul class=”toc-numbered-mark”> 2.1 <code class=”code”>error: can't find a register in class 'GENERAL_REGS' while reloading 'asm'</code> 2.2 I have installed this library with my distro’s package manager. Why does <code class=”command”>configure</code> not see it? 2.3 How do I make <code class=”command”>pkg-config</code> find my libraries? 2.4 How do I use <code class=”command”>pkg-config</code> when cross-compiling? 3 Usage <ul class=”toc-numbered-mark”> 3.1 ffmpeg does not work; what is wrong? 3.2 How do I encode single pictures into movies? 3.3 How do I encode movie to single pictures? <a id=”toc-Why-do-I-see-a-slight-quality-degradation-with-multithreaded-MPEG_002a-encoding_003f” href=”#Why-

do-I-see-a-slight-quality-degradation-with-multithreaded-MPEG_002a-encoding_003f">3.4 Why do I see a slight quality degradation with multi-threaded MPEG* encoding? 3.5 How can I read from the standard input or write to the standard output? 3.6 -f jpeg doesn’t work. 3.7 Why can I not change the frame rate? 3.8 How do I encode Xvid or DivX video with ffmpeg? 3.9 Which are good parameters for encoding high quality MPEG-4? 3.10 Which are good parameters for encoding high quality MPEG-1/MPEG-2? 3.11 Interlaced video looks very bad when encoded with ffmpeg, what is wrong? 3.12 How can I read DirectShow files? 3.13 How can I join video files? 3.14 How can I concatenate video files? <ul class="toc-numbered-mark"> 3.14.1 Concatenating using the concat filter 3.14.2 Concatenating using the concat demuxer 3.14.3 Concatenating using the concat protocol (file level) 3.14.4 Concatenating using raw audio and video 3.15 Using <samp class="option">-f lavfi</samp>, audio becomes mono for no apparent

reason. 3.16 Why does FFmpeg not see the subtitles in my VOB file? 3.17 Why was the <code class="command">ffmpeg</code> <samp class="option">-sameq</samp> option removed? What to use instead? 3.18 I have a stretched video, why does scaling not fix it? 3.19 How do I run ffmpeg as a background task? 3.20 How do I prevent ffmpeg from suspending with a message like suspended (tty output)? 4 Development <ul class="toc-numbered-mark"> 4.1 Are there examples illustrating how to use the FFmpeg libraries, particularly libavcodec and libavformat? 4.2 Can you support my C compiler XXX? 4.3 Is Microsoft Visual C++ supported? 4.4 Can you add automake, libtool or autoconf support? 4.5 Why not rewrite FFmpeg in object-oriented C++? 4.6 Why are the ffmpeg programs devoid of debugging symbols? 4.7 I do not like the LGPL, can I contribute code under the GPL instead? <a id="toc-I_0027m-using-FFmpeg-from-within-my-C-application-but-the-linker-complains-about-missing-symbols-from-the-libraries-themselves_002e" href="#I_0027m-using-FFmpeg-from-within-

my-C-application-but-the-linker-complains-about-missing-symbols-from-the-libraries-themselves_002e">4.8 I’m using FFmpeg from within my C application but the linker complains about missing symbols from the libraries themselves. 4.9 I’m using FFmpeg from within my C++ application but the linker complains about missing symbols which seem to be available. 4.10 I have a file in memory / a API different from *open/*read/ libc how do I use it with libavformat? 4.11 Where is the documentation about ffv1, msmpeg4, asv1, 4xm? 4.12 How do I feed H.263-RTP (and other codecs in RTP) to libavcodec? 4.13 AVStream.r_frame_rate is wrong, it is much larger than the frame rate. 4.14 Why is <code class="code">make fate</code> not running all tests? 4.15 Why is <code class="code">make fate</code> not finding the samples?

 <h2 class="chapter">1 General Questions# TOC</h2>

 <h3 class="section">1.1 Why doesn’t FFmpeg support feature [xyz]?# TOC</h3>

<p>Because no one has taken on that task yet. FFmpeg development is

driven by the tasks that are important to the individual developers. If there is a feature that is important to you, the best way to get it implemented is to undertake the task yourself or sponsor a developer. <h3 class="section">1.2 FFmpeg does not support codec XXX. Can you include a Windows DLL loader to support it?# TOC</h3>

No. Windows DLLs are not portable, bloated and often slow. Moreover FFmpeg strives to support all codecs natively. A DLL loader is not conducive to that goal. <h3 class="section">1.3 I cannot read this file although this format seems to be supported by ffmpeg.# TOC</h3>

Even if ffmpeg can read the container format, it may not support all its codecs. Please consult the supported codec list in the ffmpeg documentation. <h3 class="section">1.4 Which codecs are supported by Windows?# TOC</h3>

Windows does not support standard formats like MPEG very well, unless you install some additional codecs. The following list of video codecs should work on most Windows systems: <dl class="table"> <dt><samp class="option">msmpeg4v2</samp></dt> <dd> .avi/.asf </dd> <dt><samp class="option">msmpeg4</samp></dt> <dd> .asf only </dd> <dt><samp class="option">wmv1</samp></dt> <dd> .asf only </dd> <dt><samp class="option">wmv2</samp></dt> <dd> .asf only </dd> <dt><samp class="option">mpeg4</samp></dt> <dd> Only if you have some MPEG-4 codec like ffdshow or Xvid installed. </dd> <dt><samp class="option">mpeg1video</samp></dt> <dd> .mpg only </dd> </dl> Note, ASF files often have .wmv or .wma extensions in Windows. It should also be mentioned that Microsoft claims a patent on the ASF format, and may sue or threaten users who create ASF files with non-Microsoft software. It is strongly ad-

vised to avoid ASF where possible. The following list of audio codecs should work on most Windows systems: <dl class="table"> <dt><samp class="option">adpcm_ima_wav</samp></dt> <dt><samp class="option">adpcm_ms</samp></dt> <dt><samp class="option">pcm_s16le</samp></dt> <dd> always </dd> <dt><samp class="option">libmp3lame</samp></dt> <dd> If some MP3 codec like LAME is installed. </dd> </dl>

 <h2 class="chapter">2 Compilation# TOC</h2>

 <h3 class="section">2.1 <code class="code">error: can't find a register in class 'GENERAL_REGS' while reloading 'asm'</code># TOC</h3>

This is a bug in gcc. Do not report it to us. Instead, please report it to the gcc developers. Note that we will not add workarounds for gcc bugs. Also note that (some of) the gcc developers believe this is not a bug or not a bug they should fix: <a class="url" href="https://gcc.gnu.org/bugzilla/show_bug.cgi?id=11203">https://gcc.gnu.org/bugzilla/show_bug.cgi?id=
Then again, some of them do not know the difference between an undecidable problem and an NP-hard problem... <h3 class="section">2.2 I have installed this library with my distro’s package manager. Why does <code class="command">configure</code> not see it?# TOC</h3>

Distributions usually split libraries in several packages. The main package contains the files necessary to run programs using the library. The development package contains the files necessary to build programs using the library. Sometimes, docs and/or data are in a separate package too. To build FFmpeg, you need to install the development package. It is usually called <samp class="file">libfoo-dev</samp> or <samp class="file">libfoo-devel</samp>. You can remove it after the build is finished, but be sure to keep the main package. <h3 class="section">2.3 How do I make <code class="command">pkg-config</code> find my libraries?<span class="pull-

right”># TOC</h3>

Somewhere along with your libraries, there is a <samp class=”file”>.pc</samp> file (or several) in a <samp class=”file”>pkgconfig</samp> directory. You need to set environment variables to point <code class=”command”>pkg-config</code> to these files. If you need to add directories to <code class=”command”>pkg-config</code>’s search list (typical use case: library installed separately), add it to <code class=”code”>$PKG_CONFIG_PATH</code>:   <pre class=”example-preformatted”>export PKG_CONFIG_PATH=/opt/x264/lib/pkgco
</pre>

If you need to replace <code class=”command”>pkg-config</code>’s search list (typical use case: cross-compiling), set it in <code class=”code”>$PKG_CONFIG_LIBDIR</code>:   <pre class=”example-preformatted”>export PKG_CONFIG_LIBDIR=/home/me/cross/usr/lib/pkgconfig:/home/me/cross/usr/local/lib/pkgconfig
</pre>

If you need to know the library’s internal dependencies (typical use: static linking), add the <code class=”code”>--static</code> option to <code class=”command”>pkg-config</code>:   <pre class=”example-preformatted”>./configure --pkg-config-flags=--static
</pre>

 <h3 class=”section”>2.4 How do I use <code class=”command”>pkg-config</code> when cross-compiling?# TOC</h3>

The best way is to install <code class=”command”>pkg-config</code> in your cross-compilation environment. It will automatically use the cross-compilation libraries. You can also use <code class=”command”>pkg-config</code> from the host environment by specifying explicitly <code class=”code”>--pkg-config=pkg-config</code> to <code class=”command”>configure</code>. In that case, you must point <code class=”command”>pkg-config</code> to the correct directories using the <code class=”code”>PKG_CONFIG_LIBDIR</code>, as explained in the previous entry. As an intermediate solution, you can place in your cross-compilation environment a script that calls the host <code class=”command”>pkg-config</code> with <code class=”code”>PKG_CONFIG_LIBDIR</code> set. That script can look like that: <pre class=”example-preformatted”>#!/bin/sh PKG_CONFIG_LIBDIR=/path/to/cross/lib/pkgconfig

export PKG_CONFIG_LIBDIR exec /usr/bin/pkg-config "$@"
</pre>

 <h2 class="chapter">3 Usage#
TOC</h2>

 <h3 class="section">3.1 ffmpeg does not work; what is wrong?# TOC</h3>

Try a <code class="code">make distclean</code> in the ffmpeg source directory before the build. If this does not help see (https://ffmpeg.org/bugreports.html).

<h3 class="section">3.2 How do I encode single pictures into movies?# TOC</h3>

First, rename your pictures to follow a numerical sequence. For example, img1.jpg, img2.jpg, img3.jpg,... Then you may run: <pre class="example-preformatted">ffmpeg -f image2 -i img%d.jpg /tmp/a.mpg </pre>

Notice that ‘<samp class="samp">%d</samp>’ is replaced by the image number. <samp class="file">img%03d.jpg</samp> means the sequence <samp class="file">img001.jpg</samp>, <samp class="file">img002.jpg</samp>, etc. Use the <samp class="option">-start_number</samp> option to declare a starting number for the sequence. This is useful if your sequence does not start with <samp class="file">img001.jpg</samp> but is still in a numerical order. The following example will start with <samp class="file">img100.jpg</samp>:
  <pre class="example-preformatted">ffmpeg -f image2 -start_number 100 -i img%d.jpg /tmp/a.mpg </pre>

If you have large number of pictures to rename, you can use the following command to ease the burden. The command, using the bourne shell syntax, symbolically links all files in the current directory that match <code class="code">*jpg</code> to the <samp class="file">/tmp</samp> directory in the sequence of <samp class="file">img001.jpg</samp>, <samp class="file">img002.jpg</samp> and so on.
<pre class="example-preformatted">x=1; for i in *jpg; do counter=$(printf %03d $x); ln -s "$i" /tmp/img"$counter".jpg; x=$(($x+1)); done </pre>

If you want to sequence them by oldest modified first, substitute <code class=”code”>$(ls -r -t *jpg)</code> in place of <code class=”code”>*jpg</code>. Then run: <pre class=”example-preformatted”>ffmpeg -f image2 -i /tmp/img%03d.jpg /tmp/a.mpg </pre>

The same logic is used for any image format that ffmpeg reads. You can also use <code class=”command”>cat</code> to pipe images to ffmpeg: <pre class=”example-preformatted”>cat *.jpg | ffmpeg -f image2pipe -c:v mjpeg -i - output.mpg </pre>

 <h3 class=”section”>3.3 How do I encode movie to single pictures?# TOC</h3>

Use: <pre class=”example-preformatted”>ffmpeg -i movie.mpg movie%d.jpg </pre>

The <samp class=”file”>movie.mpg</samp> used as input will be converted to <samp class=”file”>movie1.jpg</samp>, <samp class=”file”>movie2.jpg</samp>, etc... Instead of relying on file format self-recognition, you may also use <dl class=”table”> <dt><samp class=”option”>-c:v ppm</samp></dt> <dt><samp class=”option”>-c:v png</samp></dt> <dt><samp class=”option”>-c:v mjpeg</samp></dt> </dl> to force the encoding. Applying that to the previous example: <pre class=”example-preformatted”>ffmpeg -i movie.mpg -f image2 -c:v mjpeg menu%d.jpg </pre>

Beware that there is no "jpeg" codec. Use "mjpeg" instead. <h3 class=”section”>3.4 Why do I see a slight quality degradation with multithreaded MPEG* encoding?# TOC</h3>

For multithreaded MPEG* encoding, the encoded slices must be independent, otherwise thread n would practically have to wait for n-1 to finish, so it’s quite logical that there is a small reduction of quality. This is not a bug. <h3 class=”section”>3.5 How can I read from the standard input or write to the standard output?<a class=”anchor hidden-xs” href=”#How-

can-I-read-from-the-standard-input-or-write-to-the-standard-output_003f" aria-hidden="true"># TOC</h3>

Use <samp class="file">-</samp> as file name. <h3 class="section">3.6 -f jpeg doesn’t work.# TOC</h3>

Try ’-f image2 test%d.jpg’. <h3 class="section">3.7 Why can I not change the frame rate?# TOC</h3>

Some codecs, like MPEG-1/2, only allow a small number of fixed frame rates. Choose a different codec with the -c:v command line option. <h3 class="section">3.8 How do I encode Xvid or DivX video with ffmpeg?# TOC</h3>

Both Xvid and DivX (version 4+) are implementations of the ISO MPEG-4 standard (note that there are many other coding formats that use this same standard). Thus, use ’-c:v mpeg4’ to encode in these formats. The default fourcc stored in an MPEG-4-coded file will be ’FMP4’. If you want a different fourcc, use the ’-vtag’ option. E.g., ’-vtag xvid’ will force the fourcc ’xvid’ to be stored as the video fourcc rather than the default. <h3 class="section">3.9 Which are good parameters for encoding high quality MPEG-4?# TOC</h3>

’-mbd rd -flags +mv4+aic -trellis 2 -cmp 2 -subcmp 2 -g 300 -pass 1/2’, things to try: ’-bf 2’, ’-mpv_flags qp_rd’, ’-mpv_flags mv0’, ’-mpv_flags skip_rd’. <a name="Which-are-good-parameters-

for-encoding-high-quality-MPEG_002d1_002fMPEG_002d2_003f”> <h3 class=”section”>3.10 Which are good parameters for encoding high quality MPEG-1/MPEG-2?# TOC</h3>

’-mbd rd -trellis 2 -cmp 2 -subcmp 2 -g 100 -pass 1/2’ but beware the ’-g 100’ might cause problems with some decoders. Things to try: ’-bf 2’, ’-mpv_flags qp_rd’, ’-mpv_flags mv0’, ’-mpv_flags skip_rd’. <h3 class=”section”>3.11 Interlaced video looks very bad when encoded with ffmpeg, what is wrong?# TOC</h3>

You should use ’-flags +ilme+ildct’ and maybe ’-flags +alt’ for interlaced material, and try ’-top 0/1’ if the result looks really messed-up. <h3 class=”section”>3.12 How can I read DirectShow files?# TOC</h3>

If you have built FFmpeg for Windows with <code class=”code”>./configure --enable-avisynth</code> and your AviSynth+ installation provides <code class=”code”>DirectShowSource</code>, then you may use any file that DirectShow can read as input. Just create an "input.avs" text file with this single line ... <pre class=”example-preformatted”>DirectShowSource("C:\path to your file\yourfile.asf") </pre> ... and then feed that text file to ffmpeg: <pre class=”example-preformatted”>ffmpeg -i input.avs </pre>

For ANY other help on AviSynth, please visit the AviSynth homepage. <h3 class=”section”>3.13 How can I join video files?# TOC</h3>

To "join" video files is quite ambiguous. The following list explains the different kinds of "joining" and points out how those are addressed in FFmpeg. To join video files may mean: <ul class="itemize mark-bullet"> To put them one after the other: this is called to concatenate them (in short: concat) and is addressed in this very faq.

To put them together in the same file, to let the user choose between the different versions (example: different audio languages): this is called to multiplex them together (in short: mux), and is done by simply invoking ffmpeg with several <samp class="option">-i</samp> options.

For audio, to put all channels together in a single stream (example: two mono streams into one stereo stream): this is sometimes called to merge them, and can be done using the <code class="code">amerge</code> filter.

For audio, to play one on top of the other: this is called to mix them, and can be done by first merging them into a single stream and then using the <code class="code">pan</code> filter to mix the channels at will.

For video, to display both together, side by side or one on top of a part of the other; it can be done using the <code class="code">overlay</code> video filter.

 <h3 class="section">3.14 How can I concatenate video files?# TOC</h3>

There are several solutions, depending on the exact circumstances. <h4 class="subsection">3.14.1 Concatenating using the concat filter# TOC</h4>

FFmpeg has a <code class="code">concat</code> filter designed specifically for that, with examples in the documentation. This operation is recommended if you need to re-encode. <h4 class="subsection">3.14.2 Concatenating using the

concat demuxer# TOC</h4>

FFmpeg has a <code class=”code”>concat</code> demuxer which you can use when you want to avoid a re-encode and your format doesn’t support file level concatenation. <h4 class=”subsection”>3.14.3 Concatenating using the concat protocol (file level)# TOC</h4>

FFmpeg has a <code class=”code”>concat</code> protocol designed specifically for that, with examples in the documentation. A few multimedia containers (MPEG-1, MPEG-2 PS, DV) allow one to concatenate video by merely concatenating the files containing them. Hence you may concatenate your multimedia files by first transcoding them to these privileged formats, then using the humble <code class=”code”>cat</code> command (or the equally humble <code class=”code”>copy</code> under Windows), and finally transcoding back to your format of choice.   <pre class=”example-preformatted”>ffmpeg -i input1.avi -qscale:v 1 intermediate1.mpg ffmpeg -i input2.avi -qscale:v 1 intermediate2.mpg cat intermediate1.mpg intermediate2.mpg > intermediate_all.mpg ffmpeg -i intermediate_all.mpg -qscale:v 2 output.avi </pre>

Additionally, you can use the <code class=”code”>concat</code> protocol instead of <code class=”code”>cat</code> or <code class=”code”>copy</code> which will avoid creation of a potentially huge intermediate file.   <pre class=”example-preformatted”>ffmpeg -i input1.avi -qscale:v 1 intermediate1.mpg ffmpeg -i input2.avi -qscale:v 1 intermediate2.mpg ffmpeg -i concat:"intermediate1.mpg|intermediate2.mpg" -c copy intermediate_all.mpg ffmpeg -i intermediate_all.mpg -qscale:v 2 output.avi </pre>

Note that you may need to escape the character "|" which is special for many shells. Another option is usage of named pipes, should your platform support it: <pre class=”example-preformatted”>mkfifo intermediate1.mpg mkfifo intermediate2.mpg ffmpeg -i input1.avi -qscale:v 1 -y intermediate1.mpg < /dev/null & ffmpeg -i input2.avi -qscale:v 1 -y intermediate2.mpg < /dev/null & cat intermediate1.mpg intermediate2.mpg |\ ffmpeg -f mpeg -i - -c:v mpeg4 -c:a libmp3lame output.avi </pre>

 <h4 class="subsection">3.14.4 Concatenating using raw audio and video# TOC</h4>

Similarly, the yuv4mpegpipe format, and the raw video, raw audio codecs also allow concatenation, and the transcoding step is almost lossless. When using multiple yuv4mpegpipe(s), the first line needs to be discarded from all but the first stream. This can be accomplished by piping through <code class="code">tail</code> as seen below. Note that when piping through <code class="code">tail</code> you must use command grouping, <code class="code">{ ;}</code>, to background properly. For example, let’s say we want to concatenate two FLV files into an output.flv file: <pre class="example-preformatted">mkfifo temp1.a mkfifo temp1.v mkfifo temp2.a mkfifo temp2.v mkfifo all.a mkfifo all.v ffmpeg -i input1.flv -vn -f u16le -c:a pcm_s16le -ac 2 -ar 44100 - > temp1.a < /dev/null & ffmpeg -i input2.flv -vn -f u16le -c:a pcm_s16le -ac 2 -ar 44100 - > temp2.a < /dev/null & ffmpeg -i input1.flv -an -f yuv4mpegpipe - > temp1.v < /dev/null & { ffmpeg -i input2.flv -an -f yuv4mpegpipe - < /dev/null | tail -n +2 > temp2.v ; } & cat temp1.a temp2.a > all.a & cat temp1.v temp2.v > all.v & ffmpeg -f u16le -c:a pcm_s16le -ac 2 -ar 44100 -i all.a \ -f yuv4mpegpipe -i all.v \ -y output.flv rm temp[12].[av] all.[av] </pre>

 <h3 class="section">3.15 Using <samp class="option">-f lavfi</samp>, audio becomes mono for no apparent reason.# TOC</h3>

Use <samp class="option">-dumpgraph -</samp> to find out exactly where the channel layout is lost. Most likely, it is through <code class="code">auto-inserted aresample</code>. Try to understand why the converting filter was needed at that place. Just before the output is a likely place, as <samp class="option">-f lavfi</samp> currently only support packed S16. Then insert the correct <code class="code">aformat</code> explicitly in the filtergraph, specifying the exact format.   <pre class="example-preformatted">aformat=sample_fmts=s16:channel_layouts=stereo </pre>

 <h3 class="section">3.16 Why does FFmpeg not see the subtitles in my VOB file?# <a class="anchor hidden-xs"href="#toc-Why-does-

FFmpeg-not-see-the-subtitles-in-my-VOB-file_003f" aria-hidden="true">TOC</h3>

VOB and a few other formats do not have a global header that describes everything present in the file. Instead, applications are supposed to scan the file to see what it contains. Since VOB files are frequently large, only the beginning is scanned. If the subtitles happen only later in the file, they will not be initially detected. Some applications, including the <code class="code">ffmpeg</code> command-line tool, can only work with streams that were detected during the initial scan; streams that are detected later are ignored. The size of the initial scan is controlled by two options: <code class="code">probesize</code> (default ~5 Mo) and <code class="code">analyzeduration</code> (default 5,000,000 µs = 5 s). For the subtitle stream to be detected, both values must be large enough. <h3 class="section">3.17 Why was the <code class="command">ffmpeg</code> <samp class="option">-sameq</samp> option removed? What to use instead?# TOC</h3>

The <samp class="option">-sameq</samp> option meant "same quantizer", and made sense only in a very limited set of cases. Unfortunately, a lot of people mistook it for "same quality" and used it in places where it did not make sense: it had roughly the expected visible effect, but achieved it in a very inefficient way. Each encoder has its own set of options to set the quality-vs-size balance, use the options for the encoder you are using to set the quality level to a point acceptable for your tastes. The most common options to do that are <samp class="option">-qscale</samp> and <samp class="option">-qmax</samp>, but you should peruse the documentation of the encoder you chose. <h3 class="section">3.18 I have a stretched video, why does scaling not fix it?# TOC</h3>

A lot of video codecs and formats can store the aspect ratio of the video: this is the ratio between the width and the height of either the full image (DAR, display aspect ratio) or individual pixels (SAR, sample aspect ratio). For example, EGA screens at resolution 640×350 had 4:3 DAR and 35:48 SAR. Most still image processing work with square pixels, i.e. 1:1 SAR, but a lot of video standards, especially from the analogic-numeric transition era, use non-square pixels. Most

processing filters in FFmpeg handle the aspect ratio to avoid stretching the image: cropping adjusts the DAR to keep the SAR constant, scaling adjusts the SAR to keep the DAR constant. If you want to stretch, or “unstretch”, the image, you need to override the information with the <code class=”code”>setdar or setsar filters</code>. Do not forget to examine carefully the original video to check whether the stretching comes from the image or from the aspect ratio information. For example, to fix a badly encoded EGA capture, use the following commands, either the first one to upscale to square pixels or the second one to set the correct aspect ratio or the third one to avoid transcoding (may not work depending on the format / codec / player / phase of the moon):   <pre class=”example-preformatted”>ffmpeg -i ega_screen.nut -vf scale=640:480,setsar=1 ega_screen_scaled.nut ffmpeg -i ega_screen.nut -vf setdar=4/3 ega_screen_anamorphic.nut ffmpeg -i ega_screen.nut -aspect 4/3 -c copy ega_screen_overridden.nut </pre>

 <h3 class=”section”>3.19 How do I run ffmpeg as a background task?# TOC</h3>

ffmpeg normally checks the console input, for entries like "q" to stop and "?" to give help, while performing operations. ffmpeg does not have a way of detecting when it is running as a background task. When it checks the console input, that can cause the process running ffmpeg in the background to suspend. To prevent those input checks, allowing ffmpeg to run as a background task, use the <code class=”code”>-nostdin</code> option in the ffmpeg invocation. This is effective whether you run ffmpeg in a shell or invoke ffmpeg in its own process via an operating system API. As an alternative, when you are running ffmpeg in a shell, you can redirect standard input to <code class=”code”>/dev/null</code> (on Linux and macOS) or <code class=”code”>NUL</code> (on Windows). You can do this redirect either on the ffmpeg invocation, or from a shell script which calls ffmpeg. For example: <pre class=”example-preformatted”>ffmpeg -nostdin -i INPUT OUTPUT </pre>

or (on Linux, macOS, and other UNIX-like shells):   <pre class=”example-preformatted”>ffmpeg -i INPUT OUTPUT </dev/null </pre>

or (on Windows):   <pre class=”example-preformatted”>ffmpeg -i INPUT OUTPUT <NUL </pre>

<a name=”How-do-I-prevent-ffmpeg-from-suspending-with-a-message-like-

suspended-_0028tty-output_0029_003f”> <h3 class=”section”>3.20 How do I prevent ffmpeg from suspending with a message like suspended (tty output)?# TOC</h3>

If you run ffmpeg in the background, you may find that its process suspends. There may be a message like suspended (tty output). The question is how to prevent the process from being suspended. For example: <pre class=”example-preformatted”>% ffmpeg -i INPUT OUTPUT > ~/tmp/log.txt & [1] 93352 % [1] + suspended (tty output) ffmpeg -i INPUT OUTPUT > </pre>

The message "tty output" notwithstanding, the problem here is that ffmpeg normally checks the console input when it runs. The operating system detects this, and suspends the process until you can bring it to the foreground and attend to it. The solution is to use the right techniques to tell ffmpeg not to consult console input. You can use the <code class=”code”>-nostdin</code> option, or redirect standard input with <code class=”code”>< /dev/null</code>. See FAQ How do I run ffmpeg as a background task? for details. <h2 class=”chapter”>4 Development# TOC</h2>

 <h3 class=”section”>4.1 Are there examples illustrating how to use the FFmpeg libraries, particularly libavcodec and libavformat?# TOC</h3>

Yes. Check the <samp class=”file”>doc/examples</samp> directory in the source repository, also available online at: https://github.com/FFmpeg/FFm
 Examples are also installed by default, usually in <code class=”code”>$PREFIX/share/ffmpeg/examples</code>. Also you may read the Developers Guide of the FFmpeg documentation. Alterna-

tively, examine the source code for one of the many open source projects that already incorporate FFmpeg at (projects.html).

<h3 class="section">4.2 Can you support my C compiler XXX?# TOC</h3>

It depends. If your compiler is C11-compliant, then patches to support it are likely to be welcome if they do not pollute the source code with <code class="code">#ifdef</code>s related to the compiler.

<h3 class="section">4.3 Is Microsoft Visual C++ supported?# TOC</h3>

Yes. Please see the Microsoft Visual C++ section in the FFmpeg documentation. <h3 class="section">4.4 Can you add automake, libtool or autoconf support?#
TOC</h3>

No. These tools are too bloated and they complicate the build.
<h3 class="section">4.5 Why not rewrite FFmpeg in object-oriented C++?# TOC</h3>

FFmpeg is already organized in a highly modular manner and does not need to be rewritten in a formal object language. Further, many of the developers favor straight C; it works for them. For more arguments on this matter, read <a class="uref" href="https://web.archive.org/web/20111004021423/http://kernel.org/pub/linux/docs/lkml/#s
Religion". <h3 class="section">4.6 Why are the ffmpeg programs devoid of debugging symbols?# TOC</h3>

The build process creates <code class=”command”>ffmpeg_g</code>, <code class=”command”>ffplay_g</code>, etc. which contain full debug information. Those binaries are stripped to create <code class=”command”>ffmpeg</code>, <code class=”command”>ffplay</code>, etc. If you need the debug information, use the *_g versions. <h3 class=”section”>4.7 I do not like the LGPL, can I contribute code under the GPL instead?# TOC</h3>

Yes, as long as the code is optional and can easily and cleanly be placed under #if CONFIG_GPL without breaking anything. So, for example, a new codec or filter would be OK under GPL while a bug fix to LGPL code would not. <h3 class=”section”>4.8 I’m using FFmpeg from within my C application but the linker complains about missing symbols from the libraries themselves.# TOC</h3>

FFmpeg builds static libraries by default. In static libraries, dependencies are not handled. That has two consequences. First, you must specify the libraries in dependency order: <code class=”code”>-lavdevice</code> must come before <code class=”code”>-lavformat</code>, <code class=”code”>-lavutil</code> must come after everything else, etc. Second, external libraries that are used in FFmpeg have to be specified too. An easy way to get the full list of required libraries in dependency order is to use <code class=”code”>pkg-config</code>.   <pre class=”example-preformatted”>c99 -o program program.c $(pkg-config --cflags --libs libavformat libavcodec) </pre>

See <samp class=”file”>doc/example/Makefile</samp> and <samp class=”file”>doc/example/pc-uninstalled</samp> for more details. <h3 class=”section”>4.9 I’m using FFmpeg from within my C++ application but the linker complains about missing symbols which seem to be available.<a class=”anchor hidden-xs” href=”#I_0027m-using-FFmpeg-from-within-my-C_002b_002b-application-but-the-linker-

complains-about-missing-symbols-which-seem-to-be-available_002e" aria-hidden="true"># TOC</h3>

FFmpeg is a pure C project, so to use the libraries within your C++ application you need to explicitly state that you are using a C library. You can do this by encompassing your FFmpeg includes using <code class="code">extern "C"</code>. See http://www.parashift.com/c++-faq-lite/mixing-c-and-cpp.html#faq-32.3 <h3 class="section">4.10 I have a file in memory / a API different from *open/*read/ libc how do I use it with libavformat?# TOC</h3>

You have to create a custom AVIOContext using <code class="code">avio_alloc_context</code>, see <samp class="file">libavformat/aviobuf.c</samp> in FFmpeg and <samp class="file">libmpdemux/demux_lavf.c</samp> in MPlayer or MPlayer2 sources. <h3 class="section">4.11 Where is the documentation about ffv1, msmpeg4, asv1, 4xm?# TOC</h3>

see https://www.ffmpeg.org/~michael/ <h3 class="section">4.12 How do I feed H.263-RTP (and other codecs in RTP) to libavcodec?# TOC</h3>

Even if peculiar since it is network oriented, RTP is a container like any other. You have to demux RTP before feeding the payload to libavcodec. In this specific case please look at RFC 4629 to see how

it should be done. <h3 class="section">4.13 AVStream.r_frame_rate is wrong, it is much larger than the frame rate.# TOC</h3>

<code class="code">r_frame_rate</code> is NOT the average frame rate, it is the smallest frame rate that can accurately represent all timestamps. So no, it is not wrong if it is larger than the average! For example, if you have mixed 25 and 30 fps content, then <code class="code">r_frame_rate</code> will be 150 (it is the least common multiple). If you are looking for the average frame rate, see <code class="code">AVStream.avg_frame_rate</code>. <h3 class="section">4.14 Why is <code class="code">make fate</code> not running all tests?# TOC</h3>

Make sure you have the fate-suite samples and the <code class="code">SAMPLES</code> Make variable or <code class="code">FATE_SAMPLES</code> environment variable or the <code class="code">--samples</code> <code class="command">configure</code> option is set to the right path. <h3 class="section">4.15 Why is <code class="code">make fate</code> not finding the samples?# TOC</h3>

Do you happen to have a <code class="code">~</code> character in the samples path to indicate a home directory? The value is used in ways where the shell cannot expand it, causing FATE to not find files. Just replace <code class="code">~</code> by the full path.   This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\fate.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Automated Testing Environment </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css">

<link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Automated Testing Environment </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Introduction 2 Using FATE from your FFmpeg source directory 3 Submitting the results to the FFmpeg result aggregation server 4 Uploading new samples to the fate suite 5 FATE makefile targets and variables <ul class="toc-numbered-mark"> 5.1 Makefile targets 5.2 Makefile variables 5.3 Examples

 <h2 class="chapter">1 Introduction# TOC</h2>

FATE is an extended regression suite on the client-side and a means for results aggregation and presentation on the server-side. The first part of this document explains how you can use FATE from your FFmpeg source directory to test your ffmpeg binary. The second part describes how you can run FATE to submit the results to FFmpeg’s FATE server. In any way you can have a look at the publicly viewable FATE results by visiting this website: http://fate.ffmpeg.org/ This is especially recommended for all people contributing source code to FFmpeg, as it can be seen if some test on some platform broke with their recent contribution. This usually happens on the platforms the developers could not test on. The second part of this document describes how you can run FATE to submit your results to FFmpeg’s FATE server. If you want to submit your results be sure to check that your combination of CPU, OS and compiler is not already listed on the above mentioned website. In

the third part you can find a comprehensive listing of FATE makefile targets and variables.

 <h2 class="chapter">2 Using FATE from your FFmpeg source directory# TOC</h2>

If you want to run FATE on your machine, FFmpeg needs to be configured first. You may specify the location of samples during configuration by adding the argument <code class="code">--samples=/path/to/fate-suite</code>.   <pre class="example-preformatted">./configure --enable-somelib --samples=/path/to/fate-suite </pre>

After successful configuration, you need to add and/or update the samples in place: <pre class="example-preformatted">make fate-rsync </pre>

Now you are ready to run FATE: <pre class="example-preformatted">make fate </pre>

See makefile variables for a list of arguments that can be added. If you did not set the samples path during configuration, or if you wish to override it just before starting FATE, you can do so in one of three ways. Either by setting a make variable: <pre class="example-preformatted">make fate-rsync SAMPLES=/path/to/fate-suite make fate SAMPLES=/path/to/fate-suite </pre>

or by setting an environment variable for the current session: <pre class="example-preformatted">export FATE_SAMPLES=/path/to/fate-suite make fate-rsync make fate </pre>

or in isolation for a single command by prepending it: <pre class="example-preformatted">FATE_SAMPLES=/path/to/fate-suite make fate-rsync FATE_SAMPLES=/path/to/fate-suite make fate </pre>

This variable can also be set in your shell profile to make it permanent.   Do not put a ’~’ character in the samples path to indicate a home directory. Because of shell nuances, this will cause FATE to fail. Also, on Windows the path has to be relative to the build path which in this case is the FFmpeg source directory. Beware that some assertions are disabled by default, so mind setting <samp class="option">--assert-level=<level></samp> at configuration time, e.g. when seeking the highest possible test coverage: <pre class="example-preformatted">./configure --assert-level=2 </pre> Note that raising the assert level could

have a performance impact. To get the complete list of tests, run the command: <pre class="example-preformatted">make fate-list </pre>

You can specify a subset of tests to run by specifying the corresponding elements from the list with the <code class="code">fate-</code> prefix, e.g. as in: <pre class="example-preformatted">make fate-ffprobe_compact fate-ffprobe_xml </pre>

This makes it easier to run a few tests in case of failure without running the complete test suite. To use a custom wrapper to run the test, pass <samp class="option">--target-exec</samp> to <code class="command">configure</code> or set the <var class="var">TARGET_EXEC</var> Make variable.

<h2 class="chapter">3 Submitting the results to the FFmpeg result aggregation server# TOC</h2>

To submit your results to the server you should run fate through the shell script <samp class="file">tests/fate.sh</samp> from the FFmpeg sources. This script needs to be invoked with a configuration file as its first argument.   <pre class="example-preformatted">tests/fate.sh /path/to/fate_config </pre>

A configuration file template with comments describing the individual configuration variables can be found at <samp class="file">doc/fate_config.sh.template</samp>. The mentioned configuration template is also available here: <pre class="verbatim">slot= # some unique identifier repo=https://git.ffmpeg.org/ffmpeg.git # the source repository #branch=release/2.6 # the branch to test samples= # path to samples directory workdir= # directory in which to do all the work #fate_recv="ssh -T fate@fate.ffmpeg.org" # command to submit report comment= # optional description build_only= # set to "yes" for a compile-only instance that skips tests skip_clean= # set to "yes" to preserve build/install directories ignore_tests=

the following are optional and map to configure options arch= cpu= toolchain= cross_prefix= ar= as= cc= cxx= ld= nm= target_os= sysroot= target_exec= target_path= target_samples= extra_cflags= extra_cxxflags= extra_objcflags= extra_ldflags= extra_libs= extra_conf= # extra configure options not covered above

#make= # name of GNU make if not 'make' makeopts= # extra options passed to 'make' #makeopts_fate= # extra options passed to 'make' when running tests, # defaulting to makeopts above if this is not set #tar= # command to create a tar archive from its arguments on stdout, # defaults

to 'tar c' #fate_targets= # targets to make when running fate; defaults to "fate", # can be set to run a subset of tests, e.g. "fate-checkasm".

#fate_environments= # a list of names of configurations to run tests for; # each round is run with variables from ${${name}_env} set.

One example of using fate_environments:

target_exec="qemu-aarch64-static" # fate_targets="fate-checkasm fate-cpu" # fate_environments="sve128 sve256" # sve128_env="QEMU_CPU=max,sve128=on" # sve256_env="QEMU_CPU=max,sve256=

The variables set by fate_environments can also be used explicitly # by target_exec, e.g. like this:

target_exec="qemu-aarch64-static -cpu \$(MY_CPU)" # fate_targets="fate-checkasm fate-cpu" # fate_environments="sve128 sve256" # sve128_env="MY_CPU=max,sve128=on" # sve256_env="MY_CPU=max,sve256=on" </pre> Create a configuration that suits your needs, based on the configuration template. The <code class="env">slot</code> configuration variable can be any string that is not yet used, but it is suggested that you name it adhering to the following pattern ‘<samp class="samp"><var class="var">arch</var>-<var class="var">os</var>-<var class="var">compiler</var>-<var class="var">compiler version</var></samp>’. The configuration file itself will be sourced in a shell script, therefore all shell features may be used. This enables you to setup the environment as you need it for your build. For your first test runs the <code class="env">fate_recv</code> variable should be empty or commented out. This will run everything as normal except that it will omit the submission of the results to the server. The following files should be present in $workdir as specified in the configuration file: <ul class="itemize mark-bullet"> configure.log compile.log test.log report version

When you have everything working properly you can create an SSH key pair and send the public key to the FATE server administrator who can be contacted at the email address fate-admin@ffmpeg.org. Configure your SSH client to use public key authentication with that key when connecting to the FATE server. Also do not forget to check the identity of the server and to accept its host key. This can usually be achieved by running your SSH client manually and killing it after you accepted the key. The FATE server’s fingerprint is: <dl class="table"> <dt>‘<samp class="samp">RSA</samp>’</dt> <dd> d3:f1:83:97:a4:75:2b:a6:fb:d6:e8:aa:81:93:97:51 </dd> <dt>‘<samp class="samp">ECDSA</samp>’</dt> <dd> 76:9f:68:32:04:1e:d5:d4:ec:47:3f:dc:fc:18:17:86 </dd> </dl>

If you have problems connecting to the FATE server, it may help

to try out the <code class=”command”>ssh</code> command with one or more <samp class=”option”>-v</samp> options. You should get detailed output concerning your SSH configuration and the authentication process. The only thing left is to automate the execution of the fate.sh script and the synchronisation of the samples directory. <h2 class=”chapter”>4 Uploading new samples to the fate suite# TOC</h2>

If you need a sample uploaded send a mail to samples-request. This is for developers who have an account on the fate suite server. If you upload new samples, please make sure they are as small as possible, space on each client, network bandwidth and so on benefit from smaller test cases. Also keep in mind older checkouts use existing sample files, that means in practice generally do not replace, remove or overwrite files as it likely would break older checkouts or releases. Also all needed samples for a commit should be uploaded, ideally 24 hours, before the push. If you need an account for frequently uploading samples or you wish to help others by doing that send a mail to ffmpeg-devel.   <pre class=”example-preformatted”>#First update your local samples copy: rsync -vauL --chmod=Dg+s,Duo+x,ug+rw,o+r,o-w,+X fate-suite.ffmpeg.org:/home/samples/fate-suite/ ~/fate-suite

#Then do a dry run checking what would be uploaded: rsync -vanL --no-g --chmod=Dg+s,Duo+x,ug+rw,o+r,o-w,+X ~/fate-suite/ fate-suite.ffmpeg.org:/home/samples/fate-suite

#Upload the files: rsync -vaL --no-g --chmod=Dg+s,Duo+x,ug+rw,o+r,o-w,+X ~/fate-suite/ fate-suite.ffmpeg.org:/home/samples/fate-suite </pre>

 <h2 class=”chapter”>5 FATE makefile targets and variables# TOC</h2>

 <h3 class=”section”>5.1 Makefile targets# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>fate-rsync</samp></dt> <dd> Download/synchronize sample files to the configured samples directory. </dd> <dt><samp class=”option”>fate-list</samp></dt> <dd> Will list all fate/regression test targets. </dd> <dt><samp class=”option”>fate-list-failing</samp></dt> <dd> List the fate tests that failed the last time they were executed. </dd> <dt><samp class=”option”>fate-clear-reports</samp></dt> <dd> Remove

the test reports from previous test executions (getting rid of potentially stale results from fate-list-failing). </dd> <dt><samp class=”option”>fate</samp></dt> <dd> Run the FATE test suite (requires the fate-suite dataset). </dd> </dl>

 <h3 class=”section”>5.2 Makefile variables# TOC</h3> <dl class=”table”> <dt><code class=”env”>V</code></dt> <dd> Verbosity level, can be set to 0, 1 or 2. <ul class=”itemize mark-bullet”> 0: show just the test arguments 1: show just the command used in the test 2: show everything

</dd> <dt><code class=”env”>SAMPLES</code></dt> <dd> Specify or override the path to the FATE samples at make time, it has a meaning only while running the regression tests. </dd> <dt><code class=”env”>THREADS</code></dt> <dd> Specify how many threads to use while running regression tests, it is quite useful to detect thread-related regressions. This variable may be set to the string "random", optionally followed by a number, like "random99", This will cause each test to use a random number of threads. If a number is specified, it is used as a maximum number of threads, otherwise 16 is the maximum. In case a test fails, the thread count used for it will be written into the errfile. </dd> <dt><code class=”env”>THREAD_TYPE</code></dt> <dd> Specify which threading strategy test, either ‘<samp class=”samp”>slice</samp>’ or ‘<samp class=”samp”>frame</samp>’, by default ‘<samp class=”samp”>slice+frame</samp>’ </dd> <dt><code class=”env”>CPUFLAGS</code></dt> <dd> Specify CPU flags. </dd> <dt><code class=”env”>TARGET_EXEC</code></dt> <dd> Specify or override the wrapper used to run the tests. The <code class=”env”>TARGET_EXEC</code> option provides a way to run FATE wrapped in <code class=”command”>valgrind</code>, <code class=”command”>qemu-user</code> or <code class=”command”>wine</code> or on remote targets through <code class=”command”>ssh</code>. </dd> <dt><code class=”env”>GEN</code></dt> <dd> Set to ‘<samp class=”samp”>1</samp>’ to generate the missing or mismatched references. </dd> <dt><code class=”env”>HWACCEL</code></dt> <dd> Specify which hardware acceleration to use while running regression tests, by default ‘<samp class=”samp”>none</samp>’ is used. </dd> <dt><code class=”env”>KEEP</code></dt> <dd> Set to ‘<samp class=”samp”>1</samp>’ to keep temp files generated by fate test(s) when test is successful. Default is ‘<samp class=”samp”>0</samp>’, which removes these files. Files are always kept when a test fails. </dd> </dl>

 <h3 class="section">5.3 Examples# TOC</h3>

 <pre class="example-preformatted">make V=1 SAMPLES=/var/fate/samples THREADS=2 CPUFLAGS=mmx fate </pre> This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-all.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> ffmpeg Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> ffmpeg Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark">

1 Synopsis

2 Description

3 Detailed description

<ul class="toc-numbered-mark">

3.1 Streamcopy

3.2 Transcoding

3.3 Filtering

<ul class="toc-numbered-mark">

3.3.1 Simple filtergraphs

3.3.2 Complex filtergraphs

3.4 Loopback decoders

4 Stream selection

<ul class="toc-numbered-mark">

4.1 Description

<ul class="toc-numbered-mark">

4.1.1 Automatic stream selection

4.1.2 Manual stream selection

4.1.3 Complex filtergraphs

4.1.4 Stream handling

4.2 Examples

5 Options

<ul class="toc-numbered-mark">

5.1 Stream specifiers

5.2 Generic options

5.3 AVOptions

5.4 Main options

5.5 Video Options

5.6 Advanced Video options

5.7 Audio Options

5.8 Advanced Audio options

5.9 Subtitle options

5.10 Advanced Subtitle options

5.11 Advanced options

5.12 Preset files

<ul class="toc-numbered-mark">

5.12.1 ffpreset files

5.12.2 avpreset files

5.13 vstats file format

6 Examples

<ul class="toc-numbered-mark">

6.1 Video and Audio grabbing

6.2 X11 grabbing

6.3 Video and Audio file format conversion

7 Syntax

<ul class="toc-numbered-mark">

7.1 Quoting and escaping

<ul class="toc-numbered-mark">

7.1.1 Examples

7.2 Date

7.3 Time duration

<ul class="toc-numbered-mark">

7.3.1 Examples

7.4 Video size

7.5 Video rate

7.6 Ratio

7.7 Color

7.8 Channel Layout

8 Expression Evaluation

9 Codec Options

10 Decoders

11 Video Decoders

<ul class="toc-numbered-mark">

11.1 av1

<ul class="toc-numbered-mark">

11.1.1 Options

11.2 hevc

<ul class="toc-numbered-mark">

11.2.1 Options

11.3 rawvideo

<ul class="toc-numbered-mark">

11.3.1 Options

11.4 libastcenc

<ul class="toc-numbered-mark">

11.4.1 Options

11.5 libdav1d

<ul class="toc-numbered-mark">

11.5.1 Options

11.6 libdavs2

11.7 libuavs3d

<ul class="toc-numbered-mark">

11.7.1 Options

11.8 libxevd

<ul class="toc-numbered-mark">

11.8.1 Options

11.9 QSV Decoders

<ul class="toc-numbered-mark">

11.9.1 Common Options

11.9.2 HEVC Options

11.10 v210

<ul class="toc-numbered-mark">

11.10.1 Options

12 Audio Decoders

<ul class="toc-numbered-mark">

12.1 ac3

<ul class="toc-numbered-mark">

12.1.1 AC-3 Decoder Options

12.2 dca

<ul class="toc-numbered-mark">

12.2.1 DCA Decoder Options

12.3 flac

<ul class="toc-numbered-mark">

12.3.1 FLAC Decoder options

12.4 ffwavesynth

12.5 libgsm

12.6 libilbc

<ul class="toc-numbered-mark">

12.6.1 Options

12.7 libmpeghdec

12.8 libopencore-amrnb

12.9 libopencore-amrwb

12.10 libopus

12.11 pcm_dvda

13 Subtitles Decoders

<ul class="toc-numbered-mark">

13.1 libaribb24

<ul class="toc-numbered-mark">

13.1.1 libaribb24 Decoder Options

13.2 libaribcaption

<ul class="toc-numbered-mark">

13.2.1 libaribcaption Decoder Options

13.2.2 libaribcaption decoder usage examples

13.3 dvbsub

<ul class="toc-numbered-mark">

13.3.1 Options

13.4 dvdsub

<ul class="toc-numbered-mark">

13.4.1 Options

13.5 libzvbi-teletext

<ul class="toc-numbered-mark">

13.5.1 Options

14 Encoders

15 Audio Encoders

<ul class="toc-numbered-mark">

15.1 aac

<ul class="toc-numbered-mark">

15.1.1 Options

15.2 ac3 and ac3_fixed

<ul class="toc-numbered-mark">

15.2.1 AC-3 Metadata

<ul class="toc-numbered-mark">

15.2.1.1 Metadata Control Options

15.2.1.2 Downmix Levels

15.2.1.3 Audio Production Information

15.2.1.4 Other Metadata Options

15.2.2 Extended Bitstream Information

<ul class="toc-numbered-mark">

15.2.2.1 Extended Bitstream Information - Part 1

15.2.2.2 Extended Bitstream Information - Part 2

15.2.3 Other AC-3 Encoding Options

15.2.4 Floating-Point-Only AC-3 Encoding Options

15.3 flac

<ul class="toc-numbered-mark">

15.3.1 Options

15.4 opus

<ul class="toc-numbered-mark">

15.4.1 Options

15.5 libfdk_aac

<ul class="toc-numbered-mark">

15.5.1 Options

15.5.2 Examples

15.6 libfdk-xhe-aac

<ul class="toc-numbered-mark">

15.6.1 Options

15.7 liblc3

<ul class="toc-numbered-mark">

15.7.1 Options

15.8 libmp3lame

<ul class="toc-numbered-mark">

15.8.1 Options

15.9 libopencore-amrnb

<ul class="toc-numbered-mark">

15.9.1 Options

15.10 libopus

<ul class="toc-numbered-mark">

15.10.1 Option Mapping

15.11 libshine

<ul class="toc-numbered-mark">

15.11.1 Options

15.12 libtwolame

<ul class="toc-numbered-mark">

15.12.1 Options

15.13 libvo-amrwbenc

<ul class="toc-numbered-mark">

15.13.1 Options

15.14 libvorbis

<ul class="toc-numbered-mark">

15.14.1 Options

15.15 mjpeg

<ul class="toc-numbered-mark">

15.15.1 Options

15.16 wavpack

<ul class="toc-numbered-mark">

15.16.1 Options

<ul class="toc-numbered-mark">

15.16.1.1 Shared options

15.16.1.2 Private options

16 Video Encoders

<ul class="toc-numbered-mark">

16.1 a64_multi, a64_multi5

16.2 Cinepak

<ul class="toc-numbered-mark">

16.2.1 Options

16.3 ffv1

<ul class="toc-numbered-mark">

16.3.1 Options

16.4 GIF

<ul class="toc-numbered-mark">

16.4.1 Options

16.5 Hap

<ul class="toc-numbered-mark">

16.5.1 Options

16.6 jpeg2000

<ul class="toc-numbered-mark">

16.6.1 Options

16.7 librav1e

<ul class="toc-numbered-mark">

16.7.1 Options

16.8 libaom-av1

<ul class="toc-numbered-mark">

16.8.1 Options

16.9 liboapv

<ul class="toc-numbered-mark">

16.9.1 Options

16.10 libsvtav1

<ul class="toc-numbered-mark">

16.10.1 Options

16.11 libsvtjpegxs

<ul class="toc-numbered-mark">

16.11.1 Options

16.12 libjxl

<ul class="toc-numbered-mark">

16.12.1 Options

16.13 libkvazaar

<ul class="toc-numbered-mark">

16.13.1 Options

16.14 libopenh264

<ul class="toc-numbered-mark">

16.14.1 Options

16.15 libtheora

<ul class="toc-numbered-mark">

16.15.1 Options

16.15.2 Examples

16.16 libvpx

<ul class="toc-numbered-mark">

16.16.1 Options

16.17 libvvenc

<ul class="toc-numbered-mark">

16.17.1 Supported Pixel Formats

16.17.2 Options

16.18 libwebp

<ul class="toc-numbered-mark">

16.18.1 Pixel Format

16.18.2 Options

16.19 libastcenc

<ul class="toc-numbered-mark">

16.19.1 Pixel Format

16.19.2 Options

16.20 libx264, libx264rgb

<ul class="toc-numbered-mark">

16.20.1 Supported Pixel Formats

16.20.2 Options

16.21 libx265

<ul class="toc-numbered-mark">

16.21.1 Options

16.22 libxavs2

<ul class="toc-numbered-mark">

16.22.1 Options

16.23 libxeve

<ul class="toc-numbered-mark">

16.23.1 Options

16.24 libxvid

<ul class="toc-numbered-mark">

16.24.1 Options

16.25 MediaCodec

16.26 MediaFoundation

<ul class="toc-numbered-mark">

16.26.1 Options

16.26.2 Examples

16.27 Microsoft RLE

<ul class="toc-numbered-mark">

16.27.1 Options

16.28 mpeg2

<ul class="toc-numbered-mark">

16.28.1 Options

16.29 NVENC encoders

<ul class="toc-numbered-mark">

16.29.1 Hardware input and devices

16.29.2 Generic options

16.29.3 Common options

<ul class="toc-numbered-mark">

16.29.3.1 Presets and tuning

16.29.3.2 Rate control

16.29.3.3 Lookahead, adaptive quantization and B-frames

16.29.3.4 SEI and side data

16.29.3.5 Other common options

16.29.4 H.264 and HEVC options

16.29.5 HEVC and AV1 options

16.29.6 H.264 options

16.29.7 HEVC options

16.29.8 AV1 options

16.30 png

<ul class="toc-numbered-mark">

16.30.1 Options

16.30.2 Private options

16.31 ProRes

<ul class="toc-numbered-mark">

16.31.1 Private Options for prores-ks

16.31.2 Speed considerations

16.32 QSV Encoders

<ul class="toc-numbered-mark">

16.32.1 Ratecontrol Method

16.32.2 Global Options -> MSDK Options

16.32.3 Common Options

16.32.4 Runtime Options

16.32.5 H264 options

16.32.6 HEVC Options

16.32.7 MPEG2 Options

16.32.8 VP9 Options

16.32.9 AV1 Options

16.33 snow

<ul class="toc-numbered-mark">

16.33.1 Options

16.34 VAAPI encoders

16.35 vbn

<ul class="toc-numbered-mark">

16.35.1 Options

16.36 vc2

<ul class="toc-numbered-mark">

16.36.1 Options

17 Subtitles Encoders

<ul class="toc-numbered-mark">

17.1 dvbsub

<ul class="toc-numbered-mark">

17.1.1 Options

17.2 dvdsub

<ul class="toc-numbered-mark">

17.2.1 Options

17.3 lrc

<ul class="toc-numbered-mark">

17.3.1 Options

18 Bitstream Filters

<ul class="toc-numbered-mark">

18.1 aac_adtstoasc

18.2 av1_metadata

18.3 chomp

18.4 dca_core

18.5 dovi_rpu

18.6 dovi_split

18.7 dump_extra

18.8 dv_error_marker

18.9 eac3_core

18.10 eia608_to_smpte436m

18.11 extract_extradata

18.12 filter_units

18.13 hapqa_extract

18.14 h264_metadata

18.15 h264_mp4toannexb

18.16 h264_redundant_pps

18.17 hevc_metadata

18.18 hevc_mp4toannexb

18.19 imxdump

18.20 jpegapp

18.21 mjpeg2jpeg

18.22 mjpegadump

18.23 mov2textsub

18.24 mpeg2_metadata

18.25 mpeg4_unpack_bframes

18.26 noise

<ul class="toc-numbered-mark">

18.26.1 Examples

18.27 null

18.28 pcm_rechunk

18.29 pgs_frame_merge

18.30 prores_metadata

18.31 remove_extra

18.32 setts

18.33 showinfo

18.34 side_data

18.35 smpte436m_to_eia608

18.36 text2movsub

18.37 trace_headers

18.38 trim

18.39 truehd_core

18.40 vp9_metadata

18.41 vp9_superframe

18.42 vp9_superframe_split

18.43 vp9_raw_reorder

19 Format Options

<ul class="toc-numbered-mark">

19.1 Format stream specifiers

20 Demuxers

<ul class="toc-numbered-mark">

20.1 aa

20.2 aac

20.3 apng

20.4 asf

20.5 astc

20.6 concat

<ul class="toc-numbered-mark">

20.6.1 Syntax

20.6.2 Options

20.6.3 Examples

20.7 dash

<ul class="toc-numbered-mark">

20.7.1 Options

20.8 dvdvideo

<ul class="toc-numbered-mark">

20.8.1 Background

20.8.2 Options

20.8.3 Examples

20.9 ea

<ul class="toc-numbered-mark">

20.9.1 Options

20.10 imf

20.11 flv, live_flv, kux

20.12 gif

20.13 hls

20.14 image2

<ul class="toc-numbered-mark">

20.14.1 Examples

20.15 jpeg_mpf

20.16 ktx

20.17 libgme

20.18 libmodplug

20.19 libopenmpt

20.20 mcc

<ul class="toc-numbered-mark">

20.20.1 Examples

20.21 mov/mp4/3gp

<ul class="toc-numbered-mark">

20.21.1 Options

20.21.2 Audible AAX

20.22 mpegts

20.23 mpjpeg

20.24 rawvideo

20.25 rcwt

<ul class="toc-numbered-mark">

20.25.1 Examples

20.26 sbg

20.27 tedcaptions

20.28 vapoursynth

20.29 w64

20.30 wav

20.31 webp

21 Muxers

<ul class="toc-numbered-mark">

21.1 Raw muxers

<ul class="toc-numbered-mark">

21.1.1 Examples

21.2 Raw PCM muxers

21.3 MPEG-1/MPEG-2 program stream muxers

<ul class="toc-numbered-mark">

21.3.1 Options

<a id="toc-MOV_002fMPEG_002d4_002fISOMBFF-muxers" href="#MOV_002fMPEG_002d4_002fI
muxers">21.4 MOV/MPEG-4/ISOMBFF muxers

<ul class="toc-numbered-mark">

21.4.1 Stereo-
scopic 3D

21.4.2 Fragmenta-
tion

21.4.3 Options

21.4.4 Examples

21.5 a64

21.6 ac4

<ul class="toc-numbered-mark">

21.6.1 Options

21.7 adts

<ul class="toc-numbered-mark">

21.7.1 Options

21.8 aea

21.9 aiff

<ul class="toc-numbered-mark">

21.9.1 Options

21.10 alp

<ul class="toc-numbered-mark">

21.10.1 Options

21.11 amr

21.12 amv

21.13 apm

21.14 apng

<ul class="toc-numbered-mark">

21.14.1 Options

21.14.2 Examples

21.15 argo_asf

<ul class="toc-numbered-mark">

21.15.1 Options

21.16 argo_cvg

<ul class="toc-numbered-mark">

21.16.1 Options

21.17 asf, asf_stream

<ul class="toc-numbered-mark">

21.17.1 Options

21.18 ass

<ul class="toc-numbered-mark">

21.18.1 Options

21.19 ast

<ul class="toc-numbered-mark">

21.19.1 Options

21.20 astc

21.21 au

21.22 avi

<ul class="toc-numbered-mark">

21.22.1 Options

21.23 avif
<ul class="toc-numbered-mark">
21.23.1 Options

21.24 avm2
21.25 bit
21.26 caf
21.27 codec2
21.28 chromaprint
<ul class="toc-numbered-mark">
21.28.1 Options

21.29 crc
<ul class="toc-numbered-mark">
21.29.1 Examples

21.30 dash
<ul class="toc-numbered-mark">
21.30.1 Options
21.30.2 Example

21.31 daud
<ul class="toc-numbered-mark">
21.31.1 Example

21.32 dv
<ul class="toc-numbered-mark">
21.32.1 Example

21.33 ffmetadata
<ul class="toc-numbered-mark">
21.33.1 Example

21.34 fifo
<ul class="toc-numbered-mark">
21.34.1 Options
21.34.2 Example

21.35 film_cpk
21.36 filmstrip
21.37 fits
21.38 flac
<ul class="toc-numbered-mark">
21.38.1 Options
21.38.2 Example

21.39 flv
<ul class="toc-numbered-mark">
21.39.1 Options

21.40 framecrc
<ul class="toc-numbered-mark">
21.40.1 Examples

21.41 framehash
<ul class="toc-numbered-mark">
21.41.1 Examples

21.42 framemd5

<ul class="toc-numbered-mark">

21.42.1 Examples

21.43 gif

<ul class="toc-numbered-mark">

21.43.1 Options

21.43.2 Example

21.44 gxf

21.45 hash

<ul class="toc-numbered-mark">

21.45.1 Examples

21.46 hds

<ul class="toc-numbered-mark">

21.46.1 Options

21.46.2 Example

21.47 hls

<ul class="toc-numbered-mark">

21.47.1 Options

21.48 iamf

21.49 ico

21.50 ilbc

21.51 image2, image2pipe

<ul class="toc-numbered-mark">

21.51.1 Options

21.51.2 Examples

21.52 ircam

21.53 iterm2

<ul class="toc-numbered-mark">

21.53.1 Options

21.53.2 Examples

21.54 jpeg_mpf

<ul class="toc-numbered-mark">

21.54.1 Examples

21.55 ivf

21.56 jacosub

21.57 ktx

<ul class="toc-numbered-mark">

21.57.1 Options

21.58 kvag

21.59 lc3

21.60 lrc

<ul class="toc-numbered-mark">

21.60.1 Options

21.60.2 Metadata

21.61 matroska

<ul class="toc-numbered-mark">

21.61.1 Metadata

21.61.2 Options

21.62 md5

<ul class="toc-numbered-mark">

21.62.1 Examples

21.63 mcc

<ul class="toc-numbered-mark">

21.63.1 Options

21.63.2 Examples

21.64 microdvd

21.65 mmf

21.66 mp3

21.67 mpegts

<ul class="toc-numbered-mark">

21.67.1 Options

21.67.2 Example

21.68 mxf, mxf_d10, mxf_opatom

<ul class="toc-numbered-mark">

21.68.1 Options

21.69 null

21.70 nut

21.71 ogg

21.72 pdv

21.73 rcwt

<ul class="toc-numbered-mark">

21.73.1 Examples

21.74 segment, stream_segment, ssegment

<ul class="toc-numbered-mark">

21.74.1 Options

21.74.2 Examples

21.75 smoothstreaming

21.76 streamhash

<ul class="toc-numbered-mark">

21.76.1 Examples

21.77 tee

<ul class="toc-numbered-mark">

21.77.1 Options

21.77.2 Examples

21.78 wav

<ul class="toc-numbered-mark">

21.78.1 Options

21.79 webm_chunk

<ul class="toc-numbered-mark">

21.79.1 Options

21.79.2 Example

21.80 webm_dash_manifest

<ul class="toc-numbered-mark">

21.80.1 Options

21.80.2 Example

21.81 whip

<ul class="toc-numbered-mark">

21.81.1 Options

22 Metadata

23 Protocol Options

24 Protocols

<ul class="toc-numbered-mark">

24.1 amqp

24.2 async

24.3 bluray

24.4 cache

24.5 concat

24.6 concatf

24.7 crypto

24.8 data

24.9 fd

24.10 file

24.11 ftp

24.12 gopher

24.13 gophers

24.14 handle

24.15 http

<ul class="toc-numbered-mark">

24.15.1 HTTP Cookies

24.16 Icecast

24.17 ipfs

24.18 mmst

24.19 mmsh

24.20 md5

24.21 pipe

24.22 prompeg

24.23 rist

24.24 rtmp

24.25 rtmpe

24.26 rtmps

24.27 rtmpt

24.28 rtmpte

24.29 rtmpts

24.30 libcurl

24.31 libsmbclient

24.32 libssh

24.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte

24.34 rtp

24.35 rtsp

<ul class="toc-numbered-mark">

24.35.1 Muxer

24.35.2 Demuxer

24.35.3 Examples

24.36 sap

<ul class="toc-numbered-mark">

24.36.1 Muxer

24.36.2 Demuxer

24.37 sctp

24.38 shared

24.39 srt

24.40 srtp

24.41 subfile

24.42 tee

24.43 tcp

24.44 tls

24.45 dtls

24.46 udp

<ul class="toc-numbered-mark">

24.46.1 Examples

24.47 unix

24.48 zmq

25 Device Options

26 Input Devices

<ul class="toc-numbered-mark">

26.1 alsa

<ul class="toc-numbered-mark">

26.1.1 Options

26.2 android_camera

<ul class="toc-numbered-mark">

26.2.1 Options

26.3 avfoundation

<ul class="toc-numbered-mark">

26.3.1 Options

26.3.2 Examples

26.4 decklink

<ul class="toc-numbered-mark">

26.4.1 Options

26.4.2 Examples

26.5 dshow

<ul class="toc-numbered-mark">

26.5.1 Options

26.5.2 Examples

26.6 fbdev

<ul class="toc-numbered-mark">

26.6.1 Options

26.7 gdigrab

<ul class="toc-numbered-mark">

26.7.1 Options

26.8 iec61883

<ul class="toc-numbered-mark">

26.8.1 Options

26.8.2 Examples

26.9 jack

<ul class="toc-numbered-mark">

26.9.1 Options

26.10 kmsgrab

<ul class="toc-numbered-mark">

26.10.1 Options

26.10.2 Examples

26.11 lavfi

<ul class="toc-numbered-mark">

26.11.1 Options

26.11.2 Examples

26.12 libcdio

<ul class="toc-numbered-mark">

26.12.1 Options

26.13 libdc1394

<ul class="toc-numbered-mark">

26.13.1 Options

26.14 openal

<ul class="toc-numbered-mark">

26.14.1 Options

26.14.2 Examples

26.15 oss

<ul class="toc-numbered-mark">

26.15.1 Options

26.16 pulse

<ul class="toc-numbered-mark">

26.16.1 Options

26.16.2 Examples

26.17 sndio

<ul class="toc-numbered-mark">

26.17.1 Options

26.18 video4linux2, v4l2

<ul class="toc-numbered-mark">

26.18.1 Options

26.19 vfwcap

<ul class="toc-numbered-mark">

26.19.1 Options

26.20 x11grab

<ul class="toc-numbered-mark">

26.20.1 Options

27 Output Devices

<ul class="toc-numbered-mark">

27.1 alsa

<ul class="toc-numbered-mark">

27.1.1 Examples

27.2 AudioToolbox

<ul class="toc-numbered-mark">

27.2.1 Options

27.2.2 Examples

27.3 caca

<ul class="toc-numbered-mark">

27.3.1 Options

27.3.2 Examples

27.4 decklink

<ul class="toc-numbered-mark">

27.4.1 Options

27.4.2 Examples

27.5 fbdev

<ul class="toc-numbered-mark">

27.5.1 Options

27.5.2 Examples

27.6 oss

27.7 pulse

<ul class="toc-numbered-mark">

27.7.1 Options

27.7.2 Examples

27.8 sndio

27.9 v4l2

27.10 xv

<ul class="toc-numbered-mark">

27.10.1 Options

27.10.2 Examples

28 Resampler Options

29 Scaler Options

30 Filtering Introduction

31 graph2dot

32 Filtergraph description

<ul class="toc-numbered-mark">

32.1 Filtergraph syntax

32.2 Notes on filtergraph escaping

33 Timeline editing

34 Changing options at runtime with a command

35 Options for filters with several inputs (framesync)

36 Audio Filters

<ul class="toc-numbered-mark">

36.1 aap

36.2 acompressor

<ul class="toc-numbered-mark">

36.2.1 Commands

36.3 acontrast

36.4 acopy

36.5 acrossfade

<ul class="toc-numbered-mark">

36.5.1 Examples

36.6 acrossover

<ul class="toc-numbered-mark">

36.6.1 Examples

36.7 acrusher

<ul class="toc-numbered-mark">

36.7.1 Commands

36.8 acue

36.9 adeclick

36.10 adeclip

36.11 adecorrelate

36.12 adelay

<ul class="toc-numbered-mark">

36.12.1 Examples

36.13 adenorm

<ul class="toc-numbered-mark">

36.13.1 Commands

36.14 aderivative, aintegral

36.15 adrc

<ul class="toc-numbered-mark">

36.15.1 Commands

36.15.2 Examples

36.16 adynamicequalizer

<ul class="toc-numbered-mark">

36.16.1 Commands

36.17 adynamicsmooth

<ul class="toc-numbered-mark">

36.17.1 Commands

36.18 aecho

<ul class="toc-numbered-mark">

36.18.1 Examples

36.19 aemphasis

<ul class="toc-numbered-mark">

36.19.1 Commands

36.20 aeval

<ul class="toc-numbered-mark">

36.20.1 Examples

36.21 aexciter

<ul class="toc-numbered-mark">

36.21.1 Commands

36.22 afade

<ul class="toc-numbered-mark">

36.22.1 Commands

36.22.2 Examples

36.23 afftdn

<ul class="toc-numbered-mark">

36.23.1 Commands

36.23.2 Examples

36.24 afftfilt

<ul class="toc-numbered-mark">

36.24.1 Examples

36.25 afir

<ul class="toc-numbered-mark">

36.25.1 Examples

36.26 aformat

36.27 afreqshift

<ul class="toc-numbered-mark">

36.27.1 Commands

36.28 afwtdn

<ul class="toc-numbered-mark">

36.28.1 Commands

36.29 agate

<ul class="toc-numbered-mark">

36.29.1 Commands

36.30 aiir

<ul class="toc-numbered-mark">

36.30.1 Examples

36.31 alimiter

36.32 allpass

<ul class="toc-numbered-mark">

36.32.1 Commands

36.33 aloop

36.34 amerge

<ul class="toc-numbered-mark">

36.34.1 Examples

36.35 amix

<ul class="toc-numbered-mark">

36.35.1 Examples

36.35.2 Commands

36.36 amultiply

36.37 anequalizer

<ul class="toc-numbered-mark">

36.37.1 Examples

36.37.2 Commands

36.38 anlmdn

<ul class="toc-numbered-mark">

36.38.1 Commands

36.39 anlmf, anlms

<ul class="toc-numbered-mark">

36.39.1 Examples

36.39.2 Commands

36.40 anull

36.41 apad

<ul class="toc-numbered-mark">

36.41.1 Examples

36.42 aphaser

36.43 aphaseshift

<ul class="toc-numbered-mark">

36.43.1 Commands

36.44 apsnr

36.45 apsyclip

<ul class="toc-numbered-mark">

36.45.1 Commands

36.46 apulsator

36.47 aresample

<ul class="toc-numbered-mark">

36.47.1 Examples

36.48 areverse

<ul class="toc-numbered-mark">

36.48.1 Examples

36.49 arls

36.50 arnndn

<ul class="toc-numbered-mark">

36.50.1 Commands

36.51 asdr

36.52 asetnsamples

36.53 asetrate

36.54 ashowinfo

36.55 asisdr

36.56 asoftclip

<ul class="toc-numbered-mark">

36.56.1 Commands

36.57 aspectralstats

36.58 asr

36.59 astats

36.60 asubboost

<ul class="toc-numbered-mark">

36.60.1 Commands

36.61 asubcut

<ul class="toc-numbered-mark">

36.61.1 Commands

36.62 asupercut

<ul class="toc-numbered-mark">

36.62.1 Commands

36.63 asuperpass

<ul class="toc-numbered-mark">

36.63.1 Commands

36.64 asuperstop

<ul class="toc-numbered-mark">

36.64.1 Commands

36.65 atempo

<ul class="toc-numbered-mark">

36.65.1 Examples

36.65.2 Commands

36.66 atilt

<ul class="toc-numbered-mark">

36.66.1 Commands

36.67 atrim

36.68 axcorrelate

<ul class="toc-numbered-mark">

36.68.1 Examples

36.69 bandpass

<ul class="toc-numbered-mark">

36.69.1 Commands

36.70 bandreject

<ul class="toc-numbered-mark">

36.70.1 Commands

36.71 bass, lowshelf

<ul class="toc-numbered-mark">

36.71.1 Commands

36.72 biquad

<ul class="toc-numbered-mark">

36.72.1 Commands

36.73 bs2b

36.74 channelmap

<ul class="toc-numbered-mark">

36.74.1 Examples

36.75 channelsplit

<ul class="toc-numbered-mark">

36.75.1 Examples

36.76 chorus

<ul class="toc-numbered-mark">

36.76.1 Examples

36.77 compand

<ul class="toc-numbered-mark">

36.77.1 Examples

36.78 compensationdelay

<ul class="toc-numbered-mark">

36.78.1 Commands

36.79 crossfeed

<ul class="toc-numbered-mark">

36.79.1 Commands

36.80 crystalizer

<ul class="toc-numbered-mark">

36.80.1 Commands

36.81 dcshift

36.82 deesser

36.83 dialoguenhance

<ul class="toc-numbered-mark">

36.83.1 Commands

36.84 drmeter

36.85 dynaudnorm

<ul class="toc-numbered-mark">

36.85.1 Commands

36.86 earwax

36.87 equalizer

<ul class="toc-numbered-mark">

36.87.1 Examples

36.87.2 Commands

36.88 extrastereo

<ul class="toc-numbered-mark">

36.88.1 Commands

36.89 firequalizer

<ul class="toc-numbered-mark">

36.89.1 Examples

36.90 flanger

36.91 haas

36.92 hdcd

36.93 headphone

<ul class="toc-numbered-mark">

36.93.1 Examples

36.94 highpass

<ul class="toc-numbered-mark">

36.94.1 Commands

36.95 join

36.96 ladspa

<ul class="toc-numbered-mark">

36.96.1 Examples

36.96.2 Commands

36.97 loudnorm

36.98 lowpass

<ul class="toc-numbered-mark">

36.98.1 Examples

36.98.2 Commands

36.99 lv2

<ul class="toc-numbered-mark">

36.99.1 Examples

36.99.2 Commands

36.100 mcompand

36.101 pan

<ul class="toc-numbered-mark">

36.101.1 Mixing examples

36.101.2 Remapping examples

36.102 replaygain

36.103 resample

36.104 rubberband

<ul class="toc-numbered-mark">

36.104.1 Commands

36.105 sidechaincompress

<ul class="toc-numbered-mark">

36.105.1 Commands

36.105.2 Examples

36.106 sidechaingate

<ul class="toc-numbered-mark">

36.106.1 Commands

36.107 silencedetect

<ul class="toc-numbered-mark">

36.107.1 Examples

36.108 silenceremove

<ul class="toc-numbered-mark">

36.108.1 Examples

36.108.2 Commands

36.109 sofalizer

<ul class="toc-numbered-mark">

36.109.1 Examples

36.110 speechnorm

<ul class="toc-numbered-mark">

36.110.1 Commands

36.110.2 Examples

36.111 stereotools

<ul class="toc-numbered-mark">

36.111.1 Commands

36.111.2 Examples

36.112 stereowiden

<ul class="toc-numbered-mark">

36.112.1 Commands

36.113 superequalizer

36.114 surround

36.115 tiltshelf

<ul class="toc-numbered-mark">

36.115.1 Commands

36.116 treble, highshelf

<ul class="toc-numbered-mark">

36.116.1 Commands

36.117 tremolo

36.118 vibrato

36.119 virtualbass

36.120 volume

<ul class="toc-numbered-mark">

36.120.1 Commands

36.120.2 Examples

36.121 volumedetect

<ul class="toc-numbered-mark">

36.121.1 Examples

36.122 whisper

<ul class="toc-numbered-mark">

36.122.1 Examples

37 Audio Sources

<ul class="toc-numbered-mark">

37.1 abuffer

<ul class="toc-numbered-mark">

37.1.1 Examples

37.2 aevalsrc

<ul class="toc-numbered-mark">

37.2.1 Examples

37.3 afdelaysrc

37.4 afireqsrc

37.5 afirsrc

37.6 anullsrc

<ul class="toc-numbered-mark">

37.6.1 Examples

37.7 flite

<ul class="toc-numbered-mark">

37.7.1 Examples

37.8 anoisesrc

<ul class="toc-numbered-mark">

37.8.1 Examples

37.9 hilbert

37.10 sinc

37.11 sine

<ul class="toc-numbered-mark">

37.11.1 Examples

38 Audio Sinks

<ul class="toc-numbered-mark">

38.1 abuffersink

38.2 anullsink

39 Video Filters

<ul class="toc-numbered-mark">

39.1 addroi

<ul class="toc-numbered-mark">

39.1.1 Examples

39.2 alphaextract

39.3 alphamerge

39.4 amplify

<ul class="toc-numbered-mark">

39.4.1 Commands

39.5 ass

<ul class="toc-numbered-mark">

39.5.1 Commands

39.6 atadenoise

<ul class="toc-numbered-mark">

39.6.1 Commands

39.7 avgblur

<ul class="toc-numbered-mark">

39.7.1 Commands

39.8 backgroundkey

<ul class="toc-numbered-mark">

39.8.1 Commands

39.9 bbox

<ul class="toc-numbered-mark">

39.9.1 Commands

39.10 bilateral

<ul class="toc-numbered-mark">

39.10.1 Commands

39.11 bitplanenoise

39.12 blackdetect, blackdetect_vulkan

39.13 blackframe

39.14 blend

<ul class="toc-numbered-mark">

39.14.1 Examples

39.14.2 Commands

39.15 blockdetect

<ul class="toc-numbered-mark">

39.15.1 Examples

39.16 blurdetect

<ul class="toc-numbered-mark">

39.16.1 Examples

39.17 bm3d

<ul class="toc-numbered-mark">

39.17.1 Examples

39.18 boxblur

<ul class="toc-numbered-mark">

39.18.1 Examples

39.19 bwdif

39.20 ccrepack

39.21 cas

<ul class="toc-numbered-mark">

39.21.1 Commands

39.22 chromahold

<ul class="toc-numbered-mark">

39.22.1 Commands

39.23 chromakey

<ul class="toc-numbered-mark">

39.23.1 Commands

39.23.2 Examples

39.24 chromanr

<ul class="toc-numbered-mark">

39.24.1 Commands

39.25 chromashift

<ul class="toc-numbered-mark">

39.25.1 Commands

39.26 ciescope

39.27 codecview

<ul class="toc-numbered-mark">

39.27.1 Examples

39.28 colorbalance

<ul class="toc-numbered-mark">

39.28.1 Examples

39.28.2 Commands

39.29 colorcontrast

<ul class="toc-numbered-mark">

39.29.1 Commands

39.30 colorcorrect

<ul class="toc-numbered-mark">

39.30.1 Commands

39.31 colorchannelmixer

<ul class="toc-numbered-mark">

39.31.1 Examples

39.31.2 Commands

39.32 colordetect

39.33 colorize

<ul class="toc-numbered-mark">

39.33.1 Commands

39.34 colorkey

<ul class="toc-numbered-mark">

39.34.1 Examples

39.34.2 Commands

39.35 colorhold

<ul class="toc-numbered-mark">

39.35.1 Commands

39.36 colorlevels

<ul class="toc-numbered-mark">

39.36.1 Examples

39.36.2 Commands

39.37 colormap

39.38 colormatrix

39.39 colorspace

39.40 colortemperature

<ul class="toc-numbered-mark">

39.40.1 Commands

39.41 convolution

<ul class="toc-numbered-mark">

39.41.1 Commands

39.41.2 Examples

39.42 convolve

39.43 copy

39.44 coreimage

<ul class="toc-numbered-mark">

39.44.1 Examples

39.45 corr

39.46 cover_rect

<ul class="toc-numbered-mark">

39.46.1 Examples

39.47 crop

<ul class="toc-numbered-mark">

39.47.1 Examples

39.47.2 Commands

39.48 cropdetect

<ul class="toc-numbered-mark">

39.48.1 Examples

39.48.2 Commands

39.49 cue

39.50 curves

<ul class="toc-numbered-mark">

39.50.1 Commands

39.50.2 Examples

39.51 datascope

<ul class="toc-numbered-mark">

39.51.1 Commands

39.52 dblur

<ul class="toc-numbered-mark">

39.52.1 Commands

39.53 dctdnoiz

<ul class="toc-numbered-mark">

39.53.1 Examples

39.54 deband

<ul class="toc-numbered-mark">

39.54.1 Commands

39.55 deblock

<ul class="toc-numbered-mark">

39.55.1 Examples

39.55.2 Commands

39.56 decimate

39.57 deconvolve

39.58 dedot

39.59 deflate

<ul class="toc-numbered-mark">

39.59.1 Commands

39.60 deflicker

39.61 deinterlace_d3d12

<ul class="toc-numbered-mark">

39.61.1 Examples

39.62 dejudder

39.63 delogo

<ul class="toc-numbered-mark">

39.63.1 Examples

39.64 derain

39.65 deshake

39.66 despill

<ul class="toc-numbered-mark">

39.66.1 Commands

39.67 detelecine

39.68 dilation

<ul class="toc-numbered-mark">

39.68.1 Commands

39.69 displace

<ul class="toc-numbered-mark">

39.69.1 Examples

39.70 dnn_classify

39.71 dnn_detect

<ul class="toc-numbered-mark">

39.71.1 Examples

39.72 dnn_processing

<ul class="toc-numbered-mark">

39.72.1 Examples

39.73 drawbox

<ul class="toc-numbered-mark">

39.73.1 Examples

39.73.2 Commands

39.74 drawgraph

39.75 drawgrid

<ul class="toc-numbered-mark">

39.75.1 Examples

39.75.2 Commands

39.76 drawtext

<ul class="toc-numbered-mark">

39.76.1 Syntax

39.76.2 Text expansion

39.76.3 Commands

39.76.4 Examples

39.77 drawvg

<ul class="toc-numbered-mark">

39.77.1 Parameters

39.77.2 Pixel Formats

39.77.3 Examples

39.78 edgedetect

<ul class="toc-numbered-mark">

39.78.1 Examples

39.79 elbg

39.80 entropy

39.81 epx

39.82 eq

<ul class="toc-numbered-mark">

39.82.1 Commands

39.83 erosion

<ul class="toc-numbered-mark">

39.83.1 Commands

39.84 estdif

<ul class="toc-numbered-mark">

39.84.1 Commands

39.85 exposure

<ul class="toc-numbered-mark">

39.85.1 Commands

39.86 extractplanes

<ul class="toc-numbered-mark">

39.86.1 Examples

39.87 fade

<ul class="toc-numbered-mark">

39.87.1 Examples

39.88 feedback

<ul class="toc-numbered-mark">

39.88.1 Examples

39.89 fftdnoiz

39.90 fftfilt

<ul class="toc-numbered-mark">

39.90.1 Examples

39.91 field

39.92 fieldhint

39.93 fieldmatch

<ul class="toc-numbered-mark">

39.93.1 p/c/n/u/b meaning

<ul class="toc-numbered-mark">

39.93.1.1 p/c/n

39.93.1.2 u/b

39.93.2 Examples

39.94 fieldorder

39.95 fillborders

<ul class="toc-numbered-mark">

39.95.1 Commands

39.96 find_rect

<ul class="toc-numbered-mark">

39.96.1 Examples

39.97 floodfill

39.98 format

<ul class="toc-numbered-mark">

39.98.1 Examples

39.99 fps

<ul class="toc-numbered-mark">

39.99.1 Examples

39.100 framepack

39.101 framerate

39.102 frc_amf

39.103 vqe_amf

39.104 framestep

39.105 freezedetect

39.106 freezeframes

39.107 frei0r

<ul class="toc-numbered-mark">

39.107.1 Examples

39.107.2 Commands

39.108 fspp

39.109 fsync

39.110 gainmap

<ul class="toc-numbered-mark">

39.110.1 Examples

39.111 gblur

<ul class="toc-numbered-mark">

39.111.1 Commands

39.112 geq

<ul class="toc-numbered-mark">

39.112.1 Examples

39.113 gradfun

<ul class="toc-numbered-mark">

39.113.1 Examples

39.114 graphmonitor

39.115 grayworld

39.116 greyedge

<ul class="toc-numbered-mark">

39.116.1 Examples

39.117 guided

<ul class="toc-numbered-mark">

39.117.1 Commands

39.117.2 Examples

39.118 haldclut

<ul class="toc-numbered-mark">

39.118.1 Commands

39.118.2 Workflow examples

<ul class="toc-numbered-mark">

39.118.2.1 Hald CLUT video stream

39.118.2.2 Hald CLUT with preview

39.119 hflip

39.120 histeq

39.121 histogram

<ul class="toc-numbered-mark">

39.121.1 Examples

39.122 hqdn3d

<ul class="toc-numbered-mark">

39.122.1 Commands

39.123 hwdownload

39.124 hwmap

39.125 hwupload

39.126 hwupload_cuda

39.127 hqx

39.128 hstack

39.129 hsvhold

39.130 hsvkey

39.131 hue

<ul class="toc-numbered-mark">

39.131.1 Examples

39.131.2 Commands

39.132 huesaturation

39.133 hysteresis

39.134 iccdetect

39.135 iccgen

39.136 identity

39.137 idet

<ul class="toc-numbered-mark">

39.137.1 Examples

39.138 il

<ul class="toc-numbered-mark">

39.138.1 Commands

39.139 inflate

<ul class="toc-numbered-mark">

39.139.1 Commands

39.140 interlace, interlace_vulkan

39.141 kerndeint

<ul class="toc-numbered-mark">

39.141.1 Examples

39.142 kirsch

<ul class="toc-numbered-mark">

39.142.1 Commands

39.143 lagfun

<ul class="toc-numbered-mark">

39.143.1 Commands

39.144 latticepal

39.145 lenscorrection

<ul class="toc-numbered-mark">

39.145.1 Options

39.145.2 Commands

39.146 lensfun

<ul class="toc-numbered-mark">

39.146.1 Examples

39.147 lcevc

39.148 libplacebo

<ul class="toc-numbered-mark">

39.148.1 Options

<ul class="toc-numbered-mark">

39.148.1.1 Output mode

39.148.1.2 Scaling

39.148.1.3 Deinterlacing

39.148.1.4 Debanding

39.148.1.5 Color adjustment

39.148.1.6 Peak detection

39.148.1.7 Tone mapping

39.148.1.8 Dithering

39.148.1.9 Custom shaders

39.148.1.10 Debugging / performance

39.148.2 Commands

39.148.3 Examples

39.149 libvmaf

<ul class="toc-numbered-mark">

39.149.1 Examples

39.150 libvmaf_cuda

<ul class="toc-numbered-mark">

39.150.1 Examples

39.151 limitdiff

<ul class="toc-numbered-mark">

39.151.1 Commands

39.152 limiter

<ul class="toc-numbered-mark">

39.152.1 Commands

39.153 loop

<ul class="toc-numbered-mark">

39.153.1 Examples

39.154 lut1d

<ul class="toc-numbered-mark">

39.154.1 Commands

39.155 lut3d

<ul class="toc-numbered-mark">

39.155.1 Commands

39.156 lumakey

<ul class="toc-numbered-mark">

39.156.1 Commands

39.157 lut, lutrgb, lutyuv

<ul class="toc-numbered-mark">

39.157.1 Commands

39.157.2 Examples

39.158 lut2, tlut2

<ul class="toc-numbered-mark">

39.158.1 Commands

39.158.2 Examples

39.159 maskedclamp

<ul class="toc-numbered-mark">

39.159.1 Commands

39.160 maskedmax

<ul class="toc-numbered-mark">

39.160.1 Commands

39.161 maskedmerge

<ul class="toc-numbered-mark">

39.161.1 Commands

39.162 maskedmin

<ul class="toc-numbered-mark">

39.162.1 Commands

39.163 maskedthreshold

<ul class="toc-numbered-mark">

39.163.1 Commands

39.164 maskfun

<ul class="toc-numbered-mark">

39.164.1 Commands

39.165 mcdeint

39.166 median

<ul class="toc-numbered-mark">

39.166.1 Commands

39.167 mergeplanes

<ul class="toc-numbered-mark">

39.167.1 Examples

39.168 mestimate

39.169 mestimate_d3d12

<ul class="toc-numbered-mark">

39.169.1 Examples

39.170 midequalizer

39.171 minterpolate

39.172 mix

<ul class="toc-numbered-mark">

39.172.1 Commands

39.173 monochrome

<ul class="toc-numbered-mark">

39.173.1 Commands

39.174 morpho

<ul class="toc-numbered-mark">

39.174.1 Commands

39.175 mpdecimate

39.176 msad

39.177 multiply

<ul class="toc-numbered-mark">

39.177.1 Commands

39.178 negate

<ul class="toc-numbered-mark">

39.178.1 Commands

39.179 nlmeans

39.180 nnedi

<ul class="toc-numbered-mark">

39.180.1 Commands

39.181 noformat

<ul class="toc-numbered-mark">

39.181.1 Examples

39.182 noise

<ul class="toc-numbered-mark">

39.182.1 Examples

39.183 normalize

<ul class="toc-numbered-mark">

39.183.1 Commands

39.183.2 Examples

39.184 null

39.185 ocio

<ul class="toc-numbered-mark">

39.185.1 Examples

39.186 ocr

39.187 ocv

<ul class="toc-numbered-mark">

39.187.1 dilate

39.187.2 erode

39.187.3 smooth

39.188 oscilloscope

<ul class="toc-numbered-mark">

39.188.1 Commands

39.188.2 Examples

39.189 overlay

<ul class="toc-numbered-mark">

39.189.1 Commands

39.189.2 Examples

39.190 owdenoise

39.191 pad

<ul class="toc-numbered-mark">

39.191.1 Examples

39.192 palettegen

<ul class="toc-numbered-mark">

39.192.1 Examples

39.193 paletteuse

<ul class="toc-numbered-mark">

39.193.1 Examples

39.194 perspective

39.195 phase

<ul class="toc-numbered-mark">

39.195.1 Commands

39.196 photosensitivity

39.197 pixdesctest

39.198 pixelize

<ul class="toc-numbered-mark">

39.198.1 Commands

39.199 pixscope

<ul class="toc-numbered-mark">

39.199.1 Commands

39.200 pp7

39.201 premultiply

39.202 premultiply_dynamic

39.203 prewitt

<ul class="toc-numbered-mark">

39.203.1 Commands

39.204 pseudocolor

<ul class="toc-numbered-mark">

39.204.1 Commands

39.204.2 Examples

39.205 psnr

<ul class="toc-numbered-mark">

39.205.1 Examples

39.206 pullup

39.207 qp

<ul class="toc-numbered-mark">

39.207.1 Examples

39.208 qrencode

<ul class="toc-numbered-mark">

39.208.1 qrencode Expressions

39.208.2 qrencode Text expansion

39.208.3 Examples

39.209 quirc

39.210 random

39.211 readeia608

<ul class="toc-numbered-mark">

39.211.1 Commands

39.211.2 Examples

39.212 readvitc

<ul class="toc-numbered-mark">

39.212.1 Examples

39.213 remap

39.214 removegrain

39.215 removelogo

39.216 repeatfields

39.217 reverse

<ul class="toc-numbered-mark">

39.217.1 Examples

39.218 rgbashift

<ul class="toc-numbered-mark">

39.218.1 Commands

39.219 roberts

<ul class="toc-numbered-mark">

39.219.1 Commands

39.220 rotate

<ul class="toc-numbered-mark">

39.220.1 Examples

39.220.2 Commands

39.221 sab

39.222 scale

<ul class="toc-numbered-mark">

39.222.1 Options

39.222.2 Examples

39.222.3 Commands

39.223 scharr

<ul class="toc-numbered-mark">

39.223.1 Commands

39.224 scroll

<ul class="toc-numbered-mark">

39.224.1 Commands

39.225 scdet

39.226 selectivecolor

<ul class="toc-numbered-mark">

39.226.1 Examples

39.227 separatefields

39.228 setdar, setsar

<ul class="toc-numbered-mark">

39.228.1 Examples

39.229 setfield

39.230 setparams

39.231 shear

<ul class="toc-numbered-mark">

39.231.1 Commands

39.232 showinfo

39.233 showpalette

39.234 shuffleframes

<ul class="toc-numbered-mark">

39.234.1 Examples

39.235 shufflepixels

39.236 shuffleplanes

<ul class="toc-numbered-mark">

39.236.1 Examples

39.237 signalstats

<ul class="toc-numbered-mark">

39.237.1 Examples

39.238 signature

<ul class="toc-numbered-mark">

39.238.1 Examples

39.239 siti

<ul class="toc-numbered-mark">

39.239.1 Examples

39.240 smartblur

39.241 sobel

<ul class="toc-numbered-mark">

39.241.1 Commands

39.242 spp

<ul class="toc-numbered-mark">

39.242.1 Commands

39.243 sr

39.244 sr_amf

<ul class="toc-numbered-mark">

39.244.1 Examples

39.245 ssim

<ul class="toc-numbered-mark">

39.245.1 Examples

39.246 stereo3d

<ul class="toc-numbered-mark">

39.246.1 Examples

39.247 streamselect, astreamselect

<ul class="toc-numbered-mark">

39.247.1 Commands

39.247.2 Examples

39.248 subtitles

39.249 super2xsai

39.250 swaprect

<ul class="toc-numbered-mark">

39.250.1 Commands

39.251 swapuv

39.252 tblend

39.253 telecine

39.254 thistogram

39.255 threshold

<ul class="toc-numbered-mark">

39.255.1 Commands

39.255.2 Examples

39.256 thumbnail

<ul class="toc-numbered-mark">

39.256.1 Examples

39.257 tile

<ul class="toc-numbered-mark">

39.257.1 Examples

39.258 tiltandshift

39.259 tinterlace

39.260 tmedian

<ul class="toc-numbered-mark">

39.260.1 Commands

39.261 tmidequalizer

39.262 tmix

<ul class="toc-numbered-mark">

39.262.1 Examples

39.262.2 Commands

39.263 tonemap

<ul class="toc-numbered-mark">

39.263.1 Options

39.264 tpad

39.265 transpose

39.266 trim

39.267 unpremultiply

39.268 unsharp

<ul class="toc-numbered-mark">

39.268.1 Examples

39.269 untile

<ul class="toc-numbered-mark">

39.269.1 Examples

39.270 uspp

39.271 v360

<ul class="toc-numbered-mark">

39.271.1 Examples

39.271.2 Commands

39.272 vaguedenoiser

39.273 varblur

<ul class="toc-numbered-mark">

39.273.1 Commands

39.274 vectorscope

39.275 vidstabdetect

<ul class="toc-numbered-mark">

39.275.1 Examples

39.276 vidstabtransform

<ul class="toc-numbered-mark">

39.276.1 Options

39.276.2 Examples

39.277 vflip

39.278 vfrdet

39.279 vibrance

<ul class="toc-numbered-mark">

39.279.1 Commands

39.280 vif

39.281 vignette

<ul class="toc-numbered-mark">

39.281.1 Expressions

39.281.2 Examples

39.282 vmafmotion

39.283 vpp_amf

<ul class="toc-numbered-mark">

39.283.1 Examples

39.284 vstack

39.285 w3fdif

<ul class="toc-numbered-mark">

39.285.1 Commands

39.286 waveform

39.287 weave, doubleweave

<ul class="toc-numbered-mark">

39.287.1 Examples

39.288 xbr

39.289 xcorrelate

39.290 xfade

<ul class="toc-numbered-mark">

39.290.1 Examples

39.291 xmedian

<ul class="toc-numbered-mark">

39.291.1 Commands

39.292 xpsnr

<ul class="toc-numbered-mark">

39.292.1 Examples

39.293 xstack

<ul class="toc-numbered-mark">

39.293.1 Examples

39.294 yadif

39.295 yaepblur

<ul class="toc-numbered-mark">

39.295.1 Commands

39.296 zoompan

<ul class="toc-numbered-mark">

39.296.1 Examples

39.297 zscale

<ul class="toc-numbered-mark">

39.297.1 Options

39.297.2 Commands

40 CUDA Video Filters

<ul class="toc-numbered-mark">

40.1 bilateral_cuda

<ul class="toc-numbered-mark">

40.1.1 Examples

40.2 bwdif_cuda

40.3 chromakey_cuda

<ul class="toc-numbered-mark">

40.3.1 Examples

40.4 colorspace_cuda

40.5 overlay_cuda

40.6 pad_cuda

<ul class="toc-numbered-mark">

40.6.1 Examples

40.7 scale_cuda

<ul class="toc-numbered-mark">

40.7.1 Examples

40.8 thumbnail_cuda

<ul class="toc-numbered-mark">

40.8.1 Example

40.9 transpose_cuda

40.10 yadif_cuda

41 OpenCL Video Filters

<ul class="toc-numbered-mark">

41.1 avgblur_opencl

<ul class="toc-numbered-mark">

41.1.1 Example

41.2 boxblur_opencl

<ul class="toc-numbered-mark">

41.2.1 Examples

41.3 colorkey_opencl

<ul class="toc-numbered-mark">

41.3.1 Examples

41.4 convolution_opencl

<ul class="toc-numbered-mark">

41.4.1 Examples

41.5 erosion_opencl

<ul class="toc-numbered-mark">

41.5.1 Example

41.6 deshake_opencl

<ul class="toc-numbered-mark">

41.6.1 Examples

41.7 dilation_opencl

<ul class="toc-numbered-mark">

41.7.1 Example

41.8 nlmeans_opencl

41.9 overlay_opencl

<ul class="toc-numbered-mark">

41.9.1 Examples

41.10 pad_opencl

41.11 prewitt_opencl

<ul class="toc-numbered-mark">

41.11.1 Example

41.12 program_opencl

41.13 remap_opencl

41.14 roberts_opencl

<ul class="toc-numbered-mark">

41.14.1 Example

41.15 sobel_opencl

<ul class="toc-numbered-mark">

41.15.1 Example

41.16 tonemap_opencl

<ul class="toc-numbered-mark">

41.16.1 Example

41.17 unsharp_opencl

<ul class="toc-numbered-mark">

41.17.1 Examples

41.18 xfade_opencl

42 VAAPI Video Filters

<ul class="toc-numbered-mark">

42.1 overlay_vaapi

<ul class="toc-numbered-mark">

42.1.1 Examples

42.2 tonemap_vaapi

<ul class="toc-numbered-mark">

42.2.1 Example

42.3 hstack_vaapi

42.4 vstack_vaapi

42.5 xstack_vaapi

42.6 pad_vaapi

42.7 drawbox_vaapi

<ul class="toc-numbered-mark">

42.7.1 Examples

43 VideoToolbox Video Filters

<ul class="toc-numbered-mark">

43.1 scale_vt

<ul class="toc-numbered-mark">

43.1.1 Examples

43.2 transpose_vt

44 Vulkan Video Filters

<ul class="toc-numbered-mark">

44.1 avgblur_vulkan

44.2 blend_vulkan

44.3 bwdif_vulkan

44.4 chromaber_vulkan

44.5 color_vulkan

44.6 vflip_vulkan

44.7 hflip_vulkan

44.8 flip_vulkan

44.9 fruc_vulkan

44.10 gblur_vulkan

44.11 nlmeans_vulkan

44.12 overlay_vulkan

44.13 transpose_vulkan

44.14 v360_vulkan

<ul class="toc-numbered-mark">

44.14.1 Examples

45 QSV Video Filters

<ul class="toc-numbered-mark">

45.1 hstack_qsv

45.2 vstack_qsv

45.3 xstack_qsv

46 Video Sources

<ul class="toc-numbered-mark">

46.1 buffer

46.2 cellauto

<ul class="toc-numbered-mark">

46.2.1 Examples

46.3 coreimagesrc

<ul class="toc-numbered-mark">

46.3.1 Examples

46.4 ddagrab

<ul class="toc-numbered-mark">

46.4.1 Examples

46.5 gfxcapture

<ul class="toc-numbered-mark">

46.5.1 Examples

46.6 gradients

<ul class="toc-numbered-mark">

46.6.1 Commands

46.7 mandelbrot

46.8 mptestsrc

46.9 frei0r_src

46.10 life

<ul class="toc-numbered-mark">

46.10.1 Examples

46.11 perlin

<ul class="toc-numbered-mark">

46.11.1 Options

46.11.2 Examples

46.12 qrencodesrc

<ul class="toc-numbered-mark">

46.12.1 Examples

46.13 allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc

<ul class="toc-numbered-mark">

46.13.1 Examples

46.13.2 Commands

46.14 openclsrc

46.15 sierpinski

46.16 zoneplate

<ul class="toc-numbered-mark">

46.16.1 Commands

46.16.2 Examples

47 Video Sinks

<ul class="toc-numbered-mark">

47.1 buffersink

47.2 nullsink

48 Multimedia Filters

<ul class="toc-numbered-mark">

48.1 a3dscope

<ul class="toc-numbered-mark">

48.1.1 Commands

48.2 abitscope

48.3 adrawgraph

48.4 agraphmonitor

48.5 ahistogram

48.6 aphasemeter

<ul class="toc-numbered-mark">

48.6.1 phasing detection

48.6.2 Examples

48.7 avectorscope

<ul class="toc-numbered-mark">

48.7.1 Examples

48.7.2 Commands

48.8 bench, abench

<ul class="toc-numbered-mark">

48.8.1 Examples

48.9 concat

<ul class="toc-numbered-mark">

48.9.1 Examples

48.9.2 Commands

48.10 ebur128

<ul class="toc-numbered-mark">

48.10.1 Examples

48.11 interleave, ainterleave

<ul class="toc-numbered-mark">

48.11.1 Examples

48.12 latency, alatency

48.13 metadata, ametadata

<ul class="toc-numbered-mark">

48.13.1 Examples

48.14 perms, aperms

48.15 realtime, arealtime

<ul class="toc-numbered-mark">

48.15.1 Commands

48.16 segment, asegment

<ul class="toc-numbered-mark">

48.16.1 Examples

48.17 select, aselect

<ul class="toc-numbered-mark">

48.17.1 Examples

48.18 sendcmd, asendcmd

<ul class="toc-numbered-mark">

48.18.1 Commands syntax

48.18.2 Examples

48.19 setpts, asetpts

<ul class="toc-numbered-mark">

48.19.1 Examples

48.19.2 Commands

48.20 setrange

48.21 settb, asettb

<ul class="toc-numbered-mark">

48.21.1 Examples

48.22 showcqt

<ul class="toc-numbered-mark">

48.22.1 Examples

48.23 showcwt

48.24 showfreqs

48.25 showspatial

48.26 showspectrum

<ul class="toc-numbered-mark">

48.26.1 Examples

48.27 showspectrumpic

<ul class="toc-numbered-mark">

48.27.1 Examples

48.28 showvolume

48.29 showwaves

<ul class="toc-numbered-mark">

48.29.1 Examples

48.30 showwavespic

<ul class="toc-numbered-mark">

48.30.1 Examples

48.31 sidedata, asidedata

48.32 spectrumsynth

<ul class="toc-numbered-mark">

48.32.1 Examples

48.33 split, asplit

<ul class="toc-numbered-mark">

48.33.1 Examples

48.34 zmq, azmq

<ul class="toc-numbered-mark">

48.34.1 Examples

49 Multimedia Sources

<ul class="toc-numbered-mark">

49.1 amovie

49.2 avsynctest

<ul class="toc-numbered-mark">

49.2.1 Commands

49.3 movie

<ul class="toc-numbered-mark">

49.3.1 Examples

49.3.2 Commands

50 External libraries

<ul class="toc-numbered-mark">

50.1 Alliance for Open Media (AOM)

50.2 ARM ASTC Encoder

50.3 AMD AMF/VCE

50.4 AviSynth

50.5 Chromaprint

50.6 codec2

50.7 dav1d

50.8 davs2

50.9 uavs3d

50.10 Game Music Emu

50.11 Intel QuickSync Video

50.12 Kvazaar

50.13 LAME

50.14 LCEVCdec

50.15 libilbc

50.16 libjxl

50.17 libvpx

50.18 ModPlug

50.19 ONNX Runtime

50.20 OpenCORE, VisualOn, and Fraunhofer libraries

<ul class="toc-numbered-mark">

50.20.1 OpenCORE AMR

50.20.2 VisualOn AMR-WB encoder library

50.20.3 Fraunhofer AAC library

50.20.4 Fraunhofer xHE-AAC library

50.20.5 Fraunhofer MPEG-H 3D Audio decoder library

50.20.6 LC3 library

50.21 OpenH264

50.22 OpenJPEG

50.23 rav1e

50.24 SVT-AV1

50.25 SVT-JPEG-XS

50.26 TwoLAME

50.27 VapourSynth

50.28 x264

50.29 x265

50.30 xavs

50.31 xavs2

50.32 eXtra-fast Essential Video Encoder (XEVE)

50.33 eXtra-fast Essential Video Decoder (XEVD)

50.34 ZVBI

51 Supported File Formats, Codecs or Features

<ul class="toc-numbered-mark">

51.1 File Formats

51.2 Image Formats

51.3 Video Codecs

51.4 Audio Codecs

51.5 Subtitle Formats

51.6 Network Protocols

51.7 Input/Output Devices

51.8 Timecode

52 See Also

53 Authors

 <h2 class="chapter">1 Synopsis# TOC</h2>

ffmpeg [<var class="var">global_options</var>] {[<var class="var">input_file_options</var>] -i <samp class="file">input_url</samp>} ... {[<var class="var">output_file_options</var>] <samp class="file">output_url</samp>} ... <h2 class="chapter">2 Description# TOC</h2>

<code class="command">ffmpeg</code> is a universal media converter. It can read a wide variety of inputs - including live grabbing/recording devices - filter, and transcode them into a plethora of output formats. <code class="command">ffmpeg</code> reads from an arbitrary number of inputs (which can be regular files, pipes, network streams, grabbing devices, etc.), specified by the <code class="code">-i</code> option, and writes to an arbitrary number of outputs, which are specified by a plain output url. Anything found on the command line which cannot be interpreted as an option is considered to be an output url. Each input or output can, in principle, contain any number of elementary streams of different types (video/audio/subtitle/attachment/data), though the allowed stream counts and/or types may be limited by the container format. Selecting which streams from which inputs will go into which output is either done automatically or with the <code class="code">-map</code> option (see the Stream selection chapter). To refer to inputs/outputs in options, you must use their indices (0-based). E.g. the first input is <code class="code">0</code>, the second is <code class="code">1</code>, etc. Similarly, streams within an input/output are referred to by their indices. E.g. <code class="code">2:3</code> refers to the fourth stream in the third input or output. Also see the Stream specifiers chapter. As a general rule, options are applied to the next specified file. Therefore, order is important, and you can have the same option on the command line multiple times. Each occurrence is then applied to the next input or output file. Exceptions from this rule are the global options (e.g. verbosity level), which should be specified first. Do not mix input and output files – first specify all input files, then all output files. Also do not mix options which belong to different files. All options apply ONLY to the next input or output file and are reset between files. Some simple examples follow. <ul class="itemize mark-bullet"> Convert an input media file to a different format, by re-encoding media streams: <pre class="example-preformatted">ffmpeg -i input.avi output.mp4 </pre>

Set the video bitrate of the output file to 64 kbit/s: <pre class="example-preformatted">ffmpeg -i input.avi -b:v 64k -bufsize 64k output.mp4 </pre>

Force the frame rate of the output file to 24 fps: <pre class="example-preformatted">ffmpeg -i input.avi -r

24 output.mp4 </pre>

Force the frame rate of the input file (valid for raw formats only) to 1 fps and the frame rate of the output file to 24 fps: <pre class=”example-preformatted”>ffmpeg -r 1 -i input.m2v -r 24 output.mp4 </pre>

The format option may be needed for raw input files.

 <h2 class=”chapter”>3 Detailed description# TOC</h2>

<code class=”command”>ffmpeg</code> builds a transcoding pipeline out of the components listed below. The program’s operation then consists of input data chunks flowing from the sources down the pipes towards the sinks, while being transformed by the components they encounter along the way. The following kinds of components are available: <ul class=”itemize mark-bullet”> Demuxers (short for "demultiplexers") read an input source in order to extract

<ul class=”itemize mark-bullet”> global properties such as metadata or chapters; list of input elementary streams and their properties

One demuxer instance is created for each <samp class=”option”>-i</samp> option, and sends encoded packets to decoders or muxers. In other literature, demuxers are sometimes called splitters, because their main function is splitting a file into elementary streams (though some files only contain one elementary stream). A schematic representation of a demuxer looks like this: <pre class=”verbatim”> demuxer
packets for stream 0 elementary stream 0
global properties packets for stream 1 and elementary stream 1 metadata .....
packets for stream N elementary stream N
read from file, network stream, grabbing device, etc. </pre> Decoders receive encoded (compressed) packets for an audio, video, or subtitle elementary stream, and decode them into raw frames (arrays of pixels for video, PCM for audio). A decoder is typically associated with (and receives its input from) an elementary stream in a demuxer, but sometimes may also exist on its own (see Loopback decoders).

A schematic representation of a decoder looks like this: <pre

class=”verbatim”> packets raw frames decoder </pre> Filtergraphs process and transform raw audio or video frames. A filtergraph consists of one or more individual filters linked into a graph. Filtergraphs come in two flavors - simple and complex, configured with the <samp class=”option”>-filter</samp> and <samp class=”option”>-filter_complex</samp> options, respectively.

A simple filtergraph is associated with an output elementary stream; it receives the input to be filtered from a decoder and sends filtered output to that output stream’s encoder. A simple video filtergraph that performs deinterlacing (using the <code class=”code”>yadif</code> deinterlacer) followed by resizing (using the <code class=”code”>scale</code> filter) can look like this: <pre class=”verbatim”> simple filtergraph frames from frames for a decoder an encoder yadif scale </pre> A complex filtergraph is standalone and not associated with any specific stream. It may have multiple (or zero) inputs, potentially of different types (audio or video), each of which receiving data either from a decoder or another complex filtergraph’s output. It also has one or more outputs that feed either an encoder or another complex filtergraph’s input. The following example diagram represents a complex filtergraph with 3 inputs and 2 outputs (all video): <pre class=”verbatim”>
complex filtergraph frames frames input 0 overlay overlay output 0 frames input 1 frames frames input 2 scale split output 1
</pre> Frames from second input are overlaid over those from the first. Frames from the third input are rescaled, then the duplicated into two identical streams. One of them is overlaid over the combined first two inputs, with the result exposed as the filtergraph’s first output. The other duplicate ends up being the filtergraph’s second output. Encoders receive raw audio, video, or subtitle frames and encode them into encoded packets. The encoding (compression) process is typically lossy - it degrades stream quality to make the output smaller; some encoders are lossless, but at the cost of much higher output size. A video or audio encoder receives its input from some filtergraph’s output, subtitle encoders receive input from a decoder (since subtitle filtering is not supported yet). Every encoder is associated with some muxer’s output elementary stream and sends its output to that

muxer.

A schematic representation of an encoder looks like this: <pre class=”verbatim”> raw frames packets encoder </pre> Muxers (short for "multiplexers") receive encoded packets for their elementary streams from encoders (the transcoding path) or directly from demuxers (the streamcopy path), interleave them (when there is more than one elementary stream), and write the resulting bytes into the output file (or pipe, network stream, etc.).

A schematic representation of a muxer looks like this: <pre class=”verbatim”> packets for stream 0 muxer elementary stream 0 global packets for stream 1 properties elementary stream 1 and metadata ..... packets for stream N elementary stream N write to file, network stream, grabbing device, etc. </pre>

 <h3 class=”section”>3.1 Streamcopy# TOC</h3> The simplest pipeline in <code class=”command”>ffmpeg</code> is single-stream streamcopy, that is copying one input elementary stream’s packets without decoding, filtering, or encoding them. As an example, consider an input file called <samp class=”file”>INPUT.mkv</samp> with 3 elementary streams, from which we take the second and write it to file <samp class=”file”>OUTPUT.mp4</samp>. A schematic representation of such a pipeline looks like this: <pre class=”verbatim”> demuxer unused elementary stream 0 INPUT.mkv packets muxer elementary stream 1 elementary stream 0 OUTPUT.mp4 unused elementary stream 2 </pre> The above pipeline can be constructed with the following commandline: <pre class=”example-preformatted”>ffmpeg -i INPUT.mkv -map 0:1 -c copy OUTPUT.mp4 </pre>

In this commandline <ul class=”itemize mark-bullet”> there is a single input <samp class=”file”>INPUT.mkv</samp>;

there are no input options for this input;

there is a single output <samp class=”file”>OUTPUT.mp4</samp>;

there are two output options for this output:

<ul class="itemize mark-bullet"> <code class="code">-map 0:1</code> selects the input stream to be used - from input with index 0 (i.e. the first one) the stream with index 1 (i.e. the second one);

<code class="code">-c copy</code> selects the <code class="code">copy</code> encoder, i.e. streamcopy with no decoding or encoding.

Streamcopy is useful for changing the elementary stream count, container format, or modifying container-level metadata. Since there is no decoding or encoding, it is very fast and there is no quality loss. However, it might not work in some cases because of a variety of factors (e.g. certain information required by the target container is not available in the source). Applying filters is obviously also impossible, since filters work on decoded frames. More complex streamcopy scenarios can be constructed - e.g. combining streams from two input files into a single output: <pre class="verbatim">
demuxer 0 packets muxer elementary stream 0 elementary stream 0 INPUT0.mkv OUTPUT.mp4
demuxer 1 packets elementary stream 1 elementary stream 0 INPUT1.aac
</pre> that can be built by the commandline <pre class="example-preformatted">ffmpeg -i INPUT0.mkv -i INPUT1.aac -map 0:0 -map 1:0 -c copy OUTPUT.mp4 </pre>

The output <samp class="option">-map</samp> option is used twice here, creating two streams in the output file - one fed by the first input and one by the second. The single instance of the <samp class="option">-c</samp> option selects streamcopy for both of those streams. You could also use multiple instances of this option together with Stream specifiers to apply different values to each stream, as will be demonstrated in following sections. A converse scenario is splitting multiple streams from a single input into multiple outputs: <pre class="verbatim">
demuxer packets muxer 0 elementary stream 0 elementary stream 0 OUTPUT0.mp4 INPUT.mkv
packets elementary stream 1 muxer 1 elementary stream 0
OUTPUT1.mp4
</pre> built with <pre class="example-preformatted">ffmpeg -i INPUT.mkv -map 0:0 -c copy OUTPUT0.mp4 -map 0:1 -c copy OUTPUT1.mp4 </pre> Note how a separate instance of the <samp class="option">-c</samp> option is needed for every output file even though their values are the same. This is because non-global options (which is most of them) only apply in the context of the file before which they are placed. These examples can of course be further generalized into arbitrary

remappings of any number of inputs into any number of outputs. <h3 class="section">3.2 Transcoding# TOC</h3> Transcoding is the process of decoding a stream and then encoding it again. Since encoding tends to be computationally expensive and in most cases degrades the stream quality (i.e. it is lossy), you should only transcode when you need to and perform streamcopy otherwise. Typical reasons to transcode are: <ul class="itemize mark-bullet"> applying filters - e.g. resizing, deinterlacing, or overlaying video; resampling or mixing audio;

you want to feed the stream to something that cannot decode the original codec. Note that <code class="command">ffmpeg</code> will transcode all audio, video, and subtitle streams unless you specify <samp class="option">-c copy</samp> for them. Consider an example pipeline that reads an input file with one audio and one video stream, transcodes the video and copies the audio into a single output file. This can be schematically represented as follows <pre class="verbatim">
demuxer audio packets stream 0 (audio)
INPUT.mkv video raw packets video
video frames stream 1 (video) decoder
video
muxer packets video stream 0 (video) encoder
(libx264) OUTPUT.mp4
stream 1 (audio)
</pre> and implemented with the following commandline: <pre class="example-preformatted">ffmpeg -i INPUT.mkv -map 0:v -map 0:a -c:v libx264 -c:a copy OUTPUT.mp4 </pre> Note how it uses stream specifiers <code class="code">:v</code> and <code class="code">:a</code> to select input streams and apply different values of the <samp class="option">-c</samp> option to them; see the Stream specifiers section for more details.

 <h3 class="section">3.3 Filtering# TOC</h3>

When transcoding, audio and video streams can be filtered before encoding, with either a simple or complex filtergraph. <h4 class="subsection">3.3.1 Simple filtergraphs# <a class="anchor hidden-xs"href="#toc-Simple-

filtergraphs" aria-hidden="true">TOC</h4>

Simple filtergraphs are those that have exactly one input and output, both of the same type (audio or video). They are configured with the per-stream <samp class="option">-filter</samp> option (with <samp class="option">-vf</samp> and <samp class="option">-af</samp> aliases for <samp class="option">-filter:v</samp> (video) and <samp class="option">-filter:a</samp> (audio) respectively). Note that simple filtergraphs are tied to their output stream, so e.g. if you have multiple audio streams, <samp class="option">-af</samp> will create a separate filtergraph for each one. Taking the transcoding example from above, adding filtering (and omitting audio, for clarity) makes it look like this: <pre class="verbatim">
                demuxer   video stream
  packets video frames INPUT.mkv   decoder
                                                    simple filtergraph
                                              yadif   scale
                                          video   muxer packets video
          video stream   encoder   OUTPUT.mp4
                                                                    </pre>
<h4 class="subsection">3.3.2 Complex filtergraphs# TOC</h4>

Complex filtergraphs are those which cannot be described as simply a linear processing chain applied to one stream. This is the case, for example, when the graph has more than one input and/or output, or when output stream type is different from input. Complex filtergraphs are configured with the <samp class="option">-filter_complex</samp> option. Note that this option is global, since a complex filtergraph, by its nature, cannot be unambiguously associated with a single stream or file. Each instance of <samp class="option">-filter_complex</samp> creates a new complex filtergraph, and there can be any number of them. A trivial example of a complex filtergraph is the <code class="code">overlay</code> filter, which has two video inputs and one video output, containing one video overlaid on top of the other. Its audio counterpart is the <code class="code">amix</code> filter. <h3 class="section">3.4 Loopback decoders# TOC</h3> While decoders are normally associated with demuxer streams, it is also possible to create "loopback" decoders that decode the output from some encoder and allow it to be fed back to complex filtergraphs. This is done with the <code class="code">-dec</code> directive, which takes as a parameter <var class="var">output_file_index</var>:<var class="var">stream_index</var> to select the output stream that should be

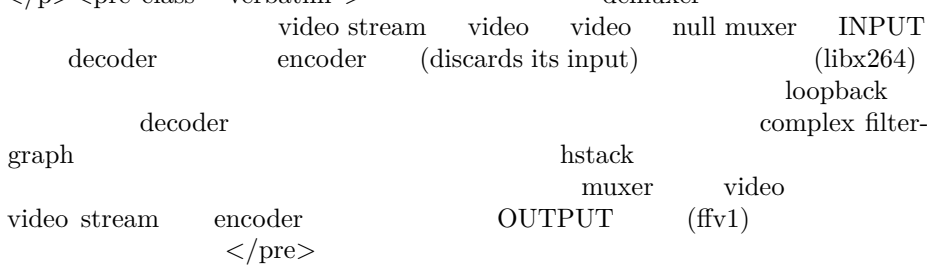
decoded. Both indices are zero-based integers: the first identifies the output file and the second identifies the stream within that file. Stream specifiers such as <code class="code">v:0</code> are not supported. Every such directive creates a new loopback decoder, indexed with successive integers starting at zero. These indices should then be used to refer to loopback decoders in complex filtergraph link labels, as described in the documentation for <samp class="option">-filter_complex</samp>. Decoding AVOptions can be passed to loopback decoders by placing them before <code class="code">-dec</code>, analogously to input/output options. E.g. the following example: <pre class="example-preformatted">ffmpeg -i INPUT \ -map 0:v:0 -c:v libx264 -crf 45 -f null - \ -threads 3 -dec 0:0 \ -filter_complex '[0:v][dec:0]hstack[stack]' \ -map '[stack]' -c:v ffv1 OUTPUT </pre>

reads an input video and <ul class="itemize mark-bullet"> (line 2) encodes it with <code class="code">libx264</code> at low quality;

(line 3) decodes this encoded stream using 3 threads;

(line 4) places decoded video side by side with the original input video;

(line 5) combined video is then losslessly encoded and written into <samp class="file">OUTPUT</samp>.

Such a transcoding pipeline can be represented with the following diagram: <pre class="verbatim">


</pre>

 <h2 class="chapter">4 Stream selection# TOC</h2>

<code class="command">ffmpeg</code> provides the <code class="code">-map</code> option for manual control of stream selection in each output file. Users can skip <code class="code">-map</code> and let ffmpeg perform automatic stream selection as described below. The <code class="code">-vn / -an / -sn / -dn</code> options can be used to skip inclusion of video, audio, subtitle

and data streams respectively, whether manually mapped or automatically selected, except for those streams which are outputs of complex filtergraphs. <h3 class="section">4.1 Description# TOC</h3> The sub-sections that follow describe the various rules that are involved in stream selection. The examples that follow next show how these rules are applied in practice. While every effort is made to accurately reflect the behavior of the program, FFmpeg is under continuous development and the code may have changed since the time of this writing. <h4 class="subsection">4.1.1 Automatic stream selection# TOC</h4>

In the absence of any map options for a particular output file, ffmpeg inspects the output format to check which type of streams can be included in it, viz. video, audio and/or subtitles. For each acceptable stream type, ffmpeg will pick one stream, when available, from among all the inputs. It will select that stream based upon the following criteria: <ul class="itemize mark-bullet"> for video, it is the stream with the highest resolution, for audio, it is the stream with the most channels, for subtitles, it is the first subtitle stream found but there’s a caveat. The output format’s default subtitle encoder can be either text-based or image-based, and only a subtitle stream of the same type will be chosen.

In the case where several streams of the same type rate equally, the stream with the lowest index is chosen. Data or attachment streams are not automatically selected and can only be included using <code class="code">-map</code>. <h4 class="subsection">4.1.2 Manual stream selection# TOC</h4>

When <code class="code">-map</code> is used, only user-mapped streams are included in that output file, with one possible exception for filtergraph outputs described below. <h4 class="subsection">4.1.3 Complex filtergraphs# TOC</h4>

If there are any complex filtergraph output streams with unlabeled pads, they will be added to the first output file. This will lead to a fatal

error if the stream type is not supported by the output format. In the absence of the map option, the inclusion of these streams leads to the automatic stream selection of their types being skipped. If map options are present, these filtergraph streams are included in addition to the mapped streams. Complex filtergraph output streams with labeled pads must be mapped once and exactly once. <h4 class="subsection">4.1.4 Stream handling# TOC</h4>

Stream handling is independent of stream selection, with an exception for subtitles described below. Stream handling is set via the <code class="code">-codec</code> option addressed to streams within a specific output file. In particular, codec options are applied by ffmpeg after the stream selection process and thus do not influence the latter. If no <code class="code">-codec</code> option is specified for a stream type, ffmpeg will select the default encoder registered by the output file muxer. An exception exists for subtitles. If a subtitle encoder is specified for an output file, the first subtitle stream found of any type, text or image, will be included. ffmpeg does not validate if the specified encoder can convert the selected stream or if the converted stream is acceptable within the output format. This applies generally as well: when the user sets an encoder manually, the stream selection process cannot check if the encoded stream can be muxed into the output file. If it cannot, ffmpeg will abort and all output files will fail to be processed. <h3 class="section">4.2 Examples# TOC</h3>

The following examples illustrate the behavior, quirks and limitations of ffmpeg’s stream selection methods. They assume the following three input files. <pre class="verbatim"> input file 'A.avi' stream 0: video 640x360 stream 1: audio 2 channels

input file 'B.mp4' stream 0: video 1920x1080 stream 1: audio 2 channels stream 2: subtitles (text) stream 3: audio 5.1 channels stream 4: subtitles (text)

input file 'C.mkv' stream 0: video 1280x720 stream 1: audio 2 channels stream 2: subtitles (image) </pre> <pre class="example-preformatted">ffmpeg -i A.avi -i B.mp4 out1.mkv out2.wav -map 1:a -c:a copy out3.mov </pre> There are three output files specified, and for the first two, no <code class="code">-map</code> options are set, so ffmpeg will select streams for these two files automatically. <samp class="file">out1.mkv</samp> is a Matroska container file and accepts video, audio and subtitle streams, so ffmpeg

will try to select one of each type.
 For video, it will select <code class=”code”>stream 0</code> from <samp class=”file”>B.mp4</samp>, which has the highest resolution among all the input video streams.
 For audio, it will select <code class=”code”>stream 3</code> from <samp class=”file”>B.mp4</samp>, since it has the greatest number of channels.
 For subtitles, it will select <code class=”code”>stream 2</code> from <samp class=”file”>B.mp4</samp>, which is the first subtitle stream from among <samp class=”file”>A.avi</samp> and <samp class=”file”>B.mp4</samp>. <samp class=”file”>out2.wav</samp> accepts only audio streams, so only <code class=”code”>stream 3</code> from <samp class=”file”>B.mp4</samp> is selected. For <samp class=”file”>out3.mov</samp>, since a <code class=”code”>-map</code> option is set, no automatic stream selection will occur. The <code class=”code”>-map 1:a</code> option will select all audio streams from the second input <samp class=”file”>B.mp4</samp>. No other streams will be included in this output file. For the first two outputs, all included streams will be transcoded. The encoders chosen will be the default ones registered by each output format, which may not match the codec of the selected input streams. For the third output, codec option for audio streams has been set to <code class=”code”>copy</code>, so no decoding-filtering-encoding operations will occur, or can occur. Packets of selected streams shall be conveyed from the input file and muxed within the output file. <pre class=”example-preformatted”>ffmpeg -i C.mkv out1.mkv -c:s dvdsub -an out2.mkv </pre> Although <samp class=”file”>out1.mkv</samp> is a Matroska container file which accepts subtitle streams, only a video and audio stream shall be selected. The subtitle stream of <samp class=”file”>C.mkv</samp> is image-based and the default subtitle encoder of the Matroska muxer is text-based, so a transcode operation for the subtitles is expected to fail and hence the stream isn’t selected. However, in <samp class=”file”>out2.mkv</samp>, a subtitle encoder is specified in the command and so, the subtitle stream is selected, in addition to the video stream. The presence of <code class=”code”>-an</code> disables audio stream selection for <samp class=”file”>out2.mkv</samp>. <pre class=”example-preformatted”>ffmpeg -i A.avi -i C.mkv -i B.mp4 -filter_complex "overlay" out1.mp4 out2.srt </pre> A filtergraph is setup here using the <code class=”code”>-filter_complex</code> option and consists of a single video filter. The <code class=”code”>overlay</code> filter requires exactly two video inputs, but none are specified, so the first two available video streams are used, those of <samp class=”file”>A.avi</samp> and <samp class=”file”>C.mkv</samp>. The output pad of the filter has no label and so is sent to the first output file <samp class=”file”>out1.mp4</samp>. Due to this, automatic selection of the video stream is skipped, which would have selected the

stream in <samp class=”file”>B.mp4</samp>. The audio stream with most channels viz. <code class=”code”>stream 3</code> in <samp class=”file”>B.mp4</samp>, is chosen automatically. No subtitle stream is chosen however, since the MP4 format has no default subtitle encoder registered, and the user hasn’t specified a subtitle encoder. The 2nd output file, <samp class=”file”>out2.srt</samp>, only accepts text-based subtitle streams. So, even though the first subtitle stream available belongs to <samp class=”file”>C.mkv</samp>, it is image-based and hence skipped. The selected stream, <code class=”code”>stream 2</code> in <samp class=”file”>B.mp4</samp>, is the first text-based subtitle stream. <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0[outv];overlay;aresample" \ -map '[outv]' -an out1.mp4 \ out2.mkv \ -map '[outv]' -map 1:a:0 out3.mkv </pre>

The above command will fail, as the output pad labelled <code class=”code”>[outv]</code> has been mapped twice. None of the output files shall be processed.   <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0[outv];overlay;aresample" \ -an out1.mp4 \ out2.mkv \ -map 1:a:0 out3.mkv </pre>

This command above will also fail as the hue filter output has a label, <code class=”code”>[outv]</code>, and hasn’t been mapped anywhere. The command should be modified as follows, <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0,split=2[outv1][outv2];overlay;aresample" \ -map '[outv1]' -an out1.mp4 \ out2.mkv \ -map '[outv2]' -map 1:a:0 out3.mkv </pre> The video stream from <samp class=”file”>B.mp4</samp> is sent to the hue filter, whose output is cloned once using the split filter, and both outputs labelled. Then a copy each is mapped to the first and third output files. The overlay filter, requiring two video inputs, uses the first two unused video streams. Those are the streams from <samp class=”file”>A.avi</samp> and <samp class=”file”>C.mkv</samp>. The overlay output isn’t labelled, so it is sent to the first output file <samp class=”file”>out1.mp4</samp>, regardless of the presence of the <code class=”code”>-map</code> option. The aresample filter is sent the first unused audio stream, that of <samp class=”file”>A.avi</samp>. Since this filter output is also unlabelled, it too is mapped to the first output file. The presence of <code class=”code”>-an</code> only suppresses automatic or manual stream selection of audio streams, not outputs sent from filtergraphs. Both these mapped streams shall be ordered before the mapped stream in <samp class=”file”>out1.mp4</samp>. The video, audio and subtitle streams mapped to <code class=”code”>out2.mkv</code> are entirely determined by automatic stream selection. <samp class=”file”>out3.mkv</samp> consists of the cloned video output from the

hue filter and the first audio stream from <samp class=”file”>B.mp4</samp>.

 <h2 class=”chapter”>5 Options<span class=”pull-
right”>#
TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or ’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’ as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class=”example-preformatted”>ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">5.1 Stream specifiers# TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class="code">a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class="code">-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class="code">-codec copy</code>

or <code class="code">-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class="table">

<dt><samp class="option"><var class="var">stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class="code">-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class=”var”>stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class=”var”>stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class=”option”><var class=”var”>stream_type</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> <var class=”var”>stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class=”var”>additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class=”var”>additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class=”option”>g:<var class=”var”>group_specifier</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class=”var”>group_specifier</var>.

if <var class=”var”>additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class=”var”>additional_stream_specifier</var>.

<var class=”var”>group_specifier</var> may be one of the following:

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class="var">additional_stream_specifier</var>.

</dd>

<dt><samp class="option">#<var class="var">stream_id</var> or i:<var class="var">stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class="option">m:<var class="var">key</var>[:<var class="var">value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class="var">key</var> having the specified value. If

<var class="var">value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class="var">key</var> or <var class="var">value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class="option">disp:<var class="var">dispositions</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class="var">dispositions</var> is a list of

one or more dispositions (as printed by the <samp class="option">-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class="option">u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class="command">ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">5.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options, including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed

information about the demuxer named <var class=”var”>demuxer_name</var>. Use the <samp class=”option”>-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class=”option”>muxer=<var class=”var”>muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class=”var”>muxer_name</var>. Use the <samp class=”option”>-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class=”option”>filter=<var class=”var”>filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class=”var”>filter_name</var>. Use the <samp class=”option”>-filters</samp> option to get a list of all filters. </dd> <dt><samp class=”option”>bsf=<var class=”var”>bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class=”var”>bitstream_filter_name</var>. Use the <samp class=”option”>-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class=”option”>protocol=<var class=”var”>protocol_name</var></samp></dt> <dd> Print detailed information about the protocol named <var class=”var”>protocol_name</var>. Use the <samp class=”option”>-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class=”option”>-version</samp></dt> <dd> Show version. </dd> <dt><samp class=”option”>-buildconf</samp></dt> <dd> Show the build configuration, one option per line. </dd> <dt><samp class=”option”>-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class=”option”>-demuxers</samp></dt> <dd> Show available demuxers. </dd> <dt><samp class=”option”>-muxers</samp></dt> <dd> Show available muxers. </dd> <dt><samp class=”option”>-devices</samp></dt> <dd> Show available devices. </dd> <dt><samp class=”option”>-codecs</samp></dt> <dd> Show all codecs known to libavcodec. Note that the term ’codec’ is used throughout this documentation as a shortcut for what is more correctly called a media bitstream format. </dd> <dt><samp class=”option”>-decoders</samp></dt> <dd> Show available decoders. </dd> <dt><samp class=”option”>-encoders</samp></dt> <dd> Show all available encoders. </dd> <dt><samp class=”option”>-bsfs</samp></dt> <dd> Show available bitstream filters. </dd> <dt><samp class=”option”>-protocols</samp></dt> <dd> Show available protocols. </dd> <dt><samp class=”option”>-filters</samp></dt> <dd> Show available libavfilter filters. </dd> <dt><samp class=”option”>-pix_fmts</samp></dt> <dd> Show available pixel formats. </dd> <dt><samp class=”option”>-sample_fmts</samp></dt> <dd> Show available sample formats. </dd> <dt><samp class=”option”>-layouts</samp></dt> <dd> Show channel names and standard channel layouts. </dd> <dt><samp class=”option”>-dispositions</samp></dt> <dd> Show stream dispositions.

</dd> <dt><samp class="option">-colors</samp></dt> <dd> Show recognized color names. </dd> <dt><samp class="option">-sources <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sources of the input device. Some devices may provide system-dependent source names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sources pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sinks of the output device. Some devices may provide system-dependent sink names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var class="var">loglevel</var></samp></dt> <dd> Set logging level and flags used by the library. The optional <var class="var">flags</var> prefix can consist of the following values: <dl class="table"> <dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates that repeated log output should not be compressed to the first line and the "Last message repeated n times" line will be omitted. </dd> <dt>‘<samp class="samp">level</samp>’</dt> <dd> Indicates that log output should add a <code class="code">[level]</code> prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var class="var">flags</var> or changing <var class="var">loglevel</var>. When setting both <var class="var">flags</var> and <var class="var">loglevel</var>, a ’+’ separator is expected between the last <var class="var">flags</var> value and before <var class="var">loglevel</var>. <var class="var">loglevel</var> is a string or a number containing one of the following values: <dl class="table"> <dt>‘<samp class="samp">quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class="samp">panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class="samp">fatal,

8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class="samp">error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class="samp">warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class="samp">info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class="samp">verbose, 40</samp>’</dt> <dd> Same as <code class="code">info</code>, except more verbose. </dd> <dt>‘<samp class="samp">debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class="samp">trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class="code">level</code> prefix, and set <var class="var">loglevel</var> to <code class="code">verbose</code>: <pre class="example-preformatted">ffmpeg -loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code class="code">level</code> prefix flag or <var class="var">loglevel</var>: <pre class="example-preformatted">ffmpeg [...] -loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class="env">AV_LOG_FORCE_NOCOLOR</code>, or can be forced setting the environment variable <code class="env">AV_LOG_FORCE_COLOR</code>. </dd> <dt><samp class="option">-report</samp></dt> <dd> Dump full command line and log output to a file named <code class="code"><var class="var">program</var>-<var class="var">YYYYMMDD</var>-<var class="var">HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class="code">-loglevel debug</code>. Setting the environment variable <code class="env">FFREPORT</code> to any value has the same effect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual). The following options are recognized: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> set the file name to use for the report; <code class="code">%p</code> is expanded to the name of the program, <code class="code">%t</code> is expanded to a timestamp, <code class="code">%%</code> is expanded to a plain <code class="code">%</code> </dd> <dt><samp class="option">level</samp></dt> <dd> set the log verbosity

level using a numerical value (see <code class=”code”>-loglevel</code>). </dd> </dl>

For example, to output a report to a file named <samp class=”file”>ffreport.log</samp> using a log level of <code class=”code”>32</code> (alias for log level <code class=”code”>info</code>):   <pre class=”example-preformatted”>FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class=”option”>-hide_banner</samp></dt> <dd> Suppress printing banner. All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class=”option”>-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class=”table”> <dt>‘<samp class=”samp”>x86</samp>’</dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>mmx</samp>’</dt> <dt>‘<samp class=”samp”>mmxext</samp>’</dt> <dt>‘<samp class=”samp”>sse</samp>’</d <dt>‘<samp class=”samp”>sse2</samp>’</dt> <dt>‘<samp class=”samp”>sse2slow</samp>’</dt> <dt>‘<samp class=”samp”>sse3</samp>’</ <dt>‘<samp class=”samp”>sse3slow</samp>’</dt> <dt>‘<samp class=”samp”>ssse3</samp>’</dt> <dt>‘<samp class=”samp”>atom</samp>’</dt <dt>‘<samp class=”samp”>sse4.1</samp>’</dt> <dt>‘<samp class=”samp”>sse4.2</samp>’</dt> <dt>‘<samp class=”samp”>avx</samp>’</dt> <dt>‘<samp class=”samp”>avx2</samp>’</dt> <dt>‘<samp class=”samp”>xop</samp>’</dt> <dt>‘<samp class=”samp”>fma3</samp>’</dt> <dt>‘<samp class=”samp”>fma4</samp>’</dt> <dt>‘<samp class=”samp”>3dnow</samp>’</dt> <dt>‘<samp class=”samp”>3dnowext</samp>&rsquo <dt>‘<samp class=”samp”>bmi1</samp>’</dt> <dt>‘<samp class=”samp”>bmi2</samp>’</dt> <dt>‘<samp class=”samp”>cmov</samp>’</dt </dl> </dd> <dt>‘<samp class=”samp”>ARM</samp>’</dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>armv5te</samp>’</dt> <dt>‘<samp class=”samp”>armv6</samp>’</dt> <dt>‘<samp class=”samp”>armv6t2</samp>’</dt> <dt>‘<samp class=”samp”>vfp</samp>’</d <dt>‘<samp class=”samp”>vfpv3</samp>’</dt> <dt>‘<samp class=”samp”>neon</samp>’</dt> <dt>‘<samp class=”samp”>setend</samp>’</d </dl> </dd> <dt>‘<samp class=”samp”>AArch64</samp>’</dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>armv8</samp>’</dt> <dt>‘<samp class=”samp”>vfp</samp>’</dt> <dt>‘<samp class=”samp”>neon</samp>’</dt> </dl> </dd> <dt>‘<samp class=”samp”>PowerPC</samp>’</dt> <dd><dl class=”table”>

<dt>‘<samp class=”samp”>altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class=”samp”>Specific Processors</samp>’</dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>pentium2</samp>’</dt>
<dt>‘<samp class=”samp”>pentium3</samp>’</dt>
<dt>‘<samp class=”samp”>pentium4</samp>’</dt>
<dt>‘<samp class=”samp”>k6</samp>’</dt> <dt>‘<samp
class=”samp”>k62</samp>’</dt> <dt>‘<samp class=”samp”>athlon</samp>’</dt
<dt>‘<samp class=”samp”>athlonxp</samp>’</dt> <dt>‘<samp
class=”samp”>k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>-cpucount <var class=”var”>count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class=”option”>-max_alloc <var class=”var”>bytes</var></samp></dt>
<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is INT_MAX. </dd> </dl>

 <h3 class=”section”>5.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and libavcodec libraries. To see the list of available AVOptions, use the <samp class=”option”>-help</samp> option. They are separated into two categories: <dl class=”table”> <dt><samp class=”option”>generic</samp></dt>
<dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class=”option”>private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class=”option”>id3v2_version</samp> private option of the MP3 muxer: <pre class=”example-preformatted”>ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class=”example-preformatted”>ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class="option">-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class="option">-option 0</samp>/<samp class="option">-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class="section">5.4 Main options# TOC</h3>

<dl class="table">

<dt><samp class="option">-f <var class="var">fmt</var> (input/output)</samp></dt>

<dd> Force input or output file format. The format is normally auto detected for input

files and guessed from the file extension for output files, so this option is not

needed in most cases.

</dd>

<dt><samp class="option">-i <var class="var">url</var> (input)</samp></

<dd> input file url

</dd>

<dt><samp class="option">-y (global)</samp></dt>

<dd> Overwrite output files without asking.

</dd>

<dt><samp class="option">-n (global)</samp></dt>

<dd> Do not overwrite output files, and exit immediately if a specified

output file already exists.

</dd>

<dt><samp class="option">-stream_loop <var class="var">number</var> (input)</samp></dt>

<dd> Set number of times input stream shall be looped. Loop 0 means no loop,

loop -1 means infinite loop.

</dd>

<dt><samp class="option">-recast_media (global)</samp></dt>

<dd> Allow forcing a decoder of a different media type than the one

detected or designated by the demuxer. Useful for decoding media

data muxed as data streams.

</dd>

<dt><samp class="option">-c[:<var class="var">stream_specifier</var>] <var class="var">codec</var> (input/output,per-stream)</samp></dt>

<dt><samp class="option">-codec[:<var class="var">stream_specifier</var>] <var class="var">codec</var> (input/output,per-stream)</samp></dt>

<dd> Select an encoder (when used before an output file) or a decoder (when used

before an input file) for one or more streams. <var class="var">codec</var> is the name of a

decoder/encoder or a special value <code class="code">copy</code> (output only) to indicate that

the stream is not to be re-encoded.

For example

<pre class="example-preformatted">ffmpeg -i INPUT -map 0 -c:v libx264 -c:a copy OUTPUT

</pre>

encodes all video streams with libx264 and copies all audio streams.

For each stream, the last matching <code class="code">c</code> option is applied, so

<pre class="example-preformatted">ffmpeg -i INPUT -map 0 -c copy -c:v:1 libx264 -c:a:137 libvorbis OUTPUT

</pre>

will copy all the streams except the second video, which will be encoded with

libx264, and the 138th audio, which will be encoded with libvorbis.

</dd>

<dt><samp class="option">-t <var class="var">duration</var> (input/output)</samp></dt>

<dd> When used as an input option (before <code class="code">-i</code>), limit the <var class="var">duration</var> of

data read from the input file.

When used as an output option (before an output url), stop writing the

output after its duration reaches <var class="var">duration</var>.

<var class="var">duration</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

-to and -t are mutually exclusive and -t has priority.

</dd>

<dt><samp class="option">-to <var class="var">position</var> (input/output)</samp></dt>

<dd> Stop writing the output or reading the input at <var class="var">position</var>.

<var class="var">position</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

-to and -t are mutually exclusive and -t has priority.

</dd>

<dt><samp class="option">-fs <var class="var">limit_size</var> (output)</samp></dt>

<dd> Set the file size limit, expressed in bytes. No further chunk of bytes is written

after the limit is exceeded. The size of the output file is slightly more than the

requested file size.

</dd>

<dt><samp class="option">-ss <var class="var">position</var> (input/output)</samp></dt>

<dd> When used as an input option (before <code class="code">-i</code>), seeks in this input file to

<var class="var">position</var>. Note that in most formats it is not possible to seek exactly,

so <code class="command">ffmpeg</code> will seek to the closest seek point before <var class="var">position</var>.

When transcoding and <samp class="option">-accurate_seek</samp> is enabled (the default), this

extra segment between the seek point and <var class="var">position</var> will be decoded and

discarded. When doing stream copy or when <samp class="option">-noaccurate_seek</samp> is used, it

will be preserved.

When used as an output option (before an output url), decodes but discards

input until the timestamps reach <var class="var">position</var>.

<var class="var">position</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

</dd>

<dt><samp class="option">-sseof <var class="var">position</var> (input)</samp></dt>

<dd>

Like the <code class="code">-ss</code> option but relative to the "end of file". That is negative

values are earlier in the file, 0 is at EOF.

</dd>

<dt><samp class="option">-isync <var class="var">input_index</var> (input)</samp></dt>

<dd> Assign an input as a sync source.

This will take the difference between the start times of the target and reference inputs and

offset the timestamps of the target file by that difference. The source timestamps of the two

inputs should derive from the same clock source for expected results. If <code class="code">copyts</code> is set

then <code class="code">start_at_zero</code> must also be set. If either of the inputs has no starting timestamp

then no sync adjustment is made.

Acceptable values are those that refer to a valid ffmpeg input index. If the sync reference is

the target index itself or <var class="var">-1</var>, then no adjustment is made to target timestamps. A sync

reference may not itself be synced to any other input.

Default value is <var class="var">-1</var>.

</dd>

<dt><samp class="option">-itsoffset <var class="var">offset</var> (input)</samp></dt>

<dd> Set the input time offset.

<var class="var">offset</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

The offset is added to the timestamps of the input files. Specifying

a positive offset means that the corresponding streams are delayed by

the time duration specified in <var class="var">offset</var>.

</dd>

<dt><samp class="option">-itsscale <var class="var">scale</var> (input,per-stream)</samp></dt>

<dd> Rescale input timestamps. <var class="var">scale</var> should be a floating point number.

</dd>

<dt><samp class="option">-timestamp <var class="var">date</var> (output)</samp></dt>

<dd> Set the recording timestamp in the container.

<var class="var">date</var> must be a date specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#date-syntax">the Date section in the ffmpeg-utils(1) manual.

</dd>

<dt><samp class="option">-metadata[:metadata_specifier] <var class="var">key</var>=<var class="var">value</var> (output,per-metadata)</samp></dt>

<dd> Set a metadata key/value pair.

An optional <var class="var">metadata_specifier</var> may be given to set metadata

on streams, chapters or programs. See <code class="code">-map_metadata</code>

documentation for details.

This option overrides metadata set with <code class="code">-map_metadata</code>. It is

also possible to delete metadata by using an empty value.

For example, for setting the title in the output file:

<pre class="example-preformatted">ffmpeg -i in.avi -metadata title="my title" out.flv

</pre>

To set the language of the first audio stream: <pre class="example-preformatted">ffmpeg -i INPUT -metadata:s:a:0 language=eng OUTPUT </pre>

</dd> <dt><samp class="option">-keep_metadata[:<var class="var">metadata_specifier</var>] <var class="var">key</var> (output,per-metadata)</samp></dt> <dd> When re-encoding a stream, certain metadata keys that are likely to become stale (such as <code class="code">iTunSMPB</code> or <code class="code">encoded_by</code>) are automatically discarded. This option prevents a specific key from being discarded, preserving its original value from the input. Without a specifier, the key is preserved in all metadata contexts (format and streams). Use <code class="code">g</code> to restrict to format (global) metadata, or <code class="code">s</code> with an optional stream specifier to restrict to matching streams only. For example, to keep <code class="code">iTunSMPB</code> in format metadata only when re-encoding: <pre class="example-preformatted">ffmpeg -i in.mp3 -keep_metadata:g iTunSMPB out.mp3 </pre>

To keep <code class="code">NUMBER_OF_BYTES</code> only for the video stream: <pre class="example-preformatted">ffmpeg -i in.mkv -keep_metadata:s:v NUMBER_OF_BYTES out.mkv </pre>

Some keys are stored in the metadata with a suffix (e.g. <code class="code">NUMBER_OF_BYTES-eng</code>). Specifying the unsuffixed family name (e.g. <code class="code">NUMBER_OF_BYTES</code>) keeps all variants. To keep only a specific one, specify the full key explicitly:

 <pre class=”example-preformatted”>ffmpeg -i in.mkv -keep_metadata:s:v NUMBER_OF_BYTES-eng out.mkv </pre>

Some (such as <code class=”code”>major_brand</code>, <code class=”code”>minor_version</code>, and <code class=”code”>compatible_brands</code>) are always removed regardless of re-encoding, because they describe the source container and are meaningless in the output. <code class=”code”>-keep_metadata</code> applies to those keys even when stream-copying. Re-encoding-only keys (such as encoder tags and Matroska stream statistics) are removed from a stream’s metadata only when that stream is being re-encoded, and from format metadata only when at least one audio or video stream is re-encoded. Only format and stream metadata are filtered. Chapter and program metadata are never modified by this mechanism. </dd> <dt><samp class=”option”>-disposition[:stream_specifier] <var class=”var”>value</var> (output,per-stream)</samp></dt> <dd> Sets the disposition flags for a stream. Default value: by default, all disposition flags are copied from the input stream, unless the output stream this option applies to is fed by a complex filtergraph - in that case no disposition flags are set by default. <var class=”var”>value</var> is a sequence of disposition flags separated by ’+’ or ’-’. A ’+’ prefix adds the given disposition, ’-’ removes it. If the first flag is also prefixed with ’+’ or ’-’, the resulting disposition is the default value updated by <var class=”var”>value</var>. If the first flag is not prefixed, the resulting disposition is <var class=”var”>value</var>. It is also possible to clear the disposition by setting it to 0. If no <code class=”code”>-disposition</code> options were specified for an output file, ffmpeg will automatically set the ’default’ disposition flag on the first stream of each type, when there are multiple streams of this type in the output file and no stream of that type is already marked as default. The <code class=”code”>-dispositions</code> option lists the known disposition flags. For example, to make the second audio stream the default stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:1 default out.mkv </pre>

To make the second subtitle stream the default stream and remove the default disposition from the first subtitle stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:s:0 0 -disposition:s:1 default out.mkv </pre>

To add an embedded cover/thumbnail: <pre class=”example-preformatted”>ffmpeg -i in.mp4 -i IMAGE -map 0 -map 1 -c copy -c:v:1 png -disposition:v:1 attached_pic out.mp4 </pre>

To add the ’original’ and remove the ’comment’ disposition flag from the first audio stream without removing its other disposition flags: <pre class=”example-

preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 +original-comment out.mkv </pre>

To remove the ’original’ and add the ’comment’ disposition flag to the first audio stream without removing its other disposition flags: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 -original+comment out.mkv </pre>

To set only the ’original’ and ’comment’ disposition flags on the first audio stream (and remove its other disposition flags): <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 original+comment out.mkv </pre>

To remove all disposition flags from the first audio stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 0 out.mkv </pre>

Not all muxers support embedded thumbnails, and those who do, only support a few formats, like JPEG or PNG. </dd> <dt><samp class=”option”>-program [title=<var class=”var”>title</var>:][program_num=<var class=”var”>program_num</var>:]st=<var class=”var”>stream</var>[:st=<var class=”var”>stream</var>...] (output)</samp></dt> <dd> Creates a program with the specified <var class=”var”>title</var>, <var class=”var”>program_num</var> and adds the specified <var class=”var”>stream</var>(s) to it. </dd> <dt><samp class=”option”>-stream_group [map=<var class=”var”>input_file_id</var>=<var class=”var”>stream_group</var>][type
class=”var”>type</var>:]st=<var class=”var”>stream</var>[:st=<var class=”var”>stream</var>][:stg=<var class=”var”>stream_group</var>][:id=<var class=”var”>stream_group_id</var>...] (output)</samp></dt> <dd> Creates a stream group of the specified <var class=”var”>type</var> and <var class=”var”>stream_group_id</var>, or by <var class=”var”>map</var>ping an input group, adding the specified <var class=”var”>stream</var>(s) and/or previously defined <var class=”var”>stream_group</var>(s) to it. <var class=”var”>type</var> can be one of the following: <dl class=”table”> <dt><samp class=”option”>iamf_audio_element</samp></dt> <dd> Groups <var class=”var”>stream</var>s that belong to the same IAMF Audio Element For this group <var class=”var”>type</var>, the following options are available <dl class=”table”> <dt><samp class=”option”>audio_element_type</samp></dt> <dd> The Audio Element type. The following values are supported: <dl class=”table”> <dt><samp class=”option”>channel</samp></dt> <dd> Scalable channel audio representation </dd> <dt><samp class=”option”>scene</samp></dt> <dd> Ambisonics representation </dd> </dl>

</dd> <dt><samp class=”option”>demixing</samp></dt> <dd> Demixing information used to reconstruct a scalable channel audio representation. This

option must be separated from the rest with a ’,’, and takes the following key=value options <dl class="table"> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to </dd> <dt><samp class="option">dmixp_mode</samp></dt> <dd> A pre-defined combination of demixing parameters </dd> </dl>

</dd> <dt><samp class="option">recon_gain</samp></dt> <dd> Recon gain information used to reconstruct a scalable channel audio representation. This option must be separated from the rest with a ’,’, and takes the following key=value options <dl class="table"> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to </dd> </dl>

</dd> <dt><samp class="option">layer</samp></dt> <dd> A layer defining a Channel Layout in the Audio Element. This option must be separated from the rest with a ’,’. Several ’,’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options <dl class="table"> <dt><samp class="option">ch_layout</samp></dt> <dd> The layer’s channel layout </dd> <dt><samp class="option">flags</samp></dt> <dd> The following flags are available: <dl class="table"> <dt><samp class="option">recon_gain</samp></dt> <dd> Whether to signal if recon_gain is present as metadata in parameter blocks within frames </dd> </dl>

</dd> <dt><samp class="option">output_gain</samp></dt> <dt><samp class="option">output_gain_flags</samp></dt> <dd> Which channels output_gain applies to. The following flags are available: <dl class="table"> <dt><samp class="option">FL</samp></dt> <dt><samp class="option">FR</samp></dt> <dt><samp class="option">BL</samp></dt> <dt><samp class="option">BR</samp></dt> <dt><samp class="option">TFL</samp></dt> <dt><samp class="option">TFR</samp></dt> </dl>

</dd> <dt><samp class="option">ambisonics_mode</samp></dt> <dd> The ambisonics mode. This has no effect if audio_element_type is set to channel. The following values are supported: <dl class="table"> <dt><samp class="option">mono</samp></dt> <dd> Each ambisonics channel is coded as an individual mono stream in the group </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">default_w</samp></dt> <dd> Default weight value </dd> </dl>

</dd> <dt><samp class="option">iamf_mix_presentation</samp></dt> <dd> Groups <var class="var">stream</var>s that belong to all IAMF Audio Element the same IAMF Mix Presentation references For this

group <var class=”var”>type</var>, the following options are available <dl class=”table”> <dt><samp class=”option”>submix</samp></dt> <dd> A sub-mix within the Mix Presentation. This option must be separated from the rest with a ’,’. Several ’,’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options <dl class=”table”> <dt><samp class=”option”>parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to, for post-processing the mixed audio signal to generate the audio signal for playback </dd> <dt><samp class=”option”>parameter_rate</samp></dt> <dd> The sample rate duration fields in parameters blocks in frames that refer to this <var class=”var”>parameter_id</var> are expressed as </dd> <dt><samp class=”option”>default_mix_gain</samp></dt> <dd> Default mix gain value to apply when there are no parameter blocks sharing the same <var class=”var”>parameter_id</var> for a given frame </dd> <dt><samp class=”option”>element</samp></dt> <dd> References an Audio Element used in this Mix Presentation to generate the final output audio signal for playback. This option must be separated from the rest with a ’|’. Several ’|’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options: <dl class=”table”> <dt><samp class=”option”>stg</samp></dt> <dd> The <var class=”var”>stream_group_id</var> for an Audio Element which this sub-mix refers to </dd> <dt><samp class=”option”>parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to, for applying any processing to the referenced and rendered Audio Element before being summed with other processed Audio Elements </dd> <dt><samp class=”option”>parameter_rate</samp></dt> <dd> The sample rate duration fields in parameters blocks in frames that refer to this <var class=”var”>parameter_id</var> are expressed as </dd> <dt><samp class=”option”>default_mix_gain</samp></dt> <dd> Default mix gain value to apply when there are no parameter blocks sharing the same <var class=”var”>parameter_id</var> for a given frame </dd> <dt><samp class=”option”>annotations</samp></dt> <dd> A key=value string describing the sub-mix element where "key" is a string conforming to BCP-47 that specifies the language for the "value" string. "key" must be the same as the one in the mix’s <var class=”var”>annotations</var> </dd> <dt><samp class=”option”>headphones_rendering_mode</samp></dt> <dd> Indicates whether the input channel-based Audio Element is rendered to stereo loudspeakers or spatialized with a binaural renderer when played back on headphones. This has no effect if the referenced Audio Element’s <var class=”var”>audio_element_type</var> is set to channel. The following values are supported: <dl class=”table”> <dt><samp class=”option”>stereo</samp></dt> <dt><samp class=”option”>binaural</samp></dt> </dl>

</dd> </dl>

</dd> <dt><samp class=”option”>layout</samp></dt> <dd> Specifies the layouts for this sub-mix on which the loudness information was measured. This option must be separated from the rest with a ’|’. Several ’|’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options: <dl class=”table”> <dt><samp class=”option”>layout_type</samp></dt> <dd> <dl class=”table”> <dt><samp class=”option”>loudspeakers</samp></dt> <dd> The layout follows the loudspeaker sound system convention of ITU-2051-3. </dd> <dt><samp class=”option”>binaural</samp></dt> <dd> The layout is binaural. </dd> </dl>

</dd> <dt><samp class=”option”>sound_system</samp></dt> <dd> Channel layout matching one of Sound Systems A to J of ITU-2051-3, plus 7.1.2 and 3.1.2 This has no effect if <var class=”var”>layout_type</var> is set to binaural. </dd> <dt><samp class=”option”>integrated_loudness</samp></dt> <dd> The program integrated loudness information, as defined in ITU-1770-4. </dd> <dt><samp class=”option”>digital_peak</samp></dt> <dd> The digital (sampled) peak value of the audio signal, as defined in ITU-1770-4. </dd> <dt><samp class=”option”>true_peak</samp></dt> <dd> The true peak of the audio signal, as defined in ITU-1770-4. </dd> <dt><samp class=”option”>dialog_anchored_loudness</samp></dt> <dd> The Dialogue loudness information, as defined in ITU-1770-4. </dd> <dt><samp class=”option”>album_anchored_loudness</samp></dt> <dd> The Album loudness information, as defined in ITU-1770-4. </dd> </dl>

</dd> </dl>

</dd> <dt><samp class=”option”>annotations</samp></dt> <dd> A key=value string string describing the mix where "key" is a string conforming to BCP-47 that specifies the language for the "value" string. "key" must be the same as the ones in all sub-mix element’s <var class=”var”>annotations</var>s </dd> </dl>

</dd> <dt><samp class=”option”>gain_map</samp></dt> <dd> Groups together a base <var class=”var”>stream</var> with a gain map, which together form a pair of renditions (e.g. HDR and SDR) as per ISO 21496-1. For this group <var class=”var”>type</var>, the following options are available <dl class=”table”> <dt><samp class=”option”>el_index</samp></dt> <dd> Index of the gain map stream within the group. Defaults to 0. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Size of the final image for presentation. </dd> </dl>

</dd> </dl>

E.g. to create an scalable 5.1 IAMF file from several WAV input files <pre class="example-preformatted">ffmpeg -i front.wav -i back.wav -i center.wav -i lfe.wav -map 0:0 -map 1:0 -map 2:0 -map 3:0 -c:a opus -stream_group type=iamf_audio_element:id=1:st=0:st=1:st=2:st=3, demixing=parameter_id=998, recon_gain=parameter_id=101, layer=ch_layout=stereo, layer=ch_layout=5.1(side), -stream_group type=iamf_mix_presentation:id=2:stg=0:annotations=en-us=Mix_Presentation, submix=parameter_id=100:parameter_rate=48000|element=stg=0:parameter_id=10 us=Scalable_Submix|layout=sound_system=stereo|layout=sound_system=5.1(side) -streamid 0:0 -streamid 1:1 -streamid 2:2 -streamid 3:3 output.iamf </pre>

To copy the two stream groups (Audio Element and Mix Presentation) from an input IAMF file with four streams into an mp4 output <pre class="example-preformatted">ffmpeg -i input.iamf -c:a copy -stream_group map=0=0:st=0:st=1:st=2:st=3 -stream_group map=0=1:stg=0 -streamid 0:0 -streamid 1:1 -streamid 2:2 -streamid 3:3 output.mp4 </pre>

</dd> <dt><samp class="option">-reinit_opts[:<var class="var">stream_specifier</var>] pts=<var class="var">pts</var>|<var class="var">key</var>=<var class="var">value</var>[:force_reinit=1][:<var class="var">key</var>=<var class="var">value</var>,pts=<var class="var">pts</var>...] (output,per-stream)</samp></dt> <dd> Creates a "," separated group of options to reconfigure an encoder at the specified output <var class="var">pts</var> in AV_TIME_BASE units. The pseudo-option <var class="var">force_reinit</var> ensures the encoder is fully reinitialized instead of simply reconfigured. </dd> <dt><samp class="option">-target <var class="var">type</var> (output)</samp></dt> <dd> Specify target file type (<code class="code">vcd</code>, <code class="code">svcd</code>, <code class="code">dvd</code>, <code class="code">dv</code>, <code class="code">dv50</code>). <var class="var">type</var> may be prefixed with <code class="code">pal-</code>, <code class="code">ntsc-</code> or <code class="code">film-</code> to use the corresponding standard. All the format options (bitrate, codecs, buffer sizes) are then set automatically. You can just type: <pre class="example-preformatted">ffmpeg -i myfile.avi -target vcd /tmp/vcd.mpg </pre>

Nevertheless you can specify additional options as long as you know they do not conflict with the standard, as in: <pre class="example-preformatted">ffmpeg -i myfile.avi -target vcd -bf 2 /tmp/vcd.mpg </pre>

The parameters set for each target are as follows. VCD <pre class="example-preformatted"><var class="var">pal</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x288 -r 25 -codec:v mpeg1video -g 15 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v

327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k

<var class="var">ntsc</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x240 -r 30000/1001 -codec:v mpeg1video -g 18 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v 327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k

<var class="var">film</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x240 -r 24000/1001 -codec:v mpeg1video -g 18 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v 327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k </pre>

SVCD <pre class="example-preformatted"><var class="var">pal</var>: -f svcd -packetsize 2324 -s 480x576 -pix_fmt yuv420p -r 25 -codec:v mpeg2video -g 15 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k

<var class="var">ntsc</var>: -f svcd -packetsize 2324 -s 480x480 -pix_fmt yuv420p -r 30000/1001 -codec:v mpeg2video -g 18 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k

<var class="var">film</var>: -f svcd -packetsize 2324 -s 480x480 -pix_fmt yuv420p -r 24000/1001 -codec:v mpeg2video -g 18 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k </pre>

DVD <pre class="example-preformatted"><var class="var">pal</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x576 -pix_fmt yuv420p -r 25 -codec:v mpeg2video -g 15 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k

<var class="var">ntsc</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x480 -pix_fmt yuv420p -r 30000/1001 -codec:v mpeg2video -g 18 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k

<var class="var">film</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x480 -pix_fmt yuv420p -r 24000/1001 -codec:v mpeg2video -g 18 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k </pre>

DV <pre class="example-preformatted"><var class="var">pal</var>: -f dv -s 720x576 -pix_fmt yuv420p -r 25 -ar 48000 -ac 2

<var class="var">ntsc</var>: -f dv -s 720x480 -pix_fmt yuv411p -r 30000/1001 -ar 48000 -ac 2

<var class="var">film</var>:

-f dv

-s 720x480 -pix_fmt yuv411p -r 24000/1001

-ar 48000 -ac 2

</pre>

The <code class=”code”>dv50</code> target is identical to the <code class=”code”>dv</code> target except that the pixel format set is <code class=”code”>yuv422p</code> for all three standards.

Any user-set value for a parameter above will override the target preset value. In that case, the output may

not comply with the target standard.

</dd>

<dt><samp class=”option”>-dn (input/output)</samp></dt>

<dd> As an input option, blocks all data streams of a file from being filtered or

being automatically selected or mapped for any output. See <code class=”code”>-discard</code>

option to disable streams individually.

As an output option, disables data recording i.e. automatic selection or

mapping of any data stream. For full manual control see the <code class=”code”>-map</code>

option.

</dd>

<dt><samp class=”option”>-dframes <var class=”var”>number</var> (output)</samp></dt>

<dd> Set the number of data frames to output. This is an obsolete alias for

<code class=”code”>-frames:d</code>, which you should use instead.

</dd>

<dt><samp class="option">-frames[:<var class="var">stream_specifier</var>] <var class="var">framecount</var> (output,per-stream)</samp></dt>

<dd> Stop writing to the stream after <var class="var">framecount</var> frames.

</dd>

<dt><samp class="option">-q[:<var class="var">stream_specifier</var>] <var class="var">q</var> (output,per-stream)</samp></dt>

<dt><samp class="option">-qscale[:<var class="var">stream_specifier</var>] <var class="var">q</var> (output,per-stream)</samp></dt>

<dd> Use fixed quality scale (VBR). The meaning of <var class="var">q</var>/<var class="var">qscale</var> is

codec-dependent.

If <var class="var">qscale</var> is used without a <var class="var">stream_specifier</var> then it applies only

to the video stream, this is to maintain compatibility with previous behavior

and as specifying the same codec specific value to 2 different codecs that is

audio and video generally is not what is intended when no stream_specifier is

used.

</dd>

<dt><samp class="option">-filter[:<var class="var">stream_specifier</var>] <var class="var">filtergraph</var> (output,per-stream)</samp></dt>

<dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to

filter the stream.

<var class="var">filtergraph</var> is a description of the filtergraph to apply to

the stream, and must have a single input and a single output of the

same type of the stream. In the filtergraph, the input is associated

to the label <code class="code">in</code>, and the output to the label <code class="code">out</code>. See

the ffmpeg-filters manual for more information about the filtergraph

syntax.

See the -filter_complex option if you

want to create filtergraphs with multiple inputs and/or outputs.

</dd>

<dt><samp class="option">-reinit_filter[:<var class="var">stream_specifier</var>] <var class="var">integer</var> (input,per-stream)</samp></dt>

<dd> This boolean option determines if the filtergraph(s) to which this stream is fed gets

reinitialized when input frame parameters change mid-stream. This option is enabled by

default as most video and all audio filters cannot handle deviation in input frame properties.

Upon reinitialization, existing filter state is lost, like e.g. the frame count <code class="code">n</code>

reference available in some filters. Any frames buffered at time of reinitialization are lost.

The properties where a change triggers reinitialization are,

for video, frame resolution or pixel format;

for audio, sample format, sample rate, channel count or channel layout.

</dd>

<dt><samp class="option">-drop_changed[:<var class="var">stream_specifier</var>] <var class="var">integer</var> (input,per-stream)</samp></dt>

<dd> This boolean option determines whether a frame with differing frame parameters mid-stream

gets dropped instead of leading to filtergraph reinitialization, as that would lead to loss

of filter state. Generally useful to avoid corrupted yet decodable packets in live streaming

inputs. Default is false.

</dd>

<dt><samp class="option">-filter_threads <var class="var">nb_threads</var> (global)</samp></dt>

<dd> Defines how many threads are used to process a filter pipeline. Each pipeline

will produce a thread pool with this many threads available for parallel processing.

The default is the number of available CPUs.

</dd>

<dt><samp class="option">-filter_buffered_frames[:<var class="var">media_type</var>] <var class="var">nb_frames</var> (global)</samp></dt>

<dd> Defines the maximum number of buffered frames allowed in a filtergraph. Under

normal circumstances, a filtergraph should not buffer more than a few frames,

especially if frames are being fed to it and read from it in a balanced way

(which is the intended behavior in ffmpeg). That said, this option allows you

to limit the total number of frames buffered across all links in a filtergraph.

If more frames are generated, filtering is aborted and an error is returned.

The default value is 0, which means no limit.

The optional suffix <var class="var">media_type</var> may be used to set a different limit

for audio and video frames, using the suffixes <code class="code">a</code> and <code class="code">v</code>,

respectively. If the suffix is omitted, the limit applies to both audio and

video frames, separately.

</dd>

<dt><samp class="option">-pre[:<var class="var">stream_specifier</var>] <var class="var">preset_name</var> (output,per-stream)</samp></dt>

<dd> Specify the preset for matching stream(s).

</dd>

<dt><samp class="option">-stats (global)</samp></dt>

<dd> Log encoding progress/statistics as "info"-level log (see <code class="code">-loglevel</code>).

It is on by default, to explicitly disable it you need to specify <code class="code">-nostats</code>.

</dd>

<dt><samp class="option">-stats_period <var class="var">time</var> (global)</samp></dt>

<dd> Set period at which encoding progress/statistics are updated. Default is 0.5 seconds.

</dd>

<dt><samp class="option">-print_graphs (global)</samp></dt>

<dd> Prints execution graph details to stderr in the format set via -print_graphs_format.

</dd>

<dt><samp class="option">-print_graphs_file <var class="var">filename</var> (global)</samp></dt>

<dd> Writes execution graph details to the specified file in the format set via -print_graphs_format.

</dd>

<dt><samp class="option">-print_graphs_format <var class="var">format</var> (global)</samp></dt>

<dd> Sets the output format (available formats are: default, compact, csv, flat, ini, json, xml, mermaid, mermaidhtml)

The default format is json.

</dd>

<dt><samp class="option">-progress <var class="var">url</var> (global)</samp></dt>

<dd> Send program-friendly progress information to <var class="var">url</var>.

Progress information is written periodically and at the end of

the encoding process. It is made of "<var class="var">key</var>=<var class="var">value</var>" lines. <var class="var">key</var>

consists of only alphanumeric characters. The last key of a sequence of

progress information is always "progress" with the value "continue" or "end".

The update period is set using <code class="code">-stats_period</code>.

For example, log progress information to stdout:

<pre class="example-preformatted">ffmpeg -progress pipe:1 -i in.mkv out.mkv

</pre>

</dd> <dt><samp class="option">-stdin</samp></dt> <dd> Enable interaction on standard input. On by default unless standard input is used as an input. To explicitly disable interaction you need to specify <code class="code">-nostdin</code>. Disabling interaction on standard input is useful, for example, if ffmpeg is in the background process group. Roughly the same result can be achieved with <code class="code">ffmpeg ... < /dev/null</code> but it requires a shell. </dd> <dt><samp class="option">-debug_ts (global)</samp></dt> <dd> Print timestamp/latency information. It is off by default. This option is mostly useful for testing and debugging purposes, and the output format may change from one version to another, so it should not be employed by portable scripts. See also the option <code class="code">-fdebug ts</code>. </dd> <dt><samp class="option">-attach <var class="var">filename</var> (output)</samp></dt> <dd> Add an attachment to the output file. This is supported by a few formats like Matroska for e.g. fonts used in rendering subtitles. Attachments are implemented as a specific type of stream, so this option will add a new stream to the file. It is then possible to use per-stream options on this stream in the usual way. Attachment streams created with this option will be created after all the

other streams (i.e. those created with <code class=”code”>-map</code> or automatic mappings). Note that for Matroska you also have to set the mimetype metadata tag: <pre class=”example-preformatted”>ffmpeg -i INPUT -attach DejaVuSans.ttf -metadata:s:2 mimetype=application/x-truetype-font out.mkv </pre> (assuming that the attachment stream will be third in the output file). </dd> <dt><samp class=”option”>-dump_attachment[:<var class=”var”>stream_specifier</var>] <var class=”var”>filename</var> (input,per-stream)</samp></dt> <dd> Extract the matching attachment stream into a file named <var class=”var”>filename</var>. If <var class=”var”>filename</var> is empty, then the value of the <code class=”code”>filename</code> metadata tag will be used. E.g. to extract the first attachment to a file named ’out.ttf’: <pre class=”example-preformatted”>ffmpeg -dump_attachment:t:0 out.ttf -i INPUT </pre> To extract all attachments to files determined by the <code class=”code”>filename</code> tag: <pre class=”example-preformatted”>ffmpeg -dump_attachment:t "" -i INPUT </pre>

Technical note – attachments are implemented as codec extradata, so this option can actually be used to extract extradata from any stream, not just attachments. </dd> </dl>

 <h3 class=”section”>5.5 Video Options# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-vframes <var class=”var”>number</var> (output)</samp></dt> <dd> Set the number of video frames to output. This is an obsolete alias for <code class=”code”>-frames:v</code>, which you should use instead. </dd> <dt><samp class=”option”>-r[:<var class=”var”>stream_specifier</var>] <var class=”var”>fps</var> (input/output,per-stream)</samp></dt> <dd> Set frame rate (Hz value, fraction or abbreviation). As an input option, ignore any timestamps stored in the file and instead generate timestamps assuming constant frame rate <var class=”var”>fps</var>. This is not the same as the <samp class=”option”>-framerate</samp> option used for some input formats like image2 or v4l2 (it used to be the same in older versions of FFmpeg). If in doubt use <samp class=”option”>-framerate</samp> instead of the input option <samp class=”option”>-r</samp>. As an output option: <dl class=”table”> <dt><samp class=”option”>video encoding</samp></dt> <dd> Duplicate or drop frames right before encoding them to achieve constant output frame rate <var class=”var”>fps</var>. </dd> <dt><samp class=”option”>video streamcopy</samp></dt> <dd> Indicate to the muxer that <var class=”var”>fps</var> is the

stream frame rate. No data is dropped or duplicated in this case. This may produce invalid files if <var class=”var”>fps</var> does not match the actual stream frame rate as determined by packet timestamps. See also the <code class=”code”>setts</code> bitstream filter. </dd> </dl>

</dd> <dt><samp class=”option”>-fpsmax[:<var class=”var”>stream_specifier</var>] <var class=”var”>fps</var> (output,per-stream)</samp></dt> <dd> Set maximum frame rate (Hz value, fraction or abbreviation). Clamps output frame rate when output framerate is auto-set and is higher than this value. Useful in batch processing or when input framerate is wrongly detected as very high. It cannot be set together with <code class=”code”>-r</code>. It is ignored during streamcopy. </dd> <dt><samp class=”option”>-s[:<var class=”var”>stream_specifier</var>] <var class=”var”>size</var> (input/output,per-stream)</samp></dt> <dd> Set frame size. As an input option, this is a shortcut for the <samp class=”option”>video_size</samp> private option, recognized by some demuxers for which the frame size is either not stored in the file or is configurable – e.g. raw video or video grabbers. As an output option, this inserts the <code class=”code”>scale</code> video filter to the end of the corresponding filtergraph. Please use the <code class=”code”>scale</code> filter directly to insert it at the beginning or some other place. The format is ‘<samp class=”samp”>wxh</samp>’ (default - same as source). </dd> <dt><samp class=”option”>-aspect[:<var class=”var”>stream_specifier</var>] <var class=”var”>aspect</var> (output,per-stream)</samp></dt> <dd> Set the video display aspect ratio specified by <var class=”var”>aspect</var>. <var class=”var”>aspect</var> can be a floating point number string, or a string of the form <var class=”var”>num</var>:<var class=”var”>den</var>, where <var class=”var”>num</var> and <var class=”var”>den</var> are the numerator and denominator of the aspect ratio. For example "4:3", "16:9", "1.3333", and "1.7777" are valid argument values. If used together with <samp class=”option”>-vcodec copy</samp>, it will affect the aspect ratio stored at container level, but not the aspect ratio stored in encoded frames, if it exists. </dd> <dt><samp class=”option”>-stereo3d[:<var class=”var”>stream_specifier</var>] <var class=”var”>layout</var> (output,per-stream)</samp></dt> <dd> Set stereoscopic 3D layout metadata on the output stream. This does not rearrange pixels; use <samp class=”option”>-vf stereo3d</samp> for that. The metadata is stored as <code class=”code”>AV_PKT_DATA_STEREO3D</code> side data on the output stream. Muxers that already consume that side data will write the corresponding container boxes or elements, including: <ul class=”itemize mark-bullet”> MP4 Google spatial-media

<code class=”code”>st3d</code> (requires <samp class=”option”>-strict unofficial</samp>) QuickTime MOV Apple <code class=”code”>vexu</code> eyes (no <code class=”code”>st3d</code>) Matroska <code class=”code”>StereoMode</code>; <samp class=”option”>-stereo3d</samp> overrides the <code class=”code”>stereo_mode</code> metadata tag, including a value inherited from the input

<var class=”var”>layout</var> is case-insensitive. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>2d</samp>’</dt> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> Monoscopic video. </dd> <dt>‘<samp class=”samp”>sbs</samp>’</dt> <dt>‘<samp class=”samp”>sbsl</samp>’</dt> <dt>‘<samp class=”samp”>side by side</samp>’</dt> <dd> Side-by-side, left eye on the left. </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dt>‘<samp class=”samp”>tbl</samp>’</dt> <dt>‘<samp class=”samp”>top and bottom</samp>’</dt> <dd> Top-and-bottom, left eye on top. </dd> </dl>

Inverted layouts such as ‘<samp class=”samp”>sbsr</samp>’ are not supported. An empty stream specifier applies to video streams only and is ignored for audio and other types. An explicit specifier on a non-video stream (for example <samp class=”option”>-stereo3d:a</samp>) is an error. Recommended form: <samp class=”option”>-stereo3d:v sbsl</samp>. </dd> <dt><samp class=”option”>-display_rotation[:<var class=”var”>stream_specifier</var>] <var class=”var”>rotation</var> (input,per-stream)</samp></dt> <dd> Set video rotation metadata. <var class=”var”>rotation</var> is a decimal number specifying the amount in degree by which the video should be rotated counter-clockwise before being displayed. This option overrides the rotation/display transform metadata stored in the file, if any. When the video is being transcoded (rather than copied) and <code class=”code”>-autorotate</code> is enabled, the video will be rotated at the filtering stage. Otherwise, the metadata will be written into the output file if the muxer supports it. If the <code class=”code”>-display_hflip</code> and/or <code class=”code”>-display_vflip</code> options are given, they are applied after the rotation specified by this option. </dd> <dt><samp class=”option”>-display_hflip[:<var class=”var”>stream_specifier</var>] (input,per-stream)</samp></dt> <dd> Set whether on display the image should be horizontally flipped. See the <code class=”code”>-display_rotation</code> option for more details. </dd> <dt><samp class=”option”>-display_vflip[:<var class=”var”>stream_specifier</var>] (input,per-stream)</samp></dt> <dd> Set whether on display the image should be vertically flipped. See the <code class=”code”>-display_rotation</code> option for more details. </dd> <dt><samp class=”option”>-mastering_display[:<var class=”var”>stream_specifier</var>] <var class=”var”>G(%u,%u)B(%u,%u)R(%u,%u)WP(%u,%u)L(%u,%

(input,per-stream)</samp></dt> <dd> Set video mastering display metadata. <var class="var">G(%u,%u)B(%u,%u)R(%u,%u)WP(%u,%u)L is a string specifying X,Y display primaries for GBR channels and white point (WP) in units of 0.00002, and max-min luminance (L) values in units of 0.0001 candela per meter square. The values are unsigned integers representing the numerator of a rational with an implicit denominator of 50000 for GBR and (WP), and implicit denominator 10000 for (L). This option overrides the mastering display metadata stored in the file, if any. </dd> <dt><samp class="option">-content_light[:<var class="var">stream_specifier</var>] <var class="var">%u,%u</var> (input,per-stream)</samp></dt> <dd> Set video content light metadata. <var class="var">%u,%u</var> is a string specifying max content light level and maximum picture average light level. This option overrides the content light metadata stored in the file, if any. </dd> <dt><samp class="option">-vn (input/output)</samp></dt> <dd> As an input option, blocks all video streams of a file from being filtered or being automatically selected or mapped for any output. See <code class="code">-discard</code> option to disable streams individually. As an output option, disables video recording i.e. automatic selection or mapping of any video stream. For full manual control see the <code class="code">-map</code> option. </dd> <dt><samp class="option">-vcodec <var class="var">codec</var> (output)</samp></dt> <dd> Set the video codec. This is an alias for <code class="code">-codec:v</code>. </dd> <dt><samp class="option">-pass[:<var class="var">stream_specifier</var>] <var class="var">n</var> (output,per-stream)</samp></dt> <dd> Select the pass number (1 or 2). It is used to do two-pass video encoding. The statistics of the video are recorded in the first pass into a log file (see also the option -passlogfile), and in the second pass that log file is used to generate the video at the exact requested bitrate. On pass 1, you may just deactivate audio and set output to null, examples for Windows and Unix: <pre class="example-preformatted">ffmpeg -i foo.mov -c:v libxvid -pass 1 -an -f rawvideo -y NUL ffmpeg -i foo.mov -c:v libxvid -pass 1 -an -f rawvideo -y /dev/null </pre>

</dd> <dt><samp class="option">-passlogfile[:<var class="var">stream_specifier</var>] <var class="var">prefix</var> (output,per-stream)</samp></dt> <dd> Set two-pass log file name prefix to <var class="var">prefix</var>, the default file name prefix is “ffmpeg2pass”. The complete file name will be <samp class="file">PREFIX-N.log</samp>, where N is a number specific to the output stream </dd> <dt><samp class="option">-vf <var class="var">filtergraph</var> (output)</samp></dt> <dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to filter the stream. This is an alias for <code class="code">-filter:v</code>, see the -

filter option. </dd> <dt><samp class="option">-autorotate</samp></dt> <dd> Automatically rotate the video according to file metadata. Enabled by default, use <samp class="option">-noautorotate</samp> to disable it. </dd> <dt><samp class="option">-autoscale</samp></dt> <dd> Automatically scale the video according to the resolution of first frame. Enabled by default, use <samp class="option">-noautoscale</samp> to disable it. When autoscale is disabled, all output frames of filter graph might not be in the same resolution and may be inadequate for some encoder/muxer. Therefore, it is not recommended to disable it unless you really know what you are doing. Disable autoscale at your own risk. </dd> </dl>

 <h3 class="section">5.6 Advanced Video options# TOC</h3>

<dl class="table"> <dt><samp class="option">-pix_fmt[:<var class="var">stream_specifier</var>] <var class="var">format</var> (input/output,per-stream)</samp></dt> <dd> Set pixel format. Use <code class="code">-pix_fmts</code> to show all the supported pixel formats. If the selected pixel format can not be selected, ffmpeg will print a warning and select the best pixel format supported by the encoder. If <var class="var">pix_fmt</var> is prefixed by a <code class="code">+</code>, ffmpeg will exit with an error if the requested pixel format can not be selected, and automatic conversions inside filtergraphs are disabled. If <var class="var">pix_fmt</var> is a single <code class="code">+</code>, ffmpeg selects the same pixel format as the input (or graph output) and automatic conversions are disabled. </dd> <dt><samp class="option">-sws_flags <var class="var">flags</var> (input/output)</samp></dt> <dd> Set default flags for the libswscale library. These flags are used by automatically inserted <code class="code">scale</code> filters and those within simple filtergraphs, if not overridden within the filtergraph definition. See the <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#scaler_005foptions">ffmpeg-scaler manual for a list of scaler options. </dd> <dt><samp class="option">-rc_override[:<var class="var">stream_specifier</var>] <var class="var">override</var> (output,per-stream)</samp></dt> <dd> Rate control override for specific intervals, formatted as "int,int,int" list separated with slashes. Two first values are the beginning and end frame numbers, last one is quantizer to use if positive, or quality factor if negative. </dd> <dt><samp class="option">-vstats</samp></dt> <dd> Dump video coding statistics to <samp class="file">vstats_HHMMSS.log</samp>. See the vstats file format section for the format description. </dd> <dt><samp class="option">-vstats_file <var class="var">file</var></samp></dt>

<dd> Dump video coding statistics to <var class="var">file</var>. See the vstats file format section for the format description. </dd> <dt><samp class="option">-vstats_version <var class="var">file</var></samp></dt> <dd> Specify which version of the vstats format to use. Default is <code class="code">2</code>. See the vstats file format section for the format description. </dd> <dt><samp class="option">-vtag <var class="var">fourcc/tag</var> (output)</samp></dt> <dd> Force video tag/fourcc. This is an alias for <code class="code">-tag:v</code>. </dd> <dt><samp class="option">-force_key_frames[:<var class="var">stream_specifier</var>] <var class="var">time</var>[,<var class="var">time</var>...] (output,per-stream)</samp></dt> <dt><samp class="option">-force_key_frames[:<var class="var">stream_specifier</var>] expr:<var class="var">expr</var> (output,per-stream)</samp></dt> <dt><samp class="option">-force_key_frames[:<var class="var">stream_specifier</var>] source (output,per-stream)</samp></dt> <dt><samp class="option">-force_key_frames[:<var class="var">stream_specifier</var>] scd_metadata (output,per-stream)</samp></dt> <dd> <var class="var">force_key_frames</var> can take arguments of the following form: <dl class="table"> <dt><samp class="option"><var class="var">time</var>[,<var class="var">time</var>...]</samp></d
<dd> If the argument consists of timestamps, ffmpeg will round the specified times to the nearest output timestamp as per the encoder time base and force a keyframe at the first frame having timestamp equal or greater than the computed timestamp. Note that if the encoder time base is too coarse, then the keyframes may be forced on frames with timestamps lower than the specified time. The default encoder time base is the inverse of the output framerate but may be set otherwise via <code class="code">-enc_time_base</code>. If one of the times is "<code class="code">chapters</code>[<var class="var">delta</var>]", it is expanded into the time of the beginning of all chapters in the file, shifted by <var class="var">delta</var>, expressed as a time in seconds. This option can be useful to ensure that a seek point is present at a chapter mark or any other designated place in the output file. For example, to insert a key frame at 5 minutes, plus key frames 0.1 second before the beginning of every chapter: <pre class="example-preformatted">-force_key_frames 0:05:00,chapters-0.1 </pre>

</dd> <dt><samp class="option">expr:<var class="var">expr</var></samp></dt> <dd> If the argument is prefixed with <code class="code">expr:</code>, the string <var class="var">expr</var> is interpreted like an expression and is evaluated for each frame. A key frame is forced in case the evaluation is non-zero. The expression in <var class="var">expr</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> the

number of current processed frame, starting from 0 </dd> <dt><samp class="option">n_forced</samp></dt> <dd> the number of forced frames </dd> <dt><samp class="option">prev_forced_n</samp></dt> <dd> the number of the previous forced frame, it is <code class="code">NAN</code> when no keyframe was forced yet </dd> <dt><samp class="option">prev_forced_t</samp></dt> <dd> the time of the previous forced frame, it is <code class="code">NAN</code> when no keyframe was forced yet </dd> <dt><samp class="option">t</samp></dt> <dd> the time of the current processed frame </dd> </dl>

For example to force a key frame every 5 seconds, you can specify: <pre class="example-preformatted">-force_key_frames expr:gte(t,n_forced*5) </pre>

To force a key frame 5 seconds after the time of the last forced one, starting from second 13: <pre class="example-preformatted">-force_key_frames expr:if(isnan(prev_forced_t),gte(t,13),gte(t,prev_forced_t+5)) </pre>

</dd> <dt><samp class="option">source</samp></dt> <dd> If the argument is <code class="code">source</code>, ffmpeg will force a key frame if the current frame being encoded is marked as a key frame in its source. In cases where this particular source frame has to be dropped, enforce the next available frame to become a key frame instead. </dd> <dt><samp class="option">scd_metadata</samp></dt> <dd> If the argument is <code class="code">scd_metadata</code>, ffmpeg will force a key frame if the current frame contains a metadata entry with the key <code class="code">lavfi.scd.time</code>. The metadata can be added by filters like <code class="code">scdet</code> and <code class="code">scdet_vulkan</code>. Avoid inserting filters that duplicate frames after <code class="code">scdet</code>, as this can cause duplicate metadata for multiple frames and repeated insertion of key frames. </dd> </dl>

Note that forcing too many keyframes is very harmful for the lookahead algorithms of certain encoders: using fixed-GOP options or similar would be more efficient. </dd> <dt><samp class="option">-apply_cropping[:<var class="var">stream_specifier</var>] <var class="var">source</var> (input,per-stream)</samp></dt> <dd> Automatically crop the video after decoding according to file metadata. Default is all. <dl class="table"> <dt><samp class="option">none (0)</samp></dt> <dd> Don’t apply any cropping metadata. </dd> <dt><samp class="option">all (1)</samp></dt> <dd> Apply both codec and container level croppping. This is the default mode. </dd> <dt><samp class="option">codec (2)</samp></dt> <dd> Apply codec level croppping. </dd> <dt><samp class="option">container (3)</samp></dt> <dd> Apply container level croppping. </dd> </dl>

</dd> <dt><samp class="option">-copyinkf[:<var class="var">stream_specifier</var>]
(output,per-stream)</samp></dt> <dd> When
doing stream copy, copy also non-key frames found at the beginning.
</dd> <dt><samp class="option">-init_hw_device <var class="var">type</var>[=<var
class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">type</var>
called <var class="var">name</var>, using the given device parameters. If no name is specified it will receive a default name of the form
"<var class="var">type</var>%d". The meaning of <var class="var">device</var> and the following arguments
depends on the device type: <dl class="table"> <dt><samp
class="option">cuda</samp></dt> <dd> <var class="var">device</var>
is the number of the CUDA device. The following options are recognized: <dl class="table"> <dt><samp class="option">primary_ctx</samp></dt>
<dd> If set to 1, uses the primary device context instead of creating a new
one. </dd> </dl>

Examples: <dl class="table"> <dt>-
init_hw_device cuda:1</dt> <dd> Choose the second device
on the system. </dd> <dt>-init_hw_device
cuda:0,primary_ctx=1</dt> <dd> Choose the first device and
use the primary device context. </dd> </dl>

</dd> <dt><samp class="option">dxva2</samp></dt> <dd> <var
class="var">device</var> is the number of the Direct3D 9 display
adapter. </dd> <dt><samp class="option">d3d11va</samp></dt>
<dd> <var class="var">device</var> is the number of the Direct3D 11
display adapter. If not specified, it will attempt to use the default Direct3D 11
display adapter or the first Direct3D 11 display adapter whose hardware VendorId is specified by ‘<samp class="samp">vendor_id</samp>’.
 Examples: <dl class="table"> <dt>-
init_hw_device d3d11va</dt> <dd> Create a d3d11va device
on the default Direct3D 11 display adapter. </dd> <dt><em
class="emph">-init_hw_device d3d11va:1</dt> <dd> Create
a d3d11va device on the Direct3D 11 display adapter specified by
index 1. </dd> <dt>-init_hw_device
d3d11va:,vendor_id=0x8086</dt> <dd> Create a d3d11va
device on the first Direct3D 11 display adapter whose hardware VendorId is
0x8086. </dd> </dl>

</dd> <dt><samp class="option">vaapi</samp></dt> <dd> <var
class="var">device</var> is either an X11 display name, a DRM render
node or a DirectX adapter index. If not specified, it will attempt to open the
default X11 display ($DISPLAY) and then the
first DRM render node (/dev/dri/renderD128),
or the default DirectX adapter on Windows. The following options are recognized: <dl class="table"> <dt><samp

class=”option”>kernel_driver</samp></dt> <dd> When <var class=”var”>device</var> is not specified, use this option to specify the name of the kernel driver associated with the desired device. This option is available only when the hardware acceleration method drm and vaapi are enabled. </dd> <dt><samp class=”option”>vendor_id</samp></dt> <dd> When <var class=”var”>device</var> and <var class=”var”>kernel_driver</var> are not specified, use this option to specify the vendor id associated with the desired device. This option is available only when the hardware acceleration method drm and vaapi are enabled and kernel_driver is not specified. </dd> </dl>

Examples: <dl class=”table”> <dt>-init_hw_device vaapi</dt> <dd> Create a vaapi device on the default device. </dd> <dt>-init_hw_device vaapi:/dev/dri/renderD129</dt> <dd> Create a vaapi device on DRM render node <samp class=”file”>/dev/dri/renderD129</samp>. </dd> <dt>-init_hw_device vaapi:1</dt> <dd> Create a vaapi device on DirectX adapter 1. </dd> <dt>-init_hw_device vaapi:,kernel_driver=i915</dt> <dd> Create a vaapi device on a device associated with kernel driver ‘<samp class=”samp”>i915</samp>’. </dd> <dt>-init_hw_device vaapi:,vendor_id=0x8086</dt> <dd> Create a vaapi device on a device associated with vendor id ‘<samp class=”samp”>0x8086</samp>’. </dd> </dl>

</dd> <dt><samp class=”option”>vdpau</samp></dt> <dd> <var class=”var”>device</var> is an X11 display name. If not specified, it will attempt to open the default X11 display ($DISPLAY). </dd> <dt><samp class=”option”>qsv</samp></dt> <dd> <var class=”var”>device</var> selects a value in ‘<samp class=”samp”>MFX_IMPL_*</samp>’. Allowed values are: <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dt><samp class=”option”>sw</samp></dt> <dt><samp class=”option”>hw</samp></dt> <dt><samp class=”option”>auto_any</samp></dt> <dt><samp class=”option”>hw_any</samp></dt> <dt><samp class=”option”>hw2</samp></dt> <dt><samp class=”option”>hw3</samp></dt> <dt><samp class=”option”>hw4</samp></dt> </dl> If not specified, ‘<samp class=”samp”>auto_any</samp>’ is used. (Note that it may be easier to achieve the desired result for QSV by creating the platform-appropriate subdevice (‘<samp class=”samp”>dxva2</samp>’ or ‘<samp class=”samp”>d3d11va</samp>’ or ‘<samp class=”samp”>vaapi</samp>’) and then deriving a QSV device from that.) The following options are recognized: <dl class=”table”> <dt><samp class=”option”>child_device</samp></dt> <dd> Specify a DRM render node on Linux or DirectX adapter on Windows. </dd> <dt><samp class=”option”>child_device_type</samp></dt> <dd> Choose platform-appropriate subdevice type. On Windows

‘<samp class="samp">d3d11va</samp>’ is used as default subdevice type when <code class="code">--enable-libvpl</code> is specified at configuration time, ‘<samp class="samp">dxva2</samp>’ is used as default subdevice type when <code class="code">--enable-libmfx</code> is specified at configuration time. On Linux user can use ‘<samp class="samp">vaapi</samp>’ only as subdevice type. </dd> </dl>

Examples: <dl class="table"> <dt>-init_hw_device qsv:hw,child_device=/dev/dri/renderD129</dt> <dd> Create a QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>&rsq on DRM render node <samp class="file">/dev/dri/renderD129</samp>. </dd> <dt>-init_hw_device qsv:hw,child_device=1</dt> <dd> Create a QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>&rsq on DirectX adapter 1. </dd> <dt>-init_hw_device qsv:hw,child_device_type=d3d11va</dt> <dd> Choose the GPU subdevice with type ‘<samp class="samp">d3d11va</samp>’ and create QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>’. </dd> <dt>-init_hw_device qsv:hw,child_device_type=dxva2</dt> <dd> Choose the GPU subdevice with type ‘<samp class="samp">dxva2</samp>’ and create QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>’. </dd> <dt>-init_hw_device qsv:hw,child_device=1,child_device_type=d3d11va <dd> Create a QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>&rsq on DirectX adapter 1 with subdevice type ‘<samp class="samp">d3d11va</samp>’. </dd> <dt>-init_hw_device vaapi=va:/dev/dri/renderD129 -init_hw_device qsv=hw1@<var class="var">va</var></dt> <dd> Create a VAAPI device called ‘<samp class="samp">va</samp>’ on <samp class="file">/dev/dri/renderD129</samp>, then derive a QSV device called ‘<samp class="samp">hw1</samp>’ from device ‘<samp class="samp">va</samp>’. </dd> </dl>

</dd> <dt><samp class="option">opencl</samp></dt> <dd> <var class="var">device</var> selects the platform and device as platform_index.device_index. The set of devices can also be filtered using the key-value pairs to find only devices matching particular platform or device strings. The strings usable as filters are: <dl class="table"> <dt><samp class="option">platform_profile</samp></dt> <dt><samp class="option">platform_version</samp><
<dt><samp class="option">platform_name</samp></dt> <dt><samp class="option">platform_vendor</samp></dt> <dt><samp class="option">platform_extensions</samp
<dt><samp class="option">device_name</samp></dt> <dt><samp class="option">device_vendor</samp></dt> <dt><samp class="option">driver_version</samp></dt>
<dt><samp class="option">device_version</samp></dt> <dt><samp class="option">device_profile</samp></dt> <dt><samp class="option">device_extensions</samp></d
<dt><samp class="option">device_type</samp></dt> </dl>

The indices and filters must together uniquely select a device.

Examples: <dl class="table"> <dt>-init_hw_device opencl:0.1</dt> <dd> Choose the second device on the first platform. </dd> <dt>-init_hw_device opencl:,device_name=Foo9000</dt> <dd> Choose the device with a name containing the string Foo9000. </dd> <dt>-init_hw_device opencl:1,device_type=gpu,device_extensions=cl_khr_fp1
<dd> Choose the GPU device on the second platform supporting the cl_khr_fp16 extension. </dd> </dl>

</dd> <dt><samp class="option">vulkan</samp></dt> <dd> If <var class="var">device</var> is an integer, it selects the device by its index in a system-dependent list of devices. If <var class="var">device</var> is any other string, it selects the first device with a name containing that string as a substring. The following options are recognized: <dl class="table"> <dt><samp class="option">debug</samp></dt> <dd> If set to 1, enables the validation layer, if installed. </dd> <dt><samp class="option">linear_images</samp></dt> <dd> If set to 1, images allocated by the hwcontext will be linear and locally mappable. </dd> <dt><samp class="option">instance_extensions</samp></dt> <dd> A plus separated list of additional instance extensions to enable. </dd> <dt><samp class="option">device_extensions</samp></dt> <dd> A plus separated list of additional device extensions to enable. </dd> </dl>

Examples: <dl class="table"> <dt>-init_hw_device vulkan:1</dt> <dd> Choose the second device on the system. </dd> <dt>-init_hw_device vulkan:RADV</dt> <dd> Choose the first device with a name containing the string RADV. </dd> <dt>-init_hw_device vulkan:0,instance_extensions=VK_KHR_wayland_surface+VK_
<dd> Choose the first device and enable the Wayland and XCB instance extensions. </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">-init_hw_device <var class="var">type</var>[=<var class="var">name</var>]@<var class="var">source</var></samp></dt> <dd> Initialise a new hardware device of type <var class="var">type</var> called <var class="var">name</var>, deriving it from the existing device with the name <var class="var">source</var>. </dd> <dt><samp class="option">-init_hw_device list</samp></dt> <dd> List all hardware device types supported in this build of ffmpeg. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. This can be used to set the device to upload to with the <code class="code">hwupload</code> filter, or the device to map to with the <code class="code">hwmap</code> filter. Other filters may also make use of this parameter when they require a hardware device. Note that this is typ-

ically only required when the input is not already in hardware frames - when it is, filters will derive the device they require from the context of the frames they receive as input. This is a global setting, so all filters will receive the same device. </dd> <dt><samp class="option">-hwaccel[:<var class="var">stream_specifier</var>] <var class="var">hwaccel</var> (input,per-stream)</samp></dt> <dd> Use hardware acceleration to decode the matching stream(s). The allowed values of <var class="var">hwaccel</var> are: <dl class="table"> <dt><samp class="option">none</samp></dt> <dd> Do not use any hardware acceleration (the default). </dd> <dt><samp class="option">auto</samp></dt> <dd> Automatically select the hardware acceleration method. </dd> <dt><samp class="option">vdpau</samp></dt> <dd> Use VDPAU (Video Decode and Presentation API for Unix) hardware acceleration. </dd> <dt><samp class="option">dxva2</samp></dt> <dd> Use DXVA2 (DirectX Video Acceleration) hardware acceleration. </dd> <dt><samp class="option">d3d11va</samp></dt> <dd> Use D3D11VA (DirectX Video Acceleration) hardware acceleration. </dd> <dt><samp class="option">vaapi</samp></dt> <dd> Use VAAPI (Video Acceleration API) hardware acceleration. </dd> <dt><samp class="option">qsv</samp></dt> <dd> Use the Intel QuickSync Video acceleration for video transcoding. Unlike most other values, this option does not enable accelerated decoding (that is used automatically whenever a qsv decoder is selected), but accelerated transcoding, without copying the frames into the system memory. For it to work, both the decoder and the encoder must support QSV acceleration and no filters must be used. </dd> <dt><samp class="option">videotoolbox</samp></dt> <dd> Use Video Toolbox hardware acceleration. </dd> </dl>

This option has no effect if the selected hwaccel is not available or not supported by the chosen decoder. Note that most acceleration methods are intended for playback and will not be faster than software decoding on modern CPUs. Additionally, <code class="command">ffmpeg</code> will usually need to copy the decoded frames from the GPU memory into the system memory, resulting in further performance loss. This option is thus mainly useful for testing. </dd> <dt><samp class="option">-hwaccel_device[:<var class="var">stream_specifier</var>] <var class="var">hwaccel_device</var> (input,per-stream)</samp></dt> <dd> Select a device to use for hardware acceleration. This option only makes sense when the <samp class="option">-hwaccel</samp> option is also specified. It can either refer to an existing device created with <samp class="option">-init_hw_device</samp> by name, or it can create a new device as if ‘<samp class="samp">-init_hw_device</samp>’ <var class="var">type</var>:<var class="var">hwaccel_device</var> were called immediately before. </dd> <dt><samp class="option">-

hwaccels</samp></dt> <dd> List all hardware acceleration components enabled in this build of ffmpeg. Actual runtime availability depends on the hardware and its suitable driver being installed. </dd> <dt><samp class="option">-fix_sub_duration_heartbeat[:<var class="var">stream_specifier</var>]</samp></dt> <dd> Set a specific output video stream as the heartbeat stream according to which to split and push through currently in-progress subtitle upon receipt of a random access packet. This lowers the latency of subtitles for which the end packet or the following subtitle has not yet been received. As a drawback, this will most likely lead to duplication of subtitle events in order to cover the full duration, so when dealing with use cases where latency of when the subtitle event is passed on to output is not relevant this option should not be utilized. Requires <samp class="option">-fix_sub_duration</samp> to be set for the relevant input subtitle stream for this to have any effect, as well as for the input subtitle stream having to be directly mapped to the same output in which the heartbeat stream resides. </dd> </dl>

 <h3 class="section">5.7 Audio Options# TOC</h3>

<dl class="table"> <dt><samp class="option">-aframes <var class="var">number</var> (output)</samp></dt> <dd> Set the number of audio frames to output. This is an obsolete alias for <code class="code">-frames:a</code>, which you should use instead. </dd> <dt><samp class="option">-ar[:<var class="var">stream_specifier</var>] <var class="var">freq</var> (input/output,per-stream)</samp></dt> <dd> Set the audio sampling frequency. For output streams it is set by default to the frequency of the corresponding input stream. For input streams this option only makes sense for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer options. </dd> <dt><samp class="option">-aq <var class="var">q</var> (output)</samp></dt> <dd> Set the audio quality (codec-specific, VBR). This is an alias for -q:a. </dd> <dt><samp class="option">-ac[:<var class="var">stream_specifier</var>] <var class="var">channels</var> (input/output,per-stream)</samp></dt> <dd> Set the number of audio channels. For output streams it is set by default to the number of input audio channels. For input streams this option only makes sense for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer options. </dd> <dt><samp class="option">-an (input/output)</samp></dt> <dd> As an input option, blocks all audio streams of a file from being filtered or being automatically selected or mapped for any output. See <code class="code">-discard</code> option to disable streams individually. As an output option, disables audio recording i.e. auto-

matic selection or mapping of any audio stream. For full manual control see the <code class="code">-map</code> option. </dd> <dt><samp class="option">-acodec <var class="var">codec</var> (input/output)</samp></dt> <dd> Set the audio codec. This is an alias for <code class="code">-codec:a</code>. </dd> <dt><samp class="option">-sample_fmt[:<var class="var">stream_specifier</var>] <var class="var">sample_fmt</var> (output,per-stream)</samp></dt> <dd> Set the audio sample format. Use <code class="code">-sample_fmts</code> to get a list of supported sample formats. </dd> <dt><samp class="option">-af <var class="var">filtergraph</var> (output)</samp></dt> <dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to filter the stream. This is an alias for <code class="code">-filter:a</code>, see the -filter option. </dd> </dl>

 <h3 class="section">5.8 Advanced Audio options# TOC</h3>

<dl class="table"> <dt><samp class="option">-atag <var class="var">fourcc/tag</var> (output)</samp></dt> <dd> Force audio tag/fourcc. This is an alias for <code class="code">-tag:a</code>. </dd> <dt><samp class="option">-ch_layout[:<var class="var">stream_specifier</var>] <var class="var">layout</var> (input/output,per-stream)</samp></dt> <dd> Alias for <code class="code">-channel_layout</code>. </dd> <dt><samp class="option">-channel_layout[:<var class="var">stream_specifier</var>] <var class="var">layout</var> (input/output,per-stream)</samp></dt> <dd> Set the audio channel layout. For output streams it is set by default to the input channel layout. For input streams it overrides the channel layout of the input. Not all decoders respect the overridden channel layout. This option also sets the channel layout for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer option. </dd> <dt><samp class="option">-guess_layout_max <var class="var">channels</var> (input,per-stream)</samp></dt> <dd> If some input channel layout is not known, try to guess only if it corresponds to at most the specified number of channels. For example, 2 tells to <code class="command">ffmpeg</code> to recognize 1 channel as mono and 2 channels as stereo but not 6 channels as 5.1. The default is to always try to guess. Use 0 to disable all guessing. Using the <code class="code">-channel_layout</code> option to explicitly specify an input layout also disables guessing. </dd> </dl>

 <h3 class="section">5.9 Subtitle op-

tions# TOC</h3>

<dl class="table"> <dt><samp class="option">-scodec <var class="var">codec</var> (input/output)</samp></dt> <dd> Set the subtitle codec. This is an alias for <code class="code">-codec:s</code>. </dd> <dt><samp class="option">-sn (input/output)</samp></dt> <dd> As an input option, blocks all subtitle streams of a file from being filtered or being automatically selected or mapped for any output. See <code class="code">-discard</code> option to disable streams individually. As an output option, disables subtitle recording i.e. automatic selection or mapping of any subtitle stream. For full manual control see the <code class="code">-map</code> option. </dd> </dl>

 <h3 class="section">5.10 Advanced Subtitle options# TOC</h3>

<dl class="table"> <dt><samp class="option">-fix_sub_duration</samp></dt> <dd> Fix subtitles durations. For each subtitle, wait for the next packet in the same stream and adjust the duration of the first to avoid overlap. This is necessary with some subtitles codecs, especially DVB subtitles, because the duration in the original packet is only a rough estimate and the end is actually marked by an empty subtitle frame. Failing to use this option when necessary can result in exaggerated durations or muxing failures due to non-monotonic timestamps. Note that this option will delay the output of all data until the next subtitle packet is decoded: it may increase memory consumption and latency a lot. </dd> <dt><samp class="option">-canvas_size <var class="var">size</var></samp></dt> <dd> Set the size of the canvas used to render subtitles. </dd> </dl>

 <h3 class="section">5.11 Advanced options# TOC</h3>

<dl class="table"> <dt><samp class="option">-map [-]<var class="var">input_file_id</var>[:<var class="var">stream_specifier</var>][:<var class="var">view_specifier</var>][:?] | <var class="var">[linklabel]</var> (output)</samp></dt> <dd> Create one or more streams in the output file. This option has two forms for specifying the data source(s): the first selects one or more streams from some input file (specified with <code class="code">-i</code>), the second takes an output from some complex filtergraph (specified with <code class="code">-filter_complex</code>). In the first form, an output stream is created for every stream from

the input file with the index <var class="var">input_file_id</var>. If <var class="var">stream_specifier</var> is given, only those streams that match the specifier are used (see the Stream specifiers section for the <var class="var">stream_specifier</var> syntax). A <code class="code">-</code> character before the stream identifier creates a "negative" mapping. It disables matching streams from already created mappings. An optional <var class="var">view_specifier</var> may be given after the stream specifier, which for multiview video specifies the view to be used. The view specifier may have one of the following formats: <dl class="table"> <dt><samp class="option">view:<var class="var">view_id</var></samp></dt> <dd> select a view by its ID; <var class="var">view_id</var> may be set to ’all’ to use all the views interleaved into one stream; </dd> <dt><samp class="option">vidx:<var class="var">view_idx</var></samp></dt> <dd> select a view by its index; i.e. 0 is the base view, 1 is the first non-base view, etc. </dd> <dt><samp class="option">vpos:<var class="var">position</var></samp></dt> <dd> select a view by its display position; <var class="var">position</var> may be <code class="code">left</code> or <code class="code">right</code> </dd> </dl> The default for transcoding is to only use the base view, i.e. the equivalent of <code class="code">vidx:0</code>. For streamcopy, view specifiers are not supported and all views are always copied. A trailing <code class="code">?</code> after the stream index will allow the map to be optional: if the map matches no streams the map will be ignored instead of failing. Note the map will still fail if an invalid input file index is used; such as if the map refers to a non-existent input. An alternative <var class="var">[linklabel]</var> form will map outputs from complex filter graphs (see the <samp class="option">-filter_complex</samp> option) to the output file. <var class="var">linklabel</var> must correspond to a defined output link label in the graph. This option may be specified multiple times, each adding more streams to the output file. Any given input stream may also be mapped any number of times as a source for different output streams, e.g. in order to use different encoding options and/or filters. The streams are created in the output in the same order in which the <code class="code">-map</code> options are given on the commandline. Using this option disables the default mappings for this output file. Examples: <dl class="table"> <dt>map everything</dt> <dd> To map ALL streams from the first input file to output <pre class="example-preformatted">ffmpeg -i INPUT -map 0 output </pre>

</dd> <dt>select specific stream</dt> <dd> If you have two audio streams in the first input file, these streams are identified by <var class="var">0:0</var> and <var class="var">0:1</var>. You can use <code class="code">-map</code> to select which streams

to place in an output file. For example: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:1 out.wav </pre> will map the second input stream in <samp class="file">INPUT</samp> to the (single) output stream in <samp class="file">out.wav</samp>. </dd> <dt>create multiple streams</dt> <dd> To select the stream with index 2 from input file <samp class="file">a.mov</samp> (specified by the identifier <var class="var">0:2</var>), and stream with index 6 from input <samp class="file">b.mov</samp> (specified by the identifier <var class="var">1:6</var>), and copy them to the output file <samp class="file">out.mov</samp>: <pre class="example-preformatted">ffmpeg -i a.mov -i b.mov -c copy -map 0:2 -map 1:6 out.mov </pre>

</dd> <dt>create multiple streams 2</dt> <dd> To select all video and the third audio stream from an input file: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:v -map 0:a:2 OUTPUT </pre>

</dd> <dt>negative map</dt> <dd> To map all the streams except the second audio, use negative mappings <pre class="example-preformatted">ffmpeg -i INPUT -map 0 -map -0:a:1 OUTPUT </pre>

</dd> <dt>optional map</dt> <dd> To map the video and audio streams from the first input, and using the trailing <code class="code">?</code>, ignore the audio mapping if no audio streams exist in the first input: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:v -map 0:a? OUTPUT </pre>

</dd> <dt>map by language</dt> <dd> To pick the English audio stream: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:m:language:eng OUTPUT </pre>

</dd> </dl>

</dd> <dt><samp class="option">-ignore_unknown</samp></dt> <dd> Ignore input streams with unknown type instead of failing if copying such streams is attempted. </dd> <dt><samp class="option">-copy_unknown</samp></dt> <dd> Allow input streams with unknown type to be copied instead of failing if copying such streams is attempted. </dd> <dt><samp class="option">-map_metadata[:<var class="var">metadata_spec_out</var>] <var class="var">infile</var>[:<var class="var">metadata_spec_in</var>] (output,per-metadata)</samp></dt> <dd> Set metadata information of the next output file from <var class="var">infile</var>. Note that those are file indices (zero-based), not filenames. Optional <var class="var">metadata_spec_in/out</var> parameters specify,

which metadata to copy. A metadata specifier can have the following forms: <dl class="table"> <dt><samp class="option"><var class="var">g</var></samp></dt> <dd> global metadata, i.e. metadata that applies to the whole file </dd> <dt><samp class="option"><var class="var">s</var>[:<var class="var">stream_spec</var>]</samp></dt> <dd> per-stream metadata. <var class="var">stream_spec</var> is a stream specifier as described in the Stream specifiers chapter. In an input metadata specifier, the first matching stream is copied from. In an output metadata specifier, all matching streams are copied to. </dd> <dt><samp class="option"><var class="var">c</var>:<var class="var">chapter_index</var></samp></dt> <dd> per-chapter metadata. <var class="var">chapter_index</var> is the zero-based chapter index. </dd> <dt><samp class="option"><var class="var">p</var>:<var class="var">program_index</var></samp></dt> <dd> per-program metadata. <var class="var">program_index</var> is the zero-based program index. </dd> </dl> If metadata specifier is omitted, it defaults to global. By default, global metadata is copied from the first input file, per-stream and per-chapter metadata is copied along with streams/chapters. These default mappings are disabled by creating any mapping of the relevant type. A negative file index can be used to create a dummy mapping that just disables automatic copying. For example to copy metadata from the first stream of the input file to global metadata of the output file: <pre class="example-preformatted">ffmpeg -i in.ogg -map_metadata 0:s:0 out.mp3 </pre>

To do the reverse, i.e. copy global metadata to all audio streams: <pre class="example-preformatted">ffmpeg -i in.mkv -map_metadata:s:a 0:g out.mkv </pre> Note that simple <code class="code">0</code> would work as well in this example, since global metadata is assumed by default. </dd> <dt><samp class="option">-map_chapters <var class="var">input_file_index</var> (output)</samp></dt> <dd> Copy chapters from input file with index <var class="var">input_file_index</var> to the next output file. If no chapter mapping is specified, then chapters are copied from the first input file with at least one chapter. Use a negative file index to disable any chapter copying. </dd> <dt><samp class="option">-benchmark (global)</samp></dt> <dd> Show benchmarking information at the end of an encode. Shows real, system and user time used and maximum memory consumption. Maximum memory consumption is not supported on all systems, it will usually display as 0 if not supported. </dd> <dt><samp class="option">-benchmark_all (global)</samp></dt> <dd> Show benchmarking information during the encode. Shows real, system and user time used in various steps (audio/video encode/decode). </dd> <dt><samp class="option">-timelimit <var class="var">duration</var>

(global)</samp></dt> <dd> Exit after ffmpeg has been running for <var class="var">duration</var> seconds in CPU user time. </dd> <dt><samp class="option">-dump (global)</samp></dt> <dd> Dump each input packet to stderr. </dd> <dt><samp class="option">-hex (global)</samp></dt> <dd> When dumping packets, also dump the payload. </dd> <dt><samp class="option">-readrate <var class="var">speed</var> (input)</samp></dt> <dd> Limit input read speed. Its value is a floating-point positive number which represents the maximum duration of media, in seconds, that should be ingested in one second of wallclock time. Default value is zero and represents no imposed limitation on speed of ingestion. Value <code class="code">1</code> represents real-time speed and is equivalent to <code class="code">-re</code>. Mainly used to simulate a capture device or live input stream (e.g. when reading from a file). Should not be used with a low value when input is an actual capture device or live stream as it may cause packet loss. It is useful for when flow speed of output packets is important, such as live streaming. </dd> <dt><samp class="option">-re (input)</samp></dt> <dd> Read input at native frame rate. This is equivalent to setting <code class="code">-readrate 1</code>. </dd> <dt><samp class="option">-readrate_initial_burst <var class="var">seconds</var></samp></dt> <dd> Set an initial read burst time, in seconds, after which <samp class="option">-re/-readrate</samp> will be enforced. </dd> <dt><samp class="option">-readrate_catchup <var class="var">speed</var> (input)</samp></dt> <dd> If either the input or output is blocked leading to actual read speed falling behind the specified readrate, then this rate takes effect till the input catches up with the specified readrate. Must not be lower than the primary readrate. </dd> <dt><samp class="option">-fps_mode[:<var class="var">stream_specifier</var>] <var class="var">parameter</var> (output,per-stream)</samp></dt> <dd> Set video sync method / framerate mode. <dl class="table"> <dt><samp class="option">passthrough</samp></dt> <dd> Each frame is passed with its timestamp from the demuxer to the muxer. </dd> <dt><samp class="option">cfr</samp></dt> <dd> Frames will be duplicated and dropped to achieve exactly the requested constant frame rate. </dd> <dt><samp class="option">vfr</samp></dt> <dd> Frames are passed through with their timestamp or dropped so as to prevent 2 frames from having the same timestamp. </dd> <dt><samp class="option">auto</samp></dt> <dd> Chooses between cfr and vfr depending on muxer capabilities. This is the default method. </dd> </dl>

Note that the timestamps may be further modified by the muxer, after this. For example, in the case that the format option <samp

class=”option”>avoid_negative_ts</samp> is enabled. With -map you can select from which stream the timestamps should be taken. You can leave either video or audio unchanged and sync the remaining stream(s) to the unchanged one. </dd> <dt><samp class=”option”>-frame_drop_threshold <var class=”var”>parameter</var></samp></dt> <dd> Frame drop threshold, which specifies how much behind video frames can be before they are dropped. In frame rate units, so 1.0 is one frame. The default is -1.1. One possible usecase is to avoid framedrops in case of noisy timestamps or to increase frame drop precision in case of exact timestamps. </dd> <dt><samp class=”option”>-apad <var class=”var”>parameters</var> (output,per-stream)</samp></dt> <dd> Pad the output audio stream(s). This is the same as applying <code class=”code”>-af apad</code>. Argument is a string of filter parameters composed the same as with the <code class=”code”>apad</code> filter. <code class=”code”>-shortest</code> must be set for this output for the option to take effect. </dd> <dt><samp class=”option”>-copyts</samp></dt> <dd> Do not process input timestamps, but keep their values without trying to sanitize them. In particular, do not remove the initial start time offset value. Note that, depending on the <samp class=”option”>fps_mode</samp> option or on specific muxer processing (e.g. in case the format option <samp class=”option”>avoid_negative_ts</samp> is enabled) the output timestamps may mismatch with the input timestamps even when this option is selected. </dd> <dt><samp class=”option”>-start_at_zero</samp></dt> <dd> When used with <samp class=”option”>copyts</samp>, shift input timestamps so they start at zero. This means that using e.g. <code class=”code”>-ss 50</code> will make output timestamps start at 50 seconds, regardless of what timestamp the input file started at. </dd> <dt><samp class=”option”>-copytb <var class=”var”>mode</var></samp></dt> <dd> Specify how to set the encoder timebase when stream copying. <var class=”var”>mode</var> is an integer numeric value, and can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>1</samp></dt> <dd> Use the demuxer timebase. The time base is copied to the output encoder from the corresponding input demuxer. This is sometimes required to avoid non monotonically increasing timestamps when copying video streams with variable frame rate. </dd> <dt><samp class=”option”>0</samp></dt> <dd> Use the decoder timebase. The time base is copied to the output encoder from the corresponding input decoder. </dd> <dt><samp class=”option”>-1</samp></dt> <dd> Try to make the choice automatically, in order to generate a sane output. </dd> </dl>

Default value is -1. </dd> <dt><samp class=”option”>-enc_time_base[:<var class=”var”>stream_specifier</var>] <var class=”var”>timebase</var> (output,per-stream)</samp></dt> <dd> Set

the encoder timebase. <var class=”var”>timebase</var> can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dd> Assign a default value according to the media type. For video - use 1/framerate, for audio - use 1/samplerate. </dd> <dt><samp class=”option”>demux</samp></dt> <dd> Use the timebase from the demuxer. </dd> <dt><samp class=”option”>filter</samp></dt> <dd> Use the timebase from the filtergraph. </dd> <dt><samp class=”option”>a positive number</samp></dt> <dd> Use the provided number as the timebase. This field can be provided as a ratio of two integers (e.g. 1:24, 1:48000) or as a decimal number (e.g. 0.04166, 2.0833e-5) </dd> </dl>

Default value is 0. </dd> <dt><samp class=”option”>-bitexact (input/output)</samp></dt> <dd> Enable bitexact mode for (de)muxer and (de/en)coder </dd> <dt><samp class=”option”>-shortest (output)</samp></dt> <dd> Finish encoding when the shortest output stream ends. Note that this option may require buffering frames, which introduces extra latency. The maximum amount of this latency may be controlled with the <code class=”code”>-shortest_buf_duration</code> option. </dd> <dt><samp class=”option”>-shortest_buf_duration <var class=”var”>duration</var> (output)</samp></dt> <dd> The <code class=”code”>-shortest</code> option may require buffering potentially large amounts of data when at least one of the streams is "sparse" (i.e. has large gaps between frames – this is typically the case for subtitles). This option controls the maximum duration of buffered frames in seconds. Larger values may allow the <code class=”code”>-shortest</code> option to produce more accurate results, but increase memory use and latency. The default value is 10 seconds. </dd> <dt><samp class=”option”>-dts_delta_threshold <var class=”var”>threshold</var></samp></dt> <dd> Timestamp discontinuity delta threshold, expressed as a decimal number of seconds. The timestamp discontinuity correction enabled by this option is only applied to input formats accepting timestamp discontinuity (for which the <code class=”code”>AVFMT_TS_DISCONT</code> flag is enabled), e.g. MPEG-TS and HLS, and is automatically disabled when employing the <code class=”code”>-copyts</code> option (unless wrapping is detected). If a timestamp discontinuity is detected whose absolute value is greater than <var class=”var”>threshold</var>, ffmpeg will remove the discontinuity by decreasing/increasing the current DTS and PTS by the corresponding delta value. The default value is 10. </dd> <dt><samp class=”option”>-dts_error_threshold <var class=”var”>threshold</var></samp></dt> <dd> Timestamp error delta threshold, expressed as a decimal number of seconds. The timestamp correction enabled by this option is only applied to input formats not accepting timestamp discontinuity (for which the <code

class=”code”>AVFMT_TS_DISCONT</code> flag is not enabled). If a timestamp discontinuity is detected whose absolute value is greater than <var class=”var”>threshold</var>, ffmpeg will drop the PTS/DTS timestamp value. The default value is <code class=”code”>3600*30</code> (30 hours), which is arbitrarily picked and quite conservative. </dd> <dt><samp class=”option”>-muxdelay <var class=”var”>seconds</var> (output)</samp></dt> <dd> Set the maximum demux-decode delay. </dd> <dt><samp class=”option”>-muxpreload <var class=”var”>seconds</var> (output)</samp></dt> <dd> Set the initial demux-decode delay. </dd> <dt><samp class=”option”>-streamid <var class=”var”>output-stream-index</var>:<var class=”var”>new-value</var> (output)</samp></dt> <dd> Assign a new stream-id value to an output stream. This option should be specified prior to the output filename to which it applies. For the situation where multiple output files exist, a streamid may be reassigned to a different value. For example, to set the stream 0 PID to 33 and the stream 1 PID to 36 for an output mpegts file: <pre class=”example-preformatted”>ffmpeg -i inurl -streamid 0:33 -streamid 1:36 out.ts </pre>

</dd> <dt><samp class=”option”>-bsf[:<var class=”var”>stream_specifier</var>] <var class=”var”>bitstream_filters</var> (input/output,per-stream)</samp></dt> <dd> Apply bitstream filters to matching streams. The filters are applied to each packet as it is received from the demuxer (when used as an input option) or before it is sent to the muxer (when used as an output option). <var class=”var”>bitstream_filters</var> is a comma-separated list of bitstream filter specifications, each of the form <pre class=”example-preformatted”><var class=”var”>filter</var>[=<var class=”var”>optname0</var>=<var class=”var”>optval0</var>:<var class=”var”>optname1</var>=<var class=”var”>optval1</var>:...] </pre> Any of the ’,=:’ characters that are to be a part of an option value need to be escaped with a backslash. Use the <code class=”code”>-bsfs</code> option to get the list of bitstream filters. E.g. <pre class=”example-preformatted”>ffmpeg -bsf:v h264_mp4toannexb -i h264.mp4 -c:v copy -an out.h264 </pre> applies the <code class=”code”>h264_mp4toannexb</code> bitstream filter (which converts MP4-encapsulated H.264 stream to Annex B) to the input video stream. On the other hand, <pre class=”example-preformatted”>ffmpeg -i file.mov -an -vn -bsf:s mov2textsub -c:s copy -f rawvideo sub.txt </pre> applies the <code class=”code”>mov2textsub</code> bitstream filter (which extracts text from MOV subtitles) to the output subtitle stream. Note, however, that since both examples use <code class=”code”>-c copy</code>, it matters little whether

the filters are applied on input or output - that would change if transcoding was happening. </dd> <dt><samp class="option">-tag[:<var class="var">stream_specifier</var>] <var class="var">codec_tag</var> (input/output,per-stream)</samp></dt> <dd> Force a tag/fourcc for matching streams. </dd> <dt><samp class="option">-timecode <var class="var">hh</var>:<var class="var">mm</var>:<var class="var">ss</var>SEP<var class="var">ff</var></samp></dt> <dd> Specify Timecode for writing. <var class="var">SEP</var> is ’:’ for non drop timecode and ’;’ (or ’.’) for drop. <pre class="example-preformatted">ffmpeg -i input.mpg -timecode 01:02:03.04 -r 30000/1001 -s ntsc output.mpg </pre>

</dd> <dt><samp class="option">-filter_complex <var class="var">filtergraph</var> (global)</samp></dt> <dd> Define a complex filtergraph, i.e. one with arbitrary number of inputs and/or outputs. For simple graphs – those with one input and one output of the same type – see the <samp class="option">-filter</samp> options. <var class="var">filtergraph</var> is a description of the filtergraph, as described in the “Filtergraph syntax” section of the ffmpeg-filters manual. This option may be specified multiple times - each use creates a new complex filtergraph. Inputs to a complex filtergraph may come from different source types, distinguished by the format of the corresponding link label: <ul class="itemize mark-bullet"> To connect an input stream, use <code class="code">[file_index:stream_specifier]</code> (i.e. the same syntax as <samp class="option">-map</samp>). If <var class="var">stream_specifier</var> matches multiple streams, the first one will be used. For multiview video, the stream specifier may be followed by the view specifier, see documentation for the <samp class="option">-map</samp> option for its syntax.

To connect a loopback decoder use [dec:<var class="var">dec_idx</var>], where <var class="var">dec_idx</var> is the index of the loopback decoder to be connected to given input. For multiview video, the decoder index may be followed by the view specifier, see documentation for the <samp class="option">-map</samp> option for its syntax.

To connect an output from another complex filtergraph, use its link label. E.g the following example:

 <pre class="example-preformatted">ffmpeg -i input.mkv \ -filter_complex '[0:v]scale=size=hd1080,split=outputs=2[for_enc][orig_scaled]' \ -c:v libx264 -map '[for_enc]' output.mkv \ -dec 0:0 \ -filter_complex '[dec:0][orig_scaled]hstack[stacked]' \ -map '[stacked]' -c:v ffv1 comparison.mkv </pre>

reads an input video and <ul class="itemize mark-bullet"> (line

2) uses a complex filtergraph with one input and two outputs to scale the video to 1920x1080 and duplicate the result to both outputs;

(line 3) encodes one scaled output with <code class="code">libx264</code> and writes the result to <samp class="file">output.mkv</samp>;

(line 4) decodes this encoded stream with a loopback decoder;

(line 5) places the output of the loopback decoder (i.e. the <code class="code">libx264</code>-encoded video) side by side with the scaled original input;

(line 6) combined video is then losslessly encoded and written into <samp class="file">comparison.mkv</samp>.

Note that the two filtergraphs cannot be combined into one, because then there would be a cycle in the transcoding pipeline (filtergraph output goes to encoding, from there to decoding, then back to the same graph), and such cycles are not allowed.

An unlabeled input will be connected to the first unused input stream of the matching type. Output link labels are referred to with <samp class="option">-map</samp>. Unlabeled outputs are added to the first output file. Note that with this option it is possible to use only lavfi sources without normal input files. For example, to overlay an image over video <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex '[0:v][1:v]overlay[out]' -map '[out]' out.mkv </pre> Here <code class="code">[0:v]</code> refers to the first video stream in the first input file, which is linked to the first (main) input of the overlay filter. Similarly the first video stream in the second input is linked to the second (overlay) input of overlay. Assuming there is only one video stream in each input file, we can omit input labels, so the above is equivalent to <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex 'overlay[out]' -map '[out]' out.mkv </pre>

Furthermore we can omit the output label and the single output from the filter graph will be added to the output file automatically, so we can simply write <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex 'overlay' out.mkv </pre>

As a special exception, you can use a bitmap subtitle stream as input: it will be converted into a video with the same size as the largest video in the file, or 720x576 if no video is present. Note that this is an experimental and temporary solution. It will be removed once libavfilter has proper support for subtitles. For example, to hardcode subtitles on top of a DVB-T recording stored in MPEG-TS format, delaying the subtitles by 1 second:

 <pre class="example-preformatted">ffmpeg -i input.ts -filter_complex \ '[#0x2ef] setpts=PTS+1/TB [sub] ; [#0x2d0] [sub] overlay' \ -sn -map '#0x2dc' output.mkv </pre> (0x2d0, 0x2dc and 0x2ef are the MPEG-TS PIDs of respectively the video, audio and subtitles streams; 0:0, 0:3 and 0:7 would have worked too) To generate 5 seconds of pure red video using lavfi <code class="code">color</code> source: <pre class="example-preformatted">ffmpeg -filter_complex 'color=c=red' -t 5 out.mkv </pre>

</dd> <dt><samp class="option">-filter_complex_threads <var class="var">nb_threads</var> (global)</samp></dt> <dd> Defines how many threads are used to process a filter_complex graph. Similar to filter_threads but used for <code class="code">-filter_complex</code> graphs only. The default is the number of available CPUs. </dd> <dt><samp class="option">-lavfi <var class="var">filtergraph</var> (global)</samp></dt> <dd> Define a complex filtergraph, i.e. one with arbitrary number of inputs and/or outputs. Equivalent to <samp class="option">-filter_complex</samp>. </dd> <dt><samp class="option">-accurate_seek (input)</samp></dt> <dd> This option enables or disables accurate seeking in input files with the <samp class="option">-ss</samp> option. It is enabled by default, so seeking is accurate when transcoding. Use <samp class="option">-noaccurate_seek</samp> to disable it, which may be useful e.g. when copying some streams and transcoding the others. </dd> <dt><samp class="option">-seek_timestamp (input)</samp></dt> <dd> This option enables or disables seeking by timestamp in input files with the <samp class="option">-ss</samp> option. It is disabled by default. If enabled, the argument to the <samp class="option">-ss</samp> option is considered an actual timestamp, and is not offset by the start time of the file. This matters only for files which do not start from timestamp 0, such as transport streams. </dd> <dt><samp class="option">-thread_queue_size <var class="var">size</var> (output)</samp></dt> <dd> This option specifies the maximum number of packets per stream that may be queued to each muxing thread. </dd> <dt><samp class="option">-sdp_file <var class="var">file</var> (global)</samp></dt> <dd> Print sdp information for an output stream to <var class="var">file</var>. This allows dumping sdp information when at least one output isn’t an rtp stream. (Requires at least one of the output formats to be rtp). </dd> <dt><samp class="option">-discard (input)</samp></dt> <dd> Allows discarding specific streams or frames from streams. Any input stream can be fully discarded, using value <code class="code">all</code> whereas selective discarding of frames from a stream occurs at the demuxer and is not supported by all demuxers. <dl class="table"> <dt><samp class="option">none</samp></dt> <dd> Discard no

frame. </dd> <dt><samp class="option">default</samp></dt> <dd> Default, which discards no frames. </dd> <dt><samp class="option">noref</samp></dt> <dd> Discard all non-reference frames. </dd> <dt><samp class="option">bidir</samp></dt> <dd> Discard all bidirectional frames. </dd> <dt><samp class="option">nokey</samp></dt> <dd> Discard all frames excepts keyframes. </dd> <dt><samp class="option">all</samp></dt> <dd> Discard all frames. </dd> </dl>

</dd> <dt><samp class="option">-abort_on <var class="var">flags</var> (global)</samp></dt> <dd> Stop and abort on various conditions. The following flags are available: <dl class="table"> <dt><samp class="option">empty_output</samp></dt> <dd> No packets were passed to the muxer, the output is empty. </dd> <dt><samp class="option">empty_output_stream</samp></dt> <dd> No packets were passed to the muxer in some of the output streams. </dd> </dl>

</dd>

<dt><samp class="option">-max_error_rate (global)</samp></dt>

<dd> Set fraction of decoding frame failures across all inputs which when crossed

ffmpeg will return exit code 69. Crossing this threshold does not terminate

processing. Range is a floating-point number between 0 to 1. Default is 2/3.

</dd>

<dt><samp class="option">-xerror (global)</samp></dt>

<dd> Stop and exit on error

</dd>

<dt><samp class="option">-max_muxing_queue_size <var class="var">packets</var> (output,per-stream)</samp></dt>

<dd> When transcoding audio and/or video streams, ffmpeg will not begin writing into

the output until it has one packet for each such stream. While waiting for that

to happen, packets for other streams are buffered. This option sets the size of

this buffer, in packets, for the matching output stream.

The default value of this option should be high enough for most uses, so only

touch this option if you are sure that you need it.

</dd>

<dt><samp class="option">-muxing_queue_data_threshold <var class="var">bytes</var> (output,per-stream)</samp></dt>

<dd> This is a minimum threshold until which the muxing queue size is not taken into

account. Defaults to 50 megabytes per stream, and is based on the overall size

of packets passed to the muxer.

</dd>

<dt><samp class="option">-auto_conversion_filters (global)</samp></dt>

<dd> Enable automatically inserting format conversion filters in all filter

graphs, including those defined by <samp class="option">-vf</samp>, <samp class="option">-af</samp>,

<samp class="option">-filter_complex</samp> and <samp class="option">-lavfi</samp>. If filter format negotiation

requires a conversion, the initialization of the filters will fail.

Conversions can still be performed by inserting the relevant conversion

filter (scale, aresample) in the graph.

On by default, to explicitly disable it you need to specify

<code class="code">-noauto_conversion_filters</code>.

</dd>

<dt><samp class="option">-bits_per_raw_sample[:<var class="var">stream_specifier</var>] <var class="var">value</var> (output,per-stream)</samp></dt>

<dd> Declare the number of bits per raw sample in the given output stream to be

<var class="var">value</var>. Note that this option sets the information provided to the

encoder/muxer, it does not change the stream to conform to this value. Setting

values that do not match the stream properties may result in encoding failures

or invalid output files.

</dd>

<dt><samp class=”option”>-stats_enc_pre[:<var class=”var”>stream_specifier</var>]
<var class=”var”>path</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_enc_post[:<var class=”var”>stream_specifier</var>]
<var class=”var”>path</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_mux_pre[:<var class=”var”>stream_specifier</var>]
<var class=”var”>path</var> (output,per-stream)</samp></dt>

<dd> Write per-frame encoding information about the matching streams
into the file

given by <var class=”var”>path</var>.

<samp class=”option”>-stats_enc_pre</samp> writes information about
raw video or audio frames right

before they are sent for encoding, while <samp class=”option”>-stats_enc_post</samp>
writes

information about encoded packets as they are received from the encoder.

<samp class=”option”>-stats_mux_pre</samp> writes information about
packets just as they are about to

be sent to the muxer. Every frame or packet produces one line in the specified

file. The format of this line is controlled by <samp class=”option”>-
stats_enc_pre_fmt</samp> /

<samp class=”option”>-stats_enc_post_fmt</samp> / <samp class=”option”>-
stats_mux_pre_fmt</samp>.

When stats for multiple streams are written into a single file, the lines

corresponding to different streams will be interleaved. The precise order of

this interleaving is not specified and not guaranteed to remain stable between

different invocations of the program, even with the same options.

</dd>

<dt><samp class=”option”>-stats_enc_pre_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_enc_post_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_mux_pre_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dd> Specify the format for the lines written with <samp class=”option”>-stats_enc_pre</samp> /

<samp class=”option”>-stats_enc_post</samp> / <samp class=”option”>-stats_mux_pre</samp>.

<var class=”var”>format_spec</var> is a string that may contain directives of the form

<var class=”var”>{fmt}</var>. <var class=”var”>format_spec</var> is backslash-escaped — use \{, \}, and \\

to write a literal {, }, or \, respectively, into the output.

The directives given with <var class=”var”>fmt</var> may be one of the following:

<dl class=”table”>

<dt><samp class=”option”>fidx</samp></dt>

<dd> Index of the output file.

</dd>

<dt><samp class=”option”>sidx</samp></dt>

<dd> Index of the output stream in the file.

</dd>

<dt><samp class=”option”>n</samp></dt>

<dd> Frame number. Pre-encoding: number of frames sent to the encoder so far.

Post-encoding: number of packets received from the encoder so far.

Muxing: number of packets submitted to the muxer for this stream so far.

</dd>

<dt><samp class="option">ni</samp></dt>

<dd> Input frame number. Index of the input frame (i.e. output by a decoder) that

corresponds to this output frame or packet. -1 if unavailable.

</dd>

<dt><samp class="option">tb</samp></dt>

<dd> Timebase in which this frame/packet’s timestamps are expressed, as a rational

number <var class="var">num/den</var>. Note that encoder and muxer may use different timebases.

</dd>

<dt><samp class="option">tbi</samp></dt>

<dd> Timebase for <var class="var">ptsi</var>, as a rational number <var class="var">num/den</var>. Available when

<var class="var">ptsi</var> is available, <var class="var">0/1</var> otherwise.

</dd>

<dt><samp class="option">pts</samp></dt>

<dd> Presentation timestamp of the frame or packet, as an integer. Should be

multiplied by the timebase to compute presentation time.

</dd>

<dt><samp class="option">ptsi</samp></dt>

<dd> Presentation timestamp of the input frame (see <var class="var">ni</var>), as an integer. Should

be multiplied by <var class="var">tbi</var> to compute presentation time. Printed as

(2^63 - 1 = 9223372036854775807) when not available.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> Presentation time of the frame or packet, as a decimal number. Equal to

<var class="var">pts</var> multiplied by <var class="var">tb</var>.

</dd>

<dt><samp class="option">ti</samp></dt>

<dd> Presentation time of the input frame (see <var class="var">ni</var>), as a decimal number. Equal

to <var class="var">ptsi</var> multiplied by <var class="var">tbi</var>. Printed as inf when not available.

</dd>

<dt><samp class="option">dts (packet)</samp></dt>

<dd> Decoding timestamp of the packet, as an integer. Should be multiplied by the

timebase to compute presentation time.

</dd>

<dt><samp class="option">dt (packet)</samp></dt>

<dd> Decoding time of the frame or packet, as a decimal number. Equal to

<var class="var">dts</var> multiplied by <var class="var">tb</var>.

</dd>

<dt><samp class="option">sn (frame,audio)</samp></dt>

<dd> Number of audio samples sent to the encoder so far.

</dd>

<dt><samp class=”option”>samp (frame,audio)</samp></dt>

<dd> Number of audio samples in the frame.

</dd>

<dt><samp class=”option”>size (packet)</samp></dt>

<dd> Size of the encoded packet in bytes.

</dd>

<dt><samp class=”option”>br (packet)</samp></dt>

<dd> Current bitrate in bits per second.

</dd>

<dt><samp class=”option”>abr (packet)</samp></dt>

<dd> Average bitrate for the whole stream so far, in bits per second, -1 if it cannot

be determined at this point.

</dd>

<dt><samp class=”option”>key (packet)</samp></dt>

<dd> Character ’K’ if the packet contains a keyframe, character ’N’ otherwise.

</dd>

</dl>

Directives tagged with packet may only be used with <samp class=”option”>-stats_enc_post_fmt</samp> and <samp class=”option”>-stats_mux_pre_fmt</samp>. Directives tagged with frame may only be used with <samp class=”option”>-stats_enc_pre_fmt</samp>. Directives tagged with audio may only be used with audio streams. The default format strings are: <dl class=”table”> <dt><samp class=”option”>pre-encoding</samp></dt> <dd> {fidx} {sidx} {n} {t} </dd> <dt><samp class=”option”>post-encoding</samp></dt> <dd> {fidx} {sidx} {n} {t} </dd> </dl> In the future, new items may be added to the end of the default formatting strings. Users who

depend on the format staying exactly the same, should prescribe it manually. Note that stats for different streams written into the same file may have different formats. </dd> </dl>

 <h3 class="section">5.12 Preset files# TOC</h3> A preset file contains a sequence of <var class="var">option</var>=<var class="var">value</var> pairs, one for each line, specifying a sequence of options which would be awkward to specify on the command line. Lines starting with the hash (’#’) character are ignored and are used to provide comments. Check the <samp class="file">presets</samp> directory in the FFmpeg source tree for examples. There are two types of preset files: ffpreset and avpreset files. <h4 class="subsection">5.12.1 ffpreset files# TOC</h4> ffpreset files are specified with the <code class="code">vpre</code>, <code class="code">apre</code>, <code class="code">spre</code>, and <code class="code">fpre</code> options. The <code class="code">fpre</code> option takes the filename of the preset instead of a preset name as input and can be used for any kind of codec. For the <code class="code">vpre</code>, <code class="code">apre</code>, and <code class="code">spre</code> options, the options specified in a preset file are applied to the currently selected codec of the same type as the preset option. The argument passed to the <code class="code">vpre</code>, <code class="code">apre</code>, and <code class="code">spre</code> preset options identifies the preset file to use according to the following rules: First ffmpeg searches for a file named <var class="var">arg</var>.ffpreset in the directories <samp class="file">$FFMPEG_DATADIR</samp> (if set), and <samp class="file">$HOME/.ffmpeg</samp>, and in the datadir defined at configuration time (usually <samp class="file">PREFIX/share/ffmpeg</samp>) or in a <samp class="file">ffpresets</samp> folder along the executable on win32, in that order. For example, if the argument is <code class="code">libvpx-1080p</code>, it will search for the file <samp class="file">libvpx-1080p.ffpreset</samp>. If no such file is found, then ffmpeg will search for a file named <var class="var">codec_name</var>-<var class="var">arg</var>.ffpreset in the above-mentioned directories, where <var class="var">codec_name</var> is the name of the codec to which the preset file options will be applied. For example, if you select the video codec with <code class="code">-vcodec libvpx</code> and use <code class="code">-vpre 1080p</code>, then it will search for the file <samp class="file">libvpx-1080p.ffpreset</samp>. <h4 class="subsection">5.12.2 avpreset files<a class="anchor hidden-xs" href="#avpreset-

files" aria-hidden="true"># TOC</h4> avpreset files are specified with the <code class="code">pre</code> option. They work similar to ffpreset files, but they only allow encoder- specific options. Therefore, an <var class="var">option</var>=<var class="var">value</var> pair specifying an encoder cannot be used. When the <code class="code">pre</code> option is specified, ffmpeg will look for files with the suffix .avpreset in the directories <samp class="file">$AVCONV_DATADIR</samp> (if set), and <samp class="file">$HOME/.avconv</samp>, and in the datadir defined at configuration time (usually <samp class="file">PREFIX/share/ffmpeg</samp>), in that order. First ffmpeg searches for a file named <var class="var">codec_name</var>-<var class="var">arg</var>.avpreset in the above-mentioned directories, where <var class="var">codec_name</var> is the name of the codec to which the preset file options will be applied. For example, if you select the video codec with <code class="code">-vcodec libvpx</code> and use <code class="code">-pre 1080p</code>, then it will search for the file <samp class="file">libvpx-1080p.avpreset</samp>. If no such file is found, then ffmpeg will search for a file named <var class="var">arg</var>.avpreset in the same directories. <h3 class="section">5.13 vstats file format# TOC</h3> The <code class="code">-vstats</code> and <code class="code">-vstats_file</code> options enable generation of a file containing statistics about the generated video outputs. The <code class="code">-vstats_version</code> option controls the format version of the generated file. With version <code class="code">1</code> the format is: <pre class="example-preformatted">frame= <var class="var">FRAME</var> q= <var class="var">FRAME_QUALITY</var> PSNR= <var class="var">PSNR</var> f_size= <var class="var">FRAME_SIZE</var> s_size= <var class="var">STREAM_SIZE</var>kB time= <var class="var">TIMESTAMP</var> br= <var class="var">BITRATE</var>kbits/s avg_br= <var class="var">AVERAGE_BITRATE</var>kbits/s </pre>

With version <code class="code">2</code> the format is: <pre class="example-preformatted">out= <var class="var">OUT_FILE_INDEX</var> st= <var class="var">OUT_FILE_STREAM_INDEX</var> frame= <var class="var">FRAME_NUMBER</var> q= <var class="var">FRAME_QUALITY</var>f PSNR= <var class="var">PSNR</var> f_size= <var class="var">FRAME_SIZE</var> s_size= <var class="var">STREAM_SIZE</var>kB time= <var class="var">TIMESTAMP</var> br= <var class="var">BITRATE</var>kbits/s avg_br= <var class="var">AVERAGE_BITRATE</var>k </pre>

The value corresponding to each key is described below: <dl class="table"> <dt><samp class="option">avg_br</samp></dt>

<dd> average bitrate expressed in Kbits/s </dd> <dt><samp class="option">br</samp></dt> <dd> bitrate expressed in Kbits/s </dd> <dt><samp class="option">frame</samp></dt> <dd> number of encoded frame </dd> <dt><samp class="option">out</samp></dt> <dd> out file index </dd> <dt><samp class="option">PSNR</samp></dt> <dd> Peak Signal to Noise Ratio </dd> <dt><samp class="option">q</samp></dt> <dd> quality of the frame </dd> <dt><samp class="option">f_size</samp></dt> <dd> encoded packet size expressed as number of bytes </dd> <dt><samp class="option">s_size</samp></dt> <dd> stream size expressed in KiB </dd> <dt><samp class="option">st</samp></dt> <dd> out file stream index </dd> <dt><samp class="option">time</samp></dt> <dd> time of the packet </dd> <dt><samp class="option">type</samp></dt> <dd> picture type </dd> </dl>

See also the -stats_enc options for an alternative way to show encoding statistics.

 <h2 class="chapter">6 Examples# TOC</h2>

 <h3 class="section">6.1 Video and Audio grabbing# TOC</h3>

If you specify the input format and device then ffmpeg can grab video and audio directly.   <pre class="example-preformatted">ffmpeg -f oss -i /dev/dsp -f video4linux2 -i /dev/video0 /tmp/out.mpg </pre>

Or with an ALSA audio source (mono input, card id 1) instead of OSS: <pre class="example-preformatted">ffmpeg -f alsa -ac 1 -i hw:1 -f video4linux2 -i /dev/video0 /tmp/out.mpg </pre>

Note that you must activate the right video source and channel before launching ffmpeg with any TV viewer such as xawtv by Gerd Knorr. You also have to set the audio recording levels correctly with a standard mixer. <h3 class="section">6.2 X11 grabbing# TOC</h3>

Grab the X11 display with ffmpeg via   <pre class="example-preformatted">ffmpeg -f x11grab -video_size cif -framerate 25

-i :0.0 /tmp/out.mpg </pre>

0.0 is display.screen number of your X11 server, same as the DISPLAY environment variable.   <pre class="example-preformatted">ffmpeg -f x11grab -video_size cif -framerate 25 -i :0.0+10,20 /tmp/out.mpg </pre>

0.0 is display.screen number of your X11 server, same as the DISPLAY environment variable. 10 is the x-offset and 20 the y-offset for the grabbing. <h3 class="section">6.3 Video and Audio file format conversion# TOC</h3>

Any supported file format and protocol can serve as input to ffmpeg: Examples: <ul class="itemize mark-bullet"> You can use YUV files as input:

 <pre class="example-preformatted">ffmpeg -i /tmp/test%d.Y /tmp/out.mpg </pre>

It will use the files: <pre class="example-preformatted">/tmp/test0.Y, /tmp/test0.U, /tmp/test0.V, /tmp/test1.Y, /tmp/test1.U, /tmp/test1.V, etc... </pre>

The Y files use twice the resolution of the U and V files. They are raw files, without header. They can be generated by all decent video decoders. You must specify the size of the image with the <samp class="option">-s</samp> option if ffmpeg cannot guess it. You can input from a raw YUV420P file:

 <pre class="example-preformatted">ffmpeg -i /tmp/test.yuv /tmp/out.avi </pre>

test.yuv is a file containing raw YUV planar data. Each frame is composed of the Y plane followed by the U and V planes at half vertical and horizontal resolution. You can output to a raw YUV420P file:

 <pre class="example-preformatted">ffmpeg -i mydivx.avi hugefile.yuv </pre>

You can set several input files and output files:

 <pre class="example-preformatted">ffmpeg -i /tmp/a.wav -s 640x480 -i /tmp/a.yuv /tmp/a.mpg </pre>

Converts the audio file a.wav and the raw YUV video file a.yuv to MPEG file a.mpg. You can also do audio and video conversions at the same time:

 <pre class=”example-preformatted”>ffmpeg -i /tmp/a.wav -ar 22050 /tmp/a.mp2 </pre>

Converts a.wav to MPEG audio at 22050 Hz sample rate. You can encode to several formats at the same time and define a mapping from input stream to output streams:

 <pre class=”example-preformatted”>ffmpeg -i /tmp/a.wav -map 0:a -b:a 64k /tmp/a.mp2 -map 0:a -b:a 128k /tmp/b.mp2 </pre>

Converts a.wav to a.mp2 at 64 kbits and to b.mp2 at 128 kbits. ’-map file:index’ specifies which input stream is used for each output stream, in the order of the definition of output streams. You can transcode decrypted VOBs:

 <pre class=”example-preformatted”>ffmpeg -i snatch_1.vob -f avi -c:v mpeg4 -b:v 800k -g 300 -bf 2 -c:a libmp3lame -b:a 128k snatch.avi </pre>

This is a typical DVD ripping example; the input is a VOB file, the output an AVI file with MPEG-4 video and MP3 audio. Note that in this command we use B-frames so the MPEG-4 stream is DivX5 compatible, and GOP size is 300 which means one intra frame every 10 seconds for 29.97fps input video. Furthermore, the audio stream is MP3-encoded so you need to enable LAME support by passing <code class=”code”>--enable-libmp3lame</code> to configure. The mapping is particularly useful for DVD transcoding to get the desired audio language. NOTE: To see the supported input formats, use <code class=”code”>ffmpeg -demuxers</code>. You can extract images from a video, or create a video from many images:

For extracting images from a video: <pre class=”example-preformatted”>ffmpeg -i foo.avi -r 1 -s WxH -f image2 foo-%03d.jpeg </pre>

This will extract one video frame per second from the video and will output them in files named <samp class=”file”>foo-001.jpeg</samp>, <samp class=”file”>foo-002.jpeg</samp>, etc. Images will be rescaled to fit the new WxH values. If you want to extract just a limited number of frames, you can use the above command in combination with the <code class=”code”>-frames:v</code> or <code class=”code”>-t</code> option, or in combination with -ss to start extracting from a certain point in time. For creating a video from many images: <pre class=”example-preformatted”>ffmpeg -f image2 -framerate 12 -i foo-%03d.jpeg -s WxH foo.avi </pre>

The syntax <code class=”code”>foo-%03d.jpeg</code> specifies to use a decimal number composed of three digits padded with zeroes to express the sequence number. It is the same syntax supported by the C printf function, but only formats accepting a normal integer are suitable.

When importing an image sequence, -i also supports expanding shell-like wildcard patterns (globbing) internally, by selecting the image2-specific <code class=”code”>-pattern_type glob</code> option. For example, for creating a video from filenames matching the glob pattern <code class=”code”>foo-*.jpeg</code>: <pre class=”example-preformatted”>ffmpeg -f image2 -pattern_type glob -framerate 12 -i 'foo-*.jpeg' -s WxH foo.avi </pre>

You can put many streams of the same type in the output:

 <pre class=”example-preformatted”>ffmpeg -i test1.avi -i test2.avi -map 1:1 -map 1:0 -map 0:1 -map 0:0 -c copy -y test12.nut </pre>

The resulting output file <samp class=”file”>test12.nut</samp> will contain the first four streams from the input files in reverse order. To force CBR video output: <pre class=”example-preformatted”>ffmpeg -i myfile.avi -b 4000k -minrate 4000k -maxrate 4000k -bufsize 1835k out.m2v </pre>

The four options lmin, lmax, mblmin and mblmax use ’lambda’ units, but you may use the QP2LAMBDA constant to easily convert from ’q’ units: <pre class=”example-preformatted”>ffmpeg -i src.ext -lmax 21*QP2LAMBDA dst.ext </pre>

 <h2 class=”chapter”>7 Syntax# TOC</h2>

This section documents the syntax and formats employed by the FFmpeg libraries and tools. <h3 class=”section”>7.1 Quoting and escaping# TOC</h3>

FFmpeg adopts the following quoting and escaping mechanism, unless explicitly specified. The following rules are applied: <ul class=”itemize mark-bullet”> ‘<samp class=”samp”>'</samp>’ and ‘<samp class=”samp”>\</samp>’ are special characters (respectively used for quoting and escaping). In addition to them, there might be other special characters depending on the specific syntax where the escaping and quoting are employed.

A special character is escaped by prefixing it with a ‘<samp class=”samp”>\</samp>’.

All characters enclosed between ‘<samp class=”samp”>''</samp>’ are included literally in the parsed string. The quote character ‘<samp

class=”samp”>'</samp>’ itself cannot be quoted, so you may need to close the quote and escape it.

Leading and trailing whitespaces, unless escaped or quoted, are removed from the parsed string.

Note that you may need to add a second level of escaping when using the command line or a script, which depends on the syntax of the adopted shell language. The function <code class=”code”>av_get_token</code> defined in <samp class=”file”>libavutil/avstring.h</samp> can be used to parse a token quoted or escaped according to the rules defined above. The tool <samp class=”file”>tools/ffescape</samp> in the FFmpeg source tree can be used to automatically quote or escape a string in a script. <h4 class=”subsection”>7.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Escape the string <code class=”code”>Crime d'Amour</code> containing the <code class=”code”>'</code> special character: <pre class=”example-preformatted”>Crime d\'Amour </pre>

The string above contains a quote, so the <code class=”code”>'</code> needs to be escaped when quoting it: <pre class=”example-preformatted”>'Crime d'\''Amour' </pre>

Include leading or trailing whitespaces using quoting: <pre class=”example-preformatted”>' this string starts and ends with whitespaces ' </pre>

Escaping and quoting can be mixed together: <pre class=”example-preformatted”>' The string '\'string\'' is a string ' </pre>

To include a literal ‘<samp class=”samp”>\</samp>’ you can use either escaping or quoting: <pre class=”example-preformatted”>'c:\foo' can be written as c:\\foo </pre>

 <h3 class=”section”>7.2 Date# TOC</h3>

The accepted syntax is: <pre class=”example-preformatted”>[(YYYY-MM-DD|YYYYMMDD)[T|t|]]((HH:MM:SS[.m...]]])|(HHMMSS[.m...]]]))[Z] now </pre>

If the value is "now" it takes the current time. Time is local time unless Z is appended, in which case it is interpreted

as UTC. If the year-month-day part is not specified it takes the current year-month-day. <h3 class=”section”>7.3 Time duration# TOC</h3>

There are two accepted syntaxes for expressing time duration.   <pre class=”example-preformatted”>[-][<var class=”var”>HH</var>:]<var class=”var”>MM</var>:<var class=”var”>SS</var>[.<var class=”var”>m</var>...] </pre>

<var class=”var”>HH</var> expresses the number of hours, <var class=”var”>MM</var> the number of minutes for a maximum of 2 digits, and <var class=”var”>SS</var> the number of seconds for a maximum of 2 digits. The <var class=”var”>m</var> at the end expresses decimal value for <var class=”var”>SS</var>. or   <pre class=”example-preformatted”>[-]<var class=”var”>S</var>+[.<var class=”var”>m</var>...][s|ms|us] </pre>

<var class=”var”>S</var> expresses the number of seconds, with the optional decimal part <var class=”var”>m</var>. The optional literal suffixes ‘<samp class=”samp”>s</samp>’, ‘<samp class=”samp”>ms</samp>’ or ‘<samp class=”samp”>us</samp>’ indicate to interpret the value as seconds, milliseconds or microseconds, respectively. In both expressions, the optional ‘<samp class=”samp”>-</samp>’ indicates negative duration. <h4 class=”subsection”>7.3.1 Examples# TOC</h4>

The following examples are all valid time duration: <dl class=”table”> <dt>‘<samp class=”samp”>55</samp>’</dt> <dd> 55 seconds </dd> <dt>‘<samp class=”samp”>0.2</samp>’</dt> <dd> 0.2 seconds </dd> <dt>‘<samp class=”samp”>200ms</samp>’</dt> <dd> 200 milliseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>200000us</samp>’</dt> <dd> 200000 microseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>12:03:45</samp>’</dt> <dd> 12 hours, 03 minutes and 45 seconds </dd> <dt>‘<samp class=”samp”>23.189</samp>’</dt> <dd> 23.189 seconds </dd> </dl>

<h3 class=”section”>7.4 Video size# TOC</h3>

Specify the size of the sourced video, it may be a string of the form

<var class="var">width</var>x<var class="var">height</var>, or the name of a size abbreviation.

The following abbreviations are recognized:

<dl class="table">

<dt>‘<samp class="samp">ntsc</samp>’</dt>

<dd> 720x480

</dd>

<dt>‘<samp class="samp">pal</samp>’</dt>

<dd> 720x576

</dd>

<dt>‘<samp class="samp">qntsc</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class="samp">qpal</samp>’</dt>

<dd> 352x288

</dd>

<dt>‘<samp class="samp">sntsc</samp>’</dt>

<dd> 640x480

</dd>

<dt>‘<samp class="samp">spal</samp>’</dt>

<dd> 768x576

</dd>

<dt>‘<samp class="samp">film</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class="samp">ntsc-film</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class="samp">sqcif</samp>’</dt>

<dd> 128x96
</dd>
<dt>‘<samp class="samp">qcif</samp>’</dt>
<dd> 176x144
</dd>
<dt>‘<samp class="samp">cif</samp>’</dt>
<dd> 352x288
</dd>
<dt>‘<samp class="samp">4cif</samp>’</dt>
<dd> 704x576
</dd>
<dt>‘<samp class="samp">16cif</samp>’</dt>
<dd> 1408x1152
</dd>
<dt>‘<samp class="samp">qqvga</samp>’</dt>
<dd> 160x120
</dd>
<dt>‘<samp class="samp">qvga</samp>’</dt>
<dd> 320x240
</dd>
<dt>‘<samp class="samp">vga</samp>’</dt>
<dd> 640x480
</dd>
<dt>‘<samp class="samp">svga</samp>’</dt>
<dd> 800x600
</dd>
<dt>‘<samp class="samp">xga</samp>’</dt>
<dd> 1024x768
</dd>
<dt>‘<samp class="samp">uxga</samp>’</dt>
<dd> 1600x1200

</dd>
<dt>‘<samp class="samp">qxga</samp>’</dt>
<dd> 2048x1536
</dd>
<dt>‘<samp class="samp">sxga</samp>’</dt>
<dd> 1280x1024
</dd>
<dt>‘<samp class="samp">qsxga</samp>’</dt>
<dd> 2560x2048
</dd>
<dt>‘<samp class="samp">hsxga</samp>’</dt>
<dd> 5120x4096
</dd>
<dt>‘<samp class="samp">wvga</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">wxga</samp>’</dt>
<dd> 1366x768
</dd>
<dt>‘<samp class="samp">wsxga</samp>’</dt>
<dd> 1600x1024
</dd>
<dt>‘<samp class="samp">wuxga</samp>’</dt>
<dd> 1920x1200
</dd>
<dt>‘<samp class="samp">woxga</samp>’</dt>
<dd> 2560x1600
</dd>
<dt>‘<samp class="samp">wqsxga</samp>’</dt>
<dd> 3200x2048
</dd>

<dt>‘<samp class="samp">wquxga</samp>’</dt>
<dd> 3840x2400
</dd>
<dt>‘<samp class="samp">whsxga</samp>’</dt>
<dd> 6400x4096
</dd>
<dt>‘<samp class="samp">whuxga</samp>’</dt>
<dd> 7680x4800
</dd>
<dt>‘<samp class="samp">cga</samp>’</dt>
<dd> 320x200
</dd>
<dt>‘<samp class="samp">ega</samp>’</dt>
<dd> 640x350
</dd>
<dt>‘<samp class="samp">hd480</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">hd720</samp>’</dt>
<dd> 1280x720
</dd>
<dt>‘<samp class="samp">hd1080</samp>’</dt>
<dd> 1920x1080
</dd>
<dt>‘<samp class="samp">2k</samp>’</dt>
<dd> 2048x1080
</dd>
<dt>‘<samp class="samp">2kflat</samp>’</dt>
<dd> 1998x1080
</dd>
<dt>‘<samp class="samp">2kscope</samp>’</dt>

<dd> 2048x858
</dd>
<dt>‘<samp class=”samp”>4k</samp>’</dt>
<dd> 4096x2160
</dd>
<dt>‘<samp class=”samp”>4kflat</samp>’</dt>
<dd> 3996x2160
</dd>
<dt>‘<samp class=”samp”>4kscope</samp>’</dt>
<dd> 4096x1716
</dd>
<dt>‘<samp class=”samp”>nhd</samp>’</dt>
<dd> 640x360
</dd>
<dt>‘<samp class=”samp”>hqvga</samp>’</dt>
<dd> 240x160
</dd>
<dt>‘<samp class=”samp”>wqvga</samp>’</dt>
<dd> 400x240
</dd>
<dt>‘<samp class=”samp”>fwqvga</samp>’</dt>
<dd> 432x240
</dd>
<dt>‘<samp class=”samp”>hvga</samp>’</dt>
<dd> 480x320
</dd>
<dt>‘<samp class=”samp”>qhd</samp>’</dt>
<dd> 960x540
</dd>
<dt>‘<samp class=”samp”>2kdci</samp>’</dt>
<dd> 2048x1080

</dd>

<dt>‘<samp class="samp">4kdci</samp>’</dt>

<dd> 4096x2160

</dd>

<dt>‘<samp class="samp">uhd2160</samp>’</dt>

<dd> 3840x2160

</dd>

<dt>‘<samp class="samp">uhd4320</samp>’</dt>

<dd> 7680x4320

</dd>

</dl>

<h3 class="section">7.5 Video rate# TOC</h3>

Specify the frame rate of a video, expressed as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a float number or a valid video frame rate abbreviation. The following abbreviations are recognized: <dl class="table"> <dt>‘<samp class="samp">ntsc</samp>’</dt> <dd> 30000/1001 </dd> <dt>‘<samp class="samp">pal</samp>’</dt> <dd> 25/1 </dd> <dt>‘<samp class="samp">qntsc</samp>’</dt> <dd> 30000/1001 </dd> <dt>‘<samp class="samp">qpal</samp>’</dt> <dd> 25/1 </dd> <dt>‘<samp class="samp">sntsc</samp>’</dt> <dd> 30000/1001 </dd> <dt>‘<samp class="samp">spal</samp>’</dt> <dd> 25/1 </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> 24/1 </dd> <dt>‘<samp class="samp">ntsc-film</samp>’</dt> <dd> 24000/1001 </dd> </dl>

 <h3 class="section">7.6 Ratio# TOC</h3>

A ratio can be expressed as an expression, or in the form <var class="var">numerator</var>:<var class="var">denominator</var>. Note that a ratio with infinite (1/0) or negative value is considered valid, so you should check on the returned value if you want to exclude those values. The undefined value can be expressed using the "0:0" string.

<h3 class="section">7.7 Color# TOC</h3>

It can be the name of a color as defined below (case insensitive match) or a

<code class="code">[0x|#]RRGGBB[AA]</code> sequence, possibly followed by @ and a string

representing the alpha component.

The alpha component may be a string composed by "0x" followed by an

hexadecimal number or a decimal number between 0.0 and 1.0, which

represents the opacity value (‘<samp class="samp">0x00</samp>’ or ‘<samp class="samp">0.0</samp>’ means completely

transparent, ‘<samp class="samp">0xff</samp>’ or ‘<samp class="samp">1.0</samp>’ completely opaque). If the alpha

component is not specified then ‘<samp class="samp">0xff</samp>’ is assumed.

The string ‘<samp class="samp">random</samp>’ will result in a random color.

The following names of colors are recognized:

<dl class="table">

<dt>‘<samp class="samp">AliceBlue</samp>’</dt>

<dd> 0xF0F8FF

</dd>

<dt>‘<samp class="samp">AntiqueWhite</samp>’</dt>

<dd> 0xFAEBD7

</dd>

<dt>‘<samp class="samp">Aqua</samp>’</dt>

<dd> 0x00FFFF

</dd>

<dt>‘<samp class="samp">Aquamarine</samp>’</dt>

<dd> 0x7FFFD4
</dd>
<dt>‘<samp class=”samp”>Azure</samp>’</dt>
<dd> 0xF0FFFF
</dd>
<dt>‘<samp class=”samp”>Beige</samp>’</dt>
<dd> 0xF5F5DC
</dd>
<dt>‘<samp class=”samp”>Bisque</samp>’</dt>
<dd> 0xFFE4C4
</dd>
<dt>‘<samp class=”samp”>Black</samp>’</dt>
<dd> 0x000000
</dd>
<dt>‘<samp class=”samp”>BlanchedAlmond</samp>’</dt>
<dd> 0xFFEBCD
</dd>
<dt>‘<samp class=”samp”>Blue</samp>’</dt>
<dd> 0x0000FF
</dd>
<dt>‘<samp class=”samp”>BlueViolet</samp>’</dt>
<dd> 0x8A2BE2
</dd>
<dt>‘<samp class=”samp”>Brown</samp>’</dt>
<dd> 0xA52A2A
</dd>
<dt>‘<samp class=”samp”>BurlyWood</samp>’</dt>
<dd> 0xDEB887
</dd>
<dt>‘<samp class=”samp”>CadetBlue</samp>’</dt>
<dd> 0x5F9EA0

</dd>
<dt>‘<samp class="samp">Chartreuse</samp>’</dt>
<dd> 0x7FFF00
</dd>
<dt>‘<samp class="samp">Chocolate</samp>’</dt>
<dd> 0xD2691E
</dd>
<dt>‘<samp class="samp">Coral</samp>’</dt>
<dd> 0xFF7F50
</dd>
<dt>‘<samp class="samp">CornflowerBlue</samp>’</dt>
<dd> 0x6495ED
</dd>
<dt>‘<samp class="samp">Cornsilk</samp>’</dt>
<dd> 0xFFF8DC
</dd>
<dt>‘<samp class="samp">Crimson</samp>’</dt>
<dd> 0xDC143C
</dd>
<dt>‘<samp class="samp">Cyan</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class="samp">DarkBlue</samp>’</dt>
<dd> 0x00008B
</dd>
<dt>‘<samp class="samp">DarkCyan</samp>’</dt>
<dd> 0x008B8B
</dd>
<dt>‘<samp class="samp">DarkGoldenRod</samp>’</dt>
<dd> 0xB8860B
</dd>

<dt>‘<samp class=”samp”>DarkGray</samp>’</dt>
<dd> 0xA9A9A9
</dd>
<dt>‘<samp class=”samp”>DarkGreen</samp>’</dt>
<dd> 0x006400
</dd>
<dt>‘<samp class=”samp”>DarkKhaki</samp>’</dt>
<dd> 0xBDB76B
</dd>
<dt>‘<samp class=”samp”>DarkMagenta</samp>’</dt>
<dd> 0x8B008B
</dd>
<dt>‘<samp class=”samp”>DarkOliveGreen</samp>’</dt>
<dd> 0x556B2F
</dd>
<dt>‘<samp class=”samp”>Darkorange</samp>’</dt>
<dd> 0xFF8C00
</dd>
<dt>‘<samp class=”samp”>DarkOrchid</samp>’</dt>
<dd> 0x9932CC
</dd>
<dt>‘<samp class=”samp”>DarkRed</samp>’</dt>
<dd> 0x8B0000
</dd>
<dt>‘<samp class=”samp”>DarkSalmon</samp>’</dt>
<dd> 0xE9967A
</dd>
<dt>‘<samp class=”samp”>DarkSeaGreen</samp>’</dt>
<dd> 0x8FBC8F
</dd>
<dt>‘<samp class=”samp”>DarkSlateBlue</samp>’</dt>

<dd> 0x483D8B
</dd>
<dt>‘<samp class=”samp”>DarkSlateGray</samp>’</dt>
<dd> 0x2F4F4F
</dd>
<dt>‘<samp class=”samp”>DarkTurquoise</samp>’</dt>
<dd> 0x00CED1
</dd>
<dt>‘<samp class=”samp”>DarkViolet</samp>’</dt>
<dd> 0x9400D3
</dd>
<dt>‘<samp class=”samp”>DeepPink</samp>’</dt>
<dd> 0xFF1493
</dd>
<dt>‘<samp class=”samp”>DeepSkyBlue</samp>’</dt>
<dd> 0x00BFFF
</dd>
<dt>‘<samp class=”samp”>DimGray</samp>’</dt>
<dd> 0x696969
</dd>
<dt>‘<samp class=”samp”>DodgerBlue</samp>’</dt>
<dd> 0x1E90FF
</dd>
<dt>‘<samp class=”samp”>FireBrick</samp>’</dt>
<dd> 0xB22222
</dd>
<dt>‘<samp class=”samp”>FloralWhite</samp>’</dt>
<dd> 0xFFFAF0
</dd>
<dt>‘<samp class=”samp”>ForestGreen</samp>’</dt>
<dd> 0x228B22

</dd>
<dt>‘<samp class="samp">Fuchsia</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class="samp">Gainsboro</samp>’</dt>
<dd> 0xDCDCDC
</dd>
<dt>‘<samp class="samp">GhostWhite</samp>’</dt>
<dd> 0xF8F8FF
</dd>
<dt>‘<samp class="samp">Gold</samp>’</dt>
<dd> 0xFFD700
</dd>
<dt>‘<samp class="samp">GoldenRod</samp>’</dt>
<dd> 0xDAA520
</dd>
<dt>‘<samp class="samp">Gray</samp>’</dt>
<dd> 0x808080
</dd>
<dt>‘<samp class="samp">Green</samp>’</dt>
<dd> 0x008000
</dd>
<dt>‘<samp class="samp">GreenYellow</samp>’</dt>
<dd> 0xADFF2F
</dd>
<dt>‘<samp class="samp">HoneyDew</samp>’</dt>
<dd> 0xF0FFF0
</dd>
<dt>‘<samp class="samp">HotPink</samp>’</dt>
<dd> 0xFF69B4
</dd>

<dt>‘<samp class=”samp”>IndianRed</samp>’</dt>
<dd> 0xCD5C5C
</dd>
<dt>‘<samp class=”samp”>Indigo</samp>’</dt>
<dd> 0x4B0082
</dd>
<dt>‘<samp class=”samp”>Ivory</samp>’</dt>
<dd> 0xFFFFF0
</dd>
<dt>‘<samp class=”samp”>Khaki</samp>’</dt>
<dd> 0xF0E68C
</dd>
<dt>‘<samp class=”samp”>Lavender</samp>’</dt>
<dd> 0xE6E6FA
</dd>
<dt>‘<samp class=”samp”>LavenderBlush</samp>’</dt>
<dd> 0xFFF0F5
</dd>
<dt>‘<samp class=”samp”>LawnGreen</samp>’</dt>
<dd> 0x7CFC00
</dd>
<dt>‘<samp class=”samp”>LemonChiffon</samp>’</dt>
<dd> 0xFFFACD
</dd>
<dt>‘<samp class=”samp”>LightBlue</samp>’</dt>
<dd> 0xADD8E6
</dd>
<dt>‘<samp class=”samp”>LightCoral</samp>’</dt>
<dd> 0xF08080
</dd>
<dt>‘<samp class=”samp”>LightCyan</samp>’</dt>

<dd> 0xE0FFFF
</dd>
<dt>‘<samp class=”samp”>LightGoldenRodYellow</samp>’</dt>
<dd> 0xFAFAD2
</dd>
<dt>‘<samp class=”samp”>LightGreen</samp>’</dt>
<dd> 0x90EE90
</dd>
<dt>‘<samp class=”samp”>LightGrey</samp>’</dt>
<dd> 0xD3D3D3
</dd>
<dt>‘<samp class=”samp”>LightPink</samp>’</dt>
<dd> 0xFFB6C1
</dd>
<dt>‘<samp class=”samp”>LightSalmon</samp>’</dt>
<dd> 0xFFA07A
</dd>
<dt>‘<samp class=”samp”>LightSeaGreen</samp>’</dt>
<dd> 0x20B2AA
</dd>
<dt>‘<samp class=”samp”>LightSkyBlue</samp>’</dt>
<dd> 0x87CEFA
</dd>
<dt>‘<samp class=”samp”>LightSlateGray</samp>’</dt>
<dd> 0x778899
</dd>
<dt>‘<samp class=”samp”>LightSteelBlue</samp>’</dt>
<dd> 0xB0C4DE
</dd>
<dt>‘<samp class=”samp”>LightYellow</samp>’</dt>
<dd> 0xFFFFE0

</dd>
<dt>‘<samp class="samp">Lime</samp>’</dt>
<dd> 0x00FF00
</dd>
<dt>‘<samp class="samp">LimeGreen</samp>’</dt>
<dd> 0x32CD32
</dd>
<dt>‘<samp class="samp">Linen</samp>’</dt>
<dd> 0xFAF0E6
</dd>
<dt>‘<samp class="samp">Magenta</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class="samp">Maroon</samp>’</dt>
<dd> 0x800000
</dd>
<dt>‘<samp class="samp">MediumAquaMarine</samp>’</dt>
<dd> 0x66CDAA
</dd>
<dt>‘<samp class="samp">MediumBlue</samp>’</dt>
<dd> 0x0000CD
</dd>
<dt>‘<samp class="samp">MediumOrchid</samp>’</dt>
<dd> 0xBA55D3
</dd>
<dt>‘<samp class="samp">MediumPurple</samp>’</dt>
<dd> 0x9370D8
</dd>
<dt>‘<samp class="samp">MediumSeaGreen</samp>’</dt>
<dd> 0x3CB371
</dd>

<dt>‘<samp class=”samp”>MediumSlateBlue</samp>’</dt>
<dd> 0x7B68EE
</dd>
<dt>‘<samp class=”samp”>MediumSpringGreen</samp>’</dt>
<dd> 0x00FA9A
</dd>
<dt>‘<samp class=”samp”>MediumTurquoise</samp>’</dt>
<dd> 0x48D1CC
</dd>
<dt>‘<samp class=”samp”>MediumVioletRed</samp>’</dt>
<dd> 0xC71585
</dd>
<dt>‘<samp class=”samp”>MidnightBlue</samp>’</dt>
<dd> 0x191970
</dd>
<dt>‘<samp class=”samp”>MintCream</samp>’</dt>
<dd> 0xF5FFFA
</dd>
<dt>‘<samp class=”samp”>MistyRose</samp>’</dt>
<dd> 0xFFE4E1
</dd>
<dt>‘<samp class=”samp”>Moccasin</samp>’</dt>
<dd> 0xFFE4B5
</dd>
<dt>‘<samp class=”samp”>NavajoWhite</samp>’</dt>
<dd> 0xFFDEAD
</dd>
<dt>‘<samp class=”samp”>Navy</samp>’</dt>
<dd> 0x000080
</dd>
<dt>‘<samp class=”samp”>OldLace</samp>’</dt>

<dd> 0xFDF5E6
</dd>
<dt>‘<samp class=”samp”>Olive</samp>’</dt>
<dd> 0x808000
</dd>
<dt>‘<samp class=”samp”>OliveDrab</samp>’</dt>
<dd> 0x6B8E23
</dd>
<dt>‘<samp class=”samp”>Orange</samp>’</dt>
<dd> 0xFFA500
</dd>
<dt>‘<samp class=”samp”>OrangeRed</samp>’</dt>
<dd> 0xFF4500
</dd>
<dt>‘<samp class=”samp”>Orchid</samp>’</dt>
<dd> 0xDA70D6
</dd>
<dt>‘<samp class=”samp”>PaleGoldenRod</samp>’</dt>
<dd> 0xEEE8AA
</dd>
<dt>‘<samp class=”samp”>PaleGreen</samp>’</dt>
<dd> 0x98FB98
</dd>
<dt>‘<samp class=”samp”>PaleTurquoise</samp>’</dt>
<dd> 0xAFEEEE
</dd>
<dt>‘<samp class=”samp”>PaleVioletRed</samp>’</dt>
<dd> 0xD87093
</dd>
<dt>‘<samp class=”samp”>PapayaWhip</samp>’</dt>
<dd> 0xFFEFD5

</dd>
<dt>‘<samp class="samp">PeachPuff</samp>’</dt>
<dd> 0xFFDAB9
</dd>
<dt>‘<samp class="samp">Peru</samp>’</dt>
<dd> 0xCD853F
</dd>
<dt>‘<samp class="samp">Pink</samp>’</dt>
<dd> 0xFFC0CB
</dd>
<dt>‘<samp class="samp">Plum</samp>’</dt>
<dd> 0xDDA0DD
</dd>
<dt>‘<samp class="samp">PowderBlue</samp>’</dt>
<dd> 0xB0E0E6
</dd>
<dt>‘<samp class="samp">Purple</samp>’</dt>
<dd> 0x800080
</dd>
<dt>‘<samp class="samp">Red</samp>’</dt>
<dd> 0xFF0000
</dd>
<dt>‘<samp class="samp">RosyBrown</samp>’</dt>
<dd> 0xBC8F8F
</dd>
<dt>‘<samp class="samp">RoyalBlue</samp>’</dt>
<dd> 0x4169E1
</dd>
<dt>‘<samp class="samp">SaddleBrown</samp>’</dt>
<dd> 0x8B4513
</dd>

<dt>‘<samp class=”samp”>Salmon</samp>’</dt>
<dd> 0xFA8072
</dd>
<dt>‘<samp class=”samp”>SandyBrown</samp>’</dt>
<dd> 0xF4A460
</dd>
<dt>‘<samp class=”samp”>SeaGreen</samp>’</dt>
<dd> 0x2E8B57
</dd>
<dt>‘<samp class=”samp”>SeaShell</samp>’</dt>
<dd> 0xFFF5EE
</dd>
<dt>‘<samp class=”samp”>Sienna</samp>’</dt>
<dd> 0xA0522D
</dd>
<dt>‘<samp class=”samp”>Silver</samp>’</dt>
<dd> 0xC0C0C0
</dd>
<dt>‘<samp class=”samp”>SkyBlue</samp>’</dt>
<dd> 0x87CEEB
</dd>
<dt>‘<samp class=”samp”>SlateBlue</samp>’</dt>
<dd> 0x6A5ACD
</dd>
<dt>‘<samp class=”samp”>SlateGray</samp>’</dt>
<dd> 0x708090
</dd>
<dt>‘<samp class=”samp”>Snow</samp>’</dt>
<dd> 0xFFFAFA
</dd>
<dt>‘<samp class=”samp”>SpringGreen</samp>’</dt>

<dd> 0x00FF7F
</dd>
<dt>‘<samp class=”samp”>SteelBlue</samp>’</dt>
<dd> 0x4682B4
</dd>
<dt>‘<samp class=”samp”>Tan</samp>’</dt>
<dd> 0xD2B48C
</dd>
<dt>‘<samp class=”samp”>Teal</samp>’</dt>
<dd> 0x008080
</dd>
<dt>‘<samp class=”samp”>Thistle</samp>’</dt>
<dd> 0xD8BFD8
</dd>
<dt>‘<samp class=”samp”>Tomato</samp>’</dt>
<dd> 0xFF6347
</dd>
<dt>‘<samp class=”samp”>Turquoise</samp>’</dt>
<dd> 0x40E0D0
</dd>
<dt>‘<samp class=”samp”>Violet</samp>’</dt>
<dd> 0xEE82EE
</dd>
<dt>‘<samp class=”samp”>Wheat</samp>’</dt>
<dd> 0xF5DEB3
</dd>
<dt>‘<samp class=”samp”>White</samp>’</dt>
<dd> 0xFFFFFF
</dd>
<dt>‘<samp class=”samp”>WhiteSmoke</samp>’</dt>
<dd> 0xF5F5F5

</dd>

<dt>‘<samp class="samp">Yellow</samp>’</dt>

<dd> 0xFFFF00

</dd>

<dt>‘<samp class="samp">YellowGreen</samp>’</dt>

<dd> 0x9ACD32

</dd>

</dl>

 <h3 class="section">7.8 Channel Layout# TOC</h3>

A channel layout specifies the spatial disposition of the channels in a multi-channel audio stream. To specify a channel layout, FFmpeg makes use of a special syntax. Individual channels are identified by an id, as given by the table below: <dl class="table"> <dt>‘<samp class="samp">FL</samp>’</dt> <dd> front left </dd> <dt>‘<samp class="samp">FR</samp>’</dt> <dd> front right </dd> <dt>‘<samp class="samp">FC</samp>’</dt> <dd> front center </dd> <dt>‘<samp class="samp">LFE</samp>’</dt> <dd> low frequency </dd> <dt>‘<samp class="samp">BL</samp>’</dt> <dd> back left </dd> <dt>‘<samp class="samp">BR</samp>’</dt> <dd> back right </dd> <dt>‘<samp class="samp">FLC</samp>’</dt> <dd> front left-of-center </dd> <dt>‘<samp class="samp">FRC</samp>’</dt> <dd> front right-of-center </dd> <dt>‘<samp class="samp">BC</samp>’</dt> <dd> back center </dd> <dt>‘<samp class="samp">SL</samp>’</dt> <dd> side left </dd> <dt>‘<samp class="samp">SR</samp>’</dt> <dd> side right </dd> <dt>‘<samp class="samp">TC</samp>’</dt> <dd> top center </dd> <dt>‘<samp class="samp">TFL</samp>’</dt> <dd> top front left </dd> <dt>‘<samp class="samp">TFC</samp>’</dt> <dd> top front center </dd> <dt>‘<samp class="samp">TFR</samp>’</dt> <dd> top front right </dd> <dt>‘<samp class="samp">TBL</samp>’</dt> <dd> top back left </dd> <dt>‘<samp class="samp">TBC</samp>’</dt> <dd> top back center </dd> <dt>‘<samp class="samp">TBR</samp>’</dt> <dd> top back right </dd> <dt>‘<samp class="samp">DL</samp>’</dt> <dd> downmix left </dd> <dt>‘<samp class="samp">DR</samp>’</dt> <dd> downmix right </dd> <dt>‘<samp class="samp">WL</samp>’</dt> <dd> wide left </dd> <dt>‘<samp class="samp">WR</samp>’</dt> <dd> wide right </dd> <dt>‘<samp class="samp">SDL</samp>’</dt> <dd> surround direct left </dd> <dt>‘<samp class="samp">SDR</samp>’</dt>

<dd> surround direct right </dd> <dt>‘<samp class="samp">LFE2</samp>’</dt>
<dd> low frequency 2 </dd> </dl>

Standard channel layout compositions can be specified by using the

following identifiers:

<dl class="table">

<dt>‘<samp class="samp">mono</samp>’</dt>

<dd> FC

</dd>

<dt>‘<samp class="samp">stereo</samp>’</dt>

<dd> FL+FR

</dd>

<dt>‘<samp class="samp">2.1</samp>’</dt>

<dd> FL+FR+LFE

</dd>

<dt>‘<samp class="samp">3.0</samp>’</dt>

<dd> FL+FR+FC

</dd>

<dt>‘<samp class="samp">3.0(back)</samp>’</dt>

<dd> FL+FR+BC

</dd>

<dt>‘<samp class="samp">4.0</samp>’</dt>

<dd> FL+FR+FC+BC

</dd>

<dt>‘<samp class="samp">quad</samp>’</dt>

<dd> FL+FR+BL+BR

</dd>

<dt>‘<samp class="samp">quad(side)</samp>’</dt>

<dd> FL+FR+SL+SR

</dd>

<dt>‘<samp class="samp">3.1</samp>’</dt>

<dd> FL+FR+FC+LFE

</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR
</dd>
<dt>‘<samp class="samp">5.0(side)</samp>’</dt>
<dd> FL+FR+FC+SL+SR
</dd>
<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR
</dd>
<dt>‘<samp class="samp">5.1(side)</samp>’</dt>
<dd> FL+FR+FC+LFE+SL+SR
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> FL+FR+FC+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.0(front)</samp>’</dt>
<dd> FL+FR+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">3.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+TFL+TFR
</dd>
<dt>‘<samp class="samp">hexagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC+SL+SR
</dd>

<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1(front)</samp>’</dt>
<dd> FL+FR+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.0(front)</samp>’</dt>
<dd> FL+FR+FC+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.1(wide)</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC
</dd>
<dt>‘<samp class="samp">7.1(wide-side)</samp>’</dt>
<dd> FL+FR+FC+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">5.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR
</dd>
<dt>‘<samp class="samp">octagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR
</dd>
<dt>‘<samp class="samp">cube</samp>’</dt>
<dd> FL+FR+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">5.1.4</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR
</dd>
<dt>‘<samp class="samp">7.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.2.3</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBC+LFE2
</dd>
<dt>‘<samp class="samp">9.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">9.1.6</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR+TSL+TSR
</dd>
<dt>‘<samp class="samp">hexadecagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR+WL+WR+TBL+TBR+TBC+TFC+TFL+TFR
</dd>
<dt>‘<samp class="samp">binaural</samp>’</dt>
<dd> BIL+BIR
</dd>
<dt>‘<samp class="samp">downmix</samp>’</dt>
<dd> DL+DR
</dd>
<dt>‘<samp class="samp">22.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+BC+SL+SR+TC+TFL+TFC+TFR+TBL+TBC+TB
</dd>
</dl>

A custom channel layout can be specified as a sequence of terms, separated by ’+’. Each term can be: <ul class=”itemize mark-bullet”> the name of a single channel (e.g. ‘<samp class=”samp”>FL</samp>’, ‘<samp class=”samp”>FR</samp>’, ‘<samp class=”samp”>FC</samp>’, ‘<samp class=”samp”>LFE</samp>’, etc.), each optionally containing a custom name after a ’@’, (e.g. ‘<samp class=”samp”>FL@Left</samp>’, ‘<samp class=”samp”>FR@Right</samp>’, ‘<samp class=”samp”>FC@Center</samp>’, ‘<samp class=”samp”>LFE@Low_Frequency</samp>’, etc.)

A standard channel layout can be specified by the following: <ul class=”itemize mark-bullet”> the name of a single channel (e.g. ‘<samp class=”samp”>FL</samp>’, ‘<samp class=”samp”>FR</samp>’, ‘<samp class=”samp”>FC</samp>’, ‘<samp class=”samp”>LFE</samp>’, etc.)

the name of a standard channel layout (e.g. ‘<samp class=”samp”>mono</samp>’, ‘<samp class=”samp”>stereo</samp>’, ‘<samp class=”samp”>4.0</samp>’, ‘<samp class=”samp”>quad</samp>’, ‘<samp class=”samp”>5.0</samp>’, etc.)

a number of channels, in decimal, followed by ’c’, yielding the default channel layout for that number of channels (see the function <code class=”code”>av_channel_layout_default</code>). Note that not all channel counts have a default layout.

a number of channels, in decimal, followed by ’C’, yielding an unknown channel layout with the specified number of channels. Note that not all channel layout specification strings support unknown channel layouts.

a channel layout mask, in hexadecimal starting with "0x" (see the <code class=”code”>AV_CH_*</code> macros in <samp class=”file”>libavutil/channel_layout.h</samp>.

Before libavutil version 53 the trailing character "c" to specify a number of channels was optional, but now it is required, while a channel layout mask can also be specified as a decimal number (if and only if not followed by "c" or "C"). See also the function <code class=”code”>av_channel_layout_from_string</code> defined in <samp class=”file”>libavutil/channel_layout.h</samp>. <h2 class=”chapter”>8 Expression Evaluation# TOC</h2>

When evaluating an arithmetic expression, FFmpeg uses an internal

formula evaluator, implemented through the <samp class=”file”>libavutil/eval.h</samp>

interface.

An expression may contain unary, binary operators, constants, and

functions.

Two expressions <var class="var">expr1</var> and <var class="var">expr2</var>
can be combined to form

another expression "<var class="var">expr1</var>;<var class="var">expr2</var>".

<var class="var">expr1</var> and <var class="var">expr2</var> are evaluated in turn, and the new

expression evaluates to the value of <var class="var">expr2</var>.

The following binary operators are available: <code class="code">+</code>,
<code class="code">-</code>,

<code class="code">*</code>, <code class="code">/</code>, <code
class="code">^</code>.

The following unary operators are available: <code class="code">+</code>,
<code class="code">-</code>.

Some internal variables can be used to store and load intermediary

results. They can be accessed using the <code class="code">ld</code> and
<code class="code">st</code>

functions with an index argument varying from 0 to 9 to specify which

internal variable to access.

The following functions are available:

<dl class="table">

<dt><samp class="option">abs(x)</samp></dt>

<dd> Compute absolute value of <var class="var">x</var>.

</dd>

<dt><samp class="option">acos(x)</samp></dt>

<dd> Compute arccosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">asin(x)</samp></dt>

<dd> Compute arcsine of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan(x)</samp></dt>

<dd> Compute arctangent of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan2(y, x)</samp></dt>

<dd> Compute principal value of the arc tangent of <var class="var">y</var>/<var class="var">x</var>.

</dd>

<dt><samp class="option">between(x, min, max)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">min</var> and lesser than or

equal to <var class="var">max</var>, 0 otherwise.

</dd>

<dt><samp class="option">bitand(x, y)</samp></dt>

<dt><samp class="option">bitor(x, y)</samp></dt>

<dd> Compute bitwise and/or operation on <var class="var">x</var> and <var class="var">y</var>.

The results of the evaluation of <var class="var">x</var> and <var class="var">y</var> are converted to

integers before executing the bitwise operation.

Note that both the conversion to integer and the conversion back to

floating point can lose precision. Beware of unexpected results for

large numbers (usually 2^53 and larger).

</dd>

<dt><samp class="option">ceil(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> upwards to the nearest

integer. For example, "ceil(1.5)" is "2.0".

</dd>

<dt><samp class="option">clip(x, min, max)</samp></dt>

<dd> Return the value of <var class="var">x</var> clipped between <var class="var">min</var> and <var class="var">max</var>.

</dd>

<dt><samp class="option">cos(x)</samp></dt>

<dd> Compute cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">cosh(x)</samp></dt>

<dd> Compute hyperbolic cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">eq(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> and <var class="var">y</var> are equivalent, 0 otherwise.

</dd>

<dt><samp class="option">exp(x)</samp></dt>

<dd> Compute exponential of <var class="var">x</var> (with base <code class="code">e</code>, the Euler’s number).

</dd>

<dt><samp class=”option”>floor(expr)</samp></dt>

<dd> Round the value of expression <var class=”var”>expr</var> downwards to the nearest

integer. For example, "floor(-1.5)" is "-2.0".

</dd>

<dt><samp class=”option”>gauss(x)</samp></dt>

<dd> Compute Gauss function of <var class=”var”>x</var>, corresponding to

<code class=”code”>exp(-x*x/2) / sqrt(2*PI)</code>.

</dd>

<dt><samp class=”option”>gcd(x, y)</samp></dt>

<dd> Return the greatest common divisor of <var class=”var”>x</var> and <var class=”var”>y</var>. If both <var class=”var”>x</var> and

<var class=”var”>y</var> are 0 or either or both are less than zero then behavior is undefined.

</dd>

<dt><samp class=”option”>gt(x, y)</samp></dt>

<dd> Return 1 if <var class=”var”>x</var> is greater than <var class=”var”>y</var>, 0 otherwise.

</dd>

<dt><samp class=”option”>gte(x, y)</samp></dt>

<dd> Return 1 if <var class=”var”>x</var> is greater than or equal to <var class=”var”>y</var>, 0 otherwise.

</dd>

<dt><samp class=”option”>hypot(x, y)</samp></dt>

<dd> This function is similar to the C function with the same name; it returns

"sqrt(<var class="var">x</var>*<var class="var">x</var> + <var class="var">y</var>*<var class="var">y</var>)", the length of the hypotenuse of a

right triangle with sides of length <var class="var">x</var> and <var class="var">y</var>, or the distance of the

point (<var class="var">x</var>, <var class="var">y</var>) from the origin.

</dd>

<dt><samp class="option">if(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the result of

the evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">if(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">ifnot(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the result of the

evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">ifnot(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">isinf(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is +/-INFINITY, 0.0 otherwise.

</dd>

<dt><samp class="option">isnan(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is NAN, 0.0 otherwise.

</dd>

<dt><samp class="option">ld(idx)</samp></dt>

<dd> Load the value of the internal variable with index <var class="var">idx</var>, which was

previously stored with st(<var class="var">idx</var>, <var class="var">expr</var>).

The function returns the loaded value.

</dd>

<dt><samp class="option">lerp(x, y, z)</samp></dt>

<dd> Return linear interpolation between <var class="var">x</var> and <var class="var">y</var> by amount of <var class="var">z</var>.

</dd>

<dt><samp class="option">log(x)</samp></dt>

<dd> Compute natural logarithm of <var class="var">x</var>.

</dd>

<dt><samp class="option">lt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">lte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">max(x, y)</samp></dt>

<dd> Return the maximum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">min(x, y)</samp></dt>

<dd> Return the minimum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">mod(x, y)</samp></dt>

<dd> Compute the remainder of division of <var class="var">x</var> by <var class="var">y</var>.

</dd>

<dt><samp class="option">not(expr)</samp></dt>

<dd> Return 1.0 if <var class="var">expr</var> is zero, 0.0 otherwise.

</dd>

<dt><samp class="option">pow(x, y)</samp></dt>

<dd> Compute the power of <var class="var">x</var> elevated <var class="var">y</var>, it is equivalent to

"(<var class="var">x</var>)^(<var class="var">y</var>)".

</dd>

<dt><samp class="option">print(t)</samp></dt>

<dt><samp class="option">print(t, l)</samp></dt>

<dd> Print the value of expression <var class="var">t</var> with loglevel <var class="var">l</var>. If <var class="var">l</var> is not

specified then a default log level is used.

Return the value of the expression printed.

</dd>

<dt><samp class="option">random(idx)</samp></dt>

<dd> Return a pseudo random value between 0.0 and 1.0. <var class="var">idx</var> is the

index of the internal variable used to save the seed/state, which can be

previously stored with <code class="code">st(idx)</code>.

To initialize the seed, you need to store the seed value as a 64-bit

unsigned integer in the internal variable with index <var class="var">idx</var>.

For example, to store the seed with value <code class="code">42</code> in the internal

variable with index <code class="code">0</code> and print a few random values:

<pre class="example-preformatted">st(0,42); print(random(0)); print(random(0)); print(random(0))

</pre>

</dd> <dt><samp class="option">randomi(idx, min, max)</samp></dt> <dd> Return a pseudo random value in the interval between <var class="var">min</var> and <var class="var">max</var>. <var class="var">idx</var> is the index of the internal variable which will be used to save the seed/state, which can be previously stored with <code class="code">st(idx)</code>. To initialize the seed, you need to store the seed value as a 64-bit unsigned integer in the internal variable with index <var class="var">idx</var>. </dd> <dt><samp class="option">root(expr, max)</samp></dt> <dd> Find an input value for which the function represented by <var class="var">expr</var> with argument <var class="var">ld(0)</var> is 0 in the interval 0..<var class="var">max</var>. The expression in <var class="var">expr</var> must denote a continuous function or the result is undefined. <var class="var">ld(0)</var> is used to represent the function input value, which means that the given expression will be evaluated multiple times with various input values that the expression can access through

<code class=”code”>ld(0)</code>. When the expression evaluates to 0 then the corresponding input value will be returned. </dd> <dt><samp class=”option”>round(expr)</samp></dt> <dd> Round the value of expression <var class=”var”>expr</var> to the nearest integer. For example, "round(1.5)" is "2.0". </dd> <dt><samp class=”option”>sgn(x)</samp></dt> <dd> Compute sign of <var class=”var”>x</var>. </dd> <dt><samp class=”option”>sin(x)</samp></dt> <dd> Compute sine of <var class=”var”>x</var>. </dd> <dt><samp class=”option”>sinh(x)</samp></dt> <dd> Compute hyperbolic sine of <var class=”var”>x</var>. </dd> <dt><samp class=”option”>sqrt(expr)</samp></dt> <dd> Compute the square root of <var class=”var”>expr</var>. This is equivalent to "(<var class=”var”>expr</var>)^.5". </dd> <dt><samp class=”option”>squish(x)</samp></dt> <dd> Compute expression <code class=”code”>1/(1 + exp(4*x))</code>. </dd> <dt><samp class=”option”>st(idx, expr)</samp></dt> <dd> Store the value of the expression <var class=”var”>expr</var> in an internal variable. <var class=”var”>idx</var> specifies the index of the variable where to store the value, and it is a value ranging from 0 to 9. The function returns the value stored in the internal variable. The stored value can be retrieved with <code class=”code”>ld(var)</code>. Note: variables are currently not shared between expressions. </dd> <dt><samp class=”option”>tan(x)</samp></dt> <dd> Compute tangent of <var class=”var”>x</var>. </dd> <dt><samp class=”option”>tanh(x)</samp></dt> <dd> Compute hyperbolic tangent of <var class=”var”>x</var>. </dd> <dt><samp class=”option”>taylor(expr, x)</samp></dt> <dt><samp class=”option”>taylor(expr, x, idx)</samp></dt> <dd> Evaluate a Taylor series at <var class=”var”>x</var>, given an expression representing the <code class=”code”>ld(idx)</code>-th derivative of a function at 0. When the series does not converge the result is undefined. <var class=”var”>ld(idx)</var> is used to represent the derivative order in <var class=”var”>expr</var>, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class=”code”>ld(idx)</code>. If <var class=”var”>idx</var> is not specified then 0 is assumed. Note, when you have the derivatives at y instead of 0, <code class=”code”>taylor(expr, x-y)</code> can be used. </dd> <dt><samp class=”option”>time(0)</samp></dt> <dd> Return the current (wallclock) time in seconds. </dd> <dt><samp class=”option”>trunc(expr)</samp></dt> <dd> Round the value of expression <var class=”var”>expr</var> towards zero to the nearest integer. For example, "trunc(-1.5)" is "-1.0". </dd> <dt><samp class=”option”>while(cond, expr)</samp></dt> <dd> Evaluate expression <var class=”var”>expr</var> while the expression <var class=”var”>cond</var> is non-zero, and returns the value of the last <var class=”var”>expr</var> evaluation, or NAN if <var class=”var”>cond</var> was always false. </dd> </dl>

The following constants are available: <dl class=”table”> <dt><samp class=”option”>PI</samp></dt> <dd> area of the unit disc, approximately 3.14 </dd> <dt><samp class=”option”>E</samp></dt> <dd> exp(1) (Euler’s number), approximately 2.718 </dd> <dt><samp class=”option”>PHI</samp></dt> <dd> golden ratio (1+sqrt(5))/2, approximately 1.618 </dd> </dl>

Assuming that an expression is considered "true" if it has a non-zero value, note that: <code class=”code”>*</code> works like AND <code class=”code”>+</code> works like OR For example the construct: <pre class=”example-preformatted”>if (A AND B) then C </pre> is equivalent to: <pre class=”example-preformatted”>if(A*B, C) </pre>

In your C code, you can extend the list of unary and binary functions, and define recognized constants, so that they are available for your expressions. The evaluator also recognizes the International System unit prefixes. If ’i’ is appended after the prefix, binary prefixes are used, which are based on powers of 1024 instead of powers of 1000. The ’B’ postfix multiplies the value by 8, and can be appended after a unit prefix or used alone. This allows using for example ’KB’, ’MiB’, ’G’ and ’B’ as number postfix. The list of available International System prefixes follows, with indication of the corresponding powers of 10 and of 2. <dl class=”table”> <dt><samp class=”option”>y</samp></dt> <dd> 10^-24 / 2^-80 </dd> <dt><samp class=”option”>z</samp></dt> <dd> 10^-21 / 2^-70 </dd> <dt><samp class=”option”>a</samp></dt> <dd> 10^-18 / 2^-60 </dd> <dt><samp class=”option”>f</samp></dt> <dd> 10^-15 / 2^-50 </dd> <dt><samp class=”option”>p</samp></dt> <dd> 10^-12 / 2^-40 </dd> <dt><samp class=”option”>n</samp></dt> <dd> 10^-9 / 2^-30 </dd> <dt><samp class=”option”>u</samp></dt> <dd> 10^-6 / 2^-20 </dd> <dt><samp class=”option”>m</samp></dt> <dd> 10^-3 / 2^-10 </dd> <dt><samp class=”option”>c</samp></dt> <dd> 10^-2 </dd> <dt><samp class=”option”>d</samp></dt> <dd> 10^-1 </dd> <dt><samp class=”option”>h</samp></dt> <dd> 10^2 </dd> <dt><samp class=”option”>k</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>K</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>M</samp></dt> <dd> 10^6 / 2^20 </dd> <dt><samp class=”option”>G</samp></dt> <dd> 10^9 / 2^30 </dd> <dt><samp class=”option”>T</samp></dt> <dd> 10^12 / 2^40 </dd> <dt><samp class=”option”>P</samp></dt> <dd> 10^15 / 2^50 </dd> <dt><samp class=”option”>E</samp></dt> <dd> 10^18 / 2^60 </dd> <dt><samp class=”option”>Z</samp></dt> <dd> 10^21 / 2^70 </dd> <dt><samp class=”option”>Y</samp></dt> <dd> 10^24 / 2^80 </dd> </dl>

 <h2 class="chapter">9 Codec Options# TOC</h2>

libavcodec provides some generic global options, which can be set on all the encoders and decoders. In addition, each codec may support so-called private options, which are specific for a given codec. Sometimes, a global option may only affect a specific kind of codec, and may be nonsensical or ignored by another, so you need to be aware of the meaning of the specified options. Also some options are meant only for decoding or encoding. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVCodecContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follow: <dl class="table"> <dt><samp class="option">b <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set bitrate in bits/s. Default value is 200K. </dd> <dt><samp class="option">ab <var class="var">integer</var> (encoding,audio)</samp><
<dd> Set audio bitrate (in bits/s). Default value is 128K. </dd> <dt><samp class="option">bt <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set video bitrate tolerance (in bits/s). In 1-pass mode, bitrate tolerance specifies how far ratecontrol is willing to deviate from the target average bitrate value. This is not related to min/max bitrate. Lowering tolerance too much has an adverse effect on quality. </dd> <dt><samp class="option">flags <var class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></dt> <dd> Set generic flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">mv4</samp>’</dt> <dd> Use four motion vector by macroblock (mpeg4). </dd> <dt>‘<samp class="samp">qpel</samp>’</dt> <dd> Use 1/4 pel motion compensation. </dd> <dt>‘<samp class="samp">loop</samp>’</dt> <dd> Use loop filter. </dd> <dt>‘<samp class="samp">qscale</samp>’</dt> <dd> Use fixed qscale. </dd> <dt>‘<samp class="samp">pass1</samp>’</dt> <dd> Use internal 2pass ratecontrol in first pass mode. </dd> <dt>‘<samp class="samp">pass2</samp>’</dt> <dd> Use internal 2pass ratecontrol in second pass mode. </dd> <dt>‘<samp class="samp">gray</samp>’</dt> <dd> Only decode/encode grayscale. </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> Set error[?] variables during encoding. </dd> <dt>‘<samp class="samp">truncated</samp>’</dt> <dd> Input bitstream might be randomly truncated. </dd> <dt>‘<samp class="samp">drop_changed</samp>’</dt> <dd> Don’t output frames whose parameters differ from first decoded frame in stream. Error

AVERROR_INPUT_CHANGED is returned when a frame is dropped.
</dd> <dt>‘<samp class="samp">ildct</samp>’</dt>
<dd> Use interlaced DCT. </dd> <dt>‘<samp class="samp">low_delay</samp>’<
<dd> Force low delay. </dd> <dt>‘<samp class="samp">global_header</samp>’<
<dd> Place global headers in extradata instead of every keyframe.
</dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. (except (I)DCT). This ensures that file and data checksums are reproducible and match between platforms. Its primary use is for regression testing.
</dd> <dt>‘<samp class="samp">aic</samp>’</dt>
<dd> Apply H263 advanced intra coding / mpeg4 ac prediction.
</dd> <dt>‘<samp class="samp">ilme</samp>’</dt>
<dd> Apply interlaced motion estimation. </dd> <dt>‘<samp class="samp">cgop</samp>’</dt> <dd> Use closed gop.
</dd> <dt>‘<samp class="samp">output_corrupt</samp>’</dt>
<dd> Output even potentially corrupted frames. </dd> </dl>

</dd>

<dt><samp class="option">time_base <var class="var">rational number</var></samp></dt>

<dd> Set codec time base.

It is the fundamental unit of time (in seconds) in terms of which

frame timestamps are represented. For fixed-fps content, timebase

should be <code class="code">1 / frame_rate</code> and timestamp increments should be

identically 1.

</dd>

<dt><samp class="option">g <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set the group of picture (GOP) size. Default value is 12.

</dd>

<dt><samp class="option">ar <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set audio sampling rate (in Hz).

</dd>

<dt><samp class="option">ac <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set number of audio channels.

</dd>

<dt><samp class="option">cutoff <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set cutoff bandwidth. (Supported only by selected encoders, see

their respective documentation sections.)

</dd>

<dt><samp class="option">frame_size <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set audio frame size.

Each submitted frame except the last must contain exactly frame_size

samples per channel. May be 0 when the codec has

CODEC_CAP_VARIABLE_FRAME_SIZE set, in that case the frame size is not

restricted. It is set by some decoders to indicate constant frame

size.

</dd>

<dt><samp class="option">frame_number <var class="var">integer</var></samp></dt>

<dd> Set the frame number.

</dd>

<dt><samp class="option">delay <var class="var">integer</var></samp></dt>

<dt><samp class="option">qcomp <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale compression (VBR). It is used as a constant

in the ratecontrol equation. Recommended range for default rc_eq:

0.0-1.0.

</dd>

<dt><samp class="option">qblur <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale blur (VBR).

</dd>

<dt><samp class="option">qmin <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set min video quantizer scale (VBR). Must be included between -1 and

69, default value is 2.

</dd>

<dt><samp class="option">qmax <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max video quantizer scale (VBR). Must be included between -1 and

1024, default value is 31.

</dd>

<dt><samp class="option">qdiff <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max difference between the quantizer scale (VBR).

</dd>

<dt><samp class="option">bf <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max number of B frames between non-B-frames.

Must be an integer between -1 and 16. 0 means that B-frames are

disabled. If a value of -1 is used, it will choose an automatic value

depending on the encoder.

Default value is 0.

</dd>

<dt><samp class="option">b_qfactor <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set qp factor between P and B frames.

</dd>

<dt><samp class="option">codec_tag <var class="var">integer</var></samp></dt>

<dt><samp class="option">bug <var class="var">flags</var> (decoding,video)</samp></dt>

<dd> Workaround not auto detected encoder bugs.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">autodetect</samp>’</dt>

<dt>‘<samp class="samp">xvid_ilace</samp>’</dt>

<dd> Xvid interlacing bug (autodetected if fourcc==XVIX)

</dd>

<dt>‘<samp class="samp">ump4</samp>’</dt>

<dd> (autodetected if fourcc==UMP4)

</dd>

<dt>‘<samp class="samp">no_padding</samp>’</dt>

<dd> padding bug (autodetected)

</dd>

<dt>‘<samp class="samp">amv</samp>’</dt>

<dt>‘<samp class="samp">qpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">std_qpel</samp>’</dt>

<dd> old standard qpel (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">qpel_chroma2</samp>’</dt>

<dt>‘<samp class="samp">direct_blocksize</samp>’</dt>

<dd> direct-qpel-blocksize bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">edge</samp>’</dt>

<dd> edge padding bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">hpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">dc_clip</samp>’</dt>

<dt>‘<samp class="samp">ms</samp>’</dt>

<dd> Workaround various bugs in microsoft broken decoders.

</dd>

<dt>‘<samp class="samp">trunc</samp>’</dt>

<dd> trancated frames

</dd>

<dt>‘<samp class="samp">h264_dp_nnz</samp>’</dt>

<dd> H.264 nC derivation for partitioned slices, as done by JM rather than
by

9.2.1 (autodetected)

</dd>

</dl>

</dd> <dt><samp class="option">strict <var class="var">integer</var>
(decoding/encoding,audio,video)</samp></dt>
<dd> Specify how strictly to follow the standards. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt>
<dd> strictly conform to an older more strict version of the spec or reference
software </dd> <dt>‘<samp class="samp">strict</samp>’</dt>
<dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt>
<dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow
unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt>
<dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note:

experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> <dt><samp class="option">b_qoffset <var class="var">float</var> (encoding,video)</samp></dt> <dd> Set QP offset between P and B frames. </dd> <dt><samp class="option">err_detect <var class="var">flags</var> (decoding,audio,video)</samp></dt> <dd> Set error detection flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt> <dd> verify embedded CRCs </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> detect bitstream specification deviations </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> detect improper bitstream length </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> abort decoding on minor error detection </dd> <dt>‘<samp class="samp">ignore_err</samp>’</dt> <dd> ignore decoding errors, and continue decoding. This is useful if you want to analyze the content of a video and thus want everything to be decoded no matter what. This option will not result in a video that is pleasing to watch in case of errors. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> consider things that violate the spec and have not been seen in the wild as errors </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> consider all spec non compliancies as errors </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> consider things that a sane encoder should not do as an error </dd> </dl>

</dd> <dt><samp class="option">has_b_frames <var class="var">integer</var></samp></dt> <dt><samp class="option">block_align <var class="var">integer</var></samp></dt> <dt><samp class="option">rc_override_count <var class="var">integer</var></samp></dt> <dt><samp class="option">maxrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set max bitrate tolerance (in bits/s). Requires bufsize to be set. </dd> <dt><samp class="option">minrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set min bitrate tolerance (in bits/s). Most useful in setting up a CBR encode. It is of little use elsewise. </dd> <dt><samp class="option">bufsize <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set ratecontrol buffer size (in bits). </dd> <dt><samp class="option">i_qfactor <var class="var">float</var> (encoding,video)</sam <dd> Set QP factor between P and I frames. </dd> <dt><samp class="option">i_qoffset <var class="var">float</var> (encoding,video)</sam <dd> Set QP offset between P and I frames. </dd> <dt><samp class="option">dct <var class="var">integer</var> (encoding,video)</samp> <dd> Set DCT algorithm. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> autoselect a good one (default) </dd> <dt>‘<samp

class=”samp”>fastint</samp>’</dt> <dd> fast integer
</dd> <dt>‘<samp class=”samp”>int</samp>’</dt>
<dd> accurate integer </dd> <dt>‘<samp class=”samp”>mmx</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>faan</samp>’</dt> <dd> floating point AAN
DCT </dd> </dl>

</dd> <dt><samp class=”option”>lumi_mask <var class=”var”>float</var>
(encoding,video)</samp></dt> <dd> Compress
bright areas stronger than medium ones. </dd> <dt><samp
class=”option”>tcplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set temporal complexity masking. </dd> <dt><samp
class=”option”>scplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set spatial complexity masking. </dd> <dt><samp
class=”option”>p_mask <var class=”var”>float</var> (encoding,video)</samp
<dd> Set inter masking. </dd> <dt><samp class=”option”>dark_mask
<var class=”var”>float</var> (encoding,video)</samp></dt>
<dd> Compress dark areas stronger than medium ones.
</dd> <dt><samp class=”option”>idct <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Select IDCT implementation. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dt>‘<samp class=”samp”>int</samp>’</dt> <dt>‘<samp
class=”samp”>simple</samp>’</dt> <dt>‘<samp class=”samp”>simplemmx</samp>&rsqu
<dt>‘<samp class=”samp”>simpleauto</samp>’</dt>
<dd> Automatically pick a IDCT compatible with the simple one
 </dd> <dt>‘<samp class=”samp”>arm</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>sh4</samp>’</dt> <dt>‘<samp class=”samp”>simplearm</samp>’<
<dt>‘<samp class=”samp”>simplearmv5te</samp>’</dt>
<dt>‘<samp class=”samp”>simplearmv6</samp>’</dt>
<dt>‘<samp class=”samp”>simpleneon</samp>’</dt>
<dt>‘<samp class=”samp”>xvid</samp>’</dt> <dt>‘<samp
class=”samp”>faani</samp>’</dt> <dd> floating point AAN
IDCT </dd> </dl>

</dd> <dt><samp class=”option”>slice_count <var class=”var”>integer</var></samp></dt>
<dt><samp class=”option”>ec <var class=”var”>flags</var> (<em
class=”emph”>decoding,video)</samp></dt> <dd> Set error
concealment strategy. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>guess_mvs</samp>’</dt>
<dd> iterative motion vector (MV) search (slow) </dd> <dt>‘<samp
class=”samp”>deblock</samp>’</dt> <dd> use strong deblock
filter for damaged MBs </dd> <dt>‘<samp class=”samp”>favor_inter</samp>’</dt>
<dd> favor predicting from the previous frame instead of the current
</dd> </dl>

</dd> <dt><samp class="option">bits_per_coded_sample <var class="var">integer</var></samp></d
<dt><samp class="option">aspect <var class="var">rational number</var>
(encoding,video)</samp></dt> <dd> Set
sample aspect ratio. </dd> <dt><samp class="option">sar <var
class="var">rational number</var> (encoding,video)</samp></dt>
<dd> Set sample aspect ratio. Alias to <var class="var">aspect</var>.
 </dd> <dt><samp class="option">debug <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Print specific debug info. Possible values: <dl
class="table"> <dt>‘<samp class="samp">pict</samp>’</dt>
<dd> picture info </dd> <dt>‘<samp class="samp">rc</samp>’</dt>
<dd> rate control </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt>
<dt>‘<samp class="samp">mb_type</samp>’</dt> <dd> macroblock
(MB) type </dd> <dt>‘<samp class="samp">qp</samp>’</dt>
<dd> per-block quantization parameter (QP) </dd> <dt>‘<samp
class="samp">dct_coeff</samp>’</dt> <dt>‘<samp
class="samp">green_metadata</samp>’</dt> <dd> display
complexity metadata for the upcoming frame, GoP or for a given duration.
 </dd> <dt>‘<samp class="samp">skip</samp>’</dt>
<dt>‘<samp class="samp">startcode</samp>’</dt> <dt>‘<samp
class="samp">er</samp>’</dt> <dd> error recognition
</dd> <dt>‘<samp class="samp">mmco</samp>’</dt>
<dd> memory management control operations (H.264) </dd>
<dt>‘<samp class="samp">bugs</samp>’</dt> <dt>‘<samp
class="samp">buffers</samp>’</dt> <dd> picture buffer alloca-
tions </dd> <dt>‘<samp class="samp">thread_ops</samp>’</dt>
<dd> threading operations </dd> <dt>‘<samp class="samp">nomc</samp>’</dt
<dd> skip motion compensation </dd> </dl>

</dd> <dt><samp class="option">cmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
full pel me compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum

of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">subcmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
sub pel me compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">mbcmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
macroblock compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp

class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>ildctcmp <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
interlaced dct compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>dia_size <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
diamond type & size for motion estimation. <dl class=”table”>
<dt>‘<samp class=”samp”>(1024, INT_MAX)</samp>’</dt>
<dd> full motion estimation(slowest) </dd> <dt>‘<samp
class=”samp”>(768, 1024]</samp>’</dt> <dd> umh mo-
tion estimation </dd> <dt>‘<samp class=”samp”>(512,

768]</samp>’</dt> <dd> hex motion estimation </dd>
<dt>‘<samp class=”samp”>(256, 512]</samp>’</dt>
<dd> l2s diamond motion estimation </dd> <dt>‘<samp
class=”samp”>[2,256]</samp>’</dt> <dd> var diamond
motion estimation </dd> <dt>‘<samp class=”samp”>(-1,
2)</samp>’</dt> <dd> small diamond motion estimation
</dd> <dt>‘<samp class=”samp”>-1</samp>’</dt>
<dd> funny diamond motion estimation </dd> <dt>‘<samp
class=”samp”>(INT_MIN, -1)</samp>’</dt> <dd> sab dia-
mond motion estimation </dd> </dl>

</dd> <dt><samp class=”option”>last_pred <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
amount of motion predictors from the previous frame. </dd>
<dt><samp class=”option”>precmp <var class=”var”>integer</var> (<em
class=”emph”>encoding,video)</samp></dt> <dd> Set pre
motion estimation compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>pre_dia_size <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
diamond type & size for motion estimation pre-pass. </dd>
<dt><samp class=”option”>subq <var class=”var”>integer</var> (<em
class=”emph”>encoding,video)</samp></dt> <dd> Set sub pel
motion estimation quality. </dd> <dt><samp class=”option”>me_range
<var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set limit motion vectors range (1023 for DivX player). </dd>
<dt><samp class="option">global_quality <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">slice_flags <var class="var">integer</var></samp></dt>
<dt><samp class="option">mbd <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set mac-
roblock decision algorithm (high quality mode). Possible values:
<dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt>
<dd> use mbcmp (default) </dd> <dt>‘<samp class="samp">bits</samp>’</dt>
<dd> use fewest bits </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> use best rate distortion </dd> </dl>

</dd> <dt><samp class="option">rc_init_occupancy <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
number of bits which should be loaded into the rc buffer before de-
coding starts. </dd> <dt><samp class="option">flags2 <var
class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></d
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">fast</samp>’</dt> <dd> Allow non spec compli-
ant speedup tricks. </dd> <dt>‘<samp class="samp">noout</samp>’</dt>
<dd> Skip bitstream encoding. </dd> <dt>‘<samp
class="samp">ignorecrop</samp>’</dt> <dd> Ignore cropping
information from sps. </dd> <dt>‘<samp class="samp">local_header</samp>’</dt>
<dd> Place global headers at every keyframe instead of in extradata.
</dd> <dt>‘<samp class="samp">chunks</samp>’</dt>
<dd> Frame data might be split into multiple chunks. </dd>
<dt>‘<samp class="samp">showall</samp>’</dt> <dd> Show
all frames before the first keyframe. </dd> <dt>‘<samp
class="samp">export_mvs</samp>’</dt> <dd> Export motion
vectors into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">skip_manual</samp>’</dt>
<dd> Do not skip samples and export skip information as frame side data.
</dd> <dt>‘<samp class="samp">ass_ro_flush_noop</samp>’</dt>
<dd> Do not reset ASS ReadOrder field on flush. </dd>
<dt>‘<samp class="samp">icc_profiles</samp>’</dt>
<dd> Generate/parse embedded ICC profiles from/to colorimetry tags.
</dd> <dt>‘<samp class="samp">fixed_frame_size</samp>’</dt>
<dd> Force audio encoders to use a fixed frame size. </dd> </dl>

</dd> <dt><samp class="option">export_side_data <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">mvs</samp>’</dt> <dd> Export motion vectors
into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">prft</samp>’</dt>

<dd> Export encoder Producer Reference Time into packet side-data (see <code class="code">AV_PKT_DATA_PRFT</code>) for codecs that support it. </dd> <dt>‘<samp class="samp">venc_params</samp>’</dt> <dd> Export video encoding parameters through frame side data (see <code class="code">AV_FRAME_DATA_VIDEO_ENC_PARAMS</code>) for codecs that support it. At present, those are H.264 and VP9. </dd> <dt>‘<samp class="samp">film_grain</samp>’</dt> <dd> Export film grain parameters through frame side data (see <code class="code">AV_FRAME_DATA_FILM_GRAIN_PARAMS</code>). Supported at present by AV1 decoders. </dd> <dt>‘<samp class="samp">enhancements</samp>’</dt> <dd> Export picture enhancement metadata through frame side data, e.g. LCEVC (see <code class="code">AV_FRAME_DATA_LCEVC</code>). </dd> </dl>

</dd> <dt><samp class="option">threads <var class="var">integer</var> (decoding/encoding,video)</samp></dt> <dd> Set the number of threads to be used, in case the selected codec implementation supports multi-threading. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, 0</samp>’</dt> <dd> automatically select the number of threads to set </dd> </dl>

Default value is ‘<samp class="samp">auto</samp>’. </dd> <dt><samp class="option">dc <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set intra_dc_precision. </dd> <dt><samp class="option">nssew <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set nsse weight. </dd> <dt><samp class="option">skip_top <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the top which are skipped. </dd> <dt><samp class="option">skip_bottom <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the bottom which are skipped. </dd> <dt><samp class="option">profile <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set encoder codec profile. Default value is ‘<samp class="samp">unknown</samp>’. Encoder specific profiles are documented in the relevant encoder documentation. </dd> <dt><samp class="option">level <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set the encoder level. This level depends on the specific codec, and might correspond to the profile level. It is set by default to ‘<samp class="samp">unknown</samp>’. Possible values: <dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> </dl>

</dd> <dt><samp class="option">lowres <var class="var">integer</var> (decoding,audio,video)</samp></dt> <dd> Decode

at 1= 1/2, 2=1/4, 3=1/8 resolutions. </dd> <dt><samp
class="option">mblmin <var class="var">integer</var> (encoding,video)</sam
<dd> Set min macroblock lagrange factor (VBR). </dd>
<dt><samp class="option">mblmax <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
max macroblock lagrange factor (VBR). </dd> <dt><samp
class="option">skip_loop_filter <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dt><samp class="option">skip_idct
<var class="var">integer</var> (decoding,video)</samp></dt>
<dt><samp class="option">skip_pred <var class="var">integer</var>
(decoding,video)</samp></dt> <dt><samp
class="option">skip_frame <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dd> Make
decoder discard processing depending on the frame type selected by the
option value. <samp class="option">skip_loop_filter</samp>
skips frame loop filtering, <samp class="option">skip_idct</samp> skips
frame IDCT/dequantization, <samp class="option">skip_pred</samp>
skips prediction (intra prediction and motion compensation), <samp
class="option">skip_frame</samp> skips decoding. With both
<samp class="option">skip_pred</samp> and <samp class="option">skip_idct</samp>
set to ‘<samp class="samp">all</samp>’ the decoder may skip
all pixel operations and output frames with undefined pixels but valid metadata,
such as timestamps and frame types. Possible values: <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> Discard no frame. </dd> <dt>‘<samp class="samp">default</samp>’</dt>
<dd> Discard useless frames like 0-sized frames. </dd>
<dt>‘<samp class="samp">noref</samp>’</dt> <dd> Discard
all non-reference frames. </dd> <dt>‘<samp class="samp">bidir</samp>’</dt>
<dd> Discard all bidirectional frames. </dd> <dt>‘<samp
class="samp">nokey</samp>’</dt> <dd> Discard all frames ex-
cepts keyframes. </dd> <dt>‘<samp class="samp">nointra</samp>’</dt>
<dd> Discard all frames except I frames. </dd> <dt>‘<samp
class="samp">all</samp>’</dt> <dd> Discard all frames.
</dd> </dl>

Default value is ‘<samp class="samp">default</samp>’.
 </dd> <dt><samp class="option">bidir_refine <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Refine
the two motion vectors used in bidirectional macroblocks. </dd>
<dt><samp class="option">keyint_min <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
minimum interval between IDR-frames. </dd> <dt><samp
class="option">refs <var class="var">integer</var> (encoding,video)</samp>
<dd> Set reference frames to consider for motion compensation.
</dd> <dt><samp class="option">trellis <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dd> Set

rate-distortion optimal quantization. </dd> <dt><samp class="option">mv0_threshold
<var class="var">integer</var> (encoding,video)</samp></dt>
<dt><samp class="option">compression_level <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">bits_per_raw_sample <var class="var">integer</var></samp></dt>
<dt><samp class="option">channel_layout <var class="var">integer</var>
(decoding/encoding,audio)</samp></dt>
<dd> See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-
layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual
for the required syntax. </dd> <dt><samp class="option">rc_max_vbv_use
<var class="var">float</var> (encoding,video)</samp></dt>
<dt><samp class="option">rc_min_vbv_use <var class="var">float</var>
(encoding,video)</samp></dt> <dt><samp
class="option">color_primaries <var class="var">integer</var> (<em
class="emph">decoding/encoding,video)</samp></dt> <dd> Possible
values: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> Film </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt>
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">smpte431</samp>’</d
<dd> SMPTE 431-2 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt>
<dd> SMPTE 432-1 </dd> <dt>‘<samp class="samp">jedec-
p22</samp>’</dt> <dd> JEDEC P22 </dd> </dl>

</dd> <dt><samp class="option">color_trc <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dd> BT.709 </dd>
<dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dt>‘<samp class="samp">log100</samp>’</dt> <dd> Log
</dd> <dt>‘<samp class="samp">log_sqrt</samp>’</dt>
<dt>‘<samp class="samp">log316</samp>’</dt> <dd> Log
square root </dd> <dt>‘<samp class="samp">iec61966_2_4</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-4</samp>’</dt>
<dd> IEC 61966-2-4 </dd> <dt>‘<samp class="samp">bt1361</samp>’</dt>
<dt>‘<samp class="samp">bt1361e</samp>’</dt> <dd> BT.1361

</dd> <dt>‘<samp class="samp">iec61966_2_1</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-1</samp>’</dt>
<dd> IEC 61966-2-1 </dd> <dt>‘<samp class="samp">bt2020_10</samp>’</dt>
<dt>‘<samp class="samp">bt2020_10bit</samp>’</dt>
<dd> BT.2020 - 10 bit </dd> <dt>‘<samp class="samp">bt2020_12</samp>’</d
<dt>‘<samp class="samp">bt2020_12bit</samp>’</dt>
<dd> BT.2020 - 12 bit </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt
<dd> SMPTE ST 2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">arib-
std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>
</dl>

</dd> <dt><samp class="option">colorspace <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">rgb</samp>’</dt> <dd> RGB </dd>
<dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709
</dd> <dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">ycocg</samp>’</dt>
<dd> YCOCG </dd> <dt>‘<samp class="samp">bt2020nc</samp>’</dt>
<dt>‘<samp class="samp">bt2020_ncl</samp>’</dt>
<dd> BT.2020 NCL </dd> <dt>‘<samp class="samp">bt2020c</samp>’</dt>
<dt>‘<samp class="samp">bt2020_cl</samp>’</dt>
<dd> BT.2020 CL </dd> <dt>‘<samp class="samp">smpte2085</samp>’</dt>
<dd> SMPTE 2085 </dd> <dt>‘<samp class="samp">chroma-
derived-nc</samp>’</dt> <dd> Chroma-derived NCL </dd>
<dt>‘<samp class="samp">chroma-derived-c</samp>’</dt>
<dd> Chroma-derived CL </dd> <dt>‘<samp class="samp">ictcp</samp>’</dt>
<dd> ICtCp </dd> </dl>

</dd> <dt><samp class="option">color_range <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> If used as input parameter, it serves as a hint to the de-
coder, which color_range the input has. Possible values: <dl
class="table"> <dt>‘<samp class="samp">tv</samp>’</dt>
<dt>‘<samp class="samp">mpeg</samp>’</dt> <dt>‘<samp
class="samp">limited</samp>’</dt> <dd> MPEG (219*2^(n-
8)) </dd> <dt>‘<samp class="samp">pc</samp>’</dt>
<dt>‘<samp class="samp">jpeg</samp>’</dt> <dt>‘<samp
class="samp">full</samp>’</dt> <dd> JPEG (2^n-1) </dd>
</dl>

</dd> <dt><samp class="option">chroma_sample_location <var

class=”var”>integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>left</samp>’</dt> <dt>‘<samp class=”samp”>center</samp>’</dt>
<dt>‘<samp class=”samp”>topleft</samp>’</dt> <dt>‘<samp
class=”samp”>top</samp>’</dt> <dt>‘<samp class=”samp”>bottomleft</samp>’<
<dt>‘<samp class=”samp”>bottom</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>alpha_mode <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>premultiplied</samp>’</dt> <dt>‘<samp
class=”samp”>straight</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>log_level_offset <var class=”var”>integer</var></samp></dt>
<dd> Set the log level offset. </dd> <dt><samp class=”option”>slices
<var class=”var”>integer</var> (encoding,video)</samp></dt>
<dd> Number of slices, used in parallelized encoding. </dd>
<dt><samp class=”option”>thread_type <var class=”var”>flags</var>
(decoding/encoding,video)</samp></dt>
<dd> Select which multithreading methods to use. Use
of ‘<samp class=”samp”>frame</samp>’ will increase decod-
ing delay by one frame per thread, so clients which cannot provide future
frames should not use it. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>slice</samp>’</dt> <dd> Decode
more than one part of a single frame at once. Multithreading
using slices works only when the video was encoded with slices.
</dd> <dt>‘<samp class=”samp”>frame</samp>’</dt>
<dd> Decode more than one frame at once. </dd> </dl>

Default value is ‘<samp class=”samp”>slice+frame</samp>’.
 </dd> <dt><samp class=”option”>audio_service_type <var
class=”var”>integer</var> (encoding,audio)</samp></dt>
<dd> Set audio service type. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>ma</samp>’</dt>
<dd> Main Audio Service </dd> <dt>‘<samp class=”samp”>ef</samp>’</dt>
<dd> Effects </dd> <dt>‘<samp class=”samp”>vi</samp>’</dt>
<dd> Visually Impaired </dd> <dt>‘<samp class=”samp”>hi</samp>’</dt>
<dd> Hearing Impaired </dd> <dt>‘<samp class=”samp”>di</samp>’</dt>
<dd> Dialogue </dd> <dt>‘<samp class=”samp”>co</samp>’</dt>
<dd> Commentary </dd> <dt>‘<samp class=”samp”>em</samp>’</dt>
<dd> Emergency </dd> <dt>‘<samp class=”samp”>vo</samp>’</dt>
<dd> Voice Over </dd> <dt>‘<samp class=”samp”>ka</samp>’</dt>
<dd> Karaoke </dd> </dl>

</dd> <dt><samp class=”option”>request_sample_fmt <var class=”var”>sample_fmt</var>
(decoding,audio)</samp></dt> <dd> Set
sample format audio decoders should prefer. Default value is <code
class=”code”>none</code>. </dd> <dt><samp class=”option”>pkt_timebase

<var class=”var”>rational number</var></samp></dt> <dt><samp class=”option”>sub_charenc <var class=”var”>encoding</var> (decoding,subtitles)</samp></dt> <dd> Set the input subtitles character encoding. </dd> <dt><samp class=”option”>field_order <var class=”var”>field_order</var> (video)</samp></dt> <dd> Set/override the field order of the video. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>progressive</samp>’</dt> <dd> Progressive video </dd> <dt>‘<samp class=”samp”>tt</samp>’</dt> <dd> Interlaced video, top field coded and displayed first </dd> <dt>‘<samp class=”samp”>bb</samp>’</dt> <dd> Interlaced video, bottom field coded and displayed first </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dd> Interlaced video, top coded first, bottom displayed first </dd> <dt>‘<samp class=”samp”>bt</samp>’</dt> <dd> Interlaced video, bottom coded first, top displayed first </dd> </dl>

</dd> <dt><samp class=”option”>skip_alpha <var class=”var”>bool</var> (decoding,video)</samp></dt> <dd> Set to 1 to disable processing alpha (transparency). This works like the ‘<samp class=”samp”>gray</samp>’ flag in the <samp class=”option”>flags</samp> option which skips chroma information instead of alpha. Default is 0. </dd> <dt><samp class=”option”>codec_whitelist <var class=”var”>list</var> (input)</samp></dt> <dd> "," separated list of allowed decoders. By default all are allowed. </dd> <dt><samp class=”option”>dump_separator <var class=”var”>string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class=”example-preformatted”>ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class=”option”>max_pixels <var class=”var”>integer</var> (decoding/encoding,video)</samp></dt> <dd> Maximum number of pixels per image. This value can be used to avoid out of memory failures due to large images. </dd> <dt><samp class=”option”>apply_cropping <var class=”var”>bool</var> (decoding,video)</samp></dt> <dd> Enable cropping if cropping parameters are multiples of the required alignment for the left and top parameters. If the alignment is not met the cropping will be partially applied to maintain alignment. Default is 1 (enabled). Note: The required alignment depends on if <code class=”code”>AV_CODEC_FLAG_UNALIGNED</code> is set and the CPU. <code class=”code”>AV_CODEC_FLAG_UNALIGNED</code> cannot be changed from the command line. Also hardware decoders will not apply left/top Cropping. </dd> <dt><samp class=”option”>lcevc-dec-

params <var class="var">string</var> (decoding,video)</samp></dt>
<dd> A :-separated list of key=value parameters passed to liblcevc-dec to initialize it. </dd> </dl>

 <h2 class="chapter">10 Decoders# TOC</h2>

Decoders are configured elements in FFmpeg which allow the decoding of multimedia streams. When you configure your FFmpeg build, all the supported native decoders are enabled by default. Decoders requiring an external library must be enabled manually via the corresponding <code class="code">--enable-lib</code> option. You can list all available decoders using the configure option <code class="code">--list-decoders</code>. You can disable all the decoders with the configure option <code class="code">--disable-decoders</code> and selectively enable / disable single decoders with the options <code class="code">--enable-decoder=<var class="var">DECODER</var></code> / <code class="code">--disable-decoder=<var class="var">DECODER</var></code>. The option <code class="code">-decoders</code> of the ff* tools will display the list of enabled decoders.

 <h2 class="chapter">11 Video Decoders# TOC</h2>

A description of some of the currently available video decoders follows. <h3 class="section">11.1 av1# TOC</h3>

AOMedia Video 1 (AV1) decoder. <h4 class="subsection">11.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">operating_point</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Default is 0. </dd> </dl>

 <h3 class="section">11.2 hevc# TOC</h3> HEVC (AKA ITU-T H.265 or ISO/IEC 23008-2) decoder. The decoder supports MV-HEVC multiview streams with at most two views. Views to be output are selected by supplying a list of view IDs to the decoder (the <samp class="option">view_ids</samp> option). This op-

tion may be set either statically before decoder init, or from the <code class=”code”>get_format()</code> callback - useful for the case when the view count or IDs change dynamically during decoding. Only the base layer is decoded by default. Note that if you are using the <code class=”code”>ffmpeg</code> CLI tool, you should be using view specifiers as documented in its manual, rather than the options documented here. <h4 class=”subsection”>11.2.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>view_ids (MV-HEVC)</samp></dt> <dd> Specify a list of view IDs that should be output. This option can also be set to a single ’-1’, which will cause all views defined in the VPS to be decoded and output. </dd> <dt><samp class=”option”>view_ids_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view IDs available in the active VPS. The array is empty for single-layer video. The value of this option is guaranteed to be accurate when read from the <code class=”code”>get_format()</code> callback. It may also be set at other times (e.g. after opening the decoder), but the value is informational only and may be incorrect (e.g. when the stream contains multiple distinct VPS NALUs). </dd> <dt><samp class=”option”>view_pos_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view positions (left, right, or unspecified) available in the active VPS, as <code class=”code”>AVStereo3DView</code> values. When the array is available, its elements apply to the corresponding elements of <samp class=”option”>view_ids_available</samp>, i.e. <code class=”code”>view_pos_available[i]</code> contains the position of view with ID <code class=”code”>view_ids_available[i]</code>. Same validity restrictions as for <samp class=”option”>view_ids_available</samp> apply to this option. </dd> </dl>

 <h3 class=”section”>11.3 rawvideo# TOC</h3>

Raw video decoder. This decoder decodes rawvideo streams. <h4 class=”subsection”>11.3.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>top <var class=”var”>top_field_first</var></samp></dt
<dd> Specify the assumed field type of the input video. <dl class=”table”> <dt><samp class=”option”>-1</samp></dt> <dd> the video is assumed to be progressive (default) </dd> <dt><samp

class=”option”>0</samp></dt> <dd> bottom-field-first is assumed </dd> <dt><samp class=”option”>1</samp></dt> <dd> top-field-first is assumed </dd> </dl>

</dd> </dl>

 <h3 class=”section”>11.4 libastcenc# TOC</h3>

ARM ASTC Encoder wrapper for decoding ASTC textures. This decoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, installed as <code class=”code”>astcenc/astcenc.h</code> next to a library that <code class=”code”>-lastcenc</code> finds. Pass <code class=”code”>--enable-version3</code> together with <code class=”code”>--enable-libastcenc</code> to configure. The <samp class=”file”>.astc</samp> header does not record the intended decoding profile. Individual blocks identify their endpoint encodings, but this does not establish the sRGB versus linear interpretation. The decoder therefore uses the container metadata or <samp class=”option”>dec_profile</samp>: a KTX sRGB texture is decoded as <code class=”code”>ldr-srgb</code>, while a KTX texture in the linear format is sampled with HDR precision (<code class=”code”>hdr-ldr-a</code>, half-float output) because the linear GL internal format says nothing about LDR. A raw <code class=”code”>.astc</code> file carries no profile at all, so it falls back to <code class=”code”>ldr-srgb</code>; use <samp class=”option”>dec_profile</samp> for a raw HDR image: <pre class=”example-preformatted”>ffmpeg -dec_profile hdr-ldr-a -i input.astc output.png </pre>

 <h4 class=”subsection”>11.4.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>dec_profile <var class=”var”>int</var></samp></dt> <dd> Color profile to decode with, overriding the container metadata. It must match the profile the texture was encoded with: <code class=”code”>astcenc</code> substitutes a fixed magenta error color for blocks whose endpoints do not fit the selected profile. The values are <code class=”code”>ldr</code>, <code class=”code”>ldr-srgb</code>, <code class=”code”>hdr-ldr-a</code> and <code class=”code”>hdr</code>. Without this option the profile published by the container is used; a raw <samp class=”file”>.astc</samp> stream, which publishes none, falls back to <code class=”code”>ldr-srgb</code>. </dd> </dl>

 <h3 class=”section”>11.5 libdav1d<a class=”anchor hidden-xs” href=”#libdav1d” aria-

hidden=”true”># TOC</h3>

dav1d AV1 decoder. libdav1d allows libavcodec to decode the AOMedia Video 1 (AV1) codec. Requires the presence of the libdav1d headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libdav1d</code>. <h4 class=”subsection”>11.5.1 Options# TOC</h4>

The following options are supported by the libdav1d wrapper. <dl class=”table”> <dt><samp class=”option”>max_frame_delay</samp></dt> <dd> Set max amount of frames the decoder may buffer internally. The default value is 0 (autodetect). </dd> <dt><samp class=”option”>filmgrain</samp></dt> <dd> Apply film grain to the decoded video if present in the bitstream. Defaults to the internal default of the library. This option is deprecated and will be removed in the future. See the global option <code class=”code”>export_side_data</code> to export Film Grain parameters instead of applying it. </dd> <dt><samp class=”option”>oppoint</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Defaults to the internal default of the library. </dd> <dt><samp class=”option”>alllayers</samp></dt> <dd> Output all spatial layers of a scalable AV1 bitstream. The default value is false. </dd> </dl>

 <h3 class=”section”>11.6 libdavs2# TOC</h3>

AVS2-P2/IEEE1857.4 video decoder wrapper. This decoder allows libavcodec to decode AVS2 streams with davs2 library.

 <h3 class=”section”>11.7 libuavs3d# TOC</h3>

AVS3-P2/IEEE1857.10 video decoder. libuavs3d allows libavcodec to decode AVS3 streams. Requires the presence of the libuavs3d headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libuavs3d</code>. <h4 class=”subsection”>11.7.1 Options# TOC</h4>

The following option is supported by the libuavs3d wrapper. <dl class="table"> <dt><samp class="option">frame_threads</samp></dt> <dd> Set amount of frame threads to use during decoding. The default value is 0 (autodetect). </dd> </dl>

 <h3 class="section">11.8 libxevd# TOC</h3>

eXtra-fast Essential Video Decoder (XEVD) MPEG-5 EVC decoder wrapper. This decoder requires the presence of the libxevd headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxevd</samp>. The xevd project website is at <a class="url" href="https://github.com/mpeg5/xevd">https://github.com/mpeg5/xevd
  <h4 class="subsection">11.8.1 Options# TOC</h4>

The following options are supported by the libxevd wrapper. The xevd-equivalent options or values are listed in parentheses for easy migration. To get a more accurate and extensive documentation of the libxevd options, invoke the command <code class="code">xevd_app --help</code> or consult the libxevd documentation. <dl class="table"> <dt><samp class="option">threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class="section">11.9 QSV Decoders# TOC</h3>

The family of Intel QuickSync Video decoders (VC1, MPEG-2, H.264, HEVC, JPEG/MJPEG, VP8, VP9, AV1, VVC). <h4 class="subsection">11.9.1 Common Options# TOC</h4>

The following options are supported by all qsv decoders. <dl class="table"> <dt><samp class="option"><var class="var">async_depth</var></samp></dt> <dd> Internal parallelization depth, the higher the value the higher the latency. </dd> <dt><samp class="option"><var class="var">gpu_copy</var></samp></dt> <dd> A GPU-accelerated copy between video and system memory <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dt>‘<samp class="samp">on</samp>’</dt> <dt>‘<samp class="samp">off</samp>’</dt> </dl>

</dd> </dl>

 <h4 class="subsection">11.9.2 HEVC Options# TOC</h4> Extra options for hevc_qsv. <dl class="table"> <dt><samp class="option"><var class="var">load_plugin</var></samp></dt> <dd> A user plugin to load in an internal session <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">hevc_sw</samp>’<
<dt>‘<samp class="samp">hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugins</var></samp></dt>
<dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session </dd> </dl>

 <h3 class="section">11.10 v210#
TOC</h3>

Uncompressed 4:2:2 10-bit decoder.
<h4 class="subsection">11.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">custom_stride</samp></dt>
<dd> Set the line size of the v210 data in bytes. The default value is 0 (autodetect). You can use the special -1 value for a strideless v210 as seen in BOXX files. </dd> </dl>

 <h2 class="chapter">12 Audio Decoders# TOC</h2>

A description of some of the currently available audio decoders follows. <h3 class="section">12.1 ac3#
TOC</h3>

AC-3 audio decoder. This decoder implements part of ATSC A/52:2010 and ETSI TS 102 366, as well as the undocumented RealAudio 3 (a.k.a. dnet).
<h4 class="subsection">12.1.1 AC-3 Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-drc_scale <var class="var">value</var></samp></dt>
<dd> Dynamic Range Scale Factor. The factor to apply to dynamic range

values from the AC-3 stream. This factor is applied exponentially. The default value is 1. There are 3 notable scale factor ranges: <dl class="table"> <dt><samp class="option">drc_scale == 0</samp></dt> <dd> DRC disabled. Produces full range audio. </dd> <dt><samp class="option">0 < drc_scale <= 1</samp></dt> <dd> DRC enabled. Applies a fraction of the stream DRC value. Audio reproduction is between full range and full compression. </dd> <dt><samp class="option">drc_scale > 1</samp></dt> <dd> DRC enabled. Applies drc_scale asymmetrically. Loud sounds are fully compressed. Soft sounds are enhanced. </dd> </dl>

</dd> </dl>

 <h3 class="section">12.2 dca# TOC</h3>

DCA (DTS Coherent Acoustics) audio decoder, including the DTS-HD extensions. <h4 class="subsection">12.2.1 DCA Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-downmix <var class="var">channel_layout</var></samp
<dd> Request a smaller channel layout from the decoder: ‘<samp class="samp">stereo</samp>’, ‘<samp class="samp">5.0(side)</samp>’, ‘<samp class="samp">5.1(side)</samp>’ or ‘<samp class="samp">7.1</samp>’. A DTS:X stream in the standard presentation decodes to 7.1.4 by default. Its four height channels are carried in an extension at the end of each lossless (XLL) frame, and the encoder also folds them into the 7.1 bed, so the decoder removes them from the bed when it outputs them. Requesting ‘<samp class="samp">7.1</samp>’ outputs the bed as coded instead, with the heights folded in, and so does every smaller layout. A ‘<samp class="samp">7.1</samp>’ request has no effect on other streams. </dd> </dl>

 <h3 class="section">12.3 flac# TOC</h3>

FLAC audio decoder. This decoder aims to implement the complete FLAC specification from Xiph. <h4 class="subsection">12.3.1 FLAC Decoder options# TOC</h4>

<dl class="table"> <dt><samp class="option">-use_buggy_lpc</samp></dt> <dd> The lavc FLAC encoder used to produce buggy streams with high lpc values (like the default value). This option makes it possible to decode such streams correctly by using lavc’s old buggy lpc logic for decoding. </dd> </dl>

 <h3 class="section">12.4 ffwavesynth# TOC</h3>

Internal wave synthesizer. This decoder generates wave patterns according to predefined sequences. Its use is purely internal and the format of the data it accepts is not publicly documented. <h3 class="section">12.5 libgsm# TOC</h3>

libgsm decoder wrapper. libgsm allows libavcodec to decode the GSM full rate audio codec. Requires the presence of the libgsm headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libgsm</code>. This decoder supports both the ordinary GSM and the Microsoft variant. <h3 class="section">12.6 libilbc# TOC</h3>

libilbc decoder wrapper. libilbc allows libavcodec to decode the Internet Low Bitrate Codec (iLBC) audio codec. Requires the presence of the libilbc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libilbc</code>. <h4 class="subsection">12.6.1 Options# TOC</h4>

The following option is supported by the libilbc wrapper. <dl class="table"> <dt><samp class="option">enhance</samp></dt> <dd> Enable the enhancement of the decoded audio when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">12.7 libmpeghdec# TOC</h3>

libmpeghdec decoder wrapper. libmpeghdec allows libm-

peghdec to decode the MPEG-H 3D audio codec. Requires the presence of the libmpeghdec headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libmpeghdec --enable-nonfree</code>. <h3 class="section">12.8 libopencore-amrnb# TOC</h3>

libopencore-amrnb decoder wrapper. libopencore-amrnb allows libavcodec to decode the Adaptive Multi-Rate Narrowband audio codec. Using it requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopencore-amrnb</code>. An FFmpeg native decoder for AMR-NB exists, so users can decode AMR-NB without this library. <h3 class="section">12.9 libopencore-amrwb# TOC</h

libopencore-amrwb decoder wrapper. libopencore-amrwb allows libavcodec to decode the Adaptive Multi-Rate Wideband audio codec. Using it requires the presence of the libopencore-amrwb headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopencore-amrwb</code>. An FFmpeg native decoder for AMR-WB exists, so users can decode AMR-WB without this library. <h3 class="section">12.10 libopus# TOC</h3>

libopus decoder wrapper. libopus allows libavcodec to decode the Opus Interactive Audio Codec. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopus</code>. An FFmpeg native decoder for Opus exists, so users can decode Opus without this library. <h3 class="section">12.11 pcm_dvda# TOC</h3>

PCM DVD-Audio decoder. Decodes the LPCM variant stored in DVD-Audio AOB files. Unlike DVD-Video LPCM it supports sample rates up to 192 kHz and allows splitting the channels into two groups with different quantization.

 <h2 class="chapter">13 Subtitles Decoders<a class="anchor hidden-xs"

href=”#Subtitles-Decoders” aria-hidden=”true”># TOC</h2>

 <h3 class=”section”>13.1 libaribb24# TOC</h3>

ARIB STD-B24 caption decoder. Implements profiles A and C of the ARIB STD-B24 standard. <h4 class=”subsection”>13.1.1 libaribb24 Decoder Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-aribb24-base-path <var class=”var”>path</var></samp></dt> <dd> Sets the base path for the libaribb24 library. This is utilized for reading of configuration files (for custom unicode conversions), and for dumping of non-text symbols as images under that location. Unset by default. </dd> <dt><samp class=”option”>-aribb24-skip-ruby-text <var class=”var”>boolean</var></samp></dt> <dd> Tells the decoder wrapper to skip text blocks that contain half-height ruby text. Enabled by default. </dd> </dl>

 <h3 class=”section”>13.2 libaribcaption# TOC</h3>

Yet another ARIB STD-B24 caption decoder using external libaribcaption library. Implements profiles A and C of the Japanese ARIB STD-B24 standard, Brazilian ABNT NBR 15606-1, and Philippines version of ISDB-T. Requires the presence of the libaribcaption headers and library (https://github.com/xqq/libaribcaption) during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libaribcaption</code>. If both libaribb24 and libaribcaption are enabled, libaribcaption decoder precedes. <h4 class=”subsection”>13.2.1 libaribcaption Decoder Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-sub_type <var class=”var”>subtitle_type</var></samp>
<dd> Specifies the format of the decoded subtitles. <dl class=”table”> <dt>‘<samp class=”samp”>bitmap</samp>’</dt>

<dd> Graphical image. </dd> <dt>‘<samp class="samp">ass</samp>’</dt>
<dd> ASS formatted text. </dd> <dt>‘<samp class="samp">text</samp>’</dt>
<dd> Simple text based output without formatting. </dd> </dl>

The default is ass as same as libaribb24 decoder. Some present players (e.g., mpv) expect ASS format for ARIB caption. </dd> <dt><samp class="option">-caption_encoding <var class="var">encoding_scheme</var></samp></dt>
<dd> Specifies the encoding scheme of input subtitle text. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Automatically detect text encoding (default). </dd>
<dt>‘<samp class="samp">jis</samp>’</dt> <dd> 8bit-char JIS encoding defined in ARIB STD B24. This encoding used in Japan for ISDB captions. </dd> <dt>‘<samp class="samp">utf8</samp>’</dt>
<dd> UTF-8 encoding defined in ARIB STD B24. This encoding is used in Philippines for ISDB-T captions. </dd> <dt>‘<samp class="samp">latin</samp>’</dt> <dd> Latin character encoding defined in ABNT NBR 15606-1. This encoding is used in South America for SBTVD / ISDB-Tb captions. </dd> </dl>

</dd> <dt><samp class="option">-font <var class="var">font_name[,font_name2,...]</var></samp></
<dd> Specify comma-separated list of font family names to be used for bitmap or ass type subtitle rendering. Only first font name is used for ass type subtitle. If not specified, use internally defined default font family. </dd> <dt><samp class="option">-ass_single_rect <var class="var">boolean</var></samp></dt> <dd> ARIB STD-B24 specifies that some captions may be displayed at different positions at a time (multi-rectangle subtitle). Since some players (e.g., old mpv) can’t handle multiple ASS rectangles in a single AVSubtitle, or multiple ASS rectangles of indeterminate duration with the same start timestamp, this option can change the behavior so that all the texts are displayed in a single ASS rectangle. The default is <var class="var">false</var>. If your player cannot handle AVSubtitles with multiple ASS rectangles properly, set this option to <var class="var">true</var> or define <code class="env">ASS_SINGLE_RECT=1</code> to change default behavior at compilation. </dd> <dt><samp class="option">-force_outline_text <var class="var">boolean</var></samp></dt>
<dd> Specify whether always render outline text for all characters regardless of the indication by character style. The default is <var class="var">false</var>. </dd> <dt><samp class="option">-outline_width <var class="var">number</var> (0.0 - 3.0)</samp></dt> <dd> Specify width for outline text, in dots (relative). The default is <var class="var">1.5</var>. </dd> <dt><samp class="option">-ignore_background <var class="var">boolean</var></samp></dt> <dd> Specify whether to ignore background color rendering. The default is <var

class=”var”>false</var>. </dd> <dt><samp class=”option”>-ignore_ruby <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore rendering for ruby-like (furigana) characters. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-replace_drcs <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to render replaced DRCS characters as Unicode characters. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_ascii <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) fullwidth alphanumerics with halfwidth alphanumerics. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_japanese <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace some MSZ (Middle Size; half width) fullwidth japanese special characters with halfwidth ones. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_glyph <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) characters with halfwidth glyphs if the fonts supports it. This option works under FreeType or DirectWrite renderer with Adobe-Japan1 compliant fonts. e.g., IBM Plex Sans JP, Morisawa BIZ UDGothic, Morisawa BIZ UDMincho, Yu Gothic, Yu Mincho, and Meiryo. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-canvas_size <var class=”var”>image_size</var></samp></dt> <dd> Specify the resolution of the canvas to render subtitles to; usually, this should be frame size of input video. This only applies when <code class=”code”>-subtitle_type</code> is set to <var class=”var”>bitmap</var>. The libaribcaption decoder assumes input frame size for bitmap rendering as below: <ol class=”enumerate”> PROFILE_A : 1440 x 1080 with SAR (PAR) 4:3 PROFILE_C : 320 x 180 with SAR (PAR) 1:1

If actual frame size of input video does not match above assumption, the rendered captions may be distorted. To make the captions undistorted, add <code class=”code”>-canvas_size</code> option to specify actual input video size. Note that the <code class=”code”>-canvas_size</code> option is not required for video with different size but same aspect ratio. In such cases, the caption will be stretched or shrunk to actual video size if <code class=”code”>-canvas_size</code> option is not specified. If <code class=”code”>-canvas_size</code> option is specified with different size, the caption will be stretched or shrunk as specified size with calculated SAR. </dd> </dl>

 <h4 class=”subsection”>13.2.2 libaribcaption decoder usage examples# <a class=”anchor hidden-xs”href=”#toc-libaribcaption-decoder-usage-

examples” aria-hidden=”true”>TOC</h4>

Display MPEG-TS file with ARIB subtitle by <code class=”code”>ffplay</code> tool: <pre class=”example-preformatted”>ffplay -sub_type bitmap MPEG.TS </pre>

Display MPEG-TS file with input frame size 1920x1080 by <code class=”code”>ffplay</code> tool: <pre class=”example-preformatted”>ffplay -sub_type bitmap -canvas_size 1920x1080 MPEG.TS </pre>

Embed ARIB subtitle in transcoded video: <pre class=”example-preformatted”>ffmpeg -sub_type bitmap -i src.m2t -filter_complex "[0:v][0:s]overlay" -vcodec h264 dest.mp4 </pre>

 <h3 class=”section”>13.3 dvbsub# TOC</h3>

 <h4 class=”subsection”>13.3.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>compute_clut</samp></dt> <dd><dl class=”table”> <dt><samp class=”option”>-2</samp></dt> <dd> Compute clut once if no matching CLUT is in the stream. </dd> <dt><samp class=”option”>-1</samp></dt> <dd> Compute clut if no matching CLUT is in the stream. </dd> <dt><samp class=”option”>0</samp></dt> <dd> Never compute CLUT </dd> <dt><samp class=”option”>1</samp></dt> <dd> Always compute CLUT and override the one provided in the stream. </dd> </dl> </dd> <dt><samp class=”option”>dvb_substream</samp></dt> <dd> Selects the dvb substream, or all substreams if -1 which is default. </dd> </dl>

 <h3 class=”section”>13.4 dvdsub# TOC</h3>

This codec decodes the bitmap subtitles used in DVDs; the same subtitles can also be found in VobSub file pairs and in some Matroska files. <h4 class=”subsection”>13.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. When stored in VobSub, the palette is normally specified in the index file; in Matroska, the palette is stored in the codec extra-data in the same format as in VobSub. In DVDs, the palette is stored in the IFO file, and therefore not available when reading from dumped VOB files. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">ifo_palette</samp></dt> <dd> Specify the IFO file from which the global palette is obtained. (experimental) </dd> <dt><samp class="option">forced_subs_only</samp></dt> <dd> Only decode subtitle entries marked as forced. Some titles have forced and non-forced subtitles in the same track. Setting this flag to <code class="code">1</code> will only keep the forced subtitles. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">13.5 libzvbi-teletext# TOC</h3>

Libzvbi allows libavcodec to decode DVB teletext pages and DVB teletext subtitles. Requires the presence of the libzvbi headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libzvbi</code>. <h4 class="subsection">13.5.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">txt_page</samp></dt> <dd> List of teletext page numbers to decode. Pages that do not match the specified list are dropped. You may use the special <code class="code">*</code> string to match all pages, or <code class="code">subtitle</code> to match all subtitle pages. Default value is *. </dd> <dt><samp class="option">txt_default_region</samp></dt> <dd> Set default character set used for decoding, a value between 0 and 87 (see ETS 300 706, Section 15, Table 32). Default value is -1, which does not override the libzvbi default. This option is needed for some legacy level 1.0 transmissions which cannot signal the proper charset. </dd> <dt><samp class="option">txt_chop_top</samp></dt> <dd> Discards the top teletext line. Default value is 1. </dd> <dt><samp class="option">txt_format</samp></dt> <dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt><samp class="option">bitmap</samp></dt> <dd> The default format, you should use this for teletext pages, because certain graphics and colors

cannot be expressed in simple text or even ASS. </dd> <dt><samp class="option">text</samp></dt> <dd> Simple text based output without formatting. </dd> <dt><samp class="option">ass</samp></dt> <dd> Formatted ASS output, subtitle pages and teletext pages are returned in different styles, subtitle pages are stripped down to text, but an effort is made to keep the text alignment and the formatting. </dd> </dl> </dd> <dt><samp class="option">txt_left</samp></dt> <dd> X offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_top</samp></dt> <dd> Y offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_chop_spaces</samp></dt> <dd> Chops leading and trailing spaces and removes empty lines from the generated text. This option is useful for teletext based subtitles where empty spaces may be present at the start or at the end of the lines or empty lines may be present between the subtitle lines because of double-sized teletext characters. Default value is 1. </dd> <dt><samp class="option">txt_duration</samp></dt> <dd> Sets the display duration of the decoded teletext pages or subtitles in milliseconds. Default value is -1 which means infinity or until the next subtitle event comes. </dd> <dt><samp class="option">txt_transparent</samp></dt> <dd> Force transparent background of the generated teletext bitmaps. Default value is 0 which means an opaque background. </dd> <dt><samp class="option">txt_opacity</samp></dt> <dd> Sets the opacity (0-255) of the teletext background. If <samp class="option">txt_transparent</samp> is not set, it only affects characters between a start box and an end box, typically subtitles. Default value is 0 if <samp class="option">txt_transparent</samp> is set, 255 otherwise. </dd> </dl>

 <h2 class="chapter">14 Encoders# TOC</h2>

Encoders are configured elements in FFmpeg which allow the encoding of multimedia streams. When you configure your FFmpeg build, all the supported native encoders are enabled by default. Encoders requiring an external library must be enabled manually via the corresponding <code class="code">--enable-lib</code> option. You can list all available encoders using the configure option <code class="code">--list-encoders</code>. You can disable all the encoders with the configure option <code class="code">--disable-encoders</code> and selectively enable / disable single encoders with the options <code class="code">--enable-encoder=<var class="var">ENCODER</var></code> / <code class="code">--disable-encoder=<var class="var">ENCODER</var></code>. The option <code class="code">-encoders</code> of the ff* tools will display the list of enabled encoders.

 <h2 class="chapter">15 Audio Encoders# TOC</h2>

A description of some of the currently available audio encoders follows. <h3 class="section">15.1 aac# TOC</h3>

Advanced Audio Coding (AAC) encoder. This encoder is the default AAC encoder, natively implemented into FFmpeg. <h4 class="subsection">15.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bit rate in bits/s. Setting this automatically activates constant bit rate (CBR) mode. If this option is unspecified it is set to 128kbps. </dd> <dt><samp class="option">q</samp></dt> <dd> Set quality for variable bit rate (VBR) mode. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. With the default ‘<samp class="samp">nmr</samp>’ coder, VBR holds a constant noise-to-mask target and lets the bitrate follow the content. Higher values are finer: <code class="code">1</code> lands near 128 kbps for typical stereo music, and each doubling of the value raises the rate by roughly a third; values above about 6 make no further difference. A quality setting takes precedence over <samp class="option">b</samp> and <samp class="option">aac_rc</samp>. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency. If unspecified will allow the encoder to dynamically adjust the cutoff to improve clarity on low bitrates. </dd> <dt><samp class="option">aac_coder</samp></dt> <dd> Set AAC encoder coding method. Possible values: <dl class="table"> <dt>‘<samp class="samp">twoloop</samp>’</dt> <dd> Two loop searching (TLS) method. This is the default method. This method first sets quantizers depending on band thresholds and then tries to find an optimal combination by adding or subtracting a specific value from all quantizers and adjusting some individual quantizer a little. Will tune itself based on whether <samp class="option">aac_is</samp>, <samp class="option">aac_ms</samp> and <samp class="option">aac_pns</samp> are enabled. </dd> <dt>‘<samp class="samp">anmr</samp>’</dt> <dd> Average noise to mask ratio (ANMR) trellis-based solution. This is an experimental coder which currently produces a lower quality, is more unstable

and is slower than the default twoloop coder but has potential. Currently has no support for the <samp class=”option”>aac_is</samp> or <samp class=”option”>aac_pns</samp> options. Not currently recommended. </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dd> Constant quantizer method. Uses a cheaper version of twoloop algorithm that doesn’t try to do as many clever adjustments. Worse with low bitrates (less than 64kbps), but is better and much faster at higher bitrates. </dd> </dl>

</dd> <dt><samp class=”option”>aac_rc</samp></dt> <dd> Rate-control mode of the ‘<samp class=”samp”>nmr</samp>’ coder when a bitrate is set with <samp class=”option”>b</samp> (other coders ignore it; <samp class=”option”>q</samp> takes precedence). Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>cbr</samp>’</dt> <dd> Constant bitrate within the decoder’s bit reservoir. This is the default. </dd> <dt>‘<samp class=”samp”>abr</samp>’</dt> <dd> Average bitrate: a constant-quality target, as in VBR, whose set-point a slow servo moves to meet <samp class=”option”>b</samp> over the length of the stream. Local quality stays constant, so short files and individual passages can deviate from the requested rate; the average converges over minutes rather than seconds. Below roughly 56 kbps stereo the rate lands above the request; use ‘<samp class=”samp”>cbr</samp>’ there. </dd> </dl>

</dd> <dt><samp class=”option”>aac_ms</samp></dt> <dd> Sets mid/side coding mode. The default value of "auto" will automatically use M/S with bands which will benefit from such coding. Can be forced for all bands using the value "enable", which is mainly useful for debugging or disabled using "disable". </dd> <dt><samp class=”option”>aac_is</samp></dt> <dd> Sets intensity stereo coding tool usage. By default, it’s enabled and will automatically toggle IS for similar pairs of stereo bands if it’s beneficial. Can be disabled for debugging by setting the value to "disable". </dd> <dt><samp class=”option”>aac_pns</samp></dt> <dd> Uses perceptual noise substitution to replace low entropy high frequency bands with imperceptible white noise during the decoding process. By default, it’s enabled, but can be disabled for debugging purposes by using "disable". </dd> <dt><samp class=”option”>aac_tns</samp></dt> <dd> Enables the use of a multitap FIR filter which spans through the high frequency bands to hide quantization noise during the encoding process and is reverted by the decoder. As well as decreasing unpleasant artifacts in the high range this also reduces the entropy in the high bands and allows for more bits to be used by the mid-low bands. By default it’s enabled but can be disabled for debugging by setting the option to "disable". </dd> <dt><samp class=”option”>aac_ltp</samp></dt> <dd> Enables the use of the long term prediction extension which increases coding efficiency

in very low bandwidth situations such as encoding of voice or solo piano music by extending constant harmonic peaks in bands throughout frames. This option is implied by profile:a aac_low. Use in conjunction with <samp class="option">-ar</samp> to decrease the samplerate. </dd> <dt><samp class="option">profile</samp></dt> <dd> Sets the encoding profile, possible values: <dl class="table"> <dt>‘<samp class="samp">aac_low</samp>’</dt> <dd> The default, AAC "Low-complexity" profile. Is the most compatible and produces decent quality. </dd> <dt>‘<samp class="samp">mpeg2_aac_low</samp>’</dt> <dd> Equivalent to <code class="code">-profile:a aac_low -aac_pns 0</code>. PNS was introduced with the MPEG4 specifications. </dd> <dt>‘<samp class="samp">aac_ltp</samp>’</dt> <dd> Long term prediction profile, is enabled by and will enable the <samp class="option">aac_ltp</samp> option. Introduced in MPEG4. </dd> </dl> If this option is unspecified it is set to ‘<samp class="samp">aac_low</samp>’. </dd> </dl>

 <h3 class="section">15.2 ac3 and ac3_fixed# TOC</h3>

AC-3 audio encoders. These encoders implement part of ATSC A/52:2010 and ETSI TS 102 366. The <var class="var">ac3</var> encoder uses floating-point math, while the <var class="var">ac3_fixed</var> encoder only uses fixed-point integer math. This does not mean that one is always faster, just that one or the other may be better suited to a particular system. The <var class="var">ac3_fixed</var> encoder is not the default codec for any of the output formats, so it must be specified explicitly using the option <code class="code">-acodec ac3_fixed</code> in order to use it. <h4 class="subsection">15.2.1 AC-3 Metadata# TOC</h4>

The AC-3 metadata options are used to set parameters that describe the audio, but in most cases do not affect the audio encoding itself. Some of the options do directly affect or influence the decoding and playback of the resulting bitstream, while others are just for informational purposes. A few of the options will add bits to the output stream that could otherwise be used for audio data, and will thus affect the quality of the output. Those will be indicated accordingly with a note in the option list below. These parameters are described in detail in several publicly-available documents. <ul class="itemize mark-bullet"> A/52:2010 - Digital Audio Compression (AC-3) (E-AC-3) Standard A/54

- Guide to the Use of the ATSC Digital Television Standard <a class="uref" href="http://www.dolby.com/uploadedFiles/zz-_Shared_Assets/English_PDFs/Professional/18 Metadata Guide Dolby Digital Professional Encoding Guidelines

 <h4 class="subsubsection">15.2.1.1 Metadata Control Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-per_frame_metadata <var class="var">boolean</var></samp></dt> <dd> Allow Per-Frame Metadata. Specifies if the encoder should check for changing metadata for each frame. <dl class="table"> <dt><samp class="option">0</samp></dt> <dd> The metadata values set at initialization will be used for every frame in the stream. (default) </dd> <dt><samp class="option">1</samp></dt> <dd> Metadata values can be changed before encoding each frame. </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">15.2.1.2 Downmix Levels# TOC</h4>

<dl class="table"> <dt><samp class="option">-center_mixlev <var class="var">level</var></samp></dt> <dd> Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo. This field will only be written to the bitstream if a center channel is present. The value is specified as a scale factor. There are 3 valid values: <dl class="table"> <dt><samp class="option">0.707</samp></dt> <dd> Apply -3dB gain </dd> <dt><samp class="option">0.595</samp></dt> <dd> Apply -4.5dB gain (default) </dd> <dt><samp class="option">0.500</samp></dt> <dd> Apply -6dB gain </dd> </dl>

</dd> <dt><samp class="option">-surround_mixlev <var class="var">level</var></samp></dt> <dd> Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo. This field will only be written to the bitstream if one or more surround channels are present. The value is specified as a scale factor. There are 3 valid values: <dl class="table"> <dt><samp class="option">0.707</samp></dt> <dd> Apply -3dB gain </dd> <dt><samp class="option">0.500</samp></dt> <dd> Apply -6dB gain (default) </dd> <dt><samp class="option">0.000</samp></dt> <dd> Silence Surround Channel(s) </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">15.2.1.3 Audio Production Information# TOC</h4> Audio Production Information is optional information describing the mixing environment. Either none or both of the fields are written to the bitstream. <dl class="table"> <dt><samp class="option">-mixing_level <var class="var">number</var></samp></dt> <dd> Mixing Level. Specifies peak sound pressure level (SPL) in the production environment when the mix was mastered. Valid values are 80 to 111, or -1 for unknown or not indicated. The default value is -1, but that value cannot be used if the Audio Production Information is written to the bitstream. Therefore, if the <code class="code">room_type</code> option is not the default value, the <code class="code">mixing_level</code> option must not be -1. </dd> <dt><samp class="option">-room_type <var class="var">type</var></samp></dt> <dd> Room Type. Describes the equalization used during the final mixing session at the studio or on the dubbing stage. A large room is a dubbing stage with the industry standard X-curve equalization; a small room has flat equalization. This field will not be written to the bitstream if both the <code class="code">mixing_level</code> option and the <code class="code">room_type</code> option have the default values. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">large</samp></dt> <dd> Large Room </dd> <dt><samp class="option">2</samp></dt> <dt><samp class="option">small</samp></dt> <dd> Small Room </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">15.2.1.4 Other Metadata Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-copyright <var class="var">boolean</var></samp></dt
<dd> Copyright Indicator. Specifies whether a copyright exists for this audio. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">off</samp></dt> <dd> No Copyright Exists (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Copyright Exists </dd> </dl>

</dd> <dt><samp class="option">-dialnorm <var class="var">value</var></samp></dt> <dd> Dialogue Normalization. Indicates how far the average dialogue level of the program is below digital 100% full scale (0 dBFS). This parameter determines a level shift during audio reproduction that sets the average volume of the dialogue to a preset level. The goal is to match volume level between program sources. A value of -31dB will result in no volume level change, relative to the source volume, during audio reproduction. Valid values are whole numbers in the range -31 to -1, with -31 being the default. </dd> <dt><samp class="option">-dsur_mode <var class="var">mode</var></samp></dt> <dd> Dolby Surround Mode. Specifies whether the stereo signal uses Dolby Surround (Pro Logic). This field will only be written to the bitstream if the audio stream is stereo. Using this option does NOT mean the encoder will actually apply Dolby Surround processing. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Not Dolby Surround Encoded </dd> <dt><samp class="option">2</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Dolby Surround Encoded </dd> </dl>

</dd> <dt><samp class="option">-original <var class="var">boolean</var></samp></dt> <dd> Original Bit Stream Indicator. Specifies whether this audio is from the original source and not a copy. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Not Original Source </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Original Source (default) </dd> </dl>

</dd> </dl>

 <h4 class="subsection">15.2.2 Extended Bitstream Information# TOC</h4> The extended bitstream options are part of the Alternate Bit Stream Syntax as specified in Annex D of the A/52:2010 standard. It is grouped into 2 parts. If any one parameter in a group is specified, all values in that group will be written to the bitstream. Default values are used for those that are written but have not been specified. If the mixing levels are written, the decoder will use these values instead of the ones specified in the <code class="code">center_mixlev</code> and <code class="code">surround_mixlev</code> options if it supports the Alternate Bit Stream Syntax. <h4 class="subsubsection">15.2.2.1 Extended Bitstream Information - Part 1<a class="anchor hidden-xs" href="#Extended-Bitstream-Information-_002d-Part-1" aria-

hidden=”true”># TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-dmix_mode <var class=”var”>mode</var></samp></dt> <dd> Preferred Stereo Downmix Mode. Allows the user to select either Lt/Rt (Dolby Surround) or Lo/Ro (normal stereo) as the preferred stereo downmix mode. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dt><samp class=”option”>notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class=”option”>1</samp></dt> <dt><samp class=”option”>ltrt</samp></dt> <dd> Lt/Rt Downmix Preferred </dd> <dt><samp class=”option”>2</samp></dt> <dt><samp class=”option”>loro</samp></dt> <dd> Lo/Ro Downmix Preferred </dd> </dl>

</dd> <dt><samp class=”option”>-ltrt_cmixlev <var class=”var”>level</var></samp></dt> <dd> Lt/Rt Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo in Lt/Rt mode. <dl class=”table”> <dt><samp class=”option”>1.414</samp></dt> <dd> Apply +3dB gain </dd> <dt><samp class=”option”>1.189</samp></dt> <dd> Apply +1.5dB gain </dd> <dt><samp class=”option”>1.000</samp></dt> <dd> Apply 0dB gain </dd> <dt><samp class=”option”>0.841</samp></dt> <dd> Apply -1.5dB gain </dd> <dt><samp class=”option”>0.707</samp></dt> <dd> Apply -3.0dB gain </dd> <dt><samp class=”option”>0.595</samp></dt> <dd> Apply -4.5dB gain (default) </dd> <dt><samp class=”option”>0.500</samp></dt> <dd> Apply -6.0dB gain </dd> <dt><samp class=”option”>0.000</samp></dt> <dd> Silence Center Channel </dd> </dl>

</dd> <dt><samp class=”option”>-ltrt_surmixlev <var class=”var”>level</var></samp></dt> <dd> Lt/Rt Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo in Lt/Rt mode. <dl class=”table”> <dt><samp class=”option”>0.841</samp></dt> <dd> Apply -1.5dB gain </dd> <dt><samp class=”option”>0.707</samp></dt> <dd> Apply -3.0dB gain </dd> <dt><samp class=”option”>0.595</samp></dt> <dd> Apply -4.5dB gain </dd> <dt><samp class=”option”>0.500</samp></dt> <dd> Apply -6.0dB gain (default) </dd> <dt><samp class=”option”>0.000</samp></dt> <dd> Silence Surround Channel(s) </dd> </dl>

</dd> <dt><samp class=”option”>-loro_cmixlev <var class=”var”>level</var></samp></dt> <dd> Lo/Ro Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo in Lo/Ro mode. <dl class=”table”> <dt><samp class=”option”>1.414</samp></dt> <dd> Apply +3dB gain </dd> <dt><samp class=”option”>1.189</samp></dt> <dd> Apply +1.5dB gain </dd> <dt><samp class=”option”>1.000</samp></dt> <dd> Apply 0dB gain </dd> <dt><samp class=”option”>0.841</samp></dt> <dd> Apply -1.5dB gain </dd> <dt><samp class=”option”>0.707</samp></dt> <dd> Apply -3.0dB gain </dd> <dt><samp class=”option”>0.595</samp></dt> <dd> Apply -4.5dB gain (default) </dd> <dt><samp class=”option”>0.500</samp></dt>

<dd> Apply -6.0dB gain </dd> <dt><samp class=”option”>0.000</samp></dt>
<dd> Silence Center Channel </dd> </dl>

</dd> <dt><samp class=”option”>-loro_surmixlev <var class=”var”>level</var></samp></dt>
<dd> Lo/Ro Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo in Lo/Ro mode. <dl class=”table”> <dt><samp class=”option”>0.841</samp></dt>
<dd> Apply -1.5dB gain </dd> <dt><samp class=”option”>0.707</samp></dt>
<dd> Apply -3.0dB gain </dd> <dt><samp class=”option”>0.595</samp></dt>
<dd> Apply -4.5dB gain </dd> <dt><samp class=”option”>0.500</samp></dt>
<dd> Apply -6.0dB gain (default) </dd> <dt><samp class=”option”>0.000</samp></dt>
<dd> Silence Surround Channel(s) </dd> </dl>

</dd> </dl>

 <h4 class=”subsubsection”>15.2.2.2 Extended Bitstream Information - Part 2# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-dsurex_mode <var class=”var”>mode</var></samp></dt> <dd> Dolby Surround EX Mode. Indicates whether the stream uses Dolby Surround EX (7.1 matrixed to 5.1). Using this option does NOT mean the encoder will actually apply Dolby Surround EX processing. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dt><samp class=”option”>notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class=”option”>1</samp></dt>
<dt><samp class=”option”>on</samp></dt> <dd> Dolby Surround EX Off </dd> <dt><samp class=”option”>2</samp></dt>
<dt><samp class=”option”>off</samp></dt> <dd> Dolby Surround EX On </dd> </dl>

</dd> <dt><samp class=”option”>-dheadphone_mode <var class=”var”>mode</var></samp></dt>
<dd> Dolby Headphone Mode. Indicates whether the stream uses Dolby Headphone encoding (multi-channel matrixed to 2.0 for use with headphones). Using this option does NOT mean the encoder will actually apply Dolby Headphone processing. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dt><samp class=”option”>notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class=”option”>1</samp></dt>
<dt><samp class=”option”>on</samp></dt> <dd> Dolby Headphone Off </dd> <dt><samp class=”option”>2</samp></dt>
<dt><samp class=”option”>off</samp></dt> <dd> Dolby Headphone On </dd> </dl>

</dd> <dt><samp class=”option”>-ad_conv_type <var class=”var”>type</var></samp></dt>

<dd> A/D Converter Type. Indicates whether the audio has passed through HDCD A/D conversion. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">standard</samp></dt> <dd> Standard A/D Converter (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">hdcd</samp></dt> <dd> HDCD A/D Converter </dd> </dl>

</dd> </dl>

 <h4 class="subsection">15.2.3 Other AC-3 Encoding Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-stereo_rematrixing <var class="var">boolean</var></samp></dt> <dd> Stereo Rematrixing. Enables/Disables use of rematrixing for stereo input. This is an optional AC-3 feature that increases quality by selectively encoding the left/right channels as mid/side. This option is enabled by default, and it is highly recommended that it be left as enabled except for testing purposes. </dd> <dt><samp class="option">cutoff <var class="var">frequency</var></samp></dt> <dd> Set lowpass cutoff frequency. If unspecified, the encoder selects a default determined by various other encoding parameters. </dd> </dl>

 <h4 class="subsection">15.2.4 Floating-Point-Only AC-3 Encoding Options# TOC</h4>

These options are only valid for the floating-point encoder and do not exist for the fixed-point encoder due to the corresponding features not being implemented in fixed-point. <dl class="table"> <dt><samp class="option">-channel_coupling <var class="var">boolean</var></samp></dt> <dd> Enables/Disables use of channel coupling, which is an optional AC-3 feature that increases quality by combining high frequency information from multiple channels into a single channel. The per-channel high frequency information is sent with less accuracy in both the frequency and time domains. This allows more bits to be used for lower frequencies while preserving enough information to reconstruct the high frequencies. This option is enabled by default for the floating-point encoder and should generally be left as enabled except for testing purposes or to increase encoding speed. <dl class="table"> <dt><samp class="option">-1</samp></dt> <dt><samp class="option">auto</samp></dt> <dd> Selected by Encoder (default) </dd> <dt><samp class="option">0</samp></dt> <dt><samp

class=”option”>off</samp></dt> <dd> Disable Channel Coupling </dd> <dt><samp class=”option”>1</samp></dt> <dt><samp class=”option”>on</samp></dt> <dd> Enable Channel Coupling </dd> </dl>

</dd> <dt><samp class=”option”>-cpl_start_band <var class=”var”>number</var></samp></dt> <dd> Coupling Start Band. Sets the channel coupling start band, from 1 to 15. If a value higher than the bandwidth is used, it will be reduced to 1 less than the coupling end band. If <var class=”var”>auto</var> is used, the start band will be determined by the encoder based on the bit rate, sample rate, and channel layout. This option has no effect if channel coupling is disabled. <dl class=”table”> <dt><samp class=”option”>-1</samp></dt> <dt><samp class=”option”>auto</samp></dt> <dd> Selected by Encoder (default) </dd> </dl>

</dd> </dl>

 <h3 class=”section”>15.3 flac# TOC</h3>

FLAC (Free Lossless Audio Codec) Encoder <h4 class=”subsection”>15.3.1 Options# TOC</h4>

The following options are supported by FFmpeg’s flac encoder. <dl class=”table”> <dt><samp class=”option”>compression_level</samp></dt> <dd> Sets the compression level, which chooses defaults for many other options if they are not set explicitly. Valid values are from 0 to 12, 5 is the default. </dd> <dt><samp class=”option”>frame_size</samp></dt> <dd> Sets the size of the frames in samples per channel. </dd> <dt><samp class=”option”>lpc_coeff_precision</samp></dt> <dd> Sets the LPC coefficient precision, valid values are from 1 to 15, 15 is the default. </dd> <dt><samp class=”option”>lpc_type</samp></dt> <dd> Sets the first stage LPC algorithm <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> LPC is not used </dd> <dt>‘<samp class=”samp”>fixed</samp>’</dt> <dd> fixed LPC coefficients </dd> <dt>‘<samp class=”samp”>levinson</samp>’</dt> <dt>‘<samp class=”samp”>cholesky</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>lpc_passes</samp></dt> <dd> Number of passes to use for Cholesky factorization during LPC analysis </dd> <dt><samp class=”option”>min_partition_order</samp></dt> <dd> The minimum partition order </dd> <dt><samp class=”option”>max_partition_order</samp></dt> <dd> The maximum partition order </dd> <dt><samp class=”option”>prediction_order_method</samp></dt>

<dd><dl class="table"> <dt>‘<samp class="samp">estimation</samp>’</dt>
<dt>‘<samp class="samp">2level</samp>’</dt> <dt>‘<samp
class="samp">4level</samp>’</dt> <dt>‘<samp class="samp">8level</samp>’</d
<dt>‘<samp class="samp">search</samp>’</dt> <dd> Bruteforce
search </dd> <dt>‘<samp class="samp">log</samp>’</dt>
</dl>

</dd> <dt><samp class="option">ch_mode</samp></dt> <dd> Channel
mode <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> The mode is chosen automatically for each frame </dd>
<dt>‘<samp class="samp">indep</samp>’</dt> <dd> Channels
are independently coded </dd> <dt>‘<samp class="samp">left_side</samp>’</dt>
<dt>‘<samp class="samp">right_side</samp>’</dt>
<dt>‘<samp class="samp">mid_side</samp>’</dt> </dl>

</dd> <dt><samp class="option">exact_rice_parameters</samp></dt>
<dd> Chooses if rice parameters are calculated exactly or approximately. if set to 1 then they are chosen exactly, which slows the code down slightly and improves compression slightly. </dd> <dt><samp
class="option">multi_dim_quant</samp></dt> <dd> Multi Dimensional Quantization. If set to 1 then a 2nd stage LPC algorithm is applied after the first stage to finetune the coefficients. This is quite slow and slightly improves compression. </dd> </dl>

 <h3
class="section">15.4 opus<a class="anchor
hidden-xs" href="#opus" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-opus" aria-hidden="true">TOC</h3>

Opus encoder. This is a native FFmpeg encoder for the Opus format. Currently, it’s in development and only implements the CELT part of the codec. Its quality is usually worse and at best is equal to the libopus encoder.
<h4 class="subsection">15.4.1 Options<a
class="anchor hidden-xs" href="#Options-15" aria-hidden="true">#
TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set
bit rate in bits/s. If unspecified it uses the number of channels and the layout to make a good guess. </dd> <dt><samp class="option">opus_delay</samp></dt>
<dd> Sets the maximum delay in milliseconds. Lower delays than 20ms will very quickly decrease quality. </dd> </dl>

<h3 class="section">15.5 libfdk_aac<a class="anchor
hidden-xs" href="#libfdk_005faac" aria-hidden="true"># <a
class="anchor hidden-xs"href="#toc-libfdk_005faac" aria-hidden="true">TOC</h3>

libfdk-aac AAC (Advanced Audio Coding) encoder wrapper.

The libfdk-aac library is based on the Fraunhofer FDK AAC code from the Android project. Requires the presence of the libfdk-aac headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libfdk-aac</code>. The library is also incompatible with GPL, so if you allow the use of GPL, you should configure with <code class="code">--enable-gpl --enable-nonfree --enable-libfdk-aac</code>. This encoder has support for the AAC-HE profiles. VBR encoding, enabled through the <samp class="option">vbr</samp> or <samp class="option">flags +qscale</samp> options, is experimental and only works with some combinations of parameters. Support for encoding 7.1 audio is only available with libfdk-aac 0.1.3 or higher. For more information see the fdk-aac project at http://sourceforge.net/p/opencore-amr/fdk-aac/. <h4 class="subsection">15.5.1 Options# TOC</h4>

The following options are mapped on the shared FFmpeg codec options. <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bit rate in bits/s. If the bitrate is not explicitly specified, it is automatically set to a suitable value depending on the selected profile. In case VBR mode is enabled the option is ignored. </dd> <dt><samp class="option">ar</samp></dt> <dd> Set audio sampling rate (in Hz). </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of audio channels. </dd> <dt><samp class="option">flags +qscale</samp></dt> <dd> Enable fixed quality, VBR (Variable Bit Rate) mode. Note that VBR is implicitly enabled when the <samp class="option">vbr</samp> value is positive. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency. If not specified (or explicitly set to 0) it will use a value automatically computed by the library. Default value is 0. </dd> <dt><samp class="option">profile</samp></dt> <dd> Set audio profile. The following profiles are recognized: <dl class="table"> <dt>‘<samp class="samp">aac_low</samp>’</dt> <dd> Low Complexity AAC (LC) </dd> <dt>‘<samp class="samp">aac_he</samp>’</dt> <dd> High Efficiency AAC (HE-AAC) </dd> <dt>‘<samp class="samp">aac_he_v2</samp>’</dt> <dd> High Efficiency AAC version 2 (HE-AACv2) </dd> <dt>‘<samp class="samp">aac_ld</samp>’</dt> <dd> Low Delay AAC (LD) </dd> <dt>‘<samp class="samp">aac_eld</samp>’</dt> <dd> Enhanced Low Delay AAC (ELD) </dd> </dl>

If not specified it is set to ‘<samp class="samp">aac_low</samp>’. </dd> </dl>

The following are private options of the libfdk_aac encoder. <dl class="table"> <dt><samp class="option">afterburner</samp></dt> <dd> Enable afterburner feature if set to 1, disabled if set to 0. This improves the quality but also the required processing power. Default value is 1. </dd> <dt><samp class="option">eld_sbr</samp></dt> <dd> Enable SBR (Spectral Band Replication) for ELD if set to 1, disabled if set to 0. Default value is 0. </dd> <dt><samp class="option">eld_v2</samp></dt> <dd> Enable ELDv2 (LD-MPS extension for ELD stereo signals) for ELDv2 if set to 1, disabled if set to 0. Note that option is available when fdk-aac version (AACENCODER_LIB_VL0.AACENCODER_LIB_VL1.AACENCODER_LIB_VL2) > (4.0.0). Default value is 0. </dd> <dt><samp class="option">signaling</samp></dt> <dd> Set SBR/PS signaling style. It can assume one of the following values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> choose signaling implicitly (explicit hierarchical by default, implicit if global header is disabled) </dd> <dt>‘<samp class="samp">implicit</samp>’</dt> <dd> implicit backwards compatible signaling </dd> <dt>‘<samp class="samp">explicit_sbr</samp>’</dt> <dd> explicit SBR, implicit PS signaling </dd> <dt>‘<samp class="samp">explicit_hierarchical</samp>’</dt> <dd> explicit hierarchical signaling </dd> </dl>

Default value is ‘<samp class="samp">default</samp>’. </dd> <dt><samp class="option">latm</samp></dt> <dd> Output LATM/LOAS encapsulated data if set to 1, disabled if set to 0. Default value is 0. </dd> <dt><samp class="option">header_period</samp></dt> <dd> Set StreamMuxConfig and PCE repetition period (in frames) for sending in-band configuration buffers within LATM/LOAS transport layer. Must be a 16-bits non-negative integer. Default value is 0. </dd> <dt><samp class="option">vbr</samp></dt> <dd> Set VBR mode, from 1 to 5. 1 is lowest quality (though still pretty good) and 5 is highest quality. A value of 0 will disable VBR, and CBR (Constant Bit Rate) is enabled. Currently only the ‘<samp class="samp">aac_low</samp>’ profile supports VBR encoding. VBR modes 1-5 correspond to roughly the following average bit rates: <dl class="table"> <dt>‘<samp class="samp">1</samp>’</dt> <dd> 32 kbps/channel </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> 40 kbps/channel </dd> <dt>‘<samp class="samp">3</samp>’</dt> <dd> 48-56 kbps/channel </dd> <dt>‘<samp class="samp">4</samp>’</dt> <dd> 64 kbps/channel </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> about 80-96 kbps/channel </dd> </dl>

Default value is 0. </dd> <dt><samp class="option">frame_length</samp></dt> <dd> Set the audio frame length in samples. Default value is the internal default of the library. Refer to the library’s documentation for

information about supported values. </dd> </dl>

 <h4 class="subsection">15.5.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to convert an audio file to VBR AAC in an M4A (MP4) container: <pre class="example-preformatted">ffmpeg -i input.wav -codec:a libfdk_aac -vbr 3 output.m4a </pre>

Use <code class="command">ffmpeg</code> to convert an audio file to CBR 64k kbps AAC, using the High-Efficiency AAC profile: <pre class="example-preformatted">ffmpeg -i input.wav -c:a libfdk_aac -profile:a aac_he -b:a 64k output.m4a </pre>

 <h3 class="section">15.6 libfdk-xhe-aac# TOC</h3>

libfdk-xhe-aac (Extended High-Efficiency Advanced Audio Coding) encoder wrapper. The libfdk-xhe-aac library is based on the Fraunhofer FDK Extended High Efficiency AAC Encoder Software code from the Android project. Requires the presence of the libfdk-xhe-aac headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libfdk-xhe-aac</code>. The library is also incompatible with GPL, so if you allow the use of GPL, you should configure with <code class="code">--enable-gpl --enable-nonfree --enable-libfdk-xhe-aac</code>. <h4 class="subsection">15.6.1 Options# TOC</h4>

The following are private options of the libfdk_xhe_aac encoder. <dl class="table"> <dt><samp class="option">vbr</samp></dt> <dd> Set VBR mode, from 0 to 6. 0 is lowest quality and 6 is highest quality. A value of -1 will disable VBR, and CBR (Constant Bit Rate) is enabled. If not set, the default is CBR. </dd> <dt><samp class="option">live_loudness_level</samp></dt> <dd> Set the expected loudness level of the stream, between -31 and -10 dB LUFS. Defaults to -24 dB LUFS if not set. </dd> <dt><samp class="option">rap_length</samp></dt> <dd> The duration between encoded Random Access Points in the bitstream. </dd> <dt><samp class="option">stream_id</samp></dt> <dd> A unique ID of the encoded stream within an adaptation set. The ID is embedded into the encoded

bitstream. If not set, the encoder tries to generate a hopefully unique ID for the stream. </dd> </dl>

 <h3 class="section">15.7 liblc3# TOC</h3>

liblc3 LC3 (Low Complexity Communication Codec) encoder wrapper. Requires the presence of the liblc3 headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-liblc3</code>. This encoder has support for the Bluetooth SIG LC3 codec for the LE Audio protocol, and the following features of LC3plus: <ul class="itemize mark-bullet"> Frame duration of 2.5 and 5ms. High-Resolution mode, 48 KHz, and 96 kHz sampling rates.

For more information see the liblc3 project at <a class="url" href="https://github.com/google/liblc3">htt
  <h4 class="subsection">15.7.1 Options# TOC</h4>

The following options are mapped on the shared FFmpeg codec options. <dl class="table"> <dt><samp class="option">b <var class="var">bitrate</var></samp></dt> <dd> Set the bit rate in bits/s. This will determine the fixed size of the encoded frames, for a selected frame duration. </dd> <dt><samp class="option">ar <var class="var">frequency</var></samp></dt> <dd> Set the audio sampling rate (in Hz). </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of audio channels. </dd> <dt><samp class="option">frame_duration</samp></dt> <dd> Set the audio frame duration in milliseconds. Default value is 10ms. Allowed frame durations are 2.5ms, 5ms, 7.5ms and 10ms. LC3 (Bluetooth LE Audio), allows 7.5ms and 10ms; and LC3plus 2.5ms, 5ms and 10ms. The 10ms frame duration is available in LC3 and LC3 plus standard. In this mode, the produced bitstream can be referenced either as LC3 or LC3plus. </dd> <dt><samp class="option">high_resolution <var class="var">boolean</var></samp></dt> <dd> Enable the high-resolution mode if set to 1. The high-resolution mode is available with all LC3plus frame durations and for a sampling rate of 48 KHz, and 96 KHz. The encoder automatically turns off this mode at lower sampling rates and activates it at 96 KHz. This mode should be preferred at high bitrates. In this mode, the audio bandwidth is always up to the Nyquist frequency, compared to LC3 at 48 KHz, which limits the bandwidth to 20 KHz. </dd> </dl>

<h3 class="section">15.8 libmp3lame# TOC</h3>

LAME (Lame Ain’t an MP3 Encoder) MP3 encoder wrapper. Requires the presence of the libmp3lame headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libmp3lame</code>. See libshine for a fixed-point MP3 encoder, although with a lower quality. <h4 class="subsection">15.8.1 Options# TOC</h4>

The following options are supported by the libmp3lame wrapper. The <code class="command">lame</code>-equivalent of the options are listed in parentheses. <dl class="table"> <dt><samp class="option">b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR or ABR. LAME <code class="code">bitrate</code> is expressed in kilobits/s. </dd> <dt><samp class="option">q (-V)</samp></dt> <dd> Set constant quality setting for VBR. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> <dt><samp class="option">compression_level (-q)</samp></dt> <dd> Set algorithm quality. Valid arguments are integers in the 0-9 range, with 0 meaning highest quality but slowest, and 9 meaning fastest while producing the worst quality. </dd> <dt><samp class="option">cutoff (--lowpass)</samp></dt> <dd> Set lowpass cutoff frequency. If unspecified, the encoder dynamically adjusts the cutoff. </dd> <dt><samp class="option">strict (--strictly-enforce-ISO)</samp></dt> <dd> Enable LAME’s strict ISO buffer constraints when set to ‘<samp class="samp">strict</samp>’ or ‘<samp class="samp">very</samp>’. Other values leave LAME’s default buffer constraints unchanged. This can improve compatibility with some decoders, at the cost of reduced bit reservoir usage. </dd> <dt><samp class="option">reservoir</samp></dt> <dd> Enable use of bit reservoir when set to 1. Default value is 1. LAME has this enabled by default, but can be overridden by use <samp class="option">--nores</samp> option. </dd> <dt><samp class="option">joint_stereo (-m j)</samp></dt> <dd> Enable the encoder to use (on a frame by frame basis) either L/R stereo or mid/side stereo. Default value is 1. </dd> <dt><samp class="option">abr (--abr)</samp></dt> <dd> Enable the encoder to use ABR when set to 1. The <code class="command">lame</code> <samp class="option">--abr</samp> sets

the target bitrate, while this options only tells FFmpeg to use ABR still relies on <samp class="option">b</samp> to set bitrate. </dd> <dt><samp class="option">copyright (-c)</samp></dt> <dd> Set MPEG audio copyright flag when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">original (-o)</samp></dt> <dd> Set MPEG audio original flag when set to 1. The default value is 1 (enabled). </dd> </dl>

 <h3 class="section">15.9 libopencore-amrnb# TOC</h3>

OpenCORE Adaptive Multi-Rate Narrowband encoder. Requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopencore-amrnb --enable-version3</code>. This is a mono-only encoder. Officially it only supports 8000Hz sample rate, but you can override it by setting <samp class="option">strict</samp> to ‘<samp class="samp">unofficial</samp>’ or lower. <h4 class="subsection">15.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate in bits per second. Only the following bitrates are supported, otherwise libavcodec will round to the nearest valid bitrate. <dl class="table"> <dt><samp class="option">4750</samp></dt> <dt><samp class="option">5150</samp></dt> <dt><samp class="option">5900</samp></dt> <dt><samp class="option">6700</samp></dt> <dt><samp class="option">7400</samp></dt> <dt><samp class="option">7950</samp></dt> <dt><samp class="option">10200</samp></dt> <dt><samp class="option">12200</samp></dt> </dl>

</dd> <dt><samp class="option">dtx</samp></dt> <dd> Allow discontinuous transmission (generate comfort noise) when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">15.10 libopus# TOC</h3>

libopus Opus Interactive Audio Codec encoder wrapper. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopus</code>.

<h4 class="subsection">15.10.1 Option Mapping# TOC</h4>

Most libopus options are modelled after the <code class="command">opusenc</code> utility from opus-tools. The following is an option mapping chart describing options supported by the libopus wrapper, and their <code class="command">opusenc</code>-equivalent in parentheses. <dl class="table"> <dt><samp class="option">b (bitrate)</samp></dt> <dd> Set the bit rate in bits/s. FFmpeg’s <samp class="option">b</samp> option is expressed in bits/s, while <code class="command">opusenc</code>’s <samp class="option">bitrate</samp> in kilobits/s. </dd> <dt><samp class="option">vbr (vbr, hard-cbr, and cvbr)</samp></dt> <dd> Set VBR mode. The FFmpeg <samp class="option">vbr</samp> option has the following valid arguments, with the <code class="command">opusenc</code> equivalent options in parentheses: <dl class="table"> <dt>‘<samp class="samp">off (hard-cbr)</samp>’</dt> <dd> Use constant bit rate encoding. </dd> <dt>‘<samp class="samp">on (vbr)</samp>’</dt> <dd> Use variable bit rate encoding (the default). </dd> <dt>‘<samp class="samp">constrained (cvbr)</samp>’</dt> <dd> Use constrained variable bit rate encoding. </dd> </dl>

</dd> <dt><samp class="option">compression_level (comp)</samp></dt> <dd> Set encoding algorithm complexity. Valid options are integers in the 0-10 range. 0 gives the fastest encodes but lower quality, while 10 gives the highest quality but slowest encoding. The default is 10. </dd> <dt><samp class="option">frame_duration (framesize)</samp></dt> <dd> Set maximum frame size, or duration of a frame in milliseconds. The argument must be exactly the following: 2.5, 5, 10, 20, 40, 60. Smaller frame sizes achieve lower latency but less quality at a given bitrate. Sizes greater than 20ms are only interesting at fairly low bitrates. The default is 20ms. </dd> <dt><samp class="option">packet_loss (expect-loss)</samp></dt> <dd> Set expected packet loss percentage. The default is 0. </dd> <dt><samp class="option">fec (n/a)</samp></dt> <dd> Enable inband forward error correction. <samp class="option">packet_loss</samp> must be non-zero to take advantage - frequency of FEC ’side-data’ is proportional to expected packet loss. Default is disabled. </dd> <dt><samp class="option">application (N.A.)</samp></dt> <dd> Set intended application type. Valid options are listed below: <dl class="table"> <dt>‘<samp class="samp">voip</samp>’</dt> <dd> Favor improved speech intelligibility. </dd> <dt>‘<samp class="samp">audio</samp>’</dt> <dd> Favor faithfulness to the input (the default). </dd>

<dt>‘<samp class="samp">lowdelay</samp>’</dt> <dd> Restrict to only the lowest delay modes by disabling voice-optimized modes. </dd> </dl>

</dd> <dt><samp class="option">cutoff (N.A.)</samp></dt> <dd> Set cutoff bandwidth in Hz. The argument must be exactly one of the following: 4000, 6000, 8000, 12000, or 20000, corresponding to narrowband, mediumband, wideband, super wideband, and fullband respectively. The default is 0 (cutoff disabled). Note that libopus forces a wideband cutoff for bitrates < 15 kbps, unless CELT-only (<samp class="option">application</samp> set to ‘<samp class="samp">lowdelay</samp>’) mode is used. </dd> <dt><samp class="option">mapping_family (mapping_family)</samp></dt> <dd> Set channel mapping family to be used by the encoder. The default value of -1 uses mapping family 0 for mono and stereo inputs, and mapping family 1 otherwise. The default also disables the surround masking and LFE bandwidth optimizations in libopus, and requires that the input contains 8 channels or fewer. Other values include 0 for mono and stereo, 1 for surround sound with masking and LFE bandwidth optimizations, and 255 for independent streams with an unspecified channel layout. </dd> <dt><samp class="option">dtx (N.A.)</samp></dt> <dd> Allow discontinuous transmission when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">apply_phase_inv (N.A.) (requires libopus >= 1.2)</samp></dt> <dd> If set to 0, disables the use of phase inversion for intensity stereo, improving the quality of mono downmixes, but slightly reducing normal stereo quality. The default is 1 (phase inversion enabled). </dd> </dl>

 <h3 class="section">15.11 libshine# TOC</h3>

Shine Fixed-Point MP3 encoder wrapper. Shine is a fixed-point MP3 encoder. It has a far better performance on platforms without an FPU, e.g. armel CPUs, and some phones and tablets. However, as it is more targeted on performance than quality, it is not on par with LAME and other production-grade encoders quality-wise. Also, according to the project’s homepage, this encoder may not be free of bugs as the code was written a long time ago and the project was dead for at least 5 years. This encoder only supports stereo and mono input. This is also CBR-only. The original project (last updated in early 2007) is at http://sourceforge.net/projects/libshine-fxp/. We only support the updated fork by the Savonet/Liquidsoap project at https://github.com/savonet/shine. Requires the presence of the libshine headers and library during

configuration. You need to explicitly configure the build with <code class="code">--enable-libshine</code>. See also libmp3lame. <h4 class="subsection">15.11.1 Options#
TOC</h4>

The following options are supported by the libshine wrapper. The <code class="command">shineenc</code>-equivalent of the options are listed in parentheses. <dl class="table"> <dt><samp class="option">b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR. <code class="command">shineenc</code> <samp class="option">-b</samp> option is expressed in kilobits/s. </dd> </dl>

 <h3 class="section">15.12 libtwolame# TOC</h3>

TwoLAME MP2 encoder wrapper. Requires the presence of the libtwolame headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libtwolame</code>. <h4 class="subsection">15.12.1 Options# TOC</h4>

The following options are supported by the libtwolame wrapper. The <code class="command">twolame</code>-equivalent options follow the FFmpeg ones and are in parentheses. <dl class="table"> <dt><samp class="option">b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR. <code class="command">twolame</code> <samp class="option">b</samp> option is expressed in kilobits/s. Default value is 128k. </dd> <dt><samp class="option">q (-V)</samp></dt> <dd> Set quality for experimental VBR support. Maximum value range is from -50 to 50, useful range is from -10 to 10. The higher the value, the better the quality. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> <dt><samp class="option">mode (--mode)</samp></dt> <dd> Set the mode of the resulting audio. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose mode automatically based on the input. This is the default. </dd> <dt>‘<samp class="samp">stereo</samp>’</dt> <dd> Stereo </dd> <dt>‘<samp class="samp">joint_stereo</samp>’</dt> <dd> Joint stereo </dd> <dt>‘<samp class="samp">dual_channel</samp>’</dt>

<dd> Dual channel </dd> <dt>‘<samp class="samp">mono</samp>’</dt> <dd> Mono </dd> </dl>

</dd> <dt><samp class="option">psymodel (--psyc-mode)</samp></dt> <dd> Set psychoacoustic model to use in encoding. The argument must be an integer between -1 and 4, inclusive. The higher the value, the better the quality. The default value is 3. </dd> <dt><samp class="option">energy_levels (--energy)</samp></dt> <dd> Enable energy levels extensions when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">error_protection (--protect)</samp></dt> <dd> Enable CRC error protection when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">copyright (--copyright)</samp></dt> <dd> Set MPEG audio copyright flag when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">original (--original)</samp></dt> <dd> Set MPEG audio original flag when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">15.13 libvo-amrwbenc# TOC</h3>

VisualOn Adaptive Multi-Rate Wideband encoder. Requires the presence of the libvo-amrwbenc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libvo-amrwbenc --enable-version3</code>. This is a mono-only encoder. Officially it only supports 16000Hz sample rate, but you can override it by setting <samp class="option">strict</samp> to ‘<samp class="samp">unofficial</samp>’ or lower. <h4 class="subsection">15.13.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate in bits/s. Only the following bitrates are supported, otherwise libavcodec will round to the nearest valid bitrate. <dl class="table"> <dt>‘<samp class="samp">6600</samp>’</dt> <dt>‘<samp class="samp">8850</samp>’</dt> <dt>‘<samp class="samp">12650</samp>’</dt> <dt>‘<samp class="samp">14250</samp>’</dt> <dt>‘<samp class="samp">15850</samp>’</dt> <dt>‘<samp class="samp">18250</samp>’</dt> <dt>‘<samp class="samp">19850</samp>’</dt> <dt>‘<samp class="samp">23050</samp>’</dt> <dt>‘<samp class="samp">23850</samp>’</dt> </dl>

</dd> <dt><samp class="option">dtx</samp></dt> <dd> Allow discontinuous transmission (generate comfort noise) when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">15.14 libvorbis# TOC</h3>

libvorbis encoder wrapper. Requires the presence of the libvorbisenc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libvorbis</code>. <h4 class="subsection">15.14.1 Options# TOC</h4>

The following options are supported by the libvorbis wrapper. The <code class="command">oggenc</code>-equivalent of the options are listed in parentheses. To get a more accurate and extensive documentation of the libvorbis options, consult the libvorbisenc’s and <code class="command">oggenc</code>’s documentations. See http://xiph.org/vorbis/, http://wiki.xiph.org/Vorbis-tools, and oggenc(1). <dl class="table"> <dt><samp class="option">b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for ABR. <code class="command">oggenc</code> <samp class="option">-b</samp> is expressed in kilobits/s. </dd> <dt><samp class="option">q (-q)</samp></dt> <dd> Set constant quality setting for VBR. The value should be a float number in the range of -1.0 to 10.0. The higher the value, the better the quality. The default value is ‘<samp class="samp">3.0</samp>’. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> <dt><samp class="option">cutoff (--advanced-encode-option lowpass_frequency=N)</samp></dt> <dd> Set cutoff bandwidth in Hz, a value of 0 disables cutoff. <code class="command">oggenc</code>’s related option is expressed in kHz. The default value is ‘<samp class="samp">0</samp>’ (cutoff disabled). </dd> <dt><samp class="option">minrate (-m)</samp></dt> <dd> Set minimum bitrate expressed in bits/s. <code class="command">oggenc</code> <samp class="option">-m</samp> is expressed in kilobits/s. </dd> <dt><samp class="option">maxrate (-M)</samp></dt> <dd> Set maximum bitrate expressed in bits/s. <code class="command">oggenc</code> <samp class="option">-M</samp> is expressed in kilobits/s. This only has effect on ABR mode.

</dd> <dt><samp class="option">iblock (--advanced-encode-option impulse_noisetune=N)</samp></dt> <dd> Set noise floor bias for impulse blocks. The value is a float number from -15.0 to 0.0. A negative bias instructs the encoder to pay special attention to the crispness of transients in the encoded audio. The tradeoff for better transient response is a higher bitrate. </dd> </dl>

 <h3 class="section">15.15 mjpeg# TOC</h3>

Motion JPEG encoder. <h4 class="subsection">15.15.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">huffman</samp></dt> <dd> Set the huffman encoding strategy. Possible values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Use the default huffman tables. This is the default strategy. </dd> <dt>‘<samp class="samp">optimal</samp>’</dt> <dd> Compute and use optimal huffman tables. </dd> </dl>

</dd> <dt><samp class="option">intra_dc_precision <var class="var">integer</var></samp></dt> <dd> Set the precision of the DC coefficient minus 8. Valid range is 0 to 3. Higher values quantize DC more finely as the DC entry of the written quantization table is 8 >> <var class="var">intra_dc_precision</var> so 3 gives a DC quantizer of 1. Default is 0 which keeps the previous fixed DC quantizer of 8. Combined with <code class="code">-q:v 1</code> and an <code class="code">intra_matrix</code> of all 8s, a value of 3 produces a quantization table of all ones. </dd> </dl>

 <h3 class="section">15.16 wavpack# TOC</h3>

WavPack lossless audio encoder. <h4 class="subsection">15.16.1 Options# TOC</h4>

The equivalent options for <code class="command">wavpack</code> command line utility are listed in parentheses. <h4 class="subsubsection">15.16.1.1 Shared options# TOC</h4>

The following shared options are effective for this encoder. Only special notes about this particular encoder will be documented here. For the general meaning of the options, see the Codec Options chapter. <dl class="table"> <dt><samp class="option">frame_size (--blocksize)</samp></dt> <dd> For this encoder, the range for this option is between 128 and 131072. Default is automatically decided based on sample rate and number of channel. For the complete formula of calculating default, see <samp class="file">libavcodec/wavpackenc.c</samp>. </dd> <dt><samp class="option">compression_level (-f, -h, -hh, and -x)</samp></dt> </dl>

 <h4 class="subsubsection">15.16.1.2 Private options# TOC</h4>

<dl class="table"> <dt><samp class="option">joint_stereo (-j)</samp></dt> <dd> Set whether to enable joint stereo. Valid values are: <dl class="table"> <dt>‘<samp class="samp">on (1)</samp>’</dt> <dd> Force mid/side audio encoding. </dd> <dt>‘<samp class="samp">off (0)</samp>’</dt> <dd> Force left/right audio encoding. </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Let the encoder decide automatically. </dd> </dl>

</dd> <dt><samp class="option">optimize_mono</samp></dt> <dd> Set whether to enable optimization for mono. This option is only effective for non-mono streams. Available values: <dl class="table"> <dt>‘<samp class="samp">on</samp>’</dt> <dd> enabled </dd> <dt>‘<samp class="samp">off</samp>’</dt> <dd> disabled </dd> </dl>

</dd> </dl>

 <h2 class="chapter">16 Video Encoders# TOC</h2>

A description of some of the currently available video encoders follows. <h3 class="section">16.1 a64_multi, a64_multi5# TOC</h3>

A64 / Commodore 64 multicolor charset encoder. <code class=”code”>a64_multi5</code> is extended with 5th color (colram). <h3 class=”section”>16.2 Cinepak# TOC</h3>

Cinepak aka CVID encoder. Compatible with Windows 3.1 and vintage MacOS. <h4 class=”subsection”>16.2.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>g <var class=”var”>integer</var></samp></dt> <dd> Keyframe interval. A keyframe is inserted at least every <code class=”code”>-g</code> frames, sometimes sooner. </dd> <dt><samp class=”option”>q:v <var class=”var”>integer</var></samp></dt> <dd> Quality factor. Lower is better. Higher gives lower bitrate. The following table lists bitrates when encoding akiyo_cif.y4m for various values of <code class=”code”>-q:v</code> with <code class=”code”>-g 100</code>: <dl class=”table”> <dt><samp class=”option”><code class=”code”>-q:v 1</code> 1918 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 2</code> 1735 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 4</code> 1500 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 10</code> 1041 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 20</code> 826 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 40</code> 553 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 100</code> 394 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 200</code> 312 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 400</code> 266 kb/s</samp></dt> <dt><samp class=”option”><code class=”code”>-q:v 1000</code> 237 kb/s</samp></dt> </dl>

</dd> <dt><samp class=”option”>max_extra_cb_iterations <var class=”var”>integer</var></samp></dt> <dd> Max extra codebook recalculation passes, more is better and slower. </dd> <dt><samp class=”option”>skip_empty_cb <var class=”var”>boolean</var></samp></dt> <dd> Avoid wasting bytes, ignore vintage MacOS decoder. </dd> <dt><samp class=”option”>max_strips <var class=”var”>integer</var></samp></dt> <dt><samp class=”option”>min_strips <var class=”var”>integer</var></samp></dt> <dd> The minimum and maximum number of strips to use. Wider range sometimes improves quality. More strips is generally better quality but costs more bits. Fewer strips tend to yield more keyframes. Vintage compatible is 1..3. </dd> <dt><samp class=”option”>strip_number_adaptivity <var class=”var”>integer</var></samp></dt> <dd> How much number of strips is allowed to change between frames. Higher is better but slower. </dd> </dl>

 <h3 class=”section”>16.3 ffv1# TOC</h3>

FFv1 Encoder <h4 class=”subsection”>16.3.1 Options# TOC</h4>

The following options are supported by FFmpeg’s FFv1 encoder. <dl class=”table”> <dt><samp class=”option”>context</samp></dt> <dd> Sets the context size, 0 (default) is small, 1 is big, 2 is minimal. The minimal context set compresses better when coding many slices, as each slice has too little data for larger context sets to adapt to. </dd> <dt><samp class=”option”>coder</samp></dt> <dd> Set the coder, <dl class=”table”> <dt>‘<samp class=”samp”>rice</samp>’</dt> <dd> Golomb rice coder </dd> <dt>‘<samp class=”samp”>range_def</samp>’</dt> <dd> Range coder with default table </dd> <dt>‘<samp class=”samp”>range_tab</samp>’</dt> <dd> Range coder with custom table </dd> </dl>

</dd> <dt><samp class=”option”>slicecrc</samp></dt> <dd> -1 (default, automatic), 1 use crc with zero initial and final state, 2 use crc with non zero initial and final state </dd> <dt><samp class=”option”>qtable</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt> <dd> default, automatic </dd> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dd> use 8bit default </dd> <dt>‘<samp class=”samp”>greater8bit</samp>’</dt <dd> use >8bit default </dd> </dl>

</dd> <dt><samp class=”option”>remap_optimizer</samp></dt> <dd> 0 - 5, default 3, how much effort the encoder puts into optimizing the remap table. </dd> </dl>

 <h3 class=”section”>16.4 GIF# TOC</h3>

GIF image/animation encoder. <h4 class=”subsection”>16.4.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>gifflags <var class=”var”>integer</var></samp></dt> <dd> Sets the flags used for GIF encoding. <dl class=”table”> <dt><samp class=”option”>offsetting</samp></dt> <dd> Enables picture offsetting. Default is enabled. </dd> <dt><samp

class=”option”>transdiff</samp></dt> <dd> Enables transparency detection between frames. Default is enabled. </dd> </dl>

</dd> <dt><samp class=”option”>gifimage <var class=”var”>integer</var></samp></dt> <dd> Enables encoding one full GIF image per frame, rather than an animated GIF. Default value is <samp class=”option”>0</samp>. </dd> <dt><samp class=”option”>global_palette <var class=”var”>integer</var></samp></dt> <dd> Writes a palette to the global GIF header where feasible. If disabled, every frame will always have a palette written, even if there is a global palette supplied. Default value is <samp class=”option”>1</samp>. </dd> </dl>

 <h3 class=”section”>16.5 Hap# TOC</h3>

Vidvox Hap video encoder. <h4 class=”subsection”>16.5.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>format <var class=”var”>integer</var></samp></dt> <dd> Specifies the Hap format to encode. <dl class=”table”> <dt><samp class=”option”>hap</samp></dt> <dt><samp class=”option”>hap_alpha</samp></dt> <dt><samp class=”option”>hap_q</samp></dt> </dl>

Default value is <samp class=”option”>hap</samp>. </dd> <dt><samp class=”option”>chunks <var class=”var”>integer</var></samp></dt> <dd> Specifies the number of chunks to split frames into, between 1 and 64. This permits multithreaded decoding of large frames, potentially at the cost of data-rate. The encoder may modify this value to divide frames evenly. Default value is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>compressor <var class=”var”>integer</var></samp></dt> <dd> Specifies the second-stage compressor to use. If set to <samp class=”option”>none</samp>, <samp class=”option”>chunks</samp> will be limited to 1, as chunked uncompressed frames offer no benefit. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dt><samp class=”option”>snappy</samp></dt> </dl>

Default value is <samp class=”option”>snappy</samp>. </dd> </dl>

 <h3 class=”section”>16.6 jpeg2000# TOC</h3>

The native jpeg 2000 encoder is lossy by default, the <code class=”code”>-q:v</code> option can be used to set the encoding quality. Lossless encoding can be selected with <code class=”code”>-pred 1</code>. <a

name=”Options-31”> <h4 class=”subsection”>16.6.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>format <var class=”var”>integer</var></samp></dt> <dd> Can be set to either <code class=”code”>j2k</code> or <code class=”code”>jp2</code> (the default) that makes it possible to store non-rgb pix_fmts. </dd> <dt><samp class=”option”>tile_width <var class=”var”>integer</var></samp></dt> <dd> Sets tile width. Range is 1 to 1073741824. Default is 256. </dd> <dt><samp class=”option”>tile_height <var class=”var”>integer</var></samp></dt> <dd> Sets tile height. Range is 1 to 1073741824. Default is 256. </dd> <dt><samp class=”option”>pred <var class=”var”>integer</var></samp></dt> <dd> Allows setting the discrete wavelet transform (DWT) type <dl class=”table”> <dt><samp class=”option”>dwt97int (Lossy)</samp></dt> <dt><samp class=”option”>dwt53 (Lossless)</samp></dt> </dl> Default is <code class=”code”>dwt97int</code> </dd> <dt><samp class=”option”>sop <var class=”var”>boolean</var></samp></dt> <dd> Enable this to add SOP marker at the start of each packet. Disabled by default. </dd> <dt><samp class=”option”>eph <var class=”var”>boolean</var></samp></dt> <dd> Enable this to add EPH marker at the end of each packet header. Disabled by default. </dd> <dt><samp class=”option”>prog <var class=”var”>integer</var></samp></dt> <dd> Sets the progression order to be used by the encoder. Possible values are: <dl class=”table”> <dt><samp class=”option”>lrcp</samp></dt> <dt><samp class=”option”>rlcp</samp></dt> <dt><samp class=”option”>rpcl</samp></dt> <dt><samp class=”option”>pcrl</samp></dt> <dt><samp class=”option”>cprl</samp></dt> </dl> Set to <code class=”code”>lrcp</code> by default. </dd> <dt><samp class=”option”>layer_rates <var class=”var”>string</var></samp></dt> <dd> By default, when this option is not used, compression is done using the quality metric. This option allows for compression using compression ratio. The compression ratio for each level could be specified. The compression ratio of a layer <code class=”code”>l</code> species the what ratio of total file size is contained in the first <code class=”code”>l</code> layers. Example usage: <pre class=”example-preformatted”>ffmpeg -i input.bmp -c:v jpeg2000 -layer_rates "100,10,1" output.j2k </pre>

This would compress the image to contain 3 layers, where the data contained in the first layer would be compressed by 1000 times, compressed by 100 in the first two layers, and shall contain all data while using all 3 layers. </dd> </dl>

 <h3 class=”section”>16.7 librav1e<a class=”anchor hidden-xs” href=”#librav1e” aria-

hidden=”true”># TOC</h3>

rav1e AV1 encoder wrapper. Requires the presence of the rav1e headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-librav1e</code>. <h4 class=”subsection”>16.7.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>qmax</samp></dt> <dd> Sets the maximum quantizer to use when using bitrate mode. </dd> <dt><samp class=”option”>qmin</samp></dt> <dd> Sets the minimum quantizer to use when using bitrate mode. </dd> <dt><samp class=”option”>qp</samp></dt> <dd> Uses quantizer mode to encode at the given quantizer (0-255). </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Selects the speed preset (0-10) to encode with. </dd> <dt><samp class=”option”>tiles</samp></dt> <dd> Selects how many tiles to encode with. </dd> <dt><samp class=”option”>tile-rows</samp></dt> <dd> Selects how many rows of tiles to encode with. </dd> <dt><samp class=”option”>tile-columns</samp></dt> <dd> Selects how many columns of tiles to encode with. </dd> <dt><samp class=”option”>rav1e-params</samp></dt> <dd> Set rav1e options using a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ":". See <code class=”command”>rav1e --help</code> for a list of options. For example to specify librav1e encoding options with <samp class=”option”>-rav1e-params</samp>:   <pre class=”example-preformatted”>ffmpeg -i input -c:v librav1e -b:v 500K -rav1e-params speed=5:low_latency=true output.mp4 </pre>

</dd> </dl>

 <h3 class=”section”>16.8 libaom-av1# TOC</h3>

libaom AV1 encoder wrapper. Requires the presence of the libaom headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libaom</code>. <h4 class=”subsection”>16.8.1 Options# TOC</h4>

The wrapper supports the following standard libavcodec options:

 <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate target in bits/second. By default this will use variable-bitrate mode. If <samp class="option">maxrate</samp> and <samp class="option">minrate</samp> are also set to the same value then it will use constant-bitrate mode, otherwise if <samp class="option">crf</samp> is set as well then it will use constrained-quality mode. </dd> <dt><samp class="option">g keyint_min</samp></dt> <dd> Set key frame placement. The GOP size sets the maximum distance between key frames; if zero the output stream will be intra-only. The minimum distance is ignored unless it is the same as the GOP size, in which case key frames will always appear at a fixed interval. Not set by default, so without this option the library has completely free choice about where to place key frames. </dd> <dt><samp class="option">qmin qmax</samp></dt> <dd> Set minimum/maximum quantisation values. Valid range is from 0 to 63 (warning: this does not match the quantiser values actually used by AV1 - divide by four to map real quantiser values to this range). Defaults to min/max (no constraint). </dd> <dt><samp class="option">minrate maxrate bufsize rc_init_occupancy</samp></dt> <dd> Set rate control buffering parameters. Not used if not set - defaults to unconstrained variable bitrate. </dd> <dt><samp class="option">threads</samp></dt> <dd> Set the number of threads to use while encoding. This may require the <samp class="option">tiles</samp> or <samp class="option">row-mt</samp> options to also be set to actually use the specified number of threads fully. Defaults to the number of hardware threads supported by the host machine. </dd> <dt><samp class="option">profile</samp></dt> <dd> Set the encoding profile. Defaults to using the profile which matches the bit depth and chroma subsampling of the input. </dd> </dl>

The wrapper also has some specific options: <dl class="table"> <dt><samp class="option">cpu-used</samp></dt> <dd> Set the quality/encoding speed tradeoff. Valid range is from 0 to 8, higher numbers indicating greater speed and lower quality. The default value is 1, which will be slow and high quality. </dd> <dt><samp class="option">auto-alt-ref</samp></dt> <dd> Enable use of alternate reference frames. Defaults to the internal default of the library. </dd> <dt><samp class="option">arnr-max-frames (frames)</samp></dt> <dd> Set altref noise reduction max frame count. Default is -1. </dd> <dt><samp class="option">arnr-strength (strength)</samp></dt> <dd> Set altref noise reduction filter strength. Range is -1 to 6. Default is -1. </dd> <dt><samp class="option">aq-mode (aq-mode)</samp></dt> <dd> Set adaptive quantization mode. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class="samp">variance (1)</samp>’</dt> <dd> Variance-

based. </dd> <dt>‘<samp class="samp">complexity (2)</samp>’</dt> <dd> Complexity-based. </dd> <dt>‘<samp class="samp">cyclic (3)</samp>’</dt> <dd> Cyclic refresh. </dd> </dl>

</dd> <dt><samp class="option">tune (tune)</samp></dt> <dd> Set the distortion metric the encoder is tuned with. Default is <code class="code">psnr</code>. <dl class="table"> <dt>‘<samp class="samp">psnr (0)</samp>’</dt> <dt>‘<samp class="samp">ssim (1)</samp>’</dt> </dl>

</dd>

<dt><samp class="option">lag-in-frames</samp></dt>

<dd> Set the maximum number of frames which the encoder may keep in flight

at any one time for lookahead purposes. Defaults to the internal

default of the library.

</dd>

<dt><samp class="option">error-resilience</samp></dt>

<dd> Enable error resilience features:

<dl class="table">

<dt><samp class="option">default</samp></dt>

<dd> Improve resilience against losses of whole frames.

</dd>

</dl>

Not enabled by default.

</dd>

<dt><samp class="option">crf</samp></dt>

<dd> Set the quality/size tradeoff for constant-quality (no bitrate target)

and constrained-quality (with maximum bitrate target) modes. Valid

range is 0 to 63, higher numbers indicating lower quality and smaller

output size. Only used if set; by default only the bitrate target is

used.

</dd>

<dt><samp class="option">static-thresh</samp></dt>

<dd> Set a change threshold on blocks below which they will be skipped by

the encoder. Defined in arbitrary units as a nonnegative integer,

defaulting to zero (no blocks are skipped).

</dd>

<dt><samp class="option">drop-threshold</samp></dt>

<dd> Set a threshold for dropping frames when close to rate control bounds.

Defined as a percentage of the target buffer - when the rate control

buffer falls below this percentage, frames will be dropped until it

has refilled above the threshold. Defaults to zero (no frames are

dropped).

</dd>

<dt><samp class="option">denoise-noise-level (level)</samp></dt>

<dd> Amount of noise to be removed for grain synthesis. Grain synthesis is disabled if

this option is not set or set to 0.

</dd>

<dt><samp class="option">denoise-block-size (pixels)</samp></dt>

<dd> Block size used for denoising for grain synthesis. If not set, AV1 codec

uses the default value of 32.

</dd>

<dt><samp class="option">undershoot-pct (pct)</samp></dt>

<dd> Set datarate undershoot (min) percentage of the target bitrate. Range is -1 to 100.

Default is -1.

</dd>

<dt><samp class="option">overshoot-pct (pct)</samp></dt>

<dd> Set datarate overshoot (max) percentage of the target bitrate. Range
is -1 to 1000.

Default is -1.

</dd>

<dt><samp class="option">minsection-pct (pct)</samp></dt>

<dd> Minimum percentage variation of the GOP bitrate from the target
bitrate. If minsection-pct

is not set, the libaomenc wrapper computes it as follows: <code class="code">(minrate
* 100 / bitrate)</code>.

Range is -1 to 100. Default is -1 (unset).

</dd>

<dt><samp class="option">maxsection-pct (pct)</samp></dt>

<dd> Maximum percentage variation of the GOP bitrate from the target
bitrate. If maxsection-pct

is not set, the libaomenc wrapper computes it as follows: <code class="code">(maxrate
* 100 / bitrate)</code>.

Range is -1 to 5000. Default is -1 (unset).

</dd>

<dt><samp class="option">frame-parallel (boolean)</samp></dt>

<dd> Enable frame parallel decodability features. Default is true.

</dd>

<dt><samp class="option">tiles</samp></dt>

<dd> Set the number of tiles to encode the input video with, as columns x

rows. Larger numbers allow greater parallelism in both encoding and

decoding, but may decrease coding efficiency. Defaults to the minimum
number of tiles required by the size of the input video (this is 1x1
(that is, a single tile) for sizes up to and including 4K).

</dd>
<dt><samp class="option">tile-columns tile-rows</samp></dt>
<dd> Set the number of tiles as log2 of the number of tile rows and columns.
Provided for compatibility with libvpx/VP9.

</dd>
<dt><samp class="option">row-mt (Requires libaom >= 1.0.0-759-g90a15f4f2)</samp></dt>
<dd> Enable row based multi-threading. Disabled by default.

</dd>
<dt><samp class="option">enable-cdef (boolean)</samp></dt>
<dd> Enable Constrained Directional Enhancement Filter. The libaom-av1
encoder enables CDEF by default.

</dd>
<dt><samp class="option">enable-restoration (boolean)</samp></dt>
<dd> Enable Loop Restoration Filter. Default is true for libaom-av1.

</dd>
<dt><samp class="option">enable-global-motion (boolean)</samp></dt>
<dd> Enable the use of global motion for block prediction. Default is true.

</dd>
<dt><samp class="option">enable-intrabc (boolean)</samp></dt>
<dd> Enable block copy mode for intra block prediction. This mode is
useful for screen content. Default is true.

</dd>

<dt><samp class="option">enable-rect-partitions (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable rectangular partitions. Default is true.

</dd>

<dt><samp class="option">enable-1to4-partitions (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable 1:4/4:1 partitions. Default is true.

</dd>

<dt><samp class="option">enable-ab-partitions (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable AB shape partitions. Default is true.

</dd>

<dt><samp class="option">enable-angle-delta (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable angle delta intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-cfl-intra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable chroma predicted from luma intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-filter-intra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable filter intra predictor. Default is true.

</dd>

<dt><samp class="option">enable-intra-edge-filter (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable intra edge filter. Default is true.

</dd>

<dt><samp class="option">enable-smooth-intra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable smooth intra prediction mode. Default is true.

</dd>

<dt><samp class="option">enable-paeth-intra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable paeth predictor in intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-palette (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable palette prediction mode. Default is true.

</dd>

<dt><samp class="option">enable-flip-idtx (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable extended transform type, including FLIPADST_DCT, DCT_FLIPADST,

FLIPADST_FLIPADST, ADST_FLIPADST, FLIPADST_ADST, IDTX, V_DCT, H_DCT,

V_ADST, H_ADST, V_FLIPADST, H_FLIPADST. Default is true.

</dd>

<dt><samp class="option">enable-tx64 (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable 64-pt transform. Default is true.

</dd>

<dt><samp class="option">reduced-tx-type-set (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use reduced set of transform types. Default is false.

</dd>

<dt><samp class="option">use-intra-dct-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use DCT only for INTRA modes. Default is false.

</dd>

<dt><samp class="option">use-inter-dct-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use DCT only for INTER modes. Default is false.

</dd>

<dt><samp class="option">use-intra-default-tx-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use Default-transform only for INTRA modes. Default is false.

</dd>

<dt><samp class="option">enable-ref-frame-mvs (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable temporal mv prediction. Default is true.

</dd>

<dt><samp class="option">enable-reduced-reference-set (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use reduced set of single and compound references. Default is false.

</dd>

<dt><samp class="option">enable-obmc (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable obmc. Default is true.

</dd>

<dt><samp class="option">enable-dual-filter (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable dual filter. Default is true.

</dd>

<dt><samp class="option">enable-diff-wtd-comp (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable difference-weighted compound. Default is true.

</dd>

<dt><samp class="option">enable-dist-wtd-comp (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable distance-weighted compound. Default is true.

</dd>

<dt><samp class="option">enable-onesided-comp (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable one sided compound. Default is true.

</dd>

<dt><samp class="option">enable-interinter-wedge (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interinter wedge compound. Default is true.

</dd>

<dt><samp class="option">enable-interintra-wedge (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interintra wedge compound. Default is true.

</dd>

<dt><samp class=”option”>enable-masked-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable masked compound. Default is true.

</dd>

<dt><samp class=”option”>enable-interintra-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interintra compound. Default is true.

</dd>

<dt><samp class=”option”>enable-smooth-interintra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable smooth interintra mode. Default is true.

</dd>

<dt><samp class=”option”>aom-params</samp></dt>

<dd> Set libaom options using a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated

by ":". For a list of supported options, see <code class=”command”>aomenc --help</code> under the

section "AV1 Specific Options".

For example to specify libaom encoding options with <samp class=”option”>-aom-params</samp>:

<pre class=”example-preformatted”>ffmpeg -i input -c:v libaom-av1 -b:v 500K -aom-params tune=psnr:enable-tpl-model=1 output.mp4

</pre>

</dd> </dl>

 <h3 class=”section”>16.9 liboapv# TOC</h3>

Advanced Professional Video codec encoder wrapper. This encoder requires the presence of the liboapv headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-liboapv</samp>.   Many liboapv encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. The apv project website is at <a class="url" href="https://github.com/AcademySoftwareFoundation/openapv">https://github.com/AcademySoftwareFou
  <h4 class="subsection">16.9.1 Options# TOC</h4>

The following options are supported by the liboapv wrapper.   To get a more extensive documentation of the liboapv options, consult the liboapv documentation. <dl class="table"> <dt><samp class="option">preset</samp></dt> <dd> Set the quality-speed tradeoff [fastest, fast, medium, slow, placebo, default] </dd> <dt><samp class="option">qp</samp></dt> <dd> Set the quantization parameter value for CQP rate control mode. </dd> <dt><samp class="option">oapv-params (parse_apv_params)</samp></dt> <dd> Set liboapvenc options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ":". See the liboapv encoder user guide for a list of accepted parameters. </dd> </dl>

 <h3 class="section">16.10 libsvtav1# TOC</h3>

SVT-AV1 encoder wrapper. Requires the presence of the SVT-AV1 headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libsvtav1</code>. <h4 class="subsection">16.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Set the encoding profile. <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt>
<dt>‘<samp class="samp">professional</samp>’</dt>
</dl>

</dd> <dt><samp class="option">level</samp></dt> <dd> Set the operating point level. For example: ’4.0’ </dd> <dt><samp class="option">hielevel</samp></dt> <dd> Set the Hi-

erarchical prediction levels. <dl class="table"> <dt>‘<samp class="samp">3level</samp>’</dt> <dt>‘<samp class="samp">4level</samp>’</d <dd> This is the default. </dd> </dl>

</dd> <dt><samp class="option">tier</samp></dt> <dd> Set the operating point tier. <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dd> This is the default. </dd> <dt>‘<samp class="samp">high</samp>’</dt> </dl>

</dd> <dt><samp class="option">qmax</samp></dt> <dd> Set the maximum quantizer to use when using a bitrate mode. </dd> <dt><samp class="option">qmin</samp></dt> <dd> Set the minimum quantizer to use when using a bitrate mode. </dd> <dt><samp class="option">crf</samp></dt> <dd> Constant rate factor value used in crf rate control mode (0-63). </dd> <dt><samp class="option">qp</samp></dt> <dd> Set the quantizer used in cqp rate control mode (0-63). </dd> <dt><samp class="option">sc_detection</samp></dt> <dd> Enable scene change detection. </dd> <dt><samp class="option">la_depth</samp></dt> <dd> Set number of frames to look ahead (0-120). </dd> <dt><samp class="option">preset</samp></dt> <dd> Set the quality-speed tradeoff, in the range 0 to 13. Higher values are faster but lower quality. </dd> <dt><samp class="option">tile_rows</samp></dt> <dd> Set log2 of the number of rows of tiles to use (0-6). </dd> <dt><samp class="option">tile_columns</samp></dt> <dd> Set log2 of the number of columns of tiles to use (0-4). </dd> <dt><samp class="option">svtav1-params</samp></dt> <dd> Set SVT-AV1 options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ":". See the SVT-AV1 encoder user guide for a list of accepted parameters. </dd> </dl>

 <h3 class="section">16.11 libsvtjpegxs# TOC</h3>

SVT-JPEG-XS encoder wrapper. Requires the presence of the SVT-JPEG-XS headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libsvtjpegxs</code>. <h4 class="subsection">16.11.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">decomp_v</samp></dt> <dd> Set vertical decomposition level </dd> <dt><samp

class=”option”>decomp_h</samp></dt> <dd> Set horizontal decomposition level </dd> <dt><samp class=”option”>quantization</samp></dt> <dd> Set the quantization algorithm. <dl class=”table”> <dt>‘<samp class=”samp”>deadzone</samp>’</dt> <dt>‘<samp class=”samp”>uniform</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>coding-signs</samp></dt> <dd> Enable Signs handling strategy <dl class=”table”> <dt>‘<samp class=”samp”>disable</samp>’</dt> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dt>‘<samp class=”samp”>full</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>coding-sigf</samp></dt> <dd> Enable Significance coding </dd> <dt><samp class=”option”>coding-vpred</samp></dt> <dd> Enable Vertical Prediction coding <dl class=”table”> <dt>‘<samp class=”samp”>disable</samp>’</dt> <dt>‘<samp class=”samp”>no_residuals</samp>’</dt> <dt>‘<samp class=”samp”>no_coeffs</samp>’</dt> </dl>

</dd> </dl>

 <h3 class=”section”>16.12 libjxl# TOC</h3>

libjxl JPEG XL encoder wrapper. Requires the presence of the libjxl headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libjxl</code>. <h4 class=”subsection”>16.12.1 Options# TOC</h4>

The libjxl wrapper supports the following options: <dl class=”table”> <dt><samp class=”option”>distance</samp></dt> <dd> Set the target Butteraugli distance. This is a quality setting: lower distance yields higher quality, with distance=1.0 roughly comparable to libjpeg Quality 90 for photographic content. Setting distance=0.0 yields true lossless encoding. Valid values range between 0.0 and 15.0, and sane values rarely exceed 5.0. Setting distance=0.1 usually attains transparency for most input. The default is 1.0. </dd> <dt><samp class=”option”>effort</samp></dt> <dd> Set the encoding effort used. Higher effort values produce more consistent quality and usually produces a better quality/bpp curve, at the cost of more CPU time required. Valid values range from 1 to 9, and the default is 7. </dd> <dt><samp class=”option”>modular</samp></dt> <dd> Force the encoder to use Modular mode instead of choosing automatically. The default is to use VarDCT for lossy encoding and Modular for lossless. VarDCT is generally superior to Modular for lossy encoding but does not support lossless encoding. </dd> </dl>

 <h3 class="section">16.13 libkvazaar# TOC</h3>

Kvazaar H.265/HEVC encoder. Requires the presence of the libkvazaar headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libkvazaar</samp>. <h4 class="subsection">16.13.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set target video bitrate in bit/s and enable rate control. </dd> <dt><samp class="option">kvazaar-params</samp></dt> <dd> Set kvazaar parameters as a list of <var class="var">name</var>=<var class="var">value</var> pairs separated by commas (,). See kvazaar documentation for a list of options. </dd> </dl>

 <h3 class="section">16.14 libopenh264# TOC</h3>

Cisco libopenh264 H.264/MPEG-4 AVC encoder wrapper. This encoder requires the presence of the libopenh264 headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopenh264</code>. The library is detected using <code class="command">pkg-config</code>. For more information about the library see http://www.openh264.org. <h4 class="subsection">16.14.1 Options# TOC</h4>

The following FFmpeg global options affect the configurations of the libopenh264 encoder. <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set the bitrate (as a number of bits per second). </dd> <dt><samp class="option">g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class="option">maxrate</samp></dt> <dd> Set the max bitrate (as a number of bits per second). </dd> <dt><samp class="option">flags +global_header</samp></dt> <dd> Set global header in the bitstream. </dd> <dt><samp class="option">slices</samp></dt> <dd> Set the number of slices, used in parallelized encoding. Default value is 0. This is only used when <samp class="option">slice_mode</samp> is set to ‘<samp

class=”samp”>fixed</samp>’. </dd> <dt><samp class=”option”>loopfilter</samp></dt> <dd> Enable loop filter, if set to 1 (automatically enabled). To disable set a value of 0. </dd> <dt><samp class=”option”>profile</samp></dt> <dd> Set profile restrictions. If set to the value of ‘<samp class=”samp”>main</samp>’ enable CABAC (set the <code class=”code”>SEncParamExt.iEntropyCodingModeFlag</code> flag to 1). </dd> <dt><samp class=”option”>max_nal_size</samp></dt> <dd> Set maximum NAL size in bytes. </dd> <dt><samp class=”option”>allow_skip_frames</samp></dt> <dd> Allow skipping frames to hit the target bitrate if set to 1. </dd> </dl>

 <h3 class=”section”>16.15 libtheora# TOC</h3>

libtheora Theora encoder wrapper. Requires the presence of the libtheora headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libtheora</code>. For more information about the libtheora project see http://www.theora.org/. <h4 class=”subsection”>16.15.1 Options# TOC</h4>

The following global options are mapped to internal libtheora options which affect the quality and the bitrate of the encoded stream. <dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dd> Set the video bitrate in bit/s for CBR (Constant Bit Rate) mode. In case VBR (Variable Bit Rate) mode is enabled this option is ignored. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Used to enable constant quality mode (VBR) encoding through the <samp class=”option”>qscale</samp> flag, and to enable the <code class=”code”>pass1</code> and <code class=”code”>pass2</code> modes. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class=”option”>global_quality</samp></dt> <dd> Set the global quality as an integer in lambda units. Only relevant when VBR mode is enabled with <code class=”code”>flags +qscale</code>. The value is converted to QP units by dividing it by <code class=”code”>FF_QP2LAMBDA</code>, clipped in the [0 - 10] range, and then multiplied by 6.3 to get a value in the native libtheora range [0-63]. A higher value corresponds to a higher quality. </dd> <dt><samp class=”option”>q</samp></dt> <dd> Enable VBR mode when set to a non-negative value, and set constant quality value as a double floating point value in QP units. The value is clipped in the [0-10] range, and then multiplied by 6.3 to get a value in the native

libtheora range [0-63]. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> </dl>

 <h4 class="subsection">16.15.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set maximum constant quality (VBR) encoding with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i INPUT -codec:v libtheora -q:v 10 OUTPUT.ogg </pre>

Use <code class="command">ffmpeg</code> to convert a CBR 1000 kbps Theora video stream: <pre class="example-preformatted">ffmpeg -i INPUT -codec:v libtheora -b:v 1000k OUTPUT.ogg </pre>

 <h3 class="section">16.16 libvpx# TOC</h3>

VP8/VP9 format supported through libvpx. Requires the presence of the libvpx headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libvpx</code>. <h4 class="subsection">16.16.1 Options# TOC</h4>

The following options are supported by the libvpx wrapper. The <code class="command">vpxenc</code>-equivalent options or values are listed in parentheses for easy migration. To reduce the duplication of documentation, only the private options and some others requiring special attention are documented here. For the documentation of the undocumented generic options, see the Codec Options chapter. To get more documentation of the libvpx options, invoke the command <code class="command">ffmpeg -h encoder=libvpx</code>, <code class="command">ffmpeg -h encoder=libvpx-vp9</code> or <code class="command">vpxenc --help</code>. Further information is available in the libvpx API documentation. <dl class="table"> <dt><samp class="option">b (target-bitrate)</samp></dt> <dd> Set bitrate in bits/s. Note that FFmpeg’s <samp class="option">b</samp> option is expressed in bits/s, while <code class="command">vpxenc</code>’s <samp class="option">target-bitrate</samp> is in kilobits/s. </dd> <dt><samp class="option">g (kf-max-

dist)</samp></dt> <dt><samp class="option">keyint_min (kf-min-dist)</samp></dt> <dt><samp class="option">qmin (min-q)</samp></dt> <dd> Minimum (Best Quality) Quantizer. </dd> <dt><samp class="option">qmax (max-q)</samp></dt> <dd> Maximum (Worst Quality) Quantizer. Can be changed per-frame. </dd> <dt><samp class="option">bufsize (buf-sz, buf-optimal-sz)</samp></dt> <dd> Set ratecontrol buffer size (in bits). Note <code class="command">vpxenc</code>’s options are specified in milliseconds, the libvpx wrapper converts this value as follows: <code class="code">buf-sz = bufsize * 1000 / bitrate</code>, <code class="code">buf-optimal-sz = bufsize * 1000 / bitrate * 5 / 6</code>. </dd> <dt><samp class="option">rc_init_occupancy (buf-initial-sz)</samp></dt> <dd> Set number of bits which should be loaded into the rc buffer before decoding starts. Note <code class="command">vpxenc</code>’s option is specified in milliseconds, the libvpx wrapper converts this value as follows: <code class="code">rc_init_occupancy * 1000 / bitrate</code>. </dd> <dt><samp class="option">undershoot-pct</samp></dt> <dd> Set datarate undershoot (min) percentage of the target bitrate. </dd> <dt><samp class="option">overshoot-pct</samp></dt> <dd> Set datarate overshoot (max) percentage of the target bitrate. </dd> <dt><samp class="option">skip_threshold (drop-frame)</samp></dt> <dt><samp class="option">qcomp (bias-pct)</samp></dt> <dt><samp class="option">maxrate (maxsection-pct)</samp></dt> <dd> Set GOP max bitrate in bits/s. Note <code class="command">vpxenc</code>’s option is specified as a percentage of the target bitrate, the libvpx wrapper converts this value as follows: <code class="code">(maxrate * 100 / bitrate)</code>. </dd> <dt><samp class="option">minrate (minsection-pct)</samp></dt> <dd> Set GOP min bitrate in bits/s. Note <code class="command">vpxenc</code>’s option is specified as a percentage of the target bitrate, the libvpx wrapper converts this value as follows: <code class="code">(minrate * 100 / bitrate)</code>. </dd> <dt><samp class="option">minrate, maxrate, b end-usage=cbr</samp></dt> <dd> <code class="code">(minrate == maxrate == bitrate)</code>. </dd> <dt><samp class="option">crf (end-usage=cq, cq-level)</samp></dt> <dt><samp class="option">tune (tune)</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">psnr (psnr)</samp>’</dt> <dt>‘<samp class="samp">ssim (ssim)</samp>’</dt> </dl>

</dd> <dt><samp class="option">quality, deadline (deadline)</samp></dt>
<dd><dl class="table"> <dt>‘<samp class="samp">best</samp>’</dt>
<dd> Use best quality deadline. Poorly named and quite slow, this option should be avoided as it may give worse quality output than good.
</dd> <dt>‘<samp class="samp">good</samp>’</dt>
<dd> Use good quality deadline. This is a good trade-off between speed and quality when used with the <samp class="option">cpu-used</samp> option.
</dd> <dt>‘<samp class="samp">realtime</samp>’</dt>
<dd> Use realtime quality deadline. </dd> </dl>

</dd> <dt><samp class="option">speed, cpu-used (cpu-used)</samp></dt> <dd> Set quality/speed ratio modifier. Higher values speed up the encode at the cost of quality. </dd>
<dt><samp class="option">nr (noise-sensitivity)</samp></dt>
<dt><samp class="option">static-thresh</samp></dt> <dd> Set a change threshold on blocks below which they will be skipped by the encoder.
 </dd> <dt><samp class="option">slices (token-parts)</samp></dt> <dd> Note that FFmpeg’s <samp class="option">slices</samp> option gives the total number of partitions, while <code class="command">vpxenc</code>’s <samp class="option">token-parts</samp> is given as <code class="code">log2(partitions)</code>.
 </dd> <dt><samp class="option">max-intra-rate</samp></dt>
<dd> Set maximum I-frame bitrate as a percentage of the target bitrate. A value of 0 means unlimited. </dd> <dt><samp class="option">force_key_frames</samp></dt> <dd> <code class="code">VPX_EFLAG_FORCE_
 </dd> <dt><samp class="option">Alternate reference frame related</samp></dt> <dd><dl class="table"> <dt><samp class="option">auto-alt-ref</samp></dt> <dd> Enable use of alternate reference frames (2-pass only). Values greater than 1 enable multi-layer alternate reference frames (VP9 only). </dd> <dt><samp class="option">arnr-maxframes</samp></dt> <dd> Set altref noise reduction max frame count. </dd> <dt><samp class="option">arnr-type</samp></dt>
<dd> Set altref noise reduction filter type: backward, forward, centered.
</dd> <dt><samp class="option">arnr-strength</samp></dt>
<dd> Set altref noise reduction filter strength. </dd> <dt><samp class="option">rc-lookahead, lag-in-frames (lag-in-frames)</samp></dt> <dd> Set number of frames to look ahead for frametype and ratecontrol. </dd> <dt><samp class="option">min-gf-interval</samp></dt> <dd> Set minimum golden/alternate reference frame interval (VP9 only). </dd> </dl>

</dd> <dt><samp class="option">error-resilient</samp></dt> <dd> Enable error resiliency features. </dd> <dt><samp class="option">sharpness <var class="var">integer</var></samp></dt> <dd> Increase sharpness at the expense of lower PSNR. The valid range is [0, 7]. </dd>
<dt><samp class="option">ts-parameters</samp></dt> <dd> Sets the temporal scalability configuration using a :-separated list of key=value

pairs. For example, to specify temporal scalability parameters with <code class=”code”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v libvpx -ts-parameters ts_number_layers=3:\ ts_target_bitrate=250,500,1000:ts_rate_decimator=4,2,1:\ ts_periodicity=4:ts_layer_id=0,2,1,2:ts_layering_mode=3 OUTPUT </pre> Below is a brief explanation of each of the parameters, please refer to <code class=”code”>struct vpx_codec_enc_cfg</code> in <code class=”code”>vpx/vpx_encoder.h</code> for more details. <dl class=”table”> <dt><samp class=”option”>ts_number_layers</samp></dt> <dd> Number of temporal coding layers. </dd> <dt><samp class=”option”>ts_target_bitrate</samp></dt> <dd> Target bitrate for each temporal layer (in kbps). (bitrate should be inclusive of the lower temporal layer). </dd> <dt><samp class=”option”>ts_rate_decimator</samp></dt> <dd> Frame rate decimation factor for each temporal layer. </dd> <dt><samp class=”option”>ts_periodicity</samp></dt> <dd> Length of the sequence defining frame temporal layer membership. </dd> <dt><samp class=”option”>ts_layer_id</samp></dt> <dd> Template defining the membership of frames to temporal layers. </dd> <dt><samp class=”option”>ts_layering_mode</samp></dt> <dd> (optional) Selecting the temporal structure from a set of pre-defined temporal layering modes. Currently supports the following options. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dd> No temporal layering flags are provided internally, relies on flags being passed in using <code class=”code”>metadata</code> field in <code class=”code”>AVFrame</code> with following keys. <dl class=”table”> <dt><samp class=”option”>vp8-flags</samp></dt> <dd> Sets the flags passed into the encoder to indicate the referencing scheme for the current frame. Refer to function <code class=”code”>vpx_codec_encode</code> in <code class=”code”>vpx/vpx_encoder.h</code> for more details. </dd> <dt><samp class=”option”>temporal_id</samp></dt> <dd> Explicitly sets the temporal id of the current frame to encode. </dd> </dl> </dd> <dt><samp class=”option”>2</samp></dt> <dd> Two temporal layers. 0-1... </dd> <dt><samp class=”option”>3</samp></dt> <dd> Three temporal layers. 0-2-1-2...; with single reference frame. </dd> <dt><samp class=”option”>4</samp></dt> <dd> Same as option "3", except there is a dependency between the two temporal layer 2 frames within the temporal period. </dd> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>VP8-specific options</samp></dt> <dd><dl class=”table”> <dt><samp class=”option”>screen-content-mode</samp></dt> <dd> Screen content mode, one of: 0 (off), 1 (screen), 2 (screen with more aggressive rate control). </dd> </dl>

</dd> <dt><samp class=”option”>VP9-specific options</samp></dt> <dd><dl class=”table”> <dt><samp class=”option”>lossless</samp></dt> <dd> Enable lossless mode. </dd> <dt><samp class=”option”>tile-

columns</samp></dt> <dd> Set number of tile columns to use. Note this is given as <code class="code">log2(tile_columns)</code>. For example, 8 tile columns would be requested by setting the <samp class="option">tile-columns</samp> option to 3. </dd> <dt><samp class="option">tile-rows</samp></dt> <dd> Set number of tile rows to use. Note this is given as <code class="code">log2(tile_rows)</code>. For example, 4 tile rows would be requested by setting the <samp class="option">tile-rows</samp> option to 2. </dd> <dt><samp class="option">frame-parallel</samp></dt> <dd> Enable frame parallel decodability features. </dd> <dt><samp class="option">aq-mode</samp></dt> <dd> Set adaptive quantization mode (0: off (default), 1: variance 2: complexity, 3: cyclic refresh, 4: equator360). </dd> <dt><samp class="option">colorspace color-space</samp></dt> <dd> Set input color space. The VP9 bitstream supports signaling the following colorspaces: <dl class="table"> <dt><samp class="option">‘<samp class="samp">rgb</samp>’ sRGB</samp></dt> <dt><samp class="option">‘<samp class="samp">bt709</samp>’ bt709</samp></dt> <dt><samp class="option">‘<samp class="samp">unspecified</samp>’ unknown</samp></dt> <dt><samp class="option">‘<samp class="samp">bt470bg</samp>’ bt601</samp></dt> <dt><samp class="option">‘<samp class="samp">smpte170m</samp>’ smpte170</samp></dt> <dt><samp class="option">‘<samp class="samp">smpte240m</samp>’ smpte240</samp></dt> <dt><samp class="option">‘<samp class="samp">bt2020_ncl</samp>’ bt2020</samp></dt> </dl> </dd> <dt><samp class="option">row-mt <var class="var">boolean</var></samp></dt> <dd> Enable row based multi-threading. </dd> <dt><samp class="option">tune-content</samp></dt> <dd> Set content type: default (0), screen (1), film (2). </dd> <dt><samp class="option">corpus-complexity</samp></dt> <dd> Corpus VBR mode is a variant of standard VBR where the complexity distribution midpoint is passed in rather than calculated for a specific clip or chunk. The valid range is [0, 10000]. 0 (default) uses standard VBR. </dd> <dt><samp class="option">enable-tpl <var class="var">boolean</var></samp></dt> <dd> Enable temporal dependency model. </dd> <dt><samp class="option">ref-frame-config</samp></dt> <dd> Using per-frame metadata, set members of the structure <code class="code">vpx_svc_ref_frame_config_t</code> in <code class="code">vpx/vp8cx.h</code> to fine-control referencing schemes and frame buffer management.
Use a :-separated list of key=value pairs. For example, <pre class="example-preformatted">av_dict_set(&av_frame->metadata, "ref-frame-config", \ "rfc_update_buffer_slot=7:rfc_lst_fb_idx=0:rfc_gld_fb_idx=1:rfc_alt_fb_idx=2:rfc
</pre> <dl class="table"> <dt><samp class="option">rfc_update_buffer_slot</samp></dt> <dd> Indicates the buffer slot number to update </dd> <dt><samp class="option">rfc_update_last</samp></dt> <dd> Indicates whether

to update the LAST frame </dd> <dt><samp class="option">rfc_update_golden</samp></dt>
<dd> Indicates whether to update GOLDEN frame </dd>
<dt><samp class="option">rfc_update_alt_ref</samp></dt> <dd> Indicates
whether to update ALT_REF frame </dd> <dt><samp class="option">rfc_lst_fb_idx</samp></dt
<dd> LAST frame buffer index </dd> <dt><samp class="option">rfc_gld_fb_idx</samp></dt
<dd> GOLDEN frame buffer index </dd> <dt><samp class="option">rfc_alt_fb_idx</samp><
<dd> ALT_REF frame buffer index </dd> <dt><samp
class="option">rfc_reference_last</samp></dt> <dd> Indicates
whether to reference LAST frame </dd> <dt><samp class="option">rfc_reference_golden</samp><
<dd> Indicates whether to reference GOLDEN frame </dd>
<dt><samp class="option">rfc_reference_alt_ref</samp></dt> <dd> Indicates
whether to reference ALT_REF frame </dd> <dt><samp class="option">rfc_reference_duration</sa
<dd> Indicates frame duration </dd> </dl> </dd> </dl>

</dd> </dl>

For more information about libvpx see: http://ww
  <h3 class="section">16.17 libvvenc<span
class="pull-right"><a class="anchor hidden-xs" href="#libvvenc" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-libvvenc"
aria-hidden="true">TOC</h3>

VVenC H.266/VVC encoder wrapper. This encoder requires
the presence of the libvvenc headers and library during configuration. You
need to explicitly configure the build with <samp class="option">--enable-
libvvenc</samp>. The VVenC project website is at <a class="url"
href="https://github.com/fraunhoferhhi/vvenc">https://github.com/fraunhoferhhi/vvenc.
  <h4 class="subsection">16.17.1
Supported Pixel Formats<a class="anchor
hidden-xs" href="#Supported-Pixel-Formats" aria-hidden="true">#
<a class="anchor hidden-xs"href="#toc-Supported-Pixel-Formats" aria-
hidden="true">TOC</h4>

VVenC supports only 10-bit color spaces as input. But the internal
(encoded) bit depth can be set to 8-bit or 10-bit at runtime. <a
name="Options-42"> <h4 class="subsection">16.17.2 Options<span
class="pull-right"><a class="anchor hidden-xs" href="#Options-42" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Options-42"
aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Sets
target video bitrate. </dd> <dt><samp class="option">g</samp></dt>
<dd> Set the GOP size. Currently support for g=1 (Intra only) or
default. </dd> <dt><samp class="option">preset</samp></dt>
<dd> Set the VVenC preset. </dd> <dt><samp class="option">levelidc</samp></dt>
<dd> Set level idc. </dd> <dt><samp class="option">tier</samp></dt>
<dd> Set vvc tier. </dd> <dt><samp class="option">qp</samp></dt>
<dd> Set constant quantization parameter. </dd> <dt><samp

class=”option”>subopt <var class=”var”>boolean</var></samp></dt> <dd> Set subjective (perceptually motivated) optimization. Default is 1 (on). </dd> <dt><samp class=”option”>bitdepth8 <var class=”var”>boolean</var></samp></dt> <dd> Set 8bit coding mode instead of using 10bit. Default is 0 (off). </dd> <dt><samp class=”option”>period</samp></dt> <dd> set (intra) refresh period in seconds. </dd> <dt><samp class=”option”>vvenc-params</samp></dt> <dd> Set vvenc options using a list of <var class=”var”>key</var>=<var class=”var”>value</var> couples separated by ":". See <code class=”command”>vvencapp --fullhelp</code> or <code class=”command”>vvencFFapp --fullhelp</code> for a list of options. For example, the options might be provided as: <pre class=”example-preformatted”>intraperiod=64:decodingrefreshtype=idr:poc0idr=1:i
</pre>

For example the encoding options might be provided with <samp class=”option”>-vvenc-params</samp>:   <pre class=”example-preformatted”>ffmpeg -i input -c:v libvvenc -b 1M -vvenc-params intraperiod=64:decodingrefreshtype=idr:poc0idr=1:internalbitdepth=8 output.mp4 </pre>

</dd> </dl>

 <h3 class=”section”>16.18 libwebp# TOC</h3>

libwebp WebP Image encoder wrapper libwebp is Google’s official encoder for WebP images. It can encode in either lossy or lossless mode. Lossy images are essentially a wrapper around a VP8 frame. Lossless images are a separate codec developed by Google. <h4 class=”subsection”>16.18.1 Pixel Format# TOC</h4>

Currently, libwebp only supports YUV420 for lossy and RGB for lossless due to limitations of the format and libwebp. Alpha is supported for either mode. Because of API limitations, if RGB is passed in when encoding lossy or YUV is passed in for encoding lossless, the pixel format will automatically be converted using functions from libwebp. This is not ideal and is done only for convenience. <h4 class=”subsection”>16.18.2 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-lossless <var class=”var”>boolean</var></samp></dt> <dd> Enables/Disables use of lossless mode. Default is 0. </dd>

<dt><samp class=”option”>-compression_level <var class=”var”>integer</var></samp></dt> <dd> For lossy, this is a quality/speed tradeoff. Higher values give better quality for a given size at the cost of increased encoding time. For lossless, this is a size/speed tradeoff. Higher values give smaller size at the cost of increased encoding time. More specifically, it controls the number of extra algorithms and compression tools used, and varies the combination of these tools. This maps to the <var class=”var”>method</var> option in libwebp. The valid range is 0 to 6. Default is 4. </dd> <dt><samp class=”option”>-quality <var class=”var”>float</var></samp></dt> <dd> For lossy encoding, this controls image quality. For lossless encoding, this controls the effort and time spent in compression. Range is 0 to 100. Default is 75. </dd> <dt><samp class=”option”>-preset <var class=”var”>type</var></samp></dt> <dd> Configuration preset. This does some automatic settings based on the general type of the image. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Do not use a preset. </dd> <dt><samp class=”option”>default</samp></dt> <dd> Use the encoder default. </dd> <dt><samp class=”option”>picture</samp></dt> <dd> Digital picture, like portrait, inner shot </dd> <dt><samp class=”option”>photo</samp></dt> <dd> Outdoor photograph, with natural lighting </dd> <dt><samp class=”option”>drawing</samp></dt> <dd> Hand or line drawing, with high-contrast details </dd> <dt><samp class=”option”>icon</samp></dt> <dd> Small-sized colorful images </dd> <dt><samp class=”option”>text</samp></dt> <dd> Text-like </dd> </dl>

</dd> </dl>

 <h3 class=”section”>16.19 libastcenc# TOC</h3>

ARM ASTC Encoder wrapper. This encoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, which provides high-quality GPU texture compression to the ASTC format, installed as <code class=”code”>astcenc/astcenc.h</code> next to a library that <code class=”code”>-lastcenc</code> finds. Pass <code class=”code”>--enable-version3</code> together with <code class=”code”>--enable-libastcenc</code> to configure. The encoder supports both LDR (8-bit RGBA/RGB) and HDR (float/half RGBA) input. When float or half input is detected with an LDR profile, the profile is automatically promoted to HDR RGB with LDR alpha (<code class=”code”>HDR_RGB_LDR_A</code>); an explicitly selected HDR profile is kept. The raw bitstream can be packaged in the <samp class=”file”>.astc</samp> container (16-byte header + compressed blocks) or in the Khronos Texture 1.0 container (<samp class=”file”>.ktx</samp>). This implementation writes LDR 2D textures only through the <samp

class=”file”>.ktx</samp> muxer: HDR ASTC and 3D block footprints are not supported by the muxer and <samp class=”file”>.astc</samp> must be used for them instead. The KTX 1.0 linear ASTC GL enums do not imply LDR either, so a linear <samp class=”file”>.ktx</samp> texture may still hold HDR endpoints; such a texture is decoded with HDR precision by default. The <samp class=”file”>.ktx</samp> muxer records its row order in a <code class=”code”>KTXorientation</code> key. <h4 class=”subsection”>16.19.1 Pixel Format# TOC</h4>

The encoder accepts the following pixel formats: <ul class=”itemize mark-bullet”> <code class=”code”>rgba</code> – 8-bit packed RGBA (LDR). <code class=”code”>rgb24</code> – 8-bit packed RGB (LDR, opaque alpha synthesized). <code class=”code”>rgbaf16</code> – 16-bit floating-point RGBA (HDR). <code class=”code”>rgbaf32</code> – 32-bit floating-point RGBA (HDR). <code class=”code”>gbrapf32</code> – planar 32-bit floating-point GBR+Alpha (HDR).

swscale cannot convert to the packed floating-point formats (<code class=”code”>rgbaf16</code>, <code class=”code”>rgbaf32</code>); to encode ordinary image formats as HDR, produce a planar floating-point source instead, for example with <code class=”code”>-vf format=gbrapf32</code>. The packed formats are accepted when they are supplied directly, such as from a decoder or a custom filter graph. <h4 class=”subsection”>16.19.2 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>block_size <var class=”var”>string</var></samp></dt> <dd> ASTC block size specified as <var class=”var”>W</var>x<var class=”var”>H</var>[<code class=”code”>x</code><var class=”var”>D</var>]. When the third dimension is omitted it defaults to 1 (2D block). Valid 2D: 4x4, 5x4, 5x5, 6x5, 6x6, 8x5, 8x6, 8x8, 10x5, 10x6, 10x8, 10x10, 12x10, 12x12. Valid 3D: 3x3x3, 4x3x3, 4x4x3, 4x4x4, 5x4x4, 5x5x4, 5x5x5, 6x5x5, 6x6x5, 6x6x6. Default is <code class=”code”>8x8</code>. </dd> <dt><samp class=”option”>quality <var class=”var”>float</var></samp></dt> <dd> Compression quality from 0 (fastest) to 100 (slowest, exhaustive search). Default is <code class=”code”>60</code>. </dd> <dt><samp class=”option”>profile <var class=”var”>int</var></samp></dt> <dd> Color profile. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>ldr</samp>’</dt> <dd> Linear LDR. </dd> <dt>‘<samp class=”samp”>ldr-srgb</samp>’</dt> <dd> sRGB LDR (default). </dd> <dt>‘<samp

class=”samp”>hdr-ldr-a</samp>’</dt> <dd> HDR RGB channels with LDR alpha channel. </dd> <dt>‘<samp class=”samp”>hdr</samp>’</dt> <dd> Full HDR. </dd> </dl>

When float/half input is detected with an LDR profile, the profile is automatically promoted to <code class=”code”>hdr-ldr-a</code>. Conversely, selecting an HDR profile with 8-bit input is rejected. This option overrides <code class=”code”>AVCodecContext.profile</code>. When it is not set, the profile is taken from <code class=”code”>AVCodecContext.profile</code> (<code class=”code”>AV_PROFILE_ASTC_*</code>), where <code class=”code”>AV_PROFILE_UNKNOWN</code> selects <code class=”code”>ldr-srgb</code> as before. </dd> <dt><samp class=”option”>perceptual <var class=”var”>boolean</var></samp></dt> <dd> Use the perceptual (PSNR-weighted) error metric instead of peak-SNR. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>alpha_weight <var class=”var”>boolean</var></samp></dt> <dd> Enable per-texel alpha weighting. Default is <code class=”code”>1</code>. </dd> </dl>

 <h3 class=”section”>16.20 libx264, libx264rgb# TOC</h3

x264 H.264/MPEG-4 AVC encoder wrapper. This encoder requires the presence of the libx264 headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libx264</code>. libx264 supports an impressive number of features, including 8x8 and 4x4 adaptive spatial transform, adaptive B-frame placement, CAVLC/CABAC entropy coding, interlacing (MBAFF), lossless mode, psy optimizations for detail retention (adaptive quantization, psy-RD, psy-trellis). Many libx264 encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. Additionally the <samp class=”option”>x264opts</samp> and <samp class=”option”>x264-params</samp> private options allows one to pass a list of key=value tuples as accepted by the libx264 <code class=”code”>x264_param_parse</code> function. The x264 project website is at http://www.videolan.org/developers/x264.html The libx264rgb encoder is the same as libx264, except it accepts packed RGB pixel formats as input instead of YUV. <h4 class=”subsection”>16.20.1 Supported Pixel Formats# TOC</h4>

x264 supports 8- to 10-bit color spaces. The exact bit depth is controlled at x264’s configure time. <h4 class="subsection">16.20.2 Options# TOC</h4>

The following options are supported by the libx264 wrapper. The

<code class="command">x264</code>-equivalent options or values are listed in parentheses

for easy migration.

To reduce the duplication of documentation, only the private options

and some others requiring special attention are documented here. For

the documentation of the undocumented generic options, see

the Codec Options chapter.

To get a more accurate and extensive documentation of the libx264

options, invoke the command <code class="command">x264 --fullhelp</code> or consult

the libx264 documentation.

In the list below, note that the <code class="command">x264</code> option name is shown

in parentheses after the libavcodec corresponding name, in case there

is a direct mapping.

<dl class="table">

<dt><samp class="option">b (bitrate)</samp></dt>

<dd> Set bitrate in bits/s. Note that FFmpeg’s <samp class="option">b</samp> option is

expressed in bits/s, while <code class="command">x264</code>’s <samp class="option">bitrate</samp> is in

kilobits/s.

</dd>

<dt><samp class=”option”>bf (bframes)</samp></dt>

<dd> Number of B-frames between I and P-frames

</dd>

<dt><samp class=”option”>g (keyint)</samp></dt>

<dd> Maximum GOP size

</dd>

<dt><samp class=”option”>qmin (qpmin)</samp></dt>

<dd> Minimum quantizer scale

</dd>

<dt><samp class=”option”>qmax (qpmax)</samp></dt>

<dd> Maximum quantizer scale

</dd>

<dt><samp class=”option”>qdiff (qpstep)</samp></dt>

<dd> Maximum difference between quantizer scales

</dd>

<dt><samp class=”option”>qblur (qblur)</samp></dt>

<dd> Quantizer curve blur

</dd>

<dt><samp class=”option”>qcomp (qcomp)</samp></dt>

<dd> Quantizer curve compression factor

</dd>

<dt><samp class=”option”>refs (ref)</samp></dt>

<dd> Number of reference frames each P-frame can use. The range is <var class=”var”>0-16</var>.

</dd>

<dt><samp class=”option”>level (level)</samp></dt>

<dd> Set the <code class=”code”>x264_param_t.i_level_idc</code> value in case the value is

positive, it is ignored otherwise.

This value can be set using the <code class=”code”>AVCodecContext</code> API (e.g. by

setting the <code class=”code”>AVCodecContext</code> value directly), and is specified as

an integer mapped on a corresponding level (e.g. the value 31 maps

to H.264 level IDC "3.1", as defined in the <code class=”code”>x264_levels</code>

table). It is ignored when set to a non positive value.

Alternatively it can be set as a private option, overriding the value

set in <code class=”code”>AVCodecContext</code>, and in this case must be specified as

the level IDC identifier (e.g. "3.1"), as defined by H.264 Annex A.

</dd>

<dt><samp class=”option”>sc_threshold (scenecut)</samp></dt>

<dd> Sets the threshold for the scene change detection.

</dd>

<dt><samp class=”option”>trellis (trellis)</samp></dt>

<dd> Performs Trellis quantization to increase efficiency. Enabled by default.

</dd>

<dt><samp class=”option”>nr (nr)</samp></dt>

<dd> Noise reduction

</dd>

<dt><samp class="option">me_range (merange)</samp></dt>

<dd> Maximum range of the motion search in pixels.

</dd>

<dt><samp class="option">me_method (me)</samp></dt>

<dd> Set motion estimation method. Possible values in the decreasing order

of speed:

<dl class="table">

<dt>‘<samp class="samp">dia (dia)</samp>’</dt>

<dt>‘<samp class="samp">epzs (dia)</samp>’</dt>

<dd> Diamond search with radius 1 (fastest). ‘<samp class="samp">epzs</samp>’ is an alias for

‘<samp class="samp">dia</samp>’.

</dd>

<dt>‘<samp class="samp">hex (hex)</samp>’</dt>

<dd> Hexagonal search with radius 2.

</dd>

<dt>‘<samp class="samp">umh (umh)</samp>’</dt>

<dd> Uneven multi-hexagon search.

</dd>

<dt>‘<samp class="samp">esa (esa)</samp>’</dt>

<dd> Exhaustive search.

</dd>

<dt>‘<samp class="samp">tesa (tesa)</samp>’</dt>

<dd> Hadamard exhaustive search (slowest).

</dd>

</dl>

</dd> <dt><samp class=”option”>forced-idr</samp></dt> <dd> Normally, when forcing a I-frame type, the encoder can select any type of I-frame. This option forces it to choose an IDR-frame. </dd> <dt><samp class=”option”>subq (subme)</samp></dt> <dd> Sub-pixel motion estimation method. </dd> <dt><samp class=”option”>b_strategy (b-adapt)</samp></dt> <dd> Adaptive B-frame placement decision algorithm. Use only on first-pass. </dd> <dt><samp class=”option”>keyint_min (min-keyint)</samp></dt> <dd> Minimum GOP size. </dd> <dt><samp class=”option”>coder</samp></dt> <dd> Set entropy encoder. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>ac</samp>’</dt> <dd> Enable CABAC. </dd> <dt>‘<samp class=”samp”>vlc</samp>’</dt> <dd> Enable CAVLC and disable CABAC. It generates the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-cabac</samp> option. </dd> </dl>

</dd> <dt><samp class=”option”>cmp</samp></dt> <dd> Set full pixel motion estimation comparison algorithm. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>chroma</samp>’</dt> <dd> Enable chroma in motion estimation. </dd> <dt>‘<samp class=”samp”>sad</samp>’</dt> <dd> Ignore chroma in motion estimation. It generates the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-chroma-me</samp> option. </dd> </dl>

</dd> <dt><samp class=”option”>threads (threads)</samp></dt> <dd> Number of encoding threads. </dd> <dt><samp class=”option”>thread_type</samp></dt> <dd> Set multithreading technique. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>slice</samp>’</dt> <dd> Slice-based multithreading. It generates the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--sliced-threads</samp> option. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Frame-based multithreading. </dd> </dl>

</dd> <dt><samp class=”option”>flags</samp></dt> <dd> Set encoding flags. It can be used to disable closed GOP and enable open GOP by setting it to <code class=”code”>-cgop</code>. The result is similar to the behavior of <code class=”command”>x264</code>’s <samp class=”option”>--open-gop</samp> option. </dd> <dt><samp class=”option”>rc_init_occupancy (vbv-init)</samp></dt> <dd> Initial VBV buffer occupancy </dd> <dt><samp class=”option”>preset (preset)</samp></dt> <dd> Set the encoding preset. </dd> <dt><samp class=”option”>tune (tune)</samp></dt> <dd> Set tuning of the encoding params. </dd> <dt><samp class=”option”>profile (profile)</samp></dt> <dd> Set profile restric-

tions. </dd> <dt><samp class="option">fastfirstpass</samp></dt>
<dd> Enable fast settings when encoding first pass, when set to 1. When set to 0, it has the same effect of <code class="command">x264</code>’s <samp class="option">--slow-firstpass</samp> option. </dd>
<dt><samp class="option">crf (crf)</samp></dt>
<dd> Set the quality for constant quality mode. </dd>
<dt><samp class="option">crf_max (crf-max)</samp></dt>
<dd> In CRF mode, prevents VBV from lowering quality beyond this point. </dd> <dt><samp class="option">qp (qp)</samp></dt>
<dd> Set constant quantization rate control method parameter. </dd> <dt><samp class="option">aq-mode (aq-mode)</samp></dt> <dd> Set AQ method. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class="samp">variance (1)</samp>’</dt>
<dd> Variance AQ (complexity mask). </dd> <dt>‘<samp class="samp">autovariance (2)</samp>’</dt>
<dd> Auto-variance AQ (experimental). </dd> </dl>

</dd> <dt><samp class="option">aq-strength (aq-strength)</samp></dt> <dd> Set AQ strength, reduce blocking and blurring in flat and textured areas. </dd> <dt><samp class="option">psy</samp></dt> <dd> Use psychovisual optimizations when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-psy</samp> option. </dd> <dt><samp class="option">psy-rd (psy-rd)</samp></dt> <dd> Set strength of psychovisual optimization, in <var class="var">psy-rd</var>:<var class="var">psy-trellis</var> format. </dd> <dt><samp class="option">rc-lookahead (rc-lookahead)</samp></dt>
<dd> Set number of frames to look ahead for frametype and ratecontrol. </dd> <dt><samp class="option">weightb</samp></dt>
<dd> Enable weighted prediction for B-frames when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-weightb</samp> option. </dd>
<dt><samp class="option">weightp (weightp)</samp></dt>
<dd> Set weighted prediction method for P-frames. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class="samp">simple (1)</samp>’</d
<dd> Enable only weighted refs </dd> <dt>‘<samp class="samp">smart (2)</samp>’</dt>
<dd> Enable both weighted refs and duplicates </dd> </dl>

</dd> <dt><samp class="option">ssim (ssim)</samp></dt>
<dd> Enable calculation and printing SSIM stats after the encoding. </dd> <dt><samp class="option">intra-refresh (<em

class=”emph”>intra-refresh)</samp></dt> <dd> Enable the use of Periodic Intra Refresh instead of IDR frames when set to 1. </dd> <dt><samp class=”option”>avcintra-class (class)</samp></dt> <dd> Configure the encoder to generate AVC-Intra. Valid values are 50, 100 and 200 </dd> <dt><samp class=”option”>bluray-compat (bluray-compat)</samp></dt> <dd> Configure the encoder to be compatible with the bluray standard. It is a shorthand for setting "bluray-compat=1 force-cfr=1". </dd> <dt><samp class=”option”>b-bias (b-bias)</samp></dt> <dd> Set the influence on how often B-frames are used. </dd> <dt><samp class=”option”>b-pyramid (b-pyramid)</samp></dt> <dd> Set method for keeping of some B-frames as references. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>none (none)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class=”samp”>strict (strict)</samp>’</dt> <dd> Strictly hierarchical pyramid. </dd> <dt>‘<samp class=”samp”>normal (normal)</samp>’</dt> <dd> Non-strict (not Blu-ray compatible). </dd> </dl>

</dd> <dt><samp class=”option”>mixed-refs</samp></dt> <dd> Enable the use of one reference per partition, as opposed to one reference per macroblock when set to 1. When set to 0, it has the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-mixed-refs</samp> option. </dd> <dt><samp class=”option”>8x8dct</samp></dt> <dd> Enable adaptive spatial transform (high profile 8x8 transform) when set to 1. When set to 0, it has the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-8x8dct</samp> option. </dd> <dt><samp class=”option”>fast-pskip</samp></dt> <dd> Enable early SKIP detection on P-frames when set to 1. When set to 0, it has the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-fast-pskip</samp> option. </dd> <dt><samp class=”option”>aud (aud)</samp></dt> <dd> Enable use of access unit delimiters when set to 1. </dd> <dt><samp class=”option”>mbtree</samp></dt> <dd> Enable use macroblock tree ratecontrol when set to 1. When set to 0, it has the same effect as <code class=”command”>x264</code>’s <samp class=”option”>--no-mbtree</samp> option. </dd> <dt><samp class=”option”>deblock (deblock)</samp></dt> <dd> Set loop filter parameters, in <var class=”var”>alpha</var>:<var class=”var”>beta</var> form. </dd> <dt><samp class=”option”>cplxblur (cplxblur)</samp></
<dd> Set fluctuations reduction in QP (before curve compression). </dd> <dt><samp class=”option”>partitions (partitions)</samp></dt> <dd> Set partitions to consider as a comma-separated list of values. Pos-

sible values in the list: <dl class="table"> <dt>‘<samp class="samp">p8x8</samp>’</dt> <dd> 8x8 P-frame partition. </dd> <dt>‘<samp class="samp">p4x4</samp>’</dt> <dd> 4x4 P-frame partition. </dd> <dt>‘<samp class="samp">b8x8</samp>’</dt> <dd> 4x4 B-frame partition. </dd> <dt>‘<samp class="samp">i8x8</samp>’</dt> <dd> 8x8 I-frame partition. </dd> <dt>‘<samp class="samp">i4x4</samp>’</dt> <dd> 4x4 I-frame partition. (Enabling ‘<samp class="samp">p4x4</samp>’ requires ‘<samp class="samp">p8x8</samp>’ to be enabled. Enabling ‘<samp class="samp">i8x8</samp>’ requires adaptive spatial transform (<samp class="option">8x8dct</samp> option) to be enabled.) </dd> <dt>‘<samp class="samp">none (none)</samp>’</dt> <dd> Do not consider any partitions. </dd> <dt>‘<samp class="samp">all (all)</samp>’</dt> <dd> Consider every partition. </dd> </dl>

</dd> <dt><samp class="option">direct-pred (direct)</samp></dt> <dd> Set direct MV prediction mode. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (none)</samp>’</dt> <dd> Disable MV prediction. </dd> <dt>‘<samp class="samp">spatial (spatial)</samp>’</dt> <dd> Enable spatial predicting. </dd> <dt>‘<samp class="samp">temporal (temporal)</samp>’</dt> <dd> Enable temporal predicting. </dd> <dt>‘<samp class="samp">auto (auto)</samp>’</dt> <dd> Automatically decided. </dd> </dl>

</dd> <dt><samp class="option">slice-max-size (slice-max-size)</samp></dt> <dd> Set the limit of the size of each slice in bytes. If not specified but RTP payload size (<samp class="option">ps</samp>) is specified, that is used. </dd> <dt><samp class="option">stats (stats)</samp></dt> <dd> Set the file name for multi-pass stats. </dd> <dt><samp class="option">nal-hrd (nal-hrd)</samp></dt> <dd> Set signal HRD information (requires <samp class="option">vbv-bufsize</samp> to be set). Possible values: <dl class="table"> <dt>‘<samp class="samp">none (none)</samp>’</dt> <dd> Disable HRD information signaling. </dd> <dt>‘<samp class="samp">vbr (vbr)</samp>’</dt> <dd> Variable bit rate. </dd> <dt>‘<samp class="samp">cbr (cbr)</samp>’</dt> <dd> Constant bit rate (not allowed in MP4 container). </dd> </dl>

</dd> <dt><samp class="option">x264opts <var class="var">opts</var></samp></dt> <dt><samp class="option">x264-params <var class="var">opts</var></samp></dt>

<dd> Override the x264 configuration using a :-separated list of key=value options. The argument for both options is a list of <var class="var">key</var>=<var class="var">value</var> couples separated by ":". With <samp class="option">x264opts</samp> the value can be omitted, and the value <code class="code">1</code> is assumed in that case. For <var class="var">filter</var> and <var class="var">psy-rd</var> options values that use ":" as a separator themselves, use "," instead. They accept it as well since long ago but this is kept undocumented for some reason. For example, the options might be provided as: <pre class="example-preformatted">level=30:bframes=0:weightp=0:cabac=0:ref=1:vbv-maxrate=768:vbv-bufsize=2000:analyse=all:me=umh:no-fast-pskip=1:subq=6:8x8dct=0:trellis=0
</pre>

For example to specify libx264 encoding options with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i foo.mpg -c:v libx264 -x264opts keyint=123:min-keyint=20 -an out.mkv
</pre>

To get the complete list of the libx264 options, invoke the command <code class="command">x264 --fullhelp</code> or consult the libx264 documentation. </dd> <dt><samp class="option">a53cc <var class="var">boolean</var></samp></dt> <dd> Import closed captions (which must be ATSC compatible format) into output. Only the mpeg2 and h264 decoders provide these. Default is 1 (on). </dd> <dt><samp class="option">udu_sei <var class="var">boolean</var></samp></dt> <dd> Import user data unregistered SEI if available into output. Default is 0 (off). </dd> <dt><samp class="option">mb_info <var class="var">boolean</var></samp></dt> <dd> Set mb_info data through AVFrameSideData, only useful when used from the API. Default is 0 (off). </dd> </dl>

Encoding ffpresets for common usages are provided so they can be used with the general presets system (e.g. passing the <samp class="option">pre</samp> option). <h3 class="section">16.21 libx265# TOC</h3>

x265 H.265/HEVC encoder wrapper. This encoder requires the presence of the libx265 headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libx265</samp>. <h4 class="subsection">16.21.1 Options#
TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Sets

target video bitrate. </dd> <dt><samp class="option">bf</samp></dt>
<dt><samp class="option">g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class="option">keyint_min</samp></dt>
<dd> Minimum GOP size. </dd> <dt><samp class="option">refs</samp></dt>
<dd> Number of reference frames each P-frame can use. The range is from <var class="var">1-16</var>. </dd> <dt><samp class="option">preset</samp></dt> <dd> Set the x265 preset. </dd> <dt><samp class="option">tune</samp></dt> <dd> Set the x265 tune parameter. </dd> <dt><samp class="option">profile</samp></dt>
<dd> Set profile restrictions. </dd> <dt><samp class="option">crf</samp></dt>
<dd> Set the quality for constant quality mode. </dd>
<dt><samp class="option">qp</samp></dt> <dd> Set constant quantization rate control method parameter. </dd> <dt><samp class="option">qmin</samp></dt> <dd> Minimum quantizer scale. </dd> <dt><samp class="option">qmax</samp></dt>
<dd> Maximum quantizer scale. </dd> <dt><samp class="option">qdiff</samp></dt>
<dd> Maximum difference between quantizer scales. </dd>
<dt><samp class="option">qblur</samp></dt> <dd> Quantizer curve blur </dd> <dt><samp class="option">qcomp</samp></dt>
<dd> Quantizer curve compression factor </dd> <dt><samp class="option">i_qfactor</samp></dt> <dt><samp class="option">b_qfactor</samp></dt>
<dt><samp class="option">forced-idr</samp></dt> <dd> Normally, when forcing a I-frame type, the encoder can select any type of I-frame. This option forces it to choose an IDR-frame. </dd> <dt><samp class="option">x265-stats</samp></dt> <dd> Specify the file name for 2-pass stats. This is set automatically when using the <samp class="option">-passlogfile</samp> option. </dd> <dt><samp class="option">udu_sei <var class="var">boolean</var></samp></dt>
<dd> Import user data unregistered SEI if available into output. Default is 0 (off). </dd> <dt><samp class="option">x265-params</samp></dt> <dd> Set x265 options using a list of <var class="var">key</var>=<var class="var">value</var> couples separated by ":". See <code class="command">x265 --help</code> for a list of options. For example to specify libx265 encoding options with <samp class="option">-x265-params</samp>:
<pre class="example-preformatted">ffmpeg -i input -c:v libx265 -x265-params crf=26:psy-rd=1 output.mp4 </pre> </dd> </dl>

 <h3 class="section">16.22 libxavs2# TOC</h3>

xavs2 AVS2-P2/IEEE1857.4 encoder wrapper. This encoder requires the presence of the libxavs2 headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxavs2</samp>. The following standard libavcodec

options are used: <ul class="itemize mark-bullet"> <samp class="option">b</samp> / <samp class="option">bit_rate</samp> <samp class="option">g</samp> / <samp class="option">gop_size</samp> <samp class="option">bf</samp> / <samp class="option">max_b_frames</samp>

The encoder also has its own specific options: <h4 class="subsection">16.22.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">lcu_row_threads</samp></dt> <dd> Set the number of parallel threads for rows from 1 to 8 (default 5). </dd> <dt><samp class="option">initial_qp</samp></dt> <dd> Set the xavs2 quantization parameter from 1 to 63 (default 34). This is used to set the initial qp for the first frame. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set the xavs2 quantization parameter from 1 to 63 (default 34). This is used to set the qp value under constant-QP mode. </dd> <dt><samp class="option">max_qp</samp></dt> <dd> Set the max qp for rate control from 1 to 63 (default 55). </dd> <dt><samp class="option">min_qp</samp></dt> <dd> Set the min qp for rate control from 1 to 63 (default 20). </dd> <dt><samp class="option">speed_level</samp></dt> <dd> Set the Speed level from 0 to 9 (default 0). Higher is better but slower. </dd> <dt><samp class="option">log_level</samp></dt> <dd> Set the log level from -1 to 3 (default 0). -1: none, 0: error, 1: warning, 2: info, 3: debug. </dd> <dt><samp class="option">xavs2-params</samp></dt> <dd> Set xavs2 options using a list of <var class="var">key</var>=<var class="var">value</var> couples separated by ":". For example to specify libxavs2 encoding options with <samp class="option">-xavs2-params</samp>:   <pre class="example-preformatted">ffmpeg -i input -c:v libxavs2 -xavs2-params RdoqLevel=0 output.avs2 </pre> </dd> </dl>

 <h3 class="section">16.23 libxeve# TOC</h3>

eXtra-fast Essential Video Encoder (XEVE) MPEG-5 EVC encoder wrapper. The xeve-equivalent options or values are listed in parentheses for easy migration. This encoder requires the presence of the libxeve headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxeve</samp>.   Many libxeve encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. Additionally the xeve-params private options allows one to pass a list of key=value tuples

as accepted by the libxeve <code class=”code”>parse_xeve_params</code> function. The xeve project website is at https://github.com/mpeg5/xeve. <h4 class=”subsection”>16.23.1 Options# TOC</h4>

The following options are supported by the libxeve wrapper. The xeve-equivalent options or values are listed in parentheses for easy migration.   To reduce the duplication of documentation, only the private options and some others requiring special attention are documented here. For the documentation of the undocumented generic options, see the Codec Options chapter.   To get a more accurate and extensive documentation of the libxeve options, invoke the command <code class=”code”>xeve_app --help</code> or consult the libxeve documentation. <dl class=”table”> <dt><samp class=”option”>b (bitrate)</samp></dt> <dd> Set target video bitrate in bits/s. Note that FFmpeg’s b option is expressed in bits/s, while xeve’s bitrate is in kilobits/s. </dd> <dt><samp class=”option”>bf (bframes)</samp></dt> <dd> Set the maximum number of B frames (1,3,7,15). </dd> <dt><samp class=”option”>g (keyint)</samp></dt> <dd> Set the GOP size (I-picture period). </dd> <dt><samp class=”option”>preset (preset)</samp></dt> <dd> Set the xeve preset. Set the encoder preset value to determine encoding speed [fast, medium, slow, placebo] </dd> <dt><samp class=”option”>tune (tune)</samp></dt> <dd> Set the encoder tune parameter [psnr, zerolatency] </dd> <dt><samp class=”option”>profile (profile)</samp></dt> <dd> Set the encoder profile [0: baseline; 1: main] </dd> <dt><samp class=”option”>crf (crf)</samp></dt> <dd> Set the quality for constant quality mode. Constant rate factor <10..49> [default: 32] </dd> <dt><samp class=”option”>qp (qp)</samp></dt> <dd> Set constant quantization rate control method parameter. Quantization parameter qp <0..51> [default: 32] </dd> <dt><samp class=”option”>threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class=”section”>16.24 libxvid# TOC</h3>

Xvid MPEG-4 Part 2 encoder wrapper. This encoder

requires the presence of the libxvidcore headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libxvid --enable-gpl</code>. The native <code class=”code”>mpeg4</code> encoder supports the MPEG-4 Part 2 format, so users can encode to this format without this library. <h4 class=”subsection”>16.24.1 Options# TOC</h4>

The following options are supported by the libxvid wrapper. Some of the following options are listed but are not documented, and correspond to shared codec options. See the Codec Options chapter for their documentation. The other shared options which are not listed have no effect for the libxvid encoder. <dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dt><samp class=”option”>g</samp></dt> <dt><samp class=”option”>qmin</samp></dt> <dt><samp class=”option”>qmax</samp></dt> <dt><samp class=”option”>mpeg_quant</samp></dt> <dt><samp class=”option”>threads</samp></dt> <dt><samp class=”option”>bf</samp></dt> <dt><samp class=”option”>b_qfactor</samp></dt> <dt><samp class=”option”>b_qoffset</samp></dt> <dt><samp class=”option”>flags</samp></dt> <dd> Set specific encoding flags. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>mv4</samp>’</dt> <dd> Use four motion vector by macroblock. </dd> <dt>‘<samp class=”samp”>aic</samp>’</dt> <dd> Enable high quality AC prediction. </dd> <dt>‘<samp class=”samp”>gray</samp>’</dt> <dd> Only encode grayscale. </dd> <dt>‘<samp class=”samp”>qpel</samp>’</dt> <dd> Enable quarter-pixel motion compensation. </dd> <dt>‘<samp class=”samp”>cgop</samp>’</dt> <dd> Enable closed GOP. </dd> <dt>‘<samp class=”samp”>global_header</samp>&rsq <dd> Place global headers in extradata instead of every keyframe. </dd> </dl>

</dd> <dt><samp class=”option”>gmc</samp></dt> <dd> Enable the use of global motion compensation (GMC). Default is 0 (disabled). </dd> <dt><samp class=”option”>me_quality</samp></dt> <dd> Set motion estimation quality level. Possible values in decreasing order of speed and increasing order of quality: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Use no motion estimation (default). </dd> <dt>‘<samp class=”samp”>1, 2</samp>’</dt> <dd> Enable advanced diamond zonal search for 16x16 blocks and half-pixel refinement for 16x16 blocks. </dd> <dt>‘<samp class=”samp”>3, 4</samp>’</dt> <dd> Enable all of the things described above, plus advanced diamond zonal search for 8x8 blocks and half-pixel refinement for 8x8 blocks, also enable motion estimation on chroma planes for P and B-frames. </dd> <dt>‘<samp class=”samp”>5, 6</samp>’</dt>

<dd> Enable all of the things described above, plus extended 16x16 and 8x8 blocks search. </dd> </dl>

</dd> <dt><samp class="option">mbd</samp></dt> <dd> Set macroblock decision algorithm. Possible values in the increasing order of quality: <dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt> <dd> Use macroblock comparing function algorithm (default). </dd> <dt>‘<samp class="samp">bits</samp>’</dt> <dd> Enable rate distortion-based half pixel and quarter pixel refinement for 16x16 blocks. </dd> <dt>‘<samp class="samp">rd</samp>’</dt> <dd> Enable all of the things described above, plus rate distortion-based half pixel and quarter pixel refinement for 8x8 blocks, and rate distortion-based search using square pattern. </dd> </dl>

</dd> <dt><samp class="option">lumi_aq</samp></dt> <dd> Enable lumi masking adaptive quantization when set to 1. Default is 0 (disabled). </dd> <dt><samp class="option">variance_aq</samp></dt> <dd> Enable variance adaptive quantization when set to 1. Default is 0 (disabled). When combined with <samp class="option">lumi_aq</samp>, the resulting quality will not be better than any of the two specified individually. In other words, the resulting quality will be the worse one of the two effects. </dd> <dt><samp class="option">trellis</samp></dt> <dd> Set rate-distortion optimal quantization. </dd> <dt><samp class="option">ssim</samp></dt> <dd> Set structural similarity (SSIM) displaying method. Possible values: <dl class="table"> <dt>‘<samp class="samp">off</samp>’</dt> <dd> Disable displaying of SSIM information. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Output average SSIM at the end of encoding to stdout. The format of showing the average SSIM is: <pre class="example-preformatted">Average SSIM: %f </pre>

For users who are not familiar with C, %f means a float number, or a decimal (e.g. 0.939232). </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Output both per-frame SSIM data during encoding and average SSIM at the end of encoding to stdout. The format of per-frame information is: <pre class="example-preformatted">SSIM: avg: %1.3f min: %1.3f max: %1.3f </pre>

For users who are not familiar with C, %1.3f means a float number rounded to 3 digits after the dot (e.g. 0.932). </dd> </dl>

</dd> <dt><samp class="option">ssim_acc</samp></dt> <dd> Set SSIM accuracy. Valid options are integers within the range of 0-4, while 0 gives the most accurate result and 4 computes the fastest. </dd> </dl>

 <h3 class="section">16.25 MediaCodec<a class="anchor hidden-xs" href="#MediaCodec"

aria-hidden=”true”># TOC</h3>

MediaCodec encoder wrapper enables hardware-accelerated video encoding on Android device. It supports H.264, H.265 (HEVC), VP8, VP9, MPEG-4, and AV1 encoding (whether works or not is device dependent). Android provides two sets of APIs: Java MediaCodec and NDK MediaCodec. The MediaCodec encoder wrapper supports both. Note that the NDK MediaCodec API operates without requiring JVM, but may fail to function outside the JVM environment due to dependencies on system framework services, particularly after Android 15. <dl class=”table”> <dt><samp class=”option”>ndk_codec <var class=”var”>boolean</var></samp></dt> <dd> Use the NDK-based MediaCodec API instead of the Java API. Enabled by default if <code class=”code”>av_jni_get_java_vm()</code> return NULL. </dd> <dt><samp class=”option”>ndk_async <var class=”var”>boolean</var></samp></dt> <dd> Use NDK MediaCodec in async mode. Async mode has less overhead than poll in a loop in sync mode. The drawback of async mode is AV_CODEC_FLAG_GLOBAL_HEADER doesn’t work (use extract_extradata bsf when necessary). It doesn’t work and will be disabled automatically on devices below Android 8.0. </dd> <dt><samp class=”option”>codec_name <var class=”var”>string</var></samp></dt> <dd> A codec type can have multiple implementations on a single device, this option specify which backend to use (via MediaCodec createCodecByName API). It’s NULL by default, and encoder is created by createEncoderByType. </dd> <dt><samp class=”option”>bitrate_mode <var class=”var”>integer</var></samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>cq</samp>’</dt> <dd> Constant quality mode </dd> <dt>‘<samp class=”samp”>vbr</samp>’</dt> <dd> Variable bitrate mode </dd> <dt>‘<samp class=”samp”>cbr</samp>’</dt> <dd> Constant bitrate mode </dd> <dt>‘<samp class=”samp”>cbr_fd</samp>’</dt> <dd> Constant bitrate mode with frame drops </dd> </dl>

</dd> <dt><samp class=”option”>pts_as_dts <var class=”var”>boolean</var></samp></dt> <dd> Use PTS as DTS. This is a workaround since MediaCodec API doesn’t provide decoding timestamp. It is enabled automatically if B frame is 0. </dd> <dt><samp class=”option”>operating_rate <var class=”var”>integer</var></samp></dt> <dd> The desired operating rate that the codec will need to operate at, zero for unspecified. This is used for cases like high-speed/slow-motion video capture, where the video encoder format contains the target playback rate (e.g. 30fps), but the component must be able to handle the high operating capture rate (e.g. 240fps). This rate will be used by codec for resource planning and setting the operating points. </dd> <dt><samp

class=”option”>qp_i_min <var class=”var”>integer</var></samp></dt>
<dd> Minimum quantization parameter for I frame. </dd>
<dt><samp class=”option”>qp_p_min <var class=”var”>integer</var></samp></dt>
<dd> Minimum quantization parameter for P frame. </dd>
<dt><samp class=”option”>qp_b_min <var class=”var”>integer</var></samp></dt>
<dd> Minimum quantization parameter for B frame. </dd>
<dt><samp class=”option”>qp_i_max <var class=”var”>integer</var></samp></dt>
<dd> Maximum quantization parameter for I frame. </dd>
<dt><samp class=”option”>qp_p_max <var class=”var”>integer</var></samp></dt>
<dd> Maximum quantization parameter for P frame. </dd>
<dt><samp class=”option”>qp_b_max <var class=”var”>integer</var></samp></dt>
<dd> Maximum quantization parameter for B frame. </dd> </dl>

 <h3 class=”section”>16.26 MediaFoundation# TOC</h3>

The following wrappers for encoders in the MediaFoundation framework are available: <ul class=”itemize mark-bullet”> h264_mf hevc_mf av1_mf

These support both software and hardware encoding. Video encoders can take input in either of nv12 or yuv420p form (some encoders support both, some support only either - in practice, nv12 is the safer choice, especially among HW encoders). Hardware-accelerated encoding requires D3D11, including hardware scaling capabilities through the scale_d3d11 filter. To list all available options for the MediaFoundation encoders, use: <code class=”command”>ffmpeg -h encoder=<encoder></code> e.g. <code class=”command”>ffmpeg -h encoder=h264_mf</code> <h4 class=”subsection”>16.26.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>rate_control</samp></dt>
<dd> Select rate control mode. Available modes: <dl class=”table”>
<dt>‘<samp class=”samp”>default</samp>’</dt> <dd> Default mode </dd> <dt>‘<samp class=”samp”>cbr</samp>’</dt>
<dd> CBR mode </dd> <dt>‘<samp class=”samp”>pc_vbr</samp>’</dt>
<dd> Peak constrained VBR mode </dd> <dt>‘<samp class=”samp”>u_vbr</samp>’</dt> <dd> Unconstrained VBR mode </dd> <dt>‘<samp class=”samp”>quality</samp>’</dt>
<dd> Quality mode </dd> <dt>‘<samp class=”samp”>ld_vbr</samp>’</dt>
<dd> Low delay VBR mode (requires Windows 8+) </dd>
<dt>‘<samp class=”samp”>g_vbr</samp>’</dt> <dd> Global VBR mode (requires Windows 8+) </dd> <dt>‘<samp class=”samp”>gld_vbr</samp>’</dt> <dd> Global low delay

VBR mode (requires Windows 8+) </dd> </dl>

</dd> <dt><samp class=”option”>scenario</samp></dt> <dd> Select usage scenario. Available scenarios: <dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt> <dd> Default scenario </dd> <dt>‘<samp class=”samp”>display_remoting</samp>’</dt> <dd> Display remoting scenario </dd> <dt>‘<samp class=”samp”>video_conference</samp>’</dt> <dd> Video conference scenario </dd> <dt>‘<samp class=”samp”>archive</samp>’</dt> <dd> Archive scenario </dd> <dt>‘<samp class=”samp”>live_streaming</samp>’ <dd> Live streaming scenario </dd> <dt>‘<samp class=”samp”>camera_record</samp>’</dt> <dd> Camera record scenario </dd> <dt>‘<samp class=”samp”>display_remoting_with_feature_map</san <dd> Display remoting with feature map scenario </dd> </dl>

</dd> <dt><samp class=”option”>quality</samp></dt> <dd> Set encoding quality (0-100). -1 means default quality. </dd> <dt><samp class=”option”>hw_encoding</samp></dt> <dd> Force hardware encoding (0-1). Default is 0 (disabled). </dd> </dl>

 <h4 class=”subsection”>16.26.2 Examples# TOC</h4>

Hardware encoding: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -c:v h264_mf -hw_encoding 1 output.mp4 </pre>

Hardware-accelerated decoding with hardware encoding: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d11va -i input.mp4 -c:v h264_mf -hw_encoding 1 output.mp4 </pre>

Hardware-accelerated decoding, HW scaling and encoding with quality setting: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d11va -hwaccel_output_format d3d11 -i input.mp4 -vf scale_d3d11=1920:1080 -c:v hevc_mf -hw_encoding 1 -quality 80 output.mp4 </pre>

 <h3 class=”section”>16.27 Microsoft RLE# TOC</h3>

Microsoft RLE aka MSRLE encoder. Only 8-bit palette mode supported. Compatible with Windows 3.1 and Windows 95. <h4 class=”subsection”>16.27.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">g <var class="var">integer</var></samp></dt>
<dd> Keyframe interval. A keyframe is inserted at least every <code
class="code">-g</code> frames, sometimes sooner. </dd> </dl>

 <h3 class="section">16.28 mpeg2<span
class="pull-right"><a class="anchor hidden-xs" href="#mpeg2" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-mpeg2"
aria-hidden="true">TOC</h3>

MPEG-2 video encoder.
<h4 class="subsection">16.28.1 Options<a
class="anchor hidden-xs" href="#Options-52" aria-hidden="true">#
TOC</h4>

<dl class="table"> <dt><samp class="option">profile</samp></dt>
<dd> Select the mpeg2 profile to encode: <dl class="table">
<dt>‘<samp class="samp">422</samp>’</dt> <dt>‘<samp
class="samp">high</samp>’</dt> <dt>‘<samp class="samp">ss</samp>’</dt>
<dd> Spatially Scalable </dd> <dt>‘<samp class="samp">snr</samp>’</dt>
<dd> SNR Scalable </dd> <dt>‘<samp class="samp">main</samp>’</dt>
<dt>‘<samp class="samp">simple</samp>’</dt> </dl>

</dd> <dt><samp class="option">level</samp></dt> <dd> Select
the mpeg2 level to encode: <dl class="table"> <dt>‘<samp
class="samp">high</samp>’</dt> <dt>‘<samp class="samp">high1440</samp>’<
<dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp
class="samp">low</samp>’</dt> </dl>

</dd> <dt><samp class="option">seq_disp_ext <var class="var">integer</var></samp></dt>
<dd> Specifies if the encoder should write a sequence_display_extension
to the output. <dl class="table"> <dt><samp class="option">-
1</samp></dt> <dt><samp class="option">auto</samp></dt>
<dd> Decide automatically to write it or not (this is the default) by
checking if the data to be written is different from the default or unspecified values. </dd> <dt><samp class="option">0</samp></dt>
<dt><samp class="option">never</samp></dt> <dd> Never
write it. </dd> <dt><samp class="option">1</samp></dt>
<dt><samp class="option">always</samp></dt> <dd> Always write
it. </dd> </dl> </dd> <dt><samp class="option">video_format
<var class="var">integer</var></samp></dt> <dd> Specifies
the video_format written into the sequence display extension indicating the source of the video pictures. The default is ‘<samp
class="samp">unspecified</samp>’, can be ‘<samp class="samp">component</samp>&rsqu
‘<samp class="samp">pal</samp>’, ‘<samp class="samp">ntsc</samp>’,
‘<samp class="samp">secam</samp>’ or ‘<samp
class="samp">mac</samp>’. For maximum compatibility, use
‘<samp class="samp">component</samp>’. </dd>
<dt><samp class="option">a53cc <var class="var">boolean</var></samp></dt>

<dd> Import closed captions (which must be ATSC compatible format) into output. Default is 1 (on). </dd> </dl>

 <h3 class="section">16.29 NVENC encoders# TOC</h3>

The <code class="code">h264_nvenc</code>, <code class="code">hevc_nvenc</code> and <code class="code">av1_nvenc</code> encoders use NVIDIA NVENC hardware to encode H.264, HEVC and AV1 video, respectively. Building them requires the <code class="code">ffnvcodec</code> headers provided by nv-codec-headers. They are autodetected by configure; they can also be enabled explicitly with <samp class="option">--enable-ffnvcodec</samp> and <samp class="option">--enable-nvenc</samp>. Available options and values depend on the SDK version exposed by the headers at build time. Feature checks in <samp class="file">libavcodec/nvenc.h</samp> distinguish SDK versions 12.0, 12.1, 12.2, 13.0 and 13.1. Runtime support also depends on the GPU and driver. Use <code class="command">ffmpeg -h encoder=hevc_nvenc</code>, or substitute <code class="code">h264_nvenc</code> or <code class="code">av1_nvenc</code>, to list the options, devices and pixel formats available in a particular build. <h4 class="subsection">16.29.1 Hardware input and devices# TOC</h4>

These encoders accept system-memory frames and CUDA hardware frames (<code class="code">AV_PIX_FMT_CUDA</code>). Builds using SDK 13.1 or newer also accept CUDA array frames (<code class="code">AV_PIX_FMT_CUARRAY</code>). Builds with <code class="code">d3d11va</code> support accept D3D11 hardware frames (<code class="code">AV_PIX_FMT_D3D11</code>). The input pixel format and encoding features must be supported by the selected GPU. For CUDA decoding followed by encoding, use the input options <samp class="option">-hwaccel cuda -hwaccel_output_format cuda</samp> to keep decoded frames on the device. A hardware device can also be created with <samp class="option">-init_hw_device</samp> and supplied to the encoder, directly or through its hardware frames context. For system-memory input without a supplied hardware device, the encoder option <samp class="option">gpu</samp> selects the device. See its entry under Other common options below. <h4 class="subsection">16.29.2 Generic options#

TOC</h4>

Several generic encoding options are mapped to NVENC settings. The bitrate options <samp class=”option”>b</samp>, <samp class=”option”>maxrate</samp> and <samp class=”option”>bufsize</samp> set the average bitrate, the maximum bitrate and the VBV buffer size; the NVENC default for <samp class=”option”>b</samp> is ‘<samp class=”samp”>2M</samp>’. <samp class=”option”>g</samp> sets the GOP length, with 0 requesting intra-only encoding. <samp class=”option”>bf</samp> sets the number of B-frames between non-B-frames. <samp class=”option”>refs</samp> sets the number of reference frames in each reference list. <samp class=”option”>g</samp> and <samp class=”option”>bf</samp> default to <code class=”code”>-1</code> and <samp class=”option”>refs</samp> defaults to 0; these defaults leave the values of the preset unchanged. The generic <samp class=”option”>qmin</samp> and <samp class=”option”>qmax</samp> options are honoured in VBR mode with a warning; API users should set the encoder options of the same names listed below instead. Only applications calling the libavcodec API can change <code class=”code”>AVCodecContext.bit_rate</code>, <code class=”code”>AVCodecContext.rc_max_rate</code> and <code class=”code”>AVCodecContext.rc_buffer_size</code> during encoding. The encoder checks these fields before encoding each frame. When dynamic bitrate changes are supported and rate control is not ‘<samp class=”samp”>constqp</samp>’, changed positive values trigger reconfiguration, reset the encoder and force an IDR. A change to the display aspect ratio also triggers reconfiguration. <h4 class=”subsection”>16.29.3 Common options# TOC</h4>

The following option names are shared by all three encoders. Differences in values and build requirements are noted below. Boolean options accept 0 (disabled) and 1 (enabled), unless otherwise stated. <h4 class=”subsubsection”>16.29.3.1 Presets and tuning# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>preset <var class=”var”>integer</var></samp></dt> <dd> Select the encoding preset using a named value from the table (recommended) or its integer. The range is <code class=”code”>0</code> to <code class=”code”>9</code>; the default is ‘<samp class=”samp”>p4</samp>’ (6). <dl class=”table”> <dt>‘<samp class=”samp”>slow</samp>’</dt> <dd> Deprecated alias for high quality with two passes (0). </dd> <dt>‘<samp class=”samp”>medium</samp>’</dt> <dd> Deprecated alias for high quality with one pass (1). </dd> <dt>‘<samp

class=”samp”>fast</samp>’</dt> <dd> Deprecated alias for high performance with one pass (2). </dd> <dt>‘<samp class=”samp”>p1</samp>’</dt> <dd> Fastest, lowest quality (3). </dd> <dt>‘<samp class=”samp”>p2</samp>’</dt> <dd> Faster, lower quality (4). </dd> <dt>‘<samp class=”samp”>p3</samp>’</dt> <dd> Fast, low quality (5). </dd> <dt>‘<samp class=”samp”>p4</samp>’</dt> <dd> Medium (6); the default. </dd> <dt>‘<samp class=”samp”>p5</samp>’</dt> <dd> Slow, good quality (7). </dd> <dt>‘<samp class=”samp”>p6</samp>’</dt> <dd> Slower, better quality (8). </dd> <dt>‘<samp class=”samp”>p7</samp>’</dt> <dd> Slowest, best quality (9). </dd> </dl>

Use ‘<samp class=”samp”>p1</samp>’ through ‘<samp class=”samp”>p7</samp>’ with <samp class=”option”>tune</samp> and <samp class=”option”>multipass</samp> instead of the deprecated aliases. </dd> <dt><samp class=”option”>tune <var class=”var”>integer</var></samp></dt> <dd> Select encoding tuning using a named value from the table (recommended) or its integer. The default is ‘<samp class=”samp”>hq</samp>’ (1). H.264 accepts <code class=”code”>1</code> to <code class=”code”>4</code>; HEVC and AV1 also expose ‘<samp class=”samp”>uhq</samp>’ when built with the required SDK. <dl class=”table”> <dt>‘<samp class=”samp”>hq</samp>’</dt> <dd> High quality (1). </dd> <dt>‘<samp class=”samp”>ll</samp>’</dt> <dd> Low latency (2). </dd> <dt>‘<samp class=”samp”>ull</samp>’</dt> <dd> Ultra low latency (3). </dd> <dt>‘<samp class=”samp”>lossless</samp>’</dt> <dd> Lossless (4). </dd> <dt>‘<samp class=”samp”>uhq</samp>’</dt> <dd> Ultra high quality (5). HEVC requires SDK 12.2 or newer; AV1 requires SDK 13.0 or newer. Not available for H.264. </dd> </dl> </dd> </dl>

 <h4 class=”subsubsection”>16.29.3.2 Rate control# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>rc <var class=”var”>integer</var></samp></dt> <dd> Override preset rate control using a named value from the table (recommended) or its integer. The range is <code class=”code”>-1</code> to <code class=”code”>INT_MAX</code>; the default is <code class=”code”>-1</code> (no explicit override). <dl class=”table”> <dt>‘<samp class=”samp”>constqp</samp>’</dt> <dd> Constant QP (0). </dd> <dt>‘<samp class=”samp”>vbr</samp>’</dt> <dd> Variable bitrate (1). </dd> <dt>‘<samp class=”samp”>cbr</samp>’</dt>

<dd> Constant bitrate (2). </dd> </dl>

</dd> <dt><samp class="option">multipass <var class="var">integer</var></samp></dt>
<dd> Select multipass encoding using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">2</code>; the default is ‘<samp class="samp">disabled</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">disabled</samp>’</dt>
<dd> Single pass (0). </dd> <dt>‘<samp class="samp">qres</samp>’</dt>
<dd> Two passes, with the first pass at quarter resolution (1). </dd>
<dt>‘<samp class="samp">fullres</samp>’</dt> <dd> Two passes, with the first pass at full resolution (2). </dd> </dl>

</dd> <dt><samp class="option">cq <var class="var">float</var></samp></dt>
<dd> Set the target quality for constant quality mode in VBR rate control. The default is 0 (automatic). The range is <code class="code">0</code> to <code class="code">51</code> for H.264 and HEVC, or <code class="code">0</code> to <code class="code">63</code> for AV1. </dd> <dt><samp class="option">qp <var class="var">integer</var></samp></dt>
<dd> Set the constant quantization parameter. The default is <code class="code">-1</code> (unset). The range is <code class="code">-1</code> to <code class="code">51</code> for H.264 and HEVC, or <code class="code">-1</code> to <code class="code">255</code> for AV1. </dd> <dt><samp class="option">init_qpP <var class="var">integer</var></samp></dt> <dt><samp class="option">init_qpB <var class="var">integer</var></samp></dt> <dt><samp class="option">init_qpI <var class="var">integer</var></samp></dt> <dd> Set the initial QP for P-, B- and I-frames, respectively. Each defaults to <code class="code">-1</code> (unset). The range is <code class="code">-1</code> to <code class="code">51</code> for H.264 and HEVC, or <code class="code">-1</code> to <code class="code">255</code> for AV1. </dd> <dt><samp class="option">qmin <var class="var">integer</var></samp></dt>
<dt><samp class="option">qmax <var class="var">integer</var></samp></dt>
<dd> Set the minimum and maximum QP used for rate control. Each defaults to <code class="code">-1</code> (unset). The range is <code class="code">-1</code> to <code class="code">51</code> for H.264 and HEVC, or <code class="code">-1</code> to <code class="code">255</code> for AV1. </dd> <dt><samp class="option">qp_cb_offset <var class="var">integer</var></samp></dt> <dt><samp class="option">qp_cr_offset <var class="var">integer</var></samp></dt> <dd> Set the chroma QP offsets for the Cb and Cr channels, respectively. Each accepts <code class="code">-12</code> to <code class="code">12</code> and defaults to 0. </dd> <dt><samp class="option">strict_gop <var class="var">boolean</var></samp></dt> <dd> Minimize GOP-to-GOP rate fluctuations. The default is 0. </dd> <dt><samp class="option">ldkfs <var class="var">integer</var></samp></dt>
<dd> Set the low-delay keyframe scale, which specifies the scene-change frame size increase allowed with a single-frame VBV buffer and CBR. The range

is <code class=”code”>0</code> to <code class=”code”>255</code> and the default is 0. </dd> <dt><samp class=”option”>cbr_padding <var class=”var”>boolean</var></samp></dt> <dd> Pad the bitstream to prevent the bitrate falling below the target in CBR mode. The default is 0. </dd> </dl>

 <h4 class=”subsubsection”>16.29.3.3 Lookahead, adaptive quantization and B-frames# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>rc-lookahead <var class=”var”>integer</var></samp></dt> <dd> Set the number of frames to look ahead for rate control. The range is <code class=”code”>0</code> to <code class=”code”>INT_MAX</code> and the default is 0. </dd> <dt><samp class=”option”>lookahead_level <var class=”var”>integer</var></samp></dt> <dd> Set the lookahead level using a named value from the table (recommended) or its integer. Higher levels may improve quality at the expense of performance. Requires SDK 12.2 or newer. The range is <code class=”code”>-1</code> to <code class=”code”>15</code>; the default is <code class=”code”>-1</code> (no override). <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dt>‘<samp class=”samp”>1</samp>’</dt> <dt>‘<samp class=”samp”>2</samp>’</dt> <dt>‘<samp class=”samp”>3</samp>’</dt> <dd> Select the corresponding lookahead level. </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection (15). </dd> </dl>

</dd> <dt><samp class=”option”>no-scenecut <var class=”var”>boolean</var></samp></dt> <dd> Disable adaptive I-frame insertion at scene cuts when lookahead is enabled. The default is 0. </dd> <dt><samp class=”option”>b_adapt <var class=”var”>boolean</var></samp></dt> <dd> Enable adaptive B-frame decisions when lookahead is enabled. The default is 1. </dd> <dt><samp class=”option”>spatial-aq <var class=”var”>boolean</var></samp></dt> <dd> Enable spatial adaptive quantization. The default is 0. </dd> <dt><samp class=”option”>temporal-aq <var class=”var”>boolean</var></samp></dt> <dd> Enable temporal adaptive quantization. The default is 0. </dd> <dt><samp class=”option”>aq-strength <var class=”var”>integer</var></samp></dt> <dd> Set spatial adaptive quantization strength when <samp class=”option”>spatial-aq</samp> is enabled, from <code class=”code”>1</code> (low) to <code class=”code”>15</code> (aggressive). The default is 8. </dd> <dt><samp class=”option”>b_ref_mode <var class=”var”>integer</var></samp></dt> <dd> Select the use of B-frames as references using a named value from the table (recommended) or its integer. The default is <code class=”code”>-1</code> (no override). H.264 and HEVC accept <code

class=”code”>-1</code> to <code class=”code”>2</code>. AV1 accepts <code class=”code”>-1</code> to <code class=”code”>4</code> with SDK 13.1 or newer, or <code class=”code”>-1</code> to <code class=”code”>2</code> with older headers. <dl class=”table”> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Do not use B-frames as references (0). </dd> <dt>‘<samp class=”samp”>each</samp>’</dt> <dd> Use each B-frame as a reference (1). </dd> <dt>‘<samp class=”samp”>middle</samp>’</dt> <dd> For H.264 and HEVC, use only half the B-frames as references (2). For AV1, use every other B-frame as an Altref2 reference (2), except the last in the Altref interval. </dd> <dt>‘<samp class=”samp”>hierarchical</samp>’</dt> <dd> Use B-frames as references hierarchically (4). AV1 only; requires SDK 13.1 or newer. </dd> </dl>

</dd> <dt><samp class=”option”>weighted_pred <var class=”var”>integer</var></samp></dt> <dd> Enable weighted prediction with 1, or disable it with 0 (the default). This is a boolean option for AV1. </dd> <dt><samp class=”option”>tf_level <var class=”var”>integer</var></samp></dt> <dd> Set temporal filtering strength. The range is <code class=”code”>-1</code> to <code class=”code”>INT_MAX</code>; the default is <code class=”code”>-1</code> (no override). Requires SDK 12.2 or newer for HEVC, or SDK 13.0 or newer for H.264 and AV1. <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dt>‘<samp class=”samp”>4</samp>’</dt> <dd> Select the corresponding temporal filtering strength. </dd> </dl> </dd> </dl>

 <h4 class=”subsubsection”>16.29.3.4 SEI and side data# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>extra_sei <var class=”var”>boolean</var></samp></dt> <dd> Pass extra frame side data. The default is 1. H.264 and HEVC use SEI payloads; AV1 uses metadata OBUs. This gates the payload array used for <samp class=”option”>a53cc</samp>, HEVC and AV1 <samp class=”option”>s12m_tc</samp>, and H.264 and HEVC <samp class=”option”>udu_sei</samp>. </dd> <dt><samp class=”option”>a53cc <var class=”var”>boolean</var></samp></dt> <dd> Use <code class=”code”>AV_FRAME_DATA_A53_CC</code> side data for A53 closed captions when available. Requires <samp class=”option”>extra_sei</samp>. The default is 1. H.264 and HEVC carry it in ITU-T T.35 registered user-data SEI; AV1 uses an ITU-T T.35 metadata OBU. </dd> <dt><samp class=”option”>s12m_tc <var class=”var”>boolean</var></samp></dt> <dd> Use <code class=”code”>AV_FRAME_DATA_S12M_TIMECODE</code> side data

when available. The default is 1. H.264 uses picture timing and requires SDK 12.2 or newer; HEVC uses timecode SEI; AV1 uses timecode metadata OBUs. The HEVC and AV1 paths require <samp class="option">extra_sei</samp>; the H.264 path is independent of it. </dd> </dl>

With SDK 13.0 or newer, HEVC and AV1 also support mastering-display and content-light-level metadata. The corresponding <code class="code">AV_FRAME_DATA_MASTERING_DISPLAY_METADATA</code> and <code class="code">AV_FRAME_DATA_CONTENT_LIGHT_LEVEL</code> entries in <code class="code">AVCodecContext.decoded_side_data</code> enable output at initialization. Frame side data supplies per-frame values, falling back to those stream values when absent. This path is independent of <samp class="option">extra_sei</samp>. <h4 class="subsubsection">16.29.3.5 Other common options# TOC</h4>

<dl class="table"> <dt><samp class="option">level <var class="var">integer</var></samp></dt> <dd> Set the encoding level restriction using a named value (recommended) or its integer. The tables below list the values, numeric ranges and default integers for each codec. The default is ‘<samp class="samp">auto</samp>’ (0 for H.264 and HEVC, 24 for AV1). </dd> <dt><samp class="option">gpu <var class="var">integer</var></samp></dt> <dd> Select the NVENC-capable GPU for system-memory input without a supplied hardware device using a named value from the table (recommended) or its integer. Device indices start at 0. The range is <code class="code">-2</code> to <code class="code">INT_MAX</code> and the default is ‘<samp class="samp">any</samp>’ (-1). <dl class="table"> <dt>‘<samp class="samp">list</samp>’</dt> <dd> List the available devices (-2). </dd> <dt>‘<samp class="samp">any</samp>’</dt> <dd> Use the first available device (-1). </dd> </dl>

</dd> <dt><samp class="option">surfaces <var class="var">integer</var></samp></dt> <dd> Set the number of concurrent surfaces, from <code class="code">0</code> to <code class="code">64</code>. The default is 0 (automatic). </dd> <dt><samp class="option">delay <var class="var">integer</var></samp></dt> <dd> Delay frame output by the specified number of frames, from <code class="code">0</code> to <code class="code">INT_MAX</code>. The default is <code class="code">INT_MAX</code>; the actual delay is limited by the number of surfaces. </dd> <dt><samp class="option">rgb_mode <var class="var">integer</var></samp></dt> <dd> Select how packed RGB input is handled using a named value from the table (recommended) or its integer. The range is <code

class=”code”>0</code> to <code class=”code”>INT_MAX</code>; the default is ‘<samp class=”samp”>yuv420</samp>’ (1). <dl class=”table”> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Reject packed RGB input (0). </dd> <dt>‘<samp class=”samp”>yuv420</samp>’</dt> <dd> Convert to YUV 4:2:0 (1). </dd> <dt>‘<samp class=”samp”>yuv444</samp>’</dt> <dd> Convert to YUV 4:4:4 (2). </dd> </dl>

</dd> <dt><samp class=”option”>highbitdepth <var class=”var”>boolean</var></samp></dt> <dd> Enable 10-bit encoding for 8-bit input. The default is 0. For H.264 this requires SDK 13.0 or newer. </dd> <dt><samp class=”option”>forced-idr <var class=”var”>boolean</var></samp></dt> <dd> When forcing keyframes, force IDR frames. The range is <code class=”code”>-1</code> to <code class=”code”>1</code> and the default is 0. A value of <code class=”code”>-1</code> leaves the picture-type decision to the encoder. </dd> <dt><samp class=”option”>zerolatency <var class=”var”>boolean</var></samp></dt> <dd> Request zero-latency operation with no reordering delay. The default is 0. </dd> <dt><samp class=”option”>nonref_p <var class=”var”>boolean</var></samp></dt> <dd> Enable automatic insertion of non-reference P-frames. The default is 0. </dd> <dt><samp class=”option”>dpb_size <var class=”var”>integer</var></samp></dt> <dd> Set the decoded picture buffer size, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>. The default is 0 (automatic). </dd> <dt><samp class=”option”>intra-refresh <var class=”var”>boolean</var></samp></dt> <dd> Use periodic intra refresh instead of IDR frames. The default is 0. </dd> </dl>

 <h4 class=”subsection”>16.29.4 H.264 and HEVC options# TOC</h4>

The following options are available for <code class=”code”>h264_nvenc</code> and <code class=”code”>hevc_nvenc</code> only. <dl class=”table”> <dt><samp class=”option”>aud <var class=”var”>boolean</var></samp></dt> <dd> Use access unit delimiters. The default is 0. </dd> <dt><samp class=”option”>bluray-compat <var class=”var”>boolean</var></samp></dt> <dd> Enable Blu-ray compatibility workarounds. The default is 0. </dd> <dt><samp class=”option”>udu_sei <var class=”var”>boolean</var></samp></dt> <dd> Pass <code class=”code”>AV_FRAME_DATA_SEI_UNREGISTERED</code> frame side data as unregistered user-data SEI. Requires <samp class=”option”>extra_sei</samp>. The default is 0. </dd> <dt><samp class=”option”>single-slice-intra-refresh <var class=”var”>boolean</var></samp></dt> <dd> Use single-slice intra refresh. This also enables <samp class=”option”>intra-refresh</samp>. The default is 0. </dd> <dt><samp class=”option”>max_slice_size

<var class=”var”>integer</var></samp></dt> <dd> Set the maximum encoded slice size in bytes, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>. The default is 0 (no override). </dd> <dt><samp class=”option”>constrained-encoding <var class=”var”>boolean</var></samp></dt> <dd> Enable constrained-frame encoding, making each slice in a constrained picture independent of the other slices. The default is 0. </dd> </dl>

 <h4 class=”subsection”>16.29.5 HEVC and AV1 options# TOC</h4>

The following option is available for <code class=”code”>hevc_nvenc</code> and <code class=”code”>av1_nvenc</code> only. <dl class=”table”> <dt><samp class=”option”>split_encode_mode <var class=”var”>integer</var></samp></dt> <dd> Set split-frame encoding mode using a named value from the table (recommended) or its integer. Requires SDK 12.1 or newer. The range is <code class=”code”>0</code> to <code class=”code”>15</code> and the default is ‘<samp class=”samp”>auto</samp>’ (0). <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Enable or disable it according to the preset and tuning (0). </dd> <dt>‘<samp class=”samp”>forced</samp>’</dt> <dd> Enable it with the number of horizontal strips selected by the driver (1). </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> Force two horizontal strips when more than one NVENC engine is available (2). </dd> <dt>‘<samp class=”samp”>3</samp>’</dt> <dd> Force three horizontal strips when more than two NVENC engines are available (3). </dd> <dt>‘<samp class=”samp”>4</samp>’</dt> <dd> Force four horizontal strips when more than three NVENC engines are available (4). Requires SDK 13.0 or newer. </dd> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Disable split encoding (15). </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>16.29.6 H.264 options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>profile <var class=”var”>integer</var></samp></dt> <dd> Set the H.264 profile using a named value from the table (recommended) or its integer. The default is ‘<samp class=”samp”>high</samp>’ (2). <dl class=”table”> <dt>‘<samp class=”samp”>baseline</samp>’</dt> <dd> baseline profile (0). </dd> <dt>‘<samp class=”samp”>main</samp>’</dt> <dd> main profile (1). </dd> <dt>‘<samp class=”samp”>high</samp>’</dt> <dd> high profile (2). </dd> <dt>‘<samp class=”samp”>high10</samp>’</dt> <dd> high10 profile (3). </dd> <dt>‘<samp class=”samp”>high422</samp>’</dt

<dd> high422 profile (4). </dd> <dt>‘<samp class="samp">high444p</samp>’</
<dd> high444p profile (5). </dd> </dl>

The ‘<samp class="samp">high10</samp>’ and ‘<samp class="samp">high422</samp>’ values require SDK 13.0 or newer. With headers older than SDK 13.0, ‘<samp class="samp">high444p</samp>’ has integer value 3 instead of 5. </dd> <dt><samp class="option">level <var class="var">integer</var></samp></dt> <dd> Set the H.264 level restriction using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">62</code>; the default is ‘<samp class="samp">auto</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection (0). </dd> <dt>‘<samp class="samp">1b</samp>’</dt> <dd> Level 1b (9). </dd> <dt>‘<samp class="samp">1.0b</samp>’</dt> <dd> Level 1.0b (9). </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> Level 1 (10). </dd> <dt>‘<samp class="samp">1.0</samp>’</dt> <dd> Level 1.0 (10). </dd> <dt>‘<samp class="samp">1.1</samp>’</dt> <dd> Level 1.1 (11). </dd> <dt>‘<samp class="samp">1.2</samp>’</dt> <dd> Level 1.2 (12). </dd> <dt>‘<samp class="samp">1.3</samp>’</dt> <dd> Level 1.3 (13). </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> Level 2 (20). </dd> <dt>‘<samp class="samp">2.0</samp>’</dt> <dd> Level 2.0 (20). </dd> <dt>‘<samp class="samp">2.1</samp>’</dt> <dd> Level 2.1 (21). </dd> <dt>‘<samp class="samp">2.2</samp>’</dt> <dd> Level 2.2 (22). </dd> <dt>‘<samp class="samp">3</samp>’</dt> <dd> Level 3 (30). </dd> <dt>‘<samp class="samp">3.0</samp>’</dt> <dd> Level 3.0 (30). </dd> <dt>‘<samp class="samp">3.1</samp>’</dt> <dd> Level 3.1 (31). </dd> <dt>‘<samp class="samp">3.2</samp>’</dt> <dd> Level 3.2 (32). </dd> <dt>‘<samp class="samp">4</samp>’</dt> <dd> Level 4 (40). </dd> <dt>‘<samp class="samp">4.0</samp>’</dt> <dd> Level 4.0 (40). </dd> <dt>‘<samp class="samp">4.1</samp>’</dt> <dd> Level 4.1 (41). </dd> <dt>‘<samp class="samp">4.2</samp>’</dt> <dd> Level 4.2 (42). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> Level 5 (50). </dd> <dt>‘<samp class="samp">5.0</samp>’</dt> <dd> Level 5.0 (50). </dd> <dt>‘<samp class="samp">5.1</samp>’</dt> <dd> Level 5.1 (51). </dd> <dt>‘<samp class="samp">5.2</samp>’</dt> <dd> Level 5.2 (52). </dd> <dt>‘<samp class="samp">6.0</samp>’</dt> <dd> Level 6.0 (60). </dd> <dt>‘<samp class="samp">6.1</samp>’</dt> <dd> Level 6.1 (61). </dd> <dt>‘<samp class="samp">6.2</samp>’</dt> <dd> Level 6.2 (62). </dd> </dl>

</dd> <dt><samp class="option">coder <var class="var">integer</var></samp></dt> <dd> Set the entropy coder using a named value from the table (recommended) or its integer. The range is <code class="code">-1</code> to <code class="code">2</code>; the default is ‘<samp class="samp">default</samp>’ (-1). <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Leave

the preset entropy coding mode unchanged (-1). </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection (0). </dd> <dt>‘<samp class="samp">cabac</samp>’</dt> <dt>‘<samp class="samp">ac</samp>’</dt> <dd> CABAC (1). </dd> <dt>‘<samp class="samp">cavlc</samp>’</dt> <dt>‘<samp class="samp">vlc</samp>’</dt> <dd> CAVLC (2). </dd> </dl> </dd> </dl>

 <h4 class="subsection">16.29.7 HEVC options# TOC</h4>

<dl class="table"> <dt><samp class="option">profile <var class="var">integer</var></samp></dt> <dd> Set the HEVC profile using a named value from the table (recommended) or its integer. The default is ‘<samp class="samp">main</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dd> main profile (0). </dd> <dt>‘<samp class="samp">main10</samp>’</dt> <dd> main10 profile (1). </dd> <dt>‘<samp class="samp">rext</samp>’</dt> <dd> rext profile (2). </dd> <dt>‘<samp class="samp">mv</samp>’</dt> <dd> mv profile (3). </dd> </dl>

The ‘<samp class="samp">mv</samp>’ value requires SDK 13.0 or newer. </dd> <dt><samp class="option">level <var class="var">integer</var></samp></dt> <dd> Set the HEVC level restriction using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">186</code>; the default is ‘<samp class="samp">auto</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection (0). </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> Level 1 (30). </dd> <dt>‘<samp class="samp">1.0</samp>’</dt> <dd> Level 1.0 (30). </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> Level 2 (60). </dd> <dt>‘<samp class="samp">2.0</samp>’</dt> <dd> Level 2.0 (60). </dd> <dt>‘<samp class="samp">2.1</samp>’</dt> <dd> Level 2.1 (63). </dd> <dt>‘<samp class="samp">3</samp>’</dt> <dd> Level 3 (90). </dd> <dt>‘<samp class="samp">3.0</samp>’</dt> <dd> Level 3.0 (90). </dd> <dt>‘<samp class="samp">3.1</samp>’</dt> <dd> Level 3.1 (93). </dd> <dt>‘<samp class="samp">4</samp>’</dt> <dd> Level 4 (120). </dd> <dt>‘<samp class="samp">4.0</samp>’</dt> <dd> Level 4.0 (120). </dd> <dt>‘<samp class="samp">4.1</samp>’</dt> <dd> Level 4.1 (123). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> Level 5 (150). </dd> <dt>‘<samp class="samp">5.0</samp>’</dt> <dd> Level 5.0 (150). </dd> <dt>‘<samp class="samp">5.1</samp>’</dt> <dd> Level 5.1 (153). </dd> <dt>‘<samp class="samp">5.2</samp>’</dt> <dd> Level 5.2 (156). </dd> <dt>‘<samp class="samp">6</samp>’</dt> <dd> Level 6 (180). </dd> <dt>‘<samp class="samp">6.0</samp>’</dt>

<dd> Level 6.0 (180). </dd> <dt>‘<samp class=”samp”>6.1</samp>’</dt>
<dd> Level 6.1 (183). </dd> <dt>‘<samp class=”samp”>6.2</samp>’</dt>
<dd> Level 6.2 (186). </dd> </dl>

</dd> <dt><samp class=”option”>tier <var class=”var”>integer</var></samp></dt>
<dd> Set the HEVC encoding tier using a named value from the table
(recommended) or its integer. The range is <code class=”code”>0</code> to
<code class=”code”>1</code>; the default is ‘<samp class=”samp”>main</samp>’
(0). <dl class=”table”> <dt>‘<samp class=”samp”>main</samp>’</dt>
<dd> Main tier (0). </dd> <dt>‘<samp class=”samp”>high</samp>’</dt>
<dd> High tier (1). </dd> </dl>

</dd> <dt><samp class=”option”>unidir_b <var class=”var”>boolean</var></samp></dt>
<dd> Enable unidirectional B-frames. Requires SDK 12.2 or newer. The
default is 0. </dd> </dl>

 <h4 class=”subsection”>16.29.8 AV1
options<a class=”anchor hidden-xs” href=”#AV1-
options” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
AV1-options” aria-hidden=”true”>TOC</h4>

<dl class=”table”>

<dt><samp class=”option”>level <var class=”var”>integer</var></samp></dt>

<dd> Set the AV1 level restriction using a named value from the table

(recommended) or its integer. The range is <code class=”code”>0</code> to
<code class=”code”>24</code>; the

default is ‘<samp class=”samp”>auto</samp>’ (24).

<dl class=”table”>

<dt>‘<samp class=”samp”>2</samp>’</dt>

<dd> Level 2 (0).

</dd>

<dt>‘<samp class=”samp”>2.0</samp>’</dt>

<dd> Level 2.0 (0).

</dd>

<dt>‘<samp class=”samp”>2.1</samp>’</dt>

<dd> Level 2.1 (1).

</dd>

<dt>‘<samp class=”samp”>2.2</samp>’</dt>

<dd> Level 2.2 (2).
</dd>
<dt>‘<samp class="samp">2.3</samp>’</dt>
<dd> Level 2.3 (3).
</dd>
<dt>‘<samp class="samp">3</samp>’</dt>
<dd> Level 3 (4).
</dd>
<dt>‘<samp class="samp">3.0</samp>’</dt>
<dd> Level 3.0 (4).
</dd>
<dt>‘<samp class="samp">3.1</samp>’</dt>
<dd> Level 3.1 (5).
</dd>
<dt>‘<samp class="samp">3.2</samp>’</dt>
<dd> Level 3.2 (6).
</dd>
<dt>‘<samp class="samp">3.3</samp>’</dt>
<dd> Level 3.3 (7).
</dd>
<dt>‘<samp class="samp">4</samp>’</dt>
<dd> Level 4 (8).
</dd>
<dt>‘<samp class="samp">4.0</samp>’</dt>
<dd> Level 4.0 (8).
</dd>
<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> Level 4.1 (9).
</dd>
<dt>‘<samp class="samp">4.2</samp>’</dt>
<dd> Level 4.2 (10).

</dd>
<dt>‘<samp class="samp">4.3</samp>’</dt>
<dd> Level 4.3 (11).
</dd>
<dt>‘<samp class="samp">5</samp>’</dt>
<dd> Level 5 (12).
</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> Level 5.0 (12).
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> Level 5.1 (13).
</dd>
<dt>‘<samp class="samp">5.2</samp>’</dt>
<dd> Level 5.2 (14).
</dd>
<dt>‘<samp class="samp">5.3</samp>’</dt>
<dd> Level 5.3 (15).
</dd>
<dt>‘<samp class="samp">6</samp>’</dt>
<dd> Level 6 (16).
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> Level 6.0 (16).
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> Level 6.1 (17).
</dd>
<dt>‘<samp class="samp">6.2</samp>’</dt>
<dd> Level 6.2 (18).
</dd>

<dt>‘<samp class="samp">6.3</samp>’</dt>

<dd> Level 6.3 (19).

</dd>

<dt>‘<samp class="samp">7</samp>’</dt>

<dd> Level 7 (20).

</dd>

<dt>‘<samp class="samp">7.0</samp>’</dt>

<dd> Level 7.0 (20).

</dd>

<dt>‘<samp class="samp">7.1</samp>’</dt>

<dd> Level 7.1 (21).

</dd>

<dt>‘<samp class="samp">7.2</samp>’</dt>

<dd> Level 7.2 (22).

</dd>

<dt>‘<samp class="samp">7.3</samp>’</dt>

<dd> Level 7.3 (23).

</dd>

<dt>‘<samp class="samp">auto</samp>’</dt>

<dd> Automatic selection (24).

</dd>

</dl>

</dd> <dt><samp class="option">tier <var class="var">integer</var></samp></dt>
<dd> Set the AV1 encoding tier, from <code class="code">0</code> to
<code class="code">1</code>. The default is 0. <dl class="table">
<dt>‘<samp class="samp">0</samp>’</dt> <dd> Tier
0. </dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> Tier 1. </dd> </dl>

</dd> <dt><samp class="option">tile-rows <var class="var">integer</var></samp></dt>
<dt><samp class="option">tile-columns <var class="var">integer</var></samp></dt>
<dd> Set the number of tile rows and columns, respectively. Each accepts <code class="code">-1</code> to <code class="code">64</code>
and defaults to <code class="code">-1</code> (no override). </dd>
<dt><samp class="option">timing-info <var class="var">boolean</var></samp></dt>

<dd> Include timing information in sequence and frame headers. The default is 0. </dd> </dl>

 <h3 class="section">16.30 png# TOC</h3>

PNG image encoder. <h4 class="subsection">16.30.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">compression_level</samp></dt> <dd> Sets the compression level, from 0 to 9(default) </dd> </dl>

 <h4 class="subsection">16.30.2 Private options# TOC</h4>

<dl class="table"> <dt><samp class="option">dpi <var class="var">integer</var></samp></dt> <dd> Set physical density of pixels, in dots per inch, unset by default </dd> <dt><samp class="option">dpm <var class="var">integer</var></samp></dt> <dd> Set physical density of pixels, in dots per meter, unset by default </dd> <dt><samp class="option">pred <var class="var">method</var></samp></dt> <dd> Set prediction method (none, sub, up, avg, paeth, mixed), default is paeth </dd> </dl>

 <h3 class="section">16.31 ProRes# TOC</h3>

Apple ProRes encoder. FFmpeg contains 2 ProRes encoders, the prores-aw and prores-ks encoder. The used encoder can be chosen with the <code class="code">-vcodec</code> option. <h4 class="subsection">16.31.1 Private Options for prores-ks# TOC</h4>

<dl class="table"> <dt><samp class="option">profile <var class="var">integer</var></samp></dt> <dd> Select the ProRes profile to encode <dl class="table"> <dt>‘<samp class="samp">proxy</samp>’</dt> <dt>‘<samp class="samp">lt</samp>’</dt> <dt>‘<samp class="samp">standard</samp>’</dt <dt>‘<samp class="samp">hq</samp>’</dt> <dt>‘<samp class="samp">4444</samp>’</dt> <dt>‘<samp class="samp">4444xq</samp>’</d </dl>

</dd> <dt><samp class="option">quant_mat <var class="var">integer</var></samp></dt>
<dd> Select quantization matrix. <dl class="table"> <dt>‘<samp
class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">default</samp>’</d
<dt>‘<samp class="samp">proxy</samp>’</dt> <dt>‘<samp
class="samp">lt</samp>’</dt> <dt>‘<samp class="samp">standard</samp>’</dt
<dt>‘<samp class="samp">hq</samp>’</dt> </dl> If set to <var class="var">auto</var>, the matrix matching the profile will be picked. If not set, the matrix providing the highest quality, <var class="var">default</var>, will be picked. </dd> <dt><samp class="option">bits_per_mb <var class="var">integer</var></samp></dt>
<dd> How many bits to allot for coding one macroblock. Different profiles use between 200 and 2400 bits per macroblock, the maximum is 8000. </dd> <dt><samp class="option">mbs_per_slice <var class="var">integer</var></samp></dt> <dd> Number of macroblocks in each slice (1-8); the default value (8) should be good in almost all situations. </dd> <dt><samp class="option">vendor <var class="var">string</var></samp></dt> <dd> Override the 4-byte vendor ID. A custom vendor ID like <var class="var">apl0</var> would claim the stream was produced by the Apple encoder. </dd> <dt><samp class="option">alpha_bits <var class="var">integer</var></samp></dt>
<dd> Specify number of bits for alpha component. Possible values are <var class="var">0</var>, <var class="var">8</var> and <var class="var">16</var>. Use <var class="var">0</var> to disable alpha plane coding. </dd> </dl>

 <h4 class="subsection">16.31.2 Speed considerations# TOC</h4>

In the default mode of operation the encoder has to honor frame constraints (i.e. not produce frames with size bigger than requested) while still making output picture as good as possible. A frame containing a lot of small details is harder to compress and the encoder would spend more time searching for appropriate quantizers for each slice.
Setting a higher <samp class="option">bits_per_mb</samp> limit will improve the speed. For the fastest encoding speed set the <samp class="option">qscale</samp> parameter (4 is the recommended value) and do not set a size constraint. <h3 class="section">16.32 QSV Encoders# TOC</h3>

The family of Intel QuickSync Video encoders (MPEG-2, H.264, HEVC, JPEG/MJPEG, VP9, AV1) <h4 class="subsection">16.32.1 Ratecontrol Method<span class="pull-

right”># TOC</h4> The ratecontrol method is selected as follows: <ul class=”itemize mark-bullet”> When <samp class=”option”>global_quality</samp> is specified, a quality-based mode is used. Specifically this means either <ul class=”itemize mark-minus”> <var class=”var”>CQP</var> - constant quantizer scale, when the <samp class=”option”>qscale</samp> codec flag is also set (the <samp class=”option”>-qscale</samp> ffmpeg option).

<var class=”var”>LA_ICQ</var> - intelligent constant quality with lookahead, when the <samp class=”option”>look_ahead</samp> option is also set.

<var class=”var”>ICQ</var> – intelligent constant quality otherwise. For the ICQ modes, global quality range is 1 to 51, with 1 being the best quality.

Otherwise when the desired average bitrate is specified with the <samp class=”option”>b</samp> option, a bitrate-based mode is used. <ul class=”itemize mark-minus”> <var class=”var”>LA</var> - VBR with lookahead, when the <samp class=”option”>look_ahead</samp> option is specified.

<var class=”var”>VCM</var> - video conferencing mode, when the <samp class=”option”>vcm</samp> option is set.

<var class=”var”>CBR</var> - constant bitrate, when <samp class=”option”>maxrate</samp> is specified and equal to the average bitrate.

<var class=”var”>VBR</var> - variable bitrate, when <samp class=”option”>maxrate</samp> is specified, but is higher than the average bitrate.

<var class=”var”>AVBR</var> - average VBR mode, when <samp class=”option”>maxrate</samp> is not specified, both <samp class=”option”>avbr_accuracy</samp> and <samp class=”option”>avbr_convergence</samp> are set to non-zero. This mode is available for H264 and HEVC on Windows.

Otherwise the default ratecontrol method <var class=”var”>CQP</var> is used.

Note that depending on your system, a different mode than the one you specified may be selected by the encoder. Set the verbosity level to <var class=”var”>verbose</var> or higher to see the actual settings used by the QSV runtime. <h4 class=”subsection”>16.32.2 Global Options -> MSDK Options<a class=”anchor

hidden-xs" href="#Global-Options-_002d_003e-MSDK-Options" aria-hidden="true"># TOC</h4> Additional libavcodec global options are mapped to MSDK options as follows: <ul class="itemize mark-bullet"> <samp class="option">g/gop_size</samp> -> <samp class="option">GopPicSize</samp>

<samp class="option">bf/max_b_frames</samp>+1 -> <samp class="option">GopRefDist</samp>

<samp class="option">rc_init_occupancy/rc_initial_buffer_occupancy</samp> -> <samp class="option">InitialDelayInKB</samp>

<samp class="option">slices</samp> -> <samp class="option">NumSlice</samp>

<samp class="option">refs</samp> -> <samp class="option">NumRefFrame</samp>

<samp class="option">b_strategy/b_frame_strategy</samp> -> <samp class="option">BRefType</samp>

<samp class="option">cgop/CLOSED_GOP</samp> codec flag -> <samp class="option">GopOptFlag</samp>

For the <var class="var">CQP</var> mode, the <samp class="option">i_qfactor/i_qoffset</samp> and <samp class="option">b_qfactor/b_qoffset</samp> set the difference between <var class="var">QPP</var> and <var class="var">QPI</var>, and <var class="var">QPP</var> and <var class="var">QPB</var> respectively.

Setting the <samp class="option">coder</samp> option to the value <var class="var">vlc</var> will make the H.264 encoder use CAVLC instead of CABAC.

 <h4 class="subsection">16.32.3 Common Options# TOC</h4> Following options are used by all qsv encoders. <dl class="table"> <dt><samp class="option"><var class="var">async_depth</var></samp></dt> <dd> Specifies how many asynchronous operations an application performs before the application explicitly synchronizes the result. If zero, the value is not specified. </dd> <dt><samp class="option"><var class="var">preset</var></samp></dt> <dd> This option itemizes a range of choices from veryfast (best speed) to veryslow (best quality). <dl class="table"> <dt>‘<samp class="samp">veryfast</samp>’</dt> <dt>‘<samp class="samp">faster</samp>’</dt> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">medium</samp>’</d <dt>‘<samp class="samp">slow</samp>’</dt> <dt>‘<samp

class=”samp”>slower</samp>’</dt> <dt>‘<samp class=”samp”>veryslow</samp>’
</dl>

</dd> <dt><samp class=”option”><var class=”var”>forced_idr</var></samp></dt>
<dd> Forcing I frames as IDR frames. </dd> <dt><samp
class=”option”><var class=”var”>low_power</var></samp></dt>
<dd> For encoders set this flag to ON to reduce power consumption
and GPU usage. </dd> </dl>

 <h4 class=”subsection”>16.32.4
Runtime Options<a class=”anchor hidden-xs”
href=”#Runtime-Options” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-Runtime-Options” aria-hidden=”true”>TOC</h4>
Following options can be used during qsv encoding. <dl
class=”table”> <dt><samp class=”option”><var class=”var”>global_quality</var></samp></dt>
<dt><samp class=”option”><var class=”var”>i_quant_factor</var></samp></dt>
<dt><samp class=”option”><var class=”var”>i_quant_offset</var></samp></dt>
<dt><samp class=”option”><var class=”var”>b_quant_factor</var></samp></dt>
<dt><samp class=”option”><var class=”var”>b_quant_offset</var></samp></dt>
<dd> Supported in h264_qsv and hevc_qsv. Change these value to
reset qsv codec’s qp configuration. </dd> <dt><samp
class=”option”><var class=”var”>max_frame_size</var></samp></dt>
<dd> Supported in h264_qsv and hevc_qsv. Change this value to re-
set qsv codec’s MaxFrameSize configuration. </dd> <dt><samp
class=”option”><var class=”var”>gop_size</var></samp></dt> <dd> Change
this value to reset qsv codec’s gop configuration. </dd>
<dt><samp class=”option”><var class=”var”>int_ref_type</var></samp></dt>
<dt><samp class=”option”><var class=”var”>int_ref_cycle_size</var></samp></dt>
<dt><samp class=”option”><var class=”var”>int_ref_qp_delta</var></samp></dt>
<dt><samp class=”option”><var class=”var”>int_ref_cycle_dist</var></samp></dt>
<dd> Supported in h264_qsv and hevc_qsv. Change these value to reset
qsv codec’s Intra Refresh configuration. </dd> <dt><samp
class=”option”><var class=”var”>qmax</var></samp></dt> <dt><samp
class=”option”><var class=”var”>qmin</var></samp></dt> <dt><samp
class=”option”><var class=”var”>max_qp_i</var></samp></dt>
<dt><samp class=”option”><var class=”var”>min_qp_i</var></samp></dt>
<dt><samp class=”option”><var class=”var”>max_qp_p</var></samp></dt>
<dt><samp class=”option”><var class=”var”>min_qp_p</var></samp></dt>
<dt><samp class=”option”><var class=”var”>max_qp_b</var></samp></dt>
<dt><samp class=”option”><var class=”var”>min_qp_b</var></samp></dt>
<dd> Supported in h264_qsv. Change these value to reset qsv
codec’s max/min qp configuration. </dd> <dt><samp
class=”option”><var class=”var”>low_delay_brc</var></samp></dt>
<dd> Supported in h264_qsv, hevc_qsv and av1_qsv. Change this
value to reset qsv codec’s low_delay_brc configuration. </dd>
<dt><samp class=”option”><var class=”var”>framerate</var></samp></dt>
<dd> Change this value to reset qsv codec’s framerate configuration.

 </dd> <dt><samp class=”option”><var class=”var”>bit_rate</var></samp></dt>
<dt><samp class=”option”><var class=”var”>rc_buffer_size</var></samp></dt>
<dt><samp class=”option”><var class=”var”>rc_initial_buffer_occupancy</var></samp></dt>
<dt><samp class=”option”><var class=”var”>rc_max_rate</var></samp></dt>
<dd> Change these value to reset qsv codec’s bitrate control configuration. </dd> <dt><samp class=”option”><var class=”var”>pic_timing_sei</var></samp></dt> <dd> Supported in h264_qsv and hevc_qsv. Change this value to reset qsv codec’s pic_timing_sei configuration. </dd> <dt><samp class=”option”><var class=”var”>qsv_params</var></samp></dt> <dd> Set QSV encoder parameters as a colon-separated list of key-value pairs. The <samp class=”option”>qsv_params</samp> should be formatted as <code class=”code”>key1=value1:key2=value2:...</code>. These parameters are passed directly to the underlying Intel Quick Sync Video (QSV) encoder using the MFXSetParameter function. Example: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -c:v h264_qsv -qsv_params "CodingOption1=1:CodingOption2=2" output.mp4 </pre>

This option allows fine-grained control over various encoder-specific settings provided by the QSV encoder. </dd> </dl>

<h4 class=”subsection”>16.32.5 H264 options# TOC</h4>

These options are used by h264_qsv

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>extbrc</var></samp></dt>

<dd> Extended bitrate control.

</dd>

<dt><samp class=”option”><var class=”var”>recovery_point_sei</var></samp></dt>

<dd> Set this flag to insert the recovery point SEI message at the beginning of every

intra refresh cycle.

</dd>

<dt><samp class=”option”><var class=”var”>rdo</var></samp></dt>

<dd> Enable rate distortion optimization.

</dd>

<dt><samp class="option"><var class="var">max_frame_size</var></samp></dt>

<dd> Maximum encoded frame size in bytes.

</dd>

<dt><samp class="option"><var class="var">max_frame_size_i</var></samp></dt>

<dd> Maximum encoded frame size for I frames in bytes. If this value is set as larger

than zero, then for I frames the value set by max_frame_size is ignored.

</dd>

<dt><samp class="option"><var class="var">max_frame_size_p</var></samp></dt>

<dd> Maximum encoded frame size for P frames in bytes. If this value is set as larger

than zero, then for P frames the value set by max_frame_size is ignored.

</dd>

<dt><samp class="option"><var class="var">max_slice_size</var></samp></dt>

<dd> Maximum encoded slice size in bytes.

</dd>

<dt><samp class="option"><var class="var">bitrate_limit</var></samp></dt>

<dd> Toggle bitrate limitations.

Modifies bitrate to be in the range imposed by the QSV encoder. Setting this flag off may lead to violation of HRD conformance. Mind that specifying bitrate below the QSV encoder range might significantly affect quality. If on this option takes effect in non CQP modes: if bitrate is not in the range imposed by the QSV encoder, it will be changed to be in the range.

</dd>

<dt><samp class=”option”><var class=”var”>mbbrc</var></samp></dt>

<dd> Setting this flag enables macroblock level bitrate control that generally

improves subjective visual quality. Enabling this flag may have negative impact

on performance and objective visual quality metric.

</dd>

<dt><samp class=”option”><var class=”var”>low_delay_brc</var></samp></dt>

<dd> Setting this flag turns on or off LowDelayBRC feature in qsv plugin, which provides

more accurate bitrate control to minimize the variance of bitstream size frame

by frame. Value: -1-default 0-off 1-on

</dd>

<dt><samp class=”option”><var class=”var”>adaptive_i</var></samp></dt>

<dd> This flag controls insertion of I frames by the QSV encoder. Turn ON this flag

to allow changing of frame type from P and B to I.

</dd>

<dt><samp class=”option”><var class=”var”>adaptive_b</var></samp></dt>

<dd> This flag controls changing of frame type from B to P.

</dd>

<dt><samp class=”option”><var class=”var”>p_strategy</var></samp></dt>

<dd> Enable P-pyramid: 0-default 1-simple 2-pyramid(bf need to be set to 0).

</dd>

<dt><samp class=”option”><var class=”var”>b_strategy</var></samp></dt>

<dd> This option controls usage of B frames as reference.

</dd>
<dt><samp class=”option”><var class=”var”>dblk_idc</var></samp></dt>
<dd> This option disable deblocking. It has value in range 0~2.

</dd>
<dt><samp class=”option”><var class=”var”>cavlc</var></samp></dt>
<dd> If set, CAVLC is used; if unset, CABAC is used for encoding.

</dd>
<dt><samp class=”option”><var class=”var”>vcm</var></samp></dt>
<dd> Video conferencing mode, please see ratecontrol method.

</dd>
<dt><samp class=”option”><var class=”var”>idr_interval</var></samp></dt>
<dd> Distance (in I-frames) between IDR frames.

</dd>
<dt><samp class=”option”><var class=”var”>pic_timing_sei</var></samp></dt>
<dd> Insert picture timing SEI with pic_struct_syntax element.

</dd>
<dt><samp class=”option”><var class=”var”>single_sei_nal_unit</var></samp></dt>
<dd> Put all the SEI messages into one NALU.

</dd>
<dt><samp class=”option”><var class=”var”>max_dec_frame_buffering</var></samp></dt>
<dd> Maximum number of frames buffered in the DPB.

</dd>
<dt><samp class=”option”><var class=”var”>look_ahead</var></samp></dt>

<dd> Use VBR algorithm with look ahead.

</dd>

<dt><samp class=”option”><var class=”var”>look_ahead_depth</var></samp></dt>

<dd> Depth of look ahead in number frames.

</dd>

<dt><samp class=”option”><var class=”var”>look_ahead_downsampling</var></samp></dt>

<dd> Downscaling factor for the frames saved for the lookahead analysis.

<dl class=”table”>

<dt>‘<samp class=”samp”>unknown</samp>’</dt>

<dt>‘<samp class=”samp”>auto</samp>’</dt>

<dt>‘<samp class=”samp”>off</samp>’</dt>

<dt>‘<samp class=”samp”>2x</samp>’</dt>

<dt>‘<samp class=”samp”>4x</samp>’</dt>

</dl>

</dd> <dt><samp class=”option”><var class=”var”>int_ref_type</var></samp></dt>
<dd> Specifies intra refresh type. The major goal of intra refresh is improvement of error resilience without significant impact on encoded bitstream size caused by I frames. The SDK encoder achieves this by encoding part of each frame in refresh cycle using intra MBs. <var class=”var”>none</var> means no refresh. <var class=”var”>vertical</var> means vertical refresh, by column of MBs. <var class=”var”>horizontal</var> means horizontal refresh, by rows of MBs. <var class=”var”>slice</var> means horizontal refresh by slices without overlapping. In case of <var class=”var”>slice</var>, in_ref_cycle_size is ignored. To enable intra refresh, B frame should be set to 0. </dd> <dt><samp class=”option”><var class=”var”>int_ref_cycle_size</var></samp></dt>
<dd> Specifies number of pictures within refresh cycle starting from 2. 0 and 1 are invalid values. </dd> <dt><samp class=”option”><var class=”var”>int_ref_qp_delta</var></samp></dt> <dd> Specifies QP difference for inserted intra MBs. This is signed value in [-51, 51] range if target encoding bit-depth for luma samples is 8 and this range is [-63, 63] for 10 bit-depth or [-75, 75] for 12 bit-depth respectively. </dd> <dt><samp class=”option”><var class=”var”>int_ref_cycle_dist</var></samp></dt>
<dd> Distance between the beginnings of the intra-refresh cycles in frames. </dd> <dt><samp class=”option”><var class=”var”>profile</var></samp></dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt>

<dt>‘<samp class="samp">baseline</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt>
</dl>

</dd> <dt><samp class="option"><var class="var">a53cc</var></samp></dt>
<dd> Use A53 Closed Captions (if available). </dd> <dt><samp class="option"><var class="var">aud</var></samp></dt> <dd> Insert the Access Unit Delimiter NAL. </dd> <dt><samp class="option"><var class="var">mfmode</var></samp></dt> <dd> Multi-Frame Mode. <dl class="table"> <dt>‘<samp class="samp">off</samp>’</dt>
<dt>‘<samp class="samp">auto</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">repeat_pps</var></samp></dt>
<dd> Repeat pps for every frame. </dd> <dt><samp class="option"><var class="var">max_qp_i</var></samp></dt>
<dd> Maximum video quantizer scale for I frame. </dd>
<dt><samp class="option"><var class="var">min_qp_i</var></samp></dt>
<dd> Minimum video quantizer scale for I frame. </dd>
<dt><samp class="option"><var class="var">max_qp_p</var></samp></dt>
<dd> Maximum video quantizer scale for P frame. </dd>
<dt><samp class="option"><var class="var">min_qp_p</var></samp></dt>
<dd> Minimum video quantizer scale for P frame. </dd>
<dt><samp class="option"><var class="var">max_qp_b</var></samp></dt>
<dd> Maximum video quantizer scale for B frame. </dd>
<dt><samp class="option"><var class="var">min_qp_b</var></samp></dt>
<dd> Minimum video quantizer scale for B frame. </dd>
<dt><samp class="option"><var class="var">scenario</var></samp></dt>
<dd> Provides a hint to encoder about the scenario for the encoding session.
<dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt>
<dt>‘<samp class="samp">displayremoting</samp>’</dt>
<dt>‘<samp class="samp">videoconference</samp>’</dt>
<dt>‘<samp class="samp">archive</samp>’</dt> <dt>‘<samp class="samp">livestreaming</samp>’</dt> <dt>‘<samp class="samp">cameracapture</samp>’</dt> <dt>‘<samp class="samp">videosurveillance</samp>’</dt> <dt>‘<samp class="samp">gamestreaming</samp>’</dt> <dt>‘<samp class="samp">remotegaming</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">avbr_accuracy</var></samp></dt>
<dd> Accuracy of the AVBR ratecontrol (unit of tenth of percent).
</dd> <dt><samp class="option"><var class="var">avbr_convergence</var></samp></dt>
<dd> Convergence of the AVBR ratecontrol (unit of 100 frames)
The parameters <var class="var">avbr_accuracy</var> and <var class="var">avbr_convergence</var> are for the average variable bitrate control (AVBR) algorithm. The algorithm focuses on overall encoding quality while meeting the specified bitrate, <var class="var">target_bitrate</var>, within the accuracy range <var class="var">avbr_accuracy</var>, after a

<var class=”var”>avbr_Convergence</var> period. This method does not follow HRD and the instant bitrate is not capped or padded. </dd> <dt><samp class=”option”><var class=”var”>skip_frame</var></samp></dt> <dd> Use per-frame metadata "qsv_skip_frame" to skip frame when encoding. This option defines the usage of this metadata. <dl class=”table”> <dt>‘<samp class=”samp”>no_skip</samp>’</dt> <dd> Frame skipping is disabled. </dd> <dt>‘<samp class=”samp”>insert_dummy</samp>’</dt> <dd> Encoder inserts into bitstream frame where all macroblocks are encoded as skipped. </dd> <dt>‘<samp class=”samp”>insert_nothing</samp>’</dt> <dd> Similar to insert_dummy, but encoder inserts nothing into bitstream. The skipped frames are still used in brc. For example, gop still include skipped frames, and the frames after skipped frames will be larger in size. </dd> <dt>‘<samp class=”samp”>brc_only</samp>’</dt> <dd> skip_frame metadata indicates the number of missed frames before the current frame. </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>16.32.6 HEVC Options# TOC</h4> These options are used by hevc_qsv <dl class=”table”> <dt><samp class=”option”><var class=”var”>extbrc</var></samp></dt> <dd> Extended bitrate control. </dd> <dt><samp class=”option”><var class=”var”>recovery_point_sei</var></samp></dt> <dd> Set this flag to insert the recovery point SEI message at the beginning of every intra refresh cycle. </dd> <dt><samp class=”option”><var class=”var”>rdo</var></samp></dt> <dd> Enable rate distortion optimization. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size</var></samp></dt> <dd> Maximum encoded frame size in bytes. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size_i</var></samp></dt> <dd> Maximum encoded frame size for I frames in bytes. If this value is set as larger than zero, then for I frames the value set by max_frame_size is ignored. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size_p</var></samp></dt> <dd> Maximum encoded frame size for P frames in bytes. If this value is set as larger than zero, then for P frames the value set by max_frame_size is ignored. </dd> <dt><samp class=”option”><var class=”var”>max_slice_size</var></samp></dt> <dd> Maximum encoded slice size in bytes. </dd> <dt><samp class=”option”><var class=”var”>mbbrc</var></samp></dt> <dd> Setting this flag enables macroblock level bitrate control that generally improves subjective visual quality. Enabling this flag may have negative impact on performance and objective visual quality metric. </dd> <dt><samp class=”option”><var class=”var”>low_delay_brc</var></samp></dt> <dd> Setting this

flag turns on or off LowDelayBRC feature in qsv plugin, which provides more accurate bitrate control to minimize the variance of bitstream size frame by frame. Value: -1-default 0-off 1-on </dd> <dt><samp class="option"><var class="var">adaptive_i</var></samp></dt> <dd> This flag controls insertion of I frames by the QSV encoder. Turn ON this flag to allow changing of frame type from P and B to I. </dd> <dt><samp class="option"><var class="var">adaptive_b</var></samp></dt> <dd> This flag controls changing of frame type from B to P. </dd> <dt><samp class="option"><var class="var">p_strategy</var></samp></dt> <dd> Enable P-pyramid: 0-default 1-simple 2-pyramid(bf need to be set to 0). </dd> <dt><samp class="option"><var class="var">b_strategy</var></samp></dt> <dd> This option controls usage of B frames as reference. </dd> <dt><samp class="option"><var class="var">dblk_idc</var></samp></dt> <dd> This option disable deblocking. It has value in range 0~2. </dd> <dt><samp class="option"><var class="var">idr_interval</var></samp></dt> <dd> Distance (in I-frames) between IDR frames. <dl class="table"> <dt>‘<samp class="samp">begin_only</samp>’</dt> <dd> Output an IDR-frame only at the beginning of the stream. </dd> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugin</var></samp></dt> <dd> A user plugin to load in an internal session. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">hevc_sw</samp>’</dt> <dt>‘<samp class="samp">hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugins</var></samp></dt> <dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session. </dd> <dt><samp class="option"><var class="var">look_ahead_depth</var></samp></dt> <dd> Depth of look ahead in number frames, available when extbrc option is enabled. </dd> <dt><samp class="option"><var class="var">profile</var></samp></dt> <dd> Set the encoding profile (scc requires libmfx >= 1.32). <dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">main10</samp>’</dt> <dt>‘<samp class="samp">mainsp</samp>’</dt> <dt>‘<samp class="samp">rext</samp>’</dt> <dt>‘<samp class="samp">scc</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">tier</var></samp></dt> <dd> Set the encoding tier (only level >= 4 can support high tier). This option only takes effect when the level option is specified. <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">gpb</var></samp></dt> <dd> 1: GPB (generalized P/B frame) 0: regular P frame.

 </dd> <dt><samp class=”option”><var class=”var”>tile_cols</var></samp></dt>
<dd> Number of columns for tiled encoding. </dd> <dt><samp
class=”option”><var class=”var”>tile_rows</var></samp></dt> <dd> Number
of rows for tiled encoding. </dd> <dt><samp class=”option”><var
class=”var”>aud</var></samp></dt> <dd> Insert the Access
Unit Delimiter NAL. </dd> <dt><samp class=”option”><var
class=”var”>pic_timing_sei</var></samp></dt> <dd> Insert picture
timing SEI with pic_struct_syntax element. </dd> <dt><samp
class=”option”><var class=”var”>transform_skip</var></samp></dt>
<dd> Turn this option ON to enable transformskip. It is supported on platform equal or newer than ICL. </dd> <dt><samp class=”option”><var
class=”var”>int_ref_type</var></samp></dt> <dd> Specifies intra
refresh type. The major goal of intra refresh is improvement of error resilience
without significant impact on encoded bitstream size caused by I frames. The
SDK encoder achieves this by encoding part of each frame in refresh cycle
using intra MBs. <var class=”var”>none</var> means no refresh. <var
class=”var”>vertical</var> means vertical refresh, by column of MBs. <var
class=”var”>horizontal</var> means horizontal refresh, by rows of MBs. <var
class=”var”>slice</var> means horizontal refresh by slices without overlapping. In case of <var class=”var”>slice</var>, in_ref_cycle_size is ignored.
To enable intra refresh, B frame should be set to 0. </dd> <dt><samp
class=”option”><var class=”var”>int_ref_cycle_size</var></samp></dt>
<dd> Specifies number of pictures within refresh cycle starting from 2. 0
and 1 are invalid values. </dd> <dt><samp class=”option”><var
class=”var”>int_ref_qp_delta</var></samp></dt> <dd> Specifies
QP difference for inserted intra MBs. This is signed value in [-51, 51] range if
target encoding bit-depth for luma samples is 8 and this range is [-63, 63] for 10
bit-depth or [-75, 75] for 12 bit-depth respectively. </dd> <dt><samp
class=”option”><var class=”var”>int_ref_cycle_dist</var></samp></dt>
<dd> Distance between the beginnings of the intra-refresh cycles in frames.
 </dd> <dt><samp class=”option”><var class=”var”>max_qp_i</var></samp></dt>
<dd> Maximum video quantizer scale for I frame. </dd>
<dt><samp class=”option”><var class=”var”>min_qp_i</var></samp></dt>
<dd> Minimum video quantizer scale for I frame. </dd>
<dt><samp class=”option”><var class=”var”>max_qp_p</var></samp></dt>
<dd> Maximum video quantizer scale for P frame. </dd>
<dt><samp class=”option”><var class=”var”>min_qp_p</var></samp></dt>
<dd> Minimum video quantizer scale for P frame. </dd>
<dt><samp class=”option”><var class=”var”>max_qp_b</var></samp></dt>
<dd> Maximum video quantizer scale for B frame. </dd>
<dt><samp class=”option”><var class=”var”>min_qp_b</var></samp></dt>
<dd> Minimum video quantizer scale for B frame. </dd>
<dt><samp class=”option”><var class=”var”>scenario</var></samp></dt>
<dd> Provides a hint to encoder about the scenario for the encoding session.
<dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt>
<dt>‘<samp class=”samp”>displayremoting</samp>’</dt>

<dt>‘<samp class=”samp”>videoconference</samp>’</dt>
<dt>‘<samp class=”samp”>archive</samp>’</dt> <dt>‘<samp
class=”samp”>livestreaming</samp>’</dt> <dt>‘<samp
class=”samp”>cameracapture</samp>’</dt> <dt>‘<samp
class=”samp”>videosurveillance</samp>’</dt> <dt>‘<samp
class=”samp”>gamestreaming</samp>’</dt> <dt>‘<samp
class=”samp”>remotegaming</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>avbr_accuracy</var></samp></dt>
<dd> Accuracy of the AVBR ratecontrol (unit of tenth of percent).
</dd> <dt><samp class=”option”><var class=”var”>avbr_convergence</var></samp></dt>
<dd> Convergence of the AVBR ratecontrol (unit of 100 frames)
The parameters <var class=”var”>avbr_accuracy</var> and <var
class=”var”>avbr_convergence</var> are for the average variable bitrate
control (AVBR) algorithm. The algorithm focuses on overall encoding quality
while meeting the specified bitrate, <var class=”var”>target_bitrate</var>,
within the accuracy range <var class=”var”>avbr_accuracy</var>, after a
<var class=”var”>avbr_Convergence</var> period. This method does not
follow HRD and the instant bitrate is not capped or padded. </dd>
<dt><samp class=”option”><var class=”var”>skip_frame</var></samp></dt>
<dd> Use per-frame metadata "qsv_skip_frame" to skip frame
when encoding. This option defines the usage of this metadata. <dl
class=”table”> <dt>‘<samp class=”samp”>no_skip</samp>’</dt>
<dd> Frame skipping is disabled. </dd> <dt>‘<samp
class=”samp”>insert_dummy</samp>’</dt> <dd> Encoder
inserts into bitstream frame where all macroblocks are encoded as skipped.
</dd> <dt>‘<samp class=”samp”>insert_nothing</samp>’</dt>
<dd> Similar to insert_dummy, but encoder inserts nothing into bitstream.
The skipped frames are still used in brc. For example, gop still include skipped
frames, and the frames after skipped frames will be larger in size. </dd>
<dt>‘<samp class=”samp”>brc_only</samp>’</dt> <dd> skip_frame
metadata indicates the number of missed frames before the current frame.
</dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>16.32.7
MPEG2 Options<a class=”anchor hidden-xs”
href=”#MPEG2-Options” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-MPEG2-Options” aria-hidden=”true”>TOC</h4>
These options are used by mpeg2_qsv <dl class=”table”>
<dt><samp class=”option”><var class=”var”>profile</var></samp></dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt>
<dt>‘<samp class=”samp”>simple</samp>’</dt> <dt>‘<samp
class=”samp”>main</samp>’</dt> <dt>‘<samp class=”samp”>high</samp>’</dt>
</dl> </dd> </dl>

 <h4 class=”subsection”>16.32.8 VP9

Options# TOC</h4> These options are used by vp9_qsv <dl class=”table”> <dt><samp class=”option”><var class=”var”>profile</var></samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>profile0</samp>’</dt> <dt>‘<samp class=”samp”>profile1</samp>’</dt> <dt>‘<samp class=”samp”>profile2</samp>’</dt> <dt>‘<samp class=”samp”>profile3</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>tile_cols</var></samp></dt> <dd> Number of columns for tiled encoding (requires libmfx >= 1.29). </dd> <dt><samp class=”option”><var class=”var”>tile_rows</var></samp></dt> <dd> Number of rows for tiled encoding (requires libmfx >= 1.29). </dd> </dl>

 <h4 class=”subsection”>16.32.9 AV1 Options# TOC</h4> These options are used by av1_qsv (requires libvpl). <dl class=”table”> <dt><samp class=”option”><var class=”var”>profile</var></samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>main</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>tile_cols</var></samp></dt> <dd> Number of columns for tiled encoding. </dd> <dt><samp class=”option”><var class=”var”>tile_rows</var></samp></dt> <dd> Number of rows for tiled encoding. </dd> <dt><samp class=”option”><var class=”var”>adaptive_i</var></samp></dt> <dd> This flag controls insertion of I frames by the QSV encoder. Turn ON this flag to allow changing of frame type from P and B to I. </dd> <dt><samp class=”option”><var class=”var”>adaptive_b</var></samp></dt> <dd> This flag controls changing of frame type from B to P. </dd> <dt><samp class=”option”><var class=”var”>b_strategy</var></samp></dt> <dd> This option controls usage of B frames as reference. </dd> <dt><samp class=”option”><var class=”var”>extbrc</var></samp></dt> <dd> Extended bitrate control. </dd> <dt><samp class=”option”><var class=”var”>look_ahead_depth</var></samp></dt> <dd> Depth of look ahead in number frames, available when extbrc option is enabled. </dd> <dt><samp class=”option”><var class=”var”>low_delay_brc</var></samp></dt> <dd> Setting this flag turns on or off LowDelayBRC feature in qsv plugin, which provides more accurate bitrate control to minimize the variance of bitstream size frame by frame. Value: -1-default 0-off 1-on </dd> <dt><samp class=”option”><var class=”var”>max_frame_size</var></samp></dt> <dd> Set the allowed max size in bytes for each frame. If the frame size exceeds the limitation, encoder will adjust the QP value to control the

frame size. Invalid in CQP rate control mode. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size_i</var></samp></dt> <dd> Maximum encoded frame size for I frames in bytes. If this value is set as larger than zero, then for I frames the value set by max_frame_size is ignored. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size_p</var></samp></dt> <dd> Maximum encoded frame size for P frames in bytes. If this value is set as larger than zero, then for P frames the value set by max_frame_size is ignored. </dd> </dl>

 <h3 class=”section”>16.33 snow# TOC</h3>

 <h4 class=”subsection”>16.33.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>iterative_dia_size</samp></dt> <dd> dia size for the iterative motion estimation </dd> </dl>

 <h3 class=”section”>16.34 VAAPI encoders# TOC</h3>

Wrappers for hardware encoders accessible via VAAPI. These encoders only accept input in VAAPI hardware surfaces. If you have input in software frames, use the <samp class=”option”>hwupload</samp> filter to upload them to the GPU. The following standard libavcodec options are used: <ul class=”itemize mark-bullet”> <samp class=”option”>g</samp> / <samp class=”option”>gop_size</samp> <samp class=”option”>bf</samp> / <samp class=”option”>max_b_frames</samp> <samp class=”option”>profile</samp>

If not set, this will be determined automatically from the format of the input frames and the profiles supported by the driver. <samp class=”option”>level</samp> <samp class=”option”>b</samp> / <samp class=”option”>bit_rate</samp> <samp class=”option”>maxrate</samp> / <samp class=”option”>rc_max_rate</samp> <samp class=”option”>bufsize</samp> / <samp class=”option”>rc_buffer_size</samp> <samp class=”option”>rc_init_occupancy</samp> / <samp class=”option”>rc_initial_buffer_occupancy</samp> <samp class=”option”>compression_level</samp>

Speed / quality tradeoff: higher values are faster / worse quality. <samp class=”option”>q</samp> / <samp class=”option”>global_quality</samp>

Size / quality tradeoff: higher values are smaller / worse quality. <samp class=”option”>qmin</samp> <samp

class=”option”>qmax</samp> <samp class=”option”>i_qfactor</samp> / <samp class=”option”>i_quant_factor</samp> <samp class=”option”>i_qoffset</samp> / <samp class=”option”>i_quant_offset</samp> <samp class=”option”>b_qfactor</samp> / <samp class=”option”>b_quant_factor</samp> <samp class=”option”>b_qoffset</samp> / <samp class=”option”>b_quant_offset</samp> <samp class=”option”>slices</samp>

All encoders support the following options: <dl class=”table”> <dt><samp class=”option”>low_power</samp></dt> <dd> Some drivers/platforms offer a second encoder for some codecs intended to use less power than the default encoder; setting this option will attempt to use that encoder. Note that it may support a reduced feature set, so some other options may not be available in this mode. </dd> <dt><samp class=”option”>idr_interval</samp></dt> <dd> Set the number of normal intra frames between full-refresh (IDR) frames in open-GOP mode. The intra frames are still IRAPs, but will not include global headers and may have non-decodable leading pictures. </dd> <dt><samp class=”option”>b_depth</samp></dt> <dd> Set the B-frame reference depth. When set to one (the default), all B-frames will refer only to P- or I-frames. When set to greater values multiple layers of B-frames will be present, frames in each layer only referring to frames in higher layers. </dd> <dt><samp class=”option”>async_depth</samp></dt> <dd> Maximum processing parallelism. Increase this to improve single channel performance. This option doesn’t work if driver doesn’t implement vaSyncBuffer function. Please make sure there are enough hw_frames allocated if a large number of async_depth is used. </dd> <dt><samp class=”option”>max_frame_size</samp></dt> <dd> Set the allowed max size in bytes for each frame. If the frame size exceeds the limitation, encoder will adjust the QP value to control the frame size. Invalid in CQP rate control mode. </dd> <dt><samp class=”option”>rc_mode</samp></dt> <dd> Set the rate control mode to use. A given driver may only support a subset of modes. Possible modes: <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Choose the mode automatically based on driver support and the other options. This is the default. </dd> <dt><samp class=”option”>CQP</samp></dt> <dd> Constant-quality. </dd> <dt><samp class=”option”>CBR</samp></dt> <dd> Constant-bitrate. </dd> <dt><samp class=”option”>VBR</samp></dt> <dd> Variable-bitrate. </dd> <dt><samp class=”option”>ICQ</samp></dt> <dd> Intelligent constant-quality. </dd> <dt><samp class=”option”>QVBR</samp></dt> <dd> Quality-defined variable-bitrate. </dd> <dt><samp class=”option”>AVBR</samp></dt> <dd> Average variable bitrate. </dd> </dl>

</dd> <dt><samp class=”option”>blbrc</samp></dt> <dd> Enable block level rate control, which assigns different bitrate block by block. Invalid for CQP mode. </dd> </dl>

Each encoder also has its own specific options: <dl class="table">
<dt><samp class="option">av1_vaapi</samp></dt> <dd> <samp
class="option">profile</samp> sets the value of seq_profile.
<samp class="option">tier</samp> sets the value of seq_tier.
<samp class="option">level</samp> sets the value of seq_level_idx.
 <dl class="table"> <dt><samp class="option">tiles</samp></dt>
<dd> Set the number of tiles to encode the input video with, as columns
x rows. (default is auto, which means use minimal tile column/row num-
ber). </dd> <dt><samp class="option">tile_groups</samp></dt>
<dd> Set tile groups number. All the tiles will be distributed as evenly as
possible to each tile group. (default is 1). </dd> </dl>

</dd> <dt><samp class="option">h264_vaapi</samp></dt> <dd> <samp
class="option">profile</samp> sets the value of profile_idc
and the constraint_set*_flags. <samp
class="option">level</samp> sets the value of level_idc.
 <dl class="table"> <dt><samp class="option">coder</samp></dt>
<dd> Set entropy encoder (default is cabac).
Possible values: <dl class="table"> <dt>‘<samp class="samp">ac</samp>’</dt>
<dt>‘<samp class="samp">cabac</samp>’</dt> <dd> Use
CABAC. </dd> <dt>‘<samp class="samp">vlc</samp>’</dt>
<dt>‘<samp class="samp">cavlc</samp>’</dt> <dd> Use
CAVLC. </dd> </dl>

</dd> <dt><samp class="option">aud</samp></dt> <dd> Include
access unit delimiters in the stream (not included by default). </dd>
<dt><samp class="option">sei</samp></dt> <dd> Set SEI mes-
sage types to include. Some combination of the following values: <dl
class="table"> <dt>‘<samp class="samp">identifier</samp>’</dt>
<dd> Include a user_data_unregistered mes-
sage containing information about the encoder. </dd> <dt>‘<samp
class="samp">timing</samp>’</dt> <dd> Include pic-
ture timing parameters (buffering_period
and pic_timing messages). </dd>
<dt>‘<samp class="samp">recovery_point</samp>’</dt>
<dd> Include recovery points where appropriate (recovery_point
messages). </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">hevc_vaapi</samp></dt> <dd> <samp
class="option">profile</samp> and <samp class="option">level</samp>
set the values of general_profile_idc and
general_level_idc respectively. <dl
class="table"> <dt><samp class="option">aud</samp></dt> <dd> Include
access unit delimiters in the stream (not included by default).
</dd> <dt><samp class="option">tier</samp></dt> <dd> Set <em
class="emph">general_tier_flag. This may affect the level chosen

for the stream if it is not explicitly specified. </dd> <dt><samp class=”option”>sei</samp></dt> <dd> Set SEI message types to include. Some combination of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>hdr</samp>’</dt> <dd> Include HDR metadata if the input frames have it (mastering_display_colour_volume and content_light_level messages). </dd> </dl>

</dd> <dt><samp class=”option”>tiles</samp></dt> <dd> Set the number of tiles to encode the input video with, as columns x rows. Larger numbers allow greater parallelism in both encoding and decoding, but may decrease coding efficiency. </dd> </dl>

</dd> <dt><samp class=”option”>mjpeg_vaapi</samp></dt> <dd> Only baseline DCT encoding is supported. The encoder always uses the standard quantisation and huffman tables - <samp class=”option”>global_quality</samp> scales the standard quantisation table (range 1-100). For YUV, 4:2:0, 4:2:2 and 4:4:4 subsampling modes are supported. RGB is also supported, and will create an RGB JPEG. <dl class=”table”> <dt><samp class=”option”>jfif</samp></dt> <dd> Include JFIF header in each frame (not included by default). </dd> <dt><samp class=”option”>huffman</samp></dt> <dd> Include standard huffman tables (on by default). Turning this off will save a few hundred bytes in each output frame, but may lose compatibility with some JPEG decoders which don’t fully handle MJPEG. </dd> </dl>

</dd> <dt><samp class=”option”>mpeg2_vaapi</samp></dt> <dd> <samp class=”option”>profile</samp> and <samp class=”option”>level</samp> set the value of profile_and_level_indication. </dd> <dt><samp class=”option”>vp8_vaapi</samp></dt> <dd> B-frames are not supported. <samp class=”option”>global_quality</samp> sets the q_idx used for non-key frames (range 0-127). <dl class=”table”> <dt><samp class=”option”>loop_filter_level</samp></dt> <dt><samp class=”option”>loop_filter_sharpness</samp></dt> <dd> Manually set the loop filter parameters. </dd> </dl>

</dd> <dt><samp class=”option”>vp9_vaapi</samp></dt> <dd> <samp class=”option”>global_quality</samp> sets the q_idx used for P-frames (range 0-255). <dl class=”table”> <dt><samp class=”option”>loop_filter_level</samp></dt> <dt><samp class=”option”>loop_filter_sharpness</samp> <dd> Manually set the loop filter parameters. </dd> </dl>

B-frames are supported, but the output stream is always in encode order rather than display order. If B-frames are enabled, it may be necessary to use the <samp class=”option”>vp9_raw_reorder</samp> bitstream filter to modify the output stream to display frames in the correct order. Only normal frames are produced - the <samp class=”option”>vp9_superframe</samp> bitstream filter may be required to

produce a stream usable with all decoders. </dd> </dl>

 <h3 class="section">16.35 vbn# TOC</h3>

Vizrt Binary Image encoder. This format is used by the broadcast vendor Vizrt for quick texture streaming. Advanced features of the format such as LZW compression of texture data or generation of mipmaps are not supported. <h4 class="subsection">16.35.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">format <var class="var">string</var></samp></dt> <dd> Sets the texture compression used by the VBN file. Can be <var class="var">dxt1</var>, <var class="var">dxt5</var> or <var class="var">raw</var>. Default is <var class="var">dxt5</var>. </dd> </dl>

 <h3 class="section">16.36 vc2# TOC</h3>

SMPTE VC-2 (previously BBC Dirac Pro). This codec was primarily aimed at professional broadcasting but since it supports yuv420, yuv422 and yuv444 at 8 (limited range or full range), 10 or 12 bits, this makes it suitable for other tasks which require low overhead and low compression (like screen recording). <h4 class="subsection">16.36.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Sets target video bitrate. Usually that’s around 1:6 of the uncompressed video bitrate (e.g. for 1920x1080 50fps yuv422p10 that’s around 400Mbps). Higher values (close to the uncompressed bitrate) turn on lossless compression mode. </dd> <dt><samp class="option">field_order</samp></dt> <dd> Enables field coding when set (e.g. to tt - top field first) for interlaced inputs. Should increase compression with interlaced content as it splits the fields and encodes each separately. </dd> <dt><samp class="option">wavelet_depth</samp></dt> <dd> Sets the total amount of wavelet transforms to apply, between 1 and 5 (default). Lower values reduce compression and quality. Less capable decoders may not be able to handle values of <samp class="option">wavelet_depth</samp> over 3. </dd> <dt><samp class="option">wavelet_type</samp></dt> <dd> Sets the transform type. Currently only <var class="var">5_3</var> (LeGall) and <var class="var">9_7</var> (Deslauriers-Dubuc) are implemented, with 9_7 being the one with better compression and thus is the de-

fault. </dd> <dt><samp class="option">slice_width</samp></dt> <dt><samp class="option">slice_height</samp></dt> <dd> Sets the slice size for each slice. Larger values result in better compression. For compatibility with other more limited decoders use <samp class="option">slice_width</samp> of 32 and <samp class="option">slice_height</samp> of 8. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Sets the undershoot tolerance of the rate control system in percent. This is to prevent an expensive search from being run. </dd> <dt><samp class="option">qm</samp></dt> <dd> Sets the quantization matrix preset to use by default or when <samp class="option">wavelet_depth</samp> is set to 5 <ul class="itemize mark-minus"> <var class="var">default</var> Uses the default quantization matrix from the specifications, extended with values for the fifth level. This provides a good balance between keeping detail and omitting artifacts.

<var class="var">flat</var> Use a completely zeroed out quantization matrix. This increases PSNR but might reduce perception. Use in bogus benchmarks.

<var class="var">color</var> Reduces detail but attempts to preserve color at extremely low bitrates.

</dd> </dl>

 <h2 class="chapter">17 Subtitles Encoders# TOC</h2>

 <h3 class="section">17.1 dvbsub# TOC</h3>

This codec encodes the bitmap subtitle format that is used in DVB broadcasts and recordings. The bitmaps are typically embedded in a container such as MPEG-TS as a separate stream. <h4 class="subsection">17.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">min_bpp <var class="var">integer (2, 4, or 8)</var></samp></dt> <dd> Set a minimum bits-per-pixel value for the subtitle color lookup tables. DVB supports 2, 4, and 8 bits-per-pixel color lookup tables. This option enables forcing a particular bits-per-pixel value regardless of the number of colors. Since not all players support or properly support 2 bits-per-pixel, this value defaults to 4. </dd> </dl>

 <h3 class="section">17.2 dvdsub# TOC</h3>

This codec encodes the bitmap subtitle format that is used in DVDs. Typically they are stored in VOBSUB file pairs (*.idx + *.sub), and they can also be used in Matroska files. <h4 class="subsection">17.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">even_rows_fix</samp></dt> <dd> When set to 1, enable a work-around that makes the number of pixel rows even in all subtitles. This fixes a problem with some players that cut off the bottom row if the number is odd. The work-around just adds a fully transparent row if needed. The overhead is low, typically one byte per subtitle on average. By default, this work-around is disabled. </dd> </dl>

 <h3 class="section">17.3 lrc# TOC</h3>

This codec encodes the LRC lyrics format. <h4 class="subsection">17.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">precision</samp></dt> <dd> Specify the precision of the fractional part of the timestamp. Time base is determined based on this value. Defaults to 2 for centiseconds. </dd> </dl>

 <h2 class="chapter">18 Bitstream Filters# TOC</h2>

When you configure your FFmpeg build, all the supported bitstream filters are enabled by default. You can list all available ones using the configure option <code class="code">--list-bsfs</code>. You can disable all the bitstream filters using the configure option <code class="code">--disable-bsfs</code>, and selectively enable any bitstream filter using the

option <code class=”code”>--enable-bsf=BSF</code>, or you can disable a particular bitstream filter using the option <code class=”code”>--disable-bsf=BSF</code>. The option <code class=”code”>-bsfs</code> of the ff* tools will display the list of all the supported bitstream filters included in your build. The ff* tools have a -bsf option applied per stream, taking a comma-separated list of filters, whose parameters follow the filter name after a ’=’.   <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v filter1[=opt1=str1:opt2=str2][,filter2] OUTPUT </pre>

Below is a description of the currently available bitstream filters, with their parameters, if any. <h3 class=”section”>18.1 aac_adtstoasc# TOC</h3>

Convert MPEG-2/4 AAC ADTS to an MPEG-4 Audio Specific Configuration bitstream. This filter creates an MPEG-4 AudioSpecificConfig from an MPEG-2/4 ADTS header and removes the ADTS header. This filter is required for example when copying an AAC stream from a raw ADTS AAC or an MPEG-TS container to MP4A-LATM, to an FLV file, or to MOV/MP4 files and related formats such as 3GP or M4A. Please note that it is auto-inserted for MP4A-LATM and MOV/MP4 and related formats. <h3 class=”section”>18.2 av1_metadata# TOC</h3>

Modify metadata embedded in an AV1 stream. <dl class=”table”> <dt><samp class=”option”>td</samp></dt> <dd> Insert or remove temporal delimiter OBUs in all temporal units of the stream. <dl class=”table”> <dt>‘<samp class=”samp”>insert</samp>’</dt> <dd> Insert a TD at the beginning of every TU which does not already have one. </dd> <dt>‘<samp class=”samp”>remove</samp>’</dt> <dd> Remove the TD from the beginning of every TU which has one. </dd> </dl>

</dd> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the color description fields in the stream (see AV1 section 6.4.2). </dd> <dt><samp class=”option”>color_range</samp></dt> <dd> Set the color range in the stream (see AV1 section 6.4.2; note that this cannot be set for streams using BT.709 primaries, sRGB transfer characteristic and identity (RGB) matrix coefficients). <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dd> Limited range. </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> Full range. </dd> </dl>

</dd> <dt><samp class="option">chroma_sample_position</samp></dt>
<dd> Set the chroma sample location in the stream (see AV1 section 6.4.2). This can only be set for 4:2:0 streams. <dl class="table">
<dt>‘<samp class="samp">vertical</samp>’</dt> <dd> Left position (matching the default in MPEG-2 and H.264). </dd>
<dt>‘<samp class="samp">colocated</samp>’</dt>
<dd> Top-left position. </dd> </dl>

</dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_display_tick) in the timing info in the sequence header. </dd> <dt><samp class="option">num_ticks_per_picture</samp></dt> <dd> Set the number of ticks in each picture, to indicate that the stream has a fixed framerate. Ignored if <samp class="option">tick_rate</samp> is not also set. </dd> <dt><samp class="option">delete_padding</samp></dt>
<dd> Deletes Padding OBUs. </dd> </dl>

 <h3 class="section">18.3 chomp#
TOC</h3>

Remove zero padding at the end of a packet.
<h3 class="section">18.4 dca_core# TOC</h3>

Extract the core from a DCA/DTS stream, dropping extensions such as DTS-HD. <h3 class="section">18.5 dovi_rpu# TOC</h3>

Manipulate Dolby Vision metadata in a HEVC/AV1 bitstream, optionally enabling metadata compression. <dl class="table"> <dt><samp class="option">strip</samp></dt> <dd> If enabled, strip all Dolby Vision metadata (configuration record + RPU data blocks) from the stream. </dd> <dt><samp class="option">compression</samp></dt>
<dd> Which compression level to enable. <dl class="table">
<dt>‘<samp class="samp">none</samp>’</dt> <dd> No metadata compression. </dd> <dt>‘<samp class="samp">limited</samp>’</dt>
<dd> Limited metadata compression scheme. Should be compatible with most devices. This is the default. </dd> <dt>‘<samp class="samp">extended</samp>’</dt> <dd> Extended metadata compression. Devices are not required to support this. Note that this level currently behaves the same as ‘<samp class="samp">limited</samp>’ in libavcodec. </dd> </dl> </dd> </dl>

 <h3 class="section">18.6 dovi_split<a class="anchor hidden-xs" href="#dovi_005fsplit"

aria-hidden=”true”># TOC</h3>

Split a Dolby Vision Profile 7 multi-layer HEVC bitstream. Profile 7 carries the enhancement-layer HEVC bitstream interleaved inside the base-layer access units, wrapped in user-unspecified NAL units of type 63 (UNSPEC63), and the RPU metadata as a sibling user-unspecified NAL of type 62 (UNSPEC62). <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Which Dolby Vision components to keep in the output bitstream. <dl class=”table”> <dt>‘<samp class=”samp”>bl</samp>’</dt> <dd> Base layer only: drop every UNSPEC63 (EL) and every UNSPEC62 (RPU). The output is a plain HEVC stream with no Dolby Vision markers. This is the default. </dd> <dt>‘<samp class=”samp”>bl_rpu</samp>’</dt> <dd> Base layer with the RPU NAL kept. </dd> <dt>‘<samp class=”samp”>el</samp>’</dt> <dd> Enhancement layer only: for every UNSPEC63 NAL, strip the two-byte outer NAL header and emit the inner payload. The result is a standalone HEVC bitstream. UNSPEC62 (RPU) is dropped. </dd> <dt>‘<samp class=”samp”>el_rpu</samp>’</dt> <dd> Enhancement layer with the RPU NAL kept verbatim. Same as ‘<samp class=”samp”>el</samp>’, but the UNSPEC62 RPU NALs are also emitted alongside the unwrapped EL NALs. </dd> </dl> </dd> </dl>

 <h3 class=”section”>18.7 dump_extra# TOC</h3>

Add extradata to the beginning of the filtered packets except when said packets already exactly begin with the extradata that is intended to be added. <dl class=”table”> <dt><samp class=”option”>freq</samp></dt> <dd> The additional argument specifies which packets should be filtered. It accepts the values: <dl class=”table”> <dt>‘<samp class=”samp”>k</samp>’</dt> <dt>‘<samp class=”samp”>keyframe</samp>’</dt> <dd> add extradata to all key packets </dd> <dt>‘<samp class=”samp”>e</samp>’</dt> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> add extradata to all packets </dd> </dl> </dd> </dl>

If not specified it is assumed ‘<samp class=”samp”>k</samp>’. For example the following <code class=”command”>ffmpeg</code> command forces a global header (thus disabling individual packet headers) in the H.264 packets generated by the <code class=”code”>libx264</code> encoder, but corrects them by adding the header stored in extradata to the key packets: <pre class=”example-preformatted”>ffmpeg -i INPUT -map 0 -flags:v +global_header -c:v libx264 -bsf:v dump_extra out.ts </pre>

 <h3 class="section">18.8 dv_error_marker# TOC</h3>

Blocks in DV which are marked as damaged are replaced by blocks of the specified color. <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color to replace damaged blocks by </dd> <dt><samp class="option">sta</samp></dt> <dd> A 16 bit mask which specifies which of the 16 possible error status values are to be replaced by colored blocks. 0xFFFE is the default which replaces all non 0 error status values. <dl class="table"> <dt>‘<samp class="samp">ok</samp>’</dt> <dd> No error, no concealment </dd> <dt>‘<samp class="samp">err</samp>’</dt> <dd> Error, No concealment </dd> <dt>‘<samp class="samp">res</samp>’</dt> <dd> Reserved </dd> <dt>‘<samp class="samp">notok</samp>’</dt> <dd> Error or concealment </dd> <dt>‘<samp class="samp">notres</samp>’</dt> <dd> Not reserved </dd> <dt>‘<samp class="samp">Aa, Ba, Ca, Ab, Bb, Cb, A, B, C, a, b, erri, erru</samp>’</dt> <dd> The specific error status code </dd> </dl> see page 44-46 or section 5.5 of <a class="url" href="http://web.archive.org/web/20060927044735/http://www.smpte.org/ </dd> </dl>

 <h3 class="section">18.9 eac3_core# TOC</h3>

Extract the core from a E-AC-3 stream, dropping extra channels. <h3 class="section">18.10 eia608_to_smpte436m# TOC</h3>

Convert from a <code class="code">EIA_608</code> stream to a <code class="code">SMPTE_436M_ANC</code> data stream, wrapping the closed captions in CTA-708 CDP VANC packets. <dl class="table"> <dt><samp class="option">line_number</samp></dt> <dd> Choose which line number the generated VANC packets should go on. You generally want either line 9 (the default) or 11. </dd> <dt><samp class="option">wrapping_type</samp></dt> <dd> Choose the SMPTE 436M wrapping type, defaults to ‘<samp class="samp">vanc_frame</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">vanc_frame</samp>’</dt> <dd> VANC frame (interlaced or segmented progressive frame) </dd> <dt>‘<samp

class=”samp”>vanc_field_1</samp>’</dt> <dt>‘<samp class=”samp”>vanc_field_2</samp>’</dt> <dt>‘<samp class=”samp”>vanc_progressive_frame</samp>’</dt> </dl> </dd> <dt><samp class=”option”>sample_coding</samp></dt> <dd> Choose the SMPTE 436M sample coding, defaults to ‘<samp class=”samp”>8bit_luma</samp>’. It accepts the values: <dl class=”table”> <dt>‘<samp class=”samp”>8bit_luma</samp>’</dt> <dd> 8-bit component luma samples </dd> <dt>‘<samp class=”samp”>8bit_color_diff</samp>’</dt> <dd> 8-bit component color difference samples </dd> <dt>‘<samp class=”samp”>8bit_luma_and_color_diff</samp>’</dt> <dd> 8-bit component luma and color difference samples </dd> <dt>‘<samp class=”samp”>10bit_luma</samp>’</dt> <dd> 10-bit component luma samples </dd> <dt>‘<samp class=”samp”>10bit_color_diff</samp>’</dt> <dd> 10-bit component color difference samples </dd> <dt>‘<samp class=”samp”>10bit_luma_and_color_diff</samp>’</dt> <dd> 10-bit component luma and color difference samples </dd> <dt>‘<samp class=”samp”>8bit_luma_parity_error</samp>’</dt> <dd> 8-bit component luma samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_color_diff_parity_error</samp>’</dt> <dd> 8-bit component color difference samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_luma_and_color_diff_parity_error</samp>’</dt> <dd> 8-bit component luma and color difference samples with parity error </dd> </dl> </dd> <dt><samp class=”option”>initial_cdp_sequence_cntr</samp></dt> <dd> The initial value of the CDP’s 16-bit unsigned integer <code class=”code”>cdp_hdr_sequence_cntr</code> and <code class=”code”>cdp_ftr_sequence_cntr</code> fields. Defaults to 0. </dd> <dt><samp class=”option”>cdp_frame_rate</samp></dt> <dd> Set the CDP’s <code class=”code”>cdp_frame_rate</code> field. This doesn’t actually change the timing of the data stream, it just changes the values inserted in that field in the generated CDP packets. Defaults to ‘<samp class=”samp”>30000/1001</samp>’. </dd> </dl>

 <h3 class=”section”>18.11 extract_extradata# TOC</h3>

Extract the in-band extradata. Certain codecs allow the long-term headers (e.g. MPEG-2 sequence headers, or H.264/HEVC (VPS/)SPS/PPS) to be transmitted either "in-band" (i.e. as a part of the bitstream containing the coded frames) or "out of band" (e.g. on the container level). This latter form is called "extradata" in FFmpeg terminology. This bitstream filter detects the in-band headers and makes them available as extradata. <dl class=”table”> <dt><samp class=”option”>remove</samp></dt> <dd> When this

option is enabled, the long-term headers are removed from the bitstream after extraction. </dd> </dl>

 <h3 class="section">18.12 filter_units# TOC</h3>

Remove units with types in or not in a given set from the stream. <dl class="table"> <dt><samp class="option">pass_types</samp></dt> <dd> List of unit types or ranges of unit types to pass through while removing all others. This is specified as a ’|’-separated list of unit type values or ranges of values with ’-’. </dd> <dt><samp class="option">remove_types</samp></dt> <dd> Identical to <samp class="option">pass_types</samp>, except the units in the given set removed and all others passed through. </dd> </dl>

The types used by pass_types and remove_types correspond to NAL unit types (nal_unit_type) in H.264, HEVC and H.266 (see Table 7-1 in the H.264 and HEVC specifications or Table 5 in the H.266 specification), to marker values for JPEG (without 0xFF prefix) and to start codes without start code prefix (i.e. the byte following the 0x000001) for MPEG-2. For VP8 and VP9, every unit has type zero. Extradata is unchanged by this transformation, but note that if the stream contains inline parameter sets then the output may be unusable if they are removed. For example, to remove all non-VCL NAL units from an H.264 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=pass_types=1-5' OUTPUT </pre>

To remove all AUDs, SEI and filler from an H.265 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=35|38-40' OUTPUT </pre>

To remove all user data from a MPEG-2 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=178' OUTPUT </pre>

To remove all SEI from a H264 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=6' OUTPUT </pre>

To remove all prefix and suffix SEI from a HEVC stream, including Closed Captions and dynamic HDR: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=39|40' OUTPUT </pre>

 <h3 class="section">18.13 hapqa_extract<a class="anchor hidden-xs" href="#hapqa_005fextract"

aria-hidden=”true”># TOC</h3>

Extract Rgb or Alpha part of an HAPQA file, without recompression, in order to create an HAPQ or an HAPAlphaOnly file. <dl class=”table”> <dt><samp class=”option”>texture</samp></dt> <dd> Specifies the texture to keep. <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dt><samp class=”option”>alpha</samp></dt> </dl>

</dd> </dl>

Convert HAPQA to HAPQ <pre class=”example-preformatted”>ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=color -tag:v HapY -metadata:s:v:0 encoder="HAPQ" hapq_file.mov </pre>

Convert HAPQA to HAPAlphaOnly <pre class=”example-preformatted”>ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=alpha -tag:v HapA -metadata:s:v:0 encoder="HAPAlpha Only" hapalphaonly_file.mov </pre>

 <h3 class=”section”>18.14 h264_metadata# TOC</h3>

Modify metadata embedded in an H.264 stream. <dl class=”table”> <dt><samp class=”option”>aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class=”table”> <dt>‘<samp class=”samp”>pass</samp>’</dt> <dt>‘<samp class=”samp”>insert</samp>’</dt> <dt>‘<samp class=”samp”>remove</samp>’</
</dl>

Default is pass. </dd> <dt><samp class=”option”>sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio of the stream in the VUI parameters. See H.264 table E-1. </dd> <dt><samp class=”option”>overscan_appropriate_flag</samp></dt> <dd> Set whether the stream is suitable for display using overscan or not (see H.264 section E.2.1). </dd> <dt><samp class=”option”>video_format</samp></dt> <dt><samp class=”option”>video_full_range_flag</samp> <dd> Set the video format in the stream (see H.264 section E.2.1 and table E-2). </dd> <dt><samp class=”option”>colour_primaries</samp></dt> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.264 section E.2.1 and tables E-3, E-4 and E-5). </dd> <dt><samp class=”option”>chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.264 section E.2.1 and figure E-1). </dd> <dt><samp class=”option”>tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_tick) in the VUI

parameters. This is the smallest time unit representable in the stream, and in many cases represents the field rate of the stream (double the frame rate). </dd> <dt><samp class=”option”>fixed_frame_rate_flag</samp></dt> <dd> Set whether the stream has fixed framerate - typically this indicates that the framerate is exactly half the tick rate, but the exact meaning is dependent on interlacing and the picture structure (see H.264 section E.2.1 and table E-6). </dd> <dt><samp class=”option”>zero_new_constraint_set_flags</samp></dt> <dd> Zero constraint_set4_flag and constraint_set5_flag in the SPS. These bits were reserved in a previous version of the H.264 spec, and thus some hardware decoders require these to be zero. The result of zeroing this is still a valid bitstream. </dd> <dt><samp class=”option”>crop_left</samp></dt> <dt><samp class=”option”>crop_right</samp></dt> <dt><samp class=”option”>crop_top</samp></dt> <dt><samp class=”option”>crop_bottom</samp></dt> <dd> Set the frame cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled or the stream is interlaced (see H.264 section 7.4.2.1.1). </dd> <dt><samp class=”option”>sei_user_data</samp></dt> <dd> Insert a string as SEI unregistered user data. The argument must be of the form UUID+string, where the UUID is as hex digits possibly separated by hyphens, and the string can be anything. For example, ‘<samp class=”samp”>086f3693-b7b3-4f2c-9653-21492feee5b8+hello</samp>’ will insert the string “hello” associated with the given UUID. </dd> <dt><samp class=”option”>delete_filler</samp></dt> <dd> Deletes both filler NAL units and filler SEI messages. </dd> <dt><samp class=”option”>display_orientation</samp></dt> <dd> Insert, extract or remove Display orientation SEI messages. See H.264 section D.1.27 and D.2.27 for syntax and semantics. <dl class=”table”> <dt>‘<samp class=”samp”>pass</samp>’</dt> <dt>‘<samp class=”samp”>insert</samp>’</dt> <dt>‘<samp class=”samp”>remove</samp>’</
<dt>‘<samp class=”samp”>extract</samp>’</dt> </dl>

Default is pass. Insert mode works in conjunction with <code class=”code”>rotate</code> and <code class=”code”>flip</code> options. Any pre-existing Display orientation messages will be removed in insert or remove mode. Extract mode attaches the display matrix to the packet as side data. </dd> <dt><samp class=”option”>rotate</samp></dt> <dd> Set rotation in display orientation SEI (anticlockwise angle in degrees). Range is -360 to +360. Default is NaN. </dd> <dt><samp class=”option”>flip</samp></dt> <dd> Set flip in display orientation SEI. <dl class=”table”> <dt>‘<samp class=”samp”>horizontal</samp>’</dt> <dt>‘<samp class=”samp”>vertical</samp>’</dt> </dl>

Default is unset. </dd> <dt><samp class=”option”>level</samp></dt>

<dd> Set the level in the SPS. Refer to H.264 section A.3 and tables A-1 to A-5. The argument must be the name of a level (for example, ‘<samp class="samp">4.2</samp>’), a level_idc value (for example, ‘<samp class="samp">42</samp>’), or the special name ‘<samp class="samp">auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class="section">18.15 h264_mp4toannexb# TOC</h3

Convert an H.264 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.264 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an H.264 stream to mpegts format with <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.mp4 -codec copy -bsf:v h264_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class="code">mpegts</code>) and raw H.264 (muxer <code class="code">h264</code>) output formats. <h3 class="section">18.16 h264_redundant_pps# TOC</h3>

This applies a specific fixup to some Blu-ray BDMV H264 streams which contain redundant PPSs. The PPSs modify irrelevant parameters of the stream, confusing other transformations which require the correct extradata. The encoder used on these impacted streams adds extra PPSs throughout the stream, varying the initial QP and whether weighted prediction was enabled. This causes issues after copying the stream into a global header container, as the starting PPS is not suitable for the rest of the stream. One side effect, for example, is seeking will return garbled output until a new PPS appears. This BSF removes the extra PPSs and rewrites the slice headers such that the stream uses a single leading PPS in the global header, which resolves the issue. <h3 class="section">18.17 hevc_metadata# TOC</h3>

Modify metadata embedded in an HEVC stream. <dl class="table">

<dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</dt> </dl>

</dd> <dt><samp class="option">sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio in the stream in the VUI parameters. </dd> <dt><samp class="option">video_format</samp></dt> <dt><samp class="option">video_full_range_flag</samp></dt> <dd> Set the video format in the stream (see H.265 section E.3.1 and table E.2). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.265 section E.3.1 and tables E.3, E.4 and E.5). </dd> <dt><samp class="option">chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.265 section E.3.1 and figure E.1). </dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate in the VPS and VUI parameters (time_scale / num_units_in_tick). Combined with <samp class="option">num_ticks_poc_diff_one</samp>, this can set a constant framerate in the stream. Note that it is likely to be overridden by container parameters when the stream is in a container. </dd> <dt><samp class="option">num_ticks_poc_diff_one</samp></dt> <dd> Set poc_proportional_to_timing_flag in VPS and VUI and use this value to set num_ticks_poc_diff_one_minus1 (see H.265 sections 7.4.3.1 and E.3.1). Ignored if <samp class="option">tick_rate</samp> is not also set. </dd> <dt><samp class="option">crop_left</samp></dt> <dt><samp class="option">crop_right</samp></dt> <dt><samp class="option">crop_top</samp></dt> <dt><samp class="option">crop_bottom</samp></dt> <dd> Set the conformance window cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled (H.265 section 7.4.3.2.1). </dd> <dt><samp class="option">width</samp></dt> <dt><samp class="option">height</samp></dt> <dd> Set width and height after crop. </dd> <dt><samp class="option">level</samp></dt> <dd> Set the level in the VPS and SPS. See H.265 section A.4 and tables A.6 and A.7. The argument must be the name of a level (for example, ‘<samp class="samp">5.1</samp>’), a general_level_idc value (for example, ‘<samp class="samp">153</samp>’ for level 5.1), or the special name ‘<samp class="samp">auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class="section">18.18 hevc_mp4toannexb# <a

class=”anchor hidden-xs”href=”#toc-hevc_005fmp4toannexb” aria-hidden=”true”>TOC</h3

Convert an HEVC/H.265 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.265 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class=”code”>mpegts</code>). For example to remux an MP4 file containing an HEVC stream to mpegts format with <code class=”command”>ffmpeg</code>, you can use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT.mp4 -codec copy -bsf:v hevc_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class=”code”>mpegts</code>) and raw HEVC/H.265 (muxer <code class=”code”>h265</code> or <code class=”code”>hevc</code>) output formats. <h3 class=”section”>18.19 imxdump# TOC</h3>

Modifies the bitstream to fit in MOV and to be usable by the Final Cut Pro decoder. This filter only applies to the mpeg2video codec, and is likely not needed for Final Cut Pro 7 and newer with the appropriate <samp class=”option”>-tag:v</samp>. For example, to remux 30 MB/sec NTSC IMX to MOV: <pre class=”example-preformatted”>ffmpeg -i input.mxf -c copy -bsf:v imxdump -tag:v mx3n output.mov </pre>

 <h3 class=”section”>18.20 jpegapp# TOC</h3>

Inject JPEG APP0..APP15 segments from a length-prefixed sidecar into each MJPEG packet, after SOI. The sidecar is a sequence of records, one per encoded JPEG, in encode order: <pre class=”example-preformatted”>u32be length u8 payload[length] </pre>

<var class=”var”>length</var> does not include the 4-byte prefix and must not exceed 16 MiB. A zero <var class=”var”>length</var> leaves that packet unchanged. A non-empty payload must be a concatenation of complete APP segments (<code class=”code”>FF En</code> plus the 16-bit length that includes those two length bytes). If the payload starts with APP0 and the packet has an APP0 marker in the two bytes immediately following SOI, that APP0 segment is dropped. APP0 segments preceded by fill bytes or another segment are retained. An APP1-only payload keeps the encoder APP0: SOI, APP1, APP0, DQT, ... The filter requires <samp class=”option”>sidecar</samp>. Extra unused sidecar bytes are ig-

nored; a packet without a matching record is an error. Records follow packets in arrival order. Flushing the filter does not rewind the sidecar, so a reused context needs a sidecar positioned for the packets it will see next. A sidecar read or validation error prevents further record consumption by that filter context. To resume, create a new context with a sidecar matching the remaining packets. Flushing does not clear this error.   <pre class=”example-preformatted”>ffmpeg -i in.mp4 -c:v mjpeg -bsf:v jpegapp=sidecar=00000004DEADBEEF frame_%d.jpg </pre>

 <h3 class=”section”>18.21 mjpeg2jpeg# TOC</h3>

Convert MJPEG/AVI1 packets to full JPEG/JFIF packets. MJPEG is a video codec wherein each video frame is essentially a JPEG image. The individual frames can be extracted without loss, e.g. by   <pre class=”example-preformatted”>ffmpeg -i ../some_mjpeg.avi -c:v copy frames_%d.jpg </pre>

Unfortunately, these chunks are incomplete JPEG images, because they lack the DHT segment required for decoding. Quoting from http://www.digitalpreservation.go
 Avery Lee, writing in the rec.video.desktop newsgroup in 2001, commented that "MJPEG, or at least the MJPEG in AVIs having the MJPG fourcc, is restricted JPEG with a fixed – and *omitted* – Huffman table. The JPEG must be YCbCr colorspace, it must be 4:2:2, and it must use basic Huffman encoding, not arithmetic or progressive. . . . You can indeed extract the MJPEG frames and decode them with a regular JPEG decoder, but you have to prepend the DHT segment to them, or else the decoder won’t have any idea how to decompress the data. The exact table necessary is given in the OpenDML spec." This bitstream filter patches the header of frames extracted from an MJPEG stream (carrying the AVI1 header ID and lacking a DHT segment) to produce fully qualified JPEG images.   <pre class=”example-preformatted”>ffmpeg -i mjpeg-movie.avi -c:v copy -bsf:v mjpeg2jpeg frame_%d.jpg exiftran -i -9 frame*.jpg ffmpeg -i frame_%d.jpg -c:v copy rotated.avi </pre>

 <h3 class=”section”>18.22 mjpegadump# TOC</h3>

Add an MJPEG A header to the bitstream, to enable decoding by Quicktime. <h3 class=”section”>18.23 mov2textsub<a class=”anchor hidden-xs” href=”#mov2textsub”

aria-hidden="true"># TOC</h3>

Extract a representable text file from MOV subtitles, stripping the metadata header from each subtitle packet. See also the text2movsub filter. <h3 class="section">18.24 mpeg2_metadata# TOC</h3>

Modify metadata embedded in an MPEG-2 stream. <dl class="table"> <dt><samp class="option">display_aspect_ratio</samp></dt> <dd> Set the display aspect ratio in the stream. The following fixed values are supported: <dl class="table"> <dt><samp class="option">4/3</samp></dt> <dt><samp class="option">16/9</samp></dt> <dt><samp class="option">221/100</samp></dt> </dl> Any other value will result in square pixels being signalled instead (see H.262 section 6.3.3 and table 6-3). </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Set the frame rate in the stream. This is constructed from a table of known values combined with a small multiplier and divisor - if the supplied value is not exactly representable, the nearest representable value will be used instead (see H.262 section 6.3.3 and table 6-4). </dd> <dt><samp class="option">video_format</samp></dt> <dd> Set the video format in the stream (see H.262 section 6.3.6 and table 6-6). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.262 section 6.3.6 and tables 6-7, 6-8 and 6-9). </dd> </dl>

 <h3 class="section">18.25 mpeg4_unpack_bframes# TOC</h3>

Unpack DivX-style packed B-frames. DivX-style packed B-frames are not valid MPEG-4 and were only a workaround for the broken Video for Windows subsystem. They use more space, can cause minor AV sync issues, require more CPU power to decode (unless the player has some decoded picture queue to compensate the 2,0,2,0 frame per packet style) and cause trouble if copied into a standard container like mp4 or mpeg-ps/ts, because MPEG-4 decoders may not be able to decode them, since they are not valid MPEG-4. For example to fix an AVI file containing an MPEG-4 stream with DivX-style packed B-frames using <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.avi

-codec copy -bsf:v mpeg4_unpack_bframes OUTPUT.avi </pre>

 <h3 class="section">18.26 noise# TOC</h3>

Damages the contents of packets or simply drops them without damaging the container. Can be used for fuzzing or testing error resilience/concealment. Parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt> <dd> Accepts an expression whose evaluation per-packet determines how often bytes in that packet will be modified. A value below 0 will result in a variable frequency. Default is 0 which results in no modification. However, if neither amount nor drop is specified, amount will be set to <var class="var">-1</var>. See below for accepted variables. </dd> <dt><samp class="option">drop</samp></dt> <dd> Accepts an expression evaluated per-packet whose value determines whether that packet is dropped. Evaluation to a positive value results in the packet being dropped. Evaluation to a negative value results in a variable chance of it being dropped, roughly inverse in proportion to the magnitude of the value. Default is 0 which results in no drops. See below for accepted variables. </dd> <dt><samp class="option">dropamount</samp></dt> <dd> Accepts a non-negative integer, which assigns a variable chance of it being dropped, roughly inverse in proportion to the value. Default is 0 which results in no drops. This option is kept for backwards compatibility and is equivalent to setting drop to a negative value with the same magnitude i.e. <code class="code">dropamount=4</code> is the same as <code class="code">drop=-4</code>. Ignored if drop is also specified. </dd> </dl>

Both <code class="code">amount</code> and <code class="code">drop</code> accept expressions containing the following variables: <dl class="table"> <dt>‘<samp class="samp">n</samp>’</dt> <dd> The index of the packet, starting from zero. </dd> <dt>‘<samp class="samp">tb</samp>’</dt> <dd> The timebase for packet timestamps. </dd> <dt>‘<samp class="samp">pts</samp>’</dt> <dd> Packet presentation timestamp. </dd> <dt>‘<samp class="samp">dts</samp>’</dt> <dd> Packet decoding timestamp. </dd> <dt>‘<samp class="samp">nopts</samp>’</dt> <dd> Constant representing AV_NOPTS_VALUE. </dd> <dt>‘<samp class="samp">startpts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE PTS seen in the stream. </dd> <dt>‘<samp class="samp">startdts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE DTS seen in the stream. </dd> <dt>‘<samp class="samp">duration</samp>’</dt> <dt>‘<samp class="samp">d</samp>’</dt <dd> Packet duration, in timebase units. </dd> <dt>‘<samp class="samp">pos</samp>’</dt> <dd> Packet position in in-

put; may be -1 when unknown or not set. </dd> <dt>‘<samp class="samp">size</samp>’</dt> <dd> Packet size, in bytes. </dd> <dt>‘<samp class="samp">key</samp>’</dt> <dd> Whether packet is marked as a keyframe. </dd> <dt>‘<samp class="samp">state</samp>’</dt> <dd> A pseudo random integer, primarily derived from the content of packet payload. </dd> </dl>

 <h4 class="subsection">18.26.1 Examples# TOC</h4> Apply modification to every byte but don’t drop any packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf noise=1 output.mkv </pre>

Drop every video packet not marked as a keyframe after timestamp 30s but do not modify any of the remaining packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v noise=drop='gt(pts*tb\,30)*not(key)' output.mkv </pre>

Drop one second of audio every 10 seconds and add some random noise to the rest. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:a noise=amount=-1:drop='between(mod(pts*tb\,10)\,9\,10)' output.mkv </pre>

 <h3 class="section">18.27 null# TOC</h3> This bitstream filter passes the packets through unchanged. <h3 class="section">18.28 pcm_rechunk# TOC</h3>

Repacketize PCM audio to a fixed number of samples per packet or a fixed packet rate per second. This is similar to the <a data-manual="ffmpeg-filters" href="./ffmpeg-filters.html#asetnsamples">asetnsamples audio filter but works on audio packets instead of audio frames. <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of samples per each output audio packet. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio packet with silence, so that it will contain the same number of samples (or roughly the same number of samples, see <samp class="option">frame_rate</samp>) as the previous ones. Default value is 1. </dd> <dt><samp class="option">frame_rate, r</samp></dt> <dd> This option makes

the filter output a fixed number of packets per second instead of a fixed number of samples per packet. If the audio sample rate is not divisible by the frame rate then the number of samples will not be constant but will vary slightly so that each packet will start as close to the frame boundary as possible. Using this option has precedence over <samp class="option">nb_out_samples</samp>. </dd> </dl>

You can generate the well known 1602-1601-1602-1601-1602 pattern of 48kHz audio for NTSC frame rate using the <samp class="option">frame_rate</samp> option. <pre class="example-preformatted">ffmpeg -f lavfi -i sine=r=48000:d=1 -c pcm_s16le -bsf pcm_rechunk=r=30000/1001 -f framecrc - </pre>

 <h3 class="section">18.29 pgs_frame_merge# TOC</h3>

Merge a sequence of PGS Subtitle segments ending with an "end of display set" segment into a single packet. This is required by some containers that support PGS subtitles (muxer <code class="code">matroska</code>). <h3 class="section">18.30 prores_metadata# TOC</h3>

Modify color property metadata embedded in prores stream. <dl class="table"> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT601 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> BT601 525 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dd> DCI P3 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dd> P3 D65 </dd> </dl>

</dd> <dt><samp class="option">transfer_characteristics</samp></dt> <dd> Set the color transfer. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same transfer characteristics property (default). </dd> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT 601, BT 709, BT

2020 </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt>
<dd> SMPTE ST 2084 </dd> <dt>‘<samp class="samp">arib-
std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>
</dl>

</dd> <dt><samp class="option">matrix_coefficients</samp></dt>
<dd> Set the matrix coefficient. Available values are: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Keep the same colorspace property (default). </dd>
<dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>&rsqu
<dd> BT 601 </dd> <dt>‘<samp class="samp">bt2020nc</samp>’</dt>
</dl> </dd> </dl>

Set Rec709 colorspace for each frame of the file
<pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v
prores_metadata=color_primaries=bt709:color_trc=bt709:colorspace=bt709
output.mov </pre>

Set Hybrid Log-Gamma parameters for each frame of the file <div
class="example"> <pre class="example-preformatted">ffmpeg -i INPUT -c
copy -bsf:v prores_metadata=color_primaries=bt2020:color_trc=arib-std-
b67:colorspace=bt2020nc output.mov </pre>

 <h3 class="section">18.31 re-
move_extra<a class="anchor hidden-xs" href="#remove_005fextra"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
remove_005fextra" aria-hidden="true">TOC</h3>

Remove extradata from packets. It accepts the following param-
eter: <dl class="table"> <dt><samp class="option">freq</samp></dt>
<dd> Set which frame types to remove extradata from. <dl
class="table"> <dt>‘<samp class="samp">k</samp>’</dt>
<dd> Remove extradata from non-keyframes only. </dd>
<dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> Remove
extradata from keyframes only. </dd> <dt>‘<samp
class="samp">e, all</samp>’</dt> <dd> Remove extradata
from all frames. </dd> </dl> </dd> </dl>

 <h3 class="section">18.32 setts<span class="pull-
right">#
TOC</h3>
Set PTS and DTS in packets. It accepts the following parame-
ters: <dl class="table"> <dt><samp class="option">ts</samp></dt>
<dt><samp class="option">pts</samp></dt> <dt><samp class="option">dts</samp></dt>
<dd> Set expressions for PTS, DTS or both. </dd> <dt><samp
class="option">duration</samp></dt> <dd> Set expression for dura-
tion. </dd> <dt><samp class="option">time_base</samp></dt>
<dd> Set output time base. </dd> <dt><samp class="option">prescale</samp></dt>

<dd> Set whether to convert time fields to user-set output time base before evaluation of expressions. Defaults to 0. </dd> </dl>

The expressions are evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The count of the input packet. Starting from 0. </dd> <dt><samp class="option">TS</samp></dt> <dd> The demux timestamp in input in case of <code class="code">ts</code> or <code class="code">dts</code> option or presentation timestamp in case of <code class="code">pts</code> option. </dd> <dt><samp class="option">POS</samp></dt> <dd> The original position in the file of the packet, or undefined if undefined for the current packet </dd> <dt><samp class="option">DTS</samp></dt> <dd> The demux timestamp in input. </dd> <dt><samp class="option">PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class="option">DURATION</samp></dt> <dd> The duration in input. </dd> <dt><samp class="option">STARTDTS</samp></dt> <dd> The DTS of the first packet. </dd> <dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first packet. </dd> <dt><samp class="option">PREV_INDTS</samp></dt> <dd> The previous input DTS. </dd> <dt><samp class="option">PREV_INPTS</samp></dt <dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INDURATION</sam <dd> The previous input duration. </dd> <dt><samp class="option">PREV_OUTDTS</samp></dt> <dd> The previous output DTS. </dd> <dt><samp class="option">PREV_OUTPTS</samp></dt> <dd> The previous output PTS. </dd> <dt><samp class="option">PREV_OUTDURATION</ <dd> The previous output duration. </dd> <dt><samp class="option">NEXT_DTS</samp></dt> <dd> The next input DTS. </dd> <dt><samp class="option">NEXT_PTS</samp></dt> <dd> The next input PTS. </dd> <dt><samp class="option">NEXT_DURATION</samp></ <dd> The next input duration. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase in which time fields are denominated. The user-set output timebase if prescale is enabled, else the input timebase. </dd> <dt><samp class="option">TB_OUT</samp></dt> <dd> The output timebase. </dd> <dt><samp class="option">SR</samp></dt> <dd> The sample rate of stream packet belongs. </dd> <dt><samp class="option">NOPTS</samp></dt> <dd> The AV_NOPTS_VALUE constant. </dd> </dl>

For example, to set PTS equal to DTS (not recommended if B-frames are involved): <pre class="example-preformatted">ffmpeg -i INPUT -c:a copy -bsf:a setts=pts=DTS out.mkv </pre>

 <h3 class="section">18.33 showinfo# <a class="anchor hidden-xs"href="#toc-showinfo"

aria-hidden=”true”>TOC</h3> Log basic packet information. Mainly useful for testing, debugging, and development. <h3 class=”section”>18.34 side_data# TOC</h3> Remove side data from packets and from the stream, or select packets by their side data. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>remove</samp></dt> <dd> Set the side data types to remove, joined with ‘<samp class=”samp”>+</samp>’, from the packets and from the side data of the stream. Each type is given by its handle; <code class=”command”>ffmpeg -h bsf=side_data</code> lists them. </dd> <dt><samp class=”option”>select</samp></dt> <dd> Set the side data types of the packets to keep, joined with ‘<samp class=”samp”>+</samp>’. A packet is kept if it carries any of them, and dropped otherwise. The selection looks at the side data before <samp class=”option”>remove</samp> applies. Dropping a packet breaks the decoding of the packets that depend on it, so this is meant for analysis or for streams where each packet decodes on its own. </dd> </dl>

For example, to drop the display matrix and the replay gain of a stream: <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf:v side_data=remove=displaymatrix+replaygain output.mkv </pre>

 <h3 class=”section”>18.35 smpte436m_to_eia608# TOC</h3>

Convert from a <code class=”code”>SMPTE_436M_ANC</code> data stream to a <code class=”code”>EIA_608</code> stream, extracting the closed captions from CTA-708 CDP VANC packets, and ignoring all other data. <h3 class=”section”>18.36 text2movsub# TOC</h3>

Convert text subtitles to MOV subtitles (as used by the <code class=”code”>mov_text</code> codec) with metadata headers. See also the mov2textsub filter. <h3 class=”section”>18.37 trace_headers# TOC</h3>

Log trace output containing all syntax elements in the coded stream headers (everything above the level of individual coded blocks). This can be useful for debugging low-level stream issues. Supports AV1, H.264, H.265, (M)JPEG, MPEG-2 and VP9, but depending on the build only a subset of these may be available. <h3 class="section">18.38 trim# TOC</h3>

Drop packets outside a given range, similar to the trim and atrim filters, but operating on packets instead of decoded frames. The range is described by a <samp class="option">start</samp> and an <samp class="option">end</samp> bound. At least one of them must be set. Each bound is fully described by three independent choices: <ul class="itemize mark-bullet"> The discriminant it is compared against: the packet’s presentation timestamp, its decoding timestamp, or its index. The two bounds need not use the same one, for instance a <code class="code">dts</code> start with a <code class="code">pts</code> end. The unit its value is given in: the stream time base, milliseconds, or packets. What it is counted from: zero, or its reference packet when <samp class="option">start_rel</samp> / <samp class="option">end_rel</samp> is set. For <samp class="option">start</samp> the reference packet is the first packet carrying that discriminant. For <samp class="option">end</samp> it is the first exported packet.

The <samp class="option">start_type</samp> and <samp class="option">end_type</samp> options name a combination of the three. The <code class="code">dur_ts</code> and <code class="code">dur_t_msec</code> end types are the always relative forms of <code class="code">pts</code> and <code class="code">msec_pt</code>. A zero duration therefore produces an empty stream. Every packet is judged on its own discriminant, and on nothing else. A packet not carrying the discriminant its bound uses cannot be placed, and is dropped. <samp class="option">start_rel</samp> makes <samp class="option">start</samp> an offset from a reference packet instead of an absolute value. The reference is the first packet carrying the discriminant that bound uses. Its value is added to the offset once, and the bound stays there for the rest of the stream. <samp class="option">end_rel</samp> anchors on the first exported packet instead, at the position <samp class="option">start</samp> trimmed it to. The two bounds therefore move together. Only a discriminant that cannot come back into the range ends the stream, namely the packet index or the decoding timestamp. End of stream is signaled once one of those passes the <samp class="option">end</samp> bound. A presentation timestamp past the end only drops its packet, so that later packets still inside the range are not lost. A bound given in a time unit can fall inside a packet rather than between two. Such a packet is trimmed. Set <samp class="option">trim_packets</samp> to false to keep every packet exactly

as it came in: exported whole if its own value is inside the range, dropped otherwise. Audio is trimmed in sample space, through skip samples side data. The timestamps and duration are left untouched, and a decoder discards the signaled samples on output. This needs a sample rate and a time base. Audio missing either is rejected rather than trimmed on the timeline. Every other media type is trimmed by shifting the packet timestamps and shortening its duration. Trimming inside an audio packet needs the number of samples it decodes to. Where the codec does not give that number, the packet is handled as though <samp class="option">trim_packets</samp> were false. A skip larger than the packet carrying it continues into the packets after it. Dropping such a packet carries the remainder over to the next one. A skip that packet already carries replaces the remainder, as a decoder would, and the larger of it and this filter’s own trim wins. By default a packet outside the range is dropped, whether or not later packets depend on it. Streams of independent frames, such as ADTS or intra only video, can be cut anywhere. Cutting an inter coded stream inside a group of pictures leaves the first packets in range undecodable. Set <samp class="option">preroll</samp> to export the packets those first packets depend on as well. They are flagged for a decoder to decode and then drop, so the frames it outputs still begin at the <samp class="option">start</samp> bound. Only a decoder acts on that flag. Use <samp class="option">preroll</samp> where the stream is decoded. A muxer writes the extra packets like any other. A remux then covers more than the requested range. <samp class="option">preroll</samp> covers the start of the range only. On a stream with B-frames an <samp class="option">end</samp> bound on the presentation timestamp can drop a packet the last frames in the range need. Those frames then decode incorrectly. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Time or index marking the start of the accepted range. Packets before it are dropped. A packet straddling it is trimmed at its start, or dropped when <samp class="option">trim_packets</samp> is false. </dd> <dt><samp class="option">start_type</samp></dt> <dd> How to interpret <samp class="option">start</samp>: <dl class="table"> <dt>‘<samp class="samp">pts</samp>’</dt> <dd> Presentation timestamp, in stream time base (default). </dd> <dt>‘<samp class="samp">dts</samp>’</dt> <dd> Decoding timestamp, in stream time base. </dd> <dt>‘<samp class="samp">pkt_index</samp>’</dt> <dd> Packet index, counting from zero. </dd> <dt>‘<samp class="samp">msec_pt</samp>’</dt> <dd> Milliseconds, matched against the presentation timestamp. </dd> <dt>‘<samp class="samp">msec_dt</samp>’</dt> <dd> Milliseconds, matched against the decoding timestamp. </dd> </dl>

</dd> <dt><samp class="option">start_rel</samp></dt> <dd> Count <samp class="option">start</samp> from the reference packet rather than

from zero. Has no effect on ‘<samp class="samp">pkt_index</samp>’, whose first packet is already zero. Default false. </dd> <dt><samp class="option">end</samp></dt> <dd> Time or index marking the end of the accepted range. Signals end of stream when reached. A packet straddling it is trimmed at its end, or exported untouched when <samp class="option">trim_packets</samp> is false. </dd> <dt><samp class="option">end_type</samp></dt> <dd> How to interpret <samp class="option">end</samp>. Same values as <samp class="option">start_type</samp>, plus: <dl class="table"> <dt>‘<samp class="samp">dur_ts</samp>’</dt> <dd> Duration in stream time base. Same as ‘<samp class="samp">pts</samp>’ with <samp class="option">end_rel</samp> set. </dd> <dt>‘<samp class="samp">dur_t_msec</samp>’</dt> <dd> Duration in milliseconds. Same as ‘<samp class="samp">msec_pt</samp>’ with <samp class="option">end_rel</samp> set. </dd> </dl>

</dd> <dt><samp class="option">end_rel</samp></dt> <dd> Count <samp class="option">end</samp> from the first exported packet rather than from zero. The reference is that packet’s value on the discriminant <samp class="option">end_type</samp> names. On a timestamp it is taken at the position the packet is trimmed to. For audio that is the time of its first retained sample. The skip that packet carries can push it past the packet itself. With ‘<samp class="samp">pkt_index</samp>’ it exports a fixed number of packets, which can be combined with a timestamp-based <samp class="option">start</samp>. Always set for the duration types. Default false. </dd> <dt><samp class="option">trim_packets</samp></dt> <dd> Trim packets straddling a boundary. When false, packets are exported exactly as they came in, with no timestamp adjustment and no skip samples side data. A packet is then exported untouched if its value falls inside the range, and dropped otherwise. Default true. </dd> <dt><samp class="option">preroll</samp></dt> <dd> Export the packets a decoder needs to decode the first packet in range, rather than dropping them for falling before the <samp class="option">start</samp> bound. The filter holds every packet from the last keyframe on. It exports them ahead of the first packet in range, carrying <code class="code">AV_PKT_FLAG_DISCARD</code>. Video only. Default false. </dd> <dt><samp class="option">preroll_size</samp></dt> <dd> Maximum number of packets held for <samp class="option">preroll</samp>. A group of pictures longer than this is dropped whole, with a warning naming the count to raise it past. Default 300, which covers the keyframe intervals encoders commonly default to. </dd> </dl>

For example, to keep only the 10 first packets of an ADTS stream: <pre class="example-preformatted">ffmpeg -i INPUT.aac -c copy -bsf:a trim=end=10:end_type=pkt_index OUTPUT.aac </pre>

To keep 5 seconds starting 1.5 seconds into the stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

The same, expressed relative to the first packet so it works regardless of the stream’s starting timestamp: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:start_rel=1:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

To start on a keyframeless point of an inter coded video stream, re-encoding it so that the preroll is decoded rather than written out. The filter must run before the decoder, so it is given before the input: <pre class=”example-preformatted”>ffmpeg -bsf:v trim=start=17000:start_type=msec_pt:end=30000:end_type=dur_t_msec:preroll=1 -i INPUT.mp4 OUTPUT.mp4 </pre>

 <h3 class=”section”>18.39 truehd_core# TOC</h3>

Extract the core from a TrueHD stream, dropping ATMOS data. <h3 class=”section”>18.40 vp9_metadata# TOC</h3>

Modify metadata embedded in a VP9 stream. <dl class=”table”> <dt><samp class=”option”>color_space</samp></dt> <dd> Set the color space value in the frame header. Note that any frame set to RGB will be implicitly set to PC range and that RGB is incompatible with profiles 0 and 2. <dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</d
<dt>‘<samp class=”samp”>smpte170</samp>’</dt>
<dt>‘<samp class=”samp”>smpte240</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>rgb</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>color_range</samp></dt> <dd> Set the color range value in the frame header. Note that any value imposed by the color space will take precedence over this value. <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dt>‘<samp class=”samp”>pc</samp>’</dt> </dl> </dd> </dl>

 <h3 class=”section”>18.41 vp9_superframe<span

class=”pull-right”># TOC</h3>

Merge VP9 invisible (alt-ref) frames back into VP9 superframes. This fixes merging of split/segmented VP9 streams where the alt-ref frame was split from its visible counterpart. <h3 class=”section”>18.42 vp9_superframe_split# TOC</h3>

Split VP9 superframes into single frames. <h3 class=”section”>18.43 vp9_raw_reorder# TOC</h3>

Given a VP9 stream with correct timestamps but possibly out of order, insert additional show-existing-frame packets to correct the ordering. <h2 class=”chapter”>19 Format Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the muxers and demuxers. In addition each muxer or demuxer may support so-called private options, which are specific for that component. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class=”code”>AVFormatContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class=”table”> <dt><samp class=”option”>avioflags <var class=”var”>flags</var> (input/output)</samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>direct</samp>’</dt> <dd> Reduce buffering. </dd> </dl>

</dd> <dt><samp class=”option”>probesize <var class=”var”>integer</var> (input)</samp></dt> <dd> Set probing size in bytes, i.e. the size of the data to analyze to get stream information. A higher value will enable detecting more information in case it is dispersed into the stream, but will increase latency. Must be an integer not lesser than 32. It is 5000000 by default. </dd> <dt><samp class=”option”>max_probe_packets <var class=”var”>integer</var> (input)</samp></dt> <dd> Set the maximum number of buffered packets when probing a codec. Default is 2500 packets. </dd> <dt><samp class=”option”>packetsize <var

class=”var”>integer</var> (output)</samp></dt>
<dd> Set packet size. </dd> <dt><samp class=”option”>fflags
<var class=”var”>flags</var></samp></dt> <dd> Set format flags.
Some are implemented for a limited number of formats. Possible
values for input files: <dl class=”table”> <dt>‘<samp
class=”samp”>discardcorrupt</samp>’</dt> <dd> Discard cor-
rupted packets. </dd> <dt>‘<samp class=”samp”>fastseek</samp>’</dt>
<dd> Enable fast, but inaccurate seeks for some formats. </dd>
<dt>‘<samp class=”samp”>genpts</samp>’</dt> <dd> Generate
missing PTS if DTS is present. </dd> <dt>‘<samp
class=”samp”>igndts</samp>’</dt> <dd> Ignore DTS if
PTS is also set. In case the PTS is set, the DTS value is set to NOPTS. This is
ignored when the <code class=”code”>nofillin</code> flag is set. </dd>
<dt>‘<samp class=”samp”>ignidx</samp>’</dt> <dd> Ignore
index. </dd> <dt>‘<samp class=”samp”>nobuffer</samp>’</dt>
<dd> Reduce the latency introduced by buffering during initial input
streams analysis. </dd> <dt>‘<samp class=”samp”>nofillin</samp>’</dt>
<dd> Do not fill in missing values in packet fields that can be exactly calcu-
lated. </dd> <dt>‘<samp class=”samp”>noparse</samp>’</dt>
<dd> Disable AVParsers, this needs <code class=”code”>+nofillin</code>
too. </dd> <dt>‘<samp class=”samp”>sortdts</samp>’</dt>
<dd> Try to interleave output packets by DTS. At present, available only
for AVIs with an index. </dd> </dl>

Possible values for output files: <dl class=”table”> <dt>‘<samp
class=”samp”>autobsf</samp>’</dt> <dd> Automatically ap-
ply bitstream filters as required by the output format. Enabled by default.
</dd> <dt>‘<samp class=”samp”>bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. This
ensures that file and data checksums are reproducible and match be-
tween platforms. Its primary use is for regression testing. </dd>
<dt>‘<samp class=”samp”>flush_packets</samp>’</dt>
<dd> Write out packets immediately. </dd> <dt>‘<samp
class=”samp”>shortest</samp>’</dt> <dd> Stop muxing at the
end of the shortest stream. It may be needed to increase max_interleave_delta
to avoid flushing the longer streams before EOF. </dd> </dl>

</dd> <dt><samp class=”option”>seek2any <var class=”var”>integer</var>
(input)</samp></dt> <dd> Allow seeking
to non-keyframes on demuxer level when supported if set to 1. Default
is 0. </dd> <dt><samp class=”option”>analyzeduration <var
class=”var”>integer</var> (input)</samp></dt>
<dd> Specify how many microseconds are analyzed to probe the input.
A higher value will enable detecting more accurate information, but will
increase latency. It defaults to 5,000,000 microseconds = 5 seconds.
</dd> <dt><samp class=”option”>cryptokey <var class=”var”>hexadecimal
string</var> (input)</samp></dt> <dd> Set

decryption key. </dd> <dt><samp class="option">indexmem <var class="var">integer</var> (input)</samp></dt> <dd> Set max memory used for timestamp index (per stream). </dd> <dt><samp class="option">rtbufsize <var class="var">integer</var> (input)</samp></dt> <dd> Set max memory used for buffering real-time frames. </dd> <dt><samp class="option">fdebug <var class="var">flags</var> (input/output)</samp><
<dd> Print specific debug info. Possible values: <dl class="table"> <dt>‘<samp class="samp">ts</samp>’</dt> <dd> Print timestamp information. </dd> <dt>‘<samp class="samp">id3v2</samp>’</dt> <dd> Print ID3v2 frame data when demuxing files containing ID3v2 tags. </dd> </dl>

</dd> <dt><samp class="option">max_delay <var class="var">integer</var> (input/output)</samp></dt> <dd> Set maximum muxing or demuxing delay in microseconds. </dd> <dt><samp class="option">fpsprobesize <var class="var">integer</var> (input)</samp></dt> <dd> Set number of frames used to probe fps. </dd> <dt><samp class="option">audio_preload <var class="var">integer</var> (output)</samp></dt> <dd> Set microseconds by which audio packets should be interleaved earlier. </dd> <dt><samp class="option">chunk_duration <var class="var">integer</var> (output)</samp></dt> <dd> Set microseconds for each chunk. </dd> <dt><samp class="option">chunk_size <var class="var">integer</var> (output)</samp><
<dd> Set size in bytes for each chunk. </dd> <dt><samp class="option">err_detect, f_err_detect <var class="var">flags</var> (input)</samp></dt> <dd> Set error detection flags. <code class="code">f_err_detect</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt> <dd> Verify embedded CRCs. </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> Detect bitstream specification deviations. </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> Detect improper bitstream length. </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> Abort decoding on minor error detection. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> Consider things that violate the spec and have not been seen in the wild as errors. </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> Consider all spec non compliancies as errors. </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> Consider things that a sane encoder should not do as an error. </dd> </dl>

</dd> <dt><samp class="option">max_interleave_delta <var class="var">integer</var> (output)</samp></dt> <dd> Set maximum

buffering duration for interleaving. The duration is expressed in microseconds, and defaults to 10000000 (10 seconds). To ensure all the streams are interleaved correctly, libavformat will wait until it has at least one packet for each stream before actually writing any packets to the output file. When some streams are "sparse" (i.e. there are large gaps between successive packets), this can result in excessive buffering. This field specifies the maximum difference between the timestamps of the first and the last packet in the muxing queue, above which libavformat will output a packet regardless of whether it has queued a packet for all the streams. If set to 0, libavformat will continue buffering packets until it has a packet for each stream, regardless of the maximum timestamp difference between the buffered packets. </dd> <dt><samp class="option">use_wallclock_as_timestamps <var class="var">integer</var> (input)</samp></dt> <dd> Use wallclock as timestamps if set to 1. Default is 0. </dd> <dt><samp class="option">avoid_negative_ts <var class="var">integer</var> (output)</samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">make_non_negative</samp>’</dt> <dd> Shift timestamps to make them non-negative. Also note that this affects only leading negative timestamps, and not non-monotonic negative timestamps. </dd> <dt>‘<samp class="samp">make_zero</samp>’</dt> <dd> Shift timestamps so that the first timestamp is 0. </dd> <dt>‘<samp class="samp">auto (default)</samp>’</dt> <dd> Enables shifting when required by the target format. </dd> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Disables shifting of timestamp. </dd> </dl>

When shifting is enabled, all output timestamps are shifted by the same amount. Audio, video, and subtitles desynching and relative timestamp differences are preserved compared to how they would have been without shifting. </dd> <dt><samp class="option">skip_initial_bytes <var class="var">integer</var> (input)</samp></dt> <dd> Set number of bytes to skip before reading header and frames if set to 1. Default is 0. </dd> <dt><samp class="option">correct_ts_overflow <var class="var">integer</var> (input)</samp></dt> <dd> Correct single timestamp overflows if set to 1. Default is 1. </dd> <dt><samp class="option">flush_packets <var class="var">integer</var> (output)</samp></dt> <dd> Flush the underlying I/O stream after each packet. Default is -1 (auto), which means that the underlying protocol will decide, 1 enables it, and has the effect of reducing the latency, 0 disables it and may increase IO throughput in some cases. </dd> <dt><samp class="option">output_ts_offset <var class="var">offset</var> (output)</samp></dt> <dd> Set the output time offset. <var class="var">offset</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-

utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. The offset is added by the muxer to the output timestamps. Specifying a positive offset means that the corresponding streams are delayed bt the time duration specified in <var class=”var”>offset</var>. Default value is <code class=”code”>0</code> (meaning that no offset is applied). </dd> <dt><samp class=”option”>format_whitelist <var class=”var”>list</var> (input)</samp></dt> <dd> "," separated list of allowed demuxers. By default all are allowed. </dd> <dt><samp class=”option”>dump_separator <var class=”var”>string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class=”example-preformatted”>ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class=”option”>max_streams <var class=”var”>integer</var> (input)</samp></dt> <dd> Specifies the maximum number of streams. This can be used to reject files that would require too many resources due to a large number of streams. </dd> <dt><samp class=”option”>skip_estimate_duration_from_pts <var class=”var”>bool</var> (input)</samp></dt> <dd> Skip estimation of input duration if it requires an additional probing for PTS at end of file. At present, applicable for MPEG-PS and MPEG-TS. </dd> <dt><samp class=”option”>duration_probesize <var class=”var”>integer</var> (input)</samp></dt> <dd> Set probing size, in bytes, for input duration estimation when it actually requires an additional probing for PTS at end of file (at present: MPEG-PS and MPEG-TS). It is aimed at users interested in better durations probing for itself, or indirectly because using the concat demuxer, for example. The typical use case is an MPEG-TS CBR with a high bitrate, high video buffering and ending cleaning with similar PTS for video and audio: in such a scenario, the large physical gap between the last video packet and the last audio packet makes it necessary to read many bytes in order to get the video stream duration. Another use case is where the default probing behaviour only reaches a single video frame which is not the last one of the stream due to frame reordering, so the duration is not accurate. Setting this option has a performance impact even for small files because the probing size is fixed. Default behaviour is a general purpose trade-off, largely adaptive, but the probing size will not be extended to get streams durations at all costs. Must be an integer not lesser than 1, or 0 for default behaviour. </dd> <dt><samp class=”option”>strict, f_strict <var class=”var”>integer</var> (input/output)</samp></dt> <dd> Specify how strictly to follow the standards. <code class=”code”>f_strict</code> is deprecated and should be used only via the <code class=”command”>ffmpeg</code>

tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt> <dd> strictly conform to an older more strict version of the spec or reference software </dd> <dt>‘<samp class="samp">strict</samp>’</dt> <dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt> <dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> </dl>

 <h3 class="section">19.1 Format stream specifiers# TOC</h3>

Format stream specifiers allow selection of one or more streams that match specific properties. The exact semantics of stream specifiers is defined by the <code class="code">avformat_match_stream_specifier()</code> function declared in the <samp class="file">libavformat/avformat.h</samp> header and documented in the <a data-manual="ffmpeg" href="./ffmpeg.html#Stream-specifiers">Stream specifiers section in the ffmpeg(1) manual. <h2 class="chapter">20 Demuxers# TOC</h2>

Demuxers are configured elements in FFmpeg that can read the multimedia streams from a particular type of file. When you configure your FFmpeg build, all the supported demuxers are enabled by default. You can list all available ones using the configure option <code class="code">--list-demuxers</code>. You can disable all the demuxers using the configure option <code class="code">--disable-demuxers</code>, and selectively enable a single demuxer with the option <code class="code">--enable-demuxer=<var class="var">DEMUXER</var></code>, or disable it with the option <code class="code">--disable-demuxer=<var class="var">DEMUXER</var></code>. The option <code class="code">-demuxers</code> of the ff* tools will display the list of enabled demuxers. Use <code class="code">-formats</code> to view a combined list of enabled demuxers and muxers. The description of some of the currently available demuxers follows. <h3 class="section">20.1 aa# <a class="anchor

hidden-xs”href=”#toc-aa” aria-hidden=”true”>TOC</h3>

Audible Format 2, 3, and 4 demuxer. This demuxer is used to demux Audible Format 2, 3, and 4 (.aa) files. <h3 class=”section”>20.2 aac# TOC</h3>

Raw Audio Data Transport Stream AAC demuxer. This demuxer is used to demux an ADTS input containing a single AAC stream alongwith any ID3v1/2 or APE tags in it. <h3 class=”section”>20.3 apng# TOC</h3>

Animated Portable Network Graphics demuxer. This demuxer is used to demux APNG files. All headers, but the PNG signature, up to (but not including) the first fcTL chunk are transmitted as extradata. Frames are then split as being all the chunks between two fcTL ones, or between the last fcTL and IEND chunks. <dl class=”table”> <dt><samp class=”option”>-ignore_loop <var class=”var”>bool</var></samp></dt> <dd> Ignore the loop variable in the file if set. Default is enabled. </dd> <dt><samp class=”option”>-max_fps <var class=”var”>int</var></samp></dt> <dd> Maximum framerate in frames per second. Default of 0 imposes no limit. </dd> <dt><samp class=”option”>-default_fps <var class=”var”>int</var></samp></dt> <dd> Default framerate in frames per second when none is specified in the file (0 meaning as fast as possible). Default is 15. </dd> </dl>

 <h3 class=”section”>20.4 asf# TOC</h3>

Advanced Systems Format demuxer. This demuxer is used to demux ASF files and MMS network streams. <dl class=”table”> <dt><samp class=”option”>-no_resync_search <var class=”var”>bool</var></samp></dt> <dd> Do not try to resynchronize by looking for a certain optional start code. </dd> </dl>

 <h3 class=”section”>20.5 astc# TOC</h3>

ASTC (Adaptive Scalable Texture Compression) demuxer. This demuxer reads the 16-byte <samp class=”file”>.astc</samp> header, hands it to the decoder as extradata, and emits the compressed image as a single packet. <h3 class=”section”>20.6 concat# <a class=”anchor

hidden-xs”href=”#toc-concat” aria-hidden=”true”>TOC</h3>

Virtual concatenation script demuxer. This demuxer reads a list of files and other directives from a text file and demuxes them one after the other, as if all their packets had been muxed together. The timestamps in the files are adjusted so that the first file starts at 0 and each next file starts where the previous one finishes. Note that it is done globally and may cause gaps if all streams do not have exactly the same length. All files must have the same streams (same codecs, same time base, etc.). The duration of each file is used to adjust the timestamps of the next file: if the duration is incorrect (because it was computed using the bit-rate or because the file is truncated, for example), it can cause artifacts. The <code class=”code”>duration</code> directive can be used to override the duration stored in each file. <h4 class=”subsection”>20.6.1 Syntax# TOC</h4>

The script is a text file in extended-ASCII, with one directive per line.

Empty lines, leading spaces and lines starting with ’#’ are ignored. The

following directive is recognized:

<dl class=”table”>

<dt><samp class=”option”><code class=”code”>file <var class=”var”>path</var></code></samp></dt

<dd> Path to a file to read; special characters and spaces must be escaped with

backslash or single quotes.

All subsequent file-related directives apply to that file.

</dd>

<dt><samp class=”option”><code class=”code”>ffconcat version 1.0</code></samp></dt>

<dd> Identify the script type and version.

To make FFmpeg recognize the format automatically, this directive must

appear exactly as is (no extra space or byte-order-mark) on the very first

line of the script.

</dd>

<dt><samp class="option"><code class="code">duration <var class="var">dur</var></code></samp>

<dd> Duration of the file. This information can be specified from the file;

specifying it here may be more efficient or help if the information from the

file is not available or accurate.

If the duration is set for all files, then it is possible to seek in the

whole concatenated video.

</dd>

<dt><samp class="option"><code class="code">inpoint <var class="var">timestamp</var></code></sa

<dd> In point of the file. When the demuxer opens the file it instantly seeks to the

specified timestamp. Seeking is done so that all streams can be presented

successfully at In point.

This directive works best with intra frame codecs, because for non-intra frame

ones you will usually get extra packets before the actual In point and the

decoded content will most likely contain frames before In point too.

For each file, packets before the file In point will have timestamps less than

the calculated start timestamp of the file (negative in case of the first

file), and the duration of the files (if not specified by the <code class="code">duration</code>

directive) will be reduced based on their specified In point.

Because of potential packets before the specified In point, packet timestamps

may overlap between two concatenated files.

</dd>

<dt><samp class="option"><code class="code">outpoint <var class="var">timestamp</var></code></

<dd> Out point of the file. When the demuxer reaches the specified decoding

timestamp in any of the streams, it handles it as an end of file condition and

skips the current and all the remaining packets from all streams.

Out point is exclusive, which means that the demuxer will not output packets

with a decoding timestamp greater or equal to Out point.

This directive works best with intra frame codecs and formats where all streams

are tightly interleaved. For non-intra frame codecs you will usually get

additional packets with presentation timestamp after Out point therefore the

decoded content will most likely contain frames after Out point too. If your

streams are not tightly interleaved you may not get all the packets from all

streams before Out point and you may only will be able to decode the earliest

stream until Out point.

The duration of the files (if not specified by the <code class="code">duration</code>

directive) will be reduced based on their specified Out point.

</dd>

<dt><samp class="option"><code class="code">file_packet_metadata <var class="var">key=value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

This directive is deprecated, use <code class="code">file_packet_meta</code> instead.

</dd>

<dt><samp class="option"><code class="code">file_packet_meta <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

</dd>

<dt><samp class="option"><code class="code">option <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Option to access, open and probe the file.

Can be present multiple times.

</dd>

<dt><samp class="option"><code class="code">stream</code></samp></dt>

<dd> Introduce a stream in the virtual file.

All subsequent stream-related directives apply to the last introduced

stream.

Some streams properties must be set in order to allow identifying the

matching streams in the subfiles.

If no streams are defined in the script, the streams from the first file are

copied.

</dd>

<dt><samp class="option"><code class="code">exact_stream_id <var class="var">id</var></code></samp></dt>

<dd> Set the id of the stream.

If this directive is given, the string with the corresponding id in the

subfiles will be used.

This is especially useful for MPEG-PS (VOB) files, where the order of the

streams is not reliable.

</dd>

<dt><samp class=”option”><code class=”code”>stream_meta <var class=”var”>key</var> <var class=”var”>value</var></code></samp></dt>

<dd> Metadata for the stream.

Can be present multiple times.

</dd>

<dt><samp class=”option”><code class=”code”>stream_codec <var class=”var”>value</var></code></samp></dt>

<dd> Codec for the stream.

</dd>

<dt><samp class=”option”><code class=”code”>stream_extradata <var class=”var”>hex_string</var></code></samp></dt>

<dd> Extradata for the string, encoded in hexadecimal.

</dd>

<dt><samp class=”option”><code class=”code”>chapter <var class=”var”>id</var> <var class=”var”>start</var> <var class=”var”>end</var></code></samp></dt>

<dd> Add a chapter. <var class=”var”>id</var> is an unique identifier, possibly small and

consecutive.

</dd>

</dl>

 <h4 class=”subsection”>20.6.2 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>safe</samp></dt> <dd> If set to 1, reject unsafe file paths and directives. A file path is considered safe if it does not contain a protocol specification and is relative and all components only contain characters from the portable character set (letters, digits, period,

underscore and hyphen) and have no period at the beginning of a component. If set to 0, any file name is accepted. The default is 1. </dd> <dt><samp class="option">auto_convert</samp></dt> <dd> If set to 1, try to perform automatic conversions on packet data to make the streams concatenable. The default is 1. Currently, the only conversion is adding the h264_mp4toannexb bitstream filter to H.264 streams in MP4 format. This is necessary in particular if there are resolution changes. </dd> <dt><samp class="option">segment_time_metadata</samp></dt> <dd> If set to 1, every packet will contain the <var class="var">lavf.concat.start_time</var> and the <var class="var">lavf.concat.duration</var> packet metadata values which are the start_time and the duration of the respective file segments in the concatenated output expressed in microseconds. The duration metadata is only set if it is known based on the concat file. The default is 0. </dd> <dt><samp class="option">chapter_per_file</samp></dt> <dd> If set to 1, add a chapter for each file, named after the <code class="code">title</code> metadata of the file or, failing that, its file name, and carrying the rest of the file’s metadata. The files are opened in order when the script is read to learn their durations, stopping after the first file whose duration is unknown, which gets the last chapter; when all durations are known, this also makes the total duration known and the output seekable. The default is 0. </dd> </dl>

 <h4 class="subsection">20.6.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use absolute filenames and include some comments: <pre class="example-preformatted"># my first filename file /mnt/share/file-1.wav # my second filename including whitespace file '/mnt/share/file 2.wav' # my third filename including whitespace plus single quote file '/mnt/share/file 3'\''.wav' </pre>

Allow for input format auto-probing, use safe filenames and set the duration of the first file: <pre class="example-preformatted">ffconcat version 1.0

file file-1.wav duration 20.0

file subdir/file-2.wav </pre>

 <h3 class="section">20.7 dash# TOC</h3>

Dynamic Adaptive Streaming over HTTP demuxer. This demuxer presents all AVStreams found in the manifest. By setting the discard flags on AVStreams the caller can decide which streams to actu-

ally receive. Each stream mirrors the <code class=”code”>id</code> and <code class=”code”>bandwidth</code> properties from the <code class=”code”><Representation></code> as metadata keys named "id" and "variant_bitrate" respectively. <h4 class=”subsection”>20.7.1 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>cenc_decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>cenc_decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> </dl>

 <h3 class=”section”>20.8 dvdvideo# TOC</h3>

DVD-Video demuxer, powered by libdvdnav and libdvdread. Can directly ingest DVD titles, specifically sequential PGCs, into a conversion pipeline. Menu assets, such as background video or audio, can also be demuxed given the menu’s coordinates (at best effort). Block devices (DVD drives), ISO files, and directory structures are accepted. Activate with <code class=”code”>-f dvdvideo</code> in front of one of these inputs. This demuxer does NOT have decryption code of any kind. You are on your own working with encrypted DVDs, and should not expect support on the matter. Underlying playback is handled by libdvdnav, and structure parsing by libdvdread. FFmpeg must be built with GPL library support available as well as the configure switches <code class=”code”>--enable-libdvdnav</code> and <code class=”code”>--enable-libdvdread</code>. You will need to provide either the desired "title number" or exact PGC/PG coordinates. Many open-source DVD players and tools can aid in providing this information. If not specified, the demuxer will default to title 1 which works for many discs. However, due to the flexibility of the format, it is recommended to check manually. There are many discs that are authored strangely or with invalid headers. If the input is a real DVD drive, please note that there are some drives which may silently fail on reading bad sectors from the disc, returning random bits instead which is effectively corrupt data. This is especially prominent on aging or rotting discs. A second pass and integrity checks would be needed to detect the corruption. This is not an FFmpeg issue. <h4 class=”subsection”>20.8.1 Background<a

class=”anchor hidden-xs” href=”#Background” aria-hidden=”true”>#
TOC</h4>

DVD-Video is not a directly accessible, linear container format in the traditional sense. Instead, it allows for complex and programmatic playback of carefully muxed MPEG-PS streams that are stored in headerless VOB files. To the end-user, these streams are known simply as "titles", but the actual logical playback sequence is defined by one or more "PGCs", or Program Group Chains, within the title. The PGC is in turn comprised of multiple "PGs", or Programs", which are the actual video segments (and for a typical video feature, sequentially ordered). The PGC structure, along with stream layout and metadata, are stored in IFO files that need to be parsed. PGCs can be thought of as playlists in easier terms. An actual DVD player relies on user GUI interaction via menus and an internal VM to drive the direction of demuxing. Generally, the user would either navigate (via menus) or automatically be redirected to the PGC of their choice. During this process and the subsequent playback, the DVD player’s internal VM also maintains a state and executes instructions that can create jumps to different sectors during playback. This is why libdvdnav is involved, as a linear read of the MPEG-PS blobs on the disc (VOBs) is not enough to produce the right sequence in many cases. There are many other DVD structures (a long subject) that will not be discussed here. NAV packets, in particular, are handled by this demuxer to build accurate timing but not emitted as a stream. For a good high-level understanding, refer to: https://code.videolan.org/videolan/libdvdnav/-/blob/master/doc/dvd_structures
<h4 class=”subsection”>20.8.2 Options#
TOC</h4>

This demuxer accepts the following options: <dl class=”table”>
<dt><samp class=”option”>title <var class=”var”>int</var></samp></dt>
<dd> The title number to play. Must be set if <samp class=”option”>pgc</samp> and <samp class=”option”>pg</samp> are not set. Not applicable to menus. Default is 0 (auto), which currently only selects the first available title (title 1) and notifies the user about the implications. </dd> <dt><samp class=”option”>chapter_start <var class=”var”>int</var></samp></dt>
<dd> The chapter, or PTT (part-of-title), number to start at. Not applicable to menus. Default is 1. </dd> <dt><samp class=”option”>chapter_end <var class=”var”>int</var></samp></dt>
<dd> The chapter, or PTT (part-of-title), number to end at. Not applicable to menus. Default is 0, which is a special value to signal end at the last possible chapter. </dd> <dt><samp class=”option”>angle <var class=”var”>int</var></samp></dt> <dd> The video angle number, referring to what is essentially an additional video stream that is composed from alternate frames interleaved in the VOBs. Not applicable

to menus. Default is 1. </dd> <dt><samp class=”option”>region <var class=”var”>int</var></samp></dt> <dd> The region code to use for playback. Some discs may use this to default playback at a particular angle in different regions. This option will not affect the region code of a real DVD drive, if used as an input. Not applicable to menus. Default is 0, "world". </dd> <dt><samp class=”option”>menu <var class=”var”>bool</var></samp></dt> <dd> Demux menu assets instead of navigating a title. Requires exact coordinates of the menu (<samp class=”option”>menu_lu</samp>, <samp class=”option”>menu_vts</samp>, <samp class=”option”>pgc</samp>, <samp class=”option”>pg</samp>). Default is false. </dd> <dt><samp class=”option”>menu_lu <var class=”var”>int</var></samp></dt> <dd> The menu language to demux. In DVD, menus are grouped by language. Default is 1, the first language unit. </dd> <dt><samp class=”option”>menu_vts <var class=”var”>int</var></samp></dt> <dd> The VTS where the menu lives, or 0 if it is a VMG menu (root-level). Default is 1, menu of the first VTS. </dd> <dt><samp class=”option”>pgc <var class=”var”>int</var></samp></dt> <dd> The entry PGC to start playback, in conjunction with <samp class=”option”>pg</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Must be explicitly set for menus. Default is 0, automatically resolve from value of <samp class=”option”>title</samp>. </dd> <dt><samp class=”option”>pg <var class=”var”>int</var></samp></dt> <dd> The entry PG to start playback, in conjunction with <samp class=”option”>pgc</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Default is 1, the first PG of the PGC. </dd> <dt><samp class=”option”>preindex <var class=”var”>bool</var></samp></dt> <dd> Enable this to have accurate chapter (PTT) markers and duration measurement, which requires a slow second pass read in order to index the chapter marker timestamps from NAV packets. This is non-ideal extra work for real optical drives. It is recommended and faster to use this option with a backup of the DVD structure stored on a hard drive. Not compatible with <samp class=”option”>pgc</samp> and <samp class=”option”>pg</samp>. Default is 0, false. </dd> <dt><samp class=”option”>trim <var class=”var”>bool</var></samp></dt> <dd> Skip padding cells (i.e. cells shorter than 1 second) from the beginning. There exist many discs with filler segments at the beginning of the PGC, often with junk data intended for controlling a real DVD player’s buffering speed and with no other material data value. Not applicable to menus. Default is 1, true. </dd> </dl>

 <h4 class=”subsection”>20.8.3 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-9” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Open title 3 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -title 3 -i <path to DVD> ... </pre>

Open chapters 3-6 from title 1 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -chapter_start 3 -chapter_end 6 -title 1 -i <path to DVD> ... </pre>

Open only chapter 5 from title 1 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -chapter_start 5 -chapter_end 5 -title 1 -i <path to DVD> ... </pre>

Demux menu with language 1 from VTS 1, PGC 1, starting at PG 1: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -menu 1 -menu_lu 1 -menu_vts 1 -pgc 1 -pg 1 -i <path to DVD> ... </pre>

 <h3 class=”section”>20.9 ea# TOC</h3>

Electronic Arts Multimedia format demuxer. This format is used by various Electronic Arts games. <h4 class=”subsection”>20.9.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>merge_alpha <var class=”var”>bool</var></samp></dt> <dd> Normally the VP6 alpha channel (if exists) is returned as a secondary video stream, by setting this option you can make the demuxer return a single video stream which contains the alpha channel in addition to the ordinary video. </dd> </dl>

 <h3 class=”section”>20.10 imf# TOC</h3>

Interoperable Master Format demuxer. This demuxer presents audio and video streams found in an IMF Composition, as specified in SMPTE ST 2067-2.   <pre class=”example-preformatted”>ffmpeg [-assetmaps <path of ASSETMAP1>,<path of ASSETMAP2>,...] -i <path of CPL> ... </pre>

If <code class=”code”>-assetmaps</code> is not specified, the demuxer looks for a file called <samp class=”file”>ASSETMAP.xml</samp> in the

same directory as the CPL. <h3 class="section">20.11 flv, live_flv, kux# TOC</h3>

Adobe Flash Video Format demuxer. This demuxer is used to demux FLV files and RTMP network streams. In case of live network streams, if you force format, you may use live_flv option instead of flv to survive timestamp discontinuities. KUX is a flv variant used on the Youku platform.   <pre class="example-preformatted">ffmpeg -f flv -i myfile.flv ... ffmpeg -f live_flv -i rtmp://<any.server>/anything/key .... </pre>

<dl class="table"> <dt><samp class="option">-flv_metadata <var class="var">bool</var></samp></dt> <dd> Allocate the streams according to the onMetaData array content. </dd> <dt><samp class="option">-flv_ignore_prevtag <var class="var">bool</var></samp></dt> <dd> Ignore the size of previous tag value. </dd> <dt><samp class="option">-flv_full_metadata <var class="var">bool</var></samp></dt> <dd> Output all context of the onMetadata. </dd> </dl>

 <h3 class="section">20.12 gif# TOC</h3>

Animated GIF demuxer. It accepts the following options: <dl class="table"> <dt><samp class="option">min_delay</samp></dt> <dd> Set the minimum valid delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 2. </dd> <dt><samp class="option">max_gif_delay</samp></dt> <dd> Set the maximum valid delay between frames in hundredth of seconds. Range is 0 to 65535. Default value is 65535 (nearly eleven minutes), the maximum value allowed by the specification. </dd> <dt><samp class="option">default_delay</samp></dt> <dd> Set the default delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 10. </dd> <dt><samp class="option">ignore_loop</samp></dt> <dd> GIF files can contain information to loop a certain number of times (or infinitely). If <samp class="option">ignore_loop</samp> is set to 1, then the loop setting from the input will be ignored and looping will not occur. If set to 0, then looping will occur and will cycle the number of times according to the GIF. Default value is 1. </dd> </dl>

For example, with the overlay filter, place an infinitely looping GIF over another video: <pre class="example-preformatted">ffmpeg -i input.mp4 -ignore_loop 0 -i input.gif -filter_complex overlay=shortest=1 out.mkv </pre>

Note that in the above example the shortest option for overlay filter is used

to end the output video at the length of the shortest input file, which in this case is <samp class=”file”>input.mp4</samp> as the GIF in this example loops infinitely. <h3 class=”section”>20.13 hls# TOC</h3>

HLS demuxer Apple HTTP Live Streaming demuxer. This demuxer presents all AVStreams from all variant streams. The id field is set to the bitrate variant index number. By setting the discard flags on AVStreams (by pressing ’a’ or ’v’ in ffplay), the caller can decide which variant streams to actually receive. The total bitrate of the variant that the stream belongs to is available in a metadata key named "variant_bitrate". It is unset when the stream is shared by variants with different bitrates. The value is always available in the corresponding program’s metadata. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>live_start_index</samp></dt> <dd> segment index to start live streams at (negative values are from the end). </dd> <dt><samp class=”option”>prefer_x_start</samp></dt> <dd> prefer to use #EXT-X-START if it’s in playlist instead of live_start_index. </dd> <dt><samp class=”option”>allowed_extensions</samp></dt> <dd> ’,’ separated list of file extensions that hls is allowed to access. </dd> <dt><samp class=”option”>extension_picky</samp></dt> <dd> This blocks disallowed extensions from probing It also requires all available segments to have matching extensions to the format except mpegts, which is always allowed. It is recommended to set the whitelists correctly instead of depending on extensions Enabled by default. </dd> <dt><samp class=”option”>max_reload</samp></dt> <dd> Maximum number of times a insufficient list is attempted to be reloaded. Default value is 1000. </dd> <dt><samp class=”option”>m3u8_hold_counters</samp></dt> <dd> The maximum number of times to load m3u8 when it refreshes without new segments. Default value is 1000. </dd> <dt><samp class=”option”>http_persistent</samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP streams. Enabled by default. </dd> <dt><samp class=”option”>http_multiple</samp></dt> <dd> Use multiple HTTP connections for downloading HTTP segments. Enabled by default for HTTP/1.1 servers. </dd> <dt><samp class=”option”>http_seekable</samp></dt> <dd> Use HTTP partial requests for downloading HTTP segments. 0 = disable, 1 = enable, -1 = auto, Default is auto. </dd> <dt><samp class=”option”>seg_format_options</samp></dt> <dd> Set options for the demuxer of media segments using a list of key=value pairs separated by <code class=”code”>:</code>. </dd> <dt><samp class=”option”>seg_max_retry</samp></dt> <dd> Maximum number of times to reload a segment on error, useful when segment skip on network error is not desired. Default value is 0.

</dd> </dl>

 <h3 class="section">20.14 image2# TOC</h3>

Image file demuxer. This demuxer reads from a list of image files specified by a pattern. The syntax and meaning of the pattern is specified by the option <var class="var">pattern_type</var>. The pattern may contain a suffix which is used to automatically determine the format of the images contained in the files. The size, the pixel format, and the format of each image must be the same for all the files in the sequence. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set the frame rate for the video stream. It defaults to 25. </dd> <dt><samp class="option">loop</samp></dt> <dd> If set to 1, loop over the input. Default value is 0. </dd> <dt><samp class="option">pattern_type</samp></dt> <dd> Select the pattern type used to interpret the provided filename. <var class="var">pattern_type</var> accepts one of the following values. <dl class="table"> <dt><samp class="option">none</samp></dt> <dd> Disable pattern matching, therefore the video will only contain the specified image. You should use this option if you do not want to create sequences from multiple images and your filenames may contain special pattern characters. </dd> <dt><samp class="option">sequence</samp></dt> <dd> Select a sequence pattern type, used to specify a sequence of files indexed by sequential numbers. A sequence pattern may contain the string "%d" or "%0<var class="var">N</var>d", which specifies the position of the characters representing a sequential number in each filename matched by the pattern. If the form "%d0<var class="var">N</var>d" is used, the string representing the number in each filename is 0-padded and <var class="var">N</var> is the total number of 0-padded digits representing the number. The literal character ’%’ can be specified in the pattern with the string "%%". If the sequence pattern contains "%d" or "%0<var class="var">N</var>d", the first filename of the file list specified by the pattern must contain a number inclusively contained between <var class="var">start_number</var> and <var class="var">start_number</var>+<var class="var">start_number_range</var>-1, and all the following numbers must be sequential. For example the pattern "img-%03d.bmp" will match a sequence of filenames of the form <samp class="file">img-001.bmp</samp>, <samp class="file">img-002.bmp</samp>, ..., <samp class="file">img-010.bmp</samp>, etc.; the pattern "i%%m%%g-%d.jpg" will match a sequence of filenames of the form <samp class="file">i%m%g-1.jpg</samp>, <samp class="file">i%m%g-2.jpg</samp>, ..., <samp

class=”file”>i%m%g-10.jpg</samp>, etc. Note that the pattern must not necessarily contain "%d" or "%0<var class=”var”>N</var>d", for example to convert a single image file <samp class=”file”>img.jpeg</samp> you can employ the command: <pre class=”example-preformatted”>ffmpeg -i img.jpeg img.png </pre>

</dd> <dt><samp class=”option”>glob</samp></dt> <dd> Select a glob wildcard pattern type. The pattern is interpreted like a <code class=”code”>glob()</code> pattern. This is only selectable if libavformat was compiled with globbing support. </dd> </dl>

Default value is <var class=”var”>sequence</var>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Set the pixel format of the images to read. If not specified the pixel format is guessed from the first image file in the sequence. </dd> <dt><samp class=”option”>start_number</samp></dt> <dd> Set the index of the file matched by the image file pattern to start to read from. Default value is 0. </dd> <dt><samp class=”option”>start_number_range</samp></dt> <dd> Set the index interval range to check when looking for the first image file in the sequence, starting from <var class=”var”>start_number</var>. Default value is 5. </dd> <dt><samp class=”option”>ts_from_file</samp></dt> <dd> If set to 1, will set frame timestamp to modification time of image file. Note that monotonity of timestamps is not provided: images go in the same order as without this option. Default value is 0. If set to 2, will set frame timestamp to the modification time of the image file in nanosecond precision. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size of the images to read. If not specified the video size is guessed from the first image file in the sequence. </dd> <dt><samp class=”option”>export_path_metadata</samp></dt> <dd> If set to 1, will add two extra fields to the metadata found in input, making them also available for other filters (see <var class=”var”>drawtext</var> filter for examples). Default value is 0. The extra fields are described below: <dl class=”table”> <dt><samp class=”option”>lavf.image2dec.source_path</samp></dt> <dd> Corresponds to the full path to the input file being read. </dd> <dt><samp class=”option”>lavf.image2dec.source_basename</samp></dt> <dd> Corresponds to the name of the file being read. </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>20.14.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> for creating a video from the images in the file sequence <samp class=”file”>img-001.jpeg</samp>, <samp class=”file”>img-002.jpeg</samp>, ..., assuming

an input frame rate of 10 frames per second: <pre class=”example-preformatted”>ffmpeg -framerate 10 -i 'img-%03d.jpeg' out.mkv </pre>

As above, but start by reading from a file with index 100 in the sequence: <pre class=”example-preformatted”>ffmpeg -framerate 10 -start_number 100 -i 'img-%03d.jpeg' out.mkv </pre>

Read images matching the "*.png" glob pattern , that is all the files terminating with the ".png" suffix: <pre class=”example-preformatted”>ffmpeg -framerate 10 -pattern_type glob -i "*.png" out.mkv </pre>

 <h3 class=”section”>20.15 jpeg_mpf# TOC</h3>

JPEG Multi-Picture Format demuxer. Exposes each JPEG image stored in a CIPA DC-007 MP file as a separate stream, in the order of the MP Entries, with the First Individual Image flagged as the default stream. If the file contains an ISO 21496-1 gain map, as is the case for JPEG Ultra HDR, the primary image and the gain map are linked by a ‘<samp class=”samp”>gain_map</samp>’ stream group, and the gain map parameters are exported as frame side data by the decoder. Images of any other type (e.g. large thumbnails or multi-view images) are exposed as additional streams. This demuxer accepts the following options, which behave like the options of the same name of the ‘<samp class=”samp”>image2</samp>’ demuxer: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate of the video streams. The default is 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the images in the file. The default is 0. This requires seekable input. </dd> </dl>

 <h3 class=”section”>20.16 ktx# TOC</h3>

Khronos Texture 1.0 demuxer for ASTC textures. This demuxer reads the KTX 1.0 header and maps the ASTC GL internal format to the block footprint the decoder needs, so only files using the ASTC formats are accepted. A <code class=”code”>KTXorientation</code> key is preserved as stream display metadata, and the texture is emitted as a single packet. <h3 class=”section”>20.17 libgme# TOC</h3>

The Game Music Emu library is a collection of video game music file emulators. See https://bitbucket.org/mpyne/game-music-emu/overview for more information. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>track_index</samp></dt> <dd> Set the index of which track to demux. The demuxer can only export one track. Track indexes start at 0. Default is to pick the first track. Number of tracks is exported as <var class=”var”>tracks</var> metadata entry. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sampling rate of the exported track. Range is 1000 to 999999. Default is 44100. </dd> <dt><samp class=”option”>max_size (bytes)</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Default is 50 MiB. </dd> </dl>

 <h3 class=”section”>20.18 libmodplug# TOC</h3>

ModPlug based module demuxer See https://github.com/Konstanty/libmodplug It will export one 2-channel 16-bit 44.1 kHz audio stream. Optionally, a <code class=”code”>pal8</code> 16-color video stream can be exported with or without printed metadata. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>noise_reduction</samp></dt> <dd> Apply a simple low-pass filter. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class=”option”>reverb_depth</samp></dt> <dd> Set amount of reverb. Range 0-100. Default is 0. </dd> <dt><samp class=”option”>reverb_delay</samp></dt> <dd> Set delay in ms, clamped to 40-250 ms. Default is 0. </dd> <dt><samp class=”option”>bass_amount</samp></dt> <dd> Apply bass expansion a.k.a. XBass or megabass. Range is 0 (quiet) to 100 (loud). Default is 0. </dd> <dt><samp class=”option”>bass_range</samp></dt> <dd> Set cutoff i.e. upper-bound for bass frequencies. Range is 10-100 Hz. Default is 0. </dd> <dt><samp class=”option”>surround_depth</samp></dt> <dd> Apply a Dolby Pro-Logic surround effect. Range is 0 (quiet) to 100 (heavy). Default is 0. </dd> <dt><samp class=”option”>surround_delay</samp></dt> <dd> Set surround delay in ms, clamped to 5-40 ms. Default is 0. </dd> <dt><samp class=”option”>max_size</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Range is 0 to 100 MiB. 0 removes buffer size limit (not recommended). Default is 5 MiB. </dd> <dt><samp class=”option”>video_stream_expr</samp></dt> <dd> String which

is evaluated using the eval API to assign colors to the generated video stream. Variables which can be used are <code class=”code”>x</code>, <code class=”code”>y</code>, <code class=”code”>w</code>, <code class=”code”>h</code>, <code class=”code”>t</code>, <code class=”code”>speed</code>, <code class=”code”>tempo</code>, <code class=”code”>order</code>, <code class=”code”>pattern</code> and <code class=”code”>row</code>. </dd> <dt><samp class=”option”>video_stream</samp></dt> <dd> Generate video stream. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class=”option”>video_stream_w</samp></dt> <dd> Set video frame width in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class=”option”>video_stream_h</samp></dt> <dd> Set video frame height in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class=”option”>video_stream_ptxt</samp></dt> <dd> Print metadata on video stream. Includes <code class=”code”>speed</code>, <code class=”code”>tempo</code>, <code class=”code”>order</code>, <code class=”code”>pattern</code>, <code class=”code”>row</code> and <code class=”code”>ts</code> (time in ms). Can be 1 (on) or 0 (off). Default is 1. </dd> </dl>

 <h3 class=”section”>20.19 libopenmpt# TOC</h3>

libopenmpt based module demuxer See https://lib.openmpt.org/libopenmpt/ for more information. Some files have multiple subsongs (tracks) this can be set with the <samp class=”option”>subsong</samp> option. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>subsong</samp></dt> <dd> Set the subsong index. This can be either ’all’, ’auto’, or the index of the subsong. Subsong indexes start at 0. The default is ’auto’. The default value is to let libopenmpt choose. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> Set the channel layout. Valid values are 1, 2, and 4 channel layouts. The default value is STEREO. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate for libopenmpt to output. Range is from 1000 to INT_MAX. The value default is 48000. </dd> </dl>

 <h3 class=”section”>20.20 mcc# TOC</h3>

Demuxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-

608 and CEA-708), ancillary time code, pan-scan data, etc. By default, for backward compatibility, the MCC demuxer extracts just the EIA-608 and CEA-708 closed captions and returns a <code class="code">EIA_608</code> stream, ignoring all other VANC data. You can change it to return all VANC data in a <code class="code">SMPTE_436M_ANC</code> data stream by setting <samp class="option">-eia608_extract 0</samp> <h4 class="subsection">20.20.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert a MCC file to Scenarist (SCC) format: <pre class="example-preformatted">ffmpeg -i CC.mcc -c:s copy CC.scc </pre> Note that the SCC format only supports EIA-608, so this will discard all other data such as CEA-708 extensions. Merge a MCC file into a MXF file: <pre class="example-preformatted">ffmpeg -i video_and_audio.mxf -eia608_extract 0 -i CC.mcc -c copy -map 0 -map 1 out.mxf </pre> This retains all VANC data and inserts it into the output MXF file as a <code class="code">SMPTE_436M_ANC</code> data stream.

 <h3 class="section">20.21 mov/mp4/3gp# TOC</h3>

Demuxer for Quicktime File Format & ISO/IEC Base Media File Format (ISO/IEC 14496-12 or MPEG-4 Part 12, ISO/IEC 15444-12 or JPEG 2000 Part 12). Registered extensions: mov, mp4, m4a, 3gp, 3g2, mj2, psp, m4b, ism, ismv, isma, f4v <h4 class="subsection">20.21.1 Options# TOC</h4>

This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">enable_drefs</samp></dt> <dd> Enable loading of external tracks, disabled by default. Enabling this can theoretically leak information in some use cases. </dd> <dt><samp class="option">use_absolute_path</samp></dt> <dd> Allows loading of external tracks via absolute paths, disabled by default. Enabling this poses a security risk. It should only be enabled if the source is known to be non-malicious. </dd> <dt><samp class="option">seek_streams_individually</samp></dt> <dd> When seeking, identify the closest point in each stream individually and demux packets in that stream from identified point. This can lead to a different sequence of packets compared to demuxing linearly from the beginning. Default is true. </dd> <dt><samp

class=”option”>ignore_editlist</samp></dt> <dd> Ignore any edit list atoms. The demuxer, by default, modifies the stream index to reflect the timeline described by the edit list. Default is false. </dd> <dt><samp class=”option”>advanced_editlist</samp></dt> <dd> Modify the stream index to reflect the timeline described by the edit list. <code class=”code”>ignore_editlist</code> must be set to false for this option to be effective. If both <code class=”code”>ignore_editlist</code> and this option are set to false, then only the start of the stream index is modified to reflect initial dwell time or starting timestamp described by the edit list. Default is true. </dd> <dt><samp class=”option”>ignore_chapters</samp></dt> <dd> Don’t parse chapters. This includes GoPro ’HiLight’ tags/moments. Note that chapters are only parsed when input is seekable. Default is false. </dd> <dt><samp class=”option”>use_mfra_for</samp></dt> <dd> For seekable fragmented input, set fragment’s starting timestamp from media fragment random access box, if present. Following options are available: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Auto-detect whether to set mfra timestamps as PTS or DTS (default) </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Set mfra timestamps as DTS </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Set mfra timestamps as PTS </dd> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Don’t use mfra box to set timestamps </dd> </dl>

</dd> <dt><samp class=”option”>use_tfdt</samp></dt> <dd> For fragmented input, set fragment’s starting timestamp to <code class=”code”>baseMediaDecodeTime</code> from the <code class=”code”>tfdt</code> box. Default is enabled, which will prefer to use the <code class=”code”>tfdt</code> box to set DTS. Disable to use the <code class=”code”>earliest_presentation_time</code> from the <code class=”code”>sidx</code> box. In either case, the timestamp from the <code class=”code”>mfra</code> box will be used if it’s available and <code class=”code”>use_mfra_for</code> is set to pts or dts. </dd> <dt><samp class=”option”>export_all</samp></dt> <dd> Export unrecognized boxes within the <var class=”var”>udta</var> box as metadata entries. The first four characters of the box type are set as the key. Default is false. </dd> <dt><samp class=”option”>export_xmp</samp></dt> <dd> Export entire contents of <var class=”var”>XMP_</var> box and <var class=”var”>uuid</var> box as a string with key <code class=”code”>xmp</code>. Note that if <code class=”code”>export_all</code> is set and this option isn’t, the contents of <var class=”var”>XMP_</var> box are still exported but with key <code class=”code”>XMP_</code>. Default is false. </dd> <dt><samp class=”option”>activation_bytes</samp></dt> <dd> 4-byte key required to decrypt Audible AAX and AAX+ files. See Audible AAX subsection below. </dd> <dt><samp class=”option”>audible_fixed_key</samp></dt> <dd> Fixed key used

for handling Audible AAX/AAX+ files. It has been pre-set so should not be necessary to specify. </dd> <dt><samp class="option">decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class="option">decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class="option">max_stts_delta</samp></dt> <dd> Very high sample deltas written in a trak’s stts box may occasionally be intended but usually they are written in error or used to store a negative value for dts correction when treated as signed 32-bit integers. This option lets the user set an upper limit, beyond which the delta is clamped to 1. Values greater than the limit if negative when cast to int32 are used to adjust onward dts. Unit is the track time scale. Range is 0 to UINT_MAX. Default is <code class="code">UINT_MAX - 48000*10</code> which allows up to a 10 second dts correction for 48 kHz audio streams while accommodating 99.9% of <code class="code">uint32</code> range. </dd> <dt><samp class="option">interleaved_read</samp></dt> <dd> Interleave packets from multiple tracks at demuxer level. For badly interleaved files, this prevents playback issues caused by large gaps between packets in different tracks, as MOV/MP4 do not have packet placement requirements. However, this can cause excessive seeking on very badly interleaved files, due to seeking between tracks, so disabling it may prevent I/O issues, at the expense of playback. </dd> </dl>

 <h4 class="subsection">20.21.2 Audible AAX# TOC</h4>

Audible AAX files are encrypted M4B files, and they can be decrypted by specifying a 4 byte activation secret. <pre class="example-preformatted">ffmpeg -activation_bytes 1CEB00DA -i test.aax -vn -c:a copy output.mp4 </pre>

 <h3 class="section">20.22 mpegts# TOC</h3>

MPEG-2 transport stream demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">resync_size</samp></dt> <dd> Set size limit for looking up a new synchronization. Default value is 65536. </dd> <dt><samp class="option">skip_unknown_pmt</samp></dt> <dd> Skip PMTs for programs not defined in the PAT. Default value is 0. </dd> <dt><samp class="option">fix_teletext_pts</samp></dt> <dd> Override teletext packet PTS and DTS values with the times-

tamps calculated from the PCR of the first program which the teletext stream is part of and is not discarded. Default value is 1, set this option to 0 if you want your teletext packet PTS and DTS values untouched. </dd> <dt><samp class=”option”>ts_packetsize</samp></dt> <dd> Output option carrying the raw packet size in bytes. Show the detected raw packet size, cannot be set by the user. </dd> <dt><samp class=”option”>scan_all_pmts</samp></dt> <dd> Scan and combine all PMTs. The value is an integer with value from -1 to 1 (-1 means automatic setting, 1 means enabled, 0 means disabled). Default value is -1. </dd> <dt><samp class=”option”>merge_pmt_versions</samp></dt> <dd> Reuse existing streams when a PMT’s version is updated and elementary streams move to different PIDs. Default value is 0. </dd> <dt><samp class=”option”>max_packet_size</samp></dt> <dd> Set maximum size, in bytes, of packet emitted by the demuxer. Payloads above this size are split across multiple packets. Range is 184 to INT_MAX/2. Default is 204800 bytes. </dd> </dl>

 <h3 class=”section”>20.23 mpjpeg# TOC</h3>

MJPEG encapsulated in multi-part MIME demuxer. This demuxer allows reading of MJPEG, where each frame is represented as a part of multipart/x-mixed-replace stream. <dl class=”table”> <dt><samp class=”option”>strict_mime_boundary</samp></dt> <dd> Default implementation applies a relaxed standard to multi-part MIME boundary detection, to prevent regression with numerous existing endpoints not generating a proper MIME MJPEG stream. Turning this option on by setting it to 1 will result in a stricter check of the boundary value. </dd> </dl>

 <h3 class=”section”>20.24 rawvideo# TOC</h3>

Raw video demuxer. This demuxer allows one to read raw video data. Since there is no header specifying the assumed video parameters, the user must specify them in order to be able to decode the data correctly. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set input video frame rate. Default value is 25. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Set the input video pixel format. Default value is <code class=”code”>yuv420p</code>. </dd> <dt><samp class=”option”>stride</samp></dt> <dd> Set frame line size in bytes, only required if there are extra padding. For multiplane formats, <samp class=”option”>stride</samp> is a list of line size for each plane. </dd> <dt><samp class=”option”>video_size</samp></dt>

<dd> Set the input video size. This value must be specified explicitly. </dd> </dl>

For example to read a rawvideo file <samp class="file">input.raw</samp> with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">rgb24</code>, a video size of <code class="code">320x240</code>, and a frame rate of 10 images per second, use the command: <pre class="example-preformatted">ffplay -f rawvideo -pixel_format rgb24 -video_size 320x240 -framerate 10 input.raw </pre>

Read a rawvideo with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">yuv420p</code>, a video size of <code class="code">1080x1920</code>, has 8 bytes of padding in each line of Y plane, and 4 bytes of padding with UV plane, <pre class="example-preformatted">ffplay -f rawvideo -pixel_format yuv420p -video_size 1080x1920 -stride 1088,544,544 input.raw </pre>

 <h3 class="section">20.25 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. For more information on the format, see <a data-manual="ffmpeg-formats" href="./ffmpeg-formats.html#rcwtenc">(ffmpeg-formats)rcwtenc. This demuxer implements the specification as of March 2024, which has been stable and unchanged since April 2014. <h4 class="subsection">20.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Render CC to ASS using the built-in decoder: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin CC.ass </pre> Note that if your output appears to be empty, you may have to manually set the decoder’s <samp class="option">data_field</samp> option to pick the desired CC substream. Convert an RCWT backup to Scenarist (SCC) format: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin -c:s copy CC.scc </pre> Note that the SCC format does not support all of the possible CC extensions that can be stored in RCWT (such as EIA-708).

 <h3 class="section">20.26 sbg# TOC</h3>

SBaGen script demuxer. This demuxer reads the script language used by SBaGen http://uazu.net/sbagen/ to generate binaural beats sessions. A SBG script looks like that: <pre class="example-preformatted">-SE a: 300-2.5/3 440+4.5/0 b: 300-2.5/0 440+4.5/3 off: - NOW == a +0:07:00 == b +0:14:00 == a +0:21:00 == b +0:30:00 off </pre>

A SBG script can mix absolute and relative timestamps. If the script uses either only absolute timestamps (including the script start time) or only relative ones, then its layout is fixed, and the conversion is straightforward. On the other hand, if the script mixes both kind of timestamps, then the <var class="var">NOW</var> reference for relative timestamps will be taken from the current time of day at the time the script is read, and the script layout will be frozen according to that reference. That means that if the script is directly played, the actual times will match the absolute timestamps up to the sound controller’s clock accuracy, but if the user somehow pauses the playback or seeks, all times will be shifted accordingly. <h3 class="section">20.27 tedcaptions# TOC</h3>

JSON captions used for TED Talks. TED does not provide links to the captions, but they can be guessed from the page. The file <samp class="file">tools/bookmarklets.html</samp> from the FFmpeg source tree contains a bookmarklet to expose them. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">start_time</samp></dt> <dd> Set the start time of the TED talk, in milliseconds. The default is 15000 (15s). It is used to sync the captions with the downloadable videos, because they include a 15s intro. </dd> </dl>

Example: convert the captions to a format most players understand: <pre class="example-preformatted">ffmpeg -i http://www.ted.com/talks/subtitles/id/1/lang/en talk1-en.srt </pre>

 <h3 class="section">20.28 vapoursynth# TOC</h3>

Vapoursynth wrapper. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class="code">-f vapoursynth</code> before the input <code class="code">-i yourscript.vpy</code>. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">max_script_size</samp></dt> <dd> The demuxer buffers the entire script into memory. Adjust this value to set the maximum

buffer size, which in turn, acts as a ceiling for the size of scripts that can be read. Default is 1 MiB. </dd> </dl>

 <h3 class="section">20.29 w64# TOC</h3>

Sony Wave64 Audio demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">max_size</samp></dt> <dd> See the same option for the wav demuxer. </dd> </dl>

 <h3 class="section">20.30 wav# TOC</h3>

RIFF Wave Audio demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">ignore_length <var class="var">bool</var></samp></dt> <dd> Ignore the size of the <code class="code">data</code> chunk and keep reading until the end of the file if set. May be useful to read broken or partial files where the header was not properly updated, but will misinterpret files with non-audio chunks after the <code class="code">data</code> chunk. Default is disabled. </dd> <dt><samp class="option">max_size</samp></dt> <dd> Specify the maximum packet size in bytes for the demuxed packets. By default this is set to 0, which means that a sensible value is chosen based on the input format. </dd> </dl>

 <h3 class="section">20.31 webp# TOC</h3>

Animated WebP demuxer. It accepts the following options: <dl class="table"> <dt><samp class="option">-min_delay <var class="var">int</var></samp></dt> <dd> Set the minimum valid delay between frames in milliseconds. Range is 0 to 60000. Default value is 10. </dd> <dt><samp class="option">-max_webp_delay <var class="var">int</var></samp></dt> <dd> Set the maximum valid delay between frames in milliseconds. Range is 0 to 16777215. Default value is 16777215 (over four hours), the maximum value allowed by the specification. </dd> <dt><samp class="option">-default_delay <var class="var">int</var></samp></dt> <dd> Set the default delay between frames in milliseconds. Range is 0 to 60000. Default value is 100. </dd> <dt><samp class="option">-ignore_loop <var class="var">bool</var></samp></dt> <dd> WebP files can contain information to loop a certain number of times (or infinitely). If <samp class="option">ignore_loop</samp> is set to true, then the loop setting from the input will be ignored and looping will not occur. If set to false, then

looping will occur and will cycle the number of times according to the WebP. Default value is true. </dd> <dt><samp class="option">usebgcolor <var class="var">bool</var></samp></dt> <dd> WebP files contain a background color hint in the ANIM chunk, but the WebP specification says that viewer applications are not required to use it. If <samp class="option">usebgcolor</samp> is set to true, then the background color hint will be used, otherwise transparent black will be used for the background. Default value is false. </dd> </dl>

 <h2 class="chapter">21 Muxers#
TOC</h2>

Muxers are configured elements in FFmpeg which allow writing multimedia streams to a particular type of file. When you configure your FFmpeg build, all the supported muxers are enabled by default. You can list all available muxers using the configure option <code class="code">--list-muxers</code>. You can disable all the muxers with the configure option <code class="code">--disable-muxers</code> and selectively enable / disable single muxers with the options <code class="code">--enable-muxer=<var class="var">MUXER</var></code> / <code class="code">--disable-muxer=<var class="var">MUXER</var></code>. The option <code class="code">-muxers</code> of the ff* tools will display the list of enabled muxers. Use <code class="code">-formats</code> to view a combined list of enabled demuxers and muxers. A description of some of the currently available muxers follows. <h3 class="section">21.1 Raw muxers# TOC</h3>

This section covers raw muxers. They accept a single stream matching

the designated codec. They do not store timestamps or metadata. The

recognized extension is the same as the muxer name unless indicated

otherwise.

It comprises the following muxers. The media type and the eventual

extensions used to automatically selects the muxer from the output

extensions are also shown.

<dl class="table">

<dt>‘<samp class="samp">ac3 audio</samp>’</dt>

<dd> Dolby Digital, also known as AC-3.

</dd>

<dt>‘<samp class="samp">adx audio</samp>’</dt>

<dd> CRI Middleware ADX audio.

This muxer will write out the total sample count near the start of the

first packet when the output is seekable and the count can be stored

in 32 bits.

</dd>

<dt>‘<samp class="samp">aptx audio</samp>’</dt>

<dd> aptX (Audio Processing Technology for Bluetooth)

</dd>

<dt>‘<samp class="samp">aptx_hd audio
(aptxdh)</samp>’</dt>

<dd> aptX HD (Audio Processing Technology for Bluetooth) audio

</dd>

<dt>‘<samp class="samp">avs2 video
(avs, avs2)</samp>’</dt>

<dd> AVS2-P2 (Audio Video Standard - Second generation - Part 2) /

IEEE 1857.4 video

</dd>

<dt>‘<samp class="samp">avs3 video
(avs3)</samp>’</dt>

<dd> AVS3-P2 (Audio Video Standard - Third generation - Part 2) /

IEEE 1857.10 video

</dd>

<dt>‘<samp class="samp">cavsvideo video
(cavs)</samp>’</dt>

<dd> Chinese AVS (Audio Video Standard - First generation)

</dd>

<dt>‘<samp class="samp">codec2raw audio</samp>’</dt>

<dd> Codec 2 audio.

No extension is registered so format name has to be supplied e.g. with

the ffmpeg CLI tool <code class="code">-f codec2raw</code>.

</dd>

<dt>‘<samp class="samp">data any</samp>’</dt>

<dd> Generic data muxer.

This muxer accepts a single stream with any codec of any type. The

input stream has to be selected using the <code class="code">-map</code>
option with the

<code class="command">ffmpeg</code> CLI tool.

No extension is registered so format name has to be supplied e.g. with

the <code class="command">ffmpeg</code> CLI tool <code class="code">-
f data</code>.

</dd>

<dt>‘<samp class="samp">dfpwm audio
(dfpwm)</samp>’</dt>

<dd> Raw DFPWM1a (Dynamic Filter Pulse With Modulation) audio
muxer.

</dd>

<dt>‘<samp class="samp">dirac video
(drc, vc2)</samp>’</dt>

<dd> BBC Dirac video.

The Dirac Pro codec is a subset and is standardized as SMPTE VC-2.

</dd>

<dt>‘<samp class="samp">dnxhd video
(dnxhd, dnxhr)</samp>’</dt>

<dd> Avid DNxHD video.

It is standardized as SMPTE VC-3. Accepts DNxHR streams.

</dd>

<dt>‘<samp class="samp">dts audio</samp>’</dt>

<dd> DTS Coherent Acoustics (DCA) audio

</dd>

<dt>‘<samp class="samp">eac3 audio</samp>’</dt>

<dd> Dolby Digital Plus, also known as Enhanced AC-3

</dd>

<dt>‘<samp class="samp">evc video
(evc)</samp>’</dt>

<dd> MPEG-5 Essential Video Coding (EVC) / EVC / MPEG-5 Part 1
EVC video

</dd>

<dt>‘<samp class="samp">g722 audio</samp>’</dt>

<dd> ITU-T G.722 audio

</dd>

<dt>‘<samp class="samp">g723_1 audio
(tco, rco)</samp>’</dt>

<dd> ITU-T G.723.1 audio

</dd>

<dt>‘<samp class="samp">g726 audio</samp>’</dt>

<dd> ITU-T G.726 big-endian ("left-justified") audio.

No extension is registered so format name has to be supplied e.g. with

the <code class="command">ffmpeg</code> CLI tool <code class="code">-
f g726</code>.

</dd>

<dt>‘<samp class="samp">g726le audio</samp>’</dt>

<dd> ITU-T G.726 little-endian ("right-justified") audio.

No extension is registered so format name has to be supplied e.g. with

the <code class="command">ffmpeg</code> CLI tool <code class="code">-
f g726le</code>.

</dd>

<dt>‘<samp class="samp">gsm audio</samp>’</dt>

<dd> Global System for Mobile Communications audio

</dd>

<dt>‘<samp class="samp">h261 video</samp>’</dt>

<dd> ITU-T H.261 video

</dd>

<dt>‘<samp class="samp">h263 video</samp>’</dt>

<dd> ITU-T H.263 / H.263-1996, H.263+ / H.263-1998 / H.263 version 2
video

</dd>

<dt>‘<samp class="samp">h264 video (h264, 264)</samp>’</dt>

<dd> ITU-T H.264 / MPEG-4 Part 10 AVC video. Bitstream shall be converted

to Annex B syntax if it’s in length-prefixed mode.

</dd>

<dt>‘<samp class="samp">hevc video (hevc, h265, 265)</samp>’</dt>

<dd> ITU-T H.265 / MPEG-H Part 2 HEVC video. Bitstream shall be converted

to Annex B syntax if it’s in length-prefixed mode.

</dd>

<dt>‘<samp class="samp">m4v video</samp>’</dt>

<dd> MPEG-4 Part 2 video

</dd>

<dt>‘<samp class="samp">mjpeg video (mjpg, mjpeg)</samp>’</dt>

<dd> Motion JPEG video

</dd>

<dt>‘<samp class="samp">mlp audio</samp>’</dt>

<dd> Meridian Lossless Packing, also known as Packed PCM

</dd>

<dt>‘<samp class="samp">mp2 audio (mp2, m2a, mpa)</samp>’</dt>

<dd> MPEG-1 Audio Layer II audio

</dd>

<dt>‘<samp class="samp">mpeg1video video
(mpg, mpeg, m1v)</samp>’</dt>

<dd> MPEG-1 Part 2 video.

</dd>

<dt>‘<samp class="samp">mpeg2video video
(m2v)</samp>’</dt>

<dd> ITU-T H.262 / MPEG-2 Part 2 video

</dd>

<dt>‘<samp class="samp">obu video</samp>’</dt>

<dd> AV1 low overhead Open Bitstream Units muxer.

Temporal delimiter OBUs will be inserted in all temporal units of the

stream.

</dd>

<dt>‘<samp class="samp">rawvideo video
(yuv, rgb)</samp>’</dt>

<dd> Raw uncompressed video.

</dd>

<dt>‘<samp class="samp">sbc audio (sbc,
msbc)</samp>’</dt>

<dd> Bluetooth SIG low-complexity subband codec audio

</dd>

<dt>‘<samp class="samp">truehd audio
(thd)</samp>’</dt>

<dd> Dolby TrueHD audio

</dd>

<dt>‘<samp class="samp">vc1 video</samp>’</dt>

<dd> SMPTE 421M / VC-1 video

</dd>

</dl>

 <h4 class="subsection">21.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Store raw video frames with the ‘<samp class="samp">rawvideo</samp>’ muxer using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f lavfi -i testsrc -t 10 -s hd1080p testsrc.yuv </pre>

Since the rawvideo muxer do not store the information related to size and format, this information must be provided when demuxing the file: <pre class="example-preformatted">ffplay -video_size 1920x1080 -pixel_format rgb24 -f rawvideo testsrc.rgb </pre>

 <h3 class="section">21.2 Raw PCM muxers# TOC</h3>
This section covers raw PCM (Pulse-Code Modulation) audio muxers. They accept a single stream matching the designated codec. They do not store timestamps or metadata. The recognized extension is the same as the muxer name. It comprises the following muxers. The optional additional extension used to automatically select the muxer from the output extension is also shown in parentheses. <dl class="table"> <dt>‘<samp class="samp">alaw (al)</samp>’</dt> <dd> PCM A-law </dd> <dt>‘<samp class="samp">f32be</samp>’</dt> <dd> PCM 32-bit floating-point big-endian </dd> <dt>‘<samp class="samp">f32le</samp>’</dt> <dd> PCM 32-bit floating-point little-endian </dd> <dt>‘<samp class="samp">f64be</samp>’</dt> <dd> PCM 64-bit floating-point big-endian </dd> <dt>‘<samp class="samp">f64le</samp>’</dt> <dd> PCM 64-bit floating-point little-endian </dd> <dt>‘<samp class="samp">mulaw (ul)</samp>’</dt> <dd> PCM mu-law </dd> <dt>‘<samp class="samp">s16be</samp>’</dt> <dd> PCM signed 16-bit big-endian </dd> <dt>‘<samp class="samp">s16le</samp>’</dt> <dd> PCM signed 16-bit little-endian </dd> <dt>‘<samp class="samp">s24be</samp>’</dt> <dd> PCM signed 24-bit

big-endian </dd> <dt>‘<samp class="samp">s24le</samp>’</dt> <dd> PCM signed 24-bit little-endian </dd> <dt>‘<samp class="samp">s32be</samp>’</dt> <dd> PCM signed 32-bit big-endian </dd> <dt>‘<samp class="samp">s32le</samp>’</dt> <dd> PCM signed 32-bit little-endian </dd> <dt>‘<samp class="samp">s8 (sb)</samp>’</dt> <dd> PCM signed 8-bit </dd> <dt>‘<samp class="samp">u16be</samp>’</dt> <dd> PCM unsigned 16-bit big-endian </dd> <dt>‘<samp class="samp">u16le</samp>’</dt> <dd> PCM unsigned 16-bit little-endian </dd> <dt>‘<samp class="samp">u24be</samp>’</dt> <dd> PCM unsigned 24-bit big-endian </dd> <dt>‘<samp class="samp">u24le</samp>’</dt> <dd> PCM unsigned 24-bit little-endian </dd> <dt>‘<samp class="samp">u32be</samp>’</dt> <dd> PCM unsigned 32-bit big-endian </dd> <dt>‘<samp class="samp">u32le</samp>’</dt> <dd> PCM unsigned 32-bit little-endian </dd> <dt>‘<samp class="samp">u8 (ub)</samp>’</dt> <dd> PCM unsigned 8-bit </dd> <dt>‘<samp class="samp">vidc</samp>’</dt> <dd> PCM Archimedes VIDC </dd> </dl>

 <h3 class="section">21.3 MPEG-1/MPEG-2 program stream muxers# TOC</h3>

This section covers formats belonging to the MPEG-1 and MPEG-2 Systems family. The MPEG-1 Systems format (also known as ISO/IEEC 11172-1 or MPEG-1 program stream) has been adopted for the format of media track stored in VCD (Video Compact Disc). The MPEG-2 Systems standard (also known as ISO/IEEC 13818-1) covers two containers formats, one known as transport stream and one known as program stream; only the latter is covered here. The MPEG-2 program stream format (also known as VOB due to the corresponding file extension) is an extension of MPEG-1 program stream: in addition to support different codecs for the audio and video streams, it also stores subtitles and navigation metadata. MPEG-2 program stream has been adopted for storing media streams in SVCD and DVD storage devices. This section comprises the following muxers. <dl class="table"> <dt>‘<samp class="samp">mpeg (mpg,mpeg)</samp>’</dt> <dd> MPEG-1 Systems / MPEG-1 program stream muxer. </dd> <dt>‘<samp class="samp">vcd</samp>’</dt> <dd> MPEG-1 Systems / MPEG-1 program stream (VCD) muxer. This muxer can be used to generate tracks in the format accepted by the VCD (Video Compact Disc) storage devices. It is the same as the ‘<samp class="samp">mpeg</samp>’ muxer with a few differences.

</dd> <dt>‘<samp class="samp">vob</samp>’</dt> <dd> MPEG-2 program stream (VOB) muxer. </dd> <dt>‘<samp class="samp">dvd</samp>’</dt> <dd> MPEG-2 program stream (DVD VOB) muxer. This muxer can be used to generate tracks in the format accepted by the DVD (Digital Versatile Disc) storage devices. This is the same as the ‘<samp class="samp">vob</samp>’ muxer with a few differences. </dd> <dt>‘<samp class="samp">svcd (vob)</samp>’</dt> <dd> MPEG-2 program stream (SVCD VOB) muxer. This muxer can be used to generate tracks in the format accepted by the SVCD (Super Video Compact Disc) storage devices. This is the same as the ‘<samp class="samp">vob</samp>’ muxer with a few differences. </dd> </dl>

 <h4 class="subsection">21.3.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">muxrate <var class="var">rate</var></samp></dt> <dd> Set user-defined mux rate expressed as a number of bits/s. If not specified the automatically computed mux rate is employed. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">preload <var class="var">delay</var></samp></dt> <dd> Set initial demux-decode delay in microseconds. Default value is <code class="code">500000</code>. </dd> </dl>

 <h3 class="section">21.4 MOV/MPEG-4/ISOMBFF muxers# TOC</h3>

This section covers formats belonging to the QuickTime / MOV family, including the MPEG-4 Part 14 format and ISO base media file format (ISOBMFF). These formats share a common structure based on the ISO base media file format (ISOBMFF). The MOV format was originally developed for use with Apple QuickTime. It was later used as the basis for the MPEG-4 Part 1 (later Part 14) format, also known as ISO/IEC 14496-1. That format was then generalized into ISOBMFF, also named MPEG-4 Part 12 format, ISO/IEC 14496-12, or ISO/IEC 15444-12. It comprises the following muxers. <dl class="table"> <dt>‘<samp class="samp">3gp</samp>’</dt> <dd> Third Generation Partnership Project (3GPP) format for 3G UMTS multimedia services </dd> <dt>‘<samp class="samp">3g2</samp>’</dt> <dd> Third Generation Partnership Project 2 (3GP2 or 3GPP2) format for 3G CDMA2000 multimedia services, similar to ‘<samp class="samp">3gp</samp>’ with extensions and limitations

</dd> <dt>‘<samp class="samp">f4v</samp>’</dt> <dd> Adobe Flash Video format </dd> <dt>‘<samp class="samp">ipod</samp>’</dt> <dd> MPEG-4 audio file format, as MOV/MP4 but limited to contain only audio streams, typically played with the Apple ipod device </dd> <dt>‘<samp class="samp">ismv</samp>’</dt> <dd> Microsoft IIS (Internet Information Services) Smooth Streaming Audio/Video (ISMV or ISMA) format. This is based on MPEG-4 Part 14 format with a few incompatible variants, used to stream media files for the Microsoft IIS server. </dd> <dt>‘<samp class="samp">mov</samp>’</dt> <dd> QuickTime player format identified by the <code class="code">.mov</code> extension </dd> <dt>‘<samp class="samp">mp4</samp>’</dt> <dd> MP4 or MPEG-4 Part 14 format </dd> <dt>‘<samp class="samp">psp</samp>’</dt> <dd> PlayStation Portable MP4/MPEG-4 Part 14 format variant. This is based on MPEG-4 Part 14 format with a few incompatible variants, used to play files on PlayStation devices. </dd> </dl>

 <h4 class="subsection">21.4.1 Stereoscopic 3D# TOC</h4>

When the output stream has stereoscopic 3D side data (see the <samp class="option">-stereo3d</samp> option), these muxers write: <ul class="itemize mark-bullet"> MP4: Google spatial-media <code class="code">st3d</code> and Apple <code class="code">vexu</code> can be written for supported metadata with <samp class="option">-strict unofficial</samp>. Stricter compliance levels skip these boxes. Warnings are logged when stereoscopic or spherical side data is present, naming <code class="code">st3d</code> or <code class="code">sv3d</code>, respectively; there is no corresponding compliance warning for <code class="code">vexu</code>. Monoscopic layouts do not produce an <code class="code">eyes</code> box. MOV: Apple <code class="code">vexu</code> eyes only. <code class="code">st3d</code> is not written.

 <h4 class="subsection">21.4.2 Fragmentation# TOC</h4>

The ‘<samp class="samp">mov</samp>’, ‘<samp class="samp">mp4</samp>’, and ‘<samp class="samp">ismv</samp>’ muxers support fragmentation. Normally, a MOV/MP4 file has all the metadata about all packets stored in one location. This data is usually written at the end of the file, but it can be moved to the start for better playback by adding <code class="code">+faststart</code> to the <code

class=”code”>-movflags</code>, or using the <code class=”command”>qt-faststart</code> tool). A fragmented file consists of a number of fragments, where packets and metadata about these packets are stored together. Writing a fragmented file has the advantage that the file is decodable even if the writing is interrupted (while a normal MOV/MP4 is undecodable if it is not properly finished), and it requires less memory when writing very long files (since writing normal MOV/MP4 files stores info about every single packet in memory until the file is closed). The downside is that it is less compatible with other applications. Fragmentation is enabled by setting one of the options that define how to cut the file into fragments: <dl class=”table”> <dt><samp class=”option”>frag_duration</samp></dt> <dt><samp class=”option”>frag_size</samp></dt> <dt><samp class=”option”>min_frag_duration</samp></dt> <dt><samp class=”option”>movflags +frag_keyframe</samp></dt> <dt><samp class=”option”>movflags +frag_custom</samp></dt> </dl>

If more than one condition is specified, fragments are cut when one of the specified conditions is fulfilled. The exception to this is the option <samp class=”option”>min_frag_duration</samp>, which has to be fulfilled for any of the other conditions to apply. <h4 class=”subsection”>21.4.3 Options# TOC</h4>

<dl class=”table”>

<dt><samp class=”option”>brand <var class=”var”>brand_string</var></samp></dt>

<dd> Override major brand.

</dd>

<dt><samp class=”option”>empty_hdlr_name <var class=”var”>bool</var></samp></dt>

<dd> Enable to skip writing the name inside a <code class=”code”>hdlr</code> box.

Default is <code class=”code”>false</code>.

</dd>

<dt><samp class=”option”>encryption_key <var class=”var”>key</var></samp></dt>

<dd> set the media encryption key in hexadecimal format

</dd>

<dt><samp class=”option”>encryption_kid <var class=”var”>kid</var></samp></dt>

<dd> set the media encryption key identifier in hexadecimal format

</dd>

<dt><samp class="option">encryption_scheme <var class="var">scheme</var></samp></dt>

<dd> configure the encryption scheme, allowed values are ‘<samp class="samp">none</samp>’, and

‘<samp class="samp">cenc-aes-ctr</samp>’

</dd>

<dt><samp class="option">frag_duration <var class="var">duration</var></samp></dt>

<dd> Create fragments that are <var class="var">duration</var> microseconds long.

</dd>

<dt><samp class="option">frag_interleave <var class="var">number</var></samp></dt>

<dd> Interleave samples within fragments (max number of consecutive

samples, lower is tighter interleaving, but with more overhead. It is

set to <code class="code">0</code> by default.

</dd>

<dt><samp class="option">frag_size <var class="var">size</var></samp></dt>

<dd> create fragments that contain up to <var class="var">size</var> bytes of payload data

</dd>

<dt><samp class="option">iods_audio_profile <var class="var">profile</var></samp></dt>

<dd> specify iods number for the audio profile atom (from -1 to 255),

default is <code class="code">-1</code>

</dd>

<dt><samp class="option">iods_video_profile <var class="var">profile</var></samp></dt>

<dd> specify iods number for the video profile atom (from -1 to 255),

default is <code class=”code”>-1</code>

</dd>

<dt><samp class=”option”>ism_lookahead <var class=”var”>num_entries</var></samp></dt>

<dd> specify number of lookahead entries for ISM files (from 0 to 255),

default is <code class=”code”>0</code>

</dd>

<dt><samp class=”option”>min_frag_duration <var class=”var”>duration</var></samp></dt>

<dd> do not create fragments that are shorter than <var class=”var”>duration</var>
microseconds long

</dd>

<dt><samp class=”option”>moov_size <var class=”var”>bytes</var></samp></dt>

<dd> Reserves space for the moov atom at the beginning of the file instead
of placing the

moov atom at the end. If the space reserved is insufficient, muxing will fail.

</dd>

<dt><samp class=”option”>mov_gamma <var class=”var”>gamma</var></samp></dt>

<dd> specify gamma value for gama atom (as a decimal number from 0 to
10),

default is <code class=”code”>0.0</code>, must be set together with <code
class=”code”>+ movflags</code>

</dd>

<dt><samp class=”option”>movflags <var class=”var”>flags</var></samp></dt>

<dd> Set various muxing switches. The following flags can be used:

<dl class=”table”>

<dt>‘<samp class=”samp”>cmaf</samp>’</dt>

<dd> write CMAF (Common Media Application Format) compatible frag-
mented

MP4 output

</dd>

<dt>‘<samp class="samp">dash</samp>’</dt>

<dd> write DASH (Dynamic Adaptive Streaming over HTTP) compatible fragmented

MP4 output

</dd>

<dt>‘<samp class="samp">default_base_moof</samp>’</dt>

<dd> Similarly to the ‘<samp class="samp">omit_tfhd_offset</samp>’ flag, this flag avoids

writing the absolute base_data_offset field in tfhd atoms, but does so

by using the new default-base-is-moof flag instead. This flag is new

from 14496-12:2012. This may make the fragments easier to parse in

certain circumstances (avoiding basing track fragment location

calculations on the implicit end of the previous track fragment).

</dd>

<dt>‘<samp class="samp">delay_moov</samp>’</dt>

<dd> delay writing the initial moov until the first fragment is cut, or

until the first fragment flush

</dd>

<dt>‘<samp class="samp">disable_chpl</samp>’</dt>

<dd> Disable Nero chapter markers (chpl atom). Normally, both Nero chapters

and a QuickTime chapter track are written to the file. With this

option set, only the QuickTime chapter track will be written. Nero

chapters can cause failures when the file is reprocessed with certain

tagging programs, like mp3Tag 2.61a and iTunes 11.3, most likely other

versions are affected as well.

</dd>

<dt>‘<samp class="samp">faststart</samp>’</dt>

<dd> Run a second pass moving the index (moov atom) to the beginning of the

file. This operation can take a while, and will not work in various

situations such as fragmented output, thus it is not enabled by

default.

</dd>

<dt>‘<samp class="samp">frag_custom</samp>’</dt>

<dd> Allow the caller to manually choose when to cut fragments, by calling

<code class="code">av_write_frame(ctx, NULL)</code> to write a fragment with the packets

written so far. (This is only useful with other applications

integrating libavformat, not from <code class="command">ffmpeg</code>.)

</dd>

<dt>‘<samp class="samp">frag_discont</samp>’</dt>

<dd> signal that the next fragment is discontinuous from earlier ones

</dd>

<dt>‘<samp class="samp">frag_every_frame</samp>’</dt>

<dd> fragment at every frame

</dd>

<dt>‘<samp class="samp">frag_keyframe</samp>’</dt>

<dd> start a new fragment at each video keyframe

</dd>

<dt>‘<samp class="samp">global_sidx</samp>’</dt>

<dd> write a global sidx index at the start of the file

</dd>

<dt>‘<samp class="samp">isml</samp>’</dt>

<dd> create a live smooth streaming feed (for pushing to a publishing point)

</dd>

<dt>‘<samp class="samp">negative_cts_offsets</samp>’</dt>

<dd> Enables utilization of version 1 of the CTTS box, in which the CTS offsets can

be negative. This enables the initial sample to have DTS/CTS of zero, and

reduces the need for edit lists for some cases such as video tracks with

B-frames. Additionally, eases conformance with the DASH-IF interoperability

guidelines.

This option is implicitly set when writing ‘<samp class="samp">ismv</samp>’ (Smooth

Streaming) files.

</dd>

<dt>‘<samp class="samp">omit_tfhd_offset</samp>’</dt>

<dd> Do not write any absolute base_data_offset in tfhd atoms. This avoids

tying fragments to absolute byte positions in the file/streams.

</dd>

<dt>‘<samp class="samp">prefer_icc</samp>’</dt>

<dd> If writing colr atom prioritise usage of ICC profile if it exists in

stream packet side data.

</dd>

<dt>‘<samp class="samp">rtphint</samp>’</dt>

<dd> add RTP hinting tracks to the output file

</dd>

<dt>‘<samp class="samp">separate_moof</samp>’</dt>

<dd> Write a separate moof (movie fragment) atom for each track. Normally,

packets for all tracks are written in a moof atom (which is slightly

more efficient), but with this option set, the muxer writes one

moof/mdat pair for each track, making it easier to separate tracks.

</dd>

<dt>‘<samp class="samp">skip_sidx</samp>’</dt>

<dd> Skip writing of sidx atom. When bitrate overhead due to sidx atom is

high, this option could be used for cases where sidx atom is not

mandatory. When the ‘<samp class="samp">global_sidx</samp>’ flag is enabled, this option

is ignored.

</dd>

<dt>‘<samp class="samp">skip_trailer</samp>’</dt>

<dd> skip writing the mfra/tfra/mfro trailer for fragmented files

</dd>

<dt>‘<samp class="samp">use_metadata_tags</samp>’</dt>

<dd> use mdta atom for metadata

</dd>

<dt>‘<samp class="samp">write_colr</samp>’</dt>

<dd> write colr atom even if the color info is unspecified. This flag is

experimental, may be renamed or changed, do not use from scripts.

</dd>

<dt>‘<samp class="samp">write_gama</samp>’</dt>

<dd> write deprecated gama atom

</dd>

<dt>‘<samp class="samp">hybrid_fragmented</samp>’</dt>

<dd> For recoverability - write the output file as a fragmented file.

This allows the intermediate file to be read while being written

(in particular, if the writing process is aborted uncleanly). When

writing is finished, the file is converted to a regular, non-fragmented

file, which is more compatible and allows easier and quicker seeking.

If writing is aborted, the intermediate file can manually be

remuxed to get a regular, non-fragmented file of what had been

written into the unfinished file.

</dd>

</dl>

</dd> <dt><samp class="option">movie_timescale <var class="var">scale</var></samp></dt>
<dd> Set the timescale written in the movie header box (<code
class="code">mvhd</code>). Range is 1 to INT_MAX. Default is <code
class="code">1000</code>. </dd> <dt><samp class="option">rtpflags
<var class="var">flags</var></samp></dt> <dd> Add RTP hinting
tracks to the output file. The following flags can be used: <dl
class="table"> <dt>‘<samp class="samp">h264_mode0</samp>’</dt>
<dd> use mode 0 for H.264 in RTP </dd> <dt>‘<samp
class="samp">latm</samp>’</dt> <dd> use MP4A-LATM
packetization instead of MPEG4-GENERIC for AAC </dd>
<dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> use
RFC 2190 packetization instead of RFC 4629 for H.263 </dd>
<dt>‘<samp class="samp">send_bye</samp>’</dt>
<dd> send RTCP BYE packets when finishing </dd> <dt>‘<samp
class="samp">skip_rtcp</samp>’</dt> <dd> do not send
RTCP sender reports </dd> </dl>

</dd> <dt><samp class="option">skip_iods <var class="var">bool</var></samp></dt>
<dd> skip writing iods atom (default value is <code class="code">true</code>)
 </dd> <dt><samp class="option">use_editlist <var class="var">bool</var></samp></dt>
<dd> use edit list (default value is <code class="code">auto</code>)

 </dd> <dt><samp class="option">use_stream_ids_as_track_ids <var class="var">bool</var></samp></dt> <dd> use stream ids as track ids (default value is <code class="code">false</code>) </dd> <dt><samp class="option">video_track_timescale <var class="var">scale</var></samp></dt> <dd> Set the timescale used for video tracks. Range is <code class="code">0</code> to INT_MAX. If set to <code class="code">0</code>, the timescale is automatically set based on the native stream time base. Default is <code class="code">0</code>. </dd> <dt><samp class="option">write_btrt <var class="var">bool</var></samp></dt> <dd> Force or disable writing bitrate box inside stsd box of a track. The box contains decoding buffer size (in bytes), maximum bitrate and average bitrate for the track. The box will be skipped if none of these values can be computed. Default is <code class="code">-1</code> or <code class="code">auto</code>, which will write the box only in MP4 mode. </dd> <dt><samp class="option">write_prft <var class="var">option</var></samp></dt> <dd> Write producer time reference box (PRFT) with a specified time source for the NTP field in the PRFT box. Set value as ‘<samp class="samp">wallclock</samp>’ to specify timesource as wallclock time and ‘<samp class="samp">pts</samp>’ to specify timesource as input packets’ PTS values. </dd> <dt><samp class="option">write_tmcd <var class="var">bool</var></samp></dt> <dd> Specify <code class="code">on</code> to force writing a timecode track, <code class="code">off</code> to disable it and <code class="code">auto</code> to write a timecode track only for mov and mp4 output (default). Setting value to ‘<samp class="samp">pts</samp>’ is applicable only for a live encoding use case, where PTS values are set as as wallclock time at the source. For example, an encoding use case with decklink capture source where <samp class="option">video_pts</samp> and <samp class="option">audio_pts</samp> are set to ‘<samp class="samp">abs_wallclock</samp>’. </dd> </dl>

 <h4 class="subsection">21.4.4 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Push Smooth Streaming content in real time to a publishing point on IIS with the ‘<samp class="samp">ismv</samp>’ muxer using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -re <var class="var"><normal input/transcoding options></var> -movflags isml+frag_keyframe -f ismv http://server/publishingpoint.isml/Streams(Encoder1) </pre>

 <h3 class="section">21.5 a64<a class="anchor hidden-xs" href="#a64"

aria-hidden=”true”># TOC</h3> A64 Commodore 64 video muxer. This muxer accepts a single <code class=”code”>a64_multi</code> or <code class=”code”>a64_multi5</code> codec video stream. <h3 class=”section”>21.6 ac4# TOC</h3> Raw AC-4 audio muxer. This muxer accepts a single <code class=”code”>ac4</code> audio stream. <h4 class=”subsection”>21.6.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>write_crc <var class=”var”>bool</var></samp></dt> <dd> when enabled, write a CRC checksum for each packet to the output, default is <code class=”code”>false</code> </dd> </dl>

 <h3 class=”section”>21.7 adts# TOC</h3> Audio Data Transport Stream muxer. It accepts a single AAC stream. <h4 class=”subsection”>21.7.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>write_id3v2 <var class=”var”>bool</var></samp></dt> <dd> Enable to write ID3v2.4 tags at the start of the stream. Default is disabled. </dd> <dt><samp class=”option”>write_apetag <var class=”var”>bool</var></samp></dt> <dd> Enable to write APE tags at the end of the stream. Default is disabled. </dd> <dt><samp class=”option”>write_mpeg2 <var class=”var”>bool</var></samp></dt> <dd> Enable to set MPEG version bit in the ADTS frame header to 1 which indicates MPEG-2. Default is 0, which indicates MPEG-4. </dd> </dl>

 <h3 class=”section”>21.8 aea# TOC</h3> MD STUDIO audio muxer. This muxer accepts a single ATRAC1 audio stream with either one or two channels and a sample rate of 44100Hz. As AEA supports storing the track title, this muxer will also write the title from stream’s metadata to the container. <h3 class=”section”>21.9 aiff# <a class=”anchor

hidden-xs”href=”#toc-aiff” aria-hidden=”true”>TOC</h3>
Audio Interchange File Format muxer. <h4 class=”subsection”>21.9.1 Options#
TOC</h4>
<dl class=”table”> <dt><samp class=”option”>write_id3v2 <var class=”var”>bool</var></samp></dt>
<dd> Enable ID3v2 tags writing when set to 1. Default is 0 (disabled). </dd> <dt><samp class=”option”>id3v2_version <var class=”var”>bool</var></samp></dt> <dd> Select ID3v2 version to write. Currently only version 3 and 4 (aka. ID3v2.3 and ID3v2.4) are supported. The default is version 4. </dd> </dl>

 <h3 class=”section”>21.10 alp# TOC</h3> High Voltage Software’s Lego Racers game audio muxer. It accepts a single ADPCM_IMA_ALP stream with no more than 2 channels and a sample rate not greater than 44100 Hz. Extensions: <code class=”code”>tun</code>, <code class=”code”>pcm</code> <h4 class=”subsection”>21.10.1 Options# TOC</h4> <dl class=”table”>
<dt><samp class=”option”>type <var class=”var”>type</var></samp></dt>
<dd> Set file type. <var class=”var”>type</var> accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tun</samp>’</dt> <dd> Set file type as music. Must have a sample rate of 22050 Hz. </dd> <dt>‘<samp class=”samp”>pcm</samp>’</dt> <dd> Set file type as sfx. </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> Set file type as per output file extension. <code class=”code”>.pcm</code> results in type <code class=”code”>pcm</code> else type <code class=”code”>tun</code> is set. <var class=”var”>(default)</var>
</dd> </dl> </dd> </dl>

 <h3 class=”section”>21.11 amr#
TOC</h3>
3GPP AMR (Adaptive Multi-Rate) audio muxer. It accepts a single audio stream containing an AMR NB stream. <h3 class=”section”>21.12 amv#
TOC</h3>
AMV (Actions Media Video) format muxer.
<h3 class=”section”>21.13 apm# <a class=”anchor

hidden-xs”href=”#toc-apm” aria-hidden=”true”>TOC</h3>
Ubisoft Rayman 2 APM audio muxer. It accepts a single
ADPCM IMA APM audio stream. <h3
class=”section”>21.14 apng<a class=”anchor
hidden-xs” href=”#apng-1” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-apng-1” aria-hidden=”true”>TOC</h3>
Animated Portable Network Graphics muxer. It accepts
a single APNG video stream.
<h4 class=”subsection”>21.14.1 Options<a
class=”anchor hidden-xs” href=”#Options-71” aria-hidden=”true”>#
TOC</h4>
<dl class=”table”> <dt><samp class=”option”>final_delay <var class=”var”>delay</var></samp></dt>
<dd> Force a delay expressed in seconds after the last frame of each rep-
etition. Default value is <code class=”code”>0.0</code>. </dd>
<dt><samp class=”option”>plays <var class=”var”>repetitions</var></samp></dt>
<dd> specify how many times to play the content, <code class=”code”>0</code>
causes an infinite loop, with <code class=”code”>1</code> there is no loop
</dd> </dl>

 <h4 class=”subsection”>21.14.2 Exam-
ples<a class=”anchor hidden-xs” href=”#Examples-
15” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Examples-15” aria-hidden=”true”>TOC</h4> <ul class=”itemize
mark-bullet”> Use <code class=”command”>ffmpeg</code> to generate
an APNG output with 2 repetitions, and with a delay of half a second
after the first repetition: <pre class=”example-
preformatted”>ffmpeg -i INPUT -final_delay 0.5 -plays 2 out.apng
</pre>

 <h3 class=”section”>21.15 argo_asf<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#argo_005fasf”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
argo_005fasf” aria-hidden=”true”>TOC</h3> Argonaut
Games ASF audio muxer. It accepts a single ADPCM audio stream.
  <h4 class=”subsection”>21.15.1 Op-
tions<a class=”anchor hidden-xs” href=”#Options-
72” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Options-72” aria-hidden=”true”>TOC</h4> <dl class=”table”>
<dt><samp class=”option”>version_major <var class=”var”>version</var></samp></dt>
<dd> override file major version, specified as an integer, default
value is <code class=”code”>2</code> </dd> <dt><samp
class=”option”>version_minor <var class=”var”>version</var></samp></dt>
<dd> override file minor version, specified as an integer, default
value is <code class=”code”>1</code> </dd> <dt><samp
class=”option”>name <var class=”var”>name</var></samp></dt>
<dd> Embed file name into file, if not specified use the output file name.
The name is truncated to 8 characters. </dd> </dl>

 <h3 class="section">21.16 argo_cvg# TOC</h3> Argonaut Games CVG audio muxer. It accepts a single one-channel ADPCM 22050Hz audio stream. The <samp class="option">loop</samp> and <samp class="option">reverb</samp> options set the corresponding flags in the header which can be later retrieved to process the audio stream accordingly. <h4 class="subsection">21.16.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">skip_rate_check <var class="var">bool</var></samp></dt> <dd> skip sample rate check (default is <code class="code">false</code>) </dd> <dt><samp class="option">loop <var class="var">bool</var></samp></dt> <dd> set loop flag (default is <code class="code">false</code>) </dd> <dt><samp class="option">reverb <var class="var">boolean</var></samp></dt> <dd> set reverb flag (default is <code class="code">true</code>) </dd> </dl>

 <h3 class="section">21.17 asf, asf_stream# TOC</h3> Advanced / Active Systems (or Streaming) Format audio muxer. The ‘<samp class="samp">asf_stream</samp>’ variant should be selected for streaming. Note that Windows Media Audio (wma) and Windows Media Video (wmv) use this muxer too. <h4 class="subsection">21.17.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">packet_size <var class="var">size</var></samp></dt> <dd> Set the muxer packet size as a number of bytes. By tuning this setting you may reduce data fragmentation or muxer overhead depending on your source. Default value is <code class="code">3200</code>, minimum is <code class="code">100</code>, maximum is <code class="code">64Ki</code>. </dd> </dl>

 <h3 class="section">21.18 ass# TOC</h3> ASS/SSA (SubStation Alpha) subtitles muxer. It accepts a single ASS subtitles stream. <h4 class="subsection">21.18.1 Options#

TOC</h4> <dl class=”table”> <dt><samp class=”option”>ignore_readorder <var class=”var”>bool</var></samp></dt> <dd> Write dialogue events immediately, even if they are out-of-order, default is <code class=”code”>false</code>, otherwise they are cached until the expected time event is found. </dd> </dl>

 <h3 class=”section”>21.19 ast# TOC</h3> AST (Audio Stream) muxer. This format is used to play audio on some Nintendo Wii games. It accepts a single audio stream. The <samp class=”option”>loopstart</samp> and <samp class=”option”>loopend</samp> options can be used to define a section of the file to loop for players honoring such options. <h4 class=”subsection”>21.19.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>loopstart <var class=”var”>start</var></samp></dt> <dd> Specify loop start position expressesd in milliseconds, from <code class=”code”>-1</code> to <code class=”code”>INT_MAX</code>, in case <code class=”code”>-1</code> is set then no loop is specified (default -1) and the <samp class=”option”>loopend</samp> value is ignored. </dd> <dt><samp class=”option”>loopend <var class=”var”>end</var></samp></dt> <dd> Specify loop end position expressed in milliseconds, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>, default is <code class=”code”>0</code>, in case <code class=”code”>0</code> is set it assumes the total stream duration. </dd> </dl>

 <h3 class=”section”>21.20 astc# TOC</h3> ASTC (Adaptive Scalable Texture Compression) muxer. This muxer accepts a single ‘<samp class=”samp”>libastcenc</samp>’ video stream and writes one image: the 16-byte <samp class=”file”>.astc</samp> header, taken from the encoder’s extradata, followed by the raw ASTC bitstream. The block footprint and the image dimensions come from the same extradata, and a second packet is rejected. Use the <samp class=”file”>.ktx</samp> container for a texture that also needs its color space or row order recorded. <h3 class=”section”>21.21 au# TOC</h3> SUN AU audio muxer. It accepts a single audio stream. <h3

class=”section”>21.22 avi# TOC</h3> Audio Video Interleaved muxer. AVI is a proprietary format developed by Microsoft, and later formally specified through the Open DML specification. Because of differences in players implementations, it might be required to set some options to make sure that the generated output can be correctly played by the target player. <h4 class=”subsection”>21.22.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>flipped_raw_rgb <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>true</code>, store positive height for raw RGB bitmaps, which indicates bitmap is stored bottom-up. Note that this option does not flip the bitmap which has to be done manually beforehand, e.g. by using the ‘<samp class=”samp”>vflip</samp>’ filter. Default is <code class=”code”>false</code> and indicates bitmap is stored top down. </dd> <dt><samp class=”option”>reserve_index_space <var class=”var”>size</var></samp></dt> <dd> Reserve the specified amount of bytes for the OpenDML master index of each stream within the file header. By default additional master indexes are embedded within the data packets if there is no space left in the first master index and are linked together as a chain of indexes. This index structure can cause problems for some use cases, e.g. third-party software strictly relying on the OpenDML index specification or when file seeking is slow. Reserving enough index space in the file header avoids these problems. The required index space depends on the output file size and should be about 16 bytes per gigabyte. When this option is omitted or set to zero the necessary index space is guessed. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>write_channel_mask <var class=”var”>bool</var></samp></dt> <dd> Write the channel layout mask into the audio stream header. This option is enabled by default. Disabling the channel mask can be useful in specific scenarios, e.g. when merging multiple audio streams into one for compatibility with software that only supports a single audio stream in AVI (see <a data-manual=”ffmpeg-filters” href=”./ffmpeg-filters.html#amerge”>the "amerge" section in the ffmpeg-filters manual). </dd> </dl>

 <h3 class=”section”>21.23 avif# TOC</h3> AV1 (Alliance for Open Media Video codec 1) image format muxer. This muxers stores images encoded using the AV1 codec. It accepts one or two video streams. In case two video streams are provided, the second one shall contain a single plane storing the al-

pha mask. In case more than one image is provided, the generated output is considered an animated AVIF and the number of loops can be specified with the <samp class="option">loop</samp> option. This is based on the specification by Alliance for Open Media at url https://aomediacodec.github.io/av1-avif. <h4 class="subsection">21.23.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">loop <var class="var">count</var></samp></dt> <dd> number of times to loop an animated AVIF, <code class="code">0</code> specify an infinite loop, default is <code class="code">0</code> </dd> <dt><samp class="option">movie_timescale <var class="var">timescale</var></samp></dt> <dd> Set the timescale written in the movie header box (<code class="code">mvhd</code>). Range is 1 to INT_MAX. Default is <code class="code">1000</code>. </dd> </dl>

 <h3 class="section">21.24 avm2# TOC</h3> ShockWave Flash (SWF) / ActionScript Virtual Machine 2 (AVM2) format muxer. It accepts one audio stream, one video stream, or both. <h3 class="section">21.25 bit# TOC</h3> G.729 (.bit) file format muxer. It accepts a single G.729 audio stream. <h3 class="section">21.26 caf# TOC</h3> Apple CAF (Core Audio Format) muxer. It accepts a single audio stream. <h3 class="section">21.27 codec2# TOC</h3> Codec2 audio audio muxer. It accepts a single codec2 audio stream. <h3 class="section">21.28 chromaprint# TOC</h3> Chromaprint fingerprinter muxers. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-chromaprint</code>. This muxer feeds audio data to the Chromaprint library, which generates a fingerprint for the provided audio data. See: https://acoustid.org/chromaprint It takes a single signed native-endian 16-bit raw audio stream

of at most 2 channels. <h4 class="subsection">21.28.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">algorithm <var class="var">version</var></samp></dt> <dd> Select version of algorithm to fingerprint with. Range is <code class="code">0</code> to <code class="code">4</code>. Version <code class="code">3</code> enables silence detection. Default is <code class="code">1</code>. </dd> <dt><samp class="option">fp_format <var class="var">format</var></samp></dt> <dd> Format to output the fingerprint as. Accepts the following options: <dl class="table"> <dt>‘<samp class="samp">base64</samp>’</dt> <dd> Base64 compressed fingerprint (default) </dd> <dt>‘<samp class="samp">compressed</samp>’</dt> <dd> Binary compressed fingerprint </dd> <dt>‘<samp class="samp">raw</samp>’</dt> <dd> Binary raw fingerprint </dd> </dl>

</dd> <dt><samp class="option">silence_threshold <var class="var">threshold</var></samp></dt> <dd> Threshold for detecting silence. Range is from <code class="code">-1</code> to <code class="code">32767</code>, where <code class="code">-1</code> disables silence detection. Silence detection can only be used with version <code class="code">3</code> of the algorithm. Silence detection must be disabled for use with the AcoustID service. Default is <code class="code">-1</code>. </dd> </dl>

 <h3 class="section">21.29 crc# TOC</h3> CRC (Cyclic Redundancy Check) muxer. This muxer computes and prints the Adler-32 CRC of all the input audio and video frames. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the CRC. The output of the muxer consists of a single line of the form: CRC=0x<var class="var">CRC</var>, where <var class="var">CRC</var> is a hexadecimal number 0-padded to 8 digits containing the CRC for all the decoded input frames. See also the framecrc muxer. <h4 class="subsection">21.29.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to compute the CRC of the input, and store it in the file <samp class="file">out.crc</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f crc out.crc </pre>

Use <code class=”command”>ffmpeg</code> to print the CRC to stdout with the command: <pre class=”example-preformatted”>ffmpeg -i INPUT -f crc - </pre>

You can select the output format of each frame with <code class=”command”>ffmpeg</code> by specifying the audio and video codec and format. For example, to compute the CRC of the input audio converted to PCM unsigned 8-bit and the input video converted to MPEG-2 video, use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:a pcm_u8 -c:v mpeg2video -f crc - </pre>

 <h3 class=”section”>21.30 dash# TOC</h3> Dynamic Adaptive Streaming over HTTP (DASH) muxer. This muxer creates segments and manifest files according to the MPEG-DASH standard ISO/IEC 23009-1:2014 and following standard updates. For more information see: <ul class=”itemize mark-bullet”> ISO DASH Specification: <a class=”url” href=”http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_I
1_2014.zip”>http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_IEC_23009-1_2014.zip WebM DASH Specification: https://sites.google.com/a/webmproject.org/wiki/adaptive-streaming/webm-dash-specification

This muxer creates an MPD (Media Presentation Description) manifest file and segment files for each stream. Segment files are placed in the same directory of the MPD manifest file. The segment filename might contain pre-defined identifiers used in the manifest <code class=”code”>SegmentTemplate</code> section as defined in section 5.3.9.4.4 of the standard. Available identifiers are <code class=”code”>$RepresentationID$</code>, <code class=”code”>$Number$</code>, <code class=”code”>$Bandwidth$</code>, and <code class=”code”>$Time$</code>. In addition to the standard identifiers, an ffmpeg-specific <code class=”code”>ext</code> identifier is also supported. When specified, <code class=”command”>ffmpeg</code> will replace <code class=”code”>ext</code> in the file name with muxing format’s extensions such as <code class=”code”>mp4</code>, <code class=”code”>webm</code> etc. <h4 class=”subsection”>21.30.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>adaptation_sets <var class=”var”>adaptation_sets</var></samp></dt> <dd> Assign streams to adaptation sets, specified in the MPD manifest <code class=”code”>AdaptationSets</code> section. An adaptation set contains a set of one or more

streams accessed as a single subset, e.g. corresponding streams encoded at different size selectable by the user depending on the available bandwidth, or to different audio streams with a different language. Each adaptation set is specified with the syntax: <pre class="example-preformatted">id=<var class="var">index</var>,streams=<var class="var">streams</var> </pre>

where <var class="var">index</var> must be a numerical index, and <var class="var">streams</var> is a sequence of <code class="code">,</code>-separated stream indices. Multiple adaptation sets can be specified, separated by spaces. To map all video (or audio) streams to an adaptation set, <code class="code">v</code> (or <code class="code">a</code>) can be used as stream identifier instead of IDs. When no assignment is defined, this defaults to an adaptation set for each stream. The following optional fields can also be specified: <dl class="table"> <dt><samp class="option">descriptor</samp></dt> <dd> Define the descriptor as defined by ISO/IEC 23009-1:2014/Amd.2:2015. For example: <pre class="example-preformatted"><SupplementalProperty schemeIdUri=\"urn:mpeg:dash:srd:2014\" value=\"0,0,0,1,1,2,2\"/> </pre>

The descriptor string should be a self-closing XML tag. </dd> <dt><samp class="option">frag_duration</samp></dt> <dd> Override the global fragment duration specified with the <samp class="option">frag_duration</samp> option. </dd> <dt><samp class="option">frag_type</samp></dt> <dd> Override the global fragment type specified with the <samp class="option">frag_type</samp> option. </dd> <dt><samp class="option">seg_duration</samp></dt> <dd> Override the global segment duration specified with the <samp class="option">seg_duration</samp> option. </dd> <dt><samp class="option">trick_id</samp></dt> <dd> Mark an adaptation set as containing streams meant to be used for Trick Mode for the referenced adaptation set. </dd> </dl>

A few examples of possible values for the <samp class="option">adaptation_sets</samp> option follow: <pre class="example-preformatted">id=0,seg_duration=2,frag_
id=1,seg_duration=2,frag_type=none,streams=a </pre>

 <pre class="example-preformatted">id=0,seg_duration=2,frag_type=none,streams=
id=1,seg_duration=10,frag_type=none,trick_id=0,streams=1 </pre>

</dd> <dt><samp class="option">dash_segment_type <var class="var">type</var></samp></dt> <dd> Set DASH segment files type. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The dash segment files format will be selected based on the stream codec. This is the default mode. </dd> <dt>‘<samp class="samp">mp4</samp>’</dt> <dd> the dash segment

files will be in ISOBMFF/MP4 format </dd> <dt>‘<samp class="samp">webm</samp>’</dt> <dd> the dash segment files will be in WebM format </dd> </dl>

</dd> <dt><samp class="option">extra_window_size <var class="var">size</var></samp></dt> <dd> Set the maximum number of segments kept outside of the manifest before removing from disk. </dd> <dt><samp class="option">format_options <var class="var">options_list</var></samp></dt> <dd> Set container format (mp4/webm) options using a <code class="code">:</code>-separated list of key=value parameters. Values containing <code class="code">:</code> special characters must be escaped. </dd> <dt><samp class="option">frag_duration <var class="var">duration</var></samp></dt> <dd> Set the length in seconds of fragments within segments, fractional value can also be set. </dd> <dt><samp class="option">frag_type <var class="var">type</var></samp></dt> <dd> Set the type of interval for fragmentation. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> set one fragment per segment </dd> <dt>‘<samp class="samp">every_frame</samp>’</
<dd> fragment at every frame </dd> <dt>‘<samp class="samp">duration</samp>’</dt> <dd> fragment at specific time intervals </dd> <dt>‘<samp class="samp">pframes</samp>’</dt> <dd> fragment at keyframes and following P-Frame reordering (Video only, experimental) </dd> </dl>

</dd>

<dt><samp class="option">global_sidx <var class="var">bool</var></samp></dt>

<dd> Write global <code class="code">SIDX</code> atom. Applicable only for single file, mp4

output, non-streaming mode.

</dd>

<dt><samp class="option">hls_master_name <var class="var">file_name</var></samp></dt>

<dd> HLS master playlist name. Default is <samp class="file">master.m3u8</samp>.

</dd>

<dt><samp class="option">hls_playlist <var class="var">bool</var></samp></dt>

<dd> Generate HLS playlist files. The master playlist is generated with

filename specified by the <samp class="option">hls_master_name</samp> option. One media

playlist file is generated for each stream with filenames

<samp class="file">media_0.m3u8</samp>, <samp class="file">media_1.m3u8</samp>, etc.

</dd>

<dt><samp class="option">http_opts <var class="var">http_opts</var></samp></dt>

<dd> Specify a list of <code class="code">:</code>-separated key=value options to pass to the

underlying HTTP protocol. Applicable only for HTTP output.

</dd>

<dt><samp class="option">http_persistent <var class="var">bool</var></samp></dt>

<dd> Use persistent HTTP connections. Applicable only for HTTP output.

</dd>

<dt><samp class="option">http_user_agent <var class="var">user_agent</var></samp></dt>

<dd> Override User-Agent field in HTTP header. Applicable only for HTTP

output.

</dd>

<dt><samp class="option">ignore_io_errors <var class="var">bool</var></samp></dt>

<dd> Ignore IO errors during open and write. Useful for long-duration runs

with network output. This is disabled by default.

</dd>

<dt><samp class="option">index_correction <var class="var">bool</var></samp></dt>

<dd> Enable or disable segment index correction logic. Applicable only when

<samp class="option">use_template</samp> is enabled and <samp class="option">use_timeline</samp> is

disabled. This is disabled by default.

When enabled, the logic monitors the flow of segment indexes. If a

streams’s segment index value is not at the expected real time

position, then the logic corrects that index value.

Typically this logic is needed in live streaming use cases. The

network bandwidth fluctuations are common during long run

streaming. Each fluctuation can cause the segment indexes fall behind

the expected real time position.

</dd>

<dt><samp class="option">init_seg_name <var class="var">init_name</var></samp></dt>

<dd> DASH-templated name to use for the initialization segment. Default
is

<code class="code">init-stream$RepresentationID$.ext</code>. <code
class="code">ext</code> is replaced

with the file name extension specific for the segment format.

</dd>

<dt><samp class="option">ldash <var class="var">bool</var></samp></dt>

<dd> Enable Low-latency Dash by constraining the presence and values of

some elements. This is disabled by default.

</dd>

<dt><samp class="option">lhls <var class="var">bool</var></samp></dt>

<dd> Enable Low-latency HLS (LHLS). Add <code class="code">#EXT-
X-PREFETCH</code> tag with

current segment’s URI. hls.js player folks are trying to standardize

an open LHLS spec. The draft spec is available at

<a class="url" href="https://github.com/video-dev/hlsjs-rfcs/blob/lhls-
spec/proposals/0001-lhls.md">https://github.com/video-dev/hlsjs-rfcs/blob/lhls-
spec/proposals/0001-lhls.md.

This option tries to comply with the above open spec. It enables

<samp class="option">streaming</samp> and <samp class="option">hls_playlist</samp> options automatically.

This is an experimental feature.

Note: This is not Apple’s version LHLS. See

https://datatracker.ietf.org/doc/html/draft-pantos-hls-rfc8216bis

</dd>

<dt><samp class="option">master_m3u8_publish_rate <var class="var">segment_intervals_count</var>

<dd> Publish master playlist repeatedly every after specified number of

segment intervals.

</dd>

<dt><samp class="option">max_playback_rate <var class="var">rate</var></samp></dt>

<dd> Set the maximum playback rate indicated as appropriate for the

purposes of automatically adjusting playback latency and buffer

occupancy during normal playback by clients.

</dd>

<dt><samp class="option">media_seg_name <var class="var">segment_name</var></samp></dt>

<dd> DASH-templated name to use for the media segments. Default is

<code class="code">chunk-stream$RepresentationID$-$Number%05d$.ext</code>. <code class="code">ext</code>

is replaced with the file name extension specific for the segment

format.

</dd>

<dt><samp class="option">method <var class="var">method</var></samp></dt>

<dd> Use the given HTTP method to create output files. Generally set to
<code class="code">PUT</code>

or <code class="code">POST</code>.

</dd>

<dt><samp class="option">min_playback_rate <var class="var">rate</var></samp></dt>

<dd> Set the minimum playback rate indicated as appropriate for the

purposes of automatically adjusting playback latency and buffer

occupancy during normal playback by clients.

</dd>

<dt><samp class="option">mpd_profile <var class="var">flags</var></samp></dt>

<dd> Set one or more MPD manifest profiles.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">dash</samp>’</dt>

<dd> MPEG-DASH ISO Base media file format live profile

</dd>

<dt>‘<samp class="samp">dvb_dash</samp>’</dt>

<dd> DVB-DASH profile

</dd>

</dl>

Default value is <code class="code">dash</code>.

</dd>

<dt><samp class="option">remove_at_exit <var class="var">bool</var></samp></dt>

<dd> Enable or disable removal of all segments when finished. This is

disabled by default.

</dd>

<dt><samp class="option">seg_duration <var class="var">duration</var></samp></dt>

<dd> Set the segment length in seconds (fractional value can be set). The

value is treated as average segment duration when the

<samp class="option">use_template</samp> option is enabled and the <samp class="option">use_timeline</samp>

option is disabled and as minimum segment duration for all the other

use cases.

Default value is <code class="code">5</code>.

</dd>

<dt><samp class="option">single_file <var class="var">bool</var></samp></dt>

<dd> Enable or disable storing all segments in one file, accessed using

byte ranges. This is disabled by default.

The name of the single file can be specified with the

<samp class="option">single_file_name</samp> option, if not specified assume the basename

of the manifest file with the output format extension.

</dd>

<dt><samp class="option">single_file_name <var class="var">file_name</var></samp></dt>

<dd> DASH-templated name to use for the manifest <code class="code">baseURL</code>

element. Imply that the <samp class="option">single_file</samp> option is set to

<var class="var">true</var>. In the template, <code class="code">ext</code> is replaced with the file

name extension specific for the segment format.

</dd>

<dt><samp class="option">availability_start_time_ms <var class="var">milliseconds</var></samp></

<dd> Set the MPD <code class="code">availabilityStartTime</code> attribute as Unix epoch time in

milliseconds. If set to <code class="code">0</code>, the wall clock is used when the first

packet is written. This is useful for live outputs that need a stable

availability start time across restarts.

</dd>

<dt><samp class="option">streaming <var class="var">bool</var></samp></dt>

<dd> Enable or disable chunk streaming mode of output. In chunk streaming

mode, each frame will be a <code class="code">moof</code> fragment which forms a

chunk. This is disabled by default.

</dd>

<dt><samp class="option">suggested_presentation_delay <var class="var">duration</var></samp></d

<dd> Set the MPD <code class="code">suggestedPresentationDelay</code> attribute. If set to

<code class="code">0</code>, the last segment duration is used.

</dd>

<dt><samp class="option">target_latency <var class="var">target_latency</var></samp></dt>

<dd> Set an intended target latency in seconds for serving (fractional

value can be set). Applicable only when the <samp class="option">streaming</samp> and

<samp class="option">write_prft</samp> options are enabled. This is an informative fields

clients can use to measure the latency of the service.

</dd>

<dt><samp class="option">timeout <var class="var">timeout</var></samp></dt>

<dd> Set timeout for socket I/O operations expressed in seconds (fractional

value can be set). Applicable only for HTTP output.

</dd>

<dt><samp class="option">update_period <var class="var">period</var></samp></dt>

<dd> Set the MPD update period, for dynamic content. The unit is

second. If set to <code class="code">0</code>, the period is automatically computed.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">use_template <var class="var">bool</var></samp></dt>

<dd> Enable or disable use of <code class="code">SegmentTemplate</code> instead of

<code class="code">SegmentList</code> in the manifest. This is enabled by default.

</dd>

<dt><samp class="option">use_timeline <var class="var">bool</var></samp></dt>

<dd> Enable or disable use of <code class="code">SegmentTimeline</code> within the

<code class="code">SegmentTemplate</code> manifest section. This is enabled by default.

</dd>

<dt><samp class="option">utc_timing_url <var class="var">url</var></samp></dt>

<dd> URL of the page that will return the UTC timestamp in ISO

format, for example <code class="code">https://time.akamai.com/?iso</code>

</dd>

<dt><samp class="option">window_size <var class="var">size</var></samp></dt>

<dd> Set the maximum number of segments kept in the manifest, discard the

oldest one. This is useful for live streaming.

If the value is <code class=”code”>0</code>, all segments are kept in the

manifest. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>write_prft <var class=”var”>write_prft</var></samp></dt>

<dd> Write Producer Reference Time elements on supported streams. This also

enables writing prft boxes in the underlying muxer. Applicable only

when the <var class=”var”>utc_url</var> option is enabled. It is set to <var class=”var”>auto</var> by

default, in which case the muxer will attempt to enable it only in

modes that require it.

</dd>

</dl>

 <h4 class=”subsection”>21.30.2 Example# TOC</h4> Generate a DASH output reading from an input source in realtime using <code class=”command”>ffmpeg</code>. Two multimedia streams are generated from the input file, both containing a video stream encoded through ‘<samp class=”samp”>libx264</samp>’, and an audio stream encoded with ‘<samp class=”samp”>libfdk_aac</samp>’. The first multimedia stream contains video with a bitrate of 800k and audio at the default rate, the second with video scaled to 320x170 pixels at 300k and audio resampled at 22005 Hz. The <samp class=”option”>window_size</samp> option keeps only the latest 5 segments with the default duration of 5 seconds.   <pre class=”example-preformatted”>ffmpeg -re -i <input> -map 0 -map 0 -c:a libfdk_aac -c:v libx264 \ -b:v:0 800k -profile:v:0 main \ -b:v:1 300k -s:v:1 320x170 -profile:v:1 baseline -ar:a:1 22050 \ -bf 1 -keyint_min 120 -g 120 -sc_threshold 0 -b_strategy 0 \ -use_timeline 1 -use_template 1 -window_size 5 \ -adaptation_sets "id=0,streams=v id=1,streams=a" \ -f dash /path/to/out.mpd </pre>

 <h3 class=”section”>21.31 daud#

TOC</h3> D-Cinema audio muxer. It accepts a single 6-channels audio stream resampled at 96000 Hz encoded with the ‘<samp class=”samp”>pcm_24daud</samp>’ codec. <h4 class=”subsection”>21.31.1 Example# TOC</h4> Use <code class=”command”>ffmpeg</code> to mux input audio to a ‘<samp class=”samp”>5.1</samp>’ channel layout resampled at 96000Hz: <pre class=”example-preformatted”>ffmpeg -i INPUT -af aresample=96000,pan=5.1 slow.302 </pre>

For ffmpeg versions before 7.0 you might have to use the ‘<samp class=”samp”>asetnsamples</samp>’ filter to limit the muxed packet size, because this format does not support muxing packets larger than 65535 bytes (3640 samples). For newer ffmpeg versions audio is automatically packetized to 36000 byte (2000 sample) packets. <h3 class=”section”>21.32 dv# TOC</h3> DV (Digital Video) muxer. It accepts exactly one ‘<samp class=”samp”>dvvideo</samp>’ video stream and at most two ‘<samp class=”samp”>pcm_s16</samp>’ audio streams. More constraints are defined by the property of the video, which must correspond to a DV video supported profile, and on the framerate. <h4 class=”subsection”>21.32.1 Example# TOC</h4> Use <code class=”command”>ffmpeg</code> to convert the input: <pre class=”example-preformatted”>ffmpeg -i INPUT -s:v 720x480 -pix_fmt yuv411p -r 29.97 -ac 2 -ar 48000 -y out.dv </pre>

 <h3 class=”section”>21.33 ffmetadata# TOC</h3> FFmpeg metadata muxer. This muxer writes the streams metadata in the ‘<samp class=”samp”>ffmetadata</samp>’ format. See <a data-manual=”ffmpeg-formats” href=”./ffmpeg-formats.html#metadata”>the Metadata chapter for information about the format. <h4 class=”subsection”>21.33.1 Example# TOC</h4>

Use <code class=”command”>ffmpeg</code> to extract metadata from an input file to a <samp class=”file”>metadata.ffmeta</samp> file in ‘<samp class=”samp”>ffmetadata</samp>’ format: <pre class=”example-preformatted”>ffmpeg -i INPUT -f ffmetadata metadata.ffmeta </pre>

 <h3 class=”section”>21.34 fifo# TOC</h3> FIFO (First-In First-Out) muxer. The ‘<samp class=”samp”>fifo</samp>’ pseudo-muxer allows the separation of encoding and muxing by using a first-in-first-out queue and running the actual muxer in a separate thread. This is especially useful in combination with the tee muxer and can be used to send data to several destinations with different reliability/writing speed/latency. The target muxer is either selected from the output name or specified through the <samp class=”option”>fifo_format</samp> option. The behavior of the ‘<samp class=”samp”>fifo</samp>’ muxer if the queue fills up or if the output fails (e.g. if a packet cannot be written to the output) is selectable: <ul class=”itemize mark-bullet”> Output can be transparently restarted with configurable delay between retries based on real time or time of the processed stream.

Encoding can be blocked during temporary failure, or continue transparently dropping packets in case the FIFO queue fills up.

API users should be aware that callback functions (<code class=”code”>interrupt_callback</code>, <code class=”code”>io_open</code> and <code class=”code”>io_close</code>) used within its <code class=”code”>AVFormatContext</code> must be thread-safe. <h4 class=”subsection”>21.34.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>attempt_recovery <var class=”var”>bool</var></samp></dt> <dd> If failure occurs, attempt to recover the output. This is especially useful when used with network output, since it makes it possible to restart streaming transparently. By default this option is set to <code class=”code”>false</code>. </dd> <dt><samp class=”option”>drop_pkts_on_overflow <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>true</code>, in case the fifo queue fills up, packets will be dropped rather than blocking the encoder. This makes it possible to continue streaming without delaying the input, at the cost of omitting part of the stream. By default this option is set to <code class=”code”>false</code>, so in such cases the encoder will be blocked until the muxer processes some of the packets and none of them is lost. </dd> <dt><samp class=”option”>fifo_format

<var class=”var”>format_name</var></samp></dt> <dd> Specify the format name. Useful if it cannot be guessed from the output name suffix. </dd> <dt><samp class=”option”>format_opts <var class=”var”>options</var></samp></dt> <dd> Specify format options for the underlying muxer. Muxer options can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. </dd> <dt><samp class=”option”>max_recovery_attempts <var class=”var”>count</var></samp></dt> <dd> Set maximum number of successive unsuccessful recovery attempts after which the output fails permanently. By default this option is set to <code class=”code”>0</code> (unlimited). </dd> <dt><samp class=”option”>queue_size <var class=”var”>size</var></samp></dt> <dd> Specify size of the queue as a number of packets. Default value is <code class=”code”>60</code>. </dd> <dt><samp class=”option”>recover_any_error <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>true</code>, recovery will be attempted regardless of type of the error causing the failure. By default this option is set to <code class=”code”>false</code> and in case of certain (usually permanent) errors the recovery is not attempted even when the <samp class=”option”>attempt_recovery</samp> option is set to <code class=”code”>true</code>. </dd> <dt><samp class=”option”>recovery_wait_streamtime <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>false</code>, the real time is used when waiting for the recovery attempt (i.e. the recovery will be attempted after the time specified by the <samp class=”option”>recovery_wait_time</samp> option). If set to <code class=”code”>true</code>, the time of the processed stream is taken into account instead (i.e. the recovery will be attempted after discarding the packets corresponding to the <samp class=”option”>recovery_wait_time</samp> option). By default this option is set to <code class=”code”>false</code>. </dd> <dt><samp class=”option”>recovery_wait_time <var class=”var”>duration</var></samp></dt> <dd> Specify waiting time in seconds before the next recovery attempt after previous unsuccessful recovery attempt. Default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>restart_with_keyframe <var class=”var”>bool</var></samp></dt> <dd> Specify whether to wait for the keyframe after recovering from queue overflow or failure. This option is set to <code class=”code”>false</code> by default. </dd> <dt><samp class=”option”>timeshift <var class=”var”>duration</var></samp></dt> <dd> Buffer the specified amount of packets and delay writing the output. Note that the value of the <samp class=”option”>queue_size</samp> option must be big enough to store the packets for timeshift. At the end of the input the fifo buffer is flushed at realtime speed. </dd> </dl>

 <h4 class=”subsection”>21.34.2 Example<a class=”anchor hidden-xs” href=”#Example-

4" aria-hidden="true"># TOC</h4>

Use <code class="command">ffmpeg</code> to stream to an RTMP server, continue processing the stream at real-time rate even in case of temporary failure (network outage) and attempt to recover streaming every second indefinitely: <pre class="example-preformatted">ffmpeg -re -i ... -c:v libx264 -c:a aac -f fifo -fifo_format flv \ -drop_pkts_on_overflow 1 -attempt_recovery 1 -recovery_wait_time 1 \ -map 0:v -map 0:a rtmp://example.com/live/stream_name </pre>

 <h3 class="section">21.35 film_cpk# TOC</h3> Sega film (.cpk) muxer. This format was used as internal format for several Sega games. For more information regarding the Sega film file format, visit <a class="url" href="http://wiki.multimedia.cx/index.php?title=Sega_FILM">http://wiki.multim
 It accepts at maximum one ‘<samp class="samp">cinepak</samp>’ or raw video stream, and at maximum one audio stream. <h3 class="section">21.36 filmstrip# TOC</h3> Adobe Filmstrip muxer. This format is used by several Adobe tools to store a generated filmstrip export. It accepts a single raw video stream. <h3 class="section">21.37 fits# TOC</h3> Flexible Image Transport System (FITS) muxer. This image format is used to store astronomical data. For more information regarding the format, visit https://fits.gsfc.nasa.gov. <h3 class="section">21.38 flac# TOC</h3> Raw FLAC audio muxer. This muxer accepts exactly one FLAC audio stream. Additionally, it is possible to add images with disposition ‘<samp class="samp">attached_pic</samp>’. <h4 class="subsection">21.38.1 Options# TOC</h4> <dl class="table">
<dt><samp class="option">write_header <var class="var">bool</var></samp></dt>
<dd> write the file header if set to <code class="code">true</code>, default is <code class="code">true</code> </dd> </dl>

 <h4 class="subsection">21.38.2 Example# TOC</h4> Use <code class="command">ffmpeg</code> to store the audio stream from an input file, together with several pictures used with ‘<samp class="samp">attached_pic</samp>’ disposition: <pre class="example-preformatted">ffmpeg -i INPUT -i pic1.png -i pic2.jpg -map 0:a -map 1 -map 2 -disposition:v attached_pic OUTPUT </pre>

 <h3 class="section">21.39 flv# TOC</h3> Adobe Flash Video Format muxer. <h4 class="subsection">21.39.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">flvflags <var class="var">flags</var></samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">aac_seq_header_detect</samp>’</dt> <dd> Place AAC sequence header based on audio stream data. </dd> <dt>‘<samp class="samp">no_sequence_end</samp>’</dt> <dd> Disable sequence end tag. </dd> <dt>‘<samp class="samp">no_metadata</samp>’</dt> <dd> Disable metadata tag. </dd> <dt>‘<samp class="samp">no_duration_filesize</samp>’</dt> <dd> Disable duration and filesize in metadata when they are equal to zero at the end of stream. (Be used to non-seekable living stream). </dd> <dt>‘<samp class="samp">add_keyframe_index</samp>’</dt> <dd> Used to facilitate seeking; particularly for HTTP pseudo streaming. </dd> </dl> </dd> </dl>

 <h3 class="section">21.40 framecrc# TOC</h3>

Per-packet CRC (Cyclic Redundancy Check) testing format. This muxer computes and prints the Adler-32 CRC for each audio and video packet. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the CRC. The output of the muxer consists of a line for each audio and video packet of the form: <pre class="example-preformatted"><var class="var">stream_index</var>, <var class="var">packet_dts</var>, <var class="var">packet_pts</var>, <var class="var">packet_duration</var>, <var class="var">packet_size</var>, 0x<var class="var">CRC</var> </pre>

<var class="var">CRC</var> is a hexadecimal number 0-padded to 8 digits containing the CRC of the packet. <h4 class="subsection">21.40.1 Examples# TOC</h4>

For example to compute the CRC of the audio and video frames in <samp class="file">INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class="file">out.crc</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f framecrc out.crc </pre>

To print the information to stdout, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f framecrc - </pre>

With <code class="command">ffmpeg</code>, you can select the output format to which the audio and video frames are encoded before computing the CRC for each packet by specifying the audio and video codec. For example, to compute the CRC of each decoded input audio frame converted to PCM unsigned 8-bit and of each decoded input video frame converted to MPEG-2 video, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -c:a pcm_u8 -c:v mpeg2video -f framecrc - </pre>

See also the crc muxer. <h3 class="section">21.41 framehash# TOC</h3>

Per-packet hash testing format. This muxer computes and prints a cryptographic hash for each audio and video packet. This can be used for packet-by-packet equality checks without having to individually do a binary comparison on each. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the hash, but the output of explicit conversions to other codecs can also be used. It uses the SHA-256 cryptographic hash function by default, but supports several other algorithms. The output of the muxer consists of a line for each audio and video packet of the form: <pre class="example-preformatted"><var class="var">stream_index</var>, <var class="var">packet_dts</var>, <var class="var">packet_pts</var>, <var class="var">packet_duration</var>, <var class="var">packet_size</var>, <var class="var">hash</var> </pre>

<var class="var">hash</var> is a hexadecimal number representing the computed hash for the packet. <dl class="table"> <dt><samp class="option">hash <var class="var">algorithm</var></samp></dt> <dd> Use the cryptographic hash function specified by the string

<var class="var">algorithm</var>. Supported values include <code class="code">MD5</code>, <code class="code">murmur3</code>, <code class="code">RIPEMD128</code>, <code class="code">RIPEMD160</code>, <code class="code">RIPEMD256</code>, <code class="code">RIPEMD320</code>, <code class="code">SHA160</code>, <code class="code">SHA224</code>, <code class="code">SHA256</code> (default), <code class="code">SHA512/224</code>, <code class="code">SHA512/256</code>, <code class="code">SHA384</code>, <code class="code">SHA512</code>, <code class="code">CRC32</code> and <code class="code">adler32</code>. </dd> </dl>

 <h4 class="subsection">21.41.1 Examples# TOC</h4>

To compute the SHA-256 hash of the audio and video frames in <samp class="file">INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class="file">out.sha256</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f framehash out.sha256 </pre>

To print the information to stdout, using the MD5 hash function, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f framehash -hash md5 - </pre>

See also the hash muxer. <h3 class="section">21.42 framemd5# TOC</h3>

Per-packet MD5 testing format. This is a variant of the framehash muxer. Unlike that muxer, it defaults to using the MD5 hash function. <h4 class="subsection">21.42.1 Examples# TOC</h4>

To compute the MD5 hash of the audio and video frames in <samp class="file">INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class="file">out.md5</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f framemd5 out.md5 </pre>

To print the information to stdout, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f framemd5 - </pre>

See also the framehash and <a

class=”ref” href=”#md5”>md5 muxers. <h3 class=”section”>21.43 gif# TOC</h3> Animated GIF muxer. Note that the GIF format has a very large time base: the delay between two frames can therefore not be smaller than one centisecond. <h4 class=”subsection”>21.43.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>loop <var class=”var”>bool</var></samp></dt> <dd> Set the number of times to loop the output. Use <code class=”code”>-1</code> for no loop, <code class=”code”>0</code> for looping indefinitely (default). </dd> <dt><samp class=”option”>final_delay <var class=”var”>delay</var></samp></dt> <dd> Force the delay (expressed in centiseconds) after the last frame. Each frame ends with a delay until the next frame. The default is <code class=”code”>-1</code>, which is a special value to tell the muxer to reuse the previous delay. In case of a loop, you might want to customize this value to mark a pause for instance. </dd> </dl>

 <h4 class=”subsection”>21.43.2 Example# TOC</h4> Encode a gif looping 10 times, with a 5 seconds delay between the loops: <pre class=”example-preformatted”>ffmpeg -i INPUT -loop 10 -final_delay 500 out.gif </pre>

Note 1: if you wish to extract the frames into separate GIF files, you need to force the image2 muxer: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v gif -f image2 "out%d.gif" </pre>

 <h3 class=”section”>21.44 gxf# TOC</h3> General eXchange Format (GXF) muxer. GXF was developed by Grass Valley Group, then standardized by SMPTE as SMPTE 360M and was extended in SMPTE RDD 14-2007 to include high-definition video resolutions. It accepts at most one video stream with codec ‘<samp class=”samp”>mjpeg</samp>’, or ‘<samp class=”samp”>mpeg1video</samp>’, or ‘<samp class=”samp”>mpeg2video</samp>’, or ‘<samp class=”samp”>dvvideo</samp>’ with resolution ‘<samp class=”samp”>512x480</samp>’ or ‘<samp class=”samp”>608x576</samp>’, and several audio

streams with rate 48000Hz and codec ‘<samp class="samp">pcm16_le</samp>’.
  <h3
class="section">21.45 hash<a class="anchor
hidden-xs" href="#hash" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-hash" aria-hidden="true">TOC</h3>

Hash testing format. This muxer computes and prints a
cryptographic hash of all the input audio and video frames. This can be used
for equality checks without having to do a complete binary comparison.
By default audio frames are converted to signed 16-bit raw audio and
video frames to raw video before computing the hash, but the output of explicit
conversions to other codecs can also be used. Timestamps are ignored. It uses
the SHA-256 cryptographic hash function by default, but supports several
other algorithms. The output of the muxer consists of a single line
of the form: <var class="var">algo</var>=<var class="var">hash</var>,
where <var class="var">algo</var> is a short string representing the hash
function used, and <var class="var">hash</var> is a hexadecimal number
representing the computed hash. <dl class="table"> <dt><samp
class="option">hash <var class="var">algorithm</var></samp></dt>
<dd> Use the cryptographic hash function specified by the string
<var class="var">algorithm</var>. Supported values include <code
class="code">MD5</code>, <code class="code">murmur3</code>, <code
class="code">RIPEMD128</code>, <code class="code">RIPEMD160</code>,
<code class="code">RIPEMD256</code>, <code class="code">RIPEMD320</code>,
<code class="code">SHA160</code>, <code class="code">SHA224</code>,
<code class="code">SHA256</code> (default), <code class="code">SHA512/224</code>,
<code class="code">SHA512/256</code>, <code class="code">SHA384</code>,
<code class="code">SHA512</code>, <code class="code">CRC32</code>
and <code class="code">adler32</code>. </dd> </dl>

 <h4 class="subsection">21.45.1 Exam-
ples<a class="anchor hidden-xs" href="#Examples-
20" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-20" aria-hidden="true">TOC</h4>

To compute the SHA-256 hash of the input converted to raw audio
and video, and store it in the file <samp class="file">out.sha256</samp>:
 <pre class="example-preformatted">ffmpeg -i
INPUT -f hash out.sha256 </pre>

To print an MD5 hash to stdout use the command: <div
class="example"> <pre class="example-preformatted">ffmpeg -i INPUT
-f hash -hash md5 - </pre>

See also the framehash muxer.
  <h3
class="section">21.46 hds<a class="anchor
hidden-xs" href="#hds" aria-hidden="true"># <a class="anchor

hidden-xs”href=”#toc-hds” aria-hidden=”true”>TOC</h3> HTTP Dynamic Streaming (HDS) muxer. HTTP dynamic streaming, or HDS, is an adaptive bitrate streaming method developed by Adobe. HDS delivers MP4 video content over HTTP connections. HDS can be used for on-demand streaming or live streaming. This muxer creates an .f4m (Adobe Flash Media Manifest File) manifest, an .abst (Adobe Bootstrap File) for each stream, and segment files in a directory specified as the output. These needs to be accessed by an HDS player through HTTPS for it to be able to perform playback on the generated stream. <h4 class=”subsection”>21.46.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>extra_window_size <var class=”var”>int</var></samp></dt> <dd> number of fragments kept outside of the manifest before removing from disk </dd> <dt><samp class=”option”>min_frag_duration <var class=”var”>microseconds</var></samp></dt> <dd> minimum fragment duration (in microseconds), default value is 1 second (<code class=”code”>10000000</code>) </dd> <dt><samp class=”option”>remove_at_exit <var class=”var”>bool</var></samp></dt> <dd> remove all fragments when finished when set to <code class=”code”>true</code> </dd> <dt><samp class=”option”>window_size <var class=”var”>int</var></samp></dt> <dd> number of fragments kept in the manifest, if set to a value different from <code class=”code”>0</code>. By default all segments are kept in the output directory. </dd> </dl>

 <h4 class=”subsection”>21.46.2 Example# TOC</h4> Use <code class=”command”>ffmpeg</code> to generate HDS files to the <samp class=”file”>output.hds</samp> directory in real-time rate: <pre class=”example-preformatted”>ffmpeg -re -i INPUT -f hds -b:v 200k output.hds </pre>

 <h3 class=”section”>21.47 hls# TOC</h3>

Apple HTTP Live Streaming muxer that segments MPEG-TS according to the HTTP Live Streaming (HLS) specification. It creates a playlist file, and one or more segment files. The output filename specifies the playlist filename. By default, the muxer creates a file for each segment produced. These files have the same name as the playlist, followed by a sequential number and a .ts extension. Make sure to require a closed GOP when encoding and to set the GOP size to fit your segment

time constraint. For example, to convert an input file with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.mkv -c:v h264 -flags +cgop -g 30 -hls_time 1 out.m3u8 </pre> This example will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">out0.ts</samp>, <samp class="file">out1.ts</samp>, <samp class="file">out2.ts</samp>, etc. See also the segment muxer, which provides a more generic and flexible implementation of a segmenter, and can be used to perform HLS segmentation. <h4 class="subsection">21.47.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">hls_init_time <var class="var">duration</var></samp></dt> <dd> Set the initial target segment length. Default value is <var class="var">0</var>. <var class="var">duration</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. Segment will be cut on the next key frame after this time has passed on the first m3u8 list. After the initial playlist is filled, <code class="command">ffmpeg</code> will cut segments at duration equal to <samp class="option">hls_time</samp>. </dd> <dt><samp class="option">hls_time <var class="var">duration</var></samp></dt> <dd> Set the target segment length. Default value is 2. <var class="var">duration</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. Segment will be cut on the next key frame after this time has passed. </dd> <dt><samp class="option">hls_list_size <var class="var">size</var></samp></dt> <dd> Set the maximum number of playlist entries. If set to 0 the list file will contain all the segments. Default value is 5. </dd> <dt><samp class="option">hls_delete_threshold <var class="var">size</var></samp></dt> <dd> Set the number of unreferenced segments to keep on disk before <code class="code">hls_flags delete_segments</code> deletes them. Increase this to allow continue clients to download segments which were recently referenced in the playlist. Default value is 1, meaning segments older than <samp class="option">hls_list_size+1</samp> will be deleted. </dd> <dt><samp class="option">hls_start_number_source <var class="var">source</var></samp></dt> <dd> Start the playlist sequence number (<code class="code">#EXT-X-MEDIA-SEQUENCE</code>) according to the specified source. Unless <samp class="option">hls_flags single_file</samp> is set, it also specifies source of starting sequence numbers of segment and subtitle filenames. In any case, if <samp class="option">hls_flags append_list</samp> is set and read playlist sequence number is greater than the specified start sequence number,

then that value will be used as start value. It accepts the following values: <dl class="table"> <dt><samp class="option">generic (default)</samp></dt> <dd> Set the start numbers according to the <samp class="option">start_number</samp> option value. </dd> <dt><samp class="option">epoch</samp></dt> <dd> Set the start number as the seconds since epoch (1970-01-01 00:00:00). </dd> <dt><samp class="option">epoch_us</samp></dt> <dd> Set the start number as the microseconds since epoch (1970-01-01 00:00:00). </dd> <dt><samp class="option">datetime</samp></dt> <dd> Set the start number based on the current date/time as YYYYmmddHHMMSS. e.g. 20161231235759. </dd> </dl>

</dd> <dt><samp class="option">start_number <var class="var">number</var></samp></dt> <dd> Start the playlist sequence number (<code class="code">#EXT-X-MEDIA-SEQUENCE</code>) from the specified <var class="var">number</var> when <samp class="option">hls_start_number_source</samp> value is <var class="var">generic</var>. (This is the default case.) Unless <samp class="option">hls_flags single_file</samp> is set, it also specifies starting sequence numbers of segment and subtitle filenames. Default value is 0. </dd> <dt><samp class="option">hls_allow_cache <var class="var">bool</var></samp></dt> <dd> Explicitly set whether the client MAY (1) or MUST NOT (0) cache media segments. </dd> <dt><samp class="option">hls_base_url <var class="var">baseurl</var></samp></dt> <dd> Append <var class="var">baseurl</var> to every entry in the playlist. Useful to generate playlists with absolute paths. Note that the playlist sequence number must be unique for each segment and it is not to be confused with the segment filename sequence number which can be cyclic, for example if the <samp class="option">wrap</samp> option is specified. </dd> <dt><samp class="option">hls_segment_filename <var class="var">filename</var></samp></dt> <dd> Set the segment filename. Unless the <samp class="option">hls_flags</samp> option is set with ‘<samp class="samp">single_file</samp>’, <var class="var">filename</var> is used as a string format with the segment number appended. For example: <pre class="example-preformatted">ffmpeg -i in.nut -hls_segment_filename 'file%03d.ts' out.m3u8 </pre>

will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">file000.ts</samp>, <samp class="file">file001.ts</samp>, <samp class="file">file002.ts</samp>, etc. <var class="var">filename</var> may contain a full path or relative path specification, but only the file name part without any path will be contained in the m3u8 segment list. Should a relative path be specified, the path of the created segment files will be relative to the current working directory. When <samp class="option">strftime_mkdir</samp> is set, the whole expanded value of <var class="var">filename</var> will be written into the m3u8 segment list. When <samp class="option">var_stream_map</samp> is

set with two or more variant streams, the <var class=”var”>filename</var> pattern must contain the string "%v", and this string will be expanded to the position of variant stream index in the generated segment file names. For example: <pre class=”example-preformatted”>ffmpeg -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ -hls_segment_filename 'file_%v_%03d.ts' out_%v.m3u8 </pre>

will produce the playlists segment file sets: <samp class=”file”>file_0_000.ts</samp>, <samp class=”file”>file_0_001.ts</samp>, <samp class=”file”>file_0_002.ts</samp>, etc. and <samp class=”file”>file_1_000.ts</samp>, <samp class=”file”>file_1_001.ts</samp>, <samp class=”file”>file_1_002.ts</samp>, etc. The string "%v" may be present in the filename or in the last directory name containing the file, but only in one of them. (Additionally, %v may appear multiple times in the last sub-directory or filename.) If the string %v is present in the directory name, then sub-directories are created after expanding the directory name pattern. This enables creation of segments corresponding to different variant streams in subdirectories. For example: <pre class=”example-preformatted”>ffmpeg -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ -hls_segment_filename 'vs%v/file_%03d.ts' vs%v/out.m3u8 </pre> will produce the playlists segment file sets: <samp class=”file”>vs0/file_000.ts</samp>, <samp class=”file”>vs0/file_001.ts</samp>, <samp class=”file”>vs0/file_002.ts</samp>, etc. and <samp class=”file”>vs1/file_000.ts</samp>, <samp class=”file”>vs1/file_001.ts</samp>, <samp class=”file”>vs1/file_002.ts</samp>, etc. </dd> <dt><samp class=”option”>strftime <var class=”var”>bool</var></samp></dt> <dd> Use <code class=”code”>strftime()</code> on <var class=”var”>filename</var> to expand the segment filename with localtime. The segment number is also available in this mode, but to use it, you need to set ‘<samp class=”samp”>second_level_segment_index</samp>’ in the <samp class=”option”>hls_flag</samp> and %%d will be the specifier. For example: <pre class=”example-preformatted”>ffmpeg -i in.nut -strftime 1 -hls_segment_filename 'file-%Y%m%d-%s.ts' out.m3u8 </pre> will produce the playlist, <samp class=”file”>out.m3u8</samp>, and segment files: <samp class=”file”>file-20160215-1455569023.ts</samp>, <samp class=”file”>file-20160215-1455569024.ts</samp>, etc. Note: On some systems/environments, the <code class=”code”>%s</code> specifier is not available. See <code class=”code”>strftime()</code> documentation. For example: <pre class=”example-preformatted”>ffmpeg -i in.nut -strftime 1 -hls_flags second_level_segment_index -hls_segment_filename 'file-%Y%m%d-%%04d.ts' out.m3u8 </pre> will produce the playlist, <samp class=”file”>out.m3u8</samp>, and segment files: <samp class=”file”>file-20160215-0001.ts</samp>, <samp class=”file”>file-

20160215-0002.ts</samp>, etc. </dd> <dt><samp class="option">strftime_mkdir <var class="var">bool</var></samp></dt> <dd> Used together with <samp class="option">strftime</samp>, it will create all sub-directories which are present in the expanded values of option <samp class="option">hls_segment_filename</samp>. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -strftime_mkdir 1 -hls_segment_filename '%Y%m%d/file-%Y%m%d-%s.ts' out.m3u8 </pre> will create a directory <samp class="file">201560215</samp> (if it does not exist), and then produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">20160215/file-20160215-1455569023.ts</samp>, <samp class="file">20160215/file-20160215-1455569024.ts</samp>, etc. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -strftime_mkdir 1 -hls_segment_filename '%Y/%m/%d/file-%Y%m%d-%s.ts' out.m3u8 </pre> will create a directory hierarchy <samp class="file">2016/02/15</samp> (if any of them do not exist), and then produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">2016/02/15/file-20160215-1455569023.ts</samp>, <samp class="file">2016/02/15/file-20160215-1455569024.ts</samp>, etc. </dd> <dt><samp class="option">hls_segment_options <var class="var">options_list</var></samp></dt> <dd> Set output format options using a :-separated list of key=value parameters. Values containing <code class="code">:</code> special characters must be escaped. </dd> <dt><samp class="option">hls_key_info_file <var class="var">key_info_file</var></samp></dt> <dd> Use the information in <var class="var">key_info_file</var> for segment encryption. The first line of <var class="var">key_info_file</var> specifies the key URI written to the playlist. The key URL is used to access the encryption key during playback. The second line specifies the path to the key file used to obtain the key during the encryption process. The key file is read as a single packed array of 16 octets in binary format. The optional third line specifies the initialization vector (IV) as a hexadecimal string to be used instead of the segment sequence number (default) for encryption. Changes to <var class="var">key_info_file</var> will result in segment encryption with the new key/IV and an entry in the playlist for the new key URI/IV if <samp class="option">hls_flags periodic_rekey</samp> is enabled. Key info file format: <pre class="example-preformatted"><var class="var">key URI</var> <var class="var">key file path</var> <var class="var">IV</var> (optional) </pre>

Example key URIs: <pre class="example-preformatted">http://server/file.key /path/to/file.key file.key </pre>

Example key file paths: <pre class="example-preformatted">file.key /path/to/file.key </pre>

Example IV: <pre class="example-

preformatted”>0123456789ABCDEF0123456789ABCDEF </pre>

Key info file example: <pre class=”example-preformatted”>http://server/file.key /path/to/file.key 0123456789ABCDEF0123456789ABCDEF </pre>

Example shell script: <pre class=”example-preformatted”>#!/bin/sh BASE_URL=${1:-'.'} openssl rand 16 > file.key echo $BASE_URL/file.key > file.keyinfo echo file.key >> file.keyinfo echo $(openssl rand -hex 16) >> file.keyinfo ffmpeg -f lavfi -re -i testsrc -c:v h264 -hls_flags delete_segments \ -hls_key_info_file file.keyinfo out.m3u8 </pre>

</dd> <dt><samp class=”option”>hls_enc <var class=”var”>bool</var></samp></dt> <dd> Enable (1) or disable (0) the AES128 encryption. When enabled every segment generated is encrypted and the encryption key is saved as <var class=”var”>playlist name</var>.key. </dd> <dt><samp class=”option”>hls_enc_key <var class=”var”>key</var></samp></dt> <dd> Specify a 16-octet key to encrypt the segments, by default it is randomly generated. </dd> <dt><samp class=”option”>hls_enc_key_url <var class=”var”>keyurl</var></samp></dt> <dd> If set, <var class=”var”>keyurl</var> is prepended instead of <var class=”var”>baseurl</var> to the key filename in the playlist. </dd> <dt><samp class=”option”>hls_enc_iv <var class=”var”>iv</var></samp></dt> <dd> Specify the 16-octet initialization vector for every segment instead of the autogenerated ones. </dd> <dt><samp class=”option”>hls_segment_type <var class=”var”>flags</var></samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>mpegts</samp>’</dt> <dd> Output segment files in MPEG-2 Transport Stream format. This is compatible with all HLS versions. </dd> <dt>‘<samp class=”samp”>fmp4</samp>’</dt> <dd> Output segment files in fragmented MP4 format, similar to MPEG-DASH. fmp4 files may be used in HLS version 7 and above. </dd> </dl>

</dd>

<dt><samp class=”option”>hls_fmp4_init_filename <var class=”var”>filename</var></samp></dt>

<dd> Set filename for the fragment files header file, default filename is <samp class=”file”>init.mp4</samp>.

When <samp class=”option”>strftime</samp> is enabled, <var class=”var”>filename</var> is expanded to the segment filename with localtime.

For example:

<pre class="example-preformatted">ffmpeg -i in.nut -hls_segment_type fmp4
-strftime 1 -hls_fmp4_init_filename "%s_init.mp4" out.m3u8

</pre>

will produce init like this <samp class="file">1602678741_init.mp4</samp>.

</dd>

<dt><samp class="option">hls_fmp4_init_resend <var class="var">bool</var></samp></dt>

<dd> Resend init file after m3u8 file refresh every time, default is <var
class="var">0</var>.

When <samp class="option">var_stream_map</samp> is set with two
or more variant streams, the

<var class="var">filename</var> pattern must contain the string "%v",
this string specifies

the position of variant stream index in the generated init file names.

The string "%v" may be present in the filename or in the last direc-
tory name

containing the file. If the string is present in the directory name, then

sub-directories are created after expanding the directory name pattern. This

enables creation of init files corresponding to different variant streams in

subdirectories.

</dd>

<dt><samp class="option">hls_flags <var class="var">flags</var></samp></dt>

<dd> Possible values:

<dl class="table">

<dt>‘<samp class="samp">single_file</samp>’</dt>

<dd> If this flag is set, the muxer will store all segments in a single MPEG-
TS

file, and will use byte ranges in the playlist. HLS playlists generated with

this way will have the version number 4.

For example:

<pre class="example-preformatted">ffmpeg -i in.nut -hls_flags single_file out.m3u8

</pre>

will produce the playlist, <samp class="file">out.m3u8</samp>, and a single segment file,

<samp class="file">out.ts</samp>.

</dd>

<dt>‘<samp class="samp">delete_segments</samp>’</dt>

<dd> Segment files removed from the playlist are deleted after a period of time

equal to the duration of the segment plus the duration of the playlist.

</dd>

<dt>‘<samp class="samp">append_list</samp>’</dt>

<dd> Append new segments into the end of old segment list,

and remove the <code class="code">#EXT-X-ENDLIST</code> from the old segment list.

</dd>

<dt>‘<samp class="samp">round_durations</samp>’</dt>

<dd> Round the duration info in the playlist file segment info to integer

values, instead of using floating point.

If there are no other features requiring higher HLS versions be used,

then this will allow <code class="command">ffmpeg</code> to output a HLS version 2 m3u8.

</dd>

<dt>‘<samp class="samp">discont_start</samp>’</dt>

<dd> Add the <code class=”code”>#EXT-X-DISCONTINUITY</code> tag to the playlist, before the

first segment’s information.

</dd>

<dt>‘<samp class=”samp”>omit_endlist</samp>’</dt>

<dd> Do not append the <code class=”code”>EXT-X-ENDLIST</code> tag at the end of the playlist.

</dd>

<dt>‘<samp class=”samp”>periodic_rekey</samp>’</dt>

<dd> The file specified by <code class=”code”>hls_key_info_file</code> will be checked periodically and

detect updates to the encryption info. Be sure to replace this file atomically,

including the file containing the AES encryption key.

</dd>

<dt>‘<samp class=”samp”>independent_segments</samp>’</dt>

<dd> Add the <code class=”code”>#EXT-X-INDEPENDENT-SEGMENTS</code> tag to playlists that has video segments

and when all the segments of that playlist are guaranteed to start with a key frame.

</dd>

<dt>‘<samp class=”samp”>iframes_only</samp>’</dt>

<dd> Add the <code class=”code”>#EXT-X-I-FRAMES-ONLY</code> tag to playlists that has video segments

and can play only I-frames in the <code class=”code”>#EXT-X-BYTERANGE</code> mode.

</dd>

<dt>‘<samp class=”samp”>split_by_time</samp>’</dt>

<dd> Allow segments to start on frames other than key frames. This improves

behavior on some players when the time between key frames is inconsistent,

but may make things worse on others, and can cause some oddities during

seeking. This flag should be used with the <samp class="option">hls_time</samp> option.

</dd>

<dt>‘<samp class="samp">program_date_time</samp>’</dt>

<dd> Generate <code class="code">EXT-X-PROGRAM-DATE-TIME</code> tags.

</dd>

<dt>‘<samp class="samp">second_level_segment_index</samp>’</dt>

<dd> Make it possible to use segment indexes as %%d in the

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

<samp class="option">strftime</samp> option is on. To get fixed width numbers with trailing zeroes, %%0xd format

is available where x is the required width.

</dd>

<dt>‘<samp class="samp">second_level_segment_size</samp>’</dt>

<dd> Make it possible to use segment sizes (counted in bytes) as %%s in

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

strftime is on. To get fixed width numbers with trailing zeroes, %%0xs format

is available where x is the required width.

</dd>

<dt>‘<samp class="samp">second_level_segment_duration</samp>’</dt>

<dd> Make it possible to use segment duration (calculated in microseconds) as %%t in

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

strftime is on. To get fixed width numbers with trailing zeroes, %%0xt format

is available where x is the required width.

For example:

<pre class="example-preformatted">ffmpeg -i sample.mpeg \

-f hls -hls_time 3 -hls_list_size 5 \

-hls_flags second_level_segment_index+second_level_segment_size+second_level_segment_duration \

-strftime 1 -strftime_mkdir 1 -hls_segment_filename "segment_%Y%m%d%H%M%S_%%04d_%%08s_stream.m3u8

</pre>

will produce segments like this:

<samp class="file">segment_20170102194334_0003_00122200_0000003000000.ts</samp>, <samp class="file">segment_20170102194334_0004_00120072_0000003000000.ts</samp> etc.

</dd>

<dt>‘<samp class="samp">temp_file</samp>’</dt>

<dd> Write segment data to <samp class="file">filename.tmp</samp> and rename to filename only once the

segment is complete.

A webserver serving up segments can be configured to reject requests to *.tmp to

prevent access to in-progress segments before they have been added to the m3u8

playlist.

This flag also affects how m3u8 playlist files are created. If this flag is set,

all playlist files will be written into a temporary file and renamed after they

are complete, similarly as segments are handled. But playlists with <code class=”code”>file</code>

protocol and with <samp class=”option”>hls_playlist_type</samp> type other than ‘<samp class=”samp”>vod</samp>’ are

always written into a temporary file regardless of this flag.

Master playlist files specified with <samp class=”option”>master_pl_name</samp>, if any, with

<code class=”code”>file</code> protocol, are always written into temporary file regardless of this

flag if <samp class=”option”>master_pl_publish_rate</samp> value is other than zero.

</dd>

</dl>

</dd> <dt><samp class=”option”>hls_playlist_type <var class=”var”>type</var></samp></dt> <dd> If type is ‘<samp class=”samp”>event</samp>’, emit <code class=”code”>#EXT-X-PLAYLIST-TYPE:EVENT</code> in the m3u8 header. This forces <samp class=”option”>hls_list_size</samp> to 0; the playlist can only be appended to. If type is ‘<samp class=”samp”>vod</samp>’, emit <code class=”code”>#EXT-X-PLAYLIST-TYPE:VOD</code> in the m3u8 header. This forces <samp class=”option”>hls_list_size</samp> to 0; the playlist must not change. </dd> <dt><samp class=”option”>method <var class=”var”>method</var></samp></dt> <dd> Use the given HTTP method to create the hls files. For example: <pre class=”example-preformatted”>ffmpeg -re -i in.ts -f hls -method PUT http://example.com/live/out.m3u8 </pre> will upload all the mpegts segment files to the HTTP server using the HTTP PUT method, and update the m3u8 files every <code class=”code”>refresh</code> times using the same method. Note that the HTTP server must support the given method for uploading files. </dd> <dt><samp class=”option”>http_user_agent <var class=”var”>agent</var></samp></dt> <dd> Override User-Agent field in HTTP header. Applicable only for HTTP output. </dd> <dt><samp class=”option”>var_stream_map <var class=”var”>stream_map</var></samp></dt> <dd> Specify a map string defining how to group the audio, video and subtitle streams into different variant streams. The variant stream groups are separated by space. Expected string format is like this "a:0,v:0 a:1,v:1 ....". Here a:, v:, s: are the keys to specify audio, video and subtitle streams respectively. Allowed values are 0 to 9 (limited just based on practical usage). When there are two or more variant streams, the output

filename pattern must contain the string "%v": this string specifies the position of variant stream index in the output media playlist filenames. The string "%v" may be present in the filename or in the last directory name containing the file. If the string is present in the directory name, then sub-directories are created after expanding the directory name pattern. This enables creation of variant streams in subdirectories. A few examples follow. <ul class="itemize mark-bullet"> Create two hls variant streams. The first variant stream will contain video stream of bitrate 1000k and audio stream of bitrate 64k and the second variant stream will contain video stream of bitrate 256k and audio stream of bitrate 32k. Here, two media playlist with file names <samp class="file">out_0.m3u8</samp> and <samp class="file">out_1.m3u8</samp> will be created. <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ http://example.com/live/out_%v.m3u8 </pre>

If you want something meaningful text instead of indexes in result names, you may specify names for each or some of the variants. The following example will create two hls variant streams as in the previous one. But here, the two media playlist with file names <samp class="file">out_my_hd.m3u8</samp> and <samp class="file">out_my_sd.m3u8</samp> will be created. <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0,name:my_hd v:1,a:1,name:my_sd" \ http://example.com/live/out_%v.m3u8 </pre>

Create three hls variant streams. The first variant stream will be a video only stream with video bitrate 1000k, the second variant stream will be an audio only stream with bitrate 64k and the third variant stream will be a video only stream with bitrate 256k. Here, three media playlist with file names <samp class="file">out_0.m3u8</samp>, <samp class="file">out_1.m3u8</samp> and <samp class="file">out_2.m3u8</samp> will be created. <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k \ -map 0:v -map 0:a -map 0:v -f hls -var_stream_map "v:0 a:0 v:1" \ http://example.com/live/out_%v.m3u8 </pre>

Create the variant streams in subdirectories. Here, the first media playlist is created at <samp class="file">http://example.com/live/vs_0/out.m3u8</samp> and the second one at <samp class="file">http://example.com/live/vs_1/out.m3u8</samp>. <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ http://example.com/live/vs_%v/out.m3u8 </pre>

Create two audio only and two video only variant streams. In

addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the two video only variant streams with audio group names ’aud_low’ and ’aud_high’. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k -b:v:1 3000k \ -map 0:a -map 0:a -map 0:v -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low a:1,agroup:aud_high v:0,agroup:aud_low v:1,agroup:aud_high" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

 Create two audio only and one video only variant streams. In addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the one video only variant streams with audio group name ’aud_low’, and the audio group have default stat is NO or YES. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k \ -map 0:a -map 0:a -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low,default:yes a:1,agroup:aud_low v:0,agroup:aud_low" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

Create two audio only and one video only variant streams. In addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the one video only variant streams with audio group name ’aud_low’, and the audio group have default stat is NO or YES, and one audio have and language is named ENG, the other audio language is named CHN. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k \ -map 0:a -map 0:a -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low,default:yes,language:ENG a:1,agroup:aud_low,language:CHN v:0,agroup:aud_low" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

Create a single variant stream. Add the <code class=”code”>#EXT-X-MEDIA</code> tag with <code class=”code”>TYPE=SUBTITLES</code> in the master playlist with webvtt subtitle group name ’subtitle’ and optional subtitle name, e.g. ’English’. Make sure the input file has one text subtitle stream at least. <pre class=”example-preformatted”>ffmpeg -y -i input_with_subtitle.mkv \ -b:v:0

5250k -c:v h264 -pix_fmt yuv420p -profile:v main -level 4.1 \ -b:a:0 256k \ -c:s webvtt -c:a mp2 -ar 48000 -ac 2 -map 0:v -map 0:a:0 -map 0:s:0 \ -f hls -var_stream_map "v:0,a:0,s:0,sgroup:subtitle,sname:English" \ -master_pl_name master.m3u8 -t 300 -hls_time 10 -hls_init_time 4 -hls_list_size \ 10 -master_pl_publish_rate 10 -hls_flags \ delete_segments+discont_start+split_by_time ./tmp/video.m3u8 </pre>

</dd> <dt><samp class="option">cc_stream_map <var class="var">cc_stream_map</var></samp></
<dd> Map string which specifies different closed captions groups and their attributes. The closed captions stream groups are separated by space. Expected string format is like this "ccgroup:<group name>,instreamid:<INSTREAM-ID>,language:<language code> ....". ’ccgroup’ and ’instreamid’ are mandatory attributes. ’language’ is an optional attribute. The closed captions groups configured using this option are mapped to different variant streams by providing the same ’ccgroup’ name in the <samp class="option">var_stream_map</samp> string. For example: <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -a53cc:0 1 -a53cc:1 1 \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls \ -cc_stream_map "ccgroup:cc,instreamid:CC1,language:en cc-group:cc,instreamid:CC2,language:sp" \ -var_stream_map "v:0,a:0,ccgroup:cc v:1,a:1,ccgroup:cc" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre> will add two <code class="code">#EXT-X-MEDIA</code> tags with <code class="code">TYPE=CLOSED-CAPTIONS</code> in the master playlist for the INSTREAM-IDs ’CC1’ and ’CC2’. Also, it will add <code class="code">CLOSED-CAPTIONS</code> attribute with group name ’cc’ for the two output variant streams. If <samp class="option">var_stream_map</samp> is not set, then the first available ccgroup in <samp class="option">cc_stream_map</samp> is mapped to the output variant stream. For example: <pre class="example-preformatted">ffmpeg -re -i in.ts -b:v 1000k -b:a 64k -a53cc 1 -f hls \ -cc_stream_map "ccgroup:cc,instreamid:CC1,language:en" \ -master_pl_name master.m3u8 \ http://example.com/live/out.m3u8 </pre> this will add <code class="code">#EXT-X-MEDIA</code> tag with <code class="code">TYPE=CLOSED-CAPTIONS</code> in the master playlist with group name ’cc’, language ’en’ (english) and INSTREAM-ID ’CC1’. Also, it will add <code class="code">CLOSED-CAPTIONS</code> attribute with group name ’cc’ for the output variant stream. </dd> <dt><samp class="option">master_pl_name <var class="var">name</var></samp></dt> <dd> Create HLS master playlist with the given name. For example: <pre class="example-preformatted">ffmpeg -re -i in.ts -f hls -master_pl_name master.m3u8 http://example.com/live/out.m3u8 </pre> creates an HLS master playlist with name <samp

class=”file”>master.m3u8</samp> which is published at http://example.com/live/. </dd> <dt><samp class=”option”>master_pl_publish_rate <var class=”var”>count</var></samp></dt> <dd> Publish master play list repeatedly every after specified number of segment intervals. For example: <pre class=”example-preformatted”>ffmpeg -re -i in.ts -f hls -master_pl_name master.m3u8 \ -hls_time 2 -master_pl_publish_rate 30 http://example.com/live/out.m3u8 </pre> creates an HLS master playlist with name <samp class=”file”>master.m3u8</samp> and keeps publishing it repeatedly every after 30 segments i.e. every after 60s. </dd> <dt><samp class=”option”>http_persistent <var class=”var”>bool</var></samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP output. </dd> <dt><samp class=”option”>timeout <var class=”var”>timeout</var></samp></dt> <dd> Set timeout for socket I/O operations. Applicable only for HTTP output. </dd> <dt><samp class=”option”>ignore_io_errors <var class=”var”>bool</var></samp></dt> <dd> Ignore IO errors during open, write and delete. Useful for long-duration runs with network output. </dd> <dt><samp class=”option”>headers <var class=”var”>headers</var></samp></dt> <dd> Set custom HTTP headers, can override built in default headers. Applicable only for HTTP output. </dd> </dl>

 <h3 class=”section”>21.48 iamf# TOC</h3> Immersive Audio Model and Formats (IAMF) muxer. IAMF is used to provide immersive audio content for presentation on a wide range of devices in both streaming and offline applications. These applications include internet audio streaming, multicasting/broadcasting services, file download, gaming, communication, virtual and augmented reality, and others. In these applications, audio may be played back on a wide range of devices, e.g., headphones, mobile phones, tablets, TVs, sound bars, home theater systems, and big screens. This format was promoted and designed by Alliance for Open Media. For more information about this format, see https://aomedia.org/iamf/. <h3 class=”section”>21.49 ico# TOC</h3>

ICO file muxer. Microsoft’s icon file format (ICO) has some strict limitations that should be noted: <ul class=”itemize mark-bullet”> Size cannot exceed 256 pixels in any dimension

Only BMP and PNG images can be stored

If a BMP image is used, it must be one of the following pixel formats: <pre class=”example-preformatted”>BMP Bit Depth

FFmpeg Pixel Format 1bit pal8 4bit pal8 8bit pal8 16bit rgb555le 24bit bgr24 32bit bgra </pre>

If a BMP image is used, it must use the BITMAPINFOHEADER DIB header

If a PNG image is used, it must use the rgba pixel format

 <h3 class="section">21.50 ilbc# TOC</h3> Internet Low Bitrate Codec (iLBC) raw muxer. It accepts a single ‘<samp class="samp">ilbc</samp>’ audio stream. <h3 class="section">21.51 image2, image2pipe# TOC</h3> Image file muxer. The ‘<samp class="samp">image2</samp>’ muxer writes video frames to image files. The output filenames are specified by a pattern, which can be used to produce sequentially numbered series of files. The pattern may contain the string "%d" or "%0<var class="var">N</var>d", this string specifies the position of the characters representing a numbering in the filenames. If the form "%0<var class="var">N</var>d" is used, the string representing the number in each filename is 0-padded to <var class="var">N</var> digits. The literal character ’%’ can be specified in the pattern with the string "%%". If the pattern contains "%d" or "%0<var class="var">N</var>d", the first filename of the file list specified will contain the number 1, all the following numbers will be sequential. The pattern may contain a suffix which is used to automatically determine the format of the image files to write. For example the pattern "img-%03d.bmp" will specify a sequence of filenames of the form <samp class="file">img-001.bmp</samp>, <samp class="file">img-002.bmp</samp>, ..., <samp class="file">img-010.bmp</samp>, etc. The pattern "img%%-%d.jpg" will specify a sequence of filenames of the form <samp class="file">img%-1.jpg</samp>, <samp class="file">img%-2.jpg</samp>, ..., <samp class="file">img%-10.jpg</samp>, etc. The image muxer supports the .Y.U.V image file format. This format is special in that each image frame consists of three files, for each of the YUV420P components. To read or write this image file format, specify the name of the ’.Y’ file. The muxer will automatically open the ’.U’ and ’.V’ files as required. The ‘<samp class="samp">image2pipe</samp>’ muxer accepts the same options as the ‘<samp class="samp">image2</samp>’ muxer, but ignores the pattern verification and expansion, as it is supposed to write to the command output rather than to an actual stored file. <a

name=”Options-87”> <h4 class=”subsection”>21.51.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>frame_pts <var class=”var”>bool</var></samp></dt> <dd> If set to 1, expand the filename with the packet PTS (presentation time stamp). Default value is 0. </dd> <dt><samp class=”option”>start_number <var class=”var”>count</var></samp></dt> <dd> Start the sequence from the specified number. Default value is 1. </dd> <dt><samp class=”option”>update <var class=”var”>bool</var></samp></dt> <dd> If set to 1, the filename will always be interpreted as just a filename, not a pattern, and the corresponding file will be continuously overwritten with new images. Default value is 0. </dd> <dt><samp class=”option”>strftime <var class=”var”>bool</var></samp></dt> <dd> If set to 1, expand the filename with date and time information from <code class=”code”>strftime()</code>. Default value is 0. </dd> <dt><samp class=”option”>atomic_writing <var class=”var”>bool</var></samp></dt> <dd> Write output to a temporary file, which is renamed to target filename once writing is completed. Default is disabled. </dd> <dt><samp class=”option”>protocol_opts <var class=”var”>options_list</var></samp></dt> <dd> Set protocol options as a :-separated list of key=value parameters. Values containing the <code class=”code”>:</code> special character must be escaped. </dd> <dt><samp class=”option”>update_filemtime <var class=”var”>bool</var></samp></dt> <dd> If set to 1, set each output file’s modification time to the <code class=”code”>creation_time</code> metadata value plus the frame’s PTS offset. If <code class=”code”>creation_time</code> is missing or unparsable, a warning is logged and the option is ignored. Default value is 0. </dd> </dl>

 <h4 class=”subsection”>21.51.2 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> for creating a sequence of files <samp class=”file”>img-001.jpeg</samp>, <samp class=”file”>img-002.jpeg</samp>, ..., taking one image every second from the input video: <pre class=”example-preformatted”>ffmpeg -i in.avi -fps_mode cfr -r 1 -f image2 'img-%03d.jpeg' </pre>

Note that with <code class=”command”>ffmpeg</code>, if the format is not specified with the <code class=”code”>-f</code> option and the output filename specifies an image file format, the image2 muxer is automatically selected, so the previous command can be written as: <pre class=”example-preformatted”>ffmpeg -i in.avi -fps_mode cfr -r 1 'img-%03d.jpeg' </pre>

Note also that the pattern must not necessarily contain "%d" or "%0<var class="var">N</var>d", for example to create a single image file <samp class="file">img.jpeg</samp> from the start of the input video you can employ the command: <pre class="example-preformatted">ffmpeg -i in.avi -f image2 -frames:v 1 img.jpeg </pre>

The <samp class="option">strftime</samp> option allows you to expand the filename with date and time information. Check the documentation of the <code class="code">strftime()</code> function for the syntax.

To generate image files from the <code class="code">strftime()</code> "%Y-%m-%d_%H-%M-%S" pattern, the following <code class="command">ffmpeg</code> command can be used: <pre class="example-preformatted">ffmpeg -f v4l2 -r 1 -i /dev/video0 -f image2 -strftime 1 "%Y-%m-%d_%H-%M-%S.jpg" </pre>

Set the file name with current frame’s PTS: <pre class="example-preformatted">ffmpeg -f v4l2 -r 1 -i /dev/video0 -copyts -f image2 -frame_pts true %d.jpg </pre>

Publish contents of your desktop directly to a WebDAV server every second: <pre class="example-preformatted">ffmpeg -f x11grab -framerate 1 -i :0.0 -q:v 6 -update 1 -protocol_opts method=PUT http://example.com/desktop.jpg </pre>

 <h3 class="section">21.52 ircam# TOC</h3> Berkeley / IRCAM / CARL Sound Filesystem (BICSF) format muxer. The Berkeley/IRCAM/CARL Sound Format, developed in the 1980s, is a result of the merging of several different earlier sound file formats and systems including the csound system developed by Dr Gareth Loy at the Computer Audio Research Lab (CARL) at UC San Diego, the IRCAM sound file system developed by Rob Gross and Dan Timis at the Institut de Recherche et Coordination Acoustique / Musique in Paris and the Berkeley Fast Filesystem. It was developed initially as part of the Berkeley/IRCAM/CARL Sound Filesystem, a suite of programs designed to implement a filesystem for audio applications running under Berkeley UNIX. It was particularly popular in academic music research centres, and was used a number of times in the creation of early computer-generated compositions. This muxer accepts a single audio stream containing PCM data. <h3 class="section">21.53 iterm2# TOC</h3> iTerm2 inline image protocol muxer. This muxer writes video frames as OSC 1337 inline

images for display in terminals that support the iTerm2 image protocol. Use <samp class="option">-re</samp> to limit the output rate to the source framerate; without it, frames are emitted as fast as they are encoded, which is usually not desired for live display. Frames are sent with the multipart form of the protocol, which splits each image across several short control sequences. This avoids the per-sequence size limit that otherwise discards large frames, and requires iTerm2 3.5 or newer. The output is a self-contained byte stream and can be redirected to a file. Replaying the file with <code class="command">cat</code> displays the images in the terminal. <h4 class="subsection">21.53.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">display_width <var class="var">size</var></samp></dt> <dd> Set the displayed image width. <var class="var">size</var> can be ‘<samp class="samp">auto</samp>’, <var class="var">N</var> terminal cells, <var class="var">N</var>px pixels, or <var class="var">N</var>% of the terminal width. When unset, the terminal derives the width from the image. </dd> <dt><samp class="option">display_height <var class="var">size</var></samp></dt> <dd> Set the displayed image height. <var class="var">size</var> uses the same syntax as <samp class="option">display_width</samp>. When unset, the terminal derives the height from the image. </dd> <dt><samp class="option">keep_aspect <var class="var">bool</var></samp></dt> <dd> Preserve the input aspect ratio when scaling. Default is enabled. </dd> <dt><samp class="option">tmux <var class="var">bool</var></samp></dt> <dd> Wrap image data in tmux DCS passthrough. This requires a tmux version whose passthrough sequence size limit is large enough for image data, with passthrough enabled via <code class="command">tmux set -g allow-passthrough on</code>. Default is disabled. </dd> </dl>

 <h4 class="subsection">21.53.2 Examples# TOC</h4>

Display a video in an iTerm2 terminal: <pre class="example-preformatted">ffmpeg -re -i input.mp4 -f iterm2 - </pre>

Scale the displayed image to 40 terminal cells tall. Inside tmux, enable passthrough first with <code class="command">tmux set -g allow-passthrough on</code>, then add <samp class="option">tmux</samp>: <pre class="example-preformatted">ffmpeg -re -i input.mp4 -f iterm2 -display_height 40 -tmux 1 - </pre>

 <h3 class="section">21.54 jpeg_mpf<a class="anchor hidden-xs" href="#jpeg_005fmpf-1"

aria-hidden=”true”># TOC</h3> JPEG Multi-Picture Format muxer. Stores multiple JPEG images in a single file using the CIPA DC-007 MP extensions. The first input stream is always the primary image. Every other stream must be specified by a stream group describing what it is; see the <samp class=”option”>-stream_group</samp> option of the <code class=”command”>ffmpeg</code> tool. Note: Currently, the only implemented type of stream group is ‘<samp class=”samp”>gain_map</samp>’, which may be used to store a gain map for JPEG Ultra HDR. <h4 class=”subsection”>21.54.1 Examples# TOC</h4>

Encode an SDR tone-mapped image and an HDR gain map computed from it: <pre class=”example-preformatted”>ffmpeg -i input-hdr.png -filter_complex "[0:v] scale=out_transfer=srgb:out_primaries=bt709,split [base][sdr]; [base][0:v] gainmap [gain]" -map "[sdr]" -map "[gain]" -c:v mjpeg -stream_group type=gain_map:st=0:st=1:el_index=1 -f jpeg_mpf output.jpg </pre>

 <h3 class=”section”>21.55 ivf# TOC</h3> On2 IVF muxer. IVF was developed by On2 Technologies (formerly known as Duck Corporation), to store internally developed codecs. This muxer accepts a single ‘<samp class=”samp”>vp8</samp>’, ‘<samp class=”samp”>vp9</samp>’, or ‘<samp class=”samp”>av1</samp>’ video stream. <h3 class=”section”>21.56 jacosub# TOC</h3> JACOsub subtitle format muxer. This muxer accepts a single ‘<samp class=”samp”>jacosub</samp>’ subtitles stream. For more information about the format, see http://
  <h3 class=”section”>21.57 ktx# TOC</h3> Khronos Texture 1.0 muxer for ASTC textures. This muxer accepts a single ‘<samp class=”samp”>libastcenc</samp>’ video stream and writes one image: the ASTC block size and the image dimensions are taken from the encoder’s extradata, and a second packet is rejected. The stored row order is recorded in a <code class=”code”>KTXorientation</code> key. <h4 class=”subsection”>21.57.1 Options<a class=”anchor hidden-xs” href=”#Options-89” aria-

hidden=”true”># TOC</h4>

<dl class=”table”> <dt><samp class=”option”>srgb <var class=”var”>integer</var></samp></dt> <dd> Select the GL internal format color space. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Use the color profile published by the encoder, which is the default. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Write the linear ASTC format. </dd> <dt>‘<samp class=”samp”>srgb</samp>’</dt> <dd> Write the sRGB ASTC format. </dd> </dl>

An explicit value that contradicts a known encoder profile is rejected. A raw <samp class=”file”>.astc</samp> input does not carry a color profile, so remuxing it to <samp class=”file”>.ktx</samp> with <code class=”code”>auto</code> writes the sRGB format; pass <samp class=”option”>-srgb linear</samp> to keep a linear texture. </dd> </dl>

 <h3 class=”section”>21.58 kvag# TOC</h3> Simon & Schuster Interactive VAG muxer. This custom VAG container is used by some Simon & Schuster Interactive games such as "Real War", and "Real War: Rogue States". This muxer accepts a single ‘<samp class=”samp”>adpcm_ima_ssi</samp>’ audio stream. <h3 class=”section”>21.59 lc3# TOC</h3> Bluetooth SIG Low Complexity Communication Codec audio (LC3), or ETSI TS 103 634 Low Complexity Communication Codec plus (LC3plus). This muxer accepts a single ‘<samp class=”samp”>lc3</samp>’ audio stream. <h3 class=”section”>21.60 lrc# TOC</h3> LRC lyrics file format muxer. LRC (short for LyRiCs) is a computer file format that synchronizes song lyrics with an audio file, such as MP3, Vorbis, or MIDI. This muxer accepts a single ‘<samp class=”samp”>subrip</samp>’ or ‘<samp class=”samp”>text</samp>’ subtitles stream. <h4 class=”subsection”>21.60.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>precision <var class=”var”>number</var></samp></dt> <dd> Set the number of decimal digits used for the fractional (sub-second)

part of the timestamps. Range is <code class=”code”>1</code> to <code class=”code”>6</code>. Default is <code class=”code”>2</code>, i.e. centiseconds. </dd> </dl>

 <h4 class=”subsection”>21.60.2 Metadata# TOC</h4> The following metadata tags are converted to the format corresponding metadata: <dl class=”table”> <dt><samp class=”option”>title</samp></dt> <dt><samp class=”option”>album</samp></dt> <dt><samp class=”option”>artist</samp></dt> <dt><samp class=”option”>author</samp></dt> <dt><samp class=”option”>creator</samp></dt> <dt><samp class=”option”>encoder</samp></dt> <dt><samp class=”option”>encoder_version</samp> </dl>

If ‘<samp class=”samp”>encoder_version</samp>’ is not explicitly set, it is automatically set to the libavformat version. <h3 class=”section”>21.61 matroska# TOC</h3> Matroska container muxer. This muxer implements the matroska and webm container specs. <h4 class=”subsection”>21.61.1 Metadata# TOC</h4> The recognized metadata settings in this muxer are: <dl class=”table”> <dt><samp class=”option”>title</samp></dt> <dd> Set title name provided to a single track. This gets mapped to the FileDescription element for a stream written as attachment. </dd> <dt><samp class=”option”>language</samp></dt> <dd> Specify the language of the track in the Matroska languages form. The language can be either the 3 letters bibliographic ISO-639-2 (ISO 639-2/B) form (like "fre" for French), or a language code mixed with a country code for specialities in languages (like "fre-ca" for Canadian French). </dd> <dt><samp class=”option”>stereo_mode</samp></dt> <dd> Set stereo 3D video layout of two views in a single video track. The <samp class=”option”>-stereo3d</samp> option overrides existing stereoscopic side data and <code class=”code”>stereo_mode</code> metadata for the selected output stream. Otherwise, stream-level <code class=”code”>stereo_mode</code> metadata takes priority over global metadata, and stereoscopic side data is used when neither tag is present. The following values are recognized: <dl class=”table”> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> video is not stereo </dd> <dt>‘<samp class=”samp”>left_right</samp>’</dt> <dd> Both views are arranged side by side, Left-eye view is on the left </dd> <dt>‘<samp class=”samp”>bottom_top</samp>’</dt>

<dd> Both views are arranged in top-bottom orientation, Left-eye view is at bottom </dd> <dt>‘<samp class="samp">top_bottom</samp>’</dt> <dd> Both views are arranged in top-bottom orientation, Left-eye view is on top </dd> <dt>‘<samp class="samp">checkerboard_rl</samp>’</dt> <dd> Each view is arranged in a checkerboard interleaved pattern, Left-eye view being first </dd> <dt>‘<samp class="samp">checkerboard_lr</samp>’</dt> <dd> Each view is arranged in a checkerboard interleaved pattern, Right-eye view being first </dd> <dt>‘<samp class="samp">row_interleaved_rl</samp>’</dt> <dd> Each view is constituted by a row based interleaving, Right-eye view is first row </dd> <dt>‘<samp class="samp">row_interleaved_lr</samp>’</dt> <dd> Each view is constituted by a row based interleaving, Left-eye view is first row </dd> <dt>‘<samp class="samp">col_interleaved_rl</samp>’</dt> <dd> Both views are arranged in a column based interleaving manner, Right-eye view is first column </dd> <dt>‘<samp class="samp">col_interleaved_lr</samp>’</dt> <dd> Both views are arranged in a column based interleaving manner, Left-eye view is first column </dd> <dt>‘<samp class="samp">anaglyph_cyan_red</samp>’</dt> <dd> All frames are in anaglyph format viewable through red-cyan filters </dd> <dt>‘<samp class="samp">right_left</samp>’</dt> <dd> Both views are arranged side by side, Right-eye view is on the left </dd> <dt>‘<samp class="samp">anaglyph_green_magenta</samp>’</dt> <dd> All frames are in anaglyph format viewable through green-magenta filters </dd> <dt>‘<samp class="samp">block_lr</samp>’</dt> <dd> Both eyes laced in one Block, Left-eye view is first </dd> <dt>‘<samp class="samp">block_rl</samp>’</dt> <dd> Both eyes laced in one Block, Right-eye view is first </dd> </dl> </dd> </dl>

For example a 3D WebM clip can be created using the following command line: <pre class="example-preformatted">ffmpeg -i sample_left_right_clip.mpg -an -c:v libvpx -metadata stereo_mode=left_right -y stereo_clip.webm </pre>

 <h4 class="subsection">21.61.2 Options# TOC</h4> <dl class="table"> <dt><samp class="option">reserve_index_space <var class="var">size</var></samp></dt> <dd> By default, this muxer writes the index for seeking (called cues in Matroska terms) at the end of the file, because it cannot know in advance how much space to leave for the index at the beginning of the file. However for some use cases – e.g. streaming where seeking is possible but slow – it is useful to put the index at the beginning of the file. If this option is set to a non-zero value, the muxer will reserve <var class="var">size</var> bytes of space in the file header and then try to write the cues there when the muxing finishes. If the reserved space does not suffice, no Cues will be written, the file will be finalized and writing the trailer will return an error. A safe size

for most use cases should be about 50kB per hour of video. Note that cues are only written if the output is seekable and this option will have no effect if it is not. </dd> <dt><samp class="option">cues_to_front <var class="var">bool</var></samp></dt> <dd> If set, the muxer will write the index at the beginning of the file by shifting the main data if necessary. This can be combined with reserve_index_space in which case the data is only shifted if the initially reserved space turns out to be insufficient. This option is ignored if the output is unseekable. </dd> <dt><samp class="option">cluster_size_limit <var class="var">size</var></samp></dt> <dd> Store at most the provided amount of bytes in a cluster. If not specified, the limit is set automatically to a sensible hardcoded fixed value. </dd> <dt><samp class="option">cluster_time_limit <var class="var">duration</var></samp></dt> <dd> Store at most the provided number of milliseconds in a cluster. If not specified, the limit is set automatically to a sensible hardcoded fixed value. </dd> <dt><samp class="option">dash <var class="var">bool</var></samp></dt> <dd> Create a WebM file conforming to WebM DASH specification. By default it is set to <code class="code">false</code>. </dd> <dt><samp class="option">dash_track_number <var class="var">index</var></samp></dt> <dd> Track number for the DASH stream. By default it is set to <code class="code">1</code>. </dd> <dt><samp class="option">live <var class="var">bool</var></samp></dt> <dd> Write files assuming it is a live stream. By default it is set to <code class="code">false</code>. </dd> <dt><samp class="option">allow_raw_vfw <var class="var">bool</var></samp></dt> <dd> Allow raw VFW mode. By default it is set to <code class="code">false</code>. </dd> <dt><samp class="option">flipped_raw_rgb <var class="var">bool</var></samp></dt> <dd> If set to <code class="code">true</code>, store positive height for raw RGB bitmaps, which indicates bitmap is stored bottom-up. Note that this option does not flip the bitmap which has to be done manually beforehand, e.g. by using the ‘<samp class="samp">vflip</samp>’ filter. Default is <code class="code">false</code> and indicates bitmap is stored top down. </dd> <dt><samp class="option">write_crc32 <var class="var">bool</var></samp></dt> <dd> Write a CRC32 element inside every Level 1 element. By default it is set to <code class="code">true</code>. This option is ignored for WebM. </dd> <dt><samp class="option">default_mode <var class="var">mode</var></samp></dt> <dd> Control how the FlagDefault of the output tracks will be set. It influences which tracks players should play by default. The default mode is ‘<samp class="samp">passthrough</samp>’. <dl class="table"> <dt>‘<samp class="samp">infer</samp>’</dt> <dd> Every track with disposition default will have the FlagDefault set. Additionally, for each type of track (audio, video or subtitle), if no track with disposition default of this type exists, then the first track of this type will be marked as default (if existing). This ensures that the default

flag is set in a sensible way even if the input originated from containers that lack the concept of default tracks. </dd> <dt>‘<samp class=”samp”>infer_no_subs</samp>’</dt> <dd> This mode is the same as infer except that if no subtitle track with disposition default exists, no subtitle track will be marked as default. </dd> <dt>‘<samp class=”samp”>passthrough</samp>’</dt> <dd> In this mode the FlagDefault is set if and only if the AV_DISPOSITION_DEFAULT flag is set in the disposition of the corresponding stream. </dd> </dl> </dd> </dl>

 <h3 class=”section”>21.62 md5# TOC</h3> MD5 testing format. This is a variant of the hash muxer. Unlike that muxer, it defaults to using the MD5 hash function. See also the hash and framemd5 muxers. <h4 class=”subsection”>21.62.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> To compute the MD5 hash of the input converted to raw audio and video, and store it in the file <samp class=”file”>out.md5</samp>: <pre class=”example-preformatted”>ffmpeg -i INPUT -f md5 out.md5 </pre>

To print the MD5 hash to stdout: <pre class=”example-preformatted”>ffmpeg -i INPUT -f md5 - </pre>

 <h3 class=”section”>21.63 mcc# TOC</h3> Muxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-608 and CEA-708), ancillary time code, pan-scan data, etc. <h4 class=”subsection”>21.63.1 Options# TOC</h4>

The muxer options are: <dl class=”table”> <dt><samp class=”option”>override_time_code_rate</samp></dt> <dd> Override the <code class=”code”>Time Code Rate</code> value in the output. Defaults to trying to deduce from the stream’s <code class=”code”>time_base</code>, which often doesn’t work.

</dd> <dt><samp class="option">use_u_alias</samp></dt> <dd> Use the <code class="code">U</code> alias for the byte sequence <code class="code">E1h 00h 00h 00h</code>. Disabled by default because some <samp class="file">.mcc</samp> files disagree on whether it has 2 or 3 zero bytes. </dd> <dt><samp class="option">mcc_version</samp></dt> <dd> The MCC file format version. Must be either 1 or 2, defaults to 2. </dd> <dt><samp class="option">creation_program</samp></dt> <dd> The creation program. Defaults to this version of FFmpeg. </dd> <dt><samp class="option">creation_time</samp></dt> <dd> The creation time. Defaults to the current time. </dd> </dl>

 <h4 class="subsection">21.63.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Extract a MXF <code class="code">SMPTE_436M_ANC</code> stream from a MXF file and write it to a MCC file at 30 fps. <pre class="example-preformatted">ffmpeg -i input.mxf -c copy -map 0:d -override_time_code_rate 30 out.mcc </pre>

Extract EIA-608/CTA-708 closed captions from a <samp class="file">.mp4</samp> file and write them to a MCC file at 29.97 fps. <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=input.mp4[out+subcc]" -c:s copy -map 0:s -override_time_code_rate 30000/1001 out.mcc </pre>

 <h3 class="section">21.64 microdvd# TOC</h3> MicroDVD subtitle format muxer. This muxer accepts a single ‘<samp class="samp">microdvd</samp>’ subtitles stream. <h3 class="section">21.65 mmf# TOC</h3> Synthetic music Mobile Application Format (SMAF) format muxer. SMAF is a music data format specified by Yamaha for portable electronic devices, such as mobile phones and personal digital assistants. This muxer accepts a single ‘<samp class="samp">adpcm_yamaha</samp>’ audio stream. <h3 class="section">21.66 mp3# TOC</h3>

The MP3 muxer writes a raw MP3 stream with the following optional features: <ul class="itemize mark-bullet"> An ID3v2 metadata header at the beginning (enabled by default). Versions 2.3 and 2.4 are supported, the

<code class=”code”>id3v2_version</code> private option controls which one is used (3 or 4). Setting <code class=”code”>id3v2_version</code> to 0 disables the ID3v2 header completely.

The muxer supports writing attached pictures (APIC frames) to the ID3v2 header. The pictures are supplied to the muxer in form of a video stream with a single packet. There can be any number of those streams, each will correspond to a single APIC frame. The stream metadata tags <var class=”var”>title</var> and <var class=”var”>comment</var> map to APIC <var class=”var”>description</var> and <var class=”var”>picture type</var> respectively. See http://id3.org/id3v2.4.0-frames for allowed picture types. Note that the APIC frames must be written at the beginning, so the muxer will buffer the audio frames until it gets all the pictures. It is therefore advised to provide the pictures as soon as possible to avoid excessive buffering. A Xing/LAME frame right after the ID3v2 header (if present). It is enabled by default, but will be written only if the output is seekable. The <code class=”code”>write_xing</code> private option can be used to disable it. The frame contains various information that may be useful to the decoder, like the audio duration or encoder delay.

A legacy ID3v1 tag at the end of the file (disabled by default). It may be enabled with the <code class=”code”>write_id3v1</code> private option, but as its capabilities are very limited, its usage is not recommended.

Examples: Write an mp3 with an ID3v2.3 header and an ID3v1 footer: <pre class=”example-preformatted”>ffmpeg -i INPUT -id3v2_version 3 -write_id3v1 1 out.mp3 </pre>

To attach a picture to an mp3 file select both the audio and the picture stream with <code class=”code”>map</code>: <pre class=”example-preformatted”>ffmpeg -i input.mp3 -i cover.png -c copy -map 0 -map 1 -metadata:s:v title="Album cover" -metadata:s:v comment="Cover (Front)" out.mp3 </pre>

Write a "clean" MP3 without any extra features: <pre class=”example-preformatted”>ffmpeg -i input.wav -write_xing 0 -id3v2_version 0 out.mp3 </pre>

 <h3 class=”section”>21.67 mpegts# TOC</h3>

MPEG transport stream muxer. This muxer implements ISO 13818-1 and part of ETSI EN 300 468. The recognized metadata settings in mpegts muxer are <code class=”code”>service_provider</code>

and <code class=”code”>service_name</code>. If they are not set the default for <code class=”code”>service_provider</code> is ‘<samp class=”samp”>FFmpeg</samp>’ and the default for <code class=”code”>service_name</code> is ‘<samp class=”samp”>Service01</samp>’. <h4 class=”subsection”>21.67.1 Options# TOC</h4>

The muxer options are: <dl class=”table”> <dt><samp class=”option”>mpegts_transport_stream_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>transport_stream_id</samp>’. This identifies a transponder in DVB. Default is <code class=”code”>0x0001</code>. </dd> <dt><samp class=”option”>mpegts_original_network_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>original_network_id</samp>’. This is unique identifier of a network in DVB. Its main use is in the unique identification of a service through the path ‘<samp class=”samp”>Original_Network_ID, Transport_Stream_ID</samp>’. Default is <code class=”code”>0xff01</code>. </dd> <dt><samp class=”option”>mpegts_service_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>service_id</samp>’, also known as program in DVB. Default is <code class=”code”>0x0001</code>. </dd> <dt><samp class=”option”>mpegts_service_type <var class=”var”>integer</var></samp></dt> <dd> Set the program ‘<samp class=”samp”>service_type</samp>’. Default is <code class=”code”>digital_tv</code>. Accepts the following options: <dl class=”table”> <dt>‘<samp class=”samp”>hex_value</samp>’</dt> <dd> Any hexadecimal value between <code class=”code”>0x01</code> and <code class=”code”>0xff</code> as defined in ETSI 300 468. </dd> <dt>‘<samp class=”samp”>digital_tv</samp>’</dt> <dd> Digital TV service. </dd> <dt>‘<samp class=”samp”>digital_radio</samp>&rsquo <dd> Digital Radio service. </dd> <dt>‘<samp class=”samp”>teletext</samp>’</dt> <dd> Teletext service. </dd> <dt>‘<samp class=”samp”>advanced_codec_digital_radio</samp>’</dt> <dd> Advanced Codec Digital Radio service. </dd> <dt>‘<samp class=”samp”>mpeg2_digital_hdtv</samp>’</dt> <dd> MPEG2 Digital HDTV service. </dd> <dt>‘<samp class=”samp”>advanced_codec_digital_sdtv</sam <dd> Advanced Codec Digital SDTV service. </dd> <dt>‘<samp class=”samp”>advanced_codec_digital_hdtv</samp>’</dt> <dd> Advanced Codec Digital HDTV service. </dd> <dt>‘<samp class=”samp”>hevc_digital_hdtv</samp>’</dt> <dd> HEVC Digital Television service. </dd> </dl>

</dd> <dt><samp class=”option”>mpegts_pmt_start_pid <var class=”var”>integer</var></samp></d <dd> Set the first PID for PMTs. Default is <code class=”code”>0x1000</code>, minimum is <code class=”code”>0x0020</code>, maximum is <code class=”code”>0x1ffa</code>. This option has no effect in m2ts mode

where the PMT PID is fixed <code class=”code”>0x0100</code>. </dd> <dt><samp class=”option”>mpegts_start_pid <var class=”var”>integer</var></samp></dt> <dd> Set the first PID for elementary streams. Default is <code class=”code”>0x0100</code>, minimum is <code class=”code”>0x0020</code>, maximum is <code class=”code”>0x1ffa</code>. This option has no effect in m2ts mode where the elementary stream PIDs are fixed. </dd> <dt><samp class=”option”>mpegts_m2ts_mode <var class=”var”>boolean</var></samp></dt> <dd> Enable m2ts mode if set to <code class=”code”>1</code>. Default value is <code class=”code”>-1</code> which disables m2ts mode. </dd> <dt><samp class=”option”>muxrate <var class=”var”>integer</var></samp></dt> <dd> Set a constant muxrate. Default is VBR. </dd> <dt><samp class=”option”>pes_payload_size <var class=”var”>integer</var></samp></dt> <dd> Set minimum PES packet payload in bytes. Default is <code class=”code”>2930</code>. </dd> <dt><samp class=”option”>mpegts_flags <var class=”var”>flags</var></samp></dt> <dd> Set mpegts flags. Accepts the following options: <dl class=”table”> <dt>‘<samp class=”samp”>resend_headers</samp>’</dt> <dd> Reemit PAT/PMT before writing the next packet. </dd> <dt>‘<samp class=”samp”>latm</samp>’</dt> <dd> Use LATM packetization for AAC. </dd> <dt>‘<samp class=”samp”>pat_pmt_at_frames</samp>’</dt> <dd> Reemit PAT and PMT at each video frame. </dd> <dt>‘<samp class=”samp”>system_b</samp>’</dt> <dd> Conform to System B (DVB) instead of System A (ATSC). </dd> <dt>‘<samp class=”samp”>initial_discontinuity</samp>’</dt> <dd> Mark the initial packet of each stream as discontinuity. </dd> <dt>‘<samp class=”samp”>nit</samp>’</dt> <dd> Emit NIT table. </dd> <dt>‘<samp class=”samp”>omit_rai</samp>’</dt> <dd> Disable writing of random access indicator. </dd> </dl>

</dd> <dt><samp class=”option”>mpegts_copyts <var class=”var”>boolean</var></samp></dt> <dd> Preserve original timestamps, if value is set to <code class=”code”>1</code>. Default value is <code class=”code”>-1</code>, which results in shifting timestamps so that they start from 0. </dd> <dt><samp class=”option”>omit_video_pes_length <var class=”var”>boolean</var></samp></dt> <dd> Omit the PES packet length for video packets. Default is <code class=”code”>1</code> (true). </dd> <dt><samp class=”option”>pcr_period <var class=”var”>integer</var></samp></dt> <dd> Override the default PCR retransmission time in milliseconds. Default is <code class=”code”>-1</code> which means that the PCR interval will be determined automatically: 20 ms is used for CBR streams, the highest multiple of the frame duration which is less than 100 ms is used for VBR streams. </dd> <dt><samp class=”option”>mpegts_pcr_pid <var class=”var”>integer</var></samp></dt> <dd> Set a separate PID for PCR packets. When set to a valid PID value, PCR is carried in adaptation-field-only packets on the specified PID, separate from the elementary stream

PIDs. This is required for example by ARIB TR-B14. Default is <code class="code">-1</code> which means PCR is placed on the elementary stream PIDs. </dd> <dt><samp class="option">pat_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between PAT/PMT tables. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">sdt_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between SDT tables. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">nit_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between NIT tables. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">tables_version <var class="var">integer</var></samp></dt> <dd> Set PAT, PMT, SDT and NIT version (default <code class="code">0</code>, valid values are from 0 to 31, inclusively). This option allows updating stream structure so that standard consumer may detect the change. To do so, reopen output <code class="code">AVFormatContext</code> (in case of API usage) or restart <code class="command">ffmpeg</code> instance, cyclically changing <samp class="option">tables_version</samp> value: <pre class="example-preformatted">ffmpeg -i source1.ts -codec copy -f mpegts -tables_version 0 udp://1.1.1.1:1111 ffmpeg -i source2.ts -codec copy -f mpegts -tables_version 1 udp://1.1.1.1:1111 ... ffmpeg -i source3.ts -codec copy -f mpegts -tables_version 31 udp://1.1.1.1:1111 ffmpeg -i source1.ts -codec copy -f mpegts -tables_version 0 udp://1.1.1.1:1111 ffmpeg -i source2.ts -codec copy -f mpegts -tables_version 1 udp://1.1.1.1:1111 ... </pre> </dd> </dl>

 <h4 class="subsection">21.67.2 Example# TOC</h4>

 <pre class="example-preformatted">ffmpeg -i file.mpg -c copy \ -mpegts_original_network_id 0x1122 \ -mpegts_transport_stream_id 0x3344 \ -mpegts_service_id 0x5566 \ -mpegts_pmt_start_pid 0x1500 \ -mpegts_start_pid 0x150 \ -metadata service_provider="Some provider" \ -metadata service_name="Some Channel" \ out.ts </pre>

 <h3 class="section">21.68 mxf, mxf_d10, mxf_opatom# TOC</h

MXF muxer. <h4 class="subsection">21.68.1 Options# TOC</h4>

The muxer options are: <dl class="table"> <dt><samp class="option">store_user_comments <var class="var">bool</var></samp></dt> <dd> Set if user comments should be stored if available or never. IRT D-10 does not allow user comments. The default is thus to write them for mxf and mxf_opatom but not for mxf_d10 </dd> </dl>

 <h3 class="section">21.69 null# TOC</h3>

Null muxer. This muxer does not generate any output file, it is mainly useful for testing or benchmarking purposes. For example to benchmark decoding with <code class="command">ffmpeg</code> you can use the command: <pre class="example-preformatted">ffmpeg -benchmark -i INPUT -f null out.null </pre>

Note that the above command does not read or write the <samp class="file">out.null</samp> file, but specifying the output file is required by the <code class="command">ffmpeg</code> syntax. Alternatively you can write the command as: <pre class="example-preformatted">ffmpeg -benchmark -i INPUT -f null - </pre>

 <h3 class="section">21.70 nut# TOC</h3>

<dl class="table"> <dt><samp class="option">-syncpoints <var class="var">flags</var></samp></dt> <dd> Change the syncpoint usage in nut: <dl class="table"> <dt><samp class="option"><var class="var">default</var> use the normal low-overhead seeking aids.</samp></dt> <dt><samp class="option"><var class="var">none</var> do not use the syncpoints at all, reducing the overhead but making the stream non-seekable;</samp></dt> <dd> Use of this option is not recommended, as the resulting files are very damage sensitive and seeking is not possible. Also in general the overhead from syncpoints is negligible. Note, -<code class="code">write_index</code> 0 can be used to disable all growing data tables, allowing to mux endless streams with limited memory and without these disadvantages. </dd> <dt><samp class="option"><var class="var">timestamped</var> extend the syncpoint with a wallclock field.</samp></dt> </dl> The <var class="var">none</var> and <var class="var">timestamped</var> flags are experimental. </dd> <dt><samp class="option">-write_index <var class="var">bool</var></samp></dt> <dd> Write index at the end, the default is to write an index. </dd> </dl>

 <pre class="example-preformatted">ffmpeg -i INPUT -f_strict experimental -syncpoints none - | processor </pre>

 <h3 class="section">21.71 ogg<span class="pull-

right”>#
TOC</h3>

Ogg container muxer. <dl class=”table”> <dt><samp class=”option”>-page_duration <var class=”var”>duration</var></samp></dt>
<dd> Preferred page duration, in microseconds. The muxer will attempt to create pages that are approximately <var class=”var”>duration</var> microseconds long. This allows the user to compromise between seek granularity and container overhead. The default is 1 second. A value of 0 will fill all segments, making pages as large as possible. A value of 1 will effectively use 1 packet-per-page in most situations, giving a small seek granularity at the cost of additional container overhead. </dd> <dt><samp class=”option”>-serial_offset <var class=”var”>value</var></samp></dt>
<dd> Serial number of the first stream in bitexact mode, the others following consecutively. Chained links need distinct serial numbers, which disjoint ranges of values guarantee. The default of -1 derives each serial number from the stream’s headers instead. This makes links with different headers or tags unlikely to share a serial number, without ruling it out. Links with identical headers and tags always share one, which separately muxed outputs avoid with explicit offsets. A link the muxer starts itself on new stream metadata keeps the offset of the link before it. With an explicit offset it then reuses its serial number. With the default it does so when its headers and tags are unchanged. </dd> </dl>

 <h3 class=”section”>21.72 pdv#
TOC</h3>

Playdate Video muxer. This muxer writes the Playdate video container used by Panic’s Playdate SDK. It requires a seekable output and a single PDV video stream. <dl class=”table”> <dt><samp class=”option”>max_frames <var class=”var”>frames</var></samp></dt>
<dd> Reserve space for at most <var class=”var”>frames</var> video frames in the file header. This option is mandatory. </dd> </dl>

 <h3 class=”section”>21.73 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. It can be used to archive the original extracted CC bitstream and to produce a source file for later processing or conversion. The format allows for interoperability between ccextractor and FFmpeg, is simple to parse, and can be used to create a backup of the CC presentation. This muxer implements the specification as of March 2024, which has been stable and unchanged since April 2014. This muxer will have some nuances

from the way that ccextractor muxes RCWT. No compatibility issues when processing the output with ccextractor have been observed as a result of this so far, but mileage may vary and outputs will not be a bit-exact match. A free specification of RCWT can be found here: h
  <h4 class="subsection">21.73.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract Closed Captions to RCWT using lavfi: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=INPUT.mkv[out+subcc]" -map 0:s:0 -c:s copy -f rcwt CC.rcwt.bin </pre>

 <h3 class="section">21.74 segment, stream_segment, ssegment# TOC</h3>

Basic stream segmenter. This muxer outputs streams to a number of separate files of nearly fixed duration. Output filename pattern can be set in a fashion similar to image2, or by using a <code class="code">strftime</code> template if the <samp class="option">strftime</samp> option is enabled. <code class="code">stream_segment</code> is a variant of the muxer used to write to streaming output formats, i.e. which do not require global headers, and is recommended for outputting e.g. to MPEG transport stream segments. <code class="code">ssegment</code> is a shorter alias for <code class="code">stream_segment</code>. Every segment starts with a keyframe of the selected reference stream, which is set through the <samp class="option">reference_stream</samp> option. Note that if you want accurate splitting for a video file, you need to make the input key frames correspond to the exact splitting times expected by the segmenter, or the segment muxer will start the new segment with the key frame found next after the specified start time. The segment muxer works best with a single constant frame rate video. Optionally it can generate a list of the created segments, by setting the option <var class="var">segment_list</var>. The list type is specified by the <var class="var">segment_list_type</var> option. The entry filenames in the segment list are set by default to the basename of the corresponding segment files. See also the hls muxer, which provides a more specific implementation for HLS segmentation. <h4 class="subsection">21.74.1 Options<a class="anchor hidden-xs" href="#Options-95" aria-

hidden=”true”># TOC</h4>

The segment muxer supports the following options: <dl class=”table”> <dt><samp class=”option”>increment_tc <var class=”var”>1|0</var></samp></dt> <dd> if set to <code class=”code”>1</code>, increment timecode between each segment If this is selected, the input need to have a timecode in the first video stream. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>reference_stream <var class=”var”>specifier</var></samp></dt> <dd> Set the reference stream, as specified by the string <var class=”var”>specifier</var>. If <var class=”var”>specifier</var> is set to <code class=”code”>auto</code>, the reference is chosen automatically. Otherwise it must be a stream specifier (see the “Stream specifiers” chapter in the ffmpeg manual) which specifies the reference stream. The default value is <code class=”code”>auto</code>. </dd> <dt><samp class=”option”>segment_format <var class=”var”>format</var></samp></dt> <dd> Override the inner container format, by default it is guessed by the filename extension. </dd> <dt><samp class=”option”>segment_format_options <var class=”var”>options_list</var></samp></dt> <dd> Set output format options using a :-separated list of key=value parameters. Values containing the <code class=”code”>:</code> special character must be escaped. </dd> <dt><samp class=”option”>segment_list <var class=”var”>name</var></samp></dt> <dd> Generate also a listfile named <var class=”var”>name</var>. If not specified no listfile is generated. </dd> <dt><samp class=”option”>segment_list_flags <var class=”var”>flags</var></samp></dt> <dd> Set flags affecting the segment list generation. It currently supports the following flags: <dl class=”table”> <dt>‘<samp class=”samp”>cache</samp>’</dt> <dd> Allow caching (only affects M3U8 list files). </dd> <dt>‘<samp class=”samp”>live</samp>’</dt> <dd> Allow live-friendly file generation. </dd> </dl>

</dd> <dt><samp class=”option”>segment_list_size <var class=”var”>size</var></samp></dt> <dd> Update the list file so that it contains at most <var class=”var”>size</var> segments. If 0 the list file will contain all the segments. Default value is 0. </dd> <dt><samp class=”option”>segment_list_entry_prefix <var class=”var”>prefix</var></samp></dt> <dd> Prepend <var class=”var”>prefix</var> to each entry. Useful to generate absolute paths. By default no prefix is applied. </dd> <dt><samp class=”option”>segment_list_type <var class=”var”>type</var></samp></dt> <dd> Select the listing format. The following values are recognized: <dl class=”table”> <dt>‘<samp class=”samp”>flat</samp>’</dt> <dd> Generate a flat list for the created segments, one segment per line. </dd> <dt>‘<samp class=”samp”>csv, ext</samp>’</dt> <dd> Generate a list for the created segments, one segment per line, each line matching the format (comma-separated values): <pre class=”example-preformatted”><var class=”var”>segment_filename</var>,<var

class=”var”>segment_start_time</var>,<var class=”var”>segment_end_time</var>
</pre>

<var class=”var”>segment_filename</var> is the name of the output file generated by the muxer according to the provided pattern. CSV escaping (according to RFC4180) is applied if required. <var class=”var”>segment_start_time</var> and <var class=”var”>segment_end_time</var> specify the segment start and end time expressed in seconds. A list file with the suffix <code class=”code”>".csv"</code> or <code class=”code”>".ext"</code> will auto-select this format. ‘<samp class=”samp”>ext</samp>’ is deprecated in favor or ‘<samp class=”samp”>csv</samp>’. </dd> <dt>‘<samp class=”samp”>ffconcat</samp>’</dt> <dd> Generate an ffconcat file for the created segments. The resulting file can be read using the FFmpeg concat demuxer. A list file with the suffix <code class=”code”>".ffcat"</code> or <code class=”code”>".ffconcat"</code> will auto-select this format. </dd> <dt>‘<samp class=”samp”>m3u8</samp>’</dt> <dd> Generate an extended M3U8 file, version 3, compliant with http://tools.ietf.org/id/draft-pantos-http-live-streaming. A list file with the suffix <code class=”code”>".m3u8"</code> will auto-select this format. </dd> </dl>

If not specified the type is guessed from the list file name suffix. </dd> <dt><samp class=”option”>segment_time <var class=”var”>time</var></samp></dt> <dd> Set segment duration to <var class=”var”>time</var>, the value must be a duration specification. Default value is "2". See also the <samp class=”option”>segment_times</samp> option. Note that splitting may not be accurate, unless you force the reference stream keyframes at the given time. See the introductory notice and the examples below. </dd> <dt><samp class=”option”>min_seg_duration <var class=”var”>time</var></samp></dt> <dd> Set minimum segment duration to <var class=”var”>time</var>, the value must be a duration specification. This prevents the muxer ending segments at a duration below this value. Only effective with <code class=”code”>segment_time</code>. Default value is "0". </dd> <dt><samp class=”option”>segment_atclocktime <var class=”var”>1|0</var></samp></dt> <dd> If set to "1" split at regular clock time intervals starting from 00:00 o’clock. The <var class=”var”>time</var> value specified in <samp class=”option”>segment_time</samp> is used for setting the length of the splitting interval. For example with <samp class=”option”>segment_time</samp> set to "900" this makes it possible to create files at 12:00 o’clock, 12:15, 12:30, etc. Default value is "0". </dd> <dt><samp class=”option”>segment_clocktime_offset <var class=”var”>duration</var></samp></dt> <dd> Delay the seg-

ment splitting times with the specified duration when using <samp class=”option”>segment_atclocktime</samp>. For example with <samp class=”option”>segment_time</samp> set to "900" and <samp class=”option”>segment_clocktime_offset</samp> set to "300" this makes it possible to create files at 12:05, 12:20, 12:35, etc. Default value is "0". </dd> <dt><samp class=”option”>segment_clocktime_wrap_duration <var class=”var”>duration</var></samp></dt> <dd> Force the segmenter to only start a new segment if a packet reaches the muxer within the specified duration after the segmenting clock time. This way you can make the segmenter more resilient to backward local time jumps, such as leap seconds or transition to standard time from daylight savings time. Default is the maximum possible duration which means starting a new segment regardless of the elapsed time since the last clock time. </dd> <dt><samp class=”option”>segment_time_delta <var class=”var”>delta</var></samp></dt> <dd> Specify the accuracy time when selecting the start time for a segment, expressed as a duration specification. Default value is "0". When delta is specified a key-frame will start a new segment if its PTS satisfies the relation: <pre class=”example-preformatted”>PTS >= start_time - time_delta </pre>

This option is useful when splitting video content, which is always split at GOP boundaries, in case a key frame is found just before the specified split time. In particular may be used in combination with the <samp class=”file”>ffmpeg</samp> option <var class=”var”>force_key_frames</var>. The key frame times specified by <var class=”var”>force_key_frames</var> may not be set accurately because of rounding issues, with the consequence that a key frame time may result set just before the specified time. For constant frame rate videos a value of 1/(2*<var class=”var”>frame_rate</var>) should address the worst case mismatch between the specified time and the time set by <var class=”var”>force_key_frames</var>. </dd> <dt><samp class=”option”>segment_times <var class=”var”>times</var></samp></dt> <dd> Specify a list of split points. <var class=”var”>times</var> contains a list of comma separated duration specifications, in increasing order. See also the <samp class=”option”>segment_time</samp> option. </dd> <dt><samp class=”option”>segment_frames <var class=”var”>frames</var></samp></dt> <dd> Specify a list of split video frame numbers. <var class=”var”>frames</var> contains a list of comma separated integer numbers, in increasing order. This option specifies to start a new segment whenever a reference stream key frame is found and the sequential number (starting from 0) of the frame is greater or equal to the next value in the list. </dd> <dt><samp class=”option”>segment_wrap <var class=”var”>limit</var></samp></dt> <dd> Wrap around segment in-

dex once it reaches <var class=”var”>limit</var>. </dd> <dt><samp class=”option”>segment_start_number <var class=”var”>number</var></samp></dt> <dd> Set the sequence number of the first segment. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>segment_wrap_number <var class=”var”>number</var></samp></dt> <dd> Set the number of times the segment index is assumed to have already wrapped when generating the indexes reported in the segment list. Together with <samp class=”option”>segment_wrap</samp> and <samp class=”option”>segment_start_number</samp> it offsets the reported index, which is computed as <code class=”code”>segment_start_number + segment_wrap * segment_wrap_number</code>. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>strftime <var class=”var”>1|0</var></samp></dt> <dd> Use the <code class=”code”>strftime</code> function to define the name of the new segments to write. If this is selected, the output segment name must contain a <code class=”code”>strftime</code> function template. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>break_non_keyframes <var class=”var”>1|0</var></samp></dt> <dd> If enabled, allow segments to start on frames other than keyframes. This improves behavior on some players when the time between keyframes is inconsistent, but may make things worse on others, and can cause some oddities during seeking. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>reset_timestamps <var class=”var”>1|0</var></samp></dt> <dd> Reset timestamps at the beginning of each segment, so that each segment will start with near-zero timestamps. It is meant to ease the playback of the generated segments. May not work with some combinations of muxers/codecs. It is set to <code class=”code”>0</code> by default. </dd> <dt><samp class=”option”>initial_offset <var class=”var”>offset</var></samp></dt> <dd> Specify timestamp offset to apply to the output packet timestamps. The argument must be a time duration specification, and defaults to 0. </dd> <dt><samp class=”option”>write_empty_segments <var class=”var”>1|0</var></samp></dt> <dd> If enabled, write an empty segment if there are no packets during the period a segment would usually span. Otherwise, the segment will be filled with the next packet written. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>write_header_trailer <var class=”var”>bool</var></samp></dt> <dd> Write a header to the first segment and a trailer to the last one, instead of writing a header and a trailer to every individual segment. Disabling it (<code class=”code”>false</code>) also forces <samp class=”option”>individual_header_trailer</samp> to <code class=”code”>false</code>, so that no segment is given a header or trailer. Default value is <code class=”code”>true</code>. </dd> <dt><samp class=”option”>individual_header_trailer <var class=”var”>bool</var></samp></dt> <dd> If enabled, write a complete header and trailer to every segment, making each segment an independently usable file. If disabled (<code class=”code”>false</code>), only the first segment is given a header and only the last one a trailer. It is forced to

<code class="code">false</code> when <samp class="option">write_header_trailer</samp> is <code class="code">false</code> or when <samp class="option">segment_header_filename</samp> is set. Default value is <code class="code">true</code>. </dd> <dt><samp class="option">segment_header_filename <var class="var">name</var></samp></dt> <dd> Write the stream header to a separate file named <var class="var">name</var> instead of storing it in the segments. Setting it forces <samp class="option">write_header_trailer</samp> to <code class="code">true</code> and <samp class="option">individual_header_trailer</samp> to <code class="code">false</code>, so the header is written only once, to <var class="var">name</var>. Unset by default. </dd> </dl>

Make sure to require a closed GOP when encoding and to set the GOP size to fit your segment time constraint. <h4 class="subsection">21.74.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Remux the content of file <samp class="file">in.mkv</samp> to a list of segments <samp class="file">out-000.nut</samp>, <samp class="file">out-001.nut</samp>, etc., and write the list of generated segments to <samp class="file">out.list</samp>: <pre class="example-preformatted">ffmpeg -i in.mkv -codec hevc -flags +cgop -g 60 -map 0 -f segment -segment_list out.list out%03d.nut </pre>

Segment input and set output format options for the output segments: <pre class="example-preformatted">ffmpeg -i in.mkv -f segment -segment_time 10 -segment_format_options movflags=+faststart out%03d.mp4 </pre>

Segment the input file according to the split points specified by the <var class="var">segment_times</var> option: <pre class="example-preformatted">ffmpeg -i in.mkv -codec copy -map 0 -f segment -segment_list out.csv -segment_times 1,2,3,5,8,13,21 out%03d.nut </pre>

Use the <code class="command">ffmpeg</code> <samp class="option">force_key_frames</samp> option to force key frames in the input at the specified location, together with the segment option <samp class="option">segment_time_delta</samp> to account for possible roundings operated when setting key frame times. <pre class="example-preformatted">ffmpeg -i in.mkv -force_key_frames 1,2,3,5,8,13,21 -codec:v mpeg4 -codec:a pcm_s16le -map 0 \ -f segment -segment_list out.csv -segment_times 1,2,3,5,8,13,21 -segment_time_delta 0.05 out%03d.nut </pre> In order to force key frames on the input file, transcoding is required. Segment the input file by splitting the input file according to the frame numbers sequence specified with the <samp class="option">segment_frames</samp> option:

<pre class="example-preformatted">ffmpeg -i in.mkv -codec copy -map 0 -f segment -segment_list out.csv -segment_frames 100,200,300,500,800 out%03d.nut </pre>

Convert the <samp class="file">in.mkv</samp> to TS segments using the <code class="code">libx264</code> and <code class="code">aac</code> encoders: <pre class="example-preformatted">ffmpeg -i in.mkv -map 0 -codec:v libx264 -codec:a aac -f ssegment -segment_list out.list out%03d.ts </pre>

Segment the input file, and create an M3U8 live playlist (can be used as live HLS source): <pre class="example-preformatted">ffmpeg -re -i in.mkv -codec copy -map 0 -f segment -segment_list playlist.m3u8 \ -segment_list_flags +live -segment_time 10 out%03d.mkv </pre>

 <h3 class="section">21.75 smoothstreaming# TOC</h3>

Smooth Streaming muxer generates a set of files (Manifest, chunks) suitable for serving with conventional web server. <dl class="table"> <dt><samp class="option">window_size</samp></dt> <dd> Specify the number of fragments kept in the manifest. Default 0 (keep all). </dd> <dt><samp class="option">extra_window_size</samp></dt> <dd> Specify the number of fragments kept outside of the manifest before removing from disk. Default 5. </dd> <dt><samp class="option">lookahead_count</samp></dt> <dd> Specify the number of lookahead fragments. Default 2. </dd> <dt><samp class="option">min_frag_duration</samp></dt> <dd> Specify the minimum fragment duration (in microseconds). Default 5000000. </dd> <dt><samp class="option">remove_at_exit</samp></dt> <dd> Specify whether to remove all fragments when finished. Default 0 (do not remove). </dd> </dl>

 <h3 class="section">21.76 streamhash# TOC</h3>

Per stream hash testing format. This muxer computes and prints a cryptographic hash of all the input frames, on a per-stream basis. This can be used for equality checks without having to do a complete binary comparison. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the hash, but the output of explicit conversions to other codecs can also be used. Timestamps are ignored. It uses the SHA-256 cryptographic hash function by default, but supports several other algorithms. The

output of the muxer consists of one line per stream of the form: <var class="var">streamindex</var>,<var class="var">streamtype</var>,<var class="var">algo</var>=<var class="var">hash</var>, where <var class="var">streamindex</var> is the index of the mapped stream, <var class="var">streamtype</var> is a single character indicating the type of stream, <var class="var">algo</var> is a short string representing the hash function used, and <var class="var">hash</var> is a hexadecimal number representing the computed hash. <dl class="table"> <dt><samp class="option">hash <var class="var">algorithm</var></samp></dt> <dd> Use the cryptographic hash function specified by the string <var class="var">algorithm</var>. Supported values include <code class="code">MD5</code>, <code class="code">murmur3</code>, <code class="code">RIPEMD128</code>, <code class="code">RIPEMD160</code>, <code class="code">RIPEMD256</code>, <code class="code">RIPEMD320</code>, <code class="code">SHA160</code>, <code class="code">SHA224</code>, <code class="code">SHA256</code> (default), <code class="code">SHA512/224</code>, <code class="code">SHA512/256</code>, <code class="code">SHA384</code>, <code class="code">SHA512</code>, <code class="code">CRC32</code> and <code class="code">adler32</code>. </dd> </dl>

 <h4 class="subsection">21.76.1 Examples# TOC</h4>

To compute the SHA-256 hash of the input converted to raw audio and video, and store it in the file <samp class="file">out.sha256</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f streamhash out.sha256 </pre>

To print an MD5 hash to stdout use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f streamhash -hash md5 - </pre>

See also the hash and framehash muxers. <h3 class="section">21.77 tee# TOC</h3>

The tee muxer can be used to write the same data to several outputs, such as files or streams. It can be used, for example, to stream a video over a network and save it to disk at the same time. It is different from specifying several outputs to the <code class="command">ffmpeg</code> command-line tool. With the tee muxer, the audio and video data will be encoded only once. With conventional multiple outputs, multiple encoding operations in parallel are initiated, which can be a very expensive process.

The tee muxer is not useful when using the libavformat API directly because it is then possible to feed the same packets to several muxers directly. Since the tee muxer does not represent any particular output format, ffmpeg cannot auto-select output streams. So all streams intended for output must be specified using <code class="code">-map</code>. See the examples below. Some encoders may need different options depending on the output format; the auto-detection of this can not work with the tee muxer, so they need to be explicitly specified. The main example is the <samp class="option">global_header</samp> flag. The slave outputs are specified in the file name given to the muxer, separated by ’|’. If any of the slave name contains the ’|’ separator, leading or trailing spaces or any special character, those must be escaped (see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual). <h4 class="subsection">21.77.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">use_fifo <var class="var">bool</var></samp></dt> <dd> If set to 1, slave outputs will be processed in separate threads using the fifo muxer. This allows to compensate for different speed/latency/reliability of outputs and setup transparent recovery. By default this feature is turned off. </dd> <dt><samp class="option">fifo_options</samp></dt> <dd> Options to pass to fifo pseudo-muxer instances. See fifo. </dd> </dl>

Muxer options can be specified for each slave by prepending them as a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ’:’, between square brackets. If the options values contain a special character or the ’:’ separator, they must be escaped; note that this is a second level escaping. The following special options are also recognized: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Specify the format name. Required if it cannot be guessed from the output URL. </dd> <dt><samp class="option">bsfs[/<var class="var">spec</var>]</samp></dt> <dd> Specify a list of bitstream filters to apply to the specified output. It is possible to specify to which streams a given bitstream filter applies, by appending a stream specifier to the option separated by <code class="code">/</code>. <var class="var">spec</var> must be a stream specifier (see Format stream specifiers). If the stream specifier is not specified, the bitstream filters will be applied to all streams in the output. This will cause that output operation to fail if the output contains streams to which the bitstream filter cannot be applied e.g. <code

class=”code”>h264_mp4toannexb</code> being applied to an output containing an audio stream. Options for a bitstream filter must be specified in the form of <code class=”code”>opt=value</code>. Several bitstream filters can be specified, separated by ",". </dd> <dt><samp class=”option”>use_fifo <var class=”var”>bool</var></samp></dt> <dd> This allows to override tee muxer use_fifo option for individual slave muxer. </dd> <dt><samp class=”option”>fifo_options</samp></dt> <dd> This allows to override tee muxer fifo_options for individual slave muxer. See fifo. </dd> <dt><samp class=”option”>select</samp></dt> <dd> Select the streams that should be mapped to the slave output, specified by a stream specifier. If not specified, this defaults to all the mapped streams. This will cause that output operation to fail if the output format does not accept all mapped streams. You may use multiple stream specifiers separated by commas (<code class=”code”>,</code>) e.g.: <code class=”code”>a:0,v</code> </dd> <dt><samp class=”option”>onfail</samp></dt> <dd> Specify behaviour on output failure. This can be set to either <code class=”code”>abort</code> (which is default) or <code class=”code”>ignore</code>. <code class=”code”>abort</code> will cause whole process to fail in case of failure on this slave output. <code class=”code”>ignore</code> will ignore failure on this output, so other outputs will continue without being affected. </dd> </dl>

 <h4 class=”subsection”>21.77.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Encode something and both archive it in a WebM file and stream it as MPEG-TS over UDP: <pre class=”example-preformatted”>ffmpeg -i ... -c:v libx264 -c:a mp2 -f tee -map 0:v -map 0:a "archive-20121107.mkv|[f=mpegts]udp://10.0.1.255:1234/" </pre>

As above, but continue streaming even if output to local file fails (for example local drive fills up): <pre class=”example-preformatted”>ffmpeg -i ... -c:v libx264 -c:a mp2 -f tee -map 0:v -map 0:a "[onfail=ignore]archive-20121107.mkv|[f=mpegts]udp://10.0.1.255:1234/" </pre>

Use <code class=”command”>ffmpeg</code> to encode the input, and send the output to three different destinations. The <code class=”code”>dump_extra</code> bitstream filter is used to add extradata information to all the output video keyframes packets, as requested by the MPEG-TS format. The select option is applied to <samp class=”file”>out.aac</samp> in order to make it contain only audio packets. <pre class=”example-preformatted”>ffmpeg -i ... -map 0 -flags +global_header -c:v libx264 -c:a aac -f tee "[bsfs/v=dump_extra=freq=keyframe]out.ts|[movflags=+faststart]out.mp4|[select=

</pre>

As above, but select only stream <code class="code">a:1</code> for the audio output. Note that a second level escaping must be performed, as ":" is a special character used to separate options. <pre class="example-preformatted">ffmpeg -i ... -map 0 -flags +global_header -c:v libx264 -c:a aac -f tee "[bsfs/v=dump_extra=freq=keyframe]out.ts|[movflags=-
</pre>

 <h3 class="section">21.78 wav# TOC</h3>

RIFF Wave Audio muxer. <h4 class="subsection">21.78.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">rf64 <var class="var">mode</var></samp></dt> <dd> Choose whether to use the RF64 file format instead of RIFF. RF64 allows files larger than 4 gigabytes, but RF64 support is not as universal as RIFF. The following modes are recognized: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Start writing a standard RIFF file to retain compatibility with non-RF64-aware software, but switch to RF64 if the output file grows larger than 4 gigabytes. Reserves a small amount of extra space in the header using a <code class="code">JUNK</code> chunk, which should be ignored by all compliant RIFF readers, but some extremely simplistic WAV readers may be confused by this. </dd> <dt>‘<samp class="samp">always</samp>’</dt> <dd> Always use the RF64 format regardless of file size. The output file will only be readable by RF64-compatible software and is no longer a standard RIFF WAVE file. </dd> <dt>‘<samp class="samp">never</samp>’</dt> <dd> Always use plain RIFF, never RF64. This mode is the most compatible with legacy software, but if the output file grows beyond 4 gigabytes, the 32-bit size field in the <code class="code">data</code> chunk will no longer be able to correctly represent the size, and most audio software will not be able to read some or all of the data. (The wav demuxer’s <code class="code">ignore_length</code> option can be used to recover such a file.) This is the default mode. </dd> </dl>

</dd> <dt><samp class="option">write_bext <var class="var">bool</var></samp></dt> <dd> Add a Broadcast Wave Format <code class="code">BEXT</code> chunk containing extended audio metadata if enabled. Defaults to false. </dd> <dt><samp class="option">write_peak <var class="var">mode</var></samp></dt> <dd> Add a Peak Envelope chunk (EBU Tech 3285 Supplement 3) if enabled. Defaults to off. <dl class="table"> <dt>‘<samp class="samp">off</samp>’</dt>

<dd> Do not add the Peak Envelope chunk. </dd> <dt>‘<samp class="samp">on</samp>’</dt> <dd> Add the Peak Envelope chunk, calculating the peak values based on the <code class="code">peak_block_size</code>, <code class="code">peak_format</code>, and <code class="code">peak_ppv</code> options. </dd> <dt>‘<samp class="samp">only</samp>’</dt> <dd> Like <code class="code">on</code>, but skip writing the actual audio data (no <code class="code">data</code> chunk will be written to the file; only the Peak Envelope and other metadata chunks will be written.) </dd> </dl>

</dd> <dt><samp class="option">peak_block_size <var class="var">int</var></samp></dt> <dd> Number of audio samples used to generate each peak frame, up to 65536. Default is 256. </dd> <dt><samp class="option">peak_format <var class="var">format</var></samp></dt> <dd> The format of the peak envelope data (1: uint8, 2: uint16). Default is 2 (uint16). </dd> <dt><samp class="option">peak_ppv <var class="var">int</var></samp></dt> <dd> Number of peak points per peak value (1 or 2). Default is 2. </dd> </dl>

 <h3 class="section">21.79 webm_chunk# TOC</h3>

WebM Live Chunk Muxer. This muxer writes out WebM headers and chunks as separate files which can be consumed by clients that support WebM Live streams via DASH. <h4 class="subsection">21.79.1 Options# TOC</h4>

This muxer supports the following options: <dl class="table"> <dt><samp class="option">chunk_start_index</samp></dt> <dd> Index of the first chunk (defaults to 0). </dd> <dt><samp class="option">header</samp></dt> <dd> Filename of the header where the initialization data will be written. </dd> <dt><samp class="option">audio_chunk_duration</samp></dt> <dd> Duration of each audio chunk in milliseconds (defaults to 5000). </dd> </dl>

 <h4 class="subsection">21.79.2 Example# TOC</h4> <pre class="example-preformatted">ffmpeg -f v4l2 -i /dev/video0 \ -f alsa -i hw:0 \ -map 0:0 \ -c:v libvpx-vp9 \ -s 640x360 -keyint_min 30 -g 30 \ -f webm_chunk \ -header webm_live_video_360.hdr \ -chunk_start_index 1 \ webm_live_video_360_%d.chk \ -map 1:0 \ -c:a libvorbis \ -b:a 128k \ -f

webm_chunk \ -header webm_live_audio_128.hdr \ -chunk_start_index 1 \ -audio_chunk_duration 1000 \ webm_live_audio_128_%d.chk </pre>

 <h3 class="section">21.80 webm_dash_manifest# TOC</h3>

WebM DASH Manifest muxer. This muxer implements the WebM DASH Manifest specification to generate the DASH manifest XML. It also supports manifest generation for DASH live streams. For more information see: <ul class="itemize mark-bullet"> WebM DASH Specification: https://sites.google.com/a/webmproject.org/wiki/adaptive-streaming/webm-dash-specification ISO DASH Specification: http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_IEC_23009-1_2014.zip

 <h4 class="subsection">21.80.1 Options# TOC</h4>

This muxer supports the following options: <dl class="table"> <dt><samp class="option">adaptation_sets</samp></dt> <dd> This option has the following syntax: "id=x,streams=a,b,c id=y,streams=d,e" where x and y are the unique identifiers of the adaptation sets and a,b,c,d and e are the indices of the corresponding audio and video streams. Any number of adaptation sets can be added using this option. </dd> <dt><samp class="option">live</samp></dt> <dd> Set this to 1 to create a live stream DASH Manifest. Default: 0. </dd> <dt><samp class="option">chunk_start_index</samp></dt> <dd> Start index of the first chunk. This will go in the ‘<samp class="samp">startNumber</samp>’ attribute of the ‘<samp class="samp">SegmentTemplate</samp>’ element in the manifest. Default: 0. </dd> <dt><samp class="option">chunk_duration_ms</samp></dt> <dd> Duration of each chunk in milliseconds. This will go in the ‘<samp class="samp">duration</samp>’ attribute of the ‘<samp class="samp">SegmentTemplate</samp>’ element in the manifest. Default: 1000. </dd> <dt><samp class="option">utc_timing_url</samp></dt> <dd> URL of the page that will return the UTC timestamp in ISO format. This will go in the ‘<samp class="samp">value</samp>’ attribute of the ‘<samp class="samp">UTCTiming</samp>’ element in the manifest. Default: None. </dd> <dt><samp

class=”option”>time_shift_buffer_depth</samp></dt> <dd> Smallest time (in seconds) shifting buffer for which any Representation is guaranteed to be available. This will go in the ‘<samp class=”samp”>timeShiftBufferDepth</samp>’ attribute of the ‘<samp class=”samp”>MPD</samp>’ element. Default: 60. </dd> <dt><samp class=”option”>minimum_update_period</samp></dt> <dd> Minimum update period (in seconds) of the manifest. This will go in the ‘<samp class=”samp”>minimumUpdatePeriod</samp>’ attribute of the ‘<samp class=”samp”>MPD</samp>’ element. Default: 0. </dd> </dl>

 <h4 class=”subsection”>21.80.2 Example# TOC</h4> <pre class=”example-preformatted”>ffmpeg -f webm_dash_manifest -i video1.webm \ -f webm_dash_manifest -i video2.webm \ -f webm_dash_manifest -i audio1.webm \ -f webm_dash_manifest -i audio2.webm \ -map 0 -map 1 -map 2 -map 3 \ -c copy \ -f webm_dash_manifest \ -adaptation_sets "id=0,streams=0,1 id=1,streams=2,3" \ manifest.xml </pre>

 <h3 class=”section”>21.81 whip# TOC</h3>

WebRTC (Real-Time Communication) muxer that supports sub-second latency streaming according to the WHIP (WebRTC-HTTP ingestion protocol) specification. This is an experimental feature. It uses HTTP as a signaling protocol to exchange SDP capabilities and ICE lite candidates. Then, it uses STUN binding requests and responses to establish a session over UDP. Subsequently, it initiates a DTLS handshake to exchange the SRTP encryption keys. Lastly, it splits video and audio frames into RTP packets and encrypts them using SRTP. Ensure that you use H.264 without B frames and Opus for the audio codec. For example, to convert an input file with <code class=”command”>ffmpeg</code> to WebRTC: <pre class=”example-preformatted”>ffmpeg -re -i input.mp4 -acodec libopus -ar 48000 -ac 2 \ -vcodec libx264 -profile:v baseline -tune zerolatency -threads 1 -bf 0 \ -f whip "http://localhost:1985/rtc/v1/whip/?app=live&stream=livestream" </pre>

For this example, we have employed low latency options, resulting in an end-to-end latency of approximately 150ms. <h4 class=”subsection”>21.81.1 Options# TOC</h4>

This muxer supports the following options: <dl class=”table”>

<dt><samp class="option">handshake_timeout <var class="var">integer</var></samp></dt> <dd> Set the timeout in milliseconds for ICE and DTLS handshake. Default value is 5000. </dd> <dt><samp class="option">timeout <var class="var">integer</var></samp></dt> <dd> Set timeout in seconds for socket I/O operations. Applicable only for HTTP output. Default value is -1. </dd> <dt><samp class="option">pkt_size <var class="var">integer</var></samp></dt> <dd> Set the maximum size, in bytes, of RTP packets that send out. Default value is 1200. </dd> <dt><samp class="option">ts_buffer_size <var class="var">integer</var></samp></dt> <dd> Set the buffer size, in bytes, of underlying protocol. Default value is -1(auto). The UDP auto selects a reasonable value. </dd> <dt><samp class="option">whip_flags <var class="var">flags</var></samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">dtls_active</samp>’</dt> <dd> The muxer will try to set dtls active role and send the first client hello. </dd> </dl>

</dd> <dt><samp class="option">rtp_history <var class="var">integer</var></samp></dt> <dd> Set the number of RTP history items to store. Default value is 512. </dd> <dt><samp class="option">authorization <var class="var">string</var></samp></dt> <dd> The optional Bearer token for WHIP Authorization. </dd> <dt><samp class="option">cert_file <var class="var">string</var></samp></dt> <dd> The optional certificate file path for DTLS. </dd> <dt><samp class="option">key_file <var class="var">string</var></samp></dt> <dd> The optional private key file path for DTLS. </dd> </dl>

 <h2 class="chapter">22 Metadata# TOC</h2>

FFmpeg is able to dump metadata from media files into a simple UTF-8-encoded INI-like text file and then load it back using the metadata muxer/demuxer. The file format is as follows: <ol class="enumerate"> A file consists of a header and a number of metadata tags divided into sections, each on its own line.

 The header is a ‘<samp class="samp">;FFMETADATA</samp>’ string, followed by a version number (now 1).

 Metadata tags are of the form ‘<samp class="samp">key=value</samp>’

 Immediately after header follows global metadata

 After global metadata there may be sections with per-stream/per-chapter metadata.

 A section starts with the section name in uppercase (i.e. STREAM or CHAPTER) in brackets (‘<samp class="samp">[</samp>’,

‘<samp class="samp">]</samp>’) and ends with next section or end of file.

 At the beginning of a chapter section there may be an optional timebase to be used for start/end values. It must be in form ‘<samp class="samp">TIMEBASE=<var class="var">num</var>/<var class="var">den</var></samp>’ where <var class="var">num</var> and <var class="var">den</var> are integers. If the timebase is missing then start/end times are assumed to be in nanoseconds.

Next a chapter section must contain chapter start and end times in form ‘<samp class="samp">START=<var class="var">num</var></samp>’, ‘<samp class="samp">END=<var class="var">num</var></samp>’, where <var class="var">num</var> is a positive integer. Empty lines and lines starting with ‘<samp class="samp">;</samp>’ or ‘<samp class="samp">#</samp>’ are ignored.

 Metadata keys or values containing special characters (‘<samp class="samp">=</samp>’, ‘<samp class="samp">;</samp>’, ‘<samp class="samp">#</samp>’, ‘<samp class="samp">\</samp>’ and a newline) must be escaped with a backslash ‘<samp class="samp">\</samp>’.

 Note that whitespace in metadata (e.g. ‘<samp class="samp">foo = bar</samp>’) is considered to be a part of the tag (in the example above key is ‘<samp class="samp">foo </samp>’, value is ‘<samp class="samp"> bar</samp>’).

A ffmetadata file might look like this: <pre class="example-preformatted">;FFMETADATA1 title=bike\\shed ;this is a comment artist=FFmpeg troll team

[CHAPTER] TIMEBASE=1/1000 START=0 #chapter ends at 0:01:00 END=60000 title=chapter \#1 [STREAM] title=multi\ line </pre>

By using the ffmetadata muxer and demuxer it is possible to extract metadata from an input file to an ffmetadata file, and then transcode the file into an output file with the edited ffmetadata file. Extracting an ffmetadata file with <samp class="file">ffmpeg</samp> goes as follows: <pre class="example-preformatted">ffmpeg -i INPUT -f ffmetadata FFMETADATAFILE </pre>

Reinserting edited metadata information from the FFMETADATAFILE file can be done as: <pre class="example-preformatted">ffmpeg -i INPUT -i FFMETADATAFILE -map_metadata 1 -codec copy OUTPUT </pre>

 <h2 class="chapter">23 Protocol Options<a class="anchor hidden-xs" href="#Protocol-

Options” aria-hidden=”true”># TOC</h2>

The libavformat library provides some generic global options, which can be set on all the protocols. In addition each protocol may support so-called private options, which are specific for that component. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class=”code”>AVFormatContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class=”table”> <dt><samp class=”option”>protocol_whitelist <var class=”var”>list</var> (input)</samp></dt> <dd> Set a ","-separated list of allowed protocols. "ALL" matches all protocols. Protocols prefixed by "-" are disabled. All protocols are allowed by default but protocols used by an another protocol (nested protocols) are restricted to a per protocol subset. </dd> </dl>

 <h2 class=”chapter”>24 Protocols# TOC</h2>

Protocols are configured elements in FFmpeg that enable access to resources that require specific protocols. When you configure your FFmpeg build, all the supported protocols are enabled by default. You can list all available ones using the configure option "–list-protocols". You can disable all the protocols using the configure option "–disable-protocols", and selectively enable a protocol using the option "–enable-protocol=<var class=”var”>PROTOCOL</var>", or you can disable a particular protocol using the option "–disable-protocol=<var class=”var”>PROTOCOL</var>". The option "-protocols" of the ff* tools will display the list of supported protocols. All protocols accept the following options: <dl class=”table”> <dt><samp class=”option”>rw_timeout</samp></dt> <dd> Maximum time to wait for (network) read/write operations to complete, in microseconds. </dd> </dl>

A description of the currently available protocols follows. <h3 class=”section”>24.1 amqp# TOC</h3>

Advanced Message Queueing Protocol (AMQP) version 0-9-1 is a broker based publish-subscribe communication protocol. FFmpeg must be compiled with –enable-librabbitmq to support AMQP. A separate AMQP broker must also be run. An example open-source AMQP

broker is RabbitMQ. After starting the broker, an FFmpeg client may stream data to the broker using the command: <pre class=”example-preformatted”>ffmpeg -re -i input -f mpegts amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

Where hostname and port (default is 5672) is the address of the broker. The client may also set a user/password for authentication. The default for both fields is "guest". Name of virtual host on broker can be set with vhost. The default value is "/". Multiple subscribers may stream from the broker using the command: <pre class=”example-preformatted”>ffplay amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

In RabbitMQ all data published to the broker flows through a specific exchange, and each subscribing client has an assigned queue/buffer. When a packet arrives at an exchange, it may be copied to a client’s queue depending on the exchange and routing_key fields. The following options are supported: <dl class=”table”> <dt><samp class=”option”>exchange</samp></dt> <dd> Sets the exchange to use on the broker. RabbitMQ has several predefined exchanges: "amq.direct" is the default exchange, where the publisher and subscriber must have a matching routing_key; "amq.fanout" is the same as a broadcast operation (i.e. the data is forwarded to all queues on the fanout exchange independent of the routing_key); and "amq.topic" is similar to "amq.direct", but allows for more complex pattern matching (refer to the RabbitMQ documentation). </dd> <dt><samp class=”option”>routing_key</samp></dt> <dd> Sets the routing key. The default value is "amqp". The routing key is used on the "amq.direct" and "amq.topic" exchanges to decide whether packets are written to the queue of a subscriber. </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Maximum size of each packet sent/received to the broker. Default is 131072. Minimum is 4096 and max is any large value (representable by an int). When receiving packets, this sets an internal buffer size in FFmpeg. It should be equal to or greater than the size of the published packets to the broker. Otherwise the received message may be truncated causing decoding errors. </dd> <dt><samp class=”option”>connection_timeout</samp></dt> <dd> The timeout in seconds during the initial connection to the broker. The default value is rw_timeout, or 5 seconds if rw_timeout is not set. </dd> <dt><samp class=”option”>delivery_mode <var class=”var”>mode</var></samp></dt> <dd> Sets the delivery mode of each message sent to broker. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>persistent</samp>’</dt> <dd> Delivery mode set to "persistent" (2). This is the default value. Messages may be written to the broker’s disk depending on its setup. </dd> <dt>‘<samp class=”samp”>non-persistent</samp>’</dt> <dd> Delivery mode set to "non-

persistent" (1). Messages will stay in broker’s memory unless the broker is under memory pressure. </dd> </dl>

</dd> </dl>

 <h3 class="section">24.2 async# TOC</h3>

Asynchronous data filling wrapper for input stream. Fill data in a background thread, to decouple I/O operation from demux thread.   <pre class="example-preformatted">async:<var class="var">URL</var> async:http://host/resource async:cache:http://host/resource </pre>

 <h3 class="section">24.3 bluray# TOC</h3>

Read BluRay playlist. The accepted options are: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> BluRay angle </dd> <dt><samp class="option">chapter</samp></dt> <dd> Start chapter (1...N) </dd> <dt><samp class="option">playlist</samp></dt> <dd> Playlist to read (BDMV/PLAYLIST/?????.mpls) </dd> </dl>

Examples: Read longest playlist from BluRay mounted to /mnt/bluray: <pre class="example-preformatted">bluray:/mnt/bluray </pre>

Read angle 2 of playlist 4 from BluRay mounted to /mnt/bluray, start from chapter 2: <pre class="example-preformatted">-playlist 4 -angle 2 -chapter 2 bluray:/mnt/bluray </pre>

 <h3 class="section">24.4 cache# TOC</h3>

Caching wrapper for input stream. Cache the input stream to temporary file. It brings seeking capability to live streams. The accepted options are: <dl class="table"> <dt><samp class="option">read_ahead_limit</samp></dt> <dd> Amount in bytes that may be read ahead when seeking isn’t supported. Range is -1 to INT_MAX. -1 for unlimited. Default is 65536. </dd> </dl>

URL Syntax is <pre class="example-preformatted">cache:<var class="var">URL</var> </pre>

 <h3 class="section">24.5 concat<a class="anchor hidden-xs" href="#concat-1" aria-

hidden=”true”># TOC</h3>

Physical concatenation protocol. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concat:<var class=”var”>URL1</var>|<var class=”var”>URL2</var>|...|<var class=”var”>URLN</var> </pre>

where <var class=”var”>URL1</var>, <var class=”var”>URL2</var>, ..., <var class=”var”>URLN</var> are the urls of the resource to be concatenated, each one possibly specifying a distinct protocol. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concat:split1.mpeg\|split2.mpeg\|split3.mpeg </pre>

Note that you may need to escape the character "|" which is special for many shells. <h3 class=”section”>24.6 concatf# TOC</h3>

Physical concatenation protocol using a line break delimited list of resources. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concatf:<var class=”var”>URL</var> </pre>

where <var class=”var”>URL</var> is the url containing a line break delimited list of resources to be concatenated, each one possibly specifying a distinct protocol. Special characters must be escaped with backslash or single quotes. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#quoting_005fand_005fescaping”>the "Quoting and escaping" section in the ffmpeg-utils(1) manual. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> listed in separate lines within a file <samp class=”file”>split.txt</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concatf:split.txt </pre> Where <samp class=”file”>split.txt</samp> contains the lines: <pre class=”example-preformatted”>split1.mpeg split2.mpeg split3.mpeg </pre>

 <h3 class=”section”>24.7 crypto# TOC</h3>

AES-encrypted stream reading protocol. The accepted options are: <dl class="table"> <dt><samp class="option">key</samp></dt> <dd> Set the AES decryption key binary block from given hexadecimal representation. </dd> <dt><samp class="option">iv</samp></dt> <dd> Set the AES decryption initialization vector binary block from given hexadecimal representation. </dd> </dl>

Accepted URL formats: <pre class="example-preformatted">crypto:<var class="var">URL</var> crypto+<var class="var">URL</var> </pre>

 <h3 class="section">24.8 data# TOC</h3>

Data in-line in the URI. See <a class="url" href="http://en.wikipedia.org/wiki/Data_URI_scheme">http For example, to convert a GIF file given inline with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i " smiley.png </pre>

 <h3 class="section">24.9 fd# TOC</h3>

File descriptor access protocol. The accepted syntax is: <pre class="example-preformatted">fd: -fd <var class="var">file_descriptor</var> </pre>

If <samp class="option">fd</samp> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. Unlike the pipe protocol, fd protocol has seek support if it corresponding to a regular file. fd protocol doesn’t support pass file descriptor via URL for security. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

 <h3 class="section">24.10 file# TOC</h3>

File access protocol. Read from or write to a file. A file URL can have the form: <pre class="example-preformatted">file:<var class="var">filename</var> </pre>

where <var class="var">filename</var> is the path of the file to read. An URL that does not have a protocol prefix will be assumed to be a file URL. Depending on the build, an URL that looks like a Windows path with the drive letter at the beginning will also be assumed to be a file URL (usually not the case in builds for unix-like systems). For example to read from a file <samp class="file">input.mpeg</samp> with <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -i file:input.mpeg output.mpeg </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable for files on slow medium. </dd> <dt><samp class="option">follow</samp></dt> <dd> If set to 1, the protocol will retry reading at the end of the file, allowing reading files that still are being written. In order for this to terminate, you either need to use the rw_timeout option, or use the interrupt callback (for API users). Setting this option also ignores the file size reported by the file system. </dd> <dt><samp class="option">seekable</samp></dt> <dd> Controls if seekability is advertised on the file. 0 means non-seekable, -1 means auto (seekable for normal files, non-seekable for named pipes). Many demuxers handle seekable and non-seekable resources differently, overriding this might speed up opening certain files at the cost of losing some features (e.g. accurate seeking). </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set the maximum packet size used for file I/O. A smaller value may reduce memory usage. A higher value may increase throughput especially with networked filesystems. For reading, if explicitly set, it overrides the default internal buffer size (32 KB) and limits the maximum amount of data read per operation. For writing, this sets the size of each write operation. The default is 256 KB for regular files, 32 KB otherwise. </dd> </dl>

 <h3 class="section">24.11 ftp# TOC</h3>

FTP (File Transfer Protocol). Read from or write to remote resources using FTP protocol. Following syntax is required. <pre class="example-preformatted">ftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg </pre>

This protocol accepts the following options. <dl class="table">

<dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout in microseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>ftp-user</samp></dt> <dd> Set a user to be used for authenticating to the FTP server. This is overridden by the user in the FTP URL. </dd> <dt><samp class=”option”>ftp-password</samp></dt> <dd> Set a password to be used for authenticating to the FTP server. This is overridden by the password in the FTP URL, or by <samp class=”option”>ftp-anonymous-password</samp> if no user is set. </dd> <dt><samp class=”option”>ftp-anonymous-password</samp></dt> <dd> Password used when login as anonymous user. Typically an e-mail address should be used. </dd> <dt><samp class=”option”>ftp-write-seekable</samp></dt> <dd> Control seekability of connection during encoding. If set to 1 the resource is supposed to be seekable, if set to 0 it is assumed not to be seekable. Default value is 0. </dd> </dl>

NOTE: Protocol can be used as output, but it is recommended to not do it, unless special care is taken (tests, customized server configuration etc.). Different FTP servers behave in different way during seek operation. ff* tools may produce incomplete content due to server limitations. <h3 class=”section”>24.12 gopher# TOC</h3>

Gopher protocol. <h3 class=”section”>24.13 gophers# TOC</h3>

Gophers protocol. The Gopher protocol with TLS encapsulation. <h3 class=”section”>24.14 handle# TOC</h3>

Windows handle access protocol. The accepted syntax is: <pre class=”example-preformatted”>handle:-handle <var class=”var”>handle</var> </pre>

If <samp class=”option”>handle</samp> is not specified, by default the output handle will be used for writing, input handle for reading. This protocol is only available on Windows. It has seek support if the handle corresponds to a regular file. It doesn’t support passing the handle via URL. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is

<code class=”code”>INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class=”option”>handle</samp></dt> <dd> Set Windows handle. The supplied handle must already be valid in the calling process. When launching <code class=”command”>ffmpeg</code>, arrange for the child process to inherit the handle before passing its numeric value with <samp class=”option”>handle</samp>. Handles opened with <code class=”code”>FILE_FLAG_OVERLAPPED</code> are not supported. Closing the protocol does not close the caller’s original handle. </dd> </dl>

 <h3 class=”section”>24.15 http# TOC</h3>

HTTP (Hyper Text Transfer Protocol).

This protocol accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>seekable</samp></dt>

<dd> Control seekability of connection. If set to 1 the resource is

supposed to be seekable, if set to 0 it is assumed not to be seekable,

if set to -1 it will try to autodetect if it is seekable. Default

value is -1.

</dd>

<dt><samp class=”option”>chunked_post</samp></dt>

<dd> If set to 1 use chunked Transfer-Encoding for posts, default is 1.

</dd>

<dt><samp class=”option”>http_proxy</samp></dt>

<dd> set HTTP proxy to tunnel through e.g. http://example.com:1234

</dd>

<dt><samp class=”option”>headers</samp></dt>

<dd> Set custom HTTP headers, can override built in default headers. The value must be a string encoding the headers.

</dd>

<dt><samp class="option">content_type</samp></dt>

<dd> Set a specific content type for the POST messages or for listen mode.

</dd>

<dt><samp class="option">user_agent</samp></dt>

<dd> Override the User-Agent header. If not specified the protocol will use a

string describing the libavformat build. ("Lavf/<version>")

</dd>

<dt><samp class="option">referer</samp></dt>

<dd> Set the Referer header. Include ’Referer: URL’ header in HTTP request.

</dd>

<dt><samp class="option">multiple_requests</samp></dt>

<dd> Force persistent connections if set to 1, or disable if 0. Default is -1, which means auto (implies keep-alive when using -request_size or -initial_request_size).

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> Limit the size of requests made. This is useful for some pathological servers

that throttle unbounded range requests, as well as when expecting to seek frequently. Disabled (set to 0) by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher. Doing so allows FFmpeg

to reuse a single HTTP connection wherever possible.

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also enable

the <samp class="option">multiple_requests</samp> option, as well as setting

<samp class="option">short_seek_size</samp> to the same value or higher.

</dd>

<dt><samp class="option">post_data</samp></dt>

<dd> Set custom HTTP post data.

</dd>

<dt><samp class="option">mime_type</samp></dt>

<dd> Export the MIME type.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> Exports the HTTP response version number. Usually "1.0" or "1.1".

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Set the cookies to be sent in future requests. The format of each cookie is the

same as the value of a Set-Cookie HTTP response field. Multiple cookies can be

delimited by a newline character. A cookie without a Domain attribute is only

sent to the host of the URL it is set for.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class="option">icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class="option">icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class="option">metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bitstream, if present.

Only useful with the C API.

</dd>

<dt><samp class="option">auth_type</samp></dt>

<dd>

Set HTTP authentication type. No option for Digest, since this method requires

getting nonce parameters from the server first and can’t be used straight away like

Basic.

<dl class="table">

<dt><samp class="option">none</samp></dt>

<dd> Choose the HTTP authentication type automatically. This is the default.

</dd>

<dt><samp class="option">basic</samp></dt>

<dd>

Choose the HTTP basic authentication.

Basic authentication sends a Base64-encoded string that contains a user name and password

for the client. Base64 is not a form of encryption and should be considered the same as

sending the user name and password in clear text (Base64 is a reversible encoding).

If a resource needs to be protected, strongly consider using an authentication scheme

other than basic authentication. HTTPS/TLS should be used with basic authentication.

Without these additional security enhancements, basic authentication should not be used

to protect sensitive or valuable information.

</dd>

</dl>

</dd> <dt><samp class="option">send_expect_100</samp></dt> <dd> Send an Expect: 100-continue header for POST. If set to 1 it will send, if set to 0 it won’t, if set to -1 it will try to send if it is applicable. Default value is -1. </dd> <dt><samp class="option">location</samp></dt> <dd> An exported dictionary containing the content location. Only useful with the C API. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set initial byte offset. </dd> <dt><samp class="option">end_offset</samp></dt> <dd> Try to limit the request to bytes preceding this offset. </dd> <dt><samp class="option">method</samp></dt> <dd> When used as a client option it sets the HTTP method for the request. When used as a server option it sets the HTTP method that is going to be expected from the client(s). If the expected and the received HTTP method do not match the client will be given a Bad Request response. When unset the HTTP method is not checked for now. This will be replaced by autodetection in the future. </dd> <dt><samp class="option">reconnect</samp></dt> <dd> Reconnect automatically when disconnected before EOF is hit. </dd> <dt><samp class="option">reconnect_at_eof</samp></dt> <dd> If set then eof is treated like an error and causes reconnection, this is useful for live / endless streams. This does not apply to the deliberately established end of a resource whose complete size is unknown, i.e. when a ranged request past the end is answered with <code class="code">416</code>. Such an end is reported as a clean EOF and is not retried, unless <samp class="option">reconnect_on_http_error</samp> lists 416 by number. </dd> <dt><samp class="option">reconnect_on_network_error</samp></dt> <dd> Reconnect automatically in case of TCP/TLS errors during connect. </dd> <dt><samp class="option">reconnect_on_http_error</samp></dt> <dd> A comma separated list of HTTP status codes to reconnect on. The list can include specific status codes (e.g. ’503’) or the strings ’4xx’ / ’5xx’. A <code class="code">416</code> that ends a read or a seek past the end of a resource whose complete size is unknown is reported as EOF. It is retried first only if 416 is listed by number, for example to read a resource that is still being written. ’4xx’ does not cover it. </dd> <dt><samp

class=”option”>reconnect_streamed</samp></dt> <dd> If set then even streamed/non seekable streams will be reconnected on errors. </dd> <dt><samp class=”option”>reconnect_delay_max</samp></dt> <dd> Set the maximum delay in seconds after which to give up reconnecting. </dd> <dt><samp class=”option”>reconnect_max_retries</samp></dt> <dd> Set the maximum number of times to retry a connection. Default unset. </dd> <dt><samp class=”option”>reconnect_delay_total_max</samp></dt> <dd> Set the maximum total delay in seconds after which to give up reconnecting. </dd> <dt><samp class=”option”>respect_retry_after</samp></dt> <dd> If enabled, and a Retry-After header is encountered, its requested reconnection delay will be honored, rather than using exponential backoff. Useful for 429 and 503 errors. Default enabled. </dd> <dt><samp class=”option”>listen</samp></dt> <dd> If set to 1 enables experimental HTTP server. This can be used to send data when used as an output option, or read data from a client with HTTP POST when used as an input option. If set to 2 enables experimental multi-client HTTP server. This is not yet implemented in ffmpeg.c and thus must not be used as a command line option. <pre class=”example-preformatted”># Server side (sending): ffmpeg -i somefile.ogg -c copy -listen 1 -f ogg http://<var class=”var”>server</var>:<var class=”var”>port</var>

Client side (receiving): ffmpeg -i http://<var class=”var”>server</var>:<var class=”var”>port</var> -c copy somefile.ogg

Client can also be done with wget: wget http://<var class=”var”>server</var>:<var class=”var”>port</var> -O somefile.ogg

Server side (receiving): ffmpeg -listen 1 -i http://<var class=”var”>server</var>:<var class=”var”>port</var> -c copy somefile.ogg

Client side (sending): ffmpeg -i somefile.ogg -chunked_post 0 -c copy -f ogg http://<var class=”var”>server</var>:<var class=”var”>port</var>

Client can also be done with wget: wget --post-file=somefile.ogg http://<var class=”var”>server</var>:<var class=”var”>port</var> </pre>

</dd> <dt><samp class=”option”>resource</samp></dt> <dd> The resource requested by a client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>reply_code</samp></dt> <dd> The HTTP code returned to the client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>short_seek_size</samp></dt> <dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and new HTTP request. This is useful, for example, to make sure the same connection is used for reading large video packets with small audio packets in between. </dd> </dl>

 <h4 class=”subsection”>24.15.1 HTTP Cookies# <a class=”anchor hidden-xs”href=”#toc-

HTTP-Cookies" aria-hidden="true">TOC</h4>

Some HTTP requests will be denied unless cookie values are passed in with the request. The <samp class="option">cookies</samp> option allows these cookies to be specified. At the very least, each cookie must specify a value. HTTP requests that match both the domain and path will automatically include the cookie value in the HTTP Cookie header field. Multiple cookies can be delimited by a newline. The required syntax to play a stream specifying a cookie is: <pre class="example-preformatted">ffplay -cookies "nlqptid=nltid=tsn; path=/; domain=somedomain.com;" http://somedomain.com/somestream.m3u8 </pre>

 <h3 class="section">24.16 Icecast# TOC</h3>

Icecast protocol (stream to Icecast servers) This protocol accepts the following options: <dl class="table"> <dt><samp class="option">ice_genre</samp></dt> <dd> Set the stream genre. </dd> <dt><samp class="option">ice_name</samp></dt> <dd> Set the stream name. </dd> <dt><samp class="option">ice_description</samp></dt> <dd> Set the stream description. </dd> <dt><samp class="option">ice_url</samp></dt> <dd> Set the stream website URL. </dd> <dt><samp class="option">ice_public</samp></dt> <dd> Set if the stream should be public. The default is 0 (not public). </dd> <dt><samp class="option">user_agent</samp></dt> <dd> Override the User-Agent header. If not specified a string of the form "Lavf/<version>" will be used. </dd> <dt><samp class="option">password</samp></dt> <dd> Set the Icecast mountpoint password. </dd> <dt><samp class="option">content_type</samp></dt> <dd> Set the stream content type. This must be set if it is different from audio/mpeg. </dd> <dt><samp class="option">legacy_icecast</samp></dt> <dd> This enables support for Icecast versions < 2.4.0, that do not support the HTTP PUT method but the SOURCE method. </dd> <dt><samp class="option">tls</samp></dt> <dd> Establish a TLS (HTTPS) connection to Icecast. </dd> </dl>

 <pre class="example-preformatted">icecast://[<var class="var">username</var>[:<var class="var">password</var>]@]<var class="var">server</var>:<var class="var">port</var>/<var class="var">mountpoint</var> </pre>

 <h3 class="section">24.17 ipfs# TOC</h3>

InterPlanetary File System (IPFS) protocol support. One can access

files stored on the IPFS network through so-called gateways. These are http(s) endpoints. This protocol wraps the IPFS native protocols (ipfs:// and ipns://) to be sent to such a gateway. Users can (and should) host their own node which means this protocol will use one’s local gateway to access files on the IPFS network. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">gateway</samp></dt> <dd> Defines the gateway to use. When not set, the protocol will first try locating the local gateway by looking at <code class="code">$IPFS_GATEWAY</code>, <code class="code">$IPFS_PATH</code> and <code class="code">$HOME/.ipfs/</code>, in that order. </dd> </dl>

One can use this protocol in 2 ways. Using IPFS: <pre class="example-preformatted">ffplay ipfs://<hash> </pre>

Or the IPNS protocol (IPNS is mutable IPFS): <pre class="example-preformatted">ffplay ipns://<hash> </pre>

 <h3 class="section">24.18 mmst# TOC</h3>

MMS (Microsoft Media Server) protocol over TCP. <h3 class="section">24.19 mmsh# TOC</h3>

MMS (Microsoft Media Server) protocol over HTTP. The required syntax is: <pre class="example-preformatted">mmsh://<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">playpath</var>] </pre>

 <h3 class="section">24.20 md5# TOC</h3>

MD5 output protocol. Computes the MD5 hash of the data to be written, and on close writes this to the designated output or stdout if none is specified. It can be used to test muxers without writing an actual file. Some examples follow. <pre class="example-preformatted"># Write the MD5 hash of the encoded AVI file to the file output.avi.md5. ffmpeg -i input.flv -f avi -y md5:output.avi.md5

Write the MD5 hash of the encoded AVI file to stdout. ffmpeg -i input.flv -f avi -y md5: </pre>

Note that some formats (typically MOV) require the output protocol to be seekable, so they will fail with the MD5 output protocol. <h3 class="section">24.21 pipe<span

class=”pull-right”># TOC</h3>

UNIX pipe access protocol. Read and write from UNIX pipes. The accepted syntax is: <pre class=”example-preformatted”>pipe:[<var class=”var”>number</var>] </pre>

If <samp class=”option”>fd</samp> isn’t specified, <var class=”var”>number</var> is the number corresponding to the file descriptor of the pipe (e.g. 0 for stdin, 1 for stdout, 2 for stderr). If <var class=”var”>number</var> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. For example to read from stdin with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>cat test.wav | ffmpeg -i pipe:0 # ...this is the same as... cat test.wav | ffmpeg -i pipe: </pre>

For writing to stdout with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i test.wav -f avi pipe:1 | cat > test.avi # ...this is the same as... ffmpeg -i test.wav -f avi pipe: | cat > test.avi </pre>

This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class=”code”>INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class=”option”>fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

Note that some formats (typically MOV), require the output protocol to be seekable, so they will fail with the pipe output protocol. <h3 class=”section”>24.22 prompeg# TOC</h3>

Pro-MPEG Code of Practice #3 Release 2 FEC protocol. The Pro-MPEG CoP#3 FEC is a 2D parity-check forward error correction mechanism for MPEG-2 Transport Streams sent over RTP. This protocol must be used in conjunction with the <code class=”code”>rtp_mpegts</code> muxer and the <code class=”code”>rtp</code> protocol. The required syntax is: <pre class=”example-preformatted”>-f rtp_mpegts -fec prompeg=<var class=”var”>option</var>=<var class=”var”>val</var>... rtp://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

The destination UDP ports are <code class="code">port + 2</code> for the column FEC stream and <code class="code">port + 4</code> for the row FEC stream. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">l=<var class="var">n</var></samp></dt> <dd> The number of columns (4-20, LxD <= 100) </dd> <dt><samp class="option">d=<var class="var">n</var></samp></dt> <dd> The number of rows (4-20, LxD <= 100) </dd> </dl>

Example usage: <pre class="example-preformatted">-f rtp_mpegts -fec prompeg=l=8:d=4 rtp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">24.23 rist# TOC</h3>

Reliable Internet Streaming Transport protocol The accepted options are: <dl class="table"> <dt><samp class="option">rist_profile</samp></dt> <dd> Supported values: <dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</d <dd> This one is default. </dd> <dt>‘<samp class="samp">advanced</samp>’</ </dl>

</dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set internal RIST buffer size in milliseconds for retransmission of data. Default value is 0 which means the librist default (1 sec). Maximum value is 30 seconds. </dd> <dt><samp class="option">fifo_size</samp></dt> <dd> Size of the librist receiver output fifo in number of packets. This must be a power of 2. Defaults to 8192 (vs the librist default of 1024). </dd> <dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt> <dd> Survive in case of librist fifo buffer overrun. Default value is 0. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set maximum packet size for sending data. 1316 by default. </dd> <dt><samp class="option">log_level</samp></dt> <dd> Set loglevel for RIST logging messages. You only need to set this if you explicitly want to enable debug level messages or packet loss simulation, otherwise the regular loglevel is respected. </dd> <dt><samp class="option">secret</samp></dt> <dd> Set override of encryption secret, by default is unset. </dd> <dt><samp class="option">encryption</samp></dt> <dd> Set encryption type, by default is disabled. Acceptable values are 128 and 256. </dd> </dl>

 <h3 class="section">24.24 rtmp# TOC</h3>

Real-Time Messaging Protocol. The Real-Time Messag-

ing Protocol (RTMP) is used for streaming multimedia content across a TCP/IP network. The required syntax is: <pre class=”example-preformatted”>rtmp://[<var class=”var”>username</var>:<var class=”var”>password</var>@]<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>instance</var>][/<var class=”var”>playpath</var>] </pre>

The accepted parameters are: <dl class=”table”> <dt><samp class=”option”>username</samp></dt> <dd> An optional username (mostly for publishing). </dd> <dt><samp class=”option”>password</samp></dt> <dd> An optional password (mostly for publishing). </dd> <dt><samp class=”option”>server</samp></dt> <dd> The address of the RTMP server. </dd> <dt><samp class=”option”>port</samp></dt> <dd> The number of the TCP port to use (by default is 1935). </dd> <dt><samp class=”option”>app</samp></dt> <dd> It is the name of the application to access. It usually corresponds to the path where the application is installed on the RTMP server (e.g. <samp class=”file”>/ondemand/</samp>, <samp class=”file”>/flash/live/</samp>, etc.). You can override the value parsed from the URI through the <code class=”code”>rtmp_app</code> option, too. </dd> <dt><samp class=”option”>playpath</samp></dt> <dd> It is the path or name of the resource to play with reference to the application specified in <var class=”var”>app</var>, may be prefixed by "mp4:". You can override the value parsed from the URI through the <code class=”code”>rtmp_playpath</code> option, too. </dd> <dt><samp class=”option”>listen</samp></dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt><samp class=”option”>timeout</samp></dt> <dd> Maximum time to wait for the incoming connection. Implies listen. </dd> </dl>

Additionally, the following parameters can be set via command line options

(or in code via <code class=”code”>AVOption</code>s):

<dl class=”table”>

<dt><samp class=”option”>rtmp_app</samp></dt>

<dd> Name of application to connect on the RTMP server. This option

overrides the parameter specified in the URI.

</dd>

<dt><samp class=”option”>rtmp_buffer</samp></dt>

<dd> Set the client buffer time in milliseconds. The default is 3000.

</dd>

<dt><samp class=”option”>rtmp_conn</samp></dt>

<dd> Extra arbitrary AMF connection parameters, parsed from a string,

e.g. like <code class=”code”>B:1 S:authMe O:1 NN:code:1.23 NS:flag:ok O:0</code>.

Each value is prefixed by a single character denoting the type,

B for Boolean, N for number, S for string, O for object, or Z for null,

followed by a colon. For Booleans the data must be either 0 or 1 for

FALSE or TRUE, respectively. Likewise for Objects the data must be 0 or

1 to end or begin an object, respectively. Data items in subobjects may

be named, by prefixing the type with ’N’ and specifying the name before

the value (i.e. <code class=”code”>NB:myFlag:1</code>). This option may be used multiple

times to construct arbitrary AMF sequences.

</dd>

<dt><samp class=”option”>rtmp_enhanced_codecs</samp></dt>

<dd> Specify the list of codecs the client advertises to support in an

enhanced RTMP stream. This option should be set to a comma separated

list of fourcc values, like <code class=”code”>hvc1,av01,vp09</code> for multiple codecs

or <code class=”code”>hvc1</code> for only one codec. The specified list will be presented

in the "fourCcLive" property of the Connect Command Message.

</dd>

<dt><samp class=”option”>rtmp_flashver</samp></dt>

<dd> Version of the Flash plugin used to run the SWF player. The default

is LNX 9,0,124,2. (When publishing, the default is FMLE/3.0 (compatible;

<libavformat version>).)

</dd>

<dt><samp class=”option”>rtmp_flush_interval</samp></dt>

<dd> Number of packets flushed in the same request (RTMPT only). The default

is 10.

</dd>

<dt><samp class=”option”>rtmp_live</samp></dt>

<dd> Specify that the media is a live stream. No resuming or seeking in

live streams is possible. The default value is <code class=”code”>any</code>, which means the

subscriber first tries to play the live stream specified in the

playpath. If a live stream of that name is not found, it plays the

recorded stream. The other possible values are <code class=”code”>live</code> and

<code class=”code”>recorded</code>.

</dd>

<dt><samp class=”option”>rtmp_pageurl</samp></dt>

<dd> URL of the web page in which the media was embedded. By default no

value will be sent.

</dd>

<dt><samp class=”option”>rtmp_playpath</samp></dt>

<dd> Stream identifier to play or to publish. This option overrides the

parameter specified in the URI.

</dd>

<dt><samp class=”option”>rtmp_subscribe</samp></dt>

<dd> Name of live stream to subscribe to. By default no value will be sent.

It is only sent if the option is specified or if rtmp_live

is set to live.

</dd>

<dt><samp class="option">rtmp_swfhash</samp></dt>

<dd> SHA256 hash of the decompressed SWF file (32 bytes).

</dd>

<dt><samp class="option">rtmp_swfsize</samp></dt>

<dd> Size of the decompressed SWF file, required for SWFVerification.

</dd>

<dt><samp class="option">rtmp_swfurl</samp></dt>

<dd> URL of the SWF player for the media. By default no value will be sent.

</dd>

<dt><samp class="option">rtmp_swfverify</samp></dt>

<dd> URL to player swf file, compute hash/size automatically.

</dd>

<dt><samp class="option">rtmp_tcurl</samp></dt>

<dd> URL of the target stream. Defaults to proto://host[:port]/app.

</dd>

<dt><samp class="option">tcp_nodelay=<var class="var">1|0</var></samp></dt>

<dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0.

Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY.

</dd>

<dt><samp class=”option”>tcp_keepalive=<var class=”var”>1|0</var></samp></dt>

<dd> Enable the TCP keepalive mechanism to detect dead peers and help maintain long-lived idle connections. Default value is 0.

Only the basic keepalive option (SO_KEEPALIVE) can be enabled or disabled. Platform-specific tuning parameters such as TCP_KEEPIDLE, TCP_KEEPINTVL, or TCP_KEEPCNT are not configurable and will use the operating system’s default values.

</dd>

</dl>

For example to read with <code class=”command”>ffplay</code> a multimedia resource named "sample" from the application "vod" from an RTMP server "myserver": <pre class=”example-preformatted”>ffplay rtmp://myserver/vod/sample </pre>

To publish to a password protected server, passing the playpath and app names separately: <pre class=”example-preformatted”>ffmpeg -re -i <input> -f flv -rtmp_playpath some/long/path -rtmp_app long/app/name rtmp://username:password@myserver/ </pre>

 <h3 class=”section”>24.25 rtmpe# TOC</h3>

Encrypted Real-Time Messaging Protocol. The Encrypted Real-Time Messaging Protocol (RTMPE) is used for streaming multimedia content within standard cryptographic primitives, consisting of Diffie-Hellman key exchange and HMACSHA256, generating a pair of RC4 keys. <h3 class=”section”>24.26 rtmps# TOC</h3>

Real-Time Messaging Protocol over a secure SSL connection. The Real-Time Messaging Protocol (RTMPS) is used for streaming multimedia content across an encrypted connection. <h3 class=”section”>24.27 rtmpt# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTP. The Real-Time Messaging Protocol tunneled through HTTP (RTMPT) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">24.28 rtmpte# TOC</h3>

Encrypted Real-Time Messaging Protocol tunneled through HTTP. The Encrypted Real-Time Messaging Protocol tunneled through HTTP (RTMPTE) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">24.29 rtmpts# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTPS. The Real-Time Messaging Protocol tunneled through HTTPS (RTMPTS) is used for streaming multimedia content within HTTPS requests to traverse firewalls. <h3 class="section">24.30 libcurl# TOC</h3>

HTTP and HTTPS access through the libcurl library, available as an alternative to the native <code class="code">http</code> protocol when FFmpeg is built with <code class="code">--enable-libcurl</code>. It runs transfers on a dedicated event-loop thread shared per <code class="code">AVFormatContext</code>, so all of a demuxer’s connections (for example HLS segments) reuse libcurl’s connection cache and HTTP/2 and HTTP/3 multiplexing. The native <code class="code">http</code> protocol stays the default. There are two ways to use libcurl instead: <ul class="itemize mark-bullet"> Prefix the URL with <code class="code">libcurl:</code> to force it explicitly, for example <code class="code">libcurl:https://example.com/video.mp4</code>.

Set the <samp class="option">prefer_libcurl</samp> option to route <code class="code">http://</code> and <code class="code">https://</code> URLs through libcurl transparently.

<dl class="table"> <dt><samp class="option">prefer_libcurl</samp></dt> <dd> When set to <code class="code">1</code>, open <code class="code">http(s)</code> URLs with the libcurl protocol instead of the native one. This is a generic IO option rather than a libcurl-specific one, so it also applies on builds without libcurl (where it has no effect). Default is <code class="code">0</code>. </dd> </dl>

The libcurl protocol accepts the following options.

<dl class="table">

<dt><samp class="option">user_agent</samp></dt>

<dd> Override the <code class="code">User-Agent</code> header. Defaults to <code class="code">Lavf/<version></code>.

</dd>

<dt><samp class="option">referer</samp></dt>

<dd> Set the <code class="code">Referer</code> header.

</dd>

<dt><samp class="option">headers</samp></dt>

<dd> Set custom HTTP headers, overriding internal defaults. Multiple headers are

separated by newlines.

</dd>

<dt><samp class="option">http_proxy</samp></dt>

<dd> Set the HTTP proxy to tunnel through.

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Cookies to be sent in future requests, given as newline-delimited

<code class="code">Set-Cookie</code>-style values.

</dd>

<dt><samp class="option">location</samp></dt>

<dd> The actual location of the data after following redirects. This is an output

option, read with <code class="command">av_opt_get</code>; demuxers use it to resolve relative URLs.

</dd>

<dt><samp class=”option”>offset</samp></dt>

<dd> Initial byte offset of the request. Default is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>end_offset</samp></dt>

<dd> If greater than <code class=”code”>0</code>, limit the request to the bytes preceding this offset.

Together with <samp class=”option”>offset</samp> this requests a sub-range of the resource, as used

for HLS byte-range segments.

</dd>

<dt><samp class=”option”>seekable</samp></dt>

<dd> Control seekability of the connection. If set to <code class=”code”>0</code> the stream is treated

as non-seekable, <code class=”code”>1</code> forces seekable, and <code class=”code”>-1</code> (the default) lets it be

detected from the server response.

</dd>

<dt><samp class=”option”>tls_verify</samp></dt>

<dd> Verify the peer certificate and host name. Enabled by default.

</dd>

<dt><samp class=”option”>ca_file</samp></dt>

<dd> Path to a certificate authority bundle that overrides the system store.

</dd>

<dt><samp class=”option”>cert_file</samp></dt>

<dt><samp class=”option”>key_file</samp></dt>

<dd> Client certificate and private key files for mutual TLS.

</dd>

<dt><samp class="option">connect_timeout</samp></dt>

<dd> Connection timeout in seconds. <code class="code">0</code> (the default) uses libcurl’s own timeout.

</dd>

<dt><samp class="option">max_redirects</samp></dt>

<dd> Maximum number of redirects to follow. Default is <code class="code">16</code>.

</dd>

<dt><samp class="option">multiple_requests</samp></dt>

<dd> Reuse the connection across requests (HTTP keep-alive). Enabled by default;

setting it to <code class="code">0</code> uses a fresh connection per transfer.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> HTTP version to use: <code class="code">auto</code> (default), <code class="code">1.0</code>, <code class="code">1.1</code>, <code class="code">2</code>,

<code class="code">2tls</code>, <code class="code">2-prior-knowledge</code>, <code class="code">3</code> or <code class="code">3only</code>.

</dd>

<dt><samp class="option">buffer_size</samp></dt>

<dd> Size in bytes of the receive buffer. Default is 4 MiB.

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> If greater than <code class=”code”>0</code>, split a seekable transfer into ranged requests of at

most this many bytes each. Default is <code class=”code”>0</code> (a single request).

</dd>

<dt><samp class=”option”>initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class=”code”>request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class=”option”>short_seek_size</samp> to the same value or higher, otherwise the HTTP

connection may be closed and recreated for each subsequent initial request.

</dd>

<dt><samp class=”option”>max_retries</samp></dt>

<dd> Maximum number of retries after a recoverable error on a seekable transfer.

Default is <code class=”code”>5</code>.

</dd>

<dt><samp class=”option”>short_seek_size</samp></dt>

<dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and

new HTTP request. This is useful, for example, to make sure the same connection

is used for reading large video packets with small audio packets in between.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_status</samp></dt>

<dd> If set to 1 accept the legacy Shoutcast v1 <code class="code">ICY 200 OK</code> status line in

place of an HTTP one. Any reply whose status line starts with <code class="code">ICY 200</code>

is then treated as a successful HTTP 200.

The default is 1.

</dd>

<dt><samp class="option">icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class="option">icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class="option">icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class="option">metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bitstream, if present.

Only useful with the C API.

</dd>

</dl>

A stream carrying in-band ICY metadata is always treated as non-seekable, since its byte offsets do not address the payload. <samp class="option">seekable</samp>, <samp class="option">request_size</samp> and <samp class="option">max_retries</samp> therefore have no effect on such a stream. For more information see: https://curl.se/libcurl/. <h3 class="section">24.31 libsmbclient# TOC</h3>

libsmbclient permits one to manipulate CIFS/SMB network resources. Following syntax is required.   <pre class="example-preformatted">smb://[[domain:]user[:password@]]server[/share[/path[/file]]] </pre>

This protocol accepts the following options. <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Set timeout in milliseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">workgroup</samp></dt> <dd> Set the workgroup used for making connections. By default workgroup is not specified. </dd> </dl>

For more information see: http://www.samba.org/. <h3 class="section">24.32 libssh# TOC</h3>

Secure File Transfer Protocol via libssh Read from or write to remote resources using SFTP protocol. Following

syntax is required.   <pre class=”example-preformatted”>sftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg
</pre>

This protocol accepts the following options. <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class=”option”>private_key</samp></dt> <dd> Specify the path of the file containing private key to use during authorization. By default libssh searches for keys in the <samp class=”file”>~/.ssh/</samp> directory. </dd> <dt><samp class=”option”>verify</samp></dt> <dd> Verify the server host key against the known_hosts file before authenticating, if set to 1. Servers that are not in the file, or whose key differs from the one recorded there, are refused. Default value is 1. </dd> </dl>

Example: Play a file stored on remote server.   <pre class=”example-preformatted”>ffplay sftp://user:password@server_address:22/home/user/resource.mpeg
</pre>

 <h3 class=”section”>24.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte# TOC</h3>

Real-Time Messaging Protocol and its variants supported through librtmp. Requires the presence of the librtmp headers and library during configuration. You need to explicitly configure the build with "–enable-librtmp". If enabled this will replace the native RTMP protocol. This protocol provides most client functions and a few server functions needed to support RTMP, RTMP tunneled in HTTP (RTMPT), encrypted RTMP (RTMPE), RTMP over SSL/TLS (RTMPS) and tunneled variants of these encrypted types (RTMPTE, RTMPTS). The required syntax is: <pre class=”example-preformatted”><var class=”var”>rtmp_proto</var>://<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>playpath</var>] <var class=”var”>options</var> </pre>

where <var class=”var”>rtmp_proto</var> is one of the strings "rtmp", "rtmpt", "rtmpe", "rtmps", "rtmpte", "rtmpts" corresponding to each RTMP variant, and <var class=”var”>server</var>, <var class=”var”>port</var>,

<var class=”var”>app</var> and <var class=”var”>playpath</var> have the same meaning as specified for the RTMP native protocol. <var class=”var”>options</var> contains a list of space-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. See the librtmp manual page (man 3 librtmp) for more information. For example, to stream a file in real-time to an RTMP server using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -re -i myfile -f flv rtmp://myserver/live/mystream </pre>

To play the same stream using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay "rtmp://myserver/live/mystream live=1" </pre>

 <h3 class=”section”>24.34 rtp# TOC</h3>

Real-time Transport Protocol. The required syntax for an RTP URL is: <pre class=”example-preformatted”>rtp://<var class=”var”>hostname</var>[:<var class=”var”>port</var>][?<var class=”var”>options</var>] </pre>

<var class=”var”>port</var> specifies the RTP port to use. <var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ttl=<var class=”var”>n</var></samp></dt> <dd> Set the TTL (Time-To-Live) value (for multicast only). </dd> <dt><samp class=”option”>rtcpport=<var class=”var”>n</var></samp></dt> <dd> Set the remote RTCP port to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>localport, local_rtpport, localrtpport=<var class=”var”>n</var></samp></dt> <dd> Set the local RTP port to <var class=”var”>n</var>. Using the localport option name is deprecated and should not be used. </dd> <dt><samp class=”option”>local_rtcpport, localrtcpport=<var class=”var”>n</var>'</samp></dt> <dd> Set the local RTCP port to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>pkt_size=<var class=”var”>n</var></samp></dt> <dd> Set max packet size (in bytes) to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>buffer_size=<var class=”var”>size</var></samp></dt> <dd> Set the maximum UDP socket buffer size in bytes. </dd> <dt><samp class=”option”>connect=0|1</samp></dt> <dd> Do a <code class=”code”>connect()</code> on the UDP socket (if set to 1) or not (if set to 0). </dd> <dt><samp class=”option”>sources=<var

class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt> <dd> List allowed source IP addresses. </dd> <dt><samp class=”option”>block=<var class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt> <dd> List disallowed (blocked) source IP addresses. </dd> <dt><samp class=”option”>write_to_source=0|1</samp></dt> <dd> Send packets to the source address of the latest received packet (if set to 1) or to a default remote address (if set to 0). </dd> <dt><samp class=”option”>localaddr=<var class=”var”>addr</var></samp></dt> <dd> Local IP address of a network interface used for sending packets or joining multicast groups. </dd> <dt><samp class=”option”>timeout=<var class=”var”>n</var></samp></dt> <dd> Set timeout (in microseconds) of socket I/O operations to <var class=”var”>n</var>. </dd> </dl>

Important notes: <ol class=”enumerate”> If <samp class=”option”>rtcpport</samp> is not set the RTCP port will be set to the RTP port value plus 1.

 If <samp class=”option”>localrtpport</samp> (the local RTP port) is not set any available port will be used for the local RTP and RTCP ports.

 If <samp class=”option”>localrtcpport</samp> (the local RTCP port) is not set it will be set to the local RTP port value plus 1.

 <h3 class=”section”>24.35 rtsp# TOC</h3>

Real-Time Streaming Protocol. RTSP is not technically a protocol handler in libavformat, it is a demuxer and muxer. The demuxer supports both normal RTSP (with data transferred over RTP; this is used by e.g. Apple and Microsoft) and Real-RTSP (with data transferred over RDT). The muxer can be used to send a stream using RTSP ANNOUNCE to a server supporting it (currently Darwin Streaming Server and Mischa Spiegelmock’s RTSP server). The required syntax for a RTSP url is: <pre class=”example-preformatted”>rtsp://<var class=”var”>hostname</var>[:<var class=”var”>port</var>]/<var class=”var”>path</var> </pre>

Options can be set on the <code class=”command”>ffmpeg</code>/<code class=”command”>ffplay</code> command line, or set in code via <code class=”code”>AVOption</code>s or in <code class=”code”>avformat_open_input</code>. <h4 class=”subsection”>24.35.1 Muxer# TOC</h4> The following options are supported. <dl class=”table”> <dt><samp class=”option”>rtsp_transport</samp></dt>

<dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">latm</samp>’</dt> <dd> Use MP4A-LATM packetization instead of MPEG4-GENERIC for AAC. </dd> <dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> Use RFC 2190 packetization instead of RFC 4629 for H.263. </dd> <dt>‘<samp class="samp">skip_rtcp</samp>’</dt> <dd> Don’t send RTCP sender reports. </dd> <dt>‘<samp class="samp">h264_mode0</samp>’</dt> <dd> Use mode 0 for H.264 in RTP. </dd> <dt>‘<samp class="samp">send_bye</samp>’</dt> <dd> Send RTCP BYE packets when finishing. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’.

</dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set max send packet size (in bytes). Default value is 1472. </dd> </dl>

 <h4 class="subsection">24.35.2 Demuxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">initial_pause</samp></dt> <dd> Do not start playing the stream immediately if set to 1. Default value is 0. </dd> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> <dt>‘<samp class="samp">udp_multicast</samp>’</dt> <dd> Use UDP multicast as lower transport protocol. </dd> <dt>‘<samp class="samp">http</samp>’</dt> <dd> Use HTTP tunneling as lower transport protocol, which is useful for passing proxies. </dd> <dt>‘<samp class="samp">https</samp>’</dt>

<dd> Use HTTPs tunneling as lower transport protocol, which is useful for passing proxies and widely used for security consideration. </dd> </dl>

Multiple lower transport protocols may be specified, in that case they are tried one at a time (if the setup of one fails, the next one is tried). For the muxer, only the ‘<samp class="samp">tcp</samp>’ and ‘<samp class="samp">udp</samp>’ options are supported. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">filter_src</samp>’</dt> <dd> Accept packets only from negotiated peer address and port. </dd> <dt>‘<samp class="samp">listen</samp>’</dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt>‘<samp class="samp">prefer_tcp</samp>’</dt> <dd> Try TCP for RTP transport first, if TCP is available as RTSP RTP transport. </dd> <dt>‘<samp class="samp">satip_raw</samp>’</dt> <dd> Export raw MPEG-TS stream instead of demuxing. The flag will simply write out the raw stream, with the original PAT/PMT/PIDs intact. </dd> </dl>

Default value is ‘<samp class="samp">none</samp>’. </dd> <dt><samp class="option">allowed_media_types</samp></dt> <dd> Set media types to accept from the server. The following flags are accepted: <dl class="table"> <dt>‘<samp class="samp">video</samp>’</dt> <dt>‘<samp class="samp">audio</samp>’</d <dt>‘<samp class="samp">data</samp>’</dt> <dt>‘<samp class="samp">subtitle</samp>’</dt> </dl>

By default it accepts all media types. </dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">listen_timeout</samp></dt> <dd> Set maximum timeout (in seconds) to establish an initial connection. Setting <samp class="option">listen_timeout</samp> > 0 sets <samp class="option">rtsp_flags</samp> to ‘<samp class="samp">listen</samp>’. Default is -1 which means an infinite timeout when ‘<samp class="samp">listen</samp>’ mode is set. </dd> <dt><samp class="option">reorder_queue_size</samp></dt> <dd> Set number of packets to buffer for handling of reordered packets. </dd> <dt><samp class="option">timeout</samp></dt> <dd> Set socket TCP I/O timeout in microseconds. </dd> <dt><samp class="option">user_agent</samp></dt> <dd> Override User-Agent header. If not specified, it defaults to the libavformat identifier string. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> </dl>

When receiving data over UDP, the demuxer tries to reorder received packets (since they may arrive out of order, or packets may get lost totally). This can be disabled by setting the maximum demuxing delay to zero (via the <code class="code">max_delay</code> field of AVFormatContext). When watching multi-bitrate Real-RTSP streams with <code class="command">ffplay</code>, the streams to display can be chosen with <code class="code">-vst</code> <var class="var">n</var> and <code class="code">-ast</code> <var class="var">n</var> for video and audio respectively, and can be switched on the fly by pressing <code class="code">v</code> and <code class="code">a</code>. <h4 class="subsection">24.35.3 Examples# TOC</h4>

The following examples all make use of the <code class="command">ffplay</code> and <code class="command">ffmpeg</code> tools. <ul class="itemize mark-bullet"> Watch a stream over UDP, with a max reordering delay of 0.5 seconds: <pre class="example-preformatted">ffplay -max_delay 500000 -rtsp_transport udp rtsp://server/video.mp4 </pre>

Watch a stream tunneled over HTTP: <pre class="example-preformatted">ffplay -rtsp_transport http rtsp://server/video.mp4 </pre>

Send a stream in realtime to a RTSP server, for others to watch: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f rtsp -muxdelay 0.1 rtsp://server/live.sdp </pre>

Receive a stream in realtime: <pre class="example-preformatted">ffmpeg -rtsp_flags listen -i rtsp://ownaddress/live.sdp <var class="var">output</var> </pre>

 <h3 class="section">24.36 sap# TOC</h3>

Session Announcement Protocol (RFC 2974). This is not technically a protocol handler in libavformat, it is a muxer and demuxer. It is used for signalling of RTP streams, by announcing the SDP for the streams regularly on a separate port. <h4 class="subsection">24.36.1 Muxer# TOC</h4>

The syntax for a SAP url given to the muxer is: <pre class="example-preformatted">sap://<var class="var">destination</var>[:<var

class=”var”>port</var>][?<var class=”var”>options</var>] </pre>

The RTP packets are sent to <var class=”var”>destination</var> on port <var class=”var”>port</var>, or to port 5004 if no port is specified. <var class=”var”>options</var> is a <code class=”code”>&</code>-separated list. The following options are supported: <dl class=”table”> <dt><samp class=”option”>announce_addr=<var class=”var”>address</var></samp></dt> <dd> Specify the destination IP address for sending the announcements to. If omitted, the announcements are sent to the commonly used SAP announcement multicast address 224.2.127.254 (sap.mcast.net), or ff0e::2:7ffe if <var class=”var”>destination</var> is an IPv6 address. </dd> <dt><samp class=”option”>announce_port=<var class=”var”>port</var></samp></dt> <dd> Specify the port to send the announcements on, defaults to 9875 if not specified. </dd> <dt><samp class=”option”>ttl=<var class=”var”>ttl</var></samp></dt> <dd> Specify the time to live value for the announcements and RTP packets, defaults to 255. </dd> <dt><samp class=”option”>same_port=<var class=”var”>0|1</var></samp></dt> <dd> If set to 1, send all RTP streams on the same port pair. If zero (the default), all streams are sent on unique ports, with each stream on a port 2 numbers higher than the previous. VLC/Live555 requires this to be set to 1, to be able to receive the stream. The RTP stack in libavformat for receiving requires all streams to be sent on unique ports. </dd> </dl>

Example command lines follow. To broadcast a stream on the local subnet, for watching in VLC: <pre class=”example-preformatted”>ffmpeg -re -i <var class=”var”>input</var> -f sap sap://224.0.0.255?same_port=1 </pre>

Similarly, for watching in <code class=”command”>ffplay</code>:   <pre class=”example-preformatted”>ffmpeg -re -i <var class=”var”>input</var> -f sap sap://224.0.0.255 </pre>

And for watching in <code class=”command”>ffplay</code>, over IPv6:   <pre class=”example-preformatted”>ffmpeg -re -i <var class=”var”>input</var> -f sap sap://[ff0e::1:2:3:4] </pre>

 <h4 class=”subsection”>24.36.2 Demuxer# TOC</h4>

The syntax for a SAP url given to the demuxer is: <pre class=”example-preformatted”>sap://[<var class=”var”>address</var>][:<var class=”var”>port</var>] </pre>

<var class=”var”>address</var> is the multicast address to listen for announcements on, if omitted, the default 224.2.127.254 (sap.mcast.net) is used. <var class=”var”>port</var> is the port that is listened on, 9875 if

omitted. The demuxers listens for announcements on the given address and port. Once an announcement is received, it tries to receive that particular stream. Example command lines follow. To play back the first stream announced on the normal SAP multicast address: <pre class=”example-preformatted”>ffplay sap:// </pre>

To play back the first stream announced on one the default IPv6 SAP multicast address: <pre class=”example-preformatted”>ffplay sap://[ff0e::2:7ffe] </pre>

 <h3 class=”section”>24.37 sctp# TOC</h3>

Stream Control Transmission Protocol. The accepted URL syntax is: <pre class=”example-preformatted”>sctp://<var class=”var”>host</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>listen</samp></dt> <dd> If set to any value, listen for an incoming connection. Outgoing connection is done by default. </dd> <dt><samp class=”option”>max_streams</samp></dt> <dd> Set the maximum number of streams. By default no limit is set. </dd> </dl>

 <h3 class=”section”>24.38 shared# TOC</h3>

Thread-safe, persistent, cross-process cache wrapper for input streams. Caches the input stream to a file in the specified directory. This is similar to cache, but with a persistent cache on disk that may be shared between multiple processes or demuxers, even concurrently. The filename of the cache is obtained by hashing the input URL, so there is some risk of collision if the same URL is used for different content (e.g. as a result of extra POST data being used to select the stream ID). To avoid this, make sure to only use this protocol for URLs that uniquely identify the content; otherwise you may get a garbled mix of sources. This includes content selected by request options such as headers, cookies or credentials: data cached for one user is returned to every other user of the same cache directory. The accepted options are: <dl class=”table”> <dt><samp

class=”option”>cache_dir</samp></dt> <dd> Path to the directory where cache files are stored. This option is required. Anyone who can write to this directory or to <samp class=”option”>metadata_dir</samp> can alter the data read by all users of the cache, or make them crash, so neither may be writable by untrusted users. </dd> <dt><samp class=”option”>metadata_dir</samp></dt> <dd> Path to the directory where metadata files are stored. Optional. If not specified, metadata is stored in the same directory as the cache file. </dd> <dt><samp class=”option”>block_shift</samp></dt> <dd> Shift factor (log2) of the block size used for internal reads/writes. Defaults to 15, i.e. 32KB blocks. If this does not match the value that was specified when an existing cache file was created, the previously specified value will be used instead. </dd> <dt><samp class=”option”>cache_read_only</samp></dt> <dd> If true, use the shared cache for reads but don’t write any new blocks to it. Default is false. Note that even with this option enabled, the cache file will be initialized if it does not already exist. </dd> <dt><samp class=”option”>cache_verify</samp></dt> <dd> If true, verify any data read from the cache against the underlying input stream, and error out on any mismatches. Note that this will make the cache layer effectively useless. This is purely a debug option. </dd> <dt><samp class=”option”>cache_timeout</samp></dt> <dd> If set to a nonzero value, specifies the maximum time (in microseconds) to wait for data to become available, if another process is currently trying to fetch and cache the same block at the same time. If this timeout elapses, it’s assumed that the other process may have gotten stuck or died in the meantime. If set to zero, no waiting is done and all processes will immediately race to try and fetch the same missing blocks themselves. Defaults to 10000 (10 ms). </dd> <dt><samp class=”option”>ignore_errors</samp></dt> <dd> If true, failures of the underlying input stream are not treated as fatal, and playback continues using only the data already present in the cache file. Default is false. Any attempt to read into uncached data will result in an unrecoverable IO error. </dd> <dt><samp class=”option”>retry_errors</samp></dt> <dd> If true (the default), transient read errors from the underlying input stream are ignored and retried again. If false, any blocks that previously failed being read from will be treated as permanently inaccessible. </dd> <dt><samp class=”option”>retry_corrupt</samp></dt> <dd> If true (the default), blocks whose contents fail the CRC integrity check are re-fetched from the underlying input stream, overwriting the corrupt cached data. If false, cache corruption is treated as a fatal read error. </dd> <dt><samp class=”option”>cache_size_max</samp></dt> <dd> Maximum amount of data to cache, in bytes, rounded down to a multiple of the chosen block size. Accepts the usual size suffixes, e.g. <code class=”code”>10G</code> for 10*10^9 bytes, or <code class=”code”>10Gi</code> for 10*2^30 bytes. Defaults to 0, meaning no limit. Once this limit is reached, the protocol switches to read-only mode (see <samp class=”option”>cache_read_only</samp>).

Already cached data is never evicted, so once this limit is reached, no new data will ever be cached until the cache file is deleted. Note that this option is best-effort only, and may over-write up to N-1 blocks, where N is the number of concurrent processes using the same cache file. Note also that the underlying file system may reserve more space than written, if the underlying allocation granularity is larger than the block size. </dd> <dt><samp class="option">offset</samp></dt> <dd> Initial byte offset to start reading from. This option is not passed on to the underlying protocol; it instead gets a regular seek on the first cache miss. Defaults to 0. </dd> </dl>

URL Syntax is <pre class="example-preformatted">shared:<var class="var">URL</var> </pre>

 <h3 class="section">24.39 srt# TOC</h3>

Haivision Secure Reliable Transport Protocol via libsrt. The supported syntax for a SRT URL is: <pre class="example-preformatted">srt://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form

<var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class="code">AVOption</code>s).

The list of supported options follows.

<dl class="table">

<dt><samp class="option">connect_timeout=<var class="var">milliseconds</var></samp></dt>

<dd> Connection timeout; SRT cannot connect for RTT > 1500 msec

(2 handshake exchanges) with the default connect timeout of

3 seconds. This option applies to the caller and rendezvous

connection modes. The connect timeout is 10 times the value

set for the rendezvous mode (which can be used as a

workaround for this connection problem with earlier versions).

</dd>

<dt><samp class="option">ffs=<var class="var">bytes</var></samp></dt>

<dd> Flight Flag Size (Window Size), in bytes. FFS is actually an

internal parameter and you should set it to not less than

<samp class="option">recv_buffer_size</samp> and <samp class="option">mss</samp>. The default value

is relatively large, therefore unless you set a very large receiver buffer,

you do not need to change this option. Default value is 25600.

</dd>

<dt><samp class="option">inputbw=<var class="var">bytes/seconds</var></samp></dt>

<dd> Sender nominal input rate, in bytes per seconds. Used along with

<samp class="option">oheadbw</samp>, when <samp class="option">maxbw</samp> is set to relative (0), to

calculate maximum sending rate when recovery packets are sent

along with the main media stream:

<samp class="option">inputbw</samp> * (100 + <samp class="option">oheadbw</samp>) / 100

if <samp class="option">inputbw</samp> is not set while <samp class="option">maxbw</samp> is set to

relative (0), the actual input rate is evaluated inside

the library. Default value is 0.

</dd>

<dt><samp class="option">iptos=<var class="var">tos</var></samp></dt>

<dd> IP Type of Service. Applies to sender only. Default value is 0xB8.

</dd>

<dt><samp class="option">ipttl=<var class="var">ttl</var></samp></dt>

<dd> IP Time To Live. Applies to sender only. Default value is 64.

</dd>

<dt><samp class="option">latency=<var class="var">microseconds</var></samp></dt>

<dd> Timestamp-based Packet Delivery Delay.

Used to absorb bursts of missed packet retransmissions.

This flag sets both <samp class="option">rcvlatency</samp> and <samp class="option">peerlatency</samp>

to the same value. Note that prior to version 1.3.0

this is the only flag to set the latency, however

this is effectively equivalent to setting <samp class="option">peerlatency</samp>,

when side is sender and <samp class="option">rcvlatency</samp>

when side is receiver, and the bidirectional stream

sending is not supported.

</dd>

<dt><samp class="option">listen_timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set socket listen timeout.

</dd>

<dt><samp class="option">maxbw=<var class="var">bytes/seconds</var></samp></dt>

<dd> Maximum sending bandwidth, in bytes per seconds.

-1 infinite (CSRTCC limit is 30mbps)

0 relative to input rate (see <samp class="option">inputbw</samp>)

>0 absolute limit value

Default value is 0 (relative)

</dd>

<dt><samp class="option">mode=<var class="var">caller|listener|rendezvous</var></samp></dt>

<dd> Connection mode.

<samp class="option">caller</samp> opens client connection.

<samp class="option">listener</samp> starts server to listen for incoming connections.

<samp class="option">rendezvous</samp> use Rendez-Vous connection mode.

Default value is caller.

</dd>

<dt><samp class="option">mss=<var class="var">bytes</var></samp></dt>

<dd> Maximum Segment Size, in bytes. Used for buffer allocation

and rate calculation using a packet counter assuming fully

filled packets. The smallest MSS between the peers is

used. This is 1500 by default in the overall internet.

This is the maximum size of the UDP packet and can be

only decreased, unless you have some unusual dedicated

network settings. Default value is 1500.

</dd>

<dt><samp class="option">nakreport=<var class="var">1|0</var></samp></dt>

<dd> If set to 1, Receiver will send ‘UMSG_LOSSREPORT‘ messages

periodically until a lost packet is retransmitted or

intentionally dropped. Default value is 1.

</dd>

<dt><samp class="option">oheadbw=<var class="var">percents</var></samp></dt>

<dd> Recovery bandwidth overhead above input rate, in percents.

See <samp class="option">inputbw</samp>. Default value is 25%.

</dd>

<dt><samp class="option">passphrase=<var class="var">string</var></samp></dt>

<dd> HaiCrypt Encryption/Decryption Passphrase string, length

from 10 to 79 characters. The passphrase is the shared

secret between the sender and the receiver. It is used
to generate the Key Encrypting Key using PBKDF2
(Password-Based Key Derivation Function). It is used
only if <samp class="option">pbkeylen</samp> is non-zero. It is used on
the receiver only if the received data is encrypted.
The configured passphrase cannot be recovered (write-only).

</dd>
<dt><samp class="option">enforced_encryption=<var class="var">1|0</var></samp></dt>
<dd> If true, both connection parties must have the same password
set (including empty, that is, with no encryption). If the
password doesn’t match or only one side is unencrypted,
the connection is rejected. Default is true.

</dd>
<dt><samp class="option">kmrefreshrate=<var class="var">packets</var></samp></dt>
<dd> The number of packets to be transmitted after which the
encryption key is switched to a new key. Default is -1.
-1 means auto (0x1000000 in srt library). The range for
this option is integers in the 0 - <code class="code">INT_MAX</code>.

</dd>
<dt><samp class="option">kmpreannounce=<var class="var">packets</var></samp></dt>
<dd> The interval between when a new encryption key is sent and
when switchover occurs. This value also applies to the
subsequent interval between when switchover occurs and
when the old encryption key is decommissioned. Default is -1.
-1 means auto (0x1000 in srt library). The range for
this option is integers in the 0 - <code class="code">INT_MAX</code>.

</dd>

<dt><samp class=”option”>snddropdelay=<var class=”var”>microseconds</var></samp></dt>

<dd> The sender’s extra delay before dropping packets. This delay
is

added to the default drop delay time interval value.

Special value -1: Do not drop packets on the sender at all.

</dd>

<dt><samp class=”option”>payload_size=<var class=”var”>bytes</var></samp></dt>

<dd> Sets the maximum declared size of a packet transferred

during the single call to the sending function in Live

mode. Use 0 if this value isn’t used (which is default in

file mode).

Default is -1 (automatic), which typically means MPEG-TS;

if you are going to use SRT

to send any different kind of payload, such as, for example,

wrapping a live stream in very small frames, then you can

use a bigger maximum frame size, though not greater than

1456 bytes.

</dd>

<dt><samp class=”option”>pkt_size=<var class=”var”>bytes</var></samp></dt>

<dd> Alias for ‘<samp class=”samp”>payload_size</samp>’.

</dd>

<dt><samp class=”option”>peerlatency=<var class=”var”>microseconds</var></samp></dt>

<dd> The latency value (as described in <samp class=”option”>rcvlatency</samp>)
that is

set by the sender side as a minimum value for the receiver.

</dd>

<dt><samp class=”option”>pbkeylen=<var class=”var”>bytes</var></samp></dt>
<dd> Sender encryption key length, in bytes.
Only can be set to 0, 16, 24 and 32.
Enable sender encryption if not 0.
Not required on receiver (set to 0),
key size obtained from sender in HaiCrypt handshake.
Default value is 0.

</dd>
<dt><samp class=”option”>rcvlatency=<var class=”var”>microseconds</var></samp></dt>
<dd> The time that should elapse since the moment when the
packet was sent and the moment when it’s delivered to
the receiver application in the receiving function.
This time should be a buffer time large enough to cover
the time spent for sending, unexpectedly extended RTT
time, and the time needed to retransmit the lost UDP
packet. The effective latency value will be the maximum
of this options’ value and the value of <samp class=”option”>peerlatency</samp>
set by the peer side. Before version 1.3.0 this option
is only available as <samp class=”option”>latency</samp>.

</dd>
<dt><samp class=”option”>recv_buffer_size=<var class=”var”>bytes</var></samp></dt>
<dd> Set UDP receive buffer size, expressed in bytes.

</dd>
<dt><samp class=”option”>send_buffer_size=<var class=”var”>bytes</var></samp></dt>
<dd> Set UDP send buffer size, expressed in bytes.

</dd>
<dt><samp class=”option”>timeout=<var class=”var”>microseconds</var></samp></dt>

<dd> Set raise error timeouts for read, write and connect operations. Note
that the

SRT library has internal timeouts which can be controlled separately, the

value set here is only a cap on those.

</dd>

<dt><samp class="option">tlpktdrop=<var class="var">1|0</var></samp></dt>

<dd> Too-late Packet Drop. When enabled on receiver, it skips

missing packets that have not been delivered in time and

delivers the following packets to the application when

their time-to-play has come. It also sends a fake ACK to

the sender. When enabled on sender and enabled on the

receiving peer, the sender drops the older packets that

have no chance of being delivered in time. It was

automatically enabled in the sender if the receiver

supports it.

</dd>

<dt><samp class="option">sndbuf=<var class="var">bytes</var></samp></dt>

<dd> Set send buffer size, expressed in bytes.

</dd>

<dt><samp class="option">rcvbuf=<var class="var">bytes</var></samp></dt>

<dd> Set receive buffer size, expressed in bytes.

Receive buffer must not be greater than <samp class="option">ffs</samp>.

</dd>

<dt><samp class="option">lossmaxttl=<var class="var">packets</var></samp></dt>

<dd> The value up to which the Reorder Tolerance may grow. When

Reorder Tolerance is > 0, then packet loss report is delayed

until that number of packets come in. Reorder Tolerance
increases every time a "belated" packet has come, but it
wasn’t due to retransmission (that is, when UDP packets tend
to come out of order), with the difference between the latest
sequence and this packet’s sequence, and not more than the
value of this option. By default it’s 0, which means that this
mechanism is turned off, and the loss report is always sent
immediately upon experiencing a "gap" in sequences.

</dd>
<dt><samp class="option">minversion</samp></dt>
<dd> The minimum SRT version that is required from the peer. A connection
to a peer that does not satisfy the minimum version requirement
will be rejected.

The version format in hex is 0xXXYYZZ for x.y.z in human readable
form.

</dd>
<dt><samp class="option">streamid=<var class="var">string</var></samp></dt>
<dd> A string limited to 512 characters that can be set on the socket prior
to connecting. This stream ID will be able to be retrieved by the
listener side from the socket that is returned from srt_accept and
was connected by a socket with that set stream ID. SRT does not enforce
any special interpretation of the contents of this string.
This option doesn't make sense in Rendezvous connection; the result
might be that simply one side will override the value from the other
side and it's the matter of luck which one would win

</dd>

<dt><samp class="option">srt_streamid=<var class="var">string</var></samp></dt>

<dd> Alias for ‘<samp class="samp">streamid</samp>’
to avoid conflict with ffmpeg command line option.

</dd>

<dt><samp class="option">smoother=<var class="var">live|file</var></samp></dt>

<dd> The type of Smoother used for the transmission for that socket, which

is responsible for the transmission and congestion control. The Smoother

type must be exactly the same on both connecting parties, otherwise

the connection is rejected.

</dd>

<dt><samp class="option">messageapi=<var class="var">1|0</var></samp></dt>

<dd> When set, this socket uses the Message API, otherwise it uses Buffer

API. Note that in live mode (see <samp class="option">transtype</samp>)
there's only

message API available. In File mode you can chose to use one of two modes:

Stream API (default, when this option is false). In this mode you may

send as many data as you wish with one sending instruction, or even use

dedicated functions that read directly from a file. The internal facility

will take care of any speed and congestion control. When receiving, you

can also receive as many data as desired, the data not extracted will be

waiting for the next call. There is no boundary between data portions in

the Stream mode.

Message API. In this mode your single sending instruction passes exactly

one piece of data that has boundaries (a message). Contrary to Live mode,

this message may span across multiple UDP packets and the only size

limitation is that it shall fit as a whole in the sending buffer. The

receiver shall use as large buffer as necessary to receive the message,

otherwise the message will not be given up. When the message is not
complete (not all packets received or there was a packet loss) it will
not be given up.

</dd>
<dt><samp class="option">transtype=<var class="var">live|file</var></samp></dt>
<dd> Sets the transmission type for the socket, in particular, setting this
option sets multiple other parameters to their default values as required
for a particular transmission type.

live: Set options as for live transmission. In this mode, you should
send by one sending instruction only so many data that fit in one UDP packet,
and limited to the value defined first in <samp class="option">payload_size</samp>
(1316 is
default in this mode). There is no speed control in this mode, only the
bandwidth control, if configured, in order to not exceed the bandwidth with
the overhead transmission (retransmitted and control packets).

file: Set options as for non-live transmission. See <samp class="option">messageapi</samp>
for further explanations

</dd>
<dt><samp class="option">linger=<var class="var">seconds</var></samp></dt>
<dd> The number of seconds that the socket waits for unsent data when
closing.
Default is -1. -1 means auto (off with 0 seconds in live mode, on with 180
seconds in file mode). The range for this option is integers in the
0 - <code class="code">INT_MAX</code>.

</dd>
<dt><samp class="option">tsbpd=<var class="var">1|0</var></samp></dt>

<dd> When true, use Timestamp-based Packet Delivery mode. The default behavior

depends on the transmission type: enabled in live mode, disabled in file

mode.

</dd>

<dt><samp class="option">ipv6only=<var class="var">1|0</var></samp></dt>

<dd> Accept IPv4 or not while using the IPv6 wildcard address. This should be set

when listening on a IPv6 wildcard address.

</dd>

</dl>

For more information see: <a class="url" href="https://github.com/Haivision/srt">https://github.com/H
  <h3 class="section">24.40 srtp# TOC</h3>

Secure Real-time Transport Protocol. The accepted options are:
<dl class="table"> <dt><samp class="option">srtp_in_suite</samp></dt>
<dt><samp class="option">srtp_out_suite</samp></dt> <dd> Select input and output encoding suites. Supported values: <dl class="table"> <dt>‘<samp class="samp">AES_CM_128_HMAC_SHA1_80</samp>’</dt>
<dt>‘<samp class="samp">SRTP_AES128_CM_HMAC_SHA1_80</samp>’</dt>
<dt>‘<samp class="samp">AES_CM_128_HMAC_SHA1_32</samp>’</dt>
<dt>‘<samp class="samp">SRTP_AES128_CM_HMAC_SHA1_32</samp>’</dt>
</dl>

</dd> <dt><samp class="option">srtp_in_params</samp></dt>
<dt><samp class="option">srtp_out_params</samp></dt> <dd> Set input and output encoding parameters, which are expressed by a base64-encoded representation of a binary block. The first 16 bytes of this binary block are used as master key, the following 14 bytes are used as master salt. </dd>
</dl>

 <h3 class="section">24.41 subfile#
TOC</h3>

Virtually extract a segment of a file or another stream. The underlying stream must be seekable. Accepted options:

<dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Start offset of the extracted segment, in bytes. </dd> <dt><samp class="option">end</samp></dt> <dd> End offset of the extracted segment, in bytes. If set to 0, extract till end of file. </dd> </dl>

Examples: Extract a chapter from a DVD VOB file (start and end sectors obtained externally and multiplied by 2048): <pre class="example-preformatted">subfile,,start,153391104,end,268142592,,:/media/dvd/ </pre>

Play an AVI file directly from a TAR archive: <pre class="example-preformatted">subfile,,start,183241728,end,366490624,,:archive.tar </pre>

Play a MPEG-TS file from start offset till end: <pre class="example-preformatted">subfile,,start,32815239,end,0,,:video.ts </pre>

 <h3 class="section">24.42 tee# TOC</h3>

Writes the output to multiple protocols. The individual outputs are separated by | <pre class="example-preformatted">tee:file://path/to/local/this.avi|file://path/to/local/that.avi </pre>

 <h3 class="section">24.43 tcp# TOC</h3>

Transmission Control Protocol. The required syntax for a TCP url is: <pre class="example-preformatted">tcp://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">listen=<var class="var">2|1|0</var></samp></dt> <dd> Listen for an incoming connection. 0 disables listen, 1 enables listen in single client mode, 2 enables listen in multi-client mode. Default value is 0. </dd> <dt><samp class="option">local_addr=<var class="var">addr</var></samp></dt> <dd> Local IP address of a network interface used for tcp socket connect. </dd> <dt><samp class="option">local_port=<var class="var">port</var></samp></dt>

<dd> Local port used for tcp socket connect. </dd> <dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt> <dd> Set raise error timeout, expressed in microseconds. This option is only relevant in read mode: if no data arrived in more than this time interval, raise error. </dd> <dt><samp class="option">listen_timeout=<var class="var">milliseconds</var></samp></dt> <dd> Set listen timeout, expressed in milliseconds. </dd> <dt><samp class="option">recv_buffer_size=<var class="var">bytes</var></samp></dt> <dd> Set receive buffer size, expressed bytes. </dd> <dt><samp class="option">send_buffer_size=<var class="var">bytes</var></samp></dt> <dd> Set send buffer size, expressed bytes. </dd> <dt><samp class="option">tcp_nodelay=<var class="var">1|0</var></samp></dt> <dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0. Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY. </dd> <dt><samp class="option">tcp_mss=<var class="var">bytes</var></samp></dt> <dd> Set maximum segment size for outgoing TCP packets, expressed in bytes. </dd> </dl>

The following example shows how to setup a listening TCP connection with <code class="command">ffmpeg</code>, which is then accessed with <code class="command">ffplay</code>: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> tcp://<var class="var">hostname</var>:<var class="var">port</var>?listen ffplay tcp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">24.44 tls# TOC</h3>

Transport Layer Security (TLS) / Secure Sockets Layer (SSL) The required syntax for a TLS/SSL url is: <pre class="example-preformatted">tls://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">ca_file, cafile=<var class="var">filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and

setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class="option">tls_verify, verify=<var class="var">1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. (With other backends, the host name is validated as well.) This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class="option">cert_file, cert=<var class="var">filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class="option">key_file, key=<var class="var">filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class="option">listen=<var class="var">1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class="option">http_proxy</samp></dt> <dd> The HTTP proxy to tunnel through, e.g. <code class="code">http://example.com:1234</code>. The proxy must support the CONNECT method. </dd> </dl>

Example command lines: To create a TLS/SSL server that serves an input stream.   <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> tls://<var class="var">hostname</var>:<var class="var">port</var>?listen&cert=<var class="var">server.crt</var>&key=<var class="var">server.key</var> </pre>

To play back a stream from the TLS/SSL server using <code class="command">ffplay</code>:   <pre class="example-preformatted">ffplay tls://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">24.45 dtls# TOC</h3>

Datagram Transport Layer Security (DTLS) The required syntax for a DTLS URL is: <pre class="example-preformatted">dtls://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to

escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). DTLS shares most options with TLS, but operates over UDP instead of TCP. The list of supported options follows. <dl class="table"> <dt><samp class="option">ca_file, cafile=<var class="var">filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class="option">tls_verify, verify=<var class="var">1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class="option">cert_file, cert=<var class="var">filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class="option">key_file, key=<var class="var">filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class="option">cert_pem=<var class="var">string</var></samp></dt> <dd> Certificate PEM string </dd> <dt><samp class="option">key_pem=<var class="var">string</var></samp></dt> <dd> Private key PEM string </dd> <dt><samp class="option">listen=<var class="var">1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class="option">mtu=<var class="var">size</var></samp></dt> <dd> Set the Maximum Transmission Unit (MTU) for DTLS packets. </dd> <dt><samp class="option">use_srtp=<var class="var">1|0</var></samp></dt> <dd> Enable the use_srtp DTLS extension. This is used in WebRTC applications to establish SRTP encryption keys through the DTLS handshake. Default is disabled. </dd> <dt><samp class="option">external_sock=<var class="var">1|0</var></samp></dt> <dd> Use an external socket instead of creating a new one. This option only makes sense to pass when interacting with the code via API, enabling this from CLI will cause immediate failure. Default is disabled. </dd> </dl>

Example command lines: To create a DTLS server: <pre class="example-preformatted">ffmpeg -listen 1 -i dtls://<var class="var">hostname</var>:<var class="var">port</var>

<var class=”var”>output</var> </pre>

To create a DTLS client and send data to server: <pre class=”example-preformatted”>ffmpeg -i <var class=”var”>input</var> -f <var class=”var”>format</var> dtls://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>24.46 udp# TOC</h3>

User Datagram Protocol. The required syntax for an UDP URL is: <pre class=”example-preformatted”>udp://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form

<var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class=”code”>AVOption</code>s).

In case threading is enabled on the system, a circular buffer is used

to store the incoming data, which allows one to reduce loss of data due to

UDP socket buffer overruns. The <var class=”var”>fifo_size</var> and

<var class=”var”>overrun_nonfatal</var> options are related to this buffer.

The list of supported options follows.

<dl class=”table”>

<dt><samp class=”option”>buffer_size=<var class=”var”>size</var></samp></dt>

<dd> Set the UDP maximum socket buffer size in bytes. This is used to set either

the receive or send buffer size, depending on what the socket is used for.

Default is 32 KB for output, 384 KB for input. See also <var class=”var”>fifo_size</var>.

</dd>

<dt><samp class="option">bitrate=<var class="var">bitrate</var></samp></dt>

<dd> If set to nonzero, the output will have the specified constant bitrate if the

input has enough packets to sustain it.

</dd>

<dt><samp class="option">burst_bits=<var class="var">bits</var></samp></dt>

<dd> When using <var class="var">bitrate</var> this specifies the maximum number of bits in

packet bursts.

</dd>

<dt><samp class="option">localport=<var class="var">port</var></samp></dt>

<dd> Override the local UDP port to bind with.

</dd>

<dt><samp class="option">localaddr=<var class="var">addr</var></samp></dt>

<dd> Local IP address of a network interface used for sending packets or joining

multicast groups.

</dd>

<dt><samp class="option">pkt_size=<var class="var">size</var></samp></dt>

<dd> Set the size in bytes of UDP packets.

</dd>

<dt><samp class="option">reuse=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow reusing UDP sockets.

</dd>

<dt><samp class=”option”>ttl=<var class=”var”>ttl</var></samp></dt>

<dd> Set the time to live value (for multicast only).

</dd>

<dt><samp class=”option”>dscp=<var class=”var”>dscp</var></samp></dt>

<dd> Set the 6-bit DSCP field for outgoing packets.

</dd>

<dt><samp class=”option”>connect=<var class=”var”>1|0</var></samp></dt>

<dd> Initialize the UDP socket with <code class=”code”>connect()</code>. In this case, the

destination address can’t be changed with ff_udp_set_remote_url later.

If the destination address isn’t known at the start, this option can

be specified in ff_udp_set_remote_url, too.

This allows finding out the source address for the packets with getsockname,

and makes writes return with AVERROR(ECONNREFUSED) if "destination

unreachable" is received.

For receiving, this gives the benefit of only receiving packets from

the specified peer address/port.

</dd>

<dt><samp class=”option”>sources=<var class=”var”>address</var>[,<var class=”var”>address</var>]</samp></dt>

<dd> Only receive packets sent from the specified addresses. In case of multicast,

also subscribe to multicast traffic coming from these addresses only.

</dd>

<dt><samp class=”option”>block=<var class=”var”>address</var>[,<var class=”var”>address</var>]</samp></dt>

<dd> Ignore packets sent from the specified addresses. In case of multicast, also

exclude the source addresses in the multicast subscription.

</dd>

<dt><samp class="option">fifo_size=<var class="var">units</var></samp></dt>

<dd> Set the UDP receiving circular buffer size, expressed as a number of

packets with size of 188 bytes. If not specified defaults to 7*4096.

Set to 0 to disable the receiving circular buffer and its receive thread.

</dd>

<dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt>

<dd> Survive in case of UDP receiving circular buffer overrun. Default

value is 0.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeout, expressed in microseconds.

This option is only relevant in read mode: if no data arrived in more

than this time interval, raise error.

</dd>

<dt><samp class="option">broadcast=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow UDP broadcasting.

Note that broadcasting may not work properly on networks having

a broadcast storm protection.

</dd>

</dl>

 <h4 class="subsection">24.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to stream over UDP to a remote endpoint: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> udp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

Use <code class="command">ffmpeg</code> to stream in mpegts format over UDP using 188 sized UDP packets, using a large input buffer: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f mpegts udp://<var class="var">hostname</var>:<var class="var">port</var>?pkt_size=188&buffer_size=65535 </pre>

Use <code class="command">ffmpeg</code> to receive over UDP from a remote endpoint: <pre class="example-preformatted">ffmpeg -i udp://[<var class="var">multicast-address</var>]:<var class="var">port</var> ... </pre>

 <h3 class="section">24.47 unix# TOC</h3>

Unix local socket The required syntax for a Unix socket URL is: <pre class="example-preformatted">unix://<var class="var">filepath</var> </pre>

The following parameters can be set via command line options (or in code via <code class="code">AVOption</code>s): <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Timeout in ms. </dd> <dt><samp class="option">listen</samp></dt> <dd> Create the Unix socket in listening mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Select socket type. <dl class="table"> <dt>‘<samp class="samp">stream Corresponds to SOCK_STREAM (reliable stream-oriented)</samp>’</dt> <dt>‘<samp class="samp">datagram Corresponds to SOCK_DGRAM (unreliable packet-oriented)</samp>’</dt> <dt>‘<samp class="samp">seqpacket Corresponds to SOCK_SEQPACKET (reliable packet-oriented)</samp>’</dt> </dl> </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Maximum packet size for packet-oriented sockets (SOCK_DGRAM and SOCK_SEQPACKET). If greater than zero, this value is used as <code class="code">max_packet_size</code>. Ignored for SOCK_STREAM. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">24.48 zmq# TOC</h3>

ZeroMQ asynchronous messaging using the libzmq library. This library supports unicast streaming to multiple clients without relying

on an external server. The required syntax for streaming or connecting to a stream is: <pre class="example-preformatted">zmq:tcp://ip-address:port </pre>

Example: Create a localhost stream on port 5555: <pre class="example-preformatted">ffmpeg -re -i input -f mpegts zmq:tcp://127.0.0.1:5555 </pre>

Multiple clients may connect to the stream using: <pre class="example-preformatted">ffplay zmq:tcp://127.0.0.1:5555 </pre>

Streaming to multiple clients is implemented using a ZeroMQ Pub-Sub pattern. The server side binds to a port and publishes data. Clients connect to the server (via IP address/port) and subscribe to the stream. The order in which the server and client start generally does not matter. ffmpeg must be compiled with the –enable-libzmq option to support this protocol. Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line. The following options are supported: <dl class="table"> <dt><samp class="option">pkt_size</samp></dt> <dd> Forces the maximum packet size for sending/receiving data. The default value is 131,072 bytes. On the server side, this sets the maximum size of sent packets via ZeroMQ. On the clients, it sets an internal buffer size for receiving packets. Note that pkt_size on the clients should be equal to or greater than pkt_size on the server. Otherwise the received message may be truncated causing decoding errors. </dd> </dl>

 <h2 class="chapter">25 Device Options# TOC</h2>

The libavdevice library provides the same interface as libavformat. Namely, an input device is considered like a demuxer, and an output device like a muxer, and the interface and generic device options are the same provided by libavformat (see the ffmpeg-formats manual). In addition each input or output device may support so-called private options, which are specific for that component. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the device <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use.

 <h2 class="chapter">26 Input Devices# TOC</h2>

Input devices are configured elements in FFmpeg which enable accessing the data coming from a multimedia device attached to your system. When you configure your FFmpeg build, all the supported input devices are enabled by default. You can list all available ones using the configure option "–list-indevs". You can disable all the input devices using the configure option "–disable-indevs", and selectively enable an input device using the option "–enable-indev=<var class="var">INDEV</var>", or you can disable a particular input device using the option "–disable-indev=<var class="var">INDEV</var>". The option "-devices" of the ff* tools will display the list of supported input devices. A description of the currently available input devices follows. <h3 class="section">26.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) input device. To enable this input device during configuration you need libasound installed on your system. This device allows capturing from an ALSA device. The name of the device to capture has to be an ALSA card identifier. An ALSA identifier has the syntax: <pre class="example-preformatted">hw:<var class="var">CARD</var>[,<var class="var">DEV</var>[,<var class="var">SUBDEV</var>]] </pre>

where the <var class="var">DEV</var> and <var class="var">SUBDEV</var> components are optional. The three arguments (in order: <var class="var">CARD</var>,<var class="var">DEV</var>,<var class="var">SUBDEV</var>) specify card number or identifier, device number and subdevice number (-1 means any). To see the list of cards currently recognized by your system check the files <samp class="file">/proc/asound/cards</samp> and <samp class="file">/proc/asound/devices</samp>. For example to capture with <code class="command">ffmpeg</code> from an ALSA device with card id 0, you may run the command: <pre class="example-preformatted">ffmpeg -f alsa -i hw:0 alsaout.wav </pre>

For more information see: http://www.alsa-project.org/alsa-doc/alsa-lib/pcm.html <h4 class="subsection">26.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">ch_layout</samp></dt> <dd> Set the channel layout. When unset, the layout defaults to stereo, or to the minimum

channel count supported by the device, and is filled in from the ALSA channel map on a best-effort basis. </dd> </dl>

 <h3 class="section">26.2 android_camera# TOC</h3>

Android camera input device. This input devices uses the Android Camera2 NDK API which is available on devices with API level 24+. The availability of android_camera is autodetected during configuration. This device allows capturing from all cameras on an Android device, which are integrated into the Camera2 NDK API. The available cameras are enumerated internally and can be selected with the <var class="var">camera_index</var> parameter. The input file string is discarded. Generally the back facing camera has index 0 while the front facing camera has index 1. <h4 class="subsection">26.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Set the video size given as a string such as 640x480 or hd720. Falls back to the first available configuration reported by Android if requested video size is not available or by default. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the video framerate. Falls back to the first available configuration reported by Android if requested framerate is not available or by default (-1). </dd> <dt><samp class="option">camera_index</samp></dt> <dd> Set the index of the camera to use. Default is 0. </dd> <dt><samp class="option">input_queue_size</samp></dt> <dd> Set the maximum number of frames to buffer. Default is 5. </dd> </dl>

 <h3 class="section">26.3 avfoundation# TOC</h3>

AVFoundation input device. AVFoundation is the currently recommended framework by Apple for streamgrabbing on OSX >= 10.7 as well as on iOS. The input filename has to be given in the following syntax: <pre class="example-preformatted">-i "[[VIDEO]:[AUDIO]]" </pre> The first entry selects the video input while the latter selects the audio input. The stream has to be specified by the device name or the device index as shown by the device list. Alternatively, the video and/or audio input device can be chosen by index using the <samp class="option"> -video_device_index <INDEX> </samp>

and/or <samp class="option"> -audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class="option">-list_devices true</samp>, listing all device names and corresponding indices. There are two device name aliases: <dl class="table"> <dt><code class="code">default</code></dt> <dd> Select the AVFoundation default device of the corresponding type. </dd> <dt><code class="code">none</code></dt> <dd> Do not record the corresponding media type. This is equivalent to specifying an empty device name or index. </dd> </dl>

 <h4 class="subsection">26.3.1 Options# TOC</h4>

AVFoundation supports the following options: <dl class="table"> <dt><samp class="option">-list_devices <TRUE|FALSE></samp></dt> <dd> If set to true, a list of all available input devices is given showing their index, name, unique ID, and USB serial number (when available). </dd> <dt><samp class="option">-video_device_index <INDEX></samp></dt> <dd> Specify the video device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-video_device_id <ID></samp></dt> <dd> Specify the video device by a prefixed identifier, as shown by <samp class="option">-list_devices</samp>: <code class="code">uid:<UID></code> selects by AVFoundation unique ID, <code class="code">serial:<SERIAL></code> by USB serial number. The unique ID of a UVC device may not be stable across reboots. Only UVC devices expose a serial number. Overrides anything given in the input filename. </dd> <dt><samp class="option">-audio_device_id <ID></samp></dt> <dd> Specify the audio device by a prefixed identifier. See <samp class="option">-video_device_id</samp>; for audio, the serial is only available for class-compliant USB devices. </dd> <dt><samp class="option">-pixel_format <FORMAT></samp></dt> <dd> Request the video device to use a specific pixel format. If the specified format is not supported, a list of available formats is given and the first one in this list is used instead. Available pixel formats are: <code class="code">monob, rgb555be, rgb555le, rgb565be, rgb565le, rgb24, bgr24, 0rgb, bgr0, 0bgr, rgb0, bgr48be, uyvy422, yuva444p, yuva444p16le, yuv444p, yuv422p16, yuv422p10, yuv444p10, yuv420p, nv12, yuyv422, gray</code> </dd> <dt><samp class="option">-framerate</samp></dt> <dd> Set the grabbing frame rate. Default is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd>

<dt><samp class="option">-video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class="option">-capture_cursor</samp></dt> <dd> Capture the mouse pointer. Default is 0. </dd> <dt><samp class="option">-capture_mouse_clicks</samp></dt> <dd> Capture the screen mouse clicks. Default is 0. </dd> <dt><samp class="option">-capture_raw_data</samp></dt> <dd> Capture the raw device data. Default is 0. Using this option may result in receiving the underlying data delivered to the AVFoundation framework. E.g. for muxed devices that sends raw DV data to the framework (like tape-based camcorders), setting this option to false results in extracted video frames captured in the designated pixel format only. Setting this option to true results in receiving the raw DV stream untouched. </dd> </dl>

 <h4 class="subsection">26.3.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of AVFoundation supported devices and exit: <pre class="example-preformatted">$ ffmpeg -f avfoundation -list_devices true -i "" </pre>

Record video from video device 0 and audio from audio device 0 into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -i "0:0" out.avi </pre>

Record video from video device 2 and audio from audio device 1 into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -video_device_index 2 -i ":1" out.avi </pre>

Record video from the system default video device using the pixel format bgr0 and do not record any audio into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -pixel_format bgr0 -i "default:none" out.avi </pre>

Record raw DV data from a suitable input device and write the output into out.dv: <pre class="example-preformatted">$ ffmpeg -f avfoundation -capture_raw_data true -i "zr100:none" out.dv </pre>

 <h3 class="section">26.4 decklink# TOC</h3>

The decklink input device provides capture capabilities for Blackmagic

DeckLink devices. To enable this input device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class=”code”>--extra-cflags</code>, <code class=”code”>--extra-cxxflags</code> and <code class=”code”>--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class=”command”>widl</code>. DeckLink is very picky about the formats it supports. Pixel format of the input can be set with <samp class=”option”>raw_format</samp>. Framerate and video size must be determined for your device with <code class=”command”>-list_formats 1</code>. Audio sample rate is always 48 kHz and the number of channels can be 2, 8 or 16. Note that all audio channels are bundled in one single audio track. <h4 class=”subsection”>26.4.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>list_devices</samp></dt> <dd> If set to <samp class=”option”>true</samp>, print a list of devices and exit. Defaults to <samp class=”option”>false</samp>. This option is deprecated, please use the <code class=”code”>-sources</code> option of ffmpeg to list the available input devices. </dd> <dt><samp class=”option”>list_formats</samp></dt> <dd> If set to <samp class=”option”>true</samp>, print a list of supported formats and exit. Defaults to <samp class=”option”>false</samp>. </dd> <dt><samp class=”option”>format_code <FourCC></samp></dt> <dd> This sets the input video format to the format given by the FourCC. To see the supported values of your device(s) use <samp class=”option”>list_formats</samp>. Note that there is a FourCC <samp class=”option”>'pal '</samp> that can also be used as <samp class=”option”>pal</samp> (3 letters). Default behavior is autodetection of the input video format, if the hardware supports it. </dd> <dt><samp class=”option”>raw_format</samp></dt> <dd> Set the pixel format of the captured video. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> This is the default which means 8-bit YUV 422 or 8-bit ARGB if format autodetection is used, 8-bit YUV 422 otherwise. </dd> <dt>‘<samp class=”samp”>uyvy422</samp>’</dt> <dd> 8-bit YUV 422. </dd> <dt>‘<samp class=”samp”>yuv422p10</samp>’</dt> <dd> 10-bit YUV 422. </dd> <dt>‘<samp class=”samp”>argb</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class=”samp”>rgb10</samp>’</dt> <dd> 10-bit RGB. </dd> </dl>

</dd> <dt><samp class=”option”>teletext_lines</samp></dt> <dd> If set to nonzero, an additional teletext stream will be captured from the vertical ancillary data. Both SD PAL (576i) and HD (1080i or 1080p) sources are

supported. In case of HD sources, OP47 packets are decoded. This option is a bitmask of the SD PAL VBI lines captured, specifically lines 6 to 22, and lines 318 to 335. Line 6 is the LSB in the mask. Selected lines which do not contain teletext information will be ignored. You can use the special <samp class="option">all</samp> constant to select all possible lines, or <samp class="option">standard</samp> to skip lines 6, 318 and 319, which are not compatible with all receivers. For SD sources, ffmpeg needs to be compiled with <code class="code">--enable-libzvbi</code>. For HD sources, on older (pre-4K) DeckLink card models you have to capture in 10 bit mode. </dd> <dt><samp class="option">channels</samp></dt> <dd> Defines number of audio channels to capture. Must be ‘<samp class="samp">2</samp>’, ‘<samp class="samp">8</samp>’ or ‘<samp class="samp">16</samp>’. Defaults to ‘<samp class="samp">2</samp>’. </dd> <dt><samp class="option">duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’, ‘<samp class="samp">one_sub_device_full</sa ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Defaults to ‘<samp class="samp">unset</samp>’. Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0): ‘<samp class="samp">one_sub_device_full</samp>’, ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’ </dd> <dt><samp class="option">timecode_format</samp>< <dd> Timecode type to include in the frame and video stream metadata. Must be ‘<samp class="samp">none</samp>’, ‘<samp class="samp">rp188vitc</samp>’, ‘<samp class="samp">rp188vitc2</samp>’, ‘<samp class="samp">rp188ltc</samp>’, ‘<samp class="samp">rp188hfr</samp>’, ‘<samp class="samp">rp188any</samp>’, ‘<samp class="samp">vitc</samp>’, ‘<samp class="samp">vitc2</samp>’, or ‘<samp class="samp">serial</samp>’. Defaults to ‘<samp class="samp">none</samp>’ (not included). In order to properly support 50/60 fps timecodes, the ordering of the queried timecode types for ‘<samp class="samp">rp188any</samp>’ is HFR, VITC1, VITC2 and LTC for >30 fps content. Note that this is slightly different to the ordering

used by the DeckLink API, which is HFR, VITC1, LTC, VITC2. </dd>
<dt><samp class="option">video_input</samp></dt> <dd> Sets the
video input source. Must be ‘<samp class="samp">unset</samp>’,
‘<samp class="samp">sdi</samp>’, ‘<samp class="samp">hdmi</samp>’,
‘<samp class="samp">optical_sdi</samp>’, ‘<samp
class="samp">component</samp>’, ‘<samp class="samp">composite</samp>’
or ‘<samp class="samp">s_video</samp>’. Defaults
to ‘<samp class="samp">unset</samp>’. </dd>
<dt><samp class="option">audio_input</samp></dt> <dd> Sets the
audio input source. Must be ‘<samp class="samp">unset</samp>’,
‘<samp class="samp">embedded</samp>’, ‘<samp
class="samp">aes_ebu</samp>’, ‘<samp class="samp">analog</samp>’,
‘<samp class="samp">analog_xlr</samp>’, ‘<samp
class="samp">analog_rca</samp>’ or ‘<samp class="samp">microphone</samp>’.
Defaults to ‘<samp class="samp">unset</samp>’.
</dd> <dt><samp class="option">video_pts</samp></dt> <dd> Sets
the video packet timestamp source. Must be ‘<samp class="samp">video</samp>’,
‘<samp class="samp">audio</samp>’, ‘<samp
class="samp">reference</samp>’, ‘<samp class="samp">wallclock</samp>’
or ‘<samp class="samp">abs_wallclock</samp>’. Defaults
to ‘<samp class="samp">video</samp>’. </dd>
<dt><samp class="option">audio_pts</samp></dt> <dd> Sets the
audio packet timestamp source. Must be ‘<samp class="samp">video</samp>’,
‘<samp class="samp">audio</samp>’, ‘<samp
class="samp">reference</samp>’, ‘<samp class="samp">wallclock</samp>’
or ‘<samp class="samp">abs_wallclock</samp>’. Defaults
to ‘<samp class="samp">audio</samp>’. </dd>
<dt><samp class="option">draw_bars</samp></dt> <dd> If
set to ‘<samp class="samp">true</samp>’, color bars
are drawn in the event of a signal loss. Defaults to ‘<samp
class="samp">true</samp>’. This option is deprecated, please
use the <code class="code">signal_loss_action</code> option.
</dd> <dt><samp class="option">signal_loss_action</samp></dt>
<dd> Sets the action to take in the event of a signal loss. Accepts one of the
following values: <dl class="table"> <dt><samp class="option">1,
none</samp></dt> <dd> Do nothing on signal loss. This usually
results in black frames. </dd> <dt><samp class="option">2,
bars</samp></dt> <dd> Draw color bars on signal loss. Only sup-
ported for 8-bit input signals. </dd> <dt><samp class="option">3,
repeat</samp></dt> <dd> Repeat the last video frame on signal loss.
</dd> </dl>

Defaults to ‘<samp class="samp">bars</samp>’.
 </dd> <dt><samp class="option">queue_size</samp></dt>
<dd> Sets maximum input buffer size in bytes. If the buffering reaches
this value, incoming frames will be dropped. Defaults to ‘<samp

class=”samp”>1073741824</samp>’. </dd> <dt><samp class=”option”>audio_depth</samp></dt> <dd> Sets the audio sample bit depth. Must be ‘<samp class=”samp”>16</samp>’ or ‘<samp class=”samp”>32</samp>’. Defaults to ‘<samp class=”samp”>16</samp>’. </dd> <dt><samp class=”option”>decklink_copyts</samp></
<dd> If set to <samp class=”option”>true</samp>, timestamps are forwarded as they are without removing the initial offset. Defaults to <samp class=”option”>false</samp>. </dd> <dt><samp class=”option”>timestamp_align</samp></dt> <dd> Capture start time alignment in seconds. If set to nonzero, input frames are dropped till the system timestamp aligns with configured value. Alignment difference of up to one frame duration is tolerated. This is useful for maintaining input synchronization across N different hardware devices deployed for ’N-way’ redundancy. The system time of different hardware devices should be synchronized with protocols such as NTP or PTP, before using this option. Note that this method is not foolproof. In some border cases input synchronization may not happen due to thread scheduling jitters in the OS. Either sync could go wrong by 1 frame or in a rarer case <samp class=”option”>timestamp_align</samp> seconds. Defaults to ‘<samp class=”samp”>0</samp>’. </dd> <dt><samp class=”option”>wait_for_tc (bool)</samp></dt>
<dd> Drop frames till a frame with timecode is received. Sometimes serial timecode isn’t received with the first input frame. If that happens, the stored stream timecode will be inaccurate. If this option is set to <samp class=”option”>true</samp>, input frames are dropped till a frame with timecode is received. Option <var class=”var”>timecode_format</var> must be specified. Defaults to <samp class=”option”>false</samp>. </dd>
<dt><samp class=”option”>enable_klv(bool)</samp></dt>
<dd> If set to <samp class=”option”>true</samp>, extracts KLV data from VANC and outputs KLV packets. KLV VANC packets are joined based on MID and PSC fields and aggregated into one KLV packet. Defaults to <samp class=”option”>false</samp>. </dd> </dl>

 <h4 class=”subsection”>26.4.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> List input devices:
<pre class=”example-preformatted”>ffmpeg -sources decklink </pre>

List supported formats: <pre class=”example-preformatted”>ffmpeg -f decklink -list_formats 1 -i 'Intensity Pro' </pre>

Capture video clip at 1080i50: <pre class=”example-preformatted”>ffmpeg -format_code Hi50 -f decklink -i 'Intensity Pro' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 10 bit: <pre class=”example-preformatted”>ffmpeg -raw_format yuv422p10 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 with 16 audio channels: <pre class=”example-preformatted”>ffmpeg -channels 16 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

 <h3 class=”section”>26.5 dshow# TOC</h3>

Windows DirectShow input device. DirectShow support is enabled when FFmpeg is built with the mingw-w64 project. Currently only audio and video devices are supported. Multiple devices may be opened as separate inputs, but they may also be opened on the same input, which should improve synchronism between them. The input name should be in the format: <pre class=”example-preformatted”><var class=”var”>TYPE</var>=<var class=”var”>NAME</var>[:<var class=”var”>TYPE</var>=<var class=”var”>NAME</var>] </pre>

where <var class=”var”>TYPE</var> can be either <var class=”var”>audio</var> or <var class=”var”>video</var>, and <var class=”var”>NAME</var> is the device’s name or alternative name.. <h4 class=”subsection”>26.5.1 Options# TOC</h4>

If no options are specified, the device’s defaults are used.

If the device does not support the requested options, it will

fail to open.

<dl class=”table”>

<dt><samp class=”option”>video_size</samp></dt>

<dd> Set the video size in the captured video.

</dd>

<dt><samp class=”option”>framerate</samp></dt>

<dd> Set the frame rate in the captured video.

</dd>

<dt><samp class=”option”>sample_rate</samp></dt>

<dd> Set the sample rate (in Hz) of the captured audio.

</dd>

<dt><samp class=”option”>sample_size</samp></dt>

<dd> Set the sample size (in bits) of the captured audio.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set the number of channels in the captured audio.

</dd>

<dt><samp class=”option”>list_devices</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of devices and exit.

</dd>

<dt><samp class=”option”>list_options</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of selected device’s options

and exit.

</dd>

<dt><samp class=”option”>video_device_number</samp></dt>

<dd> Set video device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class=”option”>audio_device_number</samp></dt>

<dd> Set audio device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class="option">pixel_format</samp></dt>

<dd> Select pixel format to be used by DirectShow. This may only be set when

the video codec is not set or set to rawvideo.

</dd>

<dt><samp class="option">audio_buffer_size</samp></dt>

<dd> Set audio device buffer size in milliseconds (which can directly

impact latency, depending on the device).

Defaults to using the audio device’s

default buffer size (typically some multiple of 500ms).

Setting this value too low can degrade performance.

See also

us/library/windows/desktop/dd377582(v=vs.85).aspx

</dd>

<dt><samp class="option">video_pin_name</samp></dt>

<dd> Select video capture pin to use by name or alternative name.

</dd>

<dt><samp class="option">audio_pin_name</samp></dt>

<dd> Select audio capture pin to use by name or alternative name.

</dd>

<dt><samp class="option">crossbar_video_input_pin_number</samp></dt>

<dd> Select video input pin number for crossbar device. This will be

routed to the crossbar device’s Video Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>crossbar_audio_input_pin_number</samp></dt>

<dd> Select audio input pin number for crossbar device. This will be

routed to the crossbar device’s Audio Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_video_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display dialog

to the end user, allowing them to change video filter properties

and configurations manually.

Note that for crossbar devices, adjusting values in this dialog

may be needed at times to toggle between PAL (25 fps) and NTSC (29.97)

input frame rates, sizes, interlacing, etc. Changing these values can

enable different scan rates/frame rates and avoiding green bars at

the bottom, flickering scan lines, etc.

Note that with some devices, changing these properties can also affect future

invocations (sets new defaults) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_audio_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display dialog

to the end user, allowing them to change audio filter properties

and configurations manually.

</dd>

<dt><samp class=”option”>show_video_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens a video device.

</dd>

<dt><samp class=”option”>show_audio_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens an audio device.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV channels and frequencies.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_audio_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV audio (like mono vs. stereo, Language A,B or C).

</dd>

<dt><samp class=”option”>audio_device_load</samp></dt>

<dd> Load an audio capture filter device from file instead of searching

it by name. It may load additional parameters too, if the filter
supports the serialization of its properties to.
To use this an audio capture source has to be specified, but it can
be anything even fake one.

</dd>
<dt><samp class="option">audio_device_save</samp></dt>
<dd> Save the currently used audio capture filter device and its
parameters (if the filter supports it) to a file.
If a file with the same name exists it will be overwritten.

</dd>
<dt><samp class="option">video_device_load</samp></dt>
<dd> Load a video capture filter device from file instead of searching
it by name. It may load additional parameters too, if the filter
supports the serialization of its properties to.
To use this a video capture source has to be specified, but it can
be anything even fake one.

</dd>
<dt><samp class="option">video_device_save</samp></dt>
<dd> Save the currently used video capture filter device and its
parameters (if the filter supports it) to a file.
If a file with the same name exists it will be overwritten.

</dd>
<dt><samp class="option">use_video_device_timestamps</samp></dt>
<dd> If set to <samp class="option">false</samp>, the timestamp for video frames will be
derived from the wallclock instead of the timestamp provided by
the capture device. This allows working around devices that

provide unreliable timestamps.

</p>

</dd>

</dl>

 <h4 class="subsection">26.5.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of DirectShow supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f dshow -i dummy </pre>

Open video device <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -i video="Camera" </pre>

Open second video device with name <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -video_device_number 1 -i video="Camera" </pre>

Open video device <var class="var">Camera</var> and audio device <var class="var">Microphone</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -i video="Camera":audio="Microphone&q
</pre>

Print the list of supported options in selected device and exit: <pre class="example-preformatted">$ ffmpeg -list_options true -f dshow -i video="Camera" </pre>

Specify pin names to capture by name or alternative name, specify alternative device name: <pre class="example-preformatted">$ ffmpeg -f dshow -audio_pin_name "Audio Out" -video_pin_name 2 -i video=video="@device_pnp_\\?\pci#ven_1a0a&dev_6200&subsys_62
8f56-11d0-a3b9-00a0c9223196}\{ca465100-deb0-4d59-818f-8c477184adf6}":audio="Microphone&q
</pre>

Configure a crossbar device, specifying crossbar pins, allow user to adjust video capture properties at startup: <pre class="example-preformatted">$ ffmpeg -f dshow -show_video_device_dialog true -crossbar_video_input_pin_number 0 -crossbar_audio_input_pin_number 3 -i video="AVerMedia BDA Analog Capture":audio="AVerMedia BDA Analog Capture" </pre>

 <h3 class="section">26.6 fbdev#

TOC</h3>

Linux framebuffer input device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class=”file”>/dev/fb0</samp>. For more detailed information read the file Documentation/fb/framebuffer.txt included in the Linux source tree. See also http://linux-fbdev.sourceforge.net/, and fbset(1). To record from the framebuffer device <samp class=”file”>/dev/fb0</samp> with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 10 -i /dev/fb0 out.avi </pre>

You can take a single screenshot image with the command: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 1 -i /dev/fb0 -frames:v 1 screenshot.jpeg </pre>

 <h4 class=”subsection”>26.6.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate. Default is 25. </dd> </dl>

 <h3 class=”section”>26.7 gdigrab# TOC</h3>

Win32 GDI-based screen capture device. This device allows you to capture a region of the display on Windows. Amongst options for the input filenames are such elements as: <pre class=”example-preformatted”>desktop </pre> or <pre class=”example-preformatted”>title=<var class=”var”>window_title</var> </pre> or <pre class=”example-preformatted”>hwnd=<var class=”var”>window_hwnd</var> </pre>

The first option will capture the entire desktop, or a fixed region of the desktop. The second and third options will instead capture the contents of a single window, regardless of its position on the screen. For example, to grab the entire desktop using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f gdigrab -framerate 6 -i desktop out.mpg </pre>

Grab a 640x480 region at position <code class=”code”>10,20</code>: <pre class=”example-preformatted”>ffmpeg

-f gdigrab -framerate 6 -offset_x 10 -offset_y 20 -video_size vga -i desktop out.mpg </pre>

Grab the contents of the window named "Calculator" <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i title=Calculator out.mpg </pre>

 <h4 class="subsection">26.7.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. Use the value <code class="code">0</code> to not draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. Note that <var class="var">show_region</var> is incompatible with grabbing the contents of a single window. For example: <pre class="example-preformatted">ffmpeg -f gdigrab -show_region 1 -framerate 6 -video_size cif -offset_x 10 -offset_y 20 -i desktop out.mpg </pre>

</dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. The default is to capture the full screen if <samp class="file">desktop</samp> is selected, or the full window size if <samp class="file">title=<var class="var">window_title</var></samp> is selected. </dd> <dt><samp class="option">offset_x</samp></dt> <dd> When capturing a region with <var class="var">video_size</var>, set the distance from the left edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned to the left of your primary monitor, you will need to use a negative <var class="var">offset_x</var> value to move the region to that monitor. </dd> <dt><samp class="option">offset_y</samp></dt> <dd> When capturing a region with <var class="var">video_size</var>, set the distance from the top edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned above your primary monitor, you will need to use a negative <var class="var">offset_y</var> value to move the region to that monitor. </dd> </dl>

 <h3 class="section">26.8 iec61883<span

class=”pull-right”># TOC</h3>

FireWire DV/HDV input device using libiec61883. To enable this input device, you need libiec61883, libraw1394 and libavc1394 installed on your system. Use the configure option <code class=”code”>--enable-libiec61883</code> to compile with the device enabled. The iec61883 capture device supports capturing from a video device connected via IEEE1394 (FireWire), using libiec61883 and the new Linux FireWire stack (juju). This is the default DV/HDV input method in Linux Kernel 2.6.37 and later, since the old FireWire stack was removed. Specify the FireWire port to be used as input file, or "auto" to choose the first port connected. <h4 class=”subsection”>26.8.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>dvtype</samp></dt> <dd> Override autodetection of DV/HDV. This should only be used if auto detection does not work, or if usage of a different device type should be prohibited. Treating a DV device as HDV (or vice versa) will not work and result in undefined behavior. The values <samp class=”option”>auto</samp>, <samp class=”option”>dv</samp> and <samp class=”option”>hdv</samp> are supported. </dd> <dt><samp class=”option”>dvbuffer</samp></dt> <dd> Set maximum size of buffer for incoming data, in frames. For DV, this is an exact value. For HDV, it is not frame exact, since HDV does not have a fixed frame size. </dd> <dt><samp class=”option”>dvguid</samp></dt> <dd> Select the capture device by specifying its GUID. Capturing will only be performed from the specified device and fails if no device with the given GUID is found. This is useful to select the input if multiple devices are connected at the same time. Look at /sys/bus/firewire/devices to find out the GUIDs. </dd> </dl>

 <h4 class=”subsection”>26.8.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Grab and show the input of a FireWire DV/HDV device. <pre class=”example-preformatted”>ffplay -f iec61883 -i auto </pre>

Grab and record the input of a FireWire DV/HDV device, using a packet buffer of 100000 packets if the source is HDV. <pre class=”example-preformatted”>ffmpeg -f iec61883 -i auto -dvbuffer 100000

out.mpg </pre>

 <h3 class="section">26.9 jack# TOC</h3>

JACK input device. To enable this input device during configuration you need libjack installed on your system. A JACK input device creates one or more JACK writable clients, one for each audio channel, with name <var class="var">client_name</var>:input_<var class="var">N</var>, where <var class="var">client_name</var> is the name provided by the application, and <var class="var">N</var> is a number which identifies the channel. Each writable client will send the acquired data to the FFmpeg input device. Once you have created one or more JACK readable clients, you need to connect them to one or more JACK writable clients. To connect or disconnect JACK clients you can use the <code class="command">jack_connect</code> and <code class="command">jack_disconnect</code> programs, or do it through a graphical interface, for example with <code class="command">qjackctl</code>. To list the JACK clients and their properties you can invoke the command <code class="command">jack_lsp</code>. Follows an example which shows how to capture a JACK readable client with <code class="command">ffmpeg</code>. <pre class="example-preformatted"># Create a JACK writable client with name "ffmpeg". $ ffmpeg -f jack -i ffmpeg -y out.wav

Start the sample jack_metro readable client. $ jack_metro -b 120 -d 0.2 -f 4000

List the current JACK clients. $ jack_lsp -c system:capture_1 system:capture_2 system:playback_1 system:playback_2 ffmpeg:input_1 metro:120_bpm

Connect metro to the ffmpeg writable client. $ jack_connect metro:120_bpm ffmpeg:input_1 </pre>

For more information read: http://jackaudio.org/ <h4 class="subsection">26.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">26.10 kmsgrab# <a class="anchor hidden-xs"href="#toc-kmsgrab"

aria-hidden=”true”>TOC</h3>

KMS video input device. Captures the KMS scanout framebuffer associated with a specified CRTC or plane as a DRM object that can be passed to other hardware functions. Requires either DRM master or CAP_SYS_ADMIN to run. If you don’t understand what all of that means, you probably don’t want this. Look at <samp class=”option”>x11grab</samp> instead. <h4 class=”subsection”>26.10.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>device</samp></dt> <dd> DRM device to capture on. Defaults to <samp class=”option”>/dev/dri/card0</samp>. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Pixel format of the framebuffer. This can be autodetected if you are running Linux 5.7 or later, but needs to be provided for earlier versions. Defaults to <samp class=”option”>bgr0</samp>, which is the most common format used by the Linux console and Xorg X server. </dd> <dt><samp class=”option”>format_modifier</samp></dt> <dd> Format modifier to signal on output frames. This is necessary to import correctly into some APIs. It can be autodetected if you are running Linux 5.7 or later, but will need to be provided explicitly when needed in earlier versions. See the libdrm documentation for possible values. </dd> <dt><samp class=”option”>crtc_id</samp></dt> <dd> KMS CRTC ID to define the capture source. The first active plane on the given CRTC will be used. </dd> <dt><samp class=”option”>plane_id</samp></dt> <dd> KMS plane ID to define the capture source. Defaults to the first active plane found if neither <samp class=”option”>crtc_id</samp> nor <samp class=”option”>plane_id</samp> are specified. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Framerate to capture at. This is not synchronised to any page flipping or framebuffer changes - it just defines the interval at which the framebuffer is sampled. Sampling faster than the framebuffer update rate will generate independent frames with the same content. Defaults to <code class=”code”>30</code>. </dd> </dl>

 <h4 class=”subsection”>26.10.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Capture from the first active plane, download the result to normal frames and encode. This will only work if the framebuffer is both linear and mappable - if not, the result may be scrambled or fail to download. <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwdownload,format=bgr0' out-

put.mp4 </pre>

Capture from CRTC ID 42 at 60fps, map the result to VAAPI, convert to NV12 and encode as H.264. <pre class="example-preformatted">ffmpeg -crtc_id 42 -framerate 60 -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,scale_vaapi=w=1920:h=1080:format=nv12' -c:v h264_vaapi output.mp4 </pre>

To capture only part of a plane the output can be cropped - this can be used to capture a single window, as long as it has a known absolute position and size. For example, to capture and encode the middle quarter of a 1920x1080 plane: <pre class="example-preformatted">ffmpeg -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,crop=960:540:480:270,scale_vaapi=960:540:nv12' -c:v h264_vaapi output.mp4 </pre>

 <h3 class="section">26.11 lavfi# TOC</h3>

Libavfilter input virtual device. This input device reads data from the open output pads of a libavfilter filtergraph. For each filtergraph open output, the input device will create a corresponding stream which is mapped to the generated output. The filtergraph is specified through the option <samp class="option">graph</samp>. <h4 class="subsection">26.11.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">graph</samp></dt> <dd> Specify the filtergraph to use as input. Each video open output must be labelled by a unique string of the form "out<var class="var">N</var>", where <var class="var">N</var> is a number starting from 0 corresponding to the mapped input stream generated by the device. The first unlabelled output is automatically assigned to the "out0" label, but all the others need to be specified explicitly. The suffix "+subcc" can be appended to the output label to create an extra stream with the closed captions packets attached to that output (experimental; only for EIA-608 / CEA-708 for now). The subcc streams are created after all the normal streams, in the order of the corresponding stream. For example, if there is "out19+subcc", "out7+subcc" and up to "out42", the stream #43 is subcc for stream #7 and stream #44 is subcc for stream #19. If not specified defaults to the filename specified for the input device. </dd> <dt><samp class="option">graph_file</samp></dt> <dd> Set the filename of the filtergraph to be read and sent to the other filters. Syntax of the filtergraph is the same as the one specified by the option <var class="var">graph</var>.

 </dd> <dt><samp class="option">dumpgraph</samp></dt> <dd> Dump graph to stderr. </dd> </dl>

 <h4 class="subsection">26.11.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Create a color video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi -graph "color=c=pink [out0]" dummy </pre>

As the previous example, but use filename for specifying the graph description, and omit the "out0" label: <pre class="example-preformatted">ffplay -f lavfi color=c=pink </pre>

Create three different video test filtered sources and play them: <pre class="example-preformatted">ffplay -f lavfi -graph "testsrc [out0]; testsrc,hflip [out1]; testsrc,negate [out2]" test3 </pre>

Read an audio stream from a file using the amovie source and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "amovie=test.wav" </pre>

Read an audio stream and a video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "movie=test.avi[out0];amovie=test.wav[out1]" </pre>

Dump decoded frames to images and Closed Captions to an RCWT backup: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=test.ts[out0+subcc]" -map v frame%08d.png -map s -c copy -f rcwt subcc.bin </pre>

 <h3 class="section">26.12 libcdio# TOC</h3>

Audio-CD input device based on libcdio. To enable this input device during configuration you need libcdio installed on your system. It requires the configure option <code class="code">--enable-libcdio</code>. This device allows playing and grabbing from an Audio-CD. For example to copy with <code class="command">ffmpeg</code> the entire Audio-CD in <samp class="file">/dev/sr0</samp>, you may

run the command: <pre class="example-preformatted">ffmpeg -f libcdio -i /dev/sr0 cd.wav </pre>

 <h4 class="subsection">26.12.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">speed</samp></dt> <dd> Set drive reading speed. Default value is 0. The speed is specified CD-ROM speed units. The speed is set through the libcdio <code class="code">cdio_cddap_speed_set</code> function. On many CD-ROM drives, specifying a value too large will result in using the fastest speed. </dd> <dt><samp class="option">paranoia_mode</samp></dt> <dd> Set paranoia recovery mode flags. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dt>‘<samp class="samp">verify</samp>’</dt> <dt>‘<samp class="samp">overlap</samp>’</dt> <dt>‘<samp class="samp">neverskip</samp>&rsquo <dt>‘<samp class="samp">full</samp>’</dt> </dl>

Default value is ‘<samp class="samp">disable</samp>’. For more information about the available recovery modes, consult the paranoia project documentation. </dd> </dl>

 <h3 class="section">26.13 libdc1394# TOC</h3>

IIDC1394 input device, based on libdc1394 and libraw1394. Requires the configure option <code class="code">--enable-libdc1394</code>. <h4 class="subsection">26.13.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set the frame rate. Default is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Select the pixel format. Default is <code class="code">uyvy422</code>. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video size given as a string such as <code class="code">640x480</code> or <code class="code">hd720</code>. Default is <code class="code">qvga</code>. </dd> </dl>

 <h3 class="section">26.14 openal# TOC</h3>

The OpenAL input device provides audio capture on all systems with a working OpenAL 1.1 implementation. To enable this input device during configuration, you need OpenAL headers and libraries installed on your system, and need to configure FFmpeg with <code class="code">--enable-openal</code>. OpenAL headers and libraries should be provided as part of your OpenAL implementation, or as an additional download (an SDK). Depending on your installation you may need to specify additional flags via the <code class="code">--extra-cflags</code> and <code class="code">--extra-ldflags</code> for allowing the build system to locate the OpenAL headers and libraries. An incomplete list of OpenAL implementations follows: <dl class="table"> <dt>Creative</dt> <dd> The official Windows implementation, providing hardware acceleration with supported devices and software fallback. See http://openal.org/. </dd> <dt>OpenAL Soft</dt> <dd> Portable, open source (LGPL) software implementation. Includes backends for the most common sound APIs on the Windows, Linux, Solaris, and BSD operating systems. See <a class="url" href="http://kcat.strangesoft.net/openal.html">http://kcat.strangesoft.net/openal.html </dd> <dt>Apple</dt> <dd> OpenAL is part of Core Audio, the official Mac OS X Audio interface. See http://developer.apple.com/technologies/mac/audio-and-video.html </dd> </dl>

This device allows one to capture from an audio input device handled through OpenAL. You need to specify the name of the device to capture in the provided filename. If the empty string is provided, the device will automatically select the default device. You can get the list of the supported devices by using the option <var class="var">list_devices</var>. <h4 class="subsection">26.14.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels in the captured audio. Only the values <samp class="option">1</samp> (monaural) and <samp class="option">2</samp> (stereo) are currently supported. Defaults to <samp class="option">2</samp>. </dd> <dt><samp class="option">sample_size</samp></dt> <dd> Set the sample size (in bits) of the captured audio. Only the values <samp class="option">8</samp> and <samp class="option">16</samp> are currently supported. Defaults to <samp class="option">16</samp>. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate (in Hz) of the captured audio. Defaults to <samp class="option">44.1k</samp>. </dd> <dt><samp class="option">list_devices</samp></dt>

<dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">26.14.2 Examples# TOC</h4>

Print the list of OpenAL supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f openal -i dummy out.ogg </pre>

Capture from the OpenAL device <samp class="file">DR-BT101 via PulseAudio</samp>: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out.ogg </pre>

Capture from the default device (note the empty string ” as filename): <pre class="example-preformatted">$ ffmpeg -f openal -i '' out.ogg </pre>

Capture from two devices simultaneously, writing to two different files, within the same <code class="command">ffmpeg</code> command: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out1.ogg -f openal -i 'ALSA Default' out2.ogg </pre> Note: not all OpenAL implementations support multiple simultaneous capture - try the latest OpenAL Soft if the above does not work. <h3 class="section">26.15 oss# TOC</h3>

Open Sound System input device. The filename to provide to the input device is the device node representing the OSS input device, and is usually set to <samp class="file">/dev/dsp</samp>. For example to grab from <samp class="file">/dev/dsp</samp> using <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -f oss -i /dev/dsp /tmp/oss.wav </pre>

For more information about OSS see: <a class="url" href="http://manuals.opensound.com/usersguide/ds
  <h4 class="subsection">26.15.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd>

<dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">26.16 pulse# TOC</h3>

PulseAudio input device. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libpulse</code>. The filename to provide to the input device is a source device or the string "default" To list the PulseAudio source devices and their properties you can invoke the command <code class="command">pactl list sources</code>. More information about PulseAudio can be found on http://www.pulseaudio.org. <h4 class="subsection">26.16.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class="option">name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class="code">LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class="option">stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active streams, by default it is "record". </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Specify the samplerate in Hz, by default 48kHz is used. </dd> <dt><samp class="option">channels</samp></dt> <dd> Specify the channels in use, by default 2 (stereo) is set. </dd> <dt><samp class="option">frame_size</samp></dt> <dd> This option does nothing and is deprecated. </dd> <dt><samp class="option">fragment_size</samp></dt> <dd> Specify the size in bytes of the minimal buffering fragment in PulseAudio, it will affect the audio latency. By default it is set to 50 ms amount of data. </dd> <dt><samp class="option">wallclock</samp></dt> <dd> Set the initial PTS using the current time. Default is 1. </dd> </dl>

 <h4 class="subsection">26.16.2 Examples# TOC</h4> Record a stream from default device: <pre class="example-preformatted">ffmpeg -f pulse -i default /tmp/pulse.wav </pre>

 <h3 class="section">26.17 sndio#

TOC</h3>

sndio input device. To enable this input device during configuration you need libsndio installed on your system. The filename to provide to the input device is the device node representing the sndio input device, and is usually set to <samp class=”file”>/dev/audio0</samp>. For example to grab from <samp class=”file”>/dev/audio0</samp> using <code class=”command”>ffmpeg</code> use the command: <pre class=”example-preformatted”>ffmpeg -f sndio -i /dev/audio0 /tmp/oss.wav </pre>

 <h4 class=”subsection”>26.17.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class=”section”>26.18 video4linux2, v4l2# TOC</h3

Video4Linux2 input video device. "v4l2" can be used as alias for "video4linux2". If FFmpeg is built with v4l-utils support (by using the <code class=”code”>--enable-libv4l2</code> configure option), it is possible to use it with the <code class=”code”>-use_libv4l2</code> input device option. The name of the device to grab is a file device node, usually Linux systems tend to automatically create such nodes when the device (e.g. an USB webcam) is plugged into the system, and has a name of the kind <samp class=”file”>/dev/video<var class=”var”>N</var></samp>, where <var class=”var”>N</var> is a number associated to the device. Video4Linux2 devices usually support a limited set of <var class=”var”>width</var>x<var class=”var”>height</var> sizes and frame rates. You can check which are supported using <code class=”command”>-list_formats all</code> for Video4Linux2 devices. Some devices, like TV cards, support one or more standards. It is possible to list all the supported standards using <code class=”command”>-list_standards all</code>. The time base for the timestamps is 1 microsecond. Depending on the kernel version and configuration, the timestamps may be derived from the real time clock (origin at the Unix Epoch) or the monotonic clock (origin usually at boot time, unaffected by NTP or manual changes to the clock). The <samp class=”option”>-timestamps abs</samp> or <samp class=”option”>-ts abs</samp> option can be used to force conversion into the real time clock. Some usage examples

of the video4linux2 device with <code class="command">ffmpeg</code> and <code class="command">ffplay</code>: <ul class="itemize mark-bullet"> List supported formats for a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -list_formats all /dev/video0 </pre>

Grab and show the input of a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -framerate 30 -video_size hd720 /dev/video0 </pre>

Grab and record the input of a video4linux2 device, leave the frame rate and size as previously set: <pre class="example-preformatted">ffmpeg -f video4linux2 -input_format mjpeg -i /dev/video0 out.mpeg </pre>

For more information about Video4Linux, check http://linuxtv.org/. <h4 class="subsection">26.18.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">standard</samp></dt> <dd> Set the standard. Must be the name of a supported standard. To get a list of the supported standards, use the <samp class="option">list_standards</samp> option. </dd> <dt><samp class="option">channel</samp></dt> <dd> Set the input channel number. Default to -1, which means using the previously selected channel. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. The argument must be a string in the form <var class="var">WIDTH</var>x<var class="var">HEIGHT</var> or a valid size abbreviation. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Select the pixel format (only valid for raw video input). </dd> <dt><samp class="option">input_format</samp></dt> <dd> Set the preferred pixel format (for raw video) or a codec name. This option allows one to select the input format, when several are available. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the preferred video frame rate. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> List available formats (supported pixel formats, codecs, and frame sizes) and exit. Available values are: <dl class="table"> <dt>‘<samp class="samp">all</samp>’</dt> <dd> Show all available (compressed and non-compressed) formats. </dd> <dt>‘<samp class="samp">raw</samp>’</dt> <dd> Show only raw video (non-compressed) formats. </dd> <dt>‘<samp class="samp">compressed</samp>’</dt> <dd> Show only compressed formats. </dd> </dl>

</dd> <dt><samp class="option">list_standards</samp></dt> <dd> List supported standards and exit. Available values are: <dl

class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt>
<dd> Show all supported standards. </dd> </dl>

</dd> <dt><samp class=”option”>timestamps, ts</samp></dt>
<dd> Set type of timestamps for grabbed frames. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt>
<dd> Use timestamps from the kernel. </dd> <dt>‘<samp class=”samp”>abs</samp>’</dt> <dd> Use absolute timestamps (wall clock). </dd> <dt>‘<samp class=”samp”>mono2abs</samp>’</dt>
<dd> Force conversion from monotonic to absolute timestamps. </dd> </dl>

Default value is <code class=”code”>default</code>. </dd>
<dt><samp class=”option”>use_libv4l2</samp></dt> <dd> Use libv4l2 (v4l-utils) conversion functions. Default is 0. </dd> </dl>

 <h3 class=”section”>26.19 vfwcap# TOC</h3>

VfW (Video for Windows) capture input device. The filename passed as input is the capture driver number, ranging from 0 to 9. You may use "list" as filename to print a list of drivers. Any other filename will be interpreted as device number 0. <h4 class=”subsection”>26.19.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>video_size</samp></dt>
<dd> Set the video frame size. </dd> <dt><samp class=”option”>framerate</samp></dt>
<dd> Set the grabbing frame rate. Default value is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> </dl>

 <h3 class=”section”>26.20 x11grab# TOC</h3>

X11 video input device. To enable this input device during configuration you need libxcb installed on your system. It will be automatically detected during configuration. This device allows one to capture a region of an X11 display. The filename passed as input has the syntax: <pre class=”example-preformatted”>[<var class=”var”>hostname</var>]:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var>[+<var class=”var”>x_offset</var>,<var class=”var”>y_offset</var>] </pre>

<var class=”var”>hostname</var>:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var> specifies the X11 display name of the screen to grab from. <var class=”var”>hostname</var> can be omitted, and defaults to "localhost". The environment variable <code class=”env”>DISPLAY</code> contains the default display name. <var class=”var”>x_offset</var> and <var class=”var”>y_offset</var> specify the offsets of the grabbed area with respect to the top-left border of the X11 screen. They default to 0. Check the X11 documentation (e.g. <code class=”command”>man X</code>) for more detailed information. Use the <code class=”command”>xdpyinfo</code> program for getting basic information about the properties of your X11 display (e.g. grep for "name" or "dimensions"). For example to grab from <samp class=”file”>:0.0</samp> using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

Grab at position <code class=”code”>10,20</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

 <h4 class=”subsection”>26.20.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>select_region</samp></dt> <dd> Specify whether to select the grabbing area graphically using the pointer. A value of <code class=”code”>1</code> prompts the user to select the grabbing area graphically by clicking and dragging. A single click with no dragging will select the whole screen. A region with zero width or height will also select the whole screen. This option overwrites the <var class=”var”>video_size</var>, <var class=”var”>grab_x</var>, and <var class=”var”>grab_y</var> options. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. A value of <code class=”code”>0</code> specifies not to draw the pointer. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>follow_mouse</samp></dt> <dd> Make the grabbed area follow the mouse. The argument can be <code class=”code”>centered</code> or a number of pixels <var class=”var”>PIXELS</var>. When it is specified with "centered", the grabbing region follows the mouse pointer and keeps the pointer at the center of region; otherwise, the region follows only when the mouse pointer reaches within <var class=”var”>PIXELS</var> (greater than zero) to the edge of region. For example: <pre class=”example-preformatted”>ffmpeg -f x11grab

-follow_mouse centered -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

To follow only when the mouse pointer reaches within 100 pixels to edge: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse 100 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. </dd> <dt><samp class="option">region_border</samp></dt> <dd> Set the region border thickness if <samp class="option">-show_region 1</samp> is used. Range is 1 to 128 and default is 3 (XCB-based x11grab only). For example: <pre class="example-preformatted">ffmpeg -f x11grab -show_region 1 -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

With <var class="var">follow_mouse</var>: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse centered -show_region 1 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">window_id</samp></dt> <dd> Grab this window, instead of the whole screen. Default value is 0, which maps to the whole screen (root window). The id of a window can be found using the <code class="command">xwininfo</code> program, possibly with options -tree and -root. If the window is later enlarged, the new area is not recorded. Video ends when the window is closed, unmapped (i.e., iconified) or shrunk beyond the video size (which defaults to the initial window size). This option disables options <samp class="option">follow_mouse</samp> and <samp class="option">select_region</samp>. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. Default is the full desktop or window. </dd> <dt><samp class="option">grab_x</samp></dt> <dt><samp class="option">grab_y</samp></dt> <dd> Set the grabbing region coordinates. They are expressed as offset from the top left corner of the X11 window and correspond to the <var class="var">x_offset</var> and <var class="var">y_offset</var> parameters in the device name. The default value for both options is 0. </dd> </dl>

 <h2 class="chapter">27 Output Devices<a class="anchor hidden-xs" href="#Output-

Devices” aria-hidden=”true”># TOC</h2>

Output devices are configured elements in FFmpeg that can write multimedia data to an output device attached to your system. When you configure your FFmpeg build, all the supported output devices are enabled by default. You can list all available ones using the configure option "–list-outdevs". You can disable all the output devices using the configure option "–disable-outdevs", and selectively enable an output device using the option "–enable-outdev=<var class=”var”>OUTDEV</var>", or you can disable a particular input device using the option "–disable-outdev=<var class=”var”>OUTDEV</var>". The option "-devices" of the ff* tools will display the list of enabled output devices. A description of the currently available output devices follows. <h3 class=”section”>27.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) output device. <h4 class=”subsection”>27.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Play a file on default ALSA device: <pre class=”example-preformatted”>ffmpeg -i INPUT -f alsa default </pre>

Play a file on soundcard 1, audio device 7: <pre class=”example-preformatted”>ffmpeg -i INPUT -f alsa hw:1,7 </pre>

 <h3 class=”section”>27.2 AudioToolbox# TOC</h3>

AudioToolbox output device. Allows native output to CoreAudio devices on OSX. The output filename can be empty (or <code class=”code”>-</code>) to refer to the default system output device or a number that refers to the device index as shown using: <code class=”code”>-list_devices true</code>. Alternatively, the audio input device can be chosen by index using the <samp class=”option”>-audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class=”option”>-list_devices true</samp>, listing all device names, UIDs and corresponding indices.

 <h4 class="subsection">27.2.1 Options# TOC</h4>

AudioToolbox supports the following options: <dl class="table"> <dt><samp class="option">-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the output filename. </dd> </dl>

 <h4 class="subsection">27.2.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of supported devices and output a sine wave to the default device: <pre class="example-preformatted">$ ffmpeg -f lavfi -i sine=r=44100 -f audiotoolbox -list_devices true - </pre>

Output a sine wave to the device with the index 2, overriding any output filename: <pre class="example-preformatted">$ ffmpeg -f lavfi -i sine=r=44100 -f audiotoolbox -audio_device_index 2 - </pre>

 <h3 class="section">27.3 caca# TOC</h3>

CACA output device. This output device allows one to show a video stream in CACA window. Only one CACA window is allowed per application, so you can have only one instance of this output device in an application. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libcaca</code>. libcaca is a graphics library that outputs text instead of pixels. For more information about libcaca, check: http://caca.zoy.org/wiki/libcaca <h4 class="subsection">27.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">window_title</samp></dt> <dd> Set the CACA window title, if not specified default to the filename specified for the output device. </dd> <dt><samp class="option">window_size</samp></dt> <dd> Set the CACA window size, can be a string of the form <var class="var">width</var>x<var class="var">height</var> or a video size abbreviation. If not specified

it defaults to the size of the input video. </dd> <dt><samp class="option">driver</samp></dt> <dd> Set display driver. </dd> <dt><samp class="option">algorithm</samp></dt> <dd> Set dithering algorithm. Dithering is necessary because the picture being rendered has usually far more colours than the available palette. The accepted values are listed with <code class="code">-list_dither algorithms</code>. </dd> <dt><samp class="option">antialias</samp></dt> <dd> Set antialias method. Antialiasing smoothens the rendered image and avoids the commonly seen staircase effect. The accepted values are listed with <code class="code">-list_dither antialiases</code>. </dd> <dt><samp class="option">charset</samp></dt> <dd> Set which characters are going to be used when rendering text. The accepted values are listed with <code class="code">-list_dither charsets</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Set color to be used when rendering text. The accepted values are listed with <code class="code">-list_dither colors</code>. </dd> <dt><samp class="option">list_drivers</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of available drivers and exit. </dd> <dt><samp class="option">list_dither</samp></dt> <dd> List available dither options related to the argument. The argument must be one of <code class="code">algorithms</code>, <code class="code">antialiases</code>, <code class="code">charsets</code>, <code class="code">colors</code>. </dd> </dl>

 <h4 class="subsection">27.3.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> The following command shows the <code class="command">ffmpeg</code> output is an CACA window, forcing its size to 80x25: <pre class="example-preformatted">ffmpeg -i INPUT -c:v rawvideo -pix_fmt rgb24 -window_size 80x25 -f caca - </pre>

Show the list of available drivers and exit: <pre class="example-preformatted">ffmpeg -i INPUT -pix_fmt rgb24 -f caca -list_drivers true - </pre>

Show the list of available dither colors and exit: <pre class="example-preformatted">ffmpeg -i INPUT -pix_fmt rgb24 -f caca -list_dither colors - </pre>

 <h3 class="section">27.4 decklink# TOC</h3>

The decklink output device provides playback capabilities for Blackmagic

DeckLink devices. To enable this output device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class="code">--extra-cflags</code>, <code class="code">--extra-cxxflags</code> and <code class="code">--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class="command">widl</code>. DeckLink is very picky about the formats it supports. Pixel format is always uyvy422, framerate, field order and video size must be determined for your device with <code class="command">-list_formats 1</code>. Audio sample rate is always 48 kHz. <h4 class="subsection">27.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. This option is deprecated, please use the <code class="code">-sinks</code> option of ffmpeg to list the available output devices. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of supported formats and exit. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">preroll</samp></dt> <dd> Amount of time to preroll video in seconds. Defaults to <samp class="option">0.5</samp>. </dd> <dt><samp class="option">duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’, ‘<samp class="samp">one_sub_device_full</sa ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Defaults to ‘<samp class="samp">unset</samp>’. Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0): ‘<samp class="samp">one_sub_device_full</samp>’, ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’ </dd> <dt><samp class="option">timing_offset</samp></dt

<dd> Sets the genlock timing pixel offset on the used output. Defaults to ‘<samp class="samp">unset</samp>’. </dd> <dt><samp class="option">link</samp></dt> <dd> Sets the SDI video link configuration on the used output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">single</samp>’ link SDI, ‘<samp class="samp">dual</samp>’ link SDI or ‘<samp class="samp">quad</samp>’ link SDI. Defaults to ‘<samp class="samp">unset</samp>’. </dd> <dt><samp class="option">sqd</samp></dt> <dd> Enable Square Division Quad Split mode for Quad-link SDI output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">true</samp>’ or ‘<samp class="samp">false</samp>’. Defaults to <samp class="option">unset</samp>. </dd> <dt><samp class="option">level_a</samp></dt> <dd> Enable SMPTE Level A mode on the used output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">true</samp>’ or ‘<samp class="samp">false</samp>’. Defaults to <samp class="option">unset</samp>. </dd> <dt><samp class="option">vanc_queue_size</samp></dt> <dd> Sets maximum output buffer size in bytes for VANC data. If the buffering reaches this value, outgoing VANC data will be dropped. Defaults to ‘<samp class="samp">1048576</samp>’. </dd> </dl>

 <h4 class="subsection">27.4.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List output devices: <pre class="example-preformatted">ffmpeg -sinks decklink </pre>

List supported formats: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -list_formats 1 'DeckLink Mini Monitor' </pre>

Play video clip: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -pix_fmt uyvy422 'DeckLink Mini Monitor' </pre>

Play video clip with non-standard framerate or video size: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -pix_fmt uyvy422 -s 720x486 -r 24000/1001 'DeckLink Mini Monitor' </pre>

 <h3 class="section">27.5 fbdev# TOC</h3>

Linux framebuffer output device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class="file">/dev/fb0</samp>. For more detailed information read the file <samp class="file">Documentation/fb/framebuffer.txt</samp> included in the Linux source tree. <h4 class="subsection">27.5.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">xoffset</samp></dt> <dt><samp class="option">yoffset</samp></dt> <dd> Set x/y coordinate of top left corner. Default is 0. </dd> </dl>

 <h4 class="subsection">27.5.2 Examples# TOC</h4> Play a file on framebuffer device <samp class="file">/dev/fb0</samp>. Required pixel format depends on current framebuffer settings. <pre class="example-preformatted">ffmpeg -re -i INPUT -c:v rawvideo -pix_fmt bgra -f fbdev /dev/fb0 </pre>

See also http://linux-fbdev.sourceforge.net/, and fbset(1). <h3 class="section">27.6 oss# TOC</h3>

OSS (Open Sound System) output device. <h3 class="section">27.7 pulse# TOC</h3>

PulseAudio output device. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libpulse</code>. More information about PulseAudio can be found on http://www.pulseaudio.org <h4 class="subsection">27.7.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class="option">name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class="code">LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class="option">stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active

streams, by default it is set to the specified output name. </dd> <dt><samp class="option">device</samp></dt> <dd> Specify the device to use. Default device is used when not provided. List of output devices can be obtained with command <code class="command">pactl list sinks</code>. </dd> <dt><samp class="option">buffer_size</samp></dt> <dt><samp class="option">buffer_duration</samp></dt> <dd> Control the size and duration of the PulseAudio buffer. A small buffer gives more control, but requires more frequent updates. <samp class="option">buffer_size</samp> specifies size in bytes while <samp class="option">buffer_duration</samp> specifies duration in milliseconds. When both options are provided then the highest value is used (duration is recalculated to bytes using stream parameters). If they are set to 0 (which is default), the device will use the default PulseAudio duration value. By default PulseAudio set buffer duration to around 2 seconds. </dd> <dt><samp class="option">prebuf</samp></dt> <dd> Specify pre-buffering size in bytes. The server does not start with playback before at least <samp class="option">prebuf</samp> bytes are available in the buffer. By default this option is initialized to the same value as <samp class="option">buffer_size</samp> or <samp class="option">buffer_duration</samp> (whichever is bigger). </dd> <dt><samp class="option">minreq</samp></dt> <dd> Specify minimum request size in bytes. The server does not request less than <samp class="option">minreq</samp> bytes from the client, instead waits until the buffer is free enough to request more bytes at once. It is recommended to not set this option, which will initialize this to a value that is deemed sensible by the server. </dd> </dl>

 <h4 class="subsection">27.7.2 Examples# TOC</h4> Play a file on default device on default server: <pre class="example-preformatted">ffmpeg -i INPUT -f pulse "stream name" </pre>

 <h3 class="section">27.8 sndio#
TOC</h3>

sndio audio output device. <h3 class="section">27.9 v4l2# TOC</h3>

Video4Linux2 output device. <h3 class="section">27.10 xv# TOC</h3>

XV (XVideo) output device. This output device allows one to show a video stream in a X Window System window. <h4 class="subsection">27.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">display_name</samp></dt> <dd> Specify the hardware display name, which determines the display and communications domain to be used. The display name or DISPLAY environment variable can be a string in the format <var class="var">hostname</var>[:<var class="var">number</var>[.<var class="var">screen_number</var>]]. <var class="var">hostname</var> specifies the name of the host machine on which the display is physically attached. <var class="var">number</var> specifies the number of the display server on that host machine. <var class="var">screen_number</var> specifies the screen to be used on that server. If unspecified, it defaults to the value of the DISPLAY environment variable. For example, <code class="code">dual-headed:0.1</code> would specify screen 1 of display 0 on the machine named “dual-headed”. Check the X11 specification for more detailed information about the display name format. </dd> <dt><samp class="option">window_id</samp></dt> <dd> When set to non-zero value then device doesn’t create new window, but uses existing one with provided <var class="var">window_id</var>. By default this options is set to zero and device creates its own window. </dd> <dt><samp class="option">window_size</samp></dt> <dd> Set the created window size, can be a string of the form <var class="var">width</var>x<var class="var">height</var> or a video size abbreviation. If not specified it defaults to the size of the input video. Ignored when <var class="var">window_id</var> is set. </dd> <dt><samp class="option">window_x</samp></dt> <dt><samp class="option">window_y</samp></dt> <dd> Set the X and Y window offsets for the created window. They are both set to 0 by default. The values may be ignored by the window manager. Ignored when <var class="var">window_id</var> is set. </dd> <dt><samp class="option">window_title</samp></dt> <dd> Set the window title, if not specified default to the filename specified for the output device. Ignored when <var class="var">window_id</var> is set. </dd> </dl>

For more information about XVideo see <a class="url" href="http://www.x.org/">http://www.x.org/</a
  <h4 class="subsection">27.10.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Decode, display and encode video

input with <code class="command">ffmpeg</code> at the same time: <pre class="example-preformatted">ffmpeg -i INPUT OUTPUT -f xv display </pre>

Decode and display the input video to multiple X11 windows: <pre class="example-preformatted">ffmpeg -i INPUT -f xv normal -vf negate -f xv negated </pre>

 <h2 class="chapter">28 Resampler Options# TOC</h2>

The audio resampler supports the following named options. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, <var class="var">option</var>=<var class="var">value</var> for the aresample filter, by setting the value explicitly in the <code class="code">SwrContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. <dl class="table"> <dt><samp class="option">uchl, used_chlayout</samp></dt> <dd> Set used input channel layout. Default is unset. This option is only used for special remapping. </dd> <dt><samp class="option">isr, in_sample_rate</samp></dt> <dd> Set the input sample rate. Default value is 0. </dd> <dt><samp class="option">osr, out_sample_rate</samp></dt> <dd> Set the output sample rate. Default value is 0. </dd> <dt><samp class="option">isf, in_sample_fmt</samp></dt> <dd> Specify the input sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">osf, out_sample_fmt</samp></dt> <dd> Specify the output sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">tsf, internal_sample_fmt</samp></dt> <dd> Set the internal sample format. Default value is <code class="code">none</code>. This will automatically be chosen when it is not explicitly set. </dd> <dt><samp class="option">ichl, in_chlayout</samp></dt> <dt><samp class="option">ochl, out_chlayout</samp></dt> <dd> Set the input/output channel layout. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> <dt><samp class="option">clev, center_mix_level</samp></dt> <dd> Set the center mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">slev, surround_mix_level</samp></dt> <dd> Set the surround mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">lfe_mix_level</samp></dt> <dd> Set LFE mix into non LFE level. It is used when there is a LFE input but no LFE output. It is a value expressed in deci-

Bel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">rmvol, rematrix_volume</samp></dt> <dd> Set rematrix volume. Default value is 1.0. </dd> <dt><samp class="option">rematrix_maxval</samp></dt> <dd> Set maximum output value for rematrixing. This can be used to prevent clipping vs. preventing volume reduction. A value of 1.0 prevents clipping. </dd> <dt><samp class="option">flags, swr_flags</samp></dt> <dd> Set flags used by the converter. Default value is 0. It supports the following individual flags: <dl class="table"> <dt><samp class="option">res</samp></dt> <dd> force resampling, this flag forces resampling to be used even when the input and output sample rates match. </dd> </dl>

</dd> <dt><samp class="option">dither_scale</samp></dt> <dd> Set the dither scale. Default value is 1. </dd> <dt><samp class="option">dither_method</samp></dt>
<dd> Set dither method. Default value is 0. Supported values:
<dl class="table"> <dt>‘<samp class="samp">rectangular</samp>’</dt>
<dd> select rectangular dither </dd> <dt>‘<samp class="samp">triangular</samp>’</dt> <dd> select triangular dither </dd> <dt>‘<samp class="samp">triangular_hp</samp>’</dt>
<dd> select triangular dither with high pass </dd> <dt>‘<samp class="samp">lipshitz</samp>’</dt> <dd> select Lipshitz noise shaping dither. </dd> <dt>‘<samp class="samp">shibata</samp>’</dt>
<dd> select Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">low_shibata</samp>’</dt> <dd> select low Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">high_shibata</samp>’</
<dd> select high Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">f_weighted</samp>’</dt> <dd> select f-weighted noise shaping dither </dd> <dt>‘<samp class="samp">modified_e_weighted</samp>
<dd> select modified-e-weighted noise shaping dither </dd>
<dt>‘<samp class="samp">improved_e_weighted</samp>’</dt>
<dd> select improved-e-weighted noise shaping dither </dd> </dl>

</dd> <dt><samp class="option">resampler</samp></dt> <dd> Set resampling engine. Default value is swr. Supported values:
<dl class="table"> <dt>‘<samp class="samp">swr</samp>’</dt>
<dd> select the native SW Resampler; filter options precision and cheby are not applicable in this case. </dd> <dt>‘<samp class="samp">soxr</samp>’</dt> <dd> select the SoX Resampler (where available); compensation, and filter options filter_size, phase_shift, exact_rational, filter_type & kaiser_beta, are not applicable in this case.
</dd> </dl>

</dd>

<dt><samp class="option">filter_size</samp></dt>

<dd> For swr only, set resampling filter size, default value is 32.

</dd>

<dt><samp class="option">phase_shift</samp></dt>

<dd> For swr only, set resampling phase shift, default value is 10, and must be in

the interval [0,30].

</dd>

<dt><samp class="option">linear_interp</samp></dt>

<dd> Use linear interpolation when enabled (the default). Disable it if you want

to preserve speed instead of quality when exact_rational fails.

</dd>

<dt><samp class="option">exact_rational</samp></dt>

<dd> For swr only, when enabled, try to use exact phase_count based on input and

output sample rate. However, if it is larger than <code class="code">1 << phase_shift</code>,

the phase_count will be <code class="code">1 << phase_shift</code> as fallback. Default is enabled.

</dd>

<dt><samp class="option">cutoff</samp></dt>

<dd> Set cutoff frequency (swr: 6dB point; soxr: 0dB point) ratio; must be a float

value between 0 and 1. Default value is 0.97 with swr, and 0.91 with soxr

(which, with a sample-rate of 44100, preserves the entire audio band to 20kHz).

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> For soxr only, the precision in bits to which the resampled signal will be

calculated. The default value of 20 (which, with suitable dithering, is

appropriate for a destination bit-depth of 16) gives SoX’s ’High Quality’; a

value of 28 gives SoX’s ’Very High Quality’.

</dd>

<dt><samp class="option">cheby</samp></dt>

<dd> For soxr only, selects passband rolloff none (Chebyshev) & higher-precision

approximation for ’irrational’ ratios. Default value is 0.

</dd>

<dt><samp class="option">async</samp></dt>

<dd> For swr only, simple 1 parameter audio sync to timestamps using stretching,

squeezing, filling and trimming. Setting this to 1 will enable filling and

trimming, larger values represent the maximum amount in samples that the data

may be stretched or squeezed for each second.

Default value is 0, thus no compensation is applied to make the samples match

the audio timestamps.

</dd>

<dt><samp class="option">first_pts</samp></dt>

<dd> For swr only, assume the first pts should be this value. The time unit is 1 / sample rate.

This allows for padding/trimming at the start of stream. By default, no

assumption is made about the first frame’s expected pts, so no padding or

trimming is done. For example, this could be set to 0 to pad the beginning with

silence if an audio stream starts after the video stream or to trim any samples

with a negative pts due to encoder delay.

</dd>

<dt><samp class="option">min_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger stretching/squeezing/filling or trimming of the

data to make it match the timestamps. The default is that

stretching/squeezing/filling and trimming is disabled

(<samp class="option">min_comp</samp> = <code class="code">FLT_MAX</code>).

</dd>

<dt><samp class="option">min_hard_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger adding/dropping samples to make it match the

timestamps. This option effectively is a threshold to select between

hard (trim/fill) and soft (squeeze/stretch) compensation. Note that

all compensation is by default disabled through <samp class="option">min_comp</samp>.

The default is 0.1.

</dd>

<dt><samp class="option">comp_duration</samp></dt>

<dd> For swr only, set duration (in seconds) over which data is stretched/squeezed

to make it match the timestamps. Must be a non-negative double float value,

default value is 1.0.

</dd>

<dt><samp class="option">max_soft_comp</samp></dt>

<dd> For swr only, set maximum factor by which data is stretched/squeezed to make it

match the timestamps. Must be a non-negative double float value, default value

is 0.

</dd>

<dt><samp class="option">matrix_encoding</samp></dt>

<dd> Select matrixed stereo encoding.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">none</samp>’</dt>

<dd> select none

</dd>

<dt>‘<samp class="samp">dolby</samp>’</dt>

<dd> select Dolby

</dd>

<dt>‘<samp class="samp">dplii</samp>’</dt>

<dd> select Dolby Pro Logic II

</dd>

</dl>

Default value is <code class="code">none</code>. </dd> <dt><samp class="option">filter_type</samp></dt> <dd> For swr only, select resampling filter type. This only affects resampling operations. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cubic</samp>’</dt> <dd> select cubic </dd> <dt>‘<samp class="samp">blackman_nuttall</samp>’</dt> <dd> select Blackman Nuttall windowed sinc </dd> <dt>‘<samp class="samp">kaiser</samp>’</dt> <dd> select Kaiser windowed sinc </dd> </dl>

</dd> <dt><samp class="option">kaiser_beta</samp></dt> <dd> For swr only, set Kaiser window beta value. Must be a double float value in the interval [2,16], default value is 9. </dd> <dt><samp class="option">output_sample_bits</samp></dt> <dd> For swr only, set number of used output sample bits for dithering. Must be an integer in the interval [0,64], default value is 0, which means it’s not used. </dd> </dl>

 <h2 class="chapter">29 Scaler Options<span class="pull-

right”>#
TOC</h2>

The video scaler supports the following named options.
Options may be set by specifying -<var class=”var”>option</var>
<var class=”var”>value</var> in the FFmpeg tools, with a few API-only exceptions noted below. For programmatic use, they can be set explicitly in the <code class=”code”>SwsContext</code> options or through the <samp class=”file”>libavutil/opt.h</samp> API.
<dl class=”table”> <dd> </dd>
<dt><samp class=”option”>scaler, scaler_sub</samp></dt> <dd> Choose the scaling algorithm to use. Default value is ‘<samp class=”samp”>auto</samp>’ for both. It accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic choice. For ‘<samp class=”samp”>scaler_sub</samp>’, this means the same algorithm as ‘<samp class=”samp”>scaler</samp>’. For ‘<samp class=”samp”>scaler</samp>’, this defaults to the scaler flag selected by ‘<samp class=”samp”>sws_flags</samp>’. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt>
<dd> Bilinear filter. (AKA triangle filter) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> 2-tap cubic BC-spline (AKA Mitchell-Netravali spline). The B and C parameters can be configured by setting <code class=”code”>param0</code> and <code class=”code”>param1</code>, defaulting to 0.0 and 0.6 respectively. </dd> <dt>‘<samp class=”samp”>point, neighbor</samp>’</dt> <dd> Point sampling (AKA nearest neighbor). </dd> <dt>‘<samp class=”samp”>area</samp>’</dt>
<dd> Area averaging. Equivalent to ‘<samp class=”samp”>bilinear</samp>’ for upscaling. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt>
<dd> 2-tap Gaussian filter approximation. The sharpness parameter can be configured by setting <code class=”code”>param0</code>, defaulting to 3.0. </dd> <dt>‘<samp class=”samp”>sinc</samp>’</dt>
<dd> Unwindowed sinc filter. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Lanczos resampling (sinc windowed sinc). The number of filter taps can be configured by setting <code class=”code”>param0</code>, defaulting to 3. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt>
<dd> Unwindowed natural bicubic spline. </dd> </dl>

</dd> <dt><samp class=”option”>sws_flags</samp></dt>
<dd> Set the scaler flags. This is also used to set the scaling algorithm, though this usage is deprecated in favor of setting ‘<samp class=”samp”>scaler</samp>’. Only a single algorithm may be selected. Default value is ‘<samp class=”samp”>bicubic</samp>’. It accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>fast_bilinear</samp>’</dt>
<dd> Select fast bilinear scaling algorithm. (Deprecated)

</dd> <dt>‘<samp class="samp">bilinear</samp>’</dt>
<dd> Select bilinear scaling algorithm. (Deprecated) </dd>
<dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> Select
bicubic scaling algorithm. (Deprecated) </dd> <dt>‘<samp
class="samp">experimental</samp>’</dt> <dd> Select experimental scaling algorithm. (Deprecated) </dd> <dt>‘<samp
class="samp">neighbor</samp>’</dt> <dd> Select nearest
neighbor rescaling algorithm. (Deprecated) </dd> <dt>‘<samp
class="samp">area</samp>’</dt> <dd> Select averaging
area rescaling algorithm. (Deprecated) </dd> <dt>‘<samp
class="samp">bicublin</samp>’</dt> <dd> Select bicubic scaling algorithm for the luma component, bilinear for chroma components. (Deprecated) </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dd> Select Gaussian rescaling algorithm. (Deprecated)
</dd> <dt>‘<samp class="samp">sinc</samp>’</dt>
<dd> Select sinc rescaling algorithm. (Deprecated) </dd>
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Select
Lanczos rescaling algorithm. The default width (alpha) is 3 and can be changed
by setting <code class="code">param0</code>. (Deprecated)
</dd> <dt>‘<samp class="samp">spline</samp>’</dt>
<dd> Select natural bicubic spline rescaling algorithm. (Deprecated)
</dd> <dt>‘<samp class="samp">print_info</samp>’</dt>
<dd> Enable printing/debug logging. </dd> <dt>‘<samp
class="samp">accurate_rnd</samp>’</dt> <dd> Enable accurate rounding. </dd> <dt>‘<samp class="samp">full_chroma_int</samp>’</dt>
<dd> Enable full chroma interpolation. </dd> <dt>‘<samp
class="samp">full_chroma_inp</samp>’</dt> <dd> Select full
chroma input. </dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Enable bitexact output. </dd> <dt>‘<samp
class="samp">unstable</samp>’</dt> <dd> Allow the use of
experimental new code. May subtly affect the output or even produce wrong
results. For testing only. </dd> </dl>

</dd> <dt><samp class="option">srcw <var class="var">(API only)</var></samp></dt>
<dd> Set source width. </dd> <dt><samp class="option">srch
<var class="var">(API only)</var></samp></dt> <dd> Set
source height. </dd> <dt><samp class="option">dstw <var
class="var">(API only)</var></samp></dt> <dd> Set destination width. </dd> <dt><samp class="option">dsth <var
class="var">(API only)</var></samp></dt> <dd> Set destination height. </dd> <dt><samp class="option">src_format <var
class="var">(API only)</var></samp></dt> <dd> Set source pixel
format (must be expressed as an integer). </dd> <dt><samp
class="option">dst_format <var class="var">(API only)</var></samp></dt>
<dd> Set destination pixel format (must be expressed as an integer). </dd> <dt><samp class="option">src_range <var

class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, indicates source is full range. Default value is <code class=”code”>0</code>, which indicates source is limited range. </dd> <dt><samp class=”option”>dst_range <var class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, enable full range for destination. Default value is <code class=”code”>0</code>, which enables limited range. </dd> <dt><samp class=”option”>gamma <var class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, enable gamma correct scaling. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>param0, param1</samp></dt> <dd> Set scaling algorithm parameters. The specified values are specific of some scaling algorithms and ignored by others. The specified values are floating point number values. </dd> <dt><samp class=”option”>sws_dither</samp></dt> <dd> Set the dithering algorithm. Accepts one of the following values. Default value is ‘<samp class=”samp”>auto</samp>’. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> automatic choice </dd> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> no dithering </dd> <dt>‘<samp class=”samp”>bayer</samp>’</dt> <dd> bayer dither </dd> <dt>‘<samp class=”samp”>ed</samp>’</dt> <dd> error diffusion dither </dd> <dt>‘<samp class=”samp”>a_dither</samp>’< <dd> arithmetic dither, based using addition </dd> <dt>‘<samp class=”samp”>x_dither</samp>’</dt> <dd> arithmetic dither, based using xor (more random/less apparent patterning that a_dither). </dd> </dl>

</dd> <dt><samp class=”option”>alphablend</samp></dt> <dd> Set the alpha blending to use when the input has alpha but the output does not. Default value is ‘<samp class=”samp”>none</samp>’. <dl class=”table”> <dt>‘<samp class=”samp”>uniform_color</samp>’</dt> <dd> Blend onto a uniform background color </dd> <dt>‘<samp class=”samp”>checkerboard</samp>’</dt> <dd> Blend onto a checkerboard </dd> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No blending </dd> </dl>

</dd> <dt><samp class=”option”>sws_backends</samp></dt> <dd> Set the allowed swscale backends. This is a flags option, so multiple backends may be combined. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection. Equal to either ‘<samp class=”samp”>stable</samp>’ or ‘<samp class=”samp”>all</samp>’ depending on whether or not the ‘<samp class=”samp”>unstable</samp>’ flag is set. This is the default value. </dd> <dt>‘<samp class=”samp”>stable</samp>’</dt> <dd> All stable backends. </dd> <dt>‘<samp class=”samp”>unstable</samp>’</dt>

<dd> All unstable backends. </dd> <dt>‘<samp class="samp">all</samp>’</dt> <dd> All available backends. </dd> <dt>‘<samp class="samp">legacy</samp>’</dt> <dd> Legacy swscale code. </dd> <dt>‘<samp class="samp">c</samp>’</dt> <dd> Template-based reference code. </dd> <dt>‘<samp class="samp">memcpy</samp>’</dt> <dd> Fast path using libc <code class="code">memcpy</code>. </dd> <dt>‘<samp class="samp">x86</samp>’</dt> <dd> x86 SIMD kernels. </dd> <dt>‘<samp class="samp">aarch64</samp>’</d <dd> AArch64 NEON kernels. </dd> <dt>‘<samp class="samp">spirv</samp>’</dt> <dd> Vulkan SPIR-V backend. </dd> </dl>

</dd> </dl>

 <h2 class="chapter">30 Filtering Introduction# TOC</h2>

Filtering in FFmpeg is enabled through the libavfilter library. In libavfilter, a filter can have multiple inputs and multiple outputs. To illustrate the sorts of things that are possible, we consider the following filtergraph. <pre class="verbatim"> [main] input --> split ----------> overlay --> output | ^ |[tmp] [flip]| +-----> crop --> vflip -------+ </pre> This filtergraph splits the input stream in two streams, then sends one stream through the crop filter and the vflip filter, before merging it back with the other stream by overlaying it on top. You can use the following command to achieve this: <pre class="example-preformatted">ffmpeg -i INPUT -vf "split [main][tmp]; [tmp] crop=iw:ih/2:0:0, vflip [flip]; [main][flip] overlay=0:H/2" OUTPUT </pre>

The result will be that the top half of the video is mirrored onto the bottom half of the output video. Filters in the same linear chain are separated by commas, and distinct linear chains of filters are separated by semicolons. In our example, <var class="var">crop,vflip</var> are in one linear chain, <var class="var">split</var> and <var class="var">overlay</var> are separately in another. The points where the linear chains join are labelled by names enclosed in square brackets. In the example, the split filter generates two outputs that are associated to the labels <var class="var">[main]</var> and <var class="var">[tmp]</var>. The stream sent to the second output of <var class="var">split</var>, labelled as <var class="var">[tmp]</var>, is processed through the <var class="var">crop</var> filter, which crops away the lower half part of the video, and then vertically flipped. The <var class="var">overlay</var> filter takes in input the first unchanged output of the split filter (which was labelled as <var class="var">[main]</var>), and overlay on its lower half the

output generated by the <var class="var">crop,vflip</var> filterchain. Some filters take in input a list of parameters: they are specified after the filter name and an equal sign, and are separated from each other by a colon. There exist so-called <var class="var">source filters</var> that do not have an audio/video input, and <var class="var">sink filters</var> that will not have audio/video output.

 <h2 class="chapter">31 graph2dot# TOC</h2>

The <samp class="file">graph2dot</samp> program included in the FFmpeg <samp class="file">tools</samp> directory can be used to parse a filtergraph description and issue a corresponding textual representation in the dot language. Invoke the command: <pre class="example-preformatted">graph2dot -h </pre>

to see how to use <samp class="file">graph2dot</samp>. You can then pass the dot description to the <samp class="file">dot</samp> program (from the graphviz suite of programs) and obtain a graphical representation of the filtergraph. For example the sequence of commands: <pre class="example-preformatted">echo <var class="var">GRAPH_DESCRIPTION</var> | \ tools/graph2dot -o graph.tmp && \ dot -Tpng graph.tmp -o graph.png && \ display graph.png </pre>

can be used to create and display an image representing the graph described by the <var class="var">GRAPH_DESCRIPTION</var> string. Note that this string must be a complete self-contained graph, with its inputs and outputs explicitly defined. For example if your command line is of the form: <pre class="example-preformatted">ffmpeg -i infile -vf scale=640:360 outfile </pre> your <var class="var">GRAPH_DESCRIPTION</var> string will need to be of the form: <pre class="example-preformatted">nullsrc,scale=640:360,nullsink </pre> you may also need to set the <var class="var">nullsrc</var> parameters and add a <var class="var">format</var> filter in order to simulate a specific input file.

 <h2 class="chapter">32 Filtergraph description# TOC</h2>

A filtergraph is a directed graph of connected filters. It can contain cycles, and there can be multiple links between a pair of filters. Each link has one input pad on one side connecting it to one filter from which it takes its input, and one output pad on the other side connecting it

to one filter accepting its output. Each filter in a filtergraph is an instance of a filter class registered in the application, which defines the features and the number of input and output pads of the filter. A filter with no input pads is called a "source", and a filter with no output pads is called a "sink". <h3 class="section">32.1 Filtergraph syntax# TOC</h3>

A filtergraph has a textual representation, which is recognized by the <samp class="option">-filter</samp>/<samp class="option">-vf</samp>/<samp class="option">-af</samp> and <samp class="option">-filter_complex</samp> options in <code class="command">ffmpeg</code> and <samp class="option">-vf</samp>/<samp class="option">-af</samp> in <code class="command">ffplay</code>, and by the <code class="code">avfilter_graph_parse_ptr()</co function defined in <samp class="file">libavfilter/avfilter.h</samp>. A filterchain consists of a sequence of connected filters, each one connected to the previous one in the sequence. A filterchain is represented by a list of ","-separated filter descriptions. A filtergraph consists of a sequence of filterchains. A sequence of filterchains is represented by a list of ";"-separated filterchain descriptions. A filter is represented by a string of the form: [<var class="var">in_link_1</var>]...[<var class="var">in_link_N</var>]<var class="var">filter_name</var>@<var class="var">id</var>=<var class="var">arguments</var>[<var class="var">out_link_1</var>]...[<var class="var">out_link_M</var>] <var class="var">filter_name</var> is the name of the filter class of which the described filter is an instance of, and has to be the name of one of the filter classes registered in the program optionally followed by "@<var class="var">id</var>". The name of the filter class is optionally followed by a string "=<var class="var">arguments</var>". <var class="var">arguments</var> is a string which contains the parameters used to initialize the filter instance. It may have one of two forms: <ul class="itemize mark-bullet"> A ’:’-separated list of <var class="var">key=value</var> pairs.

A ’:’-separated list of <var class="var">value</var>. In this case, the keys are assumed to be the option names in the order they are declared. E.g. the <code class="code">fade</code> filter declares three options in this order – <samp class="option">type</samp>, <samp class="option">start_frame</samp> and <samp class="option">nb_frames</samp>. Then the parameter list <var class="var">in:0:30</var> means that the value <var class="var">in</var> is assigned to the option <samp class="option">type</samp>, <var class="var">0</var> to <samp class="option">start_frame</samp> and <var class="var">30</var> to <samp class="option">nb_frames</samp>.

A ’:’-separated list of mixed direct <var class="var">value</var> and long <var class="var">key=value</var> pairs. The direct <var class="var">value</var> must precede the <var class="var">key=value</var> pairs, and follow the same constraints order of the previous point. The following <var class="var">key=value</var> pairs can be set in any preferred order.

If the option value itself is a list of items (e.g. the <code class="code">format</code> filter takes a list of pixel formats), the items in the list are usually separated by ‘<samp class="samp">|</samp>’. The list of arguments can be quoted using the character ‘<samp class="samp">'</samp>’ as initial and ending mark, and the character ‘<samp class="samp">\</samp>’ for escaping the characters within the quoted text; otherwise the argument string is considered terminated when the next special character (belonging to the set ‘<samp class="samp">[]=;,</samp>’) is encountered. A special syntax implemented in the <code class="command">ffmpeg</code> CLI tool allows loading option values from files. This is done be prepending a slash ’/’ to the option name, then the supplied value is interpreted as a path from which the actual value is loaded. E.g. <pre class="example-preformatted">ffmpeg -i <INPUT> -vf drawtext=/text=/tmp/some_text <OUTPUT> </pre> will load the text to be drawn from <samp class="file">/tmp/some_text</samp>. API users wishing to implement a similar feature should use the <code class="code">avfilter_graph_segment_*()</code> functions together with custom IO code. The name and arguments of the filter are optionally preceded and followed by a list of link labels. A link label allows one to name a link and associate it to a filter output or input pad. The preceding labels <var class="var">in_link_1</var> ... <var class="var">in_link_N</var>, are associated to the filter input pads, the following labels <var class="var">out_link_1</var> ... <var class="var">out_link_M</var>, are associated to the output pads. When two link labels with the same name are found in the filtergraph, a link between the corresponding input and output pad is created. If an output pad is not labelled, it is linked by default to the first unlabelled input pad of the next filter in the filterchain. For example in the filterchain <pre class="example-preformatted">nullsrc, split[L1], [L2]overlay, nullsink </pre> the split filter instance has two output pads, and the overlay filter instance two input pads. The first output pad of split is labelled "L1", the first input pad of overlay is labelled "L2", and the second output pad of split is linked to the second input pad of overlay, which are both unlabelled. In a filter description, if the input label of the first filter is not specified, "in" is assumed; if the output label of the last filter is not specified, "out" is assumed. In a complete filterchain all the unlabelled filter input and output pads must be connected. A filtergraph is considered valid if all

the filter input and output pads of all the filterchains are connected. Leading and trailing whitespaces (space, tabs, or line feeds) separating tokens in the filtergraph specification are ignored. This means that the filtergraph can be expressed using empty lines and spaces to improve readability. For example, the filtergraph: <pre class="example-preformatted">testsrc,split[L1],hflip[L2];[L1][L2] hstack </pre>

can be represented as: <pre class="example-preformatted">testsrc, split [L1], hflip [L2];

[L1][L2] hstack </pre>

Libavfilter will automatically insert scale filters where format conversion is required. It is possible to specify swscale flags for those automatically inserted scalers by prepending <code class="code">sws_flags=<var class="var">flags</var>;</code> to the filtergraph description. Here is a BNF description of the filtergraph syntax: <pre class="example-preformatted"><var class="var">NAME</var> ::= sequence of alphanumeric characters and '_' <var class="var">FILTER_NAME</var> ::= <var class="var">NAME</var>["@"<var class="var">NAME</var>] <var class="var">LINKLABEL</var> ::= "[" <var class="var">NAME</var> "]" <var class="var">LINKLABELS</var> ::= <var class="var">LINKLABEL</var> [<var class="var">LINKLABELS</var>] <var class="var">FILTER_ARGUMENTS</var> ::= sequence of chars (possibly quoted) <var class="var">FILTER</var> ::= [<var class="var">LINKLABELS</var>] <var class="var">FILTER_NAME</var> ["=" <var class="var">FILTER_ARGUMENTS</var>] [<var class="var">LINKLABELS</var>] <var class="var">FILTERCHAIN</var> ::= <var class="var">FILTER</var> [,<var class="var">FILTERCHAIN</var>] <var class="var">FILTERGRAPH</var> ::= [sws_flags=<var class="var">flags</var>;] <var class="var">FILTERCHAIN</var> [;<var class="var">FILTERGRAPH</var>] </pre>

 <h3 class="section">32.2 Notes on filtergraph escaping# TOC</h3>

Filtergraph description composition entails several levels of escaping. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual for more information about the employed escaping procedure. A first level escaping affects the content of each filter option value, which may contain the special character <code class="code">:</code> used to separate values, or one of the escaping characters <code class="code">\'</code>. A

second level escaping affects the whole filter description, which may contain the escaping characters <code class=”code”>\'</code> or the special characters <code class=”code”>[],;</code> used by the filtergraph description. Finally, when you specify a filtergraph on a shell commandline, you need to perform a third level escaping for the shell special characters contained within it. For example, consider the following string to be embedded in the drawtext filter description <samp class=”option”>text</samp> value: <pre class=”example-preformatted”>this is a 'string': may contain one, or more, special characters </pre>

This string contains the <code class=”code”>'</code> special escaping character, and the <code class=”code”>:</code> special character, so it needs to be escaped in this way: <pre class=”example-preformatted”>text=this is a \'string\'\: may contain one, or more, special characters </pre>

A second level of escaping is required when embedding the filter description in a filtergraph description, in order to escape all the filtergraph special characters. Thus the example above becomes: <pre class=”example-preformatted”>drawtext=text=this is a \\\'string\\\'\\: may contain one\, or more\, special characters </pre> (note that in addition to the <code class=”code”>\'</code> escaping special characters, also <code class=”code”>,</code> needs to be escaped). Finally an additional level of escaping is needed when writing the filtergraph description in a shell command, which depends on the escaping rules of the adopted shell. For example, assuming that <code class=”code”>\</code> is special and needs to be escaped with another <code class=”code”>\</code>, the previous string will finally result in: <pre class=”example-preformatted”>-vf "drawtext=text=this is a \\\\\\'string\\\\\\'\\\\: may contain one\\, or more\\, special characters" </pre>

In order to avoid cumbersome escaping when using a commandline tool accepting a filter specification as input, it is advisable to avoid direct inclusion of the filter or options specification in the shell. For example, in case of the drawtext filter, you might prefer to use the <samp class=”option”>textfile</samp> option in place of <samp class=”option”>text</samp> to specify the text to render. <h2 class=”chapter”>33 Timeline editing# TOC</h2>

Some filters support a generic <samp class=”option”>enable</samp> option. For the filters supporting timeline editing, this option can be set to an expression which is evaluated before sending a frame to the filter. If the evaluation is non-zero, the filter will be enabled, otherwise the frame will be sent unchanged to the next filter in the filtergraph. The expression

accepts the following values: <dl class="table"> <dt>‘<samp class="samp">t</samp>’</dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt>‘<samp class="samp">n</samp>’</dt> <dd> sequential number of the input frame, starting from 0 </dd> <dt>‘<samp class="samp">pos</samp>’</dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt>‘<samp class="samp">w</samp>’</dt> <dt>‘<samp class="samp">h</samp>’</dt> <dd> width and height of the input frame if video </dd> </dl>

Additionally, these filters support an <samp class="option">enable</samp> command that can be used to re-define the expression. Like any other filtering option, the <samp class="option">enable</samp> option follows the same rules. For example, to enable a blur filter (smartblur) from 10 seconds to 3 minutes, and a curves filter starting at 3 seconds: <pre class="example-preformatted">smartblur = enable='between(t,10,3*60)', curves = enable='gte(t,3)' : preset=cross_process </pre>

See <code class="code">ffmpeg -filters</code> to view which filters have timeline support.

 <h2 class="chapter">34 Changing options at runtime with a command# TOC</h2>

Some options can be changed during the operation of the filter using a command. These options are marked ’T’ on the output of <code class="command">ffmpeg</code> <samp class="option">-h filter=<name of filter></samp>. The name of the command is the name of the option and the argument is the new value. <h2 class="chapter">35 Options for filters with several inputs (framesync)# TOC</h2>

Some filters with several inputs support a common set of options. These options can only be set by name, not with the short notation. <dl class="table"> <dt><samp class="option">eof_action</samp></dt> <dd> The action to take when EOF is encountered on the secondary input;

it accepts one of the following values: <dl class="table"> <dt><samp class="option">repeat</samp></dt> <dd> Repeat the last frame (the default). </dd> <dt><samp class="option">endall</samp></dt> <dd> End both streams. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass the main input through. </dd> </dl>

</dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> If set to 1, force the filter to extend the last frame of secondary streams until the end of the primary stream. A value of 0 disables this behavior. Default value is 1. </dd> <dt><samp class="option">ts_sync_mode</samp></dt> <dd> How strictly to sync streams based on secondary input timestamps; it accepts one of the following values: <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Frame from secondary input with the nearest lower or equal timestamp to the primary input frame. </dd> <dt><samp class="option">nearest</samp></dt> <dd> Frame from secondary input with the absolute nearest timestamp to the primary input frame. </dd> </dl> </dd> </dl>

 <h2 class="chapter">36 Audio Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class="code">--disable-filters</code>. The configure output will show the audio filters included in your build. Below is a description of the currently available audio filters. <h3 class="section">36.1 aap# TOC</h3> Apply Affine Projection algorithm to the first audio stream using the second audio stream. This adaptive filter is used to estimate unknown audio based on multiple input audio samples. Affine projection algorithm can make trade-offs between computation complexity with convergence speed. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class="option">projection</samp></dt> <dd> Set the projection order. </dd> <dt><samp class="option">mu</samp></dt> <dd> Set the filter mu. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt>

<dd> Pass the 2nd input. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class=”option”>e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class=”section”>36.2 acompressor# TOC</h3>

A compressor is mainly used to reduce the dynamic range of a signal. Especially modern music is mostly compressed at a high ratio to improve the overall loudness. It’s done to get the highest attention of a listener, "fatten" the sound and bring more "power" to the track. If a signal is compressed too much it may sound dull or "dead" afterwards or it may start to "pump" (which could be a powerful effect but can also destroy a track completely). The right compression is the key to reach a professional sound and is the high art of mixing and mastering. Because of its complex settings it may take a long time to get the right feeling for this kind of effect. Compression is done by detecting the volume above a chosen level <code class=”code”>threshold</code> and dividing it by the factor set with <code class=”code”>ratio</code>. So if you set the threshold to -12dB and your signal reaches -6dB a ratio of 2:1 will result in a signal at -9dB. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over the time. This is done by setting "Attack" and "Release". <code class=”code”>attack</code> determines how long the signal has to rise above the threshold before any reduction will occur and <code class=”code”>release</code> sets the time the signal has to fall below the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. The overall reduction of the signal can be made up afterwards with the <code class=”code”>makeup</code> setting. So compressing the peaks of a signal about 6dB and raising the makeup to this level results in a signal twice as loud than the source. To gain a softer entry in the compression the <code class=”code”>knee</code>

flattens the hard edge at the threshold in the range of the chosen decibels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal of stream rises above this level it will affect the gain reduction. By default it is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. 1:2 means that if the level rose 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the <code class="code">average</code> level between all channels of input stream or the louder(<code class="code">maximum</code>) channel of input stream affects the reduction. Default is <code class="code">average</code>. </dd> <dt><samp class="option">detection</samp></dt> <dd> Should the exact signal be taken in case of <code class="code">peak</code> or an RMS one in case of <code class="code">rms</code>. Default is <code class="code">rms</code> which is mostly smoother. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class="subsection">36.2.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.3 acontrast# TOC</h3> Simple audio dy-

namic range compression/expansion filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set contrast. Default is 33. Allowed range is between 0 and 100. </dd> </dl>

 <h3 class="section">36.4 acopy# TOC</h3>

Copy the input audio source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">36.5 acrossfade# TOC</h3>

Apply cross fade from one input audio stream to another input audio stream. The cross fade is applied for specified duration near the end of first stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs, n</samp></dt> <dd> Specify the number of inputs to crossfade. When crossfading multiple inputs, each input will be concatenated and crossfaded in sequence, similar to the concat filter. Default is 2. </dd> <dt><samp class="option">nb_samples, ns</samp></dt> <dd> Specify the number of samples for which the cross fade effect has to last. At the end of the cross fade effect the first input audio will be completely silent. Default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the cross fade effect. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. By default the duration is determined by <var class="var">nb_samples</var>. If set this option is used instead of <var class="var">nb_samples</var>. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Should first stream end overlap with second stream start. Default is enabled. </dd> <dt><samp class="option">curve1</samp></dt> <dd> Set curve for cross fade transition for first stream. </dd> <dt><samp class="option">curve2</samp></dt> <dd> Set curve for cross fade transition for second stream. For description of available curve types see afade filter description. </dd> </dl>

 <h4 class="subsection">36.5.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input to another: <pre class="example-preformatted">ffmpeg

-i first.flac -i second.flac -filter_complex acrossfade=d=10:c1=exp:c2=exp output.flac </pre>

Cross fade from one input to another but without overlapping: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:o=0:c1=exp:c2=exp output.flac </pre>

Concatenate multiple inputs with cross fade between each: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -i third.flac -filter_complex acrossfade=n=3 output.flac </pre>

 <h3 class="section">36.6 acrossover# TOC</h3> Split audio stream into several bands. This filter splits audio stream into two or more frequency ranges. Summing all streams back will give flat output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">split</samp></dt> <dd> Set split frequencies. Those must be positive and increasing. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order for each band split. This controls filter roll-off or steepness of filter transfer function. Available values are: <dl class="table"> <dt>‘<samp class="samp">2nd</samp>’</dt> <dd> 12 dB per octave. </dd> <dt>‘<samp class="samp">4th</samp>’</dt> <dd> 24 dB per octave. </dd> <dt>‘<samp class="samp">6th</samp>’</dt> <dd> 36 dB per octave. </dd> <dt>‘<samp class="samp">8th</samp>’</dt> <dd> 48 dB per octave. </dd> <dt>‘<samp class="samp">10th</samp>’</dt> <dd> 60 dB per octave. </dd> <dt>‘<samp class="samp">12th</samp>’</dt> <dd> 72 dB per octave. </dd> <dt>‘<samp class="samp">14th</samp>’</dt> <dd> 84 dB per octave. </dd> <dt>‘<samp class="samp">16th</samp>’</dt> <dd> 96 dB per octave. </dd> <dt>‘<samp class="samp">18th</samp>’</dt> <dd> 108 dB per octave. </dd> <dt>‘<samp class="samp">20th</samp>’</dt> <dd> 120 dB per octave. </dd> </dl>

Default is <var class="var">4th</var>. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">gains</samp></dt> <dd> Set output gain for each band. Default value is 1 for all bands. </dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt>

<dd> Always use double-floating point precision sample format. </dd> </dl>

Default value is <code class="code">auto</code>. </dd> </dl>

 <h4 class="subsection">36.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Split input audio stream into two bands (low and high) with split frequency of 1500 Hz, each band will be in separate stream: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but with higher filter order: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500:order=8th[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but also with additional middle band (frequencies between 1500 and 8000): <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500 8000:order=8th[LOW][MID][HIGH]' -map '[LOW]' low.wav -map '[MID]' mid.wav -map '[HIGH]' high.wav </pre>

 <h3 class="section">36.7 acrusher# TOC</h3>

Reduce audio bit resolution. This filter is bit crusher with enhanced functionality. A bit crusher is used to audibly reduce number of bits an audio signal is sampled with. This doesn’t change the bit depth at all, it just produces the effect. Material reduced in bit depth sounds more harsh and "digital". This filter is able to even round to continuous values instead of discrete bit depths. Additionally it has a D/C offset which results in different crushing of the lower and the upper half of the signal. An Anti-Aliasing setting is able to produce "softer" crushing sounds. Another feature of this filter is the logarithmic mode. This setting switches from linear distances between bits to logarithmic ones. The result is a much more "natural" sounding crusher which doesn’t gate low signals for example. The human ear has a logarithmic perception, so this kind of crushing is much more pleasant. Logarithmic crushing is also able to get anti-aliased. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set level in. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set

level out. </dd> <dt><samp class="option">bits</samp></dt>
<dd> Set bit reduction. </dd> <dt><samp class="option">mix</samp></dt>
<dd> Set mixing amount. </dd> <dt><samp class="option">mode</samp></dt>
<dd> Can be linear: <code class="code">lin</code> or logarithmic: <code class="code">log</code>. </dd> <dt><samp class="option">dc</samp></dt> <dd> Set DC. </dd>
<dt><samp class="option">aa</samp></dt> <dd> Set anti-aliasing.
 </dd> <dt><samp class="option">samples</samp></dt> <dd> Set
sample reduction. </dd> <dt><samp class="option">lfo</samp></dt>
<dd> Enable LFO. By default disabled. </dd> <dt><samp
class="option">lforange</samp></dt> <dd> Set LFO range.
</dd> <dt><samp class="option">lforate</samp></dt> <dd> Set
LFO rate. </dd> </dl>

 <h4 class="subsection">36.7.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">36.8 acue<span
class="pull-right"><a class="anchor hidden-xs" href="#acue" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-acue"
aria-hidden="true">TOC</h3>

Delay audio filtering until a given wallclock timestamp. See the <a
class="ref" href="#cue">cue filter.
<h3 class="section">36.9 adeclick<a class="anchor
hidden-xs" href="#adeclick" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-adeclick" aria-hidden="true">TOC</h3>
Remove impulsive noise from input audio. Samples detected
as impulsive noise are replaced by interpolated samples using autoregressive
modelling. <dl class="table"> <dt><samp class="option">window,
w</samp></dt> <dd> Set window size, in milliseconds. Allowed range
is from <code class="code">10</code> to <code class="code">100</code>.
Default value is <code class="code">55</code> milliseconds. This
sets size of window which will be processed at once. </dd>
<dt><samp class="option">overlap, o</samp></dt> <dd> Set
window overlap, in percentage of window size. Allowed range is from <code
class="code">50</code> to <code class="code">95</code>. Default value
is <code class="code">75</code> percent. Setting this to a very high value
increases impulsive noise removal but makes whole process much slower.
</dd> <dt><samp class="option">arorder, a</samp></dt> <dd> Set
autoregression order, in percentage of window size. Allowed range is from <code
class="code">0</code> to <code class="code">25</code>. Default value is
<code class="code">2</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp

class=”option”>threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class=”code”>1</code> to <code class=”code”>100</code>. Default value is <code class=”code”>2</code>. This controls the strength of impulsive noise which is going to be removed. The lower value, the more samples will be detected as impulsive noise. </dd> <dt><samp class=”option”>burst, b</samp></dt> <dd> Set burst fusion, in percentage of window size. Allowed range is <code class=”code”>0</code> to <code class=”code”>10</code>. Default value is <code class=”code”>2</code>. If any two samples detected as noise are spaced less than this value then any sample between those two samples will be also detected as noise. </dd> <dt><samp class=”option”>method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class=”option”>save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class=”code”>a</code>. </dd> </dl>

 <h3 class=”section”>36.10 adeclip# TOC</h3> Remove clipped samples from input audio. Samples detected as clipped are replaced by interpolated samples using autoregressive modelling. <dl class=”table”> <dt><samp class=”option”>window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range is from <code class=”code”>10</code> to <code class=”code”>100</code>. Default value is <code class=”code”>55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class=”option”>overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class=”code”>50</code> to <code class=”code”>95</code>. Default value is <code class=”code”>75</code> percent. </dd> <dt><samp class=”option”>arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class=”code”>0</code> to <code class=”code”>25</code>. Default value is <code class=”code”>8</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class=”code”>1</code> to <code class=”code”>100</code>. Default value is <code class=”code”>10</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class=”option”>hsize, n</samp></dt> <dd> Set size of histogram used to detect clips. Allowed range is from <code class=”code”>100</code> to <code class=”code”>9999</code>. Default value is <code class=”code”>1000</code>.

Higher values make clip detection less aggressive. </dd> <dt><samp class="option">method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class="table"> <dt><samp class="option">add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class="option">save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class="code">a</code>. </dd> </dl>

 <h3 class="section">36.11 adecorrelate# TOC</h3> Apply decorrelation to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">stages</samp></dt> <dd> Set decorrelation stages of filtering. Allowed range is from 1 to 16. Default value is 6. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set random seed used for setting delay in samples across channels. </dd> </dl>

 <h3 class="section">36.12 adelay# TOC</h3>

Delay one or more audio channels. Samples in delayed channel are filled with silence. The filter accepts the following option: <dl class="table"> <dt><samp class="option">delays</samp></dt> <dd> Set list of delays in milliseconds for each channel separated by ’|’. Unused delays will be silently ignored. If number of given delays is smaller than number of channels all remaining channels will not be delayed. If you want to delay exact number of samples, append ’S’ to number. If you want instead to delay in seconds, append ’s’ to number. </dd> <dt><samp class="option">all</samp></dt> <dd> Use last set delay for all remaining channels. By default is disabled. This option if enabled changes how option <code class="code">delays</code> is interpreted. </dd> </dl>

 <h4 class="subsection">36.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Delay first channel by 1.5 seconds, the third channel by 0.5 seconds and leave the second channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=1500|0|500 </pre>

Delay second channel by 500 samples, the third channel by 700 samples and leave the first channel (and any other channels that may be present) unchanged. <pre class=”example-preformatted”>adelay=0|500S|700S </pre>

Delay all channels by same number of samples: <pre class=”example-preformatted”>adelay=delays=64S:all=1 </pre>

 <h3 class=”section”>36.13 adenorm# TOC</h3> Remedy denormals in audio by adding extremely low-level noise. This filter shall be placed before any filter that can produce denormals. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>level</samp></dt> <dd> Set level of added noise in dB. Default is <code class=”code”>-351</code>. Allowed range is from -451 to -90. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set type of added noise. <dl class=”table”> <dt><samp class=”option”>dc</samp></dt> <dd> Add DC signal. </dd> <dt><samp class=”option”>ac</samp></dt> <dd> Add AC signal. </dd> <dt><samp class=”option”>square</samp></dt> <dd> Add square signal. </dd> <dt><samp class=”option”>pulse</samp></dt> <dd> Add pulse signal. </dd> </dl>

Default is <code class=”code”>dc</code>. </dd> </dl>

 <h4 class=”subsection”>36.13.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.14 aderivative, aintegral# TOC</h3>

Compute derivative/integral of audio stream. Applying both filters one after another produces original audio. <h3 class=”section”>36.15 adrc# TOC</h3>

Apply spectral dynamic range controller filter to input audio stream. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>transfer</samp></dt>

<dd> Set the transfer expression. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> current channel number </dd> <dt><samp class="option">sn</samp></dt> <dd> current sample number </dd> <dt><samp class="option">nb_channels</samp></dt> <dd> number of channels </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds </dd> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">p</samp></dt> <dd> current frequency power value, in dB </dd> <dt><samp class="option">f</samp></dt> <dd> current frequency in Hz </dd> </dl>

Default value is <code class="code">p</code>. </dd> <dt><samp class="option">attack</samp></dt> <dd> Set the attack in milliseconds. Default is <code class="code">50</code> milliseconds. Allowed range is from 1 to 1000 milliseconds. </dd> <dt><samp class="option">release</samp></dt> <dd> Set the release in milliseconds. Default is <code class="code">100</code> milliseconds. Allowed range is from 5 to 2000 milliseconds. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set which channels to filter, by default <code class="code">all</code> channels in audio stream are filtered. </dd> </dl>

 <h4 class="subsection">36.15.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">36.15.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply spectral compression to all frequencies with threshold of -50 dB and 1:6 ratio: <pre class="example-preformatted">adrc=transfer='if(gt(p,-50),-50+(p-(-50))/6,p)':attack=50:release=100 </pre>

Similar to above but with 1:2 ratio and filtering only front center channel: <pre class="example-preformatted">adrc=transfer='if(gt(p,-50),-50+(p-(-50))/2,p)':attack=50:release=100:channels=FC </pre>

Apply spectral noise gate to all frequencies with threshold of -85 dB and with short attack time and short release time: <pre class="example-preformatted">adrc=transfer='if(lte(p,-85),p-800,p)':attack=1:release=5 </pre>

Apply spectral expansion to all frequencies with threshold of

-10 dB and 1:2 ratio: <pre class=”example-preformatted”>adrc=transfer='if(lt(p,-10),-10+(p-(-10))*2,p)':attack=50:release=100 </pre>

Apply limiter to max -60 dB to all frequencies, with attack of 2 ms and release of 10 ms: <pre class=”example-preformatted”>adrc=transfer='min(p,-60)':attack=2:release=10 </pre>

 <h3 class=”section”>36.16 adynamicequalizer# TOC</h3>

Apply dynamic equalization to input audio stream.

A description of the accepted options follows.

<dl class=”table”>

<dt><samp class=”option”>threshold</samp></dt>

<dd> Set the detection threshold used to trigger equalization.

Threshold detection is using detection filter.

Default value is 0. Allowed range is from 0 to 100.

</dd>

<dt><samp class=”option”>dfrequency</samp></dt>

<dd> Set the detection frequency in Hz used for detection filter used to trigger equalization.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class=”option”>dqfactor</samp></dt>

<dd> Set the detection resonance factor for detection filter used to trigger equalization.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">tfrequency</samp></dt>

<dd> Set the target frequency of equalization filter.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">tqfactor</samp></dt>

<dd> Set the target resonance factor for target equalization filter.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to rise above

the detection threshold before equalization starts.

Default is 20. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">release</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to fall below the

detection threshold before equalization ends.

Default is 200. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">ratio</samp></dt>

<dd> Set the ratio by which the equalization gain is raised.

Default is 1. Allowed range is between 0 and 30.

</dd>

<dt><samp class="option">makeup</samp></dt>

<dd> Set the makeup offset by which the equalization gain is raised.

Default is 0. Allowed range is between 0 and 100.

</dd>

<dt><samp class="option">range</samp></dt>

<dd> Set the max allowed cut/boost amount. Default is 50.

Allowed range is from 1 to 200.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Set the mode of filter operation, can be one of the following:

<dl class="table">

<dt>‘<samp class="samp">listen</samp>’</dt>

<dd> Output only isolated detection signal.

</dd>

<dt>‘<samp class="samp">cutbelow</samp>’</dt>

<dd> Cut frequencies below detection threshold.

</dd>

<dt>‘<samp class="samp">cutabove</samp>’</dt>

<dd> Cut frequencies above detection threshold.

</dd>

<dt>‘<samp class="samp">boostbelow</samp>’</dt>

<dd> Boost frequencies below detection threshold.

</dd>

<dt>‘<samp class="samp">boostabove</samp>’</dt>

<dd> Boost frequencies above detection threshold.

</dd>

</dl>

Default mode is ‘<samp class="samp">cutbelow</samp>’.

</dd>

<dt><samp class="option">dftype</samp></dt>
<dd> Set the type of detection filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bandpass</samp>’</dt>
<dt>‘<samp class="samp">lowpass</samp>’</dt>
<dt>‘<samp class="samp">highpass</samp>’</dt>
<dt>‘<samp class="samp">peak</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bandpass</samp>’.

</dd>
<dt><samp class="option">tftype</samp></dt>
<dd> Set the type of target filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bell</samp>’</dt>
<dt>‘<samp class="samp">lowshelf</samp>’</dt>
<dt>‘<samp class="samp">highshelf</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bell</samp>’.

</dd>
<dt><samp class="option">auto</samp></dt>
<dd> Automatically gather threshold from detection filter. By default
is ‘<samp class="samp">disabled</samp>’.
This option is useful to detect threshold in certain time frame of
input audio stream, in such case option value is changed at runtime.

Available values are:
<dl class="table">

<dt>‘<samp class="samp">disabled</samp>’</dt>

<dd> Disable using automatically gathered threshold value.

</dd>

<dt>‘<samp class="samp">off</samp>’</dt>

<dd> Stop picking threshold value.

</dd>

<dt>‘<samp class="samp">on</samp>’</dt>

<dd> Start picking threshold value.

</dd>

<dt>‘<samp class="samp">adaptive</samp>’</dt>

<dd> Adaptively pick threshold value, by calculating sliding window entropy.

</dd>

</dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class="subsection">36.16.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.17 adynamicsmooth# TOC</h3>

Apply dynamic smoothing to input audio stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sensitivity</samp></dt> <dd> Set an amount of sensitivity to frequency fluctations. Default is 2. Allowed range is from 0 to 1e+06. </dd> <dt><samp class="option">basefreq</samp></dt> <dd> Set a base frequency for smoothing. Default value is 22050. Allowed range is from 2 to 1e+06. </dd> </dl>

 <h4 class="subsection">36.17.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.18 aecho# TOC</h3>

Apply echoing to the input audio. Echoes are reflected sound and can occur naturally amongst mountains (and sometimes large buildings) when talking or shouting; digital echo effects emulate this behaviour and are often used to help fill out the sound of a single instrument or vocal. The time difference between the original signal and the reflection is the <code class="code">delay</code>, and the loudness of the reflected signal is the <code class="code">decay</code>. Multiple echoes can have different delays and decays. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain of reflected signal. Default is <code class="code">0.6</code>. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain of reflected signal. Default is <code class="code">0.3</code>. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set list of time intervals in milliseconds between original signal and reflections separated by ’|’. Allowed range for each <code class="code">delay</code> is <code class="code">(0 - 90000.0]</code>. Default is <code class="code">1000</code>. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set list of loudness of reflected signals separated by ’|’. Allowed range for each <code class="code">decay</code> is <code class="code">(0 - 1.0]</code>. Default is <code class="code">0.5</code>. </dd> </dl>

 <h4 class="subsection">36.18.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make it sound as if there are twice as many instruments as are actually playing: <pre class="example-preformatted">aecho=0.8:0.88:60:0.4 </pre>

If delay is very short, then it sounds like a (metallic) robot playing music: <pre class="example-preformatted">aecho=0.8:0.88:6:0.4 </pre>

A longer delay will sound like an open air concert in the mountains: <pre class="example-preformatted">aecho=0.8:0.9:1000:0.3

</pre>

Same as above but with one more mountain: <pre class="example-preformatted">aecho=0.8:0.9:1000|1800:0.3|0.25 </pre>

 <h3 class="section">36.19 aemphasis# TOC</h3> Audio emphasis filter creates or restores material directly taken from LPs or emphased CDs with different filter curves. E.g. to store music on vinyl the signal has to be altered by a filter first to even out the disadvantages of this recording medium. Once the material is played back the inverse filter has to be applied to restore the distortion of the frequency response. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set filter mode. For restoring material use <code class="code">reproduction</code> mode, otherwise use <code class="code">production</code> mode. Default is <code class="code">reproduction</code> mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Set filter type. Selects medium. Can be one of the following: <dl class="table"> <dt><samp class="option">col</samp></dt> <dd> select Columbia. </dd> <dt><samp class="option">emi</samp></dt> <dd> select EMI. </dd> <dt><samp class="option">bsi</samp></dt> <dd> select BSI (78RPM). </dd> <dt><samp class="option">riaa</samp></dt> <dd> select RIAA. </dd> <dt><samp class="option">cd</samp></dt> <dd> select Compact Disc (CD). </dd> <dt><samp class="option">50fm</samp></dt> <dd> select 50µs (FM). </dd> <dt><samp class="option">75fm</samp></dt> <dd> select 75µs (FM). </dd> <dt><samp class="option">50kf</samp></dt> <dd> select 50µs (FM-KF). </dd> <dt><samp class="option">75kf</samp></dt> <dd> select 75µs (FM-KF). </dd> </dl> </dd> </dl>

 <h4 class="subsection">36.19.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.20 aeval# TOC</h3>

Modify an audio signal according to the specified expressions. This filter accepts one or more expressions (one for each channel), which

are evaluated and used to modify a corresponding audio signal. It accepts the following parameters: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set output channel layout. If not specified, the channel layout is specified by the number of expressions. If set to ‘<samp class="samp">same</samp>’, it will use by default the same input channel layout. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds </dd> <dt><samp class="option">nb_in_channels</samp></dt> <dt><samp class="option">nb_out_channels</samp><
<dd> input and output number of channels </dd> <dt><samp class="option">val(CH)</samp></dt> <dd> the value of input channel with number <var class="var">CH</var> </dd> </dl>

Note: this filter is slow. For faster processing you should use a dedicated filter. <h4 class="subsection">36.20.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Half volume: <pre class="example-preformatted">aeval=val(ch)/2:c=same </pre>

Invert phase of the second channel: <pre class="example-preformatted">aeval=val(0)|-val(1) </pre>

 <h3 class="section">36.21 aexciter# TOC</h3>

An exciter is used to produce high sound that is not present in the original signal. This is done by creating harmonic distortions of the signal which are restricted in range and added to the original signal. An Exciter raises the upper end of an audio signal without simply raising the higher frequencies like an equalizer would do to create a more "crisp" or "brilliant" sound. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>level_in</samp></dt> <dd> Set input level prior processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class=”option”>amount</samp></dt> <dd> Set the amount of harmonics added to original signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class=”option”>drive</samp></dt> <dd> Set the amount of newly created harmonics. Allowed range is from 0.1 to 10. Default value is 8.5. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set the octave of newly created harmonics. Allowed range is from -10 to 10. Default value is 0. </dd> <dt><samp class=”option”>freq</samp></dt> <dd> Set the lower frequency limit of producing harmonics in Hz. Allowed range is from 2000 to 12000 Hz. Default is 7500 Hz. </dd> <dt><samp class=”option”>ceil</samp></dt> <dd> Set the upper frequency limit of producing harmonics. Allowed range is from 9999 to 20000 Hz. If value is lower than 10000 Hz no limit is applied. </dd> <dt><samp class=”option”>listen</samp></dt> <dd> Mute the original signal and output only added harmonics. By default is disabled. </dd> </dl>

 <h4 class=”subsection”>36.21.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.22 afade# TOC</h3>

Apply fade-in/out effect to input audio.

A description of the accepted parameters follows.

<dl class=”table”>

<dt><samp class=”option”>type, t</samp></dt>

<dd> Specify the effect type, can be either <code class=”code”>in</code> for fade-in, or

<code class=”code”>out</code> for a fade-out effect. Default is <code class=”code”>in</code>.

</dd>

<dt><samp class="option">start_sample, ss</samp></dt>

<dd> Specify the number of the start sample for starting to apply the fade

effect. Default is 0.

</dd>

<dt><samp class="option">nb_samples, ns</samp></dt>

<dd> Specify the number of samples for which the fade effect has to last. At

the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence. Default is 44100.

</dd>

<dt><samp class="option">start_time, st</samp></dt>

<dd> Specify the start time of the fade effect. Default is 0.

The value must be specified as a time duration; see

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If set this option is used instead of <var class="var">start_sample</var>.

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Specify the duration of the fade effect. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

At the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence.

By default the duration is determined by <var class="var">nb_samples</var>.

If set this option is used instead of <var class="var">nb_samples</var>.

</dd>
<dt><samp class="option">curve</samp></dt>
<dd> Set curve for fade transition.

It accepts the following values:
<dl class="table">
<dt><samp class="option">tri</samp></dt>
<dd> select triangular, linear slope (default)
</dd>
<dt><samp class="option">qsin</samp></dt>
<dd> select quarter of sine wave
</dd>
<dt><samp class="option">hsin</samp></dt>
<dd> select half of sine wave
</dd>
<dt><samp class="option">esin</samp></dt>
<dd> select exponential sine wave
</dd>
<dt><samp class="option">log</samp></dt>
<dd> select logarithmic
</dd>
<dt><samp class="option">ipar</samp></dt>
<dd> select inverted parabola
</dd>
<dt><samp class="option">qua</samp></dt>
<dd> select quadratic
</dd>
<dt><samp class="option">cub</samp></dt>
<dd> select cubic
</dd>

<dt><samp class="option">squ</samp></dt>
<dd> select square root
</dd>
<dt><samp class="option">cbr</samp></dt>
<dd> select cubic root
</dd>
<dt><samp class="option">par</samp></dt>
<dd> select parabola
</dd>
<dt><samp class="option">exp</samp></dt>
<dd> select exponential
</dd>
<dt><samp class="option">iqsin</samp></dt>
<dd> select inverted quarter of sine wave
</dd>
<dt><samp class="option">ihsin</samp></dt>
<dd> select inverted half of sine wave
</dd>
<dt><samp class="option">dese</samp></dt>
<dd> select double-exponential seat
</dd>
<dt><samp class="option">desi</samp></dt>
<dd> select double-exponential sigmoid
</dd>
<dt><samp class="option">losi</samp></dt>
<dd> select logistic sigmoid
</dd>
<dt><samp class="option">sinc</samp></dt>
<dd> select sine cardinal function
</dd>
<dt><samp class="option">isinc</samp></dt>

<dd> select inverted sine cardinal function

</dd>

<dt><samp class="option">quat</samp></dt>

<dd> select quartic

</dd>

<dt><samp class="option">quatr</samp></dt>

<dd> select quartic root

</dd>

<dt><samp class="option">qsin2</samp></dt>

<dd> select squared quarter of sine wave

</dd>

<dt><samp class="option">hsin2</samp></dt>

<dd> select squared half of sine wave

</dd>

<dt><samp class="option">nofade</samp></dt>

<dd> no fade applied

</dd>

</dl>

</dd> <dt><samp class="option">silence</samp></dt> <dd> Set the initial gain for fade-in or final gain for fade-out. Default value is <code class="code">0.0</code>. </dd> <dt><samp class="option">unity</samp></dt> <dd> Set the initial gain for fade-out or final gain for fade-in. Default value is <code class="code">1.0</code>. </dd> </dl>

 <h4 class="subsection">36.22.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">36.22.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in first 15 seconds of audio: <pre class="example-preformatted">afade=t=in:ss=0:d=15 </pre>

Fade out last 25 seconds of a 900 seconds audio:
<pre class="example-preformatted">afade=t=out:st=875:d=25 </pre>

 <h3 class="section">36.23 afftdn<span class="pull-
right">#
TOC</h3>
Denoise audio samples with FFT. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">noise_reduction, nr</samp></dt> <dd> Set the noise reduction in dB, allowed range is 0.01 to 97. Default value is 12 dB. </dd> <dt><samp class="option">noise_floor, nf</samp></dt> <dd> Set the noise floor in dB, allowed range is -80 to -20. Default value is -50 dB. </dd> <dt><samp class="option">noise_type, nt</samp></dt> <dd> Set the noise type. It accepts the following values: <dl class="table"> <dt><samp class="option">white, w</samp></dt> <dd> Select white noise. </dd> <dt><samp class="option">vinyl, v</samp></dt> <dd> Select vinyl noise. </dd> <dt><samp class="option">shellac, s</samp></dt> <dd> Select shellac noise. </dd> <dt><samp class="option">custom, c</samp></dt> <dd> Select custom noise, defined in <code class="code">bn</code> option. Default value is white noise. </dd> </dl>

</dd> <dt><samp class="option">band_noise, bn</samp></dt> <dd> Set custom band noise profile for every one of 15 bands. Bands are separated by ’ ’ or ’|’. </dd> <dt><samp class="option">residual_floor, rf</samp></dt> <dd> Set the residual floor in dB, allowed range is -80 to -20. Default value is -38 dB. </dd> <dt><samp class="option">track_noise, tn</samp></dt> <dd> Enable noise floor tracking. By default is disabled. With this enabled, noise floor is automatically adjusted. </dd> <dt><samp class="option">track_residual, tr</samp></dt> <dd> Enable residual tracking. By default is disabled. </dd> <dt><samp class="option">output_mode, om</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">input, i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">output, o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class="option">noise, n</samp></dt> <dd> Pass only noise. Default value is <var class="var">output</var>. </dd> </dl>

</dd> <dt><samp class="option">adaptivity, ad</samp></dt> <dd> Set the adaptivity factor, used how fast to adapt gains adjustments per each frequency bin. Value <var class="var">0</var> enables instant adaptation, while higher values react much slower. Allowed range is from <var class="var">0</var> to <var class="var">1</var>. De-

fault value is <var class="var">0.5</var>. </dd> <dt><samp class="option">floor_offset, fo</samp></dt> <dd> Set the noise floor offset factor. This option is used to adjust offset applied to measured noise floor. It is only effective when noise floor tracking is enabled. Allowed range is from <var class="var">-2.0</var> to <var class="var">2.0</var>. Default value is <var class="var">1.0</var>. </dd> <dt><samp class="option">noise_link, nl</samp></dt> <dd> Set the noise link used for multichannel audio. It accepts the following values: <dl class="table"> <dt><samp class="option">none</samp></dt> <dd> Use unchanged channel’s noise floor. </dd> <dt><samp class="option">min</samp></dt> <dd> Use measured min noise floor of all channels. </dd> <dt><samp class="option">max</samp></dt> <dd> Use measured max noise floor of all channels. </dd> <dt><samp class="option">average</samp></dt> <dd> Use measured average noise floor of all channels. Default value is <var class="var">min</var>. </dd> </dl>

</dd> <dt><samp class="option">band_multiplier, bm</samp></dt> <dd> Set the band multiplier factor, used how much to spread bands across frequency bins. Allowed range is from <var class="var">0.2</var> to <var class="var">5</var>. Default value is <var class="var">1.25</var>. </dd> <dt><samp class="option">sample_noise, sn</samp></dt> <dd> Toggle capturing and measurement of noise profile from input audio. It accepts the following values: <dl class="table"> <dt><samp class="option">start, begin</samp></dt> <dd> Start sample noise capture. </dd> <dt><samp class="option">stop, end</samp></dt> <dd> Stop sample noise capture and measure new noise band profile. Default value is <code class="code">none</code>. </dd> </dl>

</dd> <dt><samp class="option">gain_smooth, gs</samp></dt> <dd> Set gain smooth spatial radius, used to smooth gains applied to each frequency bin. Useful to reduce random music noise artefacts. Higher values increases smoothing of gains. Allowed range is from <code class="code">0</code> to <code class="code">50</code>. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">36.23.1 Commands# TOC</h4>

This filter supports the some above mentioned options as commands. <h4 class="subsection">36.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Reduce white noise by 10dB, and use previously measured noise floor of -40dB: <pre class="example-preformatted">afftdn=nr=10:nf=-40 </pre>

Reduce white noise by 10dB, also set initial noise floor to -80dB and enable automatic tracking of noise floor so noise floor will gradually change during processing: <pre class="example-preformatted">afftdn=nr=10:nf=-80:tn=1 </pre>

Reduce noise by 20dB, using noise floor of -40dB and using commands to take noise profile of first 0.4 seconds of input audio: <pre class="example-preformatted">asendcmd=0.0 afftdn sn start,asendcmd=0.4 afftdn sn stop,afftdn=nr=20:nf=-40 </pre>

 <h3 class="section">36.24 afftfilt# TOC</h3> Apply arbitrary expressions to samples in frequency domain. <dl class="table"> <dt><samp class="option">real</samp></dt> <dd> Set frequency domain real expression for each separate channel separated by ’|’. Default is "re". If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">imag</samp></dt> <dd> Set frequency domain imaginary expression for each separate channel separated by ’|’. Default is "im". Each expression in <var class="var">real</var> and <var class="var">imag</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">b</samp></dt> <dd> current frequency bin number </dd> <dt><samp class="option">nb</samp></dt> <dd> number of available bins </dd> <dt><samp class="option">ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd> <dt><samp class="option">pts</samp></dt> <dd> current frame pts </dd> <dt><samp class="option">re</samp></dt> <dd> current real part of frequency bin of current channel </dd> <dt><samp class="option">im</samp></dt> <dd> current imaginary part of frequency bin of current channel </dd> <dt><samp class="option">real(b, ch)</samp></dt> <dd> Return the value of real part of frequency bin at location (<var class="var">bin</var>,<var class="var">channel</var>) </dd> <dt><samp class="option">imag(b, ch)</samp></dt> <dd> Return the value of imaginary part of frequency bin at location (<var class="var">bin</var>,<var class="var">channel</var>) </dd> </dl>

</dd> <dt><samp class="option">win_size</samp></dt> <dd> Set

window size. Allowed range is from 16 to 131072. Default is <code class="code">4096</code> </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann, hanning</samp>’</dt> <dt>‘<samp class="samp">hamming</samp>’</dt> <dt>‘<samp class="samp">blackman</samp>’</dt> <dt>‘<samp class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</ <dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’< <dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’< <dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d <dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’< <dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class="code">0.75</code>. </dd> </dl>

 <h4 class="subsection">36.24.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Leave almost only low frequencies in audio: <pre class="example-preformatted">afftfilt="'real=re * (1-clip((b/nb)*b,0,1))':imag='im * (1-clip((b/nb)*b,0,1))'" </pre>

Apply robotize effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*sin(0)':imag='hypot(re,im)*cos(0)':win_size=512:overlap= </pre>

Apply whisper effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*cos((random(0)*2-1)*2*3.14)':imag='hypot(re,im)*sin((ran 1)*2*3.14)':win_size=128:overlap=0.8" </pre>

Apply phase shift: <pre class="example-preformatted">afftfilt="real=re*cos(1)-im*sin(1):imag=re*sin(1)+im*cos(1)" </pre>

 <h3 class="section">36.25 afir# TOC</h3>

Apply an arbitrary Finite Impulse Response filter.

This filter is designed for applying long FIR filters,
up to 60 seconds long.

It can be used as component for digital crossover filters,
room equalization, cross talk cancellation, wavefield synthesis,
auralization, ambiophonics, ambisonics and spatialization.

This filter uses the streams higher than first one as FIR coefficients.
If the non-first stream holds a single channel, it will be used
for all input channels in the first stream, otherwise
the number of channels in the non-first stream must be same as
the number of channels in the first stream.

It accepts the following parameters:

<dl class="table">
<dt><samp class="option">dry</samp></dt>
<dd> Set dry gain. This sets input gain.

</dd>
<dt><samp class="option">wet</samp></dt>
<dd> Set wet gain. This sets final output gain.

</dd>
<dt><samp class="option">length</samp></dt>
<dd> Set Impulse Response filter length. Default is 1, which means whole IR is processed.

</dd>

<dt><samp class="option">gtype</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">irnorm</samp></dt>

<dd> Set norm to be applied to IR coefficients before filtering.

Allowed range is from <var class="var">-1</var> to <var class="var">2</var>.

IR coefficients are normalized with calculated vector norm set by this option.

For negative values, no norm is calculated, and IR coefficients are not modified at all.

Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">irlink</samp></dt>

<dd> For multichannel IR if this option is set to <var class="var">true</var>, all IR channels will be

normalized with maximal measured gain of all IR channels coefficients as set by <code class="code">irnorm</code> option.

When disabled, all IR coefficients in each IR channel will be normalized independently.

Default is <var class="var">true</var>.

</dd>

<dt><samp class="option">irgain</samp></dt>

<dd> Set gain to be applied to IR coefficients before filtering.

Allowed range is 0 to 1. This gain is applied after any gain applied with <var class="var">irnorm</var> option.

</dd>

<dt><samp class="option">irfmt</samp></dt>

<dd> Set format of IR stream. Can be <code class="code">mono</code> or <code class="code">input</code>.

Default is <code class="code">input</code>.

</dd>

<dt><samp class="option">maxir</samp></dt>

<dd> Set max allowed Impulse Response filter duration in seconds. Default is 30 seconds.

Allowed range is 0.1 to 60 seconds.

</dd>

<dt><samp class="option">response</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">channel</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">size</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">rate</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">minp</samp></dt>

<dd> Set minimal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">1</var> to <var class="var">65536</var>.

Lower values decreases latency at cost of higher CPU usage.

</dd>

<dt><samp class="option">maxp</samp></dt>

<dd> Set maximal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">8</var> to <var class="var">65536</var>.

Lower values may increase CPU usage.

</dd>

<dt><samp class="option">nbirs</samp></dt>

<dd> Set number of input impulse responses streams which will be switchable at runtime.

Allowed range is from <var class="var">1</var> to <var class="var">32</var>. Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">ir</samp></dt>

<dd> Set IR stream which will be used for convolution, starting from <var class="var">0</var>, should always be

lower than supplied value by <code class="code">nbirs</code> option. Default is <var class="var">0</var>.

This option can be changed at runtime via commands.

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> Set which precision to use when processing samples.

<dl class="table">

<dt><samp class="option">auto</samp></dt>

<dd> Auto pick internal sample format depending on other filters.

</dd>

<dt><samp class="option">float</samp></dt>

<dd> Always use single-floating point precision sample format.

</dd>

<dt><samp class="option">double</samp></dt>

<dd> Always use double-floating point precision sample format.

</dd>

</dl>

Default value is auto. </dd> <dt><samp class="option">irload</samp></dt>
<dd> Set when to load IR stream. Can be <code class="code">init</code>
or <code class="code">access</code>. First one load and prepares all IRs
on initialization, second one once on first access of specific IR. Default is <code
class="code">init</code>. </dd> </dl>

 <h4 class="subsection">36.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply reverb to stream using mono IR file as second input, complete command using ffmpeg: <pre class="example-preformatted">ffmpeg -i input.wav -i middle_tunnel_1way_mono.wav -lavfi afir output.wav </pre>

Apply true stereo processing given input stereo stream, and two stereo impulse responses for left and right channel, the impulse response files are files with names l_ir.wav and r_ir.wav, and setting irnorm option value: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR|c3=FR[a];amovie=l_ir.wav[LIR];amovie=r_ir.wav[RIR
</pre>

Similar to above example, but with <code class="code">irgain</code>
explicitly set to estimated value and with <code class="code">irnorm</code>
disabled: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR
5dB:irnom=-1,pan=stereo|FL<c0+c2|FR<c1+c3" </pre>

 <h3 class="section">36.26 aformat# TOC</h3>

Set output format constraints for the input audio. The framework will negotiate the most appropriate format to minimize conversions. It accepts the following parameters: <dl class="table">
<dt><samp class="option">sample_fmts, f</samp></dt> <dd> A ’|’-separated list of requested sample formats. </dd>
<dt><samp class="option">sample_rates, r</samp></dt> <dd> A

’|’-separated list of requested sample rates. </dd> <dt><samp class="option">channel_layouts, cl</samp></dt> <dd> A ’|’-separated list of requested channel layouts. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> </dl>

If a parameter is omitted, all values are allowed. Force the output to either unsigned 8-bit or signed 16-bit stereo <pre class="example-preformatted">aformat=sample_fmts=u8|s16:channel_layouts=stereo </pre>

 <h3 class="section">36.27 afreqshift# TOC</h3> Apply frequency shift to input audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Specify frequency shift. Allowed range is -INT_MAX to INT_MAX. Default value is 0.0. </dd> <dt><samp class="option">level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">36.27.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.28 afwtdn# TOC</h3> Reduce broadband noise from input samples using Wavelets. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma, allowed range is from 0 to 1. Default value is 0. This option controls strength of denoising applied to input samples. Most useful way to set this option is via decibels, eg. -45dB. </dd> <dt><samp class="option">levels</samp></dt> <dd> Set the number of wavelet levels of decomposition. Allowed range is from 1 to 12. Default value is 10. Setting this too low make denoising performance very poor. </dd> <dt><samp class="option">wavet</samp></dt> <dd> Set wavelet type for decomposition of input frame. They are sorted by number of coefficients, from lowest to highest. More coefficients means worse filtering speed, but overall better

quality. Available wavelets are: <dl class="table"> <dt>‘<samp class="samp">sym2</samp>’</dt> <dt>‘<samp class="samp">sym4</samp>’</dt <dt>‘<samp class="samp">rbior68</samp>’</dt> <dt>‘<samp class="samp">deb10</samp>’</dt> <dt>‘<samp class="samp">sym10</samp>’</ <dt>‘<samp class="samp">coif5</samp>’</dt> <dt>‘<samp class="samp">bl3</samp>’</dt> </dl>

</dd> <dt><samp class="option">percent</samp></dt> <dd> Set percent of full denoising. Allowed range is from 0 to 100 percent. Default value is 85 percent or partial denoising. </dd> <dt><samp class="option">profile</samp></dt> <dd> If enabled, first input frame will be used as noise profile. If first frame samples contain non-noise performance will be very poor. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> If enabled, input frames are analyzed for presence of noise. If noise is detected with high possibility then input frame profile will be used for processing following frames, until new noise frame is detected. </dd> <dt><samp class="option">samples</samp></dt> <dd> Set size of single frame in number of samples. Allowed range is from 512 to 65536. Default frame size is 8192 samples. </dd> <dt><samp class="option">softness</samp></dt> <dd> Set softness applied inside thresholding function. Allowed range is from 0 to 10. Default softness is 1. </dd> </dl>

 <h4 class="subsection">36.28.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.29 agate# TOC</h3>

A gate is mainly used to reduce lower parts of a signal. This kind of signal processing reduces disturbing noise between useful signals. Gating is done by detecting the volume below a chosen level <var class="var">threshold</var> and dividing it by the factor set with <var class="var">ratio</var>. The bottom of the noise floor is set via <var class="var">range</var>. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over time. This is done by setting <var class="var">attack</var> and <var class="var">release</var>. <var class="var">attack</var> determines how long the signal has to fall below the threshold before any reduction will occur and <var class="var">release</var> sets the time the signal has to rise above the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. <dl class="table">

<dt><samp class="option">level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the mode of operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. If set to <code class="code">upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class="code">downward</code> lower parts of signal will be reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is <code class="code">rms</code>. Can be <code class="code">peak</code> or <code class="code">rms</code>. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is <code class="code">average</code>. Can be <code class="code">average</code> or <code class="code">maximum</code>. </dd> </dl>

 <h4 class="subsection">36.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.30 aiir#

TOC</h3>

Apply an arbitrary Infinite Impulse Response filter. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>zeros, z</samp></dt> <dd> Set B/numerator/zeros/reflection coefficients. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set A/denominator/poles/ladder coefficients. </dd> <dt><samp class=”option”>gains, k</samp></dt> <dd> Set channels gains. </dd> <dt><samp class=”option”>dry_gain</samp></dt> <dd> Set input gain. </dd> <dt><samp class=”option”>wet_gain</samp></dt> <dd> Set output gain. </dd> <dt><samp class=”option”>format, f</samp></dt> <dd> Set coefficients format. <dl class=”table”> <dt>‘<samp class=”samp”>ll</samp>’</dt> <dd> lattice-ladder function </dd> <dt>‘<samp class=”samp”>sf</samp>’</dt> <dd> analog transfer function </dd> <dt>‘<samp class=”samp”>tf</samp>’</dt> <dd> digital transfer function </dd> <dt>‘<samp class=”samp”>zp</samp>’</dt> <dd> Z-plane zeros/poles, cartesian (default) </dd> <dt>‘<samp class=”samp”>pr</samp>’</dt> <dd> Z-plane zeros/poles, polar radians </dd> <dt>‘<samp class=”samp”>pd</samp>’</dt> <dd> Z-plane zeros/poles, polar degrees </dd> <dt>‘<samp class=”samp”>sp</samp>’</dt> <dd> S-plane zeros/poles </dd> </dl>

</dd> <dt><samp class=”option”>process, r</samp></dt> <dd> Set type of processing. <dl class=”table”> <dt>‘<samp class=”samp”>d</samp>’</dt> <dd> direct processing </dd> <dt>‘<samp class=”samp”>s</samp>’</dt> <dd> serial processing </dd> <dt>‘<samp class=”samp”>p</samp>’</dt> <dd> parallel processing </dd> </dl>

</dd> <dt><samp class=”option”>precision, e</samp></dt> <dd> Set filtering precision. <dl class=”table”> <dt>‘<samp class=”samp”>dbl</samp>’</dt> <dd> double-precision floating-point (default) </dd> <dt>‘<samp class=”samp”>flt</samp>’</dt> <dd> single-precision floating-point </dd> <dt>‘<samp class=”samp”>i32</samp>’</dt> <dd> 32-bit integers </dd> <dt>‘<samp class=”samp”>i16</samp>’</dt> <dd> 16-bit integers </dd> </dl>

</dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize filter coefficients, by default is enabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>response</samp></dt> <dd> Show IR frequency response, magnitude(magenta), phase(green) and group delay(yellow) in additional video stream. By default it is disabled. </dd> <dt><samp class=”option”>channel</samp></dt> <dd> Set for which IR channel to display frequency response. By default is first channel

displayed. This option is used only when <var class=”var”>response</var> is enabled. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set video stream size. This option is used only when <var class=”var”>response</var> is enabled. </dd> </dl>

Coefficients in <code class=”code”>tf</code> and <code class=”code”>sf</code> format are separated by spaces and are in ascending order. Coefficients in <code class=”code”>zp</code> format are separated by spaces and order of coefficients doesn’t matter. Coefficients in <code class=”code”>zp</code> format are complex numbers with <var class=”var”>i</var> imaginary unit. Different coefficients and gains can be provided for every channel, in such case use ’|’ to separate coefficients or gains. Last provided coefficients will be used for all remaining channels. <h4 class=”subsection”>36.30.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply 2 pole elliptic notch at around 5000Hz for 48000 Hz sample rate: <pre class=”example-preformatted”>aiir=k=1:z=7.95758480780967581OE-1 -2.575128568908332300 3.674839853930788710 -2.57512875289799137 7.9575862963171308801E-1:p=1 -2.86950072432325953 3.63022088054647218 -2.28075678147272232 6.3613623264774235001E-1:f=tf:r=d </pre>

Same as above but in <code class=”code”>zp</code> format: <pre class=”example-preformatted”>aiir=k=0.7957584807809675 6:z=0.80918701+0.58773007i 0.80918701-0.58773007i 0.80884700+0.58784055i 0.80884700-0.58784055i:p=0.63892345+0.59951235i 0.63892345-0.59951235i 0.79582691+0.44198673i 0.79582691-0.44198673i:f=zp:r=s </pre>

Apply 3-rd order analog normalized Butterworth low-pass filter, using analog transfer function format: <pre class=”example-preformatted”>aiir=z=1.3057 0 0 0:p=1.3057 2.3892 2.1860 1:f=sf:r=d </pre>

 <h3 class=”section”>36.31 alimiter# TOC</h3>

The limiter prevents an input signal from rising over a desired threshold. This limiter uses lookahead technology to prevent your signal from distorting. It means that there is a small delay after the signal is processed. Keep in mind that the delay it produces is the attack time you set. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp

class=”option”>limit</samp></dt> <dd> Don’t let signals above this level pass the limiter. Default is 1. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> The limiter will reach its attenuation level in this amount of time in milliseconds. Default is 5 milliseconds. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Come back from limiting to attenuation 1.0 in this amount of milliseconds. Default is 50 milliseconds. </dd> <dt><samp class=”option”>asc</samp></dt> <dd> When gain reduction is always needed ASC takes care of releasing to an average reduction level rather than reaching a reduction of 0 in the release time. </dd> <dt><samp class=”option”>asc_level</samp></dt> <dd> Select how much the release time is affected by ASC, 0 means nearly no changes in release time while 1 produces higher release times. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Auto level output signal. Default is enabled. This normalizes audio back to 0dB if enabled. </dd> <dt><samp class=”option”>latency</samp></dt> <dd> Compensate the delay introduced by using the lookahead buffer set with attack parameter. Also flush the valid audio data in the lookahead buffer when the stream hits EOF. </dd> </dl>

Depending on picked setting it is recommended to upsample input 2x or 4x times with aresample before applying this filter. <h3 class=”section”>36.32 allpass# TOC</h3>

Apply a two-pole all-pass filter with central frequency (in Hz) <var class=”var”>frequency</var>, and filter-width <var class=”var”>width</var>. An all-pass filter changes the audio’s frequency to phase relationship without changing its frequency to amplitude relationship. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt>

<dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">order, o</samp></dt> <dd> Set the filter order, can be 1 or 2. Default is 2. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl> </dd> </dl>

 <h4 class="subsection">36.32.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change allpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change allpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change allpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change allpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">36.33 aloop# TOC</h3>

Loop audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal number of samples. Default is 0. </dd>

<dt><samp class=”option”>start</samp></dt> <dd> Set first sample of loop. Default is 0. </dd> <dt><samp class=”option”>time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class=”var”>start</var> is set to <code class=”code”>-1</code>. </dd> </dl>

 <h3 class=”section”>36.34 amerge# TOC</h3>

Merge two or more audio streams into a single multi-channel stream. All inputs must have the same sample rate, and format. If inputs do not have the same duration, the output will stop with the shortest. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set the number of inputs. Default is 2. </dd> <dt><samp class=”option”>layout_mode</samp></dt> <dd> This option controls how the output channel layout is determined and if the audio channels are reordered during merge. <dl class=”table”> <dt><samp class=”option”>legacy</samp></dt> <dd> This is the mode how the filter behaved historically so it is the default. If the channel layouts of the inputs are known and disjoint, and therefore compatible, the channel layout of the output will be set accordingly and the channels will be reordered as necessary. If the channel layouts of the inputs are not disjoint, some of them are unknown, or they are using special channel layouts, such as ambisonics, the output will have all the channels of the first input then all the channels of the second input, in that order, and the channel layout of the output will be the default value corresponding to the total number of channels. For example, if the first input is in 2.1 (FL+FR+LF) and the second input is FC+BL+BR, then the output will be in 5.1, with the channels in the following order: a1, a2, b1, a3, b2, b3 (a1 is the first channel of the first input, b1 is the first channel of the second input). On the other hand, if both input are in stereo, the output channels will be in the default order: a1, a2, b1, b2, and the channel layout will be arbitrarily set to 4.0, which may or may not be the expected value. </dd> <dt><samp class=”option”>reset</samp></dt> <dd> This mode ignores the input channel layouts and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. The output channel layout will only specify the total channel count. </dd> <dt><samp class=”option”>normal</samp></dt> <dd> This mode keeps channel name and designation information from the input channels and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">36.34.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Merge two mono files into a stereo stream: <pre class="example-preformatted">amovie=left.wav [l] ; amovie=right.mp3 [r] ; [l] [r] amerge </pre>

Multiple merges assuming 1 video stream and 6 audio streams in <samp class="file">input.mkv</samp>: <pre class="example-preformatted">ffmpeg -i input.mkv -filter_complex "[0:1][0:2][0:3][0:4][0:5][0:6] amerge=inputs=6" -c:a pcm_s16le output.mkv </pre>

 <h3 class="section">36.35 amix# TOC</h3>

Mixes multiple audio inputs into a single output. Note that this filter only supports float samples (the <var class="var">amerge</var> and <var class="var">pan</var> audio filters support many formats). If the <var class="var">amix</var> input has integer samples then aresample will be automatically inserted to perform the conversion to float samples. It accepts the following parameters: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class="option">first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> <dt><samp class="option">dropout_transition</samp></dt> <dd> The transition time, in seconds, for volume renormalization when an input stream ends. The default value is 2 seconds. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input audio stream as a sequence of numbers separated by a space. If fewer weights are specified compared to number of inputs, the last weight is assigned to the remaining inputs. Default weight for each input is 1. </dd> <dt><samp class="option">normalize</samp></dt> <dd> Always scale inputs instead of only doing summation of samples. Beware of heavy clipping if inputs are not normalized prior or after filtering by this filter if this option is disabled. By default is enabled. </dd> </dl>

 <h4 class="subsection">36.35.1 Examples<a class="anchor hidden-xs" href="#Examples-

59” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> This will mix 3 input audio streams to a single output with the same duration as the first input and a dropout transition time of 3 seconds: <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex amix=inputs=3:duration=first:dropout_transition=3 OUTPUT </pre>

This will mix one vocal and one music input audio stream to a single output with the same duration as the longest input. The music will have quarter the weight as the vocals, and the inputs are not normalized: <pre class=”example-preformatted”>ffmpeg -i VOCALS -i MUSIC -filter_complex amix=inputs=2:duration=longest:dropout_transition=0:weights="1 0.25":normalize=0 OUTPUT </pre>

 <h4 class=”subsection”>36.35.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>weights</samp></dt> <dt><samp class=”option”>normalize</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class=”section”>36.36 amultiply# TOC</h3>

Multiply first audio stream with second audio stream and store result in output audio stream. Multiplication is done by multiplying each sample from first stream with sample at same position from second stream. With this element-wise multiplication one can create amplitude fades and amplitude modulations. <h3 class=”section”>36.37 anequalizer# TOC</h3>

High-order parametric multiband equalizer for each channel. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>params</samp></dt> <dd> This option string is in format: "c<var class=”var”>chn</var> f=<var class=”var”>cf</var> w=<var class=”var”>w</var> g=<var class=”var”>g</var> t=<var class=”var”>f</var> | ..." Each equalizer band is separated by ’|’. <dl class=”table”> <dt><samp class=”option”>chn</samp></dt>

<dd> Set channel number to which equalization will be applied. If input doesn’t have that channel the entry is ignored. </dd> <dt><samp class=”option”>f</samp></dt> <dd> Set central frequency for band. If input doesn’t have that frequency the entry is ignored. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set band width in Hertz. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set band gain in dB. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set filter type for band, optional, can be: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Butterworth, this is default. </dd> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Chebyshev type 1. </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> Chebyshev type 2. </dd> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>curves</samp></dt> <dd> With this option activated frequency response of anequalizer is displayed in video stream. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set video stream size. Only useful if curves option is activated. </dd> <dt><samp class=”option”>mgain</samp></dt> <dd> Set max gain that will be displayed. Only useful if curves option is activated. Setting this to a reasonable value makes it possible to display gain which is derived from neighbour bands which are too close to each other and thus produce higher gain when both are activated. </dd> <dt><samp class=”option”>fscale</samp></dt> <dd> Set frequency scale used to draw frequency response in video output. Can be linear or logarithmic. Default is logarithmic. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set color for each channel curve which is going to be displayed in video stream. This is list of color names separated by space or by ’|’. Unrecognised or missing colors will be replaced by white color. </dd> </dl>

 <h4 class=”subsection”>36.37.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Lower gain by 10 of central frequency 200Hz and width 100 Hz for first 2 channels using Chebyshev type 1 filter: <pre class=”example-preformatted”>anequalizer=c0 f=200 w=100 g=-10 t=1|c1 f=200 w=100 g=-10 t=1 </pre>

 <h4 class=”subsection”>36.37.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>change</samp></dt> <dd> Alter existing filter parameters. Syntax for the commands is : "<var

class=”var”>fN</var>|f=<var class=”var”>freq</var>|w=<var class=”var”>width</var>|g=<var class=”var”>gain</var>" <var class=”var”>fN</var> is existing filter number, starting from 0, if no such filter is available error is returned. <var class=”var”>freq</var> set new frequency parameter. <var class=”var”>width</var> set new width parameter in Hertz. <var class=”var”>gain</var> set new gain parameter in dB. Full filter invocation with asendcmd may look like this: asendcmd=c=’4.0 anequalizer change 0|f=200|w=50|g=1’,anequalizer=... </dd> </dl>

 <h3 class=”section”>36.38 anlmdn# TOC</h3>

Reduce broadband noise in audio samples using Non-Local Means algorithm. Each sample is adjusted by looking for other samples with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class=”option”>p</samp>. Patches are searched in an area of <samp class=”option”>r</samp> around the sample. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>strength, s</samp></dt> <dd> Set denoising strength. Allowed range is from 0.00001 to 10000. Default value is 0.00001. </dd> <dt><samp class=”option”>patch, p</samp></dt> <dd> Set patch radius duration. Allowed range is from 1 to 100 milliseconds. Default value is 2 milliseconds. </dd> <dt><samp class=”option”>research, r</samp></dt> <dd> Set research radius duration. Allowed range is from 2 to 300 milliseconds. Default value is 6 milliseconds. </dd> <dt><samp class=”option”>output, o</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass only noise. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>smooth, m</samp></dt> <dd> Set smooth factor. Default value is <var class=”var”>11</var>. Allowed range is from <var class=”var”>1</var> to <var class=”var”>1000</var>. </dd> </dl>

 <h4 class=”subsection”>36.38.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h3 class="section">36.39 anlmf, anlms# TOC</h3> Apply Normalized Least-Mean-(Squares|Fourth) algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by finding the filter coefficients that relate to producing the least mean square of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set filter order. </dd> <dt><samp class="option">mu</samp></dt> <dd> Set filter mu. </dd> <dt><samp class="option">eps</samp></dt> <dd> Set the filter eps. </dd> <dt><samp class="option">leakage</samp></dt> <dd> Set the filter leakage. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class="subsection">36.39.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> One of many usages of this filter is noise reduction, input audio is filtered with same samples that are delayed by fixed amount, one such example for stereo audio is: <pre class="example-preformatted">asplit[a][b],[a]adelay=32S|32S[a],[b][a]anlms=order=128:leakage=0.0005:mu=.
</pre>

 <h4 class="subsection">36.39.2 Commands<a class="anchor hidden-xs" href="#Commands-

17" aria-hidden="true"># TOC</h4>

This filter supports the same commands as options, excluding option <code class="code">order</code>. <h3 class="section">36.40 anull# TOC</h3>

Pass the audio source unchanged to the output. <h3 class="section">36.41 apad# TOC</h3>

Pad the end of an audio stream with silence. This can be used together with <code class="command">ffmpeg</code> <samp class="option">-shortest</samp> to extend audio streams to the same length as the video stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">packet_size</samp></dt> <dd> Set silence packet size. Default value is 4096. </dd> <dt><samp class="option">pad_len</samp></dt> <dd> Set the number of samples of silence to add to the end. After the value is reached, the stream is terminated. This option is mutually exclusive with <samp class="option">whole_len</samp>. </dd> <dt><samp class="option">whole_len</samp></dt> <dd> Set the minimum total number of samples in the output audio stream. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_len</samp>. </dd> <dt><samp class="option">pad_dur</samp></dt> <dd> Specify the duration of samples of silence to add. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. </dd> <dt><samp class="option">whole_dur</samp></dt> <dd> Specify the minimum total duration in the output audio stream. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_dur</samp> </dd> </dl>

If neither the <samp class="option">pad_len</samp> nor the <samp class="option">whole_len</samp> nor <samp class="option">pad_dur</samp> nor <samp class="option">whole_dur</samp> option is set, the filter will add silence to the end of the input stream indefinitely. Note that for ffmpeg 4.4 and earlier a zero <samp class="option">pad_dur</samp> or <samp class="option">whole_dur</samp> also caused the filter to add silence indefinitely.

<h4 class="subsection">36.41.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add 1024 samples of silence to the end of the input: <pre class="example-preformatted">apad=pad_len=1024 </pre>

Make sure the audio output will contain at least 10000 samples, pad the input with silence if required: <pre class="example-preformatted">apad=whole_len=10000 </pre>

Use <code class="command">ffmpeg</code> to pad the audio input with silence, so that the video stream will always result the shortest and will be converted until the end in the output file when using the <samp class="option">shortest</samp> option: <pre class="example-preformatted">ffmpeg -i VIDEO -i AUDIO -filter_complex "[1:0]apad" -shortest OUTPUT </pre>

 <h3 class="section">36.42 aphaser# TOC</h3> Add a phasing effect to the input audio. A phaser filter creates series of peaks and troughs in the frequency spectrum. The position of the peaks and troughs are modulated so that they vary over time, creating a sweeping effect. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.74 </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay in milliseconds. Default is 3.0. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set decay. Default is 0.4. </dd> <dt><samp class="option">speed</samp></dt> <dd> Set modulation speed in Hz. Default is 0.5. </dd> <dt><samp class="option">type</samp></dt> <dd> Set modulation type. Default is triangular. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">triangular, t</samp>’</dt> <dt>‘<samp class="samp">sinusoidal, s</samp>’</dt> </dl> </dd> </dl>

 <h3 class="section">36.43 aphaseshift# TOC</h3> Apply phase shift to input audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Specify phase shift. Allowed range is from -1.0 to 1.0. Default value

is 0.0. </dd> <dt><samp class="option">level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">36.43.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.44 apsnr# TOC</h3> Measure Audio Peak Signal-to-Noise Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">36.45 apsyclip# TOC</h3> Apply Psychoacoustic clipper to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">clip</samp></dt> <dd> Set the clipping start value. Default value is 0dBFS or 1. </dd> <dt><samp class="option">diff</samp></dt> <dd> Output only difference samples, useful to hear introduced distortions. By default is disabled. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> Set strength of adaptive distortion applied. Default value is 0.5. Allowed range is from 0 to 1. </dd> <dt><samp class="option">iterations</samp></dt> <dd> Set number of iterations of psychoacoustic clipper. Allowed range is from 1 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Auto level output signal. Default is disabled. This normalizes audio back to 0dBFS if enabled. </dd> </dl>

 <h4 class="subsection">36.45.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.46 apul-

sator# TOC</h3>

Audio pulsator is something between an autopanner and a tremolo. But it can produce funny stereo effects as well. Pulsator changes the volume of the left and right channel based on a LFO (low frequency oscillator) with different waveforms and shifted phases. This filter have the ability to define an offset between left and right channel. An offset of 0 means that both LFO shapes match each other. The left and right channel are altered equally - a conventional tremolo. An offset of 50% means that the shape of the right channel is exactly shifted in phase (or moved backwards about half of the frequency) - pulsator acts as an autopanner. At 1 both curves match again. Every setting in between moves the phase shift gapless between all stages and produces some "bypassing" sounds with sine and triangle waveforms. The more you set the offset near 1 (starting from the 0.5) the faster the signal passes from the left to the right speaker. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set waveform shape the LFO will use. Can be one of: sine, triangle, square, sawup or sawdown. Default is sine. </dd> <dt><samp class=”option”>amount</samp></dt> <dd> Set modulation. Define how much of original signal is affected by the LFO. </dd> <dt><samp class=”option”>offset_l</samp></dt> <dd> Set left channel offset. Default is 0. Allowed range is [0 - 1]. </dd> <dt><samp class=”option”>offset_r</samp></dt> <dd> Set right channel offset. Default is 0.5. Allowed range is [0 - 1]. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set pulse width. Default is 1. Allowed range is [0 - 2]. </dd> <dt><samp class=”option”>timing</samp></dt> <dd> Set possible timing mode. Can be one of: bpm, ms or hz. Default is hz. </dd> <dt><samp class=”option”>bpm</samp></dt> <dd> Set bpm. Default is 120. Allowed range is [30 - 300]. Only used if timing is set to bpm. </dd> <dt><samp class=”option”>ms</samp></dt> <dd> Set ms. Default is 500. Allowed range is [10 - 2000]. Only used if timing is set to ms. </dd> <dt><samp class=”option”>hz</samp></dt> <dd> Set frequency in Hz. Default is 2. Allowed range is [0.01 - 100]. Only used if timing is set to hz. </dd> </dl>

 <h3 class=”section”>36.47 aresample# TOC</h3>

Resample the input audio to the specified parameters, using the libswresample library. If none are specified then the filter will automatically convert between its input and output. This filter is also able to stretch/squeeze the audio data to make it match the timestamps or to inject silence / cut out audio to make it match the timestamps, do a combination of both or do neither. The filter accepts the syntax [<var class="var">sample_rate</var>:]<var class="var">resampler_options</var>, where <var class="var">sample_rate</var> expresses a sample rate and <var class="var">resampler_options</var> is a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ":". See the <a data-manual="ffmpeg-resampler" href="./ffmpeg-resampler.html#Resampler-Options">"Resampler Options" section in the ffmpeg-resampler(1) manual for the complete list of supported options. <h4 class="subsection">36.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Resample the input audio to 44100Hz: <pre class="example-preformatted">aresample=44100 </pre>

Stretch/squeeze samples to the given timestamps, with a maximum of 1000 samples per second compensation: <pre class="example-preformatted">aresample=async=1000 </pre>

 <h3 class="section">36.48 areverse# TOC</h3>

Reverse an audio clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">36.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">atrim=end=5,areverse </pre>

 <h3 class="section">36.49 arls# TOC</h3>
Apply Recursive Least Squares algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by recursively finding the filter coefficients that relate to

producing the minimal weighted linear least squares cost function of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class="option">lambda</samp></dt> <dd> Set the forgetting factor. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class="section">36.50 arnndn# TOC</h3>

Reduce noise from speech using Recurrent Neural Networks. This filter accepts the following options: <dl class="table"> <dt><samp class="option">model, m</samp></dt> <dd> Set train model file to load. This option is always required. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how much to mix filtered samples into final output. Allowed range is from -1 to 1. Default value is 1. Negative values are special, they set how much to keep filtered noise in the final filter output. Set this option to -1 to hear actual noise removed from input signal. </dd> </dl>

 <h4 class="subsection">36.50.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.51 asdr# TOC</h3> Measure Audio Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class=”section”>36.52 asetnsamples# TOC</h3>

Set the number of samples per each output audio frame. The last output packet may contain a different number of samples, as the filter will flush all the remaining samples when the input audio signals its end. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>nb_out_samples, n</samp></dt> <dd> Set the number of frames per each output audio frame. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class=”option”>pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio frame with zeroes, so that the last frame will contain the same number of samples as the previous ones. Default value is 1. </dd> </dl>

For example, to set the number of per-frame samples to 1234 and disable padding for the last frame, use: <pre class=”example-preformatted”>asetnsamples=n=1234:p=0 </pre>

 <h3 class=”section”>36.53 asetrate# TOC</h3>

Set the sample rate without altering the PCM data. This will result in a change of speed and pitch. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the output sample rate. Default is 44100 Hz. </dd> </dl>

 <h3 class=”section”>36.54 ashowinfo# TOC</h3>

Show a line containing various information for each input audio frame. The input audio is not modified. The shown line contains a sequence of key/value pairs of the form <var class=”var”>key</var>:<var class=”var”>value</var>. The following values are shown in the

output: <dl class="table"> <dt><samp class="option">n</samp></dt>
<dd> The (sequential) number of the input frame, starting from 0.
</dd> <dt><samp class="option">pts</samp></dt> <dd> The
presentation timestamp of the input frame, in time base units; the time base de-
pends on the filter input pad, and is usually 1/<var class="var">sample_rate</var>.
 </dd> <dt><samp class="option">pts_time</samp></dt>
<dd> The presentation timestamp of the input frame in seconds.
</dd> <dt><samp class="option">fmt</samp></dt> <dd> The sam-
ple format. </dd> <dt><samp class="option">chlayout</samp></dt>
<dd> The channel layout. </dd> <dt><samp class="option">rate</samp></dt>
<dd> The sample rate for the audio frame. </dd> <dt><samp
class="option">nb_samples</samp></dt> <dd> The number
of samples (per channel) in the frame. </dd> <dt><samp
class="option">checksum</samp></dt> <dd> The Adler-32 check-
sum (printed in hexadecimal) of the audio data. For planar audio, the data
is treated as if all the planes were concatenated. </dd> <dt><samp
class="option">plane_checksums</samp></dt> <dd> A list of Adler-
32 checksums for each data plane. </dd> </dl>

 <h3 class="section">36.55 asisdr<span class="pull-
right">#
TOC</h3>
Measure Audio Scaled-Invariant Signal-to-Distortion Ratio.
This filter takes two audio streams for input, and outputs first au-
dio stream. Results are in dB per channel at end of either input.
 <h3 class="section">36.56 asoftclip<span
class="pull-right"><a class="anchor hidden-xs" href="#asoftclip" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-asoftclip"
aria-hidden="true">TOC</h3> Apply audio soft clipping.
 Soft clipping is a type of distortion effect where the amplitude
of a signal is saturated along a smooth curve, rather than the abrupt
shape of hard-clipping. This filter accepts the following options:
 <dl class="table"> <dt><samp class="option">type</samp></dt>
<dd> Set type of soft-clipping. It accepts the following values:
<dl class="table"> <dt><samp class="option">hard</samp></dt>
<dt><samp class="option">tanh</samp></dt> <dt><samp class="option">atan</samp></dt>
<dt><samp class="option">cubic</samp></dt> <dt><samp class="option">exp</samp></dt>
<dt><samp class="option">alg</samp></dt> <dt><samp class="option">quintic</samp></dt>
<dt><samp class="option">sin</samp></dt> <dt><samp class="option">erf</samp></dt>
</dl>

</dd> <dt><samp class="option">threshold</samp></dt> <dd> Set
threshold from where to start clipping. Default value is 0dB or 1.
</dd> <dt><samp class="option">output</samp></dt> <dd> Set
gain applied to output. Default value is 0dB or 1. </dd>
<dt><samp class="option">param</samp></dt> <dd> Set addi-
tional parameter which controls sigmoid function. </dd> <dt><samp

class=”option”>oversample</samp></dt> <dd> Set oversampling factor. </dd> </dl>

 <h4 class=”subsection”>36.56.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.57 aspectralstats# TOC</h3>

Display frequency domain statistical information about the audio channels. Statistics are calculated and stored as metadata for each audio channel and for each audio frame. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>win_size</samp></dt> <dd> Set the window length in samples. Default value is 2048. Allowed range is from 32 to 65536. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann, hanning</samp>’</dt> <dt>‘<samp class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</ <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’< <dt>‘<samp class=”samp”>sine</samp>’</dt> <dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp class=”samp”>lanczos</samp>’< <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp class=”samp”>dolph</samp>’</d <dt>‘<samp class=”samp”>cauchy</samp>’</dt> <dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp class=”samp”>poisson</samp>’< <dt>‘<samp class=”samp”>bohman</samp>’</dt> <dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl> Default is <code class=”code”>hann</code>. </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set window overlap. Allowed range is from <code class=”code”>0</code> to <code class=”code”>1</code>. Default value is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>measure</samp></dt> <dd> Select the parameters which are measured. The metadata keys can be used as flags, default is <samp class=”option”>all</samp> which measures everything. <samp class=”option”>none</samp> disables all measurement. </dd> </dl>

A list of each metadata key follows: <dl class=”table”> <dt><samp class=”option”>mean</samp></dt> <dt><samp class=”option”>variance</samp></dt>

<dt><samp class="option">centroid</samp></dt> <dt><samp class="option">spread</samp></dt>
<dt><samp class="option">skewness</samp></dt> <dt><samp class="option">kurtosis</samp></dt>
<dt><samp class="option">entropy</samp></dt> <dt><samp class="option">flatness</samp></dt>
<dt><samp class="option">crest</samp></dt> <dt><samp class="option">flux</samp></dt>
<dt><samp class="option">slope</samp></dt> <dt><samp class="option">decrease</samp></dt>
<dt><samp class="option">rolloff</samp></dt> </dl>

 <h3 class="section">36.58 asr#
TOC</h3>
Automatic Speech Recognition This filter uses PocketSphinx for speech recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-pocketsphinx</code>. It accepts the following options: <dl class="table"> <dt><samp class="option">rate</samp></dt> <dd> Set sampling rate of input audio. Defaults is <code class="code">16000</code>. This need to match speech models, otherwise one will get poor results. </dd> <dt><samp class="option">hmm</samp></dt> <dd> Set dictionary containing acoustic model files. </dd> <dt><samp class="option">dict</samp></dt> <dd> Set pronunciation dictionary. </dd> <dt><samp class="option">lm</samp></dt> <dd> Set language model file. </dd> <dt><samp class="option">lmctl</samp></dt> <dd> Set language model set. </dd> <dt><samp class="option">lmname</samp></dt> <dd> Set which language model to use. </dd> <dt><samp class="option">logfn</samp></dt> <dd> Set output for log messages. </dd> </dl>

The filter exports recognized speech as the frame metadata <code class="code">lavfi.asr.text</code>. <h3 class="section">36.59 astats#
TOC</h3>

Display time domain statistical information about the audio channels. Statistics are calculated and displayed for each audio channel and, where applicable, an overall figure is also given. It accepts the following option: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Short window length in seconds, used for peak and through RMS measurement. Default is <code class="code">0.05</code> (50 milliseconds). Allowed range is <code class="code">[0 - 10]</code>. </dd> <dt><samp class="option">metadata</samp></dt> <dd> Set metadata injection. All the metadata keys are prefixed with <code class="code">lavfi.astats.X</code>, where <code class="code">X</code> is channel number starting from 1 or string <code class="code">Overall</code>. Default is disabled. Available keys for each channel are: <var class="var">Bit_depth</var> <var class="var">Crest_factor</var> <var class="var">DC_offset</var> <var

class=”var”>Dynamic_range</var> <var class=”var”>Entropy</var> <var class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var> <var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var> <var class=”var”>Min_difference</var> <var class=”var”>Min_level</var> <var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var> <var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var> <var class=”var”>Number_of_denormals</var> <var class=”var”>Peak_count</var> <var class=”var”>Abs_Peak_count</var> <var class=”var”>Peak_level</var> <var class=”var”>RMS_difference</var> <var class=”var”>RMS_peak</var> <var class=”var”>RMS_trough</var> <var class=”var”>Zero_crossings</var> <var class=”var”>Zero_crossings_rate</var> and for <code class=”code”>Overall</code>: <var class=”var”>Bit_depth</var> <var class=”var”>DC_offset</var> <var class=”var”>Entropy</var> <var class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var> <var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var> <var class=”var”>Min_difference</var> <var class=”var”>Min_level</var> <var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var> <var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var> <var class=”var”>Number_of_denormals</var> <var class=”var”>Number_of_samples</var> <var class=”var”>Peak_count</var> <var class=”var”>Abs_Peak_count</var> <var class=”var”>Peak_level</var> <var class=”var”>RMS_difference</var> <var class=”var”>RMS_level</var> <var class=”var”>RMS_peak</var> <var class=”var”>RMS_trough</var> For example, a full key looks like <code class=”code”>lavfi.astats.1.DC_offset</code> or <code class=”code”>lavfi.astats.Overall.Peak_count</code>. Read below for the description of the keys. </dd> <dt><samp class=”option”>reset</samp></dt> <dd> Set the number of frames over which cumulative stats are calculated before being reset. Default is disabled. </dd> <dt><samp class=”option”>measure_perchannel</samp></dt> <dd> Select the parameters which are measured per channel. The metadata keys can be used as flags, default is <samp class=”option”>all</samp> which measures everything. <samp class=”option”>none</samp> disables all per channel measurement. </dd> <dt><samp class=”option”>measure_overall</samp></dt> <dd> Select the parameters which are measured overall. The metadata keys can be used as flags, default is <samp class=”option”>all</samp> which measures everything. <samp class=”option”>none</samp> disables all overall measurement. </dd> </dl>

A description of the measure keys follow:

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> no measures

</dd>

<dt><samp class="option">all</samp></dt>

<dd> all measures

</dd>

<dt><samp class="option">Bit_depth</samp></dt>

<dd> overall bit depth of audio, i.e. number of bits used for each sample

</dd>

<dt><samp class="option">Crest_factor</samp></dt>

<dd> standard ratio of peak to RMS level (note: not in dB)

</dd>

<dt><samp class="option">DC_offset</samp></dt>

<dd> mean amplitude displacement from zero

</dd>

<dt><samp class="option">Dynamic_range</samp></dt>

<dd> measured dynamic range of audio in dB

</dd>

<dt><samp class="option">Entropy</samp></dt>

<dd> entropy measured across whole audio, entropy of value near 1.0 is typically measured for white noise

</dd>

<dt><samp class="option">Flat_factor</samp></dt>

<dd> flatness (i.e. consecutive samples with the same value) of the signal at its peak levels

(i.e. either <var class="var">Min_level</var> or <var class="var">Max_level</var>)

</dd>

<dt><samp class="option">Max_difference</samp></dt>

<dd> maximal difference between two consecutive samples

</dd>

<dt><samp class="option">Max_level</samp></dt>

<dd> maximal sample level

</dd>

<dt><samp class="option">Mean_difference</samp></dt>

<dd> mean difference between two consecutive samples, i.e. the average of each difference between two consecutive samples

</dd>

<dt><samp class="option">Min_difference</samp></dt>

<dd> minimal difference between two consecutive samples

</dd>

<dt><samp class="option">Min_level</samp></dt>

<dd> minimal sample level

</dd>

<dt><samp class="option">Noise_floor</samp></dt>

<dd> minimum local peak measured in dBFS over a short window

</dd>

<dt><samp class="option">Noise_floor_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained

<var class="var">Noise floor</var>

</dd>

<dt><samp class="option">Number_of_Infs</samp></dt>

<dd> number of samples with an infinite value

</dd>

<dt><samp class="option">Number_of_NaNs</samp></dt>

<dd> number of samples with a NaN (not a number) value

</dd>

<dt><samp class="option">Number_of_denormals</samp></dt>

<dd> number of samples with a subnormal value

</dd>

<dt><samp class="option">Number_of_samples</samp></dt>

<dd> number of samples

</dd>

<dt><samp class="option">Peak_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained either

<var class="var">Min_level</var> or <var class="var">Max_level</var>

</dd>

<dt><samp class="option">Abs_Peak_count</samp></dt>

<dd> number of occasions that the absolute samples taken from the signal attained

max absolute value of <var class="var">Min_level</var> and <var class="var">Max_level</var>

</dd>

<dt><samp class="option">Peak_level</samp></dt>

<dd> standard peak level measured in dBFS

</dd>

<dt><samp class=”option”>RMS_difference</samp></dt>

<dd> Root Mean Square difference between two consecutive samples

</dd>

<dt><samp class=”option”>RMS_level</samp></dt>

<dd> standard RMS level measured in dBFS

</dd>

<dt><samp class=”option”>RMS_peak</samp></dt>

<dt><samp class=”option”>RMS_trough</samp></dt>

<dd> peak and through values for RMS level measured over a short window,

measured in dBFS.

</dd>

<dt><samp class=”option”>Zero crossings</samp></dt>

<dd> number of points where the waveform crosses the zero level axis

</dd>

<dt><samp class=”option”>Zero crossings rate</samp></dt>

<dd> rate of Zero crossings and number of audio samples

</dd>

</dl>

 <h3 class=”section”>36.60 asubboost# TOC</h3> Boost subwoofer frequencies. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dry</samp></dt> <dd> Set dry gain, how much of original signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class=”option”>wet</samp></dt> <dd> Set wet gain, how much of filtered signal is kept. Allowed range is from 0 to 1. Default value is 1.0.

</dd> <dt><samp class="option">boost</samp></dt> <dd> Set max boost factor. Allowed range is from 1 to 12. Default value is 2. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set delay line decay gain value. Allowed range is from 0 to 1. Default value is 0.0. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Set delay line feedback gain value. Allowed range is from 0 to 1. Default value is 0.9. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 50 to 900. Default value is 100. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope amount for cutoff frequency. Allowed range is 0.0001 to 1. Default value is 0.5. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay. Allowed range is from 1 to 100. Default value is 20. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the channels to process. Default value is all available. </dd> </dl>

 <h4 class="subsection">36.60.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.61 asubcut# TOC</h3> Cut subwoofer frequencies. This filter allows to set custom, steeper roll off than highpass filter, and thus is able to more attenuate frequency content in stop-band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 2 to 200. Default value is 20. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">36.61.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.62 asupercut# TOC</h3> Cut super frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt>

<dd> Set cutoff frequency in Hertz. Allowed range is 20000 to 192000. Default value is 20000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">36.62.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.63 asuperpass# TOC</h3> Apply high order Butterworth band-pass filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">36.63.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.64 asuperstop# TOC</h3> Apply high order Butterworth band-stop filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">36.64.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.65 atempo# TOC</h3>

Adjust audio tempo. The filter accepts exactly one parameter, the audio tempo. If not specified then the filter will assume nominal 1.0 tempo. Tempo must be in the [0.5, 100.0] range. Note that tempo greater than 2 will skip some samples rather than blend them in. If for any reason this is a concern it is always possible to daisy-chain several instances of atempo to achieve the desired product tempo. <h4 class="subsection">36.65.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Slow down audio to 80% tempo: <pre class="example-preformatted">atempo=0.8 </pre>

To speed up audio to 300% tempo: <pre class="example-preformatted">atempo=3 </pre>

To speed up audio to 300% tempo by daisy-chaining two atempo instances: <pre class="example-preformatted">atempo=sqrt(3),atempo=sqrt(3) </pre>

 <h4 class="subsection">36.65.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class="var">tempo</var>" </dd> </dl>

 <h3 class="section">36.66 atilt# TOC</h3> Apply spectral tilt filter to audio stream. This filter apply any spectral roll-off slope over any specified frequency band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> Set central

frequency of tilt in Hz. Default is 10000 Hz. </dd> <dt><samp class=”option”>slope</samp></dt> <dd> Set slope direction of tilt. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set width of tilt. Default is 1000. Allowed range is from 100 to 10000. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set order of tilt filter. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input volume level. Allowed range is from 0 to 4. Default is 1. </dd> </dl>

 <h4 class=”subsection”>36.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.67 atrim# TOC</h3>

Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>start</samp></dt> <dd> Timestamp (in seconds) of the start of the section to keep. I.e. the audio sample with the timestamp <var class=”var”>start</var> will be the first sample in the output. </dd> <dt><samp class=”option”>end</samp></dt> <dd> Specify time of the first audio sample that will be dropped, i.e. the audio sample immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last sample in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> Same as <var class=”var”>start</var>, except this option sets the start timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> Same as <var class=”var”>end</var>, except this option sets the end timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_sample</samp></dt> <dd> The number of the first sample that should be output. </dd> <dt><samp class=”option”>end_sample</samp></dt> <dd> The number of the first sample that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the

frame timestamp, while the _sample options simply count the samples that pass through the filter. So start/end_pts and start/end_sample will give different results when the timestamps are wrong, inexact or do not start at zero. Also note that this filter does not modify the timestamps. If you wish to have the output timestamps start at zero, insert the asetpts filter after the atrim filter. If multiple start or end options are set, this filter tries to be greedy and keep all samples that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple atrim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class="itemize mark-bullet"> Drop everything except the second minute of input: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=60:120 </pre>

Keep only the first 1000 samples: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=end_sample=1000 </pre>

 <h3 class="section">36.68 axcorrelate# TOC</h3> Calculate normalized windowed cross-correlation between two input audio streams. Resulted samples are always between -1 and 1 inclusive. If result is 1 it means two input samples are highly correlated in that selected segment. Result 0 means they are not correlated at all. If result is -1 it means two input samples are out of phase, which means they cancel each other. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size</samp></dt> <dd> Set size of segment over which cross-correlation is calculated. Default is 256. Allowed range is from 2 to 131072. </dd> <dt><samp class="option">algo</samp></dt> <dd> Set algorithm for cross-correlation. Can be <code class="code">slow</code> or <code class="code">fast</code> or <code class="code">best</code>. Default is <code class="code">best</code>. Fast algorithm assumes mean values over any given segment are always zero and thus need much less calculations to make. This is generally not true, but is valid for typical audio streams. </dd> </dl>

 <h4 class="subsection">36.68.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Calculate correlation between chan-

nels in stereo audio stream: <pre class=”example-preformatted”>ffmpeg -i stereo.wav -af channelsplit,axcorrelate=size=1024:algo=fast correlation.wav </pre>

 <h3 class=”section”>36.69 bandpass# TOC</h3>

Apply a two-pole Butterworth band-pass filter with central frequency <var class=”var”>frequency</var>, and (3dB-point) band-width width. The <var class=”var”>csg</var> option selects a constant skirt gain (peak gain = Q) instead of the default: constant 0dB peak gain. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class=”code”>3000</code>. </dd> <dt><samp class=”option”>csg</samp></dt> <dd> Constant skirt gain if set to 1. Defaults to 0. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>

<dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.69.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change bandpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bandpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd>
<dt><samp class="option">width, w</samp></dt> <dd> Change bandpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">36.70 bandreject# TOC</h3>

Apply a two-pole Butterworth band-reject filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width <var class="var">width</var>. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify

the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.70.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change bandreject frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change bandreject width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bandreject width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandreject mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class=”section”>36.71 bass, low-shelf# TOC</h3>

Boost or cut the bass (lower) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class=”code”>100</code> Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt>

<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.71.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change bass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change bass gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">36.72 biquad# TOC</h3>

Apply a biquad IIR filter with the given coefficients. Where <var class="var">b0</var>, <var class="var">b1</var>, <var class="var">b2</var> and <var class="var">a0</var>, <var class="var">a1</var>, <var class="var">a2</var> are the numerator and denominator coefficients respectively. and <var class="var">channels</var>, <var class="var">c</var> specify which channels to filter, by default all available are filtered.
 <h4 class="subsection">36.72.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">a0</samp></dt> <dt><samp class="option">a1</samp></dt>
<dt><samp class="option">a2</samp></dt> <dt><samp class="option">b0</samp></dt>
<dt><samp class="option">b1</samp></dt> <dt><samp class="option">b2</samp></dt>

<dd> Change biquad parameter. Syntax for the command is : "<var class="var">value</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h3 class="section">36.73 bs2b# TOC</h3> Bauer stereo to binaural transformation, which improves headphone listening of stereo audio records. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libbs2b</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Pre-defined crossfeed level. <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Default level (fcut=700, feed=50). </dd> <dt><samp class="option">cmoy</samp></dt> <dd> Chu Moy circuit (fcut=700, feed=60). </dd> <dt><samp class="option">jmeier</samp></dt> <dd> Jan Meier circuit (fcut=650, feed=95). </dd> </dl>

</dd> <dt><samp class="option">fcut</samp></dt> <dd> Cut fre-

quency (in Hz). </dd> <dt><samp class="option">feed</samp></dt> <dd> Feed level (in Hz). </dd> </dl>

 <h3 class="section">36.74 channelmap# TOC</h3>

Remap input channels to new locations. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Map channels from input to output. The argument is a ’|’-separated list of mappings, each in the <code class="code"><var class="var">in_channel</var>-<var class="var">out_channel</var></code> or <code class="code"><var class="var">in_channel</var></code> form. <var class="var">in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the input channel layout. <var class="var">out_channel</var> is the name of the output channel or its index in the output channel layout. If <var class="var">out_channel</var> is not given then it is implicitly an index, starting with zero and increasing by one for each mapping. Mixing different types of mappings is not allowed and will result in a parse error. </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the output stream. If not specified, then filter will guess it based on the <var class="var">out_channel</var> names or the number of mappings. Guessed layouts will not necessarily contain channels in the order of the mappings. </dd> </dl>

If no mapping is present, the filter will implicitly map input channels to output channels, preserving indices. <h4 class="subsection">36.74.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a 5.1+downmix input MOV file, <pre class="example-preformatted">ffmpeg -i in.mov -filter 'channelmap=map=DL-FL|DR-FR' out.wav </pre> will create an output WAV file tagged as stereo from the downmix channels of the input. To fix a 5.1 WAV improperly encoded in AAC’s native channel order <pre class="example-preformatted">ffmpeg -i in.wav -filter 'channelmap=1|2|0|5|3|4:5.1' out.wav </pre>

 <h3 class="section">36.75 channelsplit# TOC</h3>

Split each channel from an input audio stream into a separate output stream. It accepts the following parameters: <dl class="table">

<dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the input stream. The default is "stereo". </dd> <dt><samp class="option">channels</samp></dt> <dd> A channel layout describing the channels to be extracted as separate output streams or "all" to extract each input channel as a separate stream. The default is "all". Choosing channels not present in channel layout in the input will result in an error. </dd> </dl>

 <h4 class="subsection">36.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a stereo input MP3 file, <pre class="example-preformatted">ffmpeg -i in.mp3 -filter_complex channelsplit out.mkv </pre> will create an output Matroska file with two audio streams, one containing only the left channel and the other the right channel. Split a 5.1 WAV file into per-channel files: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1[FL][FR][FC][LFE][SL][SR]' -map '[FL]' front_left.wav -map '[FR]' front_right.wav -map '[FC]' front_center.wav -map '[LFE]' lfe.wav -map '[SL]' side_left.wav -map '[SR]' side_right.wav </pre>

Extract only LFE from a 5.1 WAV file: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1:channels=LFE[LFE]' -map '[LFE]' lfe.wav </pre>

 <h3 class="section">36.76 chorus# TOC</h3> Add a chorus effect to the audio. Can make a single vocal sound like a chorus, but can also be applied to instrumentation. Chorus resembles an echo effect with a short delay, but whereas with echo the delay is constant, with chorus, it is varied using sinusoidal or triangular modulation. The modulation depth defines the range the modulated delay is played before or after the delay. Hence the delayed sound will sound slower or faster, that is the delayed sound tuned around the original one, like in a chorus where some vocals are slightly off key. It accepts the following parameters: <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.4. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set delays. A typical delay is around 40ms to 60ms. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set decays. </dd> <dt><samp class="option">speeds</samp></dt>

<dd> Set speeds. </dd> <dt><samp class=”option”>depths</samp></dt> <dd> Set depths. </dd> </dl>

 <h4 class=”subsection”>36.76.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> A single delay: <pre class=”example-preformatted”>chorus=0.7:0.9:55:0.4:0.25:2 </pre>

Two delays: <pre class=”example-preformatted”>chorus=0.6:0.9:50|60:0.4|0.32:0.25|0.4:2|1.3 </pre>

Fuller sounding chorus with three delays: <pre class=”example-preformatted”>chorus=0.5:0.9:50|60|40:0.4|0.32|0.3:0.25|0.4|0.3:2|2.3|1.3 </pre>

 <h3 class=”section”>36.77 compand# TOC</h3> Compress or expand the audio’s dynamic range. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>attacks</samp></dt> <dt><samp class=”option”>decays</samp></dt> <dd> A list of times in seconds for each channel over which the instantaneous level of the input signal is averaged to determine its volume. <var class=”var”>attacks</var> refers to increase of volume and <var class=”var”>decays</var> refers to decrease of volume. For most situations, the attack time (response to the audio getting louder) should be shorter than the decay time, because the human ear is more sensitive to sudden loud audio than sudden soft audio. A typical value for attack is 0.3 seconds and a typical value for decay is 0.8 seconds. If specified number of attacks & decays is lower than number of channels, the last set attack/decay will be used for all remaining channels. </dd> <dt><samp class=”option”>points</samp></dt> <dd> A list of points for the transfer function, specified in dB relative to the maximum possible signal amplitude. Each key points list must be defined using the following syntax: <code class=”code”>x0/y0|x1/y1|x2/y2|....</code> or <code class=”code”>x0/y0 x1/y1 x2/y2 ....</code> The input values must be in strictly increasing order but the transfer function does not have to be monotonically rising. The point <code class=”code”>0/0</code> is assumed but may be overridden (by <code class=”code”>0/out-dBn</code>). Typical values for the transfer function are <code class=”code”>-70/-70|-60/-20|1/0</code>. </dd> <dt><samp class=”option”>soft-knee</samp></dt> <dd> Set the curve radius in dB for all joints. It defaults to 0.01. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set the additional gain in dB to be applied at all points on the transfer function. This allows for easy adjustment of the overall gain. It defaults to 0. </dd> <dt><samp

class=”option”>volume</samp></dt> <dd> Set an initial volume, in dB, to be assumed for each channel when filtering starts. This permits the user to supply a nominal level initially, so that, for example, a very large gain is not applied to initial signal levels before the companding has begun to operate. A typical value for audio which is initially quiet is -90 dB. It defaults to 0. </dd> <dt><samp class=”option”>delay</samp></dt> <dd> Set a delay, in seconds. The input audio is analyzed immediately, but audio is delayed before being fed to the volume adjuster. Specifying a delay approximately equal to the attack/decay times allows the filter to effectively operate in predictive rather than reactive mode. It defaults to 0. </dd> </dl>

 <h4 class=”subsection”>36.77.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make music with both quiet and loud passages suitable for listening to in a noisy environment: <pre class=”example-preformatted”>compand=.3|.3:1|1:-90/-60|-60/-40|-40/-30|-20/-20:6:0:-90:0.2 </pre>

Another example for audio with whisper and explosion parts: <pre class=”example-preformatted”>compand=0|0:1|1:-90/-900|-70/-70|-30/-9|0/-3:6:0:0:0 </pre>

A noise gate for when the noise is at a lower level than the signal: <pre class=”example-preformatted”>compand=.1|.1:.2|.2:-900/-900|-50.1/-900|-50/-50:.01:0:-90:.1 </pre>

Here is another noise gate, this time for when the noise is at a higher level than the signal (making it, in some ways, similar to squelch): <pre class=”example-preformatted”>compand=.1|.1:.1|.1:-45.1/-45.1|-45/-900|0/-900:.01:45:-90:.1 </pre>

2:1 compression starting at -6dB: <pre class=”example-preformatted”>compand=points=-80/-80|-6/-6|0/-3.8|20/3.5 </pre>

2:1 compression starting at -9dB: <pre class=”example-preformatted”>compand=points=-80/-80|-9/-9|0/-5.3|20/2.9 </pre>

2:1 compression starting at -12dB: <pre class=”example-preformatted”>compand=points=-80/-80|-12/-12|0/-6.8|20/1.9 </pre>

2:1 compression starting at -18dB: <pre class=”example-preformatted”>compand=points=-80/-80|-18/-18|0/-9.8|20/0.7 </pre>

3:1 compression starting at -15dB: <pre class=”example-preformatted”>compand=points=-80/-80|-15/-15|0/-10.8|20/-5.2 </pre>

Compressor/Gate: <pre class=”example-preformatted”>compand=points=-80/-105|-62/-80|-15.4/-15.4|0/-12|20/-7.6 </pre>

Expander: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-169|-54/-80|-49.5/-64.6|-41.1/-41.1|-25.8/-15|-10.8/-4.5|0/0|20/8.3 </pre>

Hard limiter at -6dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-6/-6|20/-6 </pre>

Hard limiter at -12dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12/-12|20/-12 </pre>

Hard noise gate at -35 dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-115|-35.1/-80|-35/-35|20/20 </pre>

Soft limiter: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12.4/-12.4|-6/-8|0/-6.8|20/-2.8 </pre>

 <h3 class=”section”>36.78 compensationdelay# TOC</h3>

Compensation Delay Line is a metric based delay to compensate differing positions of microphones or speakers. For example, you have recorded guitar with two microphones placed in different locations. Because the front of sound wave has fixed speed in normal conditions, the phasing of microphones can vary and depends on their location and interposition. The best sound mix can be achieved when these microphones are in phase (synchronized). Note that a distance of ~30 cm between microphones makes one microphone capture the signal in antiphase to the other microphone. That makes the final mix sound moody. This filter helps to solve phasing problems by adding different delays to each microphone track and make them synchronized. The best result can be reached when you take one track as base and synchronize other tracks one by one with it. Remember that synchronization/delay tolerance depends on sample rate, too. Higher sample rates will give more tolerance. The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mm</samp></dt> <dd> Set millimeters distance. This is compensation distance for fine tuning. Default is 0. </dd> <dt><samp class=”option”>cm</samp></dt> <dd> Set cm distance. This is compensation distance for tightening distance setup. Default is 0.

</dd> <dt><samp class="option">m</samp></dt> <dd> Set meters distance. This is compensation distance for hard distance setup. Default is 0. </dd> <dt><samp class="option">dry</samp></dt> <dd> Set dry amount. Amount of unprocessed (dry) signal. Default is 0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet amount. Amount of processed (wet) signal. Default is 1. </dd> <dt><samp class="option">temp</samp></dt> <dd> Set temperature in degrees Celsius. This is the temperature of the environment. Default is 20. </dd> </dl>

 <h4 class="subsection">36.78.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.79 crossfeed# TOC</h3> Apply headphone crossfeed filter. Crossfeed is the process of blending the left and right channels of stereo audio recording. It is mainly used to reduce extreme stereo separation of low frequencies. The intent is to produce more speaker like sound to the listener. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Set strength of crossfeed. Default is 0.2. Allowed range is from 0 to 1. This sets gain of low shelf filter for side part of stereo image. Default is -6dB. Max allowed is -30db when strength is set to 1. </dd> <dt><samp class="option">range</samp></dt> <dd> Set soundstage wideness. Default is 0.5. Allowed range is from 0 to 1. This sets cut off frequency of low shelf filter. Default is cut off near 1550 Hz. With range set to 1 cut off frequency is set to 2100 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set curve slope of low shelf filter. Default is 0.5. Allowed range is from 0.01 to 1. </dd> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 0.9. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class="option">block_size</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.79.1 Commands<a class="anchor hidden-xs" href="#Commands-

34" aria-hidden="true"># TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.80 crystalizer# TOC</h3> Simple algorithm for audio noise sharpening. This filter linearly increases differences between each audio sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Sets the intensity of effect (default: 2.0). Must be in range between -10.0 to 0 (unchanged sound) to 10.0 (maximum effect). To inverse filtering use negative value. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">36.80.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.81 dcshift# TOC</h3> Apply a DC shift to the audio. This can be useful to remove a DC offset (caused perhaps by a hardware problem in the recording chain) from the audio. The effect of a DC offset is reduced headroom and hence volume. The astats filter can be used to determine if a signal has a DC offset. <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Set the DC shift, allowed range is [-1, 1]. It indicates the amount to shift the audio. </dd> <dt><samp class="option">limitergain</samp></dt> <dd> Optional. It should have a value much less than 1 (e.g. 0.05 or 0.02) and is used to prevent clipping. </dd> </dl>

 <h3 class="section">36.82 deesser# TOC</h3>

Apply de-essing to the audio samples. <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Set intensity for triggering de-essing. Allowed range is from 0 to 1. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set amount of ducking on treble part of sound. Allowed range is from 0 to 1.

Default is 0.5. </dd> <dt><samp class="option">f</samp></dt> <dd> How much of original frequency content to keep when de-essing. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">s</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass ess filtered out. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass only ess. Default value is <var class="var">o</var>. </dd> </dl>

</dd> </dl>

 <h3 class="section">36.83 dialoguenhance# TOC</h3> Enhance dialogue in stereo audio. This filter accepts stereo input and produce surround (3.0) channels output. The newly produced front center channel have enhanced speech dialogue originally available in both stereo channels. This filter outputs front left and front right channels same as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">original</samp></dt> <dd> Set the original center factor to keep in front center channel output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">enhance</samp></dt> <dd> Set the dialogue enhance factor to put in front center channel output. Allowed range is from 0 to 3. Default value is 1. </dd> <dt><samp class="option">voice</samp></dt> <dd> Set the voice detection factor. Allowed range is from 2 to 32. Default value is 2. </dd> </dl>

 <h4 class="subsection">36.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.84 drmeter# TOC</h3> Measure audio dynamic range. DR values of 14 and higher is found in very dynamic material. DR of 8 to 13 is found in transition material. And anything less that 8 have very poor dynamics and is very compressed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Set window length in seconds used to split audio into segments of equal length. Default is 3 seconds. </dd> </dl>

<h3 class="section">36.85 dynaudnorm# TOC</h3>

Dynamic Audio Normalizer.

This filter applies a certain amount of gain to the input audio in order

to bring its peak magnitude to a target level (e.g. 0 dBFS). However, in

contrast to more "simple" normalization algorithms, the Dynamic Audio

Normalizer *dynamically* re-adjusts the gain factor to the input audio.

This allows for applying extra gain to the "quiet" sections of the audio

while avoiding distortions or clipping the "loud" sections. In other words:

The Dynamic Audio Normalizer will "even out" the volume of quiet and loud

sections, in the sense that the volume of each section is brought to the

same target level. Note, however, that the Dynamic Audio Normalizer achieves

this goal *without* applying "dynamic range compressing". It will retain 100%

of the dynamic range *within* each section of the audio file.

<dl class="table">

<dt><samp class="option">framelen, f</samp></dt>

<dd> Set the frame length in milliseconds. In range from 10 to 8000 milliseconds.

Default is 500 milliseconds.

The Dynamic Audio Normalizer processes the input audio in small chunks,

referred to as frames. This is required, because a peak magnitude has no

meaning for just a single sample value. Instead, we need to determine the

peak magnitude for a contiguous sequence of sample values. While a "standard"

normalizer would simply use the peak magnitude of the complete file, the

Dynamic Audio Normalizer determines the peak magnitude individually for each frame. The length of a frame is specified in milliseconds. By default, the Dynamic Audio Normalizer uses a frame length of 500 milliseconds, which has been found to give good results with most files.

Note that the exact frame length, in number of samples, will be determined automatically, based on the sampling rate of the individual input audio file.

</dd>

<dt><samp class="option">gausssize, g</samp></dt>

<dd> Set the Gaussian filter window size. In range from 3 to 301, must be odd

number. Default is 31.

Probably the most important parameter of the Dynamic Audio Normalizer is the

<code class="code">window size</code> of the Gaussian smoothing filter. The filter’s window size

is specified in frames, centered around the current frame. For the sake of simplicity, this must be an odd number. Consequently, the default value of 31 takes into account the current frame, as well as the 15 preceding frames and the 15 subsequent frames. Using a larger window results in a stronger smoothing effect and thus in less gain variation, i.e. slower gain adaptation. Conversely, using a smaller window results in a weaker smoothing effect and thus in more gain variation, i.e. faster gain adaptation.

In other words, the more you increase this value, the more the Dynamic Audio Normalizer will behave like a "traditional" normalization filter. On the

contrary, the more you decrease this value, the more the Dynamic Audio Normalizer will behave like a dynamic range compressor.

</dd>

<dt><samp class="option">peak, p</samp></dt>

<dd> Set the target peak value. This specifies the highest permissible magnitude

level for the normalized audio input. This filter will try to approach the

target peak magnitude as closely as possible, but at the same time it also

makes sure that the normalized signal will never exceed the peak magnitude.

A frame’s maximum local gain factor is imposed directly by the target peak

magnitude. The default value is 0.95 and thus leaves a headroom of 5%*.

It is not recommended to go above this value.

</dd>

<dt><samp class="option">maxgain, m</samp></dt>

<dd> Set the maximum gain factor. In range from 1.0 to 100.0. Default is 10.0.

The Dynamic Audio Normalizer determines the maximum possible (local) gain

factor for each input frame, i.e. the maximum gain factor that does not

result in clipping or distortion. The maximum gain factor is determined by

the frame’s highest magnitude sample. However, the Dynamic Audio Normalizer

additionally bounds the frame’s maximum gain factor by a predetermined

(global) maximum gain factor. This is done in order to avoid excessive gain

factors in "silent" or almost silent frames. By default, the maximum gain

factor is 10.0, For most inputs the default value should be sufficient and

it usually is not recommended to increase this value. Though, for input

with an extremely low overall volume level, it may be necessary to allow even

higher gain factors. Note, however, that the Dynamic Audio Normalizer does

not simply apply a "hard" threshold (i.e. cut off values above the threshold).

Instead, a "sigmoid" threshold function will be applied. This way, the

gain factors will smoothly approach the threshold value, but never exceed that

value.

</dd>

<dt><samp class=”option”>targetrms, r</samp></dt>

<dd> Set the target RMS. In range from 0.0 to 1.0. Default is 0.0 - disabled.

By default, the Dynamic Audio Normalizer performs "peak" normalization.

This means that the maximum local gain factor for each frame is defined

(only) by the frame’s highest magnitude sample. This way, the samples can

be amplified as much as possible without exceeding the maximum signal

level, i.e. without clipping. Optionally, however, the Dynamic Audio

Normalizer can also take into account the frame’s root mean square,

abbreviated RMS. In electrical engineering, the RMS is commonly used to

determine the power of a time-varying signal. It is therefore considered

that the RMS is a better approximation of the "perceived loudness" than

just looking at the signal’s peak magnitude. Consequently, by adjusting all

frames to a constant RMS value, a uniform "perceived loudness" can be

established. If a target RMS value has been specified, a frame’s local gain

factor is defined as the factor that would result in exactly that RMS value.

Note, however, that the maximum local gain factor is still restricted by the

frame’s highest magnitude sample, in order to prevent clipping.

</dd>

<dt><samp class=”option”>coupling, n</samp></dt>

<dd> Enable channels coupling. By default is enabled.

By default, the Dynamic Audio Normalizer will amplify all channels by the same

amount. This means the same gain factor will be applied to all channels, i.e.

the maximum possible gain factor is determined by the "loudest" channel.

However, in some recordings, it may happen that the volume of the different

channels is uneven, e.g. one channel may be "quieter" than the other one(s).

In this case, this option can be used to disable the channel coupling. This way,

the gain factor will be determined independently for each channel, depending

only on the individual channel’s highest magnitude sample. This allows for

harmonizing the volume of the different channels.

</dd>

<dt><samp class="option">correctdc, c</samp></dt>

<dd> Enable DC bias correction. By default is disabled.

An audio signal (in the time domain) is a sequence of sample values.

In the Dynamic Audio Normalizer these sample values are represented in the

-1.0 to 1.0 range, regardless of the original input format. Normally, the

audio signal, or "waveform", should be centered around the zero point.

That means if we calculate the mean value of all samples in a file, or in a

single frame, then the result should be 0.0 or at least very close to that

value. If, however, there is a significant deviation of the mean value from

0.0, in either positive or negative direction, this is referred to as a

DC bias or DC offset. Since a DC bias is clearly undesirable, the Dynamic

Audio Normalizer provides optional DC bias correction.

With DC bias correction enabled, the Dynamic Audio Normalizer will determine

the mean value, or "DC correction" offset, of each input frame and subtract

that value from all of the frame’s sample values which ensures those samples

are centered around 0.0 again. Also, in order to avoid "gaps" at the frame

boundaries, the DC correction offset values will be interpolated smoothly

between neighbouring frames.

</dd>

<dt><samp class=”option”>altboundary, b</samp></dt>

<dd> Enable alternative boundary mode. By default is disabled.

The Dynamic Audio Normalizer takes into account a certain neighbourhood

around each frame. This includes the preceding frames as well as the

subsequent frames. However, for the "boundary" frames, located at the very

beginning and at the very end of the audio file, not all neighbouring

frames are available. In particular, for the first few frames in the audio

file, the preceding frames are not known. And, similarly, for the last few

frames in the audio file, the subsequent frames are not known. Thus, the

question arises which gain factors should be assumed for the missing frames

in the "boundary" region. The Dynamic Audio Normalizer implements two modes

to deal with this situation. The default boundary mode assumes a gain factor

of exactly 1.0 for the missing frames, resulting in a smooth "fade in" and

"fade out" at the beginning and at the end of the input, respectively.

</dd>

<dt><samp class=”option”>compress, s</samp></dt>

<dd> Set the compress factor. In range from 0.0 to 30.0. Default is 0.0.

By default, the Dynamic Audio Normalizer does not apply "traditional"

compression. This means that signal peaks will not be pruned and thus the

full dynamic range will be retained within each local neighbourhood. However,

in some cases it may be desirable to combine the Dynamic Audio Normalizer’s

normalization algorithm with a more "traditional" compression.

For this purpose, the Dynamic Audio Normalizer provides an optional compression

(thresholding) function. If (and only if) the compression feature is enabled,
all input frames will be processed by a soft knee thresholding function prior
to the actual normalization process. Put simply, the thresholding function is
going to prune all samples whose magnitude exceeds a certain threshold value.
However, the Dynamic Audio Normalizer does not simply apply a fixed threshold
value. Instead, the threshold value will be adjusted for each individual
frame.
In general, smaller parameters result in stronger compression, and vice versa.
Values below 3.0 are not recommended, because audible distortion may appear.

</dd>
<dt><samp class="option">threshold, t</samp></dt>
<dd> Set the target threshold value. This specifies the lowest permissible
magnitude level for the audio input which will be normalized.
If input frame volume is above this value frame will be normalized.
Otherwise frame may not be normalized at all. The default value is set
to 0, which means all input frames will be normalized.
This option is mostly useful if digital noise is not wanted to be amplified.

</dd>
<dt><samp class="option">channels, h</samp></dt>
<dd> Specify which channels to filter, by default all available channels are
filtered.

</dd>
<dt><samp class="option">overlap, o</samp></dt>
<dd> Specify overlap for frames. If set to 0 (default) no frame overlapping
is done.
Using >0 and <1 values will make less conservative gain adjustments, like
when framelen option is set to smaller value, if framelen option value is
compensated for non-zero overlap then gain adjustments will be smoother across
time

compared to zero overlap case.

</dd>

<dt><samp class="option">curve, v</samp></dt>

<dd> Specify the peak mapping curve expression which is going to be used when calculating

gain applied to frames. The max output frame gain will still be limited by other

options mentioned previously for this filter.

The expression can contain the following constants:

<dl class="table">

<dt><samp class="option">ch</samp></dt>

<dd> current channel number

</dd>

<dt><samp class="option">sn</samp></dt>

<dd> current sample number

</dd>

<dt><samp class="option">nb_channels</samp></dt>

<dd> number of channels

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds

</dd>

<dt><samp class="option">sr</samp></dt>

<dd> sample rate

</dd>

<dt><samp class="option">p</samp></dt>

<dd> current frame peak value

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">36.85.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.86 earwax# TOC</h3>

Make audio easier to listen to on headphones. This filter adds ‘cues’ to 44.1kHz stereo (i.e. audio CD format) audio so that when listened to on headphones the stereo image is moved from inside your head (standard for headphones) to outside and in front of the listener (standard for speakers). Ported from SoX. <h3 class="section">36.87 equalizer# TOC</h3>

Apply a two-pole peaking equalisation (EQ) filter. With this filter, the signal-level at and around a selected frequency can be increased or decreased, whilst (unlike bandpass and bandreject filters) that at all other frequencies is unchanged. In order to produce complex equalisation curves, this filter can be given several times, each with a different central frequency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency in Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Set the required gain or attenuation in dB. Beware of clipping when using a positive gain. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.87.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Attenuate 10 dB at 1000 Hz, with a bandwidth of 200 Hz: <pre class="example-preformatted">equalizer=f=1000:t=h:width=200:g=-10 </pre>

Apply 2 dB gain at 1000 Hz with Q 1 and attenuate 5 dB at 100 Hz with Q 2: <pre class="example-preformatted">equalizer=f=1000:t=q:w=1:g=2,equalizer=f=100:t=q:w=2:g=-5 </pre>

 <h4 class="subsection">36.87.2 Com-

mands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Change equalizer frequency. Syntax for the command is : "<var class=”var”>frequency</var>" </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Change equalizer width_type. Syntax for the command is : "<var class=”var”>width_type</var>" </dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Change equalizer width. Syntax for the command is : "<var class=”var”>width</var>" </dd> <dt><samp class=”option”>gain, g</samp></dt> <dd> Change equalizer gain. Syntax for the command is : "<var class=”var”>gain</var>" </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change equalizer mix. Syntax for the command is : "<var class=”var”>mix</var>" </dd> </dl>

 <h3 class=”section”>36.88 extrastereo# TOC</h3>

Linearly increases the difference between left and right channels which adds some sort of "live" effect to playback. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>m</samp></dt> <dd> Sets the difference coefficient (default: 2.5). 0.0 means mono sound (average of both channels), with 1.0 sound will be unchanged, with -1.0 left and right channels will be swapped. </dd> <dt><samp class=”option”>c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class=”subsection”>36.88.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>36.89 firequalizer# TOC</h3> Apply FIR Equalization using arbitrary frequency response. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain curve equation (in dB). The expression can contain variables: <dl class=”table”> <dt><samp class=”option”>f</samp></dt> <dd> the evaluated frequency </dd> <dt><samp class=”option”>sr</samp></dt>

<dd> sample rate </dd> <dt><samp class="option">ch</samp></dt>
<dd> channel number, set to 0 when multichannels evaluation is disabled </dd> <dt><samp class="option">chid</samp></dt>
<dd> channel id, see libavutil/channel_layout.h, set to the first channel id when multichannels evaluation is disabled </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd>
<dt><samp class="option">chlayout</samp></dt> <dd> channel_layout, see libavutil/channel_layout.h </dd> </dl> and functions:
<dl class="table"> <dt><samp class="option">gain_interpolate(f)</samp></dt>
<dd> interpolate gain on frequency f based on gain_entry </dd>
<dt><samp class="option">cubic_interpolate(f)</samp></dt> <dd> same as gain_interpolate, but smoother </dd> </dl> This option is also available as command. Default is <code class="code">gain_interpolate(f)</code>.
 </dd> <dt><samp class="option">gain_entry</samp></dt>
<dd> Set gain entry for gain_interpolate function. The expression can contain functions: <dl class="table"> <dt><samp class="option">entry(f, g)</samp></dt> <dd> store gain entry at frequency f with value g </dd> </dl> This option is also available as command. </dd> <dt><samp class="option">delay</samp></dt>
<dd> Set filter delay in seconds. Higher value means more accurate. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set filter accuracy in Hz. Lower value means more accurate. Default is <code class="code">5</code>.
 </dd> <dt><samp class="option">wfunc</samp></dt> <dd> Set window function. Acceptable values are: <dl class="table">
<dt><samp class="option">rectangular</samp></dt> <dd> rectangular window, useful when gain curve is already smooth </dd> <dt><samp class="option">hann</samp></dt> <dd> hann window (default)
</dd> <dt><samp class="option">hamming</samp></dt>
<dd> hamming window </dd> <dt><samp class="option">blackman</samp></dt>
<dd> blackman window </dd> <dt><samp class="option">nuttall3</samp></dt>
<dd> 3-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">mnuttall3</samp></dt> <dd> minimum 3-terms discontinuous nuttall window </dd> <dt><samp class="option">nuttall</samp></dt>
<dd> 4-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">bnuttall</samp></dt> <dd> minimum 4-terms discontinuous nuttall (blackman-nuttall) window </dd>
<dt><samp class="option">bharris</samp></dt> <dd> blackman-harris window </dd> <dt><samp class="option">tukey</samp></dt>
<dd> tukey window </dd> </dl>

</dd> <dt><samp class="option">fixed</samp></dt> <dd> If enabled, use fixed number of audio samples. This improves speed when filtering with large delay. Default is disabled. </dd> <dt><samp class="option">multi</samp></dt> <dd> Enable multichannels evaluation on gain. Default is disabled. </dd> <dt><samp

class=”option”>zero_phase</samp></dt> <dd> Enable zero phase mode by subtracting timestamp to compensate delay. Default is disabled. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set scale used by gain. Acceptable values are: <dl class=”table”> <dt><samp class=”option”>linlin</samp></dt> <dd> linear frequency, linear gain </dd> <dt><samp class=”option”>linlog</samp></dt> <dd> linear frequency, logarithmic (in dB) gain (default) </dd> <dt><samp class=”option”>loglin</samp></dt> <dd> logarithmic (in octave scale where 20 Hz is 0) frequency, linear gain </dd> <dt><samp class=”option”>loglog</samp></dt> <dd> logarithmic frequency, logarithmic gain </dd> </dl>

</dd> <dt><samp class=”option”>dumpfile</samp></dt> <dd> Set file for dumping, suitable for gnuplot. </dd> <dt><samp class=”option”>dumpscale</samp></dt> <dd> Set scale for dumpfile. Acceptable values are same with scale option. Default is linlog. </dd> <dt><samp class=”option”>fft2</samp></dt> <dd> Enable 2-channel convolution using complex FFT. This improves speed significantly. Default is disabled. </dd> <dt><samp class=”option”>min_phase</samp></dt> <dd> Enable minimum phase impulse response. Default is disabled. </dd> </dl>

 <h4 class=”subsection”>36.89.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> lowpass at 1000 Hz: <pre class=”example-preformatted”>firequalizer=gain='if(lt(f,1000), 0, -INF)' </pre> lowpass at 1000 Hz with gain_entry: <pre class=”example-preformatted”>firequalizer=gain_entry='entry(1000,0); entry(1001, -INF)' </pre> custom equalization: <pre class=”example-preformatted”>firequalizer=gain_entry='entry(100,0); entry(400, -4); entry(1000, -6); entry(2000, 0)' </pre> higher delay with zero phase to compensate delay: <pre class=”example-preformatted”>firequalizer=delay=0.1:fixed=on:zero_phase=on </pre> lowpass on left channel, highpass on right channel: <pre class=”example-preformatted”>firequalizer=gain='if(eq(chid,1), gain_interpolate(f), if(eq(chid,2), gain_interpolate(1e6+f), 0))' :gain_entry='entry(1000, 0); entry(1001,-INF); entry(1e6+1000,0)':multi=on </pre>

 <h3 class=”section”>36.90 flanger# TOC</h3> Apply a flanging effect to the audio. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Set base delay in milliseconds. Range from 0 to 30. Default

value is 0. </dd> <dt><samp class="option">depth</samp></dt>
<dd> Set added sweep delay in milliseconds. Range from 0 to 10. Default
value is 2. </dd> <dt><samp class="option">regen</samp></dt>
<dd> Set percentage regeneration (delayed signal feedback). Range from -
95 to 95. Default value is 0. </dd> <dt><samp class="option">width</samp></dt>
<dd> Set percentage of delayed signal mixed with original. Range from 0 to
100. Default value is 71. </dd> <dt><samp class="option">speed</samp></dt>
<dd> Set sweeps per second (Hz). Range from 0.1 to 10. Default value
is 0.5. </dd> <dt><samp class="option">shape</samp></dt>
<dd> Set swept wave shape, can be <var class="var">triangular</var> or
<var class="var">sinusoidal</var>. Default value is <var class="var">sinusoidal</var>.
 </dd> <dt><samp class="option">phase</samp></dt> <dd> Set
swept wave percentage-shift for multi channel. Range from 0 to 100. Default
value is 25. </dd> <dt><samp class="option">interp</samp></dt>
<dd> Set delay-line interpolation, <var class="var">linear</var> or <var
class="var">quadratic</var>. Default is <var class="var">linear</var>.
</dd> </dl>

 <h3 class="section">36.91 haas<span class="pull-
right">#
TOC</h3>
Apply Haas effect to audio. Note that this makes most sense
to apply on mono signals. With this filter applied to mono signals it give
some directionality and stretches its stereo image. The filter
accepts the following options: <dl class="table"> <dt><samp
class="option">level_in</samp></dt> <dd> Set input level. By
default is <var class="var">1</var>, or 0dB </dd> <dt><samp
class="option">level_out</samp></dt> <dd> Set output level. By
default is <var class="var">1</var>, or 0dB. </dd> <dt><samp
class="option">side_gain</samp></dt> <dd> Set gain applied to side
part of signal. By default is <var class="var">1</var>. </dd>
<dt><samp class="option">middle_source</samp></dt> <dd> Set
kind of middle source. Can be one of the following: <dl class="table">
<dt>‘<samp class="samp">left</samp>’</dt> <dd> Pick
left channel. </dd> <dt>‘<samp class="samp">right</samp>’</dt>
<dd> Pick right channel. </dd> <dt>‘<samp class="samp">mid</samp>’</dt>
<dd> Pick middle part signal of stereo image. </dd> <dt>‘<samp
class="samp">side</samp>’</dt> <dd> Pick side part signal of
stereo image. </dd> </dl>

</dd> <dt><samp class="option">middle_phase</samp></dt> <dd> Change
middle phase. By default is disabled. </dd> <dt><samp
class="option">left_delay</samp></dt> <dd> Set left channel de-
lay. By default is <var class="var">2.05</var> milliseconds. </dd>
<dt><samp class="option">left_balance</samp></dt> <dd> Set left
channel balance. By default is <var class="var">-1</var>. </dd>
<dt><samp class="option">left_gain</samp></dt> <dd> Set left

channel gain. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>left_phase</samp></dt> <dd> Change left phase. By default is disabled. </dd> <dt><samp class=”option”>right_delay</samp></dt> <dd> Set right channel delay. By defaults is <var class=”var”>2.12</var> milliseconds. </dd> <dt><samp class=”option”>right_balance</samp></dt> <dd> Set right channel balance. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>right_gain</samp></dt> <dd> Set right channel gain. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>right_phase</samp></dt> <dd> Change right phase. By default is enabled. </dd> </dl>

 <h3 class=”section”>36.92 hdcd# TOC</h3>

Decodes High Definition Compatible Digital (HDCD) data. A 16-bit PCM stream with embedded HDCD codes is expanded into a 20-bit PCM stream. The filter supports the Peak Extend and Low-level Gain Adjustment features of HDCD, and detects the Transient Filter flag.   <pre class=”example-preformatted”>ffmpeg -i HDCD16.flac -af hdcd OUT24.flac </pre>

When using the filter with wav, note the default encoding for wav is 16-bit, so the resulting 20-bit stream will be truncated back to 16-bit. Use something like <code class=”command”>-acodec pcm_s24le</code> after the filter to get 24-bit PCM output. <pre class=”example-preformatted”>ffmpeg -i HDCD16.wav -af hdcd OUT16.wav ffmpeg -i HDCD16.wav -af hdcd -c:a pcm_s24le OUT24.wav </pre>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>disable_autoconvert</samp></dt> <dd> Disable any automatic format conversion or resampling in the filter graph. </dd> <dt><samp class=”option”>process_stereo</samp></dt> <dd> Process the stereo channels together. If target_gain does not match between channels, consider it invalid and use the last valid target_gain. </dd> <dt><samp class=”option”>cdt_ms</samp></dt> <dd> Set the code detect timer period in ms. </dd> <dt><samp class=”option”>force_pe</samp></dt> <dd> Always extend peaks above -3dBFS even if PE isn’t signaled. </dd> <dt><samp class=”option”>analyze_mode</samp></dt> <dd> Replace audio with a solid tone and adjust the amplitude to signal some specific aspect of the decoding process. The output file can be loaded in an audio editor alongside the original to aid analysis. <code class=”code”>analyze_mode=pe:force_pe=true</code> can be used to see all samples above the PE level. Modes are: <dl class=”table”> <dt>‘<samp class=”samp”>0, off</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class=”samp”>1, lle</samp>’</dt> <dd> Gain adjustment level at each sample </dd> <dt>‘<samp

class=”samp”>2, pe</samp>’</dt> <dd> Samples where peak extend occurs </dd> <dt>‘<samp class=”samp”>3, cdt</samp>’</dt> <dd> Samples where the code detect timer is active </dd> <dt>‘<samp class=”samp”>4, tgm</samp>’</dt> <dd> Samples where the target gain does not match between channels </dd> </dl> </dd> </dl>

 <h3 class=”section”>36.93 headphone# TOC</h3>

Apply head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones. The HRIRs are provided via additional streams, for each channel one stereo input stream is needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>map</samp></dt> <dd> Set mapping of input streams for convolution. The argument is a ’|’-separated list of channel names in order as they are given as additional stream inputs for filter. This also specify number of input streams. Number of input streams must be not less than number of channels in first stream plus one. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>lfe</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set size of frame in number of samples which will be processed at once. Default value is <var class=”var”>1024</var>. Allowed range is from 1024 to 96000. </dd> <dt><samp class=”option”>hrir</samp></dt> <dd> Set format of hrir stream. Default value is <var class=”var”>stereo</var>. Alternative value is <var class=”var”>multich</var>. If value is set to <var class=”var”>stereo</var>, number of additional streams should be greater or equal to number of input channels in first input stream. Also each additional stream should have stereo number of channels. If value is set to <var class=”var”>multich</var>, number of additional streams should be exactly one. Also number of input channels of additional stream should be equal or greater than twice number of channels of first input stream. </dd> </dl>

 <h4 class=”subsection”>36.93.1 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-73” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Full example using wav files as coefficients with amovie filters for 7.1 downmix, each amovie filter use stereo file with IR coefficients as input. The files give coefficients for each position of virtual loudspeaker: <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=azi_270_ele_0_DFC.wav[sr];amovie=azi_90_ele_0_DFC.wav[sl];amovie=azi_225_ele_0_D output.wav </pre>

Full example using wav files as coefficients with amovie filters for 7.1 downmix, but now in <var class=”var”>multich</var> <var class=”var”>hrir</var> format. <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=minp.wav[hrirs];[0:a][hrirs] output.wav </pre>

 <h3 class=”section”>36.94 highpass# TOC</h3>

Apply a high-pass filter with 3dB point frequency. The filter can be either single-pole, or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 3000. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp

class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt>
<dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt>
<dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt>
<dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set
precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt>
<dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>36.94.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>frequency, f</samp></dt> <dd> Change
highpass frequency. Syntax for the command is : "<var class=”var”>frequency</var>"
 </dd> <dt><samp class=”option”>width_type, t</samp></dt>
<dd> Change highpass width_type. Syntax for the command is
: "<var class=”var”>width_type</var>" </dd>
<dt><samp class=”option”>width, w</samp></dt> <dd> Change
highpass width. Syntax for the command is : "<var class=”var”>width</var>"
 </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change
highpass mix. Syntax for the command is : "<var class=”var”>mix</var>"
</dd> </dl>

 <h3 class=”section”>36.95 join#
TOC</h3>

Join multiple input streams into one multi-channel stream. It
accepts the following parameters: <dl class=”table”> <dt><samp
class=”option”>inputs</samp></dt> <dd> The number of input
streams. It defaults to 2. </dd> <dt><samp class=”option”>channel_layout</samp></dt>
<dd> The desired output channel layout. It defaults to stereo.
</dd> <dt><samp class=”option”>map</samp></dt> <dd> Map
channels from inputs to output. The argument is a ’|’-separated

list of mappings, each in the <code class=”code”><var class=”var”>input_idx</var>.<var class=”var”>in_channel</var>-<var class=”var”>out_channel</var></code> form. <var class=”var”>input_idx</var> is the 0-based index of the input stream. <var class=”var”>in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the specified input stream. <var class=”var”>out_channel</var> is the name of the output channel. </dd> </dl>

The filter will attempt to guess the mappings when they are not specified explicitly. It does so by first trying to find an unused matching input channel and if that fails it picks the first unused input channel. Join 3 inputs (with properly set channel layouts): <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex join=inputs=3 OUTPUT </pre>

Build a 5.1 output from 6 single-channel streams: <pre class=”example-preformatted”>ffmpeg -i fl -i fr -i fc -i sl -i sr -i lfe -filter_complex 'join=inputs=6:channel_layout=5.1:map=0.0-FL|1.0-FR|2.0-FC|3.0-SL|4.0-SR|5.0-LFE' out </pre>

 <h3 class=”section”>36.96 ladspa# TOC</h3>

Load a LADSPA (Linux Audio Developer’s Simple Plugin API) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-ladspa</code>. <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> Specifies the name of LADSPA plugin library to load. If the environment variable <code class=”env”>LADSPA_PATH</code> is defined, the LADSPA plugin is searched in each one of the directories specified by the colon separated list in <code class=”env”>LADSPA_PATH</code>, otherwise in the standard LADSPA paths, which are in this order: <samp class=”file”>HOME/.ladspa/lib/</samp>, <samp class=”file”>/usr/local/lib/ladspa/</samp>, <samp class=”file”>/usr/lib/ladspa/</samp>. </dd> <dt><samp class=”option”>plugin, p</samp></dt> <dd> Specifies the plugin within the library. Some libraries contain only one plugin, but others contain many of them. If this is not set filter will list all available plugins within the specified library. </dd> <dt><samp class=”option”>controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). Controls need to be defined using the following syntax: c0=<var class=”var”>value0</var>|c1=<var class=”var”>value1</var>|c2=<var class=”var”>value2</var>|..., where <var class=”var”>valuei</var> is the value set on the <var class=”var”>i</var>-th control. Alternatively they can be also defined using the following syntax: <var class=”var”>value0</var>|<var class=”var”>value1</var>|<var class=”var”>value2</var>|...,

where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> <dt><samp class="option">latency, l</samp></dt> <dd> Enable latency compensation, by default is disabled. Only used if plugin have inputs. </dd> </dl>

 <h4 class="subsection">36.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all available plugins within amp (LADSPA example plugin) library: <pre class="example-preformatted">ladspa=file=amp </pre>

List all available controls and their valid ranges for <code class="code">vcf_notch</code> plugin from <code class="code">VCF</code> library: <pre class="example-preformatted">ladspa=f=vcf:p=vcf_notch:c=help </pre>

Simulate low quality audio equipment using <code class="code">Computer Music Toolkit</code> (CMT) plugin library: <pre class="example-preformatted">ladspa=file=cmt:plugin=lofi:controls=c0=22|c1=12|c2=12 </pre>

Add reverberation to the audio using TAP-plugins (Tom’s Audio Processing plugins): <pre class="example-preformatted">ladspa=file=tap_reverb:tap_reverb </pre>

Generate white noise, with 0.2 amplitude: <pre class="example-preformatted">ladspa=file=cmt:noise_source_white:c=c0=.2 </pre>

Generate 20 bpm clicks using plugin <code class="code">C* Click - Metronome</code> from the <code class="code">C* Audio Plugin

Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=file=caps:Click:c=c1=20' </pre>

Apply <code class="code">C* Eq10X2 - Stereo 10-band equaliser</code> effect: <pre class="example-preformatted">ladspa=caps:Eq10X2:c=c0=-48|c9=-24|c3=12|c4=2 </pre>

Increase volume by 20dB using fast lookahead limiter from Steve Harris <code class="code">SWH Plugins</code> collection: <pre class="example-preformatted">ladspa=fast_lookahead_limiter_1913:fastLookahead
</pre>

Attenuate low frequencies using Multiband EQ from Steve Harris <code class="code">SWH Plugins</code> collection: <pre class="example-preformatted">ladspa=mbeq_1197:mbeq:-24|-24|-24|0|0|0|0|0|0|0|0|0|0|0|0 </pre>

Reduce stereo image using <code class="code">Narrower</code> from the <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:Narrower
</pre>

Another white noise, now using <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:White:.2 </pre>

Some fractal noise, using <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:Fractal:c=c1=1 </pre>

Dynamic volume normalization using <code class="code">VLevel</code> plugin: <pre class="example-preformatted">ladspa=vlevel-ladspa:vlevel_mono </pre>

 <h4 class="subsection">36.96.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">cN</samp></dt> <dd> Modify the <var class="var">N</var>-th control value. If the specified value is not valid, it is ignored and prior one is kept. </dd> </dl>

 <h3 class="section">36.97 loudnorm# TOC</h3>

EBU R128 loudness normalization. Includes both dynamic and linear normalization modes. Support for both single pass (livestreams, files) and double

pass (files) modes. This algorithm can target IL, LRA, and maximum true peak. In dynamic mode, to accurately detect true peaks, the audio stream will be upsampled to 192 kHz. Use the <code class=”code”>-ar</code> option or <code class=”code”>aresample</code> filter to explicitly set an output sample rate. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>I, i</samp></dt> <dd> Set integrated loudness target. Range is -70.0 - -5.0. Default value is -24.0. </dd> <dt><samp class=”option”>LRA, lra</samp></dt> <dd> Set loudness range target. Range is 1.0 - 50.0. Default value is 7.0. </dd> <dt><samp class=”option”>TP, tp</samp></dt> <dd> Set maximum true peak. Range is -9.0 - +0.0. Default value is -2.0. </dd> <dt><samp class=”option”>measured_I, measured_i</samp></dt> <dd> Measured IL of input file. Range is -99.0 - +0.0. </dd> <dt><samp class=”option”>measured_LRA, measured_lra</samp></dt> <dd> Measured LRA of input file. Range is 0.0 - 99.0. </dd> <dt><samp class=”option”>measured_TP, measured_tp</samp></dt> <dd> Measured true peak of input file. Range is -99.0 - +99.0. </dd> <dt><samp class=”option”>measured_thresh</samp></dt> <dd> Measured threshold of input file. Range is -99.0 - +0.0. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set offset gain. Gain is applied before the true-peak limiter. Range is -99.0 - +99.0. Default is +0.0. </dd> <dt><samp class=”option”>linear</samp></dt> <dd> Normalize by linearly scaling the source audio. <code class=”code”>measured_I</code>, <code class=”code”>measured_LRA</code>, <code class=”code”>measured_TP</code>, and <code class=”code”>measured_thresh</code> must all be specified. Target LRA shouldn’t be lower than source LRA and the change in integrated loudness shouldn’t result in a true peak which exceeds the target TP. If any of these conditions aren’t met, normalization mode will revert to <var class=”var”>dynamic</var>. Options are <code class=”code”>true</code> or <code class=”code”>false</code>. Default is <code class=”code”>true</code>. </dd> <dt><samp class=”option”>dual_mono</samp></dt> <dd> Treat mono input files as "dual-mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. Options are true or false. Default is false. </dd> <dt><samp class=”option”>print_format</samp></dt> <dd> Set print format for stats. Options are summary, json, or none. Default value is none. </dd> <dt><samp class=”option”>stats_file</samp></dt> <dd> Write stats to specified file. Format is controlled by <samp class=”option”>print_format</samp>, which must be set. Specify <code class=”code”>-</code> to write to standard output. Default is unset. </dd> </dl>

 <h3 class=”section”>36.98 lowpass<span

class=”pull-right”># TOC</h3>

Apply a low-pass filter with 3dB point frequency. The filter can be either single-pole or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 500. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero

values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.98.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Lowpass only LFE channel, it LFE is not present it does nothing: <pre class="example-preformatted">lowpass=c=LFE
</pre>

 <h4 class="subsection">36.98.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change lowpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change lowpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change lowpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change lowpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">36.99 lv2# TOC</h3>

Load a LV2 (LADSPA Version 2) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-lv2</code>. <dl class="table"> <dt><samp class="option">plugin, p</samp></dt> <dd> Specifies the plugin URI. You may need to escape ’:’. </dd> <dt><samp class="option">controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if

plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> </dl>

 <h4 class="subsection">36.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply bass enhancer plugin from Calf: <pre class="example-preformatted">lv2=p=http\\\\://calf.sourceforge.net/plugins/
</pre>

Apply vinyl plugin from Calf: <pre class="example-preformatted">lv2=p=http\\\\://calf.sourceforge.net/plugins/Vinyl:c=drone=0.2|aging=0.5
</pre>

Apply bit crusher plugin from ArtyFX: <pre class="example-preformatted">lv2=p=http\\\\://www.openavproductions.com/artyfx#bitta:c=crush=0.3
</pre>

 <h4 class="subsection">36.99.2 Commands# TOC</h4>

This filter supports all options that are exported by plugin as commands. <h3 class="section">36.100 mcompand# TOC</h3> Multiband Compress or expand the audio’s dynamic range. The input audio is divided into bands using 4th order Linkwitz-Riley IIRs. This is akin to the crossover of a loudspeaker, and results in flat frequency response when absent compander action. It accepts the following parameters: <dl class="table"> <dt><samp class="option">args</samp></dt> <dd> This option syntax is: attack,decay,[attack,decay..] soft-knee points crossover_frequency [delay [initial_volume [gain]]] | attack,decay ... For explanation of each item refer to compand filter documentation. </dd> </dl>

 <h3 class="section">36.101 pan<a class="anchor hidden-xs" href="#pan"

aria-hidden=”true”># TOC</h3>

Mix channels with specific gain levels. The filter accepts the output channel layout followed by a set of channels definitions. This filter is also designed to efficiently remap the channels of an audio stream. The filter accepts parameters of the form: "<var class=”var”>l</var>|<var class=”var”>outdef</var>|<var class=”var”>outdef</var>|..." <dl class=”table”> <dt><samp class=”option”>l</samp></dt> <dd> output channel layout or number of channels </dd> <dt><samp class=”option”>outdef</samp></dt> <dd> output channel specification, of the form: "<var class=”var”>out_name</var>=[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>[(+-)[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>...]" </dd> <dt><samp class=”option”>out_name</samp></dt> <dd> output channel to define, either a channel name (FL, FR, etc.) or a channel number (c0, c1, etc.) </dd> <dt><samp class=”option”>gain</samp></dt> <dd> multiplicative coefficient for the channel, 1 leaving the volume unchanged </dd> <dt><samp class=”option”>in_name</samp></dt> <dd> input channel to use, see out_name for details; it is not possible to mix named and numbered input channels </dd> </dl>

If the ‘=’ in a channel specification is replaced by ‘<’, then the gains for that specification will be renormalized so that the total is 1, thus avoiding clipping noise. <h4 class=”subsection”>36.101.1 Mixing examples# TOC</h4>

For example, if you want to down-mix from stereo to mono, but with a bigger factor for the left channel: <pre class=”example-preformatted”>pan=1c|c0=0.9*c0+0.1*c1 </pre>

A customized down-mix to stereo that works automatically for 3-, 4-, 5- and 7-channels surround: <pre class=”example-preformatted”>pan=stereo| FL < FL + 0.5*FC + 0.6*BL + 0.6*SL | FR < FR + 0.5*FC + 0.6*BR + 0.6*SR </pre>

Note that <code class=”command”>ffmpeg</code> integrates a default down-mix (and up-mix) system that should be preferred (see "-ac" option) unless you have very specific needs. <h4 class=”subsection”>36.101.2 Remapping examples# TOC</h4>

The channel remapping will be effective if, and only if: <ul

class=”itemize mark-bullet”> gain coefficients are zeroes or ones, only one input per channel output,

If all these conditions are satisfied, the filter will notify the user ("Pure channel mapping detected"), and use an optimized and lossless method to do the remapping. For example, if you have a 5.1 source and want a stereo audio stream by dropping the extra channels: <pre class=”example-preformatted”>pan="stereo| c0=FL | c1=FR" </pre>

Given the same source, you can also switch front left and front right channels and keep the input channel layout: <pre class=”example-preformatted”>pan="5.1| c0=c1 | c1=c0 | c2=c2 | c3=c3 | c4=c4 | c5=c5" </pre>

If the input is a stereo audio stream, you can mute the front left channel (and still keep the stereo channel layout) with: <pre class=”example-preformatted”>pan="stereo|c1=c1" </pre>

Still with a stereo audio stream input, you can copy the right channel in both front left and right: <pre class=”example-preformatted”>pan="stereo| c0=FR | c1=FR" </pre>

 <h3 class=”section”>36.102 replaygain# TOC</h3>

ReplayGain scanner filter. This filter takes an audio stream as an input and outputs it unchanged. At end of filtering it displays <code class=”code”>track_gain</code> and <code class=”code”>track_peak</code>. The filter accepts the following exported read-only options: <dl class=”table”> <dt><samp class=”option”>track_gain</samp></dt> <dd> Exported track gain in dB at end of stream. </dd> <dt><samp class=”option”>track_peak</samp></dt> <dd> Exported track peak at end of stream. </dd> </dl>

 <h3 class=”section”>36.103 resample# TOC</h3>

Convert the audio sample format, sample rate and channel layout. It is not meant to be used directly. <h3 class=”section”>36.104 rubberband# TOC</h3> Apply time-stretching and pitch-shifting with librubberband. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-librubberband</code>.

The filter accepts the following options: <dl class="table">
<dt><samp class="option">tempo</samp></dt> <dd> Set tempo
scale factor. </dd> <dt><samp class="option">pitch</samp></dt>
<dd> Set pitch scale factor. </dd> <dt><samp class="option">transients</samp></dt>
<dd> Set transients detector. Possible values are: <dl class="table">
<dt><var class="var">crisp</var></dt> <dt><var class="var">mixed</var></dt>
<dt><var class="var">smooth</var></dt> </dl>

</dd> <dt><samp class="option">detector</samp></dt> <dd> Set
detector. Possible values are: <dl class="table"> <dt><var
class="var">compound</var></dt> <dt><var class="var">percussive</var></dt>
<dt><var class="var">soft</var></dt> </dl>

</dd> <dt><samp class="option">phase</samp></dt> <dd> Set
phase. Possible values are: <dl class="table"> <dt><var class="var">laminar</var></dt>
<dt><var class="var">independent</var></dt> </dl>

</dd> <dt><samp class="option">window</samp></dt> <dd> Set
processing window size. Possible values are: <dl class="table">
<dt><var class="var">standard</var></dt> <dt><var class="var">short</var></dt>
<dt><var class="var">long</var></dt> </dl>

</dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set
smoothing. Possible values are: <dl class="table"> <dt><var
class="var">off</var></dt> <dt><var class="var">on</var></dt>
</dl>

</dd> <dt><samp class="option">formant</samp></dt> <dd> Enable
formant preservation when shift pitching. Possible values are: <dl
class="table"> <dt><var class="var">shifted</var></dt> <dt><var
class="var">preserved</var></dt> </dl>

</dd> <dt><samp class="option">pitchq</samp></dt> <dd> Set
pitch quality. Possible values are: <dl class="table"> <dt><var
class="var">quality</var></dt> <dt><var class="var">speed</var></dt>
<dt><var class="var">consistency</var></dt> </dl>

</dd> <dt><samp class="option">channels</samp></dt> <dd> Set
channels. Possible values are: <dl class="table"> <dt><var
class="var">apart</var></dt> <dt><var class="var">together</var></dt>
</dl> </dd> </dl>

 <h4 class="subsection">36.104.1 Com-
mands<a class="anchor hidden-xs" href="#Commands-
44" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Commands-44" aria-hidden="true">TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">tempo</samp></dt> <dd> Change
filter tempo scale factor. Syntax for the command is : "<var

class=”var”>tempo</var>" </dd> <dt><samp class=”option”>pitch</samp></dt> <dd> Change filter pitch scale factor. Syntax for the command is : "<var class=”var”>pitch</var>" </dd> </dl>

 <h3 class=”section”>36.105 sidechaincompress# TOC</h3>

This filter acts like normal compressor but has the ability to compress detected signal using second input signal. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filtered signal then can be filtered with other filters in later stages of processing. See pan and amerge filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> If a signal of second stream raises above this level it will affect the gain reduction of first stream. By default is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class=”option”>ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. 1:2 means that if the level raised 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class=”option”>makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class=”option”>knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class=”option”>link</samp></dt> <dd> Choose if the <code class=”code”>average</code> level between all channels of side-chain stream or the louder(<code class=”code”>maximum</code>) channel of side-chain stream affects the reduction. Default is <code class=”code”>average</code>. </dd> <dt><samp class=”option”>detection</samp></dt> <dd> Should the exact signal be taken in case of <code class=”code”>peak</code> or an RMS one in case of <code class=”code”>rms</code>. Default

is <code class=”code”>rms</code> which is mainly smoother. </dd> <dt><samp class=”option”>level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class=”subsection”>36.105.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>36.105.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Full ffmpeg example taking 2 audio inputs, 1st input to be compressed depending on the signal of 2nd input and later compressed signal to be merged with 2nd input: <pre class=”example-preformatted”>ffmpeg -i main.flac -i sidechain.flac -filter_complex "[1:a]asplit=2[sc][mix];[0:a][sc]sidechaincompress[compr];[compr][mix]amerge" </pre>

 <h3 class=”section”>36.106 sidechaingate# TOC</h3>

A sidechain gate acts like a normal (wideband) gate but has the ability to filter the detected signal before sending it to the gain reduction stage. Normally a gate uses the full range signal to detect a level above the threshold. For example: If you cut all lower frequencies from your sidechain signal the gate will decrease the volume of your track only if not enough highs appear. With this technique you are able to reduce the resonation of a natural drum or remove "rumbling" of muted strokes from a heavily distorted guitar. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode of operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. If set to <code class=”code”>upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class=”code”>downward</code> lower parts of signal will be

reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is rms. Can be peak or rms. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is average. Can be average or maximum. </dd> <dt><samp class="option">level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is from 0.015625 to 64. </dd> </dl>

 <h4 class="subsection">36.106.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">36.107 silencedetect# TOC</h3>

Detect silence in an audio stream. This filter logs a message when it detects that the input audio volume is less or equal to a noise tolerance value for a duration greater or equal to the minimum detected noise duration. The printed times and duration are expressed in seconds. The <code class="code">lavfi.silence_start</code> or <code class="code">lavfi.silence_start.X</code> metadata key is

set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the silence. The <code class="code">lavfi.silence_duration</code> or <code class="code">lavfi.silence_duration.X</code> and <code class="code">lavfi.silence_end</code> or <code class="code">lavfi.silence_end.X</code> metadata keys are set on the first frame after the silence. If <samp class="option">mono</samp> is enabled, and each channel is evaluated separately, the <code class="code">.X</code> suffixed keys are used, and <code class="code">X</code> corresponds to the channel number. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or amplitude ratio. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set silence duration until notification (default is 2 seconds). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. </dd> <dt><samp class="option">mono, m</samp></dt> <dd> Process each channel separately, instead of combined. By default is disabled. </dd> </dl>

 <h4 class="subsection">36.107.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect 5 seconds of silence with -50dB noise tolerance: <pre class="example-preformatted">silencedetect=n=-50dB:d=5 </pre>

Complete example with <code class="command">ffmpeg</code> to detect silence with 0.0001 noise tolerance in <samp class="file">silence.mp3</samp>: <pre class="example-preformatted">ffmpeg -i silence.mp3 -af silencedetect=noise=0.0001 -f null - </pre>

 <h3 class="section">36.108 silenceremove# TOC</h3>

Remove silence from the beginning, middle or end of the audio.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">start_periods</samp></dt>

<dd> This value is used to indicate if audio should be trimmed at beginning of

the audio. A value of zero indicates no silence should be trimmed from the

beginning. When specifying a non-zero value, it trims audio up until it

finds non-silence. Normally, when trimming silence from beginning of audio

the <var class="var">start_periods</var> will be <code class="code">1</code> but it can be increased to higher

values to trim all audio up to specific count of non-silence periods.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_duration</samp></dt>

<dd> Specify the amount of time that non-silence must be detected before it stops

trimming audio. By increasing the duration, bursts of noises can be treated

as silence and trimmed off. Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_threshold</samp></dt>

<dd> This indicates what sample value should be treated as silence. For digital

audio, a value of <code class="code">0</code> may be fine but for audio recorded from analog,

you may wish to increase the value to account for background noise.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_silence</samp></dt>

<dd> Specify max duration of silence at beginning that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class="option">start_mode</samp></dt>

<dd> Specify mode of detection of silence end at start of multi-channel audio.

Can be <var class="var">any</var> or <var class="var">all</var>. Default is <var class="var">any</var>.

With <var class="var">any</var>, any sample from any channel that is detected as non-silence

will trigger end of silence trimming at start of audio stream.

With <var class="var">all</var>, only if every sample from every channel is detected as non-silence

will trigger end of silence trimming at start of audio stream, limited usage.

</dd>

<dt><samp class="option">stop_periods</samp></dt>

<dd> Set the count for trimming silence from the end of audio. When specifying a

positive value, it trims audio after it finds specified silence period.

To remove silence from the middle of a file, specify a <var class="var">stop_periods</var>

that is negative. This value is then treated as a positive value and is

used to indicate the effect should restart processing as specified by

<var class="var">stop_periods</var>, making it suitable for removing periods of silence

in the middle of the audio.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">stop_duration</samp></dt>

<dd> Specify a duration of silence that must exist before audio is not copied any

more. By specifying a higher duration, silence that is wanted can be left in

the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_threshold</samp></dt>

<dd> This is the same as <samp class=”option”>start_threshold</samp> but for trimming silence from

the end of audio.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_silence</samp></dt>

<dd> Specify max duration of silence at end that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>stop_mode</samp></dt>

<dd> Specify mode of detection of silence start after start of multi-channel audio.

Can be <var class=”var”>any</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>.

With <var class=”var”>any</var>, any sample from any channel that is detected as silence

will trigger start of silence trimming after start of audio stream, limited usage.

With <var class=”var”>all</var>, only if every sample from every channel is detected as silence

will trigger start of silence trimming after start of audio stream.

</dd>

<dt><samp class=”option”>detection</samp></dt>

<dd> Set how is silence detected.

<dl class="table">

<dt><samp class="option">avg</samp></dt>

<dd> Mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">rms</samp></dt>

<dd> Root squared mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">peak</samp></dt>

<dd> Maximum of absolute values of samples in moving window.

</dd>

<dt><samp class="option">median</samp></dt>

<dd> Median of absolute values of samples in moving window.

</dd>

<dt><samp class="option">ptp</samp></dt>

<dd> Absolute of max peak to min peak difference of samples in moving window.

</dd>

<dt><samp class="option">dev</samp></dt>

<dd> Standard deviation of values of samples in moving window.

</dd>

</dl>

Default value is <code class="code">rms</code>.

</dd>

<dt><samp class="option">window</samp></dt>

<dd> Set duration in number of seconds used to calculate size of window in number

of samples for detecting silence. Using <code class="code">0</code> will effectively disable

any windowing and use only single sample per channel for silence detection.

In that case it may be needed to also set <samp class="option">start_silence</samp> and/or

<samp class="option">stop_silence</samp> to nonzero values with also <samp class="option">start_duration</samp> and/or

<samp class="option">stop_duration</samp> to nonzero values.

Default value is <code class="code">0.02</code>. Allowed range is from <code class="code">0</code> to <code class="code">10</code>.

</dd>

<dt><samp class="option">timestamp</samp></dt>

<dd> Set processing mode of every audio frame output timestamp.

<dl class="table">

<dt><samp class="option">write</samp></dt>

<dd> Full timestamps rewrite, keep only the start time for the first output frame.

</dd>

<dt><samp class="option">copy</samp></dt>

<dd> Non-dropped frames are left with same timestamp as input audio frame.

</dd>

</dl>

Defaults value is <code class="code">write</code>.

</dd>

</dl>

 <h4 class="subsection">36.108.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> The following example shows how this filter can be used to start a recording that does not contain the delay at the start which usually occurs between pressing the record button and the start of the performance: <pre class="example-preformatted">silenceremove=start_periods=1:start_duration=5:start_threshold=0.02
</pre>

Trim all silence encountered from beginning to end where there is more than 1 second of silence in audio: <pre class="example-preformatted">silenceremove=stop_periods=-1:stop_duration=1:stop_threshold=-90dB </pre>

Trim all digital silence samples, using peak detection, from beginning to end where there is more than 0 samples of digital silence in audio and digital silence is detected in all channels at same positions in stream: <pre class=”example-preformatted”>silenceremove=window=0:detection=peak:stop_mode=all:start_mode=all:stop_periods=-1:stop_threshold=0 </pre>

Trim every 2nd encountered silence period from beginning to end where there is more than 1 second of silence per silence period in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB </pre>

Similar as above, but keep maximum of 0.5 seconds of silence from each trimmed period: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5 </pre>

Similar as above, but keep maximum of 1.5 seconds of silence from start of audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5:start_periods=1:start_duration=1:start_silence=1.5:stop_threshold=-90dB </pre>

 <h4 class=”subsection”>36.108.2 Commands# TOC</h4>

This filter supports some above options as commands. <h3 class=”section”>36.109 sofalizer# TOC</h3>

SOFAlizer uses head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones (audio formats up to 9 channels supported). The HRTFs are stored in SOFA files (see http://www.sofacoustics.org/ for a database). SOFAlizer is developed at the Acoustics Research Institute (ARI) of the Austrian Academy of Sciences. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libmysofa</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sofa</samp></dt> <dd> Set the SOFA file used for rendering. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set rotation of virtual loudspeakers in deg. Default is 0. </dd> <dt><samp class=”option”>elevation</samp></dt> <dd> Set eleva-

tion of virtual speakers in deg. Default is 0. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> Set distance in meters between loudspeakers and the listener with near-field HRTFs. Default is 1. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>speakers</samp></dt> <dd> Set custom positions of virtual loudspeakers. Syntax for this option is: <CH> <AZIM> <ELEV>[|<CH> <AZIM> <ELEV>|...]. Each virtual loudspeaker is described with short channel name following with azimuth and elevation in degrees. Each virtual loudspeaker description is separated by ’|’. For example to override front left and front right channel positions use: ’speakers=FL 45 15|FR 345 15’. Descriptions with unrecognised channel names are ignored. </dd> <dt><samp class=”option”>lfegain</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>framesize</samp></dt> <dd> Set custom frame size in number of samples. Default is 1024. Allowed range is from 1024 to 96000. Only used if option ‘<samp class=”samp”>type</samp>’ is set to <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Should all IRs be normalized upon importing SOFA file. By default is enabled. </dd> <dt><samp class=”option”>interpolate</samp></dt> <dd> Should nearest IRs be interpolated with neighbor IRs if exact position does not match. By default is disabled. </dd> <dt><samp class=”option”>minphase</samp></dt> <dd> Minphase all IRs upon loading of SOFA file. By default is disabled. </dd> <dt><samp class=”option”>anglestep</samp></dt> <dd> Set neighbor search angle step. Only used if option <var class=”var”>interpolate</var> is enabled. </dd> <dt><samp class=”option”>radstep</samp></dt> <dd> Set neighbor search radius step. Only used if option <var class=”var”>interpolate</var> is enabled. </dd> </dl>

 <h4 class=”subsection”>36.109.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Using ClubFritz6 sofa file: <pre class=”example-preformatted”>sofalizer=sofa=/path/to/ClubFritz6.sofa:type=freq:r
</pre>

Using ClubFritz12 sofa file and bigger radius with small rotation: <pre class=”example-preformatted”>sofalizer=sofa=/path/to/ClubFritz12.sofa:type=
</pre>

Similar as above but with custom speaker positions for front left, front right, back left and back right and also with custom gain: <pre class=”example-preformatted”>"sofalizer=sofa=/path/to/ClubFritz6.sofa:type=45|FR 315|BL 135|BR 225:gain=28" </pre>

 <h3 class=”section”>36.110 speechnorm# TOC</h3> Speech Normalizer. This filter expands or compresses each half-cycle of audio samples (local set of samples all above or all below zero and between two nearest zero crossings) depending on threshold value, so audio reaches target peak value under conditions controlled by below options. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>peak, p</samp></dt> <dd> Set the expansion target peak value. This specifies the highest allowed absolute amplitude level for the normalized audio input. Default value is 0.95. Allowed range is from 0.0 to 1.0. </dd> <dt><samp class=”option”>expansion, e</samp></dt> <dd> Set the maximum expansion factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples expansion. The maximum expansion would be such that local peak value reaches target peak value but never to surpass it and that ratio between new and previous peak value does not surpass this option value. </dd> <dt><samp class=”option”>compression, c</samp></dt> <dd> Set the maximum compression factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples compression. This option is used only if <samp class=”option”>threshold</samp> option is set to value greater than 0.0, then in such cases when local peak is lower or same as value set by <samp class=”option”>threshold</samp> all samples belonging to that peak’s half-cycle will be compressed by current compression factor. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set the threshold value. Default value is 0.0. Allowed range is from 0.0 to 1.0. This option specifies which half-cycles of samples will be compressed and which will be expanded. Any half-cycle samples with their local peak value below or same as this option value will be compressed by current compression factor, otherwise, if greater than threshold value they will be expanded with expansion factor so that it could reach peak target value but never surpass it. </dd> <dt><samp class=”option”>raise, r</samp></dt> <dd> Set the expansion raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast expansion factor is raised per each new half-cycle until it reaches <samp class=”option”>expansion</samp> value. Setting this options too high may lead to distortions. </dd> <dt><samp class=”option”>fall, f</samp></dt> <dd> Set the compression raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast compression factor is raised per each

new half-cycle until it reaches <samp class="option">compression</samp> value. </dd> <dt><samp class="option">channels, h</samp></dt> <dd> Specify which channels to filter, by default all available channels are filtered. </dd> <dt><samp class="option">invert, i</samp></dt> <dd> Enable inverted filtering, by default is disabled. This inverts interpretation of <samp class="option">threshold</samp> option. When enabled any half-cycle of samples with their local peak value below or same as <samp class="option">threshold</samp> option will be expanded otherwise it will be compressed. </dd> <dt><samp class="option">link, l</samp></dt> <dd> Link channels when calculating gain applied to each filtered channel sample, by default is disabled. When disabled each filtered channel gain calculation is independent, otherwise when this option is enabled the minimum of all possible gains for each filtered channel is used. </dd> <dt><samp class="option">rms, m</samp></dt> <dd> Set the expansion target RMS value. This specifies the highest allowed RMS level for the normalized audio input. Default value is 0.0, thus disabled. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">36.110.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">36.110.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Weak and slow amplification: <pre class="example-preformatted">speechnorm=e=3:r=0.00001:l=1 </pre>

Moderate and slow amplification: <pre class="example-preformatted">speechnorm=e=6.25:r=0.00001:l=1 </pre>

Strong and fast amplification: <pre class="example-preformatted">speechnorm=e=12.5:r=0.0001:l=1 </pre>

Very strong and fast amplification: <pre class="example-preformatted">speechnorm=e=25:r=0.0001:l=1 </pre>

Extreme and fast amplification: <pre class="example-preformatted">speechnorm=e=50:r=0.0001:l=1 </pre>

 <h3 class="section">36.111 stereotools<a class="anchor hidden-xs" href="#stereotools" aria-

hidden=”true”># TOC</h3>

This filter has some handy utilities to manage stereo signals, for converting M/S stereo recordings to L/R signal while having control over the parameters or spreading the stereo image of master track. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>balance_in</samp></dt> <dd> Set input balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>balance_out</samp></dt> <dd> Set output balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>softclip</samp></dt> <dd> Enable softclipping. Results in analog distortion instead of harsh digital 0dB clipping. Disabled by default. </dd> <dt><samp class=”option”>mutel</samp></dt> <dd> Mute the left channel. Disabled by default. </dd> <dt><samp class=”option”>muter</samp></dt> <dd> Mute the right channel. Disabled by default. </dd> <dt><samp class=”option”>phasel</samp></dt> <dd> Change the phase of the left channel. Disabled by default. </dd> <dt><samp class=”option”>phaser</samp></dt> <dd> Change the phase of the right channel. Disabled by default. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set stereo mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lr>lr</samp>’</dt> <dd> Left/Right to Left/Right, this is default. </dd> <dt>‘<samp class=”samp”>lr>ms</samp>’</dt> <dd> Left/Right to Mid/Side. </dd> <dt>‘<samp class=”samp”>ms>lr</samp>’</dt> <dd> Mid/Side to Left/Right. </dd> <dt>‘<samp class=”samp”>lr>ll</samp>’</dt> <dd> Left/Right to Left/Left. </dd> <dt>‘<samp class=”samp”>lr>rr</samp>’</dt> <dd> Left/Right to Right/Right. </dd> <dt>‘<samp class=”samp”>lr>l+r</samp>’</dt> <dd> Left/Right to Left + Right. </dd> <dt>‘<samp class=”samp”>lr>rl</samp>’</dt> <dd> Left/Right to Right/Left. </dd> <dt>‘<samp class=”samp”>ms>ll</samp>’</dt> <dd> Mid/Side to Left/Left. </dd> <dt>‘<samp class=”samp”>ms>rr</samp>’</dt> <dd> Mid/Side to Right/Right. </dd> <dt>‘<samp class=”samp”>ms>rl</samp>’</dt> <dd> Mid/Side to Right/Left. </dd> <dt>‘<samp class=”samp”>lr>l-r</samp>’</dt> <dd> Left/Right to Left - Right. </dd> </dl>

</dd> <dt><samp class=”option”>slev</samp></dt> <dd> Set level

of side signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">sbal</samp></dt> <dd> Set balance of side signal. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">mlev</samp></dt> <dd> Set level of the middle signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mpan</samp></dt> <dd> Set middle signal pan. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">base</samp></dt> <dd> Set stereo base between mono and inversed channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay in milliseconds how much to delay left from right channel and vice versa. Default is 0. Allowed range is from -20 to 20. </dd> <dt><samp class="option">sclevel</samp></dt> <dd> Set S/C level. Default is 1. Allowed range is from 1 to 100. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set the stereo phase in degrees. Default is 0. Allowed range is from 0 to 360. </dd> <dt><samp class="option">bmode_in, bmode_out</samp></dt> <dd> Set balance mode for balance_in/balance_out option. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">balance</samp>’</dt> <dd> Classic balance mode. Attenuate one channel at time. Gain is raised up to 1. </dd> <dt>‘<samp class="samp">amplitude</samp>’</dt> <dd> Similar as classic mode above but gain is raised up to 2. </dd> <dt>‘<samp class="samp">power</samp>’</dt> <dd> Equal power distribution, from -6dB to +6dB range. </dd> </dl> </dd> </dl>

 <h4 class="subsection">36.111.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">36.111.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply karaoke like effect: <pre class="example-preformatted">stereotools=mlev=0.015625 </pre>

Convert M/S signal to L/R: <pre class="example-preformatted">"stereotools=mode=ms>lr" </pre>

 <h3 class="section">36.112 stereowiden<a class="anchor hidden-xs" href="#stereowiden" aria-

hidden=”true”># TOC</h3>

This filter enhance the stereo effect by suppressing signal common to both channels and by delaying the signal of left into right and vice versa, thereby widening the stereo effect. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Time in milliseconds of the delay of left signal into right and vice versa. Default is 20 milliseconds. </dd> <dt><samp class=”option”>feedback</samp></dt> <dd> Amount of gain in delayed signal into right and vice versa. Gives a delay effect of left signal in right output and vice versa which gives widening effect. Default is 0.3. </dd> <dt><samp class=”option”>crossfeed</samp></dt> <dd> Cross feed of left into right with inverted phase. This helps in suppressing the mono. If the value is 1 it will cancel all the signal common to both channels. Default is 0.3. </dd> <dt><samp class=”option”>drymix</samp></dt> <dd> Set level of input signal of original channel. Default is 0.8. </dd> </dl>

 <h4 class=”subsection”>36.112.1 Commands# TOC</h4>

This filter supports the all above options except <code class=”code”>delay</code> as commands. <h3 class=”section”>36.113 superequalizer# TOC</h3> Apply 18 band equalizer. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>1b</samp></dt> <dd> Set 65Hz band gain. </dd> <dt><samp class=”option”>2b</samp></dt> <dd> Set 92Hz band gain. </dd> <dt><samp class=”option”>3b</samp></dt> <dd> Set 131Hz band gain. </dd> <dt><samp class=”option”>4b</samp></dt> <dd> Set 185Hz band gain. </dd> <dt><samp class=”option”>5b</samp></dt> <dd> Set 262Hz band gain. </dd> <dt><samp class=”option”>6b</samp></dt> <dd> Set 370Hz band gain. </dd> <dt><samp class=”option”>7b</samp></dt> <dd> Set 523Hz band gain. </dd> <dt><samp class=”option”>8b</samp></dt> <dd> Set 740Hz band gain. </dd> <dt><samp class=”option”>9b</samp></dt> <dd> Set 1047Hz band gain. </dd> <dt><samp class=”option”>10b</samp></dt> <dd> Set 1480Hz band gain. </dd> <dt><samp class=”option”>11b</samp></dt> <dd> Set 2093Hz band gain. </dd> <dt><samp class=”option”>12b</samp></dt> <dd> Set 2960Hz band gain. </dd> <dt><samp class=”option”>13b</samp></dt> <dd> Set 4186Hz band gain. </dd> <dt><samp class=”option”>14b</samp></dt> <dd> Set 5920Hz band gain. </dd> <dt><samp class=”option”>15b</samp></dt> <dd> Set 8372Hz band gain. </dd> <dt><samp class=”option”>16b</samp></dt>

<dd> Set 11840Hz band gain. </dd> <dt><samp class="option">17b</samp></dt>
<dd> Set 16744Hz band gain. </dd> <dt><samp class="option">18b</samp></dt>
<dd> Set 20000Hz band gain. </dd> </dl>

<h3 class="section">36.114 surround# TOC</h3>

Apply audio surround upmix filter.

This filter allows to produce multichannel output from audio stream.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">chl_out</samp></dt>

<dd> Set output channel layout. By default, this is <var class="var">5.1</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">chl_in</samp></dt>

<dd> Set input channel layout. By default, this is <var class="var">stereo</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">level_in</samp></dt>

<dd> Set input volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">level_out</samp></dt>

<dd> Set output volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe</samp></dt>

<dd> Enable LFE channel output if output channel layout has it. By default, this is enabled.

</dd>

<dt><samp class="option">lfe_low</samp></dt>

<dd> Set LFE low cut off frequency. By default, this is <var class="var">128</var> Hz.

</dd>

<dt><samp class="option">lfe_high</samp></dt>

<dd> Set LFE high cut off frequency. By default, this is <var class="var">256</var> Hz.

</dd>

<dt><samp class="option">lfe_mode</samp></dt>

<dd> Set LFE mode, can be <var class="var">add</var> or <var class="var">sub</var>. Default is <var class="var">add</var>.

In <var class="var">add</var> mode, LFE channel is created from input audio and added to output.

In <var class="var">sub</var> mode, LFE channel is created from input audio and added to output but

also all non-LFE output channels are subtracted with output LFE channel.

</dd>

<dt><samp class="option">smooth</samp></dt>

<dd> Set temporal smoothness strength, used to gradually change factors when transforming

stereo sound in time. Allowed range is from <var class="var">0.0</var> to <var class="var">1.0</var>.

Useful to improve output quality with <var class="var">focus</var> option values greater than <var class="var">0.0</var>.

Default is <var class="var">0.0</var>. Only values inside this range and without edges are effective.

</dd>

<dt><samp class="option">angle</samp></dt>

<dd> Set angle of stereo surround transform, Allowed range is from <var class="var">0</var> to <var class="var">360</var>.

Default is <var class="var">90</var>.

</dd>

<dt><samp class="option">focus</samp></dt>

<dd> Set focus of stereo surround transform, Allowed range is from <var class="var">-1</var> to <var class="var">1</var>.

Default is <var class="var">0</var>.

</dd>

<dt><samp class="option">fc_in</samp></dt>

<dd> Set front center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fc_out</samp></dt>

<dd> Set front center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_in</samp></dt>

<dd> Set front left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_out</samp></dt>

<dd> Set front left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_in</samp></dt>

<dd> Set front right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_out</samp></dt>

<dd> Set front right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_in</samp></dt>

<dd> Set side left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_out</samp></dt>

<dd> Set side left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_in</samp></dt>

<dd> Set side right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_out</samp></dt>

<dd> Set side right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_in</samp></dt>

<dd> Set back left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_out</samp></dt>

<dd> Set back left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_in</samp></dt>

<dd> Set back right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_out</samp></dt>

<dd> Set back right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_in</samp></dt>

<dd> Set back center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_out</samp></dt>

<dd> Set back center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_in</samp></dt>

<dd> Set LFE input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_out</samp></dt>

<dd> Set LFE output volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">allx</samp></dt>
<dd> Set spread usage of stereo image across X axis for all channels.
Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.
By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>
<dt><samp class="option">ally</samp></dt>
<dd> Set spread usage of stereo image across Y axis for all channels.
Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.
By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>
<dt><samp class="option">fcx, flx, frx, blx, brx, slx, srx, bcx</samp></dt>
<dd> Set spread usage of stereo image across X axis for each channel.
Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.
By default this value is <var class="var">0.5</var>.

</dd>
<dt><samp class="option">fcy, fly, fry, bly, bry, sly, sry, bcy</samp></dt>
<dd> Set spread usage of stereo image across Y axis for each channel.
Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.
By default this value is <var class="var">0.5</var>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from <var class="var">1024</var>
to <var class="var">65536</var>. Default size is <var class="var">4096</var>.

</dd>

<dt><samp class="option">win_func</samp></dt>
<dd> Set window function.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt>
<dt>‘<samp class="samp">bartlett</samp>’</dt>
<dt>‘<samp class="samp">hann, hanning</samp>’</dt>
<dt>‘<samp class="samp">hamming</samp>’</dt>
<dt>‘<samp class="samp">blackman</samp>’</dt>
<dt>‘<samp class="samp">welch</samp>’</dt>
<dt>‘<samp class="samp">flattop</samp>’</dt>
<dt>‘<samp class="samp">bharris</samp>’</dt>
<dt>‘<samp class="samp">bnuttall</samp>’</dt>
<dt>‘<samp class="samp">bhann</samp>’</dt>
<dt>‘<samp class="samp">sine</samp>’</dt>
<dt>‘<samp class="samp">nuttall</samp>’</dt>
<dt>‘<samp class="samp">lanczos</samp>’</dt>
<dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">tukey</samp>’</dt>
<dt>‘<samp class="samp">dolph</samp>’</dt>
<dt>‘<samp class="samp">cauchy</samp>’</dt>
<dt>‘<samp class="samp">parzen</samp>’</dt>
<dt>‘<samp class="samp">poisson</samp>’</dt>
<dt>‘<samp class="samp">bohman</samp>’</dt>
<dt>‘<samp class="samp">kaiser</samp>’</dt>
</dl>
Default is <code class="code">hann</code>.

</dd>
<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class=”code”>0.5</code>.

</dd>

</dl>

 <h3 class=”section”>36.115 tiltshelf# TOC</h3>

Boost or cut the lower frequencies and cut or boost higher frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class=”code”>3000</code> Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">36.115.1 Commands# TOC</h4>

This filter supports some options as commands. <h3 class="section">36.116 treble, highshelf# TOC</h3>

Boost or cut treble (upper) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at whichever is the lower of ~22 kHz and the Nyquist frequency. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">3000</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp

class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>36.116.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Change treble frequency. Syntax for the command is : "<var class=”var”>frequency</var>" </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Change treble width_type. Syntax for the command is : "<var class=”var”>width_type</var>" </dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Change treble width. Syntax for the command is : "<var class=”var”>width</var>" </dd> <dt><samp class=”option”>gain, g</samp></dt> <dd> Change treble gain. Syntax for the command is : "<var class=”var”>gain</var>" </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change treble mix. Syntax for the command is : "<var class=”var”>mix</var>"

</dd> </dl>

 <h3 class="section">36.117 tremolo# TOC</h3>

Sinusoidal amplitude modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Modulation frequencies in the subharmonic range (20 Hz or lower) will result in a tremolo effect. This filter may also be used as a ring modulator by specifying a modulation frequency higher than 20 Hz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">36.118 vibrato# TOC</h3>

Sinusoidal phase modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">36.119 virtualbass# TOC</h3>

Apply audio Virtual Bass filter. This filter accepts stereo input and produce stereo with LFE (2.1) channels output. The newly produced LFE channel have enhanced virtual bass originally obtained from both stereo channels. This filter outputs front left and front right channels unchanged as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set the virtual bass cutoff frequency. Default value is 250 Hz. Allowed range is from 100 to 500 Hz. </dd> <dt><samp class="option">strength</samp></dt> <dd> Set the virtual bass strength. Allowed range is from 0.5 to 3. Default value is 3. </dd> </dl>

 <h3 class="section">36.120 volume# <a class="anchor hidden-xs"href="#toc-volume"

aria-hidden=”true”>TOC</h3>

Adjust the input audio volume. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>volume</samp></dt> <dd> Set audio volume expression. Output values are clipped to the maximum value. The output audio volume is given by the relation: <pre class=”example-preformatted”><var class=”var”>output_volume</var> = <var class=”var”>volume</var> * <var class=”var”>input_volume</var> </pre>

The default value for <var class=”var”>volume</var> is "1.0". </dd> <dt><samp class=”option”>precision</samp></dt> <dd> This parameter represents the mathematical precision. It determines which input sample formats will be allowed, which affects the precision of the volume scaling. <dl class=”table”> <dt><samp class=”option”>fixed</samp></dt> <dd> 8-bit fixed-point; this limits input sample format to U8, S16, and S32. </dd> <dt><samp class=”option”>float</samp></dt> <dd> 32-bit floating-point; this limits input sample format to FLT. (default) </dd> <dt><samp class=”option”>double</samp></dt> <dd> 64-bit floating-point; this limits input sample format to DBL. </dd> </dl>

</dd> <dt><samp class=”option”>replaygain</samp></dt> <dd> Choose the behaviour on encountering ReplayGain side data in input frames. <dl class=”table”> <dt><samp class=”option”>drop</samp></dt> <dd> Remove ReplayGain side data, ignoring its contents (the default). </dd> <dt><samp class=”option”>ignore</samp></dt> <dd> Ignore ReplayGain side data, but leave it in the frame. </dd> <dt><samp class=”option”>track</samp></dt> <dd> Prefer the track gain, if present. </dd> <dt><samp class=”option”>album</samp></dt> <dd> Prefer the album gain, if present. </dd> </dl>

</dd> <dt><samp class=”option”>replaygain_preamp</samp></dt> <dd> Pre-amplification gain in dB to apply to the selected replaygain gain. Default value for <var class=”var”>replaygain_preamp</var> is 0.0. </dd> <dt><samp class=”option”>replaygain_noclip</samp></dt> <dd> Prevent clipping by limiting the gain applied. Default value for <var class=”var”>replaygain_noclip</var> is 1. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the volume expression is evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>once</samp>’</dt> <dd> only evaluate expression once during the filter initialization, or when the ‘<samp class=”samp”>volume</samp>’ command is sent </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expression for each incoming frame </dd> </dl>

Default value is ‘<samp class=”samp”>once</samp>’. </dd> </dl>

The volume expression can contain the following parameters. <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> frame number (starting at zero) </dd> <dt><samp class="option">nb_channels</samp></dt> <dd> number of channels </dd> <dt><samp class="option">nb_consumed_samples</samp></dt> <dd> number of samples consumed by the filter </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> number of samples in the current frame </dd> <dt><samp class="option">pos</samp></dt> <dd> original frame position in the file; deprecated, do not use </dd> <dt><samp class="option">pts</samp></dt> <dd> frame PTS </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> sample rate </dd> <dt><samp class="option">startpts</samp></dt> <dd> PTS at start of stream </dd> <dt><samp class="option">startt</samp></dt> <dd> time at start of stream </dd> <dt><samp class="option">t</samp></dt> <dd> frame time </dd> <dt><samp class="option">tb</samp></dt> <dd> timestamp timebase </dd> <dt><samp class="option">volume</samp></dt> <dd> last set volume value </dd> </dl>

Note that when <samp class="option">eval</samp> is set to ‘<samp class="samp">once</samp>’ only the <var class="var">sample_rate</var> and <var class="var">tb</var> variables are available, all other variables will evaluate to NAN. <h4 class="subsection">36.120.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">volume</samp></dt> <dd> Modify the volume expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class="subsection">36.120.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Halve the input audio volume: <pre class="example-preformatted">volume=volume=0.5 volume=volume=1/2 volume=volume=-6.0206dB </pre>

In all the above example the named key for <samp class="option">volume</samp> can be omitted, for example like in: <pre class="example-preformatted">volume=0.5 </pre>

Increase input audio power by 6 decibels using fixed-point precision: <pre class="example-preformatted">volume=volume=6dB:precision=fixed </pre>

Fade volume after time 10 with an annihilation period of 5 seconds:

 <pre class=”example-preformatted”>volume='if(lt(t,10),1,max(1-(t-10)/5,0))':eval=frame </pre>

 <h3 class=”section”>36.121 volumedetect# TOC</h3>

Detect the volume of the input video. The filter has no parameters. It supports only 16-bit signed integer samples, so the input will be converted when needed. Statistics about the volume will be printed in the log when the input stream end is reached. In particular it will show the mean volume (root mean square), maximum volume (on a per-sample basis), and the beginning of a histogram of the registered volume values (from the maximum value to a cumulated 1/1000 of the samples). All volumes are in decibels relative to the maximum PCM value. <h4 class=”subsection”>36.121.1 Examples# TOC</h4>

Here is an excerpt of the output: <pre class=”example-preformatted”>[Parsed_volumedetect_0 0xa23120] mean_volume: -27 dB [Parsed_volumedetect_0 0xa23120] max_volume: -4 dB [Parsed_volumedetect_0 0xa23120] histogram_4db: 6 [Parsed_volumedetect_0 0xa23120] histogram_5db: 62 [Parsed_volumedetect_0 0xa23120] histogram_6db: 286 [Parsed_volumedetect_0 0xa23120] histogram_7db: 1042 [Parsed_volumedetect_0 0xa23120] histogram_8db: 2551 [Parsed_volumedetect_0 0xa23120] histogram_9db: 4609 [Parsed_volumedetect_0 0xa23120] histogram_10db: 8409 </pre>

It means that: <ul class=”itemize mark-bullet”> The mean square energy is approximately -27 dB, or 10^-2.7. The largest sample is at -4 dB, or more precisely between -4 dB and -5 dB. There are 6 samples at -4 dB, 62 at -5 dB, 286 at -6 dB, etc.

In other words, raising the volume by +4 dB does not cause any clipping, raising it by +5 dB causes clipping for 6 samples, etc. <h3 class=”section”>36.122 whisper# TOC</h3>

It runs automatic speech recognition using the OpenAI’s Whisper model. It requires the whisper.cpp library (https://github.com/ggml-org/whisper.cpp) as a prerequisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-whisper</code>. The filter has following options: <dl class=”table”>

<dt><samp class="option">model</samp></dt> <dd> The file path of the downloaded whisper.cpp model (mandatory). </dd> <dt><samp class="option">language</samp></dt> <dd> The language to use for transcription. Besides a language code, the following special values enable auto-detection: <dl class="table"> <dt>‘<samp class="samp">eval</samp>’</dt> <dd> Re-run language detection on every queued chunk. This costs an extra encoder pass per chunk and the detected language may change mid-file. </dd> <dt>‘<samp class="samp">lock</samp>’</dt> <dd> Detect the language on the first chunk that produces transcribed text, then reuse it for the remaining chunks. </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Currently an alias for ‘<samp class="samp">eval</samp>’. This may change in a future version, so use ‘<samp class="samp">eval</samp>’ or ‘<samp class="samp">lock</samp>’ if the behavior matters. </dd> </dl>

Default value: <code class="code">"auto"</code> </dd> <dt><samp class="option">translate</samp></dt> <dd> If enabled, translate the transcription from the source language to English. A multilingual model is required to enable this option. Default value: <code class="code">"false"</code> </dd> <dt><samp class="option">queue</samp></dt> <dd> The maximum size that will be queued into the filter before processing the audio with whisper. Using a small value the audio stream will be processed more often, but the transcription quality will be lower and the required processing power will be higher. Using a large value (e.g. 10-20s) will produce more accurate results using less CPU (as using the whisper-cli tool), but the transcription latency will be higher, thus not useful to process real-time streams. Consider using the vad_model option associated with a large queue value. Default value: <code class="code">"3"</code> </dd> <dt><samp class="option">use_gpu</samp></dt> <dd> If the GPU support should be enabled. Default value: <code class="code">"true"</code> </dd> <dt><samp class="option">gpu_device</samp></dt> <dd> The GPU device index to use. Default value: <code class="code">"0"</code> </dd> <dt><samp class="option">destination</samp></dt> <dd> If set, the transcription output will be sent to the specified file or URL (use one of the FFmpeg AVIO protocols); otherwise, the output will be logged as info messages. The output will also be set in the "lavfi.whisper.text" frame metadata. If the destination is a file and it already exists, it will be overwritten. </dd> <dt><samp class="option">format</samp></dt> <dd> The destination format string; it could be "text" (only the transcribed text will be sent to the destination), "srt" (subtitle format) or "json". JSON segments include a <code class="code">language</code> field con-

taining the language code reported by whisper.cpp for the transcription chunk, or <code class=”code”>unknown</code> if none is available. All segments from the same chunk have the same language. Default value: <code class=”code”>"text"</code> </dd> <dt><samp class=”option”>max_len</samp></dt> <dd> Maximum segment length in characters. When set to a value greater than 0, transcription segments will be split by word to not exceed this length. This is useful for generating subtitles with shorter lines. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>vad_model</samp></dt> <dd> Path to the VAD model file. If set, the filter will load an additional voice activity detection module (https://github.com/snakers4/silero-vad) that will be used to fragment the audio queue; use this option setting a valid path obtained from the whisper.cpp repository (e.g. "../whisper.cpp/models/ggml-silero-v5.1.2.bin") and increase the queue parameter to a higher value (e.g. 20). </dd> <dt><samp class=”option”>vad_threshold</samp></dt> <dd> The VAD threshold to use. Default value: <code class=”code”>"0.5"</code> </dd> <dt><samp class=”option”>vad_min_speech_duration</samp></dt> <dd> The minimum VAD speaking duration. Default value: <code class=”code”>"0.1"</code> </dd> <dt><samp class=”option”>vad_min_silence_duration</samp></dt> <dd> The minimum VAD silence duration. Default value: <code class=”code”>"0.5"</code> </dd> </dl>

 <h4 class=”subsection”>36.122.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Run a transcription with srt file generation: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=output.srt\ :format=srt" -f null - </pre>

Run a transcription and send the output in JSON format to an HTTP service: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=http\\://localhost\\:3000\ :format=json' -f null - </pre>

Transcribe the microphone input using the VAD option: <pre class=”example-preformatted”>ffmpeg -loglevel warning -f pulse -i default \ -af 'highpass=f=200,lowpass=f=3000,whisper=model=../whisper.cpp/models/ggml-medium.bin\ :language=en\ :queue=10\ :destination=-\ :format=json\ :vad_model=../whisper.cpp/models/ggml-silero-v5.1.2.bin' -f null - </pre>

 <h2 class="chapter">37 Audio Sources# TOC</h2>

Below is a description of the currently available audio sources. <h3 class="section">37.1 abuffer# TOC</h3>

Buffer audio frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">time_base</samp></dt> <dd> The time-base which will be used for timestamps of submitted frames. It must be either a floating-point number or in <var class="var">numerator</var>/<var class="var">denominator</var> form. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> The sample rate of the incoming audio buffers. </dd> <dt><samp class="option">sample_fmt</samp></dt> <dd> The sample format of the incoming audio buffers. Either a sample format name or its corresponding integer representation from the enum AVSample-Format in <samp class="file">libavutil/samplefmt.h</samp> </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the incoming audio buffers. Either a channel layout name from channel_layout_map in <samp class="file">libavutil/channel_layout.c</samp> or its corresponding integer representation from the AV_CH_LAYOUT_* macros in <samp class="file">libavutil/channel_layout.h</samp> </dd> <dt><samp class="option">channels</samp></dt> <dd> The number of channels of the incoming audio buffers. If both <var class="var">channels</var> and <var class="var">channel_layout</var> are specified, then they must be consistent. </dd> </dl>

 <h4 class="subsection">37.1.1 Examples# TOC</h4>

 <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=s16p
</pre>

will instruct the source to accept planar 16bit signed stereo at 44100Hz. Since the sample format with name "s16p" corresponds to the number 6 and the "stereo" channel layout corresponds to the value 0x3, this is equivalent to: <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=6:channel_layout=0x3
</pre>

 <h3 class="section">37.2 aevalsrc# TOC</h3>

Generate an audio signal specified by an expression. This source accepts in input one or more expressions (one for each channel), which are evaluated and used to generate a corresponding audio signal. This source accepts the following options: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. In case the <samp class="option">channel_layout</samp> option is not specified, the selected channel layout depends on the number of provided expressions. Otherwise the last specified expression is applied to the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set the channel layout. The number of channels in the specified layout must be equal to the number of specified expressions. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default to 1024. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> </dl>

 <h4 class="subsection">37.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate silence: <pre class="example-preformatted">aevalsrc=0 </pre>

Generate a sin signal with frequency of 440 Hz, set sample rate to 8000 Hz: <pre class="example-preformatted">aevalsrc="sin(440*2*PI*t):s=8000" </pre>

Generate a two channels signal, specify the channel layout (Front Center + Back Center) explicitly: <pre class=”example-preformatted”>aevalsrc="sin(420*2*PI*t)|cos(430*2*PI*t):c=FC|BC" </pre>

Generate white noise: <pre class=”example-preformatted”>aevalsrc="-2+random(0)" </pre>

Generate an amplitude modulated signal: <pre class=”example-preformatted”>aevalsrc="sin(10*2*PI*t)*sin(880*2*PI*t)" </pre>

Generate 2.5 Hz binaural beats on a 360 Hz carrier: <pre class=”example-preformatted”>aevalsrc="0.1*sin(2*PI*(360-2.5/2)*t) | 0.1*sin(2*PI*(360+2.5/2)*t)" </pre>

 <h3 class=”section”>37.3 afdelaysrc# TOC</h3>

Generate a fractional delay FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay, d</samp></dt> <dd> Set the fractional delay. Default is 0. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set the number of filter coefficients in output audio stream. Default value is 0. </dd> <dt><samp class=”option”>channel_layout, c</samp></dt> <dd> Specifies the channel layout, and can be a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". </dd> </dl>

 <h3 class=”section”>37.4 afireqsrc# TOC</h3>

Generate a FIR equalizer coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset, p</samp></dt> <dd> Set equalizer preset. Default preset is <code class=”code”>flat</code>. Available presets are: <dl

class=”table”> <dt>‘<samp class=”samp”>custom</samp>’</dt>
<dt>‘<samp class=”samp”>flat</samp>’</dt> <dt>‘<samp
class=”samp”>acoustic</samp>’</dt> <dt>‘<samp class=”samp”>bass</samp>’</
<dt>‘<samp class=”samp”>beats</samp>’</dt> <dt>‘<samp
class=”samp”>classic</samp>’</dt> <dt>‘<samp class=”samp”>clear</samp>’</d
<dt>‘<samp class=”samp”>deep bass</samp>’</dt>
<dt>‘<samp class=”samp”>dubstep</samp>’</dt> <dt>‘<samp
class=”samp”>electronic</samp>’</dt> <dt>‘<samp
class=”samp”>hard-style</samp>’</dt> <dt>‘<samp
class=”samp”>hip-hop</samp>’</dt> <dt>‘<samp class=”samp”>jazz</samp>’</
<dt>‘<samp class=”samp”>metal</samp>’</dt> <dt>‘<samp
class=”samp”>movie</samp>’</dt> <dt>‘<samp class=”samp”>pop</samp>’</dt
<dt>‘<samp class=”samp”>r&b</samp>’</dt> <dt>‘<samp
class=”samp”>rock</samp>’</dt> <dt>‘<samp class=”samp”>vocal
booster</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gains, g</samp></dt> <dd> Set custom gains for each band. Only used if the preset option is set to <code class=”code”>custom</code>. Gains are separated by white spaces and each gain is set in dBFS. Default is <code class=”code”>0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0</code>. </dd> <dt><samp class=”option”>bands, b</samp></dt> <dd> Set the custom bands from where custom equalizer gains are set. This must be in strictly increasing order. Only used if the preset option is set to <code class=”code”>custom</code>. Bands are separated by white spaces and each band represent frequency in Hz. Default is <code class=”code”>25 40 63 100 160 250 400 630 1000 1600 2500 4000 6300 10000 16000 24000</code>. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is <code class=”code”>4096</code>. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set sample rate of output audio stream, default is <code class=”code”>44100</code>. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set number of samples per each frame in output audio stream. Default is <code class=”code”>1024</code>. </dd> <dt><samp class=”option”>interp, i</samp></dt> <dd> Set interpolation method for FIR equalizer coefficients. Can be <code class=”code”>linear</code> or <code class=”code”>cubic</code>. </dd> <dt><samp class=”option”>phase, h</samp></dt> <dd> Set phase type of FIR filter. Can be <code class=”code”>linear</code> or <code class=”code”>min</code>: minimum-phase. Default is minimum-phase filter. </dd> </dl>

 <h3 class=”section”>37.5 afirsrc#
TOC</h3>

Generate a FIR coefficients using frequency sampling method.

The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is 1025. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency points from where magnitude and phase are set. This must be in non decreasing order, and first element must be 0, while last element must be 1. Elements are separated by white spaces. </dd> <dt><samp class=”option”>magnitude, m</samp></dt> <dd> Set magnitude value for every frequency point set by <samp class=”option”>frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class=”option”>phase, p</samp></dt> <dd> Set phase value for every frequency point set by <samp class=”option”>frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>win_func, w</samp></dt> <dd> Set window function. Default is blackman. </dd> </dl>

 <h3 class=”section”>37.6 anullsrc# TOC</h3>

The null audio source, return unprocessed audio frames. It is mainly useful as a template and to be employed in analysis / debugging tools, or as the source for filters which ignore the input data (for example the sox synth filter). This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>channel_layout, cl</samp></dt> <dd> Specifies the channel layout, and can be either an integer or a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". Check the channel_layout_map definition in <samp class=”file”>libavutil/channel_layout.c</samp> for the mapping between strings and channel layout values. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specifies the sample rate, and defaults to 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per requested frames. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the audio is supposed to be generated

forever. </dd> </dl>

 <h4 class="subsection">37.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the sample rate to 48000 Hz and the channel layout to AV_CH_LAYOUT_MONO. <pre class="example-preformatted">anullsrc=r=48000:cl=4 </pre>

Do the same operation with a more obvious syntax: <pre class="example-preformatted">anullsrc=r=48000:cl=mono </pre>

All the parameters need to be explicitly defined. <h3 class="section">37.7 flite# TOC</h3>

Synthesize a voice utterance using the libflite library. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libflite</code>. Note that versions of the flite library prior to 2.0 are not thread-safe. The filter accepts the following options: <dl class="table"> <dt><samp class="option">list_voices</samp></dt> <dd> If set to 1, list the names of the available voices and exit immediately. Default value is 0. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the maximum number of samples per frame. Default value is 512. </dd> <dt><samp class="option">textfile</samp></dt> <dd> Set the filename containing the text to speak. </dd> <dt><samp class="option">text</samp></dt> <dd> Set the text to speak. </dd> <dt><samp class="option">voice, v</samp></dt> <dd> Set the voice to use for the speech synthesis. Default value is <code class="code">kal</code>. See also the <var class="var">list_voices</var> option. </dd> </dl>

 <h4 class="subsection">37.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read from file <samp class="file">speech.txt</samp>, and synthesize the text using the standard flite voice: <pre class="example-preformatted">flite=textfile=speech.txt </pre>

Read the specified text selecting the <code class="code">slt</code> voice: <pre class="example-preformatted">flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Input text to ffmpeg: <pre class="example-preformatted">ffmpeg -f lavfi -i flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Make <samp class="file">ffplay</samp> speak the specified text, using <code class="code">flite</code> and the <code class="code">lavfi</code> device: <pre class="example-preformatted">ffplay -f lavfi flite=text='No more be grieved for which that thou hast done.' </pre>

For more information about libflite, check: <a class="url" href="http://www.festvox.org/flite/">http://w <h3 class="section">37.8 anoisesrc# TOC</h3>

Generate a noise audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate. Default value is 48000 Hz. </dd> <dt><samp class="option">amplitude, a</samp></dt> <dd> Specify the amplitude (0.0 - 1.0) of the generated audio stream. Default value is 1.0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. Not specifying this option results in noise with an infinite length. </dd> <dt><samp class="option">color, colour, c</samp></dt> <dd> Specify the color of noise. Available noise colors are white, pink, brown, blue, violet and velvet. Default color is white. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Specify a value used to seed the PRNG. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per each output frame, default is 1024. </dd> <dt><samp class="option">density</samp></dt> <dd> Set the density (0.0 - 1.0) for the velvet noise generator, default is 0.05. </dd> </dl>

 <h4 class="subsection">37.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate 60 seconds of pink noise, with a 44.1 kHz sampling rate and an amplitude of 0.5: <pre class="example-preformatted">anoisesrc=d=60:c=pink:r=44100:a=0.5 </pre>

 <h3 class="section">37.9 hilbert# TOC</h3>

Generate odd-tap Hilbert transform FIR coefficients. The

resulting stream can be used with afir filter for phase-shifting the signal by 90 degrees. This is used in many matrix coding schemes and for analytic signal generation. The process is often written as a multiplication by i (or j), the imaginary unit. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set length of FIR filter, default is 22051. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. </dd> <dt><samp class="option">win_func, w</samp></dt> <dd> Set window function to be used when generating FIR coefficients. </dd> </dl>

 <h3 class="section">37.10 sinc# TOC</h3>

Generate a sinc kaiser-windowed low-pass, high-pass, band-pass, or band-reject FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class="option">hp</samp></dt> <dd> Set high-pass frequency. Default is 0. </dd> <dt><samp class="option">lp</samp></dt> <dd> Set low-pass frequency. Default is 0. If high-pass frequency is lower than low-pass frequency and low-pass frequency is higher than 0 then filter will create band-pass filter coefficients, otherwise band-reject filter coefficients. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set filter phase response. Default is 50. Allowed range is from 0 to 100. </dd> <dt><samp class="option">beta</samp></dt> <dd> Set Kaiser window beta. </dd> <dt><samp class="option">att</samp></dt> <dd> Set stop-band attenuation. Default is 120dB, allowed range is from 40 to 180 dB. </dd> <dt><samp class="option">round</samp></dt> <dd> Enable rounding, by default is disabled. </dd> <dt><samp class="option">hptaps</samp></dt> <dd> Set number of taps for high-pass filter. </dd> <dt><samp class="option">lptaps</samp></dt> <dd> Set number of taps for low-pass filter. </dd> </dl>

 <h3 class="section">37.11 sine# TOC</h3>

Generate an audio signal made of a sine wave with amplitude 1/8. The audio signal is bit-exact. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency,

f</samp></dt> <dd> Set the carrier frequency. Default is 440 Hz. </dd> <dt><samp class="option">beep_factor, b</samp></dt> <dd> Enable a periodic beep every second with frequency <var class="var">beep_factor</var> times the carrier frequency. Default is 0, meaning the beep is disabled. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate, default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. </dd> <dt><samp class="option">samples_per_frame</samp></dt> <dd> Set the number of samples per output frame. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the output audio frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the output audio frame, expressed in <var class="var">TB</var> units. </dd> <dt><samp class="option">t</samp></dt> <dd> The PTS of the output audio frame, expressed in seconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the output audio frames. </dd> </dl>

Default is <code class="code">1024</code>. </dd> </dl>

 <h4 class="subsection">37.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a simple 440 Hz sine wave: <pre class="example-preformatted">sine </pre>

Generate a 220 Hz sine wave with a 880 Hz beep each second, for 5 seconds: <pre class="example-preformatted">sine=220:4:d=5 sine=f=220:b=4:d=5 sine=frequency=220:beep_factor=4:duration=5 </pre>

Generate a 1 kHz sine wave following <code class="code">1602,1601,1602,1601,1602</code> NTSC pattern: <pre class="example-preformatted">sine=1000:samples_per_frame= 1602-not(not(eq(ld(0),1)+eq(ld(0),3)))' </pre>

 <h2 class="chapter">38 Audio Sinks# TOC</h2>

Below is a description of the currently available audio sinks. <h3 class="section">38.1 abuffersink# <a class="anchor hidden-xs"href="#toc-abuffersink"

aria-hidden=”true”>TOC</h3>

Buffer audio frames, and make them available to the end of filter chain. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVABufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class=”code”>avfilter_init_filter</code> for initialization. <h3 class=”section”>38.2 anullsink# TOC</h3>

Null audio sink; do absolutely nothing with the input audio. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class=”chapter”>39 Video Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class=”code”>--disable-filters</code>. The configure output will show the video filters included in your build. Below is a description of the currently available video filters. <h3 class=”section”>39.1 addroi# TOC</h3>

Mark a region of interest in a video frame. The frame data is passed through unchanged, but metadata is attached to the frame indicating regions of interest which can affect the behaviour of later encoding. Multiple regions can be marked by applying the filter multiple times. <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Region distance in pixels from the left edge of the frame. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Region distance in pixels from the top edge of the frame. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Region width in pixels. </dd> <dt><samp class=”option”>h</samp></dt> <dd> Region height in pixels. The parameters <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> are expressions, and may contain the following variables: <dl class=”table”> <dt><samp class=”option”>iw</samp></dt> <dd> Width of the input frame. </dd> <dt><samp class=”option”>ih</samp></dt> <dd> Height of the input frame. </dd> </dl>

</dd> <dt><samp class=”option”>qoffset</samp></dt> <dd> Quantisation offset to apply within the region. This must be a real value in the

range -1 to +1. A value of zero indicates no quality change. A negative value asks for better quality (less quantisation), while a positive value asks for worse quality (greater quantisation). The range is calibrated so that the extreme values indicate the largest possible offset - if the rest of the frame is encoded with the worst possible quality, an offset of -1 indicates that this region should be encoded with the best possible quality anyway. Intermediate values are then interpolated in some codec-dependent way. For example, in 10-bit H.264 the quantisation parameter varies between -12 and 51. A typical qoffset value of -1/10 therefore indicates that this region should be encoded with a QP around one-tenth of the full range better than the rest of the frame. So, if most of the frame were to be encoded with a QP of around 30, this region would get a QP of around 24 (an offset of approximately -1/10 * (51 - -12) = -6.3). An extreme value of -1 would indicate that this region should be encoded with the best possible quality regardless of the treatment of the rest of the frame - that is, should be encoded at a QP of -12. </dd> <dt><samp class="option">clear</samp></dt> <dd> If set to true, remove any existing regions of interest marked on the frame before adding the new one. </dd> </dl>

 <h4 class="subsection">39.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Mark the centre quarter of the frame as interesting. <pre class="example-preformatted">addroi=iw/4:ih/4:iw/2:ih/2:-1/10 </pre> Mark the 100-pixel-wide region on the left edge of the frame as very uninteresting (to be encoded at much lower quality than the rest of the frame). <pre class="example-preformatted">addroi=0:0:100:ih:+1/5 </pre>

 <h3 class="section">39.2 alphaextract# TOC</h3>

Extract the alpha component from the input as a grayscale video. This is especially useful with the <var class="var">alphamerge</var> filter. <h3 class="section">39.3 alphamerge# TOC</h3>

Add or replace the alpha component of the primary input with the grayscale value of a second input. This is intended for use with <var class="var">alphaextract</var> to allow the transmission or storage of frame sequences that have alpha in a format that doesn’t sup-

port an alpha channel. For example, to reconstruct full frames from a normal YUV-encoded video and a separate video created with <var class="var">alphaextract</var>, you might use: <pre class="example-preformatted">movie=in_alpha.mkv [alpha]; [in][alpha] alphamerge [out] </pre>

 <h3 class="section">39.4 amplify# TOC</h3>

Amplify differences between current pixel and pixels of adjacent frames in same pixel location. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set frame radius. Default is 2. Allowed range is from 1 to 63. For example radius of 3 will instruct filter to calculate average of 7 frames. </dd> <dt><samp class="option">factor</samp></dt> <dd> Set factor to amplify difference. Default is 2. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold for difference amplification. Any difference greater or equal to this value will not alter source pixel. Default is 10. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set tolerance for difference amplification. Any difference lower to this value will not alter source pixel. Default is 0. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">low</samp></dt> <dd> Set lower limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will decrease source pixel value. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will increase source pixel value. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">39.4.1 Commands# TOC</h4>

This filter supports the following commands that corresponds to option of same name: <dl class="table"> <dt><samp class="option">factor</samp></dt> <dt><samp class="option">threshold</samp></dt> <dt><samp class="option">tolerance</samp></dt> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dt><samp class="option">planes</samp></dt> </dl>

 <h3 class="section">39.5 ass<span class="pull-

right”>#
TOC</h3>

Same as the subtitles filter, except that it doesn’t require libavcodec and libavformat to work. On the other hand, it is limited to ASS (Advanced Substation Alpha) subtitles files.
This filter accepts <samp class=”option”>filename</samp>/<samp class=”option”>f</samp>, <samp class=”option”>original_size</samp>, <samp class=”option”>fontsdir</samp>, and <samp class=”option”>alpha</samp> from the subtitles filter, plus the following options: <dl class=”table”> <dt><samp class=”option”>script</samp></dt>
<dd> The ASS script to render, given inline. Exactly one of <samp class=”option”>filename</samp> and <samp class=”option”>script</samp> must be set. </dd> <dt><samp class=”option”>shaping</samp></dt>
<dd> Set the shaping engine. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class=”samp”>simple</samp>’</dt>
<dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt>
<dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew, Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd>
</dl>

The default is <code class=”code”>auto</code>. </dd> </dl>

 <h4 class=”subsection”>39.5.1 Commands# TOC</h4>

This filter supports the <samp class=”option”>script</samp> option as commands; the new script replaces the one being rendered. <h3 class=”section”>39.6 atadenoise# TOC</h3>
Apply an Adaptive Temporal Averaging Denoiser to the video input.
The filter accepts the following options: <dl class=”table”>
<dt><samp class=”option”>0a</samp></dt> <dd> Set threshold A for 1st plane. Default is 0.02. Valid range is 0 to 0.3. </dd>
<dt><samp class=”option”>0b</samp></dt> <dd> Set threshold B for 1st plane. Default is 0.04. Valid range is 0 to 5. </dd>
<dt><samp class=”option”>1a</samp></dt> <dd> Set threshold A for 2nd plane. Default is 0.02. Valid range is 0 to 0.3. </dd>
<dt><samp class=”option”>1b</samp></dt> <dd> Set threshold B for 2nd plane. Default is 0.04. Valid range is 0 to 5. </dd>

<dt><samp class="option">2a</samp></dt> <dd> Set threshold A for 3rd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">2b</samp></dt> <dd> Set threshold B for 3rd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">3a</samp></dt> <dd> Set threshold A for 4th plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">3b</samp></dt> <dd> Set threshold B for 4th plane. Default is 0.04. Valid range is 0 to 5. Threshold A is designed to react on abrupt changes in the input signal and threshold B is designed to react on continuous changes in the input signal. </dd> <dt><samp class="option">s</samp></dt> <dd> Set number of frames filter will use for averaging. Default is 9. Must be odd number in range [5, 129]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set what planes of frame filter will use for averaging. Default is all. </dd> <dt><samp class="option">a</samp></dt> <dd> Set what variant of algorithm filter will use for averaging. Default is <code class="code">p</code> parallel. Alternatively can be set to <code class="code">s</code> serial. Parallel can be faster then serial, while other way around is never true. Parallel will abort early on first change being greater then thresholds, while serial will continue processing other side of frames if they are equal or below thresholds. </dd> <dt><samp class="option">0s</samp></dt> <dt><samp class="option">1s</samp></dt> <dt><samp class="option">2s</samp></dt> <dt><samp class="option">3s</samp></dt> <dd> Set sigma for 1st plane, 2nd plane, 3rd plane or 4th plane. Default is 32767. Valid range is from 0 to 32767. This options controls weight for each pixel in radius defined by size. Default value means every pixel have same weight. Setting this option to 0 effectively disables filtering. </dd> </dl>

 <h4 class="subsection">39.6.1 Commands# TOC</h4> This filter supports same commands as options except option <code class="code">s</code>. The command accepts the same syntax of the corresponding option.
 <h3 class="section">39.7 avgblur# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sizeX</samp></dt> <dd> Set horizontal radius size. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class="option">sizeY</samp></dt> <dd> Set vertical radius size, if zero it will be same as <code class="code">sizeX</code>.

Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">39.7.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.8 backgroundkey# TOC</h3>

Turns a static background into transparency. The filter accepts the following option: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Threshold for scene change detection. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the background. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the blend amount for pixels that are not similar. </dd> </dl>

 <h4 class="subsection">39.8.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.9 bbox# TOC</h3>

Compute the bounding box for the non-black pixels in the input frame luma plane. This filter computes the bounding box containing all the pixels with a luma value greater than the minimum allowed value. The parameters describing the bounding box are printed on the filter log. The filter accepts the following option: <dl class="table"> <dt><samp class="option">min_val</samp></dt> <dd> Set the minimal luma value. Default is <code class="code">16</code>. </dd> </dl>

 <h4 class="subsection">39.9.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.10 bilateral<a class="anchor hidden-xs" href="#bilateral" aria-

hidden=”true”># TOC</h3> Apply bilateral filter, spatial smoothing while preserving edges. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight. Allowed range is 0 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate range weight. Allowed range is 0 to 1. Default is 0.1. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>39.10.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.11 bitplanenoise# TOC</h3>

Show and measure bit plane noise. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>bitplane</samp></dt> <dd> Set which plane to analyze. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>filter</samp></dt> <dd> Filter out noisy pixels from <code class=”code”>bitplane</code> set above. Default is disabled. </dd> </dl>

 <h3 class=”section”>39.12 blackdetect, blackdetect_vulkan# TOC</h3>

Detect video intervals that are (almost) completely black. Can be useful to detect chapter transitions, commercials, or invalid recordings. The filter outputs its detection analysis to both the log as well as frame metadata. If a black segment of at least the specified minimum duration is found, a line with the start and end timestamps as well as duration is printed to the log with level <code class=”code”>info</code>. In addition, a log line with level <code class=”code”>debug</code> is printed per frame showing the black amount detected for that frame. The filter also attaches metadata to the first frame of a black segment with key <code class=”code”>lavfi.black_start</code> and to the first frame after the black segment ends with key <code class=”code”>lavfi.black_end</code>. The value is the frame’s timestamp. This metadata is added regardless of the minimum duration specified. The filter

accepts the following options: <dl class="table"> <dt><samp class="option">black_min_duration, d</samp></dt> <dd> Set the minimum detected black duration expressed in seconds. It must be a non-negative floating point number. Default value is 2.0. </dd> <dt><samp class="option">picture_black_ratio_th, pic_th</samp></dt> <dd> Set the threshold for considering a picture "black". Express the minimum value for the ratio: <pre class="example-preformatted"><var class="var">nb_black_pixels</var> / <var class="var">nb_pixels</var> </pre>

for which a picture is considered black. Default value is 0.98. </dd> <dt><samp class="option">pixel_black_th, pix_th</samp></dt> <dd> Set the threshold for considering a pixel "black". The threshold expresses the maximum pixel luma value for which a pixel is considered "black". The provided value is scaled according to the following equation: <pre class="example-preformatted"><var class="var">absolute_threshold</var> = <var class="var">luma_minimum_value</var> + <var class="var">pixel_black_th</var> * <var class="var">luma_range_size</var> </pre>

<var class="var">luma_range_size</var> and <var class="var">luma_minimum_value</var> depend on the input video format, the range is [0-255] for YUV full-range formats and [16-235] for YUV non full-range formats. Default value is 0.10. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If true, check the alpha channel instead of the luma channel. Detects frames which are (almost) transparent, instead of frames which are almost black. Default value is disabled. </dd> </dl>

The following example sets the maximum pixel threshold to the minimum value, and detects only black intervals of 2 or more seconds: <pre class="example-preformatted">blackdetect=d=2:pix_th=0.00 </pre>

 <h3 class="section">39.13 blackframe# TOC</h3>

Detect frames that are (almost) completely black. Can be useful to detect chapter transitions or commercials. Output lines consist of the frame number of the detected frame, the percentage of blackness, the position in the file if known or -1 and the timestamp in seconds. In order to display the output lines, you need to set the loglevel at least to the AV_LOG_INFO value. This filter exports frame metadata <code class="code">lavfi.blackframe.pblack</code>. The value represents the percentage of pixels in the picture that are below the threshold value. It accepts the following parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt>

<dd> The percentage of the pixels that have to be below the threshold; it defaults to <code class=”code”>98</code>. </dd> <dt><samp class=”option”>threshold, thresh</samp></dt> <dd> The threshold below which a pixel value is considered black; it defaults to <code class=”code”>32</code>. </dd> </dl>

 <h3 class=”section”>39.14 blend# TOC</h3>

Blend two video frames into each other. The <code class=”code”>blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. The <code class=”code”>tblend</code> (time blend) filter takes two consecutive frames from one single stream, and outputs the result obtained by blending the new frame on top of the old frame. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>c0_mode</samp></dt> <dt><samp class=”option”>c1_mode</samp></dt> <dt><samp class=”option”>c2_mode</samp></dt> <dt><samp class=”option”>c3_mode</samp></dt> <dt><samp class=”option”>all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class=”var”>all_mode</var>. Default value is <code class=”code”>normal</code>. Available values for component modes are: <dl class=”table”> <dt>‘<samp class=”samp”>addition</samp>’</dt> <dt>‘<samp class=”samp”>and</samp>’</dt> <dt>‘<samp class=”samp”>average</samp>’</dt> <dt>‘<samp class=”samp”>bleach</samp>’</dt> <dt>‘<samp class=”samp”>burn</samp>’</dt> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dt>‘<samp class=”samp”>difference</samp>’</dt> <dt>‘<samp class=”samp”>divide</samp>’</dt> <dt>‘<samp class=”samp”>dodge</samp>’</dt> <dt>‘<samp class=”samp”>exclusion</samp>’</dt> <dt>‘<samp class=”samp”>extremity</samp>’</dt> <dt>‘<samp class=”samp”>freeze</samp>’</dt> <dt>‘<samp class=”samp”>geometric</samp>’</dt> <dt>‘<samp class=”samp”>glow</samp>’</dt> <dt>‘<samp class=”samp”>grainextract</samp>’</dt> <dt>‘<samp class=”samp”>grainmerge</samp>’</dt> <dt>‘<samp class=”samp”>hardlight</samp>’</dt> <dt>‘<samp class=”samp”>hardmix</samp>’</dt> <dt>‘<samp class=”samp”>hardoverlay</samp>’</dt> <dt>‘<samp class=”samp”>harmonic</samp>’</dt> <dt>‘<samp class=”samp”>heat</samp>’</dt> <dt>‘<samp class=”samp”>interpolate</samp>’</dt> <dt>‘<samp class=”samp”>lighten</samp>’</dt> <dt>‘<samp class=”samp”>linearlight</samp>’</dt> <dt>‘<samp class=”samp”>multiply</samp>’</dt> <dt>‘<samp class=”samp”>multiply128</samp>’</dt> <dt>‘<samp class=”samp”>negation</samp>’</dt> <dt>‘<samp class=”samp”>normal</samp>’</dt>

<dt>‘<samp class=”samp”>or</samp>’</dt> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dt>‘<samp class=”samp”>phoenix</samp>’ <dt>‘<samp class=”samp”>pinlight</samp>’</dt> <dt>‘<samp class=”samp”>reflect</samp>’</dt> <dt>‘<samp class=”samp”>screen</samp>’</ <dt>‘<samp class=”samp”>softdifference</samp>’</dt> <dt>‘<samp class=”samp”>softlight</samp>’</dt> <dt>‘<samp class=”samp”>stain</samp>’</dt> <dt>‘<samp class=”samp”>subtract</samp>’< <dt>‘<samp class=”samp”>vividlight</samp>’</dt> <dt>‘<samp class=”samp”>xor</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>c0_opacity</samp></dt> <dt><samp class=”option”>c1_opacity</samp></dt> <dt><samp class=”option”>c2_opacity</samp></dt> <dt><samp class=”option”>c3_opacity</samp></dt> <dt><samp class=”option”>all_opacity</samp></dt> <dd> Set blend opacity for specific pixel component or all pixel components in case of <var class=”var”>all_opacity</var>. Only used in combination with pixel component blend modes. </dd> <dt><samp class=”option”>c0_expr</samp></dt> <dt><samp class=”option”>c1_expr</samp></dt> <dt><samp class=”option”>c2_expr</samp></dt> <dt><samp class=”option”>c3_expr</samp></dt> <dt><samp class=”option”>all_expr</samp></dt> <dd> Set blend expression for specific pixel component or all pixel components in case of <var class=”var”>all_expr</var>. Note that related mode options will be ignored if those are set. The expressions can use the following variables: <dl class=”table”> <dt><samp class=”option”>N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class=”code”>0</code>. </dd> <dt><samp class=”option”>X</samp></dt> <dt><samp class=”option”>Y</samp></dt> <dd> the coordinates of the current sample </dd> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> the width and height of currently filtered plane </dd> <dt><samp class=”option”>SW</samp></dt> <dt><samp class=”option”>SH</samp></dt> <dd> Width and height scale for the plane being filtered. It is the ratio between the dimensions of the current plane to the luma plane, e.g. for a <code class=”code”>yuv420p</code> frame, the values are <code class=”code”>1,1</code> for the luma plane and <code class=”code”>0.5,0.5</code> for the chroma planes. </dd> <dt><samp class=”option”>T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class=”option”>TOP, A</samp></dt> <dd> Value of pixel component at current location for first video frame (top layer). </dd> <dt><samp class=”option”>BOTTOM, B</samp></dt> <dd> Value of pixel component at current location for second video frame (bottom layer). </dd> </dl> </dd> </dl>

The <code class=”code”>blend</code> filter also supports the framesync options. <h4 class=”subsection”>39.14.1 Examples<a class=”anchor hidden-xs” href=”#Examples-

93” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Apply transition from bottom layer to top layer in first 10 seconds: <pre class=”example-preformatted”>blend=all_expr='A*(if(gte(T,10),1,T/10))+B*(1-(if(gte(T,10),1,T/10)))' </pre>

Apply linear horizontal transition from top layer to bottom layer: <pre class=”example-preformatted”>blend=all_expr='A*(X/W)+B*(1-X/W)' </pre>

Apply 1x1 checkerboard effect: <pre class=”example-preformatted”>blend=all_expr='if(eq(mod(X,2),mod(Y,2)),A,B)' </pre>

Apply uncover left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(N*SW+X,W),A,B)' </pre>

Apply uncover down effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(Y-N*SH,0),A,B)' </pre>

Apply uncover up-left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(T*SH*40+Y,H)*gte((T*40*SW+X)*W/H,W),A,B)' </pre>

Split diagonally video and shows top and bottom layer on each side: <pre class=”example-preformatted”>blend=all_expr='if(gt(X,Y*(W/H)),A,B)' </pre>

Display differences between the current and the previous frame: <pre class=”example-preformatted”>tblend=all_mode=grainextract </pre>

 <h4 class=”subsection”>39.14.2 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>39.15 blockdetect# TOC</h3>

Determines blockiness of frames without altering the input frames. Based on Remco Muijs and Ihor Kirenko: "A no-reference blocking artifact measure for adaptive video processing." 2005 13th European signal processing conference. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>period_min</samp></dt>

<dt><samp class=”option”>period_max</samp></dt> <dd> Set minimum and maximum values for determining pixel grids (periods). Default values are [3,24]. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>39.15.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Determine blockiness for the first plane and search for periods within [8,32]: <pre class=”example-preformatted”>blockdetect=period_min=8:period_max=32:planes=1 </pre>

 <h3 class=”section”>39.16 blurdetect# TOC</h3>

Determines blurriness of frames without altering the input frames. Based on Marziliano, Pina, et al. "A no-reference perceptual blur metric." Allows for a block-based abbreviation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>low</samp></dt> <dt><samp class=”option”>high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class=”var”>low</var> and <var class=”var”>high</var> threshold values must be chosen in the range [0,1], and <var class=”var”>low</var> should be lesser or equal to <var class=”var”>high</var>. Default value for <var class=”var”>low</var> is <code class=”code”>20/255</code>, and default value for <var class=”var”>high</var> is <code class=”code”>50/255</code>. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> Define the radius to search around an edge pixel for local maxima. </dd> <dt><samp class=”option”>block_pct</samp></dt> <dd> Determine blurriness only for the most significant blocks, given in percentage. </dd> <dt><samp class=”option”>block_width</samp></dt> <dd> Determine blurriness for blocks of width <var class=”var”>block_width</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class=”var”>block_height</var>. </dd> <dt><samp class=”option”>block_height</samp></dt> <dd> Determine blurriness for blocks of height <var class=”var”>block_height</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class=”var”>block_width</var>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">39.16.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Determine blur for 80% of most significant 32x32 blocks: <pre class="example-preformatted">blurdetect=block_width=32:block_height=32:block_pct=80
</pre>

 <h3 class="section">39.17 bm3d#
TOC</h3>

Denoise frames using Block-Matching 3D algorithm. The filter accepts the following options. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set denoising strength. Default value is 1. Allowed range is from 0 to 999.9. The denoising algorithm is very sensitive to sigma, so adjust it according to the source. </dd> <dt><samp class="option">block</samp></dt> <dd> Set local patch size. This sets dimensions in 2D. </dd> <dt><samp class="option">bstep</samp></dt> <dd> Set sliding step for processing blocks. Default value is 4. Allowed range is from 1 to 64. Smaller values allows processing more reference blocks and is slower. </dd> <dt><samp class="option">group</samp></dt> <dd> Set maximal number of similar blocks for 3rd dimension. Default value is 1. When set to 1, no block matching is done. Larger values allows more blocks in single group. Allowed range is from 1 to 256. </dd> <dt><samp class="option">range</samp></dt> <dd> Set radius for search block matching. Default is 9. Allowed range is from 1 to INT32_MAX. </dd> <dt><samp class="option">mstep</samp></dt> <dd> Set step between two search locations for block matching. Default is 1. Allowed range is from 1 to 64. Smaller is slower. </dd> <dt><samp class="option">thmse</samp></dt> <dd> Set threshold of mean square error for block matching. Valid range is 0 to INT32_MAX. </dd> <dt><samp class="option">hdthr</samp></dt> <dd> Set thresholding parameter for hard thresholding in 3D transformed domain. Larger values results in stronger hard-thresholding filtering in frequency domain. </dd> <dt><samp class="option">estim</samp></dt> <dd> Set filtering estimation mode. Can be <code class="code">basic</code> or <code class="code">final</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">ref</samp></dt> <dd> If enabled, filter will use 2nd stream for block matching. Default is disabled for <code class="code">basic</code> value of <var class="var">estim</var> option, and always enabled if value of <var class="var">estim</var> is <code class="code">final</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is all available except alpha. </dd>

</dl>

 <h4 class="subsection">39.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Basic filtering with bm3d: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=ba
</pre>

Same as above, but filtering only luma: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic:planes=1
</pre>

Same as above, but with both estimation modes:
<pre class="example-preformatted">split[a][b],[a]bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic[a],[b
</pre>

Same as above, but prefilter with nlmeans filter instead: <pre class="example-preformatted">split[a][b],[a]nlmeans=s=3:r=7:p=
</pre>

 <h3 class="section">39.18 boxblur# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">luma_power, lp</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">chroma_power, cp</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dt><samp class="option">alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class="code">min(w,h)/2</code> for the luma and alpha planes, and of <code class="code">min(cw,ch)/2</code> for the chroma planes. Default value for <samp class="option">luma_radius</samp> is "2". If not specified, <samp class="option">chroma_radius</samp> and <samp class="option">alpha_radius</samp> default to the corresponding value set for <samp class="option">luma_radius</samp>. The expressions can contain the following constants: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp

class=”option”>h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class=”option”>cw</samp></dt> <dt><samp class=”option”>ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class=”option”>luma_power</samp> is 2. If not specified, <samp class=”option”>chroma_power</samp> and <samp class=”option”>alpha_power</samp> default to the corresponding value set for <samp class=”option”>luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class=”subsection”>39.18.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply a boxblur filter with the luma, chroma, and alpha radii set to 2: <pre class=”example-preformatted”>boxblur=luma_radius=2:luma_power=1
boxblur=2:1 </pre>

Set the luma radius to 2, and alpha and chroma radius to 0: <pre class=”example-preformatted”>boxblur=2:1:cr=0:ar=0
</pre>

Set the luma and chroma radii to a fraction of the video dimension:
 <pre class=”example-preformatted”>boxblur=luma_radius=min(h\,w)/10:luma_pov
</pre>

 <h3 class=”section”>39.19 bwdif# TOC</h3>

Deinterlace the input video ("bwdif" stands for "Bob Weaver Deinterlacing Filter"). Motion adaptive deinterlacing based on yadif with the use of w3fdif and cubic interpolation algorithms. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, send_frame</samp></dt>

<dd> Output one frame for each frame. </dd> <dt><samp class=”option”>1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>send_field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h3 class=”section”>39.20 ccrepack# TOC</h3>

Repack CEA-708 closed captioning side data This filter fixes various issues seen with commercial encoders related to upstream malformed CEA-708 payloads, specifically incorrect number of tuples (wrong cc_count for the target FPS), and incorrect ordering of tuples (i.e. the CEA-608 tuples are not at the first entries in the payload). <h3 class=”section”>39.21 cas# TOC</h3>

Apply Contrast Adaptive Sharpen filter to video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>strength</samp></dt> <dd> Set the sharpening strength. Default value is 0. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default value is to filter all planes except alpha plane. </dd> </dl>

 <h4 class=”subsection”>39.21.1 Commands<a class=”anchor hidden-xs” href=”#Commands-

62" aria-hidden="true"># TOC</h4> This filter supports same commands as options. <h3 class="section">39.22 chromahold# TOC</h3> Remove all color information for all colors except for certain one. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will not be replaced with neutral chroma. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more preserved color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">39.22.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.23 chromakey# TOC</h3> YUV colorspace color/chroma keying. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will be replaced with transparency. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like

"green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">39.23.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">39.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf chromakey=green out.png </pre>

Overlay a greenscreen-video on top of a static black background. <pre class="example-preformatted">ffmpeg -f lavfi -i color=c=black:s=1280x720 -i video.mp4 -shortest -filter_complex "[1:v]chromakey=0x70de77:0.1:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.mkv </pre>

 <h3 class="section">39.24 chromanr# TOC</h3> Reduce chrominance noise. The filter accepts the following options: <dl class="table"> <dt><samp class="option">thres</samp></dt> <dd> Set threshold for averaging chrominance values. Sum of absolute difference of Y, U and V pixel components of current pixel and neighbour pixels lower than this threshold will be used in averaging. Luma component is left unchanged and is copied to output. Default value is 30. Allowed range is from 1 to 200. </dd> <dt><samp class="option">sizew</samp></dt> <dd> Set horizontal radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">sizeh</samp></dt> <dd> Set vertical radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">stepw</samp></dt> <dd> Set horizontal step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">steph</samp></dt> <dd> Set vertical step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">threy</samp></dt> <dd> Set Y threshold for averaging chrominance values. Set finer con-

trol for max allowed difference between Y components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threu</samp></dt> <dd> Set U threshold for averaging chrominance values. Set finer control for max allowed difference between U components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threv</samp></dt> <dd> Set V threshold for averaging chrominance values. Set finer control for max allowed difference between V components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">distance</samp></dt> <dd> Set distance type used in calculations. <dl class="table"> <dt>‘<samp class="samp">manhattan</samp>’</dt> <dd> Absolute difference. </dd> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dd> Difference squared. </dd> </dl>

Default distance type is manhattan. </dd> </dl>

 <h4 class="subsection">39.24.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. <h3 class="section">39.25 chromashift# TOC</h3> Shift chroma pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cbh</samp></dt> <dd> Set amount to shift chroma-blue horizontally. </dd> <dt><samp class="option">cbv</samp></dt> <dd> Set amount to shift chroma-blue vertically. </dd> <dt><samp class="option">crh</samp></dt> <dd> Set amount to shift chroma-red horizontally. </dd> <dt><samp class="option">crv</samp></dt> <dd> Set amount to shift chroma-red vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">39.25.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.26 ciescope<a class="anchor hidden-xs" href="#ciescope" aria-

hidden=”true”># TOC</h3>

Display CIE color diagram with pixels overlaid onto it. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>system</samp></dt> <dd> Set color system. <dl class=”table”> <dt>‘<samp class=”samp”>ntsc, 470m</samp>’</dt> <dt>‘<samp class=”samp”>ebu, 470bg</samp>’</dt> <dt>‘<samp class=”samp”>smpte</samp>’</dt> <dt>‘<samp class=”samp”>240m</samp>’</dt> <dt>‘<samp class=”samp”>apple</samp>’</dt> <dt>‘<samp class=”samp”>widergb</samp>’< <dt>‘<samp class=”samp”>cie1931</samp>’</dt> <dt>‘<samp class=”samp”>rec709, hdtv</samp>’</dt> <dt>‘<samp class=”samp”>uhdtv, rec2020</samp>’</dt> <dt>‘<samp class=”samp”>dcip3</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>cie</samp></dt> <dd> Set CIE system. <dl class=”table”> <dt>‘<samp class=”samp”>xyy</samp>’</dt> <dt>‘<samp class=”samp”>ucs</samp>’</dt> <dt>‘<samp class=”samp”>luv</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gamuts</samp></dt> <dd> Set what gamuts to draw. See <code class=”code”>system</code> option for available values. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set ciescope size, by default set to 512. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity used to map input pixel values to CIE diagram. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Set contrast used to draw tongue colors that are out of active color system gamut. </dd> <dt><samp class=”option”>corrgamma</samp></dt> <dd> Correct gamma displayed on scope, by default enabled. </dd> <dt><samp class=”option”>showwhite</samp></dt> <dd> Show white point on CIE diagram, by default disabled. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set input gamma. Used only with XYZ input color space. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Fill with CIE colors. By default is enabled. </dd> </dl>

 <h3 class=”section”>39.27 codecview# TOC</h3>

Visualize information exported by some codecs. Some codecs can export information through frames using side-data or other means. For example, some MPEG based codecs export motion vectors through the <var class=”var”>export_mvs</var> flag in the codec <samp class=”option”>flags2</samp> option. The filter

accepts the following option: <dl class="table"> <dt><samp class="option">block</samp></dt> <dd> Display block partition structure using the luma plane. </dd> <dt><samp class="option">mv</samp></dt> <dd> Set motion vectors to visualize. Available flags for <var class="var">mv</var> are: <dl class="table"> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> forward predicted MVs of P-frames </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> forward predicted MVs of B-frames </dd> <dt>‘<samp class="samp">bb</samp>’</dt> <dd> backward predicted MVs of B-frames </dd> </dl>

</dd> <dt><samp class="option">qp</samp></dt> <dd> Display quantization parameters using the chroma planes. </dd> <dt><samp class="option">mv_type, mvt</samp></dt> <dd> Set motion vectors type to visualize. Includes MVs from all frames unless specified by <var class="var">frame_type</var> option. Available flags for <var class="var">mv_type</var> are: <dl class="table"> <dt>‘<samp class="samp">fp</samp>’</dt> <dd> forward predicted MVs </dd> <dt>‘<samp class="samp">bp</samp>’</dt> <dd> backward predicted MVs </dd> </dl>

</dd> <dt><samp class="option">frame_type, ft</samp></dt> <dd> Set frame type to visualize motion vectors of. Available flags for <var class="var">frame_type</var> are: <dl class="table"> <dt>‘<samp class="samp">if</samp>’</dt> <dd> intra-coded frames (I-frames) </dd> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> predicted frames (P-frames) </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> bi-directionally predicted frames (B-frames) </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.27.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Visualize forward predicted MVs of all frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv_type=fp </pre>

Visualize multi-directionals MVs of P and B-Frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv=pf+bf+bb </pre>

 <h3 class="section">39.28 colorbalance# TOC</h3> Modify

intensity of primary colors (red, green and blue) of input frames. The filter allows an input frame to be adjusted in the shadows, midtones or highlights regions for the red-cyan, green-magenta or blue-yellow balance. A positive adjustment value shifts the balance towards the primary color, a negative value towards the complementary color. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rs</samp></dt> <dt><samp class="option">gs</samp></dt> <dt><samp class="option">bs</samp></dt> <dd> Adjust red, green and blue shadows (darkest pixels). </dd> <dt><samp class="option">rm</samp></dt> <dt><samp class="option">gm</samp></dt> <dt><samp class="option">bm</samp></dt> <dd> Adjust red, green and blue midtones (medium pixels). </dd> <dt><samp class="option">rh</samp></dt> <dt><samp class="option">gh</samp></dt> <dt><samp class="option">bh</samp></dt> <dd> Adjust red, green and blue highlights (brightest pixels). Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">pl</samp></dt> <dd> Preserve lightness when changing color balance. Default is disabled. </dd> </dl>

 <h4 class="subsection">39.28.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add red color cast to shadows: <pre class="example-preformatted">colorbalance=rs=.3 </pre>

 <h4 class="subsection">39.28.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.29 colorcontrast# TOC</h3>

Adjust color contrast between RGB components. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rc</samp></dt> <dd> Set the red-cyan contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">gm</samp></dt> <dd> Set the green-magenta contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">by</samp></dt> <dd> Set the blue-yellow contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0.

</dd> <dt><samp class="option">rcw</samp></dt> <dt><samp class="option">gmw</samp></dt> <dt><samp class="option">byw</samp></dt> <dd> Set the weight of each <code class="code">rc</code>, <code class="code">gm</code>, <code class="code">by</code> option value. Default value is 0.0. Allowed range is from 0.0 to 1.0. If all weights are 0.0 filtering is disabled. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Default value is 0.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">39.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.30 colorcorrect# TOC</h3>

Adjust color white balance selectively for blacks and whites. This filter operates in YUV colorspace. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rl</samp></dt> <dd> Set the red shadow spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">bl</samp></dt> <dd> Set the blue shadow spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">rh</samp></dt> <dd> Set the red highlight spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">bh</samp></dt> <dd> Set the blue highlight spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the amount of saturation. Allowed range is from -3.0 to 3.0. Default value is 1. </dd> <dt><samp class="option">analyze</samp></dt> <dd> If set to anything other than <code class="code">manual</code> it will analyze every frame and use derived parameters for filtering output frame. Possible values are: <dl class="table"> <dt>‘<samp class="samp">manual</samp>’</dt> <dt>‘<samp class="samp">average</samp>’</dt> <dt>‘<samp class="samp">minmax</samp>’</dt> <dt>‘<samp class="samp">median</samp>’</dt> </dl> Default value is <code class="code">manual</code>. </dd> </dl>

 <h4 class="subsection">39.30.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.31

colorchannelmixer# TOC</h3>

Adjust video input frames by re-mixing color channels.
This filter modifies a color channel by adding the values associated to the other channels of the same pixels. For example if the value to modify is red, the output value will be:
<pre class="example-preformatted"><var class="var">red</var>=<var class="var">red</var>*<var class="var">rr</var> + <var class="var">blue</var>*<var class="var">rb</var> + <var class="var">green</var>*<var class="var">rg</var>
+ <var class="var">alpha</var>*<var class="var">ra</var> </pre>

The filter accepts the following options: <dl class="table">
<dt><samp class="option">rr</samp></dt> <dt><samp class="option">rg</samp></dt>
<dt><samp class="option">rb</samp></dt> <dt><samp class="option">ra</samp></dt>
<dd> Adjust contribution of input red, green, blue and alpha channels for output red channel. Default is <code class="code">1</code> for <var class="var">rr</var>, and <code class="code">0</code> for <var class="var">rg</var>, <var class="var">rb</var> and <var class="var">ra</var>. </dd> <dt><samp class="option">gr</samp></dt>
<dt><samp class="option">gg</samp></dt> <dt><samp class="option">gb</samp></dt>
<dt><samp class="option">ga</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output green channel. Default is <code class="code">1</code> for <var class="var">gg</var>, and <code class="code">0</code> for <var class="var">gr</var>, <var class="var">gb</var> and <var class="var">ga</var>.
</dd> <dt><samp class="option">br</samp></dt> <dt><samp class="option">bg</samp></dt> <dt><samp class="option">bb</samp></dt>
<dt><samp class="option">ba</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output blue channel. Default is <code class="code">1</code> for <var class="var">bb</var>, and <code class="code">0</code> for <var class="var">br</var>, <var class="var">bg</var> and <var class="var">ba</var>.
</dd> <dt><samp class="option">ar</samp></dt> <dt><samp class="option">ag</samp></dt> <dt><samp class="option">ab</samp></dt>
<dt><samp class="option">aa</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output alpha channel. Default is <code class="code">1</code> for <var class="var">aa</var>, and <code class="code">0</code> for <var class="var">ar</var>, <var class="var">ag</var> and <var class="var">ab</var>. Allowed ranges for options are <code class="code">[-2.0, 2.0]</code>. </dd>
<dt><samp class="option">pc</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt>
<dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt>

<dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class=”samp”>sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class=”samp”>nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class=”samp”>pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl>

</dd> <dt><samp class=”option”>pa</samp></dt> <dd> Set the preserve color amount when changing colors. Allowed range is from <code class=”code”>[0.0, 1.0]</code>. Default is <code class=”code”>0.0</code>, thus disabled. </dd> </dl>

 <h4 class=”subsection”>39.31.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert source to grayscale: <pre class=”example-preformatted”>colorchannelmixer=.3:.4:.3:0:.3:.4:.3:0:.3:.4:.3 </pre> Simulate sepia tones: <pre class=”example-preformatted”>colorchannelmixer=.393:.769:.189:0:.349:.686:.168:0:.272:.534:.131 </pre>

 <h4 class=”subsection”>39.31.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.32 colordetect# TOC</h3> Analyze the video frames to determine the effective value range and alpha mode. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set of properties to detect. Unavailable properties, such as alpha mode for an input image without an alpha channel, will be ignored automatically. Accepts a combination of the following flags: <dl class=”table”> <dt>‘<samp class=”samp”>color_range</samp>’</dt> <dd> Detect if the source contains luma pixels outside the limited (MPEG) range, which indicates that this is a full range YUV source. </dd> <dt>‘<samp class=”samp”>alpha_mode</samp>’</dt> <dd> Detect if the source contains color values above the alpha channel, which indicates that the alpha channel is independent (straight), rather than premultiplied. Also detects if the alpha plane is fully opaque or not. </dd> <dt>‘<samp class=”samp”>all</samp>’</dt>

<dd> Enable detection of all of the above properties. This is the default. </dd> </dl>

</dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Allow pixels to exceed the expected bounds by this fraction of the full scale value range before treating it as evidence of (respectively) full range or straight alpha. Value range is from 0 to 0.05. Defaults to 0.001 (=0.1%). </dd> </dl>

 <h3 class=”section”>39.33 colorize# TOC</h3> Overlay a solid color on the video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>hue</samp></dt> <dd> Set the color hue. Allowed range is from 0 to 360. Default value is 0. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set the color saturation. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class=”option”>lightness</samp></dt> <dd> Set the color lightness. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> Set the mix of source lightness. By default is set to 1.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class=”subsection”>39.33.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.34 colorkey# TOC</h3> RGB colorspace color keying. This filter operates on 8-bit RGB format frames by setting the alpha component of each pixel which falls within the similarity radius of the key color to 0. The alpha value for pixels outside the similarity radius depends on the value of the blend option. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dd> Set the color for which alpha will be set to 0 (full transparency). See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default is <code class=”code”>black</code>. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Set the radius from the key color within which other colors also have full transparency. The computed distance is related to the unit fractional distance in 3D space between the RGB values of the key color and the pixel’s color. Range is 0.01 to 1.0. 0.01 matches within a very small radius around the exact key color, while 1.0 matches everything.

Default is <code class=”code”>0.01</code>. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set how the alpha value for pixels that fall outside the similarity radius is computed. 0.0 makes pixels either fully transparent or fully opaque. Higher values result in semi-transparent pixels, with greater transparency the more similar the pixel color is to the key color. Range is 0.0 to 1.0. Default is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>39.34.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make every green pixel in the input image transparent: <pre class=”example-preformatted”>ffmpeg -i input.png -vf colorkey=green out.png </pre>

Overlay a greenscreen-video on top of a static background image. <pre class=”example-preformatted”>ffmpeg -i background.png -i video.mp4 -filter_complex "[1:v]colorkey=0x3BBD1E:0.3:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.flv </pre>

 <h4 class=”subsection”>39.34.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>39.35 colorhold# TOC</h3> Remove all color information for all RGB colors except for certain one. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dd> The color which will not be replaced with neutral gray. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels fully gray. Higher values result in more preserved color. </dd> </dl>

 <h4 class=”subsection”>39.35.1 Commands# TOC</h4> This filter supports same commands

as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.36 colorlevels# TOC</h3>

Adjust video input frames using levels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rimin</samp></dt> <dt><samp class="option">gimin</samp></dt> <dt><samp class="option">bimin</samp></dt> <dt><samp class="option">aimin</samp></dt> <dd> Adjust red, green, blue and alpha input black point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">rimax</samp></dt> <dt><samp class="option">gimax</samp></dt> <dt><samp class="option">bimax</samp></dt> <dt><samp class="option">aimax</samp></dt> <dd> Adjust red, green, blue and alpha input white point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">1</code>. Input levels are used to lighten highlights (bright tones), darken shadows (dark tones), change the balance of bright and dark tones. </dd> <dt><samp class="option">romin</samp></dt> <dt><samp class="option">gomin</samp></dt> <dt><samp class="option">bomin</samp></dt> <dt><samp class="option">aomin</samp></dt> <dd> Adjust red, green, blue and alpha output black point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">romax</samp></dt> <dt><samp class="option">gomax</samp></dt> <dt><samp class="option">bomax</samp></dt> <dt><samp class="option">aomax</samp></dt> <dd> Adjust red, green, blue and alpha output white point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">1</code>. Output levels allows manual selection of a constrained output level range. </dd> <dt><samp class="option">preserve</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.36.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Make video output darker: <pre class="example-preformatted">colorlevels=rimin=0.058:gimin=0.058:bimin=0.058 </pre>

Increase contrast: <pre class="example-preformatted">colorlevels=rimin=0.039:gimin=0.039:bimin=0.039:rimax=0.96:gimax=0.96:bimax=0.96 </pre>

Make video output lighter: <pre class="example-preformatted">colorlevels=rimax=0.902:gimax=0.902:bimax=0.902 </pre>

Increase brightness: <pre class="example-preformatted">colorlevels=romin=0.5:gomin=0.5:bomin=0.5 </pre>

 <h4 class="subsection">39.36.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.37 colormap# TOC</h3> Apply custom color maps to video stream. This filter needs three input video streams. First stream is video stream that is going to be filtered out. Second and third video stream specify color patches for source color to target color mapping. The filter accepts the following options: <dl class="table"> <dt><samp class="option">patch_size</samp></dt> <dd> Set the source and target video stream patch size in pixels. </dd> <dt><samp class="option">nb_patches</samp></dt> <dd> Set the max number of used patches from source and target video stream. Default value is number of patches available in additional video streams. Max allowed number of patches is <code class="code">64</code>. </dd> <dt><samp class="option">type</samp></dt> <dd> Set the adjustments used for target colors. Can be <code class="code">relative</code> or <code class="code">absolute</code>. Defaults is <code class="code">absolute</code>. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Set the kernel used to measure color differences between mapped colors. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dt>‘<samp class="samp">weuclidean</samp>’</dt> </dl>

Default is <code class=”code”>euclidean</code>. </dd> </dl>

 <h3 class=”section”>39.38 colormatrix# TOC</h3>

Convert color matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>src</samp></dt> <dt><samp class=”option”>dst</samp></dt> <dd> Specify the source and destination color matrix. Both values must be specified. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dd> BT.470 </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl> </dd> </dl>

For example to convert from BT.601 to SMPTE-240M, use the command: <pre class=”example-preformatted”>colormatrix=bt601:smpte240m </pre>

 <h3 class=”section”>39.39 colorspace# TOC</h3>

Convert colorspace, transfer characteristics or color primaries. Input video needs to have an even size. The filter accepts the following options: <dl class=”table”> <dd></dd> <dt><samp class=”option”>all</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>bt601-6-525</samp>’</dt> <dd> BT.601-6 525 </dd> <dt>‘<samp class=”samp”>bt601-6-625</samp>’</dt> <dd> BT.601-6 625 </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class=”option”>space</samp></dt> <dd> Specify output colorspace. The accepted values are:

<dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp
class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M
or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">ycgco</samp>’</dt>
<dd> YCgCo </dd> <dt>‘<samp class="samp">bt2020ncl</samp>’</dt>
<dd> BT.2020 with non-constant luminance </dd> </dl>

</dd> <dt><samp class="option">trc</samp></dt>
<dd> Specify output transfer characteristics. The accepted val-
ues are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> Constant gamma of 2.2 </dd> <dt>‘<samp
class="samp">gamma28</samp>’</dt> <dd> Constant gamma
of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M, BT.601-6 625 or BT.601-6 525 </dd>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">srgb</samp>’</dt>
<dd> SRGB </dd> <dt>‘<samp class="samp">iec61966-2-
1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp
class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4
 </dd> <dt>‘<samp class="samp">xvycc</samp>’</dt>
<dd> xvycc </dd> <dt>‘<samp class="samp">bt2020-
10</samp>’</dt> <dd> BT.2020 for 10-bits content
</dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt>
<dd> BT.2020 for 12-bits content </dd> </dl>

</dd> <dt><samp class="option">primaries</samp></dt>
<dd> Specify output color primaries. The accepted values are:
<dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp
class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M
or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> film </dd> <dt>‘<samp class="samp">smpte431</samp>’</dt>
<dd> SMPTE-431 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt>
<dd> SMPTE-432 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class="samp">jedec-
p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd>
</dl>

</dd> <dt><samp class=”option”>range</samp></dt>
<dd> Specify output color range. The accepted values are:
<dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt>
<dd> TV (restricted) range </dd> <dt>‘<samp
class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted)
range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt>
<dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt>
<dd> JPEG (full) range </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Specify
output color format. The accepted values are: <dl
class=”table”> <dt>‘<samp class=”samp”>yuv420p</samp>’</dt>
<dd> YUV 4:2:0 planar 8-bits </dd> <dt>‘<samp
class=”samp”>yuv420p10</samp>’</dt> <dd> YUV 4:2:0 pla-
nar 10-bits </dd> <dt>‘<samp class=”samp”>yuv420p12</samp>’</dt>
<dd> YUV 4:2:0 planar 12-bits </dd> <dt>‘<samp
class=”samp”>yuv422p</samp>’</dt> <dd> YUV 4:2:2 planar
8-bits </dd> <dt>‘<samp class=”samp”>yuv422p10</samp>’</dt>
<dd> YUV 4:2:2 planar 10-bits </dd> <dt>‘<samp
class=”samp”>yuv422p12</samp>’</dt> <dd> YUV 4:2:2 pla-
nar 12-bits </dd> <dt>‘<samp class=”samp”>yuv444p</samp>’</dt>
<dd> YUV 4:4:4 planar 8-bits </dd> <dt>‘<samp
class=”samp”>yuv444p10</samp>’</dt> <dd> YUV 4:4:4 pla-
nar 10-bits </dd> <dt>‘<samp class=”samp”>yuv444p12</samp>’</dt>
<dd> YUV 4:4:4 planar 12-bits </dd> </dl>

</dd> <dt><samp class=”option”>fast</samp></dt> <dd> Do a
fast conversion, which skips gamma/primary correction. This will take
significantly less CPU, but will be mathematically incorrect. To get output
compatible with that produced by the colormatrix filter, use fast=1.
</dd> <dt><samp class=”option”>dither</samp></dt> <dd> Specify
dithering mode. The accepted values are: <dl class=”table”>
<dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No
dithering </dd> <dt>‘<samp class=”samp”>fsb</samp>’</dt>
<dd> Floyd-Steinberg dithering </dd> </dl>

</dd> <dt><samp class=”option”>wpadapt</samp></dt> <dd> Whitepoint
adaptation mode. The accepted values are: <dl
class=”table”> <dt>‘<samp class=”samp”>bradford</samp>’</dt>
<dd> Bradford whitepoint adaptation </dd> <dt>‘<samp
class=”samp”>vonkries</samp>’</dt> <dd> von Kries white-
point adaptation </dd> <dt>‘<samp class=”samp”>identity</samp>’</dt>
<dd> identity whitepoint adaptation (i.e. no whitepoint adaptation)
</dd> </dl>

</dd> <dt><samp class=”option”>clipgamut</samp></dt> <dd> Controls
how to clip out-of-gamut colors that arise as a result of colorspace con-
version. The accepted values are: <dl class=”table”>

<dt>‘<samp class="samp">none</samp>’</dt> <dd> No clipping of out of gamut colors. </dd> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> Clips the RGB values to the [0, 1] range when building the gamma transfer LUTs. </dd> </dl>

</dd> <dt><samp class="option">iall</samp></dt> <dd> Override all input properties at once. Same accepted values as all. </dd> <dt><samp class="option">ispace</samp></dt> <dd> Override input colorspace. Same accepted values as space. </dd> <dt><samp class="option">iprimaries</samp></dt> <dd> Override input color primaries. Same accepted values as primaries. </dd> <dt><samp class="option">itrc</samp></dt> <dd> Override input transfer characteristics. Same accepted values as trc. </dd> <dt><samp class="option">irange</samp></dt> <dd> Override input color range. Same accepted values as range. </dd> </dl>

The filter converts the transfer characteristics, color space and color primaries to the specified user values. The output value, if not specified, is set to a default value based on the "all" property. If that property is also not specified, the filter will log an error. The output color range and format default to the same value as the input color range and format. The input transfer characteristics, color space, color primaries and color range should be set on the input data. If any of these are missing, the filter will log an error and no conversion will take place. For example to convert the input to SMPTE-240M, use the command: <pre class="example-preformatted">colorspace=smpte240m </pre>

 <h3 class="section">39.40 colortemperature# TOC</h3> Adjust color temperature in video to simulate variations in ambient color temperature. The filter accepts the following options: <dl class="table"> <dt><samp class="option">temperature</samp></dt> <dd> Set the temperature in Kelvin. Allowed range is from 1000 to 40000. Default value is 6500 K. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set mixing with filtered output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">39.40.1 Commands# TOC</h4> This fil-

ter supports same commands as options. <h3 class=”section”>39.41 convolution# TOC</h3>

Apply convolution of 3x3, 5x5, 7x7 or horizontal/vertical up to 49 elements. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>0m</samp></dt> <dt><samp class=”option”>1m</samp></dt> <dt><samp class=”option”>2m</samp></dt> <dt><samp class=”option”>3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed integers in <var class=”var”>square</var> mode, and from 1 to 49 odd number of signed integers in <var class=”var”>row</var> mode. </dd> <dt><samp class=”option”>0rdiv</samp></dt> <dt><samp class=”option”>1rdiv</samp></dt> <dt><samp class=”option”>2rdiv</samp></dt> <dt><samp class=”option”>3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be 1/sum of all matrix elements. </dd> <dt><samp class=”option”>0bias</samp></dt> <dt><samp class=”option”>1bias</samp></dt> <dt><samp class=”option”>2bias</samp></dt> <dt><samp class=”option”>3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. Default is 0.0. </dd> <dt><samp class=”option”>0mode</samp></dt> <dt><samp class=”option”>1mode</samp></dt> <dt><samp class=”option”>2mode</samp></dt> <dt><samp class=”option”>3mode</samp></dt> <dd> Set matrix mode for each plane. Can be <var class=”var”>square</var>, <var class=”var”>row</var> or <var class=”var”>column</var>. Default is <var class=”var”>square</var>. </dd> </dl>

 <h4 class=”subsection”>39.41.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>39.41.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sharpen: <pre class=”example-preformatted”>convolution="0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0" </pre>

Apply blur: <pre class=”example-preformatted”>convolution="1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9" </pre>

Apply edge enhance: <pre class=”example-preformatted”>convolution="0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128" </pre>

Apply edge detect: <pre class=”example-preformatted”>convolution="0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128" </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>convolution="1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0" </pre>

Apply emboss: <pre class=”example-preformatted”>convolution="-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2" </pre>

 <h3 class=”section”>39.42 convolve# TOC</h3>

Apply 2D convolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class=”option”>impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> </dl>

The <code class=”code”>convolve</code> filter also supports the framesync options. <h3 class=”section”>39.43 copy# TOC</h3>

Copy the input video source unchanged to the output. This is mainly useful for testing purposes. <h3 class=”section”>39.44 coreimage# TOC</h3> Video filtering on GPU using Apple’s CoreImage API on OSX. Hardware acceleration is based on an OpenGL context. Usually, this means it is processed by video hardware. However, software-based OpenGL implementations exist which means there is no guarantee for hardware processing. It depends on the respective OSX. There are many filters and image generators provided by Apple that come with a large variety of options. The filter has to

be referenced by its name along with its options. The coreimage filter accepts the following options: <dl class="table"> <dt><samp class="option">list_filters</samp></dt> <dd> List all available filters and generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class="example-preformatted">list_filters=true </pre>

</dd> <dt><samp class="option">filter</samp></dt> <dd> Specify all filters by their respective name and options. Use <var class="var">list_filters</var> to determine all valid filter names and options. Numerical options are specified by a float value and are automatically clamped to their respective value range. Vector and color options have to be specified by a list of space separated float values. Character escaping has to be done. A special option name <code class="code">default</code> is available to use default options for a filter. It is required to specify either <code class="code">default</code> or at least one of the filter options. All omitted options are used with their default values. The syntax of the filter string is as follows: <pre class="example-preformatted">filter=<NAME>@<OPTION>=<VALUE>[@<OPTION>=<VALUE&
</pre>

</dd> <dt><samp class="option">output_rect</samp></dt> <dd> Specify a rectangle where the output of the filter chain is copied into the input image. It is given by a list of space separated float values: <pre class="example-preformatted">output_rect=x\ y\ width\ height </pre> If not given, the output rectangle equals the dimensions of the input image. The output rectangle is automatically cropped at the borders of the input image. Negative values are valid for each component. <pre class="example-preformatted">output_rect=25\ 25\ 100\ 100 </pre> </dd> </dl>

Several filters can be chained for successive processing without GPU-HOST transfers allowing for fast processing of complex filter chains. Currently, only filters with zero (generators) or exactly one (filters) input image and one output image are supported. Also, transition filters are not yet usable as intended. Some filters generate output images with additional padding depending on the respective filter kernel. The padding is automatically removed to ensure the filter output has the same size as the input image. For image generators, the size of the output image is determined by the previous output image of the filter chain or the input image of the whole filterchain, respectively. The generators do not use the pixel information of this image to generate their output. However, the generated output is blended onto this image, resulting in partial or complete coverage of the output image. The coreimagesrc video source can be used for generating input images which are directly fed into the filter chain. By using it, provid-

ing input images by another video source or an input video is not required. <h4 class="subsection">39.44.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all filters available: <pre class="example-preformatted">coreimage=list_filters=true
</pre>

Use the CIBoxBlur filter with default options to blur an image: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default
</pre>

Use a filter chain with CISepiaTone at default values and CIVignetteEffect with its center at 100x100 and a radius of 50 pixels: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default#CIVignetteEf
100@inputRadius=50 </pre>

Use nullsrc and CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class="example-preformatted">ffmpeg -f lavfi -i nullsrc=s=100x100,coreimage=filter=CIQRCodeGenerator@inputMessage=ht
-frames:v 1 QRCode.png </pre>

 <h3 class="section">39.45 corr# TOC</h3>

Obtain the correlation between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max correlation is printed through the logging system. The filter stores the calculated correlation of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi corr -f null - </pre>

 <h3 class="section">39.46 cover_rect# TOC</h3>

Cover a rectangular object It accepts the following options: <dl class="table"> <dt><samp class="option">cover</samp></dt> <dd> Filepath of the optional cover image, needs to be in yuv420.

</dd> <dt><samp class="option">mode</samp></dt> <dd> Set covering mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> cover it by the supplied image </dd> <dt>‘<samp class="samp">blur</samp>’</dt> <dd> cover it by interpolating the surrounding pixels </dd> </dl>

Default value is <var class="var">blur</var>. </dd> </dl>

 <h4 class="subsection">39.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

 <h3 class="section">39.47 crop# TOC</h3>

Crop the input video to given dimensions. It accepts the following parameters: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dd> The width of the output video. It defaults to <code class="code">iw</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">w</samp>’ or ‘<samp class="samp">out_w</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">h, out_h</samp></dt> <dd> The height of the output video. It defaults to <code class="code">ih</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">h</samp>’ or ‘<samp class="samp">out_h</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">x</samp></dt> <dd> The horizontal position, in the input video, of the left edge of the output video. It defaults to <code class="code">(in_w-out_w)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">y</samp></dt> <dd> The vertical position, in the input video, of the top edge of the output video. It defaults to <code class="code">(in_h-out_h)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">keep_aspect</samp></dt> <dd> If set to 1 will force the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. It defaults to 0. </dd> <dt><samp class="option">exact</samp></dt> <dd> Enable exact cropping. If enabled, subsampled videos will be cropped at exact width/height/x/y as

specified and will not be rounded to nearest smaller value. It defaults to 0. </dd> </dl>

The <var class=”var”>out_w</var>, <var class=”var”>out_h</var>, <var class=”var”>x</var>, <var class=”var”>y</var> parameters are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output (cropped) width and height. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

The expression for <var class=”var”>out_w</var> may depend on the value of <var class=”var”>out_h</var>, and the expression for <var class=”var”>out_h</var> may depend on <var class=”var”>out_w</var>, but they cannot depend on <var class=”var”>x</var> and <var class=”var”>y</var>, as <var class=”var”>x</var> and <var class=”var”>y</var> are evaluated after <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. The <var class=”var”>x</var> and <var class=”var”>y</var> parameters specify the expressions for the position of the top-left corner of the output (non-cropped) area. They are evaluated for each frame. If the evaluated value is not valid, it is approximated to the nearest

valid value. The expression for <var class="var">x</var> may depend on <var class="var">y</var>, and the expression for <var class="var">y</var> may depend on <var class="var">x</var>. <h4 class="subsection">39.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Crop area with size 100x100 at position (12,34). <pre class="example-preformatted">crop=100:100:12:34 </pre>

Using named options, the example above becomes: <pre class="example-preformatted">crop=w=100:h=100:x=12:y=34 </pre>

Crop the central input area with size 100x100: <pre class="example-preformatted">crop=100:100 </pre>

Crop the central input area with size 2/3 of the input video: <pre class="example-preformatted">crop=2/3*in_w:2/3*in_h </pre>

Crop the input video central square: <pre class="example-preformatted">crop=out_w=in_h crop=in_h </pre>

Delimit the rectangle with the top-left corner placed at position 100:100 and the right-bottom corner corresponding to the right-bottom corner of the input image. <pre class="example-preformatted">crop=in_w-100:in_h-100:100:100 </pre>

Crop 10 pixels from the left and right borders, and 20 pixels from the top and bottom borders <pre class="example-preformatted">crop=in_w-2*10:in_h-2*20 </pre>

Keep only the bottom right quarter of the input image: <pre class="example-preformatted">crop=in_w/2:in_h/2:in_w/2:in_h/2 </pre>

Crop height for getting Greek harmony: <pre class="example-preformatted">crop=in_w:1/PHI*in_w </pre>

Apply trembling effect: <pre class="example-preformatted">crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(n/10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(n/7) </pre>

Apply erratic camera effect depending on timestamp: <pre class="example-preformatted">crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(t*10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(t*13) </pre>

Set x depending on the value of y: <pre class="example-preformatted">crop=in_w/2:in_h/2:y:10+10*sin(n/10) </pre>

 <h4 class="subsection">39.47.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dt><samp class="option">h, out_h</samp></dt> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set width/height of the output video and the horizontal/vertical position in the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">39.48 cropdetect# TOC</h3>

Auto-detect the crop size. It calculates the necessary cropping parameters and prints the recommended parameters via the logging system. The detected dimensions correspond to the non-black or video area of the input video according to <var class="var">mode</var>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Depending on <var class="var">mode</var> crop detection is based on either the mere black value of surrounding pixels or a combination of motion vectors and edge pixels. <dl class="table"> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Detect black pixels surrounding the playing video. For fine control use option <var class="var">limit</var>. </dd> <dt>‘<samp class="samp">mvedges</samp>’</dt> <dd> Detect the playing video by the motion vectors inside the video and scanning for edge pixels typically forming the border of a playing video. </dd> </dl>

</dd> <dt><samp class="option">limit</samp></dt> <dd> Set higher black value threshold, which can be optionally specified from nothing (0) to everything (255 for 8-bit based formats). An intensity value greater to the set value is considered non-black. It defaults to 24. You can also specify a value between 0.0 and 1.0 which will be scaled depending on the bitdepth of the pixel format. </dd> <dt><samp class="option">round</samp></dt> <dd> The value which the width/height should be divisible by. It defaults to 16. The offset is automatically adjusted to center the video. Use 2 to get only even dimensions (needed for 4:2:2 video). 16 is best when encoding to most

video codecs. </dd> <dt><samp class="option">skip</samp></dt> <dd> Set the number of initial frames for which evaluation is skipped. Default is 2. Range is 0 to INT_MAX. </dd> <dt><samp class="option">reset_count, reset</samp></dt> <dd> Set the counter that determines after how many frames cropdetect will reset the previously detected largest video area and start over to detect the current optimal crop area. Default value is 0. This can be useful when channel logos distort the video area. 0 indicates ’never reset’, and returns the largest area encountered during playback. </dd> <dt><samp class="option">mv_threshold</samp></dt> <dd> Set motion in pixel units as threshold for motion detection. It defaults to 8. </dd> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">5/255</code>, and default value for <var class="var">high</var> is <code class="code">15/255</code>. </dd> </dl>

 <h4 class="subsection">39.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Find video area surrounded by black borders: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf cropdetect,metadata=mode=print -f null - </pre>

Find an embedded video area, generate motion vectors beforehand: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf mestimate,cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

Find an embedded video area, use motion vectors from decoder: <pre class="example-preformatted">ffmpeg -flags2 +export_mvs -i file.mp4 -vf cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

 <h4 class="subsection">39.48.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">

<dt><samp class=”option”>limit</samp></dt> <dd> The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>39.49 cue# TOC</h3>

Delay video filtering until a given wallclock timestamp. The filter first passes on <samp class=”option”>preroll</samp> amount of frames, then it buffers at most <samp class=”option”>buffer</samp> amount of frames and waits for the cue. After reaching the cue it forwards the buffered frames and also any subsequent frames coming in its input. The filter can be used synchronize the output of multiple ffmpeg processes for realtime output devices like decklink. By putting the delay in the filtering chain and pre-buffering frames the process can pass on data to output almost immediately after the target wallclock timestamp is reached. Perfect frame accuracy cannot be guaranteed, but the result is good enough for some use cases. <dl class=”table”> <dt><samp class=”option”>cue</samp></dt> <dd> The cue timestamp expressed in a UNIX timestamp in microseconds. Default is 0. </dd> <dt><samp class=”option”>preroll</samp></dt> <dd> The duration of content to pass on as preroll expressed in seconds. Default is 0. </dd> <dt><samp class=”option”>buffer</samp></dt> <dd> The maximum duration of content to buffer before waiting for the cue expressed in seconds. Default is 0. </dd> </dl>

 <h3 class=”section”>39.50 curves# TOC</h3>

Apply color adjustments using curves. This filter is similar to the Adobe Photoshop and GIMP curves tools. Each component (red, green and blue) has its values defined by <var class=”var”>N</var> key points tied from each other using a smooth curve. The x-axis represents the pixel values from the input frame, and the y-axis the new pixel values to be set for the output frame. By default, a component curve is defined by the two points <var class=”var”>(0;0)</var> and <var class=”var”>(1;1)</var>. This creates a straight line where each original pixel value is "adjusted" to its own value, which means no change to the image. The filter allows you to redefine these two points and add some more. A new curve will be defined to pass smoothly through all these new coordinates. The new defined points need to be strictly increasing over the x-axis, and their <var class=”var”>x</var> and <var class=”var”>y</var> values must be in the <var class=”var”>[0;1]</var> interval. The curve is formed by using a natural or monotonic cubic

spline interpolation, depending on the <var class=”var”>interp</var> option (default: <code class=”code”>natural</code>). The <code class=”code”>natural</code> spline produces a smoother curve in general while the monotonic (<code class=”code”>pchip</code>) spline guarantees the transitions between the specified points to be monotonic. If the computed curves happened to go outside the vector spaces, the values will be clipped accordingly. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset</samp></dt> <dd> Select one of the available color presets. This option can be used in addition to the <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> parameters; in this case, the later options takes priority on the preset values. Available presets are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dt>‘<samp class=”samp”>color_negative</samp>&rs
<dt>‘<samp class=”samp”>cross_process</samp>’</dt>
<dt>‘<samp class=”samp”>darker</samp>’</dt> <dt>‘<samp class=”samp”>increase_contrast</samp>’</dt> <dt>‘<samp class=”samp”>lighter</samp>’</dt> <dt>‘<samp class=”samp”>linear_contrast</samp>&
<dt>‘<samp class=”samp”>medium_contrast</samp>’</dt>
<dt>‘<samp class=”samp”>negative</samp>’</dt> <dt>‘<samp class=”samp”>strong_contrast</samp>’</dt> <dt>‘<samp class=”samp”>vintage</samp>’</dt> </dl> Default is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>master, m</samp></dt> <dd> Set the master key points. These points will define a second pass mapping. It is sometimes called a "luminance" or "value" mapping. It can be used with <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> or <samp class=”option”>all</samp> since it acts like a post-processing LUT. </dd> <dt><samp class=”option”>red, r</samp></dt> <dd> Set the key points for the red component. </dd> <dt><samp class=”option”>green, g</samp></dt> <dd> Set the key points for the green component. </dd> <dt><samp class=”option”>blue, b</samp></dt> <dd> Set the key points for the blue component. </dd> <dt><samp class=”option”>all</samp></dt> <dd> Set the key points for all components (not including master). Can be used in addition to the other key points component options. In this case, the unset component(s) will fallback on this <samp class=”option”>all</samp> setting. </dd> <dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop curves file (<code class=”code”>.acv</code>) to import the settings from. </dd> <dt><samp class=”option”>plot</samp></dt> <dd> Save Gnuplot script of the curves in specified file. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the kind of interpolation. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>natural</samp>’</dt> <dd> Natural cubic spline using a piece-wise cubic polynomial that is twice continuously differ-

entiable. </dd> <dt>‘<samp class="samp">pchip</samp>’</dt>
<dd> Monotonic cubic spline using a piecewise cubic Hermite interpolating
polynomial (PCHIP). </dd> </dl>

</dd> </dl>

To avoid some filtergraph syntax conflicts, each key points list need
to be defined using the following syntax: <code class="code">x0/y0
x1/y1 x2/y2 ...</code>.
<h4 class="subsection">39.50.1 Commands<a
class="anchor hidden-xs" href="#Commands-79" aria-hidden="true">#
TOC</h4>

This filter supports same commands
as options. <h4 class="subsection">39.50.2
Examples<a class="anchor hidden-xs" href="#Examples-
109" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-109" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Increase slightly the middle level of blue:
 <pre class="example-preformatted">curves=blue='0/0
0.5/0.58 1/1' </pre>

Vintage effect: <pre class="example-
preformatted">curves=r='0/0.11 .42/.51 1/0.95':g='0/0 0.50/0.48 1/1':b='0/0.22
.49/.44 1/0.8' </pre> Here we obtain the following co-
ordinates for each components: <dl class="table"> <dt><var
class="var">red</var></dt> <dd> <code class="code">(0;0.11)
(0.42;0.51) (1;0.95)</code> </dd> <dt><var class="var">green</var></dt>
<dd> <code class="code">(0;0) (0.50;0.48) (1;1)</code> </dd>
<dt><var class="var">blue</var></dt> <dd> <code class="code">(0;0.22)
(0.49;0.44) (1;0.80)</code> </dd> </dl>

The previous example can also be achieved with the associ-
ated built-in preset: <pre class="example-
preformatted">curves=preset=vintage </pre>

Or simply: <pre class="example-
preformatted">curves=vintage </pre>

Use a Photoshop preset and redefine the points of the green compo-
nent: <pre class="example-preformatted">curves=psfile='MyCurvesPresets/purple.a
0.45/0.53 1/1' </pre>

Check out the curves of the <code class="code">cross_process</code>
profile using <code class="command">ffmpeg</code> and <code class="command">gnuplot</code>:
 <pre class="example-preformatted">ffmpeg -f lavfi
-i color -vf curves=cross_process:plot=/tmp/curves.plt -frames:v 1 -f null -
gnuplot -p /tmp/curves.plt </pre>

 <h3 class="section">39.51 datascope# TOC</h3>

Video data analysis filter. This filter shows hexadecimal pixel values of part of video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set output video size. </dd> <dt><samp class="option">x</samp></dt> <dd> Set x offset from where to pick pixels. </dd> <dt><samp class="option">y</samp></dt> <dd> Set y offset from where to pick pixels. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set scope mode, can be one of the following: <dl class="table"> <dt>‘<samp class="samp">mono</samp>’</dt> <dd> Draw hexadecimal pixel values with white color on black background. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Draw hexadecimal pixel values with input video pixel color on black background. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Draw hexadecimal pixel values on color background picked from input video, the text color is picked in such way so its always visible. </dd> </dl>

</dd> <dt><samp class="option">axis</samp></dt> <dd> Draw rows and columns numbers on left and top of video. </dd> <dt><samp class="option">opacity</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">format</samp></dt> <dd> Set display number format. Can be <code class="code">hex</code>, or <code class="code">dec</code>. Default is <code class="code">hex</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set pixel components to display. By default all pixel components are displayed. </dd> </dl>

 <h4 class="subsection">39.51.1 Commands# TOC</h4>

This filter supports same commands as options excluding <code class="code">size</code> option. <h3 class="section">39.52 dblur# TOC</h3>
Apply Directional blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> Set angle of directional blur. Default is <code class="code">45</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Set radius of directional blur. Default is <code class="code">5</code>.

</dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> </dl>

 <h4 class=”subsection”>39.52.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>39.53 dctdnoiz# TOC</h3>

Denoise frames using 2D DCT (frequency domain filtering). This filter is not designed for real time. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma, s</samp></dt> <dd> Set the noise sigma constant. This <var class=”var”>sigma</var> defines a hard threshold of <code class=”code”>3 * sigma</code>; every DCT coefficient (absolute value) below this threshold with be dropped. If you need a more advanced filtering, see <samp class=”option”>expr</samp>. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set number overlapping pixels for each block. Since the filter can be slow, you may want to reduce this value, at the cost of a less effective filter and the risk of various artefacts. If the overlapping value doesn’t permit processing the whole input width or height, a warning will be displayed and according borders won’t be denoised. Default value is <var class=”var”>blocksize</var>-1, which is the best possible setting. </dd> <dt><samp class=”option”>expr, e</samp></dt> <dd> Set the coefficient factor expression. For each coefficient of a DCT block, this expression will be evaluated as a multiplier value for the coefficient. If this is option is set, the <samp class=”option”>sigma</samp> option will be ignored. The absolute value of the coefficient can be accessed through the <var class=”var”>c</var> variable. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Set the <var class=”var”>blocksize</var> using the number of bits. <code class=”code”>1<<<var class=”var”>n</var></code> defines the <var class=”var”>blocksize</var>, which is the width and height of the processed blocks. The default value is <var class=”var”>3</var> (8x8) and can be raised to <var class=”var”>4</var> for a <var class=”var”>blocksize</var> of 16x16. Note that changing this setting has huge consequences on the speed processing. Also, a larger block size does not necessarily means a better de-noising. </dd> </dl>

 <h4 class="subsection">39.53.1 Examples# TOC</h4>

Apply a denoise with a <samp class="option">sigma</samp> of <code class="code">4.5</code>: <pre class="example-preformatted">dctdnoiz=4.5 </pre>

The same operation can be achieved using the expression system: <pre class="example-preformatted">dctdnoiz=e='gte(c, 4.5*3)' </pre>

Violent denoise using a block size of <code class="code">16x16</code>: <pre class="example-preformatted">dctdnoiz=15:n=4 </pre>

 <h3 class="section">39.54 deband# TOC</h3>

Remove banding artifacts from input video. It works by replacing banded pixels with average value of referenced pixels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">1thr</samp></dt> <dt><samp class="option">2thr</samp></dt> <dt><samp class="option">3thr</samp></dt> <dt><samp class="option">4thr</samp></dt> <dd> Set banding detection threshold for each plane. Default is 0.02. Valid range is 0.00003 to 0.5. If difference between current pixel and reference pixel is less than threshold, it will be considered as banded. </dd> <dt><samp class="option">range, r</samp></dt> <dd> Banding detection range in pixels. Default is 16. If positive, random number in range 0 to set value will be used. If negative, exact absolute value will be used. The range defines square of four pixels around current pixel. </dd> <dt><samp class="option">direction, d</samp></dt> <dd> Set direction in radians from which four pixel will be compared. If positive, random direction from 0 to set direction will be picked. If negative, exact of absolute value will be picked. For example direction 0, -PI or -2*PI radians will pick only pixels on same row and -PI/2 will pick only pixels on same column. </dd> <dt><samp class="option">blur, b</samp></dt> <dd> If enabled, current pixel is compared with average value of all four surrounding pixels. The default is enabled. If disabled current pixel is compared with all four surrounding pixels. The pixel is considered banded if only all four differences with surrounding pixels are less than threshold. </dd> <dt><samp class="option">coupling, c</samp></dt> <dd> If enabled, current pixel is changed if and only if all pixel components are banded, e.g. banding detection threshold is triggered for all color components. The default is disabled. </dd> </dl>

 <h4 class="subsection">39.54.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.55 deblock# TOC</h3>

Remove blocking artifacts from input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filter</samp></dt> <dd> Set filter type, can be <var class="var">weak</var> or <var class="var">strong</var>. Default is <var class="var">strong</var>. This controls what kind of deblocking is applied. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block, allowed range is from 4 to 512. Default is <var class="var">8</var>. </dd> <dt><samp class="option">alpha</samp></dt> <dt><samp class="option">beta</samp></dt> <dt><samp class="option">gamma</samp></dt> <dt><samp class="option">delta</samp></dt> <dd> Set blocking detection thresholds. Allowed range is 0 to 1. Defaults are: <var class="var">0.098</var> for <var class="var">alpha</var> and <var class="var">0.05</var> for the rest. Using higher threshold gives more deblocking strength. Setting <var class="var">alpha</var> controls threshold detection at exact edge of block. Remaining options controls threshold detection near the edge. Each one for below/above or left/right. Setting any of those to <var class="var">0</var> disables deblocking. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is to filter all available planes. </dd> </dl>

 <h4 class="subsection">39.55.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Deblock using weak filter and block size of 4 pixels. <pre class="example-preformatted">deblock=filter=weak:block=4 </pre>

Deblock using strong filter, block size of 4 pixels and custom thresholds for deblocking more edges. <pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta=0.05
</pre>

Similar as above, but filter only first plane.
<pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta
</pre>

Similar as above, but filter only second and third plane. <pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.0
</pre>

 <h4 class="subsection">39.55.2 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">39.56 decimate# TOC</h3>

Drop duplicated frames at regular intervals. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cycle</samp></dt> <dd> Set the number of frames from which one will be dropped. Setting this to <var class="var">N</var> means one frame in every batch of <var class="var">N</var> frames will be dropped. Default is <code class="code">5</code>. </dd> <dt><samp class="option">dupthresh</samp></dt> <dd> Set the threshold for duplicate detection. If the difference metric for a frame is less than or equal to this value, then it is declared as duplicate. Default is <code class="code">1.1</code> </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set scene change threshold. Default is <code class="code">15</code>. </dd> <dt><samp class="option">blockx</samp></dt> <dt><samp class="option">blocky</samp></dt> <dd> Set the size of the x and y-axis blocks used during metric calculations. Larger blocks give better noise suppression, but also give worse detection of small movements. Must be a power of two. Default is <code class="code">32</code>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark main input as a pre-processed input and activate clean source input stream. This allows the input to be pre-processed with various filters to help the metrics calculation while keeping the frame selection lossless. When set to <code class="code">1</code>, the first stream is for the pre-processed input, and the second stream is the clean source from where the kept frames are chosen. Default is <code class="code">0</code>. </dd> <dt><samp class="option">chroma</samp></dt> <dd> Set whether or not chroma is considered in the metric calculations. Default is <code class="code">1</code>. </dd> <dt><samp class="option">mixed</samp></dt> <dd> Set whether or not the input only partially contains content to be decimated. Default is <code class="code">false</code>. If enabled video output stream will be in variable frame rate. </dd> </dl>

 <h3 class="section">39.57 deconvolve<span

class=”pull-right”># TOC</h3>

Apply 2D deconvolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class=”option”>impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> <dt><samp class=”option”>noise</samp></dt> <dd> Set noise when doing divisions. Default is <var class=”var”>0.0000001</var>. Useful when width and height are not same and not power of 2 or if stream prior to convolving had noise. </dd> </dl>

The <code class=”code”>deconvolve</code> filter also supports the framesync options. <h3 class=”section”>39.58 dedot# TOC</h3>

Reduce cross-luminance (dot-crawl) and cross-color (rainbows) from video. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>m</samp></dt> <dd> Set mode of operation. Can be combination of <var class=”var”>dotcrawl</var> for cross-luminance reduction and/or <var class=”var”>rainbows</var> for cross-color reduction. </dd> <dt><samp class=”option”>lt</samp></dt> <dd> Set spatial luma threshold. Lower values increases reduction of cross-luminance. </dd> <dt><samp class=”option”>tl</samp></dt> <dd> Set tolerance for temporal luma. Higher values increases reduction of cross-luminance. </dd> <dt><samp class=”option”>tc</samp></dt> <dd> Set tolerance for chroma temporal variation. Higher values increases reduction of cross-color. </dd> <dt><samp class=”option”>ct</samp></dt> <dd> Set temporal chroma threshold. Lower values increases reduction of cross-color. </dd> </dl>

 <h3 class=”section”>39.59 deflate# TOC</h3>

Apply deflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values lower than the pixel. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt>

<dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">39.59.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.60 deflicker# TOC</h3>

Remove temporal frame luminance variations. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set moving-average filter size in frames. Default is 5. Allowed range is 2 - 129. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set averaging mode to smooth temporal luminance variations. Available values are: <dl class="table"> <dt>‘<samp class="samp">am</samp>’</dt> <dd> Arithmetic mean </dd> <dt>‘<samp class="samp">gm</samp>’</dt> <dd> Geometric mean </dd> <dt>‘<samp class="samp">hm</samp>’</dt> <dd> Harmonic mean </dd> <dt>‘<samp class="samp">qm</samp>’</dt> <dd> Quadratic mean </dd> <dt>‘<samp class="samp">cm</samp>’</dt> <dd> Cubic mean </dd> <dt>‘<samp class="samp">pm</samp>’</dt> <dd> Power mean </dd> <dt>‘<samp class="samp">median</samp>’</dt> <dd> Median </dd> </dl>

</dd> <dt><samp class="option">bypass</samp></dt> <dd> Do not actually modify frame. Useful when one only wants metadata. </dd> </dl>

 <h3 class="section">39.61 deinterlace_d3d12# TOC</h3>

Deinterlace the input video using D3D12 hardware-accelerated video processing. This filter uses the DirectX 12 Video Processor to perform deinterlacing on GPU, supporting both simple bob and driver-defined advanced (motion-adaptive) algorithms. It requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter automatically queries the hardware for the required number of temporal reference frames (if needed) and manages a frame queue to supply them. It accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> The deinterlacing method. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Select

the best deinterlacing method available on the hardware. If the driver supports <code class=”code”>custom</code>, that method is used; otherwise falls back to <code class=”code”>bob</code>. This is the default. </dd> <dt>‘<samp class=”samp”>bob</samp>’</dt> <dd> Bob deinterlacing. Each field is independently scaled to full frame height. Simple and fast, but may produce visible bob artifacts on moving content. </dd> <dt>‘<samp class=”samp”>custom</samp>’</dt> <dd> Driver-defined advanced deinterlacing. The exact algorithm is hardware-specific and typically employs motion-adaptive techniques with temporal reference frames for higher quality. </dd> </dl>

The default value is <code class=”code”>default</code>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Specify the interlacing mode. It accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Send one frame for each frame. The output frame rate equals the input frame rate. This is the default. </dd> <dt>‘<samp class=”samp”>field</samp>’</dt> <dd> Send one frame for each field. The output frame rate is double the input frame rate. </dd> </dl>

The default value is <code class=”code”>frame</code>. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. It accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames. This is the default. </dd> <dt>‘<samp class=”samp”>interlaced</samp>’</dt> <dd> Only deinterlace frames marked as interlaced; progressive frames are passed through unchanged. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h4 class=”subsection”>39.61.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Deinterlace an interlaced video using software decoding and the best available hardware deinterlacing method: <pre class=”example-preformatted”>ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=method=default,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

Deinterlace at field rate (double the frame rate) using bob method with hardware decoding (full d3d12 pipeline): <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format

d3d12 -i input.ts \ -vf "deinterlace_d3d12=method=bob:mode=field"
\ -c:v h264_d3d12va output.mp4 </pre>

Deinterlace only interlaced frames, passing progressive frames through: <pre class="example-preformatted">ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=deint=interlaced,hwdownload,format=nv12"
\ -c:v libx264 -crf 18 output.mp4 </pre>

 <h3 class="section">39.62 dejudder# TOC</h3>

Remove judder produced by partially interlaced telecined content. Judder can be introduced, for instance, by pullup filter. If the original source was partially telecined content then the output of <code class="code">pullup,dejudder</code> will have a variable frame rate. May change the recorded frame rate of the container. Aside from that change, this filter will not affect constant frame rate video. The option available in this filter is: <dl class="table"> <dt><samp class="option">cycle</samp></dt> <dd> Specify the length of the window over which the judder repeats. Accepts any integer greater than 1. Useful values are: <dl class="table"> <dt>‘<samp class="samp">4</samp>’</dt> <dd> If the original was telecined from 24 to 30 fps (Film to NTSC). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> If the original was telecined from 25 to 30 fps (PAL to NTSC). </dd> <dt>‘<samp class="samp">20</samp>’</dt> <dd> If a mixture of the two. </dd> </dl>

The default is ‘<samp class="samp">4</samp>’. </dd> </dl>

 <h3 class="section">39.63 delogo# TOC</h3>

Suppress a TV station logo by a simple interpolation of the surrounding pixels. Just set a rectangle covering the logo and watch it disappear (and sometimes something even uglier appear - your mileage may vary). It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the top left corner coordinates of the logo. They must be specified. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Specify the width and height of the logo to clear. They must be

specified. </dd> <dt><samp class="option">show</samp></dt> <dd> When set to 1, a green rectangle is drawn on the screen to simplify finding the right <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, and <var class="var">h</var> parameters. The default value is 0. The rectangle is drawn on the outermost pixels which will be (partly) replaced with interpolated values. The values of the next pixels immediately outside this rectangle in each direction will be used to compute the interpolated pixel values inside the rectangle. </dd> </dl>

 <h4 class="subsection">39.63.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set a rectangle covering the area with top left corner coordinates 0,0 and size 100x77: <pre class="example-preformatted">delogo=x=0:y=0:w=100:h=77 </pre>

 <h3 class="section">39.64 derain# TOC</h3>

Remove the rain in the input image/video by applying the derain methods based on convolutional neural networks. Supported models: <ul class="itemize mark-bullet"> Recurrent Squeeze-and-Excitation Context Aggregation Net (RESCAN). See http://openaccess.thecvf.com/content_ECCV_2018/pap and-Excitation_Context_ECCV_2018_paper.pdf.

Training as well as model generation scripts are provided in the repository at <a class="url" href="https://github.com/XueweiMeng/derain_filter.git">https://github.com/XueweiMeng/d The filter accepts the following options: <dl class="table"> <dt><samp class="option">filter_type</samp></dt> <dd> Specify which filter to use. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">derain</samp>’</dt> <dd> Derain filter. To conduct derain filter, you need to use a derain model. </dd> <dt>‘<samp class="samp">dehaze</samp>’</dt> <dd> Dehaze filter. To conduct dehaze filter, you need to use a dehaze model. </dd> </dl> Default value is ‘<samp class="samp">derain</samp>’. </dd> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow back-

end. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow can load files for only its format. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>39.65 deshake# TOC</h3>

Attempt to fix small changes in horizontal and/or vertical shift. This filter helps remove camera shake from hand-holding a camera, bumping a tripod, moving on a vehicle, etc. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Specify a rectangular area where to limit the search for motion vectors. If desired the search for motion vectors can be limited to a rectangular area of the frame defined by its top left corner, width and height. These parameters have the same meaning as the drawbox filter which can be used to visualise the position of the bounding box. This is useful when simultaneous movement of subjects within the frame might be confused for camera motion by the motion vector search. If any or all of <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> are set to -1 then the full frame is used. This allows later options to be set without specifying the bounding box for the motion vector search. Default - search the whole frame. </dd> <dt><samp class=”option”>rx</samp></dt> <dt><samp class=”option”>ry</samp></dt> <dd> Specify the maximum extent of movement in x and y directions in the range 0-64 pixels. Default 16. </dd> <dt><samp class=”option”>edge</samp></dt> <dd> Specify how to generate pixels to fill blanks at the edge of the frame. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>blank, 0</samp>’</dt> <dd> Fill zeroes at blank locations </dd> <dt>‘<samp class=”samp”>original, 1</samp>’</dt> <dd> Original image at blank locations </dd> <dt>‘<samp class=”samp”>clamp, 2</samp>’</dt> <dd> Extruded edge value at blank locations </dd> <dt>‘<samp class=”samp”>mirror, 3</samp>’</dt> <dd> Mirrored edge at blank locations </dd> </dl> Default value is ‘<samp class=”samp”>mirror</samp>’.

 </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Specify the blocksize to use for motion search. Range 4-128 pixels, default 8. </dd> <dt><samp class="option">contrast</samp></dt> <dd> Specify the contrast threshold for blocks. Only blocks with more than the specified contrast (difference between darkest and lightest pixels) will be considered. Range 1-255, default 125. </dd> <dt><samp class="option">search</samp></dt> <dd> Specify the search strategy. Available values are: <dl class="table"> <dt>‘<samp class="samp">exhaustive, 0</samp>’</dt> <dd> Set exhaustive search </dd> <dt>‘<samp class="samp">less, 1</samp>’</dt> <dd> Set less exhaustive search. </dd> </dl> Default value is ‘<samp class="samp">exhaustive</samp>’. </dd> <dt><samp class="option">filename</samp></dt> <dd> If set then a detailed log of the motion search is written to the specified file. </dd> </dl>

 <h3 class="section">39.66 despill# TOC</h3>

Remove unwanted contamination of foreground colors, caused by reflected color of greenscreen or bluescreen. This filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Set what type of despill to use. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how spillmap will be generated. </dd> <dt><samp class="option">expand</samp></dt> <dd> Set how much to get rid of still remaining spill. </dd> <dt><samp class="option">red</samp></dt> <dd> Controls amount of red in spill area. </dd> <dt><samp class="option">green</samp></dt> <dd> Controls amount of green in spill area. Should be -1 for greenscreen. </dd> <dt><samp class="option">blue</samp></dt> <dd> Controls amount of blue in spill area. Should be -1 for bluescreen. </dd> <dt><samp class="option">brightness</samp></dt> <dd> Controls brightness of spill area, preserving colors. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Modify alpha from generated spillmap. </dd> </dl>

 <h4 class="subsection">39.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.67 detelecine# TOC</h3>

Apply an exact inverse of the telecine operation. It requires a predefined pattern specified using the pattern option which must be the same as that passed to the telecine filter. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> A number representing position of the first frame with respect to the telecine pattern. This is to be used if the stream is cut. The default value is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>39.68 dilation# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>39.68.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.69 displace# TOC</h3>

Displace pixels as indicated by second and third input stream. It takes three input streams and outputs one stream, the first input is the source, and second and third input are displacement maps. The second input specifies how much to displace pixels along the x-axis, while the third

input specifies how much to displace pixels along the y-axis. If one of displacement map streams terminates, last frame from that displacement map will be used. Note that once generated, displacements maps can be reused over and over again. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">edge</samp></dt> <dd> Set displace behavior for pixels that are out of range. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank</samp>’</dt> <dd> Missing pixels are replaced by black pixels. </dd> <dt>‘<samp class="samp">smear</samp>’</dt> <dd> Adjacent pixels will spread out to replace missing pixels. </dd> <dt>‘<samp class="samp">wrap</samp>’</dt> <dd> Out of range pixels are wrapped so they point to pixels of other side. </dd> <dt>‘<samp class="samp">mirror</samp>’</dt> <dd> Out of range pixels will be replaced with mirrored pixels. </dd> </dl> Default is ‘<samp class="samp">smear</samp>’. </dd> </dl>

 <h4 class="subsection">39.69.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add ripple effect to rgb input of video size hd720: <pre class="example-preformatted">ffmpeg -i INPUT -f lavfi -i nullsrc=s=hd720,lutrgb=128:128:128 -f lavfi -i nullsrc=s=hd720,geq='r=128+30*sin(2*PI*X/400+T):g=128+30*sin(2*PI*X/400+T):b=128+30*sin -lavfi '[0][1][2]displace' OUTPUT </pre>

Add wave effect to rgb input of video size hd720: <pre class="example-preformatted">ffmpeg -i INPUT -f lavfi -i nullsrc=hd720,geq='r=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):g=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):b=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T))' -lavfi '[1]split[x][y],[0][x][y]displace' OUTPUT </pre>

 <h3 class="section">39.70 dnn_classify# TOC</h3>

Do classification with deep neural networks based on bounding boxes. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts only openvino now, tensorflow backends will be added. </dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and

its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0, and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend For tensorflow backend, you can set its configs with <samp class=”option”>sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs for your system. </dd> </dl>

 <h3 class=”section”>39.71 dnn_detect# TOC</h3>

Do object detection with deep neural networks. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt> <dd> OpenVINO backend. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. The TensorFlow and ONNX Runtime backends accept multiple output names separated by ‘<samp class=”samp”>&</samp>’, e.g. <samp class=”option”>output=yolo_13&yolo_26</samp>. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0 (usually it is ’background’),

and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>model_type</samp></dt> <dd> Set the detection model output format. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>ssd</samp>’</dt> <dd> Single-stage detector (default). </dd> <dt>‘<samp class=”samp”>yolo</samp>’</dt> <dd> YOLO v1/v2. </dd> <dt>‘<samp class=”samp”>yolov3</samp>’</dt> <dd> YOLOv3. </dd> <dt>‘<samp class=”samp”>yolov4</samp>’</dt> <dd> YOLOv4. </dd> </dl> </dd> <dt><samp class=”option”>anchors</samp></dt> <dd> Anchor box dimensions, separated by ‘<samp class=”samp”>&</samp>’. Required for YOLO-family models. The list is consumed in output order, so the anchors of the first output come first: with <samp class=”option”>output=yolo_13&yolo_26</samp> the leading pairs belong to <code class=”code”>yolo_13</code>. </dd> <dt><samp class=”option”>nb_classes</samp></dt> <dd> Number of detection classes. Required for YOLO-family models. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. </dd> </dl>

 <h4 class=”subsection”>39.71.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Run SSD object detection with an ONNX model (single output, DirectML on Windows): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=300:300,format=rgb24,dnn_detect=
:model=ssd.onnx:model_type=ssd:confidence=0.5:device=dml" -f null -
</pre>

Run YOLOv3 object detection with an ONNX model (two outputs, CPU): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=416:416,format=rgb24,dnn_detect=dnn_backend=onnx
:model=yolov3.onnx:output=yolo_13&yolo_26:model_type=yolov3:nb_classes=80
:anchors=116&90&156&198&373&326&30&61&62&45&59&
-f null - </pre>

 <h3 class=”section”>39.72 dnn_processing# TOC</h3>

Do image processing with deep neural networks. It works together with another filter which converts the pixel format of the Frame to what the dnn network requires. The filter accepts the following options: <dl

class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt>
<dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>tensorflow</samp>’</dt>
<dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www
and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code>
 </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt>
<dd> OpenVINO backend. To enable this backend you need to build and install the OpenVINO for C library (see https://github.com/openvinotoolkit/openvino/blob/master/build-instruction.md) and configure FFmpeg with <code class=”code”>--enable-libopenvino</code> (–extra-cflags=-I... –extra-ldflags=-L... might be needed if the header files and libraries are not installed into system path) </dd> <dt>‘<samp class=”samp”>torch</samp>’</dt>
<dd> Libtorch backend. To enable this backend you need to build and install Libtorch for C++ library. Please download cxx11 ABI version (see https://pytorch.org/get-started/locally) and configure FFmpeg with <code class=”code”>--enable-libtorch --extra-cflags=-I/libtorch_root/libtorch/include --extra-cflags=-I/libtorch_root/libtorch/include/torch/csrc/api/include --extra-ldflags=-L/libtorch_root/libtorch/lib/</code> When FFmpeg is built with CUDA support and the input frames use <code class=”code”>AV_PIX_FMT_CUDA</code> with an underlying pixel format of <code class=”code”>rgb0</code>, <code class=”code”>bgr0</code>, <code class=”code”>rgb24</code>, or <code class=”code”>bgr24</code>, the Torch backend maps GPU frame buffers directly into LibTorch CUDA tensors using zero-copy memory mapping. This avoids host-device PCIe transfers and can significantly reduce inference latency. The zero-copy path is selected automatically when the input format qualifies; no additional options are required. When the input format is not <code class=”code”>AV_PIX_FMT_CUDA</code>, the backend uses the standard CPU tensor path. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. To enable this backend you need to install the ONNX Runtime library (see https://onnxruntime.ai/) and configure FFmpeg with <code class=”code”>--enable-libonnxruntime</code>.
 The current ONNX Runtime backend expects 4-D input and output tensors with NCHW layout and 32-bit floating-point element type (ONNX <code class=”code”>FLOAT</code>); models with integer or other element types (e.g. <code class=”code”>UINT8</code>) are not supported and will be rejected at load time. Models using NHWC layout or other ranks are not yet supported. Only single-input models are supported; the backend binds exactly one input tensor when running the model. The <samp class=”option”>input</samp>

and <samp class=”option”>output</samp> options are optional for the ONNX Runtime backend; when they are omitted the backend resolves the tensor names from the session. Multiple output names may be supplied separated by ‘<samp class=”samp”>&</samp>’ (e.g. <samp class=”option”>output=out_a&out_b</samp>); however for <code class=”code”>dnn_processing</code> only the first output tensor is used for frame post-processing. The ONNX Runtime backend runs inference synchronously using a single inference request. The shared <samp class=”option”>async</samp> and <samp class=”option”>nireq</samp> options therefore have no effect for <code class=”code”>dnn_backend=onnx</code>; inference always runs synchronously regardless of their values. When FFmpeg is built with Direct3D 12 support and this build’s ONNX Runtime provides a working DirectML execution provider, setting <samp class=”option”>device=dml</samp> together with <code class=”code”>AV_PIX_FMT_D3D12</code> NV12 hardware frames as input (e.g. via <code class=”code”>-hwaccel d3d12va -hwaccel_output_format d3d12</code>) keeps frames in VRAM for the whole filter, from the decoder’s output through DNN inference and back out, without a host round-trip. Software-frame input retains the host I/O path. D3D12 input requires successful DirectML negotiation; unsupported hardware-input configurations fail during configuration. At runtime, <samp class=”file”>DirectML.dll</samp> and <samp class=”file”>d3dcompiler_47.dll</samp> must be loadable from the application directory or System32. Currently, only single-channel (luma-only), <code class=”code”>FLOAT</code>/NCHW models with a <code class=”code”>[1,1,H,W]</code> output tensor and even input/output frame dimensions are supported; the chroma plane is copied or bilinearly upscaled on the GPU without going through the model. The output frames are D3D12 hardware frames, so they must either be consumed by a D3D12 encoder or downloaded explicitly with <code class=”code”>hwdownload,format=nv12</code>. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO, Libtorch, and ONNX Runtime backends can load files for only their respective formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. For tensorflow backend, you can set its configs

with <samp class="option">sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. </dd> <dt><samp class="option">async</samp></dt> <dd> Use DNN async inference (default: 1). When enabled, frames are submitted to the backend without blocking the caller. Supported by the Torch, OpenVINO, and TensorFlow backends; the ONNX Runtime backend ignores this option. </dd> <dt><samp class="option">nireq</samp></dt> <dd> Number of concurrent inference requests (default: 0, meaning auto). When set to 0, the backend automatically determines an appropriate number of requests. Higher values increase throughput by allowing multiple frames to be in flight simultaneously. </dd> <dt><samp class="option">batch_size</samp></dt> <dd> Number of frames to batch into a single inference request (default: 1). Values greater than 1 cause the backend to accumulate frames and concatenate them along the batch dimension before running the model. This can significantly increase throughput on GPUs at the cost of slightly higher per-frame latency. Currently supported by the OpenVINO and Torch backends. </dd> <dt><samp class="option">device</samp></dt> <dd> Set the device to run the model. For the ONNX Runtime backend this selects the execution provider: <code class="code">cpu</code> (default), <code class="code">cuda</code> (NVIDIA GPU), <code class="code">dml</code> (DirectML, Windows only) or <code class="code">vitisai</code> (AMD Ryzen AI NPU). </dd> <dt><samp class="option">device_id</samp></dt> <dd> Set the device index used by GPU execution providers (e.g. <code class="code">cuda</code> or <code class="code">dml</code>) for the ONNX Runtime backend. Default is 0. </dd> <dt><samp class="option">threads_per_operation</samp></dt> <dd> ONNX Runtime backend only. Set the number of CPU threads used per ONNX Runtime operator when running with <code class="code">device=cpu</code>. Default is 0 (let ONNX Runtime choose automatically). Has no effect for GPU/NPU providers. </dd> </dl>

 <h4 class="subsection">39.72.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Remove rain in rgb24 frame with can.pb (see derain filter): <pre class="example-preformatted">./ffmpeg -i rain.jpg -vf format=rgb24,dnn_processing=dnn_backend=tensorflow:model=can.pb:input=x:output=y derain.jpg </pre>

Handle the Y channel with srcnn.pb (see sr filter) for frame with yuv420p (planar YUV formats supported): <pre class="example-preformatted">./ffmpeg -i 480p.jpg -vf format=yuv420p,scale=w=iw*2:h=ih*2,dnn_processing=dnn_backend=tensorflow:model=srcn

-y srcnn.jpg </pre>

Handle the Y channel with espcn.pb (see sr filter), which changes frame size, for format yuv420p (planar YUV formats supported), please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. <pre class="example-preformatted">./ffmpeg -i 480p.jpg -vf format=yuv420p,dnn_processing=dnn_backend=
-y tmp.espcn.jpg </pre>

Perform zero-copy GPU inference with the Libtorch backend. Because there is currently no GPU-side filter to convert from YUV to RGB, <code class="code">hwupload</code> is used to upload CPU RGB frames into device memory. Once a GPU-side YUV-to-RGB filter becomes available, the <code class="code">hwupload</code> step will no longer be necessary and all frames can stay in device memory throughout the pipeline: <pre class="example-preformatted">./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda -i input.mp4 \ -vf "format=rgb24,hwupload,dnn_processing=dnn_backend=torch:model=my_model.pt,hwdownload,form
\ -y output.mp4 </pre>

Run a single-channel (luma-only) super-resolution model on D3D12 hardware frames with the ONNX Runtime backend, keeping frames in VRAM from decode through inference (Windows only, requires a build with Direct3D 12 support and <code class="code">--enable-libonnxruntime</code> against a DirectML-capable ONNX Runtime). The D3D12 output frames are encoded directly by a D3D12 hardware encoder: <pre class="example-preformatted">./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml
\ -c:v h264_d3d12va -bf 0 -y output.mp4 </pre>

Same as above, but download the result to software frames after inference so that a software encoder can be used: <pre class="example-preformatted">./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml,hwdownloa
\ -y output.mp4 </pre>

Process rgb24 frames with a TorchScript model using the Libtorch backend: <pre class="example-preformatted">./ffmpeg -i input.jpg -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_model.pt output.jpg </pre>

Process rgb24 frames with the Libtorch backend using batch inference (four frames per request): <pre class="example-preformatted">./ffmpeg -i input.mp4 -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_mo
-y output.mp4 </pre>

 <h3 class="section">39.73 drawbox<span

class=”pull-right”># TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the box edge color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class=”code”>fill</code> will create a filled box. Default value is <code class=”code”>3</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With value <code class=”code”>1</code>, the pixels of the painted box will overwrite the video’s color and alpha pixels. Default is <code class=”code”>0</code>, which composites the box onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> and <var class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class=”option”>box_source</samp></dt>

<dd> Box source can be set as side_data_detection_bboxes if you want to use box data in detection bboxes of side data. If <var class="var">box_source</var> is set, the <var class="var">x</var>, <var class="var">y</var>, <var class="var">width</var> and <var class="var">height</var> will be ignored and still use box data in detection bboxes of side data. So please do not use this parameter if you were not sure about the box source. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. These constants allow the <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, <var class="var">h</var> and <var class="var">t</var> expressions to refer to each other, so you may for example specify <code class="code">y=x/dar</code> or <code class="code">h=w/dar</code>. </dd> </dl>

 <h4 class="subsection">39.73.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h4 class="subsection">39.73.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.74 drawgraph# <a class="anchor

hidden-xs”href=”#toc-drawgraph” aria-hidden=”true”>TOC</h3>
Draw a graph using input video metadata. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>m1</samp></dt>
<dd> Set 1st frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class=”option”>fg1</samp></dt>
<dd> Set 1st foreground color expression. </dd> <dt><samp class=”option”>m2</samp></dt> <dd> Set 2nd frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class=”option”>fg2</samp></dt> <dd> Set 2nd foreground color expression. </dd> <dt><samp class=”option”>m3</samp></dt>
<dd> Set 3rd frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class=”option”>fg3</samp></dt>
<dd> Set 3rd foreground color expression. </dd> <dt><samp class=”option”>m4</samp></dt> <dd> Set 4th frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class=”option”>fg4</samp></dt> <dd> Set 4th foreground color expression. </dd> <dt><samp class=”option”>min</samp></dt>
<dd> Set minimal value of metadata value. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Set maximal value of metadata value. </dd> <dt><samp class=”option”>bg</samp></dt>
<dd> Set graph background color. Default is white. </dd>
<dt><samp class=”option”>mode</samp></dt> <dd> Set graph mode. Available values for mode is: <dl class=”table”>
<dt>‘<samp class=”samp”>bar</samp>’</dt> <dt>‘<samp class=”samp”>dot</samp>’</dt> <dt>‘<samp class=”samp”>line</samp>’</dt>
</dl>

Default is <code class=”code”>line</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide mode.
Available values for slide is: <dl class=”table”> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Draw new frame when right border is reached. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt
<dd> Replace old columns with new ones. </dd> <dt>‘<samp class=”samp”>scroll</samp>’</dt> <dd> Scroll from right to left. </dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> Scroll from left to right. </dd> <dt>‘<samp class=”samp”>picture</samp>’</dt> <dd> Draw single picture.
</dd> </dl>

Default is <code class=”code”>frame</code>. </dd>
<dt><samp class=”option”>size</samp></dt> <dd> Set size of graph video. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. The default value is <code class=”code”>900x256</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class=”code”>25</code>. The foreground color expressions

can use the following variables: <dl class="table"> <dt><samp class="option">MIN</samp></dt> <dd> Minimal value of metadata value. </dd> <dt><samp class="option">MAX</samp></dt> <dd> Maximal value of metadata value. </dd> <dt><samp class="option">VAL</samp></dt> <dd> Current metadata key value. </dd> </dl>

The color is defined as 0xAABBGGRR. </dd> </dl>

Example using metadata from signalstats filter: <pre class="example-preformatted">signalstats,drawgraph=lavfi.signalst </pre>

Example using metadata from ebur128 filter: <pre class="example-preformatted">ebur128=metadata=1,adrawgraph= 120:max=5 </pre>

 <h3 class="section">39.75 drawgrid# TOC</h3>

Draw a grid on the input image. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The expressions which specify the coordinates of some point of grid intersection (meant to configure offset). Both default to 0. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> The expressions which specify the width and height of the grid cell, if 0 they are interpreted as the input width and height, respectively, minus <code class="code">thickness</code>, so image gets framed. Default to 0. </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the grid. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. If the special value <code class="code">invert</code> is used, the grid color is the same as the video with inverted luma. </dd> <dt><samp class="option">thickness, t</samp></dt> <dd> The expression which sets the thickness of the grid line. Default value is <code class="code">1</code>. See below for the list of accepted constants. </dd> <dt><samp class="option">replace</samp></dt> <dd> Applicable if the input has alpha. With <code class="code">1</code> the pixels of the painted grid will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the grid onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var

class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input grid cell width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y coordinates of some point of grid intersection (meant to configure offset). </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn cell. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The thickness of the drawn cell. These constants allow the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var>, <var class=”var”>h</var> and <var class=”var”>t</var> expressions to refer to each other, so you may for example specify <code class=”code”>y=x/dar</code> or <code class=”code”>h=w/dar</code>. </dd> </dl>

 <h4 class=”subsection”>39.75.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Draw a grid with cell 100x100 pixels, thickness 2 pixels, with color red and an opacity of 50%: <pre class=”example-preformatted”>drawgrid=width=100:height=100:thickness=2:color=red@0.5 </pre>

Draw a white 3x3 grid with an opacity of 50%: <pre class=”example-preformatted”>drawgrid=w=iw/3:h=ih/3:t=2:c=white@0.5 </pre>

 <h4 class=”subsection”>39.75.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>39.76 drawtext# <a class=”anchor hidden-xs”href=”#toc-drawtext”

aria-hidden=”true”>TOC</h3>

Draw a text string or text from a specified file on top of a video, using the libfreetype library. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libfreetype</code> and <code class=”code”>--enable-libharfbuzz</code>. To enable default font fallback and the <var class=”var”>font</var> option you need to configure FFmpeg with <code class=”code”>--enable-libfontconfig</code>. To enable the <var class=”var”>text_shaping</var> option, you need to configure FFmpeg with <code class=”code”>--enable-libfribidi</code>. <h4 class=”subsection”>39.76.1 Syntax# TOC</h4>

It accepts the following parameters:

<dl class=”table”>

<dt><samp class=”option”>box</samp></dt>

<dd> Used to draw a box around text using the background color.

The value must be either 1 (enable) or 0 (disable).

The default value of <var class=”var”>box</var> is 0.

</dd>

<dt><samp class=”option”>boxborderw</samp></dt>

<dd> Set the width of the border to be drawn around the box using <var class=”var”>boxcolor</var>.

The value must be specified using one of the following formats:

<ul class=”itemize mark-bullet”>

<code class=”code”>boxborderw=10</code> set the width of all the borders to 10

<code class=”code”>boxborderw=10|20</code> set the width of the top and bottom borders to 10

and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30</code> set the width of the top border to 10, the width

of the bottom border to 30 and the width of the left and right borders to 20

<code class="code">boxborderw=10|20|30|40</code> set the borders width to 10 (top), 20 (right),

30 (bottom), 40 (left)

The default value of <var class="var">boxborderw</var> is "0".

</dd>

<dt><samp class="option">boxcolor</samp></dt>

<dd> The color to be used for drawing box around text. For the syntax of this

option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">boxcolor</var> is "white".

</dd>

<dt><samp class="option">line_spacing</samp></dt>

<dd> Set the line spacing in pixels. The default value of <var class="var">line_spacing</var> is 0.

</dd>

<dt><samp class="option">text_align</samp></dt>

<dd> Set the vertical and horizontal alignment of the text with respect to the box boundaries.

The value is combination of flags, one for the vertical alignment (T=top,

M=middle, B=bottom) and one for the horizontal alignment (L=left, C=center, R=right).

Please note that tab characters are only supported with the left horizontal alignment.

</dd>

<dt><samp class="option">y_align</samp></dt>

<dd> Specify what the <var class="var">y</var> value is referred to. Possible values are:

<ul class="itemize mark-bullet">

<code class="code">text</code> the top of the highest glyph of the first text line is placed at <var class="var">y</var>

<code class="code">baseline</code> the baseline of the first text line is placed at <var class="var">y</var>

<code class="code">font</code> the baseline of the first text line is placed at <var class="var">y</var> plus the

ascent (in pixels) defined in the font metrics

The default value of <var class="var">y_align</var> is "text" for backward compatibility.

</dd>

<dt><samp class="option">borderw</samp></dt>

<dd> Set the width of the border to be drawn around the text using <var class="var">bordercolor</var>.

The default value of <var class="var">borderw</var> is 0.

</dd>

<dt><samp class="option">bordercolor</samp></dt>

<dd> Set the color to be used for drawing border around text. For the syntax of this

option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">bordercolor</var> is "black".

</dd>

<dt><samp class="option">expansion</samp></dt>

<dd> Select how the <var class="var">text</var> is expanded. Can be either <code class="code">none</code>,

<code class=”code”>strftime</code> (deprecated) or <code class=”code”>normal</code> (default). See the

Text expansion section below for details.

</dd>

<dt><samp class=”option”>basetime</samp></dt>

<dd> Set a start time for the count. Value is in microseconds. Only applied

in the deprecated <code class=”code”>strftime</code> expansion mode. To emulate in normal expansion

mode use the <code class=”code”>pts</code> function, supplying the start time (in seconds)

as the second argument.

</dd>

<dt><samp class=”option”>fix_bounds</samp></dt>

<dd> If true, check and fix text coords to avoid clipping.

</dd>

<dt><samp class=”option”>fontcolor</samp></dt>

<dd> The color to be used for drawing fonts. For the syntax of this option, check

the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

The default value of <var class=”var”>fontcolor</var> is "black".

</dd>

<dt><samp class=”option”>fontcolor_expr</samp></dt>

<dd> String which is expanded the same way as <var class=”var”>text</var> to obtain dynamic

<var class=”var”>fontcolor</var> value. By default this option has empty value and is not

processed. When this option is set, it overrides <var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> The font family to be used for drawing text. By default Sans.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> The font file to be used for drawing text. The path must be included.

This parameter is mandatory if the fontconfig support is disabled.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Draw the text applying alpha blending. The value can

be a number between 0.0 and 1.0.

The expression accepts the same variables <var class="var">x, y</var> as well.

The default value is 1.

Please see <var class="var">fontcolor_expr</var>.

</dd>

<dt><samp class="option">fontsize</samp></dt>

<dd> The font size to be used for drawing text.

The default value of <var class="var">fontsize</var> is 16.

</dd>

<dt><samp class="option">text_shaping</samp></dt>

<dd> If set to 1, attempt to shape the text (for example, reverse the order of

right-to-left text and join Arabic characters) before drawing it.

Otherwise, just draw the text exactly as given.

By default 1 (if supported).

</dd>
<dt><samp class=”option”>ft_load_flags</samp></dt>
<dd> The flags to be used for loading the fonts.

The flags map the corresponding flags supported by libfreetype, and are a combination of the following values:
<dl class=”table”>
<dt><var class=”var”>default</var></dt>
<dt><var class=”var”>no_scale</var></dt>
<dt><var class=”var”>no_hinting</var></dt>
<dt><var class=”var”>render</var></dt>
<dt><var class=”var”>no_bitmap</var></dt>
<dt><var class=”var”>vertical_layout</var></dt>
<dt><var class=”var”>force_autohint</var></dt>
<dt><var class=”var”>crop_bitmap</var></dt>
<dt><var class=”var”>pedantic</var></dt>
<dt><var class=”var”>ignore_global_advance_width</var></dt>
<dt><var class=”var”>no_recurse</var></dt>
<dt><var class=”var”>ignore_transform</var></dt>
<dt><var class=”var”>monochrome</var></dt>
<dt><var class=”var”>linear_design</var></dt>
<dt><var class=”var”>no_autohint</var></dt>
</dl>
Default value is "default".

For more information consult the documentation for the FT_LOAD_* libfreetype flags.

</dd>

<dt><samp class=”option”>shadowcolor</samp></dt>

<dd> The color to be used for drawing a shadow behind the drawn text. For the

syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the

ffmpeg-utils manual.

The default value of <var class=”var”>shadowcolor</var> is "black".

</dd>

<dt><samp class=”option”>boxw</samp></dt>

<dd> Set the width of the box to be drawn around text.

The default value of <var class=”var”>boxw</var> is computed automatically to match the text width

</dd>

<dt><samp class=”option”>boxh</samp></dt>

<dd> Set the height of the box to be drawn around text.

The default value of <var class=”var”>boxh</var> is computed automatically to match the text height

</dd>

<dt><samp class=”option”>shadowx</samp></dt>

<dt><samp class=”option”>shadowy</samp></dt>

<dd> The x and y offsets for the text shadow position with respect to the

position of the text. They can be either positive or negative

values. The default value for both is "0".

</dd>

<dt><samp class=”option”>start_number</samp></dt>

<dd> The starting frame number for the n/frame_num variable. The default value

is "0".

</dd>

<dt><samp class="option">tabsize</samp></dt>

<dd> The size in number of spaces to use for rendering the tab.

Default value is 4.

</dd>

<dt><samp class="option">timecode</samp></dt>

<dd> Set the initial timecode representation in "hh:mm:ss[:;.]ff"

format. It can be used with or without text parameter. <var class="var">timecode_rate</var>

option must be specified.

</dd>

<dt><samp class="option">timecode_rate, rate, r</samp></dt>

<dd> Set the timecode frame rate (timecode only). Value will be rounded to nearest

integer. Minimum value is "1".

Drop-frame timecode is supported for frame rates 30 & 60.

</dd>

<dt><samp class="option">tc24hmax</samp></dt>

<dd> If set to 1, the output of the timecode option will wrap around at 24 hours.

Default is 0 (disabled).

</dd>

<dt><samp class="option">text</samp></dt>

<dd> The text string to be drawn. The text must be a sequence of UTF-8

encoded characters.

This parameter is mandatory if no file is specified with the parameter

<var class="var">textfile</var>.

</dd>

<dt><samp class="option">textfile</samp></dt>

<dd> A text file containing text to be drawn. The text must be a sequence

of UTF-8 encoded characters.

This parameter is mandatory if no text string is specified with the

parameter <var class="var">text</var>.

If both <var class="var">text</var> and <var class="var">textfile</var> are specified, an error is thrown.

</dd>

<dt><samp class="option">text_source</samp></dt>

<dd> Text source should be set as side_data_detection_bboxes if you want to use text data in

detection bboxes of side data.

If text source is set, <var class="var">text</var> and <var class="var">textfile</var> will be ignored and still use

text data in detection bboxes of side data. So please do not use this parameter

if you are not sure about the text source.

</dd>

<dt><samp class="option">reload</samp></dt>

<dd> The <var class="var">textfile</var> will be reloaded at specified frame interval.

Be sure to update <var class="var">textfile</var> atomically, or it may be read partially,

or even fail.

Range is 0 to INT_MAX. Default is 0.

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> The expressions which specify the offsets where text will be drawn

within the video frame. They are relative to the top/left border of the

output image.

The default value of <var class="var">x</var> and <var class="var">y</var> is "0".

See below for the list of accepted constants and functions.

</dd>

</dl>

The parameters for <var class="var">x</var> and <var class="var">y</var> are expressions containing the

following constants and functions:

<dl class="table">

<dt><samp class="option">dar</samp></dt>

<dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var>

</dd>

<dt><samp class="option">hsub</samp></dt>

<dt><samp class="option">vsub</samp></dt>

<dd> horizontal and vertical chroma subsample values. For example for the

pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1.

</dd>

<dt><samp class="option">line_h, lh</samp></dt>
<dd> the height of each text line

</dd>
<dt><samp class="option">main_h, h, H</samp></dt>
<dd> the input height

</dd>
<dt><samp class="option">main_w, w, W</samp></dt>
<dd> the input width

</dd>
<dt><samp class="option">max_glyph_a, ascent</samp></dt>
<dd> the maximum distance from the baseline to the highest/upper grid
coordinate used to place a glyph outline point, for all the rendered
glyphs.
It is a positive value, due to the grid’s orientation with the Y axis
upwards.

</dd>
<dt><samp class="option">max_glyph_d, descent</samp></dt>
<dd> the maximum distance from the baseline to the lowest grid coordinate
used to place a glyph outline point, for all the rendered glyphs.
This is a negative value, due to the grid’s orientation, with the Y axis
upwards.

</dd>
<dt><samp class="option">max_glyph_h</samp></dt>
<dd> maximum glyph height, that is the maximum height for all the glyphs
contained in the rendered text, it is equivalent to <var class="var">ascent</var>
-

<var class="var">descent</var>.

</dd>

<dt><samp class="option">max_glyph_w</samp></dt>

<dd> maximum glyph width, that is the maximum width for all the glyphs contained in the rendered text

</dd>

<dt><samp class="option">font_a</samp></dt>

<dd> the ascent size defined in the font metrics

</dd>

<dt><samp class="option">font_d</samp></dt>

<dd> the descent size defined in the font metrics

</dd>

<dt><samp class="option">top_a</samp></dt>

<dd> the maximum ascender of the glyphs of the first text line

</dd>

<dt><samp class="option">bottom_d</samp></dt>

<dd> the maximum descender of the glyphs of the last text line

</dd>

<dt><samp class="option">n</samp></dt>

<dd> the number of input frame, starting from 0

</dd>

<dt><samp class="option">rand(min, max)</samp></dt>

<dd> return a random number included between <var class="var">min</var> and <var class="var">max</var>

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> The input sample aspect ratio.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds, NAN if the input timestamp is unknown

</dd>

<dt><samp class="option">text_h, th</samp></dt>

<dd> the height of the rendered text

</dd>

<dt><samp class="option">text_w, tw</samp></dt>

<dd> the width of the rendered text

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> the x and y offset coordinates where the text is drawn.

These parameters allow the <var class="var">x</var> and <var class="var">y</var> expressions to refer

to each other, so you can for example specify <code class="code">y=x/dar</code>.

</dd>

<dt><samp class="option">pict_type</samp></dt>

<dd> A one character description of the current frame’s picture type.

</dd>

<dt><samp class=”option”>pkt_pos</samp></dt>

<dd> The current packet’s position in the input file or stream

(in bytes, from the start of the input). A value of -1 indicates

this info is not available.

</dd>

<dt><samp class=”option”>duration</samp></dt>

<dd> The current packet’s duration, in seconds.

</dd>

<dt><samp class=”option”>pkt_size</samp></dt>

<dd> The current packet’s size (in bytes).

</dd>

</dl>

 <h4 class=”subsection”>39.76.2 Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>strftime</code>, the filter recognizes

sequences accepted by the <code class=”code”>strftime</code> C function in the provided

text and expands them accordingly. Check the documentation of

<code class=”code”>strftime</code>. This feature is deprecated in favor of <code class=”code”>normal</code>

expansion with the <code class=”code”>gmtime</code> or <code class=”code”>localtime</code> expansion

functions.

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim.

If <samp class="option">expansion</samp> is set to <code class="code">normal</code> (which is the default),

the following expansion mechanism is used.

The backslash character ‘<samp class="samp">\</samp>’, followed by any character, always expands to

the second character.

Sequences of the form <code class="code">%{...}</code> are expanded. The text between the

braces is a function name, possibly followed by arguments separated by ’:’.

If the arguments contain special characters or delimiters (’:’ or ’}’),

they should be escaped.

Note that they probably must also be escaped as the value for the <samp class="option">text</samp>

option in the filter argument string and as the filter argument in the

filtergraph description, and possibly also for the shell, that makes up to four

levels of escaping; using a text file with the <samp class="option">textfile</samp> option avoids

these problems.

The following functions are available:

<dl class="table">

<dt><code class="command">expr, e</code></dt>

<dd> The expression evaluation result.

It must take one argument specifying the expression to be evaluated,

which accepts the same constants and functions as the <var class="var">x</var> and

<var class=”var”>y</var> values. Note that not all constants should be used, for

example the text size is not known when evaluating the expression, so

the constants <var class=”var”>text_w</var> and <var class=”var”>text_h</var> will have an undefined

value.

</dd>

<dt><code class=”command”>expr_int_format, eif</code></dt>

<dd> Evaluate the expression’s value and output as formatted integer.

The first argument is the expression to be evaluated, just as for the <var class=”var”>expr</var> function.

The second argument specifies the output format. Allowed values are ‘<samp class=”samp”>x</samp>’,

‘<samp class=”samp”>X</samp>’, ‘<samp class=”samp”>d</samp>’ and ‘<samp class=”samp”>u</samp>’. They are treated exactly as in the

<code class=”code”>printf</code> function.

The third parameter is optional and sets the number of positions taken by the output.

It can be used to add padding with zeros from the left.

</dd>

<dt><code class=”command”>gmtime</code></dt>

<dd> The time at which the filter is running, expressed in UTC.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>localtime</code></dt>

<dd> The time at which the filter is running, expressed in the local time zone.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>metadata</code></dt>

<dd> Frame metadata. Takes one or two arguments.

The first argument is mandatory and specifies the metadata key.

The second argument is optional and specifies a default value, used when the

metadata key is not found or empty.

Available metadata can be identified by inspecting entries

starting with TAG included within each frame section

printed by running <code class=”code”>ffprobe -show_frames</code>.

String metadata generated in filters leading to

the drawtext filter are also available.

</dd>

<dt><code class=”command”>n, frame_num</code></dt>

<dd> The frame number, starting from 0.

</dd>

<dt><code class=”command”>pict_type</code></dt>

<dd> A one character description of the current picture type.

</dd>

<dt><code class="command">pts</code></dt>

<dd> The timestamp of the current frame.

It can take up to three arguments.

The first argument is the format of the timestamp; it defaults to <code class="code">flt</code>

for seconds as a decimal number with microsecond accuracy; <code class="code">hms</code> stands

for a formatted <var class="var">[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy.

<code class="code">gmtime</code> stands for the timestamp of the frame formatted as UTC time;

<code class="code">localtime</code> stands for the timestamp of the frame formatted as

local time zone time.

The second argument is an offset added to the timestamp.

If the format is set to <code class="code">hms</code>, a third argument <code class="code">24HH</code> may be

supplied to present the hour part of the formatted timestamp in 24h format

(00-23).

If the format is set to <code class="code">localtime</code> or <code class="code">gmtime</code>, a third

argument may be supplied: a <code class="code">strftime</code> C function format string.

By default, <var class="var">YYYY-MM-DD HH:MM:SS</var> format will be used.

</dd>

</dl>

 <h4 class="subsection">39.76.3 Commands# TOC</h4>

This filter supports altering parameters via commands: <dl class="table"> <dt><samp class="option">reinit</samp></dt> <dd> Alter existing filter parameters. Syntax for the argument is the same as for filter invocation, e.g.   <pre class="example-preformatted">fontsize=56:fontcolor=green:text='Hello World' </pre>

Full filter invocation with sendcmd would look like this: <pre class="example-preformatted">sendcmd=c='56.0 drawtext reinit fontsize=56\:fontcolor=green\:text=Hello\\ World' </pre>

If the entire argument can’t be parsed or applied as valid values then the filter will continue with its existing parameters. </dd> </dl>

The following options are also supported as commands: <ul class="itemize mark-bullet"> x y alpha fontsize fontcolor boxcolor bordercolor shadowcolor box boxw boxh boxborderw line_spacing text_align shadowx shadowy borderw

 <h4 class="subsection">39.76.4 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw "Test Text" with font FreeSerif, using the default values for the optional parameters.

 <pre class="example-preformatted">drawtext="fontfile=/usr/share/fonts/true
text='Test Text'" </pre>

Draw ’Test Text’ with font FreeSerif of size 24 at position x=100 and y=50 (counting from the top-left corner of the screen), text is yellow with a red box around it. Both the text and the box have an opacity of 20%.

 <pre class="example-preformatted">drawtext="fontfile=/usr/share/fonts/true
text='Test Text':\ x=100: y=50: fontsize=24: fontcolor=yellow@0.2: box=1: boxcolor=red@0.2" </pre>

Note that the double quotes are not necessary if spaces are not used within the parameter list. Show the text at the center of the video frame: <pre class="example-preformatted">drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=(w-text_w)/2:y=(h-text_h)/2" </pre>

Show the text at a random position, switching to a new position every 30 seconds: <pre class=”example-preformatted”>drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=if(eq(mod(t\,30)\,0)\,rand(0\,(w-text_w))\,x):y=if(eq(mod(t\,30)\,0)\,rand(0\,(h-text_h))\,y)" </pre>

Show a text line sliding from right to left in the last row of the video frame. The file <samp class=”file”>LONG_LINE</samp> is assumed to contain a single line with no newlines. <pre class=”example-preformatted”>drawtext="fontsize=15:fontfile=FreeSerif.ttf:text=LONG_LINE:y=h-line_h:x=-50*t" </pre>

Show the content of file <samp class=”file”>CREDITS</samp> off the bottom of the frame and scroll up. <pre class=”example-preformatted”>drawtext="fontsize=20:fontfile=FreeSerif.ttf:textfile=CREDITS:y=h-20*t" </pre>

Draw a single green letter "g", at the center of the input video. The glyph baseline is placed at half screen height. <pre class=”example-preformatted”>drawtext="fontsize=60:fontfile=FreeSerif.ttf:fo
max_glyph_w)/2:y=h/2-ascent" </pre>

Show text for 1 second every 3 seconds: <pre class=”example-preformatted”>drawtext="fontfile=FreeSerif.ttf:fontcolor=white:x=100:y=x/dar:enable
</pre>

Use fontconfig to set the font. Note that the colons need to be escaped. <pre class=”example-preformatted”>drawtext='fontfile=Linux Libertine O-40\\:style=Semibold:text=FFmpeg' </pre>

Draw "Test Text" with font size dependent on height of the video. <pre class=”example-preformatted”>drawtext="text='Test Text': fontsize=h/30: x=(w-text_w)/2: y=(h-text_h*2)" </pre>

Print the date of a real-time encoding (see documentation for the <code class=”code”>strftime</code> C function): <pre class=”example-preformatted”>drawtext='fontfile=FreeSans.ttf:text=%{localtime\:%a %b %d %Y}' </pre>

Show text fading in and out (appearing/disappearing): <pre class=”example-preformatted”>#!/bin/sh DS=1.0 # display start DE=10.0 # display end FID=1.5 # fade in duration FOD=5 # fade out duration ffplay -f lavfi "color,drawtext=text=TEST:fontsize=50:fontfile=FreeSerif.ttf:fontcolor_
clip(255*(1*between(t\\, $DS + $FID\\, $DE - $FOD) + ((t - $DS)/$FID)*between(t\\, $DS\\, $DS + $FID) + (-(t - $DE)/$FOD)*between(t\\, $DE - $FOD\\, $DE))\\, 0\\, 255) \\\\: x\\\\: 2 }" </pre>

Horizontally align multiple separate texts. Note that <samp class=”option”>max_glyph_a</samp> and the <samp class=”option”>fontsize</samp>

value are included in the <samp class="option">y</samp> offset. <pre class="example-preformatted">drawtext=fontfile=FreeSans.ttf:text=DOG:fontsize=max_glyph_a, drawtext=fontfile=FreeSans.ttf:text=cow:fontsize=24:x=80:y=20+24-max_glyph_a </pre>

Plot special <var class="var">lavf.image2dec.source_basename</var> metadata onto each frame if such metadata exists. Otherwise, plot the string "NA". Note that image2 demuxer must have option <samp class="option">-export_path_metadata 1</samp> for the special metadata fields to be available for filters. <pre class="example-preformatted">drawtext="fontsize=20:fontcolor=white:fontfile=FreeSans.ttf:text='%{metadata\:lavf.i
</pre>

For more information about libfreetype, check: http://www.freetype.org/. For more information about fontconfig, check: http://freedesktop.org/software/fontconfig/fontconfig-user.html. For more information about libfribidi, check: http://fribidi.org/. For more information about libharfbuzz, check: https://git

<h3 class="section">39.77 drawvg# TOC</h3>

Draw vector graphics on top of video frames, by executing a script written in a custom language called VGS (Vector Graphics Script). The documentation for the language can be found in <a data-manual="drawvg-reference" href="./drawvg-reference.html#Top">drawvg - Language Reference. A version of this reference with rendered examples can be found at the author’s site. Graphics are rendered using the cario 2D graphics library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-cairo</code>.
<h4 class="subsection">39.77.1 Parameters#
TOC</h4>

Either <code class="code">script</code> or <code class="code">file</code> must be set. <dl class="table"> <dt><samp class="option">s, script</samp></dt> <dd> Script source to draw the graphics. </dd> <dt><samp class="option">file</samp></dt> <dd> Path of the file to load the script source. </dd> </dl>

 <h4 class="subsection">39.77.2 Pixel

Formats# TOC</h4>

Since Cairo only supports RGB images, if the input video is something else (like YUV 4:2:0), before executing the script the video is converted to a format compatible with Cairo. Then, you have to use either the format filter, or the <code class=”code”>-pix_fmt</code> option, to convert it to the expected format in the output. <h4 class=”subsection”>39.77.3 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Draw the outline of an ellipse.

 <pre class=”example-preformatted”>ffmpeg -i input.webm \ -vf 'drawvg=ellipse (w/2) (h/2) (w/3) (h/3) stroke' \ -pix_fmt yuv420p \ output.webm </pre>

 Draw a square rotating in the middle of the frame.

The script for drawvg is in a file <code class=”code”>draw.vgs</code>:   <pre class=”example-preformatted”>translate (w/2) (h/2) rotate t rect -100 -100 200 200 setcolor red@0.5 fill </pre>

Then: <pre class=”example-preformatted”>ffmpeg -i input.webm -vf 'drawvg=file=draw.vgs,format=yuv420p' output.webm </pre>

 <h3 class=”section”>39.78 edgedetect# TOC</h3>

Detect and draw edges. The filter uses the Canny Edge Detection algorithm. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>low</samp></dt> <dt><samp class=”option”>high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class=”var”>low</var> and <var class=”var”>high</var> threshold values must be chosen in the range [0,1], and <var class=”var”>low</var> should be lesser or equal to <var class=”var”>high</var>. Default value for <var class=”var”>low</var> is <code class=”code”>20/255</code>, and default value for <var class=”var”>high</var> is <code class=”code”>50/255</code>.

 </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Define the drawing mode. <dl class=”table”> <dt>‘<samp class=”samp”>wires</samp>’</dt> <dd> Draw white/gray wires on black background. </dd> <dt>‘<samp class=”samp”>colormix</samp>’</dt> <dd> Mix the colors to create a paint/cartoon effect. </dd> <dt>‘<samp class=”samp”>canny</samp>’</dt> <dd> Apply Canny edge detector on all selected planes. </dd> </dl> Default value is <var class=”var”>wires</var>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Select planes for filtering. By default all available planes are filtered. </dd> </dl>

 <h4 class=”subsection”>39.78.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Standard edge detection with custom values for the hysteresis thresholding: <pre class=”example-preformatted”>edgedetect=low=0.1:high=0.4 </pre>

Painting effect without thresholding: <pre class=”example-preformatted”>edgedetect=mode=colormix:high=0 </pre>

 <h3 class=”section”>39.79 elbg# TOC</h3>

Apply a posterize effect using the ELBG (Enhanced LBG) algorithm. For each input image, the filter will compute the optimal mapping from the input to the output given the codebook length, that is the number of distinct output colors. This filter accepts the following options. <dl class=”table”> <dt><samp class=”option”>codebook_length, l</samp></dt> <dd> Set codebook length. The value must be a positive integer, and represents the number of distinct output colors. Default value is 256. </dd> <dt><samp class=”option”>nb_steps, n</samp></dt> <dd> Set the maximum number of iterations to apply for computing the optimal mapping. The higher the value the better the result and the higher the computation time. Default value is 1. </dd> <dt><samp class=”option”>seed, s</samp></dt> <dd> Set a random seed, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class=”option”>pal8</samp></dt> <dd> Set pal8 output pixel format. This option does not work with codebook length greater than 256. Default is disabled. </dd> <dt><samp class=”option”>use_alpha</samp></dt> <dd> Include alpha values in the quantization calculation. Allows creating palettized output images (e.g. PNG8) with multiple alpha smooth blending. </dd> </dl>

 <h3 class="section">39.80 entropy# TOC</h3>

Measure graylevel entropy in histogram of color channels of video frames. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Can be either <var class="var">normal</var> or <var class="var">diff</var>. Default is <var class="var">normal</var>. <var class="var">diff</var> mode measures entropy of histogram delta values, absolute differences between neighbour histogram values. </dd> </dl>

 <h3 class="section">39.81 epx# TOC</h3> Apply the EPX magnification filter which is designed for pixel art. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xEPX</code>, <code class="code">3</code> for <code class="code">3xEPX</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">39.82 eq# TOC</h3> Set brightness, contrast, saturation and approximate gamma adjustment. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. The value must be a float value in range <code class="code">-1000.0</code> to <code class="code">1000.0</code>. The default value is "1". </dd> <dt><samp class="option">brightness</samp></dt> <dd> Set the brightness expression. The value must be a float value in range <code class="code">-1.0</code> to <code class="code">1.0</code>. The default value is "0". </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the saturation expression. The value must be a float in range <code class="code">0.0</code> to <code class="code">3.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma</samp></dt> <dd> Set the gamma expression. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_r</samp></dt> <dd> Set the gamma expression for red. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_g</samp></dt> <dd> Set the gamma expression for green. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_b</samp></dt>

<dd> Set the gamma expression for blue. The value must be a float in range <code class=”code”>0.1</code> to <code class=”code”>10.0</code>. The default value is "1". </dd> <dt><samp class=”option”>gamma_weight</samp></dt> <dd> Set the gamma weight expression. It can be used to reduce the effect of a high gamma value on bright image areas, e.g. keep them from getting overamplified and just plain white. The value must be a float in range <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. A value of <code class=”code”>0.0</code> turns the gamma correction all the way down while <code class=”code”>1.0</code> leaves it at its full strength. Default is "1". </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for brightness, contrast, saturation and gamma expressions are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’. </dd> </dl>

The expressions accept the following parameters: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pos</samp></dt> <dd> byte position of the corresponding packet in the input file, NAN if unspecified; deprecated, do not use </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> </dl>

 <h4 class=”subsection”>39.82.1 Commands# TOC</h4> The filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>contrast</samp></dt> <dd> Set the contrast expression. </dd> <dt><samp class=”option”>brightness</samp></dt> <dd> Set the brightness expression. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set the saturation expression. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set the gamma expression. </dd> <dt><samp class=”option”>gamma_r</samp></dt> <dd> Set the gamma_r expression. </dd> <dt><samp class=”option”>gamma_g</samp></dt> <dd> Set gamma_g expression. </dd> <dt><samp class=”option”>gamma_b</samp></dt> <dd> Set gamma_b expression. </dd> <dt><samp class=”option”>gamma_weight</samp></dt> <dd> Set gamma_weight expression. The command accepts

the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>39.83 erosion# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>39.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.84 estdif# TOC</h3>

Deinterlace the input video ("estdif" stands for "Edge Slope Tracing Deinterlacing Filter"). Spatial only filter that uses edge slope tracing algorithm to interpolate missing lines. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp

class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> <dt><samp class=”option”>rslope</samp></dt> <dd> Specify the search radius for edge slope tracing. Default value is 1. Allowed range is from 1 to 15. </dd> <dt><samp class=”option”>redge</samp></dt> <dd> Specify the search radius for best edge matching. Default value is 2. Allowed range is from 0 to 15. </dd> <dt><samp class=”option”>ecost</samp></dt> <dd> Specify the edge cost for edge matching. Default value is 2. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>mcost</samp></dt> <dd> Specify the middle cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>dcost</samp></dt> <dd> Specify the distance cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the interpolation used. Default is 4-point interpolation. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>2p</samp></dt> <dd> Two-point interpolation. </dd> <dt><samp class=”option”>4p</samp></dt> <dd> Four-point interpolation. </dd> <dt><samp class=”option”>6p</samp></dt> <dd> Six-point interpolation. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>39.84.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>39.85 exposure# TOC</h3> Adjust exposure of the video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>exposure</samp></dt> <dd> Set the exposure correction in EV. Allowed range is from -3.0 to 3.0 EV Default value is 0 EV. </dd> <dt><samp class=”option”>black</samp></dt> <dd> Set the black level correction. Allowed range is from -1.0 to 1.0. Default value is 0. </dd>

</dl>

 <h4 class="subsection">39.85.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">39.86 extractplanes# TOC</h3>

Extract color channel components from input video stream into separate grayscale video streams. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set plane(s) to extract. Available values for planes are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

Choosing planes not available in the input will result in an error. That means you cannot select <code class="code">r</code>, <code class="code">g</code>, <code class="code">b</code> planes with <code class="code">y</code>, <code class="code">u</code>, <code class="code">v</code> planes at same time. </dd> </dl>

 <h4 class="subsection">39.86.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract luma, u and v color channel component from input video frame into 3 grayscale outputs: <pre class="example-preformatted">ffmpeg -i video.avi -filter_complex 'extractplanes=y+u+v[y][u][v]' -map '[y]' y.avi -map '[u]' u.avi -map '[v]' v.avi </pre>

 <h3 class="section">39.87 fade# TOC</h3>

Apply a fade-in/out effect to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">type, t</samp></dt> <dd> The effect type can be either "in" for a fade-in, or "out" for a fade-out effect. Default is <code class="code">in</code>. </dd> <dt><samp

class=”option”>start_frame, s</samp></dt> <dd> Specify the number of the frame to start applying the fade effect at. Default is 0. </dd> <dt><samp class=”option”>nb_frames, n</samp></dt> <dd> The number of frames that the fade effect lasts. At the end of the fade-in effect, the output video will have the same intensity as the input video. At the end of the fade-out transition, the output video will be filled with the selected <samp class=”option”>color</samp>. Default is 25. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> If set to 1, fade only alpha channel, if one exists on the input. Default value is 0. </dd> <dt><samp class=”option”>start_time, st</samp></dt> <dd> Specify the timestamp (in seconds) of the frame to start to apply the fade effect. If both start_frame and start_time are specified, the fade will start at whichever comes last. Default is 0. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> The number of seconds for which the fade effect has to last. At the end of the fade-in effect the output video will have the same intensity as the input video, at the end of the fade-out transition the output video will be filled with the selected <samp class=”option”>color</samp>. If both duration and nb_frames are specified, duration is used. Default is 0 (nb_frames is used by default). </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the fade. Default is "black". </dd> </dl>

 <h4 class=”subsection”>39.87.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Fade in the first 30 frames of video: <pre class=”example-preformatted”>fade=in:0:30 </pre>

The command above is equivalent to: <pre class=”example-preformatted”>fade=t=in:s=0:n=30 </pre>

Fade out the last 45 frames of a 200-frame video: <pre class=”example-preformatted”>fade=out:155:45 fade=type=out:start_frame=155:nb_frames=45 </pre>

Fade in the first 25 frames and fade out the last 25 frames of a 1000-frame video: <pre class=”example-preformatted”>fade=in:0:25, fade=out:975:25 </pre>

Make the first 5 frames yellow, then fade in from frame 5-24: <pre class=”example-preformatted”>fade=in:5:20:color=yellow </pre>

Fade in alpha over first 25 frames of video: <pre class=”example-preformatted”>fade=in:0:25:alpha=1 </pre>

Make the first 5.5 seconds black, then fade in for 0.5 seconds: <div

class=”example”> <pre class=”example-preformatted”>fade=t=in:st=5.5:d=0.5
</pre>

 <h3 class=”section”>39.88 feedback# TOC</h3> Apply feedback video filter. This filter pass cropped input frames to 2nd output. From there it can be filtered with other video filters. After filter receives frame from 2nd input, that frame is combined on top of original frame from 1st input and passed to 1st output. The typical usage is filter only part of frame. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the top left crop position. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the crop size. </dd> </dl>

 <h4 class=”subsection”>39.88.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Blur only top left rectangular part of video frame size 100x100 with gblur filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]gblur=8[bl
</pre>

Draw black box on top left part of video frame of size 100x100 with drawbox filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]drawbox=x=0:y=0:w=100:
</pre>

Pixelize rectangular part of video frame of size 100x100 with pixelize filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=320:y=240:w=100
</pre>

 <h3 class=”section”>39.89 fftdnoiz# TOC</h3> Denoise frames using 3D FFT (frequency domain filtering). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt> <dd> Set the noise sigma constant. This sets denoising strength. Default value is 1. Allowed range is from 0 to 30. Using very high sigma with low overlap may give blocking artifacts. </dd> <dt><samp class=”option”>amount</samp></dt>

<dd> Set amount of denoising. By default all detected noise is reduced. Default value is 1. Allowed range is from 0 to 1. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block in pixels, Default is 32, can be 8 to 256. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set block overlap. Default is 0.5. Allowed range is from 0.2 to 0.8. </dd> <dt><samp class="option">method</samp></dt> <dd> Set denoising method. Default is <code class="code">wiener</code>, can also be <code class="code">hard</code>. </dd> <dt><samp class="option">prev</samp></dt> <dd> Set number of previous frames to use for denoising. By default is set to 0. </dd> <dt><samp class="option">next</samp></dt> <dd> Set number of next frames to to use for denoising. By default is set to 0. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes which will be filtered, by default are all available filtered except alpha. </dd> </dl>

 <h3 class="section">39.90 fftfilt# TOC</h3> Apply arbitrary expressions to samples in frequency domain <dl class="table"> <dt><samp class="option">dc_Y</samp></dt> <dd> Adjust the dc value (gain) of the luma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_U</samp></dt> <dd> Adjust the dc value (gain) of the 1st chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_V</samp></dt> <dd> Adjust the dc value (gain) of the 2nd chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">weight_Y</samp></dt> <dd> Set the frequency domain weight expression for the luma plane. </dd> <dt><samp class="option">weight_U</samp></dt> <dd> Set the frequency domain weight expression for the 1st chroma plane. </dd> <dt><samp class="option">weight_V</samp></dt> <dd> Set the frequency domain weight expression for the 2nd chroma plane. </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.

 The filter accepts the following variables: </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class="option">N</samp></dt> <dd> The number of input frame, starting from 0. </dd> <dt><samp class="option">WS</samp></dt> <dt><samp class="option">HS</samp></dt> <dd> The size of FFT array for horizontal and vertical processing. </dd> </dl>

 <h4 class="subsection">39.90.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> High-pass: <pre class="example-preformatted">fftfilt=dc_Y=128:weight_Y='squish(1-(Y+X)/100)' </pre>

Low-pass: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y='squish((Y+X)/100-1)' </pre>

Sharpen: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y=
(Y+X)/100)' </pre>

Blur: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y='exp
4 * ((Y+X)/(W+H)))' </pre>

 <h3 class="section">39.91 field# TOC</h3>

Extract a single field from an interlaced image using stride arithmetic to avoid wasting CPU time. The output frames are marked as non-interlaced. The filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Specify whether to extract the top (if the value is <code class="code">0</code> or <code class="code">top</code>) or the bottom field (if the value is <code class="code">1</code> or <code class="code">bottom</code>). </dd> </dl>

 <h3 class="section">39.92 fieldhint# TOC</h3>

Create new frames by copying the top and bottom fields from surrounding frames supplied as numbers by the hint file. <dl class="table">

<dt><samp class=”option”>hint</samp></dt> <dd> Set file containing hints: absolute/relative frame numbers. There must be one line for each frame in a clip. Each line must contain two numbers separated by the comma, optionally followed by <code class=”code”>-</code> or <code class=”code”>+</code>. Numbers supplied on each line of file can not be out of [N-1,N+1] where N is current frame number for <code class=”code”>absolute</code> mode or out of [-1, 1] range for <code class=”code”>relative</code> mode. First number tells from which frame to pick up top field and second number tells from which frame to pick up bottom field. If optionally followed by <code class=”code”>+</code> output frame will be marked as interlaced, else if followed by <code class=”code”>-</code> output frame will be marked as progressive, else it will be marked same as input frame. If optionally followed by <code class=”code”>t</code> output frame will use only top field, or in case of <code class=”code”>b</code> it will use only bottom field. If line starts with <code class=”code”>#</code> or <code class=”code”>;</code> that line is skipped. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Can be item <code class=”code”>absolute</code> or <code class=”code”>relative</code> or <code class=”code”>pattern</code>. Default is <code class=”code”>absolute</code>. The <code class=”code”>pattern</code> mode is same as <code class=”code”>relative</code> mode, except at last entry of file if there are more frames to process than <code class=”code”>hint</code> file is seek back to start. </dd> </dl>

Example of first several lines of <code class=”code”>hint</code> file for <code class=”code”>relative</code> mode: <pre class=”example-preformatted”>0,0 - # first frame 1,0 - # second frame, use third's frame top field and second's frame bottom field 1,0 - # third frame, use fourth's frame top field and third's frame bottom field 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - </pre>

 <h3 class=”section”>39.93 fieldmatch# TOC</h3>

Field matching filter for inverse telecine. It is meant to reconstruct the progressive frames from a telecined stream. The filter does not drop duplicated frames, so to achieve a complete inverse telecine <code class=”code”>fieldmatch</code> needs to be followed by a decimation filter such as decimate in the filtergraph. The separation of the field matching and the decimation is notably motivated by the possibility of inserting a de-interlacing filter fallback between the two. If the source has mixed telecined and real interlaced content, <code class=”code”>fieldmatch</code> will not be able to match fields for the interlaced parts. But these remaining combed frames will be marked as interlaced, and thus can be de-interlaced by a

later filter such as yadif before decimation. In addition to the various configuration options, <code class=”code”>fieldmatch</code> can take an optional second stream, activated through the <samp class=”option”>ppsrc</samp> option. If enabled, the frames reconstruction will be based on the fields and frames from this second stream. This allows the first input to be pre-processed in order to help the various algorithms of the filter, while keeping the output lossless (assuming the fields are matched properly). Typically, a field-aware denoiser, or brightness/contrast adjustments can help. Note that this filter uses the same algorithms as TIVTC/TFM (AviSynth project) and VIVTC/VFM (VapourSynth project). The later is a light clone of TFM from which <code class=”code”>fieldmatch</code> is based on. While the semantic and usage are very close, some behaviour and options names can differ. The decimate filter currently only works for constant frame rate input. If your input has mixed telecined (30fps) and progressive content with a lower framerate like 24fps use the following filterchain to produce the necessary cfr stream: <code class=”code”>dejudder,fps=30000/1001,fieldmatch,decimate</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Specify the assumed field order of the input stream. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Auto detect parity (use FFmpeg’s internal parity value). </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Assume bottom field first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Assume top field first. </dd> </dl>

Note that it is sometimes recommended not to trust the parity announced by the stream. Default value is <var class=”var”>auto</var>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the matching mode or strategy to use. <samp class=”option”>pc</samp> mode is the safest in the sense that it won’t risk creating jerkiness due to duplicate frames when possible, but if there are bad edits or blended fields it will end up outputting combed frames when a good match might actually exist. On the other hand, <samp class=”option”>pcn_ub</samp> mode is the most risky in terms of creating jerkiness, but will almost always find a good frame if there is one. The other values are all somewhere in between <samp class=”option”>pc</samp> and <samp class=”option”>pcn_ub</samp> in terms of risking jerkiness and creating duplicate frames versus finding good matches in sections with bad edits, orphaned fields, blended fields, etc. More details about p/c/n/u/b are available in p/c/n/u/b meaning section. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> 2-way matching (p/c) </dd> <dt>‘<samp class=”samp”>pc_n</samp>’</dt>

<dd> 2-way matching, and trying 3rd match if still combed (p/c + n) </dd> <dt>‘<samp class="samp">pc_u</samp>’</dt> <dd> 2-way matching, and trying 3rd match (same order) if still combed (p/c + u) </dd> <dt>‘<samp class="samp">pc_n_ub</samp>’</dt> <dd> 2-way matching, trying 3rd match if still combed, and trying 4th/5th matches if still combed (p/c + n + u/b) </dd> <dt>‘<samp class="samp">pcn</samp>’</dt> <dd> 3-way matching (p/c/n) </dd> <dt>‘<samp class="samp">pcn_ub</samp>’</dt> <dd> 3-way matching, and trying 4th/5th matches if all 3 of the original matches are detected as combed (p/c/n + u/b) </dd> </dl>

The parenthesis at the end indicate the matches that would be used for that mode assuming <samp class="option">order</samp>=<var class="var">tff</var> (and <samp class="option">field</samp> on <var class="var">auto</var> or <var class="var">top</var>). In terms of speed <samp class="option">pc</samp> mode is by far the fastest and <samp class="option">pcn_ub</samp> is the slowest. Default value is <var class="var">pc_n</var>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark the main input stream as a pre-processed input, and enable the secondary input stream as the clean source to pick the fields from. See the filter introduction for more details. It is similar to the <samp class="option">clip2</samp> feature from VFM/TFM. Default value is <code class="code">0</code> (disabled). </dd> <dt><samp class="option">field</samp></dt> <dd> Set the field to match from. It is recommended to set this to the same value as <samp class="option">order</samp> unless you experience matching failures with that setting. In certain circumstances changing the field that is used to match from can have a large impact on matching performance. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic (same value as <samp class="option">order</samp>). </dd> <dt>‘<samp class="samp">bottom</samp>’</dt> <dd> Match from the bottom field. </dd> <dt>‘<samp class="samp">top</samp>’</dt> <dd> Match from the top field. </dd> </dl>

Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mchroma</samp></dt> <dd> Set whether or not chroma is included during the match comparisons. In most cases it is recommended to leave this enabled. You should set this to <code class="code">0</code> only if your clip has bad chroma problems such as heavy rainbowing or other artifacts. Setting this to <code class="code">0</code> could also be used to speed things up at the cost of some accuracy. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">y0</samp></dt> <dt><samp class="option">y1</samp></dt> <dd> These define an exclusion band which excludes the lines between <samp class="option">y0</samp> and <samp class="option">y1</samp> from being included in the field

matching decision. An exclusion band can be used to ignore subtitles, a logo, or other things that may interfere with the matching. <samp class=”option”>y0</samp> sets the starting scan line and <samp class=”option”>y1</samp> sets the ending line; all lines in between <samp class=”option”>y0</samp> and <samp class=”option”>y1</samp> (including <samp class=”option”>y0</samp> and <samp class=”option”>y1</samp>) will be ignored. Setting <samp class=”option”>y0</samp> and <samp class=”option”>y1</samp> to the same value will disable the feature. <samp class=”option”>y0</samp> and <samp class=”option”>y1</samp> defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>scthresh</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change on the luma plane. Good values are in the <code class=”code”>[8.0, 14.0]</code> range. Scene change detection is only relevant in case <samp class=”option”>combmatch</samp>=<var class=”var”>sc</var>. The range for <samp class=”option”>scthresh</samp> is <code class=”code”>[0.0, 100.0]</code>. Default value is <code class=”code”>12.0</code>. </dd> <dt><samp class=”option”>combmatch</samp></dt> <dd> When <samp class=”option”>combatch</samp> is not <var class=”var”>none</var>, <code class=”code”>fieldmatch</code> will take into account the combed scores of matches when deciding what match to use as the final match. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No final matching based on combed scores. </dd> <dt>‘<samp class=”samp”>sc</samp>’</dt> <dd> Combed scores are only used when a scene change is detected. </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Use combed scores all the time. </dd> </dl>

Default is <var class=”var”>sc</var>. </dd> <dt><samp class=”option”>combdbg</samp></dt> <dd> Force <code class=”code”>fieldmatch</code> to calculate the combed metrics for certain matches and print them. This setting is known as <samp class=”option”>micout</samp> in TFM/VFM vocabulary. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No forced calculation. </dd> <dt>‘<samp class=”samp”>pcn</samp>’</dt> <dd> Force p/c/n calculations. </dd> <dt>‘<samp class=”samp”>pcnub</samp>’</dt> <dd> Force p/c/n/u/b calculations. </dd> </dl>

Default value is <var class=”var”>none</var>. </dd> <dt><samp class=”option”>cthresh</samp></dt> <dd> This is the area combing threshold used for combed frame detection. This essentially controls how "strong" or "visible" combing must be to be detected. Larger values mean combing must be more visible and smaller values mean combing can be less visible or strong and still be detected. Valid settings are from <code class=”code”>-1</code> (every pixel will be detected as combed) to <code class=”code”>255</code> (no

pixel will be detected as combed). This is basically a pixel difference value. A good range is <code class=”code”>[8, 12]</code>. Default value is <code class=”code”>9</code>. </dd> <dt><samp class=”option”>chroma</samp></dt> <dd> Sets whether or not chroma is considered in the combed frame decision. Only disable this if your source has chroma problems (rainbowing, etc.) that are causing problems for the combed frame detection with chroma enabled. Actually, using <samp class=”option”>chroma</samp>=<var class=”var”>0</var> is usually more reliable, except for the case where there is chroma only combing in the source. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>blockx</samp></dt> <dt><samp class=”option”>blocky</samp></dt> <dd> Respectively set the x-axis and y-axis size of the window used during combed frame detection. This has to do with the size of the area in which <samp class=”option”>combpel</samp> pixels are required to be detected as combed for a frame to be declared combed. See the <samp class=”option”>combpel</samp> parameter description for more info. Possible values are any number that is a power of 2 starting at 4 and going up to 512. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>combpel</samp></dt> <dd> The number of combed pixels inside any of the <samp class=”option”>blocky</samp> by <samp class=”option”>blockx</samp> size blocks on the frame for the frame to be detected as combed. While <samp class=”option”>cthresh</samp> controls how "visible" the combing must be, this setting controls "how much" combing there must be in any localized area (a window defined by the <samp class=”option”>blockx</samp> and <samp class=”option”>blocky</samp> settings) on the frame. Minimum value is <code class=”code”>0</code> and maximum is <code class=”code”>blocky x blockx</code> (at which point no frames will ever be detected as combed). This setting is known as <samp class=”option”>MI</samp> in TFM/VFM vocabulary. Default value is <code class=”code”>80</code>. </dd> </dl>

 <h4 class=”subsection”>39.93.1 p/c/n/u/b meaning# TOC</h4>

 <h4 class=”subsubsection”>39.93.1.1 p/c/n# TOC</h4>

We assume the following telecined stream: <pre class=”example-preformatted”>Top fields: 1 2 2 3 4 Bottom fields: 1 2 3

4 4 </pre>

The numbers correspond to the progressive frame the fields relate to. Here, the first two frames are progressive, the 3rd and 4th are combed, and so on. When <code class=”code”>fieldmatch</code> is configured to run a matching from bottom (<samp class=”option”>field</samp>=<var class=”var”>bottom</var>) this is how this input stream get transformed: <pre class=”example-preformatted”>Input stream: T 1 2 2 3 4 B 1 2 3 4 4 <-- matching reference

Matches: c c n n c

Output stream: T 1 2 3 4 4 B 1 2 3 4 4 </pre>

As a result of the field matching, we can see that some frames get duplicated. To perform a complete inverse telecine, you need to rely on a decimation filter after this operation. See for instance the decimate filter. The same operation now matching from top fields (<samp class=”option”>field</samp>=<var class=”var”>top</var>) looks like this: <pre class=”example-preformatted”>Input stream: T 1 2 2 3 4 <-- matching reference B 1 2 3 4 4

Matches: c c p p c

Output stream: T 1 2 2 3 4 B 1 2 2 3 4 </pre>

In these examples, we can see what <var class=”var”>p</var>, <var class=”var”>c</var> and <var class=”var”>n</var> mean; basically, they refer to the frame and field of the opposite parity: <ul class=”itemize mark-bullet”> <var class=”var”>p</var> matches the field of the opposite parity in the previous frame <var class=”var”>c</var> matches the field of the opposite parity in the current frame <var class=”var”>n</var> matches the field of the opposite parity in the next frame

 <h4 class=”subsubsection”>39.93.1.2 u/b# TOC</h4>

The <var class=”var”>u</var> and <var class=”var”>b</var> matching are a bit special in the sense that they match from the opposite parity flag. In the following examples, we assume that we are currently matching the 2nd frame (Top:2, bottom:2). According to the match, a ’x’ is placed above and below each matched fields. With bottom matching (<samp class=”option”>field</samp>=<var class=”var”>bottom</var>): <pre class=”example-preformatted”>Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 1 2 2 2 2 2 2 1 3 </pre>

With top matching (<samp class="option">field</samp>=<var class="var">top</var>): <pre class="example-preformatted">Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 2 2 1 2 2 1 3 2 2 </pre>

 <h4 class="subsection">39.93.2 Examples# TOC</h4>

Simple IVTC of a top field first telecined stream: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=none, decimate </pre>

Advanced IVTC, with fallback on yadif for still combed frames: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=full, yadif=deint=interlaced, decimate </pre>

 <h3 class="section">39.94 fieldorder# TOC</h3>

Transform the field order of the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> The output field order. Valid values are <var class="var">tff</var> for top field first or <var class="var">bff</var> for bottom field first. </dd> </dl>

The default value is ‘<samp class="samp">tff</samp>’. The transformation is done by shifting the picture content up or down by one line, and filling the remaining line with appropriate picture content. This method is consistent with most broadcast field order converters. If the input video is not flagged as being interlaced, or it is already flagged as being of the required output field order, then this filter does not alter the incoming video. It is very useful when converting to or from PAL DV material, which is bottom field first. For example: <pre class="example-preformatted">ffmpeg -i in.vob -vf "fieldorder=bff" out.dv </pre>

 <h3 class="section">39.95 fillborders<span

class=”pull-right”># TOC</h3>

Fill borders of the input video, without changing video stream dimensions. Sometimes video can have garbage at the four edges and you may not want to crop video input to keep size multiple of some number. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>left</samp></dt> <dd> Number of pixels to fill from left border. </dd> <dt><samp class=”option”>right</samp></dt> <dd> Number of pixels to fill from right border. </dd> <dt><samp class=”option”>top</samp></dt> <dd> Number of pixels to fill from top border. </dd> <dt><samp class=”option”>bottom</samp></dt> <dd> Number of pixels to fill from bottom border. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set fill mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>smear</samp>’</dt> <dd> fill pixels using outermost pixels </dd> <dt>‘<samp class=”samp”>mirror</samp>’</dt> <dd> fill pixels using mirroring (half sample symmetric) </dd> <dt>‘<samp class=”samp”>fixed</samp>’</dt> <dd> fill pixels with constant value </dd> <dt>‘<samp class=”samp”>reflect</samp>’</dt> <dd> fill pixels using reflecting (whole sample symmetric) </dd> <dt>‘<samp class=”samp”>wrap</samp>’</dt> <dd> fill pixels using wrapping </dd> <dt>‘<samp class=”samp”>fade</samp>’</dt> <dd> fade pixels to constant value </dd> <dt>‘<samp class=”samp”>margins</samp>’</dt> <dd> fill pixels at top and bottom with weighted averages pixels near borders </dd> </dl>

Default is <var class=”var”>smear</var>. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Set color for pixels in fixed or fade mode. Default is <var class=”var”>black</var>. </dd> </dl>

 <h4 class=”subsection”>39.95.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>39.96 find_rect# TOC</h3>

Find a rectangular object in the input video. The object to search for must be specified as a gray8 image specified with

the <samp class="option">object</samp> option. For each possible match, a score is computed. If the score reaches the specified threshold, the object is considered found. If the input video contains multiple instances of the object, the filter will find only one of them. When an object is found, the following metadata entries are set in the matching frame: <dl class="table"> <dt><samp class="option">lavfi.rect.w</samp></dt> <dd> width of object </dd> <dt><samp class="option">lavfi.rect.h</samp></dt> <dd> height of object </dd> <dt><samp class="option">lavfi.rect.x</samp></dt> <dd> x position of object </dd> <dt><samp class="option">lavfi.rect.y</samp></dt> <dd> y position of object </dd> <dt><samp class="option">lavfi.rect.score</samp></dt> <dd> match score of the found object </dd> </dl>

It accepts the following options: <dl class="table"> <dt><samp class="option">object</samp></dt> <dd> Filepath of the object image, needs to be in gray8. </dd> <dt><samp class="option">threshold</samp></dt> <dd> Detection threshold, expressed as a decimal number in the range 0-1. A threshold value of 0.01 means only exact matches, a threshold of 0.99 means almost everything matches. Default value is 0.5. </dd> <dt><samp class="option">mipmaps</samp></dt> <dd> Number of mipmaps, default is 3. </dd> <dt><samp class="option">xmin, ymin, xmax, ymax</samp></dt> <dd> Specifies the rectangle in which to search. </dd> <dt><samp class="option">discard</samp></dt> <dd> Discard frames where object is not detected. Default is disabled. </dd> </dl>

 <h4 class="subsection">39.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

Find the position of an object in each frame using <code class="command">ffprobe</code> and write it to a log file: <pre class="example-preformatted">ffprobe -f lavfi movie=test.mp4,find_rect=object=object.pgm:threshold=0.3 \ -show_entries frame=pkt_pts_time:frame_tags=lavfi.rect.x,lavfi.rect.y \ -of csv -o find_rect.csv </pre>

 <h3 class="section">39.97 floodfill# <a class="anchor hidden-xs"href="#toc-floodfill"

aria-hidden=”true”>TOC</h3>

Flood area with values of same pixel components with another values. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set pixel x coordinate. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set pixel y coordinate. </dd> <dt><samp class=”option”>s0</samp></dt> <dd> Set source #0 component value. </dd> <dt><samp class=”option”>s1</samp></dt> <dd> Set source #1 component value. </dd> <dt><samp class=”option”>s2</samp></dt> <dd> Set source #2 component value. </dd> <dt><samp class=”option”>s3</samp></dt> <dd> Set source #3 component value. </dd> <dt><samp class=”option”>d0</samp></dt> <dd> Set destination #0 component value. </dd> <dt><samp class=”option”>d1</samp></dt> <dd> Set destination #1 component value. </dd> <dt><samp class=”option”>d2</samp></dt> <dd> Set destination #2 component value. </dd> <dt><samp class=”option”>d3</samp></dt> <dd> Set destination #3 component value. </dd> </dl>

 <h3 class=”section”>39.98 format# TOC</h3>

Convert the input video to one of the specified pixel formats. Libavfilter will try to pick one that is suitable as input to the next filter. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as "pix_fmts=yuv420p|monow|rgb24". </dd> <dt><samp class=”option”>color_spaces</samp></dt> <dd> A ’|’-separated list of color space names, such as "color_spaces=bt709|bt470bg|bt2020nc". </dd> <dt><samp class=”option”>color_ranges</samp></dt> <dd> A ’|’-separated list of color range names, such as "color_ranges=tv|pc". </dd> <dt><samp class=”option”>alpha_modes</samp></dt> <dd> A ’|’-separated list of color range names, such as "alpha_modes=straight|premultiplied". </dd> <dt><samp class=”option”>chroma_locations</samp></dt> <dd> A ’|’-separated list of chroma sample location names, such as "chroma_locations=left|topleft". </dd> </dl>

 <h4 class=”subsection”>39.98.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert the input video to the

<var class="var">yuv420p</var> format <pre class="example-preformatted">format=pix_fmts=yuv420p </pre>

Convert the input video to any of the formats in the list <pre class="example-preformatted">format=pix_fmts=yuv420p|yuv444p|yuv410p </pre>

 <h3 class="section">39.99 fps# TOC</h3>

Convert the video to specified constant frame rate by duplicating or dropping frames as necessary. It accepts the following parameters: <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> The desired output frame rate. It accepts expressions containing the following constants: <dl class="table"> <dt>‘<samp class="samp">source_fps</samp>’</dt> <dd> The input’s frame rate </dd> <dt>‘<samp class="samp">ntsc</samp>’</dt> <dd> NTSC frame rate of <code class="code">30000/1001</code> </dd> <dt>‘<samp class="samp">pal</samp>’</dt> <dd> PAL frame rate of <code class="code">25.0</code> </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> Film frame rate of <code class="code">24.0</code> </dd> <dt>‘<samp class="samp">ntsc_film</samp>’</dt> <dd> NTSC-film frame rate of <code class="code">24000/1001</code> </dd> </dl>

The default is <code class="code">25</code>. </dd> <dt><samp class="option">start_time</samp></dt> <dd> Assume the first PTS should be the given value, in seconds. This allows for padding/trimming at the start of stream. By default, no assumption is made about the first frame’s expected PTS, so no padding or trimming is done. For example, this could be set to 0 to pad the beginning with duplicates of the first frame if a video stream starts after the audio stream or to trim any frames with a negative PTS. </dd> <dt><samp class="option">round</samp></dt> <dd> Timestamp (PTS) rounding method. Possible values are: <dl class="table"> <dt><samp class="option">zero</samp></dt> <dd> round towards 0 </dd> <dt><samp class="option">inf</samp></dt> <dd> round away from 0 </dd> <dt><samp class="option">down</samp></dt> <dd> round towards -infinity </dd> <dt><samp class="option">up</samp></dt> <dd> round towards +infinity </dd> <dt><samp class="option">near</samp></dt> <dd> round to nearest </dd> </dl> The default is <code class="code">near</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> Action performed when reading the last frame. Possible values are: <dl class="table"> <dt><samp class="option">round</samp></dt> <dd> Use same timestamp rounding method as used for other

frames. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass through last frame if input duration has not been reached yet. </dd> </dl> The default is <code class="code">round</code>. If timestamp rounding would otherwise discard every frame, one frame is retained when its rounded timestamp equals both the first output timestamp and the rounded end of input. This exception extends the output beyond the rounded end of input by one output frame period. Frames whose entire interval is trimmed by <samp class="option">start_time</samp> remain discarded. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class="var">fps</var>[:<var class="var">start_time</var>[:<var class="var">round</var>]]. See also the setpts filter. <h4 class="subsection">39.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A typical usage in order to set the fps to 25: <pre class="example-preformatted">fps=fps=25 </pre>

Sets the fps to 24, using abbreviation and rounding method to round to nearest: <pre class="example-preformatted">fps=fps=film:round=near </pre>

 <h3 class="section">39.100 framepack# TOC</h3>

Pack two different video streams into a stereoscopic video, setting proper metadata on supported codecs. The two views should have the same size and framerate and processing will stop when the shorter video ends. Please note that you may conveniently adjust view properties with the scale and fps filters. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> The desired packing format. Supported values are: <dl class="table"> <dt><samp class="option">sbs</samp></dt> <dd> The views are next to each other (default). </dd> <dt><samp class="option">tab</samp></dt> <dd> The views are on top of each other. </dd> <dt><samp class="option">lines</samp></dt> <dd> The views are packed by line. </dd> <dt><samp class="option">columns</samp></dt> <dd> The views are packed by column. </dd> <dt><samp class="option">frameseq</samp></dt> <dd> The views are temporally interleaved. </dd> </dl>

</dd> </dl>

Some examples: <pre class=”example-preformatted”># Convert left and right views into a frame-sequential video ffmpeg -i LEFT -i RIGHT -filter_complex framepack=frameseq OUTPUT

Convert views into a side-by-side video with the same output resolution as the input ffmpeg -i LEFT -i RIGHT -filter_complex [0:v]scale=w=iw/2[left],[1:v]scale=w=iw/2[right],[left][right]fra
OUTPUT </pre>

 <h3 class=”section”>39.101 framerate# TOC</h3>

Change the frame rate by interpolating new video output frames from the source frames. This filter is not designed to function correctly with interlaced media. If you wish to change the frame rate of interlaced media then you are required to deinterlace before this filter and re-interlace after this filter. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frames per second. This option can also be specified as a value alone. The default is <code class=”code”>50</code>. </dd> <dt><samp class=”option”>interp_start</samp></dt> <dd> Specify the start of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>15</code>. </dd> <dt><samp class=”option”>interp_end</samp></dt> <dd> Specify the end of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>240</code>. </dd> <dt><samp class=”option”>scene</samp></dt> <dd> Specify the level at which a scene change is detected as a value between 0 and 100 to indicate a new scene; a low value reflects a low probability for the current frame to introduce a new scene, while a higher value means the current frame is more likely to be one. The default is <code class=”code”>8.2</code>. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class=”var”>flags</var> is: <dl class=”table”> <dt><samp class=”option”>scene_change_detect, scd</samp></dt> <dd> Enable scene change detection using the value of the option <var class=”var”>scene</var>. This flag is enabled by default. </dd> </dl> </dd> </dl>

 <h3 class=”section”>39.102 frc_amf# TOC</h3>

Double the frame rate, using Frame Rate Converter (FRC) provided by AMD Advanced Media Framework library for hardware acceleration. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>engine_type</samp></dt> <dd> Specify the engine used to run shaders. <dl class=”table”> <dt>‘<samp class=”samp”>dx11</samp>’</dt> <dd> DirectX 11. </dd> <dt>‘<samp class=”samp”>dx12</samp>’</dt> <dd> DirectX 12 (Default value). </dd> </dl>

</dd> <dt><samp class=”option”>enable</samp></dt> <dd> Boolean value: enable/disable FRC. Dynamic value, can be altered at a runtime without re-initializing the filter. (Default value: enabled). </dd> <dt><samp class=”option”>fallback_mode</samp></dt> <dd> Fallback behavior in case of low interpolation confidence. <dl class=”table”> <dt>‘<samp class=”samp”>duplicate</samp>’</dt> <dd> Duplicate frame. </dd> <dt>‘<samp class=”samp”>blend</samp>’</dt> <dd> Blend two frames together (Default value). </dd> </dl>

</dd> <dt><samp class=”option”>indicator</samp></dt> <dd> Boolean value: show FRC indicator square in the top left corner of the video (Default value: disabled). </dd> <dt><samp class=”option”>profile</samp></dt> <dd> Level of hierarchical motion search. <dl class=”table”> <dt>‘<samp class=”samp”>low</samp>’</dt> <dd> Less levels of hierarchical motion search. Only recommended for extremely low resolutions. </dd> <dt>‘<samp class=”samp”>high</samp>’</dt> <dd> Recommended for any resolution up to 1440p. (Default value) </dd> <dt>‘<samp class=”samp”>super</samp>’</dt> <dd> More levels of hierarchical motion search. Recommended for resolutions 1440p or higher. </dd> </dl>

</dd> <dt><samp class=”option”>mv_search_mode</samp></dt> <dd> Performance mode of the motion search. <dl class=”table”> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Conduct motion search on the full resolution of source images. </dd> <dt>‘<samp class=”samp”>performance</samp>’</dt> <dd> Conduct motion search on the down scaled source images. Recommended for APU or low end GPU for better performance. </dd> </dl>

</dd> <dt><samp class=”option”>use_future_frame</samp></dt> <dd> Boolean value: enable dependency on future frame, improves quality for the cost of latency (Default value: enabled). </dd> </dl>

 <h3 class=”section”>39.103 vqe_amf# TOC</h3>

AMD AMF VQ Enhancer filter. This filter applies AMD AMF VQ Enhance-

ment to the input video. This filter accepts the following option: <dl class="table"> <dt><samp class="option">attenuation</samp></dt> <dd> Set VQ Enhancer strength. Allowed range is from <code class="code">0.02</code> to <code class="code">0.4</code>. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">engine_type</samp></dt> <dd> Set AMF memory type used by the filter. Possible values: <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11 </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 </dd> <dt>‘<samp class="samp">vulkan</samp>’</dt> <dd> Vulkan </dd> <dt>‘<samp class="samp">opencl</samp>’</dt> <dd> OpenCL </dd> </dl>

</dd> </dl>

Example: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf vqe_amf=attenuation=0.4 -c:v hevc_amf output.mp4 </pre>

 <h3 class="section">39.104 framestep# TOC</h3>

Select one frame every N-th frame. This filter accepts the following option: <dl class="table"> <dt><samp class="option">step</samp></dt> <dd> Select frame after every <code class="code">step</code> frames. Allowed values are positive integers higher than 0. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">39.105 freezedetect# TOC</h3>

Detect frozen video. This filter logs a message and sets frame metadata when it detects that the input video has no significant change in content during a specified duration. Video freeze detection calculates the mean average absolute difference of all the components of video frames and compares it to a noise floor. The printed times and duration are expressed in seconds. The <code class="code">lavfi.freezedetect.freeze_start</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the freeze. The <code class="code">lavfi.freezedetect.freeze_duration</code> and <code class="code">lavfi.freezedetect.freeze_end</code> metadata keys are set on the first frame after the freeze. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or as a difference

ratio between 0 and 1. Default is -60dB, or 0.001. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set freeze duration until notification (default is 2 seconds). </dd> </dl>

 <h3 class=”section”>39.106 freezeframes# TOC</h3>

Freeze video frames. This filter freezes video frames using frame from 2nd input. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first</samp></dt> <dd> Set number of first frame from which to start freeze. </dd> <dt><samp class=”option”>last</samp></dt> <dd> Set number of last frame from which to end freeze. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Set number of frame from 2nd input which will be used instead of replaced frames. </dd> </dl>

 <h3 class=”section”>39.107 frei0r# TOC</h3>

Apply a frei0r effect to the input video. To enable the compilation of this filter, you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the frei0r effect to load. If the environment variable <code class=”env”>FREI0R_PATH</code> is defined, the frei0r effect is searched for in each of the directories specified by the colon-separated list in <code class=”env”>FREI0R_PATH</code>. Otherwise, the standard frei0r paths are searched, in this order: <samp class=”file”>HOME/.frei0r-1/lib/</samp>, <samp class=”file”>/usr/local/lib/frei0r-1/</samp>, <samp class=”file”>/usr/lib/frei0r-1/</samp>. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r effect. </dd> </dl>

A frei0r effect parameter can be a boolean (its value is either "y" or "n"), a double, a color (specified as <var class=”var”>R</var>/<var class=”var”>G</var>/<var class=”var”>B</var>, where <var class=”var”>R</var>, <var class=”var”>G</var>, and <var class=”var”>B</var> are floating point numbers between 0.0 and 1.0, inclusive) or a color description as specified in the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual, a position (specified as <var class=”var”>X</var>/<var class=”var”>Y</var>, where <var class=”var”>X</var>

and <var class="var">Y</var> are floating point numbers) and/or a string. The number and types of parameters depend on the loaded effect. If an effect parameter is not specified, the default value is set. <h4 class="subsection">39.107.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply the distort0r effect, setting the first two double parameters: <pre class="example-preformatted">frei0r=filter_name=distort0r:filter_params=0.5|0.01 </pre>

Apply the colordistance effect, taking a color as the first parameter: <pre class="example-preformatted">frei0r=colordistance:0.2/0.3/0.4 frei0r=colordistance:violet frei0r=colordistance:0x112233 </pre>

Apply the perspective effect, specifying the top left and top right image positions: <pre class="example-preformatted">frei0r=perspective:0.2/0.2|0.8/0.2 </pre>

For more information, see http://frei0r.dyne.org <h4 class="subsection">39.107.2 Commands# TOC</h4>

This filter supports the <samp class="option">filter_params</samp> option as commands. <h3 class="section">39.108 fspp# TOC</h3>

Apply fast and simple postprocessing. It is a faster version of spp. It splits (I)DCT into horizontal/vertical passes. Unlike the simple post- processing filter, one of them is performed once per block, not per pixel. This allows for much higher speed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 4-5. Default value is <code class="code">4</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0-63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">strength</samp></dt> <dd> Set filter strength. It accepts an integer in range -15 to 32. Lower values mean more details but also more artifacts, while higher values make the image smoother but also blurrier. Default value is <code class="code">0</code> − PSNR optimal. </dd> <dt><samp class="option">use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code

class=”code”>1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class=”code”>0</code> (not enabled). </dd> </dl>

 <h3 class=”section”>39.109 fsync# TOC</h3>

Synchronize video frames with an external mapping from a file. For each input PTS given in the map file it either drops or creates as many frames as necessary to recreate the sequence of output frames given in the map file. This filter is useful to recreate the output frames of a framerate conversion by the fps filter, recorded into a map file using the ffmpeg option <code class=”code”>-stats_mux_pre</code>, and do further processing to the corresponding frames e.g. quality comparison. Each line of the map file must contain three items per input frame, the input PTS (decimal), the output PTS (decimal) and the output TIMEBASE (decimal/decimal), separated by a space. This file format corresponds to the output of <code class=”code”>-stats_mux_pre_fmt="{ptsi} {pts} {tb}"</code>. The filter assumes the map file is sorted by increasing input PTS. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> The filename of the map file to be used. </dd> </dl>

Example: <pre class=”example-preformatted”># Convert a video to 25 fps and record a MAP_FILE file with the default format of this filter ffmpeg -i INPUT -vf fps=fps=25 -stats_mux_pre MAP_FILE -stats_mux_pre_fmt "{ptsi} {pts} {tb}" OUTPUT

Sort MAP_FILE by increasing input PTS sort -n MAP_FILE

Use INPUT, OUTPUT and the MAP_FILE from above to compare the corresponding frames in INPUT and OUTPUT via SSIM ffmpeg -i INPUT -i OUTPUT -filter_complex '[0:v]fsync=file=MAP_FILE[ref];[1:v][ref]ssim' -f null - </pre>

 <h3 class=”section”>39.110 gainmap# TOC</h3>

Compute an HDR gain map from two renditions of the same image, as defined by ISO 21496-1. The first input is the base rendition (typically SDR), and the second input is the alternate rendition (typically HDR). Both must have the same dimensions. Outputs a gain map, as either ‘<samp class=”samp”>gbrpf32</samp>’ or ‘<samp class=”samp”>grayf32</samp>’ depending on the

chosen <samp class="option">mode</samp>. <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Controls what quantity the gain map is computed against. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> One gain map channel per component. </dd> <dt>‘<samp class="samp">luma</samp>’</dt> <dd> A single gain map channel, computed from the luma. </dd> <dt>‘<samp class="samp">maxrgb</samp>’</dt> <dd> A single gain map channel, computed from ‘<samp class="samp">max(R, G, B)</samp>’. </dd> </dl>

Default is ‘<samp class="samp">rgb</samp>’. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> Chooses which colorspace to compute the gain map in. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">base</samp>’</dt> <dd> The base rendition’s colorspace. </dd> <dt>‘<samp class="samp">alternate</samp>’</dt> <dd> The alternate rendition’s colorspace. </dd> </dl>

Default is ‘<samp class="samp">base</samp>’. </dd> <dt><samp class="option">min</samp></dt> <dt><samp class="option">max</samp></dt> <dd> Bounds on the encoded gain (log2). If unspecified, these will be measured from the frame. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Encoding gamma of the resulting gain map. If unspecified, this will be optimized for the frame in question. Setting this to <code class="code">1.0</code> will produce a linear gain map, which may be more efficient in some applications. </dd> <dt><samp class="option">base_offset</samp></dt> <dt><samp class="option">alt_offset</samp></dt> <dd> Offset to prevent numerical instability near zero. Both default to <code class="code">1/64</code>. </dd> <dt><samp class="option">base_nits</samp></dt> <dt><samp class="option">alt_nits</samp></dt> <dd> Override the reference luminance of the input, in nits. If unspecified, uses the values from the input metadata. </dd> </dl>

The <code class="code">gainmap</code> filter also supports the framesync options. <h4 class="subsection">39.110.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Compute a gain map between an SDR and an HDR version of the same image: <pre class="example-preformatted">ffmpeg -i $sdr -i $hdr -filter_complex "[0:v][1:v]gainmap" $out </pre>

 <h3 class="section">39.111 gblur<span class="pull-

right”>#
TOC</h3>

Apply Gaussian blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt>
<dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>steps</samp></dt> <dd> Set number of steps for Gaussian approximation. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class=”code”>sigma</code>. Default is <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>39.111.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>39.112 geq# TOC</h3>

Apply generic equation to each pixel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>lum_expr, lum</samp></dt> <dd> Set the luma expression. </dd> <dt><samp class=”option”>cb_expr, cb</samp></dt>
<dd> Set the chrominance blue expression. </dd> <dt><samp class=”option”>cr_expr, cr</samp></dt> <dd> Set the chrominance red expression. </dd> <dt><samp class=”option”>alpha_expr, a</samp></dt> <dd> Set the alpha expression. </dd>
<dt><samp class=”option”>red_expr, r</samp></dt> <dd> Set the red expression. </dd> <dt><samp class=”option”>green_expr, g</samp></dt> <dd> Set the green expression. </dd>
<dt><samp class=”option”>blue_expr, b</samp></dt> <dd> Set the blue expression. </dd> </dl>

The colorspace is selected according to the specified options. If one of the <samp class=”option”>lum_expr</samp>, <samp class=”option”>cb_expr</samp>, or <samp class=”option”>cr_expr</samp> options is specified, the filter will automatically select a YCbCr colorspace. If one of the <samp class=”option”>red_expr</samp>, <samp class=”option”>green_expr</samp>, or <samp class=”option”>blue_expr</samp> options is specified, it will select an RGB colorspace. If one of the chrominance expression is not defined, it falls back on the other one. If no alpha expression is specified

it will evaluate to opaque value. If none of chrominance expressions are specified, they will evaluate to the luma expression. The expressions can use the following variables and functions: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class="code">0</code>. </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class="option">SW</samp></dt> <dt><samp class="option">SH</samp></dt> <dd> Width and height scale depending on the currently filtered plane. It is the ratio between the corresponding luma plane number of pixels and the current plane ones. E.g. for YUV4:2:0 the values are <code class="code">1,1</code> for the luma plane, and <code class="code">0.5,0.5</code> for chroma planes. </dd> <dt><samp class="option">T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class="option">p(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the current plane. </dd> <dt><samp class="option">lum(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the luma plane. </dd> <dt><samp class="option">cb(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the blue-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">cr(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">r(x, y)</samp></dt> <dt><samp class="option">g(x, y)</samp></dt> <dt><samp class="option">b(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red/green/blue component. Return 0 if there is no such component. </dd> <dt><samp class="option">alpha(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the alpha plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">psum(x,y), lumsum(x, y), cbsum(x,y), crsum(x,y), rsum(x,y), gsum(x,y), bsum(x,y), alphasum(x,y)</samp></dt> <dd> Sum of sample values in the rectangle from (0,0) to (x,y), this allows obtaining sums of samples within a rectangle. See the functions without the sum postfix. </dd> <dt><samp class="option">interpolation</samp></dt> <dd> Set one of interpolation methods: <dl class="table"> <dt><samp class="option">nearest, n</samp></dt> <dt><samp class="option">bilinear, b</samp></dt> </dl> Default is bilinear. </dd> </dl>

For functions, if <var class="var">x</var> and <var class="var">y</var> are outside the area, the value will be automatically clipped to the closer edge. Please note that this filter can use multiple threads in which case each slice will have its own expression state. If you want to use only a single expression state because your expressions depend on previous state then you should limit the number of filter threads to 1. <h4 class="subsection">39.112.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Flip the image horizontally: <pre class="example-preformatted">geq=p(W-X\,Y) </pre>

Generate a bidimensional sine wave, with angle <code class="code">PI/3</code> and a wavelength of 100 pixels: <pre class="example-preformatted">geq=128 + 100*sin(2*(PI/100)*(cos(PI/3)*(X-50*T) + sin(PI/3)*Y)):128:128 </pre>

Generate a fancy enigmatic moving light: <pre class="example-preformatted">nullsrc=s=256x256,geq=random(1)/hypot(X-cos(N*0.07)*W/2-W/2\,Y-sin(N*0.09)*H/2-H/2)^2*1000000*sin(N*0.02):128:128 </pre>

Generate a quick emboss effect: <pre class="example-preformatted">format=gray,geq=lum_expr='(p(X,Y)+(256-p(X-4,Y-4)))/2' </pre>

Modify RGB components depending on pixel position: <pre class="example-preformatted">geq=r='X/W*r(X,Y)':g='(1-X/W)*g(X,Y)':b='(H-Y)/H*b(X,Y)' </pre>

Create a radial gradient that is the same size as the input (also see the vignette filter): <pre class="example-preformatted">geq=lum=255*gauss((X/W-0.5)*3)*gauss((Y/H-0.5)*3)/gauss(0)/gauss(0),format=gray </pre>

 <h3 class="section">39.113 gradfun# TOC</h3>

Fix the banding artifacts that are sometimes introduced into nearly flat regions by truncation to 8-bit color depth. Interpolate the gradients that should go where the bands are, and dither them. It is designed for playback only. Do not use it prior to lossy compression, because compression tends to lose the dither and bring back the bands. It accepts the following parameters: <dl class="table"> <dt><samp

class=”option”>strength</samp></dt> <dd> The maximum amount by which the filter will change any one pixel. This is also the threshold for detecting nearly flat regions. Acceptable values range from .51 to 64; the default value is 1.2. Out-of-range values will be clipped to the valid range. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> The neighborhood to fit the gradient to. A larger radius makes for smoother gradients, but also prevents the filter from modifying the pixels near detailed regions. Acceptable values are 8-32; the default value is 16. Out-of-range values will be clipped to the valid range. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class=”var”>strength</var>[:<var class=”var”>radius</var>] <h4 class=”subsection”>39.113.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the filter with a <code class=”code”>3.5</code> strength and radius of <code class=”code”>8</code>: <pre class=”example-preformatted”>gradfun=3.5:8 </pre>

Specify radius, omitting the strength (which will fall-back to the default value): <pre class=”example-preformatted”>gradfun=radius=8 </pre>

 <h3 class=”section”>39.114 graphmonitor# TOC</h3> Show various filtergraph stats. With this filter one can debug complete filtergraph. Especially issues with links filling with queued frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set video output size. Default is <var class=”var”>hd720</var>. </dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set video opacity. Default is <var class=”var”>0.9</var>. Allowed range is from <var class=”var”>0</var> to <var class=”var”>1</var>. </dd> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set output mode flags. Available values for flags are: <dl class=”table”> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> No any filtering. Default. </dd> <dt>‘<samp class=”samp”>compact</samp>’</dt> <dd> Show only filters with queued frames. </dd> <dt>‘<samp class=”samp”>nozero</samp>’</dt> <dd> Show only filters with non-zero stats. </dd> <dt>‘<samp class=”samp”>noeof</samp>’</dt> <dd> Show only filters with non-eof stat. </dd> <dt>‘<samp

class=”samp”>nodisabled</samp>’</dt> <dd> Show only filters that are enabled in timeline. </dd> </dl>

</dd> <dt><samp class=”option”>flags, f</samp></dt> <dd> Set flags which enable which stats are shown in video. Available values for flags are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> All flags turned off. </dd> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> All flags turned on. </dd> <dt>‘<samp class=”samp”>queue</samp>’</dt> <dd> Display number of queued frames in each link. </dd> <dt>‘<samp class=”samp”>frame_count_in</samp>’</dt> <dd> Display number of frames taken from filter. </dd> <dt>‘<samp class=”samp”>frame_count_out</samp>’</dt> <dd> Display number of frames given out from filter. </dd> <dt>‘<samp class=”samp”>frame_count_delta</samp>’</dt> <dd> Display delta number of frames between above two values. </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Display current filtered frame pts. </dd> <dt>‘<samp class=”samp”>pts_delta</samp>’</dt> <dd> Display pts delta between current and previous frame. </dd> <dt>‘<samp class=”samp”>time</samp>’</dt> <dd> Display current filtered frame time. </dd> <dt>‘<samp class=”samp”>time_delta</samp>’</dt> <dd> Display time delta between current and previous frame. </dd> <dt>‘<samp class=”samp”>timebase</samp>’</dt> <dd> Display time base for filter link. </dd> <dt>‘<samp class=”samp”>format</samp>’</dt> <dd> Display used format for filter link. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Display video size or number of audio channels in case of audio used by filter link. </dd> <dt>‘<samp class=”samp”>rate</samp>’</dt> <dd> Display video frame rate or sample rate in case of audio used by filter link. </dd> <dt>‘<samp class=”samp”>eof</samp>’</dt> <dd> Display link output status. </dd> <dt>‘<samp class=”samp”>sample_count_in</samp>’</dt> <dd> Display number of samples taken from filter. </dd> <dt>‘<samp class=”samp”>sample_count_out</samp>’</dt> <dd> Display number of samples given out from filter. </dd> <dt>‘<samp class=”samp”>sample_count_delta</samp>’</dt> <dd> Display delta number of samples between above two values. </dd> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Show the timeline filter status. </dd> </dl>

</dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set upper limit for video rate of output stream, Default value is <var class=”var”>25</var>. This guarantee that output video frame rate will not be higher than this value. </dd> </dl>

 <h3 class=”section”>39.115 grayworld<span

class=”pull-right”># TOC</h3> A color constancy filter that applies color correction based on the grayworld assumption See: <a class=”url” href=”https://www.researchgate.net/publication/275213614_A_New_Color_Correction_Met
 The algorithm uses linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class=”example-preformatted”>ffmpeg -i INPUT -vf zscale=transfer=linear,grayworld,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h3 class=”section”>39.116 greyedge# TOC</h3> A color constancy variation filter which estimates scene illumination via grey edge algorithm and corrects the scene colors accordingly. See: <a class=”url” href=”https://staff.science.uva.nl/th.gevers/pub/GeversTIP07.pdf”>https://staff.science.uva.nl/th.gevers/pu
 The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>difford</samp></dt> <dd> The order of differentiation to be applied on the scene. Must be chosen in the range [0,2] and default value is 1. </dd> <dt><samp class=”option”>minknorm</samp></dt> <dd> The Minkowski parameter to be used for calculating the Minkowski distance. Must be chosen in the range [0,20] and default value is 1. Set to 0 for getting max value instead of calculating Minkowski distance. </dd> <dt><samp class=”option”>sigma</samp></dt> <dd> The standard deviation of Gaussian blur to be applied on the scene. Must be chosen in the range [0,1024.0] and default value = 1. floor(<var class=”var”>sigma</var> * break_off_sigma(3)) can’t be equal to 0 if <var class=”var”>difford</var> is greater than 0. </dd> </dl>

 <h4 class=”subsection”>39.116.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Grey Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=5:sigma=2 </pre>

Max Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=0:sigma=2 </pre>

 <h3 class=”section”>39.117 guided# TOC</h3> Apply guided filter

for edge-preserving smoothing, dehazing and so on. The filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set the box radius in pixels. Allowed range is 1 to 20. Default is 3. </dd> <dt><samp class="option">eps</samp></dt> <dd> Set regularization parameter (with square). Allowed range is 0 to 1. Default is 0.01. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set filter mode. Can be <code class="code">basic</code> or <code class="code">fast</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">sub</samp></dt> <dd> Set subsampling ratio for <code class="code">fast</code> mode. Range is 2 to 64. Default is 4. No subsampling occurs in <code class="code">basic</code> mode. </dd> <dt><samp class="option">guidance</samp></dt> <dd> Set guidance mode. Can be <code class="code">off</code> or <code class="code">on</code>. Default is <code class="code">off</code>. If <code class="code">off</code>, single input is required. If <code class="code">on</code>, two inputs of the same resolution and pixel format are required. The second input serves as the guidance. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">39.117.1 Commands# TOC</h4> This filter supports the all above options as commands. <h4 class="subsection">39.117.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Edge-preserving smoothing with guided filter: <pre class="example-preformatted">ffmpeg -i in.png -vf guided out.png </pre>

Dehazing, structure-transferring filtering, detail enhancement with guided filter. For the generation of guidance image, refer to paper "Guided Image Filtering". See: <a class="url" href="http://kaiminghe.com/publications/pami12guidedfilter.pdf
 <pre class="example-preformatted">ffmpeg -i in.png -i guidance.png -filter_complex guided=guidance=on out.png </pre>

 <h3 class="section">39.118 haldclut# TOC</h3>

Apply a Hald CLUT to a video stream. First input

is the video stream to process, and second one is the Hald CLUT. The Hald CLUT input can be a simple picture or a complete video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">clut</samp></dt> <dd> Set which CLUT video frames will be processed from second input stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> <dt><samp class="option">shortest</samp></dt> <dd> Force termination when the shortest input terminates. Default is <code class="code">0</code>. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> Continue applying the last CLUT after the end of the stream. A value of <code class="code">0</code> disable the filter after the last frame of the CLUT is reached. Default is <code class="code">1</code>. </dd> </dl>

<code class="code">haldclut</code> also has the same interpolation options as lut3d (both filters share the same internals). This filter also supports the framesync options. More information about the Hald CLUT can be found on Eskil Steenberg’s website (Hald CLUT author) at <a class="url" href="http://www.quelsolaar.com/technology/clut.html">http://www.quelsolaar.com/technology/clut.html<
  <h4 class="subsection">39.118.1 Commands# TOC</h4>

This filter supports the <code class="code">interp</code> option as commands. <h4 class="subsection">39.118.2 Workflow examples# TOC</h4>

 <h4 class="subsubsection">39.118.2.1 Hald CLUT video stream# TOC</h4>

Generate an identity Hald CLUT stream altered with various effects: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf "hue=H=2*PI*t:s=sin(2*PI*t)+1, curves=cross_process" -t 10 -c:v ffv1 clut.nut </pre>

Note: make sure you use a lossless codec. Then use it with <code class="code">haldclut</code> to apply it on some random stream:

 <pre class="example-preformatted">ffmpeg -f lavfi -i mandelbrot -i clut.nut -filter_complex '[0][1] haldclut' -t 20 mandelclut.mkv </pre>

The Hald CLUT will be applied to the 10 first seconds (duration of <samp class="file">clut.nut</samp>), then the latest picture of that CLUT stream will be applied to the remaining frames of the <code class="code">mandelbrot</code> stream. <h4 class="subsubsection">39.118.2.2 Hald CLUT with preview# TOC</h4>

A Hald CLUT is supposed to be a squared image of <code class="code">Level*Level*Level</code> by <code class="code">Level*Level*Level</code> pixels. For a given Hald CLUT, FFmpeg will select the biggest possible square starting at the top left of the picture. The remaining padding pixels (bottom or right) will be ignored. This area can be used to add a preview of the Hald CLUT. Typically, the following generated Hald CLUT will be supported by the <code class="code">haldclut</code> filter: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf " pad=iw+320 [padded_clut]; smptebars=s=320x256, split [a][b]; [padded_clut][a] overlay=W-320:h, curves=color_negative [main]; [main][b] overlay=W-320" -frames:v 1 clut.png </pre>

It contains the original and a preview of the effect of the CLUT: SMPTE color bars are displayed on the right-top, and below the same color bars processed by the color changes. Then, the effect of this Hald CLUT can be visualized with: <pre class="example-preformatted">ffplay input.mkv -vf "movie=clut.png, [in] haldclut" </pre>

 <h3 class="section">39.119 hflip# TOC</h3>

Flip the input video horizontally. For example, to horizontally flip the input video with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf "hflip" out.avi </pre>

 <h3 class="section">39.120 histeq# TOC</h3> This filter applies a global color histogram equalization on a per-frame basis. It can be used to correct video that has a compressed range of pixel intensities. The

filter redistributes the pixel intensities to equalize their distribution across the intensity range. It may be viewed as an "automatically adjusting contrast filter". This filter is useful only for correcting degraded or poorly captured source video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Determine the amount of equalization to be applied. As the strength is reduced, the distribution of pixel intensities more-and-more approaches that of the input frame. The value must be a float number in the range [0,1] and defaults to 0.200. </dd> <dt><samp class="option">intensity</samp></dt> <dd> Set the maximum intensity that can generated and scale the output values appropriately. The strength should be set as desired and then the intensity can be limited if needed to avoid washing-out. The value must be a float number in the range [0,1] and defaults to 0.210. </dd> <dt><samp class="option">antibanding</samp></dt> <dd> Set the antibanding level. If enabled the filter will randomly vary the luminance of output pixels by a small amount to avoid banding of the histogram. Possible values are <code class="code">none</code>, <code class="code">weak</code> or <code class="code">strong</code>. It defaults to <code class="code">none</code>. </dd> </dl>

 <h3 class="section">39.121 histogram# TOC</h3>

Compute and draw a color distribution histogram for the input video. The computed histogram is a representation of the color component distribution in an image. Standard histogram displays the color components distribution in an image. Displays color graph for each color component. Shows distribution of the Y, U, V, A or R, G, B components, depending on input format, in the current frame. Below each graph a color component scale meter is shown. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_height</samp></dt> <dd> Set height of level. Default value is <code class="code">200</code>. Allowed range is [50, 2048]. </dd> <dt><samp class="option">scale_height</samp></dt> <dd> Set height of color scale. Default value is <code class="code">12</code>. Allowed range is [0, 40]. </dd> <dt><samp class="option">display_mode</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default

is <code class="code">stack</code>. </dd> <dt><samp class="option">levels_mode</samp></dt> <dd> Set mode. Can be either <code class="code">linear</code>, or <code class="code">logarithmic</code>. Default is <code class="code">linear</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set what color components to display. Default is <code class="code">7</code>. </dd> <dt><samp class="option">fgopacity</samp></dt> <dd> Set foreground opacity. Default is <code class="code">0.7</code>. </dd> <dt><samp class="option">bgopacity</samp></dt> <dd> Set background opacity. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">colors_mode</samp></dt> <dd> Set colors mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">whiteonblack</samp>’</dt> <dt>‘<samp class="samp">blackonwhite</samp>’</dt> <dt>‘<samp class="samp">whiteongray</samp>’</dt> <dt>‘<samp class="samp">blackongray</samp>’</dt> <dt>‘<samp class="samp">coloronblack</samp>’</dt> <dt>‘<samp class="samp">coloronwhite</samp>’</dt> <dt>‘<samp class="samp">colorongray</samp>’</dt> <dt>‘<samp class="samp">blackoncolor</samp>’</dt> <dt>‘<samp class="samp">whiteoncolor</samp>’</dt> <dt>‘<samp class="samp">grayoncolor</samp>’</dt> </dl> Default is <code class="code">whiteonblack</code>. </dd> </dl>

 <h4 class="subsection">39.121.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Calculate and draw histogram: <pre class="example-preformatted">ffplay -i input -vf histogram </pre>

 <h3 class="section">39.122 hqdn3d# TOC</h3>

This is a high precision/quality 3d denoise filter. It aims to reduce image noise, producing smooth images and making still images really still. It should enhance compressibility. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">luma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial luma strength. It defaults to 4.0. </dd> <dt><samp class="option">chroma_spatial</samp></dt>

<dd> A non-negative floating point number which specifies spatial chroma strength. It defaults to 3.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">luma_tmp</samp></dt> <dd> A floating point number which specifies luma temporal strength. It defaults to 6.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">chroma_tmp</samp></dt> <dd> A floating point number which specifies chroma temporal strength. It defaults to <var class="var">luma_tmp</var>*<var class="var">chroma_spatial</var>/<var class="var">luma_spatial</var>. </dd> </dl>

 <h4 class="subsection">39.122.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.123 hwdownload# TOC</h3>

Download hardware frames to system memory. The input must be in hardware frames, and the output a non-hardware format. Not all formats will be supported on the output - it may be necessary to insert an additional <samp class="option">format</samp> filter immediately following in the graph to get the output in a supported format. <h3 class="section">39.124 hwmap# TOC</h3>

Map hardware frames to system memory or to another device. This filter has several different modes of operation; which one is used depends on the input and output formats: <ul class="itemize mark-bullet"> Hardware frame input, normal frame output

Map the input frames to system memory and pass them to the output. If the original hardware frame is later required (for example, after overlaying something else on part of it), the <samp class="option">hwmap</samp> filter can be used again in the next mode to retrieve it. Normal frame input, hardware frame output

If the input is actually a software-mapped hardware frame, then unmap it - that is, return the original hardware frame. Otherwise, a device must be provided. Create new hardware surfaces on that device for the output, then map them back to the software format at the input and give those frames to the preceding filter. This will then act like

the <samp class="option">hwupload</samp> filter, but may be able to avoid an additional copy when the input is already in a compatible format. Hardware frame input and output

A device must be supplied for the output, either directly or with the <samp class="option">derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). If the input frames were originally created on the output device, then unmap to retrieve the original frames. Otherwise, map the frames to the output device - create new hardware frames on the output corresponding to the frames on the input.

The following additional parameters are accepted: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set the frame mapping mode. Some combination of: <dl class="table"> <dt><var class="var">read</var></dt> <dd> The mapped frame should be readable. </dd> <dt><var class="var">write</var></dt> <dd> The mapped frame should be writeable. </dd> <dt><var class="var">overwrite</var></dt> <dd> The mapping will always overwrite the entire frame. This may improve performance in some cases, as the original contents of the frame need not be loaded. </dd> <dt><var class="var">direct</var></dt> <dd> The mapping must not involve any copying. Indirect mappings to copies of frames are created in some cases where either direct mapping is not possible or it would have unexpected properties. Setting this flag ensures that the mapping is direct and will fail if that is not possible. </dd> </dl> Defaults to <var class="var">read+write</var> if not specified. </dd> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> <dt><samp class="option">reverse</samp></dt> <dd> In a hardware to hardware mapping, map in reverse - create frames in the sink and map them back to the source. This may be necessary in some cases where a mapping in one direction is required but only the opposite direction is supported by the devices being used. This option is dangerous - it may break the preceding filter in undefined ways if there are any additional constraints on that filter’s output. Do not use it without fully understanding the implications of its use. </dd> </dl>

 <h3 class="section">39.125 hwupload# TOC</h3>

Upload system memory frames to hardware surfaces. The

device to upload to must be supplied when the filter is initialised. If using ffmpeg, select the appropriate device with the <samp class="option">-filter_hw_device</samp> option or with the <samp class="option">derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). The following additional parameters are accepted: <dl class="table"> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> </dl>

 <h3 class="section">39.126 hwupload_cuda# TOC</h3>

Upload system memory frames to a CUDA device. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">device</samp></dt> <dd> The number of the CUDA device to use </dd> </dl>

 <h3 class="section">39.127 hqx# TOC</h3>

Apply a high-quality magnification filter designed for pixel art. This filter was originally created by Maxim Stepin. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">hq2x</code>, <code class="code">3</code> for <code class="code">hq3x</code> and <code class="code">4</code> for <code class="code">hq4x</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">39.128 hstack# TOC</h3> Stack input videos horizontally. All streams must be of same pixel format and of same height. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following option: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">shortest</samp></dt>

<dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class="section">39.129 hsvhold# TOC</h3> Turns a certain HSV range into gray values. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to be gray or not. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more gray pixels, with a higher gray pixel the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class="section">39.130 hsvkey# TOC</h3> Turns a certain HSV range into transparency. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to transparent by adding alpha channel. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the

exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class=”section”>39.131 hue# TOC</h3>

Modify the hue and/or the saturation of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Specify the hue angle as a number of degrees. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>s</samp></dt> <dd> Specify the saturation in the [-10,10] range. It accepts an expression and defaults to "1". </dd> <dt><samp class=”option”>H</samp></dt> <dd> Specify the hue angle as a number of radians. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>b</samp></dt> <dd> Specify the brightness in the [-10,10] range. It accepts an expression and defaults to "0". </dd> </dl>

<samp class=”option”>h</samp> and <samp class=”option”>H</samp> are mutually exclusive, and can’t be specified at the same time. The <samp class=”option”>b</samp>, <samp class=”option”>h</samp>, <samp class=”option”>H</samp> and <samp class=”option”>s</samp> option values are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pts</samp></dt> <dd> presentation timestamp of the input frame expressed in time base units </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class=”option”>tb</samp></dt> <dd> time base of the input video </dd> </dl>

 <h4 class=”subsection”>39.131.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the hue to 90 degrees and the saturation to 1.0: <pre class=”example-preformatted”>hue=h=90:s=1 </pre>

Same command but expressing the hue in radians: <pre class=”example-preformatted”>hue=H=PI/2:s=1 </pre>

Rotate hue and make the saturation swing between 0 and 2 over a period of 1 second: <pre class=”example-preformatted”>hue="H=2*PI*t: s=sin(2*PI*t)+1" </pre>

Apply a 3 seconds saturation fade-in effect starting at 0: <pre class=”example-preformatted”>hue="s=min(t/3\,1)" </pre>

The general fade-in expression can be written as: <pre class=”example-preformatted”>hue="s=min(0\, max((t-START)/DURATION\, 1))" </pre>

Apply a 3 seconds saturation fade-out effect starting at 5 seconds: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (8-t)/3))" </pre>

The general fade-out expression can be written as: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (START+DURATION-t)/DURATION))" </pre>

 <h4 class=”subsection”>39.131.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dt><samp class=”option”>s</samp></dt> <dt><samp class=”option”>h</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> Modify the hue and/or the saturation and/or brightness of the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>39.132 huesaturation# TOC</h3> Apply hue-saturation-intensity adjustments to input video stream. This filter operates in RGB colorspace. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>hue</samp></dt> <dd> Set the hue shift in degrees to apply. Default is 0. Allowed range is from -180 to 180. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set the saturation shift. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>intensity</samp></dt>

<dd> Set the intensity shift. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set which primary and complementary colors are going to be adjusted. This options is set by providing one or multiple values. This can select multiple colors at once. By default all colors are selected. <dl class=”table”> <dt>‘<samp class=”samp”>r</samp>’</dt> <dd> Adjust reds. </dd> <dt>‘<samp class=”samp”>y</samp>’</dt> <dd> Adjust yellows. </dd> <dt>‘<samp class=”samp”>g</samp>’</dt> <dd> Adjust greens. </dd> <dt>‘<samp class=”samp”>c</samp>’</dt> <dd> Adjust cyans. </dd> <dt>‘<samp class=”samp”>b</samp>’</dt> <dd> Adjust blues. </dd> <dt>‘<samp class=”samp”>m</samp>’</dt> <dd> Adjust magentas. </dd> <dt>‘<samp class=”samp”>a</samp>’</dt> <dd> Adjust all colors. </dd> </dl>

</dd> <dt><samp class=”option”>strength</samp></dt> <dd> Set strength of filtering. Allowed range is from 0 to 100. Default value is 1. </dd> <dt><samp class=”option”>rw, gw, bw</samp></dt> <dd> Set weight for each RGB component. Allowed range is from 0 to 1. By default is set to 0.333, 0.334, 0.333. Those options are used in saturation and lightess processing. </dd> <dt><samp class=”option”>lightness</samp></dt> <dd> Set preserving lightness, by default is disabled. Adjusting hues can change lightness from original RGB triplet, with this option enabled lightness is kept at same value. </dd> </dl>

 <h3 class=”section”>39.133 hysteresis# TOC</h3>

Grow first stream into second stream by connecting components. This makes it possible to build more robust edge masks. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold which is used in filtering. If pixel component value is higher than this value filter algorithm for connecting components is activated. By default value is 0. </dd> </dl>

The <code class=”code”>hysteresis</code> filter also supports the framesync options. <h3 class=”section”>39.134 iccdetect# TOC</h3>

Detect the colorspace from an embedded ICC profile (if present), and update the frame’s tags accordingly. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>force</samp></dt> <dd> If true, the frame’s existing colorspace tags will always be overridden by values detected from an ICC profile. Otherwise, they will only be assigned if they contain <code class=”code”>unknown</code>. Enabled by default. </dd> </dl>

 <h3 class=”section”>39.135 iccgen# TOC</h3>

Generate ICC profiles and attach them to frames. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dd> Configure the colorspace that the ICC profile will be generated for. The default value of <code class=”code”>auto</code> infers the value from the input frame’s metadata, defaulting to BT.709/sRGB as appropriate. See the setparams filter for a list of possible values, but note that <code class=”code”>unknown</code> are not valid values for this filter. </dd> <dt><samp class=”option”>force</samp></dt> <dd> If true, an ICC profile will be generated even if it would overwrite an already existing ICC profile. Disabled by default. </dd> </dl>

 <h3 class=”section”>39.136 identity# TOC</h3>

Obtain the identity score between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max identity score is printed through the logging system. The filter stores the calculated identity scores of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi identity -f null - </pre>

 <h3 class=”section”>39.137 idet# TOC</h3>

Detect video interlacing type. This filter tries to detect if

the input frames are interlaced, progressive, top or bottom field first. It will also try to detect fields that are repeated between adjacent frames (a sign of telecine). Single frame detection considers only immediately adjacent frames when classifying each frame. Multiple frame detection incorporates the classification history of previous frames. The filter will log these metadata values: <dl class="table"> <dt><samp class="option">single.current_frame</samp></dt> <dd> Detected type of current frame using single-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class="option">single.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using single-frame detection. </dd> <dt><samp class="option">multiple.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using multiple-frame detection. </dd> <dt><samp class="option">single.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using single-frame detection. </dd> <dt><samp class="option">multiple.current_frame</samp></dt> <dd> Detected type of current frame using multiple-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class="option">multiple.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using multiple-frame detection. </dd> <dt><samp class="option">single.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using single-frame detection. </dd> <dt><samp class="option">multiple.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using multiple-frame detection. </dd> <dt><samp class="option">single.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using single-frame detection. </dd> <dt><samp class="option">multiple.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using multiple-frame detection. </dd> <dt><samp class="option">repeated.current_frame</samp></dt> <dd> Which field in the current frame is repeated from the last. One of “neither”, “top”, or “bottom”. </dd> <dt><samp class="option">repeated.neither</samp></dt> <dd> Cumulative number of frames with no repeated field. </dd> <dt><samp class="option">repeated.top</samp></dt> <dd> Cumulative number of frames with the top field repeated from the previous frame’s top field. </dd> <dt><samp class="option">repeated.bottom</samp></dt> <dd> Cumulative number of frames with the bottom field repeated from the previous frame’s bottom field. </dd> </dl>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">intl_thres</samp></dt> <dd> Set interlacing threshold. </dd> <dt><samp class="option">prog_thres</samp></dt> <dd> Set progressive threshold. </dd> <dt><samp class="option">rep_thres</samp></dt> <dd> Threshold for repeated field detection. </dd> <dt><samp class="option">half_life</samp></dt> <dd> Number of frames after

which a given frame’s contribution to the statistics is halved (i.e., it contributes only 0.5 to its classification). The default of 0 means that all frames seen are given full weight of 1.0 forever. </dd> <dt><samp class="option">analyze_interlaced_flag</samp></dt> <dd> When this is not 0 then idet will use the specified number of frames to determine if the interlaced flag is accurate, it will not count undetermined frames. If the flag is found to be accurate it will be used without any further computations, if it is found to be inaccurate it will be cleared without any further computations. This allows inserting the idet filter as a low computational method to clean up the interlaced flag </dd> </dl>

 <h4 class="subsection">39.137.1 Examples# TOC</h4>

Inspect the field order of the first 360 frames in a video, in verbose detail: <pre class="example-preformatted">ffmpeg -i INPUT -filter:v idet,metadata=mode=print -frames:v 360 -an -f null - </pre> The idet filter will add analysis metadata to each frame, which will then be discarded. At the end, the filter will also print a final report with statistics. <h3 class="section">39.138 il# TOC</h3>

Deinterleave or interleave fields. This filter allows one to process interlaced images fields without deinterlacing them. Deinterleaving splits the input frame into 2 fields (so called half pictures). Odd lines are moved to the top half of the output image, even lines to the bottom half. You can process (filter) them independently and then re-interleave them. The filter accepts the following options: <dl class="table"> <dt><samp class="option">luma_mode, l</samp></dt> <dt><samp class="option">chroma_mode, c</samp></dt> <dt><samp class="option">alpha_mode, a</samp></dt> <dd> Available values for <var class="var">luma_mode</var>, <var class="var">chroma_mode</var> and <var class="var">alpha_mode</var> are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do nothing. </dd> <dt>‘<samp class="samp">deinterleave, d</samp>’</dt> <dd> Deinterleave fields, placing one above the other. </dd> <dt>‘<samp class="samp">interleave, i</samp>’</dt> <dd> Interleave fields. Reverse the effect of deinterleaving. </dd> </dl> Default value is <code class="code">none</code>. </dd> <dt><samp class="option">luma_swap, ls</samp></dt> <dt><samp class="option">chroma_swap, cs</samp></dt> <dt><samp class="option">alpha_swap, as</samp></dt> <dd> Swap luma/chroma/alpha fields. Exchange even & odd lines. Default value is

<code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">39.138.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.139 inflate# TOC</h3>

Apply inflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values higher than the pixel. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">39.139.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.140 interlace, interlace_vulkan# TOC</h3>

Simple interlacing filter from progressive contents. This interleaves upper (or lower) lines from odd frames with lower (or upper) lines from even frames, halving the frame rate and preserving image height.   <pre class="example-preformatted"> Original Original New Frame Frame 'j' Frame 'j+1' (tff) ========== ========== ==========> Line 0 ----------> Frame 'j' Line 0 Line 1 Line 1 ----> Frame 'j+1' Line 1 Line 2 ----------> Frame 'j' Line 2 Line 3 Line 3 ----> Frame 'j+1' Line 3 ... ... New Frame + 1 will be generated by Frame 'j+2' and Frame 'j+3' and so on </pre>

It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">scan</samp></dt> <dd> This determines whether the interlaced frame is taken from the even (tff - default) or odd (bff) lines of the progressive frame. </dd> <dt><samp class="option">lowpass</samp></dt> <dd> Vertical lowpass filter to

avoid twitter interlacing and reduce moire patterns. <dl class="table">
<dt>‘<samp class="samp">0, off</samp>’</dt> <dd> Disable
vertical lowpass filter </dd> <dt>‘<samp class="samp">1,
linear</samp>’</dt> <dd> Enable linear filter (default)
</dd> <dt>‘<samp class="samp">2, complex</samp>’</dt>
<dd> Enable complex filter. This will slightly less reduce twitter and
moire but better retain detail and subjective sharpness impression.
</dd> </dl> </dd> </dl>

 <h3 class="section">39.141 kerndeint<span
class="pull-right"><a class="anchor hidden-xs" href="#kerndeint" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-kerndeint"
aria-hidden="true">TOC</h3>

Deinterlace input video by applying Donald Graft’s adaptive
kernel deinterling. Work on interlaced parts of a video to produce progressive
frames. The description of the accepted parameters follows.
<dl class="table"> <dt><samp class="option">thresh</samp></dt>
<dd> Set the threshold which affects the filter’s tolerance when
determining if a pixel line must be processed. It must be an integer in the range
[0,255] and defaults to 10. A value of 0 will result in applying the process on
every pixels. </dd> <dt><samp class="option">map</samp></dt>
<dd> Paint pixels exceeding the threshold value to white if set to 1. De-
fault is 0. </dd> <dt><samp class="option">order</samp></dt>
<dd> Set the fields order. Swap fields if set to 1, leave fields alone if 0. De-
fault is 0. </dd> <dt><samp class="option">sharp</samp></dt>
<dd> Enable additional sharpening if set to 1. Default is 0. </dd>
<dt><samp class="option">twoway</samp></dt> <dd> Enable
twoway sharpening if set to 1. Default is 0. </dd> </dl>

 <h4 class="subsection">39.141.1 Exam-
ples<a class="anchor hidden-xs" href="#Examples-
139" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-139" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Apply default values:
<pre class="example-preformatted">kerndeint=thresh=10:map=0:order=0:sharp=0:twoway=0
</pre>

Enable additional sharpening: <pre
class="example-preformatted">kerndeint=sharp=1 </pre>

Paint processed pixels in white: <pre
class="example-preformatted">kerndeint=map=1 </pre>

 <h3 class="section">39.142 kirsch<span
class="pull-right"><a class="anchor hidden-xs" href="#kirsch" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-kirsch"
aria-hidden="true">TOC</h3> Apply kirsch operator to

input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">39.142.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.143 lagfun# TOC</h3>

Slowly update darker pixels. This filter makes short flashes of light appear longer. This filter accepts the following options: <dl class="table"> <dt><samp class="option">decay</samp></dt> <dd> Set factor for decaying. Default is .95. Allowed range is from 0 to 1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">39.143.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.144 latticepal# TOC</h3>

Convert packed RGB/BGR input to PAL8 with a per-frame palette whose colors are placed on a face-centered cubic (FCC) lattice spanning the RGB cube. Each frame gets its own palette, built only from the lattice colors actually used by that frame. If a frame uses more than 256 lattice colors, they are reduced by iteratively dropping the color whose removal has the least impact (its pixel count times the squared distance to the nearest remaining color) and mapping its pixels to that nearest remaining color. The alpha channel, if present, is ignored unless the <samp class="option">alpha</samp> option is set. The filter accepts the following options: <dl class="table"> <dt><samp class="option">density</samp></dt>

<dd> Set the lattice density as the number of lattice steps spanning one color axis (0 to 255). Higher values give a finer lattice and thus more possible colors, making it more likely that the per-frame color reduction has to drop some of them. At a density of <code class=”code”>7</code> with 256 palette entries no reduction ever happens since the whole lattice then holds exactly 256 points. Allowed range is <code class=”code”>1</code> to <code class=”code”>255</code>. Default is <code class=”code”>20</code>. </dd> <dt><samp class=”option”>max_colors</samp></dt> <dd> Set the maximum number of palette entries to use. Frames using more lattice colors than this are reduced accordingly. Allowed range is <code class=”code”>2</code> to <code class=”code”>256</code>. Default is <code class=”code”>256</code>. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Quantize the alpha channel too, placing the palette colors on a 4 dimensional D4 lattice over RGBA instead of the FCC lattice over RGB. Inputs without an alpha channel are treated as opaque. With this option the density is limited to <code class=”code”>64</code>. Default is <code class=”code”>0</code> (alpha is ignored and the palette is fully opaque). </dd> <dt><samp class=”option”>refine</samp></dt> <dd> Redistribute the error introduced by dropping colors, which is otherwise uncompensated and can show as discolored patches. Frames that needed no reduction are not changed by this option. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No refinement. </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Run a Floyd-Steinberg error diffusion pass over the whole frame against the final palette. Most accurate on average, but redithers everything, including regions whose colors survived. </dd> <dt>‘<samp class=”samp”>residual</samp>’</dt> <dd> Diffuse only the residual between each pixel’s first pass quantized color and its final palette color. Pixels of surviving colors that receive no residual are unchanged, keeping the character of the selected dither mode outside the reduced regions. </dd> <dt>‘<samp class=”samp”>batched</samp>’</dt> <dd> Interleave color removal and rediffusion: drop half of the excess colors, rediffuse the whole frame against the survivors, recount the colors from the actual assignments and repeat. Colors that the dithering can reproduce by mixing become free to drop while extreme colors keep their pixels and are protected; this gives the best quality at the cost of about log2 of the number of dropped colors rediffusion passes. </dd> </dl>

Default is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>dither</samp></dt> <dd> Select the dithering mode. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No dithering. </dd> <dt>‘<samp class=”samp”>bayer</samp>’</dt> <dd> Ordered 8x8 dithering using the classic swscale dither matrix

with its per-channel decorrelation, offsets spanning one lattice period. </dd> <dt>‘<samp class=”samp”>blue_noise</samp>’</dt> <dd> Void-and-cluster blue noise dithering using three 64x64 masks generated at initialization, one per color channel. </dd> <dt>‘<samp class=”samp”>floyd_steinberg</samp>’</dt> <dd> Floyd-Steinberg error diffusion. </dd> </dl>

Default is <code class=”code”>floyd_steinberg</code>. </dd> </dl>

 <h3 class=”section”>39.145 lenscorrection# TOC</h3>

Correct radial lens distortion This filter can be used to correct for radial distortion as can result from the use of wide angle lenses, and thereby re-rectify the image. To find the right parameters one can use tools available for example as part of opencv or simply trial-and-error. To use opencv use the calibration sample (under samples/cpp) from the opencv sources and extract the k1 and k2 coefficients from the resulting matrix. Note that effectively the same filter is available in the open-source tools Krita and Digikam from the KDE project. In contrast to the vignette filter, which can also be used to compensate lens errors, this filter corrects the distortion of the image, whereas vignette corrects the brightness distribution, so you may want to use both filters together in certain cases, though you will have to take care of ordering, i.e. whether vignetting should be applied before or after lens correction. <h4 class=”subsection”>39.145.1 Options# TOC</h4>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cx</samp></dt> <dd> Relative x-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image width. Default is 0.5. </dd> <dt><samp class=”option”>cy</samp></dt> <dd> Relative y-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image height. Default is 0.5. </dd> <dt><samp class=”option”>k1</samp></dt> <dd> Coefficient of the quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>k2</samp></dt> <dd> Coefficient of the double quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>i</samp></dt> <dd> Set interpo-

lation type. Can be <code class=”code”>nearest</code> or <code class=”code”>bilinear</code>. Default is <code class=”code”>nearest</code>. </dd> <dt><samp class=”option”>fc</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default color is <code class=”code”>black@0</code>. </dd> </dl>

The formula that generates the correction is: <var class=”var”>r_src</var> = <var class=”var”>r_tgt</var> * (1 + <var class=”var”>k1</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^2 + <var class=”var”>k2</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^4) where <var class=”var”>r_0</var> is halve of the image diagonal and <var class=”var”>r_src</var> and <var class=”var”>r_tgt</var> are the distances from the focal point in the source and target images, respectively. <h4 class=”subsection”>39.145.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.146 lensfun# TOC</h3>

Apply lens correction via the lensfun library (<a class=”url” href=”http://lensfun.sourceforge.net/”>http:/

The <code class=”code”>lensfun</code> filter requires the camera make, camera model, and lens model

to apply the lens correction. The filter will load the lensfun database and

query it to find the corresponding camera and lens entries in the database. As

long as these entries can be found with the given options, the filter can

perform corrections on frames. Note that incomplete strings will result in the

filter choosing the best match with the given options, and the filter will

output the chosen camera and lens models (logged with level "info"). You must

provide the make, camera model, and lens model as they are required.

To obtain a list of available makes and models, leave out one or both of <code class=”code”>make</code> and

<code class=”code”>model</code> options. The filter will send the full list to the log with level <code class=”code”>INFO</code>.

The first column is the make and the second column is the model.

To obtain a list of available lenses, set any values for make and model and leave out the

<code class=”code”>lens_model</code> option. The filter will send the full list of lenses in the log with level

<code class=”code”>INFO</code>. The ffmpeg tool will exit after the list is printed.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>make</samp></dt>

<dd> The make of the camera (for example, "Canon"). This option is required.

</dd>

<dt><samp class=”option”>model</samp></dt>

<dd> The model of the camera (for example, "Canon EOS 100D"). This option is

required.

</dd>

<dt><samp class=”option”>lens_model</samp></dt>

<dd> The model of the lens (for example, "Canon EF-S 18-55mm f/3.5-5.6 IS STM"). This

option is required.

</dd>

<dt><samp class=”option”>db_path</samp></dt>

<dd> The full path to the lens database folder. If not set, the filter will attempt to

load the database from the install path when the library was built. Default is unset.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> The type of correction to apply. The following values are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>vignetting</samp>’</dt>

<dd> Enables fixing lens vignetting.

</dd>

<dt>‘<samp class=”samp”>geometry</samp>’</dt>

<dd> Enables fixing lens geometry. This is the default.

</dd>

<dt>‘<samp class=”samp”>subpixel</samp>’</dt>

<dd> Enables fixing chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>vig_geo</samp>’</dt>

<dd> Enables fixing lens vignetting and lens geometry.

</dd>

<dt>‘<samp class=”samp”>vig_subpixel</samp>’</dt>

<dd> Enables fixing lens vignetting and chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>distortion</samp>’</dt>

<dd> Enables fixing both lens geometry and chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>all</samp>’</dt>

<dd> Enables all possible corrections.

</dd>

</dl>

</dd>

<dt><samp class=”option”>focal_length</samp></dt>

<dd> The focal length of the image/video (zoom; expected constant for video). For

example, a 18–55mm lens has focal length range of [18–55], so a value in that

range should be chosen when using that lens. Default 18.

</dd>

<dt><samp class=”option”>aperture</samp></dt>

<dd> The aperture of the image/video (expected constant for video). Note that

aperture is only used for vignetting correction. Default 3.5.

</dd>

<dt><samp class=”option”>focus_distance</samp></dt>

<dd> The focus distance of the image/video (expected constant for video). Note that

focus distance is only used for vignetting and only slightly affects the

vignetting correction process. If unknown, leave it at the default value (which

is 1000).

</dd>

<dt><samp class=”option”>scale</samp></dt>

<dd> The scale factor which is applied after transformation. After correction the

video is no longer necessarily rectangular. This parameter controls how much of

the resulting image is visible. The value 0 means that a value will be chosen

automatically such that there is little or no unmapped area in the output

image. 1.0 means that no additional scaling is done. Lower values may result

in more of the corrected image being visible, while higher values may avoid

unmapped areas in the output.

</dd>

<dt><samp class=”option”>target_geometry</samp></dt>

<dd> The target geometry of the output image/video. The following values are valid

options:

<dl class=”table”>

<dt>‘<samp class=”samp”>rectilinear (default)</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye</samp>’</dt>

<dt>‘<samp class=”samp”>panoramic</samp>’</dt>

<dt>‘<samp class=”samp”>equirectangular</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_orthographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_stereographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_equisolid</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_thoby</samp>’</dt>

</dl>

</dd>

<dt><samp class=”option”>reverse</samp></dt>

<dd> Apply the reverse of image correction (instead of correcting distortion, apply

it).

</dd>

<dt><samp class=”option”>interpolation</samp></dt>

<dd> The type of interpolation used when correcting distortion. The following values

are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>nearest</samp>’</dt>

<dt>‘<samp class=”samp”>linear (default)</samp>’</dt>

<dt>‘<samp class=”samp”>lanczos</samp>’</dt>

</dl>

</dd>

</dl>

 <h4 class=”subsection”>39.146.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply lens correction with make "Canon", camera model "Canon EOS 100D", and lens model "Canon EF-S 18-55mm f/3.5-5.6 IS STM" with focal length of "18" and aperture of "8.0".

 <pre class=”example-preformatted”>ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8 -c:v h264 -b:v 8000k output.mov </pre>

Apply the same as before, but only for the first 5 seconds of video.

 <pre class=”example-preformatted”>ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8:enable='lte(t\,5)' -c:v h264 -b:v 8000k output.mov </pre>

 <h3 class=”section”>39.147 lcevc# TOC</h3>

Low Complexity Enhancement Video Codec filter based on liblcevc_dec (https://github.com/v-

novaltd/LCEVCdec). <h3 class="section">39.148 libplacebo# TOC</h3>

Flexible GPU-accelerated processing filter based on libplacebo (<a class="url" href="https://code.videolan.org/videolan/libplacebo">https://code.videolan.org/videolan/libplac
  <h4 class="subsection">39.148.1 Options# TOC</h4>

The options for this filter are divided into the following sections: <h4 class="subsubsection">39.148.1.1 Output mode# TOC</h4> These options control the overall output mode. By default, libplacebo will try to preserve the source colorimetry and size as best as it can, but it will apply any embedded film grain, dolby vision metadata or anamorphic SAR present in source frames. <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set the number of inputs. This can be used, alongside the <code class="code">idx</code> variable, to allow placing/blending multiple inputs inside the output frame. This effectively enables functionality similar to hstack, overlay, etc. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default values are <code class="code">iw</code> and <code class="code">ih</code>. Allows for the same expressions as the scale filter. </dd> <dt><samp class="option">crop_x</samp></dt> <dt><samp class="option">crop_y</samp></dt> <dd> Set the input crop x/y expressions, default values are <code class="code">(iw-cw)/2</code> and <code class="code">(ih-ch)/2</code>. </dd> <dt><samp class="option">crop_w</samp></dt> <dt><samp class="option">crop_h</samp></dt> <dd> Set the input crop width/height expressions, default values are <code class="code">iw</code> and <code class="code">ih</code>. </dd> <dt><samp class="option">pos_x</samp></dt> <dt><samp class="option">pos_y</samp></dt> <dd> Set the output placement x/y expressions, default values are <code class="code">(ow-pw)/2</code> and <code class="code">(oh-ph)/2</code>. </dd> <dt><samp class="option">pos_w</samp></dt> <dt><samp class="option">pos_h</samp></dt> <dd> Set the output placement width/height expressions, default values are <code class="code">ow</code> and <code class="code">oh</code>. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Rotate the input frame

clockwise by the specified angle. <dl class="table"> <dt>‘<samp class="samp">0, 360</samp>’</dt> <dt>‘<samp class="samp">90</samp>’</dt> <dt>‘<samp class="samp">180</samp>’</dt> <dt>‘<samp class="samp">270</samp>’</dt> </dl>

</dd> <dt><samp class="option">fps</samp></dt> <dd> Set the output frame rate. This can be rational, e.g. <code class="code">60000/1001</code>. If set to the special string <code class="code">none</code> (the default), input timestamps will instead be passed through to the output unmodified. Otherwise, the input video frames will be interpolated as necessary to rescale the video to the specified target framerate, in a manner as determined by the <samp class="option">frame_mixer</samp> option. </dd> <dt><samp class="option">format</samp></dt> <dd> Set the output format override. If unset (the default), frames will be output in the same format as the respective input frames. Otherwise, format conversion will be performed. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. Note that <samp class="option">force_divisible_by</samp> also works with <code class="code">fit_sense=constraint</code>. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> If enabled, output frames will always have a pixel aspect ratio of 1:1. If disabled (the default), any aspect ratio mismatches, including those from e.g. anamorphic video sources, are forwarded to the output pixel aspect ratio. </dd> <dt><samp class="option">normalize_sar</samp></dt> <dd> Like <samp class="option">reset_sar</samp>, but instead of stretching the video content to fill the new output aspect ratio, the content is instead padded or cropped as necessary. Mutually exclusive with <samp class="option">fit_mode</samp>. Disabled by default. </dd> <dt><samp class="option">pad_crop_ratio</samp></dt> <dd> Specifies a ratio (between <code class="code">0.0</code> and <code class="code">1.0</code>) between padding and cropping when the input aspect ratio does not match the output aspect ratio and <samp class="option">normalize_sar</samp> is in effect. The default of <code class="code">0.0</code> always pads the content with black borders, while a value of <code class="code">1.0</code> always crops off parts of the content. Intermediate values are possible, leading to a mix of the two approaches. </dd> <dt><samp class="option">fit_mode</samp></dt> <dd> Specify the content fit strategy according to a list of predefined modes. Determines how the input image is to be placed inside the destination crop rectangle (as defined by <code class="code">pos_x/y</code> and <code class="code">pos_w/h</code>). The names and their implementations are taken from the CSS ’object-fit’ property. Note that this option is mutually exclusive with <samp class="option">normalize_sar</samp>. Defaults to <code

class=”code”>fill</code>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>fill</samp>’</dt> <dd> Stretch the input to the output rectangle, ignoring aspect ratio mismatches. Note that unless <samp class=”option”>reset_sar</samp> is also enabled, the output will still have the correct pixel aspect ratio tagged. </dd> <dt>‘<samp class=”samp”>contain</samp>’</dt> <dd> Scale the input to fit inside the output, preserving aspect ratio by padding. Equivalent to <samp class=”option”>normalize_sar</samp> with <samp class=”option”>pad_crop_ratio</samp> set to <code class=”code”>0.0</code>. </dd> <dt>‘<samp class=”samp”>cover</samp>’</dt> <dd> Scale the input to fill the output, preserving aspect ratio by cropping. Equivalent to <samp class=”option”>normalize_sar</samp> with <samp class=”option”>pad_crop_ratio</samp> set to <code class=”code”>1.0</code>. </dd> <dt>‘<samp class=”samp”>none, place</samp>’</dt> <dd> Don’t scale the input. The input will be placed inside the output rectangle at its natural size; which may result in additional padding or cropping. </dd> <dt>‘<samp class=”samp”>scale_down</samp>’</dt> <dd> Scale the input down as much as needed to fit inside the output. Equivalent to either <code class=”code”>contain</code> or <code class=”code”>none</code>, depending on whether the input is larger than the output or not. </dd> </dl>

</dd> <dt><samp class=”option”>fit_sense</samp></dt> <dd> When <samp class=”option”>fit_mode</samp> is in use, this option controls how the fit strategy is applied against the specified output resolution. Mutually exclusive with <samp class=”option”>force_original_aspect_ratio</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>target</samp>’</dt> <dd> The computed output resolution is taken as the exact size of the output frame. This is the default behavior. </dd> <dt>‘<samp class=”samp”>constraint</samp>’</dt> <dd> The computed output resolution is a size reference against which the fit mode is applied, enlarging or decreasing the true frame size as needed to fit the content. </dd> </dl>

</dd> <dt><samp class=”option”>fillcolor</samp></dt> <dd> Set the color used to fill the output area not covered by the output image, for example as a result of <samp class=”option”>normalize_sar</samp>. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Defaults to <code class=”code”>black@0</code>. </dd> <dt><samp class=”option”>corner_rounding</samp></dt> <dd> Render frames with rounded corners. The value, given as a float ranging from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>, indicates the relative degree of rounding, from fully square to fully circular. In other words, it gives the radius divided by half the smaller side length. Defaults to <code class=”code”>0.0</code>. </dd>

<dt><samp class=”option”>lut</samp></dt> <dd> Specifies a custom LUT (in Adobe .cube format) to apply to the colors as part of color conversion. The exact interpretation depends on the value of <samp class=”option”>lut_type</samp>. </dd> <dt><samp class=”option”>lut_type</samp></dt> <dd> Controls the interpretation of color values fed to and from the LUT specified as <samp class=”option”>lut</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Chooses the interpretation of the LUT automatically from tagged metadata, and otherwise falls back to ‘<samp class=”samp”>native</samp>’. (Default) </dd> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Applied to raw image contents in its native RGB colorspace (non-linear light), before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>normalized</samp>’</dt> <dd> Applied to the normalized RGB image contents, in linear light, before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>conversion</samp>’</dt> <dd> Fully replaces the conversion from the image color space to the output color space. If such a LUT is present, it has the highest priority, and overrides any ICC profiles, as well as options related to tone mapping and output colorimetry (<samp class=”option”>color_primaries</samp>, <samp class=”option”>color_trc</samp>). </dd> </dl>

</dd> <dt><samp class=”option”>extra_opts</samp></dt> <dd> Pass extra libplacebo internal configuration options. These can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to configure a custom filter kernel ("EWA LanczosSharp") and use it to double the input image resolution: <pre class=”example-preformatted”>-vf "libplacebo=w=iw*2:h=ih*2:extra_opts='upscaler=custom\:upscaler_preset=ewa_
</pre>

</dd> <dt><samp class=”option”>shader_cache</samp></dt> <dd> File path of a cache directory that libplacebo will use to store and load cached shader objects. This cache is not cleaned up automatically. If the path does not end in a directory separator, the generated filenames will be effectively prefixed by the last path component. All directories must already exist.   <pre class=”example-preformatted”>-vf "libplacebo=shader_cache=/tmp/pl-shader-" </pre>

</dd> <dt><samp class=”option”>inherit_device</samp></dt> <dd> Take the Vulkan device from the hardware frames arriving on the input, instead of creating a private one from the filter’s own hardware device. This is useful when the input frames originate on another API (for example CUDA frames uploaded to Vulkan by <code class=”code”>hwupload</code>) and the filter’s device is not itself a Vulkan device: libplacebo would otherwise spin up a separate Vulkan device

that the input frames do not belong to, and the frames could not be mapped. When enabled, the filter requires Vulkan hardware frames on the input and will fail negotiation otherwise. Disabled by default.   <pre class=”example-preformatted”>-init_hw_device vulkan=vk:0 -init_hw_device cuda@vk -filter_hw_device cuda0 ... -vf "hwupload=derive_device=vulkan,libplacebo=inherit_device=1" </pre>

</dd> <dt><samp class=”option”>colorspace</samp></dt> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dt><samp class=”option”>range</samp></dt> <dt><samp class=”option”>chroma_location</samp><
<dd> Configure the colorspace that output frames will be delivered in. For any value other than the default <code class=”code”>auto</code>, conversion will be performed. With <code class=”code”>auto</code>, <samp class=”option”>color_primaries</samp> and <samp class=”option”>color_trc</samp> keep the values of the input frames, while <samp class=”option”>colorspace</samp>, <samp class=”option”>range</samp> and <samp class=”option”>chroma_location</samp> use the values negotiated with the filtergraph, which are those of the input unless a following filter requires others. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> Choose the desired output alpha mode, when the output format has an alpha channel. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>apply_filmgrain</samp></dt> <dd> Apply film grain (e.g. AV1 or H.274) if present in source frames, and strip it from the output. Enabled by default. </dd> <dt><samp class=”option”>apply_dolbyvision</samp></dt> <dd> Apply Dolby Vision RPU metadata if present in source frames, and strip it from the output. Enabled by default. Note that Dolby Vision will always output BT.2020+PQ, overriding the usual input frame metadata. These will also be picked as the values of <code class=”code”>auto</code> for the respective frame output options. </dd> </dl>

In addition to the expression constants documented for the scale filter, the <samp class=”option”>crop_w</samp>, <samp class=”option”>crop_h</samp>, <samp class=”option”>crop_x</samp>, <samp class=”option”>crop_y</samp>, <samp class=”option”>pos_w</samp>, <samp class=”option”>pos_h</samp>, <samp class=”option”>pos_x</samp> and <samp class=”option”>pos_y</samp> options can also contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_idx, idx</samp></dt> <dd> The (0-based) numeric index of the currently active input stream. </dd> <dt><samp class=”option”>crop_w, cw</samp></dt> <dt><samp class=”option”>crop_h, ch</samp></dt> <dd> The computed values of <samp class=”option”>crop_w</samp> and <samp class=”option”>crop_h</samp>. </dd> <dt><samp class=”option”>pos_w, pw</samp></dt> <dt><samp class=”option”>pos_h, ph</samp></dt> <dd> The computed values of <samp class=”option”>pos_w</samp>

and <samp class=”option”>pos_h</samp>. </dd> <dt><samp class=”option”>in_t, t</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class=”option”>out_t, ot</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The input frame number, starting with 0. </dd> </dl>

 <h4 class=”subsubsection”>39.148.1.2 Scaling# TOC</h4> The options in this section control how libplacebo performs upscaling and (if necessary) downscaling. Note that libplacebo will always internally operate on 4:4:4 content, so any sub-sampled chroma formats such as <code class=”code”>yuv420p</code> will necessarily be upsampled and downsampled as part of the rendering process. That means scaling might be in effect even if the source and destination resolution are the same. <dl class=”table”> <dt><samp class=”option”>upscaler</samp></dt> <dt><samp class=”option”>downscaler</samp></dt> <dd> Configure the filter kernel used for upscaling and downscaling. The respective defaults are <code class=”code”>spline36</code> and <code class=”code”>mitchell</code>. For a full list of possible values, pass <code class=”code”>help</code> to these options. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Forces the use of built-in GPU texture sampling (typically bilinear). Extremely fast but poor quality, especially when downscaling. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear interpolation. Can generally be done for free on GPUs, except when doing so would lead to aliasing. Fast and low quality. </dd> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Nearest-neighbour interpolation. Sharp but highly aliasing. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Algorithm that looks visually similar to nearest-neighbour interpolation but tries to preserve pixel aspect ratio. Good for pixel art, since it results in minimal distortion of the artistic appearance. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Standard sinc-sinc interpolation kernel. </dd> <dt>‘<samp class=”samp”>spline36</samp>’</dt> <dd> Cubic spline approximation of lanczos. No difference in performance, but has very slightly less ringing. </dd> <dt>‘<samp class=”samp”>ewa_lanczos</samp>’</dt> <dd> Elliptically weighted average version of lanczos, based on a jinc-jinc kernel. This is also popularly referred to as just "Jinc scaling". Slow but very high quality. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> Gaussian kernel. Has certain ideal mathematical properties, but subjectively very blurry. </dd> <dt>‘<samp

class=”samp”>mitchell</samp>’</dt> <dd> Cubic BC spline with parameters recommended by Mitchell and Netravali. Very little ringing. </dd> </dl>

</dd> <dt><samp class=”option”>frame_mixer</samp></dt> <dd> Controls the kernel used for mixing frames temporally. The default value is <code class=”code”>none</code>, which disables frame mixing. For a full list of possible values, pass <code class=”code”>help</code> to this option. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables frame mixing, giving a result equivalent to "nearest neighbour" semantics. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Oversamples the input video to create a "Smooth Motion"-type effect: if an output frame would exactly fall on the transition between two video frames, it is blended according to the relative overlap. This is the recommended option whenever preserving the original subjective appearance is desired. </dd> <dt>‘<samp class=”samp”>mitchell_clamp</samp>’</dt> <dd> Larger filter kernel that smoothly interpolates multiple frames in a manner designed to eliminate ringing and other artefacts as much as possible. This is the recommended option wherever maximum visual smoothness is desired. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linear blend/fade between frames. Especially useful for constructing e.g. slideshows. </dd> </dl>

</dd> <dt><samp class=”option”>antiringing</samp></dt> <dd> Enables anti-ringing (for non-EWA filters). The value (between <code class=”code”>0.0</code> and <code class=”code”>1.0</code>) configures the strength of the anti-ringing algorithm. May increase aliasing if set too high. Disabled by default. </dd> <dt><samp class=”option”>sigmoid</samp></dt> <dd> Enable sigmoidal compression during upscaling. Reduces ringing slightly. Enabled by default. </dd> </dl>

 <h4 class=”subsubsection”>39.148.1.3 Deinterlacing# TOC</h4> Deinterlacing is automatically supported when frames are tagged as interlaced, however frames are not deinterlaced unless a deinterlacing algorithm is chosen. <dl class=”table”> <dt><samp class=”option”>deinterlace</samp></dt> <dd> The the deinterlacing algorithm to use. <dl class=”table”> <dt>‘<samp class=”samp”>weave</samp>’</dt> <dd> No deinterlacing, weave fields together into a single frame. This is the default. </dd> <dt>‘<samp class=”samp”>bob</samp>’</dt> <dd> Naive bob deinterlacing, simply repeat each field line twice. </dd> <dt>‘<samp class=”samp”>yadif</samp>’</dt>

<dd> Yet another deinterlacing filter. See the yadif filter for more details. </dd> <dt>‘<samp class="samp">bwdif</samp>’</dt> <dd> Bob weaver deinterlacing filter. See the bwdif filter for more details. </dd> </dl>

</dd> <dt><samp class="option">skip_spatial_check</samp></dt> <dd> Skip the spatial deinterlacing check when using <code class="code">yadif</code> deinterlacing. </dd> <dt><samp class="option">send_fields</samp></dt> <dd> Output a frame for each field, rather than for each frame. Note that this will always double the tagged output frame rate, even if the input does not contain any interlaced frames. Disabled by default. </dd> </dl>

 <h4 class="subsubsection">39.148.1.4 Debanding# TOC</h4> Libplacebo comes with a built-in debanding filter that is good at counteracting many common sources of banding and blocking. Turning this on is highly recommended whenever quality is desired. <dl class="table"> <dt><samp class="option">deband</samp></dt> <dd> Enable (fast) debanding algorithm. Disabled by default. </dd> <dt><samp class="option">deband_iterations</samp></dt> <dd> Number of deband iterations of the debanding algorithm. Each iteration is performed with progressively increased radius (and diminished threshold). Recommended values are in the range <code class="code">1</code> to <code class="code">4</code>. Defaults to <code class="code">1</code>. </dd> <dt><samp class="option">deband_threshold</samp></dt> <dd> Debanding filter strength. Higher numbers lead to more aggressive debanding. Defaults to <code class="code">4.0</code>. </dd> <dt><samp class="option">deband_radius</samp></dt> <dd> Debanding filter radius. A higher radius is better for slow gradients, while a lower radius is better for steep gradients. Defaults to <code class="code">16.0</code>. </dd> <dt><samp class="option">deband_grain</samp></dt> <dd> Amount of extra output grain to add. Helps hide imperfections. Defaults to <code class="code">6.0</code>. </dd> </dl>

 <h4 class="subsubsection">39.148.1.5 Color adjustment# TOC</h4> A collection of subjective color controls. Not very rigorous, so the exact effect will vary somewhat depending on the input primaries and colorspace. <dl class="table"> <dt><samp class="option">brightness</samp></dt> <dd> Brightness boost, between <code class="code">-1.0</code> and <code class="code">1.0</code>. Defaults to <code class="code">0.0</code>. </dd> <dt><samp class="option">contrast</samp></dt>

<dd> Contrast gain, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Saturation gain, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">hue</samp></dt> <dd> Hue shift in radians, between <code class="code">-3.14</code> and <code class="code">3.14</code>. Defaults to <code class="code">0.0</code>. This will rotate the UV subvector, defaulting to BT.709 coefficients for RGB inputs. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Gamma adjustment, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">temperature</samp></dt> <dd> Color temperature adjustment. Lower values make the output warmer/redder, down to <code class="code">1667</code>, while higher values make the output cooler/bluer, up to <code class="code">25000</code>. Defaults to <code class="code">6500</code> (neutral white). </dd> <dt><samp class="option">cones</samp></dt> <dd> Cone model to use for color blindness simulation. Accepts any combination of <code class="code">l</code>, <code class="code">m</code> and <code class="code">s</code>. Here are some examples: <dl class="table"> <dt>‘<samp class="samp">m</samp>’</dt> <dd> Deuteranomaly / deuteranopia (affecting 3%-4% of the population) </dd> <dt>‘<samp class="samp">l</samp>’</dt> <dd> Protanomaly / protanopia (affecting 1%-2% of the population) </dd> <dt>‘<samp class="samp">l+m</samp>’</dt> <dd> Monochromacy (very rare) </dd> <dt>‘<samp class="samp">l+m+s</samp>’</dt> <dd> Achromatopsy (complete loss of daytime vision, extremely rare) </dd> </dl>

</dd> <dt><samp class="option">cone-strength</samp></dt> <dd> Gain factor for the cones specified by <code class="code">cones</code>, between <code class="code">0.0</code> and <code class="code">10.0</code>. A value of <code class="code">1.0</code> results in no change to color vision. A value of <code class="code">0.0</code> (the default) simulates complete loss of those cones. Values above <code class="code">1.0</code> result in exaggerating the differences between cones, which may help compensate for reduced color vision. </dd> </dl>

 <h4 class="subsubsection">39.148.1.6 Peak detection# TOC</h4> To help deal with sources that only have static HDR10 metadata (or no tagging whatsoever), libplacebo uses its own internal frame analysis compute shader to analyze source frames and adapt the tone mapping function in realtime. If this is too slow, or if exactly reproducible frame-perfect results

are needed, it’s recommended to turn this feature off. <dl class="table"> <dt><samp class="option">peak_detect</samp></dt> <dd> Enable HDR peak detection. Ignores static MaxCLL/MaxFALL values in favor of dynamic detection from the input. Note that the detected values do not get written back to the output frames, they merely guide the internal tone mapping process. Enabled by default. </dd> <dt><samp class="option">smoothing_period</samp></dt> <dd> Peak detection smoothing period, between <code class="code">0.0</code> and <code class="code">1000.0</code>. Higher values result in peak detection becoming less responsive to changes in the input. Defaults to <code class="code">20.0</code>. </dd> <dt><samp class="option">scene_threshold_low</samp></dt> <dt><samp class="option">scene_threshold_high</
<dd> Lower and upper thresholds for scene change detection. Expressed in a logarithmic scale between <code class="code">0.0</code> and <code class="code">100.0</code>. Default to <code class="code">1.0</code> and <code class="code">3.0</code>, respectively. Setting either to a negative value disables this functionality. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Which percentile of the frame brightness histogram to use as the source peak for tone-mapping. Defaults to <code class="code">99.995</code>, a fairly conservative value. Setting this to <code class="code">100.0</code> disables frame histogram measurement and instead uses the true peak brightness for tone-mapping. </dd> </dl>

 <h4 class="subsubsection">39.148.1.7 Tone mapping# TOC</h4>
The options in this section control how libplacebo performs tone-mapping and gamut-mapping when dealing with mismatches between wide-gamut or HDR content. In general, libplacebo relies on accurate source tagging and mastering display gamut information to produce the best results. <dl class="table"> <dt><samp class="option">gamut_mode</samp></dt> <dd> How to handle out-of-gamut colors that can occur as a result of colorimetric gamut mapping. <dl class="table"> <dt>‘<samp class="samp">clip</samp>’</dt> <dd> Do nothing, simply clip out-of-range colors to the RGB volume. Low quality but extremely fast. </dd> <dt>‘<samp class="samp">perceptual</samp>’</dt>
<dd> Perceptually soft-clip colors to the gamut volume. This is the default. </dd> <dt>‘<samp class="samp">relative</samp>’</dt>
<dd> Relative colorimetric hard-clip. Similar to <code class="code">perceptual</code> but without the soft knee. </dd> <dt>‘<samp class="samp">saturation</samp>’</dt>
<dd> Saturation mapping, maps primaries directly to primaries in RGB space. Not recommended except for artificial computer graphics for which a bright, saturated display is desired. </dd> <dt>‘<samp class="samp">absolute</samp>’</dt> <dd> Absolute colori-

metric hard-clip. Performs no adjustment of the white point. </dd> <dt>‘<samp class=”samp”>desaturate</samp>’</dt> <dd> Hard-desaturates out-of-gamut colors towards white, while preserving the luminance. Has a tendency to distort the visual appearance of bright objects. </dd> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dd> Linearly reduces content brightness to preserves saturated details, followed by clipping the remaining out-of-gamut colors. </dd> <dt>‘<samp class=”samp”>warn</samp>’</dt> <dd> Highlight out-of-gamut pixels (by inverting/marking them). </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly reduces chromaticity of the entire image to make it fit within the target color volume. Be careful when using this on BT.2020 sources without proper mastering metadata, as doing so will lead to excessive desaturation. </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping</samp></dt> <dd> Tonemapping algorithm to use. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection based on internal heuristics. This is the default. </dd> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Performs no tone-mapping, just clips out-of-range colors. Retains perfect color accuracy for in-range colors but completely destroys out-of-range information. Does not perform any black point adaptation. Not configurable. </dd> <dt>‘<samp class=”samp”>st2094-40</samp>’</dt> <dd> EETF from SMPTE ST 2094-40 Annex B, which applies the Bezier curves from HDR10+ dynamic metadata based on Bezier curves to perform tone-mapping. The OOTF used is adjusted based on the ratio between the targeted and actual display peak luminances. </dd> <dt>‘<samp class=”samp”>st2094-10</samp>’</dt> <dd> EETF from SMPTE ST 2094-10 Annex B.2, which takes into account the input signal average luminance in addition to the maximum/minimum. The configurable contrast parameter influences the slope of the linear output segment, defaulting to <code class=”code”>1.0</code> for no increase/decrease in contrast. Note that this does not currently include the subjective gain/offset/gamma controls defined in Annex B.3. </dd> <dt>‘<samp class=”samp”>bt.2390</samp>’</dt> <dd> EETF from the ITU-R Report BT.2390, a hermite spline roll-off with linear segment. The knee point offset is configurable. Note that this parameter defaults to <code class=”code”>1.0</code>, rather than the value of <code class=”code”>0.5</code> from the ITU-R spec. </dd> <dt>‘<samp class=”samp”>bt.2446a</samp>’</dt> <dd> EETF from ITU-R Report BT.2446, method A. Designed for well-mastered HDR sources. Can be used for both forward and inverse tone mapping. Not configurable. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Simple spline consisting of two polynomials, joined by a single pivot point. The parameter gives the pivot point (in PQ space), defaulting to <code class=”code”>0.30</code>. Can be used for both

forward and inverse tone mapping. </dd> <dt>‘<samp class=”samp”>reinhard</samp>’</dt> <dd> Simple non-linear, global tone mapping algorithm. The parameter specifies the local contrast coefficient at the display peak. Essentially, a parameter of <code class=”code”>0.5</code> implies that the reference white will be about half as bright as when clipping. Defaults to <code class=”code”>0.5</code>, which results in the simplest formulation of this function. </dd> <dt>‘<samp class=”samp”>mobius</samp>’</dt> <dd> Generalization of the reinhard tone mapping algorithm to support an additional linear slope near black. The tone mapping parameter indicates the trade-off between the linear section and the non-linear section. Essentially, for a given parameter <var class=”var”>x</var>, every color value below <var class=”var”>x</var> will be mapped linearly, while higher values get non-linearly tone-mapped. Values near <code class=”code”>1.0</code> make this curve behave like <code class=”code”>clip</code>, while values near <code class=”code”>0.0</code> make this curve behave like <code class=”code”>reinhard</code>. The default value is <code class=”code”>0.3</code>, which provides a good balance between colorimetric accuracy and preserving out-of-gamut details. </dd> <dt>‘<samp class=”samp”>hable</samp>’</dt> <dd> Piecewise, filmic tone-mapping algorithm developed by John Hable for use in Uncharted 2, inspired by a similar tone-mapping algorithm used by Kodak. Popularized by its use in video games with HDR rendering. Preserves both dark and bright details very well, but comes with the drawback of changing the average brightness quite significantly. This is sort of similar to <code class=”code”>reinhard</code> with parameter <code class=”code”>0.24</code>. </dd> <dt>‘<samp class=”samp”>gamma</samp>’</dt> <dd> Fits a gamma (power) function to transfer between the source and target color spaces, effectively resulting in a perceptual hard-knee joining two roughly linear sections. This preserves details at all scales fairly accurately, but can result in an image with a muted or dull appearance. The parameter is used as the cutoff point, defaulting to <code class=”code”>0.5</code>. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly stretches the input range to the output range, in PQ space. This will preserve all details accurately, but results in a significantly different average brightness. Can be used for inverse tone-mapping in addition to regular tone-mapping. The parameter can be used as an additional linear gain coefficient (defaulting to <code class=”code”>1.0</code>). </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping_param</samp></dt> <dd> For tunable tone mapping functions, this parameter can be used to fine-tune the curve behavior. Refer to the documentation of <code class=”code”>tonemapping</code>. The default value of <code class=”code”>0.0</code> is replaced by the curve’s preferred default setting. </dd> <dt><samp class=”option”>inverse_tonemapping</samp></dt>

<dd> If enabled, this filter will also attempt stretching SDR signals to fill HDR output color volumes. Disabled by default. </dd> <dt><samp class=”option”>tonemapping_lut_size</samp></dt> <dd> Size of the tone-mapping LUT, between <code class=”code”>2</code> and <code class=”code”>1024</code>. Defaults to <code class=”code”>256</code>. Note that this figure is squared when combined with <code class=”code”>peak_detect</code>. </dd> <dt><samp class=”option”>contrast_recovery</samp></dt> <dd> Contrast recovery strength. If set to a value above <code class=”code”>0.0</code>, the source image will be divided into high-frequency and low-frequency components, and a portion of the high-frequency image is added back onto the tone-mapped output. May cause excessive ringing artifacts for some HDR sources, but can improve the subjective sharpness and detail left over in the image after tone-mapping. Defaults to <code class=”code”>0.30</code>. </dd> <dt><samp class=”option”>contrast_smoothness</samp></dt> <dd> Contrast recovery lowpass kernel size. Defaults to <code class=”code”>3.5</code>. Increasing or decreasing this will affect the visual appearance substantially. Has no effect when <code class=”code”>contrast_recovery</code> is disabled. </dd> </dl>

 <h4 class=”subsubsection”>39.148.1.8 Dithering# TOC</h4> By default, libplacebo will dither whenever necessary, which includes rendering to any integer format below 16-bit precision. It’s recommended to always leave this on, since not doing so may result in visible banding in the output, even if the <code class=”code”>debanding</code> filter is enabled. If maximum performance is needed, use <code class=”code”>ordered_fixed</code> instead of disabling dithering. <dl class=”table”> <dt><samp class=”option”>dithering</samp></dt> <dd> Dithering method to use. Accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables dithering completely. May result in visible banding. </dd> <dt>‘<samp class=”samp”>blue</samp>’</dt> <dd> Dither with pseudo-blue noise. This is the default. </dd> <dt>‘<samp class=”samp”>ordered</samp>’</dt> <dd> Tunable ordered dither pattern. </dd> <dt>‘<samp class=”samp”>ordered_fixed</samp>’</dt> <dd> Faster ordered dither with a fixed size of <code class=”code”>6</code>. Texture-less. </dd> <dt>‘<samp class=”samp”>white</samp>’</dt> <dd> Dither with white noise. Texture-less. </dd> </dl>

</dd> <dt><samp class=”option”>dither_lut_size</samp></dt> <dd> Dither LUT size, as log base2 between <code class=”code”>1</code> and <code class=”code”>8</code>. Defaults to <code class=”code”>6</code>, corresponding to a LUT size of <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>dither_temporal</samp></dt>

<dd> Enables temporal dithering. Disabled by default. </dd>
</dl>

 <h4 class="subsubsection">39.148.1.9 Custom shaders# TOC</h4>
libplacebo supports a number of custom shaders based on the mpv .hook GLSL syntax. A collection of such shaders can be found here: https://github.com/mpv-player/mpv/wiki/User-Scripts#user-shaders A full description of the mpv shader format is beyond the scope of this section, but a summary can be found here: https://mpv.io/manual/master/#options-glsl-shader <dl class="table"> <dt><samp class="option">custom_shader_path</samp></dt>
<dd> Specifies a path to a custom shader file to load at runtime. </dd> <dt><samp class="option">custom_shader_bin</samp></dt>
<dd> Specifies a complete custom shader as a raw string. </dd>
</dl>

 <h4 class="subsubsection">39.148.1.10 Debugging / performance# TOC</h4> All of the options in this section default off. They may be of assistance when attempting to squeeze the maximum performance at the cost of quality. <dl class="table"> <dt><samp class="option">skip_aa</samp></dt>
<dd> Disable anti-aliasing when downscaling. </dd> <dt><samp class="option">disable_linear</samp></dt> <dd> Disable linear light scaling. </dd> <dt><samp class="option">disable_builtin</samp></dt>
<dd> Disable built-in GPU sampling (forces LUT). </dd>
<dt><samp class="option">disable_fbos</samp></dt> <dd> Forcibly disable FBOs, resulting in loss of almost all functionality, but offering the maximum possible speed. </dd> </dl>

 <h4 class="subsection">39.148.2 Commands# TOC</h4> This filter supports almost all of the above options as commands. The options negotiated with the filtergraph, <samp class="option">format</samp>, <samp class="option">colorspace</samp>, <samp class="option">range</samp>, <samp class="option">chroma_location</samp> and <samp class="option">alpha_mode</samp>, are fixed once the filtergraph is configured and can not be changed with commands. <h4 class="subsection">39.148.3

Examples# TOC</h4> <ul class=”itemize mark-bullet”> Tone-map input to standard gamut BT.709 output: <pre class=”example-preformatted”>libplacebo=colorspace=bt709:color_primaries=bt709 </pre>

Rescale input to fit into standard 1080p, with high quality scaling: <pre class=”example-preformatted”>libplacebo=w=1920:h=1080:force_original_aspe </pre>

Interpolate low FPS / VFR input to smoothed constant 60 fps output: <pre class=”example-preformatted”>libplacebo=fps=60:frame_mixer=mitchell_clam </pre>

Convert input to standard sRGB JPEG: <pre class=”example-preformatted”>libplacebo=format=yuv420p:colorspace=bt470bg:color_primaries=bt709 2-1:range=pc </pre>

Use higher quality debanding settings: <pre class=”example-preformatted”>libplacebo=deband=true:deband_iterations=3:deband_radius=8:deband_thr </pre>

Run this filter on the CPU, on systems with Mesa installed (and with the most expensive options disabled): <pre class=”example-preformatted”>ffmpeg ... -init_hw_device vulkan:llvmpipe ... -vf libplacebo=upscaler=none:downscaler=none:peak_detect=false </pre>

Suppress CPU-based AV1/H.274 film grain application in the decoder, in favor of doing it with this filter. Note that this is only a gain if the frames are either already on the GPU, or if you’re using libplacebo for other purposes, since otherwise the VRAM roundtrip will more than offset any expected speedup. <pre class=”example-preformatted”>ffmpeg -export_side_data +film_grain ... -vf libplacebo=apply_filmgrain=true </pre>

Interop with VAAPI hwdec to avoid round-tripping through RAM: <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -hwaccel vaapi -hwaccel_output_format vaapi ... -vf libplacebo </pre>

 <h3 class=”section”>39.149 libvmaf# TOC</h3>

Calculate the VMAF (Video Multi-Method Assessment Fusion) score for a reference/distorted pair of input videos. The first input is the distorted video, and the second input is the reference video. The

obtained VMAF score is printed through the logging system. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class="code">./configure --enable-libvmaf</code>. The filter has following options: <dl class="table"> <dt><samp class="option">model</samp></dt> <dd> A ‘|‘ delimited list of vmaf models. Each model can be configured with a number of parameters. Default value: <code class="code">"version=vmaf_v0.6.1"</code> </dd> <dt><samp class="option">feature</samp></dt> <dd> A ‘|‘ delimited list of features. Each feature can be configured with a number of parameters. </dd> <dt><samp class="option">log_path</samp></dt> <dd> Set the file path to be used to store log files. </dd> <dt><samp class="option">log_fmt</samp></dt> <dd> Set the format of the log file (xml, json, csv, or sub). </dd> <dt><samp class="option">pool</samp></dt> <dd> Set the pool method to be used for computing vmaf. Options are <code class="code">min</code>, <code class="code">harmonic_mean</code> or <code class="code">mean</code> (default). </dd> <dt><samp class="option">n_threads</samp></dt> <dd> Set number of threads to be used when initializing libvmaf. Default value: <code class="code">0</code>, no threads. </dd> <dt><samp class="option">n_subsample</samp></d <dd> Set frame subsampling interval to be used. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">39.149.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> In the examples below, a distorted video <samp class="file">distorted.mpg</samp> is compared with a reference file <samp class="file">reference.mpg</samp>.

Basic usage: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf=log_path=output.xml -f null - </pre>

Example with multiple models: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='model=version=vmaf_v0.6.1\\:name=vmaf|version=vmaf_v0.6.1neg\\:name=vmaf_neg' -f null - </pre>

Example with multiple additional features: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='feature=name=psnr|name=ciede' -f null - </pre>

Example with options and different containers: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-

STARTPTS[ref];[main][ref]libvmaf=log_fmt=json:log_path=output.json"
-f null - </pre>

 <h3 class="section">39.150 libvmaf_cuda# TOC</h3>

This is the CUDA variant of the libvmaf filter. It only accepts CUDA frames. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class="code">./configure --enable-nonfree --enable-ffnvcodec --enable-libvmaf</code>. <h4 class="subsection">39.150.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Basic usage showing CUVID hardware decoding and CUDA scaling with scale_cuda: <pre class="example-preformatted">ffmpeg \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i dis.obu \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i ref.obu \ -filter_complex " [0:v]scale_cuda=format=yuv420p[dis]; \ [1:v]scale_cuda=format=yuv420p[ref]; \ [dis][ref]libvmaf_cuda=log_fmt=json:log_path=output.json " \ -f null - </pre>

 <h3 class="section">39.151 limitdiff# TOC</h3> Apply limited difference filter using second and optionally third video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set the threshold to use when allowing certain differences between video streams. Any absolute difference value lower or exact than this threshold will pick pixel components from first video stream. </dd> <dt><samp class="option">elasticity</samp></dt> <dd> Set the elasticity of soft thresholding when processing video streams. This value multiplied with first one sets second threshold. Any absolute difference value greater or exact than second threshold will pick pixel components from second video stream. For values between those two threshold linear interpolation between first and second video stream will be used. </dd> <dt><samp class="option">reference</samp></dt> <dd> Enable the reference (third) video stream processing. By default is disabled. If set, this video stream will be used for calculating absolute difference with first video stream. </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available.

</dd> </dl>

 <h4 class="subsection">39.151.1 Commands# TOC</h4>

This filter supports the all above options as commands except option ‘<samp class="samp">reference</samp>’. <h3 class="section">39.152 limiter# TOC</h3>

Limits the pixel components values to the specified range [min, max]. The filter accepts the following options: <dl class="table"> <dt><samp class="option">min</samp></dt> <dd> Lower bound. Defaults to the lowest representable value for the input. Set to <var class="var">auto</var> to use the minimum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class="option">max</samp></dt> <dd> Upper bound. Defaults to the highest representable value for the input. Set to <var class="var">auto</var> to use the maximum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class="subsection">39.152.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.153 loop# TOC</h3>

Loop video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal size in number of frames. Default is 0. </dd> <dt><samp class="option">start</samp></dt> <dd> Set first frame of loop. Default is 0. </dd> <dt><samp class="option">time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class="var">start</var> is set to <code class="code">-1</code>. </dd> </dl>

 <h4 class="subsection">39.153.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Loop single first frame infinitely: <pre class="example-preformatted">loop=loop=-1:size=1:start=0 </pre>

Loop single first frame 10 times: <pre class="example-preformatted">loop=loop=10:size=1:start=0 </pre>

Loop 10 first frames 5 times: <pre class="example-preformatted">loop=loop=5:size=10:start=0 </pre>

 <h3 class="section">39.154 lut1d# TOC</h3>

Apply a 1D LUT to an input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> Set the 1D LUT file name. Currently supported formats: <dl class="table"> <dt>‘<samp class="samp">cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class="samp">csp</samp>’</dt> <dd> cineSpace </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> Interpolate values using the linear interpolation. </dd> <dt>‘<samp class="samp">cosine</samp>’</dt> <dd> Interpolate values using the cosine interpolation. </dd> <dt>‘<samp class="samp">cubic</samp>’</dt> <dd> Interpolate values using the cubic interpolation. </dd> <dt>‘<samp class="samp">spline</samp>’</dt> <dd> Interpolate values using the spline interpolation. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.154.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.155 lut3d<a class="anchor

hidden-xs" href="#lut3d" aria-hidden="true"># TOC</h3>

Apply a 3D LUT to an input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> Set the 3D LUT file name. Currently supported formats: <dl class="table"> <dt>‘<samp class="samp">3dl</samp>’</dt> <dd> AfterEffects </dd> <dt>‘<samp class="samp">cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class="samp">dat</samp>’</dt> <dd> DaVinci </dd> <dt>‘<samp class="samp">m3d</samp>’</dt> <dd> Pandora </dd> <dt>‘<samp class="samp">csp</samp>’</dt> <dd> cineSpace </dd> </dl> </dd> <dt><samp class="option">interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class="samp">trilinear</samp>’</dt> <dd> Interpolate values using the 8 points defining a cube. </dd> <dt>‘<samp class="samp">tetrahedral</samp>’</dt> <dd> Interpolate values using a tetrahedron. </dd> <dt>‘<samp class="samp">pyramid</samp>’</dt> <dd> Interpolate values using a pyramid. </dd> <dt>‘<samp class="samp">prism</samp>’</dt> <dd> Interpolate values using a prism. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.155.1 Commands# TOC</h4>

This filter supports the <code class="code">interp</code> option as commands. <h3 class="section">39.156 lumakey# TOC</h3>

Turn certain luma values into transparency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set the luma which will be used as base for transparency. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set the range of luma values to be keyed out. Default value is <code class="code">0.01</code>. </dd> <dt><samp class="option">softness</samp></dt> <dd> Set the range of softness. Default value is <code class="code">0</code>. Use this to control gradual transition from zero to full transparency. </dd> </dl>

 <h4 class="subsection">39.156.1 Com-

mands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.157 lut, lutrgb, lutyuv# TOC</h3>

Compute a look-up table for binding each pixel component input value to an output value, and apply it to the input video. <var class="var">lutyuv</var> applies a lookup table to a YUV input video, <var class="var">lutrgb</var> to an RGB input video. These filters accept the following parameters: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class="option">r</samp></dt> <dd> set red component expression </dd> <dt><samp class="option">g</samp></dt> <dd> set green component expression </dd> <dt><samp class="option">b</samp></dt> <dd> set blue component expression </dd> <dt><samp class="option">a</samp></dt> <dd> alpha component expression </dd> <dt><samp class="option">y</samp></dt> <dd> set Y/luma component expression </dd> <dt><samp class="option">u</samp></dt> <dd> set U/Cb component expression </dd> <dt><samp class="option">v</samp></dt> <dd> set V/Cr component expression </dd> </dl>

Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class="var">c*</var> options depends on the format in input. The <var class="var">lut</var> filter requires either YUV or RGB pixel formats in input, <var class="var">lutrgb</var> requires RGB pixel formats in input, and <var class="var">lutyuv</var> requires YUV. The expressions can contain the following constants and functions: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp

class=”option”>clipval</samp></dt> <dd> The input value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>maxval</samp></dt> <dd> The maximum value for the pixel component. </dd> <dt><samp class=”option”>minval</samp></dt> <dd> The minimum value for the pixel component. </dd> <dt><samp class=”option”>negval</samp></dt> <dd> The negated value for the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range; it corresponds to the expression "maxval-clipval+minval". </dd> <dt><samp class=”option”>clip(val)</samp></dt> <dd> The computed value in <var class=”var”>val</var>, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>gammaval(gamma)</samp></dt> <dd> The computed gamma correction value of the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. It corresponds to the expression "pow((clipval-minval)/(maxval-minval)\,<var class=”var”>gamma</var>)*(maxval-minval)+minval" </dd> </dl>

All expressions default to "clipval". <h4 class=”subsection”>39.157.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>39.157.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Negate input video: <pre class=”example-preformatted”>lutrgb="r=maxval+minval-val:g=maxval+minval-val:b=maxval+minval-val" lutyuv="y=maxval+minval-val:u=maxval+minval-val:v=maxval+minval-val" </pre>

The above is the same as: <pre class=”example-preformatted”>lutrgb="r=negval:g=negval:b=negval" lutyuv="y=negval:u=negval:v=negval" </pre>

Negate luma: <pre class=”example-preformatted”>lutyuv=y=negval </pre>

Remove chroma components, turning the video into a graytone image: <pre class=”example-preformatted”>lutyuv="u=128:v=128" </pre>

Apply a luma burning effect: <pre class=”example-preformatted”>lutyuv="y=2*val" </pre>

Remove green and blue components: <pre class=”example-preformatted”>lutrgb="g=0:b=0" </pre>

Set a constant alpha channel value on input: <pre class=”example-preformatted”>format=rgba,lutrgb=a="maxval-minval/2" </pre>

Correct luma gamma by a factor of 0.5: <pre class=”example-preformatted”>lutyuv=y=gammaval(0.5) </pre>

Discard least significant bits of luma: <pre class=”example-preformatted”>lutyuv=y='bitand(val, 128+64+32)' </pre>

Technicolor like effect: <pre class=”example-preformatted”>lutyuv=u='(val-maxval/2)*2+maxval/2':v='(val-maxval/2)*2+maxval/2' </pre>

 <h3 class=”section”>39.158 lut2, tlut2# TOC</h3>

The <code class=”code”>lut2</code> filter takes two input streams and outputs one stream. The <code class=”code”>tlut2</code> (time lut2) filter takes two consecutive frames from one single stream. This filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class=”option”>c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class=”option”>c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class=”option”>c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class=”option”>d</samp></dt> <dd> set output bit depth, only available for <code class=”code”>lut2</code> filter. By default is 0, which means bit depth is automatically picked from first input format. </dd> </dl>

The <code class=”code”>lut2</code> filter also supports the framesync options. Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in inputs. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dd> The first input value for the pixel component. </dd> <dt><samp class=”option”>y</samp></dt> <dd> The second input

value for the pixel component. </dd> <dt><samp class="option">bdx</samp></dt> <dd> The first input video bit depth. </dd> <dt><samp class="option">bdy</samp></dt> <dd> The second input video bit depth. </dd> </dl>

All expressions default to "x". <h4 class="subsection">39.158.1 Commands# TOC</h4>

This filter supports the all above options as commands except option <code class="code">d</code>. <h4 class="subsection">39.158.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Highlight differences between two RGB video streams: <pre class="example-preformatted">lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1)' </pre>

Highlight differences between two YUV video streams: <pre class="example-preformatted">lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1)' </pre>

Show max difference between two video streams: <pre class="example-preformatted">lut2='if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1)))' </pre>

 <h3 class="section">39.159 maskedclamp# TOC</h3>

Clamp the first input stream with the second input and third input stream. Returns the value of first stream to be between second input stream - <code class="code">undershoot</code> and third input stream + <code class="code">overshoot</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">undershoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">overshoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">39.159.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.160 maskedmax# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is greater than first one or from third input stream otherwise. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">39.160.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.161 maskedmerge# TOC</h3>

Merge the first input stream with the second input stream using per pixel weights in the third input stream. A value of 0 in the third stream pixel component means that pixel component from first stream is returned unchanged, while maximum value (eg. 255 for 8-bit videos) means that pixel component from second stream is returned unchanged. Intermediate values define the amount of merging between both input stream’s pixel components. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">39.161.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.162 maskedmin# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is less than first one or from third input stream otherwise. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class=”subsection”>39.162.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.163 maskedthreshold# TOC</h3>
Pick pixels comparing absolute difference of two video streams with fixed threshold. If absolute difference between pixel component of first and second video stream is equal or lower than user supplied threshold than pixel component from first video stream is picked, otherwise pixel component from second video stream is picked. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold used when picking pixels from absolute difference from two input video streams. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from second stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of filter operation. Can be <code class=”code”>abs</code> or <code class=”code”>diff</code>. Default is <code class=”code”>abs</code>. </dd> </dl>

 <h4 class=”subsection”>39.163.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h3 class="section">39.164 maskfun# TOC</h3> Create mask from input video. For example it is useful to create motion masks after <code class="code">tblend</code> filter. This filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dd> Set low threshold. Any pixel component lower or exact than this value will be set to 0. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high threshold. Any pixel component higher than this value will be set to max value allowed for current pixel format. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all available planes are filtered. </dd> <dt><samp class="option">fill</samp></dt> <dd> Fill all frame pixels with this value. </dd> <dt><samp class="option">sum</samp></dt> <dd> Set max average pixel value for frame. If sum of all pixel components is higher that this average, output frame will be completely filled with value set by <var class="var">fill</var> option. Typically useful for scene changes when used in combination with <code class="code">tblend</code> filter. </dd> </dl>

 <h4 class="subsection">39.164.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.165 mcdeint# TOC</h3>

Apply motion-compensation deinterlacing. It needs one field per frame as input and must thus be used together with yadif=1/3 or equivalent. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set the deinterlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">medium</samp>’</d <dt>‘<samp class="samp">slow</samp>’</dt> <dd> use iterative motion estimation </dd> <dt>‘<samp class="samp">extra_slow</samp>’</d <dd> like ‘<samp class="samp">slow</samp>’, but use multiple reference frames. </dd> </dl> Default value is ‘<samp class="samp">fast</samp>’. </dd> <dt><samp class="option">parity</samp></dt> <dd> Set the picture field parity assumed for the input video. It must be one of the following

values: <dl class="table"> <dt>‘<samp class="samp">0, tff</samp>’</dt> <dd> assume top field first </dd> <dt>‘<samp class="samp">1, bff</samp>’</dt> <dd> assume bottom field first </dd> </dl>

Default value is ‘<samp class="samp">bff</samp>’. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set per-block quantization parameter (QP) used by the internal encoder. Higher values should result in a smoother motion vector field but less optimal individual vectors. Default value is 1. </dd> </dl>

 <h3 class="section">39.166 median# TOC</h3>

Pick median pixel from certain rectangle defined by radius. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set horizontal radius size. Default value is <code class="code">1</code>. Allowed range is integer from 1 to 127. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> <dt><samp class="option">radiusV</samp></dt> <dd> Set vertical radius size. Default value is <code class="code">0</code>. Allowed range is integer from 0 to 127. If it is 0, value will be picked from horizontal <code class="code">radius</code> option. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">39.166.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">39.167 mergeplanes# TOC</h3>

Merge color channel components from several video streams. The filter accepts up to 4 input streams, and merge selected input planes to the output video. This filter accepts the following options: <dl

class=”table”> <dt><samp class=”option”>mapping</samp></dt> <dd> Set input to output plane mapping. Default is <code class=”code”>0</code>. The mappings is specified as a bitmap. It should be specified as a hexadecimal number in the form 0xAa[Bb[Cc[Dd]]]. ’Aa’ describes the mapping for the first plane of the output stream. ’A’ sets the number of the input stream to use (from 0 to 3), and ’a’ the plane number of the corresponding input to use (from 0 to 3). The rest of the mappings is similar, ’Bb’ describes the mapping for the output stream second plane, ’Cc’ describes the mapping for the output stream third plane and ’Dd’ describes the mapping for the output stream fourth plane. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Set output pixel format. Default is <code class=”code”>yuva444p</code>. </dd> <dt><samp class=”option”>map0s</samp></dt> <dt><samp class=”option”>map1s</samp></dt> <dt><samp class=”option”>map2s</samp></dt> <dt><samp class=”option”>map3s</samp></dt> <dd> Set input to output stream mapping for output Nth plane. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>map0p</samp></dt> <dt><samp class=”option”>map1p</samp></dt> <dt><samp class=”option”>map2p</samp></dt> <dt><samp class=”option”>map3p</samp></dt> <dd> Set input to output plane mapping for output Nth plane. Default is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>39.167.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Merge three gray video streams of same width and height into single video stream: <pre class=”example-preformatted”>[a0][a1][a2]mergeplanes=0x001020:yuv444p </pre>

Merge 1st yuv444p stream and 2nd gray video stream into yuva444p video stream: <pre class=”example-preformatted”>[a0][a1]mergeplanes=0x00010210:yuva444p </pre>

Swap Y and A plane in yuva444p stream: <pre class=”example-preformatted”>format=yuva444p,mergeplanes=0x03010200:yuva444p </pre>

Swap U and V plane in yuv420p stream: <pre class=”example-preformatted”>format=yuv420p,mergeplanes=0x000201:yuv420p </pre>

Cast a rgb24 clip to yuv444p: <pre class=”example-preformatted”>format=rgb24,mergeplanes=0x000102:yuv444p </pre>

 <h3 class="section">39.168 mestimate# TOC</h3>

Estimate and export motion vectors using block matching algorithms. Motion vectors are stored in frame side data to be used by other filters. This filter accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> Specify the motion estimation method. Accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class="samp">tss</samp>’</dt> <dd> Three step search algorithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class="samp">ds</samp>’</dt> <dd> Diamond search algorithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt> <dd> Hexagon-based search algorithm. </dd> <dt>‘<samp class="samp">epzs</samp>’</dt> <dd> Enhanced predictive zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt> <dd> Uneven multi-hexagon search algorithm. </dd> </dl> Default value is ‘<samp class="samp">esa</samp>’. </dd> <dt><samp class="option">mb_size</samp></dt> <dd> Macroblock size. Default <code class="code">16</code>. </dd> <dt><samp class="option">search_param</samp></dt> <dd> Search parameter. Default <code class="code">7</code>. </dd> </dl>

 <h3 class="section">39.169 mestimate_d3d12# TOC</h3>

Estimate and export motion vectors using D3D12 hardware-accelerated motion estimation. This filter uses GPU hardware motion estimation capabilities available in DirectX 12 Video APIs to achieve significant performance improvements over the software-based <code class="code">mestimate</code> filter. Motion vectors are stored in frame side data to be used by other filters. This filter requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter uses quarter-pixel precision for motion vector estimation. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mb_size</samp></dt> <dd> Macroblock size. Only <code class="code">8</code> and <code class="code">16</code> are

supported. Default <code class=”code”>16</code>. </dd> </dl>

 <h4 class=”subsection”>39.169.1 Examples# TOC</h4>

Estimate motion vectors using D3D12 hardware acceleration with 16x16 blocks, and visualize them: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf mestimate_d3d12=mb_size=16,hwdownload,format=nv12,codecview=mv=pf \ -c:v libx264 -preset fast -b:v 5M output.mp4 </pre>

 <h3 class=”section”>39.170 midequalizer# TOC</h3>

Apply Midway Image Equalization effect using two video streams. Midway Image Equalization adjusts a pair of images to have the same histogram, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a pair of stereo cameras. This filter has two inputs and one output, which must be of same pixel format, but may be of different sizes. The output of filter is first input adjusted with midway histogram of both inputs. This filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. Default is <code class=”code”>15</code>, which is all available planes. </dd> </dl>

 <h3 class=”section”>39.171 minterpolate# TOC</h3>

Convert the video to specified frame rate using motion interpolation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frame rate. This can be rational e.g. <code class=”code”>60000/1001</code>. Frames are dropped if <var class=”var”>fps</var> is lower than source fps. Default <code class=”code”>60</code>. </dd> <dt><samp class=”option”>mi_mode</samp></dt> <dd> Motion interpolation mode. Following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>dup</samp>’</dt> <dd> Duplicate previous or next frame for interpolating new ones. </dd> <dt>‘<samp class=”samp”>blend</samp>’</dt> <dd> Blend source frames. Interpolated frame is mean of previous and next frames. </dd> <dt>‘<samp class=”samp”>mci</samp>’</dt> <dd> Motion compensated interpolation. Following options are effective when this mode is se-

lected: <dl class="table"> <dt>‘<samp class="samp">mc_mode</samp>’</dt>
<dd> Motion compensation mode. Following values are accepted:
<dl class="table"> <dt>‘<samp class="samp">obmc</samp>’</dt>
<dd> Overlapped block motion compensation. </dd> <dt>‘<samp
class="samp">aobmc</samp>’</dt> <dd> Adaptive overlapped
block motion compensation. Window weighting coefficients are controlled
adaptively according to the reliabilities of the neighboring motion vectors to reduce oversmoothing. </dd> </dl> Default mode is ‘<samp
class="samp">obmc</samp>’. </dd> <dt>‘<samp
class="samp">me_mode</samp>’</dt> <dd> Motion estimation mode. Following values are accepted: <dl class="table">
<dt>‘<samp class="samp">bidir</samp>’</dt> <dd> Bidirectional
motion estimation. Motion vectors are estimated for each source frame in
both forward and backward directions. </dd> <dt>‘<samp
class="samp">bilat</samp>’</dt> <dd> Bilateral motion
estimation. Motion vectors are estimated directly for interpolated frame.
</dd> </dl> Default mode is ‘<samp class="samp">bilat</samp>’.
 </dd> <dt>‘<samp class="samp">me</samp>’</dt>
<dd> The algorithm to be used for motion estimation. Following values are accepted: <dl class="table"> <dt>‘<samp
class="samp">esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class="samp">tss</samp>’</dt>
<dd> Three step search algorithm. </dd> <dt>‘<samp
class="samp">tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class="samp">ntss</samp>’</dt>
<dd> New three step search algorithm. </dd> <dt>‘<samp
class="samp">fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class="samp">ds</samp>’</dt>
<dd> Diamond search algorithm. </dd> <dt>‘<samp
class="samp">hexbs</samp>’</dt> <dd> Hexagon-based
search algorithm. </dd> <dt>‘<samp class="samp">epzs</samp>’</dt>
<dd> Enhanced predictive zonal search algorithm. </dd>
<dt>‘<samp class="samp">umh</samp>’</dt> <dd> Uneven
multi-hexagon search algorithm. </dd> </dl> Default algorithm is ‘<samp class="samp">epzs</samp>’.
</dd> <dt>‘<samp class="samp">mb_size</samp>’</dt>
<dd> Macroblock size. Default <code class="code">16</code>.
</dd> <dt>‘<samp class="samp">search_param</samp>’</dt>
<dd> Motion estimation search parameter. Default <code class="code">32</code>.
 </dd> <dt>‘<samp class="samp">vsbmc</samp>’</dt>
<dd> Enable variable-size block motion compensation. Motion estimation
is applied with smaller block sizes at object boundaries in order to make them
less blurry. Default is <code class="code">0</code> (disabled). </dd>
</dl> </dd> </dl>

</dd> <dt><samp class="option">scd</samp></dt> <dd> Scene

change detection method. Scene change leads motion vectors to be in random direction. Scene change detection replace interpolated frames by duplicate ones. May not be needed for other modes. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable scene change detection. </dd> <dt>‘<samp class="samp">fdiff</samp>’</dt> <dd> Frame difference. Corresponding pixel values are compared and if it satisfies <var class="var">scd_threshold</var> scene change is detected. </dd> </dl> Default method is ‘<samp class="samp">fdiff</samp>’. </dd> <dt><samp class="option">scd_threshold</samp></ <dd> Scene change detection threshold. Default is <code class="code">10.</code>. </dd> </dl>

 <h3 class="section">39.172 mix# TOC</h3>

Mix several video input streams into one video stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input video stream as sequence. Each weight is separated by space. If number of weights is smaller than number of <var class="var">frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> <dt><samp class="option">duration</samp></dt> <dd> Specify how end of stream is determined. <dl class="table"> <dt>‘<samp class="samp">longest</samp>’</dt> <dd> The duration of the longest input. (default) </dd> <dt>‘<samp class="samp">shortest</samp>’</dt> <dd> The duration of the shortest input. </dd> <dt>‘<samp class="samp">first</samp>’</dt> <dd> The duration of the first input. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.172.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">39.173 monochrome# TOC</h3> Convert video to gray using custom color filter. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">cb</samp></dt> <dd> Set the chroma blue spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">cr</samp></dt> <dd> Set the chroma red spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the color filter size. Allowed range is from .1 to 10. Default value is 1. </dd> <dt><samp class="option">high</samp></dt> <dd> Set the highlights strength. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">39.173.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.174 morpho# TOC</h3>

This filter allows to apply main morphological grayscale transforms, erode and dilate with arbitrary structures set in second input stream. Unlike naive implementation and much slower performance in erosion and dilation filters, when speed is critical <code class="code">morpho</code> filter should be used instead. A description of accepted options follows, <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set morphological transform to apply, can be: <dl class="table"> <dt>‘<samp class="samp">erode</samp>’</dt> <dt>‘<samp class="samp">dilate</samp>’</d <dt>‘<samp class="samp">open</samp>’</dt> <dt>‘<samp class="samp">close</samp>’</dt> <dt>‘<samp class="samp">gradient</samp>’< <dt>‘<samp class="samp">tophat</samp>’</dt> <dt>‘<samp class="samp">blackhat</samp>’</dt> </dl> Default is <code class="code">erode</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all planes except alpha are filtered. </dd> <dt><samp class="option">structure</samp></dt> <dd> Set which structure video frames will be processed from second input stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">morpho</code> filter also supports the

framesync options. <h4 class=”subsection”>39.174.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>39.175 mpdecimate# TOC</h3>

Drop frames that do not differ greatly from the previous frame in order to reduce frame rate. The main use of this filter is for very-low-bitrate encoding (e.g. streaming over dialup modem), but it could in theory be used for fixing movies that were inverse-telecined incorrectly. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode. If the value is 0, the filter drops similar frames. If the value is 1, the filter keeps only frames that are one representative of sufficiently long duplicate runs. Default value is 0. </dd> <dt><samp class=”option”>max</samp></dt> <dd> For mode 0: Set the maximum number of consecutive frames which can be dropped (if positive), or the minimum interval between dropped frames (if negative). If the value is 0, the frame is dropped disregarding the number of previous sequentially dropped frames. Default value is 0. </dd> <dt><samp class=”option”>min</samp></dt> <dd> For mode 1: Set the minimum number of frame duplicates to mark the frame as non-unique and keep the first occurrence of it. Default value is 1. </dd> <dt><samp class=”option”>keep</samp></dt> <dd> For mode 0: Set the maximum number of consecutive similar frames to ignore before to start dropping them. If the value is 0, the frame is dropped disregarding the number of previous sequentially similar frames. Default value is 0. </dd> <dt><samp class=”option”>hi</samp></dt> <dt><samp class=”option”>lo</samp></dt> <dt><samp class=”option”>frac</samp></dt> <dd> Set the dropping threshold values. Values for <samp class=”option”>hi</samp> and <samp class=”option”>lo</samp> are for 8x8 pixel blocks and represent actual pixel value differences, so a threshold of 64 corresponds to 1 unit of difference for each pixel, or the same spread out differently over the block. A frame is a candidate for dropping if no 8x8 blocks differ by more than a threshold of <samp class=”option”>hi</samp>, and if no more than <samp class=”option”>frac</samp> blocks (1 meaning the whole image) differ by more than a threshold of <samp class=”option”>lo</samp>. Default value for <samp class=”option”>hi</samp> is 64*12, default value for <samp class=”option”>lo</samp> is 64*5, and default value for <samp class=”option”>frac</samp> is 0.33. </dd> </dl>

 <h3 class="section">39.176 msad# TOC</h3>

Obtain the MSAD (Mean Sum of Absolute Differences) between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max MSAD is printed through the logging system. The filter stores the calculated MSAD of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi msad -f null - </pre>

 <h3 class="section">39.177 multiply# TOC</h3> Multiply first video stream pixels values with second video stream pixels values. The filter accepts the following options: <dl class="table"> <dt><samp class="option">scale</samp></dt> <dd> Set the scale applied to second video stream. By default is <code class="code">1</code>. Allowed range is from <code class="code">0</code> to <code class="code">9</code>. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set the offset applied to second video stream. By default is <code class="code">0.5</code>. Allowed range is from <code class="code">-1</code> to <code class="code">1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify planes from input video stream that will be processed. By default all planes are processed. </dd> </dl>

 <h4 class="subsection">39.177.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">39.178 negate# TOC</h3>

Negate (invert) the input video. It accepts the following option: <dl class="table"> <dt><samp class="option">components</samp></dt> <dd> Set components to negate. Available values for compo-

nents are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

</dd> <dt><samp class="option">negate_alpha</samp></dt> <dd> With value 1, it negates the alpha component, if present. Default value is 0. </dd> </dl>

 <h4 class="subsection">39.178.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">39.179 nlmeans# TOC</h3>

Denoise frames using Non-Local Means algorithm. Each pixel is adjusted by looking for other pixels with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>x<samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp>x<samp class="option">r</samp> around the pixel. Note that the research area defines centers for patches, which means some patches will be made of pixels outside that research area. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength. Default is 1.0. Must be in range [1.0, 30.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">pc</samp></dt> <dd> Same as <samp class="option">p</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic. </dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">rc</samp></dt> <dd> Same as <samp class="option">r</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic. </dd> </dl>

 <h3 class="section">39.180 nnedi# TOC</h3>

Deinterlace video using neural network edge directed interpola-

tion. This filter accepts the following options: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dd> Mandatory option, without binary file filter can not work. Currently file can be found here: https://github.com/dubhater/vapoursynth-nnedi3/blob/master/src/nnedi3_weights.bin </dd> <dt><samp class="option">deint</samp></dt> <dd> Set which frames to deinterlace, by default it is <code class="code">all</code>. Can be <code class="code">all</code> or <code class="code">interlaced</code>. </dd> <dt><samp class="option">field</samp></dt> <dd> Set mode of operation. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">af</samp>’</dt> <dd> Use frame flags, both fields. </dd> <dt>‘<samp class="samp">a</samp>’</dt> <dd> Use frame flags, single field. </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> Use top field only. </dd> <dt>‘<samp class="samp">b</samp>’</dt> <dd> Use bottom field only. </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> Use both fields, top first. </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> Use both fields, bottom first. </dd> </dl>

</dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process, by default filter process all frames. </dd> <dt><samp class="option">nsize</samp></dt> <dd> Set size of local neighborhood around each pixel, used by the predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">s8x6</samp>’</dt> <dt>‘<samp class="samp">s16x6</samp>’</dt> <dt>‘<samp class="samp">s32x6</samp>’</d <dt>‘<samp class="samp">s48x6</samp>’</dt> <dt>‘<samp class="samp">s8x4</samp>’</dt> <dt>‘<samp class="samp">s16x4</samp>’</dt <dt>‘<samp class="samp">s32x4</samp>’</dt> </dl>

</dd> <dt><samp class="option">nns</samp></dt> <dd> Set the number of neurons in predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">n16</samp>’</dt> <dt>‘<samp class="samp">n32</samp>’</dt> <dt>‘<samp class="samp">n64</samp>’</dt> <dt>‘<samp class="samp">n128</samp>’</dt> <dt>‘<samp class="samp">n256</samp>’</dt> </dl>

</dd> <dt><samp class="option">qual</samp></dt> <dd> Controls the number of different neural network predictions that are blended together to compute the final output value. Can be <code class="code">fast</code>, default or <code class="code">slow</code>. </dd> <dt><samp class="option">etype</samp></dt> <dd> Set which set of weights to use in the predictor. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">a, abs</samp>’</dt> <dd> weights trained to minimize absolute error </dd> <dt>‘<samp class="samp">s, mse</samp>’</dt> <dd> weights trained

to minimize squared error </dd> </dl>

</dd> <dt><samp class="option">pscrn</samp></dt> <dd> Controls whether or not the prescreener neural network is used to decide which pixels should be processed by the predictor neural network and which can be handled by simple cubic interpolation. The prescreener is trained to know whether cubic interpolation will be sufficient for a pixel or whether it should be predicted by the predictor nn. The computational complexity of the prescreener nn is much less than that of the predictor nn. Since most pixels can be handled by cubic interpolation, using the prescreener generally results in much faster processing. The prescreener is pretty accurate, so the difference between using it and not using it is almost always unnoticeable. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">original</samp>’</
<dt>‘<samp class="samp">new</samp>’</dt> <dt>‘<samp
class="samp">new2</samp>’</dt> <dt>‘<samp class="samp">new3</samp>’</dt
</dl>

Default is <code class="code">new</code>. </dd> </dl>

 <h4 class="subsection">39.180.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">weights</var> option.
 <h3 class="section">39.181 noformat# TOC</h3>

Force libavfilter not to use any of the specified pixel formats for the input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as pix_fmts=yuv420p|monow|rgb24". </dd> </dl>

 <h4 class="subsection">39.181.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Force libavfilter to use a format different from <var class="var">yuv420p</var> for the input to the vflip filter: <pre class="example-preformatted">noformat=pix_fmts=yuv420p,vflip
</pre>

Convert the input video to any of the formats not contained in the list:
 <pre class="example-preformatted">noformat=yuv420p|yuv444p|yuv410p

</pre>

 <h3 class="section">39.182 noise# TOC</h3>

Add noise on video input frame. The filter accepts the following options: <dl class="table"> <dt><samp class="option">all_seed</samp></dt> <dt><samp class="option">c0_seed</samp></dt> <dt><samp class="option">c1_seed</samp></dt> <dt><samp class="option">c2_seed</samp></dt> <dt><samp class="option">c3_seed</samp></dt> <dd> Set noise seed for specific pixel component or all pixel components in case of <var class="var">all_seed</var>. Default value is <code class="code">123457</code>. </dd> <dt><samp class="option">all_strength, alls</samp></dt> <dt><samp class="option">c0_strength, c0s</samp></dt> <dt><samp class="option">c1_strength, c1s</samp></dt> <dt><samp class="option">c2_strength, c2s</samp></dt> <dt><samp class="option">c3_strength, c3s</samp></dt> <dd> Set noise strength for specific pixel component or all pixel components in case <var class="var">all_strength</var>. Default value is <code class="code">0</code>. Allowed range is [0, 100]. </dd> <dt><samp class="option">all_flags, allf</samp></dt> <dt><samp class="option">c0_flags, c0f</samp></dt> <dt><samp class="option">c1_flags, c1f</samp></dt> <dt><samp class="option">c2_flags, c2f</samp></dt> <dt><samp class="option">c3_flags, c3f</samp></dt> <dd> Set pixel component flags or set flags for all components if <var class="var">all_flags</var>. Available values for component flags are: <dl class="table"> <dt>‘<samp class="samp">a</samp>’</dt> <dd> averaged temporal noise (smoother) </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> mix random noise with a (semi)regular pattern </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> temporal noise (noise pattern changes between frames) </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> uniform noise (gaussian otherwise) </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.182.1 Examples# TOC</h4>

Add temporal and uniform noise to input video: <pre class="example-preformatted">noise=alls=20:allf=t+u </pre>

 <h3 class="section">39.183 normalize# TOC</h3>

Normalize RGB video (aka histogram stretching, contrast stretching).

See: https://en.wikipedia.org/wiki/Normalization_(image_processing) For each channel of each frame, the filter computes the input range and maps it linearly to the user-specified output range. The output range defaults to the full dynamic range from pure black to pure white. Temporal smoothing can be used on the input range to reduce flickering (rapid changes in brightness) caused when small dark or bright objects enter or leave the scene. This is similar to the auto-exposure (automatic gain control) on a video camera, and, like a video camera, it may cause a period of over- or under-exposure of the video. The R,G,B channels can be normalized independently, which may cause some color shifting, or linked together as a single channel, which prevents color shifting. Linked normalization preserves hue. Independent normalization does not, so it can be used to remove some color casts. Independent and linked normalization can be combined in any ratio. The normalize filter accepts the following options: <dl class="table"> <dt><samp class="option">blackpt</samp></dt> <dt><samp class="option">whitept</samp></dt> <dd> Colors which define the output range. The minimum input value is mapped to the <var class="var">blackpt</var>. The maximum input value is mapped to the <var class="var">whitept</var>. The defaults are black and white respectively. Specifying white for <var class="var">blackpt</var> and black for <var class="var">whitept</var> will give color-inverted, normalized video. Shades of grey can be used to reduce the dynamic range (contrast). Specifying saturated colors here can create some interesting effects. </dd> <dt><samp class="option">smoothing</samp></dt> <dd> The number of previous frames to use for temporal smoothing. The input range of each channel is smoothed using a rolling average over the current frame and the <var class="var">smoothing</var> previous frames. The default is 0 (no temporal smoothing). </dd> <dt><samp class="option">independence</samp></dt> <dd> Controls the ratio of independent (color shifting) channel normalization to linked (color preserving) normalization. 0.0 is fully linked, 1.0 is fully independent. Defaults to 1.0 (fully independent). </dd> <dt><samp class="option">strength</samp></dt> <dd> Overall strength of the filter. 1.0 is full strength. 0.0 is a rather expensive no-op. Defaults to 1.0 (full strength). </dd> </dl>

 <h4 class="subsection">39.183.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">smoothing</var> option. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">39.183.2 Examples<a class="anchor hidden-xs" href="#Examples-

151" aria-hidden="true"># TOC</h4>

Stretch video contrast to use the full dynamic range, with no temporal smoothing; may flicker depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=
</pre>

As above, but with 50 frames of temporal smoothing; flicker should be reduced, depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50
</pre>

As above, but with hue-preserving linked channel normalization: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:
</pre>

As above, but with half strength: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50:independence=0:stre
</pre>

Map the darkest input color to red, the brightest input color to cyan: <pre class="example-preformatted">normalize=blackpt=red:whitept=cyan
</pre>

 <h3 class="section">39.184 null#
TOC</h3>

Pass the video source unchanged to the output.
<h3 class="section">39.185 ocio# TOC</h3>
OpenColorIO library filter This filter allows you to do color management using the OpenColorIO library. See https://opencolorio.org/ for more details. To Enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libopencolorio</code>.
It accepts the following options: <dl class="table"> <dt><samp class="option">config</samp></dt> <dd> By default the filter will use the OCIO config defined by the OCIO environment variable, but this parameter allows you to explicitly specify its location. If you are getting started, you can use config=ocio://studio-config-v1.0.0_aces-v1.3_ocio-v2.1 which specifies one of the built in defaults. </dd> <dt><samp class="option">input</samp></dt> <dd> Set the input colorspace. </dd> <dt><samp class="option">output</samp></dt> <dd> Set the output colorspace. </dd> <dt><samp class="option">display</samp></dt>
<dd> Set the display colorspace, used in combination with view. </dd> <dt><samp class="option">view</samp></dt> <dd> Set the view colorspace, used in combination with display. </dd> <dt><samp

class=”option”>inverse</samp></dt> <dd> When used in combination with display and view, this inverts the transform, so going from a display/view to the "input colorspace". </dd> <dt><samp class=”option”>filetransform</samp></dt> <dd> Allows you to specify an external file-transform to use instead of the OCIO config file. This is useful for applying a single transform without needing a full OCIO config file. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Allow you to specify the output pix_fmt of the OCIO filter. This *has* to be a RGB colorspace, so you are limited to rgb24, rgba, rgb48, rgba64, gbrp10, gbrap10, gbrp12, gbrap12, gbrpf32 and gbrapf32; for most encoding we would recommend rgb48. Half-float (gbrpf16, gbrapf16) is accepted as input but is not a valid output format, because swscale has no half-float write support and its legacy half-float read path clamps values to [0,1]. When this option is not set, the output format defaults to the input format, except that half-float input is promoted to the matching float format. Where negotiation cannot yet tell whether the input will be half-float, the output falls back to float, so a float output may be selected even when the input turns out to be an integer format. </dd> <dt><samp class=”option”>context_params</samp></dt> <dd> Allow you to specify additional context parameters for the OCIO filter. This is a list of key=value pairs, separated by colons. </dd> </dl>

 <h4 class=”subsection”>39.185.1 Examples# TOC</h4>

Map from ACEScg to ACEScct, this assumes the OCIO file is defined with the OCIO environment variable. <pre class=”example-preformatted”>input=ACEScg:output=ACEScct:format=rgb48 </pre>

Map from ACEScg to a sRGB display using the "ACES 1.0 - SDR Video" view transform, this assumes the OCIO file is defined with the OCIO environment variable. Note you will need to wrap the argument in quotes to ensure that the spaces are interpreted correctly. <pre class=”example-preformatted”>input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

As above but using the OCIO file studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio rather than the OCIO environment variable. <pre class=”example-preformatted”>config=studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio:input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

If you are converting to YCrCb you still will want to set the color matrix for the conversion. This is a good example of combining the two. <div

class=”example”> <pre class=”example-preformatted”>ffmpeg -y -i SOURCE-FRAMES.%05d.exr -c:v libx265 -vf "ocio=input=ACEScg:output=ACEScct:format=rgb48,scale=in_col OUTPUTFILE.mov </pre>

 <h3 class=”section”>39.186 ocr# TOC</h3> Optical Character Recognition This filter uses Tesseract for optical character recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libtesseract</code>. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>datapath</samp></dt> <dd> Set datapath to tesseract data. Default is to use whatever was set at installation. </dd> <dt><samp class=”option”>language</samp></dt> <dd> Set language, default is "eng". </dd> <dt><samp class=”option”>whitelist</samp></dt> <dd> Set character whitelist. </dd> <dt><samp class=”option”>blacklist</samp></dt> <dd> Set character blacklist. </dd> </dl>

The filter exports recognized text as the frame metadata <code class=”code”>lavfi.ocr.text</code>. The filter exports confidence of recognized words as the frame metadata <code class=”code”>lavfi.ocr.confidence</code>. <h3 class=”section”>39.187 ocv# TOC</h3>

Apply a video transform using libopencv. To enable this filter, install the libopencv library and headers and configure FFmpeg with <code class=”code”>--enable-libopencv</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the libopencv filter to apply. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> The parameters to pass to the libopencv filter. If not specified, the default values are assumed. </dd> </dl>

Refer to the official libopencv documentation for more precise information: <a class=”url” href=”http://docs.opencv.org/master/modules/imgproc/doc/filtering.html”>http://docs.open Several libopencv filters are supported; see the following subsections. <h4 class=”subsection”>39.187.1 dilate# TOC</h4>

Dilate an image by using a specific structuring element. It corresponds to the libopencv function <code class=”code”>cvDilate</code>. It accepts the parameters: <var class=”var”>struct_el</var>|<var class=”var”>nb_iterations</var>. <var class=”var”>struct_el</var> represents a structuring element, and has the syntax: <var class=”var”>cols</var>x<var

class=”var”>rows</var>+<var class=”var”>anchor_x</var>x<var class=”var”>anchor_y</var>/<var class=”var”>shape</var> <var class=”var”>cols</var> and <var class=”var”>rows</var> represent the number of columns and rows of the structuring element, <var class=”var”>anchor_x</var> and <var class=”var”>anchor_y</var> the anchor point, and <var class=”var”>shape</var> the shape for the structuring element. <var class=”var”>shape</var> must be "rect", "cross", "ellipse", or "custom". If the value for <var class=”var”>shape</var> is "custom", it must be followed by a string of the form "=<var class=”var”>filename</var>". The file with name <var class=”var”>filename</var> is assumed to represent a binary image, with each printable character corresponding to a bright pixel. When a custom <var class=”var”>shape</var> is used, <var class=”var”>cols</var> and <var class=”var”>rows</var> are ignored, the number or columns and rows of the read file are assumed instead. The default value for <var class=”var”>struct_el</var> is "3x3+0x0/rect". <var class=”var”>nb_iterations</var> specifies the number of times the transform is applied to the image, and defaults to 1. Some examples: <pre class=”example-preformatted”># Use the default values ocv=dilate

Dilate using a structuring element with a 5x5 cross, iterating two times ocv=filter_name=dilate:filter_params=5x5+2x2/cross|2

Read the shape from the file diamond.shape, iterating two times. # The file diamond.shape may contain a pattern of characters like this # * # *** # ***** # *** # * # The specified columns and rows are ignored # but the anchor point coordinates are not ocv=dilate:0x0+2x2/custom=diamond.shape|2 </pre>

 <h4 class=”subsection”>39.187.2 erode# TOC</h4>

Erode an image by using a specific structuring element. It corresponds to the libopencv function <code class=”code”>cvErode</code>. It accepts the parameters: <var class=”var”>struct_el</var>:<var class=”var”>nb_iterations</var>, with the same syntax and semantics as the dilate filter. <h4 class=”subsection”>39.187.3 smooth# TOC</h4>

Smooth the input video. The filter takes the following parameters: <var class=”var”>type</var>|<var class=”var”>param1</var>|<var class=”var”>param2</var>|<var class=”var”>param3</var>|<var class=”var”>param4</var>.

 <var class="var">type</var> is the type of smooth filter to apply, and must be one of the following values: "blur", "blur_no_scale", "median", "gaussian", or "bilateral". The default value is "gaussian". The meaning of <var class="var">param1</var>, <var class="var">param2</var>, <var class="var">param3</var>, and <var class="var">param4</var> depends on the smooth type. <var class="var">param1</var> and <var class="var">param2</var> accept integer positive values or 0. <var class="var">param3</var> and <var class="var">param4</var> accept floating point values. The default value for <var class="var">param1</var> is 3. The default value for the other parameters is 0. These parameters correspond to the parameters assigned to the libopencv function <code class="code">cvSmooth</code>. <h3 class="section">39.188 oscilloscope# TOC</h3>

2D Video Oscilloscope. Useful to measure spatial impulse, step responses, chroma delays, etc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set scope center x position. </dd> <dt><samp class="option">y</samp></dt> <dd> Set scope center y position. </dd> <dt><samp class="option">s</samp></dt> <dd> Set scope size, relative to frame diagonal. </dd> <dt><samp class="option">t</samp></dt> <dd> Set scope tilt/rotation. </dd> <dt><samp class="option">o</samp></dt> <dd> Set trace opacity. </dd> <dt><samp class="option">tx</samp></dt> <dd> Set trace center x position. </dd> <dt><samp class="option">ty</samp></dt> <dd> Set trace center y position. </dd> <dt><samp class="option">tw</samp></dt> <dd> Set trace width, relative to width of frame. </dd> <dt><samp class="option">th</samp></dt> <dd> Set trace height, relative to height of frame. </dd> <dt><samp class="option">c</samp></dt> <dd> Set which components to trace. By default it traces first three components. </dd> <dt><samp class="option">g</samp></dt> <dd> Draw trace grid. By default is enabled. </dd> <dt><samp class="option">st</samp></dt> <dd> Draw some statistics. By default is enabled. </dd> <dt><samp class="option">sc</samp></dt> <dd> Draw scope. By default is enabled. </dd> </dl>

 <h4 class="subsection">39.188.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the correspond-

ing option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">39.188.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Inspect full first row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=0:s=1 </pre>

Inspect full last row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=1:s=1 </pre>

Inspect full 5th line of video frame of height 1080. <pre class="example-preformatted">oscilloscope=x=0.5:y=5/1080:s=1 </pre>

Inspect full last column of video frame. <pre class="example-preformatted">oscilloscope=x=1:y=0.5:s=1:t=1 </pre>

 <h3 class="section">39.189 overlay# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: A description of the accepted options follows. <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set the expression for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. In case the expression is invalid, it is set to a huge value (meaning that the overlay will not be displayed within the output visible area). </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for <samp class="option">x</samp>, and <samp class="option">y</samp> are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">frame</samp>’.

 </dd> <dt><samp class="option">shortest</samp></dt>
<dd> See framesync.
</dd> <dt><samp class="option">format</samp></dt> <dd> Set
the format for the output video. It accepts the following values:
<dl class="table"> <dt>‘<samp class="samp">yuv420</samp>’</dt>
<dd> force YUV 4:2:0 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv420p10</samp>’</dt> <dd> force YUV 4:2:0
10-bit planar output </dd> <dt>‘<samp class="samp">yuv422</samp>’</dt>
<dd> force YUV 4:2:2 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv422p10</samp>’</dt> <dd> force YUV 4:2:2
10-bit planar output </dd> <dt>‘<samp class="samp">yuv444</samp>’</dt>
<dd> force YUV 4:4:4 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv444p10</samp>’</dt> <dd> force YUV 4:4:4
10-bit planar output </dd> <dt>‘<samp class="samp">rgb</samp>’</dt>
<dd> force RGB 8-bit packed output </dd> <dt>‘<samp
class="samp">gbrp</samp>’</dt> <dd> force RGB 8-bit planar
output </dd> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> automatically pick format </dd> </dl>

Default value is ‘<samp class="samp">yuv420</samp>’.
 </dd> <dt><samp class="option">repeatlast</samp></dt>
<dd> See framesync.
</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set for-
mat of alpha of the overlaid video, it can be <var class="var">straight</var>
or <var class="var">premultiplied</var>, or <var class="var">auto</var>
to choose the alpha mode automatically. Default is <var class="var">auto</var>.
</dd> </dl>

The <samp class="option">x</samp>, and <samp class="option">y</samp>
expressions can contain the following parameters. <dl class="table">
<dt><samp class="option">main_w, W</samp></dt> <dt><samp
class="option">main_h, H</samp></dt> <dd> The main input
width and height. </dd> <dt><samp class="option">overlay_w,
w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt>
<dd> The overlay input width and height. </dd> <dt><samp
class="option">x</samp></dt> <dt><samp class="option">y</samp></dt>
<dd> The computed values for <var class="var">x</var> and <var
class="var">y</var>. They are evaluated for each new frame.
</dd> <dt><samp class="option">hsub</samp></dt> <dt><samp
class="option">vsub</samp></dt> <dd> horizontal and vertical
chroma subsample values of the output format. For example for the
pixel format "yuv422p" <var class="var">hsub</var> is 2
and <var class="var">vsub</var> is 1. </dd> <dt><samp
class="option">n</samp></dt> <dd> the number of input frame, start-
ing from 0 </dd> <dt><samp class="option">pos</samp></dt>
<dd> the position in the file of the input frame, NAN if unknown; depre-
cated, do not use </dd> <dt><samp class="option">t</samp></dt>

<dd> The timestamp, expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

This filter also supports the framesync options. Note that the <var class="var">n</var>, <var class="var">t</var> variables are available only when evaluation is done per frame, and will evaluate to NAN when <samp class="option">eval</samp> is set to ‘<samp class="samp">init</samp>’. Be aware that frames are taken from each input video in timestamp order, hence, if their initial timestamps differ, it is a good idea to pass the two inputs through a <var class="var">setpts=PTS-STARTPTS</var> filter to have them begin in the same zero timestamp, as the example for the <var class="var">movie</var> filter does. You can chain together more overlays but you should test the efficiency of such approach. <h4 class="subsection">39.189.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Modify the x and y of the overlay input. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class="subsection">39.189.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the overlay at 10 pixels from the bottom right corner of the main video: <pre class="example-preformatted">overlay=main_w-overlay_w-10:main_h-overlay_h-10 </pre>

Using named options the example above becomes: <pre class="example-preformatted">overlay=x=main_w-overlay_w-10:y=main_h-overlay_h-10 </pre>

Insert a transparent PNG logo in the bottom left corner of the input, using the <code class="command">ffmpeg</code> tool with the <code class="code">-filter_complex</code> option: <pre class="example-preformatted">ffmpeg -i input -i logo -filter_complex 'overlay=10:main_h-overlay_h-10' output </pre>

Insert 2 different transparent PNG logos (second logo on bottom right corner) using the <code class="command">ffmpeg</code> tool: <pre class="example-preformatted">ffmpeg -i input -i

logo1 -i logo2 -filter_complex 'overlay=x=10:y=H-h-10,overlay=x=W-w-10:y=H-h-10' output </pre>

Add a transparent color layer on top of the main video; <code class="code">WxH</code> must specify the size of the main input to the overlay filter: <pre class="example-preformatted">color=color=red@.3:size=WxH [over]; [in][over] overlay [out] </pre>

Play an original video and a filtered version (here with the deshake filter) side by side using the <code class="command">ffplay</code> tool: <pre class="example-preformatted">ffplay input.avi -vf 'split[a][b]; [a]pad=iw*2:ih[src]; [b]deshake[filt]; [src][filt]overlay=w' </pre>

The above command is the same as: <pre class="example-preformatted">ffplay input.avi -vf 'split[b], pad=iw*2[src], [b]deshake, [src]overlay=w' </pre>

Make a sliding overlay appearing from the left to the right top part of the screen starting since time 2: <pre class="example-preformatted">overlay=x='if(gte(t,2), -w+(t-2)*20, NAN)':y=0 </pre>

Compose output by putting two input videos side to side: <pre class="example-preformatted">ffmpeg -i left.avi -i right.avi -filter_complex " nullsrc=size=200x100 [background]; [0:v] setpts=PTS-STARTPTS, scale=100x100 [left]; [1:v] setpts=PTS-STARTPTS, scale=100x100 [right]; [background][left] overlay=shortest=1 [background+left]; [background+left][right] overlay=shortest=1:x=100 [left+right] " </pre>

Mask 10-20 seconds of a video by applying the delogo filter to a section <pre class="example-preformatted">ffmpeg -i test.avi -codec:v:0 wmv2 -ar 11025 -b:v 9000k -vf '[in]split[split_main][split_delogo];[split_delogo]trim=start=360:end=
masked.avi </pre>

Chain several overlays in cascade: <pre class="example-preformatted">nullsrc=s=200x200 [bg]; testsrc=s=100x100, split=4 [in0][in1][in2][in3]; [in0] lutrgb=r=0, [bg] overlay=0:0 [mid0]; [in1] lutrgb=g=0, [mid0] overlay=100:0 [mid1]; [in2] lutrgb=b=0, [mid1] overlay=0:100 [mid2]; [in3] null, [mid2] overlay=100:100 [out0] </pre>

 <h3 class="section">39.190 owdenoise# TOC</h3>

Apply Overcomplete Wavelet denoiser. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>depth</samp></dt> <dd> Set depth. Larger depth values will denoise lower frequency components more, but slow down filtering. Must be an int in the range 8-16, default is <code class=”code”>8</code>. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma strength. Must be a double value in the range 0-1000, default is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>chroma_strength, cs</samp></dt> <dd> Set chroma strength. Must be a double value in the range 0-1000, default is <code class=”code”>1.0</code>. </dd> </dl>

 <h3 class=”section”>39.191 pad# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class=”var”>x</var>, <var class=”var”>y</var> coordinates. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. The default value of <var class=”var”>color</var> is "black". </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Specify when to evaluate <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var> and <var class=”var”>y</var> expression. It accepts the following values: <dl class=”table”>

<dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offsets as specified by the <var class="var">x</var> and <var class="var">y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">39.191.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add paddings with the color "violet" to the input video. The output video size is 640x480, and the top-left corner of the input video is placed at column 0, row 40 <pre class="example-preformatted">pad=640:480:0:40:violet
</pre>

The example above is equivalent to the following command: <pre class=”example-preformatted”>pad=width=640:height=480:x=0:y=40:color=violet </pre>

Pad the input to get an output with dimensions increased by 3/2, and put the input video at the center of the padded area: <pre class=”example-preformatted”>pad="3/2*iw:3/2*ih:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a squared output with size equal to the maximum value between the input width and height, and put the input video at the center of the padded area: <pre class=”example-preformatted”>pad="max(iw\,ih):ow:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a final w/h ratio of 16:9: <pre class=”example-preformatted”>pad="ih*16/9:ih:(ow-iw)/2:(oh-ih)/2" </pre>

In case of anamorphic video, in order to set the output display aspect correctly, it is necessary to use <var class=”var”>sar</var> in the expression, according to the relation: <pre class=”example-preformatted”>(ih * X / ih) * sar = output_dar X = output_dar / sar </pre>

Thus the previous example needs to be modified to: <pre class=”example-preformatted”>pad="ih*16/9/sar:ih:(ow-iw)/2:(oh-ih)/2" </pre>

Double the output size and put the input video in the bottom-right corner of the output padded area: <pre class=”example-preformatted”>pad="2*iw:2*ih:ow-iw:oh-ih" </pre>

 <h3 class=”section”>39.192 palettegen# TOC</h3>

Generate one palette for a whole video stream. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>max_colors</samp></dt> <dd> Set the maximum number of colors to quantize in the palette. Note: the palette will still contain 256 colors; the unused palette entries will be black. </dd> <dt><samp class=”option”>reserve_transparent</samp></dt> <dd> Create a palette of 255 colors maximum and reserve the last one for transparency. Reserving the transparency color is useful for GIF optimization. If not set, the maximum of colors in the palette will be 256. You probably want to disable this option for a standalone image. Set by default. </dd> <dt><samp class=”option”>transparency_color</samp></dt> <dd> Set the color

that will be used as background for transparency. </dd> <dt><samp class="option">stats_mode</samp></dt> <dd> Set statistics mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Compute full frame histograms. </dd> <dt>‘<samp class="samp">diff</samp>’</dt> <dd> Compute histograms only for the part that differs from previous frame. This might be relevant to give more importance to the moving part of your input if the background is static. </dd> <dt>‘<samp class="samp">single</samp>’</dt> <dd> Compute new histogram for each frame. </dd> </dl>

Default value is <var class="var">full</var>. </dd> </dl>

The filter also exports the frame metadata <code class="code">lavfi.color_quant_ratio</code> (<code class="code">nb_color_in / nb_color_out</code>) which you can use to evaluate the degree of color quantization of the palette. This information is also visible at <var class="var">info</var> logging level. <h4 class="subsection">39.192.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a representative palette of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -vf palettegen palette.png </pre>

 <h3 class="section">39.193 paletteuse# TOC</h3>

Use a palette to downsample an input video stream. The filter takes two inputs: one video stream and a palette. The palette must be a 256 pixels image. It accepts the following options: <dl class="table"> <dt><samp class="option">dither</samp></dt> <dd> Select dithering mode. Available algorithms are: <dl class="table"> <dt>‘<samp class="samp">bayer</samp>’</dt> <dd> Ordered 8x8 bayer dithering (deterministic) </dd> <dt>‘<samp class="samp">heckbert</samp>’</dt> <dd> Dithering as defined by Paul Heckbert in 1982 (simple error diffusion). Note: this dithering is sometimes considered "wrong" and is included as a reference. </dd> <dt>‘<samp class="samp">floyd_steinberg</samp>’</dt> <dd> Floyd and Steingberg dithering (error diffusion) </dd> <dt>‘<samp class="samp">sierra2</samp>’</dt> <dd> Frankie Sierra dithering v2 (error diffusion) </dd> <dt>‘<samp class="samp">sierra2_4a</samp>’</dt> <dd> Frankie Sierra dithering v2 "Lite" (error diffusion) </dd>

<dt>‘<samp class="samp">sierra3</samp>’</dt> <dd> Frankie Sierra dithering v3 (error diffusion) </dd> <dt>‘<samp class="samp">burkes</samp>’</dt> <dd> Burkes dithering (error diffusion) </dd> <dt>‘<samp class="samp">atkinson</samp>’</dt> <dd> Atkinson dithering by Bill Atkinson at Apple Computer (error diffusion) </dd> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable dithering. </dd> </dl>

Default is <var class="var">sierra2_4a</var>. </dd> <dt><samp class="option">bayer_scale</samp></dt> <dd> When <var class="var">bayer</var> dithering is selected, this option defines the scale of the pattern (how much the crosshatch pattern is visible). A low value means more visible pattern for less banding, and higher value means less visible pattern at the cost of more banding. The option must be an integer value in the range [0,5]. Default is <var class="var">2</var>. </dd> <dt><samp class="option">diff_mode</samp></dt> <dd> If set, define the zone to process <dl class="table"> <dt>‘<samp class="samp">rectangle</samp>’</dt> <dd> Only the changing rectangle will be reprocessed. This is similar to GIF cropping/offsetting compression mechanism. This option can be useful for speed if only a part of the image is changing, and has use cases such as limiting the scope of the error diffusal <samp class="option">dither</samp> to the rectangle that bounds the moving scene (it leads to more deterministic output if the scene doesn’t change much, and as a result less moving noise and better GIF compression). </dd> </dl>

Default is <var class="var">none</var>. </dd> <dt><samp class="option">new</samp></dt> <dd> Take new palette for each output frame. </dd> <dt><samp class="option">alpha_threshold</samp></dt> <dd> Sets the alpha threshold for transparency. Alpha values above this threshold will be treated as completely opaque, and values below this threshold will be treated as completely transparent. The option must be an integer value in the range [0,255]. Default is <var class="var">128</var>. </dd> </dl>

 <h4 class="subsection">39.193.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use a palette (generated for example with palettegen) to encode a GIF using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -i palette.png -lavfi paletteuse output.gif </pre>

 <h3 class="section">39.194 perspective<a class="anchor hidden-xs" href="#perspective" aria-

hidden=”true”># TOC</h3>

Correct perspective of video not recorded perpendicular to the screen. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dt><samp class=”option”>x1</samp></dt> <dt><samp class=”option”>y1</samp></dt> <dt><samp class=”option”>x2</samp></dt> <dt><samp class=”option”>y2</samp></dt> <dt><samp class=”option”>x3</samp></dt> <dt><samp class=”option”>y3</samp></dt> <dd> Set coordinates expression for top left, top right, bottom left and bottom right corners. Default values are <code class=”code”>0:0:W:0:0:H:W:H</code> with which perspective will remain unchanged. If the <code class=”code”>sense</code> option is set to <code class=”code”>source</code>, then the specified points will be sent to the corners of the destination. If the <code class=”code”>sense</code> option is set to <code class=”code”>destination</code>, then the corners of the source will be sent to the specified coordinates. The expressions can use the following variables: <dl class=”table”> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> the width and height of video frame. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> </dl>

</dd> <dt><samp class=”option”>interpolation</samp></dt> <dd> Set interpolation for perspective correction. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>cubic</samp>’</dt> </dl>

Default value is ‘<samp class=”samp”>linear</samp>’. </dd> <dt><samp class=”option”>sense</samp></dt> <dd> Set interpretation of coordinate options. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>0, source</samp>’</dt> <dd> Send point in the source specified by the given coordinates to the corners of the destination. </dd> <dt>‘<samp class=”samp”>1, destination</samp>’</dt> <dd> Send the corners of the source to the point in the destination specified by the given coordinates. Default value is ‘<samp class=”samp”>source</samp>’. </dd> </dl>

</dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for coordinates <samp class=”option”>x0,y0,...x3,y3</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.
</dd> </dl>

 <h3 class="section">39.195 phase<span class="pull-
right">#
TOC</h3>

Delay interlaced video by one field time so that the field order changes.
 The intended use is to fix PAL movies that have been captured
with the opposite field order to the film-to-video transfer. A
description of the accepted parameters follows. <dl class="table">
<dt><samp class="option">mode</samp></dt> <dd> Set phase
mode. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">t</samp>’</dt> <dd> Capture
field order top-first, transfer bottom-first. Filter will delay the bottom field.
 </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> Capture field order bottom-first, transfer top-first. Filter will delay
the top field. </dd> <dt>‘<samp class="samp">p</samp>’</dt>
<dd> Capture and transfer with the same field order. This mode only
exists for the documentation of the other options to refer to, but if you actually
select it, the filter will faithfully do nothing. </dd> <dt>‘<samp
class="samp">a</samp>’</dt> <dd> Capture field or-
der determined automatically by field flags, transfer opposite. Filter
selects among ‘<samp class="samp">t</samp>’ and
‘<samp class="samp">b</samp>’ modes on a frame by
frame basis using field flags. If no field information is available, then
this works just like ‘<samp class="samp">u</samp>’.
 </dd> <dt>‘<samp class="samp">u</samp>’</dt>
<dd> Capture unknown or varying, transfer opposite. Filter selects
among ‘<samp class="samp">t</samp>’ and ‘<samp
class="samp">b</samp>’ on a frame by frame basis by analyzing the
images and selecting the alternative that produces best match between the fields.
 </dd> <dt>‘<samp class="samp">T</samp>’</dt>
<dd> Capture top-first, transfer unknown or varying. Filter selects
among ‘<samp class="samp">t</samp>’ and ‘<samp
class="samp">p</samp>’ using image analysis. </dd>
<dt>‘<samp class="samp">B</samp>’</dt> <dd> Capture
bottom-first, transfer unknown or varying. Filter selects among ‘<samp
class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’
using image analysis. </dd> <dt>‘<samp class="samp">A</samp>’</dt>
<dd> Capture determined by field flags, transfer unknown or varying.
Filter selects among ‘<samp class="samp">t</samp>’,
‘<samp class="samp">b</samp>’ and ‘<samp
class="samp">p</samp>’ using field flags and image analysis. If
no field information is available, then this works just like ‘<samp
class="samp">U</samp>’. This is the default mode.
</dd> <dt>‘<samp class="samp">U</samp>’</dt>

<dd> Both capture and transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’, ‘<samp class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’ using image analysis only. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.195.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.196 photosensitivity# TOC</h3> Reduce various flashes in video, so to help users with epilepsy. It accepts the following options: <dl class="table"> <dt><samp class="option">frames, f</samp></dt> <dd> Set how many frames to use when filtering. Default is 30. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set detection threshold factor. Default is 1. Lower is stricter. </dd> <dt><samp class="option">skip</samp></dt> <dd> Set how many pixels to skip when sampling frames. Default is 1. Allowed range is from 1 to 1024. </dd> <dt><samp class="option">bypass</samp></dt> <dd> Leave frames unchanged. Default is disabled. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the blending factor for frames that exceed the detection threshold. A value of 0 duplicates the last accepted frame, and a value of 1 blends up to the detection threshold. Default is 1. Allowed range is from 0 to 1. </dd> </dl>

 <h3 class="section">39.197 pixdesctest# TOC</h3>

Pixel format descriptor test filter, mainly useful for internal testing. The output video should be equal to the input video. For example: <pre class="example-preformatted">format=monow, pixdesctest </pre>

can be used to test the monowhite pixel format descriptor definition. <h3 class="section">39.198 pixelize# TOC</h3> Apply pixelization to video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set block

dimensions that will be used for pixelization. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set the mode of pixelization used. Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dt>‘<samp class=”samp”>min</samp>’</dt> <dt>‘<samp class=”samp”>max</samp>’</dt> </dl> Default value is <code class=”code”>avg</code>. </dd> <dt><samp class=”option”>planes, p</samp></dt> <dd> Set what planes to filter. Default is to filter all planes. </dd> </dl>

 <h4 class=”subsection”>39.198.1 Commands# TOC</h4>

This filter supports all options as commands. <h3 class=”section”>39.199 pixscope# TOC</h3>

Display sample values of color channels. Mainly useful for checking color and levels. Minimum supported resolution is 640x480. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set scope X position, relative offset on X axis. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set scope Y position, relative offset on Y axis. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set scope width. </dd> <dt><samp class=”option”>h</samp></dt> <dd> Set scope height. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Set window opacity. This window also holds statistics about pixel area. </dd> <dt><samp class=”option”>wx</samp></dt> <dd> Set window X position, relative offset on X axis. </dd> <dt><samp class=”option”>wy</samp></dt> <dd> Set window Y position, relative offset on Y axis. </dd> </dl>

 <h4 class=”subsection”>39.199.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>39.200 pp7# TOC</h3> Apply Postprocessing filter 7. It is variant of the spp filter, similar to spp = 6 with 7 point DCT, where only the center sample is used after IDCT.

 The filter accepts the following options: <dl class="table"> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0 to 63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Set medium thresholding (good results, default). </dd> </dl> </dd> </dl>

 <h3 class="section">39.201 premultiply# TOC</h3> Apply alpha premultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">39.202 premultiply_dynamic# TOC</h3> Dynamically premultiply or unpremultiply the input video stream as needed, to match the requirements of the downstream filter graph. This is roughly equivalent to either <code class="code">premultiply:inplace=yes</code> or <code class="code">unpremultiply:inplace=yes</code>, or otherwise a noop. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> </dl>

 <h3 class="section">39.203 prewitt# TOC</h3> Apply prewitt operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be

copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">39.203.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.204 pseudocolor# TOC</h3>

Alter frame colors in video with pseudocolors. This filter accepts the following options: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set pixel first component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set pixel second component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set pixel third component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set pixel fourth component expression, corresponds to the alpha component </dd> <dt><samp class="option">index, i</samp></dt> <dd> set component to use as base for altering colors </dd> <dt><samp class="option">preset, p</samp></dt> <dd> Pick one of built-in LUTs. By default is set to none. Available LUTs: <dl class="table"> <dt>‘<samp class="samp">magma</samp>’</dt> <dt>‘<samp class="samp">inferno</samp>’<
<dt>‘<samp class="samp">plasma</samp>’</dt> <dt>‘<samp class="samp">viridis</samp>’</dt> <dt>‘<samp class="samp">turbo</samp>’</
<dt>‘<samp class="samp">cividis</samp>’</dt> <dt>‘<samp class="samp">range1</samp>’</dt> <dt>‘<samp class="samp">range2</samp>’<
<dt>‘<samp class="samp">shadows</samp>’</dt> <dt>‘<samp class="samp">highlights</samp>’</dt> <dt>‘<samp class="samp">solar</samp>’</dt> <dt>‘<samp class="samp">nominal</samp>’</
<dt>‘<samp class="samp">preferred</samp>’</dt> <dt>‘<samp class="samp">total</samp>’</dt> <dt>‘<samp class="samp">spectral</samp>’</
<dt>‘<samp class="samp">cool</samp>’</dt> <dt>‘<samp class="samp">heat</samp>’</dt> <dt>‘<samp class="samp">fiery</samp>’</dt>
<dt>‘<samp class="samp">blues</samp>’</dt> <dt>‘<samp class="samp">green</samp>’</dt> <dt>‘<samp class="samp">helix</samp>’</dt
</dl>

</dd> <dt><samp class="option">opacity</samp></dt> <dd> Set opacity of output colors. Allowed range is from 0 to 1. Default value is set to

1. </dd> </dl>

Each of the expression options specifies the expression to use for computing the lookup table for the corresponding pixel component values. The expressions can contain the following constants and functions: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class="option">ymin, umin, vmin, amin</samp></dt> <dd> The minimum allowed component value. </dd> <dt><samp class="option">ymax, umax, vmax, amax</samp></dt> <dd> The maximum allowed component value. </dd> </dl>

All expressions default to "val". <h4 class="subsection">39.204.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">39.204.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Change too high luma values to gradient: <pre class="example-preformatted">pseudocolor="'if(between(val,ymax,amax) ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(umax,umin,(val-ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(vmin,vmax,(val-ymax)/(amax-ymax)),-1):-1'" </pre>

 <h3 class="section">39.205 psnr# TOC</h3>

Obtain the average, maximum and minimum PSNR (Peak Signal to Noise Ratio) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the PSNR. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average PSNR is printed through the logging system. The filter stores the accumulated MSE (mean squared error) of each frame, and at the end of the processing it is averaged across all frames equally, and the following formula is applied to obtain the PSNR: <pre class="example-preformatted">PSNR = 10*log10(MAX^2/MSE) </pre>

Where MAX is the average of the maximum values of each component of the image. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the PSNR of each individual frame. When filename equals "-" the data is sent to standard output. </dd> <dt><samp class="option">stats_version</samp></dt> <dd> Specifies which version of the stats file format to use. Details of each format are written below. Default value is 1. </dd> <dt><samp class="option">output_max</samp></dt> <dd> Determines whether the max value is output to the stats log. Default value is 0. Requires stats_version >= 2. If this is set and stats_version < 2, the filter will return an error. </dd> </dl>

This filter also supports the framesync options. The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. If a <var class="var">stats_version</var> greater than 1 is specified, a header line precedes the list of per-frame-pair stats, with key value pairs following the frame format with the following parameters: <dl class="table"> <dt><samp class="option">psnr_log_version</samp></dt> <dd> The version of the log file format. Will match <var class="var">stats_version</var>. </dd> <dt><samp class="option">fields</samp></dt> <dd> A comma separated list of the per-frame-pair parameters included in the log. </dd> </dl>

A description of each shown per-frame-pair parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">mse_avg</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames, averaged over all the image components. </dd> <dt><samp class="option">mse_y, mse_u, mse_v, mse_r, mse_g, mse_b, mse_a</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">psnr_y, psnr_u, psnr_v, psnr_r, psnr_g, psnr_b, psnr_a</samp></dt> <dd> Peak Signal to Noise ratio of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">max_avg, max_y, max_u, max_v</samp></dt> <dd> Maximum allowed value for each channel, and average over all channels. </dd> </dl>

 <h4 class="subsection">39.205.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main];

[main][ref] psnr="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The PSNR of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]psnr" -f null - </pre>

 <h3 class="section">39.206 pullup# TOC</h3>

Pulldown reversal (inverse telecine) filter, capable of handling mixed hard-telecine, 24000/1001 fps progressive, and 30000/1001 fps progressive content. The pullup filter is designed to take advantage of future context in making its decisions. This filter is stateless in the sense that it does not lock onto a pattern to follow, but it instead looks forward to the following fields in order to identify matches and rebuild progressive frames. To produce content with an even framerate, insert the fps filter after pullup, use <code class="code">fps=24000/1001</code> if the input frame rate is 29.97fps, <code class="code">fps=24</code> for 30fps and the (rare) telecined 25fps input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">jl</samp></dt> <dt><samp class="option">jr</samp></dt> <dt><samp class="option">jt</samp></dt> <dt><samp class="option">jb</samp></dt> <dd> These options set the amount of "junk" to ignore at the left, right, top, and bottom of the image, respectively. Left and right are in units of 8 pixels, while top and bottom are in units of 2 lines. The default is 8 pixels on each side. </dd> <dt><samp class="option">sb</samp></dt> <dd> Set the strict breaks. Setting this option to 1 will reduce the chances of filter generating an occasional mismatched frame, but it may also cause an excessive number of frames to be dropped during high motion sequences. Conversely, setting it to -1 will make filter match fields more easily. This may help processing of video where there is slight blurring between the fields, but may also cause there to be interlaced frames in the output. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">mp</samp></dt> <dd> Set the metric plane to use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">l</samp>’</dt> <dd> Use luma plane. </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> Use chroma blue plane. </dd> <dt>‘<samp class="samp">v</samp>’</dt> <dd> Use chroma red plane. </dd> </dl>

This option may be set to use chroma plane instead of the default luma

plane for doing filter’s computations. This may improve accuracy on very clean source material, but more likely will decrease accuracy, especially if there is chroma noise (rainbow effect) or any grayscale video. The main purpose of setting <samp class="option">mp</samp> to a chroma plane is to reduce CPU load and make pullup usable in realtime on slow machines. </dd> </dl>

For best results (without duplicated frames in the output file) it is necessary to change the output frame rate. For example, to inverse telecine NTSC input: <pre class="example-preformatted">ffmpeg -i input -vf pullup -r 24000/1001 ... </pre>

 <h3 class="section">39.207 qp# TOC</h3>

Change video quantization parameters (QP). The filter accepts the following option: <dl class="table"> <dt><samp class="option">qp</samp></dt> <dd> Set expression for quantization parameter. </dd> </dl>

The expression is evaluated through the eval API and can contain, among others, the following constants: <dl class="table"> <dt><var class="var">known</var></dt> <dd> 1 if index is not 129, 0 otherwise. </dd> <dt><var class="var">qp</var></dt> <dd> Sequential index starting from -129 to 128. </dd> </dl>

 <h4 class="subsection">39.207.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Some equation like: <pre class="example-preformatted">qp=2+2*sin(PI*qp) </pre>

 <h3 class="section">39.208 qrencode# TOC</h3> Generate a QR code using the libqrencode library (see <a class="url" href="https://fukuchi.org/works/qrencode/">https://fukuchi.o and overlay it on top of the current frame. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and overlaid into the video output according to the specified options. In case no text is specified, no QR code is overlaied. This filter accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp

class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify an expression for positioning the padded QR code top-left corner. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice. By default <var class=”var”>x</var> and <var class=”var”>y</var> are set set to <var class=”var”>0</var>, meaning that the QR code is placed in the top left corner of the input. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is

"white". For the syntax of the color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">39.208.1 qrencode Expressions# TOC</h4>

The expressions set by the options contain the following constants and functions. <dl class="table"> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">duration</samp></dt> <dd> the current frame’s duration, in seconds </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">main_h, H</samp></dt> <dd> the input height </dd> <dt><samp class="option">main_w, W</samp></dt> <dd> the input width </dd> <dt><samp class="option">n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class="option">pict_type</samp></dt> <dd> a number representing the picture type </dd> <dt><samp class="option">qr_w, w</samp></dt> <dd> the width of the encoded QR code </dd> <dt><samp class="option">rendered_qr_w, q</samp></dt> <dt><samp class="option">rendered_padded_qr_w, Q</samp></dt> <dd> the width of the rendered QR code, without and without padding. These parameters allow the <var class="var">q</var> and <var class="var">Q</var> expressions to refer to each other, so you can for example specify <code class="code">q=3/4*Q</code>. </dd> <dt><samp class="option">rand(min, max)</samp></dt> <dd> return a random number included between <var class="var">min</var> and <var class="var">max</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> the input sample aspect ratio </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> the x and y offset coordinates where the text is drawn. These parameters allow the <var class="var">x</var> and <var class="var">y</var> expressions to refer to each other, so you can for example specify <code class="code">y=x/dar</code>. </dd> </dl>

<a

name=”qrencode-Text-expansion”> <h4 class=”subsection”>39.208.2 qrencode Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim. If <samp class=”option”>expansion</samp> is set to <code class=”code”>normal</code> (which is the default), the following expansion mechanism is used. The backslash character ‘<samp class=”samp”>\</samp>’, followed by any character, always expands to the second character. Sequences of the form <code class=”code”>%{...}</code> are expanded. The text between the braces is a function name, possibly followed by arguments separated by ’:’. If the arguments contain special characters or delimiters (’:’ or ’}’), they should be escaped. Note that they probably must also be escaped as the value for the <samp class=”option”>text</samp> option in the filter argument string and as the filter argument in the filtergraph description, and possibly also for the shell, that makes up to four levels of escaping; using a text file with the <samp class=”option”>textfile</samp> option avoids these problems. The following functions are available: <dl class=”table”> <dt><code class=”command”>n, frame_num</code></dt> <dd> return the frame number </dd> <dt><code class=”command”>pts</code></dt> <dd> Return the presentation timestamp of the current frame. It can take up to two arguments. The first argument is the format of the timestamp; it defaults to <code class=”code”>flt</code> for seconds as a decimal number with microsecond accuracy; <code class=”code”>hms</code> stands for a formatted <var class=”var”>[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy. <code class=”code”>gmtime</code> stands for the timestamp of the frame formatted as UTC time; <code class=”code”>localtime</code> stands for the timestamp of the frame formatted as local time zone time. If the format is set to <code class=”code”>hms24hh</code>, the time is formatted in 24h format (00-23). The second argument is an offset added to the timestamp. If the format is set to <code class=”code”>localtime</code> or <code class=”code”>gmtime</code>, a third argument may be supplied: a <code class=”code”>strftime</code> C function format string. By default, <var class=”var”>YYYY-MM-DD HH:MM:SS</var> format will be used. </dd> <dt><code class=”command”>expr, e</code></dt> <dd> Evaluate the expression’s value and output as a double. It must take one argument specifying the expression to be evaluated, accepting the constants and functions defined in qrencode_expressions. </dd> <dt><code class=”command”>expr_formatted, ef</code></dt> <dd> Evaluate the expression’s value and output as a formatted string. The first argument is the expression to

be evaluated, just as for the <var class=”var”>expr</var> function. The second argument specifies the output format. Allowed values are ‘<samp class=”samp”>x</samp>’, ‘<samp class=”samp”>X</samp>’, ‘<samp class=”samp”>d</samp>’ and ‘<samp class=”samp”>u</samp>’. They are treated exactly as in the <code class=”code”>printf</code> function. The third parameter is optional and sets the number of positions taken by the output. It can be used to add padding with zeros from the left. </dd> <dt><code class=”command”>gmtime</code></dt> <dd> The time at which the filter is running, expressed in UTC. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>localtime</code></dt> <dd> The time at which the filter is running, expressed in the local time zone. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>metadata</code></dt> <dd> Frame metadata. Takes one or two arguments. The first argument is mandatory and specifies the metadata key. The second argument is optional and specifies a default value, used when the metadata key is not found or empty. Available metadata can be identified by inspecting entries starting with TAG included within each frame section printed by running <code class=”code”>ffprobe -show_frames</code>. String metadata generated in filters leading to the qrencode filter are also available. </dd> <dt><code class=”command”>rand(min, max)</code></dt> <dd> return a random number included between <var class=”var”>min</var> and <var class=”var”>max</var> </dd> </dl>

 <h4 class=”subsection”>39.208.3 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Generate a QR code encoding the specified text with the default size, overalaid in the top left corner of the input video, with the default size: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:bc=pink@0.5:fc=blue </pre>

Place the QR code in the bottom right corner of the input video: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:x=W-

Q:y=H-Q </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Make the QR code a fraction of the input video width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=W/5 </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencode=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencode=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencode=text=%{pts} </pre>

 <h3 class="section">39.209 quirc# TOC</h3>

Identify and decode a QR code using the libquirc library (see https://github.com/dlbeer/quirc/), and print the identified QR codes positions and payload as metadata. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libquirc</code>. For each found QR code in the input video, some metadata entries are added with the prefix <var class="var">lavfi.quirc.N</var>, where <var class="var">N</var> is the index, starting from 0, associated to the QR code. A description of each metadata value follows: <dl class="table"> <dt><samp class="option">lavfi.quirc.count</samp></dt> <dd> the number of found QR codes, it is not set in case none was found </dd> <dt><samp class="option">lavfi.quirc.N.corner.M.x</samp></dt> <dt><samp class="option">lavfi.quirc.N.coreer.M.y</samp></dt> <dd> the x/y positions of the four corners of the square containing the QR code, where <var class="var">M</var> is the index of the corner starting from 0 </dd> <dt><samp class="option">lavfi.quirc.N.payload</samp></dt> <dd> the payload of the QR code </dd> </dl>

 <h3 class="section">39.210 random# TOC</h3>

Flush video frames from internal cache of frames into a random order. No frame is discarded. Inspired by frei0r nervous filter. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> Set size in number of frames of internal cache, in range from <code class="code">2</code> to <code class="code">512</code>. Default is <code class="code">30</code>. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed for random number generator, must be an integer included between <code class="code">0</code> and <code class="code">UINT32_MAX</code>. If not specified, or if explicitly set to less than <code class="code">0</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

 <h3 class="section">39.211 readeia608# TOC</h3>

Read closed captioning (EIA-608) information from the top lines of a video frame. This filter adds frame metadata for <code class="code">lavfi.readeia608.X.cc</code> and <code class="code">lavfi.readeia608.X.line</code>, where <code class="code">X</code> is the number of the identified line with EIA-608 data (starting from 0). A description of each metadata value follows: <dl class="table"> <dt><samp class="option">lavfi.readeia608.X.cc</samp></dt> <dd> The two bytes stored as EIA-608 data (printed in hexadecimal). </dd> <dt><samp class="option">lavfi.readeia608.X.line</samp></dt> <dd> The number of the line on which the EIA-608 data was identified and read. </dd> </dl>

This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_min</samp></dt> <dd> Set the line to start scanning for EIA-608 data. Default is <code class="code">0</code>. </dd> <dt><samp class="option">scan_max</samp></dt> <dd> Set the line to end scanning for EIA-608 data. Default is <code class="code">29</code>. </dd> <dt><samp class="option">spw</samp></dt> <dd> Set the ratio of width reserved for sync code detection. Default is <code class="code">0.27</code>. Allowed range is <code class="code">[0.1 - 0.7]</code>. </dd> <dt><samp class="option">chp</samp></dt> <dd> Enable checking the parity bit. In the event of a parity error, the filter will output <code class="code">0x00</code> for that character. Default is false. </dd> <dt><samp class="option">lp</samp></dt> <dd> Lowpass lines prior to further processing. Default is enabled. </dd> </dl>

 <h4 class="subsection">39.211.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">39.211.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output a csv with presentation time and the first two lines of identified EIA-608 captioning data. <pre class="example-preformatted">ffprobe -f lavfi -i movie=captioned_video.mov,readeia608 -show_entries frame=pts_time:frame_tags=lavfi.readeia608.0.cc,lavf -of csv </pre>

 <h3 class="section">39.212 readvitc# TOC</h3>

Read vertical interval timecode (VITC) information from the top lines of a video frame. The filter adds frame metadata key <code class="code">lavfi.readvitc.tc_str</code> with the timecode value, if a valid timecode has been detected. Further metadata key <code class="code">lavfi.readvitc.found</code> is set to 0/1 depending on whether timecode data has been found or not. This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_max</samp></dt> <dd> Set the maximum number of lines to scan for VITC data. If the value is set to <code class="code">-1</code> the full video frame is scanned. Default is <code class="code">45</code>. </dd> <dt><samp class="option">thr_b</samp></dt> <dd> Set the luma threshold for black. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.2</code>. The value must be equal or less than <code class="code">thr_w</code>. </dd> <dt><samp class="option">thr_w</samp></dt> <dd> Set the luma threshold for white. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.6</code>. The value must be equal or greater than <code class="code">thr_b</code>. </dd> </dl>

 <h4 class="subsection">39.212.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect and draw VITC data onto the video frame; if no valid VITC is detected, draw <code class="code">--:--:--

:--</code> as a placeholder: <pre class="example-preformatted">ffmpeg -i input.avi -filter:v 'readvitc,drawtext=fontfile=FreeMono.ttf:text=%{metadata\\:lavfi
-\\\\\\:--\\\\\\:--\\\\\\:--}:x=(w-tw)/2:y=400-ascent' </pre>

 <h3 class="section">39.213 remap# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 16bit depth, single channel.
<dl class="table"> <dt><samp class="option">format</samp></dt>
<dd> Specify pixel format of output from this filter. Can be <code class="code">color</code> or <code class="code">gray</code>. Default is <code class="code">color</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">39.214 removegrain# TOC</h3>

The removegrain filter is a spatial denoiser for progressive video. <dl class="table"> <dt><samp class="option">m0</samp></dt>
<dd> Set mode for the first plane. </dd> <dt><samp class="option">m1</samp></dt> <dd> Set mode for the second plane. </dd> <dt><samp class="option">m2</samp></dt>
<dd> Set mode for the third plane. </dd> <dt><samp class="option">m3</samp></dt> <dd> Set mode for the fourth plane. </dd> </dl>

Range of mode is from 0 to 24. Description of each mode follows:

<dl class="table">

<dt><var class="var">0</var></dt>

<dd> Leave input plane unchanged. Default.

</dd>

<dt><var class=”var”>1</var></dt>

<dd> Clips the pixel with the minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>2</var></dt>

<dd> Clips the pixel with the second minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>3</var></dt>

<dd> Clips the pixel with the third minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>4</var></dt>

<dd> Clips the pixel with the fourth minimum and maximum of the 8 neighbour pixels.

This is equivalent to a median filter.

</dd>

<dt><var class=”var”>5</var></dt>

<dd> Line-sensitive clipping giving the minimal change.

</dd>

<dt><var class=”var”>6</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class=”var”>7</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">8</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">9</var></dt>

<dd> Line-sensitive clipping on a line where the neighbours pixels are the closest.

</dd>

<dt><var class="var">10</var></dt>

<dd> Replaces the target pixel with the closest neighbour.

</dd>

<dt><var class="var">11</var></dt>

<dd> [1 2 1] horizontal and vertical kernel blur.

</dd>

<dt><var class="var">12</var></dt>

<dd> Same as mode 11.

</dd>

<dt><var class="var">13</var></dt>

<dd> Bob mode, interpolates top field from the line where the neighbours pixels are the closest.

</dd>

<dt><var class="var">14</var></dt>

<dd> Bob mode, interpolates bottom field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class="var">15</var></dt>

<dd> Bob mode, interpolates top field. Same as 13 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">16</var></dt>

<dd> Bob mode, interpolates bottom field. Same as 14 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">17</var></dt>

<dd> Clips the pixel with the minimum and maximum of respectively the maximum and

minimum of each pair of opposite neighbour pixels.

</dd>

<dt><var class="var">18</var></dt>

<dd> Line-sensitive clipping using opposite neighbours whose greatest distance from

the current pixel is minimal.

</dd>

<dt><var class="var">19</var></dt>

<dd> Replaces the pixel with the average of its 8 neighbours.

</dd>

<dt><var class=”var”>20</var></dt>

<dd> Averages the 9 pixels ([1 1 1] horizontal and vertical blur).

</dd>

<dt><var class=”var”>21</var></dt>

<dd> Clips pixels using the averages of opposite neighbour.

</dd>

<dt><var class=”var”>22</var></dt>

<dd> Same as mode 21 but simpler and faster.

</dd>

<dt><var class=”var”>23</var></dt>

<dd> Small edge and halo removal, but reputed useless.

</dd>

<dt><var class=”var”>24</var></dt>

<dd> Similar as 23.

</dd>

</dl>

 <h3 class=”section”>39.215 removelogo# TOC</h3>

Suppress a TV station logo, using an image file to determine which pixels comprise the logo. It works by filling in the pixels that comprise the logo with neighboring pixels. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filename, f</samp></dt> <dd> Set the filter bitmap file, which can be any image format supported by libavformat. The width and height of the image file must match those of the video stream being processed. </dd> </dl>

Pixels in the provided bitmap image with a value of zero are not considered part of the logo, non-zero pixels are considered part of the logo. If you use

white (255) for the logo and black (0) for the rest, you will be safe. For making the filter bitmap, it is recommended to take a screen capture of a black frame with the logo visible, and then using a threshold filter followed by the erode filter once or twice. If needed, little splotches can be fixed manually. Remember that if logo pixels are not covered, the filter quality will be much reduced. Marking too many pixels as part of the logo does not hurt as much, but it will increase the amount of blurring needed to cover over the image and will destroy more information than necessary, and extra pixels will slow things down on a large logo. <h3 class="section">39.216 repeatfields# TOC</h3>

This filter uses the repeat_field flag from the Video ES headers and hard repeats fields based on its value. <h3 class="section">39.217 reverse# TOC</h3>

Reverse a video clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">39.217.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">trim=end=5,reverse </pre>

 <h3 class="section">39.218 rgbashift# TOC</h3> Shift R/G/B/A pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rh</samp></dt> <dd> Set amount to shift red horizontally. </dd> <dt><samp class="option">rv</samp></dt> <dd> Set amount to shift red vertically. </dd> <dt><samp class="option">gh</samp></dt> <dd> Set amount to shift green horizontally. </dd> <dt><samp class="option">gv</samp></dt> <dd> Set amount to shift green vertically. </dd> <dt><samp class="option">bh</samp></dt> <dd> Set amount to shift blue horizontally. </dd> <dt><samp class="option">bv</samp></dt> <dd> Set amount to shift blue vertically. </dd> <dt><samp class="option">ah</samp></dt> <dd> Set amount to shift alpha horizontally. </dd> <dt><samp class="option">av</samp></dt> <dd> Set amount to shift alpha

vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">39.218.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.219 roberts# TOC</h3> Apply roberts cross operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">39.219.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.220 rotate# TOC</h3>

Rotate video by an arbitrary angle expressed in radians. The filter accepts the following options: A description of the optional parameters follows. <dl class="table"> <dt><samp class="option">angle, a</samp></dt> <dd> Set an expression for the angle by which to rotate the input video clockwise, expressed as a number of radians. A negative value will result in a counter-clockwise rotation. By default it is set to "0". This expression is evaluated for each frame. </dd> <dt><samp class="option">out_w, ow</samp></dt> <dd> Set the output width expression, default value is "iw". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">out_h, oh</samp></dt> <dd> Set the output height expression, default value is "ih". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">bilinear</samp></dt> <dd> Enable bilinear interpo-

lation if set to 1, a value of 0 disables it. Default value is 1. </dd> <dt><samp class=”option”>fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the rotated image. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> </dl>

The expressions for the angle and the output size can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> sequential number of the input frame, starting from 0. It is always NAN before the first frame is filtered. </dd> <dt><samp class=”option”>t</samp></dt> <dd> time in seconds of the input frame, it is set to 0 when the filter is configured. It is always NAN before the first frame is filtered. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_w, iw</samp></dt> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> the input video width and height </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> the output width and height, that is the size of the padded area as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions </dd> <dt><samp class=”option”>rotw(a)</samp></dt> <dt><samp class=”option”>roth(a)</samp></dt> <dd> the minimal width/height required for completely containing the input video rotated by <var class=”var”>a</var> radians. These are only available when computing the <samp class=”option”>out_w</samp> and <samp class=”option”>out_h</samp> expressions. </dd> </dl>

 <h4 class=”subsection”>39.220.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Rotate the input by PI/6 radians clockwise: <pre class=”example-preformatted”>rotate=PI/6 </pre>

Rotate the input by PI/6 radians counter-clockwise: <pre class=”example-preformatted”>rotate=-PI/6 </pre>

Rotate the input by 45 degrees clockwise: <pre class=”example-preformatted”>rotate=45*PI/180 </pre>

Apply a constant rotation with period T, starting from an angle of PI/3: <pre class=”example-preformatted”>rotate=PI/3+2*PI*t/T </pre>

Make the input video rotation oscillating with a period of T seconds and an amplitude of A radians: <pre class=”example-preformatted”>rotate=A*sin(2*PI/T*t) </pre>

Rotate the video, output size is chosen so that the whole rotating input video is always completely contained in the output: <pre class=”example-preformatted”>rotate='2*PI*t:ow=hypot(iw,ih):oh=ow' </pre>

Rotate the video, reduce the output size so that no background is ever shown: <pre class=”example-preformatted”>rotate=2*PI*t:ow='min(iw,ih)/sqrt(2)':oh=ow:c=none </pre>

 <h4 class=”subsection”>39.220.2 Commands# TOC</h4>

The filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>a, angle</samp></dt> <dd> Set the angle expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>39.221 sab# TOC</h3>

Apply Shape Adaptive Blur. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dd> Set luma blur filter strength, must be a value in range 0.1-4.0, default value is 1.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>luma_pre_filter_radius, lpfr</samp></dt> <dd> Set luma pre-filter radius, must be a value in the 0.1-2.0 range, default value is 1.0. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma maximum difference between pixels to still be considered, must be a value in the 0.1-100.0 range, default value is 1.0. </dd> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dd> Set chroma blur filter strength, must be a value in range -0.9-4.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>chroma_pre_filter_radius, cpfr</samp></dt> <dd> Set chroma pre-filter radius, must be a value in the -0.9-2.0 range. </dd> <dt><samp class=”option”>chroma_strength,

cs</samp></dt> <dd> Set chroma maximum difference between pixels to still be considered, must be a value in the -0.9-100.0 range. </dd> </dl>

Each chroma option value, if not explicitly specified, is set to the corresponding luma option value. <h3 class="section">39.222 scale# TOC</h3>

Scale (resize) the input video, using the libswscale library. The scale filter forces the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the scale filter will convert the input to the requested format. <h4 class="subsection">39.222.1 Options# TOC</h4> The filter accepts the following options, any of the options supported by the libswscale scaler, as well as any of the framesync options. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#scaler_005foptions">the ffmpeg-scaler manual for the complete list of scaler options. <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class="var">width</var> or <var class="var">w</var> value is 0, the input width is used for the output. If the <var class="var">height</var> or <var class="var">h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the scale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class="option">eval</samp></dt> <dd> Specify when to evaluate <var class="var">width</var> and <var class="var">height</var> expression. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.

</dd> <dt><samp class=”option”>interl</samp></dt> <dd> Set the interlacing mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Force interlaced aware scaling. </dd> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Do not apply interlaced scaling. </dd> <dt>‘<samp class=”samp”>-1</samp>’</dt> <dd> Select interlaced aware scaling depending on whether the source frames are flagged as interlaced or not. </dd> </dl>

Default value is ‘<samp class=”samp”>0</samp>’. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Set libswscale scaling flags. See <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#sws_005fflags”>the ffmpeg-scaler manual for the complete list of values. If not explicitly specified the filter applies the default flags.

</dd> <dt><samp class=”option”>param0, param1</samp></dt> <dd> Set libswscale input parameters for scaling algorithms that need them. See <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#sws_005fparams”>the ffmpeg-scaler manual for the complete documentation. If not explicitly specified the filter applies empty parameters. </dd> <dt><samp class=”option”>intent</samp></dt> <dd> Set the ICC rendering intent to use when transforming between different color spaces. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>perceptual</samp>’</dt> <dd> Use a perceptually guided tone and gamut mapping curve. The exact details of the mapping used may change at any time and should not be relied on as stable. This intent is recommended for final viewing of image/video content in typical viewing settings. </dd> <dt>‘<samp class=”samp”>relative_colorimetric</samp>’</dt> <dd> Statically clip out-of-gamut colors using a colorimetric clipping curve which attempts to find the colorimetrically least dissimilar in-gamut color. This intent performs white point adaptation and black point adaptation. This is the default. This intent is recommended wherever faithful color reproduction is of the utmost importance, even at the cost of clipping. </dd> <dt>‘<samp class=”samp”>absolute_colorimetric</samp>’</dt> <dd> Hard clip out-of-gamut colors with no attempt at white or black point reproduction. This intent will reproduce in-gamut colors 1:1 on the output display as they would appear on the reference display, assuming the output display is appropriately calibrated. </dd> <dt>‘<samp class=”samp”>saturation</samp>’</dt> <dd> Performs saturation mapping - that is, stretches the input color volume directly onto the output color volume, in non-linear fashion that preserves the original signal appearance as much as possible. This intent is recommended for signal content evaluation, as it will not lead to any clipping. It is roughly analogous to not performing

any color mapping, although it still takes into account the mastering display primaries and any differences in encoding TRC. </dd> </dl>

</dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">in_color_matrix</samp></dt> <dt><samp class="option">out_color_matrix</samp><
<dd> Set in/output YCbCr color space type. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the color space type depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> Format conforming to International Telecommunication Union (ITU) Recommendation BT.709. </dd> <dt>‘<samp class="samp">fcc</samp>’</dt> <dd> Set color space conforming to the United States Federal Communications Commission (FCC) Code of Federal Regulations (CFR) Title 47 (2003) 73.682 (a). </dd> <dt>‘<samp class="samp">bt601</samp>’</dt> <dt>‘<samp class="samp">bt470</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> Set color space conforming to: <ul class="itemize mark-bullet"> ITU Radiocommunication Sector (ITU-R) Recommendation BT.601

ITU-R Rec. BT.470-6 (1998) Systems B, B1, and G

Society of Motion Picture and Television Engineers (SMPTE) ST 170:2004

</dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> Set color space conforming to SMPTE ST 240:1999. </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> Set color space conforming to ITU-R BT.2020 non-constant luminance system. </dd> </dl>

</dd> <dt><samp class="option">in_range</samp></dt> <dt><samp class="option">out_range</samp></dt> <dd> Set in/output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt>

<dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> <dt><samp class="option">in_chroma_loc</samp></dt> <dt><samp class="option">out_chroma_loc</samp></dt> <dd> Set in/output chroma sample location. If <samp class="option">in_chroma_loc</samp> is not specified, the chroma location tagged on the input frame is used, and untagged input is assumed to be sited like the output, so it is tagged but not converted. If <samp class="option">out_chroma_loc</samp> is not specified, the output is converted to the chroma location negotiated with the rest of the filtergraph, or keeps the input location if none was negotiated. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, unknown</samp>’</dt> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt> <dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dt><samp class="option">out_primaries</samp></dt> <dd> Set in/output RGB primaries. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dt>‘<samp class="samp">ebu3213</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_transfer</samp></dt> <dt><samp class="option">out_transfer</samp></dt> <dd> Set in/output transfer response curve (TRC). This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt>

<dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dt>‘<samp class="samp">srgb</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dt>‘<samp class="samp">xvycc</samp>’</dt> <dt>‘<samp class="samp">bt1361e</samp>’</dt> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dd> Enable decreasing or increasing output video width or height if necessary to keep the original aspect ratio. Possible values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dd> Scale the video as specified and disable this feature. </dd> <dt>‘<samp class="samp">decrease</samp>’</dt> <dd> The output video dimensions will automatically be decreased if needed. </dd> <dt>‘<samp class="samp">increase</samp>’</dt> <dd> The output video dimensions will automatically be increased if needed. </dd> </dl>

One useful instance of this option is that when you know a specific device’s maximum allowed resolution, you can use this to limit the output video to that, while retaining the aspect ratio. For example, device A allows 1280x720 playback, and your video is 1920x800. Using this option (set it to decrease) and specifying 1280x720 to the command line makes the output 1280x533. Please note that this is a different thing than specifying -1 for <samp class="option">w</samp> or <samp class="option">h</samp>, you still need to specify the output resolution for this option to work. </dd> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Ensures that both the output dimensions, width and height, are divisible by the given integer when used together with <samp class="option">force_original_aspect_ratio</samp>. This works similar to using <code class="code">-n</code> in the <samp class="option">w</samp> and <samp class="option">h</samp> options. This option respects the value set for <samp class="option">force_original_aspect_ratio</samp>, increasing or decreasing the resolution accordingly. The video’s aspect ratio may be slightly modified. This option can be handy if you need to have a video fit within or exceed a defined resolution using <samp class="option">force_original_aspect_ratio</samp> but also have encoder restrictions on width or height divisibility. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Enabling this option leads to the output SAR being reset to 1. Additionally, if the user requests proportional scaling either through the width or height expressions, e.g. <code class="code">w=-4:h=360</code> or <code class="code">w=iw/2:h=-1</code> or by enabling <code class="code">force_original_aspect_ratio</code>,

then the input DAR is taken into account and the output is scaled to produce square pixels. Default is false. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt> <dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt> <dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt> <dd> The same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt> <dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt> <dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt> <dt><var class="var">ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">n</var></dt> <dd> The (sequential) number of the input frame, starting from 0. Only available with <code class="code">eval=frame</code>. </dd> <dt><var class="var">t</var></dt> <dd> The presentation timestamp of the input frame, expressed as a number of seconds. Only available with <code class="code">eval=frame</code>. </dd> <dt><var class="var">pos</var></dt> <dd> The position (byte offset) of the frame in the input stream, or NaN if this information is unavailable and/or meaningless (for example in case of synthetic video). Only available with <code class="code">eval=frame</code>. Deprecated, do not use. </dd> <dt><var class="var">ref_w, rw</var></dt> <dt><var class="var">ref_h, rh</var></dt> <dt><var class="var">ref_a</var></dt> <dt><var class="var">ref_dar, rdar</var></dt> <dt><var class="var">ref_n</var></dt> <dt><var class="var">ref_t</var></dt> <dt><var class="var">ref_pos</var></dt> <dd> Eqvuialent to the above, but for a second reference input. If any of these variables are present, this filter accepts two inputs. </dd> </dl>

 <h4 class="subsection">39.222.2 Examples<a class="anchor hidden-xs" href="#Examples-

166" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Scale the input video to a size of 200x100 <pre class="example-preformatted">scale=w=200:h=100 </pre>

This is equivalent to: <pre class="example-preformatted">scale=200:100 </pre>

or: <pre class="example-preformatted">scale=200x100 </pre>

Specify a size abbreviation for the output size: <pre class="example-preformatted">scale=qcif </pre>

which can also be written as: <pre class="example-preformatted">scale=size=qcif </pre>

Scale the input to 2x: <pre class="example-preformatted">scale=w=2*iw:h=2*ih </pre>

The above is the same as: <pre class="example-preformatted">scale=2*in_w:2*in_h </pre>

Scale the input to 2x with forced interlaced scaling: <pre class="example-preformatted">scale=2*iw:2*ih:interl=1 </pre>

Scale the input to half size: <pre class="example-preformatted">scale=w=iw/2:h=ih/2 </pre>

Increase the width, and set the height to the same size: <pre class="example-preformatted">scale=3/2*iw:ow </pre>

Seek Greek harmony: <pre class="example-preformatted">scale=iw:1/PHI*iw scale=ih*PHI:ih </pre>

Increase the height, and set the width to 3/2 of the height: <pre class="example-preformatted">scale=w=3/2*oh:h=3/5*ih </pre>

Increase the size, making the size a multiple of the chroma subsample values: <pre class="example-preformatted">scale="trunc(3/2*iw/hsub)*hsub:t </pre>

Increase the width to a maximum of 500 pixels, keeping the same aspect ratio as the input: <pre class="example-preformatted">scale=w='min(500\, iw*3/2):h=-1' </pre>

Make pixels square by combining scale and setsar: <pre class="example-preformatted">scale='trunc(ih*dar):ih',setsar=1/1

</pre>

Make pixels square using reset_sar, making sure the resulting resolution is even (required by some codecs): <pre class=”example-preformatted”>scale='-2:ih-mod(ih,2):reset_sar=1' </pre>

Scale to target exactly, however reset SAR to 1: <pre class=”example-preformatted”>scale='400:300:reset_sar=1' </pre>

Scale to even dimensions that fit within 400x300, preserving input SAR: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_rat
</pre>

Scale to produce square pixels with even dimensions that fit within 400x300: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_
</pre>

Scale a subtitle stream (sub) to match the main video (main) in size before overlaying. ("scale2ref") <pre class=”example-preformatted”>'[main]split[a][b]; [ref][a]scale=rw:rh[c]; [b][c]overlay' </pre>

Scale a logo to 1/10th the height of a video, while preserving its display aspect ratio. <pre class=”example-preformatted”>[logo-in][video-in]scale=w=oh*dar:h=rh/10[logo-out] </pre>

 <h4 class=”subsection”>39.222.3 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>39.223 scharr# TOC</h3> Apply scharr operator to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp

class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class=”subsection”>39.223.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.224 scroll# TOC</h3> Scroll input video horizontally and/or vertically by constant speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>hpos</samp></dt> <dd> Set the initial horizontal scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> <dt><samp class=”option”>vpos</samp></dt> <dd> Set the initial vertical scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> </dl>

 <h4 class=”subsection”>39.224.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. </dd> </dl>

 <h3 class=”section”>39.225 scdet# TOC</h3>

Detect video scene change. This filter sets frame metadata with mafd between frame, the scene score, and forward the frame to the next filter, so they can use these metadata to detect scene change or others. In addition, this filter logs a message and sets frame metadata when it detects a scene change by <samp class=”option”>threshold</samp>. <code class=”code”>lavfi.scd.mafd</code> metadata keys are set with mafd for every frame. <code class=”code”>lavfi.scd.score</code>

metadata keys are set with scene change score for every frame to detect scene change. <code class="code">lavfi.scd.time</code> metadata keys are set with current filtered frame time which detect scene change with <samp class="option">threshold</samp>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold, t</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change. Good values are in the <code class="code">[8.0, 14.0]</code> range. The range for <samp class="option">threshold</samp> is <code class="code">[0., 100.]</code>. Default value is <code class="code">10.</code>. </dd> <dt><samp class="option">sc_pass, s</samp></dt> <dd> Set the flag to pass scene change frames to the next filter. Default value is <code class="code">0</code> You can enable it if you want to get snapshot of scene change frames only. </dd> </dl>

 <h3 class="section">39.226 selectivecolor#
TOC</h3>

Adjust cyan, magenta, yellow and black (CMYK) to certain ranges of colors (such as "reds", "yellows", "greens", "cyans", ...). The adjustment range is defined by the "purity" of the color (that is, how saturated it already is). This filter is similar to the Adobe Photoshop Selective Color tool. The filter accepts the following options: <dl class="table"> <dt><samp class="option">correction_method</samp></dt> <dd> Select color correction method. Available values are: <dl class="table">
<dt>‘<samp class="samp">absolute</samp>’</dt> <dd> Specified adjustments are applied "as-is" (added/subtracted to original pixel component value). </dd> <dt>‘<samp class="samp">relative</samp>’</dt>
<dd> Specified adjustments are relative to the original component value. </dd> </dl> Default is <code class="code">absolute</code>. </dd> <dt><samp class="option">reds</samp></dt> <dd> Adjustments for red pixels (pixels where the red component is the maximum) </dd>
<dt><samp class="option">yellows</samp></dt> <dd> Adjustments for yellow pixels (pixels where the blue component is the minimum) </dd>
<dt><samp class="option">greens</samp></dt> <dd> Adjustments for green pixels (pixels where the green component is the maximum) </dd> <dt><samp class="option">cyans</samp></dt> <dd> Adjustments for cyan pixels (pixels where the red component is the minimum) </dd>
<dt><samp class="option">blues</samp></dt> <dd> Adjustments for blue pixels (pixels where the blue component is the maximum) </dd> <dt><samp class="option">magentas</samp></dt>
<dd> Adjustments for magenta pixels (pixels where the green component is the minimum) </dd> <dt><samp class="option">whites</samp></dt>
<dd> Adjustments for white pixels (pixels where all components are greater

than 128) </dd> <dt><samp class="option">neutrals</samp></dt> <dd> Adjustments for all pixels except pure black and pure white </dd> <dt><samp class="option">blacks</samp></dt> <dd> Adjustments for black pixels (pixels where all components are lesser than 128) </dd> <dt><samp class="option">psfile</samp></dt> <dd> Specify a Photoshop selective color file (<code class="code">.asv</code>) to import the settings from. </dd> </dl>

All the adjustment settings (<samp class="option">reds</samp>, <samp class="option">yellows</samp>, ...) accept up to 4 space separated floating point adjustment values in the [-1,1] range, respectively to adjust the amount of cyan, magenta, yellow and black for the pixels of its range. <h4 class="subsection">39.226.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Increase cyan by 50% and reduce yellow by 33% in every green areas, and increase magenta by 27% in blue areas: <pre class="example-preformatted">selectivecolor=greens=.5 0 -.33 0:blues=0 .27 </pre>

Use a Photoshop selective color preset: <pre class="example-preformatted">selectivecolor=psfile=MySelectiveColorPresets/Misty.asv </pre>

 <h3 class="section">39.227 separatefields# TOC</h3>

The <code class="code">separatefields</code> takes a frame-based video input and splits each frame into its components fields, producing a new half height clip with twice the frame rate and twice the frame count. This filter use field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class="code">separatefields</code> filter. <h3 class="section">39.228 setdar, setsar# TOC</h3>

The <code class="code">setdar</code> filter sets the Display Aspect Ratio for the filter output video. This is done by changing the specified Sample (aka Pixel) Aspect Ratio, according to the following equation: <pre class="example-preformatted"><var class="var">DAR</var> = <var class="var">HORIZONTAL_RESOLUTION</var>

/ <var class=”var”>VERTICAL_RESOLUTION</var> * <var class=”var”>SAR</var> </pre>

Keep in mind that the <code class=”code”>setdar</code> filter does not modify the pixel dimensions of the video frame. Also, the display aspect ratio set by this filter may be changed by later filters in the filterchain, e.g. in case of scaling or if another "setdar" or a "setsar" filter is applied. The <code class=”code”>setsar</code> filter sets the Sample (aka Pixel) Aspect Ratio for the filter output video. Note that as a consequence of the application of this filter, the output display aspect ratio will change according to the equation above. Keep in mind that the sample aspect ratio set by the <code class=”code”>setsar</code> filter may be changed by later filters in the filterchain, e.g. if another "setsar" or a "setdar" filter is applied. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>r, ratio, dar (<code class=”code”>setdar</code> only), sar (<code class=”code”>setsar</code> only)</samp></dt> <dd> Set the aspect ratio used by the filter. The parameter can be a floating point number string, or an expression. If the parameter is not specified, the value "0" is assumed, meaning that the same input value is used. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Set the maximum integer value to use for expressing numerator and denominator when reducing the expressed aspect ratio to a rational. Default value is <code class=”code”>100</code>. </dd> </dl>

The parameter <var class=”var”>sar</var> is an expression containing the following constants: <dl class=”table”> <dt><samp class=”option”>w, h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Same as <var class=”var”>w</var> / <var class=”var”>h</var>. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio. It is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub, vsub</samp></dt> <dd> Horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

 <h4 class=”subsection”>39.228.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> To change the display aspect ratio to 16:9, specify one of the following: <pre class=”example-preformatted”>setdar=dar=1.77777 setdar=dar=16/9 </pre>

To change the sample aspect ratio to 10:11, specify: <pre class=”example-preformatted”>setsar=sar=10/11 </pre>

To set a display aspect ratio of 16:9, and specify a maximum integer value of 1000 in the aspect ratio reduction, use the command: <pre class=”example-preformatted”>setdar=ratio=16/9:max=1000 </pre>

 <h3 class=”section”>39.229 setfield# TOC</h3>

Force field for the output video frame. The <code class=”code”>setfield</code> filter marks the interlace type field for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters (e.g. <code class=”code”>fieldorder</code> or <code class=”code”>yadif</code>). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same field property. </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl> </dd> </dl>

 <h3 class=”section”>39.230 setparams# TOC</h3>

Force frame parameter for the output video frame. The <code class=”code”>setparams</code> filter marks interlace and color range for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by filters/encoders. <dl class=”table”> <dt><samp class=”option”>field_mode</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same field property (default). </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl>

</dd> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property (default). </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Mark the frame as unspecified color range. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Mark the frame as limited range. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Mark the frame as full range. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> </dl>

</dd> <dt><samp class="option">color_trc</samp></dt> <dd> Set the color transfer. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color trc property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">log100</samp>’</dt> <dt>‘<samp class="samp">log316</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dt>‘<samp class="samp">bt1361e</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">colorspace</samp></dt> <dd> Set the colorspace. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same col-

orspace property (default). </dd> <dt>‘<samp class="samp">gbr</samp>’</dt>
<dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp
class="samp">unknown</samp>’</dt> <dt>‘<samp
class="samp">fcc</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</d
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dt>‘<samp class="samp">ycgco</samp>’</dt> <dt>‘<samp
class="samp">bt2020nc</samp>’</dt> <dt>‘<samp
class="samp">bt2020c</samp>’</dt> <dt>‘<samp class="samp">smpte2085</samp>&rsq
<dt>‘<samp class="samp">chroma-derived-nc</samp>’</dt>
<dt>‘<samp class="samp">chroma-derived-c</samp>’</dt>
<dt>‘<samp class="samp">ictcp</samp>’</dt> </dl>

</dd> <dt><samp class="option">chroma_location</samp></dt>
<dd> Set the chroma sample location. Available values are: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Keep the same chroma location (default). </dd>
<dt>‘<samp class="samp">unspecified, unknown</samp>’</dt>
<dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp
class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</
<dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp
class="samp">bottomleft</samp>’</dt> <dt>‘<samp
class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> Set
the alpha moda. Available values are: <dl class="table"> <dt>‘<samp
class="samp">auto</samp>’</dt> <dd> Keep the same alpha
mode (default). </dd> <dt>‘<samp class="samp">unspecified,
unknown</samp>’</dt> <dt>‘<samp class="samp">premultiplied</samp>’</dt>
<dt>‘<samp class="samp">straight</samp>’</dt> </dl>
</dd> </dl>

 <h3 class="section">39.231 shear<span class="pull-
right">#
TOC</h3>
Apply shear transform to input video. This filter sup-
ports the following options: <dl class="table"> <dt><samp
class="option">shx</samp></dt> <dd> Shear factor in X-direction.
Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp
class="option">shy</samp></dt> <dd> Shear factor in Y-direction.
Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp
class="option">fillcolor, c</samp></dt> <dd> Set the color used
to fill the output area not covered by the transformed video. For the
general syntax of this option, check the <a data-manual="ffmpeg-utils"
href="./ffmpeg-utils.html#color-syntax">"Color" section in the
ffmpeg-utils manual. If the special value "none" is se-
lected then no background is printed (useful for example if the background

is never shown). Default value is "black". </dd> <dt><samp class="option">interp</samp></dt> <dd> Set interpolation type. Can be <code class="code">bilinear</code> or <code class="code">nearest</code>. Default is <code class="code">bilinear</code>. </dd> </dl>

 <h4 class="subsection">39.231.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.232 showinfo# TOC</h3>

Show a line containing various information for each input video frame. The input video is not modified. This filter supports the following options: <dl class="table"> <dt><samp class="option">checksum</samp></dt> <dd> Calculate checksums of each plane. By default enabled. </dd> <dt><samp class="option">udu_sei_as_ascii</samp></dt> <dd> Try to print user data unregistered SEI as ascii character when possible, in hex format otherwise. </dd> </dl>

The shown line contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var>. The following values are shown in the output: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of time base units. The time base unit depends on the filter input pad. </dd> <dt><samp class="option">pts_time</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of seconds. </dd> <dt><samp class="option">fmt</samp></dt> <dd> The pixel format name. </dd> <dt><samp class="option">sar</samp></dt> <dd> The sample aspect ratio of the input frame, expressed in the form <var class="var">num</var>/<var class="var">den</var>. </dd> <dt><samp class="option">s</samp></dt> <dd> The size of the input frame. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">i</samp></dt> <dd> The type of interlaced mode ("P" for "progressive", "T" for top field first, "B" for bottom field first). </dd> <dt><samp class="option">iskey</samp></dt> <dd> This

is 1 if the frame is a key frame, 0 otherwise. </dd> <dt><samp class=”option”>type</samp></dt> <dd> The picture type of the input frame ("I" for an I-frame, "P" for a P-frame, "B" for a B-frame, or "?" for an unknown type). Also refer to the documentation of the <code class=”code”>AVPictureType</code> enum and of the <code class=”code”>av_get_picture_type_char</code> function defined in <samp class=”file”>libavutil/avutil.h</samp>. </dd> <dt><samp class=”option”>checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of all the planes of the input frame. </dd> <dt><samp class=”option”>plane_checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of each plane of the input frame, expressed in the form "[<var class=”var”>c0</var> <var class=”var”>c1</var> <var class=”var”>c2</var> <var class=”var”>c3</var>]". </dd> <dt><samp class=”option”>mean</samp></dt> <dd> The mean value of pixels in each plane of the input frame, expressed in the form "[<var class=”var”>mean0</var> <var class=”var”>mean1</var> <var class=”var”>mean2</var> <var class=”var”>mean3</var>]". </dd> <dt><samp class=”option”>stdev</samp></dt> <dd> The standard deviation of pixel values in each plane of the input frame, expressed in the form "[<var class=”var”>stdev0</var> <var class=”var”>stdev1</var> <var class=”var”>stdev2</var> <var class=”var”>stdev3</var>]". </dd> </dl>

 <h3 class=”section”>39.233 showpalette# TOC</h3>

Displays the 256 colors palette of each frame. This filter is only relevant for <var class=”var”>pal8</var> pixel format frames. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>s</samp></dt> <dd> Set the size of the box used to represent one palette color entry. Default is <code class=”code”>30</code> (for a <code class=”code”>30x30</code> pixel box). </dd> </dl>

 <h3 class=”section”>39.234 shuffleframes# TOC</h3>

Reorder and/or duplicate and/or drop video frames. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mapping</samp></dt> <dd> Set the destination indexes of input frames. This is space or ’|’ separated list of indexes that maps input frames to output frames. Number of indexes also sets maximal value that each index may have. ’-1’ index have special meaning and that is to drop frame. </dd> </dl>

The first frame has the index 0. The default is to keep the input unchanged. <h4 class="subsection">39.234.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap second and third frame of every three frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=0 2 1" OUTPUT </pre>

Swap 10th and 1st frame of every ten frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=9 1 2 3 4 5 6 7 8 0" OUTPUT </pre>

 <h3 class="section">39.235 shufflepixels# TOC</h3>

Reorder pixels in video frames. This filter accepts the following options: <dl class="table"> <dt><samp class="option">direction, d</samp></dt> <dd> Set shuffle direction. Can be forward or inverse direction. Default direction is forward. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set shuffle mode. Can be horizontal, vertical or block mode. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set shuffle block_size. In case of horizontal shuffle mode only width part of size is used, and in case of vertical shuffle mode only height part of size is used. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set random seed used with shuffling pixels. Mainly useful to set to be able to reverse filtering process to get original input. For example, to reverse forward shuffle you need to use same parameters and exact same seed and to set direction to inverse. </dd> </dl>

 <h3 class="section">39.236 shuffleplanes# TOC</h3>

Reorder and/or duplicate video planes. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map0</samp></dt> <dd> The index of the input plane to be used as the first output plane. </dd> <dt><samp class="option">map1</samp></dt> <dd> The index of the input plane to be used as the second output plane. </dd> <dt><samp class="option">map2</samp></dt> <dd> The index of the input plane to be used as the third output plane. </dd> <dt><samp class="option">map3</samp></dt>

<dd> The index of the input plane to be used as the fourth output plane. </dd> </dl>

The first plane has the index 0. The default is to keep the input unchanged. <h4 class="subsection">39.236.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap the second and third planes of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf shuffleplanes=0:2:1:3 OUTPUT </pre>

<h3 class="section">39.237 signalstats# TOC</h3>

Evaluate various visual metrics that assist in determining issues associated

with the digitization of analog video media.

By default the filter will log these metadata values:

<dl class="table">

<dt><samp class="option">YMIN</samp></dt>

<dd> Display the minimal Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YLOW</samp></dt>

<dd> Display the Y value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YAVG</samp></dt>

<dd> Display the average Y value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class="option">YHIGH</samp></dt>

<dd> Display the Y value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YMAX</samp></dt>

<dd> Display the maximum Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UMIN</samp></dt>

<dd> Display the minimal U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">ULOW</samp></dt>

<dd> Display the U value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UAVG</samp></dt>

<dd> Display the average U value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>UHIGH</samp></dt>

<dd> Display the U value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>UMAX</samp></dt>

<dd> Display the maximum U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VMIN</samp></dt>

<dd> Display the minimal V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VLOW</samp></dt>

<dd> Display the V value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VAVG</samp></dt>

<dd> Display the average V value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>VHIGH</samp></dt>

<dd> Display the V value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VMAX</samp></dt>

<dd> Display the maximum V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">SATMIN</samp></dt>

<dd> Display the minimal saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class="option">SATLOW</samp></dt>

<dd> Display the saturation value at the 10% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class="option">SATAVG</samp></dt>

<dd> Display the average saturation value within the input frame. Expressed in range

of [0-~181.02].

</dd>

<dt><samp class="option">SATHIGH</samp></dt>

<dd> Display the saturation value at the 90% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATMAX</samp></dt>

<dd> Display the maximum saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>HUEMED</samp></dt>

<dd> Display the median value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>HUEAVG</samp></dt>

<dd> Display the average value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>YDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the Y

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>UDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the U

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>VDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the V

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>YBITDEPTH</samp></dt>

<dd> Display bit depth of Y plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class=”option”>UBITDEPTH</samp></dt>

<dd> Display bit depth of U plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class=”option”>VBITDEPTH</samp></dt>

<dd> Display bit depth of V plane in current frame.

Expressed in range of [0-16].

</dd>

</dl>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stat</samp></dt> <dt><samp class=”option”>out</samp></dt> <dd> <samp class=”option”>stat</samp> specify an additional form of image analysis. <samp class=”option”>out</samp> output video with the specified type of pixel highlighted. Both options accept the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tout</samp>’</dt> <dd> Identify <var class=”var”>temporal outliers</var> pixels. A <var class=”var”>temporal outlier</var> is a pixel unlike the neighboring pixels of the same field. Examples of temporal outliers include the results of video dropouts,

head clogs, or tape tracking issues. </dd> <dt>‘<samp class=”samp”>vrep</samp>’</dt> <dd> Identify <var class=”var”>vertical line repetition</var>. Vertical line repetition includes similar rows of pixels within a frame. In born-digital video vertical line repetition is common, but this pattern is uncommon in video digitized from an analog source. When it occurs in video that results from the digitization of an analog source it can indicate concealment from a dropout compensator. </dd> <dt>‘<samp class=”samp”>brng</samp>’</dt> <dd> Identify pixels that fall outside of legal broadcast range. </dd> </dl>

</dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Set the highlight color for the <samp class=”option”>out</samp> option. The default color is yellow. </dd> </dl>

 <h4 class=”subsection”>39.237.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Output data of various video metrics: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats="stat=tout+vrep+brng" -show_frames </pre>

Output specific data about the minimum and maximum values of the Y plane per frame: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats -show_entries frame_tags=lavfi.signalstats.YMAX,lavfi.signalstats.YMIN </pre>

Playback video while highlighting pixels that are outside of broadcast range in red. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats="out=brng:color=red" </pre>

Playback video with signalstats metadata drawn over the frame. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats=stat=brng+vrep+tout,drawtext=fontfile=FreeSerif.ttf:textfile=signalstat_drawtext.txt </pre>

The contents of signalstat_drawtext.txt used in the command are: <pre class=”example-preformatted”>time %{pts:hms} Y (%{metadata:lavfi.signalstats.YMIN}-%{metadata:lavfi.signalstats.YMAX}) U (%{metadata:lavfi.signalstats.UMIN}-%{metadata:lavfi.signalstats.UMAX}) V (%{metadata:lavfi.signalstats.VMIN}-%{metadata:lavfi.signalstats.VMAX}) saturation maximum: %{metadata:lavfi.signalstats.SATMAX}

</pre>

 <h3

class=”section”>39.238 signature# TOC</h3>

Calculates the MPEG-7 Video Signature. The filter can handle more than one input. In this case the matching between the inputs can be calculated additionally. The filter always passes through the first input. The signature of each stream can be written into a file. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>detectmode</samp></dt> <dd> Enable or disable the matching process. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>off</samp>’</dt> <dd> Disable the calculation of a matching (default). </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Calculate the matching for the whole video and output whether the whole video matches or only parts. </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dd> Calculate only until a matching is found or the video ends. Should be faster in some cases. </dd> </dl>

</dd> <dt><samp class=”option”>nb_inputs</samp></dt> <dd> Set the number of inputs. The option value must be a non negative integer. Default value is 1. </dd> <dt><samp class=”option”>filename</samp></dt> <dd> Set the path to which the output is written. If there is more than one input, the path must be a prototype, i.e. must contain %d or %0nd (where n is a positive integer), that will be replaced with the input number. If no filename is specified, no output will be written. This is the default. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Choose the output format. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>binary</samp>’</dt> <dd> Use the specified binary representation (default). </dd> <dt>‘<samp class=”samp”>xml</samp>’</dt> <dd> Use the specified xml representation. </dd> </dl>

</dd> <dt><samp class=”option”>th_d</samp></dt> <dd> Set threshold to detect one word as similar. The option value must be an integer greater than zero. The default value is 9000. </dd> <dt><samp class=”option”>th_dc</samp></dt> <dd> Set threshold to detect all words as similar. The option value must be an integer greater than zero. The default value is 60000. </dd> <dt><samp class=”option”>th_xh</samp></dt> <dd> Set threshold to detect frames as similar. The option value must be an integer greater than zero. The default value is 116. </dd> <dt><samp class=”option”>th_di</samp></dt> <dd> Set the minimum length of a sequence in frames to recognize it as matching sequence. The option value must be a non negative integer value. The default value is 0. </dd> <dt><samp class=”option”>th_it</samp></dt> <dd> Set the minimum relation, that matching frames to all frames must have. The

option value must be a double value between 0 and 1. The default value is 0.5. </dd> </dl>

 <h4 class="subsection">39.238.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> To calculate the signature of an input video and store it in signature.bin: <pre class="example-preformatted">ffmpeg -i input.mkv -vf signature=filename=signature.bin -map 0:v -f null - </pre>

To detect whether two videos match and store the signatures in XML format in signature0.xml and signature1.xml: <pre class="example-preformatted">ffmpeg -i input1.mkv -i input2.mkv -filter_complex "[0:v][1:v] signature=nb_inputs=2:detectmode=full:format=xml:filename=signature%d.x -map :v -f null - </pre>

 <h3 class="section">39.239 siti# TOC</h3>

Calculate Spatial Information (SI) and Temporal Information (TI) scores for a video, as defined in ITU-T Rec. P.910 (11/21): Subjective video quality assessment methods for multimedia applications. Available PDF at https://www.itu.int/rec/T-REC-P.910-202111-S/en. Note that this is a legacy implementation that corresponds to a superseded recommendation. Refer to ITU-T Rec. P.910 (07/22) for the latest version: https://www.itu.int/rec/T-REC-P.910-202207-I/en It accepts the following option: <dl class="table"> <dt><samp class="option">print_summary</samp></dt> <dd> If set to 1, Summary statistics will be printed to the console. Default 0. </dd> </dl>

 <h4 class="subsection">39.239.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> To calculate SI/TI metrics and print summary: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf siti=print_summary=1 -f null - </pre>

 <h3 class="section">39.240 smartblur<a class="anchor

hidden-xs" href="#smartblur" aria-hidden="true"># TOC</h3>

Blur the input video without impacting the outlines. It accepts the following options: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dd> Set the luma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is 1.0. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set the luma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is 1.0. </dd> <dt><samp class="option">luma_threshold, lt</samp></dt> <dd> Set the luma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is 0. </dd> <dt><samp class="option">chroma_radius, cr</samp></dt> <dd> Set the chroma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set the chroma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">chroma_threshold, ct</samp></dt> <dd> Set the chroma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set the alpha radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">alpha_strength, as</samp></dt> <dd> Set the alpha strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">alpha_threshold, at</samp></dt> <dd> Set the alpha threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30]

will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> </dl>

If a chroma or alpha option is not explicitly set, the corresponding luma value is set. <h3 class="section">39.241 sobel# TOC</h3> Apply sobel operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">39.241.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">39.242 spp# TOC</h3>

Apply a simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">6</code> - all) shifts and average the results. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-6. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">6</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding (default). </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> </dl>

</dd> <dt><samp class="option">use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class="code">1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class="code">0</code> (not enabled). </dd> </dl>

 <h4 class="subsection">39.242.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">quality, level</samp></dt> <dd> Set quality level. The value <code class="code">max</code> can be used to set the maximum level, currently <code class="code">6</code>. </dd> </dl>

 <h3 class="section">39.243 sr# TOC</h3>

Scale the input by applying one of the super-resolution methods based on convolutional neural networks. Supported models: <ul class="itemize mark-bullet"> Super-Resolution Convolutional Neural Network model (SRCNN). See https://arxiv.org/abs/1501.00092.

Efficient Sub-Pixel Convolutional Neural Network model (ESPCN). See https://arxiv.org/abs/1609.05158.

Training scripts as well as scripts for model file (.pb) saving can be found at <a class="url" href="https://github.com/XueweiMeng/sr/tree/sr_dnn_native">https://github.com/Xuewei Original repository is at <a class="url" href="https://github.com/HighVoltageRocknRoll/sr.git">https://gith The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class="url" href="https://www.tensorflow.org/install/lang_c">https://www.tensorflow.org/install/lang_ and configure FFmpeg with <code class="code">--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO backend can load files for only its format. </dd> <dt><samp class="option">scale_factor</samp></dt> <dd> Set scale factor for

SRCNN model. Allowed values are <code class=”code”>2</code>, <code class=”code”>3</code> and <code class=”code”>4</code>. Default value is <code class=”code”>2</code>. Scale factor is necessary for SRCNN model, because it accepts input upscaled using bicubic upscaling with proper scale factor. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>39.244 sr_amf# TOC</h3>

Upscale (size increasing) for the input video using AMD Advanced Media Framework library for hardware acceleration. Use advanced algorithms for upscaling with higher output quality. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>algorithm</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> <dt><var class=”var”>sr1-0</var></dt> <dd> Video SR1.0 This is a default value </dd> <dt><var class=”var”>point</var></dt> <dd> Point AMF returns an unwritten surface for this algorithm when the format is <code class=”code”>nv12</code> or <code class=”code”>p010</code>, so inputs in those formats are converted to a packed RGB format automatically. This is a driver bug rather than a documented restriction; the conversion can be dropped once a driver that scales <code class=”code”>nv12</code> and <code class=”code”>p010</code> correctly with <code class=”code”>point</code> is in wide use. </dd> <dt><var class=”var”>sr1-1</var></dt> <dd> Video SR1.1 This algorithm only supports packed RGB formats and requires a DirectX 11 or DirectX 12 device, so it is available on Windows only. Inputs in other formats such as <code class=”code”>nv12</code> or <code class=”code”>p010</code> are converted to a packed RGB format automatically. Note that the VideoSR algorithms (<code class=”code”>sr1-0</code> and <code class=”code”>sr1-1</code>) do not preserve the alpha channel (it is left zeroed); drop it after the download, e.g. with <code class=”code”>hwdownload,format=rgba,format=rgb24</code>, if an opaque result is required. </dd> </dl>

</dd> <dt><samp class=”option”>sharpness</samp></dt> <dd> Control

hq scaler sharpening. The value is a float in the range of [0.0, 2.0] </dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. The <code class="code">point</code> and <code class="code">sr1-1</code> algorithms emit the packed RGB format the scaler receives, so for them this option must name one of <code class="code">rgba</code>, <code class="code">bgra</code>, <code class="code">x2bgr10le</code> or <code class="code">rgbaf16le</code>. A named format also decides what software input is converted to, and what a hardware surface in another format is converted to on the GPU. Without it, <code class="code">x2bgr10le</code> is selected for <code class="code">p010</code> input and <code class="code">rgba</code> for other non-RGB input, and the result is tagged as full range RGB. libswscale cannot convert other software formats to <code class="code">rgbaf16le</code>; use software input already in that format or a hardware input for GPU conversion. </dd> <dt><samp class="option">keep-ratio</samp></dt> <dd> Force the scaler to keep the aspect ratio of the input image when the output size has a different aspect ratio. Default value is false. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specifies whether the output image outside the region of interest, which does not fill the entire output surface should be filled with a solid color. </dd> </dl>

 <h4 class="subsection">39.244.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Upscale to 720p, keeping aspect ratio, and read the result back into system memory. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class="example-preformatted">sr_amf=-2:720,hwdownload,format=nv12
</pre>

Upscale to 4K with algorithm video SR1.1. <pre class="example-preformatted">sr_amf=4096:2160:algorithm=sr1-1
</pre>

 <h3 class="section">39.245 ssim#
TOC</h3>

Obtain the SSIM (Structural SImilarity Metric) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the SSIM. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have

the same number of frames, which are compared one by one. The filter stores the calculated SSIM of each frame. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the SSIM of each individual frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. A description of each shown parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">Y, U, V, R, G, B</samp></dt> <dd> SSIM of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">All</samp></dt> <dd> SSIM of the compared frames for the whole frame. </dd> <dt><samp class="option">dB</samp></dt> <dd> Same as above but in dB representation. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">39.245.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] ssim="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The SSIM of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with both psnr and ssim at same time: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi "ssim;[0:v][1:v]psnr" -f null - </pre>

Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]ssim" -f null - </pre>

 <h3 class="section">39.246 stereo3d# TOC</h3>

Convert between different stereoscopic image formats. The filters accept the following options: <dl class="table"> <dt><samp

class=”option”>in</samp></dt> <dd> Set stereoscopic image format of input. Available values for input image formats are: <dl class=”table”> <dt>‘<samp class=”samp”>sbsl</samp>’</dt> <dd> side by side parallel (left eye left, right eye right) </dd> <dt>‘<samp class=”samp”>sbsr</samp>’</dt> <dd> side by side crosseye (right eye left, left eye right) </dd> <dt>‘<samp class=”samp”>sbs2l</samp>’</dt> <dd> side by side parallel with half width resolution (left eye left, right eye right) </dd> <dt>‘<samp class=”samp”>sbs2r</samp>’</dt> <dd> side by side crosseye with half width resolution (right eye left, left eye right) </dd> <dt>‘<samp class=”samp”>abl</samp>’</dt> <dt>‘<samp class=”samp”>tbl</samp>’</dt> <dd> above-below (left eye above, right eye below) </dd> <dt>‘<samp class=”samp”>abr</samp>’</dt> <dt>‘<samp class=”samp”>tbr</samp>’</dt> <dd> above-below (right eye above, left eye below) </dd> <dt>‘<samp class=”samp”>ab2l</samp>’</dt> <dt>‘<samp class=”samp”>tb2l</samp>’</dt> <dd> above-below with half height resolution (left eye above, right eye below) </dd> <dt>‘<samp class=”samp”>ab2r</samp>’</dt> <dt>‘<samp class=”samp”>tb2r</samp>’</dt> <dd> above-below with half height resolution (right eye above, left eye below) </dd> <dt>‘<samp class=”samp”>al</samp>’</dt> <dd> alternating frames (left eye first, right eye second) </dd> <dt>‘<samp class=”samp”>ar</samp>’</dt> <dd> alternating frames (right eye first, left eye second) </dd> <dt>‘<samp class=”samp”>irl</samp>’</dt> <dd> interleaved rows (left eye has top row, right eye starts on next row) </dd> <dt>‘<samp class=”samp”>irr</samp>’</dt> <dd> interleaved rows (right eye has top row, left eye starts on next row) </dd> <dt>‘<samp class=”samp”>icl</samp>’</dt> <dd> interleaved columns, left eye first </dd> <dt>‘<samp class=”samp”>icr</samp>’</dt> <dd> interleaved columns, right eye first Default value is ‘<samp class=”samp”>sbsl</samp>’. </dd> </dl>

</dd>

<dt><samp class=”option”>out</samp></dt>

<dd> Set stereoscopic image format of output.

<dl class=”table”>

<dt>‘<samp class=”samp”>sbsl</samp>’</dt>

<dd> side by side parallel (left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbsr</samp>’</dt>
<dd> side by side crosseye (right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">sbs2l</samp>’</dt>
<dd> side by side parallel with half width resolution
(left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbs2r</samp>’</dt>
<dd> side by side crosseye with half width resolution
(right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">abl</samp>’</dt>
<dt>‘<samp class="samp">tbl</samp>’</dt>
<dd> above-below (left eye above, right eye below)

</dd>
<dt>‘<samp class="samp">abr</samp>’</dt>
<dt>‘<samp class="samp">tbr</samp>’</dt>
<dd> above-below (right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">ab2l</samp>’</dt>
<dt>‘<samp class="samp">tb2l</samp>’</dt>
<dd> above-below with half height resolution
(left eye above, right eye below)

</dd>

<dt>‘<samp class="samp">ab2r</samp>’</dt>
<dt>‘<samp class="samp">tb2r</samp>’</dt>
<dd> above-below with half height resolution
(right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">al</samp>’</dt>
<dd> alternating frames (left eye first, right eye second)

</dd>
<dt>‘<samp class="samp">ar</samp>’</dt>
<dd> alternating frames (right eye first, left eye second)

</dd>
<dt>‘<samp class="samp">irl</samp>’</dt>
<dd> interleaved rows (left eye has top row, right eye starts on next row)

</dd>
<dt>‘<samp class="samp">irr</samp>’</dt>
<dd> interleaved rows (right eye has top row, left eye starts on next row)

</dd>
<dt>‘<samp class="samp">arbg</samp>’</dt>
<dd> anaglyph red/blue gray
(red filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class="samp">argg</samp>’</dt>
<dd> anaglyph red/green gray
(red filter on left eye, green filter on right eye)

</dd>

<dt>‘<samp class="samp">arcg</samp>’</dt>

<dd> anaglyph red/cyan gray

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arch</samp>’</dt>

<dd> anaglyph red/cyan half colored

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcc</samp>’</dt>

<dd> anaglyph red/cyan color

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcd</samp>’</dt>

<dd> anaglyph red/cyan color optimized with the least squares projection of dubois

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">agmg</samp>’</dt>

<dd> anaglyph green/magenta gray

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmh</samp>’</dt>

<dd> anaglyph green/magenta half colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmc</samp>’</dt>

<dd> anaglyph green/magenta colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmd</samp>’</dt>

<dd> anaglyph green/magenta color optimized with the least squares projection of dubois

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">aybg</samp>’</dt>

<dd> anaglyph yellow/blue gray

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybh</samp>’</dt>

<dd> anaglyph yellow/blue half colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybc</samp>’</dt>

<dd> anaglyph yellow/blue colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybd</samp>’</dt>

<dd> anaglyph yellow/blue color optimized with the least squares projection of dubois

(yellow filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class=”samp”>ml</samp>’</dt>
<dd> mono output (left eye only)

</dd>
<dt>‘<samp class=”samp”>mr</samp>’</dt>
<dd> mono output (right eye only)

</dd>
<dt>‘<samp class=”samp”>chl</samp>’</dt>
<dd> checkerboard, left eye first

</dd>
<dt>‘<samp class=”samp”>chr</samp>’</dt>
<dd> checkerboard, right eye first

</dd>
<dt>‘<samp class=”samp”>icl</samp>’</dt>
<dd> interleaved columns, left eye first

</dd>
<dt>‘<samp class=”samp”>icr</samp>’</dt>
<dd> interleaved columns, right eye first

</dd>
<dt>‘<samp class=”samp”>hdmi</samp>’</dt>
<dd> HDMI frame pack
</dd>
</dl>

Default value is ‘<samp class="samp">arcd</samp>’. </dd> </dl>

 <h4 class="subsection">39.246.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert input video from side by side parallel to anaglyph yellow/blue dubois: <pre class="example-preformatted">stereo3d=sbsl:aybd </pre>

Convert input video from above below (left eye above, right eye below) to side by side crosseye. <pre class="example-preformatted">stereo3d=abl:sbsr </pre>

 <h3 class="section">39.247 streamselect, astreamselect# TOC</h3> Select video or audio streams. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 2. </dd> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">39.247.1 Commands# TOC</h4>

The <code class="code">streamselect</code> and <code class="code">astreamselect</code> filter supports the following commands: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">39.247.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select first 5 seconds 1st stream and rest of time 2nd stream: <pre class="example-preformatted">sendcmd='5.0 streamselect map 1',streamselect=inputs=2:map=0 </pre>

Same as above, but for audio: <pre class="example-preformatted">asendcmd='5.0 astreamselect map 1',astreamselect=inputs=2:map=0 </pre>

 <h3 class=”section”>39.248 subtitles# TOC</h3>

Draw subtitles on top of input video using the libass library. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libass</code>. This filter also requires a build with libavcodec and libavformat to convert the passed subtitles file to ASS (Advanced Substation Alpha) subtitles format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filename, f</samp></dt> <dd> Set the filename of the subtitle file to read. It must be specified. </dd> <dt><samp class=”option”>original_size</samp></dt> <dd> Specify the size of the original video, the video for which the ASS file was composed. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Due to a misdesign in ASS aspect ratio arithmetic, this is necessary to correctly scale the fonts if the aspect ratio has been changed. </dd> <dt><samp class=”option”>fontsdir</samp></dt> <dd> Set a directory path containing fonts that can be used by the filter. These fonts will be used in addition to whatever the font provider uses. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Process alpha channel, by default alpha channel is untouched. </dd> <dt><samp class=”option”>charenc</samp></dt> <dd> Set subtitles input character encoding. <code class=”code”>subtitles</code> filter only. Only useful if not UTF-8. </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Set subtitles stream index. <code class=”code”>subtitles</code> filter only. </dd> <dt><samp class=”option”>force_style</samp></dt> <dd> Override default style or script info parameters of the subtitles. It accepts a string containing ASS style format <code class=”code”>KEY=VALUE</code> couples separated by ",". </dd> <dt><samp class=”option”>wrap_unicode</samp></dt> <dd> Break lines according to the Unicode Line Breaking Algorithm. Availability requires at least libass release 0.17.0 (or LIBASS_VERSION 0x01600010), and libass must have been built with libunibreak. The option is enabled by default except for native ASS. </dd> <dt><samp class=”option”>shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew,

Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd>
</dl>

The default is <code class="code">auto</code>. </dd> </dl>

If the first key is not specified, it is assumed that the first value specifies the <samp class="option">filename</samp>. For example, to render the file <samp class="file">sub.srt</samp> on top of the input video, use the command: <pre class="example-preformatted">subtitles=sub.srt </pre>

which is equivalent to: <pre class="example-preformatted">subtitles=filename=sub.srt </pre>

To render the default subtitles stream from file <samp class="file">video.mkv</samp>, use: <pre class="example-preformatted">subtitles=video.mkv
</pre>

To render the second subtitles stream from that file, use: <pre class="example-preformatted">subtitles=video.mkv:si=1
</pre>

To make the subtitles stream from <samp class="file">sub.srt</samp> appear in 80% transparent blue <code class="code">DejaVu Serif</code>, use: <pre class="example-preformatted">subtitles=sub.srt:force_style='Fontn
Serif,PrimaryColour=&HCCFF0000' </pre>

 <h3 class="section">39.249 super2xsai# TOC</h3>

Scale the input by 2x and smooth using the Super2xSaI (Scale and Interpolate) pixel art scaling algorithm. Useful for enlarging pixel art images without reducing sharpness. <h3 class="section">39.250 swaprect# TOC</h3>

Swap two rectangular objects in video. This filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dd> Set object width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set object height. </dd> <dt><samp class="option">x1</samp></dt> <dd> Set 1st rect x coordinate. </dd> <dt><samp class="option">y1</samp></dt> <dd> Set 1st rect y coordinate. </dd> <dt><samp class="option">x2</samp></dt> <dd> Set 2nd rect x coordinate. </dd> <dt><samp class="option">y2</samp></dt> <dd> Set 2nd rect y coordinate. All expressions are evaluated once for each frame. </dd> </dl>

The all options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>w</var> / <var class=”var”>h</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> </dl>

 <h4 class=”subsection”>39.250.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.251 swapuv# TOC</h3> Swap U & V plane. <h3 class=”section”>39.252 tblend# TOC</h3> Blend successive video frames. See blend <h3 class=”section”>39.253 telecine# TOC</h3>

Apply telecine process to the video. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> </dl>

 <pre class=”example-preformatted”>Some typical patterns:

NTSC output (30i): 27.5p: 32222 24p: 23 (classic) 24p: 2332 (preferred) 20p: 33 18p: 334 16p: 3444

PAL output (25i): 27.5p: 12222 24p: 222222222223 ("Euro pulldown") 16.67p: 33 16p: 33333334 </pre>

 <h3 class=”section”>39.254 thistogram# TOC</h3>

Compute and draw a color distribution histogram for the input video across time. Unlike histogram video filter which only shows histogram of single input frame at certain time, this filter shows also past histograms of number of frames defined by <code class=”code”>width</code> option. The computed histogram is a representation of the color component distribution in an image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dd> Set width of single color component output. Default value is <code class=”code”>0</code>. Value of <code class=”code”>0</code> means width will be picked from input video. This also set number of passed histograms to keep. Allowed range is [0, 8192]. </dd> <dt><samp class=”option”>display_mode, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class=”samp”>parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dd> Presents information identical to that in the <code class=”code”>parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class=”code”>stack</code>. </dd> <dt><samp class=”option”>levels_mode, m</samp></dt> <dd> Set mode. Can be either <code class=”code”>linear</code>, or <code class=”code”>logarithmic</code>. Default is <code class=”code”>linear</code>. </dd> <dt><samp class=”option”>components, c</samp></dt> <dd> Set what color components to display. Default is <code class=”code”>7</code>. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. Default is <code class=”code”>0.9</code>. </dd> <dt><samp class=”option”>envelope, e</samp></dt> <dd> Show envelope. Default is disabled. </dd> <dt><samp class=”option”>ecolor, ec</samp></dt> <dd> Set envelope color. Default is <code class=”code”>gold</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set

slide mode. Available values for slide is: <dl class="table">
<dt>‘<samp class="samp">frame</samp>’</dt> <dd> Draw
new frame when right border is reached. </dd> <dt>‘<samp
class="samp">replace</samp>’</dt> <dd> Replace old columns
with new ones. </dd> <dt>‘<samp class="samp">scroll</samp>’</dt>
<dd> Scroll from right to left. </dd> <dt>‘<samp
class="samp">rscroll</samp>’</dt> <dd> Scroll from left to
right. </dd> <dt>‘<samp class="samp">picture</samp>’</dt>
<dd> Draw single picture. </dd> </dl>

Default is <code class="code">replace</code>. </dd> </dl>

 <h3 class="section">39.255 threshold<span
class="pull-right"><a class="anchor hidden-xs" href="#threshold" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-threshold"
aria-hidden="true">TOC</h3>

Apply threshold effect to video stream. This filter needs
four video streams to perform thresholding. First stream is stream we are
filtering. Second stream is holding threshold values, third stream is holding
min values, and last, fourth stream is holding max values. The
filter accepts the following option: <dl class="table"> <dt><samp
class="option">planes</samp></dt> <dd> Set which planes will be
processed, unprocessed planes will be copied. By default value 0xf, all planes
will be processed. </dd> </dl>

For example if first stream pixel’s component value is less
then threshold value of pixel component from 2nd threshold stream, third
stream value will picked, otherwise fourth stream pixel component value
will be picked. Using color source filter one can perform
various types of thresholding:
<h4 class="subsection">39.255.1 Commands<a
class="anchor hidden-xs" href="#Commands-147" aria-hidden="true">#
TOC</h4>

This filter supports the all options as commands.
  <h4 class="subsection">39.255.2 Ex-
amples<a class="anchor hidden-xs" href="#Examples-
178" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-178" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Binary threshold, using gray color as
threshold: <pre class="example-preformatted">ffmpeg
-i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=black -f lavfi -i color=white
-lavfi threshold output.avi </pre>

Inverted binary threshold, using gray color as threshold: <div
class="example"> <pre class="example-preformatted">ffmpeg -i 320x240.avi
-f lavfi -i color=gray -f lavfi -i color=white -f lavfi -i color=black -lavfi threshold

output.avi </pre>

Truncate binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=gray -lavfi threshold output.avi </pre>

Threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=white -i 320x240.avi -lavfi threshold output.avi </pre>

Inverted threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=white -lavfi threshold output.avi </pre>

 <h3 class="section">39.256 thumbnail# TOC</h3> Select the most representative frame in a given sequence of consecutive frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> <dt><samp class="option">log</samp></dt> <dd> Set the log level to display picked frame stats. Default is <code class="code">info</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">39.256.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract one picture each 50 frames: <pre class="example-preformatted">thumbnail=50 </pre>

Complete example of a thumbnail creation with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf thumbnail,scale=300:200 -frames:v 1 out.png </pre>

 <h3 class="section">39.257 tile# TOC</h3>

Tile several successive frames together. The untile filter can do the reverse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size in the form <code class="code">COLUMNSxROWS</code>. Range is up to UINT_MAX cells. Default is <code class="code">6x5</code>. </dd> <dt><samp class="option">nb_frames</samp></dt> <dd> Set the maximum number of frames to render in the given area. It must be less than or equal to <var class="var">w</var>x<var class="var">h</var>. The default value is <code class="code">0</code>, meaning all the area will be used. </dd> <dt><samp class="option">margin</samp></dt> <dd> Set the outer border margin in pixels. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">padding</samp></dt> <dd> Set the inner border thickness (i.e. the number of pixels between frames). For more advanced padding options (such as having different values for the edges), refer to the pad video filter. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the unused area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set the number of frames to overlap when tiling several successive frames together. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> <dt><samp class="option">init_padding</samp></dt> <dd> Set the number of frames to initially be empty before displaying first output frame. This controls how soon will one get first output frame. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">39.257.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce 8x8 PNG tiles of all keyframes (<samp class="option">-skip_frame nokey</samp>) in a movie: <pre class="example-preformatted">ffmpeg -skip_frame nokey -i file.avi -vf 'scale=128:72,tile=8x8' -an -fps_mode passthrough keyframes%03d.png </pre> The <samp class="option">-fps_mode passthrough</samp> is necessary to prevent <code class="command">ffmpeg</code> from duplicating each output frame to accommodate the originally detected frame rate. Display <code class="code">5</code> pictures in an area of <code class="code">3x2</code> frames, with <code

class=”code”>7</code> pixels between them, and <code class=”code”>2</code> pixels of initial margin, using mixed flat and named options: <pre class=”example-preformatted”>tile=3x2:nb_frames=5:padding=7:margin=2 </pre>

 <h3 class=”section”>39.258 tiltandshift# TOC</h3> Apply tilt-and-shift effect. What happens when you invert time and space? Normally a video is composed of several frames that represent a different instant of time and shows a scene that evolves in the space captured by the frame. This filter is the antipode of that concept, taking inspiration from tilt and shift photography. A filtered frame contains the whole timeline of events composing the sequence, and this is obtained by placing a slice of pixels from each frame into a single one. However, since there are no infinite-width frames, this is done up the width of the input frame, and a video is recomposed by shifting away one column for each subsequent frame. In order to map space to time, the filter tilts each input frame as well, so that motion is preserved. This is accomplished by progressively selecting a different column from each input frame. The end result is a sort of inverted parallax, so that far away objects move much faster that the ones in the front. The ideal conditions for this video effect are when there is either very little motion and the background is static, or when there is a lot of motion and a very wide depth of field (e.g. wide panorama, while moving on a train). The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tilt</samp></dt> <dd> Tilt video while shifting (default). When unset, video will be sliding a static image, composed of the first column of each frame. </dd> <dt><samp class=”option”>start</samp></dt> <dd> What to do at the start of filtering (see below). </dd> <dt><samp class=”option”>end</samp></dt> <dd> What to do at the end of filtering (see below). </dd> <dt><samp class=”option”>hold</samp></dt> <dd> How many columns should pass through before start of filtering. </dd> <dt><samp class=”option”>pad</samp></dt> <dd> How many columns should be inserted before end of filtering. </dd> </dl>

Normally the filter shifts and tilts from the very first frame, and stops when the last one is received. However, before filtering starts, normal video may be preserved, so that the effect is slowly shifted in its place. Similarly, the last video frame may be reconstructed at the end. Alternatively it is possible to just start and end with black. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Filtering starts immediately and ends when the last frame is received. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> The first frames or the very last frame are kept intact during processing.

</dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Black is padded at the beginning or at the end of filtering. </dd> </dl>

 <h3 class="section">39.259 tinterlace# TOC</h3>

Perform various types of temporal field interlacing. Frames are counted starting from 1, so the first input frame is considered odd. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Specify the mode of the interlacing. This option can also be specified as a value alone. See below for a list of values for this option. Available values are: <dl class="table"> <dt>‘<samp class="samp">merge, 0</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at half frame rate. <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">drop_even, 1</samp>’</dt> <dd> Only output odd frames, even frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 11111 33333 11111 33333 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">drop_odd, 2</samp>’</dt> <dd> Only output even frames, odd frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 22222 44444 22222 44444 22222 44444 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">pad, 3</samp>’</dt> <dd> Expand each frame to full height, but pad alternate lines with black,

generating a frame with double height at the same input frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_top, 4</samp>’</dt> <dd> Interleave the upper field from odd frames with the lower field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<-

Output: 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_bottom, 5</samp>’</dt> <dd> Interleave the lower field from odd frames with the upper field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444

Output: 22222 44444 11111 33333 22222 44444 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">interlacex2, 6</samp>’</dt> <dd> Double frame rate with unchanged height. Frames are inserted each containing the second temporal field from the previous input frame and the first temporal field from the next input frame. This mode relies on the top_field_first flag. Useful for interlaced video displays with no field synchronisation.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 </pre>

</dd> <dt>‘<samp class="samp">mergex2, 7</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at same frame rate.

<pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 </pre>

</dd> </dl>

Numeric values are deprecated but are accepted for backward compatibility reasons. Default mode is <code class=”code”>merge</code>. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class=”var”>flags</var> is: <dl class=”table”> <dt><samp class=”option”>low_pass_filter, vlpf</samp></dt> <dd> Enable linear vertical low-pass filtering in the filter. Vertical low-pass filtering is required when creating an interlaced destination from a progressive source which contains high-frequency vertical detail. Filtering will reduce interlace ’twitter’ and Moire patterning. </dd> <dt><samp class=”option”>complex_filter, cvlpf</samp></dt> <dd> Enable complex vertical low-pass filtering. This will slightly less reduce interlace ’twitter’ and Moire patterning but better retain detail and subjective sharpness impression. </dd> <dt><samp class=”option”>bypass_il</samp></dt> <dd> Bypass already interlaced frames, only adjust the frame rate. </dd> </dl>

Vertical low-pass filtering and bypassing already interlaced frames can only be enabled for <samp class=”option”>mode</samp> <var class=”var”>interleave_top</var> and <var class=”var”>interleave_bottom</var>. </dd> </dl>

 <h3 class=”section”>39.260 tmedian# TOC</h3> Pick median pixels from several successive input video frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set radius of median filter. Default is 1. Allowed range is from 1 to 127. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>15</code>, by which all planes are processed. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Set median percentile. Default value is <code class=”code”>0.5</code>. Default value of <code class=”code”>0.5</code> will pick always median values, while <code class=”code”>0</code> will pick minimum values, and <code class=”code”>1</code> maximum values. </dd> </dl>

 <h4 class="subsection">39.260.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">radius</code>. <h3 class="section">39.261 tmidequalizer# TOC</h3>

Apply Temporal Midway Video Equalization effect. Midway Video Equalization adjusts a sequence of video frames to have the same histograms, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a video frames sequence. This filter accepts the following option: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set filtering radius. Default is <code class="code">5</code>. Allowed range is from 1 to 127. </dd> <dt><samp class="option">sigma</samp></dt> <dd> Set filtering sigma. Default is <code class="code">0.5</code>. This controls strength of filtering. Setting this option to 0 effectively does nothing. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> </dl>

 <h3 class="section">39.262 tmix# TOC</h3>

Mix successive video frames. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> The number of successive frames to mix. If unspecified, it defaults to 3. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input video frame. Each weight is separated by space. If number of weights is smaller than number of <var class="var">frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">39.262.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Average 7 successive frames: <pre class="example-preformatted">tmix=frames=7:weights="1 1 1 1 1 1 1" </pre>

Apply simple temporal convolution: <pre class="example-preformatted">tmix=frames=3:weights="-1 3 -1" </pre>

Similar as above but only showing temporal differences: <pre class="example-preformatted">tmix=frames=3:weights="-1 2 -1":scale=1 </pre>

 <h4 class="subsection">39.262.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">39.263 tonemap# TOC</h3> Tone map colors from different dynamic ranges. This filter expects data in single precision floating point, as it needs to operate on (and can output) out-of-range values. Another filter, such as zscale, is needed to convert the resulting frame to a usable format. The tonemapping algorithms implemented only work on linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,tonemap=clip,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h4 class="subsection">39.263.1 Options# TOC</h4> The filter accepts the following options. <dl class="table"> <dt><samp class="option">tonemap</samp></dt> <dd> Set the tone map algorithm to use. Possible values are: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Do not apply any tone map, only desaturate overbright pixels. </dd> <dt><var class="var">clip</var></dt> <dd> Hard-clip any out-of-range values. Use it for perfect color accuracy for in-range values, while distorting out-of-range values. </dd> <dt><var class="var">linear</var></dt>

<dd> Stretch the entire reference gamut to a linear multiple of the display. </dd> <dt><var class="var">gamma</var></dt> <dd> Fit a logarithmic transfer between the tone curves. </dd> <dt><var class="var">reinhard</var></dt> <dd> Preserve overall image brightness with a simple curve, using nonlinear contrast, which results in flattening details and degrading color accuracy. </dd> <dt><var class="var">hable</var></dt> <dd> Preserve both dark and bright details better than <var class="var">reinhard</var>, at the cost of slightly darkening everything. Use it when detail preservation is more important than color and brightness accuracy. </dd> <dt><var class="var">mobius</var></dt> <dd> Smoothly map out-of-range values, while retaining contrast and colors for in-range material as much as possible. Use it when color accuracy is more important than detail preservation. </dd> </dl>

Default is none. </dd> <dt><samp class="option">param</samp></dt> <dd> Tune the tone mapping algorithm. This affects the following algorithms: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Ignored. </dd> <dt><var class="var">linear</var></dt> <dd> Specifies the scale factor to use while stretching. Default to 1.0. </dd> <dt><var class="var">gamma</var></dt> <dd> Specifies the exponent of the function. Default to 1.8. </dd> <dt><var class="var">clip</var></dt> <dd> Specify an extra linear coefficient to multiply into the signal before clipping. Default to 1.0. </dd> <dt><var class="var">reinhard</var></dt> <dd> Specify the local contrast coefficient at the display peak. Default to 0.5, which means that in-gamut values will be about half as bright as when clipping. </dd> <dt><var class="var">hable</var></dt> <dd> Ignored. </dd> <dt><var class="var">mobius</var></dt> <dd> Specify the transition point from linear to mobius transform. Every value below this point is guaranteed to be mapped 1:1. The higher the value, the more accurate the result will be, at the cost of losing bright details. Default to 0.3, which due to the steep initial slope still preserves in-range colors fairly accurately. </dd> </dl>

</dd> <dt><samp class="option">desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by (smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default of 2.0 is somewhat conservative and will mostly just apply to skies or directly sunlit surfaces. A setting of 0.0 disables this option. This option works only if the input frame has a supported color tag. </dd> <dt><samp class="option">peak</samp></dt> <dd> Override signal/nominal/reference peak with this value. Useful when the embedded peak information in display metadata is not reliable or when tone mapping from a

lower range to a higher range. </dd> </dl>

 <h3 class="section">39.264 tpad#
TOC</h3>

Temporarily pad video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">start</samp></dt>
<dd> Specify number of delay frames before input video stream. Default is 0. </dd> <dt><samp class="option">stop</samp></dt>
<dd> Specify number of padding frames after input video stream. Set to -1 to pad indefinitely. Default is 0. </dd> <dt><samp class="option">start_mode</samp></dt> <dd> Set kind of frames added to beginning of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of first frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">stop_mode</samp></dt>
<dd> Set kind of frames added to end of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of last frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">start_duration, stop_duration</samp></dt> <dd> Specify the duration of the start/stop delay. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. These options override <var class="var">start</var> and <var class="var">stop</var>. Default is 0. </dd> <dt><samp class="option">color</samp></dt>
<dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.
The default value of <var class="var">color</var> is "black".
</dd> </dl>

 <h3 class="section">39.265 transpose# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. It accepts the following parameters: <dl class="table">
<dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">0, 4, cclock_flip</samp>’</dt>
<dd> Rotate by 90 degrees counterclockwise and vertically flip (default), that is: <pre class="example-preformatted">L.R L.l . . -> . . l.r R.r </pre>

</dd> <dt>‘<samp class="samp">1, 5, clock</samp>’</dt>
<dd> Rotate by 90 degrees clockwise, that is:
<pre class="example-preformatted">L.R l.L . . -> . . l.r r.R </pre>

</dd> <dt>‘<samp class="samp">2, 6, cclock</samp>’</dt>
<dd> Rotate by 90 degrees counterclockwise, that is: <div
class="example"> <pre class="example-preformatted">L.R R.r . . -> . .
l.r L.l </pre>

</dd> <dt>‘<samp class="samp">3, 7, clock_flip</samp>’</dt>
<dd> Rotate by 90 degrees clockwise and vertically flip, that is: <div
class="example"> <pre class="example-preformatted">L.R r.R . . -> . .
l.r l.L </pre> </dd> </dl>

For values between 4-7, the transposition is only done if the input video geometry is portrait and not landscape. These values are deprecated, the <code class="code">passthrough</code> option should be used instead. Numerical values are deprecated, and should be dropped in favor of symbolic constants. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">none</samp>’</dt> <dd> Always
apply transposition. </dd> <dt>‘<samp class="samp">portrait</samp>’</dt>
<dd> Preserve portrait geometry (when <var class="var">height</var>
>= <var class="var">width</var>). </dd> <dt>‘<samp
class="samp">landscape</samp>’</dt> <dd> Preserve
landscape geometry (when <var class="var">width</var> >= <var
class="var">height</var>). </dd> </dl>

Default value is <code class="code">none</code>. </dd> </dl>

For example to rotate by 90 degrees clockwise and preserve portrait layout:
 <pre class="example-preformatted">transpose=dir=1:passthrough=portrait
</pre>

The command above can also be specified as:
<pre class="example-preformatted">transpose=1:portrait </pre>

 <h3 class="section">39.266 trim<span class="pull-
right">#
TOC</h3>
Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table">
<dt><samp class="option">start</samp></dt> <dd> Specify the
time of the start of the kept section, i.e. the frame with the timestamp <var class="var">start</var> will be the first frame in the output. </dd> <dt><samp class="option">end</samp></dt>
<dd> Specify the time of the first frame that will be dropped,

i.e. the frame immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last frame in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> This is the same as <var class=”var”>start</var>, except this option sets the start timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> This is the same as <var class=”var”>end</var>, except this option sets the end timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> The number of the first frame that should be passed to the output. </dd> <dt><samp class=”option”>end_frame</samp></dt> <dd> The number of the first frame that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the frame timestamp, while the _frame variants simply count the frames that pass through the filter. Also note that this filter does not modify the timestamps. If you wish for the output timestamps to start at zero, insert a setpts filter after the trim filter. If multiple start or end options are set, this filter tries to be greedy and keep all the frames that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple trim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class=”itemize mark-bullet”> Drop everything except the second minute of input: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=60:120 </pre>

Keep only the first second: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=duration=1 </pre>

 <h3 class=”section”>39.267 unpremultiply# TOC</h3> Apply alpha unpremultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt>

<dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. If the format has 1 or 2 components, then luma is bit 0. If the format has 3 or 4 components: for RGB formats bit 0 is green, bit 1 is blue and bit 2 is red; for YUV formats bit 0 is luma, bit 1 is chroma-U and bit 2 is chroma-V. If present, the alpha channel is always the last bit. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">39.268 unsharp# TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_amount, la</samp></dt> <dd> Set the luma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 1.0. </dd> <dt><samp class="option">chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> <dt><samp class="option">alpha_msize_x, ax</samp></dt> <dd> Set the alpha matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_msize_y, ay</samp></dt> <dd> Set the alpha matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_amount, aa</samp></dt> <dd> Set the alpha effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class="subsection">39.268.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply strong luma sharpen effect: <pre class="example-preformatted">unsharp=luma_msize_x=7:luma_msize_y=7:luma_
</pre>

Apply a strong blur of both luma and chroma parameters: <pre class="example-preformatted">unsharp=7:7:-2:7:7:-2
</pre>

 <h3 class="section">39.269 untile# TOC</h3>

Decompose a video made of tiled images into the individual images. The frame rate of the output video is the frame rate of the input video multiplied by the number of tiles. This filter does the reverse of tile. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size (i.e. the number of lines and columns). For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">39.269.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce a 1-second video from a still image file made of 25 frames stacked vertically, like an analogic film reel: <pre class="example-preformatted">ffmpeg -r 1 -i image.jpg -vf untile=1x25 movie.mkv </pre>

 <h3 class="section">39.270 uspp# TOC</h3>

Apply ultra slow/simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">8</code> - all) shifts and average the results. The way this differs from the behavior of spp is that uspp actually encodes & decodes each case with libavcodec Snow, whereas spp uses a simplified intra only 8x8 DCT similar to MJPEG. This filter

is not available in ffmpeg versions between 5.0 and 6.0. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-8. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">8</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">codec</samp></dt> <dd> Use specified codec instead of snow. </dd> </dl>

 <h3 class="section">39.271 v360# TOC</h3>

Convert 360 videos between various formats. The filter accepts the following options: <dl class="table"> <dt><samp class="option">input</samp></dt> <dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. Available formats: <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">c3x2</samp>’</dt> <dt>‘<samp class="samp">c6x1</samp>’</dt> <dt>‘<samp class="samp">c1x6</samp>’</dt> <dd> Cubemap with 3x2/6x1/1x6 layout. Format specific options: <dl class="table"> <dt><samp class="option">in_pad</samp></dt> <dt><samp class="option">out_pad</samp></dt> <dd> Set padding proportion for the input/output cubemap. Values in decimals. Example values: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> No padding. </dd> <dt>‘<samp class="samp">0.01</samp>’</dt> <dd> 1% of face is padding. For example, with 1920x1280 resolution face size would be 640x640 and padding would be 3 pixels from each side. (640 * 0.01 = 6 pixels) </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. Maximum value is ‘<samp class="samp">0.1</samp>’. </dd> <dt><samp class="option">fin_pad</samp></dt> <dt><samp class="option">fout_pad</samp></dt> <dd> Set fixed padding for the input/output cubemap. Values in pixels. Default value is ‘<samp class="samp">0</samp>’. If greater than zero it overrides other padding options. </dd> <dt><samp class="option">in_forder</samp></dt> <dt><samp class="option">out_forder</samp></dt> <dd> Set order of faces

for the input/output cubemap. Choose one direction for each position.
Designation of directions: <dl class="table"> <dt>‘<samp
class="samp">r</samp>’</dt> <dd> right </dd>
<dt>‘<samp class="samp">l</samp>’</dt> <dd> left
</dd> <dt>‘<samp class="samp">u</samp>’</dt>
<dd> up </dd> <dt>‘<samp class="samp">d</samp>’</dt>
<dd> down </dd> <dt>‘<samp class="samp">f</samp>’</dt>
<dd> forward </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> back </dd> </dl>

Default value is ‘<samp class="samp">rludfb</samp>’.
 </dd> <dt><samp class="option">in_frot</samp></dt> <dt><samp
class="option">out_frot</samp></dt> <dd> Set rotation of faces for
the input/output cubemap. Choose one angle for each position.
Designation of angles: <dl class="table"> <dt>‘<samp
class="samp">0</samp>’</dt> <dd> 0 degrees clockwise
</dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> 90 degrees clockwise </dd> <dt>‘<samp class="samp">2</samp>’</dt>
<dd> 180 degrees clockwise </dd> <dt>‘<samp class="samp">3</samp>’</dt>
<dd> 270 degrees clockwise </dd> </dl>

Default value is ‘<samp class="samp">000000</samp>’.
</dd> </dl>

</dd> <dt>‘<samp class="samp">eac</samp>’</dt>
<dd> Equi-Angular Cubemap. </dd> <dt>‘<samp
class="samp">flat</samp>’</dt> <dt>‘<samp class="samp">gnomonic</samp>’<
<dt>‘<samp class="samp">rectilinear</samp>’</dt>
<dd> Regular video. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt>
<dd> Dual fisheye. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>

<dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>barrel</samp>’</dt> <dt>‘<samp class=”samp”>fb</samp>’</dt> <dt>‘<samp class=”samp”>barrelsplit</samp>’</dt> <dd> Facebook’s 360 formats. </dd> <dt>‘<samp class=”samp”>sg</samp>’</dt> <dd> Stereographic format. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>mercator</samp>’</dt> <dd> Mercator format. </dd> <dt>‘<samp class=”samp”>ball</samp>’</dt> <dd> Ball format, gives significant distortion toward the back. </dd> <dt>‘<samp class=”samp”>hammer</samp>’</dt> <dd> Hammer-Aitoff map projection format. </dd> <dt>‘<samp class=”samp”>sinusoidal</samp>’</dt> <dd> Sinusoidal map projection format. </dd> <dt>‘<samp class=”samp”>fisheye</samp>’</dt> <dd> Fisheye projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>pannini</samp>’</dt> <dd> Pannini projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dd> Set output pannini parameter. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dd> Set input pannini parameter. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>cylindrical</samp>’</dt> <dd> Cylindrical projection. Format specific options:

<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">perspective</samp>’</dt>
<dd> Perspective projection. (output only)
Format specific options: <dl class="table"> <dt><samp class="option">v_fov</samp></dt> <dd> Set perspective parameter. </dd> </dl>

</dd> <dt>‘<samp class="samp">tetrahedron</samp>’</dt>
<dd> Tetrahedron projection. </dd> <dt>‘<samp class="samp">tsp</samp>’</dt> <dd> Truncated square pyramid projection. </dd> <dt>‘<samp class="samp">he</samp>’</dt>
<dt>‘<samp class="samp">hequirect</samp>’</dt> <dd> Half equirectangular projection. </dd> <dt>‘<samp class="samp">equisolid</samp>’</dt
<dd> Equisolid format. Format specific options: <dl class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">og</samp>’</dt>
<dd> Orthographic format. Format specific options: <dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">octahedron</samp>’</dt>
<dd> Octahedron projection. </dd> <dt>‘<samp
class="samp">cylindricalea</samp>’</dt> <dd> Cylindrical
Equal Area projection. </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Set
interpolation method.
 Note: more complex interpolation
methods require much more memory to run. Available methods:
 <dl class="table"> <dt>‘<samp class="samp">near</samp>’</dt>
<dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Nearest
neighbour. </dd> <dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">linear</samp>’</dt> <dd> Bilinear
interpolation. </dd> <dt>‘<samp class="samp">lagrange9</samp>’</dt>
<dd> Lagrange9 interpolation. </dd> <dt>‘<samp
class="samp">cube</samp>’</dt> <dt>‘<samp class="samp">cubic</samp>’</dt
<dd> Bicubic interpolation. </dd> <dt>‘<samp class="samp">lanc</samp>’</dt
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Lanczos
interpolation. </dd> <dt>‘<samp class="samp">sp16</samp>’</dt>
<dt>‘<samp class="samp">spline16</samp>’</dt> <dd> Spline16
interpolation. </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">gaussian</samp>’</dt> <dd> Gaussian
interpolation. </dd> <dt>‘<samp class="samp">mitchell</samp>’</dt>
<dd> Mitchell interpolation. </dd> </dl>

Default value is ‘<samp class="samp">line</samp>’.
 </dd> <dt><samp class="option">w</samp></dt> <dt><samp
class="option">h</samp></dt> <dd> Set the output video res-
olution. Default resolution depends on formats.
</dd> <dt><samp class="option">in_stereo</samp></dt> <dt><samp
class="option">out_stereo</samp></dt> <dd> Set the input/output
stereo format. <dl class="table"> <dt>‘<samp class="samp">2d</samp>’</dt>
<dd> 2D mono </dd> <dt>‘<samp class="samp">sbs</samp>’</dt>
<dd> Side by side </dd> <dt>‘<samp class="samp">tb</samp>’</dt>
<dd> Top bottom </dd> </dl>

Default value is ‘<samp class="samp">2d</samp>’
for input and output format. </dd> <dt><samp class="option">yaw</samp></dt>
<dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt>
<dd> Set rotation for the output video. Values in degrees.
</dd> <dt><samp class="option">rorder</samp></dt> <dd> Set
rotation order for the output video. Choose one item for each posi-
tion. <dl class="table"> <dt>‘<samp class="samp">y,
Y</samp>’</dt> <dd> yaw </dd> <dt>‘<samp
class="samp">p, P</samp>’</dt> <dd> pitch </dd>
<dt>‘<samp class="samp">r, R</samp>’</dt> <dd> roll
</dd> </dl>

Default value is ‘<samp class="samp">ypr</samp>’.

 </dd> <dt><samp class="option">h_flip</samp></dt> <dt><samp class="option">v_flip</samp></dt> <dt><samp class="option">d_flip</samp></dt> <dd> Flip the output video horizontally(swaps left-right)/vertically(swaps up-down)/in-depth(swaps back-forward). Boolean values. </dd> <dt><samp class="option">ih_flip</samp></dt> <dt><samp class="option">iv_flip</samp></dt> <dd> Set if input video is flipped horizontally/vertically. Boolean values. </dd> <dt><samp class="option">in_trans</samp></dt> <dd> Set if input video is transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">out_trans</samp></dt> <dd> Set if output video needs to be transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">h_offset</samp></dt> <dt><samp class="option">v_offset</samp></dt> <dd> Set output horizontal/vertical off-axis offset. Default is set to 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">alpha_mask</samp></dt> <dd> Build mask in alpha plane for all unmapped pixels by marking them fully transparent. Boolean value, by default disabled. </dd> <dt><samp class="option">reset_rot</samp></dt> <dd> Reset rotation of output video. Boolean value, by default disabled. </dd> </dl>

 <h4 class="subsection">39.271.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert equirectangular video to cubemap with 3x2 layout and 1% padding using bicubic interpolation: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=e:c3x2:cubic:out_pad=0.01 output.mkv </pre> Extract back view of Equi-Angular Cubemap: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=eac:flat:yaw=180 output.mkv </pre> Convert transposed and horizontally flipped Equi-Angular Cubemap in side-by-side stereo format to equirectangular top-bottom stereo format: <pre class="example-preformatted">v360=eac:equirect:in_stereo=sbs:in_trans=1:ih_flip=1:out_stereo=tb </pre>

 <h4 class="subsection">39.271.2 Commands# TOC</h4>

This filter supports subset of above options as commands. <h3 class="section">39.272 vaguedenoiser# TOC</h3>

Apply a wavelet based denoiser. It transforms each frame from the video input into the wavelet domain, using Cohen-Daubechies-Feauveau 9/7. Then it applies some filtering to the obtained coefficients. It does an inverse wavelet transform after. Due to wavelet properties, it should give a nice smoothed result, and reduced noise, without blurring picture features. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> The filtering strength. The higher, the more filtered the video will be. Hard thresholding can use a higher threshold than soft thresholding before the video looks overfiltered. Default value is 2. </dd> <dt><samp class="option">method</samp></dt> <dd> The filtering method the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> All values under the threshold will be zeroed. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> All values under the threshold will be zeroed. All values above will be reduced by the threshold. </dd> <dt>‘<samp class="samp">garrote</samp>’</dt> <dd> Scales or nullifies coefficients - intermediary between (more) soft and (less) hard thresholding. </dd> </dl>

Default is garrote. </dd> <dt><samp class="option">nsteps</samp></dt> <dd> Number of times, the wavelet will decompose the picture. Picture can’t be decomposed beyond a particular point (typically, 8 for a 640x480 frame - as 2^9 = 512 > 480). Valid values are integers between 1 and 32. Default value is 6. </dd> <dt><samp class="option">percent</samp></dt> <dd> Partial of full denoising (limited coefficients shrinking), from 0 to 100. Default value is 85. </dd> <dt><samp class="option">planes</samp></dt> <dd> A list of the planes to process. By default all planes are processed. </dd> <dt><samp class="option">type</samp></dt> <dd> The threshold type the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">universal</samp>’</dt> <dd> Threshold used is same for all decompositions. </dd> <dt>‘<samp class="samp">bayes</samp>’</dt> <dd> Threshold used depends also on each decomposition coefficients. </dd> </dl>

Default is universal. </dd> </dl>

 <h3 class="section">39.273 varblur# TOC</h3> Apply variable blur filter by using 2nd video stream to set blur radius. The 2nd stream must have the same dimensions. This filter accepts the following options: <dl class="table"> <dt><samp class="option">min_r</samp></dt>

<dd> Set min allowed radius. Allowed range is from 0 to 254. Default is 0. </dd> <dt><samp class="option">max_r</samp></dt> <dd> Set max allowed radius. Allowed range is from 1 to 255. Default is 8. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. By default, all are used. </dd> </dl>

The <code class="code">varblur</code> filter also supports the framesync options. <h4 class="subsection">39.273.1 Commands# TOC</h4>

This filter supports all the above options as commands. <h3 class="section">39.274 vectorscope# TOC</h3>

Display 2 color component values in the two dimensional graph (which is called a vectorscope). This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set vectorscope mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">gray</samp>’</dt> <dt>‘<samp class="samp">tint</samp>’</dt> <dd> Gray values are displayed on graph, higher brightness means more pixels have same component color value on location in graph. This is the default mode. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Gray values are displayed on graph. Surrounding pixels values which are not present in video frame are drawn in gradient of 2 color components which are set by option <code class="code">x</code> and <code class="code">y</code>. The 3rd color component is static. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Actual color components values present in video frame are displayed on graph. </dd> <dt>‘<samp class="samp">color3</samp>’</dt> <dd> Similar as color2 but higher frequency of same values <code class="code">x</code> and <code class="code">y</code> on graph increases value of another color component, which is luminance by default values of <code class="code">x</code> and <code class="code">y</code>. </dd> <dt>‘<samp class="samp">color4</samp>’</dt> <dd> Actual colors present in video frame are displayed on graph. If two different colors map to same position on graph then color with higher value of component not present in graph is picked. </dd> <dt>‘<samp class="samp">color5</samp>’</dt> <dd> Gray values are displayed on graph. Similar to <code class="code">color</code> but with 3rd color component picked from radial gradient. </dd> </dl>

</dd> <dt><samp class="option">x</samp></dt> <dd> Set which color component will be represented on X-axis. Default is <code class="code">1</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set which color component will be represented on Y-axis. Default is <code class="code">2</code>. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity, used by modes: gray, color, color3 and color5 for increasing brightness of color component which represents frequency of (X, Y) location in graph. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, even darkest single pixel will be clearly highlighted. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold maximum and minimum values presented in graph over time. This way you can still spot out of range values without constantly looking at vectorscope. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set what kind of graticule to draw. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">green</samp>’</dt <dt>‘<samp class="samp">color</samp>’</dt> <dt>‘<samp class="samp">invert</samp>’</dt> </dl>

</dd> <dt><samp class="option">opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class="option">flags, f</samp></dt> <dd> Set graticule flags. <dl class="table"> <dt>‘<samp class="samp">white</samp>’</dt> <dd> Draw graticule for white point. </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Draw graticule for black point. </dd> <dt>‘<samp class="samp">name</samp>’</dt> <dd> Draw color points short names. </dd> </dl>

</dd> <dt><samp class="option">bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">lthreshold, l</samp></dt> <dd> Set low threshold for color component not represented on X or Y axis. Values lower than this value will be ignored. Default is 0. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and low threshold value of 0.1 actual threshold is 0.1 * 255 = 25. </dd> <dt><samp class="option">hthreshold, h</samp></dt> <dd> Set high threshold for color component not represented on X or Y axis. Values higher than this value will be ignored. Default is 1. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and high threshold value of 0.9 actual threshold is 0.9 * 255 = 230. </dd>

<dt><samp class="option">colorspace, c</samp></dt> <dd> Set what kind of colorspace to use when drawing graticule. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">601</samp>’</dt> <dt>‘<samp class="samp">709</samp>’</dt> </dl> Default is auto. </dd> <dt><samp class="option">tint0, t0</samp></dt> <dt><samp class="option">tint1, t1</samp></dt> <dd> Set color tint for gray/tint vectorscope mode. By default both options are zero. This means no tint, and output will remain gray. </dd> </dl>

 <h3 class="section">39.275 vidstabdetect# TOC</h3>

Analyze video stabilization/deshaking. Perform pass 1 of 2, see vidstabtransform for pass 2. This filter generates a file with relative translation and rotation transform information about subsequent frames, which is then used by the vidstabtransform filter. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">result</samp></dt> <dd> Set the path to the file used to write the transforms information. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">shakiness</samp></dt> <dd> Set how shaky the video is and how quick the camera is. It accepts an integer in the range 1-10, a value of 1 means little shakiness, a value of 10 means strong shakiness. Default value is 5. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set the accuracy of the detection process. It must be a value in the range 1-15. A value of 1 means low accuracy, a value of 15 means high accuracy. Default value is 15. </dd> <dt><samp class="option">stepsize</samp></dt> <dd> Set stepsize of the search process. The region around minimum is scanned with 1 pixel resolution. Default value is 6. </dd> <dt><samp class="option">mincontrast</samp></dt> <dd> Set minimum contrast. Below this value a local measurement field is discarded. Must be a floating point value in the range 0-1. Default value is 0.3. </dd> <dt><samp class="option">tripod</samp></dt> <dd> Set reference frame number for tripod mode. If enabled, the motion of the frames is compared to a reference frame in the filtered stream, identified by the specified number. The idea is to compensate all movements in a more-or-less static scene and keep the camera view absolutely still. If set to 0, it is disabled. The frames are counted starting from 1. </dd> <dt><samp class="option">show</samp></dt> <dd> Show fields and transforms in the resulting frames. It accepts an integer in the range 0-2.

Default value is 0, which disables any visualization. </dd> <dt><samp class=”option”>fileformat</samp></dt> <dd> Format for the transforms data file to be written. Acceptable values are <dl class=”table”> <dt>‘<samp class=”samp”>ascii</samp>’</dt> <dd> Human-readable plain text </dd> <dt>‘<samp class=”samp”>binary</samp>’</dt> <dd> Binary format, roughly 40% smaller than <code class=”code”>ascii</code>. (default) </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>39.275.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use default values: <pre class=”example-preformatted”>vidstabdetect </pre>

Analyze strongly shaky movie and put the results in file <samp class=”file”>mytransforms.trf</samp>: <pre class=”example-preformatted”>vidstabdetect=shakiness=10:accuracy=15:result="mytransforms.trf&qu
</pre>

Visualize the result of internal transformations in the resulting video: <pre class=”example-preformatted”>vidstabdetect=show=1 </pre>

Analyze a video with medium shakiness using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i input -vf vidstabdetect=shakiness=5:show=1 dummy.avi </pre>

 <h3 class=”section”>39.276 vidstabtransform# TOC</h3>

Video stabilization/deshaking: pass 2 of 2, see vidstabdetect for pass 1. Read a file with transform information for each frame and apply/compensate them. Together with the vidstabdetect filter this can be used to deshake videos. See also http://public.hronopik.de/vid.stab. It is important to also use the unsharp filter, see below. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libvidstab</code>. <h4 class=”subsection”>39.276.1 Options<a class=”anchor hidden-xs” href=”#Options-

131" aria-hidden="true"># TOC</h4>

<dl class="table"> <dt><samp class="option">input</samp></dt> <dd> Set path to the file used to read the transforms. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set the number of frames (value*2 + 1) used for lowpass filtering the camera movements. Default value is 10. For example a number of 10 means that 21 frames are used (10 in the past and 10 in the future) to smoothen the motion in the video. A larger value leads to a smoother video, but limits the acceleration of the camera (pan/tilt movements). 0 is a special case where a static camera is simulated. </dd> <dt><samp class="option">optalgo</samp></dt> <dd> Set the camera path optimization algorithm. Accepted values are: <dl class="table"> <dt>‘<samp class="samp">gauss</samp>’</dt> <dd> gaussian kernel low-pass filter on camera motion (default) </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> averaging on transformations </dd> </dl>

</dd> <dt><samp class="option">maxshift</samp></dt> <dd> Set maximal number of pixels to translate frames. Default value is -1, meaning no limit. </dd> <dt><samp class="option">maxangle</samp></dt> <dd> Set maximal angle in radians (degree*PI/180) to rotate frames. Default value is -1, meaning no limit. </dd> <dt><samp class="option">crop</samp></dt> <dd> Specify how to deal with borders that may be visible due to movement compensation. Available values are: <dl class="table"> <dt>‘<samp class="samp">keep</samp>’</dt> <dd> keep image information from previous frame (default) </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> fill the border black </dd> </dl>

</dd> <dt><samp class="option">invert</samp></dt> <dd> Invert transforms if set to 1. Default value is 0. </dd> <dt><samp class="option">relative</samp></dt> <dd> Consider transforms as relative to previous frame if set to 1, absolute if set to 0. Default value is 0. </dd> <dt><samp class="option">zoom</samp></dt> <dd> Set percentage to zoom. A positive value will result in a zoom-in effect, a negative value in a zoom-out effect. Default value is 0 (no zoom). </dd> <dt><samp class="option">optzoom</samp></dt> <dd> Set optimal zooming to avoid borders. Accepted values are: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> disabled </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> optimal static zoom value is determined (only very strong movements will lead to visible borders) (default) </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> optimal adaptive zoom

value is determined (no borders will be visible), see <samp class="option">zoomspeed</samp>
</dd> </dl>

Note that the value given at zoom is added to the one calculated here. </dd> <dt><samp class="option">zoomspeed</samp></dt>
<dd> Set percent to zoom maximally each frame (enabled when <samp class="option">optzoom</samp> is set to 2). Range is from 0 to 5, default value is 0.25. </dd> <dt><samp class="option">interpol</samp></dt>
<dd> Specify type of interpolation. Available values are:
<dl class="table"> <dt>‘<samp class="samp">no</samp>’</dt>
<dd> no interpolation </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> linear only horizontal </dd> <dt>‘<samp class="samp">bilinear</samp>’</
<dd> linear in both directions (default) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> cubic in both directions (slow) </dd> </dl>

</dd> <dt><samp class="option">tripod</samp></dt> <dd> Enable virtual tripod mode if set to 1, which is equivalent to <code class="code">relative=0:smoothing=0</code>. Default value is 0. Use also <code class="code">tripod</code> option of vidstabdetect.
</dd> <dt><samp class="option">debug</samp></dt> <dd> Increase log verbosity if set to 1. Also the detected global motions are written to the temporary file <samp class="file">global_motions.trf</samp>. Default value is 0. </dd> </dl>

 <h4 class="subsection">39.276.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> for a typical stabilization with default values:
<pre class="example-preformatted">ffmpeg -i inp.mpeg -vf vidstabtransform,unsharp=5:5:0.8:3:3:0.4 inp_stabilized.mpeg </pre>

Note the use of the unsharp filter which is always recommended. Zoom in a bit more and load transform data from a given file: <pre class="example-preformatted">vidstabtransform=zoom=5:input="mytransforms.trf"
</pre>

Smoothen the video even more: <pre class="example-preformatted">vidstabtransform=smoothing=30 </pre>

 <h3 class="section">39.277 vflip#
TOC</h3>

Flip the input video vertically. For example, to vertically

flip a video with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i in.avi -vf "vflip" out.avi </pre>

 <h3 class=”section”>39.278 vfrdet# TOC</h3>

Detect variable frame rate video. This filter tries to detect if the input is variable or constant frame rate. At end it will output number of frames detected as having variable delta pts, and ones with constant delta pts. If there was frames with variable delta, than it will also show min, max and average delta encountered. <h3 class=”section”>39.279 vibrance# TOC</h3>

Boost or alter saturation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>intensity</samp></dt> <dd> Set strength of boost if positive value or strength of alter if negative value. Default is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>rbal</samp></dt> <dd> Set the red balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>gbal</samp></dt> <dd> Set the green balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>bbal</samp></dt> <dd> Set the blue balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>rlum</samp></dt> <dd> Set the red luma coefficient. </dd> <dt><samp class=”option”>glum</samp></dt> <dd> Set the green luma coefficient. </dd> <dt><samp class=”option”>blum</samp></dt> <dd> Set the blue luma coefficient. </dd> <dt><samp class=”option”>alternate</samp></dt> <dd> If <code class=”code”>intensity</code> is negative and this is set to 1, colors will change, otherwise colors will be less saturated, more towards gray. </dd> </dl>

 <h4 class=”subsection”>39.279.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>39.280 vif# TOC</h3>

Obtain the average VIF (Visual Information Fidelity) between two input

videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average VIF score is printed through the logging system. The filter stores the calculated VIF score of each frame. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi vif -f null - </pre>

 <h3 class=”section”>39.281 vignette# TOC</h3>

Make or reverse a natural vignetting effect. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>angle, a</samp></dt> <dd> Set lens angle expression as a number of radians. The value is clipped in the <code class=”code”>[0,PI/2]</code> range. Default value: <code class=”code”>"PI/5"</code> </dd> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dd> Set center coordinates expressions. Respectively <code class=”code”>"w/2"</code> and <code class=”code”>"h/2"</code> by default. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set forward/backward mode. Available modes are: <dl class=”table”> <dt>‘<samp class=”samp”>forward</samp>’</dt> <dd> The larger the distance from the central point, the darker the image becomes. </dd> <dt>‘<samp class=”samp”>backward</samp>’</dt> <dd> The larger the distance from the central point, the brighter the image becomes. This can be used to reverse a vignette effect, though there is no automatic detection to extract the lens <samp class=”option”>angle</samp> and other settings (yet). It can also be used to create a burning effect. </dd> </dl>

Default value is ‘<samp class=”samp”>forward</samp>’. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set evaluation mode for the expressions (<samp class=”option”>angle</samp>, <samp class=”option”>x0</samp>, <samp class=”option”>y0</samp>). It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> Evaluate expressions only once during the filter initialization. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. This is way slower than the ‘<samp class=”samp”>init</samp>’ mode since it requires all the scalers to

be re-computed, but it allows advanced dynamic expressions. </dd>
</dl>

Default value is ‘<samp class="samp">init</samp>’.
 </dd> <dt><samp class="option">dither</samp></dt> <dd> Set
dithering to reduce the circular banding effects. Default is <code
class="code">1</code> (enabled). </dd> <dt><samp class="option">aspect</samp></dt>
<dd> Set vignette aspect. This setting allows one to adjust the shape of
the vignette. Setting this value to the SAR of the input will make a rectangular
vignetting following the dimensions of the video. Default is <code
class="code">1/1</code>. </dd> </dl>

 <h4 class="subsection">39.281.1 Expressions<a class="anchor hidden-xs" href="#Expressions"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Expressions" aria-hidden="true">TOC</h4>

The <samp class="option">alpha</samp>, <samp class="option">x0</samp>
and <samp class="option">y0</samp> expressions can contain the following
parameters. <dl class="table"> <dt><samp class="option">w</samp></dt>
<dt><samp class="option">h</samp></dt> <dd> input width
and height </dd> <dt><samp class="option">n</samp></dt>
<dd> the number of input frame, starting from 0 </dd>
<dt><samp class="option">pts</samp></dt> <dd> the PTS (Presentation TimeStamp) time of the filtered video frame, expressed in <var
class="var">TB</var> units, NAN if undefined </dd> <dt><samp
class="option">r</samp></dt> <dd> frame rate of the input video,
NAN if the input frame rate is unknown </dd> <dt><samp
class="option">t</samp></dt> <dd> the PTS (Presentation TimeStamp) of the filtered video frame, expressed in seconds, NAN if undefined
</dd> <dt><samp class="option">tb</samp></dt> <dd> time base
of the input video </dd> </dl>

 <h4 class="subsection">39.281.2 Examples<a class="anchor hidden-xs" href="#Examples-
187" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-187" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Apply simple strong vignetting effect:
 <pre class="example-preformatted">vignette=PI/4
</pre>

Make a flickering vignetting: <pre
class="example-preformatted">vignette='PI/4+random(1)*PI/50':eval=frame
</pre>

 <h3 class="section">39.282 vmafmotion<a class="anchor hidden-xs" href="#vmafmotion"

aria-hidden=”true”># TOC</h3>

Obtain the average VMAF motion score of a video. It is one of the component metrics of VMAF. The obtained average motion score is printed through the logging system. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stats_file</samp></dt> <dd> If specified, the filter will use the named file to save the motion score of each frame with respect to the previous frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

Example: <pre class=”example-preformatted”>ffmpeg -i ref.mpg -vf vmafmotion -f null - </pre>

 <h3 class=”section”>39.283 vpp_amf# TOC</h3>

Scale (resize) and convert colorspace, transfer characteristics or color primaries for the input video, using AMD Advanced Media Framework library for hardware acceleration. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>scale_type</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear This is the default. </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used.

</dd> <dt><samp class=”option”>force_original_aspect_ratio</samp></dt> <dt><samp class=”option”>force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class=”option”>reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class=”option”>in_color_range</samp></dt> <dd> Override input color range. </dd> <dt><samp class=”option”>out_color_range</samp><dd> Specify output color range. The accepted values for in_trc and out_trc are: <dl class=”table”> <dt>‘<samp class=”samp”>studio</samp>’</dt> <dd> Studio (or re-

stricted, or MPEG) color range. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Full (or JPEG) color range. </dd> </dl>

</dd> <dt><samp class="option">color_profile</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">in_trc</samp></dt> <dd> Override input transfer characteristics. </dd> <dt><samp class="option">out_trc</samp></dt> <dd> Specify output transfer characteristics. The accepted values for in_trc and out_trc are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dd> Constant gamma of 2.2 </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dd> Constant gamma of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> LOG </dd> <dt>‘<samp class="samp">log-sqrt</samp>’</dt> <dd> LOG_SQRT </dd> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4 </dd> <dt>‘<samp class="samp">bt1361-ecg</samp>’</dt> <dd> BT1361_ECG </dd> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for 10-bits content </dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dd> BT.2020 for 12-bits content </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dd> SMPTE2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dd> SMPTE428 </dd> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> <dd> ARIB_STD_B67 </dd> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dd> Override input color primaries. </dd> <dt><samp class="option">out_primaries</samp></dt> <dd> Specify output color primaries. The accepted values for in_primaries and out_primaries are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp

class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>film</samp>’</dt> <dd> film </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dd> SMPTE-428 </dd> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class=”samp”>jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>39.283.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class=”example-preformatted”>vpp_amf=-2:720:format=yuv420p,hwdownload,format=yuv420p </pre>

Upscale to 4K and change color profile to bt2020. <pre class=”example-preformatted”>vpp_amf=4096:2160:color_profile=bt2020 </pre>

Override input primaries and input transfer characteristics, change both to bt709.

 <pre class=”example-preformatted”>vpp_amf=color_profile=bt2020:in_trc=smpte2 </pre>

 <h3 class=”section”>39.284 vstack# TOC</h3> Stack input videos vertically. All streams must be of same pixel format and of same width. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class=”section”>39.285 w3fdif<a class=”anchor hidden-xs” href=”#w3fdif” aria-

hidden=”true”># TOC</h3>

Deinterlace the input video ("w3fdif" stands for "Weston 3 Field Deinterlacing Filter"). Based on the process described by Martin Weston for BBC R&D, and implemented based on the de-interlace algorithm written by Jim Easterbrook for BBC R&D, the Weston 3 field deinterlacing filter uses filter coefficients calculated by BBC R&D. This filter uses field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class=”code”>w3fdif</code> filter. There are two sets of filter coefficients, so called "simple" and "complex". Which set of filter coefficients is used can be set by passing an optional parameter: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set the interlacing filter coefficients. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Simple filter coefficient set. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> More-complex filter coefficient set. </dd> </dl> Default value is ‘<samp class=”samp”>complex</samp>’. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames, </dd> <dt>‘<samp class=”samp”>interlaced</samp>&rsquo <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

Default value is ‘<samp class=”samp”>all</samp>’. </dd> </dl>

 <h4 class="subsection">39.285.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">39.286 waveform# TOC</h3> Video waveform monitor. The waveform monitor plots color component intensity. By default luma only. Each column of the waveform corresponds to a column of pixels in the source video. It accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Can be either <code class="code">row</code>, or <code class="code">column</code>. Default is <code class="code">column</code>. In row mode, the graph on the left side represents color component value 0 and the right side represents value = 255. In column mode, the top side represents color component value = 0 and bottom side represents value = 255. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity. Smaller values are useful to find out how many values of the same luminance are distributed across input rows/columns. Default value is <code class="code">0.04</code>. Allowed range is [0, 1]. </dd> <dt><samp class="option">mirror, r</samp></dt> <dd> Set mirroring mode. <code class="code">0</code> means unmirrored, <code class="code">1</code> means mirrored. In mirrored mode, higher values will be represented on the left side for <code class="code">row</code> mode and at the top for <code class="code">column</code> mode. Default is <code class="code">1</code> (mirrored). </dd> <dt><samp class="option">display, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. This display mode makes it easier to spot relative differences or similarities in overlapping areas of the color components that are supposed to be identical, such as neutral whites, grays, or blacks. </dd> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">row</code> mode or one below the other in <code class="code">column</code> mode. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">column</code> mode or one below the other in <code class="code">row</code> mode. Using this display mode makes it easy to spot color casts in the highlights and shadows of an image, by comparing the contours of the top and the bottom

graphs of each waveform. Since whites, grays, and blacks are characterized by exactly equal amounts of red, green, and blue, neutral areas of the picture should display three waveforms of roughly equal width/height. If not, the correction is easy to perform by making level adjustments the three waveforms. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">components, c</samp></dt> <dd> Set which color components to display. Default is 1, which means only luma or red color component if input is in RGB colorspace. If is set for example to 7 it will display all 3 (if) available color components. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, minimum and maximum values presented in graph will be easily visible even with small <code class="code">step</code> value. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold minimum and maximum values presented in graph across time. This way you can still spot out of range values without constantly looking at waveforms. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">filter, f</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">lowpass</samp>’</dt> <dd> No filtering, this is default. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Luma and chroma combined together. </dd> <dt>‘<samp class="samp">aflat</samp>’</dt> <dd> Similar as above, but shows difference between blue and red chroma. </dd> <dt>‘<samp class="samp">xflat</samp>’</dt> <dd> Similar as above, but use different colors. </dd> <dt>‘<samp class="samp">yflat</samp>’</dt> <dd> Similar as above, but again with different colors. </dd> <dt>‘<samp class="samp">chroma</samp>’</dt> <dd> Displays only chroma. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Displays actual color value on waveform. </dd> <dt>‘<samp class="samp">acolor</samp>’</dt> <dd> Similar as above, but with luma showing frequency of chroma values. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set which graticule to display. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do not display graticule. </dd> <dt>‘<samp class="samp">green</samp>’</dt> <dd> Display green graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">orange</samp>’</dt> <dd> Display orange graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">invert</samp>’</dt> <dd> Display invert graticule showing legal broadcast ranges. </dd>

</dl>

</dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class=”option”>flags, fl</samp></dt> <dd> Set graticule flags. <dl class=”table”> <dt>‘<samp class=”samp”>numbers</samp>’</dt> <dd> Draw numbers above lines. By default enabled. </dd> <dt>‘<samp class=”samp”>dots</samp>’</dt> <dd> Draw dots instead of lines. </dd> </dl>

</dd> <dt><samp class=”option”>scale, s</samp></dt> <dd> Set scale used for displaying graticule. <dl class=”table”> <dt>‘<samp class=”samp”>digital</samp>’</dt> <dt>‘<samp class=”samp”>millivolts</samp>&rsquo <dt>‘<samp class=”samp”>ire</samp>’</dt> </dl> Default is digital. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class=”option”>tint0, t0</samp></dt> <dt><samp class=”option”>tint1, t1</samp></dt> <dd> Set tint for output. Only used with lowpass filter and when display is not overlay and input pixel formats are not RGB. </dd> <dt><samp class=”option”>fitmode, fm</samp></dt> <dd> Set sample aspect ratio of video output frames. Can be used to configure waveform so it is not stretched too much in one of directions. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Set sample aspect ration to 1/1. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Set sample aspect ratio to match input size of video </dd> </dl> Default is ‘<samp class=”samp”>none</samp>’. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set input formats for filter to pick from. Can be ‘<samp class=”samp”>all</samp>’, for selecting from all available formats, or ‘<samp class=”samp”>first</samp>’, for selecting first available format. Default is ‘<samp class=”samp”>first</samp>’. </dd> </dl>

 <h3 class=”section”>39.287 weave, doubleweave# TOC</h3>

The <code class=”code”>weave</code> takes a field-based video input and join each two sequential fields into single frame, producing a new double height clip with half the frame rate and half the frame count. The <code class=”code”>doubleweave</code> works same as <code class=”code”>weave</code> but without halving frame rate and frame count. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd> Set first field. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> Set the frame as

top-field-first. </dd> <dt>‘<samp class="samp">bottom, b</samp>’</dt> <dd> Set the frame as bottom-field-first. </dd> </dl> </dd> </dl>

 <h4 class="subsection">39.287.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interlace video using select and separatefields filter: <pre class="example-preformatted">separatefields,select=eq(mod(n,4),0)+eq(
</pre>

 <h3 class="section">39.288 xbr# TOC</h3> Apply the xBR high-quality magnification filter which is designed for pixel art. It follows a set of edge-detection rules, see https://forums.libretro.com/t/xbr-algorithm-tutorial/123. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xBR</code>, <code class="code">3</code> for <code class="code">3xBR</code> and <code class="code">4</code> for <code class="code">4xBR</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">39.289 xcorrelate# TOC</h3> Apply normalized cross-correlation between first and second input video stream. Second input video stream dimensions must be lower than first input video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">secondary</samp></dt> <dd> Set which secondary video frames will be processed from second input video stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">xcorrelate</code> filter also supports the framesync options. <h3 class="section">39.290 xfade# TOC</h3>

Apply cross fade from one input video stream to another input video stream.

The cross fade is applied for specified duration.

Both inputs must be constant frame-rate and have the same resolution, pixel format,

frame rate and timebase.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">transition</samp></dt>

<dd> Set one of available transition effects:

<dl class="table">

<dt>‘<samp class="samp">custom</samp>’</dt>

<dt>‘<samp class="samp">fade</samp>’</dt>

<dt>‘<samp class="samp">wipeleft</samp>’</dt>

<dt>‘<samp class="samp">wiperight</samp>’</dt>

<dt>‘<samp class="samp">wipeup</samp>’</dt>

<dt>‘<samp class="samp">wipedown</samp>’</dt>

<dt>‘<samp class="samp">slideleft</samp>’</dt>

<dt>‘<samp class="samp">slideright</samp>’</dt>

<dt>‘<samp class="samp">slideup</samp>’</dt>

<dt>‘<samp class="samp">slidedown</samp>’</dt>

<dt>‘<samp class="samp">circlecrop</samp>’</dt>

<dt>‘<samp class="samp">rectcrop</samp>’</dt>

<dt>‘<samp class="samp">distance</samp>’</dt>

<dt>‘<samp class="samp">fadeblack</samp>’</dt>

<dt>‘<samp class="samp">fadewhite</samp>’</dt>

<dt>‘<samp class="samp">radial</samp>’</dt>

<dt>‘<samp class="samp">smoothleft</samp>’</dt>

<dt>‘<samp class="samp">smoothright</samp>’</dt>

<dt>‘<samp class="samp">smoothup</samp>’</dt>
<dt>‘<samp class="samp">smoothdown</samp>’</dt>
<dt>‘<samp class="samp">circleopen</samp>’</dt>
<dt>‘<samp class="samp">circleclose</samp>’</dt>
<dt>‘<samp class="samp">vertopen</samp>’</dt>
<dt>‘<samp class="samp">vertclose</samp>’</dt>
<dt>‘<samp class="samp">horzopen</samp>’</dt>
<dt>‘<samp class="samp">horzclose</samp>’</dt>
<dt>‘<samp class="samp">dissolve</samp>’</dt>
<dt>‘<samp class="samp">pixelize</samp>’</dt>
<dt>‘<samp class="samp">diagtl</samp>’</dt>
<dt>‘<samp class="samp">diagtr</samp>’</dt>
<dt>‘<samp class="samp">diagbl</samp>’</dt>
<dt>‘<samp class="samp">diagbr</samp>’</dt>
<dt>‘<samp class="samp">hlslice</samp>’</dt>
<dt>‘<samp class="samp">hrslice</samp>’</dt>
<dt>‘<samp class="samp">vuslice</samp>’</dt>
<dt>‘<samp class="samp">vdslice</samp>’</dt>
<dt>‘<samp class="samp">hblur</samp>’</dt>
<dt>‘<samp class="samp">fadegrays</samp>’</dt>
<dt>‘<samp class="samp">wipetl</samp>’</dt>
<dt>‘<samp class="samp">wipetr</samp>’</dt>
<dt>‘<samp class="samp">wipebl</samp>’</dt>
<dt>‘<samp class="samp">wipebr</samp>’</dt>
<dt>‘<samp class="samp">squeezeh</samp>’</dt>
<dt>‘<samp class="samp">squeezev</samp>’</dt>
<dt>‘<samp class="samp">zoomin</samp>’</dt>
<dt>‘<samp class="samp">fadefast</samp>’</dt>
<dt>‘<samp class="samp">fadeslow</samp>’</dt>
<dt>‘<samp class="samp">hlwind</samp>’</dt>
<dt>‘<samp class="samp">hrwind</samp>’</dt>

<dt>‘<samp class="samp">vuwind</samp>’</dt>
<dt>‘<samp class="samp">vdwind</samp>’</dt>
<dt>‘<samp class="samp">coverleft</samp>’</dt>
<dt>‘<samp class="samp">coverright</samp>’</dt>
<dt>‘<samp class="samp">coverup</samp>’</dt>
<dt>‘<samp class="samp">coverdown</samp>’</dt>
<dt>‘<samp class="samp">revealleft</samp>’</dt>
<dt>‘<samp class="samp">revealright</samp>’</dt>
<dt>‘<samp class="samp">revealup</samp>’</dt>
<dt>‘<samp class="samp">revealdown</samp>’</dt>
</dl>
Default transition effect is fade.

</dd>
<dt><samp class="option">duration</samp></dt>
<dd> Set cross fade duration in seconds.
Range is 0 to 60 seconds.
Default duration is 1 second.

</dd>
<dt><samp class="option">offset</samp></dt>
<dd> Set cross fade start relative to first input stream in seconds.
Default offset is 0.

</dd>
<dt><samp class="option">expr</samp></dt>
<dd> Set expression for custom transition effect.

The expressions can use the following variables and functions:

<dl class="table">

<dt><samp class="option">X</samp></dt>
<dt><samp class="option">Y</samp></dt>
<dd> The coordinates of the current sample.

</dd>
<dt><samp class="option">W</samp></dt>
<dt><samp class="option">H</samp></dt>
<dd> The width and height of the image.

</dd>
<dt><samp class="option">P</samp></dt>
<dd> Progress of transition effect.

</dd>
<dt><samp class="option">PLANE</samp></dt>
<dd> Currently processed plane.

</dd>
<dt><samp class="option">A</samp></dt>
<dd> Return value of first input at current location and plane.

</dd>
<dt><samp class="option">B</samp></dt>
<dd> Return value of second input at current location and plane.

</dd>
<dt><samp class="option">a0(x, y)</samp></dt>
<dt><samp class="option">a1(x, y)</samp></dt>
<dt><samp class="option">a2(x, y)</samp></dt>
<dt><samp class="option">a3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of first input.

</dd>

<dt><samp class="option">b0(x, y)</samp></dt>

<dt><samp class="option">b1(x, y)</samp></dt>

<dt><samp class="option">b2(x, y)</samp></dt>

<dt><samp class="option">b3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of second input.

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">39.290.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input video to another input video, with fade transition and duration of transition of 2 seconds starting at offset of 5 seconds: <pre class="example-preformatted">ffmpeg -i first.mp4 -i second.mp4 -filter_complex xfade=transition=fade:duration=2:offset=5 output.mp4 </pre>

 <h3 class="section">39.291 xmedian# TOC</h3> Pick median pixels from several input videos. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 3. Allowed range is from 3 to 255. If number of inputs is even number, than result will be mean value between two median values. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">15</code>, by which all planes are processed. </dd> <dt><samp class="option">percentile</samp></dt>

<dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">39.291.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">inputs</code>. <h3 class="section">39.292 xpsnr# TOC</h3>

Obtain the average (across all input frames) and minimum (across all color plane averages) eXtended Perceptually weighted peak Signal-to-Noise Ratio (XPSNR) between two input videos. The XPSNR is a low-complexity psychovisually motivated distortion measurement algorithm for assessing the difference between two video streams or images. This is especially useful for objectively quantifying the distortions caused by video and image codecs, as an alternative to a formal subjective test. The logarithmic XPSNR output values are in a similar range as those of traditional psnr assessments but better reflect human impressions of visual coding quality. More details on the XPSNR measure, which essentially represents a blockwise weighted variant of the PSNR measure, can be found in the following freely available papers: <ul class="itemize mark-bullet"> C. R. Helmrich, M. Siekmann, S. Becker, S. Bosse, D. Marpe, and T. Wiegand, "XPSNR: A Low-Complexity Extension of the Perceptually Weighted Peak Signal-to-Noise Ratio for High-Resolution Video Quality Assessment," in Proc. IEEE Int. Conf. Acoustics, Speech, Sig. Process. (ICASSP), virt./online, May 2020. www.ecodis.de/xpsnr.htm

C. R. Helmrich, S. Bosse, H. Schwarz, D. Marpe, and T. Wiegand, "A Study of the Extended Perceptually Weighted Peak Signal-to-Noise Ratio (XPSNR) for Video Compression with Different Resolutions and Bit Depths," ITU Journal: ICT Discoveries, vol. 3, no. 1, pp. 65 - 72, May 2020. http://handle.itu.int/11.1002/pub/8153d78b-en

When publishing the results of XPSNR assessments obtained using, e.g., this FFmpeg filter, a reference to the above papers as a means of documentation is strongly encouraged. The filter requires two input videos. The first input is considered a (usually not distorted) reference source and is passed unchanged to the output, whereas the second input is a (distorted) test sig-

nal. Except for the bit depth, these two video inputs must have the same pixel format. In addition, for best performance, both compared input videos should be in YCbCr color format. The obtained overall XPSNR values mentioned above are printed through the logging system. In case of input with multiple color planes, we suggest reporting of the minimum XPSNR average. The following parameter, which behaves like the one for the psnr filter, is accepted: <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified, the filter will use the named file to save the XPSNR value of each individual frame and color plane. When the file name equals "-", that data is sent to standard output. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">39.292.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> XPSNR analysis of two 1080p HD videos, ref_source.yuv and test_video.yuv, both at 24 frames per second, with color format 4:2:0, bit depth 8, and output of a logfile named "xpsnr.log": <pre class="example-preformatted">ffmpeg -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i ref_source.yuv -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i test_video.yuv -lavfi xpsnr="stats_file=xpsnr.log" -f null - </pre>

XPSNR analysis of two 2160p UHD videos, ref_source.yuv with bit depth 8 and test_video.yuv with bit depth 10, both at 60 frames per second with color format 4:2:0, no logfile output: <pre class="example-preformatted">ffmpeg -s 3840x2160 -framerate 60 -pix_fmt yuv420p -i ref_source.yuv -s 3840x2160 -framerate 60 -pix_fmt yuv420p10le -i test_video.yuv -lavfi xpsnr="stats_file=-" -f null - </pre>

 <h3 class="section">39.293 xstack# TOC</h3> Stack video inputs into custom layout. All streams must be of same pixel format. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">layout</samp></dt> <dd> Specify layout of inputs. This option requires the desired layout configuration to be explicitly set by the user. This sets position of each video input in output. Each input is separated by ’|’. The first number represents the column, and the second number represents the row. Numbers start at 0 and are separated by ’_’. Optionally one can use wX and hX, where

X is video input from which to take width or height. Multiple values can be used when separated by ’+’. In such case values are summed together. Note that if inputs are of different sizes gaps may appear, as not all of the output video frame will be filled. Similarly, videos can overlap each other if their position doesn’t leave enough space for the full frame of adjoining videos. For 2 inputs, a default layout of <code class="code">0_0|w0_0</code> (equivalent to <code class="code">grid=2x1</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">grid</samp></dt> <dd> Specify a fixed size grid of inputs. This option is used to create a fixed size grid of the input streams. Set the grid size in the form <code class="code">COLUMNSxROWS</code>. There must be <code class="code">ROWS * COLUMNS</code> input streams and they will be arranged as a grid with <code class="code">ROWS</code> rows and <code class="code">COLUMNS</code> columns. When using this option, each input stream within a row must have the same height and all the rows must have the same width. If <code class="code">grid</code> is set, then <code class="code">inputs</code> option is ignored and is implicitly set to <code class="code">ROWS * COLUMNS</code>. For 2 inputs, a default grid of <code class="code">2x1</code> (equivalent to <code class="code">layout=0_0|w0_0</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">fill</samp></dt> <dd> If set to valid color, all unused pixels will be filled with that color. By default fill is set to none, so it is disabled. </dd> </dl>

 <h4 class="subsection">39.293.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Display 4 inputs into 2x2 grid.

Layout: <pre class="example-preformatted">input1(0, 0) | input3(w0, 0) input2(0, h0) | input4(w0, h0) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|w0_0|w0_h
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 4 inputs into 1x4 grid.

Layout: <pre class="example-preformatted">input1(0,

0) input2(0, h0) input3(0, h0+h1) input4(0, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|0_h0+h1|0_
</pre>

Note that if inputs are of different widths, unused space will appear.
Display 9 inputs into 3x3 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input4(w0, 0) | input7(w0+w3, 0) input2(0, h0) | input5(w0, h0) | in-
put8(w0+w3, h0) input3(0, h0+h1) | input6(w0, h0+h1) | input9(w0+w3,
h0+h1) </pre>

 <pre class="example-preformatted">xstack=inputs=9:layout=0_0|0_h0|0_h0+h1|w
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 16 inputs into 4x4 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input5(w0, 0) | input9 (w0+w4, 0) | input13(w0+w4+w8, 0) input2(0, h0)
| input6(w0, h0) | input10(w0+w4, h0) | input14(w0+w4+w8, h0) input3(0,
h0+h1) | input7(w0, h0+h1) | input11(w0+w4, h0+h1) | input15(w0+w4+w8,
h0+h1) input4(0, h0+h1+h2)| input8(w0, h0+h1+h2)| input12(w0+w4,
h0+h1+h2)| input16(w0+w4+w8, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=16:layout=0_0|0_h0|0_h0+h1|
w0+w4_h0|w0+w4_h0+h1|w0+w4_h0+h1+h2|w0+w4+w8_0|w0+w4+w8_h0|w0+w4+w8_h0+h1|w0+w4-
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.

 <h3
class="section">39.294 yadif<a class="anchor
hidden-xs" href="#yadif" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-yadif" aria-hidden="true">TOC</h3>

Deinterlace the input video ("yadif" means "yet another
deinterlacing filter"). It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt>
<dd> The interlacing mode to adopt. It accepts one of the follow-
ing values: <dl class="table"> <dt><samp class="option">0,
send_frame</samp></dt> <dd> Output one frame for each frame.
</dd> <dt><samp class="option">1, send_field</samp></dt>
<dd> Output one frame for each field. </dd> <dt><samp
class="option">2, send_frame_nospatial</samp></dt> <dd> Like
<code class="code">send_frame</code>, but it skips the spatial interlacing
check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt>

<dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">39.295 yaepblur# TOC</h3>

Apply blur filter while preserving edges ("yaepblur" means "yet another edge preserving blur filter"). The algorithm is described in "J. S. Lee, Digital image enhancement and noise filtering by use of local statistics, IEEE Trans. Pattern Anal. Mach. Intell. PAMI-2, 1980." It accepts the following parameters: <dl class="table"> <dt><samp class="option">radius, r</samp></dt> <dd> Set the window radius. Default value is 3. </dd> <dt><samp class="option">planes, p</samp></dt> <dd> Set which planes to filter. Default is only the first plane. </dd> <dt><samp class="option">sigma, s</samp></dt> <dd> Set blur strength. Default value is 128. </dd> </dl>

 <h4 class="subsection">39.295.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">39.296 zoompan<a class="anchor hidden-xs" href="#zoompan"

aria-hidden=”true”># TOC</h3>

Apply Zoom & Pan effect. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>zoom, z</samp></dt> <dd> Set the zoom expression. Range is 1-10. Default is 1. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the x and y expression. Default is 0. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Set the duration expression in number of frames. This sets for how many number of frames effect will last for single input image. Default is 90. </dd> <dt><samp class=”option”>s</samp></dt> <dd> Set the output image size, default is ’hd720’. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set the output frame rate, default is ’25’. </dd> </dl>

Each expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> Input width. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> Input height. </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dd> Output width. </dd> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> Output height. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> <dt><samp class=”option”>in_time, it</samp></dt> <dd> The input timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>out_time, time, ot</samp></dt> <dd> The output timestamp expressed in seconds. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Last calculated ’x’ and ’y’ position from ’x’ and ’y’ expression for current input frame. </dd> <dt><samp class=”option”>px</samp></dt> <dt><samp class=”option”>py</samp></dt> <dd> ’x’ and ’y’ of last output frame of previous input frame or 0 when there was not yet such frame (first input frame). </dd> <dt><samp class=”option”>zoom</samp></dt> <dd> Last calculated zoom from ’z’ expression for current input frame. </dd> <dt><samp class=”option”>pzoom</samp></dt> <dd> Last calculated zoom of last output frame of previous input frame. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> Number of output frames for current input frame. Calculated from ’d’ expression for each input frame. </dd> <dt><samp class=”option”>pduration</samp></dt> <dd> number of output frames created for previous input frame </dd> <dt><samp class=”option”>a</samp></dt> <dd> Rational number: input width / input height </dd> <dt><samp class=”option”>sar</samp></dt>

<dd> sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt>
<dd> display aspect ratio </dd> </dl>

 <h4 class=”subsection”>39.296.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Zoom in up to 1.5x and pan at same time to some spot near center of picture: <pre class=”example-preformatted”>zoompan=z='min(zoom+0.0015,1.5)':d=700:x='if(gte(zoom,1.5),x,x+1/a)':y=
</pre>

Zoom in up to 1.5x and pan always at center of picture: <pre class=”example-preformatted”>zoompan=z='min(zoom+0.0015,1.5)':d=700:x='iw/2
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Same as above but without pausing: <pre class=”example-preformatted”>zoompan=z='min(max(zoom,pzoom)+0.0015,1.5)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Zoom in 2x into center of picture only for the first second of the input video: <pre class=”example-preformatted”>zoompan=z='if(between(in_time,0,1),2,1)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

 <h3 class=”section”>39.297 zscale# TOC</h3>
Scale (resize) the input video, using the z.lib library: https://github.com/sekrit-twc/zimg. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libzimg</code>. The zscale filter forces the output display aspect ratio to be the same as the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the zscale filter will convert the input to the requested format. <h4 class=”subsection”>39.297.1 Options# TOC</h4> The filter accepts the following options. <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class=”var”>width</var> or <var class=”var”>w</var> value is 0, the

input width is used for the output. If the <var class=”var”>height</var> or <var class=”var”>h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the zscale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>dither, d</samp></dt> <dd> Set the dither type. Possible values are: <dl class=”table”> <dt><var class=”var”>none</var></dt> <dt><var class=”var”>ordered</var></dt> <dt><var class=”var”>random</var></dt> <dt><var class=”var”>error_diffusion</var></dt> </dl>

Default is none. </dd> <dt><samp class=”option”>filter, f</samp></dt> <dd> Set the resize filter type. Possible values are: <dl class=”table”> <dt><var class=”var”>point</var></dt> <dt><var class=”var”>bilinear</var></dt> <dt><var class=”var”>bicubic</var></dt> <dt><var class=”var”>spline16</var></dt> <dt><var class=”var”>spline36</var></dt> <dt><var class=”var”>spline64</var></dt> <dt><var class=”var”>lanczos</var></dt> </dl>

Default is bilinear. </dd> <dt><samp class=”option”>range, r</samp></dt> <dd> Set the color range. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>limited</var></dt> <dt><var class=”var”>full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>primaries, p</samp></dt> <dd> Set the color primaries. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>170m</var></dt> <dt><var class=”var”>240m</var></dt> <dt><var class=”var”>2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>transfer, t</samp></dt> <dd> Set the transfer characteristics. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>601</var></dt> <dt><var class=”var”>linear</var></dt> <dt><var class=”var”>2020_10</var></dt> <dt><var class=”var”>2020_12</var></dt> <dt><var class=”var”>smpte2084</var></dt> <dt><var class=”var”>iec61966-2-1</var></dt> <dt><var class=”var”>arib-std-b67</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">rangein, rin</samp></dt> <dd> Set the input color range. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">limited</var></dt> <dt><var class="var">full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primariesin, pin</samp></dt> <dd> Set the input color primaries. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">240m</var></dt> <dt><var class="var">2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transferin, tin</samp></dt> <dd> Set the input transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">601</var></dt> <dt><var class="var">linear</var></dt> <dt><var class="var">2020_10</var></dt> <dt><var class="var">2020_12</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrixin, min</samp></dt> <dd> Set the input colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

</dd> <dt><samp class="option">chromal, c</samp></dt> <dd> Set the output chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt> <dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt> <dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt> </dl>

</dd> <dt><samp class="option">chromalin, cin</samp></dt> <dd> Set the input chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt>

<dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt>
<dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt>
</dl>

</dd> <dt><samp class="option">npl</samp></dt> <dd> Set the nominal peak luminance. </dd> <dt><samp class="option">param_a</samp></dt>
<dd> Parameter A for scaling filters. Parameter "b" for bicubic, and the number of filter taps for lanczos. </dd> <dt><samp class="option">param_b</samp></dt> <dd> Parameter B for scaling filters. Parameter "c" for bicubic. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt>
<dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd>
<dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt>
<dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt>
<dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt>
<dd> The same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt>
<dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt>
<dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt>
<dt><var class="var">ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">39.297.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h2 class="chapter">40 CUDA Video Filters# TOC</h2>

Below is a description of the currently available Nvidia CUDA video filters. Prerequisites: <ul class="itemize mark-bullet"> Install Nvidia CUDA Toolkit

Note: If FFmpeg detects the Nvidia CUDA Toolkit during configuration, it will enable CUDA filters automatically without requiring any additional flags. If you want to explicitly enable them, use the following options: <ul class="itemize mark-bullet"> Configure FFmpeg with <code class="code">--enable-cuda-nvcc --enable-nonfree</code>. Configure FFmpeg with <code class="code">--enable-cuda-llvm</code>. Additional requirement: <code class="code">llvm</code> lib must be installed.

Running CUDA filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device cuda[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">cuda</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of initializing second CUDA device on the system and running scale_cuda and bilateral_cuda filters. <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -init_hw_device cuda:1 -filter_complex \ "[0:v]scale_cuda=format=yuv444p[scaled_video];[sca
\ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

Since CUDA filters operate exclusively on GPU memory, frame data must sometimes be uploaded (hwupload) to hardware surfaces associated with the appropriate CUDA device before processing, and downloaded (hwdownload) back to normal memory afterward, if required. Whether hwupload or hwdownload is necessary depends on the specific workflow: <ul class="itemize mark-bullet"> If the input frames are already in GPU memory (e.g., when using <code class="code">-hwaccel cuda</code> or <code class="code">-hwaccel_output_format cuda</code>), explicit use of <a class="ref"

href=”#hwupload”>hwupload is not needed, as the data is already in the appropriate memory space. If the input frames are in CPU memory (e.g., software-decoded frames or frames processed by CPU-based filters), it is necessary to use hwupload to transfer the data to GPU memory for CUDA processing. If the output of the CUDA filters needs to be further processed by software-based filters or saved in a format not supported by GPU-based encoders, hwdownload is required to transfer the data back to CPU memory. Note that hwupload uploads data to a surface with the same layout as the software frame, so it may be necessary to add a format filter immediately before hwupload to ensure the input is in the correct format. Similarly, hwdownload may not support all output formats, so an additional format filter may need to be inserted immediately after hwdownload in the filter graph to ensure compatibility. <h3 class=”section”>40.1 bilateral_cuda# TOC</h3> CUDA accelerated bilateral filter, an edge preserving filter. This filter is mathematically accurate thanks to the use of GPU acceleration. For best output quality, use one to one chroma subsampling, i.e. yuv444p format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight, also called sigma space. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate color range weight, also called sigma color. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>window_size</samp></dt> <dd> Set window size of the bilateral function to determine the number of neighbours to loop on. If the number entered is even, one will be added automatically. Allowed range is 1 to 255. Default is 1. </dd> </dl> <h4 class=”subsection”>40.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the bilateral filter on a video.

 <pre class=”example-preformatted”>./ffmpeg -v verbose \ -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]scale_cuda=format=yuv444p[scaled_video]; [scaled_video]bilateral_cuda=window_size=9:sigmaS=3.0:sigmaR=50.0" \ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

 <h3 class="section">40.2 bwdif_cuda# TOC</h3>

Deinterlace the input video using the bwdif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">40.3 chromakey_cuda# TOC</h3> CUDA accelerated YUV colorspace color/chroma keying. This filter works like normal chromakey filter but operates on CUDA frames. for more details and parameters see chromakey. <h4 class="subsection">40.3.1 Examples<span class="pull-

right”>#
TOC</h4>

<ul class=”itemize mark-bullet”> Make all the green pixels in the input video transparent and use it as an overlay for another video:

 <pre class=”example-preformatted”>./ffmpeg \
-hwaccel cuda -hwaccel_output_format cuda -i input_green.mp4 \ -hwaccel cuda -hwaccel_output_format cuda -i base_video.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]chromakey_cuda=0x25302D:0.1:0.12:1[overlay_video]; \ [1:v]scale_cuda=format=yuv420p[base]; \ [base][overlay_video]overlay_cuda" \ -an -sn -c:v h264_nvenc -cq 20 output.mp4 </pre>

Process two software sources, explicitly uploading the frames:

 <pre class=”example-preformatted”>./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda \ -f lavfi -i color=size=800x600:color=white,format=yuv42 \ -f lavfi -i yuvtestsrc=size=200x200,format=yuv420p \ -filter_complex \ " \ [0]hwupload[under]; \ [1]hwupload,chromakey_cuda=green:0.1:0.12[over]; \ [under][over]overlay_cuda" \ -c:v hevc_nvenc -cq 18 -preset slow output.mp4 </pre>

 <h3 class=”section”>40.4 colorspace_cuda# TOC</h3>

CUDA accelerated implementation of the colorspace filter. It is by no means feature complete compared to the software colorspace filter, and at the current time only supports color range conversion between jpeg/full and mpeg/limited range. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> </dl>

 <h3 class=”section”>40.5 overlay_cuda# TOC</h3>

Overlay one video on top of another. This is the CUDA variant of the overlay filter. It only accepts CUDA frames. The underlying input pixel formats have to match. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. They can contain the following parameters: <dl class=”table”> <dt><samp class=”option”>main_w, W</samp></dt> <dt><samp class=”option”>main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class=”option”>overlay_w, w</samp></dt> <dt><samp class=”option”>overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The ordinal index of the main input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> The byte offset position in the file of the main input frame, NAN if unknown. Deprecated, do not use. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp of the main input frame, expressed in seconds, NAN if unknown. </dd> </dl>

Default value is "0" for both expressions. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for <samp class=”option”>x</samp> and <samp class=”option”>y</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>init</samp></dt> <dd> Evaluate expressions once during filter initialization or when a command is processed. </dd> <dt><samp class=”option”>frame</samp></dt> <dd> Evaluate expressions for each incoming frame </dd> </dl>

Default value is <samp class=”option”>frame</samp>. </dd> <dt><samp class=”option”>eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options.

 <h3 class=”section”>40.6 pad_cuda<a

class=”anchor hidden-xs” href=”#pad_005fcuda” aria-hidden=”true”>#
TOC</h3>
Add paddings to an input video stream using CUDA. This filter is the CUDA-accelerated version of the pad filter. It accepts the same options and expressions and provides the same core functionality. For a detailed description of available options, please see the documentation for the pad filter. <h4 class=”subsection”>40.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Add a 200-pixel black border to all sides of a video frame: <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=iw+400:h=ih+400:x=200:y=200" -c:v h264_nvenc out.mp4 </pre>

Pad the input video to a 16:9 aspect ratio, filling with the color "blue": <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=ih*16/9/sar:h=ih:x=(ow-iw)/2:y=(oh-ih)/2:color=blue" -c:v h264_nvenc out.mp4 </pre>

 <h3 class=”section”>40.7 scale_cuda#
TOC</h3>

Scale (resize) and convert (pixel format) the input video, using accelerated CUDA kernels. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>interp_algo</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>nearest</var></dt> <dd> Nearest neighbour Used by default if input parameters match the desired output. </dd> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic This is the default. </dd> <dt><var class=”var”>lanczos</var></dt> <dd> High-quality separable Lanczos (windowed sinc). The filter radius can be set with <samp class=”option”>param</samp>; the default is 3. </dd> </dl>

</dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. Converting between <code class="code">yuv420p</code>, <code class="code">yuv422p</code>, <code class="code">yuv444p</code>, <code class="code">yuv420p10</code>, <code class="code">yuv422p10</code>, <code class="code">yuv444p10</code>, <code class="code">nv12</code>, <code class="code">nv16</code>, <code class="code">p010</code> or <code class="code">p210</code> and <code class="code">bgra</code>, <code class="code">rgba</code>, <code class="code">bgr0</code> or <code class="code">rgb0</code> is supported, also together with resizing. Resizing is then done on the RGB side, after converting from YUV or before converting to YUV, and always uses the generic filters selected with <samp class="option">interp_algo</samp>. Resizing the YUV side in a separate instance instead needs less intermediate memory and may be faster, but filters chroma differently. YUV to RGB conversion uses the YCbCr matrix and range of the input, RGB to YUV conversion those selected for the output (see <samp class="option">out_color_matrix</samp> and <samp class="option">out_range</samp>). An unspecified matrix is treated as BT.601 and an unspecified range as limited. The conversion samples chroma independently of <samp class="option">interp_algo</samp>: YUV to RGB conversion uses the nearest chroma sample, RGB to YUV conversion averages, which centers the chroma samples. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> If set to 0, every frame is processed, even if no conversion is necessary. This mode can be useful to use the filter as a buffer for a downstream frame-consumer that exhausts the limited decoder frame pool. If set to 1, frames are passed through as-is if they match the desired output parameters. This is the default behaviour. </dd> <dt><samp class="option">use_filters</samp></dt> <dd> If set to 1, filter with a generic weight LUT instead of using fixed-function shader kernels. May be faster or slower depending on the hardware. A value of <code class="code">auto</code> (the default) enables this automatically when required for correct anti-aliasing when downscaling, and for high-quality Lanczos scaling. Set this to 0 to use the legacy fixed 4-tap Lanczos kernel. Resizing during YUV to RGB or RGB to YUV conversion always uses the generic filters. </dd> <dt><samp class="option">param</samp></dt> <dd> Algorithm-specific parameter. For <var class="var">bicubic</var>, this affects the filter curve. For <var class="var">lanczos</var>, this sets the filter radius from 1 to 10 (default 3) when using the generic filter path; it has no effect with <samp class="option">use_filters=0</samp>. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class="option">out_range</samp></dt>

<dd> Set the output YCbCr sample range. One of <code class=”code”>auto</code> (the default), <code class=”code”>jpeg</code>/<code class=”code”>full</code>/<code class=”code”>pc</code>, or <code class=”code”>mpeg</code>/<code class=”code”>limited</code>/<code class=”code”>tv</code>. </dd> <dt><samp class=”option”>out_color_matrix</samp></dt> <dd> Set the output YCbCr matrix. One of <code class=”code”>auto</code> (the default), <code class=”code”>bt601</code>, <code class=”code”>smpte170m</code>, <code class=”code”>bt470</code>, <code class=”code”>bt470bg</code>, <code class=”code”>bt709</code>, <code class=”code”>fcc</code>, <code class=”code”>smpte240m</code>, <code class=”code”>ycgco</code>, <code class=”code”>bt2020</code> or <code class=”code”>bt2020nc</code>. With <code class=”code”>auto</code>, <samp class=”option”>out_range</samp> and <samp class=”option”>out_color_matrix</samp> are left to filter-graph negotiation. They follow a single range and matrix negotiated for the input, as for decoded video, and otherwise prefer BT.601 limited range. For RGB to YUV conversion, <code class=”code”>auto</code> thus means BT.601 limited range, unless the filter graph requires otherwise; converting full range YUV to RGB and back keeps the full range. Outside RGB to YUV conversion, these options describe the existing samples without remapping them; an unspecified input description may adopt the negotiated output description, while conflicting specified descriptions are rejected. Both options can only be used with YUV output; RGB output is full range. </dd> </dl>

 <h4 class=”subsection”>40.7.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. <pre class=”example-preformatted”>scale_cuda=-2:720:format=yuv420p </pre>

Upscale to 4K using nearest neighbour algorithm. <pre class=”example-preformatted”>scale_cuda=4096:2160:interp_algo=nearest </pre>

Don’t do any conversion or scaling, but copy all input frames into newly allocated ones. This can be useful to deal with a filter and encode chain that otherwise exhausts the decoders frame pool. <pre class=”example-preformatted”>scale_cuda=passthrough=0 </pre>

Convert BT.709 input to RGB for a filter that requires it, then back to BT.709 NV12 with limited range. <pre class=”example-preformatted”>scale_cuda=format=bgra,scale_cuda=format=nv12:out_color_matrix=bt70 </pre>

Scale to 720p while converting to RGB.

<pre class="example-preformatted">scale_cuda=-2:720:format=bgra
</pre>

 <h3 class="section">40.8 thumbnail_cuda# TOC</h3>

Select the most representative frame in a given sequence of consecutive frames using CUDA. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">40.8.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Thumbnails are extracted from every <var class="var">n</var>=150-frame batch, selecting one per batch. Chosen frames are then scaled with scale_cuda. <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i ./input.mp4 -vf "thumbnail_cuda=150,scale_cuda=1920:
./output/out%03d.png </pre>

 <h3 class="section">40.9 transpose_cuda# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt>

<dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class="samp">reversal</samp>’</dt> <dd> Rotate by 180 degrees. </dd> <dt>‘<samp class="samp">hflip</samp>’</dt> <dd> Flip horizontally. </dd> <dt>‘<samp class="samp">vflip</samp>’</dt> <dd> Flip vertically. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">40.10 yadif_cuda# TOC</h3>

Deinterlace the input video using the yadif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt> <dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1,

auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h2 class="chapter">41 OpenCL Video Filters# TOC</h2>

Below is a description of the currently available OpenCL video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-opencl</code>. Running OpenCL filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device opencl[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</samp> <dd> Initialise a new hardware device of type <var class="var">opencl</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device on the second platform and running avgblur_opencl filter with default parameters on it. <pre class="example-preformatted">-init_hw_device opencl=gpu:1.0 -filter_hw_device gpu -i INPUT -vf "hwupload, avgblur_opencl, hwdownload" OUTPUT </pre>

Since OpenCL filters are not able to access frame data in normal memory, all frame data needs to be uploaded(hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded(hwdownload) back to normal memory. Note that hwupload will upload to a surface with the same layout as the software frame, so it

may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it may be necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class=”section”>41.1 avgblur_opencl# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>0</code>. If zero, <code class=”code”>sizeX</code> value will be used. </dd> </dl>

 <h4 class=”subsection”>41.1.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply average blur filter with horizontal and vertical size of 3, setting each pixel of the output to the average value of the 7x7 region centered on it in the input. For pixels on the edges of the image, the region does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, avgblur_opencl=3, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.2 boxblur_opencl# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class="table">
<dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp
class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">alpha_radius,
ar</samp></dt> <dd> Set an expression for the box radius in pixels used
for blurring the corresponding input plane. The radius value must
be a non-negative number, and must not be greater than the value of the expression <code class="code">min(w,h)/2</code> for the luma and alpha planes,
and of <code class="code">min(cw,ch)/2</code> for the chroma planes.
 Default value for <samp class="option">luma_radius</samp> is
"2". If not specified, <samp class="option">chroma_radius</samp>
and <samp class="option">alpha_radius</samp> default to the corresponding value set for <samp class="option">luma_radius</samp>.
 The expressions can contain the following constants: <dl
class="table"> <dt><samp class="option">w</samp></dt> <dt><samp
class="option">h</samp></dt> <dd> The input width and height
in pixels. </dd> <dt><samp class="option">cw</samp></dt>
<dt><samp class="option">ch</samp></dt> <dd> The input
chroma image width and height in pixels. </dd> <dt><samp
class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt>
<dd> The horizontal and vertical chroma subsample values. For example,
for the pixel format "yuv422p", <var class="var">hsub</var> is
2 and <var class="var">vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class="option">luma_power, lp</samp></dt>
<dt><samp class="option">chroma_power, cp</samp></dt> <dt><samp
class="option">alpha_power, ap</samp></dt> <dd> Specify how
many times the boxblur filter is applied to the corresponding plane.
Default value for <samp class="option">luma_power</samp> is 2. If
not specified, <samp class="option">chroma_power</samp> and <samp
class="option">alpha_power</samp> default to the corresponding value set
for <samp class="option">luma_power</samp>. A value of 0
will disable the effect. </dd> </dl>

 <h4 class="subsection">41.2.1 Examples# TOC</h4>

Apply boxblur filter, setting each pixel of the output to the average value of box-radiuses <var class="var">luma_radius</var>, <var
class="var">chroma_radius</var>, <var class="var">alpha_radius</var>
for each plane respectively. The filter will apply <var class="var">luma_power</var>,
<var class="var">chroma_power</var>, <var class="var">alpha_power</var>
times onto the corresponding plane. For pixels on the edges of the image,
the radius does not extend beyond the image boundaries, and so out-of-range
coordinates are not used in the calculations. <ul class="itemize
mark-bullet"> Apply a boxblur filter with the luma, chroma, and

alpha radius set to 2 and luma, chroma, and alpha power set to 3. The filter will run 3 times with box-radius set to 2 for every plane of the image. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=luma_radius=2:luma_power=3, hwdownload" OUTPUT -i INPUT -vf "hwupload, boxblur_opencl=2:3, hwdownload" OUTPUT </pre>

Apply a boxblur filter with luma radius set to 2, luma_power to 1, chroma_radius to 4, chroma_power to 5, alpha_radius to 3 and alpha_power to 7.

For the luma plane, a 2x2 box radius will be run once. For the chroma plane, a 4x4 box radius will be run 5 times. For the alpha plane, a 3x3 box radius will be run 7 times. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=2:1:4:5:3:7, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.3 colorkey_opencl# TOC</h3>
RGB colorspace color keying. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt>
<dd> The color which will be replaced with transparency. </dd>
<dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage.
0.0 makes pixels either fully transparent, or not transparent at all.
Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h4 class=”subsection”>41.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make every semi-green pixel in the input transparent with some slight blending:
<pre class=”example-preformatted”>-i INPUT -vf "hwupload, colorkey_opencl=green:0.3:0.1, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.4 convolution_opencl# TOC</h3

Apply convolution of 3x3, 5x5, 7x7 matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>0m</samp></dt> <dt><samp class=”option”>1m</samp></dt> <dt><samp class=”option”>2m</samp></dt> <dt><samp class=”option”>3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed numbers. Default value for each plane is <code class=”code”>0 0 0 0 1 0 0 0 0</code>. </dd> <dt><samp class=”option”>0rdiv</samp></dt> <dt><samp class=”option”>1rdiv</samp></dt> <dt><samp class=”option”>2rdiv</samp></dt> <dt><samp class=”option”>3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be sum of all matrix elements. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>0bias</samp></dt> <dt><samp class=”option”>1bias</samp></dt> <dt><samp class=”option”>2bias</samp></dt> <dt><samp class=”option”>3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>41.4.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sharpen: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0, hwdownload" OUTPUT </pre>

Apply blur: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9, hwdownload" OUTPUT </pre>

Apply edge enhance: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply edge detect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1

1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0, hwdownload" OUTPUT </pre>

Apply emboss: <pre class="example-preformatted">-i INPUT -vf "hwupload, convolution_opencl=-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2, hwdownload" OUTPUT </pre>

 <h3 class="section">41.5 erosion_opencl# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">41.5.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply erosion filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local minimum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local minimum is more then threshold of the corresponding plane, output pixel will be set to input pixel - threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, erosion_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">41.6 deshake_opencl# TOC</h3> Feature-point based video stabilization filter. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>tripod</samp></dt> <dd> Simulates a tripod by preventing any camera movement whatsoever from the original frame. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>debug</samp></dt> <dd> Whether or not additional debug info should be displayed, both in the processed output and in the console. Note that in order to see console debug output you will also need to pass <code class=”code”>-v verbose</code> to ffmpeg. Viewing point matches in the output video is only supported for RGB input. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>adaptive_crop</samp></dt> <dd> Whether or not to do a tiny bit of cropping at the borders to cut down on the amount of mirrored pixels. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>refine_features</samp></dt> <dd> Whether or not feature points should be refined at a sub-pixel level. This can be turned off for a slight performance gain at the cost of precision. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>smooth_strength</samp></dt> <dd> The strength of the smoothing applied to the camera path from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. <code class=”code”>1.0</code> is the maximum smoothing strength while values less than that result in less smoothing. <code class=”code”>0.0</code> causes the filter to adaptively choose a smoothing strength on a per-frame basis. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>smooth_window_multiplier</samp>
<dd> Controls the size of the smoothing window (the number of frames buffered to determine motion information from). The size of the smoothing window is determined by multiplying the framerate of the video by this number. Acceptable values range from <code class=”code”>0.1</code> to <code class=”code”>10.0</code>. Larger values increase the amount of motion data available for determining how to smooth the camera path, potentially improving smoothness, but also increase latency and memory usage. Defaults to <code class=”code”>2.0</code>. </dd> </dl>

 <h4 class=”subsection”>41.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Stabilize a video with a fixed, medium smoothing strength: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, deshake_opencl=smooth_strength=0.5, hwdownload" OUTPUT </pre>

Stabilize a video with debugging (both in console and in rendered video): <pre class=”example-preformatted”>-i INPUT -filter_complex "[0:v]format=rgba, hwupload, deshake_opencl=debug=1,

hwdownload, format=rgba, format=yuv420p" -v verbose OUTPUT </pre>

 <h3 class="section">41.7 dilation_opencl# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">41.7.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply dilation filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local maximum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local maximum is more then threshold of the corresponding plane, output pixel will be set to input pixel + threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, dilation_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">41.8 nlmeans_opencl# TOC</h3>

Non-local Means denoise filter through OpenCL, this filter accepts same options as nlmeans. <h3 class="section">41.9 overlay_opencl<a class="anchor hidden-xs"

href=”#overlay_005fopencl” aria-hidden=”true”># TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires same memory layout for all the inputs. So, format conversion may be needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set the x coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>41.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs are yuv420p format. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre> The inputs have same memory layout for color channels , the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.10 pad_opencl# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class=”var”>x</var>, <var class=”var”>y</var> coordinates. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt>

<dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offsets as specified by the <var class="var">x</var> and <var class="var">y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> </dl>

 <h3 class="section">41.11 prewitt_opencl# TOC</h3>

Apply the Prewitt operator (<a class="url" href="https://en.wikipedia.org/wiki/Prewitt_operator">http to input video stream. The filter accepts the following option:

 <dl class="table"> <dt><samp class="option">planes</samp></dt>
<dd> Set which planes to filter. Default value is <code class="code">0xf</code>,
by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt>
<dd> Set value which will be multiplied with filtered result. Range is
<code class="code">[0.0, 65535]</code> and default value is <code
class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt>
<dd> Set value which will be added to filtered result. Range is
<code class="code">[-65535, 65535]</code> and default value is <code
class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">41.11.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply the Prewitt operator with scale
set to 2 and delta set to 10. <pre class="example-
preformatted">-i INPUT -vf "hwupload, prewitt_opencl=scale=2:delta=10,
hwdownload" OUTPUT </pre>

 <h3 class="section">41.12 program_opencl# TOC</h3>

Filter video using an OpenCL program. <dl class="table">
<dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt>
<dd> Kernel name in program. </dd> <dt><samp class="option">inputs</samp></dt>
<dd> Number of inputs to the filter. Defaults to 1. </dd>
<dt><samp class="option">size, s</samp></dt> <dd> Size of output
frames. Defaults to the same as the first input. </dd> </dl>

The <code class="code">program_opencl</code> filter also supports the
framesync options. The
program source file must contain a kernel function with the given name, which
will be run once for each plane of the output. Each run on a plane gets enqueued
as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a
pixel in the destination image. The kernel function needs to take the
following arguments: <ul class="itemize mark-bullet"> Destination
image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it.
Frame index, <var class="var">unsigned int</var>.

This is a counter starting from zero and increasing by one for each
frame. Source images, <var class="var">__read_only

image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to.

Example programs: <ul class="itemize mark-bullet"> Copy the input to the output (output must be the same size as the input). <pre class="verbatim">___kernel void copy(___write_only image2d_t destination, unsigned int index, ___read_only image2d_t source) { const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE;

int2 location = (int2)(get_global_id(0), get_global_id(1));

float4 value = read_imagef(source, sampler, location);

write_imagef(destination, location, value); } </pre> Apply a simple transformation, rotating the input by an amount increasing with the index counter. Pixel values are linearly interpolated by the sampler, and the output need not have the same dimensions as the input. <pre class="verbatim">___kernel void rotate_image(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float angle = (float)index / 100.0f;

float2 dst_dim = convert_float2(get_image_dim(dst)); float2 src_dim = convert_float2(get_image_dim(src));

float2 dst_cen = dst_dim / 2.0f; float2 src_cen = src_dim / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1));

float2 dst_pos = convert_float2(dst_loc) - dst_cen; float2 src_pos = { cos(angle) * dst_pos.x - sin(angle) * dst_pos.y, sin(angle) * dst_pos.x + cos(angle) * dst_pos.y }; src_pos = src_pos * src_dim / dst_dim;

float2 src_loc = src_pos + src_cen;

if (src_loc.x < 0.0f || src_loc.y < 0.0f || src_loc.x > src_dim.x || src_loc.y > src_dim.y) write_imagef(dst, dst_loc, 0.5f); else write_imagef(dst, dst_loc, read_imagef(src, sampler, src_loc)); } </pre> Blend two inputs together, with the amount of each input used varying with the index counter. <pre class="verbatim">___kernel void blend_images(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src1, ___read_only image2d_t src2) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float blend = (cos((float)index / 50.0f) + 1.0f) / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1)); int2 src1_loc = dst_loc * get_image_dim(src1) / get_image_dim(dst); int2 src2_loc = dst_loc * get_image_dim(src2) / get_image_dim(dst);

float4 val1 = read_imagef(src1, sampler, src1_loc); float4 val2 = read_imagef(src2, sampler, src2_loc);

write_imagef(dst, dst_loc, val1 * blend + val2 * (1.0f - blend)); } </pre>

 <h3 class="section">41.13 remap_opencl# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 32bit float pixel format, single channel. <dl class="table"> <dt><samp class="option">interp</samp></dt> <dd> Specify interpolation used for remapping of pixels. Allowed values are <code class="code">near</code> and <code class="code">linear</code>. Default value is <code class="code">linear</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">41.14 roberts_opencl# TOC</h3> Apply the Roberts cross operator (<a class="url" href="https://en.wikipedia.org/wiki/Roberts_cross">https://en.w to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">0xf</code>, by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class="code">[0.0, 65535]</code> and default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class="code">[-65535, 65535]</code> and default value is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">41.14.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the Roberts cross operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, roberts_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.15 sobel_opencl# TOC</h3>

Apply the Sobel operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Sobel_operator”>https://
to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>41.15.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sobel operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, sobel_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.16 tonemap_opencl# TOC</h3>

Perform HDR(PQ/HLG) to SDR conversion with tone-mapping. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tonemap</samp></dt> <dd> Specify the tone-mapping operator to be used. Same as tonemap option in tonemap. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Tune the tone mapping algorithm. same as param option in tonemap. </dd> <dt><samp class=”option”>desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by

(smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default value is 0.5, and the algorithm here is a little different from the cpu version tonemap currently. A setting of 0.0 disables this option. </dd> <dt><samp class="option">threshold</samp></dt> <dd> The tonemapping algorithm parameters is fine-tuned per each scene. And a threshold is used to detect whether the scene has changed or not. If the distance between the current frame average brightness and the current running average exceeds a threshold value, we would re-calculate scene average and peak brightness. The default value is 0.2. </dd> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Currently supported formats are: <dl class="table"> <dt><var class="var">p010</var></dt> <dt><var class="var">nv12</var></dt> </dl>

</dd> <dt><samp class="option">range, r</samp></dt> <dd> Set the output color range. Possible values are: <dl class="table"> <dt><var class="var">tv/mpeg</var></dt> <dt><var class="var">pc/jpeg</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is bt709. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> </dl>

 <h4 class="subsection">41.16.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(PQ/HLG) video to bt2020-transfer-characteristic p010 format using linear operator. <pre class="example-preformatted">-i INPUT -vf "format=p010,hwupload,tonemap_opencl=t=bt2020:tonemap=linear:format=p010,hwdownload,format OUTPUT </pre>

 <h3 class="section">41.17 unsharp_opencl<a class="anchor hidden-xs"

href=”#unsharp_005fopencl” aria-hidden=”true”># TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_amount, la</samp></dt> <dd> Set the luma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>1.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> <dt><samp class=”option”>chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>0.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class=”subsection”>41.17.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply strong luma sharpen effect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=luma_msize_x=7:luma_msize_y=7:luma_amount=2.5, hwdownload" OUTPUT </pre>

Apply a strong blur of both luma and chroma parameters: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=7:7:-2:7:7:-2, hwdownload" OUTPUT </pre>

 <h3 class=”section”>41.18 xfade_opencl# TOC</h3>

Cross fade two videos with custom transition effect by using OpenCL.

 It accepts the following options: <dl class="table"> <dt><samp class="option">transition</samp></dt> <dd> Set one of possible transition effects. <dl class="table"> <dt><samp class="option">custom</samp></dt> <dd> Select custom transition effect, the actual transition description will be picked from source and kernel options. </dd> <dt><samp class="option">fade</samp></dt> <dt><samp class="option">wipeleft</samp></dt> <dt><samp class="option">wiperight</samp></dt> <dt><samp class="option">wipeup</samp></dt> <dt><samp class="option">wipedown</samp></dt> <dt><samp class="option">slideleft</samp></dt> <dt><samp class="option">slideright</samp></dt> <dt><samp class="option">slideup</samp></dt> <dt><samp class="option">slidedown</samp></dt> <dd> Default transition is fade. </dd> </dl>

</dd> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file for custom transition. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Set name of kernel to use for custom transition from program source file. </dd> <dt><samp class="option">duration</samp></dt> <dd> Set duration of video transition. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set time of start of transition relative to first video. </dd> </dl>

The program source file must contain a kernel function with the given name, which will be run once for each plane of the output. Each run on a plane gets enqueued as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a pixel in the destination image. The kernel function needs to take the following arguments: <ul class="itemize mark-bullet"> Destination image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it. First Source image, <var class="var">__read_only image2d_t</var>. Second Source image, <var class="var">__read_only image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to. Transition progress, <var class="var">float</var>. This value is always between 0 and 1 inclusive.

Example programs: <ul class="itemize mark-bullet"> Apply dots curtain transition effect: <pre class="verbatim">__kernel void blend_images(__write_only image2d_t dst, __read_only image2d_t src1, __read_only image2d_t src2, float progress) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR); int2 p = (int2)(get_global_id(0), get_global_id(1)); float2 rp = (float2)(get_global_id(0), get_global_id(1)); float2 dim = (float2)(get_image_dim(src1).x, get_image_dim(src1).y); rp = rp / dim;

float2 dots = (float2)(20.0, 20.0); float2 center = (float2)(0,0); float2 unused;

float4 val1 = read_imagef(src1, sampler, p); float4 val2 = read_imagef(src2, sampler, p); bool next = distance(fract(rp * dots, &unused), (float2)(0.5, 0.5)) < (progress / distance(rp, center));

write_imagef(dst, p, next ? val1 : val2); } </pre>

 <h2 class="chapter">42 VAAPI Video Filters# TOC</h2>

VAAPI Video filters are usually used with VAAPI decoder and VAAPI encoder. Below is a description of VAAPI video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vaapi</code>. To use vaapi filters, you need to setup the vaapi device correctly. For more information, please read <a class="url" href="https://trac.ffmpeg.org/wiki/Hardware/VAAPI">https://trac.ffmpeg.org/wiki/Hardware/
  <h3 class="section">42.1 overlay_vaapi# TOC</h3>

Overlay one video on the top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set expressions for the width and height the overlaid video on the main video. Default values are ’overlay_iw’ for ’w’ and ’overlay_ih*w/overlay_iw’ for ’h’. The expressions can contain the following parameters: <dl class="table"> <dt><samp class="option">main_w, W</samp></dt> <dt><samp class="option">main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class="option">overlay_iw</samp></dt> <dt><samp class="option">overlay_ih</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class="option">overlay_w, w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt> <dd> The overlay output width and height. </dd> <dt><samp class="option">overlay_x, x</samp></dt> <dt><samp class="option">overlay_y, y</samp></dt> <dd> Position of the overlay layer inside of main </dd> </dl>

</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set transparency of overlaid video. Allowed range is 0.0 to 1.0. Higher value

means lower transparency. Default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">42.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs for this filter are yuv420p format. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_vaapi" OUTPUT </pre> Overlay an image LOGO at the offset (200, 100) from the top-left corner of the INPUT video. The inputs have same memory layout for color channels, the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_vaapi=x=200:y=100:w=400:h=300:alpha=1.0, hwdownload, format=nv12" OUTPUT </pre>

 <h3 class="section">42.2 tonemap_vaapi# TOC</h3>

Perform HDR-to-SDR or HDR-to-HDR tone-mapping. It currently only accepts HDR10 as input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Default is nv12 for HDR-to-SDR tone-mapping and p010 for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping.

</dd> <dt><samp class=”option”>display</samp></dt> <dd> Set the output mastering display colour volume. It is given by a ’|’-separated list of two values, two values are space separated. It set display primaries x & y in G, B, R order, then white point x & y, the nominal minimum & maximum display luminances. HDR-to-HDR tone-mapping will be performed when this option is set. </dd> <dt><samp class=”option”>light</samp></dt> <dd> Set the output content light level information. It accepts 2 space-separated values, the first input is the maximum light level and the second input is the maximum average light level. It is ignored for HDR-to-SDR tone-mapping, and optional for HDR-to-HDR tone-mapping. </dd> </dl>

 <h4 class=”subsection”>42.2.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Convert HDR(HDR10) video to bt2020-transfer-characteristic p010 format <pre class=”example-preformatted”>tonemap_vaapi=format=p010:t=bt2020-10 </pre> Convert HDR video to HDR video <pre class=”example-preformatted”>tonemap_vaapi=display=7500\ 3000|34000\ 16000|13250\ 34500|15635\ 16450|500\ 10000000 </pre>

 <h3 class=”section”>42.3 hstack_vaapi# TOC</h3> Stack input videos horizontally. This is the VA-API variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class=”section”>42.4 vstack_vaapi# TOC</h3> Stack input videos vertically. This is the VA-API variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping

the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">42.5 xstack_vaapi# TOC</h3> Stack video inputs into custom layout. This is the VA-API variant of the xstack filter, each input stream may have different size, this filter will scale down/up each input stream to the given output size, or the size of the first input stream. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_vaapi=inputs=4:layout=0_0_1920x1080|0_1
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h3 class="section">42.6 pad_vaapi# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an

expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var>, and <var class=”var”>y</var> options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> </dl>

 <h3 class="section">42.7 drawbox_vaapi# TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class="code">fill</code> will create a filled box. Default value is <code class="code">3</code>. See below for the list of accepted constants. </dd> <dt><samp class="option">replace</samp></dt> <dd> With value <code class="code">1</code>, the pixels of the painted box will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the box onto the input video. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var class="var">t</var> are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_h, ih</samp></dt> <dt><samp class="option">in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. </dd> </dl>

 <h4 class="subsection">42.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h2 class="chapter">43 VideoToolbox Video Filters# TOC</h2>

Below is a description of the currently available VideoToolbox video filters. VideoToolbox filter depends on VideoToolbox framework, and is auto detected when building ffmpeg for an Apple platform such as macOS. Add <code class="code">--enable-videotoolbox</code> to configure if autodetect is disabled. <h3 class="section">43.1 scale_vt# TOC</h3>

Scale and convert the color parameters using VTPixelTransferSession. The filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. </dd> <dt><samp class="option">color_matrix</samp></dt> <dd> Set the output colorspace matrix. </dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the output color primaries. </dd> <dt><samp class="option">color_transfer</samp></dt> <dd> Set the output transfer characteristics. </dd> </dl>

 <h4 class="subsection">43.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Perform HDR to SDR conversion, and scale to half size of input <pre class="example-

preformatted”>ffmpeg -hwaccel videotoolbox \ -hwaccel_output_format videotoolbox_vld \ -i hdr.mov \ -c:v hevc_videotoolbox \ -profile:v main \ -b:v 3M \ -vf scale_vt=w=iw/2:h=ih/2:color_matrix=bt709:color_primaries=bt709:color_transfer=bt709 \ -c:a copy \ -tag:v hvc1 \ sdr.mp4 </pre>

 <h3 class=”section”>43.2 transpose_vt# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class=”table”> <dt>‘<samp class=”samp”>cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class=”samp”>clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class=”samp”>cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class=”samp”>clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class=”samp”>hflip</samp>’</dt> <dd> Flip the input video horizontally. </dd> <dt>‘<samp class=”samp”>vflip</samp>’</dt> <dd> Flip the input video vertically. </dd> </dl>

</dd> <dt><samp class=”option”>passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class=”samp”>portrait</samp>’</d <dd> Preserve portrait geometry (when <var class=”var”>height</var> >= <var class=”var”>width</var>). </dd> <dt>‘<samp class=”samp”>landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class=”var”>width</var> >= <var class=”var”>height</var>). </dd> </dl>

</dd> </dl>

 <h2 class=”chapter”>44 Vulkan Video Filters# TOC</h2>

Below is a description of the currently available Vulkan video fil-

ters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vulkan</code>, and have a SPIR-V compiler (<code class="code">glslc</code> or <code class="code">glslangValidator</code>) available at build time. Running Vulkan filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device vulkan[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">vulkan</var> called <var class="var">name</var>, using the given device parameters and options in <var class="var">key=value</var>. The following options are supported: <dl class="table"> <dt><samp class="option">debug</samp></dt> <dd> Switches validation layers on if set to 1. </dd> <dt><samp class="option">linear_images</samp></dt> <dd> Allocates linear images. Does not apply to decoding. </dd> <dt><samp class="option">disable_multiplane</samp></dt> <dd> Disables multiplane images. Does not apply to decoding. </dd> <dt><samp class="option">avoid_host_import</samp></dt> <dd> Avoids the use of dynamic host memory imports, in favor of a regular memcpy() into a previously mapped buffer. </dd> </dl>

</dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device and running nlmeans_vulkan filter with default parameters on it. <pre class="example-preformatted">-init_hw_device vulkan=vk:0 -filter_hw_device vk -i INPUT -vf "hwupload,nlmeans_vulkan,hwdownload" OUTPUT
</pre>

As Vulkan filters are not able to access frame data in normal memory, all frame data needs to be uploaded (hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded (hwdownload) back to normal memory. Note that hwupload will upload to a frame with the same layout as the software frame, so it may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it is usually necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class="section">44.1

avgblur_vulkan# TOC</h3>

Apply an average blur filter, implemented on the GPU using Vulkan. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> </dl>

 <h3 class=”section”>44.2 blend_vulkan# TOC</h3>

Blend two Vulkan frames into each other. The <code class=”code”>blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>c0_mode</samp></dt> <dt><samp class=”option”>c1_mode</samp></dt> <dt><samp class=”option”>c2_mode</samp></dt> <dt><samp class=”option”>c3_mode</samp></dt> <dt><samp class=”option”>all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class=”var”>all_mode</var>. Default value is <code class=”code”>normal</code>. Available values for component modes are: <dl class=”table”> <dt>‘<samp class=”samp”>normal</samp>’</dt> <dt>‘<samp class=”samp”>multiply</samp>’
</dl>

</dd> </dl>

 <h3 class=”section”>44.3 bwdif_vulkan# TOC</h3>

Deinterlacer using bwdif, the "Bob Weaver Deinterlacing Filter" algorithm, implemented on the GPU using Vulkan. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0,

send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">44.4 chromaber_vulkan# TOC</h3>

Apply an effect that emulates chromatic aberration. Works best with RGB inputs, but provides a similar effect with YCbCr inputs too. <dl class="table"> <dt><samp class="option">dist_x</samp></dt> <dd> Horizontal displacement multiplier. Each chroma pixel’s position will be multiplied by this amount, starting from the center of the image. Default is <code class="code">0</code>. </dd> <dt><samp class="option">dist_y</samp></dt> <dd> Similarly, this sets the vertical displacement multiplier. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">44.5 color_vulkan# TOC</h3>

Video source that creates a Vulkan frame of a solid color. Useful for benchmarking, or overlaying. It accepts the following parameters: <dl class="table"> <dt><samp class="option">color</samp></dt>

<dd> The color to use. Either a name, or a hexadecimal value. The default value is <code class="code">black</code>. </dd> <dt><samp class="option">size</samp></dt> <dd> The size of the output frame. Default value is <code class="code">1920x1080</code>. </dd> <dt><samp class="option">rate</samp></dt> <dd> The framerate to output at. Default value is <code class="code">60</code> frames per second. </dd> <dt><samp class="option">duration</samp></dt> <dd> The video duration. Default value is <code class="code">-0.000001</code>. </dd> <dt><samp class="option">sar</samp></dt> <dd> The video signal aspect ratio. Default value is <code class="code">1/1</code>. </dd> <dt><samp class="option">format</samp></dt> <dd> The pixel format of the output Vulkan frames. Default value is <code class="code">yuv444p</code>. </dd> <dt><samp class="option">out_range</samp></dt> <dd> Set the output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> </dl>

 <h3 class="section">44.6 vflip_vulkan# TOC</h3>

Flips an image vertically. <h3 class="section">44.7 hflip_vulkan# TOC</h3>

Flips an image horizontally. <h3 class="section">44.8 flip_vulkan# TOC</h3>

Flips an image along both the vertical and horizontal axis. <h3 class="section">44.9 fruc_vulkan# TOC</h3>

Frame rate up-conversion using the NVIDIA Vulkan optical flow extension (<code class="code">VK_NV_optical_flow</code>), implemented on the

GPU using Vulkan. For each pair of consecutive input frames the filter computes a forward and backward optical flow field on the device’s optical flow engine, and uses those fields to synthesise motion-compensated intermediate frames at the requested output frame rate. The frame timing logic mirrors the <code class="code">framerate</code> filter. The majority of YUV and RGB pixel formats are supported. The exceptions are subsampled packed YUV formats such as <code class="code">yuyv422</code> and <code class="code">uyvy422</code>, formats with more than 16 bits integer data per component, such as <code class="code">rgb96</code> and <code class="code">gray32</code>, and Bayer formats. <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> A string describing the desired output frame rate, evaluated as an expression. The following constants are available: <code class="code">source_fps</code>. The default value is <code class="code">60</code>. </dd> <dt><samp class="option">perf</samp></dt> <dd> Optical flow performance level, trading motion-estimation quality for speed. This is the dominant cost at high resolutions. Possible values are: <dl class="table"> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> Highest quality, slowest. This is the default. </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Balanced quality and speed. </dd> <dt>‘<samp class="samp">fast</samp>’</dt> <dd> Lowest quality, fastest. </dd> </dl>

</dd> <dt><samp class="option">grid</samp></dt> <dd> Optical flow output grid size in pixels. A coarser grid computes fewer flow vectors, making it faster and less memory hungry at the cost of flow detail. Possible values are <code class="code">auto</code> (the default; selects the finest grid the device supports, usually 1x1), <code class="code">1</code>, <code class="code">2</code>, <code class="code">4</code> and <code class="code">8</code>. Only grid sizes the device advertises are accepted. </dd> </dl>

 <h3 class="section">44.10 gblur_vulkan# TOC</h3>

Apply Gaussian blur filter on Vulkan frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class="code">sigma</code>. Default is <code class="code">-1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the kernel size along the

horizontal axis. Default is <code class="code">19</code>. </dd> <dt><samp class="option">sizeV</samp></dt> <dd> Set the kernel size along the vertical axis. Default is <code class="code">0</code>, which sets to use the same value as <var class="var">size</var>. </dd> </dl>

 <h3 class="section">44.11 nlmeans_vulkan# TOC</h3>

Denoise frames using Non-Local Means algorithm, implemented on the GPU using Vulkan. Supports more pixel formats than nlmeans or nlmeans_opencl, including alpha channel support. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength for all components. Default is 1.0. Must be in range [0.0, 100.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size for all planes. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">t</samp></dt> <dd> Set parallelism. Default is 8. Must be a number in the range [1, 64]. Larger values will use more VRAM but may not result in greater speed. The optimal value is hardware and input dependent. </dd> <dt><samp class="option">s0</samp></dt> <dt><samp class="option">s1</samp></dt> <dt><samp class="option">s2</samp></dt> <dt><samp class="option">s3</samp></dt> <dd> Set denoising strength for a specific component. Default is <var class="var">1.0</var>, equal to <samp class="option">s</samp>. Must be in range [0.0, 100.0]. 0.0 disables denoising in that component. </dd> <dt><samp class="option">p0</samp></dt> <dt><samp class="option">p1</samp></dt> <dt><samp class="option">p2</samp></dt> <dt><samp class="option">p3</samp></dt> <dd> Set patch size for a specific component. Default is <var class="var">7</var>, equal to <samp class="option">p</samp>. Must be odd number in range [0, 99]. </dd> </dl>

 <h3 class="section">44.12 overlay_vulkan# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires all inputs to use the same pixel format. So, format conversion may be needed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set the x coordinate of the over-

laid video on the main video. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">44.13 transpose_vulkan# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">44.14 v360_vulkan# TOC</h3>

Convert 360 videos between various formats. Supported formats (projections) are subset of v360 filter, that are most popular 360-degree video formats. The filter accepts subset of v360 options: <dl class="table"> <dt><samp class="option">input</samp></dt>

<dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">eac</samp>’</dt> <dd> Equi-Angular Cube-map. (Output only) FIXME: current output doesn’t contain padding. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Regular video. </dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt> <dd> Dual fisheye. </dd> <dt>‘<samp class="samp">sg</samp>’</dt> <dd> Stereographic format. </dd> <dt>‘<samp class="samp">fisheye</samp>’</dt> <dd> Fisheye projection. </dd> <dt>‘<samp class="samp">gopromax</samp>’</dt> <dd> GoPro Max .360 format. (Input only) A .360 video file contains two video streams (front and rear); with this input projection the filter takes two inputs and combines them into a single stream. See the <samp class="option">overlap</samp> option. </dd> </dl>

</dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video resolution. See v360 options. </dd> <dt><samp class="option">yaw</samp></dt> <dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt> <dt><samp class="option">rorder</samp></dt> <dd> Set rotation for the output video. See v360 options. </dd> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dd> Set output horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt> <dd> Set input horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number of overlapped pixels on input .360 video. Only used with <code class="code">input=gopromax</code>. The GoPro Max .360 video contains overlapped area (See https://gopro.com/news/max-tech-specs-stitching-resolution). The filter blends overlapped images in these two areas. The number of pixels of overlapped areas depends on the video resolution. In most cases, the value is automatically calculated based on input video size, but it may be changed on GoPro Max2 or future models. The option is to adjust for unknown video resolutions. </dd> </dl>

 <h4 class="subsection">44.14.1 Examples# TOC</h4> Special

notes exist to use with <code class=”code”>input=gopromax</code>. The GoPro Max .360 video file contains two video streams, and the stream number must be specified. The stream number (e.g., <code class=”code”>[0:5]</code>) depends on the model and shooting mode. Please check actual stream number with <code class=”code”>ffprobe</code> command. <ul class=”itemize mark-bullet”> Convert GoPro Max (8bit color depth) .360 to Equirectangular projection. GoPro Max uses obsolete yuvj420p pixel format, thus convert the format to yuv420p before this filter. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]format=yuv420p,hwupload[vf],[0:5]format=yuv420p,hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=e, hwdownload,format=yuv420p" OUTPUT.mkv </pre>

Convert GoPro Max2 (10bit color depth) .360 to Dual fisheye projection. GoPro Max2 uses yuv420p10le pixel format. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]hwupload[vf],[0:4]hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=dfisheye, hwdownload,format=yuv420p10le" OUTPUT.mkv </pre>

 <h2 class=”chapter”>45 QSV Video Filters# TOC</h2>

Below is a description of the currently available QSV video filters. To enable compilation of these filters you need to configure FFmpeg with <code class=”code”>--enable-libmfx</code> or <code class=”code”>--enable-libvpl</code>. To use QSV filters, you need to setup the QSV device correctly. For more information, please read <a class=”url” href=”https://trac.ffmpeg.org/wiki/Hardware/QuickSync”>https://trac.ffmpeg.org/wiki/Hardware/QuickSy
  <h3 class=”section”>45.1 hstack_qsv# TOC</h3> Stack input videos horizontally. This is the QSV variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">45.2 vstack_qsv# TOC</h3> Stack input videos vertically. This is the QSV variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">45.3 xstack_qsv# TOC</h3> Stack video inputs into custom layout. This is the QSV variant of the xstack filter. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_qsv=inputs=4:layout=0_0_1920x1080|0_h0_1920x1080|w0_0_1920
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h2 class="chapter">46 Video Sources# TOC</h2>

Below is a description of the currently available video sources. <h3 class="section">46.1 buffer# TOC</h3>

Buffer video frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">width</samp></dt> <dd> The input video width. </dd> <dt><samp class="option">height</samp></dt> <dd> The input video height. </dd> <dt><samp class="option">pix_fmt</samp></dt> <dd> A string representing the pixel format of the buffered video frames. It may be a number corresponding to a pixel format, or a pixel format name. </dd> <dt><samp class="option">time_base</samp></dt> <dd> Specify the timebase assumed by the timestamps of the buffered frames. </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Specify the frame rate expected for the video stream. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> A string representing the color space of the buffered video frames. It may be a number corresponding to a color space, or a color space name. </dd> <dt><samp class="option">range</samp></dt> <dd> A string representing the color range of the buffered video frames. It may be a number corresponding to a color range, or a color range name. </dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> A string representing the alpha mode of the buffered video frames. It may be a number corresponding to an alpha mode, or an alpha mode name. </dd> <dt><samp class="option">chroma_location</samp></dt> <dd> A string representing the chroma sample location of the buffered video frames. It may be a number corresponding to a chroma sample location, or a chroma sample location name. When it is not set, the filtergraph chooses the chroma sample location of the output. Frames without a chroma sample location are tagged with the one negotiated for the output. </dd> <dt><samp class="option">pixel_aspect, sar</samp></dt> <dd> The sample (pixel) aspect ratio of the input video. </dd> <dt><samp class="option">hw_frames_ctx</samp></dt> <dd> When using a hardware pixel format, this should be a reference to an AVHWFramesContext describing input frames. </dd> </dl>

For example: <pre class="example-preformatted">buffer=width=320:height=240:pix_fmt=yuv410p:time_base=1/24:sar=1
</pre>

will instruct the source to accept video frames with size 320x240 and with format "yuv410p", assuming 1/24 as the timestamps timebase and square pixels (1:1 sample aspect ratio). Since the pixel format with name "yuv410p" corresponds to the number 6 (check the enum AVPixelFormat definition in <samp class="file">libavutil/pixfmt.h</samp>), this example corresponds to: <pre class="example-preformatted">buffer=size=320x240:pixfmt=6:time_base=1/24:pixel_aspect=1/1
</pre>

Alternatively, the options can be specified as a flat string, but this syntax is deprecated: <var class="var">width</var>:<var class="var">height</var>:<var class="var">pix_fmt</var>:<var class="var">time_base.num</var>:<var class="var">time_base.den</var>:<var class="var">pixel_aspect.num</var>:<var class="var">pixel_aspect.den</var> <h3 class="section">46.2 cellauto# TOC</h3>

Create a pattern generated by an elementary cellular automaton. The initial state of the cellular automaton can be defined through the <samp class="option">filename</samp> and <samp class="option">pattern</samp> options. If such options are not specified an initial state is created randomly. At each new frame a new row in the video is filled with the result of the cellular automaton next generation. The behavior when the whole frame is filled is defined by the <samp class="option">scroll</samp> option. This source accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified file. In the file, each non-whitespace character is considered an alive cell, a newline will terminate the row, and further characters in the file will be ignored. </dd> <dt><samp class="option">pattern, p</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified string. Each non-whitespace character in the string is considered an alive cell, a newline will terminate the row, and further characters in the string will be ignored. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set the video rate, that is the number of frames generated per second. Default is 25. </dd> <dt><samp class="option">random_fill_ratio, ratio</samp></dt> <dd> Set the random fill ratio for the initial cellular automaton row. It is a floating point number value ranging from 0 to 1, defaults to 1/PHI. This option is ignored when a file or a pattern is specified. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> Set the seed for filling randomly the initial row, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">rule</samp></dt> <dd> Set the cellular automaton rule, it is a number ranging from 0 to

255. Default value is 110. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the size of the output video. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. If <samp class="option">filename</samp> or <samp class="option">pattern</samp> is specified, the size is set by default to the width of the specified initial state row, and the height is set to <var class="var">width</var> * PHI. If <samp class="option">size</samp> is set, it must contain the width of the specified pattern string, and the specified pattern will be centered in the larger row. If a filename or a pattern string is not specified, the size value defaults to "320x518" (used for a randomly generated initial state). </dd> <dt><samp class="option">scroll</samp></dt> <dd> If set to 1, scroll the output upward when all the rows in the output have been already filled. If set to 0, the new generated row will be written over the top row just after the bottom row is filled. Defaults to 1. </dd> <dt><samp class="option">start_full, full</samp></dt> <dd> If set to 1, completely fill the output with generated rows before outputting the first frame. This is the default behavior, for disabling set the value to 0. </dd> <dt><samp class="option">stitch</samp></dt> <dd> If set to 1, stitch the left and right row edges together. This is the default behavior, for disabling set the value to 0. </dd> </dl>

 <h4 class="subsection">46.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read the initial state from <samp class="file">pattern</samp>, and specify an output of size 200x400. <pre class="example-preformatted">cellauto=f=pattern:s=200x400 </pre>

Generate a random initial row with a width of 200 cells, with a fill ratio of 2/3: <pre class="example-preformatted">cellauto=ratio=2/3:s=200x200 </pre>

Create a pattern generated by rule 18 starting by a single alive cell centered on an initial row with width 100: <pre class="example-preformatted">cellauto=p=@:s=100x400:full=0:rule=18 </pre>

Specify a more elaborated initial pattern: <pre class="example-preformatted">cellauto=p='@@ @ @@':s=100x400:full=0:rule=18 </pre>

<a name="coreimagesrc-

1”> <h3 class=”section”>46.3 coreimagesrc# TOC</h3> Video source generated on GPU using Apple’s CoreImage API on OSX. This video source is a specialized version of the coreimage video filter. Use a core image generator at the beginning of the applied filterchain to generate the content. The coreimagesrc video source accepts the following options: <dl class=”table”> <dt><samp class=”option”>list_generators</samp></dt> <dd> List all available generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class=”example-preformatted”>list_generators=true </pre>

</dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the size of the sourced video. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. The default value is <code class=”code”>320x240</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> </dl>

Additionally, all options of the coreimage video filter are accepted. A complete filterchain can be used for further processing of the generated input without CPU-HOST transfer. See coreimage documentation and examples for details. <h4 class=”subsection”>46.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class=”example-preformatted”>ffmpeg -f lavfi -i coreimagesrc=s=100x100:filter=CIQRCodeGenerator@inputMessage=https\\\\\://FFmpeg.org/@inputCorrecti

-frames:v 1 QRCode.png </pre> This example is equivalent to the QRCode example of coreimage without the need for a nullsrc video source.

 <h3 class=”section”>46.4 ddagrab# TOC</h3>

Captures the Windows Desktop via Desktop Duplication API. The filter exclusively returns D3D11 Hardware Frames, for on-gpu encoding or processing. So an explicit hwdownload is needed for any kind of software processing. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>output_idx</samp></dt> <dd> DXGI Output Index to capture. Usually corresponds to the index Windows has given the screen minus one, so it’s starting at 0. Defaults to output 0. </dd> <dt><samp class=”option”>draw_mouse</samp></dt> <dd> Whether to draw the mouse cursor. Defaults to true. Only affects hardware cursors. If a game or application renders its own cursor, it’ll always be captured. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Maximum framerate at which the desktop will be captured - the interval between successive frames will not be smaller than the inverse of the framerate. When <var class=”var”>dup_frames</var> is true (the default) and the desktop is not being updated often enough, the filter will duplicate a previous frame. Note that there is no background buffering going on, so when the filter is not polled often enough then the actual inter-frame interval may be significantly larger. Defaults to 30 FPS. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Specify the size of the captured video. Defaults to the full size of the screen. Cropped from the bottom/right if smaller than screen size. </dd> <dt><samp class=”option”>offset_x</samp></dt> <dd> Horizontal offset of the captured video. </dd> <dt><samp class=”option”>offset_y</samp></dt> <dd> Vertical offset of the captured video. </dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired filter output format. Defaults to 8 Bit BGRA. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Passes all supported output formats to DDA and returns what DDA decides to use. </dd> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8 Bit formats always work, and DDA will convert to them if necessary. </dd> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dd> Filter initialization will fail if 10 bit format is requested but unavailable. </dd> </dl>

</dd> <dt><samp class="option">dup_frames</samp></dt> <dd> When this option is set to true (the default), the filter will duplicate frames when the desktop has not been updated in order to maintain approximately constant target framerate. When this option is set to false, the filter will wait for the desktop to be updated (inter-frame intervals may vary significantly in this case). </dd> </dl>

 <h4 class="subsection">46.4.1 Examples# TOC</h4>

Capture primary screen and encode using nvenc: <pre class="example-preformatted">ffmpeg -f lavfi -i ddagrab -c:v h264_nvenc -cq 18 output.mp4 </pre>

You can also skip the lavfi device and directly use the filter. Also demonstrates downloading the frame and encoding with libx264. Explicit output format specification is required in this case: <pre class="example-preformatted">ffmpeg -filter_complex ddagrab=output_idx=1:framerate=60,hwdownload,fo -c:v libx264 -crf 18 output.mp4 </pre>

If you want to capture only a subsection of the desktop, this can be achieved by specifying a smaller size and its offsets into the screen: <pre class="example-preformatted">ddagrab=video_size=800x600:offset_x=100:offset_y </pre>

 <h3 class="section">46.5 gfxcapture# TOC</h3>

Capture windows or monitors using Windows.Graphics.Capture API.

This source provides low overhead capture of application windows or entire monitors.

The filter outputs hardware frames in <code class="code">d3d11</code> format; use <code class="code">hwdownload,format=</code>

if system memory frames are required.

The window to be captured can be selected via regular expressions on its title,

class name or backing executable name, by explicit native handles, or by monitor

index or explicit native handle. A window must match all provided expressions to be

selected. The first matching window will be picked, in whatever order Windows

returns them.

Explicit handles (<samp class="option">hwnd</samp>, <samp class="option">hmonitor</samp>) override pattern or index

based selection. If neither handles nor a monitor index are given, the first

window matching the provided regular expressions is captured.

This source does NOT hold a stable FPS. It returns frames at whatever rate the compositor

provides them, only capped by the <samp class="option">max_framerate</samp>.

If you need a stable rate, you need to add an fps filter to drop/duplicate frames as needed.

Note that a static source may not deliver any frames at all, which also stalls

the fps filter; see <samp class="option">dup_frames</samp>.

If the capture source disappears mid-capture (window closed, monitor disconnected), the filter will return EOF.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">window_title</samp></dt>

<dd> ECMAScript regular expression matched against the window title. Supports a

PCRE style <code class="code">(?i)</code> prefix for case-insensitive matching.

</dd>

<dt><samp class="option">window_class</samp></dt>

<dd> As <samp class="option">window_title</samp>, but matched against the window class name.

</dd>

<dt><samp class=”option”>window_exe</samp></dt>

<dd> As <samp class=”option”>window_title</samp>, but matched against the executable file name of the

window’s process.

</dd>

<dt><samp class=”option”>monitor_idx</samp></dt>

<dd> Zero-based index of the monitor to capture.

Can also be set to <code class=”code”>window</code> to capture the monitor the selected window

is displayed on at filter initialization time.

</dd>

<dt><samp class=”option”>hwnd</samp></dt>

<dd> Explicit native window handle (HWND).

</dd>

<dt><samp class=”option”>hmonitor</samp></dt>

<dd> Explicit native monitor handle (HMONITOR).

</dd>

<dt><samp class=”option”>capture_cursor</samp></dt>

<dd> Capture the mouse cursor. Enabled by default.

</dd>

<dt><samp class=”option”>capture_border</samp></dt>

<dd> Capture the full area of the window, including its window decorarions/border.

Disabled by default.

</dd>

<dt><samp class="option">display_border</samp></dt>

<dd> Draw a yellow highlight border around the captured window. Disabled by default.

</dd>

<dt><samp class="option">max_framerate</samp></dt>

<dd> Maximum capture frame rate. Accepts a video rate (e.g. <code class="code">30</code>, <code class="code">60/1</code>,

<code class="code">24000/1001</code>). The default is <code class="code">60</code> FPS.

The actual rate is the rate at which the compositor renders the window/monitor,

capped by this option.

</dd>

<dt><samp class="option">width</samp></dt>

<dd> Force the output canvas width. If zero (default) the initial captured source

width is used. If a negative number is provided, the width will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">height</samp></dt>

<dd> Force the output canvas height. If zero (default) the initial captured source

height is used. If a negative number is provided, the height will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">crop_left</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_top</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_right</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_bottom</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">premultiplied</samp></dt>

<dd> If set to 1, return frames with premultiplied alpha. Default is 0 (straight

alpha).

</dd>

<dt><samp class="option">dup_frames</samp></dt>

<dd> If enabled (the default), the last frame is output again whenever the source

delivered no new frame within one <samp class="option">max_framerate</samp> interval, and black

frames are output until the source delivered its first frame. This keeps the

output flowing while the captured content is static. The output rate is still

not constant, add an <code class="code">fps</code> filter if needed.

</dd>

<dt><samp class=”option”>resize_mode</samp></dt>

<dd> Defines how the captured content is fitted into the output canvas size.

Possible values:

<dl class=”table”>

<dt>‘<samp class=”samp”>crop</samp>’</dt>

<dd> Crop (or pad with black) to the canvas size. (default)

</dd>

<dt>‘<samp class=”samp”>scale</samp>’</dt>

<dd> Scale the source to fill the canvas, potentially altering aspect ratio.

</dd>

<dt>‘<samp class=”samp”>scale_aspect</samp>’</dt>

<dd> Scale the source to fit inside the canvas while preserving aspect ratio.

Remaining area is filled with black.

</dd>

</dl>

</dd> <dt><samp class=”option”>scale_mode</samp></dt> <dd> Scaling algorithm used when resizing is required. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Nearest neighbour (pixelated) scaling. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear filtering. (default) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> Bicubic filtering. Potentially more blurry, but fewer scaling artifacts depending on contents. </dd> </dl>

</dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired output pixel format inside the D3D11 hardware frames. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dd> 8 bit BGRA output (default) </dd> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dd> 10 bit BGR output </dd> <dt>‘<samp class=”samp”>rgbaf16</samp>’</dt> <dt>‘<samp class=”samp”>16bit</samp>’</dt> <dd> 16bit float RGBA output </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>46.5.1 Examples# TOC</h4> <ul class=”itemize

mark-bullet”> Capture a window by title (case-insensitive) at a maximum of 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_title='(?i)My Application':max_framerate=60,hwdownload,format=bgra,format=yuv420p -c:v libx264 -crf 15 capture.mp4 </pre>

Capture monitor 1 at native refresh, 10bit color depth, scale to 1920x1080 preserving aspect: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=monitor_idx=1:width=1920:height=1080:resize_mode=sc -c:v hevc_nvenc -cq 15 capture.mp4 </pre>

Capture a window by executable name, draw border, force point scaling, fixed 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_exe='^firefox.exe$':display_border=1:scale_mod -rc qvbr -qvbr_quality_level 15 -c:v h264_amf capture.mp4 </pre>

 <h3 class=”section”>46.6 gradients# TOC</h3> Generate several gradients. <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>c0, c1, c2, c3, c4, c5, c6, c7</samp></dt> <dd> Set 8 colors. Default values for colors is to pick random one. </dd> <dt><samp class=”option”>x0, y0, x1, y1</samp></dt> <dd> Set gradient line source and destination points. If negative or out of range, random ones are picked. </dd> <dt><samp class=”option”>nb_colors, n</samp></dt> <dd> Set number of colors to use at once. Allowed range is from 2 to 8. Default value is 2. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed for picking gradient line points. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set speed of gradients rotation. </dd> <dt><samp class=”option”>type, t</samp></dt> <dd> Set type of gradients. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>radial</samp>’</d

<dt>‘<samp class="samp">circular</samp>’</dt> <dt>‘<samp class="samp">spiral</samp>’</dt> <dt>‘<samp class="samp">square</samp>’</
</dl>

Default type is <var class="var">linear</var>. </dd> </dl>

 <h4 class="subsection">46.6.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">46.7 mandelbrot# TOC</h3>

Generate a Mandelbrot set fractal, and progressively zoom towards the point specified with <var class="var">start_x</var> and <var class="var">start_y</var>. This source accepts the following options: <dl class="table"> <dt><samp class="option">end_pts</samp></dt> <dd> Set the terminal pts value. Default value is 400. </dd> <dt><samp class="option">end_scale</samp></dt> <dd> Set the terminal scale value. Must be a floating point value. Default value is 0.3. </dd> <dt><samp class="option">inner</samp></dt> <dd> Set the inner coloring mode, that is the algorithm used to draw the Mandelbrot fractal internal region. It shall assume one of the following values: <dl class="table"> <dt><samp class="option">black</samp></dt> <dd> Set black mode. </dd> <dt><samp class="option">convergence</samp></dt> <dd> Show time until convergence. </dd> <dt><samp class="option">mincol</samp></dt> <dd> Set color based on point closest to the origin of the iterations. </dd> <dt><samp class="option">period</samp></dt> <dd> Set period mode. </dd> </dl>

Default value is <var class="var">mincol</var>. </dd> <dt><samp class="option">bailout</samp></dt> <dd> Set the bailout value. Default value is 10.0. </dd> <dt><samp class="option">maxiter</samp></dt> <dd> Set the maximum of iterations performed by the rendering algorithm. Default value is 7189. </dd> <dt><samp class="option">outer</samp></dt> <dd> Set outer coloring mode. It shall assume one of following values: <dl class="table"> <dt><samp class="option">iteration_count</samp></dt> <dd> Set iteration count mode. </dd> <dt><samp class="option">normalized_iteration_count</samp></dt> <dd> set normalized iteration count mode. </dd> </dl> Default value is <var class="var">normalized_iteration_count</var>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value

is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">start_scale</samp></dt> <dd> Set the initial scale value. Default value is 3.0. </dd> <dt><samp class="option">start_x</samp></dt> <dd> Set the initial x position. Must be a floating point value between -100 and 100. Default value is -0.74364388703715870475219150611477 4. </dd> <dt><samp class="option">start_y</samp></dt> <dd> Set the initial y position. Must be a floating point value between -100 and 100. Default value is -0.13182590420531197049313205638513 9. </dd> </dl>

 <h3 class="section">46.8 mptestsrc# TOC</h3>

Generate various test patterns, as generated by the MPlayer test filter. The size of the generated video is fixed, and is 512x512. This source is useful in particular for testing encoding features. This source accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class="option">test, t</samp></dt> <dd> Set the number or the name of the test to perform. Supported tests are: <dl class="table"> <dt><samp class="option">dc_luma</samp></dt> <dt><samp class="option">dc_chroma</samp></dt> <dt><samp class="option">freq_luma</samp></dt> <dt><samp class="option">freq_chroma</samp></dt> <dt><samp class="option">amp_luma</samp></dt> <dt><samp class="option">amp_chroma</samp></dt> <dt><samp class="option">cbp</samp></dt> <dt><samp class="option">mv</samp></dt> <dt><samp class="option">ring1</samp></dt> <dt><samp class="option">ring2</samp></dt> <dt><samp class="option">all</samp></dt> <dt><samp class="option">max_frames, m</samp></dt> <dd> Set the maximum number of frames generated for each test, default value is 30. </dd> </dl>

Default value is "all", which will cycle through the list of all

tests. </dd> </dl>

Some examples: <pre class=”example-preformatted”>mptestsrc=t=dc_luma </pre>

will generate a "dc_luma" test pattern. <h3 class=”section”>46.9 frei0r_src# TOC</h3>

Provide a frei0r source. To enable compilation of this filter you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. This source accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>size</samp></dt> <dd> The size of the video to generate. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> The framerate of the generated video. It may be a string of the form <var class=”var”>num</var>/<var class=”var”>den</var> or a frame rate abbreviation. </dd> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name to the frei0r source to load. For more information regarding frei0r and how to set the parameters, read the frei0r section in the video filters documentation. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r source. </dd> </dl>

For example, to generate a frei0r partik0l source with size 200x200 and frame rate 10 which is overlaid on the overlay filter main input: <pre class=”example-preformatted”>frei0r_src=size=200x200:framerate=10:filter_name=
[overlay]; [in][overlay] overlay </pre>

 <h3 class=”section”>46.10 life# TOC</h3>

Generate a life pattern.

This source is based on a generalization of John Conway’s life game.

The sourced input represents a life grid, each pixel represents a cell

which can be in one of two possible states, alive or dead. Every cell

interacts with its eight neighbours, which are the cells that are

horizontally, vertically, or diagonally adjacent.

At each interaction the grid evolves according to the adopted rule,

which specifies the number of neighbor alive cells which will make a

cell stay alive or born. The <samp class="option">rule</samp> option allows one to specify

the rule to adopt.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">filename, f</samp></dt>

<dd> Set the file from which to read the initial grid state. In the file,

each non-whitespace character is considered an alive cell, and newline

is used to delimit the end of each row.

If this option is not specified, the initial grid is generated

randomly.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the video rate, that is the number of frames generated per second.

Default is 25.

</dd>

<dt><samp class="option">random_fill_ratio, ratio</samp></dt>

<dd> Set the random fill ratio for the initial random grid. It is a

floating point number value ranging from 0 to 1, defaults to 1/PHI.

It is ignored when a file is specified.

</dd>

<dt><samp class=”option”>random_seed, seed</samp></dt>

<dd> Set the seed for filling the initial random grid, must be an integer

included between 0 and UINT32_MAX. If not specified, or if explicitly

set to -1, the filter will try to use a good random seed on a best

effort basis.

</dd>

<dt><samp class=”option”>rule</samp></dt>

<dd> Set the life rule.

A rule can be specified with a code of the kind "S<var class=”var”>NS</var>/B<var class=”var”>NB</var>",

where <var class=”var”>NS</var> and <var class=”var”>NB</var> are sequences of numbers in the range 0-8,

<var class=”var”>NS</var> specifies the number of alive neighbor cells which make a

live cell stay alive, and <var class=”var”>NB</var> the number of alive neighbor cells

which make a dead cell to become alive (i.e. to "born").

"s" and "b" can be used in place of "S" and "B", respectively.

Alternatively a rule can be specified by an 18-bits integer. The 9

high order bits are used to encode the next cell state if it is alive

for each number of neighbor alive cells, the low order bits specify

the rule for "borning" new cells. Higher order bits encode for an

higher number of neighbor cells.

For example the number 6153 = <code class=”code”>(12<<9)+9</code> specifies a stay alive

rule of 12 and a born rule of 9, which corresponds to "S23/B03".

Default value is "S23/B3", which is the original Conway’s game of life

rule, and will keep a cell alive if it has 2 or 3 neighbor alive

cells, and will born a new cell if there are three alive cells around

a dead cell.

</dd>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Set the size of the output video. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

If <samp class=”option”>filename</samp> is specified, the size is set by default to the

same size of the input file. If <samp class=”option”>size</samp> is set, it must contain

the size specified in the input file, and the initial grid defined in

that file is centered in the larger resulting area.

If a filename is not specified, the size value defaults to "320x240"

(used for a randomly generated initial grid).

</dd>

<dt><samp class=”option”>stitch</samp></dt>

<dd> If set to 1, stitch the left and right grid edges together, and the

top and bottom edges also. Defaults to 1.

</dd>

<dt><samp class=”option”>mold</samp></dt>

<dd> Set cell mold speed. If set, a dead cell will go from <samp class=”option”>death_color</samp> to

<samp class=”option”>mold_color</samp> with a step of <samp class=”option”>mold</samp>. <samp class=”option”>mold</samp> can have a

value from 0 to 255.

</dd>

<dt><samp class="option">life_color</samp></dt>

<dd> Set the color of living (or new born) cells.

</dd>

<dt><samp class="option">death_color</samp></dt>

<dd> Set the color of dead cells. If <samp class="option">mold</samp> is set, this is the first color

used to represent a dead cell.

</dd>

<dt><samp class="option">mold_color</samp></dt>

<dd> Set mold color, for definitely dead and moldy cells.

For the syntax of these 3 color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the

ffmpeg-utils manual.

</dd>

</dl>

 <h4 class="subsection">46.10.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read a grid from <samp class="file">pattern</samp>, and center it on a grid of size 300x300 pixels: <pre class="example-preformatted">life=f=pattern:s=300x300 </pre>

Generate a random grid of size 200x200, with a fill ratio of 2/3: <pre class="example-preformatted">life=ratio=2/3:s=200x200 </pre>

Specify a custom rule for evolving a randomly generated grid: <pre class="example-preformatted">life=rule=S14/B34 </pre>

Full example with slow death effect (mold) using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi life=s=300x200:mold=10:r=60:ratio=0.1:death_color=#C83232:life_color=#00f
</pre>

 <h3 class="section">46.11 perlin#
TOC</h3>
Generate Perlin noise. Perlin noise is a kind of noise with local continuity in space. This can be used to generate patterns with continuity in space and time, e.g. to simulate smoke, fluids, or terrain. In case more than one octave is specified through the <samp class="option">octaves</samp> option, Perlin noise is generated as a sum of components, each one with doubled frequency. In this case the <samp class="option">persistence</samp> option specify the ratio of the amplitude with respect to the previous component. More octave components enable to specify more high frequency details in the generated noise (e.g. small size variations due to boulders in a generated terrain). <h4 class="subsection">46.11.1 Options# TOC</h4> <dl class="table">
<dt><samp class="option">size, s</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">320x240</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate expected for the video stream, expressed as a number of frames per second. Default value is <code class="code">25</code>. </dd>
<dt><samp class="option">octaves</samp></dt> <dd> Specify the total number of components making up the noise, each one with doubled frequency. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">persistence</samp></dt>
<dd> Set the ratio used to compute the amplitude of the next octave component with respect to the previous component amplitude. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">xscale</samp></dt> <dt><samp class="option">yscale</samp></dt>
<dd> Define a scale factor used to multiple the x, y coordinates. This can be useful to define an effect with a pattern stretched along the x or y axis. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">tscale</samp></dt> <dd> Define a scale factor used to multiple the time coordinate. This can be useful to change the time variation speed. Default value is <code class="code">1</code>. </dd>
<dt><samp class="option">random_mode</samp></dt> <dd> Set random mode used to compute initial pattern. Supported values are:

<dl class="table"> <dt><samp class="option">random</samp></dt> <dd> Compute and use random seed. </dd> <dt><samp class="option">ken</samp></dt> <dd> Use the predefined initial pattern defined by Ken Perlin in the original article, can be useful to compare the output with other sources. </dd> <dt><samp class="option">seed</samp></dt> <dd> Use the value specified by <samp class="option">random_seed</samp> option. </dd> </dl>

Default value is <code class="code">random</code>. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> When <samp class="option">random_mode</samp> is set to <var class="var">random_seed</var>, use this value to compute the initial pattern. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">46.11.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Generate single component: <pre class="example-preformatted">perlin </pre>

Use Perlin noise with 7 components, each one with a halved contribution to total amplitude: <pre class="example-preformatted">perlin=octaves=7:persistence=0.5 </pre>

Chain Perlin noise with the lutyuv to generate a black&white effect: <pre class="example-preformatted">perlin=octaves=3:tscale=0.3,lutyuv=y='if(lt(val\,128)\,255\,0)' </pre>

Stretch noise along the y axis, and convert gray level to red-only signal: <pre class="example-preformatted">perlin=octaves=7:tscale=0.4:yscale=0.3,lut </pre>

 <h3 class="section">46.12 qrencodesrc# TOC</h3>

Generate a QR code using the libqrencode library (see https://fukuchi.org/works/qrencode/). To enable the compilation of this source, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and put in the video output according to the specified output size options. In case no text is specified, the QR code is not generated, but an empty colored output is returned instead. This source accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp

class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated only once, when initializing the source. See the qrencode Expressions section for details. Note that some of the constants are missing for the source (for example the <var class=”var”>x</var> or <var class=”var”>t</var> or ¸<var class=”var”>n</var>), since they only makes sense when evaluating the expression for each frame rather than at initialization time. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is "white". For the syntax of the color options,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </p></dd> </dl>

 <h4 class="subsection">46.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:bc=pink:fc=blue </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencodesrc=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencodesrc=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencodesrc=text=%{pts} </pre>

 <h3 class="section">46.13

allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc# TOC</h3>

The <code class="code">allrgb</code> source returns frames of size 4096x4096 of all rgb colors.

The <code class="code">allyuv</code> source returns frames of size 4096x4096 of all yuv colors.

The <code class="code">color</code> source provides an uniformly colored input.

The <code class="code">colorchart</code> source provides a colors checker chart.

The <code class="code">colorspectrum</code> source provides a color spectrum input.

The <code class="code">haldclutsrc</code> source provides an identity Hald CLUT. See also

haldclut filter.

The <code class="code">nullsrc</code> source returns unprocessed video frames. It is

mainly useful to be employed in analysis / debugging tools, or as the

source for filters which ignore the input data.

The <code class="code">pal75bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 75% color levels.

The <code class=”code”>pal100bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 100% color levels.

The <code class=”code”>rgbtestsrc</code> source generates an RGB test pattern useful for

detecting RGB vs BGR issues. You should see a red, green and blue

stripe from top to bottom.

The <code class=”code”>smptebars</code> source generates a color bars pattern, based on

the SMPTE Engineering Guideline EG 1-1990.

The <code class=”code”>smptehdbars</code> source generates a color bars pattern, based on

the SMPTE RP 219-2002.

The <code class=”code”>testsrc</code> source generates a test video pattern, showing a

color pattern, a scrolling gradient and a timestamp. This is mainly

intended for testing purposes.

The <code class=”code”>testsrc2</code> source is similar to testsrc, but supports more

pixel formats instead of just <code class=”code”>rgb24</code>. This allows using it as an

input for other tests without requiring a format conversion.

The <code class=”code”>yuvtestsrc</code> source generates an YUV test pattern. You should

see a y, cb and cr stripe from top to bottom.

The sources accept the following parameters:

<dl class="table">

<dt><samp class="option">level</samp></dt>

<dd> Specify the level of the Hald CLUT, only available in the <code class="code">haldclutsrc</code>

source. A level of <code class="code">N</code> generates a picture of <code class="code">N*N*N</code> by <code class="code">N*N*N</code>

pixels to be used as identity matrix for 3D lookup tables. Each component is

coded on a <code class="code">1/(N*N)</code> scale.

</dd>

<dt><samp class="option">color, c</samp></dt>

<dd> Specify the color of the source, only available in the <code class="code">color</code>

source. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the size of the sourced video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

The default value is <code class="code">320x240</code>.

This option is not available with the <code class="code">allrgb</code>, <code class="code">allyuv</code>, and

<code class="code">haldclutsrc</code> filters.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Specify the frame rate of the sourced video, as the number of frames

generated per second. It has to be a string in the format

<var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point

number or a valid video frame rate abbreviation. The default value is

"25".

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Set the duration of the sourced video. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If not specified, or the expressed duration is negative, the video is

supposed to be generated forever.

Since the frame rate is used as time base, all frames including the last one

will have their full duration. If the specified duration is not a multiple

of the frame duration, it will be rounded up.

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> Set the sample aspect ratio of the sourced video.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Specify the alpha (opacity) of the background, only available in the

<code class="code">testsrc2</code> source. The value must be between 0 (fully transparent) and

255 (fully opaque, the default).

</dd>

<dt><samp class=”option”>decimals, n</samp></dt>

<dd> Set the number of decimals to show in the timestamp, only available in the

<code class=”code”>testsrc</code> source.

The displayed timestamp value will correspond to the original

timestamp value multiplied by the power of 10 of the specified

value. Default value is 0.

</dd>

<dt><samp class=”option”>type</samp></dt>

<dd> Set the type of the color spectrum, only available in the

<code class=”code”>colorspectrum</code> source. Can be one of the following:

<dl class=”table”>

<dt>‘<samp class=”samp”>black</samp>’</dt>

<dt>‘<samp class=”samp”>white</samp>’</dt>

<dt>‘<samp class=”samp”>all</samp>’</dt>

</dl>

</dd> <dt><samp class=”option”>patch_size</samp></dt> <dd> Set patch size of single color patch, only available in the <code class=”code”>colorchart</code> source. Default is <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>preset</samp></dt> <dd> Set colorchecker colors preset, only available in the <code class=”code”>colorchart</code> source. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>reference</samp>’</dt> <dt>‘<samp class=”samp”>skintones</samp>’</dt> </dl>

Default value is <code class=”code”>reference</code>. </dd> </dl>

 <h4 class="subsection">46.13.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a video with a duration of 5.3 seconds, with size 176x144 and a frame rate of 10 frames per second: <pre class="example-preformatted">testsrc=duration=5.3:size=qcif:rate=10 </pre>

The following graph description will generate a red source with an opacity of 0.2, with size "qcif" and a frame rate of 10 frames per second: <pre class="example-preformatted">color=c=red@0.2:s=qcif:r=10 </pre>

If the input content is to be ignored, <code class="code">nullsrc</code> can be used. The following command generates noise in the luma plane by employing the <code class="code">geq</code> filter: <pre class="example-preformatted">nullsrc=s=256x256, geq=random(1)*255:128:128 </pre>

 <h4 class="subsection">46.13.2 Commands# TOC</h4>

The <code class="code">color</code> source supports the following commands: <dl class="table"> <dt><samp class="option">c, color</samp></dt> <dd> Set the color of the created image. Accepts the same syntax of the corresponding <samp class="option">color</samp> option. </dd> </dl>

 <h3 class="section">46.14 openclsrc# TOC</h3>

Generate video using an OpenCL program. <dl class="table"> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Kernel name in program. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Size of frames to generate. This must be set. </dd> <dt><samp class="option">format</samp></dt> <dd> Pixel format to use for the generated frames. This must be set. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Number of frames generated every second. Default value is ’25’. </dd> </dl>

For details of how the program loading works, see the program_opencl filter. Example

programs: <ul class="itemize mark-bullet"> Generate a colour ramp by setting pixel values from the position of the pixel in the output image. (Note that this will work with all pixel formats, but the generated output will not be the same.) <pre class="verbatim">___kernel void ramp(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 val; val.xy = val.zw = convert_float2(loc) / convert_float2(get_image_dim(dst));

write_imagef(dst, loc, val); } </pre> Generate a Sierpinski carpet pattern, panning by a single pixel each frame. <pre class="verbatim">___kernel void sierpinski_carpet(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 value = 0.0f; int x = loc.x + index; int y = loc.y + index; while (x > 0 || y > 0) { if (x % 3 == 1 && y % 3 == 1) { value = 1.0f; break; } x /= 3; y /= 3; }

write_imagef(dst, loc, value); } </pre>

 <h3 class="section">46.15 sierpinski# TOC</h3>

Generate a Sierpinski carpet/triangle fractal, and randomly pan around. This source accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed which is used for random panning. </dd> <dt><samp class="option">jump</samp></dt> <dd> Set max jump for single pan destination. Allowed range is from 1 to 10000. </dd> <dt><samp class="option">type</samp></dt> <dd> Set fractal type, can be default <code class="code">carpet</code> or <code class="code">triangle</code>. </dd> </dl>

 <h3 class="section">46.16 zoneplate# TOC</h3> Generate a zoneplate test video pattern. This source accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option,

check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "320x240". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set precision in bits for look-up table for sine calculations. Default value is 10. Allowed range is from 4 to 16. </dd> <dt><samp class=”option”>xo</samp></dt> <dd> Set horizontal axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>yo</samp></dt> <dd> Set vertical axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>to</samp></dt> <dd> Set time axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>k0</samp></dt> <dd> Set 0-order, constant added to signal phase. Default value is 0. </dd> <dt><samp class=”option”>kx</samp></dt> <dd> Set 1-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky</samp></dt> <dd> Set 1-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt</samp></dt> <dd> Set 1-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>kxt, kyt, kxy</samp></dt> <dd> Set phase factor multipliers for combination of spatial and temporal axis. Default value is 0. </dd> <dt><samp class=”option”>kx2</samp></dt> <dd> Set 2-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky2</samp></dt> <dd> Set 2-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt2</samp></dt> <dd> Set 2-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>ku</samp></dt> <dd> Set the constant added to final phase to produce chroma-blue component of signal. Default value is 0. </dd> <dt><samp class=”option”>kv</samp></dt> <dd> Set the constant added to final phase to produce chroma-red component of signal. Default value is 0. </dd> </dl>

 <h4 class=”subsection”>46.16.1 Commands# <a class=”anchor hidden-xs”href=”#toc-

Commands-159” aria-hidden=”true”>TOC</h4>

This source supports the some above options as commands. <h4 class=”subsection”>46.16.2 Examples#
TOC</h4>

<ul class=”itemize mark-bullet”> Generate horizontal color sine sweep:
 <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:kx2=256:s=wvg
426:kt=11 </pre>

Generate vertical color sine sweep: <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:ky2=156:s=wvga:yo=-240:kt=11 </pre>

Generate circular zone-plate: <pre class=”example-preformatted”>zoneplate=ku=512:kv=100:kt2=0:ky2=256:kx2=556:s=wvga:yo=0:kt=11
</pre>

 <h2 class=”chapter”>47 Video Sinks# TOC</h2>

Below is a description of the currently available video sinks. <h3 class=”section”>47.1 buffersink# TOC</h3>

Buffer video frames, and make them available to the end of the filter graph. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVBufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class=”code”>avfilter_init_filter</code> for initialization. <h3 class=”section”>47.2 nullsink# TOC</h3>

Null video sink: do absolutely nothing with the input video. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class=”chapter”>48 Multimedia Filters# TOC</h2>

Below is a description of the currently available multimedia filters. <h3 class="section">48.1 a3dscope# TOC</h3>

Convert input audio to 3d scope video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">fov</samp></dt> <dd> Set the camera field of view. Default is 90 degrees. Allowed range is from 40 to 150. </dd> <dt><samp class="option">roll</samp></dt> <dd> Set the camera roll. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set the camera pitch. </dd> <dt><samp class="option">yaw</samp></dt> <dd> Set the camera yaw. </dd> <dt><samp class="option">xzoom</samp></dt> <dd> Set the camera zoom on X-axis. </dd> <dt><samp class="option">yzoom</samp></dt> <dd> Set the camera zoom on Y-axis. </dd> <dt><samp class="option">zzoom</samp></dt> <dd> Set the camera zoom on Z-axis. </dd> <dt><samp class="option">xpos</samp></dt> <dd> Set the camera position on X-axis. </dd> <dt><samp class="option">ypos</samp></dt> <dd> Set the camera position on Y-axis. </dd> <dt><samp class="option">zpos</samp></dt> <dd> Set the camera position on Z-axis. </dd> <dt><samp class="option">length</samp></dt> <dd> Set the length of displayed audio waves in number of frames. </dd> </dl>

 <h4 class="subsection">48.1.1 Commands# TOC</h4>

Filter supports the some above options as commands. <h3 class="section">48.2 abitscope# TOC</h3>

Convert input audio to a video output, displaying the audio bit scope. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default

value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">1024x256</code>. </dd> <dt><samp class="option">colors</samp></dt> <dd> Specify list of colors separated by space or by ’|’ which will be used to draw channels. Unrecognized or missing colors will be replaced by white color. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set output mode. Can be <code class="code">bars</code> or <code class="code">trace</code>. Default is <code class="code">bars</code>. </dd> </dl>

 <h3 class="section">48.3 adrawgraph# TOC</h3> Draw a graph using input audio metadata. See drawgraph <h3 class="section">48.4 agraphmonitor# TOC</h3>

See graphmonitor. <h3 class="section">48.5 ahistogram# TOC</h3>

Convert input audio to a video output, displaying the volume histogram.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">dmode</samp></dt>

<dd> Specify how histogram is calculated.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">single</samp>’</dt>

<dd> Use single histogram for all channels.

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> Use separate histogram for each channel.

</dd>

</dl>

Default is <code class="code">single</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set frame rate, expressed as number of frames per second. Default

value is "25".

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">hd720</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set display scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

<dt>‘<samp class="samp">sqrt</samp>’</dt>

<dd> square root

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> reverse logarithmic
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">acount</samp></dt>
<dd> Set how much frames to accumulate in histogram.

Default is 1. Setting this to -1 accumulates all frames.

</dd>
<dt><samp class=”option”>rheight</samp></dt>
<dd> Set histogram ratio of window height.

</dd>
<dt><samp class=”option”>slide</samp></dt>
<dd> Set sonogram sliding.

It accepts the following values:
<dl class=”table”>
<dt>‘<samp class=”samp”>replace</samp>’</dt>
<dd> replace old rows with new ones.
</dd>
<dt>‘<samp class=”samp”>scroll</samp>’</dt>
<dd> scroll from top to bottom.
</dd>
</dl>
Default is <code class=”code”>replace</code>.

</dd>
<dt><samp class=”option”>hmode</samp></dt>
<dd> Set histogram mode.

It accepts the following values:
<dl class=”table”>
<dt>‘<samp class=”samp”>abs</samp>’</dt>
<dd> Use absolute values of samples.
</dd>
<dt>‘<samp class=”samp”>sign</samp>’</dt>

<dd> Use untouched values of samples.

</dd>

</dl>

Default is <code class="code">abs</code>.

</dd>

</dl>

 <h3 class="section">48.6 aphasemeter# TOC</h3>

Measures phase of input audio, which is exported as metadata <code class="code">lavfi.aphasemeter.phase</code>, representing mean phase of current audio frame. A video output can also be produced and is enabled by default. The audio is passed through as first output. Audio will be rematrixed to stereo if it has a different channel layout. Phase value is in range <code class="code">[-1, 1]</code> where <code class="code">-1</code> means left and right channels are completely out of phase and <code class="code">1</code> means channels are in phase. The filter accepts the following options, all related to its video output: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class="code">25</code>. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">800x400</code>. </dd> <dt><samp class="option">rc</samp></dt> <dt><samp class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt> <dd> Specify the red, green, blue contrast. Default values are <code class="code">2</code>, <code class="code">7</code> and <code class="code">1</code>. Allowed range is <code class="code">[0, 255]</code>. </dd> <dt><samp class="option">mpc</samp></dt> <dd> Set color which will be used for drawing median phase. If color is <code class="code">none</code> which is default, no median phase value will be drawn. </dd> <dt><samp class="option">video</samp></dt> <dd> Enable video output. Default is enabled. </dd> </dl>

 <h4 class="subsection">48.6.1 phasing detection# TOC</h4>

The filter also detects out of phase and mono sequences in stereo

streams. It logs the sequence start, end and duration when it lasts longer or as long as the minimum set. The filter accepts the following options for this detection: <dl class="table"> <dt><samp class="option">phasing</samp></dt> <dd> Enable mono and out of phase detection. Default is disabled. </dd> <dt><samp class="option">tolerance, t</samp></dt> <dd> Set phase tolerance for mono detection, in amplitude ratio. Default is <code class="code">0</code>. Allowed range is <code class="code">[0, 1]</code>. </dd> <dt><samp class="option">angle, a</samp></dt> <dd> Set angle threshold for out of phase detection, in degree. Default is <code class="code">170</code>. Allowed range is <code class="code">[90, 180]</code>. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set mono or out of phase duration until notification, expressed in seconds. Default is <code class="code">2</code>. </dd> </dl>

 <h4 class="subsection">48.6.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example with <code class="command">ffmpeg</code> to detect 1 second of mono with 0.001 phase tolerance: <pre class="example-preformatted">ffmpeg -i stereo.wav -af aphasemeter=video=0:phasing=1:duration=1:tolerance=0.001 -f null - </pre>

 <h3 class="section">48.7 avectorscope# TOC</h3>

Convert input audio to a video output, representing the audio vector scope. The filter is used to measure the difference between channels of stereo audio stream. A monaural signal, consisting of identical left and right signal, results in straight vertical line. Any stereo separation is visible as a deviation from this line, creating a Lissajous figure. If the straight (or deviation from it) but horizontal line appears this indicates that the left and right channels are out of phase. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set the vectorscope mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">lissajous</samp>’</dt> <dd> Lissajous rotated by 45 degrees. </dd> <dt>‘<samp class="samp">lissajous_xy</samp>’</dt> <dd> Same as above but not rotated. </dd> <dt>‘<samp class="samp">polar</samp>’</dt> <dd> Shape resembling half of circle. </dd> </dl>

Default value is ‘<samp class="samp">lissajous</samp>’.
 </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set
the video size for the output. For the syntax of this option, check
the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-
syntax">"Video size" section in the ffmpeg-utils manual.
Default value is <code class="code">400x400</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Set the
output frame rate. Default value is <code class="code">25</code>.
 </dd> <dt><samp class="option">rc</samp></dt> <dt><samp
class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt>
<dt><samp class="option">ac</samp></dt> <dd> Specify the red,
green, blue and alpha contrast. Default values are <code class="code">40</code>,
<code class="code">160</code>, <code class="code">80</code> and
<code class="code">255</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">rf</samp></dt>
<dt><samp class="option">gf</samp></dt> <dt><samp class="option">bf</samp></dt>
<dt><samp class="option">af</samp></dt> <dd> Specify the red,
green, blue and alpha fade. Default values are <code class="code">15</code>,
<code class="code">10</code>, <code class="code">5</code> and
<code class="code">5</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">zoom</samp></dt>
<dd> Set the zoom factor. Default value is <code class="code">1</code>.
Allowed range is <code class="code">[0, 10]</code>. Values lower than
<var class="var">1</var> will auto adjust zoom factor to maximal possible
value. </dd> <dt><samp class="option">draw</samp></dt>
<dd> Set the vectorscope drawing mode. Available values are:
<dl class="table"> <dt>‘<samp class="samp">dot</samp>’</dt>
<dd> Draw dot for each sample. </dd> <dt>‘<samp
class="samp">line</samp>’</dt> <dd> Draw line between pre-
vious and current sample. </dd> <dt>‘<samp class="samp">aaline</samp>’</dt>
<dd> Draw anti-aliased line between previous and current sample.
</dd> </dl>

Default value is ‘<samp class="samp">dot</samp>’.
 </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
amplitude scale of audio samples. Available values are: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root. </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic. </dd> </dl>

</dd> <dt><samp class="option">swap</samp></dt> <dd> Swap
left channel axis with right channel axis. </dd> <dt><samp
class="option">mirror</samp></dt> <dd> Mirror axis. <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> No mirror. </dd> <dt>‘<samp class="samp">x</samp>’</dt>

<dd> Mirror only x axis. </dd> <dt>‘<samp class="samp">y</samp>’</dt>
<dd> Mirror only y axis. </dd> <dt>‘<samp class="samp">xy</samp>’</dt>
<dd> Mirror both axis. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">48.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] avectorscope=zoom=1.3:rc=2:gc=200:bc=10:rf=1:gf=8:bf=7 [out0]' </pre>

 <h4 class="subsection">48.7.2 Commands# TOC</h4>

This filter supports the all above options as commands except options <code class="code">size</code> and <code class="code">rate</code>.
  <h3 class="section">48.8 bench, abench# TOC</h3>

Benchmark part of a filtergraph. The filter accepts the following options: <dl class="table"> <dt><samp class="option">action</samp></dt>
<dd> Start or stop a timer. Available values are: <dl class="table"> <dt>‘<samp class="samp">start</samp>’</dt>
<dd> Get the current time, set it as frame metadata (using the key <code class="code">lavfi.bench.start_time</code>), and forward the frame to the next filter. </dd> <dt>‘<samp class="samp">stop</samp>’</dt>
<dd> Get the current time and fetch the <code class="code">lavfi.bench.start_time</code> metadata from the input frame metadata to get the time difference. Time difference, average, maximum and minimum time (respectively <code class="code">t</code>, <code class="code">avg</code>, <code class="code">max</code> and <code class="code">min</code>) are then printed. The timestamps are expressed in seconds. </dd> </dl> </dd>
</dl>

 <h4 class="subsection">48.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Benchmark selectivecolor

filter: <pre class=”example-preformatted”>bench=start,selectivecolor=reds=-.2 .12 -.49,bench=stop </pre>

 <h3 class=”section”>48.9 concat# TOC</h3>

Concatenate audio and video streams, joining them together one after the other. The filter works on segments of synchronized video and audio streams. All segments must have the same number of streams of each type, and that will also be the number of streams at output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of segments. Default is 2. </dd> <dt><samp class=”option”>v</samp></dt> <dd> Set the number of output video streams, that is also the number of video streams in each segment. Default is 1. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Set the number of output audio streams, that is also the number of audio streams in each segment. Default is 0. </dd> <dt><samp class=”option”>unsafe</samp></dt> <dd> Activate unsafe mode: do not fail if segments have a different format. </dd> </dl>

The filter has <var class=”var”>v</var>+<var class=”var”>a</var> outputs: first <var class=”var”>v</var> video outputs, then <var class=”var”>a</var> audio outputs. There are <var class=”var”>n</var>x(<var class=”var”>v</var>+<var class=”var”>a</var>) inputs: first the inputs for the first segment, in the same order as the outputs, then the inputs for the second segment, etc. Related streams do not always have exactly the same duration, for various reasons including codec frame size or sloppy authoring. For that reason, related synchronized streams (e.g. a video and its audio track) should be concatenated at once. The concat filter will use the duration of the longest stream in each segment (except the last one), and if necessary pad shorter audio streams with silence. For this filter to work correctly, all segments must start at timestamp 0. All corresponding streams must have the same parameters in all segments; the filtering system will automatically select a common pixel format for video streams, and a common sample format, sample rate and channel layout for audio streams, but other settings, such as resolution, must be converted explicitly by the user. Different frame rates are acceptable but will result in variable frame rate at output; be sure to configure the output file to handle it. <h4 class=”subsection”>48.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Concatenate an opening, an episode and an ending, all in bilingual version (video in stream 0, audio in streams 1 and 2):

 <pre class="example-preformatted">ffmpeg -i opening.mkv -i episode.mkv -i ending.mkv -filter_complex \ '[0:0] [0:1] [0:2] [1:0] [1:1] [1:2] [2:0] [2:1] [2:2] concat=n=3:v=1:a=2 [v] [a1] [a2]' \ -map '[v]' -map '[a1]' -map '[a2]' output.mkv </pre>

Concatenate two parts, handling audio and video separately, using the (a)movie sources, and adjusting the resolution: <pre class="example-preformatted">movie=part1.mp4, scale=512:288 [v1] ; amovie=part1.mp4 [a1] ; movie=part2.mp4, scale=512:288 [v2] ; amovie=part2.mp4 [a2] ; [v1] [v2] concat [outv] ; [a1] [a2] concat=v=0:a=1 [outa] </pre> Note that a desync will happen at the stitch if the audio and video streams do not have exactly the same duration in the first file.

 <h4 class="subsection">48.9.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">next</samp></dt> <dd> Close the current segment and step to the next one </dd> </dl>

 <h3 class="section">48.10 ebur128# TOC</h3>

EBU R128 scanner filter. This filter takes an audio stream and analyzes its loudness level. By default, it logs a message at a frequency of 10Hz with the Momentary loudness (identified by <code class="code">M</code>), Short-term loudness (<code class="code">S</code>), Integrated loudness (<code class="code">I</code>) and Loudness Range (<code class="code">LRA</code>). The filter can only analyze streams which have sample format is double-precision floating point. The input stream will be converted to this specification, if needed. Users may need to insert aformat and/or aresample filters after this filter to obtain the original parameters. The filter also has a video output (see the <var class="var">video</var> option) with a real time graph to observe the loudness evolution. The graphic contains the logged message mentioned above, so it is not printed anymore when this option is set, unless the verbose logging is set. The main graphing area contains the short-term loudness (3 seconds of analysis), and the gauge on the right is for the momentary loudness (400 milliseconds), but can optionally be configured to instead display short-term loudness (see <var class="var">gauge</var>). The green area marks a +/- 1LU target range around the target loudness (-23LUFS by default, unless modified through <var class="var">target</var>). More information about the Loudness Recommendation EBU R128 on <a class="url"

href=”http://tech.ebu.ch/loudness”>http://tech.ebu.ch/loudness. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>video</samp></dt> <dd> Activate the video output. The audio stream is passed unchanged whether this option is set or no. The video stream will be the first output stream if activated. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set the video size. This option is for video only. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default and minimum resolution is <code class=”code”>640x480</code>. </dd> <dt><samp class=”option”>meter</samp></dt> <dd> Set the EBU scale meter. Default is <code class=”code”>9</code>. Common values are <code class=”code”>9</code> and <code class=”code”>18</code>, respectively for EBU scale meter +9 and EBU scale meter +18. Any other integer value between this range is allowed. </dd> <dt><samp class=”option”>metadata</samp></dt> <dd> Set metadata injection. If set to <code class=”code”>1</code>, the audio input will be segmented into 100ms output frames, each of them containing various loudness information in metadata. All the metadata keys are prefixed with <code class=”code”>lavfi.r128.</code>. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>framelog</samp></dt> <dd> Force the frame logging level. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>quiet</samp>’</dt> <dd> logging disabled </dd> <dt>‘<samp class=”samp”>info</samp>’</dt> <dd> information logging level </dd> <dt>‘<samp class=”samp”>verbose</samp>’</dt> <dd> verbose logging level </dd> </dl>

By default, the logging level is set to <var class=”var”>info</var>. If the <samp class=”option”>video</samp> or the <samp class=”option”>metadata</samp> options are set, it switches to <var class=”var”>verbose</var>. </dd> <dt><samp class=”option”>peak</samp></dt> <dd> Set peak mode(s). Available modes can be cumulated (the option is a <code class=”code”>flag</code> type). Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disable any peak mode (default). </dd> <dt>‘<samp class=”samp”>sample</samp>’</dt> <dd> Enable sample-peak mode. Simple peak mode looking for the higher sample value. It logs a message for sample-peak (identified by <code class=”code”>SPK</code>). </dd> <dt>‘<samp class=”samp”>true</samp>’</dt> <dd> Enable true-peak mode. If enabled, the peak lookup is done on an over-sampled version of the input stream for better peak accuracy. It logs a message for true-peak. (identified by <code class=”code”>TPK</code>) and true-peak per frame (identified by <code class=”code”>FTPK</code>). This mode requires a build with <code

class=”code”>libswresample</code>. </dd> </dl>

</dd> <dt><samp class=”option”>dualmono</samp></dt> <dd> Treat mono input files as "dual mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. </dd> <dt><samp class=”option”>panlaw</samp></dt> <dd> Set a specific pan law to be used for the measurement of dual mono files. This parameter is optional, and has a default value of -3.01dB. </dd> <dt><samp class=”option”>target</samp></dt> <dd> Set a specific target level (in LUFS) used as relative zero in the visualization. This parameter is optional and has a default value of -23LUFS as specified by EBU R128. However, material published online may prefer a level of -16LUFS (e.g. for use with podcasts or video platforms). </dd> <dt><samp class=”option”>gauge</samp></dt> <dd> Set the value displayed by the gauge. Valid values are <code class=”code”>momentary</code> and s <code class=”code”>shortterm</code>. By default the momentary value will be used, but in certain scenarios it may be more useful to observe the short term value instead (e.g. live mixing). </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Sets the display scale for the loudness. Valid parameters are <code class=”code”>absolute</code> (in LUFS) or <code class=”code”>relative</code> (LU) relative to the target. This only affects the video output, not the summary or continuous log output. </dd> <dt><samp class=”option”>integrated</samp></dt> <dd> Read-only exported value for measured integrated loudness, in LUFS. </dd> <dt><samp class=”option”>range</samp></dt> <dd> Read-only exported value for measured loudness range, in LU. </dd> <dt><samp class=”option”>lra_low</samp></dt> <dd> Read-only exported value for measured LRA low, in LUFS. </dd> <dt><samp class=”option”>lra_high</samp></dt> <dd> Read-only exported value for measured LRA high, in LUFS. </dd> <dt><samp class=”option”>sample_peak</samp></dt> <dd> Read-only exported value for measured sample peak, in dBFS. </dd> <dt><samp class=”option”>true_peak</samp></dt> <dd> Read-only exported value for measured true peak, in dBFS. </dd> </dl>

 <h4 class=”subsection”>48.10.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Real-time graph using <code class=”command”>ffplay</code>, with a EBU scale meter +18: <pre class=”example-preformatted”>ffplay -f lavfi -i "amovie=input.mp3,ebur128=video=1:meter=18 [out0][out1]" </pre>

Run an analysis with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -nostats -i input.mp3 -filter_complex ebur128 -f null - </pre>

 <h3 class="section">48.11 interleave, ainterleave# TOC</h3>

Temporally interleave frames from several inputs. <code class="code">interleave</code> works with video inputs, <code class="code">ainterleave</code> with audio. These filters read frames from several inputs and send the oldest queued frame to the output. Input streams must have well defined, monotonically increasing frame timestamp values. In order to submit one frame to output, these filters need to enqueue at least one frame for each input, so they cannot work in case one input is not yet terminated and will not receive incoming frames. For example consider the case when one input is a <code class="code">select</code> filter which always drops input frames. The <code class="code">interleave</code> filter will keep reading from that input, but it will never be able to send new frames to output until the input sends an end-of-stream signal. Also, depending on inputs synchronization, the filters will drop frames in case one input receives more frames than the other ones, and the queue is already filled. These filters accept the following options: <dl class="table"> <dt><samp class="option">nb_inputs, n</samp></dt> <dd> Set the number of different inputs, it is 2 by default. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class="option">first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">48.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interleave frames belonging to different streams using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i bambi.avi -i pr0n.mkv -filter_complex "[0:v][1:v] interleave" out.avi </pre>

Add flickering blur effect: <pre

class=”example-preformatted”>select='if(gt(random(0), 0.2), 1, 2)':n=2 [tmp], boxblur=2:2, [tmp] interleave </pre>

 <h3 class=”section”>48.12 latency, alatency# TOC</h3> Measure filtering latency. Report previous filter filtering latency, delay in number of audio samples for audio filters or number of video frames for video filters. On end of input stream, filter will report min and max measured latency for previous running filter in filtergraph. <h3 class=”section”>48.13 metadata, ametadata# TOC</h3>

Manipulate frame metadata. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>select</samp>’</dt> <dd> If both <code class=”code”>value</code> and <code class=”code”>key</code> is set, select frames which have such metadata. If only <code class=”code”>key</code> is set, select every frame that has such key in metadata. </dd> <dt>‘<samp class=”samp”>add</samp>’</dt> <dd> Add new metadata <code class=”code”>key</code> and <code class=”code”>value</code>. If key is already available do nothing. </dd> <dt>‘<samp class=”samp”>modify</samp>’</dt> <dd> Modify value of already present key. </dd> <dt>‘<samp class=”samp”>delete</samp>’</dt> <dd> If <code class=”code”>value</code> is set, delete only keys that have such value. Otherwise, delete key. If <code class=”code”>key</code> is not set, delete all metadata values in the frame. </dd> <dt>‘<samp class=”samp”>print</samp>’</dt> <dd> Print key and its value if metadata was found. If <code class=”code”>key</code> is not set print all metadata values available in frame. </dd> </dl>

</dd> <dt><samp class=”option”>key</samp></dt> <dd> Set key used with all modes. Must be set for all modes except <code class=”code”>print</code> and <code class=”code”>delete</code>. </dd> <dt><samp class=”option”>value</samp></dt> <dd> Set metadata value which will be used. This option is mandatory for <code class=”code”>modify</code> and <code class=”code”>add</code> mode. </dd> <dt><samp class=”option”>function</samp></dt> <dd> Which function to use when comparing metadata value and <code class=”code”>value</code>. Can be one of following: <dl class=”table”> <dt>‘<samp class=”samp”>same_str</samp>’</dt>

<dd> Values are interpreted as strings, returns true if metadata value is same as <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>starts_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value starts with the <code class=”code”>value</code> option string. </dd> <dt>‘<samp class=”samp”>less</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is less than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>equal</samp>’</dt> <dd> Values are interpreted as floats, returns true if <code class=”code”>value</code> is equal with metadata value. </dd> <dt>‘<samp class=”samp”>greater</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is greater than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>expr</samp>’</dt> <dd> Values are interpreted as floats, returns true if expression from option <code class=”code”>expr</code> evaluates to true. </dd> <dt>‘<samp class=”samp”>ends_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value ends with the <code class=”code”>value</code> option string. </dd> </dl>

</dd> <dt><samp class=”option”>expr</samp></dt> <dd> Set expression which is used when <code class=”code”>function</code> is set to <code class=”code”>expr</code>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>VALUE1, FRAMEVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> from metadata key. </dd> <dt><samp class=”option”>VALUE2, USERVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> as supplied by user in <code class=”code”>value</code> option. </dd> </dl>

</dd> <dt><samp class=”option”>file</samp></dt> <dd> If specified in <code class=”code”>print</code> mode, output is written to the named file. Instead of plain filename any writable url can be specified. Filename “-” is a shorthand for standard output. If <code class=”code”>file</code> option is not set, output is written to the log with AV_LOG_INFO loglevel. </dd> <dt><samp class=”option”>direct</samp></dt> <dd> Reduces buffering in print mode when output is written to a URL set using <var class=”var”>file</var>. </dd> </dl>

 <h4 class=”subsection”>48.13.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print all metadata values for frames with key <code class=”code”>lavfi.signalstats.YDIF</code> with val-

ues between 0 and 1. <pre class=”example-preformatted”>signalstats,metadata=print:key=lavfi.signalstats.YDIF:value=0:function=expr:expr='between
</pre> Print silencedetect output to file <samp class=”file”>metadata.txt</samp>.
 <pre class=”example-preformatted”>silencedetect,ametadata=mode=print:file=meta
</pre> Direct all metadata to a pipe with file descriptor 4.
 <pre class=”example-preformatted”>metadata=mode=print:file='pipe\:4'
</pre>

 <h3 class=”section”>48.14 perms, aperms# TOC</h3>

Set read/write permissions for the output frames. These filters are mainly aimed at developers to test direct path in the following filter in the filtergraph. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt>
<dd> Select the permissions mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt>
<dd> Do nothing. This is the default. </dd> <dt>‘<samp class=”samp”>ro</samp>’</dt> <dd> Set all the output frames read-only. </dd> <dt>‘<samp class=”samp”>rw</samp>’</dt>
<dd> Set all the output frames directly writable. </dd>
<dt>‘<samp class=”samp”>toggle</samp>’</dt> <dd> Make the frame read-only if writable, and writable if read-only. </dd>
<dt>‘<samp class=”samp”>random</samp>’</dt> <dd> Set each output frame read-only or writable randomly. </dd> </dl>

</dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set the seed for the <var class=”var”>random</var> mode, must be an integer included between <code class=”code”>0</code> and <code class=”code”>UINT32_MAX</code>. If not specified, or if explicitly set to <code class=”code”>-1</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

Note: in case of auto-inserted filter between the permission filter and the following one, the permission might not be received as expected in that following filter. Inserting a format or aformat filter before the perms/aperms filter can avoid this problem. <h3 class=”section”>48.15 realtime, arealtime# TOC</h3>

Slow down filtering to match real time approximately. These filters will pause the filtering for a variable amount of time to match the output rate with the input timestamps. They are similar to the <samp

class=”option”>re</samp> option to <code class=”code”>ffmpeg</code>. They accept the following options: <dl class=”table”> <dt><samp class=”option”>limit</samp></dt> <dd> Time limit for the pauses. Any pause longer than that will be considered a timestamp discontinuity and reset the timer. Default is 2 seconds. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Speed factor for processing. The value must be a float larger than zero. Values larger than 1.0 will result in faster than realtime processing, smaller will slow processing down. The <var class=”var”>limit</var> is automatically adapted accordingly. Default is 1.0. A processing speed faster than what is possible without these filters cannot be achieved. </dd> </dl>

 <h4 class=”subsection”>48.15.1 Commands# TOC</h4>

Both filters supports the all above options as commands. <h3 class=”section”>48.16 segment, asegment# TOC</h3

Split single input stream into multiple streams. This filter does opposite of concat filters. <code class=”code”>segment</code> works on video frames, <code class=”code”>asegment</code> on audio samples. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>timestamps</samp></dt> <dd> Timestamps of output segments separated by ’|’. The first segment will run from the beginning of the input stream. The last segment will run until the end of the input stream </dd> <dt><samp class=”option”>frames, samples</samp></dt> <dd> Exact frame/sample count to split the segments. </dd> </dl>

In all cases, prefixing an each segment with ’+’ will make it relative to the previous segment. <h4 class=”subsection”>48.16.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Split input audio stream into three output audio streams, starting at start of input audio stream and storing that in 1st output audio stream, then following at 60th second and storing than in 2nd output audio stream, and last after 150th second of input audio stream store in 3rd output audio stream: <pre class=”example-preformatted”>asegment=timestamps="60|150" </pre>

<h3 class=”section”>48.17 select, aselect<a
class=”anchor hidden-xs” href=”#select_002c-aselect” aria-hidden=”true”>#
TOC</h3>

Select frames to pass in output. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>expr, e</samp></dt> <dd> Set expression, which is evaluated for each input frame. If the expression is evaluated to zero, the frame is discarded. If the evaluation result is negative or NaN, the frame is sent to the first output; otherwise it is sent to the output with index <code class=”code”>ceil(val)-1</code>, assuming that the input index starts from 0. For example a value of <code class=”code”>1.2</code> corresponds to the output with index <code class=”code”>ceil(1.2)-1 = 2-1 = 1</code>, that is the second output. </dd> <dt><samp class=”option”>outputs, n</samp></dt> <dd> Set the number of outputs. The output to which to send the selected frame is based on the result of the evaluation. Default value is 1. </dd> </dl>

The expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the filtered frame, starting from 0. </dd> <dt><samp class=”option”>selected_n</samp></dt> <dd> The (sequential) number of the selected frame, starting from 0. </dd> <dt><samp class=”option”>prev_selected_n</samp></dt> <dd> The sequential number of the last selected frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the filtered frame, expressed in <var class=”var”>TB</var> units. It’s NAN if undefined. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The PTS of the filtered frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_pts</samp></dt> <dd> The PTS of the previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_pts</samp></dt> <dd> The PTS of the last previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_t</samp></dt> <dd> The PTS of the last previously selected frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> The first PTS in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>start_t</samp></dt> <dd> The first PTS, in seconds, in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>pict_type (video only)</samp></dt> <dd> The type of the filtered frame. It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>I</samp></dt> <dt><samp

class=”option”>P</samp></dt> <dt><samp class=”option”>B</samp></dt>
<dt><samp class=”option”>S</samp></dt> <dt><samp class=”option”>SI</samp></dt>
<dt><samp class=”option”>SP</samp></dt> <dt><samp class=”option”>BI</samp></dt>
</dl>

</dd> <dt><samp class=”option”>interlace_type (video
only)</samp></dt> <dd> The frame interlace type. It can assume one of the following values: <dl class=”table”> <dt><samp
class=”option”>PROGRESSIVE</samp></dt> <dd> The frame is progressive (not interlaced). </dd> <dt><samp class=”option”>TOPFIRST</samp></dt>
<dd> The frame is top-field-first. </dd> <dt><samp class=”option”>BOTTOMFIRST</samp><
<dd> The frame is bottom-field-first. </dd> </dl>

</dd> <dt><samp class=”option”>consumed_sample_n (audio
only)</samp></dt> <dd> the number of selected samples before
the current frame </dd> <dt><samp class=”option”>samples_n
(audio only)</samp></dt> <dd> the
number of samples in the current frame </dd> <dt><samp
class=”option”>sample_rate (audio only)</samp></dt>
<dd> the input sample rate </dd> <dt><samp class=”option”>key</samp></dt>
<dd> This is 1 if the filtered frame is a key-frame, 0 otherwise.
</dd> <dt><samp class=”option”>pos</samp></dt> <dd> the
position in the file of the filtered frame, -1 if the information is not
available (e.g. for synthetic video); deprecated, do not use
</dd> <dt><samp class=”option”>scene (video
only)</samp></dt> <dd> value between 0 and 1 to indicate
a new scene; a low value reflects a low probability for the current frame
to introduce a new scene, while a higher value means the current frame is
more likely to be one (see the example below) </dd> <dt><samp
class=”option”>concatdec_select</samp></dt> <dd> The concat
demuxer can select only part of a concat input file by setting an inpoint
and an outpoint, but the output packets may not be entirely contained
in the selected interval. By using this variable, it is possible to skip
frames generated by the concat demuxer which are not exactly contained
in the selected interval. This works by comparing the frame
pts against the <var class=”var”>lavf.concat.start_time</var> and the
<var class=”var”>lavf.concat.duration</var> packet metadata values
which are also present in the decoded frames. The <var
class=”var”>concatdec_select</var> variable is -1 if the frame pts is at least
start_time and either the duration metadata is missing or the frame pts is
less than start_time + duration, 0 otherwise, and NaN if the start_time
metadata is missing. That basically means that an input frame
is selected if its pts is within the interval set by the concat demuxer.
 </dd> <dt><samp class=”option”>iw (video
only)</samp></dt> <dd> Represents the width of the input video frame. </dd> <dt><samp class=”option”>ih <em
class=”emph”>(video only)</samp></dt> <dd> Represents the

height of the input video frame. </dd> <dt><samp class="option">view (video only)</samp></dt> <dd> View ID for multi-view video. </dd> </dl>

The default value of the select expression is "1". <h4 class="subsection">48.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select all frames in input: <pre class="example-preformatted">select </pre>

The example above is the same as: <pre class="example-preformatted">select=1 </pre>

Skip all frames: <pre class="example-preformatted">select=0 </pre>

Select only I-frames: <pre class="example-preformatted">select='eq(pict_type\,I)' </pre>

Select one frame every 100: <pre class="example-preformatted">select='not(mod(n\,100))' </pre>

Select only frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20) </pre>

Select only I-frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20)*eq(pict_type\,I) </pre>

Select frames with a minimum distance of 10 seconds: <pre class="example-preformatted">select='isnan(prev_selected_t)+gte(t-prev_selected_t\,10)' </pre>

Use aselect to select only audio frames with samples number > 100: <pre class="example-preformatted">aselect='gt(samples_n\,100)' </pre>

Create a mosaic of the first scenes: <pre class="example-preformatted">ffmpeg -i video.avi -vf select='gt(scene\,0.4)',scale=160:120,tile -frames:v 1 preview.png </pre>

Comparing <var class="var">scene</var> against a value between 0.3 and 0.5 is generally a sane choice. Send even and odd frames to separate outputs, and compose them: <pre class="example-preformatted">select=n=2:e='mod(n, 2)+1' [odd][even]; [odd] pad=h=2*ih [tmp]; [tmp][even] overlay=y=h </pre>

Select useful frames from an ffconcat file which is using inpoints and outpoints but where the source files are not intra frame only. <pre class="example-preformatted">ffmpeg -copyts -segment_time_metadata 1 -i input.ffconcat -fps_mode passthrough -vf select=concatdec_select -af aselect=concatdec_select output.avi </pre>

 <h3 class="section">48.18 sendcmd, asendcmd# TOC</h3>

Send commands to filters in the filtergraph. These filters read commands to be sent to other filters in the filtergraph. <code class="code">sendcmd</code> must be inserted between two video filters, <code class="code">asendcmd</code> must be inserted between two audio filters, but apart from that they act the same way. The specification of commands can be provided in the filter arguments with the <var class="var">commands</var> option, or in a file specified by the <var class="var">filename</var> option. These filters accept the following options: <dl class="table"> <dt><samp class="option">commands, c</samp></dt> <dd> Set the commands to be read and sent to the other filters. </dd> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the commands to be read and sent to the other filters. </dd> </dl>

 <h4 class="subsection">48.18.1 Commands syntax# TOC</h4>

A commands description consists of a sequence of interval specifications, comprising a list of commands to be executed when a particular event related to that interval occurs. The occurring event is typically the current frame time entering or leaving a given time interval. An interval is specified by the following syntax: <pre class="example-preformatted"><var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var>; </pre>

The time interval is specified by the <var class="var">START</var> and <var class="var">END</var> times. <var class="var">END</var> is optional and defaults to the maximum time. The current frame time is considered within the specified interval if it is included in the interval [<var class="var">START</var>, <var class="var">END</var>), that is when the time is greater or equal to <var class="var">START</var> and is lesser than <var class="var">END</var>. <var

class=”var”>COMMANDS</var> consists of a sequence of one or more command specifications, separated by ",", relating to that interval. The syntax of a command specification is given by: <pre class=”example-preformatted”>[<var class=”var”>FLAGS</var>] <var class=”var”>TARGET</var> <var class=”var”>COMMAND</var> <var class=”var”>ARG</var> </pre>

<var class=”var”>FLAGS</var> is optional and specifies the type of events relating to the time interval which enable sending the specified command, and must be a non-null sequence of identifier flags separated by "+" or "|" and enclosed between "[" and "]". The following flags are recognized: <dl class=”table”> <dt><samp class=”option”>enter</samp></dt> <dd> The command is sent when the current frame timestamp enters the specified interval. In other words, the command is sent when the previous frame timestamp was not in the given interval, and the current is. </dd> <dt><samp class=”option”>leave</samp></dt> <dd> The command is sent when the current frame timestamp leaves the specified interval. In other words, the command is sent when the previous frame timestamp was in the given interval, and the current is not. </dd> <dt><samp class=”option”>expr</samp></dt> <dd> The command <var class=”var”>ARG</var> is interpreted as expression and result of expression is passed as <var class=”var”>ARG</var>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>POS</samp></dt> <dd> Original position in the file of the frame, or undefined if undefined for the current frame. Deprecated, do not use. </dd> <dt><samp class=”option”>PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class=”option”>N</samp></dt> <dd> The count of the input frame for video or audio, starting from 0. </dd> <dt><samp class=”option”>T</samp></dt> <dd> The time in seconds of the current frame. </dd> <dt><samp class=”option”>TS</samp></dt> <dd> The start time in seconds of the current command interval. </dd> <dt><samp class=”option”>TE</samp></dt> <dd> The end time in seconds of the current command interval. </dd> <dt><samp class=”option”>TI</samp></dt> <dd> The interpolated time of the current command interval, TI = (T - TS) / (TE - TS). </dd> <dt><samp class=”option”>W</samp></dt> <dd> The video frame width. </dd> <dt><samp class=”option”>H</samp></dt> <dd> The video frame height. </dd> </dl>

</dd> </dl>

If <var class=”var”>FLAGS</var> is not specified, a default value of <code class=”code”>[enter]</code> is assumed. <var class=”var”>TARGET</var> specifies the target of the command, usu-

ally the name of the filter class or a specific filter instance name. <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional list of argument for the given <var class="var">COMMAND</var>. Between one interval specification and another, whitespaces, or sequences of characters starting with <code class="code">#</code> until the end of line, are ignored and can be used to annotate comments. A simplified BNF description of the commands specification syntax follows: <pre class="example-preformatted"><var class="var">COMMAND_FLAG</var> ::= "enter" | "leave" <var class="var">COMMAND_FLAGS</var> ::= <var class="var">COMMAND_FLAG</var> [(+|"|")<var class="var">COMMAND_FLAG</var>] <var class="var">COMMAND</var> ::= ["[" <var class="var">COMMAND_FLAGS</var> "]"] <var class="var">TARGET</var> <var class="var">COMMAND</var> [<var class="var">ARG</var>] <var class="var">COMMANDS</var> ::= <var class="var">COMMAND</var> [,<var class="var">COMMANDS</var>] <var class="var">INTERVAL</var> ::= <var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var> <var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[;<var class="var">INTERVALS</var>] </pre>

 <h4 class="subsection">48.18.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Specify audio tempo change at second 4: <pre class="example-preformatted">asendcmd=c='4.0 atempo tempo 1.5',atempo </pre>

Target a specific filter instance: <pre class="example-preformatted">asendcmd=c='4.0 atempo@my tempo 1.5',atempo@my </pre>

Specify a list of drawtext and hue commands in a file. <pre class="example-preformatted"># show text in the interval 5-10 5.0-10.0 [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=hello world', [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=';

desaturate the image in the interval 15-20 15.0-20.0 [enter] hue s 0, [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=nocolor', [leave] hue s 1, [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=color';

apply an exponential saturation fade-out effect, starting from time 25 25 [enter] hue s exp(25-t) </pre>

A filtergraph allowing to read and process the above command list stored in a file <samp class="file">test.cmd</samp>, can be

specified with: <pre class="example-preformatted">sendcmd=f=test.cmd,drawtext=fontfile=FreeSerif.ttf:text='',hue
</pre>

<h3 class="section">48.19 setpts, asetpts<a
class="anchor hidden-xs" href="#setpts_002c-asetpts" aria-hidden="true">#
TOC</h3>

Change the PTS (presentation timestamp) of the input frames.
<code class="code">setpts</code> works on video frames, <code
class="code">asetpts</code> on audio frames. This filter
accepts the following options: <dl class="table"> <dt><samp
class="option">expr</samp></dt> <dd> The expression which is evaluated for each frame to construct its timestamp. </dd> <dt><samp
class="option">strip_fps (video only)</samp></dt>
<dd> Boolean option which determines if the original framerate and frame duration metadata is unset. If set to true, be advised that a sane frame rate should be explicitly specified if output is sent to a constant frame rate muxer. Default is <code class="code">false</code>. </dd> </dl>

The expression is evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp
class="option">FRAME_RATE, FR</samp></dt> <dd> frame rate, only defined for constant frame-rate video </dd> <dt><samp
class="option">PTS</samp></dt> <dd> The presentation timestamp in input </dd> <dt><samp class="option">N</samp></dt>
<dd> The count of the input frame for video or the number of consumed samples, not including the current frame for audio, starting from 0.
</dd> <dt><samp class="option">NB_CONSUMED_SAMPLES</samp></dt>
<dd> The number of consumed samples, not including the current frame (only audio) </dd> <dt><samp class="option">NB_SAMPLES, S</samp></dt> <dd> The number of samples in the current frame (only audio) </dd> <dt><samp class="option">SAMPLE_RATE, SR</samp></dt> <dd> The audio sample rate. </dd>
<dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first frame. </dd> <dt><samp class="option">STARTT</samp></dt>
<dd> the time in seconds of the first frame </dd> <dt><samp
class="option">INTERLACED</samp></dt> <dd> State whether the current frame is interlaced. </dd> <dt><samp class="option">T</samp></dt>
<dd> the time in seconds of the current frame </dd> <dt><samp
class="option">POS</samp></dt> <dd> original position in the file of the frame, or undefined if undefined for the current frame; deprecated, do not use
 </dd> <dt><samp class="option">PREV_INPTS</samp></dt>
<dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INT</samp></dt>
<dd> previous input time in seconds </dd> <dt><samp
class="option">PREV_OUTPTS</samp></dt> <dd> The previous

output PTS. </dd> <dt><samp class="option">PREV_OUTT</samp></dt> <dd> previous output time in seconds </dd> <dt><samp class="option">RTCTIME</samp></dt> <dd> The wallclock (RTC) time in microseconds. This is deprecated, use time(0) instead. </dd> <dt><samp class="option">RTCSTART</samp></dt> <dd> The wallclock (RTC) time at the start of the movie in microseconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class="option">T_CHANGE</samp></dt> <dd> Time of the first frame after command was applied or time of the first frame if no commands. </dd> </dl>

 <h4 class="subsection">48.19.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Start counting PTS from zero <pre class="example-preformatted">setpts=PTS-STARTPTS </pre>

Apply fast motion effect: <pre class="example-preformatted">setpts=0.5*PTS </pre>

Apply slow motion effect: <pre class="example-preformatted">setpts=2.0*PTS </pre>

Set fixed rate of 25 frames per second: <pre class="example-preformatted">setpts=N/(25*TB) </pre>

Apply a random jitter effect of +/-100 TB units: <pre class="example-preformatted">setpts=PTS+randomi(0, -100\,100) </pre>

Set fixed rate 25 fps with some jitter: <pre class="example-preformatted">setpts='1/(25*TB) * (N + 0.05 * sin(N*2*PI/25))' </pre>

Apply an offset of 10 seconds to the input PTS: <pre class="example-preformatted">setpts=PTS+10/TB </pre>

Generate timestamps from a "live source" and rebase onto the current timebase: <pre class="example-preformatted">setpts='(RTCTIME - RTCSTART) / (TB * 1000000)' </pre>

Generate timestamps by counting samples: <pre class="example-preformatted">asetpts=N/SR/TB </pre>

 <h4 class="subsection">48.19.2 Commands# TOC</h4>

Both filters support all above options as commands. <h3 class="section">48.20 setrange# TOC</h3>

Force color range for the output video frame. The <code class="code">setrange</code> filter marks the color range property for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters. The filter accepts the following options: <dl class="table"> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property. </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Set the color range as unspecified. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Set the color range as limited. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Set the color range as full. </dd> </dl> </dd> </dl>

 <h3 class="section">48.21 settb, asettb# TOC</h3>

Set the timebase to use for the output frames timestamps. It is mainly useful for testing timebase configuration. It accepts the following parameters: <dl class="table"> <dt><samp class="option">expr, tb</samp></dt> <dd> The expression which is evaluated into the output timebase. </dd> </dl>

The value for <samp class="option">tb</samp> is an arithmetic expression representing a rational. The expression can contain the constants "AVTB" (the default timebase), "intb" (the input timebase) and "sr" (the sample rate, audio only). Default value is "intb". <h4 class="subsection">48.21.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the timebase to 1/25: <pre class="example-preformatted">settb=expr=1/25 </pre>

Set the timebase to 1/10: <pre class="example-preformatted">settb=expr=0.1 </pre>

Set the timebase to 1001/1000: <pre class="example-preformatted">settb=1+0.001 </pre>

Set the timebase to 2*intb: <pre class="example-preformatted">settb=2*intb </pre>

Set the default timebase value: <pre class="example-preformatted">settb=AVTB </pre>

<h3 class="section">48.22 showcqt# TOC</h3>

Convert input audio to a video output representing frequency spectrum

logarithmically using Brown-Puckette constant Q transform algorithm with

direct frequency domain coefficient calculation (but the transform itself

is not really constant Q, instead the Q factor is actually variable/clamped),

with musical tone scale, from E0 to D#10.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. It must be even. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">1920x1080</code>.

</dd>

<dt><samp class="option">fps, rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">bar_h</samp></dt>

<dd> Set the bargraph height. It must be even. Default value is <code class="code">-1</code> which

computes the bargraph height automatically.

</dd>

<dt><samp class="option">axis_h</samp></dt>

<dd> Set the axis height. It must be even. Default value is <code class="code">-1</code> which computes

the axis height automatically.

</dd>

<dt><samp class="option">sono_h</samp></dt>

<dd> Set the sonogram height. It must be even. Default value is <code class="code">-1</code> which

computes the sonogram height automatically.

</dd>

<dt><samp class="option">fullhd</samp></dt>

<dd> Set the fullhd resolution. This option is deprecated, use <var class="var">size</var>, <var class="var">s</var>

instead. Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">sono_v, volume</samp></dt>

<dd> Specify the sonogram volume expression. It can contain variables:

<dl class="table">

<dt><samp class="option">bar_v</samp></dt>

<dd> the <var class="var">bar_v</var> evaluated expression

</dd>

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option
</dd>
</dl>
and functions:
<dl class="table">
<dt><samp class="option">a_weighting(f)</samp></dt>
<dd> A-weighting of equal loudness
</dd>
<dt><samp class="option">b_weighting(f)</samp></dt>
<dd> B-weighting of equal loudness
</dd>
<dt><samp class="option">c_weighting(f)</samp></dt>
<dd> C-weighting of equal loudness.
</dd>
</dl>
Default value is <code class="code">16</code>.

</dd>
<dt><samp class="option">bar_v, volume2</samp></dt>
<dd> Specify the bargraph volume expression. It can contain variables:
<dl class="table">
<dt><samp class="option">sono_v</samp></dt>
<dd> the <var class="var">sono_v</var> evaluated expression
</dd>
<dt><samp class="option">frequency, freq, f</samp></dt>
<dd> the frequency where it is evaluated
</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">a_weighting(f)</samp></dt>

<dd> A-weighting of equal loudness

</dd>

<dt><samp class="option">b_weighting(f)</samp></dt>

<dd> B-weighting of equal loudness

</dd>

<dt><samp class="option">c_weighting(f)</samp></dt>

<dd> C-weighting of equal loudness.

</dd>

</dl>

Default value is <code class="code">sono_v</code>.

</dd>

<dt><samp class="option">sono_g, gamma</samp></dt>

<dd> Specify the sonogram gamma. Lower gamma makes the spectrum more contrast,

higher gamma makes the spectrum having more range. Default value is <code class="code">3</code>.

Acceptable range is <code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_g, gamma2</samp></dt>

<dd> Specify the bargraph gamma. Default value is <code class="code">1</code>. Acceptable range is

<code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_t</samp></dt>

<dd> Specify the bargraph transparency level. Lower value makes the bargraph sharper.

Default value is <code class="code">1</code>. Acceptable range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> Specify the transform timeclamp. At low frequency, there is trade-off between

accuracy in time domain and frequency domain. If timeclamp is lower,

event in time domain is represented more accurately (such as fast bass drum),

otherwise event in frequency domain is represented more accurately

(such as bass guitar). Acceptable range is <code class="code">[0.002, 1]</code>. Default value is <code class="code">0.17</code>.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set attack time in seconds. The default is <code class="code">0</code> (disabled). Otherwise, it

limits future samples by applying asymmetric windowing in time domain, useful

when low latency is required. Accepted range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">basefreq</samp></dt>

<dd> Specify the transform base frequency. Default value is <code class="code">20.01523126408007475</code>,

which is frequency 50 cents below E0. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">endfreq</samp></dt>

<dd> Specify the transform end frequency. Default value is <code class="code">20495.59681441799654</code>,

which is frequency 50 cents above D#10. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">coeffclamp</samp></dt>

<dd> This option is deprecated and ignored.

</dd>

<dt><samp class="option">tlength</samp></dt>

<dd> Specify the transform length in time domain. Use this option to control accuracy

trade-off between time domain and frequency domain at every frequency sample.

It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option.

</dd>

</dl>

Default value is <code class="code">384*tc/(384+tc*f)</code>.

</dd>

<dt><samp class="option">count</samp></dt>

<dd> Specify the transform count for every video frame. Default value is <code class="code">6</code>.

Acceptable range is <code class="code">[1, 30]</code>.

</dd>

<dt><samp class="option">fcount</samp></dt>

<dd> Specify the transform count for every single pixel. Default value is <code class="code">0</code>,

which makes it computed automatically. Acceptable range is <code class="code">[0, 10]</code>.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> Specify font file for use with freetype to draw the axis. If not specified,

use embedded font. Note that drawing with font file or embedded font is not

implemented with custom <var class="var">basefreq</var> and <var class="var">endfreq</var>, use <var class="var">axisfile</var>

option instead.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> Specify fontconfig pattern. This has lower priority than <var class="var">fontfile</var>. The

<code class="code">:</code> in the pattern may be replaced by <code class="code">|</code> to avoid unnecessary

escaping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> Specify font color expression. This is arithmetic expression that should return

integer value 0xRRGGBB. It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">midi(f)</samp></dt>

<dd> midi number of frequency f, some midi numbers: E0(16), C1(24), C2(36), A4(69)

</dd>

<dt><samp class="option">r(x), g(x), b(x)</samp></dt>

<dd> red, green, and blue value of intensity x.

</dd>

</dl>

Default value is <code class="code">st(0, (midi(f)-59.5)/12);

st(1, if(between(ld(0),0,1), 0.5-0.5*cos(2*PI*ld(0)), 0));

r(1-ld(1)) + b(ld(1))</code>.

</dd>

<dt><samp class="option">axisfile</samp></dt>

<dd> Specify image file to draw the axis. This option override <var class="var">fontfile</var> and

<var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">axis, text</samp></dt>

<dd> Enable/disable drawing text to the axis. If it is set to <code class="code">0</code>, drawing to

the axis is disabled, ignoring <var class="var">fontfile</var> and <var class="var">axisfile</var> option.

Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">csp</samp></dt>

<dd> Set colorspace. The accepted values are:

<dl class="table">
<dt>‘<samp class="samp">unspecified</samp>’</dt>
<dd> Unspecified (default)

</dd>
<dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709

</dd>
<dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC

</dd>
<dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625

</dd>
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M or BT.601-6 525

</dd>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M

</dd>
<dt>‘<samp class="samp">bt2020ncl</samp>’</dt>
<dd> BT.2020 with non-constant luminance

</dd>
</dl>

</dd> <dt><samp class="option">cscheme</samp></dt> <dd> Set spectrogram color scheme. This is list of floating point values with format <code class="code">left_r|left_g|left_b|right_r|right_g|right_b</code>. The default is <code class="code">1|0.5|0|0|0.5|1</code>. </dd> </dl>

 <h4 class="subsection">48.22.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Playing audio while showing the spectrum: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with frame rate 30 fps: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=fps=30:count=5 [out0]' </pre>

Playing at 1280x720: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=s=1280x720:count=4 [out0]' </pre>

Disable sonogram display: <pre class="example-preformatted">sono_h=0 </pre>

A1 and its harmonics: A1, A2, (near)E3, A3: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6
asplit[a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with more accuracy in frequency domain: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6*PI*55*t)+0.1*sin(8*PI*55*t), asplit[a][out1]; [a] showcqt=timeclamp=0.5 [out0]' </pre>

Custom volume: <pre class="example-preformatted">bar_v=10:sono_v=bar_v*a_weighting(f) </pre>

Custom gamma, now spectrum is linear to the amplitude. <pre class="example-preformatted">bar_g=2:sono_g=2 </pre>

Custom tlength equation: <pre class="example-preformatted">tc=0.33:tlength='st(0,0.17); 384*tc / (384 / ld(0) + tc*f /(1-ld(0))) + 384*tc / (tc*f / ld(0) + 384 /(1-ld(0)))' </pre>

Custom fontcolor and fontfile, C-note is colored green, others are colored blue: <pre class="example-preformatted">fontcolor='if(mod(floor(midi(f)+0.5),12), 0x0000FF, g(1))':fontfile=myfont.ttf </pre>

Custom font using fontconfig: <pre class="example-preformatted">font='Courier New,Monospace,mono|bold'

</pre>

Custom frequency range with custom axis using image file: <pre class="example-preformatted">axisfile=myaxis.png:basefreq=40:endfreq=10000
</pre>

 <h3 class="section">48.23 showcwt# TOC</h3>

Convert input audio to video output representing frequency spectrum

using Continuous Wavelet Transform and Morlet wavelet.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">640x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set the frequency scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">bark</samp></dt>

<dt><samp class="option">mel</samp></dt>

<dt><samp class="option">erbs</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

<dt><samp class="option">fm</samp></dt>

</dl>

Default value is <code class="code">linear</code>.

</dd>

<dt><samp class="option">iscale</samp></dt>

<dd> Set the intensity scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

</dl>

Default value is <code class="code">log</code>.

</dd>

<dt><samp class="option">min</samp></dt>

<dd> Set the minimum frequency that will be used in output.

Default is <code class="code">20</code> Hz.

</dd>

<dt><samp class="option">max</samp></dt>

<dd> Set the maximum frequency that will be used in output.

Default is <code class="code">20000</code> Hz. The real frequency upper limit

depends on input audio’s sample rate and such will be enforced

on this value when it is set to value greater than Nyquist frequency.

</dd>

<dt><samp class="option">imin</samp></dt>

<dd> Set the minimum intensity that will be used in output.

</dd>

<dt><samp class="option">imax</samp></dt>

<dd> Set the maximum intensity that will be used in output.

</dd>

<dt><samp class="option">logb</samp></dt>

<dd> Set the logarithmic basis for brightness strength when

mapping calculated magnitude values to pixel values.

Allowed range is from <code class="code">0</code> to <code class="code">1</code>.

Default value is <code class="code">0.0001</code>.

</dd>

<dt><samp class="option">deviation</samp></dt>

<dd> Set the frequency deviation.

Lower values than <code class="code">1</code> are more frequency oriented,

while higher values than <code class="code">1</code> are more time oriented.

Allowed range is from <code class="code">0</code> to <code class="code">10</code>.

Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">pps</samp></dt>

<dd> Set the number of pixel output per each second in one row.

Allowed range is from <code class="code">1</code> to <code class="code">1024</code>.

Default value is <code class=”code”>64</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set the output visual mode. Allowed values are:

<dl class=”table”>

<dt><samp class=”option”>magnitude</samp></dt>

<dd> Show magnitude.

</dd>

<dt><samp class=”option”>phase</samp></dt>

<dd> Show only phase.

</dd>

<dt><samp class=”option”>magphase</samp></dt>

<dd> Show combination of magnitude and phase.

Magnitude is mapped to brightness and phase to color.

</dd>

<dt><samp class=”option”>channel</samp></dt>

<dd> Show unique color per channel magnitude.

</dd>

<dt><samp class=”option”>stereo</samp></dt>

<dd> Show unique color per stereo difference.

</dd>

</dl>

Default value is <code class=”code”>magnitude</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set the output slide method. Allowed values are: <dl class=”table”> <dt><samp class=”option”>replace</samp></dt> <dt><samp class=”option”>scroll</samp></dt> <dt><samp class=”option”>frame</samp></dt> </dl>

</dd> <dt><samp class=”option”>direction</samp></dt> <dd> Set the direction method for output slide method. Allowed values are: <dl class=”table”> <dt><samp class=”option”>lr</samp></dt> <dd> Direction from left to right. </dd> <dt><samp class=”option”>rl</samp></dt>

<dd> Direction from right to left. </dd> <dt><samp class=”option”>ud</samp></dt> <dd> Direction from up to down. </dd> <dt><samp class=”option”>du</samp></dt> <dd> Direction from down to up. </dd> </dl>

</dd> <dt><samp class=”option”>bar</samp></dt> <dd> Set the ratio of bargraph display to display size. Default is 0. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>48.24 showfreqs# TOC</h3>

Convert input audio to video output representing the audio power spectrum.

Audio amplitude is on Y-axis while frequency is on X-axis.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Specify size of video. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

Default is <code class=”code”>1024x512</code>.

</dd>

<dt><samp class=”option”>rate, r</samp></dt>

<dd> Set video rate. Default is <code class=”code”>25</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set display mode.

This set how each frequency bin will be represented.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">bar</samp>’</dt>
<dt>‘<samp class="samp">dot</samp>’</dt>
</dl>
Default is <code class="code">bar</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root scale.

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">fscale</samp></dt>
<dd> Set frequency scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.

</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> Reverse logarithmic scale.
</dd>
</dl>
Default is <code class="code">lin</code>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from 16 to 65536.

Default is <code class="code">2048</code>

</dd>
<dt><samp class="option">win_func</samp></dt>
<dd> Set windowing function.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt>

<dt>‘<samp class="samp">hanning</samp>’</dt>

<dt>‘<samp class="samp">hamming</samp>’</dt>

<dt>‘<samp class="samp">blackman</samp>’</dt>

<dt>‘<samp class="samp">welch</samp>’</dt>

<dt>‘<samp class="samp">flattop</samp>’</dt>

<dt>‘<samp class="samp">bharris</samp>’</dt>

<dt>‘<samp class="samp">bnuttall</samp>’</dt>

<dt>‘<samp class="samp">bhann</samp>’</dt>

<dt>‘<samp class="samp">sine</samp>’</dt>

<dt>‘<samp class="samp">nuttall</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hanning</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. In range <code class="code">[0, 1]</code>.
Default is <code class="code">1</code>,

which means optimal overlap for selected window function will be picked.

</dd>

<dt><samp class="option">averaging</samp></dt>

<dd> Set time averaging. Setting this to 0 will display current maximal peaks.

Default is <code class="code">1</code>, which means time averaging is disabled.

</dd>

<dt><samp class="option">colors</samp></dt>

<dd> Specify list of colors separated by space or by ’|’ which will be used to

draw channel frequencies. Unrecognized or missing colors will be replaced

by white color.

</dd>

<dt><samp class="option">cmode</samp></dt>

<dd> Set channel display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dt>‘<samp class="samp">separate</samp>’</dt>

</dl>

Default is <code class="code">combined</code>.

</dd>

<dt><samp class="option">minamp</samp></dt>

<dd> Set minimum amplitude used in <code class="code">log</code> amplitude scaler.

</dd>

<dt><samp class=”option”>data</samp></dt>

<dd> Set data display mode.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>magnitude</samp>’</dt>

<dt>‘<samp class=”samp”>phase</samp>’</dt>

<dt>‘<samp class=”samp”>delay</samp>’</dt>

</dl>

Default is <code class=”code”>magnitude</code>.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set channels to use when processing audio. By default all are processed.

</dd>

</dl>

 <h3 class=”section”>48.25 showspatial# TOC</h3>

Convert stereo input audio to a video output, representing the spatial relationship between two channels. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>512x512</code>. </dd> <dt><samp class=”option”>win_size</samp></dt> <dd> Set window size. Allowed range is from <var class=”var”>1024</var> to <var class=”var”>65536</var>. Default size is <var class=”var”>4096</var>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann</samp>’</dt> <dt>‘<samp class=”samp”>hanning</samp>’<
<dt>‘<samp class=”samp”>hamming</samp>’</dt>

<dt>‘<samp class=”samp”>blackman</samp>’</dt>
<dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp
class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’<
<dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp
class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt
<dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp
class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</
<dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp
class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</
<dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp
class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>&rsquo
<dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl>

Default value is <code class=”code”>hann</code>. </dd>
<dt><samp class=”option”>rate, r</samp></dt> <dd> Set output
framerate. </dd> </dl>

<a name=”showspectrum-
1”> <h3 class=”section”>48.26 showspectrum<span class=”pull-
right”>#
TOC</h3>

Convert input audio to a video output, representing the audio frequency spectrum. The filter accepts the following options:
<dl class=”table”> <dt><samp class=”option”>size, s</samp></dt>
<dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>640x512</code>.
</dd> <dt><samp class=”option”>slide</samp></dt> <dd> Specify how the spectrum should slide along the window. It accepts the following values: <dl class=”table”> <dt>‘<samp
class=”samp”>replace</samp>’</dt> <dd> the samples start again on the left when they reach the right </dd> <dt>‘<samp
class=”samp”>scroll</samp>’</dt> <dd> the samples scroll from right to left </dd> <dt>‘<samp class=”samp”>fullframe</samp>’</dt>
<dd> frames are only produced when the samples reach the right
</dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> the samples scroll from left to right </dd> <dt>‘<samp
class=”samp”>lreplace</samp>’</dt> <dd> the samples start again on the right when they reach the left </dd> </dl>

Default value is <code class=”code”>replace</code>. </dd>
<dt><samp class=”option”>mode</samp></dt> <dd> Specify display mode. It accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>combined</samp>’</dt>
<dd> all channels are displayed in the same row </dd> <dt>‘<samp
class=”samp”>separate</samp>’</dt> <dd> all channels are

displayed in separate rows </dd> </dl>

Default value is ‘<samp class=”samp”>combined</samp>’.
 </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify
display color mode. It accepts the following values: <dl
class=”table”> <dt>‘<samp class=”samp”>channel</samp>’</dt>
<dd> each channel is displayed in a separate color </dd>
<dt>‘<samp class=”samp”>intensity</samp>’</dt> <dd> each
channel is displayed using the same color scheme </dd> <dt>‘<samp
class=”samp”>rainbow</samp>’</dt> <dd> each channel is dis-
played using the rainbow color scheme </dd> <dt>‘<samp
class=”samp”>moreland</samp>’</dt> <dd> each channel is
displayed using the moreland color scheme </dd> <dt>‘<samp
class=”samp”>nebulae</samp>’</dt> <dd> each channel is
displayed using the nebulae color scheme </dd> <dt>‘<samp
class=”samp”>fire</samp>’</dt> <dd> each channel is dis-
played using the fire color scheme </dd> <dt>‘<samp
class=”samp”>fiery</samp>’</dt> <dd> each channel is
displayed using the fiery color scheme </dd> <dt>‘<samp
class=”samp”>fruit</samp>’</dt> <dd> each channel is
displayed using the fruit color scheme </dd> <dt>‘<samp
class=”samp”>cool</samp>’</dt> <dd> each channel is dis-
played using the cool color scheme </dd> <dt>‘<samp
class=”samp”>magma</samp>’</dt> <dd> each channel is
displayed using the magma color scheme </dd> <dt>‘<samp
class=”samp”>green</samp>’</dt> <dd> each channel is
displayed using the green color scheme </dd> <dt>‘<samp
class=”samp”>viridis</samp>’</dt> <dd> each channel is
displayed using the viridis color scheme </dd> <dt>‘<samp
class=”samp”>plasma</samp>’</dt> <dd> each channel is
displayed using the plasma color scheme </dd> <dt>‘<samp
class=”samp”>cividis</samp>’</dt> <dd> each channel is
displayed using the cividis color scheme </dd> <dt>‘<samp
class=”samp”>terrain</samp>’</dt> <dd> each channel is
displayed using the terrain color scheme </dd> </dl>

Default value is ‘<samp class=”samp”>channel</samp>’.
 </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Specify
scale used for calculating intensity color values. It accepts
the following values: <dl class=”table”> <dt>‘<samp
class=”samp”>lin</samp>’</dt> <dd> linear </dd>
<dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> square
root, default </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt>
<dd> cubic root </dd> <dt>‘<samp class=”samp”>log</samp>’</dt>
<dd> logarithmic </dd> <dt>‘<samp class=”samp”>4thrt</samp>’</dt>
<dd> 4th root </dd> <dt>‘<samp class=”samp”>5thrt</samp>’</dt>
<dd> 5th root </dd> </dl>

Default value is ‘<samp class="samp">sqrt</samp>’.
 </dd> <dt><samp class="option">fscale</samp></dt> <dd> Specify
frequency scale. It accepts the following values: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic </dd> </dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values
provide alternative color scheme. <code class="code">0</code> is no
saturation at all. Saturation must be in [-10.0, 10.0] range. Default
value is <code class="code">1</code>. </dd> <dt><samp
class="option">win_func</samp></dt> <dd> Set window func-
tion. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp
class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp
class="samp">hamming</samp>’</dt> <dt>‘<samp
class="samp">blackman</samp>’</dt> <dt>‘<samp
class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp
class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp
class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp
class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp
class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp
class="samp">kaiser</samp>’</dt> </dl>

Default value is <code class="code">hann</code>. </dd>
<dt><samp class="option">orientation</samp></dt> <dd> Set orien-
tation of time vs frequency axis. Can be <code class="code">vertical</code>
or <code class="code">horizontal</code>. Default is <code class="code">vertical</code>.
 </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set
ratio of overlap window. Default value is <code class="code">0</code>.
When value is <code class="code">1</code> overlap is set to recommended
size for specific window function currently used. </dd> <dt><samp
class="option">gain</samp></dt> <dd> Set scale gain for calculating
intensity color values. Default value is <code class="code">1</code>.
</dd> <dt><samp class="option">data</samp></dt> <dd> Set
which data to display. Can be <code class="code">magnitude</code>,
default or <code class="code">phase</code>, or unwrapped phase: <code
class="code">uphase</code>. </dd> <dt><samp class="option">rotation</samp></dt>
<dd> Set color rotation, must be in [-1.0, 1.0] range. Default

value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set upper frame rate limit. Default is <code class=”code”>auto</code>, unlimited. </dd> <dt><samp class=”option”>legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is disabled. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

The usage is very similar to the showwaves filter; see the examples in that section. <h4 class=”subsection”>48.26.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Large window with logarithmic color scaling: <pre class=”example-preformatted”>showspectrum=s=1280x480:scale=log </pre>

Complete example for a colored and sliding spectrum per channel using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] showspectrum=mode=separate:color=intensity:slide=1:scale=cbrt [out0]' </pre>

 <h3 class=”section”>48.27 showspectrumpic# TOC</h3>

Convert input audio to a single video frame, representing the audio frequency

spectrum.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">4096x2048</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Specify display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dd> all channels are displayed in the same row

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> all channels are displayed in separate rows

</dd>

</dl>

Default value is ‘<samp class="samp">combined</samp>’.

</dd>

<dt><samp class="option">color</samp></dt>

<dd> Specify display color mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">channel</samp>’</dt>

<dd> each channel is displayed in a separate color

</dd>

<dt>‘<samp class="samp">intensity</samp>’</dt>
<dd> each channel is displayed using the same color scheme
</dd>
<dt>‘<samp class="samp">rainbow</samp>’</dt>
<dd> each channel is displayed using the rainbow color scheme
</dd>
<dt>‘<samp class="samp">moreland</samp>’</dt>
<dd> each channel is displayed using the moreland color scheme
</dd>
<dt>‘<samp class="samp">nebulae</samp>’</dt>
<dd> each channel is displayed using the nebulae color scheme
</dd>
<dt>‘<samp class="samp">fire</samp>’</dt>
<dd> each channel is displayed using the fire color scheme
</dd>
<dt>‘<samp class="samp">fiery</samp>’</dt>
<dd> each channel is displayed using the fiery color scheme
</dd>
<dt>‘<samp class="samp">fruit</samp>’</dt>
<dd> each channel is displayed using the fruit color scheme
</dd>
<dt>‘<samp class="samp">cool</samp>’</dt>
<dd> each channel is displayed using the cool color scheme
</dd>
<dt>‘<samp class="samp">magma</samp>’</dt>
<dd> each channel is displayed using the magma color scheme
</dd>
<dt>‘<samp class="samp">green</samp>’</dt>
<dd> each channel is displayed using the green color scheme
</dd>
<dt>‘<samp class="samp">viridis</samp>’</dt>

<dd> each channel is displayed using the viridis color scheme
</dd>
<dt>‘<samp class="samp">plasma</samp>’</dt>
<dd> each channel is displayed using the plasma color scheme
</dd>
<dt>‘<samp class="samp">cividis</samp>’</dt>
<dd> each channel is displayed using the cividis color scheme
</dd>
<dt>‘<samp class="samp">terrain</samp>’</dt>
<dd> each channel is displayed using the terrain color scheme
</dd>
</dl>
Default value is ‘<samp class="samp">intensity</samp>’.

</dd>
<dt><samp class="option">scale</samp></dt>
<dd> Specify scale used for calculating intensity color values.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> square root, default
</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic

</dd>

<dt>‘<samp class="samp">4thrt</samp>’</dt>

<dd> 4th root

</dd>

<dt>‘<samp class="samp">5thrt</samp>’</dt>

<dd> 5th root

</dd>

</dl>

Default value is ‘<samp class="samp">log</samp>’.

</dd>

<dt><samp class="option">fscale</samp></dt>

<dd> Specify frequency scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">lin</samp>’</dt>

<dd> linear

</dd>

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

</dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values provide alternative color scheme. <code class="code">0</code> is no saturation at all. Saturation must be in [-10.0, 10.0] range. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp

class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt> <dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</dt> <dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>’</dt> <dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl> Default value is <code class=”code”>hann</code>. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of time vs frequency axis. Can be <code class=”code”>vertical</code> or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set scale gain for calculating intensity color values. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is enabled. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

 <h4 class=”subsection”>48.27.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Extract an audio spectrogram of a whole audio track in a 1024x1024 picture using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i audio.flac -lavfi showspectrumpic=s=1024x1024 spectrogram.png </pre>

 <h3 class="section">48.28 showvolume# TOC</h3>

Convert input audio volume to a video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set video rate. </dd> <dt><samp class="option">b</samp></dt> <dd> Set border width, allowed range is [0, 5]. Default is 1. </dd> <dt><samp class="option">w</samp></dt> <dd> Set channel width, allowed range is [80, 8192]. Default is 400. </dd> <dt><samp class="option">h</samp></dt> <dd> Set channel height, allowed range is [1, 900]. Default is 20. </dd> <dt><samp class="option">f</samp></dt> <dd> Set fade, allowed range is [0, 1]. Default is 0.95. </dd> <dt><samp class="option">c</samp></dt> <dd> Set volume color expression. The expression can use the following variables: <dl class="table"> <dt><samp class="option">VOLUME</samp></dt> <dd> Current max volume of channel in dB. </dd> <dt><samp class="option">PEAK</samp></dt> <dd> Current peak. </dd> <dt><samp class="option">CHANNEL</samp></dt> <dd> Current channel number, starting from 0. </dd> </dl>

</dd> <dt><samp class="option">t</samp></dt> <dd> If set, displays channel names. Default is enabled. </dd> <dt><samp class="option">v</samp></dt> <dd> If set, displays volume values. Default is enabled. </dd> <dt><samp class="option">o</samp></dt> <dd> Set orientation, can be horizontal: <code class="code">h</code> or vertical: <code class="code">v</code>, default is <code class="code">h</code>. </dd> <dt><samp class="option">s</samp></dt> <dd> Set step size, allowed range is [0, 5]. Default is 0, which means step is disabled. </dd> <dt><samp class="option">p</samp></dt> <dd> Set background opacity, allowed range is [0, 1]. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set metering mode, can be peak: <code class="code">p</code> or rms: <code class="code">r</code>, default is <code class="code">p</code>. </dd> <dt><samp class="option">ds</samp></dt> <dd> Set display scale, can be linear: <code class="code">lin</code> or log: <code class="code">log</code>, default is <code class="code">lin</code>. </dd> <dt><samp class="option">dm</samp></dt> <dd> In second. If set to > 0., display a line for the max level in the previous seconds. default is disabled: <code class="code">0.</code> </dd> <dt><samp class="option">dmc</samp></dt> <dd> The color of the max line. Use when <code class="code">dm</code> option is set to > 0. default is: <code class="code">orange</code> </dd> </dl>

 <h3 class="section">48.29 showwaves<span

class=”pull-right”># TOC</h3>

Convert input audio to a video output, representing the samples waves. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>600x240</code>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set display mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Draw a point for each sample. </dd> <dt>‘<samp class=”samp”>line</samp>’</dt> <dd> Draw a vertical line for each sample. </dd> <dt>‘<samp class=”samp”>p2p</samp>’</dt> <dd> Draw a point for each sample and a line between them. </dd> <dt>‘<samp class=”samp”>cline</samp>’</dt> <dd> Draw a centered vertical line for each sample. </dd> </dl>

Default value is <code class=”code”>point</code>. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of samples which are printed on the same column. A larger value will decrease the frame rate. Must be a positive integer. This option can be set only if the value for <var class=”var”>rate</var> is not explicitly specified. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the (approximate) output frame rate. This is done by setting the option <var class=”var”>n</var>. Default value is "25". </dd> <dt><samp class=”option”>split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class=”option”>draw</samp></dt> <dd> Set the draw mode. This is mostly useful to set for high <var class=”var”>n</var>. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> </dl>

 <h4 class="subsection">48.29.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output the input file audio and the corresponding video representation at the same time: <pre class="example-preformatted">amovie=a.mp3,asplit[out0],showwaves[out1]
</pre>

Create a synthetic signal and show it with showwaves, forcing a frame rate of 30 frames per second: <pre class="example-preformatted">aevalsrc=sin(1*2*PI*t)*sin(880*2*PI*t):cos(2*PI*200*t),asplit[out0],showwaves=r=30[out1]
</pre>

 <h3 class="section">48.30 showwavespic# TOC</h3>

Convert input audio to a single video frame, representing the samples waves. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">600x240</code>. </dd> <dt><samp class="option">split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class="option">colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class="table"> <dt>‘<samp class="samp">lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class="option">draw</samp></dt> <dd> Set the draw mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> <dt><samp class="option">filter</samp></dt> <dd> Set the filter mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">average</samp>’</dt> <dd> Use average samples values for each drawn sample. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Use peak samples values for each drawn sample. </dd> </dl>

Default value is <code class="code">average</code>. </dd> </dl>

 <h4 class="subsection">48.30.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract a channel split representation of the wave form of a whole audio track in a 1024x800 picture using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i audio.flac -lavfi showwavespic=split_channels=1:s=1024x800 waveform.png </pre>

 <h3 class="section">48.31 sidedata, asidedata# TOC</h3

Delete frame side data, or select frames based on it. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">select</samp>’</dt> <dd> Select every frame with side data of <code class="code">type</code>. </dd> <dt>‘<samp class="samp">delete</samp>’</dt> <dd> Delete side data of <code class="code">type</code>. If <code class="code">type</code> is not set, delete all side data in the frame. </dd> </dl>

</dd> <dt><samp class="option">type</samp></dt> <dd> Set side data type used with all modes. Must be set for <code class="code">select</code> mode. Possible values are: <dl class="table"> <dt>‘<samp class="samp">PANSCAN</samp>’</dt> <dt>‘<samp class="samp">A53_CC</samp>’</dt> <dt>‘<samp class="samp">STEREO3D</samp>’</dt> <dt>‘<samp class="samp">MATRIXENCODING</samp>’</dt> <dt>‘<samp class="samp">DOWNMIX_INFO</samp>’</dt> <dt>‘<samp

class=”samp”>REPLAYGAIN</samp>’</dt> <dt>‘<samp
class=”samp”>DISPLAYMATRIX</samp>’</dt> <dt>‘<samp
class=”samp”>AFD</samp>’</dt> <dt>‘<samp class=”samp”>MOTION_VECTORS</sa
<dt>‘<samp class=”samp”>SKIP_SAMPLES</samp>’</dt>
<dt>‘<samp class=”samp”>AUDIO_SERVICE_TYPE</samp>’</dt>
<dt>‘<samp class=”samp”>MASTERING_DISPLAY_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>GOP_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>SPHERICAL</samp>’</dt>
<dt>‘<samp class=”samp”>CONTENT_LIGHT_LEVEL</samp>’</dt>
<dt>‘<samp class=”samp”>ICC_PROFILE</samp>’</dt>
<dt>‘<samp class=”samp”>S12M_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_PLUS</samp>’</dt>
<dt>‘<samp class=”samp”>REGIONS_OF_INTEREST</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_ENC_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>SEI_UNREGISTERED</samp>’</dt>
<dt>‘<samp class=”samp”>FILM_GRAIN_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BOUNDING_BOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BBOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_RPU_BUFFER</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_VIVID</samp>’</dt>
<dt>‘<samp class=”samp”>AMBIENT_VIEWING_ENVIRONMENT</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_HINT</samp>’</dt>
<dt>‘<samp class=”samp”>GAIN_MAP_PARAMS</samp>’</dt>
</dl>

</dd> </dl>

 <h3 class=”section”>48.32 spectrumsynth# TOC</h3>

Synthesize audio from 2 input video spectrums, first input stream represents magnitude across time and second represents phase across time. The filter will transform from frequency domain as displayed in videos back to time domain as presented in audio output.
This filter is primarily created for reversing processed showspectrum filter outputs, but can synthesize sound from other spectrograms too. But in such case results are going to be poor if the phase data is not available, because in such cases phase data need to be recreated, usually it’s just recreated from random noise. For best results use gray only output (<code class=”code”>channel</code> color mode in showspectrum filter) and <code class=”code”>log</code> scale for magnitude video and <code class=”code”>lin</code> scale for phase video. To produce phase, for 2nd video, use <code class=”code”>data</code> option. Inputs

videos should generally use <code class="code">fullframe</code> slide mode as that saves resources needed for decoding video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Specify sample rate of output audio, the sample rate of audio from which spectrum was generated may differ. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set number of channels represented in input video spectrums. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set scale which was used when generating magnitude input spectrum. Can be <code class="code">lin</code> or <code class="code">log</code>. Default is <code class="code">log</code>. </dd> <dt><samp class="option">slide</samp></dt> <dd> Set slide which was used when generating inputs spectrums. Can be <code class="code">replace</code>, <code class="code">scroll</code>, <code class="code">fullframe</code> or <code class="code">rscroll</code>. Default is <code class="code">fullframe</code>. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function used for resynthesis. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set window overlap. In range <code class="code">[0, 1]</code>. Default is <code class="code">1</code>, which means optimal overlap for selected window function will be picked. </dd> <dt><samp class="option">orientation</samp></dt> <dd> Set orientation of input videos. Can be <code class="code">vertical</code> or <code class="code">horizontal</code>. Default is <code class="code">vertical</code>. </dd> </dl>

 <h4 class="subsection">48.32.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> First create magnitude and phase videos from audio, assuming audio is stereo with 44100 sample rate, then resynthesize videos back to audio with spectrumsynth: <pre class="example-preformatted">ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=log:overlap=0.875:color=channel:slide=fullframe:data=magnitude -an -c:v rawvideo magnitude.nut ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=lin:overlap=0.875:color=channel:slide=fullframe:data=phase -an -c:v rawvideo phase.nut ffmpeg -i magnitude.nut -i phase.nut -lavfi spectrumsynth=channels=2:sample_rate=44100:win_func=hann:overlap=0.875:slide=fullframe output.flac </pre>

 <h3 class="section">48.33 split, asplit# TOC</h3>

Split input into several identical outputs. <code class="code">asplit</code>

works with audio input, <code class="code">split</code> with video.
The filter accepts a single parameter which specifies the number of outputs. If unspecified, it defaults to 2.
<h4 class="subsection">48.33.1 Examples#
TOC</h4>

<ul class="itemize mark-bullet"> Create two separate outputs from the same input: <pre class="example-preformatted">[in] split [out0][out1] </pre>

To create 3 or more outputs, you need to specify the number of outputs, like in: <pre class="example-preformatted">[in] asplit=3 [out0][out1][out2] </pre>

Create two separate outputs from the same input, one cropped and one padded: <pre class="example-preformatted">[in] split [splitout1][splitout2]; [splitout1] crop=100:100:0:0 [cropout]; [splitout2] pad=200:200:100:100 [padout]; </pre>

Create 5 copies of the input audio with <code class="command">ffmpeg</code>:
 <pre class="example-preformatted">ffmpeg -i INPUT -filter_complex asplit=5 OUTPUT </pre>

 <h3 class="section">48.34 zmq, azmq# TOC</h3>

Receive commands sent through a libzmq client, and forward them to filters in the filtergraph. <code class="code">zmq</code> and <code class="code">azmq</code> work as a pass-through filters. <code class="code">zmq</code> must be inserted between two video filters, <code class="code">azmq</code> between two audio filters. Both are capable to send messages to any filter type. To enable these filters you need to install the libzmq library and headers and configure FFmpeg with <code class="code">--enable-libzmq</code>. For more information about libzmq see: http://www.zeromq.org/
The <code class="code">zmq</code> and <code class="code">azmq</code> filters work as a libzmq server, which receives messages sent through a network interface defined by the <samp class="option">bind_address</samp> (or the abbreviation "<samp class="option">b</samp>") option. Default value of this option is <samp class="file">tcp://localhost:5555</samp>. You may want to alter this value to your needs, but do not forget to escape any ’:’ signs (see filtergraph escaping). The received message must be in the form: <pre class="example-preformatted"><var class="var">TARGET</var> <var class="var">COMMAND</var>

[<var class=”var”>ARG</var>] </pre>

<var class=”var”>TARGET</var> specifies the target of the command, usually the name of the filter class or a specific filter instance name. The default filter instance name uses the pattern ‘<samp class=”samp”>Parsed_<filter_name>_<index></samp>’, but you can override this by using the ‘<samp class=”samp”>filter_name@id</samp>’ syntax (see Filtergraph syntax). <var class=”var”>COMMAND</var> specifies the name of the command for the target filter. <var class=”var”>ARG</var> is optional and specifies the optional argument list for the given <var class=”var”>COMMAND</var>. Upon reception, the message is processed and the corresponding command is injected into the filtergraph. Depending on the result, the filter will send a reply to the client, adopting the format: <pre class=”example-preformatted”><var class=”var”>ERROR_CODE</var> <var class=”var”>ERROR_REASON</var> <var class=”var”>MESSAGE</var> </pre>

<var class=”var”>MESSAGE</var> is optional. <h4 class=”subsection”>48.34.1 Examples# TOC</h4>

Look at <samp class=”file”>tools/zmqsend</samp> for an example of a zmq client which can be used to send commands processed by these filters. Consider the following filtergraph generated by <code class=”command”>ffplay</code>. In this example the last overlay filter has an instance name. All other filters will have default instance names.   <pre class=”example-preformatted”>ffplay -dumpgraph 1 -f lavfi " color=s=100x100:c=red [l]; color=s=100x100:c=blue [r]; nullsrc=s=200x100, zmq [bg]; [bg][l] overlay [bg+l]; [bg+l][r] overlay@my=x=100 " </pre>

To change the color of the left side of the video, the following command can be used: <pre class=”example-preformatted”>echo Parsed_color_0 c yellow | tools/zmqsend </pre>

To change the right side: <pre class=”example-preformatted”>echo Parsed_color_1 c pink | tools/zmqsend </pre>

To change the position of the right side: <pre class=”example-preformatted”>echo overlay@my x 150 | tools/zmqsend </pre>

 <h2 class=”chapter”>49 Multimedia Sources# TOC</h2>

Below is a description of the currently available multimedia sources. <h3 class="section">49.1 amovie# TOC</h3>

This is the same as movie source, except it selects an audio stream by default. <h3 class="section">49.2 avsynctest# TOC</h3> Generate an Audio/Video Sync Test. Generated stream periodically shows flash video frame and emits beep in audio. Useful to inspect A/V sync issues. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set output video size. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">framerate, fr</samp></dt> <dd> Set output video frame rate. Default value is <code class="code">30</code>. </dd> <dt><samp class="option">samplerate, sr</samp></dt> <dd> Set output audio sample rate. Default value is <code class="code">44100</code>. </dd> <dt><samp class="option">amplitude, a</samp></dt> <dd> Set output audio beep amplitude. Default value is <code class="code">0.7</code>. </dd> <dt><samp class="option">period, p</samp></dt> <dd> Set output audio beep period in seconds. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">delay, dl</samp></dt> <dd> Set output video flash delay in number of frames. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">cycle, c</samp></dt> <dd> Enable cycling of video delays, by default is disabled. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set stream output duration. By default duration is unlimited. </dd> <dt><samp class="option">fg, bg, ag</samp></dt> <dd> Set foreground/background/additional color. </dd> </dl>

 <h4 class="subsection">49.2.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">49.3 movie# TOC</h3>

Read audio and/or video stream(s) from a movie container. It accepts the following parameters: <dl class="table"> <dt><samp

class=”option”>filename</samp></dt> <dd> The name of the resource to read (not necessarily a file; it can also be a device or a stream accessed through some protocol). </dd> <dt><samp class=”option”>format_name, f</samp></dt> <dd> Specifies the format assumed for the movie to read, and can be either the name of a container or an input device. If not specified, the format is guessed from <var class=”var”>movie_name</var> or by probing. </dd> <dt><samp class=”option”>seek_point, sp</samp></dt> <dd> Specifies the seek point in seconds. The frames will be output starting from this seek point. The parameter is evaluated with <code class=”code”>av_strtod</code>, so the numerical value may be suffixed by an IS postfix. The default value is "0". </dd> <dt><samp class=”option”>streams, s</samp></dt> <dd> Specifies the streams to read. Several streams can be specified, separated by "+". The source will then have as many outputs, in the same order. The syntax is explained in the <a data-manual=”ffmpeg” href=”./ffmpeg.html#Stream-specifiers”>"Stream specifiers" section in the ffmpeg manual. Two special names, "dv" and "da" specify respectively the default (best suited) video and audio stream. Default is "dv", or "da" if the filter is called as "amovie". </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Specifies the index of the video stream to read. If the value is -1, the most suitable video stream will be automatically selected. The default value is "-1". Deprecated. If the filter is called "amovie", it will select audio instead of video. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> Specifies how many times to read the stream in sequence. If the value is 0, the stream will be looped infinitely. Default value is "1". Note that when the movie is looped the source timestamps are not changed, so it will generate non monotonically increasing timestamps. </dd> <dt><samp class=”option”>discontinuity</samp></dt> <dd> Specifies the time difference between frames above which the point is considered a timestamp discontinuity which is removed by adjusting the later timestamps. </dd> <dt><samp class=”option”>dec_threads</samp></dt> <dd> Specifies the number of threads for decoding </dd> <dt><samp class=”option”>format_opts</samp></dt> <dd> Specify format options for the opened file. Format options can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to add protocol_whitelist and protocol_blacklist options: <pre class=”example-preformatted”>ffplay -f lavfi "movie=filename='1.sdp':format_opts='protocol_whitelist=file,rtp,udp\:protocol_blacklist=http'&quo
</pre> </dd> </dl>

It allows overlaying a second video on top of the main input of a filtergraph, as shown in this graph: <pre class=”example-preformatted”>input ----------> deltapts0 --> overlay

--> output ^ | movie --> scale--> deltapts1 -------+ </pre> <h4 class="subsection">49.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Skip 3.2 seconds from the start of the AVI file in.avi, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=in.avi:seek_point=3.2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read from a video4linux2 device, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=/dev/video0:f=video4linux2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read the first video stream and the audio stream with id 0x81 from dvd.vob; the video is connected to the pad named "video" and the audio is connected to the pad named "audio": <pre class="example-preformatted">movie=dvd.vob:s=v:0+#0x81 [video] [audio] </pre>

 <h4 class="subsection">49.3.2 Commands# TOC</h4>

Both movie and amovie support the following commands: <dl class="table"> <dt><samp class="option">seek</samp></dt> <dd> Perform seek using "av_seek_frame". The syntax is: seek <var class="var">stream_index</var>|<var class="var">timestamp</var>|<var class="var">flags</var> <ul class="itemize mark-bullet"> <var class="var">stream_index</var>: If stream_index is -1, a default stream is selected, and <var class="var">timestamp</var> is automatically converted from AV_TIME_BASE units to the stream specific time_base. <var class="var">timestamp</var>: Timestamp in AVStream.time_base units or, if no stream is specified, in AV_TIME_BASE units. <var class="var">flags</var>: Flags which select direction and seeking mode.

</dd> <dt><samp class="option">get_duration</samp></dt> <dd> Get movie duration in AV_TIME_BASE units. </dd> </dl>

 <h2 class="chapter">50 External libraries# TOC</h2>

FFmpeg can be hooked up with a number of external libraries to add support for more formats. None of them are used by default, their use has to be explicitly requested by passing the appropriate flags to <code class=”command”>./configure</code>. <h3 class=”section”>50.1 Alliance for Open Media (AOM)# TOC</h3>

FFmpeg can make use of the AOM library for AV1 decoding and encoding. Go to http://aomedia.org/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libaom</code> to configure to enable it. <h3 class=”section”>50.2 ARM ASTC Encoder# TOC</h3>

FFmpeg can make use of the Arm ASTC Encoder library for ASTC image compression and decompression in the <code class=”code”>libastcenc</code> encoder and decoder. Go to https://github.com/ARM-software/astc-encoder to obtain the sources. Version 5.4 or newer is required, because the wrappers use the four-argument <code class=”code”>astcenc_context_alloc()</code> interface introduced in that release. The upstream build does not install the public header, and the library file name depends on the configured SIMD and linkage variants, so install the pieces manually in a layout that configure can detect: the header as <code class=”code”>astcenc/astcenc.h</code> and a library that can be linked with <code class=”code”>-lastcenc</code>. Keep the library’s own file name, which is the SONAME recorded in the binary, and provide the plain name as an alias: <pre class=”example-preformatted”>cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DASTCENC_SHAREDLIB=ON cmake --build build install -Dm644 Source/astcenc.h /usr/local/include/astcenc/astcenc.h install -Dm755 build/Source/libastcenc-native-shared.so \ /usr/local/lib/libastcenc-native-shared.so ln -sf libastcenc-native-shared.so /usr/local/lib/libastcenc.so </pre>

Use the platform’s shared library suffix, and adjust the built library name for other variants (for example <samp class=”file”>libastcenc-neon-shared.so</samp> or a static <samp class=”file”>libastcenc.a</samp>), or pass <code class=”code”>--extra-cflags</code> and <code class=”code”>--extra-ldflags</code> to configure if the files are installed elsewhere. Installing only under the plain name breaks linking at run time, and the installation directory has to be in the runtime loader’s search

path. On big-endian systems, also pass <code class="code">-DASTCENC_ISA_NONE=ON</code> and <code class="code">-DASTCENC_BIG_ENDIAN=ON</code> to CMake. For this variant, install <samp class="file">libastcenc-none-shared.so</samp> under its own name and provide <samp class="file">libastcenc.so</samp> as an alias, as in the example above. The library is licensed under Apache-2.0, which requires version 3 of the (L)GPL, so pass <code class="code">--enable-version3</code> and <code class="code">--enable-libastcenc</code> to configure to enable it. <h3 class="section">50.3 AMD AMF/VCE# TOC</h3>

FFmpeg can use the AMD Advanced Media Framework library for accelerated H.264 and HEVC(only windows) encoding on hardware with Video Coding Engine (VCE). To enable support you must obtain the AMF framework header files(version 1.4.9+) from https://github.com/GPUOpen-LibrariesAndSDKs/AMF.git. Create an <code class="code">AMF/</code> directory in the system include path. Copy the contents of <code class="code">AMF/amf/public/include/</code> into that directory. Then configure FFmpeg with <code class="code">--enable-amf</code>. Initialization of amf encoder occurs in this order: 1) trying to initialize through dx11(only windows) 2) trying to initialize through dx9(only windows) 3) trying to initialize through vulkan To use h.264(AMD VCE) encoder on linux amdgru-pro version 19.20+ and amf-amdgpu-pro package(amdgru-pro contains, but does not install automatically) are required. This driver can be installed using amdgpu-pro-install script in official amd driver archive. <h3 class="section">50.4 AviSynth# TOC</h3>

FFmpeg can read AviSynth scripts as input. To enable support, pass <code class="code">--enable-avisynth</code> to configure after installing the headers provided by AviSynth+. AviSynth+ can be configured to install only the headers by either passing <code class="code">-DHEADERS_ONLY:bool=on</code> to the normal CMake-based build system, or by using the supplied <code class="code">GNUmakefile</code>. For Windows, supported AviSynth variants are AviSynth 2.6 RC1 or higher for 32-bit builds and Avi
r1718 or higher for 32-bit and 64-bit builds. For Linux, macOS, and BSD, the only supported AviSynth variant is AviSynth+, start-

ing with version 3.5.   In 2016, AviSynth+ added support for building with GCC. However, due to the eccentricities of Windows’ calling conventions, 32-bit GCC builds of AviSynth+ are not compatible with typical 32-bit builds of FFmpeg. By default, FFmpeg assumes compatibility with 32-bit MSVC builds of AviSynth+ since that is the most widely-used and entrenched build configuration. Users can override this and enable support for 32-bit GCC builds of AviSynth+ by passing <code class=”code”>-DAVSC_WIN32_GCC32</code> to <code class=”code”>--extra-cflags</code> when configuring FFmpeg. 64-bit builds of FFmpeg are not affected, and can use either MSVC or GCC builds of AviSynth+ without any special flags.   AviSynth(+) is loaded dynamically. Distributors can build FFmpeg with <code class=”code”>--enable-avisynth</code>, and the binaries will work regardless of the end user having AviSynth installed. If/when an end user would like to use AviSynth scripts, then they can install AviSynth(+) and FFmpeg will be able to find and use it to open scripts. <h3 class=”section”>50.5 Chromaprint# TOC</h3>

FFmpeg can make use of the Chromaprint library for generating audio fingerprints. Pass <code class=”code”>--enable-chromaprint</code> to configure to enable it. See https://acoustid.org/chromapri
  <h3 class=”section”>50.6 codec2# TOC</h3>

FFmpeg can make use of the codec2 library for codec2 decoding and encoding. There is currently no native decoder, so libcodec2 must be used for decoding. Go to http://freedv.org/, download "Codec 2 source archive". Build and install using CMake. Debian users can install the libcodec2-dev package instead. Once libcodec2 is installed you can pass <code class=”code”>--enable-libcodec2</code> to configure to enable it. The easiest way to use codec2 is with .c2 files, since they contain the mode information required for decoding. To encode such a file, use a .c2 file extension and give the libcodec2 encoder the -mode option: <code class=”code”>ffmpeg -i input.wav -mode 700C output.c2</code>. Playback is as simple as <code class=”code”>ffplay output.c2</code>. For a list of supported modes, run <code class=”code”>ffmpeg -h encoder=libcodec2</code>. Raw codec2 files are also supported. To make sense of them the mode in use needs to be specified as a format option: <code class=”code”>ffmpeg -f codec2raw -mode 1300 -i input.raw output.wav</code>. <h3 class=”section”>50.7 dav1d# <a class=”anchor hidden-xs”href=”#toc-dav1d”

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the dav1d library for AV1 video decoding.
Go to https://code.videolan.org/videol
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libdav1d</code> to configure to enable it.
 <h3 class=”section”>50.8 davs2<span class=”pull-
right”>#
TOC</h3>

FFmpeg can make use of the davs2 library for AVS2-P2/IEEE1857.4 video
decoding. Go to https://github.com/pku
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libdavs2</code> to configure to enable it. <div
class=”info”> libdavs2 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>50.9
uavs3d<a class=”anchor hidden-xs” href=”#uavs3d”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-uavs3d”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the uavs3d library for AVS3-P2/IEEE1857.10
video decoding. Go to https://github.co
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libuavs3d</code> to configure to enable it. <a
name=”Game-Music-Emu”> <h3 class=”section”>50.10 Game Music
Emu<a class=”anchor hidden-xs” href=”#Game-
Music-Emu” aria-hidden=”true”># <a class=”anchor hidden-
xs”href=”#toc-Game-Music-Emu” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the Game Music Emu library to read audio
from supported video game music file formats. Pass <code class=”code”>-
-enable-libgme</code> to configure to enable it. See <a class=”url”
href=”https://bitbucket.org/mpyne/game-music-emu/overview”>https://bitbucket.org/mpyne/game-
music-emu/overview.
<h3 class=”section”>50.11 Intel QuickSync Video<span class=”pull-
right”><a class=”anchor hidden-xs” href=”#Intel-QuickSync-Video”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Intel-
QuickSync-Video” aria-hidden=”true”>TOC</h3>

FFmpeg can use Intel QuickSync Video (QSV) for accelerated decoding
and encoding of multiple codecs. To use QSV, FFmpeg must be linked
against the <code class=”code”>libmfx</code> dispatcher, which loads
the actual decoding libraries. The dispatcher is open source
and can be downloaded from <a class=”url” href=”https://github.com/lu-
zero/mfx_dispatch.git”>https://github.com/lu-zero/mfx_dispatch.git.

FFmpeg needs to be configured with the <code class="code">--enable-libmfx</code> option and <code class="code">pkg-config</code> needs to be able to locate the dispatcher’s <code class="code">.pc</code> files. <h3 class="section">50.12 Kvazaar# TOC</h3>

FFmpeg can make use of the Kvazaar library for HEVC encoding. Go to <a class="url" href="https://github.com/ultravideo/kvazaar">https://github.com/ultravideo/kvaz and follow the instructions for installing the library. Then pass <code class="code">--enable-libkvazaar</code> to configure to enable it. <h3 class="section">50.13 LAME# TOC</h3>

FFmpeg can make use of the LAME library for MP3 encoding. Go to http://lame.sourceforge.net/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libmp3lame</code> to configure to enable it. <h3 class="section">50.14 LCEVCdec# TOC</h3>

FFmpeg can make use of the liblcevc_dec library for LCEVC enhancement layer decoding on supported bitstreams. Go to https://github.com/v-novaltd/LCEVCdec and follow the instructions for installing the library. Then pass <code class="code">--enable-liblcevc-dec</code> to configure to enable it.   LCEVCdec is under the BSD-3-Clause-Clear License. <h3 class="section">50.15 libilbc# TOC</h3>

iLBC is a narrowband speech codec that has been made freely available by Google as part of the WebRTC project. libilbc is a packaging friendly copy of the iLBC codec. FFmpeg can make use of the libilbc library for iLBC decoding and encoding. Go to https://github.com/TimothyGu/libilbc and follow the instructions for installing the library. Then pass <code class="code">--enable-libilbc</code> to configure to enable it. <h3 class="section">50.16 libjxl# TOC</h3>

JPEG XL is an image format intended to fully replace legacy JPEG for an extended period of life. See https://jpegxl.info/ for more information, and see <a class="url" href="https://github.com/libjxl/libjxl">https://github.com/libj for the library source. You can pass <code class="code">--enable-libjxl</code> to configure in order enable the libjxl wrapper. <h3 class="section">50.17 libvpx# TOC</h3>

FFmpeg can make use of the libvpx library for VP8/VP9 decoding and encoding. Go to <a class="url" href="http://www.webmproject.org/">http://www.webmproject.org and follow the instructions for installing the library. Then pass <code class="code">--enable-libvpx</code> to configure to enable it. <h3 class="section">50.18 ModPlug# TOC</h3>

FFmpeg can make use of this library, originating in Modplug-XMMS, to read from MOD-like music files. See <a class="url" href="https://github.com/Konstanty/libmodplug">https:/ Pass <code class="code">--enable-libmodplug</code> to configure to enable it. <h3 class="section">50.19 ONNX Runtime# TOC</h3>

FFmpeg can make use of the ONNX Runtime library as a backend for DNN based filters. Go to https://onnxruntime.ai/ and follow the instructions for installing the library. Pre-built packages are available for various platforms. Configure FFmpeg with ONNX Runtime support: <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib" </pre>

On ELF-based systems (Linux, BSD) you may also want to embed the library search path so that the installed <samp class="file">ffmpeg</samp> binary finds the shared library at run time without setting <code class="env">LD_LIBRARY_PATH</code>:   <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib -Wl,-rpath,/path/to/onnxruntime/lib" </pre>

The <code class="code">-Wl,-rpath</code> flag is ELF-specific and should be omitted on other platforms (Windows, macOS). For MinGW GCC builds, add the <code class="code">-D_stdcall=___stdcall</code>

flag for compatibility with ONNX Runtime headers. <h3 class="section">50.20 OpenCORE, VisualOn, and Fraunhofer libraries# TOC</h3>

Spun off Google Android sources, OpenCore, VisualOn and Fraunhofer libraries provide encoders for a number of audio codecs.   OpenCORE and VisualOn libraries are under the Apache License 2.0 (see http://www.apache.org/licenses/LICENSE-2.0 for details), which is incompatible to the LGPL version 2.1 and GPL version 2. You have to upgrade FFmpeg’s license to LGPL version 3 (or if you have enabled GPL components, GPL version 3) by passing <code class="code">--enable-version3</code> to configure in order to use it. The license of the Fraunhofer AAC library is incompatible with the GPL. Therefore, for GPL builds, you have to pass <code class="code">--enable-nonfree</code> to configure in order to use it. To the best of our knowledge, it is compatible with the LGPL. <h4 class="subsection">50.20.1 OpenCORE AMR# TOC</h4>

FFmpeg can make use of the OpenCORE libraries for AMR-NB decoding/encoding and AMR-WB decoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the libraries. Then pass <code class="code">--enable-libopencore-amrnb</code> and/or <code class="code">--enable-libopencore-amrwb</code> to configure to enable them. <h4 class="subsection">50.20.2 VisualOn AMR-WB encoder library# TOC</h4>

FFmpeg can make use of the VisualOn AMR-WBenc library for AMR-WB encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libvo-amrwbenc</code> to configure to enable it. <h4 class="subsection">50.20.3 Fraunhofer AAC library#

TOC</h4>

FFmpeg can make use of the Fraunhofer AAC library for AAC decoding & encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-aac</code> to configure to enable it. <h4 class=”subsection”>50.20.4 Fraunhofer xHE-AAC library# TOC</h4>

FFmpeg can make use of the Fraunhofer xHE-AAC library for xHE-AAC encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-xhe-aac</code> to configure to enable it. <h4 class=”subsection”>50.20.5 Fraunhofer MPEG-H 3D Audio decoder library# TOC</h4>

FFmpeg can make use of the Fraunhofer MPEG-H decoder library for MPEG-H 3DA decoding. Go to https://github.com/Fraunhofer-IIS/mpeghdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libmpeghdec --enable-nonfree</code> to configure to enable it. <h4 class=”subsection”>50.20.6 LC3 library# TOC</h4>

FFmpeg can make use of the Google LC3 library for LC3 decoding & encoding. Go to <a class=”url” href=”https://github.com/google/liblc3/”>https://github.com/go and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblc3</code> to configure to enable it. <h3 class=”section”>50.21 OpenH264# TOC</h3>

FFmpeg can make use of the OpenH264 library for H.264 decoding and encoding. Go to http://www.openh264.org/ and follow the instructions for installing the library. Then pass <code

class=”code”>--enable-libopenh264</code> to configure to enable it. For decoding, this library is much more limited than the built-in decoder in libavcodec; currently, this library lacks support for decoding B-frames and some other main/high profile features. (It currently only supports constrained baseline profile and CABAC.) Using it is mostly useful for testing and for taking advantage of Cisco’s patent portfolio license (http://www.openh264.org/BINARY_LICENSE.
  <h3 class=”section”>50.22 Open-JPEG# TOC</h3>

FFmpeg can use the OpenJPEG libraries for decoding/encoding J2K videos. Go to http://www.openjpeg.org/ to get the libraries and follow the installation instructions. To enable using OpenJPEG in FFmpeg, pass <code class=”code”>--enable-libopenjpeg</code> to <samp class=”file”>./configure</samp>. <h3 class=”section”>50.23 rav1e# TOC</h3>

FFmpeg can make use of rav1e (Rust AV1 Encoder) via its C bindings to encode videos. Go to https://github.com/xiph/rav1e/< and follow the instructions to build the C library. To enable using rav1e in FFmpeg, pass <code class=”code”>--enable-librav1e</code> to <samp class=”file”>./configure</samp>. <h3 class=”section”>50.24 SVT-AV1# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for AV1 library for AV1 encoding. Go to https://gitlab.com/AOMediaCodec/SVT-AV1/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libsvtav1</code> to configure to enable it. <h3 class=”section”>50.25 SVT-JPEG-XS# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for JPEG-XS library for JPEG-XS decoding and encoding. Go to https://github.com/OpenVisualCloud/SVT-JPEG-XS and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libsvtjpegxs</code> to configure to enable it. <h3 class=”section”>50.26 TwoLAME<a class=”anchor hidden-xs” href=”#TwoLAME” aria-

hidden=”true”># TOC</h3>

FFmpeg can make use of the TwoLAME library for MP2 encoding. Go to http://www.twolame.org/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libtwolame</code> to configure to enable it. <h3 class=”section”>50.27 VapourSynth# TOC</h3>

FFmpeg can read VapourSynth scripts as input. To enable support, pass <code class=”code”>--enable-vapoursynth</code> to configure. Vapoursynth is detected via <code class=”code”>pkg-config</code>. Versions 42 or greater supported. See http://www.vapoursynth.com/. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class=”code”>-f vapoursynth</code> before the input <code class=”code”>-i yourscript.vpy</code>. <h3 class=”section”>50.28 x264# TOC</h3>

FFmpeg can make use of the x264 library for H.264 encoding. Go to http://www.videolan.org/d and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libx264</code> to configure to enable it.   x264 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>50.29 x265# TOC</h3>

FFmpeg can make use of the x265 library for HEVC encoding. Go to http://x265.org/developers.html and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libx265</code> to configure to enable it.   x265 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>50.30 xavs# <a class=”anchor hidden-xs”href=”#toc-xavs”

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the xavs library for AVS encoding. Go to http://xavs.sf.net/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libxavs</code> to configure to enable it. <h3 class=”section”>50.31 xavs2# TOC</h3>

FFmpeg can make use of the xavs2 library for AVS2-P2/IEEE1857.4 video encoding. Go to https://github.com/pku and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libxavs2</code> to configure to enable it.   libxavs2 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>50.32 eXtra-fast Essential Video Encoder (XEVE)# TOC</h3>

FFmpeg can make use of the XEVE library for EVC video encoding. Go to https://github.com/mpeg5/xeve and follow the instructions for installing the XEVE library. Then pass <code class=”code”>--enable-libxeve</code> to configure to enable it. <h3 class=”section”>50.33 eXtra-fast Essential Video Decoder (XEVD)# TOC</h3>

FFmpeg can make use of the XEVD library for EVC video decoding. Go to https://github.com/mpeg5/xevd and follow the instructions for installing the XEVD library. Then pass <code class=”code”>--enable-libxevd</code> to configure to enable it. <h3 class=”section”>50.34 ZVBI# TOC</h3>

ZVBI is a VBI decoding library which can be used by FFmpeg to decode DVB teletext pages and DVB teletext subtitles. Go to <a

class=”url” href=”http://sourceforge.net/projects/zapping/”>http://sourceforge.net/projects/zapping/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libzvbi</code> to configure to enable it. <h2 class=”chapter”>51 Supported File Formats, Codecs or Features# TOC</h2>

You can use the <code class=”code”>-formats</code> and <code class=”code”>-codecs</code> options to have an exhaustive list. <h3 class=”section”>51.1 File Formats# TOC</h3>

FFmpeg supports the following file formats through the <code class=”code”>libavformat</code>

library:

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>3dostr</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>4xm</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>4X Technologies format, used in some games.</td></tr>

<tr><td width=”40%”>8088flex TMV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Enhanced Audio format, used in audiobooks.</td></tr>

<tr><td width=”40%”>AA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Format 2, 3, and 4, used in audiobooks.</td></tr>

<tr><td width=”40%”>ACT Voice</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>contains G.729 audio</td></tr>

<tr><td width=”40%”>Adobe Filmstrip</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Audio IFF (AIFF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width="40%">American Laser Games MM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in games like Mad Dog McCree.</td></tr>

<tr><td width="40%">3GPP AMR</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Amazing Studio Packed Animation File</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in game Heart Of Darkness.</td></tr>

<tr><td width="40%">Apple HTTP Live Streaming</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Artworx Data Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only format used in some Interplay games.</td></tr>

<tr><td width="40%">ADP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">AFC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">ADS/SS2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS2.</td></tr>

<tr><td width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">.astc</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Raw ASTC texture container.</td></tr>

<tr><td width="40%">ASF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Advanced / Active Streaming Format.</td></tr>

<tr><td width="40%">AST</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">AviSynth</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AVR</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on Mac.</td></tr>

<tr><td width="40%">AVS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by the Creature Shock game.</td></tr>

<tr><td width="40%">Beam Software SIFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Beam Software.</td></tr>

<tr><td width="40%">Bethesda Softworks VID</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Binary text</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only multimedia format used by some games.</td></tr>

<tr><td width="40%">Bitmap Brothers JV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Z and Z95 games.</td></tr>

<tr><td width="40%">BRP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Argonaut Games format.</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">BFSTM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo WiiU (based on BRSTM).</td></tr>

<tr><td width="40%">BRSTM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">BW64</td><td width="10%"></td><td width="10%">X</td><td width="40%">Broadcast Wave 64bit.</td></tr>

<tr><td width="40%">BWF</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">codec2 (raw)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Must be given -mode format option to decode correctly.</td></tr>

<tr><td width="40%">codec2 (.c2 files)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Contains header with version and mode info, simplifying playback.</td></tr>

<tr><td width="40%">CRI ADX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">CRI AIX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">Discworld II BMV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Delphine Software International CIN</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by Delphine Software games.</td></tr>

<tr><td width="40%">Digital Speech Standard (DSS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video format used by CD+G karaoke disks</td></tr>

<tr><td width="40%">Phantom Cine</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Commodore CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video format</td></tr>

<tr><td width="40%">Core Audio Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Apple Core Audio Format</td></tr>

<tr><td width="40%">CRC testing format</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">Creative Voice</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Created for the Sound Blaster Pro.</td></tr>

<tr><td width="40%">CRYO APC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in some games by CRYO Interactive Entertainment.</td></tr>

<tr><td width="40%">D-Cinema audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width=”40%”>Deluxe Paint Animation</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DCSTR</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DFA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in Chronomaster game</td></tr>

<tr><td width=”40%”>DirectDraw Surface</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DSD Stream File (DSF)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DV video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DXA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in the non-Windows version of the Feeble Files

game and different game cutscenes repacked for use with ScummVM.</td></tr>

<tr><td width=”40%”>Electronic Arts cdata</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Electronic Arts Multimedia</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA games; files have extensions like WVE and UV2.</td></tr>

<tr><td width=”40%”>Ensoniq Paris Audio File</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>FFM (FFserver live feed)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash (SWF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash 9 (AVM2)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Only embedded audio is decoded.</td></tr>

<tr><td width=”40%”>FLI/FLC/FLX animation</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>.fli/.flc files</td></tr>

<tr><td width=”40%”>Flash Video (FLV)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Macromedia Flash video files</td></tr>

<tr><td width=”40%”>framecrc testing format</td><td width=”10%”>X</td><td width=”10%”></td></tr>

<tr><td width="40%">FunCom ISS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in various games from FunCom like The Longest Journey.</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.726</td><td width="10%"></td><td width="10%">X</td><td width="40%">Both left- and right-justified.</td></tr>

<tr><td width="40%">G.729 BIT</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.729 raw</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GENH</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format for various games.</td></tr>

<tr><td width="40%">GIF Animation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">General eXchange Format SMPTE 360M, used by Thomson Grass Valley

playout servers.</td></tr>

<tr><td width="40%">HNM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Only version 4 supported, used in some games from Cryo Interactive</td></tr>

<tr><td width="40%">iCEDraw File</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ICO</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft Windows ICO</td></tr>

<tr><td width="40%">id Quake II CIN video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">IEC61937 encapsulation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Interchange File Format</td></tr>

<tr><td width="40%">IFV</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format used by some old CCTV DVRs.</td></tr>

<tr><td width="40%">iLBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay MVE</td><td width="10%"></td><td width="10%">X</td><td width="40%">Format used in various Interplay computer games.</td></tr>

<tr><td width="40%">Iterated Systems ClearVideo</td><td width="10%"></td><td width="10%">X</td><td width="40%">I-frames only</td></tr>

<tr><td width="40%">IV8</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format generated by IndigoVision 8000 video server.</td></tr>

<tr><td width="40%">IVF (On2)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">A format used by libvpx</td></tr>

<tr><td width="40%">Internet Video Recording</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">IRCAM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">JPEG Multi-Picture Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Includes JPEG Ultra HDR (ISO 21496-1 gain map) images.</td></tr>

<tr><td width="40%">LAF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Limitless Audio Format</td></tr>

<tr><td width="40%">LATM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LMLM4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Linux Media Labs MPEG-4 PCI boards</td></tr>

<tr><td width="40%">LOAS</td><td width="10%"></td><td width="10%">X</td><td width="40%">contains LATM multiplexed AAC audio</td></tr>

<tr><td width="40%">LRC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LVF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LXF</td><td width="10%"></td><td width="10%">X</td><td width="40%">VR native stream format, used by Leitch/Harris’ video servers.</td></tr>

<tr><td width="40%">Magic Lantern Video (MLV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width=”40%”>Matroska audio</td><td width=”10%”>X</td><td width=”10%”></td></tr>

<tr><td width=”40%”>FFmpeg metadata</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Metadata in text format.</td></tr>

<tr><td width=”40%”>MAXIS XA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Sim City 3000; file extension .xa.</td></tr>

<tr><td width=”40%”>MCA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some games from Capcom; file extension .mca.</td></tr>

<tr><td width=”40%”>MD Studio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Metal Gear Solid: The Twin Snakes</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Megalux Frame</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used by Megalux Ultimate Paint</td></tr>

<tr><td width=”40%”>MobiClip MODS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MobiClip MOFLEX</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Mobotix .mxg</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Monkey’s Audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Motion Pixels MVI</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MOV/QuickTime/MP4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>3GP, 3GP2, PSP, iPod variants supported</td></tr>

<tr><td width=”40%”>MP2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MP3</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-1 System</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>muxed audio and video, VCD format supported</td></tr>

<tr><td width="40%">MPEG-PS (program stream)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">also known as <code class="code">VOB</code> file, SVCD and DVD format supported</td></tr>

<tr><td width="40%">MPEG-TS (transport stream)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">also known as DVB Transport Stream</td></tr>

<tr><td width="40%">MPEG-4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">MPEG-4 is a variant of QuickTime.</td></tr>

<tr><td width="40%">MSF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">Mirillis FIC video</td><td width="10%"></td><td width="10%">X</td><td width="40%">No cursor rendering.</td></tr>

<tr><td width="40%">MIDI Sample Dump Standard</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MIME multipart JPEG</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">MSN TCP webcam</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by MSN Messenger webcam streams.</td></tr>

<tr><td width="40%">MTV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Material eXchange Format (MXF)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 377M, used by D-Cinema, broadcast industry.</td></tr>

<tr><td width="40%">Material eXchange Format (MXF), D-10 Mapping</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 386M, D-10/IMX Mapping.</td></tr>

<tr><td width="40%">NC camera feed</td><td width="10%"></td><td width="10%">X</td><td width="40%">NC (AVIP NC4600) camera streams</td></tr>

<tr><td width="40%">NIST SPeech HEader REsources</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Computerized Speech Lab NSP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NTT TwinVQ (VQF)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Nippon Telegraph and Telephone Corporation TwinVQ.</td></tr>

<tr><td width="40%">Nullsoft Streaming Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NUT</td><td width="10%">X</td><td width="10%">X</td><td width="40%">NUT Open Container Format</td></tr>

<tr><td width="40%">Ogg</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Playstation Portable PMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Portable Voice Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">TechnoTrend PVA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by TechnoTrend DVB PCI boards.</td></tr>

<tr><td width="40%">QCP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw ADTS (AAC)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-NB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-WB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APAC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Bonk</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw Chinese AVS video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Dirac</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DNxHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS-HD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw E-AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw EVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw FLAC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw GSM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.263</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.264</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw HEVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Ingenient MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw MLP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw NULL</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw id RoQ</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw OBU</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw OSQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw SBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TAK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TrueHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw VC-1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 16.8 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 24.0 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RDT</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">REDCODE R3D</td><td width="10%"></td><td width="10%">X</td><td width="40%">File format used by RED Digital cameras, contains JPEG 2000 frames and PCM audio.</td></tr>

<tr><td width="40%">RealMedia</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Redirector</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RedSpark</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TeXture Dictionary</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Resolume DXV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoding is only supported for the DXT1 (Normal Quality, No Alpha) texture format.</td></tr>

<tr><td width="40%">RF64</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RL2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Entertainment Software Partners.</td></tr>

<tr><td width="40%">RPL/ARMovie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Lego Mindstorms RSO</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RSD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RTMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Output is performed by publishing stream to RTMP server</td></tr>

<tr><td width="40%">RTP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RTSP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sample Dump eXchange</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SAP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SBG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDNS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SER</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Pictures SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sega FILM/CPK</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in many Sega Saturn console games.</td></tr>

<tr><td width="40%">Silicon Graphics Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra SOL</td><td width="10%"></td><td width="10%">X</td><td width="40%">.sol files used in Sierra Online games.</td></tr>

<tr><td width="40%">Sierra VMD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra CD-ROM games.</td></tr>

<tr><td width="40%">Smacker</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">SMJPEG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in certain Loki game ports.</td></tr>

<tr><td width="40%">SMPTE 337M encapsulation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in some LucasArts games.</td></tr>

<tr><td width="40%">Sony OpenMG (OMA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used in Sony Sonic Stage and Sony Vegas.</td></tr>

<tr><td width="40%">Sony PlayStation STR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sony Wave64 (W64)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SoX native format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUN AU format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUP raw PGS subtitles</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Konami PS2 games.</td></tr>

<tr><td width="40%">TDSC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Text files</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">THP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used on the Nintendo GameCube.</td></tr>

<tr><td width="40%">Tiertex Limited SEQ</td><td width="10%"></td><td width="10%">X</td><td width="40%">Tiertex .seq files used in the DOS CD-ROM version of the game Flashback.</td></tr>

<tr><td width="40%">True Audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in many Sony PS2 games.</td></tr>

<tr><td width="40%">VC-1 test bitstream</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vidvox Hap</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vivo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VPK</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Sony PS games.</td></tr>

<tr><td width="40%">Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">WebM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Televison (WTV)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Wing Commander III movie</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Origin’s Wing Commander III computer game.</td></tr>

<tr><td width="40%">Westwood Studios audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Westwood Studios VQA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Wideband Single-bit Data (WSD)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WVE</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">KTX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Khronos Texture 1.0 container for ASTC.</td></tr>

<tr><td width="40%">Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">XMV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft video container used in Xbox games.</td></tr>

<tr><td width="40%">XVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">xWMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft audio container used by XAudio 2.</td></tr>

<tr><td width="40%">eXtended BINary text (XBIN)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">YUV4MPEG pipe</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP</td><td width="10%"></td><td
width="10%">X</td></tr>

</tbody>

</table>

<code class="code">X</code> means that the feature in that column
(encoding / decoding) is supported.
<h3 class="section">51.2 Image Formats<a
class="anchor hidden-xs" href="#Image-Formats" aria-hidden="true">#
TOC</h3>

FFmpeg can read and write images for each frame of a video se-
quence. The following image formats are supported: <table
class="multitable"> <tbody><tr><td width="40%">Name</td><td
width="10%">Encoding</td><td width="10%">Decoding</td><td
width="40%">Comments</td></tr> <tr><td width="40%">.Y.U.V</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">one
raw file per component</td></tr> <tr><td width="40%">Alias PIX</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">Alias/Wavefront
PIX image format</td></tr> <tr><td width="40%">animated GIF</td><td
width="10%">X</td><td width="10%">X</td></tr> <tr><td
width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td><td
width="40%">Animated Portable Network Graphics</td></tr> <tr><td
width="40%">ASTC</td><td width="10%">E</td><td width="10%">E</td><td
width="40%">Adaptive Scalable Texture Compression, through external li-
brary libastcenc</td></tr> <tr><td width="40%">BMP</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft
BMP image</td></tr> <tr><td width="40%">BRender PIX</td><td
width="10%"></td><td width="10%">X</td><td width="40%">Argonaut
BRender 3D engine image format.</td></tr> <tr><td width="40%">CRI</td><td
width="10%"></td><td width="10%">X</td><td width="40%">Cintel
RAW</td></tr> <tr><td width="40%">DPX</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Digital Picture Exchange</td></tr>
<tr><td width="40%">EXR</td><td width="10%"></td><td width="10%">X</td><td
width="40%">OpenEXR</td></tr> <tr><td width="40%">FITS</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">Flexible
Image Transport System</td></tr> <tr><td width="40%">HDR</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">Radiance
HDR RGBE Image format</td></tr> <tr><td width="40%">IMG</td><td
width="10%"></td><td width="10%">X</td><td width="40%">GEM
Raster image</td></tr> <tr><td width="40%">JPEG</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">Progressive
JPEG is not supported.</td></tr> <tr><td width="40%">JPEG
2000</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">JPEG-LS</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">JPEG-XS</td><td

width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Supported
through external library libsvtjpegxs</td></tr> <tr><td width=”40%”>LJPEG</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”40%”>Lossless
JPEG</td></tr> <tr><td width=”40%”>Media 100</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>MSP</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Microsoft
Paint image</td></tr> <tr><td width=”40%”>PAM</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>PAM is a PNM extension
with alpha support.</td></tr> <tr><td width=”40%”>PBM</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Portable
BitMap image</td></tr> <tr><td width=”40%”>PCD</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”40%”>PhotoCD</td></tr>
<tr><td width=”40%”>PCX</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>PC Paintbrush</td></tr>
<tr><td width=”40%”>PFM</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Portable FloatMap image</td></tr>
<tr><td width=”40%”>PGM</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Portable GrayMap image</td></tr>
<tr><td width=”40%”>PGMYUV</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>PGM with U and V compo-
nents in YUV 4:2:0</td></tr> <tr><td width=”40%”>PGX</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”40%”>PGX
file decoder</td></tr> <tr><td width=”40%”>PHM</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Portable HalfFloatMap im-
age</td></tr> <tr><td width=”40%”>PIC</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”40%”>Pictor/PC Paint</td></tr>
<tr><td width=”40%”>PNG</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Portable Network Graphics im-
age</td></tr> <tr><td width=”40%”>PPM</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Portable PixelMap image</td></tr>
<tr><td width=”40%”>PSD</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”40%”>Photoshop</td></tr> <tr><td width=”40%”>PTX</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”40%”>V.Flash
PTX format</td></tr> <tr><td width=”40%”>QOI</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Quite OK Image format</td></tr>
<tr><td width=”40%”>RW2</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”40%”>Panasonic raw image, RawFormat 8 only</td></tr> <tr><td
width=”40%”>SGI</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”40%”>SGI RGB image format</td></tr> <tr><td width=”40%”>Sun
Rasterfile</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”40%”>Sun RAS image format</td></tr> <tr><td width=”40%”>TIFF</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>YUV,
JPEG and some extension is not supported yet.</td></tr> <tr><td
width=”40%”>Truevision Targa</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”40%”>Targa (.TGA) image for-
mat</td></tr> <tr><td width=”40%”>VBN</td><td width=”10%”>X</td><td

width="10%">X</td><td width="40%">Vizrt Binary Image format</td></tr>
<tr><td width="40%">WBMP</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Wireless Application Protocol
Bitmap image format</td></tr> <tr><td width="40%">WebP</td><td
width="10%">E</td><td width="10%">X</td><td width="40%">WebP
image format, encoding supported through external library libwebp</td></tr>
<tr><td width="40%">XBM</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X BitMap image format</td></tr>
<tr><td width="40%">XFace</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X-Face image format</td></tr>
<tr><td width="40%">XPM</td><td width="10%"></td><td width="10%">X</td><td
width="40%">X PixMap image format</td></tr> <tr><td width="40%">XWD</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">X
Window Dump image format</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code> means that support is provided through an external library. <h3 class="section">51.3 Video Codecs# TOC</h3>

<table class="multitable">

<tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td
width="10%">Decoding</td><td width="40%">Comments</td></tr>

<tr><td width="40%">4X Movie</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in certain computer
games.</td></tr>

<tr><td width="40%">8088flex TMV</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">A64 multicolor</td><td width="10%">X</td><td
width="10%"></td><td width="40%">Creates video suitable to be played
on a commodore 64 (multicolor mode).</td></tr>

<tr><td width="40%">Amazing Studio PAF Video</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">American Laser Games MM</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in games like Mad Dog McCree.</td></tr>

<tr><td width="40%">Amuse Graphics Movie</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">AMV Video</td><td width="10%">X</td><td

width=”10%”>X</td><td width=”40%”>Used in Chinese MP3 players.</td></tr>

<tr><td width=”40%”>ANSI/ASCII art</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple Intermediate Codec</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple MJPEG-B</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple Pixlet</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple ProRes</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: apch,apcn,apcs,apco,ap4h,ap4x</td></tr>

<tr><td width=”40%”>Apple QuickDraw</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: qdrw</td></tr>

<tr><td width=”40%”>APV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Argonaut Video</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some Argonaut games.</td></tr>

<tr><td width=”40%”>Asus v1</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ASV1</td></tr>

<tr><td width=”40%”>Asus v2</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ASV2</td></tr>

<tr><td width=”40%”>ATI VCR1</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VCR1</td></tr>

<tr><td width=”40%”>ATI VCR2</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VCR2</td></tr>

<tr><td width=”40%”>Auravision Aura</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Auravision Aura 2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Autodesk Animator Flic video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Autodesk RLE</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: AASC</td></tr>

<tr><td width=”40%”>AV1</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Supported through external libraries libaom, libdav1d, librav1e and libsvtav1</td></tr>

<tr><td width="40%">Avid 1:1 10-bit RGB Packer</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: AVrp</td></tr>

<tr><td width="40%">AVS (Audio Video Standard) video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used by the Creature Shock game.</td></tr>

<tr><td width="40%">AVS2-P2/IEEE1857.4</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libxavs2 and libdavs2</td></tr>

<tr><td width="40%">AVS3-P2/IEEE1857.10</td><td width="10%"></td><td width="10%">E</td><td width="40%">Supported through external library libuavs3d</td></tr>

<tr><td width="40%">AYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft uncompressed packed 4:4:4:4</td></tr>

<tr><td width="40%">Beam Software VB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bethesda VID video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Bink Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">BitJazz SheerVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bitmap Brothers JV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">y41p Brooktree uncompressed 4:1:1 12-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Brooktree ProSumer Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: BT20</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">C93 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Cyberia game.</td></tr>

<tr><td width="40%">CamStudio</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: CSCD</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video codec for CD+G karaoke

disks</td></tr>

<tr><td width="40%">CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video codec</td></tr>

<tr><td width="40%">Chinese AVS video</td><td width="10%">E</td><td width="10%">X</td><td width="40%">AVS1-P2, JiZhun profile, encoding through external library libxavs</td></tr>

<tr><td width="40%">Delphine Software International CIN video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">Discworld II BMV Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CineForm HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CDToons</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in various Broderbund games.</td></tr>

<tr><td width="40%">Cinepak</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Cirrus Logic AccuPak</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: CLJR</td></tr>

<tr><td width="40%">CPiA Video Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Creative YUV (CYUV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DFA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Chronomaster game.</td></tr>

<tr><td width="40%">Dirac</td><td width="10%">E</td><td width="10%">X</td><td width="40%">supported though the native vc2 (Dirac Pro) encoder</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DNxHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">aka SMPTE VC3</td></tr>

<tr><td width="40%">Duck TrueMotion 1.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: DUCK</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TM20</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0 RT</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TR20</td></tr>

<tr><td width="40%">DV (Digital Video)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Dxtory capture format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Feeble Files/ScummVM DXA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec originally used in Feeble Files game.</td></tr>

<tr><td width="40%">Electronic Arts CMV video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in NHL 95 game.</td></tr>

<tr><td width="40%">Electronic Arts Madcow video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGQ video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TQI video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 124</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 130</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">EVC / MPEG-5 Part 1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">encoding and decoding supported through external libraries libxeve and libxevd</td></tr>

<tr><td width="40%">FFmpeg video codec #1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">lossless codec (fourcc: FFV1)</td></tr>

<tr><td width="40%">Flash Screen Video v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: FSV1</td></tr>

<tr><td width="40%">Flash Screen Video v2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash Video (FLV)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sorenson H.263 used in Flash</td></tr>

<tr><td width="40%">FM Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Forward Uncompressed</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Fraps</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Go2Meeting</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M2, G2M3</td></tr>

<tr><td width="40%">Go2Webinar</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M4</td></tr>

<tr><td width="40%">Gremlin Digital Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263 / H.263-1996</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263+ / H.263-1998 / H.263 version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libx264 and OpenH264,

decoding supports data partitioning</td></tr>

<tr><td width="40%">HEVC</td><td width="10%">X</td><td width="10%">X</td><td width="40%">encoding supported through external library libx265 and libkvazaar</td></tr>

<tr><td width="40%">HNM version 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV FFmpeg variant</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IBM Ultimotion</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ULTI</td></tr>

<tr><td width="40%">id Cinematic video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Quake II.</td></tr>

<tr><td width="40%">id RoQ video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2, other computer games.</td></tr>

<tr><td width="40%">IFF ILBM</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF interleaved bitmap</td></tr>

<tr><td width="40%">IFF ByteRun1</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF run length encoded bitmap</td></tr>

<tr><td width="40%">Infinity IMM4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM6</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel H.263</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Interplay MVE video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Interplay .MVE files.</td></tr>

<tr><td width="40%">J2K</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Karl Morton’s video codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Worms games.</td></tr>

<tr><td width="40%">Kega Game Video (KGV1)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Kega emulator screen capture codec.</td></tr>

<tr><td width="40%">Lagarith</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCEVC / MPEG-5 LCEVC / MPEG-5 Part 2</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library liblcevc-dec</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) MSZH</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) ZLIB</td><td width="10%">E</td><td width="10%">E</td></tr>

<tr><td width="40%">LEAD MCMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LOCO</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LucasArts SANM/Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts games / SMUSH animations.</td></tr>

<tr><td width="40%">lossless MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MagicYUV Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mandsoft Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft ATC Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Screen 3.</td></tr>

<tr><td width="40%">Microsoft Expression Encoder Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Titanium Screen 2.</td></tr>

<tr><td width="40%">Microsoft RLE</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft Screen 1</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V7 Screen.</td></tr>

<tr><td width="40%">Microsoft Screen 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media

Video V9 Screen.</td></tr>

<tr><td width="40%">Microsoft Video 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mimic</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in MSN Messenger Webcam streams.</td></tr>

<tr><td width="40%">Miro VideoXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VIXL</td></tr>

<tr><td width="40%">MJPEG (Motion JPEG)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mobotix MxPEG video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Motion Pixels video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-1 video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-2 video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2</td><td width="10%">X</td><td width="10%">X</td><td width="40%">libxvidcore can be used alternatively for encoding.</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Newtek SpeedHQ</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Nintendo Gamecube THP video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NotchLC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo/RTjpeg</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in NuppelVideo files.</td></tr>

<tr><td width="40%">On2 VP3</td><td width="10%"></td><td width="10%">X</td><td width="40%">still experimental</td></tr>

<tr><td width="40%">On2 VP4</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP40</td></tr>

<tr><td width="40%">On2 VP5</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP50</td></tr>

<tr><td width="40%">On2 VP6</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP60,VP61,VP62</td></tr>

<tr><td width="40%">On2 VP7</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP70,VP71</td></tr>

<tr><td width="40%">VP8</td><td width="10%">E</td><td width="10%">X</td><td width="40%">fourcc: VP80, encoding supported through external library libvpx</td></tr>

<tr><td width="40%">VP9</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libvpx</td></tr>

<tr><td width="40%">Pinnacle TARGA CineWave YUV16</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: Y216</td></tr>

<tr><td width="40%">Q-team QPEG</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourccs: QPEG, Q1.0, Q1.1</td></tr>

<tr><td width="40%">QuickTime 8BPS video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QuickTime Animation (RLE) video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ’rle ’</td></tr>

<tr><td width="40%">QuickTime Graphics (SMC)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ’smc ’</td></tr>

<tr><td width="40%">QuickTime video (RPZA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: rpza</td></tr>

<tr><td width="40%">R10K AJA Kona 10-bit RGB Codec</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">R210 Quicktime Uncompressed RGB 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Raw Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 1.0</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 2.0</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 3.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">still far from ideal</td></tr>

<tr><td width="40%">RealVideo 4.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 6.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TXD (TeXture Dictionary)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Texture dictionaries used by the Renderware Engine.</td></tr>

<tr><td width="40%">RivaTuner Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’RTV1’</td></tr>

<tr><td width="40%">RL2 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">used in some games by Entertainment Software Partners</td></tr>

<tr><td width="40%">ScreenPressor</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screenpresso</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screen Recorder Gold Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 1 (MVC1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 2 (MVC2)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics RLE 8-bit video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smacker video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in Smacker.</td></tr>

<tr><td width="40%">SMPTE VC-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Snow</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental wavelet codec (fourcc: SNOW)</td></tr>

<tr><td width="40%">Sony PlayStation MDEC (Motion DECoder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: SVQ1</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 3</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SVQ3</td></tr>

<tr><td width="40%">Sunplus JPEG (SP5X)</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SP5X</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSCC</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSC2</td></tr>

<tr><td width="40%">Theora</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libtheora</td></tr>

<tr><td width="40%">Tiertex Limited SEQ video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in DOS CD-ROM FlashBack game.</td></tr>

<tr><td width="40%">Ut Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v210 QuickTime uncompressed 4:2:2 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v308 QuickTime uncompressed 4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v408 QuickTime uncompressed 4:4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v410 QuickTime uncompressed 4:4:4 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VBLE Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">vMix Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’VMX1’</td></tr>

<tr><td width="40%">VMware Screen Codec / VMware Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in videos captured by VMware.</td></tr>

<tr><td width="40%">Westwood Studios VQA (Vector Quantized Animation) video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Image</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 8</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 9</td><td width="10%"></td><td width="10%">X</td><td width="40%">not completely working</td></tr>

<tr><td width="40%">Wing Commander III / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander III .MVE files.</td></tr>

<tr><td width="40%">Wing Commander IV / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander IV.</td></tr>

<tr><td width="40%">Winnov WNV1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WMV7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">YAMAHA SMAF</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">yuv4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">libquicktime uncompressed packed 4:2:0</td></tr>

<tr><td width="40%">ZeroCodec Lossless Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ZLIB</td><td width="10%">X</td><td width="10%">X</td><td width="40%">part of LCL, encoder experimental</td></tr>

<tr><td width="40%">Zip Motion Blocks Video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoder works only in PAL8.</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <code class=”code”>E</code> means that support is provided through an external library. <h3 class=”section”>51.4 Audio Codecs# TOC</h3>

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>8SVX exponential</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>8SVX fibonacci</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAC</td><td width=”10%”>EX</td><td width=”10%”>X</td><td width=”40%”>encoding supported through internal encoder and external library libfdk-aac</td></tr>

<tr><td width=”40%”>AAC+</td><td width=”10%”>E</td><td width=”10%”>IX</td><td width=”40%”>encoding supported through external library libfdk-aac</td></tr>

<tr><td width=”40%”>AC-3</td><td width=”10%”>IX</td><td width=”10%”>IX</td></tr>

<tr><td width=”40%”>ACELP.KELVIN</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM 4X Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Yamaha AICA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM AmuseGraphics Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Argonaut Games</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM CDROM XA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Creative Technology</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>16 -> 4, 8 -> 4, 8 -> 3, 8 -> 2</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA titles.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts Maxis CDROM XS</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Sim City 3000.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R3</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts XAS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.722</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.726</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Acorn Replay</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA AMV</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in AMV files</td></tr>

<tr><td width=”40%”>ADPCM IMA Citrix</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Cunning Developments</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Electronic Arts EACS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Electronic Arts SEAD</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Funcom</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA High Voltage Software ALP</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Mobiclip MOFLEX</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA QuickTime</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width="40%">ADPCM IMA Simon & Schuster Interactive</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Ubisoft APM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Loki SDL MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Westwood</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM ISS IMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in FunCom games.</td></tr>

<tr><td width="40%">ADPCM IMA Dialogic</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK3</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Radical</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Xbox</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Microsoft</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM MS IMA</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube AFC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube DTK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo THP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Playstation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>ADPCM QT IMA</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Rhetorex</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sanyo</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM SEGA CRI ADX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in Sega Dreamcast games.</td></tr>

<tr><td width=”40%”>ADPCM Shockwave Flash</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 2-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 2.6-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 4-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM VIMA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in LucasArts SMUSH animations.</td></tr>

<tr><td width=”40%”>ADPCM Konami XMD</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Westwood Studios IMA</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in Westwood Studios games like Command and Conquer.</td></tr>

<tr><td width=”40%”>ADPCM Yamaha</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Zork</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AMR-NB</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>encoding supported through external library libopencore-amrnb</td></tr>

<tr><td width=”40%”>AMR-WB</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>encoding supported through external library libvo-amrwbenc</td></tr>

<tr><td width=”40%”>Amazing Studio PAF Audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width="40%">Apple lossless audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">QuickTime fourcc ’alac’</td></tr>

<tr><td width="40%">aptX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">aptX HD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">ATRAC1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3+</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC9</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Bink and Smacker files in many games.</td></tr>

<tr><td width="40%">Bonk audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">codec2</td><td width="10%">E</td><td width="10%">E</td><td width="40%">en/decoding supported through external library libcodec2</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Delphine Software International CIN audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Speech Standard - Standard Play mode (DSS SP)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Discworld II BMV Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">COOK</td><td width="10%"></td><td width="10%">X</td><td width="40%">All versions except 5.1 are supported.</td></tr>

<tr><td width="40%">DCA (DTS Coherent Acoustics)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">supported extensions: XCh, XXCH, X96, XBR, XLL, LBR (partially)</td></tr>

<tr><td width="40%">Dolby E</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Cuberoot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in few games.</td></tr>

<tr><td width="40%">DPCM Gremlin</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">DPCM Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Interplay</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various Interplay computer games.</td></tr>

<tr><td width="40%">DPCM Squareroot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various games.</td></tr>

<tr><td width="40%">DPCM Sierra Online</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra Online game audio files.</td></tr>

<tr><td width="40%">DPCM Sol</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Origin’s Wing Commander IV AVI files.</td></tr>

<tr><td width="40%">DPCM Xilam DERF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSP Group TrueSpeech</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DST (Direct Stream Transfer)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DV audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Enhanced AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">EVRC (Enhanced Variable Rate Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">FLAC (Free Lossless Audio Codec)</td><td width="10%">X</td><td width="10%">IX</td></tr>

<tr><td width="40%">FTR Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.729</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GSM</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">GSM Microsoft variant</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">IAC (Indeo Audio Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">iLBC (Internet Low Bitrate Codec)</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">encoding and decoding supported through external library libilbc</td></tr>

<tr><td width="40%">IMC (Intel Music Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LC3</td><td width="10%">E</td><td width="10%">E</td><td width="40%">supported through external library liblc3</td></tr>

<tr><td width="40%">MACE (Macintosh Audio Compression/Expansion) 6:1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Marian’s A-pac audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MI-SC4 (Micronas SC-4 Audio)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MLP (Meridian Lossless Packing)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in DVD-Audio discs.</td></tr>

<tr><td width="40%">Monkey’s Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MP1 (MPEG audio layer 1)</td><td width="10%"></td><td width="10%">IX</td></tr>

<tr><td width="40%">MP2 (MPEG audio layer 2)</td><td width="10%">IX</td><td width="10%">IX</td><td width="40%">encoding supported also through external library TwoLAME</td></tr>

<tr><td width="40%">MP3 (MPEG audio layer 3)</td><td width="10%">E</td><td width="10%">IX</td><td width="40%">encoding supported through external library LAME, ADU MP3 and MP3onMP4 also supported</td></tr>

<tr><td width="40%">MPEG-4 Audio Lossless Coding (ALS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-H 3D Audio</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library libmpeghdec</td></tr>

<tr><td width="40%">MobiClip FastAudio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV7</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Nellymoser Asao</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">On2 AVC (Audio for Video Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Opus</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopus</td></tr>

<tr><td width="40%">OSQ (Original Sound Quality)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM D-Cinema audio signed 24-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian for DVD-Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian in MPEG-TS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QCELP / PureVoice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">There are still some distortions.</td></tr>

<tr><td width="40%">RealAudio 1.0 (14.4K)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Real 14400 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 2.0 (28.8K)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Real 28800 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 3.0 (dnet)</td><td width="10%">IX</td><td width="10%">X</td><td width="40%">Real low bitrate AC-3 codec</td></tr>

<tr><td width="40%">RealAudio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealAudio SIPR / ACELP.NET</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SBC (low-complexity subband codec)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Smacker audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SMPTE 302M AES3 audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sonic</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Sonic lossless</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Speex</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">supported through external library libspeex</td></tr>

<tr><td width="40%">TAK (Tom’s lossless Audio Kompressor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">True Audio (TTA)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">TrueHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in HD-DVD and Blu-Ray discs.</td></tr>

<tr><td width="40%">TwinVQ (VQF flavor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ViewQuest VQC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Vorbis</td><td width="10%">E</td><td width="10%">X</td><td width="40%">A native but very primitive encoder exists.</td></tr>

<tr><td width="40%">Voxware MetaSound</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Westwood Audio (SND1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio 1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Pro</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Xbox Media Audio 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Xbox Media Audio 2</td><td width="10%"></td><td width="10%">X</td></tr>

</tbody>

</table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code> means that support is provided through an external library. <code class="code">I</code> means that an integer-only version is available, too (ensures high performance on systems without hardware floating point support). <h3 class="section">51.5 Subtitle Formats# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Muxing</td><td width="10%">Demuxing</td><td

width="10%">Encoding</td><td width="10%">Decoding</td></tr>
<tr><td width="40%">3GPP Timed Text</td><td width="10%"></td><td
width="10%"></td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">AQTitle</td><td width="10%"></td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">DVB</td><td width="10%">X</td><td
width="10%">X</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">DVB teletext</td><td width="10%"></td><td
width="10%">X</td><td width="10%"></td><td width="10%">E</td></tr>
<tr><td width="40%">DVD</td><td width="10%">X</td><td
width="10%">X</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">JACOsub</td><td width="10%">X</td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MicroDVD</td><td width="10%">X</td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MPL2</td><td width="10%"></td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MPsub (MPlayer)</td><td width="10%"></td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">PGS</td><td width="10%"></td><td width="10%"></td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">PJS
(Phoenix)</td><td width="10%"></td><td width="10%">X</td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">RealText</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">SAMI</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">Spruce format
(STL)</td><td width="10%"></td><td width="10%">X</td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">SSA/ASS</td><td
width="10%">X</td><td width="10%">X</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">SubRip (SRT)</td><td
width="10%">X</td><td width="10%">X</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">SubViewer v1</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">SubViewer</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">TED Talks cap-
tions</td><td width="10%"></td><td width="10%">X</td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">TTML</td><td
width="10%">X</td><td width="10%"></td><td width="10%">X</td><td
width="10%"></td></tr> <tr><td width="40%">VobSub (IDX+SUB)</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">VPlayer</td><td
width="10%"></td><td width="10%">X</td><td width="10%"></td><td
width="10%">X</td></tr> <tr><td width="40%">WebVTT</td><td
width="10%">X</td><td width="10%">X</td><td width="10%">X</td><td

width=”10%”>X</td></tr> <tr><td width=”40%”>XSUB</td><td width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td width=”10%”>X</td></tr> </tbody> </table>

<code class=”code”>X</code> means that the feature is supported. <code class=”code”>E</code> means that support is provided through an external library. <h3 class=”section”>51.6 Network Protocols# TOC</h3>

<table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Support</td></tr> <tr><td width=”40%”>AMQP</td><td width=”10%”>E</td></tr> <tr><td width=”40%”>file</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>FTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>Gopher</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>Gophers</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>HLS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>HTTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>HTTPS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>Icecast</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>MMSH</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>MMST</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>pipe</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>Pro-MPEG FEC</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPE</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPT</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPTE</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPTS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SAMBA</td><td width=”10%”>E</td></tr> <tr><td width=”40%”>SCTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SFTP</td><td width=”10%”>E</td></tr> <tr><td width=”40%”>TCP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>TLS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>UDP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>ZMQ</td><td width=”10%”>E</td></tr> </tbody> </table>

<code class=”code”>X</code> means that the protocol is supported. <code class=”code”>E</code> means that support is provided through an external library.

 <h3 class=”section”>51.7 Input/Output Devices<a class=”anchor hidden-

xs" href="#Input_002fOutput-Devices" aria-hidden="true"># TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Input</td><td width="10%">Output</td></tr> <tr><td width="40%">ALSA</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">BKTR</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">caca</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">DV1394</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">Lavfi virtual device</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">Linux framebuffer</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">JACK</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">LIBCDIO</td><td width="10%">X</td></tr> <tr><td width="40%">LIBDC1394</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">OpenAL</td><td width="10%">X</td></tr> <tr><td width="40%">OSS</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">PulseAudio</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">SDL</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">Video4Linux2</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">VfW capture</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">X11 grabbing</td><td width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">Win32 grabbing</td><td width="10%">X</td><td width="10%"></td></tr> </tbody> </table>

<code class="code">X</code> means that input/output is supported. <h3 class="section">51.8 Timecode# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Codec/format</td><td width="10%">Read</td><td width="10%">Write</td></tr> <tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">DV</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MOV</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MPEG1/2</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MXF</td><td width="10%">X</td><td width="10%">X</td></tr> </tbody> </table>

 <h2 class="chapter">52 See Also# <a class="anchor hidden-xs"href="#toc-See-Also"

aria-hidden=”true”>TOC</h2>

ffmpeg ffplay, ffprobe, ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class=”chapter”>53 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class=”command”>git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class=”file”>MAINTAINERS</samp> in the source code tree.

<p style=”font-size: small;”> This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-bitstream-filters.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset=”utf-8”> <title> FFmpeg Bitstream Filters Documentation </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”> <link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body> <h1> FFmpeg Bitstream Filters Documentation </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1

Description 2 Bitstream Filters <ul class="toc-numbered-mark"> 2.1 aac_adtstoasc 2.2 av1_metadata 2.3 chomp 2.4 dca_core 2.5 dovi_rpu 2.6 dovi_split 2.7 dump_extra 2.8 dv_error_marker 2.9 eac3_core 2.10 eia608_to_smpte436m 2.11 extract_extradata 2.12 filter_units 2.13 hapqa_extract 2.14 h264_metadata 2.15 h264_mp4toannexb 2.16 h264_redundant_pps 2.17 hevc_metadata 2.18 hevc_mp4toannexb 2.19 imxdump 2.20 jpegapp 2.21 mjpeg2jpeg 2.22 mjpegadump 2.23 mov2textsub 2.24 mpeg2_metadata 2.25 mpeg4_unpack_bframes 2.26 noise <ul class="toc-numbered-mark"> 2.26.1 Examples 2.27 null 2.28 pcm_rechunk 2.29 pgs_frame_merge 2.30 prores_metadata 2.31 remove_extra 2.32 setts 2.33 showinfo 2.34 side_data 2.35 smpte436m_to_eia608 2.36 text2movsub <a id="toc-trace_005fheaders"

href=”#trace_005fheaders”>2.37 trace_headers 2.38 trim 2.39 truehd_core 2.40 vp9_metadata 2.41 vp9_superframe 2.42 vp9_superframe_split 2.43 vp9_raw_reorder 3 See Also 4 Authors

 <h2 class=”chapter”>1 Description# TOC</h2>

This document describes the bitstream filters provided by the libavcodec library. A bitstream filter operates on the encoded stream data, and performs bitstream level modifications without performing decoding.

 <h2 class=”chapter”>2 Bitstream Filters# TOC</h2>

When you configure your FFmpeg build, all the supported bitstream filters are enabled by default. You can list all available ones using the configure option <code class=”code”>--list-bsfs</code>. You can disable all the bitstream filters using the configure option <code class=”code”>--disable-bsfs</code>, and selectively enable any bitstream filter using the option <code class=”code”>--enable-bsf=BSF</code>, or you can disable a particular bitstream filter using the option <code class=”code”>--disable-bsf=BSF</code>. The option <code class=”code”>-bsfs</code> of the ff* tools will display the list of all the supported bitstream filters included in your build. The ff* tools have a -bsf option applied per stream, taking a comma-separated list of filters, whose parameters follow the filter name after a ’=’.   <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v filter1[=opt1=str1:opt2=str2][,filter2] OUTPUT </pre>

Below is a description of the currently available bitstream filters, with their parameters, if any. <h3 class=”section”>2.1 aac_adtstoasc# TOC</h3>

Convert MPEG-2/4 AAC ADTS to an MPEG-4 Audio Specific Configuration bitstream. This filter creates an MPEG-4 AudioSpecificConfig

from an MPEG-2/4 ADTS header and removes the ADTS header. This filter is required for example when copying an AAC stream from a raw ADTS AAC or an MPEG-TS container to MP4A-LATM, to an FLV file, or to MOV/MP4 files and related formats such as 3GP or M4A. Please note that it is auto-inserted for MP4A-LATM and MOV/MP4 and related formats. <h3 class="section">2.2 av1_metadata# TOC</h3>

Modify metadata embedded in an AV1 stream. <dl class="table"> <dt><samp class="option">td</samp></dt> <dd> Insert or remove temporal delimiter OBUs in all temporal units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dd> Insert a TD at the beginning of every TU which does not already have one. </dd> <dt>‘<samp class="samp">remove</samp>’</dt> <dd> Remove the TD from the beginning of every TU which has one. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the color description fields in the stream (see AV1 section 6.4.2). </dd> <dt><samp class="option">color_range</samp></dt> <dd> Set the color range in the stream (see AV1 section 6.4.2; note that this cannot be set for streams using BT.709 primaries, sRGB transfer characteristic and identity (RGB) matrix coefficients). <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dd> Limited range. </dd> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> Full range. </dd> </dl>

</dd> <dt><samp class="option">chroma_sample_position</samp></dt> <dd> Set the chroma sample location in the stream (see AV1 section 6.4.2). This can only be set for 4:2:0 streams. <dl class="table"> <dt>‘<samp class="samp">vertical</samp>’</dt> <dd> Left position (matching the default in MPEG-2 and H.264). </dd> <dt>‘<samp class="samp">colocated</samp>’</dt> <dd> Top-left position. </dd> </dl>

</dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_display_tick) in the timing info in the sequence header. </dd> <dt><samp class="option">num_ticks_per_picture</samp></dt> <dd> Set the number of ticks in each picture, to indicate that the stream has a fixed framerate. Ignored if <samp class="option">tick_rate</samp> is not also set. </dd> <dt><samp class="option">delete_padding</samp></dt> <dd> Deletes Padding OBUs. </dd> </dl>

 <h3 class="section">2.3 chomp# TOC</h3>

Remove zero padding at the end of a packet. <h3 class="section">2.4 dca_core# TOC</h3>

Extract the core from a DCA/DTS stream, dropping extensions such as DTS-HD. <h3 class="section">2.5 dovi_rpu# TOC</h3>

Manipulate Dolby Vision metadata in a HEVC/AV1 bitstream, optionally enabling metadata compression. <dl class="table"> <dt><samp class="option">strip</samp></dt> <dd> If enabled, strip all Dolby Vision metadata (configuration record + RPU data blocks) from the stream. </dd> <dt><samp class="option">compression</samp></dt> <dd> Which compression level to enable. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No metadata compression. </dd> <dt>‘<samp class="samp">limited</samp>’</dt> <dd> Limited metadata compression scheme. Should be compatible with most devices. This is the default. </dd> <dt>‘<samp class="samp">extended</samp>’</dt> <dd> Extended metadata compression. Devices are not required to support this. Note that this level currently behaves the same as ‘<samp class="samp">limited</samp>’ in libavcodec. </dd> </dl> </dd> </dl>

 <h3 class="section">2.6 dovi_split# TOC</h3>

Split a Dolby Vision Profile 7 multi-layer HEVC bitstream. Profile 7 carries the enhancement-layer HEVC bitstream interleaved inside the base-layer access units, wrapped in user-unspecified NAL units of type 63 (UNSPEC63), and the RPU metadata as a sibling user-unspecified NAL of type 62 (UNSPEC62). <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Which Dolby Vision components to keep in the output bitstream. <dl class="table"> <dt>‘<samp class="samp">bl</samp>’</dt> <dd> Base layer only: drop every UNSPEC63 (EL) and every UNSPEC62 (RPU). The output is a plain HEVC stream with no Dolby Vision markers. This is the default. </dd> <dt>‘<samp class="samp">bl_rpu</samp>’</dt> <dd> Base layer with the RPU NAL kept. </dd> <dt>‘<samp class="samp">el</samp>’</dt>

<dd> Enhancement layer only: for every UNSPEC63 NAL, strip the two-byte outer NAL header and emit the inner payload. The result is a standalone HEVC bitstream. UNSPEC62 (RPU) is dropped. </dd> <dt>‘<samp class="samp">el_rpu</samp>’</dt> <dd> Enhancement layer with the RPU NAL kept verbatim. Same as ‘<samp class="samp">el</samp>’, but the UNSPEC62 RPU NALs are also emitted alongside the unwrapped EL NALs. </dd> </dl> </dd> </dl>

 <h3 class="section">2.7 dump_extra# TOC</h3>

Add extradata to the beginning of the filtered packets except when said packets already exactly begin with the extradata that is intended to be added. <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> The additional argument specifies which packets should be filtered. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">k</samp>’</dt> <dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> add extradata to all key packets </dd> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">all</samp>’</dt> <dd> add extradata to all packets </dd> </dl> </dd> </dl>

If not specified it is assumed ‘<samp class="samp">k</samp>’. For example the following <code class="command">ffmpeg</code> command forces a global header (thus disabling individual packet headers) in the H.264 packets generated by the <code class="code">libx264</code> encoder, but corrects them by adding the header stored in extradata to the key packets: <pre class="example-preformatted">ffmpeg -i INPUT -map 0 -flags:v +global_header -c:v libx264 -bsf:v dump_extra out.ts </pre>

 <h3 class="section">2.8 dv_error_marker# TOC</h3>

Blocks in DV which are marked as damaged are replaced by blocks of the specified color. <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color to replace damaged blocks by </dd> <dt><samp class="option">sta</samp></dt> <dd> A 16 bit mask which specifies which of the 16 possible error status values are to be replaced by colored blocks. 0xFFFE is the default which replaces all non 0 error status values. <dl class="table"> <dt>‘<samp class="samp">ok</samp>’</dt> <dd> No error, no concealment </dd> <dt>‘<samp class="samp">err</samp>’</dt>

<dd> Error, No concealment </dd> <dt>‘<samp class="samp">res</samp>’</dt> <dd> Reserved </dd> <dt>‘<samp class="samp">notok</samp>’</dt> <dd> Error or concealment </dd> <dt>‘<samp class="samp">notres</samp>’</dt> <dd> Not reserved </dd> <dt>‘<samp class="samp">Aa, Ba, Ca, Ab, Bb, Cb, A, B, C, a, b, erri, erru</samp>’</dt> <dd> The specific error status code </dd> </dl> see page 44-46 or section 5.5 of <a class="url" href="http://web.archive.org/web/20060927044735/http://www.smpte.org/ </dd> </dl>

 <h3 class="section">2.9 eac3_core# TOC</h3>

Extract the core from a E-AC-3 stream, dropping extra channels. <h3 class="section">2.10 eia608_to_smpte436m# TOC</h3>

Convert from a <code class="code">EIA_608</code> stream to a <code class="code">SMPTE_436M_ANC</code> data stream, wrapping the closed captions in CTA-708 CDP VANC packets. <dl class="table"> <dt><samp class="option">line_number</samp></dt> <dd> Choose which line number the generated VANC packets should go on. You generally want either line 9 (the default) or 11. </dd> <dt><samp class="option">wrapping_type</samp></dt> <dd> Choose the SMPTE 436M wrapping type, defaults to ‘<samp class="samp">vanc_frame</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">vanc_frame</samp>’</dt> <dd> VANC frame (interlaced or segmented progressive frame) </dd> <dt>‘<samp class="samp">vanc_field_1</samp>’</dt> <dt>‘<samp class="samp">vanc_field_2</samp>’</dt> <dt>‘<samp class="samp">vanc_progressive_frame</samp>’</dt> </dl> </dd> <dt><samp class="option">sample_coding</samp></dt> <dd> Choose the SMPTE 436M sample coding, defaults to ‘<samp class="samp">8bit_luma</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">8bit_luma</samp>’</dt> <dd> 8-bit component luma samples </dd> <dt>‘<samp class="samp">8bit_color_diff</samp>’</dt> <dd> 8-bit component color difference samples </dd> <dt>‘<samp class="samp">8bit_luma_and_color_diff</samp>’</dt> <dd> 8-bit component luma and color difference samples </dd> <dt>‘<samp class="samp">10bit_luma</samp>’</dt> <dd> 10-bit component luma samples </dd> <dt>‘<samp class="samp">10bit_color_diff</samp>’</dt>

<dd> 10-bit component color difference samples </dd> <dt>‘<samp class=”samp”>10bit_luma_and_color_diff</samp>’</dt> <dd> 10-bit component luma and color difference samples </dd> <dt>‘<samp class=”samp”>8bit_luma_parity_error</samp>’</dt> <dd> 8-bit component luma samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_color_diff_parity_error</samp>’</dt> <dd> 8-bit component color difference samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_luma_and_color_diff_parity_error</samp>’</dt> <dd> 8-bit component luma and color difference samples with parity error </dd> </dl> </dd> <dt><samp class=”option”>initial_cdp_sequence_cntr</samp></dt> <dd> The initial value of the CDP’s 16-bit unsigned integer <code class=”code”>cdp_hdr_sequence_cntr</code> and <code class=”code”>cdp_ftr_sequence_cntr</code> fields. Defaults to 0. </dd> <dt><samp class=”option”>cdp_frame_rate</samp></dt> <dd> Set the CDP’s <code class=”code”>cdp_frame_rate</code> field. This doesn’t actually change the timing of the data stream, it just changes the values inserted in that field in the generated CDP packets. Defaults to ‘<samp class=”samp”>30000/1001</samp>’. </dd> </dl>

 <h3 class=”section”>2.11 extract_extradata# TOC</h3>

Extract the in-band extradata. Certain codecs allow the long-term headers (e.g. MPEG-2 sequence headers, or H.264/HEVC (VPS/)SPS/PPS) to be transmitted either "in-band" (i.e. as a part of the bitstream containing the coded frames) or "out of band" (e.g. on the container level). This latter form is called "extradata" in FFmpeg terminology. This bitstream filter detects the in-band headers and makes them available as extradata. <dl class=”table”> <dt><samp class=”option”>remove</samp></dt> <dd> When this option is enabled, the long-term headers are removed from the bitstream after extraction. </dd> </dl>

 <h3 class=”section”>2.12 filter_units# TOC</h3>

Remove units with types in or not in a given set from the stream. <dl class=”table”> <dt><samp class=”option”>pass_types</samp></dt> <dd> List of unit types or ranges of unit types to pass through while removing all others. This is specified as a ’|’-separated list of unit type values or ranges of values with ’-’. </dd> <dt><samp class=”option”>remove_types</samp></dt> <dd> Identical to <samp class=”option”>pass_types</samp>, except the units in the given set removed

and all others passed through. </dd> </dl>

The types used by pass_types and remove_types correspond to NAL unit types (nal_unit_type) in H.264, HEVC and H.266 (see Table 7-1 in the H.264 and HEVC specifications or Table 5 in the H.266 specification), to marker values for JPEG (without 0xFF prefix) and to start codes without start code prefix (i.e. the byte following the 0x000001) for MPEG-2. For VP8 and VP9, every unit has type zero. Extradata is unchanged by this transformation, but note that if the stream contains inline parameter sets then the output may be unusable if they are removed. For example, to remove all non-VCL NAL units from an H.264 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=pass_types=1-5' OUTPUT </pre>

To remove all AUDs, SEI and filler from an H.265 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=35|38-40' OUTPUT </pre>

To remove all user data from a MPEG-2 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=178' OUTPUT </pre>

To remove all SEI from a H264 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=6' OUTPUT </pre>

To remove all prefix and suffix SEI from a HEVC stream, including Closed Captions and dynamic HDR: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=39|40' OUTPUT </pre>

 <h3 class="section">2.13 hapqa_extract# TOC</h3>

Extract Rgb or Alpha part of an HAPQA file, without recompression, in order to create an HAPQ or an HAPAlphaOnly file. <dl class="table"> <dt><samp class="option">texture</samp></dt> <dd> Specifies the texture to keep. <dl class="table"> <dt><samp class="option">color</samp></dt> <dt><samp class="option">alpha</samp></dt> </dl>

</dd> </dl>

Convert HAPQA to HAPQ <pre class="example-preformatted">ffmpeg -i hapqa_inputfile.mov -c copy

-bsf:v hapqa_extract=texture=color -tag:v HapY -metadata:s:v:0 encoder="HAPQ" hapq_file.mov </pre>

Convert HAPQA to HAPAlphaOnly <pre class="example-preformatted">ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=alpha -tag:v HapA -metadata:s:v:0 encoder="HAPAlpha Only" hapalphaonly_file.mov </pre>

 <h3 class="section">2.14 h264_metadata# TOC</h3>

Modify metadata embedded in an H.264 stream. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">pass</samp>’</dt> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</
</dl>

Default is pass. </dd> <dt><samp class="option">sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio of the stream in the VUI parameters. See H.264 table E-1. </dd> <dt><samp class="option">overscan_appropriate_flag</samp></dt> <dd> Set whether the stream is suitable for display using overscan or not (see H.264 section E.2.1). </dd> <dt><samp class="option">video_format</samp></dt> <dt><samp class="option">video_full_range_flag</samp> <dd> Set the video format in the stream (see H.264 section E.2.1 and table E-2). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.264 section E.2.1 and tables E-3, E-4 and E-5). </dd> <dt><samp class="option">chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.264 section E.2.1 and figure E-1). </dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_tick) in the VUI parameters. This is the smallest time unit representable in the stream, and in many cases represents the field rate of the stream (double the frame rate). </dd> <dt><samp class="option">fixed_frame_rate_flag</samp></dt> <dd> Set whether the stream has fixed framerate - typically this indicates that the framerate is exactly half the tick rate, but the exact meaning is dependent on interlacing and the picture structure (see H.264 section E.2.1 and table E-6). </dd> <dt><samp class="option">zero_new_constraint_set_flags</samp></dt> <dd> Zero constraint_set4_flag and constraint_set5_flag in the SPS. These bits were reserved in a previous version of the H.264 spec, and thus some hardware decoders require these to be zero. The result of zeroing this is still a valid bitstream. </dd> <dt><samp class="option">crop_left</samp></dt> <dt><samp class="option">crop_right</samp></dt> <dt><samp

class=”option”>crop_top</samp></dt> <dt><samp class=”option”>crop_bottom</samp></dt> <dd> Set the frame cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled or the stream is interlaced (see H.264 section 7.4.2.1.1). </dd> <dt><samp class=”option”>sei_user_data</samp></dt> <dd> Insert a string as SEI unregistered user data. The argument must be of the form UUID+string, where the UUID is as hex digits possibly separated by hyphens, and the string can be anything. For example, ‘<samp class=”samp”>086f3693-b7b3-4f2c-9653-21492feee5b8+hello</samp>’ will insert the string “hello” associated with the given UUID. </dd> <dt><samp class=”option”>delete_filler</samp></dt> <dd> Deletes both filler NAL units and filler SEI messages. </dd> <dt><samp class=”option”>display_orientation</samp></dt> <dd> Insert, extract or remove Display orientation SEI messages. See H.264 section D.1.27 and D.2.27 for syntax and semantics. <dl class=”table”> <dt>‘<samp class=”samp”>pass</samp>’</dt> <dt>‘<samp class=”samp”>insert</samp>’</dt> <dt>‘<samp class=”samp”>remove</samp>’</ <dt>‘<samp class=”samp”>extract</samp>’</dt> </dl>

Default is pass. Insert mode works in conjunction with <code class=”code”>rotate</code> and <code class=”code”>flip</code> options. Any pre-existing Display orientation messages will be removed in insert or remove mode. Extract mode attaches the display matrix to the packet as side data. </dd> <dt><samp class=”option”>rotate</samp></dt> <dd> Set rotation in display orientation SEI (anticlockwise angle in degrees). Range is -360 to +360. Default is NaN. </dd> <dt><samp class=”option”>flip</samp></dt> <dd> Set flip in display orientation SEI. <dl class=”table”> <dt>‘<samp class=”samp”>horizontal</samp>’</dt> <dt>‘<samp class=”samp”>vertical</samp>’</dt> </dl>

Default is unset. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set the level in the SPS. Refer to H.264 section A.3 and tables A-1 to A-5. The argument must be the name of a level (for example, ‘<samp class=”samp”>4.2</samp>’), a level_idc value (for example, ‘<samp class=”samp”>42</samp>’), or the special name ‘<samp class=”samp”>auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class=”section”>2.15 h264_mp4toannexb# TOC</h3

Convert an H.264 bitstream from length prefixed mode to start code

prefixed mode (as defined in the Annex B of the ITU-T H.264 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an H.264 stream to mpegts format with <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.mp4 -codec copy -bsf:v h264_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class="code">mpegts</code>) and raw H.264 (muxer <code class="code">h264</code>) output formats. <h3 class="section">2.16 h264_redundant_pps# TOC</h3>

This applies a specific fixup to some Blu-ray BDMV H264 streams which contain redundant PPSs. The PPSs modify irrelevant parameters of the stream, confusing other transformations which require the correct extradata. The encoder used on these impacted streams adds extra PPSs throughout the stream, varying the initial QP and whether weighted prediction was enabled. This causes issues after copying the stream into a global header container, as the starting PPS is not suitable for the rest of the stream. One side effect, for example, is seeking will return garbled output until a new PPS appears. This BSF removes the extra PPSs and rewrites the slice headers such that the stream uses a single leading PPS in the global header, which resolves the issue. <h3 class="section">2.17 hevc_metadata# TOC</h3>

Modify metadata embedded in an HEVC stream. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</dt> </dl>

</dd> <dt><samp class="option">sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio in the stream in the VUI parameters. </dd> <dt><samp class="option">video_format</samp></dt> <dt><samp class="option">video_full_range_flag</samp></dt> <dd> Set the video format in the stream (see H.265 section E.3.1 and table E.2). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.265 section E.3.1 and tables E.3, E.4 and E.5). </dd> <dt><samp class="option">chroma_sample_loc_type</samp></dt>

<dd> Set the chroma sample location in the stream (see H.265 section E.3.1 and figure E.1). </dd> <dt><samp class=”option”>tick_rate</samp></dt> <dd> Set the tick rate in the VPS and VUI parameters (time_scale / num_units_in_tick). Combined with <samp class=”option”>num_ticks_poc_diff_one</samp>, this can set a constant framerate in the stream. Note that it is likely to be overridden by container parameters when the stream is in a container. </dd> <dt><samp class=”option”>num_ticks_poc_diff_one</samp></dt> <dd> Set poc_proportional_to_timing_flag in VPS and VUI and use this value to set num_ticks_poc_diff_one_minus1 (see H.265 sections 7.4.3.1 and E.3.1). Ignored if <samp class=”option”>tick_rate</samp> is not also set. </dd> <dt><samp class=”option”>crop_left</samp></dt> <dt><samp class=”option”>crop_right</samp></dt> <dt><samp class=”option”>crop_top</samp></dt> <dt><samp class=”option”>crop_bottom</samp></dt> <dd> Set the conformance window cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled (H.265 section 7.4.3.2.1). </dd> <dt><samp class=”option”>width</samp></dt> <dt><samp class=”option”>height</samp></dt> <dd> Set width and height after crop. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set the level in the VPS and SPS. See H.265 section A.4 and tables A.6 and A.7. The argument must be the name of a level (for example, ‘<samp class=”samp”>5.1</samp>’), a general_level_idc value (for example, ‘<samp class=”samp”>153</samp>’ for level 5.1), or the special name ‘<samp class=”samp”>auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class=”section”>2.18 hevc_mp4toannexb# TOC</h3

Convert an HEVC/H.265 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.265 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class=”code”>mpegts</code>). For example to remux an MP4 file containing an HEVC stream to mpegts format with <code class=”command”>ffmpeg</code>, you can use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT.mp4 -codec copy -bsf:v hevc_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class=”code”>mpegts</code>) and raw HEVC/H.265 (muxer <code class=”code”>h265</code> or <code class=”code”>hevc</code>) output

formats. <h3 class="section">2.19 imxdump# TOC</h3>

Modifies the bitstream to fit in MOV and to be usable by the Final Cut Pro decoder. This filter only applies to the mpeg2video codec, and is likely not needed for Final Cut Pro 7 and newer with the appropriate <samp class="option">-tag:v</samp>. For example, to remux 30 MB/sec NTSC IMX to MOV: <pre class="example-preformatted">ffmpeg -i input.mxf -c copy -bsf:v imxdump -tag:v mx3n output.mov </pre>

 <h3 class="section">2.20 jpegapp# TOC</h3>

Inject JPEG APP0..APP15 segments from a length-prefixed sidecar into each MJPEG packet, after SOI. The sidecar is a sequence of records, one per encoded JPEG, in encode order: <pre class="example-preformatted">u32be length u8 payload[length] </pre>

<var class="var">length</var> does not include the 4-byte prefix and must not exceed 16 MiB. A zero <var class="var">length</var> leaves that packet unchanged. A non-empty payload must be a concatenation of complete APP segments (<code class="code">FF En</code> plus the 16-bit length that includes those two length bytes). If the payload starts with APP0 and the packet has an APP0 marker in the two bytes immediately following SOI, that APP0 segment is dropped. APP0 segments preceded by fill bytes or another segment are retained. An APP1-only payload keeps the encoder APP0: SOI, APP1, APP0, DQT, ... The filter requires <samp class="option">sidecar</samp>. Extra unused sidecar bytes are ignored; a packet without a matching record is an error. Records follow packets in arrival order. Flushing the filter does not rewind the sidecar, so a reused context needs a sidecar positioned for the packets it will see next. A sidecar read or validation error prevents further record consumption by that filter context. To resume, create a new context with a sidecar matching the remaining packets. Flushing does not clear this error.   <pre class="example-preformatted">ffmpeg -i in.mp4 -c:v mjpeg -bsf:v jpegapp=sidecar=00000004DEADBEEF frame_%d.jpg </pre>

 <h3 class="section">2.21 mjpeg2jpeg# TOC</h3>

Convert MJPEG/AVI1 packets to full JPEG/JFIF packets.

MJPEG is a video codec wherein each video frame is essentially a JPEG image. The individual frames can be extracted without loss, e.g. by   <pre class=”example-preformatted”>ffmpeg -i ../some_mjpeg.avi -c:v copy frames_%d.jpg </pre>

Unfortunately, these chunks are incomplete JPEG images, because they lack the DHT segment required for decoding. Quoting from <a class=”url” href=”http://www.digitalpreservation.gov/formats/fdd/fdd000063.shtml”>http://www.digitalpreservation.go
 Avery Lee, writing in the rec.video.desktop newsgroup in 2001, commented that "MJPEG, or at least the MJPEG in AVIs having the MJPG fourcc, is restricted JPEG with a fixed – and *omitted* – Huffman table. The JPEG must be YCbCr colorspace, it must be 4:2:2, and it must use basic Huffman encoding, not arithmetic or progressive. . . . You can indeed extract the MJPEG frames and decode them with a regular JPEG decoder, but you have to prepend the DHT segment to them, or else the decoder won’t have any idea how to decompress the data. The exact table necessary is given in the OpenDML spec." This bitstream filter patches the header of frames extracted from an MJPEG stream (carrying the AVI1 header ID and lacking a DHT segment) to produce fully qualified JPEG images.   <pre class=”example-preformatted”>ffmpeg -i mjpeg-movie.avi -c:v copy -bsf:v mjpeg2jpeg frame_%d.jpg exiftran -i -9 frame*.jpg ffmpeg -i frame_%d.jpg -c:v copy rotated.avi </pre>

 <h3 class=”section”>2.22 mjpegadump# TOC</h3>

Add an MJPEG A header to the bitstream, to enable decoding by Quicktime. <h3 class=”section”>2.23 mov2textsub# TOC</h3>

Extract a representable text file from MOV subtitles, stripping the metadata header from each subtitle packet. See also the text2movsub filter. <h3 class=”section”>2.24 mpeg2_metadata# TOC</h3>

Modify metadata embedded in an MPEG-2 stream. <dl class=”table”> <dt><samp class=”option”>display_aspect_ratio</samp></dt> <dd> Set the display aspect ratio in the stream. The following fixed values are supported: <dl class=”table”> <dt><samp

class=”option”>4/3</samp></dt> <dt><samp class=”option”>16/9</samp></dt> <dt><samp class=”option”>221/100</samp></dt> </dl> Any other value will result in square pixels being signalled instead (see H.262 section 6.3.3 and table 6-3). </dd> <dt><samp class=”option”>frame_rate</samp></dt> <dd> Set the frame rate in the stream. This is constructed from a table of known values combined with a small multiplier and divisor - if the supplied value is not exactly representable, the nearest representable value will be used instead (see H.262 section 6.3.3 and table 6-4). </dd> <dt><samp class=”option”>video_format</samp></dt> <dd> Set the video format in the stream (see H.262 section 6.3.6 and table 6-6). </dd> <dt><samp class=”option”>colour_primaries</samp></dt> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.262 section 6.3.6 and tables 6-7, 6-8 and 6-9). </dd> </dl>

 <h3 class=”section”>2.25 mpeg4_unpack_bframes# TOC</h3>

Unpack DivX-style packed B-frames. DivX-style packed B-frames are not valid MPEG-4 and were only a workaround for the broken Video for Windows subsystem. They use more space, can cause minor AV sync issues, require more CPU power to decode (unless the player has some decoded picture queue to compensate the 2,0,2,0 frame per packet style) and cause trouble if copied into a standard container like mp4 or mpeg-ps/ts, because MPEG-4 decoders may not be able to decode them, since they are not valid MPEG-4. For example to fix an AVI file containing an MPEG-4 stream with DivX-style packed B-frames using <code class=”command”>ffmpeg</code>, you can use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT.avi -codec copy -bsf:v mpeg4_unpack_bframes OUTPUT.avi </pre>

 <h3 class=”section”>2.26 noise# TOC</h3>

Damages the contents of packets or simply drops them without damaging the container. Can be used for fuzzing or testing error resilience/concealment. Parameters: <dl class=”table”> <dt><samp class=”option”>amount</samp></dt> <dd> Accepts an expression whose evaluation per-packet determines how often bytes in that packet will be modified. A value below 0 will result in a variable frequency. Default is 0 which results in no modification. However, if neither amount nor drop is specified, amount will be set to <var class=”var”>-1</var>. See below for accepted variables. </dd> <dt><samp

class=”option”>drop</samp></dt> <dd> Accepts an expression evaluated per-packet whose value determines whether that packet is dropped. Evaluation to a positive value results in the packet being dropped. Evaluation to a negative value results in a variable chance of it being dropped, roughly inverse in proportion to the magnitude of the value. Default is 0 which results in no drops. See below for accepted variables. </dd> <dt><samp class=”option”>dropamount</samp></dt> <dd> Accepts a non-negative integer, which assigns a variable chance of it being dropped, roughly inverse in proportion to the value. Default is 0 which results in no drops. This option is kept for backwards compatibility and is equivalent to setting drop to a negative value with the same magnitude i.e. <code class=”code”>dropamount=4</code> is the same as <code class=”code”>drop=-4</code>. Ignored if drop is also specified. </dd> </dl>

Both <code class=”code”>amount</code> and <code class=”code”>drop</code> accept expressions containing the following variables: <dl class=”table”> <dt>‘<samp class=”samp”>n</samp>’</dt> <dd> The index of the packet, starting from zero. </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dd> The timebase for packet timestamps. </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Packet presentation timestamp. </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Packet decoding timestamp. </dd> <dt>‘<samp class=”samp”>nopts</samp>’</dt> <dd> Constant representing AV_NOPTS_VALUE. </dd> <dt>‘<samp class=”samp”>startpts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE PTS seen in the stream. </dd> <dt>‘<samp class=”samp”>startdts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE DTS seen in the stream. </dd> <dt>‘<samp class=”samp”>duration</samp>’</dt> <dt>‘<samp class=”samp”>d</samp>’</dt <dd> Packet duration, in timebase units. </dd> <dt>‘<samp class=”samp”>pos</samp>’</dt> <dd> Packet position in input; may be -1 when unknown or not set. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Packet size, in bytes. </dd> <dt>‘<samp class=”samp”>key</samp>’</dt> <dd> Whether packet is marked as a keyframe. </dd> <dt>‘<samp class=”samp”>state</samp>’</dt> <dd> A pseudo random integer, primarily derived from the content of packet payload. </dd> </dl>

 <h4 class=”subsection”>2.26.1 Examples# TOC</h4> Apply modification to every byte but don’t drop any packets. <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf noise=1 output.mkv </pre>

Drop every video packet not marked as a keyframe after timestamp 30s but do not modify any of the remaining packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v noise=drop='gt(pts*tb\,30)*not(key)' output.mkv </pre>

Drop one second of audio every 10 seconds and add some random noise to the rest. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:a noise=amount=-1:drop='between(mod(pts*tb\,10)\,9\,10)' output.mkv </pre>

 <h3 class="section">2.27 null# TOC</h3>
This bitstream filter passes the packets through unchanged. <h3 class="section">2.28 pcm_rechunk# TOC</h3>

Repacketize PCM audio to a fixed number of samples per packet or a fixed packet rate per second. This is similar to the <a data-manual="ffmpeg-filters" href="./ffmpeg-filters.html#asetnsamples">asetnsamples audio filter but works on audio packets instead of audio frames. <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of samples per each output audio packet. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio packet with silence, so that it will contain the same number of samples (or roughly the same number of samples, see <samp class="option">frame_rate</samp>) as the previous ones. Default value is 1. </dd> <dt><samp class="option">frame_rate, r</samp></dt> <dd> This option makes the filter output a fixed number of packets per second instead of a fixed number of samples per packet. If the audio sample rate is not divisible by the frame rate then the number of samples will not be constant but will vary slightly so that each packet will start as close to the frame boundary as possible. Using this option has precedence over <samp class="option">nb_out_samples</samp>. </dd> </dl>

You can generate the well known 1602-1601-1602-1601-1602 pattern of 48kHz audio for NTSC frame rate using the <samp class="option">frame_rate</samp> option. <pre class="example-preformatted">ffmpeg -f lavfi -i sine=r=48000:d=1 -c pcm_s16le -bsf pcm_rechunk=r=30000/1001 -f framecrc - </pre>

 <h3 class="section">2.29 pgs_frame_merge# <a

class=”anchor hidden-xs”href=”#toc-pgs_005fframe_005fmerge” aria-hidden=”true”>TOC</h3>

Merge a sequence of PGS Subtitle segments ending with an "end of display set" segment into a single packet. This is required by some containers that support PGS subtitles (muxer <code class=”code”>matroska</code>). <h3 class=”section”>2.30 prores_metadata# TOC</h3>

Modify color property metadata embedded in prores stream. <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT601 625 </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> BT601 525 </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> DCI P3 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> P3 D65 </dd> </dl>

</dd> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dd> Set the color transfer. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same transfer characteristics property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT 601, BT 709, BT 2020 </dd> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt> <dd> SMPTE ST 2084 </dd> <dt>‘<samp class=”samp”>arib-std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd> </dl>

</dd> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the matrix coefficient. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same colorspace property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>&rsqu <dd> BT 601 </dd> <dt>‘<samp class=”samp”>bt2020nc</samp>’</dt> </dl> </dd> </dl>

Set Rec709 colorspace for each frame of the file <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf:v

prores_metadata=color_primaries=bt709:color_trc=bt709:colorspace=bt709 output.mov </pre>

Set Hybrid Log-Gamma parameters for each frame of the file <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v prores_metadata=color_primaries=bt2020:color_trc=arib-std-b67:colorspace=bt2020nc output.mov </pre>

 <h3 class="section">2.31 remove_extra# TOC</h3>

Remove extradata from packets. It accepts the following parameter: <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> Set which frame types to remove extradata from. <dl class="table"> <dt>‘<samp class="samp">k</samp>’</dt> <dd> Remove extradata from non-keyframes only. </dd> <dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> Remove extradata from keyframes only. </dd> <dt>‘<samp class="samp">e, all</samp>’</dt> <dd> Remove extradata from all frames. </dd> </dl> </dd> </dl>

 <h3 class="section">2.32 setts# TOC</h3> Set PTS and DTS in packets. It accepts the following parameters: <dl class="table"> <dt><samp class="option">ts</samp></dt> <dt><samp class="option">pts</samp></dt> <dt><samp class="option">dts</samp></dt> <dd> Set expressions for PTS, DTS or both. </dd> <dt><samp class="option">duration</samp></dt> <dd> Set expression for duration. </dd> <dt><samp class="option">time_base</samp></dt> <dd> Set output time base. </dd> <dt><samp class="option">prescale</samp></dt> <dd> Set whether to convert time fields to user-set output time base before evaluation of expressions. Defaults to 0. </dd> </dl>

The expressions are evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The count of the input packet. Starting from 0. </dd> <dt><samp class="option">TS</samp></dt> <dd> The demux timestamp in input in case of <code class="code">ts</code> or <code class="code">dts</code> option or presentation timestamp in case of <code class="code">pts</code> option. </dd> <dt><samp class="option">POS</samp></dt> <dd> The original position in the file of the packet, or undefined if undefined for the current packet </dd> <dt><samp class="option">DTS</samp></dt> <dd> The demux timestamp in input. </dd> <dt><samp class="option">PTS</samp></dt> <dd> The presentation timestamp

in input. </dd> <dt><samp class="option">DURATION</samp></dt>
<dd> The duration in input. </dd> <dt><samp class="option">STARTDTS</samp></dt>
<dd> The DTS of the first packet. </dd> <dt><samp
class="option">STARTPTS</samp></dt> <dd> The PTS of the first
packet. </dd> <dt><samp class="option">PREV_INDTS</samp></dt>
<dd> The previous input DTS. </dd> <dt><samp class="option">PREV_INPTS</samp></dt
<dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INDURATION</sam
<dd> The previous input duration. </dd> <dt><samp
class="option">PREV_OUTDTS</samp></dt> <dd> The previous
output DTS. </dd> <dt><samp class="option">PREV_OUTPTS</samp></dt>
<dd> The previous output PTS. </dd> <dt><samp class="option">PREV_OUTDURATION</
<dd> The previous output duration. </dd> <dt><samp
class="option">NEXT_DTS</samp></dt> <dd> The next input DTS.
 </dd> <dt><samp class="option">NEXT_PTS</samp></dt>
<dd> The next input PTS. </dd> <dt><samp class="option">NEXT_DURATION</samp></
<dd> The next input duration. </dd> <dt><samp class="option">TB</samp></dt>
<dd> The timebase in which time fields are denominated. The user-set
output timebase if prescale is enabled, else the input timebase. </dd>
<dt><samp class="option">TB_OUT</samp></dt> <dd> The out-
put timebase. </dd> <dt><samp class="option">SR</samp></dt>
<dd> The sample rate of stream packet belongs. </dd>
<dt><samp class="option">NOPTS</samp></dt> <dd> The
AV_NOPTS_VALUE constant. </dd> </dl>

For example, to set PTS equal to DTS (not recommended if B-
frames are involved): <pre class="example-
preformatted">ffmpeg -i INPUT -c:a copy -bsf:a setts=pts=DTS out.mkv
</pre>

 <h3 class="section">2.33 showinfo<span
class="pull-right"><a class="anchor hidden-xs" href="#showinfo" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-showinfo"
aria-hidden="true">TOC</h3> Log basic packet infor-
mation. Mainly useful for testing, debugging, and development. <a
name="side_005fdata"> <h3 class="section">2.34 side_data<span
class="pull-right"><a class="anchor hidden-xs" href="#side_005fdata"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
side_005fdata" aria-hidden="true">TOC</h3> Remove
side data from packets and from the stream, or select packets by their
side data. It accepts the following parameters: <dl
class="table"> <dt><samp class="option">remove</samp></dt>
<dd> Set the side data types to remove, joined with ‘<samp
class="samp">+</samp>’, from the packets and from the side data of
the stream. Each type is given by its handle; <code class="command">ffmpeg -
h bsf=side_data</code> lists them. </dd> <dt><samp class="option">select</samp></dt>
<dd> Set the side data types of the packets to keep, joined with
‘<samp class="samp">+</samp>’. A packet is kept if it

carries any of them, and dropped otherwise. The selection looks at the side data before <samp class="option">remove</samp> applies. Dropping a packet breaks the decoding of the packets that depend on it, so this is meant for analysis or for streams where each packet decodes on its own. </dd> </dl>

For example, to drop the display matrix and the replay gain of a stream: <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v side_data=remove=displaymatrix+replaygain output.mkv </pre>

 <h3 class="section">2.35 smpte436m_to_eia608# TOC</h3>

Convert from a <code class="code">SMPTE_436M_ANC</code> data stream to a <code class="code">EIA_608</code> stream, extracting the closed captions from CTA-708 CDP VANC packets, and ignoring all other data. <h3 class="section">2.36 text2movsub# TOC</h3>

Convert text subtitles to MOV subtitles (as used by the <code class="code">mov_text</code> codec) with metadata headers. See also the mov2textsub filter. <h3 class="section">2.37 trace_headers# TOC</h3>

Log trace output containing all syntax elements in the coded stream headers (everything above the level of individual coded blocks). This can be useful for debugging low-level stream issues. Supports AV1, H.264, H.265, (M)JPEG, MPEG-2 and VP9, but depending on the build only a subset of these may be available. <h3 class="section">2.38 trim# TOC</h3>

Drop packets outside a given range, similar to the trim and atrim filters, but operating on packets instead of decoded frames. The range is described by a <samp class="option">start</samp> and an <samp class="option">end</samp> bound. At least one of them must be set. Each bound is fully described by three independent choices: <ul class="itemize mark-bullet"> The discriminant it is compared against:

the packet’s presentation timestamp, its decoding timestamp, or its index. The two bounds need not use the same one, for instance a <code class=”code”>dts</code> start with a <code class=”code”>pts</code> end. The unit its value is given in: the stream time base, milliseconds, or packets. What it is counted from: zero, or its reference packet when <samp class=”option”>start_rel</samp> / <samp class=”option”>end_rel</samp> is set. For <samp class=”option”>start</samp> the reference packet is the first packet carrying that discriminant. For <samp class=”option”>end</samp> it is the first exported packet.

The <samp class=”option”>start_type</samp> and <samp class=”option”>end_type</samp> options name a combination of the three. The <code class=”code”>dur_ts</code> and <code class=”code”>dur_t_msec</code> end types are the always relative forms of <code class=”code”>pts</code> and <code class=”code”>msec_pt</code>. A zero duration therefore produces an empty stream. Every packet is judged on its own discriminant, and on nothing else. A packet not carrying the discriminant its bound uses cannot be placed, and is dropped. <samp class=”option”>start_rel</samp> makes <samp class=”option”>start</samp> an offset from a reference packet instead of an absolute value. The reference is the first packet carrying the discriminant that bound uses. Its value is added to the offset once, and the bound stays there for the rest of the stream. <samp class=”option”>end_rel</samp> anchors on the first exported packet instead, at the position <samp class=”option”>start</samp> trimmed it to. The two bounds therefore move together. Only a discriminant that cannot come back into the range ends the stream, namely the packet index or the decoding timestamp. End of stream is signaled once one of those passes the <samp class=”option”>end</samp> bound. A presentation timestamp past the end only drops its packet, so that later packets still inside the range are not lost. A bound given in a time unit can fall inside a packet rather than between two. Such a packet is trimmed. Set <samp class=”option”>trim_packets</samp> to false to keep every packet exactly as it came in: exported whole if its own value is inside the range, dropped otherwise. Audio is trimmed in sample space, through skip samples side data. The timestamps and duration are left untouched, and a decoder discards the signaled samples on output. This needs a sample rate and a time base. Audio missing either is rejected rather than trimmed on the timeline. Every other media type is trimmed by shifting the packet timestamps and shortening its duration. Trimming inside an audio packet needs the number of samples it decodes to. Where the codec does not give that number, the packet is handled as though <samp class=”option”>trim_packets</samp> were false. A skip larger than the packet carrying it continues into the packets after it. Dropping such a packet carries the remainder over to the next one. A skip that packet already carries replaces the remainder, as a decoder would, and the larger of it and this filter’s own trim wins. By default a packet outside the range is dropped, whether

or not later packets depend on it. Streams of independent frames, such as ADTS or intra only video, can be cut anywhere. Cutting an inter coded stream inside a group of pictures leaves the first packets in range undecodable. Set <samp class="option">preroll</samp> to export the packets those first packets depend on as well. They are flagged for a decoder to decode and then drop, so the frames it outputs still begin at the <samp class="option">start</samp> bound. Only a decoder acts on that flag. Use <samp class="option">preroll</samp> where the stream is decoded. A muxer writes the extra packets like any other. A remux then covers more than the requested range. <samp class="option">preroll</samp> covers the start of the range only. On a stream with B-frames an <samp class="option">end</samp> bound on the presentation timestamp can drop a packet the last frames in the range need. Those frames then decode incorrectly. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Time or index marking the start of the accepted range. Packets before it are dropped. A packet straddling it is trimmed at its start, or dropped when <samp class="option">trim_packets</samp> is false. </dd> <dt><samp class="option">start_type</samp></dt> <dd> How to interpret <samp class="option">start</samp>: <dl class="table"> <dt>‘<samp class="samp">pts</samp>’</dt> <dd> Presentation timestamp, in stream time base (default). </dd> <dt>‘<samp class="samp">dts</samp>’</dt> <dd> Decoding timestamp, in stream time base. </dd> <dt>‘<samp class="samp">pkt_index</samp>’</dt> <dd> Packet index, counting from zero. </dd> <dt>‘<samp class="samp">msec_pt</samp>’</dt> <dd> Milliseconds, matched against the presentation timestamp. </dd> <dt>‘<samp class="samp">msec_dt</samp>’</dt> <dd> Milliseconds, matched against the decoding timestamp. </dd> </dl>

</dd> <dt><samp class="option">start_rel</samp></dt> <dd> Count <samp class="option">start</samp> from the reference packet rather than from zero. Has no effect on ‘<samp class="samp">pkt_index</samp>’, whose first packet is already zero. Default false. </dd> <dt><samp class="option">end</samp></dt> <dd> Time or index marking the end of the accepted range. Signals end of stream when reached. A packet straddling it is trimmed at its end, or exported untouched when <samp class="option">trim_packets</samp> is false. </dd> <dt><samp class="option">end_type</samp></dt> <dd> How to interpret <samp class="option">end</samp>. Same values as <samp class="option">start_type</samp>, plus: <dl class="table"> <dt>‘<samp class="samp">dur_ts</samp>’</dt> <dd> Duration in stream time base. Same as ‘<samp class="samp">pts</samp>’ with <samp class="option">end_rel</samp> set. </dd> <dt>‘<samp class="samp">dur_t_msec</samp>’</dt> <dd> Duration in milliseconds. Same as ‘<samp class="samp">msec_pt</samp>’

with <samp class=”option”>end_rel</samp> set. </dd> </dl>

</dd> <dt><samp class=”option”>end_rel</samp></dt> <dd> Count <samp class=”option”>end</samp> from the first exported packet rather than from zero. The reference is that packet’s value on the discriminant <samp class=”option”>end_type</samp> names. On a timestamp it is taken at the position the packet is trimmed to. For audio that is the time of its first retained sample. The skip that packet carries can push it past the packet itself. With ‘<samp class=”samp”>pkt_index</samp>’ it exports a fixed number of packets, which can be combined with a timestamp-based <samp class=”option”>start</samp>. Always set for the duration types. Default false. </dd> <dt><samp class=”option”>trim_packets</samp></dt> <dd> Trim packets straddling a boundary. When false, packets are exported exactly as they came in, with no timestamp adjustment and no skip samples side data. A packet is then exported untouched if its value falls inside the range, and dropped otherwise. Default true. </dd> <dt><samp class=”option”>preroll</samp></dt> <dd> Export the packets a decoder needs to decode the first packet in range, rather than dropping them for falling before the <samp class=”option”>start</samp> bound. The filter holds every packet from the last keyframe on. It exports them ahead of the first packet in range, carrying <code class=”code”>AV_PKT_FLAG_DISCARD</code>. Video only. Default false. </dd> <dt><samp class=”option”>preroll_size</samp></dt> <dd> Maximum number of packets held for <samp class=”option”>preroll</samp>. A group of pictures longer than this is dropped whole, with a warning naming the count to raise it past. Default 300, which covers the keyframe intervals encoders commonly default to. </dd> </dl>

For example, to keep only the 10 first packets of an ADTS stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=end=10:end_type=pkt_index OUTPUT.aac </pre>

To keep 5 seconds starting 1.5 seconds into the stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

The same, expressed relative to the first packet so it works regardless of the stream’s starting timestamp: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:start_rel=1:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

To start on a keyframeless point of an inter coded video stream, re-encoding it so that the preroll is decoded rather than written out. The filter must run before the decoder, so it is given before the input: <div

class=”example”> <pre class=”example-preformatted”>ffmpeg -bsf:v trim=start=17000:start_type=msec_pt:end=30000:end_type=dur_t_msec:preroll=1 -i INPUT.mp4 OUTPUT.mp4 </pre>

 <h3 class=”section”>2.39 truehd_core# TOC</h3>

Extract the core from a TrueHD stream, dropping ATMOS data. <h3 class=”section”>2.40 vp9_metadata# TOC</h3>

Modify metadata embedded in a VP9 stream. <dl class=”table”> <dt><samp class=”option”>color_space</samp></dt> <dd> Set the color space value in the frame header. Note that any frame set to RGB will be implicitly set to PC range and that RGB is incompatible with profiles 0 and 2. <dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</d
<dt>‘<samp class=”samp”>smpte170</samp>’</dt>
<dt>‘<samp class=”samp”>smpte240</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>rgb</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>color_range</samp></dt> <dd> Set the color range value in the frame header. Note that any value imposed by the color space will take precedence over this value. <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dt>‘<samp class=”samp”>pc</samp>’</dt> </dl> </dd> </dl>

 <h3 class=”section”>2.41 vp9_superframe# TOC</h3>

Merge VP9 invisible (alt-ref) frames back into VP9 superframes. This fixes merging of split/segmented VP9 streams where the alt-ref frame was split from its visible counterpart. <h3 class=”section”>2.42 vp9_superframe_split# TOC</h3>

Split VP9 superframes into single frames. <h3 class=”section”>2.43 vp9_raw_reorder<a

class=”anchor hidden-xs” href=”#vp9_005fraw_005freorder” aria-hidden=”true”>#
TOC</h3>

Given a VP9 stream with correct timestamps but possibly out of order, insert additional show-existing-frame packets to correct the ordering.

 <h2 class=”chapter”>3 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavcodec

 <h2 class=”chapter”>4 Authors#
TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class=”command”>git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg.
 Maintainers for the specific components are listed in the file <samp class=”file”>MAINTAINERS</samp> in the source code tree.

<p style=”font-size: small;”> This document was generated using makeinfo.
  </body> </html>

===== .\ffmpeg\doc\ffmpeg-codecs.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> FFmpeg Codecs Documentation </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> FFmpeg Codecs Documentation
</h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”>

1 Description

2 Codec Options

3 Decoders

4 Video Decoders

<ul class="toc-numbered-mark">

4.1 av1

<ul class="toc-numbered-mark">

4.1.1 Options

4.2 hevc

<ul class="toc-numbered-mark">

4.2.1 Options

4.3 rawvideo

<ul class="toc-numbered-mark">

4.3.1 Options

4.4 libastcenc

<ul class="toc-numbered-mark">

4.4.1 Options

4.5 libdav1d

<ul class="toc-numbered-mark">

4.5.1 Options

4.6 libdavs2

4.7 libuavs3d

<ul class="toc-numbered-mark">

4.7.1 Options

4.8 libxevd

<ul class="toc-numbered-mark">

4.8.1 Options

4.9 QSV Decoders

<ul class="toc-numbered-mark">

4.9.1 Common Options

4.9.2 HEVC Options

4.10 v210

<ul class="toc-numbered-mark">

4.10.1 Options

5 Audio Decoders

<ul class="toc-numbered-mark">

5.1 ac3

<ul class="toc-numbered-mark">

5.1.1 AC-3 Decoder Options

5.2 dca

<ul class="toc-numbered-mark">

5.2.1 DCA Decoder Options

5.3 flac

<ul class="toc-numbered-mark">

5.3.1 FLAC Decoder options

5.4 ffwavesynth

5.5 libgsm

5.6 libilbc

<ul class="toc-numbered-mark">

5.6.1 Options

5.7 libmpeghdec

5.8 libopencore-amrnb

5.9 libopencore-amrwb

5.10 libopus

5.11 pcm_dvda

6 Subtitles Decoders

<ul class="toc-numbered-mark">

6.1 libaribb24

<ul class="toc-numbered-mark">

6.1.1 libaribb24 Decoder Options

6.2 libaribcaption

<ul class="toc-numbered-mark">

6.2.1 libaribcaption Decoder Options

6.2.2 libaribcaption decoder usage examples

6.3 dvbsub

<ul class="toc-numbered-mark">

6.3.1 Options

6.4 dvdsub

<ul class="toc-numbered-mark">

6.4.1 Options

6.5 libzvbi-teletext

<ul class="toc-numbered-mark">

6.5.1 Options

7 Encoders

8 Audio Encoders

<ul class="toc-numbered-mark">

8.1 aac

<ul class="toc-numbered-mark">

8.1.1 Options

8.2 ac3 and ac3_fixed

<ul class="toc-numbered-mark">

8.2.1 AC-3 Metadata

<ul class="toc-numbered-mark">

8.2.1.1 Metadata Control Options

8.2.1.2 Downmix Levels

8.2.1.3 Audio Production Information

8.2.1.4 Other Metadata Options

8.2.2 Extended Bitstream Information

<ul class="toc-numbered-mark">

8.2.2.1 Extended Bitstream Information - Part 1

8.2.2.2 Extended Bitstream Information - Part 2

8.2.3 Other AC-3 Encoding Options

8.2.4 Floating-Point-Only AC-3 Encoding Options

8.3 flac

<ul class="toc-numbered-mark">

8.3.1 Options

8.4 opus

<ul class="toc-numbered-mark">

8.4.1 Options

8.5 libfdk_aac

<ul class="toc-numbered-mark">

8.5.1 Options

8.5.2 Examples

8.6 libfdk-xhe-aac

<ul class="toc-numbered-mark">

8.6.1 Options

8.7 liblc3

<ul class="toc-numbered-mark">

8.7.1 Options

8.8 libmp3lame

<ul class="toc-numbered-mark">

8.8.1 Options

8.9 libopencore-amrnb

<ul class="toc-numbered-mark">

8.9.1 Options

8.10 libopus

<ul class="toc-numbered-mark">

8.10.1 Option Mapping

8.11 libshine

<ul class="toc-numbered-mark">

8.11.1 Options

8.12 libtwolame

<ul class="toc-numbered-mark">

8.12.1 Options

8.13 libvo-amrwbenc

<ul class="toc-numbered-mark">

8.13.1 Options

8.14 libvorbis

<ul class="toc-numbered-mark">

8.14.1 Options

8.15 mjpeg

<ul class="toc-numbered-mark">

8.15.1 Options

8.16 wavpack

<ul class="toc-numbered-mark">

8.16.1 Options

<ul class="toc-numbered-mark">

8.16.1.1 Shared options

8.16.1.2 Private options

9 Video Encoders

<ul class="toc-numbered-mark">

9.1 a64_multi, a64_multi5

9.2 Cinepak

<ul class="toc-numbered-mark">

9.2.1 Options

9.3 ffv1
<ul class="toc-numbered-mark">
9.3.1 Options

9.4 GIF
<ul class="toc-numbered-mark">
9.4.1 Options

9.5 Hap
<ul class="toc-numbered-mark">
9.5.1 Options

9.6 jpeg2000
<ul class="toc-numbered-mark">
9.6.1 Options

9.7 librav1e
<ul class="toc-numbered-mark">
9.7.1 Options

9.8 libaom-av1
<ul class="toc-numbered-mark">
9.8.1 Options

9.9 liboapv
<ul class="toc-numbered-mark">
9.9.1 Options

9.10 libsvtav1

<ul class="toc-numbered-mark">
9.10.1 Options

9.11 libsvtjpegxs
<ul class="toc-numbered-mark">
9.11.1 Options

9.12 libjxl
<ul class="toc-numbered-mark">
9.12.1 Options

9.13 libkvazaar
<ul class="toc-numbered-mark">
9.13.1 Options

9.14 libopenh264
<ul class="toc-numbered-mark">
9.14.1 Options

9.15 libtheora
<ul class="toc-numbered-mark">
9.15.1 Options
9.15.2 Examples

9.16 libvpx
<ul class="toc-numbered-mark">
9.16.1 Options

9.17 libvvenc
<ul class="toc-numbered-mark">

9.17.1 Supported Pixel Formats

9.17.2 Options

9.18 libwebp

<ul class="toc-numbered-mark">

9.18.1 Pixel Format

9.18.2 Options

9.19 libastcenc

<ul class="toc-numbered-mark">

9.19.1 Pixel Format

9.19.2 Options

9.20 libx264, libx264rgb

<ul class="toc-numbered-mark">

9.20.1 Supported Pixel Formats

9.20.2 Options

9.21 libx265

<ul class="toc-numbered-mark">

9.21.1 Options

9.22 libxavs2

<ul class="toc-numbered-mark">

9.22.1 Options

9.23 libxeve

<ul class="toc-numbered-mark">

9.23.1 Options

9.24 libxvid

<ul class="toc-numbered-mark">

9.24.1 Options

9.25 MediaCodec

9.26 MediaFoundation

<ul class="toc-numbered-mark">

9.26.1 Options

9.26.2 Examples

9.27 Microsoft RLE

<ul class="toc-numbered-mark">

9.27.1 Options

9.28 mpeg2

<ul class="toc-numbered-mark">

9.28.1 Options

9.29 NVENC encoders

<ul class="toc-numbered-mark">

9.29.1 Hardware input and devices

9.29.2 Generic options

9.29.3 Common options

<ul class="toc-numbered-mark">

9.29.3.1 Presets and tuning

9.29.3.2 Rate control

9.29.3.3 Lookahead, adaptive quantization and B-frames

9.29.3.4 SEI and side data

9.29.3.5 Other common options

9.29.4 H.264 and HEVC options

9.29.5 HEVC and AV1 options

9.29.6 H.264 options

9.29.7 HEVC options

9.29.8 AV1 options

9.30 png

<ul class="toc-numbered-mark">

9.30.1 Options

9.30.2 Private options

9.31 ProRes

<ul class="toc-numbered-mark">

9.31.1 Private Options for prores-ks

9.31.2 Speed considerations

9.32 QSV Encoders

<ul class="toc-numbered-mark">

9.32.1 Ratecontrol Method

9.32.2 Global Options -> MSDK Options

9.32.3 Common Options

9.32.4 Runtime Options

9.32.5 H264 options

9.32.6 HEVC Options

9.32.7 MPEG2 Options

9.32.8 VP9 Options

9.32.9 AV1 Options

9.33 snow

<ul class="toc-numbered-mark">

9.33.1 Options

9.34 VAAPI encoders

9.35 vbn

<ul class="toc-numbered-mark">

9.35.1 Options

9.36 vc2

<ul class="toc-numbered-mark">

9.36.1 Options

10 Subtitles Encoders

<ul class="toc-numbered-mark">

10.1 dvbsub

<ul class="toc-numbered-mark">

10.1.1 Options

10.2 dvdsub

<ul class="toc-numbered-mark">

10.2.1 Options

10.3 lrc

<ul class="toc-numbered-mark">

10.3.1 Options

11 See Also

12 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes the codecs (decoders and encoders) provided by the libavcodec library.

 <h2 class="chapter">2 Codec Options# TOC</h2>

libavcodec provides some generic global options, which can be set on all the encoders and decoders. In addition, each codec may support so-called private options, which are specific for a given codec. Sometimes, a global option may only affect a specific kind of codec, and may be nonsensical or ignored by another, so you need to be aware of the meaning of the specified options. Also some options are meant only for decoding or encoding. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVCodecContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follow: <dl class="table"> <dt><samp class="option">b <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set bitrate in bits/s. Default value is 200K. </dd> <dt><samp class="option">ab <var class="var">integer</var> (encoding,audio)</samp><
<dd> Set audio bitrate (in bits/s). Default value is 128K. </dd> <dt><samp class="option">bt <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set video bitrate tolerance (in bits/s). In 1-pass mode, bitrate tolerance specifies how far ratecontrol is willing to deviate from the target average bitrate value. This is not related to min/max bitrate. Lowering tolerance too much has an adverse effect on quality. </dd> <dt><samp class="option">flags <var class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></dt> <dd> Set generic flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">mv4</samp>’</dt> <dd> Use four motion vector by macroblock (mpeg4). </dd> <dt>‘<samp class="samp">qpel</samp>’</dt> <dd> Use 1/4 pel motion compensation. </dd> <dt>‘<samp class="samp">loop</samp>’</dt> <dd> Use loop filter. </dd> <dt>‘<samp class="samp">qscale</samp>’</dt> <dd> Use fixed qscale. </dd> <dt>‘<samp class="samp">pass1</samp>’</dt> <dd> Use internal 2pass ratecontrol in first pass mode. </dd> <dt>‘<samp class="samp">pass2</samp>’</dt> <dd> Use internal 2pass ratecontrol in second pass mode. </dd> <dt>‘<samp class="samp">gray</samp>’</dt> <dd> Only decode/encode grayscale. </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> Set error[?] variables during encoding. </dd> <dt>‘<samp class="samp">truncated</samp>’</dt> <dd> Input bitstream might be randomly truncated. </dd> <dt>‘<samp class="samp">drop_changed</samp>’</dt> <dd> Don’t output frames whose parameters differ from first decoded frame in stream. Error

AVERROR_INPUT_CHANGED is returned when a frame is dropped.
</dd> <dt>‘<samp class="samp">ildct</samp>’</dt>
<dd> Use interlaced DCT. </dd> <dt>‘<samp class="samp">low_delay</samp>’<
<dd> Force low delay. </dd> <dt>‘<samp class="samp">global_header</samp>’<
<dd> Place global headers in extradata instead of every keyframe.
</dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. (except (I)DCT). This ensures that file and data checksums are reproducible and match between platforms. Its primary use is for regression testing.
</dd> <dt>‘<samp class="samp">aic</samp>’</dt>
<dd> Apply H263 advanced intra coding / mpeg4 ac prediction.
</dd> <dt>‘<samp class="samp">ilme</samp>’</dt>
<dd> Apply interlaced motion estimation. </dd> <dt>‘<samp class="samp">cgop</samp>’</dt> <dd> Use closed gop.
</dd> <dt>‘<samp class="samp">output_corrupt</samp>’</dt>
<dd> Output even potentially corrupted frames. </dd> </dl>

</dd>

<dt><samp class="option">time_base <var class="var">rational number</var></samp></dt>

<dd> Set codec time base.

It is the fundamental unit of time (in seconds) in terms of which

frame timestamps are represented. For fixed-fps content, timebase

should be <code class="code">1 / frame_rate</code> and timestamp increments should be

identically 1.

</dd>

<dt><samp class="option">g <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set the group of picture (GOP) size. Default value is 12.

</dd>

<dt><samp class="option">ar <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set audio sampling rate (in Hz).

</dd>

<dt><samp class="option">ac <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set number of audio channels.

</dd>

<dt><samp class="option">cutoff <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set cutoff bandwidth. (Supported only by selected encoders, see

their respective documentation sections.)

</dd>

<dt><samp class="option">frame_size <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set audio frame size.

Each submitted frame except the last must contain exactly frame_size

samples per channel. May be 0 when the codec has

CODEC_CAP_VARIABLE_FRAME_SIZE set, in that case the frame size is not

restricted. It is set by some decoders to indicate constant frame

size.

</dd>

<dt><samp class="option">frame_number <var class="var">integer</var></samp></dt>

<dd> Set the frame number.

</dd>

<dt><samp class="option">delay <var class="var">integer</var></samp></dt>

<dt><samp class="option">qcomp <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale compression (VBR). It is used as a constant

in the ratecontrol equation. Recommended range for default rc_eq:

0.0-1.0.

</dd>

<dt><samp class="option">qblur <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale blur (VBR).

</dd>

<dt><samp class="option">qmin <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set min video quantizer scale (VBR). Must be included between -1 and

69, default value is 2.

</dd>

<dt><samp class="option">qmax <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max video quantizer scale (VBR). Must be included between -1 and

1024, default value is 31.

</dd>

<dt><samp class="option">qdiff <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max difference between the quantizer scale (VBR).

</dd>

<dt><samp class="option">bf <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max number of B frames between non-B-frames.

Must be an integer between -1 and 16. 0 means that B-frames are

disabled. If a value of -1 is used, it will choose an automatic value

depending on the encoder.

Default value is 0.

</dd>

<dt><samp class="option">b_qfactor <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set qp factor between P and B frames.

</dd>

<dt><samp class="option">codec_tag <var class="var">integer</var></samp></dt>

<dt><samp class="option">bug <var class="var">flags</var> (decoding,video)</samp></dt>

<dd> Workaround not auto detected encoder bugs.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">autodetect</samp>’</dt>

<dt>‘<samp class="samp">xvid_ilace</samp>’</dt>

<dd> Xvid interlacing bug (autodetected if fourcc==XVIX)

</dd>

<dt>‘<samp class="samp">ump4</samp>’</dt>

<dd> (autodetected if fourcc==UMP4)

</dd>

<dt>‘<samp class="samp">no_padding</samp>’</dt>

<dd> padding bug (autodetected)

</dd>

<dt>‘<samp class="samp">amv</samp>’</dt>

<dt>‘<samp class="samp">qpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">std_qpel</samp>’</dt>

<dd> old standard qpel (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">qpel_chroma2</samp>’</dt>

<dt>‘<samp class="samp">direct_blocksize</samp>’</dt>

<dd> direct-qpel-blocksize bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">edge</samp>’</dt>

<dd> edge padding bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">hpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">dc_clip</samp>’</dt>

<dt>‘<samp class="samp">ms</samp>’</dt>

<dd> Workaround various bugs in microsoft broken decoders.

</dd>

<dt>‘<samp class="samp">trunc</samp>’</dt>

<dd> trancated frames

</dd>

<dt>‘<samp class="samp">h264_dp_nnz</samp>’</dt>

<dd> H.264 nC derivation for partitioned slices, as done by JM rather than
by

9.2.1 (autodetected)

</dd>

</dl>

</dd> <dt><samp class="option">strict <var class="var">integer</var>
(decoding/encoding,audio,video)</samp></dt>
<dd> Specify how strictly to follow the standards. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt>
<dd> strictly conform to an older more strict version of the spec or reference
software </dd> <dt>‘<samp class="samp">strict</samp>’</dt>
<dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt>
<dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow
unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt>
<dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note:

experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> <dt><samp class="option">b_qoffset <var class="var">float</var> (encoding,video)</samp></dt> <dd> Set QP offset between P and B frames. </dd> <dt><samp class="option">err_detect <var class="var">flags</var> (decoding,audio,video)</samp></dt> <dd> Set error detection flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt> <dd> verify embedded CRCs </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> detect bitstream specification deviations </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> detect improper bitstream length </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> abort decoding on minor error detection </dd> <dt>‘<samp class="samp">ignore_err</samp>’</dt> <dd> ignore decoding errors, and continue decoding. This is useful if you want to analyze the content of a video and thus want everything to be decoded no matter what. This option will not result in a video that is pleasing to watch in case of errors. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> consider things that violate the spec and have not been seen in the wild as errors </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> consider all spec non compliancies as errors </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> consider things that a sane encoder should not do as an error </dd> </dl>

</dd> <dt><samp class="option">has_b_frames <var class="var">integer</var></samp></dt> <dt><samp class="option">block_align <var class="var">integer</var></samp></dt> <dt><samp class="option">rc_override_count <var class="var">integer</var></samp></dt> <dt><samp class="option">maxrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set max bitrate tolerance (in bits/s). Requires bufsize to be set. </dd> <dt><samp class="option">minrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set min bitrate tolerance (in bits/s). Most useful in setting up a CBR encode. It is of little use elsewise. </dd> <dt><samp class="option">bufsize <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set ratecontrol buffer size (in bits). </dd> <dt><samp class="option">i_qfactor <var class="var">float</var> (encoding,video)</sam <dd> Set QP factor between P and I frames. </dd> <dt><samp class="option">i_qoffset <var class="var">float</var> (encoding,video)</sam <dd> Set QP offset between P and I frames. </dd> <dt><samp class="option">dct <var class="var">integer</var> (encoding,video)</samp> <dd> Set DCT algorithm. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> autoselect a good one (default) </dd> <dt>‘<samp

class=”samp”>fastint</samp>’</dt> <dd> fast integer
</dd> <dt>‘<samp class=”samp”>int</samp>’</dt>
<dd> accurate integer </dd> <dt>‘<samp class=”samp”>mmx</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>faan</samp>’</dt> <dd> floating point AAN
DCT </dd> </dl>

</dd> <dt><samp class=”option”>lumi_mask <var class=”var”>float</var>
(encoding,video)</samp></dt> <dd> Compress
bright areas stronger than medium ones. </dd> <dt><samp
class=”option”>tcplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set temporal complexity masking. </dd> <dt><samp
class=”option”>scplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set spatial complexity masking. </dd> <dt><samp
class=”option”>p_mask <var class=”var”>float</var> (encoding,video)</samp
<dd> Set inter masking. </dd> <dt><samp class=”option”>dark_mask
<var class=”var”>float</var> (encoding,video)</samp></dt>
<dd> Compress dark areas stronger than medium ones.
</dd> <dt><samp class=”option”>idct <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Select IDCT implementation. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dt>‘<samp class=”samp”>int</samp>’</dt> <dt>‘<samp
class=”samp”>simple</samp>’</dt> <dt>‘<samp class=”samp”>simplemmx</samp>&rsqu
<dt>‘<samp class=”samp”>simpleauto</samp>’</dt>
<dd> Automatically pick a IDCT compatible with the simple one
 </dd> <dt>‘<samp class=”samp”>arm</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>sh4</samp>’</dt> <dt>‘<samp class=”samp”>simplearm</samp>’<
<dt>‘<samp class=”samp”>simplearmv5te</samp>’</dt>
<dt>‘<samp class=”samp”>simplearmv6</samp>’</dt>
<dt>‘<samp class=”samp”>simpleneon</samp>’</dt>
<dt>‘<samp class=”samp”>xvid</samp>’</dt> <dt>‘<samp
class=”samp”>faani</samp>’</dt> <dd> floating point AAN
IDCT </dd> </dl>

</dd> <dt><samp class=”option”>slice_count <var class=”var”>integer</var></samp></dt>
<dt><samp class=”option”>ec <var class=”var”>flags</var> (<em
class=”emph”>decoding,video)</samp></dt> <dd> Set error
concealment strategy. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>guess_mvs</samp>’</dt>
<dd> iterative motion vector (MV) search (slow) </dd> <dt>‘<samp
class=”samp”>deblock</samp>’</dt> <dd> use strong deblock
filter for damaged MBs </dd> <dt>‘<samp class=”samp”>favor_inter</samp>’</dt>
<dd> favor predicting from the previous frame instead of the current
</dd> </dl>

</dd> <dt><samp class="option">bits_per_coded_sample <var class="var">integer</var></samp></d
<dt><samp class="option">aspect <var class="var">rational number</var>
(encoding,video)</samp></dt> <dd> Set
sample aspect ratio. </dd> <dt><samp class="option">sar <var
class="var">rational number</var> (encoding,video)</samp></dt>
<dd> Set sample aspect ratio. Alias to <var class="var">aspect</var>.
 </dd> <dt><samp class="option">debug <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Print specific debug info. Possible values: <dl
class="table"> <dt>‘<samp class="samp">pict</samp>’</dt>
<dd> picture info </dd> <dt>‘<samp class="samp">rc</samp>’</dt>
<dd> rate control </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt>
<dt>‘<samp class="samp">mb_type</samp>’</dt> <dd> macroblock
(MB) type </dd> <dt>‘<samp class="samp">qp</samp>’</dt>
<dd> per-block quantization parameter (QP) </dd> <dt>‘<samp
class="samp">dct_coeff</samp>’</dt> <dt>‘<samp
class="samp">green_metadata</samp>’</dt> <dd> display
complexity metadata for the upcoming frame, GoP or for a given duration.
 </dd> <dt>‘<samp class="samp">skip</samp>’</dt>
<dt>‘<samp class="samp">startcode</samp>’</dt> <dt>‘<samp
class="samp">er</samp>’</dt> <dd> error recognition
</dd> <dt>‘<samp class="samp">mmco</samp>’</dt>
<dd> memory management control operations (H.264) </dd>
<dt>‘<samp class="samp">bugs</samp>’</dt> <dt>‘<samp
class="samp">buffers</samp>’</dt> <dd> picture buffer alloca-
tions </dd> <dt>‘<samp class="samp">thread_ops</samp>’</dt>
<dd> threading operations </dd> <dt>‘<samp class="samp">nomc</samp>’</dt
<dd> skip motion compensation </dd> </dl>

</dd> <dt><samp class="option">cmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
full pel me compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum

of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">subcmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
sub pel me compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">mbcmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
macroblock compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp

class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>ildctcmp <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
interlaced dct compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>dia_size <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
diamond type & size for motion estimation. <dl class=”table”>
<dt>‘<samp class=”samp”>(1024, INT_MAX)</samp>’</dt>
<dd> full motion estimation(slowest) </dd> <dt>‘<samp
class=”samp”>(768, 1024]</samp>’</dt> <dd> umh mo-
tion estimation </dd> <dt>‘<samp class=”samp”>(512,

768]</samp>’</dt> <dd> hex motion estimation </dd>
<dt>‘<samp class=”samp”>(256, 512]</samp>’</dt>
<dd> l2s diamond motion estimation </dd> <dt>‘<samp
class=”samp”>[2,256]</samp>’</dt> <dd> var diamond
motion estimation </dd> <dt>‘<samp class=”samp”>(-1,
2)</samp>’</dt> <dd> small diamond motion estimation
</dd> <dt>‘<samp class=”samp”>-1</samp>’</dt>
<dd> funny diamond motion estimation </dd> <dt>‘<samp
class=”samp”>(INT_MIN, -1)</samp>’</dt> <dd> sab dia-
mond motion estimation </dd> </dl>

</dd> <dt><samp class=”option”>last_pred <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
amount of motion predictors from the previous frame. </dd>
<dt><samp class=”option”>precmp <var class=”var”>integer</var> (<em
class=”emph”>encoding,video)</samp></dt> <dd> Set pre
motion estimation compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>pre_dia_size <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
diamond type & size for motion estimation pre-pass. </dd>
<dt><samp class=”option”>subq <var class=”var”>integer</var> (<em
class=”emph”>encoding,video)</samp></dt> <dd> Set sub pel
motion estimation quality. </dd> <dt><samp class=”option”>me_range
<var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set limit motion vectors range (1023 for DivX player). </dd>
<dt><samp class="option">global_quality <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">slice_flags <var class="var">integer</var></samp></dt>
<dt><samp class="option">mbd <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set mac-
roblock decision algorithm (high quality mode). Possible values:
<dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt>
<dd> use mbcmp (default) </dd> <dt>‘<samp class="samp">bits</samp>’</dt>
<dd> use fewest bits </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> use best rate distortion </dd> </dl>

</dd> <dt><samp class="option">rc_init_occupancy <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
number of bits which should be loaded into the rc buffer before de-
coding starts. </dd> <dt><samp class="option">flags2 <var
class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></d
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">fast</samp>’</dt> <dd> Allow non spec compli-
ant speedup tricks. </dd> <dt>‘<samp class="samp">noout</samp>’</dt>
<dd> Skip bitstream encoding. </dd> <dt>‘<samp
class="samp">ignorecrop</samp>’</dt> <dd> Ignore cropping
information from sps. </dd> <dt>‘<samp class="samp">local_header</samp>’</dt>
<dd> Place global headers at every keyframe instead of in extradata.
</dd> <dt>‘<samp class="samp">chunks</samp>’</dt>
<dd> Frame data might be split into multiple chunks. </dd>
<dt>‘<samp class="samp">showall</samp>’</dt> <dd> Show
all frames before the first keyframe. </dd> <dt>‘<samp
class="samp">export_mvs</samp>’</dt> <dd> Export motion
vectors into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">skip_manual</samp>’</dt>
<dd> Do not skip samples and export skip information as frame side data.
</dd> <dt>‘<samp class="samp">ass_ro_flush_noop</samp>’</dt>
<dd> Do not reset ASS ReadOrder field on flush. </dd>
<dt>‘<samp class="samp">icc_profiles</samp>’</dt>
<dd> Generate/parse embedded ICC profiles from/to colorimetry tags.
</dd> <dt>‘<samp class="samp">fixed_frame_size</samp>’</dt>
<dd> Force audio encoders to use a fixed frame size. </dd> </dl>

</dd> <dt><samp class="option">export_side_data <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">mvs</samp>’</dt> <dd> Export motion vectors
into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">prft</samp>’</dt>

<dd> Export encoder Producer Reference Time into packet side-data (see <code class="code">AV_PKT_DATA_PRFT</code>) for codecs that support it. </dd> <dt>‘<samp class="samp">venc_params</samp>’</dt> <dd> Export video encoding parameters through frame side data (see <code class="code">AV_FRAME_DATA_VIDEO_ENC_PARAMS</code>) for codecs that support it. At present, those are H.264 and VP9. </dd> <dt>‘<samp class="samp">film_grain</samp>’</dt> <dd> Export film grain parameters through frame side data (see <code class="code">AV_FRAME_DATA_FILM_GRAIN_PARAMS</code>). Supported at present by AV1 decoders. </dd> <dt>‘<samp class="samp">enhancements</samp>’</dt> <dd> Export picture enhancement metadata through frame side data, e.g. LCEVC (see <code class="code">AV_FRAME_DATA_LCEVC</code>). </dd> </dl>

</dd> <dt><samp class="option">threads <var class="var">integer</var> (decoding/encoding,video)</samp></dt> <dd> Set the number of threads to be used, in case the selected codec implementation supports multi-threading. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, 0</samp>’</dt> <dd> automatically select the number of threads to set </dd> </dl>

Default value is ‘<samp class="samp">auto</samp>’. </dd> <dt><samp class="option">dc <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set intra_dc_precision. </dd> <dt><samp class="option">nssew <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set nsse weight. </dd> <dt><samp class="option">skip_top <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the top which are skipped. </dd> <dt><samp class="option">skip_bottom <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the bottom which are skipped. </dd> <dt><samp class="option">profile <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set encoder codec profile. Default value is ‘<samp class="samp">unknown</samp>’. Encoder specific profiles are documented in the relevant encoder documentation. </dd> <dt><samp class="option">level <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set the encoder level. This level depends on the specific codec, and might correspond to the profile level. It is set by default to ‘<samp class="samp">unknown</samp>’. Possible values: <dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> </dl>

</dd> <dt><samp class="option">lowres <var class="var">integer</var> (decoding,audio,video)</samp></dt> <dd> Decode

at 1= 1/2, 2=1/4, 3=1/8 resolutions. </dd> <dt><samp
class="option">mblmin <var class="var">integer</var> (encoding,video)</sam
<dd> Set min macroblock lagrange factor (VBR). </dd>
<dt><samp class="option">mblmax <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
max macroblock lagrange factor (VBR). </dd> <dt><samp
class="option">skip_loop_filter <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dt><samp class="option">skip_idct
<var class="var">integer</var> (decoding,video)</samp></dt>
<dt><samp class="option">skip_pred <var class="var">integer</var>
(decoding,video)</samp></dt> <dt><samp
class="option">skip_frame <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dd> Make
decoder discard processing depending on the frame type selected by the
option value. <samp class="option">skip_loop_filter</samp>
skips frame loop filtering, <samp class="option">skip_idct</samp> skips
frame IDCT/dequantization, <samp class="option">skip_pred</samp>
skips prediction (intra prediction and motion compensation), <samp
class="option">skip_frame</samp> skips decoding. With both
<samp class="option">skip_pred</samp> and <samp class="option">skip_idct</samp>
set to ‘<samp class="samp">all</samp>’ the decoder may skip
all pixel operations and output frames with undefined pixels but valid metadata,
such as timestamps and frame types. Possible values: <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> Discard no frame. </dd> <dt>‘<samp class="samp">default</samp>’</dt>
<dd> Discard useless frames like 0-sized frames. </dd>
<dt>‘<samp class="samp">noref</samp>’</dt> <dd> Discard
all non-reference frames. </dd> <dt>‘<samp class="samp">bidir</samp>’</dt>
<dd> Discard all bidirectional frames. </dd> <dt>‘<samp
class="samp">nokey</samp>’</dt> <dd> Discard all frames ex-
cepts keyframes. </dd> <dt>‘<samp class="samp">nointra</samp>’</dt>
<dd> Discard all frames except I frames. </dd> <dt>‘<samp
class="samp">all</samp>’</dt> <dd> Discard all frames.
</dd> </dl>

Default value is ‘<samp class="samp">default</samp>’.
 </dd> <dt><samp class="option">bidir_refine <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Refine
the two motion vectors used in bidirectional macroblocks. </dd>
<dt><samp class="option">keyint_min <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
minimum interval between IDR-frames. </dd> <dt><samp
class="option">refs <var class="var">integer</var> (encoding,video)</samp>
<dd> Set reference frames to consider for motion compensation.
</dd> <dt><samp class="option">trellis <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dd> Set

rate-distortion optimal quantization. </dd> <dt><samp class="option">mv0_threshold
<var class="var">integer</var> (encoding,video)</samp></dt>
<dt><samp class="option">compression_level <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">bits_per_raw_sample <var class="var">integer</var></samp></dt>
<dt><samp class="option">channel_layout <var class="var">integer</var>
(decoding/encoding,audio)</samp></dt>
<dd> See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-
layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual
for the required syntax. </dd> <dt><samp class="option">rc_max_vbv_use
<var class="var">float</var> (encoding,video)</samp></dt>
<dt><samp class="option">rc_min_vbv_use <var class="var">float</var>
(encoding,video)</samp></dt> <dt><samp
class="option">color_primaries <var class="var">integer</var> (<em
class="emph">decoding/encoding,video)</samp></dt> <dd> Possible
values: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> Film </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt>
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">smpte431</samp>’</d
<dd> SMPTE 431-2 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt>
<dd> SMPTE 432-1 </dd> <dt>‘<samp class="samp">jedec-
p22</samp>’</dt> <dd> JEDEC P22 </dd> </dl>

</dd> <dt><samp class="option">color_trc <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dd> BT.709 </dd>
<dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dt>‘<samp class="samp">log100</samp>’</dt> <dd> Log
</dd> <dt>‘<samp class="samp">log_sqrt</samp>’</dt>
<dt>‘<samp class="samp">log316</samp>’</dt> <dd> Log
square root </dd> <dt>‘<samp class="samp">iec61966_2_4</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-4</samp>’</dt>
<dd> IEC 61966-2-4 </dd> <dt>‘<samp class="samp">bt1361</samp>’</dt>
<dt>‘<samp class="samp">bt1361e</samp>’</dt> <dd> BT.1361

</dd> <dt>‘<samp class="samp">iec61966_2_1</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-1</samp>’</dt>
<dd> IEC 61966-2-1 </dd> <dt>‘<samp class="samp">bt2020_10</samp>’</dt>
<dt>‘<samp class="samp">bt2020_10bit</samp>’</dt>
<dd> BT.2020 - 10 bit </dd> <dt>‘<samp class="samp">bt2020_12</samp>’</d
<dt>‘<samp class="samp">bt2020_12bit</samp>’</dt>
<dd> BT.2020 - 12 bit </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt
<dd> SMPTE ST 2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">arib-
std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>
</dl>

</dd> <dt><samp class="option">colorspace <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">rgb</samp>’</dt> <dd> RGB </dd>
<dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709
</dd> <dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">ycocg</samp>’</dt>
<dd> YCOCG </dd> <dt>‘<samp class="samp">bt2020nc</samp>’</dt>
<dt>‘<samp class="samp">bt2020_ncl</samp>’</dt>
<dd> BT.2020 NCL </dd> <dt>‘<samp class="samp">bt2020c</samp>’</dt>
<dt>‘<samp class="samp">bt2020_cl</samp>’</dt>
<dd> BT.2020 CL </dd> <dt>‘<samp class="samp">smpte2085</samp>’</dt>
<dd> SMPTE 2085 </dd> <dt>‘<samp class="samp">chroma-
derived-nc</samp>’</dt> <dd> Chroma-derived NCL </dd>
<dt>‘<samp class="samp">chroma-derived-c</samp>’</dt>
<dd> Chroma-derived CL </dd> <dt>‘<samp class="samp">ictcp</samp>’</dt>
<dd> ICtCp </dd> </dl>

</dd> <dt><samp class="option">color_range <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> If used as input parameter, it serves as a hint to the de-
coder, which color_range the input has. Possible values: <dl
class="table"> <dt>‘<samp class="samp">tv</samp>’</dt>
<dt>‘<samp class="samp">mpeg</samp>’</dt> <dt>‘<samp
class="samp">limited</samp>’</dt> <dd> MPEG (219*2^(n-
8)) </dd> <dt>‘<samp class="samp">pc</samp>’</dt>
<dt>‘<samp class="samp">jpeg</samp>’</dt> <dt>‘<samp
class="samp">full</samp>’</dt> <dd> JPEG (2^n-1) </dd>
</dl>

</dd> <dt><samp class="option">chroma_sample_location <var

class=”var”>integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>left</samp>’</dt> <dt>‘<samp class=”samp”>center</samp>’</dt>
<dt>‘<samp class=”samp”>topleft</samp>’</dt> <dt>‘<samp
class=”samp”>top</samp>’</dt> <dt>‘<samp class=”samp”>bottomleft</samp>’<
<dt>‘<samp class=”samp”>bottom</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>alpha_mode <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>premultiplied</samp>’</dt> <dt>‘<samp
class=”samp”>straight</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>log_level_offset <var class=”var”>integer</var></samp></dt>
<dd> Set the log level offset. </dd> <dt><samp class=”option”>slices
<var class=”var”>integer</var> (encoding,video)</samp></dt>
<dd> Number of slices, used in parallelized encoding. </dd>
<dt><samp class=”option”>thread_type <var class=”var”>flags</var>
(decoding/encoding,video)</samp></dt>
<dd> Select which multithreading methods to use. Use
of ‘<samp class=”samp”>frame</samp>’ will increase decod-
ing delay by one frame per thread, so clients which cannot provide future
frames should not use it. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>slice</samp>’</dt> <dd> Decode
more than one part of a single frame at once. Multithreading
using slices works only when the video was encoded with slices.
</dd> <dt>‘<samp class=”samp”>frame</samp>’</dt>
<dd> Decode more than one frame at once. </dd> </dl>

Default value is ‘<samp class=”samp”>slice+frame</samp>’.
 </dd> <dt><samp class=”option”>audio_service_type <var
class=”var”>integer</var> (encoding,audio)</samp></dt>
<dd> Set audio service type. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>ma</samp>’</dt>
<dd> Main Audio Service </dd> <dt>‘<samp class=”samp”>ef</samp>’</dt>
<dd> Effects </dd> <dt>‘<samp class=”samp”>vi</samp>’</dt>
<dd> Visually Impaired </dd> <dt>‘<samp class=”samp”>hi</samp>’</dt>
<dd> Hearing Impaired </dd> <dt>‘<samp class=”samp”>di</samp>’</dt>
<dd> Dialogue </dd> <dt>‘<samp class=”samp”>co</samp>’</dt>
<dd> Commentary </dd> <dt>‘<samp class=”samp”>em</samp>’</dt>
<dd> Emergency </dd> <dt>‘<samp class=”samp”>vo</samp>’</dt>
<dd> Voice Over </dd> <dt>‘<samp class=”samp”>ka</samp>’</dt>
<dd> Karaoke </dd> </dl>

</dd> <dt><samp class=”option”>request_sample_fmt <var class=”var”>sample_fmt</var>
(decoding,audio)</samp></dt> <dd> Set
sample format audio decoders should prefer. Default value is <code
class=”code”>none</code>. </dd> <dt><samp class=”option”>pkt_timebase

<var class=”var”>rational number</var></samp></dt> <dt><samp class=”option”>sub_charenc <var class=”var”>encoding</var> (decoding,subtitles)</samp></dt> <dd> Set the input subtitles character encoding. </dd> <dt><samp class=”option”>field_order <var class=”var”>field_order</var> (video)</samp></dt> <dd> Set/override the field order of the video. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>progressive</samp>’</dt> <dd> Progressive video </dd> <dt>‘<samp class=”samp”>tt</samp>’</dt> <dd> Interlaced video, top field coded and displayed first </dd> <dt>‘<samp class=”samp”>bb</samp>’</dt> <dd> Interlaced video, bottom field coded and displayed first </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dd> Interlaced video, top coded first, bottom displayed first </dd> <dt>‘<samp class=”samp”>bt</samp>’</dt> <dd> Interlaced video, bottom coded first, top displayed first </dd> </dl>

</dd> <dt><samp class=”option”>skip_alpha <var class=”var”>bool</var> (decoding,video)</samp></dt> <dd> Set to 1 to disable processing alpha (transparency). This works like the ‘<samp class=”samp”>gray</samp>’ flag in the <samp class=”option”>flags</samp> option which skips chroma information instead of alpha. Default is 0. </dd> <dt><samp class=”option”>codec_whitelist <var class=”var”>list</var> (input)</samp></dt> <dd> "," separated list of allowed decoders. By default all are allowed. </dd> <dt><samp class=”option”>dump_separator <var class=”var”>string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class=”example-preformatted”>ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class=”option”>max_pixels <var class=”var”>integer</var> (decoding/encoding,video)</samp></dt> <dd> Maximum number of pixels per image. This value can be used to avoid out of memory failures due to large images. </dd> <dt><samp class=”option”>apply_cropping <var class=”var”>bool</var> (decoding,video)</samp></dt> <dd> Enable cropping if cropping parameters are multiples of the required alignment for the left and top parameters. If the alignment is not met the cropping will be partially applied to maintain alignment. Default is 1 (enabled). Note: The required alignment depends on if <code class=”code”>AV_CODEC_FLAG_UNALIGNED</code> is set and the CPU. <code class=”code”>AV_CODEC_FLAG_UNALIGNED</code> cannot be changed from the command line. Also hardware decoders will not apply left/top Cropping. </dd> <dt><samp class=”option”>lcevc-dec-

params <var class="var">string</var> (decoding,video)</samp></dt>
<dd> A :-separated list of key=value parameters passed to liblcevc-dec to initialize it. </dd> </dl>

 <h2 class="chapter">3 Decoders# TOC</h2>

Decoders are configured elements in FFmpeg which allow the decoding of multimedia streams. When you configure your FFmpeg build, all the supported native decoders are enabled by default. Decoders requiring an external library must be enabled manually via the corresponding <code class="code">--enable-lib</code> option. You can list all available decoders using the configure option <code class="code">--list-decoders</code>. You can disable all the decoders with the configure option <code class="code">--disable-decoders</code> and selectively enable / disable single decoders with the options <code class="code">--enable-decoder=<var class="var">DECODER</var></code> / <code class="code">--disable-decoder=<var class="var">DECODER</var></code>. The option <code class="code">-decoders</code> of the ff* tools will display the list of enabled decoders.

 <h2 class="chapter">4 Video Decoders# TOC</h2>

A description of some of the currently available video decoders follows. <h3 class="section">4.1 av1# TOC</h3>

AOMedia Video 1 (AV1) decoder. <h4 class="subsection">4.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">operating_point</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Default is 0. </dd> </dl>

 <h3 class="section">4.2 hevc# TOC</h3> HEVC (AKA ITU-T H.265 or ISO/IEC 23008-2) decoder. The decoder supports MV-HEVC multiview streams with at most two views. Views to be output are selected by supplying a list of view IDs to the decoder (the <samp class="option">view_ids</samp> option). This op-

tion may be set either statically before decoder init, or from the <code class=”code”>get_format()</code> callback - useful for the case when the view count or IDs change dynamically during decoding. Only the base layer is decoded by default. Note that if you are using the <code class=”code”>ffmpeg</code> CLI tool, you should be using view specifiers as documented in its manual, rather than the options documented here. <h4 class=”subsection”>4.2.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>view_ids (MV-HEVC)</samp></dt> <dd> Specify a list of view IDs that should be output. This option can also be set to a single ’-1’, which will cause all views defined in the VPS to be decoded and output. </dd> <dt><samp class=”option”>view_ids_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view IDs available in the active VPS. The array is empty for single-layer video. The value of this option is guaranteed to be accurate when read from the <code class=”code”>get_format()</code> callback. It may also be set at other times (e.g. after opening the decoder), but the value is informational only and may be incorrect (e.g. when the stream contains multiple distinct VPS NALUs). </dd> <dt><samp class=”option”>view_pos_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view positions (left, right, or unspecified) available in the active VPS, as <code class=”code”>AVStereo3DView</code> values. When the array is available, its elements apply to the corresponding elements of <samp class=”option”>view_ids_available</samp>, i.e. <code class=”code”>view_pos_available[i]</code> contains the position of view with ID <code class=”code”>view_ids_available[i]</code>. Same validity restrictions as for <samp class=”option”>view_ids_available</samp> apply to this option. </dd> </dl>

 <h3 class=”section”>4.3 rawvideo# TOC</h3>

Raw video decoder. This decoder decodes rawvideo streams. <h4 class=”subsection”>4.3.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>top <var class=”var”>top_field_first</var></samp></dt
<dd> Specify the assumed field type of the input video. <dl class=”table”> <dt><samp class=”option”>-1</samp></dt> <dd> the video is assumed to be progressive (default) </dd> <dt><samp

class=”option”>0</samp></dt> <dd> bottom-field-first is assumed </dd> <dt><samp class=”option”>1</samp></dt> <dd> top-field-first is assumed </dd> </dl>

</dd> </dl>

 <h3 class=”section”>4.4 libastcenc# TOC</h3>

ARM ASTC Encoder wrapper for decoding ASTC textures. This decoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, installed as <code class=”code”>astcenc/astcenc.h</code> next to a library that <code class=”code”>-lastcenc</code> finds. Pass <code class=”code”>--enable-version3</code> together with <code class=”code”>--enable-libastcenc</code> to configure. The <samp class=”file”>.astc</samp> header does not record the intended decoding profile. Individual blocks identify their endpoint encodings, but this does not establish the sRGB versus linear interpretation. The decoder therefore uses the container metadata or <samp class=”option”>dec_profile</samp>: a KTX sRGB texture is decoded as <code class=”code”>ldr-srgb</code>, while a KTX texture in the linear format is sampled with HDR precision (<code class=”code”>hdr-ldr-a</code>, half-float output) because the linear GL internal format says nothing about LDR. A raw <code class=”code”>.astc</code> file carries no profile at all, so it falls back to <code class=”code”>ldr-srgb</code>; use <samp class=”option”>dec_profile</samp> for a raw HDR image: <pre class=”example-preformatted”>ffmpeg -dec_profile hdr-ldr-a -i input.astc output.png </pre>

 <h4 class=”subsection”>4.4.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>dec_profile <var class=”var”>int</var></samp></dt> <dd> Color profile to decode with, overriding the container metadata. It must match the profile the texture was encoded with: <code class=”code”>astcenc</code> substitutes a fixed magenta error color for blocks whose endpoints do not fit the selected profile. The values are <code class=”code”>ldr</code>, <code class=”code”>ldr-srgb</code>, <code class=”code”>hdr-ldr-a</code> and <code class=”code”>hdr</code>. Without this option the profile published by the container is used; a raw <samp class=”file”>.astc</samp> stream, which publishes none, falls back to <code class=”code”>ldr-srgb</code>. </dd> </dl>

 <h3 class=”section”>4.5 libdav1d<a class=”anchor hidden-xs” href=”#libdav1d” aria-

hidden=”true”># TOC</h3>

dav1d AV1 decoder. libdav1d allows libavcodec to decode the AOMedia Video 1 (AV1) codec. Requires the presence of the libdav1d headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libdav1d</code>. <h4 class=”subsection”>4.5.1 Options# TOC</h4>

The following options are supported by the libdav1d wrapper. <dl class=”table”> <dt><samp class=”option”>max_frame_delay</samp></dt> <dd> Set max amount of frames the decoder may buffer internally. The default value is 0 (autodetect). </dd> <dt><samp class=”option”>filmgrain</samp></dt> <dd> Apply film grain to the decoded video if present in the bitstream. Defaults to the internal default of the library. This option is deprecated and will be removed in the future. See the global option <code class=”code”>export_side_data</code> to export Film Grain parameters instead of applying it. </dd> <dt><samp class=”option”>oppoint</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Defaults to the internal default of the library. </dd> <dt><samp class=”option”>alllayers</samp></dt> <dd> Output all spatial layers of a scalable AV1 bitstream. The default value is false. </dd> </dl>

 <h3 class=”section”>4.6 libdavs2# TOC</h3>

AVS2-P2/IEEE1857.4 video decoder wrapper. This decoder allows libavcodec to decode AVS2 streams with davs2 library.

 <h3 class=”section”>4.7 libuavs3d# TOC</h3>

AVS3-P2/IEEE1857.10 video decoder. libuavs3d allows libavcodec to decode AVS3 streams. Requires the presence of the libuavs3d headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libuavs3d</code>. <h4 class=”subsection”>4.7.1 Options# TOC</h4>

The following option is supported by the libuavs3d wrapper. <dl class="table"> <dt><samp class="option">frame_threads</samp></dt> <dd> Set amount of frame threads to use during decoding. The default value is 0 (autodetect). </dd> </dl>

 <h3 class="section">4.8 libxevd# TOC</h3>

eXtra-fast Essential Video Decoder (XEVD) MPEG-5 EVC decoder wrapper. This decoder requires the presence of the libxevd headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxevd</samp>. The xevd project website is at https://github.com/mpeg5/xevd. <h4 class="subsection">4.8.1 Options# TOC</h4>

The following options are supported by the libxevd wrapper. The xevd-equivalent options or values are listed in parentheses for easy migration. To get a more accurate and extensive documentation of the libxevd options, invoke the command <code class="code">xevd_app --help</code> or consult the libxevd documentation. <dl class="table"> <dt><samp class="option">threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class="section">4.9 QSV Decoders# TOC</h3>

The family of Intel QuickSync Video decoders (VC1, MPEG-2, H.264, HEVC, JPEG/MJPEG, VP8, VP9, AV1, VVC). <h4 class="subsection">4.9.1 Common Options# TOC</h4>

The following options are supported by all qsv decoders. <dl class="table"> <dt><samp class="option"><var class="var">async_depth</var></samp></dt> <dd> Internal parallelization depth, the higher the value the higher the latency. </dd> <dt><samp class="option"><var class="var">gpu_copy</var></samp></dt> <dd> A GPU-accelerated copy between video and system memory <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dt>‘<samp class="samp">on</samp>’</dt> <dt>‘<samp class="samp">off</samp>’</dt> </dl>

</dd> </dl>

 <h4 class="subsection">4.9.2 HEVC Options# TOC</h4> Extra options for hevc_qsv. <dl class="table"> <dt><samp class="option"><var class="var">load_plugin</var></samp></dt> <dd> A user plugin to load in an internal session <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">hevc_sw</samp>’<
<dt>‘<samp class="samp">hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugins</var></samp></dt>
<dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session </dd> </dl>

 <h3 class="section">4.10 v210#
TOC</h3>

Uncompressed 4:2:2 10-bit decoder.
<h4 class="subsection">4.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">custom_stride</samp></dt>
<dd> Set the line size of the v210 data in bytes. The default value is 0 (autodetect). You can use the special -1 value for a strideless v210 as seen in BOXX files. </dd> </dl>

 <h2 class="chapter">5 Audio Decoders# TOC</h2>

A description of some of the currently available audio decoders follows. <h3 class="section">5.1 ac3#
TOC</h3>

AC-3 audio decoder. This decoder implements part of ATSC A/52:2010 and ETSI TS 102 366, as well as the undocumented RealAudio 3 (a.k.a. dnet).
<h4 class="subsection">5.1.1 AC-3 Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-drc_scale <var class="var">value</var></samp></dt>
<dd> Dynamic Range Scale Factor. The factor to apply to dynamic range values from the AC-3 stream. This factor is applied exponentially. The default value is 1. There are 3 notable scale factor ranges: <dl

class=”table”> <dt><samp class=”option”>drc_scale == 0</samp></dt> <dd> DRC disabled. Produces full range audio. </dd> <dt><samp class=”option”>0 < drc_scale <= 1</samp></dt> <dd> DRC enabled. Applies a fraction of the stream DRC value. Audio reproduction is between full range and full compression. </dd> <dt><samp class=”option”>drc_scale > 1</samp></dt> <dd> DRC enabled. Applies drc_scale asymmetrically. Loud sounds are fully compressed. Soft sounds are enhanced. </dd> </dl>

</dd> </dl>

 <h3 class=”section”>5.2 dca# TOC</h3>

DCA (DTS Coherent Acoustics) audio decoder, including the DTS-HD extensions. <h4 class=”subsection”>5.2.1 DCA Decoder Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-downmix <var class=”var”>channel_layout</var></samp
<dd> Request a smaller channel layout from the decoder: ‘<samp class=”samp”>stereo</samp>’, ‘<samp class=”samp”>5.0(side)</samp>’, ‘<samp class=”samp”>5.1(side)</samp>’ or ‘<samp class=”samp”>7.1</samp>’. A DTS:X stream in the standard presentation decodes to 7.1.4 by default. Its four height channels are carried in an extension at the end of each lossless (XLL) frame, and the encoder also folds them into the 7.1 bed, so the decoder removes them from the bed when it outputs them. Requesting ‘<samp class=”samp”>7.1</samp>’ outputs the bed as coded instead, with the heights folded in, and so does every smaller layout. A ‘<samp class=”samp”>7.1</samp>’ request has no effect on other streams. </dd> </dl>

 <h3 class=”section”>5.3 flac# TOC</h3>

FLAC audio decoder. This decoder aims to implement the complete FLAC specification from Xiph. <h4 class=”subsection”>5.3.1 FLAC Decoder options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-use_buggy_lpc</samp></dt> <dd> The lavc FLAC encoder used to produce buggy streams with high

lpc values (like the default value). This option makes it possible to decode such streams correctly by using lavc’s old buggy lpc logic for decoding. </dd> </dl>

 <h3 class="section">5.4 ffwavesynth# TOC</h3>

Internal wave synthesizer. This decoder generates wave patterns according to predefined sequences. Its use is purely internal and the format of the data it accepts is not publicly documented. <h3 class="section">5.5 libgsm# TOC</h3>

libgsm decoder wrapper. libgsm allows libavcodec to decode the GSM full rate audio codec. Requires the presence of the libgsm headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libgsm</code>. This decoder supports both the ordinary GSM and the Microsoft variant. <h3 class="section">5.6 libilbc# TOC</h3>

libilbc decoder wrapper. libilbc allows libavcodec to decode the Internet Low Bitrate Codec (iLBC) audio codec. Requires the presence of the libilbc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libilbc</code>. <h4 class="subsection">5.6.1 Options# TOC</h4>

The following option is supported by the libilbc wrapper. <dl class="table"> <dt><samp class="option">enhance</samp></dt> <dd> Enable the enhancement of the decoded audio when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">5.7 libmpeghdec# TOC</h3>

libmpeghdec decoder wrapper. libmpeghdec allows libmpeghdec to decode the MPEG-H 3D audio codec. Requires the presence of the libmpeghdec headers and library during configuration. You need to

explicitly configure the build with <code class=”code”>--enable-libmpeghdec --enable-nonfree</code>. <h3 class=”section”>5.8 libopencore-amrnb# TOC</h3>

libopencore-amrnb decoder wrapper. libopencore-amrnb allows libavcodec to decode the Adaptive Multi-Rate Narrowband audio codec. Using it requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopencore-amrnb</code>. An FFmpeg native decoder for AMR-NB exists, so users can decode AMR-NB without this library. <h3 class=”section”>5.9 libopencore-amrwb# TOC</h

libopencore-amrwb decoder wrapper. libopencore-amrwb allows libavcodec to decode the Adaptive Multi-Rate Wideband audio codec. Using it requires the presence of the libopencore-amrwb headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopencore-amrwb</code>. An FFmpeg native decoder for AMR-WB exists, so users can decode AMR-WB without this library. <h3 class=”section”>5.10 libopus# TOC</h3>

libopus decoder wrapper. libopus allows libavcodec to decode the Opus Interactive Audio Codec. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopus</code>. An FFmpeg native decoder for Opus exists, so users can decode Opus without this library. <h3 class=”section”>5.11 pcm_dvda# TOC</h3>

PCM DVD-Audio decoder. Decodes the LPCM variant stored in DVD-Audio AOB files. Unlike DVD-Video LPCM it supports sample rates up to 192 kHz and allows splitting the channels into two groups with different quantization.

 <h2 class=”chapter”>6 Subtitles Decoders# TOC</h2>

 <h3 class="section">6.1 libaribb24# TOC</h3>

ARIB STD-B24 caption decoder. Implements profiles A and C of the ARIB STD-B24 standard. <h4 class="subsection">6.1.1 libaribb24 Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-aribb24-base-path <var class="var">path</var></samp></dt> <dd> Sets the base path for the libaribb24 library. This is utilized for reading of configuration files (for custom unicode conversions), and for dumping of non-text symbols as images under that location. Unset by default. </dd> <dt><samp class="option">-aribb24-skip-ruby-text <var class="var">boolean</var></samp></dt> <dd> Tells the decoder wrapper to skip text blocks that contain half-height ruby text. Enabled by default. </dd> </dl>

 <h3 class="section">6.2 libaribcaption# TOC</h3>

Yet another ARIB STD-B24 caption decoder using external libaribcaption library. Implements profiles A and C of the Japanese ARIB STD-B24 standard, Brazilian ABNT NBR 15606-1, and Philippines version of ISDB-T. Requires the presence of the libaribcaption headers and library (https://github.com/xqq/libaribcaption) during configuration. You need to explicitly configure the build with <code class="code">--enable-libaribcaption</code>. If both libaribb24 and libaribcaption are enabled, libaribcaption decoder precedes. <h4 class="subsection">6.2.1 libaribcaption Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-sub_type <var class="var">subtitle_type</var></samp>
<dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt>‘<samp class="samp">bitmap</samp>’</dt>
<dd> Graphical image. </dd> <dt>‘<samp class="samp">ass</samp>’</dt>
<dd> ASS formatted text. </dd> <dt>‘<samp class="samp">text</samp>’</dt>

<dd> Simple text based output without formatting. </dd> </dl>

The default is ass as same as libaribb24 decoder. Some present players (e.g., mpv) expect ASS format for ARIB caption. </dd> <dt><samp class=”option”>-caption_encoding <var class=”var”>encoding_scheme</var></samp></dt> <dd> Specifies the encoding scheme of input subtitle text. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatically detect text encoding (default). </dd> <dt>‘<samp class=”samp”>jis</samp>’</dt> <dd> 8bit-char JIS encoding defined in ARIB STD B24. This encoding used in Japan for ISDB captions. </dd> <dt>‘<samp class=”samp”>utf8</samp>’</dt> <dd> UTF-8 encoding defined in ARIB STD B24. This encoding is used in Philippines for ISDB-T captions. </dd> <dt>‘<samp class=”samp”>latin</samp>’</dt> <dd> Latin character encoding defined in ABNT NBR 15606-1. This encoding is used in South America for SBTVD / ISDB-Tb captions. </dd> </dl>

</dd> <dt><samp class=”option”>-font <var class=”var”>font_name[,font_name2,...]</var></samp></
<dd> Specify comma-separated list of font family names to be used for bitmap or ass type subtitle rendering. Only first font name is used for ass type subtitle. If not specified, use internally defined default font family. </dd> <dt><samp class=”option”>-ass_single_rect <var class=”var”>boolean</var></samp></dt> <dd> ARIB STD-B24 specifies that some captions may be displayed at different positions at a time (multi-rectangle subtitle). Since some players (e.g., old mpv) can’t handle multiple ASS rectangles in a single AVSubtitle, or multiple ASS rectangles of indeterminate duration with the same start timestamp, this option can change the behavior so that all the texts are displayed in a single ASS rectangle. The default is <var class=”var”>false</var>. If your player cannot handle AVSubtitles with multiple ASS rectangles properly, set this option to <var class=”var”>true</var> or define <code class=”env”>ASS_SINGLE_RECT=1</code> to change default behavior at compilation. </dd> <dt><samp class=”option”>-force_outline_text <var class=”var”>boolean</var></samp></dt> <dd> Specify whether always render outline text for all characters regardless of the indication by character style. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-outline_width <var class=”var”>number</var> (0.0 - 3.0)</samp></dt> <dd> Specify width for outline text, in dots (relative). The default is <var class=”var”>1.5</var>. </dd> <dt><samp class=”option”>-ignore_background <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore background color rendering. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-ignore_ruby <var class=”var”>boolean</var></samp></dt> <dd> Specify

whether to ignore rendering for ruby-like (furigana) characters. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-replace_drcs <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to render replaced DRCS characters as Unicode characters. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_ascii <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) fullwidth alphanumerics with halfwidth alphanumerics. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_japanese <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace some MSZ (Middle Size; half width) fullwidth japanese special characters with halfwidth ones. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_glyph <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) characters with halfwidth glyphs if the fonts supports it. This option works under FreeType or DirectWrite renderer with Adobe-Japan1 compliant fonts. e.g., IBM Plex Sans JP, Morisawa BIZ UDGothic, Morisawa BIZ UDMincho, Yu Gothic, Yu Mincho, and Meiryo. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-canvas_size <var class=”var”>image_size</var></samp></dt> <dd> Specify the resolution of the canvas to render subtitles to; usually, this should be frame size of input video. This only applies when <code class=”code”>-subtitle_type</code> is set to <var class=”var”>bitmap</var>. The libaribcaption decoder assumes input frame size for bitmap rendering as below: <ol class=”enumerate”> PROFILE_A : 1440 x 1080 with SAR (PAR) 4:3 PROFILE_C : 320 x 180 with SAR (PAR) 1:1

If actual frame size of input video does not match above assumption, the rendered captions may be distorted. To make the captions undistorted, add <code class=”code”>-canvas_size</code> option to specify actual input video size. Note that the <code class=”code”>-canvas_size</code> option is not required for video with different size but same aspect ratio. In such cases, the caption will be stretched or shrunk to actual video size if <code class=”code”>-canvas_size</code> option is not specified. If <code class=”code”>-canvas_size</code> option is specified with different size, the caption will be stretched or shrunk as specified size with calculated SAR. </dd> </dl>

 <h4 class=”subsection”>6.2.2 libaribcaption decoder usage examples# TOC</h4>

Display MPEG-TS file with ARIB subtitle by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap MPEG.TS </pre>

Display MPEG-TS file with input frame size 1920x1080 by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap -canvas_size 1920x1080 MPEG.TS </pre>

Embed ARIB subtitle in transcoded video: <pre class="example-preformatted">ffmpeg -sub_type bitmap -i src.m2t -filter_complex "[0:v][0:s]overlay" -vcodec h264 dest.mp4 </pre>

 <h3 class="section">6.3 dvbsub# TOC</h3>

 <h4 class="subsection">6.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">compute_clut</samp></dt> <dd><dl class="table"> <dt><samp class="option">-2</samp></dt> <dd> Compute clut once if no matching CLUT is in the stream. </dd> <dt><samp class="option">-1</samp></dt> <dd> Compute clut if no matching CLUT is in the stream. </dd> <dt><samp class="option">0</samp></dt> <dd> Never compute CLUT </dd> <dt><samp class="option">1</samp></dt> <dd> Always compute CLUT and override the one provided in the stream. </dd> </dl> </dd> <dt><samp class="option">dvb_substream</samp></dt> <dd> Selects the dvb substream, or all substreams if -1 which is default. </dd> </dl>

 <h3 class="section">6.4 dvdsub# TOC</h3>

This codec decodes the bitmap subtitles used in DVDs; the same subtitles can also be found in VobSub file pairs and in some Matroska files. <h4 class="subsection">6.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. When stored in VobSub, the palette is normally specified in the index file; in Matroska, the palette is stored in the codec extra-data in the same format as in VobSub.

In DVDs, the palette is stored in the IFO file, and therefore not available when reading from dumped VOB files. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">ifo_palette</samp></dt> <dd> Specify the IFO file from which the global palette is obtained. (experimental) </dd> <dt><samp class="option">forced_subs_only</samp></dt> <dd> Only decode subtitle entries marked as forced. Some titles have forced and non-forced subtitles in the same track. Setting this flag to <code class="code">1</code> will only keep the forced subtitles. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">6.5 libzvbi-teletext# TOC</h3>

Libzvbi allows libavcodec to decode DVB teletext pages and DVB teletext subtitles. Requires the presence of the libzvbi headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libzvbi</code>. <h4 class="subsection">6.5.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">txt_page</samp></dt> <dd> List of teletext page numbers to decode. Pages that do not match the specified list are dropped. You may use the special <code class="code">*</code> string to match all pages, or <code class="code">subtitle</code> to match all subtitle pages. Default value is *. </dd> <dt><samp class="option">txt_default_region</samp></dt> <dd> Set default character set used for decoding, a value between 0 and 87 (see ETS 300 706, Section 15, Table 32). Default value is -1, which does not override the libzvbi default. This option is needed for some legacy level 1.0 transmissions which cannot signal the proper charset. </dd> <dt><samp class="option">txt_chop_top</samp></dt> <dd> Discards the top teletext line. Default value is 1. </dd> <dt><samp class="option">txt_format</samp></dt> <dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt><samp class="option">bitmap</samp></dt> <dd> The default format, you should use this for teletext pages, because certain graphics and colors cannot be expressed in simple text or even ASS. </dd> <dt><samp class="option">text</samp></dt> <dd> Simple text based output without formatting. </dd> <dt><samp class="option">ass</samp></dt> <dd> Formatted ASS output, subtitle pages and teletext pages are re-

turned in different styles, subtitle pages are stripped down to text, but an effort is made to keep the text alignment and the formatting. </dd> </dl> </dd> <dt><samp class="option">txt_left</samp></dt> <dd> X offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_top</samp></dt> <dd> Y offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_chop_spaces</samp></dt> <dd> Chops leading and trailing spaces and removes empty lines from the generated text. This option is useful for teletext based subtitles where empty spaces may be present at the start or at the end of the lines or empty lines may be present between the subtitle lines because of double-sized teletext characters. Default value is 1. </dd> <dt><samp class="option">txt_duration</samp></dt> <dd> Sets the display duration of the decoded teletext pages or subtitles in milliseconds. Default value is -1 which means infinity or until the next subtitle event comes. </dd> <dt><samp class="option">txt_transparent</samp></dt> <dd> Force transparent background of the generated teletext bitmaps. Default value is 0 which means an opaque background. </dd> <dt><samp class="option">txt_opacity</samp></dt> <dd> Sets the opacity (0-255) of the teletext background. If <samp class="option">txt_transparent</samp> is not set, it only affects characters between a start box and an end box, typically subtitles. Default value is 0 if <samp class="option">txt_transparent</samp> is set, 255 otherwise. </dd> </dl>

 <h2 class="chapter">7 Encoders# TOC</h2>

Encoders are configured elements in FFmpeg which allow the encoding of multimedia streams. When you configure your FFmpeg build, all the supported native encoders are enabled by default. Encoders requiring an external library must be enabled manually via the corresponding <code class="code">--enable-lib</code> option. You can list all available encoders using the configure option <code class="code">--list-encoders</code>. You can disable all the encoders with the configure option <code class="code">--disable-encoders</code> and selectively enable / disable single encoders with the options <code class="code">--enable-encoder=<var class="var">ENCODER</var></code> / <code class="code">--disable-encoder=<var class="var">ENCODER</var></code>. The option <code class="code">-encoders</code> of the ff* tools will display the list of enabled encoders.

 <h2 class="chapter">8 Audio Encoders# TOC</h2>

A description of some of the currently available audio encoders follows. <h3 class="section">8.1 aac# TOC</h3>

Advanced Audio Coding (AAC) encoder. This encoder is the default AAC encoder, natively implemented into FFmpeg. <h4 class="subsection">8.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bit rate in bits/s. Setting this automatically activates constant bit rate (CBR) mode. If this option is unspecified it is set to 128kbps. </dd> <dt><samp class="option">q</samp></dt> <dd> Set quality for variable bit rate (VBR) mode. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. With the default ‘<samp class="samp">nmr</samp>’ coder, VBR holds a constant noise-to-mask target and lets the bitrate follow the content. Higher values are finer: <code class="code">1</code> lands near 128 kbps for typical stereo music, and each doubling of the value raises the rate by roughly a third; values above about 6 make no further difference. A quality setting takes precedence over <samp class="option">b</samp> and <samp class="option">aac_rc</samp>. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency. If unspecified will allow the encoder to dynamically adjust the cutoff to improve clarity on low bitrates. </dd> <dt><samp class="option">aac_coder</samp></dt> <dd> Set AAC encoder coding method. Possible values: <dl class="table"> <dt>‘<samp class="samp">twoloop</samp>’</dt> <dd> Two loop searching (TLS) method. This is the default method. This method first sets quantizers depending on band thresholds and then tries to find an optimal combination by adding or subtracting a specific value from all quantizers and adjusting some individual quantizer a little. Will tune itself based on whether <samp class="option">aac_is</samp>, <samp class="option">aac_ms</samp> and <samp class="option">aac_pns</samp> are enabled. </dd> <dt>‘<samp class="samp">anmr</samp>’</dt> <dd> Average noise to mask ratio (ANMR) trellis-based solution. This is an experimental coder which currently produces a lower quality, is more unstable and is slower than the default twoloop coder but has potential. Currently has no support for the <samp class="option">aac_is</samp> or <samp class="option">aac_pns</samp> options. Not currently recommended. </dd> <dt>‘<samp class="samp">fast</samp>’</dt> <dd> Constant quantizer method. Uses a cheaper version of

twoloop algorithm that doesn’t try to do as many clever adjustments. Worse with low bitrates (less than 64kbps), but is better and much faster at higher bitrates. </dd> </dl>

</dd> <dt><samp class="option">aac_rc</samp></dt> <dd> Rate-control mode of the ‘<samp class="samp">nmr</samp>’ coder when a bitrate is set with <samp class="option">b</samp> (other coders ignore it; <samp class="option">q</samp> takes precedence). Possible values: <dl class="table"> <dt>‘<samp class="samp">cbr</samp>’</dt> <dd> Constant bitrate within the decoder’s bit reservoir. This is the default. </dd> <dt>‘<samp class="samp">abr</samp>’</dt> <dd> Average bitrate: a constant-quality target, as in VBR, whose set-point a slow servo moves to meet <samp class="option">b</samp> over the length of the stream. Local quality stays constant, so short files and individual passages can deviate from the requested rate; the average converges over minutes rather than seconds. Below roughly 56 kbps stereo the rate lands above the request; use ‘<samp class="samp">cbr</samp>’ there. </dd> </dl>

</dd> <dt><samp class="option">aac_ms</samp></dt> <dd> Sets mid/side coding mode. The default value of "auto" will automatically use M/S with bands which will benefit from such coding. Can be forced for all bands using the value "enable", which is mainly useful for debugging or disabled using "disable". </dd> <dt><samp class="option">aac_is</samp></dt> <dd> Sets intensity stereo coding tool usage. By default, it’s enabled and will automatically toggle IS for similar pairs of stereo bands if it’s beneficial. Can be disabled for debugging by setting the value to "disable". </dd> <dt><samp class="option">aac_pns</samp></dt> <dd> Uses perceptual noise substitution to replace low entropy high frequency bands with imperceptible white noise during the decoding process. By default, it’s enabled, but can be disabled for debugging purposes by using "disable". </dd> <dt><samp class="option">aac_tns</samp></dt> <dd> Enables the use of a multitap FIR filter which spans through the high frequency bands to hide quantization noise during the encoding process and is reverted by the decoder. As well as decreasing unpleasant artifacts in the high range this also reduces the entropy in the high bands and allows for more bits to be used by the mid-low bands. By default it’s enabled but can be disabled for debugging by setting the option to "disable". </dd> <dt><samp class="option">aac_ltp</samp></dt> <dd> Enables the use of the long term prediction extension which increases coding efficiency in very low bandwidth situations such as encoding of voice or solo piano music by extending constant harmonic peaks in bands throughout frames. This option is implied by profile:a aac_low. Use in conjunction with <samp class="option">-ar</samp> to decrease the samplerate. </dd> <dt><samp class="option">profile</samp></dt> <dd> Sets the en-

coding profile, possible values: <dl class="table"> <dt>‘<samp class="samp">aac_low</samp>’</dt> <dd> The default, AAC "Low-complexity" profile. Is the most compatible and produces decent quality. </dd> <dt>‘<samp class="samp">mpeg2_aac_low</samp>’</dt> <dd> Equivalent to <code class="code">-profile:a aac_low -aac_pns 0</code>. PNS was introduced with the MPEG4 specifications. </dd> <dt>‘<samp class="samp">aac_ltp</samp>’</dt> <dd> Long term prediction profile, is enabled by and will enable the <samp class="option">aac_ltp</samp> option. Introduced in MPEG4. </dd> </dl> If this option is unspecified it is set to ‘<samp class="samp">aac_low</samp>’. </dd> </dl>

 <h3 class="section">8.2 ac3 and ac3_fixed# TOC</h3>

AC-3 audio encoders. These encoders implement part of ATSC A/52:2010 and ETSI TS 102 366. The <var class="var">ac3</var> encoder uses floating-point math, while the <var class="var">ac3_fixed</var> encoder only uses fixed-point integer math. This does not mean that one is always faster, just that one or the other may be better suited to a particular system. The <var class="var">ac3_fixed</var> encoder is not the default codec for any of the output formats, so it must be specified explicitly using the option <code class="code">-acodec ac3_fixed</code> in order to use it. <h4 class="subsection">8.2.1 AC-3 Metadata# TOC</h4>

The AC-3 metadata options are used to set parameters that describe the audio, but in most cases do not affect the audio encoding itself. Some of the options do directly affect or influence the decoding and playback of the resulting bitstream, while others are just for informational purposes. A few of the options will add bits to the output stream that could otherwise be used for audio data, and will thus affect the quality of the output. Those will be indicated accordingly with a note in the option list below. These parameters are described in detail in several publicly-available documents. <ul class="itemize mark-bullet"> A/52:2010 - Digital Audio Compression (AC-3) (E-AC-3) Standard A/54 - Guide to the Use of the ATSC Digital Television Standard <a class="uref" href="http://www.dolby.com/uploadedFiles/zz-_Shared_Assets/English_PDFs/Professional/18 Metadata Guide Dolby Digital Professional Encoding Guidelines

 <h4 class="subsubsection">8.2.1.1 Metadata Control Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-per_frame_metadata <var class="var">boolean</var></samp></dt> <dd> Allow Per-Frame Metadata. Specifies if the encoder should check for changing metadata for each frame. <dl class="table"> <dt><samp class="option">0</samp></dt> <dd> The metadata values set at initialization will be used for every frame in the stream. (default) </dd> <dt><samp class="option">1</samp></dt> <dd> Metadata values can be changed before encoding each frame. </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">8.2.1.2 Downmix Levels# TOC</h4>

<dl class="table"> <dt><samp class="option">-center_mixlev <var class="var">level</var></samp></dt> <dd> Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo. This field will only be written to the bitstream if a center channel is present. The value is specified as a scale factor. There are 3 valid values: <dl class="table"> <dt><samp class="option">0.707</samp></dt> <dd> Apply -3dB gain </dd> <dt><samp class="option">0.595</samp></dt> <dd> Apply -4.5dB gain (default) </dd> <dt><samp class="option">0.500</samp></dt> <dd> Apply -6dB gain </dd> </dl>

</dd> <dt><samp class="option">-surround_mixlev <var class="var">level</var></samp></dt> <dd> Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo. This field will only be written to the bitstream if one or more surround channels are present. The value is specified as a scale factor. There are 3 valid values: <dl class="table"> <dt><samp class="option">0.707</samp></dt> <dd> Apply -3dB gain </dd> <dt><samp class="option">0.500</samp></dt> <dd> Apply -6dB gain (default) </dd> <dt><samp class="option">0.000</samp></dt> <dd> Silence Surround Channel(s) </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">8.2.1.3 Audio Production Information# <a class="anchor hidden-xs"href="#toc-Audio-Production-Information"

aria-hidden=”true”>TOC</h4> Audio Production Information is optional information describing the mixing environment. Either none or both of the fields are written to the bitstream. <dl class=”table”> <dt><samp class=”option”>-mixing_level <var class=”var”>number</var></samp></dt> <dd> Mixing Level. Specifies peak sound pressure level (SPL) in the production environment when the mix was mastered. Valid values are 80 to 111, or -1 for unknown or not indicated. The default value is -1, but that value cannot be used if the Audio Production Information is written to the bitstream. Therefore, if the <code class=”code”>room_type</code> option is not the default value, the <code class=”code”>mixing_level</code> option must not be -1. </dd> <dt><samp class=”option”>-room_type <var class=”var”>type</var></samp></dt> <dd> Room Type. Describes the equalization used during the final mixing session at the studio or on the dubbing stage. A large room is a dubbing stage with the industry standard X-curve equalization; a small room has flat equalization. This field will not be written to the bitstream if both the <code class=”code”>mixing_level</code> option and the <code class=”code”>room_type</code> option have the default values. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dt><samp class=”option”>notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class=”option”>1</samp></dt> <dt><samp class=”option”>large</samp></dt> <dd> Large Room </dd> <dt><samp class=”option”>2</samp></dt> <dt><samp class=”option”>small</samp></dt> <dd> Small Room </dd> </dl>

</dd> </dl>

 <h4 class=”subsubsection”>8.2.1.4 Other Metadata Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-copyright <var class=”var”>boolean</var></samp></dt
<dd> Copyright Indicator. Specifies whether a copyright exists for this audio. <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dt><samp class=”option”>off</samp></dt> <dd> No Copyright Exists (default) </dd> <dt><samp class=”option”>1</samp></dt> <dt><samp class=”option”>on</samp></dt> <dd> Copyright Exists </dd> </dl>

</dd> <dt><samp class=”option”>-dialnorm <var class=”var”>value</var></samp></dt> <dd> Dialogue Normalization. Indicates how far the average dialogue level of the program is below digital 100% full scale (0 dBFS). This parameter determines a level shift during audio reproduction that sets the average volume of the dialogue to a preset level. The goal is to match volume level between program sources. A value of -31dB will result in no volume level change, relative to

the source volume, during audio reproduction. Valid values are whole numbers in the range -31 to -1, with -31 being the default. </dd> <dt><samp class="option">-dsur_mode <var class="var">mode</var></samp></dt> <dd> Dolby Surround Mode. Specifies whether the stereo signal uses Dolby Surround (Pro Logic). This field will only be written to the bitstream if the audio stream is stereo. Using this option does NOT mean the encoder will actually apply Dolby Surround processing. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Not Dolby Surround Encoded </dd> <dt><samp class="option">2</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Dolby Surround Encoded </dd> </dl>

</dd> <dt><samp class="option">-original <var class="var">boolean</var></samp></dt> <dd> Original Bit Stream Indicator. Specifies whether this audio is from the original source and not a copy. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Not Original Source </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Original Source (default) </dd> </dl>

</dd> </dl>

 <h4 class="subsection">8.2.2 Extended Bitstream Information# TOC</h4> The extended bitstream options are part of the Alternate Bit Stream Syntax as specified in Annex D of the A/52:2010 standard. It is grouped into 2 parts. If any one parameter in a group is specified, all values in that group will be written to the bitstream. Default values are used for those that are written but have not been specified. If the mixing levels are written, the decoder will use these values instead of the ones specified in the <code class="code">center_mixlev</code> and <code class="code">surround_mixlev</code> options if it supports the Alternate Bit Stream Syntax. <h4 class="subsubsection">8.2.2.1 Extended Bitstream Information - Part 1# TOC</h4>

<dl class="table"> <dt><samp class="option">-dmix_mode <var class="var">mode</var></samp></dt> <dd> Preferred Stereo Downmix Mode. Allows the user to select either Lt/Rt (Dolby Surround) or Lo/Ro (normal stereo) as the preferred stereo downmix mode.

<dl class="table"> <dt><samp class="option">0</samp></dt>
<dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt>
<dt><samp class="option">ltrt</samp></dt> <dd> Lt/Rt Downmix Preferred </dd> <dt><samp class="option">2</samp></dt>
<dt><samp class="option">loro</samp></dt> <dd> Lo/Ro Downmix Preferred </dd> </dl>

</dd> <dt><samp class="option">-ltrt_cmixlev <var class="var">level</var></samp></dt>
<dd> Lt/Rt Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo in Lt/Rt mode.
<dl class="table"> <dt><samp class="option">1.414</samp></dt>
<dd> Apply +3dB gain </dd> <dt><samp class="option">1.189</samp></dt>
<dd> Apply +1.5dB gain </dd> <dt><samp class="option">1.000</samp></dt>
<dd> Apply 0dB gain </dd> <dt><samp class="option">0.841</samp></dt>
<dd> Apply -1.5dB gain </dd> <dt><samp class="option">0.707</samp></dt>
<dd> Apply -3.0dB gain </dd> <dt><samp class="option">0.595</samp></dt>
<dd> Apply -4.5dB gain (default) </dd> <dt><samp class="option">0.500</samp></dt>
<dd> Apply -6.0dB gain </dd> <dt><samp class="option">0.000</samp></dt>
<dd> Silence Center Channel </dd> </dl>

</dd> <dt><samp class="option">-ltrt_surmixlev <var class="var">level</var></samp></dt>
<dd> Lt/Rt Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo in Lt/Rt mode.
<dl class="table"> <dt><samp class="option">0.841</samp></dt>
<dd> Apply -1.5dB gain </dd> <dt><samp class="option">0.707</samp></dt>
<dd> Apply -3.0dB gain </dd> <dt><samp class="option">0.595</samp></dt>
<dd> Apply -4.5dB gain </dd> <dt><samp class="option">0.500</samp></dt>
<dd> Apply -6.0dB gain (default) </dd> <dt><samp class="option">0.000</samp></dt>
<dd> Silence Surround Channel(s) </dd> </dl>

</dd> <dt><samp class="option">-loro_cmixlev <var class="var">level</var></samp></dt>
<dd> Lo/Ro Center Mix Level. The amount of gain the decoder should apply to the center channel when downmixing to stereo in Lo/Ro mode.
<dl class="table"> <dt><samp class="option">1.414</samp></dt>
<dd> Apply +3dB gain </dd> <dt><samp class="option">1.189</samp></dt>
<dd> Apply +1.5dB gain </dd> <dt><samp class="option">1.000</samp></dt>
<dd> Apply 0dB gain </dd> <dt><samp class="option">0.841</samp></dt>
<dd> Apply -1.5dB gain </dd> <dt><samp class="option">0.707</samp></dt>
<dd> Apply -3.0dB gain </dd> <dt><samp class="option">0.595</samp></dt>
<dd> Apply -4.5dB gain (default) </dd> <dt><samp class="option">0.500</samp></dt>
<dd> Apply -6.0dB gain </dd> <dt><samp class="option">0.000</samp></dt>
<dd> Silence Center Channel </dd> </dl>

</dd> <dt><samp class="option">-loro_surmixlev <var class="var">level</var></samp></dt>
<dd> Lo/Ro Surround Mix Level. The amount of gain the decoder should apply to the surround channel(s) when downmixing to stereo in Lo/Ro mode.
<dl class="table"> <dt><samp class="option">0.841</samp></dt>

<dd> Apply -1.5dB gain </dd> <dt><samp class="option">0.707</samp></dt>
<dd> Apply -3.0dB gain </dd> <dt><samp class="option">0.595</samp></dt>
<dd> Apply -4.5dB gain </dd> <dt><samp class="option">0.500</samp></dt>
<dd> Apply -6.0dB gain (default) </dd> <dt><samp class="option">0.000</samp></dt>
<dd> Silence Surround Channel(s) </dd> </dl>

</dd> </dl>

 <h4 class="subsubsection">8.2.2.2 Extended Bitstream Information - Part 2# TOC</h4>

<dl class="table"> <dt><samp class="option">-dsurex_mode <var class="var">mode</var></samp></dt> <dd> Dolby Surround EX Mode. Indicates whether the stream uses Dolby Surround EX (7.1 matrixed to 5.1). Using this option does NOT mean the encoder will actually apply Dolby Surround EX processing. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Dolby Surround EX Off </dd> <dt><samp class="option">2</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Dolby Surround EX On </dd> </dl>

</dd> <dt><samp class="option">-dheadphone_mode <var class="var">mode</var></samp></dt>
<dd> Dolby Headphone Mode. Indicates whether the stream uses Dolby Headphone encoding (multi-channel matrixed to 2.0 for use with headphones). Using this option does NOT mean the encoder will actually apply Dolby Headphone processing. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">notindicated</samp></dt> <dd> Not Indicated (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Dolby Headphone Off </dd> <dt><samp class="option">2</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Dolby Headphone On </dd> </dl>

</dd> <dt><samp class="option">-ad_conv_type <var class="var">type</var></samp></dt>
<dd> A/D Converter Type. Indicates whether the audio has passed through HDCD A/D conversion. <dl class="table"> <dt><samp class="option">0</samp></dt> <dt><samp class="option">standard</samp></dt>
<dd> Standard A/D Converter (default) </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">hdcd</samp></dt>
<dd> HDCD A/D Converter </dd> </dl>

</dd> </dl>

 <h4 class="subsection">8.2.3 Other AC-3 Encoding Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-stereo_rematrixing <var class="var">boolean</var></samp></dt> <dd> Stereo Rematrixing. Enables/Disables use of rematrixing for stereo input. This is an optional AC-3 feature that increases quality by selectively encoding the left/right channels as mid/side. This option is enabled by default, and it is highly recommended that it be left as enabled except for testing purposes. </dd> <dt><samp class="option">-cutoff <var class="var">frequency</var></samp></dt> <dd> Set lowpass cutoff frequency. If unspecified, the encoder selects a default determined by various other encoding parameters. </dd> </dl>

 <h4 class="subsection">8.2.4 Floating-Point-Only AC-3 Encoding Options# TOC</h4>

These options are only valid for the floating-point encoder and do not exist for the fixed-point encoder due to the corresponding features not being implemented in fixed-point. <dl class="table"> <dt><samp class="option">-channel_coupling <var class="var">boolean</var></samp></dt> <dd> Enables/Disables use of channel coupling, which is an optional AC-3 feature that increases quality by combining high frequency information from multiple channels into a single channel. The per-channel high frequency information is sent with less accuracy in both the frequency and time domains. This allows more bits to be used for lower frequencies while preserving enough information to reconstruct the high frequencies. This option is enabled by default for the floating-point encoder and should generally be left as enabled except for testing purposes or to increase encoding speed. <dl class="table"> <dt><samp class="option">-1</samp></dt> <dt><samp class="option">auto</samp></dt> <dd> Selected by Encoder (default) </dd> <dt><samp class="option">0</samp></dt> <dt><samp class="option">off</samp></dt> <dd> Disable Channel Coupling </dd> <dt><samp class="option">1</samp></dt> <dt><samp class="option">on</samp></dt> <dd> Enable Channel Coupling </dd> </dl>

</dd> <dt><samp class="option">-cpl_start_band <var class="var">number</var></samp></dt> <dd> Coupling Start Band. Sets the channel coupling start band, from

1 to 15. If a value higher than the bandwidth is used, it will be reduced to 1 less than the coupling end band. If <var class="var">auto</var> is used, the start band will be determined by the encoder based on the bit rate, sample rate, and channel layout. This option has no effect if channel coupling is disabled. <dl class="table"> <dt><samp class="option">-1</samp></dt> <dt><samp class="option">auto</samp></dt> <dd> Selected by Encoder (default) </dd> </dl>

</dd> </dl>

 <h3 class="section">8.3 flac# TOC</h3>

FLAC (Free Lossless Audio Codec) Encoder <h4 class="subsection">8.3.1 Options# TOC</h4>

The following options are supported by FFmpeg’s flac encoder. <dl class="table"> <dt><samp class="option">compression_level</samp></dt> <dd> Sets the compression level, which chooses defaults for many other options if they are not set explicitly. Valid values are from 0 to 12, 5 is the default. </dd> <dt><samp class="option">frame_size</samp></dt> <dd> Sets the size of the frames in samples per channel. </dd> <dt><samp class="option">lpc_coeff_precision</samp></dt> <dd> Sets the LPC coefficient precision, valid values are from 1 to 15, 15 is the default. </dd> <dt><samp class="option">lpc_type</samp></dt> <dd> Sets the first stage LPC algorithm <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> LPC is not used </dd> <dt>‘<samp class="samp">fixed</samp>’</dt> <dd> fixed LPC coefficients </dd> <dt>‘<samp class="samp">levinson</samp>’</dt> <dt>‘<samp class="samp">cholesky</samp>’</dt> </dl>

</dd> <dt><samp class="option">lpc_passes</samp></dt> <dd> Number of passes to use for Cholesky factorization during LPC analysis </dd> <dt><samp class="option">min_partition_order</samp></dt> <dd> The minimum partition order </dd> <dt><samp class="option">max_partition_order</samp></dt> <dd> The maximum partition order </dd> <dt><samp class="option">prediction_order_method</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">estimation</samp>’</dt> <dt>‘<samp class="samp">2level</samp>’</dt> <dt>‘<samp class="samp">4level</samp>’</dt> <dt>‘<samp class="samp">8level</samp>’</dt> <dt>‘<samp class="samp">search</samp>’</dt> <dd> Bruteforce search </dd> <dt>‘<samp class="samp">log</samp>’</dt> </dl>

</dd> <dt><samp class="option">ch_mode</samp></dt> <dd> Channel mode <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The mode is chosen automatically for each frame </dd> <dt>‘<samp class="samp">indep</samp>’</dt> <dd> Channels are independently coded </dd> <dt>‘<samp class="samp">left_side</samp>’</dt> <dt>‘<samp class="samp">right_side</samp>’</dt> <dt>‘<samp class="samp">mid_side</samp>’</dt> </dl>

</dd> <dt><samp class="option">exact_rice_parameters</samp></dt> <dd> Chooses if rice parameters are calculated exactly or approximately. if set to 1 then they are chosen exactly, which slows the code down slightly and improves compression slightly. </dd> <dt><samp class="option">multi_dim_quant</samp></dt> <dd> Multi Dimensional Quantization. If set to 1 then a 2nd stage LPC algorithm is applied after the first stage to finetune the coefficients. This is quite slow and slightly improves compression. </dd> </dl>

 <h3 class="section">8.4 opus# TOC</h3>

Opus encoder. This is a native FFmpeg encoder for the Opus format. Currently, it’s in development and only implements the CELT part of the codec. Its quality is usually worse and at best is equal to the libopus encoder. <h4 class="subsection">8.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bit rate in bits/s. If unspecified it uses the number of channels and the layout to make a good guess. </dd> <dt><samp class="option">opus_delay</samp></dt> <dd> Sets the maximum delay in milliseconds. Lower delays than 20ms will very quickly decrease quality. </dd> </dl>

 <h3 class="section">8.5 libfdk_aac# TOC</h3>

libfdk-aac AAC (Advanced Audio Coding) encoder wrapper. The libfdk-aac library is based on the Fraunhofer FDK AAC code from the Android project. Requires the presence of the libfdk-aac headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libfdk-aac</code>. The library is also incompatible with GPL, so if you allow the use of GPL, you should configure with <code class="code">--enable-gpl --enable-nonfree --enable-libfdk-aac</code>. This encoder has support for

the AAC-HE profiles. VBR encoding, enabled through the <samp class="option">vbr</samp> or <samp class="option">flags +qscale</samp> options, is experimental and only works with some combinations of parameters. Support for encoding 7.1 audio is only available with libfdk-aac 0.1.3 or higher. For more information see the fdk-aac project at http://sourceforge.net/p/opencore-amr/fdk-aac/. <h4 class="subsection">8.5.1 Options# TOC</h4>

The following options are mapped on the shared FFmpeg codec options. <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bit rate in bits/s. If the bitrate is not explicitly specified, it is automatically set to a suitable value depending on the selected profile. In case VBR mode is enabled the option is ignored. </dd> <dt><samp class="option">ar</samp></dt> <dd> Set audio sampling rate (in Hz). </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of audio channels. </dd> <dt><samp class="option">flags +qscale</samp></dt> <dd> Enable fixed quality, VBR (Variable Bit Rate) mode. Note that VBR is implicitly enabled when the <samp class="option">vbr</samp> value is positive. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency. If not specified (or explicitly set to 0) it will use a value automatically computed by the library. Default value is 0. </dd> <dt><samp class="option">profile</samp></dt> <dd> Set audio profile. The following profiles are recognized: <dl class="table"> <dt>‘<samp class="samp">aac_low</samp>’</dt> <dd> Low Complexity AAC (LC) </dd> <dt>‘<samp class="samp">aac_he</samp>’</dt> <dd> High Efficiency AAC (HE-AAC) </dd> <dt>‘<samp class="samp">aac_he_v2</samp>’</dt> <dd> High Efficiency AAC version 2 (HE-AACv2) </dd> <dt>‘<samp class="samp">aac_ld</samp>’</dt> <dd> Low Delay AAC (LD) </dd> <dt>‘<samp class="samp">aac_eld</samp>’</dt> <dd> Enhanced Low Delay AAC (ELD) </dd> </dl>

If not specified it is set to ‘<samp class="samp">aac_low</samp>’. </dd> </dl>

The following are private options of the libfdk_aac encoder. <dl class="table"> <dt><samp class="option">afterburner</samp></dt> <dd> Enable afterburner feature if set to 1, disabled if set to 0. This improves the quality but also the required processing power. Default value is 1. </dd> <dt><samp class="option">eld_sbr</samp></dt> <dd> Enable SBR (Spectral Band Replication) for ELD if set to 1, disabled if set to 0. Default value is 0. </dd> <dt><samp

class=”option”>eld_v2</samp></dt> <dd> Enable ELDv2 (LD-MPS extension for ELD stereo signals) for ELDv2 if set to 1, disabled if set to 0. Note that option is available when fdk-aac version (AACENCODER_LIB_VL0.AACENCODER_LIB_VL1.AACENCODER_LIB_VL2) > (4.0.0). Default value is 0. </dd> <dt><samp class=”option”>signaling</samp></dt> <dd> Set SBR/PS signaling style. It can assume one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt> <dd> choose signaling implicitly (explicit hierarchical by default, implicit if global header is disabled) </dd> <dt>‘<samp class=”samp”>implicit</samp>’</dt> <dd> implicit backwards compatible signaling </dd> <dt>‘<samp class=”samp”>explicit_sbr</samp>’</dt> <dd> explicit SBR, implicit PS signaling </dd> <dt>‘<samp class=”samp”>explicit_hierarchical</samp>’</dt> <dd> explicit hierarchical signaling </dd> </dl>

Default value is ‘<samp class=”samp”>default</samp>’. </dd> <dt><samp class=”option”>latm</samp></dt> <dd> Output LATM/LOAS encapsulated data if set to 1, disabled if set to 0. Default value is 0. </dd> <dt><samp class=”option”>header_period</samp></dt> <dd> Set StreamMuxConfig and PCE repetition period (in frames) for sending in-band configuration buffers within LATM/LOAS transport layer. Must be a 16-bits non-negative integer. Default value is 0. </dd> <dt><samp class=”option”>vbr</samp></dt> <dd> Set VBR mode, from 1 to 5. 1 is lowest quality (though still pretty good) and 5 is highest quality. A value of 0 will disable VBR, and CBR (Constant Bit Rate) is enabled. Currently only the ‘<samp class=”samp”>aac_low</samp>’ profile supports VBR encoding. VBR modes 1-5 correspond to roughly the following average bit rates: <dl class=”table”> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> 32 kbps/channel </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> 40 kbps/channel </dd> <dt>‘<samp class=”samp”>3</samp>’</dt> <dd> 48-56 kbps/channel </dd> <dt>‘<samp class=”samp”>4</samp>’</dt> <dd> 64 kbps/channel </dd> <dt>‘<samp class=”samp”>5</samp>’</dt> <dd> about 80-96 kbps/channel </dd> </dl>

Default value is 0. </dd> <dt><samp class=”option”>frame_length</samp></dt> <dd> Set the audio frame length in samples. Default value is the internal default of the library. Refer to the library’s documentation for information about supported values. </dd> </dl>

 <h4 class=”subsection”>8.5.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code>

to convert an audio file to VBR AAC in an M4A (MP4) container: <pre class=”example-preformatted”>ffmpeg -i input.wav -codec:a libfdk_aac -vbr 3 output.m4a </pre>

Use <code class=”command”>ffmpeg</code> to convert an audio file to CBR 64k kbps AAC, using the High-Efficiency AAC profile: <pre class=”example-preformatted”>ffmpeg -i input.wav -c:a libfdk_aac -profile:a aac_he -b:a 64k output.m4a </pre>

 <h3 class=”section”>8.6 libfdk-xhe-aac# TOC</h3>

libfdk-xhe-aac (Extended High-Efficiency Advanced Audio Coding) encoder wrapper. The libfdk-xhe-aac library is based on the Fraunhofer FDK Extended High Efficiency AAC Encoder Software code from the Android project. Requires the presence of the libfdk-xhe-aac headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libfdk-xhe-aac</code>. The library is also incompatible with GPL, so if you allow the use of GPL, you should configure with <code class=”code”>--enable-gpl --enable-nonfree --enable-libfdk-xhe-aac</code>. <h4 class=”subsection”>8.6.1 Options# TOC</h4>

The following are private options of the libfdk_xhe_aac encoder. <dl class=”table”> <dt><samp class=”option”>vbr</samp></dt> <dd> Set VBR mode, from 0 to 6. 0 is lowest quality and 6 is highest quality. A value of -1 will disable VBR, and CBR (Constant Bit Rate) is enabled. If not set, the default is CBR. </dd> <dt><samp class=”option”>live_loudness_level</samp></dt> <dd> Set the expected loudness level of the stream, between -31 and -10 dB LUFS. Defaults to -24 dB LUFS if not set. </dd> <dt><samp class=”option”>rap_length</samp></dt> <dd> The duration between encoded Random Access Points in the bitstream. </dd> <dt><samp class=”option”>stream_id</samp></dt> <dd> A unique ID of the encoded stream within an adaptation set. The ID is embedded into the encoded bitstream. If not set, the encoder tries to generate a hopefully unique ID for the stream. </dd> </dl>

 <h3 class=”section”>8.7 liblc3# TOC</h3>

liblc3 LC3 (Low Complexity Communication Codec) encoder wrap-

per. Requires the presence of the liblc3 headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-liblc3</code>. This encoder has support for the Bluetooth SIG LC3 codec for the LE Audio protocol, and the following features of LC3plus: <ul class="itemize mark-bullet"> Frame duration of 2.5 and 5ms. High-Resolution mode, 48 KHz, and 96 kHz sampling rates.

For more information see the liblc3 project at <a class="url" href="https://github.com/google/liblc3">htt
  <h4 class="subsection">8.7.1 Options# TOC</h4>

The following options are mapped on the shared FFmpeg codec options. <dl class="table"> <dt><samp class="option">b <var class="var">bitrate</var></samp></dt> <dd> Set the bit rate in bits/s. This will determine the fixed size of the encoded frames, for a selected frame duration. </dd> <dt><samp class="option">ar <var class="var">frequency</var></samp></dt> <dd> Set the audio sampling rate (in Hz). </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of audio channels. </dd> <dt><samp class="option">frame_duration</samp></dt> <dd> Set the audio frame duration in milliseconds. Default value is 10ms. Allowed frame durations are 2.5ms, 5ms, 7.5ms and 10ms. LC3 (Bluetooth LE Audio), allows 7.5ms and 10ms; and LC3plus 2.5ms, 5ms and 10ms. The 10ms frame duration is available in LC3 and LC3 plus standard. In this mode, the produced bitstream can be referenced either as LC3 or LC3plus. </dd> <dt><samp class="option">high_resolution <var class="var">boolean</var></samp></dt> <dd> Enable the high-resolution mode if set to 1. The high-resolution mode is available with all LC3plus frame durations and for a sampling rate of 48 KHz, and 96 KHz. The encoder automatically turns off this mode at lower sampling rates and activates it at 96 KHz. This mode should be preferred at high bitrates. In this mode, the audio bandwidth is always up to the Nyquist frequency, compared to LC3 at 48 KHz, which limits the bandwidth to 20 KHz. </dd> </dl>

<h3 class="section">8.8 libmp3lame# TOC</h3>

LAME (Lame Ain’t an MP3 Encoder) MP3 encoder wrapper. Requires the presence of the libmp3lame headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libmp3lame</code>. See libshine for a fixed-point MP3 encoder,

although with a lower quality. <h4 class="subsection">8.8.1 Options# TOC</h4>

The following options are supported by the libmp3lame wrapper. The <code class="command">lame</code>-equivalent of the options are listed in parentheses. <dl class="table"> <dt><samp class="option">b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR or ABR. LAME <code class="code">bitrate</code> is expressed in kilobits/s. </dd> <dt><samp class="option">q (-V)</samp></dt> <dd> Set constant quality setting for VBR. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> <dt><samp class="option">compression_level (-q)</samp></dt> <dd> Set algorithm quality. Valid arguments are integers in the 0-9 range, with 0 meaning highest quality but slowest, and 9 meaning fastest while producing the worst quality. </dd> <dt><samp class="option">cutoff (--lowpass)</samp></dt> <dd> Set lowpass cutoff frequency. If unspecified, the encoder dynamically adjusts the cutoff. </dd> <dt><samp class="option">strict (--strictly-enforce-ISO)</samp></dt> <dd> Enable LAME’s strict ISO buffer constraints when set to ‘<samp class="samp">strict</samp>’ or ‘<samp class="samp">very</samp>’. Other values leave LAME’s default buffer constraints unchanged. This can improve compatibility with some decoders, at the cost of reduced bit reservoir usage. </dd> <dt><samp class="option">reservoir</samp></dt> <dd> Enable use of bit reservoir when set to 1. Default value is 1. LAME has this enabled by default, but can be overridden by use <samp class="option">--nores</samp> option. </dd> <dt><samp class="option">joint_stereo (-m j)</samp></dt> <dd> Enable the encoder to use (on a frame by frame basis) either L/R stereo or mid/side stereo. Default value is 1. </dd> <dt><samp class="option">abr (--abr)</samp></dt> <dd> Enable the encoder to use ABR when set to 1. The <code class="command">lame</code> <samp class="option">--abr</samp> sets the target bitrate, while this options only tells FFmpeg to use ABR still relies on <samp class="option">b</samp> to set bitrate. </dd> <dt><samp class="option">copyright (-c)</samp></dt> <dd> Set MPEG audio copyright flag when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">original (-o)</samp></dt> <dd> Set MPEG audio original flag when set to 1. The default value is 1 (enabled). </dd> </dl>

 <h3 class="section">8.9 libopencore-amrnb# TOC</h3>

OpenCORE Adaptive Multi-Rate Narrowband encoder. Requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopencore-amrnb --enable-version3</code>. This is a mono-only encoder. Officially it only supports 8000Hz sample rate, but you can override it by setting <samp class="option">strict</samp> to ‘<samp class="samp">unofficial</samp>’ or lower. <h4 class="subsection">8.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate in bits per second. Only the following bitrates are supported, otherwise libavcodec will round to the nearest valid bitrate. <dl class="table"> <dt><samp class="option">4750</samp></dt> <dt><samp class="option">5150</samp></dt> <dt><samp class="option">5900</samp></dt> <dt><samp class="option">6700</samp></dt> <dt><samp class="option">7400</samp></dt> <dt><samp class="option">7950</samp></dt> <dt><samp class="option">10200</samp></dt> <dt><samp class="option">12200</samp></dt> </dl>

</dd> <dt><samp class="option">dtx</samp></dt> <dd> Allow discontinuous transmission (generate comfort noise) when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">8.10 libopus# TOC</h3>

libopus Opus Interactive Audio Codec encoder wrapper. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopus</code>. <h4 class="subsection">8.10.1 Option Mapping# TOC</h4>

Most libopus options are modelled after the <code class="command">opusenc</code> utility from opus-tools. The following is an option mapping chart describing options supported by the libopus wrapper, and their <code class="command">opusenc</code>-equivalent in parentheses. <dl class="table"> <dt><samp class="option">b (bitrate)</samp></dt>

<dd> Set the bit rate in bits/s. FFmpeg’s <samp class="option">b</samp> option is expressed in bits/s, while <code class="command">opusenc</code>’s <samp class="option">bitrate</samp> in kilobits/s. </dd> <dt><samp class="option">vbr (vbr, hard-cbr, and cvbr)</samp></dt> <dd> Set VBR mode. The FFmpeg <samp class="option">vbr</samp> option has the following valid arguments, with the <code class="command">opusenc</code> equivalent options in parentheses: <dl class="table"> <dt>‘<samp class="samp">off (hard-cbr)</samp>’</dt> <dd> Use constant bit rate encoding. </dd> <dt>‘<samp class="samp">on (vbr)</samp>’</dt> <dd> Use variable bit rate encoding (the default). </dd> <dt>‘<samp class="samp">constrained (cvbr)</samp>’</dt> <dd> Use constrained variable bit rate encoding. </dd> </dl>

</dd> <dt><samp class="option">compression_level (comp)</samp></dt> <dd> Set encoding algorithm complexity. Valid options are integers in the 0-10 range. 0 gives the fastest encodes but lower quality, while 10 gives the highest quality but slowest encoding. The default is 10. </dd> <dt><samp class="option">frame_duration (framesize)</samp></dt> <dd> Set maximum frame size, or duration of a frame in milliseconds. The argument must be exactly the following: 2.5, 5, 10, 20, 40, 60. Smaller frame sizes achieve lower latency but less quality at a given bitrate. Sizes greater than 20ms are only interesting at fairly low bitrates. The default is 20ms. </dd> <dt><samp class="option">packet_loss (expect-loss)</samp></dt> <dd> Set expected packet loss percentage. The default is 0. </dd> <dt><samp class="option">fec (n/a)</samp></dt> <dd> Enable inband forward error correction. <samp class="option">packet_loss</samp> must be non-zero to take advantage - frequency of FEC ’side-data’ is proportional to expected packet loss. Default is disabled. </dd> <dt><samp class="option">application (N.A.)</samp></dt> <dd> Set intended application type. Valid options are listed below: <dl class="table"> <dt>‘<samp class="samp">voip</samp>’</dt> <dd> Favor improved speech intelligibility. </dd> <dt>‘<samp class="samp">audio</samp>’</dt> <dd> Favor faithfulness to the input (the default). </dd> <dt>‘<samp class="samp">lowdelay</samp>’</dt> <dd> Restrict to only the lowest delay modes by disabling voice-optimized modes. </dd> </dl>

</dd> <dt><samp class="option">cutoff (N.A.)</samp></dt> <dd> Set cutoff bandwidth in Hz. The argument must be exactly one of the following: 4000, 6000, 8000, 12000, or 20000, corresponding to narrowband, mediumband, wideband, super wideband, and fullband respectively. The default is 0 (cutoff disabled). Note that libopus forces a wideband cutoff for bitrates < 15 kbps, unless CELT-only (<samp class="option">application</samp>

set to ‘<samp class="samp">lowdelay</samp>’) mode is used. </dd> <dt><samp class="option">mapping_family (mapping_family)</samp></dt> <dd> Set channel mapping family to be used by the encoder. The default value of -1 uses mapping family 0 for mono and stereo inputs, and mapping family 1 otherwise. The default also disables the surround masking and LFE bandwidth optimizations in libopus, and requires that the input contains 8 channels or fewer. Other values include 0 for mono and stereo, 1 for surround sound with masking and LFE bandwidth optimizations, and 255 for independent streams with an unspecified channel layout. </dd> <dt><samp class="option">dtx (N.A.)</samp></dt> <dd> Allow discontinuous transmission when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">apply_phase_inv (N.A.) (requires libopus >= 1.2)</samp></dt> <dd> If set to 0, disables the use of phase inversion for intensity stereo, improving the quality of mono downmixes, but slightly reducing normal stereo quality. The default is 1 (phase inversion enabled). </dd> </dl>

 <h3 class="section">8.11 libshine# TOC</h3>

Shine Fixed-Point MP3 encoder wrapper. Shine is a fixed-point MP3 encoder. It has a far better performance on platforms without an FPU, e.g. armel CPUs, and some phones and tablets. However, as it is more targeted on performance than quality, it is not on par with LAME and other production-grade encoders quality-wise. Also, according to the project’s homepage, this encoder may not be free of bugs as the code was written a long time ago and the project was dead for at least 5 years. This encoder only supports stereo and mono input. This is also CBR-only. The original project (last updated in early 2007) is at http://sourceforge.net/projects/libshine-fxp/. We only support the updated fork by the Savonet/Liquidsoap project at https://github.com/savonet/shine. Requires the presence of the libshine headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libshine</code>. See also libmp3lame. <h4 class="subsection">8.11.1 Options# TOC</h4>

The following options are supported by the libshine wrapper. The <code class="command">shineenc</code>-equivalent of the options are listed in parentheses. <dl class="table"> <dt><samp class="option">b (<em

class=”emph”>-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR. <code class=”command”>shineenc</code> <samp class=”option”>-b</samp> option is expressed in kilobits/s. </dd> </dl>

 <h3 class=”section”>8.12 libtwolame# TOC</h3>

TwoLAME MP2 encoder wrapper. Requires the presence of the libtwolame headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libtwolame</code>. <h4 class=”subsection”>8.12.1 Options# TOC</h4>

The following options are supported by the libtwolame wrapper. The <code class=”command”>twolame</code>-equivalent options follow the FFmpeg ones and are in parentheses. <dl class=”table”> <dt><samp class=”option”>b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for CBR. <code class=”command”>twolame</code> <samp class=”option”>b</samp> option is expressed in kilobits/s. Default value is 128k. </dd> <dt><samp class=”option”>q (-V)</samp></dt> <dd> Set quality for experimental VBR support. Maximum value range is from -50 to 50, useful range is from -10 to 10. The higher the value, the better the quality. This option is valid only using the <code class=”command”>ffmpeg</code> command-line tool. For library interface users, use <samp class=”option”>global_quality</samp>. </dd> <dt><samp class=”option”>mode (--mode)</samp></dt> <dd> Set the mode of the resulting audio. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Choose mode automatically based on the input. This is the default. </dd> <dt>‘<samp class=”samp”>stereo</samp>’</dt> <dd> Stereo </dd> <dt>‘<samp class=”samp”>joint_stereo</samp>’</dt> <dd> Joint stereo </dd> <dt>‘<samp class=”samp”>dual_channel</samp>’</dt> <dd> Dual channel </dd> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> Mono </dd> </dl>

</dd> <dt><samp class=”option”>psymodel (--psyc-mode)</samp></dt> <dd> Set psychoacoustic model to use in encoding. The argument must be an integer between -1 and 4, inclusive. The higher the value, the better the quality. The default value is 3. </dd> <dt><samp class=”option”>energy_levels (--energy)</samp></dt> <dd> Enable energy levels extensions when set to 1. The default value is 0 (dis-

abled). </dd> <dt><samp class="option">error_protection (--protect)</samp></dt> <dd> Enable CRC error protection when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">copyright (--copyright)</samp></dt> <dd> Set MPEG audio copyright flag when set to 1. The default value is 0 (disabled). </dd> <dt><samp class="option">original (--original)</samp></dt> <dd> Set MPEG audio original flag when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">8.13 libvo-amrwbenc# TOC</h3>

VisualOn Adaptive Multi-Rate Wideband encoder. Requires the presence of the libvo-amrwbenc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libvo-amrwbenc --enable-version3</code>. This is a mono-only encoder. Officially it only supports 16000Hz sample rate, but you can override it by setting <samp class="option">strict</samp> to ‘<samp class="samp">unofficial</samp>’ or lower. <h4 class="subsection">8.13.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate in bits/s. Only the following bitrates are supported, otherwise libavcodec will round to the nearest valid bitrate. <dl class="table"> <dt>‘<samp class="samp">6600</samp>’</dt> <dt>‘<samp class="samp">8850</samp>’</dt> <dt>‘<samp class="samp">12650</samp>’</dt> <dt>‘<samp class="samp">14250</samp>’</dt> <dt>‘<samp class="samp">15850</samp>’</dt> <dt>‘<samp class="samp">18250</samp>’</dt> <dt>‘<samp class="samp">19850</samp>’</dt> <dt>‘<samp class="samp">23050</samp>’</dt> <dt>‘<samp class="samp">23850</samp>’</dt> </dl>

</dd> <dt><samp class="option">dtx</samp></dt> <dd> Allow discontinuous transmission (generate comfort noise) when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">8.14 libvorbis# TOC</h3>

libvorbis encoder wrapper. Requires the presence of the libvorbisenc headers and library during configuration. You need to explicitly configure

the build with <code class=”code”>--enable-libvorbis</code>. <h4 class=”subsection”>8.14.1 Options# TOC</h4>

The following options are supported by the libvorbis wrapper. The <code class=”command”>oggenc</code>-equivalent of the options are listed in parentheses. To get a more accurate and extensive documentation of the libvorbis options, consult the libvorbisenc’s and <code class=”command”>oggenc</code>’s documentations. See http://xiph.org/vorbis/, http://wiki.xiph.org/Vorbis-tools, and oggenc(1). <dl class=”table”> <dt><samp class=”option”>b (-b)</samp></dt> <dd> Set bitrate expressed in bits/s for ABR. <code class=”command”>oggenc</code> <samp class=”option”>-b</samp> is expressed in kilobits/s. </dd> <dt><samp class=”option”>q (-q)</samp></dt> <dd> Set constant quality setting for VBR. The value should be a float number in the range of -1.0 to 10.0. The higher the value, the better the quality. The default value is ‘<samp class=”samp”>3.0</samp>’. This option is valid only using the <code class=”command”>ffmpeg</code> command-line tool. For library interface users, use <samp class=”option”>global_quality</samp>. </dd> <dt><samp class=”option”>cutoff (--advanced-encode-option lowpass_frequency=N)</samp></dt> <dd> Set cutoff bandwidth in Hz, a value of 0 disables cutoff. <code class=”command”>oggenc</code>’s related option is expressed in kHz. The default value is ‘<samp class=”samp”>0</samp>’ (cutoff disabled). </dd> <dt><samp class=”option”>minrate (-m)</samp></dt> <dd> Set minimum bitrate expressed in bits/s. <code class=”command”>oggenc</code> <samp class=”option”>-m</samp> is expressed in kilobits/s. </dd> <dt><samp class=”option”>maxrate (-M)</samp></dt> <dd> Set maximum bitrate expressed in bits/s. <code class=”command”>oggenc</code> <samp class=”option”>-M</samp> is expressed in kilobits/s. This only has effect on ABR mode. </dd> <dt><samp class=”option”>iblock (--advanced-encode-option impulse_noisetune=N)</samp></dt> <dd> Set noise floor bias for impulse blocks. The value is a float number from -15.0 to 0.0. A negative bias instructs the encoder to pay special attention to the crispness of transients in the encoded audio. The tradeoff for better transient response is a higher bitrate. </dd> </dl>

 <h3 class=”section”>8.15 mjpeg# TOC</h3>

Motion JPEG encoder. <h4 class="subsection">8.15.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">huffman</samp></dt> <dd> Set the huffman encoding strategy. Possible values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Use the default huffman tables. This is the default strategy. </dd> <dt>‘<samp class="samp">optimal</samp>’</dt> <dd> Compute and use optimal huffman tables. </dd> </dl>

</dd> <dt><samp class="option">intra_dc_precision <var class="var">integer</var></samp></dt> <dd> Set the precision of the DC coefficient minus 8. Valid range is 0 to 3. Higher values quantize DC more finely as the DC entry of the written quantization table is 8 >> <var class="var">intra_dc_precision</var> so 3 gives a DC quantizer of 1. Default is 0 which keeps the previous fixed DC quantizer of 8. Combined with <code class="code">-q:v 1</code> and an <code class="code">intra_matrix</code> of all 8s, a value of 3 produces a quantization table of all ones. </dd> </dl>

 <h3 class="section">8.16 wavpack# TOC</h3>

WavPack lossless audio encoder. <h4 class="subsection">8.16.1 Options# TOC</h4>

The equivalent options for <code class="command">wavpack</code> command line utility are listed in parentheses. <h4 class="subsubsection">8.16.1.1 Shared options# TOC</h4>

The following shared options are effective for this encoder. Only special notes about this particular encoder will be documented here. For the general meaning of the options, see the Codec Options chapter. <dl class="table"> <dt><samp class="option">frame_size (--blocksize)</samp></dt> <dd> For this encoder, the range for this option is between 128 and 131072. Default is automatically decided based on sample rate and number of channel. For the complete formula of calculating default, see <samp class="file">libavcodec/wavpackenc.c</samp>. </dd> <dt><samp class="option">compression_level (-f, -h, -hh, and

-x)</samp></dt> </dl>

 <h4 class="subsubsection">8.16.1.2 Private options# TOC</h4>

<dl class="table"> <dt><samp class="option">joint_stereo (-j)</samp></dt> <dd> Set whether to enable joint stereo. Valid values are: <dl class="table"> <dt>‘<samp class="samp">on (1)</samp>’</dt> <dd> Force mid/side audio encoding. </dd> <dt>‘<samp class="samp">off (0)</samp>’</dt> <dd> Force left/right audio encoding. </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Let the encoder decide automatically. </dd> </dl>

</dd> <dt><samp class="option">optimize_mono</samp></dt> <dd> Set whether to enable optimization for mono. This option is only effective for non-mono streams. Available values: <dl class="table"> <dt>‘<samp class="samp">on</samp>’</dt> <dd> enabled </dd> <dt>‘<samp class="samp">off</samp>’</dt> <dd> disabled </dd> </dl>

</dd> </dl>

 <h2 class="chapter">9 Video Encoders# TOC</h2>

A description of some of the currently available video encoders follows. <h3 class="section">9.1 a64_multi, a64_multi5# TOC</h3>

A64 / Commodore 64 multicolor charset encoder. <code class="code">a64_multi5</code> is extended with 5th color (colram). <h3 class="section">9.2 Cinepak# TOC</h3>

Cinepak aka CVID encoder. Compatible with Windows 3.1 and vintage MacOS. <h4 class="subsection">9.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">g <var class="var">integer</var></samp></dt>
<dd> Keyframe interval. A keyframe is inserted at least every
<code class="code">-g</code> frames, sometimes sooner. </dd>
<dt><samp class="option">q:v <var class="var">integer</var></samp></dt>
<dd> Quality factor. Lower is better. Higher gives lower bitrate. The
following table lists bitrates when encoding akiyo_cif.y4m for various values of
<code class="code">-q:v</code> with <code class="code">-g 100</code>:
 <dl class="table"> <dt><samp class="option"><code class="code">-
q:v 1</code> 1918 kb/s</samp></dt> <dt><samp class="option"><code
class="code">-q:v 2</code> 1735 kb/s</samp></dt> <dt><samp
class="option"><code class="code">-q:v 4</code> 1500 kb/s</samp></dt>
<dt><samp class="option"><code class="code">-q:v 10</code> 1041
kb/s</samp></dt> <dt><samp class="option"><code class="code">-q:v
20</code> 826 kb/s</samp></dt> <dt><samp class="option"><code
class="code">-q:v 40</code> 553 kb/s</samp></dt> <dt><samp
class="option"><code class="code">-q:v 100</code> 394 kb/s</samp></dt>
<dt><samp class="option"><code class="code">-q:v 200</code> 312
kb/s</samp></dt> <dt><samp class="option"><code class="code">-q:v
400</code> 266 kb/s</samp></dt> <dt><samp class="option"><code
class="code">-q:v 1000</code> 237 kb/s</samp></dt> </dl>

</dd> <dt><samp class="option">max_extra_cb_iterations <var
class="var">integer</var></samp></dt> <dd> Max extra codebook
recalculation passes, more is better and slower. </dd> <dt><samp
class="option">skip_empty_cb <var class="var">boolean</var></samp></dt>
<dd> Avoid wasting bytes, ignore vintage MacOS decoder. </dd>
<dt><samp class="option">max_strips <var class="var">integer</var></samp></dt>
<dt><samp class="option">min_strips <var class="var">integer</var></samp></dt>
<dd> The minimum and maximum number of strips to use. Wider range
sometimes improves quality. More strips is generally better quality but costs
more bits. Fewer strips tend to yield more keyframes. Vintage compatible is
1..3. </dd> <dt><samp class="option">strip_number_adaptivity
<var class="var">integer</var></samp></dt> <dd> How much num-
ber of strips is allowed to change between frames. Higher is better but slower.
 </dd> </dl>

 <h3 class="section">9.3
ffv1<a class="anchor hidden-xs" href="#ffv1"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-ffv1"
aria-hidden="true">TOC</h3>

FFv1 Encoder <h4 class="subsection">9.3.1
Options<a class="anchor hidden-xs" href="#Options-
27" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Options-27" aria-hidden="true">TOC</h4>

The following options are supported by FFmpeg’s FFv1 encoder.
 <dl class="table"> <dt><samp class="option">context</samp></dt>

<dd> Sets the context size, 0 (default) is small, 1 is big, 2 is minimal. The minimal context set compresses better when coding many slices, as each slice has too little data for larger context sets to adapt to. </dd> <dt><samp class="option">coder</samp></dt> <dd> Set the coder, <dl class="table"> <dt>‘<samp class="samp">rice</samp>’</dt> <dd> Golomb rice coder </dd> <dt>‘<samp class="samp">range_def</samp>’</dt> <dd> Range coder with default table </dd> <dt>‘<samp class="samp">range_tab</samp>’</dt> <dd> Range coder with custom table </dd> </dl>

</dd> <dt><samp class="option">slicecrc</samp></dt> <dd> -1 (default, automatic), 1 use crc with zero initial and final state, 2 use crc with non zero initial and final state </dd> <dt><samp class="option">qtable</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> default, automatic </dd> <dt>‘<samp class="samp">8bit</samp>’</dt> <dd> use 8bit default </dd> <dt>‘<samp class="samp">greater8bit</samp>’</dt> <dd> use >8bit default </dd> </dl>

</dd> <dt><samp class="option">remap_optimizer</samp></dt> <dd> 0 - 5, default 3, how much effort the encoder puts into optimizing the remap table. </dd> </dl>

 <h3 class="section">9.4 GIF# TOC</h3>

GIF image/animation encoder. <h4 class="subsection">9.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">gifflags <var class="var">integer</var></samp></dt> <dd> Sets the flags used for GIF encoding. <dl class="table"> <dt><samp class="option">offsetting</samp></dt> <dd> Enables picture offsetting. Default is enabled. </dd> <dt><samp class="option">transdiff</samp></dt> <dd> Enables transparency detection between frames. Default is enabled. </dd> </dl>

</dd> <dt><samp class="option">gifimage <var class="var">integer</var></samp></dt> <dd> Enables encoding one full GIF image per frame, rather than an animated GIF. Default value is <samp class="option">0</samp>. </dd> <dt><samp class="option">global_palette <var class="var">integer</var></samp></dt> <dd> Writes a palette to the global GIF header where feasible. If disabled, every frame will always have a palette written, even if there is a global palette supplied. Default value is <samp class="option">1</samp>. </dd> </dl>

 <h3 class="section">9.5 Hap# TOC</h3>

Vidvox Hap video encoder. <h4 class="subsection">9.5.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">format <var class="var">integer</var></samp></dt> <dd> Specifies the Hap format to encode. <dl class="table"> <dt><samp class="option">hap</samp></dt> <dt><samp class="option">hap_alpha</samp></dt> <dt><samp class="option">hap_q</samp></dt> </dl>

Default value is <samp class="option">hap</samp>. </dd> <dt><samp class="option">chunks <var class="var">integer</var></samp></dt> <dd> Specifies the number of chunks to split frames into, between 1 and 64. This permits multithreaded decoding of large frames, potentially at the cost of data-rate. The encoder may modify this value to divide frames evenly. Default value is <var class="var">1</var>. </dd> <dt><samp class="option">compressor <var class="var">integer</var></samp></dt> <dd> Specifies the second-stage compressor to use. If set to <samp class="option">none</samp>, <samp class="option">chunks</samp> will be limited to 1, as chunked uncompressed frames offer no benefit. <dl class="table"> <dt><samp class="option">none</samp></dt> <dt><samp class="option">snappy</samp></dt> </dl>

Default value is <samp class="option">snappy</samp>. </dd> </dl>

 <h3 class="section">9.6 jpeg2000# TOC</h3>

The native jpeg 2000 encoder is lossy by default, the <code class="code">-q:v</code> option can be used to set the encoding quality. Lossless encoding can be selected with <code class="code">-pred 1</code>. <h4 class="subsection">9.6.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">format <var class="var">integer</var></samp></dt> <dd> Can be set to either <code class="code">j2k</code> or <code class="code">jp2</code> (the default) that makes it possible to store non-rgb pix_fmts. </dd> <dt><samp class="option">tile_width <var class="var">integer</var></samp></dt> <dd> Sets tile width. Range is 1 to 1073741824. Default is 256. </dd> <dt><samp

class=”option”>tile_height <var class=”var”>integer</var></samp></dt>
<dd> Sets tile height. Range is 1 to 1073741824. Default is 256.
</dd> <dt><samp class=”option”>pred <var class=”var”>integer</var></samp></dt>
<dd> Allows setting the discrete wavelet transform (DWT) type <dl
class=”table”> <dt><samp class=”option”>dwt97int (Lossy)</samp></dt>
<dt><samp class=”option”>dwt53 (Lossless)</samp></dt> </dl>
Default is <code class=”code”>dwt97int</code> </dd>
<dt><samp class=”option”>sop <var class=”var”>boolean</var></samp></dt>
<dd> Enable this to add SOP marker at the start of each packet.
Disabled by default. </dd> <dt><samp class=”option”>eph
<var class=”var”>boolean</var></samp></dt> <dd> Enable
this to add EPH marker at the end of each packet header. Disabled
by default. </dd> <dt><samp class=”option”>prog <var
class=”var”>integer</var></samp></dt> <dd> Sets the progression
order to be used by the encoder. Possible values are: <dl class=”table”>
<dt><samp class=”option”>lrcp</samp></dt> <dt><samp class=”option”>rlcp</samp></dt>
<dt><samp class=”option”>rpcl</samp></dt> <dt><samp class=”option”>pcrl</samp></dt>
<dt><samp class=”option”>cprl</samp></dt> </dl> Set to
<code class=”code”>lrcp</code> by default. </dd> <dt><samp
class=”option”>layer_rates <var class=”var”>string</var></samp></dt>
<dd> By default, when this option is not used, compression is done using
the quality metric. This option allows for compression using compression
ratio. The compression ratio for each level could be specified. The compression ratio of a layer <code class=”code”>l</code> species the what
ratio of total file size is contained in the first <code class=”code”>l</code>
layers. Example usage: <pre
class=”example-preformatted”>ffmpeg -i input.bmp -c:v jpeg2000 -layer_rates
"100,10,1" output.j2k </pre>

This would compress the image to contain 3 layers, where the data contained in the first layer would be compressed by 1000 times, compressed by 100
in the first two layers, and shall contain all data while using all 3 layers.
</dd> </dl>

 <h3 class=”section”>9.7 librav1e<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#librav1e” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-librav1e”
aria-hidden=”true”>TOC</h3>

rav1e AV1 encoder wrapper. Requires the presence of the rav1e
headers and library during configuration. You need to explicitly configure
the build with <code class=”code”>--enable-librav1e</code>. <a
name=”Options-31”> <h4 class=”subsection”>9.7.1 Options<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Options-31” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Options-31”
aria-hidden=”true”>TOC</h4>

<dl class=”table”> <dt><samp class=”option”>qmax</samp></dt>

<dd> Sets the maximum quantizer to use when using bitrate mode. </dd> <dt><samp class="option">qmin</samp></dt> <dd> Sets the minimum quantizer to use when using bitrate mode. </dd> <dt><samp class="option">qp</samp></dt> <dd> Uses quantizer mode to encode at the given quantizer (0-255). </dd> <dt><samp class="option">speed</samp></dt> <dd> Selects the speed preset (0-10) to encode with. </dd> <dt><samp class="option">tiles</samp></dt> <dd> Selects how many tiles to encode with. </dd> <dt><samp class="option">tile-rows</samp></dt> <dd> Selects how many rows of tiles to encode with. </dd> <dt><samp class="option">tile-columns</samp></dt> <dd> Selects how many columns of tiles to encode with. </dd> <dt><samp class="option">rav1e-params</samp></dt> <dd> Set rav1e options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ":". See <code class="command">rav1e --help</code> for a list of options. For example to specify librav1e encoding options with <samp class="option">-rav1e-params</samp>:   <pre class="example-preformatted">ffmpeg -i input -c:v librav1e -b:v 500K -rav1e-params speed=5:low_latency=true output.mp4 </pre>

</dd> </dl>

 <h3 class="section">9.8 libaom-av1# TOC</h3>

libaom AV1 encoder wrapper. Requires the presence of the libaom headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libaom</code>. <h4 class="subsection">9.8.1 Options# TOC</h4>

The wrapper supports the following standard libavcodec options: <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set bitrate target in bits/second. By default this will use variable-bitrate mode. If <samp class="option">maxrate</samp> and <samp class="option">minrate</samp> are also set to the same value then it will use constant-bitrate mode, otherwise if <samp class="option">crf</samp> is set as well then it will use constrained-quality mode. </dd> <dt><samp class="option">g keyint_min</samp></dt> <dd> Set key frame placement. The GOP size sets the maximum distance between key frames; if zero the output stream will be intra-only. The minimum distance is ignored unless it is the same as the GOP size, in which case key frames will always appear at a fixed interval. Not set by default, so without this

option the library has completely free choice about where to place key frames. </dd> <dt><samp class="option">qmin qmax</samp></dt> <dd> Set minimum/maximum quantisation values. Valid range is from 0 to 63 (warning: this does not match the quantiser values actually used by AV1 - divide by four to map real quantiser values to this range). Defaults to min/max (no constraint). </dd> <dt><samp class="option">minrate maxrate bufsize rc_init_occupancy</samp></dt> <dd> Set rate control buffering parameters. Not used if not set - defaults to unconstrained variable bitrate. </dd> <dt><samp class="option">threads</samp></dt> <dd> Set the number of threads to use while encoding. This may require the <samp class="option">tiles</samp> or <samp class="option">row-mt</samp> options to also be set to actually use the specified number of threads fully. Defaults to the number of hardware threads supported by the host machine. </dd> <dt><samp class="option">profile</samp></dt> <dd> Set the encoding profile. Defaults to using the profile which matches the bit depth and chroma subsampling of the input. </dd> </dl>

The wrapper also has some specific options: <dl class="table"> <dt><samp class="option">cpu-used</samp></dt> <dd> Set the quality/encoding speed tradeoff. Valid range is from 0 to 8, higher numbers indicating greater speed and lower quality. The default value is 1, which will be slow and high quality. </dd> <dt><samp class="option">auto-alt-ref</samp></dt> <dd> Enable use of alternate reference frames. Defaults to the internal default of the library. </dd> <dt><samp class="option">arnr-max-frames (frames)</samp></dt> <dd> Set altref noise reduction max frame count. Default is -1. </dd> <dt><samp class="option">arnr-strength (strength)</samp></dt> <dd> Set altref noise reduction filter strength. Range is -1 to 6. Default is -1. </dd> <dt><samp class="option">aq-mode (aq-mode)</samp></dt> <dd> Set adaptive quantization mode. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class="samp">variance (1)</samp>’</dt> <dd> Variance-based. </dd> <dt>‘<samp class="samp">complexity (2)</samp>’</dt> <dd> Complexity-based. </dd> <dt>‘<samp class="samp">cyclic (3)</samp>’</dt> <dd> Cyclic refresh. </dd> </dl>

</dd> <dt><samp class="option">tune (tune)</samp></dt> <dd> Set the distortion metric the encoder is tuned with. Default is <code class="code">psnr</code>. <dl class="table"> <dt>‘<samp class="samp">psnr (0)</samp>’</dt> <dt>‘<samp class="samp">ssim (1)</samp>’</dt> </dl>

</dd>

<dt><samp class="option">lag-in-frames</samp></dt>

<dd> Set the maximum number of frames which the encoder may keep in flight

at any one time for lookahead purposes. Defaults to the internal

default of the library.

</dd>

<dt><samp class="option">error-resilience</samp></dt>

<dd> Enable error resilience features:

<dl class="table">

<dt><samp class="option">default</samp></dt>

<dd> Improve resilience against losses of whole frames.

</dd>

</dl>

Not enabled by default.

</dd>

<dt><samp class="option">crf</samp></dt>

<dd> Set the quality/size tradeoff for constant-quality (no bitrate target)

and constrained-quality (with maximum bitrate target) modes. Valid

range is 0 to 63, higher numbers indicating lower quality and smaller

output size. Only used if set; by default only the bitrate target is

used.

</dd>

<dt><samp class="option">static-thresh</samp></dt>

<dd> Set a change threshold on blocks below which they will be skipped by

the encoder. Defined in arbitrary units as a nonnegative integer,

defaulting to zero (no blocks are skipped).

</dd>

<dt><samp class="option">drop-threshold</samp></dt>

<dd> Set a threshold for dropping frames when close to rate control bounds.
Defined as a percentage of the target buffer - when the rate control
buffer falls below this percentage, frames will be dropped until it
has refilled above the threshold. Defaults to zero (no frames are
dropped).

</dd>

<dt><samp class="option">denoise-noise-level (level)</samp></dt>

<dd> Amount of noise to be removed for grain synthesis. Grain synthesis is disabled if
this option is not set or set to 0.

</dd>

<dt><samp class="option">denoise-block-size (pixels)</samp></dt>

<dd> Block size used for denoising for grain synthesis. If not set, AV1 codec
uses the default value of 32.

</dd>

<dt><samp class="option">undershoot-pct (pct)</samp></dt>

<dd> Set datarate undershoot (min) percentage of the target bitrate. Range is -1 to 100.
Default is -1.

</dd>

<dt><samp class="option">overshoot-pct (pct)</samp></dt>

<dd> Set datarate overshoot (max) percentage of the target bitrate. Range is -1 to 1000.
Default is -1.

</dd>

<dt><samp class="option">minsection-pct (pct)</samp></dt>

<dd> Minimum percentage variation of the GOP bitrate from the target bitrate. If minsection-pct

is not set, the libaomenc wrapper computes it as follows: <code class="code">(minrate * 100 / bitrate)</code>.

Range is -1 to 100. Default is -1 (unset).

</dd>

<dt><samp class="option">maxsection-pct (pct)</samp></dt>

<dd> Maximum percentage variation of the GOP bitrate from the target bitrate. If maxsection-pct

is not set, the libaomenc wrapper computes it as follows: <code class="code">(maxrate * 100 / bitrate)</code>.

Range is -1 to 5000. Default is -1 (unset).

</dd>

<dt><samp class="option">frame-parallel (boolean)</samp></dt>

<dd> Enable frame parallel decodability features. Default is true.

</dd>

<dt><samp class="option">tiles</samp></dt>

<dd> Set the number of tiles to encode the input video with, as columns x rows. Larger numbers allow greater parallelism in both encoding and

decoding, but may decrease coding efficiency. Defaults to the minimum

number of tiles required by the size of the input video (this is 1x1

(that is, a single tile) for sizes up to and including 4K).

</dd>

<dt><samp class="option">tile-columns tile-rows</samp></dt>

<dd> Set the number of tiles as log2 of the number of tile rows and columns.

Provided for compatibility with libvpx/VP9.

</dd>

<dt><samp class="option">row-mt (Requires libaom >= 1.0.0-759-g90a15f4f2)</samp></dt>

<dd> Enable row based multi-threading. Disabled by default.

</dd>

<dt><samp class="option">enable-cdef (boolean)</samp></dt>

<dd> Enable Constrained Directional Enhancement Filter. The libaom-av1 encoder enables CDEF by default.

</dd>

<dt><samp class="option">enable-restoration (boolean)</samp></dt>

<dd> Enable Loop Restoration Filter. Default is true for libaom-av1.

</dd>

<dt><samp class="option">enable-global-motion (boolean)</samp></dt>

<dd> Enable the use of global motion for block prediction. Default is true.

</dd>

<dt><samp class="option">enable-intrabc (boolean)</samp></dt>

<dd> Enable block copy mode for intra block prediction. This mode is useful for screen content. Default is true.

</dd>

<dt><samp class="option">enable-rect-partitions (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable rectangular partitions. Default is true.

</dd>

<dt><samp class="option">enable-1to4-partitions (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable 1:4/4:1 partitions. Default is true.

</dd>

<dt><samp class="option">enable-ab-partitions (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable AB shape partitions. Default is true.

</dd>

<dt><samp class="option">enable-angle-delta (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable angle delta intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-cfl-intra (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable chroma predicted from luma intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-filter-intra (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable filter intra predictor. Default is true.

</dd>

<dt><samp class="option">enable-intra-edge-filter (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable intra edge filter. Default is true.

</dd>

<dt><samp class="option">enable-smooth-intra (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable smooth intra prediction mode. Default is true.

</dd>

<dt><samp class="option">enable-paeth-intra (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable paeth predictor in intra prediction. Default is true.

</dd>

<dt><samp class="option">enable-palette (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable palette prediction mode. Default is true.

</dd>

<dt><samp class="option">enable-flip-idtx (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable extended transform type, including FLIPADST_DCT,
DCT_FLIPADST,

FLIPADST_FLIPADST, ADST_FLIPADST, FLIPADST_ADST, IDTX,
V_DCT, H_DCT,

V_ADST, H_ADST, V_FLIPADST, H_FLIPADST. Default is true.

</dd>

<dt><samp class="option">enable-tx64 (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable 64-pt transform. Default is true.

</dd>

<dt><samp class="option">reduced-tx-type-set (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use reduced set of transform types. Default is false.

</dd>

<dt><samp class="option">use-intra-dct-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use DCT only for INTRA modes. Default is false.

</dd>

<dt><samp class="option">use-inter-dct-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use DCT only for INTER modes. Default is false.

</dd>

<dt><samp class="option">use-intra-default-tx-only (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use Default-transform only for INTRA modes. Default is false.

</dd>

<dt><samp class="option">enable-ref-frame-mvs (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable temporal mv prediction. Default is true.

</dd>

<dt><samp class="option">enable-reduced-reference-set (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Use reduced set of single and compound references. Default is false.

</dd>

<dt><samp class="option">enable-obmc (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable obmc. Default is true.

</dd>

<dt><samp class="option">enable-dual-filter (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable dual filter. Default is true.

</dd>

<dt><samp class="option">enable-diff-wtd-comp (boolean)
(Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable difference-weighted compound. Default is true.

</dd>

<dt><samp class="option">enable-dist-wtd-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable distance-weighted compound. Default is true.

</dd>

<dt><samp class="option">enable-onesided-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable one sided compound. Default is true.

</dd>

<dt><samp class="option">enable-interinter-wedge (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interinter wedge compound. Default is true.

</dd>

<dt><samp class="option">enable-interintra-wedge (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interintra wedge compound. Default is true.

</dd>

<dt><samp class="option">enable-masked-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable masked compound. Default is true.

</dd>

<dt><samp class="option">enable-interintra-comp (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable interintra compound. Default is true.

</dd>

<dt><samp class="option">enable-smooth-interintra (boolean) (Requires libaom >= v2.0.0)</samp></dt>

<dd> Enable smooth interintra mode. Default is true.

</dd>

<dt><samp class="option">aom-params</samp></dt>

<dd> Set libaom options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated

by ":". For a list of supported options, see <code class="command">aomenc --help</code> under the

section "AV1 Specific Options".

For example to specify libaom encoding options with <samp class="option">-aom-params</samp>:

<pre class="example-preformatted">ffmpeg -i input -c:v libaom-av1 -b:v 500K -aom-params tune=psnr:enable-tpl-model=1 output.mp4

</pre>

</dd> </dl>

 <h3 class="section">9.9 liboapv# TOC</h3>

Advanced Professional Video codec encoder wrapper. This encoder requires the presence of the liboapv headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-liboapv</samp>.   Many liboapv encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. The apv project website is at <a class="url" href="https://github.com/AcademySoftwareFoundation/openapv">https://github.com/AcademySoftwareFou
  <h4 class="subsection">9.9.1 Options# TOC</h4>

The following options are supported by the liboapv wrapper.   To get a more extensive documentation of

the liboapv options, consult the liboapv documentation. <dl class="table"> <dt><samp class="option">preset</samp></dt> <dd> Set the quality-speed tradeoff [fastest, fast, medium, slow, placebo, default] </dd> <dt><samp class="option">qp</samp></dt> <dd> Set the quantization parameter value for CQP rate control mode. </dd> <dt><samp class="option">oapv-params (parse_apv_params)</samp></dt> <dd> Set liboapvenc options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ":". See the liboapv encoder user guide for a list of accepted parameters. </dd> </dl>

 <h3 class="section">9.10 libsvtav1# TOC</h3>

SVT-AV1 encoder wrapper. Requires the presence of the SVT-AV1 headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libsvtav1</code>. <h4 class="subsection">9.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Set the encoding profile. <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt> <dt>‘<samp class="samp">professional</samp>’</dt> </dl>

</dd> <dt><samp class="option">level</samp></dt> <dd> Set the operating point level. For example: ’4.0’ </dd> <dt><samp class="option">hielevel</samp></dt> <dd> Set the Hierarchical prediction levels. <dl class="table"> <dt>‘<samp class="samp">3level</samp>’</dt> <dt>‘<samp class="samp">4level</samp>’</dt> <dd> This is the default. </dd> </dl>

</dd> <dt><samp class="option">tier</samp></dt> <dd> Set the operating point tier. <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dd> This is the default. </dd> <dt>‘<samp class="samp">high</samp>’</dt> </dl>

</dd> <dt><samp class="option">qmax</samp></dt> <dd> Set the maximum quantizer to use when using a bitrate mode. </dd> <dt><samp class="option">qmin</samp></dt> <dd> Set the minimum quantizer to use when using a bitrate mode. </dd> <dt><samp class="option">crf</samp></dt> <dd> Constant rate factor value used in crf rate control mode (0-63). </dd>

<dt><samp class="option">qp</samp></dt> <dd> Set the quantizer used in cqp rate control mode (0-63). </dd> <dt><samp class="option">sc_detection</samp></dt> <dd> Enable scene change detection. </dd> <dt><samp class="option">la_depth</samp></dt> <dd> Set number of frames to look ahead (0-120). </dd> <dt><samp class="option">preset</samp></dt> <dd> Set the quality-speed tradeoff, in the range 0 to 13. Higher values are faster but lower quality. </dd> <dt><samp class="option">tile_rows</samp></dt> <dd> Set log2 of the number of rows of tiles to use (0-6). </dd> <dt><samp class="option">tile_columns</samp></dt> <dd> Set log2 of the number of columns of tiles to use (0-4). </dd> <dt><samp class="option">svtav1-params</samp></dt> <dd> Set SVT-AV1 options using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ":". See the SVT-AV1 encoder user guide for a list of accepted parameters. </dd> </dl>

 <h3 class="section">9.11 libsvtjpegxs# TOC</h3>

SVT-JPEG-XS encoder wrapper. Requires the presence of the SVT-JPEG-XS headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libsvtjpegxs</code>. <h4 class="subsection">9.11.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">decomp_v</samp></dt> <dd> Set vertical decomposition level </dd> <dt><samp class="option">decomp_h</samp></dt> <dd> Set horizontal decomposition level </dd> <dt><samp class="option">quantization</samp></dt> <dd> Set the quantization algorithm. <dl class="table"> <dt>‘<samp class="samp">deadzone</samp>’</dt> <dt>‘<samp class="samp">uniform</samp>’</dt> </dl>

</dd> <dt><samp class="option">coding-signs</samp></dt> <dd> Enable Signs handling strategy <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">full</samp>’</dt> </dl>

</dd> <dt><samp class="option">coding-sigf</samp></dt> <dd> Enable Significance coding </dd> <dt><samp class="option">coding-vpred</samp></dt> <dd> Enable Vertical Prediction coding <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dt>‘<samp class="samp">no_residuals</samp>’</dt>

<dt>‘<samp class="samp">no_coeffs</samp>’</dt> </dl>

</dd> </dl>

 <h3 class="section">9.12 libjxl# TOC</h3>

libjxl JPEG XL encoder wrapper. Requires the presence of the libjxl headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libjxl</code>. <h4 class="subsection">9.12.1 Options# TOC</h4>

The libjxl wrapper supports the following options: <dl class="table"> <dt><samp class="option">distance</samp></dt> <dd> Set the target Butteraugli distance. This is a quality setting: lower distance yields higher quality, with distance=1.0 roughly comparable to libjpeg Quality 90 for photographic content. Setting distance=0.0 yields true lossless encoding. Valid values range between 0.0 and 15.0, and sane values rarely exceed 5.0. Setting distance=0.1 usually attains transparency for most input. The default is 1.0. </dd> <dt><samp class="option">effort</samp></dt> <dd> Set the encoding effort used. Higher effort values produce more consistent quality and usually produces a better quality/bpp curve, at the cost of more CPU time required. Valid values range from 1 to 9, and the default is 7. </dd> <dt><samp class="option">modular</samp></dt> <dd> Force the encoder to use Modular mode instead of choosing automatically. The default is to use VarDCT for lossy encoding and Modular for lossless. VarDCT is generally superior to Modular for lossy encoding but does not support lossless encoding. </dd> </dl>

 <h3 class="section">9.13 libkvazaar# TOC</h3>

Kvazaar H.265/HEVC encoder. Requires the presence of the libkvazaar headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libkvazaar</samp>. <h4 class="subsection">9.13.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set target video bitrate in bit/s and enable rate control. </dd>

<dt><samp class=”option”>kvazaar-params</samp></dt> <dd> Set kvazaar parameters as a list of <var class=”var”>name</var>=<var class=”var”>value</var> pairs separated by commas (,). See kvazaar documentation for a list of options. </dd> </dl>

 <h3 class=”section”>9.14 libopenh264# TOC</h3>

Cisco libopenh264 H.264/MPEG-4 AVC encoder wrapper. This encoder requires the presence of the libopenh264 headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopenh264</code>. The library is detected using <code class=”command”>pkg-config</code>. For more information about the library see http://www.openh264.org. <h4 class=”subsection”>9.14.1 Options# TOC</h4>

The following FFmpeg global options affect the configurations of the libopenh264 encoder. <dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dd> Set the bitrate (as a number of bits per second). </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class=”option”>maxrate</samp></dt> <dd> Set the max bitrate (as a number of bits per second). </dd> <dt><samp class=”option”>flags +global_header</samp></dt> <dd> Set global header in the bitstream. </dd> <dt><samp class=”option”>slices</samp></dt> <dd> Set the number of slices, used in parallelized encoding. Default value is 0. This is only used when <samp class=”option”>slice_mode</samp> is set to ‘<samp class=”samp”>fixed</samp>’. </dd> <dt><samp class=”option”>loopfilter</samp></dt> <dd> Enable loop filter, if set to 1 (automatically enabled). To disable set a value of 0. </dd> <dt><samp class=”option”>profile</samp></dt> <dd> Set profile restrictions. If set to the value of ‘<samp class=”samp”>main</samp>’ enable CABAC (set the <code class=”code”>SEncParamExt.iEntropyCodingModeFlag</code> flag to 1). </dd> <dt><samp class=”option”>max_nal_size</samp></dt> <dd> Set maximum NAL size in bytes. </dd> <dt><samp class=”option”>allow_skip_frames</samp></dt> <dd> Allow skipping frames to hit the target bitrate if set to 1. </dd> </dl>

 <h3 class=”section”>9.15 libtheora# TOC</h3>

libtheora Theora encoder wrapper. Requires the presence of the libtheora headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libtheora</code>. For more information about the libtheora project see http://www.theora.org/. <h4 class="subsection">9.15.1 Options# TOC</h4>

The following global options are mapped to internal libtheora options which affect the quality and the bitrate of the encoded stream. <dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Set the video bitrate in bit/s for CBR (Constant Bit Rate) mode. In case VBR (Variable Bit Rate) mode is enabled this option is ignored. </dd> <dt><samp class="option">flags</samp></dt> <dd> Used to enable constant quality mode (VBR) encoding through the <samp class="option">qscale</samp> flag, and to enable the <code class="code">pass1</code> and <code class="code">pass2</code> modes. </dd> <dt><samp class="option">g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class="option">global_quality</samp></dt> <dd> Set the global quality as an integer in lambda units. Only relevant when VBR mode is enabled with <code class="code">flags +qscale</code>. The value is converted to QP units by dividing it by <code class="code">FF_QP2LAMBDA</code>, clipped in the [0 - 10] range, and then multiplied by 6.3 to get a value in the native libtheora range [0-63]. A higher value corresponds to a higher quality. </dd> <dt><samp class="option">q</samp></dt> <dd> Enable VBR mode when set to a non-negative value, and set constant quality value as a double floating point value in QP units. The value is clipped in the [0-10] range, and then multiplied by 6.3 to get a value in the native libtheora range [0-63]. This option is valid only using the <code class="command">ffmpeg</code> command-line tool. For library interface users, use <samp class="option">global_quality</samp>. </dd> </dl>

 <h4 class="subsection">9.15.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set maximum constant quality (VBR) encoding with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i INPUT -codec:v libtheora -q:v 10 OUTPUT.ogg </pre>

Use <code class="command">ffmpeg</code> to convert a CBR 1000 kbps Theora video stream: <pre class="example-

preformatted”>ffmpeg -i INPUT -codec:v libtheora -b:v 1000k OUTPUT.ogg </pre>

 <h3 class=”section”>9.16 libvpx# TOC</h3>

VP8/VP9 format supported through libvpx. Requires the presence of the libvpx headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libvpx</code>. <h4 class=”subsection”>9.16.1 Options# TOC</h4>

The following options are supported by the libvpx wrapper. The <code class=”command”>vpxenc</code>-equivalent options or values are listed in parentheses for easy migration. To reduce the duplication of documentation, only the private options and some others requiring special attention are documented here. For the documentation of the undocumented generic options, see the Codec Options chapter. To get more documentation of the libvpx options, invoke the command <code class=”command”>ffmpeg -h encoder=libvpx</code>, <code class=”command”>ffmpeg -h encoder=libvpx-vp9</code> or <code class=”command”>vpxenc --help</code>. Further information is available in the libvpx API documentation. <dl class=”table”> <dt><samp class=”option”>b (target-bitrate)</samp></dt> <dd> Set bitrate in bits/s. Note that FFmpeg’s <samp class=”option”>b</samp> option is expressed in bits/s, while <code class=”command”>vpxenc</code>’s <samp class=”option”>target-bitrate</samp> is in kilobits/s. </dd> <dt><samp class=”option”>g (kf-max-dist)</samp></dt> <dt><samp class=”option”>keyint_min (kf-min-dist)</samp></dt> <dt><samp class=”option”>qmin (min-q)</samp></dt> <dd> Minimum (Best Quality) Quantizer. </dd> <dt><samp class=”option”>qmax (max-q)</samp></dt> <dd> Maximum (Worst Quality) Quantizer. Can be changed per-frame. </dd> <dt><samp class=”option”>bufsize (buf-sz, buf-optimal-sz)</samp></dt> <dd> Set ratecontrol buffer size (in bits). Note <code class=”command”>vpxenc</code>’s options are specified in milliseconds, the libvpx wrapper converts this value as follows: <code class=”code”>buf-sz = bufsize * 1000 / bitrate</code>, <code class=”code”>buf-optimal-sz = bufsize * 1000 / bitrate * 5 / 6</code>. </dd> <dt><samp class=”option”>rc_init_occupancy (buf-initial-sz)</samp></dt> <dd> Set number of bits which should be loaded into the rc buffer before de-

coding starts. Note <code class=”command”>vpxenc</code>’s option is specified in milliseconds, the libvpx wrapper converts this value as follows: <code class=”code”>rc_init_occupancy * 1000 / bitrate</code>. </dd> <dt><samp class=”option”>undershoot-pct</samp></dt> <dd> Set datarate undershoot (min) percentage of the target bitrate. </dd> <dt><samp class=”option”>overshoot-pct</samp></dt> <dd> Set datarate overshoot (max) percentage of the target bitrate. </dd> <dt><samp class=”option”>skip_threshold (drop-frame)</samp></dt> <dt><samp class=”option”>qcomp (bias-pct)</samp></dt> <dt><samp class=”option”>maxrate (maxsection-pct)</samp></dt> <dd> Set GOP max bitrate in bits/s. Note <code class=”command”>vpxenc</code>’s option is specified as a percentage of the target bitrate, the libvpx wrapper converts this value as follows: <code class=”code”>(maxrate * 100 / bitrate)</code>. </dd> <dt><samp class=”option”>minrate (minsection-pct)</samp></dt> <dd> Set GOP min bitrate in bits/s. Note <code class=”command”>vpxenc</code>’s option is specified as a percentage of the target bitrate, the libvpx wrapper converts this value as follows: <code class=”code”>(minrate * 100 / bitrate)</code>. </dd> <dt><samp class=”option”>minrate, maxrate, b end-usage=cbr</samp></dt> <dd> <code class=”code”>(minrate == maxrate == bitrate)</code>. </dd> <dt><samp class=”option”>crf (end-usage=cq, cq-level)</samp></dt> <dt><samp class=”option”>tune (tune)</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>psnr (psnr)</samp>’</dt> <dt>‘<samp class=”samp”>ssim (ssim)</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>quality, deadline (deadline)</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>best</samp>’</dt> <dd> Use best quality deadline. Poorly named and quite slow, this option should be avoided as it may give worse quality output than good. </dd> <dt>‘<samp class=”samp”>good</samp>’</dt> <dd> Use good quality deadline. This is a good trade-off between speed and quality when used with the <samp class=”option”>cpu-used</samp> option. </dd> <dt>‘<samp class=”samp”>realtime</samp>’</dt> <dd> Use realtime quality deadline. </dd> </dl>

</dd> <dt><samp class=”option”>speed, cpu-used (cpu-used)</samp></dt> <dd> Set quality/speed ratio modifier. Higher values speed up the encode at the cost of quality. </dd> <dt><samp class=”option”>nr (noise-sensitivity)</samp></dt> <dt><samp class=”option”>static-thresh</samp></dt> <dd> Set a change threshold on blocks below which they will be skipped by the encoder.

 </dd> <dt><samp class="option">slices (token-parts)</samp></dt> <dd> Note that FFmpeg’s <samp class="option">slices</samp> option gives the total number of partitions, while <code class="command">vpxenc</code>’s <samp class="option">token-parts</samp> is given as <code class="code">log2(partitions)</code>. </dd> <dt><samp class="option">max-intra-rate</samp></dt> <dd> Set maximum I-frame bitrate as a percentage of the target bitrate. A value of 0 means unlimited. </dd> <dt><samp class="option">force_key_frames</samp></dt> <dd> <code class="code">VPX_EFLAG_FORCE_
 </dd> <dt><samp class="option">Alternate reference frame related</samp></dt> <dd><dl class="table"> <dt><samp class="option">auto-alt-ref</samp></dt> <dd> Enable use of alternate reference frames (2-pass only). Values greater than 1 enable multi-layer alternate reference frames (VP9 only). </dd> <dt><samp class="option">arnr-maxframes</samp></dt> <dd> Set altref noise reduction max frame count. </dd> <dt><samp class="option">arnr-type</samp></dt> <dd> Set altref noise reduction filter type: backward, forward, centered. </dd> <dt><samp class="option">arnr-strength</samp></dt> <dd> Set altref noise reduction filter strength. </dd> <dt><samp class="option">rc-lookahead, lag-in-frames (lag-in-frames)</samp></dt> <dd> Set number of frames to look ahead for frametype and ratecontrol. </dd> <dt><samp class="option">min-gf-interval</samp></dt> <dd> Set minimum golden/alternate reference frame interval (VP9 only). </dd> </dl>

</dd> <dt><samp class="option">error-resilient</samp></dt> <dd> Enable error resiliency features. </dd> <dt><samp class="option">sharpness <var class="var">integer</var></samp></dt> <dd> Increase sharpness at the expense of lower PSNR. The valid range is [0, 7]. </dd> <dt><samp class="option">ts-parameters</samp></dt> <dd> Sets the temporal scalability configuration using a :-separated list of key=value pairs. For example, to specify temporal scalability parameters with <code class="code">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i INPUT -c:v libvpx -ts-parameters ts_number_layers=3:\ ts_target_bitrate=250,500,1000:ts_rate_decimator=4,2,1:\ ts_periodicity=4:ts_layer_id=0,2,1,2:ts_layering_mode=3 OUTPUT
</pre> Below is a brief explanation of each of the parameters, please refer to <code class="code">struct vpx_codec_enc_cfg</code> in <code class="code">vpx/vpx_encoder.h</code> for more details. <dl class="table"> <dt><samp class="option">ts_number_layers</samp></dt> <dd> Number of temporal coding layers. </dd> <dt><samp class="option">ts_target_bitrate</samp></dt> <dd> Target bitrate for each temporal layer (in kbps). (bitrate should be inclusive of the lower temporal layer). </dd> <dt><samp class="option">ts_rate_decimator</samp></dt> <dd> Frame rate decimation factor for each temporal layer. </dd> <dt><samp class="option">ts_periodicity</samp></dt> <dd> Length

of the sequence defining frame temporal layer membership. </dd> <dt><samp class="option">ts_layer_id</samp></dt> <dd> Template defining the membership of frames to temporal layers. </dd> <dt><samp class="option">ts_layering_mode</samp></dt> <dd> (optional) Selecting the temporal structure from a set of pre-defined temporal layering modes. Currently supports the following options. <dl class="table"> <dt><samp class="option">0</samp></dt> <dd> No temporal layering flags are provided internally, relies on flags being passed in using <code class="code">metadata</code> field in <code class="code">AVFrame</code> with following keys. <dl class="table"> <dt><samp class="option">vp8-flags</samp></dt> <dd> Sets the flags passed into the encoder to indicate the referencing scheme for the current frame. Refer to function <code class="code">vpx_codec_encode</code> in <code class="code">vpx/vpx_encoder.h</code> for more details. </dd> <dt><samp class="option">temporal_id</samp></dt> <dd> Explicitly sets the temporal id of the current frame to encode. </dd> </dl> </dd> <dt><samp class="option">2</samp></dt> <dd> Two temporal layers. 0-1... </dd> <dt><samp class="option">3</samp></dt> <dd> Three temporal layers. 0-2-1-2...; with single reference frame. </dd> <dt><samp class="option">4</samp></dt> <dd> Same as option "3", except there is a dependency between the two temporal layer 2 frames within the temporal period. </dd> </dl> </dd> </dl>

</dd> <dt><samp class="option">VP8-specific options</samp></dt> <dd><dl class="table"> <dt><samp class="option">screen-content-mode</samp></dt> <dd> Screen content mode, one of: 0 (off), 1 (screen), 2 (screen with more aggressive rate control). </dd> </dl>

</dd> <dt><samp class="option">VP9-specific options</samp></dt> <dd><dl class="table"> <dt><samp class="option">lossless</samp></dt> <dd> Enable lossless mode. </dd> <dt><samp class="option">tile-columns</samp></dt> <dd> Set number of tile columns to use. Note this is given as <code class="code">log2(tile_columns)</code>. For example, 8 tile columns would be requested by setting the <samp class="option">tile-columns</samp> option to 3. </dd> <dt><samp class="option">tile-rows</samp></dt> <dd> Set number of tile rows to use. Note this is given as <code class="code">log2(tile_rows)</code>. For example, 4 tile rows would be requested by setting the <samp class="option">tile-rows</samp> option to 2. </dd> <dt><samp class="option">frame-parallel</samp></dt> <dd> Enable frame parallel decodability features. </dd> <dt><samp class="option">aq-mode</samp></dt> <dd> Set adaptive quantization mode (0: off (default), 1: variance 2: complexity, 3: cyclic refresh, 4: equator360). </dd> <dt><samp class="option">colorspace color-space</samp></dt> <dd> Set input color space. The VP9 bitstream supports signaling the following colorspaces:

<dl class="table"> <dt><samp class="option">‘<samp class="samp">rgb</samp>’ sRGB</samp></dt> <dt><samp class="option">‘<samp class="samp">bt709</samp>’ bt709</samp></dt> <dt><samp class="option">‘<samp class="samp">unspecified</samp>’ unknown</samp></dt> <dt><samp class="option">‘<samp class="samp">bt470bg</samp>’ bt601</samp></dt> <dt><samp class="option">‘<samp class="samp">smpte170m</samp>’ smpte170</samp></dt> <dt><samp class="option">‘<samp class="samp">smpte240m</samp>’ smpte240</samp></dt> <dt><samp class="option">‘<samp class="samp">bt2020_ncl</samp>’ bt2020</samp></dt> </dl> </dd> <dt><samp class="option">row-mt <var class="var">boolean</var></samp></dt> <dd> Enable row based multi-threading. </dd> <dt><samp class="option">tune-content</samp></dt> <dd> Set content type: default (0), screen (1), film (2). </dd> <dt><samp class="option">corpus-complexity</samp></dt> <dd> Corpus VBR mode is a variant of standard VBR where the complexity distribution midpoint is passed in rather than calculated for a specific clip or chunk. The valid range is [0, 10000]. 0 (default) uses standard VBR. </dd> <dt><samp class="option">enable-tpl <var class="var">boolean</var></samp></dt> <dd> Enable temporal dependency model. </dd> <dt><samp class="option">ref-frame-config</samp></dt> <dd> Using per-frame metadata, set members of the structure <code class="code">vpx_svc_ref_frame_config_t</code> in <code class="code">vpx/vp8cx.h</code> to fine-control referencing schemes and frame buffer management.
Use a :-separated list of key=value pairs. For example, <pre class="example-preformatted">av_dict_set(&av_frame->metadata, "ref-frame-config", \ "rfc_update_buffer_slot=7:rfc_lst_fb_idx=0:rfc_gld_fb_idx=1:rfc_alt_fb_idx=2:rfc
</pre> <dl class="table"> <dt><samp class="option">rfc_update_buffer_slot</samp></dt> <dd> Indicates the buffer slot number to update </dd> <dt><samp class="option">rfc_update_last</samp></dt> <dd> Indicates whether to update the LAST frame </dd> <dt><samp class="option">rfc_update_golden</samp></dt> <dd> Indicates whether to update GOLDEN frame </dd> <dt><samp class="option">rfc_update_alt_ref</samp></dt> <dd> Indicates whether to update ALT_REF frame </dd> <dt><samp class="option">rfc_lst_fb_idx</samp></dt> <dd> LAST frame buffer index </dd> <dt><samp class="option">rfc_gld_fb_idx</samp></dt> <dd> GOLDEN frame buffer index </dd> <dt><samp class="option">rfc_alt_fb_idx</samp></dt> <dd> ALT_REF frame buffer index </dd> <dt><samp class="option">rfc_reference_last</samp></dt> <dd> Indicates whether to reference LAST frame </dd> <dt><samp class="option">rfc_reference_golden</samp></dt> <dd> Indicates whether to reference GOLDEN frame </dd> <dt><samp class="option">rfc_reference_alt_ref</samp></dt> <dd> Indicates whether to reference ALT_REF frame </dd> <dt><samp class="option">rfc_reference_duration</samp></dt> <dd> Indicates frame duration </dd> </dl> </dd> </dl>

</dd> </dl>

For more information about libvpx see: http://www
  <h3 class=”section”>9.17 libvvenc<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#libvvenc” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-libvvenc”
aria-hidden=”true”>TOC</h3>

VVenC H.266/VVC encoder wrapper. This encoder requires
the presence of the libvvenc headers and library during configuration. You
need to explicitly configure the build with <samp class=”option”>--enable-
libvvenc</samp>. The VVenC project website is at <a class=”url”
href=”https://github.com/fraunhoferhhi/vvenc”>https://github.com/fraunhoferhhi/vvenc.
  <h4 class=”subsection”>9.17.1
Supported Pixel Formats<a class=”anchor
hidden-xs” href=”#Supported-Pixel-Formats” aria-hidden=”true”>#
<a class=”anchor hidden-xs”href=”#toc-Supported-Pixel-Formats” aria-
hidden=”true”>TOC</h4>

VVenC supports only 10-bit color spaces as input. But the internal
(encoded) bit depth can be set to 8-bit or 10-bit at runtime. <a
name=”Options-41”> <h4 class=”subsection”>9.17.2 Options<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Options-41” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Options-41”
aria-hidden=”true”>TOC</h4>

<dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dd> Sets
target video bitrate. </dd> <dt><samp class=”option”>g</samp></dt>
<dd> Set the GOP size. Currently support for g=1 (Intra only) or
default. </dd> <dt><samp class=”option”>preset</samp></dt>
<dd> Set the VVenC preset. </dd> <dt><samp class=”option”>levelidc</samp></dt>
<dd> Set level idc. </dd> <dt><samp class=”option”>tier</samp></dt>
<dd> Set vvc tier. </dd> <dt><samp class=”option”>qp</samp></dt>
<dd> Set constant quantization parameter. </dd> <dt><samp
class=”option”>subopt <var class=”var”>boolean</var></samp></dt>
<dd> Set subjective (perceptually motivated) optimization. Default
is 1 (on). </dd> <dt><samp class=”option”>bitdepth8 <var
class=”var”>boolean</var></samp></dt> <dd> Set 8bit coding
mode instead of using 10bit. Default is 0 (off). </dd> <dt><samp
class=”option”>period</samp></dt> <dd> set (intra) refresh pe-
riod in seconds. </dd> <dt><samp class=”option”>vvenc-
params</samp></dt> <dd> Set vvenc options using a list of <var
class=”var”>key</var>=<var class=”var”>value</var> couples separated
by ":". See <code class=”command”>vvencapp --fullhelp</code>
or <code class=”command”>vvencFFapp --fullhelp</code> for a list of op-
tions. For example, the options might be provided as: <div
class=”example”> <pre class=”example-preformatted”>intraperiod=64:decodingrefreshtype=idr:poc0idr=1:i
</pre>

For example the encoding options might be provided with <samp

class=”option”>-vvenc-params</samp>:   <pre class=”example-preformatted”>ffmpeg -i input -c:v libvvenc -b 1M -vvenc-params intraperiod=64:decodingrefreshtype=idr:poc0idr=1:internalbitdepth=8 output.mp4 </pre>

</dd> </dl>

 <h3 class=”section”>9.18 libwebp# TOC</h3>

libwebp WebP Image encoder wrapper libwebp is Google’s official encoder for WebP images. It can encode in either lossy or lossless mode. Lossy images are essentially a wrapper around a VP8 frame. Lossless images are a separate codec developed by Google. <h4 class=”subsection”>9.18.1 Pixel Format# TOC</h4>

Currently, libwebp only supports YUV420 for lossy and RGB for lossless due to limitations of the format and libwebp. Alpha is supported for either mode. Because of API limitations, if RGB is passed in when encoding lossy or YUV is passed in for encoding lossless, the pixel format will automatically be converted using functions from libwebp. This is not ideal and is done only for convenience. <h4 class=”subsection”>9.18.2 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-lossless <var class=”var”>boolean</var></samp></dt> <dd> Enables/Disables use of lossless mode. Default is 0. </dd> <dt><samp class=”option”>-compression_level <var class=”var”>integer</var></samp></dt> <dd> For lossy, this is a quality/speed tradeoff. Higher values give better quality for a given size at the cost of increased encoding time. For lossless, this is a size/speed tradeoff. Higher values give smaller size at the cost of increased encoding time. More specifically, it controls the number of extra algorithms and compression tools used, and varies the combination of these tools. This maps to the <var class=”var”>method</var> option in libwebp. The valid range is 0 to 6. Default is 4. </dd> <dt><samp class=”option”>-quality <var class=”var”>float</var></samp></dt> <dd> For lossy encoding, this controls image quality. For lossless encoding, this controls the effort and time spent in compression. Range is 0 to 100. Default is 75. </dd> <dt><samp class=”option”>-preset <var class=”var”>type</var></samp></dt> <dd> Configuration preset. This does some automatic settings based on the general type of the image. <dl class=”table”> <dt><samp class=”option”>none</samp></dt>

<dd> Do not use a preset. </dd> <dt><samp class=”option”>default</samp></dt>
<dd> Use the encoder default. </dd> <dt><samp class=”option”>picture</samp></dt>
<dd> Digital picture, like portrait, inner shot </dd> <dt><samp
class=”option”>photo</samp></dt> <dd> Outdoor photograph, with
natural lighting </dd> <dt><samp class=”option”>drawing</samp></dt>
<dd> Hand or line drawing, with high-contrast details </dd>
<dt><samp class=”option”>icon</samp></dt> <dd> Small-sized colorful images </dd> <dt><samp class=”option”>text</samp></dt>
<dd> Text-like </dd> </dl>

</dd> </dl>

 <h3 class=”section”>9.19 libastcenc<span
class=”pull-right”># <a class=”anchor hidden-xs”href=”#toc-libastcenc-1”
aria-hidden=”true”>TOC</h3>

ARM ASTC Encoder wrapper. This encoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, which provides high-quality GPU texture compression to the ASTC format, installed as <code class=”code”>astcenc/astcenc.h</code> next to a library that <code class=”code”>-lastcenc</code> finds. Pass <code class=”code”>--enable-version3</code> together with <code class=”code”>--enable-libastcenc</code> to configure. The encoder supports both LDR (8-bit RGBA/RGB) and HDR (float/half RGBA) input. When float or half input is detected with an LDR profile, the profile is automatically promoted to HDR RGB with LDR alpha (<code class=”code”>HDR_RGB_LDR_A</code>); an explicitly selected HDR profile is kept. The raw bitstream can be packaged in the <samp class=”file”>.astc</samp> container (16-byte header + compressed blocks) or in the Khronos Texture 1.0 container (<samp class=”file”>.ktx</samp>). This implementation writes LDR 2D textures only through the <samp class=”file”>.ktx</samp> muxer: HDR ASTC and 3D block footprints are not supported by the muxer and <samp class=”file”>.astc</samp> must be used for them instead. The KTX 1.0 linear ASTC GL enums do not imply LDR either, so a linear <samp class=”file”>.ktx</samp> texture may still hold HDR endpoints; such a texture is decoded with HDR precision by default. The <samp class=”file”>.ktx</samp> muxer records its row order in a <code class=”code”>KTXorientation</code> key. <h4 class=”subsection”>9.19.1 Pixel Format# TOC</h4>

The encoder accepts the following pixel formats: <ul class=”itemize mark-bullet”> <code class=”code”>rgba</code> – 8-bit packed RGBA (LDR). <code class=”code”>rgb24</code> – 8-bit packed RGB (LDR, opaque alpha synthesized). <code

class=”code”>rgbaf16</code> – 16-bit floating-point RGBA (HDR). <code class=”code”>rgbaf32</code> – 32-bit floating-point RGBA (HDR). <code class=”code”>gbrapf32</code> – planar 32-bit floating-point GBR+Alpha (HDR).

swscale cannot convert to the packed floating-point formats (<code class=”code”>rgbaf16</code>, <code class=”code”>rgbaf32</code>); to encode ordinary image formats as HDR, produce a planar floating-point source instead, for example with <code class=”code”>-vf format=gbrapf32</code>. The packed formats are accepted when they are supplied directly, such as from a decoder or a custom filter graph. <h4 class=”subsection”>9.19.2 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>block_size <var class=”var”>string</var></samp></dt> <dd> ASTC block size specified as <var class=”var”>W</var>x<var class=”var”>H</var>[<code class=”code”>x</code><var class=”var”>D</var>]. When the third dimension is omitted it defaults to 1 (2D block). Valid 2D: 4x4, 5x4, 5x5, 6x5, 6x6, 8x5, 8x6, 8x8, 10x5, 10x6, 10x8, 10x10, 12x10, 12x12. Valid 3D: 3x3x3, 4x3x3, 4x4x3, 4x4x4, 5x4x4, 5x5x4, 5x5x5, 6x5x5, 6x6x5, 6x6x6. Default is <code class=”code”>8x8</code>. </dd> <dt><samp class=”option”>quality <var class=”var”>float</var></samp></dt> <dd> Compression quality from 0 (fastest) to 100 (slowest, exhaustive search). Default is <code class=”code”>60</code>. </dd> <dt><samp class=”option”>profile <var class=”var”>int</var></samp></dt> <dd> Color profile. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>ldr</samp>’</dt> <dd> Linear LDR. </dd> <dt>‘<samp class=”samp”>ldr-srgb</samp>’</dt> <dd> sRGB LDR (default). </dd> <dt>‘<samp class=”samp”>hdr-ldr-a</samp>’</dt> <dd> HDR RGB channels with LDR alpha channel. </dd> <dt>‘<samp class=”samp”>hdr</samp>’</dt> <dd> Full HDR. </dd> </dl>

When float/half input is detected with an LDR profile, the profile is automatically promoted to <code class=”code”>hdr-ldr-a</code>. Conversely, selecting an HDR profile with 8-bit input is rejected. This option overrides <code class=”code”>AVCodecContext.profile</code>. When it is not set, the profile is taken from <code class=”code”>AVCodecContext.profile</code> (<code class=”code”>AV_PROFILE_ASTC_*</code>), where <code class=”code”>AV_PROFILE_UNKNOWN</code> selects <code class=”code”>ldr-srgb</code> as before. </dd> <dt><samp class=”option”>perceptual <var class=”var”>boolean</var></samp></dt> <dd> Use the perceptual (PSNR-weighted) error metric instead of peak-SNR. Default is <code class=”code”>0</code>. </dd> <dt><samp

class=”option”>alpha_weight <var class=”var”>boolean</var></samp></dt>
<dd> Enable per-texel alpha weighting. Default is <code class=”code”>1</code>.
 </dd> </dl>

 <h3 class=”section”>9.20 libx264, libx264rgb# TOC</h3

x264 H.264/MPEG-4 AVC encoder wrapper. This encoder requires the presence of the libx264 headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libx264</code>. libx264 supports an impressive number of features, including 8x8 and 4x4 adaptive spatial transform, adaptive B-frame placement, CAVLC/CABAC entropy coding, interlacing (MBAFF), lossless mode, psy optimizations for detail retention (adaptive quantization, psy-RD, psy-trellis). Many libx264 encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. Additionally the <samp class=”option”>x264opts</samp> and <samp class=”option”>x264-params</samp> private options allows one to pass a list of key=value tuples as accepted by the libx264 <code class=”code”>x264_param_parse</code> function. The x264 project website is at http://www.videolan.org/developers/x264.html
 The libx264rgb encoder is the same as libx264, except it accepts packed RGB pixel formats as input instead of YUV. <h4 class=”subsection”>9.20.1 Supported Pixel Formats# TOC</h4>

x264 supports 8- to 10-bit color spaces. The exact bit depth is controlled at x264’s configure time. <h4 class=”subsection”>9.20.2 Options# TOC</h4>

The following options are supported by the libx264 wrapper. The

<code class=”command”>x264</code>-equivalent options or values are listed in parentheses

for easy migration.

To reduce the duplication of documentation, only the private options

and some others requiring special attention are documented here. For

the documentation of the undocumented generic options, see

the Codec Options chapter.

To get a more accurate and extensive documentation of the libx264

options, invoke the command <code class=”command”>x264 --fullhelp</code> or consult

the libx264 documentation.

In the list below, note that the <code class=”command”>x264</code> option name is shown

in parentheses after the libavcodec corresponding name, in case there

is a direct mapping.

<dl class=”table”>

<dt><samp class=”option”>b (bitrate)</samp></dt>

<dd> Set bitrate in bits/s. Note that FFmpeg’s <samp class=”option”>b</samp> option is

expressed in bits/s, while <code class=”command”>x264</code>’s <samp class=”option”>bitrate</samp> is in

kilobits/s.

</dd>

<dt><samp class=”option”>bf (bframes)</samp></dt>

<dd> Number of B-frames between I and P-frames

</dd>

<dt><samp class=”option”>g (keyint)</samp></dt>

<dd> Maximum GOP size

</dd>

<dt><samp class=”option”>qmin (qpmin)</samp></dt>

<dd> Minimum quantizer scale

</dd>

<dt><samp class="option">qmax (qpmax)</samp></dt>

<dd> Maximum quantizer scale

</dd>

<dt><samp class="option">qdiff (qpstep)</samp></dt>

<dd> Maximum difference between quantizer scales

</dd>

<dt><samp class="option">qblur (qblur)</samp></dt>

<dd> Quantizer curve blur

</dd>

<dt><samp class="option">qcomp (qcomp)</samp></dt>

<dd> Quantizer curve compression factor

</dd>

<dt><samp class="option">refs (ref)</samp></dt>

<dd> Number of reference frames each P-frame can use. The range is <var class="var">0-16</var>.

</dd>

<dt><samp class="option">level (level)</samp></dt>

<dd> Set the <code class="code">x264_param_t.i_level_idc</code> value in case the value is

positive, it is ignored otherwise.

This value can be set using the <code class="code">AVCodecContext</code> API (e.g. by

setting the <code class="code">AVCodecContext</code> value directly), and is specified as

an integer mapped on a corresponding level (e.g. the value 31 maps

to H.264 level IDC "3.1", as defined in the <code class="code">x264_levels</code>

table). It is ignored when set to a non positive value.

Alternatively it can be set as a private option, overriding the value

set in <code class="code">AVCodecContext</code>, and in this case must be specified as

the level IDC identifier (e.g. "3.1"), as defined by H.264 Annex A.

</dd>

<dt><samp class="option">sc_threshold (scenecut)</samp></dt>

<dd> Sets the threshold for the scene change detection.

</dd>

<dt><samp class="option">trellis (trellis)</samp></dt>

<dd> Performs Trellis quantization to increase efficiency. Enabled by default.

</dd>

<dt><samp class="option">nr (nr)</samp></dt>

<dd> Noise reduction

</dd>

<dt><samp class="option">me_range (merange)</samp></dt>

<dd> Maximum range of the motion search in pixels.

</dd>

<dt><samp class="option">me_method (me)</samp></dt>

<dd> Set motion estimation method. Possible values in the decreasing order

of speed:

<dl class="table">

<dt>‘<samp class="samp">dia (dia)</samp>’</dt>

<dt>‘<samp class="samp">epzs (dia)</samp>’</dt>

<dd> Diamond search with radius 1 (fastest). ‘<samp class="samp">epzs</samp>’ is an alias for

‘<samp class="samp">dia</samp>’.

</dd>

<dt>‘<samp class="samp">hex (hex)</samp>’</dt>

<dd> Hexagonal search with radius 2.

</dd>

<dt>‘<samp class="samp">umh (umh)</samp>’</dt>

<dd> Uneven multi-hexagon search.

</dd>

<dt>‘<samp class="samp">esa (esa)</samp>’</dt>

<dd> Exhaustive search.

</dd>

<dt>‘<samp class="samp">tesa (tesa)</samp>’</dt>

<dd> Hadamard exhaustive search (slowest).

</dd>

</dl>

</dd> <dt><samp class="option">forced-idr</samp></dt> <dd> Normally, when forcing a I-frame type, the encoder can select any type of I-frame. This option forces it to choose an IDR-frame. </dd> <dt><samp class="option">subq (subme)</samp></dt> <dd> Sub-pixel motion estimation method. </dd> <dt><samp class="option">b_strategy (b-adapt)</samp></dt> <dd> Adaptive B-frame placement decision algorithm. Use only on first-pass. </dd> <dt><samp class="option">keyint_min (min-keyint)</samp></dt> <dd> Minimum GOP size. </dd> <dt><samp class="option">coder</samp></dt> <dd> Set entropy encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">ac</samp>’</dt> <dd> Enable CABAC. </dd> <dt>‘<samp class="samp">vlc</samp>’</dt> <dd> Enable CAVLC and disable CABAC. It generates the same effect as <code class="command">x264</code>’s <samp class="option">--no-cabac</samp> option. </dd> </dl>

</dd> <dt><samp class="option">cmp</samp></dt> <dd> Set full pixel motion estimation comparison algorithm. Possible values: <dl class="table"> <dt>‘<samp class="samp">chroma</samp>’</dt> <dd> Enable chroma in motion estimation. </dd> <dt>‘<samp class="samp">sad</samp>’</dt> <dd> Ignore chroma in motion estimation. It generates the same effect as <code class="command">x264</code>’s <samp class="option">--no-chroma-me</samp> option. </dd> </dl>

</dd> <dt><samp class="option">threads (threads)</samp></dt> <dd> Number of encoding threads. </dd> <dt><samp class="option">thread_type</samp></dt> <dd> Set multithreading technique. Possible values: <dl class="table"> <dt>‘<samp class="samp">slice</samp>’</dt> <dd> Slice-based multithreading. It generates the same effect as <code class="command">x264</code>’s <samp class="option">--sliced-threads</samp> option. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Frame-based multithreading. </dd> </dl>

</dd> <dt><samp class="option">flags</samp></dt> <dd> Set encoding flags. It can be used to disable closed GOP and enable open GOP by setting it to <code class="code">-cgop</code>. The result is similar to the behavior of <code class="command">x264</code>’s <samp class="option">--open-gop</samp> option. </dd> <dt><samp class="option">rc_init_occupancy (vbv-init)</samp></dt> <dd> Initial VBV buffer occupancy </dd> <dt><samp class="option">preset (preset)</samp></dt> <dd> Set the encoding preset. </dd> <dt><samp class="option">tune (tune)</samp></dt> <dd> Set tuning of the encoding params. </dd> <dt><samp class="option">profile (profile)</samp></dt> <dd> Set profile restrictions. </dd> <dt><samp class="option">fastfirstpass</samp></dt> <dd> Enable fast settings when encoding first pass, when set to 1. When set to 0, it has the same effect of <code class="command">x264</code>’s <samp class="option">--slow-firstpass</samp> option. </dd> <dt><samp class="option">crf (crf)</samp></dt> <dd> Set the quality for constant quality mode. </dd> <dt><samp class="option">crf_max (crf-max)</samp></dt> <dd> In CRF mode, prevents VBV from lowering quality beyond this point. </dd> <dt><samp class="option">qp (qp)</samp></dt> <dd> Set constant quantization rate control method parameter. </dd> <dt><samp class="option">aq-mode (aq-mode)</samp></dt> <dd> Set AQ method. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class="samp">variance (1)</samp>’</dt> <dd> Variance AQ (complexity mask). </dd> <dt>‘<samp class="samp">autovariance (2)</samp>’</dt>

<dd> Auto-variance AQ (experimental). </dd> </dl>

</dd> <dt><samp class="option">aq-strength (aq-strength)</samp></dt> <dd> Set AQ strength, reduce blocking and blurring in flat and textured areas. </dd> <dt><samp class="option">psy</samp></dt> <dd> Use psychovisual optimizations when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-psy</samp> option. </dd> <dt><samp class="option">psy-rd (psy-rd)</samp></dt> <dd> Set strength of psychovisual optimization, in <var class="var">psy-rd</var>:<var class="var">psy-trellis</var> format. </dd> <dt><samp class="option">rc-lookahead (rc-lookahead)</samp></dt> <dd> Set number of frames to look ahead for frametype and ratecontrol. </dd> <dt><samp class="option">weightb</samp></dt> <dd> Enable weighted prediction for B-frames when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-weightb</samp> option. </dd> <dt><samp class="option">weightp (weightp)</samp></dt> <dd> Set weighted prediction method for P-frames. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (0)</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class="samp">simple (1)</samp>’</d <dd> Enable only weighted refs </dd> <dt>‘<samp class="samp">smart (2)</samp>’</dt> <dd> Enable both weighted refs and duplicates </dd> </dl>

</dd> <dt><samp class="option">ssim (ssim)</samp></dt> <dd> Enable calculation and printing SSIM stats after the encoding. </dd> <dt><samp class="option">intra-refresh (intra-refresh)</samp></dt> <dd> Enable the use of Periodic Intra Refresh instead of IDR frames when set to 1. </dd> <dt><samp class="option">avcintra-class (class)</samp></dt> <dd> Configure the encoder to generate AVC-Intra. Valid values are 50, 100 and 200 </dd> <dt><samp class="option">bluray-compat (bluray-compat)</samp></dt> <dd> Configure the encoder to be compatible with the bluray standard. It is a shorthand for setting "bluray-compat=1 force-cfr=1". </dd> <dt><samp class="option">b-bias (b-bias)</samp></dt> <dd> Set the influence on how often B-frames are used. </dd> <dt><samp class="option">b-pyramid (b-pyramid)</samp></dt> <dd> Set method for keeping of some B-frames as references. Possible values: <dl class="table"> <dt>‘<samp class="samp">none (none)</samp>’</dt> <dd> Disabled. </dd> <dt>‘<samp class="samp">strict (strict)</samp>’</dt> <dd> Strictly

hierarchical pyramid. </dd> <dt>‘<samp class="samp">normal (normal)</samp>’</dt> <dd> Non-strict (not Blu-ray compatible). </dd> </dl>

</dd> <dt><samp class="option">mixed-refs</samp></dt> <dd> Enable the use of one reference per partition, as opposed to one reference per macroblock when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-mixed-refs</samp> option. </dd> <dt><samp class="option">8x8dct</samp></dt> <dd> Enable adaptive spatial transform (high profile 8x8 transform) when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-8x8dct</samp> option. </dd> <dt><samp class="option">fast-pskip</samp></dt> <dd> Enable early SKIP detection on P-frames when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-fast-pskip</samp> option. </dd> <dt><samp class="option">aud (aud)</samp></dt> <dd> Enable use of access unit delimiters when set to 1. </dd> <dt><samp class="option">mbtree</samp></dt> <dd> Enable use macroblock tree ratecontrol when set to 1. When set to 0, it has the same effect as <code class="command">x264</code>’s <samp class="option">--no-mbtree</samp> option. </dd> <dt><samp class="option">deblock (deblock)</samp></dt> <dd> Set loop filter parameters, in <var class="var">alpha</var>:<var class="var">beta</var> form. </dd> <dt><samp class="option">cplxblur (cplxblur)</samp></
<dd> Set fluctuations reduction in QP (before curve compression). </dd> <dt><samp class="option">partitions (partitions)</samp></dt> <dd> Set partitions to consider as a comma-separated list of values. Possible values in the list: <dl class="table"> <dt>‘<samp class="samp">p8x8</samp>’</dt> <dd> 8x8 P-frame partition. </dd> <dt>‘<samp class="samp">p4x4</samp>’</dt> <dd> 4x4 P-frame partition. </dd> <dt>‘<samp class="samp">b8x8</samp>’</dt> <dd> 4x4 B-frame partition. </dd> <dt>‘<samp class="samp">i8x8</samp>’</dt> <dd> 8x8 I-frame partition. </dd> <dt>‘<samp class="samp">i4x4</samp>’</dt> <dd> 4x4 I-frame partition. (Enabling ‘<samp class="samp">p4x4</samp>’ requires ‘<samp class="samp">p8x8</samp>’ to be enabled. Enabling ‘<samp class="samp">i8x8</samp>’ requires adaptive spatial transform (<samp class="option">8x8dct</samp> option) to be enabled.) </dd> <dt>‘<samp class="samp">none (none)</samp>’</dt> <dd> Do not consider any partitions. </dd> <dt>‘<samp class="samp">all (all)</samp>’</dt> <dd> Consider every partition. </dd> </dl>

</dd> <dt><samp class="option">direct-pred (direct)</samp></dt>
<dd> Set direct MV prediction mode. Possible values: <dl
class="table"> <dt>‘<samp class="samp">none (none)</samp>’
<dd> Disable MV prediction. </dd> <dt>‘<samp
class="samp">spatial (spatial)</samp>’</dt>
<dd> Enable spatial predicting. </dd> <dt>‘<samp
class="samp">temporal (temporal)</samp>’</dt>
<dd> Enable temporal predicting. </dd> <dt>‘<samp
class="samp">auto (auto)</samp>’</dt>
<dd> Automatically decided. </dd> </dl>

</dd> <dt><samp class="option">slice-max-size (slice-
max-size)</samp></dt> <dd> Set the limit of the size of
each slice in bytes. If not specified but RTP payload size (<samp
class="option">ps</samp>) is specified, that is used. </dd>
<dt><samp class="option">stats (stats)</samp></dt>
<dd> Set the file name for multi-pass stats. </dd> <dt><samp
class="option">nal-hrd (nal-hrd)</samp></dt>
<dd> Set signal HRD information (requires <samp class="option">vbv-
bufsize</samp> to be set). Possible values: <dl class="table">
<dt>‘<samp class="samp">none (none)</samp>’</dt>
<dd> Disable HRD information signaling. </dd> <dt>‘<samp
class="samp">vbr (vbr)</samp>’</dt>
<dd> Variable bit rate. </dd> <dt>‘<samp class="samp">cbr
(cbr)</samp>’</dt> <dd> Constant
bit rate (not allowed in MP4 container). </dd> </dl>

</dd> <dt><samp class="option">x264opts <var class="var">opts</var></samp></dt>
<dt><samp class="option">x264-params <var class="var">opts</var></samp></dt>
<dd> Override the x264 configuration using a :-separated list of key=value
options. The argument for both options is a list of <var
class="var">key</var>=<var class="var">value</var> couples separated
by ":". With <samp class="option">x264opts</samp> the
value can be omitted, and the value <code class="code">1</code> is
assumed in that case. For <var class="var">filter</var> and
<var class="var">psy-rd</var> options values that use ":" as
a separator themselves, use "," instead. They accept it as well
since long ago but this is kept undocumented for some reason. For
example, the options might be provided as: <pre
class="example-preformatted">level=30:bframes=0:weightp=0:cabac=0:ref=1:vbv-
maxrate=768:vbv-bufsize=2000:analyse=all:me=umh:no-fast-pskip=1:subq=6:8x8dct=0:trellis=0
</pre>

For example to specify libx264 encoding options with <code class="command">ffmpeg</code>:
 <pre class="example-preformatted">ffmpeg
-i foo.mpg -c:v libx264 -x264opts keyint=123:min-keyint=20 -an out.mkv
</pre>

To get the complete list of the libx264 options, invoke the command <code class="command">x264 --fullhelp</code> or consult the libx264 documentation. </dd> <dt><samp class="option">a53cc <var class="var">boolean</var></samp></dt> <dd> Import closed captions (which must be ATSC compatible format) into output. Only the mpeg2 and h264 decoders provide these. Default is 1 (on). </dd> <dt><samp class="option">udu_sei <var class="var">boolean</var></samp></dt> <dd> Import user data unregistered SEI if available into output. Default is 0 (off). </dd> <dt><samp class="option">mb_info <var class="var">boolean</var></samp></dt> <dd> Set mb_info data through AVFrameSideData, only useful when used from the API. Default is 0 (off). </dd> </dl>

Encoding ffpresets for common usages are provided so they can be used with the general presets system (e.g. passing the <samp class="option">pre</samp> option). <h3 class="section">9.21 libx265# TOC</h3>

x265 H.265/HEVC encoder wrapper. This encoder requires the presence of the libx265 headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libx265</samp>. <h4 class="subsection">9.21.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Sets target video bitrate. </dd> <dt><samp class="option">bf</samp></dt> <dt><samp class="option">g</samp></dt> <dd> Set the GOP size. </dd> <dt><samp class="option">keyint_min</samp></dt> <dd> Minimum GOP size. </dd> <dt><samp class="option">refs</samp></dt> <dd> Number of reference frames each P-frame can use. The range is from <var class="var">1-16</var>. </dd> <dt><samp class="option">preset</samp></dt> <dd> Set the x265 preset. </dd> <dt><samp class="option">tune</samp></dt> <dd> Set the x265 tune parameter. </dd> <dt><samp class="option">profile</samp></dt> <dd> Set profile restrictions. </dd> <dt><samp class="option">crf</samp></dt> <dd> Set the quality for constant quality mode. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set constant quantization rate control method parameter. </dd> <dt><samp class="option">qmin</samp></dt> <dd> Minimum quantizer scale. </dd> <dt><samp class="option">qmax</samp></dt> <dd> Maximum quantizer scale. </dd> <dt><samp class="option">qdiff</samp></dt> <dd> Maximum difference between quantizer scales. </dd> <dt><samp class="option">qblur</samp></dt> <dd> Quantizer

curve blur </dd> <dt><samp class="option">qcomp</samp></dt> <dd> Quantizer curve compression factor </dd> <dt><samp class="option">i_qfactor</samp></dt> <dt><samp class="option">b_qfactor</samp></dt> <dt><samp class="option">forced-idr</samp></dt> <dd> Normally, when forcing a I-frame type, the encoder can select any type of I-frame. This option forces it to choose an IDR-frame. </dd> <dt><samp class="option">x265-stats</samp></dt> <dd> Specify the file name for 2-pass stats. This is set automatically when using the <samp class="option">-passlogfile</samp> option. </dd> <dt><samp class="option">udu_sei <var class="var">boolean</var></samp></dt> <dd> Import user data unregistered SEI if available into output. Default is 0 (off). </dd> <dt><samp class="option">x265-params</samp></dt> <dd> Set x265 options using a list of <var class="var">key</var>=<var class="var">value</var> couples separated by ":". See <code class="command">x265 --help</code> for a list of options. For example to specify libx265 encoding options with <samp class="option">-x265-params</samp>:   <pre class="example-preformatted">ffmpeg -i input -c:v libx265 -x265-params crf=26:psy-rd=1 output.mp4 </pre> </dd> </dl>

 <h3 class="section">9.22 libxavs2# TOC</h3>

xavs2 AVS2-P2/IEEE1857.4 encoder wrapper. This encoder requires the presence of the libxavs2 headers and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxavs2</samp>. The following standard libavcodec options are used: <ul class="itemize mark-bullet"> <samp class="option">b</samp> / <samp class="option">bit_rate</samp> <samp class="option">g</samp> / <samp class="option">gop_size</samp> <samp class="option">bf</samp> / <samp class="option">max_b_frames</samp>

The encoder also has its own specific options: <h4 class="subsection">9.22.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">lcu_row_threads</samp></dt> <dd> Set the number of parallel threads for rows from 1 to 8 (default 5). </dd> <dt><samp class="option">initial_qp</samp></dt> <dd> Set the xavs2 quantization parameter from 1 to 63 (default 34). This is used to set the initial qp for the first frame. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set the xavs2 quantization parameter from 1 to 63 (default 34). This is used to set the qp value under constant-QP mode. </dd> <dt><samp

class=”option”>max_qp</samp></dt> <dd> Set the max qp for rate control from 1 to 63 (default 55). </dd> <dt><samp class=”option”>min_qp</samp></dt> <dd> Set the min qp for rate control from 1 to 63 (default 20). </dd> <dt><samp class=”option”>speed_level</samp></dt> <dd> Set the Speed level from 0 to 9 (default 0). Higher is better but slower. </dd> <dt><samp class=”option”>log_level</samp></dt> <dd> Set the log level from -1 to 3 (default 0). -1: none, 0: error, 1: warning, 2: info, 3: debug. </dd> <dt><samp class=”option”>xavs2-params</samp></dt> <dd> Set xavs2 options using a list of <var class=”var”>key</var>=<var class=”var”>value</var> couples separated by ":". For example to specify libxavs2 encoding options with <samp class=”option”>-xavs2-params</samp>:   <pre class=”example-preformatted”>ffmpeg -i input -c:v libxavs2 -xavs2-params RdoqLevel=0 output.avs2 </pre> </dd> </dl>

 <h3 class=”section”>9.23 libxeve# TOC</h3>

eXtra-fast Essential Video Encoder (XEVE) MPEG-5 EVC encoder wrapper. The xeve-equivalent options or values are listed in parentheses for easy migration. This encoder requires the presence of the libxeve headers and library during configuration. You need to explicitly configure the build with <samp class=”option”>--enable-libxeve</samp>.   Many libxeve encoder options are mapped to FFmpeg global codec options, while unique encoder options are provided through private options. Additionally the xeve-params private options allows one to pass a list of key=value tuples as accepted by the libxeve <code class=”code”>parse_xeve_params</code> function. The xeve project website is at https://github.com/mpeg5/xeve. <h4 class=”subsection”>9.23.1 Options# TOC</h4>

The following options are supported by the libxeve wrapper. The xeve-equivalent options or values are listed in parentheses for easy migration.   To reduce the duplication of documentation, only the private options and some others requiring special attention are documented here. For the documentation of the undocumented generic options, see the Codec Options chapter.   To get a more accurate and extensive documentation of the libxeve options, invoke the command <code class=”code”>xeve_app --help</code> or consult the libxeve documentation. <dl class=”table”> <dt><samp class=”option”>b (bitrate)</samp></dt> <dd> Set target

video bitrate in bits/s. Note that FFmpeg’s b option is expressed in bits/s, while xeve’s bitrate is in kilobits/s. </dd> <dt><samp class="option">bf (bframes)</samp></dt> <dd> Set the maximum number of B frames (1,3,7,15). </dd> <dt><samp class="option">g (keyint)</samp></dt> <dd> Set the GOP size (I-picture period). </dd> <dt><samp class="option">preset (preset)</samp></dt> <dd> Set the xeve preset. Set the encoder preset value to determine encoding speed [fast, medium, slow, placebo] </dd> <dt><samp class="option">tune (tune)</samp></dt> <dd> Set the encoder tune parameter [psnr, zerolatency] </dd> <dt><samp class="option">profile (profile)</samp></dt> <dd> Set the encoder profile [0: baseline; 1: main] </dd> <dt><samp class="option">crf (crf)</samp></dt> <dd> Set the quality for constant quality mode. Constant rate factor <10..49> [default: 32] </dd> <dt><samp class="option">qp (qp)</samp></dt> <dd> Set constant quantization rate control method parameter. Quantization parameter qp <0..51> [default: 32] </dd> <dt><samp class="option">threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class="section">9.24 libxvid# TOC</h3>

Xvid MPEG-4 Part 2 encoder wrapper. This encoder requires the presence of the libxvidcore headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libxvid --enable-gpl</code>. The native <code class="code">mpeg4</code> encoder supports the MPEG-4 Part 2 format, so users can encode to this format without this library. <h4 class="subsection">9.24.1 Options# TOC</h4>

The following options are supported by the libxvid wrapper. Some of the following options are listed but are not documented, and correspond to shared codec options. See the Codec Options chapter for their documentation. The other shared options which are not listed have no effect for the libxvid encoder. <dl class="table"> <dt><samp class="option">b</samp></dt> <dt><samp class="option">g</samp></dt> <dt><samp class="option">qmin</samp></dt> <dt><samp class="option">qmax</samp></dt> <dt><samp class="option">mpeg_quant</samp></dt> <dt><samp class="option">threads</samp></dt> <dt><samp class="option">bf</samp></dt> <dt><samp class="option">b_qfactor</samp></dt> <dt><samp

class=”option”>b_qoffset</samp></dt> <dt><samp class=”option”>flags</samp></dt>
<dd> Set specific encoding flags. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>mv4</samp>’</dt> <dd> Use
four motion vector by macroblock. </dd> <dt>‘<samp
class=”samp”>aic</samp>’</dt> <dd> Enable high quality AC
prediction. </dd> <dt>‘<samp class=”samp”>gray</samp>’</dt>
<dd> Only encode grayscale. </dd> <dt>‘<samp
class=”samp”>qpel</samp>’</dt> <dd> Enable quarter-pixel
motion compensation. </dd> <dt>‘<samp class=”samp”>cgop</samp>’</dt>
<dd> Enable closed GOP. </dd> <dt>‘<samp class=”samp”>global_header</samp>&rsq
<dd> Place global headers in extradata instead of every keyframe.
</dd> </dl>

</dd> <dt><samp class=”option”>gmc</samp></dt> <dd> Enable
the use of global motion compensation (GMC). Default is 0 (disabled).
 </dd> <dt><samp class=”option”>me_quality</samp></dt>
<dd> Set motion estimation quality level. Possible values in decreasing
order of speed and increasing order of quality: <dl class=”table”>
<dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Use
no motion estimation (default). </dd> <dt>‘<samp
class=”samp”>1, 2</samp>’</dt> <dd> Enable advanced dia-
mond zonal search for 16x16 blocks and half-pixel refinement for 16x16 blocks.
 </dd> <dt>‘<samp class=”samp”>3, 4</samp>’</dt>
<dd> Enable all of the things described above, plus advanced diamond
zonal search for 8x8 blocks and half-pixel refinement for 8x8 blocks, also
enable motion estimation on chroma planes for P and B-frames.
</dd> <dt>‘<samp class=”samp”>5, 6</samp>’</dt>
<dd> Enable all of the things described above, plus extended 16x16 and
8x8 blocks search. </dd> </dl>

</dd> <dt><samp class=”option”>mbd</samp></dt> <dd> Set mac-
roblock decision algorithm. Possible values in the increasing order of quality:
 <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt>
<dd> Use macroblock comparing function algorithm (default).
</dd> <dt>‘<samp class=”samp”>bits</samp>’</dt>
<dd> Enable rate distortion-based half pixel and quarter pixel refinement
for 16x16 blocks. </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> Enable all of the things described above, plus rate distortion-based
half pixel and quarter pixel refinement for 8x8 blocks, and rate distortion-based
search using square pattern. </dd> </dl>

</dd> <dt><samp class=”option”>lumi_aq</samp></dt> <dd> Enable
lumi masking adaptive quantization when set to 1. Default is 0 (disabled).
 </dd> <dt><samp class=”option”>variance_aq</samp></dt>
<dd> Enable variance adaptive quantization when set to 1. Default is 0 (dis-
abled). When combined with <samp class=”option”>lumi_aq</samp>,
the resulting quality will not be better than any of the two specified individ-

ually. In other words, the resulting quality will be the worse one of the two effects. </dd> <dt><samp class="option">trellis</samp></dt> <dd> Set rate-distortion optimal quantization. </dd> <dt><samp class="option">ssim</samp></dt> <dd> Set structural similarity (SSIM) displaying method. Possible values: <dl class="table"> <dt>‘<samp class="samp">off</samp>’</dt> <dd> Disable displaying of SSIM information. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Output average SSIM at the end of encoding to stdout. The format of showing the average SSIM is: <pre class="example-preformatted">Average SSIM: %f </pre>

For users who are not familiar with C, %f means a float number, or a decimal (e.g. 0.939232). </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Output both per-frame SSIM data during encoding and average SSIM at the end of encoding to stdout. The format of per-frame information is: <pre class="example-preformatted">SSIM: avg: %1.3f min: %1.3f max: %1.3f </pre>

For users who are not familiar with C, %1.3f means a float number rounded to 3 digits after the dot (e.g. 0.932). </dd> </dl>

</dd> <dt><samp class="option">ssim_acc</samp></dt> <dd> Set SSIM accuracy. Valid options are integers within the range of 0-4, while 0 gives the most accurate result and 4 computes the fastest. </dd> </dl>

 <h3 class="section">9.25 MediaCodec# TOC</h3>

MediaCodec encoder wrapper enables hardware-accelerated video encoding on Android device. It supports H.264, H.265 (HEVC), VP8, VP9, MPEG-4, and AV1 encoding (whether works or not is device dependent). Android provides two sets of APIs: Java MediaCodec and NDK MediaCodec. The MediaCodec encoder wrapper supports both. Note that the NDK MediaCodec API operates without requiring JVM, but may fail to function outside the JVM environment due to dependencies on system framework services, particularly after Android 15. <dl class="table"> <dt><samp class="option">ndk_codec <var class="var">boolean</var></samp></dt> <dd> Use the NDK-based MediaCodec API instead of the Java API. Enabled by default if <code class="code">av_jni_get_java_vm()</code> return NULL. </dd> <dt><samp class="option">ndk_async <var class="var">boolean</var></samp></dt> <dd> Use NDK MediaCodec in async mode. Async mode has less overhead than poll in a loop in sync mode. The drawback of async mode is AV_CODEC_FLAG_GLOBAL_HEADER

doesn’t work (use extract_extradata bsf when necessary). It doesn’t work and will be disabled automatically on devices below Android 8.0. </dd> <dt><samp class="option">codec_name <var class="var">string</var></samp></dt> <dd> A codec type can have multiple implementations on a single device, this option specify which backend to use (via MediaCodec createCodecByName API). It’s NULL by default, and encoder is created by createEncoderByType. </dd> <dt><samp class="option">bitrate_mode <var class="var">integer</var></samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">cq</samp>’</dt> <dd> Constant quality mode </dd> <dt>‘<samp class="samp">vbr</samp>’</dt> <dd> Variable bitrate mode </dd> <dt>‘<samp class="samp">cbr</samp>’</dt> <dd> Constant bitrate mode </dd> <dt>‘<samp class="samp">cbr_fd</samp>’</dt> <dd> Constant bitrate mode with frame drops </dd> </dl>

</dd> <dt><samp class="option">pts_as_dts <var class="var">boolean</var></samp></dt> <dd> Use PTS as DTS. This is a workaround since MediaCodec API doesn’t provide decoding timestamp. It is enabled automatically if B frame is 0. </dd> <dt><samp class="option">operating_rate <var class="var">integer</var></samp></dt> <dd> The desired operating rate that the codec will need to operate at, zero for unspecified. This is used for cases like high-speed/slow-motion video capture, where the video encoder format contains the target playback rate (e.g. 30fps), but the component must be able to handle the high operating capture rate (e.g. 240fps). This rate will be used by codec for resource planning and setting the operating points. </dd> <dt><samp class="option">qp_i_min <var class="var">integer</var></samp></dt> <dd> Minimum quantization parameter for I frame. </dd> <dt><samp class="option">qp_p_min <var class="var">integer</var></samp></dt> <dd> Minimum quantization parameter for P frame. </dd> <dt><samp class="option">qp_b_min <var class="var">integer</var></samp></dt> <dd> Minimum quantization parameter for B frame. </dd> <dt><samp class="option">qp_i_max <var class="var">integer</var></samp></dt> <dd> Maximum quantization parameter for I frame. </dd> <dt><samp class="option">qp_p_max <var class="var">integer</var></samp></dt> <dd> Maximum quantization parameter for P frame. </dd> <dt><samp class="option">qp_b_max <var class="var">integer</var></samp></dt> <dd> Maximum quantization parameter for B frame. </dd> </dl>

 <h3 class="section">9.26 MediaFoundation# TOC</h3>

The following wrappers for encoders in the MediaFoundation frame-

work are available: <ul class="itemize mark-bullet"> h264_mf hevc_mf av1_mf

These support both software and hardware encoding. Video encoders can take input in either of nv12 or yuv420p form (some encoders support both, some support only either - in practice, nv12 is the safer choice, especially among HW encoders). Hardware-accelerated encoding requires D3D11, including hardware scaling capabilities through the scale_d3d11 filter. To list all available options for the MediaFoundation encoders, use: <code class="command">ffmpeg -h encoder=<encoder></code> e.g. <code class="command">ffmpeg -h encoder=h264_mf</code> <h4 class="subsection">9.26.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">rate_control</samp></dt> <dd> Select rate control mode. Available modes: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Default mode </dd> <dt>‘<samp class="samp">cbr</samp>’</dt> <dd> CBR mode </dd> <dt>‘<samp class="samp">pc_vbr</samp>’</dt> <dd> Peak constrained VBR mode </dd> <dt>‘<samp class="samp">u_vbr</samp>’</dt> <dd> Unconstrained VBR mode </dd> <dt>‘<samp class="samp">quality</samp>’</dt> <dd> Quality mode </dd> <dt>‘<samp class="samp">ld_vbr</samp>’</dt> <dd> Low delay VBR mode (requires Windows 8+) </dd> <dt>‘<samp class="samp">g_vbr</samp>’</dt> <dd> Global VBR mode (requires Windows 8+) </dd> <dt>‘<samp class="samp">gld_vbr</samp>’</dt> <dd> Global low delay VBR mode (requires Windows 8+) </dd> </dl>

</dd> <dt><samp class="option">scenario</samp></dt> <dd> Select usage scenario. Available scenarios: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Default scenario </dd> <dt>‘<samp class="samp">display_remoting</samp>’</dt> <dd> Display remoting scenario </dd> <dt>‘<samp class="samp">video_conference</samp>’</dt> <dd> Video conference scenario </dd> <dt>‘<samp class="samp">archive</samp>’</dt> <dd> Archive scenario </dd> <dt>‘<samp class="samp">live_streaming</samp>’ <dd> Live streaming scenario </dd> <dt>‘<samp class="samp">camera_record</samp>’</dt> <dd> Camera record scenario </dd> <dt>‘<samp class="samp">display_remoting_with_feature_map</san <dd> Display remoting with feature map scenario </dd> </dl>

</dd> <dt><samp class="option">quality</samp></dt> <dd> Set encoding quality (0-100). -1 means default quality. </dd> <dt><samp class="option">hw_encoding</samp></dt> <dd> Force hardware encoding (0-1). Default is 0 (disabled). </dd> </dl>

 <h4 class="subsection">9.26.2 Examples# TOC</h4>

Hardware encoding: <pre class="example-preformatted">ffmpeg -i input.mp4 -c:v h264_mf -hw_encoding 1 output.mp4 </pre>

Hardware-accelerated decoding with hardware encoding: <pre class="example-preformatted">ffmpeg -hwaccel d3d11va -i input.mp4 -c:v h264_mf -hw_encoding 1 output.mp4 </pre>

Hardware-accelerated decoding, HW scaling and encoding with quality setting: <pre class="example-preformatted">ffmpeg -hwaccel d3d11va -hwaccel_output_format d3d11 -i input.mp4 -vf scale_d3d11=1920:1080 -c:v hevc_mf -hw_encoding 1 -quality 80 output.mp4 </pre>

 <h3 class="section">9.27 Microsoft RLE# TOC</h3>

Microsoft RLE aka MSRLE encoder. Only 8-bit palette mode supported. Compatible with Windows 3.1 and Windows 95. <h4 class="subsection">9.27.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">g <var class="var">integer</var></samp></dt> <dd> Keyframe interval. A keyframe is inserted at least every <code class="code">-g</code> frames, sometimes sooner. </dd> </dl>

 <h3 class="section">9.28 mpeg2# TOC</h3>

MPEG-2 video encoder. <h4 class="subsection">9.28.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Select the mpeg2 profile to encode: <dl class="table"> <dt>‘<samp class="samp">422</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt> <dt>‘<samp class="samp">ss</samp>’</dt> <dd> Spatially Scalable </dd> <dt>‘<samp class="samp">snr</samp>’</dt>

<dd> SNR Scalable </dd> <dt>‘<samp class="samp">main</samp>’</dt>
<dt>‘<samp class="samp">simple</samp>’</dt> </dl>

</dd> <dt><samp class="option">level</samp></dt> <dd> Select the mpeg2 level to encode: <dl class="table"> <dt>‘<samp class="samp">high</samp>’</dt> <dt>‘<samp class="samp">high1440</samp>’<
<dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">low</samp>’</dt> </dl>

</dd> <dt><samp class="option">seq_disp_ext <var class="var">integer</var></samp></dt>
<dd> Specifies if the encoder should write a sequence_display_extension to the output. <dl class="table"> <dt><samp class="option">-1</samp></dt> <dt><samp class="option">auto</samp></dt>
<dd> Decide automatically to write it or not (this is the default) by checking if the data to be written is different from the default or unspecified values. </dd> <dt><samp class="option">0</samp></dt>
<dt><samp class="option">never</samp></dt> <dd> Never write it. </dd> <dt><samp class="option">1</samp></dt>
<dt><samp class="option">always</samp></dt> <dd> Always write it. </dd> </dl> </dd> <dt><samp class="option">video_format <var class="var">integer</var></samp></dt> <dd> Specifies the video_format written into the sequence display extension indicating the source of the video pictures. The default is ‘<samp class="samp">unspecified</samp>’, can be ‘<samp class="samp">component</samp>&rsqu
‘<samp class="samp">pal</samp>’, ‘<samp class="samp">ntsc</samp>’, ‘<samp class="samp">secam</samp>’ or ‘<samp class="samp">mac</samp>’. For maximum compatibility, use ‘<samp class="samp">component</samp>’. </dd>
<dt><samp class="option">a53cc <var class="var">boolean</var></samp></dt>
<dd> Import closed captions (which must be ATSC compatible format) into output. Default is 1 (on). </dd> </dl>

 <h3 class="section">9.29 NVENC encoders# TOC</h3>

The <code class="code">h264_nvenc</code>, <code class="code">hevc_nvenc</code> and <code class="code">av1_nvenc</code> encoders use NVIDIA NVENC hardware to encode H.264, HEVC and AV1 video, respectively. Building them requires the <code class="code">ffnvcodec</code> headers provided by nv-codec-headers. They are autodetected by configure; they can also be enabled explicitly with <samp class="option">--enable-ffnvcodec</samp> and <samp class="option">--enable-nvenc</samp>. Available options and values depend on the SDK version exposed by the headers at build time. Feature checks in <samp class="file">libavcodec/nvenc.h</samp> distinguish SDK versions 12.0, 12.1, 12.2, 13.0 and 13.1. Runtime support also

depends on the GPU and driver. Use <code class=”command”>ffmpeg -h encoder=hevc_nvenc</code>, or substitute <code class=”code”>h264_nvenc</code> or <code class=”code”>av1_nvenc</code>, to list the options, devices and pixel formats available in a particular build. <h4 class=”subsection”>9.29.1 Hardware input and devices# TOC</h4>

These encoders accept system-memory frames and CUDA hardware frames (<code class=”code”>AV_PIX_FMT_CUDA</code>). Builds using SDK 13.1 or newer also accept CUDA array frames (<code class=”code”>AV_PIX_FMT_CUARRAY</code>). Builds with <code class=”code”>d3d11va</code> support accept D3D11 hardware frames (<code class=”code”>AV_PIX_FMT_D3D11</code>). The input pixel format and encoding features must be supported by the selected GPU. For CUDA decoding followed by encoding, use the input options <samp class=”option”>-hwaccel cuda -hwaccel_output_format cuda</samp> to keep decoded frames on the device. A hardware device can also be created with <samp class=”option”>-init_hw_device</samp> and supplied to the encoder, directly or through its hardware frames context. For system-memory input without a supplied hardware device, the encoder option <samp class=”option”>gpu</samp> selects the device. See its entry under Other common options below. <h4 class=”subsection”>9.29.2 Generic options# TOC</h4>

Several generic encoding options are mapped to NVENC settings. The bitrate options <samp class=”option”>b</samp>, <samp class=”option”>maxrate</samp> and <samp class=”option”>bufsize</samp> set the average bitrate, the maximum bitrate and the VBV buffer size; the NVENC default for <samp class=”option”>b</samp> is ‘<samp class=”samp”>2M</samp>’. <samp class=”option”>g</samp> sets the GOP length, with 0 requesting intra-only encoding. <samp class=”option”>bf</samp> sets the number of B-frames between non-B-frames. <samp class=”option”>refs</samp> sets the number of reference frames in each reference list. <samp class=”option”>g</samp> and <samp class=”option”>bf</samp> default to <code class=”code”>-1</code> and <samp class=”option”>refs</samp> defaults to 0; these defaults leave the values of the preset unchanged. The generic <samp class=”option”>qmin</samp> and <samp class=”option”>qmax</samp> options are honoured in VBR mode with a warning; API users should set the encoder options of the same names listed below instead. Only applications calling the libavcodec API can change <code class=”code”>AVCodecContext.bit_rate</code>,

<code class=”code”>AVCodecContext.rc_max_rate</code> and <code class=”code”>AVCodecContext.rc_buffer_size</code> during encoding. The encoder checks these fields before encoding each frame. When dynamic bitrate changes are supported and rate control is not ‘<samp class=”samp”>constqp</samp>’, changed positive values trigger reconfiguration, reset the encoder and force an IDR. A change to the display aspect ratio also triggers reconfiguration. <h4 class=”subsection”>9.29.3 Common options# TOC</h4>

The following option names are shared by all three encoders. Differences in values and build requirements are noted below. Boolean options accept 0 (disabled) and 1 (enabled), unless otherwise stated. <h4 class=”subsubsection”>9.29.3.1 Presets and tuning# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>preset <var class=”var”>integer</var></samp></dt> <dd> Select the encoding preset using a named value from the table (recommended) or its integer. The range is <code class=”code”>0</code> to <code class=”code”>9</code>; the default is ‘<samp class=”samp”>p4</samp>’ (6). <dl class=”table”> <dt>‘<samp class=”samp”>slow</samp>’</dt> <dd> Deprecated alias for high quality with two passes (0). </dd> <dt>‘<samp class=”samp”>medium</samp>’</dt> <dd> Deprecated alias for high quality with one pass (1). </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dd> Deprecated alias for high performance with one pass (2). </dd> <dt>‘<samp class=”samp”>p1</samp>’</dt> <dd> Fastest, lowest quality (3). </dd> <dt>‘<samp class=”samp”>p2</samp>’</dt> <dd> Faster, lower quality (4). </dd> <dt>‘<samp class=”samp”>p3</samp>’</dt> <dd> Fast, low quality (5). </dd> <dt>‘<samp class=”samp”>p4</samp>’</dt> <dd> Medium (6); the default. </dd> <dt>‘<samp class=”samp”>p5</samp>’</dt> <dd> Slow, good quality (7). </dd> <dt>‘<samp class=”samp”>p6</samp>’</dt> <dd> Slower, better quality (8). </dd> <dt>‘<samp class=”samp”>p7</samp>’</dt> <dd> Slowest, best quality (9). </dd> </dl>

Use ‘<samp class=”samp”>p1</samp>’ through ‘<samp class=”samp”>p7</samp>’ with <samp class=”option”>tune</samp> and <samp class=”option”>multipass</samp> instead of the deprecated aliases. </dd> <dt><samp class=”option”>tune <var class=”var”>integer</var></samp></dt> <dd> Select encoding

tuning using a named value from the table (recommended) or its integer. The default is ‘<samp class="samp">hq</samp>’ (1). H.264 accepts <code class="code">1</code> to <code class="code">4</code>; HEVC and AV1 also expose ‘<samp class="samp">uhq</samp>’ when built with the required SDK. <dl class="table"> <dt>‘<samp class="samp">hq</samp>’</dt> <dd> High quality (1). </dd> <dt>‘<samp class="samp">ll</samp>’</dt> <dd> Low latency (2). </dd> <dt>‘<samp class="samp">ull</samp>’</dt> <dd> Ultra low latency (3). </dd> <dt>‘<samp class="samp">lossless</samp>’</dt> <dd> Lossless (4). </dd> <dt>‘<samp class="samp">uhq</samp>’</dt> <dd> Ultra high quality (5). HEVC requires SDK 12.2 or newer; AV1 requires SDK 13.0 or newer. Not available for H.264. </dd> </dl> </dd> </dl>

 <h4 class="subsubsection">9.29.3.2 Rate control# TOC</h4>

<dl class="table"> <dt><samp class="option">rc <var class="var">integer</var></samp></dt> <dd> Override preset rate control using a named value from the table (recommended) or its integer. The range is <code class="code">-1</code> to <code class="code">INT_MAX</code>; the default is <code class="code">-1</code> (no explicit override). <dl class="table"> <dt>‘<samp class="samp">constqp</samp>’</dt> <dd> Constant QP (0). </dd> <dt>‘<samp class="samp">vbr</samp>’</dt> <dd> Variable bitrate (1). </dd> <dt>‘<samp class="samp">cbr</samp>’</dt> <dd> Constant bitrate (2). </dd> </dl>

</dd> <dt><samp class="option">multipass <var class="var">integer</var></samp></dt> <dd> Select multipass encoding using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">2</code>; the default is ‘<samp class="samp">disabled</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Single pass (0). </dd> <dt>‘<samp class="samp">qres</samp>’</dt> <dd> Two passes, with the first pass at quarter resolution (1). </dd> <dt>‘<samp class="samp">fullres</samp>’</dt> <dd> Two passes, with the first pass at full resolution (2). </dd> </dl>

</dd> <dt><samp class="option">cq <var class="var">float</var></samp></dt> <dd> Set the target quality for constant quality mode in VBR rate control. The default is 0 (automatic). The range is <code class="code">0</code> to <code class="code">51</code> for H.264 and HEVC, or <code class="code">0</code> to <code class="code">63</code> for AV1. </dd> <dt><samp class="option">qp <var class="var">integer</var></samp></dt> <dd> Set the constant quantization parameter. The default is <code class="code">-1</code> (unset). The range is <code

class=”code”>-1</code> to <code class=”code”>51</code> for H.264 and HEVC, or <code class=”code”>-1</code> to <code class=”code”>255</code> for AV1. </dd> <dt><samp class=”option”>init_qpP <var class=”var”>integer</var></samp></dt> <dt><samp class=”option”>init_qpB <var class=”var”>integer</var></samp></dt> <dt><samp class=”option”>init_qpI <var class=”var”>integer</var></samp></dt> <dd> Set the initial QP for P-, B- and I-frames, respectively. Each defaults to <code class=”code”>-1</code> (unset). The range is <code class=”code”>-1</code> to <code class=”code”>51</code> for H.264 and HEVC, or <code class=”code”>-1</code> to <code class=”code”>255</code> for AV1. </dd> <dt><samp class=”option”>qmin <var class=”var”>integer</var></samp></dt> <dt><samp class=”option”>qmax <var class=”var”>integer</var></samp></dt> <dd> Set the minimum and maximum QP used for rate control. Each defaults to <code class=”code”>-1</code> (unset). The range is <code class=”code”>-1</code> to <code class=”code”>51</code> for H.264 and HEVC, or <code class=”code”>-1</code> to <code class=”code”>255</code> for AV1. </dd> <dt><samp class=”option”>qp_cb_offset <var class=”var”>integer</var></samp></dt> <dt><samp class=”option”>qp_cr_offset <var class=”var”>integer</var></samp></dt> <dd> Set the chroma QP offsets for the Cb and Cr channels, respectively. Each accepts <code class=”code”>-12</code> to <code class=”code”>12</code> and defaults to 0. </dd> <dt><samp class=”option”>strict_gop <var class=”var”>boolean</var></samp></dt> <dd> Minimize GOP-to-GOP rate fluctuations. The default is 0. </dd> <dt><samp class=”option”>ldkfs <var class=”var”>integer</var></samp></dt> <dd> Set the low-delay keyframe scale, which specifies the scene-change frame size increase allowed with a single-frame VBV buffer and CBR. The range is <code class=”code”>0</code> to <code class=”code”>255</code> and the default is 0. </dd> <dt><samp class=”option”>cbr_padding <var class=”var”>boolean</var></samp></dt> <dd> Pad the bitstream to prevent the bitrate falling below the target in CBR mode. The default is 0. </dd> </dl>

 <h4 class=”subsubsection”>9.29.3.3 Lookahead, adaptive quantization and B-frames# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>rc-lookahead <var class=”var”>integer</var></samp></dt> <dd> Set the number of frames to look ahead for rate control. The range is <code class=”code”>0</code> to <code class=”code”>INT_MAX</code> and the default is 0. </dd> <dt><samp class=”option”>lookahead_level <var class=”var”>integer</var></samp></dt> <dd> Set the lookahead level using a named value from the table (recommended) or its integer.

Higher levels may improve quality at the expense of performance. Requires SDK 12.2 or newer. The range is <code class=”code”>-1</code> to <code class=”code”>15</code>; the default is <code class=”code”>-1</code> (no override). <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dt>‘<samp class=”samp”>1</samp>’</dt> <dt>‘<samp class=”samp”>2</samp>’</dt> <dt>‘<samp class=”samp”>3</samp>’</dt> <dd> Select the corresponding lookahead level. </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection (15). </dd> </dl>

</dd> <dt><samp class=”option”>no-scenecut <var class=”var”>boolean</var></samp></dt> <dd> Disable adaptive I-frame insertion at scene cuts when lookahead is enabled. The default is 0. </dd> <dt><samp class=”option”>b_adapt <var class=”var”>boolean</var></samp></dt> <dd> Enable adaptive B-frame decisions when lookahead is enabled. The default is 1. </dd> <dt><samp class=”option”>spatial-aq <var class=”var”>boolean</var></samp></dt> <dd> Enable spatial adaptive quantization. The default is 0. </dd> <dt><samp class=”option”>temporal-aq <var class=”var”>boolean</var></samp></dt> <dd> Enable temporal adaptive quantization. The default is 0. </dd> <dt><samp class=”option”>aq-strength <var class=”var”>integer</var></samp></dt> <dd> Set spatial adaptive quantization strength when <samp class=”option”>spatial-aq</samp> is enabled, from <code class=”code”>1</code> (low) to <code class=”code”>15</code> (aggressive). The default is 8. </dd> <dt><samp class=”option”>b_ref_mode <var class=”var”>integer</var></samp></dt> <dd> Select the use of B-frames as references using a named value from the table (recommended) or its integer. The default is <code class=”code”>-1</code> (no override). H.264 and HEVC accept <code class=”code”>-1</code> to <code class=”code”>2</code>. AV1 accepts <code class=”code”>-1</code> to <code class=”code”>4</code> with SDK 13.1 or newer, or <code class=”code”>-1</code> to <code class=”code”>2</code> with older headers. <dl class=”table”> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Do not use B-frames as references (0). </dd> <dt>‘<samp class=”samp”>each</samp>’</dt> <dd> Use each B-frame as a reference (1). </dd> <dt>‘<samp class=”samp”>middle</samp>’</dt> <dd> For H.264 and HEVC, use only half the B-frames as references (2). For AV1, use every other B-frame as an Altref2 reference (2), except the last in the Altref interval. </dd> <dt>‘<samp class=”samp”>hierarchical</samp>’</dt> <dd> Use B-frames as references hierarchically (4). AV1 only; requires SDK 13.1 or newer. </dd> </dl>

</dd> <dt><samp class=”option”>weighted_pred <var class=”var”>integer</var></samp></dt> <dd> Enable weighted prediction with 1, or disable it with 0 (the default). This is a boolean option for AV1. </dd> <dt><samp class=”option”>tf_level <var class=”var”>integer</var></samp></dt> <dd> Set temporal filtering strength. The range is <code class=”code”>-

1</code> to <code class=”code”>INT_MAX</code>; the default is <code class=”code”>-1</code> (no override). Requires SDK 12.2 or newer for HEVC, or SDK 13.0 or newer for H.264 and AV1. <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dt>‘<samp class=”samp”>4</samp>’</dt> <dd> Select the corresponding temporal filtering strength. </dd> </dl> </dd> </dl>

 <h4 class=”subsubsection”>9.29.3.4 SEI and side data# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>extra_sei <var class=”var”>boolean</var></samp></dt>
<dd> Pass extra frame side data. The default is 1. H.264 and HEVC use SEI payloads; AV1 uses metadata OBUs. This gates the payload array used for <samp class=”option”>a53cc</samp>, HEVC and AV1 <samp class=”option”>s12m_tc</samp>, and H.264 and HEVC <samp class=”option”>udu_sei</samp>. </dd> <dt><samp class=”option”>a53cc <var class=”var”>boolean</var></samp></dt> <dd> Use <code class=”code”>AV_FRAME_DATA_A53_CC</code> side data for A53 closed captions when available. Requires <samp class=”option”>extra_sei</samp>. The default is 1. H.264 and HEVC carry it in ITU-T T.35 registered user-data SEI; AV1 uses an ITU-T T.35 metadata OBU. </dd> <dt><samp class=”option”>s12m_tc <var class=”var”>boolean</var></samp></dt> <dd> Use <code class=”code”>AV_FRAME_DATA_S12M_TIMECODE</code> side data when available. The default is 1. H.264 uses picture timing and requires SDK 12.2 or newer; HEVC uses timecode SEI; AV1 uses timecode metadata OBUs. The HEVC and AV1 paths require <samp class=”option”>extra_sei</samp>; the H.264 path is independent of it. </dd> </dl>

With SDK 13.0 or newer, HEVC and AV1 also support mastering-display and content-light-level metadata. The corresponding <code class=”code”>AV_FRAME_DATA_MASTERING_DISPLAY_METADATA</code> and <code class=”code”>AV_FRAME_DATA_CONTENT_LIGHT_LEVEL</code> entries in <code class=”code”>AVCodecContext.decoded_side_data</code> enable output at initialization. Frame side data supplies per-frame values, falling back to those stream values when absent. This path is independent of <samp class=”option”>extra_sei</samp>. <h4 class=”subsubsection”>9.29.3.5 Other common options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>level <var class=”var”>integer</var></samp></dt>

<dd> Set the encoding level restriction using a named value (recommended) or its integer. The tables below list the values, numeric ranges and default integers for each codec. The default is ‘<samp class="samp">auto</samp>’ (0 for H.264 and HEVC, 24 for AV1). </dd> <dt><samp class="option">gpu <var class="var">integer</var></samp></dt> <dd> Select the NVENC-capable GPU for system-memory input without a supplied hardware device using a named value from the table (recommended) or its integer. Device indices start at 0. The range is <code class="code">-2</code> to <code class="code">INT_MAX</code> and the default is ‘<samp class="samp">any</samp>’ (-1). <dl class="table"> <dt>‘<samp class="samp">list</samp>’</dt> <dd> List the available devices (-2). </dd> <dt>‘<samp class="samp">any</samp>’</dt> <dd> Use the first available device (-1). </dd> </dl>

</dd> <dt><samp class="option">surfaces <var class="var">integer</var></samp></dt> <dd> Set the number of concurrent surfaces, from <code class="code">0</code> to <code class="code">64</code>. The default is 0 (automatic). </dd> <dt><samp class="option">delay <var class="var">integer</var></samp></dt> <dd> Delay frame output by the specified number of frames, from <code class="code">0</code> to <code class="code">INT_MAX</code>. The default is <code class="code">INT_MAX</code>; the actual delay is limited by the number of surfaces. </dd> <dt><samp class="option">rgb_mode <var class="var">integer</var></samp></dt> <dd> Select how packed RGB input is handled using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">INT_MAX</code>; the default is ‘<samp class="samp">yuv420</samp>’ (1). <dl class="table"> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Reject packed RGB input (0). </dd> <dt>‘<samp class="samp">yuv420</samp>’</dt> <dd> Convert to YUV 4:2:0 (1). </dd> <dt>‘<samp class="samp">yuv444</samp>’</dt> <dd> Convert to YUV 4:4:4 (2). </dd> </dl>

</dd> <dt><samp class="option">highbitdepth <var class="var">boolean</var></samp></dt> <dd> Enable 10-bit encoding for 8-bit input. The default is 0. For H.264 this requires SDK 13.0 or newer. </dd> <dt><samp class="option">forced-idr <var class="var">boolean</var></samp></dt> <dd> When forcing keyframes, force IDR frames. The range is <code class="code">-1</code> to <code class="code">1</code> and the default is 0. A value of <code class="code">-1</code> leaves the picture-type decision to the encoder. </dd> <dt><samp class="option">zerolatency <var class="var">boolean</var></samp></dt> <dd> Request zero-latency operation with no reordering delay. The default is 0. </dd> <dt><samp class="option">nonref_p <var class="var">boolean</var></samp></dt> <dd> Enable automatic insertion of non-reference P-frames. The default is 0. </dd> <dt><samp class="option">dpb_size

<var class=”var”>integer</var></samp></dt> <dd> Set the decoded picture buffer size, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>. The default is 0 (automatic). </dd> <dt><samp class=”option”>intra-refresh <var class=”var”>boolean</var></samp></dt> <dd> Use periodic intra refresh instead of IDR frames. The default is 0. </dd> </dl>

 <h4 class=”subsection”>9.29.4 H.264 and HEVC options# TOC</h4>

The following options are available for <code class=”code”>h264_nvenc</code> and <code class=”code”>hevc_nvenc</code> only. <dl class=”table”> <dt><samp class=”option”>aud <var class=”var”>boolean</var></samp></dt> <dd> Use access unit delimiters. The default is 0. </dd> <dt><samp class=”option”>bluray-compat <var class=”var”>boolean</var></samp></dt> <dd> Enable Blu-ray compatibility workarounds. The default is 0. </dd> <dt><samp class=”option”>udu_sei <var class=”var”>boolean</var></samp></dt> <dd> Pass <code class=”code”>AV_FRAME_DATA_SEI_UNREGISTERED</code> frame side data as unregistered user-data SEI. Requires <samp class=”option”>extra_sei</samp>. The default is 0. </dd> <dt><samp class=”option”>single-slice-intra-refresh <var class=”var”>boolean</var></samp></dt> <dd> Use single-slice intra refresh. This also enables <samp class=”option”>intra-refresh</samp>. The default is 0. </dd> <dt><samp class=”option”>max_slice_size <var class=”var”>integer</var></samp></dt> <dd> Set the maximum encoded slice size in bytes, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>. The default is 0 (no override). </dd> <dt><samp class=”option”>constrained-encoding <var class=”var”>boolean</var></samp></dt> <dd> Enable constrained-frame encoding, making each slice in a constrained picture independent of the other slices. The default is 0. </dd> </dl>

 <h4 class=”subsection”>9.29.5 HEVC and AV1 options# TOC</h4>

The following option is available for <code class=”code”>hevc_nvenc</code> and <code class=”code”>av1_nvenc</code> only. <dl class=”table”> <dt><samp class=”option”>split_encode_mode <var class=”var”>integer</var></samp></dt> <dd> Set split-frame encoding mode using a named value from the table (recommended) or its integer. Requires SDK 12.1 or newer. The range is <code class=”code”>0</code> to <code class=”code”>15</code> and the default is ‘<samp class=”samp”>auto</samp>’ (0). <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>

<dd> Enable or disable it according to the preset and tuning (0).
</dd> <dt>‘<samp class="samp">forced</samp>’</dt>
<dd> Enable it with the number of horizontal strips selected by the driver
(1). </dd> <dt>‘<samp class="samp">2</samp>’</dt>
<dd> Force two horizontal strips when more than one NVENC engine is
available (2). </dd> <dt>‘<samp class="samp">3</samp>’</dt>
<dd> Force three horizontal strips when more than two NVENC engines are
available (3). </dd> <dt>‘<samp class="samp">4</samp>’</dt>
<dd> Force four horizontal strips when more than three NVENC
engines are available (4). Requires SDK 13.0 or newer. </dd>
<dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Disable
split encoding (15). </dd> </dl> </dd> </dl>

 <h4 class="subsection">9.29.6
H.264 options<a class="anchor hidden-xs"
href="#H_002e264-options" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-H_002e264-options" aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">profile <var class="var">integer</var></samp></dt>
<dd> Set the H.264 profile using a named value from the table (recommended) or its integer. The default is ‘<samp class="samp">high</samp>’
(2). <dl class="table"> <dt>‘<samp class="samp">baseline</samp>’</dt>
<dd> baseline profile (0). </dd> <dt>‘<samp class="samp">main</samp>’</dt>
<dd> main profile (1). </dd> <dt>‘<samp class="samp">high</samp>’</dt>
<dd> high profile (2). </dd> <dt>‘<samp class="samp">high10</samp>’</dt>
<dd> high10 profile (3). </dd> <dt>‘<samp class="samp">high422</samp>’</dt
<dd> high422 profile (4). </dd> <dt>‘<samp class="samp">high444p</samp>’</
<dd> high444p profile (5). </dd> </dl>

The ‘<samp class="samp">high10</samp>’ and
‘<samp class="samp">high422</samp>’ values require
SDK 13.0 or newer. With headers older than SDK 13.0, ‘<samp
class="samp">high444p</samp>’ has integer value 3 instead of 5.
 </dd> <dt><samp class="option">level <var class="var">integer</var></samp></dt>
<dd> Set the H.264 level restriction using a named value from the table
(recommended) or its integer. The range is <code class="code">0</code> to
<code class="code">62</code>; the default is ‘<samp class="samp">auto</samp>’
(0). <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Automatic selection (0). </dd> <dt>‘<samp
class="samp">1b</samp>’</dt> <dd> Level 1b (9). </dd>
<dt>‘<samp class="samp">1.0b</samp>’</dt> <dd> Level
1.0b (9). </dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> Level 1 (10). </dd> <dt>‘<samp class="samp">1.0</samp>’</dt>
<dd> Level 1.0 (10). </dd> <dt>‘<samp class="samp">1.1</samp>’</dt>
<dd> Level 1.1 (11). </dd> <dt>‘<samp class="samp">1.2</samp>’</dt>
<dd> Level 1.2 (12). </dd> <dt>‘<samp class="samp">1.3</samp>’</dt>
<dd> Level 1.3 (13). </dd> <dt>‘<samp class="samp">2</samp>’</dt>

<dd> Level 2 (20). </dd> <dt>‘<samp class="samp">2.0</samp>’</dt>
<dd> Level 2.0 (20). </dd> <dt>‘<samp class="samp">2.1</samp>’</dt>
<dd> Level 2.1 (21). </dd> <dt>‘<samp class="samp">2.2</samp>’</dt>
<dd> Level 2.2 (22). </dd> <dt>‘<samp class="samp">3</samp>’</dt>
<dd> Level 3 (30). </dd> <dt>‘<samp class="samp">3.0</samp>’</dt>
<dd> Level 3.0 (30). </dd> <dt>‘<samp class="samp">3.1</samp>’</dt>
<dd> Level 3.1 (31). </dd> <dt>‘<samp class="samp">3.2</samp>’</dt>
<dd> Level 3.2 (32). </dd> <dt>‘<samp class="samp">4</samp>’</dt>
<dd> Level 4 (40). </dd> <dt>‘<samp class="samp">4.0</samp>’</dt>
<dd> Level 4.0 (40). </dd> <dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> Level 4.1 (41). </dd> <dt>‘<samp class="samp">4.2</samp>’</dt>
<dd> Level 4.2 (42). </dd> <dt>‘<samp class="samp">5</samp>’</dt>
<dd> Level 5 (50). </dd> <dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> Level 5.0 (50). </dd> <dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> Level 5.1 (51). </dd> <dt>‘<samp class="samp">5.2</samp>’</dt>
<dd> Level 5.2 (52). </dd> <dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> Level 6.0 (60). </dd> <dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> Level 6.1 (61). </dd> <dt>‘<samp class="samp">6.2</samp>’</dt>
<dd> Level 6.2 (62). </dd> </dl>

</dd> <dt><samp class="option">coder <var class="var">integer</var></samp></dt>
<dd> Set the entropy coder using a named value from the table (recommended) or its integer. The range is <code class="code">-1</code> to <code class="code">2</code>; the default is ‘<samp class="samp">default</samp>’ (-1). <dl class="table">
<dt>‘<samp class="samp">default</samp>’</dt> <dd> Leave the preset entropy coding mode unchanged (-1). </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection (0). </dd> <dt>‘<samp class="samp">cabac</samp>’</dt>
<dt>‘<samp class="samp">ac</samp>’</dt> <dd> CABAC (1). </dd> <dt>‘<samp class="samp">cavlc</samp>’</dt>
<dt>‘<samp class="samp">vlc</samp>’</dt> <dd> CAVLC (2). </dd> </dl> </dd> </dl>

 <h4 class="subsection">9.29.7 HEVC options# TOC</h4>

<dl class="table"> <dt><samp class="option">profile <var class="var">integer</var></samp></dt>
<dd> Set the HEVC profile using a named value from the table (recommended) or its integer. The default is ‘<samp class="samp">main</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt>
<dd> main profile (0). </dd> <dt>‘<samp class="samp">main10</samp>’</dt>
<dd> main10 profile (1). </dd> <dt>‘<samp class="samp">rext</samp>’</dt>
<dd> rext profile (2). </dd> <dt>‘<samp class="samp">mv</samp>’</dt>
<dd> mv profile (3). </dd> </dl>

The ‘<samp class="samp">mv</samp>’ value requires SDK 13.0 or newer. </dd> <dt><samp class="option">level <var class="var">integer</var></samp></dt> <dd> Set the HEVC level restriction using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">186</code>; the default is ‘<samp class="samp">auto</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection (0). </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> Level 1 (30). </dd> <dt>‘<samp class="samp">1.0</samp>’</dt> <dd> Level 1.0 (30). </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> Level 2 (60). </dd> <dt>‘<samp class="samp">2.0</samp>’</dt> <dd> Level 2.0 (60). </dd> <dt>‘<samp class="samp">2.1</samp>’</dt> <dd> Level 2.1 (63). </dd> <dt>‘<samp class="samp">3</samp>’</dt> <dd> Level 3 (90). </dd> <dt>‘<samp class="samp">3.0</samp>’</dt> <dd> Level 3.0 (90). </dd> <dt>‘<samp class="samp">3.1</samp>’</dt> <dd> Level 3.1 (93). </dd> <dt>‘<samp class="samp">4</samp>’</dt> <dd> Level 4 (120). </dd> <dt>‘<samp class="samp">4.0</samp>’</dt> <dd> Level 4.0 (120). </dd> <dt>‘<samp class="samp">4.1</samp>’</dt> <dd> Level 4.1 (123). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> Level 5 (150). </dd> <dt>‘<samp class="samp">5.0</samp>’</dt> <dd> Level 5.0 (150). </dd> <dt>‘<samp class="samp">5.1</samp>’</dt> <dd> Level 5.1 (153). </dd> <dt>‘<samp class="samp">5.2</samp>’</dt> <dd> Level 5.2 (156). </dd> <dt>‘<samp class="samp">6</samp>’</dt> <dd> Level 6 (180). </dd> <dt>‘<samp class="samp">6.0</samp>’</dt> <dd> Level 6.0 (180). </dd> <dt>‘<samp class="samp">6.1</samp>’</dt> <dd> Level 6.1 (183). </dd> <dt>‘<samp class="samp">6.2</samp>’</dt> <dd> Level 6.2 (186). </dd> </dl>

</dd> <dt><samp class="option">tier <var class="var">integer</var></samp></dt> <dd> Set the HEVC encoding tier using a named value from the table (recommended) or its integer. The range is <code class="code">0</code> to <code class="code">1</code>; the default is ‘<samp class="samp">main</samp>’ (0). <dl class="table"> <dt>‘<samp class="samp">main</samp>’</dt> <dd> Main tier (0). </dd> <dt>‘<samp class="samp">high</samp>’</dt> <dd> High tier (1). </dd> </dl>

</dd> <dt><samp class="option">unidir_b <var class="var">boolean</var></samp></dt> <dd> Enable unidirectional B-frames. Requires SDK 12.2 or newer. The default is 0. </dd> </dl>

 <h4 class="subsection">9.29.8 AV1 options# TOC</h4>

<dl class="table">

<dt><samp class=”option”>level <var class=”var”>integer</var></samp></dt>

<dd> Set the AV1 level restriction using a named value from the table

(recommended) or its integer. The range is <code class=”code”>0</code> to
<code class=”code”>24</code>; the

default is ‘<samp class=”samp”>auto</samp>’ (24).

<dl class=”table”>

<dt>‘<samp class=”samp”>2</samp>’</dt>

<dd> Level 2 (0).

</dd>

<dt>‘<samp class=”samp”>2.0</samp>’</dt>

<dd> Level 2.0 (0).

</dd>

<dt>‘<samp class=”samp”>2.1</samp>’</dt>

<dd> Level 2.1 (1).

</dd>

<dt>‘<samp class=”samp”>2.2</samp>’</dt>

<dd> Level 2.2 (2).

</dd>

<dt>‘<samp class=”samp”>2.3</samp>’</dt>

<dd> Level 2.3 (3).

</dd>

<dt>‘<samp class=”samp”>3</samp>’</dt>

<dd> Level 3 (4).

</dd>

<dt>‘<samp class=”samp”>3.0</samp>’</dt>

<dd> Level 3.0 (4).

</dd>

<dt>‘<samp class=”samp”>3.1</samp>’</dt>

<dd> Level 3.1 (5).

</dd>

<dt>‘<samp class="samp">3.2</samp>’</dt>
<dd> Level 3.2 (6).
</dd>
<dt>‘<samp class="samp">3.3</samp>’</dt>
<dd> Level 3.3 (7).
</dd>
<dt>‘<samp class="samp">4</samp>’</dt>
<dd> Level 4 (8).
</dd>
<dt>‘<samp class="samp">4.0</samp>’</dt>
<dd> Level 4.0 (8).
</dd>
<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> Level 4.1 (9).
</dd>
<dt>‘<samp class="samp">4.2</samp>’</dt>
<dd> Level 4.2 (10).
</dd>
<dt>‘<samp class="samp">4.3</samp>’</dt>
<dd> Level 4.3 (11).
</dd>
<dt>‘<samp class="samp">5</samp>’</dt>
<dd> Level 5 (12).
</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> Level 5.0 (12).
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> Level 5.1 (13).
</dd>
<dt>‘<samp class="samp">5.2</samp>’</dt>

<dd> Level 5.2 (14).
</dd>
<dt>‘<samp class="samp">5.3</samp>’</dt>
<dd> Level 5.3 (15).
</dd>
<dt>‘<samp class="samp">6</samp>’</dt>
<dd> Level 6 (16).
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> Level 6.0 (16).
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> Level 6.1 (17).
</dd>
<dt>‘<samp class="samp">6.2</samp>’</dt>
<dd> Level 6.2 (18).
</dd>
<dt>‘<samp class="samp">6.3</samp>’</dt>
<dd> Level 6.3 (19).
</dd>
<dt>‘<samp class="samp">7</samp>’</dt>
<dd> Level 7 (20).
</dd>
<dt>‘<samp class="samp">7.0</samp>’</dt>
<dd> Level 7.0 (20).
</dd>
<dt>‘<samp class="samp">7.1</samp>’</dt>
<dd> Level 7.1 (21).
</dd>
<dt>‘<samp class="samp">7.2</samp>’</dt>
<dd> Level 7.2 (22).

</dd>

<dt>‘<samp class="samp">7.3</samp>’</dt>

<dd> Level 7.3 (23).

</dd>

<dt>‘<samp class="samp">auto</samp>’</dt>

<dd> Automatic selection (24).

</dd>

</dl>

</dd> <dt><samp class="option">tier <var class="var">integer</var></samp></dt>
<dd> Set the AV1 encoding tier, from <code class="code">0</code> to
<code class="code">1</code>. The default is 0. <dl class="table">
<dt>‘<samp class="samp">0</samp>’</dt> <dd> Tier
0. </dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> Tier 1. </dd> </dl>

</dd> <dt><samp class="option">tile-rows <var class="var">integer</var></samp></dt>
<dt><samp class="option">tile-columns <var class="var">integer</var></samp></dt>
<dd> Set the number of tile rows and columns, respectively. Each accepts <code class="code">-1</code> to <code class="code">64</code>
and defaults to <code class="code">-1</code> (no override). </dd>
<dt><samp class="option">timing-info <var class="var">boolean</var></samp></dt>
<dd> Include timing information in sequence and frame headers. The
default is 0. </dd> </dl>

 <h3 class="section">9.30 png<span class="pull-
right">#
TOC</h3>

PNG image encoder. <h4
class="subsection">9.30.1 Options<a class="anchor
hidden-xs" href="#Options-52" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-Options-52" aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">compression_level</samp></dt>
<dd> Sets the compression level, from 0 to 9(default) </dd> </dl>

 <h4 class="subsection">9.30.2
Private options<a class="anchor hidden-xs"
href="#Private-options-1" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-Private-options-1" aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">dpi <var class="var">integer</var></samp></dt>
<dd> Set physical density of pixels, in dots per inch, unset by default
</dd> <dt><samp class="option">dpm <var class="var">integer</var></samp></dt>
<dd> Set physical density of pixels, in dots per meter, unset by default

</dd> <dt><samp class="option">pred <var class="var">method</var></samp></dt>
<dd> Set prediction method (none, sub, up, avg, paeth, mixed), default is paeth </dd> </dl>

 <h3 class="section">9.31 ProRes# TOC</h3>

Apple ProRes encoder. FFmpeg contains 2 ProRes encoders, the prores-aw and prores-ks encoder. The used encoder can be chosen with the <code class="code">-vcodec</code> option. <h4 class="subsection">9.31.1 Private Options for prores-ks# TOC</h4>

<dl class="table"> <dt><samp class="option">profile <var class="var">integer</var></samp></dt>
<dd> Select the ProRes profile to encode <dl class="table">
<dt>‘<samp class="samp">proxy</samp>’</dt> <dt>‘<samp class="samp">lt</samp>’</dt> <dt>‘<samp class="samp">standard</samp>’</dt>
<dt>‘<samp class="samp">hq</samp>’</dt> <dt>‘<samp class="samp">4444</samp>’</dt> <dt>‘<samp class="samp">4444xq</samp>’</dt>
</dl>

</dd> <dt><samp class="option">quant_mat <var class="var">integer</var></samp></dt>
<dd> Select quantization matrix. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">default</samp>’</dt>
<dt>‘<samp class="samp">proxy</samp>’</dt> <dt>‘<samp class="samp">lt</samp>’</dt> <dt>‘<samp class="samp">standard</samp>’</dt>
<dt>‘<samp class="samp">hq</samp>’</dt> </dl> If set to <var class="var">auto</var>, the matrix matching the profile will be picked. If not set, the matrix providing the highest quality, <var class="var">default</var>, will be picked. </dd> <dt><samp class="option">bits_per_mb <var class="var">integer</var></samp></dt>
<dd> How many bits to allot for coding one macroblock. Different profiles use between 200 and 2400 bits per macroblock, the maximum is 8000. </dd> <dt><samp class="option">mbs_per_slice <var class="var">integer</var></samp></dt> <dd> Number of macroblocks in each slice (1-8); the default value (8) should be good in almost all situations. </dd> <dt><samp class="option">vendor <var class="var">string</var></samp></dt> <dd> Override the 4-byte vendor ID. A custom vendor ID like <var class="var">apl0</var> would claim the stream was produced by the Apple encoder. </dd> <dt><samp class="option">alpha_bits <var class="var">integer</var></samp></dt>
<dd> Specify number of bits for alpha component. Possible values are <var class="var">0</var>, <var class="var">8</var> and <var

class=”var”>16</var>. Use <var class=”var”>0</var> to disable alpha plane coding. </dd> </dl>

 <h4 class=”subsection”>9.31.2 Speed considerations# TOC</h4>

In the default mode of operation the encoder has to honor frame constraints (i.e. not produce frames with size bigger than requested) while still making output picture as good as possible. A frame containing a lot of small details is harder to compress and the encoder would spend more time searching for appropriate quantizers for each slice. Setting a higher <samp class=”option”>bits_per_mb</samp> limit will improve the speed. For the fastest encoding speed set the <samp class=”option”>qscale</samp> parameter (4 is the recommended value) and do not set a size constraint. <h3 class=”section”>9.32 QSV Encoders# TOC</h3>

The family of Intel QuickSync Video encoders (MPEG-2, H.264, HEVC, JPEG/MJPEG, VP9, AV1) <h4 class=”subsection”>9.32.1 Ratecontrol Method# TOC</h4> The ratecontrol method is selected as follows: <ul class=”itemize mark-bullet”> When <samp class=”option”>global_quality</samp> is specified, a quality-based mode is used. Specifically this means either <ul class=”itemize mark-minus”> <var class=”var”>CQP</var> - constant quantizer scale, when the <samp class=”option”>qscale</samp> codec flag is also set (the <samp class=”option”>-qscale</samp> ffmpeg option).

<var class=”var”>LA_ICQ</var> - intelligent constant quality with lookahead, when the <samp class=”option”>look_ahead</samp> option is also set.

<var class=”var”>ICQ</var> – intelligent constant quality otherwise. For the ICQ modes, global quality range is 1 to 51, with 1 being the best quality.

Otherwise when the desired average bitrate is specified with the <samp class=”option”>b</samp> option, a bitrate-based mode is used. <ul class=”itemize mark-minus”> <var class=”var”>LA</var> - VBR with lookahead, when the <samp class=”option”>look_ahead</samp> option is specified.

<var class=”var”>VCM</var> - video conferencing mode, when the <samp class=”option”>vcm</samp> option is set.

<var class=”var”>CBR</var> - constant bitrate, when <samp class=”option”>maxrate</samp> is specified and equal to the average bitrate.

<var class=”var”>VBR</var> - variable bitrate, when <samp class=”option”>maxrate</samp> is specified, but is higher than the average bitrate.

<var class=”var”>AVBR</var> - average VBR mode, when <samp class=”option”>maxrate</samp> is not specified, both <samp class=”option”>avbr_accuracy</samp> and <samp class=”option”>avbr_convergence</samp> are set to non-zero. This mode is available for H264 and HEVC on Windows.

Otherwise the default ratecontrol method <var class=”var”>CQP</var> is used.

Note that depending on your system, a different mode than the one you specified may be selected by the encoder. Set the verbosity level to <var class=”var”>verbose</var> or higher to see the actual settings used by the QSV runtime. <h4 class=”subsection”>9.32.2 Global Options -> MSDK Options# TOC</h4> Additional libavcodec global options are mapped to MSDK options as follows: <ul class=”itemize mark-bullet”> <samp class=”option”>g/gop_size</samp> -> <samp class=”option”>GopPicSize</samp>

<samp class=”option”>bf/max_b_frames</samp>+1 -> <samp class=”option”>GopRefDist</samp>

<samp class=”option”>rc_init_occupancy/rc_initial_buffer_occupancy</samp> -> <samp class=”option”>InitialDelayInKB</samp>

<samp class=”option”>slices</samp> -> <samp class=”option”>NumSlice</samp>

<samp class=”option”>refs</samp> -> <samp class=”option”>NumRefFrame</samp>

<samp class=”option”>b_strategy/b_frame_strategy</samp> -> <samp class=”option”>BRefType</samp>

<samp class=”option”>cgop/CLOSED_GOP</samp> codec flag -> <samp class=”option”>GopOptFlag</samp>

For the <var class=”var”>CQP</var> mode, the <samp class=”option”>i_qfactor/i_qoffset</samp> and <samp class=”option”>b_qfactor/b_qoffset</samp> set the difference between <var class=”var”>QPP</var> and <var

class=”var”>QPI</var>, and <var class=”var”>QPP</var> and <var class=”var”>QPB</var> respectively.

Setting the <samp class=”option”>coder</samp> option to the value <var class=”var”>vlc</var> will make the H.264 encoder use CAVLC instead of CABAC.

 <h4 class=”subsection”>9.32.3 Common Options# TOC</h4>
Following options are used by all qsv encoders. <dl class=”table”>
<dt><samp class=”option”><var class=”var”>async_depth</var></samp></dt>
<dd> Specifies how many asynchronous operations an application performs before the application explicitly synchronizes the result. If zero, the value is not specified. </dd> <dt><samp class=”option”><var class=”var”>preset</var></samp></dt> <dd> This option itemizes a range of choices from veryfast (best speed) to veryslow (best quality). <dl class=”table”> <dt>‘<samp class=”samp”>veryfast</samp>’</dt>
<dt>‘<samp class=”samp”>faster</samp>’</dt> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dt>‘<samp class=”samp”>medium</samp>’</d
<dt>‘<samp class=”samp”>slow</samp>’</dt> <dt>‘<samp class=”samp”>slower</samp>’</dt> <dt>‘<samp class=”samp”>veryslow</samp>’
</dl>

</dd> <dt><samp class=”option”><var class=”var”>forced_idr</var></samp></dt>
<dd> Forcing I frames as IDR frames. </dd> <dt><samp class=”option”><var class=”var”>low_power</var></samp></dt>
<dd> For encoders set this flag to ON to reduce power consumption and GPU usage. </dd> </dl>

 <h4 class=”subsection”>9.32.4 Runtime Options# TOC</h4>
Following options can be used during qsv encoding. <dl class=”table”> <dt><samp class=”option”><var class=”var”>global_quality</var></samp></dt>
<dt><samp class=”option”><var class=”var”>i_quant_factor</var></samp></dt>
<dt><samp class=”option”><var class=”var”>i_quant_offset</var></samp></dt>
<dt><samp class=”option”><var class=”var”>b_quant_factor</var></samp></dt>
<dt><samp class=”option”><var class=”var”>b_quant_offset</var></samp></dt>
<dd> Supported in h264_qsv and hevc_qsv. Change these value to reset qsv codec’s qp configuration. </dd> <dt><samp class=”option”><var class=”var”>max_frame_size</var></samp></dt>
<dd> Supported in h264_qsv and hevc_qsv. Change this value to reset qsv codec’s MaxFrameSize configuration. </dd> <dt><samp

class=”option”><var class=”var”>gop_size</var></samp></dt> <dd> Change this value to reset qsv codec’s gop configuration. </dd> <dt><samp class=”option”><var class=”var”>int_ref_type</var></samp></dt> <dt><samp class=”option”><var class=”var”>int_ref_cycle_size</var></samp></dt> <dt><samp class=”option”><var class=”var”>int_ref_qp_delta</var></samp></dt> <dt><samp class=”option”><var class=”var”>int_ref_cycle_dist</var></samp></dt> <dd> Supported in h264_qsv and hevc_qsv. Change these value to reset qsv codec’s Intra Refresh configuration. </dd> <dt><samp class=”option”><var class=”var”>qmax</var></samp></dt> <dt><samp class=”option”><var class=”var”>qmin</var></samp></dt> <dt><samp class=”option”><var class=”var”>max_qp_i</var></samp></dt> <dt><samp class=”option”><var class=”var”>min_qp_i</var></samp></dt> <dt><samp class=”option”><var class=”var”>max_qp_p</var></samp></dt> <dt><samp class=”option”><var class=”var”>min_qp_p</var></samp></dt> <dt><samp class=”option”><var class=”var”>max_qp_b</var></samp></dt> <dt><samp class=”option”><var class=”var”>min_qp_b</var></samp></dt> <dd> Supported in h264_qsv. Change these value to reset qsv codec’s max/min qp configuration. </dd> <dt><samp class=”option”><var class=”var”>low_delay_brc</var></samp></dt> <dd> Supported in h264_qsv, hevc_qsv and av1_qsv. Change this value to reset qsv codec’s low_delay_brc configuration. </dd> <dt><samp class=”option”><var class=”var”>framerate</var></samp></dt> <dd> Change this value to reset qsv codec’s framerate configuration. </dd> <dt><samp class=”option”><var class=”var”>bit_rate</var></samp></dt> <dt><samp class=”option”><var class=”var”>rc_buffer_size</var></samp></dt> <dt><samp class=”option”><var class=”var”>rc_initial_buffer_occupancy</var></samp></dt> <dt><samp class=”option”><var class=”var”>rc_max_rate</var></samp></dt> <dd> Change these value to reset qsv codec’s bitrate control configuration. </dd> <dt><samp class=”option”><var class=”var”>pic_timing_sei</var></samp></dt> <dd> Supported in h264_qsv and hevc_qsv. Change this value to reset qsv codec’s pic_timing_sei configuration. </dd> <dt><samp class=”option”><var class=”var”>qsv_params</var></samp></dt> <dd> Set QSV encoder parameters as a colon-separated list of key-value pairs. The <samp class=”option”>qsv_params</samp> should be formatted as <code class=”code”>key1=value1:key2=value2:...</code>. These parameters are passed directly to the underlying Intel Quick Sync Video (QSV) encoder using the MFXSetParameter function. Example: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -c:v h264_qsv -qsv_params "CodingOption1=1:CodingOption2=2" output.mp4 </pre>

This option allows fine-grained control over various encoder-specific settings provided by the QSV encoder. </dd> </dl>

<h4 class="subsection">9.32.5 H264 options# TOC</h4>

These options are used by h264_qsv

<dl class="table">

<dt><samp class="option"><var class="var">extbrc</var></samp></dt>

<dd> Extended bitrate control.

</dd>

<dt><samp class="option"><var class="var">recovery_point_sei</var></samp></dt>

<dd> Set this flag to insert the recovery point SEI message at the beginning of every

intra refresh cycle.

</dd>

<dt><samp class="option"><var class="var">rdo</var></samp></dt>

<dd> Enable rate distortion optimization.

</dd>

<dt><samp class="option"><var class="var">max_frame_size</var></samp></dt>

<dd> Maximum encoded frame size in bytes.

</dd>

<dt><samp class="option"><var class="var">max_frame_size_i</var></samp></dt>

<dd> Maximum encoded frame size for I frames in bytes. If this value is set as larger

than zero, then for I frames the value set by max_frame_size is ignored.

</dd>

<dt><samp class="option"><var class="var">max_frame_size_p</var></samp></dt>

<dd> Maximum encoded frame size for P frames in bytes. If this value is set as larger

than zero, then for P frames the value set by max_frame_size is ignored.

</dd>

<dt><samp class="option"><var class="var">max_slice_size</var></samp></dt>

<dd> Maximum encoded slice size in bytes.

</dd>

<dt><samp class="option"><var class="var">bitrate_limit</var></samp></dt>

<dd> Toggle bitrate limitations.

Modifies bitrate to be in the range imposed by the QSV encoder. Setting this

flag off may lead to violation of HRD conformance. Mind that specifying bitrate

below the QSV encoder range might significantly affect quality. If on this

option takes effect in non CQP modes: if bitrate is not in the range imposed

by the QSV encoder, it will be changed to be in the range.

</dd>

<dt><samp class="option"><var class="var">mbbrc</var></samp></dt>

<dd> Setting this flag enables macroblock level bitrate control that generally

improves subjective visual quality. Enabling this flag may have negative impact

on performance and objective visual quality metric.

</dd>

<dt><samp class="option"><var class="var">low_delay_brc</var></samp></dt>

<dd> Setting this flag turns on or off LowDelayBRC feature in qsv plugin, which provides

more accurate bitrate control to minimize the variance of bitstream size frame

by frame. Value: -1-default 0-off 1-on

</dd>

<dt><samp class=”option”><var class=”var”>adaptive_i</var></samp></dt>

<dd> This flag controls insertion of I frames by the QSV encoder. Turn ON this flag

to allow changing of frame type from P and B to I.

</dd>

<dt><samp class=”option”><var class=”var”>adaptive_b</var></samp></dt>

<dd> This flag controls changing of frame type from B to P.

</dd>

<dt><samp class=”option”><var class=”var”>p_strategy</var></samp></dt>

<dd> Enable P-pyramid: 0-default 1-simple 2-pyramid(bf need to be set to 0).

</dd>

<dt><samp class=”option”><var class=”var”>b_strategy</var></samp></dt>

<dd> This option controls usage of B frames as reference.

</dd>

<dt><samp class=”option”><var class=”var”>dblk_idc</var></samp></dt>

<dd> This option disable deblocking. It has value in range 0~2.

</dd>

<dt><samp class=”option”><var class=”var”>cavlc</var></samp></dt>

<dd> If set, CAVLC is used; if unset, CABAC is used for encoding.

</dd>

<dt><samp class=”option”><var class=”var”>vcm</var></samp></dt>

<dd> Video conferencing mode, please see ratecontrol method.

</dd>

<dt><samp class=”option”><var class=”var”>idr_interval</var></samp></dt>
<dd> Distance (in I-frames) between IDR frames.

</dd>
<dt><samp class=”option”><var class=”var”>pic_timing_sei</var></samp></dt>
<dd> Insert picture timing SEI with pic_struct_syntax element.

</dd>
<dt><samp class=”option”><var class=”var”>single_sei_nal_unit</var></samp></dt>
<dd> Put all the SEI messages into one NALU.

</dd>
<dt><samp class=”option”><var class=”var”>max_dec_frame_buffering</var></samp></dt>
<dd> Maximum number of frames buffered in the DPB.

</dd>
<dt><samp class=”option”><var class=”var”>look_ahead</var></samp></dt>
<dd> Use VBR algorithm with look ahead.

</dd>
<dt><samp class=”option”><var class=”var”>look_ahead_depth</var></samp></dt>
<dd> Depth of look ahead in number frames.

</dd>
<dt><samp class=”option”><var class=”var”>look_ahead_downsampling</var></samp></dt>
<dd> Downscaling factor for the frames saved for the lookahead analysis.
<dl class=”table”>
<dt>‘<samp class=”samp”>unknown</samp>’</dt>
<dt>‘<samp class=”samp”>auto</samp>’</dt>
<dt>‘<samp class=”samp”>off</samp>’</dt>
<dt>‘<samp class=”samp”>2x</samp>’</dt>

<dt>‘<samp class="samp">4x</samp>’</dt>

</dl>

</dd> <dt><samp class="option"><var class="var">int_ref_type</var></samp></dt>
<dd> Specifies intra refresh type. The major goal of intra refresh is improvement of error resilience without significant impact on encoded bitstream size caused by I frames. The SDK encoder achieves this by encoding part of each frame in refresh cycle using intra MBs. <var class="var">none</var> means no refresh. <var class="var">vertical</var> means vertical refresh, by column of MBs. <var class="var">horizontal</var> means horizontal refresh, by rows of MBs. <var class="var">slice</var> means horizontal refresh by slices without overlapping. In case of <var class="var">slice</var>, in_ref_cycle_size is ignored. To enable intra refresh, B frame should be set to 0. </dd> <dt><samp class="option"><var class="var">int_ref_cycle_size</var></samp></dt>
<dd> Specifies number of pictures within refresh cycle starting from 2. 0 and 1 are invalid values. </dd> <dt><samp class="option"><var class="var">int_ref_qp_delta</var></samp></dt> <dd> Specifies QP difference for inserted intra MBs. This is signed value in [-51, 51] range if target encoding bit-depth for luma samples is 8 and this range is [-63, 63] for 10 bit-depth or [-75, 75] for 12 bit-depth respectively. </dd> <dt><samp class="option"><var class="var">int_ref_cycle_dist</var></samp></dt>
<dd> Distance between the beginnings of the intra-refresh cycles in frames. </dd> <dt><samp class="option"><var class="var">profile</var></samp></dt>
<dd><dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt>
<dt>‘<samp class="samp">baseline</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt>
</dl>

</dd> <dt><samp class="option"><var class="var">a53cc</var></samp></dt>
<dd> Use A53 Closed Captions (if available). </dd> <dt><samp class="option"><var class="var">aud</var></samp></dt> <dd> Insert the Access Unit Delimiter NAL. </dd> <dt><samp class="option"><var class="var">mfmode</var></samp></dt> <dd> Multi-Frame Mode. <dl class="table"> <dt>‘<samp class="samp">off</samp>’</dt>
<dt>‘<samp class="samp">auto</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">repeat_pps</var></samp></dt>
<dd> Repeat pps for every frame. </dd> <dt><samp class="option"><var class="var">max_qp_i</var></samp></dt>
<dd> Maximum video quantizer scale for I frame. </dd>
<dt><samp class="option"><var class="var">min_qp_i</var></samp></dt>
<dd> Minimum video quantizer scale for I frame. </dd>
<dt><samp class="option"><var class="var">max_qp_p</var></samp></dt>
<dd> Maximum video quantizer scale for P frame. </dd>
<dt><samp class="option"><var class="var">min_qp_p</var></samp></dt>
<dd> Minimum video quantizer scale for P frame. </dd>

<dt><samp class=”option”><var class=”var”>max_qp_b</var></samp></dt>
<dd> Maximum video quantizer scale for B frame. </dd>
<dt><samp class=”option”><var class=”var”>min_qp_b</var></samp></dt>
<dd> Minimum video quantizer scale for B frame. </dd>
<dt><samp class=”option”><var class=”var”>scenario</var></samp></dt>
<dd> Provides a hint to encoder about the scenario for the encoding session.
<dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt>
<dt>‘<samp class=”samp”>displayremoting</samp>’</dt>
<dt>‘<samp class=”samp”>videoconference</samp>’</dt>
<dt>‘<samp class=”samp”>archive</samp>’</dt> <dt>‘<samp
class=”samp”>livestreaming</samp>’</dt> <dt>‘<samp
class=”samp”>cameracapture</samp>’</dt> <dt>‘<samp
class=”samp”>videosurveillance</samp>’</dt> <dt>‘<samp
class=”samp”>gamestreaming</samp>’</dt> <dt>‘<samp
class=”samp”>remotegaming</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>avbr_accuracy</var></samp></dt>
<dd> Accuracy of the AVBR ratecontrol (unit of tenth of percent).
</dd> <dt><samp class=”option”><var class=”var”>avbr_convergence</var></samp></dt>
<dd> Convergence of the AVBR ratecontrol (unit of 100 frames)
The parameters <var class=”var”>avbr_accuracy</var> and <var
class=”var”>avbr_convergence</var> are for the average variable bitrate
control (AVBR) algorithm. The algorithm focuses on overall encoding quality
while meeting the specified bitrate, <var class=”var”>target_bitrate</var>,
within the accuracy range <var class=”var”>avbr_accuracy</var>, after a
<var class=”var”>avbr_Convergence</var> period. This method does not
follow HRD and the instant bitrate is not capped or padded. </dd>
<dt><samp class=”option”><var class=”var”>skip_frame</var></samp></dt>
<dd> Use per-frame metadata "qsv_skip_frame" to skip frame
when encoding. This option defines the usage of this metadata. <dl
class=”table”> <dt>‘<samp class=”samp”>no_skip</samp>’</dt>
<dd> Frame skipping is disabled. </dd> <dt>‘<samp
class=”samp”>insert_dummy</samp>’</dt> <dd> Encoder
inserts into bitstream frame where all macroblocks are encoded as skipped.
</dd> <dt>‘<samp class=”samp”>insert_nothing</samp>’</dt>
<dd> Similar to insert_dummy, but encoder inserts nothing into bitstream.
The skipped frames are still used in brc. For example, gop still include skipped
frames, and the frames after skipped frames will be larger in size. </dd>
<dt>‘<samp class=”samp”>brc_only</samp>’</dt> <dd> skip_frame
metadata indicates the number of missed frames before the current frame.
</dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>9.32.6
HEVC Options<a class=”anchor hidden-xs”
href=”#HEVC-Options-1” aria-hidden=”true”># <a class=”anchor

hidden-xs”href=”#toc-HEVC-Options-1” aria-hidden=”true”>TOC</h4>
These options are used by hevc_qsv <dl class=”table”>
<dt><samp class=”option”><var class=”var”>extbrc</var></samp></dt>
<dd> Extended bitrate control. </dd> <dt><samp class=”option”><var
class=”var”>recovery_point_sei</var></samp></dt> <dd> Set this
flag to insert the recovery point SEI message at the beginning of every
intra refresh cycle. </dd> <dt><samp class=”option”><var
class=”var”>rdo</var></samp></dt> <dd> Enable rate distor-
tion optimization. </dd> <dt><samp class=”option”><var
class=”var”>max_frame_size</var></samp></dt> <dd> Maximum
encoded frame size in bytes. </dd> <dt><samp class=”option”><var
class=”var”>max_frame_size_i</var></samp></dt> <dd> Maximum
encoded frame size for I frames in bytes. If this value is set as larger than zero,
then for I frames the value set by max_frame_size is ignored. </dd>
<dt><samp class=”option”><var class=”var”>max_frame_size_p</var></samp></dt>
<dd> Maximum encoded frame size for P frames in bytes. If this
value is set as larger than zero, then for P frames the value set by
max_frame_size is ignored. </dd> <dt><samp class=”option”><var
class=”var”>max_slice_size</var></samp></dt> <dd> Maximum
encoded slice size in bytes. </dd> <dt><samp class=”option”><var
class=”var”>mbbrc</var></samp></dt> <dd> Setting this flag en-
ables macroblock level bitrate control that generally improves subjective visual
quality. Enabling this flag may have negative impact on performance and ob-
jective visual quality metric. </dd> <dt><samp class=”option”><var
class=”var”>low_delay_brc</var></samp></dt> <dd> Setting this
flag turns on or off LowDelayBRC feature in qsv plugin, which provides more ac-
curate bitrate control to minimize the variance of bitstream size frame by frame.
Value: -1-default 0-off 1-on </dd> <dt><samp class=”option”><var
class=”var”>adaptive_i</var></samp></dt> <dd> This flag controls
insertion of I frames by the QSV encoder. Turn ON this flag to allow changing of
frame type from P and B to I. </dd> <dt><samp class=”option”><var
class=”var”>adaptive_b</var></samp></dt> <dd> This flag con-
trols changing of frame type from B to P. </dd> <dt><samp
class=”option”><var class=”var”>p_strategy</var></samp></dt>
<dd> Enable P-pyramid: 0-default 1-simple 2-pyramid(bf need
to be set to 0). </dd> <dt><samp class=”option”><var
class=”var”>b_strategy</var></samp></dt> <dd> This option
controls usage of B frames as reference. </dd> <dt><samp
class=”option”><var class=”var”>dblk_idc</var></samp></dt> <dd> This
option disable deblocking. It has value in range 0~2. </dd> <dt><samp
class=”option”><var class=”var”>idr_interval</var></samp></dt>
<dd> Distance (in I-frames) between IDR frames. <dl class=”table”>
<dt>‘<samp class=”samp”>begin_only</samp>’</dt>
<dd> Output an IDR-frame only at the beginning of the stream.
</dd> </dl>

</dd> <dt><samp class=”option”><var class=”var”>load_plugin</var></samp></dt>
<dd> A user plugin to load in an internal session. <dl class=”table”>
<dt>‘<samp class=”samp”>none</samp>’</dt> <dt>‘<samp
class=”samp”>hevc_sw</samp>’</dt> <dt>‘<samp
class=”samp”>hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>load_plugins</var></samp></dt>
<dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session. </dd> <dt><samp class=”option”><var class=”var”>look_ahead_depth</var></samp></d
<dd> Depth of look ahead in number frames, available when extbrc option is enabled. </dd> <dt><samp class=”option”><var class=”var”>profile</var></samp></dt> <dd> Set the encoding profile (scc requires libmfx >= 1.32). <dl class=”table”>
<dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp
class=”samp”>main</samp>’</dt> <dt>‘<samp class=”samp”>main10</samp>’</
<dt>‘<samp class=”samp”>mainsp</samp>’</dt> <dt>‘<samp
class=”samp”>rext</samp>’</dt> <dt>‘<samp class=”samp”>scc</samp>’</dt>
</dl>

</dd> <dt><samp class=”option”><var class=”var”>tier</var></samp></dt>
<dd> Set the encoding tier (only level >= 4 can support high tier). This option only takes effect when the level option is specified. <dl
class=”table”> <dt>‘<samp class=”samp”>main</samp>’</dt>
<dt>‘<samp class=”samp”>high</samp>’</dt> </dl>

</dd> <dt><samp class=”option”><var class=”var”>gpb</var></samp></dt>
<dd> 1: GPB (generalized P/B frame) 0: regular P frame.
 </dd> <dt><samp class=”option”><var class=”var”>tile_cols</var></samp></dt>
<dd> Number of columns for tiled encoding. </dd> <dt><samp
class=”option”><var class=”var”>tile_rows</var></samp></dt> <dd> Number of rows for tiled encoding. </dd> <dt><samp class=”option”><var class=”var”>aud</var></samp></dt> <dd> Insert the Access Unit Delimiter NAL. </dd> <dt><samp class=”option”><var class=”var”>pic_timing_sei</var></samp></dt> <dd> Insert picture timing SEI with pic_struct_syntax element. </dd> <dt><samp class=”option”><var class=”var”>transform_skip</var></samp></dt>
<dd> Turn this option ON to enable transformskip. It is supported on platform equal or newer than ICL. </dd> <dt><samp class=”option”><var class=”var”>int_ref_type</var></samp></dt> <dd> Specifies intra refresh type. The major goal of intra refresh is improvement of error resilience without significant impact on encoded bitstream size caused by I frames. The SDK encoder achieves this by encoding part of each frame in refresh cycle using intra MBs. <var class=”var”>none</var> means no refresh. <var class=”var”>vertical</var> means vertical refresh, by column of MBs. <var class=”var”>horizontal</var> means horizontal refresh, by rows of MBs. <var class=”var”>slice</var> means horizontal refresh by slices without overlapping. In case of <var class=”var”>slice</var>, in_ref_cycle_size is ignored.

To enable intra refresh, B frame should be set to 0. </dd> <dt><samp class="option"><var class="var">int_ref_cycle_size</var></samp></dt> <dd> Specifies number of pictures within refresh cycle starting from 2. 0 and 1 are invalid values. </dd> <dt><samp class="option"><var class="var">int_ref_qp_delta</var></samp></dt> <dd> Specifies QP difference for inserted intra MBs. This is signed value in [-51, 51] range if target encoding bit-depth for luma samples is 8 and this range is [-63, 63] for 10 bit-depth or [-75, 75] for 12 bit-depth respectively. </dd> <dt><samp class="option"><var class="var">int_ref_cycle_dist</var></samp></dt> <dd> Distance between the beginnings of the intra-refresh cycles in frames. </dd> <dt><samp class="option"><var class="var">max_qp_i</var></samp></dt> <dd> Maximum video quantizer scale for I frame. </dd> <dt><samp class="option"><var class="var">min_qp_i</var></samp></dt> <dd> Minimum video quantizer scale for I frame. </dd> <dt><samp class="option"><var class="var">max_qp_p</var></samp></dt> <dd> Maximum video quantizer scale for P frame. </dd> <dt><samp class="option"><var class="var">min_qp_p</var></samp></dt> <dd> Minimum video quantizer scale for P frame. </dd> <dt><samp class="option"><var class="var">max_qp_b</var></samp></dt> <dd> Maximum video quantizer scale for B frame. </dd> <dt><samp class="option"><var class="var">min_qp_b</var></samp></dt> <dd> Minimum video quantizer scale for B frame. </dd> <dt><samp class="option"><var class="var">scenario</var></samp></dt> <dd> Provides a hint to encoder about the scenario for the encoding session. <dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">displayremoting</samp>’</dt> <dt>‘<samp class="samp">videoconference</samp>’</dt> <dt>‘<samp class="samp">archive</samp>’</dt> <dt>‘<samp class="samp">livestreaming</samp>’</dt> <dt>‘<samp class="samp">cameracapture</samp>’</dt> <dt>‘<samp class="samp">videosurveillance</samp>’</dt> <dt>‘<samp class="samp">gamestreaming</samp>’</dt> <dt>‘<samp class="samp">remotegaming</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">avbr_accuracy</var></samp></dt> <dd> Accuracy of the AVBR ratecontrol (unit of tenth of percent). </dd> <dt><samp class="option"><var class="var">avbr_convergence</var></samp></dt> <dd> Convergence of the AVBR ratecontrol (unit of 100 frames) The parameters <var class="var">avbr_accuracy</var> and <var class="var">avbr_convergence</var> are for the average variable bitrate control (AVBR) algorithm. The algorithm focuses on overall encoding quality while meeting the specified bitrate, <var class="var">target_bitrate</var>, within the accuracy range <var class="var">avbr_accuracy</var>, after a <var class="var">avbr_Convergence</var> period. This method does not follow HRD and the instant bitrate is not capped or padded. </dd> <dt><samp class="option"><var class="var">skip_frame</var></samp></dt>

<dd> Use per-frame metadata "qsv_skip_frame" to skip frame when encoding. This option defines the usage of this metadata. <dl class="table"> <dt>‘<samp class="samp">no_skip</samp>’</dt> <dd> Frame skipping is disabled. </dd> <dt>‘<samp class="samp">insert_dummy</samp>’</dt> <dd> Encoder inserts into bitstream frame where all macroblocks are encoded as skipped. </dd> <dt>‘<samp class="samp">insert_nothing</samp>’</dt> <dd> Similar to insert_dummy, but encoder inserts nothing into bitstream. The skipped frames are still used in brc. For example, gop still include skipped frames, and the frames after skipped frames will be larger in size. </dd> <dt>‘<samp class="samp">brc_only</samp>’</dt> <dd> skip_frame metadata indicates the number of missed frames before the current frame. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">9.32.7 MPEG2 Options# TOC</h4> These options are used by mpeg2_qsv <dl class="table"> <dt><samp class="option"><var class="var">profile</var></samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">simple</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</dt> <dt>‘<samp class="samp">high</samp>’</dt>
</dl> </dd> </dl>

 <h4 class="subsection">9.32.8 VP9 Options# TOC</h4> These options are used by vp9_qsv <dl class="table"> <dt><samp class="option"><var class="var">profile</var></samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">profile0</samp>’</dt> <dt>‘<samp class="samp">profile1</samp>’</dt> <dt>‘<samp class="samp">profile2</samp>’
<dt>‘<samp class="samp">profile3</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">tile_cols</var></samp></dt> <dd> Number of columns for tiled encoding (requires libmfx >= 1.29). </dd> <dt><samp class="option"><var class="var">tile_rows</var></samp></dt> <dd> Number of rows for tiled encoding (requires libmfx >= 1.29). </dd> </dl>

 <h4 class="subsection">9.32.9 AV1 Options# TOC</h4> These

options are used by av1_qsv (requires libvpl). <dl class="table">
<dt><samp class="option"><var class="var">profile</var></samp></dt>
<dd><dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt>
<dt>‘<samp class="samp">main</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">tile_cols</var></samp></dt>
<dd> Number of columns for tiled encoding. </dd> <dt><samp
class="option"><var class="var">tile_rows</var></samp></dt> <dd> Number
of rows for tiled encoding. </dd> <dt><samp class="option"><var
class="var">adaptive_i</var></samp></dt> <dd> This flag controls
insertion of I frames by the QSV encoder. Turn ON this flag to allow changing of
frame type from P and B to I. </dd> <dt><samp class="option"><var
class="var">adaptive_b</var></samp></dt> <dd> This flag controls changing of frame type from B to P. </dd> <dt><samp
class="option"><var class="var">b_strategy</var></samp></dt>
<dd> This option controls usage of B frames as reference. </dd>
<dt><samp class="option"><var class="var">extbrc</var></samp></dt>
<dd> Extended bitrate control. </dd> <dt><samp class="option"><var
class="var">look_ahead_depth</var></samp></dt> <dd> Depth of
look ahead in number frames, available when extbrc option is enabled.
</dd> <dt><samp class="option"><var class="var">low_delay_brc</var></samp></dt>
<dd> Setting this flag turns on or off LowDelayBRC feature in qsv plugin,
which provides more accurate bitrate control to minimize the variance of
bitstream size frame by frame. Value: -1-default 0-off 1-on </dd>
<dt><samp class="option"><var class="var">max_frame_size</var></samp></dt>
<dd> Set the allowed max size in bytes for each frame. If the frame size
exceeds the limitation, encoder will adjust the QP value to control the
frame size. Invalid in CQP rate control mode. </dd> <dt><samp
class="option"><var class="var">max_frame_size_i</var></samp></dt>
<dd> Maximum encoded frame size for I frames in bytes. If this
value is set as larger than zero, then for I frames the value set by
max_frame_size is ignored. </dd> <dt><samp class="option"><var
class="var">max_frame_size_p</var></samp></dt> <dd> Maximum
encoded frame size for P frames in bytes. If this value is set as larger than zero,
then for P frames the value set by max_frame_size is ignored. </dd>
</dl>

 <h3 class="section">9.33 snow<span class="pull-
right">#
TOC</h3>

 <h4 class="subsection">9.33.1 Options<span
class="pull-right"><a class="anchor hidden-xs" href="#Options-53" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Options-53"
aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">iterative_dia_size</samp></dt>
<dd> dia size for the iterative motion estimation </dd> </dl>

 <h3 class="section">9.34 VAAPI encoders# TOC</h3>

Wrappers for hardware encoders accessible via VAAPI. These encoders only accept input in VAAPI hardware surfaces. If you have input in software frames, use the <samp class="option">hwupload</samp> filter to upload them to the GPU. The following standard libavcodec options are used: <ul class="itemize mark-bullet"> <samp class="option">g</samp> / <samp class="option">gop_size</samp> <samp class="option">bf</samp> / <samp class="option">max_b_frames</samp> <samp class="option">profile</samp>

If not set, this will be determined automatically from the format of the input frames and the profiles supported by the driver. <samp class="option">level</samp> <samp class="option">b</samp> / <samp class="option">bit_rate</samp> <samp class="option">maxrate</samp> / <samp class="option">rc_max_rate</samp> <samp class="option">bufsize</samp> / <samp class="option">rc_buffer_size</samp> <samp class="option">rc_init_occupancy</samp> / <samp class="option">rc_initial_buffer_occupancy</samp> <samp class="option">compression_level</samp>

Speed / quality tradeoff: higher values are faster / worse quality. <samp class="option">q</samp> / <samp class="option">global_quality</samp>

Size / quality tradeoff: higher values are smaller / worse quality. <samp class="option">qmin</samp> <samp class="option">qmax</samp> <samp class="option">i_qfactor</samp> / <samp class="option">i_quant_factor</samp> <samp class="option">i_qoffset</samp> / <samp class="option">i_quant_offset</samp> <samp class="option">b_qfactor</samp> / <samp class="option">b_quant_factor</samp> <samp class="option">b_qoffset</samp> / <samp class="option">b_quant_offset</samp> <samp class="option">slices</samp>

All encoders support the following options: <dl class="table"> <dt><samp class="option">low_power</samp></dt> <dd> Some drivers/platforms offer a second encoder for some codecs intended to use less power than the default encoder; setting this option will attempt to use that encoder. Note that it may support a reduced feature set, so some other options may not be available in this mode. </dd> <dt><samp class="option">idr_interval</samp></dt> <dd> Set the number of normal intra frames between full-refresh (IDR) frames in open-GOP mode. The intra frames are still IRAPs, but will not include global headers and may have non-decodable leading pictures. </dd> <dt><samp class="option">b_depth</samp></dt> <dd> Set the B-frame reference depth. When set to one (the default), all B-frames will refer only to P- or I-frames. When set to greater values multiple layers of B-frames will

be present, frames in each layer only referring to frames in higher layers. </dd> <dt><samp class=”option”>async_depth</samp></dt> <dd> Maximum processing parallelism. Increase this to improve single channel performance. This option doesn’t work if driver doesn’t implement vaSyncBuffer function. Please make sure there are enough hw_frames allocated if a large number of async_depth is used. </dd> <dt><samp class=”option”>max_frame_size</samp></dt> <dd> Set the allowed max size in bytes for each frame. If the frame size exceeds the limitation, encoder will adjust the QP value to control the frame size. Invalid in CQP rate control mode. </dd> <dt><samp class=”option”>rc_mode</samp></dt> <dd> Set the rate control mode to use. A given driver may only support a subset of modes. Possible modes: <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Choose the mode automatically based on driver support and the other options. This is the default. </dd> <dt><samp class=”option”>CQP</samp></dt> <dd> Constant-quality. </dd> <dt><samp class=”option”>CBR</samp></dt> <dd> Constant-bitrate. </dd> <dt><samp class=”option”>VBR</samp></dt> <dd> Variable-bitrate. </dd> <dt><samp class=”option”>ICQ</samp></dt> <dd> Intelligent constant-quality. </dd> <dt><samp class=”option”>QVBR</samp></dt> <dd> Quality-defined variable-bitrate. </dd> <dt><samp class=”option”>AVBR</samp></dt> <dd> Average variable bitrate. </dd> </dl>

</dd> <dt><samp class=”option”>blbrc</samp></dt> <dd> Enable block level rate control, which assigns different bitrate block by block. Invalid for CQP mode. </dd> </dl>

Each encoder also has its own specific options: <dl class=”table”> <dt><samp class=”option”>av1_vaapi</samp></dt> <dd> <samp class=”option”>profile</samp> sets the value of seq_profile. <samp class=”option”>tier</samp> sets the value of seq_tier. <samp class=”option”>level</samp> sets the value of seq_level_idx. <dl class=”table”> <dt><samp class=”option”>tiles</samp></dt> <dd> Set the number of tiles to encode the input video with, as columns x rows. (default is auto, which means use minimal tile column/row number). </dd> <dt><samp class=”option”>tile_groups</samp></dt> <dd> Set tile groups number. All the tiles will be distributed as evenly as possible to each tile group. (default is 1). </dd> </dl>

</dd> <dt><samp class=”option”>h264_vaapi</samp></dt> <dd> <samp class=”option”>profile</samp> sets the value of profile_idc and the constraint_set*_flags. <samp class=”option”>level</samp> sets the value of level_idc. <dl class=”table”> <dt><samp class=”option”>coder</samp></dt> <dd> Set entropy encoder (default is cabac). Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>ac</samp>’</dt>

<dt>‘<samp class="samp">cabac</samp>’</dt> <dd> Use CABAC. </dd> <dt>‘<samp class="samp">vlc</samp>’</dt> <dt>‘<samp class="samp">cavlc</samp>’</dt> <dd> Use CAVLC. </dd> </dl>

</dd> <dt><samp class="option">aud</samp></dt> <dd> Include access unit delimiters in the stream (not included by default). </dd> <dt><samp class="option">sei</samp></dt> <dd> Set SEI message types to include. Some combination of the following values: <dl class="table"> <dt>‘<samp class="samp">identifier</samp>’</dt> <dd> Include a user_data_unregistered message containing information about the encoder. </dd> <dt>‘<samp class="samp">timing</samp>’</dt> <dd> Include picture timing parameters (buffering_period and pic_timing messages). </dd> <dt>‘<samp class="samp">recovery_point</samp>’</dt> <dd> Include recovery points where appropriate (recovery_point messages). </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">hevc_vaapi</samp></dt> <dd> <samp class="option">profile</samp> and <samp class="option">level</samp> set the values of general_profile_idc and general_level_idc respectively. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Include access unit delimiters in the stream (not included by default). </dd> <dt><samp class="option">tier</samp></dt> <dd> Set general_tier_flag. This may affect the level chosen for the stream if it is not explicitly specified. </dd> <dt><samp class="option">sei</samp></dt> <dd> Set SEI message types to include. Some combination of the following values: <dl class="table"> <dt>‘<samp class="samp">hdr</samp>’</dt> <dd> Include HDR metadata if the input frames have it (mastering_display_colour_volume and content_light_level messages). </dd> </dl>

</dd> <dt><samp class="option">tiles</samp></dt> <dd> Set the number of tiles to encode the input video with, as columns x rows. Larger numbers allow greater parallelism in both encoding and decoding, but may decrease coding efficiency. </dd> </dl>

</dd> <dt><samp class="option">mjpeg_vaapi</samp></dt> <dd> Only baseline DCT encoding is supported. The encoder always uses the standard quantisation and huffman tables - <samp class="option">global_quality</samp> scales the standard quantisation table (range 1-100). For YUV, 4:2:0, 4:2:2 and 4:4:4 subsampling modes are supported. RGB is also supported, and will create an RGB JPEG. <dl class="table">

<dt><samp class=”option”>jfif</samp></dt> <dd> Include JFIF header in each frame (not included by default). </dd> <dt><samp class=”option”>huffman</samp></dt> <dd> Include standard huffman tables (on by default). Turning this off will save a few hundred bytes in each output frame, but may lose compatibility with some JPEG decoders which don’t fully handle MJPEG. </dd> </dl>

</dd> <dt><samp class=”option”>mpeg2_vaapi</samp></dt> <dd> <samp class=”option”>profile</samp> and <samp class=”option”>level</samp> set the value of profile_and_level_indication. </dd> <dt><samp class=”option”>vp8_vaapi</samp></dt> <dd> B-frames are not supported. <samp class=”option”>global_quality</samp> sets the q_idx used for non-key frames (range 0-127). <dl class=”table”> <dt><samp class=”option”>loop_filter_level</samp></dt> <dt><samp class=”option”>loop_filter_sharpness</samp></dt> <dd> Manually set the loop filter parameters. </dd> </dl>

</dd> <dt><samp class=”option”>vp9_vaapi</samp></dt> <dd> <samp class=”option”>global_quality</samp> sets the q_idx used for P-frames (range 0-255). <dl class=”table”> <dt><samp class=”option”>loop_filter_level</samp></dt> <dt><samp class=”option”>loop_filter_sharpness</sam
<dd> Manually set the loop filter parameters. </dd> </dl>

B-frames are supported, but the output stream is always in encode order rather than display order. If B-frames are enabled, it may be necessary to use the <samp class=”option”>vp9_raw_reorder</samp> bitstream filter to modify the output stream to display frames in the correct order. Only normal frames are produced - the <samp class=”option”>vp9_superframe</samp> bitstream filter may be required to produce a stream usable with all decoders. </dd> </dl>

 <h3 class=”section”>9.35 vbn# TOC</h3>

Vizrt Binary Image encoder. This format is used by the broadcast vendor Vizrt for quick texture streaming. Advanced features of the format such as LZW compression of texture data or generation of mipmaps are not supported. <h4 class=”subsection”>9.35.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>format <var class=”var”>string</var></samp></dt> <dd> Sets the texture compression used by the VBN file. Can be <var class=”var”>dxt1</var>, <var class=”var”>dxt5</var> or <var class=”var”>raw</var>. Default is <var class=”var”>dxt5</var>. </dd> </dl>

 <h3 class="section">9.36 vc2# TOC</h3>

SMPTE VC-2 (previously BBC Dirac Pro). This codec was primarily aimed at professional broadcasting but since it supports yuv420, yuv422 and yuv444 at 8 (limited range or full range), 10 or 12 bits, this makes it suitable for other tasks which require low overhead and low compression (like screen recording). <h4 class="subsection">9.36.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">b</samp></dt> <dd> Sets target video bitrate. Usually that’s around 1:6 of the uncompressed video bitrate (e.g. for 1920x1080 50fps yuv422p10 that’s around 400Mbps). Higher values (close to the uncompressed bitrate) turn on lossless compression mode. </dd> <dt><samp class="option">field_order</samp></dt> <dd> Enables field coding when set (e.g. to tt - top field first) for interlaced inputs. Should increase compression with interlaced content as it splits the fields and encodes each separately. </dd> <dt><samp class="option">wavelet_depth</samp></dt> <dd> Sets the total amount of wavelet transforms to apply, between 1 and 5 (default). Lower values reduce compression and quality. Less capable decoders may not be able to handle values of <samp class="option">wavelet_depth</samp> over 3. </dd> <dt><samp class="option">wavelet_type</samp></dt> <dd> Sets the transform type. Currently only <var class="var">5_3</var> (LeGall) and <var class="var">9_7</var> (Deslauriers-Dubuc) are implemented, with 9_7 being the one with better compression and thus is the default. </dd> <dt><samp class="option">slice_width</samp></dt> <dt><samp class="option">slice_height</samp></dt> <dd> Sets the slice size for each slice. Larger values result in better compression. For compatibility with other more limited decoders use <samp class="option">slice_width</samp> of 32 and <samp class="option">slice_height</samp> of 8. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Sets the undershoot tolerance of the rate control system in percent. This is to prevent an expensive search from being run. </dd> <dt><samp class="option">qm</samp></dt> <dd> Sets the quantization matrix preset to use by default or when <samp class="option">wavelet_depth</samp> is set to 5 <ul class="itemize mark-minus"> <var class="var">default</var> Uses the default quantization matrix from the specifications, extended with values for the fifth level. This provides a good balance between keeping detail and omitting artifacts.

<var class="var">flat</var> Use a completely zeroed out quantization matrix. This increases PSNR but might reduce perception. Use in bogus benchmarks.

<var class="var">color</var> Reduces detail but attempts to preserve color at extremely low bitrates.

</dd> </dl>

 <h2 class="chapter">10 Subtitles Encoders# TOC</h2>

 <h3 class="section">10.1 dvbsub# TOC</h3>

This codec encodes the bitmap subtitle format that is used in DVB broadcasts and recordings. The bitmaps are typically embedded in a container such as MPEG-TS as a separate stream. <h4 class="subsection">10.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">min_bpp <var class="var">integer (2, 4, or 8)</var></samp></dt> <dd> Set a minimum bits-per-pixel value for the subtitle color lookup tables. DVB supports 2, 4, and 8 bits-per-pixel color lookup tables. This option enables forcing a particular bits-per-pixel value regardless of the number of colors. Since not all players support or properly support 2 bits-per-pixel, this value defaults to 4. </dd> </dl>

 <h3 class="section">10.2 dvdsub# TOC</h3>

This codec encodes the bitmap subtitle format that is used in DVDs. Typically they are stored in VOBSUB file pairs (*.idx + *.sub), and they can also be used in Matroska files. <h4 class="subsection">10.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">even_rows_fix</samp></dt> <dd> When

set to 1, enable a work-around that makes the number of pixel rows even in all subtitles. This fixes a problem with some players that cut off the bottom row if the number is odd. The work-around just adds a fully transparent row if needed. The overhead is low, typically one byte per subtitle on average. By default, this work-around is disabled. </dd> </dl>

 <h3 class="section">10.3 lrc# TOC</h3>

This codec encodes the LRC lyrics format. <h4 class="subsection">10.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">precision</samp></dt> <dd> Specify the precision of the fractional part of the timestamp. Time base is determined based on this value. Defaults to 2 for centiseconds. </dd> </dl>

 <h2 class="chapter">11 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavcodec

 <h2 class="chapter">12 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-devices.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/

--> <head> <meta charset=”utf-8”> <title> FFmpeg Devices Documentation </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”> <link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body> <h1> FFmpeg Devices Documentation </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”>

1 Description

2 Device Options

3 Input Devices

<ul class=”toc-numbered-mark”>

3.1 alsa

<ul class=”toc-numbered-mark”>

3.1.1 Options

3.2 android_camera

<ul class=”toc-numbered-mark”>

3.2.1 Options

3.3 avfoundation

<ul class=”toc-numbered-mark”>

3.3.1 Options

3.3.2 Examples

3.4 decklink

<ul class=”toc-numbered-mark”>

3.4.1 Options

3.4.2 Examples

3.5 dshow
<ul class="toc-numbered-mark">
3.5.1 Options
3.5.2 Examples

3.6 fbdev
<ul class="toc-numbered-mark">
3.6.1 Options

3.7 gdigrab
<ul class="toc-numbered-mark">
3.7.1 Options

3.8 iec61883
<ul class="toc-numbered-mark">
3.8.1 Options
3.8.2 Examples

3.9 jack
<ul class="toc-numbered-mark">
3.9.1 Options

3.10 kmsgrab
<ul class="toc-numbered-mark">
3.10.1 Options
3.10.2 Examples

3.11 lavfi
<ul class="toc-numbered-mark">

3.11.1 Options
3.11.2 Examples

3.12 libcdio
<ul class="toc-numbered-mark">
3.12.1 Options

3.13 libdc1394
<ul class="toc-numbered-mark">
3.13.1 Options

3.14 openal
<ul class="toc-numbered-mark">
3.14.1 Options
3.14.2 Examples

3.15 oss
<ul class="toc-numbered-mark">
3.15.1 Options

3.16 pulse
<ul class="toc-numbered-mark">
3.16.1 Options
3.16.2 Examples

3.17 sndio
<ul class="toc-numbered-mark">
3.17.1 Options

3.18 video4linux2, v4l2

<ul class="toc-numbered-mark">

3.18.1 Options

3.19 vfwcap

<ul class="toc-numbered-mark">

3.19.1 Options

3.20 x11grab

<ul class="toc-numbered-mark">

3.20.1 Options

4 Output Devices

<ul class="toc-numbered-mark">

4.1 alsa

<ul class="toc-numbered-mark">

4.1.1 Examples

4.2 AudioToolbox

<ul class="toc-numbered-mark">

4.2.1 Options

4.2.2 Examples

4.3 caca

<ul class="toc-numbered-mark">

4.3.1 Options

4.3.2 Examples

4.4 decklink

<ul class="toc-numbered-mark">
4.4.1 Options
4.4.2 Examples

4.5 fbdev
<ul class="toc-numbered-mark">
4.5.1 Options
4.5.2 Examples

4.6 oss
4.7 pulse
<ul class="toc-numbered-mark">
4.7.1 Options
4.7.2 Examples

4.8 sndio
4.9 v4l2
4.10 xv
<ul class="toc-numbered-mark">
4.10.1 Options
4.10.2 Examples

5 See Also
6 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes the input and output devices provided by the libavdevice library.

 <h2 class="chapter">2 Device Options# TOC</h2>

The libavdevice library provides the same interface as libavformat. Namely, an input device is considered like a demuxer, and an output device like a muxer, and the interface and generic device options are the same provided by libavformat (see the ffmpeg-formats manual). In addition each input or output device may support so-called private options, which are specific for that component. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the device <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use.

 <h2 class="chapter">3 Input Devices# TOC</h2>

Input devices are configured elements in FFmpeg which enable accessing the data coming from a multimedia device attached to your system. When you configure your FFmpeg build, all the supported input devices are enabled by default. You can list all available ones using the configure option "–list-indevs". You can disable all the input devices using the configure option "–disable-indevs", and selectively enable an input device using the option "–enable-indev=<var class="var">INDEV</var>", or you can disable a particular input device using the option "–disable-indev=<var class="var">INDEV</var>". The option "-devices" of the ff* tools will display the list of supported input devices. A description of the currently available input devices follows. <h3 class="section">3.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) input device. To enable this input device during configuration you need libasound installed on your system. This device allows capturing from an ALSA device. The name of the device to capture has to be an ALSA card identifier. An ALSA identifier has the syntax: <pre class="example-preformatted">hw:<var class="var">CARD</var>[,<var class="var">DEV</var>[,<var class="var">SUBDEV</var>]] </pre>

where the <var class="var">DEV</var> and <var class="var">SUBDEV</var> components are optional. The three arguments (in order: <var class="var">CARD</var>,<var class="var">DEV</var>,<var class="var">SUBDEV</var>) specify card number or identifier, device number and subdevice number (-1 means any). To see the list of cards currently recognized by your system check the files <samp class="file">/proc/asound/cards</samp> and <samp class="file">/proc/asound/devices</samp>. For example to capture with <code class="command">ffmpeg</code> from an ALSA device with card id 0, you may run the command: <pre class="example-preformatted">ffmpeg -f alsa -i hw:0 alsaout.wav </pre>

For more information see: http://www.alsa-project.org/alsa-doc/alsa-lib/pcm.html <h4 class="subsection">3.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">ch_layout</samp></dt> <dd> Set the channel layout. When unset, the layout defaults to stereo, or to the minimum channel count supported by the device, and is filled in from the ALSA channel map on a best-effort basis. </dd> </dl>

 <h3 class="section">3.2 android_camera# TOC</h3>

Android camera input device. This input devices uses the Android Camera2 NDK API which is available on devices with API level 24+. The availability of android_camera is autodetected during configuration. This device allows capturing from all cameras on an Android device, which are integrated into the Camera2 NDK API. The available cameras are enumerated internally and can be selected with the <var class="var">camera_index</var> parameter. The input file string is discarded. Generally the back facing camera has index 0 while the front facing camera has index 1. <h4 class="subsection">3.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Set the video size given as a string such as 640x480 or hd720. Falls back to the first available configuration reported by Android if re-

quested video size is not available or by default. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the video framerate. Falls back to the first available configuration reported by Android if requested framerate is not available or by default (-1). </dd> <dt><samp class=”option”>camera_index</samp></dt> <dd> Set the index of the camera to use. Default is 0. </dd> <dt><samp class=”option”>input_queue_size</samp></dt> <dd> Set the maximum number of frames to buffer. Default is 5. </dd> </dl>

 <h3 class=”section”>3.3 avfoundation# TOC</h3>

AVFoundation input device. AVFoundation is the currently recommended framework by Apple for streamgrabbing on OSX >= 10.7 as well as on iOS. The input filename has to be given in the following syntax: <pre class=”example-preformatted”>-i "[[VIDEO]:[AUDIO]]" </pre> The first entry selects the video input while the latter selects the audio input. The stream has to be specified by the device name or the device index as shown by the device list. Alternatively, the video and/or audio input device can be chosen by index using the <samp class=”option”> -video_device_index <INDEX> </samp> and/or <samp class=”option”> -audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class=”option”>-list_devices true</samp>, listing all device names and corresponding indices. There are two device name aliases: <dl class=”table”> <dt><code class=”code”>default</code></dt> <dd> Select the AVFoundation default device of the corresponding type. </dd> <dt><code class=”code”>none</code></dt> <dd> Do not record the corresponding media type. This is equivalent to specifying an empty device name or index. </dd> </dl>

 <h4 class=”subsection”>3.3.1 Options# TOC</h4>

AVFoundation supports the following options: <dl class=”table”> <dt><samp class=”option”>-list_devices <TRUE|FALSE></samp></dt> <dd> If set to true, a list of all available input devices is given showing their index, name, unique ID, and USB serial number (when available). </dd> <dt><samp class=”option”>-video_device_index <INDEX></samp></dt> <dd> Specify the video device by its index. Overrides anything given in the input filename. </dd> <dt><samp class=”option”>-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the

input filename. </dd> <dt><samp class=”option”>-video_device_id <ID></samp></dt> <dd> Specify the video device by a prefixed identifier, as shown by <samp class=”option”>-list_devices</samp>: <code class=”code”>uid:<UID></code> selects by AVFoundation unique ID, <code class=”code”>serial:<SERIAL></code> by USB serial number. The unique ID of a UVC device may not be stable across reboots. Only UVC devices expose a serial number. Overrides anything given in the input filename. </dd> <dt><samp class=”option”>-audio_device_id <ID></samp></dt> <dd> Specify the audio device by a prefixed identifier. See <samp class=”option”>-video_device_id</samp>; for audio, the serial is only available for class-compliant USB devices. </dd> <dt><samp class=”option”>-pixel_format <FORMAT></samp></dt> <dd> Request the video device to use a specific pixel format. If the specified format is not supported, a list of available formats is given and the first one in this list is used instead. Available pixel formats are: <code class=”code”>monob, rgb555be, rgb555le, rgb565be, rgb565le, rgb24, bgr24, 0rgb, bgr0, 0bgr, rgb0, bgr48be, uyvy422, yuva444p, yuva444p16le, yuv444p, yuv422p16, yuv422p10, yuv444p10, yuv420p, nv12, yuyv422, gray</code> </dd> <dt><samp class=”option”>-framerate</samp></dt> <dd> Set the grabbing frame rate. Default is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>-video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class=”option”>-capture_cursor</samp></dt> <dd> Capture the mouse pointer. Default is 0. </dd> <dt><samp class=”option”>-capture_mouse_clicks</samp></dt> <dd> Capture the screen mouse clicks. Default is 0. </dd> <dt><samp class=”option”>-capture_raw_data</samp></dt> <dd> Capture the raw device data. Default is 0. Using this option may result in receiving the underlying data delivered to the AVFoundation framework. E.g. for muxed devices that sends raw DV data to the framework (like tape-based camcorders), setting this option to false results in extracted video frames captured in the designated pixel format only. Setting this option to true results in receiving the raw DV stream untouched. </dd> </dl>

 <h4 class=”subsection”>3.3.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print the list of AVFoundation supported devices and exit: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -list_devices true -i "" </pre>

Record video from video device 0 and audio from audio device 0 into out.avi: <pre class=”example-preformatted”>$

ffmpeg -f avfoundation -i "0:0" out.avi </pre>

Record video from video device 2 and audio from audio device 1 into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -video_device_index 2 -i ":1" out.avi </pre>

Record video from the system default video device using the pixel format bgr0 and do not record any audio into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -pixel_format bgr0 -i "default:none" out.avi </pre>

Record raw DV data from a suitable input device and write the output into out.dv: <pre class="example-preformatted">$ ffmpeg -f avfoundation -capture_raw_data true -i "zr100:none" out.dv </pre>

 <h3 class="section">3.4 decklink# TOC</h3>

The decklink input device provides capture capabilities for Blackmagic DeckLink devices. To enable this input device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class="code">--extra-cflags</code>, <code class="code">--extra-cxxflags</code> and <code class="code">--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class="command">widl</code>. DeckLink is very picky about the formats it supports. Pixel format of the input can be set with <samp class="option">raw_format</samp>. Framerate and video size must be determined for your device with <code class="command">-list_formats 1</code>. Audio sample rate is always 48 kHz and the number of channels can be 2, 8 or 16. Note that all audio channels are bundled in one single audio track. <h4 class="subsection">3.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. This option is deprecated, please use the <code class="code">-sources</code> option of ffmpeg to list the available input devices. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of supported formats and

exit. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">format_code <FourCC></samp></dt> <dd> This sets the input video format to the format given by the FourCC. To see the supported values of your device(s) use <samp class="option">list_formats</samp>. Note that there is a FourCC <samp class="option">'pal '</samp> that can also be used as <samp class="option">pal</samp> (3 letters). Default behavior is autodetection of the input video format, if the hardware supports it. </dd> <dt><samp class="option">raw_format</samp></dt> <dd> Set the pixel format of the captured video. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> This is the default which means 8-bit YUV 422 or 8-bit ARGB if format autodetection is used, 8-bit YUV 422 otherwise. </dd> <dt>‘<samp class="samp">uyvy422</samp>’</dt> <dd> 8-bit YUV 422. </dd> <dt>‘<samp class="samp">yuv422p10</samp>’</dt> <dd> 10-bit YUV 422. </dd> <dt>‘<samp class="samp">argb</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class="samp">bgra</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class="samp">rgb10</samp>’</dt> <dd> 10-bit RGB. </dd> </dl>

</dd> <dt><samp class="option">teletext_lines</samp></dt> <dd> If set to nonzero, an additional teletext stream will be captured from the vertical ancillary data. Both SD PAL (576i) and HD (1080i or 1080p) sources are supported. In case of HD sources, OP47 packets are decoded. This option is a bitmask of the SD PAL VBI lines captured, specifically lines 6 to 22, and lines 318 to 335. Line 6 is the LSB in the mask. Selected lines which do not contain teletext information will be ignored. You can use the special <samp class="option">all</samp> constant to select all possible lines, or <samp class="option">standard</samp> to skip lines 6, 318 and 319, which are not compatible with all receivers. For SD sources, ffmpeg needs to be compiled with <code class="code">--enable-libzvbi</code>. For HD sources, on older (pre-4K) DeckLink card models you have to capture in 10 bit mode. </dd> <dt><samp class="option">channels</samp></dt> <dd> Defines number of audio channels to capture. Must be ‘<samp class="samp">2</samp>’, ‘<samp class="samp">8</samp>’ or ‘<samp class="samp">16</samp>’. Defaults to ‘<samp class="samp">2</samp>’. </dd> <dt><samp class="option">duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’, ‘<samp class="samp">one_sub_device_full</sa ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Defaults to ‘<samp class="samp">unset</samp>’. Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a

profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0): ‘<samp class="samp">one_sub_device_full</samp>’, ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’ </dd> <dt><samp class="option">timecode_format</samp><
<dd> Timecode type to include in the frame and video stream metadata. Must be ‘<samp class="samp">none</samp>’, ‘<samp class="samp">rp188vitc</samp>’, ‘<samp class="samp">rp188vitc2</samp>’, ‘<samp class="samp">rp188ltc</samp>’, ‘<samp class="samp">rp188hfr</samp>’, ‘<samp class="samp">rp188any</samp>’, ‘<samp class="samp">vitc</samp>’, ‘<samp class="samp">vitc2</samp>’, or ‘<samp class="samp">serial</samp>’. Defaults to ‘<samp class="samp">none</samp>’ (not included). In order to properly support 50/60 fps timecodes, the ordering of the queried timecode types for ‘<samp class="samp">rp188any</samp>’ is HFR, VITC1, VITC2 and LTC for >30 fps content. Note that this is slightly different to the ordering used by the DeckLink API, which is HFR, VITC1, LTC, VITC2. </dd> <dt><samp class="option">video_input</samp></dt> <dd> Sets the video input source. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">sdi</samp>’, ‘<samp class="samp">hdmi</samp>’, ‘<samp class="samp">optical_sdi</samp>’, ‘<samp class="samp">component</samp>’, ‘<samp class="samp">composite</samp>’ or ‘<samp class="samp">s_video</samp>’. Defaults to ‘<samp class="samp">unset</samp>’. </dd> <dt><samp class="option">audio_input</samp></dt> <dd> Sets the audio input source. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">embedded</samp>’, ‘<samp class="samp">aes_ebu</samp>’, ‘<samp class="samp">analog</samp>’, ‘<samp class="samp">analog_xlr</samp>’, ‘<samp class="samp">analog_rca</samp>’ or ‘<samp class="samp">microphone</samp>’. Defaults to ‘<samp class="samp">unset</samp>’. </dd> <dt><samp class="option">video_pts</samp></dt> <dd> Sets the video packet timestamp source. Must be ‘<samp class="samp">video</samp>’, ‘<samp class="samp">audio</samp>’, ‘<samp class="samp">reference</samp>’, ‘<samp class="samp">wallclock</samp>’ or ‘<samp class="samp">abs_wallclock</samp>’. Defaults to ‘<samp class="samp">video</samp>’. </dd> <dt><samp class="option">audio_pts</samp></dt> <dd> Sets the audio packet timestamp source. Must be ‘<samp class="samp">video</samp>’,

‘<samp class=”samp”>audio</samp>’, ‘<samp class=”samp”>reference</samp>’, ‘<samp class=”samp”>wallclock</samp>’ or ‘<samp class=”samp”>abs_wallclock</samp>’. Defaults to ‘<samp class=”samp”>audio</samp>’. </dd> <dt><samp class=”option”>draw_bars</samp></dt> <dd> If set to ‘<samp class=”samp”>true</samp>’, color bars are drawn in the event of a signal loss. Defaults to ‘<samp class=”samp”>true</samp>’. This option is deprecated, please use the <code class=”code”>signal_loss_action</code> option. </dd> <dt><samp class=”option”>signal_loss_action</samp></dt> <dd> Sets the action to take in the event of a signal loss. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>1, none</samp></dt> <dd> Do nothing on signal loss. This usually results in black frames. </dd> <dt><samp class=”option”>2, bars</samp></dt> <dd> Draw color bars on signal loss. Only supported for 8-bit input signals. </dd> <dt><samp class=”option”>3, repeat</samp></dt> <dd> Repeat the last video frame on signal loss. </dd> </dl>

Defaults to ‘<samp class=”samp”>bars</samp>’. </dd> <dt><samp class=”option”>queue_size</samp></dt> <dd> Sets maximum input buffer size in bytes. If the buffering reaches this value, incoming frames will be dropped. Defaults to ‘<samp class=”samp”>1073741824</samp>’. </dd> <dt><samp class=”option”>audio_depth</samp></dt> <dd> Sets the audio sample bit depth. Must be ‘<samp class=”samp”>16</samp>’ or ‘<samp class=”samp”>32</samp>’. Defaults to ‘<samp class=”samp”>16</samp>’. </dd> <dt><samp class=”option”>decklink_copyts</samp></
<dd> If set to <samp class=”option”>true</samp>, timestamps are forwarded as they are without removing the initial offset. Defaults to <samp class=”option”>false</samp>. </dd> <dt><samp class=”option”>timestamp_align</samp></dt> <dd> Capture start time alignment in seconds. If set to nonzero, input frames are dropped till the system timestamp aligns with configured value. Alignment difference of up to one frame duration is tolerated. This is useful for maintaining input synchronization across N different hardware devices deployed for ’N-way’ redundancy. The system time of different hardware devices should be synchronized with protocols such as NTP or PTP, before using this option. Note that this method is not foolproof. In some border cases input synchronization may not happen due to thread scheduling jitters in the OS. Either sync could go wrong by 1 frame or in a rarer case <samp class=”option”>timestamp_align</samp> seconds. Defaults to ‘<samp class=”samp”>0</samp>’. </dd> <dt><samp class=”option”>wait_for_tc (bool)</samp></dt> <dd> Drop frames till a frame with timecode is received. Sometimes serial timecode isn’t received with the first input frame. If that happens,

the stored stream timecode will be inaccurate. If this option is set to <samp class="option">true</samp>, input frames are dropped till a frame with timecode is received. Option <var class="var">timecode_format</var> must be specified. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">enable_klv(bool)</samp></dt> <dd> If set to <samp class="option">true</samp>, extracts KLV data from VANC and outputs KLV packets. KLV VANC packets are joined based on MID and PSC fields and aggregated into one KLV packet. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">3.4.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List input devices: <pre class="example-preformatted">ffmpeg -sources decklink </pre>

List supported formats: <pre class="example-preformatted">ffmpeg -f decklink -list_formats 1 -i 'Intensity Pro' </pre>

Capture video clip at 1080i50: <pre class="example-preformatted">ffmpeg -format_code Hi50 -f decklink -i 'Intensity Pro' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 10 bit: <pre class="example-preformatted">ffmpeg -raw_format yuv422p10 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 with 16 audio channels: <pre class="example-preformatted">ffmpeg -channels 16 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

 <h3 class="section">3.5 dshow# TOC</h3>

Windows DirectShow input device. DirectShow support is enabled when FFmpeg is built with the mingw-w64 project. Currently only audio and video devices are supported. Multiple devices may be opened as separate inputs, but they may also be opened on the same input, which should improve synchronism between them. The input name should be in the format: <pre class="example-preformatted"><var class="var">TYPE</var>=<var class="var">NAME</var>[:<var class="var">TYPE</var>=<var class="var">NAME</var>] </pre>

where <var class=”var”>TYPE</var> can be either <var class=”var”>audio</var> or <var class=”var”>video</var>, and <var class=”var”>NAME</var> is the device’s name or alternative name.. <h4 class=”subsection”>3.5.1 Options# TOC</h4>

If no options are specified, the device’s defaults are used.

If the device does not support the requested options, it will

fail to open.

<dl class=”table”>

<dt><samp class=”option”>video_size</samp></dt>

<dd> Set the video size in the captured video.

</dd>

<dt><samp class=”option”>framerate</samp></dt>

<dd> Set the frame rate in the captured video.

</dd>

<dt><samp class=”option”>sample_rate</samp></dt>

<dd> Set the sample rate (in Hz) of the captured audio.

</dd>

<dt><samp class=”option”>sample_size</samp></dt>

<dd> Set the sample size (in bits) of the captured audio.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set the number of channels in the captured audio.

</dd>

<dt><samp class=”option”>list_devices</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of devices and exit.

</dd>

<dt><samp class=”option”>list_options</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of selected device’s options

and exit.

</dd>

<dt><samp class=”option”>video_device_number</samp></dt>

<dd> Set video device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class=”option”>audio_device_number</samp></dt>

<dd> Set audio device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class=”option”>pixel_format</samp></dt>

<dd> Select pixel format to be used by DirectShow. This may only be set when

the video codec is not set or set to rawvideo.

</dd>

<dt><samp class=”option”>audio_buffer_size</samp></dt>

<dd> Set audio device buffer size in milliseconds (which can directly

impact latency, depending on the device).

Defaults to using the audio device’s

default buffer size (typically some multiple of 500ms).

Setting this value too low can degrade performance.

See also

us/library/windows/desktop/dd377582(v=vs.85).aspx

</dd>

<dt><samp class=”option”>video_pin_name</samp></dt>

<dd> Select video capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>audio_pin_name</samp></dt>

<dd> Select audio capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>crossbar_video_input_pin_number</samp></dt>

<dd> Select video input pin number for crossbar device. This will be

routed to the crossbar device’s Video Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>crossbar_audio_input_pin_number</samp></dt>

<dd> Select audio input pin number for crossbar device. This will be

routed to the crossbar device’s Audio Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_video_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts,
popup a display dialog

to the end user, allowing them to change video filter properties

and configurations manually.

Note that for crossbar devices, adjusting values in this dialog

may be needed at times to toggle between PAL (25 fps) and NTSC (29.97)

input frame rates, sizes, interlacing, etc. Changing these values can

enable different scan rates/frame rates and avoiding green bars at

the bottom, flickering scan lines, etc.

Note that with some devices, changing these properties can also affect future

invocations (sets new defaults) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_audio_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display dialog

to the end user, allowing them to change audio filter properties

and configurations manually.

</dd>

<dt><samp class=”option”>show_video_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens a video device.

</dd>

<dt><samp class=”option”>show_audio_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens an audio device.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV channels and frequencies.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_audio_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV audio (like mono vs. stereo, Language A,B or C).

</dd>

<dt><samp class=”option”>audio_device_load</samp></dt>

<dd> Load an audio capture filter device from file instead of searching

it by name. It may load additional parameters too, if the filter

supports the serialization of its properties to.

To use this an audio capture source has to be specified, but it can

be anything even fake one.

</dd>

<dt><samp class=”option”>audio_device_save</samp></dt>

<dd> Save the currently used audio capture filter device and its

parameters (if the filter supports it) to a file.

If a file with the same name exists it will be overwritten.

</dd>

<dt><samp class=”option”>video_device_load</samp></dt>

<dd> Load a video capture filter device from file instead of searching

it by name. It may load additional parameters too, if the filter

supports the serialization of its properties to.

To use this a video capture source has to be specified, but it can

be anything even fake one.

</dd>

<dt><samp class="option">video_device_save</samp></dt>

<dd> Save the currently used video capture filter device and its

parameters (if the filter supports it) to a file.

If a file with the same name exists it will be overwritten.

</dd>

<dt><samp class="option">use_video_device_timestamps</samp></dt>

<dd> If set to <samp class="option">false</samp>, the timestamp for video frames will be

derived from the wallclock instead of the timestamp provided by

the capture device. This allows working around devices that

provide unreliable timestamps.

</dd>

</dl>

 <h4 class="subsection">3.5.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of DirectShow supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f dshow -i dummy </pre>

Open video device <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -i video="Camera" </pre>

Open second video device with name <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -video_device_number 1 -i video="Camera" </pre>

Open video device <var class="var">Camera</var> and audio device <var class="var">Microphone</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -i video="Camera":audio="Microphone&q
</pre>

Print the list of supported options in selected device and exit: <pre class="example-preformatted">$ ffmpeg -list_options true -f dshow -i video="Camera" </pre>

Specify pin names to capture by name or alternative name, specify alternative device name: <pre class="example-preformatted">$ ffmpeg -f dshow -audio_pin_name "Audio Out" -video_pin_name 2 -i video=video="@device_pnp_\\?\pci#ven_1a0a&dev_6200&subsys_62
8f56-11d0-a3b9-00a0c9223196}\{ca465100-deb0-4d59-818f-8c477184adf6}":audio="Microphone&qu
</pre>

Configure a crossbar device, specifying crossbar pins, allow user to adjust video capture properties at startup: <pre class="example-preformatted">$ ffmpeg -f dshow -show_video_device_dialog true -crossbar_video_input_pin_number 0 -crossbar_audio_input_pin_number 3 -i video="AVerMedia BDA Analog Capture":audio="AVerMedia BDA Analog Capture" </pre>

 <h3 class="section">3.6 fbdev# TOC</h3>

Linux framebuffer input device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class="file">/dev/fb0</samp>. For more detailed information read the file Documentation/fb/framebuffer.txt included in the Linux source tree. See also http://linux-fbdev.sourceforge.net/, and fbset(1). To record from the framebuffer device <samp class="file">/dev/fb0</samp> with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f fbdev -framerate 10 -i /dev/fb0 out.avi </pre>

You can take a single screenshot image with the command: <pre class="example-preformatted">ffmpeg -f fbdev -framerate 1 -i /dev/fb0 -frames:v 1 screenshot.jpeg </pre>

 <h4 class="subsection">3.6.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">framerate</samp></dt>
<dd> Set the frame rate. Default is 25. </dd> </dl>

 <h3 class="section">3.7 gdigrab#
TOC</h3>

Win32 GDI-based screen capture device. This device allows you to capture a region of the display on Windows. Amongst options for the input filenames are such elements as: <pre class="example-preformatted">desktop </pre> or <pre class="example-preformatted">title=<var class="var">window_title</var> </pre> or <pre class="example-preformatted">hwnd=<var class="var">window_hwnd</var> </pre>

The first option will capture the entire desktop, or a fixed region of the desktop. The second and third options will instead capture the contents of a single window, regardless of its position on the screen. For example, to grab the entire desktop using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i desktop out.mpg </pre>

Grab a 640x480 region at position <code class="code">10,20</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -offset_x 10 -offset_y 20 -video_size vga -i desktop out.mpg </pre>

Grab the contents of the window named "Calculator" <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i title=Calculator out.mpg </pre>

 <h4 class="subsection">3.7.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">draw_mouse</samp></dt>
<dd> Specify whether to draw the mouse pointer. Use the value <code class="code">0</code> to not draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">framerate</samp></dt>
<dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt>
<dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. Note that <var class="var">show_region</var> is incompatible with grabbing the contents of

a single window. For example: <pre class="example-preformatted">ffmpeg -f gdigrab -show_region 1 -framerate 6 -video_size cif -offset_x 10 -offset_y 20 -i desktop out.mpg </pre>

</dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. The default is to capture the full screen if <samp class="file">desktop</samp> is selected, or the full window size if <samp class="file">title=<var class="var">window_title</var></samp> is selected. </dd> <dt><samp class="option">offset_x</samp></dt> <dd> When capturing a region with <var class="var">video_size</var>, set the distance from the left edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned to the left of your primary monitor, you will need to use a negative <var class="var">offset_x</var> value to move the region to that monitor. </dd> <dt><samp class="option">offset_y</samp></dt> <dd> When capturing a region with <var class="var">video_size</var>, set the distance from the top edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned above your primary monitor, you will need to use a negative <var class="var">offset_y</var> value to move the region to that monitor. </dd> </dl>

 <h3 class="section">3.8 iec61883# TOC</h3>

FireWire DV/HDV input device using libiec61883. To enable this input device, you need libiec61883, libraw1394 and libavc1394 installed on your system. Use the configure option <code class="code">--enable-libiec61883</code> to compile with the device enabled. The iec61883 capture device supports capturing from a video device connected via IEEE1394 (FireWire), using libiec61883 and the new Linux FireWire stack (juju). This is the default DV/HDV input method in Linux Kernel 2.6.37 and later, since the old FireWire stack was removed. Specify the FireWire port to be used as input file, or "auto" to choose the first port connected. <h4 class="subsection">3.8.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">dvtype</samp></dt> <dd> Override autodetection of DV/HDV. This should only be used if auto detection does not work, or if usage of a different device type should be prohibited. Treating a DV device as HDV (or vice versa) will not work and result in undefined behavior. The values <samp

class=”option”>auto</samp>, <samp class=”option”>dv</samp> and <samp class=”option”>hdv</samp> are supported. </dd> <dt><samp class=”option”>dvbuffer</samp></dt> <dd> Set maximum size of buffer for incoming data, in frames. For DV, this is an exact value. For HDV, it is not frame exact, since HDV does not have a fixed frame size. </dd> <dt><samp class=”option”>dvguid</samp></dt> <dd> Select the capture device by specifying its GUID. Capturing will only be performed from the specified device and fails if no device with the given GUID is found. This is useful to select the input if multiple devices are connected at the same time. Look at /sys/bus/firewire/devices to find out the GUIDs. </dd> </dl>

 <h4 class=”subsection”>3.8.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Grab and show the input of a FireWire DV/HDV device. <pre class=”example-preformatted”>ffplay -f iec61883 -i auto </pre>

Grab and record the input of a FireWire DV/HDV device, using a packet buffer of 100000 packets if the source is HDV. <pre class=”example-preformatted”>ffmpeg -f iec61883 -i auto -dvbuffer 100000 out.mpg </pre>

 <h3 class=”section”>3.9 jack# TOC</h3>

JACK input device. To enable this input device during configuration you need libjack installed on your system. A JACK input device creates one or more JACK writable clients, one for each audio channel, with name <var class=”var”>client_name</var>:input_<var class=”var”>N</var>, where <var class=”var”>client_name</var> is the name provided by the application, and <var class=”var”>N</var> is a number which identifies the channel. Each writable client will send the acquired data to the FFmpeg input device. Once you have created one or more JACK readable clients, you need to connect them to one or more JACK writable clients. To connect or disconnect JACK clients you can use the <code class=”command”>jack_connect</code> and <code class=”command”>jack_disconnect</code> programs, or do it through a graphical interface, for example with <code class=”command”>qjackctl</code>. To list the JACK clients and their properties you can invoke the command <code class=”command”>jack_lsp</code>. Follows an example which shows how to capture a JACK readable client with <code class=”command”>ffmpeg</code>. <pre

class=”example-preformatted”># Create a JACK writable client with name "ffmpeg". $ ffmpeg -f jack -i ffmpeg -y out.wav

Start the sample jack_metro readable client. $ jack_metro -b 120 -d 0.2 -f 4000

List the current JACK clients. $ jack_lsp -c system:capture_1 system:capture_2 system:playback_1 system:playback_2 ffmpeg:input_1 metro:120_bpm

Connect metro to the ffmpeg writable client. $ jack_connect metro:120_bpm ffmpeg:input_1 </pre>

For more information read: http://jackaudio.org/ <h4 class=”subsection”>3.9.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class=”section”>3.10 kmsgrab# TOC</h3>

KMS video input device. Captures the KMS scanout framebuffer associated with a specified CRTC or plane as a DRM object that can be passed to other hardware functions. Requires either DRM master or CAP_SYS_ADMIN to run. If you don’t understand what all of that means, you probably don’t want this. Look at <samp class=”option”>x11grab</samp> instead. <h4 class=”subsection”>3.10.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>device</samp></dt> <dd> DRM device to capture on. Defaults to <samp class=”option”>/dev/dri/card0</samp>. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Pixel format of the framebuffer. This can be autodetected if you are running Linux 5.7 or later, but needs to be provided for earlier versions. Defaults to <samp class=”option”>bgr0</samp>, which is the most common format used by the Linux console and Xorg X server. </dd> <dt><samp class=”option”>format_modifier</samp></dt> <dd> Format modifier to signal on output frames. This is necessary to import correctly into some APIs. It can be autodetected if you are running Linux 5.7 or later, but will need to be provided explicitly when needed in earlier versions. See the libdrm documentation for possible values. </dd> <dt><samp

class=”option”>crtc_id</samp></dt> <dd> KMS CRTC ID to define the capture source. The first active plane on the given CRTC will be used. </dd> <dt><samp class=”option”>plane_id</samp></dt> <dd> KMS plane ID to define the capture source. Defaults to the first active plane found if neither <samp class=”option”>crtc_id</samp> nor <samp class=”option”>plane_id</samp> are specified. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Framerate to capture at. This is not synchronised to any page flipping or framebuffer changes - it just defines the interval at which the framebuffer is sampled. Sampling faster than the framebuffer update rate will generate independent frames with the same content. Defaults to <code class=”code”>30</code>. </dd> </dl>

 <h4 class=”subsection”>3.10.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Capture from the first active plane, download the result to normal frames and encode. This will only work if the framebuffer is both linear and mappable - if not, the result may be scrambled or fail to download. <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwdownload,format=bgr0' output.mp4 </pre>

Capture from CRTC ID 42 at 60fps, map the result to VAAPI, convert to NV12 and encode as H.264. <pre class=”example-preformatted”>ffmpeg -crtc_id 42 -framerate 60 -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,scale_vaapi=w=1920:h=1080:format=nv12' -c:v h264_vaapi output.mp4 </pre>

To capture only part of a plane the output can be cropped - this can be used to capture a single window, as long as it has a known absolute position and size. For example, to capture and encode the middle quarter of a 1920x1080 plane: <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,crop=960:540:480:270,scale_vaapi=960:540:nv12' -c:v h264_vaapi output.mp4 </pre>

 <h3 class=”section”>3.11 lavfi# TOC</h3>

Libavfilter input virtual device. This input device reads data from the open output pads of a libavfilter filtergraph. For each filtergraph open output, the input device will create a corresponding stream which is mapped to the generated output. The filtergraph is specified through the option <samp class=”option”>graph</samp>. <a

name=”Options-10”> <h4 class=”subsection”>3.11.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>graph</samp></dt> <dd> Specify the filtergraph to use as input. Each video open output must be labelled by a unique string of the form "out<var class=”var”>N</var>", where <var class=”var”>N</var> is a number starting from 0 corresponding to the mapped input stream generated by the device. The first unlabelled output is automatically assigned to the "out0" label, but all the others need to be specified explicitly. The suffix "+subcc" can be appended to the output label to create an extra stream with the closed captions packets attached to that output (experimental; only for EIA-608 / CEA-708 for now). The subcc streams are created after all the normal streams, in the order of the corresponding stream. For example, if there is "out19+subcc", "out7+subcc" and up to "out42", the stream #43 is subcc for stream #7 and stream #44 is subcc for stream #19. If not specified defaults to the filename specified for the input device. </dd> <dt><samp class=”option”>graph_file</samp></dt> <dd> Set the filename of the filtergraph to be read and sent to the other filters. Syntax of the filtergraph is the same as the one specified by the option <var class=”var”>graph</var>. </dd> <dt><samp class=”option”>dumpgraph</samp></dt> <dd> Dump graph to stderr. </dd> </dl>

 <h4 class=”subsection”>3.11.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Create a color video stream and play it back with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi -graph "color=c=pink [out0]" dummy </pre>

As the previous example, but use filename for specifying the graph description, and omit the "out0" label: <pre class=”example-preformatted”>ffplay -f lavfi color=c=pink </pre>

Create three different video test filtered sources and play them: <pre class=”example-preformatted”>ffplay -f lavfi -graph "testsrc [out0]; testsrc,hflip [out1]; testsrc,negate [out2]" test3 </pre>

Read an audio stream from a file using the amovie source and play it back with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi "amovie=test.wav" </pre>

Read an audio stream and a video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "movie=test.avi[out0];amovie=test.wav[out1]" </pre>

Dump decoded frames to images and Closed Captions to an RCWT backup: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=test.ts[out0+subcc]" -map v frame%08d.png -map s -c copy -f rcwt subcc.bin </pre>

 <h3 class="section">3.12 libcdio# TOC</h3>

Audio-CD input device based on libcdio. To enable this input device during configuration you need libcdio installed on your system. It requires the configure option <code class="code">--enable-libcdio</code>. This device allows playing and grabbing from an Audio-CD. For example to copy with <code class="command">ffmpeg</code> the entire Audio-CD in <samp class="file">/dev/sr0</samp>, you may run the command: <pre class="example-preformatted">ffmpeg -f libcdio -i /dev/sr0 cd.wav </pre>

 <h4 class="subsection">3.12.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">speed</samp></dt> <dd> Set drive reading speed. Default value is 0. The speed is specified CD-ROM speed units. The speed is set through the libcdio <code class="code">cdio_cddap_speed_set</code> function. On many CD-ROM drives, specifying a value too large will result in using the fastest speed. </dd> <dt><samp class="option">paranoia_mode</samp></dt> <dd> Set paranoia recovery mode flags. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dt>‘<samp class="samp">verify</samp>’</dt> <dt>‘<samp class="samp">overlap</samp>’</dt> <dt>‘<samp class="samp">neverskip</samp>’</dt> <dt>‘<samp class="samp">full</samp>’</dt> </dl>

Default value is ‘<samp class="samp">disable</samp>’. For more information about the available recovery modes, consult the paranoia project documentation. </dd> </dl>

 <h3 class="section">3.13 libdc1394# TOC</h3>

IIDC1394 input device, based on libdc1394 and libraw1394. Requires the configure option <code class="code">--enable-libdc1394</code>. <h4 class="subsection">3.13.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set the frame rate. Default is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Select the pixel format. Default is <code class="code">uyvy422</code>. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video size given as a string such as <code class="code">640x480</code> or <code class="code">hd720</code>. Default is <code class="code">qvga</code>. </dd> </dl>

 <h3 class="section">3.14 openal# TOC</h3>

The OpenAL input device provides audio capture on all systems with a working OpenAL 1.1 implementation. To enable this input device during configuration, you need OpenAL headers and libraries installed on your system, and need to configure FFmpeg with <code class="code">--enable-openal</code>. OpenAL headers and libraries should be provided as part of your OpenAL implementation, or as an additional download (an SDK). Depending on your installation you may need to specify additional flags via the <code class="code">--extra-cflags</code> and <code class="code">--extra-ldflags</code> for allowing the build system to locate the OpenAL headers and libraries. An incomplete list of OpenAL implementations follows: <dl class="table"> <dt>Creative</dt> <dd> The official Windows implementation, providing hardware acceleration with supported devices and software fallback. See http://openal.org/. </dd> <dt>OpenAL Soft</dt> <dd> Portable, open source (LGPL) software implementation. Includes backends for the most common sound APIs on the Windows, Linux, Solaris, and BSD operating systems. See <a class="url" href="http://kcat.strangesoft.net/openal.html">http://kcat.strangesoft.net/openal.html </dd> <dt>Apple</dt> <dd> OpenAL is part of Core Audio, the official Mac OS X Audio interface. See http://developer.apple.com/technologies/mac/audio-and-video.html </dd> </dl>

This device allows one to capture from an audio input device handled through OpenAL. You need to specify the name of the device to

capture in the provided filename. If the empty string is provided, the device will automatically select the default device. You can get the list of the supported devices by using the option <var class="var">list_devices</var>. <h4 class="subsection">3.14.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels in the captured audio. Only the values <samp class="option">1</samp> (monaural) and <samp class="option">2</samp> (stereo) are currently supported. Defaults to <samp class="option">2</samp>. </dd> <dt><samp class="option">sample_size</samp></dt> <dd> Set the sample size (in bits) of the captured audio. Only the values <samp class="option">8</samp> and <samp class="option">16</samp> are currently supported. Defaults to <samp class="option">16</samp>. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate (in Hz) of the captured audio. Defaults to <samp class="option">44.1k</samp>. </dd> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">3.14.2 Examples# TOC</h4>

Print the list of OpenAL supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f openal -i dummy out.ogg </pre>

Capture from the OpenAL device <samp class="file">DR-BT101 via PulseAudio</samp>: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out.ogg </pre>

Capture from the default device (note the empty string ” as filename): <pre class="example-preformatted">$ ffmpeg -f openal -i '' out.ogg </pre>

Capture from two devices simultaneously, writing to two different files, within the same <code class="command">ffmpeg</code> command: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out1.ogg -f openal -i 'ALSA Default' out2.ogg </pre> Note: not all OpenAL implementations support multiple simultaneous capture - try the latest OpenAL Soft if the above does not work. <h3 class="section">3.15

oss# TOC</h3>

Open Sound System input device. The filename to provide to the input device is the device node representing the OSS input device, and is usually set to <samp class=”file”>/dev/dsp</samp>. For example to grab from <samp class=”file”>/dev/dsp</samp> using <code class=”command”>ffmpeg</code> use the command: <pre class=”example-preformatted”>ffmpeg -f oss -i /dev/dsp /tmp/oss.wav </pre>

For more information about OSS see: <a class=”url” href=”http://manuals.opensound.com/usersguide/ds
  <h4 class=”subsection”>3.15.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class=”section”>3.16 pulse# TOC</h3>

PulseAudio input device. To enable this output device you need to configure FFmpeg with <code class=”code”>--enable-libpulse</code>. The filename to provide to the input device is a source device or the string "default" To list the PulseAudio source devices and their properties you can invoke the command <code class=”command”>pactl list sources</code>. More information about PulseAudio can be found on http://www.pulseaudio.org. <h4 class=”subsection”>3.16.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class=”option”>name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class=”code”>LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class=”option”>stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active streams, by default it is "record". </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Specify the sam-

plerate in Hz, by default 48kHz is used. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Specify the channels in use, by default 2 (stereo) is set. </dd> <dt><samp class=”option”>frame_size</samp></dt> <dd> This option does nothing and is deprecated. </dd> <dt><samp class=”option”>fragment_size</samp></dt> <dd> Specify the size in bytes of the minimal buffering fragment in PulseAudio, it will affect the audio latency. By default it is set to 50 ms amount of data. </dd> <dt><samp class=”option”>wallclock</samp></dt> <dd> Set the initial PTS using the current time. Default is 1. </dd> </dl>

 <h4 class=”subsection”>3.16.2 Examples# TOC</h4> Record a stream from default device: <pre class=”example-preformatted”>ffmpeg -f pulse -i default /tmp/pulse.wav </pre>

 <h3 class=”section”>3.17 sndio# TOC</h3>

sndio input device. To enable this input device during configuration you need libsndio installed on your system. The filename to provide to the input device is the device node representing the sndio input device, and is usually set to <samp class=”file”>/dev/audio0</samp>. For example to grab from <samp class=”file”>/dev/audio0</samp> using <code class=”command”>ffmpeg</code> use the command: <pre class=”example-preformatted”>ffmpeg -f sndio -i /dev/audio0 /tmp/oss.wav </pre>

 <h4 class=”subsection”>3.17.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class=”section”>3.18 video4linux2, v4l2# TOC</h3

Video4Linux2 input video device. "v4l2" can be used as alias for "video4linux2". If FFmpeg is built with v4l-utils support (by using the <code class=”code”>--enable-libv4l2</code>

configure option), it is possible to use it with the <code class="code">-use_libv4l2</code> input device option. The name of the device to grab is a file device node, usually Linux systems tend to automatically create such nodes when the device (e.g. an USB webcam) is plugged into the system, and has a name of the kind <samp class="file">/dev/video<var class="var">N</var></samp>, where <var class="var">N</var> is a number associated to the device. Video4Linux2 devices usually support a limited set of <var class="var">width</var>x<var class="var">height</var> sizes and frame rates. You can check which are supported using <code class="command">-list_formats all</code> for Video4Linux2 devices. Some devices, like TV cards, support one or more standards. It is possible to list all the supported standards using <code class="command">-list_standards all</code>. The time base for the timestamps is 1 microsecond. Depending on the kernel version and configuration, the timestamps may be derived from the real time clock (origin at the Unix Epoch) or the monotonic clock (origin usually at boot time, unaffected by NTP or manual changes to the clock). The <samp class="option">-timestamps abs</samp> or <samp class="option">-ts abs</samp> option can be used to force conversion into the real time clock. Some usage examples of the video4linux2 device with <code class="command">ffmpeg</code> and <code class="command">ffplay</code>: <ul class="itemize mark-bullet"> List supported formats for a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -list_formats all /dev/video0 </pre>

Grab and show the input of a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -framerate 30 -video_size hd720 /dev/video0 </pre>

Grab and record the input of a video4linux2 device, leave the frame rate and size as previously set: <pre class="example-preformatted">ffmpeg -f video4linux2 -input_format mjpeg -i /dev/video0 out.mpeg </pre>

For more information about Video4Linux, check http://linuxtv.org/. <h4 class="subsection">3.18.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">standard</samp></dt> <dd> Set the standard. Must be the name of a supported standard. To get a list of the supported standards, use the <samp class="option">list_standards</samp> option. </dd> <dt><samp class="option">channel</samp></dt> <dd> Set the input channel number. Default to -1, which means using the previously selected channel. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. The argument must be a string in the form <var class="var">WIDTH</var>x<var

class=”var”>HEIGHT</var> or a valid size abbreviation. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Select the pixel format (only valid for raw video input). </dd> <dt><samp class=”option”>input_format</samp></dt> <dd> Set the preferred pixel format (for raw video) or a codec name. This option allows one to select the input format, when several are available. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the preferred video frame rate. </dd> <dt><samp class=”option”>list_formats</samp></dt> <dd> List available formats (supported pixel formats, codecs, and frame sizes) and exit. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Show all available (compressed and non-compressed) formats. </dd> <dt>‘<samp class=”samp”>raw</samp>’</dt> <dd> Show only raw video (non-compressed) formats. </dd> <dt>‘<samp class=”samp”>compressed</samp>’</dt> <dd> Show only compressed formats. </dd> </dl>

</dd> <dt><samp class=”option”>list_standards</samp></dt> <dd> List supported standards and exit. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Show all supported standards. </dd> </dl>

</dd> <dt><samp class=”option”>timestamps, ts</samp></dt> <dd> Set type of timestamps for grabbed frames. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt> <dd> Use timestamps from the kernel. </dd> <dt>‘<samp class=”samp”>abs</samp>’</dt> <dd> Use absolute timestamps (wall clock). </dd> <dt>‘<samp class=”samp”>mono2abs</samp>’</dt> <dd> Force conversion from monotonic to absolute timestamps. </dd> </dl>

Default value is <code class=”code”>default</code>. </dd> <dt><samp class=”option”>use_libv4l2</samp></dt> <dd> Use libv4l2 (v4l-utils) conversion functions. Default is 0. </dd> </dl>

 <h3 class=”section”>3.19 vfwcap# TOC</h3>

VfW (Video for Windows) capture input device. The filename passed as input is the capture driver number, ranging from 0 to 9. You may use "list" as filename to print a list of drivers. Any other filename will be interpreted as device number 0. <h4 class=”subsection”>3.19.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>video_size</samp></dt>

<dd> Set the video frame size. </dd> <dt><samp class="option">framerate</samp></dt>
<dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>,
corresponding to a frame rate of <code class="code">30000/1001</code>.
 </dd> </dl>

 <h3 class="section">3.20 x11grab# TOC</h3>

X11 video input device. To enable this input device during configuration you need libxcb installed on your system. It will be automatically detected during configuration. This device allows one to capture a region of an X11 display. The filename passed as input has the syntax: <pre class="example-preformatted">[<var class="var">hostname</var>]:<var class="var">display_number</var>.<var class="var">screen_number</var>[+<var class="var">x_offset</var>,<var class="var">y_offset</var>] </pre>

<var class="var">hostname</var>:<var class="var">display_number</var>.<var class="var">screen_number</var> specifies the X11 display name of the screen to grab from. <var class="var">hostname</var> can be omitted, and defaults to "localhost". The environment variable <code class="env">DISPLAY</code> contains the default display name.
<var class="var">x_offset</var> and <var class="var">y_offset</var> specify the offsets of the grabbed area with respect to the top-left border of the X11 screen. They default to 0. Check the X11 documentation (e.g. <code class="command">man X</code>) for more detailed information. Use the <code class="command">xdpyinfo</code> program for getting basic information about the properties of your X11 display (e.g. grep for "name" or "dimensions").
For example to grab from <samp class="file">:0.0</samp> using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

Grab at position <code class="code">10,20</code>: <pre class="example-preformatted">ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

 <h4 class="subsection">3.20.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">select_region</samp></dt>
<dd> Specify whether to select the grabbing area graphically using the pointer. A value of <code class="code">1</code> prompts the user to select the grabbing area graphically by clicking and drag-

ging. A single click with no dragging will select the whole screen. A region with zero width or height will also select the whole screen. This option overwrites the <var class="var">video_size</var>, <var class="var">grab_x</var>, and <var class="var">grab_y</var> options. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. A value of <code class="code">0</code> specifies not to draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">follow_mouse</samp></dt> <dd> Make the grabbed area follow the mouse. The argument can be <code class="code">centered</code> or a number of pixels <var class="var">PIXELS</var>. When it is specified with "centered", the grabbing region follows the mouse pointer and keeps the pointer at the center of region; otherwise, the region follows only when the mouse pointer reaches within <var class="var">PIXELS</var> (greater than zero) to the edge of region. For example: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse centered -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

To follow only when the mouse pointer reaches within 100 pixels to edge: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse 100 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. </dd> <dt><samp class="option">region_border</samp></dt> <dd> Set the region border thickness if <samp class="option">-show_region 1</samp> is used. Range is 1 to 128 and default is 3 (XCB-based x11grab only). For example: <pre class="example-preformatted">ffmpeg -f x11grab -show_region 1 -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

With <var class="var">follow_mouse</var>: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse centered -show_region 1 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">window_id</samp></dt> <dd> Grab this window, instead of the whole screen. Default value is 0, which maps to the whole screen (root window). The id of a window can be found using the <code class="command">xwininfo</code>

program, possibly with options -tree and -root. If the window is later enlarged, the new area is not recorded. Video ends when the window is closed, unmapped (i.e., iconified) or shrunk beyond the video size (which defaults to the initial window size). This option disables options <samp class="option">follow_mouse</samp> and <samp class="option">select_region</samp>. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. Default is the full desktop or window. </dd> <dt><samp class="option">grab_x</samp></dt> <dt><samp class="option">grab_y</samp></dt> <dd> Set the grabbing region coordinates. They are expressed as offset from the top left corner of the X11 window and correspond to the <var class="var">x_offset</var> and <var class="var">y_offset</var> parameters in the device name. The default value for both options is 0. </dd> </dl>

 <h2 class="chapter">4 Output Devices# TOC</h2>

Output devices are configured elements in FFmpeg that can write multimedia data to an output device attached to your system. When you configure your FFmpeg build, all the supported output devices are enabled by default. You can list all available ones using the configure option "–list-outdevs". You can disable all the output devices using the configure option "–disable-outdevs", and selectively enable an output device using the option "–enable-outdev=<var class="var">OUTDEV</var>", or you can disable a particular input device using the option "–disable-outdev=<var class="var">OUTDEV</var>". The option "-devices" of the ff* tools will display the list of enabled output devices. A description of the currently available output devices follows. <h3 class="section">4.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) output device. <h4 class="subsection">4.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Play a file on default ALSA device: <pre class="example-preformatted">ffmpeg -i INPUT -f alsa default </pre>

Play a file on soundcard 1, audio device 7:

<pre class="example-preformatted">ffmpeg -i INPUT -f alsa hw:1,7
</pre>

 <h3 class="section">4.2 AudioToolbox# TOC</h3>

AudioToolbox output device. Allows native output to CoreAudio devices on OSX. The output filename can be empty (or <code class="code">-</code>) to refer to the default system output device or a number that refers to the device index as shown using: <code class="code">-list_devices true</code>. Alternatively, the audio input device can be chosen by index using the <samp class="option">-audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class="option">-list_devices true</samp>, listing all device names, UIDs and corresponding indices. <h4 class="subsection">4.2.1 Options# TOC</h4>

AudioToolbox supports the following options: <dl class="table"> <dt><samp class="option">-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the output filename. </dd> </dl>

 <h4 class="subsection">4.2.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of supported devices and output a sine wave to the default device: <pre class="example-preformatted">$ ffmpeg -f lavfi -i sine=r=44100 -f audiotoolbox -list_devices true - </pre>

Output a sine wave to the device with the index 2, overriding any output filename: <pre class="example-preformatted">$ ffmpeg -f lavfi -i sine=r=44100 -f audiotoolbox -audio_device_index 2 -
</pre>

 <h3 class="section">4.3 caca# TOC</h3>

CACA output device. This output device allows one

to show a video stream in CACA window. Only one CACA window is allowed per application, so you can have only one instance of this output device in an application. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libcaca</code>. libcaca is a graphics library that outputs text instead of pixels. For more information about libcaca, check: http://caca.zoy.org/wiki/libcaca <h4 class="subsection">4.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">window_title</samp></dt> <dd> Set the CACA window title, if not specified default to the filename specified for the output device. </dd> <dt><samp class="option">window_size</samp></dt> <dd> Set the CACA window size, can be a string of the form <var class="var">width</var>x<var class="var">height</var> or a video size abbreviation. If not specified it defaults to the size of the input video. </dd> <dt><samp class="option">driver</samp></dt> <dd> Set display driver. </dd> <dt><samp class="option">algorithm</samp></dt> <dd> Set dithering algorithm. Dithering is necessary because the picture being rendered has usually far more colours than the available palette. The accepted values are listed with <code class="code">-list_dither algorithms</code>. </dd> <dt><samp class="option">antialias</samp></dt> <dd> Set antialias method. Antialiasing smoothens the rendered image and avoids the commonly seen staircase effect. The accepted values are listed with <code class="code">-list_dither antialiases</code>. </dd> <dt><samp class="option">charset</samp></dt> <dd> Set which characters are going to be used when rendering text. The accepted values are listed with <code class="code">-list_dither charsets</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Set color to be used when rendering text. The accepted values are listed with <code class="code">-list_dither colors</code>. </dd> <dt><samp class="option">list_drivers</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of available drivers and exit. </dd> <dt><samp class="option">list_dither</samp></dt> <dd> List available dither options related to the argument. The argument must be one of <code class="code">algorithms</code>, <code class="code">antialiases</code>, <code class="code">charsets</code>, <code class="code">colors</code>. </dd> </dl>

 <h4 class="subsection">4.3.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> The following command shows the <code class="command">ffmpeg</code> output is an CACA window, forcing its size to 80x25: <pre class="example-preformatted">ffmpeg -i INPUT -c:v rawvideo -pix_fmt rgb24 -window_size 80x25 -f caca - </pre>

Show the list of available drivers and exit: <pre class="example-preformatted">ffmpeg -i INPUT -pix_fmt rgb24 -f caca -list_drivers true - </pre>

Show the list of available dither colors and exit: <pre class="example-preformatted">ffmpeg -i INPUT -pix_fmt rgb24 -f caca -list_dither colors - </pre>

 <h3 class="section">4.4 decklink# TOC</h3>

The decklink output device provides playback capabilities for Blackmagic DeckLink devices. To enable this output device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class="code">--extra-cflags</code>, <code class="code">--extra-cxxflags</code> and <code class="code">--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class="command">widl</code>. DeckLink is very picky about the formats it supports. Pixel format is always uyvy422, framerate, field order and video size must be determined for your device with <code class="command">-list_formats 1</code>. Audio sample rate is always 48 kHz. <h4 class="subsection">4.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. This option is deprecated, please use the <code class="code">-sinks</code> option of ffmpeg to list the available output devices. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of supported formats and exit. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">preroll</samp></dt> <dd> Amount of time to preroll video in seconds. Defaults to <samp class="option">0.5</samp>. </dd> <dt><samp class="option">duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">half</samp>’,

‘<samp class="samp">full</samp>’, ‘<samp class="samp">one_sub_device_full</sa
‘<samp class="samp">one_sub_device_half</samp>’,
‘<samp class="samp">two_sub_device_full</samp>’,
‘<samp class="samp">four_sub_device_half</samp>’ Defaults to ‘<samp class="samp">unset</samp>’.
Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0): ‘<samp class="samp">one_sub_device_full</samp>’,
‘<samp class="samp">one_sub_device_half</samp>’,
‘<samp class="samp">two_sub_device_full</samp>’,
‘<samp class="samp">four_sub_device_half</samp>’
 Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’ </dd> <dt><samp class="option">timing_offset</samp></dt
<dd> Sets the genlock timing pixel offset on the used output. Defaults to ‘<samp class="samp">unset</samp>’. </dd>
<dt><samp class="option">link</samp></dt> <dd> Sets the SDI video link configuration on the used output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">single</samp>’ link SDI, ‘<samp class="samp">dual</samp>’ link SDI or ‘<samp class="samp">quad</samp>’ link SDI. Defaults to ‘<samp class="samp">unset</samp>’.
</dd> <dt><samp class="option">sqd</samp></dt> <dd> Enable Square Division Quad Split mode for Quad-link SDI output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">true</samp>’ or ‘<samp class="samp">false</samp>’. Defaults to <samp class="option">unset</samp>. </dd> <dt><samp class="option">level_a</samp></dt> <dd> Enable SMPTE Level A mode on the used output. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">true</samp>’ or ‘<samp class="samp">false</samp>’. Defaults to <samp class="option">unset</samp>.
 </dd> <dt><samp class="option">vanc_queue_size</samp></dt>
<dd> Sets maximum output buffer size in bytes for VANC data. If the buffering reaches this value, outgoing VANC data will be dropped. Defaults to ‘<samp class="samp">1048576</samp>’. </dd>
</dl>

 <h4 class="subsection">4.4.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List output devices:

<pre class="example-preformatted">ffmpeg -sinks decklink </pre>

List supported formats: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -list_formats 1 'DeckLink Mini Monitor' </pre>

Play video clip: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -pix_fmt uyvy422 'DeckLink Mini Monitor' </pre>

Play video clip with non-standard framerate or video size: <pre class="example-preformatted">ffmpeg -i test.avi -f decklink -pix_fmt uyvy422 -s 720x486 -r 24000/1001 'DeckLink Mini Monitor' </pre>

 <h3 class="section">4.5 fbdev# TOC</h3>

Linux framebuffer output device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class="file">/dev/fb0</samp>. For more detailed information read the file <samp class="file">Documentation/fb/framebuffer.txt</samp> included in the Linux source tree. <h4 class="subsection">4.5.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">xoffset</samp></dt> <dt><samp class="option">yoffset</samp></dt> <dd> Set x/y coordinate of top left corner. Default is 0. </dd> </dl>

 <h4 class="subsection">4.5.2 Examples# TOC</h4> Play a file on framebuffer device <samp class="file">/dev/fb0</samp>. Required pixel format depends on current framebuffer settings. <pre class="example-preformatted">ffmpeg -re -i INPUT -c:v rawvideo -pix_fmt bgra -f fbdev /dev/fb0 </pre>

See also http://linux-fbdev.sourceforge.net/, and fbset(1). <h3 class="section">4.6 oss# TOC</h3>

OSS (Open Sound System) output device. <a name="pulse-

1”> <h3 class=”section”>4.7 pulse# TOC</h3>

PulseAudio output device. To enable this output device you need to configure FFmpeg with <code class=”code”>--enable-libpulse</code>. More information about PulseAudio can be found on http://www.pulseaudio.org <h4 class=”subsection”>4.7.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class=”option”>name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class=”code”>LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class=”option”>stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active streams, by default it is set to the specified output name. </dd> <dt><samp class=”option”>device</samp></dt> <dd> Specify the device to use. Default device is used when not provided. List of output devices can be obtained with command <code class=”command”>pactl list sinks</code>. </dd> <dt><samp class=”option”>buffer_size</samp></dt> <dt><samp class=”option”>buffer_duration</samp></dt> <dd> Control the size and duration of the PulseAudio buffer. A small buffer gives more control, but requires more frequent updates. <samp class=”option”>buffer_size</samp> specifies size in bytes while <samp class=”option”>buffer_duration</samp> specifies duration in milliseconds. When both options are provided then the highest value is used (duration is recalculated to bytes using stream parameters). If they are set to 0 (which is default), the device will use the default PulseAudio duration value. By default PulseAudio set buffer duration to around 2 seconds. </dd> <dt><samp class=”option”>prebuf</samp></dt> <dd> Specify pre-buffering size in bytes. The server does not start with playback before at least <samp class=”option”>prebuf</samp> bytes are available in the buffer. By default this option is initialized to the same value as <samp class=”option”>buffer_size</samp> or <samp class=”option”>buffer_duration</samp> (whichever is bigger). </dd> <dt><samp class=”option”>minreq</samp></dt> <dd> Specify minimum request size in bytes. The server does not request less than <samp class=”option”>minreq</samp> bytes from the client, instead waits until the buffer is free enough to request more bytes at once. It is recommended to not set this option, which will initialize this to a value that is deemed sensible by the server. </dd> </dl>

 <h4 class=”subsection”>4.7.2 Exam-

ples# TOC</h4> Play a file on default device on default server: <pre class="example-preformatted">ffmpeg -i INPUT -f pulse "stream name" </pre>

 <h3 class="section">4.8 sndio#
TOC</h3>

sndio audio output device. <h3 class="section">4.9 v4l2# TOC</h3>

Video4Linux2 output device. <h3 class="section">4.10 xv# TOC</h3>

XV (XVideo) output device. This output device allows one to show a video stream in a X Window System window. <h4 class="subsection">4.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">display_name</samp></dt>
<dd> Specify the hardware display name, which determines the display and communications domain to be used. The display name or DISPLAY environment variable can be a string in the format <var class="var">hostname</var>[:<var class="var">number</var>[.<var class="var">screen_number</var>]]. <var class="var">hostname</var> specifies the name of the host machine on which the display is physically attached. <var class="var">number</var> specifies the number of the display server on that host machine. <var class="var">screen_number</var> specifies the screen to be used on that server. If unspecified, it defaults to the value of the DISPLAY environment variable. For example, <code class="code">dual-headed:0.1</code> would specify screen 1 of display 0 on the machine named “dual-headed”. Check the X11 specification for more detailed information about the display name format. </dd> <dt><samp class="option">window_id</samp></dt> <dd> When set to non-zero value then device doesn’t create new window, but uses existing one with provided <var class="var">window_id</var>. By default this options is set to zero and device creates its own window. </dd> <dt><samp class="option">window_size</samp></dt>

<dd> Set the created window size, can be a string of the form <var class="var">width</var>x<var class="var">height</var> or a video size abbreviation. If not specified it defaults to the size of the input video. Ignored when <var class="var">window_id</var> is set. </dd> <dt><samp class="option">window_x</samp></dt> <dt><samp class="option">window_y</samp></dt> <dd> Set the X and Y window offsets for the created window. They are both set to 0 by default. The values may be ignored by the window manager. Ignored when <var class="var">window_id</var> is set. </dd> <dt><samp class="option">window_title</samp></dt> <dd> Set the window title, if not specified default to the filename specified for the output device. Ignored when <var class="var">window_id</var> is set. </dd> </dl>

For more information about XVideo see http://www.x.org/</a
  <h4 class="subsection">4.10.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Decode, display and encode video input with <code class="command">ffmpeg</code> at the same time: <pre class="example-preformatted">ffmpeg -i INPUT OUTPUT -f xv display </pre>

Decode and display the input video to multiple X11 windows: <pre class="example-preformatted">ffmpeg -i INPUT -f xv normal -vf negate -f xv negated </pre>

 <h2 class="chapter">5 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavdevice

 <h2 class="chapter">6 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using <a

class=”uref” href=”https://www.gnu.org/software/texinfo/”>makeinfo.
</p> </body> </html>

===== .\ffmpeg\doc\ffmpeg-filters.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> FFmpeg Filters Documentation </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> FFmpeg Filters Documentation
</h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”>

1 Description

2 Filtering Introduction

3 graph2dot

4 Filtergraph description

<ul class=”toc-numbered-mark”>

4.1 Filtergraph syntax

4.2 Notes on filtergraph escaping

5 Timeline editing

6 Changing options at runtime with a command

7 Options for filters with several inputs (framesync)

8 Audio Filters

<ul class="toc-numbered-mark">
8.1 aap
8.2 acompressor
<ul class="toc-numbered-mark">
8.2.1 Commands

8.3 acontrast
8.4 acopy
8.5 acrossfade
<ul class="toc-numbered-mark">
8.5.1 Examples

8.6 acrossover
<ul class="toc-numbered-mark">
8.6.1 Examples

8.7 acrusher
<ul class="toc-numbered-mark">
8.7.1 Commands

8.8 acue
8.9 adeclick
8.10 adeclip
8.11 adecorrelate
8.12 adelay
<ul class="toc-numbered-mark">
8.12.1 Examples

8.13 adenorm
<ul class="toc-numbered-mark">
8.13.1 Commands

8.14 aderivative, aintegral

8.15 adrc

<ul class="toc-numbered-mark">

8.15.1 Commands

8.15.2 Examples

8.16 adynamicequalizer

<ul class="toc-numbered-mark">

8.16.1 Commands

8.17 adynamicsmooth

<ul class="toc-numbered-mark">

8.17.1 Commands

8.18 aecho

<ul class="toc-numbered-mark">

8.18.1 Examples

8.19 aemphasis

<ul class="toc-numbered-mark">

8.19.1 Commands

8.20 aeval

<ul class="toc-numbered-mark">

8.20.1 Examples

8.21 aexciter

<ul class="toc-numbered-mark">

8.21.1 Commands

8.22 afade
<ul class="toc-numbered-mark">
8.22.1 Commands
8.22.2 Examples

8.23 afftdn
<ul class="toc-numbered-mark">
8.23.1 Commands
8.23.2 Examples

8.24 afftfilt
<ul class="toc-numbered-mark">
8.24.1 Examples

8.25 afir
<ul class="toc-numbered-mark">
8.25.1 Examples

8.26 aformat
8.27 afreqshift
<ul class="toc-numbered-mark">
8.27.1 Commands

8.28 afwtdn
<ul class="toc-numbered-mark">
8.28.1 Commands

8.29 agate

<ul class="toc-numbered-mark">

8.29.1 Commands

8.30 aiir

<ul class="toc-numbered-mark">

8.30.1 Examples

8.31 alimiter

8.32 allpass

<ul class="toc-numbered-mark">

8.32.1 Commands

8.33 aloop

8.34 amerge

<ul class="toc-numbered-mark">

8.34.1 Examples

8.35 amix

<ul class="toc-numbered-mark">

8.35.1 Examples

8.35.2 Commands

8.36 amultiply

8.37 anequalizer

<ul class="toc-numbered-mark">

8.37.1 Examples

8.37.2 Commands

8.38 anlmdn

<ul class="toc-numbered-mark">

8.38.1 Commands

8.39 anlmf, anlms

<ul class="toc-numbered-mark">

8.39.1 Examples

8.39.2 Commands

8.40 anull

8.41 apad

<ul class="toc-numbered-mark">

8.41.1 Examples

8.42 aphaser

8.43 aphaseshift

<ul class="toc-numbered-mark">

8.43.1 Commands

8.44 apsnr

8.45 apsyclip

<ul class="toc-numbered-mark">

8.45.1 Commands

8.46 apulsator

8.47 aresample

<ul class="toc-numbered-mark">

8.47.1 Examples

8.48 areverse

<ul class="toc-numbered-mark">

8.48.1 Examples

8.49 arls

8.50 arnndn

<ul class="toc-numbered-mark">

8.50.1 Commands

8.51 asdr

8.52 asetnsamples

8.53 asetrate

8.54 ashowinfo

8.55 asisdr

8.56 asoftclip

<ul class="toc-numbered-mark">

8.56.1 Commands

8.57 aspectralstats

8.58 asr

8.59 astats

8.60 asubboost

<ul class="toc-numbered-mark">

8.60.1 Commands

8.61 asubcut

<ul class="toc-numbered-mark">

8.61.1 Commands

8.62 asupercut

<ul class="toc-numbered-mark">

8.62.1 Commands

8.63 asuperpass

<ul class="toc-numbered-mark">

8.63.1 Commands

8.64 asuperstop

<ul class="toc-numbered-mark">

8.64.1 Commands

8.65 atempo

<ul class="toc-numbered-mark">

8.65.1 Examples

8.65.2 Commands

8.66 atilt

<ul class="toc-numbered-mark">

8.66.1 Commands

8.67 atrim

8.68 axcorrelate

<ul class="toc-numbered-mark">

8.68.1 Examples

8.69 bandpass

<ul class="toc-numbered-mark">

8.69.1 Commands

8.70 bandreject

<ul class="toc-numbered-mark">

8.70.1 Commands

8.71 bass, lowshelf

<ul class="toc-numbered-mark">

8.71.1 Commands

8.72 biquad

<ul class="toc-numbered-mark">

8.72.1 Commands

8.73 bs2b

8.74 channelmap

<ul class="toc-numbered-mark">

8.74.1 Examples

8.75 channelsplit

<ul class="toc-numbered-mark">

8.75.1 Examples

8.76 chorus

<ul class="toc-numbered-mark">

8.76.1 Examples

8.77 compand

<ul class="toc-numbered-mark">

8.77.1 Examples

8.78 compensationdelay

<ul class="toc-numbered-mark">

8.78.1 Commands

8.79 crossfeed

<ul class="toc-numbered-mark">

8.79.1 Commands

8.80 crystalizer

<ul class="toc-numbered-mark">

8.80.1 Commands

8.81 dcshift

8.82 deesser

8.83 dialoguenhance

<ul class="toc-numbered-mark">

8.83.1 Commands

8.84 drmeter

8.85 dynaudnorm

<ul class="toc-numbered-mark">

8.85.1 Commands

8.86 earwax

8.87 equalizer

<ul class="toc-numbered-mark">

8.87.1 Examples

8.87.2 Commands

8.88 extrastereo

<ul class="toc-numbered-mark">

8.88.1 Commands

8.89 firequalizer

<ul class="toc-numbered-mark">

8.89.1 Examples

8.90 flanger

8.91 haas

8.92 hdcd

8.93 headphone

<ul class="toc-numbered-mark">

8.93.1 Examples

8.94 highpass

<ul class="toc-numbered-mark">

8.94.1 Commands

8.95 join

8.96 ladspa

<ul class="toc-numbered-mark">

8.96.1 Examples

8.96.2 Commands

8.97 loudnorm

8.98 lowpass

<ul class="toc-numbered-mark">

8.98.1 Examples

8.98.2 Commands

8.99 lv2

<ul class="toc-numbered-mark">

8.99.1 Examples

8.99.2 Commands

8.100 mcompand

8.101 pan

<ul class="toc-numbered-mark">

8.101.1 Mixing examples

8.101.2 Remapping examples

8.102 replaygain

8.103 resample

8.104 rubberband

<ul class="toc-numbered-mark">

8.104.1 Commands

8.105 sidechaincompress

<ul class="toc-numbered-mark">

8.105.1 Commands

8.105.2 Examples

8.106 sidechaingate

<ul class="toc-numbered-mark">

8.106.1 Commands

8.107 silencedetect

<ul class="toc-numbered-mark">

8.107.1 Examples

8.108 silenceremove

<ul class="toc-numbered-mark">

8.108.1 Examples

8.108.2 Commands

8.109 sofalizer

<ul class="toc-numbered-mark">

8.109.1 Examples

8.110 speechnorm

<ul class="toc-numbered-mark">

8.110.1 Commands

8.110.2 Examples

8.111 stereotools

<ul class="toc-numbered-mark">

8.111.1 Commands

8.111.2 Examples

8.112 stereowiden

<ul class="toc-numbered-mark">

8.112.1 Commands

8.113 superequalizer

8.114 surround

8.115 tiltshelf

<ul class="toc-numbered-mark">

8.115.1 Commands

8.116 treble, highshelf

<ul class="toc-numbered-mark">

8.116.1 Commands

8.117 tremolo

8.118 vibrato

8.119 virtualbass

8.120 volume

<ul class="toc-numbered-mark">

8.120.1 Commands

8.120.2 Examples

8.121 volumedetect

<ul class="toc-numbered-mark">

8.121.1 Examples

8.122 whisper

<ul class="toc-numbered-mark">

8.122.1 Examples

9 Audio Sources

<ul class="toc-numbered-mark">

9.1 abuffer

<ul class="toc-numbered-mark">

9.1.1 Examples

9.2 aevalsrc

<ul class="toc-numbered-mark">

9.2.1 Examples

9.3 afdelaysrc
9.4 afireqsrc
9.5 afirsrc
9.6 anullsrc
<ul class="toc-numbered-mark">
9.6.1 Examples

9.7 flite
<ul class="toc-numbered-mark">
9.7.1 Examples

9.8 anoisesrc
<ul class="toc-numbered-mark">
9.8.1 Examples

9.9 hilbert
9.10 sinc
9.11 sine
<ul class="toc-numbered-mark">
9.11.1 Examples

10 Audio Sinks
<ul class="toc-numbered-mark">
10.1 abuffersink
10.2 anullsink

11 Video Filters
<ul class="toc-numbered-mark">

11.1 addroi

<ul class="toc-numbered-mark">

11.1.1 Examples

11.2 alphaextract

11.3 alphamerge

11.4 amplify

<ul class="toc-numbered-mark">

11.4.1 Commands

11.5 ass

<ul class="toc-numbered-mark">

11.5.1 Commands

11.6 atadenoise

<ul class="toc-numbered-mark">

11.6.1 Commands

11.7 avgblur

<ul class="toc-numbered-mark">

11.7.1 Commands

11.8 backgroundkey

<ul class="toc-numbered-mark">

11.8.1 Commands

11.9 bbox

<ul class="toc-numbered-mark">

11.9.1 Commands

11.10 bilateral

<ul class="toc-numbered-mark">

11.10.1 Commands

11.11 bitplanenoise

11.12 blackdetect, blackdetect_vulkan

11.13 blackframe

11.14 blend

<ul class="toc-numbered-mark">

11.14.1 Examples

11.14.2 Commands

11.15 blockdetect

<ul class="toc-numbered-mark">

11.15.1 Examples

11.16 blurdetect

<ul class="toc-numbered-mark">

11.16.1 Examples

11.17 bm3d

<ul class="toc-numbered-mark">

11.17.1 Examples

11.18 boxblur

<ul class="toc-numbered-mark">

11.18.1 Examples

11.19 bwdif

11.20 ccrepack

11.21 cas

<ul class="toc-numbered-mark">

11.21.1 Commands

11.22 chromahold

<ul class="toc-numbered-mark">

11.22.1 Commands

11.23 chromakey

<ul class="toc-numbered-mark">

11.23.1 Commands

11.23.2 Examples

11.24 chromanr

<ul class="toc-numbered-mark">

11.24.1 Commands

11.25 chromashift

<ul class="toc-numbered-mark">

11.25.1 Commands

11.26 ciescope

11.27 codecview

<ul class="toc-numbered-mark">

11.27.1 Examples

11.28 colorbalance

<ul class="toc-numbered-mark">

11.28.1 Examples

11.28.2 Commands

11.29 colorcontrast

<ul class="toc-numbered-mark">

11.29.1 Commands

11.30 colorcorrect

<ul class="toc-numbered-mark">

11.30.1 Commands

11.31 colorchannelmixer

<ul class="toc-numbered-mark">

11.31.1 Examples

11.31.2 Commands

11.32 colordetect

11.33 colorize

<ul class="toc-numbered-mark">

11.33.1 Commands

11.34 colorkey

<ul class="toc-numbered-mark">

11.34.1 Examples

11.34.2 Commands

11.35 colorhold

<ul class="toc-numbered-mark">

11.35.1 Commands

11.36 colorlevels

<ul class="toc-numbered-mark">

11.36.1 Examples

11.36.2 Commands

11.37 colormap

11.38 colormatrix

11.39 colorspace

11.40 colortemperature

<ul class="toc-numbered-mark">

11.40.1 Commands

11.41 convolution

<ul class="toc-numbered-mark">

11.41.1 Commands

11.41.2 Examples

11.42 convolve

11.43 copy

11.44 coreimage

<ul class="toc-numbered-mark">

11.44.1 Examples

11.45 corr

11.46 cover_rect

<ul class="toc-numbered-mark">

11.46.1 Examples

11.47 crop

<ul class="toc-numbered-mark">

11.47.1 Examples

11.47.2 Commands

11.48 cropdetect

<ul class="toc-numbered-mark">

11.48.1 Examples

11.48.2 Commands

11.49 cue

11.50 curves

<ul class="toc-numbered-mark">

11.50.1 Commands

11.50.2 Examples

11.51 datascope

<ul class="toc-numbered-mark">

11.51.1 Commands

11.52 dblur

<ul class="toc-numbered-mark">

11.52.1 Commands

11.53 dctdnoiz

<ul class="toc-numbered-mark">

11.53.1 Examples

11.54 deband

<ul class="toc-numbered-mark">

11.54.1 Commands

11.55 deblock

<ul class="toc-numbered-mark">

11.55.1 Examples

11.55.2 Commands

11.56 decimate

11.57 deconvolve

11.58 dedot

11.59 deflate

<ul class="toc-numbered-mark">

11.59.1 Commands

11.60 deflicker

11.61 deinterlace_d3d12

<ul class="toc-numbered-mark">

11.61.1 Examples

11.62 dejudder

11.63 delogo

<ul class="toc-numbered-mark">

11.63.1 Examples

11.64 derain

11.65 deshake

11.66 despill

<ul class="toc-numbered-mark">

11.66.1 Commands

11.67 detelecine

11.68 dilation

<ul class="toc-numbered-mark">

11.68.1 Commands

11.69 displace

<ul class="toc-numbered-mark">

11.69.1 Examples

11.70 dnn_classify

11.71 dnn_detect

<ul class="toc-numbered-mark">

11.71.1 Examples

11.72 dnn_processing

<ul class="toc-numbered-mark">

11.72.1 Examples

11.73 drawbox

<ul class="toc-numbered-mark">

11.73.1 Examples

11.73.2 Commands

11.74 drawgraph

11.75 drawgrid

<ul class="toc-numbered-mark">

11.75.1 Examples

11.75.2 Commands

11.76 drawtext

<ul class="toc-numbered-mark">

11.76.1 Syntax

11.76.2 Text expansion

11.76.3 Commands

11.76.4 Examples

11.77 drawvg

<ul class="toc-numbered-mark">

11.77.1 Parameters

11.77.2 Pixel Formats

11.77.3 Examples

11.78 edgedetect

<ul class="toc-numbered-mark">

11.78.1 Examples

11.79 elbg

11.80 entropy

11.81 epx

11.82 eq

<ul class="toc-numbered-mark">

11.82.1 Commands

11.83 erosion

<ul class="toc-numbered-mark">

11.83.1 Commands

11.84 estdif

<ul class="toc-numbered-mark">

11.84.1 Commands

11.85 exposure

<ul class="toc-numbered-mark">

11.85.1 Commands

11.86 extractplanes

<ul class="toc-numbered-mark">

11.86.1 Examples

11.87 fade

<ul class="toc-numbered-mark">

11.87.1 Examples

11.88 feedback

<ul class="toc-numbered-mark">

11.88.1 Examples

11.89 fftdnoiz

11.90 fftfilt

<ul class="toc-numbered-mark">

11.90.1 Examples

11.91 field

11.92 fieldhint

11.93 fieldmatch

<ul class="toc-numbered-mark">

11.93.1 p/c/n/u/b meaning

<ul class="toc-numbered-mark">

11.93.1.1 p/c/n

11.93.1.2 u/b

11.93.2 Examples

11.94 fieldorder

11.95 fillborders

<ul class="toc-numbered-mark">

11.95.1 Commands

11.96 find_rect

<ul class="toc-numbered-mark">

11.96.1 Examples

11.97 floodfill

11.98 format

<ul class="toc-numbered-mark">

11.98.1 Examples

11.99 fps

<ul class="toc-numbered-mark">

11.99.1 Examples

11.100 framepack

11.101 framerate

11.102 frc_amf

11.103 vqe_amf

11.104 framestep

11.105 freezedetect

11.106 freezeframes

11.107 frei0r

<ul class="toc-numbered-mark">

11.107.1 Examples

11.107.2 Commands

11.108 fspp

11.109 fsync

11.110 gainmap

<ul class="toc-numbered-mark">

11.110.1 Examples

11.111 gblur

<ul class="toc-numbered-mark">

11.111.1 Commands

11.112 geq

<ul class="toc-numbered-mark">

11.112.1 Examples

11.113 gradfun

<ul class="toc-numbered-mark">

11.113.1 Examples

11.114 graphmonitor

11.115 grayworld

11.116 greyedge

<ul class="toc-numbered-mark">

11.116.1 Examples

11.117 guided

<ul class="toc-numbered-mark">

11.117.1 Commands

11.117.2 Examples

11.118 haldclut

<ul class="toc-numbered-mark">

11.118.1 Commands

11.118.2 Workflow examples

<ul class="toc-numbered-mark">

11.118.2.1 Hald CLUT video stream

11.118.2.2 Hald CLUT with preview

11.119 hflip

11.120 histeq

11.121 histogram

<ul class="toc-numbered-mark">

11.121.1 Examples

11.122 hqdn3d

<ul class="toc-numbered-mark">

11.122.1 Commands

11.123 hwdownload

11.124 hwmap

11.125 hwupload

11.126 hwupload_cuda

11.127 hqx

11.128 hstack

11.129 hsvhold

11.130 hsvkey

11.131 hue

<ul class="toc-numbered-mark">

11.131.1 Examples

11.131.2 Commands

11.132 huesaturation

11.133 hysteresis

11.134 iccdetect

11.135 iccgen

11.136 identity

11.137 idet

<ul class="toc-numbered-mark">

11.137.1 Examples

11.138 il

<ul class="toc-numbered-mark">

11.138.1 Commands

11.139 inflate

<ul class="toc-numbered-mark">

11.139.1 Commands

11.140 interlace, interlace_vulkan

11.141 kerndeint

<ul class="toc-numbered-mark">

11.141.1 Examples

11.142 kirsch

<ul class="toc-numbered-mark">

11.142.1 Commands

11.143 lagfun

<ul class="toc-numbered-mark">

11.143.1 Commands

11.144 latticepal

11.145 lenscorrection

<ul class="toc-numbered-mark">

11.145.1 Options

11.145.2 Commands

11.146 lensfun

<ul class="toc-numbered-mark">

11.146.1 Examples

11.147 lcevc

11.148 libplacebo

<ul class="toc-numbered-mark">

11.148.1 Options

<ul class="toc-numbered-mark">

11.148.1.1 Output mode

11.148.1.2 Scaling

11.148.1.3 Deinterlacing

11.148.1.4 Debanding

11.148.1.5 Color adjustment

11.148.1.6 Peak detection

11.148.1.7 Tone mapping

11.148.1.8 Dithering

11.148.1.9 Custom shaders

11.148.1.10 Debugging / performance

11.148.2 Commands

11.148.3 Examples

11.149 libvmaf

<ul class="toc-numbered-mark">

11.149.1 Examples

11.150 libvmaf_cuda

<ul class="toc-numbered-mark">

11.150.1 Examples

11.151 limitdiff

<ul class="toc-numbered-mark">

11.151.1 Commands

11.152 limiter

<ul class="toc-numbered-mark">

11.152.1 Commands

11.153 loop

<ul class="toc-numbered-mark">

11.153.1 Examples

11.154 lut1d

<ul class="toc-numbered-mark">

11.154.1 Commands

11.155 lut3d

<ul class="toc-numbered-mark">

11.155.1 Commands

11.156 lumakey

<ul class="toc-numbered-mark">

11.156.1 Commands

11.157 lut, lutrgb, lutyuv

<ul class="toc-numbered-mark">

11.157.1 Commands

11.157.2 Examples

11.158 lut2, tlut2

<ul class="toc-numbered-mark">

11.158.1 Commands

11.158.2 Examples

11.159 maskedclamp

<ul class="toc-numbered-mark">

11.159.1 Commands

11.160 maskedmax

<ul class="toc-numbered-mark">

11.160.1 Commands

11.161 maskedmerge

<ul class="toc-numbered-mark">

11.161.1 Commands

11.162 maskedmin

<ul class="toc-numbered-mark">

11.162.1 Commands

11.163 maskedthreshold

<ul class="toc-numbered-mark">

11.163.1 Commands

11.164 maskfun

<ul class="toc-numbered-mark">

11.164.1 Commands

11.165 mcdeint

11.166 median

<ul class="toc-numbered-mark">

11.166.1 Commands

11.167 mergeplanes

<ul class="toc-numbered-mark">

11.167.1 Examples

11.168 mestimate

11.169 mestimate_d3d12

<ul class="toc-numbered-mark">

11.169.1 Examples

11.170 midequalizer

11.171 minterpolate

11.172 mix

<ul class="toc-numbered-mark">

11.172.1 Commands

11.173 monochrome

<ul class="toc-numbered-mark">

11.173.1 Commands

11.174 morpho

<ul class="toc-numbered-mark">

11.174.1 Commands

11.175 mpdecimate

11.176 msad

11.177 multiply

<ul class="toc-numbered-mark">

11.177.1 Commands

11.178 negate

<ul class="toc-numbered-mark">

11.178.1 Commands

11.179 nlmeans

11.180 nnedi

<ul class="toc-numbered-mark">

11.180.1 Commands

11.181 noformat

<ul class="toc-numbered-mark">

11.181.1 Examples

11.182 noise

<ul class="toc-numbered-mark">

11.182.1 Examples

11.183 normalize

<ul class="toc-numbered-mark">

11.183.1 Commands

11.183.2 Examples

11.184 null

11.185 ocio

<ul class="toc-numbered-mark">

11.185.1 Examples

11.186 ocr

11.187 ocv

<ul class="toc-numbered-mark">

11.187.1 dilate

11.187.2 erode

11.187.3 smooth

11.188 oscilloscope

<ul class="toc-numbered-mark">

11.188.1 Commands

11.188.2 Examples

11.189 overlay

<ul class="toc-numbered-mark">

11.189.1 Commands

11.189.2 Examples

11.190 owdenoise

11.191 pad

<ul class="toc-numbered-mark">

11.191.1 Examples

11.192 palettegen

<ul class="toc-numbered-mark">

11.192.1 Examples

11.193 paletteuse

<ul class="toc-numbered-mark">

11.193.1 Examples

11.194 perspective

11.195 phase

<ul class="toc-numbered-mark">

11.195.1 Commands

11.196 photosensitivity

11.197 pixdesctest

11.198 pixelize

<ul class="toc-numbered-mark">

11.198.1 Commands

11.199 pixscope

<ul class="toc-numbered-mark">

11.199.1 Commands

11.200 pp7

11.201 premultiply

11.202 premultiply_dynamic

11.203 prewitt

<ul class="toc-numbered-mark">

11.203.1 Commands

11.204 pseudocolor

<ul class="toc-numbered-mark">

11.204.1 Commands

11.204.2 Examples

11.205 psnr

<ul class="toc-numbered-mark">

11.205.1 Examples

11.206 pullup

11.207 qp

<ul class="toc-numbered-mark">

11.207.1 Examples

11.208 qrencode

<ul class="toc-numbered-mark">

11.208.1 qrencode Expressions

11.208.2 qrencode Text expansion

11.208.3 Examples

11.209 quirc

11.210 random

11.211 readeia608

<ul class="toc-numbered-mark">

11.211.1 Commands

11.211.2 Examples

11.212 readvitc

<ul class="toc-numbered-mark">

11.212.1 Examples

11.213 remap

11.214 removegrain

11.215 removelogo

11.216 repeatfields

11.217 reverse

<ul class="toc-numbered-mark">

11.217.1 Examples

11.218 rgbashift

<ul class="toc-numbered-mark">

11.218.1 Commands

11.219 roberts

<ul class="toc-numbered-mark">

11.219.1 Commands

11.220 rotate

<ul class="toc-numbered-mark">

11.220.1 Examples

11.220.2 Commands

11.221 sab

11.222 scale

<ul class="toc-numbered-mark">

11.222.1 Options

11.222.2 Examples

11.222.3 Commands

11.223 scharr

<ul class="toc-numbered-mark">

11.223.1 Commands

11.224 scroll

<ul class="toc-numbered-mark">

11.224.1 Commands

11.225 scdet

11.226 selectivecolor

<ul class="toc-numbered-mark">

11.226.1 Examples

11.227 separatefields

11.228 setdar, setsar

<ul class="toc-numbered-mark">

11.228.1 Examples

11.229 setfield

11.230 setparams

11.231 shear

<ul class="toc-numbered-mark">

11.231.1 Commands

11.232 showinfo

11.233 showpalette

11.234 shuffleframes

<ul class="toc-numbered-mark">

11.234.1 Examples

11.235 shufflepixels

11.236 shuffleplanes

<ul class="toc-numbered-mark">

11.236.1 Examples

11.237 signalstats

<ul class="toc-numbered-mark">

11.237.1 Examples

11.238 signature

<ul class="toc-numbered-mark">

11.238.1 Examples

11.239 siti

<ul class="toc-numbered-mark">

11.239.1 Examples

11.240 smartblur

11.241 sobel

<ul class="toc-numbered-mark">

11.241.1 Commands

11.242 spp

<ul class="toc-numbered-mark">

11.242.1 Commands

11.243 sr

11.244 sr_amf

<ul class="toc-numbered-mark">

11.244.1 Examples

11.245 ssim

<ul class="toc-numbered-mark">

11.245.1 Examples

11.246 stereo3d

<ul class="toc-numbered-mark">

11.246.1 Examples

11.247 streamselect, astreamselect

<ul class="toc-numbered-mark">

11.247.1 Commands

11.247.2 Examples

11.248 subtitles

11.249 super2xsai

11.250 swaprect

<ul class="toc-numbered-mark">

11.250.1 Commands

11.251 swapuv

11.252 tblend

11.253 telecine

11.254 thistogram

11.255 threshold

<ul class="toc-numbered-mark">

11.255.1 Commands

11.255.2 Examples

11.256 thumbnail

<ul class="toc-numbered-mark">

11.256.1 Examples

11.257 tile

<ul class="toc-numbered-mark">

11.257.1 Examples

11.258 tiltandshift

11.259 tinterlace

11.260 tmedian

<ul class="toc-numbered-mark">

11.260.1 Commands

11.261 tmidequalizer

11.262 tmix

<ul class="toc-numbered-mark">

11.262.1 Examples

11.262.2 Commands

11.263 tonemap

<ul class="toc-numbered-mark">

11.263.1 Options

11.264 tpad

11.265 transpose

11.266 trim

11.267 unpremultiply

11.268 unsharp

<ul class="toc-numbered-mark">

11.268.1 Examples

11.269 untile

<ul class="toc-numbered-mark">

11.269.1 Examples

11.270 uspp

11.271 v360

<ul class="toc-numbered-mark">

11.271.1 Examples

11.271.2 Commands

11.272 vaguedenoiser

11.273 varblur

<ul class="toc-numbered-mark">

11.273.1 Commands

11.274 vectorscope

11.275 vidstabdetect

<ul class="toc-numbered-mark">

11.275.1 Examples

11.276 vidstabtransform

<ul class="toc-numbered-mark">

11.276.1 Options

11.276.2 Examples

11.277 vflip

11.278 vfrdet

11.279 vibrance

<ul class="toc-numbered-mark">

11.279.1 Commands

11.280 vif

11.281 vignette

<ul class="toc-numbered-mark">

11.281.1 Expressions

11.281.2 Examples

11.282 vmafmotion

11.283 vpp_amf

<ul class="toc-numbered-mark">

11.283.1 Examples

11.284 vstack

11.285 w3fdif

<ul class="toc-numbered-mark">

11.285.1 Commands

11.286 waveform

11.287 weave, doubleweave

<ul class="toc-numbered-mark">

11.287.1 Examples

11.288 xbr

11.289 xcorrelate

11.290 xfade

<ul class="toc-numbered-mark">

11.290.1 Examples

11.291 xmedian

<ul class="toc-numbered-mark">

11.291.1 Commands

11.292 xpsnr

<ul class="toc-numbered-mark">

11.292.1 Examples

11.293 xstack

<ul class="toc-numbered-mark">

11.293.1 Examples

11.294 yadif

11.295 yaepblur

<ul class="toc-numbered-mark">

11.295.1 Commands

11.296 zoompan

<ul class="toc-numbered-mark">

11.296.1 Examples

11.297 zscale

<ul class="toc-numbered-mark">

11.297.1 Options

11.297.2 Commands

12 CUDA Video Filters

<ul class="toc-numbered-mark">

12.1 bilateral_cuda

<ul class="toc-numbered-mark">

12.1.1 Examples

12.2 bwdif_cuda

12.3 chromakey_cuda

<ul class="toc-numbered-mark">

12.3.1 Examples

12.4 colorspace_cuda

12.5 overlay_cuda

12.6 pad_cuda

<ul class="toc-numbered-mark">

12.6.1 Examples

12.7 scale_cuda

<ul class="toc-numbered-mark">

12.7.1 Examples

12.8 thumbnail_cuda

<ul class="toc-numbered-mark">

12.8.1 Example

12.9 transpose_cuda

12.10 yadif_cuda

13 OpenCL Video Filters

<ul class="toc-numbered-mark">

13.1 avgblur_opencl

<ul class="toc-numbered-mark">

13.1.1 Example

13.2 boxblur_opencl

<ul class="toc-numbered-mark">

13.2.1 Examples

13.3 colorkey_opencl

<ul class="toc-numbered-mark">

13.3.1 Examples

13.4 convolution_opencl

<ul class="toc-numbered-mark">

13.4.1 Examples

13.5 erosion_opencl

<ul class="toc-numbered-mark">

13.5.1 Example

13.6 deshake_opencl

<ul class="toc-numbered-mark">

13.6.1 Examples

13.7 dilation_opencl

<ul class="toc-numbered-mark">

13.7.1 Example

13.8 nlmeans_opencl

13.9 overlay_opencl

<ul class="toc-numbered-mark">

13.9.1 Examples

13.10 pad_opencl

13.11 prewitt_opencl

<ul class="toc-numbered-mark">

13.11.1 Example

13.12 program_opencl

13.13 remap_opencl

13.14 roberts_opencl

<ul class="toc-numbered-mark">

13.14.1 Example

13.15 sobel_opencl

<ul class="toc-numbered-mark">

13.15.1 Example

13.16 tonemap_opencl

<ul class="toc-numbered-mark">

13.16.1 Example

13.17 unsharp_opencl

<ul class="toc-numbered-mark">

13.17.1 Examples

13.18 xfade_opencl

14 VAAPI Video Filters

<ul class="toc-numbered-mark">

14.1 overlay_vaapi

<ul class="toc-numbered-mark">

14.1.1 Examples

14.2 tonemap_vaapi

<ul class="toc-numbered-mark">

14.2.1 Example

14.3 hstack_vaapi

14.4 vstack_vaapi

14.5 xstack_vaapi

14.6 pad_vaapi

14.7 drawbox_vaapi

<ul class="toc-numbered-mark">

14.7.1 Examples

15 VideoToolbox Video Filters

<ul class="toc-numbered-mark">

15.1 scale_vt

<ul class="toc-numbered-mark">

15.1.1 Examples

15.2 transpose_vt

16 Vulkan Video Filters

<ul class="toc-numbered-mark">

16.1 avgblur_vulkan

16.2 blend_vulkan

16.3 bwdif_vulkan

16.4 chromaber_vulkan

16.5 color_vulkan

16.6 vflip_vulkan

16.7 hflip_vulkan

16.8 flip_vulkan

16.9 fruc_vulkan

16.10 gblur_vulkan

16.11 nlmeans_vulkan

16.12 overlay_vulkan

16.13 transpose_vulkan

16.14 v360_vulkan

<ul class="toc-numbered-mark">

16.14.1 Examples

17 QSV Video Filters

<ul class="toc-numbered-mark">

17.1 hstack_qsv

17.2 vstack_qsv

17.3 xstack_qsv

18 Video Sources

<ul class="toc-numbered-mark">

18.1 buffer

18.2 cellauto

<ul class="toc-numbered-mark">

18.2.1 Examples

18.3 coreimagesrc

<ul class="toc-numbered-mark">

18.3.1 Examples

18.4 ddagrab

<ul class="toc-numbered-mark">

18.4.1 Examples

18.5 gfxcapture

<ul class="toc-numbered-mark">

18.5.1 Examples

18.6 gradients

<ul class="toc-numbered-mark">

18.6.1 Commands

18.7 mandelbrot

18.8 mptestsrc

18.9 frei0r_src

18.10 life

<ul class="toc-numbered-mark">

18.10.1 Examples

18.11 perlin

<ul class="toc-numbered-mark">

18.11.1 Options

18.11.2 Examples

18.12 qrencodesrc

<ul class="toc-numbered-mark">

18.12.1 Examples

18.13 allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc

<ul class="toc-numbered-mark">

18.13.1 Examples

18.13.2 Commands

18.14 openclsrc

18.15 sierpinski

18.16 zoneplate

<ul class="toc-numbered-mark">

18.16.1 Commands

18.16.2 Examples

19 Video Sinks

<ul class="toc-numbered-mark">

19.1 buffersink

19.2 nullsink

20 Multimedia Filters

<ul class="toc-numbered-mark">

20.1 a3dscope

<ul class="toc-numbered-mark">

20.1.1 Commands

20.2 abitscope

20.3 adrawgraph

20.4 agraphmonitor

20.5 ahistogram

20.6 aphasemeter

<ul class="toc-numbered-mark">

20.6.1 phasing detection

20.6.2 Examples

20.7 avectorscope

<ul class="toc-numbered-mark">

20.7.1 Examples

20.7.2 Commands

20.8 bench, abench

<ul class="toc-numbered-mark">

20.8.1 Examples

20.9 concat

<ul class="toc-numbered-mark">

20.9.1 Examples

20.9.2 Commands

20.10 ebur128

<ul class="toc-numbered-mark">

20.10.1 Examples

20.11 interleave, ainterleave

<ul class="toc-numbered-mark">

20.11.1 Examples

20.12 latency, alatency

20.13 metadata, ametadata

<ul class="toc-numbered-mark">

20.13.1 Examples

20.14 perms, aperms

20.15 realtime, arealtime

<ul class="toc-numbered-mark">

20.15.1 Commands

20.16 segment, asegment

<ul class="toc-numbered-mark">

20.16.1 Examples

20.17 select, aselect

<ul class="toc-numbered-mark">

20.17.1 Examples

20.18 sendcmd, asendcmd

<ul class="toc-numbered-mark">

20.18.1 Commands syntax

20.18.2 Examples

20.19 setpts, asetpts

<ul class="toc-numbered-mark">

20.19.1 Examples

20.19.2 Commands

20.20 setrange

20.21 settb, asettb

<ul class="toc-numbered-mark">

20.21.1 Examples

20.22 showcqt

<ul class="toc-numbered-mark">

20.22.1 Examples

20.23 showcwt

20.24 showfreqs

20.25 showspatial

20.26 showspectrum

<ul class="toc-numbered-mark">

20.26.1 Examples

20.27 showspectrumpic

<ul class="toc-numbered-mark">

20.27.1 Examples

20.28 showvolume

20.29 showwaves

<ul class="toc-numbered-mark">

20.29.1 Examples

20.30 showwavespic

<ul class="toc-numbered-mark">

20.30.1 Examples

20.31 sidedata, asidedata

20.32 spectrumsynth

<ul class="toc-numbered-mark">

20.32.1 Examples

20.33 split, asplit

<ul class="toc-numbered-mark">

20.33.1 Examples

20.34 zmq, azmq

<ul class="toc-numbered-mark">

20.34.1 Examples

21 Multimedia Sources

<ul class="toc-numbered-mark">

21.1 amovie

21.2 avsynctest

<ul class="toc-numbered-mark">

21.2.1 Commands

21.3 movie

<ul class="toc-numbered-mark">

21.3.1 Examples

21.3.2 Commands

22 See Also

23 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes filters, sources, and sinks provided by the libavfilter library.

 <h2 class="chapter">2 Filtering Introduction# TOC</h2>

Filtering in FFmpeg is enabled through the libavfilter library. In libavfilter, a filter can have multiple inputs and multiple outputs. To illustrate the sorts of things that are possible, we consider the following filtergraph. <pre class="verbatim"> [main] input --> split ----------> overlay --> output | ^ |[tmp] [flip]| +-----> crop --> vflip -------+ </pre> This filtergraph splits the input stream in two streams, then sends one stream through the crop filter and the vflip filter, before merging it back with the other stream by overlaying it on top. You can use the following command to achieve this: <pre

class=”example-preformatted”>ffmpeg -i INPUT -vf "split [main][tmp]; [tmp] crop=iw:ih/2:0:0, vflip [flip]; [main][flip] overlay=0:H/2" OUTPUT </pre>

The result will be that the top half of the video is mirrored onto the bottom half of the output video. Filters in the same linear chain are separated by commas, and distinct linear chains of filters are separated by semicolons. In our example, <var class=”var”>crop,vflip</var> are in one linear chain, <var class=”var”>split</var> and <var class=”var”>overlay</var> are separately in another. The points where the linear chains join are labelled by names enclosed in square brackets. In the example, the split filter generates two outputs that are associated to the labels <var class=”var”>[main]</var> and <var class=”var”>[tmp]</var>. The stream sent to the second output of <var class=”var”>split</var>, labelled as <var class=”var”>[tmp]</var>, is processed through the <var class=”var”>crop</var> filter, which crops away the lower half part of the video, and then vertically flipped. The <var class=”var”>overlay</var> filter takes in input the first unchanged output of the split filter (which was labelled as <var class=”var”>[main]</var>), and overlay on its lower half the output generated by the <var class=”var”>crop,vflip</var> filterchain. Some filters take in input a list of parameters: they are specified after the filter name and an equal sign, and are separated from each other by a colon. There exist so-called <var class=”var”>source filters</var> that do not have an audio/video input, and <var class=”var”>sink filters</var> that will not have audio/video output.

 <h2 class=”chapter”>3 graph2dot# TOC</h2>

The <samp class=”file”>graph2dot</samp> program included in the FFmpeg <samp class=”file”>tools</samp> directory can be used to parse a filtergraph description and issue a corresponding textual representation in the dot language. Invoke the command: <pre class=”example-preformatted”>graph2dot -h </pre>

to see how to use <samp class=”file”>graph2dot</samp>. You can then pass the dot description to the <samp class=”file”>dot</samp> program (from the graphviz suite of programs) and obtain a graphical representation of the filtergraph. For example the sequence of commands: <pre class=”example-preformatted”>echo <var class=”var”>GRAPH_DESCRIPTION</var> | \ tools/graph2dot -o graph.tmp && \ dot -Tpng graph.tmp -o graph.png && \ display graph.png </pre>

can be used to create and display an image representing the graph described by the <var class=”var”>GRAPH_DESCRIPTION</var> string.

Note that this string must be a complete self-contained graph, with its inputs and outputs explicitly defined. For example if your command line is of the form: <pre class="example-preformatted">ffmpeg -i infile -vf scale=640:360 outfile </pre> your <var class="var">GRAPH_DESCRIPTION</var> string will need to be of the form: <pre class="example-preformatted">nullsrc,scale=640:360,nullsink </pre> you may also need to set the <var class="var">nullsrc</var> parameters and add a <var class="var">format</var> filter in order to simulate a specific input file.

 <h2 class="chapter">4 Filtergraph description# TOC</h2>

A filtergraph is a directed graph of connected filters. It can contain cycles, and there can be multiple links between a pair of filters. Each link has one input pad on one side connecting it to one filter from which it takes its input, and one output pad on the other side connecting it to one filter accepting its output. Each filter in a filtergraph is an instance of a filter class registered in the application, which defines the features and the number of input and output pads of the filter. A filter with no input pads is called a "source", and a filter with no output pads is called a "sink". <h3 class="section">4.1 Filtergraph syntax# TOC</h3>

A filtergraph has a textual representation, which is recognized by the <samp class="option">-filter</samp>/<samp class="option">-vf</samp>/<samp class="option">-af</samp> and <samp class="option">-filter_complex</samp> options in <code class="command">ffmpeg</code> and <samp class="option">-vf</samp>/<samp class="option">-af</samp> in <code class="command">ffplay</code>, and by the <code class="code">avfilter_graph_parse_ptr()</co function defined in <samp class="file">libavfilter/avfilter.h</samp>. A filterchain consists of a sequence of connected filters, each one connected to the previous one in the sequence. A filterchain is represented by a list of ","-separated filter descriptions. A filtergraph consists of a sequence of filterchains. A sequence of filterchains is represented by a list of ";"-separated filterchain descriptions. A filter is represented by a string of the form: [<var class="var">in_link_1</var>]...[<var class="var">in_link_N</var>]<var class="var">filter_name</var>@<var class="var">id</var>=<var class="var">arguments</var>[<var class="var">out_link_1</var>]...[<var

class=”var”>out_link_M</var>] <var class=”var”>filter_name</var> is the name of the filter class of which the described filter is an instance of, and has to be the name of one of the filter classes registered in the program optionally followed by "@<var class=”var”>id</var>". The name of the filter class is optionally followed by a string "=<var class=”var”>arguments</var>". <var class=”var”>arguments</var> is a string which contains the parameters used to initialize the filter instance. It may have one of two forms: <ul class=”itemize mark-bullet”> A ’:’-separated list of <var class=”var”>key=value</var> pairs.

A ’:’-separated list of <var class=”var”>value</var>. In this case, the keys are assumed to be the option names in the order they are declared. E.g. the <code class=”code”>fade</code> filter declares three options in this order – <samp class=”option”>type</samp>, <samp class=”option”>start_frame</samp> and <samp class=”option”>nb_frames</samp>. Then the parameter list <var class=”var”>in:0:30</var> means that the value <var class=”var”>in</var> is assigned to the option <samp class=”option”>type</samp>, <var class=”var”>0</var> to <samp class=”option”>start_frame</samp> and <var class=”var”>30</var> to <samp class=”option”>nb_frames</samp>.

A ’:’-separated list of mixed direct <var class=”var”>value</var> and long <var class=”var”>key=value</var> pairs. The direct <var class=”var”>value</var> must precede the <var class=”var”>key=value</var> pairs, and follow the same constraints order of the previous point. The following <var class=”var”>key=value</var> pairs can be set in any preferred order.

If the option value itself is a list of items (e.g. the <code class=”code”>format</code> filter takes a list of pixel formats), the items in the list are usually separated by ‘<samp class=”samp”>|</samp>’. The list of arguments can be quoted using the character ‘<samp class=”samp”>'</samp>’ as initial and ending mark, and the character ‘<samp class=”samp”>\</samp>’ for escaping the characters within the quoted text; otherwise the argument string is considered terminated when the next special character (belonging to the set ‘<samp class=”samp”>[]=;,</samp>’) is encountered. A special syntax implemented in the <code class=”command”>ffmpeg</code> CLI tool allows loading option values from files. This is done be prepending a slash ’/’ to the option name, then the supplied value is interpreted as a path from which the actual value is loaded. E.g. <pre class=”example-preformatted”>ffmpeg -i <INPUT> -vf drawtext=/text=/tmp/some_text <OUTPUT> </pre> will load the text to be drawn from <samp class=”file”>/tmp/some_text</samp>. API users wishing to implement a similar feature should use the <code class=”code”>avfilter_graph_segment_*()</code> functions together with custom IO code. The name and arguments of the fil-

ter are optionally preceded and followed by a list of link labels. A link label allows one to name a link and associate it to a filter output or input pad. The preceding labels <var class=”var”>in_link_1</var> ... <var class=”var”>in_link_N</var>, are associated to the filter input pads, the following labels <var class=”var”>out_link_1</var> ... <var class=”var”>out_link_M</var>, are associated to the output pads. When two link labels with the same name are found in the filtergraph, a link between the corresponding input and output pad is created. If an output pad is not labelled, it is linked by default to the first unlabelled input pad of the next filter in the filterchain. For example in the filterchain <pre class=”example-preformatted”>nullsrc, split[L1], [L2]overlay, nullsink </pre> the split filter instance has two output pads, and the overlay filter instance two input pads. The first output pad of split is labelled "L1", the first input pad of overlay is labelled "L2", and the second output pad of split is linked to the second input pad of overlay, which are both unlabelled. In a filter description, if the input label of the first filter is not specified, "in" is assumed; if the output label of the last filter is not specified, "out" is assumed. In a complete filterchain all the unlabelled filter input and output pads must be connected. A filtergraph is considered valid if all the filter input and output pads of all the filterchains are connected. Leading and trailing whitespaces (space, tabs, or line feeds) separating tokens in the filtergraph specification are ignored. This means that the filtergraph can be expressed using empty lines and spaces to improve readability. For example, the filtergraph: <pre class=”example-preformatted”>testsrc,split[L1],hflip[L2];[L1][L2] hstack </pre>

can be represented as: <pre class=”example-preformatted”>testsrc, split [L1], hflip [L2];

[L1][L2] hstack </pre>

Libavfilter will automatically insert scale filters where format conversion is required. It is possible to specify swscale flags for those automatically inserted scalers by prepending <code class=”code”>sws_flags=<var class=”var”>flags</var>;</code> to the filtergraph description. Here is a BNF description of the filtergraph syntax: <pre class=”example-preformatted”><var class=”var”>NAME</var> ::= sequence of alphanumeric characters and '_' <var class=”var”>FILTER_NAME</var> ::= <var class=”var”>NAME</var>["@"<var class=”var”>NAME</var>] <var class=”var”>LINKLABEL</var> ::= "[" <var class=”var”>NAME</var> "]" <var class=”var”>LINKLABELS</var> ::= <var class=”var”>LINKLABEL</var> [<var class=”var”>LINKLABELS</var>] <var class=”var”>FILTER_ARGUMENTS</var> ::= sequence of chars (possibly quoted) <var class=”var”>FILTER</var> ::= [<var class=”var”>LINKLABELS</var>] <var class=”var”>FILTER_NAME</var>

["=" <var class="var">FILTER_ARGUMENTS</var>] [<var class="var">LINKLABELS</var>] <var class="var">FILTERCHAIN</var> ::= <var class="var">FILTER</var> [,<var class="var">FILTERCHAIN</var>] <var class="var">FILTERGRAPH</var> ::= [sws_flags=<var class="var">flags</var>;] <var class="var">FILTERCHAIN</var> [;<var class="var">FILTERGRAPH</var>] </pre>

 <h3 class="section">4.2 Notes on filtergraph escaping# TOC</h3>

Filtergraph description composition entails several levels of escaping. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual for more information about the employed escaping procedure. A first level escaping affects the content of each filter option value, which may contain the special character <code class="code">:</code> used to separate values, or one of the escaping characters <code class="code">\'</code>. A second level escaping affects the whole filter description, which may contain the escaping characters <code class="code">\'</code> or the special characters <code class="code">[],;</code> used by the filtergraph description. Finally, when you specify a filtergraph on a shell commandline, you need to perform a third level escaping for the shell special characters contained within it. For example, consider the following string to be embedded in the drawtext filter description <samp class="option">text</samp> value: <pre class="example-preformatted">this is a 'string': may contain one, or more, special characters </pre>

This string contains the <code class="code">'</code> special escaping character, and the <code class="code">:</code> special character, so it needs to be escaped in this way: <pre class="example-preformatted">text=this is a \'string\'\: may contain one, or more, special characters </pre>

A second level of escaping is required when embedding the filter description in a filtergraph description, in order to escape all the filtergraph special characters. Thus the example above becomes: <pre class="example-preformatted">drawtext=text=this is a \\\'string\\\'\\: may contain one\, or more\, special characters </pre> (note that in addition to the <code class="code">\'</code> escaping special characters, also <code class="code">,</code> needs to be escaped). Finally an additional level of escaping is needed when writing the filtergraph description in a shell command, which depends on the escaping rules of the adopted shell. For example, assuming that <code class="code">\</code> is special and

needs to be escaped with another <code class=”code”>\</code>, the previous string will finally result in: <pre class=”example-preformatted”>-vf "drawtext=text=this is a \\\\\\'string\\\\\\'\\\\: may contain one\\, or more\\, special characters" </pre>

In order to avoid cumbersome escaping when using a commandline tool accepting a filter specification as input, it is advisable to avoid direct inclusion of the filter or options specification in the shell. For example, in case of the drawtext filter, you might prefer to use the <samp class=”option”>textfile</samp> option in place of <samp class=”option”>text</samp> to specify the text to render. <h2 class=”chapter”>5 Timeline editing# TOC</h2>

Some filters support a generic <samp class=”option”>enable</samp> option. For the filters supporting timeline editing, this option can be set to an expression which is evaluated before sending a frame to the filter. If the evaluation is non-zero, the filter will be enabled, otherwise the frame will be sent unchanged to the next filter in the filtergraph. The expression accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>t</samp>’</dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt>‘<samp class=”samp”>n</samp>’</dt> <dd> sequential number of the input frame, starting from 0 </dd> <dt>‘<samp class=”samp”>pos</samp>’</dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt>‘<samp class=”samp”>w</samp>’</dt> <dt>‘<samp class=”samp”>h</samp>’</dt> <dd> width and height of the input frame if video </dd> </dl>

Additionally, these filters support an <samp class=”option”>enable</samp> command that can be used to re-define the expression. Like any other filtering option, the <samp class=”option”>enable</samp> option follows the same rules. For example, to enable a blur filter (smartblur) from 10 seconds to 3 minutes, and a curves filter starting at 3 seconds: <pre class=”example-preformatted”>smartblur = enable='between(t,10,3*60)', curves = enable='gte(t,3)' : preset=cross_process </pre>

See <code class=”code”>ffmpeg -filters</code> to view which filters have timeline support.

 <h2 class=”chapter”>6 Changing options at runtime with a command<a class=”anchor

hidden-xs" href="#Changing-options-at-runtime-with-a-command" aria-hidden="true"># TOC</h2>

Some options can be changed during the operation of the filter using a command. These options are marked ’T’ on the output of <code class="command">ffmpeg</code> <samp class="option">-h filter=<name of filter></samp>. The name of the command is the name of the option and the argument is the new value. <h2 class="chapter">7 Options for filters with several inputs (framesync)# TOC</h2>

Some filters with several inputs support a common set of options. These options can only be set by name, not with the short notation. <dl class="table"> <dt><samp class="option">eof_action</samp></dt> <dd> The action to take when EOF is encountered on the secondary input; it accepts one of the following values: <dl class="table"> <dt><samp class="option">repeat</samp></dt> <dd> Repeat the last frame (the default). </dd> <dt><samp class="option">endall</samp></dt> <dd> End both streams. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass the main input through. </dd> </dl>

</dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> If set to 1, force the filter to extend the last frame of secondary streams until the end of the primary stream. A value of 0 disables this behavior. Default value is 1. </dd> <dt><samp class="option">ts_sync_mode</samp></dt> <dd> How strictly to sync streams based on secondary input timestamps; it accepts one of the following values: <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Frame from secondary input with the nearest lower or equal timestamp to the primary input frame. </dd> <dt><samp class="option">nearest</samp></dt> <dd> Frame from secondary input with the absolute nearest timestamp to the primary input frame. </dd> </dl> </dd> </dl>

 <h2 class="chapter">8 Audio Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing

filters using <code class=”code”>--disable-filters</code>. The configure output will show the audio filters included in your build. Below is a description of the currently available audio filters. <h3 class=”section”>8.1 aap# TOC</h3> Apply Affine Projection algorithm to the first audio stream using the second audio stream. This adaptive filter is used to estimate unknown audio based on multiple input audio samples. Affine projection algorithm can make trade-offs between computation complexity with convergence speed. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class=”option”>projection</samp></dt> <dd> Set the projection order. </dd> <dt><samp class=”option”>mu</samp></dt> <dd> Set the filter mu. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class=”option”>out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class=”option”>e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class=”section”>8.2 acompressor# TOC</h3>

A compressor is mainly used to reduce the dynamic range of a signal. Especially modern music is mostly compressed at a high ratio to improve the overall loudness. It’s done to get the highest attention of a listener, "fatten" the sound and bring more "power" to the track. If a signal is compressed too much it may sound dull or "dead"

afterwards or it may start to "pump" (which could be a powerful effect but can also destroy a track completely). The right compression is the key to reach a professional sound and is the high art of mixing and mastering. Because of its complex settings it may take a long time to get the right feeling for this kind of effect. Compression is done by detecting the volume above a chosen level <code class="code">threshold</code> and dividing it by the factor set with <code class="code">ratio</code>. So if you set the threshold to -12dB and your signal reaches -6dB a ratio of 2:1 will result in a signal at -9dB. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over the time. This is done by setting "Attack" and "Release". <code class="code">attack</code> determines how long the signal has to rise above the threshold before any reduction will occur and <code class="code">release</code> sets the time the signal has to fall below the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. The overall reduction of the signal can be made up afterwards with the <code class="code">makeup</code> setting. So compressing the peaks of a signal about 6dB and raising the makeup to this level results in a signal twice as loud than the source. To gain a softer entry in the compression the <code class="code">knee</code> flattens the hard edge at the threshold in the range of the chosen decibels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal of stream rises above this level it will affect the gain reduction. By default it is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. 1:2 means that if the level rose 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose

if the <code class="code">average</code> level between all channels of input stream or the louder(<code class="code">maximum</code>) channel of input stream affects the reduction. Default is <code class="code">average</code>. </dd> <dt><samp class="option">detection</samp></dt> <dd> Should the exact signal be taken in case of <code class="code">peak</code> or an RMS one in case of <code class="code">rms</code>. Default is <code class="code">rms</code> which is mostly smoother. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class="subsection">8.2.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.3 acontrast# TOC</h3> Simple audio dynamic range compression/expansion filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set contrast. Default is 33. Allowed range is between 0 and 100. </dd> </dl>

 <h3 class="section">8.4 acopy# TOC</h3>

Copy the input audio source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">8.5 acrossfade# TOC</h3>

Apply cross fade from one input audio stream to another input audio stream. The cross fade is applied for specified duration near the end of first stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs, n</samp></dt> <dd> Specify the number of inputs to crossfade. When crossfading multiple inputs, each input will be concatenated and crossfaded in sequence, similar to the concat filter. Default is 2. </dd> <dt><samp class="option">nb_samples, ns</samp></dt> <dd> Specify the number of samples for which the cross fade effect has to last. At the end of the cross fade effect the first input audio will be completely silent. Default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify

the duration of the cross fade effect. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. By default the duration is determined by <var class="var">nb_samples</var>. If set this option is used instead of <var class="var">nb_samples</var>. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Should first stream end overlap with second stream start. Default is enabled. </dd> <dt><samp class="option">curve1</samp></dt> <dd> Set curve for cross fade transition for first stream. </dd> <dt><samp class="option">curve2</samp></dt> <dd> Set curve for cross fade transition for second stream. For description of available curve types see afade filter description. </dd> </dl>

 <h4 class="subsection">8.5.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input to another: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:c1=exp:c2=exp output.flac </pre>

Cross fade from one input to another but without overlapping: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:o=0:c1=exp:c2=exp output.flac </pre>

Concatenate multiple inputs with cross fade between each: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -i third.flac -filter_complex acrossfade=n=3 output.flac </pre>

 <h3 class="section">8.6 acrossover# TOC</h3> Split audio stream into several bands. This filter splits audio stream into two or more frequency ranges. Summing all streams back will give flat output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">split</samp></dt> <dd> Set split frequencies. Those must be positive and increasing. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order for each band split. This controls filter roll-off or steepness of filter transfer function. Available values are: <dl class="table"> <dt>‘<samp class="samp">2nd</samp>’</dt> <dd> 12 dB per octave. </dd> <dt>‘<samp class="samp">4th</samp>’</dt>

<dd> 24 dB per octave. </dd> <dt>‘<samp class="samp">6th</samp>’</dt>
<dd> 36 dB per octave. </dd> <dt>‘<samp class="samp">8th</samp>’</dt>
<dd> 48 dB per octave. </dd> <dt>‘<samp class="samp">10th</samp>’</dt>
<dd> 60 dB per octave. </dd> <dt>‘<samp class="samp">12th</samp>’</dt>
<dd> 72 dB per octave. </dd> <dt>‘<samp class="samp">14th</samp>’</dt>
<dd> 84 dB per octave. </dd> <dt>‘<samp class="samp">16th</samp>’</dt>
<dd> 96 dB per octave. </dd> <dt>‘<samp class="samp">18th</samp>’</dt>
<dd> 108 dB per octave. </dd> <dt>‘<samp class="samp">20th</samp>’</dt>
<dd> 120 dB per octave. </dd> </dl>

Default is <var class="var">4th</var>. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">gains</samp></dt> <dd> Set output gain for each band. Default value is 1 for all bands. </dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt>
<dd> Always use double-floating point precision sample format. </dd> </dl>

Default value is <code class="code">auto</code>. </dd> </dl>

 <h4 class="subsection">8.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Split input audio stream into two bands (low and high) with split frequency of 1500 Hz, each band will be in separate stream: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but with higher filter order: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500:order=8th[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but also with additional middle band (frequencies between 1500 and 8000): <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500 8000:order=8th[LOW][MID][HIGH]' -map '[LOW]' low.wav -map '[MID]' mid.wav -map '[HIGH]' high.wav </pre>

 <h3 class="section">8.7 acrusher<span

class=”pull-right”># TOC</h3>

Reduce audio bit resolution. This filter is bit crusher with enhanced functionality. A bit crusher is used to audibly reduce number of bits an audio signal is sampled with. This doesn’t change the bit depth at all, it just produces the effect. Material reduced in bit depth sounds more harsh and "digital". This filter is able to even round to continuous values instead of discrete bit depths. Additionally it has a D/C offset which results in different crushing of the lower and the upper half of the signal. An Anti-Aliasing setting is able to produce "softer" crushing sounds. Another feature of this filter is the logarithmic mode. This setting switches from linear distances between bits to logarithmic ones. The result is a much more "natural" sounding crusher which doesn’t gate low signals for example. The human ear has a logarithmic perception, so this kind of crushing is much more pleasant. Logarithmic crushing is also able to get anti-aliased. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set level in. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set level out. </dd> <dt><samp class=”option”>bits</samp></dt> <dd> Set bit reduction. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> Set mixing amount. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Can be linear: <code class=”code”>lin</code> or logarithmic: <code class=”code”>log</code>. </dd> <dt><samp class=”option”>dc</samp></dt> <dd> Set DC. </dd> <dt><samp class=”option”>aa</samp></dt> <dd> Set anti-aliasing. </dd> <dt><samp class=”option”>samples</samp></dt> <dd> Set sample reduction. </dd> <dt><samp class=”option”>lfo</samp></dt> <dd> Enable LFO. By default disabled. </dd> <dt><samp class=”option”>lforange</samp></dt> <dd> Set LFO range. </dd> <dt><samp class=”option”>lforate</samp></dt> <dd> Set LFO rate. </dd> </dl>

 <h4 class=”subsection”>8.7.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>8.8 acue# TOC</h3>

Delay audio filtering until a given wallclock timestamp. See the cue filter.

<h3 class="section">8.9 adeclick# TOC</h3> Remove impulsive noise from input audio. Samples detected as impulsive noise are replaced by interpolated samples using autoregressive modelling. <dl class="table"> <dt><samp class="option">window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range is from <code class="code">10</code> to <code class="code">100</code>. Default value is <code class="code">55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class="code">50</code> to <code class="code">95</code>. Default value is <code class="code">75</code> percent. Setting this to a very high value increases impulsive noise removal but makes whole process much slower. </dd> <dt><samp class="option">arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class="code">0</code> to <code class="code">25</code>. Default value is <code class="code">2</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class="code">1</code> to <code class="code">100</code>. Default value is <code class="code">2</code>. This controls the strength of impulsive noise which is going to be removed. The lower value, the more samples will be detected as impulsive noise. </dd> <dt><samp class="option">burst, b</samp></dt> <dd> Set burst fusion, in percentage of window size. Allowed range is <code class="code">0</code> to <code class="code">10</code>. Default value is <code class="code">2</code>. If any two samples detected as noise are spaced less than this value then any sample between those two samples will be also detected as noise. </dd> <dt><samp class="option">method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class="table"> <dt><samp class="option">add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class="option">save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class="code">a</code>. </dd> </dl>

 <h3 class="section">8.10 adeclip# TOC</h3> Remove clipped samples from input audio. Samples detected as clipped are replaced by interpolated samples using autoregressive modelling. <dl class="table"> <dt><samp class="option">window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range

is from <code class="code">10</code> to <code class="code">100</code>. Default value is <code class="code">55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class="code">50</code> to <code class="code">95</code>. Default value is <code class="code">75</code> percent. </dd> <dt><samp class="option">arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class="code">0</code> to <code class="code">25</code>. Default value is <code class="code">8</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class="code">1</code> to <code class="code">100</code>. Default value is <code class="code">10</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class="option">hsize, n</samp></dt> <dd> Set size of histogram used to detect clips. Allowed range is from <code class="code">100</code> to <code class="code">9999</code>. Default value is <code class="code">1000</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class="option">method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class="table"> <dt><samp class="option">add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class="option">save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class="code">a</code>. </dd> </dl>

 <h3 class="section">8.11 adecorrelate# TOC</h3> Apply decorrelation to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">stages</samp></dt> <dd> Set decorrelation stages of filtering. Allowed range is from 1 to 16. Default value is 6. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set random seed used for setting delay in samples across channels. </dd> </dl>

 <h3 class="section">8.12 adelay# TOC</h3>

Delay one or more audio channels. Samples in delayed channel are filled with silence. The filter accepts the following option: <dl class="table"> <dt><samp class="option">delays</samp></dt>

<dd> Set list of delays in milliseconds for each channel separated by ’|’. Unused delays will be silently ignored. If number of given delays is smaller than number of channels all remaining channels will not be delayed. If you want to delay exact number of samples, append ’S’ to number. If you want instead to delay in seconds, append ’s’ to number. </dd> <dt><samp class="option">all</samp></dt> <dd> Use last set delay for all remaining channels. By default is disabled. This option if enabled changes how option <code class="code">delays</code> is interpreted. </dd> </dl>

 <h4 class="subsection">8.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Delay first channel by 1.5 seconds, the third channel by 0.5 seconds and leave the second channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=1500|0|500 </pre>

Delay second channel by 500 samples, the third channel by 700 samples and leave the first channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=0|500S|700S </pre>

Delay all channels by same number of samples: <pre class="example-preformatted">adelay=delays=64S:all=1 </pre>

 <h3 class="section">8.13 adenorm# TOC</h3> Remedy denormals in audio by adding extremely low-level noise. This filter shall be placed before any filter that can produce denormals. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">level</samp></dt> <dd> Set level of added noise in dB. Default is <code class="code">-351</code>. Allowed range is from -451 to -90. </dd> <dt><samp class="option">type</samp></dt> <dd> Set type of added noise. <dl class="table"> <dt><samp class="option">dc</samp></dt> <dd> Add DC signal. </dd> <dt><samp class="option">ac</samp></dt> <dd> Add AC signal. </dd> <dt><samp class="option">square</samp></dt> <dd> Add square signal. </dd> <dt><samp class="option">pulse</samp></dt> <dd> Add pulse signal. </dd> </dl>

Default is <code class="code">dc</code>. </dd> </dl>

 <h4 class="subsection">8.13.1 Com-

mands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.14 aderivative, aintegral# TOC</h3>

Compute derivative/integral of audio stream. Applying both filters one after another produces original audio. <h3 class="section">8.15 adrc# TOC</h3>

Apply spectral dynamic range controller filter to input audio stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">transfer</samp></dt> <dd> Set the transfer expression. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> current channel number </dd> <dt><samp class="option">sn</samp></dt> <dd> current sample number </dd> <dt><samp class="option">nb_channels</samp></dt> <dd> number of channels </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds </dd> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">p</samp></dt> <dd> current frequency power value, in dB </dd> <dt><samp class="option">f</samp></dt> <dd> current frequency in Hz </dd> </dl>

Default value is <code class="code">p</code>. </dd> <dt><samp class="option">attack</samp></dt> <dd> Set the attack in milliseconds. Default is <code class="code">50</code> milliseconds. Allowed range is from 1 to 1000 milliseconds. </dd> <dt><samp class="option">release</samp></dt> <dd> Set the release in milliseconds. Default is <code class="code">100</code> milliseconds. Allowed range is from 5 to 2000 milliseconds. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set which channels to filter, by default <code class="code">all</code> channels in audio stream are filtered. </dd> </dl>

 <h4 class="subsection">8.15.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h4 class="subsection">8.15.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply spectral compression to all frequencies with threshold of -50 dB and 1:6 ratio: <pre class="example-preformatted">adrc=transfer='if(gt(p,-50),-50+(p-(-50))/6,p)':attack=50:release=100 </pre>

Similar to above but with 1:2 ratio and filtering only front center channel: <pre class="example-preformatted">adrc=transfer='if(gt(p,-50),-50+(p-(-50))/2,p)':attack=50:release=100:channels=FC </pre>

Apply spectral noise gate to all frequencies with threshold of -85 dB and with short attack time and short release time: <pre class="example-preformatted">adrc=transfer='if(lte(p,-85),p-800,p)':attack=1:release=5 </pre>

Apply spectral expansion to all frequencies with threshold of -10 dB and 1:2 ratio: <pre class="example-preformatted">adrc=transfer='if(lt(p,-10),-10+(p-(-10))*2,p)':attack=50:release=100 </pre>

Apply limiter to max -60 dB to all frequencies, with attack of 2 ms and release of 10 ms: <pre class="example-preformatted">adrc=transfer='min(p,-60)':attack=2:release=10 </pre>

 <h3 class="section">8.16 adynamicequalizer# TOC</h3>

Apply dynamic equalization to input audio stream.

A description of the accepted options follows.

<dl class="table">

<dt><samp class="option">threshold</samp></dt>

<dd> Set the detection threshold used to trigger equalization.

Threshold detection is using detection filter.

Default value is 0. Allowed range is from 0 to 100.

</dd>

<dt><samp class="option">dfrequency</samp></dt>

<dd> Set the detection frequency in Hz used for detection filter used to trigger equalization.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">dqfactor</samp></dt>

<dd> Set the detection resonance factor for detection filter used to trigger equalization.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">tfrequency</samp></dt>

<dd> Set the target frequency of equalization filter.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">tqfactor</samp></dt>

<dd> Set the target resonance factor for target equalization filter.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to rise above

the detection threshold before equalization starts.

Default is 20. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">release</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to fall below the

detection threshold before equalization ends.

Default is 200. Allowed range is between 1 and 2000.

</dd>

<dt><samp class=”option”>ratio</samp></dt>

<dd> Set the ratio by which the equalization gain is raised.

Default is 1. Allowed range is between 0 and 30.

</dd>

<dt><samp class=”option”>makeup</samp></dt>

<dd> Set the makeup offset by which the equalization gain is raised.

Default is 0. Allowed range is between 0 and 100.

</dd>

<dt><samp class=”option”>range</samp></dt>

<dd> Set the max allowed cut/boost amount. Default is 50.

Allowed range is from 1 to 200.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set the mode of filter operation, can be one of the following:

<dl class=”table”>

<dt>‘<samp class=”samp”>listen</samp>’</dt>

<dd> Output only isolated detection signal.

</dd>

<dt>‘<samp class=”samp”>cutbelow</samp>’</dt>

<dd> Cut frequencies below detection threshold.

</dd>

<dt>‘<samp class="samp">cutabove</samp>’</dt>
<dd> Cut frequencies above detection threshold.
</dd>
<dt>‘<samp class="samp">boostbelow</samp>’</dt>
<dd> Boost frequencies below detection threshold.
</dd>
<dt>‘<samp class="samp">boostabove</samp>’</dt>
<dd> Boost frequencies above detection threshold.
</dd>
</dl>
Default mode is ‘<samp class="samp">cutbelow</samp>’.

</dd>
<dt><samp class="option">dftype</samp></dt>
<dd> Set the type of detection filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bandpass</samp>’</dt>
<dt>‘<samp class="samp">lowpass</samp>’</dt>
<dt>‘<samp class="samp">highpass</samp>’</dt>
<dt>‘<samp class="samp">peak</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bandpass</samp>’.

</dd>
<dt><samp class="option">tftype</samp></dt>
<dd> Set the type of target filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bell</samp>’</dt>
<dt>‘<samp class="samp">lowshelf</samp>’</dt>

<dt>‘<samp class="samp">highshelf</samp>’</dt>

</dl>

Default type is ‘<samp class="samp">bell</samp>’.

</dd>

<dt><samp class="option">auto</samp></dt>

<dd> Automatically gather threshold from detection filter. By default

is ‘<samp class="samp">disabled</samp>’.

This option is useful to detect threshold in certain time frame of

input audio stream, in such case option value is changed at runtime.

Available values are:

<dl class="table">

<dt>‘<samp class="samp">disabled</samp>’</dt>

<dd> Disable using automatically gathered threshold value.

</dd>

<dt>‘<samp class="samp">off</samp>’</dt>

<dd> Stop picking threshold value.

</dd>

<dt>‘<samp class="samp">on</samp>’</dt>

<dd> Start picking threshold value.

</dd>

<dt>‘<samp class="samp">adaptive</samp>’</dt>

<dd> Adaptively pick threshold value, by calculating sliding window entropy.

</dd>

</dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt>

<dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class="subsection">8.16.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.17 adynamicsmooth# TOC</h3>

Apply dynamic smoothing to input audio stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sensitivity</samp></dt> <dd> Set an amount of sensitivity to frequency fluctations. Default is 2. Allowed range is from 0 to 1e+06. </dd> <dt><samp class="option">basefreq</samp></dt> <dd> Set a base frequency for smoothing. Default value is 22050. Allowed range is from 2 to 1e+06. </dd> </dl>

 <h4 class="subsection">8.17.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.18 aecho# TOC</h3>

Apply echoing to the input audio. Echoes are reflected sound and can occur naturally amongst mountains (and sometimes large buildings) when talking or shouting; digital echo effects emulate this behaviour and are often used to help fill out the sound of a single instrument or vocal. The time difference between the original signal and the reflection is the <code class="code">delay</code>, and the loudness of the reflected signal is the <code class="code">decay</code>. Multiple echoes can have different delays and decays. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain of reflected signal. Default is <code class="code">0.6</code>. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain of reflected signal. Default is <code class="code">0.3</code>. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set list of time intervals in milliseconds between original signal and reflections separated by ’|’. Allowed range for each <code

class=”code”>delay</code> is <code class=”code”>(0 - 90000.0]</code>. Default is <code class=”code”>1000</code>. </dd> <dt><samp class=”option”>decays</samp></dt> <dd> Set list of loudness of reflected signals separated by ’|’. Allowed range for each <code class=”code”>decay</code> is <code class=”code”>(0 - 1.0]</code>. Default is <code class=”code”>0.5</code>. </dd> </dl>

 <h4 class=”subsection”>8.18.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make it sound as if there are twice as many instruments as are actually playing: <pre class=”example-preformatted”>aecho=0.8:0.88:60:0.4 </pre>

If delay is very short, then it sounds like a (metallic) robot playing music: <pre class=”example-preformatted”>aecho=0.8:0.88:6:0.4 </pre>

A longer delay will sound like an open air concert in the mountains: <pre class=”example-preformatted”>aecho=0.8:0.9:1000:0.3 </pre>

Same as above but with one more mountain: <pre class=”example-preformatted”>aecho=0.8:0.9:1000|1800:0.3|0.25 </pre>

 <h3 class=”section”>8.19 aemphasis# TOC</h3> Audio emphasis filter creates or restores material directly taken from LPs or emphased CDs with different filter curves. E.g. to store music on vinyl the signal has to be altered by a filter first to even out the disadvantages of this recording medium. Once the material is played back the inverse filter has to be applied to restore the distortion of the frequency response. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set filter mode. For restoring material use <code class=”code”>reproduction</code> mode, otherwise use <code class=”code”>production</code> mode. Default is <code class=”code”>reproduction</code> mode. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set filter type. Selects medium. Can be one of the following: <dl class=”table”> <dt><samp class=”option”>col</samp></dt> <dd> select Columbia. </dd> <dt><samp class=”option”>emi</samp></dt> <dd> select EMI. </dd> <dt><samp class=”option”>bsi</samp></dt> <dd> select

BSI (78RPM). </dd> <dt><samp class="option">riaa</samp></dt>
<dd> select RIAA. </dd> <dt><samp class="option">cd</samp></dt>
<dd> select Compact Disc (CD). </dd> <dt><samp class="option">50fm</samp></dt>
<dd> select 50µs (FM). </dd> <dt><samp class="option">75fm</samp></dt>
<dd> select 75µs (FM). </dd> <dt><samp class="option">50kf</samp></dt>
<dd> select 50µs (FM-KF). </dd> <dt><samp class="option">75kf</samp></dt>
<dd> select 75µs (FM-KF). </dd> </dl> </dd> </dl>

 <h4 class="subsection">8.19.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">8.20 aeval# TOC</h3>

Modify an audio signal according to the specified expressions.
This filter accepts one or more expressions (one for each channel), which are evaluated and used to modify a corresponding audio signal. It accepts the following parameters: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set output channel layout. If not specified, the channel layout is specified by the number of expressions. If set to ‘<samp class="samp">same</samp>’, it will use by default the same input channel layout. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds </dd> <dt><samp class="option">nb_in_channels</samp></dt> <dt><samp class="option">nb_out_channels</samp><
<dd> input and output number of channels </dd> <dt><samp class="option">val(CH)</samp></dt> <dd> the value of input channel with number <var class="var">CH</var> </dd> </dl>

Note: this filter is slow. For faster processing you should use a dedicated filter. <h4 class="subsection">8.20.1 Examples<a class="anchor hidden-xs" href="#Examples-

5" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Half volume: <pre class="example-preformatted">aeval=val(ch)/2:c=same </pre>

Invert phase of the second channel: <pre class="example-preformatted">aeval=val(0)|-val(1) </pre>

 <h3 class="section">8.21 aexciter# TOC</h3>

An exciter is used to produce high sound that is not present in the original signal. This is done by creating harmonic distortions of the signal which are restricted in range and added to the original signal. An Exciter raises the upper end of an audio signal without simply raising the higher frequencies like an equalizer would do to create a more "crisp" or "brilliant" sound. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input level prior processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output level after processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set the amount of harmonics added to original signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">drive</samp></dt> <dd> Set the amount of newly created harmonics. Allowed range is from 0.1 to 10. Default value is 8.5. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the octave of newly created harmonics. Allowed range is from -10 to 10. Default value is 0. </dd> <dt><samp class="option">freq</samp></dt> <dd> Set the lower frequency limit of producing harmonics in Hz. Allowed range is from 2000 to 12000 Hz. Default is 7500 Hz. </dd> <dt><samp class="option">ceil</samp></dt> <dd> Set the upper frequency limit of producing harmonics. Allowed range is from 9999 to 20000 Hz. If value is lower than 10000 Hz no limit is applied. </dd> <dt><samp class="option">listen</samp></dt> <dd> Mute the original signal and output only added harmonics. By default is disabled. </dd> </dl>

 <h4 class="subsection">8.21.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3

class=”section”>8.22 afade# TOC</h3>

Apply fade-in/out effect to input audio.

A description of the accepted parameters follows.

<dl class=”table”>

<dt><samp class=”option”>type, t</samp></dt>

<dd> Specify the effect type, can be either <code class=”code”>in</code> for fade-in, or

<code class=”code”>out</code> for a fade-out effect. Default is <code class=”code”>in</code>.

</dd>

<dt><samp class=”option”>start_sample, ss</samp></dt>

<dd> Specify the number of the start sample for starting to apply the fade

effect. Default is 0.

</dd>

<dt><samp class=”option”>nb_samples, ns</samp></dt>

<dd> Specify the number of samples for which the fade effect has to last. At

the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence. Default is 44100.

</dd>

<dt><samp class=”option”>start_time, st</samp></dt>

<dd> Specify the start time of the fade effect. Default is 0.

The value must be specified as a time duration; see

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If set this option is used instead of <var class="var">start_sample</var>.

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Specify the duration of the fade effect. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

At the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence.

By default the duration is determined by <var class="var">nb_samples</var>.

If set this option is used instead of <var class="var">nb_samples</var>.

</dd>

<dt><samp class="option">curve</samp></dt>

<dd> Set curve for fade transition.

It accepts the following values:

<dl class="table">

<dt><samp class="option">tri</samp></dt>

<dd> select triangular, linear slope (default)

</dd>

<dt><samp class="option">qsin</samp></dt>

<dd> select quarter of sine wave

</dd>

<dt><samp class="option">hsin</samp></dt>

<dd> select half of sine wave

</dd>

<dt><samp class="option">esin</samp></dt>

<dd> select exponential sine wave
</dd>
<dt><samp class=”option”>log</samp></dt>
<dd> select logarithmic
</dd>
<dt><samp class=”option”>ipar</samp></dt>
<dd> select inverted parabola
</dd>
<dt><samp class=”option”>qua</samp></dt>
<dd> select quadratic
</dd>
<dt><samp class=”option”>cub</samp></dt>
<dd> select cubic
</dd>
<dt><samp class=”option”>squ</samp></dt>
<dd> select square root
</dd>
<dt><samp class=”option”>cbr</samp></dt>
<dd> select cubic root
</dd>
<dt><samp class=”option”>par</samp></dt>
<dd> select parabola
</dd>
<dt><samp class=”option”>exp</samp></dt>
<dd> select exponential
</dd>
<dt><samp class=”option”>iqsin</samp></dt>
<dd> select inverted quarter of sine wave
</dd>
<dt><samp class=”option”>ihsin</samp></dt>
<dd> select inverted half of sine wave

</dd>
<dt><samp class="option">dese</samp></dt>
<dd> select double-exponential seat
</dd>
<dt><samp class="option">desi</samp></dt>
<dd> select double-exponential sigmoid
</dd>
<dt><samp class="option">losi</samp></dt>
<dd> select logistic sigmoid
</dd>
<dt><samp class="option">sinc</samp></dt>
<dd> select sine cardinal function
</dd>
<dt><samp class="option">isinc</samp></dt>
<dd> select inverted sine cardinal function
</dd>
<dt><samp class="option">quat</samp></dt>
<dd> select quartic
</dd>
<dt><samp class="option">quatr</samp></dt>
<dd> select quartic root
</dd>
<dt><samp class="option">qsin2</samp></dt>
<dd> select squared quarter of sine wave
</dd>
<dt><samp class="option">hsin2</samp></dt>
<dd> select squared half of sine wave
</dd>
<dt><samp class="option">nofade</samp></dt>
<dd> no fade applied
</dd>

</dl>

</dd> <dt><samp class="option">silence</samp></dt> <dd> Set the initial gain for fade-in or final gain for fade-out. Default value is <code class="code">0.0</code>. </dd> <dt><samp class="option">unity</samp></dt> <dd> Set the initial gain for fade-out or final gain for fade-in. Default value is <code class="code">1.0</code>. </dd> </dl>

 <h4 class="subsection">8.22.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">8.22.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in first 15 seconds of audio: <pre class="example-preformatted">afade=t=in:ss=0:d=15 </pre>

Fade out last 25 seconds of a 900 seconds audio: <pre class="example-preformatted">afade=t=out:st=875:d=25 </pre>

 <h3 class="section">8.23 afftdn# TOC</h3> Denoise audio samples with FFT. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">noise_reduction, nr</samp></dt> <dd> Set the noise reduction in dB, allowed range is 0.01 to 97. Default value is 12 dB. </dd> <dt><samp class="option">noise_floor, nf</samp></dt> <dd> Set the noise floor in dB, allowed range is -80 to -20. Default value is -50 dB. </dd> <dt><samp class="option">noise_type, nt</samp></dt> <dd> Set the noise type. It accepts the following values: <dl class="table"> <dt><samp class="option">white, w</samp></dt> <dd> Select white noise. </dd> <dt><samp class="option">vinyl, v</samp></dt> <dd> Select vinyl noise. </dd> <dt><samp class="option">shellac, s</samp></dt> <dd> Select shellac noise. </dd> <dt><samp class="option">custom, c</samp></dt> <dd> Select custom noise, defined in <code class="code">bn</code> option. Default value is white noise. </dd> </dl>

</dd> <dt><samp class="option">band_noise, bn</samp></dt> <dd> Set custom band noise profile for every one of 15 bands. Bands are separated by ’ ’ or ’|’. </dd>

<dt><samp class=”option”>residual_floor, rf</samp></dt> <dd> Set the residual floor in dB, allowed range is -80 to -20. Default value is -38 dB. </dd> <dt><samp class=”option”>track_noise, tn</samp></dt> <dd> Enable noise floor tracking. By default is disabled. With this enabled, noise floor is automatically adjusted. </dd> <dt><samp class=”option”>track_residual, tr</samp></dt> <dd> Enable residual tracking. By default is disabled. </dd> <dt><samp class=”option”>output_mode, om</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>input, i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class=”option”>output, o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class=”option”>noise, n</samp></dt> <dd> Pass only noise. Default value is <var class=”var”>output</var>. </dd> </dl>

</dd> <dt><samp class=”option”>adaptivity, ad</samp></dt> <dd> Set the adaptivity factor, used how fast to adapt gains adjustments per each frequency bin. Value <var class=”var”>0</var> enables instant adaptation, while higher values react much slower. Allowed range is from <var class=”var”>0</var> to <var class=”var”>1</var>. Default value is <var class=”var”>0.5</var>. </dd> <dt><samp class=”option”>floor_offset, fo</samp></dt> <dd> Set the noise floor offset factor. This option is used to adjust offset applied to measured noise floor. It is only effective when noise floor tracking is enabled. Allowed range is from <var class=”var”>-2.0</var> to <var class=”var”>2.0</var>. Default value is <var class=”var”>1.0</var>. </dd> <dt><samp class=”option”>noise_link, nl</samp></dt> <dd> Set the noise link used for multichannel audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Use unchanged channel’s noise floor. </dd> <dt><samp class=”option”>min</samp></dt> <dd> Use measured min noise floor of all channels. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Use measured max noise floor of all channels. </dd> <dt><samp class=”option”>average</samp></dt> <dd> Use measured average noise floor of all channels. Default value is <var class=”var”>min</var>. </dd> </dl>

</dd> <dt><samp class=”option”>band_multiplier, bm</samp></dt> <dd> Set the band multiplier factor, used how much to spread bands across frequency bins. Allowed range is from <var class=”var”>0.2</var> to <var class=”var”>5</var>. Default value is <var class=”var”>1.25</var>. </dd> <dt><samp class=”option”>sample_noise, sn</samp></dt> <dd> Toggle capturing and measurement of noise profile from input audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>start, begin</samp></dt> <dd> Start sample noise capture. </dd> <dt><samp class=”option”>stop,

end</samp></dt> <dd> Stop sample noise capture and measure new noise band profile. Default value is <code class="code">none</code>. </dd> </dl>

</dd> <dt><samp class="option">gain_smooth, gs</samp></dt> <dd> Set gain smooth spatial radius, used to smooth gains applied to each frequency bin. Useful to reduce random music noise artefacts. Higher values increases smoothing of gains. Allowed range is from <code class="code">0</code> to <code class="code">50</code>. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">8.23.1 Commands# TOC</h4>

This filter supports the some above mentioned options as commands. <h4 class="subsection">8.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Reduce white noise by 10dB, and use previously measured noise floor of -40dB: <pre class="example-preformatted">afftdn=nr=10:nf=-40 </pre>

Reduce white noise by 10dB, also set initial noise floor to -80dB and enable automatic tracking of noise floor so noise floor will gradually change during processing: <pre class="example-preformatted">afftdn=nr=10:nf=-80:tn=1 </pre>

Reduce noise by 20dB, using noise floor of -40dB and using commands to take noise profile of first 0.4 seconds of input audio: <pre class="example-preformatted">asendcmd=0.0 afftdn sn start,asendcmd=0.4 afftdn sn stop,afftdn=nr=20:nf=-40 </pre>

 <h3 class="section">8.24 afftfilt# TOC</h3> Apply arbitrary expressions to samples in frequency domain. <dl class="table"> <dt><samp class="option">real</samp></dt> <dd> Set frequency domain real expression for each separate channel separated by ’|’. Default is "re". If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">imag</samp></dt> <dd> Set frequency domain imaginary expression for each separate channel separated by ’|’. Default is "im". Each expression in

<var class="var">real</var> and <var class="var">imag</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">b</samp></dt> <dd> current frequency bin number </dd> <dt><samp class="option">nb</samp></dt> <dd> number of available bins </dd> <dt><samp class="option">ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd> <dt><samp class="option">pts</samp></dt> <dd> current frame pts </dd> <dt><samp class="option">re</samp></dt> <dd> current real part of frequency bin of current channel </dd> <dt><samp class="option">im</samp></dt> <dd> current imaginary part of frequency bin of current channel </dd> <dt><samp class="option">real(b, ch)</samp></dt> <dd> Return the value of real part of frequency bin at location (<var class="var">bin</var>,<var class="var">channel</var>) </dd> <dt><samp class="option">imag(b, ch)</samp></dt> <dd> Return the value of imaginary part of frequency bin at location (<var class="var">bin</var>,<var class="var">channel</var>) </dd> </dl>

</dd> <dt><samp class="option">win_size</samp></dt> <dd> Set window size. Allowed range is from 16 to 131072. Default is <code class="code">4096</code> </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann, hanning</samp>’</dt> <dt>‘<samp class="samp">hamming</samp>’</dt> <dt>‘<samp class="samp">blackman</samp>’</dt> <dt>‘<samp class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class="code">0.75</code>. </dd> </dl>

 <h4 class="subsection">8.24.1 Examples<a class="anchor hidden-xs" href="#Examples-

8” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Leave almost only low frequencies in audio: <pre class=”example-preformatted”>afftfilt="'real=re * (1-clip((b/nb)*b,0,1))':imag='im * (1-clip((b/nb)*b,0,1))'" </pre>

Apply robotize effect: <pre class=”example-preformatted”>afftfilt="real='hypot(re,im)*sin(0)':imag='hypot(re,im)*cos(0)':win_size=512:overlap=
</pre>

Apply whisper effect: <pre class=”example-preformatted”>afftfilt="real='hypot(re,im)*cos((random(0)*2-1)*2*3.14)':imag='hypot(re,im)*sin((ran
1)*2*3.14)':win_size=128:overlap=0.8" </pre>

Apply phase shift: <pre class=”example-preformatted”>afftfilt="real=re*cos(1)-im*sin(1):imag=re*sin(1)+im*cos(1)"
</pre>

 <h3 class=”section”>8.25 afir# TOC</h3>

Apply an arbitrary Finite Impulse Response filter.

This filter is designed for applying long FIR filters,

up to 60 seconds long.

It can be used as component for digital crossover filters,

room equalization, cross talk cancellation, wavefield synthesis,

auralization, ambiophonics, ambisonics and spatialization.

This filter uses the streams higher than first one as FIR coefficients.

If the non-first stream holds a single channel, it will be used

for all input channels in the first stream, otherwise

the number of channels in the non-first stream must be same as

the number of channels in the first stream.

It accepts the following parameters:

<dl class="table">

<dt><samp class="option">dry</samp></dt>

<dd> Set dry gain. This sets input gain.

</dd>

<dt><samp class="option">wet</samp></dt>

<dd> Set wet gain. This sets final output gain.

</dd>

<dt><samp class="option">length</samp></dt>

<dd> Set Impulse Response filter length. Default is 1, which means whole IR is processed.

</dd>

<dt><samp class="option">gtype</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">irnorm</samp></dt>

<dd> Set norm to be applied to IR coefficients before filtering.

Allowed range is from <var class="var">-1</var> to <var class="var">2</var>.

IR coefficients are normalized with calculated vector norm set by this option.

For negative values, no norm is calculated, and IR coefficients are not modified at all.

Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">irlink</samp></dt>

<dd> For multichannel IR if this option is set to <var class="var">true</var>, all IR channels will be

normalized with maximal measured gain of all IR channels coefficients as set by <code class="code">irnorm</code> option.

When disabled, all IR coefficients in each IR channel will be normalized independently.

Default is <var class="var">true</var>.

</dd>

<dt><samp class="option">irgain</samp></dt>

<dd> Set gain to be applied to IR coefficients before filtering.

Allowed range is 0 to 1. This gain is applied after any gain applied with <var class="var">irnorm</var> option.

</dd>

<dt><samp class="option">irfmt</samp></dt>

<dd> Set format of IR stream. Can be <code class="code">mono</code> or <code class="code">input</code>.

Default is <code class="code">input</code>.

</dd>

<dt><samp class="option">maxir</samp></dt>

<dd> Set max allowed Impulse Response filter duration in seconds. Default is 30 seconds.

Allowed range is 0.1 to 60 seconds.

</dd>

<dt><samp class="option">response</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">channel</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">size</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">rate</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">minp</samp></dt>

<dd> Set minimal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">1</var> to <var class="var">65536</var>.

Lower values decreases latency at cost of higher CPU usage.

</dd>

<dt><samp class="option">maxp</samp></dt>

<dd> Set maximal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">8</var> to <var class="var">65536</var>.

Lower values may increase CPU usage.

</dd>

<dt><samp class="option">nbirs</samp></dt>

<dd> Set number of input impulse responses streams which will be switchable at runtime.

Allowed range is from <var class="var">1</var> to <var class="var">32</var>. Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">ir</samp></dt>

<dd> Set IR stream which will be used for convolution, starting from <var class="var">0</var>, should always be

lower than supplied value by <code class="code">nbirs</code> option. Default is <var class="var">0</var>.

This option can be changed at runtime via commands.

</dd>

<dt><samp class=”option”>precision</samp></dt>

<dd> Set which precision to use when processing samples.

<dl class=”table”>

<dt><samp class=”option”>auto</samp></dt>

<dd> Auto pick internal sample format depending on other filters.

</dd>

<dt><samp class=”option”>float</samp></dt>

<dd> Always use single-floating point precision sample format.

</dd>

<dt><samp class=”option”>double</samp></dt>

<dd> Always use double-floating point precision sample format.

</dd>

</dl>

Default value is auto. </dd> <dt><samp class=”option”>irload</samp></dt> <dd> Set when to load IR stream. Can be <code class=”code”>init</code> or <code class=”code”>access</code>. First one load and prepares all IRs on initialization, second one once on first access of specific IR. Default is <code class=”code”>init</code>. </dd> </dl>

 <h4 class=”subsection”>8.25.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply reverb to stream using mono IR file as second input, complete command using ffmpeg: <pre class=”example-preformatted”>ffmpeg -i input.wav -i middle_tunnel_1way_mono.wav -lavfi afir output.wav </pre>

Apply true stereo processing given input stereo stream, and two stereo impulse responses for left and right channel, the impulse response files are files with names l_ir.wav and r_ir.wav, and setting

irnorm option value: <pre class=”example-preformatted”>"pan=4C|c0=FL|c1=FL|c2=FR|c3=FR[a];amovie=l_ir.wav[LIR];amovie=r_ir.wav[RIR
</pre>

Similar to above example, but with <code class=”code”>irgain</code> explicitly set to estimated value and with <code class=”code”>irnorm</code> disabled: <pre class=”example-preformatted”>"pan=4C|c0=FL|c1=FL|c2=FR 5dB:irnom=-1,pan=stereo|FL<c0+c2|FR<c1+c3" </pre>

 <h3 class=”section”>8.26 aformat# TOC</h3>

Set output format constraints for the input audio. The framework will negotiate the most appropriate format to minimize conversions. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>sample_fmts, f</samp></dt> <dd> A ’|’-separated list of requested sample formats. </dd> <dt><samp class=”option”>sample_rates, r</samp></dt> <dd> A ’|’-separated list of requested sample rates. </dd> <dt><samp class=”option”>channel_layouts, cl</samp></dt> <dd> A ’|’-separated list of requested channel layouts. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#channel-layout-syntax”>the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> </dl>

If a parameter is omitted, all values are allowed. Force the output to either unsigned 8-bit or signed 16-bit stereo <pre class=”example-preformatted”>aformat=sample_fmts=u8|s16:channel_layouts=stereo
</pre>

 <h3 class=”section”>8.27 afreqshift# TOC</h3> Apply frequency shift to input audio samples. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>shift</samp></dt> <dd> Specify frequency shift. Allowed range is -INT_MAX to INT_MAX. Default value is 0.0. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class=”subsection”>8.27.1 Commands<a class=”anchor hidden-xs” href=”#Commands-

10" aria-hidden="true"># TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.28 afwtdn# TOC</h3> Reduce broadband noise from input samples using Wavelets. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma, allowed range is from 0 to 1. Default value is 0. This option controls strength of denoising applied to input samples. Most useful way to set this option is via decibels, eg. -45dB. </dd> <dt><samp class="option">levels</samp></dt> <dd> Set the number of wavelet levels of decomposition. Allowed range is from 1 to 12. Default value is 10. Setting this too low make denoising performance very poor. </dd> <dt><samp class="option">wavet</samp></dt> <dd> Set wavelet type for decomposition of input frame. They are sorted by number of coefficients, from lowest to highest. More coefficients means worse filtering speed, but overall better quality. Available wavelets are: <dl class="table"> <dt>‘<samp class="samp">sym2</samp>’</dt> <dt>‘<samp class="samp">sym4</samp>’</dt> <dt>‘<samp class="samp">rbior68</samp>’</dt> <dt>‘<samp class="samp">deb10</samp>’</dt> <dt>‘<samp class="samp">sym10</samp>’</dt> <dt>‘<samp class="samp">coif5</samp>’</dt> <dt>‘<samp class="samp">bl3</samp>’</dt> </dl>

</dd> <dt><samp class="option">percent</samp></dt> <dd> Set percent of full denoising. Allowed range is from 0 to 100 percent. Default value is 85 percent or partial denoising. </dd> <dt><samp class="option">profile</samp></dt> <dd> If enabled, first input frame will be used as noise profile. If first frame samples contain non-noise performance will be very poor. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> If enabled, input frames are analyzed for presence of noise. If noise is detected with high possibility then input frame profile will be used for processing following frames, until new noise frame is detected. </dd> <dt><samp class="option">samples</samp></dt> <dd> Set size of single frame in number of samples. Allowed range is from 512 to 65536. Default frame size is 8192 samples. </dd> <dt><samp class="option">softness</samp></dt> <dd> Set softness applied inside thresholding function. Allowed range is from 0 to 10. Default softness is 1. </dd> </dl>

 <h4 class="subsection">8.28.1 Commands# <a class="anchor hidden-xs"href="#toc-

Commands-11" aria-hidden="true">TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.29 agate# TOC</h3>

A gate is mainly used to reduce lower parts of a signal. This kind of signal processing reduces disturbing noise between useful signals. Gating is done by detecting the volume below a chosen level <var class="var">threshold</var> and dividing it by the factor set with <var class="var">ratio</var>. The bottom of the noise floor is set via <var class="var">range</var>. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over time. This is done by setting <var class="var">attack</var> and <var class="var">release</var>. <var class="var">attack</var> determines how long the signal has to fall below the threshold before any reduction will occur and <var class="var">release</var> sets the time the signal has to rise above the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the mode of operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. If set to <code class="code">upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class="code">downward</code> lower parts of signal will be reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1

to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is <code class="code">rms</code>. Can be <code class="code">peak</code> or <code class="code">rms</code>. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is <code class="code">average</code>. Can be <code class="code">average</code> or <code class="code">maximum</code>. </dd> </dl>

 <h4 class="subsection">8.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.30 aiir# TOC</h3>

Apply an arbitrary Infinite Impulse Response filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">zeros, z</samp></dt> <dd> Set B/numerator/zeros/reflection coefficients. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set A/denominator/poles/ladder coefficients. </dd> <dt><samp class="option">gains, k</samp></dt> <dd> Set channels gains. </dd> <dt><samp class="option">dry_gain</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">wet_gain</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">format, f</samp></dt> <dd> Set coefficients format. <dl class="table"> <dt>‘<samp class="samp">ll</samp>’</dt> <dd> lattice-ladder function </dd> <dt>‘<samp class="samp">sf</samp>’</dt> <dd> analog transfer function </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> digital transfer function </dd> <dt>‘<samp class="samp">zp</samp>’</dt> <dd> Z-plane zeros/poles, cartesian (default) </dd> <dt>‘<samp class="samp">pr</samp>’</dt> <dd> Z-plane zeros/poles, polar radians </dd> <dt>‘<samp class="samp">pd</samp>’</dt> <dd> Z-plane zeros/poles, polar degrees </dd> <dt>‘<samp class="samp">sp</samp>’</dt> <dd> S-plane zeros/poles </dd> </dl>

</dd> <dt><samp class="option">process, r</samp></dt> <dd> Set type of processing. <dl class="table"> <dt>‘<samp class="samp">d</samp>’</dt> <dd> direct processing

</dd> <dt>‘<samp class=”samp”>s</samp>’</dt>
<dd> serial processing </dd> <dt>‘<samp class=”samp”>p</samp>’</dt>
<dd> parallel processing </dd> </dl>

</dd> <dt><samp class=”option”>precision, e</samp></dt> <dd> Set
filtering precision. <dl class=”table”> <dt>‘<samp class=”samp”>dbl</samp>’</dt>
<dd> double-precision floating-point (default) </dd> <dt>‘<samp
class=”samp”>flt</samp>’</dt> <dd> single-precision floating-
point </dd> <dt>‘<samp class=”samp”>i32</samp>’</dt>
<dd> 32-bit integers </dd> <dt>‘<samp class=”samp”>i16</samp>’</dt>
<dd> 16-bit integers </dd> </dl>

</dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize
filter coefficients, by default is enabled. Enabling it will normalize magnitude re-
sponse at DC to 0dB. </dd> <dt><samp class=”option”>mix</samp></dt>
<dd> How much to use filtered signal in output. Default is 1. Range is be-
tween 0 and 1. </dd> <dt><samp class=”option”>response</samp></dt>
<dd> Show IR frequency response, magnitude(magenta), phase(green) and
group delay(yellow) in additional video stream. By default it is disabled.
</dd> <dt><samp class=”option”>channel</samp></dt> <dd> Set
for which IR channel to display frequency response. By default is first channel
displayed. This option is used only when <var class=”var”>response</var>
is enabled. </dd> <dt><samp class=”option”>size</samp></dt>
<dd> Set video stream size. This option is used only when <var
class=”var”>response</var> is enabled. </dd> </dl>

Coefficients in <code class=”code”>tf</code> and <code class=”code”>sf</code>
format are separated by spaces and are in ascending order.
Coefficients in <code class=”code”>zp</code> format are separated
by spaces and order of coefficients doesn’t matter. Coefficients
in <code class=”code”>zp</code> format are complex numbers with
<var class=”var”>i</var> imaginary unit. Different coef-
ficients and gains can be provided for every channel, in such case use
’|’ to separate coefficients or gains. Last provided coefficients
will be used for all remaining channels.
<h4 class=”subsection”>8.30.1 Examples<a
class=”anchor hidden-xs” href=”#Examples-10” aria-hidden=”true”>#
TOC</h4>

<ul class=”itemize mark-bullet”> Apply 2 pole elliptic notch at
around 5000Hz for 48000 Hz sample rate:
<pre class=”example-preformatted”>aiir=k=1:z=7.957584807809675810E-
1 -2.575128568908332300 3.674839853930788710 -2.57512875289799137
7.957586296317130880E-1:p=1 -2.86950072432325953 3.63022088054647218
-2.28075678147272232 6.361362326477423500E-1:f=tf:r=d </pre>

Same as above but in <code class=”code”>zp</code> format: <div
class=”example”> <pre class=”example-preformatted”>aiir=k=0.7957584807809675 6:z=0.80918701+0.58773

0.80918701-0.58773007i 0.80884700+0.58784055i 0.80884700-0.58784055i:p=0.63892345+0.59951235i 0.63892345-0.59951235i 0.79582691+0.44198673i 0.79582691-0.44198673i:f=zp:r=s
</pre>

Apply 3-rd order analog normalized Butterworth low-pass filter, using analog transfer function format: <pre class=”example-preformatted”>aiir=z=1.3057 0 0 0:p=1.3057 2.3892 2.1860 1:f=sf:r=d </pre>

 <h3 class=”section”>8.31 alimiter# TOC</h3>

The limiter prevents an input signal from rising over a desired threshold. This limiter uses lookahead technology to prevent your signal from distorting. It means that there is a small delay after the signal is processed. Keep in mind that the delay it produces is the attack time you set. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Don’t let signals above this level pass the limiter. Default is 1. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> The limiter will reach its attenuation level in this amount of time in milliseconds. Default is 5 milliseconds. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Come back from limiting to attenuation 1.0 in this amount of milliseconds. Default is 50 milliseconds. </dd> <dt><samp class=”option”>asc</samp></dt> <dd> When gain reduction is always needed ASC takes care of releasing to an average reduction level rather than reaching a reduction of 0 in the release time. </dd> <dt><samp class=”option”>asc_level</samp></dt> <dd> Select how much the release time is affected by ASC, 0 means nearly no changes in release time while 1 produces higher release times. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Auto level output signal. Default is enabled. This normalizes audio back to 0dB if enabled. </dd> <dt><samp class=”option”>latency</samp></dt> <dd> Compensate the delay introduced by using the lookahead buffer set with attack parameter. Also flush the valid audio data in the lookahead buffer when the stream hits EOF. </dd> </dl>

Depending on picked setting it is recommended to upsample input 2x or 4x times with aresample before applying this filter. <h3 class=”section”>8.32 allpass# TOC</h3>

Apply a two-pole all-pass filter with central frequency (in Hz) <var class=”var”>frequency</var>, and filter-width <var class=”var”>width</var>. An all-pass filter changes the audio’s frequency to phase relationship without changing its frequency to amplitude relationship. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>order, o</samp></dt> <dd> Set the filter order, can be 1 or 2. Default is 2. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>8.32.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>

<dt><samp class="option">frequency, f</samp></dt> <dd> Change allpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change allpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change allpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change allpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">8.33 aloop# TOC</h3>

Loop audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal number of samples. Default is 0. </dd> <dt><samp class="option">start</samp></dt> <dd> Set first sample of loop. Default is 0. </dd> <dt><samp class="option">time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class="var">start</var> is set to <code class="code">-1</code>. </dd> </dl>

 <h3 class="section">8.34 amerge# TOC</h3>

Merge two or more audio streams into a single multi-channel stream. All inputs must have the same sample rate, and format. If inputs do not have the same duration, the output will stop with the shortest. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set the number of inputs. Default is 2. </dd> <dt><samp class="option">layout_mode</samp></dt> <dd> This option controls how the output channel layout is determined and if the audio channels are reordered during merge. <dl class="table"> <dt><samp class="option">legacy</samp></dt> <dd> This is the mode how the filter behaved historically so it is the default. If the channel layouts of the inputs are known and disjoint, and therefore compatible, the channel layout of the output will be set accordingly and the channels will be reordered as necessary. If the channel layouts of the inputs are not disjoint, some of them are unknown, or they are using special channel layouts, such as ambisonics, the output will have all the channels of the first input then all the channels of the second input, in that order, and the channel layout

of the output will be the default value corresponding to the total number of channels. For example, if the first input is in 2.1 (FL+FR+LF) and the second input is FC+BL+BR, then the output will be in 5.1, with the channels in the following order: a1, a2, b1, a3, b2, b3 (a1 is the first channel of the first input, b1 is the first channel of the second input). On the other hand, if both input are in stereo, the output channels will be in the default order: a1, a2, b1, b2, and the channel layout will be arbitrarily set to 4.0, which may or may not be the expected value. </dd> <dt><samp class="option">reset</samp></dt> <dd> This mode ignores the input channel layouts and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. The output channel layout will only specify the total channel count. </dd> <dt><samp class="option">normal</samp></dt> <dd> This mode keeps channel name and designation information from the input channels and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">8.34.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Merge two mono files into a stereo stream: <pre class="example-preformatted">amovie=left.wav [l] ; amovie=right.mp3 [r] ; [l] [r] amerge </pre>

Multiple merges assuming 1 video stream and 6 audio streams in <samp class="file">input.mkv</samp>: <pre class="example-preformatted">ffmpeg -i input.mkv -filter_complex "[0:1][0:2][0:3][0:4][0:5][0:6] amerge=inputs=6" -c:a pcm_s16le output.mkv </pre>

 <h3 class="section">8.35 amix# TOC</h3>

Mixes multiple audio inputs into a single output. Note that this filter only supports float samples (the <var class="var">amerge</var> and <var class="var">pan</var> audio filters support many formats). If the <var class="var">amix</var> input has integer samples then aresample will be automatically inserted to perform the conversion to float samples. It accepts the following parameters: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp

class=”option”>longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class=”option”>first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> <dt><samp class=”option”>dropout_transition</samp></dt> <dd> The transition time, in seconds, for volume renormalization when an input stream ends. The default value is 2 seconds. </dd> <dt><samp class=”option”>weights</samp></dt> <dd> Specify weight of each input audio stream as a sequence of numbers separated by a space. If fewer weights are specified compared to number of inputs, the last weight is assigned to the remaining inputs. Default weight for each input is 1. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Always scale inputs instead of only doing summation of samples. Beware of heavy clipping if inputs are not normalized prior or after filtering by this filter if this option is disabled. By default is enabled. </dd> </dl>

 <h4 class=”subsection”>8.35.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> This will mix 3 input audio streams to a single output with the same duration as the first input and a dropout transition time of 3 seconds: <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex amix=inputs=3:duration=first:dropout_transition=3 OUTPUT </pre>

This will mix one vocal and one music input audio stream to a single output with the same duration as the longest input. The music will have quarter the weight as the vocals, and the inputs are not normalized: <pre class=”example-preformatted”>ffmpeg -i VOCALS -i MUSIC -filter_complex amix=inputs=2:duration=longest:dropout_transition=0:weights="1 0.25":normalize=0 OUTPUT </pre>

 <h4 class=”subsection”>8.35.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>weights</samp></dt> <dt><samp class=”option”>normalize</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class=”section”>8.36 amultiply<a class=”anchor hidden-xs” href=”#amultiply” aria-

hidden=”true”># TOC</h3>

Multiply first audio stream with second audio stream and store result in output audio stream. Multiplication is done by multiplying each sample from first stream with sample at same position from second stream. With this element-wise multiplication one can create amplitude fades and amplitude modulations. <h3 class=”section”>8.37 anequalizer# TOC</h3>

High-order parametric multiband equalizer for each channel. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>params</samp></dt> <dd> This option string is in format: "c<var class=”var”>chn</var> f=<var class=”var”>cf</var> w=<var class=”var”>w</var> g=<var class=”var”>g</var> t=<var class=”var”>f</var> | ..." Each equalizer band is separated by ’|’. <dl class=”table”> <dt><samp class=”option”>chn</samp></dt> <dd> Set channel number to which equalization will be applied. If input doesn’t have that channel the entry is ignored. </dd> <dt><samp class=”option”>f</samp></dt> <dd> Set central frequency for band. If input doesn’t have that frequency the entry is ignored. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set band width in Hertz. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set band gain in dB. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set filter type for band, optional, can be: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Butterworth, this is default. </dd> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Chebyshev type 1. </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> Chebyshev type 2. </dd> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>curves</samp></dt> <dd> With this option activated frequency response of anequalizer is displayed in video stream. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set video stream size. Only useful if curves option is activated. </dd> <dt><samp class=”option”>mgain</samp></dt> <dd> Set max gain that will be displayed. Only useful if curves option is activated. Setting this to a reasonable value makes it possible to display gain which is derived from neighbour bands which are too close to each other and thus produce higher gain when both are activated. </dd> <dt><samp class=”option”>fscale</samp></dt> <dd> Set frequency scale used to draw frequency response in video output. Can be linear or logarithmic. Default is logarithmic. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set color for each channel curve which is going to be displayed in video stream. This is list of color names

separated by space or by ’|’. Unrecognised or missing colors will be replaced by white color. </dd> </dl>

 <h4 class="subsection">8.37.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Lower gain by 10 of central frequency 200Hz and width 100 Hz for first 2 channels using Chebyshev type 1 filter: <pre class="example-preformatted">anequalizer=c0 f=200 w=100 g=-10 t=1|c1 f=200 w=100 g=-10 t=1 </pre>

 <h4 class="subsection">8.37.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">change</samp></dt> <dd> Alter existing filter parameters. Syntax for the commands is : "<var class="var">fN</var>|f=<var class="var">freq</var>|w=<var class="var">width</var>|g=<var class="var">gain</var>" <var class="var">fN</var> is existing filter number, starting from 0, if no such filter is available error is returned. <var class="var">freq</var> set new frequency parameter. <var class="var">width</var> set new width parameter in Hertz. <var class="var">gain</var> set new gain parameter in dB. Full filter invocation with asendcmd may look like this: asendcmd=c=’4.0 anequalizer change 0|f=200|w=50|g=1’,anequalizer=... </dd> </dl>

 <h3 class="section">8.38 anlmdn# TOC</h3>

Reduce broadband noise in audio samples using Non-Local Means algorithm. Each sample is adjusted by looking for other samples with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp> around the sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength, s</samp></dt> <dd> Set denoising strength. Allowed range is from 0.00001 to 10000. Default value is 0.00001. </dd> <dt><samp class="option">patch, p</samp></dt> <dd> Set patch radius duration. Allowed range is from 1 to 100 milliseconds. Default value is 2 milliseconds. </dd> <dt><samp class="option">research, r</samp></dt> <dd> Set research radius duration. Allowed range is from 2 to 300

milliseconds. Default value is 6 milliseconds. </dd> <dt><samp class="option">output, o</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass only noise. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">smooth, m</samp></dt> <dd> Set smooth factor. Default value is <var class="var">11</var>. Allowed range is from <var class="var">1</var> to <var class="var">1000</var>. </dd> </dl>

 <h4 class="subsection">8.38.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.39 anlmf, anlms# TOC</h3> Apply Normalized Least-Mean-(Squares|Fourth) algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by finding the filter coefficients that relate to producing the least mean square of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set filter order. </dd> <dt><samp class="option">mu</samp></dt> <dd> Set filter mu. </dd> <dt><samp class="option">eps</samp></dt> <dd> Set the filter eps. </dd> <dt><samp class="option">leakage</samp></dt> <dd> Set the filter leakage. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table">

<dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>8.39.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> One of many usages of this filter is noise reduction, input audio is filtered with same samples that are delayed by fixed amount, one such example for stereo audio is: <pre class=”example-preformatted”>asplit[a][b],[a]adelay=32S|32S[a],[b][a]anlms=order=128:leakage=0.0005:mu=.
</pre>

 <h4 class=”subsection”>8.39.2 Commands# TOC</h4>

This filter supports the same commands as options, excluding option <code class=”code”>order</code>. <h3 class=”section”>8.40 anull# TOC</h3>

Pass the audio source unchanged to the output. <h3 class=”section”>8.41 apad# TOC</h3>

Pad the end of an audio stream with silence. This can be used together with <code class=”command”>ffmpeg</code> <samp class=”option”>-shortest</samp> to extend audio streams to the same length as the video stream. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>packet_size</samp></dt> <dd> Set silence packet size. Default value is 4096. </dd> <dt><samp class=”option”>pad_len</samp></dt> <dd> Set the number of samples of silence to add to the end. After the value is reached, the stream is terminated. This option is mutually exclusive with <samp class=”option”>whole_len</samp>. </dd> <dt><samp class=”option”>whole_len</samp></dt> <dd> Set the minimum total number of samples in the output audio stream. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class=”option”>pad_len</samp>. </dd> <dt><samp class=”option”>pad_dur</samp></dt>

<dd> Specify the duration of samples of silence to add. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. </dd> <dt><samp class="option">whole_dur</samp></dt> <dd> Specify the minimum total duration in the output audio stream. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_dur</samp> </dd> </dl>

If neither the <samp class="option">pad_len</samp> nor the <samp class="option">whole_len</samp> nor <samp class="option">pad_dur</samp> nor <samp class="option">whole_dur</samp> option is set, the filter will add silence to the end of the input stream indefinitely. Note that for ffmpeg 4.4 and earlier a zero <samp class="option">pad_dur</samp> or <samp class="option">whole_dur</samp> also caused the filter to add silence indefinitely. <h4 class="subsection">8.41.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add 1024 samples of silence to the end of the input: <pre class="example-preformatted">apad=pad_len=1024 </pre>

Make sure the audio output will contain at least 10000 samples, pad the input with silence if required: <pre class="example-preformatted">apad=whole_len=10000 </pre>

Use <code class="command">ffmpeg</code> to pad the audio input with silence, so that the video stream will always result the shortest and will be converted until the end in the output file when using the <samp class="option">shortest</samp> option: <pre class="example-preformatted">ffmpeg -i VIDEO -i AUDIO -filter_complex "[1:0]apad" -shortest OUTPUT </pre>

 <h3 class="section">8.42 aphaser# TOC</h3> Add a phasing effect to the input audio. A phaser filter creates series of peaks and troughs in the frequency spectrum. The position of the peaks and troughs are modulated so that they vary over time, creating a sweeping effect. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp

class=”option”>out_gain</samp></dt> <dd> Set output gain. Default is 0.74 </dd> <dt><samp class=”option”>delay</samp></dt> <dd> Set delay in milliseconds. Default is 3.0. </dd> <dt><samp class=”option”>decay</samp></dt> <dd> Set decay. Default is 0.4. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set modulation speed in Hz. Default is 0.5. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set modulation type. Default is triangular. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>triangular, t</samp>’</dt> <dt>‘<samp class=”samp”>sinusoidal, s</samp>’</dt> </dl> </dd> </dl>

 <h3 class=”section”>8.43 aphaseshift# TOC</h3> Apply phase shift to input audio samples. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>shift</samp></dt> <dd> Specify phase shift. Allowed range is from -1.0 to 1.0. Default value is 0.0. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class=”subsection”>8.43.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>8.44 apsnr# TOC</h3> Measure Audio Peak Signal-to-Noise Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class=”section”>8.45 apsyclip# TOC</h3> Apply Psychoacoustic clipper to input audio stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. By

default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">clip</samp></dt> <dd> Set the clipping start value. Default value is 0dBFS or 1. </dd> <dt><samp class="option">diff</samp></dt> <dd> Output only difference samples, useful to hear introduced distortions. By default is disabled. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> Set strength of adaptive distortion applied. Default value is 0.5. Allowed range is from 0 to 1. </dd> <dt><samp class="option">iterations</samp></dt> <dd> Set number of iterations of psychoacoustic clipper. Allowed range is from 1 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Auto level output signal. Default is disabled. This normalizes audio back to 0dBFS if enabled. </dd> </dl>

 <h4 class="subsection">8.45.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.46 apulsator# TOC</h3>

Audio pulsator is something between an autopanner and a tremolo. But it can produce funny stereo effects as well. Pulsator changes the volume of the left and right channel based on a LFO (low frequency oscillator) with different waveforms and shifted phases. This filter have the ability to define an offset between left and right channel. An offset of 0 means that both LFO shapes match each other. The left and right channel are altered equally - a conventional tremolo. An offset of 50% means that the shape of the right channel is exactly shifted in phase (or moved backwards about half of the frequency) - pulsator acts as an autopanner. At 1 both curves match again. Every setting in between moves the phase shift gapless between all stages and produces some "bypassing" sounds with sine and triangle waveforms. The more you set the offset near 1 (starting from the 0.5) the faster the signal passes from the left to the right speaker. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set waveform shape the LFO will use. Can be one of: sine, triangle, square, sawup or sawdown. Default is sine. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set modulation. Define how much of original signal is affected by the LFO. </dd> <dt><samp class="option">offset_l</samp></dt>

<dd> Set left channel offset. Default is 0. Allowed range is [0 - 1]. </dd> <dt><samp class="option">offset_r</samp></dt> <dd> Set right channel offset. Default is 0.5. Allowed range is [0 - 1]. </dd> <dt><samp class="option">width</samp></dt> <dd> Set pulse width. Default is 1. Allowed range is [0 - 2]. </dd> <dt><samp class="option">timing</samp></dt> <dd> Set possible timing mode. Can be one of: bpm, ms or hz. Default is hz. </dd> <dt><samp class="option">bpm</samp></dt> <dd> Set bpm. Default is 120. Allowed range is [30 - 300]. Only used if timing is set to bpm. </dd> <dt><samp class="option">ms</samp></dt> <dd> Set ms. Default is 500. Allowed range is [10 - 2000]. Only used if timing is set to ms. </dd> <dt><samp class="option">hz</samp></dt> <dd> Set frequency in Hz. Default is 2. Allowed range is [0.01 - 100]. Only used if timing is set to hz. </dd> </dl>

 <h3 class="section">8.47 aresample# TOC</h3>

Resample the input audio to the specified parameters, using the libswresample library. If none are specified then the filter will automatically convert between its input and output. This filter is also able to stretch/squeeze the audio data to make it match the timestamps or to inject silence / cut out audio to make it match the timestamps, do a combination of both or do neither. The filter accepts the syntax [<var class="var">sample_rate</var>:]<var class="var">resampler_options</var>, where <var class="var">sample_rate</var> expresses a sample rate and <var class="var">resampler_options</var> is a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ":". See the <a data-manual="ffmpeg-resampler" href="./ffmpeg-resampler.html#Resampler-Options">"Resampler Options" section in the ffmpeg-resampler(1) manual for the complete list of supported options. <h4 class="subsection">8.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Resample the input audio to 44100Hz: <pre class="example-preformatted">aresample=44100 </pre>

Stretch/squeeze samples to the given timestamps, with a maximum of 1000 samples per second compensation: <pre class="example-preformatted">aresample=async=1000 </pre>

 <h3 class="section">8.48 areverse<span

class=”pull-right”># TOC</h3>

Reverse an audio clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class=”subsection”>8.48.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Take the first 5 seconds of a clip, and reverse it. <pre class=”example-preformatted”>atrim=end=5,areverse </pre>

 <h3 class=”section”>8.49 arls# TOC</h3> Apply Recursive Least Squares algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by recursively finding the filter coefficients that relate to producing the minimal weighted linear least squares cost function of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class=”option”>lambda</samp></dt> <dd> Set the forgetting factor. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class=”option”>out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class=”option”>e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format.

</dd> </dl> </dd> </dl>

 <h3 class="section">8.50 arnndn# TOC</h3>

Reduce noise from speech using Recurrent Neural Networks. This filter accepts the following options: <dl class="table"> <dt><samp class="option">model, m</samp></dt> <dd> Set train model file to load. This option is always required. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how much to mix filtered samples into final output. Allowed range is from -1 to 1. Default value is 1. Negative values are special, they set how much to keep filtered noise in the final filter output. Set this option to -1 to hear actual noise removed from input signal. </dd> </dl>

 <h4 class="subsection">8.50.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.51 asdr# TOC</h3> Measure Audio Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">8.52 asetnsamples# TOC</h3>

Set the number of samples per each output audio frame. The last output packet may contain a different number of samples, as the filter will flush all the remaining samples when the input audio signals its end. The filter accepts the following options: <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of frames per each output audio frame. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio frame with zeroes, so that the last frame will contain the same number of samples as the previous ones. Default value is 1. </dd> </dl>

For example, to set the number of per-frame samples to 1234 and disable padding for the last frame, use: <pre

class=”example-preformatted”>asetnsamples=n=1234:p=0 </pre>

 <h3 class=”section”>8.53 asetrate# TOC</h3>

Set the sample rate without altering the PCM data. This will result in a change of speed and pitch. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the output sample rate. Default is 44100 Hz. </dd> </dl>

 <h3 class=”section”>8.54 ashowinfo# TOC</h3>

Show a line containing various information for each input audio frame. The input audio is not modified. The shown line contains a sequence of key/value pairs of the form <var class=”var”>key</var>:<var class=”var”>value</var>. The following values are shown in the output: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The presentation timestamp of the input frame, in time base units; the time base depends on the filter input pad, and is usually 1/<var class=”var”>sample_rate</var>. </dd> <dt><samp class=”option”>pts_time</samp></dt> <dd> The presentation timestamp of the input frame in seconds. </dd> <dt><samp class=”option”>fmt</samp></dt> <dd> The sample format. </dd> <dt><samp class=”option”>chlayout</samp></dt> <dd> The channel layout. </dd> <dt><samp class=”option”>rate</samp></dt> <dd> The sample rate for the audio frame. </dd> <dt><samp class=”option”>nb_samples</samp></dt> <dd> The number of samples (per channel) in the frame. </dd> <dt><samp class=”option”>checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of the audio data. For planar audio, the data is treated as if all the planes were concatenated. </dd> <dt><samp class=”option”>plane_checksums</samp></dt> <dd> A list of Adler-32 checksums for each data plane. </dd> </dl>

 <h3 class=”section”>8.55 asisdr# TOC</h3> Measure Audio Scaled-Invariant Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class=”section”>8.56 asoftclip<span

class=”pull-right”># TOC</h3> Apply audio soft clipping. Soft clipping is a type of distortion effect where the amplitude of a signal is saturated along a smooth curve, rather than the abrupt shape of hard-clipping. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>type</samp></dt> <dd> Set type of soft-clipping. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>hard</samp></dt> <dt><samp class=”option”>tanh</samp></dt> <dt><samp class=”option”>atan</samp></dt> <dt><samp class=”option”>cubic</samp></dt> <dt><samp class=”option”>exp</samp></dt> <dt><samp class=”option”>alg</samp></dt> <dt><samp class=”option”>quintic</samp></dt> <dt><samp class=”option”>sin</samp></dt> <dt><samp class=”option”>erf</samp></dt> </dl>

</dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold from where to start clipping. Default value is 0dB or 1. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set gain applied to output. Default value is 0dB or 1. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Set additional parameter which controls sigmoid function. </dd> <dt><samp class=”option”>oversample</samp></dt> <dd> Set oversampling factor. </dd> </dl>

 <h4 class=”subsection”>8.56.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>8.57 aspectralstats# TOC</h3>

Display frequency domain statistical information about the audio channels. Statistics are calculated and stored as metadata for each audio channel and for each audio frame. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>win_size</samp></dt> <dd> Set the window length in samples. Default value is 2048. Allowed range is from 32 to 65536. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann, hanning</samp>’</dt> <dt>‘<samp class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</

<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt>
<dd> Set window overlap. Allowed range is from <code class="code">0</code> to <code class="code">1</code>. Default value is <code class="code">0.5</code>. </dd> <dt><samp class="option">measure</samp></dt>
<dd> Select the parameters which are measured. The metadata keys can be used as flags, default is <samp class="option">all</samp> which measures everything. <samp class="option">none</samp> disables all measurement. </dd> </dl>

A list of each metadata key follows: <dl class="table"> <dt><samp class="option">mean</samp></dt> <dt><samp class="option">variance</samp></dt>
<dt><samp class="option">centroid</samp></dt> <dt><samp class="option">spread</samp></dt>
<dt><samp class="option">skewness</samp></dt> <dt><samp class="option">kurtosis</samp></dt>
<dt><samp class="option">entropy</samp></dt> <dt><samp class="option">flatness</samp></dt>
<dt><samp class="option">crest</samp></dt> <dt><samp class="option">flux</samp></dt>
<dt><samp class="option">slope</samp></dt> <dt><samp class="option">decrease</samp></dt>
<dt><samp class="option">rolloff</samp></dt> </dl>

 <h3 class="section">8.58 asr#
TOC</h3>
Automatic Speech Recognition This filter uses PocketSphinx for speech recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-pocketsphinx</code>. It accepts the following options: <dl class="table">
<dt><samp class="option">rate</samp></dt> <dd> Set sampling rate of input audio. Defaults is <code class="code">16000</code>. This need to match speech models, otherwise one will get poor results. </dd> <dt><samp class="option">hmm</samp></dt> <dd> Set dictionary containing acoustic model files. </dd> <dt><samp class="option">dict</samp></dt> <dd> Set pronunciation dictionary. </dd> <dt><samp class="option">lm</samp></dt> <dd> Set language model file. </dd> <dt><samp class="option">lmctl</samp></dt>
<dd> Set language model set. </dd> <dt><samp class="option">lmname</samp></dt>
<dd> Set which language model to use. </dd> <dt><samp class="option">logfn</samp></dt> <dd> Set output for log messages. </dd> </dl>

The filter exports recognized speech as the frame metadata <code class="code">lavfi.asr.text</code>. <h3 class="section">8.59 astats# TOC</h3>

Display time domain statistical information about the audio channels. Statistics are calculated and displayed for each audio channel and, where applicable, an overall figure is also given. It accepts the following option: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Short window length in seconds, used for peak and through RMS measurement. Default is <code class="code">0.05</code> (50 milliseconds). Allowed range is <code class="code">[0 - 10]</code>. </dd> <dt><samp class="option">metadata</samp></dt> <dd> Set metadata injection. All the metadata keys are prefixed with <code class="code">lavfi.astats.X</code>, where <code class="code">X</code> is channel number starting from 1 or string <code class="code">Overall</code>. Default is disabled. Available keys for each channel are: <var class="var">Bit_depth</var> <var class="var">Crest_factor</var> <var class="var">DC_offset</var> <var class="var">Dynamic_range</var> <var class="var">Entropy</var> <var class="var">Flat_factor</var> <var class="var">Max_difference</var> <var class="var">Max_level</var> <var class="var">Mean_difference</var> <var class="var">Min_difference</var> <var class="var">Min_level</var> <var class="var">Noise_floor</var> <var class="var">Noise_floor_count</var> <var class="var">Number_of_Infs</var> <var class="var">Number_of_NaNs</var> <var class="var">Number_of_denormals</var> <var class="var">Peak_count</var> <var class="var">Abs_Peak_count</var> <var class="var">Peak_level</var> <var class="var">RMS_difference</var> <var class="var">RMS_peak</var> <var class="var">RMS_trough</var> <var class="var">Zero_crossings</var> <var class="var">Zero_crossings_rate</var> and for <code class="code">Overall</code>: <var class="var">Bit_depth</var> <var class="var">DC_offset</var> <var class="var">Entropy</var> <var class="var">Flat_factor</var> <var class="var">Max_difference</var> <var class="var">Max_level</var> <var class="var">Mean_difference</var> <var class="var">Min_difference</var> <var class="var">Min_level</var> <var class="var">Noise_floor</var> <var class="var">Noise_floor_count</var> <var class="var">Number_of_Infs</var> <var class="var">Number_of_NaNs</var> <var class="var">Number_of_denormals</var> <var class="var">Number_of_samples</var> <var class="var">Peak_count</var> <var class="var">Abs_Peak_count</var> <var class="var">Peak_level</var> <var class="var">RMS_difference</var> <var class="var">RMS_level</var> <var class="var">RMS_peak</var> <var class="var">RMS_trough</var> For example, a full key looks like <code class="code">lavfi.astats.1.DC_offset</code> or <code class="code">lavfi.astats.Overall.Peak_count</code>. Read below for the description of the keys. </dd> <dt><samp

class=”option”>reset</samp></dt> <dd> Set the number of frames over which cumulative stats are calculated before being reset. Default is disabled. </dd> <dt><samp class=”option”>measure_perchannel</samp></dt> <dd> Select the parameters which are measured per channel. The metadata keys can be used as flags, default is <samp class=”option”>all</samp> which measures everything. <samp class=”option”>none</samp> disables all per channel measurement. </dd> <dt><samp class=”option”>measure_overall</samp></dt> <dd> Select the parameters which are measured overall. The metadata keys can be used as flags, default is <samp class=”option”>all</samp> which measures everything. <samp class=”option”>none</samp> disables all overall measurement. </dd> </dl>

A description of the measure keys follow:

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> no measures

</dd>

<dt><samp class=”option”>all</samp></dt>

<dd> all measures

</dd>

<dt><samp class=”option”>Bit_depth</samp></dt>

<dd> overall bit depth of audio, i.e. number of bits used for each sample

</dd>

<dt><samp class=”option”>Crest_factor</samp></dt>

<dd> standard ratio of peak to RMS level (note: not in dB)

</dd>

<dt><samp class=”option”>DC_offset</samp></dt>

<dd> mean amplitude displacement from zero

</dd>

<dt><samp class="option">Dynamic_range</samp></dt>

<dd> measured dynamic range of audio in dB

</dd>

<dt><samp class="option">Entropy</samp></dt>

<dd> entropy measured across whole audio, entropy of value near 1.0 is typically measured for white noise

</dd>

<dt><samp class="option">Flat_factor</samp></dt>

<dd> flatness (i.e. consecutive samples with the same value) of the signal at its peak levels

(i.e. either <var class="var">Min_level</var> or <var class="var">Max_level</var>)

</dd>

<dt><samp class="option">Max_difference</samp></dt>

<dd> maximal difference between two consecutive samples

</dd>

<dt><samp class="option">Max_level</samp></dt>

<dd> maximal sample level

</dd>

<dt><samp class="option">Mean_difference</samp></dt>

<dd> mean difference between two consecutive samples, i.e. the average of each difference between two consecutive samples

</dd>

<dt><samp class="option">Min_difference</samp></dt>

<dd> minimal difference between two consecutive samples

</dd>

<dt><samp class="option">Min_level</samp></dt>

<dd> minimal sample level

</dd>

<dt><samp class="option">Noise_floor</samp></dt>

<dd> minimum local peak measured in dBFS over a short window

</dd>

<dt><samp class="option">Noise_floor_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained

<var class="var">Noise floor</var>

</dd>

<dt><samp class="option">Number_of_Infs</samp></dt>

<dd> number of samples with an infinite value

</dd>

<dt><samp class="option">Number_of_NaNs</samp></dt>

<dd> number of samples with a NaN (not a number) value

</dd>

<dt><samp class="option">Number_of_denormals</samp></dt>

<dd> number of samples with a subnormal value

</dd>

<dt><samp class="option">Number_of_samples</samp></dt>

<dd> number of samples

</dd>

<dt><samp class="option">Peak_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained either

<var class="var">Min_level</var> or <var class="var">Max_level</var>

</dd>

<dt><samp class="option">Abs_Peak_count</samp></dt>

<dd> number of occasions that the absolute samples taken from the signal attained

max absolute value of <var class="var">Min_level</var> and <var class="var">Max_level</var>

</dd>

<dt><samp class="option">Peak_level</samp></dt>

<dd> standard peak level measured in dBFS

</dd>

<dt><samp class="option">RMS_difference</samp></dt>

<dd> Root Mean Square difference between two consecutive samples

</dd>

<dt><samp class="option">RMS_level</samp></dt>

<dd> standard RMS level measured in dBFS

</dd>

<dt><samp class="option">RMS_peak</samp></dt>

<dt><samp class="option">RMS_trough</samp></dt>

<dd> peak and through values for RMS level measured over a short window,

measured in dBFS.

</dd>

<dt><samp class="option">Zero crossings</samp></dt>

<dd> number of points where the waveform crosses the zero level axis

</dd>

<dt><samp class="option">Zero crossings rate</samp></dt>

<dd> rate of Zero crossings and number of audio samples

</dd>

</dl>

 <h3 class="section">8.60 asubboost# TOC</h3> Boost subwoofer frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dry</samp></dt> <dd> Set dry gain, how much of original signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet gain, how much of filtered signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class="option">boost</samp></dt> <dd> Set max boost factor. Allowed range is from 1 to 12. Default value is 2. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set delay line decay gain value. Allowed range is from 0 to 1. Default value is 0.0. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Set delay line feedback gain value. Allowed range is from 0 to 1. Default value is 0.9. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 50 to 900. Default value is 100. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope amount for cutoff frequency. Allowed range is 0.0001 to 1. Default value is 0.5. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay. Allowed range is from 1 to 100. Default value is 20. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the channels to process. Default value is all available. </dd> </dl>

 <h4 class="subsection">8.60.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.61 asubcut# TOC</h3> Cut subwoofer frequencies. This filter allows to set custom, steeper roll off than highpass filter, and thus is able to more attenuate frequency content in stop-band. The filter accepts the following options: <dl class="table">

<dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 2 to 200. Default value is 20. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd></dl>

 <h4 class="subsection">8.61.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.62 asupercut# TOC</h3> Cut super frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 20000 to 192000. Default value is 20000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">8.62.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.63 asuperpass# TOC</h3> Apply high order Butterworth band-pass filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">8.63.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.64 asuperstop# TOC</h3> Apply high order Butterworth band-stop filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">8.64.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.65 atempo# TOC</h3>

Adjust audio tempo. The filter accepts exactly one parameter, the audio tempo. If not specified then the filter will assume nominal 1.0 tempo. Tempo must be in the [0.5, 100.0] range. Note that tempo greater than 2 will skip some samples rather than blend them in. If for any reason this is a concern it is always possible to daisy-chain several instances of atempo to achieve the desired product tempo. <h4 class="subsection">8.65.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Slow down audio to 80% tempo: <pre class="example-preformatted">atempo=0.8 </pre>

To speed up audio to 300% tempo: <pre class="example-preformatted">atempo=3 </pre>

To speed up audio to 300% tempo by daisy-chaining two atempo instances: <pre class="example-preformatted">atempo=sqrt(3),atempo=sqrt(3)
</pre>

 <h4 class="subsection">8.65.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class="var">tempo</var>" </dd> </dl>

 <h3 class="section">8.66 atilt# TOC</h3> Apply spectral tilt filter to audio stream. This filter apply any spectral roll-off slope over any specified frequency band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> Set central frequency of tilt in Hz. Default is 10000 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope direction of tilt. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of tilt. Default is 1000. Allowed range is from 100 to 10000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set order of tilt filter. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input volume level. Allowed range is from 0 to 4. Default is 1. </dd> </dl>

 <h4 class="subsection">8.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.67 atrim# TOC</h3>

Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Timestamp (in seconds) of the start of the section to keep. I.e. the audio sample with the timestamp <var class="var">start</var> will be the first sample in the output. </dd> <dt><samp class="option">end</samp></dt> <dd> Specify time of the first audio sample that will be dropped,

i.e. the audio sample immediately preceding the one with the timestamp <var class="var">end</var> will be the last sample in the output. </dd> <dt><samp class="option">start_pts</samp></dt> <dd> Same as <var class="var">start</var>, except this option sets the start timestamp in samples instead of seconds. </dd> <dt><samp class="option">end_pts</samp></dt> <dd> Same as <var class="var">end</var>, except this option sets the end timestamp in samples instead of seconds. </dd> <dt><samp class="option">duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class="option">start_sample</samp></dt> <dd> The number of the first sample that should be output. </dd> <dt><samp class="option">end_sample</samp></dt> <dd> The number of the first sample that should be dropped. </dd> </dl>

<samp class="option">start</samp>, <samp class="option">end</samp>, and <samp class="option">duration</samp> are expressed as time duration specifications; see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. Note that the first two sets of the start/end options and the <samp class="option">duration</samp> option look at the frame timestamp, while the _sample options simply count the samples that pass through the filter. So start/end_pts and start/end_sample will give different results when the timestamps are wrong, inexact or do not start at zero. Also note that this filter does not modify the timestamps. If you wish to have the output timestamps start at zero, insert the asetpts filter after the atrim filter. If multiple start or end options are set, this filter tries to be greedy and keep all samples that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple atrim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class="itemize mark-bullet"> Drop everything except the second minute of input: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=60:120 </pre>

Keep only the first 1000 samples: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=end_sample=1000 </pre>

 <h3 class="section">8.68 axcorrelate# TOC</h3> Calculate normalized windowed cross-correlation between two input audio streams. Resulted samples are always between -1 and 1 inclusive. If result is 1 it means two input samples are highly correlated in that selected

segment. Result 0 means they are not correlated at all. If result is -1 it means two input samples are out of phase, which means they cancel each other. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size</samp></dt> <dd> Set size of segment over which cross-correlation is calculated. Default is 256. Allowed range is from 2 to 131072. </dd> <dt><samp class="option">algo</samp></dt> <dd> Set algorithm for cross-correlation. Can be <code class="code">slow</code> or <code class="code">fast</code> or <code class="code">best</code>. Default is <code class="code">best</code>. Fast algorithm assumes mean values over any given segment are always zero and thus need much less calculations to make. This is generally not true, but is valid for typical audio streams. </dd> </dl>

 <h4 class="subsection">8.68.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Calculate correlation between channels in stereo audio stream: <pre class="example-preformatted">ffmpeg -i stereo.wav -af channelsplit,axcorrelate=size=1024:algo=fast correlation.wav </pre>

 <h3 class="section">8.69 bandpass# TOC</h3>

Apply a two-pole Butterworth band-pass filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width width. The <var class="var">csg</var> option selects a constant skirt gain (peak gain = Q) instead of the default: constant 0dB peak gain. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">csg</samp></dt> <dd> Constant skirt gain if set to 1. Defaults to 0. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify

the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>8.69.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Change bandpass frequency. Syntax for the command is : "<var class=”var”>frequency</var>" </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Change bandpass width_type. Syntax for the command is : "<var class=”var”>width_type</var>" </dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Change bandpass width. Syntax for the command is : "<var class=”var”>width</var>" </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change bandpass mix. Syntax for the command is : "<var class=”var”>mix</var>" </dd> </dl>

 <h3 class="section">8.70 bandreject# TOC</h3>

Apply a two-pole Butterworth band-reject filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width <var class="var">width</var>. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero

values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.70.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change bandreject frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change bandreject width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bandreject width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandreject mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">8.71 bass, lowshelf# TOC</h3>

Boost or cut the bass (lower) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">100</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. De-

fault is 2. </dd> <dt><samp class="option">mix, m</samp></dt>
<dd> How much to use filtered signal in output. Default is 1. Range
is between 0 and 1. </dd> <dt><samp class="option">channels,
c</samp></dt> <dd> Specify which channels to filter, by default all
available are filtered. </dd> <dt><samp class="option">normalize,
n</samp></dt> <dd> Normalize biquad coefficients, by default is
disabled. Enabling it will normalize magnitude response at DC to 0dB.
 </dd> <dt><samp class="option">transform, a</samp></dt>
<dd> Set transform type of IIR filter. <dl class="table">
<dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt>
<dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt>
<dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt>
<dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set
precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt>
<dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough
value (higher than impulse response length truncated when reaches near zero
values) filtering will become linear phase otherwise if not big enough it will just
produce nasty artifacts. Note that filter delay will be exactly this
many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.71.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change
bass frequency. Syntax for the command is : "<var class="var">frequency</var>"
 </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bass width_type. Syntax for the command is : "<var
class="var">width_type</var>" </dd> <dt><samp class="option">width,
w</samp></dt> <dd> Change bass width. Syntax for the command is
: "<var class="var">width</var>" </dd> <dt><samp
class="option">gain, g</samp></dt> <dd> Change bass gain. Syntax for the command is : "<var class="var">gain</var>"
</dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change
bass mix. Syntax for the command is : "<var class="var">mix</var>"
</dd> </dl>

 <h3 class="section">8.72 biquad# TOC</h3>

Apply a biquad IIR filter with the given coefficients. Where <var class="var">b0</var>, <var class="var">b1</var>, <var class="var">b2</var> and <var class="var">a0</var>, <var class="var">a1</var>, <var class="var">a2</var> are the numerator and denominator coefficients respectively. and <var class="var">channels</var>, <var class="var">c</var> specify which channels to filter, by default all available are filtered. <h4 class="subsection">8.72.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">a0</samp></dt> <dt><samp class="option">a1</samp></dt> <dt><samp class="option">a2</samp></dt> <dt><samp class="option">b0</samp></dt> <dt><samp class="option">b1</samp></dt> <dt><samp class="option">b2</samp></dt> <dd> Change biquad parameter. Syntax for the command is : "<var class="var">value</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just

produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h3 class="section">8.73 bs2b# TOC</h3> Bauer stereo to binaural transformation, which improves headphone listening of stereo audio records. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libbs2b</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Pre-defined crossfeed level. <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Default level (fcut=700, feed=50). </dd> <dt><samp class="option">cmoy</samp></dt> <dd> Chu Moy circuit (fcut=700, feed=60). </dd> <dt><samp class="option">jmeier</samp></dt> <dd> Jan Meier circuit (fcut=650, feed=95). </dd> </dl>

</dd> <dt><samp class="option">fcut</samp></dt> <dd> Cut frequency (in Hz). </dd> <dt><samp class="option">feed</samp></dt> <dd> Feed level (in Hz). </dd> </dl>

 <h3 class="section">8.74 channelmap# TOC</h3>

Remap input channels to new locations. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Map channels from input to output. The argument is a ’|’-separated list of mappings, each in the <code class="code"><var class="var">in_channel</var>-<var class="var">out_channel</var></code> or <code class="code"><var class="var">in_channel</var></code> form. <var class="var">in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the input channel layout. <var class="var">out_channel</var> is the name of the output channel or its index in the output channel layout. If <var class="var">out_channel</var> is not given then it is implicitly an index, starting with zero and increasing by one for each mapping. Mixing different types of mappings is not allowed and will result in a parse error. </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the output stream. If not specified, then filter will guess it based on the <var class="var">out_channel</var> names or the number of mappings. Guessed layouts will not necessarily contain channels in the order of the mappings. </dd> </dl>

If no mapping is present, the filter will implicitly map input channels to output channels, preserving indices.

<h4 class="subsection">8.74.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a 5.1+downmix input MOV file, <pre class="example-preformatted">ffmpeg -i in.mov -filter 'channelmap=map=DL-FL|DR-FR' out.wav </pre> will create an output WAV file tagged as stereo from the downmix channels of the input. To fix a 5.1 WAV improperly encoded in AAC’s native channel order <pre class="example-preformatted">ffmpeg -i in.wav -filter 'channelmap=1|2|0|5|3|4:5.1' out.wav </pre>

 <h3 class="section">8.75 channelsplit# TOC</h3>

Split each channel from an input audio stream into a separate output stream. It accepts the following parameters: <dl class="table"> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the input stream. The default is "stereo". </dd> <dt><samp class="option">channels</samp></dt> <dd> A channel layout describing the channels to be extracted as separate output streams or "all" to extract each input channel as a separate stream. The default is "all". Choosing channels not present in channel layout in the input will result in an error. </dd> </dl>

 <h4 class="subsection">8.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a stereo input MP3 file, <pre class="example-preformatted">ffmpeg -i in.mp3 -filter_complex channelsplit out.mkv </pre> will create an output Matroska file with two audio streams, one containing only the left channel and the other the right channel. Split a 5.1 WAV file into per-channel files: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1[FL][FR][FC][LFE][SL][SR]' -map '[FL]' front_left.wav -map '[FR]' front_right.wav -map '[FC]' front_center.wav -map '[LFE]' lfe.wav -map '[SL]' side_left.wav -map '[SR]' side_right.wav </pre>

Extract only LFE from a 5.1 WAV file: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1:channels=LFE[LFE]' -map '[LFE]' lfe.wav </pre>

 <h3 class="section">8.76 chorus# TOC</h3> Add a chorus effect to the audio. Can make a single vocal sound like a chorus, but can also be applied to instrumentation. Chorus resembles an echo effect with a short delay, but whereas with echo the delay is constant, with chorus, it is varied using sinusoidal or triangular modulation. The modulation depth defines the range the modulated delay is played before or after the delay. Hence the delayed sound will sound slower or faster, that is the delayed sound tuned around the original one, like in a chorus where some vocals are slightly off key. It accepts the following parameters: <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.4. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set delays. A typical delay is around 40ms to 60ms. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set decays. </dd> <dt><samp class="option">speeds</samp></dt> <dd> Set speeds. </dd> <dt><samp class="option">depths</samp></dt> <dd> Set depths. </dd> </dl>

 <h4 class="subsection">8.76.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A single delay: <pre class="example-preformatted">chorus=0.7:0.9:55:0.4:0.25:2 </pre>

Two delays: <pre class="example-preformatted">chorus=0.6:0.9:50|60:0.4|0.32:0.25|0.4:2|1.3 </pre>

Fuller sounding chorus with three delays: <pre class="example-preformatted">chorus=0.5:0.9:50|60|40:0.4|0.32|0.3:0.25|0.4|0.3:2|2.3|1.3 </pre>

 <h3 class="section">8.77 compand# TOC</h3> Compress or expand the audio’s dynamic range. It accepts the following parameters: <dl class="table"> <dt><samp class="option">attacks</samp></dt> <dt><samp class="option">decays</samp></dt> <dd> A list of times in seconds for each channel over which the instantaneous level of the input signal is averaged to determine its volume. <var class="var">attacks</var> refers to increase of volume and <var class="var">decays</var> refers to decrease of volume. For most situations, the attack time (response to the audio getting louder) should be shorter than the decay time, because the human ear is more

sensitive to sudden loud audio than sudden soft audio. A typical value for attack is 0.3 seconds and a typical value for decay is 0.8 seconds. If specified number of attacks & decays is lower than number of channels, the last set attack/decay will be used for all remaining channels. </dd> <dt><samp class="option">points</samp></dt> <dd> A list of points for the transfer function, specified in dB relative to the maximum possible signal amplitude. Each key points list must be defined using the following syntax: <code class="code">x0/y0|x1/y1|x2/y2|....</code> or <code class="code">x0/y0 x1/y1 x2/y2 ....</code> The input values must be in strictly increasing order but the transfer function does not have to be monotonically rising. The point <code class="code">0/0</code> is assumed but may be overridden (by <code class="code">0/out-dBn</code>). Typical values for the transfer function are <code class="code">-70/-70|-60/-20|1/0</code>. </dd> <dt><samp class="option">soft-knee</samp></dt> <dd> Set the curve radius in dB for all joints. It defaults to 0.01. </dd> <dt><samp class="option">gain</samp></dt> <dd> Set the additional gain in dB to be applied at all points on the transfer function. This allows for easy adjustment of the overall gain. It defaults to 0. </dd> <dt><samp class="option">volume</samp></dt> <dd> Set an initial volume, in dB, to be assumed for each channel when filtering starts. This permits the user to supply a nominal level initially, so that, for example, a very large gain is not applied to initial signal levels before the companding has begun to operate. A typical value for audio which is initially quiet is -90 dB. It defaults to 0. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set a delay, in seconds. The input audio is analyzed immediately, but audio is delayed before being fed to the volume adjuster. Specifying a delay approximately equal to the attack/decay times allows the filter to effectively operate in predictive rather than reactive mode. It defaults to 0. </dd> </dl>

 <h4 class="subsection">8.77.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make music with both quiet and loud passages suitable for listening to in a noisy environment: <pre class="example-preformatted">compand=.3|.3:1|1:-90/-60|-60/-40|-40/-30|-20/-20:6:0:-90:0.2 </pre>

Another example for audio with whisper and explosion parts: <pre class="example-preformatted">compand=0|0:1|1:-90/-900|-70/-70|-30/-9|0/-3:6:0:0:0 </pre>

A noise gate for when the noise is at a lower level than the signal: <pre class="example-preformatted">compand=.1|.1:.2|.2:-900/-900|-50.1/-900|-50/-50:.01:0:-90:.1 </pre>

Here is another noise gate, this time for when the noise is at a higher level than the signal (making it, in some ways, similar to squelch): <pre class=”example-preformatted”>compand=.1|.1:.1|.1:-45.1/-45.1|-45/-900|0/-900:.01:45:-90:.1 </pre>

2:1 compression starting at -6dB: <pre class=”example-preformatted”>compand=points=-80/-80|-6/-6|0/-3.8|20/3.5 </pre>

2:1 compression starting at -9dB: <pre class=”example-preformatted”>compand=points=-80/-80|-9/-9|0/-5.3|20/2.9 </pre>

2:1 compression starting at -12dB: <pre class=”example-preformatted”>compand=points=-80/-80|-12/-12|0/-6.8|20/1.9 </pre>

2:1 compression starting at -18dB: <pre class=”example-preformatted”>compand=points=-80/-80|-18/-18|0/-9.8|20/0.7 </pre>

3:1 compression starting at -15dB: <pre class=”example-preformatted”>compand=points=-80/-80|-15/-15|0/-10.8|20/-5.2 </pre>

Compressor/Gate: <pre class=”example-preformatted”>compand=points=-80/-105|-62/-80|-15.4/-15.4|0/-12|20/-7.6 </pre>

Expander: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-169|-54/-80|-49.5/-64.6|-41.1/-41.1|-25.8/-15|-10.8/-4.5|0/0|20/8.3 </pre>

Hard limiter at -6dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-6/-6|20/-6 </pre>

Hard limiter at -12dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12/-12|20/-12 </pre>

Hard noise gate at -35 dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-115|-35.1/-80|-35/-35|20/20 </pre>

Soft limiter: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12.4/-12.4|-6/-8|0/-6.8|20/-2.8 </pre>

 <h3 class=”section”>8.78 compensationdelay# TOC</h3>

Compensation Delay Line is a metric based delay to compensate differing positions of microphones or speakers. For example, you have recorded guitar with two microphones placed in different locations. Because the front of sound wave has fixed speed in normal conditions, the phasing of microphones can vary and depends on their location and interposition. The best sound mix can be achieved when these microphones are in phase (synchronized). Note that a distance of ~30 cm between microphones makes one microphone capture the signal in antiphase to the other microphone. That makes the final mix sound moody. This filter helps to solve phasing problems by adding different delays to each microphone track and make them synchronized. The best result can be reached when you take one track as base and synchronize other tracks one by one with it. Remember that synchronization/delay tolerance depends on sample rate, too. Higher sample rates will give more tolerance. The filter accepts the following parameters: <dl class="table"> <dt><samp class="option">mm</samp></dt> <dd> Set millimeters distance. This is compensation distance for fine tuning. Default is 0. </dd> <dt><samp class="option">cm</samp></dt> <dd> Set cm distance. This is compensation distance for tightening distance setup. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set meters distance. This is compensation distance for hard distance setup. Default is 0. </dd> <dt><samp class="option">dry</samp></dt> <dd> Set dry amount. Amount of unprocessed (dry) signal. Default is 0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet amount. Amount of processed (wet) signal. Default is 1. </dd> <dt><samp class="option">temp</samp></dt> <dd> Set temperature in degrees Celsius. This is the temperature of the environment. Default is 20. </dd> </dl>

 <h4 class="subsection">8.78.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.79 crossfeed# TOC</h3> Apply headphone crossfeed filter. Crossfeed is the process of blending the left and right channels of stereo audio recording. It is mainly used to reduce extreme stereo separation of low frequencies. The intent is to produce more speaker like sound to the listener. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Set strength of crossfeed. Default is 0.2. Allowed range is from 0 to 1. This sets gain of low shelf filter for side part of stereo image. Default is -6dB. Max allowed is -30db when strength

is set to 1. </dd> <dt><samp class="option">range</samp></dt> <dd> Set soundstage wideness. Default is 0.5. Allowed range is from 0 to 1. This sets cut off frequency of low shelf filter. Default is cut off near 1550 Hz. With range set to 1 cut off frequency is set to 2100 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set curve slope of low shelf filter. Default is 0.5. Allowed range is from 0.01 to 1. </dd> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 0.9. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class="option">block_size</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.79.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.80 crystalizer# TOC</h3> Simple algorithm for audio noise sharpening. This filter linearly increases differences between each audio sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Sets the intensity of effect (default: 2.0). Must be in range between -10.0 to 0 (unchanged sound) to 10.0 (maximum effect). To inverse filtering use negative value. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">8.80.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.81 dcshift# TOC</h3> Apply a DC shift to the audio. This can be useful to remove a DC offset (caused perhaps by a hardware problem in the recording chain) from the audio.

The effect of a DC offset is reduced headroom and hence volume. The astats filter can be used to determine if a signal has a DC offset. <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Set the DC shift, allowed range is [-1, 1]. It indicates the amount to shift the audio. </dd> <dt><samp class="option">limitergain</samp></dt> <dd> Optional. It should have a value much less than 1 (e.g. 0.05 or 0.02) and is used to prevent clipping. </dd> </dl>

 <h3 class="section">8.82 deesser# TOC</h3>

Apply de-essing to the audio samples. <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Set intensity for triggering de-essing. Allowed range is from 0 to 1. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set amount of ducking on treble part of sound. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">f</samp></dt> <dd> How much of original frequency content to keep when de-essing. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">s</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass ess filtered out. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass only ess. Default value is <var class="var">o</var>. </dd> </dl>

</dd> </dl>

 <h3 class="section">8.83 dialoguenhance# TOC</h3> Enhance dialogue in stereo audio. This filter accepts stereo input and produce surround (3.0) channels output. The newly produced front center channel have enhanced speech dialogue originally available in both stereo channels. This filter outputs front left and front right channels same as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">original</samp></dt> <dd> Set the original center factor to keep in front center channel output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">enhance</samp></dt> <dd> Set the dialogue enhance factor to put in front center channel output. Allowed range is from 0 to 3. Default value is 1. </dd> <dt><samp class="option">voice</samp></dt> <dd> Set the voice detection

factor. Allowed range is from 2 to 32. Default value is 2. </dd> </dl>

 <h4 class="subsection">8.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.84 drmeter# TOC</h3> Measure audio dynamic range. DR values of 14 and higher is found in very dynamic material. DR of 8 to 13 is found in transition material. And anything less that 8 have very poor dynamics and is very compressed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Set window length in seconds used to split audio into segments of equal length. Default is 3 seconds. </dd> </dl>

<h3 class="section">8.85 dynaudnorm# TOC</h3>

Dynamic Audio Normalizer.

This filter applies a certain amount of gain to the input audio in order

to bring its peak magnitude to a target level (e.g. 0 dBFS). However, in

contrast to more "simple" normalization algorithms, the Dynamic Audio

Normalizer *dynamically* re-adjusts the gain factor to the input audio.

This allows for applying extra gain to the "quiet" sections of the audio

while avoiding distortions or clipping the "loud" sections. In other words:

The Dynamic Audio Normalizer will "even out" the volume of quiet and loud

sections, in the sense that the volume of each section is brought to the

same target level. Note, however, that the Dynamic Audio Normalizer achieves

this goal *without* applying "dynamic range compressing". It will retain 100%

of the dynamic range *within* each section of the audio file.

<dl class="table">

<dt><samp class="option">framelen, f</samp></dt>

<dd> Set the frame length in milliseconds. In range from 10 to 8000 milliseconds.

Default is 500 milliseconds.

The Dynamic Audio Normalizer processes the input audio in small chunks,

referred to as frames. This is required, because a peak magnitude has no

meaning for just a single sample value. Instead, we need to determine the

peak magnitude for a contiguous sequence of sample values. While a "standard"

normalizer would simply use the peak magnitude of the complete file, the

Dynamic Audio Normalizer determines the peak magnitude individually for each

frame. The length of a frame is specified in milliseconds. By default, the

Dynamic Audio Normalizer uses a frame length of 500 milliseconds, which has

been found to give good results with most files.

Note that the exact frame length, in number of samples, will be determined

automatically, based on the sampling rate of the individual input audio file.

</dd>

<dt><samp class="option">gausssize, g</samp></dt>

<dd> Set the Gaussian filter window size. In range from 3 to 301, must be odd

number. Default is 31.

Probably the most important parameter of the Dynamic Audio Normalizer is the

<code class="code">window size</code> of the Gaussian smoothing filter. The filter’s window size

is specified in frames, centered around the current frame. For the sake of

simplicity, this must be an odd number. Consequently, the default value of 31

takes into account the current frame, as well as the 15 preceding frames and

the 15 subsequent frames. Using a larger window results in a stronger

smoothing effect and thus in less gain variation, i.e. slower gain

adaptation. Conversely, using a smaller window results in a weaker smoothing

effect and thus in more gain variation, i.e. faster gain adaptation.

In other words, the more you increase this value, the more the Dynamic Audio

Normalizer will behave like a "traditional" normalization filter. On the

contrary, the more you decrease this value, the more the Dynamic Audio

Normalizer will behave like a dynamic range compressor.

</dd>

<dt><samp class="option">peak, p</samp></dt>

<dd> Set the target peak value. This specifies the highest permissible magnitude

level for the normalized audio input. This filter will try to approach the

target peak magnitude as closely as possible, but at the same time it also

makes sure that the normalized signal will never exceed the peak magnitude.

A frame’s maximum local gain factor is imposed directly by the target peak

magnitude. The default value is 0.95 and thus leaves a headroom of 5%*.

It is not recommended to go above this value.

</dd>

<dt><samp class="option">maxgain, m</samp></dt>

<dd> Set the maximum gain factor. In range from 1.0 to 100.0. Default is 10.0.

The Dynamic Audio Normalizer determines the maximum possible (local) gain

factor for each input frame, i.e. the maximum gain factor that does not

result in clipping or distortion. The maximum gain factor is determined by

the frame’s highest magnitude sample. However, the Dynamic Audio Normalizer

additionally bounds the frame’s maximum gain factor by a predetermined

(global) maximum gain factor. This is done in order to avoid excessive gain

factors in "silent" or almost silent frames. By default, the maximum gain

factor is 10.0, For most inputs the default value should be sufficient and

it usually is not recommended to increase this value. Though, for input

with an extremely low overall volume level, it may be necessary to allow even

higher gain factors. Note, however, that the Dynamic Audio Normalizer does

not simply apply a "hard" threshold (i.e. cut off values above the threshold).

Instead, a "sigmoid" threshold function will be applied. This way, the

gain factors will smoothly approach the threshold value, but never exceed that

value.

</dd>

<dt><samp class="option">targetrms, r</samp></dt>

<dd> Set the target RMS. In range from 0.0 to 1.0. Default is 0.0 - disabled.

By default, the Dynamic Audio Normalizer performs "peak" normalization.

This means that the maximum local gain factor for each frame is defined

(only) by the frame’s highest magnitude sample. This way, the samples can

be amplified as much as possible without exceeding the maximum signal

level, i.e. without clipping. Optionally, however, the Dynamic Audio

Normalizer can also take into account the frame’s root mean square,

abbreviated RMS. In electrical engineering, the RMS is commonly used to

determine the power of a time-varying signal. It is therefore considered

that the RMS is a better approximation of the "perceived loudness" than

just looking at the signal’s peak magnitude. Consequently, by adjusting all

frames to a constant RMS value, a uniform "perceived loudness" can be

established. If a target RMS value has been specified, a frame’s local gain

factor is defined as the factor that would result in exactly that RMS value.

Note, however, that the maximum local gain factor is still restricted by the

frame’s highest magnitude sample, in order to prevent clipping.

</dd>

<dt><samp class="option">coupling, n</samp></dt>

<dd> Enable channels coupling. By default is enabled.

By default, the Dynamic Audio Normalizer will amplify all channels by the same

amount. This means the same gain factor will be applied to all channels, i.e.

the maximum possible gain factor is determined by the "loudest" channel.

However, in some recordings, it may happen that the volume of the different

channels is uneven, e.g. one channel may be "quieter" than the other one(s).

In this case, this option can be used to disable the channel coupling. This way,

the gain factor will be determined independently for each channel, depending

only on the individual channel’s highest magnitude sample. This allows for

harmonizing the volume of the different channels.

</dd>

<dt><samp class="option">correctdc, c</samp></dt>

<dd> Enable DC bias correction. By default is disabled.

An audio signal (in the time domain) is a sequence of sample values.

In the Dynamic Audio Normalizer these sample values are represented in the

-1.0 to 1.0 range, regardless of the original input format. Normally, the

audio signal, or "waveform", should be centered around the zero point.

That means if we calculate the mean value of all samples in a file, or in a

single frame, then the result should be 0.0 or at least very close to that

value. If, however, there is a significant deviation of the mean value from

0.0, in either positive or negative direction, this is referred to as a

DC bias or DC offset. Since a DC bias is clearly undesirable, the Dynamic

Audio Normalizer provides optional DC bias correction.

With DC bias correction enabled, the Dynamic Audio Normalizer will determine

the mean value, or "DC correction" offset, of each input frame and subtract

that value from all of the frame’s sample values which ensures those samples

are centered around 0.0 again. Also, in order to avoid "gaps" at the frame

boundaries, the DC correction offset values will be interpolated smoothly

between neighbouring frames.

</dd>

<dt><samp class="option">altboundary, b</samp></dt>

<dd> Enable alternative boundary mode. By default is disabled.

The Dynamic Audio Normalizer takes into account a certain neighbourhood

around each frame. This includes the preceding frames as well as the

subsequent frames. However, for the "boundary" frames, located at the very

beginning and at the very end of the audio file, not all neighbouring

frames are available. In particular, for the first few frames in the audio

file, the preceding frames are not known. And, similarly, for the last few

frames in the audio file, the subsequent frames are not known. Thus, the

question arises which gain factors should be assumed for the missing frames

in the "boundary" region. The Dynamic Audio Normalizer implements two modes

to deal with this situation. The default boundary mode assumes a gain factor

of exactly 1.0 for the missing frames, resulting in a smooth "fade in" and

"fade out" at the beginning and at the end of the input, respectively.

</dd>
<dt><samp class=”option”>compress, s</samp></dt>
<dd> Set the compress factor. In range from 0.0 to 30.0. Default is 0.0.
By default, the Dynamic Audio Normalizer does not apply "traditional"
compression. This means that signal peaks will not be pruned and thus the
full dynamic range will be retained within each local neighbourhood. However,
in some cases it may be desirable to combine the Dynamic Audio Normalizer’s
normalization algorithm with a more "traditional" compression.
For this purpose, the Dynamic Audio Normalizer provides an optional compression
(thresholding) function. If (and only if) the compression feature is enabled,
all input frames will be processed by a soft knee thresholding function prior
to the actual normalization process. Put simply, the thresholding function is
going to prune all samples whose magnitude exceeds a certain threshold value.
However, the Dynamic Audio Normalizer does not simply apply a fixed threshold
value. Instead, the threshold value will be adjusted for each individual
frame.
In general, smaller parameters result in stronger compression, and vice versa.
Values below 3.0 are not recommended, because audible distortion may appear.

</dd>
<dt><samp class=”option”>threshold, t</samp></dt>
<dd> Set the target threshold value. This specifies the lowest permissible
magnitude level for the audio input which will be normalized.
If input frame volume is above this value frame will be normalized.
Otherwise frame may not be normalized at all. The default value is set
to 0, which means all input frames will be normalized.
This option is mostly useful if digital noise is not wanted to be amplified.

</dd>

<dt><samp class=”option”>channels, h</samp></dt>

<dd> Specify which channels to filter, by default all available channels are filtered.

</dd>

<dt><samp class=”option”>overlap, o</samp></dt>

<dd> Specify overlap for frames. If set to 0 (default) no frame overlapping is done.

Using >0 and <1 values will make less conservative gain adjustments, like

when framelen option is set to smaller value, if framelen option value is

compensated for non-zero overlap then gain adjustments will be smoother across time

compared to zero overlap case.

</dd>

<dt><samp class=”option”>curve, v</samp></dt>

<dd> Specify the peak mapping curve expression which is going to be used when calculating

gain applied to frames. The max output frame gain will still be limited by other

options mentioned previously for this filter.

The expression can contain the following constants:

<dl class=”table”>

<dt><samp class=”option”>ch</samp></dt>

<dd> current channel number

</dd>

<dt><samp class=”option”>sn</samp></dt>

<dd> current sample number

</dd>

<dt><samp class="option">nb_channels</samp></dt>

<dd> number of channels

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds

</dd>

<dt><samp class="option">sr</samp></dt>

<dd> sample rate

</dd>

<dt><samp class="option">p</samp></dt>

<dd> current frame peak value

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">8.85.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">8.86 earwax# TOC</h3>

Make audio easier to listen to on headphones. This filter adds ‘cues’ to 44.1kHz stereo (i.e. audio CD format) audio so that when listened to on headphones the stereo image is moved from inside your head (standard for headphones) to outside and in front of the listener (standard for speakers). Ported from SoX. <h3 class="section">8.87 equalizer# TOC</h3>

Apply a two-pole peaking equalisation (EQ) filter. With this filter, the signal-level at and around a selected frequency can be increased or decreased, whilst (unlike bandpass and bandreject filters) that at all other frequencies is unchanged. In order to produce complex equalisation curves, this filter can be given several times, each with a different central frequency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency in Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Set the required gain or attenuation in dB. Beware of clipping when using a positive gain. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just

produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.87.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Attenuate 10 dB at 1000 Hz, with a bandwidth of 200 Hz: <pre class="example-preformatted">equalizer=f=1000:t=h:width=200:g=-10 </pre>

Apply 2 dB gain at 1000 Hz with Q 1 and attenuate 5 dB at 100 Hz with Q 2: <pre class="example-preformatted">equalizer=f=1000:t=q:w=1:g=2,equalizer=f=100:t=q:w=2:g=-5 </pre>

 <h4 class="subsection">8.87.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change equalizer frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change equalizer width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change equalizer width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change equalizer gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change equalizer mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">8.88 extrastereo# TOC</h3>

Linearly increases the difference between left and right channels which adds some sort of "live" effect to playback. The filter accepts the following options: <dl class="table"> <dt><samp class="option">m</samp></dt> <dd> Sets the difference coefficient (default: 2.5). 0.0 means mono sound (average of both channels), with 1.0 sound will be unchanged, with -1.0 left and right channels will be swapped. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">8.88.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.89 firequalizer# TOC</h3> Apply FIR Equalization using arbitrary frequency response. The filter accepts the following option: <dl class="table"> <dt><samp class="option">gain</samp></dt> <dd> Set gain curve equation (in dB). The expression can contain variables: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> the evaluated frequency </dd> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">ch</samp></dt> <dd> channel number, set to 0 when multichannels evaluation is disabled </dd> <dt><samp class="option">chid</samp></dt> <dd> channel id, see libavutil/channel_layout.h, set to the first channel id when multichannels evaluation is disabled </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd> <dt><samp class="option">chlayout</samp></dt> <dd> channel_layout, see libavutil/channel_layout.h </dd> </dl> and functions: <dl class="table"> <dt><samp class="option">gain_interpolate(f)</samp></dt> <dd> interpolate gain on frequency f based on gain_entry </dd> <dt><samp class="option">cubic_interpolate(f)</samp></dt> <dd> same as gain_interpolate, but smoother </dd> </dl> This option is also available as command. Default is <code class="code">gain_interpolate(f)</code>. </dd> <dt><samp class="option">gain_entry</samp></dt> <dd> Set gain entry for gain_interpolate function. The expression can contain functions: <dl class="table"> <dt><samp class="option">entry(f, g)</samp></dt> <dd> store gain entry at frequency f with value g </dd> </dl> This option is also available as command. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set filter delay in seconds. Higher value means more accurate. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set filter accuracy in Hz. Lower value means more accurate. Default is <code class="code">5</code>. </dd> <dt><samp class="option">wfunc</samp></dt> <dd> Set window function. Acceptable values are: <dl class="table"> <dt><samp class="option">rectangular</samp></dt> <dd> rectangular window, useful when gain curve is already smooth </dd> <dt><samp class="option">hann</samp></dt> <dd> hann window (default) </dd> <dt><samp class="option">hamming</samp></dt> <dd> hamming window </dd> <dt><samp class="option">blackman</samp></dt>

<dd> blackman window </dd> <dt><samp class="option">nuttall3</samp></dt>
<dd> 3-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">mnuttall3</samp></dt> <dd> minimum
3-terms discontinuous nuttall window </dd> <dt><samp class="option">nuttall</samp></dt>
<dd> 4-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">bnuttall</samp></dt> <dd> minimum
4-terms discontinuous nuttall (blackman-nuttall) window </dd>
<dt><samp class="option">bharris</samp></dt> <dd> blackman-
harris window </dd> <dt><samp class="option">tukey</samp></dt>
<dd> tukey window </dd> </dl>

</dd> <dt><samp class="option">fixed</samp></dt> <dd> If enabled, use fixed number of audio samples. This improves speed when filtering with large delay. Default is disabled. </dd> <dt><samp class="option">multi</samp></dt> <dd> Enable multichannels evaluation on gain. Default is disabled. </dd> <dt><samp class="option">zero_phase</samp></dt> <dd> Enable zero phase mode by subtracting timestamp to compensate delay. Default is disabled. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set scale used by gain. Acceptable values are: <dl class="table"> <dt><samp class="option">linlin</samp></dt> <dd> linear frequency, linear gain </dd> <dt><samp class="option">linlog</samp></dt> <dd> linear frequency, logarithmic (in dB) gain (default) </dd> <dt><samp class="option">loglin</samp></dt> <dd> logarithmic (in octave scale where 20 Hz is 0) frequency, linear gain </dd> <dt><samp class="option">loglog</samp></dt> <dd> logarithmic frequency, logarithmic gain </dd> </dl>

</dd> <dt><samp class="option">dumpfile</samp></dt> <dd> Set file for dumping, suitable for gnuplot. </dd> <dt><samp class="option">dumpscale</samp></dt> <dd> Set scale for dumpfile. Acceptable values are same with scale option. Default is linlog. </dd> <dt><samp class="option">fft2</samp></dt> <dd> Enable 2-channel convolution using complex FFT. This improves speed significantly. Default is disabled. </dd> <dt><samp class="option">min_phase</samp></dt> <dd> Enable minimum phase impulse response. Default is disabled. </dd> </dl>

 <h4 class="subsection">8.89.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> lowpass at 1000 Hz: <pre class="example-preformatted">firequalizer=gain='if(lt(f,1000), 0, -INF)' </pre> lowpass at 1000 Hz with gain_entry: <pre class="example-preformatted">firequalizer=gain_entry='entry(1000,0); entry(1001, -INF)' </pre> custom equalization: <div

class=”example”> <pre class=”example-preformatted”>firequalizer=gain_entry='entry(100,0); entry(400, -4); entry(1000, -6); entry(2000, 0)' </pre> higher delay with zero phase to compensate delay: <pre class=”example-preformatted”>firequalizer=delay=0.1:fixed=on:zero_phase=on </pre> lowpass on left channel, highpass on right channel: <pre class=”example-preformatted”>firequalizer=gain='if(eq(chid,1), gain_interpolate(f), if(eq(chid,2), gain_interpolate(1e6+f), 0))' :gain_entry='entry(1000, 0); entry(1001,-INF); entry(1e6+1000,0)':multi=on </pre>

 <h3 class=”section”>8.90 flanger# TOC</h3> Apply a flanging effect to the audio. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Set base delay in milliseconds. Range from 0 to 30. Default value is 0. </dd> <dt><samp class=”option”>depth</samp></dt> <dd> Set added sweep delay in milliseconds. Range from 0 to 10. Default value is 2. </dd> <dt><samp class=”option”>regen</samp></dt> <dd> Set percentage regeneration (delayed signal feedback). Range from -95 to 95. Default value is 0. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set percentage of delayed signal mixed with original. Range from 0 to 100. Default value is 71. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set sweeps per second (Hz). Range from 0.1 to 10. Default value is 0.5. </dd> <dt><samp class=”option”>shape</samp></dt> <dd> Set swept wave shape, can be <var class=”var”>triangular</var> or <var class=”var”>sinusoidal</var>. Default value is <var class=”var”>sinusoidal</var>. </dd> <dt><samp class=”option”>phase</samp></dt> <dd> Set swept wave percentage-shift for multi channel. Range from 0 to 100. Default value is 25. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Set delay-line interpolation, <var class=”var”>linear</var> or <var class=”var”>quadratic</var>. Default is <var class=”var”>linear</var>. </dd> </dl>

 <h3 class=”section”>8.91 haas# TOC</h3> Apply Haas effect to audio. Note that this makes most sense to apply on mono signals. With this filter applied to mono signals it give some directionality and stretches its stereo image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level. By default is <var class=”var”>1</var>, or 0dB </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level. By default is <var class=”var”>1</var>, or 0dB. </dd> <dt><samp class=”option”>side_gain</samp></dt> <dd> Set gain applied to side part of signal. By default is <var class=”var”>1</var>. </dd>

<dt><samp class="option">middle_source</samp></dt> <dd> Set kind of middle source. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">left</samp>’</dt> <dd> Pick left channel. </dd> <dt>‘<samp class="samp">right</samp>’</dt> <dd> Pick right channel. </dd> <dt>‘<samp class="samp">mid</samp>’</dt> <dd> Pick middle part signal of stereo image. </dd> <dt>‘<samp class="samp">side</samp>’</dt> <dd> Pick side part signal of stereo image. </dd> </dl>

</dd> <dt><samp class="option">middle_phase</samp></dt> <dd> Change middle phase. By default is disabled. </dd> <dt><samp class="option">left_delay</samp></dt> <dd> Set left channel delay. By default is <var class="var">2.05</var> milliseconds. </dd> <dt><samp class="option">left_balance</samp></dt> <dd> Set left channel balance. By default is <var class="var">-1</var>. </dd> <dt><samp class="option">left_gain</samp></dt> <dd> Set left channel gain. By default is <var class="var">1</var>. </dd> <dt><samp class="option">left_phase</samp></dt> <dd> Change left phase. By default is disabled. </dd> <dt><samp class="option">right_delay</samp></dt> <dd> Set right channel delay. By defaults is <var class="var">2.12</var> milliseconds. </dd> <dt><samp class="option">right_balance</samp></dt> <dd> Set right channel balance. By default is <var class="var">1</var>. </dd> <dt><samp class="option">right_gain</samp></dt> <dd> Set right channel gain. By default is <var class="var">1</var>. </dd> <dt><samp class="option">right_phase</samp></dt> <dd> Change right phase. By default is enabled. </dd> </dl>

 <h3 class="section">8.92 hdcd# TOC</h3>

Decodes High Definition Compatible Digital (HDCD) data. A 16-bit PCM stream with embedded HDCD codes is expanded into a 20-bit PCM stream. The filter supports the Peak Extend and Low-level Gain Adjustment features of HDCD, and detects the Transient Filter flag.   <pre class="example-preformatted">ffmpeg -i HDCD16.flac -af hdcd OUT24.flac </pre>

When using the filter with wav, note the default encoding for wav is 16-bit, so the resulting 20-bit stream will be truncated back to 16-bit. Use something like <code class="command">-acodec pcm_s24le</code> after the filter to get 24-bit PCM output. <pre class="example-preformatted">ffmpeg -i HDCD16.wav -af hdcd OUT16.wav ffmpeg -i HDCD16.wav -af hdcd -c:a pcm_s24le OUT24.wav </pre>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">disable_autoconvert</samp></dt> <dd> Disable any automatic format conversion or resampling in the filter graph.

 </dd> <dt><samp class=”option”>process_stereo</samp></dt> <dd> Process the stereo channels together. If target_gain does not match between channels, consider it invalid and use the last valid target_gain. </dd> <dt><samp class=”option”>cdt_ms</samp></dt> <dd> Set the code detect timer period in ms. </dd> <dt><samp class=”option”>force_pe</samp></dt> <dd> Always extend peaks above -3dBFS even if PE isn’t signaled. </dd> <dt><samp class=”option”>analyze_mode</samp></dt> <dd> Replace audio with a solid tone and adjust the amplitude to signal some specific aspect of the decoding process. The output file can be loaded in an audio editor alongside the original to aid analysis. <code class=”code”>analyze_mode=pe:force_pe=true</code> can be used to see all samples above the PE level. Modes are: <dl class=”table”> <dt>‘<samp class=”samp”>0, off</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class=”samp”>1, lle</samp>’</dt> <dd> Gain adjustment level at each sample </dd> <dt>‘<samp class=”samp”>2, pe</samp>’</dt> <dd> Samples where peak extend occurs </dd> <dt>‘<samp class=”samp”>3, cdt</samp>’</dt> <dd> Samples where the code detect timer is active </dd> <dt>‘<samp class=”samp”>4, tgm</samp>’</dt> <dd> Samples where the target gain does not match between channels </dd> </dl> </dd> </dl>

 <h3 class=”section”>8.93 headphone# TOC</h3>

Apply head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones. The HRIRs are provided via additional streams, for each channel one stereo input stream is needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>map</samp></dt> <dd> Set mapping of input streams for convolution. The argument is a ’|’-separated list of channel names in order as they are given as additional stream inputs for filter. This also specify number of input streams. Number of input streams must be not less than number of channels in first stream plus one. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>lfe</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set size of frame in num-

ber of samples which will be processed at once. Default value is <var class="var">1024</var>. Allowed range is from 1024 to 96000. </dd> <dt><samp class="option">hrir</samp></dt> <dd> Set format of hrir stream. Default value is <var class="var">stereo</var>. Alternative value is <var class="var">multich</var>. If value is set to <var class="var">stereo</var>, number of additional streams should be greater or equal to number of input channels in first input stream. Also each additional stream should have stereo number of channels. If value is set to <var class="var">multich</var>, number of additional streams should be exactly one. Also number of input channels of additional stream should be equal or greater than twice number of channels of first input stream. </dd> </dl>

 <h4 class="subsection">8.93.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Full example using wav files as coefficients with amovie filters for 7.1 downmix, each amovie filter use stereo file with IR coefficients as input. The files give coefficients for each position of virtual loudspeaker: <pre class="example-preformatted">ffmpeg -i input.wav -filter_complex "amovie=azi_270_ele_0_DFC.wav[sr];amovie=azi_90_ele_0_DFC.wav[sl];amovie=azi_225_ele_0_D output.wav </pre>

Full example using wav files as coefficients with amovie filters for 7.1 downmix, but now in <var class="var">multich</var> <var class="var">hrir</var> format. <pre class="example-preformatted">ffmpeg -i input.wav -filter_complex "amovie=minp.wav[hrirs];[0:a][hrirs] output.wav </pre>

 <h3 class="section">8.94 highpass# TOC</h3>

Apply a high-pass filter with 3dB point frequency. The filter can be either single-pole, or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 3000. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt>

<dd> octave </dd> <dt><samp class="option">s</samp></dt>
<dd> slope </dd> <dt><samp class="option">k</samp></dt>
<dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.94.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change highpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change highpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd>

<dt><samp class="option">width, w</samp></dt> <dd> Change highpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change highpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">8.95 join# TOC</h3>

Join multiple input streams into one multi-channel stream. It accepts the following parameters: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of input streams. It defaults to 2. </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The desired output channel layout. It defaults to stereo. </dd> <dt><samp class="option">map</samp></dt> <dd> Map channels from inputs to output. The argument is a ’|’-separated list of mappings, each in the <code class="code"><var class="var">input_idx</var>.<var class="var">in_channel</var>-<var class="var">out_channel</var></code> form. <var class="var">input_idx</var> is the 0-based index of the input stream. <var class="var">in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the specified input stream. <var class="var">out_channel</var> is the name of the output channel. </dd> </dl>

The filter will attempt to guess the mappings when they are not specified explicitly. It does so by first trying to find an unused matching input channel and if that fails it picks the first unused input channel. Join 3 inputs (with properly set channel layouts): <pre class="example-preformatted">ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex join=inputs=3 OUTPUT </pre>

Build a 5.1 output from 6 single-channel streams: <pre class="example-preformatted">ffmpeg -i fl -i fr -i fc -i sl -i sr -i lfe -filter_complex 'join=inputs=6:channel_layout=5.1:map=0.0-FL|1.0-FR|2.0-FC|3.0-SL|4.0-SR|5.0-LFE' out </pre>

 <h3 class="section">8.96 ladspa# TOC</h3>

Load a LADSPA (Linux Audio Developer’s Simple Plugin API) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-ladspa</code>. <dl class="table"> <dt><samp class="option">file, f</samp></dt> <dd> Specifies the name of LADSPA plugin library to load. If the environment variable <code class="env">LADSPA_PATH</code> is defined, the LADSPA plugin is searched in each one of the directories specified by the colon separated list in <code class="env">LADSPA_PATH</code>,

otherwise in the standard LADSPA paths, which are in this order: <samp class="file">HOME/.ladspa/lib/</samp>, <samp class="file">/usr/local/lib/ladspa/</samp>, <samp class="file">/usr/lib/ladspa/</samp>. </dd> <dt><samp class="option">plugin, p</samp></dt> <dd> Specifies the plugin within the library. Some libraries contain only one plugin, but others contain many of them. If this is not set filter will list all available plugins within the specified library. </dd> <dt><samp class="option">controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). Controls need to be defined using the following syntax: c0=<var class="var">value0</var>|c1=<var class="var">value1</var>|c2=<var class="var">value2</var>|..., where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. Alternatively they can be also defined using the following syntax: <var class="var">value0</var>|<var class="var">value1</var>|<var class="var">value2</var>|..., where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> <dt><samp class="option">latency, l</samp></dt> <dd> Enable latency compensation, by default is disabled. Only used if plugin have inputs. </dd> </dl>

 <h4 class="subsection">8.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all available plugins within amp (LADSPA example plugin) library: <pre class="example-preformatted">ladspa=file=amp </pre>

List all available controls and their valid ranges for <code class="code">vcf_notch</code> plugin from <code class="code">VCF</code>

library: <pre class=”example-preformatted”>ladspa=f=vcf:p=vcf_notch:c=help
</pre>

Simulate low quality audio equipment using <code class=”code”>Computer
Music Toolkit</code> (CMT) plugin library: <pre
class=”example-preformatted”>ladspa=file=cmt:plugin=lofi:controls=c0=22|c1=12|c2=12
</pre>

Add reverberation to the audio using TAP-plugins (Tom’s
Audio Processing plugins): <pre class=”example-
preformatted”>ladspa=file=tap_reverb:tap_reverb </pre>

Generate white noise, with 0.2 amplitude:
<pre class=”example-preformatted”>ladspa=file=cmt:noise_source_white:c=c0=.2
</pre>

Generate 20 bpm clicks using plugin <code class=”code”>C*
Click - Metronome</code> from the <code class=”code”>C* Audio Plugin
Suite</code> (CAPS) library: <pre class=”example-
preformatted”>ladspa=file=caps:Click:c=c1=20' </pre>

Apply <code class=”code”>C* Eq10X2 - Stereo 10-band
equaliser</code> effect: <pre class=”example-
preformatted”>ladspa=caps:Eq10X2:c=c0=-48|c9=-24|c3=12|c4=2 </pre>

Increase volume by 20dB using fast lookahead limiter from
Steve Harris <code class=”code”>SWH Plugins</code> collection: <div
class=”example”> <pre class=”example-preformatted”>ladspa=fast_lookahead_limiter_1913:fastLookahead
</pre>

Attenuate low frequencies using Multiband EQ from Steve Harris
<code class=”code”>SWH Plugins</code> collection:
<pre class=”example-preformatted”>ladspa=mbeq_1197:mbeq:-24|-24|-
24|0|0|0|0|0|0|0|0|0|0|0 </pre>

Reduce stereo image using <code class=”code”>Narrower</code>
from the <code class=”code”>C* Audio Plugin Suite</code> (CAPS) library:
 <pre class=”example-preformatted”>ladspa=caps:Narrower
</pre>

Another white noise, now using <code class=”code”>C* Au-
dio Plugin Suite</code> (CAPS) library: <pre
class=”example-preformatted”>ladspa=caps:White:.2 </pre>

Some fractal noise, using <code class=”code”>C* Audio Plugin
Suite</code> (CAPS) library: <pre class=”example-
preformatted”>ladspa=caps:Fractal:c=c1=1 </pre>

Dynamic volume normalization using <code class=”code”>VLevel</code>
plugin: <pre class=”example-preformatted”>ladspa=vlevel-
ladspa:vlevel_mono </pre>

 <h4 class="subsection">8.96.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">cN</samp></dt> <dd> Modify the <var class="var">N</var>-th control value. If the specified value is not valid, it is ignored and prior one is kept. </dd> </dl>

 <h3 class="section">8.97 loudnorm# TOC</h3>

EBU R128 loudness normalization. Includes both dynamic and linear normalization modes. Support for both single pass (livestreams, files) and double pass (files) modes. This algorithm can target IL, LRA, and maximum true peak. In dynamic mode, to accurately detect true peaks, the audio stream will be upsampled to 192 kHz. Use the <code class="code">-ar</code> option or <code class="code">aresample</code> filter to explicitly set an output sample rate. The filter accepts the following options: <dl class="table"> <dt><samp class="option">I, i</samp></dt> <dd> Set integrated loudness target. Range is -70.0 - -5.0. Default value is -24.0. </dd> <dt><samp class="option">LRA, lra</samp></dt> <dd> Set loudness range target. Range is 1.0 - 50.0. Default value is 7.0. </dd> <dt><samp class="option">TP, tp</samp></dt> <dd> Set maximum true peak. Range is -9.0 - +0.0. Default value is -2.0. </dd> <dt><samp class="option">measured_I, measured_i</samp></dt> <dd> Measured IL of input file. Range is -99.0 - +0.0. </dd> <dt><samp class="option">measured_LRA, measured_lra</samp></dt> <dd> Measured LRA of input file. Range is 0.0 - 99.0. </dd> <dt><samp class="option">measured_TP, measured_tp</samp></dt> <dd> Measured true peak of input file. Range is -99.0 - +99.0. </dd> <dt><samp class="option">measured_thresh</samp></dt> <dd> Measured threshold of input file. Range is -99.0 - +0.0. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set offset gain. Gain is applied before the true-peak limiter. Range is -99.0 - +99.0. Default is +0.0. </dd> <dt><samp class="option">linear</samp></dt> <dd> Normalize by linearly scaling the source audio. <code class="code">measured_I</code>, <code class="code">measured_LRA</code>, <code class="code">measured_TP</code>, and <code class="code">measured_thresh</code> must all be specified. Target LRA shouldn’t be lower than source LRA and the change in integrated loudness shouldn’t result in a true peak which exceeds the target TP. If any of these conditions aren’t met, normalization mode will revert to <var class="var">dynamic</var>. Options are <code class="code">true</code> or <code class="code">false</code>.

Default is <code class=”code”>true</code>. </dd> <dt><samp class=”option”>dual_mono</samp></dt> <dd> Treat mono input files as "dual-mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. Options are true or false. Default is false. </dd> <dt><samp class=”option”>print_format</samp></dt> <dd> Set print format for stats. Options are summary, json, or none. Default value is none. </dd> <dt><samp class=”option”>stats_file</samp></dt> <dd> Write stats to specified file. Format is controlled by <samp class=”option”>print_format</samp>, which must be set. Specify <code class=”code”>-</code> to write to standard output. Default is unset. </dd> </dl>

 <h3 class=”section”>8.98 lowpass# TOC</h3>

Apply a low-pass filter with 3dB point frequency. The filter can be either single-pole or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 500. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp

class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt>
<dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt>
<dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt>
<dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set
precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt>
<dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough
value (higher than impulse response length truncated when reaches near zero
values) filtering will become linear phase otherwise if not big enough it will just
produce nasty artifacts. Note that filter delay will be exactly this
many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>8.98.1 Examples# TOC</h4> <ul class=”itemize
mark-bullet”> Lowpass only LFE channel, it LFE is not present it does
nothing: <pre class=”example-preformatted”>lowpass=c=LFE
</pre>

 <h4 class=”subsection”>8.98.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>frequency, f</samp></dt> <dd> Change
lowpass frequency. Syntax for the command is : "<var class=”var”>frequency</var>"
 </dd> <dt><samp class=”option”>width_type, t</samp></dt>
<dd> Change lowpass width_type. Syntax for the command is
: "<var class=”var”>width_type</var>" </dd>
<dt><samp class=”option”>width, w</samp></dt> <dd> Change lowpass width. Syntax for the command is : "<var class=”var”>width</var>"
 </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change
lowpass mix. Syntax for the command is : "<var class=”var”>mix</var>"
</dd> </dl>

 <h3 class=”section”>8.99 lv2#
TOC</h3>

Load a LV2 (LADSPA Version 2) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-lv2</code>. <dl class=”table”> <dt><samp class=”option”>plugin, p</samp></dt> <dd> Specifies the plugin URI. You may need to escape ’:’. </dd> <dt><samp class=”option”>controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). If <samp class=”option”>controls</samp> is set to <code class=”code”>help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class=”option”>sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> </dl>

 <h4 class=”subsection”>8.99.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply bass enhancer plugin from Calf: <pre class=”example-preformatted”>lv2=p=http\\\\://calf.sourceforge.net/plugins/
</pre>

Apply vinyl plugin from Calf: <pre class=”example-preformatted”>lv2=p=http\\\\://calf.sourceforge.net/plugins/Vinyl:c=drone=0.2|aging=0.5
</pre>

Apply bit crusher plugin from ArtyFX: <pre class=”example-preformatted”>lv2=p=http\\\\://www.openavproductions.com/artyfx#bitta:c=crush=0.3
</pre>

 <h4 class=”subsection”>8.99.2 Commands# TOC</h4>

This filter supports all options that are exported by plugin as commands. <h3 class=”section”>8.100 mcom-

pand# TOC</h3> Multiband Compress or expand the audio’s dynamic range. The input audio is divided into bands using 4th order Linkwitz-Riley IIRs. This is akin to the crossover of a loudspeaker, and results in flat frequency response when absent compander action. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>args</samp></dt> <dd> This option syntax is: attack,decay,[attack,decay..] soft-knee points crossover_frequency [delay [initial_volume [gain]]] | attack,decay ... For explanation of each item refer to compand filter documentation. </dd> </dl>

 <h3 class=”section”>8.101 pan# TOC</h3>

Mix channels with specific gain levels. The filter accepts the output channel layout followed by a set of channels definitions. This filter is also designed to efficiently remap the channels of an audio stream. The filter accepts parameters of the form: "<var class=”var”>l</var>|<var class=”var”>outdef</var>|<var class=”var”>outdef</var>|..." <dl class=”table”> <dt><samp class=”option”>l</samp></dt> <dd> output channel layout or number of channels </dd> <dt><samp class=”option”>outdef</samp></dt> <dd> output channel specification, of the form: "<var class=”var”>out_name</var>=[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>[(+-)[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>...]" </dd> <dt><samp class=”option”>out_name</samp></dt> <dd> output channel to define, either a channel name (FL, FR, etc.) or a channel number (c0, c1, etc.) </dd> <dt><samp class=”option”>gain</samp></dt> <dd> multiplicative coefficient for the channel, 1 leaving the volume unchanged </dd> <dt><samp class=”option”>in_name</samp></dt> <dd> input channel to use, see out_name for details; it is not possible to mix named and numbered input channels </dd> </dl>

If the ‘=’ in a channel specification is replaced by ‘<’, then the gains for that specification will be renormalized so that the total is 1, thus avoiding clipping noise. <h4 class=”subsection”>8.101.1 Mixing examples# TOC</h4>

For example, if you want to down-mix from stereo to mono, but with a bigger factor for the left channel: <pre

class=”example-preformatted”>pan=1c|c0=0.9*c0+0.1*c1 </pre>

A customized down-mix to stereo that works automatically for 3-, 4-, 5- and 7-channels surround: <pre class=”example-preformatted”>pan=stereo| FL < FL + 0.5*FC + 0.6*BL + 0.6*SL | FR < FR + 0.5*FC + 0.6*BR + 0.6*SR </pre>

Note that <code class=”command”>ffmpeg</code> integrates a default down-mix (and up-mix) system that should be preferred (see "-ac" option) unless you have very specific needs. <h4 class=”subsection”>8.101.2 Remapping examples# TOC</h4>

The channel remapping will be effective if, and only if: <ul class=”itemize mark-bullet”> gain coefficients are zeroes or ones, only one input per channel output,

If all these conditions are satisfied, the filter will notify the user ("Pure channel mapping detected"), and use an optimized and lossless method to do the remapping. For example, if you have a 5.1 source and want a stereo audio stream by dropping the extra channels: <pre class=”example-preformatted”>pan="stereo| c0=FL | c1=FR" </pre>

Given the same source, you can also switch front left and front right channels and keep the input channel layout: <pre class=”example-preformatted”>pan="5.1| c0=c1 | c1=c0 | c2=c2 | c3=c3 | c4=c4 | c5=c5" </pre>

If the input is a stereo audio stream, you can mute the front left channel (and still keep the stereo channel layout) with: <pre class=”example-preformatted”>pan="stereo|c1=c1" </pre>

Still with a stereo audio stream input, you can copy the right channel in both front left and right: <pre class=”example-preformatted”>pan="stereo| c0=FR | c1=FR" </pre>

 <h3 class=”section”>8.102 replaygain# TOC</h3>

ReplayGain scanner filter. This filter takes an audio stream as an input and outputs it unchanged. At end of filtering it displays <code class=”code”>track_gain</code> and <code class=”code”>track_peak</code>. The filter accepts the following exported read-only options: <dl class=”table”> <dt><samp class=”option”>track_gain</samp></dt> <dd> Exported track gain in dB at end of stream. </dd>

<dt><samp class="option">track_peak</samp></dt> <dd> Exported track peak at end of stream. </dd> </dl>

 <h3 class="section">8.103 resample# TOC</h3>

Convert the audio sample format, sample rate and channel layout. It is not meant to be used directly. <h3 class="section">8.104 rubberband# TOC</h3> Apply time-stretching and pitch-shifting with librubberband. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-librubberband</code>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Set tempo scale factor. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set pitch scale factor. </dd> <dt><samp class="option">transients</samp></dt> <dd> Set transients detector. Possible values are: <dl class="table"> <dt><var class="var">crisp</var></dt> <dt><var class="var">mixed</var></dt> <dt><var class="var">smooth</var></dt> </dl>

</dd> <dt><samp class="option">detector</samp></dt> <dd> Set detector. Possible values are: <dl class="table"> <dt><var class="var">compound</var></dt> <dt><var class="var">percussive</var></dt> <dt><var class="var">soft</var></dt> </dl>

</dd> <dt><samp class="option">phase</samp></dt> <dd> Set phase. Possible values are: <dl class="table"> <dt><var class="var">laminar</var></dt> <dt><var class="var">independent</var></dt> </dl>

</dd> <dt><samp class="option">window</samp></dt> <dd> Set processing window size. Possible values are: <dl class="table"> <dt><var class="var">standard</var></dt> <dt><var class="var">short</var></dt> <dt><var class="var">long</var></dt> </dl>

</dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set smoothing. Possible values are: <dl class="table"> <dt><var class="var">off</var></dt> <dt><var class="var">on</var></dt> </dl>

</dd> <dt><samp class="option">formant</samp></dt> <dd> Enable formant preservation when shift pitching. Possible values are: <dl class="table"> <dt><var class="var">shifted</var></dt> <dt><var class="var">preserved</var></dt> </dl>

</dd> <dt><samp class="option">pitchq</samp></dt> <dd> Set pitch quality. Possible values are: <dl class="table"> <dt><var

class=”var”>quality</var></dt> <dt><var class=”var”>speed</var></dt>
<dt><var class=”var”>consistency</var></dt> </dl>

</dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set channels. Possible values are: <dl class=”table”> <dt><var class=”var”>apart</var></dt> <dt><var class=”var”>together</var></dt>
</dl> </dd> </dl>

 <h4 class=”subsection”>8.104.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class=”var”>tempo</var>" </dd> <dt><samp class=”option”>pitch</samp></dt>
<dd> Change filter pitch scale factor. Syntax for the command is : "<var class=”var”>pitch</var>" </dd> </dl>

 <h3 class=”section”>8.105 sidechaincompress# TOC</h3>

This filter acts like normal compressor but has the ability to compress detected signal using second input signal. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filtered signal then can be filtered with other filters in later stages of processing. See pan and amerge filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>.
 </dd> <dt><samp class=”option”>threshold</samp></dt>
<dd> If a signal of second stream raises above this level it will affect the gain reduction of first stream. By default is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class=”option”>ratio</samp></dt>
<dd> Set a ratio about which the signal is reduced. 1:2 means that if the level raised 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd>
<dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd>
<dt><samp class=”option”>release</samp></dt> <dd> Amount

of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the <code class="code">average</code> level between all channels of side-chain stream or the louder(<code class="code">maximum</code>) channel of side-chain stream affects the reduction. Default is <code class="code">average</code>. </dd> <dt><samp class="option">detection</samp></dt> <dd> Should the exact signal be taken in case of <code class="code">peak</code> or an RMS one in case of <code class="code">rms</code>. Default is <code class="code">rms</code> which is mainly smoother. </dd> <dt><samp class="option">level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class="subsection">8.105.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">8.105.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Full ffmpeg example taking 2 audio inputs, 1st input to be compressed depending on the signal of 2nd input and later compressed signal to be merged with 2nd input: <pre class="example-preformatted">ffmpeg -i main.flac -i sidechain.flac -filter_complex "[1:a]asplit=2[sc][mix];[0:a][sc]sidechaincompress[compr];[compr][mix]amerge" </pre>

 <h3 class="section">8.106 sidechaingate# TOC</h3>

A sidechain gate acts like a normal (wideband) gate but has the ability to filter the detected signal before sending it to the gain reduction stage. Normally a gate uses the full range signal to detect a level above the threshold.

For example: If you cut all lower frequencies from your sidechain signal the gate will decrease the volume of your track only if not enough highs appear. With this technique you are able to reduce the resonation of a natural drum or remove "rumbling" of muted strokes from a heavily distorted guitar. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the mode of operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. If set to <code class="code">upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class="code">downward</code> lower parts of signal will be reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is rms. Can be peak or rms. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is average. Can be average or maximum. </dd> <dt><samp class="option">level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is from 0.015625 to 64. </dd> </dl>

 <h4 class="subsection">8.106.1 Com-

mands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">8.107 silencedetect# TOC</h3>

Detect silence in an audio stream. This filter logs a message when it detects that the input audio volume is less or equal to a noise tolerance value for a duration greater or equal to the minimum detected noise duration. The printed times and duration are expressed in seconds. The <code class="code">lavfi.silence_start</code> or <code class="code">lavfi.silence_start.X</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the silence. The <code class="code">lavfi.silence_duration</code> or <code class="code">lavfi.silence_duration.X</code> and <code class="code">lavfi.silence_end</code> or <code class="code">lavfi.silence_end.X</code> metadata keys are set on the first frame after the silence. If <samp class="option">mono</samp> is enabled, and each channel is evaluated separately, the <code class="code">.X</code> suffixed keys are used, and <code class="code">X</code> corresponds to the channel number. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or amplitude ratio. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set silence duration until notification (default is 2 seconds). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. </dd> <dt><samp class="option">mono, m</samp></dt> <dd> Process each channel separately, instead of combined. By default is disabled. </dd> </dl>

 <h4 class="subsection">8.107.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect 5 seconds of silence with -50dB noise tolerance: <pre class="example-preformatted">silencedetect=n=-50dB:d=5 </pre>

Complete example with <code class="command">ffmpeg</code> to detect silence with 0.0001 noise tolerance in <samp class="file">silence.mp3</samp>:

 <pre class=”example-preformatted”>ffmpeg -i silence.mp3 -af silencedetect=noise=0.0001 -f null - </pre>

 <h3 class=”section”>8.108 silenceremove# TOC</h3>

Remove silence from the beginning, middle or end of the audio.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>start_periods</samp></dt>

<dd> This value is used to indicate if audio should be trimmed at beginning of

the audio. A value of zero indicates no silence should be trimmed from the

beginning. When specifying a non-zero value, it trims audio up until it

finds non-silence. Normally, when trimming silence from beginning of audio

the <var class=”var”>start_periods</var> will be <code class=”code”>1</code> but it can be increased to higher

values to trim all audio up to specific count of non-silence periods.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_duration</samp></dt>

<dd> Specify the amount of time that non-silence must be detected before it stops

trimming audio. By increasing the duration, bursts of noises can be treated

as silence and trimmed off. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_threshold</samp></dt>

<dd> This indicates what sample value should be treated as silence. For digital

audio, a value of <code class=”code”>0</code> may be fine but for audio recorded from analog,

you may wish to increase the value to account for background noise.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_silence</samp></dt>

<dd> Specify max duration of silence at beginning that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>start_mode</samp></dt>

<dd> Specify mode of detection of silence end at start of multi-channel audio.

Can be <var class=”var”>any</var> or <var class=”var”>all</var>. Default is <var class=”var”>any</var>.

With <var class=”var”>any</var>, any sample from any channel that is detected as non-silence

will trigger end of silence trimming at start of audio stream.

With <var class=”var”>all</var>, only if every sample from every channel is detected as non-silence

will trigger end of silence trimming at start of audio stream, limited usage.

</dd>

<dt><samp class=”option”>stop_periods</samp></dt>

<dd> Set the count for trimming silence from the end of audio. When specifying a

positive value, it trims audio after it finds specified silence period.

To remove silence from the middle of a file, specify a <var class=”var”>stop_periods</var>

that is negative. This value is then treated as a positive value and is

used to indicate the effect should restart processing as specified by

<var class=”var”>stop_periods</var>, making it suitable for removing periods of silence

in the middle of the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_duration</samp></dt>

<dd> Specify a duration of silence that must exist before audio is not copied any

more. By specifying a higher duration, silence that is wanted can be left in

the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_threshold</samp></dt>

<dd> This is the same as <samp class=”option”>start_threshold</samp> but for trimming silence from

the end of audio.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_silence</samp></dt>

<dd> Specify max duration of silence at end that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>stop_mode</samp></dt>

<dd> Specify mode of detection of silence start after start of multi-channel audio.

Can be <var class="var">any</var> or <var class="var">all</var>. Default is <var class="var">all</var>.

With <var class="var">any</var>, any sample from any channel that is detected as silence

will trigger start of silence trimming after start of audio stream, limited usage.

With <var class="var">all</var>, only if every sample from every channel is detected as silence

will trigger start of silence trimming after start of audio stream.

</dd>

<dt><samp class="option">detection</samp></dt>

<dd> Set how is silence detected.

<dl class="table">

<dt><samp class="option">avg</samp></dt>

<dd> Mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">rms</samp></dt>

<dd> Root squared mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">peak</samp></dt>

<dd> Maximum of absolute values of samples in moving window.

</dd>

<dt><samp class="option">median</samp></dt>

<dd> Median of absolute values of samples in moving window.

</dd>

<dt><samp class="option">ptp</samp></dt>

<dd> Absolute of max peak to min peak difference of samples in moving window.

</dd>

<dt><samp class="option">dev</samp></dt>

<dd> Standard deviation of values of samples in moving window.

</dd>

</dl>

Default value is <code class="code">rms</code>.

</dd>

<dt><samp class="option">window</samp></dt>

<dd> Set duration in number of seconds used to calculate size of window in number

of samples for detecting silence. Using <code class="code">0</code> will effectively disable

any windowing and use only single sample per channel for silence detection.

In that case it may be needed to also set <samp class="option">start_silence</samp> and/or

<samp class="option">stop_silence</samp> to nonzero values with also <samp class="option">start_duration</samp> and/or

<samp class="option">stop_duration</samp> to nonzero values.

Default value is <code class="code">0.02</code>. Allowed range is from <code class="code">0</code> to <code class="code">10</code>.

</dd>

<dt><samp class="option">timestamp</samp></dt>

<dd> Set processing mode of every audio frame output timestamp.

<dl class="table">

<dt><samp class="option">write</samp></dt>

<dd> Full timestamps rewrite, keep only the start time for the first output frame.

</dd>

<dt><samp class="option">copy</samp></dt>

<dd> Non-dropped frames are left with same timestamp as input audio frame.

</dd>

</dl>

Defaults value is <code class=”code”>write</code>.

</dd>

</dl>

 <h4 class=”subsection”>8.108.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> The following example shows how this filter can be used to start a recording that does not contain the delay at the start which usually occurs between pressing the record button and the start of the performance: <pre class=”example-preformatted”>silenceremove=start_periods=1:start_duration=5:start_threshold=0.02
</pre>

Trim all silence encountered from beginning to end where there is more than 1 second of silence in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-1:stop_duration=1:stop_threshold=-90dB </pre>

Trim all digital silence samples, using peak detection, from beginning to end where there is more than 0 samples of digital silence in audio and digital silence is detected in all channels at same positions in stream: <pre class=”example-preformatted”>silenceremove=window=0:detection=peak:stop_mode=all:start_mode=all:stop_periods=-1:stop_threshold=0 </pre>

Trim every 2nd encountered silence period from beginning to end where there is more than 1 second of silence per silence period in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB </pre>

Similar as above, but keep maximum of 0.5 seconds of silence from each trimmed period: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5 </pre>

Similar as above, but keep maximum of 1.5 seconds of silence from start of audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5:start_periods=1:start_duration=1:start_silence=1.5:stop_threshold=-90dB </pre>

 <h4 class=”subsection”>8.108.2 Commands# TOC</h4>

This filter supports some above options as commands. <h3 class=”section”>8.109 sofalizer# TOC</h3>

SOFAlizer uses head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones (audio formats up to 9 channels supported). The HRTFs are stored in SOFA files (see http://www.sofacoustics.org/ for a database). SOFAlizer is developed at the Acoustics Research Institute (ARI) of the Austrian Academy of Sciences. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libmysofa</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sofa</samp></dt> <dd> Set the SOFA file used for rendering. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set rotation of virtual loudspeakers in deg. Default is 0. </dd> <dt><samp class=”option”>elevation</samp></dt> <dd> Set elevation of virtual speakers in deg. Default is 0. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> Set distance in meters between loudspeakers and the listener with near-field HRTFs. Default is 1. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>speakers</samp></dt> <dd> Set custom positions of virtual loudspeakers. Syntax for this option is: <CH> <AZIM> <ELEV>[|<CH> <AZIM> <ELEV>|...]. Each virtual loudspeaker is described with short channel name following with azimuth and elevation in degrees. Each virtual loudspeaker description is separated by ’|’. For example to override front left and front right channel positions use: ’speakers=FL 45 15|FR 345 15’. Descriptions with unrecognised channel names are ignored. </dd> <dt><samp class=”option”>lfegain</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>framesize</samp></dt> <dd> Set custom frame size in number of samples. Default is 1024. Allowed range is from 1024 to 96000. Only used if option ‘<samp class=”samp”>type</samp>’ is set to <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Should all IRs be normalized upon importing SOFA file. By default is enabled. </dd> <dt><samp class=”option”>interpolate</samp></dt> <dd> Should

nearest IRs be interpolated with neighbor IRs if exact position does not match. By default is disabled. </dd> <dt><samp class="option">minphase</samp></dt> <dd> Minphase all IRs upon loading of SOFA file. By default is disabled. </dd> <dt><samp class="option">anglestep</samp></dt> <dd> Set neighbor search angle step. Only used if option <var class="var">interpolate</var> is enabled. </dd> <dt><samp class="option">radstep</samp></dt> <dd> Set neighbor search radius step. Only used if option <var class="var">interpolate</var> is enabled. </dd> </dl>

 <h4 class="subsection">8.109.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Using ClubFritz6 sofa file: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz6.sofa:type=freq:r
</pre>

Using ClubFritz12 sofa file and bigger radius with small rotation: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz12.sofa:type=
</pre>

Similar as above but with custom speaker positions for front left, front right, back left and back right and also with custom gain: <pre class="example-preformatted">"sofalizer=sofa=/path/to/ClubFritz6.sofa:type=
45|FR 315|BL 135|BR 225:gain=28" </pre>

 <h3 class="section">8.110 speechnorm# TOC</h3> Speech Normalizer. This filter expands or compresses each half-cycle of audio samples (local set of samples all above or all below zero and between two nearest zero crossings) depending on threshold value, so audio reaches target peak value under conditions controlled by below options. The filter accepts the following options: <dl class="table"> <dt><samp class="option">peak, p</samp></dt> <dd> Set the expansion target peak value. This specifies the highest allowed absolute amplitude level for the normalized audio input. Default value is 0.95. Allowed range is from 0.0 to 1.0. </dd> <dt><samp class="option">expansion, e</samp></dt> <dd> Set the maximum expansion factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples expansion. The maximum expansion would be such that local peak value reaches target peak value but never to surpass it and that ratio between new and previous peak value does not surpass this option value. </dd> <dt><samp class="option">compression, c</samp></dt> <dd> Set the maximum compression factor. Allowed range is from 1.0 to 50.0. Default

value is 2.0. This option controls maximum local half-cycle of samples compression. This option is used only if <samp class=”option”>threshold</samp> option is set to value greater than 0.0, then in such cases when local peak is lower or same as value set by <samp class=”option”>threshold</samp> all samples belonging to that peak’s half-cycle will be compressed by current compression factor. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set the threshold value. Default value is 0.0. Allowed range is from 0.0 to 1.0. This option specifies which half-cycles of samples will be compressed and which will be expanded. Any half-cycle samples with their local peak value below or same as this option value will be compressed by current compression factor, otherwise, if greater than threshold value they will be expanded with expansion factor so that it could reach peak target value but never surpass it. </dd> <dt><samp class=”option”>raise, r</samp></dt> <dd> Set the expansion raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast expansion factor is raised per each new half-cycle until it reaches <samp class=”option”>expansion</samp> value. Setting this options too high may lead to distortions. </dd> <dt><samp class=”option”>fall, f</samp></dt> <dd> Set the compression raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast compression factor is raised per each new half-cycle until it reaches <samp class=”option”>compression</samp> value. </dd> <dt><samp class=”option”>channels, h</samp></dt> <dd> Specify which channels to filter, by default all available channels are filtered. </dd> <dt><samp class=”option”>invert, i</samp></dt> <dd> Enable inverted filtering, by default is disabled. This inverts interpretation of <samp class=”option”>threshold</samp> option. When enabled any half-cycle of samples with their local peak value below or same as <samp class=”option”>threshold</samp> option will be expanded otherwise it will be compressed. </dd> <dt><samp class=”option”>link, l</samp></dt> <dd> Link channels when calculating gain applied to each filtered channel sample, by default is disabled. When disabled each filtered channel gain calculation is independent, otherwise when this option is enabled the minimum of all possible gains for each filtered channel is used. </dd> <dt><samp class=”option”>rms, m</samp></dt> <dd> Set the expansion target RMS value. This specifies the highest allowed RMS level for the normalized audio input. Default value is 0.0, thus disabled. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class=”subsection”>8.110.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>8.110.2 Examples<a class=”anchor hidden-xs” href=”#Examples-

34” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Weak and slow amplification: <pre class=”example-preformatted”>speechnorm=e=3:r=0.00001:l=1 </pre>

Moderate and slow amplification: <pre class=”example-preformatted”>speechnorm=e=6.25:r=0.00001:l=1 </pre>

Strong and fast amplification: <pre class=”example-preformatted”>speechnorm=e=12.5:r=0.0001:l=1 </pre>

Very strong and fast amplification: <pre class=”example-preformatted”>speechnorm=e=25:r=0.0001:l=1 </pre>

Extreme and fast amplification: <pre class=”example-preformatted”>speechnorm=e=50:r=0.0001:l=1 </pre>

 <h3 class=”section”>8.111 stereotools# TOC</h3>

This filter has some handy utilities to manage stereo signals, for converting M/S stereo recordings to L/R signal while having control over the parameters or spreading the stereo image of master track. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>balance_in</samp></dt> <dd> Set input balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>balance_out</samp></dt> <dd> Set output balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>softclip</samp></dt> <dd> Enable softclipping. Results in analog distortion instead of harsh digital 0dB clipping. Disabled by default. </dd> <dt><samp class=”option”>mutel</samp></dt> <dd> Mute the left channel. Disabled by default. </dd> <dt><samp class=”option”>muter</samp></dt> <dd> Mute the right channel. Disabled by default. </dd> <dt><samp class=”option”>phasel</samp></dt> <dd> Change the phase of the left channel. Disabled by default. </dd> <dt><samp class=”option”>phaser</samp></dt> <dd> Change

the phase of the right channel. Disabled by default. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set stereo mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">lr>lr</samp>’</dt> <dd> Left/Right to Left/Right, this is default. </dd> <dt>‘<samp class="samp">lr>ms</samp>’</dt> <dd> Left/Right to Mid/Side. </dd> <dt>‘<samp class="samp">ms>lr</samp>’</dt> <dd> Mid/Side to Left/Right. </dd> <dt>‘<samp class="samp">lr>ll</samp>’</dt> <dd> Left/Right to Left/Left. </dd> <dt>‘<samp class="samp">lr>rr</samp>’</dt> <dd> Left/Right to Right/Right. </dd> <dt>‘<samp class="samp">lr>l+r</samp>’</dt> <dd> Left/Right to Left + Right. </dd> <dt>‘<samp class="samp">lr>rl</samp>’</dt> <dd> Left/Right to Right/Left. </dd> <dt>‘<samp class="samp">ms>ll</samp>’</dt> <dd> Mid/Side to Left/Left. </dd> <dt>‘<samp class="samp">ms>rr</samp>’</dt> <dd> Mid/Side to Right/Right. </dd> <dt>‘<samp class="samp">ms>rl</samp>’</dt> <dd> Mid/Side to Right/Left. </dd> <dt>‘<samp class="samp">lr>l-r</samp>’</dt> <dd> Left/Right to Left - Right. </dd> </dl>

</dd> <dt><samp class="option">slev</samp></dt> <dd> Set level of side signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">sbal</samp></dt> <dd> Set balance of side signal. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">mlev</samp></dt> <dd> Set level of the middle signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mpan</samp></dt> <dd> Set middle signal pan. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">base</samp></dt> <dd> Set stereo base between mono and inversed channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay in milliseconds how much to delay left from right channel and vice versa. Default is 0. Allowed range is from -20 to 20. </dd> <dt><samp class="option">sclevel</samp></dt> <dd> Set S/C level. Default is 1. Allowed range is from 1 to 100. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set the stereo phase in degrees. Default is 0. Allowed range is from 0 to 360. </dd> <dt><samp class="option">bmode_in, bmode_out</samp></dt> <dd> Set balance mode for balance_in/balance_out option. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">balance</samp>’</dt> <dd> Classic balance mode. Attenuate one channel at time. Gain is raised up to 1. </dd> <dt>‘<samp class="samp">amplitude</samp>’</dt> <dd> Similar as classic mode above but gain is raised up to 2. </dd> <dt>‘<samp class="samp">power</samp>’</dt>

<dd> Equal power distribution, from -6dB to +6dB range. </dd>
</dl> </dd> </dl>

 <h4 class="subsection">8.111.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h4 class="subsection">8.111.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply karaoke like effect: <pre class="example-preformatted">stereotools=mlev=0.015625
</pre>

Convert M/S signal to L/R: <pre class="example-preformatted">"stereotools=mode=ms>lr"
</pre>

 <h3 class="section">8.112 stereowiden# TOC</h3>

This filter enhance the stereo effect by suppressing signal common to both channels and by delaying the signal of left into right and vice versa, thereby widening the stereo effect. The filter accepts the following options:
 <dl class="table"> <dt><samp class="option">delay</samp></dt>
<dd> Time in milliseconds of the delay of left signal into right and vice versa. Default is 20 milliseconds. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Amount of gain in delayed signal into right and vice versa. Gives a delay effect of left signal in right output and vice versa which gives widening effect. Default is 0.3. </dd> <dt><samp class="option">crossfeed</samp></dt>
<dd> Cross feed of left into right with inverted phase. This helps in suppressing the mono. If the value is 1 it will cancel all the signal common to both channels. Default is 0.3. </dd> <dt><samp class="option">drymix</samp></dt> <dd> Set level of input signal of original channel. Default is 0.8. </dd> </dl>

 <h4 class="subsection">8.112.1 Commands# TOC</h4>

This filter supports the all above options except <code class="code">delay</code>
as commands. <a

name=”superequalizer”> <h3 class=”section”>8.113 superequalizer# TOC</h3> Apply 18 band equalizer. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>1b</samp></dt> <dd> Set 65Hz band gain. </dd> <dt><samp class=”option”>2b</samp></dt> <dd> Set 92Hz band gain. </dd> <dt><samp class=”option”>3b</samp></dt> <dd> Set 131Hz band gain. </dd> <dt><samp class=”option”>4b</samp></dt> <dd> Set 185Hz band gain. </dd> <dt><samp class=”option”>5b</samp></dt> <dd> Set 262Hz band gain. </dd> <dt><samp class=”option”>6b</samp></dt> <dd> Set 370Hz band gain. </dd> <dt><samp class=”option”>7b</samp></dt> <dd> Set 523Hz band gain. </dd> <dt><samp class=”option”>8b</samp></dt> <dd> Set 740Hz band gain. </dd> <dt><samp class=”option”>9b</samp></dt> <dd> Set 1047Hz band gain. </dd> <dt><samp class=”option”>10b</samp></dt> <dd> Set 1480Hz band gain. </dd> <dt><samp class=”option”>11b</samp></dt> <dd> Set 2093Hz band gain. </dd> <dt><samp class=”option”>12b</samp></dt> <dd> Set 2960Hz band gain. </dd> <dt><samp class=”option”>13b</samp></dt> <dd> Set 4186Hz band gain. </dd> <dt><samp class=”option”>14b</samp></dt> <dd> Set 5920Hz band gain. </dd> <dt><samp class=”option”>15b</samp></dt> <dd> Set 8372Hz band gain. </dd> <dt><samp class=”option”>16b</samp></dt> <dd> Set 11840Hz band gain. </dd> <dt><samp class=”option”>17b</samp></dt> <dd> Set 16744Hz band gain. </dd> <dt><samp class=”option”>18b</samp></dt> <dd> Set 20000Hz band gain. </dd> </dl>

<h3 class=”section”>8.114 surround# TOC</h3>

Apply audio surround upmix filter.

This filter allows to produce multichannel output from audio stream.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>chl_out</samp></dt>

<dd> Set output channel layout. By default, this is <var class=”var”>5.1</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">chl_in</samp></dt>

<dd> Set input channel layout. By default, this is <var class="var">stereo</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">level_in</samp></dt>

<dd> Set input volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">level_out</samp></dt>

<dd> Set output volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe</samp></dt>

<dd> Enable LFE channel output if output channel layout has it. By default, this is enabled.

</dd>

<dt><samp class="option">lfe_low</samp></dt>

<dd> Set LFE low cut off frequency. By default, this is <var class="var">128</var> Hz.

</dd>

<dt><samp class="option">lfe_high</samp></dt>

<dd> Set LFE high cut off frequency. By default, this is <var class="var">256</var> Hz.

</dd>

<dt><samp class="option">lfe_mode</samp></dt>

<dd> Set LFE mode, can be <var class="var">add</var> or <var class="var">sub</var>. Default is <var class="var">add</var>.

In <var class="var">add</var> mode, LFE channel is created from input audio and added to output.

In <var class="var">sub</var> mode, LFE channel is created from input audio and added to output but

also all non-LFE output channels are subtracted with output LFE channel.

</dd>

<dt><samp class="option">smooth</samp></dt>

<dd> Set temporal smoothness strength, used to gradually change factors when transforming

stereo sound in time. Allowed range is from <var class="var">0.0</var> to <var class="var">1.0</var>.

Useful to improve output quality with <var class="var">focus</var> option values greater than <var class="var">0.0</var>.

Default is <var class="var">0.0</var>. Only values inside this range and without edges are effective.

</dd>

<dt><samp class="option">angle</samp></dt>

<dd> Set angle of stereo surround transform, Allowed range is from <var class="var">0</var> to <var class="var">360</var>.

Default is <var class="var">90</var>.

</dd>

<dt><samp class="option">focus</samp></dt>

<dd> Set focus of stereo surround transform, Allowed range is from <var class="var">-1</var> to <var class="var">1</var>.

Default is <var class="var">0</var>.

</dd>

<dt><samp class="option">fc_in</samp></dt>

<dd> Set front center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fc_out</samp></dt>

<dd> Set front center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_in</samp></dt>

<dd> Set front left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_out</samp></dt>

<dd> Set front left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_in</samp></dt>

<dd> Set front right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_out</samp></dt>

<dd> Set front right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_in</samp></dt>

<dd> Set side left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_out</samp></dt>

<dd> Set side left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_in</samp></dt>

<dd> Set side right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_out</samp></dt>

<dd> Set side right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_in</samp></dt>

<dd> Set back left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_out</samp></dt>

<dd> Set back left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_in</samp></dt>

<dd> Set back right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_out</samp></dt>

<dd> Set back right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_in</samp></dt>

<dd> Set back center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_out</samp></dt>

<dd> Set back center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_in</samp></dt>

<dd> Set LFE input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_out</samp></dt>

<dd> Set LFE output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">allx</samp></dt>

<dd> Set spread usage of stereo image across X axis for all channels.

Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.

By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>

<dt><samp class="option">ally</samp></dt>

<dd> Set spread usage of stereo image across Y axis for all channels.

Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.

By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>

<dt><samp class="option">fcx, flx, frx, blx, brx, slx, srx, bcx</samp></dt>

<dd> Set spread usage of stereo image across X axis for each channel.

Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.

By default this value is <var class="var">0.5</var>.

</dd>

<dt><samp class="option">fcy, fly, fry, bly, bry, sly, sry, bcy</samp></dt>

<dd> Set spread usage of stereo image across Y axis for each channel.

Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.

By default this value is <var class="var">0.5</var>.

</dd>

<dt><samp class="option">win_size</samp></dt>

<dd> Set window size. Allowed range is from <var class="var">1024</var>
to <var class="var">65536</var>. Default size is <var class="var">4096</var>.

</dd>

<dt><samp class="option">win_func</samp></dt>

<dd> Set window function.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt>

<dt>‘<samp class="samp">hann, hanning</samp>’</dt>

<dt>‘<samp class="samp">hamming</samp>’</dt>

<dt>‘<samp class="samp">blackman</samp>’</dt>

<dt>‘<samp class="samp">welch</samp>’</dt>

<dt>‘<samp class="samp">flattop</samp>’</dt>

<dt>‘<samp class="samp">bharris</samp>’</dt>

<dt>‘<samp class="samp">bnuttall</samp>’</dt>

<dt>‘<samp class="samp">bhann</samp>’</dt>

<dt>‘<samp class="samp">sine</samp>’</dt>

<dt>‘<samp class="samp">nuttall</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hann</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class="code">0.5</code>.

</dd>

</dl>

 <h3 class="section">8.115 tiltshelf# TOC</h3>

Boost or cut the lower frequencies and cut or boost higher frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">3000</code> Hz. </dd>

<dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.115.1 Commands# TOC</h4>

This filter supports some options as commands. <h3 class="section">8.116

treble, highshelf# TOC</h3>

Boost or cut treble (upper) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>gain, g</samp></dt> <dd> Give the gain at whichever is the lower of ~22 kHz and the Nyquist frequency. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class=”code”>3000</code> Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt>

<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">8.116.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change treble frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change treble width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change treble width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change treble gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change treble mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">8.117 tremolo# TOC</h3>

Sinusoidal amplitude modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Modulation frequencies in the subharmonic range (20 Hz or lower) will result in a tremolo effect. This filter may also be used as a ring modulator by specifying a modulation frequency higher than 20 Hz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">8.118 vibrato# TOC</h3>

Sinusoidal phase modulation. The filter accepts the following op-

tions: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">8.119 virtualbass# TOC</h3>

Apply audio Virtual Bass filter. This filter accepts stereo input and produce stereo with LFE (2.1) channels output. The newly produced LFE channel have enhanced virtual bass originally obtained from both stereo channels. This filter outputs front left and front right channels unchanged as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set the virtual bass cutoff frequency. Default value is 250 Hz. Allowed range is from 100 to 500 Hz. </dd> <dt><samp class="option">strength</samp></dt> <dd> Set the virtual bass strength. Allowed range is from 0.5 to 3. Default value is 3. </dd> </dl>

 <h3 class="section">8.120 volume# TOC</h3>

Adjust the input audio volume. It accepts the following parameters: <dl class="table"> <dt><samp class="option">volume</samp></dt> <dd> Set audio volume expression. Output values are clipped to the maximum value. The output audio volume is given by the relation: <pre class="example-preformatted"><var class="var">output_volume</var> = <var class="var">volume</var> * <var class="var">input_volume</var> </pre>

The default value for <var class="var">volume</var> is "1.0". </dd> <dt><samp class="option">precision</samp></dt> <dd> This parameter represents the mathematical precision. It determines which input sample formats will be allowed, which affects the precision of the volume scaling. <dl class="table"> <dt><samp class="option">fixed</samp></dt> <dd> 8-bit fixed-point; this limits input sample format to U8, S16, and S32. </dd> <dt><samp class="option">float</samp></dt> <dd> 32-bit floating-point; this limits input sample format to FLT. (default) </dd> <dt><samp class="option">double</samp></dt> <dd> 64-bit floating-point; this limits input sample format to DBL. </dd> </dl>

</dd> <dt><samp class="option">replaygain</samp></dt> <dd> Choose

the behaviour on encountering ReplayGain side data in input frames.
<dl class="table"> <dt><samp class="option">drop</samp></dt>
<dd> Remove ReplayGain side data, ignoring its contents (the default). </dd> <dt><samp class="option">ignore</samp></dt>
<dd> Ignore ReplayGain side data, but leave it in the frame. </dd>
<dt><samp class="option">track</samp></dt> <dd> Prefer the track gain, if present. </dd> <dt><samp class="option">album</samp></dt>
<dd> Prefer the album gain, if present. </dd> </dl>

</dd> <dt><samp class="option">replaygain_preamp</samp></dt>
<dd> Pre-amplification gain in dB to apply to the selected replaygain gain.
 Default value for <var class="var">replaygain_preamp</var> is 0.0. </dd> <dt><samp class="option">replaygain_noclip</samp></dt>
<dd> Prevent clipping by limiting the gain applied. Default value for <var class="var">replaygain_noclip</var> is 1. </dd>
<dt><samp class="option">eval</samp></dt> <dd> Set when the volume expression is evaluated. It accepts the following values:
<dl class="table"> <dt>‘<samp class="samp">once</samp>’</dt>
<dd> only evaluate expression once during the filter initialization, or when the ‘<samp class="samp">volume</samp>’ command is sent
 </dd> <dt>‘<samp class="samp">frame</samp>’</dt>
<dd> evaluate expression for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">once</samp>’.
</dd> </dl>

The volume expression can contain the following parameters. <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> frame number (starting at zero) </dd> <dt><samp class="option">nb_channels</samp></dt>
<dd> number of channels </dd> <dt><samp class="option">nb_consumed_samples</samp></
<dd> number of samples consumed by the filter </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> number of samples in the current frame </dd> <dt><samp class="option">pos</samp></dt>
<dd> original frame position in the file; deprecated, do not use </dd>
<dt><samp class="option">pts</samp></dt> <dd> frame PTS
</dd> <dt><samp class="option">sample_rate</samp></dt>
<dd> sample rate </dd> <dt><samp class="option">startpts</samp></dt>
<dd> PTS at start of stream </dd> <dt><samp class="option">startt</samp></dt>
<dd> time at start of stream </dd> <dt><samp class="option">t</samp></dt>
<dd> frame time </dd> <dt><samp class="option">tb</samp></dt>
<dd> timestamp timebase </dd> <dt><samp class="option">volume</samp></dt>
<dd> last set volume value </dd> </dl>

Note that when <samp class="option">eval</samp> is set to ‘<samp class="samp">once</samp>’ only the <var class="var">sample_rate</var> and <var class="var">tb</var> variables are available, all other variables will evaluate to NAN.
<h4 class="subsection">8.120.1 Commands<a

class=”anchor hidden-xs” href=”#Commands-53” aria-hidden=”true”>#
TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>volume</samp></dt> <dd> Modify the volume expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class=”subsection”>8.120.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Halve the input audio volume: <pre class=”example-preformatted”>volume=volume=0.5
volume=volume=1/2 volume=volume=-6.0206dB </pre>

In all the above example the named key for <samp class=”option”>volume</samp> can be omitted, for example like in: <pre class=”example-preformatted”>volume=0.5 </pre>

Increase input audio power by 6 decibels using fixed-point precision:
 <pre class=”example-preformatted”>volume=volume=6dB:precision=fixed
</pre>

Fade volume after time 10 with an annihilation period of 5 seconds:
 <pre class=”example-preformatted”>volume='if(lt(t,10),1,max(1-(t-10)/5,0))':eval=frame </pre>

 <h3 class=”section”>8.121 volumedetect# TOC</h3>

Detect the volume of the input video. The filter has no parameters. It supports only 16-bit signed integer samples, so the input will be converted when needed. Statistics about the volume will be printed in the log when the input stream end is reached. In particular it will show the mean volume (root mean square), maximum volume (on a per-sample basis), and the beginning of a histogram of the registered volume values (from the maximum value to a cumulated 1/1000 of the samples).
All volumes are in decibels relative to the maximum PCM value.
 <h4 class=”subsection”>8.121.1 Examples# TOC</h4>

Here is an excerpt of the output: <pre class=”example-preformatted”>[Parsed_volumedetect_0 0xa23120]

mean_volume: -27 dB [Parsed_volumedetect_0 0xa23120] max_volume: -4 dB [Parsed_volumedetect_0 0xa23120] histogram_4db: 6 [Parsed_volumedetect_0 0xa23120] histogram_5db: 62 [Parsed_volumedetect_0 0xa23120] histogram_6db: 286 [Parsed_volumedetect_0 0xa23120] histogram_7db: 1042 [Parsed_volumedetect_0 0xa23120] histogram_8db: 2551 [Parsed_volumedetect_0 0xa23120] histogram_9db: 4609 [Parsed_volumedetect_0 0xa23120] histogram_10db: 8409 </pre>

It means that: <ul class="itemize mark-bullet"> The mean square energy is approximately -27 dB, or 10^-2.7. The largest sample is at -4 dB, or more precisely between -4 dB and -5 dB. There are 6 samples at -4 dB, 62 at -5 dB, 286 at -6 dB, etc.

In other words, raising the volume by +4 dB does not cause any clipping, raising it by +5 dB causes clipping for 6 samples, etc. <h3 class="section">8.122 whisper# TOC</h3>

It runs automatic speech recognition using the OpenAI’s Whisper model. It requires the whisper.cpp library (https://github.com/ggml-org/whisper.cpp) as a prerequisite. After installing the library it can be enabled using: <code class="code">./configure --enable-whisper</code>. The filter has following options: <dl class="table"> <dt><samp class="option">model</samp></dt> <dd> The file path of the downloaded whisper.cpp model (mandatory). </dd> <dt><samp class="option">language</samp></dt> <dd> The language to use for transcription. Besides a language code, the following special values enable auto-detection: <dl class="table"> <dt>‘<samp class="samp">eval</samp>’</dt> <dd> Re-run language detection on every queued chunk. This costs an extra encoder pass per chunk and the detected language may change mid-file. </dd> <dt>‘<samp class="samp">lock</samp>’</dt> <dd> Detect the language on the first chunk that produces transcribed text, then reuse it for the remaining chunks. </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Currently an alias for ‘<samp class="samp">eval</samp>’. This may change in a future version, so use ‘<samp class="samp">eval</samp>’ or ‘<samp class="samp">lock</samp>’ if the behavior matters. </dd> </dl>

Default value: <code class="code">"auto"</code> </dd> <dt><samp class="option">translate</samp></dt> <dd> If enabled, translate the transcription from the source language to English. A multilingual model is required to enable this option. Default value: <code class="code">"false"</code> </dd> <dt><samp

class=”option”>queue</samp></dt> <dd> The maximum size that will be queued into the filter before processing the audio with whisper. Using a small value the audio stream will be processed more often, but the transcription quality will be lower and the required processing power will be higher. Using a large value (e.g. 10-20s) will produce more accurate results using less CPU (as using the whisper-cli tool), but the transcription latency will be higher, thus not useful to process real-time streams. Consider using the vad_model option associated with a large queue value. Default value: <code class=”code”>"3"</code> </dd> <dt><samp class=”option”>use_gpu</samp></dt> <dd> If the GPU support should be enabled. Default value: <code class=”code”>"true"</code> </dd> <dt><samp class=”option”>gpu_device</samp></dt> <dd> The GPU device index to use. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>destination</samp></dt> <dd> If set, the transcription output will be sent to the specified file or URL (use one of the FFmpeg AVIO protocols); otherwise, the output will be logged as info messages. The output will also be set in the "lavfi.whisper.text" frame metadata. If the destination is a file and it already exists, it will be overwritten. </dd> <dt><samp class=”option”>format</samp></dt> <dd> The destination format string; it could be "text" (only the transcribed text will be sent to the destination), "srt" (subtitle format) or "json". JSON segments include a <code class=”code”>language</code> field containing the language code reported by whisper.cpp for the transcription chunk, or <code class=”code”>unknown</code> if none is available. All segments from the same chunk have the same language. Default value: <code class=”code”>"text"</code> </dd> <dt><samp class=”option”>max_len</samp></dt> <dd> Maximum segment length in characters. When set to a value greater than 0, transcription segments will be split by word to not exceed this length. This is useful for generating subtitles with shorter lines. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>vad_model</samp></dt> <dd> Path to the VAD model file. If set, the filter will load an additional voice activity detection module (https://github.com/snakers4/silero-vad) that will be used to fragment the audio queue; use this option setting a valid path obtained from the whisper.cpp repository (e.g. "../whisper.cpp/models/ggml-silero-v5.1.2.bin") and increase the queue parameter to a higher value (e.g. 20). </dd> <dt><samp class=”option”>vad_threshold</samp></dt> <dd> The VAD threshold to use. Default value: <code class=”code”>"0.5"</code> </dd> <dt><samp class=”option”>vad_min_speech_duration</samp></dt> <dd> The minimum VAD speaking duration. Default value: <code class=”code”>"0.1"</code> </dd> <dt><samp class=”option”>vad_min_silence_duration</samp></dt> <dd> The minimum VAD silence duration. Default value: <code class=”code”>"0.5"</code>

 </dd> </dl>

 <h4 class="subsection">8.122.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Run a transcription with srt file generation: <pre class="example-preformatted">ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=output.srt\ :format=srt" -f null - </pre>

Run a transcription and send the output in JSON format to an HTTP service: <pre class="example-preformatted">ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=http\\://localhost\\:3000\ :format=json' -f null - </pre>

Transcribe the microphone input using the VAD option: <pre class="example-preformatted">ffmpeg -loglevel warning -f pulse -i default \ -af 'highpass=f=200,lowpass=f=3000,whisper=model=../whisper.cpp/models/ggml-medium.bin\ :language=en\ :queue=10\ :destination=-\ :format=json\ :vad_model=../whisper.cpp/models/ggml-silero-v5.1.2.bin' -f null - </pre>

 <h2 class="chapter">9 Audio Sources# TOC</h2>

Below is a description of the currently available audio sources. <h3 class="section">9.1 abuffer# TOC</h3>

Buffer audio frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">time_base</samp></dt> <dd> The timebase which will be used for timestamps of submitted frames. It must be either a floating-point number or in <var class="var">numerator</var>/<var class="var">denominator</var> form. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> The sample rate of the incoming audio buffers. </dd> <dt><samp class="option">sample_fmt</samp></dt> <dd> The sample format of the incoming audio buffers. Either a sample format name or its corresponding integer representation from the enum AVSample-

Format in <samp class="file">libavutil/samplefmt.h</samp> </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the incoming audio buffers. Either a channel layout name from channel_layout_map in <samp class="file">libavutil/channel_layout.c</samp> or its corresponding integer representation from the AV_CH_LAYOUT_* macros in <samp class="file">libavutil/channel_layout.h</samp> </dd> <dt><samp class="option">channels</samp></dt> <dd> The number of channels of the incoming audio buffers. If both <var class="var">channels</var> and <var class="var">channel_layout</var> are specified, then they must be consistent. </dd> </dl>

 <h4 class="subsection">9.1.1 Examples# TOC</h4>

 <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=s16p
</pre>

will instruct the source to accept planar 16bit signed stereo at 44100Hz. Since the sample format with name "s16p" corresponds to the number 6 and the "stereo" channel layout corresponds to the value 0x3, this is equivalent to: <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=6:channel_layout=0x3
</pre>

 <h3 class="section">9.2 aevalsrc# TOC</h3>

Generate an audio signal specified by an expression. This source accepts in input one or more expressions (one for each channel), which are evaluated and used to generate a corresponding audio signal. This source accepts the following options: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. In case the <samp class="option">channel_layout</samp> option is not specified, the selected channel layout depends on the number of provided expressions. Otherwise the last specified expression is applied to the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set the channel layout. The number of channels in the specified layout must be equal to the number of specified expressions. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated

audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default to 1024. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> </dl>

 <h4 class="subsection">9.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate silence: <pre class="example-preformatted">aevalsrc=0 </pre>

Generate a sin signal with frequency of 440 Hz, set sample rate to 8000 Hz: <pre class="example-preformatted">aevalsrc="sin(440*2*PI*t):s=8000" </pre>

Generate a two channels signal, specify the channel layout (Front Center + Back Center) explicitly: <pre class="example-preformatted">aevalsrc="sin(420*2*PI*t)|cos(430*2*PI*t):c=FC|BC" </pre>

Generate white noise: <pre class="example-preformatted">aevalsrc="-2+random(0)" </pre>

Generate an amplitude modulated signal: <pre class="example-preformatted">aevalsrc="sin(10*2*PI*t)*sin(880*2*PI*t)" </pre>

Generate 2.5 Hz binaural beats on a 360 Hz carrier: <pre class="example-preformatted">aevalsrc="0.1*sin(2*PI*(360-2.5/2)*t) | 0.1*sin(2*PI*(360+2.5/2)*t)" </pre>

 <h3 class="section">9.3 afdelaysrc# TOC</h3>

Generate a fractional delay FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay, d</samp></dt> <dd> Set the fractional delay. Default is 0. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set the number of filter coefficients in output audio stream. Default value is 0. </dd> <dt><samp class=”option”>channel_layout, c</samp></dt> <dd> Specifies the channel layout, and can be a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". </dd> </dl>

 <h3 class=”section”>9.4 afireqsrc# TOC</h3>

Generate a FIR equalizer coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset, p</samp></dt> <dd> Set equalizer preset. Default preset is <code class=”code”>flat</code>. Available presets are: <dl class=”table”> <dt>‘<samp class=”samp”>custom</samp>’</dt> <dt>‘<samp class=”samp”>flat</samp>’</dt> <dt>‘<samp class=”samp”>acoustic</samp>’</dt> <dt>‘<samp class=”samp”>bass</samp>’</ <dt>‘<samp class=”samp”>beats</samp>’</dt> <dt>‘<samp class=”samp”>classic</samp>’</dt> <dt>‘<samp class=”samp”>clear</samp>’</d <dt>‘<samp class=”samp”>deep bass</samp>’</dt> <dt>‘<samp class=”samp”>dubstep</samp>’</dt> <dt>‘<samp class=”samp”>electronic</samp>’</dt> <dt>‘<samp class=”samp”>hard-style</samp>’</dt> <dt>‘<samp class=”samp”>hip-hop</samp>’</dt> <dt>‘<samp class=”samp”>jazz</samp>’</ <dt>‘<samp class=”samp”>metal</samp>’</dt> <dt>‘<samp class=”samp”>movie</samp>’</dt> <dt>‘<samp class=”samp”>pop</samp>’</dt <dt>‘<samp class=”samp”>r&b</samp>’</dt> <dt>‘<samp class=”samp”>rock</samp>’</dt> <dt>‘<samp class=”samp”>vocal booster</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gains, g</samp></dt> <dd> Set custom gains for each band. Only used if the preset option is set to <code class=”code”>custom</code>. Gains are separated by white spaces and each gain is set in dBFS. Default is <code class=”code”>0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0</code>. </dd> <dt><samp class="option">bands, b</samp></dt> <dd> Set the custom bands from where custom equalizer gains are set. This must be in strictly increasing order. Only used if the preset option is set to <code class="code">custom</code>. Bands are separated by white spaces and each band represent frequency in Hz. Default is <code class="code">25 40 63 100 160 250 400 630 1000 1600 2500 4000 6300 10000 16000 24000</code>. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is <code class="code">4096</code>. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate of output audio stream, default is <code class="code">44100</code>. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame in output audio stream. Default is <code class="code">1024</code>. </dd> <dt><samp class="option">interp, i</samp></dt> <dd> Set interpolation method for FIR equalizer coefficients. Can be <code class="code">linear</code> or <code class="code">cubic</code>. </dd> <dt><samp class="option">phase, h</samp></dt> <dd> Set phase type of FIR filter. Can be <code class="code">linear</code> or <code class="code">min</code>: minimum-phase. Default is minimum-phase filter. </dd> </dl>

 <h3 class="section">9.5 afirsrc# TOC</h3>

Generate a FIR coefficients using frequency sampling method. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is 1025. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set frequency points from where magnitude and phase are set. This must be in non decreasing order, and first element must be 0, while last element must be 1. Elements are separated by white spaces. </dd> <dt><samp class="option">magnitude, m</samp></dt> <dd> Set magnitude value for every frequency point set by <samp class="option">frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class="option">phase, p</samp></dt> <dd> Set phase value for every frequency point set by <samp class="option">frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp

class=”option”>win_func, w</samp></dt> <dd> Set window function. Default is blackman. </dd> </dl>

 <h3 class=”section”>9.6 anullsrc# TOC</h3>

The null audio source, return unprocessed audio frames. It is mainly useful as a template and to be employed in analysis / debugging tools, or as the source for filters which ignore the input data (for example the sox synth filter). This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>channel_layout, cl</samp></dt> <dd> Specifies the channel layout, and can be either an integer or a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". Check the channel_layout_map definition in <samp class=”file”>libavutil/channel_layout.c</samp> for the mapping between strings and channel layout values. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specifies the sample rate, and defaults to 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per requested frames. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> </dl>

 <h4 class=”subsection”>9.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the sample rate to 48000 Hz and the channel layout to AV_CH_LAYOUT_MONO. <pre class=”example-preformatted”>anullsrc=r=48000:cl=4 </pre>

Do the same operation with a more obvious syntax: <pre class=”example-preformatted”>anullsrc=r=48000:cl=mono </pre>

All the parameters need to be explicitly defined. <h3 class=”section”>9.7 flite# TOC</h3>

Synthesize a voice utterance using the libflite library. To enable compilation of this filter you need to configure FFmpeg with <code

class=”code”>--enable-libflite</code>. Note that versions of the flite library prior to 2.0 are not thread-safe. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>list_voices</samp></dt> <dd> If set to 1, list the names of the available voices and exit immediately. Default value is 0. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the maximum number of samples per frame. Default value is 512. </dd> <dt><samp class=”option”>textfile</samp></dt> <dd> Set the filename containing the text to speak. </dd> <dt><samp class=”option”>text</samp></dt> <dd> Set the text to speak. </dd> <dt><samp class=”option”>voice, v</samp></dt> <dd> Set the voice to use for the speech synthesis. Default value is <code class=”code”>kal</code>. See also the <var class=”var”>list_voices</var> option. </dd> </dl>

 <h4 class=”subsection”>9.7.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Read from file <samp class=”file”>speech.txt</samp>, and synthesize the text using the standard flite voice: <pre class=”example-preformatted”>flite=textfile=speech.txt </pre>

Read the specified text selecting the <code class=”code”>slt</code> voice: <pre class=”example-preformatted”>flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Input text to ffmpeg: <pre class=”example-preformatted”>ffmpeg -f lavfi -i flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Make <samp class=”file”>ffplay</samp> speak the specified text, using <code class=”code”>flite</code> and the <code class=”code”>lavfi</code> device: <pre class=”example-preformatted”>ffplay -f lavfi flite=text='No more be grieved for which that thou hast done.' </pre>

For more information about libflite, check: <a class=”url” href=”http://www.festvox.org/flite/”>http://w
  <h3 class=”section”>9.8 anoisesrc# TOC</h3>

Generate a noise audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specify the sample rate. Default value is 48000 Hz. </dd> <dt><samp class=”option”>amplitude, a</samp></dt>

<dd> Specify the amplitude (0.0 - 1.0) of the generated audio stream. Default value is 1.0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. Not specifying this option results in noise with an infinite length. </dd> <dt><samp class="option">color, colour, c</samp></dt> <dd> Specify the color of noise. Available noise colors are white, pink, brown, blue, violet and velvet. Default color is white. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Specify a value used to seed the PRNG. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per each output frame, default is 1024. </dd> <dt><samp class="option">density</samp></dt> <dd> Set the density (0.0 - 1.0) for the velvet noise generator, default is 0.05. </dd> </dl>

 <h4 class="subsection">9.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate 60 seconds of pink noise, with a 44.1 kHz sampling rate and an amplitude of 0.5: <pre class="example-preformatted">anoisesrc=d=60:c=pink:r=44100:a=0.5 </pre>

 <h3 class="section">9.9 hilbert# TOC</h3>

Generate odd-tap Hilbert transform FIR coefficients. The resulting stream can be used with afir filter for phase-shifting the signal by 90 degrees. This is used in many matrix coding schemes and for analytic signal generation. The process is often written as a multiplication by i (or j), the imaginary unit. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set length of FIR filter, default is 22051. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. </dd> <dt><samp class="option">win_func, w</samp></dt> <dd> Set window function to be used when generating FIR coefficients. </dd> </dl>

 <h3 class="section">9.10 sinc# TOC</h3>

Generate a sinc kaiser-windowed low-pass, high-pass, band-pass, or band-reject FIR coefficients. The resulting stream can be used with afir filter for filtering the audio

signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class="option">hp</samp></dt> <dd> Set high-pass frequency. Default is 0. </dd> <dt><samp class="option">lp</samp></dt> <dd> Set low-pass frequency. Default is 0. If high-pass frequency is lower than low-pass frequency and low-pass frequency is higher than 0 then filter will create band-pass filter coefficients, otherwise band-reject filter coefficients. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set filter phase response. Default is 50. Allowed range is from 0 to 100. </dd> <dt><samp class="option">beta</samp></dt> <dd> Set Kaiser window beta. </dd> <dt><samp class="option">att</samp></dt> <dd> Set stop-band attenuation. Default is 120dB, allowed range is from 40 to 180 dB. </dd> <dt><samp class="option">round</samp></dt> <dd> Enable rounding, by default is disabled. </dd> <dt><samp class="option">hptaps</samp></dt> <dd> Set number of taps for high-pass filter. </dd> <dt><samp class="option">lptaps</samp></dt> <dd> Set number of taps for low-pass filter. </dd> </dl>

 <h3 class="section">9.11 sine# TOC</h3>

Generate an audio signal made of a sine wave with amplitude 1/8. The audio signal is bit-exact. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the carrier frequency. Default is 440 Hz. </dd> <dt><samp class="option">beep_factor, b</samp></dt> <dd> Enable a periodic beep every second with frequency <var class="var">beep_factor</var> times the carrier frequency. Default is 0, meaning the beep is disabled. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate, default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. </dd> <dt><samp class="option">samples_per_frame</samp></dt> <dd> Set the number of samples per output frame. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the output audio frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the output audio frame, expressed in <var class="var">TB</var> units. </dd> <dt><samp class="option">t</samp></dt> <dd> The PTS of the output audio frame, expressed in seconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the output audio frames. </dd> </dl>

Default is <code class="code">1024</code>. </dd> </dl>

 <h4 class="subsection">9.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a simple 440 Hz sine wave: <pre class="example-preformatted">sine
</pre>

Generate a 220 Hz sine wave with a 880 Hz beep each second, for 5 seconds: <pre class="example-preformatted">sine=220:4:d=5 sine=f=220:b=4:d=5 sine=frequency=220:beep_factor=4:duration=5
</pre>

Generate a 1 kHz sine wave following <code class="code">1602,1601,1602,1601,1602</code> NTSC pattern: <pre class="example-preformatted">sine=1000:samples_per_frame=
1602-not(not(eq(ld(0),1)+eq(ld(0),3)))' </pre>

 <h2 class="chapter">10 Audio Sinks# TOC</h2>

Below is a description of the currently available audio sinks. <h3 class="section">10.1 abuffersink# TOC</h3>

Buffer audio frames, and make them available to the end of filter chain. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVABufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class="code">avfilter_init_filter</code> for initialization. <h3 class="section">10.2 anullsink# TOC</h3>

Null audio sink; do absolutely nothing with the input audio. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class="chapter">11 Video Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class="code">--disable-filters</code>. The configure output will show the video filters included in your build. Below is a description of the currently available video filters. <h3 class="section">11.1 addroi# TOC</h3>

Mark a region of interest in a video frame. The frame data is passed through unchanged, but metadata is attached to the frame indicating regions of interest which can affect the behaviour of later encoding. Multiple regions can be marked by applying the filter multiple times. <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Region distance in pixels from the left edge of the frame. </dd> <dt><samp class="option">y</samp></dt> <dd> Region distance in pixels from the top edge of the frame. </dd> <dt><samp class="option">w</samp></dt> <dd> Region width in pixels. </dd> <dt><samp class="option">h</samp></dt> <dd> Region height in pixels. The parameters <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> are expressions, and may contain the following variables: <dl class="table"> <dt><samp class="option">iw</samp></dt> <dd> Width of the input frame. </dd> <dt><samp class="option">ih</samp></dt> <dd> Height of the input frame. </dd> </dl>

</dd> <dt><samp class="option">qoffset</samp></dt> <dd> Quantisation offset to apply within the region. This must be a real value in the range -1 to +1. A value of zero indicates no quality change. A negative value asks for better quality (less quantisation), while a positive value asks for worse quality (greater quantisation). The range is calibrated so that the extreme values indicate the largest possible offset - if the rest of the frame is encoded with the worst possible quality, an offset of -1 indicates that this region should be encoded with the best possible quality anyway. Intermediate values are then interpolated in some codec-dependent way. For example, in 10-bit H.264 the quantisation parameter varies between -12 and 51. A typical qoffset value of -1/10 therefore indicates that this region should be encoded with a QP around one-tenth of the full range better than the rest of the frame. So, if most of the frame were to be encoded with a QP of around 30, this region would get a QP of around 24 (an offset of approximately -1/10 * (51 - -12) = -6.3). An extreme value of -1 would indicate that this region should be encoded with the best possible quality regardless of the treatment of the rest of the frame - that is, should be encoded at a QP of -12. </dd> <dt><samp class="option">clear</samp></dt> <dd> If set to true, remove any existing regions of interest marked on the frame before adding the new one. </dd> </dl>

 <h4 class="subsection">11.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Mark the centre quarter of the frame as interesting. <pre class="example-preformatted">addroi=iw/4:ih/4:iw/2:ih/2:-1/10 </pre> Mark the 100-pixel-wide region on the left edge of the frame as very uninteresting (to be encoded at much lower quality than the rest of the frame). <pre class="example-preformatted">addroi=0:0:100:ih:+1/5 </pre>

 <h3 class="section">11.2 alphaextract# TOC</h3>

Extract the alpha component from the input as a grayscale video. This is especially useful with the <var class="var">alphamerge</var> filter. <h3 class="section">11.3 alphamerge# TOC</h3>

Add or replace the alpha component of the primary input with the grayscale value of a second input. This is intended for use with <var class="var">alphaextract</var> to allow the transmission or storage of frame sequences that have alpha in a format that doesn’t support an alpha channel. For example, to reconstruct full frames from a normal YUV-encoded video and a separate video created with <var class="var">alphaextract</var>, you might use: <pre class="example-preformatted">movie=in_alpha.mkv [alpha]; [in][alpha] alphamerge [out] </pre>

 <h3 class="section">11.4 amplify# TOC</h3>

Amplify differences between current pixel and pixels of adjacent frames in same pixel location. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set frame radius. Default is 2. Allowed range is from 1 to 63. For example radius of 3 will instruct filter to calculate average of 7 frames. </dd> <dt><samp class="option">factor</samp></dt> <dd> Set factor to amplify difference. Default is 2. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold for difference amplification. Any difference greater or equal to this

value will not alter source pixel. Default is 10. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set tolerance for difference amplification. Any difference lower to this value will not alter source pixel. Default is 0. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">low</samp></dt> <dd> Set lower limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will decrease source pixel value. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will increase source pixel value. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">11.4.1 Commands# TOC</h4>

This filter supports the following commands that corresponds to option of same name: <dl class="table"> <dt><samp class="option">factor</samp></dt> <dt><samp class="option">threshold</samp></dt> <dt><samp class="option">tolerance</samp></dt> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dt><samp class="option">planes</samp></dt> </dl>

 <h3 class="section">11.5 ass# TOC</h3>

Same as the subtitles filter, except that it doesn’t require libavcodec and libavformat to work. On the other hand, it is limited to ASS (Advanced Substation Alpha) subtitles files. This filter accepts <samp class="option">filename</samp>/<samp class="option">f</samp>, <samp class="option">original_size</samp>, <samp class="option">fontsdir</samp>, and <samp class="option">alpha</samp> from the subtitles filter, plus the following options: <dl class="table"> <dt><samp class="option">script</samp></dt> <dd> The ASS script to render, given inline. Exactly one of <samp class="option">filename</samp> and <samp class="option">script</samp> must be set. </dd> <dt><samp class="option">shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class="samp">simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class="samp">complex</samp>’</dt>

<dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew, Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd> </dl>

The default is <code class="code">auto</code>. </dd> </dl>

 <h4 class="subsection">11.5.1 Commands# TOC</h4>

This filter supports the <samp class="option">script</samp> option as commands; the new script replaces the one being rendered. <h3 class="section">11.6 atadenoise# TOC</h3> Apply an Adaptive Temporal Averaging Denoiser to the video input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">0a</samp></dt> <dd> Set threshold A for 1st plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">0b</samp></dt> <dd> Set threshold B for 1st plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">1a</samp></dt> <dd> Set threshold A for 2nd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">1b</samp></dt> <dd> Set threshold B for 2nd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">2a</samp></dt> <dd> Set threshold A for 3rd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">2b</samp></dt> <dd> Set threshold B for 3rd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">3a</samp></dt> <dd> Set threshold A for 4th plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">3b</samp></dt> <dd> Set threshold B for 4th plane. Default is 0.04. Valid range is 0 to 5. Threshold A is designed to react on abrupt changes in the input signal and threshold B is designed to react on continuous changes in the input signal. </dd> <dt><samp class="option">s</samp></dt> <dd> Set number of frames filter will use for averaging. Default is 9. Must be odd number in range [5, 129]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set what planes of frame filter will use for averaging. Default is all. </dd> <dt><samp class="option">a</samp></dt> <dd> Set what variant of algorithm filter will use for averaging. Default is <code class="code">p</code> parallel. Alternatively can be set to <code class="code">s</code> serial. Parallel can be faster then serial, while other way around is never true. Parallel will abort

early on first change being greater then thresholds, while serial will continue processing other side of frames if they are equal or below thresholds. </dd> <dt><samp class=”option”>0s</samp></dt> <dt><samp class=”option”>1s</samp></dt> <dt><samp class=”option”>2s</samp></dt> <dt><samp class=”option”>3s</samp></dt> <dd> Set sigma for 1st plane, 2nd plane, 3rd plane or 4th plane. Default is 32767. Valid range is from 0 to 32767. This options controls weight for each pixel in radius defined by size. Default value means every pixel have same weight. Setting this option to 0 effectively disables filtering. </dd> </dl>

 <h4 class=”subsection”>11.6.1 Commands# TOC</h4> This filter supports same commands as options except option <code class=”code”>s</code>. The command accepts the same syntax of the corresponding option. <h3 class=”section”>11.7 avgblur# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size, if zero it will be same as <code class=”code”>sizeX</code>. Default is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>11.7.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>11.8 backgroundkey# TOC</h3>

Turns a static background into transparency. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Threshold for scene change detection. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the background. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set the blend amount for pixels that are not similar. </dd> </dl>

 <h4 class="subsection">11.8.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.9 bbox# TOC</h3>

Compute the bounding box for the non-black pixels in the input frame luma plane. This filter computes the bounding box containing all the pixels with a luma value greater than the minimum allowed value. The parameters describing the bounding box are printed on the filter log. The filter accepts the following option: <dl class="table"> <dt><samp class="option">min_val</samp></dt> <dd> Set the minimal luma value. Default is <code class="code">16</code>. </dd> </dl>

 <h4 class="subsection">11.9.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.10 bilateral# TOC</h3> Apply bilateral filter, spatial smoothing while preserving edges. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight. Allowed range is 0 to 512. Default is 0.1. </dd> <dt><samp class="option">sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate range weight. Allowed range is 0 to 1. Default is 0.1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">11.10.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.11 bitplanenoise# TOC</h3>

Show and measure bit plane noise. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>bitplane</samp></dt> <dd> Set which plane to analyze. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>filter</samp></dt> <dd> Filter out noisy pixels from <code class=”code”>bitplane</code> set above. Default is disabled. </dd> </dl>

 <h3 class=”section”>11.12 blackdetect, blackdetect_vulkan# TOC</h3>

Detect video intervals that are (almost) completely black. Can be useful to detect chapter transitions, commercials, or invalid recordings. The filter outputs its detection analysis to both the log as well as frame metadata. If a black segment of at least the specified minimum duration is found, a line with the start and end timestamps as well as duration is printed to the log with level <code class=”code”>info</code>. In addition, a log line with level <code class=”code”>debug</code> is printed per frame showing the black amount detected for that frame. The filter also attaches metadata to the first frame of a black segment with key <code class=”code”>lavfi.black_start</code> and to the first frame after the black segment ends with key <code class=”code”>lavfi.black_end</code>. The value is the frame’s timestamp. This metadata is added regardless of the minimum duration specified. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>black_min_duration, d</samp></dt> <dd> Set the minimum detected black duration expressed in seconds. It must be a non-negative floating point number. Default value is 2.0. </dd> <dt><samp class=”option”>picture_black_ratio_th, pic_th</samp></dt> <dd> Set the threshold for considering a picture "black". Express the minimum value for the ratio: <pre class=”example-preformatted”><var class=”var”>nb_black_pixels</var> / <var class=”var”>nb_pixels</var> </pre>

for which a picture is considered black. Default value is 0.98. </dd> <dt><samp class=”option”>pixel_black_th, pix_th</samp></dt> <dd> Set the threshold for considering a pixel "black". The threshold expresses the maximum pixel luma value for which a pixel is considered "black". The provided value is scaled according to the following equation: <pre class=”example-preformatted”><var class=”var”>absolute_threshold</var> = <var class=”var”>luma_minimum_value</var> + <var class=”var”>pixel_black_th</var> * <var class=”var”>luma_range_size</var> </pre>

<var class=”var”>luma_range_size</var> and <var class=”var”>luma_minimum_value</var> depend on the input video format, the range is [0-255] for YUV full-range

formats and [16-235] for YUV non full-range formats. Default value is 0.10. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If true, check the alpha channel instead of the luma channel. Detects frames which are (almost) transparent, instead of frames which are almost black. Default value is disabled. </dd> </dl>

The following example sets the maximum pixel threshold to the minimum value, and detects only black intervals of 2 or more seconds: <pre class="example-preformatted">blackdetect=d=2:pix_th=0.00 </pre>

 <h3 class="section">11.13 blackframe# TOC</h3>

Detect frames that are (almost) completely black. Can be useful to detect chapter transitions or commercials. Output lines consist of the frame number of the detected frame, the percentage of blackness, the position in the file if known or -1 and the timestamp in seconds. In order to display the output lines, you need to set the loglevel at least to the AV_LOG_INFO value. This filter exports frame metadata <code class="code">lavfi.blackframe.pblack</code>. The value represents the percentage of pixels in the picture that are below the threshold value. It accepts the following parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt> <dd> The percentage of the pixels that have to be below the threshold; it defaults to <code class="code">98</code>. </dd> <dt><samp class="option">threshold, thresh</samp></dt> <dd> The threshold below which a pixel value is considered black; it defaults to <code class="code">32</code>. </dd> </dl>

 <h3 class="section">11.14 blend# TOC</h3>

Blend two video frames into each other. The <code class="code">blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. The <code class="code">tblend</code> (time blend) filter takes two consecutive frames from one single stream, and outputs the result obtained by blending the new frame on top of the old frame. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">c0_mode</samp></dt> <dt><samp class="option">c1_mode</samp></dt> <dt><samp class="option">c2_mode</samp></dt> <dt><samp class="option">c3_mode</samp></dt>

<dt><samp class="option">all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class="var">all_mode</var>. Default value is <code class="code">normal</code>. Available values for component modes are: <dl class="table"> <dt>‘<samp class="samp">addition</samp>’</dt> <dt>‘<samp class="samp">and</samp>’</dt> <dt>‘<samp class="samp">average</samp>’</dt> <dt>‘<samp class="samp">bleach</samp>’</dt> <dt>‘<samp class="samp">burn</samp>’</dt> <dt>‘<samp class="samp">darken</samp>’</dt> <dt>‘<samp class="samp">difference</samp>’</dt> <dt>‘<samp class="samp">divide</samp>’</dt> <dt>‘<samp class="samp">dodge</samp>’</dt> <dt>‘<samp class="samp">exclusion</samp>’</dt> <dt>‘<samp class="samp">extremity</samp>’</dt> <dt>‘<samp class="samp">freeze</samp>’</dt> <dt>‘<samp class="samp">geometric</samp>’</dt> <dt>‘<samp class="samp">glow</samp>’</dt> <dt>‘<samp class="samp">grainextract</samp>’</dt> <dt>‘<samp class="samp">grainmerge</samp>’</dt> <dt>‘<samp class="samp">hardlight</samp>’</dt> <dt>‘<samp class="samp">hardmix</samp>’</dt> <dt>‘<samp class="samp">hardoverlay</samp>’</dt> <dt>‘<samp class="samp">harmonic</samp>’</dt> <dt>‘<samp class="samp">heat</samp>’</dt> <dt>‘<samp class="samp">interpolate</samp>’</dt> <dt>‘<samp class="samp">lighten</samp>’</dt> <dt>‘<samp class="samp">linearlight</samp>’</dt> <dt>‘<samp class="samp">multiply</samp>’</dt> <dt>‘<samp class="samp">multiply128</samp>’</dt> <dt>‘<samp class="samp">negation</samp>’</dt> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">or</samp>’</dt> <dt>‘<samp class="samp">overlay</samp>’</dt> <dt>‘<samp class="samp">phoenix</samp>’</dt> <dt>‘<samp class="samp">pinlight</samp>’</dt> <dt>‘<samp class="samp">reflect</samp>’</dt> <dt>‘<samp class="samp">screen</samp>’</dt> <dt>‘<samp class="samp">softdifference</samp>’</dt> <dt>‘<samp class="samp">softlight</samp>’</dt> <dt>‘<samp class="samp">stain</samp>’</dt> <dt>‘<samp class="samp">subtract</samp>’</dt> <dt>‘<samp class="samp">vividlight</samp>’</dt> <dt>‘<samp class="samp">xor</samp>’</dt> </dl>

</dd> <dt><samp class="option">c0_opacity</samp></dt> <dt><samp class="option">c1_opacity</samp></dt> <dt><samp class="option">c2_opacity</samp></dt> <dt><samp class="option">c3_opacity</samp></dt> <dt><samp class="option">all_opacity</samp></dt> <dd> Set blend opacity for specific pixel component or all pixel components in case of <var class="var">all_opacity</var>. Only used in combination with pixel component blend modes. </dd> <dt><samp class="option">c0_expr</samp></dt> <dt><samp class="option">c1_expr</samp></dt> <dt><samp class="option">c2_expr</samp></dt> <dt><samp class="option">c3_expr</samp></dt> <dt><samp class="option">all_expr</samp></dt> <dd> Set blend expression for specific pixel component or all pixel components in case of <var class="var">all_expr</var>. Note that

related mode options will be ignored if those are set. The expressions can use the following variables: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class="code">0</code>. </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> the coordinates of the current sample </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> the width and height of currently filtered plane </dd> <dt><samp class="option">SW</samp></dt> <dt><samp class="option">SH</samp></dt> <dd> Width and height scale for the plane being filtered. It is the ratio between the dimensions of the current plane to the luma plane, e.g. for a <code class="code">yuv420p</code> frame, the values are <code class="code">1,1</code> for the luma plane and <code class="code">0.5,0.5</code> for the chroma planes. </dd> <dt><samp class="option">T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class="option">TOP, A</samp></dt> <dd> Value of pixel component at current location for first video frame (top layer). </dd> <dt><samp class="option">BOTTOM, B</samp></dt> <dd> Value of pixel component at current location for second video frame (bottom layer). </dd> </dl> </dd> </dl>

The <code class="code">blend</code> filter also supports the framesync options. <h4 class="subsection">11.14.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply transition from bottom layer to top layer in first 10 seconds: <pre class="example-preformatted">blend=all_expr='A*(if(gte(T,10),1,T/10))+B*(1-(if(gte(T,10),1,T/10)))' </pre>

Apply linear horizontal transition from top layer to bottom layer: <pre class="example-preformatted">blend=all_expr='A*(X/W)+B*(1-X/W)' </pre>

Apply 1x1 checkerboard effect: <pre class="example-preformatted">blend=all_expr='if(eq(mod(X,2),mod(Y,2)),A,B)' </pre>

Apply uncover left effect: <pre class="example-preformatted">blend=all_expr='if(gte(N*SW+X,W),A,B)' </pre>

Apply uncover down effect: <pre class="example-preformatted">blend=all_expr='if(gte(Y-N*SH,0),A,B)' </pre>

Apply uncover up-left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(T*SH*40+Y,H)*gte((T*40*SW+X)*W/H,W),A,B)' </pre>

Split diagonally video and shows top and bottom layer on each side: <pre class=”example-preformatted”>blend=all_expr='if(gt(X,Y*(W/H)),A,B)' </pre>

Display differences between the current and the previous frame: <pre class=”example-preformatted”>tblend=all_mode=grainextract </pre>

 <h4 class=”subsection”>11.14.2 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>11.15 blockdetect# TOC</h3>

Determines blockiness of frames without altering the input frames. Based on Remco Muijs and Ihor Kirenko: "A no-reference blocking artifact measure for adaptive video processing." 2005 13th European signal processing conference. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>period_min</samp></dt> <dt><samp class=”option”>period_max</samp></dt> <dd> Set minimum and maximum values for determining pixel grids (periods). Default values are [3,24]. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>11.15.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Determine blockiness for the first plane and search for periods within [8,32]: <pre class=”example-preformatted”>blockdetect=period_min=8:period_max=32:planes=1 </pre>

 <h3 class=”section”>11.16 blurdetect# TOC</h3>

Determines blurriness of frames without altering the input frames. Based on Marziliano, Pina, et al. "A no-reference perceptual blur metric." Allows for a block-based abbreviation. The

filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">20/255</code>, and default value for <var class="var">high</var> is <code class="code">50/255</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Define the radius to search around an edge pixel for local maxima. </dd> <dt><samp class="option">block_pct</samp></dt> <dd> Determine blurriness only for the most significant blocks, given in percentage. </dd> <dt><samp class="option">block_width</samp></dt> <dd> Determine blurriness for blocks of width <var class="var">block_width</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class="var">block_height</var>. </dd> <dt><samp class="option">block_height</samp></dt> <dd> Determine blurriness for blocks of height <var class="var">block_height</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class="var">block_width</var>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">11.16.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Determine blur for 80% of most significant 32x32 blocks: <pre class="example-preformatted">blurdetect=block_width=32:block_height=32:block_pct=80
</pre>

 <h3 class="section">11.17 bm3d#
TOC</h3>

Denoise frames using Block-Matching 3D algorithm. The filter accepts the following options. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set denoising strength. Default value is 1. Allowed range is from 0 to 999.9. The denoising algorithm is very sensitive to sigma, so adjust it according to the source. </dd> <dt><samp class="option">block</samp></dt> <dd> Set local patch size. This sets dimensions in 2D. </dd> <dt><samp class="option">bstep</samp></dt> <dd> Set sliding step for pro-

cessing blocks. Default value is 4. Allowed range is from 1 to 64. Smaller values allows processing more reference blocks and is slower. </dd> <dt><samp class="option">group</samp></dt> <dd> Set maximal number of similar blocks for 3rd dimension. Default value is 1. When set to 1, no block matching is done. Larger values allows more blocks in single group. Allowed range is from 1 to 256. </dd> <dt><samp class="option">range</samp></dt> <dd> Set radius for search block matching. Default is 9. Allowed range is from 1 to INT32_MAX. </dd> <dt><samp class="option">mstep</samp></dt> <dd> Set step between two search locations for block matching. Default is 1. Allowed range is from 1 to 64. Smaller is slower. </dd> <dt><samp class="option">thmse</samp></dt> <dd> Set threshold of mean square error for block matching. Valid range is 0 to INT32_MAX. </dd> <dt><samp class="option">hdthr</samp></dt> <dd> Set thresholding parameter for hard thresholding in 3D transformed domain. Larger values results in stronger hard-thresholding filtering in frequency domain. </dd> <dt><samp class="option">estim</samp></dt> <dd> Set filtering estimation mode. Can be <code class="code">basic</code> or <code class="code">final</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">ref</samp></dt> <dd> If enabled, filter will use 2nd stream for block matching. Default is disabled for <code class="code">basic</code> value of <var class="var">estim</var> option, and always enabled if value of <var class="var">estim</var> is <code class="code">final</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is all available except alpha. </dd> </dl>

 <h4 class="subsection">11.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Basic filtering with bm3d: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=ba
</pre>

Same as above, but filtering only luma: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic:planes=1
</pre>

Same as above, but with both estimation modes:
<pre class="example-preformatted">split[a][b],[a]bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic[a],[b
</pre>

Same as above, but prefilter with nlmeans
filter instead: <pre class="example-preformatted">split[a][b],[a]nlmeans=s=3:r=7:p=
</pre>

 <h3 class="section">11.18 boxblur<span

class=”pull-right”># TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class=”code”>min(w,h)/2</code> for the luma and alpha planes, and of <code class=”code”>min(cw,ch)/2</code> for the chroma planes. Default value for <samp class=”option”>luma_radius</samp> is "2". If not specified, <samp class=”option”>chroma_radius</samp> and <samp class=”option”>alpha_radius</samp> default to the corresponding value set for <samp class=”option”>luma_radius</samp>. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class=”option”>cw</samp></dt> <dt><samp class=”option”>ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class=”option”>luma_power</samp> is 2. If not specified, <samp class=”option”>chroma_power</samp> and <samp class=”option”>alpha_power</samp> default to the corresponding value set for <samp class=”option”>luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class=”subsection”>11.18.1 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-50” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Apply a boxblur filter with the luma, chroma, and alpha radii set to 2: <pre class=”example-preformatted”>boxblur=luma_radius=2:luma_power=1
boxblur=2:1 </pre>

Set the luma radius to 2, and alpha and chroma radius to 0: <pre class=”example-preformatted”>boxblur=2:1:cr=0:ar=0
</pre>

Set the luma and chroma radii to a fraction of the video dimension:
 <pre class=”example-preformatted”>boxblur=luma_radius=min(h\,w)/10:luma_po
</pre>

 <h3 class=”section”>11.19 bwdif# TOC</h3>

Deinterlace the input video ("bwdif" stands for "Bob Weaver Deinterlacing Filter"). Motion adaptive deinterlacing based on yadif with the use of w3fdif and cubic interpolation algorithms. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>send_field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h3 class=”section”>11.20 ccrepack# TOC</h3>

Repack CEA-708 closed captioning side data This filter fixes various issues seen with commercial encoders related to upstream malformed CEA-708 payloads, specifically incorrect number of tuples (wrong cc_count for the target FPS), and incorrect ordering of tuples (i.e. the CEA-608 tuples are not at the first entries in the payload). <h3 class=”section”>11.21 cas# TOC</h3>

Apply Contrast Adaptive Sharpen filter to video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>strength</samp></dt> <dd> Set the sharpening strength. Default value is 0. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default value is to filter all planes except alpha plane. </dd> </dl>

 <h4 class=”subsection”>11.21.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>11.22 chromahold# TOC</h3> Remove all color information for all colors except for certain one. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dd> The color which will not be replaced with neutral chroma. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more preserved color. </dd> <dt><samp class=”option”>yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">11.22.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.23 chromakey# TOC</h3> YUV colorspace color/chroma keying. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will be replaced with transparency. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">11.23.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">11.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf chromakey=green out.png </pre>

Overlay a greenscreen-video on top of a static black background. <pre class="example-preformatted">ffmpeg -f lavfi -i color=c=black:s=1280x720 -i video.mp4 -shortest -filter_complex

"[1:v]chromakey=0x70de77:0.1:0.2[ckout];[0:v][ckout]overlay[out]"
-map "[out]" output.mkv </pre>

 <h3 class="section">11.24 chromanr# TOC</h3> Reduce chrominance noise. The filter accepts the following options: <dl class="table"> <dt><samp class="option">thres</samp></dt> <dd> Set threshold for averaging chrominance values. Sum of absolute difference of Y, U and V pixel components of current pixel and neighbour pixels lower than this threshold will be used in averaging. Luma component is left unchanged and is copied to output. Default value is 30. Allowed range is from 1 to 200. </dd> <dt><samp class="option">sizew</samp></dt> <dd> Set horizontal radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">sizeh</samp></dt> <dd> Set vertical radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">stepw</samp></dt> <dd> Set horizontal step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">steph</samp></dt> <dd> Set vertical step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">threy</samp></dt> <dd> Set Y threshold for averaging chrominance values. Set finer control for max allowed difference between Y components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threu</samp></dt> <dd> Set U threshold for averaging chrominance values. Set finer control for max allowed difference between U components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threv</samp></dt> <dd> Set V threshold for averaging chrominance values. Set finer control for max allowed difference between V components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">distance</samp></dt> <dd> Set distance type used in calculations. <dl class="table"> <dt>‘<samp class="samp">manhattan</samp>’</dt> <dd> Absolute difference. </dd> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dd> Difference squared. </dd> </dl>

Default distance type is manhattan. </dd> </dl>

 <h4 class="subsection">11.24.1 Commands# TOC</h4> This

filter supports same commands as options. The command accepts the same syntax of the corresponding option. <h3 class="section">11.25 chromashift# TOC</h3> Shift chroma pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cbh</samp></dt> <dd> Set amount to shift chroma-blue horizontally. </dd> <dt><samp class="option">cbv</samp></dt> <dd> Set amount to shift chroma-blue vertically. </dd> <dt><samp class="option">crh</samp></dt> <dd> Set amount to shift chroma-red horizontally. </dd> <dt><samp class="option">crv</samp></dt> <dd> Set amount to shift chroma-red vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">11.25.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.26 ciescope# TOC</h3>

Display CIE color diagram with pixels overlaid onto it. The filter accepts the following options: <dl class="table"> <dt><samp class="option">system</samp></dt> <dd> Set color system. <dl class="table"> <dt>‘<samp class="samp">ntsc, 470m</samp>’</dt> <dt>‘<samp class="samp">ebu, 470bg</samp>’</dt> <dt>‘<samp class="samp">smpte</samp>’</dt> <dt>‘<samp class="samp">240m</samp>’</dt> <dt>‘<samp class="samp">apple</samp>’</dt> <dt>‘<samp class="samp">widergb</samp>’<
<dt>‘<samp class="samp">cie1931</samp>’</dt> <dt>‘<samp class="samp">rec709, hdtv</samp>’</dt> <dt>‘<samp class="samp">uhdtv, rec2020</samp>’</dt> <dt>‘<samp class="samp">dcip3</samp>’</dt> </dl>

</dd> <dt><samp class="option">cie</samp></dt> <dd> Set CIE system. <dl class="table"> <dt>‘<samp class="samp">xyy</samp>’</dt> <dt>‘<samp class="samp">ucs</samp>’</dt> <dt>‘<samp class="samp">luv</samp>’</dt> </dl>

</dd> <dt><samp class="option">gamuts</samp></dt> <dd> Set

what gamuts to draw. See <code class=”code”>system</code> option for available values. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set ciescope size, by default set to 512. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity used to map input pixel values to CIE diagram. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Set contrast used to draw tongue colors that are out of active color system gamut. </dd> <dt><samp class=”option”>corrgamma</samp></dt> <dd> Correct gamma displayed on scope, by default enabled. </dd> <dt><samp class=”option”>showwhite</samp></dt> <dd> Show white point on CIE diagram, by default disabled. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set input gamma. Used only with XYZ input color space. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Fill with CIE colors. By default is enabled. </dd> </dl>

 <h3 class=”section”>11.27 codecview# TOC</h3>

Visualize information exported by some codecs. Some codecs can export information through frames using side-data or other means. For example, some MPEG based codecs export motion vectors through the <var class=”var”>export_mvs</var> flag in the codec <samp class=”option”>flags2</samp> option. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>block</samp></dt> <dd> Display block partition structure using the luma plane. </dd> <dt><samp class=”option”>mv</samp></dt> <dd> Set motion vectors to visualize. Available flags for <var class=”var”>mv</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>pf</samp>’</dt> <dd> forward predicted MVs of P-frames </dd> <dt>‘<samp class=”samp”>bf</samp>’</dt> <dd> forward predicted MVs of B-frames </dd> <dt>‘<samp class=”samp”>bb</samp>’</dt> <dd> backward predicted MVs of B-frames </dd> </dl>

</dd> <dt><samp class=”option”>qp</samp></dt> <dd> Display quantization parameters using the chroma planes. </dd> <dt><samp class=”option”>mv_type, mvt</samp></dt> <dd> Set motion vectors type to visualize. Includes MVs from all frames unless specified by <var class=”var”>frame_type</var> option. Available flags for <var class=”var”>mv_type</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>fp</samp>’</dt> <dd> forward predicted MVs </dd> <dt>‘<samp class=”samp”>bp</samp>’</dt> <dd> backward predicted MVs </dd> </dl>

</dd> <dt><samp class=”option”>frame_type, ft</samp></dt>

<dd> Set frame type to visualize motion vectors of. Available flags for <var class="var">frame_type</var> are: <dl class="table"> <dt>‘<samp class="samp">if</samp>’</dt> <dd> intra-coded frames (I-frames) </dd> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> predicted frames (P-frames) </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> bi-directionally predicted frames (B-frames) </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.27.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Visualize forward predicted MVs of all frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv_type=fp </pre>

Visualize multi-directionals MVs of P and B-Frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv=pf+bf+bb </pre>

 <h3 class="section">11.28 colorbalance# TOC</h3> Modify intensity of primary colors (red, green and blue) of input frames. The filter allows an input frame to be adjusted in the shadows, midtones or highlights regions for the red-cyan, green-magenta or blue-yellow balance. A positive adjustment value shifts the balance towards the primary color, a negative value towards the complementary color. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rs</samp></dt> <dt><samp class="option">gs</samp></dt> <dt><samp class="option">bs</samp></dt> <dd> Adjust red, green and blue shadows (darkest pixels). </dd> <dt><samp class="option">rm</samp></dt> <dt><samp class="option">gm</samp></dt> <dt><samp class="option">bm</samp></dt> <dd> Adjust red, green and blue midtones (medium pixels). </dd> <dt><samp class="option">rh</samp></dt> <dt><samp class="option">gh</samp></dt> <dt><samp class="option">bh</samp></dt> <dd> Adjust red, green and blue highlights (brightest pixels). Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">pl</samp></dt> <dd> Preserve lightness when changing color balance. Default is disabled. </dd> </dl>

 <h4 class="subsection">11.28.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Add red color cast to shadows: <pre class="example-preformatted">colorbalance=rs=.3 </pre>

 <h4 class="subsection">11.28.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.29 colorcontrast# TOC</h3>

Adjust color contrast between RGB components. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rc</samp></dt> <dd> Set the red-cyan contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">gm</samp></dt> <dd> Set the green-magenta contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">by</samp></dt> <dd> Set the blue-yellow contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">rcw</samp></dt> <dt><samp class="option">gmw</samp></dt> <dt><samp class="option">byw</samp></dt> <dd> Set the weight of each <code class="code">rc</code>, <code class="code">gm</code>, <code class="code">by</code> option value. Default value is 0.0. Allowed range is from 0.0 to 1.0. If all weights are 0.0 filtering is disabled. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Default value is 0.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">11.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.30 colorcorrect# TOC</h3>

Adjust color white balance selectively for blacks and whites. This filter operates in YUV colorspace. The filter accepts the following options:

 <dl class="table"> <dt><samp class="option">rl</samp></dt>
<dd> Set the red shadow spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">bl</samp></dt>
<dd> Set the blue shadow spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">rh</samp></dt>
<dd> Set the red highlight spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">bh</samp></dt>
<dd> Set the blue highlight spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the amount of saturation. Allowed range is from -3.0 to 3.0. De-
fault value is 1. </dd> <dt><samp class="option">analyze</samp></dt>
<dd> If set to anything other than <code class="code">manual</code>
it will analyze every frame and use derived parameters for filtering output frame.
 Possible values are: <dl class="table"> <dt>‘<samp
class="samp">manual</samp>’</dt> <dt>‘<samp class="samp">average</samp>’
<dt>‘<samp class="samp">minmax</samp>’</dt> <dt>‘<samp
class="samp">median</samp>’</dt> </dl> Default value is
<code class="code">manual</code>. </dd> </dl>

 <h4 class="subsection">11.30.1 Com-
mands<a class="anchor hidden-xs" href="#Commands-
69" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Commands-69" aria-hidden="true">TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">11.31
colorchannelmixer<a class="anchor hidden-xs"
href="#colorchannelmixer" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-colorchannelmixer" aria-hidden="true">TOC</h3>

Adjust video input frames by re-mixing color channels.
This filter modifies a color channel by adding the values associated
to the other channels of the same pixels. For example if the value to
modify is red, the output value will be:
<pre class="example-preformatted"><var class="var">red</var>=<var
class="var">red</var>*<var class="var">rr</var> + <var class="var">blue</var>*<var
class="var">rb</var> + <var class="var">green</var>*<var class="var">rg</var>
+ <var class="var">alpha</var>*<var class="var">ra</var> </pre>

The filter accepts the following options: <dl class="table">
<dt><samp class="option">rr</samp></dt> <dt><samp class="option">rg</samp></dt>
<dt><samp class="option">rb</samp></dt> <dt><samp class="option">ra</samp></dt>
<dd> Adjust contribution of input red, green, blue and alpha chan-
nels for output red channel. Default is <code class="code">1</code>
for <var class="var">rr</var>, and <code class="code">0</code>
for <var class="var">rg</var>, <var class="var">rb</var> and <var
class="var">ra</var>. </dd> <dt><samp class="option">gr</samp></dt>
<dt><samp class="option">gg</samp></dt> <dt><samp class="option">gb</samp></dt>

<dt><samp class="option">ga</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output green channel. Default is <code class="code">1</code> for <var class="var">gg</var>, and <code class="code">0</code> for <var class="var">gr</var>, <var class="var">gb</var> and <var class="var">ga</var>. </dd> <dt><samp class="option">br</samp></dt> <dt><samp class="option">bg</samp></dt> <dt><samp class="option">bb</samp></dt> <dt><samp class="option">ba</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output blue channel. Default is <code class="code">1</code> for <var class="var">bb</var>, and <code class="code">0</code> for <var class="var">br</var>, <var class="var">bg</var> and <var class="var">ba</var>. </dd> <dt><samp class="option">ar</samp></dt> <dt><samp class="option">ag</samp></dt> <dt><samp class="option">ab</samp></dt> <dt><samp class="option">aa</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output alpha channel. Default is <code class="code">1</code> for <var class="var">aa</var>, and <code class="code">0</code> for <var class="var">ar</var>, <var class="var">ag</var> and <var class="var">ab</var>. Allowed ranges for options are <code class="code">[-2.0, 2.0]</code>. </dd> <dt><samp class="option">pc</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl>

</dd> <dt><samp class="option">pa</samp></dt> <dd> Set the preserve color amount when changing colors. Allowed range is from <code class="code">[0.0, 1.0]</code>. Default is <code class="code">0.0</code>, thus disabled. </dd> </dl>

 <h4 class="subsection">11.31.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert source to grayscale: <pre class="example-preformatted">colorchannelmixer=.3:.4:.3:0:.3:.4:.3:0:.3:.4:.3 </pre> Simulate sepia tones: <pre class="example-preformatted">colorchannelmixer=.393:.769:.189:0:.349:.686:.168:0:.272:.534:.131

</pre>

 <h4 class="subsection">11.31.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.32 colordetect# TOC</h3> Analyze the video frames to determine the effective value range and alpha mode. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set of properties to detect. Unavailable properties, such as alpha mode for an input image without an alpha channel, will be ignored automatically. Accepts a combination of the following flags: <dl class="table"> <dt>‘<samp class="samp">color_range</samp>’</dt> <dd> Detect if the source contains luma pixels outside the limited (MPEG) range, which indicates that this is a full range YUV source. </dd> <dt>‘<samp class="samp">alpha_mode</samp>’</dt> <dd> Detect if the source contains color values above the alpha channel, which indicates that the alpha channel is independent (straight), rather than premultiplied. Also detects if the alpha plane is fully opaque or not. </dd> <dt>‘<samp class="samp">all</samp>’</dt> <dd> Enable detection of all of the above properties. This is the default. </dd> </dl>

</dd> <dt><samp class="option">threshold</samp></dt> <dd> Allow pixels to exceed the expected bounds by this fraction of the full scale value range before treating it as evidence of (respectively) full range or straight alpha. Value range is from 0 to 0.05. Defaults to 0.001 (=0.1%). </dd> </dl>

 <h3 class="section">11.33 colorize# TOC</h3> Overlay a solid color on the video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the color hue. Allowed range is from 0 to 360. Default value is 0. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the color saturation. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">lightness</samp></dt> <dd> Set the color lightness. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set the mix of source lightness. By default is set to 1.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">11.33.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.34 colorkey# TOC</h3> RGB colorspace color keying. This filter operates on 8-bit RGB format frames by setting the alpha component of each pixel which falls within the similarity radius of the key color to 0. The alpha value for pixels outside the similarity radius depends on the value of the blend option. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> Set the color for which alpha will be set to 0 (full transparency). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default is <code class="code">black</code>. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set the radius from the key color within which other colors also have full transparency. The computed distance is related to the unit fractional distance in 3D space between the RGB values of the key color and the pixel’s color. Range is 0.01 to 1.0. 0.01 matches within a very small radius around the exact key color, while 1.0 matches everything. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set how the alpha value for pixels that fall outside the similarity radius is computed. 0.0 makes pixels either fully transparent or fully opaque. Higher values result in semi-transparent pixels, with greater transparency the more similar the pixel color is to the key color. Range is 0.0 to 1.0. Default is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">11.34.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf colorkey=green out.png </pre>

Overlay a greenscreen-video on top of a static background image. <pre class="example-preformatted">ffmpeg -i background.png -i video.mp4 -filter_complex "[1:v]colorkey=0x3BBD1E:0.3:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.flv </pre>

 <h4 class="subsection">11.34.2 Com-

mands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.35 colorhold# TOC</h3> Remove all color information for all RGB colors except for certain one. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will not be replaced with neutral gray. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels fully gray. Higher values result in more preserved color. </dd> </dl>

 <h4 class="subsection">11.35.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.36 colorlevels# TOC</h3>

Adjust video input frames using levels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rimin</samp></dt> <dt><samp class="option">gimin</samp></dt> <dt><samp class="option">bimin</samp></dt> <dt><samp class="option">aimin</samp></dt> <dd> Adjust red, green, blue and alpha input black point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">rimax</samp></dt> <dt><samp class="option">gimax</samp></dt> <dt><samp class="option">bimax</samp></dt> <dt><samp class="option">aimax</samp></dt> <dd> Adjust red, green, blue and alpha input white point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">1</code>. Input levels are used to lighten highlights (bright tones), darken shadows (dark tones), change the balance of bright and dark tones. </dd> <dt><samp class="option">romin</samp></dt> <dt><samp class="option">gomin</samp></dt> <dt><samp class="option">bomin</samp></dt>

<dt><samp class="option">aomin</samp></dt> <dd> Adjust red, green, blue and alpha output black point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">romax</samp></dt> <dt><samp class="option">gomax</samp></dt> <dt><samp class="option">bomax</samp></dt> <dt><samp class="option">aomax</samp></dt> <dd> Adjust red, green, blue and alpha output white point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">1</code>. Output levels allows manual selection of a constrained output level range. </dd> <dt><samp class="option">preserve</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.36.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make video output darker: <pre class="example-preformatted">colorlevels=rimin=0.058:gimin=0.058:bimin=0.058 </pre>

Increase contrast: <pre class="example-preformatted">colorlevels=rimin=0.039:gimin=0.039:bimin=0.039:rimax=0.96:gimax=0.96:bimax=0.96 </pre>

Make video output lighter: <pre class="example-preformatted">colorlevels=rimax=0.902:gimax=0.902:bimax=0.902 </pre>

Increase brightness: <pre class="example-preformatted">colorlevels=romin=0.5:gomin=0.5:bomin=0.5 </pre>

 <h4 class="subsection">11.36.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.37 colormap# TOC</h3> Apply custom color maps to video stream. This filter needs three input video streams. First stream is video stream that is going to be filtered out. Second and third video stream specify color patches for source color to target color mapping. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>patch_size</samp></dt> <dd> Set the source and target video stream patch size in pixels. </dd> <dt><samp class=”option”>nb_patches</samp></dt> <dd> Set the max number of used patches from source and target video stream. Default value is number of patches available in additional video streams. Max allowed number of patches is <code class=”code”>64</code>. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set the adjustments used for target colors. Can be <code class=”code”>relative</code> or <code class=”code”>absolute</code>. Defaults is <code class=”code”>absolute</code>. </dd> <dt><samp class=”option”>kernel</samp></dt> <dd> Set the kernel used to measure color differences between mapped colors. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>euclidean</samp>’</dt> <dt>‘<samp class=”samp”>weuclidean</samp>’</dt> </dl>

Default is <code class=”code”>euclidean</code>. </dd> </dl>

 <h3 class=”section”>11.38 colormatrix# TOC</h3>

Convert color matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>src</samp></dt> <dt><samp class=”option”>dst</samp></dt> <dd> Specify the source and destination color matrix. Both values must be specified. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dd> BT.470 </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl> </dd> </dl>

For example to convert from BT.601 to SMPTE-240M, use the command: <pre class=”example-preformatted”>colormatrix=bt601:smpte240m

</pre>

 <h3 class="section">11.39 colorspace# TOC</h3>

Convert colorspace, transfer characteristics or color primaries. Input video needs to have an even size. The filter accepts the following options: <dl class="table"> <dd></dd> <dt><samp class="option">all</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class="samp">bt601-6-525</samp>’</dt> <dd> BT.601-6 525 </dd> <dt>‘<samp class="samp">bt601-6-625</samp>’</dt> <dd> BT.601-6 625 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">space</samp></dt> <dd> Specify output colorspace. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">ycgco</samp>’</dt> <dd> YCgCo </dd> <dt>‘<samp class="samp">bt2020ncl</samp>’</dt> <dd> BT.2020 with non-constant luminance </dd> </dl>

</dd> <dt><samp class="option">trc</samp></dt> <dd> Specify output transfer characteristics. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dd> Constant gamma of 2.2 </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dd> Constant gamma of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M, BT.601-6 625 or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">srgb</samp>’</dt> <dd> SRGB </dd> <dt>‘<samp class="samp">iec61966-2-

1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4 </dd> <dt>‘<samp class="samp">xvycc</samp>’</dt> <dd> xvycc </dd> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for 10-bits content </dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dd> BT.2020 for 12-bits content </dd> </dl>

</dd> <dt><samp class="option">primaries</samp></dt> <dd> Specify output color primaries. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> film </dd> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl>

</dd> <dt><samp class="option">range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class="samp">mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class="samp">jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> <dt><samp class="option">format</samp></dt> <dd> Specify output color format. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">yuv420p</samp>’</dt> <dd> YUV 4:2:0 planar 8-bits </dd> <dt>‘<samp class="samp">yuv420p10</samp>’</dt> <dd> YUV 4:2:0 planar 10-bits </dd> <dt>‘<samp class="samp">yuv420p12</samp>’</dt> <dd> YUV 4:2:0 planar 12-bits </dd> <dt>‘<samp class="samp">yuv422p</samp>’</dt> <dd> YUV 4:2:2 planar 8-bits </dd> <dt>‘<samp class="samp">yuv422p10</samp>’</dt> <dd> YUV 4:2:2 planar 10-bits </dd> <dt>‘<samp class="samp">yuv422p12</samp>’</dt> <dd> YUV 4:2:2 planar 12-bits </dd> <dt>‘<samp class="samp">yuv444p</samp>’</dt> <dd> YUV 4:4:4 planar 8-bits </dd> <dt>‘<samp class="samp">yuv444p10</samp>’</dt> <dd> YUV 4:4:4 pla-

nar 10-bits </dd> <dt>‘<samp class="samp">yuv444p12</samp>’</dt>
<dd> YUV 4:4:4 planar 12-bits </dd> </dl>

</dd> <dt><samp class="option">fast</samp></dt> <dd> Do a fast conversion, which skips gamma/primary correction. This will take significantly less CPU, but will be mathematically incorrect. To get output compatible with that produced by the colormatrix filter, use fast=1. </dd> <dt><samp class="option">dither</samp></dt> <dd> Specify dithering mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No dithering </dd> <dt>‘<samp class="samp">fsb</samp>’</dt> <dd> Floyd-Steinberg dithering </dd> </dl>

</dd> <dt><samp class="option">wpadapt</samp></dt> <dd> Whitepoint adaptation mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bradford</samp>’</dt> <dd> Bradford whitepoint adaptation </dd> <dt>‘<samp class="samp">vonkries</samp>’</dt> <dd> von Kries white-point adaptation </dd> <dt>‘<samp class="samp">identity</samp>’</dt> <dd> identity whitepoint adaptation (i.e. no whitepoint adaptation) </dd> </dl>

</dd> <dt><samp class="option">clipgamut</samp></dt> <dd> Controls how to clip out-of-gamut colors that arise as a result of colorspace conversion. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No clipping of out of gamut colors. </dd> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> Clips the RGB values to the [0, 1] range when building the gamma transfer LUTs. </dd> </dl>

</dd> <dt><samp class="option">iall</samp></dt> <dd> Override all input properties at once. Same accepted values as all. </dd> <dt><samp class="option">ispace</samp></dt> <dd> Override input colorspace. Same accepted values as space. </dd> <dt><samp class="option">iprimaries</samp></dt> <dd> Override input color primaries. Same accepted values as primaries. </dd> <dt><samp class="option">itrc</samp></dt> <dd> Override input transfer characteristics. Same accepted values as trc. </dd> <dt><samp class="option">irange</samp></dt> <dd> Override input color range. Same accepted values as range. </dd> </dl>

The filter converts the transfer characteristics, color space and color primaries to the specified user values. The output value, if not specified, is set to a default value based on the "all" property. If that property is also not specified, the filter will log an error. The output color range and for-

mat default to the same value as the input color range and format. The input transfer characteristics, color space, color primaries and color range should be set on the input data. If any of these are missing, the filter will log an error and no conversion will take place. For example to convert the input to SMPTE-240M, use the command: <pre class="example-preformatted">colorspace=smpte240m </pre>

 <h3 class="section">11.40 colortemperature# TOC</h3>
Adjust color temperature in video to simulate variations in ambient color temperature. The filter accepts the following options: <dl class="table"> <dt><samp class="option">temperature</samp></dt>
<dd> Set the temperature in Kelvin. Allowed range is from 1000 to 40000. Default value is 6500 K. </dd> <dt><samp class="option">mix</samp></dt>
<dd> Set mixing with filtered output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">pl</samp></dt>
<dd> Set the amount of preserving lightness. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">11.40.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">11.41 convolution# TOC</h3>

Apply convolution of 3x3, 5x5, 7x7 or horizontal/vertical up to 49 elements. The filter accepts the following options: <dl class="table"> <dt><samp class="option">0m</samp></dt> <dt><samp class="option">1m</samp></dt> <dt><samp class="option">2m</samp></dt>
<dt><samp class="option">3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed integers in <var class="var">square</var> mode, and from 1 to 49 odd number of signed integers in <var class="var">row</var> mode.
</dd> <dt><samp class="option">0rdiv</samp></dt> <dt><samp class="option">1rdiv</samp></dt> <dt><samp class="option">2rdiv</samp></dt>
<dt><samp class="option">3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be 1/sum of all matrix elements. </dd> <dt><samp class="option">0bias</samp></dt>
<dt><samp class="option">1bias</samp></dt> <dt><samp class="option">2bias</samp></dt>
<dt><samp class="option">3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful

for making the overall image brighter or darker. Default is 0.0. </dd> <dt><samp class="option">0mode</samp></dt> <dt><samp class="option">1mode</samp></dt> <dt><samp class="option">2mode</samp></dt> <dt><samp class="option">3mode</samp></dt> <dd> Set matrix mode for each plane. Can be <var class="var">square</var>, <var class="var">row</var> or <var class="var">column</var>. Default is <var class="var">square</var>. </dd> </dl>

 <h4 class="subsection">11.41.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">11.41.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply sharpen: <pre class="example-preformatted">convolution="0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0" </pre>

Apply blur: <pre class="example-preformatted">convolution="1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9" </pre>

Apply edge enhance: <pre class="example-preformatted">convolution="0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128" </pre>

Apply edge detect: <pre class="example-preformatted">convolution="0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128" </pre>

Apply laplacian edge detector which includes diagonals: <pre class="example-preformatted">convolution="1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0" </pre>

Apply emboss: <pre class="example-preformatted">convolution="-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2" </pre>

 <h3 class="section">11.42 convolve# TOC</h3>

Apply 2D convolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options:

<dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">convolve</code> filter also supports the framesync options. <h3 class="section">11.43 copy# TOC</h3>

Copy the input video source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">11.44 coreimage# TOC</h3> Video filtering on GPU using Apple’s CoreImage API on OSX. Hardware acceleration is based on an OpenGL context. Usually, this means it is processed by video hardware. However, software-based OpenGL implementations exist which means there is no guarantee for hardware processing. It depends on the respective OSX. There are many filters and image generators provided by Apple that come with a large variety of options. The filter has to be referenced by its name along with its options. The coreimage filter accepts the following options: <dl class="table"> <dt><samp class="option">list_filters</samp></dt> <dd> List all available filters and generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class="example-preformatted">list_filters=true </pre>

</dd> <dt><samp class="option">filter</samp></dt> <dd> Specify all filters by their respective name and options. Use <var class="var">list_filters</var> to determine all valid filter names and options. Numerical options are specified by a float value and are automatically clamped to their respective value range. Vector and color options have to be specified by a list of space separated float values. Character escaping has to be done. A special option name <code class="code">default</code> is available to use default options for a filter. It is required to specify either <code class="code">default</code> or at least one of the filter options. All omitted options are used with their default values. The syntax of the filter string is as follows: <pre class="example-preformatted">filter=<NAME>@<OPTION>=<VALUE>[@<OPTION>=<VALUE&
</pre>

</dd> <dt><samp class="option">output_rect</samp></dt> <dd> Specify

a rectangle where the output of the filter chain is copied into the input image. It is given by a list of space separated float values: <pre class="example-preformatted">output_rect=x\ y\ width\ height </pre> If not given, the output rectangle equals the dimensions of the input image. The output rectangle is automatically cropped at the borders of the input image. Negative values are valid for each component. <pre class="example-preformatted">output_rect=25\ 25\ 100\ 100 </pre> </dd> </dl>

Several filters can be chained for successive processing without GPU-HOST transfers allowing for fast processing of complex filter chains. Currently, only filters with zero (generators) or exactly one (filters) input image and one output image are supported. Also, transition filters are not yet usable as intended. Some filters generate output images with additional padding depending on the respective filter kernel. The padding is automatically removed to ensure the filter output has the same size as the input image. For image generators, the size of the output image is determined by the previous output image of the filter chain or the input image of the whole filterchain, respectively. The generators do not use the pixel information of this image to generate their output. However, the generated output is blended onto this image, resulting in partial or complete coverage of the output image. The coreimagesrc video source can be used for generating input images which are directly fed into the filter chain. By using it, providing input images by another video source or an input video is not required. <h4 class="subsection">11.44.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all filters available: <pre class="example-preformatted">coreimage=list_filters=true </pre>

Use the CIBoxBlur filter with default options to blur an image: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default </pre>

Use a filter chain with CISepiaTone at default values and CIVignetteEffect with its center at 100x100 and a radius of 50 pixels: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default#CIVignetteEf
100@inputRadius=50 </pre>

Use nullsrc and CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class="example-preformatted">ffmpeg -f lavfi -i nullsrc=s=100x100,coreimage=filter=CIQRCodeGenerator@inputMessage=ht
-frames:v 1 QRCode.png </pre>

 <h3 class="section">11.45 corr# TOC</h3>

Obtain the correlation between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max correlation is printed through the logging system. The filter stores the calculated correlation of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi corr -f null - </pre>

 <h3 class="section">11.46 cover_rect# TOC</h3>

Cover a rectangular object It accepts the following options: <dl class="table"> <dt><samp class="option">cover</samp></dt> <dd> Filepath of the optional cover image, needs to be in yuv420. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set covering mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> cover it by the supplied image </dd> <dt>‘<samp class="samp">blur</samp>’</dt> <dd> cover it by interpolating the surrounding pixels </dd> </dl>

Default value is <var class="var">blur</var>. </dd> </dl>

 <h4 class="subsection">11.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

 <h3 class="section">11.47 crop# TOC</h3>

Crop the input video to given dimensions. It accepts the following parameters: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dd> The width of the output video. It defaults to <code class="code">iw</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">w</samp>’ or ‘<samp class="samp">out_w</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">h, out_h</samp></dt> <dd> The height of the output video. It defaults to <code class="code">ih</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">h</samp>’ or ‘<samp class="samp">out_h</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">x</samp></dt> <dd> The horizontal position, in the input video, of the left edge of the output video. It defaults to <code class="code">(in_w-out_w)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">y</samp></dt> <dd> The vertical position, in the input video, of the top edge of the output video. It defaults to <code class="code">(in_h-out_h)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">keep_aspect</samp></dt> <dd> If set to 1 will force the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. It defaults to 0. </dd> <dt><samp class="option">exact</samp></dt> <dd> Enable exact cropping. If enabled, subsampled videos will be cropped at exact width/height/x/y as specified and will not be rounded to nearest smaller value. It defaults to 0. </dd> </dl>

The <var class="var">out_w</var>, <var class="var">out_h</var>, <var class="var">x</var>, <var class="var">y</var> parameters are expressions containing the following constants: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The computed values for <var class="var">x</var> and <var class="var">y</var>. They are evaluated for each new frame. </dd> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output (cropped) width and height. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample

aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class="option">pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class="option">t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

The expression for <var class="var">out_w</var> may depend on the value of <var class="var">out_h</var>, and the expression for <var class="var">out_h</var> may depend on <var class="var">out_w</var>, but they cannot depend on <var class="var">x</var> and <var class="var">y</var>, as <var class="var">x</var> and <var class="var">y</var> are evaluated after <var class="var">out_w</var> and <var class="var">out_h</var>. The <var class="var">x</var> and <var class="var">y</var> parameters specify the expressions for the position of the top-left corner of the output (non-cropped) area. They are evaluated for each frame. If the evaluated value is not valid, it is approximated to the nearest valid value. The expression for <var class="var">x</var> may depend on <var class="var">y</var>, and the expression for <var class="var">y</var> may depend on <var class="var">x</var>. <h4 class="subsection">11.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Crop area with size 100x100 at position (12,34). <pre class="example-preformatted">crop=100:100:12:34 </pre>

Using named options, the example above becomes: <pre class="example-preformatted">crop=w=100:h=100:x=12:y=34 </pre>

Crop the central input area with size 100x100: <pre class="example-preformatted">crop=100:100 </pre>

Crop the central input area with size 2/3 of the input video: <pre class="example-preformatted">crop=2/3*in_w:2/3*in_h </pre>

Crop the input video central square: <pre

class=”example-preformatted”>crop=out_w=in_h crop=in_h </pre>

Delimit the rectangle with the top-left corner placed at position 100:100 and the right-bottom corner corresponding to the right-bottom corner of the input image. <pre class=”example-preformatted”>crop=in_w-100:in_h-100:100:100 </pre>

Crop 10 pixels from the left and right borders, and 20 pixels from the top and bottom borders <pre class=”example-preformatted”>crop=in_w-2*10:in_h-2*20 </pre>

Keep only the bottom right quarter of the input image: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:in_w/2:in_h/2 </pre>

Crop height for getting Greek harmony: <pre class=”example-preformatted”>crop=in_w:1/PHI*in_w </pre>

Apply trembling effect: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(n/10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(n/7) </pre>

Apply erratic camera effect depending on timestamp: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(t*10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(t*13) </pre>

Set x depending on the value of y: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:y:10+10*sin(n/10) </pre>

 <h4 class=”subsection”>11.47.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>w, out_w</samp></dt> <dt><samp class=”option”>h, out_h</samp></dt> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set width/height of the output video and the horizontal/vertical position in the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>11.48 cropdetect# TOC</h3>

Auto-detect the crop size. It calculates the necessary

cropping parameters and prints the recommended parameters via the logging system. The detected dimensions correspond to the non-black or video area of the input video according to <var class=”var”>mode</var>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Depending on <var class=”var”>mode</var> crop detection is based on either the mere black value of surrounding pixels or a combination of motion vectors and edge pixels. <dl class=”table”> <dt>‘<samp class=”samp”>black</samp>’</dt> <dd> Detect black pixels surrounding the playing video. For fine control use option <var class=”var”>limit</var>. </dd> <dt>‘<samp class=”samp”>mvedges</samp>’</dt> <dd> Detect the playing video by the motion vectors inside the video and scanning for edge pixels typically forming the border of a playing video. </dd> </dl>

</dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set higher black value threshold, which can be optionally specified from nothing (0) to everything (255 for 8-bit based formats). An intensity value greater to the set value is considered non-black. It defaults to 24. You can also specify a value between 0.0 and 1.0 which will be scaled depending on the bitdepth of the pixel format. </dd> <dt><samp class=”option”>round</samp></dt> <dd> The value which the width/height should be divisible by. It defaults to 16. The offset is automatically adjusted to center the video. Use 2 to get only even dimensions (needed for 4:2:2 video). 16 is best when encoding to most video codecs. </dd> <dt><samp class=”option”>skip</samp></dt> <dd> Set the number of initial frames for which evaluation is skipped. Default is 2. Range is 0 to INT_MAX. </dd> <dt><samp class=”option”>reset_count, reset</samp></dt> <dd> Set the counter that determines after how many frames cropdetect will reset the previously detected largest video area and start over to detect the current optimal crop area. Default value is 0. This can be useful when channel logos distort the video area. 0 indicates ’never reset’, and returns the largest area encountered during playback. </dd> <dt><samp class=”option”>mv_threshold</samp></dt> <dd> Set motion in pixel units as threshold for motion detection. It defaults to 8. </dd> <dt><samp class=”option”>low</samp></dt> <dt><samp class=”option”>high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class=”var”>low</var> and <var class=”var”>high</var> threshold values must be chosen in the range [0,1], and <var class=”var”>low</var> should be lesser or equal to <var class=”var”>high</var>. Default value for <var class=”var”>low</var> is <code class=”code”>5/255</code>, and default value for <var class=”var”>high</var> is <code class=”code”>15/255</code>.

</dd> </dl>

 <h4 class="subsection">11.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Find video area surrounded by black borders: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf cropdetect,metadata=mode=print -f null - </pre>

Find an embedded video area, generate motion vectors beforehand: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf mestimate,cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

Find an embedded video area, use motion vectors from decoder: <pre class="example-preformatted">ffmpeg -flags2 +export_mvs -i file.mp4 -vf cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

 <h4 class="subsection">11.48.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">limit</samp></dt> <dd> The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">11.49 cue# TOC</h3>

Delay video filtering until a given wallclock timestamp. The filter first passes on <samp class="option">preroll</samp> amount of frames, then it buffers at most <samp class="option">buffer</samp> amount of frames and waits for the cue. After reaching the cue it forwards the buffered frames and also any subsequent frames coming in its input. The filter can be used synchronize the output of multiple ffmpeg processes for realtime output devices like decklink. By putting the delay in the filtering chain and pre-buffering frames the process can pass on data to output almost immediately after the target wallclock timestamp is reached. Perfect frame accuracy cannot be guaranteed, but the result is good enough for some use cases. <dl class="table"> <dt><samp class="option">cue</samp></dt> <dd> The cue timestamp expressed in a UNIX timestamp in microseconds. Default is 0. </dd> <dt><samp class="option">preroll</samp></dt> <dd> The duration of content

to pass on as preroll expressed in seconds. Default is 0. </dd> <dt><samp class="option">buffer</samp></dt> <dd> The maximum duration of content to buffer before waiting for the cue expressed in seconds. Default is 0. </dd> </dl>

 <h3 class="section">11.50 curves# TOC</h3>

Apply color adjustments using curves. This filter is similar to the Adobe Photoshop and GIMP curves tools. Each component (red, green and blue) has its values defined by <var class="var">N</var> key points tied from each other using a smooth curve. The x-axis represents the pixel values from the input frame, and the y-axis the new pixel values to be set for the output frame. By default, a component curve is defined by the two points <var class="var">(0;0)</var> and <var class="var">(1;1)</var>. This creates a straight line where each original pixel value is "adjusted" to its own value, which means no change to the image. The filter allows you to redefine these two points and add some more. A new curve will be defined to pass smoothly through all these new coordinates. The new defined points need to be strictly increasing over the x-axis, and their <var class="var">x</var> and <var class="var">y</var> values must be in the <var class="var">[0;1]</var> interval. The curve is formed by using a natural or monotonic cubic spline interpolation, depending on the <var class="var">interp</var> option (default: <code class="code">natural</code>). The <code class="code">natural</code> spline produces a smoother curve in general while the monotonic (<code class="code">pchip</code>) spline guarantees the transitions between the specified points to be monotonic. If the computed curves happened to go outside the vector spaces, the values will be clipped accordingly. The filter accepts the following options: <dl class="table"> <dt><samp class="option">preset</samp></dt> <dd> Select one of the available color presets. This option can be used in addition to the <samp class="option">r</samp>, <samp class="option">g</samp>, <samp class="option">b</samp> parameters; in this case, the later options takes priority on the preset values. Available presets are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">color_negative</samp>&rs
<dt>‘<samp class="samp">cross_process</samp>’</dt>
<dt>‘<samp class="samp">darker</samp>’</dt> <dt>‘<samp class="samp">increase_contrast</samp>’</dt> <dt>‘<samp class="samp">lighter</samp>’</dt> <dt>‘<samp class="samp">linear_contrast</samp>&
<dt>‘<samp class="samp">medium_contrast</samp>’</dt>
<dt>‘<samp class="samp">negative</samp>’</dt> <dt>‘<samp class="samp">strong_contrast</samp>’</dt> <dt>‘<samp class="samp">vintage</samp>’</dt> </dl> Default is <code

class=”code”>none</code>. </dd> <dt><samp class=”option”>master, m</samp></dt> <dd> Set the master key points. These points will define a second pass mapping. It is sometimes called a "luminance" or "value" mapping. It can be used with <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> or <samp class=”option”>all</samp> since it acts like a post-processing LUT. </dd> <dt><samp class=”option”>red, r</samp></dt> <dd> Set the key points for the red component. </dd> <dt><samp class=”option”>green, g</samp></dt> <dd> Set the key points for the green component. </dd> <dt><samp class=”option”>blue, b</samp></dt> <dd> Set the key points for the blue component. </dd> <dt><samp class=”option”>all</samp></dt> <dd> Set the key points for all components (not including master). Can be used in addition to the other key points component options. In this case, the unset component(s) will fallback on this <samp class=”option”>all</samp> setting. </dd> <dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop curves file (<code class=”code”>.acv</code>) to import the settings from. </dd> <dt><samp class=”option”>plot</samp></dt> <dd> Save Gnuplot script of the curves in specified file. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the kind of interpolation. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>natural</samp>’</dt> <dd> Natural cubic spline using a piece-wise cubic polynomial that is twice continuously differentiable. </dd> <dt>‘<samp class=”samp”>pchip</samp>’</dt> <dd> Monotonic cubic spline using a piecewise cubic Hermite interpolating polynomial (PCHIP). </dd> </dl>

</dd> </dl>

To avoid some filtergraph syntax conflicts, each key points list need to be defined using the following syntax: <code class=”code”>x0/y0 x1/y1 x2/y2 ...</code>. <h4 class=”subsection”>11.50.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>11.50.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Increase slightly the middle level of blue: <pre class=”example-preformatted”>curves=blue='0/0 0.5/0.58 1/1' </pre>

Vintage effect: <pre class=”example-

preformatted”>curves=r='0/0.11 .42/.51 1/0.95':g='0/0 0.50/0.48 1/1':b='0/0.22 .49/.44 1/0.8' </pre> Here we obtain the following coordinates for each components: <dl class=”table”> <dt><var class=”var”>red</var></dt> <dd> <code class=”code”>(0;0.11) (0.42;0.51) (1;0.95)</code> </dd> <dt><var class=”var”>green</var></dt> <dd> <code class=”code”>(0;0) (0.50;0.48) (1;1)</code> </dd> <dt><var class=”var”>blue</var></dt> <dd> <code class=”code”>(0;0.22) (0.49;0.44) (1;0.80)</code> </dd> </dl>

The previous example can also be achieved with the associated built-in preset: <pre class=”example-preformatted”>curves=preset=vintage </pre>

Or simply: <pre class=”example-preformatted”>curves=vintage </pre>

Use a Photoshop preset and redefine the points of the green component: <pre class=”example-preformatted”>curves=psfile='MyCurvesPresets/purple.a 0.45/0.53 1/1' </pre>

Check out the curves of the <code class=”code”>cross_process</code> profile using <code class=”command”>ffmpeg</code> and <code class=”command”>gnuplot</code>: <pre class=”example-preformatted”>ffmpeg -f lavfi -i color -vf curves=cross_process:plot=/tmp/curves.plt -frames:v 1 -f null - gnuplot -p /tmp/curves.plt </pre>

 <h3 class=”section”>11.51 datascope# TOC</h3>

Video data analysis filter. This filter shows hexadecimal pixel values of part of video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set output video size. </dd> <dt><samp class=”option”>x</samp></dt> <dd> Set x offset from where to pick pixels. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set y offset from where to pick pixels. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set scope mode, can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> Draw hexadecimal pixel values with white color on black background. </dd> <dt>‘<samp class=”samp”>color</samp>’</dt> <dd> Draw hexadecimal pixel values with input video pixel color on black background. </dd> <dt>‘<samp class=”samp”>color2</samp>’</dt> <dd> Draw hexadecimal pixel values on color background picked from input video, the text color is picked in such way so its always visible. </dd> </dl>

</dd> <dt><samp class="option">axis</samp></dt> <dd> Draw rows and columns numbers on left and top of video. </dd> <dt><samp class="option">opacity</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">format</samp></dt> <dd> Set display number format. Can be <code class="code">hex</code>, or <code class="code">dec</code>. Default is <code class="code">hex</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set pixel components to display. By default all pixel components are displayed. </dd> </dl>

 <h4 class="subsection">11.51.1 Commands# TOC</h4>

This filter supports same commands as options excluding <code class="code">size</code> option. <h3 class="section">11.52 dblur# TOC</h3> Apply Directional blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> Set angle of directional blur. Default is <code class="code">45</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Set radius of directional blur. Default is <code class="code">5</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> </dl>

 <h4 class="subsection">11.52.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.53 dctdnoiz# TOC</h3>

Denoise frames using 2D DCT (frequency domain filtering). This filter is not designed for real time. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma, s</samp></dt> <dd> Set the noise sigma constant. This <var class="var">sigma</var> defines a hard threshold of <code class="code">3 * sigma</code>; every DCT coefficient (absolute value) below this threshold with be dropped. If you need a more advanced filtering, see <samp class="option">expr</samp>.

Default is <code class="code">0</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number overlapping pixels for each block. Since the filter can be slow, you may want to reduce this value, at the cost of a less effective filter and the risk of various artefacts. If the overlapping value doesn’t permit processing the whole input width or height, a warning will be displayed and according borders won’t be denoised. Default value is <var class="var">blocksize</var>-1, which is the best possible setting. </dd> <dt><samp class="option">expr, e</samp></dt> <dd> Set the coefficient factor expression. For each coefficient of a DCT block, this expression will be evaluated as a multiplier value for the coefficient. If this is option is set, the <samp class="option">sigma</samp> option will be ignored. The absolute value of the coefficient can be accessed through the <var class="var">c</var> variable. </dd> <dt><samp class="option">n</samp></dt> <dd> Set the <var class="var">blocksize</var> using the number of bits. <code class="code">1<<<var class="var">n</var></code> defines the <var class="var">blocksize</var>, which is the width and height of the processed blocks. The default value is <var class="var">3</var> (8x8) and can be raised to <var class="var">4</var> for a <var class="var">blocksize</var> of 16x16. Note that changing this setting has huge consequences on the speed processing. Also, a larger block size does not necessarily means a better de-noising. </dd> </dl>

 <h4 class="subsection">11.53.1 Examples# TOC</h4>

Apply a denoise with a <samp class="option">sigma</samp> of <code class="code">4.5</code>: <pre class="example-preformatted">dctdnoiz=4.5 </pre>

The same operation can be achieved using the expression system: <pre class="example-preformatted">dctdnoiz=e='gte(c, 4.5*3)' </pre>

Violent denoise using a block size of <code class="code">16x16</code>: <pre class="example-preformatted">dctdnoiz=15:n=4 </pre>

 <h3 class="section">11.54 deband# TOC</h3>

Remove banding artifacts from input video. It works by replacing banded pixels with average value of referenced pixels. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>1thr</samp></dt> <dt><samp class=”option”>2thr</samp></dt>
<dt><samp class=”option”>3thr</samp></dt> <dt><samp class=”option”>4thr</samp></dt>
<dd> Set banding detection threshold for each plane. Default is 0.02. Valid range is 0.00003 to 0.5. If difference between current pixel and reference pixel is less than threshold, it will be considered as banded. </dd>
<dt><samp class=”option”>range, r</samp></dt> <dd> Banding detection range in pixels. Default is 16. If positive, random number in range 0 to set value will be used. If negative, exact absolute value will be used. The range defines square of four pixels around current pixel. </dd>
<dt><samp class=”option”>direction, d</samp></dt> <dd> Set direction in radians from which four pixel will be compared. If positive, random direction from 0 to set direction will be picked. If negative, exact of absolute value will be picked. For example direction 0, -PI or -2*PI radians will pick only pixels on same row and -PI/2 will pick only pixels on same column. </dd> <dt><samp class=”option”>blur, b</samp></dt> <dd> If enabled, current pixel is compared with average value of all four surrounding pixels. The default is enabled. If disabled current pixel is compared with all four surrounding pixels. The pixel is considered banded if only all four differences with surrounding pixels are less than threshold. </dd>
<dt><samp class=”option”>coupling, c</samp></dt> <dd> If enabled, current pixel is changed if and only if all pixel components are banded, e.g. banding detection threshold is triggered for all color components. The default is disabled. </dd> </dl>

 <h4 class=”subsection”>11.54.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.55 deblock# TOC</h3>

Remove blocking artifacts from input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set filter type, can be <var class=”var”>weak</var> or <var class=”var”>strong</var>. Default is <var class=”var”>strong</var>. This controls what kind of deblocking is applied. </dd> <dt><samp class=”option”>block</samp></dt>
<dd> Set size of block, allowed range is from 4 to 512. Default is <var class=”var”>8</var>. </dd> <dt><samp class=”option”>alpha</samp></dt>
<dt><samp class=”option”>beta</samp></dt> <dt><samp class=”option”>gamma</samp></dt>
<dt><samp class=”option”>delta</samp></dt> <dd> Set blocking detection thresholds. Allowed range is 0 to 1. Defaults are: <var class=”var”>0.098</var> for <var class=”var”>alpha</var> and <var

class=”var”>0.05</var> for the rest. Using higher threshold gives more deblocking strength. Setting <var class=”var”>alpha</var> controls threshold detection at exact edge of block. Remaining options controls threshold detection near the edge. Each one for below/above or left/right. Setting any of those to <var class=”var”>0</var> disables deblocking. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is to filter all available planes. </dd> </dl>

 <h4 class=”subsection”>11.55.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Deblock using weak filter and block size of 4 pixels. <pre class=”example-preformatted”>deblock=filter=weak:block=4 </pre>

Deblock using strong filter, block size of 4 pixels and custom thresholds for deblocking more edges. <pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta=0.05
</pre>

Similar as above, but filter only first plane.
<pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta
</pre>

Similar as above, but filter only second and third plane. <pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.0
</pre>

 <h4 class=”subsection”>11.55.2 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class=”section”>11.56 decimate# TOC</h3>

Drop duplicated frames at regular intervals. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cycle</samp></dt> <dd> Set the number of frames from which one will be dropped. Setting this to <var class=”var”>N</var> means one frame in every batch of <var class=”var”>N</var> frames will be dropped. Default is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>dupthresh</samp></dt> <dd> Set the threshold for duplicate detection. If the difference metric for a

frame is less than or equal to this value, then it is declared as duplicate. Default is <code class="code">1.1</code> </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set scene change threshold. Default is <code class="code">15</code>. </dd> <dt><samp class="option">blockx</samp></dt> <dt><samp class="option">blocky</samp></dt> <dd> Set the size of the x and y-axis blocks used during metric calculations. Larger blocks give better noise suppression, but also give worse detection of small movements. Must be a power of two. Default is <code class="code">32</code>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark main input as a pre-processed input and activate clean source input stream. This allows the input to be pre-processed with various filters to help the metrics calculation while keeping the frame selection lossless. When set to <code class="code">1</code>, the first stream is for the pre-processed input, and the second stream is the clean source from where the kept frames are chosen. Default is <code class="code">0</code>. </dd> <dt><samp class="option">chroma</samp></dt> <dd> Set whether or not chroma is considered in the metric calculations. Default is <code class="code">1</code>. </dd> <dt><samp class="option">mixed</samp></dt> <dd> Set whether or not the input only partially contains content to be decimated. Default is <code class="code">false</code>. If enabled video output stream will be in variable frame rate. </dd> </dl>

 <h3 class="section">11.57 deconvolve# TOC</h3>

Apply 2D deconvolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> <dt><samp class="option">noise</samp></dt> <dd> Set noise when doing divisions. Default is <var class="var">0.0000001</var>. Useful when width and height are not same and not power of 2 or if stream prior to convolving had noise. </dd> </dl>

The <code class="code">deconvolve</code> filter also supports the framesync options. <h3 class="section">11.58 dedot#
TOC</h3>

Reduce cross-luminance (dot-crawl) and cross-color (rainbows) from

video. It accepts the following options: <dl class="table"> <dt><samp class="option">m</samp></dt> <dd> Set mode of operation. Can be combination of <var class="var">dotcrawl</var> for cross-luminance reduction and/or <var class="var">rainbows</var> for cross-color reduction. </dd> <dt><samp class="option">lt</samp></dt> <dd> Set spatial luma threshold. Lower values increases reduction of cross-luminance. </dd> <dt><samp class="option">tl</samp></dt> <dd> Set tolerance for temporal luma. Higher values increases reduction of cross-luminance. </dd> <dt><samp class="option">tc</samp></dt> <dd> Set tolerance for chroma temporal variation. Higher values increases reduction of cross-color. </dd> <dt><samp class="option">ct</samp></dt> <dd> Set temporal chroma threshold. Lower values increases reduction of cross-color. </dd> </dl>

 <h3 class="section">11.59 deflate# TOC</h3>

Apply deflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values lower than the pixel. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">11.59.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.60 deflicker# TOC</h3>

Remove temporal frame luminance variations. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set moving-average filter size in frames. Default is 5. Allowed range is 2 - 129. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set averaging mode to smooth temporal luminance variations. Available values are: <dl class="table"> <dt>‘<samp class="samp">am</samp>’</dt> <dd> Arithmetic mean </dd> <dt>‘<samp class="samp">gm</samp>’</dt> <dd> Geometric mean </dd> <dt>‘<samp class="samp">hm</samp>’</dt>

<dd> Harmonic mean </dd> <dt>‘<samp class="samp">qm</samp>’</dt>
<dd> Quadratic mean </dd> <dt>‘<samp class="samp">cm</samp>’</dt>
<dd> Cubic mean </dd> <dt>‘<samp class="samp">pm</samp>’</dt>
<dd> Power mean </dd> <dt>‘<samp class="samp">median</samp>’</dt>
<dd> Median </dd> </dl>

</dd> <dt><samp class="option">bypass</samp></dt> <dd> Do not actually modify frame. Useful when one only wants metadata. </dd> </dl>

 <h3 class="section">11.61 deinterlace_d3d12# TOC</h3>

Deinterlace the input video using D3D12 hardware-accelerated video processing. This filter uses the DirectX 12 Video Processor to perform deinterlacing on GPU, supporting both simple bob and driver-defined advanced (motion-adaptive) algorithms. It requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter automatically queries the hardware for the required number of temporal reference frames (if needed) and manages a frame queue to supply them. It accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> The deinterlacing method. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Select the best deinterlacing method available on the hardware. If the driver supports <code class="code">custom</code>, that method is used; otherwise falls back to <code class="code">bob</code>. This is the default. </dd> <dt>‘<samp class="samp">bob</samp>’</dt> <dd> Bob deinterlacing. Each field is independently scaled to full frame height. Simple and fast, but may produce visible bob artifacts on moving content. </dd> <dt>‘<samp class="samp">custom</samp>’</dt> <dd> Driver-defined advanced deinterlacing. The exact algorithm is hardware-specific and typically employs motion-adaptive techniques with temporal reference frames for higher quality. </dd> </dl>

The default value is <code class="code">default</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Specify the interlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Send one frame for each frame. The output frame rate equals the input frame rate. This is the default. </dd> <dt>‘<samp class="samp">field</samp>’</dt> <dd> Send one frame for each field. The output frame rate is double the input frame rate. </dd> </dl>

The default value is <code class="code">frame</code>. </dd>

<dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. It accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames. This is the default. </dd> <dt>‘<samp class=”samp”>interlaced</samp>’</dt> <dd> Only deinterlace frames marked as interlaced; progressive frames are passed through unchanged. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h4 class=”subsection”>11.61.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Deinterlace an interlaced video using software decoding and the best available hardware deinterlacing method: <pre class=”example-preformatted”>ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=method=default,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

Deinterlace at field rate (double the frame rate) using bob method with hardware decoding (full d3d12 pipeline): <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.ts \ -vf "deinterlace_d3d12=method=bob:mode=field" \ -c:v h264_d3d12va output.mp4 </pre>

Deinterlace only interlaced frames, passing progressive frames through: <pre class=”example-preformatted”>ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=deint=interlaced,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

 <h3 class=”section”>11.62 dejudder# TOC</h3>

Remove judder produced by partially interlaced telecined content. Judder can be introduced, for instance, by pullup filter. If the original source was partially telecined content then the output of <code class=”code”>pullup,dejudder</code> will have a variable frame rate. May change the recorded frame rate of the container. Aside from that change, this filter will not affect constant frame rate video. The option available in this filter is: <dl class=”table”> <dt><samp class=”option”>cycle</samp></dt> <dd> Specify the

length of the window over which the judder repeats. Accepts any integer greater than 1. Useful values are: <dl class="table"> <dt>‘<samp class="samp">4</samp>’</dt> <dd> If the original was telecined from 24 to 30 fps (Film to NTSC). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> If the original was telecined from 25 to 30 fps (PAL to NTSC). </dd> <dt>‘<samp class="samp">20</samp>’</dt> <dd> If a mixture of the two. </dd> </dl>

The default is ‘<samp class="samp">4</samp>’. </dd> </dl>

 <h3 class="section">11.63 delogo# TOC</h3>

Suppress a TV station logo by a simple interpolation of the surrounding pixels. Just set a rectangle covering the logo and watch it disappear (and sometimes something even uglier appear - your mileage may vary). It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the top left corner coordinates of the logo. They must be specified. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Specify the width and height of the logo to clear. They must be specified. </dd> <dt><samp class="option">show</samp></dt> <dd> When set to 1, a green rectangle is drawn on the screen to simplify finding the right <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, and <var class="var">h</var> parameters. The default value is 0. The rectangle is drawn on the outermost pixels which will be (partly) replaced with interpolated values. The values of the next pixels immediately outside this rectangle in each direction will be used to compute the interpolated pixel values inside the rectangle. </dd> </dl>

 <h4 class="subsection">11.63.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set a rectangle covering the area with top left corner coordinates 0,0 and size 100x77: <pre class="example-preformatted">delogo=x=0:y=0:w=100:h=77 </pre>

 <h3 class="section">11.64 derain# <a class="anchor

hidden-xs”href=”#toc-derain” aria-hidden=”true”>TOC</h3>

Remove the rain in the input image/video by applying the derain methods based on convolutional neural networks. Supported models: <ul class=”itemize mark-bullet”> Recurrent Squeeze-and-Excitation Context Aggregation Net (RESCAN). See http://openaccess.thecvf.com/content_ECCV_2018/pa and-Excitation_Context_ECCV_2018_paper.pdf.

Training as well as model generation scripts are provided in the repository at <a class=”url” href=”https://github.com/XueweiMeng/derain_filter.git”>https://github.com/XueweiMeng/d The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filter_type</samp></dt> <dd> Specify which filter to use. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>derain</samp>’</dt> <dd> Derain filter. To conduct derain filter, you need to use a derain model. </dd> <dt>‘<samp class=”samp”>dehaze</samp>’</dt> <dd> Dehaze filter. To conduct dehaze filter, you need to use a dehaze model. </dd> </dl> Default value is ‘<samp class=”samp”>derain</samp>’. </dd> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow can load files for only its format. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>11.65 deshake# TOC</h3>

Attempt to fix small changes in horizontal and/or vertical shift. This filter helps remove camera shake from hand-holding a camera, bumping a tripod, moving on a vehicle, etc. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Specify a rectan-

gular area where to limit the search for motion vectors. If desired the search for motion vectors can be limited to a rectangular area of the frame defined by its top left corner, width and height. These parameters have the same meaning as the drawbox filter which can be used to visualise the position of the bounding box. This is useful when simultaneous movement of subjects within the frame might be confused for camera motion by the motion vector search. If any or all of <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> are set to -1 then the full frame is used. This allows later options to be set without specifying the bounding box for the motion vector search. Default - search the whole frame. </dd> <dt><samp class="option">rx</samp></dt> <dt><samp class="option">ry</samp></dt> <dd> Specify the maximum extent of movement in x and y directions in the range 0-64 pixels. Default 16. </dd> <dt><samp class="option">edge</samp></dt> <dd> Specify how to generate pixels to fill blanks at the edge of the frame. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank, 0</samp>’</dt> <dd> Fill zeroes at blank locations </dd> <dt>‘<samp class="samp">original, 1</samp>’</dt> <dd> Original image at blank locations </dd> <dt>‘<samp class="samp">clamp, 2</samp>’</dt> <dd> Extruded edge value at blank locations </dd> <dt>‘<samp class="samp">mirror, 3</samp>’</dt> <dd> Mirrored edge at blank locations </dd> </dl> Default value is ‘<samp class="samp">mirror</samp>’. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Specify the blocksize to use for motion search. Range 4-128 pixels, default 8. </dd> <dt><samp class="option">contrast</samp></dt> <dd> Specify the contrast threshold for blocks. Only blocks with more than the specified contrast (difference between darkest and lightest pixels) will be considered. Range 1-255, default 125. </dd> <dt><samp class="option">search</samp></dt> <dd> Specify the search strategy. Available values are: <dl class="table"> <dt>‘<samp class="samp">exhaustive, 0</samp>’</dt> <dd> Set exhaustive search </dd> <dt>‘<samp class="samp">less, 1</samp>’</dt> <dd> Set less exhaustive search. </dd> </dl> Default value is ‘<samp class="samp">exhaustive</samp>’. </dd> <dt><samp class="option">filename</samp></dt> <dd> If set then a detailed log of the motion search is written to the specified file. </dd> </dl>

 <h3 class="section">11.66 despill# TOC</h3>

Remove unwanted contamination of foreground colors, caused by reflected color of greenscreen or bluescreen. This filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>type</samp></dt> <dd> Set what type of despill to use. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> Set how spillmap will be generated. </dd> <dt><samp class=”option”>expand</samp></dt> <dd> Set how much to get rid of still remaining spill. </dd> <dt><samp class=”option”>red</samp></dt> <dd> Controls amount of red in spill area. </dd> <dt><samp class=”option”>green</samp></dt> <dd> Controls amount of green in spill area. Should be -1 for greenscreen. </dd> <dt><samp class=”option”>blue</samp></dt> <dd> Controls amount of blue in spill area. Should be -1 for bluescreen. </dd> <dt><samp class=”option”>brightness</samp></dt> <dd> Controls brightness of spill area, preserving colors. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Modify alpha from generated spillmap. </dd> </dl>

 <h4 class=”subsection”>11.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.67 detelecine# TOC</h3>

Apply an exact inverse of the telecine operation. It requires a predefined pattern specified using the pattern option which must be the same as that passed to the telecine filter. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> A number representing position of the first frame with respect to the telecine pattern. This is to be used if the stream is cut. The default value is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>11.68 dilation# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel

by the local(3x3) maximum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class="subsection">11.68.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.69 displace# TOC</h3>

Displace pixels as indicated by second and third input stream. It takes three input streams and outputs one stream, the first input is the source, and second and third input are displacement maps. The second input specifies how much to displace pixels along the x-axis, while the third input specifies how much to displace pixels along the y-axis. If one of displacement map streams terminates, last frame from that displacement map will be used. Note that once generated, displacements maps can be reused over and over again. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">edge</samp></dt> <dd> Set displace behavior for pixels that are out of range. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank</samp>’</dt> <dd> Missing pixels are replaced by black pixels. </dd> <dt>‘<samp class="samp">smear</samp>’</dt> <dd> Adjacent pixels will spread out to replace missing pixels. </dd> <dt>‘<samp class="samp">wrap</samp>’</dt> <dd> Out of range pixels are wrapped so they point to pixels of other side. </dd> <dt>‘<samp class="samp">mirror</samp>’</dt> <dd> Out of range pixels will be replaced with mirrored pixels. </dd> </dl> Default is ‘<samp class="samp">smear</samp>’. </dd> </dl>

 <h4 class="subsection">11.69.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add ripple effect to rgb input

of video size hd720: <pre class=”example-preformatted”>ffmpeg -i INPUT -f lavfi -i nullsrc=s=hd720,lutrgb=128:128:128 -f lavfi -i nullsrc=s=hd720,geq='r=128+30*sin(2*PI*X/400+T):g=128+30*sin(2*PI*X/400+T):b=128+30*sin -lavfi '[0][1][2]displace' OUTPUT </pre>

Add wave effect to rgb input of video size hd720: <pre class=”example-preformatted”>ffmpeg -i INPUT -f lavfi -i nullsrc=hd720,geq='r=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):g=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):b=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T))' -lavfi '[1]split[x][y],[0][x][y]displace' OUTPUT </pre>

 <h3 class=”section”>11.70 dnn_classify# TOC</h3>

Do classification with deep neural networks based on bounding boxes. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts only openvino now, tensorflow backends will be added. </dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0, and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend For tensorflow backend, you can set its configs with <samp class=”option”>sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs for your system. </dd> </dl>

 <h3 class=”section”>11.71 dnn_detect# TOC</h3>

Do object detection with deep neural networks. The fil-

ter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt> <dd> OpenVINO backend. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. The TensorFlow and ONNX Runtime backends accept multiple output names separated by ‘<samp class=”samp”>&</samp>’, e.g. <samp class=”option”>output=yolo_13&yolo_26</samp>. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0 (usually it is ’background’), and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>model_type</samp></dt> <dd> Set the detection model output format. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>ssd</samp>’</dt> <dd> Single-stage detector (default). </dd> <dt>‘<samp class=”samp”>yolo</samp>’</dt> <dd> YOLO v1/v2. </dd> <dt>‘<samp class=”samp”>yolov3</samp>’</dt> <dd> YOLOv3. </dd> <dt>‘<samp class=”samp”>yolov4</samp>’</dt> <dd> YOLOv4. </dd> </dl> </dd> <dt><samp class=”option”>anchors</samp></dt> <dd> Anchor box dimensions, separated by ‘<samp class=”samp”>&</samp>’. Required for YOLO-family models. The list is consumed in output order, so the anchors of the first output come first: with <samp class=”option”>output=yolo_13&yolo_26</samp> the leading pairs belong to <code class=”code”>yolo_13</code>. </dd> <dt><samp class=”option”>nb_classes</samp></dt> <dd> Number of detection classes. Required for YOLO-family models. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. </dd> </dl>

 <h4 class=”subsection”>11.71.1 Examples<a class=”anchor hidden-xs” href=”#Examples-

68" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Run SSD object detection with an ONNX model (single output, DirectML on Windows): <pre class="example-preformatted">ffmpeg -i input.mp4 -vf "scale=300:300,format=rgb24,dnn_detect=:model=ssd.onnx:model_type=ssd:confidence=0.5:device=dml" -f null - </pre>

Run YOLOv3 object detection with an ONNX model (two outputs, CPU): <pre class="example-preformatted">ffmpeg -i input.mp4 -vf "scale=416:416,format=rgb24,dnn_detect=dnn_backend=onnx:model=yolov3.onnx:output=yolo_13&yolo_26:model_type=yolov3:nb_classes=80:anchors=116&90&156&198&373&326&30&61&62&45&59&119&116&90&156&198&373&326:scale=1/255:input_order=nhwc" -f null - </pre>

 <h3 class="section">11.72 dnn_processing# TOC</h3>

Do image processing with deep neural networks. It works together with another filter which converts the pixel format of the Frame to what the dnn network requires. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class="url" href="https://www.tensorflow.org/install/lang_c">https://www and configure FFmpeg with <code class="code">--enable-libtensorflow</code> </dd> <dt>‘<samp class="samp">openvino</samp>’</dt> <dd> OpenVINO backend. To enable this backend you need to build and install the OpenVINO for C library (see https://github.com/openvinotoolkit/openvino/blob/master/build-instruction.md) and configure FFmpeg with <code class="code">--enable-libopenvino</code> (–extra-cflags=-I... –extra-ldflags=-L... might be needed if the header files and libraries are not installed into system path) </dd> <dt>‘<samp class="samp">torch</samp>’</dt> <dd> Libtorch backend. To enable this backend you need to build and install Libtorch for C++ library. Please download cxx11 ABI version (see https://pytorch.org/get-started/locally) and configure FFmpeg with <code class="code">--enable-libtorch --extra-cflags=-I/libtorch_root/libtorch/include --extra-cflags=-I/libtorch_root/libtorch/include/torch/csrc/api/include --extra-ldflags=-L/libtorch_root/libtorch/lib/</code> When FFm-

peg is built with CUDA support and the input frames use <code class=”code”>AV_PIX_FMT_CUDA</code> with an underlying pixel format of <code class=”code”>rgb0</code>, <code class=”code”>bgr0</code>, <code class=”code”>rgb24</code>, or <code class=”code”>bgr24</code>, the Torch backend maps GPU frame buffers directly into LibTorch CUDA tensors using zero-copy memory mapping. This avoids host-device PCIe transfers and can significantly reduce inference latency. The zero-copy path is selected automatically when the input format qualifies; no additional options are required. When the input format is not <code class=”code”>AV_PIX_FMT_CUDA</code>, the backend uses the standard CPU tensor path. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. To enable this backend you need to install the ONNX Runtime library (see https://onnxruntime.ai/) and configure FFmpeg with <code class=”code”>--enable-libonnxruntime</code>. The current ONNX Runtime backend expects 4-D input and output tensors with NCHW layout and 32-bit floating-point element type (ONNX <code class=”code”>FLOAT</code>); models with integer or other element types (e.g. <code class=”code”>UINT8</code>) are not supported and will be rejected at load time. Models using NHWC layout or other ranks are not yet supported. Only single-input models are supported; the backend binds exactly one input tensor when running the model. The <samp class=”option”>input</samp> and <samp class=”option”>output</samp> options are optional for the ONNX Runtime backend; when they are omitted the backend resolves the tensor names from the session. Multiple output names may be supplied separated by ‘<samp class=”samp”>&</samp>’ (e.g. <samp class=”option”>output=out_a&out_b</samp>); however for <code class=”code”>dnn_processing</code> only the first output tensor is used for frame post-processing. The ONNX Runtime backend runs inference synchronously using a single inference request. The shared <samp class=”option”>async</samp> and <samp class=”option”>nireq</samp> options therefore have no effect for <code class=”code”>dnn_backend=onnx</code>; inference always runs synchronously regardless of their values. When FFmpeg is built with Direct3D 12 support and this build’s ONNX Runtime provides a working DirectML execution provider, setting <samp class=”option”>device=dml</samp> together with <code class=”code”>AV_PIX_FMT_D3D12</code> NV12 hardware frames as input (e.g. via <code class=”code”>-hwaccel d3d12va -hwaccel_output_format d3d12</code>) keeps frames in VRAM for the whole filter, from the decoder’s output through DNN inference and back out, without a host round-trip. Software-frame input retains the host I/O path. D3D12 input requires successful DirectML negotiation; unsupported hardware-input configurations fail during configuration. At runtime, <samp class=”file”>DirectML.dll</samp> and <samp class=”file”>d3dcompiler_47.dll</samp>

must be loadable from the application directory or System32. Currently, only single-channel (luma-only), <code class="code">FLOAT</code>/NCHW models with a <code class="code">[1,1,H,W]</code> output tensor and even input/output frame dimensions are supported; the chroma plane is copied or bilinearly upscaled on the GPU without going through the model. The output frames are D3D12 hardware frames, so they must either be consumed by a D3D12 encoder or downloaded explicitly with <code class="code">hwdownload,format=nv12</code>. </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO, Libtorch, and ONNX Runtime backends can load files for only their respective formats. </dd> <dt><samp class="option">input</samp></dt> <dd> Set the input name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class="option">output</samp></dt> <dd> Set the output name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class="option">backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. For tensorflow backend, you can set its configs with <samp class="option">sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. </dd> <dt><samp class="option">async</samp></dt> <dd> Use DNN async inference (default: 1). When enabled, frames are submitted to the backend without blocking the caller. Supported by the Torch, OpenVINO, and TensorFlow backends; the ONNX Runtime backend ignores this option. </dd> <dt><samp class="option">nireq</samp></dt> <dd> Number of concurrent inference requests (default: 0, meaning auto). When set to 0, the backend automatically determines an appropriate number of requests. Higher values increase throughput by allowing multiple frames to be in flight simultaneously. </dd> <dt><samp class="option">batch_size</samp></dt> <dd> Number of frames to batch into a single inference request (default: 1). Values greater than 1 cause the backend to accumulate frames and concatenate them along the batch dimension before running the model. This can significantly increase throughput on GPUs at the cost of slightly higher per-frame latency. Currently supported by the OpenVINO and Torch backends. </dd> <dt><samp class="option">device</samp></dt> <dd> Set the device to run the model. For the ONNX Runtime backend this selects the execution provider: <code class="code">cpu</code> (default), <code class="code">cuda</code> (NVIDIA GPU), <code class="code">dml</code> (DirectML, Windows only) or <code class="code">vitisai</code> (AMD Ryzen AI NPU). </dd> <dt><samp class="option">device_id</samp></dt>

<dd> Set the device index used by GPU execution providers (e.g. <code class=”code”>cuda</code> or <code class=”code”>dml</code>) for the ONNX Runtime backend. Default is 0. </dd> <dt><samp class=”option”>threads_per_operation</samp></dt> <dd> ONNX Runtime backend only. Set the number of CPU threads used per ONNX Runtime operator when running with <code class=”code”>device=cpu</code>. Default is 0 (let ONNX Runtime choose automatically). Has no effect for GPU/NPU providers. </dd> </dl>

 <h4 class=”subsection”>11.72.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Remove rain in rgb24 frame with can.pb (see derain filter): <pre class=”example-preformatted”>./ffmpeg -i rain.jpg -vf format=rgb24,dnn_processing=dnn_backend=tensorflow:model=can.pb:input=x:output=y derain.jpg </pre>

Handle the Y channel with srcnn.pb (see sr filter) for frame with yuv420p (planar YUV formats supported): <pre class=”example-preformatted”>./ffmpeg -i 480p.jpg -vf format=yuv420p,scale=w=iw*2:h=ih*2,dnn_processing=dnn_backend=tensorflow:model=srcn -y srcnn.jpg </pre>

Handle the Y channel with espcn.pb (see sr filter), which changes frame size, for format yuv420p (planar YUV formats supported), please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. <pre class=”example-preformatted”>./ffmpeg -i 480p.jpg -vf format=yuv420p,dnn_processing=dnn_backend= -y tmp.espcn.jpg </pre>

Perform zero-copy GPU inference with the Libtorch backend. Because there is currently no GPU-side filter to convert from YUV to RGB, <code class=”code”>hwupload</code> is used to upload CPU RGB frames into device memory. Once a GPU-side YUV-to-RGB filter becomes available, the <code class=”code”>hwupload</code> step will no longer be necessary and all frames can stay in device memory throughout the pipeline: <pre class=”example-preformatted”>./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda -i input.mp4 \ -vf "format=rgb24,hwupload,dnn_processing=dnn_backend=torch:model=my_model.pt,hwdownload,form \ -y output.mp4 </pre>

Run a single-channel (luma-only) super-resolution model on D3D12 hardware frames with the ONNX Runtime backend, keeping frames in VRAM from decode through inference (Windows only, requires a build with Direct3D 12 support and <code class=”code”>--enable-libonnxruntime</code> against a DirectML-capable ONNX Runtime).

The D3D12 output frames are encoded directly by a D3D12 hardware encoder: <pre class=”example-preformatted”>./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml \ -c:v h264_d3d12va -bf 0 -y output.mp4 </pre>

Same as above, but download the result to software frames after inference so that a software encoder can be used: <pre class=”example-preformatted”>./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml,hwdownloa \ -y output.mp4 </pre>

Process rgb24 frames with a TorchScript model using the Libtorch backend: <pre class=”example-preformatted”>./ffmpeg -i input.jpg -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_model.pt output.jpg </pre>

Process rgb24 frames with the Libtorch backend using batch inference (four frames per request): <pre class=”example-preformatted”>./ffmpeg -i input.mp4 -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_mo -y output.mp4 </pre>

 <h3 class=”section”>11.73 drawbox# TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the box edge color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class=”code”>fill</code> will create a filled box. Default value is <code class=”code”>3</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With value <code class=”code”>1</code>, the pixels of the painted

box will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the box onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var class="var">t</var> are expressions containing the following constants: <dl class="table"> <dt><samp class="option">dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var>. </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">in_h, ih</samp></dt> <dt><samp class="option">in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class="option">box_source</samp></dt> <dd> Box source can be set as side_data_detection_bboxes if you want to use box data in detection bboxes of side data. If <var class="var">box_source</var> is set, the <var class="var">x</var>, <var class="var">y</var>, <var class="var">width</var> and <var class="var">height</var> will be ignored and still use box data in detection bboxes of side data. So please do not use this parameter if you were not sure about the box source. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. These constants allow the <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, <var class="var">h</var> and <var class="var">t</var> expressions to refer to each other, so you may for example specify <code class="code">y=x/dar</code> or <code class="code">h=w/dar</code>. </dd> </dl>

 <h4 class="subsection">11.73.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5

</pre>

The previous example can be specified as:
<pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5
</pre>

Fill the box with pink color: <pre
class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill
</pre>

Draw a 2-pixel red 2.40:1 mask:
<pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-
t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h4 class="subsection">11.73.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.74 drawgraph# TOC</h3>
Draw a graph using input video metadata. It accepts the following parameters: <dl class="table"> <dt><samp class="option">m1</samp></dt>
<dd> Set 1st frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class="option">fg1</samp></dt>
<dd> Set 1st foreground color expression. </dd> <dt><samp class="option">m2</samp></dt> <dd> Set 2nd frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class="option">fg2</samp></dt> <dd> Set 2nd foreground color expression. </dd> <dt><samp class="option">m3</samp></dt>
<dd> Set 3rd frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class="option">fg3</samp></dt>
<dd> Set 3rd foreground color expression. </dd> <dt><samp class="option">m4</samp></dt> <dd> Set 4th frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class="option">fg4</samp></dt> <dd> Set 4th foreground color expression. </dd> <dt><samp class="option">min</samp></dt>
<dd> Set minimal value of metadata value. </dd> <dt><samp class="option">max</samp></dt> <dd> Set maximal value of metadata value. </dd> <dt><samp class="option">bg</samp></dt>
<dd> Set graph background color. Default is white. </dd>
<dt><samp class="option">mode</samp></dt> <dd> Set graph mode. Available values for mode is: <dl class="table">
<dt>‘<samp class="samp">bar</samp>’</dt> <dt>‘<samp

class=”samp”>dot</samp>’</dt> <dt>‘<samp class=”samp”>line</samp>’</dt>
</dl>

Default is <code class=”code”>line</code>. </dd> <dt><samp
class=”option”>slide</samp></dt> <dd> Set slide mode.
Available values for slide is: <dl class=”table”> <dt>‘<samp
class=”samp”>frame</samp>’</dt> <dd> Draw new frame
when right border is reached. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt
<dd> Replace old columns with new ones. </dd> <dt>‘<samp
class=”samp”>scroll</samp>’</dt> <dd> Scroll from right to
left. </dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> Scroll from left to right. </dd> <dt>‘<samp
class=”samp”>picture</samp>’</dt> <dd> Draw single picture.
</dd> </dl>

Default is <code class=”code”>frame</code>. </dd>
<dt><samp class=”option”>size</samp></dt> <dd> Set size of
graph video. For the syntax of this option, check the <a data-manual=”ffmpeg-
utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size"
section in the ffmpeg-utils manual. The default value is <code
class=”code”>900x256</code>. </dd> <dt><samp class=”option”>rate,
r</samp></dt> <dd> Set the output frame rate. Default value is <code
class=”code”>25</code>. The foreground color expressions
can use the following variables: <dl class=”table”> <dt><samp
class=”option”>MIN</samp></dt> <dd> Minimal value of metadata
value. </dd> <dt><samp class=”option”>MAX</samp></dt>
<dd> Maximal value of metadata value. </dd> <dt><samp
class=”option”>VAL</samp></dt> <dd> Current metadata key value.
</dd> </dl>

The color is defined as 0xAABBGGRR. </dd> </dl>

Example using metadata from signalstats
filter: <pre class=”example-preformatted”>signalstats,drawgraph=lavfi.signalst
</pre>

Example using metadata from ebur128
filter: <pre class=”example-preformatted”>ebur128=metadata=1,adrawgraph=
120:max=5 </pre>

 <h3 class=”section”>11.75 drawgrid<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#drawgrid” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-drawgrid”
aria-hidden=”true”>TOC</h3>

Draw a grid on the input image. It accepts the following parame-
ters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt>
<dt><samp class=”option”>y</samp></dt> <dd> The expressions
which specify the coordinates of some point of grid intersection (meant

to configure offset). Both default to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the grid cell, if 0 they are interpreted as the input width and height, respectively, minus <code class=”code”>thickness</code>, so image gets framed. Default to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the grid. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the grid color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the grid line. Default value is <code class=”code”>1</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With <code class=”code”>1</code> the pixels of the painted grid will overwrite the video’s color and alpha pixels. Default is <code class=”code”>0</code>, which composites the grid onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> and <var class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input grid cell width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y coordinates of some point of grid intersection (meant to configure offset). </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn cell. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The thickness of the drawn cell. These constants allow the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var>, <var class=”var”>h</var> and <var class=”var”>t</var> expressions to refer to each other, so you may for example specify <code class=”code”>y=x/dar</code> or <code class=”code”>h=w/dar</code>. </dd> </dl>

 <h4 class="subsection">11.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a grid with cell 100x100 pixels, thickness 2 pixels, with color red and an opacity of 50%: <pre class="example-preformatted">drawgrid=width=100:height=100:thickness=2:color=red@0.5 </pre>

Draw a white 3x3 grid with an opacity of 50%: <pre class="example-preformatted">drawgrid=w=iw/3:h=ih/3:t=2:c=white@0.5 </pre>

 <h4 class="subsection">11.75.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.76 drawtext# TOC</h3>

Draw a text string or text from a specified file on top of a video, using the libfreetype library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libfreetype</code> and <code class="code">--enable-libharfbuzz</code>. To enable default font fallback and the <var class="var">font</var> option you need to configure FFmpeg with <code class="code">--enable-libfontconfig</code>. To enable the <var class="var">text_shaping</var> option, you need to configure FFmpeg with <code class="code">--enable-libfribidi</code>. <h4 class="subsection">11.76.1 Syntax# TOC</h4>

It accepts the following parameters:

<dl class="table">

<dt><samp class="option">box</samp></dt>

<dd> Used to draw a box around text using the background color.

The value must be either 1 (enable) or 0 (disable).

The default value of <var class=”var”>box</var> is 0.

</dd>

<dt><samp class=”option”>boxborderw</samp></dt>

<dd> Set the width of the border to be drawn around the box using <var class=”var”>boxcolor</var>.

The value must be specified using one of the following formats:

<ul class=”itemize mark-bullet”>

<code class=”code”>boxborderw=10</code> set the width of all the borders to 10

<code class=”code”>boxborderw=10|20</code> set the width of the top and bottom borders to 10

and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30</code> set the width of the top border to 10, the width

of the bottom border to 30 and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30|40</code> set the borders width to 10 (top), 20 (right),

30 (bottom), 40 (left)

The default value of <var class=”var”>boxborderw</var> is "0".

</dd>

<dt><samp class=”option”>boxcolor</samp></dt>

<dd> The color to be used for drawing box around text. For the syntax of this

option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

The default value of <var class=”var”>boxcolor</var> is "white".

</dd>

<dt><samp class=”option”>line_spacing</samp></dt>

<dd> Set the line spacing in pixels. The default value of <var class="var">line_spacing</var> is 0.

</dd>

<dt><samp class="option">text_align</samp></dt>

<dd> Set the vertical and horizontal alignment of the text with respect to the box boundaries.

The value is combination of flags, one for the vertical alignment (T=top,

M=middle, B=bottom) and one for the horizontal alignment (L=left, C=center, R=right).

Please note that tab characters are only supported with the left horizontal alignment.

</dd>

<dt><samp class="option">y_align</samp></dt>

<dd> Specify what the <var class="var">y</var> value is referred to. Possible values are:

<ul class="itemize mark-bullet">

<code class="code">text</code> the top of the highest glyph of the first text line is placed at <var class="var">y</var>

<code class="code">baseline</code> the baseline of the first text line is placed at <var class="var">y</var>

<code class="code">font</code> the baseline of the first text line is placed at <var class="var">y</var> plus the

ascent (in pixels) defined in the font metrics

The default value of <var class="var">y_align</var> is "text" for backward compatibility.

</dd>

<dt><samp class="option">borderw</samp></dt>

<dd> Set the width of the border to be drawn around the text using <var class="var">bordercolor</var>.

The default value of <var class="var">borderw</var> is 0.

</dd>

<dt><samp class="option">bordercolor</samp></dt>

<dd> Set the color to be used for drawing border around text. For the syntax of this

option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">bordercolor</var> is "black".

</dd>

<dt><samp class="option">expansion</samp></dt>

<dd> Select how the <var class="var">text</var> is expanded. Can be either <code class="code">none</code>,

<code class="code">strftime</code> (deprecated) or <code class="code">normal</code> (default). See the

Text expansion section below for details.

</dd>

<dt><samp class="option">basetime</samp></dt>

<dd> Set a start time for the count. Value is in microseconds. Only applied

in the deprecated <code class="code">strftime</code> expansion mode. To emulate in normal expansion

mode use the <code class="code">pts</code> function, supplying the start time (in seconds)

as the second argument.

</dd>

<dt><samp class="option">fix_bounds</samp></dt>

<dd> If true, check and fix text coords to avoid clipping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> The color to be used for drawing fonts. For the syntax of this option, check

the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">fontcolor</var> is "black".

</dd>

<dt><samp class="option">fontcolor_expr</samp></dt>

<dd> String which is expanded the same way as <var class="var">text</var> to obtain dynamic

<var class="var">fontcolor</var> value. By default this option has empty value and is not

processed. When this option is set, it overrides <var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> The font family to be used for drawing text. By default Sans.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> The font file to be used for drawing text. The path must be included.

This parameter is mandatory if the fontconfig support is disabled.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Draw the text applying alpha blending. The value can

be a number between 0.0 and 1.0.

The expression accepts the same variables <var class="var">x, y</var> as well.

The default value is 1.

Please see <var class="var">fontcolor_expr</var>.

</dd>

<dt><samp class="option">fontsize</samp></dt>

<dd> The font size to be used for drawing text.

The default value of <var class="var">fontsize</var> is 16.

</dd>

<dt><samp class="option">text_shaping</samp></dt>

<dd> If set to 1, attempt to shape the text (for example, reverse the order of

right-to-left text and join Arabic characters) before drawing it.

Otherwise, just draw the text exactly as given.

By default 1 (if supported).

</dd>

<dt><samp class="option">ft_load_flags</samp></dt>

<dd> The flags to be used for loading the fonts.

The flags map the corresponding flags supported by libfreetype, and are

a combination of the following values:

<dl class="table">

<dt><var class="var">default</var></dt>

<dt><var class="var">no_scale</var></dt>

<dt><var class="var">no_hinting</var></dt>

<dt><var class="var">render</var></dt>

<dt><var class="var">no_bitmap</var></dt>

<dt><var class="var">vertical_layout</var></dt>

<dt><var class="var">force_autohint</var></dt>

<dt><var class="var">crop_bitmap</var></dt>

<dt><var class="var">pedantic</var></dt>

<dt><var class=”var”>ignore_global_advance_width</var></dt>

<dt><var class=”var”>no_recurse</var></dt>

<dt><var class=”var”>ignore_transform</var></dt>

<dt><var class=”var”>monochrome</var></dt>

<dt><var class=”var”>linear_design</var></dt>

<dt><var class=”var”>no_autohint</var></dt>

</dl>

Default value is "default".

For more information consult the documentation for the FT_LOAD_*

libfreetype flags.

</dd>

<dt><samp class=”option”>shadowcolor</samp></dt>

<dd> The color to be used for drawing a shadow behind the drawn text. For the

syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the

ffmpeg-utils manual.

The default value of <var class=”var”>shadowcolor</var> is "black".

</dd>

<dt><samp class=”option”>boxw</samp></dt>

<dd> Set the width of the box to be drawn around text.

The default value of <var class=”var”>boxw</var> is computed automatically to match the text width

</dd>

<dt><samp class=”option”>boxh</samp></dt>

<dd> Set the height of the box to be drawn around text.

The default value of <var class=”var”>boxh</var> is computed automatically to match the text height

</dd>

<dt><samp class=”option”>shadowx</samp></dt>

<dt><samp class=”option”>shadowy</samp></dt>

<dd> The x and y offsets for the text shadow position with respect to the

position of the text. They can be either positive or negative

values. The default value for both is "0".

</dd>

<dt><samp class=”option”>start_number</samp></dt>

<dd> The starting frame number for the n/frame_num variable. The default value

is "0".

</dd>

<dt><samp class=”option”>tabsize</samp></dt>

<dd> The size in number of spaces to use for rendering the tab.

Default value is 4.

</dd>

<dt><samp class=”option”>timecode</samp></dt>

<dd> Set the initial timecode representation in "hh:mm:ss[:;.]ff"

format. It can be used with or without text parameter. <var class=”var”>timecode_rate</var>

option must be specified.

</dd>

<dt><samp class=”option”>timecode_rate, rate, r</samp></dt>

<dd> Set the timecode frame rate (timecode only). Value will be rounded to nearest

integer. Minimum value is "1".

Drop-frame timecode is supported for frame rates 30 & 60.

</dd>

<dt><samp class=”option”>tc24hmax</samp></dt>

<dd> If set to 1, the output of the timecode option will wrap around at 24 hours.

Default is 0 (disabled).

</dd>

<dt><samp class=”option”>text</samp></dt>

<dd> The text string to be drawn. The text must be a sequence of UTF-8

encoded characters.

This parameter is mandatory if no file is specified with the parameter

<var class=”var”>textfile</var>.

</dd>

<dt><samp class=”option”>textfile</samp></dt>

<dd> A text file containing text to be drawn. The text must be a sequence

of UTF-8 encoded characters.

This parameter is mandatory if no text string is specified with the

parameter <var class=”var”>text</var>.

If both <var class=”var”>text</var> and <var class=”var”>textfile</var> are specified, an error is thrown.

</dd>

<dt><samp class=”option”>text_source</samp></dt>

<dd> Text source should be set as side_data_detection_bboxes if you want to use text data in

detection bboxes of side data.

If text source is set, <var class="var">text</var> and <var class="var">textfile</var> will be ignored and still use

text data in detection bboxes of side data. So please do not use this parameter

if you are not sure about the text source.

</dd>

<dt><samp class="option">reload</samp></dt>

<dd> The <var class="var">textfile</var> will be reloaded at specified frame interval.

Be sure to update <var class="var">textfile</var> atomically, or it may be read partially,

or even fail.

Range is 0 to INT_MAX. Default is 0.

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> The expressions which specify the offsets where text will be drawn

within the video frame. They are relative to the top/left border of the

output image.

The default value of <var class="var">x</var> and <var class="var">y</var> is "0".

See below for the list of accepted constants and functions.

</dd>

</dl>

The parameters for <var class="var">x</var> and <var class="var">y</var> are expressions containing the

following constants and functions:

<dl class="table">

<dt><samp class=”option”>dar</samp></dt>

<dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>

</dd>

<dt><samp class=”option”>hsub</samp></dt>

<dt><samp class=”option”>vsub</samp></dt>

<dd> horizontal and vertical chroma subsample values. For example for the

pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1.

</dd>

<dt><samp class=”option”>line_h, lh</samp></dt>

<dd> the height of each text line

</dd>

<dt><samp class=”option”>main_h, h, H</samp></dt>

<dd> the input height

</dd>

<dt><samp class=”option”>main_w, w, W</samp></dt>

<dd> the input width

</dd>

<dt><samp class=”option”>max_glyph_a, ascent</samp></dt>

<dd> the maximum distance from the baseline to the highest/upper grid

coordinate used to place a glyph outline point, for all the rendered

glyphs.

It is a positive value, due to the grid’s orientation with the Y axis

upwards.

</dd>
<dt><samp class="option">max_glyph_d, descent</samp></dt>
<dd> the maximum distance from the baseline to the lowest grid coordinate used to place a glyph outline point, for all the rendered glyphs.
This is a negative value, due to the grid’s orientation, with the Y axis upwards.

</dd>
<dt><samp class="option">max_glyph_h</samp></dt>
<dd> maximum glyph height, that is the maximum height for all the glyphs contained in the rendered text, it is equivalent to <var class="var">ascent</var>
-
<var class="var">descent</var>.

</dd>
<dt><samp class="option">max_glyph_w</samp></dt>
<dd> maximum glyph width, that is the maximum width for all the glyphs contained in the rendered text

</dd>
<dt><samp class="option">font_a</samp></dt>
<dd> the ascent size defined in the font metrics

</dd>
<dt><samp class="option">font_d</samp></dt>
<dd> the descent size defined in the font metrics

</dd>
<dt><samp class="option">top_a</samp></dt>
<dd> the maximum ascender of the glyphs of the first text line

</dd>

<dt><samp class="option">bottom_d</samp></dt>

<dd> the maximum descender of the glyphs of the last text line

</dd>

<dt><samp class="option">n</samp></dt>

<dd> the number of input frame, starting from 0

</dd>

<dt><samp class="option">rand(min, max)</samp></dt>

<dd> return a random number included between <var class="var">min</var> and <var class="var">max</var>

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> The input sample aspect ratio.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds, NAN if the input timestamp is unknown

</dd>

<dt><samp class="option">text_h, th</samp></dt>

<dd> the height of the rendered text

</dd>

<dt><samp class="option">text_w, tw</samp></dt>

<dd> the width of the rendered text

</dd>

<dt><samp class=”option”>x</samp></dt>

<dt><samp class=”option”>y</samp></dt>

<dd> the x and y offset coordinates where the text is drawn.

These parameters allow the <var class=”var”>x</var> and <var class=”var”>y</var> expressions to refer

to each other, so you can for example specify <code class=”code”>y=x/dar</code>.

</dd>

<dt><samp class=”option”>pict_type</samp></dt>

<dd> A one character description of the current frame’s picture type.

</dd>

<dt><samp class=”option”>pkt_pos</samp></dt>

<dd> The current packet’s position in the input file or stream

(in bytes, from the start of the input). A value of -1 indicates

this info is not available.

</dd>

<dt><samp class=”option”>duration</samp></dt>

<dd> The current packet’s duration, in seconds.

</dd>

<dt><samp class=”option”>pkt_size</samp></dt>

<dd> The current packet’s size (in bytes).

</dd>

</dl>

 <h4 class=”subsection”>11.76.2 Text expansion# TOC</h4>

If <samp class="option">expansion</samp> is set to <code class="code">strftime</code>, the filter recognizes

sequences accepted by the <code class="code">strftime</code> C function in the provided

text and expands them accordingly. Check the documentation of

<code class="code">strftime</code>. This feature is deprecated in favor of <code class="code">normal</code>

expansion with the <code class="code">gmtime</code> or <code class="code">localtime</code> expansion

functions.

If <samp class="option">expansion</samp> is set to <code class="code">none</code>, the text is printed verbatim.

If <samp class="option">expansion</samp> is set to <code class="code">normal</code> (which is the default),

the following expansion mechanism is used.

The backslash character ‘<samp class="samp">\</samp>’, followed by any character, always expands to

the second character.

Sequences of the form <code class="code">%{...}</code> are expanded. The text between the

braces is a function name, possibly followed by arguments separated by ’:’.

If the arguments contain special characters or delimiters (’:’ or ’}’),

they should be escaped.

Note that they probably must also be escaped as the value for the <samp class="option">text</samp>

option in the filter argument string and as the filter argument in the

filtergraph description, and possibly also for the shell, that makes up to four

levels of escaping; using a text file with the <samp class="option">textfile</samp> option avoids

these problems.

The following functions are available:

<dl class="table">

<dt><code class="command">expr, e</code></dt>

<dd> The expression evaluation result.

It must take one argument specifying the expression to be evaluated,

which accepts the same constants and functions as the <var class="var">x</var> and

<var class="var">y</var> values. Note that not all constants should be used, for

example the text size is not known when evaluating the expression, so

the constants <var class="var">text_w</var> and <var class="var">text_h</var> will have an undefined

value.

</dd>

<dt><code class="command">expr_int_format, eif</code></dt>

<dd> Evaluate the expression’s value and output as formatted integer.

The first argument is the expression to be evaluated, just as for the <var class="var">expr</var> function.

The second argument specifies the output format. Allowed values are ‘<samp class="samp">x</samp>’,

‘<samp class="samp">X</samp>’, ‘<samp class="samp">d</samp>’ and ‘<samp class="samp">u</samp>’. They are treated exactly as in the

<code class="code">printf</code> function.

The third parameter is optional and sets the number of positions taken by the output.

It can be used to add padding with zeros from the left.

</dd>

<dt><code class="command">gmtime</code></dt>

<dd> The time at which the filter is running, expressed in UTC.

It can accept an argument: a <code class="code">strftime</code> C function format string.

The format string is extended to support the variable <var class="var">%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class="command">localtime</code></dt>

<dd> The time at which the filter is running, expressed in the local time zone.

It can accept an argument: a <code class="code">strftime</code> C function format string.

The format string is extended to support the variable <var class="var">%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class="command">metadata</code></dt>

<dd> Frame metadata. Takes one or two arguments.

The first argument is mandatory and specifies the metadata key.

The second argument is optional and specifies a default value, used when the

metadata key is not found or empty.

Available metadata can be identified by inspecting entries

starting with TAG included within each frame section

printed by running <code class=”code”>ffprobe -show_frames</code>.

String metadata generated in filters leading to

the drawtext filter are also available.

</dd>

<dt><code class=”command”>n, frame_num</code></dt>

<dd> The frame number, starting from 0.

</dd>

<dt><code class=”command”>pict_type</code></dt>

<dd> A one character description of the current picture type.

</dd>

<dt><code class=”command”>pts</code></dt>

<dd> The timestamp of the current frame.

It can take up to three arguments.

The first argument is the format of the timestamp; it defaults to <code class=”code”>flt</code>

for seconds as a decimal number with microsecond accuracy; <code class=”code”>hms</code> stands

for a formatted <var class=”var”>[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy.

<code class=”code”>gmtime</code> stands for the timestamp of the frame formatted as UTC time;

<code class=”code”>localtime</code> stands for the timestamp of the frame formatted as

local time zone time.

The second argument is an offset added to the timestamp.

If the format is set to <code class="code">hms</code>, a third argument <code class="code">24HH</code> may be

supplied to present the hour part of the formatted timestamp in 24h format

(00-23).

If the format is set to <code class="code">localtime</code> or <code class="code">gmtime</code>, a third

argument may be supplied: a <code class="code">strftime</code> C function format string.

By default, <var class="var">YYYY-MM-DD HH:MM:SS</var> format will be used.

</dd>

</dl>

 <h4 class="subsection">11.76.3 Commands# TOC</h4>

This filter supports altering parameters via commands: <dl class="table"> <dt><samp class="option">reinit</samp></dt> <dd> Alter existing filter parameters. Syntax for the argument is the same as for filter invocation, e.g.   <pre class="example-preformatted">fontsize=56:fontcolor=green:text='Hello World' </pre>

Full filter invocation with sendcmd would look like this: <pre class="example-preformatted">sendcmd=c='56.0 drawtext reinit fontsize=56\:fontcolor=green\:text=Hello\\ World' </pre>

If the entire argument can’t be parsed or applied as valid values then the filter will continue with its existing parameters. </dd> </dl>

The following options are also supported as commands: <ul class="itemize mark-bullet"> x y alpha fontsize fontcolor boxcolor bordercolor shadowcolor box boxw boxh boxborderw line_spacing text_align shadowx shadowy borderw

 <h4 class="subsection">11.76.4 Examples<a class="anchor hidden-xs" href="#Examples-

72” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Draw "Test Text" with font FreeSerif, using the default values for the optional parameters.

 <pre class=”example-preformatted”>drawtext="fontfile=/usr/share/fonts/true
text='Test Text'" </pre>

Draw ’Test Text’ with font FreeSerif of size 24 at position x=100 and y=50 (counting from the top-left corner of the screen), text is yellow with a red box around it. Both the text and the box have an opacity of 20%.

 <pre class=”example-preformatted”>drawtext="fontfile=/usr/share/fonts/true
text='Test Text':\ x=100: y=50: fontsize=24: fontcolor=yellow@0.2: box=1:
boxcolor=red@0.2" </pre>

Note that the double quotes are not necessary if spaces are not used within the parameter list. Show the text at the center of the video frame: <pre class=”example-preformatted”>drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=(w-text_w)/2:y=(h-text_h)/2" </pre>

Show the text at a random position, switching to a new position every 30 seconds: <pre class=”example-preformatted”>drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=if(eq(mod(t\,30)\,0)\,rand(0\,(w-text_w))\,x):y=if(eq(mod(t\,30)\,0)\,rand(0\,(h-text_h))\,y)" </pre>

Show a text line sliding from right to left in the last row of the video frame. The file <samp class=”file”>LONG_LINE</samp> is assumed to contain a single line with no newlines. <pre class=”example-preformatted”>drawtext="fontsize=15:fontfile=FreeSerif.ttf:text=LONG_LINE:y=h-line_h:x=-50*t" </pre>

Show the content of file <samp class=”file”>CREDITS</samp> off the bottom of the frame and scroll up. <pre class=”example-preformatted”>drawtext="fontsize=20:fontfile=FreeSerif.ttf:textfile=CREDITS:y=h-20*t" </pre>

Draw a single green letter "g", at the center of the input video. The glyph baseline is placed at half screen height. <pre class=”example-preformatted”>drawtext="fontsize=60:fontfile=FreeSerif.ttf:fo
max_glyph_w)/2:y=h/2-ascent" </pre>

Show text for 1 second every 3 seconds: <pre class=”example-preformatted”>drawtext="fontfile=FreeSerif.ttf:fontcolor=white:x=100:y=x/dar:enable
</pre>

Use fontconfig to set the font. Note that the colons need to be escaped.

 <pre class=”example-preformatted”>drawtext='fontfile=Linux Libertine O-40\\:style=Semibold:text=FFmpeg' </pre>

Draw "Test Text" with font size dependent on height of the video. <pre class=”example-preformatted”>drawtext="text='Test Text': fontsize=h/30: x=(w-text_w)/2: y=(h-text_h*2)" </pre>

Print the date of a real-time encoding (see documentation for the <code class=”code”>strftime</code> C function): <pre class=”example-preformatted”>drawtext='fontfile=FreeSans.ttf:text=%{localtime\:%a %b %d %Y}' </pre>

Show text fading in and out (appearing/disappearing): <pre class=”example-preformatted”>#!/bin/sh DS=1.0 # display start DE=10.0 # display end FID=1.5 # fade in duration FOD=5 # fade out duration ffplay -f lavfi "color,drawtext=text=TEST:fontsize=50:fontfile=FreeSerif.ttf:fontcolor_
clip(255*(1*between(t\\, $DS + $FID\\, $DE - $FOD) + ((t - $DS)/$FID)*between(t\\, $DS\\, $DS + $FID) + (-(t - $DE)/$FOD)*between(t\\, $DE - $FOD\\, $DE))\\, 0\\, 255) \\\\: x\\\\: 2 }" </pre>

Horizontally align multiple separate texts. Note that <samp class=”option”>max_glyph_a</samp> and the <samp class=”option”>fontsize</samp> value are included in the <samp class=”option”>y</samp> offset. <pre class=”example-preformatted”>drawtext=fontfile=FreeSans.ttf:text=DOG:fontsize=
max_glyph_a, drawtext=fontfile=FreeSans.ttf:text=cow:fontsize=24:x=80:y=20+24-max_glyph_a </pre>

Plot special <var class=”var”>lavf.image2dec.source_basename</var> metadata onto each frame if such metadata exists. Otherwise, plot the string "NA". Note that image2 demuxer must have option <samp class=”option”>-export_path_metadata 1</samp> for the special metadata fields to be available for filters. <pre class=”example-preformatted”>drawtext="fontsize=20:fontcolor=white:fontfile=FreeSans.ttf:text='%{metadata\:lavf.ir
</pre>

For more information about libfreetype, check: http://www.freetype.org/. For more information about fontconfig, check: <a class=”url” href=”http://freedesktop.org/software/fontc
user.html”>http://freedesktop.org/software/fontconfig/fontconfig-user.html. For more information about libfribidi, check: http://fribidi.org/. For more information about libharfbuzz, check: https://git
  <h3 class=”section”>11.77 drawvg# TOC</h3>

Draw vector graphics on top of video frames, by executing a script written in a custom language called VGS (Vector Graphics Script). The documentation for the language can be found in <a data-manual="drawvg-reference" href="./drawvg-reference.html#Top">drawvg - Language Reference. A version of this reference with rendered examples can be found at the author’s site. Graphics are rendered using the cario 2D graphics library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-cairo</code>. <h4 class="subsection">11.77.1 Parameters# TOC</h4>

Either <code class="code">script</code> or <code class="code">file</code> must be set. <dl class="table"> <dt><samp class="option">s, script</samp></dt> <dd> Script source to draw the graphics. </dd> <dt><samp class="option">file</samp></dt> <dd> Path of the file to load the script source. </dd> </dl>

 <h4 class="subsection">11.77.2 Pixel Formats# TOC</h4>

Since Cairo only supports RGB images, if the input video is something else (like YUV 4:2:0), before executing the script the video is converted to a format compatible with Cairo. Then, you have to use either the format filter, or the <code class="code">-pix_fmt</code> option, to convert it to the expected format in the output. <h4 class="subsection">11.77.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the outline of an ellipse.

 <pre class="example-preformatted">ffmpeg -i input.webm \ -vf 'drawvg=ellipse (w/2) (h/2) (w/3) (h/3) stroke' \ -pix_fmt yuv420p \ output.webm </pre>

 Draw a square rotating in the middle of the frame.

The script for drawvg is in a file <code class="code">draw.vgs</code>:   <pre class="example-preformatted">translate (w/2) (h/2) rotate t rect -100 -100 200 200 setcolor red@0.5 fill </pre>

Then: <pre class="example-preformatted">ffmpeg -i input.webm -vf 'drawvg=file=draw.vgs,format=yuv420p' output.webm

</pre>

 <h3 class="section">11.78 edgedetect# TOC</h3>

Detect and draw edges. The filter uses the Canny Edge Detection algorithm. The filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">20/255</code>, and default value for <var class="var">high</var> is <code class="code">50/255</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Define the drawing mode. <dl class="table"> <dt>‘<samp class="samp">wires</samp>’</dt> <dd> Draw white/gray wires on black background. </dd> <dt>‘<samp class="samp">colormix</samp>’</dt> <dd> Mix the colors to create a paint/cartoon effect. </dd> <dt>‘<samp class="samp">canny</samp>’</dt> <dd> Apply Canny edge detector on all selected planes. </dd> </dl> Default value is <var class="var">wires</var>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Select planes for filtering. By default all available planes are filtered. </dd> </dl>

 <h4 class="subsection">11.78.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Standard edge detection with custom values for the hysteresis thresholding: <pre class="example-preformatted">edgedetect=low=0.1:high=0.4 </pre>

Painting effect without thresholding: <pre class="example-preformatted">edgedetect=mode=colormix:high=0 </pre>

 <h3 class="section">11.79 elbg# TOC</h3>

Apply a posterize effect using the ELBG (Enhanced LBG) algorithm. For each input image, the filter will compute the optimal mapping from the input to the output given the codebook length, that is the number of distinct output colors. This filter accepts the following options. <dl class="table"> <dt><samp class="option">codebook_length, l</samp></dt> <dd> Set codebook length. The value must be a positive integer, and represents the number of distinct output colors. Default value is 256. </dd> <dt><samp class="option">nb_steps, n</samp></dt> <dd> Set the maximum number of iterations to apply for computing the optimal mapping. The higher the value the better the result and the higher the computation time. Default value is 1. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set a random seed, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">pal8</samp></dt> <dd> Set pal8 output pixel format. This option does not work with codebook length greater than 256. Default is disabled. </dd> <dt><samp class="option">use_alpha</samp></dt> <dd> Include alpha values in the quantization calculation. Allows creating palettized output images (e.g. PNG8) with multiple alpha smooth blending. </dd> </dl>

 <h3 class="section">11.80 entropy# TOC</h3>

Measure graylevel entropy in histogram of color channels of video frames. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Can be either <var class="var">normal</var> or <var class="var">diff</var>. Default is <var class="var">normal</var>. <var class="var">diff</var> mode measures entropy of histogram delta values, absolute differences between neighbour histogram values. </dd> </dl>

 <h3 class="section">11.81 epx# TOC</h3> Apply the EPX magnification filter which is designed for pixel art. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xEPX</code>, <code class="code">3</code> for <code class="code">3xEPX</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">11.82 eq# TOC</h3> Set brightness, contrast, saturation and approximate gamma adjustment.

 The filter accepts the following options: <dl class="table">
<dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. The value must be a float value in range <code class="code">-1000.0</code> to <code class="code">1000.0</code>. The default value is "1". </dd> <dt><samp class="option">brightness</samp></dt>
<dd> Set the brightness expression. The value must be a float value in range <code class="code">-1.0</code> to <code class="code">1.0</code>. The default value is "0". </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the saturation expression. The value must be a float in range <code class="code">0.0</code> to <code class="code">3.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma</samp></dt>
<dd> Set the gamma expression. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_r</samp></dt>
<dd> Set the gamma expression for red. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_g</samp></dt>
<dd> Set the gamma expression for green. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_b</samp></dt>
<dd> Set the gamma expression for blue. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_weight</samp></dt>
<dd> Set the gamma weight expression. It can be used to reduce the effect of a high gamma value on bright image areas, e.g. keep them from getting overamplified and just plain white. The value must be a float in range <code class="code">0.0</code> to <code class="code">1.0</code>. A value of <code class="code">0.0</code> turns the gamma correction all the way down while <code class="code">1.0</code> leaves it at its full strength. Default is "1". </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for brightness, contrast, saturation and gamma expressions are evaluated. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt>
<dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.
</dd> </dl>

The expressions accept the following parameters: <dl class="table">
<dt><samp class="option">n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class="option">pos</samp></dt>
<dd> byte position of the corresponding packet in the input file, NAN if unspecified; deprecated, do not use </dd> <dt><samp class="option">r</samp></dt> <dd> frame rate of the input video,

NAN if the input frame rate is unknown </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> </dl>

 <h4 class="subsection">11.82.1 Commands# TOC</h4> The filter supports the following commands: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. </dd> <dt><samp class="option">brightness</samp></dt> <dd> Set the brightness expression. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the saturation expression. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Set the gamma expression. </dd> <dt><samp class="option">gamma_r</samp></dt> <dd> Set the gamma_r expression. </dd> <dt><samp class="option">gamma_g</samp></dt> <dd> Set gamma_g expression. </dd> <dt><samp class="option">gamma_b</samp></dt> <dd> Set gamma_b expression. </dd> <dt><samp class="option">gamma_weight</samp></dt> <dd> Set gamma_weight expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">11.83 erosion# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class="subsection">11.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.84 estdif# <a class="anchor hidden-xs"href="#toc-estdif"

aria-hidden=”true”>TOC</h3>

Deinterlace the input video ("estdif" stands for "Edge Slope Tracing Deinterlacing Filter"). Spatial only filter that uses edge slope tracing algorithm to interpolate missing lines. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> <dt><samp class=”option”>rslope</samp></dt> <dd> Specify the search radius for edge slope tracing. Default value is 1. Allowed range is from 1 to 15. </dd> <dt><samp class=”option”>redge</samp></dt> <dd> Specify the search radius for best edge matching. Default value is 2. Allowed range is from 0 to 15. </dd> <dt><samp class=”option”>ecost</samp></dt> <dd> Specify the edge cost for edge matching. Default value is 2. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>mcost</samp></dt> <dd> Specify the middle cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>dcost</samp></dt> <dd> Specify the distance cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the interpolation used. Default is 4-point interpolation. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>2p</samp></dt> <dd> Two-point in-

terpolation. </dd> <dt><samp class="option">4p</samp></dt>
<dd> Four-point interpolation. </dd> <dt><samp class="option">6p</samp></dt>
<dd> Six-point interpolation. </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.84.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">11.85 exposure# TOC</h3> Adjust exposure of the video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">exposure</samp></dt>
<dd> Set the exposure correction in EV. Allowed range is from -3.0 to 3.0 EV Default value is 0 EV. </dd> <dt><samp class="option">black</samp></dt> <dd> Set the black level correction. Allowed range is from -1.0 to 1.0. Default value is 0. </dd>
</dl>

 <h4 class="subsection">11.85.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">11.86 extractplanes# TOC</h3>

Extract color channel components from input video stream into separate grayscale video streams. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt>
<dd> Set plane(s) to extract. Available values for planes are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt>
<dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt>
<dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt>
</dl>

Choosing planes not available in the input will result in an error. That means you cannot select <code class="code">r</code>, <code class="code">g</code>, <code class="code">b</code> planes with <code class="code">y</code>, <code class="code">u</code>, <code class="code">v</code> planes at same time. </dd> </dl>

 <h4 class="subsection">11.86.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract luma, u and v color channel component from input video frame into 3 grayscale outputs: <pre class="example-preformatted">ffmpeg -i video.avi -filter_complex 'extractplanes=y+u+v[y][u][v]' -map '[y]' y.avi -map '[u]' u.avi -map '[v]' v.avi </pre>

 <h3 class="section">11.87 fade# TOC</h3>

Apply a fade-in/out effect to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">type, t</samp></dt> <dd> The effect type can be either "in" for a fade-in, or "out" for a fade-out effect. Default is <code class="code">in</code>. </dd> <dt><samp class="option">start_frame, s</samp></dt> <dd> Specify the number of the frame to start applying the fade effect at. Default is 0. </dd> <dt><samp class="option">nb_frames, n</samp></dt> <dd> The number of frames that the fade effect lasts. At the end of the fade-in effect, the output video will have the same intensity as the input video. At the end of the fade-out transition, the output video will be filled with the selected <samp class="option">color</samp>. Default is 25. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If set to 1, fade only alpha channel, if one exists on the input. Default value is 0. </dd> <dt><samp class="option">start_time, st</samp></dt> <dd> Specify the timestamp (in seconds) of the frame to start to apply the fade effect. If both start_frame and start_time are specified, the fade will start at whichever comes last. Default is 0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> The number of seconds for which the fade effect has to last. At the end of the fade-in effect the output video will have the same intensity as the input video, at the end of the fade-out transition the output video will be filled with the selected <samp class="option">color</samp>. If both duration and nb_frames are specified, duration is used. Default is 0 (nb_frames is used by default). </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the fade. Default is "black". </dd> </dl>

 <h4 class="subsection">11.87.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in the first 30 frames of video:

 <pre class=”example-preformatted”>fade=in:0:30 </pre>

The command above is equivalent to: <pre class=”example-preformatted”>fade=t=in:s=0:n=30 </pre>

Fade out the last 45 frames of a 200-frame video: <pre class=”example-preformatted”>fade=out:155:45 fade=type=out:start_frame=155:nb_frames=45 </pre>

Fade in the first 25 frames and fade out the last 25 frames of a 1000-frame video: <pre class=”example-preformatted”>fade=in:0:25, fade=out:975:25 </pre>

Make the first 5 frames yellow, then fade in from frame 5-24: <pre class=”example-preformatted”>fade=in:5:20:color=yellow </pre>

Fade in alpha over first 25 frames of video: <pre class=”example-preformatted”>fade=in:0:25:alpha=1 </pre>

Make the first 5.5 seconds black, then fade in for 0.5 seconds: <pre class=”example-preformatted”>fade=t=in:st=5.5:d=0.5 </pre>

 <h3 class=”section”>11.88 feedback# TOC</h3> Apply feedback video filter. This filter pass cropped input frames to 2nd output. From there it can be filtered with other video filters. After filter receives frame from 2nd input, that frame is combined on top of original frame from 1st input and passed to 1st output. The typical usage is filter only part of frame. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the top left crop position. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the crop size. </dd> </dl>

 <h4 class=”subsection”>11.88.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Blur only top left rectangular part of video frame size 100x100 with gblur filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]gblur=8[bl </pre>

Draw black box on top left part of video frame of size 100x100 with drawbox filter. <pre class="example-preformatted">[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]drawbox=x=0:y=0:w=100:
</pre>

Pixelize rectangular part of video frame of size 100x100 with pixelize filter. <pre class="example-preformatted">[in][blurin]feedback=x=320:y=240:w=100
</pre>

 <h3 class="section">11.89 fftdnoiz# TOC</h3> Denoise frames using 3D FFT (frequency domain filtering). The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma constant. This sets denoising strength. Default value is 1. Allowed range is from 0 to 30. Using very high sigma with low overlap may give blocking artifacts. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set amount of denoising. By default all detected noise is reduced. Default value is 1. Allowed range is from 0 to 1. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block in pixels, Default is 32, can be 8 to 256. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set block overlap. Default is 0.5. Allowed range is from 0.2 to 0.8. </dd> <dt><samp class="option">method</samp></dt> <dd> Set denoising method. Default is <code class="code">wiener</code>, can also be <code class="code">hard</code>. </dd> <dt><samp class="option">prev</samp></dt> <dd> Set number of previous frames to use for denoising. By default is set to 0. </dd> <dt><samp class="option">next</samp></dt> <dd> Set number of next frames to to use for denoising. By default is set to 0. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes which will be filtered, by default are all available filtered except alpha. </dd> </dl>

 <h3 class="section">11.90 fftfilt#
TOC</h3>
Apply arbitrary expressions to samples in frequency domain
<dl class="table"> <dt><samp class="option">dc_Y</samp></dt>
<dd> Adjust the dc value (gain) of the luma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_U</samp></dt>
<dd> Adjust the dc value (gain) of the 1st chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code

class=”code”>0</code>. </dd> <dt><samp class=”option”>dc_V</samp></dt>
<dd> Adjust the dc value (gain) of the 2nd chroma plane of the image.
The filter accepts an integer value in range <code class=”code”>0</code>
to <code class=”code”>1000</code>. The default value is set to <code
class=”code”>0</code>. </dd> <dt><samp class=”option”>weight_Y</samp></dt>
<dd> Set the frequency domain weight expression for the luma plane.
 </dd> <dt><samp class=”option”>weight_U</samp></dt>
<dd> Set the frequency domain weight expression for the 1st chroma
plane. </dd> <dt><samp class=”option”>weight_V</samp></dt>
<dd> Set the frequency domain weight expression for the 2nd chroma
plane. </dd> <dt><samp class=”option”>eval</samp></dt>
<dd> Set when the expressions are evaluated. It ac-
cepts the following values: <dl class=”table”> <dt>‘<samp
class=”samp”>init</samp>’</dt> <dd> Only evaluate expres-
sions once during the filter initialization. </dd> <dt>‘<samp
class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions
for each incoming frame. </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’.
 The filter accepts the following variables: </dd> <dt><samp
class=”option”>X</samp></dt> <dt><samp class=”option”>Y</samp></dt>
<dd> The coordinates of the current sample. </dd> <dt><samp
class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt>
<dd> The width and height of the image. </dd> <dt><samp
class=”option”>N</samp></dt> <dd> The number of input frame,
starting from 0. </dd> <dt><samp class=”option”>WS</samp></dt>
<dt><samp class=”option”>HS</samp></dt> <dd> The size of FFT
array for horizontal and vertical processing. </dd> </dl>

 <h4 class=”subsection”>11.90.1 Exam-
ples<a class=”anchor hidden-xs” href=”#Examples-
78” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Examples-78” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> High-pass:
<pre class=”example-preformatted”>fftfilt=dc_Y=128:weight_Y='squish(1-
(Y+X)/100)' </pre>

Low-pass: <pre class=”example-
preformatted”>fftfilt=dc_Y=0:weight_Y='squish((Y+X)/100-1)' </pre>

Sharpen: <pre class=”example-preformatted”>fftfilt=dc_Y=0:weight_Y=
(Y+X)/100)' </pre>

Blur: <pre class=”example-preformatted”>fftfilt=dc_Y=0:weight_Y='exp
4 * ((Y+X)/(W+H)))' </pre>

 <h3 class="section">11.91 field#
TOC</h3>

Extract a single field from an interlaced image using stride arithmetic to avoid wasting CPU time. The output frames are marked as non-interlaced. The filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Specify whether to extract the top (if the value is <code class="code">0</code> or <code class="code">top</code>) or the bottom field (if the value is <code class="code">1</code> or <code class="code">bottom</code>). </dd> </dl>

 <h3 class="section">11.92 fieldhint# TOC</h3>

Create new frames by copying the top and bottom fields from surrounding frames supplied as numbers by the hint file. <dl class="table"> <dt><samp class="option">hint</samp></dt> <dd> Set file containing hints: absolute/relative frame numbers. There must be one line for each frame in a clip. Each line must contain two numbers separated by the comma, optionally followed by <code class="code">-</code> or <code class="code">+</code>. Numbers supplied on each line of file can not be out of [N-1,N+1] where N is current frame number for <code class="code">absolute</code> mode or out of [-1, 1] range for <code class="code">relative</code> mode. First number tells from which frame to pick up top field and second number tells from which frame to pick up bottom field. If optionally followed by <code class="code">+</code> output frame will be marked as interlaced, else if followed by <code class="code">-</code> output frame will be marked as progressive, else it will be marked same as input frame. If optionally followed by <code class="code">t</code> output frame will use only top field, or in case of <code class="code">b</code> it will use only bottom field. If line starts with <code class="code">#</code> or <code class="code">;</code> that line is skipped. </dd> <dt><samp class="option">mode</samp></dt> <dd> Can be item <code class="code">absolute</code> or <code class="code">relative</code> or <code class="code">pattern</code>. Default is <code class="code">absolute</code>. The <code class="code">pattern</code> mode is same as <code class="code">relative</code> mode, except at last entry of file if there are more frames to process than <code class="code">hint</code> file is seek back to start. </dd> </dl>

Example of first several lines of <code class="code">hint</code> file for <code class="code">relative</code> mode: <pre class="example-preformatted">0,0 - # first frame 1,0 - # second frame, use third's frame top field and second's frame bottom field 1,0 - # third frame,

use fourth's frame top field and third's frame bottom field 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - </pre>

 <h3 class="section">11.93 fieldmatch# TOC</h3>

Field matching filter for inverse telecine. It is meant to reconstruct the progressive frames from a telecined stream. The filter does not drop duplicated frames, so to achieve a complete inverse telecine <code class="code">fieldmatch</code> needs to be followed by a decimation filter such as decimate in the filtergraph. The separation of the field matching and the decimation is notably motivated by the possibility of inserting a de-interlacing filter fallback between the two. If the source has mixed telecined and real interlaced content, <code class="code">fieldmatch</code> will not be able to match fields for the interlaced parts. But these remaining combed frames will be marked as interlaced, and thus can be de-interlaced by a later filter such as yadif before decimation. In addition to the various configuration options, <code class="code">fieldmatch</code> can take an optional second stream, activated through the <samp class="option">ppsrc</samp> option. If enabled, the frames reconstruction will be based on the fields and frames from this second stream. This allows the first input to be pre-processed in order to help the various algorithms of the filter, while keeping the output lossless (assuming the fields are matched properly). Typically, a field-aware denoiser, or brightness/contrast adjustments can help. Note that this filter uses the same algorithms as TIVTC/TFM (AviSynth project) and VIVTC/VFM (VapourSynth project). The later is a light clone of TFM from which <code class="code">fieldmatch</code> is based on. While the semantic and usage are very close, some behaviour and options names can differ. The decimate filter currently only works for constant frame rate input. If your input has mixed telecined (30fps) and progressive content with a lower framerate like 24fps use the following filterchain to produce the necessary cfr stream: <code class="code">dejudder,fps=30000/1001,fieldmatch,decimate</code>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Specify the assumed field order of the input stream. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Auto detect parity (use FFmpeg’s internal parity value). </dd> <dt>‘<samp class="samp">bff</samp>’</dt> <dd> Assume bottom field first. </dd> <dt>‘<samp class="samp">tff</samp>’</dt> <dd> Assume top field first. </dd> </dl>

Note that it is sometimes recommended not to trust the parity announced by the stream. Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the matching mode or strategy to use. <samp class="option">pc</samp> mode is the safest in the sense that it won’t risk creating jerkiness due to duplicate frames when possible, but if there are bad edits or blended fields it will end up outputting combed frames when a good match might actually exist. On the other hand, <samp class="option">pcn_ub</samp> mode is the most risky in terms of creating jerkiness, but will almost always find a good frame if there is one. The other values are all somewhere in between <samp class="option">pc</samp> and <samp class="option">pcn_ub</samp> in terms of risking jerkiness and creating duplicate frames versus finding good matches in sections with bad edits, orphaned fields, blended fields, etc. More details about p/c/n/u/b are available in p/c/n/u/b meaning section. Available values are: <dl class="table"> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> 2-way matching (p/c) </dd> <dt>‘<samp class="samp">pc_n</samp>’</dt> <dd> 2-way matching, and trying 3rd match if still combed (p/c + n) </dd> <dt>‘<samp class="samp">pc_u</samp>’</dt> <dd> 2-way matching, and trying 3rd match (same order) if still combed (p/c + u) </dd> <dt>‘<samp class="samp">pc_n_ub</samp>’</dt> <dd> 2-way matching, trying 3rd match if still combed, and trying 4th/5th matches if still combed (p/c + n + u/b) </dd> <dt>‘<samp class="samp">pcn</samp>’</dt> <dd> 3-way matching (p/c/n) </dd> <dt>‘<samp class="samp">pcn_ub</samp>’</dt> <dd> 3-way matching, and trying 4th/5th matches if all 3 of the original matches are detected as combed (p/c/n + u/b) </dd> </dl>

The parenthesis at the end indicate the matches that would be used for that mode assuming <samp class="option">order</samp>=<var class="var">tff</var> (and <samp class="option">field</samp> on <var class="var">auto</var> or <var class="var">top</var>). In terms of speed <samp class="option">pc</samp> mode is by far the fastest and <samp class="option">pcn_ub</samp> is the slowest. Default value is <var class="var">pc_n</var>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark the main input stream as a pre-processed input, and enable the secondary input stream as the clean source to pick the fields from. See the filter introduction for more details. It is similar to the <samp class="option">clip2</samp> feature from VFM/TFM. Default value is <code class="code">0</code> (disabled). </dd> <dt><samp class="option">field</samp></dt> <dd> Set the field to match from. It is recommended to set this to the same value as <samp class="option">order</samp> unless you experience matching failures with that setting. In certain circumstances changing the field that is used to match from can have a large impact on matching perfor-

mance. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic (same value as <samp class="option">order</samp>). </dd> <dt>‘<samp class="samp">bottom</samp>’</dt> <dd> Match from the bottom field. </dd> <dt>‘<samp class="samp">top</samp>’</dt> <dd> Match from the top field. </dd> </dl>

Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mchroma</samp></dt> <dd> Set whether or not chroma is included during the match comparisons. In most cases it is recommended to leave this enabled. You should set this to <code class="code">0</code> only if your clip has bad chroma problems such as heavy rainbowing or other artifacts. Setting this to <code class="code">0</code> could also be used to speed things up at the cost of some accuracy. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">y0</samp></dt> <dt><samp class="option">y1</samp></dt> <dd> These define an exclusion band which excludes the lines between <samp class="option">y0</samp> and <samp class="option">y1</samp> from being included in the field matching decision. An exclusion band can be used to ignore subtitles, a logo, or other things that may interfere with the matching. <samp class="option">y0</samp> sets the starting scan line and <samp class="option">y1</samp> sets the ending line; all lines in between <samp class="option">y0</samp> and <samp class="option">y1</samp> (including <samp class="option">y0</samp> and <samp class="option">y1</samp>) will be ignored. Setting <samp class="option">y0</samp> and <samp class="option">y1</samp> to the same value will disable the feature. <samp class="option">y0</samp> and <samp class="option">y1</samp> defaults to <code class="code">0</code>. </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change on the luma plane. Good values are in the <code class="code">[8.0, 14.0]</code> range. Scene change detection is only relevant in case <samp class="option">combmatch</samp>=<var class="var">sc</var>. The range for <samp class="option">scthresh</samp> is <code class="code">[0.0, 100.0]</code>. Default value is <code class="code">12.0</code>. </dd> <dt><samp class="option">combmatch</samp></dt> <dd> When <samp class="option">combatch</samp> is not <var class="var">none</var>, <code class="code">fieldmatch</code> will take into account the combed scores of matches when deciding what match to use as the final match. Available values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No final matching based on combed scores. </dd> <dt>‘<samp class="samp">sc</samp>’</dt> <dd> Combed scores are only used when a scene change is detected. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Use combed scores all the time. </dd> </dl>

Default is <var class=”var”>sc</var>. </dd> <dt><samp class=”option”>combdbg</samp></dt> <dd> Force <code class=”code”>fieldmatch</code> to calculate the combed metrics for certain matches and print them. This setting is known as <samp class=”option”>micout</samp> in TFM/VFM vocabulary. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No forced calculation. </dd> <dt>‘<samp class=”samp”>pcn</samp>’</dt> <dd> Force p/c/n calculations. </dd> <dt>‘<samp class=”samp”>pcnub</samp>’</dt> <dd> Force p/c/n/u/b calculations. </dd> </dl>

Default value is <var class=”var”>none</var>. </dd> <dt><samp class=”option”>cthresh</samp></dt> <dd> This is the area combing threshold used for combed frame detection. This essentially controls how "strong" or "visible" combing must be to be detected. Larger values mean combing must be more visible and smaller values mean combing can be less visible or strong and still be detected. Valid settings are from <code class=”code”>-1</code> (every pixel will be detected as combed) to <code class=”code”>255</code> (no pixel will be detected as combed). This is basically a pixel difference value. A good range is <code class=”code”>[8, 12]</code>. Default value is <code class=”code”>9</code>. </dd> <dt><samp class=”option”>chroma</samp></dt> <dd> Sets whether or not chroma is considered in the combed frame decision. Only disable this if your source has chroma problems (rainbowing, etc.) that are causing problems for the combed frame detection with chroma enabled. Actually, using <samp class=”option”>chroma</samp>=<var class=”var”>0</var> is usually more reliable, except for the case where there is chroma only combing in the source. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>blockx</samp></dt> <dt><samp class=”option”>blocky</samp></dt> <dd> Respectively set the x-axis and y-axis size of the window used during combed frame detection. This has to do with the size of the area in which <samp class=”option”>combpel</samp> pixels are required to be detected as combed for a frame to be declared combed. See the <samp class=”option”>combpel</samp> parameter description for more info. Possible values are any number that is a power of 2 starting at 4 and going up to 512. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>combpel</samp></dt> <dd> The number of combed pixels inside any of the <samp class=”option”>blocky</samp> by <samp class=”option”>blockx</samp> size blocks on the frame for the frame to be detected as combed. While <samp class=”option”>cthresh</samp> controls how "visible" the combing must be, this setting controls "how much" combing there must be in any localized area (a window defined by the <samp class=”option”>blockx</samp> and <samp class=”option”>blocky</samp> settings) on the frame. Minimum value is

<code class="code">0</code> and maximum is <code class="code">blocky x blockx</code> (at which point no frames will ever be detected as combed). This setting is known as <samp class="option">MI</samp> in TFM/VFM vocabulary. Default value is <code class="code">80</code>. </dd> </dl>

 <h4 class="subsection">11.93.1 p/c/n/u/b meaning# TOC</h4>

 <h4 class="subsubsection">11.93.1.1 p/c/n# TOC</h4>

We assume the following telecined stream: <pre class="example-preformatted">Top fields: 1 2 2 3 4 Bottom fields: 1 2 3 4 4 </pre>

The numbers correspond to the progressive frame the fields relate to. Here, the first two frames are progressive, the 3rd and 4th are combed, and so on. When <code class="code">fieldmatch</code> is configured to run a matching from bottom (<samp class="option">field</samp>=<var class="var">bottom</var>) this is how this input stream get transformed: <pre class="example-preformatted">Input stream: T 1 2 2 3 4 B 1 2 3 4 4 <-- matching reference

Matches: c c n n c

Output stream: T 1 2 3 4 4 B 1 2 3 4 4 </pre>

As a result of the field matching, we can see that some frames get duplicated. To perform a complete inverse telecine, you need to rely on a decimation filter after this operation. See for instance the decimate filter. The same operation now matching from top fields (<samp class="option">field</samp>=<var class="var">top</var>) looks like this: <pre class="example-preformatted">Input stream: T 1 2 2 3 4 <-- matching reference B 1 2 3 4 4

Matches: c c p p c

Output stream: T 1 2 2 3 4 B 1 2 2 3 4 </pre>

In these examples, we can see what <var class="var">p</var>, <var class="var">c</var> and <var class="var">n</var> mean; basically, they refer to the frame and field of the opposite parity: <ul class="itemize mark-bullet"> <var class="var">p</var> matches the field of the

opposite parity in the previous frame <var class=”var”>c</var> matches the field of the opposite parity in the current frame <var class=”var”>n</var> matches the field of the opposite parity in the next frame

 <h4 class=”subsubsection”>11.93.1.2 u/b# TOC</h4>

The <var class=”var”>u</var> and <var class=”var”>b</var> matching are a bit special in the sense that they match from the opposite parity flag. In the following examples, we assume that we are currently matching the 2nd frame (Top:2, bottom:2). According to the match, a ’x’ is placed above and below each matched fields. With bottom matching (<samp class=”option”>field</samp>=<var class=”var”>bottom</var>): <pre class=”example-preformatted”>Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 1 2 2 2 2 2 2 1 3 </pre>

With top matching (<samp class=”option”>field</samp>=<var class=”var”>top</var>): <pre class=”example-preformatted”>Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 2 2 1 2 2 1 3 2 2 </pre>

 <h4 class=”subsection”>11.93.2 Examples# TOC</h4>

Simple IVTC of a top field first telecined stream: <pre class=”example-preformatted”>fieldmatch=order=tff:combmatch=none, decimate </pre>

Advanced IVTC, with fallback on yadif for still combed frames: <pre class=”example-preformatted”>fieldmatch=order=tff:combmatch=full, yadif=deint=interlaced, decimate </pre>

 <h3 class=”section”>11.94 fieldorder# TOC</h3>

Transform the field order of the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> The output field order. Valid values are <var class=”var”>tff</var> for top field first or <var class=”var”>bff</var> for bottom field first. </dd> </dl>

The default value is ‘<samp class=”samp”>tff</samp>’. The transformation is done by shifting the picture content up or down by one line, and filling the remaining line with appropriate picture content. This method is consistent with most broadcast field order converters. If the input video is not flagged as being interlaced, or it is already flagged as being of the required output field order, then this filter does not alter the incoming video. It is very useful when converting to or from PAL DV material, which is bottom field first. For example: <pre class=”example-preformatted”>ffmpeg -i in.vob -vf "fieldorder=bff" out.dv </pre>

 <h3 class=”section”>11.95 fillborders# TOC</h3>

Fill borders of the input video, without changing video stream dimensions. Sometimes video can have garbage at the four edges and you may not want to crop video input to keep size multiple of some number. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>left</samp></dt> <dd> Number of pixels to fill from left border. </dd> <dt><samp class=”option”>right</samp></dt> <dd> Number of pixels to fill from right border. </dd> <dt><samp class=”option”>top</samp></dt> <dd> Number of pixels to fill from top border. </dd> <dt><samp class=”option”>bottom</samp></dt> <dd> Number of pixels to fill from bottom border. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set fill mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>smear</samp>’</dt> <dd> fill pixels using outermost pixels </dd> <dt>‘<samp class=”samp”>mirror</samp>’</dt> <dd> fill pixels using mirroring (half sample symmetric) </dd> <dt>‘<samp class=”samp”>fixed</samp>’</dt> <dd> fill pixels with constant value </dd> <dt>‘<samp class=”samp”>reflect</samp>’</dt> <dd> fill pixels using reflecting (whole sample symmetric) </dd> <dt>‘<samp class=”samp”>wrap</samp>’</dt> <dd> fill pixels using wrapping </dd> <dt>‘<samp class=”samp”>fade</samp>’</dt> <dd> fade pixels to constant value </dd> <dt>‘<samp class=”samp”>margins</samp>’</dt> <dd> fill pixels at top and bottom with weighted averages pixels near borders </dd> </dl>

Default is <var class=”var”>smear</var>. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Set color for pixels in fixed or fade mode. Default is <var class=”var”>black</var>. </dd> </dl>

 <h4 class=”subsection”>11.95.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>11.96 find_rect# TOC</h3>

Find a rectangular object in the input video. The object to search for must be specified as a gray8 image specified with the <samp class=”option”>object</samp> option. For each possible match, a score is computed. If the score reaches the specified threshold, the object is considered found. If the input video contains multiple instances of the object, the filter will find only one of them. When an object is found, the following metadata entries are set in the matching frame: <dl class=”table”> <dt><samp class=”option”>lavfi.rect.w</samp></dt> <dd> width of object </dd> <dt><samp class=”option”>lavfi.rect.h</samp></dt> <dd> height of object </dd> <dt><samp class=”option”>lavfi.rect.x</samp></dt> <dd> x position of object </dd> <dt><samp class=”option”>lavfi.rect.y</samp></dt> <dd> y position of object </dd> <dt><samp class=”option”>lavfi.rect.score</samp></dt> <dd> match score of the found object </dd> </dl>

It accepts the following options: <dl class=”table”> <dt><samp class=”option”>object</samp></dt> <dd> Filepath of the object image, needs to be in gray8. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Detection threshold, expressed as a decimal number in the range 0-1. A threshold value of 0.01 means only exact matches, a threshold of 0.99 means almost everything matches. Default value is 0.5. </dd> <dt><samp class=”option”>mipmaps</samp></dt> <dd> Number of mipmaps, default is 3. </dd> <dt><samp class=”option”>xmin, ymin, xmax, ymax</samp></dt> <dd> Specifies the rectangle in which to search. </dd> <dt><samp class=”option”>discard</samp></dt> <dd> Discard frames where object is not detected. Default is disabled. </dd> </dl>

 <h4 class=”subsection”>11.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

Find the position of an object in each frame using <code class="command">ffprobe</code> and write it to a log file: <pre class="example-preformatted">ffprobe -f lavfi movie=test.mp4,find_rect=object=object.pgm:threshold=0.3 \ -show_entries frame=pkt_pts_time:frame_tags=lavfi.rect.x,lavfi.rect.y \ -of csv -o find_rect.csv </pre>

 <h3 class="section">11.97 floodfill# TOC</h3>

Flood area with values of same pixel components with another values. It accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set pixel x coordinate. </dd> <dt><samp class="option">y</samp></dt> <dd> Set pixel y coordinate. </dd> <dt><samp class="option">s0</samp></dt> <dd> Set source #0 component value. </dd> <dt><samp class="option">s1</samp></dt> <dd> Set source #1 component value. </dd> <dt><samp class="option">s2</samp></dt> <dd> Set source #2 component value. </dd> <dt><samp class="option">s3</samp></dt> <dd> Set source #3 component value. </dd> <dt><samp class="option">d0</samp></dt> <dd> Set destination #0 component value. </dd> <dt><samp class="option">d1</samp></dt> <dd> Set destination #1 component value. </dd> <dt><samp class="option">d2</samp></dt> <dd> Set destination #2 component value. </dd> <dt><samp class="option">d3</samp></dt> <dd> Set destination #3 component value. </dd> </dl>

 <h3 class="section">11.98 format# TOC</h3>

Convert the input video to one of the specified pixel formats. Libavfilter will try to pick one that is suitable as input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as "pix_fmts=yuv420p|monow|rgb24". </dd> <dt><samp class="option">color_spaces</samp></dt>

<dd> A ’|’-separated list of color space names, such as "color_spaces=bt709|bt470bg|bt2020nc". </dd> <dt><samp class="option">color_ranges</samp></dt> <dd> A ’|’-separated list of color range names, such as "color_ranges=tv|pc". </dd> <dt><samp class="option">alpha_modes</samp></dt> <dd> A ’|’-separated list of color range names, such as "alpha_modes=straight|premultiplied". </dd> <dt><samp class="option">chroma_locations</samp></dt> <dd> A ’|’-separated list of chroma sample location names, such as "chroma_locations=left|topleft". </dd> </dl>

 <h4 class="subsection">11.98.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert the input video to the <var class="var">yuv420p</var> format <pre class="example-preformatted">format=pix_fmts=yuv420p </pre>

Convert the input video to any of the formats in the list <pre class="example-preformatted">format=pix_fmts=yuv420p|yuv444p|yuv410p </pre>

 <h3 class="section">11.99 fps# TOC</h3>

Convert the video to specified constant frame rate by duplicating or dropping frames as necessary. It accepts the following parameters: <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> The desired output frame rate. It accepts expressions containing the following constants: <dl class="table"> <dt>‘<samp class="samp">source_fps</samp>’</dt> <dd> The input’s frame rate </dd> <dt>‘<samp class="samp">ntsc</samp>’</dt> <dd> NTSC frame rate of <code class="code">30000/1001</code> </dd> <dt>‘<samp class="samp">pal</samp>’</dt> <dd> PAL frame rate of <code class="code">25.0</code> </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> Film frame rate of <code class="code">24.0</code> </dd> <dt>‘<samp class="samp">ntsc_film</samp>’</dt> <dd> NTSC-film frame rate of <code class="code">24000/1001</code> </dd> </dl>

The default is <code class="code">25</code>. </dd> <dt><samp class="option">start_time</samp></dt> <dd> Assume the first PTS should be the given value, in seconds. This allows for padding/trimming at the start of stream. By default, no assumption is made

about the first frame’s expected PTS, so no padding or trimming is done. For example, this could be set to 0 to pad the beginning with duplicates of the first frame if a video stream starts after the audio stream or to trim any frames with a negative PTS. </dd> <dt><samp class="option">round</samp></dt> <dd> Timestamp (PTS) rounding method. Possible values are: <dl class="table"> <dt><samp class="option">zero</samp></dt> <dd> round towards 0 </dd> <dt><samp class="option">inf</samp></dt> <dd> round away from 0 </dd> <dt><samp class="option">down</samp></dt> <dd> round towards -infinity </dd> <dt><samp class="option">up</samp></dt> <dd> round towards +infinity </dd> <dt><samp class="option">near</samp></dt> <dd> round to nearest </dd> </dl> The default is <code class="code">near</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> Action performed when reading the last frame. Possible values are: <dl class="table"> <dt><samp class="option">round</samp></dt> <dd> Use same timestamp rounding method as used for other frames. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass through last frame if input duration has not been reached yet. </dd> </dl> The default is <code class="code">round</code>. If timestamp rounding would otherwise discard every frame, one frame is retained when its rounded timestamp equals both the first output timestamp and the rounded end of input. This exception extends the output beyond the rounded end of input by one output frame period. Frames whose entire interval is trimmed by <samp class="option">start_time</samp> remain discarded. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class="var">fps</var>[:<var class="var">start_time</var>[:<var class="var">round</var>]]. See also the setpts filter. <h4 class="subsection">11.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A typical usage in order to set the fps to 25: <pre class="example-preformatted">fps=fps=25 </pre>

Sets the fps to 24, using abbreviation and rounding method to round to nearest: <pre class="example-preformatted">fps=fps=film:round=near </pre>

 <h3 class="section">11.100 framepack# TOC</h3>

Pack two different video streams into a stereoscopic video, setting

proper metadata on supported codecs. The two views should have the same size and framerate and processing will stop when the shorter video ends. Please note that you may conveniently adjust view properties with the scale and fps filters. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>format</samp></dt> <dd> The desired packing format. Supported values are: <dl class=”table”> <dt><samp class=”option”>sbs</samp></dt> <dd> The views are next to each other (default). </dd> <dt><samp class=”option”>tab</samp></dt> <dd> The views are on top of each other. </dd> <dt><samp class=”option”>lines</samp></dt> <dd> The views are packed by line. </dd> <dt><samp class=”option”>columns</samp></dt> <dd> The views are packed by column. </dd> <dt><samp class=”option”>frameseq</samp></dt> <dd> The views are temporally interleaved. </dd> </dl>

</dd> </dl>

Some examples: <pre class=”example-preformatted”># Convert left and right views into a frame-sequential video ffmpeg -i LEFT -i RIGHT -filter_complex framepack=frameseq OUTPUT

Convert views into a side-by-side video with the same output resolution as the input ffmpeg -i LEFT -i RIGHT -filter_complex [0:v]scale=w=iw/2[left],[1:v]scale=w=iw/2[right],[left][right]fra
OUTPUT </pre>

 <h3 class=”section”>11.101 framerate# TOC</h3>

Change the frame rate by interpolating new video output frames from the source frames. This filter is not designed to function correctly with interlaced media. If you wish to change the frame rate of interlaced media then you are required to deinterlace before this filter and re-interlace after this filter. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frames per second. This option can also be specified as a value alone. The default is <code class=”code”>50</code>. </dd> <dt><samp class=”option”>interp_start</samp></dt> <dd> Specify the start of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>15</code>. </dd> <dt><samp class=”option”>interp_end</samp></dt> <dd> Specify the end of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the de-

fault is <code class="code">240</code>. </dd> <dt><samp class="option">scene</samp></dt> <dd> Specify the level at which a scene change is detected as a value between 0 and 100 to indicate a new scene; a low value reflects a low probability for the current frame to introduce a new scene, while a higher value means the current frame is more likely to be one. The default is <code class="code">8.2</code>. </dd> <dt><samp class="option">flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class="var">flags</var> is: <dl class="table"> <dt><samp class="option">scene_change_detect, scd</samp></dt> <dd> Enable scene change detection using the value of the option <var class="var">scene</var>. This flag is enabled by default. </dd> </dl> </dd> </dl>

 <h3 class="section">11.102 frc_amf# TOC</h3>

Double the frame rate, using Frame Rate Converter (FRC) provided by AMD Advanced Media Framework library for hardware acceleration. The filter accepts the following options: <dl class="table"> <dt><samp class="option">engine_type</samp></dt> <dd> Specify the engine used to run shaders. <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11. </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 (Default value). </dd> </dl>

</dd> <dt><samp class="option">enable</samp></dt> <dd> Boolean value: enable/disable FRC. Dynamic value, can be altered at a runtime without re-initializing the filter. (Default value: enabled). </dd> <dt><samp class="option">fallback_mode</samp></dt> <dd> Fallback behavior in case of low interpolation confidence. <dl class="table"> <dt>‘<samp class="samp">duplicate</samp>’</dt> <dd> Duplicate frame. </dd> <dt>‘<samp class="samp">blend</samp>’</dt> <dd> Blend two frames together (Default value). </dd> </dl>

</dd> <dt><samp class="option">indicator</samp></dt> <dd> Boolean value: show FRC indicator square in the top left corner of the video (Default value: disabled). </dd> <dt><samp class="option">profile</samp></dt> <dd> Level of hierarchical motion search. <dl class="table"> <dt>‘<samp class="samp">low</samp>’</dt> <dd> Less levels of hierarchical motion search. Only recommended for extremely low resolutions. </dd> <dt>‘<samp class="samp">high</samp>’</dt> <dd> Recommended for any resolution up to 1440p. (Default value) </dd> <dt>‘<samp class="samp">super</samp>’</dt> <dd> More levels of hierarchical motion search. Recommended for resolutions 1440p or higher. </dd> </dl>

</dd> <dt><samp class="option">mv_search_mode</samp></dt> <dd> Performance mode of the motion search. <dl class="table"> <dt>‘<samp class="samp">native</samp>’</dt> <dd> Conduct motion search on the full resolution of source images. </dd> <dt>‘<samp class="samp">performance</samp>’</dt> <dd> Conduct motion search on the down scaled source images. Recommended for APU or low end GPU for better performance. </dd> </dl>

</dd> <dt><samp class="option">use_future_frame</samp></dt> <dd> Boolean value: enable dependency on future frame, improves quality for the cost of latency (Default value: enabled). </dd> </dl>

 <h3 class="section">11.103 vqe_amf# TOC</h3>

AMD AMF VQ Enhancer filter. This filter applies AMD AMF VQ Enhancement to the input video. This filter accepts the following option: <dl class="table"> <dt><samp class="option">attenuation</samp></dt> <dd> Set VQ Enhancer strength. Allowed range is from <code class="code">0.02</code> to <code class="code">0.4</code>. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">engine_type</samp></dt> <dd> Set AMF memory type used by the filter. Possible values: <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11 </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 </dd> <dt>‘<samp class="samp">vulkan</samp>’</dt> <dd> Vulkan </dd> <dt>‘<samp class="samp">opencl</samp>’</dt> <dd> OpenCL </dd> </dl>

</dd> </dl>

Example: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf vqe_amf=attenuation=0.4 -c:v hevc_amf output.mp4 </pre>

 <h3 class="section">11.104 framestep# TOC</h3>

Select one frame every N-th frame. This filter accepts the following option: <dl class="table"> <dt><samp class="option">step</samp></dt> <dd> Select frame after every <code class="code">step</code> frames. Allowed values are positive integers higher than 0. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">11.105 freezedetect# TOC</h3>

Detect frozen video. This filter logs a message and sets frame metadata when it detects that the input video has no significant change in content during a specified duration. Video freeze detection calculates the mean average absolute difference of all the components of video frames and compares it to a noise floor. The printed times and duration are expressed in seconds. The <code class="code">lavfi.freezedetect.freeze_start</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the freeze. The <code class="code">lavfi.freezedetect.freeze_duration</code> and <code class="code">lavfi.freezedetect.freeze_end</code> metadata keys are set on the first frame after the freeze. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or as a difference ratio between 0 and 1. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set freeze duration until notification (default is 2 seconds). </dd> </dl>

 <h3 class="section">11.106 freezeframes# TOC</h3>

Freeze video frames. This filter freezes video frames using frame from 2nd input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">first</samp></dt> <dd> Set number of first frame from which to start freeze. </dd> <dt><samp class="option">last</samp></dt> <dd> Set number of last frame from which to end freeze. </dd> <dt><samp class="option">replace</samp></dt> <dd> Set number of frame from 2nd input which will be used instead of replaced frames. </dd> </dl>

 <h3 class="section">11.107 frei0r# TOC</h3>

Apply a frei0r effect to the input video. To enable the compilation of this filter, you need to install the frei0r header and configure FFmpeg with <code class="code">--enable-frei0r</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">filter_name</samp></dt> <dd> The name of the frei0r effect to load. If the environment variable <code

class=”env”>FREI0R_PATH</code> is defined, the frei0r effect is searched for in each of the directories specified by the colon-separated list in <code class=”env”>FREI0R_PATH</code>. Otherwise, the standard frei0r paths are searched, in this order: <samp class=”file”>HOME/.frei0r-1/lib/</samp>, <samp class=”file”>/usr/local/lib/frei0r-1/</samp>, <samp class=”file”>/usr/lib/frei0r-1/</samp>. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r effect. </dd> </dl>

A frei0r effect parameter can be a boolean (its value is either "y" or "n"), a double, a color (specified as <var class=”var”>R</var>/<var class=”var”>G</var>/<var class=”var”>B</var>, where <var class=”var”>R</var>, <var class=”var”>G</var>, and <var class=”var”>B</var> are floating point numbers between 0.0 and 1.0, inclusive) or a color description as specified in the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual, a position (specified as <var class=”var”>X</var>/<var class=”var”>Y</var>, where <var class=”var”>X</var> and <var class=”var”>Y</var> are floating point numbers) and/or a string. The number and types of parameters depend on the loaded effect. If an effect parameter is not specified, the default value is set. <h4 class=”subsection”>11.107.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the distort0r effect, setting the first two double parameters: <pre class=”example-preformatted”>frei0r=filter_name=distort0r:filter_params=0.5|0.01 </pre>

Apply the colordistance effect, taking a color as the first parameter: <pre class=”example-preformatted”>frei0r=colordistance:0.2/0.3/0.4 frei0r=colordistance:violet frei0r=colordistance:0x112233 </pre>

Apply the perspective effect, specifying the top left and top right image positions: <pre class=”example-preformatted”>frei0r=perspective:0.2/0.2|0.8/0.2 </pre>

For more information, see http://frei0r.dyne.org <h4 class=”subsection”>11.107.2 Commands# TOC</h4>

This filter supports the <samp class=”option”>filter_params</samp> option as commands. <h3 class=”section”>11.108 fspp#

TOC</h3>

Apply fast and simple postprocessing. It is a faster version of spp. It splits (I)DCT into horizontal/vertical passes. Unlike the simple post- processing filter, one of them is performed once per block, not per pixel. This allows for much higher speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 4-5. Default value is <code class=”code”>4</code>. </dd> <dt><samp class=”option”>qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0-63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class=”option”>strength</samp></dt> <dd> Set filter strength. It accepts an integer in range -15 to 32. Lower values mean more details but also more artifacts, while higher values make the image smoother but also blurrier. Default value is <code class=”code”>0</code> − PSNR optimal. </dd> <dt><samp class=”option”>use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class=”code”>1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class=”code”>0</code> (not enabled). </dd> </dl>

 <h3 class=”section”>11.109 fsync# TOC</h3>

Synchronize video frames with an external mapping from a file. For each input PTS given in the map file it either drops or creates as many frames as necessary to recreate the sequence of output frames given in the map file. This filter is useful to recreate the output frames of a framerate conversion by the fps filter, recorded into a map file using the ffmpeg option <code class=”code”>-stats_mux_pre</code>, and do further processing to the corresponding frames e.g. quality comparison. Each line of the map file must contain three items per input frame, the input PTS (decimal), the output PTS (decimal) and the output TIMEBASE (decimal/decimal), separated by a space. This file format corresponds to the output of <code class=”code”>-stats_mux_pre_fmt="{ptsi} {pts} {tb}"</code>. The filter assumes the map file is sorted by increasing input PTS. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> The filename of the map file to be used. </dd> </dl>

Example: <pre class=”example-preformatted”># Convert a video to 25 fps and record a MAP_FILE file with the default format of this filter ffmpeg -i INPUT -vf fps=fps=25 -stats_mux_pre MAP_FILE

-stats_mux_pre_fmt "{ptsi} {pts} {tb}" OUTPUT

Sort MAP_FILE by increasing input PTS sort -n MAP_FILE

Use INPUT, OUTPUT and the MAP_FILE from above to compare the corresponding frames in INPUT and OUTPUT via SSIM ffmpeg -i INPUT -i OUTPUT -filter_complex '[0:v]fsync=file=MAP_FILE[ref];[1:v][ref]ssim' -f null - </pre>

 <h3 class="section">11.110 gainmap# TOC</h3>

Compute an HDR gain map from two renditions of the same image, as defined by ISO 21496-1. The first input is the base rendition (typically SDR), and the second input is the alternate rendition (typically HDR). Both must have the same dimensions. Outputs a gain map, as either ‘<samp class="samp">gbrpf32</samp>’ or ‘<samp class="samp">grayf32</samp>’ depending on the chosen <samp class="option">mode</samp>. <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Controls what quantity the gain map is computed against. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> One gain map channel per component. </dd> <dt>‘<samp class="samp">luma</samp>’</dt> <dd> A single gain map channel, computed from the luma. </dd> <dt>‘<samp class="samp">maxrgb</samp>’</dt> <dd> A single gain map channel, computed from ‘<samp class="samp">max(R, G, B)</samp>’. </dd> </dl>

Default is ‘<samp class="samp">rgb</samp>’. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> Chooses which colorspace to compute the gain map in. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">base</samp>’</dt> <dd> The base rendition’s colorspace. </dd> <dt>‘<samp class="samp">alternate</samp>’</dt> <dd> The alternate rendition’s colorspace. </dd> </dl>

Default is ‘<samp class="samp">base</samp>’. </dd> <dt><samp class="option">min</samp></dt> <dt><samp class="option">max</samp></dt> <dd> Bounds on the encoded gain (log2). If unspecified, these will be measured from the frame. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Encoding gamma of the resulting gain map. If unspecified, this will be optimized for the frame in question. Setting this to <code class="code">1.0</code> will produce a linear gain map, which may be more efficient in some applications. </dd> <dt><samp class="option">base_offset</samp></dt> <dt><samp class="option">alt_offset</samp></dt> <dd> Offset to prevent numer-

ical instability near zero. Both default to <code class=”code”>1/64</code>. </dd> <dt><samp class=”option”>base_nits</samp></dt> <dt><samp class=”option”>alt_nits</samp></dt> <dd> Override the reference luminance of the input, in nits. If unspecified, uses the values from the input metadata. </dd> </dl>

The <code class=”code”>gainmap</code> filter also supports the framesync options. <h4 class=”subsection”>11.110.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Compute a gain map between an SDR and an HDR version of the same image: <pre class=”example-preformatted”>ffmpeg -i $sdr -i $hdr -filter_complex "[0:v][1:v]gainmap" $out </pre>

 <h3 class=”section”>11.111 gblur# TOC</h3>

Apply Gaussian blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>steps</samp></dt> <dd> Set number of steps for Gaussian approximation. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class=”code”>sigma</code>. Default is <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>11.111.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>11.112 geq# TOC</h3>

Apply generic equation to each pixel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>lum_expr, lum</samp></dt> <dd> Set the luma expression. </dd> <dt><samp class=”option”>cb_expr, cb</samp></dt>

<dd> Set the chrominance blue expression. </dd> <dt><samp class=”option”>cr_expr, cr</samp></dt> <dd> Set the chrominance red expression. </dd> <dt><samp class=”option”>alpha_expr, a</samp></dt> <dd> Set the alpha expression. </dd> <dt><samp class=”option”>red_expr, r</samp></dt> <dd> Set the red expression. </dd> <dt><samp class=”option”>green_expr, g</samp></dt> <dd> Set the green expression. </dd> <dt><samp class=”option”>blue_expr, b</samp></dt> <dd> Set the blue expression. </dd> </dl>

The colorspace is selected according to the specified options. If one of the <samp class=”option”>lum_expr</samp>, <samp class=”option”>cb_expr</samp>, or <samp class=”option”>cr_expr</samp> options is specified, the filter will automatically select a YCbCr colorspace. If one of the <samp class=”option”>red_expr</samp>, <samp class=”option”>green_expr</samp>, or <samp class=”option”>blue_expr</samp> options is specified, it will select an RGB colorspace. If one of the chrominance expression is not defined, it falls back on the other one. If no alpha expression is specified it will evaluate to opaque value. If none of chrominance expressions are specified, they will evaluate to the luma expression. The expressions can use the following variables and functions: <dl class=”table”> <dt><samp class=”option”>N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class=”code”>0</code>. </dd> <dt><samp class=”option”>X</samp></dt> <dt><samp class=”option”>Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class=”option”>SW</samp></dt> <dt><samp class=”option”>SH</samp></dt> <dd> Width and height scale depending on the currently filtered plane. It is the ratio between the corresponding luma plane number of pixels and the current plane ones. E.g. for YUV4:2:0 the values are <code class=”code”>1,1</code> for the luma plane, and <code class=”code”>0.5,0.5</code> for chroma planes. </dd> <dt><samp class=”option”>T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class=”option”>p(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class=”var”>x</var>,<var class=”var”>y</var>) of the current plane. </dd> <dt><samp class=”option”>lum(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class=”var”>x</var>,<var class=”var”>y</var>) of the luma plane. </dd> <dt><samp class=”option”>cb(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class=”var”>x</var>,<var class=”var”>y</var>) of the blue-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class=”option”>cr(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class=”var”>x</var>,<var class=”var”>y</var>) of

the red-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">r(x, y)</samp></dt> <dt><samp class="option">g(x, y)</samp></dt> <dt><samp class="option">b(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red/green/blue component. Return 0 if there is no such component. </dd> <dt><samp class="option">alpha(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the alpha plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">psum(x,y), lumsum(x, y), cbsum(x,y), crsum(x,y), rsum(x,y), gsum(x,y), bsum(x,y), alphasum(x,y)</samp></dt> <dd> Sum of sample values in the rectangle from (0,0) to (x,y), this allows obtaining sums of samples within a rectangle. See the functions without the sum postfix. </dd> <dt><samp class="option">interpolation</samp></dt> <dd> Set one of interpolation methods: <dl class="table"> <dt><samp class="option">nearest, n</samp></dt> <dt><samp class="option">bilinear, b</samp></dt> </dl> Default is bilinear. </dd> </dl>

For functions, if <var class="var">x</var> and <var class="var">y</var> are outside the area, the value will be automatically clipped to the closer edge. Please note that this filter can use multiple threads in which case each slice will have its own expression state. If you want to use only a single expression state because your expressions depend on previous state then you should limit the number of filter threads to 1. <h4 class="subsection">11.112.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Flip the image horizontally: <pre class="example-preformatted">geq=p(W-X\,Y) </pre>

Generate a bidimensional sine wave, with angle <code class="code">PI/3</code> and a wavelength of 100 pixels: <pre class="example-preformatted">geq=128 + 100*sin(2*(PI/100)*(cos(PI/3)*(X-50*T) + sin(PI/3)*Y)):128:128 </pre>

Generate a fancy enigmatic moving light: <pre class="example-preformatted">nullsrc=s=256x256,geq=random(1)/hypot(X-cos(N*0.07)*W/2-W/2\,Y-sin(N*0.09)*H/2-H/2)^2*1000000*sin(N*0.02):128:128 </pre>

Generate a quick emboss effect: <pre class="example-preformatted">format=gray,geq=lum_expr='(p(X,Y)+(256-p(X-4,Y-4)))/2' </pre>

Modify RGB components depending on pixel position: <div

class=”example”> <pre class=”example-preformatted”>geq=r='X/W*r(X,Y)':g='(1-X/W)*g(X,Y)':b='(H-Y)/H*b(X,Y)' </pre>

Create a radial gradient that is the same size as the input (also see the vignette filter): <pre class=”example-preformatted”>geq=lum=255*gauss((X/W-0.5)*3)*gauss((Y/H-0.5)*3)/gauss(0)/gauss(0),format=gray </pre>

 <h3 class=”section”>11.113 gradfun# TOC</h3>

Fix the banding artifacts that are sometimes introduced into nearly flat regions by truncation to 8-bit color depth. Interpolate the gradients that should go where the bands are, and dither them. It is designed for playback only. Do not use it prior to lossy compression, because compression tends to lose the dither and bring back the bands. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>strength</samp></dt> <dd> The maximum amount by which the filter will change any one pixel. This is also the threshold for detecting nearly flat regions. Acceptable values range from .51 to 64; the default value is 1.2. Out-of-range values will be clipped to the valid range. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> The neighborhood to fit the gradient to. A larger radius makes for smoother gradients, but also prevents the filter from modifying the pixels near detailed regions. Acceptable values are 8-32; the default value is 16. Out-of-range values will be clipped to the valid range. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class=”var”>strength</var>[:<var class=”var”>radius</var>] <h4 class=”subsection”>11.113.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the filter with a <code class=”code”>3.5</code> strength and radius of <code class=”code”>8</code>: <pre class=”example-preformatted”>gradfun=3.5:8 </pre>

Specify radius, omitting the strength (which will fall-back to the default value): <pre class=”example-preformatted”>gradfun=radius=8 </pre>

 <h3 class=”section”>11.114 graphmonitor<span class=”pull-

right”>#
TOC</h3>
Show various filtergraph stats. With this filter one can debug
complete filtergraph. Especially issues with links filling with queued frames.
 The filter accepts the following options: <dl class=”table”>
<dt><samp class=”option”>size, s</samp></dt> <dd> Set video
output size. Default is <var class=”var”>hd720</var>. </dd>
<dt><samp class=”option”>opacity, o</samp></dt> <dd> Set video
opacity. Default is <var class=”var”>0.9</var>. Allowed range is from
<var class=”var”>0</var> to <var class=”var”>1</var>. </dd>
<dt><samp class=”option”>mode, m</samp></dt> <dd> Set output
mode flags. Available values for flags are: <dl class=”table”>
<dt>‘<samp class=”samp”>full</samp>’</dt> <dd> No
any filtering. Default. </dd> <dt>‘<samp class=”samp”>compact</samp>’</dt>
<dd> Show only filters with queued frames. </dd> <dt>‘<samp
class=”samp”>nozero</samp>’</dt> <dd> Show only filters
with non-zero stats. </dd> <dt>‘<samp class=”samp”>noeof</samp>’</dt>
<dd> Show only filters with non-eof stat. </dd> <dt>‘<samp
class=”samp”>nodisabled</samp>’</dt> <dd> Show only filters
that are enabled in timeline. </dd> </dl>

</dd> <dt><samp class=”option”>flags, f</samp></dt> <dd> Set
flags which enable which stats are shown in video. Available
values for flags are: <dl class=”table”> <dt>‘<samp
class=”samp”>none</samp>’</dt> <dd> All flags turned off.
 </dd> <dt>‘<samp class=”samp”>all</samp>’</dt>
<dd> All flags turned on. </dd> <dt>‘<samp class=”samp”>queue</samp>’</dt>
<dd> Display number of queued frames in each link. </dd>
<dt>‘<samp class=”samp”>frame_count_in</samp>’</dt>
<dd> Display number of frames taken from filter. </dd>
<dt>‘<samp class=”samp”>frame_count_out</samp>’</dt>
<dd> Display number of frames given out from filter. </dd>
<dt>‘<samp class=”samp”>frame_count_delta</samp>’</dt>
<dd> Display delta number of frames between above two values.
</dd> <dt>‘<samp class=”samp”>pts</samp>’</dt>
<dd> Display current filtered frame pts. </dd> <dt>‘<samp
class=”samp”>pts_delta</samp>’</dt> <dd> Display pts delta
between current and previous frame. </dd> <dt>‘<samp
class=”samp”>time</samp>’</dt> <dd> Display current filtered
frame time. </dd> <dt>‘<samp class=”samp”>time_delta</samp>’</dt>
<dd> Display time delta between current and previous frame.
</dd> <dt>‘<samp class=”samp”>timebase</samp>’</dt>
<dd> Display time base for filter link. </dd> <dt>‘<samp
class=”samp”>format</samp>’</dt> <dd> Display used format
for filter link. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt>
<dd> Display video size or number of audio channels in case of audio used

by filter link. </dd> <dt>‘<samp class="samp">rate</samp>’</dt> <dd> Display video frame rate or sample rate in case of audio used by filter link. </dd> <dt>‘<samp class="samp">eof</samp>’</dt> <dd> Display link output status. </dd> <dt>‘<samp class="samp">sample_count_in</samp>’</dt> <dd> Display number of samples taken from filter. </dd> <dt>‘<samp class="samp">sample_count_out</samp>’</dt> <dd> Display number of samples given out from filter. </dd> <dt>‘<samp class="samp">sample_count_delta</samp>’</dt> <dd> Display delta number of samples between above two values. </dd> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Show the timeline filter status. </dd> </dl>

</dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set upper limit for video rate of output stream, Default value is <var class="var">25</var>. This guarantee that output video frame rate will not be higher than this value. </dd> </dl>

 <h3 class="section">11.115 grayworld# TOC</h3> A color constancy filter that applies color correction based on the grayworld assumption See: <a class="url" href="https://www.researchgate.net/publication/275213614_A_New_Color_Correction_Met
 The algorithm uses linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,grayworld,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h3 class="section">11.116 greyedge# TOC</h3> A color constancy variation filter which estimates scene illumination via grey edge algorithm and corrects the scene colors accordingly. See: <a class="url" href="https://staff.science.uva.nl/th.gevers/pub/GeversTIP07.pdf">https://staff.science.uva.nl/th.gevers/pu
 The filter accepts the following options: <dl class="table"> <dt><samp class="option">difford</samp></dt> <dd> The order of differentiation to be applied on the scene. Must be chosen in the range [0,2] and default value is 1. </dd> <dt><samp class="option">minknorm</samp></dt> <dd> The Minkowski parameter to be used for calculating the Minkowski distance. Must be chosen in the range [0,20] and default value is 1. Set to 0 for getting max value instead of calculating Minkowski distance. </dd> <dt><samp class="option">sigma</samp></dt> <dd> The standard deviation of Gaussian blur to be applied on the scene. Must

be chosen in the range [0,1024.0] and default value = 1. floor(<var class=”var”>sigma</var> * break_off_sigma(3)) can’t be equal to 0 if <var class=”var”>difford</var> is greater than 0. </dd> </dl>

 <h4 class=”subsection”>11.116.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Grey Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=5:sigma=2 </pre>

Max Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=0:sigma=2 </pre>

 <h3 class=”section”>11.117 guided# TOC</h3> Apply guided filter for edge-preserving smoothing, dehazing and so on. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set the box radius in pixels. Allowed range is 1 to 20. Default is 3. </dd> <dt><samp class=”option”>eps</samp></dt> <dd> Set regularization parameter (with square). Allowed range is 0 to 1. Default is 0.01. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set filter mode. Can be <code class=”code”>basic</code> or <code class=”code”>fast</code>. Default is <code class=”code”>basic</code>. </dd> <dt><samp class=”option”>sub</samp></dt> <dd> Set subsampling ratio for <code class=”code”>fast</code> mode. Range is 2 to 64. Default is 4. No subsampling occurs in <code class=”code”>basic</code> mode. </dd> <dt><samp class=”option”>guidance</samp></dt> <dd> Set guidance mode. Can be <code class=”code”>off</code> or <code class=”code”>on</code>. Default is <code class=”code”>off</code>. If <code class=”code”>off</code>, single input is required. If <code class=”code”>on</code>, two inputs of the same resolution and pixel format are required. The second input serves as the guidance. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>11.117.1 Commands# TOC</h4> This filter supports the all above options as commands. <h4 class=”subsection”>11.117.2 Examples<a class=”anchor hidden-xs” href=”#Examples-

88” aria-hidden=”true”># TOC</h4> <ul class=”itemize mark-bullet”> Edge-preserving smoothing with guided filter: <pre class=”example-preformatted”>ffmpeg -i in.png -vf guided out.png </pre>

Dehazing, structure-transferring filtering, detail enhancement with guided filter. For the generation of guidance image, refer to paper "Guided Image Filtering". See: <a class=”url” href=”http://kaiminghe.com/publications/pami12guidedfilter.pdf <pre class=”example-preformatted”>ffmpeg -i in.png -i guidance.png -filter_complex guided=guidance=on out.png </pre>

 <h3 class=”section”>11.118 haldclut# TOC</h3>

Apply a Hald CLUT to a video stream. First input is the video stream to process, and second one is the Hald CLUT. The Hald CLUT input can be a simple picture or a complete video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>clut</samp></dt> <dd> Set which CLUT video frames will be processed from second input stream, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> Force termination when the shortest input terminates. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> Continue applying the last CLUT after the end of the stream. A value of <code class=”code”>0</code> disable the filter after the last frame of the CLUT is reached. Default is <code class=”code”>1</code>. </dd> </dl>

<code class=”code”>haldclut</code> also has the same interpolation options as lut3d (both filters share the same internals). This filter also supports the framesync options. More information about the Hald CLUT can be found on Eskil Steenberg’s website (Hald CLUT author) at <a class=”url” href=”http://www.quelsolaar.com/technology/clut.html”>http://www.quelsolaar.com/technology/clut.html< <h4 class=”subsection”>11.118.1 Commands# TOC</h4>

This filter supports the <code class=”code”>interp</code> option as commands.

 <h4 class="subsection">11.118.2 Workflow examples# TOC</h4>

 <h4 class="subsubsection">11.118.2.1 Hald CLUT video stream# TOC</h4>

Generate an identity Hald CLUT stream altered with various effects: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf "hue=H=2*PI*t:s=sin(2*PI*t)+1, curves=cross_process" -t 10 -c:v ffv1 clut.nut </pre>

Note: make sure you use a lossless codec. Then use it with <code class="code">haldclut</code> to apply it on some random stream: <pre class="example-preformatted">ffmpeg -f lavfi -i mandelbrot -i clut.nut -filter_complex '[0][1] haldclut' -t 20 mandelclut.mkv </pre>

The Hald CLUT will be applied to the 10 first seconds (duration of <samp class="file">clut.nut</samp>), then the latest picture of that CLUT stream will be applied to the remaining frames of the <code class="code">mandelbrot</code> stream. <h4 class="subsubsection">11.118.2.2 Hald CLUT with preview# TOC</h4>

A Hald CLUT is supposed to be a squared image of <code class="code">Level*Level*Level</code> by <code class="code">Level*Level*Level</code> pixels. For a given Hald CLUT, FFmpeg will select the biggest possible square starting at the top left of the picture. The remaining padding pixels (bottom or right) will be ignored. This area can be used to add a preview of the Hald CLUT. Typically, the following generated Hald CLUT will be supported by the <code class="code">haldclut</code> filter: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf " pad=iw+320 [padded_clut]; smptebars=s=320x256, split [a][b]; [padded_clut][a] overlay=W-320:h, curves=color_negative [main]; [main][b] overlay=W-320" -frames:v 1 clut.png </pre>

It contains the original and a preview of the effect of the CLUT: SMPTE color bars are displayed on the right-top, and below the same color bars processed by the color changes. Then, the effect of this Hald CLUT

can be visualized with: <pre class="example-preformatted">ffplay input.mkv -vf "movie=clut.png, [in] haldclut" </pre>

 <h3 class="section">11.119 hflip# TOC</h3>

Flip the input video horizontally. For example, to horizontally flip the input video with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf "hflip" out.avi </pre>

 <h3 class="section">11.120 histeq# TOC</h3> This filter applies a global color histogram equalization on a per-frame basis. It can be used to correct video that has a compressed range of pixel intensities. The filter redistributes the pixel intensities to equalize their distribution across the intensity range. It may be viewed as an "automatically adjusting contrast filter". This filter is useful only for correcting degraded or poorly captured source video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Determine the amount of equalization to be applied. As the strength is reduced, the distribution of pixel intensities more-and-more approaches that of the input frame. The value must be a float number in the range [0,1] and defaults to 0.200. </dd> <dt><samp class="option">intensity</samp></dt> <dd> Set the maximum intensity that can generated and scale the output values appropriately. The strength should be set as desired and then the intensity can be limited if needed to avoid washing-out. The value must be a float number in the range [0,1] and defaults to 0.210. </dd> <dt><samp class="option">antibanding</samp></dt> <dd> Set the antibanding level. If enabled the filter will randomly vary the luminance of output pixels by a small amount to avoid banding of the histogram. Possible values are <code class="code">none</code>, <code class="code">weak</code> or <code class="code">strong</code>. It defaults to <code class="code">none</code>. </dd> </dl>

 <h3 class="section">11.121 histogram# TOC</h3>

Compute and draw a color distribution histogram for the input video. The computed histogram is a representation of the color component distribution in an image. Standard histogram displays the color components distribution in an image. Displays color graph for

each color component. Shows distribution of the Y, U, V, A or R, G, B components, depending on input format, in the current frame. Below each graph a color component scale meter is shown. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_height</samp></dt> <dd> Set height of level. Default value is <code class="code">200</code>. Allowed range is [50, 2048]. </dd> <dt><samp class="option">scale_height</samp></dt> <dd> Set height of color scale. Default value is <code class="code">12</code>. Allowed range is [0, 40]. </dd> <dt><samp class="option">display_mode</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">levels_mode</samp></dt> <dd> Set mode. Can be either <code class="code">linear</code>, or <code class="code">logarithmic</code>. Default is <code class="code">linear</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set what color components to display. Default is <code class="code">7</code>. </dd> <dt><samp class="option">fgopacity</samp></dt> <dd> Set foreground opacity. Default is <code class="code">0.7</code>. </dd> <dt><samp class="option">bgopacity</samp></dt> <dd> Set background opacity. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">colors_mode</samp></dt> <dd> Set colors mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">whiteonblack</samp>’</dt> <dt>‘<samp class="samp">blackonwhite</samp>’</dt> <dt>‘<samp class="samp">whiteongray</samp>’</dt> <dt>‘<samp class="samp">blackongray</samp>’</dt> <dt>‘<samp class="samp">coloronblack</samp>’</dt> <dt>‘<samp class="samp">coloronwhite</samp>’</dt> <dt>‘<samp class="samp">colorongray</samp>’</dt> <dt>‘<samp class="samp">blackoncolor</samp>’</dt> <dt>‘<samp class="samp">whiteoncolor</samp>’</dt> <dt>‘<samp class="samp">grayoncolor</samp>’</dt> </dl> Default is <code class="code">whiteonblack</code>. </dd> </dl>

 <h4 class="subsection">11.121.1 Examples# <a class="anchor hidden-xs"href="#toc-

Examples-89” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Calculate and draw histogram: <pre class=”example-preformatted”>ffplay -i input -vf histogram </pre>

 <h3 class=”section”>11.122 hqdn3d# TOC</h3>

This is a high precision/quality 3d denoise filter. It aims to reduce image noise, producing smooth images and making still images really still. It should enhance compressibility. It accepts the following optional parameters: <dl class=”table”> <dt><samp class=”option”>luma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial luma strength. It defaults to 4.0. </dd> <dt><samp class=”option”>chroma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial chroma strength. It defaults to 3.0*<var class=”var”>luma_spatial</var>/4.0. </dd> <dt><samp class=”option”>luma_tmp</samp></dt> <dd> A floating point number which specifies luma temporal strength. It defaults to 6.0*<var class=”var”>luma_spatial</var>/4.0. </dd> <dt><samp class=”option”>chroma_tmp</samp></dt> <dd> A floating point number which specifies chroma temporal strength. It defaults to <var class=”var”>luma_tmp</var>*<var class=”var”>chroma_spatial</var>/<var class=”var”>luma_spatial</var>. </dd> </dl>

 <h4 class=”subsection”>11.122.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>11.123 hwdownload# TOC</h3>

Download hardware frames to system memory. The input must be in hardware frames, and the output a non-hardware format. Not all formats will be supported on the output - it may be necessary to insert an additional <samp class=”option”>format</samp> filter immediately following in the graph to get the output in a supported format. <h3 class=”section”>11.124 hwmap<a class=”anchor hidden-xs” href=”#hwmap” aria-

hidden=”true”># TOC</h3>

Map hardware frames to system memory or to another device. This filter has several different modes of operation; which one is used depends on the input and output formats: <ul class=”itemize mark-bullet”> Hardware frame input, normal frame output

Map the input frames to system memory and pass them to the output. If the original hardware frame is later required (for example, after overlaying something else on part of it), the <samp class=”option”>hwmap</samp> filter can be used again in the next mode to retrieve it. Normal frame input, hardware frame output

If the input is actually a software-mapped hardware frame, then unmap it - that is, return the original hardware frame. Otherwise, a device must be provided. Create new hardware surfaces on that device for the output, then map them back to the software format at the input and give those frames to the preceding filter. This will then act like the <samp class=”option”>hwupload</samp> filter, but may be able to avoid an additional copy when the input is already in a compatible format. Hardware frame input and output

A device must be supplied for the output, either directly or with the <samp class=”option”>derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). If the input frames were originally created on the output device, then unmap to retrieve the original frames. Otherwise, map the frames to the output device - create new hardware frames on the output corresponding to the frames on the input.

The following additional parameters are accepted: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the frame mapping mode. Some combination of: <dl class=”table”> <dt><var class=”var”>read</var></dt> <dd> The mapped frame should be readable. </dd> <dt><var class=”var”>write</var></dt> <dd> The mapped frame should be writeable. </dd> <dt><var class=”var”>overwrite</var></dt> <dd> The mapping will always overwrite the entire frame. This may improve performance in some cases, as the original contents of the frame need not be loaded. </dd> <dt><var class=”var”>direct</var></dt> <dd> The mapping must not involve any copying. Indirect mappings to copies of frames are created in some cases where either direct mapping is not possible or it would have unexpected properties. Setting this flag ensures that the mapping is direct and will fail if that is not possible. </dd> </dl> Defaults to <var class=”var”>read+write</var>

if not specified. </dd> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> <dt><samp class="option">reverse</samp></dt> <dd> In a hardware to hardware mapping, map in reverse - create frames in the sink and map them back to the source. This may be necessary in some cases where a mapping in one direction is required but only the opposite direction is supported by the devices being used. This option is dangerous - it may break the preceding filter in undefined ways if there are any additional constraints on that filter’s output. Do not use it without fully understanding the implications of its use. </dd> </dl>

 <h3 class="section">11.125 hwupload# TOC</h3>

Upload system memory frames to hardware surfaces. The device to upload to must be supplied when the filter is initialised. If using ffmpeg, select the appropriate device with the <samp class="option">-filter_hw_device</samp> option or with the <samp class="option">derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). The following additional parameters are accepted: <dl class="table"> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> </dl>

 <h3 class="section">11.126 hwupload_cuda# TOC</h3>

Upload system memory frames to a CUDA device. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">device</samp></dt> <dd> The number of the CUDA device to use </dd> </dl>

 <h3 class="section">11.127 hqx# TOC</h3>

Apply a high-quality magnification filter designed for pixel art. This filter was originally created by Maxim Stepin. It ac-

cepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">hq2x</code>, <code class="code">3</code> for <code class="code">hq3x</code> and <code class="code">4</code> for <code class="code">hq4x</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">11.128 hstack# TOC</h3> Stack input videos horizontally. All streams must be of same pixel format and of same height. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following option: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class="section">11.129 hsvhold# TOC</h3> Turns a certain HSV range into gray values. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to be gray or not. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more gray pixels, with a higher gray pixel the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class="section">11.130 hsvkey<span

class=”pull-right”># TOC</h3> Turns a certain HSV range into transparency. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to transparent by adding alpha channel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class=”option”>sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class=”option”>val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class=”section”>11.131 hue# TOC</h3>

Modify the hue and/or the saturation of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Specify the hue angle as a number of degrees. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>s</samp></dt> <dd> Specify the saturation in the [-10,10] range. It accepts an expression and defaults to "1". </dd> <dt><samp class=”option”>H</samp></dt> <dd> Specify the hue angle as a number of radians. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>b</samp></dt> <dd> Specify the brightness in the [-10,10] range. It accepts an expression and defaults to "0". </dd> </dl>

<samp class=”option”>h</samp> and <samp class=”option”>H</samp> are mutually exclusive, and can’t be specified at the same time. The <samp class=”option”>b</samp>, <samp class=”option”>h</samp>, <samp class=”option”>H</samp> and <samp class=”option”>s</samp> option values are expressions containing the following constants: <dl

class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pts</samp></dt> <dd> presentation timestamp of the input frame expressed in time base units </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class=”option”>tb</samp></dt> <dd> time base of the input video </dd> </dl>

 <h4 class=”subsection”>11.131.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the hue to 90 degrees and the saturation to 1.0: <pre class=”example-preformatted”>hue=h=90:s=1 </pre>

Same command but expressing the hue in radians: <pre class=”example-preformatted”>hue=H=PI/2:s=1 </pre>

Rotate hue and make the saturation swing between 0 and 2 over a period of 1 second: <pre class=”example-preformatted”>hue="H=2*PI*t: s=sin(2*PI*t)+1" </pre>

Apply a 3 seconds saturation fade-in effect starting at 0: <pre class=”example-preformatted”>hue="s=min(t/3\,1)" </pre>

The general fade-in expression can be written as: <pre class=”example-preformatted”>hue="s=min(0\, max((t-START)/DURATION\, 1))" </pre>

Apply a 3 seconds saturation fade-out effect starting at 5 seconds: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (8-t)/3))" </pre>

The general fade-out expression can be written as: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (START+DURATION-t)/DURATION))" </pre>

 <h4 class=”subsection”>11.131.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">b</samp></dt> <dt><samp class="option">s</samp></dt>
<dt><samp class="option">h</samp></dt> <dt><samp class="option">H</samp></dt>
<dd> Modify the hue and/or the saturation and/or brightness of the input
video. The command accepts the same syntax of the corresponding option.
 If the specified expression is not valid, it is kept at its current value.
</dd> </dl>

 <h3 class="section">11.132 huesatura-
tion<a class="anchor hidden-xs" href="#huesaturation"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
huesaturation" aria-hidden="true">TOC</h3> Apply
hue-saturation-intensity adjustments to input video stream. This
filter operates in RGB colorspace. This filter accepts the following op-
tions: <dl class="table"> <dt><samp class="option">hue</samp></dt>
<dd> Set the hue shift in degrees to apply. Default is 0. Allowed range is
from -180 to 180. </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the saturation shift. Default is 0. Allowed range is from -1
to 1. </dd> <dt><samp class="option">intensity</samp></dt>
<dd> Set the intensity shift. Default is 0. Allowed range is from -1
to 1. </dd> <dt><samp class="option">colors</samp></dt>
<dd> Set which primary and complementary colors are going to be
adjusted. This options is set by providing one or multiple values. This can
select multiple colors at once. By default all colors are selected. <dl
class="table"> <dt>‘<samp class="samp">r</samp>’</dt>
<dd> Adjust reds. </dd> <dt>‘<samp class="samp">y</samp>’</dt>
<dd> Adjust yellows. </dd> <dt>‘<samp class="samp">g</samp>’</dt>
<dd> Adjust greens. </dd> <dt>‘<samp class="samp">c</samp>’</dt>
<dd> Adjust cyans. </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> Adjust blues. </dd> <dt>‘<samp class="samp">m</samp>’</dt>
<dd> Adjust magentas. </dd> <dt>‘<samp class="samp">a</samp>’</dt>
<dd> Adjust all colors. </dd> </dl>

</dd> <dt><samp class="option">strength</samp></dt> <dd> Set
strength of filtering. Allowed range is from 0 to 100. Default value is
1. </dd> <dt><samp class="option">rw, gw, bw</samp></dt>
<dd> Set weight for each RGB component. Allowed range is from
0 to 1. By default is set to 0.333, 0.334, 0.333. Those options are
used in saturation and lightess processing. </dd> <dt><samp
class="option">lightness</samp></dt> <dd> Set preserving lightness,
by default is disabled. Adjusting hues can change lightness from original RGB
triplet, with this option enabled lightness is kept at same value. </dd>
</dl>

 <h3 class="section">11.133 hysteresis<span
class="pull-right"><a class="anchor hidden-xs" href="#hysteresis" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-hysteresis"

aria-hidden=”true”>TOC</h3>

Grow first stream into second stream by connecting components. This makes it possible to build more robust edge masks. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold which is used in filtering. If pixel component value is higher than this value filter algorithm for connecting components is activated. By default value is 0. </dd> </dl>

The <code class=”code”>hysteresis</code> filter also supports the framesync options. <h3 class=”section”>11.134 iccdetect# TOC</h3>

Detect the colorspace from an embedded ICC profile (if present), and update the frame’s tags accordingly. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>force</samp></dt> <dd> If true, the frame’s existing colorspace tags will always be overridden by values detected from an ICC profile. Otherwise, they will only be assigned if they contain <code class=”code”>unknown</code>. Enabled by default. </dd> </dl>

 <h3 class=”section”>11.135 iccgen# TOC</h3>

Generate ICC profiles and attach them to frames. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dd> Configure the colorspace that the ICC profile will be generated for. The default value of <code class=”code”>auto</code> infers the value from the input frame’s metadata, defaulting to BT.709/sRGB as appropriate. See the setparams filter for a list of possible values, but note that <code class=”code”>unknown</code> are not valid values for this filter. </dd> <dt><samp class=”option”>force</samp></dt> <dd> If true, an ICC profile will be generated even if it would overwrite an already existing ICC profile. Disabled by default. </dd> </dl>

 <h3 class=”section”>11.136 identity# <a class=”anchor hidden-xs”href=”#toc-identity”

aria-hidden=”true”>TOC</h3>

Obtain the identity score between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max identity score is printed through the logging system. The filter stores the calculated identity scores of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi identity -f null - </pre>

 <h3 class=”section”>11.137 idet# TOC</h3>

Detect video interlacing type. This filter tries to detect if the input frames are interlaced, progressive, top or bottom field first. It will also try to detect fields that are repeated between adjacent frames (a sign of telecine). Single frame detection considers only immediately adjacent frames when classifying each frame. Multiple frame detection incorporates the classification history of previous frames. The filter will log these metadata values: <dl class=”table”> <dt><samp class=”option”>single.current_frame</samp></dt> <dd> Detected type of current frame using single-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class=”option”>single.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using single-frame detection. </dd> <dt><samp class=”option”>multiple.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using multiple-frame detection. </dd> <dt><samp class=”option”>single.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using single-frame detection. </dd> <dt><samp class=”option”>multiple.current_frame</samp></dt> <dd> Detected type of current frame using multiple-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class=”option”>multiple.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using multiple-frame detection. </dd> <dt><samp class=”option”>single.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using single-frame detection. </dd> <dt><samp class=”option”>multiple.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using multiple-frame detection. </dd> <dt><samp class=”option”>single.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using single-

frame detection. </dd> <dt><samp class=”option”>multiple.undetermined</samp></dt>
<dd> Cumulative number of frames that could not be classified using multiple-frame detection. </dd> <dt><samp class=”option”>repeated.current_frame</samp></dt>
<dd> Which field in the current frame is repeated from the last. One of “neither”, “top”, or “bottom”. </dd> <dt><samp class=”option”>repeated.neither</samp></dt>
<dd> Cumulative number of frames with no repeated field. </dd>
<dt><samp class=”option”>repeated.top</samp></dt> <dd> Cumulative number of frames with the top field repeated from the previous frame’s top field. </dd> <dt><samp class=”option”>repeated.bottom</samp></dt>
<dd> Cumulative number of frames with the bottom field repeated from the previous frame’s bottom field. </dd> </dl>

The filter accepts the following options: <dl class=”table”>
<dt><samp class=”option”>intl_thres</samp></dt> <dd> Set interlacing threshold. </dd> <dt><samp class=”option”>prog_thres</samp></dt>
<dd> Set progressive threshold. </dd> <dt><samp class=”option”>rep_thres</samp></dt>
<dd> Threshold for repeated field detection. </dd> <dt><samp class=”option”>half_life</samp></dt> <dd> Number of frames after which a given frame’s contribution to the statistics is halved (i.e., it contributes only 0.5 to its classification). The default of 0 means that all frames seen are given full weight of 1.0 forever. </dd> <dt><samp class=”option”>analyze_interlaced_flag</samp></dt> <dd> When this is not 0 then idet will use the specified number of frames to determine if the interlaced flag is accurate, it will not count undetermined frames. If the flag is found to be accurate it will be used without any further computations, if it is found to be inaccurate it will be cleared without any further computations. This allows inserting the idet filter as a low computational method to clean up the interlaced flag </dd> </dl>

 <h4 class=”subsection”>11.137.1 Examples# TOC</h4>

Inspect the field order of the first 360 frames in a video, in verbose detail: <pre class=”example-preformatted”>ffmpeg -i INPUT -filter:v idet,metadata=mode=print -frames:v 360 -an -f null -
</pre> The idet filter will add analysis metadata to each frame, which will then be discarded. At the end, the filter will also print a final report with statistics. <h3 class=”section”>11.138 il# TOC</h3>

Deinterleave or interleave fields. This filter allows one to process interlaced images fields without deinterlacing them. Deinterleaving splits the input frame into 2 fields (so called half pictures). Odd lines

are moved to the top half of the output image, even lines to the bottom half. You can process (filter) them independently and then re-interleave them. The filter accepts the following options: <dl class="table"> <dt><samp class="option">luma_mode, l</samp></dt> <dt><samp class="option">chroma_mode, c</samp></dt> <dt><samp class="option">alpha_mode, a</samp></dt> <dd> Available values for <var class="var">luma_mode</var>, <var class="var">chroma_mode</var> and <var class="var">alpha_mode</var> are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do nothing. </dd> <dt>‘<samp class="samp">deinterleave, d</samp>’</dt> <dd> Deinterleave fields, placing one above the other. </dd> <dt>‘<samp class="samp">interleave, i</samp>’</dt> <dd> Interleave fields. Reverse the effect of deinterleaving. </dd> </dl> Default value is <code class="code">none</code>. </dd> <dt><samp class="option">luma_swap, ls</samp></dt> <dt><samp class="option">chroma_swap, cs</samp></dt> <dt><samp class="option">alpha_swap, as</samp></dt> <dd> Swap luma/chroma/alpha fields. Exchange even & odd lines. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">11.138.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.139 inflate# TOC</h3>

Apply inflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values higher than the pixel. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">11.139.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.140 interlace, interlace_vulkan<a class="anchor hidden-

xs" href="#interlace_002c-interlace_005fvulkan" aria-hidden="true"># TOC</h3>

Simple interlacing filter from progressive contents. This interleaves upper (or lower) lines from odd frames with lower (or upper) lines from even frames, halving the frame rate and preserving image height.   <pre class="example-preformatted"> Original Original New Frame Frame 'j' Frame 'j+1' (tff) ========== ========== ========== Line 0 ----------> Frame 'j' Line 0 Line 1 Line 1 ----> Frame 'j+1' Line 1 Line 2 ----------> Frame 'j' Line 2 Line 3 Line 3 ----> Frame 'j+1' Line 3 ... ... New Frame + 1 will be generated by Frame 'j+2' and Frame 'j+3' and so on </pre>

It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">scan</samp></dt> <dd> This determines whether the interlaced frame is taken from the even (tff - default) or odd (bff) lines of the progressive frame. </dd> <dt><samp class="option">lowpass</samp></dt> <dd> Vertical lowpass filter to avoid twitter interlacing and reduce moire patterns. <dl class="table"> <dt>‘<samp class="samp">0, off</samp>’</dt> <dd> Disable vertical lowpass filter </dd> <dt>‘<samp class="samp">1, linear</samp>’</dt> <dd> Enable linear filter (default) </dd> <dt>‘<samp class="samp">2, complex</samp>’</dt> <dd> Enable complex filter. This will slightly less reduce twitter and moire but better retain detail and subjective sharpness impression. </dd> </dl> </dd> </dl>

 <h3 class="section">11.141 kerndeint# TOC</h3>

Deinterlace input video by applying Donald Graft’s adaptive kernel deinterling. Work on interlaced parts of a video to produce progressive frames. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">thresh</samp></dt> <dd> Set the threshold which affects the filter’s tolerance when determining if a pixel line must be processed. It must be an integer in the range [0,255] and defaults to 10. A value of 0 will result in applying the process on every pixels. </dd> <dt><samp class="option">map</samp></dt> <dd> Paint pixels exceeding the threshold value to white if set to 1. Default is 0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set the fields order. Swap fields if set to 1, leave fields alone if 0. Default is 0. </dd> <dt><samp class="option">sharp</samp></dt> <dd> Enable additional sharpening if set to 1. Default is 0. </dd> <dt><samp class="option">twoway</samp></dt> <dd> Enable twoway sharpening if set to 1. Default is 0. </dd> </dl>

 <h4 class="subsection">11.141.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply default values:
<pre class="example-preformatted">kerndeint=thresh=10:map=0:order=0:sharp=0:twoway=0
</pre>

Enable additional sharpening: <pre class="example-preformatted">kerndeint=sharp=1 </pre>

Paint processed pixels in white: <pre class="example-preformatted">kerndeint=map=1 </pre>

 <h3 class="section">11.142 kirsch# TOC</h3> Apply kirsch operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">11.142.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">11.143 lagfun# TOC</h3>

Slowly update darker pixels. This filter makes short flashes of light appear longer. This filter accepts the following options: <dl class="table"> <dt><samp class="option">decay</samp></dt> <dd> Set factor for decaying. Default is .95. Allowed range is from 0 to 1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">11.143.1 Commands<a class="anchor hidden-xs" href="#Commands-

104" aria-hidden="true"># TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.144 latticepal# TOC</h3>

Convert packed RGB/BGR input to PAL8 with a per-frame palette whose colors are placed on a face-centered cubic (FCC) lattice spanning the RGB cube. Each frame gets its own palette, built only from the lattice colors actually used by that frame. If a frame uses more than 256 lattice colors, they are reduced by iteratively dropping the color whose removal has the least impact (its pixel count times the squared distance to the nearest remaining color) and mapping its pixels to that nearest remaining color. The alpha channel, if present, is ignored unless the <samp class="option">alpha</samp> option is set. The filter accepts the following options: <dl class="table"> <dt><samp class="option">density</samp></dt> <dd> Set the lattice density as the number of lattice steps spanning one color axis (0 to 255). Higher values give a finer lattice and thus more possible colors, making it more likely that the per-frame color reduction has to drop some of them. At a density of <code class="code">7</code> with 256 palette entries no reduction ever happens since the whole lattice then holds exactly 256 points. Allowed range is <code class="code">1</code> to <code class="code">255</code>. Default is <code class="code">20</code>. </dd> <dt><samp class="option">max_colors</samp></dt> <dd> Set the maximum number of palette entries to use. Frames using more lattice colors than this are reduced accordingly. Allowed range is <code class="code">2</code> to <code class="code">256</code>. Default is <code class="code">256</code>. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Quantize the alpha channel too, placing the palette colors on a 4 dimensional D4 lattice over RGBA instead of the FCC lattice over RGB. Inputs without an alpha channel are treated as opaque. With this option the density is limited to <code class="code">64</code>. Default is <code class="code">0</code> (alpha is ignored and the palette is fully opaque). </dd> <dt><samp class="option">refine</samp></dt> <dd> Redistribute the error introduced by dropping colors, which is otherwise uncompensated and can show as discolored patches. Frames that needed no reduction are not changed by this option. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No refinement. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Run a Floyd-Steinberg error diffusion pass over the whole frame against the final palette. Most accurate on average, but redithers everything, including regions whose colors survived. </dd> <dt>‘<samp class="samp">residual</samp>’</dt>

<dd> Diffuse only the residual between each pixel’s first pass quantized color and its final palette color. Pixels of surviving colors that receive no residual are unchanged, keeping the character of the selected dither mode outside the reduced regions. </dd> <dt>‘<samp class="samp">batched</samp>’</dt> <dd> Interleave color removal and rediffusion: drop half of the excess colors, rediffuse the whole frame against the survivors, recount the colors from the actual assignments and repeat. Colors that the dithering can reproduce by mixing become free to drop while extreme colors keep their pixels and are protected; this gives the best quality at the cost of about log2 of the number of dropped colors rediffusion passes. </dd> </dl>

Default is <code class="code">none</code>. </dd> <dt><samp class="option">dither</samp></dt> <dd> Select the dithering mode. Available algorithms are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No dithering. </dd> <dt>‘<samp class="samp">bayer</samp>’</dt> <dd> Ordered 8x8 dithering using the classic swscale dither matrix with its per-channel decorrelation, offsets spanning one lattice period. </dd> <dt>‘<samp class="samp">blue_noise</samp>’</dt> <dd> Void-and-cluster blue noise dithering using three 64x64 masks generated at initialization, one per color channel. </dd> <dt>‘<samp class="samp">floyd_steinberg</samp>’</dt> <dd> Floyd-Steinberg error diffusion. </dd> </dl>

Default is <code class="code">floyd_steinberg</code>. </dd> </dl>

 <h3 class="section">11.145 lenscorrection# TOC</h3>

Correct radial lens distortion This filter can be used to correct for radial distortion as can result from the use of wide angle lenses, and thereby re-rectify the image. To find the right parameters one can use tools available for example as part of opencv or simply trial-and-error. To use opencv use the calibration sample (under samples/cpp) from the opencv sources and extract the k1 and k2 coefficients from the resulting matrix. Note that effectively the same filter is available in the open-source tools Krita and Digikam from the KDE project. In contrast to the vignette filter, which can also be used to compensate lens errors, this filter corrects the distortion of the image, whereas vignette corrects the brightness distribution, so you may want to use both filters together in certain cases, though you will have to take care of ordering, i.e. whether vignetting should be applied before or after lens correction. <h4 class="subsection">11.145.1 Options<span

class=”pull-right”># TOC</h4>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cx</samp></dt> <dd> Relative x-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image width. Default is 0.5. </dd> <dt><samp class=”option”>cy</samp></dt> <dd> Relative y-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image height. Default is 0.5. </dd> <dt><samp class=”option”>k1</samp></dt> <dd> Coefficient of the quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>k2</samp></dt> <dd> Coefficient of the double quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>i</samp></dt> <dd> Set interpolation type. Can be <code class=”code”>nearest</code> or <code class=”code”>bilinear</code>. Default is <code class=”code”>nearest</code>. </dd> <dt><samp class=”option”>fc</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default color is <code class=”code”>black@0</code>. </dd> </dl>

The formula that generates the correction is: <var class=”var”>r_src</var> = <var class=”var”>r_tgt</var> * (1 + <var class=”var”>k1</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^2 + <var class=”var”>k2</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^4) where <var class=”var”>r_0</var> is halve of the image diagonal and <var class=”var”>r_src</var> and <var class=”var”>r_tgt</var> are the distances from the focal point in the source and target images, respectively. <h4 class=”subsection”>11.145.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.146 lensfun# TOC</h3>

Apply lens correction via the lensfun library (<a class=”url” href=”http://lensfun.sourceforge.net/”>http:/

The <code class=”code”>lensfun</code> filter requires the camera make, camera model, and lens model

to apply the lens correction. The filter will load the lensfun database and

query it to find the corresponding camera and lens entries in the database. As

long as these entries can be found with the given options, the filter can

perform corrections on frames. Note that incomplete strings will result in the

filter choosing the best match with the given options, and the filter will

output the chosen camera and lens models (logged with level "info"). You must

provide the make, camera model, and lens model as they are required.

To obtain a list of available makes and models, leave out one or both of <code class=”code”>make</code> and

<code class=”code”>model</code> options. The filter will send the full list to the log with level <code class=”code”>INFO</code>.

The first column is the make and the second column is the model.

To obtain a list of available lenses, set any values for make and model and leave out the

<code class=”code”>lens_model</code> option. The filter will send the full list of lenses in the log with level

<code class=”code”>INFO</code>. The ffmpeg tool will exit after the list is printed.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>make</samp></dt>

<dd> The make of the camera (for example, "Canon"). This option is required.

</dd>

<dt><samp class=”option”>model</samp></dt>

<dd> The model of the camera (for example, "Canon EOS 100D"). This option is

required.

</dd>

<dt><samp class=”option”>lens_model</samp></dt>

<dd> The model of the lens (for example, "Canon EF-S 18-55mm f/3.5-5.6 IS STM"). This

option is required.

</dd>

<dt><samp class=”option”>db_path</samp></dt>

<dd> The full path to the lens database folder. If not set, the filter will attempt to

load the database from the install path when the library was built. Default is unset.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> The type of correction to apply. The following values are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>vignetting</samp>’</dt>

<dd> Enables fixing lens vignetting.

</dd>

<dt>‘<samp class=”samp”>geometry</samp>’</dt>

<dd> Enables fixing lens geometry. This is the default.

</dd>

<dt>‘<samp class=”samp”>subpixel</samp>’</dt>

<dd> Enables fixing chromatic aberrations.

</dd>

<dt>‘<samp class="samp">vig_geo</samp>’</dt>

<dd> Enables fixing lens vignetting and lens geometry.

</dd>

<dt>‘<samp class="samp">vig_subpixel</samp>’</dt>

<dd> Enables fixing lens vignetting and chromatic aberrations.

</dd>

<dt>‘<samp class="samp">distortion</samp>’</dt>

<dd> Enables fixing both lens geometry and chromatic aberrations.

</dd>

<dt>‘<samp class="samp">all</samp>’</dt>

<dd> Enables all possible corrections.

</dd>

</dl>

</dd>

<dt><samp class="option">focal_length</samp></dt>

<dd> The focal length of the image/video (zoom; expected constant for video). For

example, a 18–55mm lens has focal length range of [18–55], so a value in that

range should be chosen when using that lens. Default 18.

</dd>

<dt><samp class="option">aperture</samp></dt>

<dd> The aperture of the image/video (expected constant for video). Note that

aperture is only used for vignetting correction. Default 3.5.

</dd>

<dt><samp class="option">focus_distance</samp></dt>

<dd> The focus distance of the image/video (expected constant for video). Note that

focus distance is only used for vignetting and only slightly affects the

vignetting correction process. If unknown, leave it at the default value (which

is 1000).

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> The scale factor which is applied after transformation. After correction the

video is no longer necessarily rectangular. This parameter controls how much of

the resulting image is visible. The value 0 means that a value will be chosen

automatically such that there is little or no unmapped area in the output

image. 1.0 means that no additional scaling is done. Lower values may result

in more of the corrected image being visible, while higher values may avoid

unmapped areas in the output.

</dd>

<dt><samp class="option">target_geometry</samp></dt>

<dd> The target geometry of the output image/video. The following values are valid

options:

<dl class="table">

<dt>‘<samp class="samp">rectilinear (default)</samp>’</dt>

<dt>‘<samp class="samp">fisheye</samp>’</dt>

<dt>‘<samp class="samp">panoramic</samp>’</dt>

<dt>‘<samp class="samp">equirectangular</samp>’</dt>

<dt>‘<samp class="samp">fisheye_orthographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_stereographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_equisolid</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_thoby</samp>’</dt>

</dl>

</dd>

<dt><samp class=”option”>reverse</samp></dt>

<dd> Apply the reverse of image correction (instead of correcting distortion, apply

it).

</dd>

<dt><samp class=”option”>interpolation</samp></dt>

<dd> The type of interpolation used when correcting distortion. The following values

are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>nearest</samp>’</dt>

<dt>‘<samp class=”samp”>linear (default)</samp>’</dt>

<dt>‘<samp class=”samp”>lanczos</samp>’</dt>

</dl>

</dd>

</dl>

 <h4 class=”subsection”>11.146.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply lens correction with make "Canon", camera model "Canon EOS 100D", and lens model "Canon EF-S 18-55mm f/3.5-5.6 IS STM" with focal length of "18" and aperture of "8.0".

 <pre class=”example-preformatted”>ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon

EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8 -c:v h264
-b:v 8000k output.mov </pre>

Apply the same as before, but only for the first 5 seconds of video.

 <pre class="example-preformatted">ffmpeg -i in-
put.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon
EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8:enable='lte(t\,5)'
-c:v h264 -b:v 8000k output.mov </pre>

 <h3 class="section">11.147 lcevc<span class="pull-
right">#
TOC</h3>

Low Complexity Enhancement Video Codec filter based on liblcevc_dec
(https://github.com/v-
novaltd/LCEVCdec). <h3
class="section">11.148 libplacebo<a class="anchor
hidden-xs" href="#libplacebo" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-libplacebo" aria-hidden="true">TOC</h3>

Flexible GPU-accelerated processing filter based on libplacebo (<a
class="url" href="https://code.videolan.org/videolan/libplacebo">https://code.videolan.org/videolan/libplac
  <h4 class="subsection">11.148.1 Op-
tions<a class="anchor hidden-xs" href="#Options-
1" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Options-1" aria-hidden="true">TOC</h4>

The options for this filter are divided into the following sections: <a
name="Output-mode"> <h4 class="subsubsection">11.148.1.1 Output
mode<a class="anchor hidden-xs" href="#Output-
mode" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Output-mode" aria-hidden="true">TOC</h4> These
options control the overall output mode. By default, libplacebo will try
to preserve the source colorimetry and size as best as it can, but it will
apply any embedded film grain, dolby vision metadata or anamorphic
SAR present in source frames. <dl class="table"> <dt><samp
class="option">inputs</samp></dt> <dd> Set the number of in-
puts. This can be used, alongside the <code class="code">idx</code>
variable, to allow placing/blending multiple inputs inside the output
frame. This effectively enables functionality similar to <a class="ref"
href="#hstack">hstack, overlay,
etc. </dd> <dt><samp class="option">w</samp></dt>
<dt><samp class="option">h</samp></dt> <dd> Set the output
video dimension expression. Default values are <code class="code">iw</code>
and <code class="code">ih</code>. Allows for the same ex-
pressions as the scale filter.

</dd> <dt><samp class="option">crop_x</samp></dt> <dt><samp class="option">crop_y</samp></dt> <dd> Set the input crop x/y expressions, default values are <code class="code">(iw-cw)/2</code> and <code class="code">(ih-ch)/2</code>. </dd> <dt><samp class="option">crop_w</samp></dt> <dt><samp class="option">crop_h</samp></dt> <dd> Set the input crop width/height expressions, default values are <code class="code">iw</code> and <code class="code">ih</code>. </dd> <dt><samp class="option">pos_x</samp></dt> <dt><samp class="option">pos_y</samp></dt> <dd> Set the output placement x/y expressions, default values are <code class="code">(ow-pw)/2</code> and <code class="code">(oh-ph)/2</code>. </dd> <dt><samp class="option">pos_w</samp></dt> <dt><samp class="option">pos_h</samp></dt> <dd> Set the output placement width/height expressions, default values are <code class="code">ow</code> and <code class="code">oh</code>. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Rotate the input frame clockwise by the specified angle. <dl class="table"> <dt>‘<samp class="samp">0, 360</samp>’</dt> <dt>‘<samp class="samp">90</samp>’</dt> <dt>‘<samp class="samp">180</samp>’</dt> <dt>‘<samp class="samp">270</samp>’</dt> </dl>

</dd> <dt><samp class="option">fps</samp></dt> <dd> Set the output frame rate. This can be rational, e.g. <code class="code">60000/1001</code>. If set to the special string <code class="code">none</code> (the default), input timestamps will instead be passed through to the output unmodified. Otherwise, the input video frames will be interpolated as necessary to rescale the video to the specified target framerate, in a manner as determined by the <samp class="option">frame_mixer</samp> option. </dd> <dt><samp class="option">format</samp></dt> <dd> Set the output format override. If unset (the default), frames will be output in the same format as the respective input frames. Otherwise, format conversion will be performed. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. Note that <samp class="option">force_divisible_by</samp> also works with <code class="code">fit_sense=constraint</code>. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> If enabled, output frames will always have a pixel aspect ratio of 1:1. If disabled (the default), any aspect ratio mismatches, including those from e.g. anamorphic video sources, are forwarded to the output pixel aspect ratio. </dd> <dt><samp class="option">normalize_sar</samp></dt> <dd> Like <samp class="option">reset_sar</samp>, but instead of stretching the video content to fill the new output aspect ratio, the content is instead padded or cropped as necessary. Mutually exclusive with <samp class="option">fit_mode</samp>. Disabled by default.

 </dd> <dt><samp class="option">pad_crop_ratio</samp></dt> <dd> Specifies a ratio (between <code class="code">0.0</code> and <code class="code">1.0</code>) between padding and cropping when the input aspect ratio does not match the output aspect ratio and <samp class="option">normalize_sar</samp> is in effect. The default of <code class="code">0.0</code> always pads the content with black borders, while a value of <code class="code">1.0</code> always crops off parts of the content. Intermediate values are possible, leading to a mix of the two approaches. </dd> <dt><samp class="option">fit_mode</samp></dt> <dd> Specify the content fit strategy according to a list of predefined modes. Determines how the input image is to be placed inside the destination crop rectangle (as defined by <code class="code">pos_x/y</code> and <code class="code">pos_w/h</code>). The names and their implementations are taken from the CSS ’object-fit’ property. Note that this option is mutually exclusive with <samp class="option">normalize_sar</samp>. Defaults to <code class="code">fill</code>. Valid values are: <dl class="table"> <dt>‘<samp class="samp">fill</samp>’</dt> <dd> Stretch the input to the output rectangle, ignoring aspect ratio mismatches. Note that unless <samp class="option">reset_sar</samp> is also enabled, the output will still have the correct pixel aspect ratio tagged. </dd> <dt>‘<samp class="samp">contain</samp>’</dt> <dd> Scale the input to fit inside the output, preserving aspect ratio by padding. Equivalent to <samp class="option">normalize_sar</samp> with <samp class="option">pad_crop_ratio</samp> set to <code class="code">0.0</code>. </dd> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> Scale the input to fill the output, preserving aspect ratio by cropping. Equivalent to <samp class="option">normalize_sar</samp> with <samp class="option">pad_crop_ratio</samp> set to <code class="code">1.0</code>. </dd> <dt>‘<samp class="samp">none, place</samp>’</dt> <dd> Don’t scale the input. The input will be placed inside the output rectangle at its natural size; which may result in additional padding or cropping. </dd> <dt>‘<samp class="samp">scale_down</samp>’</dt> <dd> Scale the input down as much as needed to fit inside the output. Equivalent to either <code class="code">contain</code> or <code class="code">none</code>, depending on whether the input is larger than the output or not. </dd> </dl>

</dd> <dt><samp class="option">fit_sense</samp></dt> <dd> When <samp class="option">fit_mode</samp> is in use, this option controls how the fit strategy is applied against the specified output resolution. Mutually exclusive with <samp class="option">force_original_aspect_ratio</samp>. Valid values are: <dl class="table"> <dt>‘<samp class="samp">target</samp>’</dt> <dd> The computed output resolution is taken as the exact size of the output frame. This is the default behavior. </dd> <dt>‘<samp

class=”samp”>constraint</samp>’</dt> <dd> The computed output resolution is a size reference against which the fit mode is applied, enlarging or decreasing the true frame size as needed to fit the content. </dd> </dl>

</dd> <dt><samp class=”option”>fillcolor</samp></dt> <dd> Set the color used to fill the output area not covered by the output image, for example as a result of <samp class=”option”>normalize_sar</samp>. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Defaults to <code class=”code”>black@0</code>. </dd> <dt><samp class=”option”>corner_rounding</samp></dt> <dd> Render frames with rounded corners. The value, given as a float ranging from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>, indicates the relative degree of rounding, from fully square to fully circular. In other words, it gives the radius divided by half the smaller side length. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>lut</samp></dt> <dd> Specifies a custom LUT (in Adobe .cube format) to apply to the colors as part of color conversion. The exact interpretation depends on the value of <samp class=”option”>lut_type</samp>. </dd> <dt><samp class=”option”>lut_type</samp></dt> <dd> Controls the interpretation of color values fed to and from the LUT specified as <samp class=”option”>lut</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Chooses the interpretation of the LUT automatically from tagged metadata, and otherwise falls back to ‘<samp class=”samp”>native</samp>’. (Default) </dd> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Applied to raw image contents in its native RGB colorspace (nonlinear light), before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>normalized</samp>’</dt> <dd> Applied to the normalized RGB image contents, in linear light, before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>conversion</samp>’</dt> <dd> Fully replaces the conversion from the image color space to the output color space. If such a LUT is present, it has the highest priority, and overrides any ICC profiles, as well as options related to tone mapping and output colorimetry (<samp class=”option”>color_primaries</samp>, <samp class=”option”>color_trc</samp>). </dd> </dl>

</dd> <dt><samp class=”option”>extra_opts</samp></dt> <dd> Pass extra libplacebo internal configuration options. These can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to configure a custom filter kernel ("EWA LanczosSharp") and use it to double the input image resolution: <pre class=”example-preformatted”>-vf "libplacebo=w=iw*2:h=ih*2:extra_opts='upscaler=custom\:upscaler_preset=ewa_

</pre>

</dd> <dt><samp class=”option”>shader_cache</samp></dt> <dd> File path of a cache directory that libplacebo will use to store and load cached shader objects. This cache is not cleaned up automatically. If the path does not end in a directory separator, the generated filenames will be effectively prefixed by the last path component. All directories must already exist.   <pre class=”example-preformatted”>-vf "libplacebo=shader_cache=/tmp/pl-shader-" </pre>

</dd> <dt><samp class=”option”>inherit_device</samp></dt> <dd> Take the Vulkan device from the hardware frames arriving on the input, instead of creating a private one from the filter’s own hardware device. This is useful when the input frames originate on another API (for example CUDA frames uploaded to Vulkan by <code class=”code”>hwupload</code>) and the filter’s device is not itself a Vulkan device: libplacebo would otherwise spin up a separate Vulkan device that the input frames do not belong to, and the frames could not be mapped. When enabled, the filter requires Vulkan hardware frames on the input and will fail negotiation otherwise. Disabled by default.   <pre class=”example-preformatted”>-init_hw_device vulkan=vk:0 -init_hw_device cuda@vk -filter_hw_device cuda0 ... -vf "hwupload=derive_device=vulkan,libplacebo=inherit_device=1" </pre>

</dd> <dt><samp class=”option”>colorspace</samp></dt> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dt><samp class=”option”>range</samp></dt> <dt><samp class=”option”>chroma_location</samp><
<dd> Configure the colorspace that output frames will be delivered in. For any value other than the default <code class=”code”>auto</code>, conversion will be performed. With <code class=”code”>auto</code>, <samp class=”option”>color_primaries</samp> and <samp class=”option”>color_trc</samp> keep the values of the input frames, while <samp class=”option”>colorspace</samp>, <samp class=”option”>range</samp> and <samp class=”option”>chroma_location</samp> use the values negotiated with the filtergraph, which are those of the input unless a following filter requires others. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> Choose the desired output alpha mode, when the output format has an alpha channel. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>apply_filmgrain</samp></dt> <dd> Apply film grain (e.g. AV1 or H.274) if present in source frames, and strip it from the output. Enabled by default. </dd> <dt><samp class=”option”>apply_dolbyvision</samp></dt> <dd> Apply Dolby Vision RPU metadata if present in source frames, and strip it from the output. Enabled by default. Note that Dolby Vision will always output BT.2020+PQ, overriding the usual input frame metadata. These will also be picked as the

values of <code class="code">auto</code> for the respective frame output options. </dd> </dl>

In addition to the expression constants documented for the scale filter, the <samp class="option">crop_w</samp>, <samp class="option">crop_h</samp>, <samp class="option">crop_x</samp>, <samp class="option">crop_y</samp>, <samp class="option">pos_w</samp>, <samp class="option">pos_h</samp>, <samp class="option">pos_x</samp> and <samp class="option">pos_y</samp> options can also contain the following constants: <dl class="table"> <dt><samp class="option">in_idx, idx</samp></dt> <dd> The (0-based) numeric index of the currently active input stream. </dd> <dt><samp class="option">crop_w, cw</samp></dt> <dt><samp class="option">crop_h, ch</samp></dt> <dd> The computed values of <samp class="option">crop_w</samp> and <samp class="option">crop_h</samp>. </dd> <dt><samp class="option">pos_w, pw</samp></dt> <dt><samp class="option">pos_h, ph</samp></dt> <dd> The computed values of <samp class="option">pos_w</samp> and <samp class="option">pos_h</samp>. </dd> <dt><samp class="option">in_t, t</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class="option">out_t, ot</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class="option">n</samp></dt> <dd> The input frame number, starting with 0. </dd> </dl>

 <h4 class="subsubsection">11.148.1.2 Scaling# TOC</h4> The options in this section control how libplacebo performs upscaling and (if necessary) downscaling. Note that libplacebo will always internally operate on 4:4:4 content, so any sub-sampled chroma formats such as <code class="code">yuv420p</code> will necessarily be upsampled and downsampled as part of the rendering process. That means scaling might be in effect even if the source and destination resolution are the same. <dl class="table"> <dt><samp class="option">upscaler</samp></dt> <dt><samp class="option">downscaler</samp></dt> <dd> Configure the filter kernel used for upscaling and downscaling. The respective defaults are <code class="code">spline36</code> and <code class="code">mitchell</code>. For a full list of possible values, pass <code class="code">help</code> to these options. The most important values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Forces the use of built-in GPU texture sampling (typically bilinear). Extremely fast but poor quality, especially when downscaling. </dd> <dt>‘<samp class="samp">bilinear</samp>’</dt> <dd> Bilinear interpolation. Can generally be done for free on GPUs, except when doing so would lead to aliasing. Fast and low quality. </dd> <dt>‘<samp

class=”samp”>nearest</samp>’</dt> <dd> Nearest-neighbour interpolation. Sharp but highly aliasing. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Algorithm that looks visually similar to nearest-neighbour interpolation but tries to preserve pixel aspect ratio. Good for pixel art, since it results in minimal distortion of the artistic appearance. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Standard sinc-sinc interpolation kernel. </dd> <dt>‘<samp class=”samp”>spline36</samp>’</dt> <dd> Cubic spline approximation of lanczos. No difference in performance, but has very slightly less ringing. </dd> <dt>‘<samp class=”samp”>ewa_lanczos</samp>’</dt> <dd> Elliptically weighted average version of lanczos, based on a jinc-jinc kernel. This is also popularly referred to as just "Jinc scaling". Slow but very high quality. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> Gaussian kernel. Has certain ideal mathematical properties, but subjectively very blurry. </dd> <dt>‘<samp class=”samp”>mitchell</samp>’</dt> <dd> Cubic BC spline with parameters recommended by Mitchell and Netravali. Very little ringing. </dd> </dl>

</dd> <dt><samp class=”option”>frame_mixer</samp></dt> <dd> Controls the kernel used for mixing frames temporally. The default value is <code class=”code”>none</code>, which disables frame mixing. For a full list of possible values, pass <code class=”code”>help</code> to this option. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables frame mixing, giving a result equivalent to "nearest neighbour" semantics. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Oversamples the input video to create a "Smooth Motion"-type effect: if an output frame would exactly fall on the transition between two video frames, it is blended according to the relative overlap. This is the recommended option whenever preserving the original subjective appearance is desired. </dd> <dt>‘<samp class=”samp”>mitchell_clamp</samp>’</dt> <dd> Larger filter kernel that smoothly interpolates multiple frames in a manner designed to eliminate ringing and other artefacts as much as possible. This is the recommended option wherever maximum visual smoothness is desired. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linear blend/fade between frames. Especially useful for constructing e.g. slideshows. </dd> </dl>

</dd> <dt><samp class=”option”>antiringing</samp></dt> <dd> Enables anti-ringing (for non-EWA filters). The value (between <code class=”code”>0.0</code> and <code class=”code”>1.0</code>) configures the strength of the anti-ringing algorithm. May increase aliasing if set too high. Disabled by default. </dd> <dt><samp class=”option”>sigmoid</samp></dt> <dd> Enable sigmoidal compression during upscaling. Reduces ringing

slightly. Enabled by default. </dd> </dl>

 <h4 class="subsubsection">11.148.1.3 Deinterlacing# TOC</h4> Deinterlacing is automatically supported when frames are tagged as interlaced, however frames are not deinterlaced unless a deinterlacing algorithm is chosen. <dl class="table"> <dt><samp class="option">deinterlace</samp></dt> <dd> The the deinterlacing algorithm to use. <dl class="table"> <dt>‘<samp class="samp">weave</samp>’</dt> <dd> No deinterlacing, weave fields together into a single frame. This is the default. </dd> <dt>‘<samp class="samp">bob</samp>’</dt> <dd> Naive bob deinterlacing, simply repeat each field line twice. </dd> <dt>‘<samp class="samp">yadif</samp>’</dt> <dd> Yet another deinterlacing filter. See the yadif filter for more details. </dd> <dt>‘<samp class="samp">bwdif</samp>’</dt> <dd> Bob weaver deinterlacing filter. See the bwdif filter for more details. </dd> </dl>

</dd> <dt><samp class="option">skip_spatial_check</samp></dt> <dd> Skip the spatial deinterlacing check when using <code class="code">yadif</code> deinterlacing. </dd> <dt><samp class="option">send_fields</samp></dt> <dd> Output a frame for each field, rather than for each frame. Note that this will always double the tagged output frame rate, even if the input does not contain any interlaced frames. Disabled by default. </dd> </dl>

 <h4 class="subsubsection">11.148.1.4 Debanding# TOC</h4> Libplacebo comes with a built-in debanding filter that is good at counteracting many common sources of banding and blocking. Turning this on is highly recommended whenever quality is desired. <dl class="table"> <dt><samp class="option">deband</samp></dt> <dd> Enable (fast) debanding algorithm. Disabled by default. </dd> <dt><samp class="option">deband_iterations</samp></dt> <dd> Number of deband iterations of the debanding algorithm. Each iteration is performed with progressively increased radius (and diminished threshold). Recommended values are in the range <code class="code">1</code> to <code class="code">4</code>. Defaults to <code class="code">1</code>. </dd> <dt><samp class="option">deband_threshold</samp></dt> <dd> Debanding filter strength. Higher numbers lead to more aggressive debanding. Defaults to <code class="code">4.0</code>. </dd> <dt><samp class="option">deband_radius</samp></dt> <dd> Debanding filter radius. A higher radius is better for slow

gradients, while a lower radius is better for steep gradients. Defaults to <code class=”code”>16.0</code>. </dd> <dt><samp class=”option”>deband_grain</samp></dt> <dd> Amount of extra output grain to add. Helps hide imperfections. Defaults to <code class=”code”>6.0</code>. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.5 Color adjustment# TOC</h4> A collection of subjective color controls. Not very rigorous, so the exact effect will vary somewhat depending on the input primaries and colorspace. <dl class=”table”> <dt><samp class=”option”>brightness</samp></dt> <dd> Brightness boost, between <code class=”code”>-1.0</code> and <code class=”code”>1.0</code>. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Contrast gain, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Saturation gain, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>hue</samp></dt> <dd> Hue shift in radians, between <code class=”code”>-3.14</code> and <code class=”code”>3.14</code>. Defaults to <code class=”code”>0.0</code>. This will rotate the UV subvector, defaulting to BT.709 coefficients for RGB inputs. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Gamma adjustment, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>temperature</samp></dt> <dd> Color temperature adjustment. Lower values make the output warmer/redder, down to <code class=”code”>1667</code>, while higher values make the output cooler/bluer, up to <code class=”code”>25000</code>. Defaults to <code class=”code”>6500</code> (neutral white). </dd> <dt><samp class=”option”>cones</samp></dt> <dd> Cone model to use for color blindness simulation. Accepts any combination of <code class=”code”>l</code>, <code class=”code”>m</code> and <code class=”code”>s</code>. Here are some examples: <dl class=”table”> <dt>‘<samp class=”samp”>m</samp>’</dt> <dd> Deuteranomaly / deuteranopia (affecting 3%-4% of the population) </dd> <dt>‘<samp class=”samp”>l</samp>’</dt> <dd> Protanomaly / protanopia (affecting 1%-2% of the population) </dd> <dt>‘<samp class=”samp”>l+m</samp>’</dt> <dd> Monochromacy (very rare) </dd> <dt>‘<samp class=”samp”>l+m+s</samp>’</dt> <dd> Achromatopsy (complete loss of daytime vision, extremely rare) </dd> </dl>

</dd> <dt><samp class=”option”>cone-strength</samp></dt> <dd> Gain

factor for the cones specified by <code class=”code”>cones</code>, between <code class=”code”>0.0</code> and <code class=”code”>10.0</code>. A value of <code class=”code”>1.0</code> results in no change to color vision. A value of <code class=”code”>0.0</code> (the default) simulates complete loss of those cones. Values above <code class=”code”>1.0</code> result in exaggerating the differences between cones, which may help compensate for reduced color vision. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.6 Peak detection# TOC</h4> To help deal with sources that only have static HDR10 metadata (or no tagging whatsoever), libplacebo uses its own internal frame analysis compute shader to analyze source frames and adapt the tone mapping function in realtime. If this is too slow, or if exactly reproducible frame-perfect results are needed, it’s recommended to turn this feature off. <dl class=”table”> <dt><samp class=”option”>peak_detect</samp></dt> <dd> Enable HDR peak detection. Ignores static MaxCLL/MaxFALL values in favor of dynamic detection from the input. Note that the detected values do not get written back to the output frames, they merely guide the internal tone mapping process. Enabled by default. </dd> <dt><samp class=”option”>smoothing_period</samp></dt> <dd> Peak detection smoothing period, between <code class=”code”>0.0</code> and <code class=”code”>1000.0</code>. Higher values result in peak detection becoming less responsive to changes in the input. Defaults to <code class=”code”>20.0</code>. </dd> <dt><samp class=”option”>scene_threshold_low</samp></dt> <dt><samp class=”option”>scene_threshold_high</
<dd> Lower and upper thresholds for scene change detection. Expressed in a logarithmic scale between <code class=”code”>0.0</code> and <code class=”code”>100.0</code>. Default to <code class=”code”>1.0</code> and <code class=”code”>3.0</code>, respectively. Setting either to a negative value disables this functionality. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Which percentile of the frame brightness histogram to use as the source peak for tone-mapping. Defaults to <code class=”code”>99.995</code>, a fairly conservative value. Setting this to <code class=”code”>100.0</code> disables frame histogram measurement and instead uses the true peak brightness for tone-mapping. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.7 Tone mapping# TOC</h4> The options in this section control how libplacebo performs tone-mapping and gamut-mapping when dealing with mismatches between wide-gamut or HDR content. In general, libplacebo relies on accurate source tagging and

mastering display gamut information to produce the best results. <dl class=”table”> <dt><samp class=”option”>gamut_mode</samp></dt> <dd> How to handle out-of-gamut colors that can occur as a result of colorimetric gamut mapping. <dl class=”table”> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Do nothing, simply clip out-of-range colors to the RGB volume. Low quality but extremely fast. </dd> <dt>‘<samp class=”samp”>perceptual</samp>’</dt> <dd> Perceptually soft-clip colors to the gamut volume. This is the default. </dd> <dt>‘<samp class=”samp”>relative</samp>’</dt> <dd> Relative colorimetric hard-clip. Similar to <code class=”code”>perceptual</code> but without the soft knee. </dd> <dt>‘<samp class=”samp”>saturation</samp>’</dt> <dd> Saturation mapping, maps primaries directly to primaries in RGB space. Not recommended except for artificial computer graphics for which a bright, saturated display is desired. </dd> <dt>‘<samp class=”samp”>absolute</samp>’</dt> <dd> Absolute colorimetric hard-clip. Performs no adjustment of the white point. </dd> <dt>‘<samp class=”samp”>desaturate</samp>’</dt> <dd> Hard-desaturates out-of-gamut colors towards white, while preserving the luminance. Has a tendency to distort the visual appearance of bright objects. </dd> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dd> Linearly reduces content brightness to preserves saturated details, followed by clipping the remaining out-of-gamut colors. </dd> <dt>‘<samp class=”samp”>warn</samp>’</dt> <dd> Highlight out-of-gamut pixels (by inverting/marking them). </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly reduces chromaticity of the entire image to make it fit within the target color volume. Be careful when using this on BT.2020 sources without proper mastering metadata, as doing so will lead to excessive desaturation. </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping</samp></dt> <dd> Tonemapping algorithm to use. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection based on internal heuristics. This is the default. </dd> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Performs no tone-mapping, just clips out-of-range colors. Retains perfect color accuracy for in-range colors but completely destroys out-of-range information. Does not perform any black point adaptation. Not configurable. </dd> <dt>‘<samp class=”samp”>st2094-40</samp>’</dt> <dd> EETF from SMPTE ST 2094-40 Annex B, which applies the Bezier curves from HDR10+ dynamic metadata based on Bezier curves to perform tone-mapping. The OOTF used is adjusted based on the ratio between the targeted and actual display peak luminances. </dd> <dt>‘<samp class=”samp”>st2094-10</samp>’</dt> <dd> EETF from SMPTE ST 2094-10 Annex B.2, which takes into account the input signal average luminance in addition to the maximum/minimum. The configurable contrast parameter influences the slope of the linear out-

put segment, defaulting to <code class=”code”>1.0</code> for no increase/decrease in contrast. Note that this does not currently include the subjective gain/offset/gamma controls defined in Annex B.3. </dd> <dt>‘<samp class=”samp”>bt.2390</samp>’</dt> <dd> EETF from the ITU-R Report BT.2390, a hermite spline roll-off with linear segment. The knee point offset is configurable. Note that this parameter defaults to <code class=”code”>1.0</code>, rather than the value of <code class=”code”>0.5</code> from the ITU-R spec. </dd> <dt>‘<samp class=”samp”>bt.2446a</samp>’</dt> <dd> EETF from ITU-R Report BT.2446, method A. Designed for well-mastered HDR sources. Can be used for both forward and inverse tone mapping. Not configurable. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Simple spline consisting of two polynomials, joined by a single pivot point. The parameter gives the pivot point (in PQ space), defaulting to <code class=”code”>0.30</code>. Can be used for both forward and inverse tone mapping. </dd> <dt>‘<samp class=”samp”>reinhard</samp>’</dt> <dd> Simple non-linear, global tone mapping algorithm. The parameter specifies the local contrast coefficient at the display peak. Essentially, a parameter of <code class=”code”>0.5</code> implies that the reference white will be about half as bright as when clipping. Defaults to <code class=”code”>0.5</code>, which results in the simplest formulation of this function. </dd> <dt>‘<samp class=”samp”>mobius</samp>’</dt> <dd> Generalization of the reinhard tone mapping algorithm to support an additional linear slope near black. The tone mapping parameter indicates the trade-off between the linear section and the non-linear section. Essentially, for a given parameter <var class=”var”>x</var>, every color value below <var class=”var”>x</var> will be mapped linearly, while higher values get non-linearly tone-mapped. Values near <code class=”code”>1.0</code> make this curve behave like <code class=”code”>clip</code>, while values near <code class=”code”>0.0</code> make this curve behave like <code class=”code”>reinhard</code>. The default value is <code class=”code”>0.3</code>, which provides a good balance between colorimetric accuracy and preserving out-of-gamut details. </dd> <dt>‘<samp class=”samp”>hable</samp>’</dt> <dd> Piecewise, filmic tone-mapping algorithm developed by John Hable for use in Uncharted 2, inspired by a similar tone-mapping algorithm used by Kodak. Popularized by its use in video games with HDR rendering. Preserves both dark and bright details very well, but comes with the drawback of changing the average brightness quite significantly. This is sort of similar to <code class=”code”>reinhard</code> with parameter <code class=”code”>0.24</code>. </dd> <dt>‘<samp class=”samp”>gamma</samp>’</dt> <dd> Fits a gamma (power) function to transfer between the source and target color spaces, effectively resulting in a perceptual hard-knee joining two roughly linear sections. This preserves details at all scales fairly accurately, but can result

in an image with a muted or dull appearance. The parameter is used as the cutoff point, defaulting to <code class=”code”>0.5</code>. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly stretches the input range to the output range, in PQ space. This will preserve all details accurately, but results in a significantly different average brightness. Can be used for inverse tone-mapping in addition to regular tone-mapping. The parameter can be used as an additional linear gain coefficient (defaulting to <code class=”code”>1.0</code>). </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping_param</samp></dt> <dd> For tunable tone mapping functions, this parameter can be used to fine-tune the curve behavior. Refer to the documentation of <code class=”code”>tonemapping</code>. The default value of <code class=”code”>0.0</code> is replaced by the curve’s preferred default setting. </dd> <dt><samp class=”option”>inverse_tonemapping</samp></dt> <dd> If enabled, this filter will also attempt stretching SDR signals to fill HDR output color volumes. Disabled by default. </dd> <dt><samp class=”option”>tonemapping_lut_size</samp></dt> <dd> Size of the tone-mapping LUT, between <code class=”code”>2</code> and <code class=”code”>1024</code>. Defaults to <code class=”code”>256</code>. Note that this figure is squared when combined with <code class=”code”>peak_detect</code>. </dd> <dt><samp class=”option”>contrast_recovery</samp></dt> <dd> Contrast recovery strength. If set to a value above <code class=”code”>0.0</code>, the source image will be divided into high-frequency and low-frequency components, and a portion of the high-frequency image is added back onto the tone-mapped output. May cause excessive ringing artifacts for some HDR sources, but can improve the subjective sharpness and detail left over in the image after tone-mapping. Defaults to <code class=”code”>0.30</code>. </dd> <dt><samp class=”option”>contrast_smoothness</samp></dt> <dd> Contrast recovery lowpass kernel size. Defaults to <code class=”code”>3.5</code>. Increasing or decreasing this will affect the visual appearance substantially. Has no effect when <code class=”code”>contrast_recovery</code> is disabled. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.8 Dithering# TOC</h4> By default, libplacebo will dither whenever necessary, which includes rendering to any integer format below 16-bit precision. It’s recommended to always leave this on, since not doing so may result in visible banding in the output, even if the <code class=”code”>debanding</code> filter is enabled. If maximum performance is needed, use <code class=”code”>ordered_fixed</code> instead of disabling dithering. <dl class=”table”> <dt><samp class=”option”>dithering</samp></dt> <dd> Dithering method to use. Accepts the following values: <dl class=”table”> <dt>‘<samp

class=”samp”>none</samp>’</dt> <dd> Disables dithering completely. May result in visible banding. </dd> <dt>‘<samp class=”samp”>blue</samp>’</dt> <dd> Dither with pseudo-blue noise. This is the default. </dd> <dt>‘<samp class=”samp”>ordered</samp>’</dt> <dd> Tunable ordered dither pattern. </dd> <dt>‘<samp class=”samp”>ordered_fixed</samp>’</dt> <dd> Faster ordered dither with a fixed size of <code class=”code”>6</code>. Texture-less. </dd> <dt>‘<samp class=”samp”>white</samp>’</dt> <dd> Dither with white noise. Texture-less. </dd> </dl>

</dd> <dt><samp class=”option”>dither_lut_size</samp></dt> <dd> Dither LUT size, as log base2 between <code class=”code”>1</code> and <code class=”code”>8</code>. Defaults to <code class=”code”>6</code>, corresponding to a LUT size of <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>dither_temporal</samp></dt> <dd> Enables temporal dithering. Disabled by default. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.9 Custom shaders# TOC</h4> libplacebo supports a number of custom shaders based on the mpv .hook GLSL syntax. A collection of such shaders can be found here: https://github.com/mpv-player/mpv/wiki/User-Scripts#user-shaders A full description of the mpv shader format is beyond the scope of this section, but a summary can be found here: https://mpv.io/manual/master/#options-glsl-shader <dl class=”table”> <dt><samp class=”option”>custom_shader_path</samp></dt> <dd> Specifies a path to a custom shader file to load at runtime. </dd> <dt><samp class=”option”>custom_shader_bin</samp></dt> <dd> Specifies a complete custom shader as a raw string. </dd> </dl>

 <h4 class=”subsubsection”>11.148.1.10 Debugging / performance# TOC</h4> All of the options in this section default off. They may be of assistance when attempting to squeeze the maximum performance at the cost of quality. <dl class=”table”> <dt><samp class=”option”>skip_aa</samp></dt> <dd> Disable anti-aliasing when downscaling. </dd> <dt><samp class=”option”>disable_linear</samp></dt> <dd> Disable linear light scaling. </dd> <dt><samp class=”option”>disable_builtin</samp></dt>

<dd> Disable built-in GPU sampling (forces LUT). </dd> <dt><samp class="option">disable_fbos</samp></dt> <dd> Forcibly disable FBOs, resulting in loss of almost all functionality, but offering the maximum possible speed. </dd> </dl>

 <h4 class="subsection">11.148.2 Commands# TOC</h4> This filter supports almost all of the above options as commands. The options negotiated with the filtergraph, <samp class="option">format</samp>, <samp class="option">colorspace</samp>, <samp class="option">range</samp>, <samp class="option">chroma_location</samp> and <samp class="option">alpha_mode</samp>, are fixed once the filtergraph is configured and can not be changed with commands. <h4 class="subsection">11.148.3 Examples# TOC</h4> <ul class="itemize mark-bullet"> Tone-map input to standard gamut BT.709 output: <pre class="example-preformatted">libplacebo=colorspace=bt709:color_primaries=bt709
</pre>

Rescale input to fit into standard 1080p, with high quality scaling: <pre class="example-preformatted">libplacebo=w=1920:h=1080:force_original_asp
</pre>

Interpolate low FPS / VFR input to smoothed constant 60 fps output: <pre class="example-preformatted">libplacebo=fps=60:frame_mixer=mitchell_clam
</pre>

Convert input to standard sRGB JPEG: <pre class="example-preformatted">libplacebo=format=yuv420p:colorspace=bt470bg:color_primaries=bt709
2-1:range=pc </pre>

Use higher quality debanding settings: <pre class="example-preformatted">libplacebo=deband=true:deband_iterations=3:deband_radius=8:deband_th
</pre>

Run this filter on the CPU, on systems with Mesa installed (and with the most expensive options disabled): <pre class="example-preformatted">ffmpeg ... -init_hw_device vulkan:llvmpipe ... -vf libplacebo=upscaler=none:downscaler=none:peak_detect=false
</pre>

Suppress CPU-based AV1/H.274 film grain application in the decoder, in favor of doing it with this filter. Note that this is only a gain if the frames are either already on the GPU, or if you’re using libplacebo for other purposes, since otherwise the VRAM roundtrip will more than offset any expected speedup. <pre

class=”example-preformatted”>ffmpeg -export_side_data +film_grain ... -vf libplacebo=apply_filmgrain=true </pre>

Interop with VAAPI hwdec to avoid round-tripping through RAM: <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -hwaccel vaapi -hwaccel_output_format vaapi ... -vf libplacebo </pre>

 <h3 class=”section”>11.149 libvmaf# TOC</h3>

Calculate the VMAF (Video Multi-Method Assessment Fusion) score for a reference/distorted pair of input videos. The first input is the distorted video, and the second input is the reference video. The obtained VMAF score is printed through the logging system. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-libvmaf</code>. The filter has following options: <dl class=”table”> <dt><samp class=”option”>model</samp></dt> <dd> A ‘|‘ delimited list of vmaf models. Each model can be configured with a number of parameters. Default value: <code class=”code”>"version=vmaf_v0.6.1"</code> </dd> <dt><samp class=”option”>feature</samp></dt> <dd> A ‘|‘ delimited list of features. Each feature can be configured with a number of parameters. </dd> <dt><samp class=”option”>log_path</samp></dt> <dd> Set the file path to be used to store log files. </dd> <dt><samp class=”option”>log_fmt</samp></dt> <dd> Set the format of the log file (xml, json, csv, or sub). </dd> <dt><samp class=”option”>pool</samp></dt> <dd> Set the pool method to be used for computing vmaf. Options are <code class=”code”>min</code>, <code class=”code”>harmonic_mean</code> or <code class=”code”>mean</code> (default). </dd> <dt><samp class=”option”>n_threads</samp></dt> <dd> Set number of threads to be used when initializing libvmaf. Default value: <code class=”code”>0</code>, no threads. </dd> <dt><samp class=”option”>n_subsample</samp></d <dd> Set frame subsampling interval to be used. </dd> </dl>

This filter also supports the framesync options. <h4 class=”subsection”>11.149.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> In the examples below, a distorted video <samp class=”file”>distorted.mpg</samp> is compared with a reference file <samp class=”file”>reference.mpg</samp>.

Basic usage: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf=log_path=output.xml -f null - </pre>

Example with multiple models: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='model=version=vmaf_v0.6.1\\:name=vmaf|version=vmaf_v0.6.1neg\\:name=vmaf_neg' -f null - </pre>

Example with multiple additional features: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='feature=name=psnr|name=ciede' -f null - </pre>

Example with options and different containers: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]libvmaf=log_fmt=json:log_path=output.json" -f null - </pre>

 <h3 class=”section”>11.150 libvmaf_cuda# TOC</h3>

This is the CUDA variant of the libvmaf filter. It only accepts CUDA frames. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-nonfree --enable-ffnvcodec --enable-libvmaf</code>. <h4 class=”subsection”>11.150.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Basic usage showing CUVID hardware decoding and CUDA scaling with scale_cuda: <pre class=”example-preformatted”>ffmpeg \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i dis.obu \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i ref.obu \ -filter_complex " [0:v]scale_cuda=format=yuv420p[dis]; \ [1:v]scale_cuda=format=yuv420p[ref]; \ [dis][ref]libvmaf_cuda=log_fmt=json:log_path=output.json " \ -f null - </pre>

 <h3 class=”section”>11.151 limitdiff# TOC</h3> Apply limited difference filter using second and optionally third video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set the

threshold to use when allowing certain differences between video streams. Any absolute difference value lower or exact than this threshold will pick pixel components from first video stream. </dd> <dt><samp class=”option”>elasticity</samp></dt> <dd> Set the elasticity of soft thresholding when processing video streams. This value multiplied with first one sets second threshold. Any absolute difference value greater or exact than second threshold will pick pixel components from second video stream. For values between those two threshold linear interpolation between first and second video stream will be used. </dd> <dt><samp class=”option”>reference</samp></dt> <dd> Enable the reference (third) video stream processing. By default is disabled. If set, this video stream will be used for calculating absolute difference with first video stream. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class=”subsection”>11.151.1 Commands# TOC</h4>

This filter supports the all above options as commands except option ‘<samp class=”samp”>reference</samp>’. <h3 class=”section”>11.152 limiter# TOC</h3>

Limits the pixel components values to the specified range [min, max]. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>min</samp></dt> <dd> Lower bound. Defaults to the lowest representable value for the input. Set to <var class=”var”>auto</var> to use the minimum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class=”option”>max</samp></dt> <dd> Upper bound. Defaults to the highest representable value for the input. Set to <var class=”var”>auto</var> to use the maximum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class=”subsection”>11.152.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.153 loop<span

class=”pull-right”># TOC</h3>

Loop video frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set maximal size in number of frames. Default is 0. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set first frame of loop. Default is 0. </dd> <dt><samp class=”option”>time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class=”var”>start</var> is set to <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>11.153.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Loop single first frame infinitely: <pre class=”example-preformatted”>loop=loop=-1:size=1:start=0 </pre>

Loop single first frame 10 times: <pre class=”example-preformatted”>loop=loop=10:size=1:start=0 </pre>

Loop 10 first frames 5 times: <pre class=”example-preformatted”>loop=loop=5:size=10:start=0 </pre>

 <h3 class=”section”>11.154 lut1d# TOC</h3>

Apply a 1D LUT to an input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> Set the 1D LUT file name. Currently supported formats: <dl class=”table”> <dt>‘<samp class=”samp”>cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class=”samp”>csp</samp>’</dt> <dd> cineSpace </dd> </dl>

</dd> <dt><samp class=”option”>interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Interpolate values using the linear interpolation. </dd> <dt>‘<samp

class=”samp”>cosine</samp>’</dt> <dd> Interpolate values using the cosine interpolation. </dd> <dt>‘<samp class=”samp”>cubic</samp>’</dt> <dd> Interpolate values using the cubic interpolation. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Interpolate values using the spline interpolation. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>11.154.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.155 lut3d# TOC</h3>

Apply a 3D LUT to an input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> Set the 3D LUT file name. Currently supported formats: <dl class=”table”> <dt>‘<samp class=”samp”>3dl</samp>’</dt> <dd> AfterEffects </dd> <dt>‘<samp class=”samp”>cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class=”samp”>dat</samp>’</dt> <dd> DaVinci </dd> <dt>‘<samp class=”samp”>m3d</samp>’</dt> <dd> Pandora </dd> <dt>‘<samp class=”samp”>csp</samp>’</dt> <dd> cineSpace </dd> </dl> </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class=”samp”>trilinear</samp>’</dt> <dd> Interpolate values using the 8 points defining a cube. </dd> <dt>‘<samp class=”samp”>tetrahedral</samp>’</dt> <dd> Interpolate values using a tetrahedron. </dd> <dt>‘<samp class=”samp”>pyramid</samp>’</dt> <dd> Interpolate values using a pyramid. </dd> <dt>‘<samp class=”samp”>prism</samp>’</dt> <dd> Interpolate values using a prism. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>11.155.1 Commands# TOC</h4>

This filter supports the <code class=”code”>interp</code> option as commands. <h3 class=”section”>11.156 lumakey<a class=”anchor hidden-xs” href=”#lumakey” aria-

hidden=”true”># TOC</h3>

Turn certain luma values into transparency. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set the luma which will be used as base for transparency. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>tolerance</samp></dt> <dd> Set the range of luma values to be keyed out. Default value is <code class=”code”>0.01</code>. </dd> <dt><samp class=”option”>softness</samp></dt> <dd> Set the range of softness. Default value is <code class=”code”>0</code>. Use this to control gradual transition from zero to full transparency. </dd> </dl>

 <h4 class=”subsection”>11.156.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>11.157 lut, lutrgb, lutyuv# TOC</h3>

Compute a look-up table for binding each pixel component input value to an output value, and apply it to the input video. <var class=”var”>lutyuv</var> applies a lookup table to a YUV input video, <var class=”var”>lutrgb</var> to an RGB input video. These filters accept the following parameters: <dl class=”table”> <dt><samp class=”option”>c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class=”option”>c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class=”option”>c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class=”option”>c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class=”option”>r</samp></dt> <dd> set red component expression </dd> <dt><samp class=”option”>g</samp></dt> <dd> set green component expression </dd> <dt><samp class=”option”>b</samp></dt> <dd> set blue component expression </dd> <dt><samp class=”option”>a</samp></dt> <dd> alpha component expression </dd> <dt><samp class=”option”>y</samp></dt> <dd> set Y/luma component expression </dd> <dt><samp

class=”option”>u</samp></dt> <dd> set U/Cb component expression </dd> <dt><samp class=”option”>v</samp></dt> <dd> set V/Cr component expression </dd> </dl>

Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in input. The <var class=”var”>lut</var> filter requires either YUV or RGB pixel formats in input, <var class=”var”>lutrgb</var> requires RGB pixel formats in input, and <var class=”var”>lutyuv</var> requires YUV. The expressions can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class=”option”>clipval</samp></dt> <dd> The input value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>maxval</samp></dt> <dd> The maximum value for the pixel component. </dd> <dt><samp class=”option”>minval</samp></dt> <dd> The minimum value for the pixel component. </dd> <dt><samp class=”option”>negval</samp></dt> <dd> The negated value for the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range; it corresponds to the expression "maxval-clipval+minval". </dd> <dt><samp class=”option”>clip(val)</samp></dt> <dd> The computed value in <var class=”var”>val</var>, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>gammaval(gamma)</samp></dt> <dd> The computed gamma correction value of the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. It corresponds to the expression "pow((clipval-minval)/(maxval-minval)\,<var class=”var”>gamma</var>)*(maxval-minval)+minval" </dd> </dl>

All expressions default to "clipval". <h4 class=”subsection”>11.157.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>11.157.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Negate input video: <pre class=”example-preformatted”>lutrgb="r=maxval+minval-val:g=maxval+minval-val:b=maxval+minval-val" lutyuv="y=maxval+minval-

val:u=maxval+minval-val:v=maxval+minval-val" </pre>

The above is the same as: <pre class="example-preformatted">lutrgb="r=negval:g=negval:b=negval" lutyuv="y=negval:u=negval:v=negval" </pre>

Negate luma: <pre class="example-preformatted">lutyuv=y=negval </pre>

Remove chroma components, turning the video into a graytone image: <pre class="example-preformatted">lutyuv="u=128:v=128" </pre>

Apply a luma burning effect: <pre class="example-preformatted">lutyuv="y=2*val" </pre>

Remove green and blue components: <pre class="example-preformatted">lutrgb="g=0:b=0" </pre>

Set a constant alpha channel value on input: <pre class="example-preformatted">format=rgba,lutrgb=a="maxval-minval/2" </pre>

Correct luma gamma by a factor of 0.5: <pre class="example-preformatted">lutyuv=y=gammaval(0.5) </pre>

Discard least significant bits of luma: <pre class="example-preformatted">lutyuv=y='bitand(val, 128+64+32)' </pre>

Technicolor like effect: <pre class="example-preformatted">lutyuv=u='(val-maxval/2)*2+maxval/2':v='(val-maxval/2)*2+maxval/2' </pre>

 <h3 class="section">11.158 lut2, tlut2# TOC</h3>

The <code class="code">lut2</code> filter takes two input streams and outputs one stream. The <code class="code">tlut2</code> (time lut2) filter takes two consecutive frames from one single stream. This filter accepts the following parameters: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class="option">d</samp></dt> <dd> set output bit depth, only available for <code class="code">lut2</code>

filter. By default is 0, which means bit depth is automatically picked from first input format. </dd> </dl>

The <code class=”code”>lut2</code> filter also supports the framesync options. Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in inputs. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dd> The first input value for the pixel component. </dd> <dt><samp class=”option”>y</samp></dt> <dd> The second input value for the pixel component. </dd> <dt><samp class=”option”>bdx</samp></dt> <dd> The first input video bit depth. </dd> <dt><samp class=”option”>bdy</samp></dt> <dd> The second input video bit depth. </dd> </dl>

All expressions default to "x". <h4 class=”subsection”>11.158.1 Commands# TOC</h4>

This filter supports the all above options as commands except option <code class=”code”>d</code>. <h4 class=”subsection”>11.158.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Highlight differences between two RGB video streams: <pre class=”example-preformatted”>lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1)' </pre>

Highlight differences between two YUV video streams: <pre class=”example-preformatted”>lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1)' </pre>

Show max difference between two video streams: <pre class=”example-preformatted”>lut2='if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1)))' </pre>

 <h3 class=”section”>11.159 maskedclamp# TOC</h3>

Clamp the first input stream with the second input and third input stream. Returns the value of first stream to be between second input stream - <code class="code">undershoot</code> and third input stream + <code class="code">overshoot</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">undershoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">overshoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">11.159.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.160 maskedmax# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is greater than first one or from third input stream otherwise. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">11.160.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.161 maskedmerge# TOC</h3>

Merge the first input stream with the second input stream using per pixel weights in the third input stream. A value of 0 in the third stream pixel component means that pixel component from first stream is returned unchanged, while maximum value (eg. 255 for 8-bit videos) means

that pixel component from second stream is returned unchanged. Intermediate values define the amount of merging between both input stream’s pixel components. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">11.161.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.162 maskedmin# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is less than first one or from third input stream otherwise. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">11.162.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.163 maskedthreshold# TOC</h3>
Pick pixels comparing absolute difference of two video streams with fixed threshold. If absolute difference between pixel component of first and second video stream is equal or lower than user supplied threshold than pixel component from first video stream is picked, otherwise pixel component from second video stream is picked. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold used when picking pixels from absolute difference from two input video streams. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set

which planes will be processed as bitmap, unprocessed planes will be copied from second stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of filter operation. Can be <code class="code">abs</code> or <code class="code">diff</code>. Default is <code class="code">abs</code>. </dd> </dl>

 <h4 class="subsection">11.163.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.164 maskfun# TOC</h3> Create mask from input video. For example it is useful to create motion masks after <code class="code">tblend</code> filter. This filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dd> Set low threshold. Any pixel component lower or exact than this value will be set to 0. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high threshold. Any pixel component higher than this value will be set to max value allowed for current pixel format. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all available planes are filtered. </dd> <dt><samp class="option">fill</samp></dt> <dd> Fill all frame pixels with this value. </dd> <dt><samp class="option">sum</samp></dt> <dd> Set max average pixel value for frame. If sum of all pixel components is higher that this average, output frame will be completely filled with value set by <var class="var">fill</var> option. Typically useful for scene changes when used in combination with <code class="code">tblend</code> filter. </dd> </dl>

 <h4 class="subsection">11.164.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.165 mcdeint# TOC</h3>

Apply motion-compensation deinterlacing. It needs one field per frame as input and must thus be used together with yadif=1/3

or equivalent. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set the deinterlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">medium</samp>’</dt> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> use iterative motion estimation </dd> <dt>‘<samp class="samp">extra_slow</samp>’</dt> <dd> like ‘<samp class="samp">slow</samp>’, but use multiple reference frames. </dd> </dl> Default value is ‘<samp class="samp">fast</samp>’. </dd> <dt><samp class="option">parity</samp></dt> <dd> Set the picture field parity assumed for the input video. It must be one of the following values: <dl class="table"> <dt>‘<samp class="samp">0, tff</samp>’</dt> <dd> assume top field first </dd> <dt>‘<samp class="samp">1, bff</samp>’</dt> <dd> assume bottom field first </dd> </dl>

Default value is ‘<samp class="samp">bff</samp>’. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set per-block quantization parameter (QP) used by the internal encoder. Higher values should result in a smoother motion vector field but less optimal individual vectors. Default value is 1. </dd> </dl>

 <h3 class="section">11.166 median# TOC</h3>

Pick median pixel from certain rectangle defined by radius. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set horizontal radius size. Default value is <code class="code">1</code>. Allowed range is integer from 1 to 127. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> <dt><samp class="option">radiusV</samp></dt> <dd> Set vertical radius size. Default value is <code class="code">0</code>. Allowed range is integer from 0 to 127. If it is 0, value will be picked from horizontal <code class="code">radius</code> option. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">11.166.1 Commands# <a class="anchor hidden-xs"href="#toc-

Commands-120" aria-hidden="true">TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">11.167 mergeplanes# TOC</h3>

Merge color channel components from several video streams. The filter accepts up to 4 input streams, and merge selected input planes to the output video. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mapping</samp></dt> <dd> Set input to output plane mapping. Default is <code class="code">0</code>. The mappings is specified as a bitmap. It should be specified as a hexadecimal number in the form 0xAa[Bb[Cc[Dd]]]. ’Aa’ describes the mapping for the first plane of the output stream. ’A’ sets the number of the input stream to use (from 0 to 3), and ’a’ the plane number of the corresponding input to use (from 0 to 3). The rest of the mappings is similar, ’Bb’ describes the mapping for the output stream second plane, ’Cc’ describes the mapping for the output stream third plane and ’Dd’ describes the mapping for the output stream fourth plane. </dd> <dt><samp class="option">format</samp></dt> <dd> Set output pixel format. Default is <code class="code">yuva444p</code>. </dd> <dt><samp class="option">map0s</samp></dt> <dt><samp class="option">map1s</samp></dt> <dt><samp class="option">map2s</samp></dt> <dt><samp class="option">map3s</samp></dt> <dd> Set input to output stream mapping for output Nth plane. Default is <code class="code">0</code>. </dd> <dt><samp class="option">map0p</samp></dt> <dt><samp class="option">map1p</samp></dt> <dt><samp class="option">map2p</samp></dt> <dt><samp class="option">map3p</samp></dt> <dd> Set input to output plane mapping for output Nth plane. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">11.167.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Merge three gray video streams of same width and height into single video stream: <pre class="example-preformatted">[a0][a1][a2]mergeplanes=0x001020:yuv444p
</pre>

Merge 1st yuv444p stream and 2nd gray video stream into yuva444p video stream: <pre class="example-preformatted">[a0][a1]mergeplanes=0x00010210:yuva444p </pre>

Swap Y and A plane in yuva444p stream:
<pre class="example-preformatted">format=yuva444p,mergeplanes=0x03010200:yuva444p
</pre>

Swap U and V plane in yuv420p stream:
<pre class="example-preformatted">format=yuv420p,mergeplanes=0x000201:yuv420p
</pre>

Cast a rgb24 clip to yuv444p: <pre
class="example-preformatted">format=rgb24,mergeplanes=0x000102:yuv444p
</pre>

 <h3 class="section">11.168 mestimate<span
class="pull-right"><a class="anchor hidden-xs" href="#mestimate" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-mestimate"
aria-hidden="true">TOC</h3>

Estimate and export motion vectors using block matching algorithms.
Motion vectors are stored in frame side data to be used by other filters.
 This filter accepts the following options: <dl class="table">
<dt><samp class="option">method</samp></dt> <dd> Specify the
motion estimation method. Accepts one of the following values: <dl
class="table"> <dt>‘<samp class="samp">esa</samp>’</dt>
<dd> Exhaustive search algorithm. </dd> <dt>‘<samp
class="samp">tss</samp>’</dt> <dd> Three step search algo-
rithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt>
<dd> Two dimensional logarithmic search algorithm. </dd>
<dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New
three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt>
<dd> Four step search algorithm. </dd> <dt>‘<samp
class="samp">ds</samp>’</dt> <dd> Diamond search algo-
rithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt>
<dd> Hexagon-based search algorithm. </dd> <dt>‘<samp
class="samp">epzs</samp>’</dt> <dd> Enhanced predictive
zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt>
<dd> Uneven multi-hexagon search algorithm. </dd> </dl>
Default value is ‘<samp class="samp">esa</samp>’.
 </dd> <dt><samp class="option">mb_size</samp></dt>
<dd> Macroblock size. Default <code class="code">16</code>.
 </dd> <dt><samp class="option">search_param</samp></dt>
<dd> Search parameter. Default <code class="code">7</code>.
</dd> </dl>

 <h3 class="section">11.169
mestimate_d3d12<a class="anchor hidden-
xs" href="#mestimate_005fd3d12" aria-hidden="true"># <a
class="anchor hidden-xs"href="#toc-mestimate_005fd3d12" aria-hidden="true">TOC</h3>

Estimate and export motion vectors using D3D12 hardware-accelerated mo-

tion estimation. This filter uses GPU hardware motion estimation capabilities available in DirectX 12 Video APIs to achieve significant performance improvements over the software-based <code class=”code”>mestimate</code> filter. Motion vectors are stored in frame side data to be used by other filters. This filter requires the input to be in <code class=”code”>d3d12</code> hardware pixel format. The filter uses quarter-pixel precision for motion vector estimation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mb_size</samp></dt> <dd> Macroblock size. Only <code class=”code”>8</code> and <code class=”code”>16</code> are supported. Default <code class=”code”>16</code>. </dd> </dl>

 <h4 class=”subsection”>11.169.1 Examples# TOC</h4>

Estimate motion vectors using D3D12 hardware acceleration with 16x16 blocks, and visualize them: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf mestimate_d3d12=mb_size=16,hwdownload,format=nv12,codecview=mv=pf \ -c:v libx264 -preset fast -b:v 5M output.mp4 </pre>

 <h3 class=”section”>11.170 midequalizer# TOC</h3>

Apply Midway Image Equalization effect using two video streams. Midway Image Equalization adjusts a pair of images to have the same histogram, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a pair of stereo cameras. This filter has two inputs and one output, which must be of same pixel format, but may be of different sizes. The output of filter is first input adjusted with midway histogram of both inputs. This filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. Default is <code class=”code”>15</code>, which is all available planes. </dd> </dl>

 <h3 class=”section”>11.171 minterpolate# TOC</h3>

Convert the video to specified frame rate using motion interpolation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frame rate. This can be rational e.g. <code class=”code”>60000/1001</code>.

Frames are dropped if <var class="var">fps</var> is lower than source fps. Default <code class="code">60</code>. </dd> <dt><samp class="option">mi_mode</samp></dt> <dd> Motion interpolation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">dup</samp>’</dt> <dd> Duplicate previous or next frame for interpolating new ones. </dd> <dt>‘<samp class="samp">blend</samp>’</dt> <dd> Blend source frames. Interpolated frame is mean of previous and next frames. </dd> <dt>‘<samp class="samp">mci</samp>’</dt> <dd> Motion compensated interpolation. Following options are effective when this mode is selected: <dl class="table"> <dt>‘<samp class="samp">mc_mode</samp>’</dt> <dd> Motion compensation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">obmc</samp>’</dt> <dd> Overlapped block motion compensation. </dd> <dt>‘<samp class="samp">aobmc</samp>’</dt> <dd> Adaptive overlapped block motion compensation. Window weighting coefficients are controlled adaptively according to the reliabilities of the neighboring motion vectors to reduce oversmoothing. </dd> </dl> Default mode is ‘<samp class="samp">obmc</samp>’. </dd> <dt>‘<samp class="samp">me_mode</samp>’</dt> <dd> Motion estimation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">bidir</samp>’</dt> <dd> Bidirectional motion estimation. Motion vectors are estimated for each source frame in both forward and backward directions. </dd> <dt>‘<samp class="samp">bilat</samp>’</dt> <dd> Bilateral motion estimation. Motion vectors are estimated directly for interpolated frame. </dd> </dl> Default mode is ‘<samp class="samp">bilat</samp>’. </dd> <dt>‘<samp class="samp">me</samp>’</dt> <dd> The algorithm to be used for motion estimation. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class="samp">tss</samp>’</dt> <dd> Three step search algorithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class="samp">ds</samp>’</dt> <dd> Diamond search algorithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt> <dd> Hexagon-based search algorithm. </dd> <dt>‘<samp class="samp">epzs</samp>’</dt> <dd> Enhanced predictive zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt> <dd> Uneven multi-hexagon search algorithm. </dd> </dl> Default algorithm is ‘<samp class="samp">epzs</samp>’. </dd> <dt>‘<samp class="samp">mb_size</samp>’</dt>

<dd> Macroblock size. Default <code class="code">16</code>.
</dd> <dt>‘<samp class="samp">search_param</samp>’</dt>
<dd> Motion estimation search parameter. Default <code class="code">32</code>.
 </dd> <dt>‘<samp class="samp">vsbmc</samp>’</dt>
<dd> Enable variable-size block motion compensation. Motion estimation
is applied with smaller block sizes at object boundaries in order to make them
less blurry. Default is <code class="code">0</code> (disabled). </dd>
</dl> </dd> </dl>

</dd> <dt><samp class="option">scd</samp></dt> <dd> Scene
change detection method. Scene change leads motion vectors to be in
random direction. Scene change detection replace interpolated frames
by duplicate ones. May not be needed for other modes. Following
values are accepted: <dl class="table"> <dt>‘<samp
class="samp">none</samp>’</dt> <dd> Disable scene change
detection. </dd> <dt>‘<samp class="samp">fdiff</samp>’</dt>
<dd> Frame difference. Corresponding pixel values are compared and
if it satisfies <var class="var">scd_threshold</var> scene change is
detected. </dd> </dl> Default method is ‘<samp
class="samp">fdiff</samp>’. </dd> <dt><samp class="option">scd_threshold</samp></
<dd> Scene change detection threshold. Default is <code class="code">10.</code>.
</dd> </dl>

 <h3 class="section">11.172 mix<span class="pull-
right">#
TOC</h3>

Mix several video input streams into one video stream. A
description of the accepted options follows. <dl class="table">
<dt><samp class="option">inputs</samp></dt> <dd> The number
of inputs. If unspecified, it defaults to 2. </dd> <dt><samp
class="option">weights</samp></dt> <dd> Specify weight of each
input video stream as sequence. Each weight is separated by space. If
number of weights is smaller than number of <var class="var">frames</var>
last specified weight will be used for all remaining unset weights.
</dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
scale, if it is set it will be multiplied with sum of each weight multiplied
with pixel values to give final destination pixel value. By default <var
class="var">scale</var> is auto scaled to sum of weights. </dd>
<dt><samp class="option">planes</samp></dt> <dd> Set which
planes to filter. Default is all. Allowed range is from 0 to 15. </dd>
<dt><samp class="option">duration</samp></dt> <dd> Specify how
end of stream is determined. <dl class="table"> <dt>‘<samp
class="samp">longest</samp>’</dt> <dd> The duration of the
longest input. (default) </dd> <dt>‘<samp class="samp">shortest</samp>’</dt>
<dd> The duration of the shortest input. </dd> <dt>‘<samp
class="samp">first</samp>’</dt> <dd> The duration of the

first input. </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.172.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">11.173 monochrome# TOC</h3> Convert video to gray using custom color filter. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">cb</samp></dt> <dd> Set the chroma blue spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">cr</samp></dt> <dd> Set the chroma red spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the color filter size. Allowed range is from .1 to 10. Default value is 1. </dd> <dt><samp class="option">high</samp></dt> <dd> Set the highlights strength. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">11.173.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.174 morpho# TOC</h3>

This filter allows to apply main morphological grayscale transforms, erode and dilate with arbitrary structures set in second input stream. Unlike naive implementation and much slower performance in erosion and dilation filters, when speed is critical <code class="code">morpho</code> filter should be used instead. A description of accepted options follows, <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set morphological transform to apply, can be: <dl class="table"> <dt>‘<samp class="samp">erode</samp>’</dt> <dt>‘<samp class="samp">dilate</samp>’</d <dt>‘<samp class="samp">open</samp>’</dt> <dt>‘<samp

class=”samp”>close</samp>’</dt> <dt>‘<samp class=”samp”>gradient</samp>’<
<dt>‘<samp class=”samp”>tophat</samp>’</dt> <dt>‘<samp
class=”samp”>blackhat</samp>’</dt> </dl> Default is <code
class=”code”>erode</code>. </dd> <dt><samp class=”option”>planes</samp></dt>
<dd> Set planes to filter, by default all planes except alpha are filtered. </dd> <dt><samp class=”option”>structure</samp></dt>
<dd> Set which structure video frames will be processed from second input stream, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> </dl>

The <code class=”code”>morpho</code> filter also supports the framesync options. <h4 class=”subsection”>11.174.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>11.175 mpdecimate# TOC</h3>

Drop frames that do not differ greatly from the previous frame in order to reduce frame rate. The main use of this filter is for very-low-bitrate encoding (e.g. streaming over dialup modem), but it could in theory be used for fixing movies that were inverse-telecined incorrectly. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode. If the value is 0, the filter drops similar frames. If the value is 1, the filter keeps only frames that are one representative of sufficiently long duplicate runs. Default value is 0. </dd> <dt><samp class=”option”>max</samp></dt> <dd> For mode 0: Set the maximum number of consecutive frames which can be dropped (if positive), or the minimum interval between dropped frames (if negative). If the value is 0, the frame is dropped disregarding the number of previous sequentially dropped frames. Default value is 0. </dd> <dt><samp class=”option”>min</samp></dt> <dd> For mode 1: Set the minimum number of frame duplicates to mark the frame as non-unique and keep the first occurrence of it. Default value is 1. </dd> <dt><samp class=”option”>keep</samp></dt> <dd> For mode 0: Set the maximum number of consecutive similar frames to ignore before to start dropping them. If the value is 0, the frame is dropped disregarding the number of previous sequentially similar frames. Default value is 0. </dd> <dt><samp class=”option”>hi</samp></dt> <dt><samp class=”option”>lo</samp></dt> <dt><samp class=”option”>frac</samp></dt>
<dd> Set the dropping threshold values. Values for <samp

class=”option”>hi</samp> and <samp class=”option”>lo</samp> are for 8x8 pixel blocks and represent actual pixel value differences, so a threshold of 64 corresponds to 1 unit of difference for each pixel, or the same spread out differently over the block. A frame is a candidate for dropping if no 8x8 blocks differ by more than a threshold of <samp class=”option”>hi</samp>, and if no more than <samp class=”option”>frac</samp> blocks (1 meaning the whole image) differ by more than a threshold of <samp class=”option”>lo</samp>. Default value for <samp class=”option”>hi</samp> is 64*12, default value for <samp class=”option”>lo</samp> is 64*5, and default value for <samp class=”option”>frac</samp> is 0.33. </dd> </dl>

 <h3 class=”section”>11.176 msad# TOC</h3>

Obtain the MSAD (Mean Sum of Absolute Differences) between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max MSAD is printed through the logging system. The filter stores the calculated MSAD of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi msad -f null - </pre>

 <h3 class=”section”>11.177 multiply# TOC</h3> Multiply first video stream pixels values with second video stream pixels values. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>scale</samp></dt> <dd> Set the scale applied to second video stream. By default is <code class=”code”>1</code>. Allowed range is from <code class=”code”>0</code> to <code class=”code”>9</code>. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set the offset applied to second video stream. By default is <code class=”code”>0.5</code>. Allowed range is from <code class=”code”>-1</code> to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify planes from input video stream that will be processed. By default all planes are processed. </dd> </dl>

 <h4 class=”subsection”>11.177.1 Commands<a class=”anchor hidden-xs” href=”#Commands-

124" aria-hidden="true"># TOC</h4>

This filter supports same commands as options. <h3 class="section">11.178 negate# TOC</h3>

Negate (invert) the input video. It accepts the following option: <dl class="table"> <dt><samp class="option">components</samp></dt> <dd> Set components to negate. Available values for components are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

</dd> <dt><samp class="option">negate_alpha</samp></dt> <dd> With value 1, it negates the alpha component, if present. Default value is 0. </dd> </dl>

 <h4 class="subsection">11.178.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">11.179 nlmeans# TOC</h3>

Denoise frames using Non-Local Means algorithm. Each pixel is adjusted by looking for other pixels with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>x<samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp>x<samp class="option">r</samp> around the pixel. Note that the research area defines centers for patches, which means some patches will be made of pixels outside that research area. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength. Default is 1.0. Must be in range [1.0, 30.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">pc</samp></dt> <dd> Same as <samp class="option">p</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic.

</dd> <dt><samp class=”option”>r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class=”option”>rc</samp></dt> <dd> Same as <samp class=”option”>r</samp> but for chroma planes. The default value is <var class=”var”>0</var> and means automatic. </dd> </dl>

 <h3 class=”section”>11.180 nnedi# TOC</h3>

Deinterlace video using neural network edge directed interpolation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>weights</samp></dt> <dd> Mandatory option, without binary file filter can not work. Currently file can be found here: https://github.com/dubhater/vapoursynth-nnedi3/blob/master/src/nnedi3_weights.bin </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Set which frames to deinterlace, by default it is <code class=”code”>all</code>. Can be <code class=”code”>all</code> or <code class=”code”>interlaced</code>. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Set mode of operation. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>af</samp>’</dt> <dd> Use frame flags, both fields. </dd> <dt>‘<samp class=”samp”>a</samp>’</dt> <dd> Use frame flags, single field. </dd> <dt>‘<samp class=”samp”>t</samp>’</dt> <dd> Use top field only. </dd> <dt>‘<samp class=”samp”>b</samp>’</dt> <dd> Use bottom field only. </dd> <dt>‘<samp class=”samp”>tf</samp>’</dt> <dd> Use both fields, top first. </dd> <dt>‘<samp class=”samp”>bf</samp>’</dt> <dd> Use both fields, bottom first. </dd> </dl>

</dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process, by default filter process all frames. </dd> <dt><samp class=”option”>nsize</samp></dt> <dd> Set size of local neighborhood around each pixel, used by the predictor neural network. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>s8x6</samp>’</dt> <dt>‘<samp class=”samp”>s16x6</samp>’</dt> <dt>‘<samp class=”samp”>s32x6</samp>’</d <dt>‘<samp class=”samp”>s48x6</samp>’</dt> <dt>‘<samp class=”samp”>s8x4</samp>’</dt> <dt>‘<samp class=”samp”>s16x4</samp>’</dt <dt>‘<samp class=”samp”>s32x4</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>nns</samp></dt> <dd> Set the number of neurons in predictor neural network. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>n16</samp>’</dt> <dt>‘<samp class=”samp”>n32</samp>’</dt> <dt>‘<samp class=”samp”>n64</samp>’</dt> <dt>‘<samp class=”samp”>n128</samp>’</dt>

<dt>‘<samp class="samp">n256</samp>’</dt> </dl>

</dd> <dt><samp class="option">qual</samp></dt> <dd> Controls the number of different neural network predictions that are blended together to compute the final output value. Can be <code class="code">fast</code>, default or <code class="code">slow</code>. </dd> <dt><samp class="option">etype</samp></dt> <dd> Set which set of weights to use in the predictor. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">a, abs</samp>’</dt> <dd> weights trained to minimize absolute error </dd> <dt>‘<samp class="samp">s, mse</samp>’</dt> <dd> weights trained to minimize squared error </dd> </dl>

</dd> <dt><samp class="option">pscrn</samp></dt> <dd> Controls whether or not the prescreener neural network is used to decide which pixels should be processed by the predictor neural network and which can be handled by simple cubic interpolation. The prescreener is trained to know whether cubic interpolation will be sufficient for a pixel or whether it should be predicted by the predictor nn. The computational complexity of the prescreener nn is much less than that of the predictor nn. Since most pixels can be handled by cubic interpolation, using the prescreener generally results in much faster processing. The prescreener is pretty accurate, so the difference between using it and not using it is almost always unnoticeable. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">original</samp>’</
<dt>‘<samp class="samp">new</samp>’</dt> <dt>‘<samp class="samp">new2</samp>’</dt> <dt>‘<samp class="samp">new3</samp>’</dt
</dl>

Default is <code class="code">new</code>. </dd> </dl>

 <h4 class="subsection">11.180.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">weights</var> option.
 <h3 class="section">11.181 noformat# TOC</h3>

Force libavfilter not to use any of the specified pixel formats for the input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as pix_fmts=yuv420p|monow|rgb24". </dd> </dl>

 <h4 class="subsection">11.181.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Force libavfilter to use a format different from <var class="var">yuv420p</var> for the input to the vflip filter: <pre class="example-preformatted">noformat=pix_fmts=yuv420p,vflip
</pre>

Convert the input video to any of the formats not contained in the list: <pre class="example-preformatted">noformat=yuv420p|yuv444p|yuv410p
</pre>

 <h3 class="section">11.182 noise#
TOC</h3>

Add noise on video input frame. The filter accepts the following options: <dl class="table"> <dt><samp class="option">all_seed</samp></dt>
<dt><samp class="option">c0_seed</samp></dt> <dt><samp class="option">c1_seed</samp></dt>
<dt><samp class="option">c2_seed</samp></dt> <dt><samp class="option">c3_seed</samp></dt>
<dd> Set noise seed for specific pixel component or all pixel components in case of <var class="var">all_seed</var>. Default value is <code class="code">123457</code>. </dd> <dt><samp class="option">all_strength, alls</samp></dt> <dt><samp class="option">c0_strength, c0s</samp></dt> <dt><samp class="option">c1_strength, c1s</samp></dt>
<dt><samp class="option">c2_strength, c2s</samp></dt> <dt><samp class="option">c3_strength, c3s</samp></dt> <dd> Set noise strength for specific pixel component or all pixel components in case <var class="var">all_strength</var>. Default value is <code class="code">0</code>. Allowed range is [0, 100]. </dd> <dt><samp class="option">all_flags, allf</samp></dt> <dt><samp class="option">c0_flags, c0f</samp></dt>
<dt><samp class="option">c1_flags, c1f</samp></dt> <dt><samp class="option">c2_flags, c2f</samp></dt> <dt><samp class="option">c3_flags, c3f</samp></dt> <dd> Set pixel component flags or set flags for all components if <var class="var">all_flags</var>. Available values for component flags are: <dl class="table"> <dt>‘<samp class="samp">a</samp>’</dt> <dd> averaged temporal noise (smoother) </dd> <dt>‘<samp class="samp">p</samp>’</dt>
<dd> mix random noise with a (semi)regular pattern </dd>
<dt>‘<samp class="samp">t</samp>’</dt> <dd> temporal noise (noise pattern changes between frames) </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> uniform noise (gaussian otherwise) </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.182.1 Examples# <a class="anchor hidden-xs"href="#toc-

Examples-103” aria-hidden=”true”>TOC</h4>

Add temporal and uniform noise to input video: <pre class=”example-preformatted”>noise=alls=20:allf=t+u </pre>

 <h3 class=”section”>11.183 normalize# TOC</h3>

Normalize RGB video (aka histogram stretching, contrast stretching). See: https://en.wikipedia.org/wiki/Normalization_(image_processing) For each channel of each frame, the filter computes the input range and maps it linearly to the user-specified output range. The output range defaults to the full dynamic range from pure black to pure white. Temporal smoothing can be used on the input range to reduce flickering (rapid changes in brightness) caused when small dark or bright objects enter or leave the scene. This is similar to the auto-exposure (automatic gain control) on a video camera, and, like a video camera, it may cause a period of over- or under-exposure of the video. The R,G,B channels can be normalized independently, which may cause some color shifting, or linked together as a single channel, which prevents color shifting. Linked normalization preserves hue. Independent normalization does not, so it can be used to remove some color casts. Independent and linked normalization can be combined in any ratio. The normalize filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>blackpt</samp></dt> <dt><samp class=”option”>whitept</samp></dt> <dd> Colors which define the output range. The minimum input value is mapped to the <var class=”var”>blackpt</var>. The maximum input value is mapped to the <var class=”var”>whitept</var>. The defaults are black and white respectively. Specifying white for <var class=”var”>blackpt</var> and black for <var class=”var”>whitept</var> will give color-inverted, normalized video. Shades of grey can be used to reduce the dynamic range (contrast). Specifying saturated colors here can create some interesting effects. </dd> <dt><samp class=”option”>smoothing</samp></dt> <dd> The number of previous frames to use for temporal smoothing. The input range of each channel is smoothed using a rolling average over the current frame and the <var class=”var”>smoothing</var> previous frames. The default is 0 (no temporal smoothing). </dd> <dt><samp class=”option”>independence</samp></dt> <dd> Controls the ratio of independent (color shifting) channel normalization to linked (color preserving) normalization. 0.0 is fully linked, 1.0 is fully independent. Defaults to 1.0 (fully independent). </dd> <dt><samp class=”option”>strength</samp></dt> <dd> Overall strength of the filter. 1.0 is full strength. 0.0 is a rather expensive no-op. Defaults to 1.0 (full strength). </dd> </dl>

 <h4 class=”subsection”>11.183.1 Com-

mands# TOC</h4> This filter supports same commands as options, excluding <var class="var">smoothing</var> option. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">11.183.2 Examples# TOC</h4>

Stretch video contrast to use the full dynamic range, with no temporal smoothing; may flicker depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=
</pre>

As above, but with 50 frames of temporal smoothing; flicker should be reduced, depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50
</pre>

As above, but with hue-preserving linked channel normalization: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:
</pre>

As above, but with half strength: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50:independence=0:stre
</pre>

Map the darkest input color to red, the brightest input color to cyan: <pre class="example-preformatted">normalize=blackpt=red:whitept=cyan
</pre>

 <h3 class="section">11.184 null# TOC</h3>

Pass the video source unchanged to the output. <h3 class="section">11.185 ocio# TOC</h3> OpenColorIO library filter This filter allows you to do color management using the OpenColorIO library. See https://opencolorio.org/ for more details. To Enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libopencolorio</code>. It accepts the following options: <dl class="table"> <dt><samp class="option">config</samp></dt> <dd> By default the filter will use the OCIO config defined by the OCIO environment variable, but this

parameter allows you to explicitly specify its location. If you are getting started, you can use config=ocio://studio-config-v1.0.0_aces-v1.3_ocio-v2.1 which specifies one of the built in defaults. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input colorspace. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output colorspace. </dd> <dt><samp class=”option”>display</samp></dt> <dd> Set the display colorspace, used in combination with view. </dd> <dt><samp class=”option”>view</samp></dt> <dd> Set the view colorspace, used in combination with display. </dd> <dt><samp class=”option”>inverse</samp></dt> <dd> When used in combination with display and view, this inverts the transform, so going from a display/view to the "input colorspace". </dd> <dt><samp class=”option”>filetransform</samp></dt> <dd> Allows you to specify an external file-transform to use instead of the OCIO config file. This is useful for applying a single transform without needing a full OCIO config file. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Allow you to specify the output pix_fmt of the OCIO filter. This *has* to be a RGB colorspace, so you are limited to rgb24, rgba, rgb48, rgba64, gbrp10, gbrap10, gbrp12, gbrap12, gbrpf32 and gbrapf32; for most encoding we would recommend rgb48. Half-float (gbrpf16, gbrapf16) is accepted as input but is not a valid output format, because swscale has no half-float write support and its legacy half-float read path clamps values to [0,1]. When this option is not set, the output format defaults to the input format, except that half-float input is promoted to the matching float format. Where negotiation cannot yet tell whether the input will be half-float, the output falls back to float, so a float output may be selected even when the input turns out to be an integer format. </dd> <dt><samp class=”option”>context_params</samp></dt> <dd> Allow you to specify additional context parameters for the OCIO filter. This is a list of key=value pairs, separated by colons. </dd> </dl>

 <h4 class=”subsection”>11.185.1 Examples# TOC</h4>

Map from ACEScg to ACEScct, this assumes the OCIO file is defined with the OCIO environment variable. <pre class=”example-preformatted”>input=ACEScg:output=ACEScct:format=rgb48 </pre>

Map from ACEScg to a sRGB display using the "ACES 1.0 - SDR Video" view transform, this assumes the OCIO file is defined with the OCIO environment variable. Note you will need to wrap the argument in quotes to ensure that the spaces are interpreted correctly. <pre class=”example-preformatted”>input=ACEScg:display=sRGB - Display:view=ACES 1.0 -

SDR Video:format=rgb48 </pre>

As above but using the OCIO file studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio rather than the OCIO environment variable. <pre class=”example-preformatted”>config=studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio:input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

If you are converting to YCrCb you still will want to set the color matrix for the conversion. This is a good example of combining the two. <pre class=”example-preformatted”>ffmpeg -y -i SOURCE-FRAMES.%05d.exr -c:v libx265 -vf "ocio=input=ACEScg:output=ACEScct:format=rgb48,scale=in_col
OUTPUTFILE.mov </pre>

 <h3 class=”section”>11.186 ocr# TOC</h3> Optical Character Recognition This filter uses Tesseract for optical character recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libtesseract</code>. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>datapath</samp></dt> <dd> Set datapath to tesseract data. Default is to use whatever was set at installation. </dd> <dt><samp class=”option”>language</samp></dt> <dd> Set language, default is "eng". </dd> <dt><samp class=”option”>whitelist</samp></dt> <dd> Set character whitelist. </dd> <dt><samp class=”option”>blacklist</samp></dt> <dd> Set character blacklist. </dd> </dl>

The filter exports recognized text as the frame metadata <code class=”code”>lavfi.ocr.text</code>. The filter exports confidence of recognized words as the frame metadata <code class=”code”>lavfi.ocr.confidence</code>. <h3 class=”section”>11.187 ocv# TOC</h3>

Apply a video transform using libopencv. To enable this filter, install the libopencv library and headers and configure FFmpeg with <code class=”code”>--enable-libopencv</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the libopencv filter to apply. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> The parameters to pass to the libopencv filter. If not specified, the default values are assumed. </dd> </dl>

Refer to the official libopencv documentation for more precise information: <a class=”url” href=”http://docs.opencv.org/master/modules/imgproc/doc/filtering.html”>http://docs.open
 Several libopencv filters are supported; see the following subsections.

<h4 class="subsection">11.187.1 dilate# TOC</h4>

Dilate an image by using a specific structuring element. It corresponds to the libopencv function <code class="code">cvDilate</code>. It accepts the parameters: <var class="var">struct_el</var>|<var class="var">nb_iterations</var>. <var class="var">struct_el</var> represents a structuring element, and has the syntax: <var class="var">cols</var>x<var class="var">rows</var>+<var class="var">anchor_x</var>x<var class="var">anchor_y</var>/<var class="var">shape</var> <var class="var">cols</var> and <var class="var">rows</var> represent the number of columns and rows of the structuring element, <var class="var">anchor_x</var> and <var class="var">anchor_y</var> the anchor point, and <var class="var">shape</var> the shape for the structuring element. <var class="var">shape</var> must be "rect", "cross", "ellipse", or "custom". If the value for <var class="var">shape</var> is "custom", it must be followed by a string of the form "=<var class="var">filename</var>". The file with name <var class="var">filename</var> is assumed to represent a binary image, with each printable character corresponding to a bright pixel. When a custom <var class="var">shape</var> is used, <var class="var">cols</var> and <var class="var">rows</var> are ignored, the number or columns and rows of the read file are assumed instead. The default value for <var class="var">struct_el</var> is "3x3+0x0/rect". <var class="var">nb_iterations</var> specifies the number of times the transform is applied to the image, and defaults to 1. Some examples: <pre class="example-preformatted"># Use the default values ocv=dilate

Dilate using a structuring element with a 5x5 cross, iterating two times ocv=filter_name=dilate:filter_params=5x5+2x2/cross|2

Read the shape from the file diamond.shape, iterating two times. # The file diamond.shape may contain a pattern of characters like this # * # *** # ***** # *** # * # The specified columns and rows are ignored # but the anchor point coordinates are not ocv=dilate:0x0+2x2/custom=diamond.shape|2 </pre>

 <h4 class="subsection">11.187.2 erode# TOC</h4>

Erode an image by using a specific structuring element. It corresponds to the libopencv function <code class="code">cvErode</code>. It accepts the parameters: <var class="var">struct_el</var>:<var class="var">nb_iterations</var>, with the same syntax and seman-

tics as the dilate filter. <h4 class="subsection">11.187.3 smooth# TOC</h4>

Smooth the input video. The filter takes the following parameters: <var class="var">type</var>|<var class="var">param1</var>|<var class="var">param2</var>|<var class="var">param3</var>|<var class="var">param4</var>. <var class="var">type</var> is the type of smooth filter to apply, and must be one of the following values: "blur", "blur_no_scale", "median", "gaussian", or "bilateral". The default value is "gaussian". The meaning of <var class="var">param1</var>, <var class="var">param2</var>, <var class="var">param3</var>, and <var class="var">param4</var> depends on the smooth type. <var class="var">param1</var> and <var class="var">param2</var> accept integer positive values or 0. <var class="var">param3</var> and <var class="var">param4</var> accept floating point values. The default value for <var class="var">param1</var> is 3. The default value for the other parameters is 0. These parameters correspond to the parameters assigned to the libopencv function <code class="code">cvSmooth</code>. <h3 class="section">11.188 oscilloscope# TOC</h3>

2D Video Oscilloscope. Useful to measure spatial impulse, step responses, chroma delays, etc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set scope center x position. </dd> <dt><samp class="option">y</samp></dt> <dd> Set scope center y position. </dd> <dt><samp class="option">s</samp></dt> <dd> Set scope size, relative to frame diagonal. </dd> <dt><samp class="option">t</samp></dt> <dd> Set scope tilt/rotation. </dd> <dt><samp class="option">o</samp></dt> <dd> Set trace opacity. </dd> <dt><samp class="option">tx</samp></dt> <dd> Set trace center x position. </dd> <dt><samp class="option">ty</samp></dt> <dd> Set trace center y position. </dd> <dt><samp class="option">tw</samp></dt> <dd> Set trace width, relative to width of frame. </dd> <dt><samp class="option">th</samp></dt> <dd> Set trace height, relative to height of frame. </dd> <dt><samp class="option">c</samp></dt> <dd> Set which components to trace. By default it traces first three components. </dd> <dt><samp class="option">g</samp></dt> <dd> Draw trace grid. By default is enabled. </dd> <dt><samp class="option">st</samp></dt> <dd> Draw some statistics. By default

is enabled. </dd> <dt><samp class="option">sc</samp></dt> <dd> Draw scope. By default is enabled. </dd> </dl>

 <h4 class="subsection">11.188.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">11.188.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Inspect full first row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=0:s=1 </pre>

Inspect full last row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=1:s=1 </pre>

Inspect full 5th line of video frame of height 1080. <pre class="example-preformatted">oscilloscope=x=0.5:y=5/1080:s=1 </pre>

Inspect full last column of video frame. <pre class="example-preformatted">oscilloscope=x=1:y=0.5:s=1:t=1 </pre>

 <h3 class="section">11.189 overlay# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: A description of the accepted options follows. <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set the expression for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. In case the expression is invalid, it is set to a huge value (meaning that the overlay will not be displayed within the output visible area). </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for

<samp class="option">x</samp>, and <samp class="option">y</samp> are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">frame</samp>’. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">format</samp></dt> <dd> Set the format for the output video. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">yuv420</samp>’</dt> <dd> force YUV 4:2:0 8-bit planar output </dd> <dt>‘<samp class="samp">yuv420p10</samp>’</dt> <dd> force YUV 4:2:0 10-bit planar output </dd> <dt>‘<samp class="samp">yuv422</samp>’</dt> <dd> force YUV 4:2:2 8-bit planar output </dd> <dt>‘<samp class="samp">yuv422p10</samp>’</dt> <dd> force YUV 4:2:2 10-bit planar output </dd> <dt>‘<samp class="samp">yuv444</samp>’</dt> <dd> force YUV 4:4:4 8-bit planar output </dd> <dt>‘<samp class="samp">yuv444p10</samp>’</dt> <dd> force YUV 4:4:4 10-bit planar output </dd> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> force RGB 8-bit packed output </dd> <dt>‘<samp class="samp">gbrp</samp>’</dt> <dd> force RGB 8-bit planar output </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> automatically pick format </dd> </dl>

Default value is ‘<samp class="samp">yuv420</samp>’. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Set format of alpha of the overlaid video, it can be <var class="var">straight</var> or <var class="var">premultiplied</var>, or <var class="var">auto</var> to choose the alpha mode automatically. Default is <var class="var">auto</var>. </dd> </dl>

The <samp class="option">x</samp>, and <samp class="option">y</samp> expressions can contain the following parameters. <dl class="table"> <dt><samp class="option">main_w, W</samp></dt> <dt><samp class="option">main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class="option">overlay_w, w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The computed values for <var class="var">x</var> and <var class="var">y</var>. They are evaluated for each new frame.

</dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values of the output format. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class="option">pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class="option">t</samp></dt> <dd> The timestamp, expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

This filter also supports the framesync options. Note that the <var class="var">n</var>, <var class="var">t</var> variables are available only when evaluation is done per frame, and will evaluate to NAN when <samp class="option">eval</samp> is set to ‘<samp class="samp">init</samp>’. Be aware that frames are taken from each input video in timestamp order, hence, if their initial timestamps differ, it is a good idea to pass the two inputs through a <var class="var">setpts=PTS-STARTPTS</var> filter to have them begin in the same zero timestamp, as the example for the <var class="var">movie</var> filter does. You can chain together more overlays but you should test the efficiency of such approach. <h4 class="subsection">11.189.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Modify the x and y of the overlay input. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class="subsection">11.189.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the overlay at 10 pixels from the bottom right corner of the main video: <pre class="example-preformatted">overlay=main_w-overlay_w-10:main_h-overlay_h-10 </pre>

Using named options the example above becomes: <pre class="example-preformatted">overlay=x=main_w-overlay_w-10:y=main_h-overlay_h-10 </pre>

Insert a transparent PNG logo in the bottom left corner of the

input, using the <code class="command">ffmpeg</code> tool with the <code class="code">-filter_complex</code> option: <pre class="example-preformatted">ffmpeg -i input -i logo -filter_complex 'overlay=10:main_h-overlay_h-10' output </pre>

Insert 2 different transparent PNG logos (second logo on bottom right corner) using the <code class="command">ffmpeg</code> tool: <pre class="example-preformatted">ffmpeg -i input -i logo1 -i logo2 -filter_complex 'overlay=x=10:y=H-h-10,overlay=x=W-w-10:y=H-h-10' output </pre>

Add a transparent color layer on top of the main video; <code class="code">WxH</code> must specify the size of the main input to the overlay filter: <pre class="example-preformatted">color=color=red@.3:size=WxH [over]; [in][over] overlay [out] </pre>

Play an original video and a filtered version (here with the deshake filter) side by side using the <code class="command">ffplay</code> tool: <pre class="example-preformatted">ffplay input.avi -vf 'split[a][b]; [a]pad=iw*2:ih[src]; [b]deshake[filt]; [src][filt]overlay=w' </pre>

The above command is the same as: <pre class="example-preformatted">ffplay input.avi -vf 'split[b], pad=iw*2[src], [b]deshake, [src]overlay=w' </pre>

Make a sliding overlay appearing from the left to the right top part of the screen starting since time 2: <pre class="example-preformatted">overlay=x='if(gte(t,2), -w+(t-2)*20, NAN)':y=0 </pre>

Compose output by putting two input videos side to side: <pre class="example-preformatted">ffmpeg -i left.avi -i right.avi -filter_complex " nullsrc=size=200x100 [background]; [0:v] setpts=PTS-STARTPTS, scale=100x100 [left]; [1:v] setpts=PTS-STARTPTS, scale=100x100 [right]; [background][left] overlay=shortest=1 [background+left]; [background+left][right] overlay=shortest=1:x=100 [left+right] " </pre>

Mask 10-20 seconds of a video by applying the delogo filter to a section <pre class="example-preformatted">ffmpeg -i test.avi -codec:v:0 wmv2 -ar 11025 -b:v 9000k -vf '[in]split[split_main][split_delogo];[split_delogo]trim=start=360:end=
masked.avi </pre>

Chain several overlays in cascade: <pre class="example-preformatted">nullsrc=s=200x200 [bg]; testsrc=s=100x100, split=4 [in0][in1][in2][in3]; [in0] lutrgb=r=0, [bg] overlay=0:0 [mid0]; [in1] lutrgb=g=0, [mid0] overlay=100:0 [mid1]; [in2] lutrgb=b=0, [mid1] overlay=0:100 [mid2]; [in3] null, [mid2] overlay=100:100 [out0] </pre>

 <h3 class="section">11.190 owdenoise# TOC</h3>

Apply Overcomplete Wavelet denoiser. The filter accepts the following options: <dl class="table"> <dt><samp class="option">depth</samp></dt> <dd> Set depth. Larger depth values will denoise lower frequency components more, but slow down filtering. Must be an int in the range 8-16, default is <code class="code">8</code>. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set luma strength. Must be a double value in the range 0-1000, default is <code class="code">1.0</code>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set chroma strength. Must be a double value in the range 0-1000, default is <code class="code">1.0</code>. </dd> </dl>

 <h3 class="section">11.191 pad# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following parameters: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class="var">width</var> or <var class="var">height</var> is 0, the corresponding input size is used for the output. The <var class="var">width</var> expression can reference the value set by the <var class="var">height</var> expression, and vice versa. The default value of <var class="var">width</var> and <var class="var">height</var> is 0. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section

in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> <dt><samp class="option">eval</samp></dt> <dd> Specify when to evaluate <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var> and <var class="var">y</var> expression. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offsets as specified by the <var class="var">x</var> and <var class="var">y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">11.191.1 Examples# <a class="anchor hidden-xs"href="#toc-

Examples-108" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Add paddings with the color "violet" to the input video. The output video size is 640x480, and the top-left corner of the input video is placed at column 0, row 40 <pre class="example-preformatted">pad=640:480:0:40:violet
</pre>

The example above is equivalent to the following command: <pre class="example-preformatted">pad=width=640:height=480:x=0:y=40:color=violet
</pre>

Pad the input to get an output with dimensions increased by 3/2, and put the input video at the center of the padded area: <pre class="example-preformatted">pad="3/2*iw:3/2*ih:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a squared output with size equal to the maximum value between the input width and height, and put the input video at the center of the padded area: <pre class="example-preformatted">pad="max(iw\,ih):ow:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a final w/h ratio of 16:9: <pre class="example-preformatted">pad="ih*16/9:ih:(ow-iw)/2:(oh-ih)/2" </pre>

In case of anamorphic video, in order to set the output display aspect correctly, it is necessary to use <var class="var">sar</var> in the expression, according to the relation: <pre class="example-preformatted">(ih * X / ih) * sar = output_dar X = output_dar / sar
</pre>

Thus the previous example needs to be modified to: <pre class="example-preformatted">pad="ih*16/9/sar:ih:(ow-iw)/2:(oh-ih)/2" </pre>

Double the output size and put the input video in the bottom-right corner of the output padded area: <pre class="example-preformatted">pad="2*iw:2*ih:ow-iw:oh-ih"
</pre>

 <h3 class="section">11.192 palettegen# TOC</h3>

Generate one palette for a whole video stream. It accepts the following options: <dl class="table"> <dt><samp class="option">max_colors</samp></dt> <dd> Set the maximum number of colors to quantize in the palette. Note: the palette will still contain

256 colors; the unused palette entries will be black. </dd> <dt><samp class="option">reserve_transparent</samp></dt> <dd> Create a palette of 255 colors maximum and reserve the last one for transparency. Reserving the transparency color is useful for GIF optimization. If not set, the maximum of colors in the palette will be 256. You probably want to disable this option for a standalone image. Set by default. </dd> <dt><samp class="option">transparency_color</samp></dt> <dd> Set the color that will be used as background for transparency. </dd> <dt><samp class="option">stats_mode</samp></dt> <dd> Set statistics mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Compute full frame histograms. </dd> <dt>‘<samp class="samp">diff</samp>’</dt> <dd> Compute histograms only for the part that differs from previous frame. This might be relevant to give more importance to the moving part of your input if the background is static. </dd> <dt>‘<samp class="samp">single</samp>’</dt> <dd> Compute new histogram for each frame. </dd> </dl>

Default value is <var class="var">full</var>. </dd> </dl>

The filter also exports the frame metadata <code class="code">lavfi.color_quant_ratio</code> (<code class="code">nb_color_in / nb_color_out</code>) which you can use to evaluate the degree of color quantization of the palette. This information is also visible at <var class="var">info</var> logging level. <h4 class="subsection">11.192.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a representative palette of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -vf palettegen palette.png </pre>

 <h3 class="section">11.193 paletteuse# TOC</h3>

Use a palette to downsample an input video stream. The filter takes two inputs: one video stream and a palette. The palette must be a 256 pixels image. It accepts the following options: <dl class="table"> <dt><samp class="option">dither</samp></dt> <dd> Select dithering mode. Available algorithms are: <dl class="table"> <dt>‘<samp class="samp">bayer</samp>’</dt> <dd> Ordered 8x8 bayer dithering (deterministic) </dd> <dt>‘<samp class="samp">heckbert</samp>’</dt> <dd> Dithering as defined by Paul Heckbert in 1982 (simple error diffusion). Note: this dither-

ing is sometimes considered "wrong" and is included as a reference. </dd> <dt>‘<samp class="samp">floyd_steinberg</samp>’</dt> <dd> Floyd and Steingberg dithering (error diffusion) </dd> <dt>‘<samp class="samp">sierra2</samp>’</dt> <dd> Frankie Sierra dithering v2 (error diffusion) </dd> <dt>‘<samp class="samp">sierra2_4a</samp>’</dt> <dd> Frankie Sierra dithering v2 "Lite" (error diffusion) </dd> <dt>‘<samp class="samp">sierra3</samp>’</dt> <dd> Frankie Sierra dithering v3 (error diffusion) </dd> <dt>‘<samp class="samp">burkes</samp>’</dt> <dd> Burkes dithering (error diffusion) </dd> <dt>‘<samp class="samp">atkinson</samp>’</dt> <dd> Atkinson dithering by Bill Atkinson at Apple Computer (error diffusion) </dd> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable dithering. </dd> </dl>

Default is <var class="var">sierra2_4a</var>. </dd> <dt><samp class="option">bayer_scale</samp></dt> <dd> When <var class="var">bayer</var> dithering is selected, this option defines the scale of the pattern (how much the crosshatch pattern is visible). A low value means more visible pattern for less banding, and higher value means less visible pattern at the cost of more banding. The option must be an integer value in the range [0,5]. Default is <var class="var">2</var>. </dd> <dt><samp class="option">diff_mode</samp></dt> <dd> If set, define the zone to process <dl class="table"> <dt>‘<samp class="samp">rectangle</samp>’</dt> <dd> Only the changing rectangle will be reprocessed. This is similar to GIF cropping/offsetting compression mechanism. This option can be useful for speed if only a part of the image is changing, and has use cases such as limiting the scope of the error diffusal <samp class="option">dither</samp> to the rectangle that bounds the moving scene (it leads to more deterministic output if the scene doesn’t change much, and as a result less moving noise and better GIF compression). </dd> </dl>

Default is <var class="var">none</var>. </dd> <dt><samp class="option">new</samp></dt> <dd> Take new palette for each output frame. </dd> <dt><samp class="option">alpha_threshold</samp></dt> <dd> Sets the alpha threshold for transparency. Alpha values above this threshold will be treated as completely opaque, and values below this threshold will be treated as completely transparent. The option must be an integer value in the range [0,255]. Default is <var class="var">128</var>. </dd> </dl>

 <h4 class="subsection">11.193.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use a palette (generated for example

with palettegen) to encode a GIF using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i input.mkv -i palette.png -lavfi paletteuse output.gif </pre>

 <h3 class=”section”>11.194 perspective# TOC</h3>

Correct perspective of video not recorded perpendicular to the screen. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dt><samp class=”option”>x1</samp></dt> <dt><samp class=”option”>y1</samp></dt> <dt><samp class=”option”>x2</samp></dt> <dt><samp class=”option”>y2</samp></dt> <dt><samp class=”option”>x3</samp></dt> <dt><samp class=”option”>y3</samp></dt> <dd> Set coordinates expression for top left, top right, bottom left and bottom right corners. Default values are <code class=”code”>0:0:W:0:0:H:W:H</code> with which perspective will remain unchanged. If the <code class=”code”>sense</code> option is set to <code class=”code”>source</code>, then the specified points will be sent to the corners of the destination. If the <code class=”code”>sense</code> option is set to <code class=”code”>destination</code>, then the corners of the source will be sent to the specified coordinates. The expressions can use the following variables: <dl class=”table”> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> the width and height of video frame. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> </dl>

</dd> <dt><samp class=”option”>interpolation</samp></dt> <dd> Set interpolation for perspective correction. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>cubic</samp>’</dt> </dl>

Default value is ‘<samp class=”samp”>linear</samp>’. </dd> <dt><samp class=”option”>sense</samp></dt> <dd> Set interpretation of coordinate options. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>0, source</samp>’</dt> <dd> Send point in the source specified by the given coordinates to the corners of the destination. </dd> <dt>‘<samp class=”samp”>1, destination</samp>’</dt> <dd> Send the corners of the source to the point in the destination specified by the given coordinates. Default value is ‘<samp class=”samp”>source</samp>’. </dd> </dl>

</dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when

the expressions for coordinates <samp class="option">x0,y0,...x3,y3</samp> are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> </dl>

 <h3 class="section">11.195 phase# TOC</h3>

Delay interlaced video by one field time so that the field order changes. The intended use is to fix PAL movies that have been captured with the opposite field order to the film-to-video transfer. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set phase mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">t</samp>’</dt> <dd> Capture field order top-first, transfer bottom-first. Filter will delay the bottom field. </dd> <dt>‘<samp class="samp">b</samp>’</dt> <dd> Capture field order bottom-first, transfer top-first. Filter will delay the top field. </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> Capture and transfer with the same field order. This mode only exists for the documentation of the other options to refer to, but if you actually select it, the filter will faithfully do nothing. </dd> <dt>‘<samp class="samp">a</samp>’</dt> <dd> Capture field order determined automatically by field flags, transfer opposite. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp class="samp">b</samp>’ modes on a frame by frame basis using field flags. If no field information is available, then this works just like ‘<samp class="samp">u</samp>’. </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> Capture unknown or varying, transfer opposite. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp class="samp">b</samp>’ on a frame by frame basis by analyzing the images and selecting the alternative that produces best match between the fields. </dd> <dt>‘<samp class="samp">T</samp>’</dt> <dd> Capture top-first, transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp class="samp">p</samp>’ using image analysis. </dd> <dt>‘<samp class="samp">B</samp>’</dt> <dd> Capture bottom-first, transfer unknown or varying. Filter selects among ‘<samp class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’

using image analysis. </dd> <dt>‘<samp class="samp">A</samp>’</dt> <dd> Capture determined by field flags, transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’, ‘<samp class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’ using field flags and image analysis. If no field information is available, then this works just like ‘<samp class="samp">U</samp>’. This is the default mode. </dd> <dt>‘<samp class="samp">U</samp>’</dt> <dd> Both capture and transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’, ‘<samp class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’ using image analysis only. </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.195.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.196 photosensitivity# TOC</h3> Reduce various flashes in video, so to help users with epilepsy. It accepts the following options: <dl class="table"> <dt><samp class="option">frames, f</samp></dt> <dd> Set how many frames to use when filtering. Default is 30. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set detection threshold factor. Default is 1. Lower is stricter. </dd> <dt><samp class="option">skip</samp></dt> <dd> Set how many pixels to skip when sampling frames. Default is 1. Allowed range is from 1 to 1024. </dd> <dt><samp class="option">bypass</samp></dt> <dd> Leave frames unchanged. Default is disabled. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the blending factor for frames that exceed the detection threshold. A value of 0 duplicates the last accepted frame, and a value of 1 blends up to the detection threshold. Default is 1. Allowed range is from 0 to 1. </dd> </dl>

 <h3 class="section">11.197 pixdesctest# TOC</h3>

Pixel format descriptor test filter, mainly useful for internal testing. The output video should be equal to the input video. For example: <pre class="example-preformatted">format=monow, pixdesctest </pre>

can be used to test the monowhite pixel format descriptor definition. <h3 class="section">11.198 pixelize# TOC</h3> Apply pixelization to video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set block dimensions that will be used for pixelization. Default value is <code class="code">16</code>. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set the mode of pixelization used. Possible values are: <dl class="table"> <dt>‘<samp class="samp">avg</samp>’</dt> <dt>‘<samp class="samp">min</samp>’</dt> <dt>‘<samp class="samp">max</samp>’</dt> </dl> Default value is <code class="code">avg</code>. </dd> <dt><samp class="option">planes, p</samp></dt> <dd> Set what planes to filter. Default is to filter all planes. </dd> </dl>

 <h4 class="subsection">11.198.1 Commands# TOC</h4>

This filter supports all options as commands. <h3 class="section">11.199 pixscope# TOC</h3>

Display sample values of color channels. Mainly useful for checking color and levels. Minimum supported resolution is 640x480. The filters accept the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set scope X position, relative offset on X axis. </dd> <dt><samp class="option">y</samp></dt> <dd> Set scope Y position, relative offset on Y axis. </dd> <dt><samp class="option">w</samp></dt> <dd> Set scope width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set scope height. </dd> <dt><samp class="option">o</samp></dt> <dd> Set window opacity. This window also holds statistics about pixel area. </dd> <dt><samp class="option">wx</samp></dt> <dd> Set window X position, relative offset on X axis. </dd> <dt><samp class="option">wy</samp></dt> <dd> Set window Y position, relative offset on Y axis. </dd> </dl>

 <h4 class="subsection">11.199.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>11.200 pp7# TOC</h3> Apply Postprocessing filter 7. It is variant of the spp filter, similar to spp = 6 with 7 point DCT, where only the center sample is used after IDCT. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0 to 63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class=”table”> <dt>‘<samp class=”samp”>hard</samp>’</dt> <dd> Set hard thresholding. </dd> <dt>‘<samp class=”samp”>soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> <dt>‘<samp class=”samp”>medium</samp>’</dt> <dd> Set medium thresholding (good results, default). </dd> </dl> </dd> </dl>

 <h3 class=”section”>11.201 premultiply# TOC</h3> Apply alpha premultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class=”section”>11.202 premultiply_dynamic# TOC</h3> Dynamically premultiply or unpremultiply the input video stream as needed, to match the requirements of the downstream filter graph. This is roughly equivalent to either <code class=”code”>premultiply:inplace=yes</code> or <code class=”code”>unpremultiply:inplace=yes</code>, or otherwise a noop. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> </dl>

 <h3 class="section">11.203 prewitt# TOC</h3> Apply prewitt operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">11.203.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.204 pseudocolor# TOC</h3>

Alter frame colors in video with pseudocolors. This filter accepts the following options: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set pixel first component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set pixel second component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set pixel third component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set pixel fourth component expression, corresponds to the alpha component </dd> <dt><samp class="option">index, i</samp></dt> <dd> set component to use as base for altering colors </dd> <dt><samp class="option">preset, p</samp></dt> <dd> Pick one of built-in LUTs. By default is set to none. Available LUTs: <dl class="table"> <dt>‘<samp class="samp">magma</samp>’</dt> <dt>‘<samp class="samp">inferno</samp>’<
<dt>‘<samp class="samp">plasma</samp>’</dt> <dt>‘<samp class="samp">viridis</samp>’</dt> <dt>‘<samp class="samp">turbo</samp>’</
<dt>‘<samp class="samp">cividis</samp>’</dt> <dt>‘<samp class="samp">range1</samp>’</dt> <dt>‘<samp class="samp">range2</samp>’<
<dt>‘<samp class="samp">shadows</samp>’</dt> <dt>‘<samp class="samp">highlights</samp>’</dt> <dt>‘<samp class="samp">solar</samp>’</dt> <dt>‘<samp class="samp">nominal</samp>’</
<dt>‘<samp class="samp">preferred</samp>’</dt> <dt>‘<samp class="samp">total</samp>’</dt> <dt>‘<samp class="samp">spectral</samp>’</

<dt>‘<samp class="samp">cool</samp>’</dt> <dt>‘<samp class="samp">heat</samp>’</dt> <dt>‘<samp class="samp">fiery</samp>’</dt> <dt>‘<samp class="samp">blues</samp>’</dt> <dt>‘<samp class="samp">green</samp>’</dt> <dt>‘<samp class="samp">helix</samp>’</dt> </dl>

</dd> <dt><samp class="option">opacity</samp></dt> <dd> Set opacity of output colors. Allowed range is from 0 to 1. Default value is set to 1. </dd> </dl>

Each of the expression options specifies the expression to use for computing the lookup table for the corresponding pixel component values. The expressions can contain the following constants and functions: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class="option">ymin, umin, vmin, amin</samp></dt> <dd> The minimum allowed component value. </dd> <dt><samp class="option">ymax, umax, vmax, amax</samp></dt> <dd> The maximum allowed component value. </dd> </dl>

All expressions default to "val". <h4 class="subsection">11.204.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">11.204.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Change too high luma values to gradient: <pre class="example-preformatted">pseudocolor="'if(between(val,ymax,amax),lerp(umin,umax,(val-ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(umax,umin,(val-ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(vmin,vmax,(val-ymax)/(amax-ymax)),-1):-1'" </pre>

 <h3 class="section">11.205 psnr# TOC</h3>

Obtain the average, maximum and minimum PSNR (Peak Signal to Noise Ratio) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the PSNR. Both video inputs must have the

same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average PSNR is printed through the logging system. The filter stores the accumulated MSE (mean squared error) of each frame, and at the end of the processing it is averaged across all frames equally, and the following formula is applied to obtain the PSNR: <pre class="example-preformatted">PSNR = 10*log10(MAX^2/MSE) </pre>

Where MAX is the average of the maximum values of each component of the image. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the PSNR of each individual frame. When filename equals "-" the data is sent to standard output. </dd> <dt><samp class="option">stats_version</samp></dt> <dd> Specifies which version of the stats file format to use. Details of each format are written below. Default value is 1. </dd> <dt><samp class="option">output_max</samp></dt> <dd> Determines whether the max value is output to the stats log. Default value is 0. Requires stats_version >= 2. If this is set and stats_version < 2, the filter will return an error. </dd> </dl>

This filter also supports the framesync options. The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. If a <var class="var">stats_version</var> greater than 1 is specified, a header line precedes the list of per-frame-pair stats, with key value pairs following the frame format with the following parameters: <dl class="table"> <dt><samp class="option">psnr_log_version</samp></dt> <dd> The version of the log file format. Will match <var class="var">stats_version</var>. </dd> <dt><samp class="option">fields</samp></dt> <dd> A comma separated list of the per-frame-pair parameters included in the log. </dd> </dl>

A description of each shown per-frame-pair parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">mse_avg</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames, averaged over all the image components. </dd> <dt><samp class="option">mse_y, mse_u, mse_v, mse_r, mse_g, mse_b, mse_a</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">psnr_y, psnr_u, psnr_v, psnr_r, psnr_g, psnr_b, psnr_a</samp></dt> <dd> Peak Signal to Noise ratio of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">max_avg,

max_y, max_u, max_v</samp></dt> <dd> Maximum allowed value for each channel, and average over all channels. </dd> </dl>

 <h4 class="subsection">11.205.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] psnr="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The PSNR of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]psnr" -f null - </pre>

 <h3 class="section">11.206 pullup# TOC</h3>

Pulldown reversal (inverse telecine) filter, capable of handling mixed hard-telecine, 24000/1001 fps progressive, and 30000/1001 fps progressive content. The pullup filter is designed to take advantage of future context in making its decisions. This filter is stateless in the sense that it does not lock onto a pattern to follow, but it instead looks forward to the following fields in order to identify matches and rebuild progressive frames. To produce content with an even framerate, insert the fps filter after pullup, use <code class="code">fps=24000/1001</code> if the input frame rate is 29.97fps, <code class="code">fps=24</code> for 30fps and the (rare) telecined 25fps input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">jl</samp></dt> <dt><samp class="option">jr</samp></dt> <dt><samp class="option">jt</samp></dt> <dt><samp class="option">jb</samp></dt> <dd> These options set the amount of "junk" to ignore at the left, right, top, and bottom of the image, respectively. Left and right are in units of 8 pixels, while top and bottom are in units of 2 lines. The default is 8 pixels on each side. </dd> <dt><samp class="option">sb</samp></dt> <dd> Set the strict breaks. Setting this option to 1 will reduce the chances of filter generating an occasional mismatched frame, but it may also cause an excessive number of frames to be dropped during high motion sequences. Conversely, setting it to -1 will make filter match fields more easily. This may help processing of video where there is slight blurring between the fields, but may also cause there to be interlaced frames in the output. Default value is <code class="code">0</code>. </dd> <dt><samp

class=”option”>mp</samp></dt> <dd> Set the metric plane to use. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>l</samp>’</dt> <dd> Use luma plane. </dd> <dt>‘<samp class=”samp”>u</samp>’</dt> <dd> Use chroma blue plane. </dd> <dt>‘<samp class=”samp”>v</samp>’</dt> <dd> Use chroma red plane. </dd> </dl>

This option may be set to use chroma plane instead of the default luma plane for doing filter’s computations. This may improve accuracy on very clean source material, but more likely will decrease accuracy, especially if there is chroma noise (rainbow effect) or any grayscale video. The main purpose of setting <samp class=”option”>mp</samp> to a chroma plane is to reduce CPU load and make pullup usable in realtime on slow machines. </dd> </dl>

For best results (without duplicated frames in the output file) it is necessary to change the output frame rate. For example, to inverse telecine NTSC input: <pre class=”example-preformatted”>ffmpeg -i input -vf pullup -r 24000/1001 ... </pre>

 <h3 class=”section”>11.207 qp# TOC</h3>

Change video quantization parameters (QP). The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>qp</samp></dt> <dd> Set expression for quantization parameter. </dd> </dl>

The expression is evaluated through the eval API and can contain, among others, the following constants: <dl class=”table”> <dt><var class=”var”>known</var></dt> <dd> 1 if index is not 129, 0 otherwise. </dd> <dt><var class=”var”>qp</var></dt> <dd> Sequential index starting from -129 to 128. </dd> </dl>

 <h4 class=”subsection”>11.207.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Some equation like: <pre class=”example-preformatted”>qp=2+2*sin(PI*qp) </pre>

 <h3 class=”section”>11.208 qrencode# TOC</h3> Generate a QR code using the libqrencode library (see <a class=”url” href=”https://fukuchi.org/works/qrencode/”>https://fukuchi.o

and overlay it on top of the current frame. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and overlaid into the video output according to the specified options. In case no text is specified, no QR code is overlaied. This filter accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp class="option">padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class="var">qrcode_width</var> expression can reference the value set by the <var class="var">padded_qrcode_width</var> expression, and vice versa. By default <var class="var">padded_qrcode_width</var> is set to <var class="var">qrcode_width</var>, meaning that there is no padding. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify an expression for positioning the padded QR code top-left corner. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice. By default <var class="var">x</var> and <var class="var">y</var> are set set to <var class="var">0</var>, meaning that the QR code is placed in the top left corner of the input. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class="option">case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class="option">level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class="var">L</var>. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">L</samp>’</dt> <dt>‘<samp class="samp">M</samp>’</dt> <dt>‘<samp class="samp">Q</samp>’</dt> <dt>‘<samp class="samp">H</samp>’</dt> </dl>

</dd> <dt><samp class="option">expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class="code">none</code>, or <code class="code">normal</code> (default). See the qrencode Text expansion section below for details. </dd> <dt><samp class="option">text</samp></dt> <dt><samp class="option">textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an

empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section below for details. </dd> <dt><samp class="option">background_color, bc</samp></dt> <dt><samp class="option">foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class="var">foreground_color</var> is "black", the default value of <var class="var">background_color</var> is "white". For the syntax of the color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">11.208.1 qrencode Expressions# TOC</h4>

The expressions set by the options contain the following constants and functions. <dl class="table"> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">duration</samp></dt> <dd> the current frame’s duration, in seconds </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">main_h, H</samp></dt> <dd> the input height </dd> <dt><samp class="option">main_w, W</samp></dt> <dd> the input width </dd> <dt><samp class="option">n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class="option">pict_type</samp></dt> <dd> a number representing the picture type </dd> <dt><samp class="option">qr_w, w</samp></dt> <dd> the width of the encoded QR code </dd> <dt><samp class="option">rendered_qr_w, q</samp></dt> <dt><samp class="option">rendered_padded_qr_w, Q</samp></dt> <dd> the width of the rendered QR code, without and without padding. These parameters allow the <var class="var">q</var> and <var class="var">Q</var> expressions to refer to each other, so you can for example specify <code class="code">q=3/4*Q</code>. </dd> <dt><samp class="option">rand(min, max)</samp></dt> <dd> return a random number included between <var class="var">min</var> and <var class="var">max</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> the input sample aspect ratio </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in sec-

onds, NAN if the input timestamp is unknown </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> the x and y offset coordinates where the text is drawn. These parameters allow the <var class="var">x</var> and <var class="var">y</var> expressions to refer to each other, so you can for example specify <code class="code">y=x/dar</code>. </dd> </dl>

 <h4 class="subsection">11.208.2 qrencode Text expansion# TOC</h4>

If <samp class="option">expansion</samp> is set to <code class="code">none</code>, the text is printed verbatim. If <samp class="option">expansion</samp> is set to <code class="code">normal</code> (which is the default), the following expansion mechanism is used. The backslash character ‘<samp class="samp">\</samp>’, followed by any character, always expands to the second character. Sequences of the form <code class="code">%{...}</code> are expanded. The text between the braces is a function name, possibly followed by arguments separated by ’:’. If the arguments contain special characters or delimiters (’:’ or ’}’), they should be escaped. Note that they probably must also be escaped as the value for the <samp class="option">text</samp> option in the filter argument string and as the filter argument in the filtergraph description, and possibly also for the shell, that makes up to four levels of escaping; using a text file with the <samp class="option">textfile</samp> option avoids these problems. The following functions are available: <dl class="table"> <dt><code class="command">n, frame_num</code></dt> <dd> return the frame number </dd> <dt><code class="command">pts</code></dt> <dd> Return the presentation timestamp of the current frame. It can take up to two arguments. The first argument is the format of the timestamp; it defaults to <code class="code">flt</code> for seconds as a decimal number with microsecond accuracy; <code class="code">hms</code> stands for a formatted <var class="var">[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy. <code class="code">gmtime</code> stands for the timestamp of the frame formatted as UTC time; <code class="code">localtime</code> stands for the timestamp of the frame formatted as local time zone time. If the format is set to <code class="code">hms24hh</code>, the time is formatted in 24h format (00-23). The second argument is an offset added to the timestamp. If the format is set to <code class="code">localtime</code> or <code class="code">gmtime</code>, a third argument may be supplied: a <code class="code">strftime</code> C function format string. By default, <var class="var">YYYY-MM-DD HH:MM:SS</var> format will be used. </dd> <dt><code class="command">expr, e</code></dt>

<dd> Evaluate the expression’s value and output as a double. It must take one argument specifying the expression to be evaluated, accepting the constants and functions defined in qrencode_expressions. </dd> <dt><code class="command">expr_formatted, ef</code></dt> <dd> Evaluate the expression’s value and output as a formatted string. The first argument is the expression to be evaluated, just as for the <var class="var">expr</var> function. The second argument specifies the output format. Allowed values are ‘<samp class="samp">x</samp>’, ‘<samp class="samp">X</samp>’, ‘<samp class="samp">d</samp>’ and ‘<samp class="samp">u</samp>’. They are treated exactly as in the <code class="code">printf</code> function. The third parameter is optional and sets the number of positions taken by the output. It can be used to add padding with zeros from the left. </dd> <dt><code class="command">gmtime</code></dt> <dd> The time at which the filter is running, expressed in UTC. It can accept an argument: a <code class="code">strftime</code> C function format string. The format string is extended to support the variable <var class="var">%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class="command">localtime</code></dt> <dd> The time at which the filter is running, expressed in the local time zone. It can accept an argument: a <code class="code">strftime</code> C function format string. The format string is extended to support the variable <var class="var">%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class="command">metadata</code></dt> <dd> Frame metadata. Takes one or two arguments. The first argument is mandatory and specifies the metadata key. The second argument is optional and specifies a default value, used when the metadata key is not found or empty. Available metadata can be identified by inspecting entries starting with TAG included within each frame section printed by running <code class="code">ffprobe -show_frames</code>. String metadata generated in filters leading to the qrencode filter are also available. </dd> <dt><code class="command">rand(min, max)</code></dt> <dd> return a random number included between <var class="var">min</var> and <var class="var">max</var> </dd> </dl>

 <h4 class="subsection">11.208.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size, overalaid in the top left corner of the input video, with the default size: <pre class="example-

preformatted”>qrencode=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors:
<pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:bc=pink@0.5:fc=blue
</pre>

Place the QR code in the bottom right corner of the input video: <div
class=”example”> <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:x=W-
Q:y=H-Q </pre>

Generate a QR code with width of 200 pixels and padding, making
the padded width 4/3 of the QR code width: <pre
class=”example-preformatted”>qrencode=text=www.ffmpeg.org:q=200:Q=4/3*q
</pre>

Generate a QR code with padded width of 200 pixels and padding,
making the QR code width 3/4 of the padded width:
<pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:Q=200:q=3/4*Q
</pre>

Make the QR code a fraction of the input video width: <div
class=”example”> <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:q=W/5
</pre>

Generate a QR code encoding the frame number:
<pre class=”example-preformatted”>qrencode=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <div
class=”example”> <pre class=”example-preformatted”>qrencode=text=%{gmtime}
</pre>

Generate a QR code encoding the timestamp expressed as a float:
 <pre class=”example-preformatted”>qrencode=text=%{pts}
</pre>

 <h3 class=”section”>11.209 quirc<span class=”pull-
right”>#
TOC</h3>

Identify and decode a QR code using the libquirc library (see <a class=”url”
href=”https://github.com/dlbeer/quirc/”>https://github.com/dlbeer/quirc/),
and print the identified QR codes positions and payload as metadata.
To enable the compilation of this filter, you need to configure FFmpeg with
<code class=”code”>--enable-libquirc</code>. For each found QR
code in the input video, some metadata entries are added with the prefix <var
class=”var”>lavfi.quirc.N</var>, where <var class=”var”>N</var> is the
index, starting from 0, associated to the QR code. A description
of each metadata value follows: <dl class=”table”> <dt><samp
class=”option”>lavfi.quirc.count</samp></dt> <dd> the number of

found QR codes, it is not set in case none was found </dd> <dt><samp class="option">lavfi.quirc.N.corner.M.x</samp></dt> <dt><samp class="option">lavfi.quirc.N.coreer.M.y</samp></dt> <dd> the x/y positions of the four corners of the square containing the QR code, where <var class="var">M</var> is the index of the corner starting from 0 </dd> <dt><samp class="option">lavfi.quirc.N.payload</samp></dt> <dd> the payload of the QR code </dd> </dl>

 <h3 class="section">11.210 random# TOC</h3>

Flush video frames from internal cache of frames into a random order. No frame is discarded. Inspired by frei0r nervous filter. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> Set size in number of frames of internal cache, in range from <code class="code">2</code> to <code class="code">512</code>. Default is <code class="code">30</code>. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed for random number generator, must be an integer included between <code class="code">0</code> and <code class="code">UINT32_MAX</code>. If not specified, or if explicitly set to less than <code class="code">0</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

 <h3 class="section">11.211 readeia608# TOC</h3>

Read closed captioning (EIA-608) information from the top lines of a video frame. This filter adds frame metadata for <code class="code">lavfi.readeia608.X.cc</code> and <code class="code">lavfi.readeia608.X.line</code>, where <code class="code">X</code> is the number of the identified line with EIA-608 data (starting from 0). A description of each metadata value follows: <dl class="table"> <dt><samp class="option">lavfi.readeia608.X.cc</samp></dt> <dd> The two bytes stored as EIA-608 data (printed in hexadecimal). </dd> <dt><samp class="option">lavfi.readeia608.X.line</samp></dt> <dd> The number of the line on which the EIA-608 data was identified and read. </dd> </dl>

This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_min</samp></dt> <dd> Set the line to start scanning for EIA-608 data. Default is <code class="code">0</code>. </dd> <dt><samp class="option">scan_max</samp></dt> <dd> Set the line to end scanning for EIA-608 data. Default is <code class="code">29</code>. </dd> <dt><samp class="option">spw</samp></dt> <dd> Set the ratio of width reserved for sync code detection. Default is

<code class=”code”>0.27</code>. Allowed range is <code class=”code”>[0.1 - 0.7]</code>. </dd> <dt><samp class=”option”>chp</samp></dt> <dd> Enable checking the parity bit. In the event of a parity error, the filter will output <code class=”code”>0x00</code> for that character. Default is false. </dd> <dt><samp class=”option”>lp</samp></dt> <dd> Lowpass lines prior to further processing. Default is enabled. </dd> </dl>

 <h4 class=”subsection”>11.211.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>11.211.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Output a csv with presentation time and the first two lines of identified EIA-608 captioning data. <pre class=”example-preformatted”>ffprobe -f lavfi -i movie=captioned_video.mov,readeia608 -show_entries frame=pts_time:frame_tags=lavfi.readeia608.0.cc,lavf -of csv </pre>

 <h3 class=”section”>11.212 readvitc# TOC</h3>

Read vertical interval timecode (VITC) information from the top lines of a video frame. The filter adds frame metadata key <code class=”code”>lavfi.readvitc.tc_str</code> with the timecode value, if a valid timecode has been detected. Further metadata key <code class=”code”>lavfi.readvitc.found</code> is set to 0/1 depending on whether timecode data has been found or not. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>scan_max</samp></dt> <dd> Set the maximum number of lines to scan for VITC data. If the value is set to <code class=”code”>-1</code> the full video frame is scanned. Default is <code class=”code”>45</code>. </dd> <dt><samp class=”option”>thr_b</samp></dt> <dd> Set the luma threshold for black. Accepts float numbers in the range [0.0,1.0], default value is <code class=”code”>0.2</code>. The value must be equal or less than <code class=”code”>thr_w</code>. </dd> <dt><samp class=”option”>thr_w</samp></dt> <dd> Set the luma threshold for white. Accepts float numbers in the range [0.0,1.0], default value is <code class=”code”>0.6</code>. The value must be equal or greater than <code

class=”code”>thr_b</code>. </dd> </dl>

 <h4 class=”subsection”>11.212.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Detect and draw VITC data onto the video frame; if no valid VITC is detected, draw <code class=”code”>--:--:--:--</code> as a placeholder: <pre class=”example-preformatted”>ffmpeg -i input.avi -filter:v 'readvitc,drawtext=fontfile=FreeMono.ttf:text=%{metadata\\:lavfi.readvitc.tc_str\\:-\\\\\\:--\\\\\\:--\\\\\\:--}:x=(w-tw)/2:y=400-ascent' </pre>

 <h3 class=”section”>11.213 remap# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 16bit depth, single channel. <dl class=”table”> <dt><samp class=”option”>format</samp></dt> <dd> Specify pixel format of output from this filter. Can be <code class=”code”>color</code> or <code class=”code”>gray</code>. Default is <code class=”code”>color</code>. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default color is <code class=”code”>black</code>. </dd> </dl>

 <h3 class=”section”>11.214 removegrain# TOC</h3>

The removegrain filter is a spatial denoiser for progressive video. <dl class=”table”> <dt><samp class=”option”>m0</samp></dt> <dd> Set mode for the first plane. </dd> <dt><samp class=”option”>m1</samp></dt> <dd> Set mode for the second plane. </dd> <dt><samp class=”option”>m2</samp></dt> <dd> Set mode for the third plane. </dd> <dt><samp class=”option”>m3</samp></dt> <dd> Set mode for the fourth plane. </dd> </dl>

Range of mode is from 0 to 24. Description of each mode follows:

<dl class="table">

<dt><var class="var">0</var></dt>

<dd> Leave input plane unchanged. Default.

</dd>

<dt><var class="var">1</var></dt>

<dd> Clips the pixel with the minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">2</var></dt>

<dd> Clips the pixel with the second minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">3</var></dt>

<dd> Clips the pixel with the third minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">4</var></dt>

<dd> Clips the pixel with the fourth minimum and maximum of the 8 neighbour pixels.

This is equivalent to a median filter.

</dd>

<dt><var class="var">5</var></dt>

<dd> Line-sensitive clipping giving the minimal change.

</dd>

<dt><var class=”var”>6</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class=”var”>7</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class=”var”>8</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class=”var”>9</var></dt>

<dd> Line-sensitive clipping on a line where the neighbours pixels are the closest.

</dd>

<dt><var class=”var”>10</var></dt>

<dd> Replaces the target pixel with the closest neighbour.

</dd>

<dt><var class=”var”>11</var></dt>

<dd> [1 2 1] horizontal and vertical kernel blur.

</dd>

<dt><var class=”var”>12</var></dt>

<dd> Same as mode 11.

</dd>

<dt><var class=”var”>13</var></dt>

<dd> Bob mode, interpolates top field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class="var">14</var></dt>

<dd> Bob mode, interpolates bottom field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class="var">15</var></dt>

<dd> Bob mode, interpolates top field. Same as 13 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">16</var></dt>

<dd> Bob mode, interpolates bottom field. Same as 14 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">17</var></dt>

<dd> Clips the pixel with the minimum and maximum of respectively the maximum and

minimum of each pair of opposite neighbour pixels.

</dd>

<dt><var class="var">18</var></dt>

<dd> Line-sensitive clipping using opposite neighbours whose greatest distance from

the current pixel is minimal.

</dd>

<dt><var class="var">19</var></dt>

<dd> Replaces the pixel with the average of its 8 neighbours.

</dd>

<dt><var class="var">20</var></dt>

<dd> Averages the 9 pixels ([1 1 1] horizontal and vertical blur).

</dd>

<dt><var class="var">21</var></dt>

<dd> Clips pixels using the averages of opposite neighbour.

</dd>

<dt><var class="var">22</var></dt>

<dd> Same as mode 21 but simpler and faster.

</dd>

<dt><var class="var">23</var></dt>

<dd> Small edge and halo removal, but reputed useless.

</dd>

<dt><var class="var">24</var></dt>

<dd> Similar as 23.

</dd>

</dl>

 <h3 class="section">11.215 removelogo# TOC</h3>

Suppress a TV station logo, using an image file to determine which pixels comprise the logo. It works by filling in the pixels that comprise the logo with neighboring pixels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt>

<dd> Set the filter bitmap file, which can be any image format supported by libavformat. The width and height of the image file must match those of the video stream being processed. </dd> </dl>

Pixels in the provided bitmap image with a value of zero are not considered part of the logo, non-zero pixels are considered part of the logo. If you use white (255) for the logo and black (0) for the rest, you will be safe. For making the filter bitmap, it is recommended to take a screen capture of a black frame with the logo visible, and then using a threshold filter followed by the erode filter once or twice. If needed, little splotches can be fixed manually. Remember that if logo pixels are not covered, the filter quality will be much reduced. Marking too many pixels as part of the logo does not hurt as much, but it will increase the amount of blurring needed to cover over the image and will destroy more information than necessary, and extra pixels will slow things down on a large logo. <h3 class="section">11.216 repeatfields# TOC</h3>

This filter uses the repeat_field flag from the Video ES headers and hard repeats fields based on its value. <h3 class="section">11.217 reverse# TOC</h3>

Reverse a video clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">11.217.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">trim=end=5,reverse </pre>

 <h3 class="section">11.218 rgbashift# TOC</h3> Shift R/G/B/A pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rh</samp></dt> <dd> Set amount to shift red horizontally. </dd> <dt><samp class="option">rv</samp></dt> <dd> Set amount to shift red vertically. </dd> <dt><samp class="option">gh</samp></dt> <dd> Set amount to shift green horizontally. </dd> <dt><samp class="option">gv</samp></dt> <dd> Set amount to shift green

vertically. </dd> <dt><samp class="option">bh</samp></dt> <dd> Set amount to shift blue horizontally. </dd> <dt><samp class="option">bv</samp></dt> <dd> Set amount to shift blue vertically. </dd> <dt><samp class="option">ah</samp></dt> <dd> Set amount to shift alpha horizontally. </dd> <dt><samp class="option">av</samp></dt> <dd> Set amount to shift alpha vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">11.218.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.219 roberts# TOC</h3> Apply roberts cross operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">11.219.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.220 rotate# TOC</h3>

Rotate video by an arbitrary angle expressed in radians. The filter accepts the following options: A description of the optional parameters follows. <dl class="table"> <dt><samp class="option">angle, a</samp></dt> <dd> Set an expression for the angle by which to rotate the input video clockwise, expressed as a number of radians. A negative value will result in a counter-clockwise rotation. By default it is set to "0". This expression is evaluated for each frame. </dd> <dt><samp class="option">out_w, ow</samp></dt>

<dd> Set the output width expression, default value is "iw". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">out_h, oh</samp></dt> <dd> Set the output height expression, default value is "ih". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">bilinear</samp></dt> <dd> Enable bilinear interpolation if set to 1, a value of 0 disables it. Default value is 1. </dd> <dt><samp class="option">fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the rotated image. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> </dl>

The expressions for the angle and the output size can contain the following constants and functions: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 0. It is always NAN before the first frame is filtered. </dd> <dt><samp class="option">t</samp></dt> <dd> time in seconds of the input frame, it is set to 0 when the filter is configured. It is always NAN before the first frame is filtered. </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">in_w, iw</samp></dt> <dt><samp class="option">in_h, ih</samp></dt> <dd> the input video width and height </dd> <dt><samp class="option">out_w, ow</samp></dt> <dt><samp class="option">out_h, oh</samp></dt> <dd> the output width and height, that is the size of the padded area as specified by the <var class="var">width</var> and <var class="var">height</var> expressions </dd> <dt><samp class="option">rotw(a)</samp></dt> <dt><samp class="option">roth(a)</samp></dt> <dd> the minimal width/height required for completely containing the input video rotated by <var class="var">a</var> radians. These are only available when computing the <samp class="option">out_w</samp> and <samp class="option">out_h</samp> expressions. </dd> </dl>

 <h4 class="subsection">11.220.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Rotate the input by PI/6 radians clockwise: <pre class="example-preformatted">rotate=PI/6 </pre>

Rotate the input by PI/6 radians counter-clockwise: <pre class=”example-preformatted”>rotate=-PI/6 </pre>

Rotate the input by 45 degrees clockwise: <pre class=”example-preformatted”>rotate=45*PI/180 </pre>

Apply a constant rotation with period T, starting from an angle of PI/3: <pre class=”example-preformatted”>rotate=PI/3+2*PI*t/T </pre>

Make the input video rotation oscillating with a period of T seconds and an amplitude of A radians: <pre class=”example-preformatted”>rotate=A*sin(2*PI/T*t) </pre>

Rotate the video, output size is chosen so that the whole rotating input video is always completely contained in the output: <pre class=”example-preformatted”>rotate='2*PI*t:ow=hypot(iw,ih):oh=ow' </pre>

Rotate the video, reduce the output size so that no background is ever shown: <pre class=”example-preformatted”>rotate=2*PI*t:ow='min(iw,ih)/sqrt(2)':oh=ow:c=none </pre>

 <h4 class=”subsection”>11.220.2 Commands# TOC</h4>

The filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>a, angle</samp></dt> <dd> Set the angle expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>11.221 sab# TOC</h3>

Apply Shape Adaptive Blur. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dd> Set luma blur filter strength, must be a value in range 0.1-4.0, default value is 1.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>luma_pre_filter_radius, lpfr</samp></dt> <dd> Set luma pre-filter radius, must be a value in the 0.1-2.0 range, default value is 1.0. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma maximum difference between pixels to still be considered, must be a value in the 0.1-100.0 range, default value is 1.0.

</dd> <dt><samp class="option">chroma_radius, cr</samp></dt> <dd> Set chroma blur filter strength, must be a value in range -0.9-4.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class="option">chroma_pre_filter_radius, cpfr</samp></dt> <dd> Set chroma pre-filter radius, must be a value in the -0.9-2.0 range. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set chroma maximum difference between pixels to still be considered, must be a value in the -0.9-100.0 range. </dd> </dl>

Each chroma option value, if not explicitly specified, is set to the corresponding luma option value. <h3 class="section">11.222 scale# TOC</h3>

Scale (resize) the input video, using the libswscale library. The scale filter forces the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the scale filter will convert the input to the requested format. <h4 class="subsection">11.222.1 Options# TOC</h4> The filter accepts the following options, any of the options supported by the libswscale scaler, as well as any of the framesync options. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#scaler_005foptions">the ffmpeg-scaler manual for the complete list of scaler options. <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class="var">width</var> or <var class="var">w</var> value is 0, the input width is used for the output. If the <var class="var">height</var> or <var class="var">h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the scale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class="option">eval</samp></dt> <dd> Specify when to evaluate <var class="var">width</var> and <var class="var">height</var> expression. It accepts the following values: <dl class="table"> <dt>‘<samp

class=”samp”>init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’.

</dd> <dt><samp class=”option”>interl</samp></dt> <dd> Set the interlacing mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Force interlaced aware scaling. </dd> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Do not apply interlaced scaling. </dd> <dt>‘<samp class=”samp”>-1</samp>’</dt> <dd> Select interlaced aware scaling depending on whether the source frames are flagged as interlaced or not. </dd> </dl>

Default value is ‘<samp class=”samp”>0</samp>’. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Set libswscale scaling flags. See <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#sws_005fflags”>the ffmpeg-scaler manual for the complete list of values. If not explicitly specified the filter applies the default flags.

</dd> <dt><samp class=”option”>param0, param1</samp></dt> <dd> Set libswscale input parameters for scaling algorithms that need them. See <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#sws_005fparams”>the ffmpeg-scaler manual for the complete documentation. If not explicitly specified the filter applies empty parameters. </dd> <dt><samp class=”option”>intent</samp></dt> <dd> Set the ICC rendering intent to use when transforming between different color spaces. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>perceptual</samp>’</dt> <dd> Use a perceptually guided tone and gamut mapping curve. The exact details of the mapping used may change at any time and should not be relied on as stable. This intent is recommended for final viewing of image/video content in typical viewing settings. </dd> <dt>‘<samp class=”samp”>relative_colorimetric</samp>’</dt> <dd> Statically clip out-of-gamut colors using a colorimetric clipping curve which attempts to find the colorimetrically least dissimilar in-gamut color. This intent performs white point adaptation and black point adaptation. This is the default. This intent is recommended wherever faithful color reproduction is of the utmost importance, even at the cost of clipping. </dd> <dt>‘<samp class=”samp”>absolute_colorimetric</samp>’</dt> <dd> Hard clip out-of-gamut colors with no attempt at white or black point reproduction. This intent will reproduce in-gamut colors 1:1 on the output display as they would appear on the reference display, assuming the output display is appropriately calibrated. </dd> <dt>‘<samp

class=”samp”>saturation</samp>’</dt> <dd> Performs saturation mapping - that is, stretches the input color volume directly onto the output color volume, in non-linear fashion that preserves the original signal appearance as much as possible. This intent is recommended for signal content evaluation, as it will not lead to any clipping. It is roughly analogous to not performing any color mapping, although it still takes into account the mastering display primaries and any differences in encoding TRC. </dd> </dl>

</dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>in_color_matrix</samp></dt> <dt><samp class=”option”>out_color_matrix</samp><
<dd> Set in/output YCbCr color space type. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the color space type depends on the pixel format. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> Format conforming to International Telecommunication Union (ITU) Recommendation BT.709. </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> Set color space conforming to the United States Federal Communications Commission (FCC) Code of Federal Regulations (CFR) Title 47 (2003) 73.682 (a). </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> Set color space conforming to: <ul class=”itemize mark-bullet”> ITU Radiocommunication Sector (ITU-R) Recommendation BT.601

ITU-R Rec. BT.470-6 (1998) Systems B, B1, and G

Society of Motion Picture and Television Engineers (SMPTE) ST 170:2004

</dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> Set color space conforming to SMPTE ST 240:1999. </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> Set color space conforming to ITU-R BT.2020 non-constant luminance system. </dd> </dl>

</dd> <dt><samp class=”option”>in_range</samp></dt> <dt><samp class=”option”>out_range</samp></dt> <dd> Set in/output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel for-

mat. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> <dt><samp class="option">in_chroma_loc</samp></dt> <dt><samp class="option">out_chroma_loc</samp></dt> <dd> Set in/output chroma sample location. If <samp class="option">in_chroma_loc</samp> is not specified, the chroma location tagged on the input frame is used, and untagged input is assumed to be sited like the output, so it is tagged but not converted. If <samp class="option">out_chroma_loc</samp> is not specified, the output is converted to the chroma location negotiated with the rest of the filtergraph, or keeps the input location if none was negotiated. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, unknown</samp>’</dt> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt> <dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dt><samp class="option">out_primaries</samp></dt> <dd> Set in/output RGB primaries. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dt>‘<samp class="samp">ebu3213</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_transfer</samp></dt> <dt><samp class="option">out_transfer</samp></dt> <dd> Set in/output transfer response curve (TRC). This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt>

<dt>‘<samp class=”samp”>bt470m</samp>’</dt> <dt>‘<samp class=”samp”>gamma22</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dt>‘<samp class=”samp”>gamma28</samp>&rsqu
<dt>‘<samp class=”samp”>smpte170m</samp>’</dt>
<dt>‘<samp class=”samp”>smpte240m</samp>’</dt>
<dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>iec61966-2-1</samp>’</dt> <dt>‘<samp class=”samp”>srgb</samp>’</dt> <dt>‘<samp class=”samp”>iec61966-2-4</samp>’</dt> <dt>‘<samp class=”samp”>xvycc</samp>’</dt>
<dt>‘<samp class=”samp”>bt1361e</samp>’</dt> <dt>‘<samp class=”samp”>bt2020-10</samp>’</dt> <dt>‘<samp class=”samp”>bt2020-12</samp>’</dt> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dt>‘<samp class=”samp”>arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>force_original_aspect_ratio</samp></dt> <dd> Enable decreasing or increasing output video width or height if necessary to keep the original aspect ratio. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>disable</samp>’</dt> <dd> Scale the video as specified and disable this feature. </dd> <dt>‘<samp class=”samp”>decrease</samp>’</dt> <dd> The output video dimensions will automatically be decreased if needed. </dd> <dt>‘<samp class=”samp”>increase</samp>’</dt> <dd> The output video dimensions will automatically be increased if needed. </dd> </dl>

One useful instance of this option is that when you know a specific device’s maximum allowed resolution, you can use this to limit the output video to that, while retaining the aspect ratio. For example, device A allows 1280x720 playback, and your video is 1920x800. Using this option (set it to decrease) and specifying 1280x720 to the command line makes the output 1280x533. Please note that this is a different thing than specifying -1 for <samp class=”option”>w</samp> or <samp class=”option”>h</samp>, you still need to specify the output resolution for this option to work. </dd> <dt><samp class=”option”>force_divisible_by</samp></dt> <dd> Ensures that both the output dimensions, width and height, are divisible by the given integer when used together with <samp class=”option”>force_original_aspect_ratio</samp>. This works similar to using <code class=”code”>-n</code> in the <samp class=”option”>w</samp> and <samp class=”option”>h</samp> options. This option respects the value set for <samp class=”option”>force_original_aspect_ratio</samp>, increasing or decreasing the resolution accordingly. The video’s aspect ratio may be slightly modified. This option can be handy if you need to have a video fit within or exceed a defined resolution using <samp class=”option”>force_original_aspect_ratio</samp> but also have encoder restrictions on width or height divisibility. </dd> <dt><samp

class=”option”>reset_sar</samp></dt> <dd> Enabling this option leads to the output SAR being reset to 1. Additionally, if the user requests proportional scaling either through the width or height expressions, e.g. <code class=”code”>w=-4:h=360</code> or <code class=”code”>w=iw/2:h=-1</code> or by enabling <code class=”code”>force_original_aspect_ratio</code>, then the input DAR is taken into account and the output is scaled to produce square pixels. Default is false. </dd> </dl>

The values of the <samp class=”option”>w</samp> and <samp class=”option”>h</samp> options are expressions containing the following constants: <dl class=”table”> <dt><var class=”var”>in_w</var></dt> <dt><var class=”var”>in_h</var></dt> <dd> The input width and height </dd> <dt><var class=”var”>iw</var></dt> <dt><var class=”var”>ih</var></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><var class=”var”>out_w</var></dt> <dt><var class=”var”>out_h</var></dt> <dd> The output (scaled) width and height </dd> <dt><var class=”var”>ow</var></dt> <dt><var class=”var”>oh</var></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var> </dd> <dt><var class=”var”>a</var></dt> <dd> The same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><var class=”var”>sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class=”var”>dar</var></dt> <dd> The input display aspect ratio. Calculated from <code class=”code”>(iw / ih) * sar</code>. </dd> <dt><var class=”var”>hsub</var></dt> <dt><var class=”var”>vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><var class=”var”>ohsub</var></dt> <dt><var class=”var”>ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><var class=”var”>n</var></dt> <dd> The (sequential) number of the input frame, starting from 0. Only available with <code class=”code”>eval=frame</code>. </dd> <dt><var class=”var”>t</var></dt> <dd> The presentation timestamp of the input frame, expressed as a number of seconds. Only available with <code class=”code”>eval=frame</code>. </dd> <dt><var class=”var”>pos</var></dt> <dd> The position (byte offset) of the frame in the input stream, or NaN if this information is unavailable and/or meaningless (for example in case of synthetic video). Only available with <code class=”code”>eval=frame</code>. Deprecated, do not use. </dd> <dt><var class=”var”>ref_w, rw</var></dt> <dt><var class=”var”>ref_h, rh</var></dt> <dt><var class=”var”>ref_a</var></dt> <dt><var class=”var”>ref_dar, rdar</var></dt> <dt><var class=”var”>ref_n</var></dt> <dt><var class=”var”>ref_t</var></dt>

<dt><var class="var">ref_pos</var></dt> <dd> Eqvuialent to the above, but for a second reference input. If any of these variables are present, this filter accepts two inputs. </dd> </dl>

 <h4 class="subsection">11.222.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Scale the input video to a size of 200x100 <pre class="example-preformatted">scale=w=200:h=100 </pre>

This is equivalent to: <pre class="example-preformatted">scale=200:100 </pre>

or: <pre class="example-preformatted">scale=200x100 </pre>

Specify a size abbreviation for the output size: <pre class="example-preformatted">scale=qcif </pre>

which can also be written as: <pre class="example-preformatted">scale=size=qcif </pre>

Scale the input to 2x: <pre class="example-preformatted">scale=w=2*iw:h=2*ih </pre>

The above is the same as: <pre class="example-preformatted">scale=2*in_w:2*in_h </pre>

Scale the input to 2x with forced interlaced scaling: <pre class="example-preformatted">scale=2*iw:2*ih:interl=1 </pre>

Scale the input to half size: <pre class="example-preformatted">scale=w=iw/2:h=ih/2 </pre>

Increase the width, and set the height to the same size: <pre class="example-preformatted">scale=3/2*iw:ow </pre>

Seek Greek harmony: <pre class="example-preformatted">scale=iw:1/PHI*iw scale=ih*PHI:ih </pre>

Increase the height, and set the width to 3/2 of the height: <pre class="example-preformatted">scale=w=3/2*oh:h=3/5*ih </pre>

Increase the size, making the size a multiple of the chroma subsample values: <pre class="example-preformatted">scale="trunc(3/2*iw/hsub)*hsub:t
</pre>

Increase the width to a maximum of 500 pixels, keeping the same aspect ratio as the input: <pre class=”example-preformatted”>scale=w='min(500\, iw*3/2):h=-1' </pre>

Make pixels square by combining scale and setsar: <pre class=”example-preformatted”>scale='trunc(ih*dar):ih',setsar=1/1 </pre>

Make pixels square using reset_sar, making sure the resulting resolution is even (required by some codecs): <pre class=”example-preformatted”>scale='-2:ih-mod(ih,2):reset_sar=1' </pre>

Scale to target exactly, however reset SAR to 1: <pre class=”example-preformatted”>scale='400:300:reset_sar=1' </pre>

Scale to even dimensions that fit within 400x300, preserving input SAR: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_rat </pre>

Scale to produce square pixels with even dimensions that fit within 400x300: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_ </pre>

Scale a subtitle stream (sub) to match the main video (main) in size before overlaying. ("scale2ref") <pre class=”example-preformatted”>'[main]split[a][b]; [ref][a]scale=rw:rh[c]; [b][c]overlay' </pre>

Scale a logo to 1/10th the height of a video, while preserving its display aspect ratio. <pre class=”example-preformatted”>[logo-in][video-in]scale=w=oh*dar:h=rh/10[logo-out] </pre>

 <h4 class=”subsection”>11.222.3 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>11.223 scharr# TOC</h3> Apply scharr operator to input video stream. The filter accepts the following option:

<dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">11.223.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.224 scroll# TOC</h3> Scroll input video horizontally and/or vertically by constant speed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class="option">vertical, v</samp></dt> <dd> Set the vertical scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class="option">hpos</samp></dt> <dd> Set the initial horizontal scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> <dt><samp class="option">vpos</samp></dt> <dd> Set the initial vertical scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> </dl>

 <h4 class="subsection">11.224.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. </dd> <dt><samp class="option">vertical, v</samp></dt> <dd> Set the vertical scrolling speed. </dd> </dl>

 <h3 class="section">11.225 scdet# TOC</h3>

Detect video scene change. This filter sets frame metadata with mafd between frame, the scene score, and forward the frame to the next

filter, so they can use these metadata to detect scene change or others. In addition, this filter logs a message and sets frame metadata when it detects a scene change by <samp class="option">threshold</samp>. <code class="code">lavfi.scd.mafd</code> metadata keys are set with mafd for every frame. <code class="code">lavfi.scd.score</code> metadata keys are set with scene change score for every frame to detect scene change. <code class="code">lavfi.scd.time</code> metadata keys are set with current filtered frame time which detect scene change with <samp class="option">threshold</samp>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold, t</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change. Good values are in the <code class="code">[8.0, 14.0]</code> range. The range for <samp class="option">threshold</samp> is <code class="code">[0., 100.]</code>. Default value is <code class="code">10.</code>. </dd> <dt><samp class="option">sc_pass, s</samp></dt> <dd> Set the flag to pass scene change frames to the next filter. Default value is <code class="code">0</code> You can enable it if you want to get snapshot of scene change frames only. </dd> </dl>

 <h3 class="section">11.226 selectivecolor# TOC</h3>

Adjust cyan, magenta, yellow and black (CMYK) to certain ranges of colors (such as "reds", "yellows", "greens", "cyans", ...). The adjustment range is defined by the "purity" of the color (that is, how saturated it already is). This filter is similar to the Adobe Photoshop Selective Color tool. The filter accepts the following options: <dl class="table"> <dt><samp class="option">correction_method</samp></dt> <dd> Select color correction method. Available values are: <dl class="table"> <dt>‘<samp class="samp">absolute</samp>’</dt> <dd> Specified adjustments are applied "as-is" (added/subtracted to original pixel component value). </dd> <dt>‘<samp class="samp">relative</samp>’</dt> <dd> Specified adjustments are relative to the original component value. </dd> </dl> Default is <code class="code">absolute</code>. </dd> <dt><samp class="option">reds</samp></dt> <dd> Adjustments for red pixels (pixels where the red component is the maximum) </dd> <dt><samp class="option">yellows</samp></dt> <dd> Adjustments for yellow pixels (pixels where the blue component is the minimum) </dd> <dt><samp class="option">greens</samp></dt> <dd> Adjustments for green pixels (pixels where the green component is the maximum) </dd> <dt><samp class="option">cyans</samp></dt> <dd> Adjustments for cyan pixels (pixels where the red component is the minimum) </dd> <dt><samp class="option">blues</samp></dt> <dd> Adjustments

for blue pixels (pixels where the blue component is the maximum) </dd> <dt><samp class=”option”>magentas</samp></dt> <dd> Adjustments for magenta pixels (pixels where the green component is the minimum) </dd> <dt><samp class=”option”>whites</samp></dt> <dd> Adjustments for white pixels (pixels where all components are greater than 128) </dd> <dt><samp class=”option”>neutrals</samp></dt> <dd> Adjustments for all pixels except pure black and pure white </dd> <dt><samp class=”option”>blacks</samp></dt> <dd> Adjustments for black pixels (pixels where all components are lesser than 128) </dd> <dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop selective color file (<code class=”code”>.asv</code>) to import the settings from. </dd> </dl>

All the adjustment settings (<samp class=”option”>reds</samp>, <samp class=”option”>yellows</samp>, ...) accept up to 4 space separated floating point adjustment values in the [-1,1] range, respectively to adjust the amount of cyan, magenta, yellow and black for the pixels of its range. <h4 class=”subsection”>11.226.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Increase cyan by 50% and reduce yellow by 33% in every green areas, and increase magenta by 27% in blue areas: <pre class=”example-preformatted”>selectivecolor=greens=.5 0 -.33 0:blues=0 .27 </pre>

Use a Photoshop selective color preset: <pre class=”example-preformatted”>selectivecolor=psfile=MySelectiveColorPresets/Misty.asv </pre>

 <h3 class=”section”>11.227 separatefields# TOC</h3>

The <code class=”code”>separatefields</code> takes a frame-based video input and splits each frame into its components fields, producing a new half height clip with twice the frame rate and twice the frame count. This filter use field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class=”code”>separatefields</code> filter. <h3 class=”section”>11.228 setdar, setsar# TOC</h3>

The <code class=”code”>setdar</code> filter sets the Display Aspect

Ratio for the filter output video. This is done by changing the specified Sample (aka Pixel) Aspect Ratio, according to the following equation: <pre class=”example-preformatted”><var class=”var”>DAR</var> = <var class=”var”>HORIZONTAL_RESOLUTION</var> / <var class=”var”>VERTICAL_RESOLUTION</var> * <var class=”var”>SAR</var> </pre>

Keep in mind that the <code class=”code”>setdar</code> filter does not modify the pixel dimensions of the video frame. Also, the display aspect ratio set by this filter may be changed by later filters in the filterchain, e.g. in case of scaling or if another "setdar" or a "setsar" filter is applied. The <code class=”code”>setsar</code> filter sets the Sample (aka Pixel) Aspect Ratio for the filter output video. Note that as a consequence of the application of this filter, the output display aspect ratio will change according to the equation above. Keep in mind that the sample aspect ratio set by the <code class=”code”>setsar</code> filter may be changed by later filters in the filterchain, e.g. if another "setsar" or a "setdar" filter is applied. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>r, ratio, dar (<code class=”code”>setdar</code> only), sar (<code class=”code”>setsar</code> only)</samp></dt> <dd> Set the aspect ratio used by the filter. The parameter can be a floating point number string, or an expression. If the parameter is not specified, the value "0" is assumed, meaning that the same input value is used. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Set the maximum integer value to use for expressing numerator and denominator when reducing the expressed aspect ratio to a rational. Default value is <code class=”code”>100</code>. </dd> </dl>

The parameter <var class=”var”>sar</var> is an expression containing the following constants: <dl class=”table”> <dt><samp class=”option”>w, h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Same as <var class=”var”>w</var> / <var class=”var”>h</var>. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio. It is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub, vsub</samp></dt> <dd> Horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

 <h4 class=”subsection”>11.228.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> To change the display aspect ratio to 16:9, specify one of the following: <pre class="example-preformatted">setdar=dar=1.77777 setdar=dar=16/9 </pre>

To change the sample aspect ratio to 10:11, specify: <pre class="example-preformatted">setsar=sar=10/11 </pre>

To set a display aspect ratio of 16:9, and specify a maximum integer value of 1000 in the aspect ratio reduction, use the command: <pre class="example-preformatted">setdar=ratio=16/9:max=1000 </pre>

 <h3 class="section">11.229 setfield# TOC</h3>

Force field for the output video frame. The <code class="code">setfield</code> filter marks the interlace type field for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters (e.g. <code class="code">fieldorder</code> or <code class="code">yadif</code>). The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same field property. </dd> <dt>‘<samp class="samp">bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class="samp">tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class="samp">prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl> </dd> </dl>

 <h3 class="section">11.230 setparams# TOC</h3>

Force frame parameter for the output video frame. The <code class="code">setparams</code> filter marks interlace and color range for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by filters/encoders. <dl class="table"> <dt><samp class="option">field_mode</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same field property (default). </dd> <dt>‘<samp class="samp">bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp

class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl>

</dd> <dt><samp class=”option”>range</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color range property (default). </dd> <dt>‘<samp class=”samp”>unspecified, unknown</samp>’</dt> <dd> Mark the frame as unspecified color range. </dd> <dt>‘<samp class=”samp”>limited, tv, mpeg</samp>’</dt> <dd> Mark the frame as limited range. </dd> <dt>‘<samp class=”samp”>full, pc, jpeg</samp>’</dt> <dd> Mark the frame as full range. </dd> </dl>

</dd> <dt><samp class=”option”>color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt470m</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dt>‘<samp class=”samp”>film</samp>’</dt> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dt>‘<samp class=”samp”>jedec-p22</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>color_trc</samp></dt> <dd> Set the color transfer. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color trc property (default). </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt470m</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>log100</samp>’</dt> <dt>‘<samp class=”samp”>log316</samp>’</dt> <dt>‘<samp class=”samp”>iec61966-2-4</samp>’</dt> <dt>‘<samp class=”samp”>bt1361e</samp>’</dt> <dt>‘<samp class=”samp”>iec61966-2-1</samp>’</dt> <dt>‘<samp class=”samp”>bt2020-10</samp>’</dt> <dt>‘<samp class=”samp”>bt2020-12</samp>’</dt> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dt>‘<samp class=”samp”>arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">colorspace</samp></dt> <dd> Set the colorspace. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same colorspace property (default). </dd> <dt>‘<samp class="samp">gbr</samp>’</dt> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">fcc</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">ycgco</samp>’</dt> <dt>‘<samp class="samp">bt2020nc</samp>’</dt> <dt>‘<samp class="samp">bt2020c</samp>’</dt> <dt>‘<samp class="samp">smpte2085</samp>’</dt> <dt>‘<samp class="samp">chroma-derived-nc</samp>’</dt> <dt>‘<samp class="samp">chroma-derived-c</samp>’</dt> <dt>‘<samp class="samp">ictcp</samp>’</dt> </dl>

</dd> <dt><samp class="option">chroma_location</samp></dt> <dd> Set the chroma sample location. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same chroma location (default). </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt> <dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> Set the alpha moda. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same alpha mode (default). </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dt>‘<samp class="samp">premultiplied</samp>’</dt> <dt>‘<samp class="samp">straight</samp>’</dt> </dl> </dd> </dl>

 <h3 class="section">11.231 shear# TOC</h3> Apply shear transform to input video. This filter supports the following options: <dl class="table"> <dt><samp class="option">shx</samp></dt> <dd> Shear factor in X-direction. Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp class="option">shy</samp></dt> <dd> Shear factor in Y-direction. Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp class="option">fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the transformed video. For the general syntax of this option, check the <a data-manual="ffmpeg-utils"

href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> <dt><samp class="option">interp</samp></dt> <dd> Set interpolation type. Can be <code class="code">bilinear</code> or <code class="code">nearest</code>. Default is <code class="code">bilinear</code>. </dd> </dl>

 <h4 class="subsection">11.231.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.232 showinfo# TOC</h3>

Show a line containing various information for each input video frame. The input video is not modified. This filter supports the following options: <dl class="table"> <dt><samp class="option">checksum</samp></dt> <dd> Calculate checksums of each plane. By default enabled. </dd> <dt><samp class="option">udu_sei_as_ascii</samp></dt> <dd> Try to print user data unregistered SEI as ascii character when possible, in hex format otherwise. </dd> </dl>

The shown line contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var>. The following values are shown in the output: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of time base units. The time base unit depends on the filter input pad. </dd> <dt><samp class="option">pts_time</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of seconds. </dd> <dt><samp class="option">fmt</samp></dt> <dd> The pixel format name. </dd> <dt><samp class="option">sar</samp></dt> <dd> The sample aspect ratio of the input frame, expressed in the form <var class="var">num</var>/<var class="var">den</var>. </dd> <dt><samp class="option">s</samp></dt> <dd> The size of the input frame. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">i</samp></dt> <dd> The

type of interlaced mode ("P" for "progressive", "T" for top field first, "B" for bottom field first). </dd> <dt><samp class="option">iskey</samp></dt> <dd> This is 1 if the frame is a key frame, 0 otherwise. </dd> <dt><samp class="option">type</samp></dt> <dd> The picture type of the input frame ("I" for an I-frame, "P" for a P-frame, "B" for a B-frame, or "?" for an unknown type). Also refer to the documentation of the <code class="code">AVPictureType</code> enum and of the <code class="code">av_get_picture_type_char</code> function defined in <samp class="file">libavutil/avutil.h</samp>. </dd> <dt><samp class="option">checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of all the planes of the input frame. </dd> <dt><samp class="option">plane_checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of each plane of the input frame, expressed in the form "[<var class="var">c0</var> <var class="var">c1</var> <var class="var">c2</var> <var class="var">c3</var>]". </dd> <dt><samp class="option">mean</samp></dt> <dd> The mean value of pixels in each plane of the input frame, expressed in the form "[<var class="var">mean0</var> <var class="var">mean1</var> <var class="var">mean2</var> <var class="var">mean3</var>]". </dd> <dt><samp class="option">stdev</samp></dt> <dd> The standard deviation of pixel values in each plane of the input frame, expressed in the form "[<var class="var">stdev0</var> <var class="var">stdev1</var> <var class="var">stdev2</var> <var class="var">stdev3</var>]". </dd> </dl>

 <h3 class="section">11.233 showpalette# TOC</h3>

Displays the 256 colors palette of each frame. This filter is only relevant for <var class="var">pal8</var> pixel format frames. It accepts the following option: <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set the size of the box used to represent one palette color entry. Default is <code class="code">30</code> (for a <code class="code">30x30</code> pixel box). </dd> </dl>

 <h3 class="section">11.234 shuffleframes# TOC</h3>

Reorder and/or duplicate and/or drop video frames. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mapping</samp></dt> <dd> Set the destination indexes of input frames. This is space or ’|’ separated list of indexes that maps input frames to output frames. Number of indexes also sets

maximal value that each index may have. ’-1’ index have special meaning and that is to drop frame. </dd> </dl>

The first frame has the index 0. The default is to keep the input unchanged. <h4 class="subsection">11.234.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap second and third frame of every three frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=0 2 1" OUTPUT </pre>

Swap 10th and 1st frame of every ten frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=9 1 2 3 4 5 6 7 8 0" OUTPUT </pre>

 <h3 class="section">11.235 shufflepixels# TOC</h3>

Reorder pixels in video frames. This filter accepts the following options: <dl class="table"> <dt><samp class="option">direction, d</samp></dt> <dd> Set shuffle direction. Can be forward or inverse direction. Default direction is forward. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set shuffle mode. Can be horizontal, vertical or block mode. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set shuffle block_size. In case of horizontal shuffle mode only width part of size is used, and in case of vertical shuffle mode only height part of size is used. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set random seed used with shuffling pixels. Mainly useful to set to be able to reverse filtering process to get original input. For example, to reverse forward shuffle you need to use same parameters and exact same seed and to set direction to inverse. </dd> </dl>

 <h3 class="section">11.236 shuffleplanes# TOC</h3>

Reorder and/or duplicate video planes. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map0</samp></dt> <dd> The index of the input plane to be used as the first output plane. </dd> <dt><samp class="option">map1</samp></dt> <dd> The index of the input plane to be used as the second output

plane. </dd> <dt><samp class="option">map2</samp></dt> <dd> The index of the input plane to be used as the third output plane. </dd> <dt><samp class="option">map3</samp></dt> <dd> The index of the input plane to be used as the fourth output plane. </dd> </dl>

The first plane has the index 0. The default is to keep the input unchanged. <h4 class="subsection">11.236.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap the second and third planes of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf shuffleplanes=0:2:1:3 OUTPUT </pre>

<h3 class="section">11.237 signalstats# TOC</h3>

Evaluate various visual metrics that assist in determining issues associated

with the digitization of analog video media.

By default the filter will log these metadata values:

<dl class="table">

<dt><samp class="option">YMIN</samp></dt>

<dd> Display the minimal Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YLOW</samp></dt>

<dd> Display the Y value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>YAVG</samp></dt>

<dd> Display the average Y value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>YHIGH</samp></dt>

<dd> Display the Y value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>YMAX</samp></dt>

<dd> Display the maximum Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>UMIN</samp></dt>

<dd> Display the minimal U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>ULOW</samp></dt>

<dd> Display the U value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>UAVG</samp></dt>

<dd> Display the average U value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class="option">UHIGH</samp></dt>

<dd> Display the U value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UMAX</samp></dt>

<dd> Display the maximum U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VMIN</samp></dt>

<dd> Display the minimal V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VLOW</samp></dt>

<dd> Display the V value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VAVG</samp></dt>

<dd> Display the average V value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>VHIGH</samp></dt>

<dd> Display the V value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VMAX</samp></dt>

<dd> Display the maximum V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>SATMIN</samp></dt>

<dd> Display the minimal saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATLOW</samp></dt>

<dd> Display the saturation value at the 10% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATAVG</samp></dt>

<dd> Display the average saturation value within the input frame. Expressed in range

of [0-~181.02].

</dd>

<dt><samp class=”option”>SATHIGH</samp></dt>

<dd> Display the saturation value at the 90% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATMAX</samp></dt>

<dd> Display the maximum saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>HUEMED</samp></dt>

<dd> Display the median value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>HUEAVG</samp></dt>

<dd> Display the average value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>YDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the Y

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>UDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the U

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>VDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the V

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>YBITDEPTH</samp></dt>

<dd> Display bit depth of Y plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class=”option”>UBITDEPTH</samp></dt>

<dd> Display bit depth of U plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class=”option”>VBITDEPTH</samp></dt>

<dd> Display bit depth of V plane in current frame.

Expressed in range of [0-16].

</dd>

</dl>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stat</samp></dt> <dt><samp class=”option”>out</samp></dt> <dd> <samp class=”option”>stat</samp> specify an additional form of image analysis. <samp class=”option”>out</samp> output video with the specified type of pixel highlighted. Both options accept the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tout</samp>’</dt> <dd> Identify <var

class=”var”>temporal outliers</var> pixels. A <var class=”var”>temporal outlier</var> is a pixel unlike the neighboring pixels of the same field. Examples of temporal outliers include the results of video dropouts, head clogs, or tape tracking issues. </dd> <dt>‘<samp class=”samp”>vrep</samp>’</dt> <dd> Identify <var class=”var”>vertical line repetition</var>. Vertical line repetition includes similar rows of pixels within a frame. In born-digital video vertical line repetition is common, but this pattern is uncommon in video digitized from an analog source. When it occurs in video that results from the digitization of an analog source it can indicate concealment from a dropout compensator. </dd> <dt>‘<samp class=”samp”>brng</samp>’</dt> <dd> Identify pixels that fall outside of legal broadcast range. </dd> </dl>

</dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Set the highlight color for the <samp class=”option”>out</samp> option. The default color is yellow. </dd> </dl>

 <h4 class=”subsection”>11.237.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Output data of various video metrics: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats="stat=tout+vrep+brng" -show_frames </pre>

Output specific data about the minimum and maximum values of the Y plane per frame: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats -show_entries frame_tags=lavfi.signalstats.YMAX,lavfi.signalstats.YMIN </pre>

Playback video while highlighting pixels that are outside of broadcast range in red. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats="out=brng:color=red" </pre>

Playback video with signalstats metadata drawn over the frame. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats=stat=brng+vrep+tout,drawtext=fontfile=FreeSerif.ttf:textfile=signalstat_drawtext.txt </pre>

The contents of signalstat_drawtext.txt used in the command are: <pre class=”example-preformatted”>time %{pts:hms} Y (%{metadata:lavfi.signalstats.YMIN}-%{metadata:lavfi.signalstats.YMAX})
U (%{metadata:lavfi.signalstats.UMIN}-%{metadata:lavfi.signalstats.UMAX})
V (%{metadata:lavfi.signalstats.VMIN}-%{metadata:lavfi.signalstats.VMAX})
saturation maximum: %{metadata:lavfi.signalstats.SATMAX}

</pre>

 <h3 class=”section”>11.238 signature# TOC</h3>

Calculates the MPEG-7 Video Signature. The filter can handle more than one input. In this case the matching between the inputs can be calculated additionally. The filter always passes through the first input. The signature of each stream can be written into a file. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>detectmode</samp></dt> <dd> Enable or disable the matching process. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>off</samp>’</dt> <dd> Disable the calculation of a matching (default). </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Calculate the matching for the whole video and output whether the whole video matches or only parts. </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dd> Calculate only until a matching is found or the video ends. Should be faster in some cases. </dd> </dl>

</dd> <dt><samp class=”option”>nb_inputs</samp></dt> <dd> Set the number of inputs. The option value must be a non negative integer. Default value is 1. </dd> <dt><samp class=”option”>filename</samp></dt> <dd> Set the path to which the output is written. If there is more than one input, the path must be a prototype, i.e. must contain %d or %0nd (where n is a positive integer), that will be replaced with the input number. If no filename is specified, no output will be written. This is the default. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Choose the output format. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>binary</samp>’</dt> <dd> Use the specified binary representation (default). </dd> <dt>‘<samp class=”samp”>xml</samp>’</dt> <dd> Use the specified xml representation. </dd> </dl>

</dd> <dt><samp class=”option”>th_d</samp></dt> <dd> Set threshold to detect one word as similar. The option value must be an integer greater than zero. The default value is 9000. </dd> <dt><samp class=”option”>th_dc</samp></dt> <dd> Set threshold to detect all words as similar. The option value must be an integer greater than zero. The default value is 60000. </dd> <dt><samp class=”option”>th_xh</samp></dt> <dd> Set threshold to detect frames as similar. The option value must be an integer greater than zero. The default value is 116. </dd> <dt><samp class=”option”>th_di</samp></dt> <dd> Set the minimum length of a sequence in frames to recognize it as matching sequence. The option value must be a non negative integer value. The default value is 0.

</dd> <dt><samp class="option">th_it</samp></dt> <dd> Set the minimum relation, that matching frames to all frames must have. The option value must be a double value between 0 and 1. The default value is 0.5. </dd> </dl>

 <h4 class="subsection">11.238.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> To calculate the signature of an input video and store it in signature.bin: <pre class="example-preformatted">ffmpeg -i input.mkv -vf signature=filename=signature.bin -map 0:v -f null - </pre>

To detect whether two videos match and store the signatures in XML format in signature0.xml and signature1.xml: <pre class="example-preformatted">ffmpeg -i input1.mkv -i input2.mkv -filter_complex "[0:v][1:v] signature=nb_inputs=2:detectmode=full:format=xml:filename=signature%d.x -map :v -f null - </pre>

 <h3 class="section">11.239 siti# TOC</h3>

Calculate Spatial Information (SI) and Temporal Information (TI) scores for a video, as defined in ITU-T Rec. P.910 (11/21): Subjective video quality assessment methods for multimedia applications. Available PDF at https://www.itu.int/rec/T-REC-P.910-202111-S/en. Note that this is a legacy implementation that corresponds to a superseded recommendation. Refer to ITU-T Rec. P.910 (07/22) for the latest version: https://www.itu.int/rec/T-REC-P.910-202207-I/en It accepts the following option: <dl class="table"> <dt><samp class="option">print_summary</samp></dt> <dd> If set to 1, Summary statistics will be printed to the console. Default 0. </dd> </dl>

 <h4 class="subsection">11.239.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> To calculate SI/TI metrics and print summary: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf siti=print_summary=1 -f null - </pre>

 <h3 class="section">11.240 smartblur# TOC</h3>

Blur the input video without impacting the outlines. It accepts the following options: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dd> Set the luma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is 1.0. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set the luma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is 1.0. </dd> <dt><samp class="option">luma_threshold, lt</samp></dt> <dd> Set the luma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is 0. </dd> <dt><samp class="option">chroma_radius, cr</samp></dt> <dd> Set the chroma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set the chroma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">chroma_threshold, ct</samp></dt> <dd> Set the chroma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set the alpha radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">alpha_strength, as</samp></dt> <dd> Set the alpha strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">alpha_threshold, at</samp></dt> <dd> Set the alpha threshold used as a coefficient to determine whether

a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> </dl>

If a chroma or alpha option is not explicitly set, the corresponding luma value is set. <h3 class="section">11.241 sobel# TOC</h3> Apply sobel operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">11.241.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">11.242 spp# TOC</h3>

Apply a simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">6</code> - all) shifts and average the results. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-6. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">6</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding (default). </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier).

</dd> </dl>

</dd> <dt><samp class="option">use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class="code">1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class="code">0</code> (not enabled). </dd> </dl>

 <h4 class="subsection">11.242.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">quality, level</samp></dt> <dd> Set quality level. The value <code class="code">max</code> can be used to set the maximum level, currently <code class="code">6</code>. </dd> </dl>

 <h3 class="section">11.243 sr# TOC</h3>

Scale the input by applying one of the super-resolution methods based on convolutional neural networks. Supported models: <ul class="itemize mark-bullet"> Super-Resolution Convolutional Neural Network model (SRCNN). See https://arxiv.org/abs/1501.00092.

Efficient Sub-Pixel Convolutional Neural Network model (ESPCN). See https://arxiv.org/abs/1609.05158.

Training scripts as well as scripts for model file (.pb) saving can be found at <a class="url" href="https://github.com/XueweiMeng/sr/tree/sr_dnn_native">https://github.com/Xuewei
Original repository is at <a class="url" href="https://github.com/HighVoltageRocknRoll/sr.git">https://gith
 The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class="url" href="https://www.tensorflow.org/install/lang_c">https://www.tensorflow.org/install/lang_c
and configure FFmpeg with <code class="code">--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO backend can load files for only its format. </dd> <dt><samp

class=”option”>scale_factor</samp></dt> <dd> Set scale factor for SRCNN model. Allowed values are <code class=”code”>2</code>, <code class=”code”>3</code> and <code class=”code”>4</code>. Default value is <code class=”code”>2</code>. Scale factor is necessary for SRCNN model, because it accepts input upscaled using bicubic upscaling with proper scale factor. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>11.244 sr_amf# TOC</h3>

Upscale (size increasing) for the input video using AMD Advanced Media Framework library for hardware acceleration. Use advanced algorithms for upscaling with higher output quality. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>algorithm</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> <dt><var class=”var”>sr1-0</var></dt> <dd> Video SR1.0 This is a default value </dd> <dt><var class=”var”>point</var></dt> <dd> Point AMF returns an unwritten surface for this algorithm when the format is <code class=”code”>nv12</code> or <code class=”code”>p010</code>, so inputs in those formats are converted to a packed RGB format automatically. This is a driver bug rather than a documented restriction; the conversion can be dropped once a driver that scales <code class=”code”>nv12</code> and <code class=”code”>p010</code> correctly with <code class=”code”>point</code> is in wide use. </dd> <dt><var class=”var”>sr1-1</var></dt> <dd> Video SR1.1 This algorithm only supports packed RGB formats and requires a DirectX 11 or DirectX 12 device, so it is available on Windows only. Inputs in other formats such as <code class=”code”>nv12</code> or <code class=”code”>p010</code> are converted to a packed RGB format automatically. Note that the VideoSR algorithms (<code class=”code”>sr1-0</code> and <code class=”code”>sr1-1</code>) do not preserve the alpha channel (it is left zeroed); drop it after the download, e.g. with <code class=”code”>hwdownload,format=rgba,format=rgb24</code>, if an opaque result is required. </dd> </dl>

</dd> <dt><samp class=”option”>sharpness</samp></dt> <dd> Control hq scaler sharpening. The value is a float in the range of [0.0, 2.0] </dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. The <code class=”code”>point</code> and <code class=”code”>sr1-1</code> algorithms emit the packed RGB format the scaler receives, so for them this option must name one of <code class=”code”>rgba</code>, <code class=”code”>bgra</code>, <code class=”code”>x2bgr10le</code> or <code class=”code”>rgbaf16le</code>. A named format also decides what software input is converted to, and what a hardware surface in another format is converted to on the GPU. Without it, <code class=”code”>x2bgr10le</code> is selected for <code class=”code”>p010</code> input and <code class=”code”>rgba</code> for other non-RGB input, and the result is tagged as full range RGB. libswscale cannot convert other software formats to <code class=”code”>rgbaf16le</code>; use software input already in that format or a hardware input for GPU conversion. </dd> <dt><samp class=”option”>keep-ratio</samp></dt> <dd> Force the scaler to keep the aspect ratio of the input image when the output size has a different aspect ratio. Default value is false. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Specifies whether the output image outside the region of interest, which does not fill the entire output surface should be filled with a solid color. </dd> </dl>

 <h4 class=”subsection”>11.244.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Upscale to 720p, keeping aspect ratio, and read the result back into system memory. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class=”example-preformatted”>sr_amf=-2:720,hwdownload,format=nv12
</pre>

Upscale to 4K with algorithm video SR1.1. <pre class=”example-preformatted”>sr_amf=4096:2160:algorithm=sr1-1
</pre>

 <h3 class=”section”>11.245 ssim# TOC</h3>

Obtain the SSIM (Structural SImilarity Metric) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the SSIM. Both video inputs must have the same resolution and pixel

format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The filter stores the calculated SSIM of each frame. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the SSIM of each individual frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. A description of each shown parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">Y, U, V, R, G, B</samp></dt> <dd> SSIM of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">All</samp></dt> <dd> SSIM of the compared frames for the whole frame. </dd> <dt><samp class="option">dB</samp></dt> <dd> Same as above but in dB representation. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">11.245.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] ssim="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The SSIM of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with both psnr and ssim at same time: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi "ssim;[0:v][1:v]psnr" -f null - </pre>

Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]ssim" -f null - </pre>

 <h3 class="section">11.246 stereo3d# TOC</h3>

Convert between different stereoscopic image formats. The

filters accept the following options: <dl class="table"> <dt><samp class="option">in</samp></dt> <dd> Set stereoscopic image format of input. Available values for input image formats are: <dl class="table"> <dt>‘<samp class="samp">sbsl</samp>’</dt> <dd> side by side parallel (left eye left, right eye right) </dd> <dt>‘<samp class="samp">sbsr</samp>’</dt> <dd> side by side crosseye (right eye left, left eye right) </dd> <dt>‘<samp class="samp">sbs2l</samp>’</dt> <dd> side by side parallel with half width resolution (left eye left, right eye right) </dd> <dt>‘<samp class="samp">sbs2r</samp>’</dt> <dd> side by side crosseye with half width resolution (right eye left, left eye right) </dd> <dt>‘<samp class="samp">abl</samp>’</dt> <dt>‘<samp class="samp">tbl</samp>’</dt> <dd> above-below (left eye above, right eye below) </dd> <dt>‘<samp class="samp">abr</samp>’</dt> <dt>‘<samp class="samp">tbr</samp>’</dt> <dd> above-below (right eye above, left eye below) </dd> <dt>‘<samp class="samp">ab2l</samp>’</dt> <dt>‘<samp class="samp">tb2l</samp>’</dt> <dd> above-below with half height resolution (left eye above, right eye below) </dd> <dt>‘<samp class="samp">ab2r</samp>’</dt> <dt>‘<samp class="samp">tb2r</samp>’</dt> <dd> above-below with half height resolution (right eye above, left eye below) </dd> <dt>‘<samp class="samp">al</samp>’</dt> <dd> alternating frames (left eye first, right eye second) </dd> <dt>‘<samp class="samp">ar</samp>’</dt> <dd> alternating frames (right eye first, left eye second) </dd> <dt>‘<samp class="samp">irl</samp>’</dt> <dd> interleaved rows (left eye has top row, right eye starts on next row) </dd> <dt>‘<samp class="samp">irr</samp>’</dt> <dd> interleaved rows (right eye has top row, left eye starts on next row) </dd> <dt>‘<samp class="samp">icl</samp>’</dt> <dd> interleaved columns, left eye first </dd> <dt>‘<samp class="samp">icr</samp>’</dt> <dd> interleaved columns, right eye first Default value is ‘<samp class="samp">sbsl</samp>’. </dd> </dl>

</dd>

<dt><samp class="option">out</samp></dt>

<dd> Set stereoscopic image format of output.

<dl class="table">

<dt>‘<samp class="samp">sbsl</samp>’</dt>

<dd> side by side parallel (left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbsr</samp>’</dt>
<dd> side by side crosseye (right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">sbs2l</samp>’</dt>
<dd> side by side parallel with half width resolution
(left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbs2r</samp>’</dt>
<dd> side by side crosseye with half width resolution
(right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">abl</samp>’</dt>
<dt>‘<samp class="samp">tbl</samp>’</dt>
<dd> above-below (left eye above, right eye below)

</dd>
<dt>‘<samp class="samp">abr</samp>’</dt>
<dt>‘<samp class="samp">tbr</samp>’</dt>
<dd> above-below (right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">ab2l</samp>’</dt>
<dt>‘<samp class="samp">tb2l</samp>’</dt>
<dd> above-below with half height resolution
(left eye above, right eye below)

</dd>

<dt>‘<samp class="samp">ab2r</samp>’</dt>
<dt>‘<samp class="samp">tb2r</samp>’</dt>
<dd> above-below with half height resolution
(right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">al</samp>’</dt>
<dd> alternating frames (left eye first, right eye second)

</dd>
<dt>‘<samp class="samp">ar</samp>’</dt>
<dd> alternating frames (right eye first, left eye second)

</dd>
<dt>‘<samp class="samp">irl</samp>’</dt>
<dd> interleaved rows (left eye has top row, right eye starts on next row)

</dd>
<dt>‘<samp class="samp">irr</samp>’</dt>
<dd> interleaved rows (right eye has top row, left eye starts on next row)

</dd>
<dt>‘<samp class="samp">arbg</samp>’</dt>
<dd> anaglyph red/blue gray
(red filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class="samp">argg</samp>’</dt>
<dd> anaglyph red/green gray
(red filter on left eye, green filter on right eye)

</dd>

<dt>‘<samp class="samp">arcg</samp>’</dt>

<dd> anaglyph red/cyan gray

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arch</samp>’</dt>

<dd> anaglyph red/cyan half colored

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcc</samp>’</dt>

<dd> anaglyph red/cyan color

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcd</samp>’</dt>

<dd> anaglyph red/cyan color optimized with the least squares projection of dubois

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">agmg</samp>’</dt>

<dd> anaglyph green/magenta gray

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmh</samp>’</dt>

<dd> anaglyph green/magenta half colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmc</samp>’</dt>

<dd> anaglyph green/magenta colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmd</samp>’</dt>

<dd> anaglyph green/magenta color optimized with the least squares projection of dubois

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">aybg</samp>’</dt>

<dd> anaglyph yellow/blue gray

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybh</samp>’</dt>

<dd> anaglyph yellow/blue half colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybc</samp>’</dt>

<dd> anaglyph yellow/blue colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybd</samp>’</dt>

<dd> anaglyph yellow/blue color optimized with the least squares projection of dubois

(yellow filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class=”samp”>ml</samp>’</dt>
<dd> mono output (left eye only)

</dd>
<dt>‘<samp class=”samp”>mr</samp>’</dt>
<dd> mono output (right eye only)

</dd>
<dt>‘<samp class=”samp”>chl</samp>’</dt>
<dd> checkerboard, left eye first

</dd>
<dt>‘<samp class=”samp”>chr</samp>’</dt>
<dd> checkerboard, right eye first

</dd>
<dt>‘<samp class=”samp”>icl</samp>’</dt>
<dd> interleaved columns, left eye first

</dd>
<dt>‘<samp class=”samp”>icr</samp>’</dt>
<dd> interleaved columns, right eye first

</dd>
<dt>‘<samp class=”samp”>hdmi</samp>’</dt>
<dd> HDMI frame pack
</dd>
</dl>

Default value is ‘<samp class="samp">arcd</samp>’. </dd> </dl>

 <h4 class="subsection">11.246.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert input video from side by side parallel to anaglyph yellow/blue dubois: <pre class="example-preformatted">stereo3d=sbsl:aybd </pre>

Convert input video from above below (left eye above, right eye below) to side by side crosseye. <pre class="example-preformatted">stereo3d=abl:sbsr </pre>

 <h3 class="section">11.247 streamselect, astreamselect# TOC</h3> Select video or audio streams. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 2. </dd> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">11.247.1 Commands# TOC</h4>

The <code class="code">streamselect</code> and <code class="code">astreamselect</code> filter supports the following commands: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">11.247.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select first 5 seconds 1st stream and rest of time 2nd stream: <pre class="example-preformatted">sendcmd='5.0 streamselect map 1',streamselect=inputs=2:map=0 </pre>

Same as above, but for audio: <pre class="example-preformatted">asendcmd='5.0 astreamselect map 1',astreamselect=inputs=2:map=0 </pre>

 <h3 class="section">11.248 subtitles# TOC</h3>

Draw subtitles on top of input video using the libass library. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libass</code>. This filter also requires a build with libavcodec and libavformat to convert the passed subtitles file to ASS (Advanced Substation Alpha) subtitles format. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the subtitle file to read. It must be specified. </dd> <dt><samp class="option">original_size</samp></dt> <dd> Specify the size of the original video, the video for which the ASS file was composed. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Due to a misdesign in ASS aspect ratio arithmetic, this is necessary to correctly scale the fonts if the aspect ratio has been changed. </dd> <dt><samp class="option">fontsdir</samp></dt> <dd> Set a directory path containing fonts that can be used by the filter. These fonts will be used in addition to whatever the font provider uses. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Process alpha channel, by default alpha channel is untouched. </dd> <dt><samp class="option">charenc</samp></dt> <dd> Set subtitles input character encoding. <code class="code">subtitles</code> filter only. Only useful if not UTF-8. </dd> <dt><samp class="option">stream_index, si</samp></dt> <dd> Set subtitles stream index. <code class="code">subtitles</code> filter only. </dd> <dt><samp class="option">force_style</samp></dt> <dd> Override default style or script info parameters of the subtitles. It accepts a string containing ASS style format <code class="code">KEY=VALUE</code> couples separated by ",". </dd> <dt><samp class="option">wrap_unicode</samp></dt> <dd> Break lines according to the Unicode Line Breaking Algorithm. Availability requires at least libass release 0.17.0 (or LIBASS_VERSION 0x01600010), and libass must have been built with libunibreak. The option is enabled by default except for native ASS. </dd> <dt><samp class="option">shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class="samp">simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class="samp">complex</samp>’</dt> <dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew,

Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd>
</dl>

The default is <code class="code">auto</code>. </dd> </dl>

If the first key is not specified, it is assumed that the first value specifies the <samp class="option">filename</samp>. For example, to render the file <samp class="file">sub.srt</samp> on top of the input video, use the command: <pre class="example-preformatted">subtitles=sub.srt </pre>

which is equivalent to: <pre class="example-preformatted">subtitles=filename=sub.srt </pre>

To render the default subtitles stream from file <samp class="file">video.mkv</samp>, use: <pre class="example-preformatted">subtitles=video.mkv
</pre>

To render the second subtitles stream from that file, use: <pre class="example-preformatted">subtitles=video.mkv:si=1
</pre>

To make the subtitles stream from <samp class="file">sub.srt</samp> appear in 80% transparent blue <code class="code">DejaVu Serif</code>, use: <pre class="example-preformatted">subtitles=sub.srt:force_style='Fontn
Serif,PrimaryColour=&HCCFF0000' </pre>

 <h3 class="section">11.249 super2xsai# TOC</h3>

Scale the input by 2x and smooth using the Super2xSaI (Scale and Interpolate) pixel art scaling algorithm. Useful for enlarging pixel art images without reducing sharpness. <h3 class="section">11.250 swaprect# TOC</h3>

Swap two rectangular objects in video. This filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dd> Set object width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set object height. </dd> <dt><samp class="option">x1</samp></dt> <dd> Set 1st rect x coordinate. </dd> <dt><samp class="option">y1</samp></dt> <dd> Set 1st rect y coordinate. </dd> <dt><samp class="option">x2</samp></dt> <dd> Set 2nd rect x coordinate. </dd> <dt><samp class="option">y2</samp></dt> <dd> Set 2nd rect y coordinate. All expressions are evaluated once for each frame. </dd> </dl>

The all options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>w</var> / <var class=”var”>h</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> </dl>

 <h4 class=”subsection”>11.250.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.251 swapuv# TOC</h3> Swap U & V plane. <h3 class=”section”>11.252 tblend# TOC</h3> Blend successive video frames. See blend <h3 class=”section”>11.253 telecine# TOC</h3>

Apply telecine process to the video. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> </dl>

 <pre class=”example-preformatted”>Some typical patterns:

NTSC output (30i): 27.5p: 32222 24p: 23 (classic) 24p: 2332 (preferred) 20p: 33 18p: 334 16p: 3444

PAL output (25i): 27.5p: 12222 24p: 222222222223 ("Euro pulldown") 16.67p: 33 16p: 33333334 </pre>

 <h3 class=”section”>11.254 thistogram# TOC</h3>

Compute and draw a color distribution histogram for the input video across time. Unlike histogram video filter which only shows histogram of single input frame at certain time, this filter shows also past histograms of number of frames defined by <code class=”code”>width</code> option. The computed histogram is a representation of the color component distribution in an image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dd> Set width of single color component output. Default value is <code class=”code”>0</code>. Value of <code class=”code”>0</code> means width will be picked from input video. This also set number of passed histograms to keep. Allowed range is [0, 8192]. </dd> <dt><samp class=”option”>display_mode, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class=”samp”>parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dd> Presents information identical to that in the <code class=”code”>parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class=”code”>stack</code>. </dd> <dt><samp class=”option”>levels_mode, m</samp></dt> <dd> Set mode. Can be either <code class=”code”>linear</code>, or <code class=”code”>logarithmic</code>. Default is <code class=”code”>linear</code>. </dd> <dt><samp class=”option”>components, c</samp></dt> <dd> Set what color components to display. Default is <code class=”code”>7</code>. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. Default is <code class=”code”>0.9</code>. </dd> <dt><samp class=”option”>envelope, e</samp></dt> <dd> Show envelope. Default is disabled. </dd> <dt><samp class=”option”>ecolor, ec</samp></dt> <dd> Set envelope color. Default is <code class=”code”>gold</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set

slide mode. Available values for slide is: <dl class="table">
<dt>‘<samp class="samp">frame</samp>’</dt> <dd> Draw
new frame when right border is reached. </dd> <dt>‘<samp
class="samp">replace</samp>’</dt> <dd> Replace old columns
with new ones. </dd> <dt>‘<samp class="samp">scroll</samp>’</dt>
<dd> Scroll from right to left. </dd> <dt>‘<samp
class="samp">rscroll</samp>’</dt> <dd> Scroll from left to
right. </dd> <dt>‘<samp class="samp">picture</samp>’</dt>
<dd> Draw single picture. </dd> </dl>

Default is <code class="code">replace</code>. </dd> </dl>

 <h3 class="section">11.255 threshold<span
class="pull-right"><a class="anchor hidden-xs" href="#threshold" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-threshold"
aria-hidden="true">TOC</h3>

Apply threshold effect to video stream. This filter needs
four video streams to perform thresholding. First stream is stream we are
filtering. Second stream is holding threshold values, third stream is holding
min values, and last, fourth stream is holding max values. The
filter accepts the following option: <dl class="table"> <dt><samp
class="option">planes</samp></dt> <dd> Set which planes will be
processed, unprocessed planes will be copied. By default value 0xf, all planes
will be processed. </dd> </dl>

For example if first stream pixel’s component value is less
then threshold value of pixel component from 2nd threshold stream, third
stream value will picked, otherwise fourth stream pixel component value
will be picked. Using color source filter one can perform
various types of thresholding:
<h4 class="subsection">11.255.1 Commands<a
class="anchor hidden-xs" href="#Commands-147" aria-hidden="true">#
TOC</h4>

This filter supports the all options as commands.
  <h4 class="subsection">11.255.2 Ex-
amples<a class="anchor hidden-xs" href="#Examples-
131" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-131" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Binary threshold, using gray color as
threshold: <pre class="example-preformatted">ffmpeg
-i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=black -f lavfi -i color=white
-lavfi threshold output.avi </pre>

Inverted binary threshold, using gray color as threshold: <div
class="example"> <pre class="example-preformatted">ffmpeg -i 320x240.avi
-f lavfi -i color=gray -f lavfi -i color=white -f lavfi -i color=black -lavfi threshold

output.avi </pre>

Truncate binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=gray -lavfi threshold output.avi </pre>

Threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=white -i 320x240.avi -lavfi threshold output.avi </pre>

Inverted threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=white -lavfi threshold output.avi </pre>

 <h3 class="section">11.256 thumbnail# TOC</h3> Select the most representative frame in a given sequence of consecutive frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> <dt><samp class="option">log</samp></dt> <dd> Set the log level to display picked frame stats. Default is <code class="code">info</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">11.256.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract one picture each 50 frames: <pre class="example-preformatted">thumbnail=50 </pre>

Complete example of a thumbnail creation with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf thumbnail,scale=300:200 -frames:v 1 out.png </pre>

 <h3 class="section">11.257 tile# TOC</h3>

Tile several successive frames together. The untile filter can do the reverse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size in the form <code class="code">COLUMNSxROWS</code>. Range is up to UINT_MAX cells. Default is <code class="code">6x5</code>. </dd> <dt><samp class="option">nb_frames</samp></dt> <dd> Set the maximum number of frames to render in the given area. It must be less than or equal to <var class="var">w</var>x<var class="var">h</var>. The default value is <code class="code">0</code>, meaning all the area will be used. </dd> <dt><samp class="option">margin</samp></dt> <dd> Set the outer border margin in pixels. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">padding</samp></dt> <dd> Set the inner border thickness (i.e. the number of pixels between frames). For more advanced padding options (such as having different values for the edges), refer to the pad video filter. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the unused area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set the number of frames to overlap when tiling several successive frames together. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> <dt><samp class="option">init_padding</samp></dt> <dd> Set the number of frames to initially be empty before displaying first output frame. This controls how soon will one get first output frame. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">11.257.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce 8x8 PNG tiles of all keyframes (<samp class="option">-skip_frame nokey</samp>) in a movie: <pre class="example-preformatted">ffmpeg -skip_frame nokey -i file.avi -vf 'scale=128:72,tile=8x8' -an -fps_mode passthrough keyframes%03d.png </pre> The <samp class="option">-fps_mode passthrough</samp> is necessary to prevent <code class="command">ffmpeg</code> from duplicating each output frame to accommodate the originally detected frame rate. Display <code class="code">5</code> pictures in an area of <code class="code">3x2</code> frames, with <code

class=”code”>7</code> pixels between them, and <code class=”code”>2</code> pixels of initial margin, using mixed flat and named options: <pre class=”example-preformatted”>tile=3x2:nb_frames=5:padding=7:margin=2 </pre>

 <h3 class=”section”>11.258 tiltandshift# TOC</h3> Apply tilt-and-shift effect. What happens when you invert time and space? Normally a video is composed of several frames that represent a different instant of time and shows a scene that evolves in the space captured by the frame. This filter is the antipode of that concept, taking inspiration from tilt and shift photography. A filtered frame contains the whole timeline of events composing the sequence, and this is obtained by placing a slice of pixels from each frame into a single one. However, since there are no infinite-width frames, this is done up the width of the input frame, and a video is recomposed by shifting away one column for each subsequent frame. In order to map space to time, the filter tilts each input frame as well, so that motion is preserved. This is accomplished by progressively selecting a different column from each input frame. The end result is a sort of inverted parallax, so that far away objects move much faster that the ones in the front. The ideal conditions for this video effect are when there is either very little motion and the background is static, or when there is a lot of motion and a very wide depth of field (e.g. wide panorama, while moving on a train). The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tilt</samp></dt> <dd> Tilt video while shifting (default). When unset, video will be sliding a static image, composed of the first column of each frame. </dd> <dt><samp class=”option”>start</samp></dt> <dd> What to do at the start of filtering (see below). </dd> <dt><samp class=”option”>end</samp></dt> <dd> What to do at the end of filtering (see below). </dd> <dt><samp class=”option”>hold</samp></dt> <dd> How many columns should pass through before start of filtering. </dd> <dt><samp class=”option”>pad</samp></dt> <dd> How many columns should be inserted before end of filtering. </dd> </dl>

Normally the filter shifts and tilts from the very first frame, and stops when the last one is received. However, before filtering starts, normal video may be preserved, so that the effect is slowly shifted in its place. Similarly, the last video frame may be reconstructed at the end. Alternatively it is possible to just start and end with black. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Filtering starts immediately and ends when the last frame is received. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> The first frames or the very last frame are kept intact during processing.

</dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Black is padded at the beginning or at the end of filtering. </dd> </dl>

 <h3 class="section">11.259 tinterlace# TOC</h3>

Perform various types of temporal field interlacing. Frames are counted starting from 1, so the first input frame is considered odd. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Specify the mode of the interlacing. This option can also be specified as a value alone. See below for a list of values for this option. Available values are: <dl class="table"> <dt>‘<samp class="samp">merge, 0</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at half frame rate. <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">drop_even, 1</samp>’</dt> <dd> Only output odd frames, even frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 11111 33333 11111 33333 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">drop_odd, 2</samp>’</dt> <dd> Only output even frames, odd frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 22222 44444 22222 44444 22222 44444 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">pad, 3</samp>’</dt> <dd> Expand each frame to full height, but pad alternate lines with black,

generating a frame with double height at the same input frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_top, 4</samp>’</dt> <dd> Interleave the upper field from odd frames with the lower field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<-

Output: 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_bottom, 5</samp>’</dt> <dd> Interleave the lower field from odd frames with the upper field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444

Output: 22222 44444 11111 33333 22222 44444 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">interlacex2, 6</samp>’</dt> <dd> Double frame rate with unchanged height. Frames are inserted each containing the second temporal field from the previous input frame and the first temporal field from the next input frame. This mode relies on the top_field_first flag. Useful for interlaced video displays with no field synchronisation.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 </pre>

</dd> <dt>‘<samp class="samp">mergex2, 7</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at same frame rate.

<pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 </pre>

</dd> </dl>

Numeric values are deprecated but are accepted for backward compatibility reasons. Default mode is <code class=”code”>merge</code>. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class=”var”>flags</var> is: <dl class=”table”> <dt><samp class=”option”>low_pass_filter, vlpf</samp></dt> <dd> Enable linear vertical low-pass filtering in the filter. Vertical low-pass filtering is required when creating an interlaced destination from a progressive source which contains high-frequency vertical detail. Filtering will reduce interlace ’twitter’ and Moire patterning. </dd> <dt><samp class=”option”>complex_filter, cvlpf</samp></dt> <dd> Enable complex vertical low-pass filtering. This will slightly less reduce interlace ’twitter’ and Moire patterning but better retain detail and subjective sharpness impression. </dd> <dt><samp class=”option”>bypass_il</samp></dt> <dd> Bypass already interlaced frames, only adjust the frame rate. </dd> </dl>

Vertical low-pass filtering and bypassing already interlaced frames can only be enabled for <samp class=”option”>mode</samp> <var class=”var”>interleave_top</var> and <var class=”var”>interleave_bottom</var>. </dd> </dl>

 <h3 class=”section”>11.260 tmedian# TOC</h3> Pick median pixels from several successive input video frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set radius of median filter. Default is 1. Allowed range is from 1 to 127. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>15</code>, by which all planes are processed. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Set median percentile. Default value is <code class=”code”>0.5</code>. Default value of <code class=”code”>0.5</code> will pick always median values, while <code class=”code”>0</code> will pick minimum values, and <code class=”code”>1</code> maximum values. </dd> </dl>

 <h4 class="subsection">11.260.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">radius</code>. <h3 class="section">11.261 tmidequalizer# TOC</h3>

Apply Temporal Midway Video Equalization effect. Midway Video Equalization adjusts a sequence of video frames to have the same histograms, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a video frames sequence. This filter accepts the following option: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set filtering radius. Default is <code class="code">5</code>. Allowed range is from 1 to 127. </dd> <dt><samp class="option">sigma</samp></dt> <dd> Set filtering sigma. Default is <code class="code">0.5</code>. This controls strength of filtering. Setting this option to 0 effectively does nothing. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> </dl>

 <h3 class="section">11.262 tmix# TOC</h3>

Mix successive video frames. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> The number of successive frames to mix. If unspecified, it defaults to 3. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input video frame. Each weight is separated by space. If number of weights is smaller than number of <var class="var">frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">11.262.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Average 7 successive frames: <pre class="example-preformatted">tmix=frames=7:weights="1 1 1 1 1 1 1" </pre>

Apply simple temporal convolution: <pre class="example-preformatted">tmix=frames=3:weights="-1 3 -1" </pre>

Similar as above but only showing temporal differences: <pre class="example-preformatted">tmix=frames=3:weights="-1 2 -1":scale=1 </pre>

 <h4 class="subsection">11.262.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">11.263 tonemap# TOC</h3> Tone map colors from different dynamic ranges. This filter expects data in single precision floating point, as it needs to operate on (and can output) out-of-range values. Another filter, such as zscale, is needed to convert the resulting frame to a usable format. The tonemapping algorithms implemented only work on linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,tonemap=clip,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h4 class="subsection">11.263.1 Options# TOC</h4> The filter accepts the following options. <dl class="table"> <dt><samp class="option">tonemap</samp></dt> <dd> Set the tone map algorithm to use. Possible values are: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Do not apply any tone map, only desaturate overbright pixels. </dd> <dt><var class="var">clip</var></dt> <dd> Hard-clip any out-of-range values. Use it for perfect color accuracy for in-range values, while distorting out-of-range values. </dd> <dt><var class="var">linear</var></dt>

<dd> Stretch the entire reference gamut to a linear multiple of the display. </dd> <dt><var class="var">gamma</var></dt> <dd> Fit a logarithmic transfer between the tone curves. </dd> <dt><var class="var">reinhard</var></dt> <dd> Preserve overall image brightness with a simple curve, using nonlinear contrast, which results in flattening details and degrading color accuracy. </dd> <dt><var class="var">hable</var></dt> <dd> Preserve both dark and bright details better than <var class="var">reinhard</var>, at the cost of slightly darkening everything. Use it when detail preservation is more important than color and brightness accuracy. </dd> <dt><var class="var">mobius</var></dt> <dd> Smoothly map out-of-range values, while retaining contrast and colors for in-range material as much as possible. Use it when color accuracy is more important than detail preservation. </dd> </dl>

Default is none. </dd> <dt><samp class="option">param</samp></dt> <dd> Tune the tone mapping algorithm. This affects the following algorithms: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Ignored. </dd> <dt><var class="var">linear</var></dt> <dd> Specifies the scale factor to use while stretching. Default to 1.0. </dd> <dt><var class="var">gamma</var></dt> <dd> Specifies the exponent of the function. Default to 1.8. </dd> <dt><var class="var">clip</var></dt> <dd> Specify an extra linear coefficient to multiply into the signal before clipping. Default to 1.0. </dd> <dt><var class="var">reinhard</var></dt> <dd> Specify the local contrast coefficient at the display peak. Default to 0.5, which means that in-gamut values will be about half as bright as when clipping. </dd> <dt><var class="var">hable</var></dt> <dd> Ignored. </dd> <dt><var class="var">mobius</var></dt> <dd> Specify the transition point from linear to mobius transform. Every value below this point is guaranteed to be mapped 1:1. The higher the value, the more accurate the result will be, at the cost of losing bright details. Default to 0.3, which due to the steep initial slope still preserves in-range colors fairly accurately. </dd> </dl>

</dd> <dt><samp class="option">desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by (smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default of 2.0 is somewhat conservative and will mostly just apply to skies or directly sunlit surfaces. A setting of 0.0 disables this option. This option works only if the input frame has a supported color tag. </dd> <dt><samp class="option">peak</samp></dt> <dd> Override signal/nominal/reference peak with this value. Useful when the embedded peak information in display metadata is not reliable or when tone mapping from a

lower range to a higher range. </dd> </dl>

 <h3 class="section">11.264 tpad# TOC</h3>

Temporarily pad video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Specify number of delay frames before input video stream. Default is 0. </dd> <dt><samp class="option">stop</samp></dt> <dd> Specify number of padding frames after input video stream. Set to -1 to pad indefinitely. Default is 0. </dd> <dt><samp class="option">start_mode</samp></dt> <dd> Set kind of frames added to beginning of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of first frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">stop_mode</samp></dt> <dd> Set kind of frames added to end of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of last frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">start_duration, stop_duration</samp></dt> <dd> Specify the duration of the start/stop delay. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. These options override <var class="var">start</var> and <var class="var">stop</var>. Default is 0. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> </dl>

 <h3 class="section">11.265 transpose# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">0, 4, cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip (default), that is: <pre class="example-preformatted">L.R L.l . . -> . . l.r R.r </pre>

</dd> <dt>‘<samp class="samp">1, 5, clock</samp>’</dt>
<dd> Rotate by 90 degrees clockwise, that is:
<pre class="example-preformatted">L.R l.L . . -> . . l.r r.R </pre>

</dd> <dt>‘<samp class="samp">2, 6, cclock</samp>’</dt>
<dd> Rotate by 90 degrees counterclockwise, that is: <div
class="example"> <pre class="example-preformatted">L.R R.r . . -> . .
l.r L.l </pre>

</dd> <dt>‘<samp class="samp">3, 7, clock_flip</samp>’</dt>
<dd> Rotate by 90 degrees clockwise and vertically flip, that is: <div
class="example"> <pre class="example-preformatted">L.R r.R . . -> . .
l.r l.L </pre> </dd> </dl>

For values between 4-7, the transposition is only done if the input video geometry is portrait and not landscape. These values are deprecated, the <code class="code">passthrough</code> option should be used instead. Numerical values are deprecated, and should be dropped in favor of symbolic constants. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">none</samp>’</dt> <dd> Always
apply transposition. </dd> <dt>‘<samp class="samp">portrait</samp>’</dt>
<dd> Preserve portrait geometry (when <var class="var">height</var>
>= <var class="var">width</var>). </dd> <dt>‘<samp
class="samp">landscape</samp>’</dt> <dd> Preserve
landscape geometry (when <var class="var">width</var> >= <var
class="var">height</var>). </dd> </dl>

Default value is <code class="code">none</code>. </dd> </dl>

For example to rotate by 90 degrees clockwise and preserve portrait layout:
 <pre class="example-preformatted">transpose=dir=1:passthrough=portrait
</pre>

The command above can also be specified as:
<pre class="example-preformatted">transpose=1:portrait </pre>

 <h3 class="section">11.266 trim<span class="pull-
right">#
TOC</h3>
Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table">
<dt><samp class="option">start</samp></dt> <dd> Specify the
time of the start of the kept section, i.e. the frame with the timestamp <var class="var">start</var> will be the first frame in the output. </dd> <dt><samp class="option">end</samp></dt>
<dd> Specify the time of the first frame that will be dropped,

i.e. the frame immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last frame in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> This is the same as <var class=”var”>start</var>, except this option sets the start timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> This is the same as <var class=”var”>end</var>, except this option sets the end timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> The number of the first frame that should be passed to the output. </dd> <dt><samp class=”option”>end_frame</samp></dt> <dd> The number of the first frame that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the frame timestamp, while the _frame variants simply count the frames that pass through the filter. Also note that this filter does not modify the timestamps. If you wish for the output timestamps to start at zero, insert a setpts filter after the trim filter. If multiple start or end options are set, this filter tries to be greedy and keep all the frames that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple trim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class=”itemize mark-bullet”> Drop everything except the second minute of input: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=60:120 </pre>

Keep only the first second: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=duration=1 </pre>

 <h3 class=”section”>11.267 unpremultiply# TOC</h3> Apply alpha unpremultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt>

<dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. If the format has 1 or 2 components, then luma is bit 0. If the format has 3 or 4 components: for RGB formats bit 0 is green, bit 1 is blue and bit 2 is red; for YUV formats bit 0 is luma, bit 1 is chroma-U and bit 2 is chroma-V. If present, the alpha channel is always the last bit. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">11.268 unsharp# TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_amount, la</samp></dt> <dd> Set the luma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 1.0. </dd> <dt><samp class="option">chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> <dt><samp class="option">alpha_msize_x, ax</samp></dt> <dd> Set the alpha matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_msize_y, ay</samp></dt> <dd> Set the alpha matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_amount, aa</samp></dt> <dd> Set the alpha effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class="subsection">11.268.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply strong luma sharpen effect: <pre class="example-preformatted">unsharp=luma_msize_x=7:luma_msize_y=7:luma_
</pre>

Apply a strong blur of both luma and chroma parameters: <pre class="example-preformatted">unsharp=7:7:-2:7:7:-2
</pre>

 <h3 class="section">11.269 untile# TOC</h3>

Decompose a video made of tiled images into the individual images. The frame rate of the output video is the frame rate of the input video multiplied by the number of tiles. This filter does the reverse of tile. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size (i.e. the number of lines and columns). For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">11.269.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce a 1-second video from a still image file made of 25 frames stacked vertically, like an analogic film reel: <pre class="example-preformatted">ffmpeg -r 1 -i image.jpg -vf untile=1x25 movie.mkv </pre>

 <h3 class="section">11.270 uspp# TOC</h3>

Apply ultra slow/simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">8</code> - all) shifts and average the results. The way this differs from the behavior of spp is that uspp actually encodes & decodes each case with libavcodec Snow, whereas spp uses a simplified intra only 8x8 DCT similar to MJPEG. This filter

is not available in ffmpeg versions between 5.0 and 6.0. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-8. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">8</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">codec</samp></dt> <dd> Use specified codec instead of snow. </dd> </dl>

 <h3 class="section">11.271 v360# TOC</h3>

Convert 360 videos between various formats. The filter accepts the following options: <dl class="table"> <dt><samp class="option">input</samp></dt> <dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. Available formats: <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">c3x2</samp>’</dt> <dt>‘<samp class="samp">c6x1</samp>’</dt> <dt>‘<samp class="samp">c1x6</samp>’</dt> <dd> Cubemap with 3x2/6x1/1x6 layout. Format specific options: <dl class="table"> <dt><samp class="option">in_pad</samp></dt> <dt><samp class="option">out_pad</samp></dt> <dd> Set padding proportion for the input/output cubemap. Values in decimals. Example values: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> No padding. </dd> <dt>‘<samp class="samp">0.01</samp>’</dt> <dd> 1% of face is padding. For example, with 1920x1280 resolution face size would be 640x640 and padding would be 3 pixels from each side. (640 * 0.01 = 6 pixels) </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. Maximum value is ‘<samp class="samp">0.1</samp>’. </dd> <dt><samp class="option">fin_pad</samp></dt> <dt><samp class="option">fout_pad</samp></dt> <dd> Set fixed padding for the input/output cubemap. Values in pixels. Default value is ‘<samp class="samp">0</samp>’. If greater than zero it overrides other padding options. </dd> <dt><samp class="option">in_forder</samp></dt> <dt><samp class="option">out_forder</samp></dt> <dd> Set order of faces

for the input/output cubemap. Choose one direction for each position.
Designation of directions: <dl class="table"> <dt>‘<samp
class="samp">r</samp>’</dt> <dd> right </dd>
<dt>‘<samp class="samp">l</samp>’</dt> <dd> left
</dd> <dt>‘<samp class="samp">u</samp>’</dt>
<dd> up </dd> <dt>‘<samp class="samp">d</samp>’</dt>
<dd> down </dd> <dt>‘<samp class="samp">f</samp>’</dt>
<dd> forward </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> back </dd> </dl>

Default value is ‘<samp class="samp">rludfb</samp>’.
 </dd> <dt><samp class="option">in_frot</samp></dt> <dt><samp
class="option">out_frot</samp></dt> <dd> Set rotation of faces for
the input/output cubemap. Choose one angle for each position.
Designation of angles: <dl class="table"> <dt>‘<samp
class="samp">0</samp>’</dt> <dd> 0 degrees clockwise
</dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> 90 degrees clockwise </dd> <dt>‘<samp class="samp">2</samp>’</dt>
<dd> 180 degrees clockwise </dd> <dt>‘<samp class="samp">3</samp>’</dt>
<dd> 270 degrees clockwise </dd> </dl>

Default value is ‘<samp class="samp">000000</samp>’.
</dd> </dl>

</dd> <dt>‘<samp class="samp">eac</samp>’</dt>
<dd> Equi-Angular Cubemap. </dd> <dt>‘<samp
class="samp">flat</samp>’</dt> <dt>‘<samp class="samp">gnomonic</samp>’<
<dt>‘<samp class="samp">rectilinear</samp>’</dt>
<dd> Regular video. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt>
<dd> Dual fisheye. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>

<dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>barrel</samp>’</dt> <dt>‘<samp class=”samp”>fb</samp>’</dt> <dt>‘<samp class=”samp”>barrelsplit</samp>’</dt> <dd> Facebook’s 360 formats. </dd> <dt>‘<samp class=”samp”>sg</samp>’</dt> <dd> Stereographic format. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>mercator</samp>’</dt> <dd> Mercator format. </dd> <dt>‘<samp class=”samp”>ball</samp>’</dt> <dd> Ball format, gives significant distortion toward the back. </dd> <dt>‘<samp class=”samp”>hammer</samp>’</dt> <dd> Hammer-Aitoff map projection format. </dd> <dt>‘<samp class=”samp”>sinusoidal</samp>’</dt> <dd> Sinusoidal map projection format. </dd> <dt>‘<samp class=”samp”>fisheye</samp>’</dt> <dd> Fisheye projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>pannini</samp>’</dt> <dd> Pannini projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dd> Set output pannini parameter. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dd> Set input pannini parameter. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>cylindrical</samp>’</dt> <dd> Cylindrical projection. Format specific options:

<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">perspective</samp>’</dt>
<dd> Perspective projection. (output only)
Format specific options: <dl class="table"> <dt><samp class="option">v_fov</samp></dt> <dd> Set perspective parameter. </dd> </dl>

</dd> <dt>‘<samp class="samp">tetrahedron</samp>’</dt>
<dd> Tetrahedron projection. </dd> <dt>‘<samp class="samp">tsp</samp>’</dt> <dd> Truncated square pyramid projection. </dd> <dt>‘<samp class="samp">he</samp>’</dt>
<dt>‘<samp class="samp">hequirect</samp>’</dt> <dd> Half equirectangular projection. </dd> <dt>‘<samp class="samp">equisolid</samp>’</dt
<dd> Equisolid format. Format specific options: <dl class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">og</samp>’</dt>
<dd> Orthographic format. Format specific options:
<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">octahedron</samp>’</dt>
<dd> Octahedron projection. </dd> <dt>‘<samp
class="samp">cylindricalea</samp>’</dt> <dd> Cylindrical
Equal Area projection. </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Set
interpolation method.
 Note: more complex interpolation
methods require much more memory to run. Available methods:
 <dl class="table"> <dt>‘<samp class="samp">near</samp>’</dt>
<dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Nearest
neighbour. </dd> <dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">linear</samp>’</dt> <dd> Bilinear
interpolation. </dd> <dt>‘<samp class="samp">lagrange9</samp>’</dt>
<dd> Lagrange9 interpolation. </dd> <dt>‘<samp
class="samp">cube</samp>’</dt> <dt>‘<samp class="samp">cubic</samp>’</dt
<dd> Bicubic interpolation. </dd> <dt>‘<samp class="samp">lanc</samp>’</dt
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Lanczos
interpolation. </dd> <dt>‘<samp class="samp">sp16</samp>’</dt>
<dt>‘<samp class="samp">spline16</samp>’</dt> <dd> Spline16
interpolation. </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">gaussian</samp>’</dt> <dd> Gaussian
interpolation. </dd> <dt>‘<samp class="samp">mitchell</samp>’</dt>
<dd> Mitchell interpolation. </dd> </dl>

Default value is ‘<samp class="samp">line</samp>’.
 </dd> <dt><samp class="option">w</samp></dt> <dt><samp
class="option">h</samp></dt> <dd> Set the output video res-
olution. Default resolution depends on formats.
</dd> <dt><samp class="option">in_stereo</samp></dt> <dt><samp
class="option">out_stereo</samp></dt> <dd> Set the input/output
stereo format. <dl class="table"> <dt>‘<samp class="samp">2d</samp>’</dt>
<dd> 2D mono </dd> <dt>‘<samp class="samp">sbs</samp>’</dt>
<dd> Side by side </dd> <dt>‘<samp class="samp">tb</samp>’</dt>
<dd> Top bottom </dd> </dl>

Default value is ‘<samp class="samp">2d</samp>’
for input and output format. </dd> <dt><samp class="option">yaw</samp></dt>
<dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt>
<dd> Set rotation for the output video. Values in degrees.
</dd> <dt><samp class="option">rorder</samp></dt> <dd> Set
rotation order for the output video. Choose one item for each posi-
tion. <dl class="table"> <dt>‘<samp class="samp">y,
Y</samp>’</dt> <dd> yaw </dd> <dt>‘<samp
class="samp">p, P</samp>’</dt> <dd> pitch </dd>
<dt>‘<samp class="samp">r, R</samp>’</dt> <dd> roll
</dd> </dl>

Default value is ‘<samp class="samp">ypr</samp>’.

 </dd> <dt><samp class="option">h_flip</samp></dt> <dt><samp class="option">v_flip</samp></dt> <dt><samp class="option">d_flip</samp></dt> <dd> Flip the output video horizontally(swaps left-right)/vertically(swaps up-down)/in-depth(swaps back-forward). Boolean values. </dd> <dt><samp class="option">ih_flip</samp></dt> <dt><samp class="option">iv_flip</samp></dt> <dd> Set if input video is flipped horizontally/vertically. Boolean values. </dd> <dt><samp class="option">in_trans</samp></dt> <dd> Set if input video is transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">out_trans</samp></dt> <dd> Set if output video needs to be transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">h_offset</samp></dt> <dt><samp class="option">v_offset</samp></dt> <dd> Set output horizontal/vertical off-axis offset. Default is set to 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">alpha_mask</samp></dt> <dd> Build mask in alpha plane for all unmapped pixels by marking them fully transparent. Boolean value, by default disabled. </dd> <dt><samp class="option">reset_rot</samp></dt> <dd> Reset rotation of output video. Boolean value, by default disabled. </dd> </dl>

 <h4 class="subsection">11.271.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert equirectangular video to cubemap with 3x2 layout and 1% padding using bicubic interpolation: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=e:c3x2:cubic:out_pad=0.01 output.mkv </pre> Extract back view of Equi-Angular Cubemap: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=eac:flat:yaw=180 output.mkv </pre> Convert transposed and horizontally flipped Equi-Angular Cubemap in side-by-side stereo format to equirectangular top-bottom stereo format: <pre class="example-preformatted">v360=eac:equirect:in_stereo=sbs:in_trans=1:ih_flip=1:out_stereo=tb </pre>

 <h4 class="subsection">11.271.2 Commands# TOC</h4>

This filter supports subset of above options as commands. <h3 class="section">11.272 vaguedenoiser# TOC</h3>

Apply a wavelet based denoiser. It transforms each frame from the video input into the wavelet domain, using Cohen-Daubechies-Feauveau 9/7. Then it applies some filtering to the obtained coefficients. It does an inverse wavelet transform after. Due to wavelet properties, it should give a nice smoothed result, and reduced noise, without blurring picture features. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> The filtering strength. The higher, the more filtered the video will be. Hard thresholding can use a higher threshold than soft thresholding before the video looks overfiltered. Default value is 2. </dd> <dt><samp class="option">method</samp></dt> <dd> The filtering method the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> All values under the threshold will be zeroed. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> All values under the threshold will be zeroed. All values above will be reduced by the threshold. </dd> <dt>‘<samp class="samp">garrote</samp>’</dt> <dd> Scales or nullifies coefficients - intermediary between (more) soft and (less) hard thresholding. </dd> </dl>

Default is garrote. </dd> <dt><samp class="option">nsteps</samp></dt> <dd> Number of times, the wavelet will decompose the picture. Picture can’t be decomposed beyond a particular point (typically, 8 for a 640x480 frame - as 2^9 = 512 > 480). Valid values are integers between 1 and 32. Default value is 6. </dd> <dt><samp class="option">percent</samp></dt> <dd> Partial of full denoising (limited coefficients shrinking), from 0 to 100. Default value is 85. </dd> <dt><samp class="option">planes</samp></dt> <dd> A list of the planes to process. By default all planes are processed. </dd> <dt><samp class="option">type</samp></dt> <dd> The threshold type the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">universal</samp>’</dt> <dd> Threshold used is same for all decompositions. </dd> <dt>‘<samp class="samp">bayes</samp>’</dt> <dd> Threshold used depends also on each decomposition coefficients. </dd> </dl>

Default is universal. </dd> </dl>

 <h3 class="section">11.273 varblur# TOC</h3> Apply variable blur filter by using 2nd video stream to set blur radius. The 2nd stream must have the same dimensions. This filter accepts the following options: <dl class="table"> <dt><samp class="option">min_r</samp></dt>

<dd> Set min allowed radius. Allowed range is from 0 to 254. Default is 0. </dd> <dt><samp class="option">max_r</samp></dt> <dd> Set max allowed radius. Allowed range is from 1 to 255. Default is 8. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. By default, all are used. </dd> </dl>

The <code class="code">varblur</code> filter also supports the framesync options. <h4 class="subsection">11.273.1 Commands# TOC</h4>

This filter supports all the above options as commands. <h3 class="section">11.274 vectorscope# TOC</h3>

Display 2 color component values in the two dimensional graph (which is called a vectorscope). This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set vectorscope mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">gray</samp>’</dt> <dt>‘<samp class="samp">tint</samp>’</dt> <dd> Gray values are displayed on graph, higher brightness means more pixels have same component color value on location in graph. This is the default mode. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Gray values are displayed on graph. Surrounding pixels values which are not present in video frame are drawn in gradient of 2 color components which are set by option <code class="code">x</code> and <code class="code">y</code>. The 3rd color component is static. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Actual color components values present in video frame are displayed on graph. </dd> <dt>‘<samp class="samp">color3</samp>’</dt> <dd> Similar as color2 but higher frequency of same values <code class="code">x</code> and <code class="code">y</code> on graph increases value of another color component, which is luminance by default values of <code class="code">x</code> and <code class="code">y</code>. </dd> <dt>‘<samp class="samp">color4</samp>’</dt> <dd> Actual colors present in video frame are displayed on graph. If two different colors map to same position on graph then color with higher value of component not present in graph is picked. </dd> <dt>‘<samp class="samp">color5</samp>’</dt> <dd> Gray values are displayed on graph. Similar to <code class="code">color</code> but with 3rd color component picked from radial gradient. </dd> </dl>

</dd> <dt><samp class="option">x</samp></dt> <dd> Set which color component will be represented on X-axis. Default is <code class="code">1</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set which color component will be represented on Y-axis. Default is <code class="code">2</code>. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity, used by modes: gray, color, color3 and color5 for increasing brightness of color component which represents frequency of (X, Y) location in graph. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, even darkest single pixel will be clearly highlighted. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold maximum and minimum values presented in graph over time. This way you can still spot out of range values without constantly looking at vectorscope. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set what kind of graticule to draw. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">green</samp>’</dt <dt>‘<samp class="samp">color</samp>’</dt> <dt>‘<samp class="samp">invert</samp>’</dt> </dl>

</dd> <dt><samp class="option">opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class="option">flags, f</samp></dt> <dd> Set graticule flags. <dl class="table"> <dt>‘<samp class="samp">white</samp>’</dt> <dd> Draw graticule for white point. </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Draw graticule for black point. </dd> <dt>‘<samp class="samp">name</samp>’</dt> <dd> Draw color points short names. </dd> </dl>

</dd> <dt><samp class="option">bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">lthreshold, l</samp></dt> <dd> Set low threshold for color component not represented on X or Y axis. Values lower than this value will be ignored. Default is 0. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and low threshold value of 0.1 actual threshold is 0.1 * 255 = 25. </dd> <dt><samp class="option">hthreshold, h</samp></dt> <dd> Set high threshold for color component not represented on X or Y axis. Values higher than this value will be ignored. Default is 1. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and high threshold value of 0.9 actual threshold is 0.9 * 255 = 230. </dd>

<dt><samp class="option">colorspace, c</samp></dt> <dd> Set what kind of colorspace to use when drawing graticule. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">601</samp>’</dt> <dt>‘<samp class="samp">709</samp>’</dt> </dl> Default is auto. </dd> <dt><samp class="option">tint0, t0</samp></dt> <dt><samp class="option">tint1, t1</samp></dt> <dd> Set color tint for gray/tint vectorscope mode. By default both options are zero. This means no tint, and output will remain gray. </dd> </dl>

 <h3 class="section">11.275 vidstabdetect# TOC</h3>

Analyze video stabilization/deshaking. Perform pass 1 of 2, see vidstabtransform for pass 2. This filter generates a file with relative translation and rotation transform information about subsequent frames, which is then used by the vidstabtransform filter. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">result</samp></dt> <dd> Set the path to the file used to write the transforms information. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">shakiness</samp></dt> <dd> Set how shaky the video is and how quick the camera is. It accepts an integer in the range 1-10, a value of 1 means little shakiness, a value of 10 means strong shakiness. Default value is 5. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set the accuracy of the detection process. It must be a value in the range 1-15. A value of 1 means low accuracy, a value of 15 means high accuracy. Default value is 15. </dd> <dt><samp class="option">stepsize</samp></dt> <dd> Set stepsize of the search process. The region around minimum is scanned with 1 pixel resolution. Default value is 6. </dd> <dt><samp class="option">mincontrast</samp></dt> <dd> Set minimum contrast. Below this value a local measurement field is discarded. Must be a floating point value in the range 0-1. Default value is 0.3. </dd> <dt><samp class="option">tripod</samp></dt> <dd> Set reference frame number for tripod mode. If enabled, the motion of the frames is compared to a reference frame in the filtered stream, identified by the specified number. The idea is to compensate all movements in a more-or-less static scene and keep the camera view absolutely still. If set to 0, it is disabled. The frames are counted starting from 1. </dd> <dt><samp class="option">show</samp></dt> <dd> Show fields and transforms in the resulting frames. It accepts an integer in the range 0-2.

Default value is 0, which disables any visualization. </dd> <dt><samp class="option">fileformat</samp></dt> <dd> Format for the transforms data file to be written. Acceptable values are <dl class="table"> <dt>‘<samp class="samp">ascii</samp>’</dt> <dd> Human-readable plain text </dd> <dt>‘<samp class="samp">binary</samp>’</dt> <dd> Binary format, roughly 40% smaller than <code class="code">ascii</code>. (default) </dd> </dl>

</dd> </dl>

 <h4 class="subsection">11.275.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use default values: <pre class="example-preformatted">vidstabdetect </pre>

Analyze strongly shaky movie and put the results in file <samp class="file">mytransforms.trf</samp>: <pre class="example-preformatted">vidstabdetect=shakiness=10:accuracy=15:result="mytransforms.trf&qu
</pre>

Visualize the result of internal transformations in the resulting video: <pre class="example-preformatted">vidstabdetect=show=1 </pre>

Analyze a video with medium shakiness using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input -vf vidstabdetect=shakiness=5:show=1 dummy.avi </pre>

 <h3 class="section">11.276 vidstabtransform# TOC</h3>

Video stabilization/deshaking: pass 2 of 2, see vidstabdetect for pass 1. Read a file with transform information for each frame and apply/compensate them. Together with the vidstabdetect filter this can be used to deshake videos. See also http://public.hronopik.de/vid.stab. It is important to also use the unsharp filter, see below. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. <h4 class="subsection">11.276.1 Options<a class="anchor hidden-xs" href="#Options-4" aria-

hidden=”true”># TOC</h4>

<dl class=”table”> <dt><samp class=”option”>input</samp></dt> <dd> Set path to the file used to read the transforms. Default value is <samp class=”file”>transforms.trf</samp>. </dd> <dt><samp class=”option”>smoothing</samp></dt> <dd> Set the number of frames (value*2 + 1) used for lowpass filtering the camera movements. Default value is 10. For example a number of 10 means that 21 frames are used (10 in the past and 10 in the future) to smoothen the motion in the video. A larger value leads to a smoother video, but limits the acceleration of the camera (pan/tilt movements). 0 is a special case where a static camera is simulated. </dd> <dt><samp class=”option”>optalgo</samp></dt> <dd> Set the camera path optimization algorithm. Accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dd> gaussian kernel low-pass filter on camera motion (default) </dd> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dd> averaging on transformations </dd> </dl>

</dd> <dt><samp class=”option”>maxshift</samp></dt> <dd> Set maximal number of pixels to translate frames. Default value is -1, meaning no limit. </dd> <dt><samp class=”option”>maxangle</samp></dt> <dd> Set maximal angle in radians (degree*PI/180) to rotate frames. Default value is -1, meaning no limit. </dd> <dt><samp class=”option”>crop</samp></dt> <dd> Specify how to deal with borders that may be visible due to movement compensation. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>keep</samp>’</dt> <dd> keep image information from previous frame (default) </dd> <dt>‘<samp class=”samp”>black</samp>’</dt> <dd> fill the border black </dd> </dl>

</dd> <dt><samp class=”option”>invert</samp></dt> <dd> Invert transforms if set to 1. Default value is 0. </dd> <dt><samp class=”option”>relative</samp></dt> <dd> Consider transforms as relative to previous frame if set to 1, absolute if set to 0. Default value is 0. </dd> <dt><samp class=”option”>zoom</samp></dt> <dd> Set percentage to zoom. A positive value will result in a zoom-in effect, a negative value in a zoom-out effect. Default value is 0 (no zoom). </dd> <dt><samp class=”option”>optzoom</samp></dt> <dd> Set optimal zooming to avoid borders. Accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> disabled </dd> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> optimal static zoom value is determined (only very strong movements will lead to visible borders) (default) </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> optimal adaptive zoom

value is determined (no borders will be visible), see <samp class="option">zoomspeed</samp>
</dd> </dl>

Note that the value given at zoom is added to the one calculated here. </dd> <dt><samp class="option">zoomspeed</samp></dt>
<dd> Set percent to zoom maximally each frame (enabled when <samp class="option">optzoom</samp> is set to 2). Range is from 0 to 5, default value is 0.25. </dd> <dt><samp class="option">interpol</samp></dt>
<dd> Specify type of interpolation. Available values are:
<dl class="table"> <dt>‘<samp class="samp">no</samp>’</dt>
<dd> no interpolation </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> linear only horizontal </dd> <dt>‘<samp class="samp">bilinear</samp>’</
<dd> linear in both directions (default) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> cubic in both directions (slow) </dd> </dl>

</dd> <dt><samp class="option">tripod</samp></dt> <dd> Enable virtual tripod mode if set to 1, which is equivalent to <code class="code">relative=0:smoothing=0</code>. Default value is 0. Use also <code class="code">tripod</code> option of vidstabdetect.
</dd> <dt><samp class="option">debug</samp></dt> <dd> Increase log verbosity if set to 1. Also the detected global motions are written to the temporary file <samp class="file">global_motions.trf</samp>. Default value is 0. </dd> </dl>

 <h4 class="subsection">11.276.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> for a typical stabilization with default values:
<pre class="example-preformatted">ffmpeg -i inp.mpeg -vf vidstabtransform,unsharp=5:5:0.8:3:3:0.4 inp_stabilized.mpeg </pre>

Note the use of the unsharp filter which is always recommended. Zoom in a bit more and load transform data from a given file: <pre class="example-preformatted">vidstabtransform=zoom=5:input="mytransforms.trf"
</pre>

Smoothen the video even more: <pre class="example-preformatted">vidstabtransform=smoothing=30 </pre>

 <h3 class="section">11.277 vflip#
TOC</h3>

Flip the input video vertically. For example, to vertically

flip a video with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i in.avi -vf "vflip" out.avi </pre>

 <h3 class=”section”>11.278 vfrdet# TOC</h3>

Detect variable frame rate video. This filter tries to detect if the input is variable or constant frame rate. At end it will output number of frames detected as having variable delta pts, and ones with constant delta pts. If there was frames with variable delta, than it will also show min, max and average delta encountered. <h3 class=”section”>11.279 vibrance# TOC</h3>

Boost or alter saturation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>intensity</samp></dt> <dd> Set strength of boost if positive value or strength of alter if negative value. Default is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>rbal</samp></dt> <dd> Set the red balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>gbal</samp></dt> <dd> Set the green balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>bbal</samp></dt> <dd> Set the blue balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>rlum</samp></dt> <dd> Set the red luma coefficient. </dd> <dt><samp class=”option”>glum</samp></dt> <dd> Set the green luma coefficient. </dd> <dt><samp class=”option”>blum</samp></dt> <dd> Set the blue luma coefficient. </dd> <dt><samp class=”option”>alternate</samp></dt> <dd> If <code class=”code”>intensity</code> is negative and this is set to 1, colors will change, otherwise colors will be less saturated, more towards gray. </dd> </dl>

 <h4 class=”subsection”>11.279.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>11.280 vif# TOC</h3>

Obtain the average VIF (Visual Information Fidelity) between two input

videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average VIF score is printed through the logging system. The filter stores the calculated VIF score of each frame. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi vif -f null - </pre>

 <h3 class=”section”>11.281 vignette# TOC</h3>

Make or reverse a natural vignetting effect. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>angle, a</samp></dt> <dd> Set lens angle expression as a number of radians. The value is clipped in the <code class=”code”>[0,PI/2]</code> range. Default value: <code class=”code”>"PI/5"</code> </dd> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dd> Set center coordinates expressions. Respectively <code class=”code”>"w/2"</code> and <code class=”code”>"h/2"</code> by default. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set forward/backward mode. Available modes are: <dl class=”table”> <dt>‘<samp class=”samp”>forward</samp>’</dt> <dd> The larger the distance from the central point, the darker the image becomes. </dd> <dt>‘<samp class=”samp”>backward</samp>’</dt> <dd> The larger the distance from the central point, the brighter the image becomes. This can be used to reverse a vignette effect, though there is no automatic detection to extract the lens <samp class=”option”>angle</samp> and other settings (yet). It can also be used to create a burning effect. </dd> </dl>

Default value is ‘<samp class=”samp”>forward</samp>’. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set evaluation mode for the expressions (<samp class=”option”>angle</samp>, <samp class=”option”>x0</samp>, <samp class=”option”>y0</samp>). It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> Evaluate expressions only once during the filter initialization. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. This is way slower than the ‘<samp class=”samp”>init</samp>’ mode since it requires all the scalers to

be re-computed, but it allows advanced dynamic expressions. </dd>
</dl>

Default value is ‘<samp class="samp">init</samp>’.
 </dd> <dt><samp class="option">dither</samp></dt> <dd> Set
dithering to reduce the circular banding effects. Default is <code
class="code">1</code> (enabled). </dd> <dt><samp class="option">aspect</samp></dt>
<dd> Set vignette aspect. This setting allows one to adjust the shape of
the vignette. Setting this value to the SAR of the input will make a rectangular
vignetting following the dimensions of the video. Default is <code
class="code">1/1</code>. </dd> </dl>

 <h4 class="subsection">11.281.1 Expressions<a class="anchor hidden-xs" href="#Expressions"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Expressions" aria-hidden="true">TOC</h4>

The <samp class="option">alpha</samp>, <samp class="option">x0</samp>
and <samp class="option">y0</samp> expressions can contain the following
parameters. <dl class="table"> <dt><samp class="option">w</samp></dt>
<dt><samp class="option">h</samp></dt> <dd> input width
and height </dd> <dt><samp class="option">n</samp></dt>
<dd> the number of input frame, starting from 0 </dd>
<dt><samp class="option">pts</samp></dt> <dd> the PTS (Presentation TimeStamp) time of the filtered video frame, expressed in <var
class="var">TB</var> units, NAN if undefined </dd> <dt><samp
class="option">r</samp></dt> <dd> frame rate of the input video,
NAN if the input frame rate is unknown </dd> <dt><samp
class="option">t</samp></dt> <dd> the PTS (Presentation TimeStamp) of the filtered video frame, expressed in seconds, NAN if undefined
</dd> <dt><samp class="option">tb</samp></dt> <dd> time base
of the input video </dd> </dl>

 <h4 class="subsection">11.281.2 Examples<a class="anchor hidden-xs" href="#Examples-
140" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-140" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Apply simple strong vignetting effect:
 <pre class="example-preformatted">vignette=PI/4
</pre>

Make a flickering vignetting: <pre
class="example-preformatted">vignette='PI/4+random(1)*PI/50':eval=frame
</pre>

 <h3 class="section">11.282 vmafmotion<a class="anchor hidden-xs" href="#vmafmotion"

aria-hidden=”true”># TOC</h3>

Obtain the average VMAF motion score of a video. It is one of the component metrics of VMAF. The obtained average motion score is printed through the logging system. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stats_file</samp></dt> <dd> If specified, the filter will use the named file to save the motion score of each frame with respect to the previous frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

Example: <pre class=”example-preformatted”>ffmpeg -i ref.mpg -vf vmafmotion -f null - </pre>

 <h3 class=”section”>11.283 vpp_amf# TOC</h3>

Scale (resize) and convert colorspace, transfer characteristics or color primaries for the input video, using AMD Advanced Media Framework library for hardware acceleration. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>scale_type</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear This is the default. </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used.

</dd> <dt><samp class=”option”>force_original_aspect_ratio</samp></dt> <dt><samp class=”option”>force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class=”option”>reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class=”option”>in_color_range</samp></dt> <dd> Override input color range. </dd> <dt><samp class=”option”>out_color_range</samp><dd> Specify output color range. The accepted values for in_trc and out_trc are: <dl class=”table”> <dt>‘<samp class=”samp”>studio</samp>’</dt> <dd> Studio (or re-

stricted, or MPEG) color range. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Full (or JPEG) color range. </dd> </dl>

</dd> <dt><samp class="option">color_profile</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">in_trc</samp></dt> <dd> Override input transfer characteristics. </dd> <dt><samp class="option">out_trc</samp></dt> <dd> Specify output transfer characteristics. The accepted values for in_trc and out_trc are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dd> Constant gamma of 2.2 </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dd> Constant gamma of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> LOG </dd> <dt>‘<samp class="samp">log-sqrt</samp>’</dt> <dd> LOG_SQRT </dd> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4 </dd> <dt>‘<samp class="samp">bt1361-ecg</samp>’</dt> <dd> BT1361_ECG </dd> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for 10-bits content </dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dd> BT.2020 for 12-bits content </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dd> SMPTE2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dd> SMPTE428 </dd> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> <dd> ARIB_STD_B67 </dd> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dd> Override input color primaries. </dd> <dt><samp class="option">out_primaries</samp></dt> <dd> Specify output color primaries. The accepted values for in_primaries and out_primaries are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp

class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>film</samp>’</dt> <dd> film </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dd> SMPTE-428 </dd> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class=”samp”>jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>11.283.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class=”example-preformatted”>vpp_amf=-2:720:format=yuv420p,hwdownload,format=yuv420p </pre>

Upscale to 4K and change color profile to bt2020. <pre class=”example-preformatted”>vpp_amf=4096:2160:color_profile=bt2020 </pre>

Override input primaries and input transfer characteristics, change both to bt709.

 <pre class=”example-preformatted”>vpp_amf=color_profile=bt2020:in_trc=smpte2 </pre>

 <h3 class=”section”>11.284 vstack# TOC</h3> Stack input videos vertically. All streams must be of same pixel format and of same width. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class=”section”>11.285 w3fdif<a class=”anchor hidden-xs” href=”#w3fdif” aria-

hidden=”true”># TOC</h3>

Deinterlace the input video ("w3fdif" stands for "Weston 3 Field Deinterlacing Filter"). Based on the process described by Martin Weston for BBC R&D, and implemented based on the de-interlace algorithm written by Jim Easterbrook for BBC R&D, the Weston 3 field deinterlacing filter uses filter coefficients calculated by BBC R&D. This filter uses field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class=”code”>w3fdif</code> filter. There are two sets of filter coefficients, so called "simple" and "complex". Which set of filter coefficients is used can be set by passing an optional parameter: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set the interlacing filter coefficients. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Simple filter coefficient set. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> More-complex filter coefficient set. </dd> </dl> Default value is ‘<samp class=”samp”>complex</samp>’. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames, </dd> <dt>‘<samp class=”samp”>interlaced</samp>&rsquo <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

Default value is ‘<samp class=”samp”>all</samp>’. </dd> </dl>

 <h4 class="subsection">11.285.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">11.286 waveform# TOC</h3> Video waveform monitor. The waveform monitor plots color component intensity. By default luma only. Each column of the waveform corresponds to a column of pixels in the source video. It accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Can be either <code class="code">row</code>, or <code class="code">column</code>. Default is <code class="code">column</code>. In row mode, the graph on the left side represents color component value 0 and the right side represents value = 255. In column mode, the top side represents color component value = 0 and bottom side represents value = 255. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity. Smaller values are useful to find out how many values of the same luminance are distributed across input rows/columns. Default value is <code class="code">0.04</code>. Allowed range is [0, 1]. </dd> <dt><samp class="option">mirror, r</samp></dt> <dd> Set mirroring mode. <code class="code">0</code> means unmirrored, <code class="code">1</code> means mirrored. In mirrored mode, higher values will be represented on the left side for <code class="code">row</code> mode and at the top for <code class="code">column</code> mode. Default is <code class="code">1</code> (mirrored). </dd> <dt><samp class="option">display, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. This display mode makes it easier to spot relative differences or similarities in overlapping areas of the color components that are supposed to be identical, such as neutral whites, grays, or blacks. </dd> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">row</code> mode or one below the other in <code class="code">column</code> mode. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">column</code> mode or one below the other in <code class="code">row</code> mode. Using this display mode makes it easy to spot color casts in the highlights and shadows of an image, by comparing the contours of the top and the bottom

graphs of each waveform. Since whites, grays, and blacks are characterized by exactly equal amounts of red, green, and blue, neutral areas of the picture should display three waveforms of roughly equal width/height. If not, the correction is easy to perform by making level adjustments the three waveforms. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">components, c</samp></dt> <dd> Set which color components to display. Default is 1, which means only luma or red color component if input is in RGB colorspace. If is set for example to 7 it will display all 3 (if) available color components. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, minimum and maximum values presented in graph will be easily visible even with small <code class="code">step</code> value. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold minimum and maximum values presented in graph across time. This way you can still spot out of range values without constantly looking at waveforms. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">filter, f</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">lowpass</samp>’</dt> <dd> No filtering, this is default. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Luma and chroma combined together. </dd> <dt>‘<samp class="samp">aflat</samp>’</dt> <dd> Similar as above, but shows difference between blue and red chroma. </dd> <dt>‘<samp class="samp">xflat</samp>’</dt> <dd> Similar as above, but use different colors. </dd> <dt>‘<samp class="samp">yflat</samp>’</dt> <dd> Similar as above, but again with different colors. </dd> <dt>‘<samp class="samp">chroma</samp>’</dt> <dd> Displays only chroma. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Displays actual color value on waveform. </dd> <dt>‘<samp class="samp">acolor</samp>’</dt> <dd> Similar as above, but with luma showing frequency of chroma values. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set which graticule to display. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do not display graticule. </dd> <dt>‘<samp class="samp">green</samp>’</dt> <dd> Display green graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">orange</samp>’</dt> <dd> Display orange graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">invert</samp>’</dt> <dd> Display invert graticule showing legal broadcast ranges. </dd>

</dl>

</dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class=”option”>flags, fl</samp></dt> <dd> Set graticule flags. <dl class=”table”> <dt>‘<samp class=”samp”>numbers</samp>’</dt> <dd> Draw numbers above lines. By default enabled. </dd> <dt>‘<samp class=”samp”>dots</samp>’</dt> <dd> Draw dots instead of lines. </dd> </dl>

</dd> <dt><samp class=”option”>scale, s</samp></dt> <dd> Set scale used for displaying graticule. <dl class=”table”> <dt>‘<samp class=”samp”>digital</samp>’</dt> <dt>‘<samp class=”samp”>millivolts</samp>&rsquo <dt>‘<samp class=”samp”>ire</samp>’</dt> </dl> Default is digital. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class=”option”>tint0, t0</samp></dt> <dt><samp class=”option”>tint1, t1</samp></dt> <dd> Set tint for output. Only used with lowpass filter and when display is not overlay and input pixel formats are not RGB. </dd> <dt><samp class=”option”>fitmode, fm</samp></dt> <dd> Set sample aspect ratio of video output frames. Can be used to configure waveform so it is not stretched too much in one of directions. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Set sample aspect ration to 1/1. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Set sample aspect ratio to match input size of video </dd> </dl> Default is ‘<samp class=”samp”>none</samp>’. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set input formats for filter to pick from. Can be ‘<samp class=”samp”>all</samp>’, for selecting from all available formats, or ‘<samp class=”samp”>first</samp>’, for selecting first available format. Default is ‘<samp class=”samp”>first</samp>’. </dd> </dl>

 <h3 class=”section”>11.287 weave, doubleweave# TOC</h3>

The <code class=”code”>weave</code> takes a field-based video input and join each two sequential fields into single frame, producing a new double height clip with half the frame rate and half the frame count. The <code class=”code”>doubleweave</code> works same as <code class=”code”>weave</code> but without halving frame rate and frame count. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd> Set first field. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> Set the frame as

top-field-first. </dd> <dt>‘<samp class="samp">bottom, b</samp>’</dt> <dd> Set the frame as bottom-field-first. </dd> </dl> </dd> </dl>

 <h4 class="subsection">11.287.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interlace video using select and separatefields filter: <pre class="example-preformatted">separatefields,select=eq(mod(n,4),0)+eq(
</pre>

 <h3 class="section">11.288 xbr# TOC</h3> Apply the xBR high-quality magnification filter which is designed for pixel art. It follows a set of edge-detection rules, see https://forums.libretro.com/t/xbr-algorithm-tutorial/123. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xBR</code>, <code class="code">3</code> for <code class="code">3xBR</code> and <code class="code">4</code> for <code class="code">4xBR</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">11.289 xcorrelate# TOC</h3> Apply normalized cross-correlation between first and second input video stream. Second input video stream dimensions must be lower than first input video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">secondary</samp></dt> <dd> Set which secondary video frames will be processed from second input video stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">xcorrelate</code> filter also supports the framesync options. <h3 class="section">11.290 xfade# TOC</h3>

Apply cross fade from one input video stream to another input video stream.

The cross fade is applied for specified duration.

Both inputs must be constant frame-rate and have the same resolution, pixel format,

frame rate and timebase.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">transition</samp></dt>

<dd> Set one of available transition effects:

<dl class="table">

<dt>‘<samp class="samp">custom</samp>’</dt>

<dt>‘<samp class="samp">fade</samp>’</dt>

<dt>‘<samp class="samp">wipeleft</samp>’</dt>

<dt>‘<samp class="samp">wiperight</samp>’</dt>

<dt>‘<samp class="samp">wipeup</samp>’</dt>

<dt>‘<samp class="samp">wipedown</samp>’</dt>

<dt>‘<samp class="samp">slideleft</samp>’</dt>

<dt>‘<samp class="samp">slideright</samp>’</dt>

<dt>‘<samp class="samp">slideup</samp>’</dt>

<dt>‘<samp class="samp">slidedown</samp>’</dt>

<dt>‘<samp class="samp">circlecrop</samp>’</dt>

<dt>‘<samp class="samp">rectcrop</samp>’</dt>

<dt>‘<samp class="samp">distance</samp>’</dt>

<dt>‘<samp class="samp">fadeblack</samp>’</dt>

<dt>‘<samp class="samp">fadewhite</samp>’</dt>

<dt>‘<samp class="samp">radial</samp>’</dt>

<dt>‘<samp class="samp">smoothleft</samp>’</dt>

<dt>‘<samp class="samp">smoothright</samp>’</dt>

<dt>‘<samp class="samp">smoothup</samp>’</dt>
<dt>‘<samp class="samp">smoothdown</samp>’</dt>
<dt>‘<samp class="samp">circleopen</samp>’</dt>
<dt>‘<samp class="samp">circleclose</samp>’</dt>
<dt>‘<samp class="samp">vertopen</samp>’</dt>
<dt>‘<samp class="samp">vertclose</samp>’</dt>
<dt>‘<samp class="samp">horzopen</samp>’</dt>
<dt>‘<samp class="samp">horzclose</samp>’</dt>
<dt>‘<samp class="samp">dissolve</samp>’</dt>
<dt>‘<samp class="samp">pixelize</samp>’</dt>
<dt>‘<samp class="samp">diagtl</samp>’</dt>
<dt>‘<samp class="samp">diagtr</samp>’</dt>
<dt>‘<samp class="samp">diagbl</samp>’</dt>
<dt>‘<samp class="samp">diagbr</samp>’</dt>
<dt>‘<samp class="samp">hlslice</samp>’</dt>
<dt>‘<samp class="samp">hrslice</samp>’</dt>
<dt>‘<samp class="samp">vuslice</samp>’</dt>
<dt>‘<samp class="samp">vdslice</samp>’</dt>
<dt>‘<samp class="samp">hblur</samp>’</dt>
<dt>‘<samp class="samp">fadegrays</samp>’</dt>
<dt>‘<samp class="samp">wipetl</samp>’</dt>
<dt>‘<samp class="samp">wipetr</samp>’</dt>
<dt>‘<samp class="samp">wipebl</samp>’</dt>
<dt>‘<samp class="samp">wipebr</samp>’</dt>
<dt>‘<samp class="samp">squeezeh</samp>’</dt>
<dt>‘<samp class="samp">squeezev</samp>’</dt>
<dt>‘<samp class="samp">zoomin</samp>’</dt>
<dt>‘<samp class="samp">fadefast</samp>’</dt>
<dt>‘<samp class="samp">fadeslow</samp>’</dt>
<dt>‘<samp class="samp">hlwind</samp>’</dt>
<dt>‘<samp class="samp">hrwind</samp>’</dt>

<dt>‘<samp class="samp">vuwind</samp>’</dt>
<dt>‘<samp class="samp">vdwind</samp>’</dt>
<dt>‘<samp class="samp">coverleft</samp>’</dt>
<dt>‘<samp class="samp">coverright</samp>’</dt>
<dt>‘<samp class="samp">coverup</samp>’</dt>
<dt>‘<samp class="samp">coverdown</samp>’</dt>
<dt>‘<samp class="samp">revealleft</samp>’</dt>
<dt>‘<samp class="samp">revealright</samp>’</dt>
<dt>‘<samp class="samp">revealup</samp>’</dt>
<dt>‘<samp class="samp">revealdown</samp>’</dt>
</dl>
Default transition effect is fade.

</dd>
<dt><samp class="option">duration</samp></dt>
<dd> Set cross fade duration in seconds.
Range is 0 to 60 seconds.
Default duration is 1 second.

</dd>
<dt><samp class="option">offset</samp></dt>
<dd> Set cross fade start relative to first input stream in seconds.
Default offset is 0.

</dd>
<dt><samp class="option">expr</samp></dt>
<dd> Set expression for custom transition effect.

The expressions can use the following variables and functions:

<dl class="table">

<dt><samp class="option">X</samp></dt>
<dt><samp class="option">Y</samp></dt>
<dd> The coordinates of the current sample.

</dd>
<dt><samp class="option">W</samp></dt>
<dt><samp class="option">H</samp></dt>
<dd> The width and height of the image.

</dd>
<dt><samp class="option">P</samp></dt>
<dd> Progress of transition effect.

</dd>
<dt><samp class="option">PLANE</samp></dt>
<dd> Currently processed plane.

</dd>
<dt><samp class="option">A</samp></dt>
<dd> Return value of first input at current location and plane.

</dd>
<dt><samp class="option">B</samp></dt>
<dd> Return value of second input at current location and plane.

</dd>
<dt><samp class="option">a0(x, y)</samp></dt>
<dt><samp class="option">a1(x, y)</samp></dt>
<dt><samp class="option">a2(x, y)</samp></dt>
<dt><samp class="option">a3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of first input.

</dd>

<dt><samp class="option">b0(x, y)</samp></dt>

<dt><samp class="option">b1(x, y)</samp></dt>

<dt><samp class="option">b2(x, y)</samp></dt>

<dt><samp class="option">b3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of second input.

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">11.290.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input video to another input video, with fade transition and duration of transition of 2 seconds starting at offset of 5 seconds: <pre class="example-preformatted">ffmpeg -i first.mp4 -i second.mp4 -filter_complex xfade=transition=fade:duration=2:offset=5 output.mp4 </pre>

 <h3 class="section">11.291 xmedian# TOC</h3> Pick median pixels from several input videos. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 3. Allowed range is from 3 to 255. If number of inputs is even number, than result will be mean value between two median values. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">15</code>, by which all planes are processed. </dd> <dt><samp class="option">percentile</samp></dt>

<dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">11.291.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">inputs</code>. <h3 class="section">11.292 xpsnr# TOC</h3>

Obtain the average (across all input frames) and minimum (across all color plane averages) eXtended Perceptually weighted peak Signal-to-Noise Ratio (XPSNR) between two input videos. The XPSNR is a low-complexity psychovisually motivated distortion measurement algorithm for assessing the difference between two video streams or images. This is especially useful for objectively quantifying the distortions caused by video and image codecs, as an alternative to a formal subjective test. The logarithmic XPSNR output values are in a similar range as those of traditional psnr assessments but better reflect human impressions of visual coding quality. More details on the XPSNR measure, which essentially represents a blockwise weighted variant of the PSNR measure, can be found in the following freely available papers: <ul class="itemize mark-bullet"> C. R. Helmrich, M. Siekmann, S. Becker, S. Bosse, D. Marpe, and T. Wiegand, "XPSNR: A Low-Complexity Extension of the Perceptually Weighted Peak Signal-to-Noise Ratio for High-Resolution Video Quality Assessment," in Proc. IEEE Int. Conf. Acoustics, Speech, Sig. Process. (ICASSP), virt./online, May 2020. www.ecodis.de/xpsnr.htm

C. R. Helmrich, S. Bosse, H. Schwarz, D. Marpe, and T. Wiegand, "A Study of the Extended Perceptually Weighted Peak Signal-to-Noise Ratio (XPSNR) for Video Compression with Different Resolutions and Bit Depths," ITU Journal: ICT Discoveries, vol. 3, no. 1, pp. 65 - 72, May 2020. http://handle.itu.int/11.1002/pub/8153d78b-en

When publishing the results of XPSNR assessments obtained using, e.g., this FFmpeg filter, a reference to the above papers as a means of documentation is strongly encouraged. The filter requires two input videos. The first input is considered a (usually not distorted) reference source and is passed unchanged to the output, whereas the second input is a (distorted) test sig-

nal. Except for the bit depth, these two video inputs must have the same pixel format. In addition, for best performance, both compared input videos should be in YCbCr color format. The obtained overall XPSNR values mentioned above are printed through the logging system. In case of input with multiple color planes, we suggest reporting of the minimum XPSNR average. The following parameter, which behaves like the one for the psnr filter, is accepted: <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified, the filter will use the named file to save the XPSNR value of each individual frame and color plane. When the file name equals "-", that data is sent to standard output. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">11.292.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> XPSNR analysis of two 1080p HD videos, ref_source.yuv and test_video.yuv, both at 24 frames per second, with color format 4:2:0, bit depth 8, and output of a logfile named "xpsnr.log": <pre class="example-preformatted">ffmpeg -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i ref_source.yuv -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i test_video.yuv -lavfi xpsnr="stats_file=xpsnr.log" -f null - </pre>

XPSNR analysis of two 2160p UHD videos, ref_source.yuv with bit depth 8 and test_video.yuv with bit depth 10, both at 60 frames per second with color format 4:2:0, no logfile output: <pre class="example-preformatted">ffmpeg -s 3840x2160 -framerate 60 -pix_fmt yuv420p -i ref_source.yuv -s 3840x2160 -framerate 60 -pix_fmt yuv420p10le -i test_video.yuv -lavfi xpsnr="stats_file=-" -f null - </pre>

 <h3 class="section">11.293 xstack# TOC</h3> Stack video inputs into custom layout. All streams must be of same pixel format. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">layout</samp></dt> <dd> Specify layout of inputs. This option requires the desired layout configuration to be explicitly set by the user. This sets position of each video input in output. Each input is separated by ’|’. The first number represents the column, and the second number represents the row. Numbers start at 0 and are separated by ’_’. Optionally one can use wX and hX, where

X is video input from which to take width or height. Multiple values can be used when separated by ’+’. In such case values are summed together. Note that if inputs are of different sizes gaps may appear, as not all of the output video frame will be filled. Similarly, videos can overlap each other if their position doesn’t leave enough space for the full frame of adjoining videos. For 2 inputs, a default layout of <code class="code">0_0|w0_0</code> (equivalent to <code class="code">grid=2x1</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">grid</samp></dt> <dd> Specify a fixed size grid of inputs. This option is used to create a fixed size grid of the input streams. Set the grid size in the form <code class="code">COLUMNSxROWS</code>. There must be <code class="code">ROWS * COLUMNS</code> input streams and they will be arranged as a grid with <code class="code">ROWS</code> rows and <code class="code">COLUMNS</code> columns. When using this option, each input stream within a row must have the same height and all the rows must have the same width. If <code class="code">grid</code> is set, then <code class="code">inputs</code> option is ignored and is implicitly set to <code class="code">ROWS * COLUMNS</code>. For 2 inputs, a default grid of <code class="code">2x1</code> (equivalent to <code class="code">layout=0_0|w0_0</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">fill</samp></dt> <dd> If set to valid color, all unused pixels will be filled with that color. By default fill is set to none, so it is disabled. </dd> </dl>

 <h4 class="subsection">11.293.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Display 4 inputs into 2x2 grid.

Layout: <pre class="example-preformatted">input1(0, 0) | input3(w0, 0) input2(0, h0) | input4(w0, h0) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|w0_0|w0_h
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 4 inputs into 1x4 grid.

Layout: <pre class="example-preformatted">input1(0,

0) input2(0, h0) input3(0, h0+h1) input4(0, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|0_h0+h1|0_
</pre>

Note that if inputs are of different widths, unused space will appear.
Display 9 inputs into 3x3 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input4(w0, 0) | input7(w0+w3, 0) input2(0, h0) | input5(w0, h0) | in-
put8(w0+w3, h0) input3(0, h0+h1) | input6(w0, h0+h1) | input9(w0+w3,
h0+h1) </pre>

 <pre class="example-preformatted">xstack=inputs=9:layout=0_0|0_h0|0_h0+h1|w
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 16 inputs into 4x4 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input5(w0, 0) | input9 (w0+w4, 0) | input13(w0+w4+w8, 0) input2(0, h0)
| input6(w0, h0) | input10(w0+w4, h0) | input14(w0+w4+w8, h0) input3(0,
h0+h1) | input7(w0, h0+h1) | input11(w0+w4, h0+h1) | input15(w0+w4+w8,
h0+h1) input4(0, h0+h1+h2)| input8(w0, h0+h1+h2)| input12(w0+w4,
h0+h1+h2)| input16(w0+w4+w8, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=16:layout=0_0|0_h0|0_h0+h1|
w0+w4_h0|w0+w4_h0+h1|w0+w4_h0+h1+h2|w0+w4+w8_0|w0+w4+w8_h0|w0+w4+w8_h0+h1|w0+w4-
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.

 <h3
class="section">11.294 yadif<a class="anchor
hidden-xs" href="#yadif" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-yadif" aria-hidden="true">TOC</h3>

Deinterlace the input video ("yadif" means "yet another
deinterlacing filter"). It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt>
<dd> The interlacing mode to adopt. It accepts one of the follow-
ing values: <dl class="table"> <dt><samp class="option">0,
send_frame</samp></dt> <dd> Output one frame for each frame.
</dd> <dt><samp class="option">1, send_field</samp></dt>
<dd> Output one frame for each field. </dd> <dt><samp
class="option">2, send_frame_nospatial</samp></dt> <dd> Like
<code class="code">send_frame</code>, but it skips the spatial interlacing
check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt>

<dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">11.295 yaepblur# TOC</h3>

Apply blur filter while preserving edges ("yaepblur" means "yet another edge preserving blur filter"). The algorithm is described in "J. S. Lee, Digital image enhancement and noise filtering by use of local statistics, IEEE Trans. Pattern Anal. Mach. Intell. PAMI-2, 1980." It accepts the following parameters: <dl class="table"> <dt><samp class="option">radius, r</samp></dt> <dd> Set the window radius. Default value is 3. </dd> <dt><samp class="option">planes, p</samp></dt> <dd> Set which planes to filter. Default is only the first plane. </dd> <dt><samp class="option">sigma, s</samp></dt> <dd> Set blur strength. Default value is 128. </dd> </dl>

 <h4 class="subsection">11.295.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">11.296 zoompan<a class="anchor hidden-xs" href="#zoompan"

aria-hidden=”true”># TOC</h3>

Apply Zoom & Pan effect. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>zoom, z</samp></dt> <dd> Set the zoom expression. Range is 1-10. Default is 1. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the x and y expression. Default is 0. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Set the duration expression in number of frames. This sets for how many number of frames effect will last for single input image. Default is 90. </dd> <dt><samp class=”option”>s</samp></dt> <dd> Set the output image size, default is ’hd720’. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set the output frame rate, default is ’25’. </dd> </dl>

Each expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> Input width. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> Input height. </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dd> Output width. </dd> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> Output height. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> <dt><samp class=”option”>in_time, it</samp></dt> <dd> The input timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>out_time, time, ot</samp></dt> <dd> The output timestamp expressed in seconds. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Last calculated ’x’ and ’y’ position from ’x’ and ’y’ expression for current input frame. </dd> <dt><samp class=”option”>px</samp></dt> <dt><samp class=”option”>py</samp></dt> <dd> ’x’ and ’y’ of last output frame of previous input frame or 0 when there was not yet such frame (first input frame). </dd> <dt><samp class=”option”>zoom</samp></dt> <dd> Last calculated zoom from ’z’ expression for current input frame. </dd> <dt><samp class=”option”>pzoom</samp></dt> <dd> Last calculated zoom of last output frame of previous input frame. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> Number of output frames for current input frame. Calculated from ’d’ expression for each input frame. </dd> <dt><samp class=”option”>pduration</samp></dt> <dd> number of output frames created for previous input frame </dd> <dt><samp class=”option”>a</samp></dt> <dd> Rational number: input width / input height </dd> <dt><samp class=”option”>sar</samp></dt>

<dd> sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt>
<dd> display aspect ratio </dd> </dl>

 <h4 class="subsection">11.296.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Zoom in up to 1.5x and pan at same time to some spot near center of picture: <pre class="example-preformatted">zoompan=z='min(zoom+0.0015,1.5)':d=700:x='if(gte(zoom,1.5),x,x+1/a)':y=
</pre>

Zoom in up to 1.5x and pan always at center of picture: <pre class="example-preformatted">zoompan=z='min(zoom+0.0015,1.5)':d=700:x='iw/2
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Same as above but without pausing: <pre class="example-preformatted">zoompan=z='min(max(zoom,pzoom)+0.0015,1.5)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Zoom in 2x into center of picture only for the first second of the input video: <pre class="example-preformatted">zoompan=z='if(between(in_time,0,1),2,1)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

 <h3 class="section">11.297 zscale# TOC</h3>
Scale (resize) the input video, using the z.lib library: https://github.com/sekrit-twc/zimg. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libzimg</code>. The zscale filter forces the output display aspect ratio to be the same as the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the zscale filter will convert the input to the requested format. <h4 class="subsection">11.297.1 Options# TOC</h4> The filter accepts the following options. <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class="var">width</var> or <var class="var">w</var> value is 0, the input width is used for the output. If the

<var class=”var”>height</var> or <var class=”var”>h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the zscale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>dither, d</samp></dt> <dd> Set the dither type. Possible values are: <dl class=”table”> <dt><var class=”var”>none</var></dt> <dt><var class=”var”>ordered</var></dt> <dt><var class=”var”>random</var></dt> <dt><var class=”var”>error_diffusion</var></dt> </dl>

Default is none. </dd> <dt><samp class=”option”>filter, f</samp></dt> <dd> Set the resize filter type. Possible values are: <dl class=”table”> <dt><var class=”var”>point</var></dt> <dt><var class=”var”>bilinear</var></dt> <dt><var class=”var”>bicubic</var></dt> <dt><var class=”var”>spline16</var></dt> <dt><var class=”var”>spline36</var></dt> <dt><var class=”var”>spline64</var></dt> <dt><var class=”var”>lanczos</var></dt> </dl>

Default is bilinear. </dd> <dt><samp class=”option”>range, r</samp></dt> <dd> Set the color range. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>limited</var></dt> <dt><var class=”var”>full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>primaries, p</samp></dt> <dd> Set the color primaries. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>170m</var></dt> <dt><var class=”var”>240m</var></dt> <dt><var class=”var”>2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>transfer, t</samp></dt> <dd> Set the transfer characteristics. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>601</var></dt> <dt><var class=”var”>linear</var></dt> <dt><var class=”var”>2020_10</var></dt> <dt><var class=”var”>2020_12</var></dt> <dt><var class=”var”>smpte2084</var></dt> <dt><var class=”var”>iec61966-2-1</var></dt> <dt><var class=”var”>arib-std-b67</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">rangein, rin</samp></dt> <dd> Set the input color range. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">limited</var></dt> <dt><var class="var">full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primariesin, pin</samp></dt> <dd> Set the input color primaries. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">240m</var></dt> <dt><var class="var">2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transferin, tin</samp></dt> <dd> Set the input transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">601</var></dt> <dt><var class="var">linear</var></dt> <dt><var class="var">2020_10</var></dt> <dt><var class="var">2020_12</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrixin, min</samp></dt> <dd> Set the input colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

</dd> <dt><samp class="option">chromal, c</samp></dt> <dd> Set the output chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt> <dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt> <dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt> </dl>

</dd> <dt><samp class="option">chromalin, cin</samp></dt> <dd> Set the input chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt>

<dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt>
<dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt>
</dl>

</dd> <dt><samp class="option">npl</samp></dt> <dd> Set the nominal peak luminance. </dd> <dt><samp class="option">param_a</samp></dt>
<dd> Parameter A for scaling filters. Parameter "b" for bicubic, and the number of filter taps for lanczos. </dd> <dt><samp class="option">param_b</samp></dt> <dd> Parameter B for scaling filters. Parameter "c" for bicubic. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt>
<dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd>
<dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt>
<dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt>
<dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt>
<dd> The same as <var class="var">iw</var> / <var class="var">ih</var>
 </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt>
<dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt>
<dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt>
<dt><var class="var">ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">11.297.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h2 class="chapter">12 CUDA Video Filters# TOC</h2>

Below is a description of the currently available Nvidia CUDA video filters. Prerequisites: <ul class="itemize mark-bullet"> Install Nvidia CUDA Toolkit

Note: If FFmpeg detects the Nvidia CUDA Toolkit during configuration, it will enable CUDA filters automatically without requiring any additional flags. If you want to explicitly enable them, use the following options: <ul class="itemize mark-bullet"> Configure FFmpeg with <code class="code">--enable-cuda-nvcc --enable-nonfree</code>. Configure FFmpeg with <code class="code">--enable-cuda-llvm</code>. Additional requirement: <code class="code">llvm</code> lib must be installed.

Running CUDA filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device cuda[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">cuda</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of initializing second CUDA device on the system and running scale_cuda and bilateral_cuda filters. <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -init_hw_device cuda:1 -filter_complex \ "[0:v]scale_cuda=format=yuv444p[scaled_video];[sca
\ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

Since CUDA filters operate exclusively on GPU memory, frame data must sometimes be uploaded (hwupload) to hardware surfaces associated with the appropriate CUDA device before processing, and downloaded (hwdownload) back to normal memory afterward, if required. Whether hwupload or hwdownload is necessary depends on the specific workflow: <ul class="itemize mark-bullet"> If the input frames are already in GPU memory (e.g., when using <code class="code">-hwaccel cuda</code> or <code class="code">-hwaccel_output_format cuda</code>), explicit use of <a class="ref"

href=”#hwupload”>hwupload is not needed, as the data is already in the appropriate memory space. If the input frames are in CPU memory (e.g., software-decoded frames or frames processed by CPU-based filters), it is necessary to use hwupload to transfer the data to GPU memory for CUDA processing. If the output of the CUDA filters needs to be further processed by software-based filters or saved in a format not supported by GPU-based encoders, hwdownload is required to transfer the data back to CPU memory. Note that hwupload uploads data to a surface with the same layout as the software frame, so it may be necessary to add a format filter immediately before hwupload to ensure the input is in the correct format. Similarly, hwdownload may not support all output formats, so an additional format filter may need to be inserted immediately after hwdownload in the filter graph to ensure compatibility. <h3 class=”section”>12.1 bilateral_cuda# TOC</h3> CUDA accelerated bilateral filter, an edge preserving filter. This filter is mathematically accurate thanks to the use of GPU acceleration. For best output quality, use one to one chroma subsampling, i.e. yuv444p format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight, also called sigma space. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate color range weight, also called sigma color. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>window_size</samp></dt> <dd> Set window size of the bilateral function to determine the number of neighbours to loop on. If the number entered is even, one will be added automatically. Allowed range is 1 to 255. Default is 1. </dd> </dl> <h4 class=”subsection”>12.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the bilateral filter on a video.

 <pre class=”example-preformatted”>./ffmpeg -v verbose \ -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]scale_cuda=format=yuv444p[scaled_video]; [scaled_video]bilateral_cuda=window_size=9:sigmaS=3.0:sigmaR=50.0" \ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

 <h3 class="section">12.2 bwdif_cuda# TOC</h3>

Deinterlace the input video using the bwdif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">12.3 chromakey_cuda# TOC</h3> CUDA accelerated YUV colorspace color/chroma keying. This filter works like normal chromakey filter but operates on CUDA frames. for more details and parameters see chromakey. <h4 class="subsection">12.3.1 Examples<span class="pull-

right”>#
TOC</h4>

<ul class=”itemize mark-bullet”> Make all the green pixels in the input video transparent and use it as an overlay for another video:

 <pre class=”example-preformatted”>./ffmpeg \
-hwaccel cuda -hwaccel_output_format cuda -i input_green.mp4 \ -hwaccel cuda -hwaccel_output_format cuda -i base_video.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]chromakey_cuda=0x25302D:0.1:0.12:1[overlay_video]; \ [1:v]scale_cuda=format=yuv420p[base]; \ [base][overlay_video]overlay_cuda" \ -an -sn -c:v h264_nvenc -cq 20 output.mp4 </pre>

Process two software sources, explicitly uploading the frames:

 <pre class=”example-preformatted”>./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda \ -f lavfi -i color=size=800x600:color=white,format=yuv42 \ -f lavfi -i yuvtestsrc=size=200x200,format=yuv420p \ -filter_complex \ " \ [0]hwupload[under]; \ [1]hwupload,chromakey_cuda=green:0.1:0.12[over]; \ [under][over]overlay_cuda" \ -c:v hevc_nvenc -cq 18 -preset slow output.mp4 </pre>

 <h3 class=”section”>12.4 colorspace_cuda# TOC</h3>

CUDA accelerated implementation of the colorspace filter. It is by no means feature complete compared to the software colorspace filter, and at the current time only supports color range conversion between jpeg/full and mpeg/limited range. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> </dl>

 <h3 class=”section”>12.5 overlay_cuda# TOC</h3>

Overlay one video on top of another. This is the CUDA variant of the overlay filter. It only accepts CUDA frames. The underlying input pixel formats have to match. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. They can contain the following parameters: <dl class=”table”> <dt><samp class=”option”>main_w, W</samp></dt> <dt><samp class=”option”>main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class=”option”>overlay_w, w</samp></dt> <dt><samp class=”option”>overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The ordinal index of the main input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> The byte offset position in the file of the main input frame, NAN if unknown. Deprecated, do not use. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp of the main input frame, expressed in seconds, NAN if unknown. </dd> </dl>

Default value is "0" for both expressions. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for <samp class=”option”>x</samp> and <samp class=”option”>y</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>init</samp></dt> <dd> Evaluate expressions once during filter initialization or when a command is processed. </dd> <dt><samp class=”option”>frame</samp></dt> <dd> Evaluate expressions for each incoming frame </dd> </dl>

Default value is <samp class=”option”>frame</samp>. </dd> <dt><samp class=”option”>eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options.

 <h3 class=”section”>12.6 pad_cuda<a

class=”anchor hidden-xs” href=”#pad_005fcuda” aria-hidden=”true”>#
TOC</h3>
Add paddings to an input video stream using CUDA. This filter is the CUDA-accelerated version of the pad filter. It accepts the same options and expressions and provides the same core functionality. For a detailed description of available options, please see the documentation for the pad filter. <h4 class=”subsection”>12.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Add a 200-pixel black border to all sides of a video frame: <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=iw+400:h=ih+400:x=200:y=200" -c:v h264_nvenc out.mp4 </pre>

Pad the input video to a 16:9 aspect ratio, filling with the color "blue": <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=ih*16/9/sar:h=ih:x=(ow-iw)/2:y=(oh-ih)/2:color=blue" -c:v h264_nvenc out.mp4 </pre>

 <h3 class=”section”>12.7 scale_cuda#
TOC</h3>

Scale (resize) and convert (pixel format) the input video, using accelerated CUDA kernels. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>interp_algo</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>nearest</var></dt> <dd> Nearest neighbour Used by default if input parameters match the desired output. </dd> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic This is the default. </dd> <dt><var class=”var”>lanczos</var></dt> <dd> High-quality separable Lanczos (windowed sinc). The filter radius can be set with <samp class=”option”>param</samp>; the default is 3. </dd> </dl>

</dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. Converting between <code class="code">yuv420p</code>, <code class="code">yuv422p</code>, <code class="code">yuv444p</code>, <code class="code">yuv420p10</code>, <code class="code">yuv422p10</code>, <code class="code">yuv444p10</code>, <code class="code">nv12</code>, <code class="code">nv16</code>, <code class="code">p010</code> or <code class="code">p210</code> and <code class="code">bgra</code>, <code class="code">rgba</code>, <code class="code">bgr0</code> or <code class="code">rgb0</code> is supported, also together with resizing. Resizing is then done on the RGB side, after converting from YUV or before converting to YUV, and always uses the generic filters selected with <samp class="option">interp_algo</samp>. Resizing the YUV side in a separate instance instead needs less intermediate memory and may be faster, but filters chroma differently. YUV to RGB conversion uses the YCbCr matrix and range of the input, RGB to YUV conversion those selected for the output (see <samp class="option">out_color_matrix</samp> and <samp class="option">out_range</samp>). An unspecified matrix is treated as BT.601 and an unspecified range as limited. The conversion samples chroma independently of <samp class="option">interp_algo</samp>: YUV to RGB conversion uses the nearest chroma sample, RGB to YUV conversion averages, which centers the chroma samples. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> If set to 0, every frame is processed, even if no conversion is necessary. This mode can be useful to use the filter as a buffer for a downstream frame-consumer that exhausts the limited decoder frame pool. If set to 1, frames are passed through as-is if they match the desired output parameters. This is the default behaviour. </dd> <dt><samp class="option">use_filters</samp></dt> <dd> If set to 1, filter with a generic weight LUT instead of using fixed-function shader kernels. May be faster or slower depending on the hardware. A value of <code class="code">auto</code> (the default) enables this automatically when required for correct anti-aliasing when downscaling, and for high-quality Lanczos scaling. Set this to 0 to use the legacy fixed 4-tap Lanczos kernel. Resizing during YUV to RGB or RGB to YUV conversion always uses the generic filters. </dd> <dt><samp class="option">param</samp></dt> <dd> Algorithm-specific parameter. For <var class="var">bicubic</var>, this affects the filter curve. For <var class="var">lanczos</var>, this sets the filter radius from 1 to 10 (default 3) when using the generic filter path; it has no effect with <samp class="option">use_filters=0</samp>. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class="option">out_range</samp></dt>

<dd> Set the output YCbCr sample range. One of <code class="code">auto</code> (the default), <code class="code">jpeg</code>/<code class="code">full</code>/<code class="code">pc</code>, or <code class="code">mpeg</code>/<code class="code">limited</code>/<code class="code">tv</code>. </dd> <dt><samp class="option">out_color_matrix</samp></dt> <dd> Set the output YCbCr matrix. One of <code class="code">auto</code> (the default), <code class="code">bt601</code>, <code class="code">smpte170m</code>, <code class="code">bt470</code>, <code class="code">bt470bg</code>, <code class="code">bt709</code>, <code class="code">fcc</code>, <code class="code">smpte240m</code>, <code class="code">ycgco</code>, <code class="code">bt2020</code> or <code class="code">bt2020nc</code>. With <code class="code">auto</code>, <samp class="option">out_range</samp> and <samp class="option">out_color_matrix</samp> are left to filter-graph negotiation. They follow a single range and matrix negotiated for the input, as for decoded video, and otherwise prefer BT.601 limited range. For RGB to YUV conversion, <code class="code">auto</code> thus means BT.601 limited range, unless the filter graph requires otherwise; converting full range YUV to RGB and back keeps the full range. Outside RGB to YUV conversion, these options describe the existing samples without remapping them; an unspecified input description may adopt the negotiated output description, while conflicting specified descriptions are rejected. Both options can only be used with YUV output; RGB output is full range. </dd> </dl>

 <h4 class="subsection">12.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. <pre class="example-preformatted">scale_cuda=-2:720:format=yuv420p </pre>

Upscale to 4K using nearest neighbour algorithm. <pre class="example-preformatted">scale_cuda=4096:2160:interp_algo=nearest </pre>

Don’t do any conversion or scaling, but copy all input frames into newly allocated ones. This can be useful to deal with a filter and encode chain that otherwise exhausts the decoders frame pool. <pre class="example-preformatted">scale_cuda=passthrough=0 </pre>

Convert BT.709 input to RGB for a filter that requires it, then back to BT.709 NV12 with limited range. <pre class="example-preformatted">scale_cuda=format=bgra,scale_cuda=format=nv12:out_color_matrix=bt70 </pre>

Scale to 720p while converting to RGB.

<pre class="example-preformatted">scale_cuda=-2:720:format=bgra
</pre>

 <h3 class="section">12.8 thumbnail_cuda# TOC</h3>

Select the most representative frame in a given sequence of consecutive frames using CUDA. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">12.8.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Thumbnails are extracted from every <var class="var">n</var>=150-frame batch, selecting one per batch. Chosen frames are then scaled with scale_cuda.
 <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i ./input.mp4 -vf "thumbnail_cuda=150,scale_cuda=1920:
./output/out%03d.png </pre>

 <h3 class="section">12.9 transpose_cuda# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt>

<dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class="samp">reversal</samp>’</dt> <dd> Rotate by 180 degrees. </dd> <dt>‘<samp class="samp">hflip</samp>’</dt> <dd> Flip horizontally. </dd> <dt>‘<samp class="samp">vflip</samp>’</dt> <dd> Flip vertically. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">12.10 yadif_cuda# TOC</h3>

Deinterlace the input video using the yadif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt> <dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1,

auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h2 class="chapter">13 OpenCL Video Filters# TOC</h2>

Below is a description of the currently available OpenCL video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-opencl</code>. Running OpenCL filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device opencl[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</samp> <dd> Initialise a new hardware device of type <var class="var">opencl</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device on the second platform and running avgblur_opencl filter with default parameters on it. <pre class="example-preformatted">-init_hw_device opencl=gpu:1.0 -filter_hw_device gpu -i INPUT -vf "hwupload, avgblur_opencl, hwdownload" OUTPUT </pre>

Since OpenCL filters are not able to access frame data in normal memory, all frame data needs to be uploaded(hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded(hwdownload) back to normal memory. Note that hwupload will upload to a surface with the same layout as the software frame, so it

may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it may be necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class=”section”>13.1 avgblur_opencl# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>0</code>. If zero, <code class=”code”>sizeX</code> value will be used. </dd> </dl>

 <h4 class=”subsection”>13.1.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply average blur filter with horizontal and vertical size of 3, setting each pixel of the output to the average value of the 7x7 region centered on it in the input. For pixels on the edges of the image, the region does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, avgblur_opencl=3, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.2 boxblur_opencl# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class="code">min(w,h)/2</code> for the luma and alpha planes, and of <code class="code">min(cw,ch)/2</code> for the chroma planes. Default value for <samp class="option">luma_radius</samp> is "2". If not specified, <samp class="option">chroma_radius</samp> and <samp class="option">alpha_radius</samp> default to the corresponding value set for <samp class="option">luma_radius</samp>. The expressions can contain the following constants: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class="option">cw</samp></dt> <dt><samp class="option">ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class="option">luma_power, lp</samp></dt> <dt><samp class="option">chroma_power, cp</samp></dt> <dt><samp class="option">alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class="option">luma_power</samp> is 2. If not specified, <samp class="option">chroma_power</samp> and <samp class="option">alpha_power</samp> default to the corresponding value set for <samp class="option">luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class="subsection">13.2.1 Examples# TOC</h4>

Apply boxblur filter, setting each pixel of the output to the average value of box-radiuses <var class="var">luma_radius</var>, <var class="var">chroma_radius</var>, <var class="var">alpha_radius</var> for each plane respectively. The filter will apply <var class="var">luma_power</var>, <var class="var">chroma_power</var>, <var class="var">alpha_power</var> times onto the corresponding plane. For pixels on the edges of the image, the radius does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <ul class="itemize mark-bullet"> Apply a boxblur filter with the luma, chroma, and

alpha radius set to 2 and luma, chroma, and alpha power set to 3. The filter will run 3 times with box-radius set to 2 for every plane of the image. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=luma_radius=2:luma_power=3, hwdownload" OUTPUT -i INPUT -vf "hwupload, boxblur_opencl=2:3, hwdownload" OUTPUT </pre>

Apply a boxblur filter with luma radius set to 2, luma_power to 1, chroma_radius to 4, chroma_power to 5, alpha_radius to 3 and alpha_power to 7.

For the luma plane, a 2x2 box radius will be run once. For the chroma plane, a 4x4 box radius will be run 5 times. For the alpha plane, a 3x3 box radius will be run 7 times. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=2:1:4:5:3:7, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.3 colorkey_opencl# TOC</h3>
RGB colorspace color keying. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt>
<dd> The color which will be replaced with transparency. </dd>
<dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage.
0.0 makes pixels either fully transparent, or not transparent at all.
Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h4 class=”subsection”>13.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make every semi-green pixel in the input transparent with some slight blending:
<pre class=”example-preformatted”>-i INPUT -vf "hwupload, colorkey_opencl=green:0.3:0.1, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.4 convolution_opencl# TOC</h3

Apply convolution of 3x3, 5x5, 7x7 matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>0m</samp></dt> <dt><samp class=”option”>1m</samp></dt> <dt><samp class=”option”>2m</samp></dt> <dt><samp class=”option”>3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed numbers. Default value for each plane is <code class=”code”>0 0 0 0 1 0 0 0 0</code>. </dd> <dt><samp class=”option”>0rdiv</samp></dt> <dt><samp class=”option”>1rdiv</samp></dt> <dt><samp class=”option”>2rdiv</samp></dt> <dt><samp class=”option”>3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be sum of all matrix elements. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>0bias</samp></dt> <dt><samp class=”option”>1bias</samp></dt> <dt><samp class=”option”>2bias</samp></dt> <dt><samp class=”option”>3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>13.4.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sharpen: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0, hwdownload" OUTPUT </pre>

Apply blur: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9, hwdownload" OUTPUT </pre>

Apply edge enhance: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply edge detect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1

1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0, hwdownload" OUTPUT </pre>

Apply emboss: <pre class="example-preformatted">-i INPUT -vf "hwupload, convolution_opencl=-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2, hwdownload" OUTPUT </pre>

 <h3 class="section">13.5 erosion_opencl# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">13.5.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply erosion filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local minimum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local minimum is more then threshold of the corresponding plane, output pixel will be set to input pixel - threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, erosion_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">13.6 deshake_opencl# TOC</h3> Feature-point based video stabilization filter. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>tripod</samp></dt> <dd> Simulates a tripod by preventing any camera movement whatsoever from the original frame. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>debug</samp></dt> <dd> Whether or not additional debug info should be displayed, both in the processed output and in the console. Note that in order to see console debug output you will also need to pass <code class=”code”>-v verbose</code> to ffmpeg. Viewing point matches in the output video is only supported for RGB input. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>adaptive_crop</samp></dt> <dd> Whether or not to do a tiny bit of cropping at the borders to cut down on the amount of mirrored pixels. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>refine_features</samp></dt> <dd> Whether or not feature points should be refined at a sub-pixel level. This can be turned off for a slight performance gain at the cost of precision. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>smooth_strength</samp></dt> <dd> The strength of the smoothing applied to the camera path from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. <code class=”code”>1.0</code> is the maximum smoothing strength while values less than that result in less smoothing. <code class=”code”>0.0</code> causes the filter to adaptively choose a smoothing strength on a per-frame basis. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>smooth_window_multiplier</samp>
<dd> Controls the size of the smoothing window (the number of frames buffered to determine motion information from). The size of the smoothing window is determined by multiplying the framerate of the video by this number. Acceptable values range from <code class=”code”>0.1</code> to <code class=”code”>10.0</code>. Larger values increase the amount of motion data available for determining how to smooth the camera path, potentially improving smoothness, but also increase latency and memory usage. Defaults to <code class=”code”>2.0</code>. </dd> </dl>

 <h4 class=”subsection”>13.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Stabilize a video with a fixed, medium smoothing strength: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, deshake_opencl=smooth_strength=0.5, hwdownload" OUTPUT </pre>

Stabilize a video with debugging (both in console and in rendered video): <pre class=”example-preformatted”>-i INPUT -filter_complex "[0:v]format=rgba, hwupload, deshake_opencl=debug=1,

hwdownload, format=rgba, format=yuv420p" -v verbose OUTPUT </pre>

 <h3 class="section">13.7 dilation_opencl# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">13.7.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply dilation filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local maximum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local maximum is more then threshold of the corresponding plane, output pixel will be set to input pixel + threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, dilation_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">13.8 nlmeans_opencl# TOC</h3>

Non-local Means denoise filter through OpenCL, this filter accepts same options as nlmeans. <h3 class="section">13.9 overlay_opencl<a class="anchor hidden-xs"

href=”#overlay_005fopencl” aria-hidden=”true”># TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires same memory layout for all the inputs. So, format conversion may be needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set the x coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>13.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs are yuv420p format. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre> The inputs have same memory layout for color channels , the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.10 pad_opencl# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class=”var”>x</var>, <var class=”var”>y</var> coordinates. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt>

<dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var>, and <var class=”var”>y</var> options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> </dl>

 <h3 class=”section”>13.11 prewitt_opencl# TOC</h3>

Apply the Prewitt operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Prewitt_operator”>http
to input video stream. The filter accepts the following option:

 <dl class="table"> <dt><samp class="option">planes</samp></dt>
<dd> Set which planes to filter. Default value is <code class="code">0xf</code>,
by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt>
<dd> Set value which will be multiplied with filtered result. Range is
<code class="code">[0.0, 65535]</code> and default value is <code
class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt>
<dd> Set value which will be added to filtered result. Range is
<code class="code">[-65535, 65535]</code> and default value is <code
class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">13.11.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply the Prewitt operator with scale
set to 2 and delta set to 10. <pre class="example-
preformatted">-i INPUT -vf "hwupload, prewitt_opencl=scale=2:delta=10,
hwdownload" OUTPUT </pre>

 <h3 class="section">13.12 program_opencl# TOC</h3>

Filter video using an OpenCL program. <dl class="table">
<dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt>
<dd> Kernel name in program. </dd> <dt><samp class="option">inputs</samp></dt>
<dd> Number of inputs to the filter. Defaults to 1. </dd>
<dt><samp class="option">size, s</samp></dt> <dd> Size of output
frames. Defaults to the same as the first input. </dd> </dl>

The <code class="code">program_opencl</code> filter also supports the
framesync options. The
program source file must contain a kernel function with the given name, which
will be run once for each plane of the output. Each run on a plane gets enqueued
as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a
pixel in the destination image. The kernel function needs to take the
following arguments: <ul class="itemize mark-bullet"> Destination
image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it.
Frame index, <var class="var">unsigned int</var>.

This is a counter starting from zero and increasing by one for each
frame. Source images, <var class="var">__read_only

image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to.

Example programs: <ul class="itemize mark-bullet"> Copy the input to the output (output must be the same size as the input). <pre class="verbatim">___kernel void copy(___write_only image2d_t destination, unsigned int index, ___read_only image2d_t source) { const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE;

int2 location = (int2)(get_global_id(0), get_global_id(1));

float4 value = read_imagef(source, sampler, location);

write_imagef(destination, location, value); } </pre> Apply a simple transformation, rotating the input by an amount increasing with the index counter. Pixel values are linearly interpolated by the sampler, and the output need not have the same dimensions as the input. <pre class="verbatim">___kernel void rotate_image(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float angle = (float)index / 100.0f;

float2 dst_dim = convert_float2(get_image_dim(dst)); float2 src_dim = convert_float2(get_image_dim(src));

float2 dst_cen = dst_dim / 2.0f; float2 src_cen = src_dim / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1));

float2 dst_pos = convert_float2(dst_loc) - dst_cen; float2 src_pos = { cos(angle) * dst_pos.x - sin(angle) * dst_pos.y, sin(angle) * dst_pos.x + cos(angle) * dst_pos.y }; src_pos = src_pos * src_dim / dst_dim;

float2 src_loc = src_pos + src_cen;

if (src_loc.x < 0.0f || src_loc.y < 0.0f || src_loc.x > src_dim.x || src_loc.y > src_dim.y) write_imagef(dst, dst_loc, 0.5f); else write_imagef(dst, dst_loc, read_imagef(src, sampler, src_loc)); } </pre> Blend two inputs together, with the amount of each input used varying with the index counter. <pre class="verbatim">___kernel void blend_images(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src1, ___read_only image2d_t src2) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float blend = (cos((float)index / 50.0f) + 1.0f) / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1)); int2 src1_loc = dst_loc * get_image_dim(src1) / get_image_dim(dst); int2 src2_loc = dst_loc * get_image_dim(src2) / get_image_dim(dst);

float4 val1 = read_imagef(src1, sampler, src1_loc); float4 val2 = read_imagef(src2, sampler, src2_loc);

write_imagef(dst, dst_loc, val1 * blend + val2 * (1.0f - blend)); } </pre>

 <h3 class="section">13.13 remap_opencl# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 32bit float pixel format, single channel. <dl class="table"> <dt><samp class="option">interp</samp></dt> <dd> Specify interpolation used for remapping of pixels. Allowed values are <code class="code">near</code> and <code class="code">linear</code>. Default value is <code class="code">linear</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">13.14 roberts_opencl# TOC</h3> Apply the Roberts cross operator (<a class="url" href="https://en.wikipedia.org/wiki/Roberts_cross">https://en.w to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">0xf</code>, by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class="code">[0.0, 65535]</code> and default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class="code">[-65535, 65535]</code> and default value is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">13.14.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the Roberts cross operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, roberts_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.15 sobel_opencl# TOC</h3>

Apply the Sobel operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Sobel_operator”>https://
to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>13.15.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sobel operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, sobel_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.16 tonemap_opencl# TOC</h3>

Perform HDR(PQ/HLG) to SDR conversion with tone-mapping. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tonemap</samp></dt> <dd> Specify the tone-mapping operator to be used. Same as tonemap option in tonemap. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Tune the tone mapping algorithm. same as param option in tonemap. </dd> <dt><samp class=”option”>desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by

(smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default value is 0.5, and the algorithm here is a little different from the cpu version tonemap currently. A setting of 0.0 disables this option. </dd> <dt><samp class="option">threshold</samp></dt> <dd> The tonemapping algorithm parameters is fine-tuned per each scene. And a threshold is used to detect whether the scene has changed or not. If the distance between the current frame average brightness and the current running average exceeds a threshold value, we would re-calculate scene average and peak brightness. The default value is 0.2. </dd> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Currently supported formats are: <dl class="table"> <dt><var class="var">p010</var></dt> <dt><var class="var">nv12</var></dt> </dl>

</dd> <dt><samp class="option">range, r</samp></dt> <dd> Set the output color range. Possible values are: <dl class="table"> <dt><var class="var">tv/mpeg</var></dt> <dt><var class="var">pc/jpeg</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is bt709. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> </dl>

 <h4 class="subsection">13.16.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(PQ/HLG) video to bt2020-transfer-characteristic p010 format using linear operator. <pre class="example-preformatted">-i INPUT -vf "format=p010,hwupload,tonemap_opencl=t=bt2020:tonemap=linear:format=p010,hwdownload,format OUTPUT </pre>

 <h3 class="section">13.17 unsharp_opencl<a class="anchor hidden-xs"

href=”#unsharp_005fopencl” aria-hidden=”true”># TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_amount, la</samp></dt> <dd> Set the luma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>1.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> <dt><samp class=”option”>chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>0.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class=”subsection”>13.17.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply strong luma sharpen effect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=luma_msize_x=7:luma_msize_y=7:luma_amount=2.5, hwdownload" OUTPUT </pre>

Apply a strong blur of both luma and chroma parameters: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=7:7:-2:7:7:-2, hwdownload" OUTPUT </pre>

 <h3 class=”section”>13.18 xfade_opencl# TOC</h3>

Cross fade two videos with custom transition effect by using OpenCL.

 It accepts the following options: <dl class="table"> <dt><samp class="option">transition</samp></dt> <dd> Set one of possible transition effects. <dl class="table"> <dt><samp class="option">custom</samp></dt> <dd> Select custom transition effect, the actual transition description will be picked from source and kernel options. </dd> <dt><samp class="option">fade</samp></dt> <dt><samp class="option">wipeleft</samp></dt> <dt><samp class="option">wiperight</samp></dt> <dt><samp class="option">wipeup</samp></dt> <dt><samp class="option">wipedown</samp></dt> <dt><samp class="option">slideleft</samp></dt> <dt><samp class="option">slideright</samp></dt> <dt><samp class="option">slideup</samp></dt> <dt><samp class="option">slidedown</samp></dt> <dd> Default transition is fade. </dd> </dl>

</dd> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file for custom transition. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Set name of kernel to use for custom transition from program source file. </dd> <dt><samp class="option">duration</samp></dt> <dd> Set duration of video transition. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set time of start of transition relative to first video. </dd> </dl>

The program source file must contain a kernel function with the given name, which will be run once for each plane of the output. Each run on a plane gets enqueued as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a pixel in the destination image. The kernel function needs to take the following arguments: <ul class="itemize mark-bullet"> Destination image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it. First Source image, <var class="var">__read_only image2d_t</var>. Second Source image, <var class="var">__read_only image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to. Transition progress, <var class="var">float</var>. This value is always between 0 and 1 inclusive.

Example programs: <ul class="itemize mark-bullet"> Apply dots curtain transition effect: <pre class="verbatim">__kernel void blend_images(__write_only image2d_t dst, __read_only image2d_t src1, __read_only image2d_t src2, float progress) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR); int2 p = (int2)(get_global_id(0), get_global_id(1)); float2 rp = (float2)(get_global_id(0), get_global_id(1)); float2 dim = (float2)(get_image_dim(src1).x, get_image_dim(src1).y); rp = rp / dim;

float2 dots = (float2)(20.0, 20.0); float2 center = (float2)(0,0); float2 unused;

float4 val1 = read_imagef(src1, sampler, p); float4 val2 = read_imagef(src2, sampler, p); bool next = distance(fract(rp * dots, &unused), (float2)(0.5, 0.5)) < (progress / distance(rp, center));

write_imagef(dst, p, next ? val1 : val2); } </pre>

 <h2 class="chapter">14 VAAPI Video Filters# TOC</h2>

VAAPI Video filters are usually used with VAAPI decoder and VAAPI encoder. Below is a description of VAAPI video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vaapi</code>. To use vaapi filters, you need to setup the vaapi device correctly. For more information, please read <a class="url" href="https://trac.ffmpeg.org/wiki/Hardware/VAAPI">https://trac.ffmpeg.org/wiki/Hardware/
  <h3 class="section">14.1 overlay_vaapi# TOC</h3>

Overlay one video on the top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set expressions for the width and height the overlaid video on the main video. Default values are ’overlay_iw’ for ’w’ and ’overlay_ih*w/overlay_iw’ for ’h’. The expressions can contain the following parameters: <dl class="table"> <dt><samp class="option">main_w, W</samp></dt> <dt><samp class="option">main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class="option">overlay_iw</samp></dt> <dt><samp class="option">overlay_ih</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class="option">overlay_w, w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt> <dd> The overlay output width and height. </dd> <dt><samp class="option">overlay_x, x</samp></dt> <dt><samp class="option">overlay_y, y</samp></dt> <dd> Position of the overlay layer inside of main </dd> </dl>

</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set transparency of overlaid video. Allowed range is 0.0 to 1.0. Higher value

means lower transparency. Default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">14.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs for this filter are yuv420p format. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_vaapi" OUTPUT </pre> Overlay an image LOGO at the offset (200, 100) from the top-left corner of the INPUT video. The inputs have same memory layout for color channels, the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_vaapi=x=200:y=100:w=400:h=300:alpha=1.0, hwdownload, format=nv12" OUTPUT </pre>

 <h3 class="section">14.2 tonemap_vaapi# TOC</h3>

Perform HDR-to-SDR or HDR-to-HDR tone-mapping. It currently only accepts HDR10 as input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Default is nv12 for HDR-to-SDR tone-mapping and p010 for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping.

</dd> <dt><samp class="option">display</samp></dt> <dd> Set the output mastering display colour volume. It is given by a ’|’-separated list of two values, two values are space separated. It set display primaries x & y in G, B, R order, then white point x & y, the nominal minimum & maximum display luminances. HDR-to-HDR tone-mapping will be performed when this option is set. </dd> <dt><samp class="option">light</samp></dt> <dd> Set the output content light level information. It accepts 2 space-separated values, the first input is the maximum light level and the second input is the maximum average light level. It is ignored for HDR-to-SDR tone-mapping, and optional for HDR-to-HDR tone-mapping. </dd> </dl>

 <h4 class="subsection">14.2.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(HDR10) video to bt2020-transfer-characteristic p010 format <pre class="example-preformatted">tonemap_vaapi=format=p010:t=bt2020-10 </pre> Convert HDR video to HDR video <pre class="example-preformatted">tonemap_vaapi=display=7500\ 3000|34000\ 16000|13250\ 34500|15635\ 16450|500\ 10000000 </pre>

 <h3 class="section">14.3 hstack_vaapi# TOC</h3> Stack input videos horizontally. This is the VA-API variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class="option">height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">14.4 vstack_vaapi# TOC</h3> Stack input videos vertically. This is the VA-API variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping

the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">14.5 xstack_vaapi# TOC</h3> Stack video inputs into custom layout. This is the VA-API variant of the xstack filter, each input stream may have different size, this filter will scale down/up each input stream to the given output size, or the size of the first input stream. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_vaapi=inputs=4:layout=0_0_1920x1080|0_1
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h3 class="section">14.6 pad_vaapi# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an

expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var>, and <var class=”var”>y</var> options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> </dl>

 <h3 class="section">14.7 drawbox_vaapi# TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class="code">fill</code> will create a filled box. Default value is <code class="code">3</code>. See below for the list of accepted constants. </dd> <dt><samp class="option">replace</samp></dt> <dd> With value <code class="code">1</code>, the pixels of the painted box will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the box onto the input video. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var class="var">t</var> are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_h, ih</samp></dt> <dt><samp class="option">in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. </dd> </dl>

 <h4 class="subsection">14.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h2 class="chapter">15 VideoToolbox Video Filters# TOC</h2>

Below is a description of the currently available VideoToolbox video filters. VideoToolbox filter depends on VideoToolbox framework, and is auto detected when building ffmpeg for an Apple platform such as macOS. Add <code class="code">--enable-videotoolbox</code> to configure if autodetect is disabled. <h3 class="section">15.1 scale_vt# TOC</h3>

Scale and convert the color parameters using VTPixelTransferSession. The filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. </dd> <dt><samp class="option">color_matrix</samp></dt> <dd> Set the output colorspace matrix. </dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the output color primaries. </dd> <dt><samp class="option">color_transfer</samp></dt> <dd> Set the output transfer characteristics. </dd> </dl>

 <h4 class="subsection">15.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Perform HDR to SDR conversion, and scale to half size of input <pre class="example-

preformatted”>ffmpeg -hwaccel videotoolbox \ -hwaccel_output_format videotoolbox_vld \ -i hdr.mov \ -c:v hevc_videotoolbox \ -profile:v main \ -b:v 3M \ -vf scale_vt=w=iw/2:h=ih/2:color_matrix=bt709:color_primaries=bt709:color_transfer=bt709 \ -c:a copy \ -tag:v hvc1 \ sdr.mp4 </pre>

 <h3 class=”section”>15.2 transpose_vt# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class=”table”> <dt>‘<samp class=”samp”>cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class=”samp”>clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class=”samp”>cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class=”samp”>clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class=”samp”>hflip</samp>’</dt> <dd> Flip the input video horizontally. </dd> <dt>‘<samp class=”samp”>vflip</samp>’</dt> <dd> Flip the input video vertically. </dd> </dl>

</dd> <dt><samp class=”option”>passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class=”samp”>portrait</samp>’</d <dd> Preserve portrait geometry (when <var class=”var”>height</var> >= <var class=”var”>width</var>). </dd> <dt>‘<samp class=”samp”>landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class=”var”>width</var> >= <var class=”var”>height</var>). </dd> </dl>

</dd> </dl>

 <h2 class=”chapter”>16 Vulkan Video Filters# TOC</h2>

Below is a description of the currently available Vulkan video fil-

ters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vulkan</code>, and have a SPIR-V compiler (<code class="code">glslc</code> or <code class="code">glslangValidator</code>) available at build time. Running Vulkan filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device vulkan[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">vulkan</var> called <var class="var">name</var>, using the given device parameters and options in <var class="var">key=value</var>. The following options are supported: <dl class="table"> <dt><samp class="option">debug</samp></dt> <dd> Switches validation layers on if set to 1. </dd> <dt><samp class="option">linear_images</samp></dt> <dd> Allocates linear images. Does not apply to decoding. </dd> <dt><samp class="option">disable_multiplane</samp></dt> <dd> Disables multiplane images. Does not apply to decoding. </dd> <dt><samp class="option">avoid_host_import</samp></dt> <dd> Avoids the use of dynamic host memory imports, in favor of a regular memcpy() into a previously mapped buffer. </dd> </dl>

</dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device and running nlmeans_vulkan filter with default parameters on it. <pre class="example-preformatted">-init_hw_device vulkan=vk:0 -filter_hw_device vk -i INPUT -vf "hwupload,nlmeans_vulkan,hwdownload" OUTPUT
</pre>

As Vulkan filters are not able to access frame data in normal memory, all frame data needs to be uploaded (hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded (hwdownload) back to normal memory. Note that hwupload will upload to a frame with the same layout as the software frame, so it may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it is usually necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class="section">16.1

avgblur_vulkan# TOC</h3>

Apply an average blur filter, implemented on the GPU using Vulkan. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> </dl>

 <h3 class=”section”>16.2 blend_vulkan# TOC</h3>

Blend two Vulkan frames into each other. The <code class=”code”>blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>c0_mode</samp></dt> <dt><samp class=”option”>c1_mode</samp></dt> <dt><samp class=”option”>c2_mode</samp></dt> <dt><samp class=”option”>c3_mode</samp></dt> <dt><samp class=”option”>all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class=”var”>all_mode</var>. Default value is <code class=”code”>normal</code>. Available values for component modes are: <dl class=”table”> <dt>‘<samp class=”samp”>normal</samp>’</dt> <dt>‘<samp class=”samp”>multiply</samp>’
</dl>

</dd> </dl>

 <h3 class=”section”>16.3 bwdif_vulkan# TOC</h3>

Deinterlacer using bwdif, the "Bob Weaver Deinterlacing Filter" algorithm, implemented on the GPU using Vulkan. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0,

send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">16.4 chromaber_vulkan# TOC</h3>

Apply an effect that emulates chromatic aberration. Works best with RGB inputs, but provides a similar effect with YCbCr inputs too. <dl class="table"> <dt><samp class="option">dist_x</samp></dt> <dd> Horizontal displacement multiplier. Each chroma pixel’s position will be multiplied by this amount, starting from the center of the image. Default is <code class="code">0</code>. </dd> <dt><samp class="option">dist_y</samp></dt> <dd> Similarly, this sets the vertical displacement multiplier. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">16.5 color_vulkan# TOC</h3>

Video source that creates a Vulkan frame of a solid color. Useful for benchmarking, or overlaying. It accepts the following parameters: <dl class="table"> <dt><samp class="option">color</samp></dt>

<dd> The color to use. Either a name, or a hexadecimal value. The default value is <code class="code">black</code>. </dd> <dt><samp class="option">size</samp></dt> <dd> The size of the output frame. Default value is <code class="code">1920x1080</code>. </dd> <dt><samp class="option">rate</samp></dt> <dd> The framerate to output at. Default value is <code class="code">60</code> frames per second. </dd> <dt><samp class="option">duration</samp></dt> <dd> The video duration. Default value is <code class="code">-0.000001</code>. </dd> <dt><samp class="option">sar</samp></dt> <dd> The video signal aspect ratio. Default value is <code class="code">1/1</code>. </dd> <dt><samp class="option">format</samp></dt> <dd> The pixel format of the output Vulkan frames. Default value is <code class="code">yuv444p</code>. </dd> <dt><samp class="option">out_range</samp></dt> <dd> Set the output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> </dl>

 <h3 class="section">16.6 vflip_vulkan# TOC</h3>

Flips an image vertically. <h3 class="section">16.7 hflip_vulkan# TOC</h3>

Flips an image horizontally. <h3 class="section">16.8 flip_vulkan# TOC</h3>

Flips an image along both the vertical and horizontal axis. <h3 class="section">16.9 fruc_vulkan# TOC</h3>

Frame rate up-conversion using the NVIDIA Vulkan optical flow extension (<code class="code">VK_NV_optical_flow</code>), implemented on the

GPU using Vulkan. For each pair of consecutive input frames the filter computes a forward and backward optical flow field on the device’s optical flow engine, and uses those fields to synthesise motion-compensated intermediate frames at the requested output frame rate. The frame timing logic mirrors the <code class="code">framerate</code> filter. The majority of YUV and RGB pixel formats are supported. The exceptions are subsampled packed YUV formats such as <code class="code">yuyv422</code> and <code class="code">uyvy422</code>, formats with more than 16 bits integer data per component, such as <code class="code">rgb96</code> and <code class="code">gray32</code>, and Bayer formats. <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> A string describing the desired output frame rate, evaluated as an expression. The following constants are available: <code class="code">source_fps</code>. The default value is <code class="code">60</code>. </dd> <dt><samp class="option">perf</samp></dt> <dd> Optical flow performance level, trading motion-estimation quality for speed. This is the dominant cost at high resolutions. Possible values are: <dl class="table"> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> Highest quality, slowest. This is the default. </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Balanced quality and speed. </dd> <dt>‘<samp class="samp">fast</samp>’</dt> <dd> Lowest quality, fastest. </dd> </dl>

</dd> <dt><samp class="option">grid</samp></dt> <dd> Optical flow output grid size in pixels. A coarser grid computes fewer flow vectors, making it faster and less memory hungry at the cost of flow detail. Possible values are <code class="code">auto</code> (the default; selects the finest grid the device supports, usually 1x1), <code class="code">1</code>, <code class="code">2</code>, <code class="code">4</code> and <code class="code">8</code>. Only grid sizes the device advertises are accepted. </dd> </dl>

 <h3 class="section">16.10 gblur_vulkan# TOC</h3>

Apply Gaussian blur filter on Vulkan frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class="code">sigma</code>. Default is <code class="code">-1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the kernel size along the

horizontal axis. Default is <code class="code">19</code>. </dd> <dt><samp class="option">sizeV</samp></dt> <dd> Set the kernel size along the vertical axis. Default is <code class="code">0</code>, which sets to use the same value as <var class="var">size</var>. </dd> </dl>

 <h3 class="section">16.11 nlmeans_vulkan# TOC</h3>

Denoise frames using Non-Local Means algorithm, implemented on the GPU using Vulkan. Supports more pixel formats than nlmeans or nlmeans_opencl, including alpha channel support. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength for all components. Default is 1.0. Must be in range [0.0, 100.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size for all planes. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">t</samp></dt> <dd> Set parallelism. Default is 8. Must be a number in the range [1, 64]. Larger values will use more VRAM but may not result in greater speed. The optimal value is hardware and input dependent. </dd> <dt><samp class="option">s0</samp></dt> <dt><samp class="option">s1</samp></dt> <dt><samp class="option">s2</samp></dt> <dt><samp class="option">s3</samp></dt> <dd> Set denoising strength for a specific component. Default is <var class="var">1.0</var>, equal to <samp class="option">s</samp>. Must be in range [0.0, 100.0]. 0.0 disables denoising in that component. </dd> <dt><samp class="option">p0</samp></dt> <dt><samp class="option">p1</samp></dt> <dt><samp class="option">p2</samp></dt> <dt><samp class="option">p3</samp></dt> <dd> Set patch size for a specific component. Default is <var class="var">7</var>, equal to <samp class="option">p</samp>. Must be odd number in range [0, 99]. </dd> </dl>

 <h3 class="section">16.12 overlay_vulkan# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires all inputs to use the same pixel format. So, format conversion may be needed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set the x coordinate of the over-

laid video on the main video. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">16.13 transpose_vulkan# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">16.14 v360_vulkan# TOC</h3>

Convert 360 videos between various formats. Supported formats (projections) are subset of v360 filter, that are most popular 360-degree video formats. The filter accepts subset of v360 options: <dl class="table"> <dt><samp class="option">input</samp></dt>

<dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">eac</samp>’</dt> <dd> Equi-Angular Cube-map. (Output only) FIXME: current output doesn’t contain padding. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Regular video. </dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt> <dd> Dual fisheye. </dd> <dt>‘<samp class="samp">sg</samp>’</dt> <dd> Stereographic format. </dd> <dt>‘<samp class="samp">fisheye</samp>’</dt> <dd> Fisheye projection. </dd> <dt>‘<samp class="samp">gopromax</samp>’</dt> <dd> GoPro Max .360 format. (Input only) A .360 video file contains two video streams (front and rear); with this input projection the filter takes two inputs and combines them into a single stream. See the <samp class="option">overlap</samp> option. </dd> </dl>

</dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video resolution. See v360 options. </dd> <dt><samp class="option">yaw</samp></dt> <dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt> <dt><samp class="option">rorder</samp></dt> <dd> Set rotation for the output video. See v360 options. </dd> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dd> Set output horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt> <dd> Set input horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number of overlapped pixels on input .360 video. Only used with <code class="code">input=gopromax</code>. The GoPro Max .360 video contains overlapped area (See https://gopro.com/news/max-tech-specs-stitching-resolution). The filter blends overlapped images in these two areas. The number of pixels of overlapped areas depends on the video resolution. In most cases, the value is automatically calculated based on input video size, but it may be changed on GoPro Max2 or future models. The option is to adjust for unknown video resolutions. </dd> </dl>

 <h4 class="subsection">16.14.1 Examples# TOC</h4> Special

notes exist to use with <code class=”code”>input=gopromax</code>. The GoPro Max .360 video file contains two video streams, and the stream number must be specified. The stream number (e.g., <code class=”code”>[0:5]</code>) depends on the model and shooting mode. Please check actual stream number with <code class=”code”>ffprobe</code> command. <ul class=”itemize mark-bullet”> Convert GoPro Max (8bit color depth) .360 to Equirectangular projection. GoPro Max uses obsolete yuvj420p pixel format, thus convert the format to yuv420p before this filter. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]format=yuv420p,hwupload[vf],[0:5]format=yuv420p,hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=e, hwdownload,format=yuv420p" OUTPUT.mkv </pre>

Convert GoPro Max2 (10bit color depth) .360 to Dual fisheye projection. GoPro Max2 uses yuv420p10le pixel format. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]hwupload[vf],[0:4]hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=dfisheye, hwdownload,format=yuv420p10le" OUTPUT.mkv </pre>

 <h2 class=”chapter”>17 QSV Video Filters# TOC</h2>

Below is a description of the currently available QSV video filters. To enable compilation of these filters you need to configure FFmpeg with <code class=”code”>--enable-libmfx</code> or <code class=”code”>--enable-libvpl</code>. To use QSV filters, you need to setup the QSV device correctly. For more information, please read <a class=”url” href=”https://trac.ffmpeg.org/wiki/Hardware/QuickSync”>https://trac.ffmpeg.org/wiki/Hardware/QuickSy
  <h3 class=”section”>17.1 hstack_qsv# TOC</h3> Stack input videos horizontally. This is the QSV variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">17.2 vstack_qsv# TOC</h3> Stack input videos vertically. This is the QSV variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">17.3 xstack_qsv# TOC</h3> Stack video inputs into custom layout. This is the QSV variant of the xstack filter. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_qsv=inputs=4:layout=0_0_1920x1080|0_h0_1920x1080|w0_0_1920
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h2 class="chapter">18 Video Sources# TOC</h2>

Below is a description of the currently available video sources. <h3 class="section">18.1 buffer# TOC</h3>

Buffer video frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">width</samp></dt> <dd> The input video width. </dd> <dt><samp class="option">height</samp></dt> <dd> The input video height. </dd> <dt><samp class="option">pix_fmt</samp></dt> <dd> A string representing the pixel format of the buffered video frames. It may be a number corresponding to a pixel format, or a pixel format name. </dd> <dt><samp class="option">time_base</samp></dt> <dd> Specify the timebase assumed by the timestamps of the buffered frames. </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Specify the frame rate expected for the video stream. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> A string representing the color space of the buffered video frames. It may be a number corresponding to a color space, or a color space name. </dd> <dt><samp class="option">range</samp></dt> <dd> A string representing the color range of the buffered video frames. It may be a number corresponding to a color range, or a color range name. </dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> A string representing the alpha mode of the buffered video frames. It may be a number corresponding to an alpha mode, or an alpha mode name. </dd> <dt><samp class="option">chroma_location</samp></dt> <dd> A string representing the chroma sample location of the buffered video frames. It may be a number corresponding to a chroma sample location, or a chroma sample location name. When it is not set, the filtergraph chooses the chroma sample location of the output. Frames without a chroma sample location are tagged with the one negotiated for the output. </dd> <dt><samp class="option">pixel_aspect, sar</samp></dt> <dd> The sample (pixel) aspect ratio of the input video. </dd> <dt><samp class="option">hw_frames_ctx</samp></dt> <dd> When using a hardware pixel format, this should be a reference to an AVHWFramesContext describing input frames. </dd> </dl>

For example: <pre class="example-preformatted">buffer=width=320:height=240:pix_fmt=yuv410p:time_base=1/24:sar=1
</pre>

will instruct the source to accept video frames with size 320x240 and with format "yuv410p", assuming 1/24 as the timestamps timebase and square pixels (1:1 sample aspect ratio). Since the pixel format with name "yuv410p" corresponds to the number 6 (check the enum AVPixelFormat definition in <samp class="file">libavutil/pixfmt.h</samp>), this example corresponds to: <pre class="example-preformatted">buffer=size=320x240:pixfmt=6:time_base=1/24:pixel_aspect=1/1
</pre>

Alternatively, the options can be specified as a flat string, but this syntax is deprecated: <var class="var">width</var>:<var class="var">height</var>:<var class="var">pix_fmt</var>:<var class="var">time_base.num</var>:<var class="var">time_base.den</var>:<var class="var">pixel_aspect.num</var>:<var class="var">pixel_aspect.den</var> <h3 class="section">18.2 cellauto# TOC</h3>

Create a pattern generated by an elementary cellular automaton. The initial state of the cellular automaton can be defined through the <samp class="option">filename</samp> and <samp class="option">pattern</samp> options. If such options are not specified an initial state is created randomly. At each new frame a new row in the video is filled with the result of the cellular automaton next generation. The behavior when the whole frame is filled is defined by the <samp class="option">scroll</samp> option. This source accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified file. In the file, each non-whitespace character is considered an alive cell, a newline will terminate the row, and further characters in the file will be ignored. </dd> <dt><samp class="option">pattern, p</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified string. Each non-whitespace character in the string is considered an alive cell, a newline will terminate the row, and further characters in the string will be ignored. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set the video rate, that is the number of frames generated per second. Default is 25. </dd> <dt><samp class="option">random_fill_ratio, ratio</samp></dt> <dd> Set the random fill ratio for the initial cellular automaton row. It is a floating point number value ranging from 0 to 1, defaults to 1/PHI. This option is ignored when a file or a pattern is specified. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> Set the seed for filling randomly the initial row, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">rule</samp></dt> <dd> Set the cellular automaton rule, it is a number ranging from 0 to

255. Default value is 110. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the size of the output video. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. If <samp class="option">filename</samp> or <samp class="option">pattern</samp> is specified, the size is set by default to the width of the specified initial state row, and the height is set to <var class="var">width</var> * PHI. If <samp class="option">size</samp> is set, it must contain the width of the specified pattern string, and the specified pattern will be centered in the larger row. If a filename or a pattern string is not specified, the size value defaults to "320x518" (used for a randomly generated initial state). </dd> <dt><samp class="option">scroll</samp></dt> <dd> If set to 1, scroll the output upward when all the rows in the output have been already filled. If set to 0, the new generated row will be written over the top row just after the bottom row is filled. Defaults to 1. </dd> <dt><samp class="option">start_full, full</samp></dt> <dd> If set to 1, completely fill the output with generated rows before outputting the first frame. This is the default behavior, for disabling set the value to 0. </dd> <dt><samp class="option">stitch</samp></dt> <dd> If set to 1, stitch the left and right row edges together. This is the default behavior, for disabling set the value to 0. </dd> </dl>

 <h4 class="subsection">18.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read the initial state from <samp class="file">pattern</samp>, and specify an output of size 200x400. <pre class="example-preformatted">cellauto=f=pattern:s=200x400 </pre>

Generate a random initial row with a width of 200 cells, with a fill ratio of 2/3: <pre class="example-preformatted">cellauto=ratio=2/3:s=200x200 </pre>

Create a pattern generated by rule 18 starting by a single alive cell centered on an initial row with width 100: <pre class="example-preformatted">cellauto=p=@:s=100x400:full=0:rule=18 </pre>

Specify a more elaborated initial pattern: <pre class="example-preformatted">cellauto=p='@@ @ @@':s=100x400:full=0:rule=18 </pre>

<a name="coreimagesrc-

1”> <h3 class=”section”>18.3 coreimagesrc# TOC</h3> Video source generated on GPU using Apple’s CoreImage API on OSX. This video source is a specialized version of the coreimage video filter. Use a core image generator at the beginning of the applied filterchain to generate the content. The coreimagesrc video source accepts the following options: <dl class=”table”> <dt><samp class=”option”>list_generators</samp></dt> <dd> List all available generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class=”example-preformatted”>list_generators=true </pre>

</dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the size of the sourced video. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. The default value is <code class=”code”>320x240</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> </dl>

Additionally, all options of the coreimage video filter are accepted. A complete filterchain can be used for further processing of the generated input without CPU-HOST transfer. See coreimage documentation and examples for details. <h4 class=”subsection”>18.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class=”example-preformatted”>ffmpeg -f lavfi -i coreimagesrc=s=100x100:filter=CIQRCodeGenerator@inputMessage=https\\\\\://FFmpeg.org/@inputCorrecti

-frames:v 1 QRCode.png </pre> This example is equivalent to the QRCode example of coreimage without the need for a nullsrc video source.

 <h3 class=”section”>18.4 ddagrab# TOC</h3>

Captures the Windows Desktop via Desktop Duplication API. The filter exclusively returns D3D11 Hardware Frames, for on-gpu encoding or processing. So an explicit hwdownload is needed for any kind of software processing. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>output_idx</samp></dt> <dd> DXGI Output Index to capture. Usually corresponds to the index Windows has given the screen minus one, so it’s starting at 0. Defaults to output 0. </dd> <dt><samp class=”option”>draw_mouse</samp></dt> <dd> Whether to draw the mouse cursor. Defaults to true. Only affects hardware cursors. If a game or application renders its own cursor, it’ll always be captured. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Maximum framerate at which the desktop will be captured - the interval between successive frames will not be smaller than the inverse of the framerate. When <var class=”var”>dup_frames</var> is true (the default) and the desktop is not being updated often enough, the filter will duplicate a previous frame. Note that there is no background buffering going on, so when the filter is not polled often enough then the actual inter-frame interval may be significantly larger. Defaults to 30 FPS. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Specify the size of the captured video. Defaults to the full size of the screen. Cropped from the bottom/right if smaller than screen size. </dd> <dt><samp class=”option”>offset_x</samp></dt> <dd> Horizontal offset of the captured video. </dd> <dt><samp class=”option”>offset_y</samp></dt> <dd> Vertical offset of the captured video. </dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired filter output format. Defaults to 8 Bit BGRA. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Passes all supported output formats to DDA and returns what DDA decides to use. </dd> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8 Bit formats always work, and DDA will convert to them if necessary. </dd> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dd> Filter initialization will fail if 10 bit format is requested but unavailable. </dd> </dl>

</dd> <dt><samp class="option">dup_frames</samp></dt> <dd> When this option is set to true (the default), the filter will duplicate frames when the desktop has not been updated in order to maintain approximately constant target framerate. When this option is set to false, the filter will wait for the desktop to be updated (inter-frame intervals may vary significantly in this case). </dd> </dl>

 <h4 class="subsection">18.4.1 Examples# TOC</h4>

Capture primary screen and encode using nvenc: <pre class="example-preformatted">ffmpeg -f lavfi -i ddagrab -c:v h264_nvenc -cq 18 output.mp4 </pre>

You can also skip the lavfi device and directly use the filter. Also demonstrates downloading the frame and encoding with libx264. Explicit output format specification is required in this case: <pre class="example-preformatted">ffmpeg -filter_complex ddagrab=output_idx=1:framerate=60,hwdownload,fo -c:v libx264 -crf 18 output.mp4 </pre>

If you want to capture only a subsection of the desktop, this can be achieved by specifying a smaller size and its offsets into the screen: <pre class="example-preformatted">ddagrab=video_size=800x600:offset_x=100:offset_y </pre>

 <h3 class="section">18.5 gfxcapture# TOC</h3>

Capture windows or monitors using Windows.Graphics.Capture API.

This source provides low overhead capture of application windows or entire monitors.

The filter outputs hardware frames in <code class="code">d3d11</code> format; use <code class="code">hwdownload,format=</code>

if system memory frames are required.

The window to be captured can be selected via regular expressions on its title,

class name or backing executable name, by explicit native handles, or by monitor

index or explicit native handle. A window must match all provided expressions to be

selected. The first matching window will be picked, in whatever order Windows

returns them.

Explicit handles (<samp class="option">hwnd</samp>, <samp class="option">hmonitor</samp>) override pattern or index

based selection. If neither handles nor a monitor index are given, the first

window matching the provided regular expressions is captured.

This source does NOT hold a stable FPS. It returns frames at whatever rate the compositor

provides them, only capped by the <samp class="option">max_framerate</samp>.

If you need a stable rate, you need to add an fps filter to drop/duplicate frames as needed.

Note that a static source may not deliver any frames at all, which also stalls

the fps filter; see <samp class="option">dup_frames</samp>.

If the capture source disappears mid-capture (window closed, monitor disconnected), the filter will return EOF.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">window_title</samp></dt>

<dd> ECMAScript regular expression matched against the window title. Supports a

PCRE style <code class="code">(?i)</code> prefix for case-insensitive matching.

</dd>

<dt><samp class="option">window_class</samp></dt>

<dd> As <samp class="option">window_title</samp>, but matched against the window class name.

</dd>

<dt><samp class=”option”>window_exe</samp></dt>

<dd> As <samp class=”option”>window_title</samp>, but matched against the executable file name of the

window’s process.

</dd>

<dt><samp class=”option”>monitor_idx</samp></dt>

<dd> Zero-based index of the monitor to capture.

Can also be set to <code class=”code”>window</code> to capture the monitor the selected window

is displayed on at filter initialization time.

</dd>

<dt><samp class=”option”>hwnd</samp></dt>

<dd> Explicit native window handle (HWND).

</dd>

<dt><samp class=”option”>hmonitor</samp></dt>

<dd> Explicit native monitor handle (HMONITOR).

</dd>

<dt><samp class=”option”>capture_cursor</samp></dt>

<dd> Capture the mouse cursor. Enabled by default.

</dd>

<dt><samp class=”option”>capture_border</samp></dt>

<dd> Capture the full area of the window, including its window decorarions/border.

Disabled by default.

</dd>

<dt><samp class="option">display_border</samp></dt>

<dd> Draw a yellow highlight border around the captured window. Disabled by default.

</dd>

<dt><samp class="option">max_framerate</samp></dt>

<dd> Maximum capture frame rate. Accepts a video rate (e.g. <code class="code">30</code>, <code class="code">60/1</code>,

<code class="code">24000/1001</code>). The default is <code class="code">60</code> FPS.

The actual rate is the rate at which the compositor renders the window/monitor,

capped by this option.

</dd>

<dt><samp class="option">width</samp></dt>

<dd> Force the output canvas width. If zero (default) the initial captured source

width is used. If a negative number is provided, the width will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">height</samp></dt>

<dd> Force the output canvas height. If zero (default) the initial captured source

height is used. If a negative number is provided, the height will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">crop_left</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_top</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_right</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_bottom</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">premultiplied</samp></dt>

<dd> If set to 1, return frames with premultiplied alpha. Default is 0 (straight

alpha).

</dd>

<dt><samp class="option">dup_frames</samp></dt>

<dd> If enabled (the default), the last frame is output again whenever the source

delivered no new frame within one <samp class="option">max_framerate</samp> interval, and black

frames are output until the source delivered its first frame. This keeps the

output flowing while the captured content is static. The output rate is still

not constant, add an <code class="code">fps</code> filter if needed.

</dd>

<dt><samp class=”option”>resize_mode</samp></dt>

<dd> Defines how the captured content is fitted into the output canvas size.

Possible values:

<dl class=”table”>

<dt>‘<samp class=”samp”>crop</samp>’</dt>

<dd> Crop (or pad with black) to the canvas size. (default)

</dd>

<dt>‘<samp class=”samp”>scale</samp>’</dt>

<dd> Scale the source to fill the canvas, potentially altering aspect ratio.

</dd>

<dt>‘<samp class=”samp”>scale_aspect</samp>’</dt>

<dd> Scale the source to fit inside the canvas while preserving aspect ratio.

Remaining area is filled with black.

</dd>

</dl>

</dd> <dt><samp class=”option”>scale_mode</samp></dt> <dd> Scaling algorithm used when resizing is required. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Nearest neighbour (pixelated) scaling. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear filtering. (default) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> Bicubic filtering. Potentially more blurry, but fewer scaling artifacts depending on contents. </dd> </dl>

</dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired output pixel format inside the D3D11 hardware frames. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dd> 8 bit BGRA output (default) </dd> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dd> 10 bit BGR output </dd> <dt>‘<samp class=”samp”>rgbaf16</samp>’</dt> <dt>‘<samp class=”samp”>16bit</samp>’</dt> <dd> 16bit float RGBA output </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>18.5.1 Examples# TOC</h4> <ul class=”itemize

mark-bullet”> Capture a window by title (case-insensitive) at a maximum of 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_title='(?i)My Application':max_framerate=60,hwdownload,format=bgra,format=yuv420p -c:v libx264 -crf 15 capture.mp4 </pre>

Capture monitor 1 at native refresh, 10bit color depth, scale to 1920x1080 preserving aspect: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=monitor_idx=1:width=1920:height=1080:resize_mode=sc -c:v hevc_nvenc -cq 15 capture.mp4 </pre>

Capture a window by executable name, draw border, force point scaling, fixed 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_exe='^firefox.exe$':display_border=1:scale_mod -rc qvbr -qvbr_quality_level 15 -c:v h264_amf capture.mp4 </pre>

 <h3 class=”section”>18.6 gradients# TOC</h3> Generate several gradients. <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>c0, c1, c2, c3, c4, c5, c6, c7</samp></dt> <dd> Set 8 colors. Default values for colors is to pick random one. </dd> <dt><samp class=”option”>x0, y0, x1, y1</samp></dt> <dd> Set gradient line source and destination points. If negative or out of range, random ones are picked. </dd> <dt><samp class=”option”>nb_colors, n</samp></dt> <dd> Set number of colors to use at once. Allowed range is from 2 to 8. Default value is 2. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed for picking gradient line points. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set speed of gradients rotation. </dd> <dt><samp class=”option”>type, t</samp></dt> <dd> Set type of gradients. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>radial</samp>’</d

<dt>‘<samp class="samp">circular</samp>’</dt> <dt>‘<samp class="samp">spiral</samp>’</dt> <dt>‘<samp class="samp">square</samp>’</dt> </dl>

Default type is <var class="var">linear</var>. </dd> </dl>

 <h4 class="subsection">18.6.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">18.7 mandelbrot# TOC</h3>

Generate a Mandelbrot set fractal, and progressively zoom towards the point specified with <var class="var">start_x</var> and <var class="var">start_y</var>. This source accepts the following options: <dl class="table"> <dt><samp class="option">end_pts</samp></dt> <dd> Set the terminal pts value. Default value is 400. </dd> <dt><samp class="option">end_scale</samp></dt> <dd> Set the terminal scale value. Must be a floating point value. Default value is 0.3. </dd> <dt><samp class="option">inner</samp></dt> <dd> Set the inner coloring mode, that is the algorithm used to draw the Mandelbrot fractal internal region. It shall assume one of the following values: <dl class="table"> <dt><samp class="option">black</samp></dt> <dd> Set black mode. </dd> <dt><samp class="option">convergence</samp></dt> <dd> Show time until convergence. </dd> <dt><samp class="option">mincol</samp></dt> <dd> Set color based on point closest to the origin of the iterations. </dd> <dt><samp class="option">period</samp></dt> <dd> Set period mode. </dd> </dl>

Default value is <var class="var">mincol</var>. </dd> <dt><samp class="option">bailout</samp></dt> <dd> Set the bailout value. Default value is 10.0. </dd> <dt><samp class="option">maxiter</samp></dt> <dd> Set the maximum of iterations performed by the rendering algorithm. Default value is 7189. </dd> <dt><samp class="option">outer</samp></dt> <dd> Set outer coloring mode. It shall assume one of following values: <dl class="table"> <dt><samp class="option">iteration_count</samp></dt> <dd> Set iteration count mode. </dd> <dt><samp class="option">normalized_iteration_count</samp></dt> <dd> set normalized iteration count mode. </dd> </dl> Default value is <var class="var">normalized_iteration_count</var>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value

is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">start_scale</samp></dt> <dd> Set the initial scale value. Default value is 3.0. </dd> <dt><samp class="option">start_x</samp></dt> <dd> Set the initial x position. Must be a floating point value between -100 and 100. Default value is -0.74364388703715870475219150611477 4. </dd> <dt><samp class="option">start_y</samp></dt> <dd> Set the initial y position. Must be a floating point value between -100 and 100. Default value is -0.13182590420531197049313205638513 9. </dd> </dl>

 <h3 class="section">18.8 mptestsrc# TOC</h3>

Generate various test patterns, as generated by the MPlayer test filter. The size of the generated video is fixed, and is 512x512. This source is useful in particular for testing encoding features. This source accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class="option">test, t</samp></dt> <dd> Set the number or the name of the test to perform. Supported tests are: <dl class="table"> <dt><samp class="option">dc_luma</samp></dt> <dt><samp class="option">dc_chroma</samp></dt> <dt><samp class="option">freq_luma</samp></dt> <dt><samp class="option">freq_chroma</samp></dt> <dt><samp class="option">amp_luma</samp></dt> <dt><samp class="option">amp_chroma</samp></dt> <dt><samp class="option">cbp</samp></dt> <dt><samp class="option">mv</samp></dt> <dt><samp class="option">ring1</samp></dt> <dt><samp class="option">ring2</samp></dt> <dt><samp class="option">all</samp></dt> <dt><samp class="option">max_frames, m</samp></dt> <dd> Set the maximum number of frames generated for each test, default value is 30. </dd> </dl>

Default value is "all", which will cycle through the list of all

tests. </dd> </dl>

Some examples: <pre class=”example-preformatted”>mptestsrc=t=dc_luma </pre>

will generate a "dc_luma" test pattern. <h3 class=”section”>18.9 frei0r_src# TOC</h3>

Provide a frei0r source. To enable compilation of this filter you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. This source accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>size</samp></dt> <dd> The size of the video to generate. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> The framerate of the generated video. It may be a string of the form <var class=”var”>num</var>/<var class=”var”>den</var> or a frame rate abbreviation. </dd> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name to the frei0r source to load. For more information regarding frei0r and how to set the parameters, read the frei0r section in the video filters documentation. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r source. </dd> </dl>

For example, to generate a frei0r partik0l source with size 200x200 and frame rate 10 which is overlaid on the overlay filter main input: <pre class=”example-preformatted”>frei0r_src=size=200x200:framerate=10:filter_name=
[overlay]; [in][overlay] overlay </pre>

 <h3 class=”section”>18.10 life# TOC</h3>

Generate a life pattern.

This source is based on a generalization of John Conway’s life game.

The sourced input represents a life grid, each pixel represents a cell

which can be in one of two possible states, alive or dead. Every cell

interacts with its eight neighbours, which are the cells that are

horizontally, vertically, or diagonally adjacent.

At each interaction the grid evolves according to the adopted rule,

which specifies the number of neighbor alive cells which will make a

cell stay alive or born. The <samp class="option">rule</samp> option allows one to specify

the rule to adopt.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">filename, f</samp></dt>

<dd> Set the file from which to read the initial grid state. In the file,

each non-whitespace character is considered an alive cell, and newline

is used to delimit the end of each row.

If this option is not specified, the initial grid is generated

randomly.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the video rate, that is the number of frames generated per second.

Default is 25.

</dd>

<dt><samp class="option">random_fill_ratio, ratio</samp></dt>

<dd> Set the random fill ratio for the initial random grid. It is a

floating point number value ranging from 0 to 1, defaults to 1/PHI.

It is ignored when a file is specified.

</dd>

<dt><samp class=”option”>random_seed, seed</samp></dt>

<dd> Set the seed for filling the initial random grid, must be an integer

included between 0 and UINT32_MAX. If not specified, or if explicitly

set to -1, the filter will try to use a good random seed on a best

effort basis.

</dd>

<dt><samp class=”option”>rule</samp></dt>

<dd> Set the life rule.

A rule can be specified with a code of the kind "S<var class=”var”>NS</var>/B<var class=”var”>NB</var>",

where <var class=”var”>NS</var> and <var class=”var”>NB</var> are sequences of numbers in the range 0-8,

<var class=”var”>NS</var> specifies the number of alive neighbor cells which make a

live cell stay alive, and <var class=”var”>NB</var> the number of alive neighbor cells

which make a dead cell to become alive (i.e. to "born").

"s" and "b" can be used in place of "S" and "B", respectively.

Alternatively a rule can be specified by an 18-bits integer. The 9

high order bits are used to encode the next cell state if it is alive

for each number of neighbor alive cells, the low order bits specify

the rule for "borning" new cells. Higher order bits encode for an

higher number of neighbor cells.

For example the number 6153 = <code class=”code”>(12<<9)+9</code> specifies a stay alive

rule of 12 and a born rule of 9, which corresponds to "S23/B03".

Default value is "S23/B3", which is the original Conway’s game of life

rule, and will keep a cell alive if it has 2 or 3 neighbor alive

cells, and will born a new cell if there are three alive cells around

a dead cell.

</dd>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Set the size of the output video. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

If <samp class=”option”>filename</samp> is specified, the size is set by default to the

same size of the input file. If <samp class=”option”>size</samp> is set, it must contain

the size specified in the input file, and the initial grid defined in

that file is centered in the larger resulting area.

If a filename is not specified, the size value defaults to "320x240"

(used for a randomly generated initial grid).

</dd>

<dt><samp class=”option”>stitch</samp></dt>

<dd> If set to 1, stitch the left and right grid edges together, and the

top and bottom edges also. Defaults to 1.

</dd>

<dt><samp class=”option”>mold</samp></dt>

<dd> Set cell mold speed. If set, a dead cell will go from <samp class=”option”>death_color</samp> to

<samp class=”option”>mold_color</samp> with a step of <samp class=”option”>mold</samp>. <samp class=”option”>mold</samp> can have a

value from 0 to 255.

</dd>

<dt><samp class="option">life_color</samp></dt>

<dd> Set the color of living (or new born) cells.

</dd>

<dt><samp class="option">death_color</samp></dt>

<dd> Set the color of dead cells. If <samp class="option">mold</samp> is set, this is the first color

used to represent a dead cell.

</dd>

<dt><samp class="option">mold_color</samp></dt>

<dd> Set mold color, for definitely dead and moldy cells.

For the syntax of these 3 color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the

ffmpeg-utils manual.

</dd>

</dl>

 <h4 class="subsection">18.10.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read a grid from <samp class="file">pattern</samp>, and center it on a grid of size 300x300 pixels: <pre class="example-preformatted">life=f=pattern:s=300x300 </pre>

Generate a random grid of size 200x200, with a fill ratio of 2/3: <pre class="example-preformatted">life=ratio=2/3:s=200x200 </pre>

Specify a custom rule for evolving a randomly generated grid: <pre class="example-preformatted">life=rule=S14/B34 </pre>

Full example with slow death effect (mold) using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi life=s=300x200:mold=10:r=60:ratio=0.1:death_color=#C83232:life_color=#00f
</pre>

 <h3 class="section">18.11 perlin#
TOC</h3>
Generate Perlin noise. Perlin noise is a kind of noise with local continuity in space. This can be used to generate patterns with continuity in space and time, e.g. to simulate smoke, fluids, or terrain. In case more than one octave is specified through the <samp class="option">octaves</samp> option, Perlin noise is generated as a sum of components, each one with doubled frequency. In this case the <samp class="option">persistence</samp> option specify the ratio of the amplitude with respect to the previous component. More octave components enable to specify more high frequency details in the generated noise (e.g. small size variations due to boulders in a generated terrain). <h4 class="subsection">18.11.1 Options# TOC</h4> <dl class="table">
<dt><samp class="option">size, s</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">320x240</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate expected for the video stream, expressed as a number of frames per second. Default value is <code class="code">25</code>. </dd>
<dt><samp class="option">octaves</samp></dt> <dd> Specify the total number of components making up the noise, each one with doubled frequency. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">persistence</samp></dt>
<dd> Set the ratio used to compute the amplitude of the next octave component with respect to the previous component amplitude. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">xscale</samp></dt> <dt><samp class="option">yscale</samp></dt>
<dd> Define a scale factor used to multiple the x, y coordinates. This can be useful to define an effect with a pattern stretched along the x or y axis. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">tscale</samp></dt> <dd> Define a scale factor used to multiple the time coordinate. This can be useful to change the time variation speed. Default value is <code class="code">1</code>. </dd>
<dt><samp class="option">random_mode</samp></dt> <dd> Set random mode used to compute initial pattern. Supported values are:

<dl class="table"> <dt><samp class="option">random</samp></dt> <dd> Compute and use random seed. </dd> <dt><samp class="option">ken</samp></dt> <dd> Use the predefined initial pattern defined by Ken Perlin in the original article, can be useful to compare the output with other sources. </dd> <dt><samp class="option">seed</samp></dt> <dd> Use the value specified by <samp class="option">random_seed</samp> option. </dd> </dl>

Default value is <code class="code">random</code>. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> When <samp class="option">random_mode</samp> is set to <var class="var">random_seed</var>, use this value to compute the initial pattern. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">18.11.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Generate single component: <pre class="example-preformatted">perlin </pre>

Use Perlin noise with 7 components, each one with a halved contribution to total amplitude: <pre class="example-preformatted">perlin=octaves=7:persistence=0.5 </pre>

Chain Perlin noise with the lutyuv to generate a black&white effect: <pre class="example-preformatted">perlin=octaves=3:tscale=0.3,lutyuv=y='if(lt(val\,128)\,255\,0)' </pre>

Stretch noise along the y axis, and convert gray level to red-only signal: <pre class="example-preformatted">perlin=octaves=7:tscale=0.4:yscale=0.3,lut </pre>

 <h3 class="section">18.12 qrencodesrc# TOC</h3>

Generate a QR code using the libqrencode library (see https://fukuchi.org/works/qrencode/). To enable the compilation of this source, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and put in the video output according to the specified output size options. In case no text is specified, the QR code is not generated, but an empty colored output is returned instead. This source accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp

class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated only once, when initializing the source. See the qrencode Expressions section for details. Note that some of the constants are missing for the source (for example the <var class=”var”>x</var> or <var class=”var”>t</var> or ¸<var class=”var”>n</var>), since they only makes sense when evaluating the expression for each frame rather than at initialization time. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is "white". For the syntax of the color options,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </p></dd> </dl>

 <h4 class="subsection">18.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:bc=pink:fc=blue </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencodesrc=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencodesrc=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencodesrc=text=%{pts} </pre>

 <h3 class="section">18.13

allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc# TOC</h3>

The <code class="code">allrgb</code> source returns frames of size 4096x4096 of all rgb colors.

The <code class="code">allyuv</code> source returns frames of size 4096x4096 of all yuv colors.

The <code class="code">color</code> source provides an uniformly colored input.

The <code class="code">colorchart</code> source provides a colors checker chart.

The <code class="code">colorspectrum</code> source provides a color spectrum input.

The <code class="code">haldclutsrc</code> source provides an identity Hald CLUT. See also

haldclut filter.

The <code class="code">nullsrc</code> source returns unprocessed video frames. It is

mainly useful to be employed in analysis / debugging tools, or as the

source for filters which ignore the input data.

The <code class="code">pal75bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 75% color levels.

The <code class=”code”>pal100bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 100% color levels.

The <code class=”code”>rgbtestsrc</code> source generates an RGB test pattern useful for

detecting RGB vs BGR issues. You should see a red, green and blue

stripe from top to bottom.

The <code class=”code”>smptebars</code> source generates a color bars pattern, based on

the SMPTE Engineering Guideline EG 1-1990.

The <code class=”code”>smptehdbars</code> source generates a color bars pattern, based on

the SMPTE RP 219-2002.

The <code class=”code”>testsrc</code> source generates a test video pattern, showing a

color pattern, a scrolling gradient and a timestamp. This is mainly

intended for testing purposes.

The <code class=”code”>testsrc2</code> source is similar to testsrc, but supports more

pixel formats instead of just <code class=”code”>rgb24</code>. This allows using it as an

input for other tests without requiring a format conversion.

The <code class=”code”>yuvtestsrc</code> source generates an YUV test pattern. You should

see a y, cb and cr stripe from top to bottom.

The sources accept the following parameters:

<dl class="table">

<dt><samp class="option">level</samp></dt>

<dd> Specify the level of the Hald CLUT, only available in the <code class="code">haldclutsrc</code>

source. A level of <code class="code">N</code> generates a picture of <code class="code">N*N*N</code> by <code class="code">N*N*N</code>

pixels to be used as identity matrix for 3D lookup tables. Each component is

coded on a <code class="code">1/(N*N)</code> scale.

</dd>

<dt><samp class="option">color, c</samp></dt>

<dd> Specify the color of the source, only available in the <code class="code">color</code>

source. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the size of the sourced video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

The default value is <code class="code">320x240</code>.

This option is not available with the <code class="code">allrgb</code>, <code class="code">allyuv</code>, and

<code class="code">haldclutsrc</code> filters.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Specify the frame rate of the sourced video, as the number of frames

generated per second. It has to be a string in the format

<var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point

number or a valid video frame rate abbreviation. The default value is

"25".

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Set the duration of the sourced video. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If not specified, or the expressed duration is negative, the video is

supposed to be generated forever.

Since the frame rate is used as time base, all frames including the last one

will have their full duration. If the specified duration is not a multiple

of the frame duration, it will be rounded up.

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> Set the sample aspect ratio of the sourced video.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Specify the alpha (opacity) of the background, only available in the

<code class="code">testsrc2</code> source. The value must be between 0 (fully transparent) and

255 (fully opaque, the default).

</dd>

<dt><samp class="option">decimals, n</samp></dt>

<dd> Set the number of decimals to show in the timestamp, only available in the

<code class="code">testsrc</code> source.

The displayed timestamp value will correspond to the original

timestamp value multiplied by the power of 10 of the specified

value. Default value is 0.

</dd>

<dt><samp class="option">type</samp></dt>

<dd> Set the type of the color spectrum, only available in the

<code class="code">colorspectrum</code> source. Can be one of the following:

<dl class="table">

<dt>‘<samp class="samp">black</samp>’</dt>

<dt>‘<samp class="samp">white</samp>’</dt>

<dt>‘<samp class="samp">all</samp>’</dt>

</dl>

</dd> <dt><samp class="option">patch_size</samp></dt> <dd> Set patch size of single color patch, only available in the <code class="code">colorchart</code> source. Default is <code class="code">64x64</code>. </dd> <dt><samp class="option">preset</samp></dt> <dd> Set colorchecker colors preset, only available in the <code class="code">colorchart</code> source. Available values are: <dl class="table"> <dt>‘<samp class="samp">reference</samp>’</dt> <dt>‘<samp class="samp">skintones</samp>’</dt> </dl>

Default value is <code class="code">reference</code>. </dd> </dl>

 <h4 class="subsection">18.13.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a video with a duration of 5.3 seconds, with size 176x144 and a frame rate of 10 frames per second: <pre class="example-preformatted">testsrc=duration=5.3:size=qcif:rate=10 </pre>

The following graph description will generate a red source with an opacity of 0.2, with size "qcif" and a frame rate of 10 frames per second: <pre class="example-preformatted">color=c=red@0.2:s=qcif:r=10 </pre>

If the input content is to be ignored, <code class="code">nullsrc</code> can be used. The following command generates noise in the luma plane by employing the <code class="code">geq</code> filter: <pre class="example-preformatted">nullsrc=s=256x256, geq=random(1)*255:128:128 </pre>

 <h4 class="subsection">18.13.2 Commands# TOC</h4>

The <code class="code">color</code> source supports the following commands: <dl class="table"> <dt><samp class="option">c, color</samp></dt> <dd> Set the color of the created image. Accepts the same syntax of the corresponding <samp class="option">color</samp> option. </dd> </dl>

 <h3 class="section">18.14 openclsrc# TOC</h3>

Generate video using an OpenCL program. <dl class="table"> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Kernel name in program. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Size of frames to generate. This must be set. </dd> <dt><samp class="option">format</samp></dt> <dd> Pixel format to use for the generated frames. This must be set. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Number of frames generated every second. Default value is ’25’. </dd> </dl>

For details of how the program loading works, see the program_opencl filter. Example

programs: <ul class="itemize mark-bullet"> Generate a colour ramp by setting pixel values from the position of the pixel in the output image. (Note that this will work with all pixel formats, but the generated output will not be the same.) <pre class="verbatim">___kernel void ramp(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 val; val.xy = val.zw = convert_float2(loc) / convert_float2(get_image_dim(dst));

write_imagef(dst, loc, val); } </pre> Generate a Sierpinski carpet pattern, panning by a single pixel each frame. <pre class="verbatim">___kernel void sierpinski_carpet(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 value = 0.0f; int x = loc.x + index; int y = loc.y + index; while (x > 0 || y > 0) { if (x % 3 == 1 & y % 3 == 1) { value = 1.0f; break; } x /= 3; y /= 3; }

write_imagef(dst, loc, value); } </pre>

 <h3 class="section">18.15 sierpinski# TOC</h3>

Generate a Sierpinski carpet/triangle fractal, and randomly pan around. This source accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed which is used for random panning. </dd> <dt><samp class="option">jump</samp></dt> <dd> Set max jump for single pan destination. Allowed range is from 1 to 10000. </dd> <dt><samp class="option">type</samp></dt> <dd> Set fractal type, can be default <code class="code">carpet</code> or <code class="code">triangle</code>. </dd> </dl>

 <h3 class="section">18.16 zoneplate# TOC</h3> Generate a zoneplate test video pattern. This source accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option,

check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "320x240". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set precision in bits for look-up table for sine calculations. Default value is 10. Allowed range is from 4 to 16. </dd> <dt><samp class=”option”>xo</samp></dt> <dd> Set horizontal axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>yo</samp></dt> <dd> Set vertical axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>to</samp></dt> <dd> Set time axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>k0</samp></dt> <dd> Set 0-order, constant added to signal phase. Default value is 0. </dd> <dt><samp class=”option”>kx</samp></dt> <dd> Set 1-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky</samp></dt> <dd> Set 1-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt</samp></dt> <dd> Set 1-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>kxt, kyt, kxy</samp></dt> <dd> Set phase factor multipliers for combination of spatial and temporal axis. Default value is 0. </dd> <dt><samp class=”option”>kx2</samp></dt> <dd> Set 2-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky2</samp></dt> <dd> Set 2-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt2</samp></dt> <dd> Set 2-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>ku</samp></dt> <dd> Set the constant added to final phase to produce chroma-blue component of signal. Default value is 0. </dd> <dt><samp class=”option”>kv</samp></dt> <dd> Set the constant added to final phase to produce chroma-red component of signal. Default value is 0. </dd> </dl>

 <h4 class=”subsection”>18.16.1 Commands# <a class=”anchor hidden-xs”href=”#toc-

Commands-159” aria-hidden=”true”>TOC</h4>

This source supports the some above options as commands. <h4 class=”subsection”>18.16.2 Examples#
TOC</h4>

<ul class=”itemize mark-bullet”> Generate horizontal color sine sweep:
 <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:kx2=256:s=wvg
426:kt=11 </pre>

Generate vertical color sine sweep: <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:ky2=156:s=wvga:yo=-240:kt=11 </pre>

Generate circular zone-plate: <pre class=”example-preformatted”>zoneplate=ku=512:kv=100:kt2=0:ky2=256:kx2=556:s=wvga:yo=0:kt=11 </pre>

 <h2 class=”chapter”>19 Video Sinks# TOC</h2>

Below is a description of the currently available video sinks. <h3 class=”section”>19.1 buffersink# TOC</h3>

Buffer video frames, and make them available to the end of the filter graph. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVBufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class=”code”>avfilter_init_filter</code> for initialization. <h3 class=”section”>19.2 nullsink# TOC</h3>

Null video sink: do absolutely nothing with the input video. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class=”chapter”>20 Multimedia Filters# TOC</h2>

Below is a description of the currently available multimedia filters. <h3 class="section">20.1 a3dscope# TOC</h3>

Convert input audio to 3d scope video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">fov</samp></dt> <dd> Set the camera field of view. Default is 90 degrees. Allowed range is from 40 to 150. </dd> <dt><samp class="option">roll</samp></dt> <dd> Set the camera roll. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set the camera pitch. </dd> <dt><samp class="option">yaw</samp></dt> <dd> Set the camera yaw. </dd> <dt><samp class="option">xzoom</samp></dt> <dd> Set the camera zoom on X-axis. </dd> <dt><samp class="option">yzoom</samp></dt> <dd> Set the camera zoom on Y-axis. </dd> <dt><samp class="option">zzoom</samp></dt> <dd> Set the camera zoom on Z-axis. </dd> <dt><samp class="option">xpos</samp></dt> <dd> Set the camera position on X-axis. </dd> <dt><samp class="option">ypos</samp></dt> <dd> Set the camera position on Y-axis. </dd> <dt><samp class="option">zpos</samp></dt> <dd> Set the camera position on Z-axis. </dd> <dt><samp class="option">length</samp></dt> <dd> Set the length of displayed audio waves in number of frames. </dd> </dl>

 <h4 class="subsection">20.1.1 Commands# TOC</h4>

Filter supports the some above options as commands. <h3 class="section">20.2 abitscope# TOC</h3>

Convert input audio to a video output, displaying the audio bit scope. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default

value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">1024x256</code>. </dd> <dt><samp class="option">colors</samp></dt> <dd> Specify list of colors separated by space or by ’|’ which will be used to draw channels. Unrecognized or missing colors will be replaced by white color. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set output mode. Can be <code class="code">bars</code> or <code class="code">trace</code>. Default is <code class="code">bars</code>. </dd> </dl>

 <h3 class="section">20.3 adrawgraph# TOC</h3> Draw a graph using input audio metadata. See drawgraph <h3 class="section">20.4 agraphmonitor# TOC</h3>

See graphmonitor. <h3 class="section">20.5 ahistogram# TOC</h3>

Convert input audio to a video output, displaying the volume histogram.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">dmode</samp></dt>

<dd> Specify how histogram is calculated.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">single</samp>’</dt>

<dd> Use single histogram for all channels.

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> Use separate histogram for each channel.

</dd>

</dl>

Default is <code class="code">single</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set frame rate, expressed as number of frames per second. Default

value is "25".

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">hd720</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set display scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

<dt>‘<samp class="samp">sqrt</samp>’</dt>

<dd> square root

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> reverse logarithmic
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">acount</samp></dt>
<dd> Set how much frames to accumulate in histogram.

Default is 1. Setting this to -1 accumulates all frames.

</dd>
<dt><samp class=”option”>rheight</samp></dt>
<dd> Set histogram ratio of window height.

</dd>
<dt><samp class=”option”>slide</samp></dt>
<dd> Set sonogram sliding.

It accepts the following values:
<dl class=”table”>
<dt>‘<samp class=”samp”>replace</samp>’</dt>
<dd> replace old rows with new ones.
</dd>
<dt>‘<samp class=”samp”>scroll</samp>’</dt>
<dd> scroll from top to bottom.
</dd>
</dl>
Default is <code class=”code”>replace</code>.

</dd>
<dt><samp class=”option”>hmode</samp></dt>
<dd> Set histogram mode.

It accepts the following values:
<dl class=”table”>
<dt>‘<samp class=”samp”>abs</samp>’</dt>
<dd> Use absolute values of samples.
</dd>
<dt>‘<samp class=”samp”>sign</samp>’</dt>

<dd> Use untouched values of samples.

</dd>

</dl>

Default is <code class="code">abs</code>.

</dd>

</dl>

 <h3 class="section">20.6 aphasemeter# TOC</h3>

Measures phase of input audio, which is exported as metadata <code class="code">lavfi.aphasemeter.phase</code>, representing mean phase of current audio frame. A video output can also be produced and is enabled by default. The audio is passed through as first output. Audio will be rematrixed to stereo if it has a different channel layout. Phase value is in range <code class="code">[-1, 1]</code> where <code class="code">-1</code> means left and right channels are completely out of phase and <code class="code">1</code> means channels are in phase. The filter accepts the following options, all related to its video output: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class="code">25</code>. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">800x400</code>. </dd> <dt><samp class="option">rc</samp></dt> <dt><samp class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt> <dd> Specify the red, green, blue contrast. Default values are <code class="code">2</code>, <code class="code">7</code> and <code class="code">1</code>. Allowed range is <code class="code">[0, 255]</code>. </dd> <dt><samp class="option">mpc</samp></dt> <dd> Set color which will be used for drawing median phase. If color is <code class="code">none</code> which is default, no median phase value will be drawn. </dd> <dt><samp class="option">video</samp></dt> <dd> Enable video output. Default is enabled. </dd> </dl>

 <h4 class="subsection">20.6.1 phasing detection# TOC</h4>

The filter also detects out of phase and mono sequences in stereo

streams. It logs the sequence start, end and duration when it lasts longer or as long as the minimum set. The filter accepts the following options for this detection: <dl class="table"> <dt><samp class="option">phasing</samp></dt> <dd> Enable mono and out of phase detection. Default is disabled. </dd> <dt><samp class="option">tolerance, t</samp></dt> <dd> Set phase tolerance for mono detection, in amplitude ratio. Default is <code class="code">0</code>. Allowed range is <code class="code">[0, 1]</code>. </dd> <dt><samp class="option">angle, a</samp></dt> <dd> Set angle threshold for out of phase detection, in degree. Default is <code class="code">170</code>. Allowed range is <code class="code">[90, 180]</code>. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set mono or out of phase duration until notification, expressed in seconds. Default is <code class="code">2</code>. </dd> </dl>

 <h4 class="subsection">20.6.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example with <code class="command">ffmpeg</code> to detect 1 second of mono with 0.001 phase tolerance: <pre class="example-preformatted">ffmpeg -i stereo.wav -af aphasemeter=video=0:phasing=1:duration=1:tolerance=0.001 -f null - </pre>

 <h3 class="section">20.7 avectorscope# TOC</h3>

Convert input audio to a video output, representing the audio vector scope. The filter is used to measure the difference between channels of stereo audio stream. A monaural signal, consisting of identical left and right signal, results in straight vertical line. Any stereo separation is visible as a deviation from this line, creating a Lissajous figure. If the straight (or deviation from it) but horizontal line appears this indicates that the left and right channels are out of phase. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set the vectorscope mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">lissajous</samp>’</dt> <dd> Lissajous rotated by 45 degrees. </dd> <dt>‘<samp class="samp">lissajous_xy</samp>’</dt> <dd> Same as above but not rotated. </dd> <dt>‘<samp class="samp">polar</samp>’</dt> <dd> Shape resembling half of circle. </dd> </dl>

Default value is ‘<samp class="samp">lissajous</samp>’.
 </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set
the video size for the output. For the syntax of this option, check
the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-
syntax">"Video size" section in the ffmpeg-utils manual.
Default value is <code class="code">400x400</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Set the
output frame rate. Default value is <code class="code">25</code>.
 </dd> <dt><samp class="option">rc</samp></dt> <dt><samp
class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt>
<dt><samp class="option">ac</samp></dt> <dd> Specify the red,
green, blue and alpha contrast. Default values are <code class="code">40</code>,
<code class="code">160</code>, <code class="code">80</code> and
<code class="code">255</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">rf</samp></dt>
<dt><samp class="option">gf</samp></dt> <dt><samp class="option">bf</samp></dt>
<dt><samp class="option">af</samp></dt> <dd> Specify the red,
green, blue and alpha fade. Default values are <code class="code">15</code>,
<code class="code">10</code>, <code class="code">5</code> and
<code class="code">5</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">zoom</samp></dt>
<dd> Set the zoom factor. Default value is <code class="code">1</code>.
Allowed range is <code class="code">[0, 10]</code>. Values lower than
<var class="var">1</var> will auto adjust zoom factor to maximal possible
value. </dd> <dt><samp class="option">draw</samp></dt>
<dd> Set the vectorscope drawing mode. Available values are:
<dl class="table"> <dt>‘<samp class="samp">dot</samp>’</dt>
<dd> Draw dot for each sample. </dd> <dt>‘<samp
class="samp">line</samp>’</dt> <dd> Draw line between pre-
vious and current sample. </dd> <dt>‘<samp class="samp">aaline</samp>’</dt>
<dd> Draw anti-aliased line between previous and current sample.
</dd> </dl>

Default value is ‘<samp class="samp">dot</samp>’.
 </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
amplitude scale of audio samples. Available values are: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root. </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic. </dd> </dl>

</dd> <dt><samp class="option">swap</samp></dt> <dd> Swap
left channel axis with right channel axis. </dd> <dt><samp
class="option">mirror</samp></dt> <dd> Mirror axis. <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> No mirror. </dd> <dt>‘<samp class="samp">x</samp>’</dt>

<dd> Mirror only x axis. </dd> <dt>‘<samp class="samp">y</samp>’</dt>
<dd> Mirror only y axis. </dd> <dt>‘<samp class="samp">xy</samp>’</dt>
<dd> Mirror both axis. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">20.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] avectorscope=zoom=1.3:rc=2:gc=200:bc=10:rf=1:gf=8:bf=7 [out0]' </pre>

 <h4 class="subsection">20.7.2 Commands# TOC</h4>

This filter supports the all above options as commands except options <code class="code">size</code> and <code class="code">rate</code>.
  <h3 class="section">20.8 bench, abench# TOC</h3>

Benchmark part of a filtergraph. The filter accepts the following options: <dl class="table"> <dt><samp class="option">action</samp></dt>
<dd> Start or stop a timer. Available values are: <dl class="table"> <dt>‘<samp class="samp">start</samp>’</dt>
<dd> Get the current time, set it as frame metadata (using the key <code class="code">lavfi.bench.start_time</code>), and forward the frame to the next filter. </dd> <dt>‘<samp class="samp">stop</samp>’</dt>
<dd> Get the current time and fetch the <code class="code">lavfi.bench.start_time</code> metadata from the input frame metadata to get the time difference. Time difference, average, maximum and minimum time (respectively <code class="code">t</code>, <code class="code">avg</code>, <code class="code">max</code> and <code class="code">min</code>) are then printed. The timestamps are expressed in seconds. </dd> </dl> </dd>
</dl>

 <h4 class="subsection">20.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Benchmark selectivecolor

filter: <pre class=”example-preformatted”>bench=start,selectivecolor=reds=-.2 .12 -.49,bench=stop </pre>

 <h3 class=”section”>20.9 concat# TOC</h3>

Concatenate audio and video streams, joining them together one after the other. The filter works on segments of synchronized video and audio streams. All segments must have the same number of streams of each type, and that will also be the number of streams at output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of segments. Default is 2. </dd> <dt><samp class=”option”>v</samp></dt> <dd> Set the number of output video streams, that is also the number of video streams in each segment. Default is 1. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Set the number of output audio streams, that is also the number of audio streams in each segment. Default is 0. </dd> <dt><samp class=”option”>unsafe</samp></dt> <dd> Activate unsafe mode: do not fail if segments have a different format. </dd> </dl>

The filter has <var class=”var”>v</var>+<var class=”var”>a</var> outputs: first <var class=”var”>v</var> video outputs, then <var class=”var”>a</var> audio outputs. There are <var class=”var”>n</var>x(<var class=”var”>v</var>+<var class=”var”>a</var>) inputs: first the inputs for the first segment, in the same order as the outputs, then the inputs for the second segment, etc. Related streams do not always have exactly the same duration, for various reasons including codec frame size or sloppy authoring. For that reason, related synchronized streams (e.g. a video and its audio track) should be concatenated at once. The concat filter will use the duration of the longest stream in each segment (except the last one), and if necessary pad shorter audio streams with silence. For this filter to work correctly, all segments must start at timestamp 0. All corresponding streams must have the same parameters in all segments; the filtering system will automatically select a common pixel format for video streams, and a common sample format, sample rate and channel layout for audio streams, but other settings, such as resolution, must be converted explicitly by the user. Different frame rates are acceptable but will result in variable frame rate at output; be sure to configure the output file to handle it. <h4 class=”subsection”>20.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Concatenate an opening, an episode and an ending, all in bilingual version (video in stream 0, audio in streams 1 and 2):

 <pre class="example-preformatted">ffmpeg -i opening.mkv -i episode.mkv -i ending.mkv -filter_complex \ '[0:0] [0:1] [0:2] [1:0] [1:1] [1:2] [2:0] [2:1] [2:2] concat=n=3:v=1:a=2 [v] [a1] [a2]' \ -map '[v]' -map '[a1]' -map '[a2]' output.mkv </pre>

Concatenate two parts, handling audio and video separately, using the (a)movie sources, and adjusting the resolution: <pre class="example-preformatted">movie=part1.mp4, scale=512:288 [v1] ; amovie=part1.mp4 [a1] ; movie=part2.mp4, scale=512:288 [v2] ; amovie=part2.mp4 [a2] ; [v1] [v2] concat [outv] ; [a1] [a2] concat=v=0:a=1 [outa] </pre> Note that a desync will happen at the stitch if the audio and video streams do not have exactly the same duration in the first file.

 <h4 class="subsection">20.9.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">next</samp></dt> <dd> Close the current segment and step to the next one </dd> </dl>

 <h3 class="section">20.10 ebur128# TOC</h3>

EBU R128 scanner filter. This filter takes an audio stream and analyzes its loudness level. By default, it logs a message at a frequency of 10Hz with the Momentary loudness (identified by <code class="code">M</code>), Short-term loudness (<code class="code">S</code>), Integrated loudness (<code class="code">I</code>) and Loudness Range (<code class="code">LRA</code>). The filter can only analyze streams which have sample format is double-precision floating point. The input stream will be converted to this specification, if needed. Users may need to insert aformat and/or aresample filters after this filter to obtain the original parameters. The filter also has a video output (see the <var class="var">video</var> option) with a real time graph to observe the loudness evolution. The graphic contains the logged message mentioned above, so it is not printed anymore when this option is set, unless the verbose logging is set. The main graphing area contains the short-term loudness (3 seconds of analysis), and the gauge on the right is for the momentary loudness (400 milliseconds), but can optionally be configured to instead display short-term loudness (see <var class="var">gauge</var>). The green area marks a +/- 1LU target range around the target loudness (-23LUFS by default, unless modified through <var class="var">target</var>). More information about the Loudness Recommendation EBU R128 on <a class="url"

href=”http://tech.ebu.ch/loudness”>http://tech.ebu.ch/loudness. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>video</samp></dt> <dd> Activate the video output. The audio stream is passed unchanged whether this option is set or no. The video stream will be the first output stream if activated. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set the video size. This option is for video only. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default and minimum resolution is <code class=”code”>640x480</code>. </dd> <dt><samp class=”option”>meter</samp></dt> <dd> Set the EBU scale meter. Default is <code class=”code”>9</code>. Common values are <code class=”code”>9</code> and <code class=”code”>18</code>, respectively for EBU scale meter +9 and EBU scale meter +18. Any other integer value between this range is allowed. </dd> <dt><samp class=”option”>metadata</samp></dt> <dd> Set metadata injection. If set to <code class=”code”>1</code>, the audio input will be segmented into 100ms output frames, each of them containing various loudness information in metadata. All the metadata keys are prefixed with <code class=”code”>lavfi.r128.</code>. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>framelog</samp></dt> <dd> Force the frame logging level. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>quiet</samp>’</dt> <dd> logging disabled </dd> <dt>‘<samp class=”samp”>info</samp>’</dt> <dd> information logging level </dd> <dt>‘<samp class=”samp”>verbose</samp>’</dt> <dd> verbose logging level </dd> </dl>

By default, the logging level is set to <var class=”var”>info</var>. If the <samp class=”option”>video</samp> or the <samp class=”option”>metadata</samp> options are set, it switches to <var class=”var”>verbose</var>. </dd> <dt><samp class=”option”>peak</samp></dt> <dd> Set peak mode(s). Available modes can be cumulated (the option is a <code class=”code”>flag</code> type). Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disable any peak mode (default). </dd> <dt>‘<samp class=”samp”>sample</samp>’</dt> <dd> Enable sample-peak mode. Simple peak mode looking for the higher sample value. It logs a message for sample-peak (identified by <code class=”code”>SPK</code>). </dd> <dt>‘<samp class=”samp”>true</samp>’</dt> <dd> Enable true-peak mode. If enabled, the peak lookup is done on an over-sampled version of the input stream for better peak accuracy. It logs a message for true-peak. (identified by <code class=”code”>TPK</code>) and true-peak per frame (identified by <code class=”code”>FTPK</code>). This mode requires a build with <code

class=”code”>libswresample</code>. </dd> </dl>

</dd> <dt><samp class=”option”>dualmono</samp></dt> <dd> Treat mono input files as "dual mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. </dd> <dt><samp class=”option”>panlaw</samp></dt> <dd> Set a specific pan law to be used for the measurement of dual mono files. This parameter is optional, and has a default value of -3.01dB. </dd> <dt><samp class=”option”>target</samp></dt> <dd> Set a specific target level (in LUFS) used as relative zero in the visualization. This parameter is optional and has a default value of -23LUFS as specified by EBU R128. However, material published online may prefer a level of -16LUFS (e.g. for use with podcasts or video platforms). </dd> <dt><samp class=”option”>gauge</samp></dt> <dd> Set the value displayed by the gauge. Valid values are <code class=”code”>momentary</code> and s <code class=”code”>shortterm</code>. By default the momentary value will be used, but in certain scenarios it may be more useful to observe the short term value instead (e.g. live mixing). </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Sets the display scale for the loudness. Valid parameters are <code class=”code”>absolute</code> (in LUFS) or <code class=”code”>relative</code> (LU) relative to the target. This only affects the video output, not the summary or continuous log output. </dd> <dt><samp class=”option”>integrated</samp></dt> <dd> Read-only exported value for measured integrated loudness, in LUFS. </dd> <dt><samp class=”option”>range</samp></dt> <dd> Read-only exported value for measured loudness range, in LU. </dd> <dt><samp class=”option”>lra_low</samp></dt> <dd> Read-only exported value for measured LRA low, in LUFS. </dd> <dt><samp class=”option”>lra_high</samp></dt> <dd> Read-only exported value for measured LRA high, in LUFS. </dd> <dt><samp class=”option”>sample_peak</samp></dt> <dd> Read-only exported value for measured sample peak, in dBFS. </dd> <dt><samp class=”option”>true_peak</samp></dt> <dd> Read-only exported value for measured true peak, in dBFS. </dd> </dl>

 <h4 class=”subsection”>20.10.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Real-time graph using <code class=”command”>ffplay</code>, with a EBU scale meter +18: <pre class=”example-preformatted”>ffplay -f lavfi -i "amovie=input.mp3,ebur128=video=1:meter=18 [out0][out1]" </pre>

Run an analysis with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -nostats -i input.mp3 -filter_complex ebur128 -f null - </pre>

 <h3 class="section">20.11 interleave, ainterleave# TOC</h3>

Temporally interleave frames from several inputs. <code class="code">interleave</code> works with video inputs, <code class="code">ainterleave</code> with audio. These filters read frames from several inputs and send the oldest queued frame to the output. Input streams must have well defined, monotonically increasing frame timestamp values. In order to submit one frame to output, these filters need to enqueue at least one frame for each input, so they cannot work in case one input is not yet terminated and will not receive incoming frames. For example consider the case when one input is a <code class="code">select</code> filter which always drops input frames. The <code class="code">interleave</code> filter will keep reading from that input, but it will never be able to send new frames to output until the input sends an end-of-stream signal. Also, depending on inputs synchronization, the filters will drop frames in case one input receives more frames than the other ones, and the queue is already filled. These filters accept the following options: <dl class="table"> <dt><samp class="option">nb_inputs, n</samp></dt> <dd> Set the number of different inputs, it is 2 by default. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class="option">first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">20.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interleave frames belonging to different streams using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i bambi.avi -i pr0n.mkv -filter_complex "[0:v][1:v] interleave" out.avi </pre>

Add flickering blur effect: <pre

class=”example-preformatted”>select='if(gt(random(0), 0.2), 1, 2)':n=2 [tmp], boxblur=2:2, [tmp] interleave </pre>

 <h3 class=”section”>20.12 latency, alatency# TOC</h3> Measure filtering latency. Report previous filter filtering latency, delay in number of audio samples for audio filters or number of video frames for video filters. On end of input stream, filter will report min and max measured latency for previous running filter in filtergraph. <h3 class=”section”>20.13 metadata, ametadata# TOC</h3>

Manipulate frame metadata. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>select</samp>’</dt> <dd> If both <code class=”code”>value</code> and <code class=”code”>key</code> is set, select frames which have such metadata. If only <code class=”code”>key</code> is set, select every frame that has such key in metadata. </dd> <dt>‘<samp class=”samp”>add</samp>’</dt> <dd> Add new metadata <code class=”code”>key</code> and <code class=”code”>value</code>. If key is already available do nothing. </dd> <dt>‘<samp class=”samp”>modify</samp>’</dt> <dd> Modify value of already present key. </dd> <dt>‘<samp class=”samp”>delete</samp>’</dt> <dd> If <code class=”code”>value</code> is set, delete only keys that have such value. Otherwise, delete key. If <code class=”code”>key</code> is not set, delete all metadata values in the frame. </dd> <dt>‘<samp class=”samp”>print</samp>’</dt> <dd> Print key and its value if metadata was found. If <code class=”code”>key</code> is not set print all metadata values available in frame. </dd> </dl>

</dd> <dt><samp class=”option”>key</samp></dt> <dd> Set key used with all modes. Must be set for all modes except <code class=”code”>print</code> and <code class=”code”>delete</code>. </dd> <dt><samp class=”option”>value</samp></dt> <dd> Set metadata value which will be used. This option is mandatory for <code class=”code”>modify</code> and <code class=”code”>add</code> mode. </dd> <dt><samp class=”option”>function</samp></dt> <dd> Which function to use when comparing metadata value and <code class=”code”>value</code>. Can be one of following: <dl class=”table”> <dt>‘<samp class=”samp”>same_str</samp>’</dt>

<dd> Values are interpreted as strings, returns true if metadata value is same as <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>starts_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value starts with the <code class=”code”>value</code> option string. </dd> <dt>‘<samp class=”samp”>less</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is less than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>equal</samp>’</dt> <dd> Values are interpreted as floats, returns true if <code class=”code”>value</code> is equal with metadata value. </dd> <dt>‘<samp class=”samp”>greater</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is greater than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>expr</samp>’</dt> <dd> Values are interpreted as floats, returns true if expression from option <code class=”code”>expr</code> evaluates to true. </dd> <dt>‘<samp class=”samp”>ends_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value ends with the <code class=”code”>value</code> option string. </dd> </dl>

</dd> <dt><samp class=”option”>expr</samp></dt> <dd> Set expression which is used when <code class=”code”>function</code> is set to <code class=”code”>expr</code>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>VALUE1, FRAMEVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> from metadata key. </dd> <dt><samp class=”option”>VALUE2, USERVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> as supplied by user in <code class=”code”>value</code> option. </dd> </dl>

</dd> <dt><samp class=”option”>file</samp></dt> <dd> If specified in <code class=”code”>print</code> mode, output is written to the named file. Instead of plain filename any writable url can be specified. Filename “-” is a shorthand for standard output. If <code class=”code”>file</code> option is not set, output is written to the log with AV_LOG_INFO loglevel. </dd> <dt><samp class=”option”>direct</samp></dt> <dd> Reduces buffering in print mode when output is written to a URL set using <var class=”var”>file</var>. </dd> </dl>

 <h4 class=”subsection”>20.13.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print all metadata values for frames with key <code class=”code”>lavfi.signalstats.YDIF</code> with val-

ues between 0 and 1. <pre class=”example-preformatted”>signalstats,metadata=print:key=lavfi.signalstats.YDIF:value=0:function=expr:expr='between
</pre> Print silencedetect output to file <samp class=”file”>metadata.txt</samp>.
 <pre class=”example-preformatted”>silencedetect,ametadata=mode=print:file=meta
</pre> Direct all metadata to a pipe with file descriptor 4.
 <pre class=”example-preformatted”>metadata=mode=print:file='pipe\:4'
</pre>

 <h3 class=”section”>20.14 perms, aperms# TOC</h3>

Set read/write permissions for the output frames. These filters are mainly aimed at developers to test direct path in the following filter in the filtergraph. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt>
<dd> Select the permissions mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt>
<dd> Do nothing. This is the default. </dd> <dt>‘<samp class=”samp”>ro</samp>’</dt> <dd> Set all the output frames read-only. </dd> <dt>‘<samp class=”samp”>rw</samp>’</dt>
<dd> Set all the output frames directly writable. </dd>
<dt>‘<samp class=”samp”>toggle</samp>’</dt> <dd> Make the frame read-only if writable, and writable if read-only. </dd>
<dt>‘<samp class=”samp”>random</samp>’</dt> <dd> Set each output frame read-only or writable randomly. </dd> </dl>

</dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set the seed for the <var class=”var”>random</var> mode, must be an integer included between <code class=”code”>0</code> and <code class=”code”>UINT32_MAX</code>. If not specified, or if explicitly set to <code class=”code”>-1</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

Note: in case of auto-inserted filter between the permission filter and the following one, the permission might not be received as expected in that following filter. Inserting a format or aformat filter before the perms/aperms filter can avoid this problem. <h3 class=”section”>20.15 realtime, arealtime# TOC</h3>

Slow down filtering to match real time approximately. These filters will pause the filtering for a variable amount of time to match the output rate with the input timestamps. They are similar to the <samp

class=”option”>re</samp> option to <code class=”code”>ffmpeg</code>. They accept the following options: <dl class=”table”> <dt><samp class=”option”>limit</samp></dt> <dd> Time limit for the pauses. Any pause longer than that will be considered a timestamp discontinuity and reset the timer. Default is 2 seconds. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Speed factor for processing. The value must be a float larger than zero. Values larger than 1.0 will result in faster than realtime processing, smaller will slow processing down. The <var class=”var”>limit</var> is automatically adapted accordingly. Default is 1.0. A processing speed faster than what is possible without these filters cannot be achieved. </dd> </dl>

 <h4 class=”subsection”>20.15.1 Commands# TOC</h4>

Both filters supports the all above options as commands. <h3 class=”section”>20.16 segment, asegment# TOC</h3

Split single input stream into multiple streams. This filter does opposite of concat filters. <code class=”code”>segment</code> works on video frames, <code class=”code”>asegment</code> on audio samples. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>timestamps</samp></dt> <dd> Timestamps of output segments separated by ’|’. The first segment will run from the beginning of the input stream. The last segment will run until the end of the input stream </dd> <dt><samp class=”option”>frames, samples</samp></dt> <dd> Exact frame/sample count to split the segments. </dd> </dl>

In all cases, prefixing an each segment with ’+’ will make it relative to the previous segment. <h4 class=”subsection”>20.16.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Split input audio stream into three output audio streams, starting at start of input audio stream and storing that in 1st output audio stream, then following at 60th second and storing than in 2nd output audio stream, and last after 150th second of input audio stream store in 3rd output audio stream: <pre class=”example-preformatted”>asegment=timestamps="60|150" </pre>

<h3 class=”section”>20.17 select, aselect<a
class=”anchor hidden-xs” href=”#select_002c-aselect” aria-hidden=”true”>#
TOC</h3>

Select frames to pass in output. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>expr, e</samp></dt> <dd> Set expression, which is evaluated for each input frame. If the expression is evaluated to zero, the frame is discarded. If the evaluation result is negative or NaN, the frame is sent to the first output; otherwise it is sent to the output with index <code class=”code”>ceil(val)-1</code>, assuming that the input index starts from 0. For example a value of <code class=”code”>1.2</code> corresponds to the output with index <code class=”code”>ceil(1.2)-1 = 2-1 = 1</code>, that is the second output. </dd> <dt><samp class=”option”>outputs, n</samp></dt> <dd> Set the number of outputs. The output to which to send the selected frame is based on the result of the evaluation. Default value is 1. </dd> </dl>

The expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the filtered frame, starting from 0. </dd> <dt><samp class=”option”>selected_n</samp></dt> <dd> The (sequential) number of the selected frame, starting from 0. </dd> <dt><samp class=”option”>prev_selected_n</samp></dt> <dd> The sequential number of the last selected frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the filtered frame, expressed in <var class=”var”>TB</var> units. It’s NAN if undefined. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The PTS of the filtered frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_pts</samp></dt> <dd> The PTS of the previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_pts</samp></dt> <dd> The PTS of the last previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_t</samp></dt> <dd> The PTS of the last previously selected frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> The first PTS in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>start_t</samp></dt> <dd> The first PTS, in seconds, in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>pict_type (video only)</samp></dt> <dd> The type of the filtered frame. It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>I</samp></dt> <dt><samp

class=”option”>P</samp></dt> <dt><samp class=”option”>B</samp></dt>
<dt><samp class=”option”>S</samp></dt> <dt><samp class=”option”>SI</samp></dt>
<dt><samp class=”option”>SP</samp></dt> <dt><samp class=”option”>BI</samp></dt>
</dl>

</dd> <dt><samp class=”option”>interlace_type (video
only)</samp></dt> <dd> The frame interlace type. It can assume one of the following values: <dl class=”table”> <dt><samp
class=”option”>PROGRESSIVE</samp></dt> <dd> The frame is progressive (not interlaced). </dd> <dt><samp class=”option”>TOPFIRST</samp></dt>
<dd> The frame is top-field-first. </dd> <dt><samp class=”option”>BOTTOMFIRST</samp><
<dd> The frame is bottom-field-first. </dd> </dl>

</dd> <dt><samp class=”option”>consumed_sample_n (audio
only)</samp></dt> <dd> the number of selected samples before
the current frame </dd> <dt><samp class=”option”>samples_n
(audio only)</samp></dt> <dd> the
number of samples in the current frame </dd> <dt><samp
class=”option”>sample_rate (audio only)</samp></dt>
<dd> the input sample rate </dd> <dt><samp class=”option”>key</samp></dt>
<dd> This is 1 if the filtered frame is a key-frame, 0 otherwise.
</dd> <dt><samp class=”option”>pos</samp></dt> <dd> the
position in the file of the filtered frame, -1 if the information is not
available (e.g. for synthetic video); deprecated, do not use
</dd> <dt><samp class=”option”>scene (video
only)</samp></dt> <dd> value between 0 and 1 to indicate
a new scene; a low value reflects a low probability for the current frame
to introduce a new scene, while a higher value means the current frame is
more likely to be one (see the example below) </dd> <dt><samp
class=”option”>concatdec_select</samp></dt> <dd> The concat
demuxer can select only part of a concat input file by setting an inpoint
and an outpoint, but the output packets may not be entirely contained
in the selected interval. By using this variable, it is possible to skip
frames generated by the concat demuxer which are not exactly contained
in the selected interval. This works by comparing the frame
pts against the <var class=”var”>lavf.concat.start_time</var> and the
<var class=”var”>lavf.concat.duration</var> packet metadata values
which are also present in the decoded frames. The <var
class=”var”>concatdec_select</var> variable is -1 if the frame pts is at least
start_time and either the duration metadata is missing or the frame pts is
less than start_time + duration, 0 otherwise, and NaN if the start_time
metadata is missing. That basically means that an input frame
is selected if its pts is within the interval set by the concat demuxer.
 </dd> <dt><samp class=”option”>iw (video
only)</samp></dt> <dd> Represents the width of the input video frame. </dd> <dt><samp class=”option”>ih <em
class=”emph”>(video only)</samp></dt> <dd> Represents the

height of the input video frame. </dd> <dt><samp class="option">view (video only)</samp></dt> <dd> View ID for multi-view video. </dd> </dl>

The default value of the select expression is "1". <h4 class="subsection">20.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select all frames in input: <pre class="example-preformatted">select </pre>

The example above is the same as: <pre class="example-preformatted">select=1 </pre>

Skip all frames: <pre class="example-preformatted">select=0 </pre>

Select only I-frames: <pre class="example-preformatted">select='eq(pict_type\,I)' </pre>

Select one frame every 100: <pre class="example-preformatted">select='not(mod(n\,100))' </pre>

Select only frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20) </pre>

Select only I-frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20)*eq(pict_type\,I) </pre>

Select frames with a minimum distance of 10 seconds: <pre class="example-preformatted">select='isnan(prev_selected_t)+gte(t-prev_selected_t\,10)' </pre>

Use aselect to select only audio frames with samples number > 100: <pre class="example-preformatted">aselect='gt(samples_n\,100)' </pre>

Create a mosaic of the first scenes: <pre class="example-preformatted">ffmpeg -i video.avi -vf select='gt(scene\,0.4)',scale=160:120,tile -frames:v 1 preview.png </pre>

Comparing <var class="var">scene</var> against a value between 0.3 and 0.5 is generally a sane choice. Send even and odd frames to separate outputs, and compose them: <pre class="example-preformatted">select=n=2:e='mod(n, 2)+1' [odd][even]; [odd] pad=h=2*ih [tmp]; [tmp][even] overlay=y=h </pre>

Select useful frames from an ffconcat file which is using inpoints and outpoints but where the source files are not intra frame only. <pre class="example-preformatted">ffmpeg -copyts -segment_time_metadata 1 -i input.ffconcat -fps_mode passthrough -vf select=concatdec_select -af aselect=concatdec_select output.avi </pre>

 <h3 class="section">20.18 sendcmd, asendcmd# TOC</h3>

Send commands to filters in the filtergraph. These filters read commands to be sent to other filters in the filtergraph. <code class="code">sendcmd</code> must be inserted between two video filters, <code class="code">asendcmd</code> must be inserted between two audio filters, but apart from that they act the same way. The specification of commands can be provided in the filter arguments with the <var class="var">commands</var> option, or in a file specified by the <var class="var">filename</var> option. These filters accept the following options: <dl class="table"> <dt><samp class="option">commands, c</samp></dt> <dd> Set the commands to be read and sent to the other filters. </dd> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the commands to be read and sent to the other filters. </dd> </dl>

 <h4 class="subsection">20.18.1 Commands syntax# TOC</h4>

A commands description consists of a sequence of interval specifications, comprising a list of commands to be executed when a particular event related to that interval occurs. The occurring event is typically the current frame time entering or leaving a given time interval. An interval is specified by the following syntax: <pre class="example-preformatted"><var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var>; </pre>

The time interval is specified by the <var class="var">START</var> and <var class="var">END</var> times. <var class="var">END</var> is optional and defaults to the maximum time. The current frame time is considered within the specified interval if it is included in the interval [<var class="var">START</var>, <var class="var">END</var>), that is when the time is greater or equal to <var class="var">START</var> and is lesser than <var class="var">END</var>. <var

class=”var”>COMMANDS</var> consists of a sequence of one or more command specifications, separated by ",", relating to that interval. The syntax of a command specification is given by: <pre class=”example-preformatted”>[<var class=”var”>FLAGS</var>] <var class=”var”>TARGET</var> <var class=”var”>COMMAND</var> <var class=”var”>ARG</var> </pre>

<var class=”var”>FLAGS</var> is optional and specifies the type of events relating to the time interval which enable sending the specified command, and must be a non-null sequence of identifier flags separated by "+" or "|" and enclosed between "[" and "]". The following flags are recognized: <dl class=”table”> <dt><samp class=”option”>enter</samp></dt> <dd> The command is sent when the current frame timestamp enters the specified interval. In other words, the command is sent when the previous frame timestamp was not in the given interval, and the current is. </dd> <dt><samp class=”option”>leave</samp></dt> <dd> The command is sent when the current frame timestamp leaves the specified interval. In other words, the command is sent when the previous frame timestamp was in the given interval, and the current is not. </dd> <dt><samp class=”option”>expr</samp></dt> <dd> The command <var class=”var”>ARG</var> is interpreted as expression and result of expression is passed as <var class=”var”>ARG</var>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>POS</samp></dt> <dd> Original position in the file of the frame, or undefined if undefined for the current frame. Deprecated, do not use. </dd> <dt><samp class=”option”>PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class=”option”>N</samp></dt> <dd> The count of the input frame for video or audio, starting from 0. </dd> <dt><samp class=”option”>T</samp></dt> <dd> The time in seconds of the current frame. </dd> <dt><samp class=”option”>TS</samp></dt> <dd> The start time in seconds of the current command interval. </dd> <dt><samp class=”option”>TE</samp></dt> <dd> The end time in seconds of the current command interval. </dd> <dt><samp class=”option”>TI</samp></dt> <dd> The interpolated time of the current command interval, TI = (T - TS) / (TE - TS). </dd> <dt><samp class=”option”>W</samp></dt> <dd> The video frame width. </dd> <dt><samp class=”option”>H</samp></dt> <dd> The video frame height. </dd> </dl>

</dd> </dl>

If <var class=”var”>FLAGS</var> is not specified, a default value of <code class=”code”>[enter]</code> is assumed. <var class=”var”>TARGET</var> specifies the target of the command, usu-

ally the name of the filter class or a specific filter instance name. <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional list of argument for the given <var class="var">COMMAND</var>. Between one interval specification and another, whitespaces, or sequences of characters starting with <code class="code">#</code> until the end of line, are ignored and can be used to annotate comments. A simplified BNF description of the commands specification syntax follows: <pre class="example-preformatted"><var class="var">COMMAND_FLAG</var> ::= "enter" | "leave" <var class="var">COMMAND_FLAGS</var> ::= <var class="var">COMMAND_FLAG</var> [(+|"|")<var class="var">COMMAND_FLAG</var>] <var class="var">COMMAND</var> ::= ["[" <var class="var">COMMAND_FLAGS</var> "]"] <var class="var">TARGET</var> <var class="var">COMMAND</var> [<var class="var">ARG</var>] <var class="var">COMMANDS</var> ::= <var class="var">COMMAND</var> [,<var class="var">COMMANDS</var>] <var class="var">INTERVAL</var> ::= <var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var> <var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[;<var class="var">INTERVALS</var>] </pre>

 <h4 class="subsection">20.18.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Specify audio tempo change at second 4: <pre class="example-preformatted">asendcmd=c='4.0 atempo tempo 1.5',atempo </pre>

Target a specific filter instance: <pre class="example-preformatted">asendcmd=c='4.0 atempo@my tempo 1.5',atempo@my </pre>

Specify a list of drawtext and hue commands in a file. <pre class="example-preformatted"># show text in the interval 5-10 5.0-10.0 [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=hello world', [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=';

desaturate the image in the interval 15-20 15.0-20.0 [enter] hue s 0, [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=nocolor', [leave] hue s 1, [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=color';

apply an exponential saturation fade-out effect, starting from time 25 25 [enter] hue s exp(25-t) </pre>

A filtergraph allowing to read and process the above command list stored in a file <samp class="file">test.cmd</samp>, can be

specified with: <pre class="example-preformatted">sendcmd=f=test.cmd,drawtext=fontfile=FreeSerif.ttf:text='',hue
</pre>

<h3 class="section">20.19 setpts, asetpts<a
class="anchor hidden-xs" href="#setpts_002c-asetpts" aria-hidden="true">#
TOC</h3>

Change the PTS (presentation timestamp) of the input frames.
<code class="code">setpts</code> works on video frames, <code
class="code">asetpts</code> on audio frames. This filter
accepts the following options: <dl class="table"> <dt><samp
class="option">expr</samp></dt> <dd> The expression which is evaluated for each frame to construct its timestamp. </dd> <dt><samp
class="option">strip_fps (video only)</samp></dt>
<dd> Boolean option which determines if the original framerate and frame duration metadata is unset. If set to true, be advised that a sane frame rate should be explicitly specified if output is sent to a constant frame rate muxer. Default is <code class="code">false</code>. </dd> </dl>

The expression is evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp
class="option">FRAME_RATE, FR</samp></dt> <dd> frame rate, only defined for constant frame-rate video </dd> <dt><samp
class="option">PTS</samp></dt> <dd> The presentation timestamp in input </dd> <dt><samp class="option">N</samp></dt>
<dd> The count of the input frame for video or the number of consumed samples, not including the current frame for audio, starting from 0.
</dd> <dt><samp class="option">NB_CONSUMED_SAMPLES</samp></dt>
<dd> The number of consumed samples, not including the current frame (only audio) </dd> <dt><samp class="option">NB_SAMPLES, S</samp></dt> <dd> The number of samples in the current frame (only audio) </dd> <dt><samp class="option">SAMPLE_RATE, SR</samp></dt> <dd> The audio sample rate. </dd>
<dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first frame. </dd> <dt><samp class="option">STARTT</samp></dt>
<dd> the time in seconds of the first frame </dd> <dt><samp
class="option">INTERLACED</samp></dt> <dd> State whether the current frame is interlaced. </dd> <dt><samp class="option">T</samp></dt>
<dd> the time in seconds of the current frame </dd> <dt><samp
class="option">POS</samp></dt> <dd> original position in the file of the frame, or undefined if undefined for the current frame; deprecated, do not use
 </dd> <dt><samp class="option">PREV_INPTS</samp></dt>
<dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INT</samp></dt>
<dd> previous input time in seconds </dd> <dt><samp
class="option">PREV_OUTPTS</samp></dt> <dd> The previous

output PTS. </dd> <dt><samp class="option">PREV_OUTT</samp></dt> <dd> previous output time in seconds </dd> <dt><samp class="option">RTCTIME</samp></dt> <dd> The wallclock (RTC) time in microseconds. This is deprecated, use time(0) instead. </dd> <dt><samp class="option">RTCSTART</samp></dt> <dd> The wallclock (RTC) time at the start of the movie in microseconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class="option">T_CHANGE</samp></dt> <dd> Time of the first frame after command was applied or time of the first frame if no commands. </dd> </dl>

 <h4 class="subsection">20.19.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Start counting PTS from zero <pre class="example-preformatted">setpts=PTS-STARTPTS </pre>

Apply fast motion effect: <pre class="example-preformatted">setpts=0.5*PTS </pre>

Apply slow motion effect: <pre class="example-preformatted">setpts=2.0*PTS </pre>

Set fixed rate of 25 frames per second: <pre class="example-preformatted">setpts=N/(25*TB) </pre>

Apply a random jitter effect of +/-100 TB units: <pre class="example-preformatted">setpts=PTS+randomi(0, -100\,100) </pre>

Set fixed rate 25 fps with some jitter: <pre class="example-preformatted">setpts='1/(25*TB) * (N + 0.05 * sin(N*2*PI/25))' </pre>

Apply an offset of 10 seconds to the input PTS: <pre class="example-preformatted">setpts=PTS+10/TB </pre>

Generate timestamps from a "live source" and rebase onto the current timebase: <pre class="example-preformatted">setpts='(RTCTIME - RTCSTART) / (TB * 1000000)' </pre>

Generate timestamps by counting samples: <pre class="example-preformatted">asetpts=N/SR/TB </pre>

 <h4 class="subsection">20.19.2 Commands# TOC</h4>

Both filters support all above options as commands. <h3 class="section">20.20 setrange# TOC</h3>

Force color range for the output video frame. The <code class="code">setrange</code> filter marks the color range property for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters. The filter accepts the following options: <dl class="table"> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property. </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Set the color range as unspecified. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Set the color range as limited. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Set the color range as full. </dd> </dl> </dd> </dl>

 <h3 class="section">20.21 settb, asettb# TOC</h3>

Set the timebase to use for the output frames timestamps. It is mainly useful for testing timebase configuration. It accepts the following parameters: <dl class="table"> <dt><samp class="option">expr, tb</samp></dt> <dd> The expression which is evaluated into the output timebase. </dd> </dl>

The value for <samp class="option">tb</samp> is an arithmetic expression representing a rational. The expression can contain the constants "AVTB" (the default timebase), "intb" (the input timebase) and "sr" (the sample rate, audio only). Default value is "intb". <h4 class="subsection">20.21.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the timebase to 1/25: <pre class="example-preformatted">settb=expr=1/25 </pre>

Set the timebase to 1/10: <pre class="example-preformatted">settb=expr=0.1 </pre>

Set the timebase to 1001/1000: <pre class="example-preformatted">settb=1+0.001 </pre>

Set the timebase to 2*intb: <pre class="example-preformatted">settb=2*intb </pre>

Set the default timebase value: <pre class="example-preformatted">settb=AVTB </pre>

<h3 class="section">20.22 showcqt# TOC</h3>

Convert input audio to a video output representing frequency spectrum

logarithmically using Brown-Puckette constant Q transform algorithm with

direct frequency domain coefficient calculation (but the transform itself

is not really constant Q, instead the Q factor is actually variable/clamped),

with musical tone scale, from E0 to D#10.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. It must be even. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">1920x1080</code>.

</dd>

<dt><samp class="option">fps, rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">bar_h</samp></dt>

<dd> Set the bargraph height. It must be even. Default value is <code class="code">-1</code> which

computes the bargraph height automatically.

</dd>

<dt><samp class="option">axis_h</samp></dt>

<dd> Set the axis height. It must be even. Default value is <code class="code">-1</code> which computes

the axis height automatically.

</dd>

<dt><samp class="option">sono_h</samp></dt>

<dd> Set the sonogram height. It must be even. Default value is <code class="code">-1</code> which

computes the sonogram height automatically.

</dd>

<dt><samp class="option">fullhd</samp></dt>

<dd> Set the fullhd resolution. This option is deprecated, use <var class="var">size</var>, <var class="var">s</var>

instead. Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">sono_v, volume</samp></dt>

<dd> Specify the sonogram volume expression. It can contain variables:

<dl class="table">

<dt><samp class="option">bar_v</samp></dt>

<dd> the <var class="var">bar_v</var> evaluated expression

</dd>

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option
</dd>
</dl>
and functions:
<dl class="table">
<dt><samp class="option">a_weighting(f)</samp></dt>
<dd> A-weighting of equal loudness
</dd>
<dt><samp class="option">b_weighting(f)</samp></dt>
<dd> B-weighting of equal loudness
</dd>
<dt><samp class="option">c_weighting(f)</samp></dt>
<dd> C-weighting of equal loudness.
</dd>
</dl>
Default value is <code class="code">16</code>.

</dd>
<dt><samp class="option">bar_v, volume2</samp></dt>
<dd> Specify the bargraph volume expression. It can contain variables:
<dl class="table">
<dt><samp class="option">sono_v</samp></dt>
<dd> the <var class="var">sono_v</var> evaluated expression
</dd>
<dt><samp class="option">frequency, freq, f</samp></dt>
<dd> the frequency where it is evaluated
</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">a_weighting(f)</samp></dt>

<dd> A-weighting of equal loudness

</dd>

<dt><samp class="option">b_weighting(f)</samp></dt>

<dd> B-weighting of equal loudness

</dd>

<dt><samp class="option">c_weighting(f)</samp></dt>

<dd> C-weighting of equal loudness.

</dd>

</dl>

Default value is <code class="code">sono_v</code>.

</dd>

<dt><samp class="option">sono_g, gamma</samp></dt>

<dd> Specify the sonogram gamma. Lower gamma makes the spectrum more contrast,

higher gamma makes the spectrum having more range. Default value is <code class="code">3</code>.

Acceptable range is <code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_g, gamma2</samp></dt>

<dd> Specify the bargraph gamma. Default value is <code class="code">1</code>. Acceptable range is

<code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_t</samp></dt>

<dd> Specify the bargraph transparency level. Lower value makes the bargraph sharper.

Default value is <code class="code">1</code>. Acceptable range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> Specify the transform timeclamp. At low frequency, there is trade-off between

accuracy in time domain and frequency domain. If timeclamp is lower,

event in time domain is represented more accurately (such as fast bass drum),

otherwise event in frequency domain is represented more accurately

(such as bass guitar). Acceptable range is <code class="code">[0.002, 1]</code>. Default value is <code class="code">0.17</code>.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set attack time in seconds. The default is <code class="code">0</code> (disabled). Otherwise, it

limits future samples by applying asymmetric windowing in time domain, useful

when low latency is required. Accepted range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">basefreq</samp></dt>

<dd> Specify the transform base frequency. Default value is <code class="code">20.01523126408007475</code>,

which is frequency 50 cents below E0. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">endfreq</samp></dt>

<dd> Specify the transform end frequency. Default value is <code class="code">20495.59681441799654</code>,

which is frequency 50 cents above D#10. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">coeffclamp</samp></dt>

<dd> This option is deprecated and ignored.

</dd>

<dt><samp class="option">tlength</samp></dt>

<dd> Specify the transform length in time domain. Use this option to control accuracy

trade-off between time domain and frequency domain at every frequency sample.

It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option.

</dd>

</dl>

Default value is <code class="code">384*tc/(384+tc*f)</code>.

</dd>

<dt><samp class="option">count</samp></dt>

<dd> Specify the transform count for every video frame. Default value is <code class="code">6</code>.

Acceptable range is <code class="code">[1, 30]</code>.

</dd>

<dt><samp class="option">fcount</samp></dt>

<dd> Specify the transform count for every single pixel. Default value is <code class="code">0</code>,

which makes it computed automatically. Acceptable range is <code class="code">[0, 10]</code>.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> Specify font file for use with freetype to draw the axis. If not specified,

use embedded font. Note that drawing with font file or embedded font is not

implemented with custom <var class="var">basefreq</var> and <var class="var">endfreq</var>, use <var class="var">axisfile</var>

option instead.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> Specify fontconfig pattern. This has lower priority than <var class="var">fontfile</var>. The

<code class="code">:</code> in the pattern may be replaced by <code class="code">|</code> to avoid unnecessary

escaping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> Specify font color expression. This is arithmetic expression that should return

integer value 0xRRGGBB. It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">midi(f)</samp></dt>

<dd> midi number of frequency f, some midi numbers: E0(16), C1(24), C2(36), A4(69)

</dd>

<dt><samp class="option">r(x), g(x), b(x)</samp></dt>

<dd> red, green, and blue value of intensity x.

</dd>

</dl>

Default value is <code class="code">st(0, (midi(f)-59.5)/12);

st(1, if(between(ld(0),0,1), 0.5-0.5*cos(2*PI*ld(0)), 0));

r(1-ld(1)) + b(ld(1))</code>.

</dd>

<dt><samp class="option">axisfile</samp></dt>

<dd> Specify image file to draw the axis. This option override <var class="var">fontfile</var> and

<var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">axis, text</samp></dt>

<dd> Enable/disable drawing text to the axis. If it is set to <code class="code">0</code>, drawing to

the axis is disabled, ignoring <var class="var">fontfile</var> and <var class="var">axisfile</var> option.

Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">csp</samp></dt>

<dd> Set colorspace. The accepted values are:

<dl class="table">
<dt>‘<samp class="samp">unspecified</samp>’</dt>
<dd> Unspecified (default)

</dd>
<dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709

</dd>
<dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC

</dd>
<dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625

</dd>
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M or BT.601-6 525

</dd>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M

</dd>
<dt>‘<samp class="samp">bt2020ncl</samp>’</dt>
<dd> BT.2020 with non-constant luminance

</dd>
</dl>

</dd> <dt><samp class="option">cscheme</samp></dt> <dd> Set spectrogram color scheme. This is list of floating point values with format <code class="code">left_r|left_g|left_b|right_r|right_g|right_b</code>. The default is <code class="code">1|0.5|0|0|0.5|1</code>. </dd> </dl>

 <h4 class="subsection">20.22.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Playing audio while showing the spectrum: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with frame rate 30 fps: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=fps=30:count=5 [out0]' </pre>

Playing at 1280x720: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=s=1280x720:count=4 [out0]' </pre>

Disable sonogram display: <pre class="example-preformatted">sono_h=0 </pre>

A1 and its harmonics: A1, A2, (near)E3, A3: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6
asplit[a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with more accuracy in frequency domain: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6*PI*55*t)+0.1*sin(8*PI*55*t), asplit[a][out1]; [a] showcqt=timeclamp=0.5 [out0]' </pre>

Custom volume: <pre class="example-preformatted">bar_v=10:sono_v=bar_v*a_weighting(f) </pre>

Custom gamma, now spectrum is linear to the amplitude. <pre class="example-preformatted">bar_g=2:sono_g=2 </pre>

Custom tlength equation: <pre class="example-preformatted">tc=0.33:tlength='st(0,0.17); 384*tc / (384 / ld(0) + tc*f /(1-ld(0))) + 384*tc / (tc*f / ld(0) + 384 /(1-ld(0)))' </pre>

Custom fontcolor and fontfile, C-note is colored green, others are colored blue: <pre class="example-preformatted">fontcolor='if(mod(floor(midi(f)+0.5),12), 0x0000FF, g(1))':fontfile=myfont.ttf </pre>

Custom font using fontconfig: <pre class="example-preformatted">font='Courier New,Monospace,mono|bold'

</pre>

Custom frequency range with custom axis using image file: <pre class="example-preformatted">axisfile=myaxis.png:basefreq=40:endfreq=10000
</pre>

 <h3 class="section">20.23 showcwt# TOC</h3>

Convert input audio to video output representing frequency spectrum

using Continuous Wavelet Transform and Morlet wavelet.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">640x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set the frequency scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">bark</samp></dt>

<dt><samp class="option">mel</samp></dt>

<dt><samp class="option">erbs</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

<dt><samp class="option">fm</samp></dt>

</dl>

Default value is <code class="code">linear</code>.

</dd>

<dt><samp class="option">iscale</samp></dt>

<dd> Set the intensity scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

</dl>

Default value is <code class="code">log</code>.

</dd>

<dt><samp class="option">min</samp></dt>

<dd> Set the minimum frequency that will be used in output.

Default is <code class="code">20</code> Hz.

</dd>

<dt><samp class="option">max</samp></dt>

<dd> Set the maximum frequency that will be used in output.

Default is <code class="code">20000</code> Hz. The real frequency upper limit

depends on input audio’s sample rate and such will be enforced
on this value when it is set to value greater than Nyquist frequency.

</dd>
<dt><samp class="option">imin</samp></dt>
<dd> Set the minimum intensity that will be used in output.

</dd>
<dt><samp class="option">imax</samp></dt>
<dd> Set the maximum intensity that will be used in output.

</dd>
<dt><samp class="option">logb</samp></dt>
<dd> Set the logarithmic basis for brightness strength when
mapping calculated magnitude values to pixel values.
Allowed range is from <code class="code">0</code> to <code class="code">1</code>.
Default value is <code class="code">0.0001</code>.

</dd>
<dt><samp class="option">deviation</samp></dt>
<dd> Set the frequency deviation.
Lower values than <code class="code">1</code> are more frequency oriented,
while higher values than <code class="code">1</code> are more time oriented.
Allowed range is from <code class="code">0</code> to <code class="code">10</code>.
Default value is <code class="code">1</code>.

</dd>
<dt><samp class="option">pps</samp></dt>
<dd> Set the number of pixel output per each second in one row.
Allowed range is from <code class="code">1</code> to <code class="code">1024</code>.

Default value is <code class=”code”>64</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set the output visual mode. Allowed values are:

<dl class=”table”>

<dt><samp class=”option”>magnitude</samp></dt>

<dd> Show magnitude.

</dd>

<dt><samp class=”option”>phase</samp></dt>

<dd> Show only phase.

</dd>

<dt><samp class=”option”>magphase</samp></dt>

<dd> Show combination of magnitude and phase.

Magnitude is mapped to brightness and phase to color.

</dd>

<dt><samp class=”option”>channel</samp></dt>

<dd> Show unique color per channel magnitude.

</dd>

<dt><samp class=”option”>stereo</samp></dt>

<dd> Show unique color per stereo difference.

</dd>

</dl>

Default value is <code class=”code”>magnitude</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set the output slide method. Allowed values are: <dl class=”table”> <dt><samp class=”option”>replace</samp></dt> <dt><samp class=”option”>scroll</samp></dt> <dt><samp class=”option”>frame</samp></dt> </dl>

</dd> <dt><samp class=”option”>direction</samp></dt> <dd> Set the direction method for output slide method. Allowed values are: <dl class=”table”> <dt><samp class=”option”>lr</samp></dt> <dd> Direction from left to right. </dd> <dt><samp class=”option”>rl</samp></dt>

<dd> Direction from right to left. </dd> <dt><samp class="option">ud</samp></dt> <dd> Direction from up to down. </dd> <dt><samp class="option">du</samp></dt> <dd> Direction from down to up. </dd> </dl>

</dd> <dt><samp class="option">bar</samp></dt> <dd> Set the ratio of bargraph display to display size. Default is 0. </dd> <dt><samp class="option">rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">20.24 showfreqs# TOC</h3>

Convert input audio to video output representing the audio power spectrum.

Audio amplitude is on Y-axis while frequency is on X-axis.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify size of video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default is <code class="code">1024x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set video rate. Default is <code class="code">25</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Set display mode.

This set how each frequency bin will be represented.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">bar</samp>’</dt>
<dt>‘<samp class="samp">dot</samp>’</dt>
</dl>
Default is <code class="code">bar</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root scale.

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">fscale</samp></dt>
<dd> Set frequency scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.

</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> Reverse logarithmic scale.
</dd>
</dl>
Default is <code class="code">lin</code>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from 16 to 65536.

Default is <code class="code">2048</code>

</dd>
<dt><samp class="option">win_func</samp></dt>
<dd> Set windowing function.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt>

<dt>‘<samp class="samp">hanning</samp>’</dt>

<dt>‘<samp class="samp">hamming</samp>’</dt>

<dt>‘<samp class="samp">blackman</samp>’</dt>

<dt>‘<samp class="samp">welch</samp>’</dt>

<dt>‘<samp class="samp">flattop</samp>’</dt>

<dt>‘<samp class="samp">bharris</samp>’</dt>

<dt>‘<samp class="samp">bnuttall</samp>’</dt>

<dt>‘<samp class="samp">bhann</samp>’</dt>

<dt>‘<samp class="samp">sine</samp>’</dt>

<dt>‘<samp class="samp">nuttall</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hanning</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. In range <code class="code">[0, 1]</code>.
Default is <code class="code">1</code>,

which means optimal overlap for selected window function will be picked.

</dd>

<dt><samp class="option">averaging</samp></dt>

<dd> Set time averaging. Setting this to 0 will display current maximal peaks.

Default is <code class="code">1</code>, which means time averaging is disabled.

</dd>

<dt><samp class="option">colors</samp></dt>

<dd> Specify list of colors separated by space or by ’|’ which will be used to

draw channel frequencies. Unrecognized or missing colors will be replaced

by white color.

</dd>

<dt><samp class="option">cmode</samp></dt>

<dd> Set channel display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dt>‘<samp class="samp">separate</samp>’</dt>

</dl>

Default is <code class="code">combined</code>.

</dd>

<dt><samp class="option">minamp</samp></dt>

<dd> Set minimum amplitude used in <code class="code">log</code> amplitude scaler.

</dd>

<dt><samp class=”option”>data</samp></dt>

<dd> Set data display mode.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>magnitude</samp>’</dt>

<dt>‘<samp class=”samp”>phase</samp>’</dt>

<dt>‘<samp class=”samp”>delay</samp>’</dt>

</dl>

Default is <code class=”code”>magnitude</code>.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set channels to use when processing audio. By default all are processed.

</dd>

</dl>

 <h3 class=”section”>20.25 showspatial# TOC</h3>

Convert stereo input audio to a video output, representing the spatial relationship between two channels. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>512x512</code>. </dd> <dt><samp class=”option”>win_size</samp></dt> <dd> Set window size. Allowed range is from <var class=”var”>1024</var> to <var class=”var”>65536</var>. Default size is <var class=”var”>4096</var>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann</samp>’</dt> <dt>‘<samp class=”samp”>hanning</samp>’<
<dt>‘<samp class=”samp”>hamming</samp>’</dt>

<dt>‘<samp class=”samp”>blackman</samp>’</dt>
<dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp
class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’<
<dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp
class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt
<dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp
class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</
<dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp
class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</
<dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp
class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>&rsquo
<dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl>

Default value is <code class=”code”>hann</code>. </dd>
<dt><samp class=”option”>rate, r</samp></dt> <dd> Set output
framerate. </dd> </dl>

<a name=”showspectrum-
1”> <h3 class=”section”>20.26 showspectrum<span class=”pull-
right”>#
TOC</h3>

Convert input audio to a video output, representing the audio frequency spectrum. The filter accepts the following options:
<dl class=”table”> <dt><samp class=”option”>size, s</samp></dt>
<dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>640x512</code>.
</dd> <dt><samp class=”option”>slide</samp></dt> <dd> Specify how the spectrum should slide along the window. It accepts the following values: <dl class=”table”> <dt>‘<samp
class=”samp”>replace</samp>’</dt> <dd> the samples start again on the left when they reach the right </dd> <dt>‘<samp
class=”samp”>scroll</samp>’</dt> <dd> the samples scroll from right to left </dd> <dt>‘<samp class=”samp”>fullframe</samp>’</dt>
<dd> frames are only produced when the samples reach the right
</dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> the samples scroll from left to right </dd> <dt>‘<samp
class=”samp”>lreplace</samp>’</dt> <dd> the samples start again on the right when they reach the left </dd> </dl>

Default value is <code class=”code”>replace</code>. </dd>
<dt><samp class=”option”>mode</samp></dt> <dd> Specify display mode. It accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>combined</samp>’</dt>
<dd> all channels are displayed in the same row </dd> <dt>‘<samp
class=”samp”>separate</samp>’</dt> <dd> all channels are

displayed in separate rows </dd> </dl>

Default value is ‘<samp class="samp">combined</samp>’.
 </dd> <dt><samp class="option">color</samp></dt> <dd> Specify
display color mode. It accepts the following values: <dl
class="table"> <dt>‘<samp class="samp">channel</samp>’</dt>
<dd> each channel is displayed in a separate color </dd>
<dt>‘<samp class="samp">intensity</samp>’</dt> <dd> each
channel is displayed using the same color scheme </dd> <dt>‘<samp
class="samp">rainbow</samp>’</dt> <dd> each channel is dis-
played using the rainbow color scheme </dd> <dt>‘<samp
class="samp">moreland</samp>’</dt> <dd> each channel is
displayed using the moreland color scheme </dd> <dt>‘<samp
class="samp">nebulae</samp>’</dt> <dd> each channel is
displayed using the nebulae color scheme </dd> <dt>‘<samp
class="samp">fire</samp>’</dt> <dd> each channel is dis-
played using the fire color scheme </dd> <dt>‘<samp
class="samp">fiery</samp>’</dt> <dd> each channel is
displayed using the fiery color scheme </dd> <dt>‘<samp
class="samp">fruit</samp>’</dt> <dd> each channel is
displayed using the fruit color scheme </dd> <dt>‘<samp
class="samp">cool</samp>’</dt> <dd> each channel is dis-
played using the cool color scheme </dd> <dt>‘<samp
class="samp">magma</samp>’</dt> <dd> each channel is
displayed using the magma color scheme </dd> <dt>‘<samp
class="samp">green</samp>’</dt> <dd> each channel is
displayed using the green color scheme </dd> <dt>‘<samp
class="samp">viridis</samp>’</dt> <dd> each channel is
displayed using the viridis color scheme </dd> <dt>‘<samp
class="samp">plasma</samp>’</dt> <dd> each channel is
displayed using the plasma color scheme </dd> <dt>‘<samp
class="samp">cividis</samp>’</dt> <dd> each channel is
displayed using the cividis color scheme </dd> <dt>‘<samp
class="samp">terrain</samp>’</dt> <dd> each channel is
displayed using the terrain color scheme </dd> </dl>

Default value is ‘<samp class="samp">channel</samp>’.
 </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
scale used for calculating intensity color values. It accepts
the following values: <dl class="table"> <dt>‘<samp
class="samp">lin</samp>’</dt> <dd> linear </dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> square
root, default </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic </dd> <dt>‘<samp class="samp">4thrt</samp>’</dt>
<dd> 4th root </dd> <dt>‘<samp class="samp">5thrt</samp>’</dt>
<dd> 5th root </dd> </dl>

Default value is ‘<samp class="samp">sqrt</samp>’.
 </dd> <dt><samp class="option">fscale</samp></dt> <dd> Specify
frequency scale. It accepts the following values: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic </dd> </dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values
provide alternative color scheme. <code class="code">0</code> is no
saturation at all. Saturation must be in [-10.0, 10.0] range. Default
value is <code class="code">1</code>. </dd> <dt><samp
class="option">win_func</samp></dt> <dd> Set window func-
tion. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp
class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp
class="samp">hamming</samp>’</dt> <dt>‘<samp
class="samp">blackman</samp>’</dt> <dt>‘<samp
class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp
class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp
class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp
class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp
class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp
class="samp">kaiser</samp>’</dt> </dl>

Default value is <code class="code">hann</code>. </dd>
<dt><samp class="option">orientation</samp></dt> <dd> Set orien-
tation of time vs frequency axis. Can be <code class="code">vertical</code>
or <code class="code">horizontal</code>. Default is <code class="code">vertical</code>.
 </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set
ratio of overlap window. Default value is <code class="code">0</code>.
When value is <code class="code">1</code> overlap is set to recommended
size for specific window function currently used. </dd> <dt><samp
class="option">gain</samp></dt> <dd> Set scale gain for calculating
intensity color values. Default value is <code class="code">1</code>.
</dd> <dt><samp class="option">data</samp></dt> <dd> Set
which data to display. Can be <code class="code">magnitude</code>,
default or <code class="code">phase</code>, or unwrapped phase: <code
class="code">uphase</code>. </dd> <dt><samp class="option">rotation</samp></dt>
<dd> Set color rotation, must be in [-1.0, 1.0] range. Default

value is <code class="code">0</code>. </dd> <dt><samp class="option">start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class="code">0</code>. </dd> <dt><samp class="option">stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class="code">0</code>. </dd> <dt><samp class="option">fps</samp></dt> <dd> Set upper frame rate limit. Default is <code class="code">auto</code>, unlimited. </dd> <dt><samp class="option">legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is disabled. </dd> <dt><samp class="option">drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class="option">limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class="option">opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

The usage is very similar to the showwaves filter; see the examples in that section. <h4 class="subsection">20.26.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Large window with logarithmic color scaling: <pre class="example-preformatted">showspectrum=s=1280x480:scale=log </pre>

Complete example for a colored and sliding spectrum per channel using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] showspectrum=mode=separate:color=intensity:slide=1:scale=cbrt [out0]' </pre>

 <h3 class="section">20.27 showspectrumpic# TOC</h3>

Convert input audio to a single video frame, representing the audio frequency

spectrum.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">4096x2048</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Specify display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dd> all channels are displayed in the same row

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> all channels are displayed in separate rows

</dd>

</dl>

Default value is ‘<samp class="samp">combined</samp>’.

</dd>

<dt><samp class="option">color</samp></dt>

<dd> Specify display color mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">channel</samp>’</dt>

<dd> each channel is displayed in a separate color

</dd>

<dt>‘<samp class="samp">intensity</samp>’</dt>
<dd> each channel is displayed using the same color scheme
</dd>
<dt>‘<samp class="samp">rainbow</samp>’</dt>
<dd> each channel is displayed using the rainbow color scheme
</dd>
<dt>‘<samp class="samp">moreland</samp>’</dt>
<dd> each channel is displayed using the moreland color scheme
</dd>
<dt>‘<samp class="samp">nebulae</samp>’</dt>
<dd> each channel is displayed using the nebulae color scheme
</dd>
<dt>‘<samp class="samp">fire</samp>’</dt>
<dd> each channel is displayed using the fire color scheme
</dd>
<dt>‘<samp class="samp">fiery</samp>’</dt>
<dd> each channel is displayed using the fiery color scheme
</dd>
<dt>‘<samp class="samp">fruit</samp>’</dt>
<dd> each channel is displayed using the fruit color scheme
</dd>
<dt>‘<samp class="samp">cool</samp>’</dt>
<dd> each channel is displayed using the cool color scheme
</dd>
<dt>‘<samp class="samp">magma</samp>’</dt>
<dd> each channel is displayed using the magma color scheme
</dd>
<dt>‘<samp class="samp">green</samp>’</dt>
<dd> each channel is displayed using the green color scheme
</dd>
<dt>‘<samp class="samp">viridis</samp>’</dt>

<dd> each channel is displayed using the viridis color scheme
</dd>
<dt>‘<samp class="samp">plasma</samp>’</dt>
<dd> each channel is displayed using the plasma color scheme
</dd>
<dt>‘<samp class="samp">cividis</samp>’</dt>
<dd> each channel is displayed using the cividis color scheme
</dd>
<dt>‘<samp class="samp">terrain</samp>’</dt>
<dd> each channel is displayed using the terrain color scheme
</dd>
</dl>
Default value is ‘<samp class="samp">intensity</samp>’.

</dd>
<dt><samp class="option">scale</samp></dt>
<dd> Specify scale used for calculating intensity color values.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> square root, default
</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic

</dd>

<dt>‘<samp class="samp">4thrt</samp>’</dt>

<dd> 4th root

</dd>

<dt>‘<samp class="samp">5thrt</samp>’</dt>

<dd> 5th root

</dd>

</dl>

Default value is ‘<samp class="samp">log</samp>’.

</dd>

<dt><samp class="option">fscale</samp></dt>

<dd> Specify frequency scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">lin</samp>’</dt>

<dd> linear

</dd>

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

</dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values provide alternative color scheme. <code class="code">0</code> is no saturation at all. Saturation must be in [-10.0, 10.0] range. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp

class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt> <dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</dt> <dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>’</dt> <dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl> Default value is <code class=”code”>hann</code>. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of time vs frequency axis. Can be <code class=”code”>vertical</code> or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set scale gain for calculating intensity color values. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is enabled. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

 <h4 class=”subsection”>20.27.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Extract an audio spectrogram of a whole audio track in a 1024x1024 picture using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i audio.flac -lavfi showspectrumpic=s=1024x1024 spectrogram.png </pre>

 <h3 class="section">20.28 showvolume# TOC</h3>

Convert input audio volume to a video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set video rate. </dd> <dt><samp class="option">b</samp></dt> <dd> Set border width, allowed range is [0, 5]. Default is 1. </dd> <dt><samp class="option">w</samp></dt> <dd> Set channel width, allowed range is [80, 8192]. Default is 400. </dd> <dt><samp class="option">h</samp></dt> <dd> Set channel height, allowed range is [1, 900]. Default is 20. </dd> <dt><samp class="option">f</samp></dt> <dd> Set fade, allowed range is [0, 1]. Default is 0.95. </dd> <dt><samp class="option">c</samp></dt> <dd> Set volume color expression. The expression can use the following variables: <dl class="table"> <dt><samp class="option">VOLUME</samp></dt> <dd> Current max volume of channel in dB. </dd> <dt><samp class="option">PEAK</samp></dt> <dd> Current peak. </dd> <dt><samp class="option">CHANNEL</samp></dt> <dd> Current channel number, starting from 0. </dd> </dl>

</dd> <dt><samp class="option">t</samp></dt> <dd> If set, displays channel names. Default is enabled. </dd> <dt><samp class="option">v</samp></dt> <dd> If set, displays volume values. Default is enabled. </dd> <dt><samp class="option">o</samp></dt> <dd> Set orientation, can be horizontal: <code class="code">h</code> or vertical: <code class="code">v</code>, default is <code class="code">h</code>. </dd> <dt><samp class="option">s</samp></dt> <dd> Set step size, allowed range is [0, 5]. Default is 0, which means step is disabled. </dd> <dt><samp class="option">p</samp></dt> <dd> Set background opacity, allowed range is [0, 1]. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set metering mode, can be peak: <code class="code">p</code> or rms: <code class="code">r</code>, default is <code class="code">p</code>. </dd> <dt><samp class="option">ds</samp></dt> <dd> Set display scale, can be linear: <code class="code">lin</code> or log: <code class="code">log</code>, default is <code class="code">lin</code>. </dd> <dt><samp class="option">dm</samp></dt> <dd> In second. If set to > 0., display a line for the max level in the previous seconds. default is disabled: <code class="code">0.</code> </dd> <dt><samp class="option">dmc</samp></dt> <dd> The color of the max line. Use when <code class="code">dm</code> option is set to > 0. default is: <code class="code">orange</code> </dd> </dl>

 <h3 class="section">20.29 showwaves<span

class=”pull-right”># TOC</h3>

Convert input audio to a video output, representing the samples waves. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>600x240</code>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set display mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Draw a point for each sample. </dd> <dt>‘<samp class=”samp”>line</samp>’</dt> <dd> Draw a vertical line for each sample. </dd> <dt>‘<samp class=”samp”>p2p</samp>’</dt> <dd> Draw a point for each sample and a line between them. </dd> <dt>‘<samp class=”samp”>cline</samp>’</dt> <dd> Draw a centered vertical line for each sample. </dd> </dl>

Default value is <code class=”code”>point</code>. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of samples which are printed on the same column. A larger value will decrease the frame rate. Must be a positive integer. This option can be set only if the value for <var class=”var”>rate</var> is not explicitly specified. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the (approximate) output frame rate. This is done by setting the option <var class=”var”>n</var>. Default value is "25". </dd> <dt><samp class=”option”>split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class=”option”>draw</samp></dt> <dd> Set the draw mode. This is mostly useful to set for high <var class=”var”>n</var>. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> </dl>

 <h4 class="subsection">20.29.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output the input file audio and the corresponding video representation at the same time: <pre class="example-preformatted">amovie=a.mp3,asplit[out0],showwaves[out1]
</pre>

Create a synthetic signal and show it with showwaves, forcing a frame rate of 30 frames per second: <pre class="example-preformatted">aevalsrc=sin(1*2*PI*t)*sin(880*2*PI*t):cos(2*PI*200*t),asplit[out0],showwaves=r=30[out1]
</pre>

 <h3 class="section">20.30 showwavespic# TOC</h3>

Convert input audio to a single video frame, representing the samples waves. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">600x240</code>. </dd> <dt><samp class="option">split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class="option">colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class="table"> <dt>‘<samp class="samp">lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class="option">draw</samp></dt> <dd> Set the draw mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> <dt><samp class="option">filter</samp></dt> <dd> Set the filter mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">average</samp>’</dt> <dd> Use average samples values for each drawn sample. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Use peak samples values for each drawn sample. </dd> </dl>

Default value is <code class="code">average</code>. </dd> </dl>

 <h4 class="subsection">20.30.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract a channel split representation of the wave form of a whole audio track in a 1024x800 picture using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i audio.flac -lavfi showwavespic=split_channels=1:s=1024x800 waveform.png </pre>

 <h3 class="section">20.31 sidedata, asidedata# TOC</h3

Delete frame side data, or select frames based on it. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">select</samp>’</dt> <dd> Select every frame with side data of <code class="code">type</code>. </dd> <dt>‘<samp class="samp">delete</samp>’</dt> <dd> Delete side data of <code class="code">type</code>. If <code class="code">type</code> is not set, delete all side data in the frame. </dd> </dl>

</dd> <dt><samp class="option">type</samp></dt> <dd> Set side data type used with all modes. Must be set for <code class="code">select</code> mode. Possible values are: <dl class="table"> <dt>‘<samp class="samp">PANSCAN</samp>’</dt> <dt>‘<samp class="samp">A53_CC</samp>’</dt> <dt>‘<samp class="samp">STEREO3D</samp>’</dt> <dt>‘<samp class="samp">MATRIXENCODING</samp>’</dt> <dt>‘<samp class="samp">DOWNMIX_INFO</samp>’</dt> <dt>‘<samp

class=”samp”>REPLAYGAIN</samp>’</dt> <dt>‘<samp
class=”samp”>DISPLAYMATRIX</samp>’</dt> <dt>‘<samp
class=”samp”>AFD</samp>’</dt> <dt>‘<samp class=”samp”>MOTION_VECTORS</sa
<dt>‘<samp class=”samp”>SKIP_SAMPLES</samp>’</dt>
<dt>‘<samp class=”samp”>AUDIO_SERVICE_TYPE</samp>’</dt>
<dt>‘<samp class=”samp”>MASTERING_DISPLAY_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>GOP_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>SPHERICAL</samp>’</dt>
<dt>‘<samp class=”samp”>CONTENT_LIGHT_LEVEL</samp>’</dt>
<dt>‘<samp class=”samp”>ICC_PROFILE</samp>’</dt>
<dt>‘<samp class=”samp”>S12M_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_PLUS</samp>’</dt>
<dt>‘<samp class=”samp”>REGIONS_OF_INTEREST</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_ENC_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>SEI_UNREGISTERED</samp>’</dt>
<dt>‘<samp class=”samp”>FILM_GRAIN_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BOUNDING_BOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BBOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_RPU_BUFFER</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_VIVID</samp>’</dt>
<dt>‘<samp class=”samp”>AMBIENT_VIEWING_ENVIRONMENT</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_HINT</samp>’</dt>
<dt>‘<samp class=”samp”>GAIN_MAP_PARAMS</samp>’</dt>
</dl>

</dd> </dl>

 <h3 class=”section”>20.32 spectrumsynth# TOC</h3>

Synthesize audio from 2 input video spectrums, first input stream represents magnitude across time and second represents phase across time. The filter will transform from frequency domain as displayed in videos back to time domain as presented in audio output.
This filter is primarily created for reversing processed showspectrum filter outputs, but can synthesize sound from other spectrograms too. But in such case results are going to be poor if the phase data is not available, because in such cases phase data need to be recreated, usually it’s just recreated from random noise. For best results use gray only output (<code class=”code”>channel</code> color mode in showspectrum filter) and <code class=”code”>log</code> scale for magnitude video and <code class=”code”>lin</code> scale for phase video. To produce phase, for 2nd video, use <code class=”code”>data</code> option. Inputs

videos should generally use <code class=”code”>fullframe</code> slide mode as that saves resources needed for decoding video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Specify sample rate of output audio, the sample rate of audio from which spectrum was generated may differ. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set number of channels represented in input video spectrums. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set scale which was used when generating magnitude input spectrum. Can be <code class=”code”>lin</code> or <code class=”code”>log</code>. Default is <code class=”code”>log</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide which was used when generating inputs spectrums. Can be <code class=”code”>replace</code>, <code class=”code”>scroll</code>, <code class=”code”>fullframe</code> or <code class=”code”>rscroll</code>. Default is <code class=”code”>fullframe</code>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function used for resynthesis. </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set window overlap. In range <code class=”code”>[0, 1]</code>. Default is <code class=”code”>1</code>, which means optimal overlap for selected window function will be picked. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of input videos. Can be <code class=”code”>vertical</code> or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>. </dd> </dl>

 <h4 class=”subsection”>20.32.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> First create magnitude and phase videos from audio, assuming audio is stereo with 44100 sample rate, then resynthesize videos back to audio with spectrumsynth: <pre class=”example-preformatted”>ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=log:overlap=0.875:color=channel:slide=fullframe:data=magnitude -an -c:v rawvideo magnitude.nut ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=lin:overlap=0.875:color=channel:slide=fullframe:data=phase -an -c:v rawvideo phase.nut ffmpeg -i magnitude.nut -i phase.nut -lavfi spectrumsynth=channels=2:sample_rate=44100:win_func=hann:overlap=0.875:slide=fullframe output.flac </pre>

 <h3 class=”section”>20.33 split, asplit# TOC</h3>

Split input into several identical outputs. <code class=”code”>asplit</code>

works with audio input, <code class="code">split</code> with video.
The filter accepts a single parameter which specifies the number of outputs. If unspecified, it defaults to 2.
<h4 class="subsection">20.33.1 Examples#
TOC</h4>

<ul class="itemize mark-bullet"> Create two separate outputs from the same input: <pre class="example-preformatted">[in] split [out0][out1] </pre>

To create 3 or more outputs, you need to specify the number of outputs, like in: <pre class="example-preformatted">[in] asplit=3 [out0][out1][out2] </pre>

Create two separate outputs from the same input, one cropped and one padded: <pre class="example-preformatted">[in] split [splitout1][splitout2]; [splitout1] crop=100:100:0:0 [cropout]; [splitout2] pad=200:200:100:100 [padout]; </pre>

Create 5 copies of the input audio with <code class="command">ffmpeg</code>:
 <pre class="example-preformatted">ffmpeg -i INPUT -filter_complex asplit=5 OUTPUT </pre>

 <h3 class="section">20.34 zmq, azmq# TOC</h3>

Receive commands sent through a libzmq client, and forward them to filters in the filtergraph. <code class="code">zmq</code> and <code class="code">azmq</code> work as a pass-through filters. <code class="code">zmq</code> must be inserted between two video filters, <code class="code">azmq</code> between two audio filters. Both are capable to send messages to any filter type. To enable these filters you need to install the libzmq library and headers and configure FFmpeg with <code class="code">--enable-libzmq</code>. For more information about libzmq see: http://www.zeromq.org/
The <code class="code">zmq</code> and <code class="code">azmq</code> filters work as a libzmq server, which receives messages sent through a network interface defined by the <samp class="option">bind_address</samp> (or the abbreviation "<samp class="option">b</samp>") option. Default value of this option is <samp class="file">tcp://localhost:5555</samp>. You may want to alter this value to your needs, but do not forget to escape any ’:’ signs (see filtergraph escaping). The received message must be in the form: <pre class="example-preformatted"><var class="var">TARGET</var> <var class="var">COMMAND</var>

[<var class="var">ARG</var>] </pre>

<var class="var">TARGET</var> specifies the target of the command, usually the name of the filter class or a specific filter instance name. The default filter instance name uses the pattern ‘<samp class="samp">Parsed_<filter_name>_<index></samp>’, but you can override this by using the ‘<samp class="samp">filter_name@id</samp>’ syntax (see Filtergraph syntax). <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional argument list for the given <var class="var">COMMAND</var>. Upon reception, the message is processed and the corresponding command is injected into the filtergraph. Depending on the result, the filter will send a reply to the client, adopting the format: <pre class="example-preformatted"><var class="var">ERROR_CODE</var> <var class="var">ERROR_REASON</var> <var class="var">MESSAGE</var> </pre>

<var class="var">MESSAGE</var> is optional. <h4 class="subsection">20.34.1 Examples# TOC</h4>

Look at <samp class="file">tools/zmqsend</samp> for an example of a zmq client which can be used to send commands processed by these filters. Consider the following filtergraph generated by <code class="command">ffplay</code>. In this example the last overlay filter has an instance name. All other filters will have default instance names.   <pre class="example-preformatted">ffplay -dumpgraph 1 -f lavfi " color=s=100x100:c=red [l]; color=s=100x100:c=blue [r]; nullsrc=s=200x100, zmq [bg]; [bg][l] overlay [bg+l]; [bg+l][r] overlay@my=x=100 " </pre>

To change the color of the left side of the video, the following command can be used: <pre class="example-preformatted">echo Parsed_color_0 c yellow | tools/zmqsend </pre>

To change the right side: <pre class="example-preformatted">echo Parsed_color_1 c pink | tools/zmqsend </pre>

To change the position of the right side: <pre class="example-preformatted">echo overlay@my x 150 | tools/zmqsend </pre>

 <h2 class="chapter">21 Multimedia Sources# TOC</h2>

Below is a description of the currently available multimedia sources. <h3 class="section">21.1 amovie# TOC</h3>

This is the same as movie source, except it selects an audio stream by default. <h3 class="section">21.2 avsynctest# TOC</h3> Generate an Audio/Video Sync Test. Generated stream periodically shows flash video frame and emits beep in audio. Useful to inspect A/V sync issues. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set output video size. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">framerate, fr</samp></dt> <dd> Set output video frame rate. Default value is <code class="code">30</code>. </dd> <dt><samp class="option">samplerate, sr</samp></dt> <dd> Set output audio sample rate. Default value is <code class="code">44100</code>. </dd> <dt><samp class="option">amplitude, a</samp></dt> <dd> Set output audio beep amplitude. Default value is <code class="code">0.7</code>. </dd> <dt><samp class="option">period, p</samp></dt> <dd> Set output audio beep period in seconds. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">delay, dl</samp></dt> <dd> Set output video flash delay in number of frames. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">cycle, c</samp></dt> <dd> Enable cycling of video delays, by default is disabled. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set stream output duration. By default duration is unlimited. </dd> <dt><samp class="option">fg, bg, ag</samp></dt> <dd> Set foreground/background/additional color. </dd> </dl>

 <h4 class="subsection">21.2.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">21.3 movie# TOC</h3>

Read audio and/or video stream(s) from a movie container. It accepts the following parameters: <dl class="table"> <dt><samp

class=”option”>filename</samp></dt> <dd> The name of the resource to read (not necessarily a file; it can also be a device or a stream accessed through some protocol). </dd> <dt><samp class=”option”>format_name, f</samp></dt> <dd> Specifies the format assumed for the movie to read, and can be either the name of a container or an input device. If not specified, the format is guessed from <var class=”var”>movie_name</var> or by probing. </dd> <dt><samp class=”option”>seek_point, sp</samp></dt> <dd> Specifies the seek point in seconds. The frames will be output starting from this seek point. The parameter is evaluated with <code class=”code”>av_strtod</code>, so the numerical value may be suffixed by an IS postfix. The default value is "0". </dd> <dt><samp class=”option”>streams, s</samp></dt> <dd> Specifies the streams to read. Several streams can be specified, separated by "+". The source will then have as many outputs, in the same order. The syntax is explained in the <a data-manual=”ffmpeg” href=”./ffmpeg.html#Stream-specifiers”>"Stream specifiers" section in the ffmpeg manual. Two special names, "dv" and "da" specify respectively the default (best suited) video and audio stream. Default is "dv", or "da" if the filter is called as "amovie". </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Specifies the index of the video stream to read. If the value is -1, the most suitable video stream will be automatically selected. The default value is "-1". Deprecated. If the filter is called "amovie", it will select audio instead of video. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> Specifies how many times to read the stream in sequence. If the value is 0, the stream will be looped infinitely. Default value is "1". Note that when the movie is looped the source timestamps are not changed, so it will generate non monotonically increasing timestamps. </dd> <dt><samp class=”option”>discontinuity</samp></dt> <dd> Specifies the time difference between frames above which the point is considered a timestamp discontinuity which is removed by adjusting the later timestamps. </dd> <dt><samp class=”option”>dec_threads</samp></dt> <dd> Specifies the number of threads for decoding </dd> <dt><samp class=”option”>format_opts</samp></dt> <dd> Specify format options for the opened file. Format options can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to add protocol_whitelist and protocol_blacklist options: <pre class=”example-preformatted”>ffplay -f lavfi "movie=filename='1.sdp':format_opts='protocol_whitelist=file,rtp,udp\:protocol_blacklist=http'&quo
</pre> </dd> </dl>

It allows overlaying a second video on top of the main input of a filtergraph, as shown in this graph: <pre class=”example-preformatted”>input ----------> deltapts0 --> overlay

--> output ^ | movie --> scale--> deltapts1 -------+ </pre> <h4 class="subsection">21.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Skip 3.2 seconds from the start of the AVI file in.avi, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=in.avi:seek_point=3.2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read from a video4linux2 device, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=/dev/video0:f=video4linux2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read the first video stream and the audio stream with id 0x81 from dvd.vob; the video is connected to the pad named "video" and the audio is connected to the pad named "audio": <pre class="example-preformatted">movie=dvd.vob:s=v:0+#0x81 [video] [audio] </pre>

 <h4 class="subsection">21.3.2 Commands# TOC</h4>

Both movie and amovie support the following commands: <dl class="table"> <dt><samp class="option">seek</samp></dt> <dd> Perform seek using "av_seek_frame". The syntax is: seek <var class="var">stream_index</var>|<var class="var">timestamp</var>|<var class="var">flags</var> <ul class="itemize mark-bullet"> <var class="var">stream_index</var>: If stream_index is -1, a default stream is selected, and <var class="var">timestamp</var> is automatically converted from AV_TIME_BASE units to the stream specific time_base. <var class="var">timestamp</var>: Timestamp in AVStream.time_base units or, if no stream is specified, in AV_TIME_BASE units. <var class="var">flags</var>: Flags which select direction and seeking mode.

</dd> <dt><samp class="option">get_duration</samp></dt> <dd> Get movie duration in AV_TIME_BASE units. </dd> </dl>

 <h2 class="chapter">22 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavfilter

 <h2 class="chapter">23 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-formats.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Formats Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Formats Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark">

1 Description

2 Format Options

<ul class="toc-numbered-mark">

2.1 Format stream specifiers

3 Demuxers

<ul class="toc-numbered-mark">
3.1 aa
3.2 aac
3.3 apng
3.4 asf
3.5 astc
3.6 concat
<ul class="toc-numbered-mark">
3.6.1 Syntax
3.6.2 Options
3.6.3 Examples

3.7 dash
<ul class="toc-numbered-mark">
3.7.1 Options

3.8 dvdvideo
<ul class="toc-numbered-mark">
3.8.1 Background
3.8.2 Options
3.8.3 Examples

3.9 ea
<ul class="toc-numbered-mark">
3.9.1 Options

3.10 imf
3.11 flv, live_flv, kux
3.12 gif
3.13 hls

3.14 image2

<ul class="toc-numbered-mark">

3.14.1 Examples

3.15 jpeg_mpf

3.16 ktx

3.17 libgme

3.18 libmodplug

3.19 libopenmpt

3.20 mcc

<ul class="toc-numbered-mark">

3.20.1 Examples

3.21 mov/mp4/3gp

<ul class="toc-numbered-mark">

3.21.1 Options

3.21.2 Audible AAX

3.22 mpegts

3.23 mpjpeg

3.24 rawvideo

3.25 rcwt

<ul class="toc-numbered-mark">

3.25.1 Examples

3.26 sbg

3.27 tedcaptions

3.28 vapoursynth

3.29 w64

3.30 wav

3.31 webp

4 Muxers

<ul class="toc-numbered-mark">

4.1 Raw muxers

<ul class="toc-numbered-mark">

4.1.1 Examples

4.2 Raw PCM muxers

4.3 MPEG-1/MPEG-2 program stream muxers

<ul class="toc-numbered-mark">

4.3.1 Options

<a id="toc-MOV_002fMPEG_002d4_002fISOMBFF-muxers" href="#MOV_002fMPEG_002d4_002fI
muxers">4.4 MOV/MPEG-4/ISOMBFF muxers

<ul class="toc-numbered-mark">

4.4.1 Stereoscopic 3D

4.4.2 Fragmentation

4.4.3 Options

4.4.4 Examples

4.5 a64

4.6 ac4

<ul class="toc-numbered-mark">

4.6.1 Options

4.7 adts

<ul class="toc-numbered-mark">
4.7.1 Options

4.8 aea
4.9 aiff
<ul class="toc-numbered-mark">
4.9.1 Options

4.10 alp
<ul class="toc-numbered-mark">
4.10.1 Options

4.11 amr
4.12 amv
4.13 apm
4.14 apng
<ul class="toc-numbered-mark">
4.14.1 Options
4.14.2 Examples

4.15 argo_asf
<ul class="toc-numbered-mark">
4.15.1 Options

4.16 argo_cvg
<ul class="toc-numbered-mark">
4.16.1 Options

4.17 asf, asf_stream
<ul class="toc-numbered-mark">

4.17.1 Options

4.18 ass
<ul class="toc-numbered-mark">
4.18.1 Options

4.19 ast
<ul class="toc-numbered-mark">
4.19.1 Options

4.20 astc
4.21 au
4.22 avi
<ul class="toc-numbered-mark">
4.22.1 Options

4.23 avif
<ul class="toc-numbered-mark">
4.23.1 Options

4.24 avm2
4.25 bit
4.26 caf
4.27 codec2
4.28 chromaprint
<ul class="toc-numbered-mark">
4.28.1 Options

4.29 crc
<ul class="toc-numbered-mark">
4.29.1 Examples

4.30 dash
<ul class="toc-numbered-mark">
4.30.1 Options
4.30.2 Example

4.31 daud
<ul class="toc-numbered-mark">
4.31.1 Example

4.32 dv
<ul class="toc-numbered-mark">
4.32.1 Example

4.33 ffmetadata
<ul class="toc-numbered-mark">
4.33.1 Example

4.34 fifo
<ul class="toc-numbered-mark">
4.34.1 Options
4.34.2 Example

4.35 film_cpk
4.36 filmstrip
4.37 fits
4.38 flac
<ul class="toc-numbered-mark">
4.38.1 Options
4.38.2 Example

4.39 flv
<ul class="toc-numbered-mark">
4.39.1 Options

4.40 framecrc
<ul class="toc-numbered-mark">
4.40.1 Examples

4.41 framehash
<ul class="toc-numbered-mark">
4.41.1 Examples

4.42 framemd5
<ul class="toc-numbered-mark">
4.42.1 Examples

4.43 gif
<ul class="toc-numbered-mark">
4.43.1 Options
4.43.2 Example

4.44 gxf
4.45 hash
<ul class="toc-numbered-mark">
4.45.1 Examples

4.46 hds
<ul class="toc-numbered-mark">
4.46.1 Options
4.46.2 Example

4.47 hls

<ul class="toc-numbered-mark">

4.47.1 Options

4.48 iamf

4.49 ico

4.50 ilbc

4.51 image2, image2pipe

<ul class="toc-numbered-mark">

4.51.1 Options

4.51.2 Examples

4.52 ircam

4.53 iterm2

<ul class="toc-numbered-mark">

4.53.1 Options

4.53.2 Examples

4.54 jpeg_mpf

<ul class="toc-numbered-mark">

4.54.1 Examples

4.55 ivf

4.56 jacosub

4.57 ktx

<ul class="toc-numbered-mark">

4.57.1 Options

4.58 kvag

4.59 lc3

4.60 lrc
<ul class="toc-numbered-mark">
4.60.1 Options
4.60.2 Metadata

4.61 matroska
<ul class="toc-numbered-mark">
4.61.1 Metadata
4.61.2 Options

4.62 md5
<ul class="toc-numbered-mark">
4.62.1 Examples

4.63 mcc
<ul class="toc-numbered-mark">
4.63.1 Options
4.63.2 Examples

4.64 microdvd
4.65 mmf
4.66 mp3
4.67 mpegts
<ul class="toc-numbered-mark">
4.67.1 Options
4.67.2 Example

4.68 mxf, mxf_d10, mxf_opatom
<ul class="toc-numbered-mark">
4.68.1 Options

4.69 null

4.70 nut

4.71 ogg

4.72 pdv

4.73 rcwt

<ul class="toc-numbered-mark">

4.73.1 Examples

4.74 segment, stream_segment, ssegment

<ul class="toc-numbered-mark">

4.74.1 Options

4.74.2 Examples

4.75 smoothstreaming

4.76 streamhash

<ul class="toc-numbered-mark">

4.76.1 Examples

4.77 tee

<ul class="toc-numbered-mark">

4.77.1 Options

4.77.2 Examples

4.78 wav

<ul class="toc-numbered-mark">

4.78.1 Options

4.79 webm_chunk

<ul class="toc-numbered-mark">

4.79.1 Options

4.79.2 Example

4.80 webm_dash_manifest

<ul class="toc-numbered-mark">

4.80.1 Options

4.80.2 Example

4.81 whip

<ul class="toc-numbered-mark">

4.81.1 Options

5 Metadata

6 See Also

7 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes the supported formats (muxers and demuxers) provided by the libavformat library.

 <h2 class="chapter">2 Format Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the muxers and demuxers. In addition each muxer or demuxer may support so-called private options, which are specific for that component.

Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class="table"> <dt><samp class="option">avioflags <var class="var">flags</var> (input/output)</samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">direct</samp>’</dt> <dd> Reduce buffering. </dd> </dl>

</dd> <dt><samp class="option">probesize <var class="var">integer</var> (input)</samp></dt> <dd> Set probing size in bytes, i.e. the size of the data to analyze to get stream information. A higher value will enable detecting more information in case it is dispersed into the stream, but will increase latency. Must be an integer not lesser than 32. It is 5000000 by default. </dd> <dt><samp class="option">max_probe_packets <var class="var">integer</var> (input)</samp></dt> <dd> Set the maximum number of buffered packets when probing a codec. Default is 2500 packets. </dd> <dt><samp class="option">packetsize <var class="var">integer</var> (output)</samp></dt> <dd> Set packet size. </dd> <dt><samp class="option">fflags <var class="var">flags</var></samp></dt> <dd> Set format flags. Some are implemented for a limited number of formats. Possible values for input files: <dl class="table"> <dt>‘<samp class="samp">discardcorrupt</samp>’</dt> <dd> Discard corrupted packets. </dd> <dt>‘<samp class="samp">fastseek</samp>’</dt> <dd> Enable fast, but inaccurate seeks for some formats. </dd> <dt>‘<samp class="samp">genpts</samp>’</dt> <dd> Generate missing PTS if DTS is present. </dd> <dt>‘<samp class="samp">igndts</samp>’</dt> <dd> Ignore DTS if PTS is also set. In case the PTS is set, the DTS value is set to NOPTS. This is ignored when the <code class="code">nofillin</code> flag is set. </dd> <dt>‘<samp class="samp">ignidx</samp>’</dt> <dd> Ignore index. </dd> <dt>‘<samp class="samp">nobuffer</samp>’</dt> <dd> Reduce the latency introduced by buffering during initial input streams analysis. </dd> <dt>‘<samp class="samp">nofillin</samp>’</dt> <dd> Do not fill in missing values in packet fields that can be exactly calculated. </dd> <dt>‘<samp class="samp">noparse</samp>’</dt> <dd> Disable AVParsers, this needs <code class="code">+nofillin</code> too. </dd> <dt>‘<samp class="samp">sortdts</samp>’</dt> <dd> Try to interleave output packets by DTS. At present, available only for AVIs with an index. </dd> </dl>

Possible values for output files: <dl class="table"> <dt>‘<samp class="samp">autobsf</samp>’</dt> <dd> Automatically apply bitstream filters as required by the output format. Enabled by default.

</dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. This ensures that file and data checksums are reproducible and match between platforms. Its primary use is for regression testing. </dd>
<dt>‘<samp class="samp">flush_packets</samp>’</dt>
<dd> Write out packets immediately. </dd> <dt>‘<samp class="samp">shortest</samp>’</dt> <dd> Stop muxing at the end of the shortest stream. It may be needed to increase max_interleave_delta to avoid flushing the longer streams before EOF. </dd> </dl>

</dd> <dt><samp class="option">seek2any <var class="var">integer</var> (input)</samp></dt> <dd> Allow seeking to non-keyframes on demuxer level when supported if set to 1. Default is 0. </dd> <dt><samp class="option">analyzeduration <var class="var">integer</var> (input)</samp></dt>
<dd> Specify how many microseconds are analyzed to probe the input. A higher value will enable detecting more accurate information, but will increase latency. It defaults to 5,000,000 microseconds = 5 seconds.
</dd> <dt><samp class="option">cryptokey <var class="var">hexadecimal string</var> (input)</samp></dt> <dd> Set decryption key. </dd> <dt><samp class="option">indexmem <var class="var">integer</var> (input)</samp></dt>
<dd> Set max memory used for timestamp index (per stream).
</dd> <dt><samp class="option">rtbufsize <var class="var">integer</var> (input)</samp></dt> <dd> Set max memory used for buffering real-time frames. </dd> <dt><samp class="option">fdebug <var class="var">flags</var> (input/output)</samp><
<dd> Print specific debug info. Possible values: <dl class="table"> <dt>‘<samp class="samp">ts</samp>’</dt>
<dd> Print timestamp information. </dd> <dt>‘<samp class="samp">id3v2</samp>’</dt> <dd> Print ID3v2 frame data when demuxing files containing ID3v2 tags. </dd> </dl>

</dd> <dt><samp class="option">max_delay <var class="var">integer</var> (input/output)</samp></dt> <dd> Set maximum muxing or demuxing delay in microseconds. </dd>
<dt><samp class="option">fpsprobesize <var class="var">integer</var> (input)</samp></dt> <dd> Set number of frames used to probe fps. </dd> <dt><samp class="option">audio_preload <var class="var">integer</var> (output)</samp></dt>
<dd> Set microseconds by which audio packets should be interleaved earlier. </dd> <dt><samp class="option">chunk_duration <var class="var">integer</var> (output)</samp></dt>
<dd> Set microseconds for each chunk. </dd> <dt><samp class="option">chunk_size <var class="var">integer</var> (output)</samp>
<dd> Set size in bytes for each chunk. </dd> <dt><samp class="option">err_detect, f_err_detect <var class="var">flags</var>

(input)</samp></dt> <dd> Set error detection flags. <code class="code">f_err_detect</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt> <dd> Verify embedded CRCs. </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> Detect bitstream specification deviations. </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> Detect improper bitstream length. </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> Abort decoding on minor error detection. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> Consider things that violate the spec and have not been seen in the wild as errors. </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> Consider all spec non compliancies as errors. </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> Consider things that a sane encoder should not do as an error. </dd> </dl>

</dd> <dt><samp class="option">max_interleave_delta <var class="var">integer</var> (output)</samp></dt> <dd> Set maximum buffering duration for interleaving. The duration is expressed in microseconds, and defaults to 10000000 (10 seconds). To ensure all the streams are interleaved correctly, libavformat will wait until it has at least one packet for each stream before actually writing any packets to the output file. When some streams are "sparse" (i.e. there are large gaps between successive packets), this can result in excessive buffering. This field specifies the maximum difference between the timestamps of the first and the last packet in the muxing queue, above which libavformat will output a packet regardless of whether it has queued a packet for all the streams. If set to 0, libavformat will continue buffering packets until it has a packet for each stream, regardless of the maximum timestamp difference between the buffered packets. </dd> <dt><samp class="option">use_wallclock_as_timestamps <var class="var">integer</var> (input)</samp></dt> <dd> Use wallclock as timestamps if set to 1. Default is 0. </dd> <dt><samp class="option">avoid_negative_ts <var class="var">integer</var> (output)</samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">make_non_negative</samp>’</dt> <dd> Shift timestamps to make them non-negative. Also note that this affects only leading negative timestamps, and not non-monotonic negative timestamps. </dd> <dt>‘<samp class="samp">make_zero</samp>’</dt> <dd> Shift timestamps so that the first timestamp is 0. </dd> <dt>‘<samp class="samp">auto (default)</samp>’</dt> <dd> Enables shifting when required by the target format. </dd> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Disables shifting of timestamp. </dd> </dl>

When shifting is enabled, all output timestamps are shifted by the same amount. Audio, video, and subtitles desynching and relative timestamp differences are preserved compared to how they would have been without shifting. </dd> <dt><samp class="option">skip_initial_bytes <var class="var">integer</var> (input)</samp></dt> <dd> Set number of bytes to skip before reading header and frames if set to 1. Default is 0. </dd> <dt><samp class="option">correct_ts_overflow <var class="var">integer</var> (input)</samp></dt> <dd> Correct single timestamp overflows if set to 1. Default is 1. </dd> <dt><samp class="option">flush_packets <var class="var">integer</var> (output)</samp></dt> <dd> Flush the underlying I/O stream after each packet. Default is -1 (auto), which means that the underlying protocol will decide, 1 enables it, and has the effect of reducing the latency, 0 disables it and may increase IO throughput in some cases. </dd> <dt><samp class="option">output_ts_offset <var class="var">offset</var> (output)</samp></dt> <dd> Set the output time offset. <var class="var">offset</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. The offset is added by the muxer to the output timestamps. Specifying a positive offset means that the corresponding streams are delayed bt the time duration specified in <var class="var">offset</var>. Default value is <code class="code">0</code> (meaning that no offset is applied). </dd> <dt><samp class="option">format_whitelist <var class="var">list</var> (input)</samp></dt> <dd> "," separated list of allowed demuxers. By default all are allowed. </dd> <dt><samp class="option">dump_separator <var class="var">string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class="example-preformatted">ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class="option">max_streams <var class="var">integer</var> (input)</samp></dt> <dd> Specifies the maximum number of streams. This can be used to reject files that would require too many resources due to a large number of streams. </dd> <dt><samp class="option">skip_estimate_duration_from_pts <var class="var">bool</var> (input)</samp></dt> <dd> Skip estimation of input duration if it requires an additional probing for PTS at end of file. At present, applicable for MPEG-PS and MPEG-TS. </dd> <dt><samp class="option">duration_probesize <var class="var">integer</var> (input)</samp></dt>

<dd> Set probing size, in bytes, for input duration estimation when it actually requires an additional probing for PTS at end of file (at present: MPEG-PS and MPEG-TS). It is aimed at users interested in better durations probing for itself, or indirectly because using the concat demuxer, for example. The typical use case is an MPEG-TS CBR with a high bitrate, high video buffering and ending cleaning with similar PTS for video and audio: in such a scenario, the large physical gap between the last video packet and the last audio packet makes it necessary to read many bytes in order to get the video stream duration. Another use case is where the default probing behaviour only reaches a single video frame which is not the last one of the stream due to frame reordering, so the duration is not accurate. Setting this option has a performance impact even for small files because the probing size is fixed. Default behaviour is a general purpose trade-off, largely adaptive, but the probing size will not be extended to get streams durations at all costs. Must be an integer not lesser than 1, or 0 for default behaviour. </dd> <dt><samp class="option">strict, f_strict <var class="var">integer</var> (input/output)</samp></dt> <dd> Specify how strictly to follow the standards. <code class="code">f_strict</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt> <dd> strictly conform to an older more strict version of the spec or reference software </dd> <dt>‘<samp class="samp">strict</samp>’</dt> <dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt> <dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> </dl>

 <h3 class="section">2.1 Format stream specifiers# TOC</h3>

Format stream specifiers allow selection of one or more streams that match specific properties. The exact semantics of stream specifiers is defined by the <code class="code">avformat_match_stream_specifier()</code> function declared in the <samp class="file">libavformat/avformat.h</samp> header and documented in the <a data-manual="ffmpeg" href="./ffmpeg.html#Stream-specifiers">Stream specifiers section in the ffmpeg(1) manual. <h2 class="chapter">3 Demuxers<span

class=”pull-right”># TOC</h2>

Demuxers are configured elements in FFmpeg that can read the multimedia streams from a particular type of file. When you configure your FFmpeg build, all the supported demuxers are enabled by default. You can list all available ones using the configure option <code class=”code”>--list-demuxers</code>. You can disable all the demuxers using the configure option <code class=”code”>--disable-demuxers</code>, and selectively enable a single demuxer with the option <code class=”code”>--enable-demuxer=<var class=”var”>DEMUXER</var></code>, or disable it with the option <code class=”code”>--disable-demuxer=<var class=”var”>DEMUXER</var></code>. The option <code class=”code”>-demuxers</code> of the ff* tools will display the list of enabled demuxers. Use <code class=”code”>-formats</code> to view a combined list of enabled demuxers and muxers. The description of some of the currently available demuxers follows. <h3 class=”section”>3.1 aa# TOC</h3>

Audible Format 2, 3, and 4 demuxer. This demuxer is used to demux Audible Format 2, 3, and 4 (.aa) files. <h3 class=”section”>3.2 aac# TOC</h3>

Raw Audio Data Transport Stream AAC demuxer. This demuxer is used to demux an ADTS input containing a single AAC stream alongwith any ID3v1/2 or APE tags in it. <h3 class=”section”>3.3 apng# TOC</h3>

Animated Portable Network Graphics demuxer. This demuxer is used to demux APNG files. All headers, but the PNG signature, up to (but not including) the first fcTL chunk are transmitted as extradata. Frames are then split as being all the chunks between two fcTL ones, or between the last fcTL and IEND chunks. <dl class=”table”> <dt><samp class=”option”>-ignore_loop <var class=”var”>bool</var></samp></dt> <dd> Ignore the loop variable in the file if set. Default is enabled. </dd> <dt><samp class=”option”>-max_fps <var class=”var”>int</var></samp></dt> <dd> Maximum framerate in frames per second. Default of 0 imposes no limit. </dd> <dt><samp class=”option”>-default_fps <var class=”var”>int</var></samp></dt> <dd> Default framerate in frames per second when none is specified in the file (0 meaning as fast as possible). Default is 15. </dd> </dl>

 <h3 class="section">3.4 asf# TOC</h3>

Advanced Systems Format demuxer. This demuxer is used to demux ASF files and MMS network streams. <dl class="table"> <dt><samp class="option">-no_resync_search <var class="var">bool</var></samp></dt> <dd> Do not try to resynchronize by looking for a certain optional start code. </dd> </dl>

 <h3 class="section">3.5 astc# TOC</h3>

ASTC (Adaptive Scalable Texture Compression) demuxer. This demuxer reads the 16-byte <samp class="file">.astc</samp> header, hands it to the decoder as extradata, and emits the compressed image as a single packet. <h3 class="section">3.6 concat# TOC</h3>

Virtual concatenation script demuxer. This demuxer reads a list of files and other directives from a text file and demuxes them one after the other, as if all their packets had been muxed together. The timestamps in the files are adjusted so that the first file starts at 0 and each next file starts where the previous one finishes. Note that it is done globally and may cause gaps if all streams do not have exactly the same length. All files must have the same streams (same codecs, same time base, etc.). The duration of each file is used to adjust the timestamps of the next file: if the duration is incorrect (because it was computed using the bit-rate or because the file is truncated, for example), it can cause artifacts. The <code class="code">duration</code> directive can be used to override the duration stored in each file. <h4 class="subsection">3.6.1 Syntax# TOC</h4>

The script is a text file in extended-ASCII, with one directive per line.

Empty lines, leading spaces and lines starting with ’#’ are ignored. The

following directive is recognized:

<dl class="table">

<dt><samp class="option"><code class="code">file <var class="var">path</var></code></samp></dt

<dd> Path to a file to read; special characters and spaces must be escaped with

backslash or single quotes.

All subsequent file-related directives apply to that file.

</dd>

<dt><samp class="option"><code class="code">ffconcat version 1.0</code></samp></dt>

<dd> Identify the script type and version.

To make FFmpeg recognize the format automatically, this directive must

appear exactly as is (no extra space or byte-order-mark) on the very first

line of the script.

</dd>

<dt><samp class="option"><code class="code">duration <var class="var">dur</var></code></samp>

<dd> Duration of the file. This information can be specified from the file;

specifying it here may be more efficient or help if the information from the

file is not available or accurate.

If the duration is set for all files, then it is possible to seek in the

whole concatenated video.

</dd>

<dt><samp class="option"><code class="code">inpoint <var class="var">timestamp</var></code></sa

<dd> In point of the file. When the demuxer opens the file it instantly seeks to the

specified timestamp. Seeking is done so that all streams can be presented

successfully at In point.

This directive works best with intra frame codecs, because for non-intra frame

ones you will usually get extra packets before the actual In point and the
decoded content will most likely contain frames before In point too.

For each file, packets before the file In point will have timestamps less than
the calculated start timestamp of the file (negative in case of the first
file), and the duration of the files (if not specified by the <code class="code">duration</code>
directive) will be reduced based on their specified In point.

Because of potential packets before the specified In point, packet timestamps
may overlap between two concatenated files.

</dd>
<dt><samp class="option"><code class="code">outpoint <var class="var">timestamp</var></code></
<dd> Out point of the file. When the demuxer reaches the specified decoding
timestamp in any of the streams, it handles it as an end of file condition and
skips the current and all the remaining packets from all streams.

Out point is exclusive, which means that the demuxer will not output packets
with a decoding timestamp greater or equal to Out point.

This directive works best with intra frame codecs and formats where all streams
are tightly interleaved. For non-intra frame codecs you will usually get
additional packets with presentation timestamp after Out point therefore the
decoded content will most likely contain frames after Out point too. If your
streams are not tightly interleaved you may not get all the packets from all
streams before Out point and you may only will be able to decode the earliest
stream until Out point.

The duration of the files (if not specified by the <code class=”code”>duration</code>

directive) will be reduced based on their specified Out point.

</dd>

<dt><samp class=”option”><code class=”code”>file_packet_metadata <var class=”var”>key=value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

This directive is deprecated, use <code class=”code”>file_packet_meta</code> instead.

</dd>

<dt><samp class=”option”><code class=”code”>file_packet_meta <var class=”var”>key</var> <var class=”var”>value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

</dd>

<dt><samp class=”option”><code class=”code”>option <var class=”var”>key</var> <var class=”var”>value</var></code></samp></dt>

<dd> Option to access, open and probe the file.

Can be present multiple times.

</dd>

<dt><samp class=”option”><code class=”code”>stream</code></samp></dt>

<dd> Introduce a stream in the virtual file.

All subsequent stream-related directives apply to the last introduced

stream.

Some streams properties must be set in order to allow identifying the

matching streams in the subfiles.

If no streams are defined in the script, the streams from the first file are

copied.

</dd>

<dt><samp class="option"><code class="code">exact_stream_id <var class="var">id</var></code></samp></dt>

<dd> Set the id of the stream.

If this directive is given, the string with the corresponding id in the

subfiles will be used.

This is especially useful for MPEG-PS (VOB) files, where the order of the

streams is not reliable.

</dd>

<dt><samp class="option"><code class="code">stream_meta <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Metadata for the stream.

Can be present multiple times.

</dd>

<dt><samp class="option"><code class="code">stream_codec <var class="var">value</var></code></samp></dt>

<dd> Codec for the stream.

</dd>

<dt><samp class="option"><code class="code">stream_extradata <var class="var">hex_string</var></code></samp></dt>

<dd> Extradata for the string, encoded in hexadecimal.

</dd>

<dt><samp class="option"><code class="code">chapter <var class="var">id</var> <var class="var">start</var> <var class="var">end</var></code></samp></dt>

<dd> Add a chapter. <var class=”var”>id</var> is an unique identifier, possibly small and

consecutive.

</dd>

</dl>

 <h4 class=”subsection”>3.6.2 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>safe</samp></dt> <dd> If set to 1, reject unsafe file paths and directives. A file path is considered safe if it does not contain a protocol specification and is relative and all components only contain characters from the portable character set (letters, digits, period, underscore and hyphen) and have no period at the beginning of a component. If set to 0, any file name is accepted. The default is 1. </dd> <dt><samp class=”option”>auto_convert</samp></dt> <dd> If set to 1, try to perform automatic conversions on packet data to make the streams concatenable. The default is 1. Currently, the only conversion is adding the h264_mp4toannexb bitstream filter to H.264 streams in MP4 format. This is necessary in particular if there are resolution changes. </dd> <dt><samp class=”option”>segment_time_metadata</samp></dt> <dd> If set to 1, every packet will contain the <var class=”var”>lavf.concat.start_time</var> and the <var class=”var”>lavf.concat.duration</var> packet metadata values which are the start_time and the duration of the respective file segments in the concatenated output expressed in microseconds. The duration metadata is only set if it is known based on the concat file. The default is 0. </dd> <dt><samp class=”option”>chapter_per_file</samp></dt> <dd> If set to 1, add a chapter for each file, named after the <code class=”code”>title</code> metadata of the file or, failing that, its file name, and carrying the rest of the file’s metadata. The files are opened in order when the script is read to learn their durations, stopping after the first file whose duration is unknown, which gets the last chapter; when all durations are known, this also makes the total duration known and the output seekable. The default is 0. </dd> </dl>

 <h4 class=”subsection”>3.6.3 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use absolute filenames and include some

comments: <pre class=”example-preformatted”># my first filename file /mnt/share/file-1.wav # my second filename including whitespace file '/mnt/share/file 2.wav' # my third filename including whitespace plus single quote file '/mnt/share/file 3'\''.wav' </pre>

Allow for input format auto-probing, use safe filenames and set the duration of the first file: <pre class=”example-preformatted”>ffconcat version 1.0

file file-1.wav duration 20.0

file subdir/file-2.wav </pre>

 <h3 class=”section”>3.7 dash# TOC</h3>

Dynamic Adaptive Streaming over HTTP demuxer. This demuxer presents all AVStreams found in the manifest. By setting the discard flags on AVStreams the caller can decide which streams to actually receive. Each stream mirrors the <code class=”code”>id</code> and <code class=”code”>bandwidth</code> properties from the <code class=”code”><Representation></code> as metadata keys named "id" and "variant_bitrate" respectively. <h4 class=”subsection”>3.7.1 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>cenc_decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>cenc_decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> </dl>

 <h3 class=”section”>3.8 dvdvideo# TOC</h3>

DVD-Video demuxer, powered by libdvdnav and libdvdread. Can directly ingest DVD titles, specifically sequential PGCs, into a conversion pipeline. Menu assets, such as background video or audio, can also be demuxed given the menu’s coordinates (at best effort). Block devices (DVD drives), ISO files, and directory structures are accepted. Activate with <code class=”code”>-f dvdvideo</code> in front of one of these inputs. This demuxer does NOT have decryption code

of any kind. You are on your own working with encrypted DVDs, and should not expect support on the matter. Underlying playback is handled by libdvdnav, and structure parsing by libdvdread. FFmpeg must be built with GPL library support available as well as the configure switches <code class=”code”>--enable-libdvdnav</code> and <code class=”code”>--enable-libdvdread</code>. You will need to provide either the desired "title number" or exact PGC/PG coordinates. Many open-source DVD players and tools can aid in providing this information. If not specified, the demuxer will default to title 1 which works for many discs. However, due to the flexibility of the format, it is recommended to check manually. There are many discs that are authored strangely or with invalid headers. If the input is a real DVD drive, please note that there are some drives which may silently fail on reading bad sectors from the disc, returning random bits instead which is effectively corrupt data. This is especially prominent on aging or rotting discs. A second pass and integrity checks would be needed to detect the corruption. This is not an FFmpeg issue. <h4 class=”subsection”>3.8.1 Background# TOC</h4>

DVD-Video is not a directly accessible, linear container format in the traditional sense. Instead, it allows for complex and programmatic playback of carefully muxed MPEG-PS streams that are stored in headerless VOB files. To the end-user, these streams are known simply as "titles", but the actual logical playback sequence is defined by one or more "PGCs", or Program Group Chains, within the title. The PGC is in turn comprised of multiple "PGs", or Programs", which are the actual video segments (and for a typical video feature, sequentially ordered). The PGC structure, along with stream layout and metadata, are stored in IFO files that need to be parsed. PGCs can be thought of as playlists in easier terms. An actual DVD player relies on user GUI interaction via menus and an internal VM to drive the direction of demuxing. Generally, the user would either navigate (via menus) or automatically be redirected to the PGC of their choice. During this process and the subsequent playback, the DVD player’s internal VM also maintains a state and executes instructions that can create jumps to different sectors during playback. This is why libdvdnav is involved, as a linear read of the MPEG-PS blobs on the disc (VOBs) is not enough to produce the right sequence in many cases. There are many other DVD structures (a long subject) that will not be discussed here. NAV packets, in particular, are handled by this demuxer to build accurate timing but not emitted as a stream. For a good high-level understanding, refer to: https://code.videolan.org/videolan/libdvdnav/-/blob/master/doc/dvd_structures <h4 class=”subsection”>3.8.2 Options<a class=”anchor

hidden-xs" href="#Options-2" aria-hidden="true"># TOC</h4>

This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">title <var class="var">int</var></samp></dt> <dd> The title number to play. Must be set if <samp class="option">pgc</samp> and <samp class="option">pg</samp> are not set. Not applicable to menus. Default is 0 (auto), which currently only selects the first available title (title 1) and notifies the user about the implications. </dd> <dt><samp class="option">chapter_start <var class="var">int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to start at. Not applicable to menus. Default is 1. </dd> <dt><samp class="option">chapter_end <var class="var">int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to end at. Not applicable to menus. Default is 0, which is a special value to signal end at the last possible chapter. </dd> <dt><samp class="option">angle <var class="var">int</var></samp></dt> <dd> The video angle number, referring to what is essentially an additional video stream that is composed from alternate frames interleaved in the VOBs. Not applicable to menus. Default is 1. </dd> <dt><samp class="option">region <var class="var">int</var></samp></dt> <dd> The region code to use for playback. Some discs may use this to default playback at a particular angle in different regions. This option will not affect the region code of a real DVD drive, if used as an input. Not applicable to menus. Default is 0, "world". </dd> <dt><samp class="option">menu <var class="var">bool</var></samp></dt> <dd> Demux menu assets instead of navigating a title. Requires exact coordinates of the menu (<samp class="option">menu_lu</samp>, <samp class="option">menu_vts</samp>, <samp class="option">pgc</samp>, <samp class="option">pg</samp>). Default is false. </dd> <dt><samp class="option">menu_lu <var class="var">int</var></samp></dt> <dd> The menu language to demux. In DVD, menus are grouped by language. Default is 1, the first language unit. </dd> <dt><samp class="option">menu_vts <var class="var">int</var></samp></dt> <dd> The VTS where the menu lives, or 0 if it is a VMG menu (root-level). Default is 1, menu of the first VTS. </dd> <dt><samp class="option">pgc <var class="var">int</var></samp></dt> <dd> The entry PGC to start playback, in conjunction with <samp class="option">pg</samp>. Alternative to setting <samp class="option">title</samp>. Chapter markers are not supported at this time. Must be explicitly set for menus. Default is 0, automatically resolve from value of <samp class="option">title</samp>. </dd> <dt><samp class="option">pg <var class="var">int</var></samp></dt> <dd> The entry PG to start playback, in conjunction with <samp class="option">pgc</samp>. Alternative to setting <samp class="option">title</samp>. Chapter markers are not supported at this time. Default is 1, the first PG of

the PGC. </dd> <dt><samp class="option">preindex <var class="var">bool</var></samp></dt> <dd> Enable this to have accurate chapter (PTT) markers and duration measurement, which requires a slow second pass read in order to index the chapter marker timestamps from NAV packets. This is non-ideal extra work for real optical drives. It is recommended and faster to use this option with a backup of the DVD structure stored on a hard drive. Not compatible with <samp class="option">pgc</samp> and <samp class="option">pg</samp>. Default is 0, false. </dd> <dt><samp class="option">trim <var class="var">bool</var></samp></dt> <dd> Skip padding cells (i.e. cells shorter than 1 second) from the beginning. There exist many discs with filler segments at the beginning of the PGC, often with junk data intended for controlling a real DVD player’s buffering speed and with no other material data value. Not applicable to menus. Default is 1, true. </dd> </dl>

 <h4 class="subsection">3.8.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Open title 3 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -title 3 -i <path to DVD> ... </pre>

Open chapters 3-6 from title 1 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -chapter_start 3 -chapter_end 6 -title 1 -i <path to DVD> ... </pre>

Open only chapter 5 from title 1 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -chapter_start 5 -chapter_end 5 -title 1 -i <path to DVD> ... </pre>

Demux menu with language 1 from VTS 1, PGC 1, starting at PG 1: <pre class="example-preformatted">ffmpeg -f dvdvideo -menu 1 -menu_lu 1 -menu_vts 1 -pgc 1 -pg 1 -i <path to DVD> ... </pre>

 <h3 class="section">3.9 ea# TOC</h3>

Electronic Arts Multimedia format demuxer. This format is used by various Electronic Arts games. <h4 class="subsection">3.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">merge_alpha <var class="var">bool</var></samp></dt> <dd> Normally the VP6 alpha channel (if exists) is returned as a secondary video stream, by setting this option you can make the demuxer return a single video stream which contains the alpha channel in addition to the ordinary video. </dd> </dl>

 <h3 class="section">3.10 imf# TOC</h3>

Interoperable Master Format demuxer. This demuxer presents audio and video streams found in an IMF Composition, as specified in SMPTE ST 2067-2.   <pre class="example-preformatted">ffmpeg [-assetmaps <path of ASSETMAP1>,<path of ASSETMAP2>,...] -i <path of CPL> ... </pre>

If <code class="code">-assetmaps</code> is not specified, the demuxer looks for a file called <samp class="file">ASSETMAP.xml</samp> in the same directory as the CPL. <h3 class="section">3.11 flv, live_flv, kux# TOC</h3>

Adobe Flash Video Format demuxer. This demuxer is used to demux FLV files and RTMP network streams. In case of live network streams, if you force format, you may use live_flv option instead of flv to survive timestamp discontinuities. KUX is a flv variant used on the Youku platform.   <pre class="example-preformatted">ffmpeg -f flv -i myfile.flv ... ffmpeg -f live_flv -i rtmp://<any.server>/anything/key .... </pre>

<dl class="table"> <dt><samp class="option">-flv_metadata <var class="var">bool</var></samp></dt> <dd> Allocate the streams according to the onMetaData array content. </dd> <dt><samp class="option">-flv_ignore_prevtag <var class="var">bool</var></samp></dt> <dd> Ignore the size of previous tag value. </dd> <dt><samp class="option">-flv_full_metadata <var class="var">bool</var></samp></dt> <dd> Output all context of the onMetadata. </dd> </dl>

 <h3 class="section">3.12 gif# TOC</h3>

Animated GIF demuxer. It accepts the following options: <dl class="table"> <dt><samp class="option">min_delay</samp></dt> <dd> Set the minimum valid delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 2. </dd> <dt><samp

class=”option”>max_gif_delay</samp></dt> <dd> Set the maximum valid delay between frames in hundredth of seconds. Range is 0 to 65535. Default value is 65535 (nearly eleven minutes), the maximum value allowed by the specification. </dd> <dt><samp class=”option”>default_delay</samp></dt> <dd> Set the default delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 10. </dd> <dt><samp class=”option”>ignore_loop</samp></dt> <dd> GIF files can contain information to loop a certain number of times (or infinitely). If <samp class=”option”>ignore_loop</samp> is set to 1, then the loop setting from the input will be ignored and looping will not occur. If set to 0, then looping will occur and will cycle the number of times according to the GIF. Default value is 1. </dd> </dl>

For example, with the overlay filter, place an infinitely looping GIF over another video: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -ignore_loop 0 -i input.gif -filter_complex overlay=shortest=1 out.mkv </pre>

Note that in the above example the shortest option for overlay filter is used to end the output video at the length of the shortest input file, which in this case is <samp class=”file”>input.mp4</samp> as the GIF in this example loops infinitely. <h3 class=”section”>3.13 hls# TOC</h3>

HLS demuxer Apple HTTP Live Streaming demuxer. This demuxer presents all AVStreams from all variant streams. The id field is set to the bitrate variant index number. By setting the discard flags on AVStreams (by pressing ’a’ or ’v’ in ffplay), the caller can decide which variant streams to actually receive. The total bitrate of the variant that the stream belongs to is available in a metadata key named "variant_bitrate". It is unset when the stream is shared by variants with different bitrates. The value is always available in the corresponding program’s metadata. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>live_start_index</samp></dt> <dd> segment index to start live streams at (negative values are from the end). </dd> <dt><samp class=”option”>prefer_x_start</samp></dt> <dd> prefer to use #EXT-X-START if it’s in playlist instead of live_start_index. </dd> <dt><samp class=”option”>allowed_extensions</samp></dt> <dd> ’,’ separated list of file extensions that hls is allowed to access. </dd> <dt><samp class=”option”>extension_picky</samp></dt> <dd> This blocks disallowed extensions from probing It also requires all available segments to have matching extensions to the format except mpegts, which is always allowed. It is recommended to set the whitelists correctly instead of depending on extensions Enabled by default. </dd> <dt><samp

class=”option”>max_reload</samp></dt> <dd> Maximum number of times a insufficient list is attempted to be reloaded. Default value is 1000. </dd> <dt><samp class=”option”>m3u8_hold_counters</samp></dt> <dd> The maximum number of times to load m3u8 when it refreshes without new segments. Default value is 1000. </dd> <dt><samp class=”option”>http_persistent</samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP streams. Enabled by default. </dd> <dt><samp class=”option”>http_multiple</samp></dt> <dd> Use multiple HTTP connections for downloading HTTP segments. Enabled by default for HTTP/1.1 servers. </dd> <dt><samp class=”option”>http_seekable</samp></dt> <dd> Use HTTP partial requests for downloading HTTP segments. 0 = disable, 1 = enable, -1 = auto, Default is auto. </dd> <dt><samp class=”option”>seg_format_options</samp></dt> <dd> Set options for the demuxer of media segments using a list of key=value pairs separated by <code class=”code”>:</code>. </dd> <dt><samp class=”option”>seg_max_retry</samp></dt> <dd> Maximum number of times to reload a segment on error, useful when segment skip on network error is not desired. Default value is 0. </dd> </dl>

 <h3 class=”section”>3.14 image2# TOC</h3>

Image file demuxer. This demuxer reads from a list of image files specified by a pattern. The syntax and meaning of the pattern is specified by the option <var class=”var”>pattern_type</var>. The pattern may contain a suffix which is used to automatically determine the format of the images contained in the files. The size, the pixel format, and the format of each image must be the same for all the files in the sequence. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate for the video stream. It defaults to 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the input. Default value is 0. </dd> <dt><samp class=”option”>pattern_type</samp></dt> <dd> Select the pattern type used to interpret the provided filename. <var class=”var”>pattern_type</var> accepts one of the following values. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Disable pattern matching, therefore the video will only contain the specified image. You should use this option if you do not want to create sequences from multiple images and your filenames may contain special pattern characters. </dd> <dt><samp class=”option”>sequence</samp></dt> <dd> Select a sequence pattern type, used to specify a sequence of files indexed by sequential numbers. A sequence pattern may contain the string "%d" or

"%0<var class=”var”>N</var>d", which specifies the position of the characters representing a sequential number in each filename matched by the pattern. If the form "%d0<var class=”var”>N</var>d" is used, the string representing the number in each filename is 0-padded and <var class=”var”>N</var> is the total number of 0-padded digits representing the number. The literal character ’%’ can be specified in the pattern with the string "%%". If the sequence pattern contains "%d" or "%0<var class=”var”>N</var>d", the first filename of the file list specified by the pattern must contain a number inclusively contained between <var class=”var”>start_number</var> and <var class=”var”>start_number</var>+<var class=”var”>start_number_range</var>-1, and all the following numbers must be sequential. For example the pattern "img-%03d.bmp" will match a sequence of filenames of the form <samp class=”file”>img-001.bmp</samp>, <samp class=”file”>img-002.bmp</samp>, ..., <samp class=”file”>img-010.bmp</samp>, etc.; the pattern "i%%m%%g-%d.jpg" will match a sequence of filenames of the form <samp class=”file”>i%m%g-1.jpg</samp>, <samp class=”file”>i%m%g-2.jpg</samp>, ..., <samp class=”file”>i%m%g-10.jpg</samp>, etc. Note that the pattern must not necessarily contain "%d" or "%0<var class=”var”>N</var>d", for example to convert a single image file <samp class=”file”>img.jpeg</samp> you can employ the command: <pre class=”example-preformatted”>ffmpeg -i img.jpeg img.png </pre>

</dd> <dt><samp class=”option”>glob</samp></dt> <dd> Select a glob wildcard pattern type. The pattern is interpreted like a <code class=”code”>glob()</code> pattern. This is only selectable if libavformat was compiled with globbing support. </dd> </dl>

Default value is <var class=”var”>sequence</var>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Set the pixel format of the images to read. If not specified the pixel format is guessed from the first image file in the sequence. </dd> <dt><samp class=”option”>start_number</samp></dt> <dd> Set the index of the file matched by the image file pattern to start to read from. Default value is 0. </dd> <dt><samp class=”option”>start_number_range</samp></dt> <dd> Set the index interval range to check when looking for the first image file in the sequence, starting from <var class=”var”>start_number</var>. Default value is 5. </dd> <dt><samp class=”option”>ts_from_file</samp></dt> <dd> If set to 1, will set frame timestamp to modification time of image file. Note that monotonity of timestamps is not provided: images go in the same order as without this option. Default value is 0. If set to 2, will set frame timestamp to the modification time of the image file in nanosecond precision. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size of the images to read. If not specified the video size is guessed from the first image file in the sequence. </dd> <dt><samp

class=”option”>export_path_metadata</samp></dt> <dd> If set to 1, will add two extra fields to the metadata found in input, making them also available for other filters (see <var class=”var”>drawtext</var> filter for examples). Default value is 0. The extra fields are described below: <dl class=”table”> <dt><samp class=”option”>lavf.image2dec.source_path</samp></dt> <dd> Corresponds to the full path to the input file being read. </dd> <dt><samp class=”option”>lavf.image2dec.source_basename</samp></dt> <dd> Corresponds to the name of the file being read. </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>3.14.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> for creating a video from the images in the file sequence <samp class=”file”>img-001.jpeg</samp>, <samp class=”file”>img-002.jpeg</samp>, ..., assuming an input frame rate of 10 frames per second: <pre class=”example-preformatted”>ffmpeg -framerate 10 -i 'img-%03d.jpeg' out.mkv </pre>

As above, but start by reading from a file with index 100 in the sequence: <pre class=”example-preformatted”>ffmpeg -framerate 10 -start_number 100 -i 'img-%03d.jpeg' out.mkv </pre>

Read images matching the "*.png" glob pattern , that is all the files terminating with the ".png" suffix: <pre class=”example-preformatted”>ffmpeg -framerate 10 -pattern_type glob -i "*.png" out.mkv </pre>

 <h3 class=”section”>3.15 jpeg_mpf# TOC</h3>

JPEG Multi-Picture Format demuxer. Exposes each JPEG image stored in a CIPA DC-007 MP file as a separate stream, in the order of the MP Entries, with the First Individual Image flagged as the default stream. If the file contains an ISO 21496-1 gain map, as is the case for JPEG Ultra HDR, the primary image and the gain map are linked by a ‘<samp class=”samp”>gain_map</samp>’ stream group, and the gain map parameters are exported as frame side data by the decoder. Images of any other type (e.g. large thumbnails or multi-view images) are exposed as additional streams. This demuxer accepts the following options, which behave like the options of the same name of the ‘<samp class=”samp”>image2</samp>’ demuxer: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the

frame rate of the video streams. The default is 25. </dd> <dt><samp class="option">loop</samp></dt> <dd> If set to 1, loop over the images in the file. The default is 0. This requires seekable input. </dd> </dl>

 <h3 class="section">3.16 ktx# TOC</h3>

Khronos Texture 1.0 demuxer for ASTC textures. This demuxer reads the KTX 1.0 header and maps the ASTC GL internal format to the block footprint the decoder needs, so only files using the ASTC formats are accepted. A <code class="code">KTXorientation</code> key is preserved as stream display metadata, and the texture is emitted as a single packet. <h3 class="section">3.17 libgme# TOC</h3>

The Game Music Emu library is a collection of video game music file emulators. See https://bitbucket.org/mpyne/game-music-emu/overview for more information. It accepts the following options: <dl class="table"> <dt><samp class="option">track_index</samp></dt> <dd> Set the index of which track to demux. The demuxer can only export one track. Track indexes start at 0. Default is to pick the first track. Number of tracks is exported as <var class="var">tracks</var> metadata entry. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sampling rate of the exported track. Range is 1000 to 999999. Default is 44100. </dd> <dt><samp class="option">max_size (bytes)</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Default is 50 MiB. </dd> </dl>

 <h3 class="section">3.18 libmodplug# TOC</h3>

ModPlug based module demuxer See https://github.com/Konstanty/libmodplug It will export one 2-channel 16-bit 44.1 kHz audio stream. Optionally, a <code class="code">pal8</code> 16-color video stream can be exported with or without printed metadata. It accepts the following options: <dl class="table"> <dt><samp class="option">noise_reduction</samp></dt> <dd> Apply a simple low-pass filter. Can be 1 (on) or 0 (off). Default is 0. </dd>

<dt><samp class=”option”>reverb_depth</samp></dt> <dd> Set amount of reverb. Range 0-100. Default is 0. </dd> <dt><samp class=”option”>reverb_delay</samp></dt> <dd> Set delay in ms, clamped to 40-250 ms. Default is 0. </dd> <dt><samp class=”option”>bass_amount</samp></dt> <dd> Apply bass expansion a.k.a. XBass or megabass. Range is 0 (quiet) to 100 (loud). Default is 0. </dd> <dt><samp class=”option”>bass_range</samp></dt> <dd> Set cutoff i.e. upper-bound for bass frequencies. Range is 10-100 Hz. Default is 0. </dd> <dt><samp class=”option”>surround_depth</samp></dt> <dd> Apply a Dolby Pro-Logic surround effect. Range is 0 (quiet) to 100 (heavy). Default is 0. </dd> <dt><samp class=”option”>surround_delay</samp></dt> <dd> Set surround delay in ms, clamped to 5-40 ms. Default is 0. </dd> <dt><samp class=”option”>max_size</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Range is 0 to 100 MiB. 0 removes buffer size limit (not recommended). Default is 5 MiB. </dd> <dt><samp class=”option”>video_stream_expr</samp></dt> <dd> String which is evaluated using the eval API to assign colors to the generated video stream. Variables which can be used are <code class=”code”>x</code>, <code class=”code”>y</code>, <code class=”code”>w</code>, <code class=”code”>h</code>, <code class=”code”>t</code>, <code class=”code”>speed</code>, <code class=”code”>tempo</code>, <code class=”code”>order</code>, <code class=”code”>pattern</code> and <code class=”code”>row</code>. </dd> <dt><samp class=”option”>video_stream</samp></dt> <dd> Generate video stream. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class=”option”>video_stream_w</samp></dt> <dd> Set video frame width in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class=”option”>video_stream_h</samp></dt> <dd> Set video frame height in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class=”option”>video_stream_ptxt</samp></dt> <dd> Print metadata on video stream. Includes <code class=”code”>speed</code>, <code class=”code”>tempo</code>, <code class=”code”>order</code>, <code class=”code”>pattern</code>, <code class=”code”>row</code> and <code class=”code”>ts</code> (time in ms). Can be 1 (on) or 0 (off). Default is 1. </dd> </dl>

 <h3 class=”section”>3.19 libopenmpt# TOC</h3>

libopenmpt based module demuxer See https://lib.openmpt.org/libopenmpt/ for more information. Some files have multiple subsongs (tracks) this

can be set with the <samp class=”option”>subsong</samp> option. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>subsong</samp></dt> <dd> Set the subsong index. This can be either ’all’, ’auto’, or the index of the subsong. Subsong indexes start at 0. The default is ’auto’. The default value is to let libopenmpt choose. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> Set the channel layout. Valid values are 1, 2, and 4 channel layouts. The default value is STEREO. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate for libopenmpt to output. Range is from 1000 to INT_MAX. The value default is 48000. </dd> </dl>

 <h3 class=”section”>3.20 mcc# TOC</h3>

Demuxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-608 and CEA-708), ancillary time code, pan-scan data, etc. By default, for backward compatibility, the MCC demuxer extracts just the EIA-608 and CEA-708 closed captions and returns a <code class=”code”>EIA_608</code> stream, ignoring all other VANC data. You can change it to return all VANC data in a <code class=”code”>SMPTE_436M_ANC</code> data stream by setting <samp class=”option”>-eia608_extract 0</samp> <h4 class=”subsection”>3.20.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert a MCC file to Scenarist (SCC) format: <pre class=”example-preformatted”>ffmpeg -i CC.mcc -c:s copy CC.scc </pre> Note that the SCC format only supports EIA-608, so this will discard all other data such as CEA-708 extensions. Merge a MCC file into a MXF file: <pre class=”example-preformatted”>ffmpeg -i video_and_audio.mxf -eia608_extract 0 -i CC.mcc -c copy -map 0 -map 1 out.mxf </pre> This retains all VANC data and inserts it into the output MXF file as a <code class=”code”>SMPTE_436M_ANC</code> data stream.

 <h3 class=”section”>3.21 mov/mp4/3gp# TOC</h3>

Demuxer for Quicktime File Format & ISO/IEC Base Media File Format (ISO/IEC 14496-12 or MPEG-4 Part 12, ISO/IEC 15444-

12 or JPEG 2000 Part 12). Registered extensions: mov, mp4, m4a, 3gp, 3g2, mj2, psp, m4b, ism, ismv, isma, f4v <h4 class="subsection">3.21.1 Options# TOC</h4>

This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">enable_drefs</samp></dt> <dd> Enable loading of external tracks, disabled by default. Enabling this can theoretically leak information in some use cases. </dd> <dt><samp class="option">use_absolute_path</samp></dt> <dd> Allows loading of external tracks via absolute paths, disabled by default. Enabling this poses a security risk. It should only be enabled if the source is known to be non-malicious. </dd> <dt><samp class="option">seek_streams_individually</samp></dt> <dd> When seeking, identify the closest point in each stream individually and demux packets in that stream from identified point. This can lead to a different sequence of packets compared to demuxing linearly from the beginning. Default is true. </dd> <dt><samp class="option">ignore_editlist</samp></dt> <dd> Ignore any edit list atoms. The demuxer, by default, modifies the stream index to reflect the timeline described by the edit list. Default is false. </dd> <dt><samp class="option">advanced_editlist</samp></dt> <dd> Modify the stream index to reflect the timeline described by the edit list. <code class="code">ignore_editlist</code> must be set to false for this option to be effective. If both <code class="code">ignore_editlist</code> and this option are set to false, then only the start of the stream index is modified to reflect initial dwell time or starting timestamp described by the edit list. Default is true. </dd> <dt><samp class="option">ignore_chapters</samp></dt> <dd> Don’t parse chapters. This includes GoPro ’HiLight’ tags/moments. Note that chapters are only parsed when input is seekable. Default is false. </dd> <dt><samp class="option">use_mfra_for</samp></dt> <dd> For seekable fragmented input, set fragment’s starting timestamp from media fragment random access box, if present. Following options are available: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Auto-detect whether to set mfra timestamps as PTS or DTS (default) </dd> <dt>‘<samp class="samp">dts</samp>’</dt> <dd> Set mfra timestamps as DTS </dd> <dt>‘<samp class="samp">pts</samp>’</dt> <dd> Set mfra timestamps as PTS </dd> <dt>‘<samp class="samp">0</samp>’</dt> <dd> Don’t use mfra box to set timestamps </dd> </dl>

</dd> <dt><samp class="option">use_tfdt</samp></dt> <dd> For fragmented input, set fragment’s starting timestamp to <code class="code">baseMediaDecodeTime</code> from the <code class="code">tfdt</code>

box. Default is enabled, which will prefer to use the <code class=”code”>tfdt</code> box to set DTS. Disable to use the <code class=”code”>earliest_presentation_time</code> from the <code class=”code”>sidx</code> box. In either case, the timestamp from the <code class=”code”>mfra</code> box will be used if it’s available and <code class=”code”>use_mfra_for</code> is set to pts or dts. </dd> <dt><samp class=”option”>export_all</samp></dt> <dd> Export unrecognized boxes within the <var class=”var”>udta</var> box as metadata entries. The first four characters of the box type are set as the key. Default is false. </dd> <dt><samp class=”option”>export_xmp</samp></dt> <dd> Export entire contents of <var class=”var”>XMP_</var> box and <var class=”var”>uuid</var> box as a string with key <code class=”code”>xmp</code>. Note that if <code class=”code”>export_all</code> is set and this option isn’t, the contents of <var class=”var”>XMP_</var> box are still exported but with key <code class=”code”>XMP_</code>. Default is false. </dd> <dt><samp class=”option”>activation_bytes</samp></dt> <dd> 4-byte key required to decrypt Audible AAX and AAX+ files. See Audible AAX subsection below. </dd> <dt><samp class=”option”>audible_fixed_key</samp></dt> <dd> Fixed key used for handling Audible AAX/AAX+ files. It has been pre-set so should not be necessary to specify. </dd> <dt><samp class=”option”>decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>max_stts_delta</samp></dt> <dd> Very high sample deltas written in a trak’s stts box may occasionally be intended but usually they are written in error or used to store a negative value for dts correction when treated as signed 32-bit integers. This option lets the user set an upper limit, beyond which the delta is clamped to 1. Values greater than the limit if negative when cast to int32 are used to adjust onward dts. Unit is the track time scale. Range is 0 to UINT_MAX. Default is <code class=”code”>UINT_MAX - 48000*10</code> which allows up to a 10 second dts correction for 48 kHz audio streams while accommodating 99.9% of <code class=”code”>uint32</code> range. </dd> <dt><samp class=”option”>interleaved_read</samp></dt> <dd> Interleave packets from multiple tracks at demuxer level. For badly interleaved files, this prevents playback issues caused by large gaps between packets in different tracks, as MOV/MP4 do not have packet placement requirements. However, this can cause excessive seeking on very badly interleaved files, due to seeking between tracks, so disabling it may prevent I/O issues, at the expense of playback. </dd> </dl>

 <h4 class=”subsection”>3.21.2 Audible AAX# <a class=”anchor hidden-xs”href=”#toc-

Audible-AAX" aria-hidden="true">TOC</h4>

Audible AAX files are encrypted M4B files, and they can be decrypted by specifying a 4 byte activation secret. <pre class="example-preformatted">ffmpeg -activation_bytes 1CEB00DA -i test.aax -vn -c:a copy output.mp4 </pre>

 <h3 class="section">3.22 mpegts# TOC</h3>

MPEG-2 transport stream demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">resync_size</samp></dt> <dd> Set size limit for looking up a new synchronization. Default value is 65536. </dd> <dt><samp class="option">skip_unknown_pmt</samp></dt> <dd> Skip PMTs for programs not defined in the PAT. Default value is 0. </dd> <dt><samp class="option">fix_teletext_pts</samp></dt> <dd> Override teletext packet PTS and DTS values with the timestamps calculated from the PCR of the first program which the teletext stream is part of and is not discarded. Default value is 1, set this option to 0 if you want your teletext packet PTS and DTS values untouched. </dd> <dt><samp class="option">ts_packetsize</samp></dt> <dd> Output option carrying the raw packet size in bytes. Show the detected raw packet size, cannot be set by the user. </dd> <dt><samp class="option">scan_all_pmts</samp></dt> <dd> Scan and combine all PMTs. The value is an integer with value from -1 to 1 (-1 means automatic setting, 1 means enabled, 0 means disabled). Default value is -1. </dd> <dt><samp class="option">merge_pmt_versions</samp></dt> <dd> Reuse existing streams when a PMT’s version is updated and elementary streams move to different PIDs. Default value is 0. </dd> <dt><samp class="option">max_packet_size</samp></dt> <dd> Set maximum size, in bytes, of packet emitted by the demuxer. Payloads above this size are split across multiple packets. Range is 184 to INT_MAX/2. Default is 204800 bytes. </dd> </dl>

 <h3 class="section">3.23 mpjpeg# TOC</h3>

MJPEG encapsulated in multi-part MIME demuxer. This demuxer allows reading of MJPEG, where each frame is represented as a part of multipart/x-mixed-replace stream. <dl class="table"> <dt><samp class="option">strict_mime_boundary</samp></dt> <dd> Default implementation applies a relaxed standard to multi-part MIME boundary detection, to prevent regression with numerous existing endpoints not generating

a proper MIME MJPEG stream. Turning this option on by setting it to 1 will result in a stricter check of the boundary value. </dd> </dl>

 <h3 class="section">3.24 rawvideo# TOC</h3>

Raw video demuxer. This demuxer allows one to read raw video data. Since there is no header specifying the assumed video parameters, the user must specify them in order to be able to decode the data correctly. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set input video frame rate. Default value is 25. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Set the input video pixel format. Default value is <code class="code">yuv420p</code>. </dd> <dt><samp class="option">stride</samp></dt> <dd> Set frame line size in bytes, only required if there are extra padding. For multiplane formats, <samp class="option">stride</samp> is a list of line size for each plane. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the input video size. This value must be specified explicitly. </dd> </dl>

For example to read a rawvideo file <samp class="file">input.raw</samp> with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">rgb24</code>, a video size of <code class="code">320x240</code>, and a frame rate of 10 images per second, use the command: <pre class="example-preformatted">ffplay -f rawvideo -pixel_format rgb24 -video_size 320x240 -framerate 10 input.raw </pre>

Read a rawvideo with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">yuv420p</code>, a video size of <code class="code">1080x1920</code>, has 8 bytes of padding in each line of Y plane, and 4 bytes of padding with UV plane, <pre class="example-preformatted">ffplay -f rawvideo -pixel_format yuv420p -video_size 1080x1920 -stride 1088,544,544 input.raw </pre>

 <h3 class="section">3.25 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. For more information on the format, see <a data-manual="ffmpeg-formats" href="./ffmpeg-formats.html#rcwtenc">(ffmpeg-formats)rcwtenc. This demuxer implements the specification as of March 2024, which has been stable and unchanged since April 2014.

  <h4 class="subsection">3.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Render CC to ASS using the built-in decoder: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin CC.ass </pre> Note that if your output appears to be empty, you may have to manually set the decoder’s <samp class="option">data_field</samp> option to pick the desired CC substream. Convert an RCWT backup to Scenarist (SCC) format: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin -c:s copy CC.scc </pre> Note that the SCC format does not support all of the possible CC extensions that can be stored in RCWT (such as EIA-708).

 <h3 class="section">3.26 sbg# TOC</h3>

SBaGen script demuxer. This demuxer reads the script language used by SBaGen http://uazu.net/sbagen/ to generate binaural beats sessions. A SBG script looks like that: <pre class="example-preformatted">-SE a: 300-2.5/3 440+4.5/0 b: 300-2.5/0 440+4.5/3 off: - NOW == a +0:07:00 == b +0:14:00 == a +0:21:00 == b +0:30:00 off </pre>

A SBG script can mix absolute and relative timestamps. If the script uses either only absolute timestamps (including the script start time) or only relative ones, then its layout is fixed, and the conversion is straightforward. On the other hand, if the script mixes both kind of timestamps, then the <var class="var">NOW</var> reference for relative timestamps will be taken from the current time of day at the time the script is read, and the script layout will be frozen according to that reference. That means that if the script is directly played, the actual times will match the absolute timestamps up to the sound controller’s clock accuracy, but if the user somehow pauses the playback or seeks, all times will be shifted accordingly. <h3 class="section">3.27 tedcaptions# TOC</h3>

JSON captions used for TED Talks. TED does not provide links to the captions, but they can be guessed from the page. The file <samp class="file">tools/bookmarklets.html</samp> from the FFmpeg source tree contains a bookmarklet to expose them. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">start_time</samp></dt> <dd> Set the

start time of the TED talk, in milliseconds. The default is 15000 (15s). It is used to sync the captions with the downloadable videos, because they include a 15s intro. </dd> </dl>

Example: convert the captions to a format most players understand: <pre class="example-preformatted">ffmpeg -i http://www.ted.com/talks/subtitles/id/1/lang/en talk1-en.srt </pre>

 <h3 class="section">3.28 vapoursynth# TOC</h3>

Vapoursynth wrapper. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class="code">-f vapoursynth</code> before the input <code class="code">-i yourscript.vpy</code>. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">max_script_size</samp></dt> <dd> The demuxer buffers the entire script into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of scripts that can be read. Default is 1 MiB. </dd> </dl>

 <h3 class="section">3.29 w64# TOC</h3>

Sony Wave64 Audio demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">max_size</samp></dt> <dd> See the same option for the wav demuxer. </dd> </dl>

 <h3 class="section">3.30 wav# TOC</h3>

RIFF Wave Audio demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">ignore_length <var class="var">bool</var></samp></dt> <dd> Ignore the size of the <code class="code">data</code> chunk and keep reading until the end of the file if set. May be useful to read broken or partial files where the header was not properly updated, but will misinterpret files with non-audio chunks after the <code class="code">data</code> chunk. Default is disabled. </dd> <dt><samp class="option">max_size</samp></dt> <dd> Specify the maximum packet size in bytes for the demuxed packets. By default this is set to 0, which means that a sensible value is chosen based on the input format. </dd> </dl>

 <h3 class="section">3.31 webp# TOC</h3>

Animated WebP demuxer. It accepts the following options: <dl class="table"> <dt><samp class="option">-min_delay <var class="var">int</var></samp></dt> <dd> Set the minimum valid delay between frames in milliseconds. Range is 0 to 60000. Default value is 10. </dd> <dt><samp class="option">-max_webp_delay <var class="var">int</var></samp></dt> <dd> Set the maximum valid delay between frames in milliseconds. Range is 0 to 16777215. Default value is 16777215 (over four hours), the maximum value allowed by the specification. </dd> <dt><samp class="option">-default_delay <var class="var">int</var></samp></dt> <dd> Set the default delay between frames in milliseconds. Range is 0 to 60000. Default value is 100. </dd> <dt><samp class="option">-ignore_loop <var class="var">bool</var></samp></dt> <dd> WebP files can contain information to loop a certain number of times (or infinitely). If <samp class="option">ignore_loop</samp> is set to true, then the loop setting from the input will be ignored and looping will not occur. If set to false, then looping will occur and will cycle the number of times according to the WebP. Default value is true. </dd> <dt><samp class="option">usebgcolor <var class="var">bool</var></samp></dt> <dd> WebP files contain a background color hint in the ANIM chunk, but the WebP specification says that viewer applications are not required to use it. If <samp class="option">usebgcolor</samp> is set to true, then the background color hint will be used, otherwise transparent black will be used for the background. Default value is false. </dd> </dl>

 <h2 class="chapter">4 Muxers# TOC</h2>

Muxers are configured elements in FFmpeg which allow writing multimedia streams to a particular type of file. When you configure your FFmpeg build, all the supported muxers are enabled by default. You can list all available muxers using the configure option <code class="code">--list-muxers</code>. You can disable all the muxers with the configure option <code class="code">--disable-muxers</code> and selectively enable / disable single muxers with the options <code class="code">--enable-muxer=<var class="var">MUXER</var></code> / <code class="code">--disable-muxer=<var class="var">MUXER</var></code>. The option <code class="code">-muxers</code> of the ff* tools will display the list of enabled muxers. Use <code class="code">-formats</code> to view a combined list of enabled demuxers and muxers. A description of some of the currently available muxers follows. <a

name=”Raw-muxers”> <h3 class=”section”>4.1 Raw muxers# TOC</h3>

This section covers raw muxers. They accept a single stream matching the designated codec. They do not store timestamps or metadata. The recognized extension is the same as the muxer name unless indicated otherwise.

It comprises the following muxers. The media type and the eventual extensions used to automatically selects the muxer from the output extensions are also shown.

<dl class=”table”>

<dt>‘<samp class=”samp”>ac3 audio</samp>’</dt>

<dd> Dolby Digital, also known as AC-3.

</dd>

<dt>‘<samp class=”samp”>adx audio</samp>’</dt>

<dd> CRI Middleware ADX audio.

This muxer will write out the total sample count near the start of the first packet when the output is seekable and the count can be stored in 32 bits.

</dd>

<dt>‘<samp class=”samp”>aptx audio</samp>’</dt>

<dd> aptX (Audio Processing Technology for Bluetooth)

</dd>

<dt>‘<samp class=”samp”>aptx_hd audio (aptxdh)</samp>’</dt>

<dd> aptX HD (Audio Processing Technology for Bluetooth) audio

</dd>

<dt>‘<samp class="samp">avs2 video (avs, avs2)</samp>’</dt>

<dd> AVS2-P2 (Audio Video Standard - Second generation - Part 2) /

IEEE 1857.4 video

</dd>

<dt>‘<samp class="samp">avs3 video (avs3)</samp>’</dt>

<dd> AVS3-P2 (Audio Video Standard - Third generation - Part 2) /

IEEE 1857.10 video

</dd>

<dt>‘<samp class="samp">cavsvideo video (cavs)</samp>’</dt>

<dd> Chinese AVS (Audio Video Standard - First generation)

</dd>

<dt>‘<samp class="samp">codec2raw audio</samp>’</dt>

<dd> Codec 2 audio.

No extension is registered so format name has to be supplied e.g. with

the ffmpeg CLI tool <code class="code">-f codec2raw</code>.

</dd>

<dt>‘<samp class="samp">data any</samp>’</dt>

<dd> Generic data muxer.

This muxer accepts a single stream with any codec of any type. The

input stream has to be selected using the <code class=”code”>-map</code> option with the

<code class=”command”>ffmpeg</code> CLI tool.

No extension is registered so format name has to be supplied e.g. with

the <code class=”command”>ffmpeg</code> CLI tool <code class=”code”>-f data</code>.

</dd>

<dt>‘<samp class=”samp”>dfpwm audio (dfpwm)</samp>’</dt>

<dd> Raw DFPWM1a (Dynamic Filter Pulse With Modulation) audio muxer.

</dd>

<dt>‘<samp class=”samp”>dirac video (drc, vc2)</samp>’</dt>

<dd> BBC Dirac video.

The Dirac Pro codec is a subset and is standardized as SMPTE VC-2.

</dd>

<dt>‘<samp class=”samp”>dnxhd video (dnxhd, dnxhr)</samp>’</dt>

<dd> Avid DNxHD video.

It is standardized as SMPTE VC-3. Accepts DNxHR streams.

</dd>

<dt>‘<samp class=”samp”>dts audio</samp>’</dt>

<dd> DTS Coherent Acoustics (DCA) audio

</dd>

<dt>‘<samp class="samp">eac3 audio</samp>’</dt>

<dd> Dolby Digital Plus, also known as Enhanced AC-3

</dd>

<dt>‘<samp class="samp">evc video
(evc)</samp>’</dt>

<dd> MPEG-5 Essential Video Coding (EVC) / EVC / MPEG-5 Part 1
EVC video

</dd>

<dt>‘<samp class="samp">g722 audio</samp>’</dt>

<dd> ITU-T G.722 audio

</dd>

<dt>‘<samp class="samp">g723_1 audio
(tco, rco)</samp>’</dt>

<dd> ITU-T G.723.1 audio

</dd>

<dt>‘<samp class="samp">g726 audio</samp>’</dt>

<dd> ITU-T G.726 big-endian ("left-justified") audio.

No extension is registered so format name has to be supplied e.g. with

the <code class="command">ffmpeg</code> CLI tool <code class="code">-
f g726</code>.

</dd>

<dt>‘<samp class="samp">g726le audio</samp>’</dt>

<dd> ITU-T G.726 little-endian ("right-justified") audio.

No extension is registered so format name has to be supplied e.g. with

the <code class=”command”>ffmpeg</code> CLI tool <code class=”code”>-f g726le</code>.

</dd>

<dt>‘<samp class=”samp”>gsm audio</samp>’</dt>

<dd> Global System for Mobile Communications audio

</dd>

<dt>‘<samp class=”samp”>h261 video</samp>’</dt>

<dd> ITU-T H.261 video

</dd>

<dt>‘<samp class=”samp”>h263 video</samp>’</dt>

<dd> ITU-T H.263 / H.263-1996, H.263+ / H.263-1998 / H.263 version 2 video

</dd>

<dt>‘<samp class=”samp”>h264 video (h264, 264)</samp>’</dt>

<dd> ITU-T H.264 / MPEG-4 Part 10 AVC video. Bitstream shall be converted

to Annex B syntax if it’s in length-prefixed mode.

</dd>

<dt>‘<samp class=”samp”>hevc video (hevc, h265, 265)</samp>’</dt>

<dd> ITU-T H.265 / MPEG-H Part 2 HEVC video. Bitstream shall be converted

to Annex B syntax if it’s in length-prefixed mode.

</dd>

<dt>‘<samp class=”samp”>m4v video</samp>’</dt>

<dd> MPEG-4 Part 2 video

</dd>

<dt>‘<samp class="samp">mjpeg video
(mjpg, mjpeg)</samp>’</dt>

<dd> Motion JPEG video

</dd>

<dt>‘<samp class="samp">mlp audio</samp>’</dt>

<dd> Meridian Lossless Packing, also known as Packed PCM

</dd>

<dt>‘<samp class="samp">mp2 audio
(mp2, m2a, mpa)</samp>’</dt>

<dd> MPEG-1 Audio Layer II audio

</dd>

<dt>‘<samp class="samp">mpeg1video video
(mpg, mpeg, m1v)</samp>’</dt>

<dd> MPEG-1 Part 2 video.

</dd>

<dt>‘<samp class="samp">mpeg2video video
(m2v)</samp>’</dt>

<dd> ITU-T H.262 / MPEG-2 Part 2 video

</dd>

<dt>‘<samp class="samp">obu video</samp>’</dt>

<dd> AV1 low overhead Open Bitstream Units muxer.

Temporal delimiter OBUs will be inserted in all temporal units of the

stream.

</dd>

<dt>‘<samp class="samp">rawvideo video (yuv, rgb)</samp>’</dt>

<dd> Raw uncompressed video.

</dd>

<dt>‘<samp class="samp">sbc audio (sbc, msbc)</samp>’</dt>

<dd> Bluetooth SIG low-complexity subband codec audio

</dd>

<dt>‘<samp class="samp">truehd audio (thd)</samp>’</dt>

<dd> Dolby TrueHD audio

</dd>

<dt>‘<samp class="samp">vc1 video</samp>’</dt>

<dd> SMPTE 421M / VC-1 video

</dd>

</dl>

 <h4 class="subsection">4.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Store raw video frames with the ‘<samp class="samp">rawvideo</samp>’ muxer using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f lavfi -i testsrc -t 10 -s hd1080p testsrc.yuv </pre>

Since the rawvideo muxer do not store the information related to size and format, this information must be provided when demuxing the file: <pre class="example-preformatted">ffplay -video_size 1920x1080 -pixel_format rgb24 -f rawvideo testsrc.rgb </pre>

 <h3 class="section">4.2 Raw PCM muxers# TOC</h3> This section covers raw PCM (Pulse-Code Modulation) audio muxers. They accept a single stream matching the designated codec. They do not store timestamps or metadata. The recognized extension is the same as the muxer name. It comprises the following muxers. The optional additional extension used to automatically select the muxer from the output extension is also shown in parentheses. <dl class="table"> <dt>‘<samp class="samp">alaw (al)</samp>’</dt> <dd> PCM A-law </dd> <dt>‘<samp class="samp">f32be</samp>’</dt> <dd> PCM 32-bit floating-point big-endian </dd> <dt>‘<samp class="samp">f32le</samp>’</dt> <dd> PCM 32-bit floating-point little-endian </dd> <dt>‘<samp class="samp">f64be</samp>’</dt> <dd> PCM 64-bit floating-point big-endian </dd> <dt>‘<samp class="samp">f64le</samp>’</dt> <dd> PCM 64-bit floating-point little-endian </dd> <dt>‘<samp class="samp">mulaw (ul)</samp>’</dt> <dd> PCM mu-law </dd> <dt>‘<samp class="samp">s16be</samp>’</dt> <dd> PCM signed 16-bit big-endian </dd> <dt>‘<samp class="samp">s16le</samp>’</dt> <dd> PCM signed 16-bit little-endian </dd> <dt>‘<samp class="samp">s24be</samp>’</dt> <dd> PCM signed 24-bit big-endian </dd> <dt>‘<samp class="samp">s24le</samp>’</dt> <dd> PCM signed 24-bit little-endian </dd> <dt>‘<samp class="samp">s32be</samp>’</dt> <dd> PCM signed 32-bit big-endian </dd> <dt>‘<samp class="samp">s32le</samp>’</dt> <dd> PCM signed 32-bit little-endian </dd> <dt>‘<samp class="samp">s8 (sb)</samp>’</dt> <dd> PCM signed 8-bit </dd> <dt>‘<samp class="samp">u16be</samp>’</dt> <dd> PCM unsigned 16-bit big-endian </dd> <dt>‘<samp class="samp">u16le</samp>’</dt> <dd> PCM unsigned 16-bit little-endian </dd> <dt>‘<samp class="samp">u24be</samp>’</dt> <dd> PCM unsigned 24-bit big-endian </dd> <dt>‘<samp class="samp">u24le</samp>’</dt> <dd> PCM unsigned 24-bit little-endian </dd> <dt>‘<samp class="samp">u32be</samp>’</dt> <dd> PCM unsigned 32-bit big-endian </dd> <dt>‘<samp class="samp">u32le</samp>’</dt> <dd> PCM unsigned 32-bit little-endian </dd> <dt>‘<samp class="samp">u8 (ub)</samp>’</dt> <dd> PCM unsigned 8-bit </dd> <dt>‘<samp class="samp">vidc</samp>’</dt> <dd> PCM Archimedes VIDC </dd> </dl>

 <h3 class="section">4.3 MPEG-1/MPEG-2 program stream muxers<a class="anchor hidden-xs" href="#MPEG_002d1_002fMPEG_002d2-

program-stream-muxers" aria-hidden="true"># TOC</h3>

This section covers formats belonging to the MPEG-1 and MPEG-2 Systems family. The MPEG-1 Systems format (also known as ISO/IEEC 11172-1 or MPEG-1 program stream) has been adopted for the format of media track stored in VCD (Video Compact Disc). The MPEG-2 Systems standard (also known as ISO/IEEC 13818-1) covers two containers formats, one known as transport stream and one known as program stream; only the latter is covered here. The MPEG-2 program stream format (also known as VOB due to the corresponding file extension) is an extension of MPEG-1 program stream: in addition to support different codecs for the audio and video streams, it also stores subtitles and navigation metadata. MPEG-2 program stream has been adopted for storing media streams in SVCD and DVD storage devices. This section comprises the following muxers. <dl class="table"> <dt>‘<samp class="samp">mpeg (mpg,mpeg)</samp>’</dt> <dd> MPEG-1 Systems / MPEG-1 program stream muxer. </dd> <dt>‘<samp class="samp">vcd</samp>’</dt> <dd> MPEG-1 Systems / MPEG-1 program stream (VCD) muxer. This muxer can be used to generate tracks in the format accepted by the VCD (Video Compact Disc) storage devices. It is the same as the ‘<samp class="samp">mpeg</samp>’ muxer with a few differences. </dd> <dt>‘<samp class="samp">vob</samp>’</dt> <dd> MPEG-2 program stream (VOB) muxer. </dd> <dt>‘<samp class="samp">dvd</samp>’</dt> <dd> MPEG-2 program stream (DVD VOB) muxer. This muxer can be used to generate tracks in the format accepted by the DVD (Digital Versatile Disc) storage devices. This is the same as the ‘<samp class="samp">vob</samp>’ muxer with a few differences. </dd> <dt>‘<samp class="samp">svcd (vob)</samp>’</dt> <dd> MPEG-2 program stream (SVCD VOB) muxer. This muxer can be used to generate tracks in the format accepted by the SVCD (Super Video Compact Disc) storage devices. This is the same as the ‘<samp class="samp">vob</samp>’ muxer with a few differences. </dd> </dl>

 <h4 class="subsection">4.3.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">muxrate <var class="var">rate</var></samp></dt> <dd> Set user-defined mux rate expressed as a number of bits/s. If not specified the automatically computed mux rate is employed. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">preload <var class="var">delay</var></samp></dt>

<dd> Set initial demux-decode delay in microseconds. Default value is <code class="code">500000</code>. </dd> </dl>

 <h3 class="section">4.4 MOV/MPEG-4/ISOMBFF muxers# TOC</h3>

This section covers formats belonging to the QuickTime / MOV family, including the MPEG-4 Part 14 format and ISO base media file format (ISOBMFF). These formats share a common structure based on the ISO base media file format (ISOBMFF). The MOV format was originally developed for use with Apple QuickTime. It was later used as the basis for the MPEG-4 Part 1 (later Part 14) format, also known as ISO/IEC 14496-1. That format was then generalized into ISOBMFF, also named MPEG-4 Part 12 format, ISO/IEC 14496-12, or ISO/IEC 15444-12. It comprises the following muxers. <dl class="table"> <dt>‘<samp class="samp">3gp</samp>’</dt> <dd> Third Generation Partnership Project (3GPP) format for 3G UMTS multimedia services </dd> <dt>‘<samp class="samp">3g2</samp>’</dt> <dd> Third Generation Partnership Project 2 (3GP2 or 3GPP2) format for 3G CDMA2000 multimedia services, similar to ‘<samp class="samp">3gp</samp>’ with extensions and limitations </dd> <dt>‘<samp class="samp">f4v</samp>’</dt> <dd> Adobe Flash Video format </dd> <dt>‘<samp class="samp">ipod</samp>’</dt> <dd> MPEG-4 audio file format, as MOV/MP4 but limited to contain only audio streams, typically played with the Apple ipod device </dd> <dt>‘<samp class="samp">ismv</samp>’</dt> <dd> Microsoft IIS (Internet Information Services) Smooth Streaming Audio/Video (ISMV or ISMA) format. This is based on MPEG-4 Part 14 format with a few incompatible variants, used to stream media files for the Microsoft IIS server. </dd> <dt>‘<samp class="samp">mov</samp>’</dt> <dd> QuickTime player format identified by the <code class="code">.mov</code> extension </dd> <dt>‘<samp class="samp">mp4</samp>’</dt> <dd> MP4 or MPEG-4 Part 14 format </dd> <dt>‘<samp class="samp">psp</samp>’</dt> <dd> PlayStation Portable MP4/MPEG-4 Part 14 format variant. This is based on MPEG-4 Part 14 format with a few incompatible variants, used to play files on PlayStation devices. </dd> </dl>

 <h4 class="subsection">4.4.1 Stereoscopic 3D# TOC</h4>

When the output stream has stereoscopic 3D side data (see the <samp

class=”option”>-stereo3d</samp> option), these muxers write: <ul class=”itemize mark-bullet”> MP4: Google spatial-media <code class=”code”>st3d</code> and Apple <code class=”code”>vexu</code> can be written for supported metadata with <samp class=”option”>-strict unofficial</samp>. Stricter compliance levels skip these boxes. Warnings are logged when stereoscopic or spherical side data is present, naming <code class=”code”>st3d</code> or <code class=”code”>sv3d</code>, respectively; there is no corresponding compliance warning for <code class=”code”>vexu</code>. Monoscopic layouts do not produce an <code class=”code”>eyes</code> box. MOV: Apple <code class=”code”>vexu</code> eyes only. <code class=”code”>st3d</code> is not written.

 <h4 class=”subsection”>4.4.2 Fragmentation# TOC</h4>

The ‘<samp class=”samp”>mov</samp>’, ‘<samp class=”samp”>mp4</samp>’, and ‘<samp class=”samp”>ismv</samp>’ muxers support fragmentation. Normally, a MOV/MP4 file has all the metadata about all packets stored in one location. This data is usually written at the end of the file, but it can be moved to the start for better playback by adding <code class=”code”>+faststart</code> to the <code class=”code”>-movflags</code>, or using the <code class=”command”>qt-faststart</code> tool). A fragmented file consists of a number of fragments, where packets and metadata about these packets are stored together. Writing a fragmented file has the advantage that the file is decodable even if the writing is interrupted (while a normal MOV/MP4 is undecodable if it is not properly finished), and it requires less memory when writing very long files (since writing normal MOV/MP4 files stores info about every single packet in memory until the file is closed). The downside is that it is less compatible with other applications. Fragmentation is enabled by setting one of the options that define how to cut the file into fragments: <dl class=”table”> <dt><samp class=”option”>frag_duration</samp></dt> <dt><samp class=”option”>frag_size</samp></dt> <dt><samp class=”option”>min_frag_duration</samp></dt> <dt><samp class=”option”>movflags +frag_keyframe</samp></dt> <dt><samp class=”option”>movflags +frag_custom</samp></dt> </dl>

If more than one condition is specified, fragments are cut when one of the specified conditions is fulfilled. The exception to this is the option <samp class=”option”>min_frag_duration</samp>, which has to be fulfilled for any of the other conditions to apply. <h4 class=”subsection”>4.4.3 Options# TOC</h4>

<dl class="table">

<dt><samp class="option">brand <var class="var">brand_string</var></samp></dt>

<dd> Override major brand.

</dd>

<dt><samp class="option">empty_hdlr_name <var class="var">bool</var></samp></dt>

<dd> Enable to skip writing the name inside a <code class="code">hdlr</code>
box.

Default is <code class="code">false</code>.

</dd>

<dt><samp class="option">encryption_key <var class="var">key</var></samp></dt>

<dd> set the media encryption key in hexadecimal format

</dd>

<dt><samp class="option">encryption_kid <var class="var">kid</var></samp></dt>

<dd> set the media encryption key identifier in hexadecimal format

</dd>

<dt><samp class="option">encryption_scheme <var class="var">scheme</var></samp></dt>

<dd> configure the encryption scheme, allowed values are ‘<samp
class="samp">none</samp>’, and

‘<samp class="samp">cenc-aes-ctr</samp>’

</dd>

<dt><samp class="option">frag_duration <var class="var">duration</var></samp></dt>

<dd> Create fragments that are <var class="var">duration</var> mi-
croseconds long.

</dd>

<dt><samp class="option">frag_interleave <var class="var">number</var></samp></dt>

<dd> Interleave samples within fragments (max number of consecutive

samples, lower is tighter interleaving, but with more overhead. It is

set to <code class="code">0</code> by default.

</dd>

<dt><samp class="option">frag_size <var class="var">size</var></samp></dt>

<dd> create fragments that contain up to <var class="var">size</var>
bytes of payload data

</dd>

<dt><samp class="option">iods_audio_profile <var class="var">profile</var></samp></dt>

<dd> specify iods number for the audio profile atom (from -1 to 255),

default is <code class="code">-1</code>

</dd>

<dt><samp class="option">iods_video_profile <var class="var">profile</var></samp></dt>

<dd> specify iods number for the video profile atom (from -1 to 255),

default is <code class="code">-1</code>

</dd>

<dt><samp class="option">ism_lookahead <var class="var">num_entries</var></samp></dt>

<dd> specify number of lookahead entries for ISM files (from 0 to 255),

default is <code class="code">0</code>

</dd>

<dt><samp class="option">min_frag_duration <var class="var">duration</var></samp></dt>

<dd> do not create fragments that are shorter than <var class="var">duration</var>
microseconds long

</dd>

<dt><samp class="option">moov_size <var class="var">bytes</var></samp></dt>

<dd> Reserves space for the moov atom at the beginning of the file instead
of placing the

moov atom at the end. If the space reserved is insufficient, muxing will fail.

</dd>

<dt><samp class=”option”>mov_gamma <var class=”var”>gamma</var></samp></dt>

<dd> specify gamma value for gama atom (as a decimal number from 0 to 10),

default is <code class=”code”>0.0</code>, must be set together with <code class=”code”>+ movflags</code>

</dd>

<dt><samp class=”option”>movflags <var class=”var”>flags</var></samp></dt>

<dd> Set various muxing switches. The following flags can be used:

<dl class=”table”>

<dt>‘<samp class=”samp”>cmaf</samp>’</dt>

<dd> write CMAF (Common Media Application Format) compatible fragmented

MP4 output

</dd>

<dt>‘<samp class=”samp”>dash</samp>’</dt>

<dd> write DASH (Dynamic Adaptive Streaming over HTTP) compatible fragmented

MP4 output

</dd>

<dt>‘<samp class=”samp”>default_base_moof</samp>’</dt>

<dd> Similarly to the ‘<samp class=”samp”>omit_tfhd_offset</samp>’ flag, this flag avoids

writing the absolute base_data_offset field in tfhd atoms, but does so

by using the new default-base-is-moof flag instead. This flag is new

from 14496-12:2012. This may make the fragments easier to parse in

certain circumstances (avoiding basing track fragment location

calculations on the implicit end of the previous track fragment).

</dd>

<dt>‘<samp class="samp">delay_moov</samp>’</dt>

<dd> delay writing the initial moov until the first fragment is cut, or

until the first fragment flush

</dd>

<dt>‘<samp class="samp">disable_chpl</samp>’</dt>

<dd> Disable Nero chapter markers (chpl atom). Normally, both Nero chapters

and a QuickTime chapter track are written to the file. With this

option set, only the QuickTime chapter track will be written. Nero

chapters can cause failures when the file is reprocessed with certain

tagging programs, like mp3Tag 2.61a and iTunes 11.3, most likely other

versions are affected as well.

</dd>

<dt>‘<samp class="samp">faststart</samp>’</dt>

<dd> Run a second pass moving the index (moov atom) to the beginning of the

file. This operation can take a while, and will not work in various

situations such as fragmented output, thus it is not enabled by

default.

</dd>

<dt>‘<samp class="samp">frag_custom</samp>’</dt>

<dd> Allow the caller to manually choose when to cut fragments, by calling

<code class="code">av_write_frame(ctx, NULL)</code> to write a fragment with the packets

written so far. (This is only useful with other applications

integrating libavformat, not from <code class="command">ffmpeg</code>.)

</dd>
<dt>‘<samp class="samp">frag_discont</samp>’</dt>
<dd> signal that the next fragment is discontinuous from earlier ones

</dd>
<dt>‘<samp class="samp">frag_every_frame</samp>’</dt>
<dd> fragment at every frame

</dd>
<dt>‘<samp class="samp">frag_keyframe</samp>’</dt>
<dd> start a new fragment at each video keyframe

</dd>
<dt>‘<samp class="samp">global_sidx</samp>’</dt>
<dd> write a global sidx index at the start of the file

</dd>
<dt>‘<samp class="samp">isml</samp>’</dt>
<dd> create a live smooth streaming feed (for pushing to a publishing point)

</dd>
<dt>‘<samp class="samp">negative_cts_offsets</samp>’</dt>
<dd> Enables utilization of version 1 of the CTTS box, in which the CTS offsets can
be negative. This enables the initial sample to have DTS/CTS of zero, and
reduces the need for edit lists for some cases such as video tracks with
B-frames. Additionally, eases conformance with the DASH-IF interoperability
guidelines.

This option is implicitly set when writing ‘<samp class="samp">ismv</samp>’ (Smooth

Streaming) files.

</dd>

<dt>‘<samp class="samp">omit_tfhd_offset</samp>’</dt>

<dd> Do not write any absolute base_data_offset in tfhd atoms. This avoids

tying fragments to absolute byte positions in the file/streams.

</dd>

<dt>‘<samp class="samp">prefer_icc</samp>’</dt>

<dd> If writing colr atom prioritise usage of ICC profile if it exists in

stream packet side data.

</dd>

<dt>‘<samp class="samp">rtphint</samp>’</dt>

<dd> add RTP hinting tracks to the output file

</dd>

<dt>‘<samp class="samp">separate_moof</samp>’</dt>

<dd> Write a separate moof (movie fragment) atom for each track. Normally,

packets for all tracks are written in a moof atom (which is slightly

more efficient), but with this option set, the muxer writes one

moof/mdat pair for each track, making it easier to separate tracks.

</dd>

<dt>‘<samp class="samp">skip_sidx</samp>’</dt>

<dd> Skip writing of sidx atom. When bitrate overhead due to sidx atom is

high, this option could be used for cases where sidx atom is not

mandatory. When the ‘<samp class="samp">global_sidx</samp>’ flag is enabled, this option

is ignored.

</dd>

<dt>‘<samp class="samp">skip_trailer</samp>’</dt>

<dd> skip writing the mfra/tfra/mfro trailer for fragmented files

</dd>

<dt>‘<samp class="samp">use_metadata_tags</samp>’</dt>

<dd> use mdta atom for metadata

</dd>

<dt>‘<samp class="samp">write_colr</samp>’</dt>

<dd> write colr atom even if the color info is unspecified. This flag is

experimental, may be renamed or changed, do not use from scripts.

</dd>

<dt>‘<samp class="samp">write_gama</samp>’</dt>

<dd> write deprecated gama atom

</dd>

<dt>‘<samp class="samp">hybrid_fragmented</samp>’</dt>

<dd> For recoverability - write the output file as a fragmented file.

This allows the intermediate file to be read while being written

(in particular, if the writing process is aborted uncleanly). When

writing is finished, the file is converted to a regular, non-fragmented

file, which is more compatible and allows easier and quicker seeking.

If writing is aborted, the intermediate file can manually be

remuxed to get a regular, non-fragmented file of what had been

written into the unfinished file.

</dd>

</dl>

</dd> <dt><samp class="option">movie_timescale <var class="var">scale</var></samp></dt> <dd> Set the timescale written in the movie header box (<code class="code">mvhd</code>). Range is 1 to INT_MAX. Default is <code class="code">1000</code>. </dd> <dt><samp class="option">rtpflags <var class="var">flags</var></samp></dt> <dd> Add RTP hinting tracks to the output file. The following flags can be used: <dl class="table"> <dt>‘<samp class="samp">h264_mode0</samp>’</dt> <dd> use mode 0 for H.264 in RTP </dd> <dt>‘<samp class="samp">latm</samp>’</dt> <dd> use MP4A-LATM packetization instead of MPEG4-GENERIC for AAC </dd> <dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> use RFC 2190 packetization instead of RFC 4629 for H.263 </dd> <dt>‘<samp class="samp">send_bye</samp>’</dt> <dd> send RTCP BYE packets when finishing </dd> <dt>‘<samp class="samp">skip_rtcp</samp>’</dt> <dd> do not send RTCP sender reports </dd> </dl>

</dd> <dt><samp class="option">skip_iods <var class="var">bool</var></samp></dt> <dd> skip writing iods atom (default value is <code class="code">true</code>) </dd> <dt><samp class="option">use_editlist <var class="var">bool</var></samp></dt> <dd> use edit list (default value is <code class="code">auto</code>) </dd> <dt><samp class="option">use_stream_ids_as_track_ids <var class="var">bool</var></samp></dt> <dd> use stream ids as track ids (default value is <code class="code">false</code>) </dd> <dt><samp class="option">video_track_timescale <var class="var">scale</var></samp></dt> <dd> Set the timescale used for video tracks. Range is <code class="code">0</code> to INT_MAX. If set to <code class="code">0</code>, the timescale is automatically set based on the native stream time base. Default is <code class="code">0</code>. </dd> <dt><samp class="option">write_btrt <var class="var">bool</var></samp></dt> <dd> Force or disable writing bitrate box inside stsd box of a track. The box contains decoding buffer size (in bytes), maximum bitrate and average bitrate for the track. The box will be skipped if none of these values can be computed. Default is <code class="code">-1</code> or <code class="code">auto</code>, which will write the box only in MP4 mode. </dd> <dt><samp class="option">write_prft <var class="var">option</var></samp></dt> <dd> Write producer time reference box (PRFT) with a specified time source for the NTP field in the PRFT box. Set value as ‘<samp class="samp">wallclock</samp>’ to specify timesource as wallclock time and ‘<samp class="samp">pts</samp>’ to specify timesource as input packets’ PTS values. </dd> <dt><samp class="option">write_tmcd <var class="var">bool</var></samp></dt>

<dd> Specify <code class="code">on</code> to force writing a timecode track, <code class="code">off</code> to disable it and <code class="code">auto</code> to write a timecode track only for mov and mp4 output (default). Setting value to ‘<samp class="samp">pts</samp>’ is applicable only for a live encoding use case, where PTS values are set as as wallclock time at the source. For example, an encoding use case with decklink capture source where <samp class="option">video_pts</samp> and <samp class="option">audio_pts</samp> are set to ‘<samp class="samp">abs_wallclock</samp>’. </dd> </dl>

 <h4 class="subsection">4.4.4 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Push Smooth Streaming content in real time to a publishing point on IIS with the ‘<samp class="samp">ismv</samp>’ muxer using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -re <var class="var"><normal input/transcoding options></var> -movflags isml+frag_keyframe -f ismv http://server/publishingpoint.isml/Streams(Encoder1) </pre>

 <h3 class="section">4.5 a64# TOC</h3> A64 Commodore 64 video muxer. This muxer accepts a single <code class="code">a64_multi</code> or <code class="code">a64_multi5</code> codec video stream. <h3 class="section">4.6 ac4# TOC</h3> Raw AC-4 audio muxer. This muxer accepts a single <code class="code">ac4</code> audio stream. <h4 class="subsection">4.6.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">write_crc <var class="var">bool</var></samp></dt> <dd> when enabled, write a CRC checksum for each packet to the output, default is <code class="code">false</code> </dd> </dl>

 <h3 class="section">4.7 adts# TOC</h3> Audio Data Transport Stream muxer. It accepts a single AAC

stream. <h4 class="subsection">4.7.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">write_id3v2 <var class="var">bool</var></samp></dt> <dd> Enable to write ID3v2.4 tags at the start of the stream. Default is disabled. </dd> <dt><samp class="option">write_apetag <var class="var">bool</var></samp></dt> <dd> Enable to write APE tags at the end of the stream. Default is disabled. </dd> <dt><samp class="option">write_mpeg2 <var class="var">bool</var></samp></dt> <dd> Enable to set MPEG version bit in the ADTS frame header to 1 which indicates MPEG-2. Default is 0, which indicates MPEG-4. </dd> </dl>

 <h3 class="section">4.8 aea# TOC</h3> MD STUDIO audio muxer. This muxer accepts a single ATRAC1 audio stream with either one or two channels and a sample rate of 44100Hz. As AEA supports storing the track title, this muxer will also write the title from stream’s metadata to the container. <h3 class="section">4.9 aiff# TOC</h3> Audio Interchange File Format muxer. <h4 class="subsection">4.9.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">write_id3v2 <var class="var">bool</var></samp></dt> <dd> Enable ID3v2 tags writing when set to 1. Default is 0 (disabled). </dd> <dt><samp class="option">id3v2_version <var class="var">bool</var></samp></dt> <dd> Select ID3v2 version to write. Currently only version 3 and 4 (aka. ID3v2.3 and ID3v2.4) are supported. The default is version 4. </dd> </dl>

 <h3 class="section">4.10 alp# TOC</h3> High Voltage Software’s Lego Racers game audio muxer. It accepts a single ADPCM_IMA_ALP stream with no more than 2 channels and a sample rate not greater than 44100 Hz. Extensions: <code class="code">tun</code>, <code class="code">pcm</code> <h4 class="subsection">4.10.1 Options<a class="anchor hidden-xs" href="#Options-10" aria-

hidden=”true”># TOC</h4> <dl class=”table”> <dt><samp class=”option”>type <var class=”var”>type</var></samp></dt> <dd> Set file type. <var class=”var”>type</var> accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tun</samp>’</dt> <dd> Set file type as music. Must have a sample rate of 22050 Hz. </dd> <dt>‘<samp class=”samp”>pcm</samp>’</dt> <dd> Set file type as sfx. </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Set file type as per output file extension. <code class=”code”>.pcm</code> results in type <code class=”code”>pcm</code> else type <code class=”code”>tun</code> is set. <var class=”var”>(default)</var> </dd> </dl> </dd> </dl>

 <h3 class=”section”>4.11 amr# TOC</h3> 3GPP AMR (Adaptive Multi-Rate) audio muxer. It accepts a single audio stream containing an AMR NB stream. <h3 class=”section”>4.12 amv# TOC</h3> AMV (Actions Media Video) format muxer. <h3 class=”section”>4.13 apm# TOC</h3> Ubisoft Rayman 2 APM audio muxer. It accepts a single ADPCM IMA APM audio stream. <h3 class=”section”>4.14 apng# TOC</h3> Animated Portable Network Graphics muxer. It accepts a single APNG video stream. <h4 class=”subsection”>4.14.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>final_delay <var class=”var”>delay</var></samp></dt> <dd> Force a delay expressed in seconds after the last frame of each repetition. Default value is <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>plays <var class=”var”>repetitions</var></samp></dt> <dd> specify how many times to play the content, <code class=”code”>0</code> causes an infinite loop, with <code class=”code”>1</code> there is no loop </dd> </dl>

 <h4 class=”subsection”>4.14.2 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-7” aria-hidden=”true”>TOC</h4> <ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> to generate an APNG output with 2 repetitions, and with a delay of half a second after the first repetition: <pre class=”example-preformatted”>ffmpeg -i INPUT -final_delay 0.5 -plays 2 out.apng </pre>

 <h3 class=”section”>4.15 argo_asf# TOC</h3> Argonaut Games ASF audio muxer. It accepts a single ADPCM audio stream. <h4 class=”subsection”>4.15.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>version_major <var class=”var”>version</var></samp></dt> <dd> override file major version, specified as an integer, default value is <code class=”code”>2</code> </dd> <dt><samp class=”option”>version_minor <var class=”var”>version</var></samp></dt> <dd> override file minor version, specified as an integer, default value is <code class=”code”>1</code> </dd> <dt><samp class=”option”>name <var class=”var”>name</var></samp></dt> <dd> Embed file name into file, if not specified use the output file name. The name is truncated to 8 characters. </dd> </dl>

 <h3 class=”section”>4.16 argo_cvg# TOC</h3> Argonaut Games CVG audio muxer. It accepts a single one-channel ADPCM 22050Hz audio stream. The <samp class=”option”>loop</samp> and <samp class=”option”>reverb</samp> options set the corresponding flags in the header which can be later retrieved to process the audio stream accordingly. <h4 class=”subsection”>4.16.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>skip_rate_check <var class=”var”>bool</var></samp></dt> <dd> skip sample rate check (default is <code class=”code”>false</code>) </dd> <dt><samp class=”option”>loop <var class=”var”>bool</var></samp></dt> <dd> set loop flag (default is <code class=”code”>false</code>) </dd> <dt><samp class=”option”>reverb <var class=”var”>boolean</var></samp></dt> <dd> set reverb flag (default is <code class=”code”>true</code>) </dd> </dl>

<h3 class="section">4.17 asf, asf_stream# TOC</h3> Advanced / Active Systems (or Streaming) Format audio muxer. The ‘<samp class="samp">asf_stream</samp>’ variant should be selected for streaming. Note that Windows Media Audio (wma) and Windows Media Video (wmv) use this muxer too. <h4 class="subsection">4.17.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">packet_size <var class="var">size</var></samp></dt> <dd> Set the muxer packet size as a number of bytes. By tuning this setting you may reduce data fragmentation or muxer overhead depending on your source. Default value is <code class="code">3200</code>, minimum is <code class="code">100</code>, maximum is <code class="code">64Ki</code>. </dd> </dl>

 <h3 class="section">4.18 ass# TOC</h3> ASS/SSA (SubStation Alpha) subtitles muxer. It accepts a single ASS subtitles stream. <h4 class="subsection">4.18.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">ignore_readorder <var class="var">bool</var></samp></dt> <dd> Write dialogue events immediately, even if they are out-of-order, default is <code class="code">false</code>, otherwise they are cached until the expected time event is found. </dd> </dl>

 <h3 class="section">4.19 ast# TOC</h3> AST (Audio Stream) muxer. This format is used to play audio on some Nintendo Wii games. It accepts a single audio stream. The <samp class="option">loopstart</samp> and <samp class="option">loopend</samp> options can be used to define a section of the file to loop for players honoring such options. <h4 class="subsection">4.19.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">loopstart <var class="var">start</var></samp></dt> <dd> Specify loop start position expressesd in milliseconds, from <code

class=”code”>-1</code> to <code class=”code”>INT_MAX</code>, in case <code class=”code”>-1</code> is set then no loop is specified (default -1) and the <samp class=”option”>loopend</samp> value is ignored. </dd> <dt><samp class=”option”>loopend <var class=”var”>end</var></samp></dt> <dd> Specify loop end position expressed in milliseconds, from <code class=”code”>0</code> to <code class=”code”>INT_MAX</code>, default is <code class=”code”>0</code>, in case <code class=”code”>0</code> is set it assumes the total stream duration. </dd> </dl>

 <h3 class=”section”>4.20 astc# TOC</h3> ASTC (Adaptive Scalable Texture Compression) muxer. This muxer accepts a single ‘<samp class=”samp”>libastcenc</samp>’ video stream and writes one image: the 16-byte <samp class=”file”>.astc</samp> header, taken from the encoder’s extradata, followed by the raw ASTC bitstream. The block footprint and the image dimensions come from the same extradata, and a second packet is rejected. Use the <samp class=”file”>.ktx</samp> container for a texture that also needs its color space or row order recorded. <h3 class=”section”>4.21 au# TOC</h3> SUN AU audio muxer. It accepts a single audio stream. <h3 class=”section”>4.22 avi# TOC</h3> Audio Video Interleaved muxer. AVI is a proprietary format developed by Microsoft, and later formally specified through the Open DML specification. Because of differences in players implementations, it might be required to set some options to make sure that the generated output can be correctly played by the target player. <h4 class=”subsection”>4.22.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>flipped_raw_rgb <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>true</code>, store positive height for raw RGB bitmaps, which indicates bitmap is stored bottom-up. Note that this option does not flip the bitmap which has to be done manually beforehand, e.g. by using the ‘<samp class=”samp”>vflip</samp>’ filter. Default is <code class=”code”>false</code> and indicates bitmap is stored top down. </dd> <dt><samp class=”option”>reserve_index_space <var class=”var”>size</var></samp></dt> <dd> Reserve the specified

amount of bytes for the OpenDML master index of each stream within the file header. By default additional master indexes are embedded within the data packets if there is no space left in the first master index and are linked together as a chain of indexes. This index structure can cause problems for some use cases, e.g. third-party software strictly relying on the OpenDML index specification or when file seeking is slow. Reserving enough index space in the file header avoids these problems. The required index space depends on the output file size and should be about 16 bytes per gigabyte. When this option is omitted or set to zero the necessary index space is guessed. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">write_channel_mask <var class="var">bool</var></samp></dt> <dd> Write the channel layout mask into the audio stream header. This option is enabled by default. Disabling the channel mask can be useful in specific scenarios, e.g. when merging multiple audio streams into one for compatibility with software that only supports a single audio stream in AVI (see <a data-manual="ffmpeg-filters" href="./ffmpeg-filters.html#amerge">the "amerge" section in the ffmpeg-filters manual). </dd> </dl>

 <h3 class="section">4.23 avif# TOC</h3>
AV1 (Alliance for Open Media Video codec 1) image format muxer.
This muxers stores images encoded using the AV1 codec.
It accepts one or two video streams. In case two video streams are provided, the second one shall contain a single plane storing the alpha mask. In case more than one image is provided, the generated output is considered an animated AVIF and the number of loops can be specified with the <samp class="option">loop</samp> option. This is based on the specification by Alliance for Open Media at url https://aomediacodec.github.io/av1-avif. <h4 class="subsection">4.23.1 Options# TOC</h4>
<dl class="table"> <dt><samp class="option">loop <var class="var">count</var></samp></dt>
<dd> number of times to loop an animated AVIF, <code class="code">0</code> specify an infinite loop, default is <code class="code">0</code> </dd>
<dt><samp class="option">movie_timescale <var class="var">timescale</var></samp></dt>
<dd> Set the timescale written in the movie header box (<code class="code">mvhd</code>). Range is 1 to INT_MAX. Default is <code class="code">1000</code>. </dd> </dl>

 <h3 class="section">4.24 avm2# TOC</h3>
ShockWave Flash (SWF) / ActionScript Virtual Machine 2 (AVM2)

format muxer. It accepts one audio stream, one video stream, or both. <h3 class="section">4.25 bit# TOC</h3> G.729 (.bit) file format muxer. It accepts a single G.729 audio stream. <h3 class="section">4.26 caf# TOC</h3> Apple CAF (Core Audio Format) muxer. It accepts a single audio stream. <h3 class="section">4.27 codec2# TOC</h3> Codec2 audio audio muxer. It accepts a single codec2 audio stream. <h3 class="section">4.28 chromaprint# TOC</h3> Chromaprint fingerprinter muxers. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-chromaprint</code>. This muxer feeds audio data to the Chromaprint library, which generates a fingerprint for the provided audio data. See: https://acoustid.org/chromaprint It takes a single signed native-endian 16-bit raw audio stream of at most 2 channels. <h4 class="subsection">4.28.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">algorithm <var class="var">version</var></samp></dt> <dd> Select version of algorithm to fingerprint with. Range is <code class="code">0</code> to <code class="code">4</code>. Version <code class="code">3</code> enables silence detection. Default is <code class="code">1</code>. </dd> <dt><samp class="option">fp_format <var class="var">format</var></samp></dt> <dd> Format to output the fingerprint as. Accepts the following options: <dl class="table"> <dt>‘<samp class="samp">base64</samp>’</dt> <dd> Base64 compressed fingerprint (default) </dd> <dt>‘<samp class="samp">compressed</samp>’</dt> <dd> Binary compressed fingerprint </dd> <dt>‘<samp class="samp">raw</samp>’</dt> <dd> Binary raw fingerprint </dd> </dl>

</dd> <dt><samp class="option">silence_threshold <var class="var">threshold</var></samp></dt> <dd> Threshold for detecting silence. Range is from <code class="code">-1</code> to <code class="code">32767</code>, where <code class="code">-

1</code> disables silence detection. Silence detection can only be used with version <code class=”code”>3</code> of the algorithm. Silence detection must be disabled for use with the AcoustID service. Default is <code class=”code”>-1</code>. </dd> </dl>

 <h3 class=”section”>4.29 crc# TOC</h3> CRC (Cyclic Redundancy Check) muxer. This muxer computes and prints the Adler-32 CRC of all the input audio and video frames. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the CRC. The output of the muxer consists of a single line of the form: CRC=0x<var class=”var”>CRC</var>, where <var class=”var”>CRC</var> is a hexadecimal number 0-padded to 8 digits containing the CRC for all the decoded input frames. See also the framecrc muxer. <h4 class=”subsection”>4.29.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> to compute the CRC of the input, and store it in the file <samp class=”file”>out.crc</samp>: <pre class=”example-preformatted”>ffmpeg -i INPUT -f crc out.crc </pre>

Use <code class=”command”>ffmpeg</code> to print the CRC to stdout with the command: <pre class=”example-preformatted”>ffmpeg -i INPUT -f crc - </pre>

You can select the output format of each frame with <code class=”command”>ffmpeg</code> by specifying the audio and video codec and format. For example, to compute the CRC of the input audio converted to PCM unsigned 8-bit and the input video converted to MPEG-2 video, use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:a pcm_u8 -c:v mpeg2video -f crc - </pre>

 <h3 class=”section”>4.30 dash# TOC</h3> Dynamic Adaptive Streaming over HTTP (DASH) muxer. This muxer creates segments and manifest files according to the MPEG-DASH standard ISO/IEC 23009-1:2014 and following standard updates. For more information see: <ul class=”itemize mark-bullet”> ISO DASH Specification: <a class=”url” href=”http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_I
1_2014.zip”>http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_IEC_23009-

1_2014.zip WebM DASH Specification: https://sites.google.com/a/webmproject.org/wiki/adaptive-streaming/webm-dash-specification

This muxer creates an MPD (Media Presentation Description) manifest file and segment files for each stream. Segment files are placed in the same directory of the MPD manifest file. The segment filename might contain pre-defined identifiers used in the manifest <code class="code">SegmentTemplate</code> section as defined in section 5.3.9.4.4 of the standard. Available identifiers are <code class="code">$RepresentationID$</code>, <code class="code">$Number$</code>, <code class="code">$Bandwidth$</code>, and <code class="code">$Time$</code>. In addition to the standard identifiers, an ffmpeg-specific <code class="code">ext</code> identifier is also supported. When specified, <code class="command">ffmpeg</code> will replace <code class="code">ext</code> in the file name with muxing format’s extensions such as <code class="code">mp4</code>, <code class="code">webm</code> etc. <h4 class="subsection">4.30.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">adaptation_sets <var class="var">adaptation_sets</var></samp></dt> <dd> Assign streams to adaptation sets, specified in the MPD manifest <code class="code">AdaptationSets</code> section. An adaptation set contains a set of one or more streams accessed as a single subset, e.g. corresponding streams encoded at different size selectable by the user depending on the available bandwidth, or to different audio streams with a different language. Each adaptation set is specified with the syntax: <pre class="example-preformatted">id=<var class="var">index</var>,streams=<var class="var">streams</var> </pre>

where <var class="var">index</var> must be a numerical index, and <var class="var">streams</var> is a sequence of <code class="code">,</code>-separated stream indices. Multiple adaptation sets can be specified, separated by spaces. To map all video (or audio) streams to an adaptation set, <code class="code">v</code> (or <code class="code">a</code>) can be used as stream identifier instead of IDs. When no assignment is defined, this defaults to an adaptation set for each stream. The following optional fields can also be specified: <dl class="table"> <dt><samp class="option">descriptor</samp></dt> <dd> Define the descriptor as defined by ISO/IEC 23009-1:2014/Amd.2:2015. For example: <pre class="example-preformatted"><SupplementalProperty schemeIdUri=\"urn:mpeg:dash:srd:2014\" value=\"0,0,0,1,1,2,2\"/> </pre>

The descriptor string should be a self-closing XML tag. </dd>
<dt><samp class="option">frag_duration</samp></dt> <dd> Override
the global fragment duration specified with the <samp class="option">frag_duration</samp>
option. </dd> <dt><samp class="option">frag_type</samp></dt>
<dd> Override the global fragment type specified with the <samp
class="option">frag_type</samp> option. </dd> <dt><samp
class="option">seg_duration</samp></dt> <dd> Override the global
segment duration specified with the <samp class="option">seg_duration</samp>
option. </dd> <dt><samp class="option">trick_id</samp></dt>
<dd> Mark an adaptation set as containing streams meant to be used for
Trick Mode for the referenced adaptation set. </dd> </dl>

A few examples of possible values for the <samp class="option">adaptation_sets</samp>
option follow: <pre class="example-preformatted">id=0,seg_duration=2,frag_
id=1,seg_duration=2,frag_type=none,streams=a </pre>

 <pre class="example-preformatted">id=0,seg_duration=2,frag_type=none,streams=
id=1,seg_duration=10,frag_type=none,trick_id=0,streams=1 </pre>

</dd> <dt><samp class="option">dash_segment_type <var class="var">type</var></samp></dt>
<dd> Set DASH segment files type. Possible values: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> The dash segment files format will be selected based on the
stream codec. This is the default mode. </dd> <dt>‘<samp
class="samp">mp4</samp>’</dt> <dd> the dash segment
files will be in ISOBMFF/MP4 format </dd> <dt>‘<samp
class="samp">webm</samp>’</dt> <dd> the dash segment
files will be in WebM format </dd> </dl>

</dd> <dt><samp class="option">extra_window_size <var class="var">size</var></samp></dt>
<dd> Set the maximum number of segments kept outside of the manifest be-
fore removing from disk. </dd> <dt><samp class="option">format_options
<var class="var">options_list</var></samp></dt> <dd> Set con-
tainer format (mp4/webm) options using a <code class="code">:</code>-
separated list of key=value parameters. Values containing <code class="code">:</code>
special characters must be escaped. </dd> <dt><samp class="option">frag_duration
<var class="var">duration</var></samp></dt> <dd> Set the
length in seconds of fragments within segments, fractional value can
also be set. </dd> <dt><samp class="option">frag_type <var
class="var">type</var></samp></dt> <dd> Set the type of inter-
val for fragmentation. Possible values: <dl class="table">
<dt>‘<samp class="samp">auto</samp>’</dt> <dd> set
one fragment per segment </dd> <dt>‘<samp class="samp">every_frame</samp>’</
<dd> fragment at every frame </dd> <dt>‘<samp
class="samp">duration</samp>’</dt> <dd> fragment at spe-
cific time intervals </dd> <dt>‘<samp class="samp">pframes</samp>’</dt>
<dd> fragment at keyframes and following P-Frame reordering (Video
only, experimental) </dd> </dl>

</dd>

<dt><samp class="option">global_sidx <var class="var">bool</var></samp></dt>

<dd> Write global <code class="code">SIDX</code> atom. Applicable only for single file, mp4

output, non-streaming mode.

</dd>

<dt><samp class="option">hls_master_name <var class="var">file_name</var></samp></dt>

<dd> HLS master playlist name. Default is <samp class="file">master.m3u8</samp>.

</dd>

<dt><samp class="option">hls_playlist <var class="var">bool</var></samp></dt>

<dd> Generate HLS playlist files. The master playlist is generated with

filename specified by the <samp class="option">hls_master_name</samp> option. One media

playlist file is generated for each stream with filenames

<samp class="file">media_0.m3u8</samp>, <samp class="file">media_1.m3u8</samp>, etc.

</dd>

<dt><samp class="option">http_opts <var class="var">http_opts</var></samp></dt>

<dd> Specify a list of <code class="code">:</code>-separated key=value options to pass to the

underlying HTTP protocol. Applicable only for HTTP output.

</dd>

<dt><samp class="option">http_persistent <var class="var">bool</var></samp></dt>

<dd> Use persistent HTTP connections. Applicable only for HTTP output.

</dd>

<dt><samp class="option">http_user_agent <var class="var">user_agent</var></samp></dt>

<dd> Override User-Agent field in HTTP header. Applicable only for HTTP

output.

</dd>

<dt><samp class="option">ignore_io_errors <var class="var">bool</var></samp></dt>

<dd> Ignore IO errors during open and write. Useful for long-duration runs

with network output. This is disabled by default.

</dd>

<dt><samp class="option">index_correction <var class="var">bool</var></samp></dt>

<dd> Enable or disable segment index correction logic. Applicable only when

<samp class="option">use_template</samp> is enabled and <samp class="option">use_timeline</samp> is

disabled. This is disabled by default.

When enabled, the logic monitors the flow of segment indexes. If a

streams’s segment index value is not at the expected real time

position, then the logic corrects that index value.

Typically this logic is needed in live streaming use cases. The

network bandwidth fluctuations are common during long run

streaming. Each fluctuation can cause the segment indexes fall behind

the expected real time position.

</dd>

<dt><samp class="option">init_seg_name <var class="var">init_name</var></samp></dt>

<dd> DASH-templated name to use for the initialization segment. Default is

<code class="code">init-stream$RepresentationID$.ext</code>. <code class="code">ext</code> is replaced

with the file name extension specific for the segment format.

</dd>

<dt><samp class=”option”>ldash <var class=”var”>bool</var></samp></dt>

<dd> Enable Low-latency Dash by constraining the presence and values of

some elements. This is disabled by default.

</dd>

<dt><samp class=”option”>lhls <var class=”var”>bool</var></samp></dt>

<dd> Enable Low-latency HLS (LHLS). Add <code class=”code”>#EXT-X-PREFETCH</code> tag with

current segment’s URI. hls.js player folks are trying to standardize

an open LHLS spec. The draft spec is available at

https://github.com/video-dev/hlsjs-rfcs/blob/lhls-spec/proposals/0001-lhls.md.

This option tries to comply with the above open spec. It enables

<samp class=”option”>streaming</samp> and <samp class=”option”>hls_playlist</samp> options automatically.

This is an experimental feature.

Note: This is not Apple’s version LHLS. See

https://datatracker.ietf.org/doc/html/draft-pantos-hls-rfc8216bis

</dd>

<dt><samp class=”option”>master_m3u8_publish_rate <var class=”var”>segment_intervals_count</var>

<dd> Publish master playlist repeatedly every after specified number of

segment intervals.

</dd>

<dt><samp class="option">max_playback_rate <var class="var">rate</var></samp></dt>

<dd> Set the maximum playback rate indicated as appropriate for the

purposes of automatically adjusting playback latency and buffer

occupancy during normal playback by clients.

</dd>

<dt><samp class="option">media_seg_name <var class="var">segment_name</var></samp></dt>

<dd> DASH-templated name to use for the media segments. Default is

<code class="code">chunk-stream$RepresentationID$-$Number%05d$.ext</code>.
<code class="code">ext</code>

is replaced with the file name extension specific for the segment

format.

</dd>

<dt><samp class="option">method <var class="var">method</var></samp></dt>

<dd> Use the given HTTP method to create output files. Generally set to
<code class="code">PUT</code>

or <code class="code">POST</code>.

</dd>

<dt><samp class="option">min_playback_rate <var class="var">rate</var></samp></dt>

<dd> Set the minimum playback rate indicated as appropriate for the

purposes of automatically adjusting playback latency and buffer

occupancy during normal playback by clients.

</dd>

<dt><samp class="option">mpd_profile <var class="var">flags</var></samp></dt>

<dd> Set one or more MPD manifest profiles.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">dash</samp>’</dt>
<dd> MPEG-DASH ISO Base media file format live profile
</dd>
<dt>‘<samp class="samp">dvb_dash</samp>’</dt>
<dd> DVB-DASH profile
</dd>
</dl>
Default value is <code class="code">dash</code>.

</dd>
<dt><samp class="option">remove_at_exit <var class="var">bool</var></samp></dt>
<dd> Enable or disable removal of all segments when finished. This is
disabled by default.

</dd>
<dt><samp class="option">seg_duration <var class="var">duration</var></samp></dt>
<dd> Set the segment length in seconds (fractional value can be set). The
value is treated as average segment duration when the
<samp class="option">use_template</samp> option is enabled and the
<samp class="option">use_timeline</samp>
option is disabled and as minimum segment duration for all the other
use cases.

Default value is <code class="code">5</code>.

</dd>
<dt><samp class="option">single_file <var class="var">bool</var></samp></dt>
<dd> Enable or disable storing all segments in one file, accessed using
byte ranges. This is disabled by default.

The name of the single file can be specified with the

<samp class="option">single_file_name</samp> option, if not specified assume the basename

of the manifest file with the output format extension.

</dd>

<dt><samp class="option">single_file_name <var class="var">file_name</var></samp></dt>

<dd> DASH-templated name to use for the manifest <code class="code">baseURL</code>

element. Imply that the <samp class="option">single_file</samp> option is set to

<var class="var">true</var>. In the template, <code class="code">ext</code> is replaced with the file

name extension specific for the segment format.

</dd>

<dt><samp class="option">availability_start_time_ms <var class="var">milliseconds</var></samp></

<dd> Set the MPD <code class="code">availabilityStartTime</code> attribute as Unix epoch time in

milliseconds. If set to <code class="code">0</code>, the wall clock is used when the first

packet is written. This is useful for live outputs that need a stable

availability start time across restarts.

</dd>

<dt><samp class="option">streaming <var class="var">bool</var></samp></dt>

<dd> Enable or disable chunk streaming mode of output. In chunk streaming

mode, each frame will be a <code class="code">moof</code> fragment which forms a

chunk. This is disabled by default.

</dd>

<dt><samp class="option">suggested_presentation_delay <var class="var">duration</var></samp></d

<dd> Set the MPD <code class="code">suggestedPresentationDelay</code>
attribute. If set to

<code class="code">0</code>, the last segment duration is used.

</dd>

<dt><samp class="option">target_latency <var class="var">target_latency</var></samp></dt>

<dd> Set an intended target latency in seconds for serving (fractional

value can be set). Applicable only when the <samp class="option">streaming</samp>
and

<samp class="option">write_prft</samp> options are enabled. This is an
informative fields

clients can use to measure the latency of the service.

</dd>

<dt><samp class="option">timeout <var class="var">timeout</var></samp></dt>

<dd> Set timeout for socket I/O operations expressed in seconds (fractional

value can be set). Applicable only for HTTP output.

</dd>

<dt><samp class="option">update_period <var class="var">period</var></samp></dt>

<dd> Set the MPD update period, for dynamic content. The unit is

second. If set to <code class="code">0</code>, the period is automatically
computed.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">use_template <var class="var">bool</var></samp></dt>

<dd> Enable or disable use of <code class="code">SegmentTemplate</code>
instead of

<code class="code">SegmentList</code> in the manifest. This is enabled by
default.

</dd>

<dt><samp class="option">use_timeline <var class="var">bool</var></samp></dt>

<dd> Enable or disable use of <code class="code">SegmentTimeline</code> within the

<code class="code">SegmentTemplate</code> manifest section. This is enabled by default.

</dd>

<dt><samp class="option">utc_timing_url <var class="var">url</var></samp></dt>

<dd> URL of the page that will return the UTC timestamp in ISO

format, for example <code class="code">https://time.akamai.com/?iso</code>

</dd>

<dt><samp class="option">window_size <var class="var">size</var></samp></dt>

<dd> Set the maximum number of segments kept in the manifest, discard the

oldest one. This is useful for live streaming.

If the value is <code class="code">0</code>, all segments are kept in the

manifest. Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">write_prft <var class="var">write_prft</var></samp></dt>

<dd> Write Producer Reference Time elements on supported streams. This also

enables writing prft boxes in the underlying muxer. Applicable only

when the <var class="var">utc_url</var> option is enabled. It is set to <var class="var">auto</var> by

default, in which case the muxer will attempt to enable it only in

modes that require it.

</dd>

</dl>

 <h4 class="subsection">4.30.2 Example# TOC</h4> Generate a DASH output reading from an input source in realtime using <code class="command">ffmpeg</code>. Two multimedia streams are generated from the input file, both containing a video stream encoded through ‘<samp class="samp">libx264</samp>’, and an audio stream encoded with ‘<samp class="samp">libfdk_aac</samp>’. The first multimedia stream contains video with a bitrate of 800k and audio at the default rate, the second with video scaled to 320x170 pixels at 300k and audio resampled at 22005 Hz. The <samp class="option">window_size</samp> option keeps only the latest 5 segments with the default duration of 5 seconds.   <pre class="example-preformatted">ffmpeg -re -i <input> -map 0 -map 0 -c:a libfdk_aac -c:v libx264 \ -b:v:0 800k -profile:v:0 main \ -b:v:1 300k -s:v:1 320x170 -profile:v:1 baseline -ar:a:1 22050 \ -bf 1 -keyint_min 120 -g 120 -sc_threshold 0 -b_strategy 0 \ -use_timeline 1 -use_template 1 -window_size 5 \ -adaptation_sets "id=0,streams=v id=1,streams=a" \ -f dash /path/to/out.mpd </pre>

 <h3 class="section">4.31 daud# TOC</h3> D-Cinema audio muxer. It accepts a single 6-channels audio stream resampled at 96000 Hz encoded with the ‘<samp class="samp">pcm_24daud</samp>’ codec. <h4 class="subsection">4.31.1 Example# TOC</h4> Use <code class="command">ffmpeg</code> to mux input audio to a ‘<samp class="samp">5.1</samp>’ channel layout resampled at 96000Hz: <pre class="example-preformatted">ffmpeg -i INPUT -af aresample=96000,pan=5.1 slow.302 </pre>

For ffmpeg versions before 7.0 you might have to use the ‘<samp class="samp">asetnsamples</samp>’ filter to limit the muxed packet size, because this format does not support muxing packets larger than 65535 bytes (3640 samples). For newer ffmpeg versions audio is automatically packetized to 36000 byte (2000 sample) packets. <h3 class="section">4.32 dv# TOC</h3> DV (Digital Video) muxer. It accepts exactly one ‘<samp class="samp">dvvideo</samp>’ video stream and at most two ‘<samp class="samp">pcm_s16</samp>’ audio streams. More constraints are defined by the property of the video, which must cor-

respond to a DV video supported profile, and on the framerate. <h4 class="subsection">4.32.1 Example# TOC</h4> Use <code class="command">ffmpeg</code> to convert the input: <pre class="example-preformatted">ffmpeg -i INPUT -s:v 720x480 -pix_fmt yuv411p -r 29.97 -ac 2 -ar 48000 -y out.dv </pre>

 <h3 class="section">4.33 ffmetadata# TOC</h3> FFmpeg metadata muxer. This muxer writes the streams metadata in the ‘<samp class="samp">ffmetadata</samp>’ format. See <a data-manual="ffmpeg-formats" href="./ffmpeg-formats.html#metadata">the Metadata chapter for information about the format. <h4 class="subsection">4.33.1 Example# TOC</h4>

Use <code class="command">ffmpeg</code> to extract metadata from an input file to a <samp class="file">metadata.ffmeta</samp> file in ‘<samp class="samp">ffmetadata</samp>’ format: <pre class="example-preformatted">ffmpeg -i INPUT -f ffmetadata metadata.ffmeta </pre>

 <h3 class="section">4.34 fifo# TOC</h3> FIFO (First-In First-Out) muxer. The ‘<samp class="samp">fifo</samp>’ pseudo-muxer allows the separation of encoding and muxing by using a first-in-first-out queue and running the actual muxer in a separate thread. This is especially useful in combination with the tee muxer and can be used to send data to several destinations with different reliability/writing speed/latency. The target muxer is either selected from the output name or specified through the <samp class="option">fifo_format</samp> option. The behavior of the ‘<samp class="samp">fifo</samp>’ muxer if the queue fills up or if the output fails (e.g. if a packet cannot be written to the output) is selectable: <ul class="itemize mark-bullet"> Output can be transparently restarted with configurable delay between retries based on real time or time of the processed stream.

Encoding can be blocked during temporary failure, or continue transparently dropping packets in case the FIFO queue fills up.

API users should be aware that callback functions (<code class="code">interrupt_callback</code>, <code class="code">io_open</code> and <code class="code">io_close</code>) used within its <code class="code">AVFormatContext</code> must be thread-safe. <h4 class="subsection">4.34.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">attempt_recovery <var class="var">bool</var></samp></dt> <dd> If failure occurs, attempt to recover the output. This is especially useful when used with network output, since it makes it possible to restart streaming transparently. By default this option is set to <code class="code">false</code>. </dd> <dt><samp class="option">drop_pkts_on_overflow <var class="var">bool</var></samp></dt> <dd> If set to <code class="code">true</code>, in case the fifo queue fills up, packets will be dropped rather than blocking the encoder. This makes it possible to continue streaming without delaying the input, at the cost of omitting part of the stream. By default this option is set to <code class="code">false</code>, so in such cases the encoder will be blocked until the muxer processes some of the packets and none of them is lost. </dd> <dt><samp class="option">fifo_format <var class="var">format_name</var></samp></dt> <dd> Specify the format name. Useful if it cannot be guessed from the output name suffix. </dd> <dt><samp class="option">format_opts <var class="var">options</var></samp></dt> <dd> Specify format options for the underlying muxer. Muxer options can be specified as a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ’:’. </dd> <dt><samp class="option">max_recovery_attempts <var class="var">count</var></samp></dt> <dd> Set maximum number of successive unsuccessful recovery attempts after which the output fails permanently. By default this option is set to <code class="code">0</code> (unlimited). </dd> <dt><samp class="option">queue_size <var class="var">size</var></samp></dt> <dd> Specify size of the queue as a number of packets. Default value is <code class="code">60</code>. </dd> <dt><samp class="option">recover_any_error <var class="var">bool</var></samp></dt> <dd> If set to <code class="code">true</code>, recovery will be attempted regardless of type of the error causing the failure. By default this option is set to <code class="code">false</code> and in case of certain (usually permanent) errors the recovery is not attempted even when the <samp class="option">attempt_recovery</samp> option is set to <code class="code">true</code>. </dd> <dt><samp class="option">recovery_wait_streamtime <var class="var">bool</var></samp></dt> <dd> If set to <code class="code">false</code>, the real time is used when waiting for the recovery attempt (i.e. the recovery will be attempted after the time specified by the <samp class="option">recovery_wait_time</samp> option). If set to <code class="code">true</code>, the

time of the processed stream is taken into account instead (i.e. the recovery will be attempted after discarding the packets corresponding to the <samp class="option">recovery_wait_time</samp> option). By default this option is set to <code class="code">false</code>. </dd> <dt><samp class="option">recovery_wait_time <var class="var">duration</var></samp></dt> <dd> Specify waiting time in seconds before the next recovery attempt after previous unsuccessful recovery attempt. Default value is <code class="code">5</code>. </dd> <dt><samp class="option">restart_with_keyframe <var class="var">bool</var></samp></dt> <dd> Specify whether to wait for the keyframe after recovering from queue overflow or failure. This option is set to <code class="code">false</code> by default. </dd> <dt><samp class="option">timeshift <var class="var">duration</var></samp></dt> <dd> Buffer the specified amount of packets and delay writing the output. Note that the value of the <samp class="option">queue_size</samp> option must be big enough to store the packets for timeshift. At the end of the input the fifo buffer is flushed at realtime speed. </dd> </dl>

 <h4 class="subsection">4.34.2 Example# TOC</h4>

Use <code class="command">ffmpeg</code> to stream to an RTMP server, continue processing the stream at real-time rate even in case of temporary failure (network outage) and attempt to recover streaming every second indefinitely: <pre class="example-preformatted">ffmpeg -re -i ... -c:v libx264 -c:a aac -f fifo -fifo_format flv \ -drop_pkts_on_overflow 1 -attempt_recovery 1 -recovery_wait_time 1 \ -map 0:v -map 0:a rtmp://example.com/live/stream_name </pre>

 <h3 class="section">4.35 film_cpk# TOC</h3> Sega film (.cpk) muxer. This format was used as internal format for several Sega games. For more information regarding the Sega film file format, visit http://wiki.multim
 It accepts at maximum one ‘<samp class="samp">cinepak</samp>’ or raw video stream, and at maximum one audio stream. <h3 class="section">4.36 filmstrip# TOC</h3> Adobe Filmstrip muxer. This format is used by several Adobe tools to store a generated filmstrip export. It accepts a single raw video stream. <h3 class="section">4.37 fits<a class="anchor hidden-xs" href="#fits"

aria-hidden=”true”># TOC</h3> Flexible Image Transport System (FITS) muxer. This image format is used to store astronomical data. For more information regarding the format, visit https://fits.gsfc.nasa.gov. <h3 class=”section”>4.38 flac# TOC</h3> Raw FLAC audio muxer. This muxer accepts exactly one FLAC audio stream. Additionally, it is possible to add images with disposition ‘<samp class=”samp”>attached_pic</samp>’. <h4 class=”subsection”>4.38.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>write_header <var class=”var”>bool</var></samp></dt> <dd> write the file header if set to <code class=”code”>true</code>, default is <code class=”code”>true</code> </dd> </dl>

 <h4 class=”subsection”>4.38.2 Example# TOC</h4> Use <code class=”command”>ffmpeg</code> to store the audio stream from an input file, together with several pictures used with ‘<samp class=”samp”>attached_pic</samp>’ disposition: <pre class=”example-preformatted”>ffmpeg -i INPUT -i pic1.png -i pic2.jpg -map 0:a -map 1 -map 2 -disposition:v attached_pic OUTPUT </pre>

 <h3 class=”section”>4.39 flv# TOC</h3> Adobe Flash Video Format muxer. <h4 class=”subsection”>4.39.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>flvflags <var class=”var”>flags</var></samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>aac_seq_header_detect</samp>’</dt> <dd> Place AAC sequence header based on audio stream data. </dd> <dt>‘<samp class=”samp”>no_sequence_end</samp>’</dt> <dd> Disable sequence end tag. </dd> <dt>‘<samp class=”samp”>no_metadata</samp>’</dt> <dd> Disable metadata tag. </dd> <dt>‘<samp class=”samp”>no_duration_filesize</samp>’</dt> <dd> Disable duration and filesize in metadata when they are equal to zero at the end of stream. (Be used to non-seekable living stream). </dd>

<dt>‘<samp class="samp">add_keyframe_index</samp>’</dt>
<dd> Used to facilitate seeking; particularly for HTTP pseudo streaming.
</dd> </dl> </dd> </dl>

 <h3
class="section">4.40 framecrc<a class="anchor
hidden-xs" href="#framecrc" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-framecrc" aria-hidden="true">TOC</h3>

Per-packet CRC (Cyclic Redundancy Check) testing format.
This muxer computes and prints the Adler-32 CRC for each audio and video packet. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the CRC. The output of the muxer consists of a line for each audio and video packet of the form: <pre class="example-preformatted"><var class="var">stream_index</var>, <var class="var">packet_dts</var>, <var class="var">packet_pts</var>, <var class="var">packet_duration</var>, <var class="var">packet_size</var>, 0x<var class="var">CRC</var> </pre>

<var class="var">CRC</var> is a hexadecimal number 0-padded to 8 digits containing the CRC of the packet.
<h4 class="subsection">4.40.1 Examples<a
class="anchor hidden-xs" href="#Examples-9" aria-hidden="true">#
TOC</h4>

For example to compute the CRC of the audio and video frames in <samp class="file">INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class="file">out.crc</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f framecrc out.crc </pre>

To print the information to stdout, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f framecrc - </pre>

With <code class="command">ffmpeg</code>, you can select the output format to which the audio and video frames are encoded before computing the CRC for each packet by specifying the audio and video codec. For example, to compute the CRC of each decoded input audio frame converted to PCM unsigned 8-bit and of each decoded input video frame converted to MPEG-2 video, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -c:a pcm_u8 -c:v mpeg2video -f framecrc - </pre>

See also the crc muxer. <a
class="anchor" id="framehash"> <h3
class="section">4.41 framehash<a class="anchor
hidden-xs" href="#framehash" aria-hidden="true"># <a class="anchor

hidden-xs”href=”#toc-framehash” aria-hidden=”true”>TOC</h3>

Per-packet hash testing format. This muxer computes and prints a cryptographic hash for each audio and video packet. This can be used for packet-by-packet equality checks without having to individually do a binary comparison on each. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the hash, but the output of explicit conversions to other codecs can also be used. It uses the SHA-256 cryptographic hash function by default, but supports several other algorithms. The output of the muxer consists of a line for each audio and video packet of the form: <pre class=”example-preformatted”><var class=”var”>stream_index</var>, <var class=”var”>packet_dts</var>, <var class=”var”>packet_pts</var>, <var class=”var”>packet_duration</var>, <var class=”var”>packet_size</var>, <var class=”var”>hash</var> </pre>

<var class=”var”>hash</var> is a hexadecimal number representing the computed hash for the packet. <dl class=”table”> <dt><samp class=”option”>hash <var class=”var”>algorithm</var></samp></dt> <dd> Use the cryptographic hash function specified by the string <var class=”var”>algorithm</var>. Supported values include <code class=”code”>MD5</code>, <code class=”code”>murmur3</code>, <code class=”code”>RIPEMD128</code>, <code class=”code”>RIPEMD160</code>, <code class=”code”>RIPEMD256</code>, <code class=”code”>RIPEMD320</code>, <code class=”code”>SHA160</code>, <code class=”code”>SHA224</code>, <code class=”code”>SHA256</code> (default), <code class=”code”>SHA512/224</code>, <code class=”code”>SHA512/256</code>, <code class=”code”>SHA384</code>, <code class=”code”>SHA512</code>, <code class=”code”>CRC32</code> and <code class=”code”>adler32</code>. </dd> </dl>

 <h4 class=”subsection”>4.41.1 Examples# TOC</h4>

To compute the SHA-256 hash of the audio and video frames in <samp class=”file”>INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class=”file”>out.sha256</samp>: <pre class=”example-preformatted”>ffmpeg -i INPUT -f framehash out.sha256 </pre>

To print the information to stdout, using the MD5 hash function, use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT -f framehash -hash md5 - </pre>

See also the hash muxer. <h3 class=”section”>4.42 framemd5<a class=”anchor

hidden-xs" href="#framemd5" aria-hidden="true"># TOC</h3>

Per-packet MD5 testing format. This is a variant of the framehash muxer. Unlike that muxer, it defaults to using the MD5 hash function. <h4 class="subsection">4.42.1 Examples# TOC</h4>

To compute the MD5 hash of the audio and video frames in <samp class="file">INPUT</samp>, converted to raw audio and video packets, and store it in the file <samp class="file">out.md5</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f framemd5 out.md5 </pre>

To print the information to stdout, use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f framemd5 - </pre>

See also the framehash and md5 muxers. <h3 class="section">4.43 gif# TOC</h3> Animated GIF muxer. Note that the GIF format has a very large time base: the delay between two frames can therefore not be smaller than one centisecond. <h4 class="subsection">4.43.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">loop <var class="var">bool</var></samp></dt> <dd> Set the number of times to loop the output. Use <code class="code">-1</code> for no loop, <code class="code">0</code> for looping indefinitely (default). </dd> <dt><samp class="option">final_delay <var class="var">delay</var></samp></dt> <dd> Force the delay (expressed in centiseconds) after the last frame. Each frame ends with a delay until the next frame. The default is <code class="code">-1</code>, which is a special value to tell the muxer to reuse the previous delay. In case of a loop, you might want to customize this value to mark a pause for instance. </dd> </dl>

 <h4 class="subsection">4.43.2 Example# TOC</h4> Encode a gif looping 10

times, with a 5 seconds delay between the loops: <pre class="example-preformatted">ffmpeg -i INPUT -loop 10 -final_delay 500 out.gif </pre>

Note 1: if you wish to extract the frames into separate GIF files, you need to force the image2 muxer: <pre class="example-preformatted">ffmpeg -i INPUT -c:v gif -f image2 "out%d.gif" </pre>

 <h3 class="section">4.44 gxf# TOC</h3> General eXchange Format (GXF) muxer. GXF was developed by Grass Valley Group, then standardized by SMPTE as SMPTE 360M and was extended in SMPTE RDD 14-2007 to include high-definition video resolutions. It accepts at most one video stream with codec ‘<samp class="samp">mjpeg</samp>’, or ‘<samp class="samp">mpeg1video</samp>’, or ‘<samp class="samp">mpeg2video</samp>’, or ‘<samp class="samp">dvvideo</samp>’ with resolution ‘<samp class="samp">512x480</samp>’ or ‘<samp class="samp">608x576</samp>’, and several audio streams with rate 48000Hz and codec ‘<samp class="samp">pcm16_le</samp>’. <h3 class="section">4.45 hash# TOC</h3>

Hash testing format. This muxer computes and prints a cryptographic hash of all the input audio and video frames. This can be used for equality checks without having to do a complete binary comparison. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the hash, but the output of explicit conversions to other codecs can also be used. Timestamps are ignored. It uses the SHA-256 cryptographic hash function by default, but supports several other algorithms. The output of the muxer consists of a single line of the form: <var class="var">algo</var>=<var class="var">hash</var>, where <var class="var">algo</var> is a short string representing the hash function used, and <var class="var">hash</var> is a hexadecimal number representing the computed hash. <dl class="table"> <dt><samp class="option">hash <var class="var">algorithm</var></samp></dt> <dd> Use the cryptographic hash function specified by the string <var class="var">algorithm</var>. Supported values include <code class="code">MD5</code>, <code class="code">murmur3</code>, <code class="code">RIPEMD128</code>, <code class="code">RIPEMD160</code>, <code class="code">RIPEMD256</code>, <code class="code">RIPEMD320</code>, <code class="code">SHA160</code>, <code class="code">SHA224</code>, <code class="code">SHA256</code> (default), <code class="code">SHA512/224</code>,

<code class="code">SHA512/256</code>, <code class="code">SHA384</code>, <code class="code">SHA512</code>, <code class="code">CRC32</code> and <code class="code">adler32</code>. </dd> </dl>

 <h4 class="subsection">4.45.1 Examples# TOC</h4>

To compute the SHA-256 hash of the input converted to raw audio and video, and store it in the file <samp class="file">out.sha256</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f hash out.sha256 </pre>

To print an MD5 hash to stdout use the command: <pre class="example-preformatted">ffmpeg -i INPUT -f hash -hash md5 - </pre>

See also the framehash muxer. <h3 class="section">4.46 hds# TOC</h3> HTTP Dynamic Streaming (HDS) muxer. HTTP dynamic streaming, or HDS, is an adaptive bitrate streaming method developed by Adobe. HDS delivers MP4 video content over HTTP connections. HDS can be used for on-demand streaming or live streaming. This muxer creates an .f4m (Adobe Flash Media Manifest File) manifest, an .abst (Adobe Bootstrap File) for each stream, and segment files in a directory specified as the output. These needs to be accessed by an HDS player through HTTPS for it to be able to perform playback on the generated stream. <h4 class="subsection">4.46.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">extra_window_size <var class="var">int</var></samp></dt> <dd> number of fragments kept outside of the manifest before removing from disk </dd> <dt><samp class="option">min_frag_duration <var class="var">microseconds</var></samp></dt> <dd> minimum fragment duration (in microseconds), default value is 1 second (<code class="code">10000000</code>) </dd> <dt><samp class="option">remove_at_exit <var class="var">bool</var></samp></dt> <dd> remove all fragments when finished when set to <code class="code">true</code> </dd> <dt><samp class="option">window_size <var class="var">int</var></samp></dt> <dd> number of fragments kept in the manifest, if set to a value different from <code class="code">0</code>. By default all segments are kept in the output directory. </dd> </dl>

 <h4 class="subsection">4.46.2 Example# TOC</h4> Use <code class="command">ffmpeg</code> to generate HDS files to the <samp class="file">output.hds</samp> directory in real-time rate: <pre class="example-preformatted">ffmpeg -re -i INPUT -f hds -b:v 200k output.hds </pre>

 <h3 class="section">4.47 hls# TOC</h3>

Apple HTTP Live Streaming muxer that segments MPEG-TS according to the HTTP Live Streaming (HLS) specification. It creates a playlist file, and one or more segment files. The output filename specifies the playlist filename. By default, the muxer creates a file for each segment produced. These files have the same name as the playlist, followed by a sequential number and a .ts extension. Make sure to require a closed GOP when encoding and to set the GOP size to fit your segment time constraint. For example, to convert an input file with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.mkv -c:v h264 -flags +cgop -g 30 -hls_time 1 out.m3u8 </pre> This example will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">out0.ts</samp>, <samp class="file">out1.ts</samp>, <samp class="file">out2.ts</samp>, etc. See also the segment muxer, which provides a more generic and flexible implementation of a segmenter, and can be used to perform HLS segmentation. <h4 class="subsection">4.47.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">hls_init_time <var class="var">duration</var></samp></dt> <dd> Set the initial target segment length. Default value is <var class="var">0</var>. <var class="var">duration</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. Segment will be cut on the next key frame after this time has passed on the first m3u8 list. After the initial playlist is filled, <code class="command">ffmpeg</code> will cut segments at duration equal to <samp class="option">hls_time</samp>. </dd> <dt><samp class="option">hls_time <var class="var">duration</var></samp></dt> <dd> Set the target segment length. Default value is 2. <var class="var">duration</var> must be a time duration specifi-

cation, see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. Segment will be cut on the next key frame after this time has passed. </dd> <dt><samp class=”option”>hls_list_size <var class=”var”>size</var></samp></dt> <dd> Set the maximum number of playlist entries. If set to 0 the list file will contain all the segments. Default value is 5. </dd> <dt><samp class=”option”>hls_delete_threshold <var class=”var”>size</var></samp></dt> <dd> Set the number of unreferenced segments to keep on disk before <code class=”code”>hls_flags delete_segments</code> deletes them. Increase this to allow continue clients to download segments which were recently referenced in the playlist. Default value is 1, meaning segments older than <samp class=”option”>hls_list_size+1</samp> will be deleted. </dd> <dt><samp class=”option”>hls_start_number_source <var class=”var”>source</var></samp></dt> <dd> Start the playlist sequence number (<code class=”code”>#EXT-X-MEDIA-SEQUENCE</code>) according to the specified source. Unless <samp class=”option”>hls_flags single_file</samp> is set, it also specifies source of starting sequence numbers of segment and subtitle filenames. In any case, if <samp class=”option”>hls_flags append_list</samp> is set and read playlist sequence number is greater than the specified start sequence number, then that value will be used as start value. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>generic (default)</samp></dt> <dd> Set the start numbers according to the <samp class=”option”>start_number</samp> option value. </dd> <dt><samp class=”option”>epoch</samp></dt> <dd> Set the start number as the seconds since epoch (1970-01-01 00:00:00). </dd> <dt><samp class=”option”>epoch_us</samp></dt> <dd> Set the start number as the microseconds since epoch (1970-01-01 00:00:00). </dd> <dt><samp class=”option”>datetime</samp></dt> <dd> Set the start number based on the current date/time as YYYYmmddHHMMSS. e.g. 20161231235759. </dd> </dl>

</dd> <dt><samp class=”option”>start_number <var class=”var”>number</var></samp></dt> <dd> Start the playlist sequence number (<code class=”code”>#EXT-X-MEDIA-SEQUENCE</code>) from the specified <var class=”var”>number</var> when <samp class=”option”>hls_start_number_source</samp> value is <var class=”var”>generic</var>. (This is the default case.) Unless <samp class=”option”>hls_flags single_file</samp> is set, it also specifies starting sequence numbers of segment and subtitle filenames. Default value is 0. </dd> <dt><samp class=”option”>hls_allow_cache <var class=”var”>bool</var></samp></dt> <dd> Explicitly set whether the client MAY (1) or MUST NOT (0) cache media segments. </dd> <dt><samp class=”option”>hls_base_url <var class=”var”>baseurl</var></samp></dt> <dd> Append <var class=”var”>baseurl</var> to every entry in the playlist. Useful to generate playlists with absolute paths. Note that the playlist sequence number must be unique for each segment and it

is not to be confused with the segment filename sequence number which can be cyclic, for example if the <samp class="option">wrap</samp> option is specified. </dd> <dt><samp class="option">hls_segment_filename <var class="var">filename</var></samp></dt> <dd> Set the segment filename. Unless the <samp class="option">hls_flags</samp> option is set with ‘<samp class="samp">single_file</samp>’, <var class="var">filename</var> is used as a string format with the segment number appended. For example: <pre class="example-preformatted">ffmpeg -i in.nut -hls_segment_filename 'file%03d.ts' out.m3u8 </pre>

will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">file000.ts</samp>, <samp class="file">file001.ts</samp>, <samp class="file">file002.ts</samp>, etc. <var class="var">filename</var> may contain a full path or relative path specification, but only the file name part without any path will be contained in the m3u8 segment list. Should a relative path be specified, the path of the created segment files will be relative to the current working directory. When <samp class="option">strftime_mkdir</samp> is set, the whole expanded value of <var class="var">filename</var> will be written into the m3u8 segment list. When <samp class="option">var_stream_map</samp> is set with two or more variant streams, the <var class="var">filename</var> pattern must contain the string "%v", and this string will be expanded to the position of variant stream index in the generated segment file names. For example: <pre class="example-preformatted">ffmpeg -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ -hls_segment_filename 'file_%v_%03d.ts' out_%v.m3u8 </pre>

will produce the playlists segment file sets: <samp class="file">file_0_000.ts</samp>, <samp class="file">file_0_001.ts</samp>, <samp class="file">file_0_002.ts</samp>, etc. and <samp class="file">file_1_000.ts</samp>, <samp class="file">file_1_001.ts</samp>, <samp class="file">file_1_002.ts</samp>, etc. The string "%v" may be present in the filename or in the last directory name containing the file, but only in one of them. (Additionally, %v may appear multiple times in the last sub-directory or filename.) If the string %v is present in the directory name, then sub-directories are created after expanding the directory name pattern. This enables creation of segments corresponding to different variant streams in subdirectories. For example: <pre class="example-preformatted">ffmpeg -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ -hls_segment_filename 'vs%v/file_%03d.ts' vs%v/out.m3u8 </pre> will produce the playlists segment file sets: <samp class="file">vs0/file_000.ts</samp>, <samp class="file">vs0/file_001.ts</samp>, <samp class="file">vs0/file_002.ts</samp>, etc. and <samp class="file">vs1/file_000.ts</samp>, <samp class="file">vs1/file_001.ts</samp>,

<samp class="file">vs1/file_002.ts</samp>, etc. </dd> <dt><samp class="option">strftime <var class="var">bool</var></samp></dt> <dd> Use <code class="code">strftime()</code> on <var class="var">filename</var> to expand the segment filename with localtime. The segment number is also available in this mode, but to use it, you need to set ‘<samp class="samp">second_level_segment_index</samp>’ in the <samp class="option">hls_flag</samp> and %%d will be the specifier. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -hls_segment_filename 'file-%Y%m%d-%s.ts' out.m3u8 </pre> will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">file-20160215-1455569023.ts</samp>, <samp class="file">file-20160215-1455569024.ts</samp>, etc. Note: On some systems/environments, the <code class="code">%s</code> specifier is not available. See <code class="code">strftime()</code> documentation. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -hls_flags second_level_segment_index -hls_segment_filename 'file-%Y%m%d-%%04d.ts' out.m3u8 </pre> will produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">file-20160215-0001.ts</samp>, <samp class="file">file-20160215-0002.ts</samp>, etc. </dd> <dt><samp class="option">strftime_mkdir <var class="var">bool</var></samp></dt> <dd> Used together with <samp class="option">strftime</samp>, it will create all subdirectories which are present in the expanded values of option <samp class="option">hls_segment_filename</samp>. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -strftime_mkdir 1 -hls_segment_filename '%Y%m%d/file-%Y%m%d-%s.ts' out.m3u8 </pre> will create a directory <samp class="file">201560215</samp> (if it does not exist), and then produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">20160215/file-20160215-1455569023.ts</samp>, <samp class="file">20160215/file-20160215-1455569024.ts</samp>, etc. For example: <pre class="example-preformatted">ffmpeg -i in.nut -strftime 1 -strftime_mkdir 1 -hls_segment_filename '%Y/%m/%d/file-%Y%m%d-%s.ts' out.m3u8 </pre> will create a directory hierarchy <samp class="file">2016/02/15</samp> (if any of them do not exist), and then produce the playlist, <samp class="file">out.m3u8</samp>, and segment files: <samp class="file">2016/02/15/file-20160215-1455569023.ts</samp>, <samp class="file">2016/02/15/file-20160215-1455569024.ts</samp>, etc. </dd> <dt><samp class="option">hls_segment_options <var class="var">options_list</var></samp></dt> <dd> Set output format options using a :-separated list of key=value parameters. Values containing <code class="code">:</code> special characters must be escaped. </dd> <dt><samp class="option">hls_key_info_file <var class="var">key_info_file</var></samp></dt> <dd> Use the information in <var class="var">key_info_file</var> for segment encryption.

The first line of <var class="var">key_info_file</var> specifies the key URI written to the playlist. The key URL is used to access the encryption key during playback. The second line specifies the path to the key file used to obtain the key during the encryption process. The key file is read as a single packed array of 16 octets in binary format. The optional third line specifies the initialization vector (IV) as a hexadecimal string to be used instead of the segment sequence number (default) for encryption. Changes to <var class="var">key_info_file</var> will result in segment encryption with the new key/IV and an entry in the playlist for the new key URI/IV if <samp class="option">hls_flags periodic_rekey</samp> is enabled. Key info file format: <pre class="example-preformatted"><var class="var">key URI</var> <var class="var">key file path</var> <var class="var">IV</var> (optional) </pre>

Example key URIs: <pre class="example-preformatted">http://server/file.key /path/to/file.key file.key </pre>

Example key file paths: <pre class="example-preformatted">file.key /path/to/file.key </pre>

Example IV: <pre class="example-preformatted">0123456789ABCDEF0123456789ABCDEF </pre>

Key info file example: <pre class="example-preformatted">http://server/file.key /path/to/file.key 0123456789ABCDEF0123456789ABCDEF
</pre>

Example shell script: <pre class="example-preformatted">#!/bin/sh BASE_URL=${1:-'.'} openssl rand 16 > file.key echo $BASE_URL/file.key > file.keyinfo echo file.key >> file.keyinfo echo $(openssl rand -hex 16) >> file.keyinfo ffmpeg -f lavfi -re -i testsrc -c:v h264 -hls_flags delete_segments \ -hls_key_info_file file.keyinfo out.m3u8
</pre>

</dd> <dt><samp class="option">hls_enc <var class="var">bool</var></samp></dt> <dd> Enable (1) or disable (0) the AES128 encryption. When enabled every segment generated is encrypted and the encryption key is saved as <var class="var">playlist name</var>.key. </dd> <dt><samp class="option">hls_enc_key <var class="var">key</var></samp></dt> <dd> Specify a 16-octet key to encrypt the segments, by default it is randomly generated. </dd> <dt><samp class="option">hls_enc_key_url <var class="var">keyurl</var></samp></dt> <dd> If set, <var class="var">keyurl</var> is prepended instead of <var class="var">baseurl</var> to the key filename in the playlist. </dd> <dt><samp class="option">hls_enc_iv <var class="var">iv</var></samp></dt> <dd> Specify the 16-octet initialization vector for every segment instead of the autogenerated ones. </dd> <dt><samp class="option">hls_segment_type <var class="var">flags</var></samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">mpegts</samp>’</dt>

<dd> Output segment files in MPEG-2 Transport Stream format. This is compatible with all HLS versions. </dd> <dt>‘<samp class=”samp”>fmp4</samp>’</dt> <dd> Output segment files in fragmented MP4 format, similar to MPEG-DASH. fmp4 files may be used in HLS version 7 and above. </dd> </dl>

</dd>

<dt><samp class=”option”>hls_fmp4_init_filename <var class=”var”>filename</var></samp></dt>

<dd> Set filename for the fragment files header file, default filename is <samp class=”file”>init.mp4</samp>.

When <samp class=”option”>strftime</samp> is enabled, <var class=”var”>filename</var> is expanded to the segment filename with localtime.

For example:

<pre class=”example-preformatted”>ffmpeg -i in.nut -hls_segment_type fmp4 -strftime 1 -hls_fmp4_init_filename "%s_init.mp4" out.m3u8

</pre>

will produce init like this <samp class=”file”>1602678741_init.mp4</samp>.

</dd>

<dt><samp class=”option”>hls_fmp4_init_resend <var class=”var”>bool</var></samp></dt>

<dd> Resend init file after m3u8 file refresh every time, default is <var class=”var”>0</var>.

When <samp class=”option”>var_stream_map</samp> is set with two or more variant streams, the

<var class=”var”>filename</var> pattern must contain the string "%v", this string specifies

the position of variant stream index in the generated init file names.

The string "%v" may be present in the filename or in the last directory name

containing the file. If the string is present in the directory name, then

sub-directories are created after expanding the directory name pattern. This

enables creation of init files corresponding to different variant streams in

subdirectories.

</dd>

<dt><samp class="option">hls_flags <var class="var">flags</var></samp></dt>

<dd> Possible values:

<dl class="table">

<dt>‘<samp class="samp">single_file</samp>’</dt>

<dd> If this flag is set, the muxer will store all segments in a single MPEG-TS

file, and will use byte ranges in the playlist. HLS playlists generated with

this way will have the version number 4.

For example:

<pre class="example-preformatted">ffmpeg -i in.nut -hls_flags single_file out.m3u8

</pre>

will produce the playlist, <samp class="file">out.m3u8</samp>, and a single segment file,

<samp class="file">out.ts</samp>.

</dd>

<dt>‘<samp class="samp">delete_segments</samp>’</dt>

<dd> Segment files removed from the playlist are deleted after a period of time

equal to the duration of the segment plus the duration of the playlist.

</dd>

<dt>‘<samp class="samp">append_list</samp>’</dt>

<dd> Append new segments into the end of old segment list,

and remove the <code class=”code”>#EXT-X-ENDLIST</code> from the old segment list.

</dd>

<dt>‘<samp class=”samp”>round_durations</samp>’</dt>

<dd> Round the duration info in the playlist file segment info to integer

values, instead of using floating point.

If there are no other features requiring higher HLS versions be used,

then this will allow <code class=”command”>ffmpeg</code> to output a HLS version 2 m3u8.

</dd>

<dt>‘<samp class=”samp”>discont_start</samp>’</dt>

<dd> Add the <code class=”code”>#EXT-X-DISCONTINUITY</code> tag to the playlist, before the

first segment’s information.

</dd>

<dt>‘<samp class=”samp”>omit_endlist</samp>’</dt>

<dd> Do not append the <code class=”code”>EXT-X-ENDLIST</code> tag at the end of the playlist.

</dd>

<dt>‘<samp class=”samp”>periodic_rekey</samp>’</dt>

<dd> The file specified by <code class=”code”>hls_key_info_file</code> will be checked periodically and

detect updates to the encryption info. Be sure to replace this file atomically,

including the file containing the AES encryption key.

</dd>

<dt>‘<samp class=”samp”>independent_segments</samp>’</dt>

<dd> Add the <code class=”code”>#EXT-X-INDEPENDENT-SEGMENTS</code> tag to playlists that has video segments

and when all the segments of that playlist are guaranteed to start with a key frame.

</dd>

<dt>‘<samp class="samp">iframes_only</samp>’</dt>

<dd> Add the <code class="code">#EXT-X-I-FRAMES-ONLY</code> tag to playlists that has video segments

and can play only I-frames in the <code class="code">#EXT-X-BYTERANGE</code> mode.

</dd>

<dt>‘<samp class="samp">split_by_time</samp>’</dt>

<dd> Allow segments to start on frames other than key frames. This improves

behavior on some players when the time between key frames is inconsistent,

but may make things worse on others, and can cause some oddities during

seeking. This flag should be used with the <samp class="option">hls_time</samp> option.

</dd>

<dt>‘<samp class="samp">program_date_time</samp>’</dt>

<dd> Generate <code class="code">EXT-X-PROGRAM-DATE-TIME</code> tags.

</dd>

<dt>‘<samp class="samp">second_level_segment_index</samp>’</dt>

<dd> Make it possible to use segment indexes as %%d in the

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

<samp class="option">strftime</samp> option is on. To get fixed width numbers with trailing zeroes, %%0xd format

is available where x is the required width.

</dd>

<dt>‘<samp class="samp">second_level_segment_size</samp>’</dt>

<dd> Make it possible to use segment sizes (counted in bytes) as %%s in

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

strftime is on. To get fixed width numbers with trailing zeroes, %%0xs format

is available where x is the required width.

</dd>

<dt>‘<samp class="samp">second_level_segment_duration</samp>’</dt>

<dd> Make it possible to use segment duration (calculated in microseconds) as %%t in

<samp class="option">hls_segment_filename</samp> option expression besides date/time values when

strftime is on. To get fixed width numbers with trailing zeroes, %%0xt format

is available where x is the required width.

For example:

<pre class="example-preformatted">ffmpeg -i sample.mpeg \

-f hls -hls_time 3 -hls_list_size 5 \

-hls_flags second_level_segment_index+second_level_segment_size+second_level_segment_duration \

-strftime 1 -strftime_mkdir 1 -hls_segment_filename "segment_%Y%m%d%H%M%S_%%04d_%%08s_ stream.m3u8

</pre>

will produce segments like this:

<samp class="file">segment_20170102194334_0003_00122200_0000003000000.ts</samp>, <samp class="file">segment_20170102194334_0004_00120072_0000003000000.ts</samp> etc.

</dd>

<dt>‘<samp class="samp">temp_file</samp>’</dt>

<dd> Write segment data to <samp class=”file”>filename.tmp</samp> and rename to filename only once the

segment is complete.

A webserver serving up segments can be configured to reject requests to *.tmp to

prevent access to in-progress segments before they have been added to the m3u8

playlist.

This flag also affects how m3u8 playlist files are created. If this flag is set,

all playlist files will be written into a temporary file and renamed after they

are complete, similarly as segments are handled. But playlists with <code class=”code”>file</code>

protocol and with <samp class=”option”>hls_playlist_type</samp> type other than ‘<samp class=”samp”>vod</samp>’ are

always written into a temporary file regardless of this flag.

Master playlist files specified with <samp class=”option”>master_pl_name</samp>, if any, with

<code class=”code”>file</code> protocol, are always written into temporary file regardless of this

flag if <samp class=”option”>master_pl_publish_rate</samp> value is other than zero.

</dd>

</dl>

</dd> <dt><samp class=”option”>hls_playlist_type <var class=”var”>type</var></samp></dt> <dd> If type is ‘<samp class=”samp”>event</samp>’, emit <code class=”code”>#EXT-X-PLAYLIST-TYPE:EVENT</code> in the m3u8 header. This forces <samp class=”option”>hls_list_size</samp> to 0; the playlist can only be appended to. If type is ‘<samp class=”samp”>vod</samp>’, emit <code class=”code”>#EXT-X-PLAYLIST-TYPE:VOD</code> in the m3u8 header. This forces <samp class=”option”>hls_list_size</samp> to 0; the playlist must not change. </dd> <dt><samp class=”option”>method <var class=”var”>method</var></samp></dt> <dd> Use the given HTTP method to create the hls files. For example: <pre class=”example-preformatted”>ffmpeg -re -i in.ts

-f hls -method PUT http://example.com/live/out.m3u8 </pre> will upload all the mpegts segment files to the HTTP server using the HTTP PUT method, and update the m3u8 files every <code class=”code”>refresh</code> times using the same method. Note that the HTTP server must support the given method for uploading files. </dd> <dt><samp class=”option”>http_user_agent <var class=”var”>agent</var></samp></dt> <dd> Override User-Agent field in HTTP header. Applicable only for HTTP output. </dd> <dt><samp class=”option”>var_stream_map <var class=”var”>stream_map</var></samp></dt> <dd> Specify a map string defining how to group the audio, video and subtitle streams into different variant streams. The variant stream groups are separated by space. Expected string format is like this "a:0,v:0 a:1,v:1 ....". Here a:, v:, s: are the keys to specify audio, video and subtitle streams respectively. Allowed values are 0 to 9 (limited just based on practical usage). When there are two or more variant streams, the output filename pattern must contain the string "%v": this string specifies the position of variant stream index in the output media playlist filenames. The string "%v" may be present in the filename or in the last directory name containing the file. If the string is present in the directory name, then sub-directories are created after expanding the directory name pattern. This enables creation of variant streams in subdirectories. A few examples follow. <ul class=”itemize mark-bullet”> Create two hls variant streams. The first variant stream will contain video stream of bitrate 1000k and audio stream of bitrate 64k and the second variant stream will contain video stream of bitrate 256k and audio stream of bitrate 32k. Here, two media playlist with file names <samp class=”file”>out_0.m3u8</samp> and <samp class=”file”>out_1.m3u8</samp> will be created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ http://example.com/live/out_%v.m3u8 </pre>

If you want something meaningful text instead of indexes in result names, you may specify names for each or some of the variants. The following example will create two hls variant streams as in the previous one. But here, the two media playlist with file names <samp class=”file”>out_my_hd.m3u8</samp> and <samp class=”file”>out_my_sd.m3u8</samp> will be created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0,name:my_hd v:1,a:1,name:my_sd" \ http://example.com/live/out_%v.m3u8 </pre>

Create three hls variant streams. The first variant stream will be a video only stream with video bitrate 1000k, the second variant stream will be an audio only stream with bitrate 64k and the third variant stream will be a video only stream with bitrate 256k. Here, three media

playlist with file names <samp class=”file”>out_0.m3u8</samp>, <samp class=”file”>out_1.m3u8</samp> and <samp class=”file”>out_2.m3u8</samp> will be created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k \ -map 0:v -map 0:a -map 0:v -f hls -var_stream_map "v:0 a:0 v:1" \ http://example.com/live/out_%v.m3u8 </pre>

Create the variant streams in subdirectories. Here, the first media playlist is created at <samp class=”file”>http://example.com/live/vs_0/out.m3u8</samp> and the second one at <samp class=”file”>http://example.com/live/vs_1/out.m3u8</samp>. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls -var_stream_map "v:0,a:0 v:1,a:1" \ http://example.com/live/vs_%v/out.m3u8 </pre>

Create two audio only and two video only variant streams. In addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the two video only variant streams with audio group names ’aud_low’ and ’aud_high’. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k -b:v:1 3000k \ -map 0:a -map 0:a -map 0:v -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low a:1,agroup:aud_high v:0,agroup:aud_low v:1,agroup:aud_high" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

 Create two audio only and one video only variant streams. In addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the one video only variant streams with audio group name ’aud_low’, and the audio group have default stat is NO or YES. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k \ -map 0:a -map 0:a -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low,default:yes a:1,agroup:aud_low v:0,agroup:aud_low" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

Create two audio only and one video only variant streams. In addition to the <code class=”code”>#EXT-X-STREAM-INF</code> tag for each variant stream in the master playlist, the <code class=”code”>#EXT-X-MEDIA</code> tag is also added for the two audio only variant streams and they are mapped to the one video only variant streams with audio group name ’aud_low’, and the audio group

have default stat is NO or YES, and one audio have and language is named ENG, the other audio language is named CHN. By default, a single hls variant containing all the encoded streams is created. <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:a:0 32k -b:a:1 64k -b:v:0 1000k \ -map 0:a -map 0:a -map 0:v -f hls \ -var_stream_map "a:0,agroup:aud_low,default:yes,language:ENG a:1,agroup:aud_low,language:CHN v:0,agroup:aud_low" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre>

Create a single variant stream. Add the <code class=”code”>#EXT-X-MEDIA</code> tag with <code class=”code”>TYPE=SUBTITLES</code> in the master playlist with webvtt subtitle group name ’subtitle’ and optional subtitle name, e.g. ’English’. Make sure the input file has one text subtitle stream at least. <pre class=”example-preformatted”>ffmpeg -y -i input_with_subtitle.mkv \ -b:v:0 5250k -c:v h264 -pix_fmt yuv420p -profile:v main -level 4.1 \ -b:a:0 256k \ -c:s webvtt -c:a mp2 -ar 48000 -ac 2 -map 0:v -map 0:a:0 -map 0:s:0 \ -f hls -var_stream_map "v:0,a:0,s:0,sgroup:subtitle,sname:English" \ -master_pl_name master.m3u8 -t 300 -hls_time 10 -hls_init_time 4 -hls_list_size \ 10 -master_pl_publish_rate 10 -hls_flags \ delete_segments+discont_start+split_by_time ./tmp/video.m3u8 </pre>

</dd> <dt><samp class=”option”>cc_stream_map <var class=”var”>cc_stream_map</var></samp></
<dd> Map string which specifies different closed captions groups and their attributes. The closed captions stream groups are separated by space. Expected string format is like this "ccgroup:<group name>,instreamid:<INSTREAM-ID>,language:<language code> ....". ’ccgroup’ and ’instreamid’ are mandatory attributes. ’language’ is an optional attribute. The closed captions groups configured using this option are mapped to different variant streams by providing the same ’ccgroup’ name in the <samp class=”option”>var_stream_map</samp> string. For example: <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v:0 1000k -b:v:1 256k -b:a:0 64k -b:a:1 32k \ -a53cc:0 1 -a53cc:1 1 \ -map 0:v -map 0:a -map 0:v -map 0:a -f hls \ -cc_stream_map "ccgroup:cc,instreamid:CC1,language:en cc-group:cc,instreamid:CC2,language:sp" \ -var_stream_map "v:0,a:0,ccgroup:cc v:1,a:1,ccgroup:cc" \ -master_pl_name master.m3u8 \ http://example.com/live/out_%v.m3u8 </pre> will add two <code class=”code”>#EXT-X-MEDIA</code> tags with <code class=”code”>TYPE=CLOSED-CAPTIONS</code> in the master playlist for the INSTREAM-IDs ’CC1’ and ’CC2’. Also, it will add <code class=”code”>CLOSED-CAPTIONS</code> attribute with group name ’cc’ for the two output variant streams. If <samp class=”option”>var_stream_map</samp> is not set, then the first available ccgroup in <samp class=”option”>cc_stream_map</samp> is mapped to the output variant stream. For example: <pre class=”example-preformatted”>ffmpeg -re -i in.ts -b:v

1000k -b:a 64k -a53cc 1 -f hls \ -cc_stream_map "ccgroup:cc,instreamid:CC1,language:en" \ -master_pl_name master.m3u8 \ http://example.com/live/out.m3u8 </pre> this will add <code class="code">#EXT-X-MEDIA</code> tag with <code class="code">TYPE=CLOSED-CAPTIONS</code> in the master playlist with group name ’cc’, language ’en’ (english) and INSTREAM-ID ’CC1’. Also, it will add <code class="code">CLOSED-CAPTIONS</code> attribute with group name ’cc’ for the output variant stream. </dd> <dt><samp class="option">master_pl_name <var class="var">name</var></samp></dt> <dd> Create HLS master playlist with the given name. For example: <pre class="example-preformatted">ffmpeg -re -i in.ts -f hls -master_pl_name master.m3u8 http://example.com/live/out.m3u8 </pre> creates an HLS master playlist with name <samp class="file">master.m3u8</samp> which is published at http://example.com/live/. </dd> <dt><samp class="option">master_pl_publish_rate <var class="var">count</var></samp></dt <dd> Publish master play list repeatedly every after specified number of segment intervals. For example: <pre class="example-preformatted">ffmpeg -re -i in.ts -f hls -master_pl_name master.m3u8 \ -hls_time 2 -master_pl_publish_rate 30 http://example.com/live/out.m3u8 </pre> creates an HLS master playlist with name <samp class="file">master.m3u8</samp> and keeps publishing it repeatedly every after 30 segments i.e. every after 60s. </dd> <dt><samp class="option">http_persistent <var class="var">bool</var></samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP output. </dd> <dt><samp class="option">timeout <var class="var">timeout</var></samp></dt> <dd> Set timeout for socket I/O operations. Applicable only for HTTP output. </dd> <dt><samp class="option">ignore_io_errors <var class="var">bool</var></samp></dt> <dd> Ignore IO errors during open, write and delete. Useful for long-duration runs with network output. </dd> <dt><samp class="option">headers <var class="var">headers</var></samp></dt> <dd> Set custom HTTP headers, can override built in default headers. Applicable only for HTTP output. </dd> </dl>

 <h3 class="section">4.48 iamf# TOC</h3> Immersive Audio Model and Formats (IAMF) muxer. IAMF is used to provide immersive audio content for presentation on a wide range of devices in both streaming and offline applications. These applications include internet audio streaming, multicasting/broadcasting services, file download, gaming, communication, virtual and augmented reality, and others. In these applications, audio may be played back on a wide range of devices, e.g., headphones, mobile phones, tablets, TVs, sound bars, home theater systems, and big screens.

 This format was promoted and designed by Alliance for Open Media. For more information about this format, see https://aomedia.org/iamf/. <h3 class=”section”>4.49 ico# TOC</h3>

ICO file muxer. Microsoft’s icon file format (ICO) has some strict limitations that should be noted: <ul class=”itemize mark-bullet”> Size cannot exceed 256 pixels in any dimension

Only BMP and PNG images can be stored

If a BMP image is used, it must be one of the following pixel formats: <pre class=”example-preformatted”>BMP Bit Depth FFmpeg Pixel Format 1bit pal8 4bit pal8 8bit pal8 16bit rgb555le 24bit bgr24 32bit bgra </pre>

If a BMP image is used, it must use the BITMAPINFOHEADER DIB header

If a PNG image is used, it must use the rgba pixel format

 <h3 class=”section”>4.50 ilbc# TOC</h3> Internet Low Bitrate Codec (iLBC) raw muxer. It accepts a single ‘<samp class=”samp”>ilbc</samp>’ audio stream. <h3 class=”section”>4.51 image2, image2pipe# TOC</h3> Image file muxer. The ‘<samp class=”samp”>image2</samp>’ muxer writes video frames to image files. The output filenames are specified by a pattern, which can be used to produce sequentially numbered series of files. The pattern may contain the string "%d" or "%0<var class=”var”>N</var>d", this string specifies the position of the characters representing a numbering in the filenames. If the form "%0<var class=”var”>N</var>d" is used, the string representing the number in each filename is 0-padded to <var class=”var”>N</var> digits. The literal character ’%’ can be specified in the pattern with the string "%%". If the pattern contains "%d" or "%0<var class=”var”>N</var>d", the first filename of the file list specified will contain the number 1, all the following numbers will be sequential. The pattern may contain a suffix which is used to automatically determine the format of the image files to write. For example the pattern "img-%03d.bmp" will specify a sequence of filenames of the

form <samp class="file">img-001.bmp</samp>, <samp class="file">img-002.bmp</samp>, ..., <samp class="file">img-010.bmp</samp>, etc. The pattern "img%%-%d.jpg" will specify a sequence of filenames of the form <samp class="file">img%-1.jpg</samp>, <samp class="file">img%-2.jpg</samp>, ..., <samp class="file">img%-10.jpg</samp>, etc. The image muxer supports the .Y.U.V image file format. This format is special in that each image frame consists of three files, for each of the YUV420P components. To read or write this image file format, specify the name of the ’.Y’ file. The muxer will automatically open the ’.U’ and ’.V’ files as required. The ‘<samp class="samp">image2pipe</samp>’ muxer accepts the same options as the ‘<samp class="samp">image2</samp>’ muxer, but ignores the pattern verification and expansion, as it is supposed to write to the command output rather than to an actual stored file. <h4 class="subsection">4.51.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">frame_pts <var class="var">bool</var></samp></dt> <dd> If set to 1, expand the filename with the packet PTS (presentation time stamp). Default value is 0. </dd> <dt><samp class="option">start_number <var class="var">count</var></samp></dt> <dd> Start the sequence from the specified number. Default value is 1. </dd> <dt><samp class="option">update <var class="var">bool</var></samp></dt> <dd> If set to 1, the filename will always be interpreted as just a filename, not a pattern, and the corresponding file will be continuously overwritten with new images. Default value is 0. </dd> <dt><samp class="option">strftime <var class="var">bool</var></samp></dt> <dd> If set to 1, expand the filename with date and time information from <code class="code">strftime()</code>. Default value is 0. </dd> <dt><samp class="option">atomic_writing <var class="var">bool</var></samp></dt> <dd> Write output to a temporary file, which is renamed to target filename once writing is completed. Default is disabled. </dd> <dt><samp class="option">protocol_opts <var class="var">options_list</var></samp></dt> <dd> Set protocol options as a :-separated list of key=value parameters. Values containing the <code class="code">:</code> special character must be escaped. </dd> <dt><samp class="option">update_filemtime <var class="var">bool</var></samp></dt> <dd> If set to 1, set each output file’s modification time to the <code class="code">creation_time</code> metadata value plus the frame’s PTS offset. If <code class="code">creation_time</code> is missing or unparsable, a warning is logged and the option is ignored. Default value is 0. </dd> </dl>

 <h4 class="subsection">4.51.2 Examples<a class="anchor hidden-xs" href="#Examples-

13” aria-hidden=”true”># TOC</h4> <ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> for creating a sequence of files <samp class=”file”>img-001.jpeg</samp>, <samp class=”file”>img-002.jpeg</samp>, ..., taking one image every second from the input video: <pre class=”example-preformatted”>ffmpeg -i in.avi -fps_mode cfr -r 1 -f image2 'img-%03d.jpeg' </pre>

Note that with <code class=”command”>ffmpeg</code>, if the format is not specified with the <code class=”code”>-f</code> option and the output filename specifies an image file format, the image2 muxer is automatically selected, so the previous command can be written as: <pre class=”example-preformatted”>ffmpeg -i in.avi -fps_mode cfr -r 1 'img-%03d.jpeg' </pre>

Note also that the pattern must not necessarily contain "%d" or "%0<var class=”var”>N</var>d", for example to create a single image file <samp class=”file”>img.jpeg</samp> from the start of the input video you can employ the command: <pre class=”example-preformatted”>ffmpeg -i in.avi -f image2 -frames:v 1 img.jpeg </pre>

The <samp class=”option”>strftime</samp> option allows you to expand the filename with date and time information. Check the documentation of the <code class=”code”>strftime()</code> function for the syntax.

To generate image files from the <code class=”code”>strftime()</code> "%Y-%m-%d_%H-%M-%S" pattern, the following <code class=”command”>ffmpeg</code> command can be used: <pre class=”example-preformatted”>ffmpeg -f v4l2 -r 1 -i /dev/video0 -f image2 -strftime 1 "%Y-%m-%d_%H-%M-%S.jpg" </pre>

Set the file name with current frame’s PTS: <pre class=”example-preformatted”>ffmpeg -f v4l2 -r 1 -i /dev/video0 -copyts -f image2 -frame_pts true %d.jpg </pre>

Publish contents of your desktop directly to a WebDAV server every second: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 1 -i :0.0 -q:v 6 -update 1 -protocol_opts method=PUT http://example.com/desktop.jpg </pre>

 <h3 class=”section”>4.52 ircam#
TOC</h3>
Berkeley / IRCAM / CARL Sound Filesystem (BICSF) format muxer. The Berkeley/IRCAM/CARL Sound Format, developed in the 1980s, is a result of the merging of several different earlier sound file formats

and systems including the csound system developed by Dr Gareth Loy at the Computer Audio Research Lab (CARL) at UC San Diego, the IRCAM sound file system developed by Rob Gross and Dan Timis at the Institut de Recherche et Coordination Acoustique / Musique in Paris and the Berkeley Fast Filesystem. It was developed initially as part of the Berkeley/IRCAM/CARL Sound Filesystem, a suite of programs designed to implement a filesystem for audio applications running under Berkeley UNIX. It was particularly popular in academic music research centres, and was used a number of times in the creation of early computer-generated compositions. This muxer accepts a single audio stream containing PCM data. <h3 class="section">4.53 iterm2# TOC</h3> iTerm2 inline image protocol muxer. This muxer writes video frames as OSC 1337 inline images for display in terminals that support the iTerm2 image protocol. Use <samp class="option">-re</samp> to limit the output rate to the source framerate; without it, frames are emitted as fast as they are encoded, which is usually not desired for live display. Frames are sent with the multipart form of the protocol, which splits each image across several short control sequences. This avoids the per-sequence size limit that otherwise discards large frames, and requires iTerm2 3.5 or newer. The output is a self-contained byte stream and can be redirected to a file. Replaying the file with <code class="command">cat</code> displays the images in the terminal. <h4 class="subsection">4.53.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">display_width <var class="var">size</var></samp></dt> <dd> Set the displayed image width. <var class="var">size</var> can be ‘<samp class="samp">auto</samp>’, <var class="var">N</var> terminal cells, <var class="var">N</var>px pixels, or <var class="var">N</var>% of the terminal width. When unset, the terminal derives the width from the image. </dd> <dt><samp class="option">display_height <var class="var">size</var></samp></dt> <dd> Set the displayed image height. <var class="var">size</var> uses the same syntax as <samp class="option">display_width</samp>. When unset, the terminal derives the height from the image. </dd> <dt><samp class="option">keep_aspect <var class="var">bool</var></samp></dt> <dd> Preserve the input aspect ratio when scaling. Default is enabled. </dd> <dt><samp class="option">tmux <var class="var">bool</var></samp></dt> <dd> Wrap image data in tmux DCS passthrough. This requires a tmux version whose passthrough sequence size limit is large enough for image data, with passthrough enabled via <code class="command">tmux set -g allow-passthrough on</code>. Default is disabled. </dd> </dl>

 <h4 class="subsection">4.53.2 Examples# TOC</h4>

Display a video in an iTerm2 terminal: <pre class="example-preformatted">ffmpeg -re -i input.mp4 -f iterm2 - </pre>

Scale the displayed image to 40 terminal cells tall. Inside tmux, enable passthrough first with <code class="command">tmux set -g allow-passthrough on</code>, then add <samp class="option">tmux</samp>: <pre class="example-preformatted">ffmpeg -re -i input.mp4 -f iterm2 -display_height 40 -tmux 1 - </pre>

 <h3 class="section">4.54 jpeg_mpf# TOC</h3> JPEG Multi-Picture Format muxer. Stores multiple JPEG images in a single file using the CIPA DC-007 MP extensions. The first input stream is always the primary image. Every other stream must be specified by a stream group describing what it is; see the <samp class="option">-stream_group</samp> option of the <code class="command">ffmpeg</code> tool. Note: Currently, the only implemented type of stream group is ‘<samp class="samp">gain_map</samp>’, which may be used to store a gain map for JPEG Ultra HDR. <h4 class="subsection">4.54.1 Examples# TOC</h4>

Encode an SDR tone-mapped image and an HDR gain map computed from it: <pre class="example-preformatted">ffmpeg -i input-hdr.png -filter_complex "[0:v] scale=out_transfer=srgb:out_primaries=bt709,split [base][sdr]; [base][0:v] gainmap [gain]" -map "[sdr]" -map "[gain]" -c:v mjpeg -stream_group type=gain_map:st=0:st=1:el_index=1 -f jpeg_mpf output.jpg </pre>

 <h3 class="section">4.55 ivf# TOC</h3> On2 IVF muxer. IVF was developed by On2 Technologies (formerly known as Duck Corporation), to store internally developed codecs. This muxer accepts a single ‘<samp class="samp">vp8</samp>’, ‘<samp class="samp">vp9</samp>’, or ‘<samp class="samp">av1</samp>’ video stream. <h3 class="section">4.56 jacosub<a class="anchor hidden-xs" href="#jacosub" aria-

hidden=”true”># TOC</h3> JACOsub subtitle format muxer. This muxer accepts a single ‘<samp class=”samp”>jacosub</samp>’ subtitles stream. For more information about the format, see http://
  <h3 class=”section”>4.57 ktx# TOC</h3> Khronos Texture 1.0 muxer for ASTC textures. This muxer accepts a single ‘<samp class=”samp”>libastcenc</samp>’ video stream and writes one image: the ASTC block size and the image dimensions are taken from the encoder’s extradata, and a second packet is rejected. The stored row order is recorded in a <code class=”code”>KTXorientation</code> key.
 <h4 class=”subsection”>4.57.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>srgb <var class=”var”>integer</var></samp></dt>
<dd> Select the GL internal format color space. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> Use the color profile published by the encoder, which is the default. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt>
<dd> Write the linear ASTC format. </dd> <dt>‘<samp class=”samp”>srgb</samp>’</dt> <dd> Write the sRGB ASTC format. </dd> </dl>

An explicit value that contradicts a known encoder profile is rejected. A raw <samp class=”file”>.astc</samp> input does not carry a color profile, so remuxing it to <samp class=”file”>.ktx</samp> with <code class=”code”>auto</code> writes the sRGB format; pass <samp class=”option”>-srgb linear</samp> to keep a linear texture. </dd>
</dl>

 <h3 class=”section”>4.58 kvag#
TOC</h3>
Simon & Schuster Interactive VAG muxer. This custom VAG container is used by some Simon & Schuster Interactive games such as "Real War", and "Real War: Rogue States". This muxer accepts a single ‘<samp class=”samp”>adpcm_ima_ssi</samp>’ audio stream.
 <h3 class=”section”>4.59 lc3#
TOC</h3>
Bluetooth SIG Low Complexity Communication Codec audio (LC3), or

ETSI TS 103 634 Low Complexity Communication Codec plus (LC3plus).
This muxer accepts a single ‘<samp class="samp">lc3</samp>’
audio stream. <h3 class="section">4.60
lrc<a class="anchor hidden-xs" href="#lrc" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-lrc" aria-
hidden="true">TOC</h3> LRC lyrics file format muxer.
 LRC (short for LyRiCs) is a computer file format that synchronizes
song lyrics with an audio file, such as MP3, Vorbis, or MIDI. This
muxer accepts a single ‘<samp class="samp">subrip</samp>’
or ‘<samp class="samp">text</samp>’ subtitles stream.
 <h4 class="subsection">4.60.1 Options<span
class="pull-right"><a class="anchor hidden-xs" href="#Options-30" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Options-
30" aria-hidden="true">TOC</h4> <dl class="table">
<dt><samp class="option">precision <var class="var">number</var></samp></dt>
<dd> Set the number of decimal digits used for the fractional (sub-second)
part of the timestamps. Range is <code class="code">1</code> to <code
class="code">6</code>. Default is <code class="code">2</code>, i.e.
centiseconds. </dd> </dl>

 <h4 class="subsection">4.60.2 Metadata<span
class="pull-right"><a class="anchor hidden-xs" href="#Metadata" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Metadata"
aria-hidden="true">TOC</h4> The following meta-
data tags are converted to the format corresponding metadata:
<dl class="table"> <dt><samp class="option">title</samp></dt>
<dt><samp class="option">album</samp></dt> <dt><samp class="option">artist</samp></dt>
<dt><samp class="option">author</samp></dt> <dt><samp class="option">creator</samp></dt>
<dt><samp class="option">encoder</samp></dt> <dt><samp class="option">encoder_version</samp>
</dl>

If ‘<samp class="samp">encoder_version</samp>’ is
not explicitly set, it is automatically set to the libavformat version.
 <h3 class="section">4.61 matroska<span
class="pull-right"><a class="anchor hidden-xs" href="#matroska" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-matroska"
aria-hidden="true">TOC</h3> Matroska container
muxer. This muxer implements the matroska and webm container
specs. <h4 class="subsection">4.61.1
Metadata<a class="anchor hidden-xs" href="#Metadata-
1" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Metadata-1" aria-hidden="true">TOC</h4> The
recognized metadata settings in this muxer are: <dl class="table">
<dt><samp class="option">title</samp></dt> <dd> Set title name
provided to a single track. This gets mapped to the FileDescription el-
ement for a stream written as attachment. </dd> <dt><samp
class="option">language</samp></dt> <dd> Specify the language

of the track in the Matroska languages form. The language can be either the 3 letters bibliographic ISO-639-2 (ISO 639-2/B) form (like "fre" for French), or a language code mixed with a country code for specialities in languages (like "fre-ca" for Canadian French). </dd> <dt><samp class="option">stereo_mode</samp></dt> <dd> Set stereo 3D video layout of two views in a single video track. The <samp class="option">-stereo3d</samp> option overrides existing stereoscopic side data and <code class="code">stereo_mode</code> metadata for the selected output stream. Otherwise, stream-level <code class="code">stereo_mode</code> metadata takes priority over global metadata, and stereoscopic side data is used when neither tag is present. The following values are recognized: <dl class="table"> <dt>‘<samp class="samp">mono</samp>’</dt> <dd> video is not stereo </dd> <dt>‘<samp class="samp">left_right</samp>’</dt> <dd> Both views are arranged side by side, Left-eye view is on the left </dd> <dt>‘<samp class="samp">bottom_top</samp>’</dt> <dd> Both views are arranged in top-bottom orientation, Left-eye view is at bottom </dd> <dt>‘<samp class="samp">top_bottom</samp>’</dt> <dd> Both views are arranged in top-bottom orientation, Left-eye view is on top </dd> <dt>‘<samp class="samp">checkerboard_rl</samp>’</dt> <dd> Each view is arranged in a checkerboard interleaved pattern, Left-eye view being first </dd> <dt>‘<samp class="samp">checkerboard_lr</samp>’</dt> <dd> Each view is arranged in a checkerboard interleaved pattern, Right-eye view being first </dd> <dt>‘<samp class="samp">row_interleaved_rl</samp>’</d <dd> Each view is constituted by a row based interleaving, Right-eye view is first row </dd> <dt>‘<samp class="samp">row_interleaved_lr</samp>’</dt> <dd> Each view is constituted by a row based interleaving, Left-eye view is first row </dd> <dt>‘<samp class="samp">col_interleaved_rl</samp>’</dt> <dd> Both views are arranged in a column based interleaving manner, Right-eye view is first column </dd> <dt>‘<samp class="samp">col_interleaved_lr</samp>’</dt> <dd> Both views are arranged in a column based interleaving manner, Left-eye view is first column </dd> <dt>‘<samp class="samp">anaglyph_cyan_red</samp>’</dt> <dd> All frames are in anaglyph format viewable through red-cyan filters </dd> <dt>‘<samp class="samp">right_left</samp>’</dt> <dd> Both views are arranged side by side, Right-eye view is on the left </dd> <dt>‘<samp class="samp">anaglyph_green_magenta</samp>’</dt> <dd> All frames are in anaglyph format viewable through green-magenta filters </dd> <dt>‘<samp class="samp">block_lr</samp>’</dt> <dd> Both eyes laced in one Block, Left-eye view is first </dd> <dt>‘<samp class="samp">block_rl</samp>’</dt> <dd> Both eyes laced in one Block, Right-eye view is first </dd> </dl> </dd> </dl>

For example a 3D WebM clip can be created using the following command line: <pre class="example-

preformatted”>ffmpeg -i sample_left_right_clip.mpg -an -c:v libvpx -metadata stereo_mode=left_right -y stereo_clip.webm </pre>

 <h4 class=”subsection”>4.61.2 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>reserve_index_space <var class=”var”>size</var></samp></dt> <dd> By default, this muxer writes the index for seeking (called cues in Matroska terms) at the end of the file, because it cannot know in advance how much space to leave for the index at the beginning of the file. However for some use cases – e.g. streaming where seeking is possible but slow – it is useful to put the index at the beginning of the file. If this option is set to a non-zero value, the muxer will reserve <var class=”var”>size</var> bytes of space in the file header and then try to write the cues there when the muxing finishes. If the reserved space does not suffice, no Cues will be written, the file will be finalized and writing the trailer will return an error. A safe size for most use cases should be about 50kB per hour of video. Note that cues are only written if the output is seekable and this option will have no effect if it is not. </dd> <dt><samp class=”option”>cues_to_front <var class=”var”>bool</var></samp></dt> <dd> If set, the muxer will write the index at the beginning of the file by shifting the main data if necessary. This can be combined with reserve_index_space in which case the data is only shifted if the initially reserved space turns out to be insufficient. This option is ignored if the output is unseekable. </dd> <dt><samp class=”option”>cluster_size_limit <var class=”var”>size</var></samp></dt> <dd> Store at most the provided amount of bytes in a cluster. If not specified, the limit is set automatically to a sensible hardcoded fixed value. </dd> <dt><samp class=”option”>cluster_time_limit <var class=”var”>duration</var></samp></dt> <dd> Store at most the provided number of milliseconds in a cluster. If not specified, the limit is set automatically to a sensible hardcoded fixed value. </dd> <dt><samp class=”option”>dash <var class=”var”>bool</var></samp></dt> <dd> Create a WebM file conforming to WebM DASH specification. By default it is set to <code class=”code”>false</code>. </dd> <dt><samp class=”option”>dash_track_number <var class=”var”>index</var></samp></dt> <dd> Track number for the DASH stream. By default it is set to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>live <var class=”var”>bool</var></samp></dt> <dd> Write files assuming it is a live stream. By default it is set to <code class=”code”>false</code>. </dd> <dt><samp class=”option”>allow_raw_vfw <var class=”var”>bool</var></samp></dt> <dd> Allow raw VFW mode. By default it is set to <code class=”code”>false</code>. </dd> <dt><samp class=”option”>flipped_raw_rgb <var class=”var”>bool</var></samp></dt> <dd> If set to <code class=”code”>true</code>, store positive height

for raw RGB bitmaps, which indicates bitmap is stored bottom-up. Note that this option does not flip the bitmap which has to be done manually beforehand, e.g. by using the ‘<samp class="samp">vflip</samp>’ filter. Default is <code class="code">false</code> and indicates bitmap is stored top down. </dd> <dt><samp class="option">write_crc32 <var class="var">bool</var></samp></dt> <dd> Write a CRC32 element inside every Level 1 element. By default it is set to <code class="code">true</code>. This option is ignored for WebM. </dd> <dt><samp class="option">default_mode <var class="var">mode</var></samp></dt> <dd> Control how the FlagDefault of the output tracks will be set. It influences which tracks players should play by default. The default mode is ‘<samp class="samp">passthrough</samp>’. <dl class="table"> <dt>‘<samp class="samp">infer</samp>’</dt> <dd> Every track with disposition default will have the FlagDefault set. Additionally, for each type of track (audio, video or subtitle), if no track with disposition default of this type exists, then the first track of this type will be marked as default (if existing). This ensures that the default flag is set in a sensible way even if the input originated from containers that lack the concept of default tracks. </dd> <dt>‘<samp class="samp">infer_no_subs</samp>’</dt> <dd> This mode is the same as infer except that if no subtitle track with disposition default exists, no subtitle track will be marked as default. </dd> <dt>‘<samp class="samp">passthrough</samp>’</dt> <dd> In this mode the FlagDefault is set if and only if the AV_DISPOSITION_DEFAULT flag is set in the disposition of the corresponding stream. </dd> </dl> </dd> </dl>

 <h3 class="section">4.62 md5# TOC</h3> MD5 testing format. This is a variant of the hash muxer. Unlike that muxer, it defaults to using the MD5 hash function. See also the hash and framemd5 muxers. <h4 class="subsection">4.62.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> To compute the MD5 hash of the input converted to raw audio and video, and store it in the file <samp class="file">out.md5</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f md5 out.md5 </pre>

To print the MD5 hash to stdout: <pre class="example-preformatted">ffmpeg -i INPUT -f md5 - </pre>

 <h3 class=”section”>4.63 mcc# TOC</h3> Muxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-608 and CEA-708), ancillary time code, pan-scan data, etc. <h4 class=”subsection”>4.63.1 Options# TOC</h4>

The muxer options are: <dl class=”table”> <dt><samp class=”option”>override_time_code_rate</samp></dt> <dd> Override the <code class=”code”>Time Code Rate</code> value in the output. Defaults to trying to deduce from the stream’s <code class=”code”>time_base</code>, which often doesn’t work. </dd> <dt><samp class=”option”>use_u_alias</samp></dt> <dd> Use the <code class=”code”>U</code> alias for the byte sequence <code class=”code”>E1h 00h 00h 00h</code>. Disabled by default because some <samp class=”file”>.mcc</samp> files disagree on whether it has 2 or 3 zero bytes. </dd> <dt><samp class=”option”>mcc_version</samp></dt> <dd> The MCC file format version. Must be either 1 or 2, defaults to 2. </dd> <dt><samp class=”option”>creation_program</samp></dt> <dd> The creation program. Defaults to this version of FFmpeg. </dd> <dt><samp class=”option”>creation_time</samp></dt> <dd> The creation time. Defaults to the current time. </dd> </dl>

 <h4 class=”subsection”>4.63.2 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Extract a MXF <code class=”code”>SMPTE_436M_ANC</code> stream from a MXF file and write it to a MCC file at 30 fps. <pre class=”example-preformatted”>ffmpeg -i input.mxf -c copy -map 0:d -override_time_code_rate 30 out.mcc </pre>

Extract EIA-608/CTA-708 closed captions from a <samp class=”file”>.mp4</samp> file and write them to a MCC file at 29.97 fps. <pre class=”example-preformatted”>ffmpeg -f lavfi -i "movie=input.mp4[out+subcc]" -c:s copy -map 0:s -override_time_code_rate 30000/1001 out.mcc </pre>

 <h3 class=”section”>4.64 microdvd# TOC</h3> MicroDVD subtitle

format muxer. This muxer accepts a single ‘<samp class="samp">microdvd</samp>’ subtitles stream. <h3 class="section">4.65 mmf# TOC</h3> Synthetic music Mobile Application Format (SMAF) format muxer. SMAF is a music data format specified by Yamaha for portable electronic devices, such as mobile phones and personal digital assistants. This muxer accepts a single ‘<samp class="samp">adpcm_yamaha</samp>’ audio stream. <h3 class="section">4.66 mp3# TOC</h3>

The MP3 muxer writes a raw MP3 stream with the following optional features: <ul class="itemize mark-bullet"> An ID3v2 metadata header at the beginning (enabled by default). Versions 2.3 and 2.4 are supported, the <code class="code">id3v2_version</code> private option controls which one is used (3 or 4). Setting <code class="code">id3v2_version</code> to 0 disables the ID3v2 header completely.

The muxer supports writing attached pictures (APIC frames) to the ID3v2 header. The pictures are supplied to the muxer in form of a video stream with a single packet. There can be any number of those streams, each will correspond to a single APIC frame. The stream metadata tags <var class="var">title</var> and <var class="var">comment</var> map to APIC <var class="var">description</var> and <var class="var">picture type</var> respectively. See http://id3.org/id3v2.4.0-frames for allowed picture types. Note that the APIC frames must be written at the beginning, so the muxer will buffer the audio frames until it gets all the pictures. It is therefore advised to provide the pictures as soon as possible to avoid excessive buffering. A Xing/LAME frame right after the ID3v2 header (if present). It is enabled by default, but will be written only if the output is seekable. The <code class="code">write_xing</code> private option can be used to disable it. The frame contains various information that may be useful to the decoder, like the audio duration or encoder delay.

A legacy ID3v1 tag at the end of the file (disabled by default). It may be enabled with the <code class="code">write_id3v1</code> private option, but as its capabilities are very limited, its usage is not recommended.

Examples: Write an mp3 with an ID3v2.3 header and an ID3v1 footer: <pre class="example-preformatted">ffmpeg -i INPUT -id3v2_version 3 -write_id3v1 1 out.mp3 </pre>

To attach a picture to an mp3 file select both the audio and the picture stream with <code class=”code”>map</code>: <pre class=”example-preformatted”>ffmpeg -i input.mp3 -i cover.png -c copy -map 0 -map 1 -metadata:s:v title="Album cover" -metadata:s:v comment="Cover (Front)" out.mp3 </pre>

Write a "clean" MP3 without any extra features: <pre class=”example-preformatted”>ffmpeg -i input.wav -write_xing 0 -id3v2_version 0 out.mp3 </pre>

 <h3 class=”section”>4.67 mpegts# TOC</h3>

MPEG transport stream muxer. This muxer implements ISO 13818-1 and part of ETSI EN 300 468. The recognized metadata settings in mpegts muxer are <code class=”code”>service_provider</code> and <code class=”code”>service_name</code>. If they are not set the default for <code class=”code”>service_provider</code> is ‘<samp class=”samp”>FFmpeg</samp>’ and the default for <code class=”code”>service_name</code> is ‘<samp class=”samp”>Service01</samp>’. <h4 class=”subsection”>4.67.1 Options# TOC</h4>

The muxer options are: <dl class=”table”> <dt><samp class=”option”>mpegts_transport_stream_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>transport_stream_id</samp>’. This identifies a transponder in DVB. Default is <code class=”code”>0x0001</code>. </dd> <dt><samp class=”option”>mpegts_original_network_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>original_network_id</samp>’. This is unique identifier of a network in DVB. Its main use is in the unique identification of a service through the path ‘<samp class=”samp”>Original_Network_ID, Transport_Stream_ID</samp>’. Default is <code class=”code”>0xff01</code>. </dd> <dt><samp class=”option”>mpegts_service_id <var class=”var”>integer</var></samp></dt> <dd> Set the ‘<samp class=”samp”>service_id</samp>’, also known as program in DVB. Default is <code class=”code”>0x0001</code>. </dd> <dt><samp class=”option”>mpegts_service_type <var class=”var”>integer</var></samp></dt> <dd> Set the program ‘<samp class=”samp”>service_type</samp>’. Default is <code class=”code”>digital_tv</code>. Accepts the following options: <dl class=”table”> <dt>‘<samp class=”samp”>hex_value</samp>’</dt> <dd> Any hexadecimal value between <code class=”code”>0x01</code> and <code class=”code”>0xff</code> as defined in ETSI 300 468. </dd> <dt>‘<samp class=”samp”>digital_tv</samp>’</dt>

<dd> Digital TV service. </dd> <dt>‘<samp class="samp">digital_radio</samp>&rsquo
<dd> Digital Radio service. </dd> <dt>‘<samp
class="samp">teletext</samp>’</dt> <dd> Teletext service.
</dd> <dt>‘<samp class="samp">advanced_codec_digital_radio</samp>’</dt>
<dd> Advanced Codec Digital Radio service. </dd> <dt>‘<samp
class="samp">mpeg2_digital_hdtv</samp>’</dt> <dd> MPEG2
Digital HDTV service. </dd> <dt>‘<samp class="samp">advanced_codec_digital_sdtv</sam
<dd> Advanced Codec Digital SDTV service. </dd> <dt>‘<samp
class="samp">advanced_codec_digital_hdtv</samp>’</dt>
<dd> Advanced Codec Digital HDTV service. </dd> <dt>‘<samp
class="samp">hevc_digital_hdtv</samp>’</dt> <dd> HEVC
Digital Television service. </dd> </dl>

</dd> <dt><samp class="option">mpegts_pmt_start_pid <var class="var">integer</var></samp></d
<dd> Set the first PID for PMTs. Default is <code class="code">0x1000</code>,
minimum is <code class="code">0x0020</code>, maximum is <code
class="code">0x1ffa</code>. This option has no effect in m2ts mode
where the PMT PID is fixed <code class="code">0x0100</code>.
 </dd> <dt><samp class="option">mpegts_start_pid <var
class="var">integer</var></samp></dt> <dd> Set the first PID
for elementary streams. Default is <code class="code">0x0100</code>,
minimum is <code class="code">0x0020</code>, maximum is <code
class="code">0x1ffa</code>. This option has no effect in m2ts mode
where the elementary stream PIDs are fixed. </dd> <dt><samp
class="option">mpegts_m2ts_mode <var class="var">boolean</var></samp></dt>
<dd> Enable m2ts mode if set to <code class="code">1</code>. De-
fault value is <code class="code">-1</code> which disables m2ts mode.
</dd> <dt><samp class="option">muxrate <var class="var">integer</var></samp></dt>
<dd> Set a constant muxrate. Default is VBR. </dd> <dt><samp
class="option">pes_payload_size <var class="var">integer</var></samp></dt>
<dd> Set minimum PES packet payload in bytes. Default is <code
class="code">2930</code>. </dd> <dt><samp class="option">mpegts_flags
<var class="var">flags</var></samp></dt> <dd> Set mpegts flags.
Accepts the following options: <dl class="table"> <dt>‘<samp
class="samp">resend_headers</samp>’</dt> <dd> Reemit
PAT/PMT before writing the next packet. </dd> <dt>‘<samp
class="samp">latm</samp>’</dt> <dd> Use LATM packetiza-
tion for AAC. </dd> <dt>‘<samp class="samp">pat_pmt_at_frames</samp>’</dt>
<dd> Reemit PAT and PMT at each video frame. </dd>
<dt>‘<samp class="samp">system_b</samp>’</dt>
<dd> Conform to System B (DVB) instead of System A (ATSC).
</dd> <dt>‘<samp class="samp">initial_discontinuity</samp>’</dt>
<dd> Mark the initial packet of each stream as discontinuity. </dd>
<dt>‘<samp class="samp">nit</samp>’</dt> <dd> Emit
NIT table. </dd> <dt>‘<samp class="samp">omit_rai</samp>’</dt>
<dd> Disable writing of random access indicator. </dd> </dl>

</dd> <dt><samp class="option">mpegts_copyts <var class="var">boolean</var></samp></dt> <dd> Preserve original timestamps, if value is set to <code class="code">1</code>. Default value is <code class="code">-1</code>, which results in shifting timestamps so that they start from 0. </dd> <dt><samp class="option">omit_video_pes_length <var class="var">boolean</var></samp></dt> <dd> Omit the PES packet length for video packets. Default is <code class="code">1</code> (true). </dd> <dt><samp class="option">pcr_period <var class="var">integer</var></samp></dt> <dd> Override the default PCR retransmission time in milliseconds. Default is <code class="code">-1</code> which means that the PCR interval will be determined automatically: 20 ms is used for CBR streams, the highest multiple of the frame duration which is less than 100 ms is used for VBR streams. </dd> <dt><samp class="option">mpegts_pcr_pid <var class="var">integer</var></samp></dt> <dd> Set a separate PID for PCR packets. When set to a valid PID value, PCR is carried in adaptation-field-only packets on the specified PID, separate from the elementary stream PIDs. This is required for example by ARIB TR-B14. Default is <code class="code">-1</code> which means PCR is placed on the elementary stream PIDs. </dd> <dt><samp class="option">pat_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between PAT/PMT tables. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">sdt_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between SDT tables. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">nit_period <var class="var">duration</var></samp></dt> <dd> Maximum time in seconds between NIT tables. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">tables_version <var class="var">integer</var></samp></dt> <dd> Set PAT, PMT, SDT and NIT version (default <code class="code">0</code>, valid values are from 0 to 31, inclusively). This option allows updating stream structure so that standard consumer may detect the change. To do so, reopen output <code class="code">AVFormatContext</code> (in case of API usage) or restart <code class="command">ffmpeg</code> instance, cyclically changing <samp class="option">tables_version</samp> value: <pre class="example-preformatted">ffmpeg -i source1.ts -codec copy -f mpegts -tables_version 0 udp://1.1.1.1:1111 ffmpeg -i source2.ts -codec copy -f mpegts -tables_version 1 udp://1.1.1.1:1111 ... ffmpeg -i source3.ts -codec copy -f mpegts -tables_version 31 udp://1.1.1.1:1111 ffmpeg -i source1.ts -codec copy -f mpegts -tables_version 0 udp://1.1.1.1:1111 ffmpeg -i source2.ts -codec copy -f mpegts -tables_version 1 udp://1.1.1.1:1111 ... </pre> </dd> </dl>

 <h4 class="subsection">4.67.2 Example# TOC</h4>

 <pre class="example-preformatted">ffmpeg -i file.mpg

-c copy \ -mpegts_original_network_id 0x1122 \ -mpegts_transport_stream_id 0x3344 \ -mpegts_service_id 0x5566 \ -mpegts_pmt_start_pid 0x1500 \ -mpegts_start_pid 0x150 \ -metadata service_provider="Some provider" \ -metadata service_name="Some Channel" \ out.ts </pre>

 <h3 class="section">4.68 mxf, mxf_d10, mxf_opatom# TOC</

MXF muxer. <h4 class="subsection">4.68.1 Options# TOC</h4>

The muxer options are: <dl class="table"> <dt><samp class="option">store_user_comments <var class="var">bool</var></samp></dt> <dd> Set if user comments should be stored if available or never. IRT D-10 does not allow user comments. The default is thus to write them for mxf and mxf_opatom but not for mxf_d10 </dd> </dl>

 <h3 class="section">4.69 null# TOC</h3>

Null muxer. This muxer does not generate any output file, it is mainly useful for testing or benchmarking purposes. For example to benchmark decoding with <code class="command">ffmpeg</code> you can use the command: <pre class="example-preformatted">ffmpeg -benchmark -i INPUT -f null out.null </pre>

Note that the above command does not read or write the <samp class="file">out.null</samp> file, but specifying the output file is required by the <code class="command">ffmpeg</code> syntax. Alternatively you can write the command as: <pre class="example-preformatted">ffmpeg -benchmark -i INPUT -f null - </pre>

 <h3 class="section">4.70 nut# TOC</h3>

<dl class="table"> <dt><samp class="option">-syncpoints <var class="var">flags</var></samp></dt> <dd> Change the syncpoint usage in nut: <dl class="table"> <dt><samp class="option"><var class="var">default</var> use the normal low-overhead seeking aids.</samp></dt> <dt><samp class="option"><var class="var">none</var> do not use the syncpoints at all, reducing the overhead but making the stream non-seekable;</samp></dt> <dd> Use

of this option is not recommended, as the resulting files are very damage sensitive and seeking is not possible. Also in general the overhead from syncpoints is negligible. Note, -<code class="code">write_index</code> 0 can be used to disable all growing data tables, allowing to mux endless streams with limited memory and without these disadvantages. </dd> <dt><samp class="option"><var class="var">timestamped</var> extend the syncpoint with a wallclock field.</samp></dt> </dl> The <var class="var">none</var> and <var class="var">timestamped</var> flags are experimental. </dd> <dt><samp class="option">-write_index <var class="var">bool</var></samp></dt> <dd> Write index at the end, the default is to write an index. </dd> </dl>

 <pre class="example-preformatted">ffmpeg -i INPUT -f_strict experimental -syncpoints none - | processor </pre>

 <h3 class="section">4.71 ogg# TOC</h3>

Ogg container muxer. <dl class="table"> <dt><samp class="option">-page_duration <var class="var">duration</var></samp></dt> <dd> Preferred page duration, in microseconds. The muxer will attempt to create pages that are approximately <var class="var">duration</var> microseconds long. This allows the user to compromise between seek granularity and container overhead. The default is 1 second. A value of 0 will fill all segments, making pages as large as possible. A value of 1 will effectively use 1 packet-per-page in most situations, giving a small seek granularity at the cost of additional container overhead. </dd> <dt><samp class="option">-serial_offset <var class="var">value</var></samp></dt> <dd> Serial number of the first stream in bitexact mode, the others following consecutively. Chained links need distinct serial numbers, which disjoint ranges of values guarantee. The default of -1 derives each serial number from the stream’s headers instead. This makes links with different headers or tags unlikely to share a serial number, without ruling it out. Links with identical headers and tags always share one, which separately muxed outputs avoid with explicit offsets. A link the muxer starts itself on new stream metadata keeps the offset of the link before it. With an explicit offset it then reuses its serial number. With the default it does so when its headers and tags are unchanged. </dd> </dl>

 <h3 class="section">4.72 pdv# TOC</h3>

Playdate Video muxer. This muxer writes the Playdate video container used by Panic’s Playdate SDK. It requires a seekable output and a single PDV video stream. <dl class="table"> <dt><samp class="option">max_frames <var class="var">frames</var></samp></dt>

<dd> Reserve space for at most <var class="var">frames</var> video frames in the file header. This option is mandatory. </dd> </dl>

 <h3 class="section">4.73 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. It can be used to archive the original extracted CC bitstream and to produce a source file for later processing or conversion. The format allows for interoperability between ccextractor and FFmpeg, is simple to parse, and can be used to create a backup of the CC presentation. This muxer implements the specification as of March 2024, which has been stable and unchanged since April 2014. This muxer will have some nuances from the way that ccextractor muxes RCWT. No compatibility issues when processing the output with ccextractor have been observed as a result of this so far, but mileage may vary and outputs will not be a bit-exact match. A free specification of RCWT can be found here: <a class="url" href="https://github.com/CCExtractor/ccextractor/blob/master/docs/BINARY_FILE_FORMAT.TXT">h
  <h4 class="subsection">4.73.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract Closed Captions to RCWT using lavfi: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=INPUT.mkv[out+subcc]" -map 0:s:0 -c:s copy -f rcwt CC.rcwt.bin </pre>

 <h3 class="section">4.74 segment, stream_segment, ssegment# TOC</h3>

Basic stream segmenter. This muxer outputs streams to a number of separate files of nearly fixed duration. Output filename pattern can be set in a fashion similar to image2, or by using a <code class="code">strftime</code> template if the <samp class="option">strftime</samp> option is enabled. <code class="code">stream_segment</code> is a variant of the muxer used to write to streaming output formats, i.e. which do not require global headers, and is recommended for outputting e.g. to MPEG transport stream segments. <code class="code">ssegment</code> is a shorter alias for <code class="code">stream_segment</code>. Every segment starts

with a keyframe of the selected reference stream, which is set through the <samp class="option">reference_stream</samp> option. Note that if you want accurate splitting for a video file, you need to make the input key frames correspond to the exact splitting times expected by the segmenter, or the segment muxer will start the new segment with the key frame found next after the specified start time. The segment muxer works best with a single constant frame rate video. Optionally it can generate a list of the created segments, by setting the option <var class="var">segment_list</var>. The list type is specified by the <var class="var">segment_list_type</var> option. The entry filenames in the segment list are set by default to the basename of the corresponding segment files. See also the hls muxer, which provides a more specific implementation for HLS segmentation. <h4 class="subsection">4.74.1 Options# TOC</h4>

The segment muxer supports the following options: <dl class="table"> <dt><samp class="option">increment_tc <var class="var">1|0</var></samp></dt> <dd> if set to <code class="code">1</code>, increment timecode between each segment If this is selected, the input need to have a timecode in the first video stream. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">reference_stream <var class="var">specifier</var></samp></dt> <dd> Set the reference stream, as specified by the string <var class="var">specifier</var>. If <var class="var">specifier</var> is set to <code class="code">auto</code>, the reference is chosen automatically. Otherwise it must be a stream specifier (see the “Stream specifiers” chapter in the ffmpeg manual) which specifies the reference stream. The default value is <code class="code">auto</code>. </dd> <dt><samp class="option">segment_format <var class="var">format</var></samp></dt> <dd> Override the inner container format, by default it is guessed by the filename extension. </dd> <dt><samp class="option">segment_format_options <var class="var">options_list</var></samp></dt> <dd> Set output format options using a :-separated list of key=value parameters. Values containing the <code class="code">:</code> special character must be escaped. </dd> <dt><samp class="option">segment_list <var class="var">name</var></samp></dt> <dd> Generate also a listfile named <var class="var">name</var>. If not specified no listfile is generated. </dd> <dt><samp class="option">segment_list_flags <var class="var">flags</var></samp></dt> <dd> Set flags affecting the segment list generation. It currently supports the following flags: <dl class="table"> <dt>‘<samp class="samp">cache</samp>’</dt> <dd> Allow caching (only affects M3U8 list files). </dd> <dt>‘<samp class="samp">live</samp>’</dt> <dd> Allow live-friendly file generation. </dd> </dl>

</dd> <dt><samp class="option">segment_list_size <var class="var">size</var></samp></dt>
<dd> Update the list file so that it contains at most <var class="var">size</var>
segments. If 0 the list file will contain all the segments. Default value is 0.
 </dd> <dt><samp class="option">segment_list_entry_prefix
<var class="var">prefix</var></samp></dt> <dd> Prepend <var
class="var">prefix</var> to each entry. Useful to generate absolute
paths. By default no prefix is applied. </dd> <dt><samp
class="option">segment_list_type <var class="var">type</var></samp></dt>
<dd> Select the listing format. The following values are recog-
nized: <dl class="table"> <dt>‘<samp class="samp">flat</samp>’</dt>
<dd> Generate a flat list for the created segments, one segment per line.
 </dd> <dt>‘<samp class="samp">csv, ext</samp>’</dt>
<dd> Generate a list for the created segments, one segment per line,
each line matching the format (comma-separated values): <div
class="example"> <pre class="example-preformatted"><var class="var">segment_filename</var>,<var
class="var">segment_start_time</var>,<var class="var">segment_end_time</var>
</pre>

<var class="var">segment_filename</var> is the name of the
output file generated by the muxer according to the provided pat-
tern. CSV escaping (according to RFC4180) is applied if required.
 <var class="var">segment_start_time</var> and <var
class="var">segment_end_time</var> specify the segment start and
end time expressed in seconds. A list file with the suffix <code
class="code">".csv"</code> or <code class="code">".ext"</code>
will auto-select this format. ‘<samp class="samp">ext</samp>’
is deprecated in favor or ‘<samp class="samp">csv</samp>’.
 </dd> <dt>‘<samp class="samp">ffconcat</samp>’</dt>
<dd> Generate an ffconcat file for the created segments. The resulting file
can be read using the FFmpeg concat de-
muxer. A list file with the suffix <code class="code">".ffcat"</code>
or <code class="code">".ffconcat"</code> will auto-select this for-
mat. </dd> <dt>‘<samp class="samp">m3u8</samp>’</dt>
<dd> Generate an extended M3U8 file, version 3, compliant with <a
class="url" href="http://tools.ietf.org/id/draft-pantos-http-live-streaming">http://tools.ietf.org/id/draft-
pantos-http-live-streaming. A list file with the suffix <code
class="code">".m3u8"</code> will auto-select this format.
</dd> </dl>

If not specified the type is guessed from the list file name suf-
fix. </dd> <dt><samp class="option">segment_time <var
class="var">time</var></samp></dt> <dd> Set segment dura-
tion to <var class="var">time</var>, the value must be a duration
specification. Default value is "2". See also the <samp
class="option">segment_times</samp> option. Note that
splitting may not be accurate, unless you force the reference stream key-
frames at the given time. See the introductory notice and the examples

below. </dd> <dt><samp class=”option”>min_seg_duration <var class=”var”>time</var></samp></dt> <dd> Set minimum segment duration to <var class=”var”>time</var>, the value must be a duration specification. This prevents the muxer ending segments at a duration below this value. Only effective with <code class=”code”>segment_time</code>. Default value is "0". </dd> <dt><samp class=”option”>segment_atclocktime <var class=”var”>1|0</var></samp></dt> <dd> If set to "1" split at regular clock time intervals starting from 00:00 o’clock. The <var class=”var”>time</var> value specified in <samp class=”option”>segment_time</samp> is used for setting the length of the splitting interval. For example with <samp class=”option”>segment_time</samp> set to "900" this makes it possible to create files at 12:00 o’clock, 12:15, 12:30, etc. Default value is "0". </dd> <dt><samp class=”option”>segment_clocktime_offset <var class=”var”>duration</var></samp></dt> <dd> Delay the segment splitting times with the specified duration when using <samp class=”option”>segment_atclocktime</samp>. For example with <samp class=”option”>segment_time</samp> set to "900" and <samp class=”option”>segment_clocktime_offset</samp> set to "300" this makes it possible to create files at 12:05, 12:20, 12:35, etc. Default value is "0". </dd> <dt><samp class=”option”>segment_clocktime_wrap_duration <var class=”var”>duration</var></samp></dt> <dd> Force the segmenter to only start a new segment if a packet reaches the muxer within the specified duration after the segmenting clock time. This way you can make the segmenter more resilient to backward local time jumps, such as leap seconds or transition to standard time from daylight savings time. Default is the maximum possible duration which means starting a new segment regardless of the elapsed time since the last clock time. </dd> <dt><samp class=”option”>segment_time_delta <var class=”var”>delta</var></samp></dt> <dd> Specify the accuracy time when selecting the start time for a segment, expressed as a duration specification. Default value is "0". When delta is specified a key-frame will start a new segment if its PTS satisfies the relation: <pre class=”example-preformatted”>PTS >= start_time - time_delta </pre>

This option is useful when splitting video content, which is always split at GOP boundaries, in case a key frame is found just before the specified split time. In particular may be used in combination with the <samp class=”file”>ffmpeg</samp> option <var class=”var”>force_key_frames</var>. The key frame times specified by <var class=”var”>force_key_frames</var> may not be set accurately because of rounding issues, with the consequence that a key frame time may result set just before the specified time. For constant frame rate videos a value of 1/(2*<var class=”var”>frame_rate</var>) should ad-

dress the worst case mismatch between the specified time and the time set by <var class="var">force_key_frames</var>. </dd> <dt><samp class="option">segment_times <var class="var">times</var></samp></dt> <dd> Specify a list of split points. <var class="var">times</var> contains a list of comma separated duration specifications, in increasing order. See also the <samp class="option">segment_time</samp> option. </dd> <dt><samp class="option">segment_frames <var class="var">frames</var></samp></dt> <dd> Specify a list of split video frame numbers. <var class="var">frames</var> contains a list of comma separated integer numbers, in increasing order. This option specifies to start a new segment whenever a reference stream key frame is found and the sequential number (starting from 0) of the frame is greater or equal to the next value in the list. </dd> <dt><samp class="option">segment_wrap <var class="var">limit</var></samp></dt> <dd> Wrap around segment index once it reaches <var class="var">limit</var>. </dd> <dt><samp class="option">segment_start_number <var class="var">number</var></samp></dt> <dd> Set the sequence number of the first segment. Defaults to <code class="code">0</code>. </dd> <dt><samp class="option">segment_wrap_number <var class="var">number</var></samp></dt> <dd> Set the number of times the segment index is assumed to have already wrapped when generating the indexes reported in the segment list. Together with <samp class="option">segment_wrap</samp> and <samp class="option">segment_start_number</samp> it offsets the reported index, which is computed as <code class="code">segment_start_number + segment_wrap * segment_wrap_number</code>. Defaults to <code class="code">0</code>. </dd> <dt><samp class="option">strftime <var class="var">1|0</var></samp></dt> <dd> Use the <code class="code">strftime</code> function to define the name of the new segments to write. If this is selected, the output segment name must contain a <code class="code">strftime</code> function template. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">break_non_keyframes <var class="var">1|0</var></samp></dt> <dd> If enabled, allow segments to start on frames other than keyframes. This improves behavior on some players when the time between keyframes is inconsistent, but may make things worse on others, and can cause some oddities during seeking. Defaults to <code class="code">0</code>. </dd> <dt><samp class="option">reset_timestamps <var class="var">1|0</var></samp></dt> <dd> Reset timestamps at the beginning of each segment, so that each segment will start with near-zero timestamps. It is meant to ease the playback of the generated segments. May not work with some combinations of muxers/codecs. It is set to <code class="code">0</code> by default. </dd> <dt><samp class="option">initial_offset <var class="var">offset</var></samp></dt> <dd> Specify timestamp offset to apply to the output packet timestamps. The argument must be a time duration specification, and defaults to 0. </dd> <dt><samp class="option">write_empty_segments <var class="var">1|0</var></samp></dt>

<dd> If enabled, write an empty segment if there are no packets during the period a segment would usually span. Otherwise, the segment will be filled with the next packet written. Defaults to <code class="code">0</code>. </dd> <dt><samp class="option">write_header_trailer <var class="var">bool</var></samp></dt> <dd> Write a header to the first segment and a trailer to the last one, instead of writing a header and a trailer to every individual segment. Disabling it (<code class="code">false</code>) also forces <samp class="option">individual_header_trailer</samp> to <code class="code">false</code>, so that no segment is given a header or trailer. Default value is <code class="code">true</code>. </dd> <dt><samp class="option">individual_header_trailer <var class="var">bool</var></samp></dt> <dd> If enabled, write a complete header and trailer to every segment, making each segment an independently usable file. If disabled (<code class="code">false</code>), only the first segment is given a header and only the last one a trailer. It is forced to <code class="code">false</code> when <samp class="option">write_header_trailer</samp> is <code class="code">false</code> or when <samp class="option">segment_header_filename</samp> is set. Default value is <code class="code">true</code>. </dd> <dt><samp class="option">segment_header_filename <var class="var">name</var></samp></dt> <dd> Write the stream header to a separate file named <var class="var">name</var> instead of storing it in the segments. Setting it forces <samp class="option">write_header_trailer</samp> to <code class="code">true</code> and <samp class="option">individual_header_trailer</samp> to <code class="code">false</code>, so the header is written only once, to <var class="var">name</var>. Unset by default. </dd> </dl>

Make sure to require a closed GOP when encoding and to set the GOP size to fit your segment time constraint. <h4 class="subsection">4.74.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Remux the content of file <samp class="file">in.mkv</samp> to a list of segments <samp class="file">out-000.nut</samp>, <samp class="file">out-001.nut</samp>, etc., and write the list of generated segments to <samp class="file">out.list</samp>: <pre class="example-preformatted">ffmpeg -i in.mkv -codec hevc -flags +cgop -g 60 -map 0 -f segment -segment_list out.list out%03d.nut </pre>

Segment input and set output format options for the output segments: <pre class="example-preformatted">ffmpeg -i in.mkv -f segment -segment_time 10 -segment_format_options movflags=+faststart out%03d.mp4 </pre>

Segment the input file according to the split points specified by the <var class="var">segment_times</var> option: <pre class="example-preformatted">ffmpeg -i in.mkv -codec copy -map 0 -f

segment -segment_list out.csv -segment_times 1,2,3,5,8,13,21 out%03d.nut </pre>

Use the <code class="command">ffmpeg</code> <samp class="option">force_key_frames</samp> option to force key frames in the input at the specified location, together with the segment option <samp class="option">segment_time_delta</samp> to account for possible roundings operated when setting key frame times. <pre class="example-preformatted">ffmpeg -i in.mkv -force_key_frames 1,2,3,5,8,13,21 -codec:v mpeg4 -codec:a pcm_s16le -map 0 \ -f segment -segment_list out.csv -segment_times 1,2,3,5,8,13,21 -segment_time_delta 0.05 out%03d.nut </pre> In order to force key frames on the input file, transcoding is required. Segment the input file by splitting the input file according to the frame numbers sequence specified with the <samp class="option">segment_frames</samp> option: <pre class="example-preformatted">ffmpeg -i in.mkv -codec copy -map 0 -f segment -segment_list out.csv -segment_frames 100,200,300,500,800 out%03d.nut </pre>

Convert the <samp class="file">in.mkv</samp> to TS segments using the <code class="code">libx264</code> and <code class="code">aac</code> encoders: <pre class="example-preformatted">ffmpeg -i in.mkv -map 0 -codec:v libx264 -codec:a aac -f ssegment -segment_list out.list out%03d.ts </pre>

Segment the input file, and create an M3U8 live playlist (can be used as live HLS source): <pre class="example-preformatted">ffmpeg -re -i in.mkv -codec copy -map 0 -f segment -segment_list playlist.m3u8 \ -segment_list_flags +live -segment_time 10 out%03d.mkv </pre>

 <h3 class="section">4.75 smoothstreaming# TOC</h3>

Smooth Streaming muxer generates a set of files (Manifest, chunks) suitable for serving with conventional web server. <dl class="table"> <dt><samp class="option">window_size</samp></dt> <dd> Specify the number of fragments kept in the manifest. Default 0 (keep all). </dd> <dt><samp class="option">extra_window_size</samp></dt> <dd> Specify the number of fragments kept outside of the manifest before removing from disk. Default 5. </dd> <dt><samp class="option">lookahead_count</samp></dt> <dd> Specify the number of lookahead fragments. Default 2. </dd> <dt><samp class="option">min_frag_duration</samp></dt> <dd> Specify the minimum fragment duration (in microseconds). Default 5000000. </dd> <dt><samp class="option">remove_at_exit</samp></dt>

<dd> Specify whether to remove all fragments when finished. Default 0 (do not remove). </dd> </dl>

 <h3 class="section">4.76 streamhash# TOC</h3>

Per stream hash testing format. This muxer computes and prints a cryptographic hash of all the input frames, on a per-stream basis. This can be used for equality checks without having to do a complete binary comparison. By default audio frames are converted to signed 16-bit raw audio and video frames to raw video before computing the hash, but the output of explicit conversions to other codecs can also be used. Timestamps are ignored. It uses the SHA-256 cryptographic hash function by default, but supports several other algorithms. The output of the muxer consists of one line per stream of the form: <var class="var">streamindex</var>,<var class="var">streamtype</var>,<var class="var">algo</var>=<var class="var">hash</var>, where <var class="var">streamindex</var> is the index of the mapped stream, <var class="var">streamtype</var> is a single character indicating the type of stream, <var class="var">algo</var> is a short string representing the hash function used, and <var class="var">hash</var> is a hexadecimal number representing the computed hash. <dl class="table"> <dt><samp class="option">hash <var class="var">algorithm</var></samp></dt> <dd> Use the cryptographic hash function specified by the string <var class="var">algorithm</var>. Supported values include <code class="code">MD5</code>, <code class="code">murmur3</code>, <code class="code">RIPEMD128</code>, <code class="code">RIPEMD160</code>, <code class="code">RIPEMD256</code>, <code class="code">RIPEMD320</code>, <code class="code">SHA160</code>, <code class="code">SHA224</code>, <code class="code">SHA256</code> (default), <code class="code">SHA512/224</code>, <code class="code">SHA512/256</code>, <code class="code">SHA384</code>, <code class="code">SHA512</code>, <code class="code">CRC32</code> and <code class="code">adler32</code>. </dd> </dl>

 <h4 class="subsection">4.76.1 Examples# TOC</h4>

To compute the SHA-256 hash of the input converted to raw audio and video, and store it in the file <samp class="file">out.sha256</samp>: <pre class="example-preformatted">ffmpeg -i INPUT -f streamhash out.sha256 </pre>

To print an MD5 hash to stdout use the command: <pre class="example-preformatted">ffmpeg -i INPUT

-f streamhash -hash md5 - </pre>

See also the hash and framehash muxers. <h3 class="section">4.77 tee# TOC</h3>

The tee muxer can be used to write the same data to several outputs, such as files or streams. It can be used, for example, to stream a video over a network and save it to disk at the same time. It is different from specifying several outputs to the <code class="command">ffmpeg</code> command-line tool. With the tee muxer, the audio and video data will be encoded only once. With conventional multiple outputs, multiple encoding operations in parallel are initiated, which can be a very expensive process. The tee muxer is not useful when using the libavformat API directly because it is then possible to feed the same packets to several muxers directly. Since the tee muxer does not represent any particular output format, ffmpeg cannot auto-select output streams. So all streams intended for output must be specified using <code class="code">-map</code>. See the examples below. Some encoders may need different options depending on the output format; the auto-detection of this can not work with the tee muxer, so they need to be explicitly specified. The main example is the <samp class="option">global_header</samp> flag. The slave outputs are specified in the file name given to the muxer, separated by ’|’. If any of the slave name contains the ’|’ separator, leading or trailing spaces or any special character, those must be escaped (see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual). <h4 class="subsection">4.77.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">use_fifo <var class="var">bool</var></samp></dt> <dd> If set to 1, slave outputs will be processed in separate threads using the fifo muxer. This allows to compensate for different speed/latency/reliability of outputs and setup transparent recovery. By default this feature is turned off. </dd> <dt><samp class="option">fifo_options</samp></dt> <dd> Options to pass to fifo pseudo-muxer instances. See fifo. </dd> </dl>

Muxer options can be specified for each slave by prepending them as a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ’:’, between square brackets. If the options values contain a special character or the ’:’ separator, they must be

escaped; note that this is a second level escaping. The following special options are also recognized: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Specify the format name. Required if it cannot be guessed from the output URL. </dd> <dt><samp class="option">bsfs[/<var class="var">spec</var>]</samp></dt> <dd> Specify a list of bitstream filters to apply to the specified output. It is possible to specify to which streams a given bitstream filter applies, by appending a stream specifier to the option separated by <code class="code">/</code>. <var class="var">spec</var> must be a stream specifier (see Format stream specifiers). If the stream specifier is not specified, the bitstream filters will be applied to all streams in the output. This will cause that output operation to fail if the output contains streams to which the bitstream filter cannot be applied e.g. <code class="code">h264_mp4toannexb</code> being applied to an output containing an audio stream. Options for a bitstream filter must be specified in the form of <code class="code">opt=value</code>. Several bitstream filters can be specified, separated by ",". </dd> <dt><samp class="option">use_fifo <var class="var">bool</var></samp></dt> <dd> This allows to override tee muxer use_fifo option for individual slave muxer. </dd> <dt><samp class="option">fifo_options</samp></dt> <dd> This allows to override tee muxer fifo_options for individual slave muxer. See fifo. </dd> <dt><samp class="option">select</samp></dt> <dd> Select the streams that should be mapped to the slave output, specified by a stream specifier. If not specified, this defaults to all the mapped streams. This will cause that output operation to fail if the output format does not accept all mapped streams. You may use multiple stream specifiers separated by commas (<code class="code">,</code>) e.g.: <code class="code">a:0,v</code> </dd> <dt><samp class="option">onfail</samp></dt> <dd> Specify behaviour on output failure. This can be set to either <code class="code">abort</code> (which is default) or <code class="code">ignore</code>. <code class="code">abort</code> will cause whole process to fail in case of failure on this slave output. <code class="code">ignore</code> will ignore failure on this output, so other outputs will continue without being affected. </dd> </dl>

 <h4 class="subsection">4.77.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Encode something and both archive it in a WebM file and stream it as MPEG-TS over UDP: <pre class="example-preformatted">ffmpeg -i ... -c:v libx264 -c:a mp2 -f tee -map 0:v -map 0:a "archive-20121107.mkv|[f=mpegts]udp://10.0.1.255:1234/" </pre>

As above, but continue streaming even if output to local file fails (for example local drive fills up): <pre class=”example-preformatted”>ffmpeg -i ... -c:v libx264 -c:a mp2 -f tee -map 0:v -map 0:a "[onfail=ignore]archive-20121107.mkv|[f=mpegts]udp://10.0.1.255:1234/"
</pre>

Use <code class=”command”>ffmpeg</code> to encode the input, and send the output to three different destinations. The <code class=”code”>dump_extra</code> bitstream filter is used to add extradata information to all the output video keyframes packets, as requested by the MPEG-TS format. The select option is applied to <samp class=”file”>out.aac</samp> in order to make it contain only audio packets. <pre class=”example-preformatted”>ffmpeg -i ... -map 0 -flags +global_header -c:v libx264 -c:a aac -f tee "[bsfs/v=dump_extra=freq=keyframe]out.ts|[movflags=+faststart]out.mp4|[select=
</pre>

As above, but select only stream <code class=”code”>a:1</code> for the audio output. Note that a second level escaping must be performed, as ":" is a special character used to separate options. <pre class=”example-preformatted”>ffmpeg -i ... -map 0 -flags +global_header -c:v libx264 -c:a aac -f tee "[bsfs/v=dump_extra=freq=keyframe]out.ts|[movflags=-
</pre>

 <h3 class=”section”>4.78 wav#
TOC</h3>

RIFF Wave Audio muxer. <h4 class=”subsection”>4.78.1 Options#
TOC</h4>

<dl class=”table”> <dt><samp class=”option”>rf64 <var class=”var”>mode</var></samp></dt>
<dd> Choose whether to use the RF64 file format instead of RIFF. RF64 allows files larger than 4 gigabytes, but RF64 support is not as universal as RIFF. The following modes are recognized: <dl class=”table”>
<dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Start writing a standard RIFF file to retain compatibility with non-RF64-aware software, but switch to RF64 if the output file grows larger than 4 gigabytes. Reserves a small amount of extra space in the header using a <code class=”code”>JUNK</code> chunk, which should be ignored by all compliant RIFF readers, but some extremely simplistic WAV readers may be confused by this. </dd> <dt>‘<samp class=”samp”>always</samp>’</dt> <dd> Always use the RF64 format regardless of file size. The output file will only be readable by RF64-compatible software and is no longer a standard RIFF WAVE file.
</dd> <dt>‘<samp class=”samp”>never</samp>’</dt>
<dd> Always use plain RIFF, never RF64. This mode is the most compat-

ible with legacy software, but if the output file grows beyond 4 gigabytes, the 32-bit size field in the <code class=”code”>data</code> chunk will no longer be able to correctly represent the size, and most audio software will not be able to read some or all of the data. (The wav demuxer’s <code class=”code”>ignore_length</code> option can be used to recover such a file.) This is the default mode. </dd> </dl>

</dd> <dt><samp class=”option”>write_bext <var class=”var”>bool</var></samp></dt> <dd> Add a Broadcast Wave Format <code class=”code”>BEXT</code> chunk containing extended audio metadata if enabled. Defaults to false. </dd> <dt><samp class=”option”>write_peak <var class=”var”>mode</var></samp></dt> <dd> Add a Peak Envelope chunk (EBU Tech 3285 Supplement 3) if enabled. Defaults to off. <dl class=”table”> <dt>‘<samp class=”samp”>off</samp>’</dt> <dd> Do not add the Peak Envelope chunk. </dd> <dt>‘<samp class=”samp”>on</samp>’</dt> <dd> Add the Peak Envelope chunk, calculating the peak values based on the <code class=”code”>peak_block_size</code>, <code class=”code”>peak_format</code>, and <code class=”code”>peak_ppv</code> options. </dd> <dt>‘<samp class=”samp”>only</samp>’</dt> <dd> Like <code class=”code”>on</code>, but skip writing the actual audio data (no <code class=”code”>data</code> chunk will be written to the file; only the Peak Envelope and other metadata chunks will be written.) </dd> </dl>

</dd> <dt><samp class=”option”>peak_block_size <var class=”var”>int</var></samp></dt> <dd> Number of audio samples used to generate each peak frame, up to 65536. Default is 256. </dd> <dt><samp class=”option”>peak_format <var class=”var”>format</var></samp></dt> <dd> The format of the peak envelope data (1: uint8, 2: uint16). Default is 2 (uint16). </dd> <dt><samp class=”option”>peak_ppv <var class=”var”>int</var></samp></dt> <dd> Number of peak points per peak value (1 or 2). Default is 2. </dd> </dl>

 <h3 class=”section”>4.79 webm_chunk# TOC</h3>

WebM Live Chunk Muxer. This muxer writes out WebM headers and chunks as separate files which can be consumed by clients that support WebM Live streams via DASH. <h4 class=”subsection”>4.79.1 Options# TOC</h4>

This muxer supports the following options: <dl class=”table”> <dt><samp class=”option”>chunk_start_index</samp></dt> <dd> Index

of the first chunk (defaults to 0). </dd> <dt><samp class="option">header</samp></dt> <dd> Filename of the header where the initialization data will be written. </dd> <dt><samp class="option">audio_chunk_duration</samp></dt> <dd> Duration of each audio chunk in milliseconds (defaults to 5000). </dd> </dl>

 <h4 class="subsection">4.79.2 Example# TOC</h4> <pre class="example-preformatted">ffmpeg -f v4l2 -i /dev/video0 \ -f alsa -i hw:0 \ -map 0:0 \ -c:v libvpx-vp9 \ -s 640x360 -keyint_min 30 -g 30 \ -f webm_chunk \ -header webm_live_video_360.hdr \ -chunk_start_index 1 \ webm_live_video_360_%d.chk \ -map 1:0 \ -c:a libvorbis \ -b:a 128k \ -f webm_chunk \ -header webm_live_audio_128.hdr \ -chunk_start_index 1 \ -audio_chunk_duration 1000 \ webm_live_audio_128_%d.chk </pre>

 <h3 class="section">4.80 webm_dash_manifest# TOC</h3>

WebM DASH Manifest muxer. This muxer implements the WebM DASH Manifest specification to generate the DASH manifest XML. It also supports manifest generation for DASH live streams. For more information see: <ul class="itemize mark-bullet"> WebM DASH Specification: https://sites.google.com/a/webmproject.org/wiki/adaptive-streaming/webm-dash-specification ISO DASH Specification: http://standards.iso.org/ittf/PubliclyAvailableStandards/c065274_ISO_IEC_23009-1_2014.zip

 <h4 class="subsection">4.80.1 Options# TOC</h4>

This muxer supports the following options: <dl class="table"> <dt><samp class="option">adaptation_sets</samp></dt> <dd> This option has the following syntax: "id=x,streams=a,b,c id=y,streams=d,e" where x and y are the unique identifiers of the adaptation sets and a,b,c,d and e are the indices of the corresponding audio and video streams. Any number of adaptation sets can be added using this option. </dd> <dt><samp class="option">live</samp></dt> <dd> Set this to 1 to create a live stream DASH Manifest. Default: 0.

</dd> <dt><samp class="option">chunk_start_index</samp></dt> <dd> Start index of the first chunk. This will go in the ‘<samp class="samp">startNumber</samp>’ attribute of the ‘<samp class="samp">SegmentTemplate</samp>’ element in the manifest. Default: 0. </dd> <dt><samp class="option">chunk_duration_ms</samp></dt> <dd> Duration of each chunk in milliseconds. This will go in the ‘<samp class="samp">duration</samp>’ attribute of the ‘<samp class="samp">SegmentTemplate</samp>’ element in the manifest. Default: 1000. </dd> <dt><samp class="option">utc_timing_url</samp></dt> <dd> URL of the page that will return the UTC timestamp in ISO format. This will go in the ‘<samp class="samp">value</samp>’ attribute of the ‘<samp class="samp">UTCTiming</samp>’ element in the manifest. Default: None. </dd> <dt><samp class="option">time_shift_buffer_depth</samp></dt> <dd> Smallest time (in seconds) shifting buffer for which any Representation is guaranteed to be available. This will go in the ‘<samp class="samp">timeShiftBufferDepth</samp>’ attribute of the ‘<samp class="samp">MPD</samp>’ element. Default: 60. </dd> <dt><samp class="option">minimum_update_period</samp></dt> <dd> Minimum update period (in seconds) of the manifest. This will go in the ‘<samp class="samp">minimumUpdatePeriod</samp>’ attribute of the ‘<samp class="samp">MPD</samp>’ element. Default: 0. </dd> </dl>

 <h4 class="subsection">4.80.2 Example# TOC</h4> <pre class="example-preformatted">ffmpeg -f webm_dash_manifest -i video1.webm \ -f webm_dash_manifest -i video2.webm \ -f webm_dash_manifest -i audio1.webm \ -f webm_dash_manifest -i audio2.webm \ -map 0 -map 1 -map 2 -map 3 \ -c copy \ -f webm_dash_manifest \ -adaptation_sets "id=0,streams=0,1 id=1,streams=2,3" \ manifest.xml </pre>

 <h3 class="section">4.81 whip# TOC</h3>

WebRTC (Real-Time Communication) muxer that supports sub-second latency streaming according to the WHIP (WebRTC-HTTP ingestion protocol) specification. This is an experimental feature. It uses HTTP as a signaling protocol to exchange SDP capabilities and ICE lite candidates. Then, it uses STUN binding requests and responses to establish a session over UDP. Subsequently, it initiates a DTLS handshake to exchange the SRTP encryption keys. Lastly, it splits video and audio frames into RTP packets and encrypts them us-

ing SRTP. Ensure that you use H.264 without B frames and Opus for the audio codec. For example, to convert an input file with <code class=”command”>ffmpeg</code> to WebRTC: <pre class=”example-preformatted”>ffmpeg -re -i input.mp4 -acodec libopus -ar 48000 -ac 2 \ -vcodec libx264 -profile:v baseline -tune zerolatency -threads 1 -bf 0 \ -f whip "http://localhost:1985/rtc/v1/whip/?app=live&stream=livestream" </pre>

For this example, we have employed low latency options, resulting in an end-to-end latency of approximately 150ms. <h4 class=”subsection”>4.81.1 Options# TOC</h4>

This muxer supports the following options: <dl class=”table”> <dt><samp class=”option”>handshake_timeout <var class=”var”>integer</var></samp></dt> <dd> Set the timeout in milliseconds for ICE and DTLS handshake. Default value is 5000. </dd> <dt><samp class=”option”>timeout <var class=”var”>integer</var></samp></dt> <dd> Set timeout in seconds for socket I/O operations. Applicable only for HTTP output. Default value is -1. </dd> <dt><samp class=”option”>pkt_size <var class=”var”>integer</var></samp></dt> <dd> Set the maximum size, in bytes, of RTP packets that send out. Default value is 1200. </dd> <dt><samp class=”option”>ts_buffer_size <var class=”var”>integer</var></samp></dt> <dd> Set the buffer size, in bytes, of underlying protocol. Default value is -1(auto). The UDP auto selects a reasonable value. </dd> <dt><samp class=”option”>whip_flags <var class=”var”>flags</var></samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>dtls_active</samp>’</dt> <dd> The muxer will try to set dtls active role and send the first client hello. </dd> </dl>

</dd> <dt><samp class=”option”>rtp_history <var class=”var”>integer</var></samp></dt> <dd> Set the number of RTP history items to store. Default value is 512. </dd> <dt><samp class=”option”>authorization <var class=”var”>string</var></samp></dt> <dd> The optional Bearer token for WHIP Authorization. </dd> <dt><samp class=”option”>cert_file <var class=”var”>string</var></samp></dt> <dd> The optional certificate file path for DTLS. </dd> <dt><samp class=”option”>key_file <var class=”var”>string</var></samp></dt> <dd> The optional private key file path for DTLS. </dd> </dl>

 <h2 class=”chapter”>5 Metadata# TOC</h2>

FFmpeg is able to dump metadata from media files into a simple

UTF-8-encoded INI-like text file and then load it back using the metadata muxer/demuxer. The file format is as follows: <ol class=”enumerate”> A file consists of a header and a number of metadata tags divided into sections, each on its own line.

 The header is a ‘<samp class=”samp”>;FFMETADATA</samp>’ string, followed by a version number (now 1).

 Metadata tags are of the form ‘<samp class=”samp”>key=value</samp>’

 Immediately after header follows global metadata

 After global metadata there may be sections with per-stream/per-chapter metadata.

 A section starts with the section name in uppercase (i.e. STREAM or CHAPTER) in brackets (‘<samp class=”samp”>[</samp>’, ‘<samp class=”samp”>]</samp>’) and ends with next section or end of file.

 At the beginning of a chapter section there may be an optional timebase to be used for start/end values. It must be in form ‘<samp class=”samp”>TIMEBASE=<var class=”var”>num</var>/<var class=”var”>den</var></samp>’ where <var class=”var”>num</var> and <var class=”var”>den</var> are integers. If the timebase is missing then start/end times are assumed to be in nanoseconds.

Next a chapter section must contain chapter start and end times in form ‘<samp class=”samp”>START=<var class=”var”>num</var></samp>’, ‘<samp class=”samp”>END=<var class=”var”>num</var></samp>’, where <var class=”var”>num</var> is a positive integer. Empty lines and lines starting with ‘<samp class=”samp”>;</samp>’ or ‘<samp class=”samp”>#</samp>’ are ignored.

 Metadata keys or values containing special characters (‘<samp class=”samp”>=</samp>’, ‘<samp class=”samp”>;</samp>’, ‘<samp class=”samp”>#</samp>’, ‘<samp class=”samp”>\</samp>’ and a newline) must be escaped with a backslash ‘<samp class=”samp”>\</samp>’.

 Note that whitespace in metadata (e.g. ‘<samp class=”samp”>foo = bar</samp>’) is considered to be a part of the tag (in the example above key is ‘<samp class=”samp”>foo </samp>’, value is ‘<samp class=”samp”> bar</samp>’).

A ffmetadata file might look like this: <pre class=”example-preformatted”>;FFMETADATA1 title=bike\\shed ;this is a comment artist=FFmpeg troll team

[CHAPTER] TIMEBASE=1/1000 START=0 #chapter ends at 0:01:00 END=60000 title=chapter \#1 [STREAM] title=multi\ line </pre>

By using the ffmetadata muxer and demuxer it is possible to extract metadata from an input file to an ffmetadata file, and then transcode the file into an output file with the edited ffmetadata file. Extracting an ffmetadata file with <samp class="file">ffmpeg</samp> goes as follows: <pre class="example-preformatted">ffmpeg -i INPUT -f ffmetadata FFMETADATAFILE </pre>

Reinserting edited metadata information from the FFMETADATAFILE file can be done as: <pre class="example-preformatted">ffmpeg -i INPUT -i FFMETADATAFILE -map_metadata 1 -codec copy OUTPUT </pre>

 <h2 class="chapter">6 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavformat

 <h2 class="chapter">7 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-protocols.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Protocols Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Protocols Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 Protocol Options 3 Protocols <ul class="toc-numbered-mark"> 3.1 amqp 3.2 async 3.3 bluray 3.4 cache 3.5 concat 3.6 concatf 3.7 crypto 3.8 data 3.9 fd 3.10 file 3.11 ftp 3.12 gopher 3.13 gophers 3.14 handle 3.15 http <ul class="toc-numbered-mark"> 3.15.1 HTTP Cookies 3.16 Icecast 3.17 ipfs 3.18 mmst 3.19 mmsh 3.20 md5 3.21 pipe 3.22 prompeg 3.23 rist 3.24 rtmp 3.25 rtmpe 3.26 rtmps 3.27 rtmpt 3.28 rtmpte 3.29 rtmpts 3.30 libcurl 3.31 libsmbclient 3.32 libssh 3.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte 3.34 rtp 3.35 rtsp <ul class="toc-numbered-mark"> 3.35.1 Muxer 3.35.2 Demuxer 3.35.3 Examples 3.36 sap <ul class="toc-numbered-mark">

3.36.1 Muxer 3.36.2 Demuxer 3.37 sctp 3.38 shared 3.39 srt 3.40 srtp 3.41 subfile 3.42 tee 3.43 tcp 3.44 tls 3.45 dtls 3.46 udp <ul class="toc-numbered-mark"> 3.46.1 Examples 3.47 unix 3.48 zmq 4 See Also 5 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes the input and output protocols provided by the libavformat library.

 <h2 class="chapter">2 Protocol Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the protocols. In addition each protocol may support so-called private options, which are specific for that component. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class="table"> <dt><samp class="option">protocol_whitelist <var class="var">list</var> (input)</samp></dt> <dd> Set a ","-separated list of allowed protocols. "ALL" matches all protocols. Protocols prefixed by "-" are disabled. All protocols are allowed by default but protocols used by an another protocol (nested protocols) are restricted to a per protocol subset. </dd> </dl>

 <h2 class="chapter">3 Protocols# <a class="anchor hidden-xs"href="#toc-Protocols"

aria-hidden=”true”>TOC</h2>

Protocols are configured elements in FFmpeg that enable access to resources that require specific protocols. When you configure your FFmpeg build, all the supported protocols are enabled by default. You can list all available ones using the configure option "–list-protocols". You can disable all the protocols using the configure option "–disable-protocols", and selectively enable a protocol using the option "–enable-protocol=<var class=”var”>PROTOCOL</var>", or you can disable a particular protocol using the option "–disable-protocol=<var class=”var”>PROTOCOL</var>". The option "-protocols" of the ff* tools will display the list of supported protocols. All protocols accept the following options: <dl class=”table”> <dt><samp class=”option”>rw_timeout</samp></dt> <dd> Maximum time to wait for (network) read/write operations to complete, in microseconds. </dd> </dl>

A description of the currently available protocols follows. <h3 class=”section”>3.1 amqp#
TOC</h3>

Advanced Message Queueing Protocol (AMQP) version 0-9-1 is a broker based publish-subscribe communication protocol. FFmpeg must be compiled with –enable-librabbitmq to support AMQP. A separate AMQP broker must also be run. An example open-source AMQP broker is RabbitMQ. After starting the broker, an FFmpeg client may stream data to the broker using the command: <pre class=”example-preformatted”>ffmpeg -re -i input -f mpegts amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

Where hostname and port (default is 5672) is the address of the broker. The client may also set a user/password for authentication. The default for both fields is "guest". Name of virtual host on broker can be set with vhost. The default value is "/". Multiple subscribers may stream from the broker using the command: <pre class=”example-preformatted”>ffplay amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

In RabbitMQ all data published to the broker flows through a specific exchange, and each subscribing client has an assigned queue/buffer. When a packet arrives at an exchange, it may be copied to a client’s queue depending on the exchange and routing_key fields. The following options are supported: <dl class=”table”> <dt><samp class=”option”>exchange</samp></dt> <dd> Sets the exchange to use on the broker. RabbitMQ has several predefined exchanges: "amq.direct" is the default exchange, where the publisher and

subscriber must have a matching routing_key; "amq.fanout" is the same as a broadcast operation (i.e. the data is forwarded to all queues on the fanout exchange independent of the routing_key); and "amq.topic" is similar to "amq.direct", but allows for more complex pattern matching (refer to the RabbitMQ documentation). </dd> <dt><samp class="option">routing_key</samp></dt> <dd> Sets the routing key. The default value is "amqp". The routing key is used on the "amq.direct" and "amq.topic" exchanges to decide whether packets are written to the queue of a subscriber. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Maximum size of each packet sent/received to the broker. Default is 131072. Minimum is 4096 and max is any large value (representable by an int). When receiving packets, this sets an internal buffer size in FFmpeg. It should be equal to or greater than the size of the published packets to the broker. Otherwise the received message may be truncated causing decoding errors. </dd> <dt><samp class="option">connection_timeout</samp></dt> <dd> The timeout in seconds during the initial connection to the broker. The default value is rw_timeout, or 5 seconds if rw_timeout is not set. </dd> <dt><samp class="option">delivery_mode <var class="var">mode</var></samp></dt> <dd> Sets the delivery mode of each message sent to broker. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">persistent</samp>’</dt> <dd> Delivery mode set to "persistent" (2). This is the default value. Messages may be written to the broker’s disk depending on its setup. </dd> <dt>‘<samp class="samp">non-persistent</samp>’</dt> <dd> Delivery mode set to "non-persistent" (1). Messages will stay in broker’s memory unless the broker is under memory pressure. </dd> </dl>

</dd> </dl>

 <h3 class="section">3.2 async# TOC</h3>

Asynchronous data filling wrapper for input stream. Fill data in a background thread, to decouple I/O operation from demux thread.   <pre class="example-preformatted">async:<var class="var">URL</var> async:http://host/resource async:cache:http://host/resource </pre>

 <h3 class="section">3.3 bluray# TOC</h3>

Read BluRay playlist. The accepted options are: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> BluRay angle </dd> <dt><samp class="option">chapter</samp></dt>

<dd> Start chapter (1...N) </dd> <dt><samp class=”option”>playlist</samp></dt>
<dd> Playlist to read (BDMV/PLAYLIST/?????.mpls) </dd>
</dl>

Examples: Read longest playlist from BluRay mounted to /mnt/bluray: <pre class=”example-preformatted”>bluray:/mnt/bluray </pre>

Read angle 2 of playlist 4 from BluRay mounted to /mnt/bluray, start from chapter 2: <pre class=”example-preformatted”>-playlist 4 -angle 2 -chapter 2 bluray:/mnt/bluray </pre>

 <h3 class=”section”>3.4 cache# TOC</h3>

Caching wrapper for input stream. Cache the input stream to temporary file. It brings seeking capability to live streams. The accepted options are: <dl class=”table”> <dt><samp class=”option”>read_ahead_limit</samp></dt> <dd> Amount in bytes that may be read ahead when seeking isn’t supported. Range is -1 to INT_MAX. -1 for unlimited. Default is 65536. </dd> </dl>

URL Syntax is <pre class=”example-preformatted”>cache:<var class=”var”>URL</var> </pre>

 <h3 class=”section”>3.5 concat#
TOC</h3>

Physical concatenation protocol. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concat:<var class=”var”>URL1</var>|<var class=”var”>URL2</var>|...|<var class=”var”>URLN</var> </pre>

where <var class=”var”>URL1</var>, <var class=”var”>URL2</var>, ..., <var class=”var”>URLN</var> are the urls of the resource to be concatenated, each one possibly specifying a distinct protocol. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concat:split1.mpeg\|split2.mpeg\|split3.mpeg
</pre>

Note that you may need to escape the character "|" which is special for many shells. <h3 class=”section”>3.6 concatf<a class=”anchor

hidden-xs" href="#concatf" aria-hidden="true"># TOC</h3>

Physical concatenation protocol using a line break delimited list of resources. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class="example-preformatted">concatf:<var class="var">URL</var> </pre>

where <var class="var">URL</var> is the url containing a line break delimited list of resources to be concatenated, each one possibly specifying a distinct protocol. Special characters must be escaped with backslash or single quotes. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual. For example to read a sequence of files <samp class="file">split1.mpeg</samp>, <samp class="file">split2.mpeg</samp>, <samp class="file">split3.mpeg</samp> listed in separate lines within a file <samp class="file">split.txt</samp> with <code class="command">ffplay</code> use the command: <pre class="example-preformatted">ffplay concatf:split.txt </pre> Where <samp class="file">split.txt</samp> contains the lines: <pre class="example-preformatted">split1.mpeg split2.mpeg split3.mpeg </pre>

 <h3 class="section">3.7 crypto#
TOC</h3>

AES-encrypted stream reading protocol. The accepted options are: <dl class="table"> <dt><samp class="option">key</samp></dt>
<dd> Set the AES decryption key binary block from given hexadecimal representation. </dd> <dt><samp class="option">iv</samp></dt>
<dd> Set the AES decryption initialization vector binary block from given hexadecimal representation. </dd> </dl>

Accepted URL formats: <pre class="example-preformatted">crypto:<var class="var">URL</var> crypto+<var class="var">URL</var>
</pre>

 <h3 class="section">3.8 data#
TOC</h3>

Data in-line in the URI. See <a class="url" href="http://en.wikipedia.org/wiki/Data_URI_scheme">http
 For example, to convert a GIF file given inline with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i "
smiley.png </pre>

 <h3 class="section">3.9 fd# TOC</h3>

File descriptor access protocol. The accepted syntax is: <pre class="example-preformatted">fd: -fd <var class="var">file_descriptor</var> </pre>

If <samp class="option">fd</samp> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. Unlike the pipe protocol, fd protocol has seek support if it corresponding to a regular file. fd protocol doesn’t support pass file descriptor via URL for security. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

 <h3 class="section">3.10 file# TOC</h3>

File access protocol. Read from or write to a file. A file URL can have the form: <pre class="example-preformatted">file:<var class="var">filename</var> </pre>

where <var class="var">filename</var> is the path of the file to read. An URL that does not have a protocol prefix will be assumed to be a file URL. Depending on the build, an URL that looks like a Windows path with the drive letter at the beginning will also be assumed to be a file URL (usually not the case in builds for unix-like systems). For example to read from a file <samp class="file">input.mpeg</samp> with <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -i file:input.mpeg output.mpeg </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable for files on slow medium. </dd> <dt><samp class="option">follow</samp></dt> <dd> If set

to 1, the protocol will retry reading at the end of the file, allowing reading files that still are being written. In order for this to terminate, you either need to use the rw_timeout option, or use the interrupt callback (for API users). Setting this option also ignores the file size reported by the file system. </dd> <dt><samp class="option">seekable</samp></dt> <dd> Controls if seekability is advertised on the file. 0 means non-seekable, -1 means auto (seekable for normal files, non-seekable for named pipes). Many demuxers handle seekable and non-seekable resources differently, overriding this might speed up opening certain files at the cost of losing some features (e.g. accurate seeking). </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set the maximum packet size used for file I/O. A smaller value may reduce memory usage. A higher value may increase throughput especially with networked filesystems. For reading, if explicitly set, it overrides the default internal buffer size (32 KB) and limits the maximum amount of data read per operation. For writing, this sets the size of each write operation. The default is 256 KB for regular files, 32 KB otherwise. </dd> </dl>

 <h3 class="section">3.11 ftp# TOC</h3>

FTP (File Transfer Protocol). Read from or write to remote resources using FTP protocol. Following syntax is required. <pre class="example-preformatted">ftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg
</pre>

This protocol accepts the following options. <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Set timeout in microseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class="option">ftp-user</samp></dt> <dd> Set a user to be used for authenticating to the FTP server. This is overridden by the user in the FTP URL. </dd> <dt><samp class="option">ftp-password</samp></dt> <dd> Set a password to be used for authenticating to the FTP server. This is overridden by the password in the FTP URL, or by <samp class="option">ftp-anonymous-password</samp> if no user is set. </dd> <dt><samp class="option">ftp-anonymous-password</samp></dt> <dd> Password used when login as anonymous user. Typically an e-mail address should be used. </dd> <dt><samp class="option">ftp-write-seekable</samp></dt> <dd> Control seekability of connection during encoding. If set to 1 the resource is supposed to be seekable, if set to 0 it is assumed not to be seekable. Default value is 0. </dd> </dl>

NOTE: Protocol can be used as output, but it is recommended to not do it, unless special care is taken (tests, customized server configuration

etc.). Different FTP servers behave in different way during seek operation. ff* tools may produce incomplete content due to server limitations. <h3 class=”section”>3.12 gopher# TOC</h3>

Gopher protocol. <h3 class=”section”>3.13 gophers# TOC</h3>

Gophers protocol. The Gopher protocol with TLS encapsulation. <h3 class=”section”>3.14 handle# TOC</h3>

Windows handle access protocol. The accepted syntax is: <pre class=”example-preformatted”>handle:-handle <var class=”var”>handle</var> </pre>

If <samp class=”option”>handle</samp> is not specified, by default the output handle will be used for writing, input handle for reading. This protocol is only available on Windows. It has seek support if the handle corresponds to a regular file. It doesn’t support passing the handle via URL. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class=”code”>INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class=”option”>handle</samp></dt> <dd> Set Windows handle. The supplied handle must already be valid in the calling process. When launching <code class=”command”>ffmpeg</code>, arrange for the child process to inherit the handle before passing its numeric value with <samp class=”option”>handle</samp>. Handles opened with <code class=”code”>FILE_FLAG_OVERLAPPED</code> are not supported. Closing the protocol does not close the caller’s original handle. </dd> </dl>

 <h3 class=”section”>3.15 http# TOC</h3>

HTTP (Hyper Text Transfer Protocol).

This protocol accepts the following options:

<dl class="table">
<dt><samp class="option">seekable</samp></dt>
<dd> Control seekability of connection. If set to 1 the resource is
supposed to be seekable, if set to 0 it is assumed not to be seekable,
if set to -1 it will try to autodetect if it is seekable. Default
value is -1.

</dd>
<dt><samp class="option">chunked_post</samp></dt>
<dd> If set to 1 use chunked Transfer-Encoding for posts, default is 1.

</dd>
<dt><samp class="option">http_proxy</samp></dt>
<dd> set HTTP proxy to tunnel through e.g. http://example.com:1234

</dd>
<dt><samp class="option">headers</samp></dt>
<dd> Set custom HTTP headers, can override built in default headers. The
value must be a string encoding the headers.

</dd>
<dt><samp class="option">content_type</samp></dt>
<dd> Set a specific content type for the POST messages or for listen mode.

</dd>
<dt><samp class="option">user_agent</samp></dt>
<dd> Override the User-Agent header. If not specified the protocol will use
a
string describing the libavformat build. ("Lavf/<version>")

</dd>

<dt><samp class="option">referer</samp></dt>

<dd> Set the Referer header. Include ’Referer: URL’ header in HTTP request.

</dd>

<dt><samp class="option">multiple_requests</samp></dt>

<dd> Force persistent connections if set to 1, or disable if 0. Default is -1,

which means auto (implies keep-alive when using -request_size or

-initial_request_size).

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> Limit the size of requests made. This is useful for some pathological servers

that throttle unbounded range requests, as well as when expecting to seek

frequently. Disabled (set to 0) by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher. Doing so allows FFmpeg

to reuse a single HTTP connection wherever possible.

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also enable

the <samp class="option">multiple_requests</samp> option, as well as setting

<samp class="option">short_seek_size</samp> to the same value or higher.

</dd>

<dt><samp class="option">post_data</samp></dt>

<dd> Set custom HTTP post data.

</dd>

<dt><samp class="option">mime_type</samp></dt>

<dd> Export the MIME type.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> Exports the HTTP response version number. Usually "1.0" or "1.1".

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Set the cookies to be sent in future requests. The format of each cookie is the

same as the value of a Set-Cookie HTTP response field. Multiple cookies can be

delimited by a newline character. A cookie without a Domain attribute is only

sent to the host of the URL it is set for.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class="option">icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class="option">icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class="option">metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

<dt><samp class="option">auth_type</samp></dt>

<dd>

Set HTTP authentication type. No option for Digest, since this method requires

getting nonce parameters from the server first and can’t be used straight away like

Basic.

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> Choose the HTTP authentication type automatically. This is the default.

</dd>

<dt><samp class=”option”>basic</samp></dt>

<dd>

Choose the HTTP basic authentication.

Basic authentication sends a Base64-encoded string that contains a user name and password

for the client. Base64 is not a form of encryption and should be considered the same as

sending the user name and password in clear text (Base64 is a reversible encoding).

If a resource needs to be protected, strongly consider using an authentication scheme

other than basic authentication. HTTPS/TLS should be used with basic authentication.

Without these additional security enhancements, basic authentication should not be used

to protect sensitive or valuable information.

</dd>

</dl>

</dd> <dt><samp class=”option”>send_expect_100</samp></dt> <dd> Send an Expect: 100-continue header for POST. If set to 1 it will send, if set to 0 it won’t, if set to -1 it will try to send if it is applicable. Default value is -1. </dd> <dt><samp class=”option”>location</samp></dt> <dd> An exported dictionary containing the content location. Only useful with the C API. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set initial byte offset. </dd> <dt><samp class=”option”>end_offset</samp></dt>

<dd> Try to limit the request to bytes preceding this offset. </dd> <dt><samp class="option">method</samp></dt> <dd> When used as a client option it sets the HTTP method for the request. When used as a server option it sets the HTTP method that is going to be expected from the client(s). If the expected and the received HTTP method do not match the client will be given a Bad Request response. When unset the HTTP method is not checked for now. This will be replaced by autodetection in the future. </dd> <dt><samp class="option">reconnect</samp></dt> <dd> Reconnect automatically when disconnected before EOF is hit. </dd> <dt><samp class="option">reconnect_at_eof</samp></dt> <dd> If set then eof is treated like an error and causes reconnection, this is useful for live / endless streams. This does not apply to the deliberately established end of a resource whose complete size is unknown, i.e. when a ranged request past the end is answered with <code class="code">416</code>. Such an end is reported as a clean EOF and is not retried, unless <samp class="option">reconnect_on_http_error</samp> lists 416 by number. </dd> <dt><samp class="option">reconnect_on_network_error</samp></dt> <dd> Reconnect automatically in case of TCP/TLS errors during connect. </dd> <dt><samp class="option">reconnect_on_http_error</samp></dt> <dd> A comma separated list of HTTP status codes to reconnect on. The list can include specific status codes (e.g. ’503’) or the strings ’4xx’ / ’5xx’. A <code class="code">416</code> that ends a read or a seek past the end of a resource whose complete size is unknown is reported as EOF. It is retried first only if 416 is listed by number, for example to read a resource that is still being written. ’4xx’ does not cover it. </dd> <dt><samp class="option">reconnect_streamed</samp></dt> <dd> If set then even streamed/non seekable streams will be reconnected on errors. </dd> <dt><samp class="option">reconnect_delay_max</samp></dt> <dd> Set the maximum delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">reconnect_max_retries</samp></dt> <dd> Set the maximum number of times to retry a connection. Default unset. </dd> <dt><samp class="option">reconnect_delay_total_max</samp></dt> <dd> Set the maximum total delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">respect_retry_after</samp></dt> <dd> If enabled, and a Retry-After header is encountered, its requested reconnection delay will be honored, rather than using exponential backoff. Useful for 429 and 503 errors. Default enabled. </dd> <dt><samp class="option">listen</samp></dt> <dd> If set to 1 enables experimental HTTP server. This can be used to send data when used as an output option, or read data from a client with HTTP POST when used as an input option. If set to 2 enables experimental multi-client HTTP server. This is not yet implemented in ffmpeg.c and thus must not be used as a command line option. <pre class="example-preformatted"># Server side (sending): ffmpeg -i somefile.ogg -c copy -listen 1 -f ogg http://<var class="var">server</var>:<var class="var">port</var>

Client side (receiving): ffmpeg -i http://<var class="var">server</var>:<var class="var">port</var> -c copy somefile.ogg

Client can also be done with wget: wget http://<var class="var">server</var>:<var class="var">port</var> -O somefile.ogg

Server side (receiving): ffmpeg -listen 1 -i http://<var class="var">server</var>:<var class="var">port</var> -c copy somefile.ogg

Client side (sending): ffmpeg -i somefile.ogg -chunked_post 0 -c copy -f ogg http://<var class="var">server</var>:<var class="var">port</var>

Client can also be done with wget: wget --post-file=somefile.ogg http://<var class="var">server</var>:<var class="var">port</var> </pre>

</dd> <dt><samp class="option">resource</samp></dt> <dd> The resource requested by a client, when the experimental HTTP server is in use. </dd> <dt><samp class="option">reply_code</samp></dt> <dd> The HTTP code returned to the client, when the experimental HTTP server is in use. </dd> <dt><samp class="option">short_seek_size</samp></dt> <dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and new HTTP request. This is useful, for example, to make sure the same connection is used for reading large video packets with small audio packets in between. </dd> </dl>

 <h4 class="subsection">3.15.1 HTTP Cookies# TOC</h4>

Some HTTP requests will be denied unless cookie values are passed in with the request. The <samp class="option">cookies</samp> option allows these cookies to be specified. At the very least, each cookie must specify a value. HTTP requests that match both the domain and path will automatically include the cookie value in the HTTP Cookie header field. Multiple cookies can be delimited by a newline. The required syntax to play a stream specifying a cookie is: <pre class="example-preformatted">ffplay -cookies "nlqptid=nltid=tsn; path=/; domain=somedomain.com;" http://somedomain.com/somestream.m3u8 </pre>

 <h3 class="section">3.16 Icecast# TOC</h3>

Icecast protocol (stream to Icecast servers) This protocol accepts the following options: <dl class="table"> <dt><samp class="option">ice_genre</samp></dt> <dd> Set the stream genre. </dd> <dt><samp class="option">ice_name</samp></dt> <dd> Set the stream name. </dd> <dt><samp class="option">ice_description</samp></dt>

<dd> Set the stream description. </dd> <dt><samp class="option">ice_url</samp></dt> <dd> Set the stream website URL. </dd> <dt><samp class="option">ice_public</samp></dt> <dd> Set if the stream should be public. The default is 0 (not public). </dd> <dt><samp class="option">user_agent</samp></dt> <dd> Override the User-Agent header. If not specified a string of the form "Lavf/<version>" will be used. </dd> <dt><samp class="option">password</samp></dt> <dd> Set the Icecast mountpoint password. </dd> <dt><samp class="option">content_type</samp></dt> <dd> Set the stream content type. This must be set if it is different from audio/mpeg. </dd> <dt><samp class="option">legacy_icecast</samp></dt> <dd> This enables support for Icecast versions < 2.4.0, that do not support the HTTP PUT method but the SOURCE method. </dd> <dt><samp class="option">tls</samp></dt> <dd> Establish a TLS (HTTPS) connection to Icecast. </dd> </dl>

 <pre class="example-preformatted">icecast://[<var class="var">username</var>[:<var class="var">password</var>]@]<var class="var">server</var>:<var class="var">port</var>/<var class="var">mountpoint</var> </pre>

 <h3 class="section">3.17 ipfs# TOC</h3>

InterPlanetary File System (IPFS) protocol support. One can access files stored on the IPFS network through so-called gateways. These are http(s) endpoints. This protocol wraps the IPFS native protocols (ipfs:// and ipns://) to be sent to such a gateway. Users can (and should) host their own node which means this protocol will use one’s local gateway to access files on the IPFS network. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">gateway</samp></dt> <dd> Defines the gateway to use. When not set, the protocol will first try locating the local gateway by looking at <code class="code">$IPFS_GATEWAY</code>, <code class="code">$IPFS_PATH</code> and <code class="code">$HOME/.ipfs/</code>, in that order. </dd> </dl>

One can use this protocol in 2 ways. Using IPFS: <pre class="example-preformatted">ffplay ipfs://<hash> </pre>

Or the IPNS protocol (IPNS is mutable IPFS): <pre class="example-preformatted">ffplay ipns://<hash> </pre>

 <h3 class="section">3.18 mmst# TOC</h3>

MMS (Microsoft Media Server) protocol over TCP. <h3 class="section">3.19 mmsh# TOC</h3>

MMS (Microsoft Media Server) protocol over HTTP. The required syntax is: <pre class="example-preformatted">mmsh://<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">playpath</var>] </pre>

 <h3 class="section">3.20 md5# TOC</h3>

MD5 output protocol. Computes the MD5 hash of the data to be written, and on close writes this to the designated output or stdout if none is specified. It can be used to test muxers without writing an actual file. Some examples follow. <pre class="example-preformatted"># Write the MD5 hash of the encoded AVI file to the file output.avi.md5. ffmpeg -i input.flv -f avi -y md5:output.avi.md5

Write the MD5 hash of the encoded AVI file to stdout. ffmpeg -i input.flv -f avi -y md5: </pre>

Note that some formats (typically MOV) require the output protocol to be seekable, so they will fail with the MD5 output protocol. <h3 class="section">3.21 pipe# TOC</h3>

UNIX pipe access protocol. Read and write from UNIX pipes. The accepted syntax is: <pre class="example-preformatted">pipe:[<var class="var">number</var>] </pre>

If <samp class="option">fd</samp> isn’t specified, <var class="var">number</var> is the number corresponding to the file descriptor of the pipe (e.g. 0 for stdin, 1 for stdout, 2 for stderr). If <var class="var">number</var> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. For example to read from stdin with <code class="command">ffmpeg</code>: <pre class="example-preformatted">cat test.wav | ffmpeg -i pipe:0 # ...this is the same as... cat test.wav | ffmpeg -i pipe: </pre>

For writing to stdout with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i test.wav -f avi pipe:1 | cat > test.avi # ...this is the same as... ffmpeg -i test.wav -f avi pipe: | cat > test.avi </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

Note that some formats (typically MOV), require the output protocol to be seekable, so they will fail with the pipe output protocol. <h3 class="section">3.22 prompeg# TOC</h3>

Pro-MPEG Code of Practice #3 Release 2 FEC protocol. The Pro-MPEG CoP#3 FEC is a 2D parity-check forward error correction mechanism for MPEG-2 Transport Streams sent over RTP. This protocol must be used in conjunction with the <code class="code">rtp_mpegts</code> muxer and the <code class="code">rtp</code> protocol. The required syntax is: <pre class="example-preformatted">-f rtp_mpegts -fec prompeg=<var class="var">option</var>=<var class="var">val</var>... rtp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

The destination UDP ports are <code class="code">port + 2</code> for the column FEC stream and <code class="code">port + 4</code> for the row FEC stream. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">l=<var class="var">n</var></samp></dt> <dd> The number of columns (4-20, LxD <= 100) </dd> <dt><samp class="option">d=<var class="var">n</var></samp></dt> <dd> The number of rows (4-20, LxD <= 100) </dd> </dl>

Example usage: <pre class="example-preformatted">-f rtp_mpegts -fec prompeg=l=8:d=4 rtp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">3.23 rist# TOC</h3>

Reliable Internet Streaming Transport protocol The accepted options are: <dl class="table"> <dt><samp class="option">rist_profile</samp></dt> <dd> Supported values: <dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</d <dd> This one is default. </dd> <dt>‘<samp class="samp">advanced</samp>’</ </dl>

</dd> <dt><samp class=”option”>buffer_size</samp></dt> <dd> Set internal RIST buffer size in milliseconds for retransmission of data. Default value is 0 which means the librist default (1 sec). Maximum value is 30 seconds. </dd> <dt><samp class=”option”>fifo_size</samp></dt> <dd> Size of the librist receiver output fifo in number of packets. This must be a power of 2. Defaults to 8192 (vs the librist default of 1024). </dd> <dt><samp class=”option”>overrun_nonfatal=<var class=”var”>1|0</var></samp></dt> <dd> Survive in case of librist fifo buffer overrun. Default value is 0. </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Set maximum packet size for sending data. 1316 by default. </dd> <dt><samp class=”option”>log_level</samp></dt> <dd> Set loglevel for RIST logging messages. You only need to set this if you explicitly want to enable debug level messages or packet loss simulation, otherwise the regular loglevel is respected. </dd> <dt><samp class=”option”>secret</samp></dt> <dd> Set override of encryption secret, by default is unset. </dd> <dt><samp class=”option”>encryption</samp></dt> <dd> Set encryption type, by default is disabled. Acceptable values are 128 and 256. </dd> </dl>

 <h3 class=”section”>3.24 rtmp# TOC</h3>

Real-Time Messaging Protocol. The Real-Time Messaging Protocol (RTMP) is used for streaming multimedia content across a TCP/IP network. The required syntax is: <pre class=”example-preformatted”>rtmp://[<var class=”var”>username</var>:<var class=”var”>password</var>@]<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>instance</var>][/<var class=”var”>playpath</var>] </pre>

The accepted parameters are: <dl class=”table”> <dt><samp class=”option”>username</samp></dt> <dd> An optional username (mostly for publishing). </dd> <dt><samp class=”option”>password</samp></dt> <dd> An optional password (mostly for publishing). </dd> <dt><samp class=”option”>server</samp></dt> <dd> The address of the RTMP server. </dd> <dt><samp class=”option”>port</samp></dt> <dd> The number of the TCP port to use (by default is 1935). </dd> <dt><samp class=”option”>app</samp></dt> <dd> It is the name of the application to access. It usually corresponds to the path where the application is installed on the RTMP server (e.g. <samp class=”file”>/ondemand/</samp>, <samp class=”file”>/flash/live/</samp>, etc.). You can override the value parsed from the URI through the <code class=”code”>rtmp_app</code> option, too. </dd> <dt><samp class=”option”>playpath</samp></dt> <dd> It is the path or name of the resource to play with reference to the appli-

cation specified in <var class="var">app</var>, may be prefixed by "mp4:". You can override the value parsed from the URI through the <code class="code">rtmp_playpath</code> option, too. </dd> <dt><samp class="option">listen</samp></dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt><samp class="option">timeout</samp></dt> <dd> Maximum time to wait for the incoming connection. Implies listen. </dd> </dl>

Additionally, the following parameters can be set via command line options

(or in code via <code class="code">AVOption</code>s):

<dl class="table">

<dt><samp class="option">rtmp_app</samp></dt>

<dd> Name of application to connect on the RTMP server. This option

overrides the parameter specified in the URI.

</dd>

<dt><samp class="option">rtmp_buffer</samp></dt>

<dd> Set the client buffer time in milliseconds. The default is 3000.

</dd>

<dt><samp class="option">rtmp_conn</samp></dt>

<dd> Extra arbitrary AMF connection parameters, parsed from a string,

e.g. like <code class="code">B:1 S:authMe O:1 NN:code:1.23 NS:flag:ok O:0</code>.

Each value is prefixed by a single character denoting the type,

B for Boolean, N for number, S for string, O for object, or Z for null,

followed by a colon. For Booleans the data must be either 0 or 1 for

FALSE or TRUE, respectively. Likewise for Objects the data must be 0 or

1 to end or begin an object, respectively. Data items in subobjects may

be named, by prefixing the type with ’N’ and specifying the name before

the value (i.e. <code class="code">NB:myFlag:1</code>). This option may be used multiple

times to construct arbitrary AMF sequences.

</dd>

<dt><samp class=”option”>rtmp_enhanced_codecs</samp></dt>

<dd> Specify the list of codecs the client advertises to support in an

enhanced RTMP stream. This option should be set to a comma separated

list of fourcc values, like <code class=”code”>hvc1,av01,vp09</code> for multiple codecs

or <code class=”code”>hvc1</code> for only one codec. The specified list will be presented

in the "fourCcLive" property of the Connect Command Message.

</dd>

<dt><samp class=”option”>rtmp_flashver</samp></dt>

<dd> Version of the Flash plugin used to run the SWF player. The default

is LNX 9,0,124,2. (When publishing, the default is FMLE/3.0 (compatible;

<libavformat version>).)

</dd>

<dt><samp class=”option”>rtmp_flush_interval</samp></dt>

<dd> Number of packets flushed in the same request (RTMPT only). The default

is 10.

</dd>

<dt><samp class=”option”>rtmp_live</samp></dt>

<dd> Specify that the media is a live stream. No resuming or seeking in

live streams is possible. The default value is <code class=”code”>any</code>, which means the

subscriber first tries to play the live stream specified in the

playpath. If a live stream of that name is not found, it plays the

recorded stream. The other possible values are <code class=”code”>live</code> and

<code class=”code”>recorded</code>.

</dd>

<dt><samp class="option">rtmp_pageurl</samp></dt>

<dd> URL of the web page in which the media was embedded. By default no

value will be sent.

</dd>

<dt><samp class="option">rtmp_playpath</samp></dt>

<dd> Stream identifier to play or to publish. This option overrides the

parameter specified in the URI.

</dd>

<dt><samp class="option">rtmp_subscribe</samp></dt>

<dd> Name of live stream to subscribe to. By default no value will be sent.

It is only sent if the option is specified or if rtmp_live

is set to live.

</dd>

<dt><samp class="option">rtmp_swfhash</samp></dt>

<dd> SHA256 hash of the decompressed SWF file (32 bytes).

</dd>

<dt><samp class="option">rtmp_swfsize</samp></dt>

<dd> Size of the decompressed SWF file, required for SWFVerification.

</dd>

<dt><samp class="option">rtmp_swfurl</samp></dt>

<dd> URL of the SWF player for the media. By default no value will be sent.

</dd>

<dt><samp class=”option”>rtmp_swfverify</samp></dt>

<dd> URL to player swf file, compute hash/size automatically.

</dd>

<dt><samp class=”option”>rtmp_tcurl</samp></dt>

<dd> URL of the target stream. Defaults to proto://host[:port]/app.

</dd>

<dt><samp class=”option”>tcp_nodelay=<var class=”var”>1|0</var></samp></dt>

<dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0.

Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY.

</dd>

<dt><samp class=”option”>tcp_keepalive=<var class=”var”>1|0</var></samp></dt>

<dd> Enable the TCP keepalive mechanism to detect dead peers and help maintain long-lived idle connections. Default value is 0.

Only the basic keepalive option (SO_KEEPALIVE) can be enabled or disabled. Platform-specific tuning parameters such as TCP_KEEPIDLE, TCP_KEEPINTVL, or TCP_KEEPCNT are not configurable and will use the operating system’s default values.

</dd>

</dl>

For example to read with <code class=”command”>ffplay</code> a multimedia resource named "sample" from the application "vod" from an RTMP server "myserver": <pre class=”example-preformatted”>ffplay rtmp://myserver/vod/sample </pre>

To publish to a password protected server, passing the playpath and app names separately: <pre class="example-preformatted">ffmpeg -re -i <input> -f flv -rtmp_playpath some/long/path -rtmp_app long/app/name rtmp://username:password@myserver/ </pre>

 <h3 class="section">3.25 rtmpe# TOC</h3>

Encrypted Real-Time Messaging Protocol. The Encrypted Real-Time Messaging Protocol (RTMPE) is used for streaming multimedia content within standard cryptographic primitives, consisting of Diffie-Hellman key exchange and HMACSHA256, generating a pair of RC4 keys. <h3 class="section">3.26 rtmps# TOC</h3>

Real-Time Messaging Protocol over a secure SSL connection. The Real-Time Messaging Protocol (RTMPS) is used for streaming multimedia content across an encrypted connection. <h3 class="section">3.27 rtmpt# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTP. The Real-Time Messaging Protocol tunneled through HTTP (RTMPT) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">3.28 rtmpte# TOC</h3>

Encrypted Real-Time Messaging Protocol tunneled through HTTP. The Encrypted Real-Time Messaging Protocol tunneled through HTTP (RTMPTE) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">3.29 rtmpts# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTPS. The Real-Time Messaging Protocol tunneled through HTTPS (RTMPTS) is used for streaming multimedia content within HTTPS requests to traverse firewalls. <h3 class="section">3.30 libcurl# TOC</h3>

HTTP and HTTPS access through the libcurl library, available as an alternative to the native <code class=”code”>http</code> protocol when FFmpeg is built with <code class=”code”>--enable-libcurl</code>. It runs transfers on a dedicated event-loop thread shared per <code class=”code”>AVFormatContext</code>, so all of a demuxer’s connections (for example HLS segments) reuse libcurl’s connection cache and HTTP/2 and HTTP/3 multiplexing. The native <code class=”code”>http</code> protocol stays the default. There are two ways to use libcurl instead: <ul class=”itemize mark-bullet”> Prefix the URL with <code class=”code”>libcurl:</code> to force it explicitly, for example <code class=”code”>libcurl:https://example.com/video.mp4</code>.

Set the <samp class=”option”>prefer_libcurl</samp> option to route <code class=”code”>http://</code> and <code class=”code”>https://</code> URLs through libcurl transparently.

<dl class=”table”> <dt><samp class=”option”>prefer_libcurl</samp></dt> <dd> When set to <code class=”code”>1</code>, open <code class=”code”>http(s)</code> URLs with the libcurl protocol instead of the native one. This is a generic IO option rather than a libcurl-specific one, so it also applies on builds without libcurl (where it has no effect). Default is <code class=”code”>0</code>. </dd> </dl>

The libcurl protocol accepts the following options.

<dl class=”table”>

<dt><samp class=”option”>user_agent</samp></dt>

<dd> Override the <code class=”code”>User-Agent</code> header. Defaults to <code class=”code”>Lavf/<version></code>.

</dd>

<dt><samp class=”option”>referer</samp></dt>

<dd> Set the <code class=”code”>Referer</code> header.

</dd>

<dt><samp class=”option”>headers</samp></dt>

<dd> Set custom HTTP headers, overriding internal defaults. Multiple headers are

separated by newlines.

</dd>

<dt><samp class="option">http_proxy</samp></dt>

<dd> Set the HTTP proxy to tunnel through.

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Cookies to be sent in future requests, given as newline-delimited

<code class="code">Set-Cookie</code>-style values.

</dd>

<dt><samp class="option">location</samp></dt>

<dd> The actual location of the data after following redirects. This is an output

option, read with <code class="command">av_opt_get</code>; demuxers use it to resolve relative URLs.

</dd>

<dt><samp class="option">offset</samp></dt>

<dd> Initial byte offset of the request. Default is <code class="code">0</code>.

</dd>

<dt><samp class="option">end_offset</samp></dt>

<dd> If greater than <code class="code">0</code>, limit the request to the bytes preceding this offset.

Together with <samp class="option">offset</samp> this requests a sub-range of the resource, as used

for HLS byte-range segments.

</dd>

<dt><samp class="option">seekable</samp></dt>

<dd> Control seekability of the connection. If set to <code class="code">0</code> the stream is treated

as non-seekable, <code class=”code”>1</code> forces seekable, and <code class=”code”>-1</code> (the default) lets it be

detected from the server response.

</dd>

<dt><samp class=”option”>tls_verify</samp></dt>

<dd> Verify the peer certificate and host name. Enabled by default.

</dd>

<dt><samp class=”option”>ca_file</samp></dt>

<dd> Path to a certificate authority bundle that overrides the system store.

</dd>

<dt><samp class=”option”>cert_file</samp></dt>

<dt><samp class=”option”>key_file</samp></dt>

<dd> Client certificate and private key files for mutual TLS.

</dd>

<dt><samp class=”option”>connect_timeout</samp></dt>

<dd> Connection timeout in seconds. <code class=”code”>0</code> (the default) uses libcurl’s own timeout.

</dd>

<dt><samp class=”option”>max_redirects</samp></dt>

<dd> Maximum number of redirects to follow. Default is <code class=”code”>16</code>.

</dd>

<dt><samp class=”option”>multiple_requests</samp></dt>

<dd> Reuse the connection across requests (HTTP keep-alive). Enabled by default;

setting it to <code class="code">0</code> uses a fresh connection per transfer.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> HTTP version to use: <code class="code">auto</code> (default), <code class="code">1.0</code>, <code class="code">1.1</code>, <code class="code">2</code>,

<code class="code">2tls</code>, <code class="code">2-prior-knowledge</code>, <code class="code">3</code> or <code class="code">3only</code>.

</dd>

<dt><samp class="option">buffer_size</samp></dt>

<dd> Size in bytes of the receive buffer. Default is 4 MiB.

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> If greater than <code class="code">0</code>, split a seekable transfer into ranged requests of at

most this many bytes each. Default is <code class="code">0</code> (a single request).

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher, otherwise the HTTP

connection may be closed and recreated for each subsequent initial request.

</dd>

<dt><samp class="option">max_retries</samp></dt>

<dd> Maximum number of retries after a recoverable error on a seekable transfer.

Default is <code class="code">5</code>.

</dd>

<dt><samp class="option">short_seek_size</samp></dt>

<dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and

new HTTP request. This is useful, for example, to make sure the same connection

is used for reading large video packets with small audio packets in between.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_status</samp></dt>

<dd> If set to 1 accept the legacy Shoutcast v1 <code class="code">ICY 200 OK</code> status line in

place of an HTTP one. Any reply whose status line starts with <code class=”code”>ICY 200</code>

is then treated as a successful HTTP 200.

The default is 1.

</dd>

<dt><samp class=”option”>icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class=”option”>icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class=”option”>icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class=”option”>metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

</dl>

A stream carrying in-band ICY metadata is always treated as non-seekable, since its byte offsets do not address the payload. <samp class=”option”>seekable</samp>, <samp class=”option”>request_size</samp> and <samp class=”option”>max_retries</samp> therefore have no effect on such a stream. For more information see: https://curl.se/libcurl/. <h3 class=”section”>3.31 libsmbclient<span

class=”pull-right”># TOC</h3>

libsmbclient permits one to manipulate CIFS/SMB network resources. Following syntax is required.   <pre class=”example-preformatted”>smb://[[domain:]user[:password@]]server[/share[/path[/file]]] </pre>

This protocol accepts the following options. <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout in milliseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class=”option”>workgroup</samp></dt> <dd> Set the workgroup used for making connections. By default workgroup is not specified. </dd> </dl>

For more information see: http://www.samba.org/. <h3 class=”section”>3.32 libssh# TOC</h3>

Secure File Transfer Protocol via libssh Read from or write to remote resources using SFTP protocol. Following syntax is required.   <pre class=”example-preformatted”>sftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg </pre>

This protocol accepts the following options. <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class=”option”>private_key</samp></dt> <dd> Specify the path of the file containing private key to use during authorization. By default libssh searches for keys in the <samp class=”file”>~/.ssh/</samp> directory. </dd> <dt><samp class=”option”>verify</samp></dt> <dd> Verify the server host key against the known_hosts file before authenticating, if set to 1. Servers that are not in the file, or whose key differs from the one recorded there, are refused. Default value is 1. </dd> </dl>

Example: Play a file stored on remote server.   <pre class=”example-preformatted”>ffplay sftp://user:password@server_address:22/home/user/resource.mpeg

</pre>

 <h3 class="section">3.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte# TOC</h3>

Real-Time Messaging Protocol and its variants supported through librtmp. Requires the presence of the librtmp headers and library during configuration. You need to explicitly configure the build with "–enable-librtmp". If enabled this will replace the native RTMP protocol. This protocol provides most client functions and a few server functions needed to support RTMP, RTMP tunneled in HTTP (RTMPT), encrypted RTMP (RTMPE), RTMP over SSL/TLS (RTMPS) and tunneled variants of these encrypted types (RTMPTE, RTMPTS). The required syntax is: <pre class="example-preformatted"><var class="var">rtmp_proto</var>://<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">playpath</var>] <var class="var">options</var> </pre>

where <var class="var">rtmp_proto</var> is one of the strings "rtmp", "rtmpt", "rtmpe", "rtmps", "rtmpte", "rtmpts" corresponding to each RTMP variant, and <var class="var">server</var>, <var class="var">port</var>, <var class="var">app</var> and <var class="var">playpath</var> have the same meaning as specified for the RTMP native protocol. <var class="var">options</var> contains a list of space-separated options of the form <var class="var">key</var>=<var class="var">val</var>. See the librtmp manual page (man 3 librtmp) for more information. For example, to stream a file in real-time to an RTMP server using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -re -i myfile -f flv rtmp://myserver/live/mystream </pre>

To play the same stream using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay "rtmp://myserver/live/mystream live=1" </pre>

 <h3 class="section">3.34 rtp# TOC</h3>

Real-time Transport Protocol. The required syntax for an RTP URL is: <pre class="example-preformatted">rtp://<var class="var">hostname</var>[:<var class="var">port</var>][?<var class="var">options</var>] </pre>

<var class="var">port</var> specifies the RTP port to use. <var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">ttl=<var class="var">n</var></samp></dt> <dd> Set the TTL (Time-To-Live) value (for multicast only). </dd> <dt><samp class="option">rtcpport=<var class="var">n</var></samp></dt> <dd> Set the remote RTCP port to <var class="var">n</var>. </dd> <dt><samp class="option">localport, local_rtpport, localrtpport=<var class="var">n</var></samp></dt> <dd> Set the local RTP port to <var class="var">n</var>. Using the localport option name is deprecated and should not be used. </dd> <dt><samp class="option">local_rtcpport, localrtcpport=<var class="var">n</var>'</samp></dt> <dd> Set the local RTCP port to <var class="var">n</var>. </dd> <dt><samp class="option">pkt_size=<var class="var">n</var></samp></dt> <dd> Set max packet size (in bytes) to <var class="var">n</var>. </dd> <dt><samp class="option">buffer_size=<var class="var">size</var></samp></dt> <dd> Set the maximum UDP socket buffer size in bytes. </dd> <dt><samp class="option">connect=0|1</samp></dt> <dd> Do a <code class="code">connect()</code> on the UDP socket (if set to 1) or not (if set to 0). </dd> <dt><samp class="option">sources=<var class="var">ip</var>[,<var class="var">ip</var>]</samp></dt> <dd> List allowed source IP addresses. </dd> <dt><samp class="option">block=<var class="var">ip</var>[,<var class="var">ip</var>]</samp></dt> <dd> List disallowed (blocked) source IP addresses. </dd> <dt><samp class="option">write_to_source=0|1</samp></dt> <dd> Send packets to the source address of the latest received packet (if set to 1) or to a default remote address (if set to 0). </dd> <dt><samp class="option">localaddr=<var class="var">addr</var></samp></dt> <dd> Local IP address of a network interface used for sending packets or joining multicast groups. </dd> <dt><samp class="option">timeout=<var class="var">n</var></samp></dt> <dd> Set timeout (in microseconds) of socket I/O operations to <var class="var">n</var>. </dd> </dl>

Important notes: <ol class="enumerate"> If <samp class="option">rtcpport</samp> is not set the RTCP port will be set to the RTP port value plus 1.

 If <samp class="option">localrtpport</samp> (the local RTP port) is not set any available port will be used for the local RTP and RTCP ports.

 If <samp class="option">localrtcpport</samp> (the local RTCP port) is not set it will be set to the local RTP port value plus 1.

 <h3 class="section">3.35 rtsp# TOC</h3>

Real-Time Streaming Protocol. RTSP is not technically a protocol handler in libavformat, it is a demuxer and muxer. The demuxer supports both normal RTSP (with data transferred over RTP; this is used by e.g. Apple and Microsoft) and Real-RTSP (with data transferred over RDT). The muxer can be used to send a stream using RTSP ANNOUNCE to a server supporting it (currently Darwin Streaming Server and Mischa Spiegelmock’s RTSP server). The required syntax for a RTSP url is: <pre class="example-preformatted">rtsp://<var class="var">hostname</var>[:<var class="var">port</var>]/<var class="var">path</var> </pre>

Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line, or set in code via <code class="code">AVOption</code>s or in <code class="code">avformat_open_input</code>. <h4 class="subsection">3.35.1 Muxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">latm</samp>’</dt> <dd> Use MP4A-LATM packetization instead of MPEG4-GENERIC for AAC. </dd> <dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> Use RFC 2190 packetization instead of RFC 4629 for H.263. </dd> <dt>‘<samp class="samp">skip_rtcp</samp>’</dt> <dd> Don’t send RTCP sender reports. </dd> <dt>‘<samp class="samp">h264_mode0</samp>’</dt> <dd> Use mode 0 for H.264 in RTP. </dd> <dt>‘<samp class="samp">send_bye</samp>’</dt> <dd> Send RTCP BYE packets when finishing. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’.

</dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set max send packet size (in bytes). Default value is 1472. </dd> </dl>

 <h4 class="subsection">3.35.2 Demuxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">initial_pause</samp></dt> <dd> Do not start playing the stream immediately if set to 1. Default value is 0. </dd> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> <dt>‘<samp class="samp">udp_multicast</samp>’</dt> <dd> Use UDP multicast as lower transport protocol. </dd> <dt>‘<samp class="samp">http</samp>’</dt> <dd> Use HTTP tunneling as lower transport protocol, which is useful for passing proxies. </dd> <dt>‘<samp class="samp">https</samp>’</dt> <dd> Use HTTPs tunneling as lower transport protocol, which is useful for passing proxies and widely used for security consideration. </dd> </dl>

Multiple lower transport protocols may be specified, in that case they are tried one at a time (if the setup of one fails, the next one is tried). For the muxer, only the ‘<samp class="samp">tcp</samp>’ and ‘<samp class="samp">udp</samp>’ options are supported. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">filter_src</samp>’</dt> <dd> Accept packets only from negotiated peer address and port. </dd> <dt>‘<samp class="samp">listen</samp>’</dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt>‘<samp class="samp">prefer_tcp</samp>’</dt> <dd> Try TCP for RTP transport first, if TCP is available as RTSP RTP transport. </dd> <dt>‘<samp class="samp">satip_raw</samp>’</dt> <dd> Export raw MPEG-TS stream instead of demuxing. The flag will simply write out the raw stream, with the original PAT/PMT/PIDs intact. </dd> </dl>

Default value is ‘<samp class=”samp”>none</samp>’.
 </dd> <dt><samp class=”option”>allowed_media_types</samp></dt>
<dd> Set media types to accept from the server. The following flags are accepted: <dl class=”table”> <dt>‘<samp
class=”samp”>video</samp>’</dt> <dt>‘<samp class=”samp”>audio</samp>’</d
<dt>‘<samp class=”samp”>data</samp>’</dt> <dt>‘<samp
class=”samp”>subtitle</samp>’</dt> </dl>

By default it accepts all media types. </dd> <dt><samp
class=”option”>min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp
class=”option”>max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp
class=”option”>listen_timeout</samp></dt> <dd> Set maximum
timeout (in seconds) to establish an initial connection. Setting <samp
class=”option”>listen_timeout</samp> > 0 sets <samp class=”option”>rtsp_flags</samp>
to ‘<samp class=”samp”>listen</samp>’. Default is -1 which
means an infinite timeout when ‘<samp class=”samp”>listen</samp>’
mode is set. </dd> <dt><samp class=”option”>reorder_queue_size</samp></dt>
<dd> Set number of packets to buffer for handling of reordered packets. </dd> <dt><samp class=”option”>timeout</samp></dt>
<dd> Set socket TCP I/O timeout in microseconds. </dd>
<dt><samp class=”option”>user_agent</samp></dt> <dd> Override
User-Agent header. If not specified, it defaults to the libavformat identifier
string. </dd> <dt><samp class=”option”>buffer_size</samp></dt>
<dd> Set the maximum socket buffer size in bytes. </dd> </dl>

When receiving data over UDP, the demuxer tries to reorder received
packets (since they may arrive out of order, or packets may get lost totally).
This can be disabled by setting the maximum demuxing delay to zero (via
the <code class=”code”>max_delay</code> field of AVFormatContext).
 When watching multi-bitrate Real-RTSP streams with <code
class=”command”>ffplay</code>, the streams to display can be chosen
with <code class=”code”>-vst</code> <var class=”var”>n</var> and
<code class=”code”>-ast</code> <var class=”var”>n</var> for video
and audio respectively, and can be switched on the fly by pressing <code
class=”code”>v</code> and <code class=”code”>a</code>. <a
name=”Examples”> <h4 class=”subsection”>3.35.3 Examples<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Examples” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Examples”
aria-hidden=”true”>TOC</h4>

The following examples all make use of the <code class=”command”>ffplay</code>
and <code class=”command”>ffmpeg</code> tools. <ul class=”itemize
mark-bullet”> Watch a stream over UDP, with a max reordering
delay of 0.5 seconds: <pre class=”example-
preformatted”>ffplay -max_delay 500000 -rtsp_transport udp rtsp://server/video.mp4

</pre>

Watch a stream tunneled over HTTP: <pre class="example-preformatted">ffplay -rtsp_transport http rtsp://server/video.mp4 </pre>

Send a stream in realtime to a RTSP server, for others to watch: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f rtsp -muxdelay 0.1 rtsp://server/live.sdp </pre>

Receive a stream in realtime: <pre class="example-preformatted">ffmpeg -rtsp_flags listen -i rtsp://ownaddress/live.sdp <var class="var">output</var> </pre>

 <h3 class="section">3.36 sap# TOC</h3>

Session Announcement Protocol (RFC 2974). This is not technically a protocol handler in libavformat, it is a muxer and demuxer. It is used for signalling of RTP streams, by announcing the SDP for the streams regularly on a separate port. <h4 class="subsection">3.36.1 Muxer# TOC</h4>

The syntax for a SAP url given to the muxer is: <pre class="example-preformatted">sap://<var class="var">destination</var>[:<var class="var">port</var>][?<var class="var">options</var>] </pre>

The RTP packets are sent to <var class="var">destination</var> on port <var class="var">port</var>, or to port 5004 if no port is specified. <var class="var">options</var> is a <code class="code">&</code>-separated list. The following options are supported: <dl class="table"> <dt><samp class="option">announce_addr=<var class="var">address</var></samp></dt> <dd> Specify the destination IP address for sending the announcements to. If omitted, the announcements are sent to the commonly used SAP announcement multicast address 224.2.127.254 (sap.mcast.net), or ff0e::2:7ffe if <var class="var">destination</var> is an IPv6 address. </dd> <dt><samp class="option">announce_port=<var class="var">port</var></samp></dt> <dd> Specify the port to send the announcements on, defaults to 9875 if not specified. </dd> <dt><samp class="option">ttl=<var class="var">ttl</var></samp></dt> <dd> Specify the time to live value for the announcements and RTP packets, defaults to 255. </dd> <dt><samp class="option">same_port=<var class="var">0|1</var></samp></dt> <dd> If set to 1, send all RTP streams on the same port pair. If zero (the default), all streams are sent on unique ports, with each stream on a port 2 numbers higher than the previous.

VLC/Live555 requires this to be set to 1, to be able to receive the stream. The RTP stack in libavformat for receiving requires all streams to be sent on unique ports. </dd> </dl>

Example command lines follow. To broadcast a stream on the local subnet, for watching in VLC: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255?same_port=1 </pre>

Similarly, for watching in <code class="command">ffplay</code>:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255 </pre>

And for watching in <code class="command">ffplay</code>, over IPv6:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://[ff0e::1:2:3:4] </pre>

 <h4 class="subsection">3.36.2 Demuxer# TOC</h4>

The syntax for a SAP url given to the demuxer is: <pre class="example-preformatted">sap://[<var class="var">address</var>][:<var class="var">port</var>] </pre>

<var class="var">address</var> is the multicast address to listen for announcements on, if omitted, the default 224.2.127.254 (sap.mcast.net) is used. <var class="var">port</var> is the port that is listened on, 9875 if omitted. The demuxers listens for announcements on the given address and port. Once an announcement is received, it tries to receive that particular stream. Example command lines follow. To play back the first stream announced on the normal SAP multicast address: <pre class="example-preformatted">ffplay sap:// </pre>

To play back the first stream announced on one the default IPv6 SAP multicast address: <pre class="example-preformatted">ffplay sap://[ff0e::2:7ffe] </pre>

 <h3 class="section">3.37 sctp# TOC</h3>

Stream Control Transmission Protocol. The accepted URL syntax is: <pre class="example-preformatted">sctp://<var class="var">host</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>.

Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">listen</samp></dt> <dd> If set to any value, listen for an incoming connection. Outgoing connection is done by default. </dd> <dt><samp class="option">max_streams</samp></dt> <dd> Set the maximum number of streams. By default no limit is set. </dd> </dl>

 <h3 class="section">3.38 shared#
TOC</h3>

Thread-safe, persistent, cross-process cache wrapper for input streams. Caches the input stream to a file in the specified directory. This is similar to cache, but with a persistent cache on disk that may be shared between multiple processes or demuxers, even concurrently. The filename of the cache is obtained by hashing the input URL, so there is some risk of collision if the same URL is used for different content (e.g. as a result of extra POST data being used to select the stream ID). To avoid this, make sure to only use this protocol for URLs that uniquely identify the content; otherwise you may get a garbled mix of sources. This includes content selected by request options such as headers, cookies or credentials: data cached for one user is returned to every other user of the same cache directory. The accepted options are: <dl class="table"> <dt><samp class="option">cache_dir</samp></dt> <dd> Path to the directory where cache files are stored. This option is required. Anyone who can write to this directory or to <samp class="option">metadata_dir</samp> can alter the data read by all users of the cache, or make them crash, so neither may be writable by untrusted users. </dd> <dt><samp class="option">metadata_dir</samp></dt> <dd> Path to the directory where metadata files are stored. Optional. If not specified, metadata is stored in the same directory as the cache file. </dd> <dt><samp class="option">block_shift</samp></dt> <dd> Shift factor (log2) of the block size used for internal reads/writes. Defaults to 15, i.e. 32KB blocks. If this does not match the value that was specified when an existing cache file was created, the previously specified value will be used instead. </dd> <dt><samp class="option">cache_read_only</samp></dt> <dd> If true, use the shared cache for reads but don’t write any new blocks to it. Default is false. Note that even with this option enabled, the cache file will be initialized if it does not already exist. </dd> <dt><samp class="option">cache_verify</samp></dt> <dd> If true, verify any data read from the cache against the underlying input stream, and error out on any mismatches. Note that this will make the cache layer effectively useless. This is purely a debug option. </dd> <dt><samp class="option">cache_timeout</samp></dt> <dd> If set to a nonzero

value, specifies the maximum time (in microseconds) to wait for data to become available, if another process is currently trying to fetch and cache the same block at the same time. If this timeout elapses, it’s assumed that the other process may have gotten stuck or died in the meantime. If set to zero, no waiting is done and all processes will immediately race to try and fetch the same missing blocks themselves. Defaults to 10000 (10 ms). </dd> <dt><samp class="option">ignore_errors</samp></dt> <dd> If true, failures of the underlying input stream are not treated as fatal, and playback continues using only the data already present in the cache file. Default is false. Any attempt to read into uncached data will result in an unrecoverable IO error. </dd> <dt><samp class="option">retry_errors</samp></dt> <dd> If true (the default), transient read errors from the underlying input stream are ignored and retried again. If false, any blocks that previously failed being read from will be treated as permanently inaccessible. </dd> <dt><samp class="option">retry_corrupt</samp></dt> <dd> If true (the default), blocks whose contents fail the CRC integrity check are re-fetched from the underlying input stream, overwriting the corrupt cached data. If false, cache corruption is treated as a fatal read error. </dd> <dt><samp class="option">cache_size_max</samp></dt> <dd> Maximum amount of data to cache, in bytes, rounded down to a multiple of the chosen block size. Accepts the usual size suffixes, e.g. <code class="code">10G</code> for 10*10^9 bytes, or <code class="code">10Gi</code> for 10*2^30 bytes. Defaults to 0, meaning no limit. Once this limit is reached, the protocol switches to read-only mode (see <samp class="option">cache_read_only</samp>). Already cached data is never evicted, so once this limit is reached, no new data will ever be cached until the cache file is deleted. Note that this option is best-effort only, and may over-write up to N-1 blocks, where N is the number of concurrent processes using the same cache file. Note also that the underlying file system may reserve more space than written, if the underlying allocation granularity is larger than the block size. </dd> <dt><samp class="option">offset</samp></dt> <dd> Initial byte offset to start reading from. This option is not passed on to the underlying protocol; it instead gets a regular seek on the first cache miss. Defaults to 0. </dd> </dl>

URL Syntax is <pre class="example-preformatted">shared:<var class="var">URL</var> </pre>

 <h3 class="section">3.39 srt# TOC</h3>

Haivision Secure Reliable Transport Protocol via libsrt. The supported syntax for a SRT URL is: <pre class="example-preformatted">srt://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form

<var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class=”code”>AVOption</code>s).

The list of supported options follows.

<dl class=”table”>

<dt><samp class=”option”>connect_timeout=<var class=”var”>milliseconds</var></samp></dt>

<dd> Connection timeout; SRT cannot connect for RTT > 1500 msec

(2 handshake exchanges) with the default connect timeout of

3 seconds. This option applies to the caller and rendezvous

connection modes. The connect timeout is 10 times the value

set for the rendezvous mode (which can be used as a

workaround for this connection problem with earlier versions).

</dd>

<dt><samp class=”option”>ffs=<var class=”var”>bytes</var></samp></dt>

<dd> Flight Flag Size (Window Size), in bytes. FFS is actually an

internal parameter and you should set it to not less than

<samp class=”option”>recv_buffer_size</samp> and <samp class=”option”>mss</samp>. The default value

is relatively large, therefore unless you set a very large receiver buffer,

you do not need to change this option. Default value is 25600.

</dd>

<dt><samp class=”option”>inputbw=<var class=”var”>bytes/seconds</var></samp></dt>

<dd> Sender nominal input rate, in bytes per seconds. Used along with

<samp class="option">oheadbw</samp>, when <samp class="option">maxbw</samp> is set to relative (0), to

calculate maximum sending rate when recovery packets are sent

along with the main media stream:

<samp class="option">inputbw</samp> * (100 + <samp class="option">oheadbw</samp>) / 100

if <samp class="option">inputbw</samp> is not set while <samp class="option">maxbw</samp> is set to

relative (0), the actual input rate is evaluated inside

the library. Default value is 0.

</dd>

<dt><samp class="option">iptos=<var class="var">tos</var></samp></dt>

<dd> IP Type of Service. Applies to sender only. Default value is 0xB8.

</dd>

<dt><samp class="option">ipttl=<var class="var">ttl</var></samp></dt>

<dd> IP Time To Live. Applies to sender only. Default value is 64.

</dd>

<dt><samp class="option">latency=<var class="var">microseconds</var></samp></dt>

<dd> Timestamp-based Packet Delivery Delay.

Used to absorb bursts of missed packet retransmissions.

This flag sets both <samp class="option">rcvlatency</samp> and <samp class="option">peerlatency</samp>

to the same value. Note that prior to version 1.3.0

this is the only flag to set the latency, however

this is effectively equivalent to setting <samp class="option">peerlatency</samp>,

when side is sender and <samp class="option">rcvlatency</samp>

when side is receiver, and the bidirectional stream

sending is not supported.

</dd>

<dt><samp class="option">listen_timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set socket listen timeout.

</dd>

<dt><samp class="option">maxbw=<var class="var">bytes/seconds</var></samp></dt>

<dd> Maximum sending bandwidth, in bytes per seconds.

-1 infinite (CSRTCC limit is 30mbps)

0 relative to input rate (see <samp class="option">inputbw</samp>)

>0 absolute limit value

Default value is 0 (relative)

</dd>

<dt><samp class="option">mode=<var class="var">caller|listener|rendezvous</var></samp></dt>

<dd> Connection mode.

<samp class="option">caller</samp> opens client connection.

<samp class="option">listener</samp> starts server to listen for incoming connections.

<samp class="option">rendezvous</samp> use Rendez-Vous connection mode.

Default value is caller.

</dd>

<dt><samp class="option">mss=<var class="var">bytes</var></samp></dt>

<dd> Maximum Segment Size, in bytes. Used for buffer allocation

and rate calculation using a packet counter assuming fully

filled packets. The smallest MSS between the peers is

used. This is 1500 by default in the overall internet.

This is the maximum size of the UDP packet and can be

only decreased, unless you have some unusual dedicated

network settings. Default value is 1500.

</dd>

<dt><samp class="option">nakreport=<var class="var">1|0</var></samp></dt>

<dd> If set to 1, Receiver will send ‘UMSG_LOSSREPORT‘ messages

periodically until a lost packet is retransmitted or

intentionally dropped. Default value is 1.

</dd>

<dt><samp class="option">oheadbw=<var class="var">percents</var></samp></dt>

<dd> Recovery bandwidth overhead above input rate, in percents.

See <samp class="option">inputbw</samp>. Default value is 25%.

</dd>

<dt><samp class="option">passphrase=<var class="var">string</var></samp></dt>

<dd> HaiCrypt Encryption/Decryption Passphrase string, length

from 10 to 79 characters. The passphrase is the shared

secret between the sender and the receiver. It is used

to generate the Key Encrypting Key using PBKDF2

(Password-Based Key Derivation Function). It is used

only if <samp class="option">pbkeylen</samp> is non-zero. It is used on

the receiver only if the received data is encrypted.

The configured passphrase cannot be recovered (write-only).

</dd>

<dt><samp class="option">enforced_encryption=<var class="var">1|0</var></samp></dt>

<dd> If true, both connection parties must have the same password

set (including empty, that is, with no encryption). If the

password doesn’t match or only one side is unencrypted,

the connection is rejected. Default is true.

</dd>
<dt><samp class="option">kmrefreshrate=<var class="var">packets</var></samp></dt>
<dd> The number of packets to be transmitted after which the
encryption key is switched to a new key. Default is -1.
-1 means auto (0x1000000 in srt library). The range for
this option is integers in the 0 - <code class="code">INT_MAX</code>.

</dd>
<dt><samp class="option">kmpreannounce=<var class="var">packets</var></samp></dt>
<dd> The interval between when a new encryption key is sent and
when switchover occurs. This value also applies to the
subsequent interval between when switchover occurs and
when the old encryption key is decommissioned. Default is -1.
-1 means auto (0x1000 in srt library). The range for
this option is integers in the 0 - <code class="code">INT_MAX</code>.

</dd>
<dt><samp class="option">snddropdelay=<var class="var">microseconds</var></samp></dt>
<dd> The sender’s extra delay before dropping packets. This delay
is
added to the default drop delay time interval value.

Special value -1: Do not drop packets on the sender at all.

</dd>
<dt><samp class="option">payload_size=<var class="var">bytes</var></samp></dt>
<dd> Sets the maximum declared size of a packet transferred
during the single call to the sending function in Live
mode. Use 0 if this value isn’t used (which is default in
file mode).
Default is -1 (automatic), which typically means MPEG-TS;

if you are going to use SRT

to send any different kind of payload, such as, for example,

wrapping a live stream in very small frames, then you can

use a bigger maximum frame size, though not greater than

1456 bytes.

</dd>

<dt><samp class="option">pkt_size=<var class="var">bytes</var></samp></dt>

<dd> Alias for ‘<samp class="samp">payload_size</samp>’.

</dd>

<dt><samp class="option">peerlatency=<var class="var">microseconds</var></samp></dt>

<dd> The latency value (as described in <samp class="option">rcvlatency</samp>)
that is

set by the sender side as a minimum value for the receiver.

</dd>

<dt><samp class="option">pbkeylen=<var class="var">bytes</var></samp></dt>

<dd> Sender encryption key length, in bytes.

Only can be set to 0, 16, 24 and 32.

Enable sender encryption if not 0.

Not required on receiver (set to 0),

key size obtained from sender in HaiCrypt handshake.

Default value is 0.

</dd>

<dt><samp class="option">rcvlatency=<var class="var">microseconds</var></samp></dt>

<dd> The time that should elapse since the moment when the

packet was sent and the moment when it’s delivered to

the receiver application in the receiving function.

This time should be a buffer time large enough to cover

the time spent for sending, unexpectedly extended RTT
time, and the time needed to retransmit the lost UDP
packet. The effective latency value will be the maximum
of this options’ value and the value of <samp class="option">peerlatency</samp>
set by the peer side. Before version 1.3.0 this option
is only available as <samp class="option">latency</samp>.

</dd>
<dt><samp class="option">recv_buffer_size=<var class="var">bytes</var></samp></dt>
<dd> Set UDP receive buffer size, expressed in bytes.

</dd>
<dt><samp class="option">send_buffer_size=<var class="var">bytes</var></samp></dt>
<dd> Set UDP send buffer size, expressed in bytes.

</dd>
<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>
<dd> Set raise error timeouts for read, write and connect operations. Note
that the
SRT library has internal timeouts which can be controlled separately, the
value set here is only a cap on those.

</dd>
<dt><samp class="option">tlpktdrop=<var class="var">1|0</var></samp></dt>
<dd> Too-late Packet Drop. When enabled on receiver, it skips
missing packets that have not been delivered in time and
delivers the following packets to the application when
their time-to-play has come. It also sends a fake ACK to
the sender. When enabled on sender and enabled on the
receiving peer, the sender drops the older packets that
have no chance of being delivered in time. It was

automatically enabled in the sender if the receiver

supports it.

</dd>

<dt><samp class=”option”>sndbuf=<var class=”var”>bytes</var></samp></dt>

<dd> Set send buffer size, expressed in bytes.

</dd>

<dt><samp class=”option”>rcvbuf=<var class=”var”>bytes</var></samp></dt>

<dd> Set receive buffer size, expressed in bytes.

Receive buffer must not be greater than <samp class=”option”>ffs</samp>.

</dd>

<dt><samp class=”option”>lossmaxttl=<var class=”var”>packets</var></samp></dt>

<dd> The value up to which the Reorder Tolerance may grow. When

Reorder Tolerance is > 0, then packet loss report is delayed

until that number of packets come in. Reorder Tolerance

increases every time a "belated" packet has come, but it

wasn’t due to retransmission (that is, when UDP packets tend

to come out of order), with the difference between the latest

sequence and this packet’s sequence, and not more than the

value of this option. By default it’s 0, which means that this

mechanism is turned off, and the loss report is always sent

immediately upon experiencing a "gap" in sequences.

</dd>

<dt><samp class=”option”>minversion</samp></dt>

<dd> The minimum SRT version that is required from the peer. A connection

to a peer that does not satisfy the minimum version requirement

will be rejected.

The version format in hex is 0xXXYYZZ for x.y.z in human readable

form.

</dd>

<dt><samp class="option">streamid=<var class="var">string</var></samp></dt>

<dd> A string limited to 512 characters that can be set on the socket prior

to connecting. This stream ID will be able to be retrieved by the

listener side from the socket that is returned from srt_accept and

was connected by a socket with that set stream ID. SRT does not enforce

any special interpretation of the contents of this string.

This option doesn't make sense in Rendezvous connection; the result

might be that simply one side will override the value from the other

side and it's the matter of luck which one would win

</dd>

<dt><samp class="option">srt_streamid=<var class="var">string</var></samp></dt>

<dd> Alias for ‘<samp class="samp">streamid</samp>’
to avoid conflict with ffmpeg command line option.

</dd>

<dt><samp class="option">smoother=<var class="var">live|file</var></samp></dt>

<dd> The type of Smoother used for the transmission for that socket, which

is responsible for the transmission and congestion control. The Smoother

type must be exactly the same on both connecting parties, otherwise

the connection is rejected.

</dd>

<dt><samp class="option">messageapi=<var class="var">1|0</var></samp></dt>

<dd> When set, this socket uses the Message API, otherwise it uses Buffer

API. Note that in live mode (see <samp class="option">transtype</samp>) there's only

message API available. In File mode you can chose to use one of two modes:

Stream API (default, when this option is false). In this mode you may send as many data as you wish with one sending instruction, or even use dedicated functions that read directly from a file. The internal facility will take care of any speed and congestion control. When receiving, you can also receive as many data as desired, the data not extracted will be waiting for the next call. There is no boundary between data portions in the Stream mode.

Message API. In this mode your single sending instruction passes exactly one piece of data that has boundaries (a message). Contrary to Live mode, this message may span across multiple UDP packets and the only size limitation is that it shall fit as a whole in the sending buffer. The receiver shall use as large buffer as necessary to receive the message, otherwise the message will not be given up. When the message is not complete (not all packets received or there was a packet loss) it will not be given up.

</dd>

<dt><samp class="option">transtype=<var class="var">live|file</var></samp></dt>

<dd> Sets the transmission type for the socket, in particular, setting this option sets multiple other parameters to their default values as required for a particular transmission type.

live: Set options as for live transmission. In this mode, you should send by one sending instruction only so many data that fit in one UDP packet, and limited to the value defined first in <samp class="option">payload_size</samp> (1316 is

default in this mode). There is no speed control in this mode, only the

bandwidth control, if configured, in order to not exceed the bandwidth with

the overhead transmission (retransmitted and control packets).

file: Set options as for non-live transmission. See <samp class="option">messageapi</samp>

for further explanations

</dd>

<dt><samp class="option">linger=<var class="var">seconds</var></samp></dt>

<dd> The number of seconds that the socket waits for unsent data when closing.

Default is -1. -1 means auto (off with 0 seconds in live mode, on with 180

seconds in file mode). The range for this option is integers in the

0 - <code class="code">INT_MAX</code>.

</dd>

<dt><samp class="option">tsbpd=<var class="var">1|0</var></samp></dt>

<dd> When true, use Timestamp-based Packet Delivery mode. The default behavior

depends on the transmission type: enabled in live mode, disabled in file

mode.

</dd>

<dt><samp class="option">ipv6only=<var class="var">1|0</var></samp></dt>

<dd> Accept IPv4 or not while using the IPv6 wildcard address. This should be set

when listening on a IPv6 wildcard address.

</dd>

</dl>

For more information see: <a class="url" href="https://github.com/Haivision/srt">https://github.com/H
  <h3 class="section">3.40 srtp<span

class=”pull-right”># TOC</h3>

Secure Real-time Transport Protocol. The accepted options are: <dl class=”table”> <dt><samp class=”option”>srtp_in_suite</samp></dt> <dt><samp class=”option”>srtp_out_suite</samp></dt> <dd> Select input and output encoding suites. Supported values: <dl class=”table”> <dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_80</samp>’</d <dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_80</samp>’</dt> <dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_32</samp>’</dt> <dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_32</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>srtp_in_params</samp></dt> <dt><samp class=”option”>srtp_out_params</samp></dt> <dd> Set input and output encoding parameters, which are expressed by a base64-encoded representation of a binary block. The first 16 bytes of this binary block are used as master key, the following 14 bytes are used as master salt. </dd> </dl>

 <h3 class=”section”>3.41 subfile# TOC</h3>

Virtually extract a segment of a file or another stream. The underlying stream must be seekable. Accepted options: <dl class=”table”> <dt><samp class=”option”>start</samp></dt> <dd> Start offset of the extracted segment, in bytes. </dd> <dt><samp class=”option”>end</samp></dt> <dd> End offset of the extracted segment, in bytes. If set to 0, extract till end of file. </dd> </dl>

Examples: Extract a chapter from a DVD VOB file (start and end sectors obtained externally and multiplied by 2048): <pre class=”example-preformatted”>subfile,,start,153391104,end,268142592,,:/media/dvd/ </pre>

Play an AVI file directly from a TAR archive: <pre class=”example-preformatted”>subfile,,start,183241728,end,366490624,,:archive.tar </pre>

Play a MPEG-TS file from start offset till end: <pre class=”example-preformatted”>subfile,,start,32815239,end,0,,:video.ts </pre>

 <h3 class=”section”>3.42 tee# TOC</h3>

Writes the output to multiple protocols. The individual outputs are separated by | <pre class=”example-preformatted”>tee:file://path/to/local/this.avi|file://path/to/local/that.avi
</pre>

 <h3 class=”section”>3.43 tcp#
TOC</h3>

Transmission Control Protocol. The required syntax for a TCP url is: <pre class=”example-preformatted”>tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”>
<dt><samp class=”option”>listen=<var class=”var”>2|1|0</var></samp></dt>
<dd> Listen for an incoming connection. 0 disables listen, 1 enables listen in single client mode, 2 enables listen in multi-client mode. Default value is 0. </dd> <dt><samp class=”option”>local_addr=<var class=”var”>addr</var></samp></dt> <dd> Local IP address of a network interface used for tcp socket connect. </dd> <dt><samp class=”option”>local_port=<var class=”var”>port</var></samp></dt>
<dd> Local port used for tcp socket connect. </dd> <dt><samp class=”option”>timeout=<var class=”var”>microseconds</var></samp></dt>
<dd> Set raise error timeout, expressed in microseconds.
This option is only relevant in read mode: if no data arrived in more than this time interval, raise error. </dd> <dt><samp class=”option”>listen_timeout=<var class=”var”>milliseconds</var></samp></dt>
<dd> Set listen timeout, expressed in milliseconds. </dd>
<dt><samp class=”option”>recv_buffer_size=<var class=”var”>bytes</var></samp></dt>
<dd> Set receive buffer size, expressed bytes. </dd> <dt><samp class=”option”>send_buffer_size=<var class=”var”>bytes</var></samp></dt>
<dd> Set send buffer size, expressed bytes. </dd> <dt><samp class=”option”>tcp_nodelay=<var class=”var”>1|0</var></samp></dt>
<dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0. Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY. </dd> <dt><samp class=”option”>tcp_mss=<var class=”var”>bytes</var></samp></dt>
<dd> Set maximum segment size for outgoing TCP packets, expressed in bytes. </dd> </dl>

The following example shows how to setup a listening TCP con-

nection with <code class=”command”>ffmpeg</code>, which is then accessed with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffmpeg -i <var class=”var”>input</var> -f <var class=”var”>format</var> tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var>?listen ffplay tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>3.44 tls# TOC</h3>

Transport Layer Security (TLS) / Secure Sockets Layer (SSL) The required syntax for a TLS/SSL url is: <pre class=”example-preformatted”>tls://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ca_file, cafile=<var class=”var”>filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class=”option”>tls_verify, verify=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. (With other backends, the host name is validated as well.) This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class=”option”>cert_file, cert=<var class=”var”>filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class=”option”>key_file, key=<var class=”var”>filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class=”option”>listen=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd>

<dt><samp class="option">http_proxy</samp></dt> <dd> The HTTP proxy to tunnel through, e.g. <code class="code">http://example.com:1234</code>. The proxy must support the CONNECT method. </dd> </dl>

Example command lines: To create a TLS/SSL server that serves an input stream.   <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> tls://<var class="var">hostname</var>:<var class="var">port</var>?listen&cert=<var class="var">server.crt</var>&key=<var class="var">server.key</var> </pre>

To play back a stream from the TLS/SSL server using <code class="command">ffplay</code>:   <pre class="example-preformatted">ffplay tls://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">3.45 dtls# TOC</h3>

Datagram Transport Layer Security (DTLS) The required syntax for a DTLS URL is: <pre class="example-preformatted">dtls://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). DTLS shares most options with TLS, but operates over UDP instead of TCP. The list of supported options follows. <dl class="table"> <dt><samp class="option">ca_file, cafile=<var class="var">filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class="option">tls_verify, verify=<var class="var">1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class="option">cert_file, cert=<var class="var">filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer.

(When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class=”option”>key_file, key=<var class=”var”>filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class=”option”>cert_pem=<var class=”var”>string</var></samp></dt> <dd> Certificate PEM string </dd> <dt><samp class=”option”>key_pem=<var class=”var”>string</var></samp></dt> <dd> Private key PEM string </dd> <dt><samp class=”option”>listen=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class=”option”>mtu=<var class=”var”>size</var></samp></dt> <dd> Set the Maximum Transmission Unit (MTU) for DTLS packets. </dd> <dt><samp class=”option”>use_srtp=<var class=”var”>1|0</var></samp></dt> <dd> Enable the use_srtp DTLS extension. This is used in WebRTC applications to establish SRTP encryption keys through the DTLS handshake. Default is disabled. </dd> <dt><samp class=”option”>external_sock=<var class=”var”>1|0</var></samp></dt> <dd> Use an external socket instead of creating a new one. This option only makes sense to pass when interacting with the code via API, enabling this from CLI will cause immediate failure. Default is disabled. </dd> </dl>

Example command lines: To create a DTLS server: <pre class=”example-preformatted”>ffmpeg -listen 1 -i dtls://<var class=”var”>hostname</var>:<var class=”var”>port</var> <var class=”var”>output</var> </pre>

To create a DTLS client and send data to server: <pre class=”example-preformatted”>ffmpeg -i <var class=”var”>input</var> -f <var class=”var”>format</var> dtls://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>3.46 udp# TOC</h3>

User Datagram Protocol. The required syntax for an UDP URL is: <pre class=”example-preformatted”>udp://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form

<var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class="code">AVOption</code>s).

In case threading is enabled on the system, a circular buffer is used

to store the incoming data, which allows one to reduce loss of data due to

UDP socket buffer overruns. The <var class="var">fifo_size</var> and

<var class="var">overrun_nonfatal</var> options are related to this buffer.

The list of supported options follows.

<dl class="table">

<dt><samp class="option">buffer_size=<var class="var">size</var></samp></dt>

<dd> Set the UDP maximum socket buffer size in bytes. This is used to set either

the receive or send buffer size, depending on what the socket is used for.

Default is 32 KB for output, 384 KB for input. See also <var class="var">fifo_size</var>.

</dd>

<dt><samp class="option">bitrate=<var class="var">bitrate</var></samp></dt>

<dd> If set to nonzero, the output will have the specified constant bitrate if the

input has enough packets to sustain it.

</dd>

<dt><samp class="option">burst_bits=<var class="var">bits</var></samp></dt>

<dd> When using <var class="var">bitrate</var> this specifies the maximum number of bits in

packet bursts.

</dd>

<dt><samp class="option">localport=<var class="var">port</var></samp></dt>

<dd> Override the local UDP port to bind with.

</dd>

<dt><samp class="option">localaddr=<var class="var">addr</var></samp></dt>

<dd> Local IP address of a network interface used for sending packets or joining

multicast groups.

</dd>

<dt><samp class="option">pkt_size=<var class="var">size</var></samp></dt>

<dd> Set the size in bytes of UDP packets.

</dd>

<dt><samp class="option">reuse=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow reusing UDP sockets.

</dd>

<dt><samp class="option">ttl=<var class="var">ttl</var></samp></dt>

<dd> Set the time to live value (for multicast only).

</dd>

<dt><samp class="option">dscp=<var class="var">dscp</var></samp></dt>

<dd> Set the 6-bit DSCP field for outgoing packets.

</dd>

<dt><samp class="option">connect=<var class="var">1|0</var></samp></dt>

<dd> Initialize the UDP socket with <code class="code">connect()</code>. In this case, the

destination address can’t be changed with ff_udp_set_remote_url later.

If the destination address isn’t known at the start, this option can

be specified in ff_udp_set_remote_url, too.

This allows finding out the source address for the packets with getsockname,

and makes writes return with AVERROR(ECONNREFUSED) if "destination

unreachable" is received.

For receiving, this gives the benefit of only receiving packets from

the specified peer address/port.

</dd>

<dt><samp class="option">sources=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Only receive packets sent from the specified addresses. In case of multicast,

also subscribe to multicast traffic coming from these addresses only.

</dd>

<dt><samp class="option">block=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Ignore packets sent from the specified addresses. In case of multicast, also

exclude the source addresses in the multicast subscription.

</dd>

<dt><samp class="option">fifo_size=<var class="var">units</var></samp></dt>

<dd> Set the UDP receiving circular buffer size, expressed as a number of

packets with size of 188 bytes. If not specified defaults to 7*4096.

Set to 0 to disable the receiving circular buffer and its receive thread.

</dd>

<dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt>

<dd> Survive in case of UDP receiving circular buffer overrun. Default

value is 0.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeout, expressed in microseconds.

This option is only relevant in read mode: if no data arrived in more

than this time interval, raise error.

</dd>

<dt><samp class="option">broadcast=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow UDP broadcasting.

Note that broadcasting may not work properly on networks having

a broadcast storm protection.

</dd>

</dl>

 <h4 class="subsection">3.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to stream over UDP to a remote endpoint: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> udp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

Use <code class="command">ffmpeg</code> to stream in mpegts format over UDP using 188 sized UDP packets, using a large input buffer: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f mpegts udp://<var class="var">hostname</var>:<var class="var">port</var>?pkt_size=188&buffer_size=65535 </pre>

Use <code class="command">ffmpeg</code> to receive over UDP from a remote endpoint: <pre class="example-preformatted">ffmpeg -i udp://[<var class="var">multicast-address</var>]:<var class="var">port</var> ... </pre>

 <h3 class="section">3.47 unix# TOC</h3>

Unix local socket The required syntax for a Unix socket URL is: <pre class=”example-preformatted”>unix://<var class=”var”>filepath</var> </pre>

The following parameters can be set via command line options (or in code via <code class=”code”>AVOption</code>s): <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Timeout in ms. </dd> <dt><samp class=”option”>listen</samp></dt> <dd> Create the Unix socket in listening mode. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Select socket type. <dl class=”table”> <dt>‘<samp class=”samp”>stream Corresponds to SOCK_STREAM (reliable stream-oriented)</samp>’</dt> <dt>‘<samp class=”samp”>datagram Corresponds to SOCK_DGRAM (unreliable packet-oriented)</samp>’</dt> <dt>‘<samp class=”samp”>seqpacket Corresponds to SOCK_SEQPACKET (reliable packet-oriented)</samp>’</dt> </dl> </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Maximum packet size for packet-oriented sockets (SOCK_DGRAM and SOCK_SEQPACKET). If greater than zero, this value is used as <code class=”code”>max_packet_size</code>. Ignored for SOCK_STREAM. Default is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>3.48 zmq# TOC</h3>

ZeroMQ asynchronous messaging using the libzmq library. This library supports unicast streaming to multiple clients without relying on an external server. The required syntax for streaming or connecting to a stream is: <pre class=”example-preformatted”>zmq:tcp://ip-address:port </pre>

Example: Create a localhost stream on port 5555: <pre class=”example-preformatted”>ffmpeg -re -i input -f mpegts zmq:tcp://127.0.0.1:5555 </pre>

Multiple clients may connect to the stream using: <pre class=”example-preformatted”>ffplay zmq:tcp://127.0.0.1:5555 </pre>

Streaming to multiple clients is implemented using a ZeroMQ Pub-Sub pattern. The server side binds to a port and publishes data. Clients connect to the server (via IP address/port) and subscribe to the stream. The order in which the server and client start generally does not matter. ffmpeg must be compiled with the –enable-libzmq option to support this protocol. Options can be set on the <code class=”command”>ffmpeg</code>/<code class=”command”>ffplay</code> command line. The following options are supported: <dl class=”table”> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Forces the

maximum packet size for sending/receiving data. The default value is 131,072 bytes. On the server side, this sets the maximum size of sent packets via ZeroMQ. On the clients, it sets an internal buffer size for receiving packets. Note that pkt_size on the clients should be equal to or greater than pkt_size on the server. Otherwise the received message may be truncated causing decoding errors. </dd> </dl>

 <h2 class="chapter">4 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavformat

 <h2 class="chapter">5 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-resampler.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Resampler Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Resampler Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description <a id="toc-Resampler-Options" href="#Resampler-

Options”>2 Resampler Options 3 See Also 4 Authors

 <h2 class=”chapter”>1 Description# TOC</h2>

The FFmpeg resampler provides a high-level interface to the libswresample library audio resampling utilities. In particular it allows one to perform audio resampling, audio channel layout rematrixing, and convert audio format and packing layout.

 <h2 class=”chapter”>2 Resampler Options# TOC</h2>

The audio resampler supports the following named options. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, <var class=”var”>option</var>=<var class=”var”>value</var> for the aresample filter, by setting the value explicitly in the <code class=”code”>SwrContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use. <dl class=”table”> <dt><samp class=”option”>uchl, used_chlayout</samp></dt> <dd> Set used input channel layout. Default is unset. This option is only used for special remapping. </dd> <dt><samp class=”option”>isr, in_sample_rate</samp></dt> <dd> Set the input sample rate. Default value is 0. </dd> <dt><samp class=”option”>osr, out_sample_rate</samp></dt> <dd> Set the output sample rate. Default value is 0. </dd> <dt><samp class=”option”>isf, in_sample_fmt</samp></dt> <dd> Specify the input sample format. It is set by default to <code class=”code”>none</code>. </dd> <dt><samp class=”option”>osf, out_sample_fmt</samp></dt> <dd> Specify the output sample format. It is set by default to <code class=”code”>none</code>. </dd> <dt><samp class=”option”>tsf, internal_sample_fmt</samp></dt> <dd> Set the internal sample format. Default value is <code class=”code”>none</code>. This will automatically be chosen when it is not explicitly set. </dd> <dt><samp class=”option”>ichl, in_chlayout</samp></dt> <dt><samp class=”option”>ochl, out_chlayout</samp></dt> <dd> Set the input/output channel layout. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#channel-layout-syntax”>the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> <dt><samp class=”option”>clev, center_mix_level</samp></dt> <dd> Set the center mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class=”option”>slev,

surround_mix_level</samp></dt> <dd> Set the surround mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">lfe_mix_level</samp></dt> <dd> Set LFE mix into non LFE level. It is used when there is a LFE input but no LFE output. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">rmvol, rematrix_volume</samp></dt> <dd> Set rematrix volume. Default value is 1.0. </dd> <dt><samp class="option">rematrix_maxval</samp></dt> <dd> Set maximum output value for rematrixing. This can be used to prevent clipping vs. preventing volume reduction. A value of 1.0 prevents clipping. </dd> <dt><samp class="option">flags, swr_flags</samp></dt> <dd> Set flags used by the converter. Default value is 0. It supports the following individual flags: <dl class="table"> <dt><samp class="option">res</samp></dt> <dd> force resampling, this flag forces resampling to be used even when the input and output sample rates match. </dd> </dl>

</dd> <dt><samp class="option">dither_scale</samp></dt> <dd> Set the dither scale. Default value is 1. </dd> <dt><samp class="option">dither_method</samp></dt> <dd> Set dither method. Default value is 0. Supported values: <dl class="table"> <dt>‘<samp class="samp">rectangular</samp>’</dt> <dd> select rectangular dither </dd> <dt>‘<samp class="samp">triangular</samp>’</dt> <dd> select triangular dither </dd> <dt>‘<samp class="samp">triangular_hp</samp>’</dt> <dd> select triangular dither with high pass </dd> <dt>‘<samp class="samp">lipshitz</samp>’</dt> <dd> select Lipshitz noise shaping dither. </dd> <dt>‘<samp class="samp">shibata</samp>’</dt> <dd> select Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">low_shibata</samp>’</dt> <dd> select low Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">high_shibata</samp>’</ <dd> select high Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">f_weighted</samp>’</dt> <dd> select f-weighted noise shaping dither </dd> <dt>‘<samp class="samp">modified_e_weighted</samp> <dd> select modified-e-weighted noise shaping dither </dd> <dt>‘<samp class="samp">improved_e_weighted</samp>’</dt> <dd> select improved-e-weighted noise shaping dither </dd> </dl>

</dd> <dt><samp class="option">resampler</samp></dt> <dd> Set resampling engine. Default value is swr. Supported values: <dl class="table"> <dt>‘<samp class="samp">swr</samp>’</dt> <dd> select the native SW Resampler; filter options precision and cheby are not applicable in this case. </dd> <dt>‘<samp class="samp">soxr</samp>’</dt> <dd> select the SoX Resampler (where available); compensation, and filter options filter_size, phase_shift, exact_rational, filter_type & kaiser_beta, are not applicable in this case. </dd> </dl>

</dd>

<dt><samp class=”option”>filter_size</samp></dt>

<dd> For swr only, set resampling filter size, default value is 32.

</dd>

<dt><samp class=”option”>phase_shift</samp></dt>

<dd> For swr only, set resampling phase shift, default value is 10, and must be in

the interval [0,30].

</dd>

<dt><samp class=”option”>linear_interp</samp></dt>

<dd> Use linear interpolation when enabled (the default). Disable it if you want

to preserve speed instead of quality when exact_rational fails.

</dd>

<dt><samp class=”option”>exact_rational</samp></dt>

<dd> For swr only, when enabled, try to use exact phase_count based on input and

output sample rate. However, if it is larger than <code class=”code”>1 << phase_shift</code>,

the phase_count will be <code class=”code”>1 << phase_shift</code> as fallback. Default is enabled.

</dd>

<dt><samp class=”option”>cutoff</samp></dt>

<dd> Set cutoff frequency (swr: 6dB point; soxr: 0dB point) ratio; must be a float

value between 0 and 1. Default value is 0.97 with swr, and 0.91 with soxr

(which, with a sample-rate of 44100, preserves the entire audio band to 20kHz).

</dd>

<dt><samp class=”option”>precision</samp></dt>

<dd> For soxr only, the precision in bits to which the resampled signal will be

calculated. The default value of 20 (which, with suitable dithering, is

appropriate for a destination bit-depth of 16) gives SoX’s ’High Quality’; a

value of 28 gives SoX’s ’Very High Quality’.

</dd>

<dt><samp class=”option”>cheby</samp></dt>

<dd> For soxr only, selects passband rolloff none (Chebyshev) & higher-precision

approximation for ’irrational’ ratios. Default value is 0.

</dd>

<dt><samp class=”option”>async</samp></dt>

<dd> For swr only, simple 1 parameter audio sync to timestamps using stretching,

squeezing, filling and trimming. Setting this to 1 will enable filling and

trimming, larger values represent the maximum amount in samples that the data

may be stretched or squeezed for each second.

Default value is 0, thus no compensation is applied to make the samples match

the audio timestamps.

</dd>

<dt><samp class=”option”>first_pts</samp></dt>

<dd> For swr only, assume the first pts should be this value. The time unit is 1 / sample rate.

This allows for padding/trimming at the start of stream. By default, no

assumption is made about the first frame’s expected pts, so no padding or

trimming is done. For example, this could be set to 0 to pad the beginning with

silence if an audio stream starts after the video stream or to trim any samples

with a negative pts due to encoder delay.

</dd>

<dt><samp class="option">min_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger stretching/squeezing/filling or trimming of the

data to make it match the timestamps. The default is that

stretching/squeezing/filling and trimming is disabled

(<samp class="option">min_comp</samp> = <code class="code">FLT_MAX</code>).

</dd>

<dt><samp class="option">min_hard_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger adding/dropping samples to make it match the

timestamps. This option effectively is a threshold to select between

hard (trim/fill) and soft (squeeze/stretch) compensation. Note that

all compensation is by default disabled through <samp class="option">min_comp</samp>.

The default is 0.1.

</dd>

<dt><samp class="option">comp_duration</samp></dt>

<dd> For swr only, set duration (in seconds) over which data is stretched/squeezed

to make it match the timestamps. Must be a non-negative double float value,

default value is 1.0.

</dd>

<dt><samp class="option">max_soft_comp</samp></dt>

<dd> For swr only, set maximum factor by which data is stretched/squeezed to make it

match the timestamps. Must be a non-negative double float value, default value

is 0.

</dd>

<dt><samp class="option">matrix_encoding</samp></dt>

<dd> Select matrixed stereo encoding.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">none</samp>’</dt>

<dd> select none

</dd>

<dt>‘<samp class="samp">dolby</samp>’</dt>

<dd> select Dolby

</dd>

<dt>‘<samp class="samp">dplii</samp>’</dt>

<dd> select Dolby Pro Logic II

</dd>

</dl>

Default value is <code class="code">none</code>. </dd> <dt><samp class="option">filter_type</samp></dt> <dd> For swr only, select resampling filter type. This only affects resampling operations. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cubic</samp>’</dt> <dd> select cubic </dd> <dt>‘<samp class="samp">blackman_nuttall</samp>’</dt> <dd> select Blackman Nuttall windowed sinc </dd> <dt>‘<samp class="samp">kaiser</samp>’</dt> <dd> select Kaiser windowed sinc </dd> </dl>

</dd> <dt><samp class="option">kaiser_beta</samp></dt> <dd> For swr only, set Kaiser window beta value. Must be a double float value in the interval [2,16], default value is 9. </dd> <dt><samp class="option">output_sample_bits</samp></dt> <dd> For swr only, set number of used output sample bits for dithering. Must be an integer in

the interval [0,64], default value is 0, which means it’s not used.
</dd> </dl>

 <h2 class="chapter">3 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libswresample

 <h2 class="chapter">4 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-scaler.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Scaler Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Scaler Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 Scaler Options 3 See Also 4 Authors

 <h2 class="chapter">1 Description# TOC</h2>

The FFmpeg rescaler provides a high-level interface to the libswscale library image conversion utilities. In particular it allows one to perform image rescaling and pixel format conversion.

 <h2 class="chapter">2 Scaler Options# TOC</h2>

The video scaler supports the following named options. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, with a few API-only exceptions noted below. For programmatic use, they can be set explicitly in the <code class="code">SwsContext</code> options or through the <samp class="file">libavutil/opt.h</samp> API. <dl class="table"> <dd> </dd> <dt><samp class="option">scaler, scaler_sub</samp></dt> <dd> Choose the scaling algorithm to use. Default value is ‘<samp class="samp">auto</samp>’ for both. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Aumotic choice. For ‘<samp class="samp">scaler_sub</samp>’, this means the same algorithm as ‘<samp class="samp">scaler</samp>’. For ‘<samp class="samp">scaler</samp>’, this defaults to the scaler flag selected by ‘<samp class="samp">sws_flags</samp>’. </dd> <dt>‘<samp class="samp">bilinear</samp>’</dt> <dd> Bilinear filter. (AKA triangle filter) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> 2-tap cubic BC-spline (AKA Mitchell-Netravali spline). The B and C parameters can be configured by setting <code class="code">param0</code> and <code class="code">param1</code>, defaulting to 0.0 and 0.6 respectively. </dd> <dt>‘<samp class="samp">point, neighbor</samp>’</dt> <dd> Point sampling (AKA nearest neighbor). </dd> <dt>‘<samp class="samp">area</samp>’</dt> <dd> Area averaging. Equivalent to ‘<samp class="samp">bilinear</samp>’ for upscaling. </dd> <dt>‘<samp class="samp">gaussian</samp>’</dt> <dd> 2-tap Gaussian filter approximation. The sharpness parameter can be configured by setting <code class="code">param0</code>, defaulting to 3.0. </dd> <dt>‘<samp class="samp">sinc</samp>’</dt> <dd> Unwindowed sinc filter. </dd> <dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Lanczos resampling (sinc windowed sinc). The number of filter taps can be configured by setting <code class="code">param0</code>, defaulting to 3.

</dd> <dt>‘<samp class="samp">spline</samp>’</dt>
<dd> Unwindowed natural bicubic spline. </dd> </dl>

</dd> <dt><samp class="option">sws_flags</samp></dt>
<dd> Set the scaler flags. This is also used to set the scaling algorithm, though this usage is deprecated in favor of setting ‘<samp class="samp">scaler</samp>’. Only a single algorithm may be selected. Default value is ‘<samp class="samp">bicubic</samp>’. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">fast_bilinear</samp>’</dt>
<dd> Select fast bilinear scaling algorithm. (Deprecated)
</dd> <dt>‘<samp class="samp">bilinear</samp>’</dt>
<dd> Select bilinear scaling algorithm. (Deprecated) </dd>
<dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> Select bicubic scaling algorithm. (Deprecated) </dd> <dt>‘<samp class="samp">experimental</samp>’</dt> <dd> Select experimental scaling algorithm. (Deprecated) </dd> <dt>‘<samp class="samp">neighbor</samp>’</dt> <dd> Select nearest neighbor rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class="samp">area</samp>’</dt> <dd> Select averaging area rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class="samp">bicublin</samp>’</dt> <dd> Select bicubic scaling algorithm for the luma component, bilinear for chroma components. (Deprecated) </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dd> Select Gaussian rescaling algorithm. (Deprecated)
</dd> <dt>‘<samp class="samp">sinc</samp>’</dt>
<dd> Select sinc rescaling algorithm. (Deprecated) </dd>
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Select Lanczos rescaling algorithm. The default width (alpha) is 3 and can be changed by setting <code class="code">param0</code>. (Deprecated)
</dd> <dt>‘<samp class="samp">spline</samp>’</dt>
<dd> Select natural bicubic spline rescaling algorithm. (Deprecated)
</dd> <dt>‘<samp class="samp">print_info</samp>’</dt>
<dd> Enable printing/debug logging. </dd> <dt>‘<samp class="samp">accurate_rnd</samp>’</dt> <dd> Enable accurate rounding. </dd> <dt>‘<samp class="samp">full_chroma_int</samp>’</dt>
<dd> Enable full chroma interpolation. </dd> <dt>‘<samp class="samp">full_chroma_inp</samp>’</dt> <dd> Select full chroma input. </dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Enable bitexact output. </dd> <dt>‘<samp class="samp">unstable</samp>’</dt> <dd> Allow the use of experimental new code. May subtly affect the output or even produce wrong results. For testing only. </dd> </dl>

</dd> <dt><samp class="option">srcw <var class="var">(API only)</var></samp></dt>
<dd> Set source width. </dd> <dt><samp class="option">srch <var class="var">(API only)</var></samp></dt> <dd> Set

source height. </dd> <dt><samp class="option">dstw <var class="var">(API only)</var></samp></dt> <dd> Set destination width. </dd> <dt><samp class="option">dsth <var class="var">(API only)</var></samp></dt> <dd> Set destination height. </dd> <dt><samp class="option">src_format <var class="var">(API only)</var></samp></dt> <dd> Set source pixel format (must be expressed as an integer). </dd> <dt><samp class="option">dst_format <var class="var">(API only)</var></samp></dt>
<dd> Set destination pixel format (must be expressed as an integer). </dd> <dt><samp class="option">src_range <var class="var">(boolean)</var></samp></dt> <dd> If value is set to <code class="code">1</code>, indicates source is full range. Default value is <code class="code">0</code>, which indicates source is limited range. </dd> <dt><samp class="option">dst_range <var class="var">(boolean)</var></samp></dt> <dd> If value is set to <code class="code">1</code>, enable full range for destination. Default value is <code class="code">0</code>, which enables limited range. </dd> <dt><samp class="option">gamma <var class="var">(boolean)</var></samp></dt> <dd> If value is set to <code class="code">1</code>, enable gamma correct scaling. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">param0, param1</samp></dt> <dd> Set scaling algorithm parameters. The specified values are specific of some scaling algorithms and ignored by others. The specified values are floating point number values. </dd>
<dt><samp class="option">sws_dither</samp></dt> <dd> Set the dithering algorithm. Accepts one of the following values. Default value is ‘<samp class="samp">auto</samp>’. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> automatic choice </dd> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> no dithering </dd> <dt>‘<samp class="samp">bayer</samp>’</dt>
<dd> bayer dither </dd> <dt>‘<samp class="samp">ed</samp>’</dt>
<dd> error diffusion dither </dd> <dt>‘<samp class="samp">a_dither</samp>’<
<dd> arithmetic dither, based using addition </dd> <dt>‘<samp class="samp">x_dither</samp>’</dt> <dd> arithmetic dither, based using xor (more random/less apparent patterning that a_dither).
</dd> </dl>

</dd> <dt><samp class="option">alphablend</samp></dt> <dd> Set the alpha blending to use when the input has alpha but the output does not. Default value is ‘<samp class="samp">none</samp>’. <dl class="table"> <dt>‘<samp class="samp">uniform_color</samp>’</dt>
<dd> Blend onto a uniform background color </dd> <dt>‘<samp class="samp">checkerboard</samp>’</dt> <dd> Blend onto a checkerboard </dd> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> No blending </dd> </dl>

</dd> <dt><samp class="option">sws_backends</samp></dt> <dd> Set the allowed swscale backends. This is a flags option, so multiple backends may be combined. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection. Equal to either ‘<samp class="samp">stable</samp>’ or ‘<samp class="samp">all</samp>’ depending on whether or not the ‘<samp class="samp">unstable</samp>’ flag is set. This is the default value. </dd> <dt>‘<samp class="samp">stable</samp>’</dt> <dd> All stable backends. </dd> <dt>‘<samp class="samp">unstable</samp>’</dt> <dd> All unstable backends. </dd> <dt>‘<samp class="samp">all</samp>’</dt> <dd> All available backends. </dd> <dt>‘<samp class="samp">legacy</samp>’</dt> <dd> Legacy swscale code. </dd> <dt>‘<samp class="samp">c</samp>’</dt> <dd> Template-based reference code. </dd> <dt>‘<samp class="samp">memcpy</samp>’</dt> <dd> Fast path using libc <code class="code">memcpy</code>. </dd> <dt>‘<samp class="samp">x86</samp>’</dt> <dd> x86 SIMD kernels. </dd> <dt>‘<samp class="samp">aarch64</samp>’</d <dd> AArch64 NEON kernels. </dd> <dt>‘<samp class="samp">spirv</samp>’</dt> <dd> Vulkan SPIR-V backend. </dd> </dl>

</dd> </dl>

 <h2 class="chapter">3 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libswscale

 <h2 class="chapter">4 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg-utils.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Utilities Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Utilities Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 Syntax <ul class="toc-numbered-mark"> 2.1 Quoting and escaping <ul class="toc-numbered-mark"> 2.1.1 Examples 2.2 Date 2.3 Time duration <ul class="toc-numbered-mark"> 2.3.1 Examples 2.4 Video size 2.5 Video rate 2.6 Ratio 2.7 Color 2.8 Channel Layout 3 Expression Evaluation 4 See Also 5 Authors

 <h2 class="chapter">1 Description# TOC</h2>

This document describes some generic features and utilities provided by the libavutil library.

 <h2 class="chapter">2 Syntax# TOC</h2>

This section documents the syntax and formats employed by the FFmpeg libraries and tools. <a

name=”Quoting-and-escaping”> <h3 class=”section”>2.1 Quoting and escaping# TOC</h3>

FFmpeg adopts the following quoting and escaping mechanism, unless explicitly specified. The following rules are applied: <ul class=”itemize mark-bullet”> ‘<samp class=”samp”>'</samp>’ and ‘<samp class=”samp”>\</samp>’ are special characters (respectively used for quoting and escaping). In addition to them, there might be other special characters depending on the specific syntax where the escaping and quoting are employed.

A special character is escaped by prefixing it with a ‘<samp class=”samp”>\</samp>’.

All characters enclosed between ‘<samp class=”samp”>''</samp>’ are included literally in the parsed string. The quote character ‘<samp class=”samp”>'</samp>’ itself cannot be quoted, so you may need to close the quote and escape it.

Leading and trailing whitespaces, unless escaped or quoted, are removed from the parsed string.

Note that you may need to add a second level of escaping when using the command line or a script, which depends on the syntax of the adopted shell language. The function <code class=”code”>av_get_token</code> defined in <samp class=”file”>libavutil/avstring.h</samp> can be used to parse a token quoted or escaped according to the rules defined above. The tool <samp class=”file”>tools/ffescape</samp> in the FFmpeg source tree can be used to automatically quote or escape a string in a script. <h4 class=”subsection”>2.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Escape the string <code class=”code”>Crime d'Amour</code> containing the <code class=”code”>'</code> special character: <pre class=”example-preformatted”>Crime d\'Amour </pre>

The string above contains a quote, so the <code class=”code”>'</code> needs to be escaped when quoting it: <pre class=”example-preformatted”>'Crime d'\''Amour' </pre>

Include leading or trailing whitespaces using quoting: <pre class=”example-preformatted”>' this string starts and ends with whitespaces ' </pre>

Escaping and quoting can be mixed together:

<pre class="example-preformatted">' The string '\'string\'' is a string '
</pre>

To include a literal ‘<samp class="samp">\</samp>’ you can use either escaping or quoting: <pre class="example-preformatted">'c:\foo' can be written as c:\\foo </pre>

 <h3 class="section">2.2 Date# TOC</h3>

The accepted syntax is: <pre class="example-preformatted">[(YYYY-MM-DD|YYYYMMDD)[T|t|]](HH:MM:SS[.m...]]])|(HHMMSS[.m...]]]))[Z]
now </pre>

If the value is "now" it takes the current time.
Time is local time unless Z is appended, in which case it is interpreted as UTC. If the year-month-day part is not specified it takes the current year-month-day. <h3 class="section">2.3 Time duration# TOC</h3>

There are two accepted syntaxes for expressing time duration.
 <pre class="example-preformatted">[-][<var class="var">HH</var>:]<var class="var">MM</var>:<var class="var">SS</var>[.<var class="var">m</var>...] </pre>

<var class="var">HH</var> expresses the number of hours, <var class="var">MM</var> the number of minutes for a maximum of 2 digits, and <var class="var">SS</var> the number of seconds for a maximum of 2 digits. The <var class="var">m</var> at the end expresses decimal value for <var class="var">SS</var>. or   <pre class="example-preformatted">[-]<var class="var">S</var>+[.<var class="var">m</var>...][s|ms|us]
</pre>

<var class="var">S</var> expresses the number of seconds, with the optional decimal part <var class="var">m</var>. The optional literal suffixes ‘<samp class="samp">s</samp>’, ‘<samp class="samp">ms</samp>’ or ‘<samp class="samp">us</samp>’ indicate to interpret the value as seconds, milliseconds or microseconds, respectively. In both expressions, the optional ‘<samp class="samp">-</samp>’ indicates negative duration. <h4 class="subsection">2.3.1 Examples<a class="anchor hidden-xs" href="#Examples-1" aria-

hidden=”true”># TOC</h4>

The following examples are all valid time duration: <dl class=”table”> <dt>‘<samp class=”samp”>55</samp>’</dt> <dd> 55 seconds </dd> <dt>‘<samp class=”samp”>0.2</samp>’</dt> <dd> 0.2 seconds </dd> <dt>‘<samp class=”samp”>200ms</samp>’</dt> <dd> 200 milliseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>200000us</samp>’</dt> <dd> 200000 microseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>12:03:45</samp>’</dt> <dd> 12 hours, 03 minutes and 45 seconds </dd> <dt>‘<samp class=”samp”>23.189</samp>’</dt> <dd> 23.189 seconds </dd> </dl>

<h3 class=”section”>2.4 Video size# TOC</h3>

Specify the size of the sourced video, it may be a string of the form

<var class=”var”>width</var>x<var class=”var”>height</var>, or the name of a size abbreviation.

The following abbreviations are recognized:

<dl class=”table”>

<dt>‘<samp class=”samp”>ntsc</samp>’</dt>

<dd> 720x480

</dd>

<dt>‘<samp class=”samp”>pal</samp>’</dt>

<dd> 720x576

</dd>

<dt>‘<samp class=”samp”>qntsc</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class=”samp”>qpal</samp>’</dt>

<dd> 352x288

</dd>

<dt>‘<samp class=”samp”>sntsc</samp>’</dt>

<dd> 640x480
</dd>
<dt>‘<samp class="samp">spal</samp>’</dt>
<dd> 768x576
</dd>
<dt>‘<samp class="samp">film</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">ntsc-film</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">sqcif</samp>’</dt>
<dd> 128x96
</dd>
<dt>‘<samp class="samp">qcif</samp>’</dt>
<dd> 176x144
</dd>
<dt>‘<samp class="samp">cif</samp>’</dt>
<dd> 352x288
</dd>
<dt>‘<samp class="samp">4cif</samp>’</dt>
<dd> 704x576
</dd>
<dt>‘<samp class="samp">16cif</samp>’</dt>
<dd> 1408x1152
</dd>
<dt>‘<samp class="samp">qqvga</samp>’</dt>
<dd> 160x120
</dd>
<dt>‘<samp class="samp">qvga</samp>’</dt>
<dd> 320x240

</dd>
<dt>‘<samp class="samp">vga</samp>’</dt>
<dd> 640x480
</dd>
<dt>‘<samp class="samp">svga</samp>’</dt>
<dd> 800x600
</dd>
<dt>‘<samp class="samp">xga</samp>’</dt>
<dd> 1024x768
</dd>
<dt>‘<samp class="samp">uxga</samp>’</dt>
<dd> 1600x1200
</dd>
<dt>‘<samp class="samp">qxga</samp>’</dt>
<dd> 2048x1536
</dd>
<dt>‘<samp class="samp">sxga</samp>’</dt>
<dd> 1280x1024
</dd>
<dt>‘<samp class="samp">qsxga</samp>’</dt>
<dd> 2560x2048
</dd>
<dt>‘<samp class="samp">hsxga</samp>’</dt>
<dd> 5120x4096
</dd>
<dt>‘<samp class="samp">wvga</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">wxga</samp>’</dt>
<dd> 1366x768
</dd>

<dt>‘<samp class=”samp”>wsxga</samp>’</dt>
<dd> 1600x1024
</dd>
<dt>‘<samp class=”samp”>wuxga</samp>’</dt>
<dd> 1920x1200
</dd>
<dt>‘<samp class=”samp”>woxga</samp>’</dt>
<dd> 2560x1600
</dd>
<dt>‘<samp class=”samp”>wqsxga</samp>’</dt>
<dd> 3200x2048
</dd>
<dt>‘<samp class=”samp”>wquxga</samp>’</dt>
<dd> 3840x2400
</dd>
<dt>‘<samp class=”samp”>whsxga</samp>’</dt>
<dd> 6400x4096
</dd>
<dt>‘<samp class=”samp”>whuxga</samp>’</dt>
<dd> 7680x4800
</dd>
<dt>‘<samp class=”samp”>cga</samp>’</dt>
<dd> 320x200
</dd>
<dt>‘<samp class=”samp”>ega</samp>’</dt>
<dd> 640x350
</dd>
<dt>‘<samp class=”samp”>hd480</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class=”samp”>hd720</samp>’</dt>

<dd> 1280x720
</dd>
<dt>‘<samp class="samp">hd1080</samp>’</dt>
<dd> 1920x1080
</dd>
<dt>‘<samp class="samp">2k</samp>’</dt>
<dd> 2048x1080
</dd>
<dt>‘<samp class="samp">2kflat</samp>’</dt>
<dd> 1998x1080
</dd>
<dt>‘<samp class="samp">2kscope</samp>’</dt>
<dd> 2048x858
</dd>
<dt>‘<samp class="samp">4k</samp>’</dt>
<dd> 4096x2160
</dd>
<dt>‘<samp class="samp">4kflat</samp>’</dt>
<dd> 3996x2160
</dd>
<dt>‘<samp class="samp">4kscope</samp>’</dt>
<dd> 4096x1716
</dd>
<dt>‘<samp class="samp">nhd</samp>’</dt>
<dd> 640x360
</dd>
<dt>‘<samp class="samp">hqvga</samp>’</dt>
<dd> 240x160
</dd>
<dt>‘<samp class="samp">wqvga</samp>’</dt>
<dd> 400x240

</dd>

<dt>‘<samp class="samp">fwqvga</samp>’</dt>

<dd> 432x240

</dd>

<dt>‘<samp class="samp">hvga</samp>’</dt>

<dd> 480x320

</dd>

<dt>‘<samp class="samp">qhd</samp>’</dt>

<dd> 960x540

</dd>

<dt>‘<samp class="samp">2kdci</samp>’</dt>

<dd> 2048x1080

</dd>

<dt>‘<samp class="samp">4kdci</samp>’</dt>

<dd> 4096x2160

</dd>

<dt>‘<samp class="samp">uhd2160</samp>’</dt>

<dd> 3840x2160

</dd>

<dt>‘<samp class="samp">uhd4320</samp>’</dt>

<dd> 7680x4320

</dd>

</dl>

<h3 class="section">2.5 Video rate<a class="anchor
hidden-xs" href="#Video-rate" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-Video-rate" aria-hidden="true">TOC</h3>

Specify the frame rate of a video, expressed as the number of
frames generated per second. It has to be a string in the format <var
class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>,
an integer number, a float number or a valid video frame rate abbre-
viation. The following abbreviations are recognized: <dl
class="table"> <dt>‘<samp class="samp">ntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">pal</samp>’</dt>

<dd> 25/1 </dd> <dt>‘<samp class=”samp”>qntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class=”samp”>qpal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class=”samp”>sntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class=”samp”>spal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class=”samp”>film</samp>’</dt>
<dd> 24/1 </dd> <dt>‘<samp class=”samp”>ntsc-film</samp>’</dt> <dd> 24000/1001 </dd> </dl>

 <h3 class=”section”>2.6 Ratio# TOC</h3>

A ratio can be expressed as an expression, or in the form <var class=”var”>numerator</var>:<var class=”var”>denominator</var>.
Note that a ratio with infinite (1/0) or negative value is considered valid, so you should check on the returned value if you want to exclude those values.
The undefined value can be expressed using the "0:0" string. <h3 class=”section”>2.7 Color# TOC</h3>

It can be the name of a color as defined below (case insensitive match) or a

<code class=”code”>[0x|#]RRGGBB[AA]</code> sequence, possibly followed by @ and a string

representing the alpha component.

The alpha component may be a string composed by "0x" followed by an

hexadecimal number or a decimal number between 0.0 and 1.0, which

represents the opacity value (‘<samp class=”samp”>0x00</samp>’ or ‘<samp class=”samp”>0.0</samp>’ means completely

transparent, ‘<samp class=”samp”>0xff</samp>’ or ‘<samp class=”samp”>1.0</samp>’ completely opaque). If the alpha

component is not specified then ‘<samp class=”samp”>0xff</samp>’ is assumed.

The string ‘<samp class=”samp”>random</samp>’ will result in a random color.

The following names of colors are recognized:
<dl class="table">
<dt>‘<samp class="samp">AliceBlue</samp>’</dt>
<dd> 0xF0F8FF
</dd>
<dt>‘<samp class="samp">AntiqueWhite</samp>’</dt>
<dd> 0xFAEBD7
</dd>
<dt>‘<samp class="samp">Aqua</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class="samp">Aquamarine</samp>’</dt>
<dd> 0x7FFFD4
</dd>
<dt>‘<samp class="samp">Azure</samp>’</dt>
<dd> 0xF0FFFF
</dd>
<dt>‘<samp class="samp">Beige</samp>’</dt>
<dd> 0xF5F5DC
</dd>
<dt>‘<samp class="samp">Bisque</samp>’</dt>
<dd> 0xFFE4C4
</dd>
<dt>‘<samp class="samp">Black</samp>’</dt>
<dd> 0x000000
</dd>
<dt>‘<samp class="samp">BlanchedAlmond</samp>’</dt>
<dd> 0xFFEBCD
</dd>
<dt>‘<samp class="samp">Blue</samp>’</dt>
<dd> 0x0000FF

</dd>
<dt>‘<samp class=”samp”>BlueViolet</samp>’</dt>
<dd> 0x8A2BE2
</dd>
<dt>‘<samp class=”samp”>Brown</samp>’</dt>
<dd> 0xA52A2A
</dd>
<dt>‘<samp class=”samp”>BurlyWood</samp>’</dt>
<dd> 0xDEB887
</dd>
<dt>‘<samp class=”samp”>CadetBlue</samp>’</dt>
<dd> 0x5F9EA0
</dd>
<dt>‘<samp class=”samp”>Chartreuse</samp>’</dt>
<dd> 0x7FFF00
</dd>
<dt>‘<samp class=”samp”>Chocolate</samp>’</dt>
<dd> 0xD2691E
</dd>
<dt>‘<samp class=”samp”>Coral</samp>’</dt>
<dd> 0xFF7F50
</dd>
<dt>‘<samp class=”samp”>CornflowerBlue</samp>’</dt>
<dd> 0x6495ED
</dd>
<dt>‘<samp class=”samp”>Cornsilk</samp>’</dt>
<dd> 0xFFF8DC
</dd>
<dt>‘<samp class=”samp”>Crimson</samp>’</dt>
<dd> 0xDC143C
</dd>

<dt>‘<samp class=”samp”>Cyan</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class=”samp”>DarkBlue</samp>’</dt>
<dd> 0x00008B
</dd>
<dt>‘<samp class=”samp”>DarkCyan</samp>’</dt>
<dd> 0x008B8B
</dd>
<dt>‘<samp class=”samp”>DarkGoldenRod</samp>’</dt>
<dd> 0xB8860B
</dd>
<dt>‘<samp class=”samp”>DarkGray</samp>’</dt>
<dd> 0xA9A9A9
</dd>
<dt>‘<samp class=”samp”>DarkGreen</samp>’</dt>
<dd> 0x006400
</dd>
<dt>‘<samp class=”samp”>DarkKhaki</samp>’</dt>
<dd> 0xBDB76B
</dd>
<dt>‘<samp class=”samp”>DarkMagenta</samp>’</dt>
<dd> 0x8B008B
</dd>
<dt>‘<samp class=”samp”>DarkOliveGreen</samp>’</dt>
<dd> 0x556B2F
</dd>
<dt>‘<samp class=”samp”>Darkorange</samp>’</dt>
<dd> 0xFF8C00
</dd>
<dt>‘<samp class=”samp”>DarkOrchid</samp>’</dt>

<dd> 0x9932CC
</dd>
<dt>‘<samp class=”samp”>DarkRed</samp>’</dt>
<dd> 0x8B0000
</dd>
<dt>‘<samp class=”samp”>DarkSalmon</samp>’</dt>
<dd> 0xE9967A
</dd>
<dt>‘<samp class=”samp”>DarkSeaGreen</samp>’</dt>
<dd> 0x8FBC8F
</dd>
<dt>‘<samp class=”samp”>DarkSlateBlue</samp>’</dt>
<dd> 0x483D8B
</dd>
<dt>‘<samp class=”samp”>DarkSlateGray</samp>’</dt>
<dd> 0x2F4F4F
</dd>
<dt>‘<samp class=”samp”>DarkTurquoise</samp>’</dt>
<dd> 0x00CED1
</dd>
<dt>‘<samp class=”samp”>DarkViolet</samp>’</dt>
<dd> 0x9400D3
</dd>
<dt>‘<samp class=”samp”>DeepPink</samp>’</dt>
<dd> 0xFF1493
</dd>
<dt>‘<samp class=”samp”>DeepSkyBlue</samp>’</dt>
<dd> 0x00BFFF
</dd>
<dt>‘<samp class=”samp”>DimGray</samp>’</dt>
<dd> 0x696969

</dd>
<dt>‘<samp class=”samp”>DodgerBlue</samp>’</dt>
<dd> 0x1E90FF
</dd>
<dt>‘<samp class=”samp”>FireBrick</samp>’</dt>
<dd> 0xB22222
</dd>
<dt>‘<samp class=”samp”>FloralWhite</samp>’</dt>
<dd> 0xFFFAF0
</dd>
<dt>‘<samp class=”samp”>ForestGreen</samp>’</dt>
<dd> 0x228B22
</dd>
<dt>‘<samp class=”samp”>Fuchsia</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class=”samp”>Gainsboro</samp>’</dt>
<dd> 0xDCDCDC
</dd>
<dt>‘<samp class=”samp”>GhostWhite</samp>’</dt>
<dd> 0xF8F8FF
</dd>
<dt>‘<samp class=”samp”>Gold</samp>’</dt>
<dd> 0xFFD700
</dd>
<dt>‘<samp class=”samp”>GoldenRod</samp>’</dt>
<dd> 0xDAA520
</dd>
<dt>‘<samp class=”samp”>Gray</samp>’</dt>
<dd> 0x808080
</dd>

<dt>‘<samp class=”samp”>Green</samp>’</dt>
<dd> 0x008000
</dd>
<dt>‘<samp class=”samp”>GreenYellow</samp>’</dt>
<dd> 0xADFF2F
</dd>
<dt>‘<samp class=”samp”>HoneyDew</samp>’</dt>
<dd> 0xF0FFF0
</dd>
<dt>‘<samp class=”samp”>HotPink</samp>’</dt>
<dd> 0xFF69B4
</dd>
<dt>‘<samp class=”samp”>IndianRed</samp>’</dt>
<dd> 0xCD5C5C
</dd>
<dt>‘<samp class=”samp”>Indigo</samp>’</dt>
<dd> 0x4B0082
</dd>
<dt>‘<samp class=”samp”>Ivory</samp>’</dt>
<dd> 0xFFFFF0
</dd>
<dt>‘<samp class=”samp”>Khaki</samp>’</dt>
<dd> 0xF0E68C
</dd>
<dt>‘<samp class=”samp”>Lavender</samp>’</dt>
<dd> 0xE6E6FA
</dd>
<dt>‘<samp class=”samp”>LavenderBlush</samp>’</dt>
<dd> 0xFFF0F5
</dd>
<dt>‘<samp class=”samp”>LawnGreen</samp>’</dt>

<dd> 0x7CFC00
</dd>
<dt>‘<samp class=”samp”>LemonChiffon</samp>’</dt>
<dd> 0xFFFACD
</dd>
<dt>‘<samp class=”samp”>LightBlue</samp>’</dt>
<dd> 0xADD8E6
</dd>
<dt>‘<samp class=”samp”>LightCoral</samp>’</dt>
<dd> 0xF08080
</dd>
<dt>‘<samp class=”samp”>LightCyan</samp>’</dt>
<dd> 0xE0FFFF
</dd>
<dt>‘<samp class=”samp”>LightGoldenRodYellow</samp>’</dt>
<dd> 0xFAFAD2
</dd>
<dt>‘<samp class=”samp”>LightGreen</samp>’</dt>
<dd> 0x90EE90
</dd>
<dt>‘<samp class=”samp”>LightGrey</samp>’</dt>
<dd> 0xD3D3D3
</dd>
<dt>‘<samp class=”samp”>LightPink</samp>’</dt>
<dd> 0xFFB6C1
</dd>
<dt>‘<samp class=”samp”>LightSalmon</samp>’</dt>
<dd> 0xFFA07A
</dd>
<dt>‘<samp class=”samp”>LightSeaGreen</samp>’</dt>
<dd> 0x20B2AA

</dd>
<dt>‘<samp class="samp">LightSkyBlue</samp>’</dt>
<dd> 0x87CEFA
</dd>
<dt>‘<samp class="samp">LightSlateGray</samp>’</dt>
<dd> 0x778899
</dd>
<dt>‘<samp class="samp">LightSteelBlue</samp>’</dt>
<dd> 0xB0C4DE
</dd>
<dt>‘<samp class="samp">LightYellow</samp>’</dt>
<dd> 0xFFFFE0
</dd>
<dt>‘<samp class="samp">Lime</samp>’</dt>
<dd> 0x00FF00
</dd>
<dt>‘<samp class="samp">LimeGreen</samp>’</dt>
<dd> 0x32CD32
</dd>
<dt>‘<samp class="samp">Linen</samp>’</dt>
<dd> 0xFAF0E6
</dd>
<dt>‘<samp class="samp">Magenta</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class="samp">Maroon</samp>’</dt>
<dd> 0x800000
</dd>
<dt>‘<samp class="samp">MediumAquaMarine</samp>’</dt>
<dd> 0x66CDAA
</dd>

<dt>‘<samp class=”samp”>MediumBlue</samp>’</dt>
<dd> 0x0000CD
</dd>
<dt>‘<samp class=”samp”>MediumOrchid</samp>’</dt>
<dd> 0xBA55D3
</dd>
<dt>‘<samp class=”samp”>MediumPurple</samp>’</dt>
<dd> 0x9370D8
</dd>
<dt>‘<samp class=”samp”>MediumSeaGreen</samp>’</dt>
<dd> 0x3CB371
</dd>
<dt>‘<samp class=”samp”>MediumSlateBlue</samp>’</dt>
<dd> 0x7B68EE
</dd>
<dt>‘<samp class=”samp”>MediumSpringGreen</samp>’</dt>
<dd> 0x00FA9A
</dd>
<dt>‘<samp class=”samp”>MediumTurquoise</samp>’</dt>
<dd> 0x48D1CC
</dd>
<dt>‘<samp class=”samp”>MediumVioletRed</samp>’</dt>
<dd> 0xC71585
</dd>
<dt>‘<samp class=”samp”>MidnightBlue</samp>’</dt>
<dd> 0x191970
</dd>
<dt>‘<samp class=”samp”>MintCream</samp>’</dt>
<dd> 0xF5FFFA
</dd>
<dt>‘<samp class=”samp”>MistyRose</samp>’</dt>

<dd> 0xFFE4E1
</dd>
<dt>‘<samp class=”samp”>Moccasin</samp>’</dt>
<dd> 0xFFE4B5
</dd>
<dt>‘<samp class=”samp”>NavajoWhite</samp>’</dt>
<dd> 0xFFDEAD
</dd>
<dt>‘<samp class=”samp”>Navy</samp>’</dt>
<dd> 0x000080
</dd>
<dt>‘<samp class=”samp”>OldLace</samp>’</dt>
<dd> 0xFDF5E6
</dd>
<dt>‘<samp class=”samp”>Olive</samp>’</dt>
<dd> 0x808000
</dd>
<dt>‘<samp class=”samp”>OliveDrab</samp>’</dt>
<dd> 0x6B8E23
</dd>
<dt>‘<samp class=”samp”>Orange</samp>’</dt>
<dd> 0xFFA500
</dd>
<dt>‘<samp class=”samp”>OrangeRed</samp>’</dt>
<dd> 0xFF4500
</dd>
<dt>‘<samp class=”samp”>Orchid</samp>’</dt>
<dd> 0xDA70D6
</dd>
<dt>‘<samp class=”samp”>PaleGoldenRod</samp>’</dt>
<dd> 0xEEE8AA

</dd>
<dt>‘<samp class="samp">PaleGreen</samp>’</dt>
<dd> 0x98FB98
</dd>
<dt>‘<samp class="samp">PaleTurquoise</samp>’</dt>
<dd> 0xAFEEEE
</dd>
<dt>‘<samp class="samp">PaleVioletRed</samp>’</dt>
<dd> 0xD87093
</dd>
<dt>‘<samp class="samp">PapayaWhip</samp>’</dt>
<dd> 0xFFEFD5
</dd>
<dt>‘<samp class="samp">PeachPuff</samp>’</dt>
<dd> 0xFFDAB9
</dd>
<dt>‘<samp class="samp">Peru</samp>’</dt>
<dd> 0xCD853F
</dd>
<dt>‘<samp class="samp">Pink</samp>’</dt>
<dd> 0xFFC0CB
</dd>
<dt>‘<samp class="samp">Plum</samp>’</dt>
<dd> 0xDDA0DD
</dd>
<dt>‘<samp class="samp">PowderBlue</samp>’</dt>
<dd> 0xB0E0E6
</dd>
<dt>‘<samp class="samp">Purple</samp>’</dt>
<dd> 0x800080
</dd>

<dt>‘<samp class=”samp”>Red</samp>’</dt>
<dd> 0xFF0000
</dd>
<dt>‘<samp class=”samp”>RosyBrown</samp>’</dt>
<dd> 0xBC8F8F
</dd>
<dt>‘<samp class=”samp”>RoyalBlue</samp>’</dt>
<dd> 0x4169E1
</dd>
<dt>‘<samp class=”samp”>SaddleBrown</samp>’</dt>
<dd> 0x8B4513
</dd>
<dt>‘<samp class=”samp”>Salmon</samp>’</dt>
<dd> 0xFA8072
</dd>
<dt>‘<samp class=”samp”>SandyBrown</samp>’</dt>
<dd> 0xF4A460
</dd>
<dt>‘<samp class=”samp”>SeaGreen</samp>’</dt>
<dd> 0x2E8B57
</dd>
<dt>‘<samp class=”samp”>SeaShell</samp>’</dt>
<dd> 0xFFF5EE
</dd>
<dt>‘<samp class=”samp”>Sienna</samp>’</dt>
<dd> 0xA0522D
</dd>
<dt>‘<samp class=”samp”>Silver</samp>’</dt>
<dd> 0xC0C0C0
</dd>
<dt>‘<samp class=”samp”>SkyBlue</samp>’</dt>

<dd> 0x87CEEB
</dd>
<dt>‘<samp class=”samp”>SlateBlue</samp>’</dt>
<dd> 0x6A5ACD
</dd>
<dt>‘<samp class=”samp”>SlateGray</samp>’</dt>
<dd> 0x708090
</dd>
<dt>‘<samp class=”samp”>Snow</samp>’</dt>
<dd> 0xFFFAFA
</dd>
<dt>‘<samp class=”samp”>SpringGreen</samp>’</dt>
<dd> 0x00FF7F
</dd>
<dt>‘<samp class=”samp”>SteelBlue</samp>’</dt>
<dd> 0x4682B4
</dd>
<dt>‘<samp class=”samp”>Tan</samp>’</dt>
<dd> 0xD2B48C
</dd>
<dt>‘<samp class=”samp”>Teal</samp>’</dt>
<dd> 0x008080
</dd>
<dt>‘<samp class=”samp”>Thistle</samp>’</dt>
<dd> 0xD8BFD8
</dd>
<dt>‘<samp class=”samp”>Tomato</samp>’</dt>
<dd> 0xFF6347
</dd>
<dt>‘<samp class=”samp”>Turquoise</samp>’</dt>
<dd> 0x40E0D0

</dd>

<dt>‘<samp class="samp">Violet</samp>’</dt>

<dd> 0xEE82EE

</dd>

<dt>‘<samp class="samp">Wheat</samp>’</dt>

<dd> 0xF5DEB3

</dd>

<dt>‘<samp class="samp">White</samp>’</dt>

<dd> 0xFFFFFF

</dd>

<dt>‘<samp class="samp">WhiteSmoke</samp>’</dt>

<dd> 0xF5F5F5

</dd>

<dt>‘<samp class="samp">Yellow</samp>’</dt>

<dd> 0xFFFF00

</dd>

<dt>‘<samp class="samp">YellowGreen</samp>’</dt>

<dd> 0x9ACD32

</dd>

</dl>

 <h3 class="section">2.8 Channel Layout# TOC</h3>

A channel layout specifies the spatial disposition of the channels in a multi-channel audio stream. To specify a channel layout, FFmpeg makes use of a special syntax. Individual channels are identified by an id, as given by the table below: <dl class="table"> <dt>‘<samp class="samp">FL</samp>’</dt> <dd> front left </dd> <dt>‘<samp class="samp">FR</samp>’</dt> <dd> front right </dd> <dt>‘<samp class="samp">FC</samp>’</dt> <dd> front center </dd> <dt>‘<samp class="samp">LFE</samp>’</dt> <dd> low frequency </dd> <dt>‘<samp class="samp">BL</samp>’</dt> <dd> back left </dd> <dt>‘<samp class="samp">BR</samp>’</dt>

<dd> back right </dd> <dt>‘<samp class="samp">FLC</samp>’</dt>
<dd> front left-of-center </dd> <dt>‘<samp class="samp">FRC</samp>’</dt>
<dd> front right-of-center </dd> <dt>‘<samp class="samp">BC</samp>’</dt>
<dd> back center </dd> <dt>‘<samp class="samp">SL</samp>’</dt>
<dd> side left </dd> <dt>‘<samp class="samp">SR</samp>’</dt>
<dd> side right </dd> <dt>‘<samp class="samp">TC</samp>’</dt>
<dd> top center </dd> <dt>‘<samp class="samp">TFL</samp>’</dt>
<dd> top front left </dd> <dt>‘<samp class="samp">TFC</samp>’</dt>
<dd> top front center </dd> <dt>‘<samp class="samp">TFR</samp>’</dt>
<dd> top front right </dd> <dt>‘<samp class="samp">TBL</samp>’</dt>
<dd> top back left </dd> <dt>‘<samp class="samp">TBC</samp>’</dt>
<dd> top back center </dd> <dt>‘<samp class="samp">TBR</samp>’</dt>
<dd> top back right </dd> <dt>‘<samp class="samp">DL</samp>’</dt>
<dd> downmix left </dd> <dt>‘<samp class="samp">DR</samp>’</dt>
<dd> downmix right </dd> <dt>‘<samp class="samp">WL</samp>’</dt>
<dd> wide left </dd> <dt>‘<samp class="samp">WR</samp>’</dt>
<dd> wide right </dd> <dt>‘<samp class="samp">SDL</samp>’</dt>
<dd> surround direct left </dd> <dt>‘<samp class="samp">SDR</samp>’</dt>
<dd> surround direct right </dd> <dt>‘<samp class="samp">LFE2</samp>’</dt
<dd> low frequency 2 </dd> </dl>

Standard channel layout compositions can be specified by using the

following identifiers:

<dl class="table">

<dt>‘<samp class="samp">mono</samp>’</dt>

<dd> FC

</dd>

<dt>‘<samp class="samp">stereo</samp>’</dt>

<dd> FL+FR

</dd>

<dt>‘<samp class="samp">2.1</samp>’</dt>

<dd> FL+FR+LFE

</dd>

<dt>‘<samp class="samp">3.0</samp>’</dt>

<dd> FL+FR+FC

</dd>

<dt>‘<samp class="samp">3.0(back)</samp>’</dt>

<dd> FL+FR+BC

</dd>
<dt>‘<samp class="samp">4.0</samp>’</dt>
<dd> FL+FR+FC+BC
</dd>
<dt>‘<samp class="samp">quad</samp>’</dt>
<dd> FL+FR+BL+BR
</dd>
<dt>‘<samp class="samp">quad(side)</samp>’</dt>
<dd> FL+FR+SL+SR
</dd>
<dt>‘<samp class="samp">3.1</samp>’</dt>
<dd> FL+FR+FC+LFE
</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR
</dd>
<dt>‘<samp class="samp">5.0(side)</samp>’</dt>
<dd> FL+FR+FC+SL+SR
</dd>
<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR
</dd>
<dt>‘<samp class="samp">5.1(side)</samp>’</dt>
<dd> FL+FR+FC+LFE+SL+SR
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> FL+FR+FC+BC+SL+SR
</dd>

<dt>‘<samp class="samp">6.0(front)</samp>’</dt>
<dd> FL+FR+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">3.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+TFL+TFR
</dd>
<dt>‘<samp class="samp">hexagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1(front)</samp>’</dt>
<dd> FL+FR+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.0(front)</samp>’</dt>
<dd> FL+FR+FC+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.1(wide)</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC
</dd>
<dt>‘<samp class="samp">7.1(wide-side)</samp>’</dt>

<dd> FL+FR+FC+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">5.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR
</dd>
<dt>‘<samp class="samp">octagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR
</dd>
<dt>‘<samp class="samp">cube</samp>’</dt>
<dd> FL+FR+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">5.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR
</dd>
<dt>‘<samp class="samp">7.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.2.3</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBC+LFE2
</dd>
<dt>‘<samp class="samp">9.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">9.1.6</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR+TSL+TSR
</dd>
<dt>‘<samp class="samp">hexadecagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR+WL+WR+TBL+TBR+TBC+TFC+TFL+TFR

</dd>

<dt>‘<samp class="samp">binaural</samp>’</dt>

<dd> BIL+BIR

</dd>

<dt>‘<samp class="samp">downmix</samp>’</dt>

<dd> DL+DR

</dd>

<dt>‘<samp class="samp">22.2</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+BC+SL+SR+TC+TFL+TFC+TFR+TBL+TBC+TBR

</dd>

</dl>

A custom channel layout can be specified as a sequence of terms, separated by ’+’. Each term can be: <ul class="itemize mark-bullet"> the name of a single channel (e.g. ‘<samp class="samp">FL</samp>’, ‘<samp class="samp">FR</samp>’, ‘<samp class="samp">FC</samp>’, ‘<samp class="samp">LFE</samp>’, etc.), each optionally containing a custom name after a ’@’, (e.g. ‘<samp class="samp">FL@Left</samp>’, ‘<samp class="samp">FR@Right</samp>’, ‘<samp class="samp">FC@Center</samp>’, ‘<samp class="samp">LFE@Low_Frequency</samp>’, etc.)

A standard channel layout can be specified by the following: <ul class="itemize mark-bullet"> the name of a single channel (e.g. ‘<samp class="samp">FL</samp>’, ‘<samp class="samp">FR</samp>’, ‘<samp class="samp">FC</samp>’, ‘<samp class="samp">LFE</samp>’, etc.)

the name of a standard channel layout (e.g. ‘<samp class="samp">mono</samp>’, ‘<samp class="samp">stereo</samp>’, ‘<samp class="samp">4.0</samp>’, ‘<samp class="samp">quad</samp>’, ‘<samp class="samp">5.0</samp>’, etc.)

a number of channels, in decimal, followed by ’c’, yielding the default channel layout for that number of channels (see the function <code class="code">av_channel_layout_default</code>). Note that not all channel counts have a default layout.

a number of channels, in decimal, followed by ’C’, yielding an unknown channel layout with the specified number of channels. Note that not all channel layout specification strings support unknown channel layouts.

a channel layout mask, in hexadecimal starting with "0x" (see the <code class="code">AV_CH_*</code> macros in <samp class="file">libavutil/channel_layout.h</samp>.

Before libavutil version 53 the trailing character "c" to specify a number of channels was optional, but now it is required, while a channel layout mask can also be specified as a decimal number (if and only if not followed by "c" or "C"). See also the function <code class="code">av_channel_layout_from_string</code> defined in <samp class="file">libavutil/channel_layout.h</samp>. <h2 class="chapter">3 Expression Evaluation# TOC</h2>

When evaluating an arithmetic expression, FFmpeg uses an internal

formula evaluator, implemented through the <samp class="file">libavutil/eval.h</samp>

interface.

An expression may contain unary, binary operators, constants, and

functions.

Two expressions <var class="var">expr1</var> and <var class="var">expr2</var> can be combined to form

another expression "<var class="var">expr1</var>;<var class="var">expr2</var>".

<var class="var">expr1</var> and <var class="var">expr2</var> are evaluated in turn, and the new

expression evaluates to the value of <var class="var">expr2</var>.

The following binary operators are available: <code class="code">+</code>, <code class="code">-</code>,

<code class="code">*</code>, <code class="code">/</code>, <code class="code">^</code>.

The following unary operators are available: <code class="code">+</code>, <code class="code">-</code>.

Some internal variables can be used to store and load intermediary

results. They can be accessed using the <code class="code">ld</code> and <code class="code">st</code>

functions with an index argument varying from 0 to 9 to specify which

internal variable to access.

The following functions are available:

<dl class="table">

<dt><samp class="option">abs(x)</samp></dt>

<dd> Compute absolute value of <var class="var">x</var>.

</dd>

<dt><samp class="option">acos(x)</samp></dt>

<dd> Compute arccosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">asin(x)</samp></dt>

<dd> Compute arcsine of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan(x)</samp></dt>

<dd> Compute arctangent of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan2(y, x)</samp></dt>

<dd> Compute principal value of the arc tangent of <var class="var">y</var>/<var class="var">x</var>.

</dd>

<dt><samp class="option">between(x, min, max)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">min</var> and lesser than or

equal to <var class="var">max</var>, 0 otherwise.

</dd>

<dt><samp class="option">bitand(x, y)</samp></dt>

<dt><samp class="option">bitor(x, y)</samp></dt>

<dd> Compute bitwise and/or operation on <var class="var">x</var> and <var class="var">y</var>.

The results of the evaluation of <var class="var">x</var> and <var class="var">y</var> are converted to

integers before executing the bitwise operation.

Note that both the conversion to integer and the conversion back to

floating point can lose precision. Beware of unexpected results for

large numbers (usually 2^53 and larger).

</dd>

<dt><samp class="option">ceil(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> upwards to the nearest

integer. For example, "ceil(1.5)" is "2.0".

</dd>

<dt><samp class="option">clip(x, min, max)</samp></dt>

<dd> Return the value of <var class="var">x</var> clipped between <var class="var">min</var> and <var class="var">max</var>.

</dd>

<dt><samp class="option">cos(x)</samp></dt>

<dd> Compute cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">cosh(x)</samp></dt>

<dd> Compute hyperbolic cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">eq(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> and <var class="var">y</var> are equivalent, 0 otherwise.

</dd>

<dt><samp class="option">exp(x)</samp></dt>

<dd> Compute exponential of <var class="var">x</var> (with base <code class="code">e</code>, the Euler’s number).

</dd>

<dt><samp class="option">floor(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> downwards to the nearest

integer. For example, "floor(-1.5)" is "-2.0".

</dd>

<dt><samp class="option">gauss(x)</samp></dt>

<dd> Compute Gauss function of <var class="var">x</var>, corresponding to

<code class="code">exp(-x*x/2) / sqrt(2*PI)</code>.

</dd>

<dt><samp class="option">gcd(x, y)</samp></dt>

<dd> Return the greatest common divisor of <var class="var">x</var> and <var class="var">y</var>. If both <var class="var">x</var> and

<var class="var">y</var> are 0 or either or both are less than zero then behavior is undefined.

</dd>

<dt><samp class="option">gt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">gte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">hypot(x, y)</samp></dt>

<dd> This function is similar to the C function with the same name; it returns

"sqrt(<var class="var">x</var>*<var class="var">x</var> + <var class="var">y</var>*<var class="var">y</var>)", the length of the hypotenuse of a

right triangle with sides of length <var class="var">x</var> and <var class="var">y</var>, or the distance of the

point (<var class="var">x</var>, <var class="var">y</var>) from the origin.

</dd>

<dt><samp class="option">if(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the result of

the evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">if(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">ifnot(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the result of the

evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">ifnot(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">isinf(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is +/-INFINITY, 0.0 otherwise.

</dd>

<dt><samp class="option">isnan(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is NAN, 0.0 otherwise.

</dd>

<dt><samp class="option">ld(idx)</samp></dt>

<dd> Load the value of the internal variable with index <var class="var">idx</var>, which was

previously stored with st(<var class="var">idx</var>, <var class="var">expr</var>).

The function returns the loaded value.

</dd>

<dt><samp class="option">lerp(x, y, z)</samp></dt>

<dd> Return linear interpolation between <var class="var">x</var> and <var class="var">y</var> by amount of <var class="var">z</var>.

</dd>

<dt><samp class="option">log(x)</samp></dt>

<dd> Compute natural logarithm of <var class="var">x</var>.

</dd>

<dt><samp class="option">lt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">lte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">max(x, y)</samp></dt>

<dd> Return the maximum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">min(x, y)</samp></dt>

<dd> Return the minimum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">mod(x, y)</samp></dt>

<dd> Compute the remainder of division of <var class="var">x</var> by <var class="var">y</var>.

</dd>

<dt><samp class="option">not(expr)</samp></dt>

<dd> Return 1.0 if <var class="var">expr</var> is zero, 0.0 otherwise.

</dd>

<dt><samp class="option">pow(x, y)</samp></dt>

<dd> Compute the power of <var class="var">x</var> elevated <var class="var">y</var>, it is equivalent to

"(<var class="var">x</var>)^(<var class="var">y</var>)".

</dd>

<dt><samp class="option">print(t)</samp></dt>

<dt><samp class="option">print(t, l)</samp></dt>

<dd> Print the value of expression <var class="var">t</var> with loglevel <var class="var">l</var>. If <var class="var">l</var> is not

specified then a default log level is used.

Return the value of the expression printed.

</dd>

<dt><samp class="option">random(idx)</samp></dt>

<dd> Return a pseudo random value between 0.0 and 1.0. <var class="var">idx</var> is the

index of the internal variable used to save the seed/state, which can be

previously stored with <code class="code">st(idx)</code>.

To initialize the seed, you need to store the seed value as a 64-bit

unsigned integer in the internal variable with index <var class="var">idx</var>.

For example, to store the seed with value <code class="code">42</code> in the internal

variable with index <code class="code">0</code> and print a few random values:

<pre class="example-preformatted">st(0,42); print(random(0)); print(random(0)); print(random(0))

</pre>

</dd> <dt><samp class="option">randomi(idx, min, max)</samp></dt> <dd> Return a pseudo random value in the interval between <var class="var">min</var> and <var class="var">max</var>. <var class="var">idx</var> is the index of the internal variable which will be used to save the seed/state, which can be previously stored with <code class="code">st(idx)</code>. To initialize the seed, you need to store the seed value as a 64-bit unsigned integer in the internal variable with index <var class="var">idx</var>. </dd> <dt><samp class="option">root(expr, max)</samp></dt> <dd> Find an input value for which the function represented by <var class="var">expr</var> with argument <var class="var">ld(0)</var> is 0 in the interval 0..<var class="var">max</var>. The expression in <var class="var">expr</var> must denote a continuous function or the result is undefined. <var class="var">ld(0)</var> is used to represent the function input value, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class="code">ld(0)</code>. When the expression evaluates to 0 then the corresponding input value will be returned. </dd> <dt><samp class="option">round(expr)</samp></dt> <dd> Round the value of expression <var class="var">expr</var> to the nearest integer. For example, "round(1.5)" is "2.0". </dd> <dt><samp class="option">sgn(x)</samp></dt> <dd> Compute sign of <var class="var">x</var>. </dd> <dt><samp class="option">sin(x)</samp></dt> <dd> Compute sine of <var class="var">x</var>. </dd> <dt><samp class="option">sinh(x)</samp></dt> <dd> Compute hyperbolic sine of <var class="var">x</var>. </dd> <dt><samp class="option">sqrt(expr)</samp></dt> <dd> Compute the square root of <var class="var">expr</var>. This is equivalent to "(<var class="var">expr</var>)^.5". </dd> <dt><samp class="option">squish(x)</samp></dt> <dd> Compute expression <code class="code">1/(1 + exp(4*x))</code>. </dd> <dt><samp class="option">st(idx, expr)</samp></dt> <dd> Store the value of the expression <var class="var">expr</var> in an internal variable. <var class="var">idx</var> specifies the index of the variable where to store the value, and it is a value ranging from 0 to 9. The function returns the value stored in the internal variable. The stored value can be retrieved with <code class="code">ld(var)</code>. Note: variables are currently not shared between expressions. </dd> <dt><samp class="option">tan(x)</samp></dt> <dd> Compute tangent of <var class="var">x</var>. </dd> <dt><samp class="option">tanh(x)</samp></dt> <dd> Compute hyperbolic tangent of <var class="var">x</var>. </dd> <dt><samp class="option">taylor(expr, x)</samp></dt> <dt><samp class="option">taylor(expr, x, idx)</samp></dt> <dd> Evaluate a Taylor series at <var class="var">x</var>, given an expression representing the <code class="code">ld(idx)</code>-th derivative of a function at 0. When the series does not converge the result is undefined. <var class="var">ld(idx)</var> is

used to represent the derivative order in <var class=”var”>expr</var>, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class=”code”>ld(idx)</code>. If <var class=”var”>idx</var> is not specified then 0 is assumed. Note, when you have the derivatives at y instead of 0, <code class=”code”>taylor(expr, x-y)</code> can be used. </dd> <dt><samp class=”option”>time(0)</samp></dt> <dd> Return the current (wallclock) time in seconds. </dd> <dt><samp class=”option”>trunc(expr)</samp></dt> <dd> Round the value of expression <var class=”var”>expr</var> towards zero to the nearest integer. For example, "trunc(-1.5)" is "-1.0". </dd> <dt><samp class=”option”>while(cond, expr)</samp></dt> <dd> Evaluate expression <var class=”var”>expr</var> while the expression <var class=”var”>cond</var> is non-zero, and returns the value of the last <var class=”var”>expr</var> evaluation, or NAN if <var class=”var”>cond</var> was always false. </dd> </dl>

The following constants are available: <dl class=”table”> <dt><samp class=”option”>PI</samp></dt> <dd> area of the unit disc, approximately 3.14 </dd> <dt><samp class=”option”>E</samp></dt> <dd> exp(1) (Euler’s number), approximately 2.718 </dd> <dt><samp class=”option”>PHI</samp></dt> <dd> golden ratio (1+sqrt(5))/2, approximately 1.618 </dd> </dl>

Assuming that an expression is considered "true" if it has a non-zero value, note that: <code class=”code”>*</code> works like AND <code class=”code”>+</code> works like OR For example the construct: <pre class=”example-preformatted”>if (A AND B) then C </pre> is equivalent to: <pre class=”example-preformatted”>if(A*B, C) </pre>

In your C code, you can extend the list of unary and binary functions, and define recognized constants, so that they are available for your expressions. The evaluator also recognizes the International System unit prefixes. If ’i’ is appended after the prefix, binary prefixes are used, which are based on powers of 1024 instead of powers of 1000. The ’B’ postfix multiplies the value by 8, and can be appended after a unit prefix or used alone. This allows using for example ’KB’, ’MiB’, ’G’ and ’B’ as number postfix. The list of available International System prefixes follows, with indication of the corresponding powers of 10 and of 2. <dl class=”table”> <dt><samp class=”option”>y</samp></dt> <dd> 10^-24 / 2^-80 </dd> <dt><samp class=”option”>z</samp></dt> <dd> 10^-21 / 2^-70 </dd> <dt><samp class=”option”>a</samp></dt> <dd> 10^-18 / 2^-60 </dd> <dt><samp class=”option”>f</samp></dt> <dd> 10^-15 / 2^-50 </dd> <dt><samp class=”option”>p</samp></dt>

<dd> 10^-12 / 2^-40 </dd> <dt><samp class="option">n</samp></dt>
<dd> 10^-9 / 2^-30 </dd> <dt><samp class="option">u</samp></dt>
<dd> 10^-6 / 2^-20 </dd> <dt><samp class="option">m</samp></dt>
<dd> 10^-3 / 2^-10 </dd> <dt><samp class="option">c</samp></dt>
<dd> 10^-2 </dd> <dt><samp class="option">d</samp></dt>
<dd> 10^-1 </dd> <dt><samp class="option">h</samp></dt>
<dd> 10^2 </dd> <dt><samp class="option">k</samp></dt>
<dd> 10^3 / 2^10 </dd> <dt><samp class="option">K</samp></dt>
<dd> 10^3 / 2^10 </dd> <dt><samp class="option">M</samp></dt>
<dd> 10^6 / 2^20 </dd> <dt><samp class="option">G</samp></dt>
<dd> 10^9 / 2^30 </dd> <dt><samp class="option">T</samp></dt>
<dd> 10^12 / 2^40 </dd> <dt><samp class="option">P</samp></dt>
<dd> 10^15 / 2^50 </dd> <dt><samp class="option">E</samp></dt>
<dd> 10^18 / 2^60 </dd> <dt><samp class="option">Z</samp></dt>
<dd> 10^21 / 2^70 </dd> <dt><samp class="option">Y</samp></dt>
<dd> 10^24 / 2^80 </dd> </dl>

 <h2 class="chapter">4 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, libavutil

 <h2 class="chapter">5 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffmpeg.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> ffmpeg Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link

rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body>
 <h1> ffmpeg Documentation </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1 Synopsis 2 Description 3 Detailed description <ul class=”toc-numbered-mark”> 3.1 Streamcopy 3.2 Transcoding 3.3 Filtering <ul class=”toc-numbered-mark”> 3.3.1 Simple filtergraphs 3.3.2 Complex filtergraphs 3.4 Loopback decoders 4 Stream selection <ul class=”toc-numbered-mark”> 4.1 Description <ul class=”toc-numbered-mark”> 4.1.1 Automatic stream selection 4.1.2 Manual stream selection 4.1.3 Complex filtergraphs 4.1.4 Stream handling 4.2 Examples 5 Options <ul class=”toc-numbered-mark”> 5.1 Stream specifiers 5.2 Generic options 5.3 AVOptions 5.4 Main options 5.5 Video Options 5.6 Advanced Video options 5.7 Audio Options 5.8 Advanced Audio options 5.9 Subtitle options 5.10 Advanced Subtitle options <a id=”toc-Advanced-options” href=”#Advanced-

options”>5.11 Advanced options 5.12 Preset files <ul class=”toc-numbered-mark”> 5.12.1 ffpreset files 5.12.2 avpreset files 5.13 vstats file format 6 Examples <ul class=”toc-numbered-mark”> 6.1 Video and Audio grabbing 6.2 X11 grabbing 6.3 Video and Audio file format conversion 7 See Also 8 Authors

 <h2 class=”chapter”>1 Synopsis# TOC</h2>

ffmpeg [<var class=”var”>global_options</var>] {[<var class=”var”>input_file_options</var>] -i <samp class=”file”>input_url</samp>} ... {[<var class=”var”>output_file_options</var>] <samp class=”file”>output_url</samp>} ... <h2 class=”chapter”>2 Description# TOC</h2>

<code class=”command”>ffmpeg</code> is a universal media converter. It can read a wide variety of inputs - including live grabbing/recording devices - filter, and transcode them into a plethora of output formats. <code class=”command”>ffmpeg</code> reads from an arbitrary number of inputs (which can be regular files, pipes, network streams, grabbing devices, etc.), specified by the <code class=”code”>-i</code> option, and writes to an arbitrary number of outputs, which are specified by a plain output url. Anything found on the command line which cannot be interpreted as an option is considered to be an output url. Each input or output can, in principle, contain any number of elementary streams of different types (video/audio/subtitle/attachment/data), though the allowed stream counts and/or types may be limited by the container format. Selecting which streams from which inputs will go into which output is either done automatically or with the <code class=”code”>-map</code> option (see the Stream selection chapter). To refer to inputs/outputs in options, you must use their indices (0-based). E.g. the first input is <code class=”code”>0</code>, the second is <code class=”code”>1</code>, etc. Similarly, streams within an input/output are referred to by their indices. E.g. <code class=”code”>2:3</code> refers to

the fourth stream in the third input or output. Also see the Stream specifiers chapter. As a general rule, options are applied to the next specified file. Therefore, order is important, and you can have the same option on the command line multiple times. Each occurrence is then applied to the next input or output file. Exceptions from this rule are the global options (e.g. verbosity level), which should be specified first. Do not mix input and output files – first specify all input files, then all output files. Also do not mix options which belong to different files. All options apply ONLY to the next input or output file and are reset between files. Some simple examples follow. <ul class=”itemize mark-bullet”> Convert an input media file to a different format, by re-encoding media streams: <pre class=”example-preformatted”>ffmpeg -i input.avi output.mp4 </pre>

Set the video bitrate of the output file to 64 kbit/s: <pre class=”example-preformatted”>ffmpeg -i input.avi -b:v 64k -bufsize 64k output.mp4 </pre>

Force the frame rate of the output file to 24 fps: <pre class=”example-preformatted”>ffmpeg -i input.avi -r 24 output.mp4 </pre>

Force the frame rate of the input file (valid for raw formats only) to 1 fps and the frame rate of the output file to 24 fps: <pre class=”example-preformatted”>ffmpeg -r 1 -i input.m2v -r 24 output.mp4 </pre>

The format option may be needed for raw input files.

 <h2 class=”chapter”>3 Detailed description# TOC</h2>

<code class=”command”>ffmpeg</code> builds a transcoding pipeline out of the components listed below. The program’s operation then consists of input data chunks flowing from the sources down the pipes towards the sinks, while being transformed by the components they encounter along the way. The following kinds of components are available: <ul class=”itemize mark-bullet”> Demuxers (short for "demultiplexers") read an input source in order to extract

<ul class=”itemize mark-bullet”> global properties such as metadata or chapters; list of input elementary streams and their properties

One demuxer instance is created for each <samp class=”option”>-i</samp> option, and sends encoded packets to decoders or muxers.

 In other literature, demuxers are sometimes called splitters, because their main function is splitting a file into elementary streams (though some files only contain one elementary stream). A schematic representation of a demuxer looks like this: <pre class=”verbatim”> demuxer packets for stream 0 elementary stream 0 global properties packets for stream 1 and elementary stream 1 metadata ..... packets for stream N elementary stream N read from file, network stream, grabbing device, etc. </pre> Decoders receive encoded (compressed) packets for an audio, video, or subtitle elementary stream, and decode them into raw frames (arrays of pixels for video, PCM for audio). A decoder is typically associated with (and receives its input from) an elementary stream in a demuxer, but sometimes may also exist on its own (see Loopback decoders).

A schematic representation of a decoder looks like this: <pre class=”verbatim”> packets raw frames decoder </pre> Filtergraphs process and transform raw audio or video frames. A filtergraph consists of one or more individual filters linked into a graph. Filtergraphs come in two flavors - simple and complex, configured with the <samp class=”option”>-filter</samp> and <samp class=”option”>-filter_complex</samp> options, respectively.

A simple filtergraph is associated with an output elementary stream; it receives the input to be filtered from a decoder and sends filtered output to that output stream’s encoder. A simple video filtergraph that performs deinterlacing (using the <code class=”code”>yadif</code> deinterlacer) followed by resizing (using the <code class=”code”>scale</code> filter) can look like this: <pre class=”verbatim”> simple filtergraph frames from frames for a decoder an encoder yadif scale </pre> A complex filtergraph is standalone and not associated with any specific stream. It may have multiple (or zero) inputs, potentially of different types (audio or video), each of which receiving data either from a decoder or another complex filtergraph’s output. It also has one or more outputs that feed either an encoder or another complex filtergraph’s input. The following example diagram represents a complex filtergraph with 3 inputs and 2 outputs (all video): <pre class=”verbatim”>
complex filtergraph frames

frames input 0 overlay overlay output 0 frames input 1 frames frames input 2 scale split output 1 </pre> Frames from second input are overlaid over those from the first. Frames from the third input are rescaled, then the duplicated into two identical streams. One of them is overlaid over the combined first two inputs, with the result exposed as the filtergraph’s first output. The other duplicate ends up being the filtergraph’s second output. Encoders receive raw audio, video, or subtitle frames and encode them into encoded packets. The encoding (compression) process is typically lossy - it degrades stream quality to make the output smaller; some encoders are lossless, but at the cost of much higher output size. A video or audio encoder receives its input from some filtergraph’s output, subtitle encoders receive input from a decoder (since subtitle filtering is not supported yet). Every encoder is associated with some muxer’s output elementary stream and sends its output to that muxer.

A schematic representation of an encoder looks like this: <pre class="verbatim"> raw frames packets encoder </pre> Muxers (short for "multiplexers") receive encoded packets for their elementary streams from encoders (the transcoding path) or directly from demuxers (the streamcopy path), interleave them (when there is more than one elementary stream), and write the resulting bytes into the output file (or pipe, network stream, etc.).

A schematic representation of a muxer looks like this: <pre class="verbatim"> packets for stream 0 muxer elementary stream 0 global packets for stream 1 properties elementary stream 1 and metadata ..... packets for stream N elementary stream N write to file, network stream, grabbing device, etc. </pre>

 <h3 class="section">3.1 Streamcopy# TOC</h3> The simplest pipeline in <code class="command">ffmpeg</code> is single-stream streamcopy, that is copying one input elementary stream’s packets without decoding, filtering, or encoding them. As an example, consider an

input file called <samp class="file">INPUT.mkv</samp> with 3 elementary streams, from which we take the second and write it to file <samp class="file">OUTPUT.mp4</samp>. A schematic representation of such a pipeline looks like this: <pre class="verbatim">
demuxer unused elementary stream 0 INPUT.mkv
packets muxer elementary
stream 1 elementary stream 0 OUTPUT.mp4
unused elementary stream 2
</pre> The above pipeline can be constructed with the following commandline:
<pre class="example-preformatted">ffmpeg -i INPUT.mkv -map 0:1 -c copy
OUTPUT.mp4 </pre>

In this commandline <ul class="itemize mark-bullet"> there is a single input <samp class="file">INPUT.mkv</samp>;

there are no input options for this input;

there is a single output <samp class="file">OUTPUT.mp4</samp>;

there are two output options for this output:

<ul class="itemize mark-bullet"> <code class="code">-map 0:1</code> selects the input stream to be used - from input with index 0 (i.e. the first one) the stream with index 1 (i.e. the second one);

<code class="code">-c copy</code> selects the <code class="code">copy</code> encoder, i.e. streamcopy with no decoding or encoding.

Streamcopy is useful for changing the elementary stream count, container format, or modifying container-level metadata. Since there is no decoding or encoding, it is very fast and there is no quality loss. However, it might not work in some cases because of a variety of factors (e.g. certain information required by the target container is not available in the source). Applying filters is obviously also impossible, since filters work on decoded frames. More complex streamcopy scenarios can be constructed - e.g. combining streams from two input files into a single output: <pre class="verbatim">
demuxer
0 packets muxer elementary stream 0 elementary
stream 0 INPUT0.mkv OUTPUT.mp4
demuxer 1 packets elementary stream 1 elementary stream 0 INPUT1.aac
</pre> that can be built by the commandline <pre class="example-preformatted">ffmpeg -i INPUT0.mkv -i INPUT1.aac -map 0:0 -map 1:0 -c copy OUTPUT.mp4 </pre>

The output <samp class="option">-map</samp> option is used twice

here, creating two streams in the output file - one fed by the first input and one by the second. The single instance of the <samp class="option">-c</samp> option selects streamcopy for both of those streams. You could also use multiple instances of this option together with Stream specifiers to apply different values to each stream, as will be demonstrated in following sections. A converse scenario is splitting multiple streams from a single input into multiple outputs: <pre class="verbatim">
demuxer packets muxer 0 elementary stream 0 elementary stream 0 OUTPUT0.mp4 INPUT.mkv
packets elementary stream 1
muxer 1 elementary stream 0
OUTPUT1.mp4 </pre> built with <pre class="example-preformatted">ffmpeg -i INPUT.mkv -map 0:0 -c copy OUTPUT0.mp4 -map 0:1 -c copy OUTPUT1.mp4 </pre> Note how a separate instance of the <samp class="option">-c</samp> option is needed for every output file even though their values are the same. This is because non-global options (which is most of them) only apply in the context of the file before which they are placed. These examples can of course be further generalized into arbitrary remappings of any number of inputs into any number of outputs. <h3 class="section">3.2 Transcoding# TOC</h3> Transcoding is the process of decoding a stream and then encoding it again. Since encoding tends to be computationally expensive and in most cases degrades the stream quality (i.e. it is lossy), you should only transcode when you need to and perform streamcopy otherwise. Typical reasons to transcode are: <ul class="itemize mark-bullet"> applying filters - e.g. resizing, deinterlacing, or overlaying video; resampling or mixing audio;

you want to feed the stream to something that cannot decode the original codec. Note that <code class="command">ffmpeg</code> will transcode all audio, video, and subtitle streams unless you specify <samp class="option">-c copy</samp> for them. Consider an example pipeline that reads an input file with one audio and one video stream, transcodes the video and copies the audio into a single output file. This can be schematically represented as follows <pre class="verbatim">
demuxer audio packets stream 0 (audio)
INPUT.mkv video raw packets video
video frames stream 1 (video) decoder
video
muxer packets video stream 0 (video) encoder
(libx264) OUTPUT.mp4

stream 1 (audio)
</pre> and implemented with the following commandline: <pre class="example-preformatted">ffmpeg -i INPUT.mkv -map 0:v -map 0:a -c:v libx264 -c:a copy OUTPUT.mp4 </pre> Note how it uses stream specifiers <code class="code">:v</code> and <code class="code">:a</code> to select input streams and apply different values of the <samp class="option">-c</samp> option to them; see the Stream specifiers section for more details.

 <h3 class="section">3.3 Filtering# TOC</h3>

When transcoding, audio and video streams can be filtered before encoding, with either a simple or complex filtergraph. <h4 class="subsection">3.3.1 Simple filtergraphs# TOC</h4>

Simple filtergraphs are those that have exactly one input and output, both of the same type (audio or video). They are configured with the per-stream <samp class="option">-filter</samp> option (with <samp class="option">-vf</samp> and <samp class="option">-af</samp> aliases for <samp class="option">-filter:v</samp> (video) and <samp class="option">-filter:a</samp> (audio) respectively). Note that simple filtergraphs are tied to their output stream, so e.g. if you have multiple audio streams, <samp class="option">-af</samp> will create a separate filtergraph for each one. Taking the transcoding example from above, adding filtering (and omitting audio, for clarity) makes it look like this: <pre class="verbatim">
demuxer video stream
packets video frames INPUT.mkv decoder
simple filtergraph
yadif scale
video muxer packets video
video stream encoder OUTPUT.mp4
</pre>
<h4 class="subsection">3.3.2 Complex filtergraphs# TOC</h4>

Complex filtergraphs are those which cannot be described as simply a linear processing chain applied to one stream. This is the case, for example, when the graph has more than one input and/or output, or when output stream

type is different from input. Complex filtergraphs are configured with the <samp class="option">-filter_complex</samp> option. Note that this option is global, since a complex filtergraph, by its nature, cannot be unambiguously associated with a single stream or file. Each instance of <samp class="option">-filter_complex</samp> creates a new complex filtergraph, and there can be any number of them. A trivial example of a complex filtergraph is the <code class="code">overlay</code> filter, which has two video inputs and one video output, containing one video overlaid on top of the other. Its audio counterpart is the <code class="code">amix</code> filter. <h3 class="section">3.4 Loopback decoders# TOC</h3> While decoders are normally associated with demuxer streams, it is also possible to create "loopback" decoders that decode the output from some encoder and allow it to be fed back to complex filtergraphs. This is done with the <code class="code">-dec</code> directive, which takes as a parameter <var class="var">output_file_index</var>:<var class="var">stream_index</var> to select the output stream that should be decoded. Both indices are zero-based integers: the first identifies the output file and the second identifies the stream within that file. Stream specifiers such as <code class="code">v:0</code> are not supported. Every such directive creates a new loopback decoder, indexed with successive integers starting at zero. These indices should then be used to refer to loopback decoders in complex filtergraph link labels, as described in the documentation for <samp class="option">-filter_complex</samp>. Decoding AVOptions can be passed to loopback decoders by placing them before <code class="code">-dec</code>, analogously to input/output options. E.g. the following example: <pre class="example-preformatted">ffmpeg -i INPUT \ -map 0:v:0 -c:v libx264 -crf 45 -f null - \ -threads 3 -dec 0:0 \ -filter_complex '[0:v][dec:0]hstack[stack]' \ -map '[stack]' -c:v ffv1 OUTPUT </pre>

reads an input video and <ul class="itemize mark-bullet"> (line 2) encodes it with <code class="code">libx264</code> at low quality;

(line 3) decodes this encoded stream using 3 threads;

(line 4) places decoded video side by side with the original input video;

(line 5) combined video is then losslessly encoded and written into <samp class="file">OUTPUT</samp>.

Such a transcoding pipeline can be represented with the following diagram: <pre class="verbatim">   demuxer

video stream video video null muxer INPUT
decoder encoder (discards its input) (libx264)
loopback
decoder complex filter-
graph hstack
muxer video
video stream encoder OUTPUT (ffv1)
</pre>

 <h2 class="chapter">4 Stream selection# TOC</h2>

<code class="command">ffmpeg</code> provides the <code class="code">-map</code> option for manual control of stream selection in each output file. Users can skip <code class="code">-map</code> and let ffmpeg perform automatic stream selection as described below. The <code class="code">-vn / -an / -sn / -dn</code> options can be used to skip inclusion of video, audio, subtitle and data streams respectively, whether manually mapped or automatically selected, except for those streams which are outputs of complex filtergraphs. <h3 class="section">4.1 Description# TOC</h3> The sub-sections that follow describe the various rules that are involved in stream selection. The examples that follow next show how these rules are applied in practice. While every effort is made to accurately reflect the behavior of the program, FFmpeg is under continuous development and the code may have changed since the time of this writing. <h4 class="subsection">4.1.1 Automatic stream selection# TOC</h4>

In the absence of any map options for a particular output file, ffmpeg inspects the output format to check which type of streams can be included in it, viz. video, audio and/or subtitles. For each acceptable stream type, ffmpeg will pick one stream, when available, from among all the inputs. It will select that stream based upon the following criteria: <ul class="itemize mark-bullet"> for video, it is the stream with the highest resolution, for audio, it is the stream with the most channels, for subtitles, it is the first subtitle stream found but there’s a caveat. The output format’s default subtitle encoder can be either text-based or image-based, and only a subtitle stream of the same type will be

chosen.

In the case where several streams of the same type rate equally, the stream with the lowest index is chosen. Data or attachment streams are not automatically selected and can only be included using <code class="code">-map</code>. <h4 class="subsection">4.1.2 Manual stream selection# TOC</h4>

When <code class="code">-map</code> is used, only user-mapped streams are included in that output file, with one possible exception for filtergraph outputs described below. <h4 class="subsection">4.1.3 Complex filtergraphs# TOC</h4>

If there are any complex filtergraph output streams with unlabeled pads, they will be added to the first output file. This will lead to a fatal error if the stream type is not supported by the output format. In the absence of the map option, the inclusion of these streams leads to the automatic stream selection of their types being skipped. If map options are present, these filtergraph streams are included in addition to the mapped streams. Complex filtergraph output streams with labeled pads must be mapped once and exactly once. <h4 class="subsection">4.1.4 Stream handling# TOC</h4>

Stream handling is independent of stream selection, with an exception for subtitles described below. Stream handling is set via the <code class="code">-codec</code> option addressed to streams within a specific output file. In particular, codec options are applied by ffmpeg after the stream selection process and thus do not influence the latter. If no <code class="code">-codec</code> option is specified for a stream type, ffmpeg will select the default encoder registered by the output file muxer. An exception exists for subtitles. If a subtitle encoder is specified for an output file, the first subtitle stream found of any type, text or image, will be included. ffmpeg does not validate if the specified encoder can convert the selected stream or if the converted stream is acceptable within the output format. This applies generally as well: when the user sets an encoder manually, the stream selection process cannot check if the encoded stream can be muxed into the output file. If it cannot, ffmpeg will abort and all output files will fail to be processed. <h3 class="section">4.2 Examples<span

class=”pull-right”># TOC</h3>

The following examples illustrate the behavior, quirks and limitations of ffmpeg’s stream selection methods. They assume the following three input files. <pre class=”verbatim”> input file 'A.avi' stream 0: video 640x360 stream 1: audio 2 channels

input file 'B.mp4' stream 0: video 1920x1080 stream 1: audio 2 channels stream 2: subtitles (text) stream 3: audio 5.1 channels stream 4: subtitles (text)

input file 'C.mkv' stream 0: video 1280x720 stream 1: audio 2 channels stream 2: subtitles (image) </pre> <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 out1.mkv out2.wav -map 1:a -c:a copy out3.mov </pre> There are three output files specified, and for the first two, no <code class=”code”>-map</code> options are set, so ffmpeg will select streams for these two files automatically. <samp class=”file”>out1.mkv</samp> is a Matroska container file and accepts video, audio and subtitle streams, so ffmpeg will try to select one of each type.
 For video, it will select <code class=”code”>stream 0</code> from <samp class=”file”>B.mp4</samp>, which has the highest resolution among all the input video streams.
 For audio, it will select <code class=”code”>stream 3</code> from <samp class=”file”>B.mp4</samp>, since it has the greatest number of channels.
 For subtitles, it will select <code class=”code”>stream 2</code> from <samp class=”file”>B.mp4</samp>, which is the first subtitle stream from among <samp class=”file”>A.avi</samp> and <samp class=”file”>B.mp4</samp>. <samp class=”file”>out2.wav</samp> accepts only audio streams, so only <code class=”code”>stream 3</code> from <samp class=”file”>B.mp4</samp> is selected. For <samp class=”file”>out3.mov</samp>, since a <code class=”code”>-map</code> option is set, no automatic stream selection will occur. The <code class=”code”>-map 1:a</code> option will select all audio streams from the second input <samp class=”file”>B.mp4</samp>. No other streams will be included in this output file. For the first two outputs, all included streams will be transcoded. The encoders chosen will be the default ones registered by each output format, which may not match the codec of the selected input streams. For the third output, codec option for audio streams has been set to <code class=”code”>copy</code>, so no decoding-filtering-encoding operations will occur, or can occur. Packets of selected streams shall be conveyed from the input file and muxed within the output file. <pre class=”example-preformatted”>ffmpeg -i C.mkv out1.mkv -c:s dvdsub -an out2.mkv </pre> Although <samp

class=”file”>out1.mkv</samp> is a Matroska container file which accepts subtitle streams, only a video and audio stream shall be selected. The subtitle stream of <samp class=”file”>C.mkv</samp> is image-based and the default subtitle encoder of the Matroska muxer is text-based, so a transcode operation for the subtitles is expected to fail and hence the stream isn’t selected. However, in <samp class=”file”>out2.mkv</samp>, a subtitle encoder is specified in the command and so, the subtitle stream is selected, in addition to the video stream. The presence of <code class=”code”>-an</code> disables audio stream selection for <samp class=”file”>out2.mkv</samp>. <pre class=”example-preformatted”>ffmpeg -i A.avi -i C.mkv -i B.mp4 -filter_complex "overlay" out1.mp4 out2.srt </pre> A filtergraph is setup here using the <code class=”code”>-filter_complex</code> option and consists of a single video filter. The <code class=”code”>overlay</code> filter requires exactly two video inputs, but none are specified, so the first two available video streams are used, those of <samp class=”file”>A.avi</samp> and <samp class=”file”>C.mkv</samp>. The output pad of the filter has no label and so is sent to the first output file <samp class=”file”>out1.mp4</samp>. Due to this, automatic selection of the video stream is skipped, which would have selected the stream in <samp class=”file”>B.mp4</samp>. The audio stream with most channels viz. <code class=”code”>stream 3</code> in <samp class=”file”>B.mp4</samp>, is chosen automatically. No subtitle stream is chosen however, since the MP4 format has no default subtitle encoder registered, and the user hasn’t specified a subtitle encoder. The 2nd output file, <samp class=”file”>out2.srt</samp>, only accepts text-based subtitle streams. So, even though the first subtitle stream available belongs to <samp class=”file”>C.mkv</samp>, it is image-based and hence skipped. The selected stream, <code class=”code”>stream 2</code> in <samp class=”file”>B.mp4</samp>, is the first text-based subtitle stream. <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0[outv];overlay;aresample" \ -map '[outv]' -an out1.mp4 \ out2.mkv \ -map '[outv]' -map 1:a:0 out3.mkv </pre>

The above command will fail, as the output pad labelled <code class=”code”>[outv]</code> has been mapped twice. None of the output files shall be processed.   <pre class=”example-preformatted”>ffmpeg -i A.avi -i B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0[outv];overlay;aresample" \ -an out1.mp4 \ out2.mkv \ -map 1:a:0 out3.mkv </pre>

This command above will also fail as the hue filter output has a label, <code class=”code”>[outv]</code>, and hasn’t been mapped anywhere. The command should be modified as follows, <pre class=”example-preformatted”>ffmpeg -i A.avi -i

B.mp4 -i C.mkv -filter_complex "[1:v]hue=s=0,split=2[outv1][outv2];overlay;aresample"
\ -map '[outv1]' -an out1.mp4 \ out2.mkv \ -map '[outv2]' -map 1:a:0 out3.mkv
</pre> The video stream from <samp class="file">B.mp4</samp>
is sent to the hue filter, whose output is cloned once using the split filter,
and both outputs labelled. Then a copy each is mapped to the first and
third output files. The overlay filter, requiring two video inputs,
uses the first two unused video streams. Those are the streams from <samp
class="file">A.avi</samp> and <samp class="file">C.mkv</samp>. The
overlay output isn’t labelled, so it is sent to the first output file <samp
class="file">out1.mp4</samp>, regardless of the presence of the <code
class="code">-map</code> option. The aresample filter is sent
the first unused audio stream, that of <samp class="file">A.avi</samp>.
Since this filter output is also unlabelled, it too is mapped to the first output
file. The presence of <code class="code">-an</code> only suppresses
automatic or manual stream selection of audio streams, not outputs sent from
filtergraphs. Both these mapped streams shall be ordered before the mapped
stream in <samp class="file">out1.mp4</samp>. The video,
audio and subtitle streams mapped to <code class="code">out2.mkv</code>
are entirely determined by automatic stream selection. <samp
class="file">out3.mkv</samp> consists of the cloned video output from the
hue filter and the first audio stream from <samp class="file">B.mp4</samp>.

 <h2 class="chapter">5 Options<span class="pull-
right">#
TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or
’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix
will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the
SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’
as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class="example-preformatted">ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">5.1 Stream specifiers# TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class="code">a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class="code">-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class="code">-codec copy</code>

or <code class="code">-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class="table">

<dt><samp class="option"><var class="var">stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class="code">-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class="var">stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class="var">stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class="option"><var class="var">stream_type</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> <var class="var">stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class="var">additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class="var">additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class="option">g:<var class="var">group_specifier</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class="var">group_specifier</var>.

if <var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class="var">additional_stream_specifier</var>.

<var class="var">group_specifier</var> may be one of the following:

<dl class="table">

<dt><samp class="option"><var class="var">group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class="var">additional_stream_specifier</var>.

</dd>

<dt><samp class="option">#<var class="var">stream_id</var> or i:<var class="var">stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class="option">m:<var class="var">key</var>[:<var class="var">value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class="var">key</var> having the specified value. If

<var class="var">value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class="var">key</var> or <var class="var">value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class="option">disp:<var class="var">dispositions</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class="var">dispositions</var> is a list of

one or more dispositions (as printed by the <samp class="option">-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class="option">u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class="command">ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">5.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options, including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed information about the demuxer named <var class="var">demuxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class="option">muxer=<var class="var">muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class="var">muxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class="option">filter=<var class="var">filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class="var">filter_name</var>. Use the <samp class="option">-filters</samp> option to get a list of all filters. </dd> <dt><samp class="option">bsf=<var class="var">bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class="var">bitstream_filter_name</var>. Use the <samp class="option">-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class="option">protocol=<var class="var">protocol_name</var></samp></dt> <dd> Print detailed information about the protocol named <var class="var">protocol_name</var>. Use the <samp class="option">-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class="option">-version</samp></dt> <dd> Show

version. </dd> <dt><samp class="option">-buildconf</samp></dt>
<dd> Show the build configuration, one option per line. </dd>
<dt><samp class="option">-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class="option">-demuxers</samp></dt> <dd> Show available demuxers. </dd>
<dt><samp class="option">-muxers</samp></dt> <dd> Show available muxers. </dd> <dt><samp class="option">-devices</samp></dt>
<dd> Show available devices. </dd> <dt><samp class="option">-codecs</samp></dt> <dd> Show all codecs known to libavcodec. Note that the term ’codec’ is used throughout this documentation as a shortcut for what is more correctly called a media bitstream format. </dd> <dt><samp class="option">-decoders</samp></dt> <dd> Show available decoders. </dd>
<dt><samp class="option">-encoders</samp></dt> <dd> Show all available encoders. </dd> <dt><samp class="option">-bsfs</samp></dt> <dd> Show available bitstream filters. </dd>
<dt><samp class="option">-protocols</samp></dt> <dd> Show available protocols. </dd> <dt><samp class="option">-filters</samp></dt>
<dd> Show available libavfilter filters. </dd> <dt><samp class="option">-pix_fmts</samp></dt> <dd> Show available pixel formats. </dd> <dt><samp class="option">-sample_fmts</samp></dt>
<dd> Show available sample formats. </dd> <dt><samp class="option">-layouts</samp></dt> <dd> Show channel names and standard channel layouts. </dd> <dt><samp class="option">-dispositions</samp></dt> <dd> Show stream dispositions.
</dd> <dt><samp class="option">-colors</samp></dt> <dd> Show recognized color names. </dd> <dt><samp class="option">-sources <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt>
<dd> Show autodetected sources of the input device. Some devices may provide system-dependent source names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sources pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sinks of the output device. Some devices may provide system-dependent sink names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var class="var">loglevel</var></samp></dt> <dd> Set logging level and flags used by the library. The optional <var class="var">flags</var>

prefix can consist of the following values: <dl class="table"> <dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates that repeated log output should not be compressed to the first line and the "Last message repeated n times" line will be omitted. </dd> <dt>‘<samp class="samp">level</samp>’</dt> <dd> Indicates that log output should add a <code class="code">[level]</code> prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var class="var">flags</var> or changing <var class="var">loglevel</var>. When setting both <var class="var">flags</var> and <var class="var">loglevel</var>, a ’+’ separator is expected between the last <var class="var">flags</var> value and before <var class="var">loglevel</var>. <var class="var">loglevel</var> is a string or a number containing one of the following values: <dl class="table"> <dt>‘<samp class="samp">quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class="samp">panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class="samp">fatal, 8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class="samp">error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class="samp">warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class="samp">info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class="samp">verbose, 40</samp>’</dt> <dd> Same as <code class="code">info</code>, except more verbose. </dd> <dt>‘<samp class="samp">debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class="samp">trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class="code">level</code> prefix, and set <var class="var">loglevel</var> to <code class="code">verbose</code>: <pre class="example-preformatted">ffmpeg -loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code

class=”code”>level</code> prefix flag or <var class=”var”>loglevel</var>: <pre class=”example-preformatted”>ffmpeg [...] -loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class=”env”>AV_LOG_FORCE_NOCOLOR</code>, or can be forced setting the environment variable <code class=”env”>AV_LOG_FORCE_COLOR</code>. </dd> <dt><samp class=”option”>-report</samp></dt> <dd> Dump full command line and log output to a file named <code class=”code”><var class=”var”>program</var>-<var class=”var”>YYYYMMDD</var>-<var class=”var”>HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class=”code”>-loglevel debug</code>. Setting the environment variable <code class=”env”>FFREPORT</code> to any value has the same effect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual). The following options are recognized: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> set the file name to use for the report; <code class=”code”>%p</code> is expanded to the name of the program, <code class=”code”>%t</code> is expanded to a timestamp, <code class=”code”>%%</code> is expanded to a plain <code class=”code”>%</code> </dd> <dt><samp class=”option”>level</samp></dt> <dd> set the log verbosity level using a numerical value (see <code class=”code”>-loglevel</code>). </dd> </dl>

For example, to output a report to a file named <samp class=”file”>ffreport.log</samp> using a log level of <code class=”code”>32</code> (alias for log level <code class=”code”>info</code>):   <pre class=”example-preformatted”>FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class=”option”>-hide_banner</samp></dt> <dd> Suppress printing banner. All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class=”option”>-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class=”table”> <dt>‘<samp

class=”samp”>x86</samp>’</dt> <dd><dl class=”table”>
<dt>‘<samp class=”samp”>mmx</samp>’</dt> <dt>‘<samp
class=”samp”>mmxext</samp>’</dt> <dt>‘<samp class=”samp”>sse</samp>’</d
<dt>‘<samp class=”samp”>sse2</samp>’</dt> <dt>‘<samp
class=”samp”>sse2slow</samp>’</dt> <dt>‘<samp class=”samp”>sse3</samp>’</
<dt>‘<samp class=”samp”>sse3slow</samp>’</dt> <dt>‘<samp
class=”samp”>ssse3</samp>’</dt> <dt>‘<samp class=”samp”>atom</samp>’</dt
<dt>‘<samp class=”samp”>sse4.1</samp>’</dt> <dt>‘<samp
class=”samp”>sse4.2</samp>’</dt> <dt>‘<samp class=”samp”>avx</samp>’</dt>
<dt>‘<samp class=”samp”>avx2</samp>’</dt> <dt>‘<samp
class=”samp”>xop</samp>’</dt> <dt>‘<samp class=”samp”>fma3</samp>’</dt>
<dt>‘<samp class=”samp”>fma4</samp>’</dt> <dt>‘<samp
class=”samp”>3dnow</samp>’</dt> <dt>‘<samp class=”samp”>3dnowext</samp>&rsquo
<dt>‘<samp class=”samp”>bmi1</samp>’</dt> <dt>‘<samp
class=”samp”>bmi2</samp>’</dt> <dt>‘<samp class=”samp”>cmov</samp>’</dt
</dl> </dd> <dt>‘<samp class=”samp”>ARM</samp>’</dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>armv5te</samp>’</dt>
<dt>‘<samp class=”samp”>armv6</samp>’</dt> <dt>‘<samp
class=”samp”>armv6t2</samp>’</dt> <dt>‘<samp class=”samp”>vfp</samp>’</d
<dt>‘<samp class=”samp”>vfpv3</samp>’</dt> <dt>‘<samp
class=”samp”>neon</samp>’</dt> <dt>‘<samp class=”samp”>setend</samp>’</d
</dl> </dd> <dt>‘<samp class=”samp”>AArch64</samp>’</dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>armv8</samp>’</dt>
<dt>‘<samp class=”samp”>vfp</samp>’</dt> <dt>‘<samp
class=”samp”>neon</samp>’</dt> </dl> </dd> <dt>‘<samp
class=”samp”>PowerPC</samp>’</dt> <dd><dl class=”table”>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class=”samp”>Specific Processors</samp>’</dt>
<dd><dl class=”table”> <dt>‘<samp class=”samp”>pentium2</samp>’</dt>
<dt>‘<samp class=”samp”>pentium3</samp>’</dt>
<dt>‘<samp class=”samp”>pentium4</samp>’</dt>
<dt>‘<samp class=”samp”>k6</samp>’</dt> <dt>‘<samp
class=”samp”>k62</samp>’</dt> <dt>‘<samp class=”samp”>athlon</samp>’</dt
<dt>‘<samp class=”samp”>athlonxp</samp>’</dt> <dt>‘<samp
class=”samp”>k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>-cpucount <var class=”var”>count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class=”option”>-max_alloc <var class=”var”>bytes</var></samp></dt>
<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is

INT_MAX. </dd> </dl>

 <h3 class="section">5.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and libavcodec libraries. To see the list of available AVOptions, use the <samp class="option">-help</samp> option. They are separated into two categories: <dl class="table"> <dt><samp class="option">generic</samp></dt> <dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class="option">private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class="option">id3v2_version</samp> private option of the MP3 muxer: <pre class="example-preformatted">ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class="example-preformatted">ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class="option">-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class="option">-option 0</samp>/<samp class="option">-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class="section">5.4 Main options# TOC</h3>

<dl class="table">

<dt><samp class="option">-f <var class="var">fmt</var> (input/output)</samp></dt>

<dd> Force input or output file format. The format is normally auto detected for input

files and guessed from the file extension for output files, so this option is not

needed in most cases.

</dd>

<dt><samp class="option">-i <var class="var">url</var> (input)</samp></

<dd> input file url

</dd>

<dt><samp class="option">-y (global)</samp></dt>

<dd> Overwrite output files without asking.

</dd>

<dt><samp class="option">-n (global)</samp></dt>

<dd> Do not overwrite output files, and exit immediately if a specified

output file already exists.

</dd>

<dt><samp class="option">-stream_loop <var class="var">number</var>
(input)</samp></dt>

<dd> Set number of times input stream shall be looped. Loop 0 means no
loop,

loop -1 means infinite loop.

</dd>

<dt><samp class="option">-recast_media (global)</samp></dt>

<dd> Allow forcing a decoder of a different media type than the one

detected or designated by the demuxer. Useful for decoding media

data muxed as data streams.

</dd>

<dt><samp class="option">-c[:<var class="var">stream_specifier</var>] <var class="var">codec</var> (input/output,per-stream)</samp></dt>

<dt><samp class="option">-codec[:<var class="var">stream_specifier</var>] <var class="var">codec</var> (input/output,per-stream)</samp></dt>

<dd> Select an encoder (when used before an output file) or a decoder (when used

before an input file) for one or more streams. <var class="var">codec</var> is the name of a

decoder/encoder or a special value <code class="code">copy</code> (output only) to indicate that

the stream is not to be re-encoded.

For example

<pre class="example-preformatted">ffmpeg -i INPUT -map 0 -c:v libx264 -c:a copy OUTPUT

</pre>

encodes all video streams with libx264 and copies all audio streams.

For each stream, the last matching <code class="code">c</code> option is applied, so

<pre class="example-preformatted">ffmpeg -i INPUT -map 0 -c copy -c:v:1 libx264 -c:a:137 libvorbis OUTPUT

</pre>

will copy all the streams except the second video, which will be encoded with

libx264, and the 138th audio, which will be encoded with libvorbis.

</dd>

<dt><samp class="option">-t <var class="var">duration</var> (input/output)</samp></dt>

<dd> When used as an input option (before <code class="code">-i</code>), limit the <var class="var">duration</var> of

data read from the input file.

When used as an output option (before an output url), stop writing the

output after its duration reaches <var class="var">duration</var>.

<var class="var">duration</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

-to and -t are mutually exclusive and -t has priority.

</dd>

<dt><samp class="option">-to <var class="var">position</var> (input/output)</samp></dt>

<dd> Stop writing the output or reading the input at <var class="var">position</var>.

<var class="var">position</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

-to and -t are mutually exclusive and -t has priority.

</dd>

<dt><samp class="option">-fs <var class="var">limit_size</var> (output)</samp></dt>

<dd> Set the file size limit, expressed in bytes. No further chunk of bytes is written

after the limit is exceeded. The size of the output file is slightly more than the

requested file size.

</dd>

<dt><samp class="option">-ss <var class="var">position</var> (input/output)</samp></dt>

<dd> When used as an input option (before <code class="code">-i</code>), seeks in this input file to

<var class="var">position</var>. Note that in most formats it is not possible to seek exactly,

so <code class="command">ffmpeg</code> will seek to the closest seek point before <var class="var">position</var>.

When transcoding and <samp class="option">-accurate_seek</samp> is enabled (the default), this

extra segment between the seek point and <var class="var">position</var> will be decoded and

discarded. When doing stream copy or when <samp class="option">-noaccurate_seek</samp> is used, it

will be preserved.

When used as an output option (before an output url), decodes but discards

input until the timestamps reach <var class="var">position</var>.

<var class="var">position</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

</dd>

<dt><samp class="option">-sseof <var class="var">position</var> (input)</samp></dt>

<dd>

Like the <code class="code">-ss</code> option but relative to the "end of file". That is negative

values are earlier in the file, 0 is at EOF.

</dd>

<dt><samp class="option">-isync <var class="var">input_index</var> (input)</samp></dt>

<dd> Assign an input as a sync source.

This will take the difference between the start times of the target and reference inputs and

offset the timestamps of the target file by that difference. The source timestamps of the two

inputs should derive from the same clock source for expected results. If <code class="code">copyts</code> is set

then <code class="code">start_at_zero</code> must also be set. If either of the inputs has no starting timestamp

then no sync adjustment is made.

Acceptable values are those that refer to a valid ffmpeg input index. If the sync reference is

the target index itself or <var class="var">-1</var>, then no adjustment is made to target timestamps. A sync

reference may not itself be synced to any other input.

Default value is <var class="var">-1</var>.

</dd>

<dt><samp class="option">-itsoffset <var class="var">offset</var> (input)</samp></dt>

<dd> Set the input time offset.

<var class="var">offset</var> must be a time duration specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual.

The offset is added to the timestamps of the input files. Specifying

a positive offset means that the corresponding streams are delayed by

the time duration specified in <var class="var">offset</var>.

</dd>

<dt><samp class="option">-itsscale <var class="var">scale</var> (input,per-stream)</samp></dt>

<dd> Rescale input timestamps. <var class="var">scale</var> should be a floating point number.

</dd>

<dt><samp class="option">-timestamp <var class="var">date</var> (output)</samp></dt>

<dd> Set the recording timestamp in the container.

<var class="var">date</var> must be a date specification,

see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#date-syntax">the Date section in the ffmpeg-utils(1) manual.

</dd>

<dt><samp class="option">-metadata[:metadata_specifier] <var class="var">key</var>=<var class="var">value</var> (output,per-metadata)</samp></dt>

<dd> Set a metadata key/value pair.

An optional <var class="var">metadata_specifier</var> may be given to set metadata

on streams, chapters or programs. See <code class="code">-map_metadata</code>

documentation for details.

This option overrides metadata set with <code class="code">-map_metadata</code>. It is

also possible to delete metadata by using an empty value.

For example, for setting the title in the output file:

<pre class="example-preformatted">ffmpeg -i in.avi -metadata title="my title" out.flv

</pre>

To set the language of the first audio stream: <pre class="example-preformatted">ffmpeg -i INPUT -metadata:s:a:0 language=eng OUTPUT </pre>

</dd> <dt><samp class="option">-keep_metadata[:<var class="var">metadata_specifier</var>] <var class="var">key</var> (output,per-metadata)</samp></dt> <dd> When re-encoding a stream, certain metadata keys that are likely to become stale (such as <code class="code">iTunSMPB</code> or <code class="code">encoded_by</code>) are automatically discarded. This option prevents a specific key from being discarded, preserving its original value from the input. Without a specifier, the key is preserved in all metadata contexts (format and streams). Use <code class="code">g</code> to restrict to format (global) metadata, or <code class="code">s</code> with an optional stream specifier to restrict to matching streams only. For example, to keep <code class="code">iTunSMPB</code> in format metadata only when re-encoding: <pre class="example-preformatted">ffmpeg -i in.mp3 -keep_metadata:g iTunSMPB out.mp3 </pre>

To keep <code class="code">NUMBER_OF_BYTES</code> only for the video stream: <pre class="example-preformatted">ffmpeg -i in.mkv -keep_metadata:s:v NUMBER_OF_BYTES out.mkv </pre>

Some keys are stored in the metadata with a suffix (e.g. <code class="code">NUMBER_OF_BYTES-eng</code>). Specifying the unsuffixed family name (e.g. <code class="code">NUMBER_OF_BYTES</code>) keeps all variants. To keep only a specific one, specify the full key explicitly: <pre class="example-preformatted">ffmpeg -i in.mkv -keep_metadata:s:v NUMBER_OF_BYTES-eng out.mkv </pre>

Some (such as <code class="code">major_brand</code>, <code class="code">minor_version</code>, and <code class="code">compatible_brands</code>) are always removed regardless of re-encoding, because they describe the source container and are meaningless in the output. <code class="code">-keep_metadata</code> applies to those keys even when stream-copying. Re-encoding-only keys (such as encoder tags and Matroska stream statistics) are removed from a stream’s metadata only when that stream is being re-encoded, and from format metadata only when at least one audio or video stream is re-encoded. Only format and stream metadata are filtered. Chapter and program metadata are never modified by this mechanism. </dd> <dt><samp class="option">-disposition[:stream_specifier] <var class="var">value</var> (output,per-stream)</samp></dt> <dd> Sets the disposition flags for a stream. Default value: by default, all disposition flags are copied from the input stream, unless the output stream

this option applies to is fed by a complex filtergraph - in that case no disposition flags are set by default. <var class=”var”>value</var> is a sequence of disposition flags separated by ’+’ or ’-’. A ’+’ prefix adds the given disposition, ’-’ removes it. If the first flag is also prefixed with ’+’ or ’-’, the resulting disposition is the default value updated by <var class=”var”>value</var>. If the first flag is not prefixed, the resulting disposition is <var class=”var”>value</var>. It is also possible to clear the disposition by setting it to 0. If no <code class=”code”>-disposition</code> options were specified for an output file, ffmpeg will automatically set the ’default’ disposition flag on the first stream of each type, when there are multiple streams of this type in the output file and no stream of that type is already marked as default. The <code class=”code”>-dispositions</code> option lists the known disposition flags. For example, to make the second audio stream the default stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:1 default out.mkv </pre>

To make the second subtitle stream the default stream and remove the default disposition from the first subtitle stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:s:0 0 -disposition:s:1 default out.mkv </pre>

To add an embedded cover/thumbnail: <pre class=”example-preformatted”>ffmpeg -i in.mp4 -i IMAGE -map 0 -map 1 -c copy -c:v:1 png -disposition:v:1 attached_pic out.mp4 </pre>

To add the ’original’ and remove the ’comment’ disposition flag from the first audio stream without removing its other disposition flags: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 +original-comment out.mkv </pre>

To remove the ’original’ and add the ’comment’ disposition flag to the first audio stream without removing its other disposition flags: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 -original+comment out.mkv </pre>

To set only the ’original’ and ’comment’ disposition flags on the first audio stream (and remove its other disposition flags): <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 original+comment out.mkv </pre>

To remove all disposition flags from the first audio stream: <pre class=”example-preformatted”>ffmpeg -i in.mkv -c copy -disposition:a:0 0 out.mkv </pre>

Not all muxers support embedded thumbnails, and those who do,

only support a few formats, like JPEG or PNG. </dd> <dt><samp class="option">-program [title=<var class="var">title</var>:][program_num=<var class="var">program_num</var>:]st=<var class="var">stream</var>[:st=<var class="var">stream</var>...] (output)</samp></dt> <dd> Creates a program with the specified <var class="var">title</var>, <var class="var">program_num</var> and adds the specified <var class="var">stream</var>(s) to it. </dd> <dt><samp class="option">-stream_group [map=<var class="var">input_file_id</var>=<var class="var">stream_group</var>][type class="var">type</var>:]st=<var class="var">stream</var>[:st=<var class="var">stream</var>][:stg=<var class="var">stream_group</var>][:id=<var class="var">stream_group_id</var>...] (output)</samp></dt> <dd> Creates a stream group of the specified <var class="var">type</var> and <var class="var">stream_group_id</var>, or by <var class="var">map</var>ping an input group, adding the specified <var class="var">stream</var>(s) and/or previously defined <var class="var">stream_group</var>(s) to it. <var class="var">type</var> can be one of the following: <dl class="table"> <dt><samp class="option">iamf_audio_element</samp></dt> <dd> Groups <var class="var">stream</var>s that belong to the same IAMF Audio Element For this group <var class="var">type</var>, the following options are available <dl class="table"> <dt><samp class="option">audio_element_type</samp></dt> <dd> The Audio Element type. The following values are supported: <dl class="table"> <dt><samp class="option">channel</samp></dt> <dd> Scalable channel audio representation </dd> <dt><samp class="option">scene</samp></dt> <dd> Ambisonics representation </dd> </dl>

</dd> <dt><samp class="option">demixing</samp></dt> <dd> Demixing information used to reconstruct a scalable channel audio representation. This option must be separated from the rest with a ’,’, and takes the following key=value options <dl class="table"> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to </dd> <dt><samp class="option">dmixp_mode</samp></dt> <dd> A pre-defined combination of demixing parameters </dd> </dl>

</dd> <dt><samp class="option">recon_gain</samp></dt> <dd> Recon gain information used to reconstruct a scalable channel audio representation. This option must be separated from the rest with a ’,’, and takes the following key=value options <dl class="table"> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to </dd> </dl>

</dd> <dt><samp class="option">layer</samp></dt> <dd> A layer defining a Channel Layout in the Audio Element. This option must be separated from the rest with a ’,’. Several ’,’ separated entries can be defined, and at least one must be set.

It takes the following ":"-separated key=value options <dl class="table"> <dt><samp class="option">ch_layout</samp></dt> <dd> The layer’s channel layout </dd> <dt><samp class="option">flags</samp></dt> <dd> The following flags are available: <dl class="table"> <dt><samp class="option">recon_gain</samp></dt> <dd> Whether to signal if recon_gain is present as metadata in parameter blocks within frames </dd> </dl>

</dd> <dt><samp class="option">output_gain</samp></dt> <dt><samp class="option">output_gain_flags</samp></dt> <dd> Which channels output_gain applies to. The following flags are available: <dl class="table"> <dt><samp class="option">FL</samp></dt> <dt><samp class="option">FR</samp></dt> <dt><samp class="option">BL</samp></dt> <dt><samp class="option">BR</samp></dt> <dt><samp class="option">TFL</samp></dt> <dt><samp class="option">TFR</samp></dt> </dl>

</dd> <dt><samp class="option">ambisonics_mode</samp></dt> <dd> The ambisonics mode. This has no effect if audio_element_type is set to channel. The following values are supported: <dl class="table"> <dt><samp class="option">mono</samp></dt> <dd> Each ambisonics channel is coded as an individual mono stream in the group </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">default_w</samp></dt> <dd> Default weight value </dd> </dl>

</dd> <dt><samp class="option">iamf_mix_presentation</samp></dt> <dd> Groups <var class="var">stream</var>s that belong to all IAMF Audio Element the same IAMF Mix Presentation references For this group <var class="var">type</var>, the following options are available <dl class="table"> <dt><samp class="option">submix</samp></dt> <dd> A sub-mix within the Mix Presentation. This option must be separated from the rest with a ’,’. Several ’,’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options <dl class="table"> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to, for post-processing the mixed audio signal to generate the audio signal for playback </dd> <dt><samp class="option">parameter_rate</samp></dt> <dd> The sample rate duration fields in parameters blocks in frames that refer to this <var class="var">parameter_id</var> are expressed as </dd> <dt><samp class="option">default_mix_gain</samp></dt> <dd> Default mix gain value to apply when there are no parameter blocks sharing the same <var class="var">parameter_id</var> for a given frame </dd> <dt><samp class="option">element</samp></dt> <dd> References an Audio Element used in this Mix Presentation to

generate the final output audio signal for playback. This option must be separated from the rest with a ’|’. Several ’|’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options: <dl class="table"> <dt><samp class="option">stg</samp></dt> <dd> The <var class="var">stream_group_id</var> for an Audio Element which this sub-mix refers to </dd> <dt><samp class="option">parameter_id</samp></dt> <dd> An identifier parameters blocks in frames may refer to, for applying any processing to the referenced and rendered Audio Element before being summed with other processed Audio Elements </dd> <dt><samp class="option">parameter_rate</samp></dt> <dd> The sample rate duration fields in parameters blocks in frames that refer to this <var class="var">parameter_id</var> are expressed as </dd> <dt><samp class="option">default_mix_gain</samp></dt> <dd> Default mix gain value to apply when there are no parameter blocks sharing the same <var class="var">parameter_id</var> for a given frame </dd> <dt><samp class="option">annotations</samp></dt> <dd> A key=value string describing the sub-mix element where "key" is a string conforming to BCP-47 that specifies the language for the "value" string. "key" must be the same as the one in the mix’s <var class="var">annotations</var> </dd> <dt><samp class="option">headphones_rendering_mode</samp></dt> <dd> Indicates whether the input channel-based Audio Element is rendered to stereo loudspeakers or spatialized with a binaural renderer when played back on headphones. This has no effect if the referenced Audio Element’s <var class="var">audio_element_type</var> is set to channel. The following values are supported: <dl class="table"> <dt><samp class="option">stereo</samp></dt> <dt><samp class="option">binaural</samp></dt> </dl>

</dd> </dl>

</dd> <dt><samp class="option">layout</samp></dt> <dd> Specifies the layouts for this sub-mix on which the loudness information was measured. This option must be separated from the rest with a ’|’. Several ’|’ separated entries can be defined, and at least one must be set. It takes the following ":"-separated key=value options: <dl class="table"> <dt><samp class="option">layout_type</samp></dt> <dd> <dl class="table"> <dt><samp class="option">loudspeakers</samp></dt> <dd> The layout follows the loudspeaker sound system convention of ITU-2051-3. </dd> <dt><samp class="option">binaural</samp></dt> <dd> The layout is binaural. </dd> </dl>

</dd> <dt><samp class="option">sound_system</samp></dt> <dd> Channel layout matching one of Sound Systems A to J of ITU-2051-3, plus 7.1.2 and 3.1.2 This has no effect if <var class="var">layout_type</var> is set to binaural.

</dd> <dt><samp class="option">integrated_loudness</samp></dt>
<dd> The program integrated loudness information, as defined in ITU-1770-4. </dd> <dt><samp class="option">digital_peak</samp></dt>
<dd> The digital (sampled) peak value of the audio signal, as defined in ITU-1770-4. </dd> <dt><samp class="option">true_peak</samp></dt>
<dd> The true peak of the audio signal, as defined in ITU-1770-4. </dd> <dt><samp class="option">dialog_anchored_loudness</samp></dt>
<dd> The Dialogue loudness information, as defined in ITU-1770-4. </dd> <dt><samp class="option">album_anchored_loudness</samp></dt>
<dd> The Album loudness information, as defined in ITU-1770-4. </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">annotations</samp></dt> <dd> A key=value string string describing the mix where "key" is a string conforming to BCP-47 that specifies the language for the "value" string. "key" must be the same as the ones in all sub-mix element’s <var class="var">annotations</var>s </dd> </dl>

</dd> <dt><samp class="option">gain_map</samp></dt> <dd> Groups together a base <var class="var">stream</var> with a gain map, which together form a pair of renditions (e.g. HDR and SDR) as per ISO 21496-1. For this group <var class="var">type</var>, the following options are available <dl class="table"> <dt><samp class="option">el_index</samp></dt> <dd> Index of the gain map stream within the group. Defaults to 0. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Size of the final image for presentation. </dd> </dl>

</dd> </dl>

E.g. to create an scalable 5.1 IAMF file from several WAV input files <pre class="example-preformatted">ffmpeg -i front.wav -i back.wav -i center.wav -i lfe.wav -map 0:0 -map 1:0 -map 2:0 -map 3:0 -c:a opus -stream_group type=iamf_audio_element:id=1:st=0:st=1:st=2:st=3, demixing=parameter_id=998, recon_gain=parameter_id=101, layer=ch_layout=stereo, layer=ch_layout=5.1(side), -stream_group type=iamf_mix_presentation:id=2:stg=0:annotations=en-us=Mix_Presentation, submix=parameter_id=100:parameter_rate=48000|element=stg=0:parameter_id=10 us=Scalable_Submix|layout=sound_system=stereo|layout=sound_system=5.1(side) -streamid 0:0 -streamid 1:1 -streamid 2:2 -streamid 3:3 output.iamf </pre>

To copy the two stream groups (Audio Element and Mix Presentation) from an input IAMF file with four streams into an mp4 output <pre class="example-preformatted">ffmpeg -i input.iamf -c:a copy -stream_group map=0=0:st=0:st=1:st=2:st=3 -stream_group map=0=1:stg=0 -streamid 0:0 -streamid 1:1 -streamid 2:2 -streamid 3:3 output.mp4 </pre>

</dd> <dt><samp class="option">-reinit_opts[:<var class="var">stream_specifier</var>] pts=<var class="var">pts</var>|<var class="var">key</var>=<var class="var">value</var>[:force_reinit=1][:<var class="var">key</var>=<var class="var">value</var>,pts=<var class="var">pts</var>...] (output,per-stream)</samp></dt> <dd> Creates a "," separated group of options to reconfigure an encoder at the specified output <var class="var">pts</var> in AV_TIME_BASE units. The pseudo-option <var class="var">force_reinit</var> ensures the encoder is fully reinitialized instead of simply reconfigured. </dd> <dt><samp class="option">-target <var class="var">type</var> (output)</samp></dt> <dd> Specify target file type (<code class="code">vcd</code>, <code class="code">svcd</code>, <code class="code">dvd</code>, <code class="code">dv</code>, <code class="code">dv50</code>). <var class="var">type</var> may be prefixed with <code class="code">pal-</code>, <code class="code">ntsc-</code> or <code class="code">film-</code> to use the corresponding standard. All the format options (bitrate, codecs, buffer sizes) are then set automatically. You can just type: <pre class="example-preformatted">ffmpeg -i myfile.avi -target vcd /tmp/vcd.mpg </pre>

Nevertheless you can specify additional options as long as you know they do not conflict with the standard, as in: <pre class="example-preformatted">ffmpeg -i myfile.avi -target vcd -bf 2 /tmp/vcd.mpg </pre>

The parameters set for each target are as follows. VCD <pre class="example-preformatted"><var class="var">pal</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x288 -r 25 -codec:v mpeg1video -g 15 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v 327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k

<var class="var">ntsc</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x240 -r 30000/1001 -codec:v mpeg1video -g 18 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v 327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k

<var class="var">film</var>: -f vcd -muxrate 1411200 -muxpreload 0.44 -packetsize 2324 -s 352x240 -r 24000/1001 -codec:v mpeg1video -g 18 -b:v 1150k -maxrate:v 1150k -minrate:v 1150k -bufsize:v 327680 -ar 44100 -ac 2 -codec:a mp2 -b:a 224k </pre>

SVCD <pre class="example-preformatted"><var class="var">pal</var>: -f svcd -packetsize 2324 -s 480x576 -pix_fmt yuv420p -r 25 -codec:v mpeg2video -g 15 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k

<var class="var">ntsc</var>: -f svcd -packetsize 2324 -s 480x480 -pix_fmt

yuv420p -r 30000/1001 -codec:v mpeg2video -g 18 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k

<var class=”var”>film</var>: -f svcd -packetsize 2324 -s 480x480 -pix_fmt yuv420p -r 24000/1001 -codec:v mpeg2video -g 18 -b:v 2040k -maxrate:v 2516k -minrate:v 0 -bufsize:v 1835008 -scan_offset 1 -ar 44100 -codec:a mp2 -b:a 224k </pre>

DVD <pre class=”example-preformatted”><var class=”var”>pal</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x576 -pix_fmt yuv420p -r 25 -codec:v mpeg2video -g 15 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k

<var class=”var”>ntsc</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x480 -pix_fmt yuv420p -r 30000/1001 -codec:v mpeg2video -g 18 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k

<var class=”var”>film</var>: -f dvd -muxrate 10080k -packetsize 2048 -s 720x480 -pix_fmt yuv420p -r 24000/1001 -codec:v mpeg2video -g 18 -b:v 6000k -maxrate:v 9000k -minrate:v 0 -bufsize:v 1835008 -ar 48000 -codec:a ac3 -b:a 448k </pre>

DV <pre class=”example-preformatted”><var class=”var”>pal</var>: -f dv -s 720x576 -pix_fmt yuv420p -r 25 -ar 48000 -ac 2

<var class=”var”>ntsc</var>: -f dv -s 720x480 -pix_fmt yuv411p -r 30000/1001 -ar 48000 -ac 2

<var class=”var”>film</var>:

-f dv

-s 720x480 -pix_fmt yuv411p -r 24000/1001

-ar 48000 -ac 2

</pre>

The <code class=”code”>dv50</code> target is identical to the <code class=”code”>dv</code> target except that the pixel format set is <code class=”code”>yuv422p</code> for all three standards.

Any user-set value for a parameter above will override the target preset value. In that case, the output may

not comply with the target standard.

</dd>

<dt><samp class="option">-dn (input/output)</samp></dt>

<dd> As an input option, blocks all data streams of a file from being filtered or

being automatically selected or mapped for any output. See <code class="code">-discard</code>

option to disable streams individually.

As an output option, disables data recording i.e. automatic selection or

mapping of any data stream. For full manual control see the <code class="code">-map</code>

option.

</dd>

<dt><samp class="option">-dframes <var class="var">number</var> (output)</samp></dt>

<dd> Set the number of data frames to output. This is an obsolete alias for

<code class="code">-frames:d</code>, which you should use instead.

</dd>

<dt><samp class="option">-frames[:<var class="var">stream_specifier</var>] <var class="var">framecount</var> (output,per-stream)</samp></dt>

<dd> Stop writing to the stream after <var class="var">framecount</var> frames.

</dd>

<dt><samp class="option">-q[:<var class="var">stream_specifier</var>] <var class="var">q</var> (output,per-stream)</samp></dt>

<dt><samp class="option">-qscale[:<var class="var">stream_specifier</var>] <var class="var">q</var> (output,per-stream)</samp></dt>

<dd> Use fixed quality scale (VBR). The meaning of <var class="var">q</var>/<var class="var">qscale</var> is

codec-dependent.

If <var class="var">qscale</var> is used without a <var class="var">stream_specifier</var> then it applies only

to the video stream, this is to maintain compatibility with previous behavior

and as specifying the same codec specific value to 2 different codecs that is

audio and video generally is not what is intended when no stream_specifier is

used.

</dd>

<dt><samp class="option">-filter[:<var class="var">stream_specifier</var>] <var class="var">filtergraph</var> (output,per-stream)</samp></dt>

<dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to

filter the stream.

<var class="var">filtergraph</var> is a description of the filtergraph to apply to

the stream, and must have a single input and a single output of the

same type of the stream. In the filtergraph, the input is associated

to the label <code class="code">in</code>, and the output to the label <code class="code">out</code>. See

the ffmpeg-filters manual for more information about the filtergraph

syntax.

See the -filter_complex option if you

want to create filtergraphs with multiple inputs and/or outputs.

</dd>

<dt><samp class="option">-reinit_filter[:<var class="var">stream_specifier</var>] <var class="var">integer</var> (input,per-stream)</samp></dt>

<dd> This boolean option determines if the filtergraph(s) to which this stream is fed gets

reinitialized when input frame parameters change mid-stream. This option is enabled by

default as most video and all audio filters cannot handle deviation in input frame properties.

Upon reinitialization, existing filter state is lost, like e.g. the frame count <code class=”code”>n</code>

reference available in some filters. Any frames buffered at time of reinitialization are lost.

The properties where a change triggers reinitialization are,

for video, frame resolution or pixel format;

for audio, sample format, sample rate, channel count or channel layout.

</dd>

<dt><samp class=”option”>-drop_changed[:<var class=”var”>stream_specifier</var>] <var class=”var”>integer</var> (input,per-stream)</samp></dt>

<dd> This boolean option determines whether a frame with differing frame parameters mid-stream

gets dropped instead of leading to filtergraph reinitialization, as that would lead to loss

of filter state. Generally useful to avoid corrupted yet decodable packets in live streaming

inputs. Default is false.

</dd>

<dt><samp class=”option”>-filter_threads <var class=”var”>nb_threads</var> (global)</samp></dt>

<dd> Defines how many threads are used to process a filter pipeline. Each pipeline

will produce a thread pool with this many threads available for parallel processing.

The default is the number of available CPUs.

</dd>

<dt><samp class=”option”>-filter_buffered_frames[:<var class=”var”>media_type</var>] <var class=”var”>nb_frames</var> (global)</samp></dt>

<dd> Defines the maximum number of buffered frames allowed in a filtergraph. Under

normal circumstances, a filtergraph should not buffer more than a few frames,

especially if frames are being fed to it and read from it in a balanced way

(which is the intended behavior in ffmpeg). That said, this option allows you

to limit the total number of frames buffered across all links in a filtergraph.

If more frames are generated, filtering is aborted and an error is returned.

The default value is 0, which means no limit.

The optional suffix <var class="var">media_type</var> may be used to set a different limit

for audio and video frames, using the suffixes <code class="code">a</code> and <code class="code">v</code>,

respectively. If the suffix is omitted, the limit applies to both audio and

video frames, separately.

</dd>

<dt><samp class="option">-pre[:<var class="var">stream_specifier</var>] <var class="var">preset_name</var> (output,per-stream)</samp></dt>

<dd> Specify the preset for matching stream(s).

</dd>

<dt><samp class="option">-stats (global)</samp></dt>

<dd> Log encoding progress/statistics as "info"-level log (see <code class="code">-loglevel</code>).

It is on by default, to explicitly disable it you need to specify <code class="code">-nostats</code>.

</dd>

<dt><samp class="option">-stats_period <var class="var">time</var> (global)</samp></dt>

<dd> Set period at which encoding progress/statistics are updated. Default is 0.5 seconds.

</dd>

<dt><samp class="option">-print_graphs (global)</samp></dt>

<dd> Prints execution graph details to stderr in the format set via -print_graphs_format.

</dd>

<dt><samp class="option">-print_graphs_file <var class="var">filename</var> (global)</samp></dt>

<dd> Writes execution graph details to the specified file in the format set via -print_graphs_format.

</dd>

<dt><samp class="option">-print_graphs_format <var class="var">format</var> (global)</samp></dt>

<dd> Sets the output format (available formats are: default, compact, csv, flat, ini, json, xml, mermaid, mermaidhtml)

The default format is json.

</dd>

<dt><samp class="option">-progress <var class="var">url</var> (global)</samp></dt>

<dd> Send program-friendly progress information to <var class="var">url</var>.

Progress information is written periodically and at the end of

the encoding process. It is made of "<var class="var">key</var>=<var class="var">value</var>" lines. <var class="var">key</var>

consists of only alphanumeric characters. The last key of a sequence of

progress information is always "progress" with the value "continue" or "end".

The update period is set using <code class="code">-stats_period</code>.

For example, log progress information to stdout:

<pre class="example-preformatted">ffmpeg -progress pipe:1 -i in.mkv out.mkv

</pre>

</dd> <dt><samp class="option">-stdin</samp></dt> <dd> Enable interaction on standard input. On by default unless standard input is used as an input. To explicitly disable interaction you need to specify <code class="code">-nostdin</code>. Disabling interaction on standard input is useful, for example, if ffmpeg is in the background process group. Roughly the same result can be achieved with <code class="code">ffmpeg ... < /dev/null</code> but it requires a shell. </dd> <dt><samp class="option">-debug_ts (global)</samp></dt> <dd> Print timestamp/latency information. It is off by default. This option is mostly useful for testing and debugging purposes, and the output format may change from one version to another, so it should not be employed by portable scripts. See also the option <code class="code">-fdebug ts</code>. </dd> <dt><samp class="option">-attach <var class="var">filename</var> (output)</samp></dt> <dd> Add an attachment to the output file. This is supported by a few formats like Matroska for e.g. fonts used in rendering subtitles. Attachments are implemented as a specific type of stream, so this option will add a new stream to the file. It is then possible to use per-stream options on this stream in the usual way. Attachment streams created with this option will be created after all the other streams (i.e. those created with <code class="code">-map</code> or automatic mappings). Note that for Matroska you also have to set the mimetype metadata tag: <pre class="example-preformatted">ffmpeg -i INPUT -attach DejaVuSans.ttf -metadata:s:2 mimetype=application/x-truetype-font out.mkv </pre> (assuming that the attachment stream will be third in the output file). </dd> <dt><samp class="option">-dump_attachment[:<var class="var">stream_specifier</var>] <var class="var">filename</var> (input,per-stream)</samp></dt> <dd> Extract the matching attachment stream into a file named <var class="var">filename</var>. If <var class="var">filename</var> is empty, then the value of the <code class="code">filename</code> metadata tag will be used. E.g. to extract the first attachment to a file named ’out.ttf’: <pre class="example-preformatted">ffmpeg -dump_attachment:t:0 out.ttf -i INPUT </pre> To extract all attachments to files determined by the <code class="code">filename</code> tag: <pre class="example-preformatted">ffmpeg -dump_attachment:t "" -i INPUT </pre>

Technical note – attachments are implemented as codec extradata, so this option can actually be used to extract extradata from any stream, not just attachments. </dd> </dl>

 <h3 class="section">5.5 Video Options# TOC</h3>

<dl class="table"> <dt><samp class="option">-vframes <var class="var">number</var> (output)</samp></dt> <dd> Set the number of video frames to output. This is an obsolete alias for <code class="code">-frames:v</code>, which you should use instead. </dd> <dt><samp class="option">-r[:<var class="var">stream_specifier</var>] <var class="var">fps</var> (input/output,per-stream)</samp></dt> <dd> Set frame rate (Hz value, fraction or abbreviation). As an input option, ignore any timestamps stored in the file and instead generate timestamps assuming constant frame rate <var class="var">fps</var>. This is not the same as the <samp class="option">-framerate</samp> option used for some input formats like image2 or v4l2 (it used to be the same in older versions of FFmpeg). If in doubt use <samp class="option">-framerate</samp> instead of the input option <samp class="option">-r</samp>. As an output option: <dl class="table"> <dt><samp class="option">video encoding</samp></dt> <dd> Duplicate or drop frames right before encoding them to achieve constant output frame rate <var class="var">fps</var>. </dd> <dt><samp class="option">video streamcopy</samp></dt> <dd> Indicate to the muxer that <var class="var">fps</var> is the stream frame rate. No data is dropped or duplicated in this case. This may produce invalid files if <var class="var">fps</var> does not match the actual stream frame rate as determined by packet timestamps. See also the <code class="code">setts</code> bitstream filter. </dd> </dl>

</dd> <dt><samp class="option">-fpsmax[:<var class="var">stream_specifier</var>] <var class="var">fps</var> (output,per-stream)</samp></dt> <dd> Set maximum frame rate (Hz value, fraction or abbreviation). Clamps output frame rate when output framerate is auto-set and is higher than this value. Useful in batch processing or when input framerate is wrongly detected as very high. It cannot be set together with <code class="code">-r</code>. It is ignored during streamcopy. </dd> <dt><samp class="option">-s[:<var class="var">stream_specifier</var>] <var class="var">size</var> (input/output,per-stream)</samp></dt> <dd> Set frame size. As an input option, this is a shortcut for the <samp class="option">video_size</samp> private option, recognized by some demuxers for which the frame size is either not stored in the file or is configurable – e.g. raw video or video grabbers.

As an output option, this inserts the <code class=”code”>scale</code> video filter to the end of the corresponding filtergraph. Please use the <code class=”code”>scale</code> filter directly to insert it at the beginning or some other place. The format is ‘<samp class=”samp”>wxh</samp>’ (default - same as source). </dd> <dt><samp class=”option”>-aspect[:<var class=”var”>stream_specifier</var>] <var class=”var”>aspect</var> (output,per-stream)</samp></dt> <dd> Set the video display aspect ratio specified by <var class=”var”>aspect</var>. <var class=”var”>aspect</var> can be a floating point number string, or a string of the form <var class=”var”>num</var>:<var class=”var”>den</var>, where <var class=”var”>num</var> and <var class=”var”>den</var> are the numerator and denominator of the aspect ratio. For example "4:3", "16:9", "1.3333", and "1.7777" are valid argument values. If used together with <samp class=”option”>-vcodec copy</samp>, it will affect the aspect ratio stored at container level, but not the aspect ratio stored in encoded frames, if it exists. </dd> <dt><samp class=”option”>-stereo3d[:<var class=”var”>stream_specifier</var>] <var class=”var”>layout</var> (output,per-stream)</samp></dt> <dd> Set stereoscopic 3D layout metadata on the output stream. This does not rearrange pixels; use <samp class=”option”>-vf stereo3d</samp> for that. The metadata is stored as <code class=”code”>AV_PKT_DATA_STEREO3D</code> side data on the output stream. Muxers that already consume that side data will write the corresponding container boxes or elements, including: <ul class=”itemize mark-bullet”> MP4 Google spatial-media <code class=”code”>st3d</code> (requires <samp class=”option”>-strict unofficial</samp>) QuickTime MOV Apple <code class=”code”>vexu</code> eyes (no <code class=”code”>st3d</code>) Matroska <code class=”code”>StereoMode</code>; <samp class=”option”>-stereo3d</samp> overrides the <code class=”code”>stereo_mode</code> metadata tag, including a value inherited from the input

<var class=”var”>layout</var> is case-insensitive. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>2d</samp>’</dt> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> Monoscopic video. </dd> <dt>‘<samp class=”samp”>sbs</samp>’</dt> <dt>‘<samp class=”samp”>sbsl</samp>’</dt> <dt>‘<samp class=”samp”>side by side</samp>’</dt> <dd> Side-by-side, left eye on the left. </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dt>‘<samp class=”samp”>tbl</samp>’</dt> <dt>‘<samp class=”samp”>top and bottom</samp>’</dt> <dd> Top-and-bottom, left eye on top. </dd> </dl>

Inverted layouts such as ‘<samp class=”samp”>sbsr</samp>’ are not supported. An empty stream specifier applies to video streams only

and is ignored for audio and other types. An explicit specifier on a non-video stream (for example <samp class="option">-stereo3d:a</samp>) is an error. Recommended form: <samp class="option">-stereo3d:v sbsl</samp>. </dd> <dt><samp class="option">-display_rotation[:<var class="var">stream_specifier</var>] <var class="var">rotation</var> (input,per-stream)</samp></dt> <dd> Set video rotation metadata. <var class="var">rotation</var> is a decimal number specifying the amount in degree by which the video should be rotated counter-clockwise before being displayed. This option overrides the rotation/display transform metadata stored in the file, if any. When the video is being transcoded (rather than copied) and <code class="code">-autorotate</code> is enabled, the video will be rotated at the filtering stage. Otherwise, the metadata will be written into the output file if the muxer supports it. If the <code class="code">-display_hflip</code> and/or <code class="code">-display_vflip</code> options are given, they are applied after the rotation specified by this option. </dd> <dt><samp class="option">-display_hflip[:<var class="var">stream_specifier</var>] (input,per-stream)</samp></dt> <dd> Set whether on display the image should be horizontally flipped. See the <code class="code">-display_rotation</code> option for more details. </dd> <dt><samp class="option">-display_vflip[:<var class="var">stream_specifier</var>] (input,per-stream)</samp></dt> <dd> Set whether on display the image should be vertically flipped. See the <code class="code">-display_rotation</code> option for more details. </dd> <dt><samp class="option">-mastering_display[:<var class="var">stream_specifier</var>] <var class="var">G(%u,%u)B(%u,%u)R(%u,%u)WP(%u,%u)L(%u,% (input,per-stream)</samp></dt> <dd> Set video mastering display metadata. <var class="var">G(%u,%u)B(%u,%u)R(%u,%u)WP(%u,%u)L is a string specifying X,Y display primaries for GBR channels and white point (WP) in units of 0.00002, and max-min luminance (L) values in units of 0.0001 candela per meter square. The values are unsigned integers representing the numerator of a rational with an implicit denominator of 50000 for GBR and (WP), and implicit denominator 10000 for (L). This option overrides the mastering display metadata stored in the file, if any. </dd> <dt><samp class="option">-content_light[:<var class="var">stream_specifier</var>] <var class="var">%u,%u</var> (input,per-stream)</samp></dt> <dd> Set video content light metadata. <var class="var">%u,%u</var> is a string specifying max content light level and maximum picture average light level. This option overrides the content light metadata stored in the file, if any. </dd> <dt><samp class="option">-vn (input/output)</samp></dt> <dd> As an input option, blocks all video streams of a file from being filtered or being automatically selected or mapped for any output. See <code class="code">-discard</code> option to disable streams individually.

 As an output option, disables video recording i.e. automatic selection or mapping of any video stream. For full manual control see the <code class="code">-map</code> option. </dd> <dt><samp class="option">-vcodec <var class="var">codec</var> (output)</samp></dt> <dd> Set the video codec. This is an alias for <code class="code">-codec:v</code>. </dd> <dt><samp class="option">-pass[:<var class="var">stream_specifier</var>] <var class="var">n</var> (output,per-stream)</samp></dt> <dd> Select the pass number (1 or 2). It is used to do two-pass video encoding. The statistics of the video are recorded in the first pass into a log file (see also the option -passlogfile), and in the second pass that log file is used to generate the video at the exact requested bitrate. On pass 1, you may just deactivate audio and set output to null, examples for Windows and Unix: <pre class="example-preformatted">ffmpeg -i foo.mov -c:v libxvid -pass 1 -an -f rawvideo -y NUL ffmpeg -i foo.mov -c:v libxvid -pass 1 -an -f rawvideo -y /dev/null </pre>

</dd> <dt><samp class="option">-passlogfile[:<var class="var">stream_specifier</var>] <var class="var">prefix</var> (output,per-stream)</samp></dt> <dd> Set two-pass log file name prefix to <var class="var">prefix</var>, the default file name prefix is “ffmpeg2pass”. The complete file name will be <samp class="file">PREFIX-N.log</samp>, where N is a number specific to the output stream </dd> <dt><samp class="option">-vf <var class="var">filtergraph</var> (output)</samp></dt> <dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to filter the stream. This is an alias for <code class="code">-filter:v</code>, see the -filter option. </dd> <dt><samp class="option">-autorotate</samp></dt> <dd> Automatically rotate the video according to file metadata. Enabled by default, use <samp class="option">-noautorotate</samp> to disable it. </dd> <dt><samp class="option">-autoscale</samp></dt> <dd> Automatically scale the video according to the resolution of first frame. Enabled by default, use <samp class="option">-noautoscale</samp> to disable it. When autoscale is disabled, all output frames of filter graph might not be in the same resolution and may be inadequate for some encoder/muxer. Therefore, it is not recommended to disable it unless you really know what you are doing. Disable autoscale at your own risk. </dd> </dl>

 <h3 class="section">5.6 Advanced Video options# TOC</h3>

<dl class="table"> <dt><samp class="option">-pix_fmt[:<var class="var">stream_specifier</var>] <var class="var">format</var> (input/output,per-stream)</samp></dt> <dd> Set pixel format. Use <code

class=”code”>-pix_fmts</code> to show all the supported pixel formats. If the selected pixel format can not be selected, ffmpeg will print a warning and select the best pixel format supported by the encoder. If <var class=”var”>pix_fmt</var> is prefixed by a <code class=”code”>+</code>, ffmpeg will exit with an error if the requested pixel format can not be selected, and automatic conversions inside filtergraphs are disabled. If <var class=”var”>pix_fmt</var> is a single <code class=”code”>+</code>, ffmpeg selects the same pixel format as the input (or graph output) and automatic conversions are disabled. </dd> <dt><samp class=”option”>-sws_flags <var class=”var”>flags</var> (input/output)</samp></dt> <dd> Set default flags for the libswscale library. These flags are used by automatically inserted <code class=”code”>scale</code> filters and those within simple filtergraphs, if not overridden within the filtergraph definition. See the <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#scaler_005foptions”>ffmpeg-scaler manual for a list of scaler options. </dd> <dt><samp class=”option”>-rc_override[:<var class=”var”>stream_specifier</var>] <var class=”var”>override</var> (output,per-stream)</samp></dt> <dd> Rate control override for specific intervals, formatted as "int,int,int" list separated with slashes. Two first values are the beginning and end frame numbers, last one is quantizer to use if positive, or quality factor if negative. </dd> <dt><samp class=”option”>-vstats</samp></dt> <dd> Dump video coding statistics to <samp class=”file”>vstats_HHMMSS.log</samp>. See the vstats file format section for the format description. </dd> <dt><samp class=”option”>-vstats_file <var class=”var”>file</var></samp></dt> <dd> Dump video coding statistics to <var class=”var”>file</var>. See the vstats file format section for the format description. </dd> <dt><samp class=”option”>-vstats_version <var class=”var”>file</var></samp></dt> <dd> Specify which version of the vstats format to use. Default is <code class=”code”>2</code>. See the vstats file format section for the format description. </dd> <dt><samp class=”option”>-vtag <var class=”var”>fourcc/tag</var> (output)</samp></dt> <dd> Force video tag/fourcc. This is an alias for <code class=”code”>-tag:v</code>. </dd> <dt><samp class=”option”>-force_key_frames[:<var class=”var”>stream_specifier</var>] <var class=”var”>time</var>[,<var class=”var”>time</var>...] (output,per-stream)</samp></dt> <dt><samp class=”option”>-force_key_frames[:<var class=”var”>stream_specifier</var>] expr:<var class=”var”>expr</var> (output,per-stream)</samp></dt> <dt><samp class=”option”>-force_key_frames[:<var class=”var”>stream_specifier</var>] source (output,per-stream)</samp></dt> <dt><samp class=”option”>-force_key_frames[:<var class=”var”>stream_specifier</var>] scd_metadata (output,per-

stream)</samp></dt> <dd> <var class="var">force_key_frames</var>
can take arguments of the following form: <dl class="table">
<dt><samp class="option"><var class="var">time</var>[,<var class="var">time</var>...]</samp></d
<dd> If the argument consists of timestamps, ffmpeg will round the specified times to the nearest output timestamp as per the encoder time base and force a keyframe at the first frame having timestamp equal or greater than the computed timestamp. Note that if the encoder time base is too coarse, then the keyframes may be forced on frames with timestamps lower than the specified time. The default encoder time base is the inverse of the output framerate but may be set otherwise via <code class="code">-enc_time_base</code>.
If one of the times is "<code class="code">chapters</code>[<var class="var">delta</var>]", it is expanded into the time of the beginning of all chapters in the file, shifted by <var class="var">delta</var>, expressed as a time in seconds. This option can be useful to ensure that a seek point is present at a chapter mark or any other designated place in the output file. For example, to insert a key frame at 5 minutes, plus key frames 0.1 second before the beginning of every chapter:
<pre class="example-preformatted">-force_key_frames 0:05:00,chapters-0.1
</pre>

</dd> <dt><samp class="option">expr:<var class="var">expr</var></samp></dt>
<dd> If the argument is prefixed with <code class="code">expr:</code>, the string <var class="var">expr</var> is interpreted like an expression and is evaluated for each frame. A key frame is forced in case the evaluation is non-zero. The expression in <var class="var">expr</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> the number of current processed frame, starting from 0 </dd> <dt><samp class="option">n_forced</samp></dt> <dd> the number of forced frames </dd> <dt><samp class="option">prev_forced_n</samp></dt>
<dd> the number of the previous forced frame, it is <code class="code">NAN</code> when no keyframe was forced yet </dd> <dt><samp class="option">prev_forced_t</samp></dt>
<dd> the time of the previous forced frame, it is <code class="code">NAN</code> when no keyframe was forced yet </dd> <dt><samp class="option">t</samp></dt>
<dd> the time of the current processed frame </dd> </dl>

For example to force a key frame every 5 seconds, you can specify: <pre class="example-preformatted">-force_key_frames expr:gte(t,n_forced*5) </pre>

To force a key frame 5 seconds after the time of the last forced one, starting from second 13: <pre class="example-preformatted">-force_key_frames expr:if(isnan(prev_forced_t),gte(t,13),gte(t,prev_forced_t+5))
</pre>

</dd> <dt><samp class="option">source</samp></dt> <dd> If the argument is <code class="code">source</code>, ffmpeg will force a key frame if the current frame being encoded is marked as a key frame in

its source. In cases where this particular source frame has to be dropped, enforce the next available frame to become a key frame instead. </dd> <dt><samp class="option">scd_metadata</samp></dt> <dd> If the argument is <code class="code">scd_metadata</code>, ffmpeg will force a key frame if the current frame contains a metadata entry with the key <code class="code">lavfi.scd.time</code>. The metadata can be added by filters like <code class="code">scdet</code> and <code class="code">scdet_vulkan</code>. Avoid inserting filters that duplicate frames after <code class="code">scdet</code>, as this can cause duplicate metadata for multiple frames and repeated insertion of key frames. </dd> </dl>

Note that forcing too many keyframes is very harmful for the lookahead algorithms of certain encoders: using fixed-GOP options or similar would be more efficient. </dd> <dt><samp class="option">-apply_cropping[:<var class="var">stream_specifier</var>] <var class="var">source</var> (input,per-stream)</samp></dt> <dd> Automatically crop the video after decoding according to file metadata. Default is all. <dl class="table"> <dt><samp class="option">none (0)</samp></dt> <dd> Don’t apply any cropping metadata. </dd> <dt><samp class="option">all (1)</samp></dt> <dd> Apply both codec and container level croppping. This is the default mode. </dd> <dt><samp class="option">codec (2)</samp></dt> <dd> Apply codec level croppping. </dd> <dt><samp class="option">container (3)</samp></dt> <dd> Apply container level croppping. </dd> </dl>

</dd> <dt><samp class="option">-copyinkf[:<var class="var">stream_specifier</var>] (output,per-stream)</samp></dt> <dd> When doing stream copy, copy also non-key frames found at the beginning. </dd> <dt><samp class="option">-init_hw_device <var class="var">type</var>[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">type</var> called <var class="var">name</var>, using the given device parameters. If no name is specified it will receive a default name of the form "<var class="var">type</var>%d". The meaning of <var class="var">device</var> and the following arguments depends on the device type: <dl class="table"> <dt><samp class="option">cuda</samp></dt> <dd> <var class="var">device</var> is the number of the CUDA device. The following options are recognized: <dl class="table"> <dt><samp class="option">primary_ctx</samp></dt> <dd> If set to 1, uses the primary device context instead of creating a new one. </dd> </dl>

Examples: <dl class="table"> <dt>-init_hw_device cuda:1</dt> <dd> Choose the second device on the system. </dd> <dt>-init_hw_device

cuda:0,primary_ctx=1</dt> <dd> Choose the first device and use the primary device context. </dd> </dl>

</dd> <dt><samp class="option">dxva2</samp></dt> <dd> <var class="var">device</var> is the number of the Direct3D 9 display adapter. </dd> <dt><samp class="option">d3d11va</samp></dt> <dd> <var class="var">device</var> is the number of the Direct3D 11 display adapter. If not specified, it will attempt to use the default Direct3D 11 display adapter or the first Direct3D 11 display adapter whose hardware VendorId is specified by ‘<samp class="samp">vendor_id</samp>’. Examples: <dl class="table"> <dt>-init_hw_device d3d11va</dt> <dd> Create a d3d11va device on the default Direct3D 11 display adapter. </dd> <dt>-init_hw_device d3d11va:1</dt> <dd> Create a d3d11va device on the Direct3D 11 display adapter specified by index 1. </dd> <dt>-init_hw_device d3d11va:,vendor_id=0x8086</dt> <dd> Create a d3d11va device on the first Direct3D 11 display adapter whose hardware VendorId is 0x8086. </dd> </dl>

</dd> <dt><samp class="option">vaapi</samp></dt> <dd> <var class="var">device</var> is either an X11 display name, a DRM render node or a DirectX adapter index. If not specified, it will attempt to open the default X11 display ($DISPLAY) and then the first DRM render node (/dev/dri/renderD128), or the default DirectX adapter on Windows. The following options are recognized: <dl class="table"> <dt><samp class="option">kernel_driver</samp></dt> <dd> When <var class="var">device</var> is not specified, use this option to specify the name of the kernel driver associated with the desired device. This option is available only when the hardware acceleration method drm and vaapi are enabled. </dd> <dt><samp class="option">vendor_id</samp></dt> <dd> When <var class="var">device</var> and <var class="var">kernel_driver</var> are not specified, use this option to specify the vendor id associated with the desired device. This option is available only when the hardware acceleration method drm and vaapi are enabled and kernel_driver is not specified. </dd> </dl>

Examples: <dl class="table"> <dt>-init_hw_device vaapi</dt> <dd> Create a vaapi device on the default device. </dd> <dt>-init_hw_device vaapi:/dev/dri/renderD129</dt> <dd> Create a vaapi device on DRM render node <samp class="file">/dev/dri/renderD129</samp>. </dd> <dt>-init_hw_device vaapi:1</dt> <dd> Create a vaapi device on DirectX adapter 1. </dd> <dt><em

class=”emph”>-init_hw_device vaapi:,kernel_driver=i915</dt>
<dd> Create a vaapi device on a device associated with kernel driver
‘<samp class=”samp”>i915</samp>’. </dd> <dt><em
class=”emph”>-init_hw_device vaapi:,vendor_id=0x8086</dt>
<dd> Create a vaapi device on a device associated with vendor id
‘<samp class=”samp”>0x8086</samp>’. </dd> </dl>

</dd> <dt><samp class=”option”>vdpau</samp></dt> <dd> <var
class=”var”>device</var> is an X11 display name. If not specified, it will attempt to open the default X11 display ($DISPLAY).
 </dd> <dt><samp class=”option”>qsv</samp></dt> <dd> <var
class=”var”>device</var> selects a value in ‘<samp class=”samp”>MFX_IMPL_*</samp>’.
Allowed values are: <dl class=”table”> <dt><samp class=”option”>auto</samp></dt>
<dt><samp class=”option”>sw</samp></dt> <dt><samp class=”option”>hw</samp></dt>
<dt><samp class=”option”>auto_any</samp></dt> <dt><samp
class=”option”>hw_any</samp></dt> <dt><samp class=”option”>hw2</samp></dt>
<dt><samp class=”option”>hw3</samp></dt> <dt><samp class=”option”>hw4</samp></dt>
</dl> If not specified, ‘<samp class=”samp”>auto_any</samp>’
is used. (Note that it may be easier to achieve the desired result
for QSV by creating the platform-appropriate subdevice (‘<samp
class=”samp”>dxva2</samp>’ or ‘<samp class=”samp”>d3d11va</samp>’
or ‘<samp class=”samp”>vaapi</samp>’) and then deriving
a QSV device from that.) The following options are recognized:
<dl class=”table”> <dt><samp class=”option”>child_device</samp></dt>
<dd> Specify a DRM render node on Linux or DirectX adapter on Windows.
</dd> <dt><samp class=”option”>child_device_type</samp></dt>
<dd> Choose platform-appropriate subdevice type. On Windows
‘<samp class=”samp”>d3d11va</samp>’ is used as default
subdevice type when <code class=”code”>--enable-libvpl</code> is specified
at configuration time, ‘<samp class=”samp”>dxva2</samp>’
is used as default subdevice type when <code class=”code”>--enable-
libmfx</code> is specified at configuration time. On Linux user can use
‘<samp class=”samp”>vaapi</samp>’ only as subdevice type.
</dd> </dl>

Examples: <dl class=”table”> <dt>-
init_hw_device qsv:hw,child_device=/dev/dri/renderD129</dt>
<dd> Create a QSV device with ‘<samp class=”samp”>MFX_IMPL_HARDWARE</samp>&rsq
on DRM render node <samp class=”file”>/dev/dri/renderD129</samp>.
 </dd> <dt>-init_hw_device qsv:hw,child_device=1</dt>
<dd> Create a QSV device with ‘<samp class=”samp”>MFX_IMPL_HARDWARE</samp>&rsq
on DirectX adapter 1. </dd> <dt>-init_hw_device
qsv:hw,child_device_type=d3d11va</dt> <dd> Choose the GPU
subdevice with type ‘<samp class=”samp”>d3d11va</samp>’
and create QSV device with ‘<samp class=”samp”>MFX_IMPL_HARDWARE</samp>’.
 </dd> <dt>-init_hw_device qsv:hw,child_device_type=dxva2</dt>
<dd> Choose the GPU subdevice with type ‘<samp class=”samp”>dxva2</samp>’

and create QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>’.
 </dd> <dt>-init_hw_device qsv:hw,child_device=1,child_device_type=d3d11va
<dd> Create a QSV device with ‘<samp class="samp">MFX_IMPL_HARDWARE</samp>&rsq
on DirectX adapter 1 with subdevice type ‘<samp class="samp">d3d11va</samp>’.
 </dd> <dt>-init_hw_device vaapi=va:/dev/dri/renderD129
-init_hw_device qsv=hw1@<var class="var">va</var></dt>
<dd> Create a VAAPI device called ‘<samp class="samp">va</samp>’
on <samp class="file">/dev/dri/renderD129</samp>, then derive a QSV
device called ‘<samp class="samp">hw1</samp>’ from device
‘<samp class="samp">va</samp>’. </dd> </dl>

</dd> <dt><samp class="option">opencl</samp></dt> <dd> <var
class="var">device</var> selects the platform and device as <em
class="emph">platform_index.device_index. The set
of devices can also be filtered using the key-value pairs to find only de-
vices matching particular platform or device strings. The
strings usable as filters are: <dl class="table"> <dt><samp
class="option">platform_profile</samp></dt> <dt><samp class="option">platform_version</samp><
<dt><samp class="option">platform_name</samp></dt> <dt><samp
class="option">platform_vendor</samp></dt> <dt><samp class="option">platform_extensions</samp
<dt><samp class="option">device_name</samp></dt> <dt><samp
class="option">device_vendor</samp></dt> <dt><samp class="option">driver_version</samp></dt>
<dt><samp class="option">device_version</samp></dt> <dt><samp
class="option">device_profile</samp></dt> <dt><samp class="option">device_extensions</samp></d
<dt><samp class="option">device_type</samp></dt> </dl>

The indices and filters must together uniquely select a device.
Examples: <dl class="table"> <dt>-
init_hw_device opencl:0.1</dt> <dd> Choose the second device
on the first platform. </dd> <dt>-init_hw_device
opencl:,device_name=Foo9000</dt> <dd> Choose the device
with a name containing the string Foo9000.
</dd> <dt>-init_hw_device opencl:1,device_type=gpu,device_extensions=cl_khr_fp1
<dd> Choose the GPU device on the second platform supporting the <em
class="emph">cl_khr_fp16 extension. </dd> </dl>

</dd> <dt><samp class="option">vulkan</samp></dt> <dd> If
<var class="var">device</var> is an integer, it selects the device by its index
in a system-dependent list of devices. If <var class="var">device</var>
is any other string, it selects the first device with a name containing that
string as a substring. The following options are recognized:
<dl class="table"> <dt><samp class="option">debug</samp></dt>
<dd> If set to 1, enables the validation layer, if installed. </dd>
<dt><samp class="option">linear_images</samp></dt> <dd> If set
to 1, images allocated by the hwcontext will be linear and locally mappable.
</dd> <dt><samp class="option">instance_extensions</samp></dt>
<dd> A plus separated list of additional instance extensions to enable.

</dd> <dt><samp class="option">device_extensions</samp></dt>
<dd> A plus separated list of additional device extensions to enable.
</dd> </dl>

Examples: <dl class="table"> <dt>-
init_hw_device vulkan:1</dt> <dd> Choose the second device
on the system. </dd> <dt>-init_hw_device
vulkan:RADV</dt> <dd> Choose the first device with a
name containing the string RADV. </dd>
<dt>-init_hw_device vulkan:0,instance_extensions=VK_KHR_wayland_surface+VK_
<dd> Choose the first device and enable the Wayland and XCB instance
extensions. </dd> </dl>

</dd> </dl>

</dd> <dt><samp class="option">-init_hw_device <var class="var">type</var>[=<var
class="var">name</var>]@<var class="var">source</var></samp></dt>
<dd> Initialise a new hardware device of type <var class="var">type</var>
called <var class="var">name</var>, deriving it from the existing device
with the name <var class="var">source</var>. </dd> <dt><samp
class="option">-init_hw_device list</samp></dt> <dd> List all hard-
ware device types supported in this build of ffmpeg. </dd> <dt><samp
class="option">-filter_hw_device <var class="var">name</var></samp></dt>
<dd> Pass the hardware device called <var class="var">name</var> to
all filters in any filter graph. This can be used to set the device to upload to with
the <code class="code">hwupload</code> filter, or the device to map to with
the <code class="code">hwmap</code> filter. Other filters may also make
use of this parameter when they require a hardware device. Note that this is typ-
ically only required when the input is not already in hardware frames - when it
is, filters will derive the device they require from the context of the frames they
receive as input. This is a global setting, so all filters will receive
the same device. </dd> <dt><samp class="option">-hwaccel[:<var
class="var">stream_specifier</var>] <var class="var">hwaccel</var>
(input,per-stream)</samp></dt> <dd> Use
hardware acceleration to decode the matching stream(s). The allowed val-
ues of <var class="var">hwaccel</var> are: <dl class="table">
<dt><samp class="option">none</samp></dt> <dd> Do not use
any hardware acceleration (the default). </dd> <dt><samp
class="option">auto</samp></dt> <dd> Automatically select the hard-
ware acceleration method. </dd> <dt><samp class="option">vdpau</samp></dt>
<dd> Use VDPAU (Video Decode and Presentation API for Unix) hard-
ware acceleration. </dd> <dt><samp class="option">dxva2</samp></dt>
<dd> Use DXVA2 (DirectX Video Acceleration) hardware accelera-
tion. </dd> <dt><samp class="option">d3d11va</samp></dt>
<dd> Use D3D11VA (DirectX Video Acceleration) hardware acceler-
ation. </dd> <dt><samp class="option">vaapi</samp></dt>
<dd> Use VAAPI (Video Acceleration API) hardware acceleration.

</dd> <dt><samp class="option">qsv</samp></dt> <dd> Use the Intel QuickSync Video acceleration for video transcoding. Unlike most other values, this option does not enable accelerated decoding (that is used automatically whenever a qsv decoder is selected), but accelerated transcoding, without copying the frames into the system memory. For it to work, both the decoder and the encoder must support QSV acceleration and no filters must be used. </dd> <dt><samp class="option">videotoolbox</samp></dt> <dd> Use Video Toolbox hardware acceleration. </dd> </dl>

This option has no effect if the selected hwaccel is not available or not supported by the chosen decoder. Note that most acceleration methods are intended for playback and will not be faster than software decoding on modern CPUs. Additionally, <code class="command">ffmpeg</code> will usually need to copy the decoded frames from the GPU memory into the system memory, resulting in further performance loss. This option is thus mainly useful for testing. </dd> <dt><samp class="option">-hwaccel_device[:<var class="var">stream_specifier</var>] <var class="var">hwaccel_device</var> (input,per-stream)</samp></dt> <dd> Select a device to use for hardware acceleration. This option only makes sense when the <samp class="option">-hwaccel</samp> option is also specified. It can either refer to an existing device created with <samp class="option">-init_hw_device</samp> by name, or it can create a new device as if ‘<samp class="samp">-init_hw_device</samp>’ <var class="var">type</var>:<var class="var">hwaccel_device</var> were called immediately before. </dd> <dt><samp class="option">-hwaccels</samp></dt> <dd> List all hardware acceleration components enabled in this build of ffmpeg. Actual runtime availability depends on the hardware and its suitable driver being installed. </dd> <dt><samp class="option">-fix_sub_duration_heartbeat[:<var class="var">stream_specifier</var>]</samp></dt> <dd> Set a specific output video stream as the heartbeat stream according to which to split and push through currently in-progress subtitle upon receipt of a random access packet. This lowers the latency of subtitles for which the end packet or the following subtitle has not yet been received. As a drawback, this will most likely lead to duplication of subtitle events in order to cover the full duration, so when dealing with use cases where latency of when the subtitle event is passed on to output is not relevant this option should not be utilized. Requires <samp class="option">-fix_sub_duration</samp> to be set for the relevant input subtitle stream for this to have any effect, as well as for the input subtitle stream having to be directly mapped to the same output in which the heartbeat stream resides. </dd> </dl>

 <h3 class="section">5.7 Audio Options# <a class="anchor hidden-xs"href="#toc-

Audio-Options" aria-hidden="true">TOC</h3>

<dl class="table"> <dt><samp class="option">-aframes <var class="var">number</var> (output)</samp></dt> <dd> Set the number of audio frames to output. This is an obsolete alias for <code class="code">-frames:a</code>, which you should use instead. </dd> <dt><samp class="option">-ar[:<var class="var">stream_specifier</var>] <var class="var">freq</var> (input/output,per-stream)</samp></dt> <dd> Set the audio sampling frequency. For output streams it is set by default to the frequency of the corresponding input stream. For input streams this option only makes sense for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer options. </dd> <dt><samp class="option">-aq <var class="var">q</var> (output)</samp></dt> <dd> Set the audio quality (codec-specific, VBR). This is an alias for -q:a. </dd> <dt><samp class="option">-ac[:<var class="var">stream_specifier</var>] <var class="var">channels</var> (input/output,per-stream)</samp></dt> <dd> Set the number of audio channels. For output streams it is set by default to the number of input audio channels. For input streams this option only makes sense for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer options. </dd> <dt><samp class="option">-an (input/output)</samp></dt> <dd> As an input option, blocks all audio streams of a file from being filtered or being automatically selected or mapped for any output. See <code class="code">-discard</code> option to disable streams individually. As an output option, disables audio recording i.e. automatic selection or mapping of any audio stream. For full manual control see the <code class="code">-map</code> option. </dd> <dt><samp class="option">-acodec <var class="var">codec</var> (input/output)</samp></dt> <dd> Set the audio codec. This is an alias for <code class="code">-codec:a</code>. </dd> <dt><samp class="option">-sample_fmt[:<var class="var">stream_specifier</var>] <var class="var">sample_fmt</var> (output,per-stream)</samp></dt> <dd> Set the audio sample format. Use <code class="code">-sample_fmts</code> to get a list of supported sample formats. </dd> <dt><samp class="option">-af <var class="var">filtergraph</var> (output)</samp></dt> <dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to filter the stream. This is an alias for <code class="code">-filter:a</code>, see the -filter option. </dd> </dl>

 <h3 class="section">5.8 Advanced Audio options# <a class="anchor hidden-xs"href="#toc-Advanced-Audio-options" aria-

hidden=”true”>TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-atag <var class=”var”>fourcc/tag</var> (output)</samp></dt> <dd> Force audio tag/fourcc. This is an alias for <code class=”code”>-tag:a</code>. </dd> <dt><samp class=”option”>-ch_layout[:<var class=”var”>stream_specifier</var>] <var class=”var”>layout</var> (input/output,per-stream)</samp></dt> <dd> Alias for <code class=”code”>-channel_layout</code>. </dd> <dt><samp class=”option”>-channel_layout[:<var class=”var”>stream_specifier</var>] <var class=”var”>layout</var> (input/output,per-stream)</samp></dt> <dd> Set the audio channel layout. For output streams it is set by default to the input channel layout. For input streams it overrides the channel layout of the input. Not all decoders respect the overridden channel layout. This option also sets the channel layout for audio grabbing devices and raw demuxers and is mapped to the corresponding demuxer option. </dd> <dt><samp class=”option”>-guess_layout_max <var class=”var”>channels</var> (input,per-stream)</samp></dt> <dd> If some input channel layout is not known, try to guess only if it corresponds to at most the specified number of channels. For example, 2 tells to <code class=”command”>ffmpeg</code> to recognize 1 channel as mono and 2 channels as stereo but not 6 channels as 5.1. The default is to always try to guess. Use 0 to disable all guessing. Using the <code class=”code”>-channel_layout</code> option to explicitly specify an input layout also disables guessing. </dd> </dl>

 <h3 class=”section”>5.9 Subtitle options# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-scodec <var class=”var”>codec</var> (input/output)</samp></dt> <dd> Set the subtitle codec. This is an alias for <code class=”code”>-codec:s</code>. </dd> <dt><samp class=”option”>-sn (input/output)</samp></dt> <dd> As an input option, blocks all subtitle streams of a file from being filtered or being automatically selected or mapped for any output. See <code class=”code”>-discard</code> option to disable streams individually. As an output option, disables subtitle recording i.e. automatic selection or mapping of any subtitle stream. For full manual control see the <code class=”code”>-map</code> option. </dd> </dl>

 <h3 class=”section”>5.10 Advanced Subtitle options# TOC</h3>

<dl class="table"> <dt><samp class="option">-fix_sub_duration</samp></dt> <dd> Fix subtitles durations. For each subtitle, wait for the next packet in the same stream and adjust the duration of the first to avoid overlap. This is necessary with some subtitles codecs, especially DVB subtitles, because the duration in the original packet is only a rough estimate and the end is actually marked by an empty subtitle frame. Failing to use this option when necessary can result in exaggerated durations or muxing failures due to non-monotonic timestamps. Note that this option will delay the output of all data until the next subtitle packet is decoded: it may increase memory consumption and latency a lot. </dd> <dt><samp class="option">-canvas_size <var class="var">size</var></samp></dt> <dd> Set the size of the canvas used to render subtitles. </dd> </dl>

 <h3 class="section">5.11 Advanced options# TOC</h3>

<dl class="table"> <dt><samp class="option">-map [-]<var class="var">input_file_id</var>[:<var class="var">stream_specifier</var>][:<var class="var">view_specifier</var>][:?] | <var class="var">[linklabel]</var> (output)</samp></dt> <dd> Create one or more streams in the output file. This option has two forms for specifying the data source(s): the first selects one or more streams from some input file (specified with <code class="code">-i</code>), the second takes an output from some complex filtergraph (specified with <code class="code">-filter_complex</code>). In the first form, an output stream is created for every stream from the input file with the index <var class="var">input_file_id</var>. If <var class="var">stream_specifier</var> is given, only those streams that match the specifier are used (see the Stream specifiers section for the <var class="var">stream_specifier</var> syntax). A <code class="code">-</code> character before the stream identifier creates a "negative" mapping. It disables matching streams from already created mappings. An optional <var class="var">view_specifier</var> may be given after the stream specifier, which for multiview video specifies the view to be used. The view specifier may have one of the following formats: <dl class="table"> <dt><samp class="option">view:<var class="var">view_id</var></samp></dt> <dd> select a view by its ID; <var class="var">view_id</var> may be set to ’all’ to use all the views interleaved into one stream; </dd> <dt><samp class="option">vidx:<var class="var">view_idx</var></samp></dt> <dd> select a view by its index; i.e. 0 is the base view, 1 is the first non-base view, etc. </dd> <dt><samp class="option">vpos:<var class="var">position</var></samp></dt> <dd> select a view by its display position; <var class="var">position</var> may be <code class="code">left</code> or <code class="code">right</code> </dd>

</dl> The default for transcoding is to only use the base view, i.e. the equivalent of <code class="code">vidx:0</code>. For streamcopy, view specifiers are not supported and all views are always copied. A trailing <code class="code">?</code> after the stream index will allow the map to be optional: if the map matches no streams the map will be ignored instead of failing. Note the map will still fail if an invalid input file index is used; such as if the map refers to a non-existent input. An alternative <var class="var">[linklabel]</var> form will map outputs from complex filter graphs (see the <samp class="option">-filter_complex</samp> option) to the output file. <var class="var">linklabel</var> must correspond to a defined output link label in the graph. This option may be specified multiple times, each adding more streams to the output file. Any given input stream may also be mapped any number of times as a source for different output streams, e.g. in order to use different encoding options and/or filters. The streams are created in the output in the same order in which the <code class="code">-map</code> options are given on the commandline. Using this option disables the default mappings for this output file. Examples: <dl class="table"> <dt>map everything</dt> <dd> To map ALL streams from the first input file to output <pre class="example-preformatted">ffmpeg -i INPUT -map 0 output </pre>

</dd> <dt>select specific stream</dt> <dd> If you have two audio streams in the first input file, these streams are identified by <var class="var">0:0</var> and <var class="var">0:1</var>. You can use <code class="code">-map</code> to select which streams to place in an output file. For example: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:1 out.wav </pre> will map the second input stream in <samp class="file">INPUT</samp> to the (single) output stream in <samp class="file">out.wav</samp>. </dd> <dt>create multiple streams</dt> <dd> To select the stream with index 2 from input file <samp class="file">a.mov</samp> (specified by the identifier <var class="var">0:2</var>), and stream with index 6 from input <samp class="file">b.mov</samp> (specified by the identifier <var class="var">1:6</var>), and copy them to the output file <samp class="file">out.mov</samp>: <pre class="example-preformatted">ffmpeg -i a.mov -i b.mov -c copy -map 0:2 -map 1:6 out.mov </pre>

</dd> <dt>create multiple streams 2</dt> <dd> To select all video and the third audio stream from an input file: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:v -map 0:a:2 OUTPUT </pre>

</dd> <dt>negative map</dt> <dd> To map all the streams except the second audio, use negative mappings <pre class="example-preformatted">ffmpeg -i INPUT

-map 0 -map -0:a:1 OUTPUT </pre>

</dd> <dt>optional map</dt> <dd> To map the video and audio streams from the first input, and using the trailing <code class="code">?</code>, ignore the audio mapping if no audio streams exist in the first input: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:v -map 0:a? OUTPUT </pre>

</dd> <dt>map by language</dt> <dd> To pick the English audio stream: <pre class="example-preformatted">ffmpeg -i INPUT -map 0:m:language:eng OUTPUT </pre>

</dd> </dl>

</dd> <dt><samp class="option">-ignore_unknown</samp></dt> <dd> Ignore input streams with unknown type instead of failing if copying such streams is attempted. </dd> <dt><samp class="option">-copy_unknown</samp></dt> <dd> Allow input streams with unknown type to be copied instead of failing if copying such streams is attempted. </dd> <dt><samp class="option">-map_metadata[:<var class="var">metadata_spec_out</var>] <var class="var">infile</var>[:<var class="var">metadata_spec_in</var>] (output,per-metadata)</samp></dt> <dd> Set metadata information of the next output file from <var class="var">infile</var>. Note that those are file indices (zero-based), not filenames. Optional <var class="var">metadata_spec_in/out</var> parameters specify, which metadata to copy. A metadata specifier can have the following forms: <dl class="table"> <dt><samp class="option"><var class="var">g</var></samp></dt> <dd> global metadata, i.e. metadata that applies to the whole file </dd> <dt><samp class="option"><var class="var">s</var>[:<var class="var">stream_spec</var>]</samp></dt> <dd> per-stream metadata. <var class="var">stream_spec</var> is a stream specifier as described in the Stream specifiers chapter. In an input metadata specifier, the first matching stream is copied from. In an output metadata specifier, all matching streams are copied to. </dd> <dt><samp class="option"><var class="var">c</var>:<var class="var">chapter_index</var></samp></dt> <dd> per-chapter metadata. <var class="var">chapter_index</var> is the zero-based chapter index. </dd> <dt><samp class="option"><var class="var">p</var>:<var class="var">program_index</var></samp></dt> <dd> per-program metadata. <var class="var">program_index</var> is the zero-based program index. </dd> </dl> If metadata specifier is omitted, it defaults to global. By default, global metadata is copied from the first input file, per-stream and per-chapter metadata is copied along with streams/chapters. These default mappings are disabled by creating any mapping of the relevant type. A negative file index can be

used to create a dummy mapping that just disables automatic copying. For example to copy metadata from the first stream of the input file to global metadata of the output file: <pre class="example-preformatted">ffmpeg -i in.ogg -map_metadata 0:s:0 out.mp3 </pre>

To do the reverse, i.e. copy global metadata to all audio streams: <pre class="example-preformatted">ffmpeg -i in.mkv -map_metadata:s:a 0:g out.mkv </pre> Note that simple <code class="code">0</code> would work as well in this example, since global metadata is assumed by default. </dd> <dt><samp class="option">-map_chapters <var class="var">input_file_index</var> (output)</samp></dt> <dd> Copy chapters from input file with index <var class="var">input_file_index</var> to the next output file. If no chapter mapping is specified, then chapters are copied from the first input file with at least one chapter. Use a negative file index to disable any chapter copying. </dd> <dt><samp class="option">-benchmark (global)</samp></dt> <dd> Show benchmarking information at the end of an encode. Shows real, system and user time used and maximum memory consumption. Maximum memory consumption is not supported on all systems, it will usually display as 0 if not supported. </dd> <dt><samp class="option">-benchmark_all (global)</samp></dt> <dd> Show benchmarking information during the encode. Shows real, system and user time used in various steps (audio/video encode/decode). </dd> <dt><samp class="option">-timelimit <var class="var">duration</var> (global)</samp></dt> <dd> Exit after ffmpeg has been running for <var class="var">duration</var> seconds in CPU user time. </dd> <dt><samp class="option">-dump (global)</samp></dt> <dd> Dump each input packet to stderr. </dd> <dt><samp class="option">-hex (global)</samp></dt> <dd> When dumping packets, also dump the payload. </dd> <dt><samp class="option">-readrate <var class="var">speed</var> (input)</samp></dt> <dd> Limit input read speed. Its value is a floating-point positive number which represents the maximum duration of media, in seconds, that should be ingested in one second of wallclock time. Default value is zero and represents no imposed limitation on speed of ingestion. Value <code class="code">1</code> represents real-time speed and is equivalent to <code class="code">-re</code>. Mainly used to simulate a capture device or live input stream (e.g. when reading from a file). Should not be used with a low value when input is an actual capture device or live stream as it may cause packet loss. It is useful for when flow speed of output packets is important, such as live streaming. </dd> <dt><samp class="option">-re (input)</samp></dt> <dd> Read input at native frame rate. This is equivalent to set-

ting <code class="code">-readrate 1</code>. </dd> <dt><samp class="option">-readrate_initial_burst <var class="var">seconds</var></samp></dt> <dd> Set an initial read burst time, in seconds, after which <samp class="option">-re/-readrate</samp> will be enforced. </dd> <dt><samp class="option">-readrate_catchup <var class="var">speed</var> (input)</samp></dt> <dd> If either the input or output is blocked leading to actual read speed falling behind the specified readrate, then this rate takes effect till the input catches up with the specified readrate. Must not be lower than the primary readrate. </dd> <dt><samp class="option">-fps_mode[:<var class="var">stream_specifier</var>] <var class="var">parameter</var> (output,per-stream)</samp></dt> <dd> Set video sync method / framerate mode. <dl class="table"> <dt><samp class="option">passthrough</samp></dt> <dd> Each frame is passed with its timestamp from the demuxer to the muxer. </dd> <dt><samp class="option">cfr</samp></dt> <dd> Frames will be duplicated and dropped to achieve exactly the requested constant frame rate. </dd> <dt><samp class="option">vfr</samp></dt> <dd> Frames are passed through with their timestamp or dropped so as to prevent 2 frames from having the same timestamp. </dd> <dt><samp class="option">auto</samp></dt> <dd> Chooses between cfr and vfr depending on muxer capabilities. This is the default method. </dd> </dl>

Note that the timestamps may be further modified by the muxer, after this. For example, in the case that the format option <samp class="option">avoid_negative_ts</samp> is enabled. With -map you can select from which stream the timestamps should be taken. You can leave either video or audio unchanged and sync the remaining stream(s) to the unchanged one. </dd> <dt><samp class="option">-frame_drop_threshold <var class="var">parameter</var></samp></dt> <dd> Frame drop threshold, which specifies how much behind video frames can be before they are dropped. In frame rate units, so 1.0 is one frame. The default is -1.1. One possible usecase is to avoid framedrops in case of noisy timestamps or to increase frame drop precision in case of exact timestamps. </dd> <dt><samp class="option">-apad <var class="var">parameters</var> (output,per-stream)</samp></dt> <dd> Pad the output audio stream(s). This is the same as applying <code class="code">-af apad</code>. Argument is a string of filter parameters composed the same as with the <code class="code">apad</code> filter. <code class="code">-shortest</code> must be set for this output for the option to take effect. </dd> <dt><samp class="option">-copyts</samp></dt> <dd> Do not process input timestamps, but keep their values without trying to sanitize them. In particular, do not remove the initial start time offset value. Note that, depending on the <samp

class=”option”>fps_mode</samp> option or on specific muxer processing (e.g. in case the format option <samp class=”option”>avoid_negative_ts</samp> is enabled) the output timestamps may mismatch with the input timestamps even when this option is selected. </dd> <dt><samp class=”option”>-start_at_zero</samp></dt> <dd> When used with <samp class=”option”>copyts</samp>, shift input timestamps so they start at zero. This means that using e.g. <code class=”code”>-ss 50</code> will make output timestamps start at 50 seconds, regardless of what timestamp the input file started at. </dd> <dt><samp class=”option”>-copytb <var class=”var”>mode</var></samp></dt> <dd> Specify how to set the encoder timebase when stream copying. <var class=”var”>mode</var> is an integer numeric value, and can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>1</samp></dt> <dd> Use the demuxer timebase. The time base is copied to the output encoder from the corresponding input demuxer. This is sometimes required to avoid non monotonically increasing timestamps when copying video streams with variable frame rate. </dd> <dt><samp class=”option”>0</samp></dt> <dd> Use the decoder timebase. The time base is copied to the output encoder from the corresponding input decoder. </dd> <dt><samp class=”option”>-1</samp></dt> <dd> Try to make the choice automatically, in order to generate a sane output. </dd> </dl>

Default value is -1. </dd> <dt><samp class=”option”>-enc_time_base[:<var class=”var”>stream_specifier</var>] <var class=”var”>timebase</var> (output,per-stream)</samp></dt> <dd> Set the encoder timebase. <var class=”var”>timebase</var> can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>0</samp></dt> <dd> Assign a default value according to the media type. For video - use 1/framerate, for audio - use 1/samplerate. </dd> <dt><samp class=”option”>demux</samp></dt> <dd> Use the timebase from the demuxer. </dd> <dt><samp class=”option”>filter</samp></dt> <dd> Use the timebase from the filtergraph. </dd> <dt><samp class=”option”>a positive number</samp></dt> <dd> Use the provided number as the timebase. This field can be provided as a ratio of two integers (e.g. 1:24, 1:48000) or as a decimal number (e.g. 0.04166, 2.0833e-5) </dd> </dl>

Default value is 0. </dd> <dt><samp class=”option”>-bitexact (input/output)</samp></dt> <dd> Enable bitexact mode for (de)muxer and (de/en)coder </dd> <dt><samp class=”option”>-shortest (output)</samp></dt> <dd> Finish encoding when the shortest output stream ends. Note that this option may require buffering frames, which introduces extra latency. The maximum amount of this latency may be controlled with the <code class=”code”>-shortest_buf_duration</code> option. </dd> <dt><samp class=”option”>-shortest_buf_duration <var

class=”var”>duration</var> (output)</samp></dt> <dd> The <code class=”code”>-shortest</code> option may require buffering potentially large amounts of data when at least one of the streams is "sparse" (i.e. has large gaps between frames – this is typically the case for subtitles). This option controls the maximum duration of buffered frames in seconds. Larger values may allow the <code class=”code”>-shortest</code> option to produce more accurate results, but increase memory use and latency. The default value is 10 seconds. </dd> <dt><samp class=”option”>-dts_delta_threshold <var class=”var”>threshold</var></samp></dt> <dd> Timestamp discontinuity delta threshold, expressed as a decimal number of seconds. The timestamp discontinuity correction enabled by this option is only applied to input formats accepting timestamp discontinuity (for which the <code class=”code”>AVFMT_TS_DISCONT</code> flag is enabled), e.g. MPEG-TS and HLS, and is automatically disabled when employing the <code class=”code”>-copyts</code> option (unless wrapping is detected). If a timestamp discontinuity is detected whose absolute value is greater than <var class=”var”>threshold</var>, ffmpeg will remove the discontinuity by decreasing/increasing the current DTS and PTS by the corresponding delta value. The default value is 10. </dd> <dt><samp class=”option”>-dts_error_threshold <var class=”var”>threshold</var></samp></dt> <dd> Timestamp error delta threshold, expressed as a decimal number of seconds. The timestamp correction enabled by this option is only applied to input formats not accepting timestamp discontinuity (for which the <code class=”code”>AVFMT_TS_DISCONT</code> flag is not enabled). If a timestamp discontinuity is detected whose absolute value is greater than <var class=”var”>threshold</var>, ffmpeg will drop the PTS/DTS timestamp value. The default value is <code class=”code”>3600*30</code> (30 hours), which is arbitrarily picked and quite conservative. </dd> <dt><samp class=”option”>-muxdelay <var class=”var”>seconds</var> (output)</samp></dt> <dd> Set the maximum demux-decode delay. </dd> <dt><samp class=”option”>-muxpreload <var class=”var”>seconds</var> (output)</samp></dt> <dd> Set the initial demux-decode delay. </dd> <dt><samp class=”option”>-streamid <var class=”var”>output-stream-index</var>:<var class=”var”>new-value</var> (output)</samp></dt> <dd> Assign a new stream-id value to an output stream. This option should be specified prior to the output filename to which it applies. For the situation where multiple output files exist, a streamid may be reassigned to a different value. For example, to set the stream 0 PID to 33 and the stream 1 PID to 36 for an output mpegts file: <pre class=”example-preformatted”>ffmpeg -i inurl -streamid 0:33 -streamid 1:36 out.ts </pre>

</dd> <dt><samp class=”option”>-bsf[:<var class=”var”>stream_specifier</var>]

<var class=”var”>bitstream_filters</var> (input/output,per-stream)</samp></dt> <dd> Apply bitstream filters to matching streams. The filters are applied to each packet as it is received from the demuxer (when used as an input option) or before it is sent to the muxer (when used as an output option). <var class=”var”>bitstream_filters</var> is a comma-separated list of bitstream filter specifications, each of the form <pre class=”example-preformatted”><var class=”var”>filter</var>[=<var class=”var”>optname0</var>=<var class=”var”>optval0</var>:<var class=”var”>optname1</var>=<var class=”var”>optval1</var>:...] </pre> Any of the ’,=:’ characters that are to be a part of an option value need to be escaped with a backslash. Use the <code class=”code”>-bsfs</code> option to get the list of bitstream filters. E.g. <pre class=”example-preformatted”>ffmpeg -bsf:v h264_mp4toannexb -i h264.mp4 -c:v copy -an out.h264 </pre> applies the <code class=”code”>h264_mp4toannexb</code> bitstream filter (which converts MP4-encapsulated H.264 stream to Annex B) to the input video stream. On the other hand, <pre class=”example-preformatted”>ffmpeg -i file.mov -an -vn -bsf:s mov2textsub -c:s copy -f rawvideo sub.txt </pre> applies the <code class=”code”>mov2textsub</code> bitstream filter (which extracts text from MOV subtitles) to the output subtitle stream. Note, however, that since both examples use <code class=”code”>-c copy</code>, it matters little whether the filters are applied on input or output - that would change if transcoding was happening. </dd> <dt><samp class=”option”>-tag[:<var class=”var”>stream_specifier</var>] <var class=”var”>codec_tag</var> (input/output,per-stream)</samp></dt> <dd> Force a tag/fourcc for matching streams. </dd> <dt><samp class=”option”>-timecode <var class=”var”>hh</var>:<var class=”var”>mm</var>:<var class=”var”>ss</var>SEP<var class=”var”>ff</var></samp></dt> <dd> Specify Timecode for writing. <var class=”var”>SEP</var> is ’:’ for non drop timecode and ’;’ (or ’.’) for drop. <pre class=”example-preformatted”>ffmpeg -i input.mpg -timecode 01:02:03.04 -r 30000/1001 -s ntsc output.mpg </pre>

</dd> <dt><samp class=”option”>-filter_complex <var class=”var”>filtergraph</var> (global)</samp></dt> <dd> Define a complex filtergraph, i.e. one with arbitrary number of inputs and/or outputs. For simple graphs – those with one input and one output of the same type – see the <samp class=”option”>-filter</samp> options. <var class=”var”>filtergraph</var> is a description of the filtergraph, as described in the “Filtergraph syntax” section of the ffmpeg-filters

manual. This option may be specified multiple times - each use creates a new complex filtergraph. Inputs to a complex filtergraph may come from different source types, distinguished by the format of the corresponding link label: <ul class="itemize mark-bullet"> To connect an input stream, use <code class="code">[file_index:stream_specifier]</code> (i.e. the same syntax as <samp class="option">-map</samp>). If <var class="var">stream_specifier</var> matches multiple streams, the first one will be used. For multiview video, the stream specifier may be followed by the view specifier, see documentation for the <samp class="option">-map</samp> option for its syntax.

To connect a loopback decoder use [dec:<var class="var">dec_idx</var>], where <var class="var">dec_idx</var> is the index of the loopback decoder to be connected to given input. For multiview video, the decoder index may be followed by the view specifier, see documentation for the <samp class="option">-map</samp> option for its syntax.

To connect an output from another complex filtergraph, use its link label. E.g the following example:

 <pre class="example-preformatted">ffmpeg -i input.mkv \ -filter_complex '[0:v]scale=size=hd1080,split=outputs=2[for_enc][orig_scaled]' \ -c:v libx264 -map '[for_enc]' output.mkv \ -dec 0:0 \ -filter_complex '[dec:0][orig_scaled]hstack[stacked]' \ -map '[stacked]' -c:v ffv1 comparison.mkv </pre>

reads an input video and <ul class="itemize mark-bullet"> (line 2) uses a complex filtergraph with one input and two outputs to scale the video to 1920x1080 and duplicate the result to both outputs;

(line 3) encodes one scaled output with <code class="code">libx264</code> and writes the result to <samp class="file">output.mkv</samp>;

(line 4) decodes this encoded stream with a loopback decoder;

(line 5) places the output of the loopback decoder (i.e. the <code class="code">libx264</code>-encoded video) side by side with the scaled original input;

(line 6) combined video is then losslessly encoded and written into <samp class="file">comparison.mkv</samp>.

Note that the two filtergraphs cannot be combined into one, because then there would be a cycle in the transcoding pipeline (filtergraph output goes to encoding, from there to decoding, then back to the same graph), and such cycles are not allowed.

An unlabeled input will be connected to the first unused input stream of the matching type. Output link labels are referred to with

<samp class="option">-map</samp>. Unlabeled outputs are added to the first output file. Note that with this option it is possible to use only lavfi sources without normal input files. For example, to overlay an image over video <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex '[0:v][1:v]overlay[out]' -map '[out]' out.mkv </pre> Here <code class="code">[0:v]</code> refers to the first video stream in the first input file, which is linked to the first (main) input of the overlay filter. Similarly the first video stream in the second input is linked to the second (overlay) input of overlay. Assuming there is only one video stream in each input file, we can omit input labels, so the above is equivalent to <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex 'overlay[out]' -map '[out]' out.mkv </pre>

Furthermore we can omit the output label and the single output from the filter graph will be added to the output file automatically, so we can simply write <pre class="example-preformatted">ffmpeg -i video.mkv -i image.png -filter_complex 'overlay' out.mkv </pre>

As a special exception, you can use a bitmap subtitle stream as input: it will be converted into a video with the same size as the largest video in the file, or 720x576 if no video is present. Note that this is an experimental and temporary solution. It will be removed once libavfilter has proper support for subtitles. For example, to hardcode subtitles on top of a DVB-T recording stored in MPEG-TS format, delaying the subtitles by 1 second: <pre class="example-preformatted">ffmpeg -i input.ts -filter_complex \ '[#0x2ef] setpts=PTS+1/TB [sub] ; [#0x2d0] [sub] overlay' \ -sn -map '#0x2dc' output.mkv </pre> (0x2d0, 0x2dc and 0x2ef are the MPEG-TS PIDs of respectively the video, audio and subtitles streams; 0:0, 0:3 and 0:7 would have worked too) To generate 5 seconds of pure red video using lavfi <code class="code">color</code> source: <pre class="example-preformatted">ffmpeg -filter_complex 'color=c=red' -t 5 out.mkv </pre>

</dd> <dt><samp class="option">-filter_complex_threads <var class="var">nb_threads</var> (global)</samp></dt> <dd> Defines how many threads are used to process a filter_complex graph. Similar to filter_threads but used for <code class="code">-filter_complex</code> graphs only. The default is the number of available CPUs. </dd> <dt><samp class="option">-lavfi <var class="var">filtergraph</var> (global)</samp></dt> <dd> Define a complex filtergraph, i.e. one with arbitrary number of inputs and/or outputs. Equivalent to <samp class="option">-filter_complex</samp>. </dd> <dt><samp class="option">-accurate_seek (input)</samp></dt> <dd> This option enables or disables accurate seeking in input files with the <samp class="option">-ss</samp> option. It is en-

abled by default, so seeking is accurate when transcoding. Use <samp class=”option”>-noaccurate_seek</samp> to disable it, which may be useful e.g. when copying some streams and transcoding the others. </dd> <dt><samp class=”option”>-seek_timestamp (input)</samp></dt> <dd> This option enables or disables seeking by timestamp in input files with the <samp class=”option”>-ss</samp> option. It is disabled by default. If enabled, the argument to the <samp class=”option”>-ss</samp> option is considered an actual timestamp, and is not offset by the start time of the file. This matters only for files which do not start from timestamp 0, such as transport streams. </dd> <dt><samp class=”option”>-thread_queue_size <var class=”var”>size</var> (output)</samp></dt> <dd> This option specifies the maximum number of packets per stream that may be queued to each muxing thread. </dd> <dt><samp class=”option”>-sdp_file <var class=”var”>file</var> (global)</samp></dt> <dd> Print sdp information for an output stream to <var class=”var”>file</var>. This allows dumping sdp information when at least one output isn’t an rtp stream. (Requires at least one of the output formats to be rtp). </dd> <dt><samp class=”option”>-discard (input)</samp></dt> <dd> Allows discarding specific streams or frames from streams. Any input stream can be fully discarded, using value <code class=”code”>all</code> whereas selective discarding of frames from a stream occurs at the demuxer and is not supported by all demuxers. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Discard no frame. </dd> <dt><samp class=”option”>default</samp></dt> <dd> Default, which discards no frames. </dd> <dt><samp class=”option”>noref</samp></dt> <dd> Discard all non-reference frames. </dd> <dt><samp class=”option”>bidir</samp></dt> <dd> Discard all bidirectional frames. </dd> <dt><samp class=”option”>nokey</samp></dt> <dd> Discard all frames excepts keyframes. </dd> <dt><samp class=”option”>all</samp></dt> <dd> Discard all frames. </dd> </dl>

</dd> <dt><samp class=”option”>-abort_on <var class=”var”>flags</var> (global)</samp></dt> <dd> Stop and abort on various conditions. The following flags are available: <dl class=”table”> <dt><samp class=”option”>empty_output</samp></dt> <dd> No packets were passed to the muxer, the output is empty. </dd> <dt><samp class=”option”>empty_output_stream</samp></dt> <dd> No packets were passed to the muxer in some of the output streams. </dd> </dl>

</dd>

<dt><samp class=”option”>-max_error_rate (global)</samp></dt>

<dd> Set fraction of decoding frame failures across all inputs which when

crossed

ffmpeg will return exit code 69. Crossing this threshold does not terminate

processing. Range is a floating-point number between 0 to 1. Default is 2/3.

</dd>

<dt><samp class="option">-xerror (global)</samp></dt>

<dd> Stop and exit on error

</dd>

<dt><samp class="option">-max_muxing_queue_size <var class="var">packets</var> (output,per-stream)</samp></dt>

<dd> When transcoding audio and/or video streams, ffmpeg will not begin writing into

the output until it has one packet for each such stream. While waiting for that

to happen, packets for other streams are buffered. This option sets the size of

this buffer, in packets, for the matching output stream.

The default value of this option should be high enough for most uses, so only

touch this option if you are sure that you need it.

</dd>

<dt><samp class="option">-muxing_queue_data_threshold <var class="var">bytes</var> (output,per-stream)</samp></dt>

<dd> This is a minimum threshold until which the muxing queue size is not taken into

account. Defaults to 50 megabytes per stream, and is based on the overall size

of packets passed to the muxer.

</dd>

<dt><samp class="option">-auto_conversion_filters (global)</samp></dt>

<dd> Enable automatically inserting format conversion filters in all filter

graphs, including those defined by <samp class="option">-vf</samp>, <samp class="option">-af</samp>,

<samp class="option">-filter_complex</samp> and <samp class="option">-lavfi</samp>. If filter format negotiation

requires a conversion, the initialization of the filters will fail.

Conversions can still be performed by inserting the relevant conversion

filter (scale, aresample) in the graph.

On by default, to explicitly disable it you need to specify

<code class="code">-noauto_conversion_filters</code>.

</dd>

<dt><samp class="option">-bits_per_raw_sample[:<var class="var">stream_specifier</var>] <var class="var">value</var> (output,per-stream)</samp></dt>

<dd> Declare the number of bits per raw sample in the given output stream to be

<var class="var">value</var>. Note that this option sets the information provided to the

encoder/muxer, it does not change the stream to conform to this value. Setting

values that do not match the stream properties may result in encoding failures

or invalid output files.

</dd>

<dt><samp class="option">-stats_enc_pre[:<var class="var">stream_specifier</var>] <var class="var">path</var> (output,per-stream)</samp></dt>

<dt><samp class="option">-stats_enc_post[:<var class="var">stream_specifier</var>] <var class="var">path</var> (output,per-stream)</samp></dt>

<dt><samp class="option">-stats_mux_pre[:<var class="var">stream_specifier</var>] <var class="var">path</var> (output,per-stream)</samp></dt>

<dd> Write per-frame encoding information about the matching streams into the file

given by <var class="var">path</var>.

<samp class="option">-stats_enc_pre</samp> writes information about raw video or audio frames right

before they are sent for encoding, while <samp class=”option”>-stats_enc_post</samp> writes

information about encoded packets as they are received from the encoder.

<samp class=”option”>-stats_mux_pre</samp> writes information about packets just as they are about to

be sent to the muxer. Every frame or packet produces one line in the specified

file. The format of this line is controlled by <samp class=”option”>-stats_enc_pre_fmt</samp> /

<samp class=”option”>-stats_enc_post_fmt</samp> / <samp class=”option”>-stats_mux_pre_fmt</samp>.

When stats for multiple streams are written into a single file, the lines

corresponding to different streams will be interleaved. The precise order of

this interleaving is not specified and not guaranteed to remain stable between

different invocations of the program, even with the same options.

</dd>

<dt><samp class=”option”>-stats_enc_pre_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_enc_post_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dt><samp class=”option”>-stats_mux_pre_fmt[:<var class=”var”>stream_specifier</var>] <var class=”var”>format_spec</var> (output,per-stream)</samp></dt>

<dd> Specify the format for the lines written with <samp class=”option”>-stats_enc_pre</samp> /

<samp class=”option”>-stats_enc_post</samp> / <samp class=”option”>-stats_mux_pre</samp>.

<var class=”var”>format_spec</var> is a string that may contain directives of the form

<var class=”var”>{fmt}</var>. <var class=”var”>format_spec</var> is backslash-escaped — use \{, \}, and \\

to write a literal {, }, or \, respectively, into the output.

The directives given with <var class="var">fmt</var> may be one of the following:

<dl class="table">

<dt><samp class="option">fidx</samp></dt>

<dd> Index of the output file.

</dd>

<dt><samp class="option">sidx</samp></dt>

<dd> Index of the output stream in the file.

</dd>

<dt><samp class="option">n</samp></dt>

<dd> Frame number. Pre-encoding: number of frames sent to the encoder so far.

Post-encoding: number of packets received from the encoder so far.

Muxing: number of packets submitted to the muxer for this stream so far.

</dd>

<dt><samp class="option">ni</samp></dt>

<dd> Input frame number. Index of the input frame (i.e. output by a decoder) that

corresponds to this output frame or packet. -1 if unavailable.

</dd>

<dt><samp class="option">tb</samp></dt>

<dd> Timebase in which this frame/packet’s timestamps are expressed, as a rational

number <var class="var">num/den</var>. Note that encoder and muxer may use different timebases.

</dd>

<dt><samp class="option">tbi</samp></dt>

<dd> Timebase for <var class="var">ptsi</var>, as a rational number <var class="var">num/den</var>. Available when

<var class="var">ptsi</var> is available, <var class="var">0/1</var> otherwise.

</dd>

<dt><samp class="option">pts</samp></dt>

<dd> Presentation timestamp of the frame or packet, as an integer. Should be

multiplied by the timebase to compute presentation time.

</dd>

<dt><samp class="option">ptsi</samp></dt>

<dd> Presentation timestamp of the input frame (see <var class="var">ni</var>), as an integer. Should

be multiplied by <var class="var">tbi</var> to compute presentation time. Printed as

(2^63 - 1 = 9223372036854775807) when not available.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> Presentation time of the frame or packet, as a decimal number. Equal to

<var class="var">pts</var> multiplied by <var class="var">tb</var>.

</dd>

<dt><samp class="option">ti</samp></dt>

<dd> Presentation time of the input frame (see <var class="var">ni</var>), as a decimal number. Equal

to <var class="var">ptsi</var> multiplied by <var class="var">tbi</var>. Printed as inf when not available.

</dd>

<dt><samp class="option">dts (packet)</samp></dt>

<dd> Decoding timestamp of the packet, as an integer. Should be multiplied by the

timebase to compute presentation time.

</dd>

<dt><samp class="option">dt (packet)</samp></dt>

<dd> Decoding time of the frame or packet, as a decimal number. Equal to

<var class="var">dts</var> multiplied by <var class="var">tb</var>.

</dd>

<dt><samp class="option">sn (frame,audio)</samp></dt>

<dd> Number of audio samples sent to the encoder so far.

</dd>

<dt><samp class="option">samp (frame,audio)</samp></dt>

<dd> Number of audio samples in the frame.

</dd>

<dt><samp class="option">size (packet)</samp></dt>

<dd> Size of the encoded packet in bytes.

</dd>

<dt><samp class="option">br (packet)</samp></dt>

<dd> Current bitrate in bits per second.

</dd>

<dt><samp class="option">abr (packet)</samp></dt>

<dd> Average bitrate for the whole stream so far, in bits per second, -1 if it cannot

be determined at this point.

</dd>

<dt><samp class="option">key (packet)</samp></dt>

<dd> Character ’K’ if the packet contains a keyframe, character ’N’ otherwise.

</dd>

</dl>

Directives tagged with packet may only be used with <samp class="option">-stats_enc_post_fmt</samp> and <samp class="option">-stats_mux_pre_fmt</samp>. Directives tagged with frame may only be used with <samp class="option">-stats_enc_pre_fmt</samp>. Directives tagged with audio may only be used with audio streams. The default format strings are: <dl class="table"> <dt><samp class="option">pre-encoding</samp></dt> <dd> {fidx} {sidx} {n} {t} </dd> <dt><samp class="option">post-encoding</samp></dt> <dd> {fidx} {sidx} {n} {t} </dd> </dl> In the future, new items may be added to the end of the default formatting strings. Users who depend on the format staying exactly the same, should prescribe it manually. Note that stats for different streams written into the same file may have different formats. </dd> </dl>

 <h3 class="section">5.12 Preset files# TOC</h3> A preset file contains a sequence of <var class="var">option</var>=<var class="var">value</var> pairs, one for each line, specifying a sequence of options which would be awkward to specify on the command line. Lines starting with the hash (’#’) character are ignored and are used to provide comments. Check the <samp class="file">presets</samp> directory in the FFmpeg source tree for examples. There are two types of preset files: ffpreset and avpreset files. <h4 class="subsection">5.12.1 ffpreset files# TOC</h4> ffpreset files are specified with the <code class="code">vpre</code>, <code class="code">apre</code>, <code class="code">spre</code>, and <code class="code">fpre</code> options. The <code class="code">fpre</code>

option takes the filename of the preset instead of a preset name as input and can be used for any kind of codec. For the <code class=”code”>vpre</code>, <code class=”code”>apre</code>, and <code class=”code”>spre</code> options, the options specified in a preset file are applied to the currently selected codec of the same type as the preset option. The argument passed to the <code class=”code”>vpre</code>, <code class=”code”>apre</code>, and <code class=”code”>spre</code> preset options identifies the preset file to use according to the following rules: First ffmpeg searches for a file named <var class=”var”>arg</var>.ffpreset in the directories <samp class=”file”>$FFMPEG_DATADIR</samp> (if set), and <samp class=”file”>$HOME/.ffmpeg</samp>, and in the datadir defined at configuration time (usually <samp class=”file”>PREFIX/share/ffmpeg</samp>) or in a <samp class=”file”>ffpresets</samp> folder along the executable on win32, in that order. For example, if the argument is <code class=”code”>libvpx-1080p</code>, it will search for the file <samp class=”file”>libvpx-1080p.ffpreset</samp>. If no such file is found, then ffmpeg will search for a file named <var class=”var”>codec_name</var>-<var class=”var”>arg</var>.ffpreset in the above-mentioned directories, where <var class=”var”>codec_name</var> is the name of the codec to which the preset file options will be applied. For example, if you select the video codec with <code class=”code”>-vcodec libvpx</code> and use <code class=”code”>-vpre 1080p</code>, then it will search for the file <samp class=”file”>libvpx-1080p.ffpreset</samp>. <h4 class=”subsection”>5.12.2 avpreset files# TOC</h4> avpreset files are specified with the <code class=”code”>pre</code> option. They work similar to ffpreset files, but they only allow encoder- specific options. Therefore, an <var class=”var”>option</var>=<var class=”var”>value</var> pair specifying an encoder cannot be used. When the <code class=”code”>pre</code> option is specified, ffmpeg will look for files with the suffix .avpreset in the directories <samp class=”file”>$AVCONV_DATADIR</samp> (if set), and <samp class=”file”>$HOME/.avconv</samp>, and in the datadir defined at configuration time (usually <samp class=”file”>PREFIX/share/ffmpeg</samp>), in that order. First ffmpeg searches for a file named <var class=”var”>codec_name</var>-<var class=”var”>arg</var>.avpreset in the above-mentioned directories, where <var class=”var”>codec_name</var> is the name of the codec to which the preset file options will be applied. For example, if you select the video codec with <code class=”code”>-vcodec libvpx</code> and use <code class=”code”>-pre 1080p</code>, then it will search for the file <samp class=”file”>libvpx-1080p.avpreset</samp>. If no such file is found, then ffmpeg will search for a file named <var class=”var”>arg</var>.avpreset in the same directories. <a name=”vstats-

file-format”> <h3 class=”section”>5.13 vstats file format# TOC</h3> The <code class=”code”>-vstats</code> and <code class=”code”>-vstats_file</code> options enable generation of a file containing statistics about the generated video outputs. The <code class=”code”>-vstats_version</code> option controls the format version of the generated file. With version <code class=”code”>1</code> the format is: <pre class=”example-preformatted”>frame= <var class=”var”>FRAME</var> q= <var class=”var”>FRAME_QUALITY</var> PSNR= <var class=”var”>PSNR</var> f_size= <var class=”var”>FRAME_SIZE</var> s_size= <var class=”var”>STREAM_SIZE</var>kB time= <var class=”var”>TIMESTAMP</var> br= <var class=”var”>BITRATE</var>kbits/s avg_br= <var class=”var”>AVERAGE_BITRATE</var>kbits/s </pre>

With version <code class=”code”>2</code> the format is: <pre class=”example-preformatted”>out= <var class=”var”>OUT_FILE_INDEX</var> st= <var class=”var”>OUT_FILE_STREAM_INDEX</var> frame= <var class=”var”>FRAME_NUMBER</var> q= <var class=”var”>FRAME_QUALITY</var>f PSNR= <var class=”var”>PSNR</var> f_size= <var class=”var”>FRAME_SIZE</var> s_size= <var class=”var”>STREAM_SIZE</var>kB time= <var class=”var”>TIMESTAMP</var> br= <var class=”var”>BITRATE</var>kbits/s avg_br= <var class=”var”>AVERAGE_BITRATE</var>k </pre>

The value corresponding to each key is described below: <dl class=”table”> <dt><samp class=”option”>avg_br</samp></dt> <dd> average bitrate expressed in Kbits/s </dd> <dt><samp class=”option”>br</samp></dt> <dd> bitrate expressed in Kbits/s </dd> <dt><samp class=”option”>frame</samp></dt> <dd> number of encoded frame </dd> <dt><samp class=”option”>out</samp></dt> <dd> out file index </dd> <dt><samp class=”option”>PSNR</samp></dt> <dd> Peak Signal to Noise Ratio </dd> <dt><samp class=”option”>q</samp></dt> <dd> quality of the frame </dd> <dt><samp class=”option”>f_size</samp></dt> <dd> encoded packet size expressed as number of bytes </dd> <dt><samp class=”option”>s_size</samp></dt> <dd> stream size expressed in KiB </dd> <dt><samp class=”option”>st</samp></dt> <dd> out file stream index </dd> <dt><samp class=”option”>time</samp></dt> <dd> time of the packet </dd> <dt><samp class=”option”>type</samp></dt> <dd> picture type </dd> </dl>

See also the -stats_enc options for an alternative way to show encoding statistics.

 <h2 class=”chapter”>6 Examples# TOC</h2>

 <h3 class="section">6.1 Video and Audio grabbing# TOC</h3>

If you specify the input format and device then ffmpeg can grab video and audio directly.   <pre class="example-preformatted">ffmpeg -f oss -i /dev/dsp -f video4linux2 -i /dev/video0 /tmp/out.mpg </pre>

Or with an ALSA audio source (mono input, card id 1) instead of OSS: <pre class="example-preformatted">ffmpeg -f alsa -ac 1 -i hw:1 -f video4linux2 -i /dev/video0 /tmp/out.mpg </pre>

Note that you must activate the right video source and channel before launching ffmpeg with any TV viewer such as xawtv by Gerd Knorr. You also have to set the audio recording levels correctly with a standard mixer. <h3 class="section">6.2 X11 grabbing# TOC</h3>

Grab the X11 display with ffmpeg via   <pre class="example-preformatted">ffmpeg -f x11grab -video_size cif -framerate 25 -i :0.0 /tmp/out.mpg </pre>

0.0 is display.screen number of your X11 server, same as the DISPLAY environment variable.   <pre class="example-preformatted">ffmpeg -f x11grab -video_size cif -framerate 25 -i :0.0+10,20 /tmp/out.mpg </pre>

0.0 is display.screen number of your X11 server, same as the DISPLAY environment variable. 10 is the x-offset and 20 the y-offset for the grabbing. <h3 class="section">6.3 Video and Audio file format conversion# TOC</h3>

Any supported file format and protocol can serve as input to ffmpeg: Examples: <ul class="itemize mark-bullet"> You can use YUV files as input:

 <pre class="example-preformatted">ffmpeg -i /tmp/test%d.Y /tmp/out.mpg </pre>

It will use the files: <pre class="example-preformatted">/tmp/test0.Y, /tmp/test0.U, /tmp/test0.V, /tmp/test1.Y,

/tmp/test1.U, /tmp/test1.V, etc... </pre>

The Y files use twice the resolution of the U and V files. They are raw files, without header. They can be generated by all decent video decoders. You must specify the size of the image with the <samp class="option">-s</samp> option if ffmpeg cannot guess it. You can input from a raw YUV420P file:

 <pre class="example-preformatted">ffmpeg -i /tmp/test.yuv /tmp/out.avi </pre>

test.yuv is a file containing raw YUV planar data. Each frame is composed of the Y plane followed by the U and V planes at half vertical and horizontal resolution. You can output to a raw YUV420P file:

 <pre class="example-preformatted">ffmpeg -i mydivx.avi hugefile.yuv </pre>

You can set several input files and output files:

 <pre class="example-preformatted">ffmpeg -i /tmp/a.wav -s 640x480 -i /tmp/a.yuv /tmp/a.mpg </pre>

Converts the audio file a.wav and the raw YUV video file a.yuv to MPEG file a.mpg. You can also do audio and video conversions at the same time:

 <pre class="example-preformatted">ffmpeg -i /tmp/a.wav -ar 22050 /tmp/a.mp2 </pre>

Converts a.wav to MPEG audio at 22050 Hz sample rate. You can encode to several formats at the same time and define a mapping from input stream to output streams:

 <pre class="example-preformatted">ffmpeg -i /tmp/a.wav -map 0:a -b:a 64k /tmp/a.mp2 -map 0:a -b:a 128k /tmp/b.mp2 </pre>

Converts a.wav to a.mp2 at 64 kbits and to b.mp2 at 128 kbits. ’-map file:index’ specifies which input stream is used for each output stream, in the order of the definition of output streams. You can transcode decrypted VOBs:

 <pre class="example-preformatted">ffmpeg -i snatch_1.vob -f avi -c:v mpeg4 -b:v 800k -g 300 -bf 2 -c:a libmp3lame -b:a 128k snatch.avi </pre>

This is a typical DVD ripping example; the input is a VOB file, the output an AVI file with MPEG-4 video and MP3 audio. Note that in this command we use B-frames so the MPEG-4 stream is DivX5 compatible, and GOP size is 300 which means one intra frame every 10 seconds for 29.97fps input video. Furthermore, the audio stream is MP3-encoded so you need to enable LAME

support by passing <code class=”code”>--enable-libmp3lame</code> to configure. The mapping is particularly useful for DVD transcoding to get the desired audio language. NOTE: To see the supported input formats, use <code class=”code”>ffmpeg -demuxers</code>. You can extract images from a video, or create a video from many images:

For extracting images from a video: <pre class=”example-preformatted”>ffmpeg -i foo.avi -r 1 -s WxH -f image2 foo-%03d.jpeg </pre>

This will extract one video frame per second from the video and will output them in files named <samp class=”file”>foo-001.jpeg</samp>, <samp class=”file”>foo-002.jpeg</samp>, etc. Images will be rescaled to fit the new WxH values. If you want to extract just a limited number of frames, you can use the above command in combination with the <code class=”code”>-frames:v</code> or <code class=”code”>-t</code> option, or in combination with -ss to start extracting from a certain point in time. For creating a video from many images: <pre class=”example-preformatted”>ffmpeg -f image2 -framerate 12 -i foo-%03d.jpeg -s WxH foo.avi </pre>

The syntax <code class=”code”>foo-%03d.jpeg</code> specifies to use a decimal number composed of three digits padded with zeroes to express the sequence number. It is the same syntax supported by the C printf function, but only formats accepting a normal integer are suitable. When importing an image sequence, -i also supports expanding shell-like wildcard patterns (globbing) internally, by selecting the image2-specific <code class=”code”>-pattern_type glob</code> option. For example, for creating a video from filenames matching the glob pattern <code class=”code”>foo-*.jpeg</code>: <pre class=”example-preformatted”>ffmpeg -f image2 -pattern_type glob -framerate 12 -i 'foo-*.jpeg' -s WxH foo.avi </pre>

You can put many streams of the same type in the output:

 <pre class=”example-preformatted”>ffmpeg -i test1.avi -i test2.avi -map 1:1 -map 1:0 -map 0:1 -map 0:0 -c copy -y test12.nut </pre>

The resulting output file <samp class=”file”>test12.nut</samp> will contain the first four streams from the input files in reverse order. To force CBR video output: <pre class=”example-preformatted”>ffmpeg -i myfile.avi -b 4000k -minrate 4000k -maxrate 4000k -bufsize 1835k out.m2v </pre>

The four options lmin, lmax, mblmin and mblmax use ’lambda’ units, but you may use the QP2LAMBDA constant to easily convert from ’q’ units: <pre class=”example-preformatted”>ffmpeg -i src.ext -lmax 21*QP2LAMBDA dst.ext </pre>

 <h2 class="chapter">7 See Also# TOC</h2>

ffmpeg-all, ffplay, ffprobe, ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class="chapter">8 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffplay-all.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> ffplay Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> ffplay Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark">

1 Synopsis

2 Description

3 Options

<ul class="toc-numbered-mark">

3.1 Stream specifiers

3.2 Generic options

3.3 AVOptions

3.4 Main options

3.5 Advanced options

3.6 While playing

4 Syntax

<ul class="toc-numbered-mark">

4.1 Quoting and escaping

<ul class="toc-numbered-mark">

4.1.1 Examples

4.2 Date

4.3 Time duration

<ul class="toc-numbered-mark">

4.3.1 Examples

4.4 Video size

4.5 Video rate

4.6 Ratio

4.7 Color

4.8 Channel Layout

5 Expression Evaluation

6 Codec Options

7 Decoders

8 Video Decoders

<ul class="toc-numbered-mark">

8.1 av1

<ul class="toc-numbered-mark">

8.1.1 Options

8.2 hevc

<ul class="toc-numbered-mark">

8.2.1 Options

8.3 rawvideo

<ul class="toc-numbered-mark">

8.3.1 Options

8.4 libastcenc

<ul class="toc-numbered-mark">

8.4.1 Options

8.5 libdav1d

<ul class="toc-numbered-mark">

8.5.1 Options

8.6 libdavs2

8.7 libuavs3d

<ul class="toc-numbered-mark">

8.7.1 Options

8.8 libxevd

<ul class="toc-numbered-mark">

8.8.1 Options

8.9 QSV Decoders

<ul class="toc-numbered-mark">

8.9.1 Common Options

8.9.2 HEVC Options

8.10 v210

<ul class="toc-numbered-mark">

8.10.1 Options

9 Audio Decoders

<ul class="toc-numbered-mark">

9.1 ac3

<ul class="toc-numbered-mark">

9.1.1 AC-3 Decoder Options

9.2 dca

<ul class="toc-numbered-mark">

9.2.1 DCA Decoder Options

9.3 flac

<ul class="toc-numbered-mark">

9.3.1 FLAC Decoder options

9.4 ffwavesynth

9.5 libgsm

9.6 libilbc

<ul class="toc-numbered-mark">

9.6.1 Options

9.7 libmpeghdec

9.8 libopencore-amrnb

9.9 libopencore-amrwb

9.10 libopus

9.11 pcm_dvda

10 Subtitles Decoders

<ul class="toc-numbered-mark">

10.1 libaribb24

<ul class="toc-numbered-mark">

10.1.1 libaribb24 Decoder Options

10.2 libaribcaption

<ul class="toc-numbered-mark">

10.2.1 libaribcaption Decoder Options

10.2.2 libaribcaption decoder usage examples

10.3 dvbsub

<ul class="toc-numbered-mark">

10.3.1 Options

10.4 dvdsub

<ul class="toc-numbered-mark">

10.4.1 Options

10.5 libzvbi-teletext

<ul class="toc-numbered-mark">

10.5.1 Options

11 Bitstream Filters

<ul class="toc-numbered-mark">

11.1 aac_adtstoasc

11.2 av1_metadata

11.3 chomp

11.4 dca_core

11.5 dovi_rpu

11.6 dovi_split

11.7 dump_extra

11.8 dv_error_marker

11.9 eac3_core

11.10 eia608_to_smpte436m

11.11 extract_extradata

11.12 filter_units

11.13 hapqa_extract

11.14 h264_metadata

11.15 h264_mp4toannexb

11.16 h264_redundant_pps

11.17 hevc_metadata

11.18 hevc_mp4toannexb

11.19 imxdump

11.20 jpegapp

11.21 mjpeg2jpeg

11.22 mjpegadump

11.23 mov2textsub

11.24 mpeg2_metadata

11.25 mpeg4_unpack_bframes

11.26 noise

<ul class="toc-numbered-mark">

11.26.1 Examples

11.27 null

11.28 pcm_rechunk

11.29 pgs_frame_merge

11.30 prores_metadata

11.31 remove_extra

11.32 setts

11.33 showinfo

11.34 side_data

11.35 smpte436m_to_eia608

11.36 text2movsub

11.37 trace_headers

11.38 trim

11.39 truehd_core

11.40 vp9_metadata

11.41 vp9_superframe

11.42 vp9_superframe_split

11.43 vp9_raw_reorder

12 Format Options

<ul class="toc-numbered-mark">

12.1 Format stream specifiers

13 Demuxers

<ul class="toc-numbered-mark">

13.1 aa
13.2 aac
13.3 apng
13.4 asf
13.5 astc
13.6 concat
<ul class="toc-numbered-mark">
13.6.1 Syntax
13.6.2 Options
13.6.3 Examples

13.7 dash
<ul class="toc-numbered-mark">
13.7.1 Options

13.8 dvdvideo
<ul class="toc-numbered-mark">
13.8.1 Background
13.8.2 Options
13.8.3 Examples

13.9 ea
<ul class="toc-numbered-mark">
13.9.1 Options

13.10 imf
13.11 flv, live_flv, kux
13.12 gif
13.13 hls
13.14 image2

<ul class="toc-numbered-mark">
13.14.1 Examples

13.15 jpeg_mpf
13.16 ktx
13.17 libgme
13.18 libmodplug
13.19 libopenmpt
13.20 mcc
<ul class="toc-numbered-mark">
13.20.1 Examples

13.21 mov/mp4/3gp
<ul class="toc-numbered-mark">
13.21.1 Options
13.21.2 Audible AAX

13.22 mpegts
13.23 mpjpeg
13.24 rawvideo
13.25 rcwt
<ul class="toc-numbered-mark">
13.25.1 Examples

13.26 sbg
13.27 tedcaptions
13.28 vapoursynth
13.29 w64
13.30 wav

13.31 webp

14 Metadata

15 Protocol Options

16 Protocols

<ul class="toc-numbered-mark">

16.1 amqp

16.2 async

16.3 bluray

16.4 cache

16.5 concat

16.6 concatf

16.7 crypto

16.8 data

16.9 fd

16.10 file

16.11 ftp

16.12 gopher

16.13 gophers

16.14 handle

16.15 http

<ul class="toc-numbered-mark">

16.15.1 HTTP Cookies

16.16 Icecast

16.17 ipfs

16.18 mmst

16.19 mmsh

16.20 md5

16.21 pipe
16.22 prompeg
16.23 rist
16.24 rtmp
16.25 rtmpe
16.26 rtmps
16.27 rtmpt
16.28 rtmpte
16.29 rtmpts
16.30 libcurl
16.31 libsmbclient
16.32 libssh
16.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte
16.34 rtp
16.35 rtsp
<ul class="toc-numbered-mark">
16.35.1 Muxer
16.35.2 Demuxer
16.35.3 Examples

16.36 sap
<ul class="toc-numbered-mark">
16.36.1 Muxer
16.36.2 Demuxer

16.37 sctp
16.38 shared
16.39 srt
16.40 srtp

16.41 subfile
16.42 tee
16.43 tcp
16.44 tls
16.45 dtls
16.46 udp
<ul class="toc-numbered-mark">
16.46.1 Examples

16.47 unix
16.48 zmq

17 Device Options
18 Input Devices
<ul class="toc-numbered-mark">
18.1 alsa
<ul class="toc-numbered-mark">
18.1.1 Options

18.2 android_camera
<ul class="toc-numbered-mark">
18.2.1 Options

18.3 avfoundation
<ul class="toc-numbered-mark">
18.3.1 Options
18.3.2 Examples

18.4 decklink
<ul class="toc-numbered-mark">
18.4.1 Options
18.4.2 Examples

18.5 dshow
<ul class="toc-numbered-mark">
18.5.1 Options
18.5.2 Examples

18.6 fbdev
<ul class="toc-numbered-mark">
18.6.1 Options

18.7 gdigrab
<ul class="toc-numbered-mark">
18.7.1 Options

18.8 iec61883
<ul class="toc-numbered-mark">
18.8.1 Options
18.8.2 Examples

18.9 jack
<ul class="toc-numbered-mark">
18.9.1 Options

18.10 kmsgrab
<ul class="toc-numbered-mark">
18.10.1 Options

18.10.2 Examples

18.11 lavfi

<ul class="toc-numbered-mark">

18.11.1 Options

18.11.2 Examples

18.12 libcdio

<ul class="toc-numbered-mark">

18.12.1 Options

18.13 libdc1394

<ul class="toc-numbered-mark">

18.13.1 Options

18.14 openal

<ul class="toc-numbered-mark">

18.14.1 Options

18.14.2 Examples

18.15 oss

<ul class="toc-numbered-mark">

18.15.1 Options

18.16 pulse

<ul class="toc-numbered-mark">

18.16.1 Options

18.16.2 Examples

18.17 sndio

<ul class="toc-numbered-mark">

18.17.1 Options

18.18 video4linux2, v4l2

<ul class="toc-numbered-mark">

18.18.1 Options

18.19 vfwcap

<ul class="toc-numbered-mark">

18.19.1 Options

18.20 x11grab

<ul class="toc-numbered-mark">

18.20.1 Options

19 Resampler Options

20 Scaler Options

21 Filtering Introduction

22 graph2dot

23 Filtergraph description

<ul class="toc-numbered-mark">

23.1 Filtergraph syntax

23.2 Notes on filtergraph escaping

24 Timeline editing

25 Changing options at runtime with a command

26 Options for filters with several inputs (framesync)

27 Audio Filters

<ul class="toc-numbered-mark">

27.1 aap

27.2 acompressor

<ul class="toc-numbered-mark">

27.2.1 Commands

27.3 acontrast

27.4 acopy

27.5 acrossfade

<ul class="toc-numbered-mark">

27.5.1 Examples

27.6 acrossover

<ul class="toc-numbered-mark">

27.6.1 Examples

27.7 acrusher

<ul class="toc-numbered-mark">

27.7.1 Commands

27.8 acue

27.9 adeclick

27.10 adeclip

27.11 adecorrelate

27.12 adelay

<ul class="toc-numbered-mark">

27.12.1 Examples

27.13 adenorm

<ul class="toc-numbered-mark">

27.13.1 Commands

27.14 aderivative, aintegral

27.15 adrc

<ul class="toc-numbered-mark">

27.15.1 Commands

27.15.2 Examples

27.16 adynamicequalizer

<ul class="toc-numbered-mark">

27.16.1 Commands

27.17 adynamicsmooth

<ul class="toc-numbered-mark">

27.17.1 Commands

27.18 aecho

<ul class="toc-numbered-mark">

27.18.1 Examples

27.19 aemphasis

<ul class="toc-numbered-mark">

27.19.1 Commands

27.20 aeval

<ul class="toc-numbered-mark">

27.20.1 Examples

27.21 aexciter

<ul class="toc-numbered-mark">

27.21.1 Commands

27.22 afade

<ul class="toc-numbered-mark">

27.22.1 Commands

27.22.2 Examples

27.23 afftdn

<ul class="toc-numbered-mark">

27.23.1 Commands

27.23.2 Examples

27.24 afftfilt

<ul class="toc-numbered-mark">

27.24.1 Examples

27.25 afir

<ul class="toc-numbered-mark">

27.25.1 Examples

27.26 aformat

27.27 afreqshift

<ul class="toc-numbered-mark">

27.27.1 Commands

27.28 afwtdn

<ul class="toc-numbered-mark">

27.28.1 Commands

27.29 agate

<ul class="toc-numbered-mark">

27.29.1 Commands

27.30 aiir

<ul class="toc-numbered-mark">

27.30.1 Examples

27.31 alimiter

27.32 allpass

<ul class="toc-numbered-mark">

27.32.1 Commands

27.33 aloop

27.34 amerge

<ul class="toc-numbered-mark">

27.34.1 Examples

27.35 amix

<ul class="toc-numbered-mark">

27.35.1 Examples

27.35.2 Commands

27.36 amultiply

27.37 anequalizer

<ul class="toc-numbered-mark">

27.37.1 Examples

27.37.2 Commands

27.38 anlmdn

<ul class="toc-numbered-mark">

27.38.1 Commands

27.39 anlmf, anlms

<ul class="toc-numbered-mark">

27.39.1 Examples

27.39.2 Commands

27.40 anull

27.41 apad

<ul class="toc-numbered-mark">

27.41.1 Examples

27.42 aphaser

27.43 aphaseshift

<ul class="toc-numbered-mark">

27.43.1 Commands

27.44 apsnr

27.45 apsyclip

<ul class="toc-numbered-mark">

27.45.1 Commands

27.46 apulsator

27.47 aresample

<ul class="toc-numbered-mark">

27.47.1 Examples

27.48 areverse

<ul class="toc-numbered-mark">

27.48.1 Examples

27.49 arls

27.50 arnndn

<ul class="toc-numbered-mark">

27.50.1 Commands

27.51 asdr

27.52 asetnsamples

27.53 asetrate

27.54 ashowinfo

27.55 asisdr

27.56 asoftclip

<ul class="toc-numbered-mark">

27.56.1 Commands

27.57 aspectralstats

27.58 asr

27.59 astats

27.60 asubboost

<ul class="toc-numbered-mark">

27.60.1 Commands

27.61 asubcut

<ul class="toc-numbered-mark">

27.61.1 Commands

27.62 asupercut

<ul class="toc-numbered-mark">

27.62.1 Commands

27.63 asuperpass

<ul class="toc-numbered-mark">

27.63.1 Commands

27.64 asuperstop

<ul class="toc-numbered-mark">

27.64.1 Commands

27.65 atempo

<ul class="toc-numbered-mark">

27.65.1 Examples

27.65.2 Commands

27.66 atilt

<ul class="toc-numbered-mark">

27.66.1 Commands

27.67 atrim

27.68 axcorrelate

<ul class="toc-numbered-mark">

27.68.1 Examples

27.69 bandpass

<ul class="toc-numbered-mark">

27.69.1 Commands

27.70 bandreject

<ul class="toc-numbered-mark">

27.70.1 Commands

27.71 bass, lowshelf

<ul class="toc-numbered-mark">

27.71.1 Commands

27.72 biquad

<ul class="toc-numbered-mark">

27.72.1 Commands

27.73 bs2b

27.74 channelmap

<ul class="toc-numbered-mark">

27.74.1 Examples

27.75 channelsplit

<ul class="toc-numbered-mark">

27.75.1 Examples

27.76 chorus

<ul class="toc-numbered-mark">

27.76.1 Examples

27.77 compand

<ul class="toc-numbered-mark">

27.77.1 Examples

27.78 compensationdelay

<ul class="toc-numbered-mark">

27.78.1 Commands

27.79 crossfeed

<ul class="toc-numbered-mark">

27.79.1 Commands

27.80 crystalizer

<ul class="toc-numbered-mark">

27.80.1 Commands

27.81 dcshift

27.82 deesser

27.83 dialoguenhance

<ul class="toc-numbered-mark">

27.83.1 Commands

27.84 drmeter

27.85 dynaudnorm

<ul class="toc-numbered-mark">

27.85.1 Commands

27.86 earwax

27.87 equalizer

<ul class="toc-numbered-mark">

27.87.1 Examples

27.87.2 Commands

27.88 extrastereo

<ul class="toc-numbered-mark">

27.88.1 Commands

27.89 firequalizer

<ul class="toc-numbered-mark">

27.89.1 Examples

27.90 flanger

27.91 haas

27.92 hdcd

27.93 headphone

<ul class="toc-numbered-mark">

27.93.1 Examples

27.94 highpass

<ul class="toc-numbered-mark">

27.94.1 Commands

27.95 join

27.96 ladspa

<ul class="toc-numbered-mark">

27.96.1 Examples

27.96.2 Commands

27.97 loudnorm

27.98 lowpass

<ul class="toc-numbered-mark">

27.98.1 Examples

27.98.2 Commands

27.99 lv2

<ul class="toc-numbered-mark">

27.99.1 Examples

27.99.2 Commands

27.100 mcompand

27.101 pan

<ul class="toc-numbered-mark">

27.101.1 Mixing examples

27.101.2 Remapping examples

27.102 replaygain

27.103 resample

27.104 rubberband

<ul class="toc-numbered-mark">

27.104.1 Commands

27.105 sidechaincompress

<ul class="toc-numbered-mark">

27.105.1 Commands

27.105.2 Examples

27.106 sidechaingate

<ul class="toc-numbered-mark">

27.106.1 Commands

27.107 silencedetect

<ul class="toc-numbered-mark">

27.107.1 Examples

27.108 silenceremove

<ul class="toc-numbered-mark">

27.108.1 Examples

27.108.2 Commands

27.109 sofalizer

<ul class="toc-numbered-mark">

27.109.1 Examples

27.110 speechnorm

<ul class="toc-numbered-mark">

27.110.1 Commands

27.110.2 Examples

27.111 stereotools

<ul class="toc-numbered-mark">

27.111.1 Commands

27.111.2 Examples

27.112 stereowiden

<ul class="toc-numbered-mark">

27.112.1 Commands

27.113 superequalizer

27.114 surround

27.115 tiltshelf

<ul class="toc-numbered-mark">

27.115.1 Commands

27.116 treble, highshelf

<ul class="toc-numbered-mark">

27.116.1 Commands

27.117 tremolo

27.118 vibrato

27.119 virtualbass

27.120 volume

<ul class="toc-numbered-mark">

27.120.1 Commands

27.120.2 Examples

27.121 volumedetect

<ul class="toc-numbered-mark">

27.121.1 Examples

27.122 whisper

<ul class="toc-numbered-mark">

27.122.1 Examples

28 Audio Sources

<ul class="toc-numbered-mark">
28.1 abuffer
<ul class="toc-numbered-mark">
28.1.1 Examples

28.2 aevalsrc
<ul class="toc-numbered-mark">
28.2.1 Examples

28.3 afdelaysrc
28.4 afireqsrc
28.5 afirsrc
28.6 anullsrc
<ul class="toc-numbered-mark">
28.6.1 Examples

28.7 flite
<ul class="toc-numbered-mark">
28.7.1 Examples

28.8 anoisesrc
<ul class="toc-numbered-mark">
28.8.1 Examples

28.9 hilbert
28.10 sinc
28.11 sine
<ul class="toc-numbered-mark">
28.11.1 Examples

29 Audio Sinks
<ul class="toc-numbered-mark">
29.1 abuffersink
29.2 anullsink

30 Video Filters
<ul class="toc-numbered-mark">
30.1 addroi
<ul class="toc-numbered-mark">
30.1.1 Examples

30.2 alphaextract
30.3 alphamerge
30.4 amplify
<ul class="toc-numbered-mark">
30.4.1 Commands

30.5 ass
<ul class="toc-numbered-mark">
30.5.1 Commands

30.6 atadenoise
<ul class="toc-numbered-mark">
30.6.1 Commands

30.7 avgblur
<ul class="toc-numbered-mark">

30.7.1 Commands

30.8 backgroundkey

<ul class="toc-numbered-mark">

30.8.1 Commands

30.9 bbox

<ul class="toc-numbered-mark">

30.9.1 Commands

30.10 bilateral

<ul class="toc-numbered-mark">

30.10.1 Commands

30.11 bitplanenoise

30.12 blackdetect, blackdetect_vulkan

30.13 blackframe

30.14 blend

<ul class="toc-numbered-mark">

30.14.1 Examples

30.14.2 Commands

30.15 blockdetect

<ul class="toc-numbered-mark">

30.15.1 Examples

30.16 blurdetect

<ul class="toc-numbered-mark">

30.16.1 Examples

30.17 bm3d

<ul class="toc-numbered-mark">

30.17.1 Examples

30.18 boxblur

<ul class="toc-numbered-mark">

30.18.1 Examples

30.19 bwdif

30.20 ccrepack

30.21 cas

<ul class="toc-numbered-mark">

30.21.1 Commands

30.22 chromahold

<ul class="toc-numbered-mark">

30.22.1 Commands

30.23 chromakey

<ul class="toc-numbered-mark">

30.23.1 Commands

30.23.2 Examples

30.24 chromanr

<ul class="toc-numbered-mark">

30.24.1 Commands

30.25 chromashift

<ul class="toc-numbered-mark">

30.25.1 Commands

30.26 ciescope

30.27 codecview

<ul class="toc-numbered-mark">

30.27.1 Examples

30.28 colorbalance

<ul class="toc-numbered-mark">

30.28.1 Examples

30.28.2 Commands

30.29 colorcontrast

<ul class="toc-numbered-mark">

30.29.1 Commands

30.30 colorcorrect

<ul class="toc-numbered-mark">

30.30.1 Commands

30.31 colorchannelmixer

<ul class="toc-numbered-mark">

30.31.1 Examples

30.31.2 Commands

30.32 colordetect

30.33 colorize

<ul class="toc-numbered-mark">

30.33.1 Commands

30.34 colorkey

<ul class="toc-numbered-mark">

30.34.1 Examples

30.34.2 Commands

30.35 colorhold

<ul class="toc-numbered-mark">

30.35.1 Commands

30.36 colorlevels

<ul class="toc-numbered-mark">

30.36.1 Examples

30.36.2 Commands

30.37 colormap

30.38 colormatrix

30.39 colorspace

30.40 colortemperature

<ul class="toc-numbered-mark">

30.40.1 Commands

30.41 convolution

<ul class="toc-numbered-mark">

30.41.1 Commands

30.41.2 Examples

30.42 convolve

30.43 copy

30.44 coreimage

<ul class="toc-numbered-mark">

30.44.1 Examples

30.45 corr

30.46 cover_rect

<ul class="toc-numbered-mark">

30.46.1 Examples

30.47 crop

<ul class="toc-numbered-mark">

30.47.1 Examples

30.47.2 Commands

30.48 cropdetect

<ul class="toc-numbered-mark">

30.48.1 Examples

30.48.2 Commands

30.49 cue

30.50 curves

<ul class="toc-numbered-mark">

30.50.1 Commands

30.50.2 Examples

30.51 datascope

<ul class="toc-numbered-mark">

30.51.1 Commands

30.52 dblur

<ul class="toc-numbered-mark">

30.52.1 Commands

30.53 dctdnoiz

<ul class="toc-numbered-mark">

30.53.1 Examples

30.54 deband

<ul class="toc-numbered-mark">

30.54.1 Commands

30.55 deblock

<ul class="toc-numbered-mark">

30.55.1 Examples

30.55.2 Commands

30.56 decimate

30.57 deconvolve

30.58 dedot

30.59 deflate

<ul class="toc-numbered-mark">

30.59.1 Commands

30.60 deflicker

30.61 deinterlace_d3d12

<ul class="toc-numbered-mark">

30.61.1 Examples

30.62 dejudder

30.63 delogo

<ul class="toc-numbered-mark">

30.63.1 Examples

30.64 derain

30.65 deshake

30.66 despill

<ul class="toc-numbered-mark">

30.66.1 Commands

30.67 detelecine

30.68 dilation

<ul class="toc-numbered-mark">

30.68.1 Commands

30.69 displace

<ul class="toc-numbered-mark">

30.69.1 Examples

30.70 dnn_classify

30.71 dnn_detect

<ul class="toc-numbered-mark">

30.71.1 Examples

30.72 dnn_processing

<ul class="toc-numbered-mark">

30.72.1 Examples

30.73 drawbox

<ul class="toc-numbered-mark">

30.73.1 Examples

30.73.2 Commands

30.74 drawgraph

30.75 drawgrid

<ul class="toc-numbered-mark">

30.75.1 Examples

30.75.2 Commands

30.76 drawtext

<ul class="toc-numbered-mark">

30.76.1 Syntax

30.76.2 Text expansion

30.76.3 Commands

30.76.4 Examples

30.77 drawvg

<ul class="toc-numbered-mark">

30.77.1 Parameters

30.77.2 Pixel Formats

30.77.3 Examples

30.78 edgedetect

<ul class="toc-numbered-mark">

30.78.1 Examples

30.79 elbg

30.80 entropy

30.81 epx

30.82 eq

<ul class="toc-numbered-mark">

30.82.1 Commands

30.83 erosion

<ul class="toc-numbered-mark">

30.83.1 Commands

30.84 estdif

<ul class="toc-numbered-mark">

30.84.1 Commands

30.85 exposure

<ul class="toc-numbered-mark">

30.85.1 Commands

30.86 extractplanes

<ul class="toc-numbered-mark">

30.86.1 Examples

30.87 fade

<ul class="toc-numbered-mark">

30.87.1 Examples

30.88 feedback

<ul class="toc-numbered-mark">

30.88.1 Examples

30.89 fftdnoiz

30.90 fftfilt

<ul class="toc-numbered-mark">

30.90.1 Examples

30.91 field

30.92 fieldhint

30.93 fieldmatch

<ul class="toc-numbered-mark">

30.93.1 p/c/n/u/b meaning

<ul class="toc-numbered-mark">

30.93.1.1 p/c/n

30.93.1.2 u/b

30.93.2 Examples

30.94 fieldorder

30.95 fillborders

<ul class="toc-numbered-mark">

30.95.1 Commands

30.96 find_rect

<ul class="toc-numbered-mark">

30.96.1 Examples

30.97 floodfill

30.98 format

<ul class="toc-numbered-mark">

30.98.1 Examples

30.99 fps

<ul class="toc-numbered-mark">

30.99.1 Examples

30.100 framepack

30.101 framerate

30.102 frc_amf

30.103 vqe_amf

30.104 framestep

30.105 freezedetect

30.106 freezeframes

30.107 frei0r

<ul class="toc-numbered-mark">

30.107.1 Examples

30.107.2 Commands

30.108 fspp

30.109 fsync

30.110 gainmap

<ul class="toc-numbered-mark">

30.110.1 Examples

30.111 gblur

<ul class="toc-numbered-mark">

30.111.1 Commands

30.112 geq

<ul class="toc-numbered-mark">

30.112.1 Examples

30.113 gradfun

<ul class="toc-numbered-mark">

30.113.1 Examples

30.114 graphmonitor

30.115 grayworld

30.116 greyedge

<ul class="toc-numbered-mark">

30.116.1 Examples

30.117 guided

<ul class="toc-numbered-mark">

30.117.1 Commands

30.117.2 Examples

30.118 haldclut

<ul class="toc-numbered-mark">

30.118.1 Commands

30.118.2 Workflow examples

<ul class="toc-numbered-mark">

30.118.2.1 Hald CLUT video stream

30.118.2.2 Hald CLUT with preview

30.119 hflip

30.120 histeq

30.121 histogram

<ul class="toc-numbered-mark">

30.121.1 Examples

30.122 hqdn3d

<ul class="toc-numbered-mark">

30.122.1 Commands

30.123 hwdownload

30.124 hwmap

30.125 hwupload

30.126 hwupload_cuda

30.127 hqx

30.128 hstack

30.129 hsvhold

30.130 hsvkey

30.131 hue

<ul class="toc-numbered-mark">

30.131.1 Examples

30.131.2 Commands

30.132 huesaturation

30.133 hysteresis

30.134 iccdetect

30.135 iccgen

30.136 identity

30.137 idet

<ul class="toc-numbered-mark">

30.137.1 Examples

30.138 il

<ul class="toc-numbered-mark">

30.138.1 Commands

30.139 inflate

<ul class="toc-numbered-mark">

30.139.1 Commands

30.140 interlace, interlace_vulkan

30.141 kerndeint

<ul class="toc-numbered-mark">

30.141.1 Examples

30.142 kirsch

<ul class="toc-numbered-mark">

30.142.1 Commands

30.143 lagfun

<ul class="toc-numbered-mark">

30.143.1 Commands

30.144 latticepal

30.145 lenscorrection

<ul class="toc-numbered-mark">

30.145.1 Options

30.145.2 Commands

30.146 lensfun

<ul class="toc-numbered-mark">

30.146.1 Examples

30.147 lcevc

30.148 libplacebo

<ul class="toc-numbered-mark">

30.148.1 Options

<ul class="toc-numbered-mark">

30.148.1.1 Output mode

30.148.1.2 Scaling

30.148.1.3 Deinterlacing

30.148.1.4 Debanding

30.148.1.5 Color adjustment

30.148.1.6 Peak detection

30.148.1.7 Tone mapping

30.148.1.8 Dithering

30.148.1.9 Custom shaders

30.148.1.10 Debugging / performance

30.148.2 Commands

30.148.3 Examples

30.149 libvmaf

<ul class="toc-numbered-mark">

30.149.1 Examples

30.150 libvmaf_cuda

<ul class="toc-numbered-mark">

30.150.1 Examples

30.151 limitdiff

<ul class="toc-numbered-mark">

30.151.1 Commands

30.152 limiter

<ul class="toc-numbered-mark">

30.152.1 Commands

30.153 loop

<ul class="toc-numbered-mark">

30.153.1 Examples

30.154 lut1d

<ul class="toc-numbered-mark">

30.154.1 Commands

30.155 lut3d

<ul class="toc-numbered-mark">

30.155.1 Commands

30.156 lumakey

<ul class="toc-numbered-mark">

30.156.1 Commands

30.157 lut, lutrgb, lutyuv

<ul class="toc-numbered-mark">

30.157.1 Commands

30.157.2 Examples

30.158 lut2, tlut2

<ul class="toc-numbered-mark">

30.158.1 Commands

30.158.2 Examples

30.159 maskedclamp

<ul class="toc-numbered-mark">

30.159.1 Commands

30.160 maskedmax

<ul class="toc-numbered-mark">

30.160.1 Commands

30.161 maskedmerge

<ul class="toc-numbered-mark">

30.161.1 Commands

30.162 maskedmin

<ul class="toc-numbered-mark">

30.162.1 Commands

30.163 maskedthreshold

<ul class="toc-numbered-mark">

30.163.1 Commands

30.164 maskfun

<ul class="toc-numbered-mark">

30.164.1 Commands

30.165 mcdeint

30.166 median

<ul class="toc-numbered-mark">

30.166.1 Commands

30.167 mergeplanes

<ul class="toc-numbered-mark">

30.167.1 Examples

30.168 mestimate

30.169 mestimate_d3d12

<ul class="toc-numbered-mark">

30.169.1 Examples

30.170 midequalizer

30.171 minterpolate

30.172 mix

<ul class="toc-numbered-mark">

30.172.1 Commands

30.173 monochrome

<ul class="toc-numbered-mark">

30.173.1 Commands

30.174 morpho

<ul class="toc-numbered-mark">

30.174.1 Commands

30.175 mpdecimate

30.176 msad

30.177 multiply

<ul class="toc-numbered-mark">

30.177.1 Commands

30.178 negate

<ul class="toc-numbered-mark">

30.178.1 Commands

30.179 nlmeans

30.180 nnedi

<ul class="toc-numbered-mark">

30.180.1 Commands

30.181 noformat

<ul class="toc-numbered-mark">

30.181.1 Examples

30.182 noise

<ul class="toc-numbered-mark">

30.182.1 Examples

30.183 normalize

<ul class="toc-numbered-mark">

30.183.1 Commands

30.183.2 Examples

30.184 null

30.185 ocio

<ul class="toc-numbered-mark">

30.185.1 Examples

30.186 ocr

30.187 ocv

<ul class="toc-numbered-mark">

30.187.1 dilate

30.187.2 erode

30.187.3 smooth

30.188 oscilloscope

<ul class="toc-numbered-mark">

30.188.1 Commands

30.188.2 Examples

30.189 overlay

<ul class="toc-numbered-mark">

30.189.1 Commands

30.189.2 Examples

30.190 owdenoise

30.191 pad

<ul class="toc-numbered-mark">

30.191.1 Examples

30.192 palettegen

<ul class="toc-numbered-mark">

30.192.1 Examples

30.193 paletteuse

<ul class="toc-numbered-mark">

30.193.1 Examples

30.194 perspective

30.195 phase

<ul class="toc-numbered-mark">

30.195.1 Commands

30.196 photosensitivity

30.197 pixdesctest

30.198 pixelize

<ul class="toc-numbered-mark">

30.198.1 Commands

30.199 pixscope

<ul class="toc-numbered-mark">

30.199.1 Commands

30.200 pp7

30.201 premultiply

30.202 premultiply_dynamic

30.203 prewitt

<ul class="toc-numbered-mark">

30.203.1 Commands

30.204 pseudocolor

<ul class="toc-numbered-mark">

30.204.1 Commands

30.204.2 Examples

30.205 psnr

<ul class="toc-numbered-mark">

30.205.1 Examples

30.206 pullup

30.207 qp

<ul class="toc-numbered-mark">

30.207.1 Examples

30.208 qrencode

<ul class="toc-numbered-mark">

30.208.1 qrencode Expressions

30.208.2 qrencode Text expansion

30.208.3 Examples

30.209 quirc

30.210 random

30.211 readeia608

<ul class="toc-numbered-mark">

30.211.1 Commands

30.211.2 Examples

30.212 readvitc

<ul class="toc-numbered-mark">

30.212.1 Examples

30.213 remap

30.214 removegrain

30.215 removelogo

30.216 repeatfields

30.217 reverse

<ul class="toc-numbered-mark">

30.217.1 Examples

30.218 rgbashift

<ul class="toc-numbered-mark">

30.218.1 Commands

30.219 roberts

<ul class="toc-numbered-mark">

30.219.1 Commands

30.220 rotate

<ul class="toc-numbered-mark">

30.220.1 Examples

30.220.2 Commands

30.221 sab

30.222 scale

<ul class="toc-numbered-mark">

30.222.1 Options

30.222.2 Examples

30.222.3 Commands

30.223 scharr

<ul class="toc-numbered-mark">

30.223.1 Commands

30.224 scroll

<ul class="toc-numbered-mark">

30.224.1 Commands

30.225 scdet

30.226 selectivecolor

<ul class="toc-numbered-mark">

30.226.1 Examples

30.227 separatefields

30.228 setdar, setsar

<ul class="toc-numbered-mark">

30.228.1 Examples

30.229 setfield

30.230 setparams

30.231 shear

<ul class="toc-numbered-mark">

30.231.1 Commands

30.232 showinfo

30.233 showpalette

30.234 shuffleframes

<ul class="toc-numbered-mark">

30.234.1 Examples

30.235 shufflepixels

30.236 shuffleplanes

<ul class="toc-numbered-mark">

30.236.1 Examples

30.237 signalstats

<ul class="toc-numbered-mark">

30.237.1 Examples

30.238 signature

<ul class="toc-numbered-mark">

30.238.1 Examples

30.239 siti

<ul class="toc-numbered-mark">

30.239.1 Examples

30.240 smartblur

30.241 sobel

<ul class="toc-numbered-mark">

30.241.1 Commands

30.242 spp

<ul class="toc-numbered-mark">

30.242.1 Commands

30.243 sr

30.244 sr_amf

<ul class="toc-numbered-mark">

30.244.1 Examples

30.245 ssim

<ul class="toc-numbered-mark">

30.245.1 Examples

30.246 stereo3d

<ul class="toc-numbered-mark">

30.246.1 Examples

30.247 streamselect, astreamselect

<ul class="toc-numbered-mark">

30.247.1 Commands

30.247.2 Examples

30.248 subtitles

30.249 super2xsai

30.250 swaprect

<ul class="toc-numbered-mark">

30.250.1 Commands

30.251 swapuv

30.252 tblend

30.253 telecine

30.254 thistogram

30.255 threshold

<ul class="toc-numbered-mark">

30.255.1 Commands

30.255.2 Examples

30.256 thumbnail

<ul class="toc-numbered-mark">

30.256.1 Examples

30.257 tile

<ul class="toc-numbered-mark">

30.257.1 Examples

30.258 tiltandshift

30.259 tinterlace

30.260 tmedian

<ul class="toc-numbered-mark">

30.260.1 Commands

30.261 tmidequalizer

30.262 tmix

<ul class="toc-numbered-mark">

30.262.1 Examples

30.262.2 Commands

30.263 tonemap

<ul class="toc-numbered-mark">

30.263.1 Options

30.264 tpad

30.265 transpose

30.266 trim

30.267 unpremultiply

30.268 unsharp

<ul class="toc-numbered-mark">

30.268.1 Examples

30.269 untile

<ul class="toc-numbered-mark">

30.269.1 Examples

30.270 uspp

30.271 v360

<ul class="toc-numbered-mark">

30.271.1 Examples

30.271.2 Commands

30.272 vaguedenoiser

30.273 varblur

<ul class="toc-numbered-mark">

30.273.1 Commands

30.274 vectorscope

30.275 vidstabdetect

<ul class="toc-numbered-mark">

30.275.1 Examples

30.276 vidstabtransform

<ul class="toc-numbered-mark">

30.276.1 Options

30.276.2 Examples

30.277 vflip

30.278 vfrdet

30.279 vibrance

<ul class="toc-numbered-mark">

30.279.1 Commands

30.280 vif

30.281 vignette

<ul class="toc-numbered-mark">

30.281.1 Expressions

30.281.2 Examples

30.282 vmafmotion

30.283 vpp_amf

<ul class="toc-numbered-mark">

30.283.1 Examples

30.284 vstack

30.285 w3fdif

<ul class="toc-numbered-mark">

30.285.1 Commands

30.286 waveform

30.287 weave, doubleweave

<ul class="toc-numbered-mark">

30.287.1 Examples

30.288 xbr

30.289 xcorrelate

30.290 xfade

<ul class="toc-numbered-mark">

30.290.1 Examples

30.291 xmedian

<ul class="toc-numbered-mark">

30.291.1 Commands

30.292 xpsnr

<ul class="toc-numbered-mark">

30.292.1 Examples

30.293 xstack

<ul class="toc-numbered-mark">

30.293.1 Examples

30.294 yadif

30.295 yaepblur

<ul class="toc-numbered-mark">

30.295.1 Commands

30.296 zoompan

<ul class="toc-numbered-mark">

30.296.1 Examples

30.297 zscale

<ul class="toc-numbered-mark">

30.297.1 Options

30.297.2 Commands

31 CUDA Video Filters

<ul class="toc-numbered-mark">

31.1 bilateral_cuda

<ul class="toc-numbered-mark">

31.1.1 Examples

31.2 bwdif_cuda

31.3 chromakey_cuda

<ul class="toc-numbered-mark">

31.3.1 Examples

31.4 colorspace_cuda

31.5 overlay_cuda

31.6 pad_cuda

<ul class="toc-numbered-mark">

31.6.1 Examples

31.7 scale_cuda

<ul class="toc-numbered-mark">

31.7.1 Examples

31.8 thumbnail_cuda

<ul class="toc-numbered-mark">

31.8.1 Example

31.9 transpose_cuda

31.10 yadif_cuda

32 OpenCL Video Filters

<ul class="toc-numbered-mark">

32.1 avgblur_opencl

<ul class="toc-numbered-mark">

32.1.1 Example

32.2 boxblur_opencl

<ul class="toc-numbered-mark">

32.2.1 Examples

32.3 colorkey_opencl

<ul class="toc-numbered-mark">

32.3.1 Examples

32.4 convolution_opencl

<ul class="toc-numbered-mark">

32.4.1 Examples

32.5 erosion_opencl

<ul class="toc-numbered-mark">

32.5.1 Example

32.6 deshake_opencl

<ul class="toc-numbered-mark">

32.6.1 Examples

32.7 dilation_opencl

<ul class="toc-numbered-mark">

32.7.1 Example

32.8 nlmeans_opencl

32.9 overlay_opencl

<ul class="toc-numbered-mark">

32.9.1 Examples

32.10 pad_opencl

32.11 prewitt_opencl

<ul class="toc-numbered-mark">

32.11.1 Example

32.12 program_opencl

32.13 remap_opencl

32.14 roberts_opencl

<ul class="toc-numbered-mark">

32.14.1 Example

32.15 sobel_opencl

<ul class="toc-numbered-mark">

32.15.1 Example

32.16 tonemap_opencl

<ul class="toc-numbered-mark">

32.16.1 Example

32.17 unsharp_opencl

<ul class="toc-numbered-mark">

32.17.1 Examples

32.18 xfade_opencl

33 VAAPI Video Filters

<ul class="toc-numbered-mark">

33.1 overlay_vaapi

<ul class="toc-numbered-mark">

33.1.1 Examples

33.2 tonemap_vaapi

<ul class="toc-numbered-mark">

33.2.1 Example

33.3 hstack_vaapi

33.4 vstack_vaapi

33.5 xstack_vaapi

33.6 pad_vaapi

33.7 drawbox_vaapi

<ul class="toc-numbered-mark">

33.7.1 Examples

34 VideoToolbox Video Filters

<ul class="toc-numbered-mark">

34.1 scale_vt

<ul class="toc-numbered-mark">

34.1.1 Examples

34.2 transpose_vt

35 Vulkan Video Filters

<ul class="toc-numbered-mark">

35.1 avgblur_vulkan

35.2 blend_vulkan

35.3 bwdif_vulkan

35.4 chromaber_vulkan

35.5 color_vulkan

35.6 vflip_vulkan

35.7 hflip_vulkan

35.8 flip_vulkan

35.9 fruc_vulkan

35.10 gblur_vulkan

35.11 nlmeans_vulkan

35.12 overlay_vulkan

35.13 transpose_vulkan

35.14 v360_vulkan

<ul class="toc-numbered-mark">

35.14.1 Examples

36 QSV Video Filters

<ul class="toc-numbered-mark">

36.1 hstack_qsv

36.2 vstack_qsv

36.3 xstack_qsv

37 Video Sources

<ul class="toc-numbered-mark">

37.1 buffer

37.2 cellauto

<ul class="toc-numbered-mark">

37.2.1 Examples

37.3 coreimagesrc

<ul class="toc-numbered-mark">

37.3.1 Examples

37.4 ddagrab

<ul class="toc-numbered-mark">

37.4.1 Examples

37.5 gfxcapture

<ul class="toc-numbered-mark">

37.5.1 Examples

37.6 gradients

<ul class="toc-numbered-mark">

37.6.1 Commands

37.7 mandelbrot

37.8 mptestsrc

37.9 frei0r_src

37.10 life

<ul class="toc-numbered-mark">

37.10.1 Examples

37.11 perlin

<ul class="toc-numbered-mark">

37.11.1 Options

37.11.2 Examples

37.12 qrencodesrc

<ul class="toc-numbered-mark">

37.12.1 Examples

37.13 allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc

<ul class="toc-numbered-mark">

37.13.1 Examples

37.13.2 Commands

37.14 openclsrc

37.15 sierpinski

37.16 zoneplate

<ul class="toc-numbered-mark">

37.16.1 Commands

37.16.2 Examples

38 Video Sinks

<ul class="toc-numbered-mark">

38.1 buffersink

38.2 nullsink

39 Multimedia Filters

<ul class="toc-numbered-mark">

39.1 a3dscope

<ul class="toc-numbered-mark">

39.1.1 Commands

39.2 abitscope

39.3 adrawgraph

39.4 agraphmonitor

39.5 ahistogram

39.6 aphasemeter

<ul class="toc-numbered-mark">

39.6.1 phasing detection

39.6.2 Examples

39.7 avectorscope

<ul class="toc-numbered-mark">

39.7.1 Examples

39.7.2 Commands

39.8 bench, abench

<ul class="toc-numbered-mark">

39.8.1 Examples

39.9 concat

<ul class="toc-numbered-mark">

39.9.1 Examples

39.9.2 Commands

39.10 ebur128

<ul class="toc-numbered-mark">

39.10.1 Examples

39.11 interleave, ainterleave

<ul class="toc-numbered-mark">

39.11.1 Examples

39.12 latency, alatency

39.13 metadata, ametadata

<ul class="toc-numbered-mark">

39.13.1 Examples

39.14 perms, aperms

39.15 realtime, arealtime

<ul class="toc-numbered-mark">

39.15.1 Commands

39.16 segment, asegment

<ul class="toc-numbered-mark">

39.16.1 Examples

39.17 select, aselect

<ul class="toc-numbered-mark">

39.17.1 Examples

39.18 sendcmd, asendcmd

<ul class="toc-numbered-mark">

39.18.1 Commands syntax

39.18.2 Examples

39.19 setpts, asetpts

<ul class="toc-numbered-mark">

39.19.1 Examples

39.19.2 Commands

39.20 setrange

39.21 settb, asettb

<ul class="toc-numbered-mark">

39.21.1 Examples

39.22 showcqt

<ul class="toc-numbered-mark">

39.22.1 Examples

39.23 showcwt

39.24 showfreqs

39.25 showspatial

39.26 showspectrum

<ul class="toc-numbered-mark">

39.26.1 Examples

39.27 showspectrumpic

<ul class="toc-numbered-mark">

39.27.1 Examples

39.28 showvolume

39.29 showwaves

<ul class="toc-numbered-mark">

39.29.1 Examples

39.30 showwavespic

<ul class="toc-numbered-mark">

39.30.1 Examples

39.31 sidedata, asidedata

39.32 spectrumsynth

<ul class="toc-numbered-mark">

39.32.1 Examples

39.33 split, asplit

<ul class="toc-numbered-mark">

39.33.1 Examples

39.34 zmq, azmq

<ul class="toc-numbered-mark">

39.34.1 Examples

40 Multimedia Sources

<ul class="toc-numbered-mark">

40.1 amovie

40.2 avsynctest

<ul class="toc-numbered-mark">

40.2.1 Commands

40.3 movie

<ul class="toc-numbered-mark">

40.3.1 Examples

40.3.2 Commands

41 External libraries

<ul class="toc-numbered-mark">

41.1 Alliance for Open Media (AOM)

41.2 ARM ASTC Encoder

41.3 AMD AMF/VCE

41.4 AviSynth

41.5 Chromaprint

41.6 codec2

41.7 dav1d

41.8 davs2

41.9 uavs3d

41.10 Game Music Emu

41.11 Intel QuickSync Video

41.12 Kvazaar

41.13 LAME

41.14 LCEVCdec

41.15 libilbc

41.16 libjxl

41.17 libvpx

41.18 ModPlug

41.19 ONNX Runtime

41.20 OpenCORE, VisualOn, and Fraunhofer libraries

<ul class="toc-numbered-mark">

41.20.1 OpenCORE AMR

41.20.2 VisualOn AMR-WB encoder library

41.20.3 Fraunhofer AAC library

41.20.4 Fraunhofer xHE-AAC library

41.20.5 Fraunhofer MPEG-H 3D Audio decoder library

41.20.6 LC3 library

41.21 OpenH264

41.22 OpenJPEG

41.23 rav1e

41.24 SVT-AV1

41.25 SVT-JPEG-XS

41.26 TwoLAME

41.27 VapourSynth

41.28 x264

41.29 x265

41.30 xavs

41.31 xavs2

41.32 eXtra-fast Essential Video Encoder (XEVE)

41.33 eXtra-fast Essential Video Decoder (XEVD)

41.34 ZVBI

42 Supported File Formats, Codecs or Features

<ul class="toc-numbered-mark">

42.1 File Formats

42.2 Image Formats

42.3 Video Codecs

42.4 Audio Codecs

42.5 Subtitle Formats

42.6 Network Protocols

42.7 Input/Output Devices

42.8 Timecode

43 See Also

44 Authors

 <h2 class="chapter">1 Synopsis# TOC</h2>

ffplay [<var class="var">options</var>] [<samp class="file">input_url</samp>] <h2 class="chapter">2 Description# TOC</h2>

FFplay is a very simple and portable media player using the FFmpeg libraries and the SDL library. It is mostly used as a testbed for the various FFmpeg APIs. <h2 class="chapter">3 Options# TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or ’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’ as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class="example-preformatted">ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">3.1 Stream specifiers# TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class="code">a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class="code">-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class="code">-codec copy</code>

or <code class=”code”>-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class=”code”>-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class=”var”>stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class=”var”>stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class=”option”><var class=”var”>stream_type</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> <var class=”var”>stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class=”var”>additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class=”var”>additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class=”option”>g:<var class=”var”>group_specifier</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class="var">group_specifier</var>.

if <var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class="var">additional_stream_specifier</var>.

<var class="var">group_specifier</var> may be one of the following:

<dl class="table">

<dt><samp class="option"><var class="var">group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class="var">additional_stream_specifier</var>.

</dd>

<dt><samp class="option">#<var class="var">stream_id</var> or i:<var class="var">stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class="option">m:<var class="var">key</var>[:<var class="var">value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class="var">key</var> having the specified value. If

<var class="var">value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class="var">key</var> or <var class="var">value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class="option">disp:<var class="var">dispositions</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class="var">dispositions</var> is a list of

one or more dispositions (as printed by the <samp class="option">-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class="option">u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class="command">ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">3.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options,

including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed information about the demuxer named <var class="var">demuxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class="option">muxer=<var class="var">muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class="var">muxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class="option">filter=<var class="var">filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class="var">filter_name</var>. Use the <samp class="option">-filters</samp> option to get a list of all filters. </dd> <dt><samp class="option">bsf=<var class="var">bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class="var">bitstream_filter_name</var>. Use the <samp class="option">-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class="option">protocol=<var class="var">protocol_name</var></samp></dt> <dd> Print detailed information about the protocol named <var class="var">protocol_name</var>. Use the <samp class="option">-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class="option">-version</samp></dt> <dd> Show version. </dd> <dt><samp class="option">-buildconf</samp></dt> <dd> Show the build configuration, one option per line. </dd> <dt><samp class="option">-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class="option">-demuxers</samp></dt> <dd> Show available demuxers. </dd> <dt><samp class="option">-muxers</samp></dt> <dd> Show available muxers. </dd> <dt><samp class="option">-devices</samp></dt> <dd> Show available devices. </dd> <dt><samp class="option">-codecs</samp></dt> <dd> Show all codecs known to libavcodec. Note that the term ’codec’ is used throughout this documentation as a shortcut for what is more correctly called a media bitstream format. </dd> <dt><samp class="option">-decoders</samp></dt> <dd> Show available decoders. </dd> <dt><samp class="option">-encoders</samp></dt> <dd> Show

all available encoders. </dd> <dt><samp class="option">-bsfs</samp></dt> <dd> Show available bitstream filters. </dd> <dt><samp class="option">-protocols</samp></dt> <dd> Show available protocols. </dd> <dt><samp class="option">-filters</samp></dt> <dd> Show available libavfilter filters. </dd> <dt><samp class="option">-pix_fmts</samp></dt> <dd> Show available pixel formats. </dd> <dt><samp class="option">-sample_fmts</samp></dt> <dd> Show available sample formats. </dd> <dt><samp class="option">-layouts</samp></dt> <dd> Show channel names and standard channel layouts. </dd> <dt><samp class="option">-dispositions</samp></dt> <dd> Show stream dispositions. </dd> <dt><samp class="option">-colors</samp></dt> <dd> Show recognized color names. </dd> <dt><samp class="option">-sources <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sources of the input device. Some devices may provide system-dependent source names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sources pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sinks of the output device. Some devices may provide system-dependent sink names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var class="var">loglevel</var></samp></dt> <dd> Set logging level and flags used by the library. The optional <var class="var">flags</var> prefix can consist of the following values: <dl class="table"> <dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates that repeated log output should not be compressed to the first line and the "Last message repeated n times" line will be omitted. </dd> <dt>‘<samp class="samp">level</samp>’</dt> <dd> Indicates that log output should add a <code class="code">[level]</code> prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var

class=”var”>flags</var> or changing <var class=”var”>loglevel</var>. When setting both <var class=”var”>flags</var> and <var class=”var”>loglevel</var>, a ’+’ separator is expected between the last <var class=”var”>flags</var> value and before <var class=”var”>loglevel</var>. <var class=”var”>loglevel</var> is a string or a number containing one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class=”samp”>panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class=”samp”>fatal, 8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class=”samp”>error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class=”samp”>warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class=”samp”>info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class=”samp”>verbose, 40</samp>’</dt> <dd> Same as <code class=”code”>info</code>, except more verbose. </dd> <dt>‘<samp class=”samp”>debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class=”samp”>trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class=”code”>level</code> prefix, and set <var class=”var”>loglevel</var> to <code class=”code”>verbose</code>: <pre class=”example-preformatted”>ffmpeg -loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code class=”code”>level</code> prefix flag or <var class=”var”>loglevel</var>: <pre class=”example-preformatted”>ffmpeg [...] -loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class=”env”>AV_LOG_FORCE_NOCOLOR</code>, or can be forced setting the environment variable <code class=”env”>AV_LOG_FORCE_COLOR</code>. </dd> <dt><samp class=”option”>-report</samp></dt> <dd> Dump full command line and log output to a file named <code class=”code”><var class=”var”>program</var>-<var class=”var”>YYYYMMDD</var>-<var class=”var”>HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class=”code”>-loglevel debug</code>. Setting the environment variable <code class=”env”>FFREPORT</code> to any value has the same ef-

fect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual). The following options are recognized: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> set the file name to use for the report; <code class="code">%p</code> is expanded to the name of the program, <code class="code">%t</code> is expanded to a timestamp, <code class="code">%%</code> is expanded to a plain <code class="code">%</code> </dd> <dt><samp class="option">level</samp></dt> <dd> set the log verbosity level using a numerical value (see <code class="code">-loglevel</code>). </dd> </dl>

For example, to output a report to a file named <samp class="file">ffreport.log</samp> using a log level of <code class="code">32</code> (alias for log level <code class="code">info</code>):   <pre class="example-preformatted">FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class="option">-hide_banner</samp></dt> <dd> Suppress printing banner. All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class="option">-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class="example-preformatted">ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class="table"> <dt>‘<samp class="samp">x86</samp>’</dt> <dd><dl class="table"> <dt>‘<samp class="samp">mmx</samp>’</dt> <dt>‘<samp class="samp">mmxext</samp>’</dt> <dt>‘<samp class="samp">sse</samp>’</d <dt>‘<samp class="samp">sse2</samp>’</dt> <dt>‘<samp class="samp">sse2slow</samp>’</dt> <dt>‘<samp class="samp">sse3</samp>’</ <dt>‘<samp class="samp">sse3slow</samp>’</dt> <dt>‘<samp class="samp">ssse3</samp>’</dt> <dt>‘<samp class="samp">atom</samp>’</dt <dt>‘<samp class="samp">sse4.1</samp>’</dt> <dt>‘<samp class="samp">sse4.2</samp>’</dt> <dt>‘<samp class="samp">avx</samp>’</dt> <dt>‘<samp class="samp">avx2</samp>’</dt> <dt>‘<samp class="samp">xop</samp>’</dt> <dt>‘<samp class="samp">fma3</samp>’</dt> <dt>‘<samp class="samp">fma4</samp>’</dt> <dt>‘<samp class="samp">3dnow</samp>’</dt> <dt>‘<samp class="samp">3dnowext</samp>&rsquo <dt>‘<samp class="samp">bmi1</samp>’</dt> <dt>‘<samp class="samp">bmi2</samp>’</dt> <dt>‘<samp class="samp">cmov</samp>’</dt

</dl> </dd> <dt>‘<samp class="samp">ARM</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv5te</samp>’</dt>
<dt>‘<samp class="samp">armv6</samp>’</dt> <dt>‘<samp
class="samp">armv6t2</samp>’</dt> <dt>‘<samp class="samp">vfp</samp>’</d
<dt>‘<samp class="samp">vfpv3</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> <dt>‘<samp class="samp">setend</samp>’</d
</dl> </dd> <dt>‘<samp class="samp">AArch64</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv8</samp>’</dt>
<dt>‘<samp class="samp">vfp</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> </dl> </dd> <dt>‘<samp
class="samp">PowerPC</samp>’</dt> <dd><dl class="table">
<dt>‘<samp class="samp">altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class="samp">Specific Processors</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">pentium2</samp>’</dt>
<dt>‘<samp class="samp">pentium3</samp>’</dt>
<dt>‘<samp class="samp">pentium4</samp>’</dt>
<dt>‘<samp class="samp">k6</samp>’</dt> <dt>‘<samp
class="samp">k62</samp>’</dt> <dt>‘<samp class="samp">athlon</samp>’</dt
<dt>‘<samp class="samp">athlonxp</samp>’</dt> <dt>‘<samp
class="samp">k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class="option">-cpucount <var class="var">count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class="example-preformatted">ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class="option">-max_alloc <var class="var">bytes</var></samp></dt>
<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is INT_MAX. </dd> </dl>

 <h3 class="section">3.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and libavcodec libraries. To see the list of available AVOptions, use the <samp class="option">-help</samp> option. They are separated into two categories: <dl class="table"> <dt><samp class="option">generic</samp></dt>
<dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class="option">private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their

corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class="option">id3v2_version</samp> private option of the MP3 muxer: <pre class="example-preformatted">ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class="example-preformatted">ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class="option">-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class="option">-option 0</samp>/<samp class="option">-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class="section">3.4 Main options# TOC</h3>

<dl class="table"> <dt><samp class="option">-x <var class="var">width</var></samp></dt> <dd> Force displayed width. </dd> <dt><samp class="option">-y <var class="var">height</var></samp></dt> <dd> Force displayed height. </dd> <dt><samp class="option">-fs</samp></dt> <dd> Start in fullscreen mode. </dd> <dt><samp class="option">-an</samp></dt> <dd> Disable audio. </dd> <dt><samp class="option">-vn</samp></dt> <dd> Disable video. </dd> <dt><samp class="option">-sn</samp></dt> <dd> Disable subtitles. </dd> <dt><samp class="option">-ss <var class="var">pos</var></samp></dt> <dd> Seek to <var class="var">pos</var>. Note that in most formats it is not possible to seek exactly, so <code class="command">ffplay</code> will seek to the nearest seek point to <var class="var">pos</var>. <var class="var">pos</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. </dd> <dt><samp class="option">-t <var class="var">duration</var></samp></dt> <dd> Play <var class="var">duration</var> seconds of audio/video. <var class="var">duration</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual. </dd> <dt><samp class="option">-bytes</samp></dt> <dd> Seek by bytes. </dd> <dt><samp class="option">-

seek_interval</samp></dt> <dd> Set custom interval, in seconds, for seeking using left/right keys. Default is 10 seconds. </dd> <dt><samp class="option">-nodisp</samp></dt> <dd> Disable graphical display. </dd> <dt><samp class="option">-noborder</samp></dt> <dd> Borderless window. </dd> <dt><samp class="option">-alwaysontop</samp></dt> <dd> Window always on top. Available on: X11 with SDL >= 2.0.5, Windows SDL >= 2.0.6. </dd> <dt><samp class="option">-volume</samp></dt> <dd> Set the startup volume. 0 means silence, 100 means no volume reduction or amplification. Negative values are treated as 0, values above 100 are treated as 100. </dd> <dt><samp class="option">-f <var class="var">fmt</var></samp></dt> <dd> Force format. </dd> <dt><samp class="option">-window_title <var class="var">title</var></samp></dt> <dd> Set window title (default is the input filename). </dd> <dt><samp class="option">-left <var class="var">title</var></samp></dt> <dd> Set the x position for the left of the window (default is a centered window). </dd> <dt><samp class="option">-top <var class="var">title</var></samp></dt> <dd> Set the y position for the top of the window (default is a centered window). </dd> <dt><samp class="option">-loop <var class="var">number</var></samp></dt> <dd> Loops movie playback <number> times. 0 means forever. </dd> <dt><samp class="option">-showmode <var class="var">mode</var></samp></dt> <dd> Set the show mode to use. Available values for <var class="var">mode</var> are: <dl class="table"> <dt>‘<samp class="samp">0, video</samp>’</dt> <dd> show video </dd> <dt>‘<samp class="samp">1, waves</samp>’</dt> <dd> show audio waves </dd> <dt>‘<samp class="samp">2, rdft</samp>’</dt> <dd> show audio frequency band using RDFT ((Inverse) Real Discrete Fourier Transform) </dd> <dt>‘<samp class="samp">3, phase</samp>’</dt> <dd> like rdft, with stereo content whose left channel leads in phase shown in blue and whose right channel leads shown in yellow </dd> </dl>

Default value is "video", if video is not present or cannot be played "rdft" is automatically selected. You can interactively cycle through the available show modes by pressing the key <kbd class="key">w</kbd>. </dd> <dt><samp class="option">-vf <var class="var">filtergraph</var></samp></dt> <dd> Create the filtergraph specified by <var class="var">filtergraph</var> and use it to filter the video stream. <var class="var">filtergraph</var> is a description of the filtergraph to apply to the stream, and must have a single video input and a single video output. In the filtergraph, the input is associated to the label <code class="code">in</code>, and the output to the label <code class="code">out</code>. See the ffmpeg-filters manual for more information about the filtergraph syntax. You can specify this parameter multiple times and cycle through the

specified filtergraphs along with the show modes by pressing the key <kbd class=”key”>w</kbd>. </dd> <dt><samp class=”option”>-af <var class=”var”>filtergraph</var></samp></dt> <dd> <var class=”var”>filtergraph</var> is a description of the filtergraph to apply to the input audio. Use the option "-filters" to show all the available filters (including sources and sinks). </dd> <dt><samp class=”option”>-i <var class=”var”>input_url</var></samp></dt> <dd> Read <var class=”var”>input_url</var>. </dd> </dl>

<h3 class=”section”>3.5 Advanced options# TOC</h3>

<dl class=”table”>

<dt><samp class=”option”>-stats</samp></dt>

<dd> Print several playback statistics, in particular show the stream

duration, the codec parameters, the current position in the stream and

the audio/video synchronisation drift. It is shown by default, unless the

log level is lower than <code class=”code”>info</code>. Its display can be forced by manually

specifying this option. To disable it, you need to specify <code class=”code”>-nostats</code>.

</dd>

<dt><samp class=”option”>-fast</samp></dt>

<dd> Non-spec-compliant optimizations.

</dd>

<dt><samp class=”option”>-genpts</samp></dt>

<dd> Generate pts.

</dd>

<dt><samp class=”option”>-sync <var class=”var”>type</var></samp></dt>

<dd> Set the master clock to audio (<code class=”code”>type=audio</code>), video

(<code class=”code”>type=video</code>) or external (<code class=”code”>type=ext</code>). Default is audio. The

master clock is used to control audio-video synchronization. Most media

players use audio as master clock, but in some cases (streaming or high

quality broadcast) it is necessary to change that. This option is mainly

used for debugging purposes.

</dd>

<dt><samp class="option">-ast <var class="var">audio_stream_specifier</var></samp></dt>

<dd> Select the desired audio stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" audio stream is selected in the program of the

already selected video stream.

</dd>

<dt><samp class="option">-vst <var class="var">video_stream_specifier</var></samp></dt>

<dd> Select the desired video stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" video stream is selected.

</dd>

<dt><samp class="option">-sst <var class="var">subtitle_stream_specifier</var></samp></dt>

<dd> Select the desired subtitle stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" subtitle stream is selected in the program of the

already selected video or audio stream.

</dd>

<dt><samp class="option">-autoexit</samp></dt>

<dd> Exit when video is done playing.

</dd>

<dt><samp class="option">-exitonkeydown</samp></dt>

<dd> Exit if any key is pressed.

</dd>

<dt><samp class="option">-exitonmousedown</samp></dt>

<dd> Exit if any mouse button is pressed.

</dd>

<dt><samp class="option">-codec:<var class="var">media_specifier</var>
<var class="var">codec_name</var></samp></dt>

<dd> Force a specific decoder implementation for the stream identified by

<var class="var">media_specifier</var>, which can assume the values <code
class="code">a</code> (audio),

<code class="code">v</code> (video), and <code class="code">s</code>
subtitle.

</dd>

<dt><samp class="option">-acodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific audio decoder.

</dd>

<dt><samp class="option">-vcodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific video decoder.

</dd>

<dt><samp class="option">-scodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific subtitle decoder.

</dd>

<dt><samp class="option">-autorotate</samp></dt>

<dd> Automatically rotate the video according to file metadata. Enabled
by

default, use <samp class="option">-noautorotate</samp> to disable it.

</dd>

<dt><samp class="option">-framedrop</samp></dt>

<dd> Drop video frames if video is out of sync. Enabled by default if the master

clock is not set to video. Use this option to enable frame dropping for all

master clock sources, use <samp class="option">-noframedrop</samp> to disable it.

</dd>

<dt><samp class="option">-infbuf</samp></dt>

<dd> Do not limit the input buffer size, read as much data as possible from the

input as soon as possible. Enabled by default for realtime streams, where data

may be dropped if not read in time. Use this option to enable infinite buffers

for all inputs, use <samp class="option">-noinfbuf</samp> to disable it.

</dd>

<dt><samp class="option">-filter_threads <var class="var">nb_threads</var></samp></dt>

<dd> Defines how many threads are used to process a filter pipeline. Each pipeline

will produce a thread pool with this many threads available for parallel

processing. The default is 0 which means that the thread count will be

determined by the number of available CPUs.

</dd>

<dt><samp class="option">-enable_vulkan</samp></dt>

<dd> Use vulkan renderer rather than SDL builtin renderer. Depends on libplacebo.

</dd>

<dt><samp class="option">-vulkan_params</samp></dt>

<dd>

Vulkan configuration using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by

":".

</dd>

<dt><samp class="option">-hwaccel</samp></dt>

<dd> Use HW accelerated decoding. Enable this option will enable vulkan renderer

automatically.

</dd>

<dt><samp class="option">-video_bg <var class="var">pattern</var></samp></dt>

<dd> Set the video background pattern used for transparent videos. For solid color

patterns you can use the color name or color code as specified in the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">Color section in the ffmpeg-utils(1) manual.

You can also use the special <code class="code">tiles</code> keyword for a checker board style

pattern. This is also the default. The <code class="code">none</code> pattern does rendering

without alpha blending effectively ignoring alpha channel.

</dd>

</dl>

 <h3 class="section">3.6 While playing# TOC</h3>

<dl class="table">

<dt><kbd class="key">q, ESC</kbd></dt>

<dd> Quit.

</dd>

<dt><kbd class=”key”>f</kbd></dt>
<dd> Toggle full screen.

</dd>
<dt><kbd class=”key”>p, SPC</kbd></dt>
<dd> Pause.

</dd>
<dt><kbd class=”key”>m</kbd></dt>
<dd> Toggle mute.

</dd>
<dt><kbd class=”key”>9, 0</kbd></dt>
<dt><kbd class=”key”>/, *</kbd></dt>
<dd> Decrease and increase volume respectively.

</dd>
<dt><kbd class=”key”>a</kbd></dt>
<dd> Cycle audio channel in the current program.

</dd>
<dt><kbd class=”key”>v</kbd></dt>
<dd> Cycle video channel.

</dd>
<dt><kbd class=”key”>t</kbd></dt>
<dd> Cycle subtitle channel in the current program.

</dd>
<dt><kbd class=”key”>c</kbd></dt>
<dd> Cycle program.

</dd>

<dt><kbd class=”key”>w</kbd></dt>

<dd> Cycle video filters or show modes.

</dd>

<dt><kbd class=”key”>s</kbd></dt>

<dd> Step to the next frame.

Pause if the stream is not already paused, step to the next video

frame, and pause.

</dd>

<dt><kbd class=”key”>left/right</kbd></dt>

<dd> Seek backward/forward by 10 seconds or a custom interval if -seek_interval is set.

</dd>

<dt><kbd class=”key”>down/up</kbd></dt>

<dd> Seek backward/forward 1 minute.

</dd>

<dt><kbd class=”key”>page down/page up</kbd></dt>

<dd> Seek to previous/next chapter or backward/forward 10 minutes if no chapters.

The chapter list is shown briefly, marking the chapter being played at its

left edge. It needs libass and the SDL renderer, so it is unavailable

in builds without libass and with the Vulkan renderer.

</dd>

<dt><kbd class=”key”>l</kbd></dt>

<dd> Keep the chapter list on screen. While it is kept, typing filters the list by

artist and title, down/up move its selection and escape closes it.

</dd>

<dt><kbd class=”key”>o</kbd></dt>

<dd> Show the files in the directory of the input. Enter or a click plays the

selected file or opens the selected directory, escape closes the list. Typing

filters the list by name.

</dd>

<dt><kbd class=”key”>enter</kbd></dt>

<dd> Seek to the chapter selected in the chapter list while it is shown.

</dd>

<dt><kbd class=”key”>mouse wheel</kbd></dt>

<dd> Move the chapter list selection while it is shown.

</dd>

<dt><kbd class=”key”>mouse over the (i) symbol</kbd></dt>

<dd> Show the metadata of that chapter list entry.

</dd>

<dt><kbd class=”key”>left mouse click</kbd></dt>

<dd> Seek to the clicked chapter list entry, or sort the list by the clicked column

button. Clicking the same button again flips the order, and ties are resolved

by the columns clicked before. The shuffle button orders the list by a seed

that its left half lowers and its right half raises.

</dd>

<dt><kbd class=”key”>right mouse click</kbd></dt>

<dd> Seek to percentage in file corresponding to fraction of width.

</dd>

<dt><kbd class=”key”>left mouse double-click</kbd></dt>

<dd> Toggle full screen.

</dd>

</dl>

 <h2 class=”chapter”>4 Syntax# TOC</h2>

This section documents the syntax and formats employed by the FFmpeg libraries and tools. <h3 class=”section”>4.1 Quoting and escaping# TOC</h3>

FFmpeg adopts the following quoting and escaping mechanism, unless explicitly specified. The following rules are applied: <ul class=”itemize mark-bullet”> ‘<samp class=”samp”>'</samp>’ and ‘<samp class=”samp”>\</samp>’ are special characters (respectively used for quoting and escaping). In addition to them, there might be other special characters depending on the specific syntax where the escaping and quoting are employed.

A special character is escaped by prefixing it with a ‘<samp class=”samp”>\</samp>’.

All characters enclosed between ‘<samp class=”samp”>''</samp>’ are included literally in the parsed string. The quote character ‘<samp class=”samp”>'</samp>’ itself cannot be quoted, so you may need to close the quote and escape it.

Leading and trailing whitespaces, unless escaped or quoted, are removed from the parsed string.

Note that you may need to add a second level of escaping when using the command line or a script, which depends on the syntax of the adopted shell language. The function <code class=”code”>av_get_token</code> defined in <samp class=”file”>libavutil/avstring.h</samp> can be used to parse a token quoted or escaped according to the rules defined above.

The tool <samp class=”file”>tools/ffescape</samp> in the FFmpeg source tree can be used to automatically quote or escape a string in a script. <h4 class=”subsection”>4.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Escape the string <code class=”code”>Crime d'Amour</code> containing the <code class=”code”>'</code> special character: <pre class=”example-preformatted”>Crime d\'Amour </pre>

The string above contains a quote, so the <code class=”code”>'</code> needs to be escaped when quoting it: <pre class=”example-preformatted”>'Crime d'\''Amour' </pre>

Include leading or trailing whitespaces using quoting: <pre class=”example-preformatted”>' this string starts and ends with whitespaces ' </pre>

Escaping and quoting can be mixed together: <pre class=”example-preformatted”>' The string '\'string\'' is a string ' </pre>

To include a literal ‘<samp class=”samp”>\</samp>’ you can use either escaping or quoting: <pre class=”example-preformatted”>'c:\foo' can be written as c:\\foo </pre>

 <h3 class=”section”>4.2 Date# TOC</h3>

The accepted syntax is: <pre class=”example-preformatted”>[(YYYY-MM-DD|YYYYMMDD)[T|t|]]((HH:MM:SS[.m...]]])|(HHMMSS[.m...]]]))[Z] now </pre>

If the value is "now" it takes the current time. Time is local time unless Z is appended, in which case it is interpreted as UTC. If the year-month-day part is not specified it takes the current year-month-day. <h3 class=”section”>4.3 Time duration# TOC</h3>

There are two accepted syntaxes for expressing time duration.   <pre class=”example-preformatted”>[-][<var

class=”var”>HH</var>:]<var class=”var”>MM</var>:<var class=”var”>SS</var>[.<var class=”var”>m</var>...] </pre>

<var class=”var”>HH</var> expresses the number of hours, <var class=”var”>MM</var> the number of minutes for a maximum of 2 digits, and <var class=”var”>SS</var> the number of seconds for a maximum of 2 digits. The <var class=”var”>m</var> at the end expresses decimal value for <var class=”var”>SS</var>. or   <pre class=”example-preformatted”>[-]<var class=”var”>S</var>+[.<var class=”var”>m</var>...][s|ms|us] </pre>

<var class=”var”>S</var> expresses the number of seconds, with the optional decimal part <var class=”var”>m</var>. The optional literal suffixes ‘<samp class=”samp”>s</samp>’, ‘<samp class=”samp”>ms</samp>’ or ‘<samp class=”samp”>us</samp>’ indicate to interpret the value as seconds, milliseconds or microseconds, respectively. In both expressions, the optional ‘<samp class=”samp”>-</samp>’ indicates negative duration. <h4 class=”subsection”>4.3.1 Examples# TOC</h4>

The following examples are all valid time duration: <dl class=”table”> <dt>‘<samp class=”samp”>55</samp>’</dt> <dd> 55 seconds </dd> <dt>‘<samp class=”samp”>0.2</samp>’</dt> <dd> 0.2 seconds </dd> <dt>‘<samp class=”samp”>200ms</samp>’</dt> <dd> 200 milliseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>200000us</samp>’</dt> <dd> 200000 microseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>12:03:45</samp>’</dt> <dd> 12 hours, 03 minutes and 45 seconds </dd> <dt>‘<samp class=”samp”>23.189</samp>’</dt> <dd> 23.189 seconds </dd> </dl>

<h3 class=”section”>4.4 Video size# TOC</h3>

Specify the size of the sourced video, it may be a string of the form

<var class=”var”>width</var>x<var class=”var”>height</var>, or the name of a size abbreviation.

The following abbreviations are recognized:

<dl class=”table”>

<dt>‘<samp class="samp">ntsc</samp>’</dt>
<dd> 720x480
</dd>
<dt>‘<samp class="samp">pal</samp>’</dt>
<dd> 720x576
</dd>
<dt>‘<samp class="samp">qntsc</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">qpal</samp>’</dt>
<dd> 352x288
</dd>
<dt>‘<samp class="samp">sntsc</samp>’</dt>
<dd> 640x480
</dd>
<dt>‘<samp class="samp">spal</samp>’</dt>
<dd> 768x576
</dd>
<dt>‘<samp class="samp">film</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">ntsc-film</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">sqcif</samp>’</dt>
<dd> 128x96
</dd>
<dt>‘<samp class="samp">qcif</samp>’</dt>
<dd> 176x144
</dd>
<dt>‘<samp class="samp">cif</samp>’</dt>

<dd> 352x288
</dd>
<dt>‘<samp class="samp">4cif</samp>’</dt>
<dd> 704x576
</dd>
<dt>‘<samp class="samp">16cif</samp>’</dt>
<dd> 1408x1152
</dd>
<dt>‘<samp class="samp">qqvga</samp>’</dt>
<dd> 160x120
</dd>
<dt>‘<samp class="samp">qvga</samp>’</dt>
<dd> 320x240
</dd>
<dt>‘<samp class="samp">vga</samp>’</dt>
<dd> 640x480
</dd>
<dt>‘<samp class="samp">svga</samp>’</dt>
<dd> 800x600
</dd>
<dt>‘<samp class="samp">xga</samp>’</dt>
<dd> 1024x768
</dd>
<dt>‘<samp class="samp">uxga</samp>’</dt>
<dd> 1600x1200
</dd>
<dt>‘<samp class="samp">qxga</samp>’</dt>
<dd> 2048x1536
</dd>
<dt>‘<samp class="samp">sxga</samp>’</dt>
<dd> 1280x1024

</dd>
<dt>‘<samp class="samp">qsxga</samp>’</dt>
<dd> 2560x2048
</dd>
<dt>‘<samp class="samp">hsxga</samp>’</dt>
<dd> 5120x4096
</dd>
<dt>‘<samp class="samp">wvga</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">wxga</samp>’</dt>
<dd> 1366x768
</dd>
<dt>‘<samp class="samp">wsxga</samp>’</dt>
<dd> 1600x1024
</dd>
<dt>‘<samp class="samp">wuxga</samp>’</dt>
<dd> 1920x1200
</dd>
<dt>‘<samp class="samp">woxga</samp>’</dt>
<dd> 2560x1600
</dd>
<dt>‘<samp class="samp">wqsxga</samp>’</dt>
<dd> 3200x2048
</dd>
<dt>‘<samp class="samp">wquxga</samp>’</dt>
<dd> 3840x2400
</dd>
<dt>‘<samp class="samp">whsxga</samp>’</dt>
<dd> 6400x4096
</dd>

<dt>‘<samp class=”samp”>whuxga</samp>’</dt>
<dd> 7680x4800
</dd>
<dt>‘<samp class=”samp”>cga</samp>’</dt>
<dd> 320x200
</dd>
<dt>‘<samp class=”samp”>ega</samp>’</dt>
<dd> 640x350
</dd>
<dt>‘<samp class=”samp”>hd480</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class=”samp”>hd720</samp>’</dt>
<dd> 1280x720
</dd>
<dt>‘<samp class=”samp”>hd1080</samp>’</dt>
<dd> 1920x1080
</dd>
<dt>‘<samp class=”samp”>2k</samp>’</dt>
<dd> 2048x1080
</dd>
<dt>‘<samp class=”samp”>2kflat</samp>’</dt>
<dd> 1998x1080
</dd>
<dt>‘<samp class=”samp”>2kscope</samp>’</dt>
<dd> 2048x858
</dd>
<dt>‘<samp class=”samp”>4k</samp>’</dt>
<dd> 4096x2160
</dd>
<dt>‘<samp class=”samp”>4kflat</samp>’</dt>

<dd> 3996x2160
</dd>
<dt>‘<samp class=”samp”>4kscope</samp>’</dt>
<dd> 4096x1716
</dd>
<dt>‘<samp class=”samp”>nhd</samp>’</dt>
<dd> 640x360
</dd>
<dt>‘<samp class=”samp”>hqvga</samp>’</dt>
<dd> 240x160
</dd>
<dt>‘<samp class=”samp”>wqvga</samp>’</dt>
<dd> 400x240
</dd>
<dt>‘<samp class=”samp”>fwqvga</samp>’</dt>
<dd> 432x240
</dd>
<dt>‘<samp class=”samp”>hvga</samp>’</dt>
<dd> 480x320
</dd>
<dt>‘<samp class=”samp”>qhd</samp>’</dt>
<dd> 960x540
</dd>
<dt>‘<samp class=”samp”>2kdci</samp>’</dt>
<dd> 2048x1080
</dd>
<dt>‘<samp class=”samp”>4kdci</samp>’</dt>
<dd> 4096x2160
</dd>
<dt>‘<samp class=”samp”>uhd2160</samp>’</dt>
<dd> 3840x2160

</dd>

<dt>‘<samp class="samp">uhd4320</samp>’</dt>

<dd> 7680x4320

</dd>

</dl>

<h3 class="section">4.5 Video rate<a class="anchor
hidden-xs" href="#Video-rate" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-Video-rate" aria-hidden="true">TOC</h3>

Specify the frame rate of a video, expressed as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a float number or a valid video frame rate abbreviation. The following abbreviations are recognized: <dl class="table"> <dt>‘<samp class="samp">ntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">pal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">qntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">qpal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">sntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">spal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> 24/1 </dd> <dt>‘<samp class="samp">ntsc-film</samp>’</dt> <dd> 24000/1001 </dd> </dl>

 <h3 class="section">4.6 Ratio# TOC</h3>

A ratio can be expressed as an expression, or in the form <var class="var">numerator</var>:<var class="var">denominator</var>.
Note that a ratio with infinite (1/0) or negative value is considered valid, so you should check on the returned value if you want to exclude those values.
The undefined value can be expressed using the "0:0" string.
<h3 class="section">4.7 Color# TOC</h3>

It can be the name of a color as defined below (case insensitive match) or a

<code class="code">[0x|#]RRGGBB[AA]</code> sequence, possibly followed by @ and a string

representing the alpha component.

The alpha component may be a string composed by "0x" followed by an

hexadecimal number or a decimal number between 0.0 and 1.0, which

represents the opacity value (‘<samp class="samp">0x00</samp>’ or ‘<samp class="samp">0.0</samp>’ means completely

transparent, ‘<samp class="samp">0xff</samp>’ or ‘<samp class="samp">1.0</samp>’ completely opaque). If the alpha

component is not specified then ‘<samp class="samp">0xff</samp>’ is assumed.

The string ‘<samp class="samp">random</samp>’ will result in a random color.

The following names of colors are recognized:

<dl class="table">

<dt>‘<samp class="samp">AliceBlue</samp>’</dt>

<dd> 0xF0F8FF

</dd>

<dt>‘<samp class="samp">AntiqueWhite</samp>’</dt>

<dd> 0xFAEBD7

</dd>

<dt>‘<samp class="samp">Aqua</samp>’</dt>

<dd> 0x00FFFF

</dd>

<dt>‘<samp class="samp">Aquamarine</samp>’</dt>

<dd> 0x7FFFD4

</dd>

<dt>‘<samp class="samp">Azure</samp>’</dt>

<dd> 0xF0FFFF

</dd>

<dt>‘<samp class="samp">Beige</samp>’</dt>

<dd> 0xF5F5DC
</dd>
<dt>‘<samp class=”samp”>Bisque</samp>’</dt>
<dd> 0xFFE4C4
</dd>
<dt>‘<samp class=”samp”>Black</samp>’</dt>
<dd> 0x000000
</dd>
<dt>‘<samp class=”samp”>BlanchedAlmond</samp>’</dt>
<dd> 0xFFEBCD
</dd>
<dt>‘<samp class=”samp”>Blue</samp>’</dt>
<dd> 0x0000FF
</dd>
<dt>‘<samp class=”samp”>BlueViolet</samp>’</dt>
<dd> 0x8A2BE2
</dd>
<dt>‘<samp class=”samp”>Brown</samp>’</dt>
<dd> 0xA52A2A
</dd>
<dt>‘<samp class=”samp”>BurlyWood</samp>’</dt>
<dd> 0xDEB887
</dd>
<dt>‘<samp class=”samp”>CadetBlue</samp>’</dt>
<dd> 0x5F9EA0
</dd>
<dt>‘<samp class=”samp”>Chartreuse</samp>’</dt>
<dd> 0x7FFF00
</dd>
<dt>‘<samp class=”samp”>Chocolate</samp>’</dt>
<dd> 0xD2691E

</dd>
<dt>‘<samp class="samp">Coral</samp>’</dt>
<dd> 0xFF7F50
</dd>
<dt>‘<samp class="samp">CornflowerBlue</samp>’</dt>
<dd> 0x6495ED
</dd>
<dt>‘<samp class="samp">Cornsilk</samp>’</dt>
<dd> 0xFFF8DC
</dd>
<dt>‘<samp class="samp">Crimson</samp>’</dt>
<dd> 0xDC143C
</dd>
<dt>‘<samp class="samp">Cyan</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class="samp">DarkBlue</samp>’</dt>
<dd> 0x00008B
</dd>
<dt>‘<samp class="samp">DarkCyan</samp>’</dt>
<dd> 0x008B8B
</dd>
<dt>‘<samp class="samp">DarkGoldenRod</samp>’</dt>
<dd> 0xB8860B
</dd>
<dt>‘<samp class="samp">DarkGray</samp>’</dt>
<dd> 0xA9A9A9
</dd>
<dt>‘<samp class="samp">DarkGreen</samp>’</dt>
<dd> 0x006400
</dd>

<dt>‘<samp class=”samp”>DarkKhaki</samp>’</dt>
<dd> 0xBDB76B
</dd>
<dt>‘<samp class=”samp”>DarkMagenta</samp>’</dt>
<dd> 0x8B008B
</dd>
<dt>‘<samp class=”samp”>DarkOliveGreen</samp>’</dt>
<dd> 0x556B2F
</dd>
<dt>‘<samp class=”samp”>Darkorange</samp>’</dt>
<dd> 0xFF8C00
</dd>
<dt>‘<samp class=”samp”>DarkOrchid</samp>’</dt>
<dd> 0x9932CC
</dd>
<dt>‘<samp class=”samp”>DarkRed</samp>’</dt>
<dd> 0x8B0000
</dd>
<dt>‘<samp class=”samp”>DarkSalmon</samp>’</dt>
<dd> 0xE9967A
</dd>
<dt>‘<samp class=”samp”>DarkSeaGreen</samp>’</dt>
<dd> 0x8FBC8F
</dd>
<dt>‘<samp class=”samp”>DarkSlateBlue</samp>’</dt>
<dd> 0x483D8B
</dd>
<dt>‘<samp class=”samp”>DarkSlateGray</samp>’</dt>
<dd> 0x2F4F4F
</dd>
<dt>‘<samp class=”samp”>DarkTurquoise</samp>’</dt>

<dd> 0x00CED1
</dd>
<dt>‘<samp class=”samp”>DarkViolet</samp>’</dt>
<dd> 0x9400D3
</dd>
<dt>‘<samp class=”samp”>DeepPink</samp>’</dt>
<dd> 0xFF1493
</dd>
<dt>‘<samp class=”samp”>DeepSkyBlue</samp>’</dt>
<dd> 0x00BFFF
</dd>
<dt>‘<samp class=”samp”>DimGray</samp>’</dt>
<dd> 0x696969
</dd>
<dt>‘<samp class=”samp”>DodgerBlue</samp>’</dt>
<dd> 0x1E90FF
</dd>
<dt>‘<samp class=”samp”>FireBrick</samp>’</dt>
<dd> 0xB22222
</dd>
<dt>‘<samp class=”samp”>FloralWhite</samp>’</dt>
<dd> 0xFFFAF0
</dd>
<dt>‘<samp class=”samp”>ForestGreen</samp>’</dt>
<dd> 0x228B22
</dd>
<dt>‘<samp class=”samp”>Fuchsia</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class=”samp”>Gainsboro</samp>’</dt>
<dd> 0xDCDCDC

</dd>
<dt>‘<samp class="samp">GhostWhite</samp>’</dt>
<dd> 0xF8F8FF
</dd>
<dt>‘<samp class="samp">Gold</samp>’</dt>
<dd> 0xFFD700
</dd>
<dt>‘<samp class="samp">GoldenRod</samp>’</dt>
<dd> 0xDAA520
</dd>
<dt>‘<samp class="samp">Gray</samp>’</dt>
<dd> 0x808080
</dd>
<dt>‘<samp class="samp">Green</samp>’</dt>
<dd> 0x008000
</dd>
<dt>‘<samp class="samp">GreenYellow</samp>’</dt>
<dd> 0xADFF2F
</dd>
<dt>‘<samp class="samp">HoneyDew</samp>’</dt>
<dd> 0xF0FFF0
</dd>
<dt>‘<samp class="samp">HotPink</samp>’</dt>
<dd> 0xFF69B4
</dd>
<dt>‘<samp class="samp">IndianRed</samp>’</dt>
<dd> 0xCD5C5C
</dd>
<dt>‘<samp class="samp">Indigo</samp>’</dt>
<dd> 0x4B0082
</dd>

<dt>‘<samp class=”samp”>Ivory</samp>’</dt>
<dd> 0xFFFFF0
</dd>
<dt>‘<samp class=”samp”>Khaki</samp>’</dt>
<dd> 0xF0E68C
</dd>
<dt>‘<samp class=”samp”>Lavender</samp>’</dt>
<dd> 0xE6E6FA
</dd>
<dt>‘<samp class=”samp”>LavenderBlush</samp>’</dt>
<dd> 0xFFF0F5
</dd>
<dt>‘<samp class=”samp”>LawnGreen</samp>’</dt>
<dd> 0x7CFC00
</dd>
<dt>‘<samp class=”samp”>LemonChiffon</samp>’</dt>
<dd> 0xFFFACD
</dd>
<dt>‘<samp class=”samp”>LightBlue</samp>’</dt>
<dd> 0xADD8E6
</dd>
<dt>‘<samp class=”samp”>LightCoral</samp>’</dt>
<dd> 0xF08080
</dd>
<dt>‘<samp class=”samp”>LightCyan</samp>’</dt>
<dd> 0xE0FFFF
</dd>
<dt>‘<samp class=”samp”>LightGoldenRodYellow</samp>’</dt>
<dd> 0xFAFAD2
</dd>
<dt>‘<samp class=”samp”>LightGreen</samp>’</dt>

<dd> 0x90EE90
</dd>
<dt>‘<samp class=”samp”>LightGrey</samp>’</dt>
<dd> 0xD3D3D3
</dd>
<dt>‘<samp class=”samp”>LightPink</samp>’</dt>
<dd> 0xFFB6C1
</dd>
<dt>‘<samp class=”samp”>LightSalmon</samp>’</dt>
<dd> 0xFFA07A
</dd>
<dt>‘<samp class=”samp”>LightSeaGreen</samp>’</dt>
<dd> 0x20B2AA
</dd>
<dt>‘<samp class=”samp”>LightSkyBlue</samp>’</dt>
<dd> 0x87CEFA
</dd>
<dt>‘<samp class=”samp”>LightSlateGray</samp>’</dt>
<dd> 0x778899
</dd>
<dt>‘<samp class=”samp”>LightSteelBlue</samp>’</dt>
<dd> 0xB0C4DE
</dd>
<dt>‘<samp class=”samp”>LightYellow</samp>’</dt>
<dd> 0xFFFFE0
</dd>
<dt>‘<samp class=”samp”>Lime</samp>’</dt>
<dd> 0x00FF00
</dd>
<dt>‘<samp class=”samp”>LimeGreen</samp>’</dt>
<dd> 0x32CD32

</dd>
<dt>‘<samp class=”samp”>Linen</samp>’</dt>
<dd> 0xFAF0E6
</dd>
<dt>‘<samp class=”samp”>Magenta</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class=”samp”>Maroon</samp>’</dt>
<dd> 0x800000
</dd>
<dt>‘<samp class=”samp”>MediumAquaMarine</samp>’</dt>
<dd> 0x66CDAA
</dd>
<dt>‘<samp class=”samp”>MediumBlue</samp>’</dt>
<dd> 0x0000CD
</dd>
<dt>‘<samp class=”samp”>MediumOrchid</samp>’</dt>
<dd> 0xBA55D3
</dd>
<dt>‘<samp class=”samp”>MediumPurple</samp>’</dt>
<dd> 0x9370D8
</dd>
<dt>‘<samp class=”samp”>MediumSeaGreen</samp>’</dt>
<dd> 0x3CB371
</dd>
<dt>‘<samp class=”samp”>MediumSlateBlue</samp>’</dt>
<dd> 0x7B68EE
</dd>
<dt>‘<samp class=”samp”>MediumSpringGreen</samp>’</dt>
<dd> 0x00FA9A
</dd>

<dt>‘<samp class=”samp”>MediumTurquoise</samp>’</dt>
<dd> 0x48D1CC
</dd>
<dt>‘<samp class=”samp”>MediumVioletRed</samp>’</dt>
<dd> 0xC71585
</dd>
<dt>‘<samp class=”samp”>MidnightBlue</samp>’</dt>
<dd> 0x191970
</dd>
<dt>‘<samp class=”samp”>MintCream</samp>’</dt>
<dd> 0xF5FFFA
</dd>
<dt>‘<samp class=”samp”>MistyRose</samp>’</dt>
<dd> 0xFFE4E1
</dd>
<dt>‘<samp class=”samp”>Moccasin</samp>’</dt>
<dd> 0xFFE4B5
</dd>
<dt>‘<samp class=”samp”>NavajoWhite</samp>’</dt>
<dd> 0xFFDEAD
</dd>
<dt>‘<samp class=”samp”>Navy</samp>’</dt>
<dd> 0x000080
</dd>
<dt>‘<samp class=”samp”>OldLace</samp>’</dt>
<dd> 0xFDF5E6
</dd>
<dt>‘<samp class=”samp”>Olive</samp>’</dt>
<dd> 0x808000
</dd>
<dt>‘<samp class=”samp”>OliveDrab</samp>’</dt>

<dd> 0x6B8E23
</dd>
<dt>‘<samp class=”samp”>Orange</samp>’</dt>
<dd> 0xFFA500
</dd>
<dt>‘<samp class=”samp”>OrangeRed</samp>’</dt>
<dd> 0xFF4500
</dd>
<dt>‘<samp class=”samp”>Orchid</samp>’</dt>
<dd> 0xDA70D6
</dd>
<dt>‘<samp class=”samp”>PaleGoldenRod</samp>’</dt>
<dd> 0xEEE8AA
</dd>
<dt>‘<samp class=”samp”>PaleGreen</samp>’</dt>
<dd> 0x98FB98
</dd>
<dt>‘<samp class=”samp”>PaleTurquoise</samp>’</dt>
<dd> 0xAFEEEE
</dd>
<dt>‘<samp class=”samp”>PaleVioletRed</samp>’</dt>
<dd> 0xD87093
</dd>
<dt>‘<samp class=”samp”>PapayaWhip</samp>’</dt>
<dd> 0xFFEFD5
</dd>
<dt>‘<samp class=”samp”>PeachPuff</samp>’</dt>
<dd> 0xFFDAB9
</dd>
<dt>‘<samp class=”samp”>Peru</samp>’</dt>
<dd> 0xCD853F

</dd>
<dt>‘<samp class="samp">Pink</samp>’</dt>
<dd> 0xFFC0CB
</dd>
<dt>‘<samp class="samp">Plum</samp>’</dt>
<dd> 0xDDA0DD
</dd>
<dt>‘<samp class="samp">PowderBlue</samp>’</dt>
<dd> 0xB0E0E6
</dd>
<dt>‘<samp class="samp">Purple</samp>’</dt>
<dd> 0x800080
</dd>
<dt>‘<samp class="samp">Red</samp>’</dt>
<dd> 0xFF0000
</dd>
<dt>‘<samp class="samp">RosyBrown</samp>’</dt>
<dd> 0xBC8F8F
</dd>
<dt>‘<samp class="samp">RoyalBlue</samp>’</dt>
<dd> 0x4169E1
</dd>
<dt>‘<samp class="samp">SaddleBrown</samp>’</dt>
<dd> 0x8B4513
</dd>
<dt>‘<samp class="samp">Salmon</samp>’</dt>
<dd> 0xFA8072
</dd>
<dt>‘<samp class="samp">SandyBrown</samp>’</dt>
<dd> 0xF4A460
</dd>

<dt>‘<samp class=”samp”>SeaGreen</samp>’</dt>
<dd> 0x2E8B57
</dd>
<dt>‘<samp class=”samp”>SeaShell</samp>’</dt>
<dd> 0xFFF5EE
</dd>
<dt>‘<samp class=”samp”>Sienna</samp>’</dt>
<dd> 0xA0522D
</dd>
<dt>‘<samp class=”samp”>Silver</samp>’</dt>
<dd> 0xC0C0C0
</dd>
<dt>‘<samp class=”samp”>SkyBlue</samp>’</dt>
<dd> 0x87CEEB
</dd>
<dt>‘<samp class=”samp”>SlateBlue</samp>’</dt>
<dd> 0x6A5ACD
</dd>
<dt>‘<samp class=”samp”>SlateGray</samp>’</dt>
<dd> 0x708090
</dd>
<dt>‘<samp class=”samp”>Snow</samp>’</dt>
<dd> 0xFFFAFA
</dd>
<dt>‘<samp class=”samp”>SpringGreen</samp>’</dt>
<dd> 0x00FF7F
</dd>
<dt>‘<samp class=”samp”>SteelBlue</samp>’</dt>
<dd> 0x4682B4
</dd>
<dt>‘<samp class=”samp”>Tan</samp>’</dt>

<dd> 0xD2B48C
</dd>
<dt>‘<samp class=”samp”>Teal</samp>’</dt>
<dd> 0x008080
</dd>
<dt>‘<samp class=”samp”>Thistle</samp>’</dt>
<dd> 0xD8BFD8
</dd>
<dt>‘<samp class=”samp”>Tomato</samp>’</dt>
<dd> 0xFF6347
</dd>
<dt>‘<samp class=”samp”>Turquoise</samp>’</dt>
<dd> 0x40E0D0
</dd>
<dt>‘<samp class=”samp”>Violet</samp>’</dt>
<dd> 0xEE82EE
</dd>
<dt>‘<samp class=”samp”>Wheat</samp>’</dt>
<dd> 0xF5DEB3
</dd>
<dt>‘<samp class=”samp”>White</samp>’</dt>
<dd> 0xFFFFFF
</dd>
<dt>‘<samp class=”samp”>WhiteSmoke</samp>’</dt>
<dd> 0xF5F5F5
</dd>
<dt>‘<samp class=”samp”>Yellow</samp>’</dt>
<dd> 0xFFFF00
</dd>
<dt>‘<samp class=”samp”>YellowGreen</samp>’</dt>
<dd> 0x9ACD32

</dd>

</dl>

 <h3 class=”section”>4.8 Channel Layout# TOC</h3>

A channel layout specifies the spatial disposition of the channels in a multi-channel audio stream. To specify a channel layout, FFmpeg makes use of a special syntax. Individual channels are identified by an id, as given by the table below: <dl class=”table”> <dt>‘<samp class=”samp”>FL</samp>’</dt> <dd> front left </dd> <dt>‘<samp class=”samp”>FR</samp>’</dt> <dd> front right </dd> <dt>‘<samp class=”samp”>FC</samp>’</dt> <dd> front center </dd> <dt>‘<samp class=”samp”>LFE</samp>’</dt> <dd> low frequency </dd> <dt>‘<samp class=”samp”>BL</samp>’</dt> <dd> back left </dd> <dt>‘<samp class=”samp”>BR</samp>’</dt> <dd> back right </dd> <dt>‘<samp class=”samp”>FLC</samp>’</dt> <dd> front left-of-center </dd> <dt>‘<samp class=”samp”>FRC</samp>’</dt> <dd> front right-of-center </dd> <dt>‘<samp class=”samp”>BC</samp>’</dt> <dd> back center </dd> <dt>‘<samp class=”samp”>SL</samp>’</dt> <dd> side left </dd> <dt>‘<samp class=”samp”>SR</samp>’</dt> <dd> side right </dd> <dt>‘<samp class=”samp”>TC</samp>’</dt> <dd> top center </dd> <dt>‘<samp class=”samp”>TFL</samp>’</dt> <dd> top front left </dd> <dt>‘<samp class=”samp”>TFC</samp>’</dt> <dd> top front center </dd> <dt>‘<samp class=”samp”>TFR</samp>’</dt> <dd> top front right </dd> <dt>‘<samp class=”samp”>TBL</samp>’</dt> <dd> top back left </dd> <dt>‘<samp class=”samp”>TBC</samp>’</dt> <dd> top back center </dd> <dt>‘<samp class=”samp”>TBR</samp>’</dt> <dd> top back right </dd> <dt>‘<samp class=”samp”>DL</samp>’</dt> <dd> downmix left </dd> <dt>‘<samp class=”samp”>DR</samp>’</dt> <dd> downmix right </dd> <dt>‘<samp class=”samp”>WL</samp>’</dt> <dd> wide left </dd> <dt>‘<samp class=”samp”>WR</samp>’</dt> <dd> wide right </dd> <dt>‘<samp class=”samp”>SDL</samp>’</dt> <dd> surround direct left </dd> <dt>‘<samp class=”samp”>SDR</samp>’</dt> <dd> surround direct right </dd> <dt>‘<samp class=”samp”>LFE2</samp>’</dt> <dd> low frequency 2 </dd> </dl>

Standard channel layout compositions can be specified by using the

following identifiers:

<dl class=”table”>

<dt>‘<samp class=”samp”>mono</samp>’</dt>

<dd> FC

</dd>
<dt>‘<samp class="samp">stereo</samp>’</dt>
<dd> FL+FR
</dd>
<dt>‘<samp class="samp">2.1</samp>’</dt>
<dd> FL+FR+LFE
</dd>
<dt>‘<samp class="samp">3.0</samp>’</dt>
<dd> FL+FR+FC
</dd>
<dt>‘<samp class="samp">3.0(back)</samp>’</dt>
<dd> FL+FR+BC
</dd>
<dt>‘<samp class="samp">4.0</samp>’</dt>
<dd> FL+FR+FC+BC
</dd>
<dt>‘<samp class="samp">quad</samp>’</dt>
<dd> FL+FR+BL+BR
</dd>
<dt>‘<samp class="samp">quad(side)</samp>’</dt>
<dd> FL+FR+SL+SR
</dd>
<dt>‘<samp class="samp">3.1</samp>’</dt>
<dd> FL+FR+FC+LFE
</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR
</dd>
<dt>‘<samp class="samp">5.0(side)</samp>’</dt>
<dd> FL+FR+FC+SL+SR
</dd>

<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR
</dd>
<dt>‘<samp class="samp">5.1(side)</samp>’</dt>
<dd> FL+FR+FC+LFE+SL+SR
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> FL+FR+FC+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.0(front)</samp>’</dt>
<dd> FL+FR+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">3.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+TFL+TFR
</dd>
<dt>‘<samp class="samp">hexagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1(front)</samp>’</dt>
<dd> FL+FR+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.0</samp>’</dt>

<dd> FL+FR+FC+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.0(front)</samp>’</dt>
<dd> FL+FR+FC+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.1(wide)</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC
</dd>
<dt>‘<samp class="samp">7.1(wide-side)</samp>’</dt>
<dd> FL+FR+FC+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">5.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR
</dd>
<dt>‘<samp class="samp">octagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR
</dd>
<dt>‘<samp class="samp">cube</samp>’</dt>
<dd> FL+FR+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">5.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR
</dd>
<dt>‘<samp class="samp">7.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBL+TBR

</dd>

<dt>‘<samp class="samp">7.2.3</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBC+LFE2

</dd>

<dt>‘<samp class="samp">9.1.4</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR

</dd>

<dt>‘<samp class="samp">9.1.6</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR+TSL+TSR

</dd>

<dt>‘<samp class="samp">hexadecagonal</samp>’</dt>

<dd> FL+FR+FC+BL+BR+BC+SL+SR+WL+WR+TBL+TBR+TBC+TFC+TFL+TFR

</dd>

<dt>‘<samp class="samp">binaural</samp>’</dt>

<dd> BIL+BIR

</dd>

<dt>‘<samp class="samp">downmix</samp>’</dt>

<dd> DL+DR

</dd>

<dt>‘<samp class="samp">22.2</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+BC+SL+SR+TC+TFL+TFC+TFR+TBL+TBC+TBI

</dd>

</dl>

A custom channel layout can be specified as a sequence of terms, separated by ’+’. Each term can be: <ul class="itemize mark-bullet"> the name of a single channel (e.g. ‘<samp class="samp">FL</samp>’, ‘<samp class="samp">FR</samp>’, ‘<samp class="samp">FC</samp>’, ‘<samp class="samp">LFE</samp>’, etc.), each optionally containing a custom name after a ’@’, (e.g. ‘<samp class="samp">FL@Left</samp>’, ‘<samp class="samp">FR@Right</samp>’, ‘<samp class="samp">FC@Center</samp>’, ‘<samp class="samp">LFE@Low_Frequency</samp>’, etc.)

A standard channel layout can be specified by the following: <ul class=”itemize mark-bullet”> the name of a single channel (e.g. ‘<samp class=”samp”>FL</samp>’, ‘<samp class=”samp”>FR</samp>’, ‘<samp class=”samp”>FC</samp>’, ‘<samp class=”samp”>LFE</samp>’, etc.)

the name of a standard channel layout (e.g. ‘<samp class=”samp”>mono</samp>’, ‘<samp class=”samp”>stereo</samp>’, ‘<samp class=”samp”>4.0</samp>’, ‘<samp class=”samp”>quad</samp>’, ‘<samp class=”samp”>5.0</samp>’, etc.)

a number of channels, in decimal, followed by ’c’, yielding the default channel layout for that number of channels (see the function <code class=”code”>av_channel_layout_default</code>). Note that not all channel counts have a default layout.

a number of channels, in decimal, followed by ’C’, yielding an unknown channel layout with the specified number of channels. Note that not all channel layout specification strings support unknown channel layouts.

a channel layout mask, in hexadecimal starting with "0x" (see the <code class=”code”>AV_CH_*</code> macros in <samp class=”file”>libavutil/channel_layout.h</samp>.

Before libavutil version 53 the trailing character "c" to specify a number of channels was optional, but now it is required, while a channel layout mask can also be specified as a decimal number (if and only if not followed by "c" or "C"). See also the function <code class=”code”>av_channel_layout_from_string</code> defined in <samp class=”file”>libavutil/channel_layout.h</samp>. <h2 class=”chapter”>5 Expression Evaluation# TOC</h2>

When evaluating an arithmetic expression, FFmpeg uses an internal

formula evaluator, implemented through the <samp class=”file”>libavutil/eval.h</samp>

interface.

An expression may contain unary, binary operators, constants, and

functions.

Two expressions <var class=”var”>expr1</var> and <var class=”var”>expr2</var> can be combined to form

another expression "<var class=”var”>expr1</var>;<var class=”var”>expr2</var>".

<var class="var">expr1</var> and <var class="var">expr2</var> are evaluated in turn, and the new

expression evaluates to the value of <var class="var">expr2</var>.

The following binary operators are available: <code class="code">+</code>, <code class="code">-</code>,

<code class="code">*</code>, <code class="code">/</code>, <code class="code">^</code>.

The following unary operators are available: <code class="code">+</code>, <code class="code">-</code>.

Some internal variables can be used to store and load intermediary

results. They can be accessed using the <code class="code">ld</code> and <code class="code">st</code>

functions with an index argument varying from 0 to 9 to specify which

internal variable to access.

The following functions are available:

<dl class="table">

<dt><samp class="option">abs(x)</samp></dt>

<dd> Compute absolute value of <var class="var">x</var>.

</dd>

<dt><samp class="option">acos(x)</samp></dt>

<dd> Compute arccosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">asin(x)</samp></dt>

<dd> Compute arcsine of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan(x)</samp></dt>

<dd> Compute arctangent of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan2(y, x)</samp></dt>

<dd> Compute principal value of the arc tangent of <var class="var">y</var>/<var class="var">x</var>.

</dd>

<dt><samp class="option">between(x, min, max)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">min</var> and lesser than or

equal to <var class="var">max</var>, 0 otherwise.

</dd>

<dt><samp class="option">bitand(x, y)</samp></dt>

<dt><samp class="option">bitor(x, y)</samp></dt>

<dd> Compute bitwise and/or operation on <var class="var">x</var> and <var class="var">y</var>.

The results of the evaluation of <var class="var">x</var> and <var class="var">y</var> are converted to

integers before executing the bitwise operation.

Note that both the conversion to integer and the conversion back to

floating point can lose precision. Beware of unexpected results for

large numbers (usually 2^53 and larger).

</dd>

<dt><samp class="option">ceil(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> upwards to the nearest

integer. For example, "ceil(1.5)" is "2.0".

</dd>

<dt><samp class="option">clip(x, min, max)</samp></dt>

<dd> Return the value of <var class="var">x</var> clipped between <var class="var">min</var> and <var class="var">max</var>.

</dd>

<dt><samp class="option">cos(x)</samp></dt>

<dd> Compute cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">cosh(x)</samp></dt>

<dd> Compute hyperbolic cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">eq(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> and <var class="var">y</var> are equivalent, 0 otherwise.

</dd>

<dt><samp class="option">exp(x)</samp></dt>

<dd> Compute exponential of <var class="var">x</var> (with base <code class="code">e</code>, the Euler’s number).

</dd>

<dt><samp class="option">floor(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> downwards to the nearest

integer. For example, "floor(-1.5)" is "-2.0".

</dd>

<dt><samp class="option">gauss(x)</samp></dt>

<dd> Compute Gauss function of <var class="var">x</var>, corresponding to

<code class="code">exp(-x*x/2) / sqrt(2*PI)</code>.

</dd>

<dt><samp class="option">gcd(x, y)</samp></dt>

<dd> Return the greatest common divisor of <var class="var">x</var> and <var class="var">y</var>. If both <var class="var">x</var> and

<var class="var">y</var> are 0 or either or both are less than zero then behavior is undefined.

</dd>

<dt><samp class="option">gt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">gte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">hypot(x, y)</samp></dt>

<dd> This function is similar to the C function with the same name; it returns

"sqrt(<var class="var">x</var>*<var class="var">x</var> + <var class="var">y</var>*<var class="var">y</var>)", the length of the hypotenuse of a

right triangle with sides of length <var class="var">x</var> and <var class="var">y</var>, or the distance of the

point (<var class="var">x</var>, <var class="var">y</var>) from the origin.

</dd>

<dt><samp class="option">if(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the result of

the evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">if(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is non-zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">ifnot(x, y)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the result of the

evaluation of <var class="var">y</var>, return 0 otherwise.

</dd>

<dt><samp class="option">ifnot(x, y, z)</samp></dt>

<dd> Evaluate <var class="var">x</var>, and if the result is zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">isinf(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is +/-INFINITY, 0.0 otherwise.

</dd>

<dt><samp class="option">isnan(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is NAN, 0.0 otherwise.

</dd>

<dt><samp class="option">ld(idx)</samp></dt>

<dd> Load the value of the internal variable with index <var class="var">idx</var>, which was

previously stored with st(<var class="var">idx</var>, <var class="var">expr</var>).

The function returns the loaded value.

</dd>

<dt><samp class="option">lerp(x, y, z)</samp></dt>

<dd> Return linear interpolation between <var class="var">x</var> and <var class="var">y</var> by amount of <var class="var">z</var>.

</dd>

<dt><samp class="option">log(x)</samp></dt>

<dd> Compute natural logarithm of <var class="var">x</var>.

</dd>

<dt><samp class="option">lt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">lte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">max(x, y)</samp></dt>

<dd> Return the maximum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">min(x, y)</samp></dt>

<dd> Return the minimum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">mod(x, y)</samp></dt>

<dd> Compute the remainder of division of <var class="var">x</var> by <var class="var">y</var>.

</dd>

<dt><samp class="option">not(expr)</samp></dt>

<dd> Return 1.0 if <var class="var">expr</var> is zero, 0.0 otherwise.

</dd>

<dt><samp class="option">pow(x, y)</samp></dt>

<dd> Compute the power of <var class="var">x</var> elevated <var class="var">y</var>, it is equivalent to

"(<var class="var">x</var>)^(<var class="var">y</var>)".

</dd>

<dt><samp class="option">print(t)</samp></dt>

<dt><samp class="option">print(t, l)</samp></dt>

<dd> Print the value of expression <var class="var">t</var> with loglevel <var class="var">l</var>. If <var class="var">l</var> is not

specified then a default log level is used.

Return the value of the expression printed.

</dd>

<dt><samp class="option">random(idx)</samp></dt>

<dd> Return a pseudo random value between 0.0 and 1.0. <var class="var">idx</var> is the

index of the internal variable used to save the seed/state, which can be

previously stored with <code class="code">st(idx)</code>.

To initialize the seed, you need to store the seed value as a 64-bit

unsigned integer in the internal variable with index <var class="var">idx</var>.

For example, to store the seed with value <code class="code">42</code> in the internal

variable with index <code class="code">0</code> and print a few random values:

<pre class="example-preformatted">st(0,42); print(random(0)); print(random(0)); print(random(0))

</pre>

</dd> <dt><samp class="option">randomi(idx, min, max)</samp></dt> <dd> Return a pseudo random value in the interval between <var class="var">min</var> and <var class="var">max</var>. <var class="var">idx</var> is the index of the internal variable which will be used to save the seed/state, which can be previously stored with <code class="code">st(idx)</code>. To initialize the seed, you need to store the seed value as a 64-bit unsigned integer in the internal variable with index <var class="var">idx</var>. </dd> <dt><samp class="option">root(expr, max)</samp></dt> <dd> Find an input value for which the function represented by <var class="var">expr</var> with argument <var class="var">ld(0)</var> is 0 in the interval 0..<var class="var">max</var>. The expression in <var class="var">expr</var> must denote a continuous function or the result is undefined. <var class="var">ld(0)</var> is used to represent the function input value, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class="code">ld(0)</code>. When the expression evaluates to 0 then the corresponding input value will be returned. </dd> <dt><samp class="option">round(expr)</samp></dt> <dd> Round the value of expression <var class="var">expr</var> to the nearest integer. For example, "round(1.5)" is "2.0". </dd> <dt><samp class="option">sgn(x)</samp></dt> <dd> Compute sign of <var class="var">x</var>. </dd> <dt><samp class="option">sin(x)</samp></dt> <dd> Compute sine of <var class="var">x</var>. </dd>

<dt><samp class="option">sinh(x)</samp></dt> <dd> Compute hyperbolic sine of <var class="var">x</var>. </dd> <dt><samp class="option">sqrt(expr)</samp></dt> <dd> Compute the square root of <var class="var">expr</var>. This is equivalent to "(<var class="var">expr</var>)^.5". </dd> <dt><samp class="option">squish(x)</samp></dt> <dd> Compute expression <code class="code">1/(1 + exp(4*x))</code>. </dd> <dt><samp class="option">st(idx, expr)</samp></dt> <dd> Store the value of the expression <var class="var">expr</var> in an internal variable. <var class="var">idx</var> specifies the index of the variable where to store the value, and it is a value ranging from 0 to 9. The function returns the value stored in the internal variable. The stored value can be retrieved with <code class="code">ld(var)</code>. Note: variables are currently not shared between expressions. </dd> <dt><samp class="option">tan(x)</samp></dt> <dd> Compute tangent of <var class="var">x</var>. </dd> <dt><samp class="option">tanh(x)</samp></dt> <dd> Compute hyperbolic tangent of <var class="var">x</var>. </dd> <dt><samp class="option">taylor(expr, x)</samp></dt> <dt><samp class="option">taylor(expr, x, idx)</samp></dt> <dd> Evaluate a Taylor series at <var class="var">x</var>, given an expression representing the <code class="code">ld(idx)</code>-th derivative of a function at 0. When the series does not converge the result is undefined. <var class="var">ld(idx)</var> is used to represent the derivative order in <var class="var">expr</var>, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class="code">ld(idx)</code>. If <var class="var">idx</var> is not specified then 0 is assumed. Note, when you have the derivatives at y instead of 0, <code class="code">taylor(expr, x-y)</code> can be used. </dd> <dt><samp class="option">time(0)</samp></dt> <dd> Return the current (wallclock) time in seconds. </dd> <dt><samp class="option">trunc(expr)</samp></dt> <dd> Round the value of expression <var class="var">expr</var> towards zero to the nearest integer. For example, "trunc(-1.5)" is "-1.0". </dd> <dt><samp class="option">while(cond, expr)</samp></dt> <dd> Evaluate expression <var class="var">expr</var> while the expression <var class="var">cond</var> is non-zero, and returns the value of the last <var class="var">expr</var> evaluation, or NAN if <var class="var">cond</var> was always false. </dd> </dl>

The following constants are available: <dl class="table"> <dt><samp class="option">PI</samp></dt> <dd> area of the unit disc, approximately 3.14 </dd> <dt><samp class="option">E</samp></dt> <dd> exp(1) (Euler’s number), approximately 2.718 </dd> <dt><samp class="option">PHI</samp></dt> <dd> golden ratio (1+sqrt(5))/2, approximately 1.618 </dd> </dl>

Assuming that an expression is considered "true" if it has a non-

zero value, note that: <code class=”code”>*</code> works like AND <code class=”code”>+</code> works like OR For example the construct: <pre class=”example-preformatted”>if (A AND B) then C </pre> is equivalent to: <pre class=”example-preformatted”>if(A*B, C) </pre>

In your C code, you can extend the list of unary and binary functions, and define recognized constants, so that they are available for your expressions. The evaluator also recognizes the International System unit prefixes. If ’i’ is appended after the prefix, binary prefixes are used, which are based on powers of 1024 instead of powers of 1000. The ’B’ postfix multiplies the value by 8, and can be appended after a unit prefix or used alone. This allows using for example ’KB’, ’MiB’, ’G’ and ’B’ as number postfix. The list of available International System prefixes follows, with indication of the corresponding powers of 10 and of 2. <dl class=”table”> <dt><samp class=”option”>y</samp></dt> <dd> 10^-24 / 2^-80 </dd> <dt><samp class=”option”>z</samp></dt> <dd> 10^-21 / 2^-70 </dd> <dt><samp class=”option”>a</samp></dt> <dd> 10^-18 / 2^-60 </dd> <dt><samp class=”option”>f</samp></dt> <dd> 10^-15 / 2^-50 </dd> <dt><samp class=”option”>p</samp></dt> <dd> 10^-12 / 2^-40 </dd> <dt><samp class=”option”>n</samp></dt> <dd> 10^-9 / 2^-30 </dd> <dt><samp class=”option”>u</samp></dt> <dd> 10^-6 / 2^-20 </dd> <dt><samp class=”option”>m</samp></dt> <dd> 10^-3 / 2^-10 </dd> <dt><samp class=”option”>c</samp></dt> <dd> 10^-2 </dd> <dt><samp class=”option”>d</samp></dt> <dd> 10^-1 </dd> <dt><samp class=”option”>h</samp></dt> <dd> 10^2 </dd> <dt><samp class=”option”>k</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>K</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>M</samp></dt> <dd> 10^6 / 2^20 </dd> <dt><samp class=”option”>G</samp></dt> <dd> 10^9 / 2^30 </dd> <dt><samp class=”option”>T</samp></dt> <dd> 10^12 / 2^40 </dd> <dt><samp class=”option”>P</samp></dt> <dd> 10^15 / 2^50 </dd> <dt><samp class=”option”>E</samp></dt> <dd> 10^18 / 2^60 </dd> <dt><samp class=”option”>Z</samp></dt> <dd> 10^21 / 2^70 </dd> <dt><samp class=”option”>Y</samp></dt> <dd> 10^24 / 2^80 </dd> </dl>

 <h2 class=”chapter”>6 Codec Options# TOC</h2>

libavcodec provides some generic global options, which can be set on all the encoders and decoders. In addition, each codec may support so-called private options, which are specific for a given codec. Sometimes, a

global option may only affect a specific kind of codec, and may be nonsensical or ignored by another, so you need to be aware of the meaning of the specified options. Also some options are meant only for decoding or encoding. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVCodecContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follow: <dl class="table"> <dt><samp class="option">b <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set bitrate in bits/s. Default value is 200K. </dd> <dt><samp class="option">ab <var class="var">integer</var> (encoding,audio)</samp><
<dd> Set audio bitrate (in bits/s). Default value is 128K. </dd> <dt><samp class="option">bt <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set video bitrate tolerance (in bits/s). In 1-pass mode, bitrate tolerance specifies how far ratecontrol is willing to deviate from the target average bitrate value. This is not related to min/max bitrate. Lowering tolerance too much has an adverse effect on quality. </dd> <dt><samp class="option">flags <var class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></dt> <dd> Set generic flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">mv4</samp>’</dt> <dd> Use four motion vector by macroblock (mpeg4). </dd> <dt>‘<samp class="samp">qpel</samp>’</dt> <dd> Use 1/4 pel motion compensation. </dd> <dt>‘<samp class="samp">loop</samp>’</dt> <dd> Use loop filter. </dd> <dt>‘<samp class="samp">qscale</samp>’</dt> <dd> Use fixed qscale. </dd> <dt>‘<samp class="samp">pass1</samp>’</dt> <dd> Use internal 2pass ratecontrol in first pass mode. </dd> <dt>‘<samp class="samp">pass2</samp>’</dt> <dd> Use internal 2pass ratecontrol in second pass mode. </dd> <dt>‘<samp class="samp">gray</samp>’</dt> <dd> Only decode/encode grayscale. </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> Set error[?] variables during encoding. </dd> <dt>‘<samp class="samp">truncated</samp>’</dt> <dd> Input bitstream might be randomly truncated. </dd> <dt>‘<samp class="samp">drop_changed</samp>’</dt> <dd> Don’t output frames whose parameters differ from first decoded frame in stream. Error AVERROR_INPUT_CHANGED is returned when a frame is dropped. </dd> <dt>‘<samp class="samp">ildct</samp>’</dt> <dd> Use interlaced DCT. </dd> <dt>‘<samp class="samp">low_delay</samp>’<
<dd> Force low delay. </dd> <dt>‘<samp class="samp">global_header</samp>’<
<dd> Place global headers in extradata instead of every keyframe. </dd> <dt>‘<samp class="samp">bitexact</samp>’</dt> <dd> Only write platform-, build- and time-independent data. (except (I)DCT). This ensures that file and data checksums are reproducible

and match between platforms. Its primary use is for regression testing. </dd> <dt>‘<samp class=”samp”>aic</samp>’</dt> <dd> Apply H263 advanced intra coding / mpeg4 ac prediction. </dd> <dt>‘<samp class=”samp”>ilme</samp>’</dt> <dd> Apply interlaced motion estimation. </dd> <dt>‘<samp class=”samp”>cgop</samp>’</dt> <dd> Use closed gop. </dd> <dt>‘<samp class=”samp”>output_corrupt</samp>’</dt> <dd> Output even potentially corrupted frames. </dd> </dl>

</dd>

<dt><samp class=”option”>time_base <var class=”var”>rational number</var></samp></dt>

<dd> Set codec time base.

It is the fundamental unit of time (in seconds) in terms of which

frame timestamps are represented. For fixed-fps content, timebase

should be <code class=”code”>1 / frame_rate</code> and timestamp increments should be

identically 1.

</dd>

<dt><samp class=”option”>g <var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set the group of picture (GOP) size. Default value is 12.

</dd>

<dt><samp class=”option”>ar <var class=”var”>integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set audio sampling rate (in Hz).

</dd>

<dt><samp class=”option”>ac <var class=”var”>integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set number of audio channels.

</dd>

<dt><samp class="option">cutoff <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set cutoff bandwidth. (Supported only by selected encoders, see

their respective documentation sections.)

</dd>

<dt><samp class="option">frame_size <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set audio frame size.

Each submitted frame except the last must contain exactly frame_size

samples per channel. May be 0 when the codec has

CODEC_CAP_VARIABLE_FRAME_SIZE set, in that case the frame size is not

restricted. It is set by some decoders to indicate constant frame

size.

</dd>

<dt><samp class="option">frame_number <var class="var">integer</var></samp></dt>

<dd> Set the frame number.

</dd>

<dt><samp class="option">delay <var class="var">integer</var></samp></dt>

<dt><samp class="option">qcomp <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale compression (VBR). It is used as a constant

in the ratecontrol equation. Recommended range for default rc_eq:

0.0-1.0.

</dd>

<dt><samp class="option">qblur <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale blur (VBR).

</dd>

<dt><samp class="option">qmin <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set min video quantizer scale (VBR). Must be included between -1 and

69, default value is 2.

</dd>

<dt><samp class="option">qmax <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max video quantizer scale (VBR). Must be included between -1 and

1024, default value is 31.

</dd>

<dt><samp class="option">qdiff <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max difference between the quantizer scale (VBR).

</dd>

<dt><samp class="option">bf <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max number of B frames between non-B-frames.

Must be an integer between -1 and 16. 0 means that B-frames are

disabled. If a value of -1 is used, it will choose an automatic value

depending on the encoder.

Default value is 0.

</dd>

<dt><samp class="option">b_qfactor <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set qp factor between P and B frames.

</dd>

<dt><samp class="option">codec_tag <var class="var">integer</var></samp></dt>

<dt><samp class="option">bug <var class="var">flags</var> (decoding,video)</samp></dt>

<dd> Workaround not auto detected encoder bugs.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">autodetect</samp>’</dt>

<dt>‘<samp class="samp">xvid_ilace</samp>’</dt>

<dd> Xvid interlacing bug (autodetected if fourcc==XVIX)

</dd>

<dt>‘<samp class="samp">ump4</samp>’</dt>

<dd> (autodetected if fourcc==UMP4)

</dd>

<dt>‘<samp class="samp">no_padding</samp>’</dt>

<dd> padding bug (autodetected)

</dd>

<dt>‘<samp class="samp">amv</samp>’</dt>

<dt>‘<samp class="samp">qpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">std_qpel</samp>’</dt>

<dd> old standard qpel (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">qpel_chroma2</samp>’</dt>

<dt>‘<samp class="samp">direct_blocksize</samp>’</dt>

<dd> direct-qpel-blocksize bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">edge</samp>’</dt>

<dd> edge padding bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">hpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">dc_clip</samp>’</dt>

<dt>‘<samp class="samp">ms</samp>’</dt>

<dd> Workaround various bugs in microsoft broken decoders.

</dd>

<dt>‘<samp class="samp">trunc</samp>’</dt>

<dd> trancated frames

</dd>

<dt>‘<samp class="samp">h264_dp_nnz</samp>’</dt>

<dd> H.264 nC derivation for partitioned slices, as done by JM rather than by

9.2.1 (autodetected)

</dd>

</dl>

</dd> <dt><samp class="option">strict <var class="var">integer</var> (decoding/encoding,audio,video)</samp></dt> <dd> Specify how strictly to follow the standards. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt> <dd> strictly conform to an older more strict version of the spec or reference software </dd> <dt>‘<samp class="samp">strict</samp>’</dt> <dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt> <dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> <dt><samp class="option">b_qoffset <var class="var">float</var> (encoding,video)</samp></dt> <dd> Set QP offset between P and B frames. </dd> <dt><samp class="option">err_detect <var class="var">flags</var> (decoding,audio,video)</samp></dt> <dd> Set error detection flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt>

<dd> verify embedded CRCs </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> detect bitstream specification deviations </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> detect improper bitstream length </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> abort decoding on minor error detection </dd> <dt>‘<samp class="samp">ignore_err</samp>’</dt> <dd> ignore decoding errors, and continue decoding. This is useful if you want to analyze the content of a video and thus want everything to be decoded no matter what. This option will not result in a video that is pleasing to watch in case of errors. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> consider things that violate the spec and have not been seen in the wild as errors </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> consider all spec non compliancies as errors </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> consider things that a sane encoder should not do as an error </dd> </dl>

</dd> <dt><samp class="option">has_b_frames <var class="var">integer</var></samp></dt> <dt><samp class="option">block_align <var class="var">integer</var></samp></dt> <dt><samp class="option">rc_override_count <var class="var">integer</var></samp></dt> <dt><samp class="option">maxrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set max bitrate tolerance (in bits/s). Requires bufsize to be set. </dd> <dt><samp class="option">minrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set min bitrate tolerance (in bits/s). Most useful in setting up a CBR encode. It is of little use elsewise. </dd> <dt><samp class="option">bufsize <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set ratecontrol buffer size (in bits). </dd> <dt><samp class="option">i_qfactor <var class="var">float</var> (encoding,video)</sam <dd> Set QP factor between P and I frames. </dd> <dt><samp class="option">i_qoffset <var class="var">float</var> (encoding,video)</sam <dd> Set QP offset between P and I frames. </dd> <dt><samp class="option">dct <var class="var">integer</var> (encoding,video)</samp> <dd> Set DCT algorithm. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> autoselect a good one (default) </dd> <dt>‘<samp class="samp">fastint</samp>’</dt> <dd> fast integer </dd> <dt>‘<samp class="samp">int</samp>’</dt> <dd> accurate integer </dd> <dt>‘<samp class="samp">mmx</samp>’</dt> <dt>‘<samp class="samp">altivec</samp>’</dt> <dt>‘<samp class="samp">faan</samp>’</dt> <dd> floating point AAN DCT </dd> </dl>

</dd> <dt><samp class="option">lumi_mask <var class="var">float</var> (encoding,video)</samp></dt> <dd> Compress

bright areas stronger than medium ones. </dd> <dt><samp
class="option">tcplx_mask <var class="var">float</var> (encoding,video)</s
<dd> Set temporal complexity masking. </dd> <dt><samp
class="option">scplx_mask <var class="var">float</var> (encoding,video)</s
<dd> Set spatial complexity masking. </dd> <dt><samp
class="option">p_mask <var class="var">float</var> (encoding,video)</samp
<dd> Set inter masking. </dd> <dt><samp class="option">dark_mask
<var class="var">float</var> (encoding,video)</samp></dt>
<dd> Compress dark areas stronger than medium ones.
</dd> <dt><samp class="option">idct <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Select IDCT implementation. Possible values: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dt>‘<samp class="samp">int</samp>’</dt> <dt>‘<samp
class="samp">simple</samp>’</dt> <dt>‘<samp class="samp">simplemmx</samp>&rsqu
<dt>‘<samp class="samp">simpleauto</samp>’</dt>
<dd> Automatically pick a IDCT compatible with the simple one
 </dd> <dt>‘<samp class="samp">arm</samp>’</dt>
<dt>‘<samp class="samp">altivec</samp>’</dt> <dt>‘<samp
class="samp">sh4</samp>’</dt> <dt>‘<samp class="samp">simplearm</samp>’<
<dt>‘<samp class="samp">simplearmv5te</samp>’</dt>
<dt>‘<samp class="samp">simplearmv6</samp>’</dt>
<dt>‘<samp class="samp">simpleneon</samp>’</dt>
<dt>‘<samp class="samp">xvid</samp>’</dt> <dt>‘<samp
class="samp">faani</samp>’</dt> <dd> floating point AAN
IDCT </dd> </dl>

</dd> <dt><samp class="option">slice_count <var class="var">integer</var></samp></dt>
<dt><samp class="option">ec <var class="var">flags</var> (<em
class="emph">decoding,video)</samp></dt> <dd> Set error
concealment strategy. Possible values: <dl class="table">
<dt>‘<samp class="samp">guess_mvs</samp>’</dt>
<dd> iterative motion vector (MV) search (slow) </dd> <dt>‘<samp
class="samp">deblock</samp>’</dt> <dd> use strong deblock
filter for damaged MBs </dd> <dt>‘<samp class="samp">favor_inter</samp>’</dt>
<dd> favor predicting from the previous frame instead of the current
</dd> </dl>

</dd> <dt><samp class="option">bits_per_coded_sample <var class="var">integer</var></samp></d
<dt><samp class="option">aspect <var class="var">rational number</var>
(encoding,video)</samp></dt> <dd> Set
sample aspect ratio. </dd> <dt><samp class="option">sar <var
class="var">rational number</var> (encoding,video)</samp></dt>
<dd> Set sample aspect ratio. Alias to <var class="var">aspect</var>.
 </dd> <dt><samp class="option">debug <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Print specific debug info. Possible values: <dl

class=”table”> <dt>‘<samp class=”samp”>pict</samp>’</dt>
<dd> picture info </dd> <dt>‘<samp class=”samp”>rc</samp>’</dt>
<dd> rate control </dd> <dt>‘<samp class=”samp”>bitstream</samp>’</dt>
<dt>‘<samp class=”samp”>mb_type</samp>’</dt> <dd> macroblock
(MB) type </dd> <dt>‘<samp class=”samp”>qp</samp>’</dt>
<dd> per-block quantization parameter (QP) </dd> <dt>‘<samp
class=”samp”>dct_coeff</samp>’</dt> <dt>‘<samp
class=”samp”>green_metadata</samp>’</dt> <dd> display
complexity metadata for the upcoming frame, GoP or for a given duration.
 </dd> <dt>‘<samp class=”samp”>skip</samp>’</dt>
<dt>‘<samp class=”samp”>startcode</samp>’</dt> <dt>‘<samp
class=”samp”>er</samp>’</dt> <dd> error recognition
</dd> <dt>‘<samp class=”samp”>mmco</samp>’</dt>
<dd> memory management control operations (H.264) </dd>
<dt>‘<samp class=”samp”>bugs</samp>’</dt> <dt>‘<samp
class=”samp”>buffers</samp>’</dt> <dd> picture buffer alloca-
tions </dd> <dt>‘<samp class=”samp”>thread_ops</samp>’</dt>
<dd> threading operations </dd> <dt>‘<samp class=”samp”>nomc</samp>’</dt
<dd> skip motion compensation </dd> </dl>

</dd> <dt><samp class=”option”>cmp <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
full pel me compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>subcmp <var class=”var”>integer</var>

(encoding,video)</samp></dt> <dd> Set sub pel me compare function. Possible values: <dl class="table"> <dt>‘<samp class="samp">sad</samp>’</dt> <dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp class="samp">sse</samp>’</dt> <dd> sum of squared errors </dd> <dt>‘<samp class="samp">satd</samp>’</dt> <dd> sum of absolute Hadamard transformed differences </dd> <dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum of absolute DCT transformed differences </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> sum of squared quantization errors (avoid, low quality) </dd> <dt>‘<samp class="samp">bit</samp>’</dt> <dd> number of bits needed for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt> <dd> rate distortion optimal, slow </dd> <dt>‘<samp class="samp">zero</samp>’</dt> <dd> 0 </dd> <dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt> <dd> sum of squared vertical differences </dd> <dt>‘<samp class="samp">nsse</samp>’</dt> <dd> noise preserving sum of squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt> <dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt> <dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">mbcmp <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set macroblock compare function. Possible values: <dl class="table"> <dt>‘<samp class="samp">sad</samp>’</dt> <dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp class="samp">sse</samp>’</dt> <dd> sum of squared errors </dd> <dt>‘<samp class="samp">satd</samp>’</dt> <dd> sum of absolute Hadamard transformed differences </dd> <dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum of absolute DCT transformed differences </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> sum of squared quantization errors (avoid, low quality) </dd> <dt>‘<samp class="samp">bit</samp>’</dt> <dd> number of bits needed for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt> <dd> rate distortion optimal, slow </dd> <dt>‘<samp class="samp">zero</samp>’</dt> <dd> 0 </dd> <dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt> <dd> sum of squared vertical differences </dd> <dt>‘<samp class="samp">nsse</samp>’</dt> <dd> noise preserving sum of squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>

<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt> <dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">ildctcmp <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set interlaced dct compare function. Possible values: <dl class="table"> <dt>‘<samp class="samp">sad</samp>’</dt> <dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp class="samp">sse</samp>’</dt> <dd> sum of squared errors </dd> <dt>‘<samp class="samp">satd</samp>’</dt> <dd> sum of absolute Hadamard transformed differences </dd> <dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum of absolute DCT transformed differences </dd> <dt>‘<samp class="samp">psnr</samp>’</dt> <dd> sum of squared quantization errors (avoid, low quality) </dd> <dt>‘<samp class="samp">bit</samp>’</dt> <dd> number of bits needed for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt> <dd> rate distortion optimal, slow </dd> <dt>‘<samp class="samp">zero</samp>’</dt> <dd> 0 </dd> <dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt> <dd> sum of squared vertical differences </dd> <dt>‘<samp class="samp">nsse</samp>’</dt> <dd> noise preserving sum of squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt> <dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt> <dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">dia_size <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set diamond type & size for motion estimation. <dl class="table"> <dt>‘<samp class="samp">(1024, INT_MAX)</samp>’</dt> <dd> full motion estimation(slowest) </dd> <dt>‘<samp class="samp">(768, 1024]</samp>’</dt> <dd> umh motion estimation </dd> <dt>‘<samp class="samp">(512, 768]</samp>’</dt> <dd> hex motion estimation </dd> <dt>‘<samp class="samp">(256, 512]</samp>’</dt> <dd> l2s diamond motion estimation </dd> <dt>‘<samp class="samp">[2,256]</samp>’</dt> <dd> var diamond motion estimation </dd> <dt>‘<samp class="samp">(-1, 2)</samp>’</dt> <dd> small diamond motion estimation </dd> <dt>‘<samp class="samp">-1</samp>’</dt> <dd> funny diamond motion estimation </dd> <dt>‘<samp class="samp">(INT_MIN, -1)</samp>’</dt> <dd> sab dia-

mond motion estimation </dd> </dl>

</dd> <dt><samp class="option">last_pred <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
amount of motion predictors from the previous frame. </dd>
<dt><samp class="option">precmp <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set pre
motion estimation compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">pre_dia_size <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
diamond type & size for motion estimation pre-pass. </dd>
<dt><samp class="option">subq <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set sub pel
motion estimation quality. </dd> <dt><samp class="option">me_range
<var class="var">integer</var> (encoding,video)</samp></dt>
<dd> Set limit motion vectors range (1023 for DivX player). </dd>
<dt><samp class="option">global_quality <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">slice_flags <var class="var">integer</var></samp></dt>
<dt><samp class="option">mbd <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set mac-
roblock decision algorithm (high quality mode). Possible values:
<dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt>
<dd> use mbcmp (default) </dd> <dt>‘<samp class="samp">bits</samp>’</dt>

<dd> use fewest bits </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> use best rate distortion </dd> </dl>

</dd> <dt><samp class="option">rc_init_occupancy <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
number of bits which should be loaded into the rc buffer before de-
coding starts. </dd> <dt><samp class="option">flags2 <var
class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></d
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">fast</samp>’</dt> <dd> Allow non spec compli-
ant speedup tricks. </dd> <dt>‘<samp class="samp">noout</samp>’</dt>
<dd> Skip bitstream encoding. </dd> <dt>‘<samp
class="samp">ignorecrop</samp>’</dt> <dd> Ignore cropping
information from sps. </dd> <dt>‘<samp class="samp">local_header</samp>’</dt>
<dd> Place global headers at every keyframe instead of in extradata.
</dd> <dt>‘<samp class="samp">chunks</samp>’</dt>
<dd> Frame data might be split into multiple chunks. </dd>
<dt>‘<samp class="samp">showall</samp>’</dt> <dd> Show
all frames before the first keyframe. </dd> <dt>‘<samp
class="samp">export_mvs</samp>’</dt> <dd> Export motion
vectors into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">skip_manual</samp>’</dt>
<dd> Do not skip samples and export skip information as frame side data.
</dd> <dt>‘<samp class="samp">ass_ro_flush_noop</samp>’</dt>
<dd> Do not reset ASS ReadOrder field on flush. </dd>
<dt>‘<samp class="samp">icc_profiles</samp>’</dt>
<dd> Generate/parse embedded ICC profiles from/to colorimetry tags.
</dd> <dt>‘<samp class="samp">fixed_frame_size</samp>’</dt>
<dd> Force audio encoders to use a fixed frame size. </dd> </dl>

</dd> <dt><samp class="option">export_side_data <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">mvs</samp>’</dt> <dd> Export motion vectors
into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">prft</samp>’</dt>
<dd> Export encoder Producer Reference Time into packet side-data (see
<code class="code">AV_PKT_DATA_PRFT</code>) for codecs that sup-
port it. </dd> <dt>‘<samp class="samp">venc_params</samp>’</dt>
<dd> Export video encoding parameters through frame side data (see
<code class="code">AV_FRAME_DATA_VIDEO_ENC_PARAMS</code>)
for codecs that support it. At present, those are H.264 and VP9. </dd>
<dt>‘<samp class="samp">film_grain</samp>’</dt>
<dd> Export film grain parameters through frame side data (see <code
class="code">AV_FRAME_DATA_FILM_GRAIN_PARAMS</code>).

Supported at present by AV1 decoders. </dd> <dt>‘<samp class=”samp”>enhancements</samp>’</dt> <dd> Export picture enhancement metadata through frame side data, e.g. LCEVC (see <code class=”code”>AV_FRAME_DATA_LCEVC</code>). </dd> </dl>

</dd> <dt><samp class=”option”>threads <var class=”var”>integer</var> (decoding/encoding,video)</samp></dt> <dd> Set the number of threads to be used, in case the selected codec implementation supports multi-threading. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto, 0</samp>’</dt> <dd> automatically select the number of threads to set </dd> </dl>

Default value is ‘<samp class=”samp”>auto</samp>’. </dd> <dt><samp class=”option”>dc <var class=”var”>integer</var> (encoding,video)</samp></dt> <dd> Set intra_dc_precision. </dd> <dt><samp class=”option”>nssew <var class=”var”>integer</var> (encoding,video)</samp></dt> <dd> Set nsse weight. </dd> <dt><samp class=”option”>skip_top <var class=”var”>integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the top which are skipped. </dd> <dt><samp class=”option”>skip_bottom <var class=”var”>integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the bottom which are skipped. </dd> <dt><samp class=”option”>profile <var class=”var”>integer</var> (encoding,audio,video)</samp></dt> <dd> Set encoder codec profile. Default value is ‘<samp class=”samp”>unknown</samp>’. Encoder specific profiles are documented in the relevant encoder documentation. </dd> <dt><samp class=”option”>level <var class=”var”>integer</var> (encoding,audio,video)</samp></dt> <dd> Set the encoder level. This level depends on the specific codec, and might correspond to the profile level. It is set by default to ‘<samp class=”samp”>unknown</samp>’. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>lowres <var class=”var”>integer</var> (decoding,audio,video)</samp></dt> <dd> Decode at 1= 1/2, 2=1/4, 3=1/8 resolutions. </dd> <dt><samp class=”option”>mblmin <var class=”var”>integer</var> (encoding,video)</sam
<dd> Set min macroblock lagrange factor (VBR). </dd> <dt><samp class=”option”>mblmax <var class=”var”>integer</var> (encoding,video)</samp></dt> <dd> Set max macroblock lagrange factor (VBR). </dd> <dt><samp class=”option”>skip_loop_filter <var class=”var”>integer</var> (decoding,video)</samp></dt> <dt><samp class=”option”>skip_idct <var class=”var”>integer</var> (decoding,video)</samp></dt>

<dt><samp class="option">skip_pred <var class="var">integer</var>
(decoding,video)</samp></dt> <dt><samp
class="option">skip_frame <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dd> Make
decoder discard processing depending on the frame type selected by the
option value. <samp class="option">skip_loop_filter</samp>
skips frame loop filtering, <samp class="option">skip_idct</samp> skips
frame IDCT/dequantization, <samp class="option">skip_pred</samp>
skips prediction (intra prediction and motion compensation), <samp
class="option">skip_frame</samp> skips decoding. With both
<samp class="option">skip_pred</samp> and <samp class="option">skip_idct</samp>
set to ‘<samp class="samp">all</samp>’ the decoder may skip
all pixel operations and output frames with undefined pixels but valid metadata,
such as timestamps and frame types. Possible values: <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> Discard no frame. </dd> <dt>‘<samp class="samp">default</samp>’</dt>
<dd> Discard useless frames like 0-sized frames. </dd>
<dt>‘<samp class="samp">noref</samp>’</dt> <dd> Discard
all non-reference frames. </dd> <dt>‘<samp class="samp">bidir</samp>’</dt>
<dd> Discard all bidirectional frames. </dd> <dt>‘<samp
class="samp">nokey</samp>’</dt> <dd> Discard all frames ex-
cepts keyframes. </dd> <dt>‘<samp class="samp">nointra</samp>’</dt>
<dd> Discard all frames except I frames. </dd> <dt>‘<samp
class="samp">all</samp>’</dt> <dd> Discard all frames.
</dd> </dl>

Default value is ‘<samp class="samp">default</samp>’.
 </dd> <dt><samp class="option">bidir_refine <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Refine
the two motion vectors used in bidirectional macroblocks. </dd>
<dt><samp class="option">keyint_min <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
minimum interval between IDR-frames. </dd> <dt><samp
class="option">refs <var class="var">integer</var> (encoding,video)</samp>
<dd> Set reference frames to consider for motion compensation.
</dd> <dt><samp class="option">trellis <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dd> Set
rate-distortion optimal quantization. </dd> <dt><samp class="option">mv0_threshold
<var class="var">integer</var> (encoding,video)</samp></dt>
<dt><samp class="option">compression_level <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">bits_per_raw_sample <var class="var">integer</var></samp></dt>
<dt><samp class="option">channel_layout <var class="var">integer</var>
(decoding/encoding,audio)</samp></dt>
<dd> See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-
layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax. </dd> <dt><samp class="option">rc_max_vbv_use
<var class="var">float</var> (encoding,video)</samp></dt>
<dt><samp class="option">rc_min_vbv_use <var class="var">float</var>
(encoding,video)</samp></dt> <dt><samp
class="option">color_primaries <var class="var">integer</var> (<em
class="emph">decoding/encoding,video)</samp></dt> <dd> Possible
values: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> Film </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt>
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">smpte431</samp>’</d
<dd> SMPTE 431-2 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt>
<dd> SMPTE 432-1 </dd> <dt>‘<samp class="samp">jedec-
p22</samp>’</dt> <dd> JEDEC P22 </dd> </dl>

</dd> <dt><samp class="option">color_trc <var class="var">integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dd> BT.709 </dd>
<dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dt>‘<samp class="samp">log100</samp>’</dt> <dd> Log
</dd> <dt>‘<samp class="samp">log_sqrt</samp>’</dt>
<dt>‘<samp class="samp">log316</samp>’</dt> <dd> Log
square root </dd> <dt>‘<samp class="samp">iec61966_2_4</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-4</samp>’</dt>
<dd> IEC 61966-2-4 </dd> <dt>‘<samp class="samp">bt1361</samp>’</dt>
<dt>‘<samp class="samp">bt1361e</samp>’</dt> <dd> BT.1361
</dd> <dt>‘<samp class="samp">iec61966_2_1</samp>’</dt>
<dt>‘<samp class="samp">iec61966-2-1</samp>’</dt>
<dd> IEC 61966-2-1 </dd> <dt>‘<samp class="samp">bt2020_10</samp>’</dt>
<dt>‘<samp class="samp">bt2020_10bit</samp>’</dt>
<dd> BT.2020 - 10 bit </dd> <dt>‘<samp class="samp">bt2020_12</samp>’</d
<dt>‘<samp class="samp">bt2020_12bit</samp>’</dt>
<dd> BT.2020 - 12 bit </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt
<dd> SMPTE ST 2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt
<dt>‘<samp class="samp">smpte428_1</samp>’</dt>

<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>
</dl>

</dd> <dt><samp class="option">colorspace <var class="var">integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> RGB </dd>
<dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE 240 M </dd> <dt>‘<samp class="samp">ycocg</samp>’</dt>
<dd> YCOCG </dd> <dt>‘<samp class="samp">bt2020nc</samp>’</dt>
<dt>‘<samp class="samp">bt2020_ncl</samp>’</dt>
<dd> BT.2020 NCL </dd> <dt>‘<samp class="samp">bt2020c</samp>’</dt>
<dt>‘<samp class="samp">bt2020_cl</samp>’</dt>
<dd> BT.2020 CL </dd> <dt>‘<samp class="samp">smpte2085</samp>’</dt>
<dd> SMPTE 2085 </dd> <dt>‘<samp class="samp">chroma-derived-nc</samp>’</dt> <dd> Chroma-derived NCL </dd>
<dt>‘<samp class="samp">chroma-derived-c</samp>’</dt>
<dd> Chroma-derived CL </dd> <dt>‘<samp class="samp">ictcp</samp>’</dt>
<dd> ICtCp </dd> </dl>

</dd> <dt><samp class="option">color_range <var class="var">integer</var> (decoding/encoding,video)</samp></dt>
<dd> If used as input parameter, it serves as a hint to the decoder, which color_range the input has. Possible values: <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt>
<dt>‘<samp class="samp">mpeg</samp>’</dt> <dt>‘<samp class="samp">limited</samp>’</dt> <dd> MPEG (219*2^(n-8)) </dd> <dt>‘<samp class="samp">pc</samp>’</dt>
<dt>‘<samp class="samp">jpeg</samp>’</dt> <dt>‘<samp class="samp">full</samp>’</dt> <dd> JPEG (2^n-1) </dd>
</dl>

</dd> <dt><samp class="option">chroma_sample_location <var class="var">integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt>
<dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt>
<dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">alpha_mode <var class="var">integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class="table"> <dt>‘<samp

class=”samp”>premultiplied</samp>’</dt> <dt>‘<samp class=”samp”>straight</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>log_level_offset <var class=”var”>integer</var></samp></dt> <dd> Set the log level offset. </dd> <dt><samp class=”option”>slices <var class=”var”>integer</var> (encoding,video)</samp></dt> <dd> Number of slices, used in parallelized encoding. </dd> <dt><samp class=”option”>thread_type <var class=”var”>flags</var> (decoding/encoding,video)</samp></dt> <dd> Select which multithreading methods to use. Use of ‘<samp class=”samp”>frame</samp>’ will increase decoding delay by one frame per thread, so clients which cannot provide future frames should not use it. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>slice</samp>’</dt> <dd> Decode more than one part of a single frame at once. Multithreading using slices works only when the video was encoded with slices. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Decode more than one frame at once. </dd> </dl>

Default value is ‘<samp class=”samp”>slice+frame</samp>’. </dd> <dt><samp class=”option”>audio_service_type <var class=”var”>integer</var> (encoding,audio)</samp></dt> <dd> Set audio service type. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>ma</samp>’</dt> <dd> Main Audio Service </dd> <dt>‘<samp class=”samp”>ef</samp>’</dt> <dd> Effects </dd> <dt>‘<samp class=”samp”>vi</samp>’</dt> <dd> Visually Impaired </dd> <dt>‘<samp class=”samp”>hi</samp>’</dt> <dd> Hearing Impaired </dd> <dt>‘<samp class=”samp”>di</samp>’</dt> <dd> Dialogue </dd> <dt>‘<samp class=”samp”>co</samp>’</dt> <dd> Commentary </dd> <dt>‘<samp class=”samp”>em</samp>’</dt> <dd> Emergency </dd> <dt>‘<samp class=”samp”>vo</samp>’</dt> <dd> Voice Over </dd> <dt>‘<samp class=”samp”>ka</samp>’</dt> <dd> Karaoke </dd> </dl>

</dd> <dt><samp class=”option”>request_sample_fmt <var class=”var”>sample_fmt</var> (decoding,audio)</samp></dt> <dd> Set sample format audio decoders should prefer. Default value is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>pkt_timebase <var class=”var”>rational number</var></samp></dt> <dt><samp class=”option”>sub_charenc <var class=”var”>encoding</var> (decoding,subtitles)</samp></dt> <dd> Set the input subtitles character encoding. </dd> <dt><samp class=”option”>field_order <var class=”var”>field_order</var> (video)</samp></dt> <dd> Set/override the field order of the video. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>progressive</samp>’</dt> <dd> Progressive video </dd> <dt>‘<samp class=”samp”>tt</samp>’</dt>

<dd> Interlaced video, top field coded and displayed first </dd> <dt>‘<samp class="samp">bb</samp>’</dt> <dd> Interlaced video, bottom field coded and displayed first </dd> <dt>‘<samp class="samp">tb</samp>’</dt> <dd> Interlaced video, top coded first, bottom displayed first </dd> <dt>‘<samp class="samp">bt</samp>’</dt> <dd> Interlaced video, bottom coded first, top displayed first </dd> </dl>

</dd> <dt><samp class="option">skip_alpha <var class="var">bool</var> (decoding,video)</samp></dt> <dd> Set to 1 to disable processing alpha (transparency). This works like the ‘<samp class="samp">gray</samp>’ flag in the <samp class="option">flags</samp> option which skips chroma information instead of alpha. Default is 0. </dd> <dt><samp class="option">codec_whitelist <var class="var">list</var> (input)</samp></dt> <dd> "," separated list of allowed decoders. By default all are allowed. </dd> <dt><samp class="option">dump_separator <var class="var">string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class="example-preformatted">ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class="option">max_pixels <var class="var">integer</var> (decoding/encoding,video)</samp></dt> <dd> Maximum number of pixels per image. This value can be used to avoid out of memory failures due to large images. </dd> <dt><samp class="option">apply_cropping <var class="var">bool</var> (decoding,video)</samp></dt> <dd> Enable cropping if cropping parameters are multiples of the required alignment for the left and top parameters. If the alignment is not met the cropping will be partially applied to maintain alignment. Default is 1 (enabled). Note: The required alignment depends on if <code class="code">AV_CODEC_FLAG_UNALIGNED</code> is set and the CPU. <code class="code">AV_CODEC_FLAG_UNALIGNED</code> cannot be changed from the command line. Also hardware decoders will not apply left/top Cropping. </dd> <dt><samp class="option">lcevc_dec_params <var class="var">string</var> (decoding,video)</samp></dt> <dd> A :-separated list of key=value parameters passed to liblcevc_dec to initialize it. </dd> </dl>

 <h2 class="chapter">7 Decoders# TOC</h2>

Decoders are configured elements in FFmpeg which allow the decoding

of multimedia streams. When you configure your FFmpeg build, all the supported native decoders are enabled by default. Decoders requiring an external library must be enabled manually via the corresponding <code class="code">--enable-lib</code> option. You can list all available decoders using the configure option <code class="code">--list-decoders</code>. You can disable all the decoders with the configure option <code class="code">--disable-decoders</code> and selectively enable / disable single decoders with the options <code class="code">--enable-decoder=<var class="var">DECODER</var></code> / <code class="code">--disable-decoder=<var class="var">DECODER</var></code>. The option <code class="code">-decoders</code> of the ff* tools will display the list of enabled decoders.

 <h2 class="chapter">8 Video Decoders# TOC</h2>

A description of some of the currently available video decoders follows. <h3 class="section">8.1 av1# TOC</h3>

AOMedia Video 1 (AV1) decoder. <h4 class="subsection">8.1.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">operating_point</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Default is 0. </dd> </dl>

 <h3 class="section">8.2 hevc# TOC</h3> HEVC (AKA ITU-T H.265 or ISO/IEC 23008-2) decoder. The decoder supports MV-HEVC multiview streams with at most two views. Views to be output are selected by supplying a list of view IDs to the decoder (the <samp class="option">view_ids</samp> option). This option may be set either statically before decoder init, or from the <code class="code">get_format()</code> callback - useful for the case when the view count or IDs change dynamically during decoding. Only the base layer is decoded by default. Note that if you are using the <code class="code">ffmpeg</code> CLI tool, you should be using view specifiers as documented in its manual, rather than the options documented here. <h4 class="subsection">8.2.1 Options# <a class="anchor hidden-xs"href="#toc-Options-2"

aria-hidden=”true”>TOC</h4>

<dl class=”table”> <dt><samp class=”option”>view_ids (MV-HEVC)</samp></dt> <dd> Specify a list of view IDs that should be output. This option can also be set to a single ’-1’, which will cause all views defined in the VPS to be decoded and output. </dd> <dt><samp class=”option”>view_ids_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view IDs available in the active VPS. The array is empty for single-layer video. The value of this option is guaranteed to be accurate when read from the <code class=”code”>get_format()</code> callback. It may also be set at other times (e.g. after opening the decoder), but the value is informational only and may be incorrect (e.g. when the stream contains multiple distinct VPS NALUs). </dd> <dt><samp class=”option”>view_pos_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view positions (left, right, or unspecified) available in the active VPS, as <code class=”code”>AVStereo3DView</code> values. When the array is available, its elements apply to the corresponding elements of <samp class=”option”>view_ids_available</samp>, i.e. <code class=”code”>view_pos_available[i]</code> contains the position of view with ID <code class=”code”>view_ids_available[i]</code>. Same validity restrictions as for <samp class=”option”>view_ids_available</samp> apply to this option. </dd> </dl>

 <h3 class=”section”>8.3 rawvideo# TOC</h3>

Raw video decoder. This decoder decodes rawvideo streams. <h4 class=”subsection”>8.3.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>top <var class=”var”>top_field_first</var></samp></dt> <dd> Specify the assumed field type of the input video. <dl class=”table”> <dt><samp class=”option”>-1</samp></dt> <dd> the video is assumed to be progressive (default) </dd> <dt><samp class=”option”>0</samp></dt> <dd> bottom-field-first is assumed </dd> <dt><samp class=”option”>1</samp></dt> <dd> top-field-first is assumed </dd> </dl>

</dd> </dl>

 <h3 class=”section”>8.4 libastcenc# TOC</h3>

ARM ASTC Encoder wrapper for decoding ASTC textures. This decoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, installed as <code class="code">astcenc/astcenc.h</code> next to a library that <code class="code">-lastcenc</code> finds. Pass <code class="code">--enable-version3</code> together with <code class="code">--enable-libastcenc</code> to configure. The <samp class="file">.astc</samp> header does not record the intended decoding profile. Individual blocks identify their endpoint encodings, but this does not establish the sRGB versus linear interpretation. The decoder therefore uses the container metadata or <samp class="option">dec_profile</samp>: a KTX sRGB texture is decoded as <code class="code">ldr-srgb</code>, while a KTX texture in the linear format is sampled with HDR precision (<code class="code">hdr-ldr-a</code>, half-float output) because the linear GL internal format says nothing about LDR. A raw <code class="code">.astc</code> file carries no profile at all, so it falls back to <code class="code">ldr-srgb</code>; use <samp class="option">dec_profile</samp> for a raw HDR image: <pre class="example-preformatted">ffmpeg -dec_profile hdr-ldr-a -i input.astc output.png </pre>

 <h4 class="subsection">8.4.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">dec_profile <var class="var">int</var></samp></dt> <dd> Color profile to decode with, overriding the container metadata. It must match the profile the texture was encoded with: <code class="code">astcenc</code> substitutes a fixed magenta error color for blocks whose endpoints do not fit the selected profile. The values are <code class="code">ldr</code>, <code class="code">ldr-srgb</code>, <code class="code">hdr-ldr-a</code> and <code class="code">hdr</code>. Without this option the profile published by the container is used; a raw <samp class="file">.astc</samp> stream, which publishes none, falls back to <code class="code">ldr-srgb</code>. </dd> </dl>

 <h3 class="section">8.5 libdav1d# TOC</h3>

dav1d AV1 decoder. libdav1d allows libavcodec to decode the AOMedia Video 1 (AV1) codec. Requires the presence of the libdav1d headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libdav1d</code>. <h4 class="subsection">8.5.1 Options# <a class="anchor hidden-xs"href="#toc-Options-5"

aria-hidden=”true”>TOC</h4>

The following options are supported by the libdav1d wrapper. <dl class=”table”> <dt><samp class=”option”>max_frame_delay</samp></dt> <dd> Set max amount of frames the decoder may buffer internally. The default value is 0 (autodetect). </dd> <dt><samp class=”option”>filmgrain</samp></dt> <dd> Apply film grain to the decoded video if present in the bitstream. Defaults to the internal default of the library. This option is deprecated and will be removed in the future. See the global option <code class=”code”>export_side_data</code> to export Film Grain parameters instead of applying it. </dd> <dt><samp class=”option”>oppoint</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Defaults to the internal default of the library. </dd> <dt><samp class=”option”>alllayers</samp></dt> <dd> Output all spatial layers of a scalable AV1 bitstream. The default value is false. </dd> </dl>

 <h3 class=”section”>8.6 libdavs2# TOC</h3>

AVS2-P2/IEEE1857.4 video decoder wrapper. This decoder allows libavcodec to decode AVS2 streams with davs2 library.

 <h3 class=”section”>8.7 libuavs3d# TOC</h3>

AVS3-P2/IEEE1857.10 video decoder. libuavs3d allows libavcodec to decode AVS3 streams. Requires the presence of the libuavs3d headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libuavs3d</code>. <h4 class=”subsection”>8.7.1 Options# TOC</h4>

The following option is supported by the libuavs3d wrapper. <dl class=”table”> <dt><samp class=”option”>frame_threads</samp></dt> <dd> Set amount of frame threads to use during decoding. The default value is 0 (autodetect). </dd> </dl>

 <h3 class=”section”>8.8 libxevd# TOC</h3>

eXtra-fast Essential Video Decoder (XEVD) MPEG-5 EVC decoder wrapper. This decoder requires the presence of the libxevd headers

and library during configuration. You need to explicitly configure the build with <samp class="option">--enable-libxevd</samp>. The xevd project website is at <a class="url" href="https://github.com/mpeg5/xevd">https://github.com/mpeg5/xevd
  <h4 class="subsection">8.8.1 Options# TOC</h4>

The following options are supported by the libxevd wrapper. The xevd-equivalent options or values are listed in parentheses for easy migration. To get a more accurate and extensive documentation of the libxevd options, invoke the command <code class="code">xevd_app --help</code> or consult the libxevd documentation. <dl class="table"> <dt><samp class="option">threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class="section">8.9 QSV Decoders# TOC</h3>

The family of Intel QuickSync Video decoders (VC1, MPEG-2, H.264, HEVC, JPEG/MJPEG, VP8, VP9, AV1, VVC). <h4 class="subsection">8.9.1 Common Options# TOC</h4>

The following options are supported by all qsv decoders. <dl class="table"> <dt><samp class="option"><var class="var">async_depth</var></samp></dt> <dd> Internal parallelization depth, the higher the value the higher the latency. </dd> <dt><samp class="option"><var class="var">gpu_copy</var></samp></dt> <dd> A GPU-accelerated copy between video and system memory <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dt>‘<samp class="samp">on</samp>’</dt> <dt>‘<samp class="samp">off</samp>’</dt> </dl>

</dd> </dl>

 <h4 class="subsection">8.9.2 HEVC Options# TOC</h4> Extra options for hevc_qsv. <dl class="table"> <dt><samp class="option"><var class="var">load_plugin</var></samp></dt> <dd> A user plugin to load in an internal session <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">hevc_sw</samp>’<
<dt>‘<samp class="samp">hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugins</var></samp></dt> <dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session </dd> </dl>

 <h3 class="section">8.10 v210# TOC</h3>

Uncompressed 4:2:2 10-bit decoder. <h4 class="subsection">8.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">custom_stride</samp></dt> <dd> Set the line size of the v210 data in bytes. The default value is 0 (autodetect). You can use the special -1 value for a strideless v210 as seen in BOXX files. </dd> </dl>

 <h2 class="chapter">9 Audio Decoders# TOC</h2>

A description of some of the currently available audio decoders follows. <h3 class="section">9.1 ac3# TOC</h3>

AC-3 audio decoder. This decoder implements part of ATSC A/52:2010 and ETSI TS 102 366, as well as the undocumented RealAudio 3 (a.k.a. dnet). <h4 class="subsection">9.1.1 AC-3 Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-drc_scale <var class="var">value</var></samp></dt> <dd> Dynamic Range Scale Factor. The factor to apply to dynamic range values from the AC-3 stream. This factor is applied exponentially. The default value is 1. There are 3 notable scale factor ranges: <dl class="table"> <dt><samp class="option">drc_scale == 0</samp></dt> <dd> DRC disabled. Produces full range audio. </dd> <dt><samp class="option">0 < drc_scale <= 1</samp></dt> <dd> DRC enabled. Applies a fraction of the stream DRC value. Audio reproduction is between full range and full compression. </dd> <dt><samp class="option">drc_scale > 1</samp></dt> <dd> DRC enabled. Applies drc_scale asymmetrically. Loud sounds are fully compressed. Soft sounds are enhanced. </dd> </dl>

</dd> </dl>

 <h3 class="section">9.2 dca# TOC</h3>

DCA (DTS Coherent Acoustics) audio decoder, including the DTS-HD extensions. <h4 class="subsection">9.2.1 DCA Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-downmix <var class="var">channel_layout</var></samp
<dd> Request a smaller channel layout from the decoder: ‘<samp class="samp">stereo</samp>’, ‘<samp class="samp">5.0(side)</samp>’, ‘<samp class="samp">5.1(side)</samp>’ or ‘<samp class="samp">7.1</samp>’. A DTS:X stream in the standard presentation decodes to 7.1.4 by default. Its four height channels are carried in an extension at the end of each lossless (XLL) frame, and the encoder also folds them into the 7.1 bed, so the decoder removes them from the bed when it outputs them. Requesting ‘<samp class="samp">7.1</samp>’ outputs the bed as coded instead, with the heights folded in, and so does every smaller layout. A ‘<samp class="samp">7.1</samp>’ request has no effect on other streams. </dd> </dl>

 <h3 class="section">9.3 flac# TOC</h3>

FLAC audio decoder. This decoder aims to implement the complete FLAC specification from Xiph. <h4 class="subsection">9.3.1 FLAC Decoder options# TOC</h4>

<dl class="table"> <dt><samp class="option">-use_buggy_lpc</samp></dt>
<dd> The lavc FLAC encoder used to produce buggy streams with high lpc values (like the default value). This option makes it possible to decode such streams correctly by using lavc’s old buggy lpc logic for decoding. </dd> </dl>

 <h3 class="section">9.4 ffwavesynth# TOC</h3>

Internal wave synthesizer. This decoder generates wave patterns according to predefined sequences. Its use is purely inter-

nal and the format of the data it accepts is not publicly documented. <h3 class="section">9.5 libgsm# TOC</h3>

libgsm decoder wrapper. libgsm allows libavcodec to decode the GSM full rate audio codec. Requires the presence of the libgsm headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libgsm</code>. This decoder supports both the ordinary GSM and the Microsoft variant. <h3 class="section">9.6 libilbc# TOC</h3>

libilbc decoder wrapper. libilbc allows libavcodec to decode the Internet Low Bitrate Codec (iLBC) audio codec. Requires the presence of the libilbc headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libilbc</code>. <h4 class="subsection">9.6.1 Options# TOC</h4>

The following option is supported by the libilbc wrapper. <dl class="table"> <dt><samp class="option">enhance</samp></dt> <dd> Enable the enhancement of the decoded audio when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class="section">9.7 libmpeghdec# TOC</h3>

libmpeghdec decoder wrapper. libmpeghdec allows libmpeghdec to decode the MPEG-H 3D audio codec. Requires the presence of the libmpeghdec headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libmpeghdec --enable-nonfree</code>. <h3 class="section">9.8 libopencore-amrnb# TOC</h3>

libopencore-amrnb decoder wrapper. libopencore-amrnb allows libavcodec to decode the Adaptive Multi-Rate Narrowband audio codec. Using it requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class="code">-

-enable-libopencore-amrnb</code>. An FFmpeg native decoder for AMR-NB exists, so users can decode AMR-NB without this library. <h3 class="section">9.9 libopencore-amrwb# TOC</h

libopencore-amrwb decoder wrapper. libopencore-amrwb allows libavcodec to decode the Adaptive Multi-Rate Wideband audio codec. Using it requires the presence of the libopencore-amrwb headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopencore-amrwb</code>. An FFmpeg native decoder for AMR-WB exists, so users can decode AMR-WB without this library. <h3 class="section">9.10 libopus# TOC</h3>

libopus decoder wrapper. libopus allows libavcodec to decode the Opus Interactive Audio Codec. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libopus</code>. An FFmpeg native decoder for Opus exists, so users can decode Opus without this library. <h3 class="section">9.11 pcm_dvda# TOC</h3>

PCM DVD-Audio decoder. Decodes the LPCM variant stored in DVD-Audio AOB files. Unlike DVD-Video LPCM it supports sample rates up to 192 kHz and allows splitting the channels into two groups with different quantization.

 <h2 class="chapter">10 Subtitles Decoders# TOC</h2>

 <h3 class="section">10.1 libaribb24# TOC</h3>

ARIB STD-B24 caption decoder. Implements profiles A and C of the ARIB STD-B24 standard. <h4 class="subsection">10.1.1 libaribb24 Decoder Options# <a

class=”anchor hidden-xs”href=”#toc-libaribb24-Decoder-Options” aria-hidden=”true”>TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-aribb24-base-path <var class=”var”>path</var></samp></dt> <dd> Sets the base path for the libaribb24 library. This is utilized for reading of configuration files (for custom unicode conversions), and for dumping of non-text symbols as images under that location. Unset by default. </dd> <dt><samp class=”option”>-aribb24-skip-ruby-text <var class=”var”>boolean</var></samp></dt> <dd> Tells the decoder wrapper to skip text blocks that contain half-height ruby text. Enabled by default. </dd> </dl>

 <h3 class=”section”>10.2 libaribcaption# TOC</h3>

Yet another ARIB STD-B24 caption decoder using external libaribcaption library. Implements profiles A and C of the Japanese ARIB STD-B24 standard, Brazilian ABNT NBR 15606-1, and Philippines version of ISDB-T. Requires the presence of the libaribcaption headers and library (https://github.com/xqq/libaribcaption) during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libaribcaption</code>. If both libaribb24 and libaribcaption are enabled, libaribcaption decoder precedes. <h4 class=”subsection”>10.2.1 libaribcaption Decoder Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-sub_type <var class=”var”>subtitle_type</var></samp>
<dd> Specifies the format of the decoded subtitles. <dl class=”table”> <dt>‘<samp class=”samp”>bitmap</samp>’</dt>
<dd> Graphical image. </dd> <dt>‘<samp class=”samp”>ass</samp>’</dt>
<dd> ASS formatted text. </dd> <dt>‘<samp class=”samp”>text</samp>’</dt>
<dd> Simple text based output without formatting. </dd> </dl>

The default is ass as same as libaribb24 decoder. Some present players (e.g., mpv) expect ASS format for ARIB caption. </dd> <dt><samp class=”option”>-caption_encoding <var class=”var”>encoding_scheme</var></samp></dt>
<dd> Specifies the encoding scheme of input subtitle text. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> Automatically detect text encoding (default). </dd>
<dt>‘<samp class=”samp”>jis</samp>’</dt> <dd> 8bit-

char JIS encoding defined in ARIB STD B24. This encoding used in Japan for ISDB captions. </dd> <dt>‘<samp class=”samp”>utf8</samp>’</dt> <dd> UTF-8 encoding defined in ARIB STD B24. This encoding is used in Philippines for ISDB-T captions. </dd> <dt>‘<samp class=”samp”>latin</samp>’</dt> <dd> Latin character encoding defined in ABNT NBR 15606-1. This encoding is used in South America for SBTVD / ISDB-Tb captions. </dd> </dl>

</dd> <dt><samp class=”option”>-font <var class=”var”>font_name[,font_name2,...]</var></samp></
<dd> Specify comma-separated list of font family names to be used for bitmap or ass type subtitle rendering. Only first font name is used for ass type subtitle. If not specified, use internally defined default font family. </dd> <dt><samp class=”option”>-ass_single_rect <var class=”var”>boolean</var></samp></dt> <dd> ARIB STD-B24 specifies that some captions may be displayed at different positions at a time (multi-rectangle subtitle). Since some players (e.g., old mpv) can’t handle multiple ASS rectangles in a single AVSubtitle, or multiple ASS rectangles of indeterminate duration with the same start timestamp, this option can change the behavior so that all the texts are displayed in a single ASS rectangle. The default is <var class=”var”>false</var>. If your player cannot handle AVSubtitles with multiple ASS rectangles properly, set this option to <var class=”var”>true</var> or define <code class=”env”>ASS_SINGLE_RECT=1</code> to change default behavior at compilation. </dd> <dt><samp class=”option”>-force_outline_text <var class=”var”>boolean</var></samp></dt> <dd> Specify whether always render outline text for all characters regardless of the indication by character style. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-outline_width <var class=”var”>number</var> (0.0 - 3.0)</samp></dt> <dd> Specify width for outline text, in dots (relative). The default is <var class=”var”>1.5</var>. </dd> <dt><samp class=”option”>-ignore_background <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore background color rendering. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-ignore_ruby <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore rendering for ruby-like (furigana) characters. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-replace_drcs <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to render replaced DRCS characters as Unicode characters. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_ascii <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) fullwidth alphanumerics with halfwidth alphanumerics. The default is <var class=”var”>true</var>.

 </dd> <dt><samp class="option">-replace_msz_japanese <var class="var">boolean</var></samp></dt> <dd> Specify whether to replace some MSZ (Middle Size; half width) fullwidth japanese special characters with halfwidth ones. The default is <var class="var">true</var>. </dd> <dt><samp class="option">-replace_msz_glyph <var class="var">boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) characters with halfwidth glyphs if the fonts supports it. This option works under FreeType or DirectWrite renderer with Adobe-Japan1 compliant fonts. e.g., IBM Plex Sans JP, Morisawa BIZ UDGothic, Morisawa BIZ UDMincho, Yu Gothic, Yu Mincho, and Meiryo. The default is <var class="var">true</var>. </dd> <dt><samp class="option">-canvas_size <var class="var">image_size</var></samp></dt> <dd> Specify the resolution of the canvas to render subtitles to; usually, this should be frame size of input video. This only applies when <code class="code">-subtitle_type</code> is set to <var class="var">bitmap</var>. The libaribcaption decoder assumes input frame size for bitmap rendering as below: <ol class="enumerate"> PROFILE_A : 1440 x 1080 with SAR (PAR) 4:3 PROFILE_C : 320 x 180 with SAR (PAR) 1:1

If actual frame size of input video does not match above assumption, the rendered captions may be distorted. To make the captions undistorted, add <code class="code">-canvas_size</code> option to specify actual input video size. Note that the <code class="code">-canvas_size</code> option is not required for video with different size but same aspect ratio. In such cases, the caption will be stretched or shrunk to actual video size if <code class="code">-canvas_size</code> option is not specified. If <code class="code">-canvas_size</code> option is specified with different size, the caption will be stretched or shrunk as specified size with calculated SAR. </dd> </dl>

 <h4 class="subsection">10.2.2 libaribcaption decoder usage examples# TOC</h4>

Display MPEG-TS file with ARIB subtitle by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap MPEG.TS </pre>

Display MPEG-TS file with input frame size 1920x1080 by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap -canvas_size 1920x1080 MPEG.TS </pre>

Embed ARIB subtitle in transcoded video:

<pre class="example-preformatted">ffmpeg -sub_type bitmap -i src.m2t -filter_complex "[0:v][0:s]overlay" -vcodec h264 dest.mp4 </pre>

 <h3 class="section">10.3 dvbsub# TOC</h3>

 <h4 class="subsection">10.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">compute_clut</samp></dt> <dd><dl class="table"> <dt><samp class="option">-2</samp></dt> <dd> Compute clut once if no matching CLUT is in the stream. </dd> <dt><samp class="option">-1</samp></dt> <dd> Compute clut if no matching CLUT is in the stream. </dd> <dt><samp class="option">0</samp></dt> <dd> Never compute CLUT </dd> <dt><samp class="option">1</samp></dt> <dd> Always compute CLUT and override the one provided in the stream. </dd> </dl> </dd> <dt><samp class="option">dvb_substream</samp></dt> <dd> Selects the dvb substream, or all substreams if -1 which is default. </dd> </dl>

 <h3 class="section">10.4 dvdsub# TOC</h3>

This codec decodes the bitmap subtitles used in DVDs; the same subtitles can also be found in VobSub file pairs and in some Matroska files. <h4 class="subsection">10.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. When stored in VobSub, the palette is normally specified in the index file; in Matroska, the palette is stored in the codec extra-data in the same format as in VobSub. In DVDs, the palette is stored in the IFO file, and therefore not available when reading from dumped VOB files. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">ifo_palette</samp></dt> <dd> Specify

the IFO file from which the global palette is obtained. (experimental) </dd> <dt><samp class="option">forced_subs_only</samp></dt> <dd> Only decode subtitle entries marked as forced. Some titles have forced and non-forced subtitles in the same track. Setting this flag to <code class="code">1</code> will only keep the forced subtitles. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">10.5 libzvbi-teletext# TOC</h3>

Libzvbi allows libavcodec to decode DVB teletext pages and DVB teletext subtitles. Requires the presence of the libzvbi headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libzvbi</code>. <h4 class="subsection">10.5.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">txt_page</samp></dt> <dd> List of teletext page numbers to decode. Pages that do not match the specified list are dropped. You may use the special <code class="code">*</code> string to match all pages, or <code class="code">subtitle</code> to match all subtitle pages. Default value is *. </dd> <dt><samp class="option">txt_default_region</samp></dt> <dd> Set default character set used for decoding, a value between 0 and 87 (see ETS 300 706, Section 15, Table 32). Default value is -1, which does not override the libzvbi default. This option is needed for some legacy level 1.0 transmissions which cannot signal the proper charset. </dd> <dt><samp class="option">txt_chop_top</samp></dt> <dd> Discards the top teletext line. Default value is 1. </dd> <dt><samp class="option">txt_format</samp></dt> <dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt><samp class="option">bitmap</samp></dt> <dd> The default format, you should use this for teletext pages, because certain graphics and colors cannot be expressed in simple text or even ASS. </dd> <dt><samp class="option">text</samp></dt> <dd> Simple text based output without formatting. </dd> <dt><samp class="option">ass</samp></dt> <dd> Formatted ASS output, subtitle pages and teletext pages are returned in different styles, subtitle pages are stripped down to text, but an effort is made to keep the text alignment and the formatting. </dd> </dl> </dd> <dt><samp class="option">txt_left</samp></dt> <dd> X offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_top</samp></dt> <dd> Y offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_chop_spaces</samp></dt> <dd> Chops lead-

ing and trailing spaces and removes empty lines from the generated text. This option is useful for teletext based subtitles where empty spaces may be present at the start or at the end of the lines or empty lines may be present between the subtitle lines because of double-sized teletext characters. Default value is 1. </dd> <dt><samp class="option">txt_duration</samp></dt> <dd> Sets the display duration of the decoded teletext pages or subtitles in milliseconds. Default value is -1 which means infinity or until the next subtitle event comes. </dd> <dt><samp class="option">txt_transparent</samp></dt> <dd> Force transparent background of the generated teletext bitmaps. Default value is 0 which means an opaque background. </dd> <dt><samp class="option">txt_opacity</samp></dt> <dd> Sets the opacity (0-255) of the teletext background. If <samp class="option">txt_transparent</samp> is not set, it only affects characters between a start box and an end box, typically subtitles. Default value is 0 if <samp class="option">txt_transparent</samp> is set, 255 otherwise. </dd> </dl>

 <h2 class="chapter">11 Bitstream Filters# TOC</h2>

When you configure your FFmpeg build, all the supported bitstream filters are enabled by default. You can list all available ones using the configure option <code class="code">--list-bsfs</code>. You can disable all the bitstream filters using the configure option <code class="code">--disable-bsfs</code>, and selectively enable any bitstream filter using the option <code class="code">--enable-bsf=BSF</code>, or you can disable a particular bitstream filter using the option <code class="code">--disable-bsf=BSF</code>. The option <code class="code">-bsfs</code> of the ff* tools will display the list of all the supported bitstream filters included in your build. The ff* tools have a -bsf option applied per stream, taking a comma-separated list of filters, whose parameters follow the filter name after a ’=’.   <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v filter1[=opt1=str1:opt2=str2][,filter2] OUTPUT </pre>

Below is a description of the currently available bitstream filters, with their parameters, if any. <h3 class="section">11.1 aac_adtstoasc# TOC</h3>

Convert MPEG-2/4 AAC ADTS to an MPEG-4 Audio Specific Configuration bitstream. This filter creates an MPEG-4 AudioSpecificConfig from an MPEG-2/4 ADTS header and removes the ADTS header. This filter is required for example when copying an AAC stream from a raw ADTS AAC or an MPEG-TS container to MP4A-LATM, to an FLV file,

or to MOV/MP4 files and related formats such as 3GP or M4A. Please note that it is auto-inserted for MP4A-LATM and MOV/MP4 and related formats. <h3 class="section">11.2 av1_metadata# TOC</h3>

Modify metadata embedded in an AV1 stream. <dl class="table"> <dt><samp class="option">td</samp></dt> <dd> Insert or remove temporal delimiter OBUs in all temporal units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dd> Insert a TD at the beginning of every TU which does not already have one. </dd> <dt>‘<samp class="samp">remove</samp>’</dt> <dd> Remove the TD from the beginning of every TU which has one. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the color description fields in the stream (see AV1 section 6.4.2). </dd> <dt><samp class="option">color_range</samp></dt> <dd> Set the color range in the stream (see AV1 section 6.4.2; note that this cannot be set for streams using BT.709 primaries, sRGB transfer characteristic and identity (RGB) matrix coefficients). <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dd> Limited range. </dd> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> Full range. </dd> </dl>

</dd> <dt><samp class="option">chroma_sample_position</samp></dt> <dd> Set the chroma sample location in the stream (see AV1 section 6.4.2). This can only be set for 4:2:0 streams. <dl class="table"> <dt>‘<samp class="samp">vertical</samp>’</dt> <dd> Left position (matching the default in MPEG-2 and H.264). </dd> <dt>‘<samp class="samp">colocated</samp>’</dt> <dd> Top-left position. </dd> </dl>

</dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_display_tick) in the timing info in the sequence header. </dd> <dt><samp class="option">num_ticks_per_picture</samp></dt> <dd> Set the number of ticks in each picture, to indicate that the stream has a fixed framerate. Ignored if <samp class="option">tick_rate</samp> is not also set. </dd> <dt><samp class="option">delete_padding</samp></dt> <dd> Deletes Padding OBUs. </dd> </dl>

 <h3 class="section">11.3 chomp# TOC</h3>

Remove zero padding at the end of a packet. <h3 class="section">11.4 dca_core# TOC</h3>

Extract the core from a DCA/DTS stream, dropping extensions such as DTS-HD. <h3 class="section">11.5 dovi_rpu# TOC</h3>

Manipulate Dolby Vision metadata in a HEVC/AV1 bitstream, optionally enabling metadata compression. <dl class="table"> <dt><samp class="option">strip</samp></dt> <dd> If enabled, strip all Dolby Vision metadata (configuration record + RPU data blocks) from the stream. </dd> <dt><samp class="option">compression</samp></dt> <dd> Which compression level to enable. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No metadata compression. </dd> <dt>‘<samp class="samp">limited</samp>’</dt> <dd> Limited metadata compression scheme. Should be compatible with most devices. This is the default. </dd> <dt>‘<samp class="samp">extended</samp>’</dt> <dd> Extended metadata compression. Devices are not required to support this. Note that this level currently behaves the same as ‘<samp class="samp">limited</samp>’ in libavcodec. </dd> </dl> </dd> </dl>

 <h3 class="section">11.6 dovi_split# TOC</h3>

Split a Dolby Vision Profile 7 multi-layer HEVC bitstream. Profile 7 carries the enhancement-layer HEVC bitstream interleaved inside the base-layer access units, wrapped in user-unspecified NAL units of type 63 (UNSPEC63), and the RPU metadata as a sibling user-unspecified NAL of type 62 (UNSPEC62). <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Which Dolby Vision components to keep in the output bitstream. <dl class="table"> <dt>‘<samp class="samp">bl</samp>’</dt> <dd> Base layer only: drop every UNSPEC63 (EL) and every UNSPEC62 (RPU). The output is a plain HEVC stream with no Dolby Vision markers. This is the default. </dd> <dt>‘<samp class="samp">bl_rpu</samp>’</dt> <dd> Base layer with the RPU NAL kept. </dd> <dt>‘<samp class="samp">el</samp>’</dt> <dd> Enhancement layer only: for every UNSPEC63 NAL, strip the two-byte outer NAL header and emit the inner payload. The result is a standalone HEVC bitstream. UNSPEC62 (RPU) is dropped. </dd> <dt>‘<samp class="samp">el_rpu</samp>’</dt> <dd> Enhancement

layer with the RPU NAL kept verbatim. Same as ‘<samp class="samp">el</samp>’, but the UNSPEC62 RPU NALs are also emitted alongside the unwrapped EL NALs. </dd> </dl> </dd> </dl>

 <h3 class="section">11.7 dump_extra# TOC</h3>

Add extradata to the beginning of the filtered packets except when said packets already exactly begin with the extradata that is intended to be added. <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> The additional argument specifies which packets should be filtered. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">k</samp>’</dt> <dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> add extradata to all key packets </dd> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">all</samp>’</dt> <dd> add extradata to all packets </dd> </dl> </dd> </dl>

If not specified it is assumed ‘<samp class="samp">k</samp>’. For example the following <code class="command">ffmpeg</code> command forces a global header (thus disabling individual packet headers) in the H.264 packets generated by the <code class="code">libx264</code> encoder, but corrects them by adding the header stored in extradata to the key packets: <pre class="example-preformatted">ffmpeg -i INPUT -map 0 -flags:v +global_header -c:v libx264 -bsf:v dump_extra out.ts </pre>

 <h3 class="section">11.8 dv_error_marker# TOC</h3>

Blocks in DV which are marked as damaged are replaced by blocks of the specified color. <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color to replace damaged blocks by </dd> <dt><samp class="option">sta</samp></dt> <dd> A 16 bit mask which specifies which of the 16 possible error status values are to be replaced by colored blocks. 0xFFFE is the default which replaces all non 0 error status values. <dl class="table"> <dt>‘<samp class="samp">ok</samp>’</dt> <dd> No error, no concealment </dd> <dt>‘<samp class="samp">err</samp>’</dt> <dd> Error, No concealment </dd> <dt>‘<samp class="samp">res</samp>’</dt> <dd> Reserved </dd> <dt>‘<samp class="samp">notok</samp>’</dt> <dd> Error or concealment </dd> <dt>‘<samp class="samp">notres</samp>’</dt>

<dd> Not reserved </dd> <dt>‘<samp class="samp">Aa, Ba, Ca, Ab, Bb, Cb, A, B, C, a, b, erri, erru</samp>’</dt> <dd> The specific error status code </dd> </dl> see page 44-46 or section 5.5 of <a class="url" href="http://web.archive.org/web/20060927044735/http://www.smpte.org/ </dd> </dl>

 <h3 class="section">11.9 eac3_core# TOC</h3>

Extract the core from a E-AC-3 stream, dropping extra channels. <h3 class="section">11.10 eia608_to_smpte436m# TOC</h3>

Convert from a <code class="code">EIA_608</code> stream to a <code class="code">SMPTE_436M_ANC</code> data stream, wrapping the closed captions in CTA-708 CDP VANC packets. <dl class="table"> <dt><samp class="option">line_number</samp></dt> <dd> Choose which line number the generated VANC packets should go on. You generally want either line 9 (the default) or 11. </dd> <dt><samp class="option">wrapping_type</samp></dt> <dd> Choose the SMPTE 436M wrapping type, defaults to ‘<samp class="samp">vanc_frame</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">vanc_frame</samp>’</dt> <dd> VANC frame (interlaced or segmented progressive frame) </dd> <dt>‘<samp class="samp">vanc_field_1</samp>’</dt> <dt>‘<samp class="samp">vanc_field_2</samp>’</dt> <dt>‘<samp class="samp">vanc_progressive_frame</samp>’</dt> </dl> </dd> <dt><samp class="option">sample_coding</samp></dt> <dd> Choose the SMPTE 436M sample coding, defaults to ‘<samp class="samp">8bit_luma</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">8bit_luma</samp>’</dt> <dd> 8-bit component luma samples </dd> <dt>‘<samp class="samp">8bit_color_diff</samp>’</dt> <dd> 8-bit component color difference samples </dd> <dt>‘<samp class="samp">8bit_luma_and_color_diff</samp>’</dt> <dd> 8-bit component luma and color difference samples </dd> <dt>‘<samp class="samp">10bit_luma</samp>’</dt> <dd> 10-bit component luma samples </dd> <dt>‘<samp class="samp">10bit_color_diff</samp>’</dt> <dd> 10-bit component color difference samples </dd> <dt>‘<samp class="samp">10bit_luma_and_color_diff</samp>’</dt> <dd> 10-bit component luma and color difference samples </dd> <dt>‘<samp class="samp">8bit_luma_parity_error</samp>’</dt> <dd> 8-

bit component luma samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_color_diff_parity_error</samp>’</dt> <dd> 8-bit component color difference samples with parity error </dd> <dt>‘<samp class=”samp”>8bit_luma_and_color_diff_parity_error</samp>’</dt> <dd> 8-bit component luma and color difference samples with parity error </dd> </dl> </dd> <dt><samp class=”option”>initial_cdp_sequence_cntr</samp></dt> <dd> The initial value of the CDP’s 16-bit unsigned integer <code class=”code”>cdp_hdr_sequence_cntr</code> and <code class=”code”>cdp_ftr_sequence_cntr</code> fields. Defaults to 0. </dd> <dt><samp class=”option”>cdp_frame_rate</samp></dt> <dd> Set the CDP’s <code class=”code”>cdp_frame_rate</code> field. This doesn’t actually change the timing of the data stream, it just changes the values inserted in that field in the generated CDP packets. Defaults to ‘<samp class=”samp”>30000/1001</samp>’. </dd> </dl>

 <h3 class=”section”>11.11 extract_extradata# TOC</h3>

Extract the in-band extradata. Certain codecs allow the long-term headers (e.g. MPEG-2 sequence headers, or H.264/HEVC (VPS/)SPS/PPS) to be transmitted either "in-band" (i.e. as a part of the bitstream containing the coded frames) or "out of band" (e.g. on the container level). This latter form is called "extradata" in FFmpeg terminology. This bitstream filter detects the in-band headers and makes them available as extradata. <dl class=”table”> <dt><samp class=”option”>remove</samp></dt> <dd> When this option is enabled, the long-term headers are removed from the bitstream after extraction. </dd> </dl>

 <h3 class=”section”>11.12 filter_units# TOC</h3>

Remove units with types in or not in a given set from the stream. <dl class=”table”> <dt><samp class=”option”>pass_types</samp></dt> <dd> List of unit types or ranges of unit types to pass through while removing all others. This is specified as a ’|’-separated list of unit type values or ranges of values with ’-’. </dd> <dt><samp class=”option”>remove_types</samp></dt> <dd> Identical to <samp class=”option”>pass_types</samp>, except the units in the given set removed and all others passed through. </dd> </dl>

The types used by pass_types and remove_types correspond to NAL unit types (nal_unit_type) in H.264, HEVC and H.266 (see Table 7-1 in

the H.264 and HEVC specifications or Table 5 in the H.266 specification), to marker values for JPEG (without 0xFF prefix) and to start codes without start code prefix (i.e. the byte following the 0x000001) for MPEG-2. For VP8 and VP9, every unit has type zero. Extradata is unchanged by this transformation, but note that if the stream contains inline parameter sets then the output may be unusable if they are removed. For example, to remove all non-VCL NAL units from an H.264 stream: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=pass_types=1-5' OUTPUT </pre>

To remove all AUDs, SEI and filler from an H.265 stream: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=35|38-40' OUTPUT </pre>

To remove all user data from a MPEG-2 stream, including Closed Captions: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=178' OUTPUT </pre>

To remove all SEI from a H264 stream, including Closed Captions: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=6' OUTPUT </pre>

To remove all prefix and suffix SEI from a HEVC stream, including Closed Captions and dynamic HDR: <pre class=”example-preformatted”>ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=39|40' OUTPUT </pre>

 <h3 class=”section”>11.13 hapqa_extract# TOC</h3>

Extract Rgb or Alpha part of an HAPQA file, without recompression, in order to create an HAPQ or an HAPAlphaOnly file. <dl class=”table”> <dt><samp class=”option”>texture</samp></dt> <dd> Specifies the texture to keep. <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dt><samp class=”option”>alpha</samp></dt> </dl>

</dd> </dl>

Convert HAPQA to HAPQ <pre class=”example-preformatted”>ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=color -tag:v HapY -metadata:s:v:0 encoder="HAPQ" hapq_file.mov </pre>

Convert HAPQA to HAPAlphaOnly <pre class=”example-preformatted”>ffmpeg -i hapqa_inputfile.mov -c

copy -bsf:v hapqa_extract=texture=alpha -tag:v HapA -metadata:s:v:0 encoder="HAPAlpha Only" hapalphaonly_file.mov </pre>

 <h3 class="section">11.14 h264_metadata# TOC</h3>

Modify metadata embedded in an H.264 stream. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">pass</samp>’</dt> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</
</dl>

Default is pass. </dd> <dt><samp class="option">sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio of the stream in the VUI parameters. See H.264 table E-1. </dd> <dt><samp class="option">overscan_appropriate_flag</samp></dt> <dd> Set whether the stream is suitable for display using overscan or not (see H.264 section E.2.1). </dd> <dt><samp class="option">video_format</samp></dt> <dt><samp class="option">video_full_range_flag</samp> <dd> Set the video format in the stream (see H.264 section E.2.1 and table E-2). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.264 section E.2.1 and tables E-3, E-4 and E-5). </dd> <dt><samp class="option">chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.264 section E.2.1 and figure E-1). </dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_tick) in the VUI parameters. This is the smallest time unit representable in the stream, and in many cases represents the field rate of the stream (double the frame rate). </dd> <dt><samp class="option">fixed_frame_rate_flag</samp></dt> <dd> Set whether the stream has fixed framerate - typically this indicates that the framerate is exactly half the tick rate, but the exact meaning is dependent on interlacing and the picture structure (see H.264 section E.2.1 and table E-6). </dd> <dt><samp class="option">zero_new_constraint_set_flags</samp></dt> <dd> Zero constraint_set4_flag and constraint_set5_flag in the SPS. These bits were reserved in a previous version of the H.264 spec, and thus some hardware decoders require these to be zero. The result of zeroing this is still a valid bitstream. </dd> <dt><samp class="option">crop_left</samp></dt> <dt><samp class="option">crop_right</samp></dt> <dt><samp class="option">crop_top</samp></dt> <dt><samp class="option">crop_bottom</samp></dt> <dd> Set the frame cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the

chroma is subsampled or the stream is interlaced (see H.264 section 7.4.2.1.1). </dd> <dt><samp class="option">sei_user_data</samp></dt> <dd> Insert a string as SEI unregistered user data. The argument must be of the form UUID+string, where the UUID is as hex digits possibly separated by hyphens, and the string can be anything. For example, ‘<samp class="samp">086f3693-b7b3-4f2c-9653-21492feee5b8+hello</samp>’ will insert the string “hello” associated with the given UUID. </dd> <dt><samp class="option">delete_filler</samp></dt> <dd> Deletes both filler NAL units and filler SEI messages. </dd> <dt><samp class="option">display_orientation</samp></dt> <dd> Insert, extract or remove Display orientation SEI messages. See H.264 section D.1.27 and D.2.27 for syntax and semantics. <dl class="table"> <dt>‘<samp class="samp">pass</samp>’</dt> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</dt> <dt>‘<samp class="samp">extract</samp>’</dt> </dl>

Default is pass. Insert mode works in conjunction with <code class="code">rotate</code> and <code class="code">flip</code> options. Any pre-existing Display orientation messages will be removed in insert or remove mode. Extract mode attaches the display matrix to the packet as side data. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Set rotation in display orientation SEI (anticlockwise angle in degrees). Range is -360 to +360. Default is NaN. </dd> <dt><samp class="option">flip</samp></dt> <dd> Set flip in display orientation SEI. <dl class="table"> <dt>‘<samp class="samp">horizontal</samp>’</dt> <dt>‘<samp class="samp">vertical</samp>’</dt> </dl>

Default is unset. </dd> <dt><samp class="option">level</samp></dt> <dd> Set the level in the SPS. Refer to H.264 section A.3 and tables A-1 to A-5. The argument must be the name of a level (for example, ‘<samp class="samp">4.2</samp>’), a level_idc value (for example, ‘<samp class="samp">42</samp>’), or the special name ‘<samp class="samp">auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class="section">11.15 h264_mp4toannexb# TOC</h3

Convert an H.264 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.264 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an H.264 stream

to mpegts format with <code class=”command”>ffmpeg</code>, you can use the command: <pre class=”example-preformatted”>ffmpeg -i INPUT.mp4 -codec copy -bsf:v h264_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class=”code”>mpegts</code>) and raw H.264 (muxer <code class=”code”>h264</code>) output formats. <h3 class=”section”>11.16 h264_redundant_pps# TOC</h3>

This applies a specific fixup to some Blu-ray BDMV H264 streams which contain redundant PPSs. The PPSs modify irrelevant parameters of the stream, confusing other transformations which require the correct extradata. The encoder used on these impacted streams adds extra PPSs throughout the stream, varying the initial QP and whether weighted prediction was enabled. This causes issues after copying the stream into a global header container, as the starting PPS is not suitable for the rest of the stream. One side effect, for example, is seeking will return garbled output until a new PPS appears. This BSF removes the extra PPSs and rewrites the slice headers such that the stream uses a single leading PPS in the global header, which resolves the issue. <h3 class=”section”>11.17 hevc_metadata# TOC</h3>

Modify metadata embedded in an HEVC stream. <dl class=”table”> <dt><samp class=”option”>aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class=”table”> <dt>‘<samp class=”samp”>insert</samp>’</dt> <dt>‘<samp class=”samp”>remove</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio in the stream in the VUI parameters. </dd> <dt><samp class=”option”>video_format</samp></dt> <dt><samp class=”option”>video_full_range_flag</samp></dt> <dd> Set the video format in the stream (see H.265 section E.3.1 and table E.2). </dd> <dt><samp class=”option”>colour_primaries</samp></dt> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.265 section E.3.1 and tables E.3, E.4 and E.5). </dd> <dt><samp class=”option”>chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.265 section E.3.1 and figure E.1). </dd> <dt><samp class=”option”>tick_rate</samp></dt> <dd> Set the tick rate in the VPS and VUI parameters (time_scale /

num_units_in_tick). Combined with <samp class="option">num_ticks_poc_diff_one</samp>, this can set a constant framerate in the stream. Note that it is likely to be overridden by container parameters when the stream is in a container. </dd> <dt><samp class="option">num_ticks_poc_diff_one</samp></dt> <dd> Set poc_proportional_to_timing_flag in VPS and VUI and use this value to set num_ticks_poc_diff_one_minus1 (see H.265 sections 7.4.3.1 and E.3.1). Ignored if <samp class="option">tick_rate</samp> is not also set. </dd> <dt><samp class="option">crop_left</samp></dt> <dt><samp class="option">crop_right</samp></dt> <dt><samp class="option">crop_top</samp></dt> <dt><samp class="option">crop_bottom</samp></dt> <dd> Set the conformance window cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled (H.265 section 7.4.3.2.1). </dd> <dt><samp class="option">width</samp></dt> <dt><samp class="option">height</samp></dt> <dd> Set width and height after crop. </dd> <dt><samp class="option">level</samp></dt> <dd> Set the level in the VPS and SPS. See H.265 section A.4 and tables A.6 and A.7. The argument must be the name of a level (for example, ‘<samp class="samp">5.1</samp>’), a general_level_idc value (for example, ‘<samp class="samp">153</samp>’ for level 5.1), or the special name ‘<samp class="samp">auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class="section">11.18 hevc_mp4toannexb# TOC</h3

Convert an HEVC/H.265 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.265 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an HEVC stream to mpegts format with <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.mp4 -codec copy -bsf:v hevc_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class="code">mpegts</code>) and raw HEVC/H.265 (muxer <code class="code">h265</code> or <code class="code">hevc</code>) output formats. <h3 class="section">11.19 imxdump# <a class="anchor hidden-xs"href="#toc-

imxdump" aria-hidden="true">TOC</h3>

Modifies the bitstream to fit in MOV and to be usable by the Final Cut Pro decoder. This filter only applies to the mpeg2video codec, and is likely not needed for Final Cut Pro 7 and newer with the appropriate <samp class="option">-tag:v</samp>. For example, to remux 30 MB/sec NTSC IMX to MOV: <pre class="example-preformatted">ffmpeg -i input.mxf -c copy -bsf:v imxdump -tag:v mx3n output.mov </pre>

 <h3 class="section">11.20 jpegapp# TOC</h3>

Inject JPEG APP0..APP15 segments from a length-prefixed sidecar into each MJPEG packet, after SOI. The sidecar is a sequence of records, one per encoded JPEG, in encode order: <pre class="example-preformatted">u32be length u8 payload[length] </pre>

<var class="var">length</var> does not include the 4-byte prefix and must not exceed 16 MiB. A zero <var class="var">length</var> leaves that packet unchanged. A non-empty payload must be a concatenation of complete APP segments (<code class="code">FF En</code> plus the 16-bit length that includes those two length bytes). If the payload starts with APP0 and the packet has an APP0 marker in the two bytes immediately following SOI, that APP0 segment is dropped. APP0 segments preceded by fill bytes or another segment are retained. An APP1-only payload keeps the encoder APP0: SOI, APP1, APP0, DQT, ... The filter requires <samp class="option">sidecar</samp>. Extra unused sidecar bytes are ignored; a packet without a matching record is an error. Records follow packets in arrival order. Flushing the filter does not rewind the sidecar, so a reused context needs a sidecar positioned for the packets it will see next. A sidecar read or validation error prevents further record consumption by that filter context. To resume, create a new context with a sidecar matching the remaining packets. Flushing does not clear this error.   <pre class="example-preformatted">ffmpeg -i in.mp4 -c:v mjpeg -bsf:v jpegapp=sidecar=00000004DEADBEEF frame_%d.jpg </pre>

 <h3 class="section">11.21 mjpeg2jpeg# TOC</h3>

Convert MJPEG/AVI1 packets to full JPEG/JFIF packets. MJPEG is a video codec wherein each video frame is essentially a JPEG image. The individual frames can be extracted without loss, e.g. by

  <pre class=”example-preformatted”>ffmpeg -i ../some_mjpeg.avi -c:v copy frames_%d.jpg </pre>

Unfortunately, these chunks are incomplete JPEG images, because they lack the DHT segment required for decoding. Quoting from <a class=”url” href=”http://www.digitalpreservation.gov/formats/fdd/fdd000063.shtml”>http://www.digitalpreservation.go
 Avery Lee, writing in the rec.video.desktop newsgroup in 2001, commented that "MJPEG, or at least the MJPEG in AVIs having the MJPG fourcc, is restricted JPEG with a fixed – and *omitted* – Huffman table. The JPEG must be YCbCr colorspace, it must be 4:2:2, and it must use basic Huffman encoding, not arithmetic or progressive. . . . You can indeed extract the MJPEG frames and decode them with a regular JPEG decoder, but you have to prepend the DHT segment to them, or else the decoder won’t have any idea how to decompress the data. The exact table necessary is given in the OpenDML spec." This bitstream filter patches the header of frames extracted from an MJPEG stream (carrying the AVI1 header ID and lacking a DHT segment) to produce fully qualified JPEG images.   <pre class=”example-preformatted”>ffmpeg -i mjpeg-movie.avi -c:v copy -bsf:v mjpeg2jpeg frame_%d.jpg exiftran -i -9 frame*.jpg ffmpeg -i frame_%d.jpg -c:v copy rotated.avi </pre>

 <h3 class=”section”>11.22 mjpegadump# TOC</h3>

Add an MJPEG A header to the bitstream, to enable decoding by Quicktime. <h3 class=”section”>11.23 mov2textsub# TOC</h3>

Extract a representable text file from MOV subtitles, stripping the metadata header from each subtitle packet. See also the text2movsub filter. <h3 class=”section”>11.24 mpeg2_metadata# TOC</h3>

Modify metadata embedded in an MPEG-2 stream. <dl class=”table”> <dt><samp class=”option”>display_aspect_ratio</samp></dt> <dd> Set the display aspect ratio in the stream. The following fixed values are supported: <dl class=”table”> <dt><samp class=”option”>4/3</samp></dt> <dt><samp class=”option”>16/9</samp></dt> <dt><samp class=”option”>221/100</samp></dt> </dl> Any other

value will result in square pixels being signalled instead (see H.262 section 6.3.3 and table 6-3). </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Set the frame rate in the stream. This is constructed from a table of known values combined with a small multiplier and divisor - if the supplied value is not exactly representable, the nearest representable value will be used instead (see H.262 section 6.3.3 and table 6-4). </dd> <dt><samp class="option">video_format</samp></dt> <dd> Set the video format in the stream (see H.262 section 6.3.6 and table 6-6). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.262 section 6.3.6 and tables 6-7, 6-8 and 6-9). </dd> </dl>

 <h3 class="section">11.25 mpeg4_unpack_bframes# TOC</h3>

Unpack DivX-style packed B-frames. DivX-style packed B-frames are not valid MPEG-4 and were only a workaround for the broken Video for Windows subsystem. They use more space, can cause minor AV sync issues, require more CPU power to decode (unless the player has some decoded picture queue to compensate the 2,0,2,0 frame per packet style) and cause trouble if copied into a standard container like mp4 or mpeg-ps/ts, because MPEG-4 decoders may not be able to decode them, since they are not valid MPEG-4. For example to fix an AVI file containing an MPEG-4 stream with DivX-style packed B-frames using <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.avi -codec copy -bsf:v mpeg4_unpack_bframes OUTPUT.avi </pre>

 <h3 class="section">11.26 noise# TOC</h3>

Damages the contents of packets or simply drops them without damaging the container. Can be used for fuzzing or testing error resilience/concealment. Parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt> <dd> Accepts an expression whose evaluation per-packet determines how often bytes in that packet will be modified. A value below 0 will result in a variable frequency. Default is 0 which results in no modification. However, if neither amount nor drop is specified, amount will be set to <var class="var">-1</var>. See below for accepted variables. </dd> <dt><samp class="option">drop</samp></dt> <dd> Accepts an expression evaluated per-packet whose value determines whether that packet is dropped.

Evaluation to a positive value results in the packet being dropped. Evaluation to a negative value results in a variable chance of it being dropped, roughly inverse in proportion to the magnitude of the value. Default is 0 which results in no drops. See below for accepted variables. </dd> <dt><samp class=”option”>dropamount</samp></dt> <dd> Accepts a non-negative integer, which assigns a variable chance of it being dropped, roughly inverse in proportion to the value. Default is 0 which results in no drops. This option is kept for backwards compatibility and is equivalent to setting drop to a negative value with the same magnitude i.e. <code class=”code”>dropamount=4</code> is the same as <code class=”code”>drop=-4</code>. Ignored if drop is also specified. </dd> </dl>

Both <code class=”code”>amount</code> and <code class=”code”>drop</code> accept expressions containing the following variables: <dl class=”table”> <dt>‘<samp class=”samp”>n</samp>’</dt> <dd> The index of the packet, starting from zero. </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dd> The timebase for packet timestamps. </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Packet presentation timestamp. </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Packet decoding timestamp. </dd> <dt>‘<samp class=”samp”>nopts</samp>’</dt> <dd> Constant representing AV_NOPTS_VALUE. </dd> <dt>‘<samp class=”samp”>startpts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE PTS seen in the stream. </dd> <dt>‘<samp class=”samp”>startdts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE DTS seen in the stream. </dd> <dt>‘<samp class=”samp”>duration</samp>’</dt> <dt>‘<samp class=”samp”>d</samp>’</dt <dd> Packet duration, in timebase units. </dd> <dt>‘<samp class=”samp”>pos</samp>’</dt> <dd> Packet position in input; may be -1 when unknown or not set. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Packet size, in bytes. </dd> <dt>‘<samp class=”samp”>key</samp>’</dt> <dd> Whether packet is marked as a keyframe. </dd> <dt>‘<samp class=”samp”>state</samp>’</dt> <dd> A pseudo random integer, primarily derived from the content of packet payload. </dd> </dl>

 <h4 class=”subsection”>11.26.1 Examples# TOC</h4> Apply modification to every byte but don’t drop any packets. <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf noise=1 output.mkv </pre>

Drop every video packet not marked as a keyframe after times-

tamp 30s but do not modify any of the remaining packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v noise=drop='gt(pts*tb\,30)*not(key)' output.mkv </pre>

Drop one second of audio every 10 seconds and add some random noise to the rest. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:a noise=amount=-1:drop='between(mod(pts*tb\,10)\,9\,10)' output.mkv </pre>

 <h3 class="section">11.27 null# TOC</h3> This bitstream filter passes the packets through unchanged. <h3 class="section">11.28 pcm_rechunk# TOC</h3>

Repacketize PCM audio to a fixed number of samples per packet or a fixed packet rate per second. This is similar to the <a data-manual="ffmpeg-filters" href="./ffmpeg-filters.html#asetnsamples">asetnsamples audio filter but works on audio packets instead of audio frames. <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of samples per each output audio packet. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio packet with silence, so that it will contain the same number of samples (or roughly the same number of samples, see <samp class="option">frame_rate</samp>) as the previous ones. Default value is 1. </dd> <dt><samp class="option">frame_rate, r</samp></dt> <dd> This option makes the filter output a fixed number of packets per second instead of a fixed number of samples per packet. If the audio sample rate is not divisible by the frame rate then the number of samples will not be constant but will vary slightly so that each packet will start as close to the frame boundary as possible. Using this option has precedence over <samp class="option">nb_out_samples</samp>. </dd> </dl>

You can generate the well known 1602-1601-1602-1601-1602 pattern of 48kHz audio for NTSC frame rate using the <samp class="option">frame_rate</samp> option. <pre class="example-preformatted">ffmpeg -f lavfi -i sine=r=48000:d=1 -c pcm_s16le -bsf pcm_rechunk=r=30000/1001 -f framecrc - </pre>

 <h3 class="section">11.29 pgs_frame_merge# <a class="anchor hidden-xs"href="#toc-pgs_005fframe_005fmerge" aria-

hidden=”true”>TOC</h3>

Merge a sequence of PGS Subtitle segments ending with an "end of display set" segment into a single packet. This is required by some containers that support PGS subtitles (muxer <code class=”code”>matroska</code>). <h3 class=”section”>11.30 prores_metadata# TOC</h3>

Modify color property metadata embedded in prores stream. <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT601 625 </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> BT601 525 </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> DCI P3 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> P3 D65 </dd> </dl>

</dd> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dd> Set the color transfer. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same transfer characteristics property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT 601, BT 709, BT 2020 </dd> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt> <dd> SMPTE ST 2084 </dd> <dt>‘<samp class=”samp”>arib-std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd> </dl>

</dd> <dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set the matrix coefficient. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same colorspace property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>&rsqu <dd> BT 601 </dd> <dt>‘<samp class=”samp”>bt2020nc</samp>’</dt> </dl> </dd> </dl>

Set Rec709 colorspace for each frame of the file <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf:v prores_metadata=color_primaries=bt709:color_trc=bt709:colorspace=bt709

output.mov </pre>

Set Hybrid Log-Gamma parameters for each frame of the file <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf:v prores_metadata=color_primaries=bt2020:color_trc=arib-std-b67:colorspace=bt2020nc output.mov </pre>

 <h3 class=”section”>11.31 remove_extra# TOC</h3>

Remove extradata from packets. It accepts the following parameter: <dl class=”table”> <dt><samp class=”option”>freq</samp></dt> <dd> Set which frame types to remove extradata from. <dl class=”table”> <dt>‘<samp class=”samp”>k</samp>’</dt> <dd> Remove extradata from non-keyframes only. </dd> <dt>‘<samp class=”samp”>keyframe</samp>’</dt> <dd> Remove extradata from keyframes only. </dd> <dt>‘<samp class=”samp”>e, all</samp>’</dt> <dd> Remove extradata from all frames. </dd> </dl> </dd> </dl>

 <h3 class=”section”>11.32 setts# TOC</h3> Set PTS and DTS in packets. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>ts</samp></dt> <dt><samp class=”option”>pts</samp></dt> <dt><samp class=”option”>dts</samp></dt> <dd> Set expressions for PTS, DTS or both. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> Set expression for duration. </dd> <dt><samp class=”option”>time_base</samp></dt> <dd> Set output time base. </dd> <dt><samp class=”option”>prescale</samp></dt> <dd> Set whether to convert time fields to user-set output time base before evaluation of expressions. Defaults to 0. </dd> </dl>

The expressions are evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>N</samp></dt> <dd> The count of the input packet. Starting from 0. </dd> <dt><samp class=”option”>TS</samp></dt> <dd> The demux timestamp in input in case of <code class=”code”>ts</code> or <code class=”code”>dts</code> option or presentation timestamp in case of <code class=”code”>pts</code> option. </dd> <dt><samp class=”option”>POS</samp></dt> <dd> The original position in the file of the packet, or undefined if undefined for the current packet </dd> <dt><samp class=”option”>DTS</samp></dt> <dd> The demux timestamp in input. </dd> <dt><samp class=”option”>PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class=”option”>DURATION</samp></dt>

<dd> The duration in input. </dd> <dt><samp class="option">STARTDTS</samp></dt>
<dd> The DTS of the first packet. </dd> <dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first packet. </dd> <dt><samp class="option">PREV_INDTS</samp></dt>
<dd> The previous input DTS. </dd> <dt><samp class="option">PREV_INPTS</samp></dt
<dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INDURATION</sam
<dd> The previous input duration. </dd> <dt><samp class="option">PREV_OUTDTS</samp></dt> <dd> The previous output DTS. </dd> <dt><samp class="option">PREV_OUTPTS</samp></dt>
<dd> The previous output PTS. </dd> <dt><samp class="option">PREV_OUTDURATION</
<dd> The previous output duration. </dd> <dt><samp class="option">NEXT_DTS</samp></dt> <dd> The next input DTS. </dd> <dt><samp class="option">NEXT_PTS</samp></dt>
<dd> The next input PTS. </dd> <dt><samp class="option">NEXT_DURATION</samp></
<dd> The next input duration. </dd> <dt><samp class="option">TB</samp></dt>
<dd> The timebase in which time fields are denominated. The user-set output timebase if prescale is enabled, else the input timebase. </dd>
<dt><samp class="option">TB_OUT</samp></dt> <dd> The output timebase. </dd> <dt><samp class="option">SR</samp></dt>
<dd> The sample rate of stream packet belongs. </dd>
<dt><samp class="option">NOPTS</samp></dt> <dd> The AV_NOPTS_VALUE constant. </dd> </dl>

For example, to set PTS equal to DTS (not recommended if B-frames are involved): <pre class="example-preformatted">ffmpeg -i INPUT -c:a copy -bsf:a setts=pts=DTS out.mkv
</pre>

 <h3 class="section">11.33 showinfo# TOC</h3> Log basic packet information. Mainly useful for testing, debugging, and development. <h3 class="section">11.34 side_data# TOC</h3> Remove side data from packets and from the stream, or select packets by their side data. It accepts the following parameters: <dl class="table"> <dt><samp class="option">remove</samp></dt>
<dd> Set the side data types to remove, joined with ‘<samp class="samp">+</samp>’, from the packets and from the side data of the stream. Each type is given by its handle; <code class="command">ffmpeg -h bsf=side_data</code> lists them. </dd> <dt><samp class="option">select</samp></dt>
<dd> Set the side data types of the packets to keep, joined with ‘<samp class="samp">+</samp>’. A packet is kept if it carries any of them, and dropped otherwise. The selection looks at the side

data before <samp class="option">remove</samp> applies. Dropping a packet breaks the decoding of the packets that depend on it, so this is meant for analysis or for streams where each packet decodes on its own. </dd> </dl>

For example, to drop the display matrix and the replay gain of a stream: <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v side_data=remove=displaymatrix+replaygain output.mkv </pre>

 <h3 class="section">11.35 smpte436m_to_eia608# TOC</h3>

Convert from a <code class="code">SMPTE_436M_ANC</code> data stream to a <code class="code">EIA_608</code> stream, extracting the closed captions from CTA-708 CDP VANC packets, and ignoring all other data. <h3 class="section">11.36 text2movsub# TOC</h3>

Convert text subtitles to MOV subtitles (as used by the <code class="code">mov_text</code> codec) with metadata headers. See also the mov2textsub filter. <h3 class="section">11.37 trace_headers# TOC</h3>

Log trace output containing all syntax elements in the coded stream headers (everything above the level of individual coded blocks). This can be useful for debugging low-level stream issues. Supports AV1, H.264, H.265, (M)JPEG, MPEG-2 and VP9, but depending on the build only a subset of these may be available. <h3 class="section">11.38 trim# TOC</h3>

Drop packets outside a given range, similar to the trim and atrim filters, but operating on packets instead of decoded frames. The range is described by a <samp class="option">start</samp> and an <samp class="option">end</samp> bound. At least one of them must be set. Each bound is fully described by three independent choices: <ul class="itemize mark-bullet"> The discriminant it is compared against: the packet’s presentation timestamp, its decoding timestamp, or its

index. The two bounds need not use the same one, for instance a <code class=”code”>dts</code> start with a <code class=”code”>pts</code> end. The unit its value is given in: the stream time base, milliseconds, or packets. What it is counted from: zero, or its reference packet when <samp class=”option”>start_rel</samp> / <samp class=”option”>end_rel</samp> is set. For <samp class=”option”>start</samp> the reference packet is the first packet carrying that discriminant. For <samp class=”option”>end</samp> it is the first exported packet.

The <samp class=”option”>start_type</samp> and <samp class=”option”>end_type</samp> options name a combination of the three. The <code class=”code”>dur_ts</code> and <code class=”code”>dur_t_msec</code> end types are the always relative forms of <code class=”code”>pts</code> and <code class=”code”>msec_pt</code>. A zero duration therefore produces an empty stream. Every packet is judged on its own discriminant, and on nothing else. A packet not carrying the discriminant its bound uses cannot be placed, and is dropped. <samp class=”option”>start_rel</samp> makes <samp class=”option”>start</samp> an offset from a reference packet instead of an absolute value. The reference is the first packet carrying the discriminant that bound uses. Its value is added to the offset once, and the bound stays there for the rest of the stream. <samp class=”option”>end_rel</samp> anchors on the first exported packet instead, at the position <samp class=”option”>start</samp> trimmed it to. The two bounds therefore move together. Only a discriminant that cannot come back into the range ends the stream, namely the packet index or the decoding timestamp. End of stream is signaled once one of those passes the <samp class=”option”>end</samp> bound. A presentation timestamp past the end only drops its packet, so that later packets still inside the range are not lost. A bound given in a time unit can fall inside a packet rather than between two. Such a packet is trimmed. Set <samp class=”option”>trim_packets</samp> to false to keep every packet exactly as it came in: exported whole if its own value is inside the range, dropped otherwise. Audio is trimmed in sample space, through skip samples side data. The timestamps and duration are left untouched, and a decoder discards the signaled samples on output. This needs a sample rate and a time base. Audio missing either is rejected rather than trimmed on the timeline. Every other media type is trimmed by shifting the packet timestamps and shortening its duration. Trimming inside an audio packet needs the number of samples it decodes to. Where the codec does not give that number, the packet is handled as though <samp class=”option”>trim_packets</samp> were false. A skip larger than the packet carrying it continues into the packets after it. Dropping such a packet carries the remainder over to the next one. A skip that packet already carries replaces the remainder, as a decoder would, and the larger of it and this filter’s own trim wins. By default a packet outside the range is dropped, whether or not later packets depend on it. Streams of independent frames, such as

ADTS or intra only video, can be cut anywhere. Cutting an inter coded stream inside a group of pictures leaves the first packets in range undecodable. Set <samp class=”option”>preroll</samp> to export the packets those first packets depend on as well. They are flagged for a decoder to decode and then drop, so the frames it outputs still begin at the <samp class=”option”>start</samp> bound. Only a decoder acts on that flag. Use <samp class=”option”>preroll</samp> where the stream is decoded. A muxer writes the extra packets like any other. A remux then covers more than the requested range. <samp class=”option”>preroll</samp> covers the start of the range only. On a stream with B-frames an <samp class=”option”>end</samp> bound on the presentation timestamp can drop a packet the last frames in the range need. Those frames then decode incorrectly. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>start</samp></dt> <dd> Time or index marking the start of the accepted range. Packets before it are dropped. A packet straddling it is trimmed at its start, or dropped when <samp class=”option”>trim_packets</samp> is false. </dd> <dt><samp class=”option”>start_type</samp></dt> <dd> How to interpret <samp class=”option”>start</samp>: <dl class=”table”> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Presentation timestamp, in stream time base (default). </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Decoding timestamp, in stream time base. </dd> <dt>‘<samp class=”samp”>pkt_index</samp>’</dt> <dd> Packet index, counting from zero. </dd> <dt>‘<samp class=”samp”>msec_pt</samp>’</dt> <dd> Milliseconds, matched against the presentation timestamp. </dd> <dt>‘<samp class=”samp”>msec_dt</samp>’</dt> <dd> Milliseconds, matched against the decoding timestamp. </dd> </dl>

</dd> <dt><samp class=”option”>start_rel</samp></dt> <dd> Count <samp class=”option”>start</samp> from the reference packet rather than from zero. Has no effect on ‘<samp class=”samp”>pkt_index</samp>’, whose first packet is already zero. Default false. </dd> <dt><samp class=”option”>end</samp></dt> <dd> Time or index marking the end of the accepted range. Signals end of stream when reached. A packet straddling it is trimmed at its end, or exported untouched when <samp class=”option”>trim_packets</samp> is false. </dd> <dt><samp class=”option”>end_type</samp></dt> <dd> How to interpret <samp class=”option”>end</samp>. Same values as <samp class=”option”>start_type</samp>, plus: <dl class=”table”> <dt>‘<samp class=”samp”>dur_ts</samp>’</dt> <dd> Duration in stream time base. Same as ‘<samp class=”samp”>pts</samp>’ with <samp class=”option”>end_rel</samp> set. </dd> <dt>‘<samp class=”samp”>dur_t_msec</samp>’</dt> <dd> Duration in milliseconds. Same as ‘<samp class=”samp”>msec_pt</samp>’ with <samp class=”option”>end_rel</samp> set. </dd> </dl>

</dd> <dt><samp class=”option”>end_rel</samp></dt> <dd> Count <samp class=”option”>end</samp> from the first exported packet rather than from zero. The reference is that packet’s value on the discriminant <samp class=”option”>end_type</samp> names. On a timestamp it is taken at the position the packet is trimmed to. For audio that is the time of its first retained sample. The skip that packet carries can push it past the packet itself. With ‘<samp class=”samp”>pkt_index</samp>’ it exports a fixed number of packets, which can be combined with a timestamp-based <samp class=”option”>start</samp>. Always set for the duration types. Default false. </dd> <dt><samp class=”option”>trim_packets</samp></dt> <dd> Trim packets straddling a boundary. When false, packets are exported exactly as they came in, with no timestamp adjustment and no skip samples side data. A packet is then exported untouched if its value falls inside the range, and dropped otherwise. Default true. </dd> <dt><samp class=”option”>preroll</samp></dt> <dd> Export the packets a decoder needs to decode the first packet in range, rather than dropping them for falling before the <samp class=”option”>start</samp> bound. The filter holds every packet from the last keyframe on. It exports them ahead of the first packet in range, carrying <code class=”code”>AV_PKT_FLAG_DISCARD</code>. Video only. Default false. </dd> <dt><samp class=”option”>preroll_size</samp></dt> <dd> Maximum number of packets held for <samp class=”option”>preroll</samp>. A group of pictures longer than this is dropped whole, with a warning naming the count to raise it past. Default 300, which covers the keyframe intervals encoders commonly default to. </dd> </dl>

For example, to keep only the 10 first packets of an ADTS stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=end=10:end_type=pkt_index OUTPUT.aac </pre>

To keep 5 seconds starting 1.5 seconds into the stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

The same, expressed relative to the first packet so it works regardless of the stream’s starting timestamp: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:start_rel=1:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

To start on a keyframeless point of an inter coded video stream, re-encoding it so that the preroll is decoded rather than written out. The filter must run before the decoder, so it is given before the input: <pre class=”example-preformatted”>ffmpeg -bsf:v trim=start=17000:start_type=msec_pt:end=30000:end_type=dur_t_msec:preroll=1

-i INPUT.mp4 OUTPUT.mp4 </pre>

 <h3 class="section">11.39 truehd_core# TOC</h3>

Extract the core from a TrueHD stream, dropping ATMOS data. <h3 class="section">11.40 vp9_metadata# TOC</h3>

Modify metadata embedded in a VP9 stream. <dl class="table"> <dt><samp class="option">color_space</samp></dt> <dd> Set the color space value in the frame header. Note that any frame set to RGB will be implicitly set to PC range and that RGB is incompatible with profiles 0 and 2. <dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt601</samp>’</dt> <dt>‘<samp class="samp">bt709</samp>’</d
<dt>‘<samp class="samp">smpte170</samp>’</dt>
<dt>‘<samp class="samp">smpte240</samp>’</dt>
<dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">rgb</samp>’</dt> </dl>

</dd> <dt><samp class="option">color_range</samp></dt> <dd> Set the color range value in the frame header. Note that any value imposed by the color space will take precedence over this value. <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dt>‘<samp class="samp">pc</samp>’</dt> </dl> </dd> </dl>

 <h3 class="section">11.41 vp9_superframe# TOC</h3>

Merge VP9 invisible (alt-ref) frames back into VP9 superframes. This fixes merging of split/segmented VP9 streams where the alt-ref frame was split from its visible counterpart. <h3 class="section">11.42 vp9_superframe_split# TOC</h3>

Split VP9 superframes into single frames. <h3 class="section">11.43 vp9_raw_reorder#

TOC</h3>

Given a VP9 stream with correct timestamps but possibly out of order, insert additional show-existing-frame packets to correct the ordering. <h2 class=”chapter”>12 Format Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the muxers and demuxers. In addition each muxer or demuxer may support so-called private options, which are specific for that component. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class=”code”>AVFormatContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class=”table”> <dt><samp class=”option”>avioflags <var class=”var”>flags</var> (input/output)</samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>direct</samp>’</dt> <dd> Reduce buffering. </dd> </dl>

</dd> <dt><samp class=”option”>probesize <var class=”var”>integer</var> (input)</samp></dt> <dd> Set probing size in bytes, i.e. the size of the data to analyze to get stream information. A higher value will enable detecting more information in case it is dispersed into the stream, but will increase latency. Must be an integer not lesser than 32. It is 5000000 by default. </dd> <dt><samp class=”option”>max_probe_packets <var class=”var”>integer</var> (input)</samp></dt> <dd> Set the maximum number of buffered packets when probing a codec. Default is 2500 packets. </dd> <dt><samp class=”option”>packetsize <var class=”var”>integer</var> (output)</samp></dt> <dd> Set packet size. </dd> <dt><samp class=”option”>fflags <var class=”var”>flags</var></samp></dt> <dd> Set format flags. Some are implemented for a limited number of formats. Possible values for input files: <dl class=”table”> <dt>‘<samp class=”samp”>discardcorrupt</samp>’</dt> <dd> Discard corrupted packets. </dd> <dt>‘<samp class=”samp”>fastseek</samp>’</dt> <dd> Enable fast, but inaccurate seeks for some formats. </dd> <dt>‘<samp class=”samp”>genpts</samp>’</dt> <dd> Generate missing PTS if DTS is present. </dd> <dt>‘<samp class=”samp”>igndts</samp>’</dt> <dd> Ignore DTS if PTS is also set. In case the PTS is set, the DTS value is set to NOPTS. This is ignored when the <code class=”code”>nofillin</code> flag is set. </dd> <dt>‘<samp class=”samp”>ignidx</samp>’</dt> <dd> Ignore

index. </dd> <dt>‘<samp class="samp">nobuffer</samp>’</dt>
<dd> Reduce the latency introduced by buffering during initial input
streams analysis. </dd> <dt>‘<samp class="samp">nofillin</samp>’</dt>
<dd> Do not fill in missing values in packet fields that can be exactly calcu-
lated. </dd> <dt>‘<samp class="samp">noparse</samp>’</dt>
<dd> Disable AVParsers, this needs <code class="code">+nofillin</code>
too. </dd> <dt>‘<samp class="samp">sortdts</samp>’</dt>
<dd> Try to interleave output packets by DTS. At present, available only
for AVIs with an index. </dd> </dl>

Possible values for output files: <dl class="table"> <dt>‘<samp
class="samp">autobsf</samp>’</dt> <dd> Automatically ap-
ply bitstream filters as required by the output format. Enabled by default.
</dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. This
ensures that file and data checksums are reproducible and match be-
tween platforms. Its primary use is for regression testing. </dd>
<dt>‘<samp class="samp">flush_packets</samp>’</dt>
<dd> Write out packets immediately. </dd> <dt>‘<samp
class="samp">shortest</samp>’</dt> <dd> Stop muxing at the
end of the shortest stream. It may be needed to increase max_interleave_delta
to avoid flushing the longer streams before EOF. </dd> </dl>

</dd> <dt><samp class="option">seek2any <var class="var">integer</var>
(input)</samp></dt> <dd> Allow seeking
to non-keyframes on demuxer level when supported if set to 1. Default
is 0. </dd> <dt><samp class="option">analyzeduration <var
class="var">integer</var> (input)</samp></dt>
<dd> Specify how many microseconds are analyzed to probe the input.
A higher value will enable detecting more accurate information, but will
increase latency. It defaults to 5,000,000 microseconds = 5 seconds.
</dd> <dt><samp class="option">cryptokey <var class="var">hexadecimal
string</var> (input)</samp></dt> <dd> Set
decryption key. </dd> <dt><samp class="option">indexmem <var
class="var">integer</var> (input)</samp></dt>
<dd> Set max memory used for timestamp index (per stream).
</dd> <dt><samp class="option">rtbufsize <var class="var">integer</var>
(input)</samp></dt> <dd> Set max
memory used for buffering real-time frames. </dd> <dt><samp
class="option">fdebug <var class="var">flags</var> (input/output)</samp><
<dd> Print specific debug info. Possible values: <dl
class="table"> <dt>‘<samp class="samp">ts</samp>’</dt>
<dd> Print timestamp information. </dd> <dt>‘<samp
class="samp">id3v2</samp>’</dt> <dd> Print ID3v2 frame
data when demuxing files containing ID3v2 tags. </dd> </dl>

</dd> <dt><samp class="option">max_delay <var class="var">integer</var>

(input/output)</samp></dt> <dd> Set maximum muxing or demuxing delay in microseconds. </dd> <dt><samp class="option">fpsprobesize <var class="var">integer</var> (input)</samp></dt> <dd> Set number of frames used to probe fps. </dd> <dt><samp class="option">audio_preload <var class="var">integer</var> (output)</samp></dt> <dd> Set microseconds by which audio packets should be interleaved earlier. </dd> <dt><samp class="option">chunk_duration <var class="var">integer</var> (output)</samp></dt> <dd> Set microseconds for each chunk. </dd> <dt><samp class="option">chunk_size <var class="var">integer</var> (output)</samp>
<dd> Set size in bytes for each chunk. </dd> <dt><samp class="option">err_detect, f_err_detect <var class="var">flags</var> (input)</samp></dt> <dd> Set error detection flags. <code class="code">f_err_detect</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt> <dd> Verify embedded CRCs. </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> Detect bitstream specification deviations. </dd> <dt>‘<samp class="samp">buffer</samp>’</dt> <dd> Detect improper bitstream length. </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> Abort decoding on minor error detection. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> Consider things that violate the spec and have not been seen in the wild as errors. </dd> <dt>‘<samp class="samp">compliant</samp>’</dt> <dd> Consider all spec non compliancies as errors. </dd> <dt>‘<samp class="samp">aggressive</samp>’</dt> <dd> Consider things that a sane encoder should not do as an error. </dd> </dl>

</dd> <dt><samp class="option">max_interleave_delta <var class="var">integer</var> (output)</samp></dt> <dd> Set maximum buffering duration for interleaving. The duration is expressed in microseconds, and defaults to 10000000 (10 seconds). To ensure all the streams are interleaved correctly, libavformat will wait until it has at least one packet for each stream before actually writing any packets to the output file. When some streams are "sparse" (i.e. there are large gaps between successive packets), this can result in excessive buffering. This field specifies the maximum difference between the timestamps of the first and the last packet in the muxing queue, above which libavformat will output a packet regardless of whether it has queued a packet for all the streams. If set to 0, libavformat will continue buffering packets until it has a packet for each stream, regardless of the maximum timestamp difference between the buffered packets. </dd> <dt><samp class="option">use_wallclock_as_timestamps <var

class=”var”>integer</var> (input)</samp></dt>
<dd> Use wallclock as timestamps if set to 1. Default is 0. </dd>
<dt><samp class=”option”>avoid_negative_ts <var class=”var”>integer</var>
(output)</samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>make_non_negative</samp>’</dt>
<dd> Shift timestamps to make them non-negative. Also note that this affects only leading negative timestamps, and not non-monotonic negative timestamps. </dd> <dt>‘<samp class=”samp”>make_zero</samp>’</dt>
<dd> Shift timestamps so that the first timestamp is 0. </dd>
<dt>‘<samp class=”samp”>auto (default)</samp>’</dt>
<dd> Enables shifting when required by the target format. </dd>
<dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Disables shifting of timestamp. </dd> </dl>

When shifting is enabled, all output timestamps are shifted by the same amount. Audio, video, and subtitles desynching and relative timestamp differences are preserved compared to how they would have been without shifting. </dd> <dt><samp class=”option”>skip_initial_bytes <var class=”var”>integer</var> (input)</samp></dt>
<dd> Set number of bytes to skip before reading header and frames if set to 1. Default is 0. </dd> <dt><samp class=”option”>correct_ts_overflow
<var class=”var”>integer</var> (input)</samp></dt>
<dd> Correct single timestamp overflows if set to 1. Default is 1. </dd> <dt><samp class=”option”>flush_packets <var class=”var”>integer</var> (output)</samp></dt>
<dd> Flush the underlying I/O stream after each packet. Default is -1 (auto), which means that the underlying protocol will decide, 1 enables it, and has the effect of reducing the latency, 0 disables it and may increase IO throughput in some cases. </dd> <dt><samp class=”option”>output_ts_offset <var class=”var”>offset</var> (output)</samp></dt> <dd> Set the output time offset. <var class=”var”>offset</var> must be a time duration specification, see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. The offset is added by the muxer to the output timestamps. Specifying a positive offset means that the corresponding streams are delayed bt the time duration specified in <var class=”var”>offset</var>. Default value is <code class=”code”>0</code> (meaning that no offset is applied). </dd>
<dt><samp class=”option”>format_whitelist <var class=”var”>list</var>
(input)</samp></dt> <dd> "," separated list of allowed demuxers. By default all are allowed. </dd>
<dt><samp class=”option”>dump_separator <var class=”var”>string</var>
(input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation:

 <pre class="example-preformatted">ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class="option">max_streams <var class="var">integer</var> (input)</samp></dt> <dd> Specifies the maximum number of streams. This can be used to reject files that would require too many resources due to a large number of streams. </dd> <dt><samp class="option">skip_estimate_duration_from_pts <var class="var">bool</var> (input)</samp></dt> <dd> Skip estimation of input duration if it requires an additional probing for PTS at end of file. At present, applicable for MPEG-PS and MPEG-TS. </dd> <dt><samp class="option">duration_probesize <var class="var">integer</var> (input)</samp></dt> <dd> Set probing size, in bytes, for input duration estimation when it actually requires an additional probing for PTS at end of file (at present: MPEG-PS and MPEG-TS). It is aimed at users interested in better durations probing for itself, or indirectly because using the concat demuxer, for example. The typical use case is an MPEG-TS CBR with a high bitrate, high video buffering and ending cleaning with similar PTS for video and audio: in such a scenario, the large physical gap between the last video packet and the last audio packet makes it necessary to read many bytes in order to get the video stream duration. Another use case is where the default probing behaviour only reaches a single video frame which is not the last one of the stream due to frame reordering, so the duration is not accurate. Setting this option has a performance impact even for small files because the probing size is fixed. Default behaviour is a general purpose trade-off, largely adaptive, but the probing size will not be extended to get streams durations at all costs. Must be an integer not lesser than 1, or 0 for default behaviour. </dd> <dt><samp class="option">strict, f_strict <var class="var">integer</var> (input/output)</samp></dt> <dd> Specify how strictly to follow the standards. <code class="code">f_strict</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt> <dd> strictly conform to an older more strict version of the spec or reference software </dd> <dt>‘<samp class="samp">strict</samp>’</dt> <dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt> <dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> </dl>

 <h3 class="section">12.1 Format stream specifiers# TOC</h3>

Format stream specifiers allow selection of one or more streams that match specific properties. The exact semantics of stream specifiers is defined by the <code class="code">avformat_match_stream_specifier()</code> function declared in the <samp class="file">libavformat/avformat.h</samp> header and documented in the <a data-manual="ffmpeg" href="./ffmpeg.html#Stream-specifiers">Stream specifiers section in the ffmpeg(1) manual. <h2 class="chapter">13 Demuxers# TOC</h2>

Demuxers are configured elements in FFmpeg that can read the multimedia streams from a particular type of file. When you configure your FFmpeg build, all the supported demuxers are enabled by default. You can list all available ones using the configure option <code class="code">--list-demuxers</code>. You can disable all the demuxers using the configure option <code class="code">--disable-demuxers</code>, and selectively enable a single demuxer with the option <code class="code">--enable-demuxer=<var class="var">DEMUXER</var></code>, or disable it with the option <code class="code">--disable-demuxer=<var class="var">DEMUXER</var></code>. The option <code class="code">-demuxers</code> of the ff* tools will display the list of enabled demuxers. Use <code class="code">-formats</code> to view a combined list of enabled demuxers and muxers. The description of some of the currently available demuxers follows. <h3 class="section">13.1 aa# TOC</h3>

Audible Format 2, 3, and 4 demuxer. This demuxer is used to demux Audible Format 2, 3, and 4 (.aa) files. <h3 class="section">13.2 aac# TOC</h3>

Raw Audio Data Transport Stream AAC demuxer. This demuxer is used to demux an ADTS input containing a single AAC stream alongwith any ID3v1/2 or APE tags in it. <h3 class="section">13.3 apng# TOC</h3>

Animated Portable Network Graphics demuxer. This demuxer is used to demux APNG files. All headers, but the PNG signature, up to (but not including) the first fcTL chunk are transmitted as extradata. Frames are then split as being all the chunks between two fcTL ones, or between the last fcTL and IEND chunks. <dl class="table"> <dt><samp class="option">-ignore_loop <var class="var">bool</var></samp></dt> <dd> Ignore the loop variable in the file if set. Default is enabled. </dd> <dt><samp class="option">-max_fps <var class="var">int</var></samp></dt> <dd> Maximum framerate in frames per second. Default of 0 imposes no limit. </dd> <dt><samp class="option">-default_fps <var class="var">int</var></samp></dt> <dd> Default framerate in frames per second when none is specified in the file (0 meaning as fast as possible). Default is 15. </dd> </dl>

 <h3 class="section">13.4 asf# TOC</h3>

Advanced Systems Format demuxer. This demuxer is used to demux ASF files and MMS network streams. <dl class="table"> <dt><samp class="option">-no_resync_search <var class="var">bool</var></samp></dt> <dd> Do not try to resynchronize by looking for a certain optional start code. </dd> </dl>

 <h3 class="section">13.5 astc# TOC</h3>

ASTC (Adaptive Scalable Texture Compression) demuxer. This demuxer reads the 16-byte <samp class="file">.astc</samp> header, hands it to the decoder as extradata, and emits the compressed image as a single packet. <h3 class="section">13.6 concat# TOC</h3>

Virtual concatenation script demuxer. This demuxer reads a list of files and other directives from a text file and demuxes them one after the other, as if all their packets had been muxed together. The timestamps in the files are adjusted so that the first file starts at 0 and each next file starts where the previous one finishes. Note that it is done globally and may cause gaps if all streams do not have exactly the same length. All files must have the same streams (same codecs, same time base, etc.). The duration of each file is used to adjust the timestamps of the next file: if the duration is incorrect (because it was computed using the bit-rate or because the file is truncated, for example), it can cause artifacts. The <code class="code">duration</code> directive can be used to override the duration stored in each file.

 <h4 class="subsection">13.6.1 Syntax# TOC</h4>

The script is a text file in extended-ASCII, with one directive per line.

Empty lines, leading spaces and lines starting with ’#’ are ignored. The

following directive is recognized:

<dl class="table">

<dt><samp class="option"><code class="code">file <var class="var">path</var></code></samp></dt

<dd> Path to a file to read; special characters and spaces must be escaped with

backslash or single quotes.

All subsequent file-related directives apply to that file.

</dd>

<dt><samp class="option"><code class="code">ffconcat version 1.0</code></samp></dt>

<dd> Identify the script type and version.

To make FFmpeg recognize the format automatically, this directive must

appear exactly as is (no extra space or byte-order-mark) on the very first

line of the script.

</dd>

<dt><samp class="option"><code class="code">duration <var class="var">dur</var></code></samp>

<dd> Duration of the file. This information can be specified from the file;

specifying it here may be more efficient or help if the information from the

file is not available or accurate.

If the duration is set for all files, then it is possible to seek in the

whole concatenated video.

</dd>

<dt><samp class="option"><code class="code">inpoint <var class="var">timestamp</var></code></sa

<dd> In point of the file. When the demuxer opens the file it instantly seeks to the

specified timestamp. Seeking is done so that all streams can be presented

successfully at In point.

This directive works best with intra frame codecs, because for non-intra frame

ones you will usually get extra packets before the actual In point and the

decoded content will most likely contain frames before In point too.

For each file, packets before the file In point will have timestamps less than

the calculated start timestamp of the file (negative in case of the first

file), and the duration of the files (if not specified by the <code class="code">duration</code>

directive) will be reduced based on their specified In point.

Because of potential packets before the specified In point, packet timestamps

may overlap between two concatenated files.

</dd>

<dt><samp class="option"><code class="code">outpoint <var class="var">timestamp</var></code></

<dd> Out point of the file. When the demuxer reaches the specified decoding

timestamp in any of the streams, it handles it as an end of file condition and

skips the current and all the remaining packets from all streams.

Out point is exclusive, which means that the demuxer will not output packets

with a decoding timestamp greater or equal to Out point.

This directive works best with intra frame codecs and formats where all streams

are tightly interleaved. For non-intra frame codecs you will usually get

additional packets with presentation timestamp after Out point therefore the

decoded content will most likely contain frames after Out point too. If your

streams are not tightly interleaved you may not get all the packets from all

streams before Out point and you may only will be able to decode the earliest

stream until Out point.

The duration of the files (if not specified by the <code class="code">duration</code>

directive) will be reduced based on their specified Out point.

</dd>

<dt><samp class="option"><code class="code">file_packet_metadata <var class="var">key=value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

This directive is deprecated, use <code class="code">file_packet_meta</code> instead.

</dd>

<dt><samp class="option"><code class="code">file_packet_meta <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

</dd>

<dt><samp class=”option”><code class=”code”>option <var class=”var”>key</var> <var class=”var”>value</var></code></samp></dt>

<dd> Option to access, open and probe the file.

Can be present multiple times.

</dd>

<dt><samp class=”option”><code class=”code”>stream</code></samp></dt>

<dd> Introduce a stream in the virtual file.

All subsequent stream-related directives apply to the last introduced

stream.

Some streams properties must be set in order to allow identifying the

matching streams in the subfiles.

If no streams are defined in the script, the streams from the first file are

copied.

</dd>

<dt><samp class=”option”><code class=”code”>exact_stream_id <var class=”var”>id</var></code></samp></dt>

<dd> Set the id of the stream.

If this directive is given, the string with the corresponding id in the

subfiles will be used.

This is especially useful for MPEG-PS (VOB) files, where the order of the

streams is not reliable.

</dd>

<dt><samp class=”option”><code class=”code”>stream_meta <var class=”var”>key</var> <var class=”var”>value</var></code></samp></dt>

<dd> Metadata for the stream.

Can be present multiple times.

</dd>

<dt><samp class=”option”><code class=”code”>stream_codec <var class=”var”>value</var></code></samp></dt>

<dd> Codec for the stream.

</dd>

<dt><samp class=”option”><code class=”code”>stream_extradata <var class=”var”>hex_string</var></code></samp></dt>

<dd> Extradata for the string, encoded in hexadecimal.

</dd>

<dt><samp class=”option”><code class=”code”>chapter <var class=”var”>id</var> <var class=”var”>start</var> <var class=”var”>end</var></code></samp></dt>

<dd> Add a chapter. <var class=”var”>id</var> is an unique identifier, possibly small and

consecutive.

</dd>

</dl>

 <h4 class=”subsection”>13.6.2 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>safe</samp></dt> <dd> If set to 1, reject unsafe file paths and directives. A file path is considered safe if it does not contain a protocol specification and is relative and all components only contain characters from the portable character set (letters, digits, period, underscore and hyphen) and have no period at the beginning of a component. If set to 0, any file name is accepted. The default is 1. </dd> <dt><samp class=”option”>auto_convert</samp></dt> <dd> If set to 1, try to perform automatic conversions on packet data to make the streams concatenable. The default is 1. Currently, the only conversion is adding the h264_mp4toannexb bitstream filter to H.264 streams in MP4 format. This is necessary in particular if there are resolution changes. </dd> <dt><samp class=”option”>segment_time_metadata</samp></dt> <dd> If set to 1, every packet will contain the <var class=”var”>lavf.concat.start_time</var> and the <var class=”var”>lavf.concat.duration</var> packet metadata values

which are the start_time and the duration of the respective file segments in the concatenated output expressed in microseconds. The duration metadata is only set if it is known based on the concat file. The default is 0. </dd> <dt><samp class="option">chapter_per_file</samp></dt> <dd> If set to 1, add a chapter for each file, named after the <code class="code">title</code> metadata of the file or, failing that, its file name, and carrying the rest of the file’s metadata. The files are opened in order when the script is read to learn their durations, stopping after the first file whose duration is unknown, which gets the last chapter; when all durations are known, this also makes the total duration known and the output seekable. The default is 0. </dd> </dl>

 <h4 class="subsection">13.6.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use absolute filenames and include some comments: <pre class="example-preformatted"># my first filename file /mnt/share/file-1.wav # my second filename including whitespace file '/mnt/share/file 2.wav' # my third filename including whitespace plus single quote file '/mnt/share/file 3'\''.wav' </pre>

Allow for input format auto-probing, use safe filenames and set the duration of the first file: <pre class="example-preformatted">ffconcat version 1.0

file file-1.wav duration 20.0

file subdir/file-2.wav </pre>

 <h3 class="section">13.7 dash# TOC</h3>

Dynamic Adaptive Streaming over HTTP demuxer. This demuxer presents all AVStreams found in the manifest. By setting the discard flags on AVStreams the caller can decide which streams to actually receive. Each stream mirrors the <code class="code">id</code> and <code class="code">bandwidth</code> properties from the <code class="code"><Representation></code> as metadata keys named "id" and "variant_bitrate" respectively. <h4 class="subsection">13.7.1 Options# TOC</h4>

This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">cenc_decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption

(CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>cenc_decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> </dl>

 <h3 class=”section”>13.8 dvdvideo# TOC</h3>

DVD-Video demuxer, powered by libdvdnav and libdvdread. Can directly ingest DVD titles, specifically sequential PGCs, into a conversion pipeline. Menu assets, such as background video or audio, can also be demuxed given the menu’s coordinates (at best effort). Block devices (DVD drives), ISO files, and directory structures are accepted. Activate with <code class=”code”>-f dvdvideo</code> in front of one of these inputs. This demuxer does NOT have decryption code of any kind. You are on your own working with encrypted DVDs, and should not expect support on the matter. Underlying playback is handled by libdvdnav, and structure parsing by libdvdread. FFmpeg must be built with GPL library support available as well as the configure switches <code class=”code”>--enable-libdvdnav</code> and <code class=”code”>--enable-libdvdread</code>. You will need to provide either the desired "title number" or exact PGC/PG coordinates. Many open-source DVD players and tools can aid in providing this information. If not specified, the demuxer will default to title 1 which works for many discs. However, due to the flexibility of the format, it is recommended to check manually. There are many discs that are authored strangely or with invalid headers. If the input is a real DVD drive, please note that there are some drives which may silently fail on reading bad sectors from the disc, returning random bits instead which is effectively corrupt data. This is especially prominent on aging or rotting discs. A second pass and integrity checks would be needed to detect the corruption. This is not an FFmpeg issue. <h4 class=”subsection”>13.8.1 Background# TOC</h4>

DVD-Video is not a directly accessible, linear container format in the traditional sense. Instead, it allows for complex and programmatic playback of carefully muxed MPEG-PS streams that are stored in headerless VOB files. To the end-user, these streams are known simply as "titles", but the actual logical playback sequence is defined by one or more "PGCs", or Program Group Chains, within the title. The PGC is in turn comprised of multiple "PGs", or Programs", which are the actual video segments (and for a typical video feature, sequentially ordered). The PGC structure, along with stream layout and metadata, are stored in IFO files

that need to be parsed. PGCs can be thought of as playlists in easier terms. An actual DVD player relies on user GUI interaction via menus and an internal VM to drive the direction of demuxing. Generally, the user would either navigate (via menus) or automatically be redirected to the PGC of their choice. During this process and the subsequent playback, the DVD player’s internal VM also maintains a state and executes instructions that can create jumps to different sectors during playback. This is why libdvdnav is involved, as a linear read of the MPEG-PS blobs on the disc (VOBs) is not enough to produce the right sequence in many cases. There are many other DVD structures (a long subject) that will not be discussed here. NAV packets, in particular, are handled by this demuxer to build accurate timing but not emitted as a stream. For a good high-level understanding, refer to: https://code.videolan.org/videolan/libdvdnav/-/blob/master/doc/dvd_structures <h4 class="subsection">13.8.2 Options# TOC</h4>

This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">title <var class="var">int</var></samp></dt> <dd> The title number to play. Must be set if <samp class="option">pgc</samp> and <samp class="option">pg</samp> are not set. Not applicable to menus. Default is 0 (auto), which currently only selects the first available title (title 1) and notifies the user about the implications. </dd> <dt><samp class="option">chapter_start <var class="var">int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to start at. Not applicable to menus. Default is 1. </dd> <dt><samp class="option">chapter_end <var class="var">int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to end at. Not applicable to menus. Default is 0, which is a special value to signal end at the last possible chapter. </dd> <dt><samp class="option">angle <var class="var">int</var></samp></dt> <dd> The video angle number, referring to what is essentially an additional video stream that is composed from alternate frames interleaved in the VOBs. Not applicable to menus. Default is 1. </dd> <dt><samp class="option">region <var class="var">int</var></samp></dt> <dd> The region code to use for playback. Some discs may use this to default playback at a particular angle in different regions. This option will not affect the region code of a real DVD drive, if used as an input. Not applicable to menus. Default is 0, "world". </dd> <dt><samp class="option">menu <var class="var">bool</var></samp></dt> <dd> Demux menu assets instead of navigating a title. Requires exact coordinates of the menu (<samp class="option">menu_lu</samp>, <samp class="option">menu_vts</samp>, <samp class="option">pgc</samp>, <samp class="option">pg</samp>). Default is false. </dd>

<dt><samp class=”option”>menu_lu <var class=”var”>int</var></samp></dt> <dd> The menu language to demux. In DVD, menus are grouped by language. Default is 1, the first language unit. </dd> <dt><samp class=”option”>menu_vts <var class=”var”>int</var></samp></dt> <dd> The VTS where the menu lives, or 0 if it is a VMG menu (root-level). Default is 1, menu of the first VTS. </dd> <dt><samp class=”option”>pgc <var class=”var”>int</var></samp></dt> <dd> The entry PGC to start playback, in conjunction with <samp class=”option”>pg</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Must be explicitly set for menus. Default is 0, automatically resolve from value of <samp class=”option”>title</samp>. </dd> <dt><samp class=”option”>pg <var class=”var”>int</var></samp></dt> <dd> The entry PG to start playback, in conjunction with <samp class=”option”>pgc</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Default is 1, the first PG of the PGC. </dd> <dt><samp class=”option”>preindex <var class=”var”>bool</var></samp></dt> <dd> Enable this to have accurate chapter (PTT) markers and duration measurement, which requires a slow second pass read in order to index the chapter marker timestamps from NAV packets. This is non-ideal extra work for real optical drives. It is recommended and faster to use this option with a backup of the DVD structure stored on a hard drive. Not compatible with <samp class=”option”>pgc</samp> and <samp class=”option”>pg</samp>. Default is 0, false. </dd> <dt><samp class=”option”>trim <var class=”var”>bool</var></samp></dt> <dd> Skip padding cells (i.e. cells shorter than 1 second) from the beginning. There exist many discs with filler segments at the beginning of the PGC, often with junk data intended for controlling a real DVD player’s buffering speed and with no other material data value. Not applicable to menus. Default is 1, true. </dd> </dl>

 <h4 class=”subsection”>13.8.3 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Open title 3 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -title 3 -i <path to DVD> ... </pre>

Open chapters 3-6 from title 1 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvdvideo -chapter_start 3 -chapter_end 6 -title 1 -i <path to DVD> ... </pre>

Open only chapter 5 from title 1 from a given DVD structure: <pre class=”example-preformatted”>ffmpeg -f dvd-

video -chapter_start 5 -chapter_end 5 -title 1 -i <path to DVD> ... </pre>

Demux menu with language 1 from VTS 1, PGC 1, starting at PG 1: <pre class="example-preformatted">ffmpeg -f dvd-video -menu 1 -menu_lu 1 -menu_vts 1 -pgc 1 -pg 1 -i <path to DVD> ... </pre>

 <h3 class="section">13.9 ea# TOC</h3>

Electronic Arts Multimedia format demuxer. This format is used by various Electronic Arts games. <h4 class="subsection">13.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">merge_alpha <var class="var">bool</var></samp></dt> <dd> Normally the VP6 alpha channel (if exists) is returned as a secondary video stream, by setting this option you can make the demuxer return a single video stream which contains the alpha channel in addition to the ordinary video. </dd> </dl>

 <h3 class="section">13.10 imf# TOC</h3>

Interoperable Master Format demuxer. This demuxer presents audio and video streams found in an IMF Composition, as specified in SMPTE ST 2067-2.   <pre class="example-preformatted">ffmpeg [-assetmaps <path of ASSETMAP1>,<path of ASSETMAP2>,...] -i <path of CPL> ... </pre>

If <code class="code">-assetmaps</code> is not specified, the demuxer looks for a file called <samp class="file">ASSETMAP.xml</samp> in the same directory as the CPL. <h3 class="section">13.11 flv, live_flv, kux# TOC</h3>

Adobe Flash Video Format demuxer. This demuxer is used to demux FLV files and RTMP network streams. In case of live network streams, if you force format, you may use live_flv option instead of flv to survive timestamp discontinuities. KUX is a flv variant used on the Youku platform.   <pre class="example-preformatted">ffmpeg -f flv -i myfile.flv ... ffmpeg -f live_flv -i rtmp://<any.server>/anything/key .... </pre>

<dl class="table"> <dt><samp class="option">-flv_metadata <var class="var">bool</var></samp></dt> <dd> Allocate the streams according to the onMetaData array content. </dd> <dt><samp class="option">-flv_ignore_prevtag <var class="var">bool</var></samp></dt> <dd> Ignore the size of previous tag value. </dd> <dt><samp class="option">-flv_full_metadata <var class="var">bool</var></samp></dt> <dd> Output all context of the onMetadata. </dd> </dl>

 <h3 class="section">13.12 gif# TOC</h3>

Animated GIF demuxer. It accepts the following options: <dl class="table"> <dt><samp class="option">min_delay</samp></dt> <dd> Set the minimum valid delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 2. </dd> <dt><samp class="option">max_gif_delay</samp></dt> <dd> Set the maximum valid delay between frames in hundredth of seconds. Range is 0 to 65535. Default value is 65535 (nearly eleven minutes), the maximum value allowed by the specification. </dd> <dt><samp class="option">default_delay</samp></dt> <dd> Set the default delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 10. </dd> <dt><samp class="option">ignore_loop</samp></dt> <dd> GIF files can contain information to loop a certain number of times (or infinitely). If <samp class="option">ignore_loop</samp> is set to 1, then the loop setting from the input will be ignored and looping will not occur. If set to 0, then looping will occur and will cycle the number of times according to the GIF. Default value is 1. </dd> </dl>

For example, with the overlay filter, place an infinitely looping GIF over another video: <pre class="example-preformatted">ffmpeg -i input.mp4 -ignore_loop 0 -i input.gif -filter_complex overlay=shortest=1 out.mkv </pre>

Note that in the above example the shortest option for overlay filter is used to end the output video at the length of the shortest input file, which in this case is <samp class="file">input.mp4</samp> as the GIF in this example loops infinitely. <h3 class="section">13.13 hls# TOC</h3>

HLS demuxer Apple HTTP Live Streaming demuxer. This demuxer presents all AVStreams from all variant streams. The id field is set to the bitrate variant index number. By setting the discard flags on AVStreams (by pressing ’a’ or ’v’ in ffplay), the caller can decide which variant streams to actually receive. The total bitrate of the variant that the stream belongs to is available in a

metadata key named "variant_bitrate". It is unset when the stream is shared by variants with different bitrates. The value is always available in the corresponding program’s metadata. It accepts the following options: <dl class="table"> <dt><samp class="option">live_start_index</samp></dt> <dd> segment index to start live streams at (negative values are from the end). </dd> <dt><samp class="option">prefer_x_start</samp></dt> <dd> prefer to use #EXT-X-START if it’s in playlist instead of live_start_index. </dd> <dt><samp class="option">allowed_extensions</samp></dt> <dd> ’,’ separated list of file extensions that hls is allowed to access. </dd> <dt><samp class="option">extension_picky</samp></dt> <dd> This blocks disallowed extensions from probing It also requires all available segments to have matching extensions to the format except mpegts, which is always allowed. It is recommended to set the whitelists correctly instead of depending on extensions Enabled by default. </dd> <dt><samp class="option">max_reload</samp></dt> <dd> Maximum number of times a insufficient list is attempted to be reloaded. Default value is 1000. </dd> <dt><samp class="option">m3u8_hold_counters</samp></dt> <dd> The maximum number of times to load m3u8 when it refreshes without new segments. Default value is 1000. </dd> <dt><samp class="option">http_persistent</samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP streams. Enabled by default. </dd> <dt><samp class="option">http_multiple</samp></dt> <dd> Use multiple HTTP connections for downloading HTTP segments. Enabled by default for HTTP/1.1 servers. </dd> <dt><samp class="option">http_seekable</samp></dt> <dd> Use HTTP partial requests for downloading HTTP segments. 0 = disable, 1 = enable, -1 = auto, Default is auto. </dd> <dt><samp class="option">seg_format_options</samp></dt> <dd> Set options for the demuxer of media segments using a list of key=value pairs separated by <code class="code">:</code>. </dd> <dt><samp class="option">seg_max_retry</samp></dt> <dd> Maximum number of times to reload a segment on error, useful when segment skip on network error is not desired. Default value is 0. </dd> </dl>

 <h3 class="section">13.14 image2# TOC</h3>

Image file demuxer. This demuxer reads from a list of image files specified by a pattern. The syntax and meaning of the pattern is specified by the option <var class="var">pattern_type</var>. The pattern may contain a suffix which is used to automatically determine the format of the images contained in the files. The size, the pixel format, and the format of each image must be the same for all the files in the sequence. This demuxer accepts the following options: <dl

class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate for the video stream. It defaults to 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the input. Default value is 0. </dd> <dt><samp class=”option”>pattern_type</samp></dt> <dd> Select the pattern type used to interpret the provided filename. <var class=”var”>pattern_type</var> accepts one of the following values. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Disable pattern matching, therefore the video will only contain the specified image. You should use this option if you do not want to create sequences from multiple images and your filenames may contain special pattern characters. </dd> <dt><samp class=”option”>sequence</samp></dt> <dd> Select a sequence pattern type, used to specify a sequence of files indexed by sequential numbers. A sequence pattern may contain the string "%d" or "%0<var class=”var”>N</var>d", which specifies the position of the characters representing a sequential number in each filename matched by the pattern. If the form "%d0<var class=”var”>N</var>d" is used, the string representing the number in each filename is 0-padded and <var class=”var”>N</var> is the total number of 0-padded digits representing the number. The literal character ’%’ can be specified in the pattern with the string "%%". If the sequence pattern contains "%d" or "%0<var class=”var”>N</var>d", the first filename of the file list specified by the pattern must contain a number inclusively contained between <var class=”var”>start_number</var> and <var class=”var”>start_number</var>+<var class=”var”>start_number_range</var>-1, and all the following numbers must be sequential. For example the pattern "img-%03d.bmp" will match a sequence of filenames of the form <samp class=”file”>img-001.bmp</samp>, <samp class=”file”>img-002.bmp</samp>, ..., <samp class=”file”>img-010.bmp</samp>, etc.; the pattern "i%%m%%g-%d.jpg" will match a sequence of filenames of the form <samp class=”file”>i%m%g-1.jpg</samp>, <samp class=”file”>i%m%g-2.jpg</samp>, ..., <samp class=”file”>i%m%g-10.jpg</samp>, etc. Note that the pattern must not necessarily contain "%d" or "%0<var class=”var”>N</var>d", for example to convert a single image file <samp class=”file”>img.jpeg</samp> you can employ the command: <pre class=”example-preformatted”>ffmpeg -i img.jpeg img.png </pre>

</dd> <dt><samp class=”option”>glob</samp></dt> <dd> Select a glob wildcard pattern type. The pattern is interpreted like a <code class=”code”>glob()</code> pattern. This is only selectable if libavformat was compiled with globbing support. </dd> </dl>

Default value is <var class=”var”>sequence</var>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Set

the pixel format of the images to read. If not specified the pixel format is guessed from the first image file in the sequence. </dd> <dt><samp class="option">start_number</samp></dt> <dd> Set the index of the file matched by the image file pattern to start to read from. Default value is 0. </dd> <dt><samp class="option">start_number_range</samp></dt> <dd> Set the index interval range to check when looking for the first image file in the sequence, starting from <var class="var">start_number</var>. Default value is 5. </dd> <dt><samp class="option">ts_from_file</samp></dt> <dd> If set to 1, will set frame timestamp to modification time of image file. Note that monotonity of timestamps is not provided: images go in the same order as without this option. Default value is 0. If set to 2, will set frame timestamp to the modification time of the image file in nanosecond precision. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video size of the images to read. If not specified the video size is guessed from the first image file in the sequence. </dd> <dt><samp class="option">export_path_metadata</samp></dt> <dd> If set to 1, will add two extra fields to the metadata found in input, making them also available for other filters (see <var class="var">drawtext</var> filter for examples). Default value is 0. The extra fields are described below: <dl class="table"> <dt><samp class="option">lavf.image2dec.source_path</samp></dt> <dd> Corresponds to the full path to the input file being read. </dd> <dt><samp class="option">lavf.image2dec.source_basename</samp></dt> <dd> Corresponds to the name of the file being read. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">13.14.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> for creating a video from the images in the file sequence <samp class="file">img-001.jpeg</samp>, <samp class="file">img-002.jpeg</samp>, ..., assuming an input frame rate of 10 frames per second: <pre class="example-preformatted">ffmpeg -framerate 10 -i 'img-%03d.jpeg' out.mkv </pre>

As above, but start by reading from a file with index 100 in the sequence: <pre class="example-preformatted">ffmpeg -framerate 10 -start_number 100 -i 'img-%03d.jpeg' out.mkv </pre>

Read images matching the "*.png" glob pattern , that is all the files terminating with the ".png" suffix: <pre class="example-preformatted">ffmpeg -framerate 10 -pattern_type glob -i "*.png" out.mkv </pre>

 <h3 class="section">13.15 jpeg_mpf<a class="anchor hidden-xs" href="#jpeg_005fmpf"

aria-hidden=”true”># TOC</h3>

JPEG Multi-Picture Format demuxer. Exposes each JPEG image stored in a CIPA DC-007 MP file as a separate stream, in the order of the MP Entries, with the First Individual Image flagged as the default stream. If the file contains an ISO 21496-1 gain map, as is the case for JPEG Ultra HDR, the primary image and the gain map are linked by a ‘<samp class=”samp”>gain_map</samp>’ stream group, and the gain map parameters are exported as frame side data by the decoder. Images of any other type (e.g. large thumbnails or multi-view images) are exposed as additional streams. This demuxer accepts the following options, which behave like the options of the same name of the ‘<samp class=”samp”>image2</samp>’ demuxer: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate of the video streams. The default is 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the images in the file. The default is 0. This requires seekable input. </dd> </dl>

 <h3 class=”section”>13.16 ktx# TOC</h3>

Khronos Texture 1.0 demuxer for ASTC textures. This demuxer reads the KTX 1.0 header and maps the ASTC GL internal format to the block footprint the decoder needs, so only files using the ASTC formats are accepted. A <code class=”code”>KTXorientation</code> key is preserved as stream display metadata, and the texture is emitted as a single packet. <h3 class=”section”>13.17 libgme# TOC</h3>

The Game Music Emu library is a collection of video game music file emulators. See https://bitbucket.org/mpyne/game-music-emu/overview for more information. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>track_index</samp></dt> <dd> Set the index of which track to demux. The demuxer can only export one track. Track indexes start at 0. Default is to pick the first track. Number of tracks is exported as <var class=”var”>tracks</var> metadata entry. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sampling rate of the exported track. Range is 1000 to 999999. Default is 44100. </dd> <dt><samp class=”option”>max_size (bytes)</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read.

Default is 50 MiB. </dd> </dl>

 <h3 class="section">13.18 libmodplug# TOC</h3>

ModPlug based module demuxer See https://github.com/Konstanty/libmodplug It will export one 2-channel 16-bit 44.1 kHz audio stream. Optionally, a <code class="code">pal8</code> 16-color video stream can be exported with or without printed metadata. It accepts the following options: <dl class="table"> <dt><samp class="option">noise_reduction</samp></dt> <dd> Apply a simple low-pass filter. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class="option">reverb_depth</samp></dt> <dd> Set amount of reverb. Range 0-100. Default is 0. </dd> <dt><samp class="option">reverb_delay</samp></dt> <dd> Set delay in ms, clamped to 40-250 ms. Default is 0. </dd> <dt><samp class="option">bass_amount</samp></dt> <dd> Apply bass expansion a.k.a. XBass or megabass. Range is 0 (quiet) to 100 (loud). Default is 0. </dd> <dt><samp class="option">bass_range</samp></dt> <dd> Set cutoff i.e. upper-bound for bass frequencies. Range is 10-100 Hz. Default is 0. </dd> <dt><samp class="option">surround_depth</samp></dt> <dd> Apply a Dolby Pro-Logic surround effect. Range is 0 (quiet) to 100 (heavy). Default is 0. </dd> <dt><samp class="option">surround_delay</samp></dt> <dd> Set surround delay in ms, clamped to 5-40 ms. Default is 0. </dd> <dt><samp class="option">max_size</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Range is 0 to 100 MiB. 0 removes buffer size limit (not recommended). Default is 5 MiB. </dd> <dt><samp class="option">video_stream_expr</samp></dt> <dd> String which is evaluated using the eval API to assign colors to the generated video stream. Variables which can be used are <code class="code">x</code>, <code class="code">y</code>, <code class="code">w</code>, <code class="code">h</code>, <code class="code">t</code>, <code class="code">speed</code>, <code class="code">tempo</code>, <code class="code">order</code>, <code class="code">pattern</code> and <code class="code">row</code>. </dd> <dt><samp class="option">video_stream</samp></dt> <dd> Generate video stream. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class="option">video_stream_w</samp></dt> <dd> Set video frame width in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class="option">video_stream_h</samp></dt> <dd> Set video frame height in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp

class=”option”>video_stream_ptxt</samp></dt> <dd> Print metadata on video stream. Includes <code class=”code”>speed</code>, <code class=”code”>tempo</code>, <code class=”code”>order</code>, <code class=”code”>pattern</code>, <code class=”code”>row</code> and <code class=”code”>ts</code> (time in ms). Can be 1 (on) or 0 (off). Default is 1. </dd> </dl>

 <h3 class=”section”>13.19 libopenmpt# TOC</h3>

libopenmpt based module demuxer See https://lib.openmpt.org/libopenmpt/ for more information. Some files have multiple subsongs (tracks) this can be set with the <samp class=”option”>subsong</samp> option. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>subsong</samp></dt> <dd> Set the subsong index. This can be either ’all’, ’auto’, or the index of the subsong. Subsong indexes start at 0. The default is ’auto’. The default value is to let libopenmpt choose. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> Set the channel layout. Valid values are 1, 2, and 4 channel layouts. The default value is STEREO. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate for libopenmpt to output. Range is from 1000 to INT_MAX. The value default is 48000. </dd> </dl>

 <h3 class=”section”>13.20 mcc# TOC</h3>

Demuxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-608 and CEA-708), ancillary time code, pan-scan data, etc. By default, for backward compatibility, the MCC demuxer extracts just the EIA-608 and CEA-708 closed captions and returns a <code class=”code”>EIA_608</code> stream, ignoring all other VANC data. You can change it to return all VANC data in a <code class=”code”>SMPTE_436M_ANC</code> data stream by setting <samp class=”option”>-eia608_extract 0</samp> <h4 class=”subsection”>13.20.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert a MCC file to Scenarist (SCC) format: <pre class=”example-preformatted”>ffmpeg -i CC.mcc -c:s copy CC.scc </pre> Note that the SCC for-

mat only supports EIA-608, so this will discard all other data such as CEA-708 extensions. Merge a MCC file into a MXF file: <pre class=”example-preformatted”>ffmpeg -i video_and_audio.mxf -eia608_extract 0 -i CC.mcc -c copy -map 0 -map 1 out.mxf </pre> This retains all VANC data and inserts it into the output MXF file as a <code class=”code”>SMPTE_436M_ANC</code> data stream.

 <h3 class=”section”>13.21 mov/mp4/3gp# TOC</h3>

Demuxer for Quicktime File Format & ISO/IEC Base Media File Format (ISO/IEC 14496-12 or MPEG-4 Part 12, ISO/IEC 15444-12 or JPEG 2000 Part 12). Registered extensions: mov, mp4, m4a, 3gp, 3g2, mj2, psp, m4b, ism, ismv, isma, f4v <h4 class=”subsection”>13.21.1 Options# TOC</h4>

This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>enable_drefs</samp></dt> <dd> Enable loading of external tracks, disabled by default. Enabling this can theoretically leak information in some use cases. </dd> <dt><samp class=”option”>use_absolute_path</samp></dt> <dd> Allows loading of external tracks via absolute paths, disabled by default. Enabling this poses a security risk. It should only be enabled if the source is known to be non-malicious. </dd> <dt><samp class=”option”>seek_streams_individually</samp></dt> <dd> When seeking, identify the closest point in each stream individually and demux packets in that stream from identified point. This can lead to a different sequence of packets compared to demuxing linearly from the beginning. Default is true. </dd> <dt><samp class=”option”>ignore_editlist</samp></dt> <dd> Ignore any edit list atoms. The demuxer, by default, modifies the stream index to reflect the timeline described by the edit list. Default is false. </dd> <dt><samp class=”option”>advanced_editlist</samp></dt> <dd> Modify the stream index to reflect the timeline described by the edit list. <code class=”code”>ignore_editlist</code> must be set to false for this option to be effective. If both <code class=”code”>ignore_editlist</code> and this option are set to false, then only the start of the stream index is modified to reflect initial dwell time or starting timestamp described by the edit list. Default is true. </dd> <dt><samp class=”option”>ignore_chapters</samp></dt> <dd> Don’t parse chapters. This includes GoPro ’HiLight’ tags/moments. Note that chapters are only parsed when input is seekable. Default is false.

 </dd> <dt><samp class=”option”>use_mfra_for</samp></dt>
<dd> For seekable fragmented input, set fragment’s starting timestamp from media fragment random access box, if present. Following options are available: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Auto-detect whether to set mfra timestamps as PTS or DTS (default) </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt>
<dd> Set mfra timestamps as DTS </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Set mfra timestamps as PTS </dd> <dt>‘<samp class=”samp”>0</samp>’</dt>
<dd> Don’t use mfra box to set timestamps </dd> </dl>

</dd> <dt><samp class=”option”>use_tfdt</samp></dt> <dd> For fragmented input, set fragment’s starting timestamp to <code class=”code”>baseMediaDecodeTime</code> from the <code class=”code”>tfdt</code> box. Default is enabled, which will prefer to use the <code class=”code”>tfdt</code> box to set DTS. Disable to use the <code class=”code”>earliest_presentation_time</code> from the <code class=”code”>sidx</code> box. In either case, the timestamp from the <code class=”code”>mfra</code> box will be used if it’s available and <code class=”code”>use_mfra_for</code> is set to pts or dts. </dd> <dt><samp class=”option”>export_all</samp></dt>
<dd> Export unrecognized boxes within the <var class=”var”>udta</var> box as metadata entries. The first four characters of the box type are set as the key. Default is false. </dd> <dt><samp class=”option”>export_xmp</samp></dt>
<dd> Export entire contents of <var class=”var”>XMP_</var> box and <var class=”var”>uuid</var> box as a string with key <code class=”code”>xmp</code>. Note that if <code class=”code”>export_all</code> is set and this option isn’t, the contents of <var class=”var”>XMP_</var> box are still exported but with key <code class=”code”>XMP_</code>. Default is false. </dd> <dt><samp class=”option”>activation_bytes</samp></dt>
<dd> 4-byte key required to decrypt Audible AAX and AAX+ files. See Audible AAX subsection below. </dd> <dt><samp class=”option”>audible_fixed_key</samp></dt> <dd> Fixed key used for handling Audible AAX/AAX+ files. It has been pre-set so should not be necessary to specify. </dd> <dt><samp class=”option”>decryption_key</samp></dt>
<dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>decryption_keys</samp></dt>
<dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>max_stts_delta</samp></dt>
<dd> Very high sample deltas written in a trak’s stts box may occasionally be intended but usually they are written in error or used to store a negative value for dts correction when treated as signed 32-bit integers. This option lets the user set an upper limit, beyond which the delta is clamped to 1. Values greater than the limit if negative when cast to int32 are used to adjust onward

dts. Unit is the track time scale. Range is 0 to UINT_MAX. Default is <code class="code">UINT_MAX - 48000*10</code> which allows up to a 10 second dts correction for 48 kHz audio streams while accommodating 99.9% of <code class="code">uint32</code> range. </dd> <dt><samp class="option">interleaved_read</samp></dt> <dd> Interleave packets from multiple tracks at demuxer level. For badly interleaved files, this prevents playback issues caused by large gaps between packets in different tracks, as MOV/MP4 do not have packet placement requirements. However, this can cause excessive seeking on very badly interleaved files, due to seeking between tracks, so disabling it may prevent I/O issues, at the expense of playback. </dd> </dl>

 <h4 class="subsection">13.21.2 Audible AAX# TOC</h4>

Audible AAX files are encrypted M4B files, and they can be decrypted by specifying a 4 byte activation secret. <pre class="example-preformatted">ffmpeg -activation_bytes 1CEB00DA -i test.aax -vn -c:a copy output.mp4 </pre>

 <h3 class="section">13.22 mpegts# TOC</h3>

MPEG-2 transport stream demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">resync_size</samp></dt> <dd> Set size limit for looking up a new synchronization. Default value is 65536. </dd> <dt><samp class="option">skip_unknown_pmt</samp></dt> <dd> Skip PMTs for programs not defined in the PAT. Default value is 0. </dd> <dt><samp class="option">fix_teletext_pts</samp></dt> <dd> Override teletext packet PTS and DTS values with the timestamps calculated from the PCR of the first program which the teletext stream is part of and is not discarded. Default value is 1, set this option to 0 if you want your teletext packet PTS and DTS values untouched. </dd> <dt><samp class="option">ts_packetsize</samp></dt> <dd> Output option carrying the raw packet size in bytes. Show the detected raw packet size, cannot be set by the user. </dd> <dt><samp class="option">scan_all_pmts</samp></dt> <dd> Scan and combine all PMTs. The value is an integer with value from -1 to 1 (-1 means automatic setting, 1 means enabled, 0 means disabled). Default value is -1. </dd> <dt><samp class="option">merge_pmt_versions</samp></dt> <dd> Reuse existing streams when a PMT’s version is updated and elementary streams move to different PIDs. Default value is 0. </dd> <dt><samp class="option">max_packet_size</samp></dt> <dd> Set

maximum size, in bytes, of packet emitted by the demuxer. Payloads above this size are split across multiple packets. Range is 184 to INT_MAX/2. Default is 204800 bytes. </dd> </dl>

 <h3 class="section">13.23 mpjpeg# TOC</h3>

MJPEG encapsulated in multi-part MIME demuxer. This demuxer allows reading of MJPEG, where each frame is represented as a part of multipart/x-mixed-replace stream. <dl class="table"> <dt><samp class="option">strict_mime_boundary</samp></dt> <dd> Default implementation applies a relaxed standard to multi-part MIME boundary detection, to prevent regression with numerous existing endpoints not generating a proper MIME MJPEG stream. Turning this option on by setting it to 1 will result in a stricter check of the boundary value. </dd> </dl>

 <h3 class="section">13.24 rawvideo# TOC</h3>

Raw video demuxer. This demuxer allows one to read raw video data. Since there is no header specifying the assumed video parameters, the user must specify them in order to be able to decode the data correctly. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set input video frame rate. Default value is 25. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Set the input video pixel format. Default value is <code class="code">yuv420p</code>. </dd> <dt><samp class="option">stride</samp></dt> <dd> Set frame line size in bytes, only required if there are extra padding. For multiplane formats, <samp class="option">stride</samp> is a list of line size for each plane. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the input video size. This value must be specified explicitly. </dd> </dl>

For example to read a rawvideo file <samp class="file">input.raw</samp> with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">rgb24</code>, a video size of <code class="code">320x240</code>, and a frame rate of 10 images per second, use the command: <pre class="example-preformatted">ffplay -f rawvideo -pixel_format rgb24 -video_size 320x240 -framerate 10 input.raw </pre>

Read a rawvideo with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">yuv420p</code>, a video size of <code class="code">1080x1920</code>, has 8 bytes of padding in each line of Y

plane, and 4 bytes of padding with UV plane, <pre class="example-preformatted">ffplay -f rawvideo -pixel_format yuv420p -video_size 1080x1920 -stride 1088,544,544 input.raw </pre>

 <h3 class="section">13.25 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. For more information on the format, see <a data-manual="ffmpeg-formats" href="./ffmpeg-formats.html#rcwtenc">(ffmpeg-formats)rcwtenc. This demuxer implements the specification as of March 2024, which has been stable and unchanged since April 2014. <h4 class="subsection">13.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Render CC to ASS using the built-in decoder: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin CC.ass </pre> Note that if your output appears to be empty, you may have to manually set the decoder’s <samp class="option">data_field</samp> option to pick the desired CC substream. Convert an RCWT backup to Scenarist (SCC) format: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin -c:s copy CC.scc </pre> Note that the SCC format does not support all of the possible CC extensions that can be stored in RCWT (such as EIA-708).

 <h3 class="section">13.26 sbg# TOC</h3>

SBaGen script demuxer. This demuxer reads the script language used by SBaGen http://uazu.net/sbagen/ to generate binaural beats sessions. A SBG script looks like that: <pre class="example-preformatted">-SE a: 300-2.5/3 440+4.5/0 b: 300-2.5/0 440+4.5/3 off: - NOW == a +0:07:00 == b +0:14:00 == a +0:21:00 == b +0:30:00 off </pre>

A SBG script can mix absolute and relative timestamps. If the script uses either only absolute timestamps (including the script start time) or only relative ones, then its layout is fixed, and the conversion is straightforward. On the other hand, if the script mixes both kind of timestamps, then the <var class="var">NOW</var> reference for relative timestamps will be taken from the current time of day at the time the script is read, and the script layout will be frozen according to that reference. That means that if the

script is directly played, the actual times will match the absolute timestamps up to the sound controller’s clock accuracy, but if the user somehow pauses the playback or seeks, all times will be shifted accordingly. <h3 class="section">13.27 tedcaptions# TOC</h3>

JSON captions used for TED Talks. TED does not provide links to the captions, but they can be guessed from the page. The file <samp class="file">tools/bookmarklets.html</samp> from the FFmpeg source tree contains a bookmarklet to expose them. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">start_time</samp></dt> <dd> Set the start time of the TED talk, in milliseconds. The default is 15000 (15s). It is used to sync the captions with the downloadable videos, because they include a 15s intro. </dd> </dl>

Example: convert the captions to a format most players understand: <pre class="example-preformatted">ffmpeg -i http://www.ted.com/talks/subtitles/id/1/lang/en talk1-en.srt </pre>

 <h3 class="section">13.28 vapoursynth# TOC</h3>

Vapoursynth wrapper. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class="code">-f vapoursynth</code> before the input <code class="code">-i yourscript.vpy</code>. This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">max_script_size</samp></dt> <dd> The demuxer buffers the entire script into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of scripts that can be read. Default is 1 MiB. </dd> </dl>

 <h3 class="section">13.29 w64# TOC</h3>

Sony Wave64 Audio demuxer. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">max_size</samp></dt> <dd> See the same option for the wav demuxer. </dd> </dl>

 <h3 class="section">13.30 wav# <a class="anchor hidden-xs"href="#toc-wav"

aria-hidden=”true”>TOC</h3>

RIFF Wave Audio demuxer. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>ignore_length <var class=”var”>bool</var></samp></dt> <dd> Ignore the size of the <code class=”code”>data</code> chunk and keep reading until the end of the file if set. May be useful to read broken or partial files where the header was not properly updated, but will misinterpret files with non-audio chunks after the <code class=”code”>data</code> chunk. Default is disabled. </dd> <dt><samp class=”option”>max_size</samp></dt> <dd> Specify the maximum packet size in bytes for the demuxed packets. By default this is set to 0, which means that a sensible value is chosen based on the input format. </dd> </dl>

 <h3 class=”section”>13.31 webp# TOC</h3>

Animated WebP demuxer. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>-min_delay <var class=”var”>int</var></samp></dt> <dd> Set the minimum valid delay between frames in milliseconds. Range is 0 to 60000. Default value is 10. </dd> <dt><samp class=”option”>-max_webp_delay <var class=”var”>int</var></samp></dt> <dd> Set the maximum valid delay between frames in milliseconds. Range is 0 to 16777215. Default value is 16777215 (over four hours), the maximum value allowed by the specification. </dd> <dt><samp class=”option”>-default_delay <var class=”var”>int</var></samp></dt> <dd> Set the default delay between frames in milliseconds. Range is 0 to 60000. Default value is 100. </dd> <dt><samp class=”option”>-ignore_loop <var class=”var”>bool</var></samp></dt> <dd> WebP files can contain information to loop a certain number of times (or infinitely). If <samp class=”option”>ignore_loop</samp> is set to true, then the loop setting from the input will be ignored and looping will not occur. If set to false, then looping will occur and will cycle the number of times according to the WebP. Default value is true. </dd> <dt><samp class=”option”>usebgcolor <var class=”var”>bool</var></samp></dt> <dd> WebP files contain a background color hint in the ANIM chunk, but the WebP specification says that viewer applications are not required to use it. If <samp class=”option”>usebgcolor</samp> is set to true, then the background color hint will be used, otherwise transparent black will be used for the background. Default value is false. </dd> </dl>

 <h2 class=”chapter”>14 Metadata# TOC</h2>

FFmpeg is able to dump metadata from media files into a simple UTF-8-encoded INI-like text file and then load it back using the metadata muxer/demuxer. The file format is as follows: <ol class=”enumerate”> A file consists of a header and a number of metadata tags divided into sections, each on its own line.

 The header is a ‘<samp class=”samp”>;FFMETADATA</samp>’ string, followed by a version number (now 1).

 Metadata tags are of the form ‘<samp class=”samp”>key=value</samp>’

 Immediately after header follows global metadata

 After global metadata there may be sections with per-stream/per-chapter metadata.

 A section starts with the section name in uppercase (i.e. STREAM or CHAPTER) in brackets (‘<samp class=”samp”>[</samp>’, ‘<samp class=”samp”>]</samp>’) and ends with next section or end of file.

 At the beginning of a chapter section there may be an optional timebase to be used for start/end values. It must be in form ‘<samp class=”samp”>TIMEBASE=<var class=”var”>num</var>/<var class=”var”>den</var></samp>’ where <var class=”var”>num</var> and <var class=”var”>den</var> are integers. If the timebase is missing then start/end times are assumed to be in nanoseconds.

Next a chapter section must contain chapter start and end times in form ‘<samp class=”samp”>START=<var class=”var”>num</var></samp>’, ‘<samp class=”samp”>END=<var class=”var”>num</var></samp>’, where <var class=”var”>num</var> is a positive integer. Empty lines and lines starting with ‘<samp class=”samp”>;</samp>’ or ‘<samp class=”samp”>#</samp>’ are ignored.

 Metadata keys or values containing special characters (‘<samp class=”samp”>=</samp>’, ‘<samp class=”samp”>;</samp>’, ‘<samp class=”samp”>#</samp>’, ‘<samp class=”samp”>\</samp>’ and a newline) must be escaped with a backslash ‘<samp class=”samp”>\</samp>’.

 Note that whitespace in metadata (e.g. ‘<samp class=”samp”>foo = bar</samp>’) is considered to be a part of the tag (in the example above key is ‘<samp class=”samp”>foo </samp>’, value is ‘<samp class=”samp”> bar</samp>’).

A ffmetadata file might look like this: <pre class=”example-preformatted”>;FFMETADATA1 title=bike\\shed ;this is a comment artist=FFmpeg troll team

[CHAPTER] TIMEBASE=1/1000 START=0 #chapter ends at 0:01:00 END=60000 title=chapter \#1 [STREAM] title=multi\ line </pre>

By using the ffmetadata muxer and demuxer it is possible to extract metadata from an input file to an ffmetadata file, and then transcode the file into an output file with the edited ffmetadata file. Extracting an ffmetadata file with <samp class="file">ffmpeg</samp> goes as follows: <pre class="example-preformatted">ffmpeg -i INPUT -f ffmetadata FFMETADATAFILE </pre>

Reinserting edited metadata information from the FFMETADATAFILE file can be done as: <pre class="example-preformatted">ffmpeg -i INPUT -i FFMETADATAFILE -map_metadata 1 -codec copy OUTPUT </pre>

 <h2 class="chapter">15 Protocol Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the protocols. In addition each protocol may support so-called private options, which are specific for that component. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class="table"> <dt><samp class="option">protocol_whitelist <var class="var">list</var> (input)</samp></dt> <dd> Set a ","-separated list of allowed protocols. "ALL" matches all protocols. Protocols prefixed by "-" are disabled. All protocols are allowed by default but protocols used by an another protocol (nested protocols) are restricted to a per protocol subset. </dd> </dl>

 <h2 class="chapter">16 Protocols# TOC</h2>

Protocols are configured elements in FFmpeg that enable access to resources that require specific protocols. When you configure your FFmpeg build, all the supported protocols are enabled by default. You can list all available ones using the configure option "–list-protocols". You can disable all the protocols using the configure option "–disable-protocols", and selectively enable a protocol using the option "–enable-protocol=<var class="var">PROTOCOL</var>", or you can disable a particular protocol using the option "–disable-protocol=<var

class=”var”>PROTOCOL</var>". The option "-protocols" of the ff* tools will display the list of supported protocols. All protocols accept the following options: <dl class=”table”> <dt><samp class=”option”>rw_timeout</samp></dt> <dd> Maximum time to wait for (network) read/write operations to complete, in microseconds. </dd> </dl>

A description of the currently available protocols follows. <h3 class=”section”>16.1 amqp# TOC</h3>

Advanced Message Queueing Protocol (AMQP) version 0-9-1 is a broker based publish-subscribe communication protocol. FFmpeg must be compiled with –enable-librabbitmq to support AMQP. A separate AMQP broker must also be run. An example open-source AMQP broker is RabbitMQ. After starting the broker, an FFmpeg client may stream data to the broker using the command: <pre class=”example-preformatted”>ffmpeg -re -i input -f mpegts amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

Where hostname and port (default is 5672) is the address of the broker. The client may also set a user/password for authentication. The default for both fields is "guest". Name of virtual host on broker can be set with vhost. The default value is "/". Multiple subscribers may stream from the broker using the command: <pre class=”example-preformatted”>ffplay amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

In RabbitMQ all data published to the broker flows through a specific exchange, and each subscribing client has an assigned queue/buffer. When a packet arrives at an exchange, it may be copied to a client’s queue depending on the exchange and routing_key fields. The following options are supported: <dl class=”table”> <dt><samp class=”option”>exchange</samp></dt> <dd> Sets the exchange to use on the broker. RabbitMQ has several predefined exchanges: "amq.direct" is the default exchange, where the publisher and subscriber must have a matching routing_key; "amq.fanout" is the same as a broadcast operation (i.e. the data is forwarded to all queues on the fanout exchange independent of the routing_key); and "amq.topic" is similar to "amq.direct", but allows for more complex pattern matching (refer to the RabbitMQ documentation). </dd> <dt><samp class=”option”>routing_key</samp></dt> <dd> Sets the routing key. The default value is "amqp". The routing key is used on the "amq.direct" and "amq.topic" exchanges to decide whether packets are written to the queue of a subscriber. </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Maximum size of each packet sent/received to the broker. Default is 131072. Minimum

is 4096 and max is any large value (representable by an int). When receiving packets, this sets an internal buffer size in FFmpeg. It should be equal to or greater than the size of the published packets to the broker. Otherwise the received message may be truncated causing decoding errors. </dd> <dt><samp class="option">connection_timeout</samp></dt> <dd> The timeout in seconds during the initial connection to the broker. The default value is rw_timeout, or 5 seconds if rw_timeout is not set. </dd> <dt><samp class="option">delivery_mode <var class="var">mode</var></samp></dt> <dd> Sets the delivery mode of each message sent to broker. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">persistent</samp>’</dt> <dd> Delivery mode set to "persistent" (2). This is the default value. Messages may be written to the broker’s disk depending on its setup. </dd> <dt>‘<samp class="samp">non-persistent</samp>’</dt> <dd> Delivery mode set to "non-persistent" (1). Messages will stay in broker’s memory unless the broker is under memory pressure. </dd> </dl>

</dd> </dl>

 <h3 class="section">16.2 async# TOC</h3>

Asynchronous data filling wrapper for input stream. Fill data in a background thread, to decouple I/O operation from demux thread.   <pre class="example-preformatted">async:<var class="var">URL</var> async:http://host/resource async:cache:http://host/resource </pre>

 <h3 class="section">16.3 bluray# TOC</h3>

Read BluRay playlist. The accepted options are: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> BluRay angle </dd> <dt><samp class="option">chapter</samp></dt> <dd> Start chapter (1...N) </dd> <dt><samp class="option">playlist</samp></dt> <dd> Playlist to read (BDMV/PLAYLIST/?????.mpls) </dd> </dl>

Examples: Read longest playlist from BluRay mounted to /mnt/bluray: <pre class="example-preformatted">bluray:/mnt/bluray </pre>

Read angle 2 of playlist 4 from BluRay mounted to /mnt/bluray, start from chapter 2: <pre class="example-preformatted">-playlist 4 -angle 2 -chapter 2 bluray:/mnt/bluray </pre>

 <h3

class=”section”>16.4 cache# TOC</h3>

Caching wrapper for input stream. Cache the input stream to temporary file. It brings seeking capability to live streams. The accepted options are: <dl class=”table”> <dt><samp class=”option”>read_ahead_limit</samp></dt> <dd> Amount in bytes that may be read ahead when seeking isn’t supported. Range is -1 to INT_MAX. -1 for unlimited. Default is 65536. </dd> </dl>

URL Syntax is <pre class=”example-preformatted”>cache:<var class=”var”>URL</var> </pre>

 <h3 class=”section”>16.5 concat# TOC</h3>

Physical concatenation protocol. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concat:<var class=”var”>URL1</var>|<var class=”var”>URL2</var>|...|<var class=”var”>URLN</var> </pre>

where <var class=”var”>URL1</var>, <var class=”var”>URL2</var>, ..., <var class=”var”>URLN</var> are the urls of the resource to be concatenated, each one possibly specifying a distinct protocol. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concat:split1.mpeg\|split2.mpeg\|split3.mpeg </pre>

Note that you may need to escape the character "|" which is special for many shells. <h3 class=”section”>16.6 concatf# TOC</h3>

Physical concatenation protocol using a line break delimited list of resources. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concatf:<var class=”var”>URL</var> </pre>

where <var class=”var”>URL</var> is the url containing a line break delimited list of resources to be concatenated, each one possibly specifying a distinct protocol. Special characters must be escaped with

backslash or single quotes. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#quoting_005fand_005fescaping”>the "Quoting and escaping" section in the ffmpeg-utils(1) manual. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> listed in separate lines within a file <samp class=”file”>split.txt</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concatf:split.txt </pre> Where <samp class=”file”>split.txt</samp> contains the lines: <pre class=”example-preformatted”>split1.mpeg split2.mpeg split3.mpeg </pre>

 <h3 class=”section”>16.7 crypto# TOC</h3>

AES-encrypted stream reading protocol. The accepted options are: <dl class=”table”> <dt><samp class=”option”>key</samp></dt> <dd> Set the AES decryption key binary block from given hexadecimal representation. </dd> <dt><samp class=”option”>iv</samp></dt> <dd> Set the AES decryption initialization vector binary block from given hexadecimal representation. </dd> </dl>

Accepted URL formats: <pre class=”example-preformatted”>crypto:<var class=”var”>URL</var> crypto+<var class=”var”>URL</var> </pre>

 <h3 class=”section”>16.8 data# TOC</h3>

Data in-line in the URI. See <a class=”url” href=”http://en.wikipedia.org/wiki/Data_URI_scheme”>http For example, to convert a GIF file given inline with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i " smiley.png </pre>

 <h3 class=”section”>16.9 fd# TOC</h3>

File descriptor access protocol. The accepted syntax is: <pre class=”example-preformatted”>fd: -fd <var class=”var”>file_descriptor</var> </pre>

If <samp class=”option”>fd</samp> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. Unlike the pipe protocol, fd protocol has seek support if it corresponding to a regular file. fd protocol doesn’t support pass file descriptor via URL for security. This protocol accepts the following options:

<dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

 <h3 class="section">16.10 file# TOC</h3>

File access protocol. Read from or write to a file. A file URL can have the form: <pre class="example-preformatted">file:<var class="var">filename</var> </pre>

where <var class="var">filename</var> is the path of the file to read. An URL that does not have a protocol prefix will be assumed to be a file URL. Depending on the build, an URL that looks like a Windows path with the drive letter at the beginning will also be assumed to be a file URL (usually not the case in builds for unix-like systems). For example to read from a file <samp class="file">input.mpeg</samp> with <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -i file:input.mpeg output.mpeg </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable for files on slow medium. </dd> <dt><samp class="option">follow</samp></dt> <dd> If set to 1, the protocol will retry reading at the end of the file, allowing reading files that still are being written. In order for this to terminate, you either need to use the rw_timeout option, or use the interrupt callback (for API users). Setting this option also ignores the file size reported by the file system. </dd> <dt><samp class="option">seekable</samp></dt> <dd> Controls if seekability is advertised on the file. 0 means non-seekable, -1 means auto (seekable for normal files, non-seekable for named pipes). Many demuxers handle seekable and non-seekable resources differently, overriding this might speed up opening certain files at the cost of losing some features (e.g. accurate seeking). </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set the maximum packet size used for file I/O. A smaller value may reduce memory usage. A higher

value may increase throughput especially with networked filesystems. For reading, if explicitly set, it overrides the default internal buffer size (32 KB) and limits the maximum amount of data read per operation. For writing, this sets the size of each write operation. The default is 256 KB for regular files, 32 KB otherwise. </dd> </dl>

 <h3 class="section">16.11 ftp# TOC</h3>

FTP (File Transfer Protocol). Read from or write to remote resources using FTP protocol. Following syntax is required. <pre class="example-preformatted">ftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg </pre>

This protocol accepts the following options. <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Set timeout in microseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class="option">ftp-user</samp></dt> <dd> Set a user to be used for authenticating to the FTP server. This is overridden by the user in the FTP URL. </dd> <dt><samp class="option">ftp-password</samp></dt> <dd> Set a password to be used for authenticating to the FTP server. This is overridden by the password in the FTP URL, or by <samp class="option">ftp-anonymous-password</samp> if no user is set. </dd> <dt><samp class="option">ftp-anonymous-password</samp></dt> <dd> Password used when login as anonymous user. Typically an e-mail address should be used. </dd> <dt><samp class="option">ftp-write-seekable</samp></dt> <dd> Control seekability of connection during encoding. If set to 1 the resource is supposed to be seekable, if set to 0 it is assumed not to be seekable. Default value is 0. </dd> </dl>

NOTE: Protocol can be used as output, but it is recommended to not do it, unless special care is taken (tests, customized server configuration etc.). Different FTP servers behave in different way during seek operation. ff* tools may produce incomplete content due to server limitations. <h3 class="section">16.12 gopher# TOC</h3>

Gopher protocol. <h3 class="section">16.13 gophers# TOC</h3>

Gophers protocol. The Gopher protocol with TLS en-

capsulation. <h3 class="section">16.14 handle# TOC</h3>

Windows handle access protocol. The accepted syntax is: <pre class="example-preformatted">handle: -handle <var class="var">handle</var> </pre>

If <samp class="option">handle</samp> is not specified, by default the output handle will be used for writing, input handle for reading. This protocol is only available on Windows. It has seek support if the handle corresponds to a regular file. It doesn’t support passing the handle via URL. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">handle</samp></dt> <dd> Set Windows handle. The supplied handle must already be valid in the calling process. When launching <code class="command">ffmpeg</code>, arrange for the child process to inherit the handle before passing its numeric value with <samp class="option">handle</samp>. Handles opened with <code class="code">FILE_FLAG_OVERLAPPED</code> are not supported. Closing the protocol does not close the caller’s original handle. </dd> </dl>

 <h3 class="section">16.15 http# TOC</h3>

HTTP (Hyper Text Transfer Protocol).

This protocol accepts the following options:

<dl class="table">

<dt><samp class="option">seekable</samp></dt>

<dd> Control seekability of connection. If set to 1 the resource is

supposed to be seekable, if set to 0 it is assumed not to be seekable,

if set to -1 it will try to autodetect if it is seekable. Default

value is -1.

</dd>
<dt><samp class="option">chunked_post</samp></dt>
<dd> If set to 1 use chunked Transfer-Encoding for posts, default is 1.

</dd>
<dt><samp class="option">http_proxy</samp></dt>
<dd> set HTTP proxy to tunnel through e.g. http://example.com:1234

</dd>
<dt><samp class="option">headers</samp></dt>
<dd> Set custom HTTP headers, can override built in default headers. The
value must be a string encoding the headers.

</dd>
<dt><samp class="option">content_type</samp></dt>
<dd> Set a specific content type for the POST messages or for listen mode.

</dd>
<dt><samp class="option">user_agent</samp></dt>
<dd> Override the User-Agent header. If not specified the protocol will use
a
string describing the libavformat build. ("Lavf/<version>")

</dd>
<dt><samp class="option">referer</samp></dt>
<dd> Set the Referer header. Include ’Referer: URL’ header
in HTTP request.

</dd>
<dt><samp class="option">multiple_requests</samp></dt>
<dd> Force persistent connections if set to 1, or disable if 0. Default is -1,

which means auto (implies keep-alive when using -request_size or

-initial_request_size).

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> Limit the size of requests made. This is useful for some pathological servers

that throttle unbounded range requests, as well as when expecting to seek

frequently. Disabled (set to 0) by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher. Doing so allows FFmpeg

to reuse a single HTTP connection wherever possible.

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also enable

the <samp class="option">multiple_requests</samp> option, as well as setting

<samp class="option">short_seek_size</samp> to the same value or higher.

</dd>

<dt><samp class=”option”>post_data</samp></dt>

<dd> Set custom HTTP post data.

</dd>

<dt><samp class=”option”>mime_type</samp></dt>

<dd> Export the MIME type.

</dd>

<dt><samp class=”option”>http_version</samp></dt>

<dd> Exports the HTTP response version number. Usually "1.0" or "1.1".

</dd>

<dt><samp class=”option”>cookies</samp></dt>

<dd> Set the cookies to be sent in future requests. The format of each cookie is the

same as the value of a Set-Cookie HTTP response field. Multiple cookies can be

delimited by a newline character. A cookie without a Domain attribute is only

sent to the host of the URL it is set for.

</dd>

<dt><samp class=”option”>icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class=”option”>icy_metadata_headers</samp> and <samp class=”option”>icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class=”option”>icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class="option">icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class="option">icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class="option">metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

<dt><samp class="option">auth_type</samp></dt>

<dd>

Set HTTP authentication type. No option for Digest, since this method requires

getting nonce parameters from the server first and can’t be used straight away like

Basic.

<dl class="table">

<dt><samp class="option">none</samp></dt>

<dd> Choose the HTTP authentication type automatically. This is the default.

</dd>

<dt><samp class="option">basic</samp></dt>

<dd>

Choose the HTTP basic authentication.

Basic authentication sends a Base64-encoded string that contains a user name and password

for the client. Base64 is not a form of encryption and should be considered the same as

sending the user name and password in clear text (Base64 is a reversible encoding).

If a resource needs to be protected, strongly consider using an authentication scheme

other than basic authentication. HTTPS/TLS should be used with basic authentication.

Without these additional security enhancements, basic authentication should not be used

to protect sensitive or valuable information.

</dd>

</dl>

</dd> <dt><samp class="option">send_expect_100</samp></dt> <dd> Send an Expect: 100-continue header for POST. If set to 1 it will send, if set to 0 it won’t, if set to -1 it will try to send if it is applicable. Default value is -1. </dd> <dt><samp class="option">location</samp></dt> <dd> An exported dictionary containing the content location. Only useful with the C API. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set initial byte offset. </dd> <dt><samp class="option">end_offset</samp></dt> <dd> Try to limit the request to bytes preceding this offset. </dd> <dt><samp class="option">method</samp></dt> <dd> When used as a client option it sets the HTTP method for the request. When used as a server option it sets the HTTP method that is going to be expected from the client(s). If the expected and the received HTTP method do not match the client will be given a Bad Request response. When unset the HTTP method is not checked for now. This will be replaced by autodetection in the future. </dd> <dt><samp class="option">reconnect</samp></dt> <dd> Reconnect automatically when disconnected before EOF is hit. </dd> <dt><samp class="option">reconnect_at_eof</samp></dt> <dd> If set then eof is treated like an error and causes reconnection, this is useful for live / endless streams. This does not apply to the deliberately established end of a resource whose complete size is unknown, i.e. when a ranged request past the end is answered with <code class="code">416</code>.

Such an end is reported as a clean EOF and is not retried, unless <samp class="option">reconnect_on_http_error</samp> lists 416 by number. </dd> <dt><samp class="option">reconnect_on_network_error</samp></dt> <dd> Reconnect automatically in case of TCP/TLS errors during connect. </dd> <dt><samp class="option">reconnect_on_http_error</samp></dt> <dd> A comma separated list of HTTP status codes to reconnect on. The list can include specific status codes (e.g. ’503’) or the strings ’4xx’ / ’5xx’. A <code class="code">416</code> that ends a read or a seek past the end of a resource whose complete size is unknown is reported as EOF. It is retried first only if 416 is listed by number, for example to read a resource that is still being written. ’4xx’ does not cover it. </dd> <dt><samp class="option">reconnect_streamed</samp></dt> <dd> If set then even streamed/non seekable streams will be reconnected on errors. </dd> <dt><samp class="option">reconnect_delay_max</samp></dt> <dd> Set the maximum delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">reconnect_max_retries</samp></dt> <dd> Set the maximum number of times to retry a connection. Default unset. </dd> <dt><samp class="option">reconnect_delay_total_max</samp></dt> <dd> Set the maximum total delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">respect_retry_after</samp></dt> <dd> If enabled, and a Retry-After header is encountered, its requested reconnection delay will be honored, rather than using exponential backoff. Useful for 429 and 503 errors. Default enabled. </dd> <dt><samp class="option">listen</samp></dt> <dd> If set to 1 enables experimental HTTP server. This can be used to send data when used as an output option, or read data from a client with HTTP POST when used as an input option. If set to 2 enables experimental multi-client HTTP server. This is not yet implemented in ffmpeg.c and thus must not be used as a command line option. <pre class="example-preformatted"># Server side (sending): ffmpeg -i somefile.ogg -c copy -listen 1 -f ogg http://<var class="var">server</var>:<var class="var">port</var>

Client side (receiving): ffmpeg -i http://<var class="var">server</var>:<var class="var">port</var> -c copy somefile.ogg

Client can also be done with wget: wget http://<var class="var">server</var>:<var class="var">port</var> -O somefile.ogg

Server side (receiving): ffmpeg -listen 1 -i http://<var class="var">server</var>:<var class="var">port</var> -c copy somefile.ogg

Client side (sending): ffmpeg -i somefile.ogg -chunked_post 0 -c copy -f ogg http://<var class="var">server</var>:<var class="var">port</var>

Client can also be done with wget: wget --post-file=somefile.ogg http://<var class="var">server</var>:<var class="var">port</var> </pre>

</dd> <dt><samp class="option">resource</samp></dt> <dd> The

resource requested by a client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>reply_code</samp></dt> <dd> The HTTP code returned to the client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>short_seek_size</samp></dt> <dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and new HTTP request. This is useful, for example, to make sure the same connection is used for reading large video packets with small audio packets in between. </dd> </dl>

 <h4 class=”subsection”>16.15.1 HTTP Cookies# TOC</h4>

Some HTTP requests will be denied unless cookie values are passed in with the request. The <samp class=”option”>cookies</samp> option allows these cookies to be specified. At the very least, each cookie must specify a value. HTTP requests that match both the domain and path will automatically include the cookie value in the HTTP Cookie header field. Multiple cookies can be delimited by a newline. The required syntax to play a stream specifying a cookie is: <pre class=”example-preformatted”>ffplay -cookies "nlqptid=nltid=tsn; path=/; domain=somedomain.com;" http://somedomain.com/somestream.m3u8 </pre>

 <h3 class=”section”>16.16 Icecast# TOC</h3>

Icecast protocol (stream to Icecast servers) This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>ice_genre</samp></dt> <dd> Set the stream genre. </dd> <dt><samp class=”option”>ice_name</samp></dt> <dd> Set the stream name. </dd> <dt><samp class=”option”>ice_description</samp></dt> <dd> Set the stream description. </dd> <dt><samp class=”option”>ice_url</samp></dt> <dd> Set the stream website URL. </dd> <dt><samp class=”option”>ice_public</samp></dt> <dd> Set if the stream should be public. The default is 0 (not public). </dd> <dt><samp class=”option”>user_agent</samp></dt> <dd> Override the User-Agent header. If not specified a string of the form "Lavf/<version>" will be used. </dd> <dt><samp class=”option”>password</samp></dt> <dd> Set the Icecast mountpoint password. </dd> <dt><samp class=”option”>content_type</samp></dt> <dd> Set the stream content type. This must be set if it is different from audio/mpeg. </dd> <dt><samp class=”option”>legacy_icecast</samp></dt> <dd> This enables support for Icecast versions < 2.4.0, that do not support the HTTP PUT method but the SOURCE method. </dd> <dt><samp

class=”option”>tls</samp></dt> <dd> Establish a TLS (HTTPS) connection to Icecast. </dd> </dl>

 <pre class=”example-preformatted”>icecast://[<var class=”var”>username</var>[:<var class=”var”>password</var>]@]<var class=”var”>server</var>:<var class=”var”>port</var>/<var class=”var”>mountpoint</var> </pre>

 <h3 class=”section”>16.17 ipfs# TOC</h3>

InterPlanetary File System (IPFS) protocol support. One can access files stored on the IPFS network through so-called gateways. These are http(s) endpoints. This protocol wraps the IPFS native protocols (ipfs:// and ipns://) to be sent to such a gateway. Users can (and should) host their own node which means this protocol will use one’s local gateway to access files on the IPFS network. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>gateway</samp></dt> <dd> Defines the gateway to use. When not set, the protocol will first try locating the local gateway by looking at <code class=”code”>$IPFS_GATEWAY</code>, <code class=”code”>$IPFS_PATH</code> and <code class=”code”>$HOME/.ipfs/</code>, in that order. </dd> </dl>

One can use this protocol in 2 ways. Using IPFS: <pre class=”example-preformatted”>ffplay ipfs://<hash> </pre>

Or the IPNS protocol (IPNS is mutable IPFS): <pre class=”example-preformatted”>ffplay ipns://<hash> </pre>

 <h3 class=”section”>16.18 mmst# TOC</h3>

MMS (Microsoft Media Server) protocol over TCP. <h3 class=”section”>16.19 mmsh# TOC</h3>

MMS (Microsoft Media Server) protocol over HTTP. The required syntax is: <pre class=”example-preformatted”>mmsh://<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>playpath</var>] </pre>

 <h3 class=”section”>16.20 md5# TOC</h3>

MD5 output protocol. Computes the MD5 hash of the data to be written, and on close writes this to the designated output or stdout if none is specified. It can be used to test muxers without writing an actual file. Some examples follow. <pre class="example-preformatted"># Write the MD5 hash of the encoded AVI file to the file output.avi.md5. ffmpeg -i input.flv -f avi -y md5:output.avi.md5

Write the MD5 hash of the encoded AVI file to stdout. ffmpeg -i input.flv -f avi -y md5: </pre>

Note that some formats (typically MOV) require the output protocol to be seekable, so they will fail with the MD5 output protocol. <h3 class="section">16.21 pipe# TOC</h3>

UNIX pipe access protocol. Read and write from UNIX pipes. The accepted syntax is: <pre class="example-preformatted">pipe:[<var class="var">number</var>] </pre>

If <samp class="option">fd</samp> isn’t specified, <var class="var">number</var> is the number corresponding to the file descriptor of the pipe (e.g. 0 for stdin, 1 for stdout, 2 for stderr). If <var class="var">number</var> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. For example to read from stdin with <code class="command">ffmpeg</code>: <pre class="example-preformatted">cat test.wav | ffmpeg -i pipe:0 # ...this is the same as... cat test.wav | ffmpeg -i pipe: </pre>

For writing to stdout with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i test.wav -f avi pipe:1 | cat > test.avi # ...this is the same as... ffmpeg -i test.wav -f avi pipe: | cat > test.avi </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

Note that some formats (typically MOV), require the output protocol to be seekable, so they will fail with the pipe output protocol. <h3 class="section">16.22 prompeg<a class="anchor hidden-xs" href="#prompeg" aria-

hidden=”true”># TOC</h3>

Pro-MPEG Code of Practice #3 Release 2 FEC protocol. The Pro-MPEG CoP#3 FEC is a 2D parity-check forward error correction mechanism for MPEG-2 Transport Streams sent over RTP. This protocol must be used in conjunction with the <code class=”code”>rtp_mpegts</code> muxer and the <code class=”code”>rtp</code> protocol. The required syntax is: <pre class=”example-preformatted”>-f rtp_mpegts -fec prompeg=<var class=”var”>option</var>=<var class=”var”>val</var>... rtp://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

The destination UDP ports are <code class=”code”>port + 2</code> for the column FEC stream and <code class=”code”>port + 4</code> for the row FEC stream. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>l=<var class=”var”>n</var></samp></dt> <dd> The number of columns (4-20, LxD <= 100) </dd> <dt><samp class=”option”>d=<var class=”var”>n</var></samp></dt> <dd> The number of rows (4-20, LxD <= 100) </dd> </dl>

Example usage: <pre class=”example-preformatted”>-f rtp_mpegts -fec prompeg=l=8:d=4 rtp://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>16.23 rist# TOC</h3>

Reliable Internet Streaming Transport protocol The accepted options are: <dl class=”table”> <dt><samp class=”option”>rist_profile</samp></dt> <dd> Supported values: <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dt>‘<samp class=”samp”>main</samp>’</d <dd> This one is default. </dd> <dt>‘<samp class=”samp”>advanced</samp>’</ </dl>

</dd> <dt><samp class=”option”>buffer_size</samp></dt> <dd> Set internal RIST buffer size in milliseconds for retransmission of data. Default value is 0 which means the librist default (1 sec). Maximum value is 30 seconds. </dd> <dt><samp class=”option”>fifo_size</samp></dt> <dd> Size of the librist receiver output fifo in number of packets. This must be a power of 2. Defaults to 8192 (vs the librist default of 1024). </dd> <dt><samp class=”option”>overrun_nonfatal=<var class=”var”>1|0</var></samp></dt> <dd> Survive in case of librist fifo buffer overrun. Default value is 0. </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Set maximum packet size for sending data. 1316 by default. </dd> <dt><samp class=”option”>log_level</samp></dt> <dd> Set loglevel for RIST

logging messages. You only need to set this if you explicitly want to enable debug level messages or packet loss simulation, otherwise the regular loglevel is respected. </dd> <dt><samp class="option">secret</samp></dt> <dd> Set override of encryption secret, by default is unset. </dd> <dt><samp class="option">encryption</samp></dt> <dd> Set encryption type, by default is disabled. Acceptable values are 128 and 256. </dd> </dl>

 <h3 class="section">16.24 rtmp# TOC</h3>

Real-Time Messaging Protocol. The Real-Time Messaging Protocol (RTMP) is used for streaming multimedia content across a TCP/IP network. The required syntax is: <pre class="example-preformatted">rtmp://[<var class="var">username</var>:<var class="var">password</var>@]<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">instance</var>][/<var class="var">playpath</var>] </pre>

The accepted parameters are: <dl class="table"> <dt><samp class="option">username</samp></dt> <dd> An optional username (mostly for publishing). </dd> <dt><samp class="option">password</samp></dt> <dd> An optional password (mostly for publishing). </dd> <dt><samp class="option">server</samp></dt> <dd> The address of the RTMP server. </dd> <dt><samp class="option">port</samp></dt> <dd> The number of the TCP port to use (by default is 1935). </dd> <dt><samp class="option">app</samp></dt> <dd> It is the name of the application to access. It usually corresponds to the path where the application is installed on the RTMP server (e.g. <samp class="file">/ondemand/</samp>, <samp class="file">/flash/live/</samp>, etc.). You can override the value parsed from the URI through the <code class="code">rtmp_app</code> option, too. </dd> <dt><samp class="option">playpath</samp></dt> <dd> It is the path or name of the resource to play with reference to the application specified in <var class="var">app</var>, may be prefixed by "mp4:". You can override the value parsed from the URI through the <code class="code">rtmp_playpath</code> option, too. </dd> <dt><samp class="option">listen</samp></dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt><samp class="option">timeout</samp></dt> <dd> Maximum time to wait for the incoming connection. Implies listen. </dd> </dl>

Additionally, the following parameters can be set via command line options

(or in code via <code class="code">AVOption</code>s):

<dl class="table">

<dt><samp class="option">rtmp_app</samp></dt>

<dd> Name of application to connect on the RTMP server. This option

overrides the parameter specified in the URI.

</dd>

<dt><samp class="option">rtmp_buffer</samp></dt>

<dd> Set the client buffer time in milliseconds. The default is 3000.

</dd>

<dt><samp class="option">rtmp_conn</samp></dt>

<dd> Extra arbitrary AMF connection parameters, parsed from a string,

e.g. like <code class="code">B:1 S:authMe O:1 NN:code:1.23 NS:flag:ok O:0</code>.

Each value is prefixed by a single character denoting the type,

B for Boolean, N for number, S for string, O for object, or Z for null,

followed by a colon. For Booleans the data must be either 0 or 1 for

FALSE or TRUE, respectively. Likewise for Objects the data must be 0 or

1 to end or begin an object, respectively. Data items in subobjects may

be named, by prefixing the type with ’N’ and specifying the name before

the value (i.e. <code class="code">NB:myFlag:1</code>). This option may be used multiple

times to construct arbitrary AMF sequences.

</dd>

<dt><samp class="option">rtmp_enhanced_codecs</samp></dt>

<dd> Specify the list of codecs the client advertises to support in an

enhanced RTMP stream. This option should be set to a comma separated

list of fourcc values, like <code class="code">hvc1,av01,vp09</code> for multiple codecs

or <code class="code">hvc1</code> for only one codec. The specified list will be presented

in the "fourCcLive" property of the Connect Command Message.

</dd>

<dt><samp class="option">rtmp_flashver</samp></dt>

<dd> Version of the Flash plugin used to run the SWF player. The default

is LNX 9,0,124,2. (When publishing, the default is FMLE/3.0 (compatible;

<libavformat version>).)

</dd>

<dt><samp class="option">rtmp_flush_interval</samp></dt>

<dd> Number of packets flushed in the same request (RTMPT only). The default

is 10.

</dd>

<dt><samp class="option">rtmp_live</samp></dt>

<dd> Specify that the media is a live stream. No resuming or seeking in

live streams is possible. The default value is <code class="code">any</code>, which means the

subscriber first tries to play the live stream specified in the

playpath. If a live stream of that name is not found, it plays the

recorded stream. The other possible values are <code class="code">live</code> and

<code class="code">recorded</code>.

</dd>

<dt><samp class="option">rtmp_pageurl</samp></dt>

<dd> URL of the web page in which the media was embedded. By default no

value will be sent.

</dd>

<dt><samp class="option">rtmp_playpath</samp></dt>

<dd> Stream identifier to play or to publish. This option overrides the

parameter specified in the URI.

</dd>

<dt><samp class="option">rtmp_subscribe</samp></dt>

<dd> Name of live stream to subscribe to. By default no value will be sent.

It is only sent if the option is specified or if rtmp_live

is set to live.

</dd>

<dt><samp class="option">rtmp_swfhash</samp></dt>

<dd> SHA256 hash of the decompressed SWF file (32 bytes).

</dd>

<dt><samp class="option">rtmp_swfsize</samp></dt>

<dd> Size of the decompressed SWF file, required for SWFVerification.

</dd>

<dt><samp class="option">rtmp_swfurl</samp></dt>

<dd> URL of the SWF player for the media. By default no value will be sent.

</dd>

<dt><samp class="option">rtmp_swfverify</samp></dt>

<dd> URL to player swf file, compute hash/size automatically.

</dd>

<dt><samp class="option">rtmp_tcurl</samp></dt>

<dd> URL of the target stream. Defaults to proto://host[:port]/app.

</dd>

<dt><samp class=”option”>tcp_nodelay=<var class=”var”>1|0</var></samp></dt>

<dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0.

Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY.

</dd>

<dt><samp class=”option”>tcp_keepalive=<var class=”var”>1|0</var></samp></dt>

<dd> Enable the TCP keepalive mechanism to detect dead peers and help maintain long-lived idle connections. Default value is 0.

Only the basic keepalive option (SO_KEEPALIVE) can be enabled or disabled. Platform-specific tuning parameters such as TCP_KEEPIDLE, TCP_KEEPINTVL, or TCP_KEEPCNT are not configurable and will use the operating system’s default values.

</dd>

</dl>

For example to read with <code class=”command”>ffplay</code> a multimedia resource named "sample" from the application "vod" from an RTMP server "myserver": <pre class=”example-preformatted”>ffplay rtmp://myserver/vod/sample </pre>

To publish to a password protected server, passing the playpath and app names separately: <pre class=”example-preformatted”>ffmpeg -re -i <input> -f flv -rtmp_playpath some/long/path -rtmp_app long/app/name rtmp://username:password@myserver/ </pre>

 <h3 class=”section”>16.25 rtmpe# TOC</h3>

Encrypted Real-Time Messaging Protocol. The Encrypted Real-Time Messaging Protocol (RTMPE) is used for streaming multimedia content within standard cryptographic primitives, consisting of Diffie-Hellman key exchange and HMACSHA256, generating a pair of

RC4 keys. <h3 class="section">16.26 rtmps# TOC</h3>

Real-Time Messaging Protocol over a secure SSL connection. The Real-Time Messaging Protocol (RTMPS) is used for streaming multimedia content across an encrypted connection. <h3 class="section">16.27 rtmpt# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTP. The Real-Time Messaging Protocol tunneled through HTTP (RTMPT) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">16.28 rtmpte# TOC</h3>

Encrypted Real-Time Messaging Protocol tunneled through HTTP. The Encrypted Real-Time Messaging Protocol tunneled through HTTP (RTMPTE) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class="section">16.29 rtmpts# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTPS. The Real-Time Messaging Protocol tunneled through HTTPS (RTMPTS) is used for streaming multimedia content within HTTPS requests to traverse firewalls. <h3 class="section">16.30 libcurl# TOC</h3>

HTTP and HTTPS access through the libcurl library, available as an alternative to the native <code class="code">http</code> protocol when FFmpeg is built with <code class="code">--enable-libcurl</code>. It runs transfers on a dedicated event-loop thread shared per <code class="code">AVFormatContext</code>, so all of a demuxer’s connections (for example HLS segments) reuse libcurl’s connection cache and HTTP/2 and HTTP/3 multiplexing. The native <code class="code">http</code> protocol stays the default. There are two ways to use libcurl instead: <ul class="itemize mark-bullet"> Prefix the URL with <code class="code">libcurl:</code> to force it explicitly, for example <code class="code">libcurl:https://example.com/video.mp4</code>.

Set the <samp class="option">prefer_libcurl</samp> option to

route <code class=”code”>http://</code> and <code class=”code”>https://</code> URLs through libcurl transparently.

<dl class=”table”> <dt><samp class=”option”>prefer_libcurl</samp></dt> <dd> When set to <code class=”code”>1</code>, open <code class=”code”>http(s)</code> URLs with the libcurl protocol instead of the native one. This is a generic IO option rather than a libcurl-specific one, so it also applies on builds without libcurl (where it has no effect). Default is <code class=”code”>0</code>. </dd> </dl>

The libcurl protocol accepts the following options.

<dl class=”table”>

<dt><samp class=”option”>user_agent</samp></dt>

<dd> Override the <code class=”code”>User-Agent</code> header. Defaults to <code class=”code”>Lavf/<version></code>.

</dd>

<dt><samp class=”option”>referer</samp></dt>

<dd> Set the <code class=”code”>Referer</code> header.

</dd>

<dt><samp class=”option”>headers</samp></dt>

<dd> Set custom HTTP headers, overriding internal defaults. Multiple headers are

separated by newlines.

</dd>

<dt><samp class=”option”>http_proxy</samp></dt>

<dd> Set the HTTP proxy to tunnel through.

</dd>

<dt><samp class=”option”>cookies</samp></dt>

<dd> Cookies to be sent in future requests, given as newline-delimited

<code class=”code”>Set-Cookie</code>-style values.

</dd>

<dt><samp class="option">location</samp></dt>

<dd> The actual location of the data after following redirects. This is an output

option, read with <code class="command">av_opt_get</code>; demuxers use it to resolve relative URLs.

</dd>

<dt><samp class="option">offset</samp></dt>

<dd> Initial byte offset of the request. Default is <code class="code">0</code>.

</dd>

<dt><samp class="option">end_offset</samp></dt>

<dd> If greater than <code class="code">0</code>, limit the request to the bytes preceding this offset.

Together with <samp class="option">offset</samp> this requests a sub-range of the resource, as used

for HLS byte-range segments.

</dd>

<dt><samp class="option">seekable</samp></dt>

<dd> Control seekability of the connection. If set to <code class="code">0</code> the stream is treated

as non-seekable, <code class="code">1</code> forces seekable, and <code class="code">-1</code> (the default) lets it be

detected from the server response.

</dd>

<dt><samp class="option">tls_verify</samp></dt>

<dd> Verify the peer certificate and host name. Enabled by default.

</dd>

<dt><samp class=”option”>ca_file</samp></dt>

<dd> Path to a certificate authority bundle that overrides the system store.

</dd>

<dt><samp class=”option”>cert_file</samp></dt>

<dt><samp class=”option”>key_file</samp></dt>

<dd> Client certificate and private key files for mutual TLS.

</dd>

<dt><samp class=”option”>connect_timeout</samp></dt>

<dd> Connection timeout in seconds. <code class=”code”>0</code> (the default) uses libcurl’s own timeout.

</dd>

<dt><samp class=”option”>max_redirects</samp></dt>

<dd> Maximum number of redirects to follow. Default is <code class=”code”>16</code>.

</dd>

<dt><samp class=”option”>multiple_requests</samp></dt>

<dd> Reuse the connection across requests (HTTP keep-alive). Enabled by default;

setting it to <code class=”code”>0</code> uses a fresh connection per transfer.

</dd>

<dt><samp class=”option”>http_version</samp></dt>

<dd> HTTP version to use: <code class=”code”>auto</code> (default), <code class=”code”>1.0</code>, <code class=”code”>1.1</code>, <code class=”code”>2</code>,

<code class=”code”>2tls</code>, <code class=”code”>2-prior-knowledge</code>, <code class=”code”>3</code> or <code class=”code”>3only</code>.

</dd>

<dt><samp class=”option”>buffer_size</samp></dt>

<dd> Size in bytes of the receive buffer. Default is 4 MiB.

</dd>

<dt><samp class=”option”>request_size</samp></dt>

<dd> If greater than <code class=”code”>0</code>, split a seekable transfer into ranged requests of at

most this many bytes each. Default is <code class=”code”>0</code> (a single request).

</dd>

<dt><samp class=”option”>initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class=”code”>request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class=”option”>short_seek_size</samp> to the same value or higher, otherwise the HTTP

connection may be closed and recreated for each subsequent initial request.

</dd>

<dt><samp class=”option”>max_retries</samp></dt>

<dd> Maximum number of retries after a recoverable error on a seekable transfer.

Default is <code class=”code”>5</code>.

</dd>

<dt><samp class="option">short_seek_size</samp></dt>

<dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and

new HTTP request. This is useful, for example, to make sure the same connection

is used for reading large video packets with small audio packets in between.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_status</samp></dt>

<dd> If set to 1 accept the legacy Shoutcast v1 <code class="code">ICY 200 OK</code> status line in

place of an HTTP one. Any reply whose status line starts with <code class="code">ICY 200</code>

is then treated as a successful HTTP 200.

The default is 1.

</dd>

<dt><samp class="option">icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class=”option”>icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class=”option”>icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class=”option”>metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

</dl>

A stream carrying in-band ICY metadata is always treated as non-seekable, since its byte offsets do not address the payload. <samp class=”option”>seekable</samp>, <samp class=”option”>request_size</samp> and <samp class=”option”>max_retries</samp> therefore have no effect on such a stream. For more information see: https://curl.se/libcurl/. <h3 class=”section”>16.31 libsmbclient# TOC</h3>

libsmbclient permits one to manipulate CIFS/SMB network resources. Following syntax is required.   <pre class=”example-preformatted”>smb://[[domain:]user[:password@]]server[/share[/path[/file]]] </pre>

This protocol accepts the following options. <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout in milliseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class=”option”>workgroup</samp></dt> <dd> Set the workgroup used for making connections. By default workgroup is not specified. </dd> </dl>

For more information see: http://www.samba.org/.
  <h3 class="section">16.32 libssh# TOC</h3>

Secure File Transfer Protocol via libssh Read from or write to remote resources using SFTP protocol. Following syntax is required.   <pre class="example-preformatted">sftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg
</pre>

This protocol accepts the following options. <dl class="table">
<dt><samp class="option">timeout</samp></dt> <dd> Set timeout of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd>
<dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">private_key</samp></dt>
<dd> Specify the path of the file containing private key to use during authorization. By default libssh searches for keys in the <samp class="file">~/.ssh/</samp> directory. </dd> <dt><samp class="option">verify</samp></dt> <dd> Verify the server host key against the known_hosts file before authenticating, if set to 1. Servers that are not in the file, or whose key differs from the one recorded there, are refused. Default value is 1. </dd> </dl>

Example: Play a file stored on remote server.
<pre class="example-preformatted">ffplay sftp://user:password@server_address:22/home/user/resource.mpeg
</pre>

 <h3 class="section">16.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte# TOC</h3>

Real-Time Messaging Protocol and its variants supported through librtmp. Requires the presence of the librtmp headers and library during configuration. You need to explicitly configure the build with "–enable-librtmp". If enabled this will replace the native RTMP protocol.
This protocol provides most client functions and a few server functions needed to support RTMP, RTMP tunneled in HTTP (RTMPT), encrypted RTMP (RTMPE), RTMP over SSL/TLS (RTMPS) and tunneled variants of these encrypted types (RTMPTE, RTMPTS). The required syntax is: <pre class="example-preformatted"><var

class=”var”>rtmp_proto</var>://<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>playpath</var>] <var class=”var”>options</var> </pre>

where <var class=”var”>rtmp_proto</var> is one of the strings "rtmp", "rtmpt", "rtmpe", "rtmps", "rtmpte", "rtmpts" corresponding to each RTMP variant, and <var class=”var”>server</var>, <var class=”var”>port</var>, <var class=”var”>app</var> and <var class=”var”>playpath</var> have the same meaning as specified for the RTMP native protocol. <var class=”var”>options</var> contains a list of space-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. See the librtmp manual page (man 3 librtmp) for more information. For example, to stream a file in real-time to an RTMP server using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -re -i myfile -f flv rtmp://myserver/live/mystream </pre>

To play the same stream using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay "rtmp://myserver/live/mystream live=1" </pre>

 <h3 class=”section”>16.34 rtp# TOC</h3>

Real-time Transport Protocol. The required syntax for an RTP URL is: <pre class=”example-preformatted”>rtp://<var class=”var”>hostname</var>[:<var class=”var”>port</var>][?<var class=”var”>options</var>] </pre>

<var class=”var”>port</var> specifies the RTP port to use. <var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ttl=<var class=”var”>n</var></samp></dt> <dd> Set the TTL (Time-To-Live) value (for multicast only). </dd> <dt><samp class=”option”>rtcpport=<var class=”var”>n</var></samp></dt> <dd> Set the remote RTCP port to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>localport, local_rtpport, localrtpport=<var class=”var”>n</var></samp></dt> <dd> Set the local RTP port to <var class=”var”>n</var>. Using the localport option name is deprecated and should not be used. </dd> <dt><samp class=”option”>local_rtcpport, localrtcpport=<var class=”var”>n</var>'</samp></dt> <dd> Set the local RTCP port to

<var class=”var”>n</var>. </dd> <dt><samp class=”option”>pkt_size=<var class=”var”>n</var></samp></dt> <dd> Set max packet size (in bytes) to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>buffer_size=<var class=”var”>size</var></samp></dt> <dd> Set the maximum UDP socket buffer size in bytes. </dd> <dt><samp class=”option”>connect=0|1</samp></dt> <dd> Do a <code class=”code”>connect()</code> on the UDP socket (if set to 1) or not (if set to 0). </dd> <dt><samp class=”option”>sources=<var class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt> <dd> List allowed source IP addresses. </dd> <dt><samp class=”option”>block=<var class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt> <dd> List disallowed (blocked) source IP addresses. </dd> <dt><samp class=”option”>write_to_source=0|1</samp></dt> <dd> Send packets to the source address of the latest received packet (if set to 1) or to a default remote address (if set to 0). </dd> <dt><samp class=”option”>localaddr=<var class=”var”>addr</var></samp></dt> <dd> Local IP address of a network interface used for sending packets or joining multicast groups. </dd> <dt><samp class=”option”>timeout=<var class=”var”>n</var></samp></dt> <dd> Set timeout (in microseconds) of socket I/O operations to <var class=”var”>n</var>. </dd> </dl>

Important notes: <ol class=”enumerate”> If <samp class=”option”>rtcpport</samp> is not set the RTCP port will be set to the RTP port value plus 1.

 If <samp class=”option”>localrtpport</samp> (the local RTP port) is not set any available port will be used for the local RTP and RTCP ports.

 If <samp class=”option”>localrtcpport</samp> (the local RTCP port) is not set it will be set to the local RTP port value plus 1.

 <h3 class=”section”>16.35 rtsp# TOC</h3>

Real-Time Streaming Protocol. RTSP is not technically a protocol handler in libavformat, it is a demuxer and muxer. The demuxer supports both normal RTSP (with data transferred over RTP; this is used by e.g. Apple and Microsoft) and Real-RTSP (with data transferred over RDT). The muxer can be used to send a stream using RTSP ANNOUNCE to a server supporting it (currently Darwin Streaming Server and Mischa Spiegelmock’s RTSP server). The required syntax for a RTSP url is: <pre class=”example-preformatted”>rtsp://<var class=”var”>hostname</var>[:<var class=”var”>port</var>]/<var class=”var”>path</var> </pre>

Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line, or set in code via <code class="code">AVOption</code>s or in <code class="code">avformat_open_input</code>. <h4 class="subsection">16.35.1 Muxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">latm</samp>’</dt> <dd> Use MP4A-LATM packetization instead of MPEG4-GENERIC for AAC. </dd> <dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> Use RFC 2190 packetization instead of RFC 4629 for H.263. </dd> <dt>‘<samp class="samp">skip_rtcp</samp>’</dt> <dd> Don’t send RTCP sender reports. </dd> <dt>‘<samp class="samp">h264_mode0</samp>’</dt> <dd> Use mode 0 for H.264 in RTP. </dd> <dt>‘<samp class="samp">send_bye</samp>’</dt> <dd> Send RTCP BYE packets when finishing. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’.

</dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set max send packet size (in bytes). Default value is 1472. </dd> </dl>

 <h4 class="subsection">16.35.2 Demuxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">initial_pause</samp></dt> <dd> Do not start playing the stream immediately if set to 1. Default value is 0. </dd> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt>

<dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> <dt>‘<samp class="samp">udp_multicast</samp>’</dt> <dd> Use UDP multicast as lower transport protocol. </dd> <dt>‘<samp class="samp">http</samp>’</dt> <dd> Use HTTP tunneling as lower transport protocol, which is useful for passing proxies. </dd> <dt>‘<samp class="samp">https</samp>’</dt> <dd> Use HTTPs tunneling as lower transport protocol, which is useful for passing proxies and widely used for security consideration. </dd> </dl>

Multiple lower transport protocols may be specified, in that case they are tried one at a time (if the setup of one fails, the next one is tried). For the muxer, only the ‘<samp class="samp">tcp</samp>’ and ‘<samp class="samp">udp</samp>’ options are supported. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">filter_src</samp>’</dt> <dd> Accept packets only from negotiated peer address and port. </dd> <dt>‘<samp class="samp">listen</samp>’</dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt>‘<samp class="samp">prefer_tcp</samp>’</dt> <dd> Try TCP for RTP transport first, if TCP is available as RTSP RTP transport. </dd> <dt>‘<samp class="samp">satip_raw</samp>’</dt> <dd> Export raw MPEG-TS stream instead of demuxing. The flag will simply write out the raw stream, with the original PAT/PMT/PIDs intact. </dd> </dl>

Default value is ‘<samp class="samp">none</samp>’. </dd> <dt><samp class="option">allowed_media_types</samp></dt> <dd> Set media types to accept from the server. The following flags are accepted: <dl class="table"> <dt>‘<samp class="samp">video</samp>’</dt> <dt>‘<samp class="samp">audio</samp>’</d <dt>‘<samp class="samp">data</samp>’</dt> <dt>‘<samp class="samp">subtitle</samp>’</dt> </dl>

By default it accepts all media types. </dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">listen_timeout</samp></dt> <dd> Set maximum timeout (in seconds) to establish an initial connection. Setting <samp class="option">listen_timeout</samp> > 0 sets <samp class="option">rtsp_flags</samp> to ‘<samp class="samp">listen</samp>’. Default is -1 which means an infinite timeout when ‘<samp class="samp">listen</samp>’

mode is set. </dd> <dt><samp class="option">reorder_queue_size</samp></dt> <dd> Set number of packets to buffer for handling of reordered packets. </dd> <dt><samp class="option">timeout</samp></dt> <dd> Set socket TCP I/O timeout in microseconds. </dd> <dt><samp class="option">user_agent</samp></dt> <dd> Override User-Agent header. If not specified, it defaults to the libavformat identifier string. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> </dl>

When receiving data over UDP, the demuxer tries to reorder received packets (since they may arrive out of order, or packets may get lost totally). This can be disabled by setting the maximum demuxing delay to zero (via the <code class="code">max_delay</code> field of AVFormatContext). When watching multi-bitrate Real-RTSP streams with <code class="command">ffplay</code>, the streams to display can be chosen with <code class="code">-vst</code> <var class="var">n</var> and <code class="code">-ast</code> <var class="var">n</var> for video and audio respectively, and can be switched on the fly by pressing <code class="code">v</code> and <code class="code">a</code>. <h4 class="subsection">16.35.3 Examples# TOC</h4>

The following examples all make use of the <code class="command">ffplay</code> and <code class="command">ffmpeg</code> tools. <ul class="itemize mark-bullet"> Watch a stream over UDP, with a max reordering delay of 0.5 seconds: <pre class="example-preformatted">ffplay -max_delay 500000 -rtsp_transport udp rtsp://server/video.mp4 </pre>

Watch a stream tunneled over HTTP: <pre class="example-preformatted">ffplay -rtsp_transport http rtsp://server/video.mp4 </pre>

Send a stream in realtime to a RTSP server, for others to watch: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f rtsp -muxdelay 0.1 rtsp://server/live.sdp </pre>

Receive a stream in realtime: <pre class="example-preformatted">ffmpeg -rtsp_flags listen -i rtsp://ownaddress/live.sdp <var class="var">output</var> </pre>

 <h3 class="section">16.36 sap# TOC</h3>

Session Announcement Protocol (RFC 2974). This is not technically a pro-

tocol handler in libavformat, it is a muxer and demuxer. It is used for signalling of RTP streams, by announcing the SDP for the streams regularly on a separate port. <h4 class="subsection">16.36.1 Muxer# TOC</h4>

The syntax for a SAP url given to the muxer is: <pre class="example-preformatted">sap://<var class="var">destination</var>[:<var class="var">port</var>][?<var class="var">options</var>] </pre>

The RTP packets are sent to <var class="var">destination</var> on port <var class="var">port</var>, or to port 5004 if no port is specified. <var class="var">options</var> is a <code class="code">&</code>-separated list. The following options are supported: <dl class="table"> <dt><samp class="option">announce_addr=<var class="var">address</var></samp></dt> <dd> Specify the destination IP address for sending the announcements to. If omitted, the announcements are sent to the commonly used SAP announcement multicast address 224.2.127.254 (sap.mcast.net), or ff0e::2:7ffe if <var class="var">destination</var> is an IPv6 address. </dd> <dt><samp class="option">announce_port=<var class="var">port</var></samp></dt> <dd> Specify the port to send the announcements on, defaults to 9875 if not specified. </dd> <dt><samp class="option">ttl=<var class="var">ttl</var></samp></dt> <dd> Specify the time to live value for the announcements and RTP packets, defaults to 255. </dd> <dt><samp class="option">same_port=<var class="var">0|1</var></samp></dt> <dd> If set to 1, send all RTP streams on the same port pair. If zero (the default), all streams are sent on unique ports, with each stream on a port 2 numbers higher than the previous. VLC/Live555 requires this to be set to 1, to be able to receive the stream. The RTP stack in libavformat for receiving requires all streams to be sent on unique ports. </dd> </dl>

Example command lines follow. To broadcast a stream on the local subnet, for watching in VLC: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255?same_port=1 </pre>

Similarly, for watching in <code class="command">ffplay</code>:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255 </pre>

And for watching in <code class="command">ffplay</code>, over IPv6:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://[ff0e::1:2:3:4] </pre>

 <h4 class="subsection">16.36.2 Demuxer<a class="anchor hidden-xs" href="#Demuxer-

1" aria-hidden="true"># TOC</h4>

The syntax for a SAP url given to the demuxer is: <pre class="example-preformatted">sap://[<var class="var">address</var>][:<var class="var">port</var>] </pre>

<var class="var">address</var> is the multicast address to listen for announcements on, if omitted, the default 224.2.127.254 (sap.mcast.net) is used. <var class="var">port</var> is the port that is listened on, 9875 if omitted. The demuxers listens for announcements on the given address and port. Once an announcement is received, it tries to receive that particular stream. Example command lines follow. To play back the first stream announced on the normal SAP multicast address: <pre class="example-preformatted">ffplay sap:// </pre>

To play back the first stream announced on one the default IPv6 SAP multicast address: <pre class="example-preformatted">ffplay sap://[ff0e::2:7ffe] </pre>

 <h3 class="section">16.37 sctp# TOC</h3>

Stream Control Transmission Protocol. The accepted URL syntax is: <pre class="example-preformatted">sctp://<var class="var">host</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">listen</samp></dt> <dd> If set to any value, listen for an incoming connection. Outgoing connection is done by default. </dd> <dt><samp class="option">max_streams</samp></dt> <dd> Set the maximum number of streams. By default no limit is set. </dd> </dl>

 <h3 class="section">16.38 shared# TOC</h3>

Thread-safe, persistent, cross-process cache wrapper for input streams. Caches the input stream to a file in the specified directory. This is similar to cache, but with a persistent cache on disk

that may be shared between multiple processes or demuxers, even concurrently. The filename of the cache is obtained by hashing the input URL, so there is some risk of collision if the same URL is used for different content (e.g. as a result of extra POST data being used to select the stream ID). To avoid this, make sure to only use this protocol for URLs that uniquely identify the content; otherwise you may get a garbled mix of sources. This includes content selected by request options such as headers, cookies or credentials: data cached for one user is returned to every other user of the same cache directory. The accepted options are: <dl class="table"> <dt><samp class="option">cache_dir</samp></dt> <dd> Path to the directory where cache files are stored. This option is required. Anyone who can write to this directory or to <samp class="option">metadata_dir</samp> can alter the data read by all users of the cache, or make them crash, so neither may be writable by untrusted users. </dd> <dt><samp class="option">metadata_dir</samp></dt> <dd> Path to the directory where metadata files are stored. Optional. If not specified, metadata is stored in the same directory as the cache file. </dd> <dt><samp class="option">block_shift</samp></dt> <dd> Shift factor (log2) of the block size used for internal reads/writes. Defaults to 15, i.e. 32KB blocks. If this does not match the value that was specified when an existing cache file was created, the previously specified value will be used instead. </dd> <dt><samp class="option">cache_read_only</samp></dt> <dd> If true, use the shared cache for reads but don’t write any new blocks to it. Default is false. Note that even with this option enabled, the cache file will be initialized if it does not already exist. </dd> <dt><samp class="option">cache_verify</samp></dt> <dd> If true, verify any data read from the cache against the underlying input stream, and error out on any mismatches. Note that this will make the cache layer effectively useless. This is purely a debug option. </dd> <dt><samp class="option">cache_timeout</samp></dt> <dd> If set to a nonzero value, specifies the maximum time (in microseconds) to wait for data to become available, if another process is currently trying to fetch and cache the same block at the same time. If this timeout elapses, it’s assumed that the other process may have gotten stuck or died in the meantime. If set to zero, no waiting is done and all processes will immediately race to try and fetch the same missing blocks themselves. Defaults to 10000 (10 ms). </dd> <dt><samp class="option">ignore_errors</samp></dt> <dd> If true, failures of the underlying input stream are not treated as fatal, and playback continues using only the data already present in the cache file. Default is false. Any attempt to read into uncached data will result in an unrecoverable IO error. </dd> <dt><samp class="option">retry_errors</samp></dt> <dd> If true (the default), transient read errors from the underlying input stream are ignored and retried again. If false, any blocks that previously failed being read from will be treated as permanently inaccessible. </dd> <dt><samp class="option">retry_corrupt</samp></dt> <dd> If true (the default), blocks whose contents fail the CRC integrity

check are re-fetched from the underlying input stream, overwriting the corrupt cached data. If false, cache corruption is treated as a fatal read error. </dd> <dt><samp class="option">cache_size_max</samp></dt> <dd> Maximum amount of data to cache, in bytes, rounded down to a multiple of the chosen block size. Accepts the usual size suffixes, e.g. <code class="code">10G</code> for 10*10^9 bytes, or <code class="code">10Gi</code> for 10*2^30 bytes. Defaults to 0, meaning no limit. Once this limit is reached, the protocol switches to read-only mode (see <samp class="option">cache_read_only</samp>). Already cached data is never evicted, so once this limit is reached, no new data will ever be cached until the cache file is deleted. Note that this option is best-effort only, and may over-write up to N-1 blocks, where N is the number of concurrent processes using the same cache file. Note also that the underlying file system may reserve more space than written, if the underlying allocation granularity is larger than the block size. </dd> <dt><samp class="option">offset</samp></dt> <dd> Initial byte offset to start reading from. This option is not passed on to the underlying protocol; it instead gets a regular seek on the first cache miss. Defaults to 0. </dd> </dl>

URL Syntax is <pre class="example-preformatted">shared:<var class="var">URL</var> </pre>

 <h3 class="section">16.39 srt# TOC</h3>

Haivision Secure Reliable Transport Protocol via libsrt. The supported syntax for a SRT URL is: <pre class="example-preformatted">srt://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form

<var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class="code">AVOption</code>s).

The list of supported options follows.

<dl class="table">

<dt><samp class="option">connect_timeout=<var class="var">milliseconds</var></samp></dt>

<dd> Connection timeout; SRT cannot connect for RTT > 1500 msec

(2 handshake exchanges) with the default connect timeout of

3 seconds. This option applies to the caller and rendezvous

connection modes. The connect timeout is 10 times the value

set for the rendezvous mode (which can be used as a

workaround for this connection problem with earlier versions).

</dd>

<dt><samp class="option">ffs=<var class="var">bytes</var></samp></dt>

<dd> Flight Flag Size (Window Size), in bytes. FFS is actually an

internal parameter and you should set it to not less than

<samp class="option">recv_buffer_size</samp> and <samp class="option">mss</samp>. The default value

is relatively large, therefore unless you set a very large receiver buffer,

you do not need to change this option. Default value is 25600.

</dd>

<dt><samp class="option">inputbw=<var class="var">bytes/seconds</var></samp></dt>

<dd> Sender nominal input rate, in bytes per seconds. Used along with

<samp class="option">oheadbw</samp>, when <samp class="option">maxbw</samp> is set to relative (0), to

calculate maximum sending rate when recovery packets are sent

along with the main media stream:

<samp class="option">inputbw</samp> * (100 + <samp class="option">oheadbw</samp>) / 100

if <samp class="option">inputbw</samp> is not set while <samp class="option">maxbw</samp> is set to

relative (0), the actual input rate is evaluated inside

the library. Default value is 0.

</dd>

<dt><samp class=”option”>iptos=<var class=”var”>tos</var></samp></dt>

<dd> IP Type of Service. Applies to sender only. Default value is 0xB8.

</dd>

<dt><samp class=”option”>ipttl=<var class=”var”>ttl</var></samp></dt>

<dd> IP Time To Live. Applies to sender only. Default value is 64.

</dd>

<dt><samp class=”option”>latency=<var class=”var”>microseconds</var></samp></dt>

<dd> Timestamp-based Packet Delivery Delay.

Used to absorb bursts of missed packet retransmissions.

This flag sets both <samp class=”option”>rcvlatency</samp> and <samp class=”option”>peerlatency</samp>

to the same value. Note that prior to version 1.3.0

this is the only flag to set the latency, however

this is effectively equivalent to setting <samp class=”option”>peerlatency</samp>,

when side is sender and <samp class=”option”>rcvlatency</samp>

when side is receiver, and the bidirectional stream

sending is not supported.

</dd>

<dt><samp class=”option”>listen_timeout=<var class=”var”>microseconds</var></samp></dt>

<dd> Set socket listen timeout.

</dd>

<dt><samp class=”option”>maxbw=<var class=”var”>bytes/seconds</var></samp></dt>

<dd> Maximum sending bandwidth, in bytes per seconds.

-1 infinite (CSRTCC limit is 30mbps)

0 relative to input rate (see <samp class=”option”>inputbw</samp>)

>0 absolute limit value

Default value is 0 (relative)

</dd>

<dt><samp class=”option”>mode=<var class=”var”>caller|listener|rendezvous</var></samp></dt>

<dd> Connection mode.

<samp class=”option”>caller</samp> opens client connection.

<samp class=”option”>listener</samp> starts server to listen for incoming connections.

<samp class=”option”>rendezvous</samp> use Rendez-Vous connection mode.

Default value is caller.

</dd>

<dt><samp class=”option”>mss=<var class=”var”>bytes</var></samp></dt>

<dd> Maximum Segment Size, in bytes. Used for buffer allocation

and rate calculation using a packet counter assuming fully

filled packets. The smallest MSS between the peers is

used. This is 1500 by default in the overall internet.

This is the maximum size of the UDP packet and can be

only decreased, unless you have some unusual dedicated

network settings. Default value is 1500.

</dd>

<dt><samp class=”option”>nakreport=<var class=”var”>1|0</var></samp></dt>

<dd> If set to 1, Receiver will send ‘UMSG_LOSSREPORT‘ messages

periodically until a lost packet is retransmitted or

intentionally dropped. Default value is 1.

</dd>

<dt><samp class=”option”>oheadbw=<var class=”var”>percents</var></samp></dt>

<dd> Recovery bandwidth overhead above input rate, in percents.

See <samp class="option">inputbw</samp>. Default value is 25%.

</dd>
<dt><samp class="option">passphrase=<var class="var">string</var></samp></dt>
<dd> HaiCrypt Encryption/Decryption Passphrase string, length
from 10 to 79 characters. The passphrase is the shared
secret between the sender and the receiver. It is used
to generate the Key Encrypting Key using PBKDF2
(Password-Based Key Derivation Function). It is used
only if <samp class="option">pbkeylen</samp> is non-zero. It is used on
the receiver only if the received data is encrypted.
The configured passphrase cannot be recovered (write-only).

</dd>
<dt><samp class="option">enforced_encryption=<var class="var">1|0</var></samp></dt>
<dd> If true, both connection parties must have the same password
set (including empty, that is, with no encryption). If the
password doesn’t match or only one side is unencrypted,
the connection is rejected. Default is true.

</dd>
<dt><samp class="option">kmrefreshrate=<var class="var">packets</var></samp></dt>
<dd> The number of packets to be transmitted after which the
encryption key is switched to a new key. Default is -1.
-1 means auto (0x1000000 in srt library). The range for
this option is integers in the 0 - <code class="code">INT_MAX</code>.

</dd>
<dt><samp class="option">kmpreannounce=<var class="var">packets</var></samp></dt>
<dd> The interval between when a new encryption key is sent and
when switchover occurs. This value also applies to the

subsequent interval between when switchover occurs and
when the old encryption key is decommissioned. Default is -1.
-1 means auto (0x1000 in srt library). The range for
this option is integers in the 0 - <code class=”code”>INT_MAX</code>.

</dd>
<dt><samp class=”option”>snddropdelay=<var class=”var”>microseconds</var></samp></dt>
<dd> The sender’s extra delay before dropping packets. This delay
is
added to the default drop delay time interval value.

Special value -1: Do not drop packets on the sender at all.

</dd>
<dt><samp class=”option”>payload_size=<var class=”var”>bytes</var></samp></dt>
<dd> Sets the maximum declared size of a packet transferred
during the single call to the sending function in Live
mode. Use 0 if this value isn’t used (which is default in
file mode).
Default is -1 (automatic), which typically means MPEG-TS;
if you are going to use SRT
to send any different kind of payload, such as, for example,
wrapping a live stream in very small frames, then you can
use a bigger maximum frame size, though not greater than
1456 bytes.

</dd>
<dt><samp class=”option”>pkt_size=<var class=”var”>bytes</var></samp></dt>
<dd> Alias for ‘<samp class=”samp”>payload_size</samp>’.

</dd>

<dt><samp class=”option”>peerlatency=<var class=”var”>microseconds</var></samp></dt>

<dd> The latency value (as described in <samp class=”option”>rcvlatency</samp>)
that is

set by the sender side as a minimum value for the receiver.

</dd>

<dt><samp class=”option”>pbkeylen=<var class=”var”>bytes</var></samp></dt>

<dd> Sender encryption key length, in bytes.

Only can be set to 0, 16, 24 and 32.

Enable sender encryption if not 0.

Not required on receiver (set to 0),

key size obtained from sender in HaiCrypt handshake.

Default value is 0.

</dd>

<dt><samp class=”option”>rcvlatency=<var class=”var”>microseconds</var></samp></dt>

<dd> The time that should elapse since the moment when the

packet was sent and the moment when it’s delivered to

the receiver application in the receiving function.

This time should be a buffer time large enough to cover

the time spent for sending, unexpectedly extended RTT

time, and the time needed to retransmit the lost UDP

packet. The effective latency value will be the maximum

of this options’ value and the value of <samp class=”option”>peerlatency</samp>

set by the peer side. Before version 1.3.0 this option

is only available as <samp class=”option”>latency</samp>.

</dd>

<dt><samp class=”option”>recv_buffer_size=<var class=”var”>bytes</var></samp></dt>

<dd> Set UDP receive buffer size, expressed in bytes.

</dd>

<dt><samp class="option">send_buffer_size=<var class="var">bytes</var></samp></dt>

<dd> Set UDP send buffer size, expressed in bytes.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeouts for read, write and connect operations. Note
that the

SRT library has internal timeouts which can be controlled separately, the

value set here is only a cap on those.

</dd>

<dt><samp class="option">tlpktdrop=<var class="var">1|0</var></samp></dt>

<dd> Too-late Packet Drop. When enabled on receiver, it skips

missing packets that have not been delivered in time and

delivers the following packets to the application when

their time-to-play has come. It also sends a fake ACK to

the sender. When enabled on sender and enabled on the

receiving peer, the sender drops the older packets that

have no chance of being delivered in time. It was

automatically enabled in the sender if the receiver

supports it.

</dd>

<dt><samp class="option">sndbuf=<var class="var">bytes</var></samp></dt>

<dd> Set send buffer size, expressed in bytes.

</dd>

<dt><samp class="option">rcvbuf=<var class="var">bytes</var></samp></dt>

<dd> Set receive buffer size, expressed in bytes.

Receive buffer must not be greater than <samp class=”option”>ffs</samp>.

</dd>
<dt><samp class=”option”>lossmaxttl=<var class=”var”>packets</var></samp></dt>
<dd> The value up to which the Reorder Tolerance may grow. When
Reorder Tolerance is > 0, then packet loss report is delayed
until that number of packets come in. Reorder Tolerance
increases every time a "belated" packet has come, but it
wasn’t due to retransmission (that is, when UDP packets tend
to come out of order), with the difference between the latest
sequence and this packet’s sequence, and not more than the
value of this option. By default it’s 0, which means that this
mechanism is turned off, and the loss report is always sent
immediately upon experiencing a "gap" in sequences.

</dd>
<dt><samp class=”option”>minversion</samp></dt>
<dd> The minimum SRT version that is required from the peer. A connection
to a peer that does not satisfy the minimum version requirement
will be rejected.

The version format in hex is 0xXXYYZZ for x.y.z in human readable
form.

</dd>
<dt><samp class=”option”>streamid=<var class=”var”>string</var></samp></dt>
<dd> A string limited to 512 characters that can be set on the socket prior
to connecting. This stream ID will be able to be retrieved by the
listener side from the socket that is returned from srt_accept and
was connected by a socket with that set stream ID. SRT does not enforce

any special interpretation of the contents of this string.

This option doesn't make sense in Rendezvous connection; the result

might be that simply one side will override the value from the other

side and it's the matter of luck which one would win

</dd>

<dt><samp class="option">srt_streamid=<var class="var">string</var></samp></dt>

<dd> Alias for ‘<samp class="samp">streamid</samp>’
to avoid conflict with ffmpeg command line option.

</dd>

<dt><samp class="option">smoother=<var class="var">live|file</var></samp></dt>

<dd> The type of Smoother used for the transmission for that socket, which

is responsible for the transmission and congestion control. The Smoother

type must be exactly the same on both connecting parties, otherwise

the connection is rejected.

</dd>

<dt><samp class="option">messageapi=<var class="var">1|0</var></samp></dt>

<dd> When set, this socket uses the Message API, otherwise it uses Buffer

API. Note that in live mode (see <samp class="option">transtype</samp>)
there's only

message API available. In File mode you can chose to use one of two modes:

Stream API (default, when this option is false). In this mode you may

send as many data as you wish with one sending instruction, or even use

dedicated functions that read directly from a file. The internal facility

will take care of any speed and congestion control. When receiving, you

can also receive as many data as desired, the data not extracted will be

waiting for the next call. There is no boundary between data portions in

the Stream mode.

Message API. In this mode your single sending instruction passes exactly
one piece of data that has boundaries (a message). Contrary to Live mode,
this message may span across multiple UDP packets and the only size
limitation is that it shall fit as a whole in the sending buffer. The
receiver shall use as large buffer as necessary to receive the message,
otherwise the message will not be given up. When the message is not
complete (not all packets received or there was a packet loss) it will
not be given up.

</dd>
<dt><samp class="option">transtype=<var class="var">live|file</var></samp></dt>
<dd> Sets the transmission type for the socket, in particular, setting this
option sets multiple other parameters to their default values as required
for a particular transmission type.

live: Set options as for live transmission. In this mode, you should
send by one sending instruction only so many data that fit in one UDP packet,
and limited to the value defined first in <samp class="option">payload_size</samp>
(1316 is
default in this mode). There is no speed control in this mode, only the
bandwidth control, if configured, in order to not exceed the bandwidth with
the overhead transmission (retransmitted and control packets).

file: Set options as for non-live transmission. See <samp class="option">messageapi</samp>
for further explanations

</dd>
<dt><samp class="option">linger=<var class="var">seconds</var></samp></dt>
<dd> The number of seconds that the socket waits for unsent data when
closing.

Default is -1. -1 means auto (off with 0 seconds in live mode, on with 180

seconds in file mode). The range for this option is integers in the

0 - <code class=”code”>INT_MAX</code>.

</dd>

<dt><samp class=”option”>tsbpd=<var class=”var”>1|0</var></samp></dt>

<dd> When true, use Timestamp-based Packet Delivery mode. The default behavior

depends on the transmission type: enabled in live mode, disabled in file

mode.

</dd>

<dt><samp class=”option”>ipv6only=<var class=”var”>1|0</var></samp></dt>

<dd> Accept IPv4 or not while using the IPv6 wildcard address. This should be set

when listening on a IPv6 wildcard address.

</dd>

</dl>

For more information see: <a class=”url” href=”https://github.com/Haivision/srt”>https://github.com/H
  <h3 class=”section”>16.40 srtp# TOC</h3>

Secure Real-time Transport Protocol. The accepted options are:
<dl class=”table”> <dt><samp class=”option”>srtp_in_suite</samp></dt>
<dt><samp class=”option”>srtp_out_suite</samp></dt> <dd> Select input and output encoding suites. Supported values: <dl class=”table”> <dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_80</samp>’</d
<dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_80</samp>’</dt>
<dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_32</samp>’</dt>
<dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_32</samp>’</dt>
</dl>

</dd> <dt><samp class=”option”>srtp_in_params</samp></dt>
<dt><samp class=”option”>srtp_out_params</samp></dt> <dd> Set input and output encoding parameters, which are expressed by a base64-encoded

representation of a binary block. The first 16 bytes of this binary block are used as master key, the following 14 bytes are used as master salt. </dd> </dl>

 <h3 class="section">16.41 subfile# TOC</h3>

Virtually extract a segment of a file or another stream. The underlying stream must be seekable. Accepted options: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Start offset of the extracted segment, in bytes. </dd> <dt><samp class="option">end</samp></dt> <dd> End offset of the extracted segment, in bytes. If set to 0, extract till end of file. </dd> </dl>

Examples: Extract a chapter from a DVD VOB file (start and end sectors obtained externally and multiplied by 2048): <pre class="example-preformatted">subfile,,start,153391104,end,268142592,,:/media/dvd/ </pre>

Play an AVI file directly from a TAR archive: <pre class="example-preformatted">subfile,,start,183241728,end,366490624,,:archive.tar </pre>

Play a MPEG-TS file from start offset till end: <pre class="example-preformatted">subfile,,start,32815239,end,0,,:video.ts </pre>

 <h3 class="section">16.42 tee# TOC</h3>

Writes the output to multiple protocols. The individual outputs are separated by | <pre class="example-preformatted">tee:file://path/to/local/this.avi|file://path/to/local/that.avi </pre>

 <h3 class="section">16.43 tcp# TOC</h3>

Transmission Control Protocol. The required syntax for a TCP url is: <pre class="example-preformatted">tcp://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to

escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">listen=<var class="var">2|1|0</var></samp></dt> <dd> Listen for an incoming connection. 0 disables listen, 1 enables listen in single client mode, 2 enables listen in multi-client mode. Default value is 0. </dd> <dt><samp class="option">local_addr=<var class="var">addr</var></samp></dt> <dd> Local IP address of a network interface used for tcp socket connect. </dd> <dt><samp class="option">local_port=<var class="var">port</var></samp></dt> <dd> Local port used for tcp socket connect. </dd> <dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt> <dd> Set raise error timeout, expressed in microseconds. This option is only relevant in read mode: if no data arrived in more than this time interval, raise error. </dd> <dt><samp class="option">listen_timeout=<var class="var">milliseconds</var></samp></dt> <dd> Set listen timeout, expressed in milliseconds. </dd> <dt><samp class="option">recv_buffer_size=<var class="var">bytes</var></samp></dt> <dd> Set receive buffer size, expressed bytes. </dd> <dt><samp class="option">send_buffer_size=<var class="var">bytes</var></samp></dt> <dd> Set send buffer size, expressed bytes. </dd> <dt><samp class="option">tcp_nodelay=<var class="var">1|0</var></samp></dt> <dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0. Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY. </dd> <dt><samp class="option">tcp_mss=<var class="var">bytes</var></samp></dt> <dd> Set maximum segment size for outgoing TCP packets, expressed in bytes. </dd> </dl>

The following example shows how to setup a listening TCP connection with <code class="command">ffmpeg</code>, which is then accessed with <code class="command">ffplay</code>: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> tcp://<var class="var">hostname</var>:<var class="var">port</var>?listen ffplay tcp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">16.44 tls# TOC</h3>

Transport Layer Security (TLS) / Secure Sockets Layer (SSL) The required syntax for a TLS/SSL url is: <pre class="example-preformatted">tls://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ca_file, cafile=<var class=”var”>filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class=”option”>tls_verify, verify=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. (With other backends, the host name is validated as well.) This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class=”option”>cert_file, cert=<var class=”var”>filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class=”option”>key_file, key=<var class=”var”>filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class=”option”>listen=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class=”option”>http_proxy</samp></dt> <dd> The HTTP proxy to tunnel through, e.g. <code class=”code”>http://example.com:1234</code>. The proxy must support the CONNECT method. </dd> </dl>

Example command lines: To create a TLS/SSL server that serves an input stream.   <pre class=”example-preformatted”>ffmpeg -i <var class=”var”>input</var> -f <var class=”var”>format</var> tls://<var class=”var”>hostname</var>:<var class=”var”>port</var>?listen&cert=<var class=”var”>server.crt</var>&key=<var class=”var”>server.key</var> </pre>

To play back a stream from the TLS/SSL server using <code class=”command”>ffplay</code>:   <pre class=”example-preformatted”>ffplay tls://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>16.45 dtls<span class=”pull-

right”>#
TOC</h3>

Datagram Transport Layer Security (DTLS) The required syntax for a DTLS URL is: <pre class=”example-preformatted”>dtls://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). DTLS shares most options with TLS, but operates over UDP instead of TCP. The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ca_file, cafile=<var class=”var”>filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class=”option”>tls_verify, verify=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class=”option”>cert_file, cert=<var class=”var”>filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class=”option”>key_file, key=<var class=”var”>filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class=”option”>cert_pem=<var class=”var”>string</var></samp></dt> <dd> Certificate PEM string </dd> <dt><samp class=”option”>key_pem=<var class=”var”>string</var></samp></dt> <dd> Private key PEM string </dd> <dt><samp class=”option”>listen=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class=”option”>mtu=<var class=”var”>size</var></samp></dt> <dd> Set the Maximum Transmission Unit (MTU) for DTLS packets. </dd> <dt><samp class=”option”>use_srtp=<var class=”var”>1|0</var></samp></dt> <dd> Enable the use_srtp DTLS extension. This is used in WebRTC applications to establish SRTP encryption keys through the

DTLS handshake. Default is disabled. </dd> <dt><samp class="option">external_sock=<var class="var">1|0</var></samp></dt> <dd> Use an external socket instead of creating a new one. This option only makes sense to pass when interacting with the code via API, enabling this from CLI will cause immediate failure. Default is disabled. </dd> </dl>

Example command lines: To create a DTLS server: <pre class="example-preformatted">ffmpeg -listen 1 -i dtls://<var class="var">hostname</var>:<var class="var">port</var> <var class="var">output</var> </pre>

To create a DTLS client and send data to server: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> dtls://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">16.46 udp# TOC</h3>

User Datagram Protocol. The required syntax for an UDP URL is: <pre class="example-preformatted">udp://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form

<var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class="code">AVOption</code>s).

In case threading is enabled on the system, a circular buffer is used

to store the incoming data, which allows one to reduce loss of data due to

UDP socket buffer overruns. The <var class="var">fifo_size</var> and

<var class="var">overrun_nonfatal</var> options are related to this buffer.

The list of supported options follows.

<dl class="table">

<dt><samp class="option">buffer_size=<var class="var">size</var></samp></dt>

<dd> Set the UDP maximum socket buffer size in bytes. This is used to set either

the receive or send buffer size, depending on what the socket is used for.

Default is 32 KB for output, 384 KB for input. See also <var class="var">fifo_size</var>.

</dd>

<dt><samp class="option">bitrate=<var class="var">bitrate</var></samp></dt>

<dd> If set to nonzero, the output will have the specified constant bitrate if the

input has enough packets to sustain it.

</dd>

<dt><samp class="option">burst_bits=<var class="var">bits</var></samp></dt>

<dd> When using <var class="var">bitrate</var> this specifies the maximum number of bits in

packet bursts.

</dd>

<dt><samp class="option">localport=<var class="var">port</var></samp></dt>

<dd> Override the local UDP port to bind with.

</dd>

<dt><samp class="option">localaddr=<var class="var">addr</var></samp></dt>

<dd> Local IP address of a network interface used for sending packets or joining

multicast groups.

</dd>

<dt><samp class="option">pkt_size=<var class="var">size</var></samp></dt>

<dd> Set the size in bytes of UDP packets.

</dd>

<dt><samp class="option">reuse=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow reusing UDP sockets.

</dd>

<dt><samp class="option">ttl=<var class="var">ttl</var></samp></dt>

<dd> Set the time to live value (for multicast only).

</dd>

<dt><samp class="option">dscp=<var class="var">dscp</var></samp></dt>

<dd> Set the 6-bit DSCP field for outgoing packets.

</dd>

<dt><samp class="option">connect=<var class="var">1|0</var></samp></dt>

<dd> Initialize the UDP socket with <code class="code">connect()</code>. In this case, the

destination address can’t be changed with ff_udp_set_remote_url later.

If the destination address isn’t known at the start, this option can

be specified in ff_udp_set_remote_url, too.

This allows finding out the source address for the packets with getsockname,

and makes writes return with AVERROR(ECONNREFUSED) if "destination

unreachable" is received.

For receiving, this gives the benefit of only receiving packets from

the specified peer address/port.

</dd>

<dt><samp class="option">sources=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Only receive packets sent from the specified addresses. In case of multicast,

also subscribe to multicast traffic coming from these addresses only.

</dd>

<dt><samp class="option">block=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Ignore packets sent from the specified addresses. In case of multicast, also

exclude the source addresses in the multicast subscription.

</dd>

<dt><samp class="option">fifo_size=<var class="var">units</var></samp></dt>

<dd> Set the UDP receiving circular buffer size, expressed as a number of

packets with size of 188 bytes. If not specified defaults to 7*4096.

Set to 0 to disable the receiving circular buffer and its receive thread.

</dd>

<dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt>

<dd> Survive in case of UDP receiving circular buffer overrun. Default

value is 0.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeout, expressed in microseconds.

This option is only relevant in read mode: if no data arrived in more

than this time interval, raise error.

</dd>

<dt><samp class="option">broadcast=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow UDP broadcasting.

Note that broadcasting may not work properly on networks having

a broadcast storm protection.

</dd>

</dl>

 <h4 class="subsection">16.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to stream over UDP to a remote endpoint: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> udp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

Use <code class="command">ffmpeg</code> to stream in mpegts format over UDP using 188 sized UDP packets, using a large input buffer: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f mpegts udp://<var class="var">hostname</var>:<var class="var">port</var>?pkt_size=188&buffer_size=65535 </pre>

Use <code class="command">ffmpeg</code> to receive over UDP from a remote endpoint: <pre class="example-preformatted">ffmpeg -i udp://[<var class="var">multicast-address</var>]:<var class="var">port</var> ... </pre>

 <h3 class="section">16.47 unix# TOC</h3>

Unix local socket The required syntax for a Unix socket URL is: <pre class="example-preformatted">unix://<var class="var">filepath</var> </pre>

The following parameters can be set via command line options (or in code via <code class="code">AVOption</code>s): <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Timeout in ms. </dd> <dt><samp class="option">listen</samp></dt> <dd> Create the Unix socket in listening mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Select socket type. <dl class="table"> <dt>‘<samp class="samp">stream Corresponds to SOCK_STREAM (reliable stream-oriented)</samp>’</dt> <dt>‘<samp class="samp">datagram Corresponds to SOCK_DGRAM (unreliable packet-oriented)</samp>’</dt> <dt>‘<samp class="samp">seqpacket Corresponds to SOCK_SEQPACKET (reliable packet-oriented)</samp>’</dt> </dl> </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Maximum packet size for packet-oriented sockets (SOCK_DGRAM and SOCK_SEQPACKET). If

greater than zero, this value is used as <code class="code">max_packet_size</code>. Ignored for SOCK_STREAM. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">16.48 zmq# TOC</h3>

ZeroMQ asynchronous messaging using the libzmq library. This library supports unicast streaming to multiple clients without relying on an external server. The required syntax for streaming or connecting to a stream is: <pre class="example-preformatted">zmq:tcp://ip-address:port </pre>

Example: Create a localhost stream on port 5555: <pre class="example-preformatted">ffmpeg -re -i input -f mpegts zmq:tcp://127.0.0.1:5555 </pre>

Multiple clients may connect to the stream using: <pre class="example-preformatted">ffplay zmq:tcp://127.0.0.1:5555 </pre>

Streaming to multiple clients is implemented using a ZeroMQ Pub-Sub pattern. The server side binds to a port and publishes data. Clients connect to the server (via IP address/port) and subscribe to the stream. The order in which the server and client start generally does not matter. ffmpeg must be compiled with the –enable-libzmq option to support this protocol. Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line. The following options are supported: <dl class="table"> <dt><samp class="option">pkt_size</samp></dt> <dd> Forces the maximum packet size for sending/receiving data. The default value is 131,072 bytes. On the server side, this sets the maximum size of sent packets via ZeroMQ. On the clients, it sets an internal buffer size for receiving packets. Note that pkt_size on the clients should be equal to or greater than pkt_size on the server. Otherwise the received message may be truncated causing decoding errors. </dd> </dl>

 <h2 class="chapter">17 Device Options# TOC</h2>

The libavdevice library provides the same interface as libavformat. Namely, an input device is considered like a demuxer, and an output device like a muxer, and the interface and generic device options are the same provided by libavformat (see the ffmpeg-formats manual). In addition each input or output device may support so-called private options, which are specific for that component. Options may be set by

specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the device <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use.

 <h2 class="chapter">18 Input Devices# TOC</h2>

Input devices are configured elements in FFmpeg which enable accessing the data coming from a multimedia device attached to your system. When you configure your FFmpeg build, all the supported input devices are enabled by default. You can list all available ones using the configure option "–list-indevs". You can disable all the input devices using the configure option "–disable-indevs", and selectively enable an input device using the option "–enable-indev=<var class="var">INDEV</var>", or you can disable a particular input device using the option "–disable-indev=<var class="var">INDEV</var>". The option "-devices" of the ff* tools will display the list of supported input devices. A description of the currently available input devices follows. <h3 class="section">18.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) input device. To enable this input device during configuration you need libasound installed on your system. This device allows capturing from an ALSA device. The name of the device to capture has to be an ALSA card identifier. An ALSA identifier has the syntax: <pre class="example-preformatted">hw:<var class="var">CARD</var>[,<var class="var">DEV</var>[,<var class="var">SUBDEV</var>]] </pre>

where the <var class="var">DEV</var> and <var class="var">SUBDEV</var> components are optional. The three arguments (in order: <var class="var">CARD</var>,<var class="var">DEV</var>,<var class="var">SUBDEV</var>) specify card number or identifier, device number and subdevice number (-1 means any). To see the list of cards currently recognized by your system check the files <samp class="file">/proc/asound/cards</samp> and <samp class="file">/proc/asound/devices</samp>. For example to capture with <code class="command">ffmpeg</code> from an ALSA device with card id 0, you may run the command: <pre class="example-preformatted">ffmpeg -f alsa -i hw:0 alsaout.wav </pre>

For more information see: <a class="url" href="http://www.alsa-

project.org/alsa-doc/alsa-lib/pcm.html”>http://www.alsa-project.org/alsa-doc/alsa-lib/pcm.html <h4 class=”subsection”>18.1.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class=”option”>ch_layout</samp></dt> <dd> Set the channel layout. When unset, the layout defaults to stereo, or to the minimum channel count supported by the device, and is filled in from the ALSA channel map on a best-effort basis. </dd> </dl>

 <h3 class=”section”>18.2 android_camera# TOC</h3>

Android camera input device. This input devices uses the Android Camera2 NDK API which is available on devices with API level 24+. The availability of android_camera is autodetected during configuration. This device allows capturing from all cameras on an Android device, which are integrated into the Camera2 NDK API. The available cameras are enumerated internally and can be selected with the <var class=”var”>camera_index</var> parameter. The input file string is discarded. Generally the back facing camera has index 0 while the front facing camera has index 1. <h4 class=”subsection”>18.2.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size given as a string such as 640x480 or hd720. Falls back to the first available configuration reported by Android if requested video size is not available or by default. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the video framerate. Falls back to the first available configuration reported by Android if requested framerate is not available or by default (-1). </dd> <dt><samp class=”option”>camera_index</samp></dt> <dd> Set the index of the camera to use. Default is 0. </dd> <dt><samp class=”option”>input_queue_size</samp></dt> <dd> Set the maximum number of frames to buffer. Default is 5. </dd> </dl>

 <h3 class=”section”>18.3 avfoundation# TOC</h3>

AVFoundation input device. AVFoundation is the currently recommended framework by Apple for streamgrabbing on OSX >= 10.7 as well as on iOS. The input filename has to be given in the following syntax: <pre class="example-preformatted">-i "[[VIDEO]:[AUDIO]]" </pre> The first entry selects the video input while the latter selects the audio input. The stream has to be specified by the device name or the device index as shown by the device list. Alternatively, the video and/or audio input device can be chosen by index using the <samp class="option"> -video_device_index <INDEX> </samp> and/or <samp class="option"> -audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class="option">-list_devices true</samp>, listing all device names and corresponding indices. There are two device name aliases: <dl class="table"> <dt><code class="code">default</code></dt> <dd> Select the AVFoundation default device of the corresponding type. </dd> <dt><code class="code">none</code></dt> <dd> Do not record the corresponding media type. This is equivalent to specifying an empty device name or index. </dd> </dl>

 <h4 class="subsection">18.3.1 Options# TOC</h4>

AVFoundation supports the following options: <dl class="table"> <dt><samp class="option">-list_devices <TRUE|FALSE></samp></dt> <dd> If set to true, a list of all available input devices is given showing their index, name, unique ID, and USB serial number (when available). </dd> <dt><samp class="option">-video_device_index <INDEX></samp></dt> <dd> Specify the video device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-video_device_id <ID></samp></dt> <dd> Specify the video device by a prefixed identifier, as shown by <samp class="option">-list_devices</samp>: <code class="code">uid:<UID></code> selects by AVFoundation unique ID, <code class="code">serial:<SERIAL></code> by USB serial number. The unique ID of a UVC device may not be stable across reboots. Only UVC devices expose a serial number. Overrides anything given in the input filename. </dd> <dt><samp class="option">-audio_device_id <ID></samp></dt> <dd> Specify the audio device by a prefixed identifier. See <samp class="option">-video_device_id</samp>; for audio, the serial is only available for class-compliant USB devices. </dd> <dt><samp class="option">-pixel_format <FORMAT></samp></dt> <dd> Request

the video device to use a specific pixel format. If the specified format is not supported, a list of available formats is given and the first one in this list is used instead. Available pixel formats are: <code class=”code”>monob, rgb555be, rgb555le, rgb565be, rgb565le, rgb24, bgr24, 0rgb, bgr0, 0bgr, rgb0, bgr48be, uyvy422, yuva444p, yuva444p16le, yuv444p, yuv422p16, yuv422p10, yuv444p10, yuv420p, nv12, yuyv422, gray</code> </dd> <dt><samp class=”option”>-framerate</samp></dt> <dd> Set the grabbing frame rate. Default is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>-video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class=”option”>-capture_cursor</samp></dt> <dd> Capture the mouse pointer. Default is 0. </dd> <dt><samp class=”option”>-capture_mouse_clicks</samp></dt> <dd> Capture the screen mouse clicks. Default is 0. </dd> <dt><samp class=”option”>-capture_raw_data</samp></dt> <dd> Capture the raw device data. Default is 0. Using this option may result in receiving the underlying data delivered to the AVFoundation framework. E.g. for muxed devices that sends raw DV data to the framework (like tape-based camcorders), setting this option to false results in extracted video frames captured in the designated pixel format only. Setting this option to true results in receiving the raw DV stream untouched. </dd> </dl>

 <h4 class=”subsection”>18.3.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print the list of AVFoundation supported devices and exit: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -list_devices true -i "" </pre>

Record video from video device 0 and audio from audio device 0 into out.avi: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -i "0:0" out.avi </pre>

Record video from video device 2 and audio from audio device 1 into out.avi: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -video_device_index 2 -i ":1" out.avi </pre>

Record video from the system default video device using the pixel format bgr0 and do not record any audio into out.avi: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -pixel_format bgr0 -i "default:none" out.avi </pre>

Record raw DV data from a suitable input device and write the output into out.dv: <pre class=”example-preformatted”>$

ffmpeg -f avfoundation -capture_raw_data true -i "zr100:none" out.dv </pre>

 <h3 class="section">18.4 decklink# TOC</h3>

The decklink input device provides capture capabilities for Blackmagic DeckLink devices. To enable this input device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class="code">--extra-cflags</code>, <code class="code">--extra-cxxflags</code> and <code class="code">--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class="command">widl</code>. DeckLink is very picky about the formats it supports. Pixel format of the input can be set with <samp class="option">raw_format</samp>. Framerate and video size must be determined for your device with <code class="command">-list_formats 1</code>. Audio sample rate is always 48 kHz and the number of channels can be 2, 8 or 16. Note that all audio channels are bundled in one single audio track. <h4 class="subsection">18.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. This option is deprecated, please use the <code class="code">-sources</code> option of ffmpeg to list the available input devices. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of supported formats and exit. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">format_code <FourCC></samp></dt> <dd> This sets the input video format to the format given by the FourCC. To see the supported values of your device(s) use <samp class="option">list_formats</samp>. Note that there is a FourCC <samp class="option">'pal '</samp> that can also be used as <samp class="option">pal</samp> (3 letters). Default behavior is autodetection of the input video format, if the hardware supports it. </dd> <dt><samp class="option">raw_format</samp></dt> <dd> Set the pixel format of the captured video. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> This is the default which means 8-bit YUV 422 or 8-bit ARGB if format autodetection is used, 8-bit YUV 422 otherwise. </dd>

<dt>‘<samp class="samp">uyvy422</samp>’</dt> <dd> 8-bit YUV 422. </dd> <dt>‘<samp class="samp">yuv422p10</samp>’</dt> <dd> 10-bit YUV 422. </dd> <dt>‘<samp class="samp">argb</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class="samp">bgra</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class="samp">rgb10</samp>’</dt> <dd> 10-bit RGB. </dd> </dl>

</dd> <dt><samp class="option">teletext_lines</samp></dt> <dd> If set to nonzero, an additional teletext stream will be captured from the vertical ancillary data. Both SD PAL (576i) and HD (1080i or 1080p) sources are supported. In case of HD sources, OP47 packets are decoded. This option is a bitmask of the SD PAL VBI lines captured, specifically lines 6 to 22, and lines 318 to 335. Line 6 is the LSB in the mask. Selected lines which do not contain teletext information will be ignored. You can use the special <samp class="option">all</samp> constant to select all possible lines, or <samp class="option">standard</samp> to skip lines 6, 318 and 319, which are not compatible with all receivers. For SD sources, ffmpeg needs to be compiled with <code class="code">--enable-libzvbi</code>. For HD sources, on older (pre-4K) DeckLink card models you have to capture in 10 bit mode. </dd> <dt><samp class="option">channels</samp></dt> <dd> Defines number of audio channels to capture. Must be ‘<samp class="samp">2</samp>’, ‘<samp class="samp">8</samp>’ or ‘<samp class="samp">16</samp>’. Defaults to ‘<samp class="samp">2</samp>’. </dd> <dt><samp class="option">duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class="samp">unset</samp>’, ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’, ‘<samp class="samp">one_sub_device_full</sa ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Defaults to ‘<samp class="samp">unset</samp>’. Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0): ‘<samp class="samp">one_sub_device_full</samp>’, ‘<samp class="samp">one_sub_device_half</samp>’, ‘<samp class="samp">two_sub_device_full</samp>’, ‘<samp class="samp">four_sub_device_half</samp>’ Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class="samp">half</samp>’, ‘<samp class="samp">full</samp>’ </dd> <dt><samp class="option">timecode_format</samp>< <dd> Timecode type to include in the frame and video stream metadata. Must be ‘<samp class="samp">none</samp>’,

‘<samp class=”samp”>rp188vitc</samp>’, ‘<samp class=”samp”>rp188vitc2</samp>’, ‘<samp class=”samp”>rp188ltc</samp>’, ‘<samp class=”samp”>rp188hfr</samp>’, ‘<samp class=”samp”>rp188any</samp>’, ‘<samp class=”samp”>vitc</samp>’, ‘<samp class=”samp”>vitc2</samp>’, or ‘<samp class=”samp”>serial</samp>’. Defaults to ‘<samp class=”samp”>none</samp>’ (not included). </p> <p>In order to properly support 50/60 fps timecodes, the ordering of the queried timecode types for ‘<samp class=”samp”>rp188any</samp>’ is HFR, VITC1, VITC2 and LTC for >30 fps content. Note that this is slightly different to the ordering used by the DeckLink API, which is HFR, VITC1, LTC, VITC2. </p> </dd> <dt><samp class=”option”>video_input</samp></dt> <dd><p>Sets the video input source. Must be ‘<samp class=”samp”>unset</samp>’, ‘<samp class=”samp”>sdi</samp>’, ‘<samp class=”samp”>hdmi</samp>’, ‘<samp class=”samp”>optical_sdi</samp>’, ‘<samp class=”samp”>component</samp>’, ‘<samp class=”samp”>composite</samp>’ or ‘<samp class=”samp”>s_video</samp>’. Defaults to ‘<samp class=”samp”>unset</samp>’. </p> </dd> <dt><samp class=”option”>audio_input</samp></dt> <dd><p>Sets the audio input source. Must be ‘<samp class=”samp”>unset</samp>’, ‘<samp class=”samp”>embedded</samp>’, ‘<samp class=”samp”>aes_ebu</samp>’, ‘<samp class=”samp”>analog</samp>’, ‘<samp class=”samp”>analog_xlr</samp>’, ‘<samp class=”samp”>analog_rca</samp>’ or ‘<samp class=”samp”>microphone</samp>’. Defaults to ‘<samp class=”samp”>unset</samp>’. </p> </dd> <dt><samp class=”option”>video_pts</samp></dt> <dd><p>Sets the video packet timestamp source. Must be ‘<samp class=”samp”>video</samp>’, ‘<samp class=”samp”>audio</samp>’, ‘<samp class=”samp”>reference</samp>’, ‘<samp class=”samp”>wallclock</samp>’ or ‘<samp class=”samp”>abs_wallclock</samp>’. Defaults to ‘<samp class=”samp”>video</samp>’. </p> </dd> <dt><samp class=”option”>audio_pts</samp></dt> <dd><p>Sets the audio packet timestamp source. Must be ‘<samp class=”samp”>video</samp>’, ‘<samp class=”samp”>audio</samp>’, ‘<samp class=”samp”>reference</samp>’, ‘<samp class=”samp”>wallclock</samp>’ or ‘<samp class=”samp”>abs_wallclock</samp>’. Defaults to ‘<samp class=”samp”>audio</samp>’. </p> </dd> <dt><samp class=”option”>draw_bars</samp></dt> <dd><p>If set to ‘<samp class=”samp”>true</samp>’, color bars are drawn in the event of a signal loss. Defaults to ‘<samp class=”samp”>true</samp>’. This option is deprecated, please use the <code class=”code”>signal_loss_action</code> option. </p> </dd> <dt><samp class=”option”>signal_loss_action</samp></dt> <dd><p>Sets the action to take in the event of a signal loss. Accepts one of the following values: </p> <dl class=”table”> <dt><samp class=”option”>1, none</samp></dt> <dd><p>Do nothing on signal loss. This usually

results in black frames. </dd> <dt><samp class="option">2, bars</samp></dt> <dd> Draw color bars on signal loss. Only supported for 8-bit input signals. </dd> <dt><samp class="option">3, repeat</samp></dt> <dd> Repeat the last video frame on signal loss. </dd> </dl>

Defaults to ‘<samp class="samp">bars</samp>’. </dd> <dt><samp class="option">queue_size</samp></dt> <dd> Sets maximum input buffer size in bytes. If the buffering reaches this value, incoming frames will be dropped. Defaults to ‘<samp class="samp">1073741824</samp>’. </dd> <dt><samp class="option">audio_depth</samp></dt> <dd> Sets the audio sample bit depth. Must be ‘<samp class="samp">16</samp>’ or ‘<samp class="samp">32</samp>’. Defaults to ‘<samp class="samp">16</samp>’. </dd> <dt><samp class="option">decklink_copyts</samp></
<dd> If set to <samp class="option">true</samp>, timestamps are forwarded as they are without removing the initial offset. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">timestamp_align</samp></dt> <dd> Capture start time alignment in seconds. If set to nonzero, input frames are dropped till the system timestamp aligns with configured value. Alignment difference of up to one frame duration is tolerated. This is useful for maintaining input synchronization across N different hardware devices deployed for ’N-way’ redundancy. The system time of different hardware devices should be synchronized with protocols such as NTP or PTP, before using this option. Note that this method is not foolproof. In some border cases input synchronization may not happen due to thread scheduling jitters in the OS. Either sync could go wrong by 1 frame or in a rarer case <samp class="option">timestamp_align</samp> seconds. Defaults to ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">wait_for_tc (bool)</samp></dt>
<dd> Drop frames till a frame with timecode is received. Sometimes serial timecode isn’t received with the first input frame. If that happens, the stored stream timecode will be inaccurate. If this option is set to <samp class="option">true</samp>, input frames are dropped till a frame with timecode is received. Option <var class="var">timecode_format</var> must be specified. Defaults to <samp class="option">false</samp>. </dd>
<dt><samp class="option">enable_klv(bool)</samp></dt>
<dd> If set to <samp class="option">true</samp>, extracts KLV data from VANC and outputs KLV packets. KLV VANC packets are joined based on MID and PSC fields and aggregated into one KLV packet. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">18.4.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List input devices:
<pre class="example-preformatted">ffmpeg -sources decklink </pre>

List supported formats: <pre class="example-
preformatted">ffmpeg -f decklink -list_formats 1 -i 'Intensity Pro' </pre>

Capture video clip at 1080i50: <pre
class="example-preformatted">ffmpeg -format_code Hi50 -f decklink -i
'Intensity Pro' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 10 bit: <pre
class="example-preformatted">ffmpeg -raw_format yuv422p10 -format_code
Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi
</pre>

Capture video clip at 1080i50 with 16 audio channels: <div
class="example"> <pre class="example-preformatted">ffmpeg -channels 16
-format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v
copy output.avi </pre>

 <h3 class="section">18.5 dshow<span class="pull-
right">#
TOC</h3>

Windows DirectShow input device. DirectShow support
is enabled when FFmpeg is built with the mingw-w64 project. Currently
only audio and video devices are supported. Multiple devices
may be opened as separate inputs, but they may also be opened on the
same input, which should improve synchronism between them.
The input name should be in the format:
<pre class="example-preformatted"><var class="var">TYPE</var>=<var
class="var">NAME</var>[:<var class="var">TYPE</var>=<var class="var">NAME</var>]
</pre>

where <var class="var">TYPE</var> can be either <var class="var">audio</var>
or <var class="var">video</var>, and <var class="var">NAME</var> is
the device’s name or alternative name.. <a name="Options-
22"> <h4 class="subsection">18.5.1 Options<span class="pull-
right">#
TOC</h4>

If no options are specified, the device’s defaults are used.

If the device does not support the requested options, it will

fail to open.

<dl class="table">

<dt><samp class=”option”>video_size</samp></dt>

<dd> Set the video size in the captured video.

</dd>

<dt><samp class=”option”>framerate</samp></dt>

<dd> Set the frame rate in the captured video.

</dd>

<dt><samp class=”option”>sample_rate</samp></dt>

<dd> Set the sample rate (in Hz) of the captured audio.

</dd>

<dt><samp class=”option”>sample_size</samp></dt>

<dd> Set the sample size (in bits) of the captured audio.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set the number of channels in the captured audio.

</dd>

<dt><samp class=”option”>list_devices</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of devices and exit.

</dd>

<dt><samp class=”option”>list_options</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of selected device’s options

and exit.

</dd>

<dt><samp class="option">video_device_number</samp></dt>

<dd> Set video device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class="option">audio_device_number</samp></dt>

<dd> Set audio device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class="option">pixel_format</samp></dt>

<dd> Select pixel format to be used by DirectShow. This may only be set when

the video codec is not set or set to rawvideo.

</dd>

<dt><samp class="option">audio_buffer_size</samp></dt>

<dd> Set audio device buffer size in milliseconds (which can directly

impact latency, depending on the device).

Defaults to using the audio device’s

default buffer size (typically some multiple of 500ms).

Setting this value too low can degrade performance.

See also

us/library/windows/desktop/dd377582(v=vs.85).aspx

</dd>

<dt><samp class="option">video_pin_name</samp></dt>

<dd> Select video capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>audio_pin_name</samp></dt>

<dd> Select audio capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>crossbar_video_input_pin_number</samp></dt>

<dd> Select video input pin number for crossbar device. This will be

routed to the crossbar device’s Video Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>crossbar_audio_input_pin_number</samp></dt>

<dd> Select audio input pin number for crossbar device. This will be

routed to the crossbar device’s Audio Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_video_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts,
popup a display dialog

to the end user, allowing them to change video filter properties

and configurations manually.

Note that for crossbar devices, adjusting values in this dialog

may be needed at times to toggle between PAL (25 fps) and NTSC (29.97)

input frame rates, sizes, interlacing, etc. Changing these values can

enable different scan rates/frame rates and avoiding green bars at

the bottom, flickering scan lines, etc.

Note that with some devices, changing these properties can also affect future

invocations (sets new defaults) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_audio_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display dialog

to the end user, allowing them to change audio filter properties

and configurations manually.

</dd>

<dt><samp class=”option”>show_video_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens a video device.

</dd>

<dt><samp class=”option”>show_audio_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens an audio device.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV channels and frequencies.

</dd>

<dt><samp class=”option”>show_analog_tv_tuner_audio_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually
modify TV audio (like mono vs. stereo, Language A,B or C).

</dd>
<dt><samp class=”option”>audio_device_load</samp></dt>
<dd> Load an audio capture filter device from file instead of searching
it by name. It may load additional parameters too, if the filter
supports the serialization of its properties to.
To use this an audio capture source has to be specified, but it can
be anything even fake one.

</dd>
<dt><samp class=”option”>audio_device_save</samp></dt>
<dd> Save the currently used audio capture filter device and its
parameters (if the filter supports it) to a file.
If a file with the same name exists it will be overwritten.

</dd>
<dt><samp class=”option”>video_device_load</samp></dt>
<dd> Load a video capture filter device from file instead of searching
it by name. It may load additional parameters too, if the filter
supports the serialization of its properties to.
To use this a video capture source has to be specified, but it can
be anything even fake one.

</dd>
<dt><samp class=”option”>video_device_save</samp></dt>
<dd> Save the currently used video capture filter device and its
parameters (if the filter supports it) to a file.
If a file with the same name exists it will be overwritten.

</dd>

<dt><samp class=”option”>use_video_device_timestamps</samp></dt>

<dd> If set to <samp class=”option”>false</samp>, the timestamp for video frames will be

derived from the wallclock instead of the timestamp provided by

the capture device. This allows working around devices that

provide unreliable timestamps.

</dd>

</dl>

 <h4 class=”subsection”>18.5.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print the list of DirectShow supported devices and exit: <pre class=”example-preformatted”>$ ffmpeg -list_devices true -f dshow -i dummy </pre>

Open video device <var class=”var”>Camera</var>: <pre class=”example-preformatted”>$ ffmpeg -f dshow -i video="Camera" </pre>

Open second video device with name <var class=”var”>Camera</var>: <pre class=”example-preformatted”>$ ffmpeg -f dshow -video_device_number 1 -i video="Camera" </pre>

Open video device <var class=”var”>Camera</var> and audio device <var class=”var”>Microphone</var>: <pre class=”example-preformatted”>$ ffmpeg -f dshow -i video="Camera":audio="Microphone&q
</pre>

Print the list of supported options in selected device and exit: <pre class=”example-preformatted”>$ ffmpeg -list_options true -f dshow -i video="Camera" </pre>

Specify pin names to capture by name or alternative name, specify alternative device name: <pre class=”example-preformatted”>$ ffmpeg -f dshow -audio_pin_name "Audio Out" -video_pin_name 2 -i video=video="@device_pnp_\\?\pci#ven_1a0a&dev_6200&subsys_62
8f56-11d0-a3b9-00a0c9223196}\{ca465100-deb0-4d59-818f-8c477184adf6}":audio="Microphone&qu
</pre>

Configure a crossbar device, specifying crossbar pins, allow user to adjust video capture properties at startup: <pre

class=”example-preformatted”>$ ffmpeg -f dshow -show_video_device_dialog true -crossbar_video_input_pin_number 0 -crossbar_audio_input_pin_number 3 -i video="AVerMedia BDA Analog Capture":audio="AVerMedia BDA Analog Capture" </pre>

 <h3 class=”section”>18.6 fbdev# TOC</h3>

Linux framebuffer input device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class=”file”>/dev/fb0</samp>. For more detailed information read the file Documentation/fb/framebuffer.txt included in the Linux source tree. See also http://linux-fbdev.sourceforge.net/, and fbset(1). To record from the framebuffer device <samp class=”file”>/dev/fb0</samp> with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 10 -i /dev/fb0 out.avi </pre>

You can take a single screenshot image with the command: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 1 -i /dev/fb0 -frames:v 1 screenshot.jpeg </pre>

 <h4 class=”subsection”>18.6.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate. Default is 25. </dd> </dl>

 <h3 class=”section”>18.7 gdigrab# TOC</h3>

Win32 GDI-based screen capture device. This device allows you to capture a region of the display on Windows. Amongst options for the input filenames are such elements as: <pre class=”example-preformatted”>desktop </pre> or <pre class=”example-preformatted”>title=<var class=”var”>window_title</var> </pre> or <pre class=”example-preformatted”>hwnd=<var class=”var”>window_hwnd</var> </pre>

The first option will capture the entire desktop, or a fixed region of the desktop. The second and third options will instead capture the contents of a single window, regardless of its position on the screen. For example, to grab the entire desktop using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i desktop out.mpg </pre>

Grab a 640x480 region at position <code class="code">10,20</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -offset_x 10 -offset_y 20 -video_size vga -i desktop out.mpg </pre>

Grab the contents of the window named "Calculator" <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i title=Calculator out.mpg </pre>

 <h4 class="subsection">18.7.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. Use the value <code class="code">0</code> to not draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. Note that <var class="var">show_region</var> is incompatible with grabbing the contents of a single window. For example: <pre class="example-preformatted">ffmpeg -f gdigrab -show_region 1 -framerate 6 -video_size cif -offset_x 10 -offset_y 20 -i desktop out.mpg </pre>

</dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. The default is to capture the full screen if <samp class="file">desktop</samp> is selected, or the full window size if <samp class="file">title=<var class="var">window_title</var></samp> is selected. </dd> <dt><samp class="option">offset_x</samp></dt> <dd> When capturing a region with <var class="var">video_size</var>, set the distance from the left edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned to the left of your primary monitor, you will need to use a negative <var class="var">offset_x</var> value to move the region to that monitor. </dd> <dt><samp

class=”option”>offset_y</samp></dt> <dd> When capturing a region with <var class=”var”>video_size</var>, set the distance from the top edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned above your primary monitor, you will need to use a negative <var class=”var”>offset_y</var> value to move the region to that monitor. </dd> </dl>

 <h3 class=”section”>18.8 iec61883# TOC</h3>

FireWire DV/HDV input device using libiec61883. To enable this input device, you need libiec61883, libraw1394 and libavc1394 installed on your system. Use the configure option <code class=”code”>--enable-libiec61883</code> to compile with the device enabled. The iec61883 capture device supports capturing from a video device connected via IEEE1394 (FireWire), using libiec61883 and the new Linux FireWire stack (juju). This is the default DV/HDV input method in Linux Kernel 2.6.37 and later, since the old FireWire stack was removed. Specify the FireWire port to be used as input file, or "auto" to choose the first port connected. <h4 class=”subsection”>18.8.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>dvtype</samp></dt> <dd> Override autodetection of DV/HDV. This should only be used if auto detection does not work, or if usage of a different device type should be prohibited. Treating a DV device as HDV (or vice versa) will not work and result in undefined behavior. The values <samp class=”option”>auto</samp>, <samp class=”option”>dv</samp> and <samp class=”option”>hdv</samp> are supported. </dd> <dt><samp class=”option”>dvbuffer</samp></dt> <dd> Set maximum size of buffer for incoming data, in frames. For DV, this is an exact value. For HDV, it is not frame exact, since HDV does not have a fixed frame size. </dd> <dt><samp class=”option”>dvguid</samp></dt> <dd> Select the capture device by specifying its GUID. Capturing will only be performed from the specified device and fails if no device with the given GUID is found. This is useful to select the input if multiple devices are connected at the same time. Look at /sys/bus/firewire/devices to find out the GUIDs. </dd> </dl>

 <h4 class=”subsection”>18.8.2 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-13" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Grab and show the input of a FireWire DV/HDV device. <pre class="example-preformatted">ffplay -f iec61883 -i auto </pre>

Grab and record the input of a FireWire DV/HDV device, using a packet buffer of 100000 packets if the source is HDV. <pre class="example-preformatted">ffmpeg -f iec61883 -i auto -dvbuffer 100000 out.mpg </pre>

 <h3 class="section">18.9 jack# TOC</h3>

JACK input device. To enable this input device during configuration you need libjack installed on your system. A JACK input device creates one or more JACK writable clients, one for each audio channel, with name <var class="var">client_name</var>:input_<var class="var">N</var>, where <var class="var">client_name</var> is the name provided by the application, and <var class="var">N</var> is a number which identifies the channel. Each writable client will send the acquired data to the FFmpeg input device. Once you have created one or more JACK readable clients, you need to connect them to one or more JACK writable clients. To connect or disconnect JACK clients you can use the <code class="command">jack_connect</code> and <code class="command">jack_disconnect</code> programs, or do it through a graphical interface, for example with <code class="command">qjackctl</code>. To list the JACK clients and their properties you can invoke the command <code class="command">jack_lsp</code>. Follows an example which shows how to capture a JACK readable client with <code class="command">ffmpeg</code>. <pre class="example-preformatted"># Create a JACK writable client with name "ffmpeg". $ ffmpeg -f jack -i ffmpeg -y out.wav

Start the sample jack_metro readable client. $ jack_metro -b 120 -d 0.2 -f 4000

List the current JACK clients. $ jack_lsp -c system:capture_1 system:capture_2 system:playback_1 system:playback_2 ffmpeg:input_1 metro:120_bpm

Connect metro to the ffmpeg writable client. $ jack_connect metro:120_bpm ffmpeg:input_1 </pre>

For more information read: http://jackaudio.org/ <h4 class="subsection">18.9.1 Options<a class="anchor hidden-xs" href="#Options-

26" aria-hidden="true"># TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">18.10 kmsgrab# TOC</h3>

KMS video input device. Captures the KMS scanout framebuffer associated with a specified CRTC or plane as a DRM object that can be passed to other hardware functions. Requires either DRM master or CAP_SYS_ADMIN to run. If you don’t understand what all of that means, you probably don’t want this. Look at <samp class="option">x11grab</samp> instead. <h4 class="subsection">18.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">device</samp></dt> <dd> DRM device to capture on. Defaults to <samp class="option">/dev/dri/card0</samp>. </dd> <dt><samp class="option">format</samp></dt> <dd> Pixel format of the framebuffer. This can be autodetected if you are running Linux 5.7 or later, but needs to be provided for earlier versions. Defaults to <samp class="option">bgr0</samp>, which is the most common format used by the Linux console and Xorg X server. </dd> <dt><samp class="option">format_modifier</samp></dt> <dd> Format modifier to signal on output frames. This is necessary to import correctly into some APIs. It can be autodetected if you are running Linux 5.7 or later, but will need to be provided explicitly when needed in earlier versions. See the libdrm documentation for possible values. </dd> <dt><samp class="option">crtc_id</samp></dt> <dd> KMS CRTC ID to define the capture source. The first active plane on the given CRTC will be used. </dd> <dt><samp class="option">plane_id</samp></dt> <dd> KMS plane ID to define the capture source. Defaults to the first active plane found if neither <samp class="option">crtc_id</samp> nor <samp class="option">plane_id</samp> are specified. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Framerate to capture at. This is not synchronised to any page flipping or framebuffer changes - it just defines the interval at which the framebuffer is sampled. Sampling faster than the framebuffer update rate will generate independent frames with the same content. Defaults to <code class="code">30</code>. </dd> </dl>

 <h4 class="subsection">18.10.2 Exam-

ples# TOC</h4>

<ul class=”itemize mark-bullet”> Capture from the first active plane, download the result to normal frames and encode. This will only work if the framebuffer is both linear and mappable - if not, the result may be scrambled or fail to download. <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwdownload,format=bgr0' output.mp4 </pre>

Capture from CRTC ID 42 at 60fps, map the result to VAAPI, convert to NV12 and encode as H.264. <pre class=”example-preformatted”>ffmpeg -crtc_id 42 -framerate 60 -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,scale_vaapi=w=1920:h=1080:format=nv12' -c:v h264_vaapi output.mp4 </pre>

To capture only part of a plane the output can be cropped - this can be used to capture a single window, as long as it has a known absolute position and size. For example, to capture and encode the middle quarter of a 1920x1080 plane: <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,crop=960:540:480:270,scale_vaapi=960:540:nv12' -c:v h264_vaapi output.mp4 </pre>

 <h3 class=”section”>18.11 lavfi# TOC</h3>

Libavfilter input virtual device. This input device reads data from the open output pads of a libavfilter filtergraph. For each filtergraph open output, the input device will create a corresponding stream which is mapped to the generated output. The filtergraph is specified through the option <samp class=”option”>graph</samp>. <h4 class=”subsection”>18.11.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>graph</samp></dt> <dd> Specify the filtergraph to use as input. Each video open output must be labelled by a unique string of the form "out<var class=”var”>N</var>", where <var class=”var”>N</var> is a number starting from 0 corresponding to the mapped input stream generated by the device. The first unlabelled output is automatically assigned to the "out0" label, but all the others need to be specified explicitly. The suffix "+subcc" can be appended to the output label to create an extra stream with the closed captions packets attached to that output

(experimental; only for EIA-608 / CEA-708 for now). The subcc streams are created after all the normal streams, in the order of the corresponding stream. For example, if there is "out19+subcc", "out7+subcc" and up to "out42", the stream #43 is subcc for stream #7 and stream #44 is subcc for stream #19. If not specified defaults to the filename specified for the input device. </dd> <dt><samp class="option">graph_file</samp></dt> <dd> Set the filename of the filtergraph to be read and sent to the other filters. Syntax of the filtergraph is the same as the one specified by the option <var class="var">graph</var>. </dd> <dt><samp class="option">dumpgraph</samp></dt> <dd> Dump graph to stderr. </dd> </dl>

 <h4 class="subsection">18.11.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Create a color video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi -graph "color=c=pink [out0]" dummy </pre>

As the previous example, but use filename for specifying the graph description, and omit the "out0" label: <pre class="example-preformatted">ffplay -f lavfi color=c=pink </pre>

Create three different video test filtered sources and play them: <pre class="example-preformatted">ffplay -f lavfi -graph "testsrc [out0]; testsrc,hflip [out1]; testsrc,negate [out2]" test3 </pre>

Read an audio stream from a file using the amovie source and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "amovie=test.wav" </pre>

Read an audio stream and a video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "movie=test.avi[out0];amovie=test.wav[out1]" </pre>

Dump decoded frames to images and Closed Captions to an RCWT backup: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=test.ts[out0+subcc]" -map v frame%08d.png -map s -c copy -f rcwt subcc.bin </pre>

 <h3 class="section">18.12 libcdio<a class="anchor hidden-xs" href="#libcdio" aria-

hidden=”true”># TOC</h3>

Audio-CD input device based on libcdio. To enable this input device during configuration you need libcdio installed on your system. It requires the configure option <code class=”code”>--enable-libcdio</code>. This device allows playing and grabbing from an Audio-CD. For example to copy with <code class=”command”>ffmpeg</code> the entire Audio-CD in <samp class=”file”>/dev/sr0</samp>, you may run the command: <pre class=”example-preformatted”>ffmpeg -f libcdio -i /dev/sr0 cd.wav </pre>

 <h4 class=”subsection”>18.12.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>speed</samp></dt> <dd> Set drive reading speed. Default value is 0. The speed is specified CD-ROM speed units. The speed is set through the libcdio <code class=”code”>cdio_cddap_speed_set</code> function. On many CD-ROM drives, specifying a value too large will result in using the fastest speed. </dd> <dt><samp class=”option”>paranoia_mode</samp></dt> <dd> Set paranoia recovery mode flags. It accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>disable</samp>’</dt> <dt>‘<samp class=”samp”>verify</samp>’</dt> <dt>‘<samp class=”samp”>overlap</samp>’</dt> <dt>‘<samp class=”samp”>neverskip</samp>&rsquo <dt>‘<samp class=”samp”>full</samp>’</dt> </dl>

Default value is ‘<samp class=”samp”>disable</samp>’. For more information about the available recovery modes, consult the paranoia project documentation. </dd> </dl>

 <h3 class=”section”>18.13 libdc1394# TOC</h3>

IIDC1394 input device, based on libdc1394 and libraw1394. Requires the configure option <code class=”code”>--enable-libdc1394</code>. <h4 class=”subsection”>18.13.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate. Default is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Select the pixel format. Default is <code class=”code”>uyvy422</code>.

 </dd> <dt><samp class="option">video_size</samp></dt>
<dd> Set the video size given as a string such as <code class="code">640x480</code>
or <code class="code">hd720</code>. Default is <code class="code">qvga</code>.
</dd> </dl>

 <h3 class="section">18.14 openal<span
class="pull-right"><a class="anchor hidden-xs" href="#openal" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-openal"
aria-hidden="true">TOC</h3>

The OpenAL input device provides audio capture on all systems with
a working OpenAL 1.1 implementation. To enable this in-
put device during configuration, you need OpenAL headers and libraries
installed on your system, and need to configure FFmpeg with <code
class="code">--enable-openal</code>. OpenAL headers and
libraries should be provided as part of your OpenAL implementation,
or as an additional download (an SDK). Depending on your installation
you may need to specify additional flags via the <code class="code">--
extra-cflags</code> and <code class="code">--extra-ldflags</code> for
allowing the build system to locate the OpenAL headers and libraries.
An incomplete list of OpenAL implementations follows: <dl
class="table"> <dt>Creative</dt>
<dd> The official Windows implementation, providing hardware ac-
celeration with supported devices and software fallback. See <a class="url"
href="http://openal.org/">http://openal.org/. </dd> <dt><strong
class="strong">OpenAL Soft</dt> <dd> Portable, open
source (LGPL) software implementation. Includes backends for the most com-
mon sound APIs on the Windows, Linux, Solaris, and BSD operating systems.
See http://kcat.strangesoft.net/openal.html<
</dd> <dt>Apple</dt> <dd> OpenAL
is part of Core Audio, the official Mac OS X Audio interface. See <a
class="url" href="http://developer.apple.com/technologies/mac/audio-
and-video.html">http://developer.apple.com/technologies/mac/audio-and-
video.html </dd> </dl>

This device allows one to capture from an audio input device handled
through OpenAL. You need to specify the name of the device to
capture in the provided filename. If the empty string is provided, the device will
automatically select the default device. You can get the list of the supported
devices by using the option <var class="var">list_devices</var>. <a
name="Options-31"> <h4 class="subsection">18.14.1 Options<span
class="pull-right"><a class="anchor hidden-xs" href="#Options-31" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Options-31"
aria-hidden="true">TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt>
<dd> Set the number of channels in the captured audio. Only
the values <samp class="option">1</samp> (monaural) and <samp

class=”option”>2</samp> (stereo) are currently supported. Defaults to <samp class=”option”>2</samp>. </dd> <dt><samp class=”option”>sample_size</samp></dt> <dd> Set the sample size (in bits) of the captured audio. Only the values <samp class=”option”>8</samp> and <samp class=”option”>16</samp> are currently supported. Defaults to <samp class=”option”>16</samp>. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate (in Hz) of the captured audio. Defaults to <samp class=”option”>44.1k</samp>. </dd> <dt><samp class=”option”>list_devices</samp></dt> <dd> If set to <samp class=”option”>true</samp>, print a list of devices and exit. Defaults to <samp class=”option”>false</samp>. </dd> </dl>

 <h4 class=”subsection”>18.14.2 Examples# TOC</h4>

Print the list of OpenAL supported devices and exit: <pre class=”example-preformatted”>$ ffmpeg -list_devices true -f openal -i dummy out.ogg </pre>

Capture from the OpenAL device <samp class=”file”>DR-BT101 via PulseAudio</samp>: <pre class=”example-preformatted”>$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out.ogg </pre>

Capture from the default device (note the empty string ” as filename): <pre class=”example-preformatted”>$ ffmpeg -f openal -i '' out.ogg </pre>

Capture from two devices simultaneously, writing to two different files, within the same <code class=”command”>ffmpeg</code> command: <pre class=”example-preformatted”>$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out1.ogg -f openal -i 'ALSA Default' out2.ogg </pre> Note: not all OpenAL implementations support multiple simultaneous capture - try the latest OpenAL Soft if the above does not work. <h3 class=”section”>18.15 oss# TOC</h3>

Open Sound System input device. The filename to provide to the input device is the device node representing the OSS input device, and is usually set to <samp class=”file”>/dev/dsp</samp>. For example to grab from <samp class=”file”>/dev/dsp</samp> using <code class=”command”>ffmpeg</code> use the command: <pre class=”example-preformatted”>ffmpeg -f oss -i /dev/dsp /tmp/oss.wav </pre>

For more information about OSS see: <a class="url" href="http://manuals.opensound.com/usersguide/ds
  <h4 class="subsection">18.15.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">18.16 pulse# TOC</h3>

PulseAudio input device. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libpulse</code>. The filename to provide to the input device is a source device or the string "default" To list the PulseAudio source devices and their properties you can invoke the command <code class="command">pactl list sources</code>. More information about PulseAudio can be found on http://www.pulseaudio.org. <h4 class="subsection">18.16.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class="option">name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class="code">LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class="option">stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active streams, by default it is "record". </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Specify the samplerate in Hz, by default 48kHz is used. </dd> <dt><samp class="option">channels</samp></dt> <dd> Specify the channels in use, by default 2 (stereo) is set. </dd> <dt><samp class="option">frame_size</samp></dt> <dd> This option does nothing and is deprecated. </dd> <dt><samp class="option">fragment_size</samp></dt> <dd> Specify the size in bytes of the minimal buffering fragment in PulseAudio, it will affect the audio latency. By default it is set to 50 ms amount of data. </dd> <dt><samp class="option">wallclock</samp></dt> <dd> Set the initial PTS using the current time. Default is 1. </dd> </dl>

 <h4 class="subsection">18.16.2 Exam-

ples# TOC</h4> Record a stream from default device: <pre class="example-preformatted">ffmpeg -f pulse -i default /tmp/pulse.wav </pre>

 <h3 class="section">18.17 sndio# TOC</h3>

sndio input device. To enable this input device during configuration you need libsndio installed on your system. The filename to provide to the input device is the device node representing the sndio input device, and is usually set to <samp class="file">/dev/audio0</samp>. For example to grab from <samp class="file">/dev/audio0</samp> using <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -f sndio -i /dev/audio0 /tmp/oss.wav </pre>

 <h4 class="subsection">18.17.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">18.18 video4linux2, v4l2# TOC</h3

Video4Linux2 input video device. "v4l2" can be used as alias for "video4linux2". If FFmpeg is built with v4l-utils support (by using the <code class="code">--enable-libv4l2</code> configure option), it is possible to use it with the <code class="code">-use_libv4l2</code> input device option. The name of the device to grab is a file device node, usually Linux systems tend to automatically create such nodes when the device (e.g. an USB webcam) is plugged into the system, and has a name of the kind <samp class="file">/dev/video<var class="var">N</var></samp>, where <var class="var">N</var> is a number associated to the device. Video4Linux2 devices usually support a limited set of <var class="var">width</var>x<var class="var">height</var> sizes and frame rates. You can check which are supported using <code class="command">-list_formats all</code> for Video4Linux2 devices. Some devices, like TV cards, support one or more standards. It is possible to list all the supported standards using <code

class=”command”>-list_standards all</code>. The time base for the timestamps is 1 microsecond. Depending on the kernel version and configuration, the timestamps may be derived from the real time clock (origin at the Unix Epoch) or the monotonic clock (origin usually at boot time, unaffected by NTP or manual changes to the clock). The <samp class=”option”>-timestamps abs</samp> or <samp class=”option”>-ts abs</samp> option can be used to force conversion into the real time clock. Some usage examples of the video4linux2 device with <code class=”command”>ffmpeg</code> and <code class=”command”>ffplay</code>: <ul class=”itemize mark-bullet”> List supported formats for a video4linux2 device: <pre class=”example-preformatted”>ffplay -f video4linux2 -list_formats all /dev/video0 </pre>

Grab and show the input of a video4linux2 device: <pre class=”example-preformatted”>ffplay -f video4linux2 -framerate 30 -video_size hd720 /dev/video0 </pre>

Grab and record the input of a video4linux2 device, leave the frame rate and size as previously set: <pre class=”example-preformatted”>ffmpeg -f video4linux2 -input_format mjpeg -i /dev/video0 out.mpeg </pre>

For more information about Video4Linux, check http://linuxtv.org/. <h4 class=”subsection”>18.18.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>standard</samp></dt> <dd> Set the standard. Must be the name of a supported standard. To get a list of the supported standards, use the <samp class=”option”>list_standards</samp> option. </dd> <dt><samp class=”option”>channel</samp></dt> <dd> Set the input channel number. Default to -1, which means using the previously selected channel. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video frame size. The argument must be a string in the form <var class=”var”>WIDTH</var>x<var class=”var”>HEIGHT</var> or a valid size abbreviation. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Select the pixel format (only valid for raw video input). </dd> <dt><samp class=”option”>input_format</samp></dt> <dd> Set the preferred pixel format (for raw video) or a codec name. This option allows one to select the input format, when several are available. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the preferred video frame rate. </dd> <dt><samp class=”option”>list_formats</samp></dt> <dd> List available formats (supported pixel formats, codecs, and frame sizes) and exit. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Show all available (compressed and non-compressed) formats. </dd>

<dt>‘<samp class="samp">raw</samp>’</dt> <dd> Show only raw video (non-compressed) formats. </dd> <dt>‘<samp class="samp">compressed</samp>’</dt> <dd> Show only compressed formats. </dd> </dl>

</dd> <dt><samp class="option">list_standards</samp></dt> <dd> List supported standards and exit. Available values are: <dl class="table"> <dt>‘<samp class="samp">all</samp>’</dt> <dd> Show all supported standards. </dd> </dl>

</dd> <dt><samp class="option">timestamps, ts</samp></dt> <dd> Set type of timestamps for grabbed frames. Available values are: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Use timestamps from the kernel. </dd> <dt>‘<samp class="samp">abs</samp>’</dt> <dd> Use absolute timestamps (wall clock). </dd> <dt>‘<samp class="samp">mono2abs</samp>’</dt> <dd> Force conversion from monotonic to absolute timestamps. </dd> </dl>

Default value is <code class="code">default</code>. </dd> <dt><samp class="option">use_libv4l2</samp></dt> <dd> Use libv4l2 (v4l-utils) conversion functions. Default is 0. </dd> </dl>

 <h3 class="section">18.19 vfwcap# TOC</h3>

VfW (Video for Windows) capture input device. The filename passed as input is the capture driver number, ranging from 0 to 9. You may use "list" as filename to print a list of drivers. Any other filename will be interpreted as device number 0. <h4 class="subsection">18.19.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> </dl>

 <h3 class="section">18.20 x11grab# TOC</h3>

X11 video input device. To enable this input device during configuration you need libxcb installed on your system. It will

be automatically detected during configuration. This device allows one to capture a region of an X11 display. The filename passed as input has the syntax: <pre class=”example-preformatted”>[<var class=”var”>hostname</var>]:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var>[+<var class=”var”>x_offset</var>,<var class=”var”>y_offset</var>] </pre>

<var class=”var”>hostname</var>:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var> specifies the X11 display name of the screen to grab from. <var class=”var”>hostname</var> can be omitted, and defaults to "localhost". The environment variable <code class=”env”>DISPLAY</code> contains the default display name. <var class=”var”>x_offset</var> and <var class=”var”>y_offset</var> specify the offsets of the grabbed area with respect to the top-left border of the X11 screen. They default to 0. Check the X11 documentation (e.g. <code class=”command”>man X</code>) for more detailed information. Use the <code class=”command”>xdpyinfo</code> program for getting basic information about the properties of your X11 display (e.g. grep for "name" or "dimensions"). For example to grab from <samp class=”file”>:0.0</samp> using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

Grab at position <code class=”code”>10,20</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

 <h4 class=”subsection”>18.20.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>select_region</samp></dt> <dd> Specify whether to select the grabbing area graphically using the pointer. A value of <code class=”code”>1</code> prompts the user to select the grabbing area graphically by clicking and dragging. A single click with no dragging will select the whole screen. A region with zero width or height will also select the whole screen. This option overwrites the <var class=”var”>video_size</var>, <var class=”var”>grab_x</var>, and <var class=”var”>grab_y</var> options. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. A value of <code class=”code”>0</code> specifies not to draw the pointer. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>follow_mouse</samp></dt> <dd> Make the grabbed area follow the mouse. The argument can be <code class=”code”>centered</code> or a number of pixels

<var class=”var”>PIXELS</var>. When it is specified with "centered", the grabbing region follows the mouse pointer and keeps the pointer at the center of region; otherwise, the region follows only when the mouse pointer reaches within <var class=”var”>PIXELS</var> (greater than zero) to the edge of region. For example: <pre class=”example-preformatted”>ffmpeg -f x11grab -follow_mouse centered -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

To follow only when the mouse pointer reaches within 100 pixels to edge: <pre class=”example-preformatted”>ffmpeg -f x11grab -follow_mouse 100 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>show_region</samp></dt> <dd> Show grabbed region on screen. If <var class=”var”>show_region</var> is specified with <code class=”code”>1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. </dd> <dt><samp class=”option”>region_border</samp></dt> <dd> Set the region border thickness if <samp class=”option”>-show_region 1</samp> is used. Range is 1 to 128 and default is 3 (XCB-based x11grab only). For example: <pre class=”example-preformatted”>ffmpeg -f x11grab -show_region 1 -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

With <var class=”var”>follow_mouse</var>: <pre class=”example-preformatted”>ffmpeg -f x11grab -follow_mouse centered -show_region 1 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class=”option”>window_id</samp></dt> <dd> Grab this window, instead of the whole screen. Default value is 0, which maps to the whole screen (root window). The id of a window can be found using the <code class=”command”>xwininfo</code> program, possibly with options -tree and -root. If the window is later enlarged, the new area is not recorded. Video ends when the window is closed, unmapped (i.e., iconified) or shrunk beyond the video size (which defaults to the initial window size). This option disables options <samp class=”option”>follow_mouse</samp> and <samp class=”option”>select_region</samp>. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video frame size. Default is the full desktop or window. </dd> <dt><samp class=”option”>grab_x</samp></dt> <dt><samp class=”option”>grab_y</samp></dt> <dd> Set the grabbing region coordinates. They are expressed as offset from the top left corner of the X11

window and correspond to the <var class="var">x_offset</var> and <var class="var">y_offset</var> parameters in the device name. The default value for both options is 0. </dd> </dl>

 <h2 class="chapter">19 Resampler Options# TOC</h2>

The audio resampler supports the following named options. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, <var class="var">option</var>=<var class="var">value</var> for the aresample filter, by setting the value explicitly in the <code class="code">SwrContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. <dl class="table"> <dt><samp class="option">uchl, used_chlayout</samp></dt> <dd> Set used input channel layout. Default is unset. This option is only used for special remapping. </dd> <dt><samp class="option">isr, in_sample_rate</samp></dt> <dd> Set the input sample rate. Default value is 0. </dd> <dt><samp class="option">osr, out_sample_rate</samp></dt> <dd> Set the output sample rate. Default value is 0. </dd> <dt><samp class="option">isf, in_sample_fmt</samp></dt> <dd> Specify the input sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">osf, out_sample_fmt</samp></dt> <dd> Specify the output sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">tsf, internal_sample_fmt</samp></dt> <dd> Set the internal sample format. Default value is <code class="code">none</code>. This will automatically be chosen when it is not explicitly set. </dd> <dt><samp class="option">ichl, in_chlayout</samp></dt> <dt><samp class="option">ochl, out_chlayout</samp></dt> <dd> Set the input/output channel layout. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> <dt><samp class="option">clev, center_mix_level</samp></dt> <dd> Set the center mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">slev, surround_mix_level</samp></dt> <dd> Set the surround mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">lfe_mix_level</samp></dt> <dd> Set LFE mix into non LFE level. It is used when there is a LFE input but no LFE output. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">rmvol, rematrix_volume</samp></dt> <dd> Set rematrix volume. Default value is 1.0. </dd> <dt><samp class="option">rematrix_maxval</samp></dt> <dd> Set maxi-

mum output value for rematrixing. This can be used to prevent clipping vs. preventing volume reduction. A value of 1.0 prevents clipping. </dd> <dt><samp class="option">flags, swr_flags</samp></dt> <dd> Set flags used by the converter. Default value is 0. It supports the following individual flags: <dl class="table"> <dt><samp class="option">res</samp></dt> <dd> force resampling, this flag forces resampling to be used even when the input and output sample rates match. </dd> </dl>

</dd> <dt><samp class="option">dither_scale</samp></dt> <dd> Set the dither scale. Default value is 1. </dd> <dt><samp class="option">dither_method</samp></dt>
<dd> Set dither method. Default value is 0. Supported values: <dl class="table"> <dt>‘<samp class="samp">rectangular</samp>’</dt> <dd> select rectangular dither </dd> <dt>‘<samp class="samp">triangular</samp>’</dt> <dd> select triangular dither </dd> <dt>‘<samp class="samp">triangular_hp</samp>’</dt> <dd> select triangular dither with high pass </dd> <dt>‘<samp class="samp">lipshitz</samp>’</dt> <dd> select Lipshitz noise shaping dither. </dd> <dt>‘<samp class="samp">shibata</samp>’</dt> <dd> select Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">low_shibata</samp>’</dt> <dd> select low Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">high_shibata</samp>’</
<dd> select high Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">f_weighted</samp>’</dt> <dd> select f-weighted noise shaping dither </dd> <dt>‘<samp class="samp">modified_e_weighted</samp>
<dd> select modified-e-weighted noise shaping dither </dd> <dt>‘<samp class="samp">improved_e_weighted</samp>’</dt> <dd> select improved-e-weighted noise shaping dither </dd> </dl>

</dd> <dt><samp class="option">resampler</samp></dt> <dd> Set resampling engine. Default value is swr. Supported values: <dl class="table"> <dt>‘<samp class="samp">swr</samp>’</dt> <dd> select the native SW Resampler; filter options precision and cheby are not applicable in this case. </dd> <dt>‘<samp class="samp">soxr</samp>’</dt> <dd> select the SoX Resampler (where available); compensation, and filter options filter_size, phase_shift, exact_rational, filter_type & kaiser_beta, are not applicable in this case. </dd> </dl>

</dd>

<dt><samp class="option">filter_size</samp></dt>

<dd> For swr only, set resampling filter size, default value is 32.

</dd>

<dt><samp class="option">phase_shift</samp></dt>

<dd> For swr only, set resampling phase shift, default value is 10, and must be in

the interval [0,30].

</dd>

<dt><samp class="option">linear_interp</samp></dt>

<dd> Use linear interpolation when enabled (the default). Disable it if you want

to preserve speed instead of quality when exact_rational fails.

</dd>

<dt><samp class="option">exact_rational</samp></dt>

<dd> For swr only, when enabled, try to use exact phase_count based on input and

output sample rate. However, if it is larger than <code class="code">1 << phase_shift</code>,

the phase_count will be <code class="code">1 << phase_shift</code> as fallback. Default is enabled.

</dd>

<dt><samp class="option">cutoff</samp></dt>

<dd> Set cutoff frequency (swr: 6dB point; soxr: 0dB point) ratio; must be a float

value between 0 and 1. Default value is 0.97 with swr, and 0.91 with soxr

(which, with a sample-rate of 44100, preserves the entire audio band to 20kHz).

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> For soxr only, the precision in bits to which the resampled signal will be

calculated. The default value of 20 (which, with suitable dithering, is

appropriate for a destination bit-depth of 16) gives SoX’s ’High Quality’; a

value of 28 gives SoX’s ’Very High Quality’.

</dd>

<dt><samp class="option">cheby</samp></dt>

<dd> For soxr only, selects passband rolloff none (Chebyshev) & higher-precision

approximation for ’irrational’ ratios. Default value is 0.

</dd>

<dt><samp class="option">async</samp></dt>

<dd> For swr only, simple 1 parameter audio sync to timestamps using stretching,

squeezing, filling and trimming. Setting this to 1 will enable filling and

trimming, larger values represent the maximum amount in samples that the data

may be stretched or squeezed for each second.

Default value is 0, thus no compensation is applied to make the samples match

the audio timestamps.

</dd>

<dt><samp class="option">first_pts</samp></dt>

<dd> For swr only, assume the first pts should be this value. The time unit is 1 / sample rate.

This allows for padding/trimming at the start of stream. By default, no

assumption is made about the first frame’s expected pts, so no padding or

trimming is done. For example, this could be set to 0 to pad the beginning with

silence if an audio stream starts after the video stream or to trim any samples

with a negative pts due to encoder delay.

</dd>

<dt><samp class="option">min_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger stretching/squeezing/filling or trimming of the

data to make it match the timestamps. The default is that

stretching/squeezing/filling and trimming is disabled

(<samp class="option">min_comp</samp> = <code class="code">FLT_MAX</code>).

</dd>

<dt><samp class="option">min_hard_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger adding/dropping samples to make it match the

timestamps. This option effectively is a threshold to select between

hard (trim/fill) and soft (squeeze/stretch) compensation. Note that

all compensation is by default disabled through <samp class="option">min_comp</samp>.

The default is 0.1.

</dd>

<dt><samp class="option">comp_duration</samp></dt>

<dd> For swr only, set duration (in seconds) over which data is stretched/squeezed

to make it match the timestamps. Must be a non-negative double float value,

default value is 1.0.

</dd>

<dt><samp class="option">max_soft_comp</samp></dt>

<dd> For swr only, set maximum factor by which data is stretched/squeezed to make it

match the timestamps. Must be a non-negative double float value, default value

is 0.

</dd>

<dt><samp class=”option”>matrix_encoding</samp></dt>

<dd> Select matrixed stereo encoding.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>none</samp>’</dt>

<dd> select none

</dd>

<dt>‘<samp class=”samp”>dolby</samp>’</dt>

<dd> select Dolby

</dd>

<dt>‘<samp class=”samp”>dplii</samp>’</dt>

<dd> select Dolby Pro Logic II

</dd>

</dl>

Default value is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>filter_type</samp></dt> <dd> For swr only, select resampling filter type. This only affects resampling operations. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>cubic</samp>’</dt> <dd> select cubic </dd> <dt>‘<samp class=”samp”>blackman_nuttall</samp>’</dt> <dd> select Blackman Nuttall windowed sinc </dd> <dt>‘<samp class=”samp”>kaiser</samp>’</dt> <dd> select Kaiser windowed sinc </dd> </dl>

</dd> <dt><samp class=”option”>kaiser_beta</samp></dt> <dd> For swr only, set Kaiser window beta value. Must be a double float value in the interval [2,16], default value is 9. </dd> <dt><samp class=”option”>output_sample_bits</samp></dt> <dd> For swr only, set number of used output sample bits for dithering. Must be an integer in the interval [0,64], default value is 0, which means it’s not used. </dd> </dl>

 <h2 class=”chapter”>20 Scaler Options# TOC</h2>

The video scaler supports the following named options. Options may be set by specifying -<var class=”var”>option</var>

<var class=”var”>value</var> in the FFmpeg tools, with a few API-only exceptions noted below. For programmatic use, they can be set explicitly in the <code class=”code”>SwsContext</code> options or through the <samp class=”file”>libavutil/opt.h</samp> API. <dl class=”table”> <dd> </dd> <dt><samp class=”option”>scaler, scaler_sub</samp></dt> <dd> Choose the scaling algorithm to use. Default value is ‘<samp class=”samp”>auto</samp>’ for both. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Aumotic choice. For ‘<samp class=”samp”>scaler_sub</samp>’, this means the same algorithm as ‘<samp class=”samp”>scaler</samp>’. For ‘<samp class=”samp”>scaler</samp>’, this defaults to the scaler flag selected by ‘<samp class=”samp”>sws_flags</samp>’. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear filter. (AKA triangle filter) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> 2-tap cubic BC-spline (AKA Mitchell-Netravali spline). The B and C parameters can be configured by setting <code class=”code”>param0</code> and <code class=”code”>param1</code>, defaulting to 0.0 and 0.6 respectively. </dd> <dt>‘<samp class=”samp”>point, neighbor</samp>’</dt> <dd> Point sampling (AKA nearest neighbor). </dd> <dt>‘<samp class=”samp”>area</samp>’</dt> <dd> Area averaging. Equivalent to ‘<samp class=”samp”>bilinear</samp>’ for upscaling. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> 2-tap Gaussian filter approximation. The sharpness parameter can be configured by setting <code class=”code”>param0</code>, defaulting to 3.0. </dd> <dt>‘<samp class=”samp”>sinc</samp>’</dt> <dd> Unwindowed sinc filter. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Lanczos resampling (sinc windowed sinc). The number of filter taps can be configured by setting <code class=”code”>param0</code>, defaulting to 3. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Unwindowed natural bicubic spline. </dd> </dl>

</dd> <dt><samp class=”option”>sws_flags</samp></dt> <dd> Set the scaler flags. This is also used to set the scaling algorithm, though this usage is deprecated in favor of setting ‘<samp class=”samp”>scaler</samp>’. Only a single algorithm may be selected. Default value is ‘<samp class=”samp”>bicubic</samp>’. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>fast_bilinear</samp>’</dt> <dd> Select fast bilinear scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Select bilinear scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> Select bicubic scaling algorithm. (Deprecated) </dd> <dt>‘<samp

class=”samp”>experimental</samp>’</dt> <dd> Select experimental scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>neighbor</samp>’</dt> <dd> Select nearest neighbor rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>area</samp>’</dt> <dd> Select averaging area rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bicublin</samp>’</dt> <dd> Select bicubic scaling algorithm for the luma component, bilinear for chroma components. (Deprecated) </dd> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dd> Select Gaussian rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>sinc</samp>’</dt> <dd> Select sinc rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Select Lanczos rescaling algorithm. The default width (alpha) is 3 and can be changed by setting <code class=”code”>param0</code>. (Deprecated) </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Select natural bicubic spline rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>print_info</samp>’</dt> <dd> Enable printing/debug logging. </dd> <dt>‘<samp class=”samp”>accurate_rnd</samp>’</dt> <dd> Enable accurate rounding. </dd> <dt>‘<samp class=”samp”>full_chroma_int</samp>’</dt> <dd> Enable full chroma interpolation. </dd> <dt>‘<samp class=”samp”>full_chroma_inp</samp>’</dt> <dd> Select full chroma input. </dd> <dt>‘<samp class=”samp”>bitexact</samp>’</dt> <dd> Enable bitexact output. </dd> <dt>‘<samp class=”samp”>unstable</samp>’</dt> <dd> Allow the use of experimental new code. May subtly affect the output or even produce wrong results. For testing only. </dd> </dl>

</dd> <dt><samp class=”option”>srcw <var class=”var”>(API only)</var></samp></dt> <dd> Set source width. </dd> <dt><samp class=”option”>srch <var class=”var”>(API only)</var></samp></dt> <dd> Set source height. </dd> <dt><samp class=”option”>dstw <var class=”var”>(API only)</var></samp></dt> <dd> Set destination width. </dd> <dt><samp class=”option”>dsth <var class=”var”>(API only)</var></samp></dt> <dd> Set destination height. </dd> <dt><samp class=”option”>src_format <var class=”var”>(API only)</var></samp></dt> <dd> Set source pixel format (must be expressed as an integer). </dd> <dt><samp class=”option”>dst_format <var class=”var”>(API only)</var></samp></dt> <dd> Set destination pixel format (must be expressed as an integer). </dd> <dt><samp class=”option”>src_range <var class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, indicates source is full range. Default value is <code class=”code”>0</code>, which indicates source is limited range. </dd> <dt><samp class=”option”>dst_range

<var class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, enable full range for destination. Default value is <code class=”code”>0</code>, which enables limited range. </dd> <dt><samp class=”option”>gamma <var class=”var”>(boolean)</var></samp></dt> <dd> If value is set to <code class=”code”>1</code>, enable gamma correct scaling. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>param0, param1</samp></dt> <dd> Set scaling algorithm parameters. The specified values are specific of some scaling algorithms and ignored by others. The specified values are floating point number values. </dd> <dt><samp class=”option”>sws_dither</samp></dt> <dd> Set the dithering algorithm. Accepts one of the following values. Default value is ‘<samp class=”samp”>auto</samp>’. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> automatic choice </dd> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> no dithering </dd> <dt>‘<samp class=”samp”>bayer</samp>’</dt> <dd> bayer dither </dd> <dt>‘<samp class=”samp”>ed</samp>’</dt> <dd> error diffusion dither </dd> <dt>‘<samp class=”samp”>a_dither</samp>’< <dd> arithmetic dither, based using addition </dd> <dt>‘<samp class=”samp”>x_dither</samp>’</dt> <dd> arithmetic dither, based using xor (more random/less apparent patterning that a_dither). </dd> </dl>

</dd> <dt><samp class=”option”>alphablend</samp></dt> <dd> Set the alpha blending to use when the input has alpha but the output does not. Default value is ‘<samp class=”samp”>none</samp>’. <dl class=”table”> <dt>‘<samp class=”samp”>uniform_color</samp>’</dt> <dd> Blend onto a uniform background color </dd> <dt>‘<samp class=”samp”>checkerboard</samp>’</dt> <dd> Blend onto a checkerboard </dd> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No blending </dd> </dl>

</dd> <dt><samp class=”option”>sws_backends</samp></dt> <dd> Set the allowed swscale backends. This is a flags option, so multiple backends may be combined. <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection. Equal to either ‘<samp class=”samp”>stable</samp>’ or ‘<samp class=”samp”>all</samp>’ depending on whether or not the ‘<samp class=”samp”>unstable</samp>’ flag is set. This is the default value. </dd> <dt>‘<samp class=”samp”>stable</samp>’</dt> <dd> All stable backends. </dd> <dt>‘<samp class=”samp”>unstable</samp>’</dt> <dd> All unstable backends. </dd> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> All available backends. </dd> <dt>‘<samp class=”samp”>legacy</samp>’</dt> <dd> Legacy swscale code. </dd> <dt>‘<samp

class=”samp”>c</samp>’</dt> <dd> Template-based reference code. </dd> <dt>‘<samp class=”samp”>memcpy</samp>’</dt> <dd> Fast path using libc <code class=”code”>memcpy</code>. </dd> <dt>‘<samp class=”samp”>x86</samp>’</dt> <dd> x86 SIMD kernels. </dd> <dt>‘<samp class=”samp”>aarch64</samp>’</d <dd> AArch64 NEON kernels. </dd> <dt>‘<samp class=”samp”>spirv</samp>’</dt> <dd> Vulkan SPIR-V backend. </dd> </dl>

</dd> </dl>

 <h2 class=”chapter”>21 Filtering Introduction# TOC</h2>

Filtering in FFmpeg is enabled through the libavfilter library. In libavfilter, a filter can have multiple inputs and multiple outputs. To illustrate the sorts of things that are possible, we consider the following filtergraph. <pre class=”verbatim”> [main] input --> split ----------> overlay --> output | ^ |[tmp] [flip]| +-----> crop --> vflip -------+ </pre> This filtergraph splits the input stream in two streams, then sends one stream through the crop filter and the vflip filter, before merging it back with the other stream by overlaying it on top. You can use the following command to achieve this: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf "split [main][tmp]; [tmp] crop=iw:ih/2:0:0, vflip [flip]; [main][flip] overlay=0:H/2" OUTPUT </pre>

The result will be that the top half of the video is mirrored onto the bottom half of the output video. Filters in the same linear chain are separated by commas, and distinct linear chains of filters are separated by semicolons. In our example, <var class=”var”>crop,vflip</var> are in one linear chain, <var class=”var”>split</var> and <var class=”var”>overlay</var> are separately in another. The points where the linear chains join are labelled by names enclosed in square brackets. In the example, the split filter generates two outputs that are associated to the labels <var class=”var”>[main]</var> and <var class=”var”>[tmp]</var>. The stream sent to the second output of <var class=”var”>split</var>, labelled as <var class=”var”>[tmp]</var>, is processed through the <var class=”var”>crop</var> filter, which crops away the lower half part of the video, and then vertically flipped. The <var class=”var”>overlay</var> filter takes in input the first unchanged output of the split filter (which was labelled as <var class=”var”>[main]</var>), and overlay on its lower half the output generated by the <var class=”var”>crop,vflip</var> filterchain. Some filters take in input a list of parameters: they are specified after the filter name and an equal sign, and are separated from each other by a colon. There exist so-called <var class=”var”>source filters</var> that

do not have an audio/video input, and <var class="var">sink filters</var> that will not have audio/video output.

 <h2 class="chapter">22 graph2dot# TOC</h2>

The <samp class="file">graph2dot</samp> program included in the FFmpeg <samp class="file">tools</samp> directory can be used to parse a filtergraph description and issue a corresponding textual representation in the dot language. Invoke the command: <pre class="example-preformatted">graph2dot -h </pre>

to see how to use <samp class="file">graph2dot</samp>. You can then pass the dot description to the <samp class="file">dot</samp> program (from the graphviz suite of programs) and obtain a graphical representation of the filtergraph. For example the sequence of commands: <pre class="example-preformatted">echo <var class="var">GRAPH_DESCRIPTION</var> | \ tools/graph2dot -o graph.tmp && \ dot -Tpng graph.tmp -o graph.png && \ display graph.png </pre>

can be used to create and display an image representing the graph described by the <var class="var">GRAPH_DESCRIPTION</var> string. Note that this string must be a complete self-contained graph, with its inputs and outputs explicitly defined. For example if your command line is of the form: <pre class="example-preformatted">ffmpeg -i infile -vf scale=640:360 outfile </pre> your <var class="var">GRAPH_DESCRIPTION</var> string will need to be of the form: <pre class="example-preformatted">nullsrc,scale=640:360,nullsink </pre> you may also need to set the <var class="var">nullsrc</var> parameters and add a <var class="var">format</var> filter in order to simulate a specific input file.

 <h2 class="chapter">23 Filtergraph description# TOC</h2>

A filtergraph is a directed graph of connected filters. It can contain cycles, and there can be multiple links between a pair of filters. Each link has one input pad on one side connecting it to one filter from which it takes its input, and one output pad on the other side connecting it to one filter accepting its output. Each filter in a filtergraph is an instance of a filter class registered in the application, which defines the features and the number of input and output pads of the filter. A filter with no input pads is called a "source",

and a filter with no output pads is called a "sink". <h3 class=”section”>23.1 Filtergraph syntax# TOC</h3>

A filtergraph has a textual representation, which is recognized by the <samp class=”option”>-filter</samp>/<samp class=”option”>-vf</samp>/<samp class=”option”>-af</samp> and <samp class=”option”>-filter_complex</samp> options in <code class=”command”>ffmpeg</code> and <samp class=”option”>-vf</samp>/<samp class=”option”>-af</samp> in <code class=”command”>ffplay</code>, and by the <code class=”code”>avfilter_graph_parse_ptr()</co function defined in <samp class=”file”>libavfilter/avfilter.h</samp>. A filterchain consists of a sequence of connected filters, each one connected to the previous one in the sequence. A filterchain is represented by a list of ","-separated filter descriptions. A filtergraph consists of a sequence of filterchains. A sequence of filterchains is represented by a list of ";"-separated filterchain descriptions. A filter is represented by a string of the form: [<var class=”var”>in_link_1</var>]...[<var class=”var”>in_link_N</var>]<var class=”var”>filter_name</var>@<var class=”var”>id</var>=<var class=”var”>arguments</var>[<var class=”var”>out_link_1</var>]...[<var class=”var”>out_link_M</var>] <var class=”var”>filter_name</var> is the name of the filter class of which the described filter is an instance of, and has to be the name of one of the filter classes registered in the program optionally followed by "@<var class=”var”>id</var>". The name of the filter class is optionally followed by a string "=<var class=”var”>arguments</var>". <var class=”var”>arguments</var> is a string which contains the parameters used to initialize the filter instance. It may have one of two forms: <ul class=”itemize mark-bullet”> A ’:’-separated list of <var class=”var”>key=value</var> pairs.

A ’:’-separated list of <var class=”var”>value</var>. In this case, the keys are assumed to be the option names in the order they are declared. E.g. the <code class=”code”>fade</code> filter declares three options in this order – <samp class=”option”>type</samp>, <samp class=”option”>start_frame</samp> and <samp class=”option”>nb_frames</samp>. Then the parameter list <var class=”var”>in:0:30</var> means that the value <var class=”var”>in</var> is assigned to the option <samp class=”option”>type</samp>, <var class=”var”>0</var> to <samp class=”option”>start_frame</samp> and <var class=”var”>30</var> to <samp class=”option”>nb_frames</samp>.

A ’:’-separated list of mixed direct <var class=”var”>value</var> and long <var class=”var”>key=value</var> pairs. The direct <var class=”var”>value</var> must precede the <var class=”var”>key=value</var>

pairs, and follow the same constraints order of the previous point. The following <var class="var">key=value</var> pairs can be set in any preferred order.

If the option value itself is a list of items (e.g. the <code class="code">format</code> filter takes a list of pixel formats), the items in the list are usually separated by ‘<samp class="samp">|</samp>’. The list of arguments can be quoted using the character ‘<samp class="samp">'</samp>’ as initial and ending mark, and the character ‘<samp class="samp">\</samp>’ for escaping the characters within the quoted text; otherwise the argument string is considered terminated when the next special character (belonging to the set ‘<samp class="samp">[]=;,</samp>’) is encountered. A special syntax implemented in the <code class="command">ffmpeg</code> CLI tool allows loading option values from files. This is done be prepending a slash ’/’ to the option name, then the supplied value is interpreted as a path from which the actual value is loaded. E.g. <pre class="example-preformatted">ffmpeg -i <INPUT> -vf drawtext=/text=/tmp/some_text <OUTPUT> </pre> will load the text to be drawn from <samp class="file">/tmp/some_text</samp>. API users wishing to implement a similar feature should use the <code class="code">avfilter_graph_segment_*()</code> functions together with custom IO code. The name and arguments of the filter are optionally preceded and followed by a list of link labels. A link label allows one to name a link and associate it to a filter output or input pad. The preceding labels <var class="var">in_link_1</var> ... <var class="var">in_link_N</var>, are associated to the filter input pads, the following labels <var class="var">out_link_1</var> ... <var class="var">out_link_M</var>, are associated to the output pads. When two link labels with the same name are found in the filtergraph, a link between the corresponding input and output pad is created. If an output pad is not labelled, it is linked by default to the first unlabelled input pad of the next filter in the filterchain. For example in the filterchain <pre class="example-preformatted">nullsrc, split[L1], [L2]overlay, nullsink </pre> the split filter instance has two output pads, and the overlay filter instance two input pads. The first output pad of split is labelled "L1", the first input pad of overlay is labelled "L2", and the second output pad of split is linked to the second input pad of overlay, which are both unlabelled. In a filter description, if the input label of the first filter is not specified, "in" is assumed; if the output label of the last filter is not specified, "out" is assumed. In a complete filterchain all the unlabelled filter input and output pads must be connected. A filtergraph is considered valid if all the filter input and output pads of all the filterchains are connected. Leading and trailing whitespaces (space, tabs, or line feeds) separating tokens in the filtergraph specification are ignored. This means that the

filtergraph can be expressed using empty lines and spaces to improve readability. For example, the filtergraph: <pre class="example-preformatted">testsrc,split[L1],hflip[L2];[L1][L2] hstack </pre>

can be represented as: <pre class="example-preformatted">testsrc, split [L1], hflip [L2];

[L1][L2] hstack </pre>

Libavfilter will automatically insert scale filters where format conversion is required. It is possible to specify swscale flags for those automatically inserted scalers by prepending <code class="code">sws_flags=<var class="var">flags</var>;</code> to the filtergraph description. Here is a BNF description of the filtergraph syntax: <pre class="example-preformatted"><var class="var">NAME</var> ::= sequence of alphanumeric characters and '_' <var class="var">FILTER_NAME</var> ::= <var class="var">NAME</var>["@"<var class="var">NAME</var>] <var class="var">LINKLABEL</var> ::= "[" <var class="var">NAME</var> "]" <var class="var">LINKLABELS</var> ::= <var class="var">LINKLABEL</var> [<var class="var">LINKLABELS</var>] <var class="var">FILTER_ARGUMENTS</var> ::= sequence of chars (possibly quoted) <var class="var">FILTER</var> ::= [<var class="var">LINKLABELS</var>] <var class="var">FILTER_NAME</var> ["=" <var class="var">FILTER_ARGUMENTS</var>] [<var class="var">LINKLABELS</var>] <var class="var">FILTERCHAIN</var> ::= <var class="var">FILTER</var> [,<var class="var">FILTERCHAIN</var>] <var class="var">FILTERGRAPH</var> ::= [sws_flags=<var class="var">flags</var>;] <var class="var">FILTERCHAIN</var> [;<var class="var">FILTERGRAPH</var>] </pre>

 <h3 class="section">23.2 Notes on filtergraph escaping# TOC</h3>

Filtergraph description composition entails several levels of escaping. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#quoting_005fand_005fescaping">the "Quoting and escaping" section in the ffmpeg-utils(1) manual for more information about the employed escaping procedure. A first level escaping affects the content of each filter option value, which may contain the special character <code class="code">:</code> used to separate values, or one of the escaping characters <code class="code">\'</code>. A second level escaping affects the whole filter description, which may contain the escaping characters <code class="code">\'</code> or the special characters <code class="code">[],;</code> used by the filtergraph description.

Finally, when you specify a filtergraph on a shell commandline, you need to perform a third level escaping for the shell special characters contained within it. For example, consider the following string to be embedded in the drawtext filter description <samp class=”option”>text</samp> value: <pre class=”example-preformatted”>this is a 'string': may contain one, or more, special characters </pre>

This string contains the <code class=”code”>'</code> special escaping character, and the <code class=”code”>:</code> special character, so it needs to be escaped in this way: <pre class=”example-preformatted”>text=this is a \'string\'\: may contain one, or more, special characters </pre>

A second level of escaping is required when embedding the filter description in a filtergraph description, in order to escape all the filtergraph special characters. Thus the example above becomes: <pre class=”example-preformatted”>drawtext=text=this is a \\\'string\\\'\\: may contain one\, or more\, special characters </pre> (note that in addition to the <code class=”code”>\'</code> escaping special characters, also <code class=”code”>,</code> needs to be escaped). Finally an additional level of escaping is needed when writing the filtergraph description in a shell command, which depends on the escaping rules of the adopted shell. For example, assuming that <code class=”code”>\</code> is special and needs to be escaped with another <code class=”code”>\</code>, the previous string will finally result in: <pre class=”example-preformatted”>-vf "drawtext=text=this is a \\\\\\'string\\\\\\'\\\\: may contain one\\, or more\\, special characters" </pre>

In order to avoid cumbersome escaping when using a commandline tool accepting a filter specification as input, it is advisable to avoid direct inclusion of the filter or options specification in the shell. For example, in case of the drawtext filter, you might prefer to use the <samp class=”option”>textfile</samp> option in place of <samp class=”option”>text</samp> to specify the text to render. <h2 class=”chapter”>24 Timeline editing# TOC</h2>

Some filters support a generic <samp class=”option”>enable</samp> option. For the filters supporting timeline editing, this option can be set to an expression which is evaluated before sending a frame to the filter. If the evaluation is non-zero, the filter will be enabled, otherwise the frame will be sent unchanged to the next filter in the filtergraph. The expression accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>t</samp>’</dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd>

<dt>‘<samp class="samp">n</samp>’</dt> <dd> sequential number of the input frame, starting from 0 </dd> <dt>‘<samp class="samp">pos</samp>’</dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt>‘<samp class="samp">w</samp>’</dt> <dt>‘<samp class="samp">h</samp>’</dt> <dd> width and height of the input frame if video </dd> </dl>

Additionally, these filters support an <samp class="option">enable</samp> command that can be used to re-define the expression. Like any other filtering option, the <samp class="option">enable</samp> option follows the same rules. For example, to enable a blur filter (smartblur) from 10 seconds to 3 minutes, and a curves filter starting at 3 seconds: <pre class="example-preformatted">smartblur = enable='between(t,10,3*60)', curves = enable='gte(t,3)' : preset=cross_process </pre>

See <code class="code">ffmpeg -filters</code> to view which filters have timeline support.

 <h2 class="chapter">25 Changing options at runtime with a command# TOC</h2>

Some options can be changed during the operation of the filter using a command. These options are marked ’T’ on the output of <code class="command">ffmpeg</code> <samp class="option">-h filter=<name of filter></samp>. The name of the command is the name of the option and the argument is the new value. <h2 class="chapter">26 Options for filters with several inputs (framesync)# TOC</h2>

Some filters with several inputs support a common set of options. These options can only be set by name, not with the short notation. <dl class="table"> <dt><samp class="option">eof_action</samp></dt> <dd> The action to take when EOF is encountered on the secondary input; it accepts one of the following values: <dl class="table"> <dt><samp class="option">repeat</samp></dt> <dd> Repeat the last frame (the default). </dd> <dt><samp class="option">endall</samp></dt>

<dd> End both streams. </dd> <dt><samp class="option">pass</samp></dt>
<dd> Pass the main input through. </dd> </dl>

</dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">repeatlast</samp></dt>
<dd> If set to 1, force the filter to extend the last frame of secondary streams until the end of the primary stream. A value of 0 disables this behavior. Default value is 1. </dd> <dt><samp class="option">ts_sync_mode</samp></dt> <dd> How strictly to sync streams based on secondary input timestamps; it accepts one of the following values: <dl class="table"> <dt><samp class="option">default</samp></dt>
<dd> Frame from secondary input with the nearest lower or equal timestamp to the primary input frame. </dd> <dt><samp class="option">nearest</samp></dt> <dd> Frame from secondary input with the absolute nearest timestamp to the primary input frame. </dd> </dl> </dd> </dl>

 <h2 class="chapter">27 Audio Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class="code">--disable-filters</code>. The configure output will show the audio filters included in your build. Below is a description of the currently available audio filters.
<h3 class="section">27.1 aap# TOC</h3>
Apply Affine Projection algorithm to the first audio stream using the second audio stream. This adaptive filter is used to estimate unknown audio based on multiple input audio samples. Affine projection algorithm can make trade-offs between computation complexity with convergence speed.
A description of the accepted options follows. <dl class="table">
<dt><samp class="option">order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class="option">projection</samp></dt>
<dd> Set the projection order. </dd> <dt><samp class="option">mu</samp></dt>
<dd> Set the filter mu. </dd> <dt><samp class="option">delta</samp></dt>
<dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class="option">out_mode</samp></dt>
<dd> Set the filter output samples. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt>
<dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt>
<dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt>
<dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass

difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class="section">27.2 acompressor# TOC</h3>

A compressor is mainly used to reduce the dynamic range of a signal. Especially modern music is mostly compressed at a high ratio to improve the overall loudness. It’s done to get the highest attention of a listener, "fatten" the sound and bring more "power" to the track. If a signal is compressed too much it may sound dull or "dead" afterwards or it may start to "pump" (which could be a powerful effect but can also destroy a track completely). The right compression is the key to reach a professional sound and is the high art of mixing and mastering. Because of its complex settings it may take a long time to get the right feeling for this kind of effect. Compression is done by detecting the volume above a chosen level <code class="code">threshold</code> and dividing it by the factor set with <code class="code">ratio</code>. So if you set the threshold to -12dB and your signal reaches -6dB a ratio of 2:1 will result in a signal at -9dB. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over the time. This is done by setting "Attack" and "Release". <code class="code">attack</code> determines how long the signal has to rise above the threshold before any reduction will occur and <code class="code">release</code> sets the time the signal has to fall below the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. The overall reduction of the signal can be made up afterwards with the <code class="code">makeup</code> setting. So compressing the peaks of a signal about 6dB and raising the makeup to this level results in a signal twice as loud than the source. To gain a softer entry in the compression the <code class="code">knee</code> flattens the hard edge at the threshold in the range of the chosen decibels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input

gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal of stream rises above this level it will affect the gain reduction. By default it is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. 1:2 means that if the level rose 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the <code class="code">average</code> level between all channels of input stream or the louder(<code class="code">maximum</code>) channel of input stream affects the reduction. Default is <code class="code">average</code>. </dd> <dt><samp class="option">detection</samp></dt> <dd> Should the exact signal be taken in case of <code class="code">peak</code> or an RMS one in case of <code class="code">rms</code>. Default is <code class="code">rms</code> which is mostly smoother. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class="subsection">27.2.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.3 acontrast# TOC</h3> Simple audio dynamic range compression/expansion filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set contrast. Default

is 33. Allowed range is between 0 and 100. </dd> </dl>

 <h3 class="section">27.4 acopy# TOC</h3>

Copy the input audio source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">27.5 acrossfade# TOC</h3>

Apply cross fade from one input audio stream to another input audio stream. The cross fade is applied for specified duration near the end of first stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs, n</samp></dt> <dd> Specify the number of inputs to crossfade. When crossfading multiple inputs, each input will be concatenated and crossfaded in sequence, similar to the concat filter. Default is 2. </dd> <dt><samp class="option">nb_samples, ns</samp></dt> <dd> Specify the number of samples for which the cross fade effect has to last. At the end of the cross fade effect the first input audio will be completely silent. Default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the cross fade effect. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. By default the duration is determined by <var class="var">nb_samples</var>. If set this option is used instead of <var class="var">nb_samples</var>. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Should first stream end overlap with second stream start. Default is enabled. </dd> <dt><samp class="option">curve1</samp></dt> <dd> Set curve for cross fade transition for first stream. </dd> <dt><samp class="option">curve2</samp></dt> <dd> Set curve for cross fade transition for second stream. For description of available curve types see afade filter description. </dd> </dl>

 <h4 class="subsection">27.5.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input to another: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:c1=exp:c2=exp output.flac </pre>

Cross fade from one input to another but without overlapping: <div

class=”example”> <pre class=”example-preformatted”>ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:o=0:c1=exp:c2=exp output.flac </pre>

Concatenate multiple inputs with cross fade between each: <pre class=”example-preformatted”>ffmpeg -i first.flac -i second.flac -i third.flac -filter_complex acrossfade=n=3 output.flac </pre>

 <h3 class=”section”>27.6 acrossover# TOC</h3> Split audio stream into several bands. This filter splits audio stream into two or more frequency ranges. Summing all streams back will give flat output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>split</samp></dt> <dd> Set split frequencies. Those must be positive and increasing. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order for each band split. This controls filter roll-off or steepness of filter transfer function. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>2nd</samp>’</dt> <dd> 12 dB per octave. </dd> <dt>‘<samp class=”samp”>4th</samp>’</dt> <dd> 24 dB per octave. </dd> <dt>‘<samp class=”samp”>6th</samp>’</dt> <dd> 36 dB per octave. </dd> <dt>‘<samp class=”samp”>8th</samp>’</dt> <dd> 48 dB per octave. </dd> <dt>‘<samp class=”samp”>10th</samp>’</dt> <dd> 60 dB per octave. </dd> <dt>‘<samp class=”samp”>12th</samp>’</dt> <dd> 72 dB per octave. </dd> <dt>‘<samp class=”samp”>14th</samp>’</dt> <dd> 84 dB per octave. </dd> <dt>‘<samp class=”samp”>16th</samp>’</dt> <dd> 96 dB per octave. </dd> <dt>‘<samp class=”samp”>18th</samp>’</dt> <dd> 108 dB per octave. </dd> <dt>‘<samp class=”samp”>20th</samp>’</dt> <dd> 120 dB per octave. </dd> </dl>

Default is <var class=”var”>4th</var>. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class=”option”>gains</samp></dt> <dd> Set output gain for each band. Default value is 1 for all bands. </dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl>

Default value is <code class=”code”>auto</code>. </dd> </dl>

 <h4 class="subsection">27.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Split input audio stream into two bands (low and high) with split frequency of 1500 Hz, each band will be in separate stream: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but with higher filter order: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500:order=8th[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but also with additional middle band (frequencies between 1500 and 8000): <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500 8000:order=8th[LOW][MID][HIGH]' -map '[LOW]' low.wav -map '[MID]' mid.wav -map '[HIGH]' high.wav </pre>

 <h3 class="section">27.7 acrusher# TOC</h3>

Reduce audio bit resolution. This filter is bit crusher with enhanced functionality. A bit crusher is used to audibly reduce number of bits an audio signal is sampled with. This doesn’t change the bit depth at all, it just produces the effect. Material reduced in bit depth sounds more harsh and "digital". This filter is able to even round to continuous values instead of discrete bit depths. Additionally it has a D/C offset which results in different crushing of the lower and the upper half of the signal. An Anti-Aliasing setting is able to produce "softer" crushing sounds. Another feature of this filter is the logarithmic mode. This setting switches from linear distances between bits to logarithmic ones. The result is a much more "natural" sounding crusher which doesn’t gate low signals for example. The human ear has a logarithmic perception, so this kind of crushing is much more pleasant. Logarithmic crushing is also able to get anti-aliased. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set level in. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set level out. </dd> <dt><samp class="option">bits</samp></dt> <dd> Set bit reduction. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set mixing amount. </dd> <dt><samp class="option">mode</samp></dt> <dd> Can be linear: <code class="code">lin</code> or logarith-

mic: <code class="code">log</code>. </dd> <dt><samp class="option">dc</samp></dt> <dd> Set DC. </dd> <dt><samp class="option">aa</samp></dt> <dd> Set anti-aliasing. </dd> <dt><samp class="option">samples</samp></dt> <dd> Set sample reduction. </dd> <dt><samp class="option">lfo</samp></dt> <dd> Enable LFO. By default disabled. </dd> <dt><samp class="option">lforange</samp></dt> <dd> Set LFO range. </dd> <dt><samp class="option">lforate</samp></dt> <dd> Set LFO rate. </dd> </dl>

 <h4 class="subsection">27.7.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.8 acue# TOC</h3>

Delay audio filtering until a given wallclock timestamp. See the cue filter. <h3 class="section">27.9 adeclick# TOC</h3> Remove impulsive noise from input audio. Samples detected as impulsive noise are replaced by interpolated samples using autoregressive modelling. <dl class="table"> <dt><samp class="option">window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range is from <code class="code">10</code> to <code class="code">100</code>. Default value is <code class="code">55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class="code">50</code> to <code class="code">95</code>. Default value is <code class="code">75</code> percent. Setting this to a very high value increases impulsive noise removal but makes whole process much slower. </dd> <dt><samp class="option">arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class="code">0</code> to <code class="code">25</code>. Default value is <code class="code">2</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class="code">1</code> to <code class="code">100</code>. Default value is <code class="code">2</code>. This controls the strength of impulsive noise which is going to be re-

moved. The lower value, the more samples will be detected as impulsive noise. </dd> <dt><samp class="option">burst, b</samp></dt> <dd> Set burst fusion, in percentage of window size. Allowed range is <code class="code">0</code> to <code class="code">10</code>. Default value is <code class="code">2</code>. If any two samples detected as noise are spaced less than this value then any sample between those two samples will be also detected as noise. </dd> <dt><samp class="option">method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class="table"> <dt><samp class="option">add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class="option">save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class="code">a</code>. </dd> </dl>

 <h3 class="section">27.10 adeclip# TOC</h3> Remove clipped samples from input audio. Samples detected as clipped are replaced by interpolated samples using autoregressive modelling. <dl class="table"> <dt><samp class="option">window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range is from <code class="code">10</code> to <code class="code">100</code>. Default value is <code class="code">55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class="code">50</code> to <code class="code">95</code>. Default value is <code class="code">75</code> percent. </dd> <dt><samp class="option">arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class="code">0</code> to <code class="code">25</code>. Default value is <code class="code">8</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class="code">1</code> to <code class="code">100</code>. Default value is <code class="code">10</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class="option">hsize, n</samp></dt> <dd> Set size of histogram used to detect clips. Allowed range is from <code class="code">100</code> to <code class="code">9999</code>. Default value is <code class="code">1000</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class="option">method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class="table"> <dt><samp class="option">add, a</samp></dt> <dd> Select overlap-

add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class="option">save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class="code">a</code>. </dd> </dl>

 <h3 class="section">27.11 adecorrelate# TOC</h3> Apply decorrelation to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">stages</samp></dt> <dd> Set decorrelation stages of filtering. Allowed range is from 1 to 16. Default value is 6. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set random seed used for setting delay in samples across channels. </dd> </dl>

 <h3 class="section">27.12 adelay# TOC</h3>

Delay one or more audio channels. Samples in delayed channel are filled with silence. The filter accepts the following option: <dl class="table"> <dt><samp class="option">delays</samp></dt> <dd> Set list of delays in milliseconds for each channel separated by ’|’. Unused delays will be silently ignored. If number of given delays is smaller than number of channels all remaining channels will not be delayed. If you want to delay exact number of samples, append ’S’ to number. If you want instead to delay in seconds, append ’s’ to number. </dd> <dt><samp class="option">all</samp></dt> <dd> Use last set delay for all remaining channels. By default is disabled. This option if enabled changes how option <code class="code">delays</code> is interpreted. </dd> </dl>

 <h4 class="subsection">27.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Delay first channel by 1.5 seconds, the third channel by 0.5 seconds and leave the second channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=1500|0|500 </pre>

Delay second channel by 500 samples, the third channel by 700 samples and leave the first channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=0|500S|700S </pre>

Delay all channels by same number of samples:
<pre class="example-preformatted">adelay=delays=64S:all=1 </pre>

 <h3 class="section">27.13 adenorm# TOC</h3> Remedy denormals in audio by adding extremely low-level noise. This filter shall be placed before any filter that can produce denormals. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">level</samp></dt> <dd> Set level of added noise in dB. Default is <code class="code">-351</code>. Allowed range is from -451 to -90. </dd> <dt><samp class="option">type</samp></dt> <dd> Set type of added noise. <dl class="table"> <dt><samp class="option">dc</samp></dt> <dd> Add DC signal. </dd> <dt><samp class="option">ac</samp></dt> <dd> Add AC signal. </dd> <dt><samp class="option">square</samp></dt> <dd> Add square signal. </dd> <dt><samp class="option">pulse</samp></dt> <dd> Add pulse signal. </dd> </dl>

Default is <code class="code">dc</code>. </dd> </dl>

 <h4 class="subsection">27.13.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">27.14 aderivative, aintegral# TOC</h3>

Compute derivative/integral of audio stream. Applying both filters one after another produces original audio. <h3 class="section">27.15 adrc# TOC</h3>

Apply spectral dynamic range controller filter to input audio stream.
 A description of the accepted options follows. <dl class="table"> <dt><samp class="option">transfer</samp></dt> <dd> Set the transfer expression. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> current channel number </dd> <dt><samp class="option">sn</samp></dt> <dd> current sample number </dd> <dt><samp class="option">nb_channels</samp></dt>

<dd> number of channels </dd> <dt><samp class=”option”>t</samp></dt>
<dd> timestamp expressed in seconds </dd> <dt><samp
class=”option”>sr</samp></dt> <dd> sample rate </dd>
<dt><samp class=”option”>p</samp></dt> <dd> current frequency
power value, in dB </dd> <dt><samp class=”option”>f</samp></dt>
<dd> current frequency in Hz </dd> </dl>

Default value is <code class=”code”>p</code>. </dd>
<dt><samp class=”option”>attack</samp></dt> <dd> Set the
attack in milliseconds. Default is <code class=”code”>50</code> mil-
liseconds. Allowed range is from 1 to 1000 milliseconds. </dd>
<dt><samp class=”option”>release</samp></dt> <dd> Set the release
in milliseconds. Default is <code class=”code”>100</code> milliseconds.
Allowed range is from 5 to 2000 milliseconds. </dd> <dt><samp
class=”option”>channels</samp></dt> <dd> Set which channels to
filter, by default <code class=”code”>all</code> channels in audio stream
are filtered. </dd> </dl>

 <h4 class=”subsection”>27.15.1 Com-
mands<a class=”anchor hidden-xs” href=”#Commands-
3” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Commands-3” aria-hidden=”true”>TOC</h4>

This filter supports the all above options as commands.
  <h4 class=”subsection”>27.15.2 Exam-
ples<a class=”anchor hidden-xs” href=”#Examples-
21” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Examples-21” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Apply spectral compression to all
frequencies with threshold of -50 dB and 1:6 ratio:
<pre class=”example-preformatted”>adrc=transfer='if(gt(p,-50),-50+(p-(-
50))/6,p)':attack=50:release=100 </pre>

Similar to above but with 1:2 ratio and filtering only front center chan-
nel: <pre class=”example-preformatted”>adrc=transfer='if(gt(p,-
50),-50+(p-(-50))/2,p)':attack=50:release=100:channels=FC </pre>

Apply spectral noise gate to all frequencies with threshold of -85 dB
and with short attack time and short release time: <pre
class=”example-preformatted”>adrc=transfer='if(lte(p,-85),p-800,p)':attack=1:release=5
</pre>

Apply spectral expansion to all frequencies with threshold of
-10 dB and 1:2 ratio: <pre class=”example-
preformatted”>adrc=transfer='if(lt(p,-10),-10+(p-(-10))*2,p)':attack=50:release=100
</pre>

Apply limiter to max -60 dB to all frequencies, with attack of
2 ms and release of 10 ms: <pre class=”example-

preformatted”>adrc=transfer='min(p,-60)':attack=2:release=10 </pre>

 <h3 class=”section”>27.16 adynamicequalizer# TOC</h3>

Apply dynamic equalization to input audio stream.

A description of the accepted options follows.

<dl class=”table”>

<dt><samp class=”option”>threshold</samp></dt>

<dd> Set the detection threshold used to trigger equalization.

Threshold detection is using detection filter.

Default value is 0. Allowed range is from 0 to 100.

</dd>

<dt><samp class=”option”>dfrequency</samp></dt>

<dd> Set the detection frequency in Hz used for detection filter used to trigger equalization.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class=”option”>dqfactor</samp></dt>

<dd> Set the detection resonance factor for detection filter used to trigger equalization.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class=”option”>tfrequency</samp></dt>

<dd> Set the target frequency of equalization filter.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">tqfactor</samp></dt>

<dd> Set the target resonance factor for target equalization filter.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to rise above

the detection threshold before equalization starts.

Default is 20. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">release</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to fall below the

detection threshold before equalization ends.

Default is 200. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">ratio</samp></dt>

<dd> Set the ratio by which the equalization gain is raised.

Default is 1. Allowed range is between 0 and 30.

</dd>

<dt><samp class="option">makeup</samp></dt>

<dd> Set the makeup offset by which the equalization gain is raised.

Default is 0. Allowed range is between 0 and 100.

</dd>

<dt><samp class="option">range</samp></dt>

<dd> Set the max allowed cut/boost amount. Default is 50.
Allowed range is from 1 to 200.

</dd>
<dt><samp class=”option”>mode</samp></dt>
<dd> Set the mode of filter operation, can be one of the following:

<dl class=”table”>
<dt>‘<samp class=”samp”>listen</samp>’</dt>
<dd> Output only isolated detection signal.
</dd>
<dt>‘<samp class=”samp”>cutbelow</samp>’</dt>
<dd> Cut frequencies below detection threshold.
</dd>
<dt>‘<samp class=”samp”>cutabove</samp>’</dt>
<dd> Cut frequencies above detection threshold.
</dd>
<dt>‘<samp class=”samp”>boostbelow</samp>’</dt>
<dd> Boost frequencies below detection threshold.
</dd>
<dt>‘<samp class=”samp”>boostabove</samp>’</dt>
<dd> Boost frequencies above detection threshold.
</dd>
</dl>
Default mode is ‘<samp class=”samp”>cutbelow</samp>’.

</dd>
<dt><samp class=”option”>dftype</samp></dt>
<dd> Set the type of detection filter, can be one of the following:

<dl class=”table”>

<dt>‘<samp class="samp">bandpass</samp>’</dt>
<dt>‘<samp class="samp">lowpass</samp>’</dt>
<dt>‘<samp class="samp">highpass</samp>’</dt>
<dt>‘<samp class="samp">peak</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bandpass</samp>’.

</dd>
<dt><samp class="option">tftype</samp></dt>
<dd> Set the type of target filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bell</samp>’</dt>
<dt>‘<samp class="samp">lowshelf</samp>’</dt>
<dt>‘<samp class="samp">highshelf</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bell</samp>’.

</dd>
<dt><samp class="option">auto</samp></dt>
<dd> Automatically gather threshold from detection filter. By default
is ‘<samp class="samp">disabled</samp>’.
This option is useful to detect threshold in certain time frame of
input audio stream, in such case option value is changed at runtime.

Available values are:
<dl class="table">
<dt>‘<samp class="samp">disabled</samp>’</dt>
<dd> Disable using automatically gathered threshold value.
</dd>
<dt>‘<samp class="samp">off</samp>’</dt>

<dd> Stop picking threshold value.

</dd>

<dt>‘<samp class="samp">on</samp>’</dt>

<dd> Start picking threshold value.

</dd>

<dt>‘<samp class="samp">adaptive</samp>’</dt>

<dd> Adaptively pick threshold value, by calculating sliding window entropy.

</dd>

</dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class="subsection">27.16.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.17 adynamicsmooth# TOC</h3>

Apply dynamic smoothing to input audio stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sensitivity</samp></dt> <dd> Set an amount of sensitivity to frequency fluctations. Default is 2. Allowed range is from 0 to 1e+06. </dd> <dt><samp class="option">basefreq</samp></dt> <dd> Set a base frequency for smoothing. Default value is 22050. Allowed range is from 2 to 1e+06. </dd> </dl>

 <h4 class="subsection">27.17.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.18 aecho# TOC</h3>

Apply echoing to the input audio. Echoes are reflected sound and can occur naturally amongst mountains (and sometimes large buildings) when talking or shouting; digital echo effects emulate this behaviour and are often used to help fill out the sound of a single instrument or vocal. The time difference between the original signal and the reflection is the <code class="code">delay</code>, and the loudness of the reflected signal is the <code class="code">decay</code>. Multiple echoes can have different delays and decays. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain of reflected signal. Default is <code class="code">0.6</code>. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain of reflected signal. Default is <code class="code">0.3</code>. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set list of time intervals in milliseconds between original signal and reflections separated by ’|’. Allowed range for each <code class="code">delay</code> is <code class="code">(0 - 90000.0]</code>. Default is <code class="code">1000</code>. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set list of loudness of reflected signals separated by ’|’. Allowed range for each <code class="code">decay</code> is <code class="code">(0 - 1.0]</code>. Default is <code class="code">0.5</code>. </dd> </dl>

 <h4 class="subsection">27.18.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make it sound as if there are twice as many instruments as are actually playing: <pre class="example-preformatted">aecho=0.8:0.88:60:0.4 </pre>

If delay is very short, then it sounds like a (metallic) robot playing music: <pre class="example-preformatted">aecho=0.8:0.88:6:0.4 </pre>

A longer delay will sound like an open air concert in the mountains: <pre class="example-preformatted">aecho=0.8:0.9:1000:0.3 </pre>

Same as above but with one more mountain: <pre class="example-preformatted">aecho=0.8:0.9:1000|1800:0.3|0.25 </pre>

 <h3 class="section">27.19 aemphasis# TOC</h3> Audio emphasis filter creates or restores material directly taken from LPs or emphased CDs with different filter curves. E.g. to store music on vinyl the signal has to be altered by a filter first to even out the disadvantages of this recording medium. Once the material is played back the inverse filter has to be applied to restore the distortion of the frequency response. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set filter mode. For restoring material use <code class="code">reproduction</code> mode, otherwise use <code class="code">production</code> mode. Default is <code class="code">reproduction</code> mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Set filter type. Selects medium. Can be one of the following: <dl class="table"> <dt><samp class="option">col</samp></dt> <dd> select Columbia. </dd> <dt><samp class="option">emi</samp></dt> <dd> select EMI. </dd> <dt><samp class="option">bsi</samp></dt> <dd> select BSI (78RPM). </dd> <dt><samp class="option">riaa</samp></dt> <dd> select RIAA. </dd> <dt><samp class="option">cd</samp></dt> <dd> select Compact Disc (CD). </dd> <dt><samp class="option">50fm</samp></dt> <dd> select 50µs (FM). </dd> <dt><samp class="option">75fm</samp></dt> <dd> select 75µs (FM). </dd> <dt><samp class="option">50kf</samp></dt> <dd> select 50µs (FM-KF). </dd> <dt><samp class="option">75kf</samp></dt> <dd> select 75µs (FM-KF). </dd> </dl> </dd> </dl>

 <h4 class="subsection">27.19.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.20 aeval# TOC</h3>

Modify an audio signal according to the specified expressions. This filter accepts one or more expressions (one for each channel), which are evaluated and used to modify a corresponding audio signal. It accepts the following parameters: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. If the number of input channels is greater than the number of expressions, the last specified expres-

sion is used for the remaining output channels. </dd> <dt><samp class=”option”>channel_layout, c</samp></dt> <dd> Set output channel layout. If not specified, the channel layout is specified by the number of expressions. If set to ‘<samp class=”samp”>same</samp>’, it will use by default the same input channel layout. </dd> </dl>

Each expression in <var class=”var”>exprs</var> can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class=”option”>n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class=”option”>s</samp></dt> <dd> sample rate </dd> <dt><samp class=”option”>t</samp></dt> <dd> time of the evaluated sample expressed in seconds </dd> <dt><samp class=”option”>nb_in_channels</samp></dt> <dt><samp class=”option”>nb_out_channels</samp></
<dd> input and output number of channels </dd> <dt><samp class=”option”>val(CH)</samp></dt> <dd> the value of input channel with number <var class=”var”>CH</var> </dd> </dl>

Note: this filter is slow. For faster processing you should use a dedicated filter. <h4 class=”subsection”>27.20.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Half volume: <pre class=”example-preformatted”>aeval=val(ch)/2:c=same </pre>

Invert phase of the second channel: <pre class=”example-preformatted”>aeval=val(0)|-val(1) </pre>

 <h3 class=”section”>27.21 aexciter# TOC</h3>

An exciter is used to produce high sound that is not present in the original signal. This is done by creating harmonic distortions of the signal which are restricted in range and added to the original signal. An Exciter raises the upper end of an audio signal without simply raising the higher frequencies like an equalizer would do to create a more "crisp" or "brilliant" sound. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level prior processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class=”option”>amount</samp></dt>

<dd> Set the amount of harmonics added to original signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">drive</samp></dt> <dd> Set the amount of newly created harmonics. Allowed range is from 0.1 to 10. Default value is 8.5. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the octave of newly created harmonics. Allowed range is from -10 to 10. Default value is 0. </dd> <dt><samp class="option">freq</samp></dt> <dd> Set the lower frequency limit of producing harmonics in Hz. Allowed range is from 2000 to 12000 Hz. Default is 7500 Hz. </dd> <dt><samp class="option">ceil</samp></dt> <dd> Set the upper frequency limit of producing harmonics. Allowed range is from 9999 to 20000 Hz. If value is lower than 10000 Hz no limit is applied. </dd> <dt><samp class="option">listen</samp></dt> <dd> Mute the original signal and output only added harmonics. By default is disabled. </dd> </dl>

 <h4 class="subsection">27.21.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.22 afade# TOC</h3>

Apply fade-in/out effect to input audio.

A description of the accepted parameters follows.

<dl class="table">

<dt><samp class="option">type, t</samp></dt>

<dd> Specify the effect type, can be either <code class="code">in</code> for fade-in, or

<code class="code">out</code> for a fade-out effect. Default is <code class="code">in</code>.

</dd>

<dt><samp class="option">start_sample, ss</samp></dt>

<dd> Specify the number of the start sample for starting to apply the fade

effect. Default is 0.

</dd>

<dt><samp class="option">nb_samples, ns</samp></dt>

<dd> Specify the number of samples for which the fade effect has to last. At

the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence. Default is 44100.

</dd>

<dt><samp class="option">start_time, st</samp></dt>

<dd> Specify the start time of the fade effect. Default is 0.

The value must be specified as a time duration; see

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If set this option is used instead of <var class="var">start_sample</var>.

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Specify the duration of the fade effect. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

At the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence.

By default the duration is determined by <var class="var">nb_samples</var>.

If set this option is used instead of <var class="var">nb_samples</var>.

</dd>

<dt><samp class="option">curve</samp></dt>

<dd> Set curve for fade transition.

It accepts the following values:
<dl class="table">
<dt><samp class="option">tri</samp></dt>
<dd> select triangular, linear slope (default)
</dd>
<dt><samp class="option">qsin</samp></dt>
<dd> select quarter of sine wave
</dd>
<dt><samp class="option">hsin</samp></dt>
<dd> select half of sine wave
</dd>
<dt><samp class="option">esin</samp></dt>
<dd> select exponential sine wave
</dd>
<dt><samp class="option">log</samp></dt>
<dd> select logarithmic
</dd>
<dt><samp class="option">ipar</samp></dt>
<dd> select inverted parabola
</dd>
<dt><samp class="option">qua</samp></dt>
<dd> select quadratic
</dd>
<dt><samp class="option">cub</samp></dt>
<dd> select cubic
</dd>
<dt><samp class="option">squ</samp></dt>
<dd> select square root
</dd>

<dt><samp class="option">cbr</samp></dt>
<dd> select cubic root
</dd>
<dt><samp class="option">par</samp></dt>
<dd> select parabola
</dd>
<dt><samp class="option">exp</samp></dt>
<dd> select exponential
</dd>
<dt><samp class="option">iqsin</samp></dt>
<dd> select inverted quarter of sine wave
</dd>
<dt><samp class="option">ihsin</samp></dt>
<dd> select inverted half of sine wave
</dd>
<dt><samp class="option">dese</samp></dt>
<dd> select double-exponential seat
</dd>
<dt><samp class="option">desi</samp></dt>
<dd> select double-exponential sigmoid
</dd>
<dt><samp class="option">losi</samp></dt>
<dd> select logistic sigmoid
</dd>
<dt><samp class="option">sinc</samp></dt>
<dd> select sine cardinal function
</dd>
<dt><samp class="option">isinc</samp></dt>
<dd> select inverted sine cardinal function
</dd>
<dt><samp class="option">quat</samp></dt>

<dd> select quartic

</dd>

<dt><samp class="option">quatr</samp></dt>

<dd> select quartic root

</dd>

<dt><samp class="option">qsin2</samp></dt>

<dd> select squared quarter of sine wave

</dd>

<dt><samp class="option">hsin2</samp></dt>

<dd> select squared half of sine wave

</dd>

<dt><samp class="option">nofade</samp></dt>

<dd> no fade applied

</dd>

</dl>

</dd> <dt><samp class="option">silence</samp></dt> <dd> Set the initial gain for fade-in or final gain for fade-out. Default value is <code class="code">0.0</code>. </dd> <dt><samp class="option">unity</samp></dt> <dd> Set the initial gain for fade-out or final gain for fade-in. Default value is <code class="code">1.0</code>. </dd> </dl>

 <h4 class="subsection">27.22.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">27.22.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in first 15 seconds of audio: <pre class="example-preformatted">afade=t=in:ss=0:d=15 </pre>

Fade out last 25 seconds of a 900 seconds audio: <pre class="example-preformatted">afade=t=out:st=875:d=25 </pre>

 <h3 class="section">27.23 afftdn# TOC</h3> Denoise audio samples with FFT. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">noise_reduction, nr</samp></dt> <dd> Set the noise reduction in dB, allowed range is 0.01 to 97. Default value is 12 dB. </dd> <dt><samp class="option">noise_floor, nf</samp></dt> <dd> Set the noise floor in dB, allowed range is -80 to -20. Default value is -50 dB. </dd> <dt><samp class="option">noise_type, nt</samp></dt> <dd> Set the noise type. It accepts the following values: <dl class="table"> <dt><samp class="option">white, w</samp></dt> <dd> Select white noise. </dd> <dt><samp class="option">vinyl, v</samp></dt> <dd> Select vinyl noise. </dd> <dt><samp class="option">shellac, s</samp></dt> <dd> Select shellac noise. </dd> <dt><samp class="option">custom, c</samp></dt> <dd> Select custom noise, defined in <code class="code">bn</code> option. Default value is white noise. </dd> </dl>

</dd> <dt><samp class="option">band_noise, bn</samp></dt> <dd> Set custom band noise profile for every one of 15 bands. Bands are separated by ’ ’ or ’|’. </dd> <dt><samp class="option">residual_floor, rf</samp></dt> <dd> Set the residual floor in dB, allowed range is -80 to -20. Default value is -38 dB. </dd> <dt><samp class="option">track_noise, tn</samp></dt> <dd> Enable noise floor tracking. By default is disabled. With this enabled, noise floor is automatically adjusted. </dd> <dt><samp class="option">track_residual, tr</samp></dt> <dd> Enable residual tracking. By default is disabled. </dd> <dt><samp class="option">output_mode, om</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">input, i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">output, o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class="option">noise, n</samp></dt> <dd> Pass only noise. Default value is <var class="var">output</var>. </dd> </dl>

</dd> <dt><samp class="option">adaptivity, ad</samp></dt> <dd> Set the adaptivity factor, used how fast to adapt gains adjustments per each frequency bin. Value <var class="var">0</var> enables instant adaptation, while higher values react much slower. Allowed range is from <var class="var">0</var> to <var class="var">1</var>. Default value is <var class="var">0.5</var>. </dd> <dt><samp class="option">floor_offset, fo</samp></dt> <dd> Set the noise floor offset factor. This option is used to adjust offset applied to measured noise floor. It is only effective when noise floor tracking is enabled. Allowed

range is from <var class=”var”>-2.0</var> to <var class=”var”>2.0</var>. Default value is <var class=”var”>1.0</var>. </dd> <dt><samp class=”option”>noise_link, nl</samp></dt> <dd> Set the noise link used for multichannel audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Use unchanged channel’s noise floor. </dd> <dt><samp class=”option”>min</samp></dt> <dd> Use measured min noise floor of all channels. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Use measured max noise floor of all channels. </dd> <dt><samp class=”option”>average</samp></dt> <dd> Use measured average noise floor of all channels. Default value is <var class=”var”>min</var>. </dd> </dl>

</dd> <dt><samp class=”option”>band_multiplier, bm</samp></dt> <dd> Set the band multiplier factor, used how much to spread bands across frequency bins. Allowed range is from <var class=”var”>0.2</var> to <var class=”var”>5</var>. Default value is <var class=”var”>1.25</var>. </dd> <dt><samp class=”option”>sample_noise, sn</samp></dt> <dd> Toggle capturing and measurement of noise profile from input audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>start, begin</samp></dt> <dd> Start sample noise capture. </dd> <dt><samp class=”option”>stop, end</samp></dt> <dd> Stop sample noise capture and measure new noise band profile. Default value is <code class=”code”>none</code>. </dd> </dl>

</dd> <dt><samp class=”option”>gain_smooth, gs</samp></dt> <dd> Set gain smooth spatial radius, used to smooth gains applied to each frequency bin. Useful to reduce random music noise artefacts. Higher values increases smoothing of gains. Allowed range is from <code class=”code”>0</code> to <code class=”code”>50</code>. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>27.23.1 Commands# TOC</h4>

This filter supports the some above mentioned options as commands. <h4 class=”subsection”>27.23.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Reduce white noise by 10dB, and use previously measured noise floor of -40dB: <pre class=”example-preformatted”>afftdn=nr=10:nf=-40 </pre>

Reduce white noise by 10dB, also set initial noise floor to -80dB

and enable automatic tracking of noise floor so noise floor will gradually change during processing: <pre class=”example-preformatted”>afftdn=nr=10:nf=-80:tn=1 </pre>

Reduce noise by 20dB, using noise floor of -40dB and using commands to take noise profile of first 0.4 seconds of input audio: <pre class=”example-preformatted”>asendcmd=0.0 afftdn sn start,asendcmd=0.4 afftdn sn stop,afftdn=nr=20:nf=-40 </pre>

 <h3 class=”section”>27.24 afftfilt# TOC</h3> Apply arbitrary expressions to samples in frequency domain. <dl class=”table”> <dt><samp class=”option”>real</samp></dt> <dd> Set frequency domain real expression for each separate channel separated by ’|’. Default is "re". If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class=”option”>imag</samp></dt> <dd> Set frequency domain imaginary expression for each separate channel separated by ’|’. Default is "im". Each expression in <var class=”var”>real</var> and <var class=”var”>imag</var> can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>sr</samp></dt> <dd> sample rate </dd> <dt><samp class=”option”>b</samp></dt> <dd> current frequency bin number </dd> <dt><samp class=”option”>nb</samp></dt> <dd> number of available bins </dd> <dt><samp class=”option”>ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class=”option”>chs</samp></dt> <dd> number of channels </dd> <dt><samp class=”option”>pts</samp></dt> <dd> current frame pts </dd> <dt><samp class=”option”>re</samp></dt> <dd> current real part of frequency bin of current channel </dd> <dt><samp class=”option”>im</samp></dt> <dd> current imaginary part of frequency bin of current channel </dd> <dt><samp class=”option”>real(b, ch)</samp></dt> <dd> Return the value of real part of frequency bin at location (<var class=”var”>bin</var>,<var class=”var”>channel</var>) </dd> <dt><samp class=”option”>imag(b, ch)</samp></dt> <dd> Return the value of imaginary part of frequency bin at location (<var class=”var”>bin</var>,<var class=”var”>channel</var>) </dd> </dl>

</dd> <dt><samp class=”option”>win_size</samp></dt> <dd> Set window size. Allowed range is from 16 to 131072. Default is <code class=”code”>4096</code> </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann, hanning</samp>’</dt> <dt>‘<samp class="samp">hamming</samp>’</dt> <dt>‘<samp class="samp">blackman</samp>’</dt> <dt>‘<samp class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</dt> <dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’</dt> <dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’</dt> <dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</dt> <dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’</dt> <dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class="code">0.75</code>. </dd> </dl>

 <h4 class="subsection">27.24.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Leave almost only low frequencies in audio: <pre class="example-preformatted">afftfilt="real=re * (1-clip((b/nb)*b,0,1))':imag='im * (1-clip((b/nb)*b,0,1))'" </pre>

Apply robotize effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*sin(0)':imag='hypot(re,im)*cos(0)':win_size=512:overlap=
</pre>

Apply whisper effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*cos((random(0)*2-1)*2*3.14)':imag='hypot(re,im)*sin((ran 1)*2*3.14)':win_size=128:overlap=0.8" </pre>

Apply phase shift: <pre class="example-preformatted">afftfilt="real=re*cos(1)-im*sin(1):imag=re*sin(1)+im*cos(1)" </pre>

 <h3 class="section">27.25 afir# TOC</h3>

Apply an arbitrary Finite Impulse Response filter.

This filter is designed for applying long FIR filters,
up to 60 seconds long.

It can be used as component for digital crossover filters,
room equalization, cross talk cancellation, wavefield synthesis,
auralization, ambiophonics, ambisonics and spatialization.

This filter uses the streams higher than first one as FIR coefficients.
If the non-first stream holds a single channel, it will be used
for all input channels in the first stream, otherwise
the number of channels in the non-first stream must be same as
the number of channels in the first stream.

It accepts the following parameters:

<dl class="table">
<dt><samp class="option">dry</samp></dt>
<dd> Set dry gain. This sets input gain.

</dd>
<dt><samp class="option">wet</samp></dt>
<dd> Set wet gain. This sets final output gain.

</dd>
<dt><samp class="option">length</samp></dt>
<dd> Set Impulse Response filter length. Default is 1, which means whole IR is processed.

</dd>
<dt><samp class="option">gtype</samp></dt>
<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">irnorm</samp></dt>

<dd> Set norm to be applied to IR coefficients before filtering.

Allowed range is from <var class="var">-1</var> to <var class="var">2</var>.

IR coefficients are normalized with calculated vector norm set by this option.

For negative values, no norm is calculated, and IR coefficients are not modified at all.

Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">irlink</samp></dt>

<dd> For multichannel IR if this option is set to <var class="var">true</var>, all IR channels will be

normalized with maximal measured gain of all IR channels coefficients as set by <code class="code">irnorm</code> option.

When disabled, all IR coefficients in each IR channel will be normalized independently.

Default is <var class="var">true</var>.

</dd>

<dt><samp class="option">irgain</samp></dt>

<dd> Set gain to be applied to IR coefficients before filtering.

Allowed range is 0 to 1. This gain is applied after any gain applied with <var class="var">irnorm</var> option.

</dd>

<dt><samp class="option">irfmt</samp></dt>

<dd> Set format of IR stream. Can be <code class="code">mono</code> or <code class="code">input</code>.

Default is <code class="code">input</code>.

</dd>

<dt><samp class="option">maxir</samp></dt>

<dd> Set max allowed Impulse Response filter duration in seconds. Default is 30 seconds.

Allowed range is 0.1 to 60 seconds.

</dd>

<dt><samp class="option">response</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">channel</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">size</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">rate</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">minp</samp></dt>

<dd> Set minimal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">1</var> to <var class="var">65536</var>.

Lower values decreases latency at cost of higher CPU usage.

</dd>

<dt><samp class="option">maxp</samp></dt>

<dd> Set maximal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">8</var> to <var class="var">65536</var>.

Lower values may increase CPU usage.

</dd>

<dt><samp class="option">nbirs</samp></dt>

<dd> Set number of input impulse responses streams which will be switchable at runtime.

Allowed range is from <var class="var">1</var> to <var class="var">32</var>. Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">ir</samp></dt>

<dd> Set IR stream which will be used for convolution, starting from <var class="var">0</var>, should always be

lower than supplied value by <code class="code">nbirs</code> option. Default is <var class="var">0</var>.

This option can be changed at runtime via commands.

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> Set which precision to use when processing samples.

<dl class="table">

<dt><samp class="option">auto</samp></dt>

<dd> Auto pick internal sample format depending on other filters.

</dd>

<dt><samp class="option">float</samp></dt>

<dd> Always use single-floating point precision sample format.

</dd>

<dt><samp class="option">double</samp></dt>

<dd> Always use double-floating point precision sample format.

</dd>

</dl>

Default value is auto. </dd> <dt><samp class="option">irload</samp></dt>
<dd> Set when to load IR stream. Can be <code class="code">init</code>
or <code class="code">access</code>. First one load and prepares all IRs
on initialization, second one once on first access of specific IR. Default is <code
class="code">init</code>. </dd> </dl>

 <h4 class="subsection">27.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply reverb to stream using mono IR file as second input, complete command using ffmpeg: <pre class="example-preformatted">ffmpeg -i input.wav -i middle_tunnel_1way_mono.wav -lavfi afir output.wav </pre>

Apply true stereo processing given input stereo stream, and two stereo impulse responses for left and right channel, the impulse response files are files with names l_ir.wav and r_ir.wav, and setting irnorm option value: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR|c3=FR[a];amovie=l_ir.wav[LIR];amovie=r_ir.wav[RIR
</pre>

Similar to above example, but with <code class="code">irgain</code>
explicitly set to estimated value and with <code class="code">irnorm</code>
disabled: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR
5dB:irnom=-1,pan=stereo|FL<c0+c2|FR<c1+c3" </pre>

 <h3 class="section">27.26 aformat# TOC</h3>

Set output format constraints for the input audio. The framework will negotiate the most appropriate format to minimize conversions. It accepts the following parameters: <dl class="table">
<dt><samp class="option">sample_fmts, f</samp></dt> <dd> A ’|’-separated list of requested sample formats. </dd>
<dt><samp class="option">sample_rates, r</samp></dt> <dd> A ’|’-separated list of requested sample rates. </dd>
<dt><samp class="option">channel_layouts, cl</samp></dt> <dd> A ’|’-separated list of requested channel layouts. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> </dl>

If a parameter is omitted, all values are allowed. Force the output to either unsigned 8-bit or signed 16-bit stereo <pre class="example-preformatted">aformat=sample_fmts=u8|s16:channel_layouts=stereo </pre>

 <h3 class="section">27.27 afreqshift# TOC</h3> Apply frequency shift to input audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Specify frequency shift. Allowed range is -INT_MAX to INT_MAX. Default value is 0.0. </dd> <dt><samp class="option">level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">27.27.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.28 afwtdn# TOC</h3> Reduce broadband noise from input samples using Wavelets. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma, allowed range is from 0 to 1. Default value is 0. This option controls strength of denoising applied to input samples. Most useful way to set this option is via decibels, eg. -45dB. </dd> <dt><samp class="option">levels</samp></dt> <dd> Set the number of wavelet levels of decomposition. Allowed range is from 1 to 12. Default value is 10. Setting this too low make denoising performance very poor. </dd> <dt><samp class="option">wavet</samp></dt> <dd> Set wavelet type for decomposition of input frame. They are sorted by number of coefficients, from lowest to highest. More coefficients means worse filtering speed, but overall better quality. Available wavelets are: <dl class="table"> <dt>‘<samp class="samp">sym2</samp>’</dt> <dt>‘<samp class="samp">sym4</samp>’</dt <dt>‘<samp class="samp">rbior68</samp>’</dt> <dt>‘<samp

class=”samp”>deb10</samp>’</dt> <dt>‘<samp class=”samp”>sym10</samp>’</
<dt>‘<samp class=”samp”>coif5</samp>’</dt> <dt>‘<samp
class=”samp”>bl3</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>percent</samp></dt> <dd> Set percent of full denoising. Allowed range is from 0 to 100 percent. Default value is 85 percent or partial denoising. </dd> <dt><samp class=”option”>profile</samp></dt> <dd> If enabled, first input frame will be used as noise profile. If first frame samples contain non-noise performance will be very poor. </dd> <dt><samp class=”option”>adaptive</samp></dt> <dd> If enabled, input frames are analyzed for presence of noise. If noise is detected with high possibility then input frame profile will be used for processing following frames, until new noise frame is detected. </dd> <dt><samp class=”option”>samples</samp></dt> <dd> Set size of single frame in number of samples. Allowed range is from 512 to 65536. Default frame size is 8192 samples. </dd> <dt><samp class=”option”>softness</samp></dt> <dd> Set softness applied inside thresholding function. Allowed range is from 0 to 10. Default softness is 1. </dd> </dl>

 <h4 class=”subsection”>27.28.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.29 agate# TOC</h3>

A gate is mainly used to reduce lower parts of a signal. This kind of signal processing reduces disturbing noise between useful signals. Gating is done by detecting the volume below a chosen level <var class=”var”>threshold</var> and dividing it by the factor set with <var class=”var”>ratio</var>. The bottom of the noise floor is set via <var class=”var”>range</var>. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over time. This is done by setting <var class=”var”>attack</var> and <var class=”var”>release</var>. <var class=”var”>attack</var> determines how long the signal has to fall below the threshold before any reduction will occur and <var class=”var”>release</var> sets the time the signal has to rise above the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the

mode of operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. If set to <code class=”code”>upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class=”code”>downward</code> lower parts of signal will be reduced. </dd> <dt><samp class=”option”>range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class=”option”>ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class=”option”>makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class=”option”>knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class=”option”>detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is <code class=”code”>rms</code>. Can be <code class=”code”>peak</code> or <code class=”code”>rms</code>. </dd> <dt><samp class=”option”>link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is <code class=”code”>average</code>. Can be <code class=”code”>average</code> or <code class=”code”>maximum</code>. </dd> </dl>

 <h4 class=”subsection”>27.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.30 aiir# TOC</h3>

Apply an arbitrary Infinite Impulse Response filter. It

accepts the following parameters: <dl class="table"> <dt><samp class="option">zeros, z</samp></dt> <dd> Set B/numerator/zeros/reflection coefficients. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set A/denominator/poles/ladder coefficients. </dd> <dt><samp class="option">gains, k</samp></dt> <dd> Set channels gains. </dd> <dt><samp class="option">dry_gain</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">wet_gain</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">format, f</samp></dt> <dd> Set coefficients format. <dl class="table"> <dt>‘<samp class="samp">ll</samp>’</dt> <dd> lattice-ladder function </dd> <dt>‘<samp class="samp">sf</samp>’</dt> <dd> analog transfer function </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> digital transfer function </dd> <dt>‘<samp class="samp">zp</samp>’</dt> <dd> Z-plane zeros/poles, cartesian (default) </dd> <dt>‘<samp class="samp">pr</samp>’</dt> <dd> Z-plane zeros/poles, polar radians </dd> <dt>‘<samp class="samp">pd</samp>’</dt> <dd> Z-plane zeros/poles, polar degrees </dd> <dt>‘<samp class="samp">sp</samp>’</dt> <dd> S-plane zeros/poles </dd> </dl>

</dd> <dt><samp class="option">process, r</samp></dt> <dd> Set type of processing. <dl class="table"> <dt>‘<samp class="samp">d</samp>’</dt> <dd> direct processing </dd> <dt>‘<samp class="samp">s</samp>’</dt> <dd> serial processing </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> parallel processing </dd> </dl>

</dd> <dt><samp class="option">precision, e</samp></dt> <dd> Set filtering precision. <dl class="table"> <dt>‘<samp class="samp">dbl</samp>’</dt> <dd> double-precision floating-point (default) </dd> <dt>‘<samp class="samp">flt</samp>’</dt> <dd> single-precision floating-point </dd> <dt>‘<samp class="samp">i32</samp>’</dt> <dd> 32-bit integers </dd> <dt>‘<samp class="samp">i16</samp>’</dt> <dd> 16-bit integers </dd> </dl>

</dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize filter coefficients, by default is enabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">response</samp></dt> <dd> Show IR frequency response, magnitude(magenta), phase(green) and group delay(yellow) in additional video stream. By default it is disabled. </dd> <dt><samp class="option">channel</samp></dt> <dd> Set for which IR channel to display frequency response. By default is first channel displayed. This option is used only when <var class="var">response</var> is enabled. </dd> <dt><samp class="option">size</samp></dt>

<dd> Set video stream size. This option is used only when <var class=”var”>response</var> is enabled. </dd> </dl>

Coefficients in <code class=”code”>tf</code> and <code class=”code”>sf</code> format are separated by spaces and are in ascending order. Coefficients in <code class=”code”>zp</code> format are separated by spaces and order of coefficients doesn’t matter. Coefficients in <code class=”code”>zp</code> format are complex numbers with <var class=”var”>i</var> imaginary unit. Different coefficients and gains can be provided for every channel, in such case use ’|’ to separate coefficients or gains. Last provided coefficients will be used for all remaining channels. <h4 class=”subsection”>27.30.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply 2 pole elliptic notch at around 5000Hz for 48000 Hz sample rate: <pre class=”example-preformatted”>aiir=k=1:z=7.957584807809675810E-1 -2.575128568908332300 3.674839853930788710 -2.57512875289799137 7.957586296317130880E-1:p=1 -2.86950072432325953 3.63022088054647218 -2.28075678147272232 6.361362326477423500E-1:f=tf:r=d </pre>

Same as above but in <code class=”code”>zp</code> format: <pre class=”example-preformatted”>aiir=k=0.7957584807809675 6:z=0.80918701+0.58773 0.80918701-0.58773007i 0.80884700+0.58784055i 0.80884700-0.58784055i:p=0.63892345+0.59951235i 0.63892345-0.59951235i 0.79582691+0.44198673i 0.79582691-0.44198673i:f=zp:r=s </pre>

Apply 3-rd order analog normalized Butterworth low-pass filter, using analog transfer function format: <pre class=”example-preformatted”>aiir=z=1.3057 0 0 0:p=1.3057 2.3892 2.1860 1:f=sf:r=d </pre>

 <h3 class=”section”>27.31 alimiter# TOC</h3>

The limiter prevents an input signal from rising over a desired threshold. This limiter uses lookahead technology to prevent your signal from distorting. It means that there is a small delay after the signal is processed. Keep in mind that the delay it produces is the attack time you set. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Don’t let signals above this level pass the limiter. Default is 1. </dd> <dt><samp

class=”option”>attack</samp></dt> <dd> The limiter will reach its attenuation level in this amount of time in milliseconds. Default is 5 milliseconds. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Come back from limiting to attenuation 1.0 in this amount of milliseconds. Default is 50 milliseconds. </dd> <dt><samp class=”option”>asc</samp></dt> <dd> When gain reduction is always needed ASC takes care of releasing to an average reduction level rather than reaching a reduction of 0 in the release time. </dd> <dt><samp class=”option”>asc_level</samp></dt> <dd> Select how much the release time is affected by ASC, 0 means nearly no changes in release time while 1 produces higher release times. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Auto level output signal. Default is enabled. This normalizes audio back to 0dB if enabled. </dd> <dt><samp class=”option”>latency</samp></dt> <dd> Compensate the delay introduced by using the lookahead buffer set with attack parameter. Also flush the valid audio data in the lookahead buffer when the stream hits EOF. </dd> </dl>

Depending on picked setting it is recommended to upsample input 2x or 4x times with aresample before applying this filter. <h3 class=”section”>27.32 allpass# TOC</h3>

Apply a two-pole all-pass filter with central frequency (in Hz) <var class=”var”>frequency</var>, and filter-width <var class=”var”>width</var>. An all-pass filter changes the audio’s frequency to phase relationship without changing its frequency to amplitude relationship. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt>

<dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">order, o</samp></dt> <dd> Set the filter order, can be 1 or 2. Default is 2. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl> </dd> </dl>

 <h4 class="subsection">27.32.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change allpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change allpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change allpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change allpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.33 aloop# TOC</h3>

Loop audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal number of samples. Default is 0. </dd> <dt><samp class="option">start</samp></dt> <dd> Set first sample of loop. Default is 0. </dd> <dt><samp class="option">time</samp></dt>

<dd> Set the time of loop start in seconds. Only used if option named <var class="var">start</var> is set to <code class="code">-1</code>. </dd> </dl>

 <h3 class="section">27.34 amerge# TOC</h3>

Merge two or more audio streams into a single multi-channel stream. All inputs must have the same sample rate, and format. If inputs do not have the same duration, the output will stop with the shortest. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set the number of inputs. Default is 2. </dd> <dt><samp class="option">layout_mode</samp></dt> <dd> This option controls how the output channel layout is determined and if the audio channels are reordered during merge. <dl class="table"> <dt><samp class="option">legacy</samp></dt> <dd> This is the mode how the filter behaved historically so it is the default. If the channel layouts of the inputs are known and disjoint, and therefore compatible, the channel layout of the output will be set accordingly and the channels will be reordered as necessary. If the channel layouts of the inputs are not disjoint, some of them are unknown, or they are using special channel layouts, such as ambisonics, the output will have all the channels of the first input then all the channels of the second input, in that order, and the channel layout of the output will be the default value corresponding to the total number of channels. For example, if the first input is in 2.1 (FL+FR+LF) and the second input is FC+BL+BR, then the output will be in 5.1, with the channels in the following order: a1, a2, b1, a3, b2, b3 (a1 is the first channel of the first input, b1 is the first channel of the second input). On the other hand, if both input are in stereo, the output channels will be in the default order: a1, a2, b1, b2, and the channel layout will be arbitrarily set to 4.0, which may or may not be the expected value. </dd> <dt><samp class="option">reset</samp></dt> <dd> This mode ignores the input channel layouts and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. The output channel layout will only specify the total channel count. </dd> <dt><samp class="option">normal</samp></dt> <dd> This mode keeps channel name and designation information from the input channels and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">27.34.1 Examples<a class="anchor hidden-xs" href="#Examples-

29” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Merge two mono files into a stereo stream: <pre class=”example-preformatted”>amovie=left.wav [l] ; amovie=right.mp3 [r] ; [l] [r] amerge </pre>

Multiple merges assuming 1 video stream and 6 audio streams in <samp class=”file”>input.mkv</samp>: <pre class=”example-preformatted”>ffmpeg -i input.mkv -filter_complex "[0:1][0:2][0:3][0:4][0:5][0:6] amerge=inputs=6" -c:a pcm_s16le output.mkv </pre>

 <h3 class=”section”>27.35 amix# TOC</h3>

Mixes multiple audio inputs into a single output. Note that this filter only supports float samples (the <var class=”var”>amerge</var> and <var class=”var”>pan</var> audio filters support many formats). If the <var class=”var”>amix</var> input has integer samples then aresample will be automatically inserted to perform the conversion to float samples. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> How to determine the end-of-stream. <dl class=”table”> <dt><samp class=”option”>longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class=”option”>first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> <dt><samp class=”option”>dropout_transition</samp></dt> <dd> The transition time, in seconds, for volume renormalization when an input stream ends. The default value is 2 seconds. </dd> <dt><samp class=”option”>weights</samp></dt> <dd> Specify weight of each input audio stream as a sequence of numbers separated by a space. If fewer weights are specified compared to number of inputs, the last weight is assigned to the remaining inputs. Default weight for each input is 1. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Always scale inputs instead of only doing summation of samples. Beware of heavy clipping if inputs are not normalized prior or after filtering by this filter if this option is disabled. By default is enabled. </dd> </dl>

 <h4 class=”subsection”>27.35.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> This will mix 3 input audio streams to a single output with the same duration as the first input and a dropout transition time of 3 seconds: <pre class="example-preformatted">ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex amix=inputs=3:duration=first:dropout_transition=3 OUTPUT </pre>

This will mix one vocal and one music input audio stream to a single output with the same duration as the longest input. The music will have quarter the weight as the vocals, and the inputs are not normalized: <pre class="example-preformatted">ffmpeg -i VOCALS -i MUSIC -filter_complex amix=inputs=2:duration=longest:dropout_transition=0:weights="1 0.25":normalize=0 OUTPUT </pre>

 <h4 class="subsection">27.35.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">normalize</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">27.36 amultiply# TOC</h3>

Multiply first audio stream with second audio stream and store result in output audio stream. Multiplication is done by multiplying each sample from first stream with sample at same position from second stream. With this element-wise multiplication one can create amplitude fades and amplitude modulations. <h3 class="section">27.37 anequalizer# TOC</h3>

High-order parametric multiband equalizer for each channel. It accepts the following parameters: <dl class="table"> <dt><samp class="option">params</samp></dt> <dd> This option string is in format: "c<var class="var">chn</var> f=<var class="var">cf</var> w=<var class="var">w</var> g=<var class="var">g</var> t=<var class="var">f</var> | ..." Each equalizer band is separated by ’|’. <dl class="table"> <dt><samp class="option">chn</samp></dt> <dd> Set channel number to which equalization will be applied. If input doesn’t have that channel the entry is ignored. </dd>

<dt><samp class=”option”>f</samp></dt> <dd> Set central frequency for band. If input doesn’t have that frequency the entry is ignored. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set band width in Hertz. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set band gain in dB. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set filter type for band, optional, can be: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Butterworth, this is default. </dd> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Chebyshev type 1. </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> Chebyshev type 2. </dd> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>curves</samp></dt> <dd> With this option activated frequency response of anequalizer is displayed in video stream. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set video stream size. Only useful if curves option is activated. </dd> <dt><samp class=”option”>mgain</samp></dt> <dd> Set max gain that will be displayed. Only useful if curves option is activated. Setting this to a reasonable value makes it possible to display gain which is derived from neighbour bands which are too close to each other and thus produce higher gain when both are activated. </dd> <dt><samp class=”option”>fscale</samp></dt> <dd> Set frequency scale used to draw frequency response in video output. Can be linear or logarithmic. Default is logarithmic. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set color for each channel curve which is going to be displayed in video stream. This is list of color names separated by space or by ’|’. Unrecognised or missing colors will be replaced by white color. </dd> </dl>

 <h4 class=”subsection”>27.37.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Lower gain by 10 of central frequency 200Hz and width 100 Hz for first 2 channels using Chebyshev type 1 filter: <pre class=”example-preformatted”>anequalizer=c0 f=200 w=100 g=-10 t=1|c1 f=200 w=100 g=-10 t=1 </pre>

 <h4 class=”subsection”>27.37.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>change</samp></dt> <dd> Alter existing filter parameters. Syntax for the commands is : "<var class=”var”>fN</var>|f=<var class=”var”>freq</var>|w=<var class=”var”>width</var>|g=<var class=”var”>gain</var>" <var class=”var”>fN</var> is

existing filter number, starting from 0, if no such filter is available error is returned. <var class=”var”>freq</var> set new frequency parameter. <var class=”var”>width</var> set new width parameter in Hertz. <var class=”var”>gain</var> set new gain parameter in dB. Full filter invocation with asendcmd may look like this: asendcmd=c=’4.0 anequalizer change 0|f=200|w=50|g=1’,anequalizer=... </dd> </dl>

 <h3 class=”section”>27.38 anlmdn# TOC</h3>

Reduce broadband noise in audio samples using Non-Local Means algorithm. Each sample is adjusted by looking for other samples with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class=”option”>p</samp>. Patches are searched in an area of <samp class=”option”>r</samp> around the sample. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>strength, s</samp></dt> <dd> Set denoising strength. Allowed range is from 0.00001 to 10000. Default value is 0.00001. </dd> <dt><samp class=”option”>patch, p</samp></dt> <dd> Set patch radius duration. Allowed range is from 1 to 100 milliseconds. Default value is 2 milliseconds. </dd> <dt><samp class=”option”>research, r</samp></dt> <dd> Set research radius duration. Allowed range is from 2 to 300 milliseconds. Default value is 6 milliseconds. </dd> <dt><samp class=”option”>output, o</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass only noise. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>smooth, m</samp></dt> <dd> Set smooth factor. Default value is <var class=”var”>11</var>. Allowed range is from <var class=”var”>1</var> to <var class=”var”>1000</var>. </dd> </dl>

 <h4 class=”subsection”>27.38.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.39 anlmf, anlms<a class=”anchor hidden-xs”

href=”#anlmf_002c-anlms” aria-hidden=”true”># TOC</h3> Apply Normalized Least-Mean-(Squares|Fourth) algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by finding the filter coefficients that relate to producing the least mean square of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order. </dd> <dt><samp class=”option”>mu</samp></dt> <dd> Set filter mu. </dd> <dt><samp class=”option”>eps</samp></dt> <dd> Set the filter eps. </dd> <dt><samp class=”option”>leakage</samp></dt> <dd> Set the filter leakage. </dd> <dt><samp class=”option”>out_mode</samp></dt> <dd> It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class=”option”>e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>27.39.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> One of many usages of this filter is noise reduction, input audio is filtered with same samples that are delayed by fixed amount, one such example for stereo audio is: <pre class=”example-preformatted”>asplit[a][b],[a]adelay=32S|32S[a],[b][a]anlms=order=128:leakage=0.0005:mu=.
</pre>

 <h4 class=”subsection”>27.39.2 Commands# TOC</h4>

This filter supports the same commands as options, excluding option <code class="code">order</code>. <h3 class="section">27.40 anull# TOC</h3>

Pass the audio source unchanged to the output. <h3 class="section">27.41 apad# TOC</h3>

Pad the end of an audio stream with silence. This can be used together with <code class="command">ffmpeg</code> <samp class="option">-shortest</samp> to extend audio streams to the same length as the video stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">packet_size</samp></dt> <dd> Set silence packet size. Default value is 4096. </dd> <dt><samp class="option">pad_len</samp></dt> <dd> Set the number of samples of silence to add to the end. After the value is reached, the stream is terminated. This option is mutually exclusive with <samp class="option">whole_len</samp>. </dd> <dt><samp class="option">whole_len</samp></dt> <dd> Set the minimum total number of samples in the output audio stream. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_len</samp>. </dd> <dt><samp class="option">pad_dur</samp></dt> <dd> Specify the duration of samples of silence to add. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. </dd> <dt><samp class="option">whole_dur</samp></dt> <dd> Specify the minimum total duration in the output audio stream. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_dur</samp> </dd> </dl>

If neither the <samp class="option">pad_len</samp> nor the <samp class="option">whole_len</samp> nor <samp class="option">pad_dur</samp> nor <samp class="option">whole_dur</samp> option is set, the filter will add silence to the end of the input stream indefinitely. Note that for ffmpeg 4.4 and earlier a zero <samp class="option">pad_dur</samp> or <samp class="option">whole_dur</samp> also caused the filter to add silence indefinitely. <h4 class="subsection">27.41.1 Examples#

TOC</h4>

<ul class=”itemize mark-bullet”> Add 1024 samples of silence to the end of the input: <pre class=”example-preformatted”>apad=pad_len=1024 </pre>

Make sure the audio output will contain at least 10000 samples, pad the input with silence if required: <pre class=”example-preformatted”>apad=whole_len=10000 </pre>

Use <code class=”command”>ffmpeg</code> to pad the audio input with silence, so that the video stream will always result the shortest and will be converted until the end in the output file when using the <samp class=”option”>shortest</samp> option: <pre class=”example-preformatted”>ffmpeg -i VIDEO -i AUDIO -filter_complex "[1:0]apad" -shortest OUTPUT </pre>

 <h3 class=”section”>27.42 aphaser# TOC</h3> Add a phasing effect to the input audio. A phaser filter creates series of peaks and troughs in the frequency spectrum. The position of the peaks and troughs are modulated so that they vary over time, creating a sweeping effect. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class=”option”>out_gain</samp></dt> <dd> Set output gain. Default is 0.74 </dd> <dt><samp class=”option”>delay</samp></dt> <dd> Set delay in milliseconds. Default is 3.0. </dd> <dt><samp class=”option”>decay</samp></dt> <dd> Set decay. Default is 0.4. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set modulation speed in Hz. Default is 0.5. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set modulation type. Default is triangular. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>triangular, t</samp>’</dt> <dt>‘<samp class=”samp”>sinusoidal, s</samp>’</dt> </dl> </dd> </dl>

 <h3 class=”section”>27.43 aphaseshift# TOC</h3> Apply phase shift to input audio samples. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>shift</samp></dt> <dd> Specify phase shift. Allowed range is from -1.0 to 1.0. Default value is 0.0. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set output gain applied to final output. Allowed range is

from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">27.43.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.44 apsnr# TOC</h3> Measure Audio Peak Signal-to-Noise Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">27.45 apsyclip# TOC</h3> Apply Psychoacoustic clipper to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">clip</samp></dt> <dd> Set the clipping start value. Default value is 0dBFS or 1. </dd> <dt><samp class="option">diff</samp></dt> <dd> Output only difference samples, useful to hear introduced distortions. By default is disabled. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> Set strength of adaptive distortion applied. Default value is 0.5. Allowed range is from 0 to 1. </dd> <dt><samp class="option">iterations</samp></dt> <dd> Set number of iterations of psychoacoustic clipper. Allowed range is from 1 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Auto level output signal. Default is disabled. This normalizes audio back to 0dBFS if enabled. </dd> </dl>

 <h4 class="subsection">27.45.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.46 apulsator# <a class="anchor hidden-xs"href="#toc-

apulsator" aria-hidden="true">TOC</h3>

Audio pulsator is something between an autopanner and a tremolo. But it can produce funny stereo effects as well. Pulsator changes the volume of the left and right channel based on a LFO (low frequency oscillator) with different waveforms and shifted phases. This filter have the ability to define an offset between left and right channel. An offset of 0 means that both LFO shapes match each other. The left and right channel are altered equally - a conventional tremolo. An offset of 50% means that the shape of the right channel is exactly shifted in phase (or moved backwards about half of the frequency) - pulsator acts as an autopanner. At 1 both curves match again. Every setting in between moves the phase shift gapless between all stages and produces some "bypassing" sounds with sine and triangle waveforms. The more you set the offset near 1 (starting from the 0.5) the faster the signal passes from the left to the right speaker. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set waveform shape the LFO will use. Can be one of: sine, triangle, square, sawup or sawdown. Default is sine. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set modulation. Define how much of original signal is affected by the LFO. </dd> <dt><samp class="option">offset_l</samp></dt> <dd> Set left channel offset. Default is 0. Allowed range is [0 - 1]. </dd> <dt><samp class="option">offset_r</samp></dt> <dd> Set right channel offset. Default is 0.5. Allowed range is [0 - 1]. </dd> <dt><samp class="option">width</samp></dt> <dd> Set pulse width. Default is 1. Allowed range is [0 - 2]. </dd> <dt><samp class="option">timing</samp></dt> <dd> Set possible timing mode. Can be one of: bpm, ms or hz. Default is hz. </dd> <dt><samp class="option">bpm</samp></dt> <dd> Set bpm. Default is 120. Allowed range is [30 - 300]. Only used if timing is set to bpm. </dd> <dt><samp class="option">ms</samp></dt> <dd> Set ms. Default is 500. Allowed range is [10 - 2000]. Only used if timing is set to ms. </dd> <dt><samp class="option">hz</samp></dt> <dd> Set frequency in Hz. Default is 2. Allowed range is [0.01 - 100]. Only used if timing is set to hz. </dd> </dl>

 <h3 class="section">27.47 aresample# TOC</h3>

Resample the input audio to the specified parameters, using the libswresample library. If none are specified then the filter will auto-

matically convert between its input and output. This filter is also able to stretch/squeeze the audio data to make it match the timestamps or to inject silence / cut out audio to make it match the timestamps, do a combination of both or do neither. The filter accepts the syntax [<var class="var">sample_rate</var>:]<var class="var">resampler_options</var>, where <var class="var">sample_rate</var> expresses a sample rate and <var class="var">resampler_options</var> is a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ":". See the <a data-manual="ffmpeg-resampler" href="./ffmpeg-resampler.html#Resampler-Options">"Resampler Options" section in the ffmpeg-resampler(1) manual for the complete list of supported options. <h4 class="subsection">27.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Resample the input audio to 44100Hz: <pre class="example-preformatted">aresample=44100 </pre>

Stretch/squeeze samples to the given timestamps, with a maximum of 1000 samples per second compensation: <pre class="example-preformatted">aresample=async=1000 </pre>

 <h3 class="section">27.48 areverse# TOC</h3>

Reverse an audio clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">27.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">atrim=end=5,areverse </pre>

 <h3 class="section">27.49 arls# TOC</h3>
Apply Recursive Least Squares algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by recursively finding the filter coefficients that relate to producing the minimal weighted linear least squares cost function of the error signal (difference between the desired, 2nd input audio stream and

the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class="option">lambda</samp></dt> <dd> Set the forgetting factor. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class="section">27.50 arnndn# TOC</h3>

Reduce noise from speech using Recurrent Neural Networks. This filter accepts the following options: <dl class="table"> <dt><samp class="option">model, m</samp></dt> <dd> Set train model file to load. This option is always required. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how much to mix filtered samples into final output. Allowed range is from -1 to 1. Default value is 1. Negative values are special, they set how much to keep filtered noise in the final filter output. Set this option to -1 to hear actual noise removed from input signal. </dd> </dl>

 <h4 class="subsection">27.50.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h3 class="section">27.51 asdr# TOC</h3> Measure Audio Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">27.52 asetnsamples# TOC</h3>

Set the number of samples per each output audio frame. The last output packet may contain a different number of samples, as the filter will flush all the remaining samples when the input audio signals its end. The filter accepts the following options: <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of frames per each output audio frame. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio frame with zeroes, so that the last frame will contain the same number of samples as the previous ones. Default value is 1. </dd> </dl>

For example, to set the number of per-frame samples to 1234 and disable padding for the last frame, use: <pre class="example-preformatted">asetnsamples=n=1234:p=0 </pre>

 <h3 class="section">27.53 asetrate# TOC</h3>

Set the sample rate without altering the PCM data. This will result in a change of speed and pitch. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set the output sample rate. Default is 44100 Hz. </dd> </dl>

 <h3 class="section">27.54 ashowinfo# TOC</h3>

Show a line containing various information for each input audio frame. The input audio is not modified. The shown line contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var>. The following values are shown in the output: <dl class="table"> <dt><samp class="option">n</samp></dt>

<dd> The (sequential) number of the input frame, starting from 0.
</dd> <dt><samp class="option">pts</samp></dt> <dd> The
presentation timestamp of the input frame, in time base units; the time base depends on the filter input pad, and is usually 1/<var class="var">sample_rate</var>.
 </dd> <dt><samp class="option">pts_time</samp></dt>
<dd> The presentation timestamp of the input frame in seconds.
</dd> <dt><samp class="option">fmt</samp></dt> <dd> The sample format. </dd> <dt><samp class="option">chlayout</samp></dt>
<dd> The channel layout. </dd> <dt><samp class="option">rate</samp></dt>
<dd> The sample rate for the audio frame. </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> The number of samples (per channel) in the frame. </dd> <dt><samp class="option">checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of the audio data. For planar audio, the data is treated as if all the planes were concatenated. </dd> <dt><samp class="option">plane_checksums</samp></dt> <dd> A list of Adler-32 checksums for each data plane. </dd> </dl>

 <h3 class="section">27.55 asisdr#
TOC</h3>
Measure Audio Scaled-Invariant Signal-to-Distortion Ratio.
This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input.
 <h3 class="section">27.56 asoftclip# TOC</h3> Apply audio soft clipping.
 Soft clipping is a type of distortion effect where the amplitude of a signal is saturated along a smooth curve, rather than the abrupt shape of hard-clipping. This filter accepts the following options:
 <dl class="table"> <dt><samp class="option">type</samp></dt>
<dd> Set type of soft-clipping. It accepts the following values:
<dl class="table"> <dt><samp class="option">hard</samp></dt>
<dt><samp class="option">tanh</samp></dt> <dt><samp class="option">atan</samp></dt>
<dt><samp class="option">cubic</samp></dt> <dt><samp class="option">exp</samp></dt>
<dt><samp class="option">alg</samp></dt> <dt><samp class="option">quintic</samp></dt>
<dt><samp class="option">sin</samp></dt> <dt><samp class="option">erf</samp></dt>
</dl>

</dd> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold from where to start clipping. Default value is 0dB or 1.
</dd> <dt><samp class="option">output</samp></dt> <dd> Set gain applied to output. Default value is 0dB or 1. </dd>
<dt><samp class="option">param</samp></dt> <dd> Set additional parameter which controls sigmoid function. </dd> <dt><samp class="option">oversample</samp></dt> <dd> Set oversampling

factor. </dd> </dl>

 <h4 class="subsection">27.56.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.57 aspectralstats# TOC</h3>

Display frequency domain statistical information about the audio channels. Statistics are calculated and stored as metadata for each audio channel and for each audio frame. It accepts the following option: <dl class="table"> <dt><samp class="option">win_size</samp></dt> <dd> Set the window length in samples. Default value is 2048. Allowed range is from 32 to 65536. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann, hanning</samp>’</dt> <dt>‘<samp class="samp">hamming</samp>’</dt> <dt>‘<samp class="samp">blackman</samp>’</dt> <dt>‘<samp class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</ <dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’< <dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’< <dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d <dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’< <dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set window overlap. Allowed range is from <code class="code">0</code> to <code class="code">1</code>. Default value is <code class="code">0.5</code>. </dd> <dt><samp class="option">measure</samp></dt> <dd> Select the parameters which are measured. The metadata keys can be used as flags, default is <samp class="option">all</samp> which measures everything. <samp class="option">none</samp> disables all measurement. </dd> </dl>

A list of each metadata key follows: <dl class="table"> <dt><samp class="option">mean</samp></dt> <dt><samp class="option">variance</samp></dt> <dt><samp class="option">centroid</samp></dt> <dt><samp class="option">spread</samp></dt>

<dt><samp class="option">skewness</samp></dt> <dt><samp class="option">kurtosis</samp></dt>
<dt><samp class="option">entropy</samp></dt> <dt><samp class="option">flatness</samp></dt>
<dt><samp class="option">crest</samp></dt> <dt><samp class="option">flux</samp></dt>
<dt><samp class="option">slope</samp></dt> <dt><samp class="option">decrease</samp></dt>
<dt><samp class="option">rolloff</samp></dt> </dl>

 <h3 class="section">27.58 asr#
TOC</h3>
Automatic Speech Recognition This filter uses PocketSphinx for speech recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-pocketsphinx</code>. It accepts the following options: <dl class="table">
<dt><samp class="option">rate</samp></dt> <dd> Set sampling rate of input audio. Defaults is <code class="code">16000</code>. This need to match speech models, otherwise one will get poor results.
</dd> <dt><samp class="option">hmm</samp></dt> <dd> Set dictionary containing acoustic model files. </dd> <dt><samp class="option">dict</samp></dt> <dd> Set pronunciation dictionary. </dd> <dt><samp class="option">lm</samp></dt> <dd> Set language model file. </dd> <dt><samp class="option">lmctl</samp></dt>
<dd> Set language model set. </dd> <dt><samp class="option">lmname</samp></dt>
<dd> Set which language model to use. </dd> <dt><samp class="option">logfn</samp></dt> <dd> Set output for log messages. </dd> </dl>

The filter exports recognized speech as the frame metadata <code class="code">lavfi.asr.text</code>. <h3 class="section">27.59 astats#
TOC</h3>

Display time domain statistical information about the audio channels. Statistics are calculated and displayed for each audio channel and, where applicable, an overall figure is also given. It accepts the following option: <dl class="table"> <dt><samp class="option">length</samp></dt>
<dd> Short window length in seconds, used for peak and through RMS measurement. Default is <code class="code">0.05</code> (50 milliseconds). Allowed range is <code class="code">[0 - 10]</code>. </dd> <dt><samp class="option">metadata</samp></dt>
<dd> Set metadata injection. All the metadata keys are prefixed with <code class="code">lavfi.astats.X</code>, where <code class="code">X</code> is channel number starting from 1 or string <code class="code">Overall</code>. Default is disabled. Available keys for each channel are: <var class="var">Bit_depth</var> <var class="var">Crest_factor</var> <var class="var">DC_offset</var> <var class="var">Dynamic_range</var> <var class="var">Entropy</var> <var

class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var>
<var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var>
<var class=”var”>Min_difference</var> <var class=”var”>Min_level</var>
<var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var>
<var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var>
<var class=”var”>Number_of_denormals</var> <var class=”var”>Peak_count</var>
<var class=”var”>Abs_Peak_count</var> <var class=”var”>Peak_level</var>
<var class=”var”>RMS_difference</var> <var class=”var”>RMS_peak</var>
<var class=”var”>RMS_trough</var> <var class=”var”>Zero_crossings</var>
<var class=”var”>Zero_crossings_rate</var> and for <code
class=”code”>Overall</code>: <var class=”var”>Bit_depth</var> <var
class=”var”>DC_offset</var> <var class=”var”>Entropy</var> <var
class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var>
<var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var>
<var class=”var”>Min_difference</var> <var class=”var”>Min_level</var>
<var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var>
<var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var>
<var class=”var”>Number_of_denormals</var> <var class=”var”>Number_of_samples</var>
<var class=”var”>Peak_count</var> <var class=”var”>Abs_Peak_count</var>
<var class=”var”>Peak_level</var> <var class=”var”>RMS_difference</var>
<var class=”var”>RMS_level</var> <var class=”var”>RMS_peak</var>
<var class=”var”>RMS_trough</var> For example, a full
key looks like <code class=”code”>lavfi.astats.1.DC_offset</code> or
<code class=”code”>lavfi.astats.Overall.Peak_count</code>.
Read below for the description of the keys. </dd> <dt><samp
class=”option”>reset</samp></dt> <dd> Set the number of frames
over which cumulative stats are calculated before being reset. Default is disabled.
 </dd> <dt><samp class=”option”>measure_perchannel</samp></dt>
<dd> Select the parameters which are measured per channel. The metadata
keys can be used as flags, default is <samp class=”option”>all</samp> which
measures everything. <samp class=”option”>none</samp> disables all per
channel measurement. </dd> <dt><samp class=”option”>measure_overall</samp></dt>
<dd> Select the parameters which are measured overall. The metadata
keys can be used as flags, default is <samp class=”option”>all</samp>
which measures everything. <samp class=”option”>none</samp> disables all
overall measurement. </dd> </dl>

A description of the measure keys follow:

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> no measures

</dd>

<dt><samp class="option">all</samp></dt>

<dd> all measures

</dd>

<dt><samp class="option">Bit_depth</samp></dt>

<dd> overall bit depth of audio, i.e. number of bits used for each sample

</dd>

<dt><samp class="option">Crest_factor</samp></dt>

<dd> standard ratio of peak to RMS level (note: not in dB)

</dd>

<dt><samp class="option">DC_offset</samp></dt>

<dd> mean amplitude displacement from zero

</dd>

<dt><samp class="option">Dynamic_range</samp></dt>

<dd> measured dynamic range of audio in dB

</dd>

<dt><samp class="option">Entropy</samp></dt>

<dd> entropy measured across whole audio, entropy of value near 1.0 is typically measured for white noise

</dd>

<dt><samp class="option">Flat_factor</samp></dt>

<dd> flatness (i.e. consecutive samples with the same value) of the signal at its peak levels

(i.e. either <var class="var">Min_level</var> or <var class="var">Max_level</var>)

</dd>

<dt><samp class="option">Max_difference</samp></dt>

<dd> maximal difference between two consecutive samples

</dd>

<dt><samp class="option">Max_level</samp></dt>

<dd> maximal sample level

</dd>

<dt><samp class="option">Mean_difference</samp></dt>

<dd> mean difference between two consecutive samples, i.e. the average of each difference between two consecutive samples

</dd>

<dt><samp class="option">Min_difference</samp></dt>

<dd> minimal difference between two consecutive samples

</dd>

<dt><samp class="option">Min_level</samp></dt>

<dd> minimal sample level

</dd>

<dt><samp class="option">Noise_floor</samp></dt>

<dd> minimum local peak measured in dBFS over a short window

</dd>

<dt><samp class="option">Noise_floor_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained

<var class="var">Noise floor</var>

</dd>

<dt><samp class=”option”>Number_of_Infs</samp></dt>

<dd> number of samples with an infinite value

</dd>

<dt><samp class=”option”>Number_of_NaNs</samp></dt>

<dd> number of samples with a NaN (not a number) value

</dd>

<dt><samp class=”option”>Number_of_denormals</samp></dt>

<dd> number of samples with a subnormal value

</dd>

<dt><samp class=”option”>Number_of_samples</samp></dt>

<dd> number of samples

</dd>

<dt><samp class=”option”>Peak_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained either

<var class=”var”>Min_level</var> or <var class=”var”>Max_level</var>

</dd>

<dt><samp class=”option”>Abs_Peak_count</samp></dt>

<dd> number of occasions that the absolute samples taken from the signal attained

max absolute value of <var class=”var”>Min_level</var> and <var class=”var”>Max_level</var>

</dd>

<dt><samp class=”option”>Peak_level</samp></dt>

<dd> standard peak level measured in dBFS

</dd>

<dt><samp class="option">RMS_difference</samp></dt>

<dd> Root Mean Square difference between two consecutive samples

</dd>

<dt><samp class="option">RMS_level</samp></dt>

<dd> standard RMS level measured in dBFS

</dd>

<dt><samp class="option">RMS_peak</samp></dt>

<dt><samp class="option">RMS_trough</samp></dt>

<dd> peak and through values for RMS level measured over a short window, measured in dBFS.

</dd>

<dt><samp class="option">Zero crossings</samp></dt>

<dd> number of points where the waveform crosses the zero level axis

</dd>

<dt><samp class="option">Zero crossings rate</samp></dt>

<dd> rate of Zero crossings and number of audio samples

</dd>

</dl>

 <h3 class="section">27.60 asubboost# TOC</h3> Boost subwoofer frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dry</samp></dt> <dd> Set dry gain, how much of original signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet gain, how much of filtered signal is kept. Allowed range is from 0 to 1. Default value is 1.0.

</dd> <dt><samp class="option">boost</samp></dt> <dd> Set max boost factor. Allowed range is from 1 to 12. Default value is 2. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set delay line decay gain value. Allowed range is from 0 to 1. Default value is 0.0. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Set delay line feedback gain value. Allowed range is from 0 to 1. Default value is 0.9. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 50 to 900. Default value is 100. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope amount for cutoff frequency. Allowed range is 0.0001 to 1. Default value is 0.5. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay. Allowed range is from 1 to 100. Default value is 20. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the channels to process. Default value is all available. </dd> </dl>

 <h4 class="subsection">27.60.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.61 asubcut# TOC</h3> Cut subwoofer frequencies. This filter allows to set custom, steeper roll off than highpass filter, and thus is able to more attenuate frequency content in stop-band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 2 to 200. Default value is 20. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">27.61.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.62 asupercut# TOC</h3> Cut super frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt>

<dd> Set cutoff frequency in Hertz. Allowed range is 20000 to 192000. Default value is 20000. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class=”subsection”>27.62.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.63 asuperpass# TOC</h3> Apply high order Butterworth band-pass filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class=”option”>qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class=”subsection”>27.63.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.64 asuperstop# TOC</h3> Apply high order Butterworth band-stop filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class=”option”>qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">27.64.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.65 atempo# TOC</h3>

Adjust audio tempo. The filter accepts exactly one parameter, the audio tempo. If not specified then the filter will assume nominal 1.0 tempo. Tempo must be in the [0.5, 100.0] range. Note that tempo greater than 2 will skip some samples rather than blend them in. If for any reason this is a concern it is always possible to daisy-chain several instances of atempo to achieve the desired product tempo. <h4 class="subsection">27.65.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Slow down audio to 80% tempo: <pre class="example-preformatted">atempo=0.8 </pre>

To speed up audio to 300% tempo: <pre class="example-preformatted">atempo=3 </pre>

To speed up audio to 300% tempo by daisy-chaining two atempo instances: <pre class="example-preformatted">atempo=sqrt(3),atempo=sqrt(3) </pre>

 <h4 class="subsection">27.65.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class="var">tempo</var>" </dd> </dl>

 <h3 class="section">27.66 atilt# TOC</h3> Apply spectral tilt filter to audio stream. This filter apply any spectral roll-off slope over any specified frequency band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> Set central

frequency of tilt in Hz. Default is 10000 Hz. </dd> <dt><samp class=”option”>slope</samp></dt> <dd> Set slope direction of tilt. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set width of tilt. Default is 1000. Allowed range is from 100 to 10000. </dd> <dt><samp class=”option”>order</samp></dt> <dd> Set order of tilt filter. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Set input volume level. Allowed range is from 0 to 4. Default is 1. </dd> </dl>

 <h4 class=”subsection”>27.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>27.67 atrim# TOC</h3>

Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>start</samp></dt> <dd> Timestamp (in seconds) of the start of the section to keep. I.e. the audio sample with the timestamp <var class=”var”>start</var> will be the first sample in the output. </dd> <dt><samp class=”option”>end</samp></dt> <dd> Specify time of the first audio sample that will be dropped, i.e. the audio sample immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last sample in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> Same as <var class=”var”>start</var>, except this option sets the start timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> Same as <var class=”var”>end</var>, except this option sets the end timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_sample</samp></dt> <dd> The number of the first sample that should be output. </dd> <dt><samp class=”option”>end_sample</samp></dt> <dd> The number of the first sample that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the

frame timestamp, while the _sample options simply count the samples that pass through the filter. So start/end_pts and start/end_sample will give different results when the timestamps are wrong, inexact or do not start at zero. Also note that this filter does not modify the timestamps. If you wish to have the output timestamps start at zero, insert the asetpts filter after the atrim filter. If multiple start or end options are set, this filter tries to be greedy and keep all samples that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple atrim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class="itemize mark-bullet"> Drop everything except the second minute of input: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=60:120 </pre>

Keep only the first 1000 samples: <pre class="example-preformatted">ffmpeg -i INPUT -af atrim=end_sample=1000 </pre>

 <h3 class="section">27.68 axcorrelate# TOC</h3> Calculate normalized windowed cross-correlation between two input audio streams. Resulted samples are always between -1 and 1 inclusive. If result is 1 it means two input samples are highly correlated in that selected segment. Result 0 means they are not correlated at all. If result is -1 it means two input samples are out of phase, which means they cancel each other. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size</samp></dt> <dd> Set size of segment over which cross-correlation is calculated. Default is 256. Allowed range is from 2 to 131072. </dd> <dt><samp class="option">algo</samp></dt> <dd> Set algorithm for cross-correlation. Can be <code class="code">slow</code> or <code class="code">fast</code> or <code class="code">best</code>. Default is <code class="code">best</code>. Fast algorithm assumes mean values over any given segment are always zero and thus need much less calculations to make. This is generally not true, but is valid for typical audio streams. </dd> </dl>

 <h4 class="subsection">27.68.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Calculate correlation between chan-

nels in stereo audio stream: <pre class=”example-preformatted”>ffmpeg -i stereo.wav -af channelsplit,axcorrelate=size=1024:algo=fast correlation.wav </pre>

 <h3 class=”section”>27.69 bandpass# TOC</h3>

Apply a two-pole Butterworth band-pass filter with central frequency <var class=”var”>frequency</var>, and (3dB-point) band-width width. The <var class=”var”>csg</var> option selects a constant skirt gain (peak gain = Q) instead of the default: constant 0dB peak gain. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class=”code”>3000</code>. </dd> <dt><samp class=”option”>csg</samp></dt> <dd> Constant skirt gain if set to 1. Defaults to 0. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>

<dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.69.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change bandpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bandpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd>
<dt><samp class="option">width, w</samp></dt> <dd> Change bandpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.70 bandreject# TOC</h3>

Apply a two-pole Butterworth band-reject filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width <var class="var">width</var>. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify

the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.70.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change bandreject frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change bandreject width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bandreject width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandreject mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.71 bass, lowshelf# TOC</h3>

Boost or cut the bass (lower) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">100</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt>

<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.71.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change bass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change bass gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.72 biquad# TOC</h3>

Apply a biquad IIR filter with the given coefficients. Where <var class="var">b0</var>, <var class="var">b1</var>, <var class="var">b2</var> and <var class="var">a0</var>, <var class="var">a1</var>, <var class="var">a2</var> are the numerator and denominator coefficients respectively. and <var class="var">channels</var>, <var class="var">c</var> specify which channels to filter, by default all available are filtered.
 <h4 class="subsection">27.72.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">a0</samp></dt> <dt><samp class="option">a1</samp></dt>
<dt><samp class="option">a2</samp></dt> <dt><samp class="option">b0</samp></dt>
<dt><samp class="option">b1</samp></dt> <dt><samp class="option">b2</samp></dt>

<dd> Change biquad parameter. Syntax for the command is : "<var class="var">value</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h3 class="section">27.73 bs2b# TOC</h3> Bauer stereo to binaural transformation, which improves headphone listening of stereo audio records. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libbs2b</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Pre-defined crossfeed level. <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Default level (fcut=700, feed=50). </dd> <dt><samp class="option">cmoy</samp></dt> <dd> Chu Moy circuit (fcut=700, feed=60). </dd> <dt><samp class="option">jmeier</samp></dt> <dd> Jan Meier circuit (fcut=650, feed=95). </dd> </dl>

</dd> <dt><samp class="option">fcut</samp></dt> <dd> Cut fre-

quency (in Hz). </dd> <dt><samp class="option">feed</samp></dt> <dd> Feed level (in Hz). </dd> </dl>

 <h3 class="section">27.74 channelmap# TOC</h3>

Remap input channels to new locations. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Map channels from input to output. The argument is a ’|’-separated list of mappings, each in the <code class="code"><var class="var">in_channel</var>-<var class="var">out_channel</var></code> or <code class="code"><var class="var">in_channel</var></code> form. <var class="var">in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the input channel layout. <var class="var">out_channel</var> is the name of the output channel or its index in the output channel layout. If <var class="var">out_channel</var> is not given then it is implicitly an index, starting with zero and increasing by one for each mapping. Mixing different types of mappings is not allowed and will result in a parse error. </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the output stream. If not specified, then filter will guess it based on the <var class="var">out_channel</var> names or the number of mappings. Guessed layouts will not necessarily contain channels in the order of the mappings. </dd> </dl>

If no mapping is present, the filter will implicitly map input channels to output channels, preserving indices. <h4 class="subsection">27.74.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a 5.1+downmix input MOV file, <pre class="example-preformatted">ffmpeg -i in.mov -filter 'channelmap=map=DL-FL|DR-FR' out.wav </pre> will create an output WAV file tagged as stereo from the downmix channels of the input. To fix a 5.1 WAV improperly encoded in AAC’s native channel order <pre class="example-preformatted">ffmpeg -i in.wav -filter 'channelmap=1|2|0|5|3|4:5.1' out.wav </pre>

 <h3 class="section">27.75 channelsplit# TOC</h3>

Split each channel from an input audio stream into a separate output stream. It accepts the following parameters: <dl class="table">

<dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the input stream. The default is "stereo". </dd> <dt><samp class="option">channels</samp></dt> <dd> A channel layout describing the channels to be extracted as separate output streams or "all" to extract each input channel as a separate stream. The default is "all". Choosing channels not present in channel layout in the input will result in an error. </dd> </dl>

 <h4 class="subsection">27.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a stereo input MP3 file, <pre class="example-preformatted">ffmpeg -i in.mp3 -filter_complex channelsplit out.mkv </pre> will create an output Matroska file with two audio streams, one containing only the left channel and the other the right channel. Split a 5.1 WAV file into per-channel files: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1[FL][FR][FC][LFE][SL][SR]' -map '[FL]' front_left.wav -map '[FR]' front_right.wav -map '[FC]' front_center.wav -map '[LFE]' lfe.wav -map '[SL]' side_left.wav -map '[SR]' side_right.wav </pre>

Extract only LFE from a 5.1 WAV file: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1:channels=LFE[LFE]' -map '[LFE]' lfe.wav </pre>

 <h3 class="section">27.76 chorus# TOC</h3> Add a chorus effect to the audio. Can make a single vocal sound like a chorus, but can also be applied to instrumentation. Chorus resembles an echo effect with a short delay, but whereas with echo the delay is constant, with chorus, it is varied using sinusoidal or triangular modulation. The modulation depth defines the range the modulated delay is played before or after the delay. Hence the delayed sound will sound slower or faster, that is the delayed sound tuned around the original one, like in a chorus where some vocals are slightly off key. It accepts the following parameters: <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.4. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set delays. A typical delay is around 40ms to 60ms. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set decays. </dd> <dt><samp class="option">speeds</samp></dt>

<dd> Set speeds. </dd> <dt><samp class=”option”>depths</samp></dt> <dd> Set depths. </dd> </dl>

 <h4 class=”subsection”>27.76.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> A single delay: <pre class=”example-preformatted”>chorus=0.7:0.9:55:0.4:0.25:2 </pre>

Two delays: <pre class=”example-preformatted”>chorus=0.6:0.9:50|60:0.4|0.32:0.25|0.4:2|1.3 </pre>

Fuller sounding chorus with three delays: <pre class=”example-preformatted”>chorus=0.5:0.9:50|60|40:0.4|0.32|0.3:0.25|0.4|0.3:2|2.3|1.3 </pre>

 <h3 class=”section”>27.77 compand# TOC</h3> Compress or expand the audio’s dynamic range. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>attacks</samp></dt> <dt><samp class=”option”>decays</samp></dt> <dd> A list of times in seconds for each channel over which the instantaneous level of the input signal is averaged to determine its volume. <var class=”var”>attacks</var> refers to increase of volume and <var class=”var”>decays</var> refers to decrease of volume. For most situations, the attack time (response to the audio getting louder) should be shorter than the decay time, because the human ear is more sensitive to sudden loud audio than sudden soft audio. A typical value for attack is 0.3 seconds and a typical value for decay is 0.8 seconds. If specified number of attacks & decays is lower than number of channels, the last set attack/decay will be used for all remaining channels. </dd> <dt><samp class=”option”>points</samp></dt> <dd> A list of points for the transfer function, specified in dB relative to the maximum possible signal amplitude. Each key points list must be defined using the following syntax: <code class=”code”>x0/y0|x1/y1|x2/y2|....</code> or <code class=”code”>x0/y0 x1/y1 x2/y2 ....</code> The input values must be in strictly increasing order but the transfer function does not have to be monotonically rising. The point <code class=”code”>0/0</code> is assumed but may be overridden (by <code class=”code”>0/out-dBn</code>). Typical values for the transfer function are <code class=”code”>-70/-70|-60/-20|1/0</code>. </dd> <dt><samp class=”option”>soft-knee</samp></dt> <dd> Set the curve radius in dB for all joints. It defaults to 0.01. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set the additional gain in dB to be applied at all points on the transfer function. This allows for easy adjustment of the overall gain. It defaults to 0. </dd> <dt><samp

class=”option”>volume</samp></dt> <dd> Set an initial volume, in dB, to be assumed for each channel when filtering starts. This permits the user to supply a nominal level initially, so that, for example, a very large gain is not applied to initial signal levels before the companding has begun to operate. A typical value for audio which is initially quiet is -90 dB. It defaults to 0. </dd> <dt><samp class=”option”>delay</samp></dt> <dd> Set a delay, in seconds. The input audio is analyzed immediately, but audio is delayed before being fed to the volume adjuster. Specifying a delay approximately equal to the attack/decay times allows the filter to effectively operate in predictive rather than reactive mode. It defaults to 0. </dd> </dl>

 <h4 class=”subsection”>27.77.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make music with both quiet and loud passages suitable for listening to in a noisy environment: <pre class=”example-preformatted”>compand=.3|.3:1|1:-90/-60|-60/-40|-40/-30|-20/-20:6:0:-90:0.2 </pre>

Another example for audio with whisper and explosion parts: <pre class=”example-preformatted”>compand=0|0:1|1:-90/-900|-70/-70|-30/-9|0/-3:6:0:0:0 </pre>

A noise gate for when the noise is at a lower level than the signal: <pre class=”example-preformatted”>compand=.1|.1:.2|.2:-900/-900|-50.1/-900|-50/-50:.01:0:-90:.1 </pre>

Here is another noise gate, this time for when the noise is at a higher level than the signal (making it, in some ways, similar to squelch): <pre class=”example-preformatted”>compand=.1|.1:.1|.1:-45.1/-45.1|-45/-900|0/-900:.01:45:-90:.1 </pre>

2:1 compression starting at -6dB: <pre class=”example-preformatted”>compand=points=-80/-80|-6/-6|0/-3.8|20/3.5 </pre>

2:1 compression starting at -9dB: <pre class=”example-preformatted”>compand=points=-80/-80|-9/-9|0/-5.3|20/2.9 </pre>

2:1 compression starting at -12dB: <pre class=”example-preformatted”>compand=points=-80/-80|-12/-12|0/-6.8|20/1.9 </pre>

2:1 compression starting at -18dB: <pre class=”example-preformatted”>compand=points=-80/-80|-18/-18|0/-9.8|20/0.7 </pre>

3:1 compression starting at -15dB: <pre class=”example-preformatted”>compand=points=-80/-80|-15/-15|0/-10.8|20/-5.2 </pre>

Compressor/Gate: <pre class=”example-preformatted”>compand=points=-80/-105|-62/-80|-15.4/-15.4|0/-12|20/-7.6 </pre>

Expander: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-169|-54/-80|-49.5/-64.6|-41.1/-41.1|-25.8/-15|-10.8/-4.5|0/0|20/8.3 </pre>

Hard limiter at -6dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-6/-6|20/-6 </pre>

Hard limiter at -12dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12/-12|20/-12 </pre>

Hard noise gate at -35 dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-115|-35.1/-80|-35/-35|20/20 </pre>

Soft limiter: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12.4/-12.4|-6/-8|0/-6.8|20/-2.8 </pre>

 <h3 class=”section”>27.78 compensationdelay# TOC</h3>

Compensation Delay Line is a metric based delay to compensate differing positions of microphones or speakers. For example, you have recorded guitar with two microphones placed in different locations. Because the front of sound wave has fixed speed in normal conditions, the phasing of microphones can vary and depends on their location and interposition. The best sound mix can be achieved when these microphones are in phase (synchronized). Note that a distance of ~30 cm between microphones makes one microphone capture the signal in antiphase to the other microphone. That makes the final mix sound moody. This filter helps to solve phasing problems by adding different delays to each microphone track and make them synchronized. The best result can be reached when you take one track as base and synchronize other tracks one by one with it. Remember that synchronization/delay tolerance depends on sample rate, too. Higher sample rates will give more tolerance. The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mm</samp></dt> <dd> Set millimeters distance. This is compensation distance for fine tuning. Default is 0. </dd> <dt><samp class=”option”>cm</samp></dt> <dd> Set cm distance. This is compensation distance for tightening distance setup. Default is 0.

</dd> <dt><samp class="option">m</samp></dt> <dd> Set meters distance. This is compensation distance for hard distance setup. Default is 0. </dd> <dt><samp class="option">dry</samp></dt> <dd> Set dry amount. Amount of unprocessed (dry) signal. Default is 0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet amount. Amount of processed (wet) signal. Default is 1. </dd> <dt><samp class="option">temp</samp></dt> <dd> Set temperature in degrees Celsius. This is the temperature of the environment. Default is 20. </dd> </dl>

 <h4 class="subsection">27.78.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.79 crossfeed# TOC</h3> Apply headphone crossfeed filter. Crossfeed is the process of blending the left and right channels of stereo audio recording. It is mainly used to reduce extreme stereo separation of low frequencies. The intent is to produce more speaker like sound to the listener. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Set strength of crossfeed. Default is 0.2. Allowed range is from 0 to 1. This sets gain of low shelf filter for side part of stereo image. Default is -6dB. Max allowed is -30db when strength is set to 1. </dd> <dt><samp class="option">range</samp></dt> <dd> Set soundstage wideness. Default is 0.5. Allowed range is from 0 to 1. This sets cut off frequency of low shelf filter. Default is cut off near 1550 Hz. With range set to 1 cut off frequency is set to 2100 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set curve slope of low shelf filter. Default is 0.5. Allowed range is from 0.01 to 1. </dd> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 0.9. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class="option">block_size</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.79.1 Commands<a class="anchor hidden-xs" href="#Commands-

34" aria-hidden="true"># TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.80 crystalizer# TOC</h3> Simple algorithm for audio noise sharpening. This filter linearly increases differences between each audio sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Sets the intensity of effect (default: 2.0). Must be in range between -10.0 to 0 (unchanged sound) to 10.0 (maximum effect). To inverse filtering use negative value. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">27.80.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.81 dcshift# TOC</h3> Apply a DC shift to the audio. This can be useful to remove a DC offset (caused perhaps by a hardware problem in the recording chain) from the audio. The effect of a DC offset is reduced headroom and hence volume. The astats filter can be used to determine if a signal has a DC offset. <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Set the DC shift, allowed range is [-1, 1]. It indicates the amount to shift the audio. </dd> <dt><samp class="option">limitergain</samp></dt> <dd> Optional. It should have a value much less than 1 (e.g. 0.05 or 0.02) and is used to prevent clipping. </dd> </dl>

 <h3 class="section">27.82 deesser# TOC</h3>

Apply de-essing to the audio samples. <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Set intensity for triggering de-essing. Allowed range is from 0 to 1. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set amount of ducking on treble part of sound. Allowed range is from 0 to 1.

Default is 0.5. </dd> <dt><samp class="option">f</samp></dt> <dd> How much of original frequency content to keep when de-essing. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">s</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass ess filtered out. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass only ess. Default value is <var class="var">o</var>. </dd> </dl>

</dd> </dl>

 <h3 class="section">27.83 dialoguenhance# TOC</h3> Enhance dialogue in stereo audio. This filter accepts stereo input and produce surround (3.0) channels output. The newly produced front center channel have enhanced speech dialogue originally available in both stereo channels. This filter outputs front left and front right channels same as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">original</samp></dt> <dd> Set the original center factor to keep in front center channel output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">enhance</samp></dt> <dd> Set the dialogue enhance factor to put in front center channel output. Allowed range is from 0 to 3. Default value is 1. </dd> <dt><samp class="option">voice</samp></dt> <dd> Set the voice detection factor. Allowed range is from 2 to 32. Default value is 2. </dd> </dl>

 <h4 class="subsection">27.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.84 drmeter# TOC</h3> Measure audio dynamic range. DR values of 14 and higher is found in very dynamic material. DR of 8 to 13 is found in transition material. And anything less that 8 have very poor dynamics and is very compressed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Set window length in seconds used to split audio into segments of equal length. Default is 3 seconds. </dd> </dl>

<h3 class="section">27.85 dynaudnorm# TOC</h3>

Dynamic Audio Normalizer.

This filter applies a certain amount of gain to the input audio in order

to bring its peak magnitude to a target level (e.g. 0 dBFS). However, in

contrast to more "simple" normalization algorithms, the Dynamic Audio

Normalizer *dynamically* re-adjusts the gain factor to the input audio.

This allows for applying extra gain to the "quiet" sections of the audio

while avoiding distortions or clipping the "loud" sections. In other words:

The Dynamic Audio Normalizer will "even out" the volume of quiet and loud

sections, in the sense that the volume of each section is brought to the

same target level. Note, however, that the Dynamic Audio Normalizer achieves

this goal *without* applying "dynamic range compressing". It will retain 100%

of the dynamic range *within* each section of the audio file.

<dl class="table">

<dt><samp class="option">framelen, f</samp></dt>

<dd> Set the frame length in milliseconds. In range from 10 to 8000 milliseconds.

Default is 500 milliseconds.

The Dynamic Audio Normalizer processes the input audio in small chunks,

referred to as frames. This is required, because a peak magnitude has no

meaning for just a single sample value. Instead, we need to determine the

peak magnitude for a contiguous sequence of sample values. While a "standard"

normalizer would simply use the peak magnitude of the complete file, the

Dynamic Audio Normalizer determines the peak magnitude individually for each frame. The length of a frame is specified in milliseconds. By default, the Dynamic Audio Normalizer uses a frame length of 500 milliseconds, which has been found to give good results with most files.

Note that the exact frame length, in number of samples, will be determined automatically, based on the sampling rate of the individual input audio file.

</dd>

<dt><samp class="option">gausssize, g</samp></dt>

<dd> Set the Gaussian filter window size. In range from 3 to 301, must be odd

number. Default is 31.

Probably the most important parameter of the Dynamic Audio Normalizer is the

<code class="code">window size</code> of the Gaussian smoothing filter. The filter’s window size

is specified in frames, centered around the current frame. For the sake of simplicity, this must be an odd number. Consequently, the default value of 31 takes into account the current frame, as well as the 15 preceding frames and the 15 subsequent frames. Using a larger window results in a stronger smoothing effect and thus in less gain variation, i.e. slower gain adaptation. Conversely, using a smaller window results in a weaker smoothing effect and thus in more gain variation, i.e. faster gain adaptation.

In other words, the more you increase this value, the more the Dynamic Audio Normalizer will behave like a "traditional" normalization filter. On the

contrary, the more you decrease this value, the more the Dynamic Audio Normalizer will behave like a dynamic range compressor.

</dd>

<dt><samp class="option">peak, p</samp></dt>

<dd> Set the target peak value. This specifies the highest permissible magnitude

level for the normalized audio input. This filter will try to approach the

target peak magnitude as closely as possible, but at the same time it also

makes sure that the normalized signal will never exceed the peak magnitude.

A frame’s maximum local gain factor is imposed directly by the target peak

magnitude. The default value is 0.95 and thus leaves a headroom of 5%*.

It is not recommended to go above this value.

</dd>

<dt><samp class="option">maxgain, m</samp></dt>

<dd> Set the maximum gain factor. In range from 1.0 to 100.0. Default is 10.0.

The Dynamic Audio Normalizer determines the maximum possible (local) gain

factor for each input frame, i.e. the maximum gain factor that does not

result in clipping or distortion. The maximum gain factor is determined by

the frame’s highest magnitude sample. However, the Dynamic Audio Normalizer

additionally bounds the frame’s maximum gain factor by a predetermined

(global) maximum gain factor. This is done in order to avoid excessive gain

factors in "silent" or almost silent frames. By default, the maximum gain

factor is 10.0, For most inputs the default value should be sufficient and

it usually is not recommended to increase this value. Though, for input

with an extremely low overall volume level, it may be necessary to allow even

higher gain factors. Note, however, that the Dynamic Audio Normalizer does

not simply apply a "hard" threshold (i.e. cut off values above the threshold).

Instead, a "sigmoid" threshold function will be applied. This way, the

gain factors will smoothly approach the threshold value, but never exceed that

value.

</dd>

<dt><samp class=”option”>targetrms, r</samp></dt>

<dd> Set the target RMS. In range from 0.0 to 1.0. Default is 0.0 - disabled.

By default, the Dynamic Audio Normalizer performs "peak" normalization.

This means that the maximum local gain factor for each frame is defined

(only) by the frame’s highest magnitude sample. This way, the samples can

be amplified as much as possible without exceeding the maximum signal

level, i.e. without clipping. Optionally, however, the Dynamic Audio

Normalizer can also take into account the frame’s root mean square,

abbreviated RMS. In electrical engineering, the RMS is commonly used to

determine the power of a time-varying signal. It is therefore considered

that the RMS is a better approximation of the "perceived loudness" than

just looking at the signal’s peak magnitude. Consequently, by adjusting all

frames to a constant RMS value, a uniform "perceived loudness" can be

established. If a target RMS value has been specified, a frame’s local gain

factor is defined as the factor that would result in exactly that RMS value.

Note, however, that the maximum local gain factor is still restricted by the

frame’s highest magnitude sample, in order to prevent clipping.

</dd>

<dt><samp class=”option”>coupling, n</samp></dt>

<dd> Enable channels coupling. By default is enabled.

By default, the Dynamic Audio Normalizer will amplify all channels by the same

amount. This means the same gain factor will be applied to all channels, i.e.

the maximum possible gain factor is determined by the "loudest" channel.

However, in some recordings, it may happen that the volume of the different

channels is uneven, e.g. one channel may be "quieter" than the other one(s).

In this case, this option can be used to disable the channel coupling. This way,

the gain factor will be determined independently for each channel, depending

only on the individual channel’s highest magnitude sample. This allows for

harmonizing the volume of the different channels.

</dd>

<dt><samp class="option">correctdc, c</samp></dt>

<dd> Enable DC bias correction. By default is disabled.

An audio signal (in the time domain) is a sequence of sample values.

In the Dynamic Audio Normalizer these sample values are represented in the

-1.0 to 1.0 range, regardless of the original input format. Normally, the

audio signal, or "waveform", should be centered around the zero point.

That means if we calculate the mean value of all samples in a file, or in a

single frame, then the result should be 0.0 or at least very close to that

value. If, however, there is a significant deviation of the mean value from

0.0, in either positive or negative direction, this is referred to as a

DC bias or DC offset. Since a DC bias is clearly undesirable, the Dynamic

Audio Normalizer provides optional DC bias correction.

With DC bias correction enabled, the Dynamic Audio Normalizer will determine

the mean value, or "DC correction" offset, of each input frame and subtract

that value from all of the frame’s sample values which ensures those samples

are centered around 0.0 again. Also, in order to avoid "gaps" at the frame

boundaries, the DC correction offset values will be interpolated smoothly

between neighbouring frames.

</dd>

<dt><samp class=”option”>altboundary, b</samp></dt>

<dd> Enable alternative boundary mode. By default is disabled.

The Dynamic Audio Normalizer takes into account a certain neighbourhood

around each frame. This includes the preceding frames as well as the

subsequent frames. However, for the "boundary" frames, located at the very

beginning and at the very end of the audio file, not all neighbouring

frames are available. In particular, for the first few frames in the audio

file, the preceding frames are not known. And, similarly, for the last few

frames in the audio file, the subsequent frames are not known. Thus, the

question arises which gain factors should be assumed for the missing frames

in the "boundary" region. The Dynamic Audio Normalizer implements two modes

to deal with this situation. The default boundary mode assumes a gain factor

of exactly 1.0 for the missing frames, resulting in a smooth "fade in" and

"fade out" at the beginning and at the end of the input, respectively.

</dd>

<dt><samp class=”option”>compress, s</samp></dt>

<dd> Set the compress factor. In range from 0.0 to 30.0. Default is 0.0.

By default, the Dynamic Audio Normalizer does not apply "traditional"

compression. This means that signal peaks will not be pruned and thus the

full dynamic range will be retained within each local neighbourhood. However,

in some cases it may be desirable to combine the Dynamic Audio Normalizer’s

normalization algorithm with a more "traditional" compression.

For this purpose, the Dynamic Audio Normalizer provides an optional compression

(thresholding) function. If (and only if) the compression feature is enabled,
all input frames will be processed by a soft knee thresholding function prior
to the actual normalization process. Put simply, the thresholding function is
going to prune all samples whose magnitude exceeds a certain threshold value.
However, the Dynamic Audio Normalizer does not simply apply a fixed threshold
value. Instead, the threshold value will be adjusted for each individual
frame.
In general, smaller parameters result in stronger compression, and vice versa.
Values below 3.0 are not recommended, because audible distortion may appear.

</dd>
<dt><samp class="option">threshold, t</samp></dt>
<dd> Set the target threshold value. This specifies the lowest permissible
magnitude level for the audio input which will be normalized.
If input frame volume is above this value frame will be normalized.
Otherwise frame may not be normalized at all. The default value is set
to 0, which means all input frames will be normalized.
This option is mostly useful if digital noise is not wanted to be amplified.

</dd>
<dt><samp class="option">channels, h</samp></dt>
<dd> Specify which channels to filter, by default all available channels are
filtered.

</dd>
<dt><samp class="option">overlap, o</samp></dt>
<dd> Specify overlap for frames. If set to 0 (default) no frame overlapping
is done.
Using >0 and <1 values will make less conservative gain adjustments, like
when framelen option is set to smaller value, if framelen option value is
compensated for non-zero overlap then gain adjustments will be smoother across
time

compared to zero overlap case.

</dd>

<dt><samp class="option">curve, v</samp></dt>

<dd> Specify the peak mapping curve expression which is going to be used when calculating

gain applied to frames. The max output frame gain will still be limited by other

options mentioned previously for this filter.

The expression can contain the following constants:

<dl class="table">

<dt><samp class="option">ch</samp></dt>

<dd> current channel number

</dd>

<dt><samp class="option">sn</samp></dt>

<dd> current sample number

</dd>

<dt><samp class="option">nb_channels</samp></dt>

<dd> number of channels

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds

</dd>

<dt><samp class="option">sr</samp></dt>

<dd> sample rate

</dd>

<dt><samp class="option">p</samp></dt>

<dd> current frame peak value

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">27.85.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.86 earwax# TOC</h3>

Make audio easier to listen to on headphones. This filter adds ‘cues’ to 44.1kHz stereo (i.e. audio CD format) audio so that when listened to on headphones the stereo image is moved from inside your head (standard for headphones) to outside and in front of the listener (standard for speakers). Ported from SoX. <h3 class="section">27.87 equalizer# TOC</h3>

Apply a two-pole peaking equalisation (EQ) filter. With this filter, the signal-level at and around a selected frequency can be increased or decreased, whilst (unlike bandpass and bandreject filters) that at all other frequencies is unchanged. In order to produce complex equalisation curves, this filter can be given several times, each with a different central frequency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency in Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Set the required gain or attenuation in dB. Beware of clipping when using a positive gain. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.87.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Attenuate 10 dB at 1000 Hz, with a bandwidth of 200 Hz: <pre class="example-preformatted">equalizer=f=1000:t=h:width=200:g=-10 </pre>

Apply 2 dB gain at 1000 Hz with Q 1 and attenuate 5 dB at 100 Hz with Q 2: <pre class="example-preformatted">equalizer=f=1000:t=q:w=1:g=2,equalizer=f=100:t=q:w=2:g=-5 </pre>

 <h4 class="subsection">27.87.2 Com-

mands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change equalizer frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change equalizer width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change equalizer width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change equalizer gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change equalizer mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.88 extrastereo# TOC</h3>

Linearly increases the difference between left and right channels which adds some sort of "live" effect to playback. The filter accepts the following options: <dl class="table"> <dt><samp class="option">m</samp></dt> <dd> Sets the difference coefficient (default: 2.5). 0.0 means mono sound (average of both channels), with 1.0 sound will be unchanged, with -1.0 left and right channels will be swapped. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">27.88.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.89 firequalizer# TOC</h3> Apply FIR Equalization using arbitrary frequency response. The filter accepts the following option: <dl class="table"> <dt><samp class="option">gain</samp></dt> <dd> Set gain curve equation (in dB). The expression can contain variables: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> the evaluated frequency </dd> <dt><samp class="option">sr</samp></dt>

<dd> sample rate </dd> <dt><samp class="option">ch</samp></dt>
<dd> channel number, set to 0 when multichannels evaluation is disabled </dd> <dt><samp class="option">chid</samp></dt>
<dd> channel id, see libavutil/channel_layout.h, set to the first channel id when multichannels evaluation is disabled </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd>
<dt><samp class="option">chlayout</samp></dt> <dd> channel_layout, see libavutil/channel_layout.h </dd> </dl> and functions:
<dl class="table"> <dt><samp class="option">gain_interpolate(f)</samp></dt>
<dd> interpolate gain on frequency f based on gain_entry </dd>
<dt><samp class="option">cubic_interpolate(f)</samp></dt> <dd> same as gain_interpolate, but smoother </dd> </dl> This option is also available as command. Default is <code class="code">gain_interpolate(f)</code>.
 </dd> <dt><samp class="option">gain_entry</samp></dt>
<dd> Set gain entry for gain_interpolate function. The expression can contain functions: <dl class="table"> <dt><samp class="option">entry(f, g)</samp></dt> <dd> store gain entry at frequency f with value g </dd> </dl> This option is also available as command. </dd> <dt><samp class="option">delay</samp></dt>
<dd> Set filter delay in seconds. Higher value means more accurate. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set filter accuracy in Hz. Lower value means more accurate. Default is <code class="code">5</code>.
 </dd> <dt><samp class="option">wfunc</samp></dt> <dd> Set window function. Acceptable values are: <dl class="table">
<dt><samp class="option">rectangular</samp></dt> <dd> rectangular window, useful when gain curve is already smooth </dd> <dt><samp class="option">hann</samp></dt> <dd> hann window (default)
</dd> <dt><samp class="option">hamming</samp></dt>
<dd> hamming window </dd> <dt><samp class="option">blackman</samp></dt>
<dd> blackman window </dd> <dt><samp class="option">nuttall3</samp></dt>
<dd> 3-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">mnuttall3</samp></dt> <dd> minimum 3-terms discontinuous nuttall window </dd> <dt><samp class="option">nuttall</samp></dt>
<dd> 4-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">bnuttall</samp></dt> <dd> minimum 4-terms discontinuous nuttall (blackman-nuttall) window </dd>
<dt><samp class="option">bharris</samp></dt> <dd> blackman-harris window </dd> <dt><samp class="option">tukey</samp></dt>
<dd> tukey window </dd> </dl>

</dd> <dt><samp class="option">fixed</samp></dt> <dd> If enabled, use fixed number of audio samples. This improves speed when filtering with large delay. Default is disabled. </dd> <dt><samp class="option">multi</samp></dt> <dd> Enable multichannels evaluation on gain. Default is disabled. </dd> <dt><samp

class=”option”>zero_phase</samp></dt> <dd> Enable zero phase mode by subtracting timestamp to compensate delay. Default is disabled. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set scale used by gain. Acceptable values are: <dl class=”table”> <dt><samp class=”option”>linlin</samp></dt> <dd> linear frequency, linear gain </dd> <dt><samp class=”option”>linlog</samp></dt> <dd> linear frequency, logarithmic (in dB) gain (default) </dd> <dt><samp class=”option”>loglin</samp></dt> <dd> logarithmic (in octave scale where 20 Hz is 0) frequency, linear gain </dd> <dt><samp class=”option”>loglog</samp></dt> <dd> logarithmic frequency, logarithmic gain </dd> </dl>

</dd> <dt><samp class=”option”>dumpfile</samp></dt> <dd> Set file for dumping, suitable for gnuplot. </dd> <dt><samp class=”option”>dumpscale</samp></dt> <dd> Set scale for dumpfile. Acceptable values are same with scale option. Default is linlog. </dd> <dt><samp class=”option”>fft2</samp></dt> <dd> Enable 2-channel convolution using complex FFT. This improves speed significantly. Default is disabled. </dd> <dt><samp class=”option”>min_phase</samp></dt> <dd> Enable minimum phase impulse response. Default is disabled. </dd> </dl>

 <h4 class=”subsection”>27.89.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> lowpass at 1000 Hz: <pre class=”example-preformatted”>firequalizer=gain='if(lt(f,1000), 0, -INF)' </pre> lowpass at 1000 Hz with gain_entry: <pre class=”example-preformatted”>firequalizer=gain_entry='entry(1000,0); entry(1001, -INF)' </pre> custom equalization: <pre class=”example-preformatted”>firequalizer=gain_entry='entry(100,0); entry(400, -4); entry(1000, -6); entry(2000, 0)' </pre> higher delay with zero phase to compensate delay: <pre class=”example-preformatted”>firequalizer=delay=0.1:fixed=on:zero_phase=on </pre> lowpass on left channel, highpass on right channel: <pre class=”example-preformatted”>firequalizer=gain='if(eq(chid,1), gain_interpolate(f), if(eq(chid,2), gain_interpolate(1e6+f), 0))' :gain_entry='entry(1000, 0); entry(1001,-INF); entry(1e6+1000,0)':multi=on </pre>

 <h3 class=”section”>27.90 flanger# TOC</h3> Apply a flanging effect to the audio. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Set base delay in milliseconds. Range from 0 to 30. Default

value is 0. </dd> <dt><samp class="option">depth</samp></dt>
<dd> Set added sweep delay in milliseconds. Range from 0 to 10. Default
value is 2. </dd> <dt><samp class="option">regen</samp></dt>
<dd> Set percentage regeneration (delayed signal feedback). Range from -
95 to 95. Default value is 0. </dd> <dt><samp class="option">width</samp></dt>
<dd> Set percentage of delayed signal mixed with original. Range from 0 to
100. Default value is 71. </dd> <dt><samp class="option">speed</samp></dt>
<dd> Set sweeps per second (Hz). Range from 0.1 to 10. Default value
is 0.5. </dd> <dt><samp class="option">shape</samp></dt>
<dd> Set swept wave shape, can be <var class="var">triangular</var> or
<var class="var">sinusoidal</var>. Default value is <var class="var">sinusoidal</var>.
 </dd> <dt><samp class="option">phase</samp></dt> <dd> Set
swept wave percentage-shift for multi channel. Range from 0 to 100. Default
value is 25. </dd> <dt><samp class="option">interp</samp></dt>
<dd> Set delay-line interpolation, <var class="var">linear</var> or <var
class="var">quadratic</var>. Default is <var class="var">linear</var>.
</dd> </dl>

 <h3 class="section">27.91 haas<span class="pull-
right">#
TOC</h3>
Apply Haas effect to audio. Note that this makes most sense
to apply on mono signals. With this filter applied to mono signals it give
some directionality and stretches its stereo image. The filter
accepts the following options: <dl class="table"> <dt><samp
class="option">level_in</samp></dt> <dd> Set input level. By
default is <var class="var">1</var>, or 0dB </dd> <dt><samp
class="option">level_out</samp></dt> <dd> Set output level. By
default is <var class="var">1</var>, or 0dB. </dd> <dt><samp
class="option">side_gain</samp></dt> <dd> Set gain applied to side
part of signal. By default is <var class="var">1</var>. </dd>
<dt><samp class="option">middle_source</samp></dt> <dd> Set
kind of middle source. Can be one of the following: <dl class="table">
<dt>‘<samp class="samp">left</samp>’</dt> <dd> Pick
left channel. </dd> <dt>‘<samp class="samp">right</samp>’</dt>
<dd> Pick right channel. </dd> <dt>‘<samp class="samp">mid</samp>’</dt>
<dd> Pick middle part signal of stereo image. </dd> <dt>‘<samp
class="samp">side</samp>’</dt> <dd> Pick side part signal of
stereo image. </dd> </dl>

</dd> <dt><samp class="option">middle_phase</samp></dt> <dd> Change
middle phase. By default is disabled. </dd> <dt><samp
class="option">left_delay</samp></dt> <dd> Set left channel de-
lay. By default is <var class="var">2.05</var> milliseconds. </dd>
<dt><samp class="option">left_balance</samp></dt> <dd> Set left
channel balance. By default is <var class="var">-1</var>. </dd>
<dt><samp class="option">left_gain</samp></dt> <dd> Set left

channel gain. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>left_phase</samp></dt> <dd> Change left phase. By default is disabled. </dd> <dt><samp class=”option”>right_delay</samp></dt> <dd> Set right channel delay. By defaults is <var class=”var”>2.12</var> milliseconds. </dd> <dt><samp class=”option”>right_balance</samp></dt> <dd> Set right channel balance. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>right_gain</samp></dt> <dd> Set right channel gain. By default is <var class=”var”>1</var>. </dd> <dt><samp class=”option”>right_phase</samp></dt> <dd> Change right phase. By default is enabled. </dd> </dl>

 <h3 class=”section”>27.92 hdcd# TOC</h3>

Decodes High Definition Compatible Digital (HDCD) data. A 16-bit PCM stream with embedded HDCD codes is expanded into a 20-bit PCM stream. The filter supports the Peak Extend and Low-level Gain Adjustment features of HDCD, and detects the Transient Filter flag.   <pre class=”example-preformatted”>ffmpeg -i HDCD16.flac -af hdcd OUT24.flac </pre>

When using the filter with wav, note the default encoding for wav is 16-bit, so the resulting 20-bit stream will be truncated back to 16-bit. Use something like <code class=”command”>-acodec pcm_s24le</code> after the filter to get 24-bit PCM output. <pre class=”example-preformatted”>ffmpeg -i HDCD16.wav -af hdcd OUT16.wav ffmpeg -i HDCD16.wav -af hdcd -c:a pcm_s24le OUT24.wav </pre>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>disable_autoconvert</samp></dt> <dd> Disable any automatic format conversion or resampling in the filter graph. </dd> <dt><samp class=”option”>process_stereo</samp></dt> <dd> Process the stereo channels together. If target_gain does not match between channels, consider it invalid and use the last valid target_gain. </dd> <dt><samp class=”option”>cdt_ms</samp></dt> <dd> Set the code detect timer period in ms. </dd> <dt><samp class=”option”>force_pe</samp></dt> <dd> Always extend peaks above -3dBFS even if PE isn’t signaled. </dd> <dt><samp class=”option”>analyze_mode</samp></dt> <dd> Replace audio with a solid tone and adjust the amplitude to signal some specific aspect of the decoding process. The output file can be loaded in an audio editor alongside the original to aid analysis. <code class=”code”>analyze_mode=pe:force_pe=true</code> can be used to see all samples above the PE level. Modes are: <dl class=”table”> <dt>‘<samp class=”samp”>0, off</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class=”samp”>1, lle</samp>’</dt> <dd> Gain adjustment level at each sample </dd> <dt>‘<samp

class=”samp”>2, pe</samp>’</dt> <dd> Samples where peak extend occurs </dd> <dt>‘<samp class=”samp”>3, cdt</samp>’</dt> <dd> Samples where the code detect timer is active </dd> <dt>‘<samp class=”samp”>4, tgm</samp>’</dt> <dd> Samples where the target gain does not match between channels </dd> </dl> </dd> </dl>

 <h3 class=”section”>27.93 headphone# TOC</h3>

Apply head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones. The HRIRs are provided via additional streams, for each channel one stereo input stream is needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>map</samp></dt> <dd> Set mapping of input streams for convolution. The argument is a ’|’-separated list of channel names in order as they are given as additional stream inputs for filter. This also specify number of input streams. Number of input streams must be not less than number of channels in first stream plus one. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>lfe</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set size of frame in number of samples which will be processed at once. Default value is <var class=”var”>1024</var>. Allowed range is from 1024 to 96000. </dd> <dt><samp class=”option”>hrir</samp></dt> <dd> Set format of hrir stream. Default value is <var class=”var”>stereo</var>. Alternative value is <var class=”var”>multich</var>. If value is set to <var class=”var”>stereo</var>, number of additional streams should be greater or equal to number of input channels in first input stream. Also each additional stream should have stereo number of channels. If value is set to <var class=”var”>multich</var>, number of additional streams should be exactly one. Also number of input channels of additional stream should be equal or greater than twice number of channels of first input stream. </dd> </dl>

 <h4 class=”subsection”>27.93.1 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-44” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Full example using wav files as coefficients with amovie filters for 7.1 downmix, each amovie filter use stereo file with IR coefficients as input. The files give coefficients for each position of virtual loudspeaker: <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=azi_270_ele_0_DFC.wav[sr];amovie=azi_90_ele_0_DFC.wav[sl];amovie=azi_225_ele_0_D output.wav </pre>

Full example using wav files as coefficients with amovie filters for 7.1 downmix, but now in <var class=”var”>multich</var> <var class=”var”>hrir</var> format. <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=minp.wav[hrirs];[0:a][hrirs] output.wav </pre>

 <h3 class=”section”>27.94 highpass# TOC</h3>

Apply a high-pass filter with 3dB point frequency. The filter can be either single-pole, or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 3000. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp

class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt>
<dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt>
<dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt>
<dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set
precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt>
<dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>27.94.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>frequency, f</samp></dt> <dd> Change
highpass frequency. Syntax for the command is : "<var class=”var”>frequency</var>"
 </dd> <dt><samp class=”option”>width_type, t</samp></dt>
<dd> Change highpass width_type. Syntax for the command is
: "<var class=”var”>width_type</var>" </dd>
<dt><samp class=”option”>width, w</samp></dt> <dd> Change
highpass width. Syntax for the command is : "<var class=”var”>width</var>"
 </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change
highpass mix. Syntax for the command is : "<var class=”var”>mix</var>"
</dd> </dl>

 <h3 class=”section”>27.95 join#
TOC</h3>

Join multiple input streams into one multi-channel stream. It
accepts the following parameters: <dl class=”table”> <dt><samp
class=”option”>inputs</samp></dt> <dd> The number of input
streams. It defaults to 2. </dd> <dt><samp class=”option”>channel_layout</samp></dt>
<dd> The desired output channel layout. It defaults to stereo.
</dd> <dt><samp class=”option”>map</samp></dt> <dd> Map
channels from inputs to output. The argument is a ’|’-separated

list of mappings, each in the <code class=”code”><var class=”var”>input_idx</var>.<var class=”var”>in_channel</var>-<var class=”var”>out_channel</var></code> form. <var class=”var”>input_idx</var> is the 0-based index of the input stream. <var class=”var”>in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the specified input stream. <var class=”var”>out_channel</var> is the name of the output channel. </dd> </dl>

The filter will attempt to guess the mappings when they are not specified explicitly. It does so by first trying to find an unused matching input channel and if that fails it picks the first unused input channel. Join 3 inputs (with properly set channel layouts): <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex join=inputs=3 OUTPUT </pre>

Build a 5.1 output from 6 single-channel streams: <pre class=”example-preformatted”>ffmpeg -i fl -i fr -i fc -i sl -i sr -i lfe -filter_complex 'join=inputs=6:channel_layout=5.1:map=0.0-FL|1.0-FR|2.0-FC|3.0-SL|4.0-SR|5.0-LFE' out </pre>

 <h3 class=”section”>27.96 ladspa# TOC</h3>

Load a LADSPA (Linux Audio Developer’s Simple Plugin API) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-ladspa</code>. <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> Specifies the name of LADSPA plugin library to load. If the environment variable <code class=”env”>LADSPA_PATH</code> is defined, the LADSPA plugin is searched in each one of the directories specified by the colon separated list in <code class=”env”>LADSPA_PATH</code>, otherwise in the standard LADSPA paths, which are in this order: <samp class=”file”>HOME/.ladspa/lib/</samp>, <samp class=”file”>/usr/local/lib/ladspa/</samp>, <samp class=”file”>/usr/lib/ladspa/</samp>. </dd> <dt><samp class=”option”>plugin, p</samp></dt> <dd> Specifies the plugin within the library. Some libraries contain only one plugin, but others contain many of them. If this is not set filter will list all available plugins within the specified library. </dd> <dt><samp class=”option”>controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). Controls need to be defined using the following syntax: c0=<var class=”var”>value0</var>|c1=<var class=”var”>value1</var>|c2=<var class=”var”>value2</var>|..., where <var class=”var”>valuei</var> is the value set on the <var class=”var”>i</var>-th control. Alternatively they can be also defined using the following syntax: <var class=”var”>value0</var>|<var class=”var”>value1</var>|<var class=”var”>value2</var>|...,

where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> <dt><samp class="option">latency, l</samp></dt> <dd> Enable latency compensation, by default is disabled. Only used if plugin have inputs. </dd> </dl>

 <h4 class="subsection">27.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all available plugins within amp (LADSPA example plugin) library: <pre class="example-preformatted">ladspa=file=amp </pre>

List all available controls and their valid ranges for <code class="code">vcf_notch</code> plugin from <code class="code">VCF</code> library: <pre class="example-preformatted">ladspa=f=vcf:p=vcf_notch:c=help </pre>

Simulate low quality audio equipment using <code class="code">Computer Music Toolkit</code> (CMT) plugin library: <pre class="example-preformatted">ladspa=file=cmt:plugin=lofi:controls=c0=22|c1=12|c2=12 </pre>

Add reverberation to the audio using TAP-plugins (Tom’s Audio Processing plugins): <pre class="example-preformatted">ladspa=file=tap_reverb:tap_reverb </pre>

Generate white noise, with 0.2 amplitude: <pre class="example-preformatted">ladspa=file=cmt:noise_source_white:c=c0=.2 </pre>

Generate 20 bpm clicks using plugin <code class="code">C* Click - Metronome</code> from the <code class="code">C* Audio Plugin

Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=file=caps:Click:c=c1=20' </pre>

Apply <code class="code">C* Eq10X2 - Stereo 10-band equaliser</code> effect: <pre class="example-preformatted">ladspa=caps:Eq10X2:c=c0=-48|c9=-24|c3=12|c4=2 </pre>

Increase volume by 20dB using fast lookahead limiter from Steve Harris <code class="code">SWH Plugins</code> collection: <pre class="example-preformatted">ladspa=fast_lookahead_limiter_1913:fastLookahead
</pre>

Attenuate low frequencies using Multiband EQ from Steve Harris <code class="code">SWH Plugins</code> collection: <pre class="example-preformatted">ladspa=mbeq_1197:mbeq:-24|-24|-24|0|0|0|0|0|0|0|0|0|0|0 </pre>

Reduce stereo image using <code class="code">Narrower</code> from the <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:Narrower
</pre>

Another white noise, now using <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:White:.2 </pre>

Some fractal noise, using <code class="code">C* Audio Plugin Suite</code> (CAPS) library: <pre class="example-preformatted">ladspa=caps:Fractal:c=c1=1 </pre>

Dynamic volume normalization using <code class="code">VLevel</code> plugin: <pre class="example-preformatted">ladspa=vlevel-ladspa:vlevel_mono </pre>

 <h4 class="subsection">27.96.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">cN</samp></dt> <dd> Modify the <var class="var">N</var>-th control value. If the specified value is not valid, it is ignored and prior one is kept. </dd> </dl>

 <h3 class="section">27.97 loudnorm# TOC</h3>

EBU R128 loudness normalization. Includes both dynamic and linear normalization modes. Support for both single pass (livestreams, files) and double

pass (files) modes. This algorithm can target IL, LRA, and maximum true peak. In dynamic mode, to accurately detect true peaks, the audio stream will be upsampled to 192 kHz. Use the <code class=”code”>-ar</code> option or <code class=”code”>aresample</code> filter to explicitly set an output sample rate. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>I, i</samp></dt> <dd> Set integrated loudness target. Range is -70.0 - -5.0. Default value is -24.0. </dd> <dt><samp class=”option”>LRA, lra</samp></dt> <dd> Set loudness range target. Range is 1.0 - 50.0. Default value is 7.0. </dd> <dt><samp class=”option”>TP, tp</samp></dt> <dd> Set maximum true peak. Range is -9.0 - +0.0. Default value is -2.0. </dd> <dt><samp class=”option”>measured_I, measured_i</samp></dt> <dd> Measured IL of input file. Range is -99.0 - +0.0. </dd> <dt><samp class=”option”>measured_LRA, measured_lra</samp></dt> <dd> Measured LRA of input file. Range is 0.0 - 99.0. </dd> <dt><samp class=”option”>measured_TP, measured_tp</samp></dt> <dd> Measured true peak of input file. Range is -99.0 - +99.0. </dd> <dt><samp class=”option”>measured_thresh</samp></dt> <dd> Measured threshold of input file. Range is -99.0 - +0.0. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set offset gain. Gain is applied before the true-peak limiter. Range is -99.0 - +99.0. Default is +0.0. </dd> <dt><samp class=”option”>linear</samp></dt> <dd> Normalize by linearly scaling the source audio. <code class=”code”>measured_I</code>, <code class=”code”>measured_LRA</code>, <code class=”code”>measured_TP</code>, and <code class=”code”>measured_thresh</code> must all be specified. Target LRA shouldn’t be lower than source LRA and the change in integrated loudness shouldn’t result in a true peak which exceeds the target TP. If any of these conditions aren’t met, normalization mode will revert to <var class=”var”>dynamic</var>. Options are <code class=”code”>true</code> or <code class=”code”>false</code>. Default is <code class=”code”>true</code>. </dd> <dt><samp class=”option”>dual_mono</samp></dt> <dd> Treat mono input files as "dual-mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. Options are true or false. Default is false. </dd> <dt><samp class=”option”>print_format</samp></dt> <dd> Set print format for stats. Options are summary, json, or none. Default value is none. </dd> <dt><samp class=”option”>stats_file</samp></dt> <dd> Write stats to specified file. Format is controlled by <samp class=”option”>print_format</samp>, which must be set. Specify <code class=”code”>-</code> to write to standard output. Default is unset. </dd> </dl>

 <h3 class=”section”>27.98 lowpass<span

class=”pull-right”># TOC</h3>

Apply a low-pass filter with 3dB point frequency. The filter can be either single-pole or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 500. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero

values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.98.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Lowpass only LFE channel, it LFE is not present it does nothing: <pre class="example-preformatted">lowpass=c=LFE
</pre>

 <h4 class="subsection">27.98.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change lowpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change lowpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change lowpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change lowpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">27.99 lv2# TOC</h3>

Load a LV2 (LADSPA Version 2) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-lv2</code>. <dl class="table"> <dt><samp class="option">plugin, p</samp></dt> <dd> Specifies the plugin URI. You may need to escape ’:’. </dd> <dt><samp class="option">controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if

plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> </dl>

 <h4 class="subsection">27.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply bass enhancer plugin from Calf: <pre class="example-preformatted">lv2=p=http\\\\://calf.sourceforge.net/plugins/
</pre>

Apply vinyl plugin from Calf: <pre class="example-preformatted">lv2=p=http\\\\://calf.sourceforge.net/plugins/Vinyl:c=drone=0.2|aging=0.5
</pre>

Apply bit crusher plugin from ArtyFX: <pre class="example-preformatted">lv2=p=http\\\\://www.openavproductions.com/artyfx#bitta:c=crush=0.3
</pre>

 <h4 class="subsection">27.99.2 Commands# TOC</h4>

This filter supports all options that are exported by plugin as commands. <h3 class="section">27.100 mcompand# TOC</h3> Multiband Compress or expand the audio’s dynamic range. The input audio is divided into bands using 4th order Linkwitz-Riley IIRs. This is akin to the crossover of a loudspeaker, and results in flat frequency response when absent compander action. It accepts the following parameters: <dl class="table"> <dt><samp class="option">args</samp></dt> <dd> This option syntax is: attack,decay,[attack,decay..] soft-knee points crossover_frequency [delay [initial_volume [gain]]] | attack,decay ... For explanation of each item refer to compand filter documentation. </dd> </dl>

 <h3 class="section">27.101 pan<a class="anchor hidden-xs" href="#pan"

aria-hidden=”true”># TOC</h3>

Mix channels with specific gain levels. The filter accepts the output channel layout followed by a set of channels definitions. This filter is also designed to efficiently remap the channels of an audio stream. The filter accepts parameters of the form: "<var class=”var”>l</var>|<var class=”var”>outdef</var>|<var class=”var”>outdef</var>|..." <dl class=”table”> <dt><samp class=”option”>l</samp></dt> <dd> output channel layout or number of channels </dd> <dt><samp class=”option”>outdef</samp></dt> <dd> output channel specification, of the form: "<var class=”var”>out_name</var>=[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>[(+-)[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>...]" </dd> <dt><samp class=”option”>out_name</samp></dt> <dd> output channel to define, either a channel name (FL, FR, etc.) or a channel number (c0, c1, etc.) </dd> <dt><samp class=”option”>gain</samp></dt> <dd> multiplicative coefficient for the channel, 1 leaving the volume unchanged </dd> <dt><samp class=”option”>in_name</samp></dt> <dd> input channel to use, see out_name for details; it is not possible to mix named and numbered input channels </dd> </dl>

If the ‘=’ in a channel specification is replaced by ‘<’, then the gains for that specification will be renormalized so that the total is 1, thus avoiding clipping noise. <h4 class=”subsection”>27.101.1 Mixing examples# TOC</h4>

For example, if you want to down-mix from stereo to mono, but with a bigger factor for the left channel: <pre class=”example-preformatted”>pan=1c|c0=0.9*c0+0.1*c1 </pre>

A customized down-mix to stereo that works automatically for 3-, 4-, 5- and 7-channels surround: <pre class=”example-preformatted”>pan=stereo| FL < FL + 0.5*FC + 0.6*BL + 0.6*SL | FR < FR + 0.5*FC + 0.6*BR + 0.6*SR </pre>

Note that <code class=”command”>ffmpeg</code> integrates a default down-mix (and up-mix) system that should be preferred (see "-ac" option) unless you have very specific needs. <h4 class=”subsection”>27.101.2 Remapping examples# TOC</h4>

The channel remapping will be effective if, and only if: <ul

class=”itemize mark-bullet”> gain coefficients are zeroes or ones, only one input per channel output,

If all these conditions are satisfied, the filter will notify the user ("Pure channel mapping detected"), and use an optimized and lossless method to do the remapping. For example, if you have a 5.1 source and want a stereo audio stream by dropping the extra channels: <pre class=”example-preformatted”>pan="stereo| c0=FL | c1=FR" </pre>

Given the same source, you can also switch front left and front right channels and keep the input channel layout: <pre class=”example-preformatted”>pan="5.1| c0=c1 | c1=c0 | c2=c2 | c3=c3 | c4=c4 | c5=c5" </pre>

If the input is a stereo audio stream, you can mute the front left channel (and still keep the stereo channel layout) with: <pre class=”example-preformatted”>pan="stereo|c1=c1" </pre>

Still with a stereo audio stream input, you can copy the right channel in both front left and right: <pre class=”example-preformatted”>pan="stereo| c0=FR | c1=FR" </pre>

 <h3 class=”section”>27.102 replaygain# TOC</h3>

ReplayGain scanner filter. This filter takes an audio stream as an input and outputs it unchanged. At end of filtering it displays <code class=”code”>track_gain</code> and <code class=”code”>track_peak</code>. The filter accepts the following exported read-only options: <dl class=”table”> <dt><samp class=”option”>track_gain</samp></dt> <dd> Exported track gain in dB at end of stream. </dd> <dt><samp class=”option”>track_peak</samp></dt> <dd> Exported track peak at end of stream. </dd> </dl>

 <h3 class=”section”>27.103 resample# TOC</h3>

Convert the audio sample format, sample rate and channel layout. It is not meant to be used directly. <h3 class=”section”>27.104 rubberband# TOC</h3> Apply time-stretching and pitch-shifting with librubberband. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-librubberband</code>.

The filter accepts the following options: <dl class="table">
<dt><samp class="option">tempo</samp></dt> <dd> Set tempo
scale factor. </dd> <dt><samp class="option">pitch</samp></dt>
<dd> Set pitch scale factor. </dd> <dt><samp class="option">transients</samp></dt>
<dd> Set transients detector. Possible values are: <dl class="table">
<dt><var class="var">crisp</var></dt> <dt><var class="var">mixed</var></dt>
<dt><var class="var">smooth</var></dt> </dl>

</dd> <dt><samp class="option">detector</samp></dt> <dd> Set
detector. Possible values are: <dl class="table"> <dt><var
class="var">compound</var></dt> <dt><var class="var">percussive</var></dt>
<dt><var class="var">soft</var></dt> </dl>

</dd> <dt><samp class="option">phase</samp></dt> <dd> Set
phase. Possible values are: <dl class="table"> <dt><var class="var">laminar</var></dt>
<dt><var class="var">independent</var></dt> </dl>

</dd> <dt><samp class="option">window</samp></dt> <dd> Set
processing window size. Possible values are: <dl class="table">
<dt><var class="var">standard</var></dt> <dt><var class="var">short</var></dt>
<dt><var class="var">long</var></dt> </dl>

</dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set
smoothing. Possible values are: <dl class="table"> <dt><var
class="var">off</var></dt> <dt><var class="var">on</var></dt>
</dl>

</dd> <dt><samp class="option">formant</samp></dt> <dd> Enable
formant preservation when shift pitching. Possible values are: <dl
class="table"> <dt><var class="var">shifted</var></dt> <dt><var
class="var">preserved</var></dt> </dl>

</dd> <dt><samp class="option">pitchq</samp></dt> <dd> Set
pitch quality. Possible values are: <dl class="table"> <dt><var
class="var">quality</var></dt> <dt><var class="var">speed</var></dt>
<dt><var class="var">consistency</var></dt> </dl>

</dd> <dt><samp class="option">channels</samp></dt> <dd> Set
channels. Possible values are: <dl class="table"> <dt><var
class="var">apart</var></dt> <dt><var class="var">together</var></dt>
</dl> </dd> </dl>

 <h4 class="subsection">27.104.1 Com-
mands<a class="anchor hidden-xs" href="#Commands-
44" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Commands-44" aria-hidden="true">TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">tempo</samp></dt> <dd> Change
filter tempo scale factor. Syntax for the command is : "<var

class=”var”>tempo</var>" </dd> <dt><samp class=”option”>pitch</samp></dt> <dd> Change filter pitch scale factor. Syntax for the command is : "<var class=”var”>pitch</var>" </dd> </dl>

 <h3 class=”section”>27.105 sidechaincompress# TOC</h3>

This filter acts like normal compressor but has the ability to compress detected signal using second input signal. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filtered signal then can be filtered with other filters in later stages of processing. See pan and amerge filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> If a signal of second stream raises above this level it will affect the gain reduction of first stream. By default is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class=”option”>ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. 1:2 means that if the level raised 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class=”option”>makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class=”option”>knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class=”option”>link</samp></dt> <dd> Choose if the <code class=”code”>average</code> level between all channels of side-chain stream or the louder(<code class=”code”>maximum</code>) channel of side-chain stream affects the reduction. Default is <code class=”code”>average</code>. </dd> <dt><samp class=”option”>detection</samp></dt> <dd> Should the exact signal be taken in case of <code class=”code”>peak</code> or an RMS one in case of <code class=”code”>rms</code>. Default

is <code class=”code”>rms</code> which is mainly smoother. </dd> <dt><samp class=”option”>level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class=”subsection”>27.105.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>27.105.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Full ffmpeg example taking 2 audio inputs, 1st input to be compressed depending on the signal of 2nd input and later compressed signal to be merged with 2nd input: <pre class=”example-preformatted”>ffmpeg -i main.flac -i sidechain.flac -filter_complex "[1:a]asplit=2[sc][mix];[0:a][sc]sidechaincompress[compr];[compr][mix]amerge" </pre>

 <h3 class=”section”>27.106 sidechaingate# TOC</h3>

A sidechain gate acts like a normal (wideband) gate but has the ability to filter the detected signal before sending it to the gain reduction stage. Normally a gate uses the full range signal to detect a level above the threshold. For example: If you cut all lower frequencies from your sidechain signal the gate will decrease the volume of your track only if not enough highs appear. With this technique you are able to reduce the resonation of a natural drum or remove "rumbling" of muted strokes from a heavily distorted guitar. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode of operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. If set to <code class=”code”>upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class=”code”>downward</code> lower parts of signal will be

reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is rms. Can be peak or rms. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is average. Can be average or maximum. </dd> <dt><samp class="option">level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is from 0.015625 to 64. </dd> </dl>

 <h4 class="subsection">27.106.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">27.107 silencedetect# TOC</h3>

Detect silence in an audio stream. This filter logs a message when it detects that the input audio volume is less or equal to a noise tolerance value for a duration greater or equal to the minimum detected noise duration. The printed times and duration are expressed in seconds. The <code class="code">lavfi.silence_start</code> or <code class="code">lavfi.silence_start.X</code> metadata key is

set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the silence. The <code class="code">lavfi.silence_duration</code> or <code class="code">lavfi.silence_duration.X</code> and <code class="code">lavfi.silence_end</code> or <code class="code">lavfi.silence_end.X</code> metadata keys are set on the first frame after the silence. If <samp class="option">mono</samp> is enabled, and each channel is evaluated separately, the <code class="code">.X</code> suffixed keys are used, and <code class="code">X</code> corresponds to the channel number. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or amplitude ratio. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set silence duration until notification (default is 2 seconds). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. </dd> <dt><samp class="option">mono, m</samp></dt> <dd> Process each channel separately, instead of combined. By default is disabled. </dd> </dl>

 <h4 class="subsection">27.107.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect 5 seconds of silence with -50dB noise tolerance: <pre class="example-preformatted">silencedetect=n=-50dB:d=5 </pre>

Complete example with <code class="command">ffmpeg</code> to detect silence with 0.0001 noise tolerance in <samp class="file">silence.mp3</samp>: <pre class="example-preformatted">ffmpeg -i silence.mp3 -af silencedetect=noise=0.0001 -f null - </pre>

 <h3 class="section">27.108 silenceremove# TOC</h3>

Remove silence from the beginning, middle or end of the audio.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">start_periods</samp></dt>

<dd> This value is used to indicate if audio should be trimmed at beginning of

the audio. A value of zero indicates no silence should be trimmed from the

beginning. When specifying a non-zero value, it trims audio up until it

finds non-silence. Normally, when trimming silence from beginning of audio

the <var class="var">start_periods</var> will be <code class="code">1</code> but it can be increased to higher

values to trim all audio up to specific count of non-silence periods.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_duration</samp></dt>

<dd> Specify the amount of time that non-silence must be detected before it stops

trimming audio. By increasing the duration, bursts of noises can be treated

as silence and trimmed off. Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_threshold</samp></dt>

<dd> This indicates what sample value should be treated as silence. For digital

audio, a value of <code class="code">0</code> may be fine but for audio recorded from analog,

you may wish to increase the value to account for background noise.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">start_silence</samp></dt>

<dd> Specify max duration of silence at beginning that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class="option">start_mode</samp></dt>

<dd> Specify mode of detection of silence end at start of multi-channel audio.

Can be <var class="var">any</var> or <var class="var">all</var>. Default is <var class="var">any</var>.

With <var class="var">any</var>, any sample from any channel that is detected as non-silence

will trigger end of silence trimming at start of audio stream.

With <var class="var">all</var>, only if every sample from every channel is detected as non-silence

will trigger end of silence trimming at start of audio stream, limited usage.

</dd>

<dt><samp class="option">stop_periods</samp></dt>

<dd> Set the count for trimming silence from the end of audio. When specifying a

positive value, it trims audio after it finds specified silence period.

To remove silence from the middle of a file, specify a <var class="var">stop_periods</var>

that is negative. This value is then treated as a positive value and is

used to indicate the effect should restart processing as specified by

<var class="var">stop_periods</var>, making it suitable for removing periods of silence

in the middle of the audio.

Default value is <code class="code">0</code>.

</dd>

<dt><samp class="option">stop_duration</samp></dt>

<dd> Specify a duration of silence that must exist before audio is not copied any

more. By specifying a higher duration, silence that is wanted can be left in

the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_threshold</samp></dt>

<dd> This is the same as <samp class=”option”>start_threshold</samp> but for trimming silence from

the end of audio.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_silence</samp></dt>

<dd> Specify max duration of silence at end that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>stop_mode</samp></dt>

<dd> Specify mode of detection of silence start after start of multi-channel audio.

Can be <var class=”var”>any</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>.

With <var class=”var”>any</var>, any sample from any channel that is detected as silence

will trigger start of silence trimming after start of audio stream, limited usage.

With <var class=”var”>all</var>, only if every sample from every channel is detected as silence

will trigger start of silence trimming after start of audio stream.

</dd>

<dt><samp class=”option”>detection</samp></dt>

<dd> Set how is silence detected.

<dl class="table">

<dt><samp class="option">avg</samp></dt>

<dd> Mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">rms</samp></dt>

<dd> Root squared mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">peak</samp></dt>

<dd> Maximum of absolute values of samples in moving window.

</dd>

<dt><samp class="option">median</samp></dt>

<dd> Median of absolute values of samples in moving window.

</dd>

<dt><samp class="option">ptp</samp></dt>

<dd> Absolute of max peak to min peak difference of samples in moving window.

</dd>

<dt><samp class="option">dev</samp></dt>

<dd> Standard deviation of values of samples in moving window.

</dd>

</dl>

Default value is <code class="code">rms</code>.

</dd>

<dt><samp class="option">window</samp></dt>

<dd> Set duration in number of seconds used to calculate size of window in number

of samples for detecting silence. Using <code class="code">0</code> will effectively disable

any windowing and use only single sample per channel for silence detection.

In that case it may be needed to also set <samp class="option">start_silence</samp> and/or

<samp class="option">stop_silence</samp> to nonzero values with also <samp class="option">start_duration</samp> and/or

<samp class="option">stop_duration</samp> to nonzero values.

Default value is <code class="code">0.02</code>. Allowed range is from <code class="code">0</code> to <code class="code">10</code>.

</dd>

<dt><samp class="option">timestamp</samp></dt>

<dd> Set processing mode of every audio frame output timestamp.

<dl class="table">

<dt><samp class="option">write</samp></dt>

<dd> Full timestamps rewrite, keep only the start time for the first output frame.

</dd>

<dt><samp class="option">copy</samp></dt>

<dd> Non-dropped frames are left with same timestamp as input audio frame.

</dd>

</dl>

Defaults value is <code class="code">write</code>.

</dd>

</dl>

 <h4 class="subsection">27.108.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> The following example shows how this filter can be used to start a recording that does not contain the delay at the start which usually occurs between pressing the record button and the start of the performance: <pre class="example-preformatted">silenceremove=start_periods=1:start_duration=5:start_threshold=0.02
</pre>

Trim all silence encountered from beginning to end where there is more than 1 second of silence in audio: <pre class="example-preformatted">silenceremove=stop_periods=-1:stop_duration=1:stop_threshold=-90dB </pre>

Trim all digital silence samples, using peak detection, from beginning to end where there is more than 0 samples of digital silence in audio and digital silence is detected in all channels at same positions in stream: <pre class=”example-preformatted”>silenceremove=window=0:detection=peak:stop_mode=all:start_mode=all:stop_periods=-1:stop_threshold=0 </pre>

Trim every 2nd encountered silence period from beginning to end where there is more than 1 second of silence per silence period in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB </pre>

Similar as above, but keep maximum of 0.5 seconds of silence from each trimmed period: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5 </pre>

Similar as above, but keep maximum of 1.5 seconds of silence from start of audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5:start_periods=1:start_duration=1:start_silence=1.5:stop_threshold=-90dB </pre>

 <h4 class=”subsection”>27.108.2 Commands# TOC</h4>

This filter supports some above options as commands. <h3 class=”section”>27.109 sofalizer# TOC</h3>

SOFAlizer uses head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones (audio formats up to 9 channels supported). The HRTFs are stored in SOFA files (see http://www.sofacoustics.org/ for a database). SOFAlizer is developed at the Acoustics Research Institute (ARI) of the Austrian Academy of Sciences. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libmysofa</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sofa</samp></dt> <dd> Set the SOFA file used for rendering. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set rotation of virtual loudspeakers in deg. Default is 0. </dd> <dt><samp class=”option”>elevation</samp></dt> <dd> Set eleva-

tion of virtual speakers in deg. Default is 0. </dd> <dt><samp class="option">radius</samp></dt> <dd> Set distance in meters between loudspeakers and the listener with near-field HRTFs. Default is 1. </dd> <dt><samp class="option">type</samp></dt> <dd> Set processing type. Can be <var class="var">time</var> or <var class="var">freq</var>. <var class="var">time</var> is processing audio in time domain which is slow. <var class="var">freq</var> is processing audio in frequency domain which is fast. Default is <var class="var">freq</var>. </dd> <dt><samp class="option">speakers</samp></dt> <dd> Set custom positions of virtual loudspeakers. Syntax for this option is: <CH> <AZIM> <ELEV>[|<CH> <AZIM> <ELEV>|...]. Each virtual loudspeaker is described with short channel name following with azimuth and elevation in degrees. Each virtual loudspeaker description is separated by ’|’. For example to override front left and front right channel positions use: ’speakers=FL 45 15|FR 345 15’. Descriptions with unrecognised channel names are ignored. </dd> <dt><samp class="option">lfegain</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class="option">framesize</samp></dt> <dd> Set custom frame size in number of samples. Default is 1024. Allowed range is from 1024 to 96000. Only used if option ‘<samp class="samp">type</samp>’ is set to <var class="var">freq</var>. </dd> <dt><samp class="option">normalize</samp></dt> <dd> Should all IRs be normalized upon importing SOFA file. By default is enabled. </dd> <dt><samp class="option">interpolate</samp></dt> <dd> Should nearest IRs be interpolated with neighbor IRs if exact position does not match. By default is disabled. </dd> <dt><samp class="option">minphase</samp></dt> <dd> Minphase all IRs upon loading of SOFA file. By default is disabled. </dd> <dt><samp class="option">anglestep</samp></dt> <dd> Set neighbor search angle step. Only used if option <var class="var">interpolate</var> is enabled. </dd> <dt><samp class="option">radstep</samp></dt> <dd> Set neighbor search radius step. Only used if option <var class="var">interpolate</var> is enabled. </dd> </dl>

 <h4 class="subsection">27.109.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Using ClubFritz6 sofa file: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz6.sofa:type=freq:r
</pre>

Using ClubFritz12 sofa file and bigger radius with small rotation: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz12.sofa:type=
</pre>

Similar as above but with custom speaker positions for front left, front right, back left and back right and also with custom gain: <pre class=”example-preformatted”>"sofalizer=sofa=/path/to/ClubFritz6.sofa:type=45|FR 315|BL 135|BR 225:gain=28" </pre>

 <h3 class=”section”>27.110 speechnorm# TOC</h3> Speech Normalizer. This filter expands or compresses each half-cycle of audio samples (local set of samples all above or all below zero and between two nearest zero crossings) depending on threshold value, so audio reaches target peak value under conditions controlled by below options. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>peak, p</samp></dt> <dd> Set the expansion target peak value. This specifies the highest allowed absolute amplitude level for the normalized audio input. Default value is 0.95. Allowed range is from 0.0 to 1.0. </dd> <dt><samp class=”option”>expansion, e</samp></dt> <dd> Set the maximum expansion factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples expansion. The maximum expansion would be such that local peak value reaches target peak value but never to surpass it and that ratio between new and previous peak value does not surpass this option value. </dd> <dt><samp class=”option”>compression, c</samp></dt> <dd> Set the maximum compression factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples compression. This option is used only if <samp class=”option”>threshold</samp> option is set to value greater than 0.0, then in such cases when local peak is lower or same as value set by <samp class=”option”>threshold</samp> all samples belonging to that peak’s half-cycle will be compressed by current compression factor. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set the threshold value. Default value is 0.0. Allowed range is from 0.0 to 1.0. This option specifies which half-cycles of samples will be compressed and which will be expanded. Any half-cycle samples with their local peak value below or same as this option value will be compressed by current compression factor, otherwise, if greater than threshold value they will be expanded with expansion factor so that it could reach peak target value but never surpass it. </dd> <dt><samp class=”option”>raise, r</samp></dt> <dd> Set the expansion raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast expansion factor is raised per each new half-cycle until it reaches <samp class=”option”>expansion</samp> value. Setting this options too high may lead to distortions. </dd> <dt><samp class=”option”>fall, f</samp></dt> <dd> Set the compression raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast compression factor is raised per each

new half-cycle until it reaches <samp class="option">compression</samp> value. </dd> <dt><samp class="option">channels, h</samp></dt> <dd> Specify which channels to filter, by default all available channels are filtered. </dd> <dt><samp class="option">invert, i</samp></dt> <dd> Enable inverted filtering, by default is disabled. This inverts interpretation of <samp class="option">threshold</samp> option. When enabled any half-cycle of samples with their local peak value below or same as <samp class="option">threshold</samp> option will be expanded otherwise it will be compressed. </dd> <dt><samp class="option">link, l</samp></dt> <dd> Link channels when calculating gain applied to each filtered channel sample, by default is disabled. When disabled each filtered channel gain calculation is independent, otherwise when this option is enabled the minimum of all possible gains for each filtered channel is used. </dd> <dt><samp class="option">rms, m</samp></dt> <dd> Set the expansion target RMS value. This specifies the highest allowed RMS level for the normalized audio input. Default value is 0.0, thus disabled. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">27.110.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">27.110.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Weak and slow amplification: <pre class="example-preformatted">speechnorm=e=3:r=0.00001:l=1 </pre>

Moderate and slow amplification: <pre class="example-preformatted">speechnorm=e=6.25:r=0.00001:l=1 </pre>

Strong and fast amplification: <pre class="example-preformatted">speechnorm=e=12.5:r=0.0001:l=1 </pre>

Very strong and fast amplification: <pre class="example-preformatted">speechnorm=e=25:r=0.0001:l=1 </pre>

Extreme and fast amplification: <pre class="example-preformatted">speechnorm=e=50:r=0.0001:l=1 </pre>

 <h3 class="section">27.111 stereotools<a class="anchor hidden-xs" href="#stereotools" aria-

hidden=”true”># TOC</h3>

This filter has some handy utilities to manage stereo signals, for converting M/S stereo recordings to L/R signal while having control over the parameters or spreading the stereo image of master track. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>balance_in</samp></dt> <dd> Set input balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>balance_out</samp></dt> <dd> Set output balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>softclip</samp></dt> <dd> Enable softclipping. Results in analog distortion instead of harsh digital 0dB clipping. Disabled by default. </dd> <dt><samp class=”option”>mutel</samp></dt> <dd> Mute the left channel. Disabled by default. </dd> <dt><samp class=”option”>muter</samp></dt> <dd> Mute the right channel. Disabled by default. </dd> <dt><samp class=”option”>phasel</samp></dt> <dd> Change the phase of the left channel. Disabled by default. </dd> <dt><samp class=”option”>phaser</samp></dt> <dd> Change the phase of the right channel. Disabled by default. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set stereo mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lr>lr</samp>’</dt> <dd> Left/Right to Left/Right, this is default. </dd> <dt>‘<samp class=”samp”>lr>ms</samp>’</dt> <dd> Left/Right to Mid/Side. </dd> <dt>‘<samp class=”samp”>ms>lr</samp>’</dt> <dd> Mid/Side to Left/Right. </dd> <dt>‘<samp class=”samp”>lr>ll</samp>’</dt> <dd> Left/Right to Left/Left. </dd> <dt>‘<samp class=”samp”>lr>rr</samp>’</dt> <dd> Left/Right to Right/Right. </dd> <dt>‘<samp class=”samp”>lr>l+r</samp>’</dt> <dd> Left/Right to Left + Right. </dd> <dt>‘<samp class=”samp”>lr>rl</samp>’</dt> <dd> Left/Right to Right/Left. </dd> <dt>‘<samp class=”samp”>ms>ll</samp>’</dt> <dd> Mid/Side to Left/Left. </dd> <dt>‘<samp class=”samp”>ms>rr</samp>’</dt> <dd> Mid/Side to Right/Right. </dd> <dt>‘<samp class=”samp”>ms>rl</samp>’</dt> <dd> Mid/Side to Right/Left. </dd> <dt>‘<samp class=”samp”>lr>l-r</samp>’</dt> <dd> Left/Right to Left - Right. </dd> </dl>

</dd> <dt><samp class=”option”>slev</samp></dt> <dd> Set level

of side signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">sbal</samp></dt> <dd> Set balance of side signal. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">mlev</samp></dt> <dd> Set level of the middle signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mpan</samp></dt> <dd> Set middle signal pan. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">base</samp></dt> <dd> Set stereo base between mono and inversed channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay in milliseconds how much to delay left from right channel and vice versa. Default is 0. Allowed range is from -20 to 20. </dd> <dt><samp class="option">sclevel</samp></dt> <dd> Set S/C level. Default is 1. Allowed range is from 1 to 100. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set the stereo phase in degrees. Default is 0. Allowed range is from 0 to 360. </dd> <dt><samp class="option">bmode_in, bmode_out</samp></dt> <dd> Set balance mode for balance_in/balance_out option. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">balance</samp>’</dt> <dd> Classic balance mode. Attenuate one channel at time. Gain is raised up to 1. </dd> <dt>‘<samp class="samp">amplitude</samp>’</dt> <dd> Similar as classic mode above but gain is raised up to 2. </dd> <dt>‘<samp class="samp">power</samp>’</dt> <dd> Equal power distribution, from -6dB to +6dB range. </dd> </dl> </dd> </dl>

 <h4 class="subsection">27.111.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">27.111.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply karaoke like effect: <pre class="example-preformatted">stereotools=mlev=0.015625 </pre>

Convert M/S signal to L/R: <pre class="example-preformatted">"stereotools=mode=ms>lr" </pre>

 <h3 class="section">27.112 stereowiden<a class="anchor hidden-xs" href="#stereowiden" aria-

hidden=”true”># TOC</h3>

This filter enhance the stereo effect by suppressing signal common to both channels and by delaying the signal of left into right and vice versa, thereby widening the stereo effect. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Time in milliseconds of the delay of left signal into right and vice versa. Default is 20 milliseconds. </dd> <dt><samp class=”option”>feedback</samp></dt> <dd> Amount of gain in delayed signal into right and vice versa. Gives a delay effect of left signal in right output and vice versa which gives widening effect. Default is 0.3. </dd> <dt><samp class=”option”>crossfeed</samp></dt> <dd> Cross feed of left into right with inverted phase. This helps in suppressing the mono. If the value is 1 it will cancel all the signal common to both channels. Default is 0.3. </dd> <dt><samp class=”option”>drymix</samp></dt> <dd> Set level of input signal of original channel. Default is 0.8. </dd> </dl>

 <h4 class=”subsection”>27.112.1 Commands# TOC</h4>

This filter supports the all above options except <code class=”code”>delay</code> as commands. <h3 class=”section”>27.113 superequalizer# TOC</h3> Apply 18 band equalizer. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>1b</samp></dt> <dd> Set 65Hz band gain. </dd> <dt><samp class=”option”>2b</samp></dt> <dd> Set 92Hz band gain. </dd> <dt><samp class=”option”>3b</samp></dt> <dd> Set 131Hz band gain. </dd> <dt><samp class=”option”>4b</samp></dt> <dd> Set 185Hz band gain. </dd> <dt><samp class=”option”>5b</samp></dt> <dd> Set 262Hz band gain. </dd> <dt><samp class=”option”>6b</samp></dt> <dd> Set 370Hz band gain. </dd> <dt><samp class=”option”>7b</samp></dt> <dd> Set 523Hz band gain. </dd> <dt><samp class=”option”>8b</samp></dt> <dd> Set 740Hz band gain. </dd> <dt><samp class=”option”>9b</samp></dt> <dd> Set 1047Hz band gain. </dd> <dt><samp class=”option”>10b</samp></dt> <dd> Set 1480Hz band gain. </dd> <dt><samp class=”option”>11b</samp></dt> <dd> Set 2093Hz band gain. </dd> <dt><samp class=”option”>12b</samp></dt> <dd> Set 2960Hz band gain. </dd> <dt><samp class=”option”>13b</samp></dt> <dd> Set 4186Hz band gain. </dd> <dt><samp class=”option”>14b</samp></dt> <dd> Set 5920Hz band gain. </dd> <dt><samp class=”option”>15b</samp></dt> <dd> Set 8372Hz band gain. </dd> <dt><samp class=”option”>16b</samp></dt>

<dd> Set 11840Hz band gain. </dd> <dt><samp class="option">17b</samp></dt>
<dd> Set 16744Hz band gain. </dd> <dt><samp class="option">18b</samp></dt>
<dd> Set 20000Hz band gain. </dd> </dl>

<h3 class="section">27.114 surround# TOC</h3>

Apply audio surround upmix filter.

This filter allows to produce multichannel output from audio stream.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">chl_out</samp></dt>

<dd> Set output channel layout. By default, this is <var class="var">5.1</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">chl_in</samp></dt>

<dd> Set input channel layout. By default, this is <var class="var">stereo</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">level_in</samp></dt>

<dd> Set input volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">level_out</samp></dt>

<dd> Set output volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe</samp></dt>

<dd> Enable LFE channel output if output channel layout has it. By default, this is enabled.

</dd>

<dt><samp class="option">lfe_low</samp></dt>

<dd> Set LFE low cut off frequency. By default, this is <var class="var">128</var> Hz.

</dd>

<dt><samp class="option">lfe_high</samp></dt>

<dd> Set LFE high cut off frequency. By default, this is <var class="var">256</var> Hz.

</dd>

<dt><samp class="option">lfe_mode</samp></dt>

<dd> Set LFE mode, can be <var class="var">add</var> or <var class="var">sub</var>. Default is <var class="var">add</var>.

In <var class="var">add</var> mode, LFE channel is created from input audio and added to output.

In <var class="var">sub</var> mode, LFE channel is created from input audio and added to output but

also all non-LFE output channels are subtracted with output LFE channel.

</dd>

<dt><samp class="option">smooth</samp></dt>

<dd> Set temporal smoothness strength, used to gradually change factors when transforming

stereo sound in time. Allowed range is from <var class="var">0.0</var> to <var class="var">1.0</var>.

Useful to improve output quality with <var class="var">focus</var> option values greater than <var class="var">0.0</var>.

Default is <var class="var">0.0</var>. Only values inside this range and without edges are effective.

</dd>

<dt><samp class="option">angle</samp></dt>

<dd> Set angle of stereo surround transform, Allowed range is from <var class="var">0</var> to <var class="var">360</var>.

Default is <var class="var">90</var>.

</dd>

<dt><samp class="option">focus</samp></dt>

<dd> Set focus of stereo surround transform, Allowed range is from <var class="var">-1</var> to <var class="var">1</var>.

Default is <var class="var">0</var>.

</dd>

<dt><samp class="option">fc_in</samp></dt>

<dd> Set front center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fc_out</samp></dt>

<dd> Set front center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_in</samp></dt>

<dd> Set front left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_out</samp></dt>

<dd> Set front left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_in</samp></dt>

<dd> Set front right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_out</samp></dt>

<dd> Set front right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_in</samp></dt>

<dd> Set side left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_out</samp></dt>

<dd> Set side left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_in</samp></dt>

<dd> Set side right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sr_out</samp></dt>

<dd> Set side right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_in</samp></dt>

<dd> Set back left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bl_out</samp></dt>

<dd> Set back left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_in</samp></dt>

<dd> Set back right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">br_out</samp></dt>

<dd> Set back right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_in</samp></dt>

<dd> Set back center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_out</samp></dt>

<dd> Set back center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_in</samp></dt>

<dd> Set LFE input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_out</samp></dt>

<dd> Set LFE output volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">allx</samp></dt>
<dd> Set spread usage of stereo image across X axis for all channels.
Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.
By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>
<dt><samp class="option">ally</samp></dt>
<dd> Set spread usage of stereo image across Y axis for all channels.
Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.
By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>
<dt><samp class="option">fcx, flx, frx, blx, brx, slx, srx, bcx</samp></dt>
<dd> Set spread usage of stereo image across X axis for each channel.
Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.
By default this value is <var class="var">0.5</var>.

</dd>
<dt><samp class="option">fcy, fly, fry, bly, bry, sly, sry, bcy</samp></dt>
<dd> Set spread usage of stereo image across Y axis for each channel.
Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.
By default this value is <var class="var">0.5</var>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from <var class="var">1024</var>
to <var class="var">65536</var>. Default size is <var class="var">4096</var>.

</dd>

<dt><samp class="option">win_func</samp></dt>
<dd> Set window function.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt>
<dt>‘<samp class="samp">bartlett</samp>’</dt>
<dt>‘<samp class="samp">hann, hanning</samp>’</dt>
<dt>‘<samp class="samp">hamming</samp>’</dt>
<dt>‘<samp class="samp">blackman</samp>’</dt>
<dt>‘<samp class="samp">welch</samp>’</dt>
<dt>‘<samp class="samp">flattop</samp>’</dt>
<dt>‘<samp class="samp">bharris</samp>’</dt>
<dt>‘<samp class="samp">bnuttall</samp>’</dt>
<dt>‘<samp class="samp">bhann</samp>’</dt>
<dt>‘<samp class="samp">sine</samp>’</dt>
<dt>‘<samp class="samp">nuttall</samp>’</dt>
<dt>‘<samp class="samp">lanczos</samp>’</dt>
<dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">tukey</samp>’</dt>
<dt>‘<samp class="samp">dolph</samp>’</dt>
<dt>‘<samp class="samp">cauchy</samp>’</dt>
<dt>‘<samp class="samp">parzen</samp>’</dt>
<dt>‘<samp class="samp">poisson</samp>’</dt>
<dt>‘<samp class="samp">bohman</samp>’</dt>
<dt>‘<samp class="samp">kaiser</samp>’</dt>
</dl>
Default is <code class="code">hann</code>.

</dd>
<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class=”code”>0.5</code>.

</dd>

</dl>

 <h3 class=”section”>27.115 tiltshelf# TOC</h3>

Boost or cut the lower frequencies and cut or boost higher frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class=”code”>3000</code> Hz. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">27.115.1 Commands# TOC</h4>

This filter supports some options as commands. <h3 class="section">27.116 treble, highshelf# TOC</h3>

Boost or cut treble (upper) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at whichever is the lower of ~22 kHz and the Nyquist frequency. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">3000</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp

class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt> <dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt> <dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt> <dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>27.116.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Change treble frequency. Syntax for the command is : "<var class=”var”>frequency</var>" </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Change treble width_type. Syntax for the command is : "<var class=”var”>width_type</var>" </dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Change treble width. Syntax for the command is : "<var class=”var”>width</var>" </dd> <dt><samp class=”option”>gain, g</samp></dt> <dd> Change treble gain. Syntax for the command is : "<var class=”var”>gain</var>" </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change treble mix. Syntax for the command is : "<var class=”var”>mix</var>"

</dd> </dl>

 <h3 class="section">27.117 tremolo# TOC</h3>

Sinusoidal amplitude modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Modulation frequencies in the subharmonic range (20 Hz or lower) will result in a tremolo effect. This filter may also be used as a ring modulator by specifying a modulation frequency higher than 20 Hz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">27.118 vibrato# TOC</h3>

Sinusoidal phase modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">27.119 virtualbass# TOC</h3>

Apply audio Virtual Bass filter. This filter accepts stereo input and produce stereo with LFE (2.1) channels output. The newly produced LFE channel have enhanced virtual bass originally obtained from both stereo channels. This filter outputs front left and front right channels unchanged as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set the virtual bass cutoff frequency. Default value is 250 Hz. Allowed range is from 100 to 500 Hz. </dd> <dt><samp class="option">strength</samp></dt> <dd> Set the virtual bass strength. Allowed range is from 0.5 to 3. Default value is 3. </dd> </dl>

 <h3 class="section">27.120 volume# <a class="anchor hidden-xs"href="#toc-volume"

aria-hidden=”true”>TOC</h3>

Adjust the input audio volume. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>volume</samp></dt> <dd> Set audio volume expression. Output values are clipped to the maximum value. The output audio volume is given by the relation: <pre class=”example-preformatted”><var class=”var”>output_volume</var> = <var class=”var”>volume</var> * <var class=”var”>input_volume</var> </pre>

The default value for <var class=”var”>volume</var> is "1.0". </dd> <dt><samp class=”option”>precision</samp></dt> <dd> This parameter represents the mathematical precision. It determines which input sample formats will be allowed, which affects the precision of the volume scaling. <dl class=”table”> <dt><samp class=”option”>fixed</samp></dt> <dd> 8-bit fixed-point; this limits input sample format to U8, S16, and S32. </dd> <dt><samp class=”option”>float</samp></dt> <dd> 32-bit floating-point; this limits input sample format to FLT. (default) </dd> <dt><samp class=”option”>double</samp></dt> <dd> 64-bit floating-point; this limits input sample format to DBL. </dd> </dl>

</dd> <dt><samp class=”option”>replaygain</samp></dt> <dd> Choose the behaviour on encountering ReplayGain side data in input frames. <dl class=”table”> <dt><samp class=”option”>drop</samp></dt> <dd> Remove ReplayGain side data, ignoring its contents (the default). </dd> <dt><samp class=”option”>ignore</samp></dt> <dd> Ignore ReplayGain side data, but leave it in the frame. </dd> <dt><samp class=”option”>track</samp></dt> <dd> Prefer the track gain, if present. </dd> <dt><samp class=”option”>album</samp></dt> <dd> Prefer the album gain, if present. </dd> </dl>

</dd> <dt><samp class=”option”>replaygain_preamp</samp></dt> <dd> Pre-amplification gain in dB to apply to the selected replaygain gain. Default value for <var class=”var”>replaygain_preamp</var> is 0.0. </dd> <dt><samp class=”option”>replaygain_noclip</samp></dt> <dd> Prevent clipping by limiting the gain applied. Default value for <var class=”var”>replaygain_noclip</var> is 1. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the volume expression is evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>once</samp>’</dt> <dd> only evaluate expression once during the filter initialization, or when the ‘<samp class=”samp”>volume</samp>’ command is sent </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expression for each incoming frame </dd> </dl>

Default value is ‘<samp class=”samp”>once</samp>’. </dd> </dl>

The volume expression can contain the following parameters. <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> frame number (starting at zero) </dd> <dt><samp class="option">nb_channels</samp></dt> <dd> number of channels </dd> <dt><samp class="option">nb_consumed_samples</samp></dt> <dd> number of samples consumed by the filter </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> number of samples in the current frame </dd> <dt><samp class="option">pos</samp></dt> <dd> original frame position in the file; deprecated, do not use </dd> <dt><samp class="option">pts</samp></dt> <dd> frame PTS </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> sample rate </dd> <dt><samp class="option">startpts</samp></dt> <dd> PTS at start of stream </dd> <dt><samp class="option">startt</samp></dt> <dd> time at start of stream </dd> <dt><samp class="option">t</samp></dt> <dd> frame time </dd> <dt><samp class="option">tb</samp></dt> <dd> timestamp timebase </dd> <dt><samp class="option">volume</samp></dt> <dd> last set volume value </dd> </dl>

Note that when <samp class="option">eval</samp> is set to ‘<samp class="samp">once</samp>’ only the <var class="var">sample_rate</var> and <var class="var">tb</var> variables are available, all other variables will evaluate to NAN. <h4 class="subsection">27.120.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">volume</samp></dt> <dd> Modify the volume expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class="subsection">27.120.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Halve the input audio volume: <pre class="example-preformatted">volume=volume=0.5 volume=volume=1/2 volume=volume=-6.0206dB </pre>

In all the above example the named key for <samp class="option">volume</samp> can be omitted, for example like in: <pre class="example-preformatted">volume=0.5 </pre>

Increase input audio power by 6 decibels using fixed-point precision: <pre class="example-preformatted">volume=volume=6dB:precision=fixed </pre>

Fade volume after time 10 with an annihilation period of 5 seconds:

 <pre class=”example-preformatted”>volume='if(lt(t,10),1,max(1-(t-10)/5,0))':eval=frame </pre>

 <h3 class=”section”>27.121 volumedetect# TOC</h3>

Detect the volume of the input video. The filter has no parameters. It supports only 16-bit signed integer samples, so the input will be converted when needed. Statistics about the volume will be printed in the log when the input stream end is reached. In particular it will show the mean volume (root mean square), maximum volume (on a per-sample basis), and the beginning of a histogram of the registered volume values (from the maximum value to a cumulated 1/1000 of the samples). All volumes are in decibels relative to the maximum PCM value. <h4 class=”subsection”>27.121.1 Examples# TOC</h4>

Here is an excerpt of the output: <pre class=”example-preformatted”>[Parsed_volumedetect_0 0xa23120] mean_volume: -27 dB [Parsed_volumedetect_0 0xa23120] max_volume: -4 dB [Parsed_volumedetect_0 0xa23120] histogram_4db: 6 [Parsed_volumedetect_0 0xa23120] histogram_5db: 62 [Parsed_volumedetect_0 0xa23120] histogram_6db: 286 [Parsed_volumedetect_0 0xa23120] histogram_7db: 1042 [Parsed_volumedetect_0 0xa23120] histogram_8db: 2551 [Parsed_volumedetect_0 0xa23120] histogram_9db: 4609 [Parsed_volumedetect_0 0xa23120] histogram_10db: 8409 </pre>

It means that: <ul class=”itemize mark-bullet”> The mean square energy is approximately -27 dB, or 10^-2.7. The largest sample is at -4 dB, or more precisely between -4 dB and -5 dB. There are 6 samples at -4 dB, 62 at -5 dB, 286 at -6 dB, etc.

In other words, raising the volume by +4 dB does not cause any clipping, raising it by +5 dB causes clipping for 6 samples, etc. <h3 class=”section”>27.122 whisper# TOC</h3>

It runs automatic speech recognition using the OpenAI’s Whisper model. It requires the whisper.cpp library (https://github.com/ggml-org/whisper.cpp) as a prerequisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-whisper</code>. The filter has following options: <dl class=”table”>

<dt><samp class=”option”>model</samp></dt> <dd> The file path of the downloaded whisper.cpp model (mandatory). </dd> <dt><samp class=”option”>language</samp></dt> <dd> The language to use for transcription. Besides a language code, the following special values enable auto-detection: <dl class=”table”> <dt>‘<samp class=”samp”>eval</samp>’</dt> <dd> Re-run language detection on every queued chunk. This costs an extra encoder pass per chunk and the detected language may change mid-file. </dd> <dt>‘<samp class=”samp”>lock</samp>’</dt> <dd> Detect the language on the first chunk that produces transcribed text, then reuse it for the remaining chunks. </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Currently an alias for ‘<samp class=”samp”>eval</samp>’. This may change in a future version, so use ‘<samp class=”samp”>eval</samp>’ or ‘<samp class=”samp”>lock</samp>’ if the behavior matters. </dd> </dl>

Default value: <code class=”code”>"auto"</code> </dd> <dt><samp class=”option”>translate</samp></dt> <dd> If enabled, translate the transcription from the source language to English. A multilingual model is required to enable this option. Default value: <code class=”code”>"false"</code> </dd> <dt><samp class=”option”>queue</samp></dt> <dd> The maximum size that will be queued into the filter before processing the audio with whisper. Using a small value the audio stream will be processed more often, but the transcription quality will be lower and the required processing power will be higher. Using a large value (e.g. 10-20s) will produce more accurate results using less CPU (as using the whisper-cli tool), but the transcription latency will be higher, thus not useful to process real-time streams. Consider using the vad_model option associated with a large queue value. Default value: <code class=”code”>"3"</code> </dd> <dt><samp class=”option”>use_gpu</samp></dt> <dd> If the GPU support should be enabled. Default value: <code class=”code”>"true"</code> </dd> <dt><samp class=”option”>gpu_device</samp></dt> <dd> The GPU device index to use. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>destination</samp></dt> <dd> If set, the transcription output will be sent to the specified file or URL (use one of the FFmpeg AVIO protocols); otherwise, the output will be logged as info messages. The output will also be set in the "lavfi.whisper.text" frame metadata. If the destination is a file and it already exists, it will be overwritten. </dd> <dt><samp class=”option”>format</samp></dt> <dd> The destination format string; it could be "text" (only the transcribed text will be sent to the destination), "srt" (subtitle format) or "json". JSON segments include a <code class=”code”>language</code> field con-

taining the language code reported by whisper.cpp for the transcription chunk, or <code class=”code”>unknown</code> if none is available. All segments from the same chunk have the same language. Default value: <code class=”code”>"text"</code> </dd> <dt><samp class=”option”>max_len</samp></dt> <dd> Maximum segment length in characters. When set to a value greater than 0, transcription segments will be split by word to not exceed this length. This is useful for generating subtitles with shorter lines. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>vad_model</samp></dt> <dd> Path to the VAD model file. If set, the filter will load an additional voice activity detection module (https://github.com/snakers4/silero-vad) that will be used to fragment the audio queue; use this option setting a valid path obtained from the whisper.cpp repository (e.g. "../whisper.cpp/models/ggml-silero-v5.1.2.bin") and increase the queue parameter to a higher value (e.g. 20). </dd> <dt><samp class=”option”>vad_threshold</samp></dt> <dd> The VAD threshold to use. Default value: <code class=”code”>"0.5"</code> </dd> <dt><samp class=”option”>vad_min_speech_duration</samp></dt> <dd> The minimum VAD speaking duration. Default value: <code class=”code”>"0.1"</code> </dd> <dt><samp class=”option”>vad_min_silence_duration</samp></dt> <dd> The minimum VAD silence duration. Default value: <code class=”code”>"0.5"</code> </dd> </dl>

 <h4 class=”subsection”>27.122.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Run a transcription with srt file generation: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=output.srt\ :format=srt" -f null - </pre>

Run a transcription and send the output in JSON format to an HTTP service: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=http\\://localhost\\:3000\ :format=json' -f null - </pre>

Transcribe the microphone input using the VAD option: <pre class=”example-preformatted”>ffmpeg -loglevel warning -f pulse -i default \ -af 'highpass=f=200,lowpass=f=3000,whisper=model=../whisper.cpp/models/ggml-medium.bin\ :language=en\ :queue=10\ :destination=-\ :format=json\ :vad_model=../whisper.cpp/models/ggml-silero-v5.1.2.bin' -f null - </pre>

 <h2 class="chapter">28 Audio Sources# TOC</h2>

Below is a description of the currently available audio sources. <h3 class="section">28.1 abuffer# TOC</h3>

Buffer audio frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">time_base</samp></dt> <dd> The time-base which will be used for timestamps of submitted frames. It must be either a floating-point number or in <var class="var">numerator</var>/<var class="var">denominator</var> form. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> The sample rate of the incoming audio buffers. </dd> <dt><samp class="option">sample_fmt</samp></dt> <dd> The sample format of the incoming audio buffers. Either a sample format name or its corresponding integer representation from the enum AVSample-Format in <samp class="file">libavutil/samplefmt.h</samp> </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the incoming audio buffers. Either a channel layout name from channel_layout_map in <samp class="file">libavutil/channel_layout.c</samp> or its corresponding integer representation from the AV_CH_LAYOUT_* macros in <samp class="file">libavutil/channel_layout.h</samp> </dd> <dt><samp class="option">channels</samp></dt> <dd> The number of channels of the incoming audio buffers. If both <var class="var">channels</var> and <var class="var">channel_layout</var> are specified, then they must be consistent. </dd> </dl>

 <h4 class="subsection">28.1.1 Examples# TOC</h4>

 <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=s16p
</pre>

will instruct the source to accept planar 16bit signed stereo at 44100Hz. Since the sample format with name "s16p" corresponds to the number 6 and the "stereo" channel layout corresponds to the value 0x3, this is equivalent to: <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=6:channel_layout=0x3
</pre>

 <h3 class="section">28.2 aevalsrc# TOC</h3>

Generate an audio signal specified by an expression. This source accepts in input one or more expressions (one for each channel), which are evaluated and used to generate a corresponding audio signal. This source accepts the following options: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. In case the <samp class="option">channel_layout</samp> option is not specified, the selected channel layout depends on the number of provided expressions. Otherwise the last specified expression is applied to the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set the channel layout. The number of channels in the specified layout must be equal to the number of specified expressions. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default to 1024. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> </dl>

 <h4 class="subsection">28.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate silence: <pre class="example-preformatted">aevalsrc=0 </pre>

Generate a sin signal with frequency of 440 Hz, set sample rate to 8000 Hz: <pre class="example-preformatted">aevalsrc="sin(440*2*PI*t):s=8000" </pre>

Generate a two channels signal, specify the channel layout (Front Center + Back Center) explicitly: <pre class=”example-preformatted”>aevalsrc="sin(420*2*PI*t)|cos(430*2*PI*t):c=FC|BC" </pre>

Generate white noise: <pre class=”example-preformatted”>aevalsrc="-2+random(0)" </pre>

Generate an amplitude modulated signal: <pre class=”example-preformatted”>aevalsrc="sin(10*2*PI*t)*sin(880*2*PI*t)" </pre>

Generate 2.5 Hz binaural beats on a 360 Hz carrier: <pre class=”example-preformatted”>aevalsrc="0.1*sin(2*PI*(360-2.5/2)*t) | 0.1*sin(2*PI*(360+2.5/2)*t)" </pre>

 <h3 class=”section”>28.3 afdelaysrc# TOC</h3>

Generate a fractional delay FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay, d</samp></dt> <dd> Set the fractional delay. Default is 0. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set the number of filter coefficients in output audio stream. Default value is 0. </dd> <dt><samp class=”option”>channel_layout, c</samp></dt> <dd> Specifies the channel layout, and can be a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". </dd> </dl>

 <h3 class=”section”>28.4 afireqsrc# TOC</h3>

Generate a FIR equalizer coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset, p</samp></dt> <dd> Set equalizer preset. Default preset is <code class=”code”>flat</code>. Available presets are: <dl

class=”table”> <dt>‘<samp class=”samp”>custom</samp>’</dt>
<dt>‘<samp class=”samp”>flat</samp>’</dt> <dt>‘<samp
class=”samp”>acoustic</samp>’</dt> <dt>‘<samp class=”samp”>bass</samp>’</
<dt>‘<samp class=”samp”>beats</samp>’</dt> <dt>‘<samp
class=”samp”>classic</samp>’</dt> <dt>‘<samp class=”samp”>clear</samp>’</d
<dt>‘<samp class=”samp”>deep bass</samp>’</dt>
<dt>‘<samp class=”samp”>dubstep</samp>’</dt> <dt>‘<samp
class=”samp”>electronic</samp>’</dt> <dt>‘<samp
class=”samp”>hard-style</samp>’</dt> <dt>‘<samp
class=”samp”>hip-hop</samp>’</dt> <dt>‘<samp class=”samp”>jazz</samp>’</
<dt>‘<samp class=”samp”>metal</samp>’</dt> <dt>‘<samp
class=”samp”>movie</samp>’</dt> <dt>‘<samp class=”samp”>pop</samp>’</dt
<dt>‘<samp class=”samp”>r&b</samp>’</dt> <dt>‘<samp
class=”samp”>rock</samp>’</dt> <dt>‘<samp class=”samp”>vocal
booster</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gains, g</samp></dt> <dd> Set custom gains for each band. Only used if the preset option is set to <code class=”code”>custom</code>. Gains are separated by white spaces and each gain is set in dBFS. Default is <code class=”code”>0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0</code>. </dd> <dt><samp class=”option”>bands, b</samp></dt> <dd> Set the custom bands from where custom equalizer gains are set. This must be in strictly increasing order. Only used if the preset option is set to <code class=”code”>custom</code>. Bands are separated by white spaces and each band represent frequency in Hz. Default is <code class=”code”>25 40 63 100 160 250 400 630 1000 1600 2500 4000 6300 10000 16000 24000</code>. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is <code class=”code”>4096</code>. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set sample rate of output audio stream, default is <code class=”code”>44100</code>. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set number of samples per each frame in output audio stream. Default is <code class=”code”>1024</code>. </dd> <dt><samp class=”option”>interp, i</samp></dt> <dd> Set interpolation method for FIR equalizer coefficients. Can be <code class=”code”>linear</code> or <code class=”code”>cubic</code>. </dd> <dt><samp class=”option”>phase, h</samp></dt> <dd> Set phase type of FIR filter. Can be <code class=”code”>linear</code> or <code class=”code”>min</code>: minimum-phase. Default is minimum-phase filter. </dd> </dl>

 <h3 class=”section”>28.5 afirsrc#
TOC</h3>

Generate a FIR coefficients using frequency sampling method.

The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is 1025. </dd> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency points from where magnitude and phase are set. This must be in non decreasing order, and first element must be 0, while last element must be 1. Elements are separated by white spaces. </dd> <dt><samp class=”option”>magnitude, m</samp></dt> <dd> Set magnitude value for every frequency point set by <samp class=”option”>frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class=”option”>phase, p</samp></dt> <dd> Set phase value for every frequency point set by <samp class=”option”>frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>win_func, w</samp></dt> <dd> Set window function. Default is blackman. </dd> </dl>

 <h3 class=”section”>28.6 anullsrc# TOC</h3>

The null audio source, return unprocessed audio frames. It is mainly useful as a template and to be employed in analysis / debugging tools, or as the source for filters which ignore the input data (for example the sox synth filter). This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>channel_layout, cl</samp></dt> <dd> Specifies the channel layout, and can be either an integer or a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". Check the channel_layout_map definition in <samp class=”file”>libavutil/channel_layout.c</samp> for the mapping between strings and channel layout values. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specifies the sample rate, and defaults to 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per requested frames. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the audio is supposed to be generated

forever. </dd> </dl>

 <h4 class="subsection">28.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the sample rate to 48000 Hz and the channel layout to AV_CH_LAYOUT_MONO. <pre class="example-preformatted">anullsrc=r=48000:cl=4 </pre>

Do the same operation with a more obvious syntax: <pre class="example-preformatted">anullsrc=r=48000:cl=mono </pre>

All the parameters need to be explicitly defined. <h3 class="section">28.7 flite# TOC</h3>

Synthesize a voice utterance using the libflite library. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libflite</code>. Note that versions of the flite library prior to 2.0 are not thread-safe. The filter accepts the following options: <dl class="table"> <dt><samp class="option">list_voices</samp></dt> <dd> If set to 1, list the names of the available voices and exit immediately. Default value is 0. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the maximum number of samples per frame. Default value is 512. </dd> <dt><samp class="option">textfile</samp></dt> <dd> Set the filename containing the text to speak. </dd> <dt><samp class="option">text</samp></dt> <dd> Set the text to speak. </dd> <dt><samp class="option">voice, v</samp></dt> <dd> Set the voice to use for the speech synthesis. Default value is <code class="code">kal</code>. See also the <var class="var">list_voices</var> option. </dd> </dl>

 <h4 class="subsection">28.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read from file <samp class="file">speech.txt</samp>, and synthesize the text using the standard flite voice: <pre class="example-preformatted">flite=textfile=speech.txt </pre>

Read the specified text selecting the <code class="code">slt</code> voice: <pre class="example-preformatted">flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Input text to ffmpeg: <pre class="example-preformatted">ffmpeg -f lavfi -i flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Make <samp class="file">ffplay</samp> speak the specified text, using <code class="code">flite</code> and the <code class="code">lavfi</code> device: <pre class="example-preformatted">ffplay -f lavfi flite=text='No more be grieved for which that thou hast done.' </pre>

For more information about libflite, check: <a class="url" href="http://www.festvox.org/flite/">http://w <h3 class="section">28.8 anoisesrc# TOC</h3>

Generate a noise audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate. Default value is 48000 Hz. </dd> <dt><samp class="option">amplitude, a</samp></dt> <dd> Specify the amplitude (0.0 - 1.0) of the generated audio stream. Default value is 1.0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. Not specifying this option results in noise with an infinite length. </dd> <dt><samp class="option">color, colour, c</samp></dt> <dd> Specify the color of noise. Available noise colors are white, pink, brown, blue, violet and velvet. Default color is white. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Specify a value used to seed the PRNG. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per each output frame, default is 1024. </dd> <dt><samp class="option">density</samp></dt> <dd> Set the density (0.0 - 1.0) for the velvet noise generator, default is 0.05. </dd> </dl>

 <h4 class="subsection">28.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate 60 seconds of pink noise, with a 44.1 kHz sampling rate and an amplitude of 0.5: <pre class="example-preformatted">anoisesrc=d=60:c=pink:r=44100:a=0.5 </pre>

 <h3 class="section">28.9 hilbert# TOC</h3>

Generate odd-tap Hilbert transform FIR coefficients. The

resulting stream can be used with afir filter for phase-shifting the signal by 90 degrees. This is used in many matrix coding schemes and for analytic signal generation. The process is often written as a multiplication by i (or j), the imaginary unit. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set length of FIR filter, default is 22051. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. </dd> <dt><samp class="option">win_func, w</samp></dt> <dd> Set window function to be used when generating FIR coefficients. </dd> </dl>

 <h3 class="section">28.10 sinc# TOC</h3>

Generate a sinc kaiser-windowed low-pass, high-pass, band-pass, or band-reject FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class="option">hp</samp></dt> <dd> Set high-pass frequency. Default is 0. </dd> <dt><samp class="option">lp</samp></dt> <dd> Set low-pass frequency. Default is 0. If high-pass frequency is lower than low-pass frequency and low-pass frequency is higher than 0 then filter will create band-pass filter coefficients, otherwise band-reject filter coefficients. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set filter phase response. Default is 50. Allowed range is from 0 to 100. </dd> <dt><samp class="option">beta</samp></dt> <dd> Set Kaiser window beta. </dd> <dt><samp class="option">att</samp></dt> <dd> Set stop-band attenuation. Default is 120dB, allowed range is from 40 to 180 dB. </dd> <dt><samp class="option">round</samp></dt> <dd> Enable rounding, by default is disabled. </dd> <dt><samp class="option">hptaps</samp></dt> <dd> Set number of taps for high-pass filter. </dd> <dt><samp class="option">lptaps</samp></dt> <dd> Set number of taps for low-pass filter. </dd> </dl>

 <h3 class="section">28.11 sine# TOC</h3>

Generate an audio signal made of a sine wave with amplitude 1/8. The audio signal is bit-exact. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency,

f</samp></dt> <dd> Set the carrier frequency. Default is 440 Hz. </dd> <dt><samp class="option">beep_factor, b</samp></dt> <dd> Enable a periodic beep every second with frequency <var class="var">beep_factor</var> times the carrier frequency. Default is 0, meaning the beep is disabled. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate, default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. </dd> <dt><samp class="option">samples_per_frame</samp></dt> <dd> Set the number of samples per output frame. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the output audio frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the output audio frame, expressed in <var class="var">TB</var> units. </dd> <dt><samp class="option">t</samp></dt> <dd> The PTS of the output audio frame, expressed in seconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the output audio frames. </dd> </dl>

Default is <code class="code">1024</code>. </dd> </dl>

 <h4 class="subsection">28.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a simple 440 Hz sine wave: <pre class="example-preformatted">sine </pre>

Generate a 220 Hz sine wave with a 880 Hz beep each second, for 5 seconds: <pre class="example-preformatted">sine=220:4:d=5 sine=f=220:b=4:d=5 sine=frequency=220:beep_factor=4:duration=5 </pre>

Generate a 1 kHz sine wave following <code class="code">1602,1601,1602,1601,1602</code> NTSC pattern: <pre class="example-preformatted">sine=1000:samples_per_frame= 1602-not(not(eq(ld(0),1)+eq(ld(0),3)))' </pre>

 <h2 class="chapter">29 Audio Sinks# TOC</h2>

Below is a description of the currently available audio sinks. <h3 class="section">29.1 abuffersink# <a class="anchor hidden-xs"href="#toc-abuffersink"

aria-hidden=”true”>TOC</h3>

Buffer audio frames, and make them available to the end of filter chain. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVABufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class=”code”>avfilter_init_filter</code> for initialization. <h3 class=”section”>29.2 anullsink# TOC</h3>

Null audio sink; do absolutely nothing with the input audio. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class=”chapter”>30 Video Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class=”code”>--disable-filters</code>. The configure output will show the video filters included in your build. Below is a description of the currently available video filters. <h3 class=”section”>30.1 addroi# TOC</h3>

Mark a region of interest in a video frame. The frame data is passed through unchanged, but metadata is attached to the frame indicating regions of interest which can affect the behaviour of later encoding. Multiple regions can be marked by applying the filter multiple times. <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Region distance in pixels from the left edge of the frame. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Region distance in pixels from the top edge of the frame. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Region width in pixels. </dd> <dt><samp class=”option”>h</samp></dt> <dd> Region height in pixels. The parameters <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> are expressions, and may contain the following variables: <dl class=”table”> <dt><samp class=”option”>iw</samp></dt> <dd> Width of the input frame. </dd> <dt><samp class=”option”>ih</samp></dt> <dd> Height of the input frame. </dd> </dl>

</dd> <dt><samp class=”option”>qoffset</samp></dt> <dd> Quantisation offset to apply within the region. This must be a real value in the

range -1 to +1. A value of zero indicates no quality change. A negative value asks for better quality (less quantisation), while a positive value asks for worse quality (greater quantisation). The range is calibrated so that the extreme values indicate the largest possible offset - if the rest of the frame is encoded with the worst possible quality, an offset of -1 indicates that this region should be encoded with the best possible quality anyway. Intermediate values are then interpolated in some codec-dependent way. For example, in 10-bit H.264 the quantisation parameter varies between -12 and 51. A typical qoffset value of -1/10 therefore indicates that this region should be encoded with a QP around one-tenth of the full range better than the rest of the frame. So, if most of the frame were to be encoded with a QP of around 30, this region would get a QP of around 24 (an offset of approximately -1/10 * (51 - -12) = -6.3). An extreme value of -1 would indicate that this region should be encoded with the best possible quality regardless of the treatment of the rest of the frame - that is, should be encoded at a QP of -12. </dd> <dt><samp class="option">clear</samp></dt> <dd> If set to true, remove any existing regions of interest marked on the frame before adding the new one. </dd> </dl>

 <h4 class="subsection">30.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Mark the centre quarter of the frame as interesting. <pre class="example-preformatted">addroi=iw/4:ih/4:iw/2:ih/2:-1/10 </pre> Mark the 100-pixel-wide region on the left edge of the frame as very uninteresting (to be encoded at much lower quality than the rest of the frame). <pre class="example-preformatted">addroi=0:0:100:ih:+1/5 </pre>

 <h3 class="section">30.2 alphaextract# TOC</h3>

Extract the alpha component from the input as a grayscale video. This is especially useful with the <var class="var">alphamerge</var> filter. <h3 class="section">30.3 alphamerge# TOC</h3>

Add or replace the alpha component of the primary input with the grayscale value of a second input. This is intended for use with <var class="var">alphaextract</var> to allow the transmission or storage of frame sequences that have alpha in a format that doesn’t sup-

port an alpha channel. For example, to reconstruct full frames from a normal YUV-encoded video and a separate video created with <var class="var">alphaextract</var>, you might use: <pre class="example-preformatted">movie=in_alpha.mkv [alpha]; [in][alpha] alphamerge [out] </pre>

 <h3 class="section">30.4 amplify# TOC</h3>

Amplify differences between current pixel and pixels of adjacent frames in same pixel location. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set frame radius. Default is 2. Allowed range is from 1 to 63. For example radius of 3 will instruct filter to calculate average of 7 frames. </dd> <dt><samp class="option">factor</samp></dt> <dd> Set factor to amplify difference. Default is 2. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold for difference amplification. Any difference greater or equal to this value will not alter source pixel. Default is 10. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set tolerance for difference amplification. Any difference lower to this value will not alter source pixel. Default is 0. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">low</samp></dt> <dd> Set lower limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will decrease source pixel value. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will increase source pixel value. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">30.4.1 Commands# TOC</h4>

This filter supports the following commands that corresponds to option of same name: <dl class="table"> <dt><samp class="option">factor</samp></dt> <dt><samp class="option">threshold</samp></dt> <dt><samp class="option">tolerance</samp></dt> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dt><samp class="option">planes</samp></dt> </dl>

 <h3 class="section">30.5 ass<span class="pull-

right”>#
TOC</h3>

Same as the subtitles filter, except that it doesn’t require libavcodec and libavformat to work. On the other hand, it is limited to ASS (Advanced Substation Alpha) subtitles files.
This filter accepts <samp class=”option”>filename</samp>/<samp class=”option”>f</samp>, <samp class=”option”>original_size</samp>, <samp class=”option”>fontsdir</samp>, and <samp class=”option”>alpha</samp> from the subtitles filter, plus the following options: <dl class=”table”> <dt><samp class=”option”>script</samp></dt>
<dd> The ASS script to render, given inline. Exactly one of <samp class=”option”>filename</samp> and <samp class=”option”>script</samp> must be set. </dd> <dt><samp class=”option”>shaping</samp></dt>
<dd> Set the shaping engine. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class=”samp”>simple</samp>’</dt>
<dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt>
<dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew, Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd>
</dl>

The default is <code class=”code”>auto</code>. </dd> </dl>

 <h4 class=”subsection”>30.5.1 Commands# TOC</h4>

This filter supports the <samp class=”option”>script</samp> option as commands; the new script replaces the one being rendered. <h3 class=”section”>30.6 atadenoise# TOC</h3>
Apply an Adaptive Temporal Averaging Denoiser to the video input.
The filter accepts the following options: <dl class=”table”>
<dt><samp class=”option”>0a</samp></dt> <dd> Set threshold A for 1st plane. Default is 0.02. Valid range is 0 to 0.3. </dd>
<dt><samp class=”option”>0b</samp></dt> <dd> Set threshold B for 1st plane. Default is 0.04. Valid range is 0 to 5. </dd>
<dt><samp class=”option”>1a</samp></dt> <dd> Set threshold A for 2nd plane. Default is 0.02. Valid range is 0 to 0.3. </dd>
<dt><samp class=”option”>1b</samp></dt> <dd> Set threshold B for 2nd plane. Default is 0.04. Valid range is 0 to 5. </dd>

<dt><samp class="option">2a</samp></dt> <dd> Set threshold A for 3rd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">2b</samp></dt> <dd> Set threshold B for 3rd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">3a</samp></dt> <dd> Set threshold A for 4th plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">3b</samp></dt> <dd> Set threshold B for 4th plane. Default is 0.04. Valid range is 0 to 5. Threshold A is designed to react on abrupt changes in the input signal and threshold B is designed to react on continuous changes in the input signal. </dd> <dt><samp class="option">s</samp></dt> <dd> Set number of frames filter will use for averaging. Default is 9. Must be odd number in range [5, 129]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set what planes of frame filter will use for averaging. Default is all. </dd> <dt><samp class="option">a</samp></dt> <dd> Set what variant of algorithm filter will use for averaging. Default is <code class="code">p</code> parallel. Alternatively can be set to <code class="code">s</code> serial. Parallel can be faster then serial, while other way around is never true. Parallel will abort early on first change being greater then thresholds, while serial will continue processing other side of frames if they are equal or below thresholds. </dd> <dt><samp class="option">0s</samp></dt> <dt><samp class="option">1s</samp></dt> <dt><samp class="option">2s</samp></dt> <dt><samp class="option">3s</samp></dt> <dd> Set sigma for 1st plane, 2nd plane, 3rd plane or 4th plane. Default is 32767. Valid range is from 0 to 32767. This options controls weight for each pixel in radius defined by size. Default value means every pixel have same weight. Setting this option to 0 effectively disables filtering. </dd> </dl>

 <h4 class="subsection">30.6.1 Commands# TOC</h4> This filter supports same commands as options except option <code class="code">s</code>. The command accepts the same syntax of the corresponding option.

 <h3 class="section">30.7 avgblur# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sizeX</samp></dt> <dd> Set horizontal radius size. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class="option">sizeY</samp></dt> <dd> Set vertical radius size, if zero it will be same as <code class="code">sizeX</code>.

Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">30.7.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.8 backgroundkey# TOC</h3>

Turns a static background into transparency. The filter accepts the following option: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Threshold for scene change detection. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the background. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the blend amount for pixels that are not similar. </dd> </dl>

 <h4 class="subsection">30.8.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.9 bbox# TOC</h3>

Compute the bounding box for the non-black pixels in the input frame luma plane. This filter computes the bounding box containing all the pixels with a luma value greater than the minimum allowed value. The parameters describing the bounding box are printed on the filter log. The filter accepts the following option: <dl class="table"> <dt><samp class="option">min_val</samp></dt> <dd> Set the minimal luma value. Default is <code class="code">16</code>. </dd> </dl>

 <h4 class="subsection">30.9.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.10 bilateral<a class="anchor hidden-xs" href="#bilateral" aria-

hidden=”true”># TOC</h3> Apply bilateral filter, spatial smoothing while preserving edges. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight. Allowed range is 0 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate range weight. Allowed range is 0 to 1. Default is 0.1. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>30.10.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.11 bitplanenoise# TOC</h3>

Show and measure bit plane noise. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>bitplane</samp></dt> <dd> Set which plane to analyze. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>filter</samp></dt> <dd> Filter out noisy pixels from <code class=”code”>bitplane</code> set above. Default is disabled. </dd> </dl>

 <h3 class=”section”>30.12 blackdetect, blackdetect_vulkan# TOC</h3>

Detect video intervals that are (almost) completely black. Can be useful to detect chapter transitions, commercials, or invalid recordings. The filter outputs its detection analysis to both the log as well as frame metadata. If a black segment of at least the specified minimum duration is found, a line with the start and end timestamps as well as duration is printed to the log with level <code class=”code”>info</code>. In addition, a log line with level <code class=”code”>debug</code> is printed per frame showing the black amount detected for that frame. The filter also attaches metadata to the first frame of a black segment with key <code class=”code”>lavfi.black_start</code> and to the first frame after the black segment ends with key <code class=”code”>lavfi.black_end</code>. The value is the frame’s timestamp. This metadata is added regardless of the minimum duration specified. The filter

accepts the following options: <dl class="table"> <dt><samp class="option">black_min_duration, d</samp></dt> <dd> Set the minimum detected black duration expressed in seconds. It must be a non-negative floating point number. Default value is 2.0. </dd> <dt><samp class="option">picture_black_ratio_th, pic_th</samp></dt> <dd> Set the threshold for considering a picture "black". Express the minimum value for the ratio: <pre class="example-preformatted"><var class="var">nb_black_pixels</var> / <var class="var">nb_pixels</var> </pre>

for which a picture is considered black. Default value is 0.98. </dd> <dt><samp class="option">pixel_black_th, pix_th</samp></dt> <dd> Set the threshold for considering a pixel "black". The threshold expresses the maximum pixel luma value for which a pixel is considered "black". The provided value is scaled according to the following equation: <pre class="example-preformatted"><var class="var">absolute_threshold</var> = <var class="var">luma_minimum_value</var> + <var class="var">pixel_black_th</var> * <var class="var">luma_range_size</var> </pre>

<var class="var">luma_range_size</var> and <var class="var">luma_minimum_value</var> depend on the input video format, the range is [0-255] for YUV full-range formats and [16-235] for YUV non full-range formats. Default value is 0.10. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If true, check the alpha channel instead of the luma channel. Detects frames which are (almost) transparent, instead of frames which are almost black. Default value is disabled. </dd> </dl>

The following example sets the maximum pixel threshold to the minimum value, and detects only black intervals of 2 or more seconds: <pre class="example-preformatted">blackdetect=d=2:pix_th=0.00 </pre>

 <h3 class="section">30.13 blackframe# TOC</h3>

Detect frames that are (almost) completely black. Can be useful to detect chapter transitions or commercials. Output lines consist of the frame number of the detected frame, the percentage of blackness, the position in the file if known or -1 and the timestamp in seconds. In order to display the output lines, you need to set the loglevel at least to the AV_LOG_INFO value. This filter exports frame metadata <code class="code">lavfi.blackframe.pblack</code>. The value represents the percentage of pixels in the picture that are below the threshold value. It accepts the following parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt>

<dd> The percentage of the pixels that have to be below the threshold; it defaults to <code class=”code”>98</code>. </dd> <dt><samp class=”option”>threshold, thresh</samp></dt> <dd> The threshold below which a pixel value is considered black; it defaults to <code class=”code”>32</code>. </dd> </dl>

 <h3 class=”section”>30.14 blend# TOC</h3>

Blend two video frames into each other. The <code class=”code”>blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. The <code class=”code”>tblend</code> (time blend) filter takes two consecutive frames from one single stream, and outputs the result obtained by blending the new frame on top of the old frame. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>c0_mode</samp></dt> <dt><samp class=”option”>c1_mode</samp></dt> <dt><samp class=”option”>c2_mode</samp></dt> <dt><samp class=”option”>c3_mode</samp></dt> <dt><samp class=”option”>all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class=”var”>all_mode</var>. Default value is <code class=”code”>normal</code>. Available values for component modes are: <dl class=”table”> <dt>‘<samp class=”samp”>addition</samp>’</dt> <dt>‘<samp class=”samp”>and</samp>’</dt> <dt>‘<samp class=”samp”>average</samp>’</dt> <dt>‘<samp class=”samp”>bleach</samp>’</dt> <dt>‘<samp class=”samp”>burn</samp>’</dt> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dt>‘<samp class=”samp”>difference</samp>’</dt> <dt>‘<samp class=”samp”>divide</samp>’</dt> <dt>‘<samp class=”samp”>dodge</samp>’</dt> <dt>‘<samp class=”samp”>exclusion</samp>’</dt> <dt>‘<samp class=”samp”>extremity</samp>’</dt> <dt>‘<samp class=”samp”>freeze</samp>’</dt> <dt>‘<samp class=”samp”>geometric</samp>’</dt> <dt>‘<samp class=”samp”>glow</samp>’</dt> <dt>‘<samp class=”samp”>grainextract</samp>’</dt> <dt>‘<samp class=”samp”>grainmerge</samp>’</dt> <dt>‘<samp class=”samp”>hardlight</samp>’</dt> <dt>‘<samp class=”samp”>hardmix</samp>’</dt> <dt>‘<samp class=”samp”>hardoverlay</samp>’</dt> <dt>‘<samp class=”samp”>harmonic</samp>’</dt> <dt>‘<samp class=”samp”>heat</samp>’</dt> <dt>‘<samp class=”samp”>interpolate</samp>’</dt> <dt>‘<samp class=”samp”>lighten</samp>’</dt> <dt>‘<samp class=”samp”>linearlight</samp>’</dt> <dt>‘<samp class=”samp”>multiply</samp>’</dt> <dt>‘<samp class=”samp”>multiply128</samp>’</dt> <dt>‘<samp class=”samp”>negation</samp>’</dt> <dt>‘<samp class=”samp”>normal</samp>’</dt>

<dt>‘<samp class=”samp”>or</samp>’</dt> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dt>‘<samp class=”samp”>phoenix</samp>’ <dt>‘<samp class=”samp”>pinlight</samp>’</dt> <dt>‘<samp class=”samp”>reflect</samp>’</dt> <dt>‘<samp class=”samp”>screen</samp>’</ <dt>‘<samp class=”samp”>softdifference</samp>’</dt> <dt>‘<samp class=”samp”>softlight</samp>’</dt> <dt>‘<samp class=”samp”>stain</samp>’</dt> <dt>‘<samp class=”samp”>subtract</samp>’< <dt>‘<samp class=”samp”>vividlight</samp>’</dt> <dt>‘<samp class=”samp”>xor</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>c0_opacity</samp></dt> <dt><samp class=”option”>c1_opacity</samp></dt> <dt><samp class=”option”>c2_opacity</samp></dt> <dt><samp class=”option”>c3_opacity</samp></dt> <dt><samp class=”option”>all_opacity</samp></dt> <dd> Set blend opacity for specific pixel component or all pixel components in case of <var class=”var”>all_opacity</var>. Only used in combination with pixel component blend modes. </dd> <dt><samp class=”option”>c0_expr</samp></dt> <dt><samp class=”option”>c1_expr</samp></dt> <dt><samp class=”option”>c2_expr</samp></dt> <dt><samp class=”option”>c3_expr</samp></dt> <dt><samp class=”option”>all_expr</samp></dt> <dd> Set blend expression for specific pixel component or all pixel components in case of <var class=”var”>all_expr</var>. Note that related mode options will be ignored if those are set. The expressions can use the following variables: <dl class=”table”> <dt><samp class=”option”>N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class=”code”>0</code>. </dd> <dt><samp class=”option”>X</samp></dt> <dt><samp class=”option”>Y</samp></dt> <dd> the coordinates of the current sample </dd> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> the width and height of currently filtered plane </dd> <dt><samp class=”option”>SW</samp></dt> <dt><samp class=”option”>SH</samp></dt> <dd> Width and height scale for the plane being filtered. It is the ratio between the dimensions of the current plane to the luma plane, e.g. for a <code class=”code”>yuv420p</code> frame, the values are <code class=”code”>1,1</code> for the luma plane and <code class=”code”>0.5,0.5</code> for the chroma planes. </dd> <dt><samp class=”option”>T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class=”option”>TOP, A</samp></dt> <dd> Value of pixel component at current location for first video frame (top layer). </dd> <dt><samp class=”option”>BOTTOM, B</samp></dt> <dd> Value of pixel component at current location for second video frame (bottom layer). </dd> </dl> </dd> </dl>

The <code class=”code”>blend</code> filter also supports the framesync options. <h4 class=”subsection”>30.14.1 Examples<a class=”anchor hidden-xs” href=”#Examples-

64” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Apply transition from bottom layer to top layer in first 10 seconds: <pre class=”example-preformatted”>blend=all_expr='A*(if(gte(T,10),1,T/10))+B*(1-(if(gte(T,10),1,T/10)))' </pre>

Apply linear horizontal transition from top layer to bottom layer: <pre class=”example-preformatted”>blend=all_expr='A*(X/W)+B*(1-X/W)' </pre>

Apply 1x1 checkerboard effect: <pre class=”example-preformatted”>blend=all_expr='if(eq(mod(X,2),mod(Y,2)),A,B)' </pre>

Apply uncover left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(N*SW+X,W),A,B)' </pre>

Apply uncover down effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(Y-N*SH,0),A,B)' </pre>

Apply uncover up-left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(T*SH*40+Y,H)*gte((T*40*SW+X)*W/H,W),A,B)' </pre>

Split diagonally video and shows top and bottom layer on each side: <pre class=”example-preformatted”>blend=all_expr='if(gt(X,Y*(W/H)),A,B)' </pre>

Display differences between the current and the previous frame: <pre class=”example-preformatted”>tblend=all_mode=grainextract </pre>

 <h4 class=”subsection”>30.14.2 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>30.15 blockdetect# TOC</h3>

Determines blockiness of frames without altering the input frames. Based on Remco Muijs and Ihor Kirenko: "A no-reference blocking artifact measure for adaptive video processing." 2005 13th European signal processing conference. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>period_min</samp></dt>

<dt><samp class=”option”>period_max</samp></dt> <dd> Set minimum and maximum values for determining pixel grids (periods). Default values are [3,24]. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>30.15.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Determine blockiness for the first plane and search for periods within [8,32]: <pre class=”example-preformatted”>blockdetect=period_min=8:period_max=32:planes=1
</pre>

 <h3 class=”section”>30.16 blurdetect# TOC</h3>

Determines blurriness of frames without altering the input frames. Based on Marziliano, Pina, et al. "A no-reference perceptual blur metric." Allows for a block-based abbreviation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>low</samp></dt> <dt><samp class=”option”>high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class=”var”>low</var> and <var class=”var”>high</var> threshold values must be chosen in the range [0,1], and <var class=”var”>low</var> should be lesser or equal to <var class=”var”>high</var>. Default value for <var class=”var”>low</var> is <code class=”code”>20/255</code>, and default value for <var class=”var”>high</var> is <code class=”code”>50/255</code>. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> Define the radius to search around an edge pixel for local maxima. </dd> <dt><samp class=”option”>block_pct</samp></dt> <dd> Determine blurriness only for the most significant blocks, given in percentage. </dd> <dt><samp class=”option”>block_width</samp></dt> <dd> Determine blurriness for blocks of width <var class=”var”>block_width</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class=”var”>block_height</var>. </dd> <dt><samp class=”option”>block_height</samp></dt> <dd> Determine blurriness for blocks of height <var class=”var”>block_height</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class=”var”>block_width</var>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">30.16.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Determine blur for 80% of most significant 32x32 blocks: <pre class="example-preformatted">blurdetect=block_width=32:block_height=32:block_pct=80
</pre>

 <h3 class="section">30.17 bm3d#
TOC</h3>

Denoise frames using Block-Matching 3D algorithm. The filter accepts the following options. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set denoising strength. Default value is 1. Allowed range is from 0 to 999.9. The denoising algorithm is very sensitive to sigma, so adjust it according to the source. </dd> <dt><samp class="option">block</samp></dt> <dd> Set local patch size. This sets dimensions in 2D. </dd> <dt><samp class="option">bstep</samp></dt> <dd> Set sliding step for processing blocks. Default value is 4. Allowed range is from 1 to 64. Smaller values allows processing more reference blocks and is slower. </dd> <dt><samp class="option">group</samp></dt> <dd> Set maximal number of similar blocks for 3rd dimension. Default value is 1. When set to 1, no block matching is done. Larger values allows more blocks in single group. Allowed range is from 1 to 256. </dd> <dt><samp class="option">range</samp></dt> <dd> Set radius for search block matching. Default is 9. Allowed range is from 1 to INT32_MAX. </dd> <dt><samp class="option">mstep</samp></dt> <dd> Set step between two search locations for block matching. Default is 1. Allowed range is from 1 to 64. Smaller is slower. </dd> <dt><samp class="option">thmse</samp></dt> <dd> Set threshold of mean square error for block matching. Valid range is 0 to INT32_MAX. </dd> <dt><samp class="option">hdthr</samp></dt> <dd> Set thresholding parameter for hard thresholding in 3D transformed domain. Larger values results in stronger hard-thresholding filtering in frequency domain. </dd> <dt><samp class="option">estim</samp></dt> <dd> Set filtering estimation mode. Can be <code class="code">basic</code> or <code class="code">final</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">ref</samp></dt> <dd> If enabled, filter will use 2nd stream for block matching. Default is disabled for <code class="code">basic</code> value of <var class="var">estim</var> option, and always enabled if value of <var class="var">estim</var> is <code class="code">final</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is all available except alpha. </dd>

</dl>

 <h4 class="subsection">30.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Basic filtering with bm3d: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=ba
</pre>

Same as above, but filtering only luma: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic:planes=1
</pre>

Same as above, but with both estimation modes: <pre class="example-preformatted">split[a][b],[a]bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic[a],[b
</pre>

Same as above, but prefilter with nlmeans filter instead: <pre class="example-preformatted">split[a][b],[a]nlmeans=s=3:r=7:p=
</pre>

 <h3 class="section">30.18 boxblur# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">luma_power, lp</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">chroma_power, cp</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dt><samp class="option">alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class="code">min(w,h)/2</code> for the luma and alpha planes, and of <code class="code">min(cw,ch)/2</code> for the chroma planes. Default value for <samp class="option">luma_radius</samp> is "2". If not specified, <samp class="option">chroma_radius</samp> and <samp class="option">alpha_radius</samp> default to the corresponding value set for <samp class="option">luma_radius</samp>. The expressions can contain the following constants: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp

class=”option”>h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class=”option”>cw</samp></dt> <dt><samp class=”option”>ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class=”option”>luma_power</samp> is 2. If not specified, <samp class=”option”>chroma_power</samp> and <samp class=”option”>alpha_power</samp> default to the corresponding value set for <samp class=”option”>luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class=”subsection”>30.18.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply a boxblur filter with the luma, chroma, and alpha radii set to 2: <pre class=”example-preformatted”>boxblur=luma_radius=2:luma_power=1
boxblur=2:1 </pre>

Set the luma radius to 2, and alpha and chroma radius to 0: <pre class=”example-preformatted”>boxblur=2:1:cr=0:ar=0
</pre>

Set the luma and chroma radii to a fraction of the video dimension:
 <pre class=”example-preformatted”>boxblur=luma_radius=min(h\,w)/10:luma_pow
</pre>

 <h3 class=”section”>30.19 bwdif# TOC</h3>

Deinterlace the input video ("bwdif" stands for "Bob Weaver Deinterlacing Filter"). Motion adaptive deinterlacing based on yadif with the use of w3fdif and cubic interpolation algorithms. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, send_frame</samp></dt>

<dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">30.20 ccrepack# TOC</h3>

Repack CEA-708 closed captioning side data This filter fixes various issues seen with commercial encoders related to upstream malformed CEA-708 payloads, specifically incorrect number of tuples (wrong cc_count for the target FPS), and incorrect ordering of tuples (i.e. the CEA-608 tuples are not at the first entries in the payload). <h3 class="section">30.21 cas# TOC</h3>

Apply Contrast Adaptive Sharpen filter to video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Set the sharpening strength. Default value is 0. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default value is to filter all planes except alpha plane. </dd> </dl>

 <h4 class="subsection">30.21.1 Commands<a class="anchor hidden-xs" href="#Commands-

62" aria-hidden="true"># TOC</h4> This filter supports same commands as options. <h3 class="section">30.22 chromahold# TOC</h3> Remove all color information for all colors except for certain one. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will not be replaced with neutral chroma. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more preserved color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">30.22.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.23 chromakey# TOC</h3> YUV colorspace color/chroma keying. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will be replaced with transparency. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like

"green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">30.23.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">30.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf chromakey=green out.png </pre>

Overlay a greenscreen-video on top of a static black background. <pre class="example-preformatted">ffmpeg -f lavfi -i color=c=black:s=1280x720 -i video.mp4 -shortest -filter_complex "[1:v]chromakey=0x70de77:0.1:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.mkv </pre>

 <h3 class="section">30.24 chromanr# TOC</h3> Reduce chrominance noise. The filter accepts the following options: <dl class="table"> <dt><samp class="option">thres</samp></dt> <dd> Set threshold for averaging chrominance values. Sum of absolute difference of Y, U and V pixel components of current pixel and neighbour pixels lower than this threshold will be used in averaging. Luma component is left unchanged and is copied to output. Default value is 30. Allowed range is from 1 to 200. </dd> <dt><samp class="option">sizew</samp></dt> <dd> Set horizontal radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">sizeh</samp></dt> <dd> Set vertical radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">stepw</samp></dt> <dd> Set horizontal step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">steph</samp></dt> <dd> Set vertical step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">threy</samp></dt> <dd> Set Y threshold for averaging chrominance values. Set finer con-

trol for max allowed difference between Y components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threu</samp></dt> <dd> Set U threshold for averaging chrominance values. Set finer control for max allowed difference between U components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threv</samp></dt> <dd> Set V threshold for averaging chrominance values. Set finer control for max allowed difference between V components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">distance</samp></dt> <dd> Set distance type used in calculations. <dl class="table"> <dt>‘<samp class="samp">manhattan</samp>’</dt> <dd> Absolute difference. </dd> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dd> Difference squared. </dd> </dl>

Default distance type is manhattan. </dd> </dl>

 <h4 class="subsection">30.24.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. <h3 class="section">30.25 chromashift# TOC</h3> Shift chroma pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cbh</samp></dt> <dd> Set amount to shift chroma-blue horizontally. </dd> <dt><samp class="option">cbv</samp></dt> <dd> Set amount to shift chroma-blue vertically. </dd> <dt><samp class="option">crh</samp></dt> <dd> Set amount to shift chroma-red horizontally. </dd> <dt><samp class="option">crv</samp></dt> <dd> Set amount to shift chroma-red vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">30.25.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.26 ciescope<a class="anchor hidden-xs" href="#ciescope" aria-

hidden=”true”># TOC</h3>

Display CIE color diagram with pixels overlaid onto it. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>system</samp></dt> <dd> Set color system. <dl class=”table”> <dt>‘<samp class=”samp”>ntsc, 470m</samp>’</dt> <dt>‘<samp class=”samp”>ebu, 470bg</samp>’</dt> <dt>‘<samp class=”samp”>smpte</samp>’</dt> <dt>‘<samp class=”samp”>240m</samp>’</dt> <dt>‘<samp class=”samp”>apple</samp>’</dt> <dt>‘<samp class=”samp”>widergb</samp>’< <dt>‘<samp class=”samp”>cie1931</samp>’</dt> <dt>‘<samp class=”samp”>rec709, hdtv</samp>’</dt> <dt>‘<samp class=”samp”>uhdtv, rec2020</samp>’</dt> <dt>‘<samp class=”samp”>dcip3</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>cie</samp></dt> <dd> Set CIE system. <dl class=”table”> <dt>‘<samp class=”samp”>xyy</samp>’</dt> <dt>‘<samp class=”samp”>ucs</samp>’</dt> <dt>‘<samp class=”samp”>luv</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gamuts</samp></dt> <dd> Set what gamuts to draw. See <code class=”code”>system</code> option for available values. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set ciescope size, by default set to 512. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity used to map input pixel values to CIE diagram. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Set contrast used to draw tongue colors that are out of active color system gamut. </dd> <dt><samp class=”option”>corrgamma</samp></dt> <dd> Correct gamma displayed on scope, by default enabled. </dd> <dt><samp class=”option”>showwhite</samp></dt> <dd> Show white point on CIE diagram, by default disabled. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set input gamma. Used only with XYZ input color space. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Fill with CIE colors. By default is enabled. </dd> </dl>

 <h3 class=”section”>30.27 codecview# TOC</h3>

Visualize information exported by some codecs. Some codecs can export information through frames using side-data or other means. For example, some MPEG based codecs export motion vectors through the <var class=”var”>export_mvs</var> flag in the codec <samp class=”option”>flags2</samp> option. The filter

accepts the following option: <dl class="table"> <dt><samp class="option">block</samp></dt> <dd> Display block partition structure using the luma plane. </dd> <dt><samp class="option">mv</samp></dt> <dd> Set motion vectors to visualize. Available flags for <var class="var">mv</var> are: <dl class="table"> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> forward predicted MVs of P-frames </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> forward predicted MVs of B-frames </dd> <dt>‘<samp class="samp">bb</samp>’</dt> <dd> backward predicted MVs of B-frames </dd> </dl>

</dd> <dt><samp class="option">qp</samp></dt> <dd> Display quantization parameters using the chroma planes. </dd> <dt><samp class="option">mv_type, mvt</samp></dt> <dd> Set motion vectors type to visualize. Includes MVs from all frames unless specified by <var class="var">frame_type</var> option. Available flags for <var class="var">mv_type</var> are: <dl class="table"> <dt>‘<samp class="samp">fp</samp>’</dt> <dd> forward predicted MVs </dd> <dt>‘<samp class="samp">bp</samp>’</dt> <dd> backward predicted MVs </dd> </dl>

</dd> <dt><samp class="option">frame_type, ft</samp></dt> <dd> Set frame type to visualize motion vectors of. Available flags for <var class="var">frame_type</var> are: <dl class="table"> <dt>‘<samp class="samp">if</samp>’</dt> <dd> intra-coded frames (I-frames) </dd> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> predicted frames (P-frames) </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> bi-directionally predicted frames (B-frames) </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.27.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Visualize forward predicted MVs of all frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv_type=fp </pre>

Visualize multi-directionals MVs of P and B-Frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv=pf+bf+bb </pre>

 <h3 class="section">30.28 colorbalance# TOC</h3> Modify

intensity of primary colors (red, green and blue) of input frames. The filter allows an input frame to be adjusted in the shadows, midtones or highlights regions for the red-cyan, green-magenta or blue-yellow balance. A positive adjustment value shifts the balance towards the primary color, a negative value towards the complementary color. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rs</samp></dt> <dt><samp class="option">gs</samp></dt> <dt><samp class="option">bs</samp></dt> <dd> Adjust red, green and blue shadows (darkest pixels). </dd> <dt><samp class="option">rm</samp></dt> <dt><samp class="option">gm</samp></dt> <dt><samp class="option">bm</samp></dt> <dd> Adjust red, green and blue midtones (medium pixels). </dd> <dt><samp class="option">rh</samp></dt> <dt><samp class="option">gh</samp></dt> <dt><samp class="option">bh</samp></dt> <dd> Adjust red, green and blue highlights (brightest pixels). Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">pl</samp></dt> <dd> Preserve lightness when changing color balance. Default is disabled. </dd> </dl>

 <h4 class="subsection">30.28.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add red color cast to shadows: <pre class="example-preformatted">colorbalance=rs=.3 </pre>

 <h4 class="subsection">30.28.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.29 colorcontrast# TOC</h3>

Adjust color contrast between RGB components. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rc</samp></dt> <dd> Set the red-cyan contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">gm</samp></dt> <dd> Set the green-magenta contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">by</samp></dt> <dd> Set the blue-yellow contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0.

</dd> <dt><samp class="option">rcw</samp></dt> <dt><samp class="option">gmw</samp></dt> <dt><samp class="option">byw</samp></dt> <dd> Set the weight of each <code class="code">rc</code>, <code class="code">gm</code>, <code class="code">by</code> option value. Default value is 0.0. Allowed range is from 0.0 to 1.0. If all weights are 0.0 filtering is disabled. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Default value is 0.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">30.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.30 colorcorrect# TOC</h3>

Adjust color white balance selectively for blacks and whites. This filter operates in YUV colorspace. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rl</samp></dt> <dd> Set the red shadow spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">bl</samp></dt> <dd> Set the blue shadow spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">rh</samp></dt> <dd> Set the red highlight spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">bh</samp></dt> <dd> Set the blue highlight spot. Allowed range is from -1.0 to 1.0. Default value is 0. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the amount of saturation. Allowed range is from -3.0 to 3.0. Default value is 1. </dd> <dt><samp class="option">analyze</samp></dt> <dd> If set to anything other than <code class="code">manual</code> it will analyze every frame and use derived parameters for filtering output frame. Possible values are: <dl class="table"> <dt>‘<samp class="samp">manual</samp>’</dt> <dt>‘<samp class="samp">average</samp>’</dt> <dt>‘<samp class="samp">minmax</samp>’</dt> <dt>‘<samp class="samp">median</samp>’</dt> </dl> Default value is <code class="code">manual</code>. </dd> </dl>

 <h4 class="subsection">30.30.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.31

colorchannelmixer# TOC</h3>

Adjust video input frames by re-mixing color channels.
This filter modifies a color channel by adding the values associated to the other channels of the same pixels. For example if the value to modify is red, the output value will be: <pre class="example-preformatted"><var class="var">red</var>=<var class="var">red</var>*<var class="var">rr</var> + <var class="var">blue</var>*<var class="var">rb</var> + <var class="var">green</var>*<var class="var">rg</var> + <var class="var">alpha</var>*<var class="var">ra</var> </pre>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">rr</samp></dt> <dt><samp class="option">rg</samp></dt> <dt><samp class="option">rb</samp></dt> <dt><samp class="option">ra</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output red channel. Default is <code class="code">1</code> for <var class="var">rr</var>, and <code class="code">0</code> for <var class="var">rg</var>, <var class="var">rb</var> and <var class="var">ra</var>. </dd> <dt><samp class="option">gr</samp></dt> <dt><samp class="option">gg</samp></dt> <dt><samp class="option">gb</samp></dt> <dt><samp class="option">ga</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output green channel. Default is <code class="code">1</code> for <var class="var">gg</var>, and <code class="code">0</code> for <var class="var">gr</var>, <var class="var">gb</var> and <var class="var">ga</var>. </dd> <dt><samp class="option">br</samp></dt> <dt><samp class="option">bg</samp></dt> <dt><samp class="option">bb</samp></dt> <dt><samp class="option">ba</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output blue channel. Default is <code class="code">1</code> for <var class="var">bb</var>, and <code class="code">0</code> for <var class="var">br</var>, <var class="var">bg</var> and <var class="var">ba</var>. </dd> <dt><samp class="option">ar</samp></dt> <dt><samp class="option">ag</samp></dt> <dt><samp class="option">ab</samp></dt> <dt><samp class="option">aa</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output alpha channel. Default is <code class="code">1</code> for <var class="var">aa</var>, and <code class="code">0</code> for <var class="var">ar</var>, <var class="var">ag</var> and <var class="var">ab</var>. Allowed ranges for options are <code class="code">[-2.0, 2.0]</code>. </dd> <dt><samp class="option">pc</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt>

<dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class=”samp”>sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class=”samp”>nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class=”samp”>pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl>

</dd> <dt><samp class=”option”>pa</samp></dt> <dd> Set the preserve color amount when changing colors. Allowed range is from <code class=”code”>[0.0, 1.0]</code>. Default is <code class=”code”>0.0</code>, thus disabled. </dd> </dl>

 <h4 class=”subsection”>30.31.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert source to grayscale: <pre class=”example-preformatted”>colorchannelmixer=.3:.4:.3:0:.3:.4:.3:0:.3:.4:.3 </pre> Simulate sepia tones: <pre class=”example-preformatted”>colorchannelmixer=.393:.769:.189:0:.349:.686:.168:0:.272:.534:.131 </pre>

 <h4 class=”subsection”>30.31.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.32 colordetect# TOC</h3> Analyze the video frames to determine the effective value range and alpha mode. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set of properties to detect. Unavailable properties, such as alpha mode for an input image without an alpha channel, will be ignored automatically. Accepts a combination of the following flags: <dl class=”table”> <dt>‘<samp class=”samp”>color_range</samp>’</dt> <dd> Detect if the source contains luma pixels outside the limited (MPEG) range, which indicates that this is a full range YUV source. </dd> <dt>‘<samp class=”samp”>alpha_mode</samp>’</dt> <dd> Detect if the source contains color values above the alpha channel, which indicates that the alpha channel is independent (straight), rather than premultiplied. Also detects if the alpha plane is fully opaque or not. </dd> <dt>‘<samp class=”samp”>all</samp>’</dt>

<dd> Enable detection of all of the above properties. This is the default. </dd> </dl>

</dd> <dt><samp class="option">threshold</samp></dt> <dd> Allow pixels to exceed the expected bounds by this fraction of the full scale value range before treating it as evidence of (respectively) full range or straight alpha. Value range is from 0 to 0.05. Defaults to 0.001 (=0.1%). </dd> </dl>

 <h3 class="section">30.33 colorize# TOC</h3> Overlay a solid color on the video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the color hue. Allowed range is from 0 to 360. Default value is 0. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the color saturation. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">lightness</samp></dt> <dd> Set the color lightness. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set the mix of source lightness. By default is set to 1.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">30.33.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.34 colorkey# TOC</h3> RGB colorspace color keying. This filter operates on 8-bit RGB format frames by setting the alpha component of each pixel which falls within the similarity radius of the key color to 0. The alpha value for pixels outside the similarity radius depends on the value of the blend option. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> Set the color for which alpha will be set to 0 (full transparency). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default is <code class="code">black</code>. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set the radius from the key color within which other colors also have full transparency. The computed distance is related to the unit fractional distance in 3D space between the RGB values of the key color and the pixel’s color. Range is 0.01 to 1.0. 0.01 matches within a very small radius around the exact key color, while 1.0 matches everything.

Default is <code class=”code”>0.01</code>. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set how the alpha value for pixels that fall outside the similarity radius is computed. 0.0 makes pixels either fully transparent or fully opaque. Higher values result in semi-transparent pixels, with greater transparency the more similar the pixel color is to the key color. Range is 0.0 to 1.0. Default is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>30.34.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make every green pixel in the input image transparent: <pre class=”example-preformatted”>ffmpeg -i input.png -vf colorkey=green out.png </pre>

Overlay a greenscreen-video on top of a static background image. <pre class=”example-preformatted”>ffmpeg -i background.png -i video.mp4 -filter_complex "[1:v]colorkey=0x3BBD1E:0.3:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.flv </pre>

 <h4 class=”subsection”>30.34.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>30.35 colorhold# TOC</h3> Remove all color information for all RGB colors except for certain one. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dd> The color which will not be replaced with neutral gray. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels fully gray. Higher values result in more preserved color. </dd> </dl>

 <h4 class=”subsection”>30.35.1 Commands# TOC</h4> This filter supports same commands

as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.36 colorlevels# TOC</h3>

Adjust video input frames using levels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rimin</samp></dt> <dt><samp class="option">gimin</samp></dt> <dt><samp class="option">bimin</samp></dt> <dt><samp class="option">aimin</samp></dt> <dd> Adjust red, green, blue and alpha input black point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">rimax</samp></dt> <dt><samp class="option">gimax</samp></dt> <dt><samp class="option">bimax</samp></dt> <dt><samp class="option">aimax</samp></dt> <dd> Adjust red, green, blue and alpha input white point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">1</code>. Input levels are used to lighten highlights (bright tones), darken shadows (dark tones), change the balance of bright and dark tones. </dd> <dt><samp class="option">romin</samp></dt> <dt><samp class="option">gomin</samp></dt> <dt><samp class="option">bomin</samp></dt> <dt><samp class="option">aomin</samp></dt> <dd> Adjust red, green, blue and alpha output black point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">romax</samp></dt> <dt><samp class="option">gomax</samp></dt> <dt><samp class="option">bomax</samp></dt> <dt><samp class="option">aomax</samp></dt> <dd> Adjust red, green, blue and alpha output white point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">1</code>. Output levels allows manual selection of a constrained output level range. </dd> <dt><samp class="option">preserve</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.36.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Make video output darker: <pre class="example-preformatted">colorlevels=rimin=0.058:gimin=0.058:bimin=0.058 </pre>

Increase contrast: <pre class="example-preformatted">colorlevels=rimin=0.039:gimin=0.039:bimin=0.039:rimax=0.96:gimax=0.96:bimax=0.96 </pre>

Make video output lighter: <pre class="example-preformatted">colorlevels=rimax=0.902:gimax=0.902:bimax=0.902 </pre>

Increase brightness: <pre class="example-preformatted">colorlevels=romin=0.5:gomin=0.5:bomin=0.5 </pre>

 <h4 class="subsection">30.36.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.37 colormap# TOC</h3> Apply custom color maps to video stream. This filter needs three input video streams. First stream is video stream that is going to be filtered out. Second and third video stream specify color patches for source color to target color mapping. The filter accepts the following options: <dl class="table"> <dt><samp class="option">patch_size</samp></dt> <dd> Set the source and target video stream patch size in pixels. </dd> <dt><samp class="option">nb_patches</samp></dt> <dd> Set the max number of used patches from source and target video stream. Default value is number of patches available in additional video streams. Max allowed number of patches is <code class="code">64</code>. </dd> <dt><samp class="option">type</samp></dt> <dd> Set the adjustments used for target colors. Can be <code class="code">relative</code> or <code class="code">absolute</code>. Defaults is <code class="code">absolute</code>. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Set the kernel used to measure color differences between mapped colors. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dt>‘<samp class="samp">weuclidean</samp>’</dt> </dl>

Default is <code class=”code”>euclidean</code>. </dd> </dl>

 <h3 class=”section”>30.38 colormatrix# TOC</h3>

Convert color matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>src</samp></dt> <dt><samp class=”option”>dst</samp></dt> <dd> Specify the source and destination color matrix. Both values must be specified. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dd> BT.470 </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl> </dd> </dl>

For example to convert from BT.601 to SMPTE-240M, use the command: <pre class=”example-preformatted”>colormatrix=bt601:smpte240m </pre>

 <h3 class=”section”>30.39 colorspace# TOC</h3>

Convert colorspace, transfer characteristics or color primaries. Input video needs to have an even size. The filter accepts the following options: <dl class=”table”> <dd></dd> <dt><samp class=”option”>all</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>bt601-6-525</samp>’</dt> <dd> BT.601-6 525 </dd> <dt>‘<samp class=”samp”>bt601-6-625</samp>’</dt> <dd> BT.601-6 625 </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class=”option”>space</samp></dt> <dd> Specify output colorspace. The accepted values are:

<dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp
class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M
or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">ycgco</samp>’</dt>
<dd> YCgCo </dd> <dt>‘<samp class="samp">bt2020ncl</samp>’</dt>
<dd> BT.2020 with non-constant luminance </dd> </dl>

</dd> <dt><samp class="option">trc</samp></dt>
<dd> Specify output transfer characteristics. The accepted val-
ues are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> Constant gamma of 2.2 </dd> <dt>‘<samp
class="samp">gamma28</samp>’</dt> <dd> Constant gamma
of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M, BT.601-6 625 or BT.601-6 525 </dd>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">srgb</samp>’</dt>
<dd> SRGB </dd> <dt>‘<samp class="samp">iec61966-2-
1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp
class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4
 </dd> <dt>‘<samp class="samp">xvycc</samp>’</dt>
<dd> xvycc </dd> <dt>‘<samp class="samp">bt2020-
10</samp>’</dt> <dd> BT.2020 for 10-bits content
</dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt>
<dd> BT.2020 for 12-bits content </dd> </dl>

</dd> <dt><samp class="option">primaries</samp></dt>
<dd> Specify output color primaries. The accepted values are:
<dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt>
<dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp
class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M
or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> film </dd> <dt>‘<samp class="samp">smpte431</samp>’</dt>
<dd> SMPTE-431 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt>
<dd> SMPTE-432 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class="samp">jedec-
p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd>
</dl>

</dd> <dt><samp class=”option”>range</samp></dt>
<dd> Specify output color range. The accepted values are:
<dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt>
<dd> TV (restricted) range </dd> <dt>‘<samp
class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted)
range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt>
<dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt>
<dd> JPEG (full) range </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Specify
output color format. The accepted values are: <dl
class=”table”> <dt>‘<samp class=”samp”>yuv420p</samp>’</dt>
<dd> YUV 4:2:0 planar 8-bits </dd> <dt>‘<samp
class=”samp”>yuv420p10</samp>’</dt> <dd> YUV 4:2:0 pla-
nar 10-bits </dd> <dt>‘<samp class=”samp”>yuv420p12</samp>’</dt>
<dd> YUV 4:2:0 planar 12-bits </dd> <dt>‘<samp
class=”samp”>yuv422p</samp>’</dt> <dd> YUV 4:2:2 planar
8-bits </dd> <dt>‘<samp class=”samp”>yuv422p10</samp>’</dt>
<dd> YUV 4:2:2 planar 10-bits </dd> <dt>‘<samp
class=”samp”>yuv422p12</samp>’</dt> <dd> YUV 4:2:2 pla-
nar 12-bits </dd> <dt>‘<samp class=”samp”>yuv444p</samp>’</dt>
<dd> YUV 4:4:4 planar 8-bits </dd> <dt>‘<samp
class=”samp”>yuv444p10</samp>’</dt> <dd> YUV 4:4:4 pla-
nar 10-bits </dd> <dt>‘<samp class=”samp”>yuv444p12</samp>’</dt>
<dd> YUV 4:4:4 planar 12-bits </dd> </dl>

</dd> <dt><samp class=”option”>fast</samp></dt> <dd> Do a
fast conversion, which skips gamma/primary correction. This will take
significantly less CPU, but will be mathematically incorrect. To get output
compatible with that produced by the colormatrix filter, use fast=1.
</dd> <dt><samp class=”option”>dither</samp></dt> <dd> Specify
dithering mode. The accepted values are: <dl class=”table”>
<dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No
dithering </dd> <dt>‘<samp class=”samp”>fsb</samp>’</dt>
<dd> Floyd-Steinberg dithering </dd> </dl>

</dd> <dt><samp class=”option”>wpadapt</samp></dt> <dd> Whitepoint
adaptation mode. The accepted values are: <dl
class=”table”> <dt>‘<samp class=”samp”>bradford</samp>’</dt>
<dd> Bradford whitepoint adaptation </dd> <dt>‘<samp
class=”samp”>vonkries</samp>’</dt> <dd> von Kries white-
point adaptation </dd> <dt>‘<samp class=”samp”>identity</samp>’</dt>
<dd> identity whitepoint adaptation (i.e. no whitepoint adaptation)
</dd> </dl>

</dd> <dt><samp class=”option”>clipgamut</samp></dt> <dd> Controls
how to clip out-of-gamut colors that arise as a result of colorspace con-
version. The accepted values are: <dl class=”table”>

<dt>‘<samp class="samp">none</samp>’</dt> <dd> No clipping of out of gamut colors. </dd> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> Clips the RGB values to the [0, 1] range when building the gamma transfer LUTs. </dd> </dl>

</dd> <dt><samp class="option">iall</samp></dt> <dd> Override all input properties at once. Same accepted values as all. </dd> <dt><samp class="option">ispace</samp></dt> <dd> Override input colorspace. Same accepted values as space. </dd> <dt><samp class="option">iprimaries</samp></dt> <dd> Override input color primaries. Same accepted values as primaries. </dd> <dt><samp class="option">itrc</samp></dt> <dd> Override input transfer characteristics. Same accepted values as trc. </dd> <dt><samp class="option">irange</samp></dt> <dd> Override input color range. Same accepted values as range. </dd> </dl>

The filter converts the transfer characteristics, color space and color primaries to the specified user values. The output value, if not specified, is set to a default value based on the "all" property. If that property is also not specified, the filter will log an error. The output color range and format default to the same value as the input color range and format. The input transfer characteristics, color space, color primaries and color range should be set on the input data. If any of these are missing, the filter will log an error and no conversion will take place. For example to convert the input to SMPTE-240M, use the command: <pre class="example-preformatted">colorspace=smpte240m </pre>

 <h3 class="section">30.40 colortemperature# TOC</h3> Adjust color temperature in video to simulate variations in ambient color temperature. The filter accepts the following options: <dl class="table"> <dt><samp class="option">temperature</samp></dt> <dd> Set the temperature in Kelvin. Allowed range is from 1000 to 40000. Default value is 6500 K. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set mixing with filtered output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">30.40.1 Commands# TOC</h4> This fil-

ter supports same commands as options. <h3 class=”section”>30.41 convolution# TOC</h3>

Apply convolution of 3x3, 5x5, 7x7 or horizontal/vertical up to 49 elements. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>0m</samp></dt> <dt><samp class=”option”>1m</samp></dt> <dt><samp class=”option”>2m</samp></dt> <dt><samp class=”option”>3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed integers in <var class=”var”>square</var> mode, and from 1 to 49 odd number of signed integers in <var class=”var”>row</var> mode. </dd> <dt><samp class=”option”>0rdiv</samp></dt> <dt><samp class=”option”>1rdiv</samp></dt> <dt><samp class=”option”>2rdiv</samp></dt> <dt><samp class=”option”>3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be 1/sum of all matrix elements. </dd> <dt><samp class=”option”>0bias</samp></dt> <dt><samp class=”option”>1bias</samp></dt> <dt><samp class=”option”>2bias</samp></dt> <dt><samp class=”option”>3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. Default is 0.0. </dd> <dt><samp class=”option”>0mode</samp></dt> <dt><samp class=”option”>1mode</samp></dt> <dt><samp class=”option”>2mode</samp></dt> <dt><samp class=”option”>3mode</samp></dt> <dd> Set matrix mode for each plane. Can be <var class=”var”>square</var>, <var class=”var”>row</var> or <var class=”var”>column</var>. Default is <var class=”var”>square</var>. </dd> </dl>

 <h4 class=”subsection”>30.41.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>30.41.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sharpen: <pre class=”example-preformatted”>convolution="0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0" </pre>

Apply blur: <pre class=”example-preformatted”>convolution="1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9" </pre>

Apply edge enhance: <pre class=”example-preformatted”>convolution="0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128" </pre>

Apply edge detect: <pre class=”example-preformatted”>convolution="0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128" </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>convolution="1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0" </pre>

Apply emboss: <pre class=”example-preformatted”>convolution="-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2" </pre>

 <h3 class=”section”>30.42 convolve# TOC</h3>

Apply 2D convolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class=”option”>impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> </dl>

The <code class=”code”>convolve</code> filter also supports the framesync options. <h3 class=”section”>30.43 copy# TOC</h3>

Copy the input video source unchanged to the output. This is mainly useful for testing purposes. <h3 class=”section”>30.44 coreimage# TOC</h3> Video filtering on GPU using Apple’s CoreImage API on OSX. Hardware acceleration is based on an OpenGL context. Usually, this means it is processed by video hardware. However, software-based OpenGL implementations exist which means there is no guarantee for hardware processing. It depends on the respective OSX. There are many filters and image generators provided by Apple that come with a large variety of options. The filter has to

be referenced by its name along with its options. The coreimage filter accepts the following options: <dl class="table"> <dt><samp class="option">list_filters</samp></dt> <dd> List all available filters and generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class="example-preformatted">list_filters=true </pre>

</dd> <dt><samp class="option">filter</samp></dt> <dd> Specify all filters by their respective name and options. Use <var class="var">list_filters</var> to determine all valid filter names and options. Numerical options are specified by a float value and are automatically clamped to their respective value range. Vector and color options have to be specified by a list of space separated float values. Character escaping has to be done. A special option name <code class="code">default</code> is available to use default options for a filter. It is required to specify either <code class="code">default</code> or at least one of the filter options. All omitted options are used with their default values. The syntax of the filter string is as follows: <pre class="example-preformatted">filter=<NAME>@<OPTION>=<VALUE>[@<OPTION>=<VALUE&
</pre>

</dd> <dt><samp class="option">output_rect</samp></dt> <dd> Specify a rectangle where the output of the filter chain is copied into the input image. It is given by a list of space separated float values: <pre class="example-preformatted">output_rect=x\ y\ width\ height </pre> If not given, the output rectangle equals the dimensions of the input image. The output rectangle is automatically cropped at the borders of the input image. Negative values are valid for each component. <pre class="example-preformatted">output_rect=25\ 25\ 100\ 100 </pre> </dd> </dl>

Several filters can be chained for successive processing without GPU-HOST transfers allowing for fast processing of complex filter chains. Currently, only filters with zero (generators) or exactly one (filters) input image and one output image are supported. Also, transition filters are not yet usable as intended. Some filters generate output images with additional padding depending on the respective filter kernel. The padding is automatically removed to ensure the filter output has the same size as the input image. For image generators, the size of the output image is determined by the previous output image of the filter chain or the input image of the whole filterchain, respectively. The generators do not use the pixel information of this image to generate their output. However, the generated output is blended onto this image, resulting in partial or complete coverage of the output image. The coreimagesrc video source can be used for generating input images which are directly fed into the filter chain. By using it,

providing input images by another video source or an input video is not required. <h4 class="subsection">30.44.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all filters available: <pre class="example-preformatted">coreimage=list_filters=true </pre>

Use the CIBoxBlur filter with default options to blur an image: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default </pre>

Use a filter chain with CISepiaTone at default values and CIVignetteEffect with its center at 100x100 and a radius of 50 pixels: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default#CIVignetteEf
100@inputRadius=50 </pre>

Use nullsrc and CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class="example-preformatted">ffmpeg -f lavfi -i nullsrc=s=100x100,coreimage=filter=CIQRCodeGenerator@inputMessage=ht
-frames:v 1 QRCode.png </pre>

 <h3 class="section">30.45 corr# TOC</h3>

Obtain the correlation between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max correlation is printed through the logging system. The filter stores the calculated correlation of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi corr -f null - </pre>

 <h3 class="section">30.46 cover_rect# TOC</h3>

Cover a rectangular object It accepts the following options: <dl class="table"> <dt><samp class="option">cover</samp></dt> <dd> Filepath of the optional cover image, needs to be in yuv420.

</dd> <dt><samp class="option">mode</samp></dt> <dd> Set covering mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> cover it by the supplied image </dd> <dt>‘<samp class="samp">blur</samp>’</dt> <dd> cover it by interpolating the surrounding pixels </dd> </dl>

Default value is <var class="var">blur</var>. </dd> </dl>

 <h4 class="subsection">30.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

 <h3 class="section">30.47 crop# TOC</h3>

Crop the input video to given dimensions. It accepts the following parameters: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dd> The width of the output video. It defaults to <code class="code">iw</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">w</samp>’ or ‘<samp class="samp">out_w</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">h, out_h</samp></dt> <dd> The height of the output video. It defaults to <code class="code">ih</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">h</samp>’ or ‘<samp class="samp">out_h</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">x</samp></dt> <dd> The horizontal position, in the input video, of the left edge of the output video. It defaults to <code class="code">(in_w-out_w)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">y</samp></dt> <dd> The vertical position, in the input video, of the top edge of the output video. It defaults to <code class="code">(in_h-out_h)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">keep_aspect</samp></dt> <dd> If set to 1 will force the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. It defaults to 0. </dd> <dt><samp class="option">exact</samp></dt> <dd> Enable exact cropping. If enabled, subsampled videos will be cropped at exact width/height/x/y as

specified and will not be rounded to nearest smaller value. It defaults to 0. </dd> </dl>

The <var class=”var”>out_w</var>, <var class=”var”>out_h</var>, <var class=”var”>x</var>, <var class=”var”>y</var> parameters are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output (cropped) width and height. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

The expression for <var class=”var”>out_w</var> may depend on the value of <var class=”var”>out_h</var>, and the expression for <var class=”var”>out_h</var> may depend on <var class=”var”>out_w</var>, but they cannot depend on <var class=”var”>x</var> and <var class=”var”>y</var>, as <var class=”var”>x</var> and <var class=”var”>y</var> are evaluated after <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. The <var class=”var”>x</var> and <var class=”var”>y</var> parameters specify the expressions for the position of the top-left corner of the output (non-cropped) area. They are evaluated for each frame. If the evaluated value is not valid, it is approximated to the nearest

valid value. The expression for <var class="var">x</var> may depend on <var class="var">y</var>, and the expression for <var class="var">y</var> may depend on <var class="var">x</var>. <h4 class="subsection">30.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Crop area with size 100x100 at position (12,34). <pre class="example-preformatted">crop=100:100:12:34 </pre>

Using named options, the example above becomes: <pre class="example-preformatted">crop=w=100:h=100:x=12:y=34 </pre>

Crop the central input area with size 100x100: <pre class="example-preformatted">crop=100:100 </pre>

Crop the central input area with size 2/3 of the input video: <pre class="example-preformatted">crop=2/3*in_w:2/3*in_h </pre>

Crop the input video central square: <pre class="example-preformatted">crop=out_w=in_h crop=in_h </pre>

Delimit the rectangle with the top-left corner placed at position 100:100 and the right-bottom corner corresponding to the right-bottom corner of the input image. <pre class="example-preformatted">crop=in_w-100:in_h-100:100:100 </pre>

Crop 10 pixels from the left and right borders, and 20 pixels from the top and bottom borders <pre class="example-preformatted">crop=in_w-2*10:in_h-2*20 </pre>

Keep only the bottom right quarter of the input image: <pre class="example-preformatted">crop=in_w/2:in_h/2:in_w/2:in_h/2 </pre>

Crop height for getting Greek harmony: <pre class="example-preformatted">crop=in_w:1/PHI*in_w </pre>

Apply trembling effect: <pre class="example-preformatted">crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(n/10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(n/7) </pre>

Apply erratic camera effect depending on timestamp: <pre class="example-preformatted">crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(t*10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(t*13) </pre>

Set x depending on the value of y: <pre class="example-preformatted">crop=in_w/2:in_h/2:y:10+10*sin(n/10) </pre>

 <h4 class="subsection">30.47.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dt><samp class="option">h, out_h</samp></dt> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set width/height of the output video and the horizontal/vertical position in the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">30.48 cropdetect# TOC</h3>

Auto-detect the crop size. It calculates the necessary cropping parameters and prints the recommended parameters via the logging system. The detected dimensions correspond to the non-black or video area of the input video according to <var class="var">mode</var>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Depending on <var class="var">mode</var> crop detection is based on either the mere black value of surrounding pixels or a combination of motion vectors and edge pixels. <dl class="table"> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Detect black pixels surrounding the playing video. For fine control use option <var class="var">limit</var>. </dd> <dt>‘<samp class="samp">mvedges</samp>’</dt> <dd> Detect the playing video by the motion vectors inside the video and scanning for edge pixels typically forming the border of a playing video. </dd> </dl>

</dd> <dt><samp class="option">limit</samp></dt> <dd> Set higher black value threshold, which can be optionally specified from nothing (0) to everything (255 for 8-bit based formats). An intensity value greater to the set value is considered non-black. It defaults to 24. You can also specify a value between 0.0 and 1.0 which will be scaled depending on the bitdepth of the pixel format. </dd> <dt><samp class="option">round</samp></dt> <dd> The value which the width/height should be divisible by. It defaults to 16. The offset is automatically adjusted to center the video. Use 2 to get only even dimensions (needed for 4:2:2 video). 16 is best when encoding to most

video codecs. </dd> <dt><samp class="option">skip</samp></dt> <dd> Set the number of initial frames for which evaluation is skipped. Default is 2. Range is 0 to INT_MAX. </dd> <dt><samp class="option">reset_count, reset</samp></dt> <dd> Set the counter that determines after how many frames cropdetect will reset the previously detected largest video area and start over to detect the current optimal crop area. Default value is 0. This can be useful when channel logos distort the video area. 0 indicates ’never reset’, and returns the largest area encountered during playback. </dd> <dt><samp class="option">mv_threshold</samp></dt> <dd> Set motion in pixel units as threshold for motion detection. It defaults to 8. </dd> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">5/255</code>, and default value for <var class="var">high</var> is <code class="code">15/255</code>. </dd> </dl>

 <h4 class="subsection">30.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Find video area surrounded by black borders: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf cropdetect,metadata=mode=print -f null - </pre>

Find an embedded video area, generate motion vectors beforehand: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf mestimate,cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

Find an embedded video area, use motion vectors from decoder: <pre class="example-preformatted">ffmpeg -flags2 +export_mvs -i file.mp4 -vf cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

 <h4 class="subsection">30.48.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">

<dt><samp class=”option”>limit</samp></dt> <dd> The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>30.49 cue# TOC</h3>

Delay video filtering until a given wallclock timestamp. The filter first passes on <samp class=”option”>preroll</samp> amount of frames, then it buffers at most <samp class=”option”>buffer</samp> amount of frames and waits for the cue. After reaching the cue it forwards the buffered frames and also any subsequent frames coming in its input. The filter can be used synchronize the output of multiple ffmpeg processes for realtime output devices like decklink. By putting the delay in the filtering chain and pre-buffering frames the process can pass on data to output almost immediately after the target wallclock timestamp is reached. Perfect frame accuracy cannot be guaranteed, but the result is good enough for some use cases. <dl class=”table”> <dt><samp class=”option”>cue</samp></dt> <dd> The cue timestamp expressed in a UNIX timestamp in microseconds. Default is 0. </dd> <dt><samp class=”option”>preroll</samp></dt> <dd> The duration of content to pass on as preroll expressed in seconds. Default is 0. </dd> <dt><samp class=”option”>buffer</samp></dt> <dd> The maximum duration of content to buffer before waiting for the cue expressed in seconds. Default is 0. </dd> </dl>

 <h3 class=”section”>30.50 curves# TOC</h3>

Apply color adjustments using curves. This filter is similar to the Adobe Photoshop and GIMP curves tools. Each component (red, green and blue) has its values defined by <var class=”var”>N</var> key points tied from each other using a smooth curve. The x-axis represents the pixel values from the input frame, and the y-axis the new pixel values to be set for the output frame. By default, a component curve is defined by the two points <var class=”var”>(0;0)</var> and <var class=”var”>(1;1)</var>. This creates a straight line where each original pixel value is "adjusted" to its own value, which means no change to the image. The filter allows you to redefine these two points and add some more. A new curve will be defined to pass smoothly through all these new coordinates. The new defined points need to be strictly increasing over the x-axis, and their <var class=”var”>x</var> and <var class=”var”>y</var> values must be in the <var class=”var”>[0;1]</var> interval. The curve is formed by using a natural or monotonic cubic

spline interpolation, depending on the <var class=”var”>interp</var> option (default: <code class=”code”>natural</code>). The <code class=”code”>natural</code> spline produces a smoother curve in general while the monotonic (<code class=”code”>pchip</code>) spline guarantees the transitions between the specified points to be monotonic. If the computed curves happened to go outside the vector spaces, the values will be clipped accordingly. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset</samp></dt> <dd> Select one of the available color presets. This option can be used in addition to the <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> parameters; in this case, the later options takes priority on the preset values. Available presets are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dt>‘<samp class=”samp”>color_negative</samp>&rs
<dt>‘<samp class=”samp”>cross_process</samp>’</dt>
<dt>‘<samp class=”samp”>darker</samp>’</dt> <dt>‘<samp class=”samp”>increase_contrast</samp>’</dt> <dt>‘<samp class=”samp”>lighter</samp>’</dt> <dt>‘<samp class=”samp”>linear_contrast</samp>&
<dt>‘<samp class=”samp”>medium_contrast</samp>’</dt>
<dt>‘<samp class=”samp”>negative</samp>’</dt> <dt>‘<samp class=”samp”>strong_contrast</samp>’</dt> <dt>‘<samp class=”samp”>vintage</samp>’</dt> </dl> Default is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>master, m</samp></dt> <dd> Set the master key points. These points will define a second pass mapping. It is sometimes called a "luminance" or "value" mapping. It can be used with <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> or <samp class=”option”>all</samp> since it acts like a post-processing LUT. </dd> <dt><samp class=”option”>red, r</samp></dt> <dd> Set the key points for the red component. </dd> <dt><samp class=”option”>green, g</samp></dt> <dd> Set the key points for the green component. </dd> <dt><samp class=”option”>blue, b</samp></dt> <dd> Set the key points for the blue component. </dd> <dt><samp class=”option”>all</samp></dt> <dd> Set the key points for all components (not including master). Can be used in addition to the other key points component options. In this case, the unset component(s) will fallback on this <samp class=”option”>all</samp> setting. </dd> <dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop curves file (<code class=”code”>.acv</code>) to import the settings from. </dd> <dt><samp class=”option”>plot</samp></dt> <dd> Save Gnuplot script of the curves in specified file. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the kind of interpolation. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>natural</samp>’</dt> <dd> Natural cubic spline using a piece-wise cubic polynomial that is twice continuously differ-

entiable. </dd> <dt>‘<samp class="samp">pchip</samp>’</dt>
<dd> Monotonic cubic spline using a piecewise cubic Hermite interpolating
polynomial (PCHIP). </dd> </dl>

</dd> </dl>

To avoid some filtergraph syntax conflicts, each key points list need
to be defined using the following syntax: <code class="code">x0/y0
x1/y1 x2/y2 ...</code>.
<h4 class="subsection">30.50.1 Commands<a
class="anchor hidden-xs" href="#Commands-79" aria-hidden="true">#
TOC</h4>

This filter supports same commands
as options. <h4 class="subsection">30.50.2
Examples<a class="anchor hidden-xs" href="#Examples-
80" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-80" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Increase slightly the middle level of blue:
 <pre class="example-preformatted">curves=blue='0/0
0.5/0.58 1/1' </pre>

Vintage effect: <pre class="example-
preformatted">curves=r='0/0.11 .42/.51 1/0.95':g='0/0 0.50/0.48 1/1':b='0/0.22
.49/.44 1/0.8' </pre> Here we obtain the following co-
ordinates for each components: <dl class="table"> <dt><var
class="var">red</var></dt> <dd> <code class="code">(0;0.11)
(0.42;0.51) (1;0.95)</code> </dd> <dt><var class="var">green</var></dt>
<dd> <code class="code">(0;0) (0.50;0.48) (1;1)</code> </dd>
<dt><var class="var">blue</var></dt> <dd> <code class="code">(0;0.22)
(0.49;0.44) (1;0.80)</code> </dd> </dl>

The previous example can also be achieved with the associ-
ated built-in preset: <pre class="example-
preformatted">curves=preset=vintage </pre>

Or simply: <pre class="example-
preformatted">curves=vintage </pre>

Use a Photoshop preset and redefine the points of the green compo-
nent: <pre class="example-preformatted">curves=psfile='MyCurvesPresets/purple.a
0.45/0.53 1/1' </pre>

Check out the curves of the <code class="code">cross_process</code>
profile using <code class="command">ffmpeg</code> and <code class="command">gnuplot</code>:
 <pre class="example-preformatted">ffmpeg -f lavfi
-i color -vf curves=cross_process:plot=/tmp/curves.plt -frames:v 1 -f null -
gnuplot -p /tmp/curves.plt </pre>

 <h3 class="section">30.51 datascope# TOC</h3>

Video data analysis filter. This filter shows hexadecimal pixel values of part of video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set output video size. </dd> <dt><samp class="option">x</samp></dt> <dd> Set x offset from where to pick pixels. </dd> <dt><samp class="option">y</samp></dt> <dd> Set y offset from where to pick pixels. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set scope mode, can be one of the following: <dl class="table"> <dt>‘<samp class="samp">mono</samp>’</dt> <dd> Draw hexadecimal pixel values with white color on black background. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Draw hexadecimal pixel values with input video pixel color on black background. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Draw hexadecimal pixel values on color background picked from input video, the text color is picked in such way so its always visible. </dd> </dl>

</dd> <dt><samp class="option">axis</samp></dt> <dd> Draw rows and columns numbers on left and top of video. </dd> <dt><samp class="option">opacity</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">format</samp></dt> <dd> Set display number format. Can be <code class="code">hex</code>, or <code class="code">dec</code>. Default is <code class="code">hex</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set pixel components to display. By default all pixel components are displayed. </dd> </dl>

 <h4 class="subsection">30.51.1 Commands# TOC</h4>

This filter supports same commands as options excluding <code class="code">size</code> option. <h3 class="section">30.52 dblur# TOC</h3> Apply Directional blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> Set angle of directional blur. Default is <code class="code">45</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Set radius of directional blur. Default is <code class="code">5</code>.

</dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> </dl>

 <h4 class="subsection">30.52.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.53 dctdnoiz# TOC</h3>

Denoise frames using 2D DCT (frequency domain filtering). This filter is not designed for real time. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma, s</samp></dt> <dd> Set the noise sigma constant. This <var class="var">sigma</var> defines a hard threshold of <code class="code">3 * sigma</code>; every DCT coefficient (absolute value) below this threshold with be dropped. If you need a more advanced filtering, see <samp class="option">expr</samp>. Default is <code class="code">0</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number overlapping pixels for each block. Since the filter can be slow, you may want to reduce this value, at the cost of a less effective filter and the risk of various artefacts. If the overlapping value doesn’t permit processing the whole input width or height, a warning will be displayed and according borders won’t be denoised. Default value is <var class="var">blocksize</var>-1, which is the best possible setting. </dd> <dt><samp class="option">expr, e</samp></dt> <dd> Set the coefficient factor expression. For each coefficient of a DCT block, this expression will be evaluated as a multiplier value for the coefficient. If this is option is set, the <samp class="option">sigma</samp> option will be ignored. The absolute value of the coefficient can be accessed through the <var class="var">c</var> variable. </dd> <dt><samp class="option">n</samp></dt> <dd> Set the <var class="var">blocksize</var> using the number of bits. <code class="code">1<<<var class="var">n</var></code> defines the <var class="var">blocksize</var>, which is the width and height of the processed blocks. The default value is <var class="var">3</var> (8x8) and can be raised to <var class="var">4</var> for a <var class="var">blocksize</var> of 16x16. Note that changing this setting has huge consequences on the speed processing. Also, a larger block size does not necessarily means a better de-noising. </dd> </dl>

 <h4 class="subsection">30.53.1 Examples# TOC</h4>

Apply a denoise with a <samp class="option">sigma</samp> of <code class="code">4.5</code>: <pre class="example-preformatted">dctdnoiz=4.5 </pre>

The same operation can be achieved using the expression system: <pre class="example-preformatted">dctdnoiz=e='gte(c, 4.5*3)' </pre>

Violent denoise using a block size of <code class="code">16x16</code>: <pre class="example-preformatted">dctdnoiz=15:n=4 </pre>

 <h3 class="section">30.54 deband# TOC</h3>

Remove banding artifacts from input video. It works by replacing banded pixels with average value of referenced pixels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">1thr</samp></dt> <dt><samp class="option">2thr</samp></dt> <dt><samp class="option">3thr</samp></dt> <dt><samp class="option">4thr</samp></dt> <dd> Set banding detection threshold for each plane. Default is 0.02. Valid range is 0.00003 to 0.5. If difference between current pixel and reference pixel is less than threshold, it will be considered as banded. </dd> <dt><samp class="option">range, r</samp></dt> <dd> Banding detection range in pixels. Default is 16. If positive, random number in range 0 to set value will be used. If negative, exact absolute value will be used. The range defines square of four pixels around current pixel. </dd> <dt><samp class="option">direction, d</samp></dt> <dd> Set direction in radians from which four pixel will be compared. If positive, random direction from 0 to set direction will be picked. If negative, exact of absolute value will be picked. For example direction 0, -PI or -2*PI radians will pick only pixels on same row and -PI/2 will pick only pixels on same column. </dd> <dt><samp class="option">blur, b</samp></dt> <dd> If enabled, current pixel is compared with average value of all four surrounding pixels. The default is enabled. If disabled current pixel is compared with all four surrounding pixels. The pixel is considered banded if only all four differences with surrounding pixels are less than threshold. </dd> <dt><samp class="option">coupling, c</samp></dt> <dd> If enabled, current pixel is changed if and only if all pixel components are banded, e.g. banding detection threshold is triggered for all color components. The default is disabled. </dd> </dl>

 <h4 class="subsection">30.54.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.55 deblock# TOC</h3>

Remove blocking artifacts from input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filter</samp></dt> <dd> Set filter type, can be <var class="var">weak</var> or <var class="var">strong</var>. Default is <var class="var">strong</var>. This controls what kind of deblocking is applied. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block, allowed range is from 4 to 512. Default is <var class="var">8</var>. </dd> <dt><samp class="option">alpha</samp></dt> <dt><samp class="option">beta</samp></dt> <dt><samp class="option">gamma</samp></dt> <dt><samp class="option">delta</samp></dt> <dd> Set blocking detection thresholds. Allowed range is 0 to 1. Defaults are: <var class="var">0.098</var> for <var class="var">alpha</var> and <var class="var">0.05</var> for the rest. Using higher threshold gives more deblocking strength. Setting <var class="var">alpha</var> controls threshold detection at exact edge of block. Remaining options controls threshold detection near the edge. Each one for below/above or left/right. Setting any of those to <var class="var">0</var> disables deblocking. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is to filter all available planes. </dd> </dl>

 <h4 class="subsection">30.55.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Deblock using weak filter and block size of 4 pixels. <pre class="example-preformatted">deblock=filter=weak:block=4 </pre>

Deblock using strong filter, block size of 4 pixels and custom thresholds for deblocking more edges. <pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta=0.05
</pre>

Similar as above, but filter only first plane.
<pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta
</pre>

Similar as above, but filter only second and third plane. <pre class="example-preformatted">deblock=filter=strong:block=4:alpha=0.12:beta=0.0
</pre>

 <h4 class="subsection">30.55.2 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">30.56 decimate# TOC</h3>

Drop duplicated frames at regular intervals. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cycle</samp></dt> <dd> Set the number of frames from which one will be dropped. Setting this to <var class="var">N</var> means one frame in every batch of <var class="var">N</var> frames will be dropped. Default is <code class="code">5</code>. </dd> <dt><samp class="option">dupthresh</samp></dt> <dd> Set the threshold for duplicate detection. If the difference metric for a frame is less than or equal to this value, then it is declared as duplicate. Default is <code class="code">1.1</code> </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set scene change threshold. Default is <code class="code">15</code>. </dd> <dt><samp class="option">blockx</samp></dt> <dt><samp class="option">blocky</samp></dt> <dd> Set the size of the x and y-axis blocks used during metric calculations. Larger blocks give better noise suppression, but also give worse detection of small movements. Must be a power of two. Default is <code class="code">32</code>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark main input as a pre-processed input and activate clean source input stream. This allows the input to be pre-processed with various filters to help the metrics calculation while keeping the frame selection lossless. When set to <code class="code">1</code>, the first stream is for the pre-processed input, and the second stream is the clean source from where the kept frames are chosen. Default is <code class="code">0</code>. </dd> <dt><samp class="option">chroma</samp></dt> <dd> Set whether or not chroma is considered in the metric calculations. Default is <code class="code">1</code>. </dd> <dt><samp class="option">mixed</samp></dt> <dd> Set whether or not the input only partially contains content to be decimated. Default is <code class="code">false</code>. If enabled video output stream will be in variable frame rate. </dd> </dl>

 <h3 class="section">30.57 deconvolve<span

class=”pull-right”># TOC</h3>

Apply 2D deconvolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class=”option”>impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> <dt><samp class=”option”>noise</samp></dt> <dd> Set noise when doing divisions. Default is <var class=”var”>0.0000001</var>. Useful when width and height are not same and not power of 2 or if stream prior to convolving had noise. </dd> </dl>

The <code class=”code”>deconvolve</code> filter also supports the framesync options. <h3 class=”section”>30.58 dedot# TOC</h3>

Reduce cross-luminance (dot-crawl) and cross-color (rainbows) from video. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>m</samp></dt> <dd> Set mode of operation. Can be combination of <var class=”var”>dotcrawl</var> for cross-luminance reduction and/or <var class=”var”>rainbows</var> for cross-color reduction. </dd> <dt><samp class=”option”>lt</samp></dt> <dd> Set spatial luma threshold. Lower values increases reduction of cross-luminance. </dd> <dt><samp class=”option”>tl</samp></dt> <dd> Set tolerance for temporal luma. Higher values increases reduction of cross-luminance. </dd> <dt><samp class=”option”>tc</samp></dt> <dd> Set tolerance for chroma temporal variation. Higher values increases reduction of cross-color. </dd> <dt><samp class=”option”>ct</samp></dt> <dd> Set temporal chroma threshold. Lower values increases reduction of cross-color. </dd> </dl>

 <h3 class=”section”>30.59 deflate# TOC</h3>

Apply deflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values lower than the pixel. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt>

<dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">30.59.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.60 deflicker# TOC</h3>

Remove temporal frame luminance variations. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set moving-average filter size in frames. Default is 5. Allowed range is 2 - 129. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set averaging mode to smooth temporal luminance variations. Available values are: <dl class="table"> <dt>‘<samp class="samp">am</samp>’</dt> <dd> Arithmetic mean </dd> <dt>‘<samp class="samp">gm</samp>’</dt> <dd> Geometric mean </dd> <dt>‘<samp class="samp">hm</samp>’</dt> <dd> Harmonic mean </dd> <dt>‘<samp class="samp">qm</samp>’</dt> <dd> Quadratic mean </dd> <dt>‘<samp class="samp">cm</samp>’</dt> <dd> Cubic mean </dd> <dt>‘<samp class="samp">pm</samp>’</dt> <dd> Power mean </dd> <dt>‘<samp class="samp">median</samp>’</dt> <dd> Median </dd> </dl>

</dd> <dt><samp class="option">bypass</samp></dt> <dd> Do not actually modify frame. Useful when one only wants metadata. </dd> </dl>

 <h3 class="section">30.61 deinterlace_d3d12# TOC</h3>

Deinterlace the input video using D3D12 hardware-accelerated video processing. This filter uses the DirectX 12 Video Processor to perform deinterlacing on GPU, supporting both simple bob and driver-defined advanced (motion-adaptive) algorithms. It requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter automatically queries the hardware for the required number of temporal reference frames (if needed) and manages a frame queue to supply them. It accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> The deinterlacing method. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Select

the best deinterlacing method available on the hardware. If the driver supports <code class="code">custom</code>, that method is used; otherwise falls back to <code class="code">bob</code>. This is the default. </dd> <dt>‘<samp class="samp">bob</samp>’</dt> <dd> Bob deinterlacing. Each field is independently scaled to full frame height. Simple and fast, but may produce visible bob artifacts on moving content. </dd> <dt>‘<samp class="samp">custom</samp>’</dt> <dd> Driver-defined advanced deinterlacing. The exact algorithm is hardware-specific and typically employs motion-adaptive techniques with temporal reference frames for higher quality. </dd> </dl>

The default value is <code class="code">default</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Specify the interlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Send one frame for each frame. The output frame rate equals the input frame rate. This is the default. </dd> <dt>‘<samp class="samp">field</samp>’</dt> <dd> Send one frame for each field. The output frame rate is double the input frame rate. </dd> </dl>

The default value is <code class="code">frame</code>. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">all</samp>’</dt> <dd> Deinterlace all frames. This is the default. </dd> <dt>‘<samp class="samp">interlaced</samp>’</dt> <dd> Only deinterlace frames marked as interlaced; progressive frames are passed through unchanged. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h4 class="subsection">30.61.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Deinterlace an interlaced video using software decoding and the best available hardware deinterlacing method: <pre class="example-preformatted">ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=method=default,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

Deinterlace at field rate (double the frame rate) using bob method with hardware decoding (full d3d12 pipeline): <pre class="example-preformatted">ffmpeg -hwaccel d3d12va -hwaccel_output_format

d3d12 -i input.ts \ -vf "deinterlace_d3d12=method=bob:mode=field"
\ -c:v h264_d3d12va output.mp4 </pre>

Deinterlace only interlaced frames, passing progressive frames through: <pre class="example-preformatted">ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=deint=interlaced,hwdownload,format=nv12"
\ -c:v libx264 -crf 18 output.mp4 </pre>

 <h3 class="section">30.62 dejudder# TOC</h3>

Remove judder produced by partially interlaced telecined content. Judder can be introduced, for instance, by pullup filter. If the original source was partially telecined content then the output of <code class="code">pullup,dejudder</code> will have a variable frame rate. May change the recorded frame rate of the container. Aside from that change, this filter will not affect constant frame rate video. The option available in this filter is: <dl class="table"> <dt><samp class="option">cycle</samp></dt> <dd> Specify the length of the window over which the judder repeats. Accepts any integer greater than 1. Useful values are: <dl class="table"> <dt>‘<samp class="samp">4</samp>’</dt> <dd> If the original was telecined from 24 to 30 fps (Film to NTSC). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> If the original was telecined from 25 to 30 fps (PAL to NTSC). </dd> <dt>‘<samp class="samp">20</samp>’</dt> <dd> If a mixture of the two. </dd> </dl>

The default is ‘<samp class="samp">4</samp>’. </dd> </dl>

 <h3 class="section">30.63 delogo# TOC</h3>

Suppress a TV station logo by a simple interpolation of the surrounding pixels. Just set a rectangle covering the logo and watch it disappear (and sometimes something even uglier appear - your mileage may vary). It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the top left corner coordinates of the logo. They must be specified. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Specify the width and height of the logo to clear. They must be

specified. </dd> <dt><samp class="option">show</samp></dt> <dd> When set to 1, a green rectangle is drawn on the screen to simplify finding the right <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, and <var class="var">h</var> parameters. The default value is 0. The rectangle is drawn on the outermost pixels which will be (partly) replaced with interpolated values. The values of the next pixels immediately outside this rectangle in each direction will be used to compute the interpolated pixel values inside the rectangle. </dd> </dl>

 <h4 class="subsection">30.63.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set a rectangle covering the area with top left corner coordinates 0,0 and size 100x77: <pre class="example-preformatted">delogo=x=0:y=0:w=100:h=77 </pre>

 <h3 class="section">30.64 derain# TOC</h3>

Remove the rain in the input image/video by applying the derain methods based on convolutional neural networks. Supported models: <ul class="itemize mark-bullet"> Recurrent Squeeze-and-Excitation Context Aggregation Net (RESCAN). See http://openaccess.thecvf.com/content_ECCV_2018/pap and-Excitation_Context_ECCV_2018_paper.pdf.

Training as well as model generation scripts are provided in the repository at <a class="url" href="https://github.com/XueweiMeng/derain_filter.git">https://github.com/XueweiMeng/d The filter accepts the following options: <dl class="table"> <dt><samp class="option">filter_type</samp></dt> <dd> Specify which filter to use. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">derain</samp>’</dt> <dd> Derain filter. To conduct derain filter, you need to use a derain model. </dd> <dt>‘<samp class="samp">dehaze</samp>’</dt> <dd> Dehaze filter. To conduct dehaze filter, you need to use a dehaze model. </dd> </dl> Default value is ‘<samp class="samp">derain</samp>’. </dd> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow back-

end. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow can load files for only its format. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>30.65 deshake# TOC</h3>

Attempt to fix small changes in horizontal and/or vertical shift. This filter helps remove camera shake from hand-holding a camera, bumping a tripod, moving on a vehicle, etc. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Specify a rectangular area where to limit the search for motion vectors. If desired the search for motion vectors can be limited to a rectangular area of the frame defined by its top left corner, width and height. These parameters have the same meaning as the drawbox filter which can be used to visualise the position of the bounding box. This is useful when simultaneous movement of subjects within the frame might be confused for camera motion by the motion vector search. If any or all of <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> are set to -1 then the full frame is used. This allows later options to be set without specifying the bounding box for the motion vector search. Default - search the whole frame. </dd> <dt><samp class=”option”>rx</samp></dt> <dt><samp class=”option”>ry</samp></dt> <dd> Specify the maximum extent of movement in x and y directions in the range 0-64 pixels. Default 16. </dd> <dt><samp class=”option”>edge</samp></dt> <dd> Specify how to generate pixels to fill blanks at the edge of the frame. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>blank, 0</samp>’</dt> <dd> Fill zeroes at blank locations </dd> <dt>‘<samp class=”samp”>original, 1</samp>’</dt> <dd> Original image at blank locations </dd> <dt>‘<samp class=”samp”>clamp, 2</samp>’</dt> <dd> Extruded edge value at blank locations </dd> <dt>‘<samp class=”samp”>mirror, 3</samp>’</dt> <dd> Mirrored edge at blank locations </dd> </dl> Default value is ‘<samp class=”samp”>mirror</samp>’.

 </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Specify the blocksize to use for motion search. Range 4-128 pixels, default 8. </dd> <dt><samp class="option">contrast</samp></dt> <dd> Specify the contrast threshold for blocks. Only blocks with more than the specified contrast (difference between darkest and lightest pixels) will be considered. Range 1-255, default 125. </dd> <dt><samp class="option">search</samp></dt> <dd> Specify the search strategy. Available values are: <dl class="table"> <dt>‘<samp class="samp">exhaustive, 0</samp>’</dt> <dd> Set exhaustive search </dd> <dt>‘<samp class="samp">less, 1</samp>’</dt> <dd> Set less exhaustive search. </dd> </dl> Default value is ‘<samp class="samp">exhaustive</samp>’. </dd> <dt><samp class="option">filename</samp></dt> <dd> If set then a detailed log of the motion search is written to the specified file. </dd> </dl>

 <h3 class="section">30.66 despill# TOC</h3>

Remove unwanted contamination of foreground colors, caused by reflected color of greenscreen or bluescreen. This filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Set what type of despill to use. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how spillmap will be generated. </dd> <dt><samp class="option">expand</samp></dt> <dd> Set how much to get rid of still remaining spill. </dd> <dt><samp class="option">red</samp></dt> <dd> Controls amount of red in spill area. </dd> <dt><samp class="option">green</samp></dt> <dd> Controls amount of green in spill area. Should be -1 for greenscreen. </dd> <dt><samp class="option">blue</samp></dt> <dd> Controls amount of blue in spill area. Should be -1 for bluescreen. </dd> <dt><samp class="option">brightness</samp></dt> <dd> Controls brightness of spill area, preserving colors. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Modify alpha from generated spillmap. </dd> </dl>

 <h4 class="subsection">30.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.67 detelecine# TOC</h3>

Apply an exact inverse of the telecine operation. It requires a predefined pattern specified using the pattern option which must be the same as that passed to the telecine filter. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> A number representing position of the first frame with respect to the telecine pattern. This is to be used if the stream is cut. The default value is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>30.68 dilation# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>30.68.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.69 displace# TOC</h3>

Displace pixels as indicated by second and third input stream. It takes three input streams and outputs one stream, the first input is the source, and second and third input are displacement maps. The second input specifies how much to displace pixels along the x-axis, while the third

input specifies how much to displace pixels along the y-axis. If one of displacement map streams terminates, last frame from that displacement map will be used. Note that once generated, displacements maps can be reused over and over again. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">edge</samp></dt> <dd> Set displace behavior for pixels that are out of range. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank</samp>’</dt> <dd> Missing pixels are replaced by black pixels. </dd> <dt>‘<samp class="samp">smear</samp>’</dt> <dd> Adjacent pixels will spread out to replace missing pixels. </dd> <dt>‘<samp class="samp">wrap</samp>’</dt> <dd> Out of range pixels are wrapped so they point to pixels of other side. </dd> <dt>‘<samp class="samp">mirror</samp>’</dt> <dd> Out of range pixels will be replaced with mirrored pixels. </dd> </dl> Default is ‘<samp class="samp">smear</samp>’. </dd> </dl>

 <h4 class="subsection">30.69.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add ripple effect to rgb input of video size hd720: <pre class="example-preformatted">ffmpeg -i INPUT -f lavfi -i nullsrc=s=hd720,lutrgb=128:128:128 -f lavfi -i nullsrc=s=hd720,geq='r=128+30*sin(2*PI*X/400+T):g=128+30*sin(2*PI*X/400+T):b=128+30*sin -lavfi '[0][1][2]displace' OUTPUT </pre>

Add wave effect to rgb input of video size hd720: <pre class="example-preformatted">ffmpeg -i INPUT -f lavfi -i nullsrc=hd720,geq='r=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):g=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):b=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T))' -lavfi '[1]split[x][y],[0][x][y]displace' OUTPUT </pre>

 <h3 class="section">30.70 dnn_classify# TOC</h3>

Do classification with deep neural networks based on bounding boxes. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts only openvino now, tensorflow backends will be added. </dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and

its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0, and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend For tensorflow backend, you can set its configs with <samp class=”option”>sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs for your system. </dd> </dl>

 <h3 class=”section”>30.71 dnn_detect# TOC</h3>

Do object detection with deep neural networks. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt> <dd> OpenVINO backend. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. The TensorFlow and ONNX Runtime backends accept multiple output names separated by ‘<samp class=”samp”>&</samp>’, e.g. <samp class=”option”>output=yolo_13&yolo_26</samp>. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0 (usually it is ’background’),

and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>model_type</samp></dt> <dd> Set the detection model output format. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>ssd</samp>’</dt> <dd> Single-stage detector (default). </dd> <dt>‘<samp class=”samp”>yolo</samp>’</dt> <dd> YOLO v1/v2. </dd> <dt>‘<samp class=”samp”>yolov3</samp>’</dt> <dd> YOLOv3. </dd> <dt>‘<samp class=”samp”>yolov4</samp>’</dt> <dd> YOLOv4. </dd> </dl> </dd> <dt><samp class=”option”>anchors</samp></dt> <dd> Anchor box dimensions, separated by ‘<samp class=”samp”>&</samp>’. Required for YOLO-family models. The list is consumed in output order, so the anchors of the first output come first: with <samp class=”option”>output=yolo_13&yolo_26</samp> the leading pairs belong to <code class=”code”>yolo_13</code>. </dd> <dt><samp class=”option”>nb_classes</samp></dt> <dd> Number of detection classes. Required for YOLO-family models. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. </dd> </dl>

 <h4 class=”subsection”>30.71.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Run SSD object detection with an ONNX model (single output, DirectML on Windows): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=300:300,format=rgb24,dnn_detect=
:model=ssd.onnx:model_type=ssd:confidence=0.5:device=dml" -f null -
</pre>

Run YOLOv3 object detection with an ONNX model (two outputs, CPU): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=416:416,format=rgb24,dnn_detect=dnn_backend=onnx
:model=yolov3.onnx:output=yolo_13&yolo_26:model_type=yolov3:nb_classes=80
:anchors=116&90&156&198&373&326&30&61&62&45&59&
-f null - </pre>

 <h3 class=”section”>30.72 dnn_processing# TOC</h3>

Do image processing with deep neural networks. It works together with another filter which converts the pixel format of the Frame to what the dnn network requires. The filter accepts the following options: <dl

class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt>
<dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>tensorflow</samp>’</dt>
<dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www
and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code>
 </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt>
<dd> OpenVINO backend. To enable this backend you need to build and install the OpenVINO for C library (see https://github.com/openvinotoolkit/openvino/blob/master/build-instruction.md) and configure FFmpeg with <code class=”code”>--enable-libopenvino</code> (–extra-cflags=-I... –extra-ldflags=-L... might be needed if the header files and libraries are not installed into system path) </dd> <dt>‘<samp class=”samp”>torch</samp>’</dt>
<dd> Libtorch backend. To enable this backend you need to build and install Libtorch for C++ library. Please download cxx11 ABI version (see https://pytorch.org/get-started/locally) and configure FFmpeg with <code class=”code”>--enable-libtorch --extra-cflags=-I/libtorch_root/libtorch/include --extra-cflags=-I/libtorch_root/libtorch/include/torch/csrc/api/include --extra-ldflags=-L/libtorch_root/libtorch/lib/</code> When FFmpeg is built with CUDA support and the input frames use <code class=”code”>AV_PIX_FMT_CUDA</code> with an underlying pixel format of <code class=”code”>rgb0</code>, <code class=”code”>bgr0</code>, <code class=”code”>rgb24</code>, or <code class=”code”>bgr24</code>, the Torch backend maps GPU frame buffers directly into LibTorch CUDA tensors using zero-copy memory mapping. This avoids host-device PCIe transfers and can significantly reduce inference latency. The zero-copy path is selected automatically when the input format qualifies; no additional options are required. When the input format is not <code class=”code”>AV_PIX_FMT_CUDA</code>, the backend uses the standard CPU tensor path. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. To enable this backend you need to install the ONNX Runtime library (see https://onnxruntime.ai/) and configure FFmpeg with <code class=”code”>--enable-libonnxruntime</code>.
 The current ONNX Runtime backend expects 4-D input and output tensors with NCHW layout and 32-bit floating-point element type (ONNX <code class=”code”>FLOAT</code>); models with integer or other element types (e.g. <code class=”code”>UINT8</code>) are not supported and will be rejected at load time. Models using NHWC layout or other ranks are not yet supported. Only single-input models are supported; the backend binds exactly one input tensor when running the model. The <samp class=”option”>input</samp>

and <samp class=”option”>output</samp> options are optional for the ONNX Runtime backend; when they are omitted the backend resolves the tensor names from the session. Multiple output names may be supplied separated by ‘<samp class=”samp”>&</samp>’ (e.g. <samp class=”option”>output=out_a&out_b</samp>); however for <code class=”code”>dnn_processing</code> only the first output tensor is used for frame post-processing. The ONNX Runtime backend runs inference synchronously using a single inference request. The shared <samp class=”option”>async</samp> and <samp class=”option”>nireq</samp> options therefore have no effect for <code class=”code”>dnn_backend=onnx</code>; inference always runs synchronously regardless of their values. When FFmpeg is built with Direct3D 12 support and this build’s ONNX Runtime provides a working DirectML execution provider, setting <samp class=”option”>device=dml</samp> together with <code class=”code”>AV_PIX_FMT_D3D12</code> NV12 hardware frames as input (e.g. via <code class=”code”>-hwaccel d3d12va -hwaccel_output_format d3d12</code>) keeps frames in VRAM for the whole filter, from the decoder’s output through DNN inference and back out, without a host round-trip. Software-frame input retains the host I/O path. D3D12 input requires successful DirectML negotiation; unsupported hardware-input configurations fail during configuration. At runtime, <samp class=”file”>DirectML.dll</samp> and <samp class=”file”>d3dcompiler_47.dll</samp> must be loadable from the application directory or System32. Currently, only single-channel (luma-only), <code class=”code”>FLOAT</code>/NCHW models with a <code class=”code”>[1,1,H,W]</code> output tensor and even input/output frame dimensions are supported; the chroma plane is copied or bilinearly upscaled on the GPU without going through the model. The output frames are D3D12 hardware frames, so they must either be consumed by a D3D12 encoder or downloaded explicitly with <code class=”code”>hwdownload,format=nv12</code>. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO, Libtorch, and ONNX Runtime backends can load files for only their respective formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. For tensorflow backend, you can set its configs

with <samp class="option">sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. </dd> <dt><samp class="option">async</samp></dt> <dd> Use DNN async inference (default: 1). When enabled, frames are submitted to the backend without blocking the caller. Supported by the Torch, OpenVINO, and TensorFlow backends; the ONNX Runtime backend ignores this option. </dd> <dt><samp class="option">nireq</samp></dt> <dd> Number of concurrent inference requests (default: 0, meaning auto). When set to 0, the backend automatically determines an appropriate number of requests. Higher values increase throughput by allowing multiple frames to be in flight simultaneously. </dd> <dt><samp class="option">batch_size</samp></dt> <dd> Number of frames to batch into a single inference request (default: 1). Values greater than 1 cause the backend to accumulate frames and concatenate them along the batch dimension before running the model. This can significantly increase throughput on GPUs at the cost of slightly higher per-frame latency. Currently supported by the OpenVINO and Torch backends. </dd> <dt><samp class="option">device</samp></dt> <dd> Set the device to run the model. For the ONNX Runtime backend this selects the execution provider: <code class="code">cpu</code> (default), <code class="code">cuda</code> (NVIDIA GPU), <code class="code">dml</code> (DirectML, Windows only) or <code class="code">vitisai</code> (AMD Ryzen AI NPU). </dd> <dt><samp class="option">device_id</samp></dt> <dd> Set the device index used by GPU execution providers (e.g. <code class="code">cuda</code> or <code class="code">dml</code>) for the ONNX Runtime backend. Default is 0. </dd> <dt><samp class="option">threads_per_operation</samp></dt> <dd> ONNX Runtime backend only. Set the number of CPU threads used per ONNX Runtime operator when running with <code class="code">device=cpu</code>. Default is 0 (let ONNX Runtime choose automatically). Has no effect for GPU/NPU providers. </dd> </dl>

 <h4 class="subsection">30.72.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Remove rain in rgb24 frame with can.pb (see derain filter): <pre class="example-preformatted">./ffmpeg -i rain.jpg -vf format=rgb24,dnn_processing=dnn_backend=tensorflow:model=can.pb:input=x:output=y derain.jpg </pre>

Handle the Y channel with srcnn.pb (see sr filter) for frame with yuv420p (planar YUV formats supported): <pre class="example-preformatted">./ffmpeg -i 480p.jpg -vf format=yuv420p,scale=w=iw*2:h=ih*2,dnn_processing=dnn_backend=tensorflow:model=srcn

-y srcnn.jpg </pre>

Handle the Y channel with espcn.pb (see sr filter), which changes frame size, for format yuv420p (planar YUV formats supported), please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. <pre class="example-preformatted">./ffmpeg -i 480p.jpg -vf format=yuv420p,dnn_processing=dnn_backend=
-y tmp.espcn.jpg </pre>

Perform zero-copy GPU inference with the Libtorch backend. Because there is currently no GPU-side filter to convert from YUV to RGB, <code class="code">hwupload</code> is used to upload CPU RGB frames into device memory. Once a GPU-side YUV-to-RGB filter becomes available, the <code class="code">hwupload</code> step will no longer be necessary and all frames can stay in device memory throughout the pipeline: <pre class="example-preformatted">./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda -i input.mp4 \ -vf "format=rgb24,hwupload,dnn_processing=dnn_backend=torch:model=my_model.pt,hwdownload,form
\ -y output.mp4 </pre>

Run a single-channel (luma-only) super-resolution model on D3D12 hardware frames with the ONNX Runtime backend, keeping frames in VRAM from decode through inference (Windows only, requires a build with Direct3D 12 support and <code class="code">--enable-libonnxruntime</code> against a DirectML-capable ONNX Runtime). The D3D12 output frames are encoded directly by a D3D12 hardware encoder: <pre class="example-preformatted">./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml
\ -c:v h264_d3d12va -bf 0 -y output.mp4 </pre>

Same as above, but download the result to software frames after inference so that a software encoder can be used: <pre class="example-preformatted">./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml,hwdownloa
\ -y output.mp4 </pre>

Process rgb24 frames with a TorchScript model using the Libtorch backend: <pre class="example-preformatted">./ffmpeg -i input.jpg -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_model.pt output.jpg </pre>

Process rgb24 frames with the Libtorch backend using batch inference (four frames per request): <pre class="example-preformatted">./ffmpeg -i input.mp4 -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_mo
-y output.mp4 </pre>

 <h3 class="section">30.73 drawbox<span

class=”pull-right”># TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the box edge color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class=”code”>fill</code> will create a filled box. Default value is <code class=”code”>3</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With value <code class=”code”>1</code>, the pixels of the painted box will overwrite the video’s color and alpha pixels. Default is <code class=”code”>0</code>, which composites the box onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> and <var class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class=”option”>box_source</samp></dt>

<dd> Box source can be set as side_data_detection_bboxes if you want to use box data in detection bboxes of side data. If <var class="var">box_source</var> is set, the <var class="var">x</var>, <var class="var">y</var>, <var class="var">width</var> and <var class="var">height</var> will be ignored and still use box data in detection bboxes of side data. So please do not use this parameter if you were not sure about the box source. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. These constants allow the <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, <var class="var">h</var> and <var class="var">t</var> expressions to refer to each other, so you may for example specify <code class="code">y=x/dar</code> or <code class="code">h=w/dar</code>. </dd> </dl>

 <h4 class="subsection">30.73.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h4 class="subsection">30.73.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.74 drawgraph# <a class="anchor

hidden-xs”href=”#toc-drawgraph” aria-hidden=”true”>TOC</h3>
Draw a graph using input video metadata. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>m1</samp></dt>
<dd> Set 1st frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class=”option”>fg1</samp></dt>
<dd> Set 1st foreground color expression. </dd> <dt><samp class=”option”>m2</samp></dt> <dd> Set 2nd frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class=”option”>fg2</samp></dt> <dd> Set 2nd foreground color expression. </dd> <dt><samp class=”option”>m3</samp></dt>
<dd> Set 3rd frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class=”option”>fg3</samp></dt>
<dd> Set 3rd foreground color expression. </dd> <dt><samp class=”option”>m4</samp></dt> <dd> Set 4th frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class=”option”>fg4</samp></dt> <dd> Set 4th foreground color expression. </dd> <dt><samp class=”option”>min</samp></dt>
<dd> Set minimal value of metadata value. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Set maximal value of metadata value. </dd> <dt><samp class=”option”>bg</samp></dt>
<dd> Set graph background color. Default is white. </dd>
<dt><samp class=”option”>mode</samp></dt> <dd> Set graph mode. Available values for mode is: <dl class=”table”>
<dt>‘<samp class=”samp”>bar</samp>’</dt> <dt>‘<samp class=”samp”>dot</samp>’</dt> <dt>‘<samp class=”samp”>line</samp>’</dt>
</dl>

Default is <code class=”code”>line</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide mode.
Available values for slide is: <dl class=”table”> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Draw new frame when right border is reached. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt
<dd> Replace old columns with new ones. </dd> <dt>‘<samp class=”samp”>scroll</samp>’</dt> <dd> Scroll from right to left. </dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> Scroll from left to right. </dd> <dt>‘<samp class=”samp”>picture</samp>’</dt> <dd> Draw single picture.
</dd> </dl>

Default is <code class=”code”>frame</code>. </dd>
<dt><samp class=”option”>size</samp></dt> <dd> Set size of graph video. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. The default value is <code class=”code”>900x256</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class=”code”>25</code>. The foreground color expressions

can use the following variables: <dl class="table"> <dt><samp class="option">MIN</samp></dt> <dd> Minimal value of metadata value. </dd> <dt><samp class="option">MAX</samp></dt> <dd> Maximal value of metadata value. </dd> <dt><samp class="option">VAL</samp></dt> <dd> Current metadata key value. </dd> </dl>

The color is defined as 0xAABBGGRR. </dd> </dl>

Example using metadata from signalstats filter: <pre class="example-preformatted">signalstats,drawgraph=lavfi.signalst </pre>

Example using metadata from ebur128 filter: <pre class="example-preformatted">ebur128=metadata=1,adrawgraph= 120:max=5 </pre>

 <h3 class="section">30.75 drawgrid# TOC</h3>

Draw a grid on the input image. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The expressions which specify the coordinates of some point of grid intersection (meant to configure offset). Both default to 0. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> The expressions which specify the width and height of the grid cell, if 0 they are interpreted as the input width and height, respectively, minus <code class="code">thickness</code>, so image gets framed. Default to 0. </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the grid. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. If the special value <code class="code">invert</code> is used, the grid color is the same as the video with inverted luma. </dd> <dt><samp class="option">thickness, t</samp></dt> <dd> The expression which sets the thickness of the grid line. Default value is <code class="code">1</code>. See below for the list of accepted constants. </dd> <dt><samp class="option">replace</samp></dt> <dd> Applicable if the input has alpha. With <code class="code">1</code> the pixels of the painted grid will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the grid onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var

class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input grid cell width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y coordinates of some point of grid intersection (meant to configure offset). </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn cell. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The thickness of the drawn cell. These constants allow the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var>, <var class=”var”>h</var> and <var class=”var”>t</var> expressions to refer to each other, so you may for example specify <code class=”code”>y=x/dar</code> or <code class=”code”>h=w/dar</code>. </dd> </dl>

 <h4 class=”subsection”>30.75.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Draw a grid with cell 100x100 pixels, thickness 2 pixels, with color red and an opacity of 50%: <pre class=”example-preformatted”>drawgrid=width=100:height=100:thickness=2:color=red@0.5 </pre>

Draw a white 3x3 grid with an opacity of 50%: <pre class=”example-preformatted”>drawgrid=w=iw/3:h=ih/3:t=2:c=white@0.5 </pre>

 <h4 class=”subsection”>30.75.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>30.76 drawtext# <a class=”anchor hidden-xs”href=”#toc-drawtext”

aria-hidden=”true”>TOC</h3>

Draw a text string or text from a specified file on top of a video, using the libfreetype library. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libfreetype</code> and <code class=”code”>--enable-libharfbuzz</code>. To enable default font fallback and the <var class=”var”>font</var> option you need to configure FFmpeg with <code class=”code”>--enable-libfontconfig</code>. To enable the <var class=”var”>text_shaping</var> option, you need to configure FFmpeg with <code class=”code”>--enable-libfribidi</code>. <h4 class=”subsection”>30.76.1 Syntax# TOC</h4>

It accepts the following parameters:

<dl class=”table”>

<dt><samp class=”option”>box</samp></dt>

<dd> Used to draw a box around text using the background color.

The value must be either 1 (enable) or 0 (disable).

The default value of <var class=”var”>box</var> is 0.

</dd>

<dt><samp class=”option”>boxborderw</samp></dt>

<dd> Set the width of the border to be drawn around the box using <var class=”var”>boxcolor</var>.

The value must be specified using one of the following formats:

<ul class=”itemize mark-bullet”>

<code class=”code”>boxborderw=10</code> set the width of all the borders to 10

<code class=”code”>boxborderw=10|20</code> set the width of the top and bottom borders to 10

and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30</code> set the width of the top border to 10, the width

of the bottom border to 30 and the width of the left and right borders to 20

<code class="code">boxborderw=10|20|30|40</code> set the borders width to 10 (top), 20 (right),

30 (bottom), 40 (left)

The default value of <var class="var">boxborderw</var> is "0".

</dd>

<dt><samp class="option">boxcolor</samp></dt>

<dd> The color to be used for drawing box around text. For the syntax of this

option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">boxcolor</var> is "white".

</dd>

<dt><samp class="option">line_spacing</samp></dt>

<dd> Set the line spacing in pixels. The default value of <var class="var">line_spacing</var> is 0.

</dd>

<dt><samp class="option">text_align</samp></dt>

<dd> Set the vertical and horizontal alignment of the text with respect to the box boundaries.

The value is combination of flags, one for the vertical alignment (T=top,

M=middle, B=bottom) and one for the horizontal alignment (L=left, C=center, R=right).

Please note that tab characters are only supported with the left horizontal alignment.

</dd>

<dt><samp class="option">y_align</samp></dt>

<dd> Specify what the <var class=”var”>y</var> value is referred to. Possible values are:

<ul class=”itemize mark-bullet”>

<code class=”code”>text</code> the top of the highest glyph of the first text line is placed at <var class=”var”>y</var>

<code class=”code”>baseline</code> the baseline of the first text line is placed at <var class=”var”>y</var>

<code class=”code”>font</code> the baseline of the first text line is placed at <var class=”var”>y</var> plus the

ascent (in pixels) defined in the font metrics

The default value of <var class=”var”>y_align</var> is "text" for backward compatibility.

</dd>

<dt><samp class=”option”>borderw</samp></dt>

<dd> Set the width of the border to be drawn around the text using <var class=”var”>bordercolor</var>.

The default value of <var class=”var”>borderw</var> is 0.

</dd>

<dt><samp class=”option”>bordercolor</samp></dt>

<dd> Set the color to be used for drawing border around text. For the syntax of this

option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

The default value of <var class=”var”>bordercolor</var> is "black".

</dd>

<dt><samp class=”option”>expansion</samp></dt>

<dd> Select how the <var class=”var”>text</var> is expanded. Can be either <code class=”code”>none</code>,

<code class=”code”>strftime</code> (deprecated) or <code class=”code”>normal</code> (default). See the

Text expansion section below for details.

</dd>

<dt><samp class=”option”>basetime</samp></dt>

<dd> Set a start time for the count. Value is in microseconds. Only applied

in the deprecated <code class=”code”>strftime</code> expansion mode. To emulate in normal expansion

mode use the <code class=”code”>pts</code> function, supplying the start time (in seconds)

as the second argument.

</dd>

<dt><samp class=”option”>fix_bounds</samp></dt>

<dd> If true, check and fix text coords to avoid clipping.

</dd>

<dt><samp class=”option”>fontcolor</samp></dt>

<dd> The color to be used for drawing fonts. For the syntax of this option, check

the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

The default value of <var class=”var”>fontcolor</var> is "black".

</dd>

<dt><samp class=”option”>fontcolor_expr</samp></dt>

<dd> String which is expanded the same way as <var class=”var”>text</var> to obtain dynamic

<var class=”var”>fontcolor</var> value. By default this option has empty value and is not

processed. When this option is set, it overrides <var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> The font family to be used for drawing text. By default Sans.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> The font file to be used for drawing text. The path must be included.

This parameter is mandatory if the fontconfig support is disabled.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Draw the text applying alpha blending. The value can

be a number between 0.0 and 1.0.

The expression accepts the same variables <var class="var">x, y</var> as well.

The default value is 1.

Please see <var class="var">fontcolor_expr</var>.

</dd>

<dt><samp class="option">fontsize</samp></dt>

<dd> The font size to be used for drawing text.

The default value of <var class="var">fontsize</var> is 16.

</dd>

<dt><samp class="option">text_shaping</samp></dt>

<dd> If set to 1, attempt to shape the text (for example, reverse the order of

right-to-left text and join Arabic characters) before drawing it.

Otherwise, just draw the text exactly as given.

By default 1 (if supported).

</dd>
<dt><samp class=”option”>ft_load_flags</samp></dt>
<dd> The flags to be used for loading the fonts.

The flags map the corresponding flags supported by libfreetype, and are a combination of the following values:
<dl class=”table”>
<dt><var class=”var”>default</var></dt>
<dt><var class=”var”>no_scale</var></dt>
<dt><var class=”var”>no_hinting</var></dt>
<dt><var class=”var”>render</var></dt>
<dt><var class=”var”>no_bitmap</var></dt>
<dt><var class=”var”>vertical_layout</var></dt>
<dt><var class=”var”>force_autohint</var></dt>
<dt><var class=”var”>crop_bitmap</var></dt>
<dt><var class=”var”>pedantic</var></dt>
<dt><var class=”var”>ignore_global_advance_width</var></dt>
<dt><var class=”var”>no_recurse</var></dt>
<dt><var class=”var”>ignore_transform</var></dt>
<dt><var class=”var”>monochrome</var></dt>
<dt><var class=”var”>linear_design</var></dt>
<dt><var class=”var”>no_autohint</var></dt>
</dl>
Default value is "default".

For more information consult the documentation for the FT_LOAD_* libfreetype flags.

</dd>

<dt><samp class="option">shadowcolor</samp></dt>

<dd> The color to be used for drawing a shadow behind the drawn text. For the

syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the

ffmpeg-utils manual.

The default value of <var class="var">shadowcolor</var> is "black".

</dd>

<dt><samp class="option">boxw</samp></dt>

<dd> Set the width of the box to be drawn around text.

The default value of <var class="var">boxw</var> is computed automatically to match the text width

</dd>

<dt><samp class="option">boxh</samp></dt>

<dd> Set the height of the box to be drawn around text.

The default value of <var class="var">boxh</var> is computed automatically to match the text height

</dd>

<dt><samp class="option">shadowx</samp></dt>

<dt><samp class="option">shadowy</samp></dt>

<dd> The x and y offsets for the text shadow position with respect to the

position of the text. They can be either positive or negative

values. The default value for both is "0".

</dd>

<dt><samp class="option">start_number</samp></dt>

<dd> The starting frame number for the n/frame_num variable. The default value

is "0".

</dd>

<dt><samp class="option">tabsize</samp></dt>

<dd> The size in number of spaces to use for rendering the tab.

Default value is 4.

</dd>

<dt><samp class="option">timecode</samp></dt>

<dd> Set the initial timecode representation in "hh:mm:ss[:;.]ff"

format. It can be used with or without text parameter. <var class="var">timecode_rate</var>

option must be specified.

</dd>

<dt><samp class="option">timecode_rate, rate, r</samp></dt>

<dd> Set the timecode frame rate (timecode only). Value will be rounded to nearest

integer. Minimum value is "1".

Drop-frame timecode is supported for frame rates 30 & 60.

</dd>

<dt><samp class="option">tc24hmax</samp></dt>

<dd> If set to 1, the output of the timecode option will wrap around at 24 hours.

Default is 0 (disabled).

</dd>

<dt><samp class="option">text</samp></dt>

<dd> The text string to be drawn. The text must be a sequence of UTF-8

encoded characters.

This parameter is mandatory if no file is specified with the parameter

<var class="var">textfile</var>.

</dd>

<dt><samp class="option">textfile</samp></dt>

<dd> A text file containing text to be drawn. The text must be a sequence

of UTF-8 encoded characters.

This parameter is mandatory if no text string is specified with the

parameter <var class="var">text</var>.

If both <var class="var">text</var> and <var class="var">textfile</var> are specified, an error is thrown.

</dd>

<dt><samp class="option">text_source</samp></dt>

<dd> Text source should be set as side_data_detection_bboxes if you want to use text data in

detection bboxes of side data.

If text source is set, <var class="var">text</var> and <var class="var">textfile</var> will be ignored and still use

text data in detection bboxes of side data. So please do not use this parameter

if you are not sure about the text source.

</dd>

<dt><samp class="option">reload</samp></dt>

<dd> The <var class="var">textfile</var> will be reloaded at specified frame interval.

Be sure to update <var class="var">textfile</var> atomically, or it may be read partially,

or even fail.

Range is 0 to INT_MAX. Default is 0.

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> The expressions which specify the offsets where text will be drawn

within the video frame. They are relative to the top/left border of the

output image.

The default value of <var class="var">x</var> and <var class="var">y</var> is "0".

See below for the list of accepted constants and functions.

</dd>

</dl>

The parameters for <var class="var">x</var> and <var class="var">y</var> are expressions containing the

following constants and functions:

<dl class="table">

<dt><samp class="option">dar</samp></dt>

<dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var>

</dd>

<dt><samp class="option">hsub</samp></dt>

<dt><samp class="option">vsub</samp></dt>

<dd> horizontal and vertical chroma subsample values. For example for the

pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1.

</dd>

<dt><samp class="option">line_h, lh</samp></dt>
<dd> the height of each text line

</dd>
<dt><samp class="option">main_h, h, H</samp></dt>
<dd> the input height

</dd>
<dt><samp class="option">main_w, w, W</samp></dt>
<dd> the input width

</dd>
<dt><samp class="option">max_glyph_a, ascent</samp></dt>
<dd> the maximum distance from the baseline to the highest/upper grid coordinate used to place a glyph outline point, for all the rendered glyphs.
It is a positive value, due to the grid’s orientation with the Y axis upwards.

</dd>
<dt><samp class="option">max_glyph_d, descent</samp></dt>
<dd> the maximum distance from the baseline to the lowest grid coordinate used to place a glyph outline point, for all the rendered glyphs.
This is a negative value, due to the grid’s orientation, with the Y axis upwards.

</dd>
<dt><samp class="option">max_glyph_h</samp></dt>
<dd> maximum glyph height, that is the maximum height for all the glyphs contained in the rendered text, it is equivalent to <var class="var">ascent</var>
-

<var class="var">descent</var>.

</dd>

<dt><samp class="option">max_glyph_w</samp></dt>

<dd> maximum glyph width, that is the maximum width for all the glyphs contained in the rendered text

</dd>

<dt><samp class="option">font_a</samp></dt>

<dd> the ascent size defined in the font metrics

</dd>

<dt><samp class="option">font_d</samp></dt>

<dd> the descent size defined in the font metrics

</dd>

<dt><samp class="option">top_a</samp></dt>

<dd> the maximum ascender of the glyphs of the first text line

</dd>

<dt><samp class="option">bottom_d</samp></dt>

<dd> the maximum descender of the glyphs of the last text line

</dd>

<dt><samp class="option">n</samp></dt>

<dd> the number of input frame, starting from 0

</dd>

<dt><samp class="option">rand(min, max)</samp></dt>

<dd> return a random number included between <var class="var">min</var> and <var class="var">max</var>

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> The input sample aspect ratio.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds, NAN if the input timestamp is unknown

</dd>

<dt><samp class="option">text_h, th</samp></dt>

<dd> the height of the rendered text

</dd>

<dt><samp class="option">text_w, tw</samp></dt>

<dd> the width of the rendered text

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> the x and y offset coordinates where the text is drawn.

These parameters allow the <var class="var">x</var> and <var class="var">y</var> expressions to refer

to each other, so you can for example specify <code class="code">y=x/dar</code>.

</dd>

<dt><samp class="option">pict_type</samp></dt>

<dd> A one character description of the current frame’s picture type.

</dd>

<dt><samp class=”option”>pkt_pos</samp></dt>

<dd> The current packet’s position in the input file or stream

(in bytes, from the start of the input). A value of -1 indicates

this info is not available.

</dd>

<dt><samp class=”option”>duration</samp></dt>

<dd> The current packet’s duration, in seconds.

</dd>

<dt><samp class=”option”>pkt_size</samp></dt>

<dd> The current packet’s size (in bytes).

</dd>

</dl>

 <h4 class=”subsection”>30.76.2 Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>strftime</code>, the filter recognizes

sequences accepted by the <code class=”code”>strftime</code> C function in the provided

text and expands them accordingly. Check the documentation of

<code class=”code”>strftime</code>. This feature is deprecated in favor of <code class=”code”>normal</code>

expansion with the <code class=”code”>gmtime</code> or <code class=”code”>localtime</code> expansion

functions.

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim.

If <samp class=”option”>expansion</samp> is set to <code class=”code”>normal</code> (which is the default),

the following expansion mechanism is used.

The backslash character ‘<samp class=”samp”>\</samp>’, followed by any character, always expands to

the second character.

Sequences of the form <code class=”code”>%{...}</code> are expanded. The text between the

braces is a function name, possibly followed by arguments separated by ’:’.

If the arguments contain special characters or delimiters (’:’ or ’}’),

they should be escaped.

Note that they probably must also be escaped as the value for the <samp class=”option”>text</samp>

option in the filter argument string and as the filter argument in the

filtergraph description, and possibly also for the shell, that makes up to four

levels of escaping; using a text file with the <samp class=”option”>textfile</samp> option avoids

these problems.

The following functions are available:

<dl class=”table”>

<dt><code class=”command”>expr, e</code></dt>

<dd> The expression evaluation result.

It must take one argument specifying the expression to be evaluated,

which accepts the same constants and functions as the <var class=”var”>x</var> and

<var class=”var”>y</var> values. Note that not all constants should be used, for

example the text size is not known when evaluating the expression, so

the constants <var class=”var”>text_w</var> and <var class=”var”>text_h</var> will have an undefined

value.

</dd>

<dt><code class=”command”>expr_int_format, eif</code></dt>

<dd> Evaluate the expression’s value and output as formatted integer.

The first argument is the expression to be evaluated, just as for the <var class=”var”>expr</var> function.

The second argument specifies the output format. Allowed values are ‘<samp class=”samp”>x</samp>’,

‘<samp class=”samp”>X</samp>’, ‘<samp class=”samp”>d</samp>’ and ‘<samp class=”samp”>u</samp>’. They are treated exactly as in the

<code class=”code”>printf</code> function.

The third parameter is optional and sets the number of positions taken by the output.

It can be used to add padding with zeros from the left.

</dd>

<dt><code class=”command”>gmtime</code></dt>

<dd> The time at which the filter is running, expressed in UTC.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>localtime</code></dt>

<dd> The time at which the filter is running, expressed in the local time zone.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>metadata</code></dt>

<dd> Frame metadata. Takes one or two arguments.

The first argument is mandatory and specifies the metadata key.

The second argument is optional and specifies a default value, used when the

metadata key is not found or empty.

Available metadata can be identified by inspecting entries

starting with TAG included within each frame section

printed by running <code class=”code”>ffprobe -show_frames</code>.

String metadata generated in filters leading to

the drawtext filter are also available.

</dd>

<dt><code class=”command”>n, frame_num</code></dt>

<dd> The frame number, starting from 0.

</dd>

<dt><code class=”command”>pict_type</code></dt>

<dd> A one character description of the current picture type.

</dd>

<dt><code class="command">pts</code></dt>

<dd> The timestamp of the current frame.

It can take up to three arguments.

The first argument is the format of the timestamp; it defaults to <code class="code">flt</code>

for seconds as a decimal number with microsecond accuracy; <code class="code">hms</code> stands

for a formatted <var class="var">[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy.

<code class="code">gmtime</code> stands for the timestamp of the frame formatted as UTC time;

<code class="code">localtime</code> stands for the timestamp of the frame formatted as

local time zone time.

The second argument is an offset added to the timestamp.

If the format is set to <code class="code">hms</code>, a third argument <code class="code">24HH</code> may be

supplied to present the hour part of the formatted timestamp in 24h format

(00-23).

If the format is set to <code class="code">localtime</code> or <code class="code">gmtime</code>, a third

argument may be supplied: a <code class="code">strftime</code> C function format string.

By default, <var class="var">YYYY-MM-DD HH:MM:SS</var> format will be used.

</dd>

</dl>

 <h4 class="subsection">30.76.3 Commands# TOC</h4>

This filter supports altering parameters via commands: <dl class="table"> <dt><samp class="option">reinit</samp></dt> <dd> Alter existing filter parameters. Syntax for the argument is the same as for filter invocation, e.g.   <pre class="example-preformatted">fontsize=56:fontcolor=green:text='Hello World' </pre>

Full filter invocation with sendcmd would look like this: <pre class="example-preformatted">sendcmd=c='56.0 drawtext reinit fontsize=56\:fontcolor=green\:text=Hello\\ World' </pre>

If the entire argument can’t be parsed or applied as valid values then the filter will continue with its existing parameters. </dd> </dl>

The following options are also supported as commands: <ul class="itemize mark-bullet"> x y alpha fontsize fontcolor boxcolor bordercolor shadowcolor box boxw boxh boxborderw line_spacing text_align shadowx shadowy borderw

 <h4 class="subsection">30.76.4 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw "Test Text" with font FreeSerif, using the default values for the optional parameters.

 <pre class="example-preformatted">drawtext="fontfile=/usr/share/fonts/true
text='Test Text'" </pre>

Draw ’Test Text’ with font FreeSerif of size 24 at position x=100 and y=50 (counting from the top-left corner of the screen), text is yellow with a red box around it. Both the text and the box have an opacity of 20%.

 <pre class="example-preformatted">drawtext="fontfile=/usr/share/fonts/true
text='Test Text':\ x=100: y=50: fontsize=24: fontcolor=yellow@0.2: box=1: boxcolor=red@0.2" </pre>

Note that the double quotes are not necessary if spaces are not used within the parameter list. Show the text at the center of the video frame: <pre class="example-preformatted">drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=(w-text_w)/2:y=(h-text_h)/2" </pre>

Show the text at a random position, switching to a new position every 30 seconds: <pre class="example-preformatted">drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=if(eq(mod(t\,30)\,0)\,rand(0\,(w-text_w))\,x):y=if(eq(mod(t\,30)\,0)\,rand(0\,(h-text_h))\,y)" </pre>

Show a text line sliding from right to left in the last row of the video frame. The file <samp class="file">LONG_LINE</samp> is assumed to contain a single line with no newlines. <pre class="example-preformatted">drawtext="fontsize=15:fontfile=FreeSerif.ttf:text=LONG_LINE:y=h-line_h:x=-50*t" </pre>

Show the content of file <samp class="file">CREDITS</samp> off the bottom of the frame and scroll up. <pre class="example-preformatted">drawtext="fontsize=20:fontfile=FreeSerif.ttf:textfile=CREDITS:y=h-20*t" </pre>

Draw a single green letter "g", at the center of the input video. The glyph baseline is placed at half screen height. <pre class="example-preformatted">drawtext="fontsize=60:fontfile=FreeSerif.ttf:fo
max_glyph_w)/2:y=h/2-ascent" </pre>

Show text for 1 second every 3 seconds: <pre class="example-preformatted">drawtext="fontfile=FreeSerif.ttf:fontcolor=white:x=100:y=x/dar:enable
</pre>

Use fontconfig to set the font. Note that the colons need to be escaped. <pre class="example-preformatted">drawtext='fontfile=Linux Libertine O-40\\:style=Semibold:text=FFmpeg' </pre>

Draw "Test Text" with font size dependent on height of the video. <pre class="example-preformatted">drawtext="text='Test Text': fontsize=h/30: x=(w-text_w)/2: y=(h-text_h*2)" </pre>

Print the date of a real-time encoding (see documentation for the <code class="code">strftime</code> C function): <pre class="example-preformatted">drawtext='fontfile=FreeSans.ttf:text=%{localtime\:%a %b %d %Y}' </pre>

Show text fading in and out (appearing/disappearing): <pre class="example-preformatted">#!/bin/sh DS=1.0 # display start DE=10.0 # display end FID=1.5 # fade in duration FOD=5 # fade out duration ffplay -f lavfi "color,drawtext=text=TEST:fontsize=50:fontfile=FreeSerif.ttf:fontcolor_
clip(255*(1*between(t\\, $DS + $FID\\, $DE - $FOD) + ((t - $DS)/$FID)*between(t\\, $DS\\, $DS + $FID) + (-(t - $DE)/$FOD)*between(t\\, $DE - $FOD\\, $DE))\\, 0\\, 255) \\\\: x\\\\: 2 }" </pre>

Horizontally align multiple separate texts. Note that <samp class="option">max_glyph_a</samp> and the <samp class="option">fontsize</samp>

value are included in the <samp class="option">y</samp> offset. <pre class="example-preformatted">drawtext=fontfile=FreeSans.ttf:text=DOG:fontsize=max_glyph_a, drawtext=fontfile=FreeSans.ttf:text=cow:fontsize=24:x=80:y=20+24-max_glyph_a </pre>

Plot special <var class="var">lavf.image2dec.source_basename</var> metadata onto each frame if such metadata exists. Otherwise, plot the string "NA". Note that image2 demuxer must have option <samp class="option">-export_path_metadata 1</samp> for the special metadata fields to be available for filters. <pre class="example-preformatted">drawtext="fontsize=20:fontcolor=white:fontfile=FreeSans.ttf:text='%{metadata\:lavf.i
</pre>

For more information about libfreetype, check: http://www.freetype.org/. For more information about fontconfig, check: http://freedesktop.org/software/fontconfig/fontconfig-user.html. For more information about libfribidi, check: http://fribidi.org/. For more information about libharfbuzz, check: https://git

<h3 class="section">30.77 drawvg# TOC</h3>

Draw vector graphics on top of video frames, by executing a script written in a custom language called VGS (Vector Graphics Script). The documentation for the language can be found in <a data-manual="drawvg-reference" href="./drawvg-reference.html#Top">drawvg - Language Reference. A version of this reference with rendered examples can be found at the author’s site. Graphics are rendered using the cario 2D graphics library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-cairo</code>.
<h4 class="subsection">30.77.1 Parameters#
TOC</h4>

Either <code class="code">script</code> or <code class="code">file</code> must be set. <dl class="table"> <dt><samp class="option">s, script</samp></dt> <dd> Script source to draw the graphics. </dd> <dt><samp class="option">file</samp></dt> <dd> Path of the file to load the script source. </dd> </dl>

 <h4 class="subsection">30.77.2 Pixel

Formats# TOC</h4>

Since Cairo only supports RGB images, if the input video is something else (like YUV 4:2:0), before executing the script the video is converted to a format compatible with Cairo. Then, you have to use either the format filter, or the <code class="code">-pix_fmt</code> option, to convert it to the expected format in the output. <h4 class="subsection">30.77.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the outline of an ellipse.

 <pre class="example-preformatted">ffmpeg -i input.webm \ -vf 'drawvg=ellipse (w/2) (h/2) (w/3) (h/3) stroke' \ -pix_fmt yuv420p \ output.webm </pre>

 Draw a square rotating in the middle of the frame.

The script for drawvg is in a file <code class="code">draw.vgs</code>:   <pre class="example-preformatted">translate (w/2) (h/2) rotate t rect -100 -100 200 200 setcolor red@0.5 fill </pre>

Then: <pre class="example-preformatted">ffmpeg -i input.webm -vf 'drawvg=file=draw.vgs,format=yuv420p' output.webm </pre>

 <h3 class="section">30.78 edgedetect# TOC</h3>

Detect and draw edges. The filter uses the Canny Edge Detection algorithm. The filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">20/255</code>, and default value for <var class="var">high</var> is <code class="code">50/255</code>.

 </dd> <dt><samp class="option">mode</samp></dt> <dd> Define the drawing mode. <dl class="table"> <dt>‘<samp class="samp">wires</samp>’</dt> <dd> Draw white/gray wires on black background. </dd> <dt>‘<samp class="samp">colormix</samp>’</dt> <dd> Mix the colors to create a paint/cartoon effect. </dd> <dt>‘<samp class="samp">canny</samp>’</dt> <dd> Apply Canny edge detector on all selected planes. </dd> </dl> Default value is <var class="var">wires</var>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Select planes for filtering. By default all available planes are filtered. </dd> </dl>

 <h4 class="subsection">30.78.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Standard edge detection with custom values for the hysteresis thresholding: <pre class="example-preformatted">edgedetect=low=0.1:high=0.4 </pre>

Painting effect without thresholding: <pre class="example-preformatted">edgedetect=mode=colormix:high=0 </pre>

 <h3 class="section">30.79 elbg# TOC</h3>

Apply a posterize effect using the ELBG (Enhanced LBG) algorithm. For each input image, the filter will compute the optimal mapping from the input to the output given the codebook length, that is the number of distinct output colors. This filter accepts the following options. <dl class="table"> <dt><samp class="option">codebook_length, l</samp></dt> <dd> Set codebook length. The value must be a positive integer, and represents the number of distinct output colors. Default value is 256. </dd> <dt><samp class="option">nb_steps, n</samp></dt> <dd> Set the maximum number of iterations to apply for computing the optimal mapping. The higher the value the better the result and the higher the computation time. Default value is 1. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set a random seed, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">pal8</samp></dt> <dd> Set pal8 output pixel format. This option does not work with codebook length greater than 256. Default is disabled. </dd> <dt><samp class="option">use_alpha</samp></dt> <dd> Include alpha values in the quantization calculation. Allows creating palettized output images (e.g. PNG8) with multiple alpha smooth blending. </dd> </dl>

 <h3 class="section">30.80 entropy# TOC</h3>

Measure graylevel entropy in histogram of color channels of video frames. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Can be either <var class="var">normal</var> or <var class="var">diff</var>. Default is <var class="var">normal</var>. <var class="var">diff</var> mode measures entropy of histogram delta values, absolute differences between neighbour histogram values. </dd> </dl>

 <h3 class="section">30.81 epx# TOC</h3> Apply the EPX magnification filter which is designed for pixel art. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xEPX</code>, <code class="code">3</code> for <code class="code">3xEPX</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">30.82 eq# TOC</h3> Set brightness, contrast, saturation and approximate gamma adjustment. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. The value must be a float value in range <code class="code">-1000.0</code> to <code class="code">1000.0</code>. The default value is "1". </dd> <dt><samp class="option">brightness</samp></dt> <dd> Set the brightness expression. The value must be a float value in range <code class="code">-1.0</code> to <code class="code">1.0</code>. The default value is "0". </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the saturation expression. The value must be a float in range <code class="code">0.0</code> to <code class="code">3.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma</samp></dt> <dd> Set the gamma expression. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_r</samp></dt> <dd> Set the gamma expression for red. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_g</samp></dt> <dd> Set the gamma expression for green. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_b</samp></dt>

<dd> Set the gamma expression for blue. The value must be a float in range <code class=”code”>0.1</code> to <code class=”code”>10.0</code>. The default value is "1". </dd> <dt><samp class=”option”>gamma_weight</samp></dt> <dd> Set the gamma weight expression. It can be used to reduce the effect of a high gamma value on bright image areas, e.g. keep them from getting overamplified and just plain white. The value must be a float in range <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. A value of <code class=”code”>0.0</code> turns the gamma correction all the way down while <code class=”code”>1.0</code> leaves it at its full strength. Default is "1". </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for brightness, contrast, saturation and gamma expressions are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’. </dd> </dl>

The expressions accept the following parameters: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pos</samp></dt> <dd> byte position of the corresponding packet in the input file, NAN if unspecified; deprecated, do not use </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> </dl>

 <h4 class=”subsection”>30.82.1 Commands# TOC</h4> The filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>contrast</samp></dt> <dd> Set the contrast expression. </dd> <dt><samp class=”option”>brightness</samp></dt> <dd> Set the brightness expression. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set the saturation expression. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set the gamma expression. </dd> <dt><samp class=”option”>gamma_r</samp></dt> <dd> Set the gamma_r expression. </dd> <dt><samp class=”option”>gamma_g</samp></dt> <dd> Set gamma_g expression. </dd> <dt><samp class=”option”>gamma_b</samp></dt> <dd> Set gamma_b expression. </dd> <dt><samp class=”option”>gamma_weight</samp></dt> <dd> Set gamma_weight expression. The command accepts

the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>30.83 erosion# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>30.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.84 estdif# TOC</h3>

Deinterlace the input video ("estdif" stands for "Edge Slope Tracing Deinterlacing Filter"). Spatial only filter that uses edge slope tracing algorithm to interpolate missing lines. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp

class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> <dt><samp class=”option”>rslope</samp></dt> <dd> Specify the search radius for edge slope tracing. Default value is 1. Allowed range is from 1 to 15. </dd> <dt><samp class=”option”>redge</samp></dt> <dd> Specify the search radius for best edge matching. Default value is 2. Allowed range is from 0 to 15. </dd> <dt><samp class=”option”>ecost</samp></dt> <dd> Specify the edge cost for edge matching. Default value is 2. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>mcost</samp></dt> <dd> Specify the middle cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>dcost</samp></dt> <dd> Specify the distance cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the interpolation used. Default is 4-point interpolation. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>2p</samp></dt> <dd> Two-point interpolation. </dd> <dt><samp class=”option”>4p</samp></dt> <dd> Four-point interpolation. </dd> <dt><samp class=”option”>6p</samp></dt> <dd> Six-point interpolation. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>30.84.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>30.85 exposure# TOC</h3> Adjust exposure of the video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>exposure</samp></dt> <dd> Set the exposure correction in EV. Allowed range is from -3.0 to 3.0 EV Default value is 0 EV. </dd> <dt><samp class=”option”>black</samp></dt> <dd> Set the black level correction. Allowed range is from -1.0 to 1.0. Default value is 0. </dd>

</dl>

 <h4 class="subsection">30.85.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">30.86 extractplanes# TOC</h3>

Extract color channel components from input video stream into separate grayscale video streams. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set plane(s) to extract. Available values for planes are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

Choosing planes not available in the input will result in an error. That means you cannot select <code class="code">r</code>, <code class="code">g</code>, <code class="code">b</code> planes with <code class="code">y</code>, <code class="code">u</code>, <code class="code">v</code> planes at same time. </dd> </dl>

 <h4 class="subsection">30.86.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract luma, u and v color channel component from input video frame into 3 grayscale outputs: <pre class="example-preformatted">ffmpeg -i video.avi -filter_complex 'extractplanes=y+u+v[y][u][v]' -map '[y]' y.avi -map '[u]' u.avi -map '[v]' v.avi </pre>

 <h3 class="section">30.87 fade# TOC</h3>

Apply a fade-in/out effect to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">type, t</samp></dt> <dd> The effect type can be either "in" for a fade-in, or "out" for a fade-out effect. Default is <code class="code">in</code>. </dd> <dt><samp

class=”option”>start_frame, s</samp></dt> <dd> Specify the number of the frame to start applying the fade effect at. Default is 0. </dd> <dt><samp class=”option”>nb_frames, n</samp></dt> <dd> The number of frames that the fade effect lasts. At the end of the fade-in effect, the output video will have the same intensity as the input video. At the end of the fade-out transition, the output video will be filled with the selected <samp class=”option”>color</samp>. Default is 25. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> If set to 1, fade only alpha channel, if one exists on the input. Default value is 0. </dd> <dt><samp class=”option”>start_time, st</samp></dt> <dd> Specify the timestamp (in seconds) of the frame to start to apply the fade effect. If both start_frame and start_time are specified, the fade will start at whichever comes last. Default is 0. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> The number of seconds for which the fade effect has to last. At the end of the fade-in effect the output video will have the same intensity as the input video, at the end of the fade-out transition the output video will be filled with the selected <samp class=”option”>color</samp>. If both duration and nb_frames are specified, duration is used. Default is 0 (nb_frames is used by default). </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the fade. Default is "black". </dd> </dl>

 <h4 class=”subsection”>30.87.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Fade in the first 30 frames of video: <pre class=”example-preformatted”>fade=in:0:30 </pre>

The command above is equivalent to: <pre class=”example-preformatted”>fade=t=in:s=0:n=30 </pre>

Fade out the last 45 frames of a 200-frame video: <pre class=”example-preformatted”>fade=out:155:45 fade=type=out:start_frame=155:nb_frames=45 </pre>

Fade in the first 25 frames and fade out the last 25 frames of a 1000-frame video: <pre class=”example-preformatted”>fade=in:0:25, fade=out:975:25 </pre>

Make the first 5 frames yellow, then fade in from frame 5-24: <pre class=”example-preformatted”>fade=in:5:20:color=yellow </pre>

Fade in alpha over first 25 frames of video: <pre class=”example-preformatted”>fade=in:0:25:alpha=1 </pre>

Make the first 5.5 seconds black, then fade in for 0.5 seconds: <div

class=”example”> <pre class=”example-preformatted”>fade=t=in:st=5.5:d=0.5
</pre>

 <h3 class=”section”>30.88 feedback# TOC</h3> Apply feedback video filter. This filter pass cropped input frames to 2nd output. From there it can be filtered with other video filters. After filter receives frame from 2nd input, that frame is combined on top of original frame from 1st input and passed to 1st output. The typical usage is filter only part of frame. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the top left crop position. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the crop size. </dd> </dl>

 <h4 class=”subsection”>30.88.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Blur only top left rectangular part of video frame size 100x100 with gblur filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]gblur=8[bl
</pre>

Draw black box on top left part of video frame of size 100x100 with drawbox filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]drawbox=x=0:y=0:w=100:
</pre>

Pixelize rectangular part of video frame of size 100x100 with pixelize filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=320:y=240:w=100
</pre>

 <h3 class=”section”>30.89 fftdnoiz# TOC</h3> Denoise frames using 3D FFT (frequency domain filtering). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt> <dd> Set the noise sigma constant. This sets denoising strength. Default value is 1. Allowed range is from 0 to 30. Using very high sigma with low overlap may give blocking artifacts. </dd> <dt><samp class=”option”>amount</samp></dt>

<dd> Set amount of denoising. By default all detected noise is reduced. Default value is 1. Allowed range is from 0 to 1. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block in pixels, Default is 32, can be 8 to 256. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set block overlap. Default is 0.5. Allowed range is from 0.2 to 0.8. </dd> <dt><samp class="option">method</samp></dt> <dd> Set denoising method. Default is <code class="code">wiener</code>, can also be <code class="code">hard</code>. </dd> <dt><samp class="option">prev</samp></dt> <dd> Set number of previous frames to use for denoising. By default is set to 0. </dd> <dt><samp class="option">next</samp></dt> <dd> Set number of next frames to to use for denoising. By default is set to 0. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes which will be filtered, by default are all available filtered except alpha. </dd> </dl>

 <h3 class="section">30.90 fftfilt# TOC</h3> Apply arbitrary expressions to samples in frequency domain <dl class="table"> <dt><samp class="option">dc_Y</samp></dt> <dd> Adjust the dc value (gain) of the luma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_U</samp></dt> <dd> Adjust the dc value (gain) of the 1st chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_V</samp></dt> <dd> Adjust the dc value (gain) of the 2nd chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">weight_Y</samp></dt> <dd> Set the frequency domain weight expression for the luma plane. </dd> <dt><samp class="option">weight_U</samp></dt> <dd> Set the frequency domain weight expression for the 1st chroma plane. </dd> <dt><samp class="option">weight_V</samp></dt> <dd> Set the frequency domain weight expression for the 2nd chroma plane. </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.

 The filter accepts the following variables: </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class="option">N</samp></dt> <dd> The number of input frame, starting from 0. </dd> <dt><samp class="option">WS</samp></dt> <dt><samp class="option">HS</samp></dt> <dd> The size of FFT array for horizontal and vertical processing. </dd> </dl>

 <h4 class="subsection">30.90.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> High-pass: <pre class="example-preformatted">fftfilt=dc_Y=128:weight_Y='squish(1-(Y+X)/100)' </pre>

Low-pass: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y='squish((Y+X)/100-1)' </pre>

Sharpen: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y=
(Y+X)/100)' </pre>

Blur: <pre class="example-preformatted">fftfilt=dc_Y=0:weight_Y='exp
4 * ((Y+X)/(W+H)))' </pre>

 <h3 class="section">30.91 field# TOC</h3>

Extract a single field from an interlaced image using stride arithmetic to avoid wasting CPU time. The output frames are marked as non-interlaced. The filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Specify whether to extract the top (if the value is <code class="code">0</code> or <code class="code">top</code>) or the bottom field (if the value is <code class="code">1</code> or <code class="code">bottom</code>). </dd> </dl>

 <h3 class="section">30.92 fieldhint# TOC</h3>

Create new frames by copying the top and bottom fields from surrounding frames supplied as numbers by the hint file. <dl class="table">

<dt><samp class=”option”>hint</samp></dt> <dd> Set file containing hints: absolute/relative frame numbers. There must be one line for each frame in a clip. Each line must contain two numbers separated by the comma, optionally followed by <code class=”code”>-</code> or <code class=”code”>+</code>. Numbers supplied on each line of file can not be out of [N-1,N+1] where N is current frame number for <code class=”code”>absolute</code> mode or out of [-1, 1] range for <code class=”code”>relative</code> mode. First number tells from which frame to pick up top field and second number tells from which frame to pick up bottom field. If optionally followed by <code class=”code”>+</code> output frame will be marked as interlaced, else if followed by <code class=”code”>-</code> output frame will be marked as progressive, else it will be marked same as input frame. If optionally followed by <code class=”code”>t</code> output frame will use only top field, or in case of <code class=”code”>b</code> it will use only bottom field. If line starts with <code class=”code”>#</code> or <code class=”code”>;</code> that line is skipped. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Can be item <code class=”code”>absolute</code> or <code class=”code”>relative</code> or <code class=”code”>pattern</code>. Default is <code class=”code”>absolute</code>. The <code class=”code”>pattern</code> mode is same as <code class=”code”>relative</code> mode, except at last entry of file if there are more frames to process than <code class=”code”>hint</code> file is seek back to start. </dd> </dl>

Example of first several lines of <code class=”code”>hint</code> file for <code class=”code”>relative</code> mode: <pre class=”example-preformatted”>0,0 - # first frame 1,0 - # second frame, use third's frame top field and second's frame bottom field 1,0 - # third frame, use fourth's frame top field and third's frame bottom field 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - </pre>

 <h3 class=”section”>30.93 fieldmatch# TOC</h3>

Field matching filter for inverse telecine. It is meant to reconstruct the progressive frames from a telecined stream. The filter does not drop duplicated frames, so to achieve a complete inverse telecine <code class=”code”>fieldmatch</code> needs to be followed by a decimation filter such as decimate in the filtergraph. The separation of the field matching and the decimation is notably motivated by the possibility of inserting a de-interlacing filter fallback between the two. If the source has mixed telecined and real interlaced content, <code class=”code”>fieldmatch</code> will not be able to match fields for the interlaced parts. But these remaining combed frames will be marked as interlaced, and thus can be de-interlaced by a

later filter such as yadif before decimation. In addition to the various configuration options, <code class=”code”>fieldmatch</code> can take an optional second stream, activated through the <samp class=”option”>ppsrc</samp> option. If enabled, the frames reconstruction will be based on the fields and frames from this second stream. This allows the first input to be pre-processed in order to help the various algorithms of the filter, while keeping the output lossless (assuming the fields are matched properly). Typically, a field-aware denoiser, or brightness/contrast adjustments can help. Note that this filter uses the same algorithms as TIVTC/TFM (AviSynth project) and VIVTC/VFM (VapourSynth project). The later is a light clone of TFM from which <code class=”code”>fieldmatch</code> is based on. While the semantic and usage are very close, some behaviour and options names can differ. The decimate filter currently only works for constant frame rate input. If your input has mixed telecined (30fps) and progressive content with a lower framerate like 24fps use the following filterchain to produce the necessary cfr stream: <code class=”code”>dejudder,fps=30000/1001,fieldmatch,decimate</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Specify the assumed field order of the input stream. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Auto detect parity (use FFmpeg’s internal parity value). </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Assume bottom field first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Assume top field first. </dd> </dl>

Note that it is sometimes recommended not to trust the parity announced by the stream. Default value is <var class=”var”>auto</var>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the matching mode or strategy to use. <samp class=”option”>pc</samp> mode is the safest in the sense that it won’t risk creating jerkiness due to duplicate frames when possible, but if there are bad edits or blended fields it will end up outputting combed frames when a good match might actually exist. On the other hand, <samp class=”option”>pcn_ub</samp> mode is the most risky in terms of creating jerkiness, but will almost always find a good frame if there is one. The other values are all somewhere in between <samp class=”option”>pc</samp> and <samp class=”option”>pcn_ub</samp> in terms of risking jerkiness and creating duplicate frames versus finding good matches in sections with bad edits, orphaned fields, blended fields, etc. More details about p/c/n/u/b are available in p/c/n/u/b meaning section. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> 2-way matching (p/c) </dd> <dt>‘<samp class=”samp”>pc_n</samp>’</dt>

<dd> 2-way matching, and trying 3rd match if still combed (p/c + n) </dd> <dt>‘<samp class="samp">pc_u</samp>’</dt> <dd> 2-way matching, and trying 3rd match (same order) if still combed (p/c + u) </dd> <dt>‘<samp class="samp">pc_n_ub</samp>’</dt> <dd> 2-way matching, trying 3rd match if still combed, and trying 4th/5th matches if still combed (p/c + n + u/b) </dd> <dt>‘<samp class="samp">pcn</samp>’</dt> <dd> 3-way matching (p/c/n) </dd> <dt>‘<samp class="samp">pcn_ub</samp>’</dt> <dd> 3-way matching, and trying 4th/5th matches if all 3 of the original matches are detected as combed (p/c/n + u/b) </dd> </dl>

The parenthesis at the end indicate the matches that would be used for that mode assuming <samp class="option">order</samp>=<var class="var">tff</var> (and <samp class="option">field</samp> on <var class="var">auto</var> or <var class="var">top</var>). In terms of speed <samp class="option">pc</samp> mode is by far the fastest and <samp class="option">pcn_ub</samp> is the slowest. Default value is <var class="var">pc_n</var>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark the main input stream as a pre-processed input, and enable the secondary input stream as the clean source to pick the fields from. See the filter introduction for more details. It is similar to the <samp class="option">clip2</samp> feature from VFM/TFM. Default value is <code class="code">0</code> (disabled). </dd> <dt><samp class="option">field</samp></dt> <dd> Set the field to match from. It is recommended to set this to the same value as <samp class="option">order</samp> unless you experience matching failures with that setting. In certain circumstances changing the field that is used to match from can have a large impact on matching performance. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic (same value as <samp class="option">order</samp>). </dd> <dt>‘<samp class="samp">bottom</samp>’</dt> <dd> Match from the bottom field. </dd> <dt>‘<samp class="samp">top</samp>’</dt> <dd> Match from the top field. </dd> </dl>

Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mchroma</samp></dt> <dd> Set whether or not chroma is included during the match comparisons. In most cases it is recommended to leave this enabled. You should set this to <code class="code">0</code> only if your clip has bad chroma problems such as heavy rainbowing or other artifacts. Setting this to <code class="code">0</code> could also be used to speed things up at the cost of some accuracy. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">y0</samp></dt> <dt><samp class="option">y1</samp></dt> <dd> These define an exclusion band which excludes the lines between <samp class="option">y0</samp> and <samp class="option">y1</samp> from being included in the field

matching decision. An exclusion band can be used to ignore subtitles, a logo, or other things that may interfere with the matching. <samp class="option">y0</samp> sets the starting scan line and <samp class="option">y1</samp> sets the ending line; all lines in between <samp class="option">y0</samp> and <samp class="option">y1</samp> (including <samp class="option">y0</samp> and <samp class="option">y1</samp>) will be ignored. Setting <samp class="option">y0</samp> and <samp class="option">y1</samp> to the same value will disable the feature. <samp class="option">y0</samp> and <samp class="option">y1</samp> defaults to <code class="code">0</code>. </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change on the luma plane. Good values are in the <code class="code">[8.0, 14.0]</code> range. Scene change detection is only relevant in case <samp class="option">combmatch</samp>=<var class="var">sc</var>. The range for <samp class="option">scthresh</samp> is <code class="code">[0.0, 100.0]</code>. Default value is <code class="code">12.0</code>. </dd> <dt><samp class="option">combmatch</samp></dt> <dd> When <samp class="option">combatch</samp> is not <var class="var">none</var>, <code class="code">fieldmatch</code> will take into account the combed scores of matches when deciding what match to use as the final match. Available values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No final matching based on combed scores. </dd> <dt>‘<samp class="samp">sc</samp>’</dt> <dd> Combed scores are only used when a scene change is detected. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Use combed scores all the time. </dd> </dl>

Default is <var class="var">sc</var>. </dd> <dt><samp class="option">combdbg</samp></dt> <dd> Force <code class="code">fieldmatch</code> to calculate the combed metrics for certain matches and print them. This setting is known as <samp class="option">micout</samp> in TFM/VFM vocabulary. Available values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No forced calculation. </dd> <dt>‘<samp class="samp">pcn</samp>’</dt> <dd> Force p/c/n calculations. </dd> <dt>‘<samp class="samp">pcnub</samp>’</dt> <dd> Force p/c/n/u/b calculations. </dd> </dl>

Default value is <var class="var">none</var>. </dd> <dt><samp class="option">cthresh</samp></dt> <dd> This is the area combing threshold used for combed frame detection. This essentially controls how "strong" or "visible" combing must be to be detected. Larger values mean combing must be more visible and smaller values mean combing can be less visible or strong and still be detected. Valid settings are from <code class="code">-1</code> (every pixel will be detected as combed) to <code class="code">255</code> (no

pixel will be detected as combed). This is basically a pixel difference value. A good range is <code class=”code”>[8, 12]</code>. Default value is <code class=”code”>9</code>. </dd> <dt><samp class=”option”>chroma</samp></dt> <dd> Sets whether or not chroma is considered in the combed frame decision. Only disable this if your source has chroma problems (rainbowing, etc.) that are causing problems for the combed frame detection with chroma enabled. Actually, using <samp class=”option”>chroma</samp>=<var class=”var”>0</var> is usually more reliable, except for the case where there is chroma only combing in the source. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>blockx</samp></dt> <dt><samp class=”option”>blocky</samp></dt> <dd> Respectively set the x-axis and y-axis size of the window used during combed frame detection. This has to do with the size of the area in which <samp class=”option”>combpel</samp> pixels are required to be detected as combed for a frame to be declared combed. See the <samp class=”option”>combpel</samp> parameter description for more info. Possible values are any number that is a power of 2 starting at 4 and going up to 512. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>combpel</samp></dt> <dd> The number of combed pixels inside any of the <samp class=”option”>blocky</samp> by <samp class=”option”>blockx</samp> size blocks on the frame for the frame to be detected as combed. While <samp class=”option”>cthresh</samp> controls how "visible" the combing must be, this setting controls "how much" combing there must be in any localized area (a window defined by the <samp class=”option”>blockx</samp> and <samp class=”option”>blocky</samp> settings) on the frame. Minimum value is <code class=”code”>0</code> and maximum is <code class=”code”>blocky x blockx</code> (at which point no frames will ever be detected as combed). This setting is known as <samp class=”option”>MI</samp> in TFM/VFM vocabulary. Default value is <code class=”code”>80</code>. </dd> </dl>

 <h4 class=”subsection”>30.93.1 p/c/n/u/b meaning# TOC</h4>

 <h4 class=”subsubsection”>30.93.1.1 p/c/n# TOC</h4>

We assume the following telecined stream: <pre class=”example-preformatted”>Top fields: 1 2 2 3 4 Bottom fields: 1 2 3

4 4 </pre>

The numbers correspond to the progressive frame the fields relate to. Here, the first two frames are progressive, the 3rd and 4th are combed, and so on. When <code class=”code”>fieldmatch</code> is configured to run a matching from bottom (<samp class=”option”>field</samp>=<var class=”var”>bottom</var>) this is how this input stream get transformed: <pre class=”example-preformatted”>Input stream: T 1 2 2 3 4 B 1 2 3 4 4 <-- matching reference

Matches: c c n n c

Output stream: T 1 2 3 4 4 B 1 2 3 4 4 </pre>

As a result of the field matching, we can see that some frames get duplicated. To perform a complete inverse telecine, you need to rely on a decimation filter after this operation. See for instance the decimate filter. The same operation now matching from top fields (<samp class=”option”>field</samp>=<var class=”var”>top</var>) looks like this: <pre class=”example-preformatted”>Input stream: T 1 2 2 3 4 <-- matching reference B 1 2 3 4 4

Matches: c c p p c

Output stream: T 1 2 2 3 4 B 1 2 2 3 4 </pre>

In these examples, we can see what <var class=”var”>p</var>, <var class=”var”>c</var> and <var class=”var”>n</var> mean; basically, they refer to the frame and field of the opposite parity: <ul class=”itemize mark-bullet”> <var class=”var”>p</var> matches the field of the opposite parity in the previous frame <var class=”var”>c</var> matches the field of the opposite parity in the current frame <var class=”var”>n</var> matches the field of the opposite parity in the next frame

 <h4 class=”subsubsection”>30.93.1.2 u/b# TOC</h4>

The <var class=”var”>u</var> and <var class=”var”>b</var> matching are a bit special in the sense that they match from the opposite parity flag. In the following examples, we assume that we are currently matching the 2nd frame (Top:2, bottom:2). According to the match, a ’x’ is placed above and below each matched fields. With bottom matching (<samp class=”option”>field</samp>=<var class=”var”>bottom</var>): <pre class=”example-preformatted”>Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 1 2 2 2 2 2 2 1 3 </pre>

With top matching (<samp class="option">field</samp>=<var class="var">top</var>): <pre class="example-preformatted">Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 2 2 1 2 2 1 3 2 2 </pre>

 <h4 class="subsection">30.93.2 Examples# TOC</h4>

Simple IVTC of a top field first telecined stream: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=none, decimate </pre>

Advanced IVTC, with fallback on yadif for still combed frames: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=full, yadif=deint=interlaced, decimate </pre>

 <h3 class="section">30.94 fieldorder# TOC</h3>

Transform the field order of the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> The output field order. Valid values are <var class="var">tff</var> for top field first or <var class="var">bff</var> for bottom field first. </dd> </dl>

The default value is ‘<samp class="samp">tff</samp>’. The transformation is done by shifting the picture content up or down by one line, and filling the remaining line with appropriate picture content. This method is consistent with most broadcast field order converters. If the input video is not flagged as being interlaced, or it is already flagged as being of the required output field order, then this filter does not alter the incoming video. It is very useful when converting to or from PAL DV material, which is bottom field first. For example: <pre class="example-preformatted">ffmpeg -i in.vob -vf "fieldorder=bff" out.dv </pre>

 <h3 class="section">30.95 fillborders<span

class=”pull-right”># TOC</h3>

Fill borders of the input video, without changing video stream dimensions. Sometimes video can have garbage at the four edges and you may not want to crop video input to keep size multiple of some number. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>left</samp></dt> <dd> Number of pixels to fill from left border. </dd> <dt><samp class=”option”>right</samp></dt> <dd> Number of pixels to fill from right border. </dd> <dt><samp class=”option”>top</samp></dt> <dd> Number of pixels to fill from top border. </dd> <dt><samp class=”option”>bottom</samp></dt> <dd> Number of pixels to fill from bottom border. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set fill mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>smear</samp>’</dt> <dd> fill pixels using outermost pixels </dd> <dt>‘<samp class=”samp”>mirror</samp>’</dt> <dd> fill pixels using mirroring (half sample symmetric) </dd> <dt>‘<samp class=”samp”>fixed</samp>’</dt> <dd> fill pixels with constant value </dd> <dt>‘<samp class=”samp”>reflect</samp>’</dt> <dd> fill pixels using reflecting (whole sample symmetric) </dd> <dt>‘<samp class=”samp”>wrap</samp>’</dt> <dd> fill pixels using wrapping </dd> <dt>‘<samp class=”samp”>fade</samp>’</dt> <dd> fade pixels to constant value </dd> <dt>‘<samp class=”samp”>margins</samp>’</dt> <dd> fill pixels at top and bottom with weighted averages pixels near borders </dd> </dl>

Default is <var class=”var”>smear</var>. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Set color for pixels in fixed or fade mode. Default is <var class=”var”>black</var>. </dd> </dl>

 <h4 class=”subsection”>30.95.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>30.96 find_rect# TOC</h3>

Find a rectangular object in the input video. The object to search for must be specified as a gray8 image specified with

the <samp class=”option”>object</samp> option. For each possible match, a score is computed. If the score reaches the specified threshold, the object is considered found. If the input video contains multiple instances of the object, the filter will find only one of them. When an object is found, the following metadata entries are set in the matching frame: <dl class=”table”> <dt><samp class=”option”>lavfi.rect.w</samp></dt> <dd> width of object </dd> <dt><samp class=”option”>lavfi.rect.h</samp></dt> <dd> height of object </dd> <dt><samp class=”option”>lavfi.rect.x</samp></dt> <dd> x position of object </dd> <dt><samp class=”option”>lavfi.rect.y</samp></dt> <dd> y position of object </dd> <dt><samp class=”option”>lavfi.rect.score</samp></dt> <dd> match score of the found object </dd> </dl>

It accepts the following options: <dl class=”table”> <dt><samp class=”option”>object</samp></dt> <dd> Filepath of the object image, needs to be in gray8. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Detection threshold, expressed as a decimal number in the range 0-1. A threshold value of 0.01 means only exact matches, a threshold of 0.99 means almost everything matches. Default value is 0.5. </dd> <dt><samp class=”option”>mipmaps</samp></dt> <dd> Number of mipmaps, default is 3. </dd> <dt><samp class=”option”>xmin, ymin, xmax, ymax</samp></dt> <dd> Specifies the rectangle in which to search. </dd> <dt><samp class=”option”>discard</samp></dt> <dd> Discard frames where object is not detected. Default is disabled. </dd> </dl>

 <h4 class=”subsection”>30.96.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Cover a rectangular object by the supplied image of a given video using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

Find the position of an object in each frame using <code class=”command”>ffprobe</code> and write it to a log file: <pre class=”example-preformatted”>ffprobe -f lavfi movie=test.mp4,find_rect=object=object.pgm:threshold=0.3 \ -show_entries frame=pkt_pts_time:frame_tags=lavfi.rect.x,lavfi.rect.y \ -of csv -o find_rect.csv </pre>

 <h3 class=”section”>30.97 floodfill# <a class=”anchor hidden-xs”href=”#toc-floodfill”

aria-hidden=”true”>TOC</h3>

Flood area with values of same pixel components with another values. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set pixel x coordinate. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set pixel y coordinate. </dd> <dt><samp class=”option”>s0</samp></dt> <dd> Set source #0 component value. </dd> <dt><samp class=”option”>s1</samp></dt> <dd> Set source #1 component value. </dd> <dt><samp class=”option”>s2</samp></dt> <dd> Set source #2 component value. </dd> <dt><samp class=”option”>s3</samp></dt> <dd> Set source #3 component value. </dd> <dt><samp class=”option”>d0</samp></dt> <dd> Set destination #0 component value. </dd> <dt><samp class=”option”>d1</samp></dt> <dd> Set destination #1 component value. </dd> <dt><samp class=”option”>d2</samp></dt> <dd> Set destination #2 component value. </dd> <dt><samp class=”option”>d3</samp></dt> <dd> Set destination #3 component value. </dd> </dl>

 <h3 class=”section”>30.98 format# TOC</h3>

Convert the input video to one of the specified pixel formats. Libavfilter will try to pick one that is suitable as input to the next filter. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as "pix_fmts=yuv420p|monow|rgb24". </dd> <dt><samp class=”option”>color_spaces</samp></dt> <dd> A ’|’-separated list of color space names, such as "color_spaces=bt709|bt470bg|bt2020nc". </dd> <dt><samp class=”option”>color_ranges</samp></dt> <dd> A ’|’-separated list of color range names, such as "color_ranges=tv|pc". </dd> <dt><samp class=”option”>alpha_modes</samp></dt> <dd> A ’|’-separated list of color range names, such as "alpha_modes=straight|premultiplied". </dd> <dt><samp class=”option”>chroma_locations</samp></dt> <dd> A ’|’-separated list of chroma sample location names, such as "chroma_locations=left|topleft". </dd> </dl>

 <h4 class=”subsection”>30.98.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert the input video to the

<var class="var">yuv420p</var> format <pre class="example-preformatted">format=pix_fmts=yuv420p </pre>

Convert the input video to any of the formats in the list <pre class="example-preformatted">format=pix_fmts=yuv420p|yuv444p|yuv410p </pre>

 <h3 class="section">30.99 fps# TOC</h3>

Convert the video to specified constant frame rate by duplicating or dropping frames as necessary. It accepts the following parameters: <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> The desired output frame rate. It accepts expressions containing the following constants: <dl class="table"> <dt>‘<samp class="samp">source_fps</samp>’</dt> <dd> The input’s frame rate </dd> <dt>‘<samp class="samp">ntsc</samp>’</dt> <dd> NTSC frame rate of <code class="code">30000/1001</code> </dd> <dt>‘<samp class="samp">pal</samp>’</dt> <dd> PAL frame rate of <code class="code">25.0</code> </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> Film frame rate of <code class="code">24.0</code> </dd> <dt>‘<samp class="samp">ntsc_film</samp>’</dt> <dd> NTSC-film frame rate of <code class="code">24000/1001</code> </dd> </dl>

The default is <code class="code">25</code>. </dd> <dt><samp class="option">start_time</samp></dt> <dd> Assume the first PTS should be the given value, in seconds. This allows for padding/trimming at the start of stream. By default, no assumption is made about the first frame’s expected PTS, so no padding or trimming is done. For example, this could be set to 0 to pad the beginning with duplicates of the first frame if a video stream starts after the audio stream or to trim any frames with a negative PTS. </dd> <dt><samp class="option">round</samp></dt> <dd> Timestamp (PTS) rounding method. Possible values are: <dl class="table"> <dt><samp class="option">zero</samp></dt> <dd> round towards 0 </dd> <dt><samp class="option">inf</samp></dt> <dd> round away from 0 </dd> <dt><samp class="option">down</samp></dt> <dd> round towards -infinity </dd> <dt><samp class="option">up</samp></dt> <dd> round towards +infinity </dd> <dt><samp class="option">near</samp></dt> <dd> round to nearest </dd> </dl> The default is <code class="code">near</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> Action performed when reading the last frame. Possible values are: <dl class="table"> <dt><samp class="option">round</samp></dt> <dd> Use same timestamp rounding method as used for other

frames. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass through last frame if input duration has not been reached yet. </dd> </dl> The default is <code class="code">round</code>. If timestamp rounding would otherwise discard every frame, one frame is retained when its rounded timestamp equals both the first output timestamp and the rounded end of input. This exception extends the output beyond the rounded end of input by one output frame period. Frames whose entire interval is trimmed by <samp class="option">start_time</samp> remain discarded. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class="var">fps</var>[:<var class="var">start_time</var>[:<var class="var">round</var>]]. See also the setpts filter. <h4 class="subsection">30.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A typical usage in order to set the fps to 25: <pre class="example-preformatted">fps=fps=25 </pre>

Sets the fps to 24, using abbreviation and rounding method to round to nearest: <pre class="example-preformatted">fps=fps=film:round=near </pre>

 <h3 class="section">30.100 framepack# TOC</h3>

Pack two different video streams into a stereoscopic video, setting proper metadata on supported codecs. The two views should have the same size and framerate and processing will stop when the shorter video ends. Please note that you may conveniently adjust view properties with the scale and fps filters. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> The desired packing format. Supported values are: <dl class="table"> <dt><samp class="option">sbs</samp></dt> <dd> The views are next to each other (default). </dd> <dt><samp class="option">tab</samp></dt> <dd> The views are on top of each other. </dd> <dt><samp class="option">lines</samp></dt> <dd> The views are packed by line. </dd> <dt><samp class="option">columns</samp></dt> <dd> The views are packed by column. </dd> <dt><samp class="option">frameseq</samp></dt> <dd> The views are temporally interleaved. </dd> </dl>

</dd> </dl>

Some examples: <pre class=”example-preformatted”># Convert left and right views into a frame-sequential video ffmpeg -i LEFT -i RIGHT -filter_complex framepack=frameseq OUTPUT

Convert views into a side-by-side video with the same output resolution as the input ffmpeg -i LEFT -i RIGHT -filter_complex [0:v]scale=w=iw/2[left],[1:v]scale=w=iw/2[right],[left][right]fra
OUTPUT </pre>

 <h3 class=”section”>30.101 framerate# TOC</h3>

Change the frame rate by interpolating new video output frames from the source frames. This filter is not designed to function correctly with interlaced media. If you wish to change the frame rate of interlaced media then you are required to deinterlace before this filter and re-interlace after this filter. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frames per second. This option can also be specified as a value alone. The default is <code class=”code”>50</code>. </dd> <dt><samp class=”option”>interp_start</samp></dt> <dd> Specify the start of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>15</code>. </dd> <dt><samp class=”option”>interp_end</samp></dt> <dd> Specify the end of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>240</code>. </dd> <dt><samp class=”option”>scene</samp></dt> <dd> Specify the level at which a scene change is detected as a value between 0 and 100 to indicate a new scene; a low value reflects a low probability for the current frame to introduce a new scene, while a higher value means the current frame is more likely to be one. The default is <code class=”code”>8.2</code>. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class=”var”>flags</var> is: <dl class=”table”> <dt><samp class=”option”>scene_change_detect, scd</samp></dt> <dd> Enable scene change detection using the value of the option <var class=”var”>scene</var>. This flag is enabled by default. </dd> </dl> </dd> </dl>

 <h3 class=”section”>30.102 frc_amf# TOC</h3>

Double the frame rate, using Frame Rate Converter (FRC) provided by AMD Advanced Media Framework library for hardware acceleration. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>engine_type</samp></dt> <dd> Specify the engine used to run shaders. <dl class=”table”> <dt>‘<samp class=”samp”>dx11</samp>’</dt> <dd> DirectX 11. </dd> <dt>‘<samp class=”samp”>dx12</samp>’</dt> <dd> DirectX 12 (Default value). </dd> </dl>

</dd> <dt><samp class=”option”>enable</samp></dt> <dd> Boolean value: enable/disable FRC. Dynamic value, can be altered at a runtime without re-initializing the filter. (Default value: enabled). </dd> <dt><samp class=”option”>fallback_mode</samp></dt> <dd> Fallback behavior in case of low interpolation confidence. <dl class=”table”> <dt>‘<samp class=”samp”>duplicate</samp>’</dt> <dd> Duplicate frame. </dd> <dt>‘<samp class=”samp”>blend</samp>’</dt> <dd> Blend two frames together (Default value). </dd> </dl>

</dd> <dt><samp class=”option”>indicator</samp></dt> <dd> Boolean value: show FRC indicator square in the top left corner of the video (Default value: disabled). </dd> <dt><samp class=”option”>profile</samp></dt> <dd> Level of hierarchical motion search. <dl class=”table”> <dt>‘<samp class=”samp”>low</samp>’</dt> <dd> Less levels of hierarchical motion search. Only recommended for extremely low resolutions. </dd> <dt>‘<samp class=”samp”>high</samp>’</dt> <dd> Recommended for any resolution up to 1440p. (Default value) </dd> <dt>‘<samp class=”samp”>super</samp>’</dt> <dd> More levels of hierarchical motion search. Recommended for resolutions 1440p or higher. </dd> </dl>

</dd> <dt><samp class=”option”>mv_search_mode</samp></dt> <dd> Performance mode of the motion search. <dl class=”table”> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Conduct motion search on the full resolution of source images. </dd> <dt>‘<samp class=”samp”>performance</samp>’</dt> <dd> Conduct motion search on the down scaled source images. Recommended for APU or low end GPU for better performance. </dd> </dl>

</dd> <dt><samp class=”option”>use_future_frame</samp></dt> <dd> Boolean value: enable dependency on future frame, improves quality for the cost of latency (Default value: enabled). </dd> </dl>

 <h3 class=”section”>30.103 vqe_amf# TOC</h3>

AMD AMF VQ Enhancer filter. This filter applies AMD AMF VQ Enhance-

ment to the input video. This filter accepts the following option: <dl class="table"> <dt><samp class="option">attenuation</samp></dt> <dd> Set VQ Enhancer strength. Allowed range is from <code class="code">0.02</code> to <code class="code">0.4</code>. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">engine_type</samp></dt> <dd> Set AMF memory type used by the filter. Possible values: <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11 </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 </dd> <dt>‘<samp class="samp">vulkan</samp>’</dt> <dd> Vulkan </dd> <dt>‘<samp class="samp">opencl</samp>’</dt> <dd> OpenCL </dd> </dl>

</dd> </dl>

Example: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf vqe_amf=attenuation=0.4 -c:v hevc_amf output.mp4 </pre>

 <h3 class="section">30.104 framestep# TOC</h3>

Select one frame every N-th frame. This filter accepts the following option: <dl class="table"> <dt><samp class="option">step</samp></dt> <dd> Select frame after every <code class="code">step</code> frames. Allowed values are positive integers higher than 0. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">30.105 freezedetect# TOC</h3>

Detect frozen video. This filter logs a message and sets frame metadata when it detects that the input video has no significant change in content during a specified duration. Video freeze detection calculates the mean average absolute difference of all the components of video frames and compares it to a noise floor. The printed times and duration are expressed in seconds. The <code class="code">lavfi.freezedetect.freeze_start</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the freeze. The <code class="code">lavfi.freezedetect.freeze_duration</code> and <code class="code">lavfi.freezedetect.freeze_end</code> metadata keys are set on the first frame after the freeze. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or as a difference

ratio between 0 and 1. Default is -60dB, or 0.001. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set freeze duration until notification (default is 2 seconds). </dd> </dl>

 <h3 class=”section”>30.106 freezeframes# TOC</h3>

Freeze video frames. This filter freezes video frames using frame from 2nd input. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first</samp></dt> <dd> Set number of first frame from which to start freeze. </dd> <dt><samp class=”option”>last</samp></dt> <dd> Set number of last frame from which to end freeze. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Set number of frame from 2nd input which will be used instead of replaced frames. </dd> </dl>

 <h3 class=”section”>30.107 frei0r# TOC</h3>

Apply a frei0r effect to the input video. To enable the compilation of this filter, you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the frei0r effect to load. If the environment variable <code class=”env”>FREI0R_PATH</code> is defined, the frei0r effect is searched for in each of the directories specified by the colon-separated list in <code class=”env”>FREI0R_PATH</code>. Otherwise, the standard frei0r paths are searched, in this order: <samp class=”file”>HOME/.frei0r-1/lib/</samp>, <samp class=”file”>/usr/local/lib/frei0r-1/</samp>, <samp class=”file”>/usr/lib/frei0r-1/</samp>. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r effect. </dd> </dl>

A frei0r effect parameter can be a boolean (its value is either "y" or "n"), a double, a color (specified as <var class=”var”>R</var>/<var class=”var”>G</var>/<var class=”var”>B</var>, where <var class=”var”>R</var>, <var class=”var”>G</var>, and <var class=”var”>B</var> are floating point numbers between 0.0 and 1.0, inclusive) or a color description as specified in the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual, a position (specified as <var class=”var”>X</var>/<var class=”var”>Y</var>, where <var class=”var”>X</var>

and <var class=”var”>Y</var> are floating point numbers) and/or a string. The number and types of parameters depend on the loaded effect. If an effect parameter is not specified, the default value is set. <h4 class=”subsection”>30.107.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the distort0r effect, setting the first two double parameters: <pre class=”example-preformatted”>frei0r=filter_name=distort0r:filter_params=0.5|0.01 </pre>

Apply the colordistance effect, taking a color as the first parameter: <pre class=”example-preformatted”>frei0r=colordistance:0.2/0.3/0.4 frei0r=colordistance:violet frei0r=colordistance:0x112233 </pre>

Apply the perspective effect, specifying the top left and top right image positions: <pre class=”example-preformatted”>frei0r=perspective:0.2/0.2|0.8/0.2 </pre>

For more information, see http://frei0r.dyne.org <h4 class=”subsection”>30.107.2 Commands# TOC</h4>

This filter supports the <samp class=”option”>filter_params</samp> option as commands. <h3 class=”section”>30.108 fspp# TOC</h3>

Apply fast and simple postprocessing. It is a faster version of spp. It splits (I)DCT into horizontal/vertical passes. Unlike the simple post- processing filter, one of them is performed once per block, not per pixel. This allows for much higher speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 4-5. Default value is <code class=”code”>4</code>. </dd> <dt><samp class=”option”>qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0-63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class=”option”>strength</samp></dt> <dd> Set filter strength. It accepts an integer in range -15 to 32. Lower values mean more details but also more artifacts, while higher values make the image smoother but also blurrier. Default value is <code class=”code”>0</code> − PSNR optimal. </dd> <dt><samp class=”option”>use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code

class=”code”>1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class=”code”>0</code> (not enabled). </dd> </dl>

 <h3 class=”section”>30.109 fsync# TOC</h3>

Synchronize video frames with an external mapping from a file. For each input PTS given in the map file it either drops or creates as many frames as necessary to recreate the sequence of output frames given in the map file. This filter is useful to recreate the output frames of a framerate conversion by the fps filter, recorded into a map file using the ffmpeg option <code class=”code”>-stats_mux_pre</code>, and do further processing to the corresponding frames e.g. quality comparison. Each line of the map file must contain three items per input frame, the input PTS (decimal), the output PTS (decimal) and the output TIMEBASE (decimal/decimal), separated by a space. This file format corresponds to the output of <code class=”code”>-stats_mux_pre_fmt="{ptsi} {pts} {tb}"</code>. The filter assumes the map file is sorted by increasing input PTS. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> The filename of the map file to be used. </dd> </dl>

Example: <pre class=”example-preformatted”># Convert a video to 25 fps and record a MAP_FILE file with the default format of this filter ffmpeg -i INPUT -vf fps=fps=25 -stats_mux_pre MAP_FILE -stats_mux_pre_fmt "{ptsi} {pts} {tb}" OUTPUT

Sort MAP_FILE by increasing input PTS sort -n MAP_FILE

Use INPUT, OUTPUT and the MAP_FILE from above to compare the corresponding frames in INPUT and OUTPUT via SSIM ffmpeg -i INPUT -i OUTPUT -filter_complex '[0:v]fsync=file=MAP_FILE[ref];[1:v][ref]ssim' -f null - </pre>

 <h3 class=”section”>30.110 gainmap# TOC</h3>

Compute an HDR gain map from two renditions of the same image, as defined by ISO 21496-1. The first input is the base rendition (typically SDR), and the second input is the alternate rendition (typically HDR). Both must have the same dimensions. Outputs a gain map, as either ‘<samp class=”samp”>gbrpf32</samp>’ or ‘<samp class=”samp”>grayf32</samp>’ depending on the

chosen <samp class=”option”>mode</samp>. <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Controls what quantity the gain map is computed against. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>rgb</samp>’</dt> <dd> One gain map channel per component. </dd> <dt>‘<samp class=”samp”>luma</samp>’</dt> <dd> A single gain map channel, computed from the luma. </dd> <dt>‘<samp class=”samp”>maxrgb</samp>’</dt> <dd> A single gain map channel, computed from ‘<samp class=”samp”>max(R, G, B)</samp>’. </dd> </dl>

Default is ‘<samp class=”samp”>rgb</samp>’. </dd> <dt><samp class=”option”>colorspace</samp></dt> <dd> Chooses which colorspace to compute the gain map in. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>base</samp>’</dt> <dd> The base rendition’s colorspace. </dd> <dt>‘<samp class=”samp”>alternate</samp>’</dt> <dd> The alternate rendition’s colorspace. </dd> </dl>

Default is ‘<samp class=”samp”>base</samp>’. </dd> <dt><samp class=”option”>min</samp></dt> <dt><samp class=”option”>max</samp></dt> <dd> Bounds on the encoded gain (log2). If unspecified, these will be measured from the frame. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Encoding gamma of the resulting gain map. If unspecified, this will be optimized for the frame in question. Setting this to <code class=”code”>1.0</code> will produce a linear gain map, which may be more efficient in some applications. </dd> <dt><samp class=”option”>base_offset</samp></dt> <dt><samp class=”option”>alt_offset</samp></dt> <dd> Offset to prevent numerical instability near zero. Both default to <code class=”code”>1/64</code>. </dd> <dt><samp class=”option”>base_nits</samp></dt> <dt><samp class=”option”>alt_nits</samp></dt> <dd> Override the reference luminance of the input, in nits. If unspecified, uses the values from the input metadata. </dd> </dl>

The <code class=”code”>gainmap</code> filter also supports the framesync options. <h4 class=”subsection”>30.110.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Compute a gain map between an SDR and an HDR version of the same image: <pre class=”example-preformatted”>ffmpeg -i $sdr -i $hdr -filter_complex "[0:v][1:v]gainmap" $out </pre>

 <h3 class=”section”>30.111 gblur<span class=”pull-

right”>#
TOC</h3>

Apply Gaussian blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt>
<dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>steps</samp></dt> <dd> Set number of steps for Gaussian approximation. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class=”code”>sigma</code>. Default is <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>30.111.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>30.112 geq# TOC</h3>

Apply generic equation to each pixel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>lum_expr, lum</samp></dt> <dd> Set the luma expression. </dd> <dt><samp class=”option”>cb_expr, cb</samp></dt>
<dd> Set the chrominance blue expression. </dd> <dt><samp class=”option”>cr_expr, cr</samp></dt> <dd> Set the chrominance red expression. </dd> <dt><samp class=”option”>alpha_expr, a</samp></dt> <dd> Set the alpha expression. </dd>
<dt><samp class=”option”>red_expr, r</samp></dt> <dd> Set the red expression. </dd> <dt><samp class=”option”>green_expr, g</samp></dt> <dd> Set the green expression. </dd>
<dt><samp class=”option”>blue_expr, b</samp></dt> <dd> Set the blue expression. </dd> </dl>

The colorspace is selected according to the specified options. If one of the <samp class=”option”>lum_expr</samp>, <samp class=”option”>cb_expr</samp>, or <samp class=”option”>cr_expr</samp> options is specified, the filter will automatically select a YCbCr colorspace. If one of the <samp class=”option”>red_expr</samp>, <samp class=”option”>green_expr</samp>, or <samp class=”option”>blue_expr</samp> options is specified, it will select an RGB colorspace. If one of the chrominance expression is not defined, it falls back on the other one. If no alpha expression is specified

it will evaluate to opaque value. If none of chrominance expressions are specified, they will evaluate to the luma expression. The expressions can use the following variables and functions: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class="code">0</code>. </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class="option">SW</samp></dt> <dt><samp class="option">SH</samp></dt> <dd> Width and height scale depending on the currently filtered plane. It is the ratio between the corresponding luma plane number of pixels and the current plane ones. E.g. for YUV4:2:0 the values are <code class="code">1,1</code> for the luma plane, and <code class="code">0.5,0.5</code> for chroma planes. </dd> <dt><samp class="option">T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class="option">p(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the current plane. </dd> <dt><samp class="option">lum(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the luma plane. </dd> <dt><samp class="option">cb(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the blue-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">cr(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">r(x, y)</samp></dt> <dt><samp class="option">g(x, y)</samp></dt> <dt><samp class="option">b(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red/green/blue component. Return 0 if there is no such component. </dd> <dt><samp class="option">alpha(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the alpha plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">psum(x,y), lumsum(x, y), cbsum(x,y), crsum(x,y), rsum(x,y), gsum(x,y), bsum(x,y), alphasum(x,y)</samp></dt> <dd> Sum of sample values in the rectangle from (0,0) to (x,y), this allows obtaining sums of samples within a rectangle. See the functions without the sum postfix. </dd> <dt><samp class="option">interpolation</samp></dt> <dd> Set one of interpolation methods: <dl class="table"> <dt><samp class="option">nearest, n</samp></dt> <dt><samp class="option">bilinear, b</samp></dt> </dl> Default is bilinear. </dd> </dl>

For functions, if <var class="var">x</var> and <var class="var">y</var> are outside the area, the value will be automatically clipped to the closer edge. Please note that this filter can use multiple threads in which case each slice will have its own expression state. If you want to use only a single expression state because your expressions depend on previous state then you should limit the number of filter threads to 1. <h4 class="subsection">30.112.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Flip the image horizontally: <pre class="example-preformatted">geq=p(W-X\,Y) </pre>

Generate a bidimensional sine wave, with angle <code class="code">PI/3</code> and a wavelength of 100 pixels: <pre class="example-preformatted">geq=128 + 100*sin(2*(PI/100)*(cos(PI/3)*(X-50*T) + sin(PI/3)*Y)):128:128 </pre>

Generate a fancy enigmatic moving light: <pre class="example-preformatted">nullsrc=s=256x256,geq=random(1)/hypot(X-cos(N*0.07)*W/2-W/2\,Y-sin(N*0.09)*H/2-H/2)^2*1000000*sin(N*0.02):128:128 </pre>

Generate a quick emboss effect: <pre class="example-preformatted">format=gray,geq=lum_expr='(p(X,Y)+(256-p(X-4,Y-4)))/2' </pre>

Modify RGB components depending on pixel position: <pre class="example-preformatted">geq=r='X/W*r(X,Y)':g='(1-X/W)*g(X,Y)':b='(H-Y)/H*b(X,Y)' </pre>

Create a radial gradient that is the same size as the input (also see the vignette filter): <pre class="example-preformatted">geq=lum=255*gauss((X/W-0.5)*3)*gauss((Y/H-0.5)*3)/gauss(0)/gauss(0),format=gray </pre>

 <h3 class="section">30.113 gradfun# TOC</h3>

Fix the banding artifacts that are sometimes introduced into nearly flat regions by truncation to 8-bit color depth. Interpolate the gradients that should go where the bands are, and dither them. It is designed for playback only. Do not use it prior to lossy compression, because compression tends to lose the dither and bring back the bands. It accepts the following parameters: <dl class="table"> <dt><samp

class=”option”>strength</samp></dt> <dd> The maximum amount by which the filter will change any one pixel. This is also the threshold for detecting nearly flat regions. Acceptable values range from .51 to 64; the default value is 1.2. Out-of-range values will be clipped to the valid range. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> The neighborhood to fit the gradient to. A larger radius makes for smoother gradients, but also prevents the filter from modifying the pixels near detailed regions. Acceptable values are 8-32; the default value is 16. Out-of-range values will be clipped to the valid range. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class=”var”>strength</var>[:<var class=”var”>radius</var>] <h4 class=”subsection”>30.113.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the filter with a <code class=”code”>3.5</code> strength and radius of <code class=”code”>8</code>: <pre class=”example-preformatted”>gradfun=3.5:8 </pre>

Specify radius, omitting the strength (which will fall-back to the default value): <pre class=”example-preformatted”>gradfun=radius=8 </pre>

 <h3 class=”section”>30.114 graphmonitor# TOC</h3> Show various filtergraph stats. With this filter one can debug complete filtergraph. Especially issues with links filling with queued frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set video output size. Default is <var class=”var”>hd720</var>. </dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set video opacity. Default is <var class=”var”>0.9</var>. Allowed range is from <var class=”var”>0</var> to <var class=”var”>1</var>. </dd> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set output mode flags. Available values for flags are: <dl class=”table”> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> No any filtering. Default. </dd> <dt>‘<samp class=”samp”>compact</samp>’</dt> <dd> Show only filters with queued frames. </dd> <dt>‘<samp class=”samp”>nozero</samp>’</dt> <dd> Show only filters with non-zero stats. </dd> <dt>‘<samp class=”samp”>noeof</samp>’</dt> <dd> Show only filters with non-eof stat. </dd> <dt>‘<samp

class=”samp”>nodisabled</samp>’</dt> <dd> Show only filters that are enabled in timeline. </dd> </dl>

</dd> <dt><samp class=”option”>flags, f</samp></dt> <dd> Set flags which enable which stats are shown in video. Available values for flags are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> All flags turned off. </dd> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> All flags turned on. </dd> <dt>‘<samp class=”samp”>queue</samp>’</dt> <dd> Display number of queued frames in each link. </dd> <dt>‘<samp class=”samp”>frame_count_in</samp>’</dt> <dd> Display number of frames taken from filter. </dd> <dt>‘<samp class=”samp”>frame_count_out</samp>’</dt> <dd> Display number of frames given out from filter. </dd> <dt>‘<samp class=”samp”>frame_count_delta</samp>’</dt> <dd> Display delta number of frames between above two values. </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Display current filtered frame pts. </dd> <dt>‘<samp class=”samp”>pts_delta</samp>’</dt> <dd> Display pts delta between current and previous frame. </dd> <dt>‘<samp class=”samp”>time</samp>’</dt> <dd> Display current filtered frame time. </dd> <dt>‘<samp class=”samp”>time_delta</samp>’</dt> <dd> Display time delta between current and previous frame. </dd> <dt>‘<samp class=”samp”>timebase</samp>’</dt> <dd> Display time base for filter link. </dd> <dt>‘<samp class=”samp”>format</samp>’</dt> <dd> Display used format for filter link. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Display video size or number of audio channels in case of audio used by filter link. </dd> <dt>‘<samp class=”samp”>rate</samp>’</dt> <dd> Display video frame rate or sample rate in case of audio used by filter link. </dd> <dt>‘<samp class=”samp”>eof</samp>’</dt> <dd> Display link output status. </dd> <dt>‘<samp class=”samp”>sample_count_in</samp>’</dt> <dd> Display number of samples taken from filter. </dd> <dt>‘<samp class=”samp”>sample_count_out</samp>’</dt> <dd> Display number of samples given out from filter. </dd> <dt>‘<samp class=”samp”>sample_count_delta</samp>’</dt> <dd> Display delta number of samples between above two values. </dd> <dt>‘<samp class=”samp”>disabled</samp>’</dt> <dd> Show the timeline filter status. </dd> </dl>

</dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set upper limit for video rate of output stream, Default value is <var class=”var”>25</var>. This guarantee that output video frame rate will not be higher than this value. </dd> </dl>

 <h3 class=”section”>30.115 grayworld<span

class=”pull-right”># TOC</h3> A color constancy filter that applies color correction based on the grayworld assumption See: <a class=”url” href=”https://www.researchgate.net/publication/275213614_A_New_Color_Correction_Met
 The algorithm uses linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class=”example-preformatted”>ffmpeg -i INPUT -vf zscale=transfer=linear,grayworld,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h3 class=”section”>30.116 greyedge# TOC</h3> A color constancy variation filter which estimates scene illumination via grey edge algorithm and corrects the scene colors accordingly. See: <a class=”url” href=”https://staff.science.uva.nl/th.gevers/pub/GeversTIP07.pdf”>https://staff.science.uva.nl/th.gevers/pu
 The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>difford</samp></dt> <dd> The order of differentiation to be applied on the scene. Must be chosen in the range [0,2] and default value is 1. </dd> <dt><samp class=”option”>minknorm</samp></dt> <dd> The Minkowski parameter to be used for calculating the Minkowski distance. Must be chosen in the range [0,20] and default value is 1. Set to 0 for getting max value instead of calculating Minkowski distance. </dd> <dt><samp class=”option”>sigma</samp></dt> <dd> The standard deviation of Gaussian blur to be applied on the scene. Must be chosen in the range [0,1024.0] and default value = 1. floor(<var class=”var”>sigma</var> * break_off_sigma(3)) can’t be equal to 0 if <var class=”var”>difford</var> is greater than 0. </dd> </dl>

 <h4 class=”subsection”>30.116.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Grey Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=5:sigma=2 </pre>

Max Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=0:sigma=2 </pre>

 <h3 class=”section”>30.117 guided# TOC</h3> Apply guided filter

for edge-preserving smoothing, dehazing and so on. The filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set the box radius in pixels. Allowed range is 1 to 20. Default is 3. </dd> <dt><samp class="option">eps</samp></dt> <dd> Set regularization parameter (with square). Allowed range is 0 to 1. Default is 0.01. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set filter mode. Can be <code class="code">basic</code> or <code class="code">fast</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">sub</samp></dt> <dd> Set subsampling ratio for <code class="code">fast</code> mode. Range is 2 to 64. Default is 4. No subsampling occurs in <code class="code">basic</code> mode. </dd> <dt><samp class="option">guidance</samp></dt> <dd> Set guidance mode. Can be <code class="code">off</code> or <code class="code">on</code>. Default is <code class="code">off</code>. If <code class="code">off</code>, single input is required. If <code class="code">on</code>, two inputs of the same resolution and pixel format are required. The second input serves as the guidance. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">30.117.1 Commands# TOC</h4> This filter supports the all above options as commands. <h4 class="subsection">30.117.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Edge-preserving smoothing with guided filter: <pre class="example-preformatted">ffmpeg -i in.png -vf guided out.png </pre>

Dehazing, structure-transferring filtering, detail enhancement with guided filter. For the generation of guidance image, refer to paper "Guided Image Filtering". See: <a class="url" href="http://kaiminghe.com/publications/pami12guidedfilter.pdf
 <pre class="example-preformatted">ffmpeg -i in.png -i guidance.png -filter_complex guided=guidance=on out.png </pre>

 <h3 class="section">30.118 haldclut# TOC</h3>

Apply a Hald CLUT to a video stream. First input

is the video stream to process, and second one is the Hald CLUT. The Hald CLUT input can be a simple picture or a complete video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">clut</samp></dt> <dd> Set which CLUT video frames will be processed from second input stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> <dt><samp class="option">shortest</samp></dt> <dd> Force termination when the shortest input terminates. Default is <code class="code">0</code>. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> Continue applying the last CLUT after the end of the stream. A value of <code class="code">0</code> disable the filter after the last frame of the CLUT is reached. Default is <code class="code">1</code>. </dd> </dl>

<code class="code">haldclut</code> also has the same interpolation options as lut3d (both filters share the same internals). This filter also supports the framesync options. More information about the Hald CLUT can be found on Eskil Steenberg’s website (Hald CLUT author) at <a class="url" href="http://www.quelsolaar.com/technology/clut.html">http://www.quelsolaar.com/technology/clut.html<
  <h4 class="subsection">30.118.1 Commands# TOC</h4>

This filter supports the <code class="code">interp</code> option as commands. <h4 class="subsection">30.118.2 Workflow examples# TOC</h4>

 <h4 class="subsubsection">30.118.2.1 Hald CLUT video stream# TOC</h4>

Generate an identity Hald CLUT stream altered with various effects: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf "hue=H=2*PI*t:s=sin(2*PI*t)+1, curves=cross_process" -t 10 -c:v ffv1 clut.nut </pre>

Note: make sure you use a lossless codec. Then use it with <code class="code">haldclut</code> to apply it on some random stream:

 <pre class="example-preformatted">ffmpeg -f lavfi -i mandelbrot -i clut.nut -filter_complex '[0][1] haldclut' -t 20 mandelclut.mkv </pre>

The Hald CLUT will be applied to the 10 first seconds (duration of <samp class="file">clut.nut</samp>), then the latest picture of that CLUT stream will be applied to the remaining frames of the <code class="code">mandelbrot</code> stream. <h4 class="subsubsection">30.118.2.2 Hald CLUT with preview# TOC</h4>

A Hald CLUT is supposed to be a squared image of <code class="code">Level*Level*Level</code> by <code class="code">Level*Level*Level</code> pixels. For a given Hald CLUT, FFmpeg will select the biggest possible square starting at the top left of the picture. The remaining padding pixels (bottom or right) will be ignored. This area can be used to add a preview of the Hald CLUT. Typically, the following generated Hald CLUT will be supported by the <code class="code">haldclut</code> filter: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf " pad=iw+320 [padded_clut]; smptebars=s=320x256, split [a][b]; [padded_clut][a] overlay=W-320:h, curves=color_negative [main]; [main][b] overlay=W-320" -frames:v 1 clut.png </pre>

It contains the original and a preview of the effect of the CLUT: SMPTE color bars are displayed on the right-top, and below the same color bars processed by the color changes. Then, the effect of this Hald CLUT can be visualized with: <pre class="example-preformatted">ffplay input.mkv -vf "movie=clut.png, [in] haldclut" </pre>

 <h3 class="section">30.119 hflip# TOC</h3>

Flip the input video horizontally. For example, to horizontally flip the input video with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf "hflip" out.avi </pre>

 <h3 class="section">30.120 histeq# TOC</h3> This filter applies a global color histogram equalization on a per-frame basis. It can be used to correct video that has a compressed range of pixel intensities. The

filter redistributes the pixel intensities to equalize their distribution across the intensity range. It may be viewed as an "automatically adjusting contrast filter". This filter is useful only for correcting degraded or poorly captured source video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Determine the amount of equalization to be applied. As the strength is reduced, the distribution of pixel intensities more-and-more approaches that of the input frame. The value must be a float number in the range [0,1] and defaults to 0.200. </dd> <dt><samp class="option">intensity</samp></dt> <dd> Set the maximum intensity that can generated and scale the output values appropriately. The strength should be set as desired and then the intensity can be limited if needed to avoid washing-out. The value must be a float number in the range [0,1] and defaults to 0.210. </dd> <dt><samp class="option">antibanding</samp></dt> <dd> Set the antibanding level. If enabled the filter will randomly vary the luminance of output pixels by a small amount to avoid banding of the histogram. Possible values are <code class="code">none</code>, <code class="code">weak</code> or <code class="code">strong</code>. It defaults to <code class="code">none</code>. </dd> </dl>

 <h3 class="section">30.121 histogram# TOC</h3>

Compute and draw a color distribution histogram for the input video. The computed histogram is a representation of the color component distribution in an image. Standard histogram displays the color components distribution in an image. Displays color graph for each color component. Shows distribution of the Y, U, V, A or R, G, B components, depending on input format, in the current frame. Below each graph a color component scale meter is shown. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_height</samp></dt> <dd> Set height of level. Default value is <code class="code">200</code>. Allowed range is [50, 2048]. </dd> <dt><samp class="option">scale_height</samp></dt> <dd> Set height of color scale. Default value is <code class="code">12</code>. Allowed range is [0, 40]. </dd> <dt><samp class="option">display_mode</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default

is <code class=”code”>stack</code>. </dd> <dt><samp class=”option”>levels_mode</samp></dt> <dd> Set mode. Can be either <code class=”code”>linear</code>, or <code class=”code”>logarithmic</code>. Default is <code class=”code”>linear</code>. </dd> <dt><samp class=”option”>components</samp></dt> <dd> Set what color components to display. Default is <code class=”code”>7</code>. </dd> <dt><samp class=”option”>fgopacity</samp></dt> <dd> Set foreground opacity. Default is <code class=”code”>0.7</code>. </dd> <dt><samp class=”option”>bgopacity</samp></dt> <dd> Set background opacity. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>colors_mode</samp></dt> <dd> Set colors mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>whiteonblack</samp>’</dt> <dt>‘<samp class=”samp”>blackonwhite</samp>’</dt> <dt>‘<samp class=”samp”>whiteongray</samp>’</dt> <dt>‘<samp class=”samp”>blackongray</samp>’</dt> <dt>‘<samp class=”samp”>coloronblack</samp>’</dt> <dt>‘<samp class=”samp”>coloronwhite</samp>’</dt> <dt>‘<samp class=”samp”>colorongray</samp>’</dt> <dt>‘<samp class=”samp”>blackoncolor</samp>’</dt> <dt>‘<samp class=”samp”>whiteoncolor</samp>’</dt> <dt>‘<samp class=”samp”>grayoncolor</samp>’</dt> </dl> Default is <code class=”code”>whiteonblack</code>. </dd> </dl>

 <h4 class=”subsection”>30.121.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Calculate and draw histogram: <pre class=”example-preformatted”>ffplay -i input -vf histogram </pre>

 <h3 class=”section”>30.122 hqdn3d# TOC</h3>

This is a high precision/quality 3d denoise filter. It aims to reduce image noise, producing smooth images and making still images really still. It should enhance compressibility. It accepts the following optional parameters: <dl class=”table”> <dt><samp class=”option”>luma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial luma strength. It defaults to 4.0. </dd> <dt><samp class=”option”>chroma_spatial</samp></dt>

<dd> A non-negative floating point number which specifies spatial chroma strength. It defaults to 3.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">luma_tmp</samp></dt> <dd> A floating point number which specifies luma temporal strength. It defaults to 6.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">chroma_tmp</samp></dt> <dd> A floating point number which specifies chroma temporal strength. It defaults to <var class="var">luma_tmp</var>*<var class="var">chroma_spatial</var>/<var class="var">luma_spatial</var>. </dd> </dl>

 <h4 class="subsection">30.122.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.123 hwdownload# TOC</h3>

Download hardware frames to system memory. The input must be in hardware frames, and the output a non-hardware format. Not all formats will be supported on the output - it may be necessary to insert an additional <samp class="option">format</samp> filter immediately following in the graph to get the output in a supported format. <h3 class="section">30.124 hwmap# TOC</h3>

Map hardware frames to system memory or to another device. This filter has several different modes of operation; which one is used depends on the input and output formats: <ul class="itemize mark-bullet"> Hardware frame input, normal frame output

Map the input frames to system memory and pass them to the output. If the original hardware frame is later required (for example, after overlaying something else on part of it), the <samp class="option">hwmap</samp> filter can be used again in the next mode to retrieve it. Normal frame input, hardware frame output

If the input is actually a software-mapped hardware frame, then unmap it - that is, return the original hardware frame. Otherwise, a device must be provided. Create new hardware surfaces on that device for the output, then map them back to the software format at the input and give those frames to the preceding filter. This will then act like

the <samp class=”option”>hwupload</samp> filter, but may be able to avoid an additional copy when the input is already in a compatible format. Hardware frame input and output

A device must be supplied for the output, either directly or with the <samp class=”option”>derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). If the input frames were originally created on the output device, then unmap to retrieve the original frames. Otherwise, map the frames to the output device - create new hardware frames on the output corresponding to the frames on the input.

The following additional parameters are accepted: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the frame mapping mode. Some combination of: <dl class=”table”> <dt><var class=”var”>read</var></dt> <dd> The mapped frame should be readable. </dd> <dt><var class=”var”>write</var></dt> <dd> The mapped frame should be writeable. </dd> <dt><var class=”var”>overwrite</var></dt> <dd> The mapping will always overwrite the entire frame. This may improve performance in some cases, as the original contents of the frame need not be loaded. </dd> <dt><var class=”var”>direct</var></dt> <dd> The mapping must not involve any copying. Indirect mappings to copies of frames are created in some cases where either direct mapping is not possible or it would have unexpected properties. Setting this flag ensures that the mapping is direct and will fail if that is not possible. </dd> </dl> Defaults to <var class=”var”>read+write</var> if not specified. </dd> <dt><samp class=”option”>derive_device <var class=”var”>type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class=”var”>type</var> from the device the input frames exist on. </dd> <dt><samp class=”option”>reverse</samp></dt> <dd> In a hardware to hardware mapping, map in reverse - create frames in the sink and map them back to the source. This may be necessary in some cases where a mapping in one direction is required but only the opposite direction is supported by the devices being used. This option is dangerous - it may break the preceding filter in undefined ways if there are any additional constraints on that filter’s output. Do not use it without fully understanding the implications of its use. </dd> </dl>

 <h3 class=”section”>30.125 hwupload# TOC</h3>

Upload system memory frames to hardware surfaces. The

device to upload to must be supplied when the filter is initialised. If using ffmpeg, select the appropriate device with the <samp class="option">-filter_hw_device</samp> option or with the <samp class="option">derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). The following additional parameters are accepted: <dl class="table"> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> </dl>

 <h3 class="section">30.126 hwupload_cuda# TOC</h3>

Upload system memory frames to a CUDA device. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">device</samp></dt> <dd> The number of the CUDA device to use </dd> </dl>

 <h3 class="section">30.127 hqx# TOC</h3>

Apply a high-quality magnification filter designed for pixel art. This filter was originally created by Maxim Stepin. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">hq2x</code>, <code class="code">3</code> for <code class="code">hq3x</code> and <code class="code">4</code> for <code class="code">hq4x</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">30.128 hstack# TOC</h3> Stack input videos horizontally. All streams must be of same pixel format and of same height. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following option: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">shortest</samp></dt>

<dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class="section">30.129 hsvhold# TOC</h3> Turns a certain HSV range into gray values. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to be gray or not. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more gray pixels, with a higher gray pixel the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class="section">30.130 hsvkey# TOC</h3> Turns a certain HSV range into transparency. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to transparent by adding alpha channel. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the

exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class=”section”>30.131 hue# TOC</h3>

Modify the hue and/or the saturation of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Specify the hue angle as a number of degrees. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>s</samp></dt> <dd> Specify the saturation in the [-10,10] range. It accepts an expression and defaults to "1". </dd> <dt><samp class=”option”>H</samp></dt> <dd> Specify the hue angle as a number of radians. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>b</samp></dt> <dd> Specify the brightness in the [-10,10] range. It accepts an expression and defaults to "0". </dd> </dl>

<samp class=”option”>h</samp> and <samp class=”option”>H</samp> are mutually exclusive, and can’t be specified at the same time. The <samp class=”option”>b</samp>, <samp class=”option”>h</samp>, <samp class=”option”>H</samp> and <samp class=”option”>s</samp> option values are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pts</samp></dt> <dd> presentation timestamp of the input frame expressed in time base units </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class=”option”>tb</samp></dt> <dd> time base of the input video </dd> </dl>

 <h4 class=”subsection”>30.131.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the hue to 90 degrees and the saturation to 1.0: <pre class=”example-preformatted”>hue=h=90:s=1 </pre>

Same command but expressing the hue in radians: <pre class=”example-preformatted”>hue=H=PI/2:s=1 </pre>

Rotate hue and make the saturation swing between 0 and 2 over a period of 1 second: <pre class=”example-preformatted”>hue="H=2*PI*t: s=sin(2*PI*t)+1" </pre>

Apply a 3 seconds saturation fade-in effect starting at 0: <pre class=”example-preformatted”>hue="s=min(t/3\,1)" </pre>

The general fade-in expression can be written as: <pre class=”example-preformatted”>hue="s=min(0\, max((t-START)/DURATION\, 1))" </pre>

Apply a 3 seconds saturation fade-out effect starting at 5 seconds: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (8-t)/3))" </pre>

The general fade-out expression can be written as: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (START+DURATION-t)/DURATION))" </pre>

 <h4 class=”subsection”>30.131.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>b</samp></dt> <dt><samp class=”option”>s</samp></dt> <dt><samp class=”option”>h</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> Modify the hue and/or the saturation and/or brightness of the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>30.132 huesaturation# TOC</h3> Apply hue-saturation-intensity adjustments to input video stream. This filter operates in RGB colorspace. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>hue</samp></dt> <dd> Set the hue shift in degrees to apply. Default is 0. Allowed range is from -180 to 180. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set the saturation shift. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>intensity</samp></dt>

<dd> Set the intensity shift. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set which primary and complementary colors are going to be adjusted. This options is set by providing one or multiple values. This can select multiple colors at once. By default all colors are selected. <dl class=”table”> <dt>‘<samp class=”samp”>r</samp>’</dt> <dd> Adjust reds. </dd> <dt>‘<samp class=”samp”>y</samp>’</dt> <dd> Adjust yellows. </dd> <dt>‘<samp class=”samp”>g</samp>’</dt> <dd> Adjust greens. </dd> <dt>‘<samp class=”samp”>c</samp>’</dt> <dd> Adjust cyans. </dd> <dt>‘<samp class=”samp”>b</samp>’</dt> <dd> Adjust blues. </dd> <dt>‘<samp class=”samp”>m</samp>’</dt> <dd> Adjust magentas. </dd> <dt>‘<samp class=”samp”>a</samp>’</dt> <dd> Adjust all colors. </dd> </dl>

</dd> <dt><samp class=”option”>strength</samp></dt> <dd> Set strength of filtering. Allowed range is from 0 to 100. Default value is 1. </dd> <dt><samp class=”option”>rw, gw, bw</samp></dt> <dd> Set weight for each RGB component. Allowed range is from 0 to 1. By default is set to 0.333, 0.334, 0.333. Those options are used in saturation and lightess processing. </dd> <dt><samp class=”option”>lightness</samp></dt> <dd> Set preserving lightness, by default is disabled. Adjusting hues can change lightness from original RGB triplet, with this option enabled lightness is kept at same value. </dd> </dl>

 <h3 class=”section”>30.133 hysteresis# TOC</h3>

Grow first stream into second stream by connecting components. This makes it possible to build more robust edge masks. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold which is used in filtering. If pixel component value is higher than this value filter algorithm for connecting components is activated. By default value is 0. </dd> </dl>

The <code class=”code”>hysteresis</code> filter also supports the framesync options. <h3 class=”section”>30.134 iccdetect# TOC</h3>

Detect the colorspace from an embedded ICC profile (if present), and update the frame’s tags accordingly. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>force</samp></dt> <dd> If true, the frame’s existing colorspace tags will always be overridden by values detected from an ICC profile. Otherwise, they will only be assigned if they contain <code class=”code”>unknown</code>. Enabled by default. </dd> </dl>

 <h3 class=”section”>30.135 iccgen# TOC</h3>

Generate ICC profiles and attach them to frames. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dd> Configure the colorspace that the ICC profile will be generated for. The default value of <code class=”code”>auto</code> infers the value from the input frame’s metadata, defaulting to BT.709/sRGB as appropriate. See the setparams filter for a list of possible values, but note that <code class=”code”>unknown</code> are not valid values for this filter. </dd> <dt><samp class=”option”>force</samp></dt> <dd> If true, an ICC profile will be generated even if it would overwrite an already existing ICC profile. Disabled by default. </dd> </dl>

 <h3 class=”section”>30.136 identity# TOC</h3>

Obtain the identity score between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max identity score is printed through the logging system. The filter stores the calculated identity scores of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi identity -f null - </pre>

 <h3 class=”section”>30.137 idet# TOC</h3>

Detect video interlacing type. This filter tries to detect if

the input frames are interlaced, progressive, top or bottom field first. It will also try to detect fields that are repeated between adjacent frames (a sign of telecine). Single frame detection considers only immediately adjacent frames when classifying each frame. Multiple frame detection incorporates the classification history of previous frames. The filter will log these metadata values: <dl class="table"> <dt><samp class="option">single.current_frame</samp></dt> <dd> Detected type of current frame using single-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class="option">single.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using single-frame detection. </dd> <dt><samp class="option">multiple.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using multiple-frame detection. </dd> <dt><samp class="option">single.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using single-frame detection. </dd> <dt><samp class="option">multiple.current_frame</samp></dt> <dd> Detected type of current frame using multiple-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class="option">multiple.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using multiple-frame detection. </dd> <dt><samp class="option">single.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using single-frame detection. </dd> <dt><samp class="option">multiple.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using multiple-frame detection. </dd> <dt><samp class="option">single.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using single-frame detection. </dd> <dt><samp class="option">multiple.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using multiple-frame detection. </dd> <dt><samp class="option">repeated.current_frame</samp></dt> <dd> Which field in the current frame is repeated from the last. One of “neither”, “top”, or “bottom”. </dd> <dt><samp class="option">repeated.neither</samp></dt> <dd> Cumulative number of frames with no repeated field. </dd> <dt><samp class="option">repeated.top</samp></dt> <dd> Cumulative number of frames with the top field repeated from the previous frame’s top field. </dd> <dt><samp class="option">repeated.bottom</samp></dt> <dd> Cumulative number of frames with the bottom field repeated from the previous frame’s bottom field. </dd> </dl>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">intl_thres</samp></dt> <dd> Set interlacing threshold. </dd> <dt><samp class="option">prog_thres</samp></dt> <dd> Set progressive threshold. </dd> <dt><samp class="option">rep_thres</samp></dt> <dd> Threshold for repeated field detection. </dd> <dt><samp class="option">half_life</samp></dt> <dd> Number of frames after

which a given frame’s contribution to the statistics is halved (i.e., it contributes only 0.5 to its classification). The default of 0 means that all frames seen are given full weight of 1.0 forever. </dd> <dt><samp class="option">analyze_interlaced_flag</samp></dt> <dd> When this is not 0 then idet will use the specified number of frames to determine if the interlaced flag is accurate, it will not count undetermined frames. If the flag is found to be accurate it will be used without any further computations, if it is found to be inaccurate it will be cleared without any further computations. This allows inserting the idet filter as a low computational method to clean up the interlaced flag </dd> </dl>

 <h4 class="subsection">30.137.1 Examples# TOC</h4>

Inspect the field order of the first 360 frames in a video, in verbose detail: <pre class="example-preformatted">ffmpeg -i INPUT -filter:v idet,metadata=mode=print -frames:v 360 -an -f null - </pre> The idet filter will add analysis metadata to each frame, which will then be discarded. At the end, the filter will also print a final report with statistics. <h3 class="section">30.138 il# TOC</h3>

Deinterleave or interleave fields. This filter allows one to process interlaced images fields without deinterlacing them. Deinterleaving splits the input frame into 2 fields (so called half pictures). Odd lines are moved to the top half of the output image, even lines to the bottom half. You can process (filter) them independently and then re-interleave them. The filter accepts the following options: <dl class="table"> <dt><samp class="option">luma_mode, l</samp></dt> <dt><samp class="option">chroma_mode, c</samp></dt> <dt><samp class="option">alpha_mode, a</samp></dt> <dd> Available values for <var class="var">luma_mode</var>, <var class="var">chroma_mode</var> and <var class="var">alpha_mode</var> are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do nothing. </dd> <dt>‘<samp class="samp">deinterleave, d</samp>’</dt> <dd> Deinterleave fields, placing one above the other. </dd> <dt>‘<samp class="samp">interleave, i</samp>’</dt> <dd> Interleave fields. Reverse the effect of deinterleaving. </dd> </dl> Default value is <code class="code">none</code>. </dd> <dt><samp class="option">luma_swap, ls</samp></dt> <dt><samp class="option">chroma_swap, cs</samp></dt> <dt><samp class="option">alpha_swap, as</samp></dt> <dd> Swap luma/chroma/alpha fields. Exchange even & odd lines. Default value is

<code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">30.138.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.139 inflate# TOC</h3>

Apply inflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values higher than the pixel. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">30.139.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.140 interlace, interlace_vulkan# TOC</h3>

Simple interlacing filter from progressive contents. This interleaves upper (or lower) lines from odd frames with lower (or upper) lines from even frames, halving the frame rate and preserving image height.   <pre class="example-preformatted"> Original Original New Frame Frame 'j' Frame 'j+1' (tff) ========== ========== ========== ========== Line 0 ----------> Frame 'j' Line 0 Line 1 Line 1 ----> Frame 'j+1' Line 1 Line 2 ----------> Frame 'j' Line 2 Line 3 Line 3 ----> Frame 'j+1' Line 3 ... ... New Frame + 1 will be generated by Frame 'j+2' and Frame 'j+3' and so on </pre>

It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">scan</samp></dt> <dd> This determines whether the interlaced frame is taken from the even (tff - default) or odd (bff) lines of the progressive frame. </dd> <dt><samp class="option">lowpass</samp></dt> <dd> Vertical lowpass filter to

avoid twitter interlacing and reduce moire patterns. <dl class="table">
<dt>‘<samp class="samp">0, off</samp>’</dt> <dd> Disable
vertical lowpass filter </dd> <dt>‘<samp class="samp">1,
linear</samp>’</dt> <dd> Enable linear filter (default)
</dd> <dt>‘<samp class="samp">2, complex</samp>’</dt>
<dd> Enable complex filter. This will slightly less reduce twitter and
moire but better retain detail and subjective sharpness impression.
</dd> </dl> </dd> </dl>

 <h3 class="section">30.141 kerndeint<span
class="pull-right"><a class="anchor hidden-xs" href="#kerndeint" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-kerndeint"
aria-hidden="true">TOC</h3>

Deinterlace input video by applying Donald Graft’s adaptive
kernel deinterling. Work on interlaced parts of a video to produce progressive
frames. The description of the accepted parameters follows.
<dl class="table"> <dt><samp class="option">thresh</samp></dt>
<dd> Set the threshold which affects the filter’s tolerance when
determining if a pixel line must be processed. It must be an integer in the range
[0,255] and defaults to 10. A value of 0 will result in applying the process on
every pixels. </dd> <dt><samp class="option">map</samp></dt>
<dd> Paint pixels exceeding the threshold value to white if set to 1. De-
fault is 0. </dd> <dt><samp class="option">order</samp></dt>
<dd> Set the fields order. Swap fields if set to 1, leave fields alone if 0. De-
fault is 0. </dd> <dt><samp class="option">sharp</samp></dt>
<dd> Enable additional sharpening if set to 1. Default is 0. </dd>
<dt><samp class="option">twoway</samp></dt> <dd> Enable
twoway sharpening if set to 1. Default is 0. </dd> </dl>

 <h4 class="subsection">30.141.1 Exam-
ples<a class="anchor hidden-xs" href="#Examples-
110" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-110" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Apply default values:
<pre class="example-preformatted">kerndeint=thresh=10:map=0:order=0:sharp=0:twoway=0
</pre>

Enable additional sharpening: <pre
class="example-preformatted">kerndeint=sharp=1 </pre>

Paint processed pixels in white: <pre
class="example-preformatted">kerndeint=map=1 </pre>

 <h3 class="section">30.142 kirsch<span
class="pull-right"><a class="anchor hidden-xs" href="#kirsch" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-kirsch"
aria-hidden="true">TOC</h3> Apply kirsch operator to

input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">30.142.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.143 lagfun# TOC</h3>

Slowly update darker pixels. This filter makes short flashes of light appear longer. This filter accepts the following options: <dl class="table"> <dt><samp class="option">decay</samp></dt> <dd> Set factor for decaying. Default is .95. Allowed range is from 0 to 1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">30.143.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.144 latticepal# TOC</h3>

Convert packed RGB/BGR input to PAL8 with a per-frame palette whose colors are placed on a face-centered cubic (FCC) lattice spanning the RGB cube. Each frame gets its own palette, built only from the lattice colors actually used by that frame. If a frame uses more than 256 lattice colors, they are reduced by iteratively dropping the color whose removal has the least impact (its pixel count times the squared distance to the nearest remaining color) and mapping its pixels to that nearest remaining color. The alpha channel, if present, is ignored unless the <samp class="option">alpha</samp> option is set. The filter accepts the following options: <dl class="table"> <dt><samp class="option">density</samp></dt>

<dd> Set the lattice density as the number of lattice steps spanning one color axis (0 to 255). Higher values give a finer lattice and thus more possible colors, making it more likely that the per-frame color reduction has to drop some of them. At a density of <code class=”code”>7</code> with 256 palette entries no reduction ever happens since the whole lattice then holds exactly 256 points. Allowed range is <code class=”code”>1</code> to <code class=”code”>255</code>. Default is <code class=”code”>20</code>. </dd> <dt><samp class=”option”>max_colors</samp></dt> <dd> Set the maximum number of palette entries to use. Frames using more lattice colors than this are reduced accordingly. Allowed range is <code class=”code”>2</code> to <code class=”code”>256</code>. Default is <code class=”code”>256</code>. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Quantize the alpha channel too, placing the palette colors on a 4 dimensional D4 lattice over RGBA instead of the FCC lattice over RGB. Inputs without an alpha channel are treated as opaque. With this option the density is limited to <code class=”code”>64</code>. Default is <code class=”code”>0</code> (alpha is ignored and the palette is fully opaque). </dd> <dt><samp class=”option”>refine</samp></dt> <dd> Redistribute the error introduced by dropping colors, which is otherwise uncompensated and can show as discolored patches. Frames that needed no reduction are not changed by this option. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No refinement. </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Run a Floyd-Steinberg error diffusion pass over the whole frame against the final palette. Most accurate on average, but redithers everything, including regions whose colors survived. </dd> <dt>‘<samp class=”samp”>residual</samp>’</dt> <dd> Diffuse only the residual between each pixel’s first pass quantized color and its final palette color. Pixels of surviving colors that receive no residual are unchanged, keeping the character of the selected dither mode outside the reduced regions. </dd> <dt>‘<samp class=”samp”>batched</samp>’</dt> <dd> Interleave color removal and rediffusion: drop half of the excess colors, rediffuse the whole frame against the survivors, recount the colors from the actual assignments and repeat. Colors that the dithering can reproduce by mixing become free to drop while extreme colors keep their pixels and are protected; this gives the best quality at the cost of about log2 of the number of dropped colors rediffusion passes. </dd> </dl>

Default is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>dither</samp></dt> <dd> Select the dithering mode. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No dithering. </dd> <dt>‘<samp class=”samp”>bayer</samp>’</dt> <dd> Ordered 8x8 dithering using the classic swscale dither matrix

with its per-channel decorrelation, offsets spanning one lattice period. </dd> <dt>‘<samp class=”samp”>blue_noise</samp>’</dt> <dd> Void-and-cluster blue noise dithering using three 64x64 masks generated at initialization, one per color channel. </dd> <dt>‘<samp class=”samp”>floyd_steinberg</samp>’</dt> <dd> Floyd-Steinberg error diffusion. </dd> </dl>

Default is <code class=”code”>floyd_steinberg</code>. </dd> </dl>

 <h3 class=”section”>30.145 lenscorrection# TOC</h3>

Correct radial lens distortion This filter can be used to correct for radial distortion as can result from the use of wide angle lenses, and thereby re-rectify the image. To find the right parameters one can use tools available for example as part of opencv or simply trial-and-error. To use opencv use the calibration sample (under samples/cpp) from the opencv sources and extract the k1 and k2 coefficients from the resulting matrix. Note that effectively the same filter is available in the open-source tools Krita and Digikam from the KDE project. In contrast to the vignette filter, which can also be used to compensate lens errors, this filter corrects the distortion of the image, whereas vignette corrects the brightness distribution, so you may want to use both filters together in certain cases, though you will have to take care of ordering, i.e. whether vignetting should be applied before or after lens correction. <h4 class=”subsection”>30.145.1 Options# TOC</h4>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cx</samp></dt> <dd> Relative x-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image width. Default is 0.5. </dd> <dt><samp class=”option”>cy</samp></dt> <dd> Relative y-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image height. Default is 0.5. </dd> <dt><samp class=”option”>k1</samp></dt> <dd> Coefficient of the quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>k2</samp></dt> <dd> Coefficient of the double quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>i</samp></dt> <dd> Set interpo-

lation type. Can be <code class=”code”>nearest</code> or <code class=”code”>bilinear</code>. Default is <code class=”code”>nearest</code>. </dd> <dt><samp class=”option”>fc</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default color is <code class=”code”>black@0</code>. </dd> </dl>

The formula that generates the correction is: <var class=”var”>r_src</var> = <var class=”var”>r_tgt</var> * (1 + <var class=”var”>k1</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^2 + <var class=”var”>k2</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^4) where <var class=”var”>r_0</var> is halve of the image diagonal and <var class=”var”>r_src</var> and <var class=”var”>r_tgt</var> are the distances from the focal point in the source and target images, respectively. <h4 class=”subsection”>30.145.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.146 lensfun# TOC</h3>

Apply lens correction via the lensfun library (<a class=”url” href=”http://lensfun.sourceforge.net/”>http:/

The <code class=”code”>lensfun</code> filter requires the camera make, camera model, and lens model

to apply the lens correction. The filter will load the lensfun database and

query it to find the corresponding camera and lens entries in the database. As

long as these entries can be found with the given options, the filter can

perform corrections on frames. Note that incomplete strings will result in the

filter choosing the best match with the given options, and the filter will

output the chosen camera and lens models (logged with level "info"). You must

provide the make, camera model, and lens model as they are required.

To obtain a list of available makes and models, leave out one or both of <code class=”code”>make</code> and

<code class=”code”>model</code> options. The filter will send the full list to the log with level <code class=”code”>INFO</code>.

The first column is the make and the second column is the model.

To obtain a list of available lenses, set any values for make and model and leave out the

<code class=”code”>lens_model</code> option. The filter will send the full list of lenses in the log with level

<code class=”code”>INFO</code>. The ffmpeg tool will exit after the list is printed.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>make</samp></dt>

<dd> The make of the camera (for example, "Canon"). This option is required.

</dd>

<dt><samp class=”option”>model</samp></dt>

<dd> The model of the camera (for example, "Canon EOS 100D"). This option is

required.

</dd>

<dt><samp class=”option”>lens_model</samp></dt>

<dd> The model of the lens (for example, "Canon EF-S 18-55mm f/3.5-5.6 IS STM"). This

option is required.

</dd>

<dt><samp class=”option”>db_path</samp></dt>

<dd> The full path to the lens database folder. If not set, the filter will attempt to

load the database from the install path when the library was built. Default is unset.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> The type of correction to apply. The following values are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>vignetting</samp>’</dt>

<dd> Enables fixing lens vignetting.

</dd>

<dt>‘<samp class=”samp”>geometry</samp>’</dt>

<dd> Enables fixing lens geometry. This is the default.

</dd>

<dt>‘<samp class=”samp”>subpixel</samp>’</dt>

<dd> Enables fixing chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>vig_geo</samp>’</dt>

<dd> Enables fixing lens vignetting and lens geometry.

</dd>

<dt>‘<samp class=”samp”>vig_subpixel</samp>’</dt>

<dd> Enables fixing lens vignetting and chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>distortion</samp>’</dt>

<dd> Enables fixing both lens geometry and chromatic aberrations.

</dd>

<dt>‘<samp class=”samp”>all</samp>’</dt>

<dd> Enables all possible corrections.

</dd>

</dl>

</dd>

<dt><samp class=”option”>focal_length</samp></dt>

<dd> The focal length of the image/video (zoom; expected constant for video). For

example, a 18–55mm lens has focal length range of [18–55], so a value in that

range should be chosen when using that lens. Default 18.

</dd>

<dt><samp class=”option”>aperture</samp></dt>

<dd> The aperture of the image/video (expected constant for video). Note that

aperture is only used for vignetting correction. Default 3.5.

</dd>

<dt><samp class=”option”>focus_distance</samp></dt>

<dd> The focus distance of the image/video (expected constant for video). Note that

focus distance is only used for vignetting and only slightly affects the

vignetting correction process. If unknown, leave it at the default value (which

is 1000).

</dd>

<dt><samp class=”option”>scale</samp></dt>

<dd> The scale factor which is applied after transformation. After correction the

video is no longer necessarily rectangular. This parameter controls how much of

the resulting image is visible. The value 0 means that a value will be chosen

automatically such that there is little or no unmapped area in the output

image. 1.0 means that no additional scaling is done. Lower values may result

in more of the corrected image being visible, while higher values may avoid

unmapped areas in the output.

</dd>

<dt><samp class=”option”>target_geometry</samp></dt>

<dd> The target geometry of the output image/video. The following values are valid

options:

<dl class=”table”>

<dt>‘<samp class=”samp”>rectilinear (default)</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye</samp>’</dt>

<dt>‘<samp class=”samp”>panoramic</samp>’</dt>

<dt>‘<samp class=”samp”>equirectangular</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_orthographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_stereographic</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_equisolid</samp>’</dt>

<dt>‘<samp class=”samp”>fisheye_thoby</samp>’</dt>

</dl>

</dd>

<dt><samp class=”option”>reverse</samp></dt>

<dd> Apply the reverse of image correction (instead of correcting distortion, apply

it).

</dd>

<dt><samp class=”option”>interpolation</samp></dt>

<dd> The type of interpolation used when correcting distortion. The following values

are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>nearest</samp>’</dt>

<dt>‘<samp class=”samp”>linear (default)</samp>’</dt>

<dt>‘<samp class=”samp”>lanczos</samp>’</dt>

</dl>

</dd>

</dl>

 <h4 class=”subsection”>30.146.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply lens correction with make "Canon", camera model "Canon EOS 100D", and lens model "Canon EF-S 18-55mm f/3.5-5.6 IS STM" with focal length of "18" and aperture of "8.0".

 <pre class=”example-preformatted”>ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8 -c:v h264 -b:v 8000k output.mov </pre>

Apply the same as before, but only for the first 5 seconds of video.

 <pre class=”example-preformatted”>ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8:enable='lte(t\,5)' -c:v h264 -b:v 8000k output.mov </pre>

 <h3 class=”section”>30.147 lcevc# TOC</h3>

Low Complexity Enhancement Video Codec filter based on liblcevc_dec (https://github.com/v-

novaltd/LCEVCdec). <h3 class="section">30.148 libplacebo# TOC</h3>

Flexible GPU-accelerated processing filter based on libplacebo (<a class="url" href="https://code.videolan.org/videolan/libplacebo">https://code.videolan.org/videolan/libplac
  <h4 class="subsection">30.148.1 Options# TOC</h4>

The options for this filter are divided into the following sections: <h4 class="subsubsection">30.148.1.1 Output mode# TOC</h4> These options control the overall output mode. By default, libplacebo will try to preserve the source colorimetry and size as best as it can, but it will apply any embedded film grain, dolby vision metadata or anamorphic SAR present in source frames. <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set the number of inputs. This can be used, alongside the <code class="code">idx</code> variable, to allow placing/blending multiple inputs inside the output frame. This effectively enables functionality similar to hstack, overlay, etc. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default values are <code class="code">iw</code> and <code class="code">ih</code>. Allows for the same expressions as the scale filter. </dd> <dt><samp class="option">crop_x</samp></dt> <dt><samp class="option">crop_y</samp></dt> <dd> Set the input crop x/y expressions, default values are <code class="code">(iw-cw)/2</code> and <code class="code">(ih-ch)/2</code>. </dd> <dt><samp class="option">crop_w</samp></dt> <dt><samp class="option">crop_h</samp></dt> <dd> Set the input crop width/height expressions, default values are <code class="code">iw</code> and <code class="code">ih</code>. </dd> <dt><samp class="option">pos_x</samp></dt> <dt><samp class="option">pos_y</samp></dt> <dd> Set the output placement x/y expressions, default values are <code class="code">(ow-pw)/2</code> and <code class="code">(oh-ph)/2</code>. </dd> <dt><samp class="option">pos_w</samp></dt> <dt><samp class="option">pos_h</samp></dt> <dd> Set the output placement width/height expressions, default values are <code class="code">ow</code> and <code class="code">oh</code>. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Rotate the input frame

clockwise by the specified angle. <dl class="table"> <dt>‘<samp class="samp">0, 360</samp>’</dt> <dt>‘<samp class="samp">90</samp>’</dt> <dt>‘<samp class="samp">180</samp>’</dt> <dt>‘<samp class="samp">270</samp>’</dt> </dl>

</dd> <dt><samp class="option">fps</samp></dt> <dd> Set the output frame rate. This can be rational, e.g. <code class="code">60000/1001</code>. If set to the special string <code class="code">none</code> (the default), input timestamps will instead be passed through to the output unmodified. Otherwise, the input video frames will be interpolated as necessary to rescale the video to the specified target framerate, in a manner as determined by the <samp class="option">frame_mixer</samp> option. </dd> <dt><samp class="option">format</samp></dt> <dd> Set the output format override. If unset (the default), frames will be output in the same format as the respective input frames. Otherwise, format conversion will be performed. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. Note that <samp class="option">force_divisible_by</samp> also works with <code class="code">fit_sense=constraint</code>. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> If enabled, output frames will always have a pixel aspect ratio of 1:1. If disabled (the default), any aspect ratio mismatches, including those from e.g. anamorphic video sources, are forwarded to the output pixel aspect ratio. </dd> <dt><samp class="option">normalize_sar</samp></dt> <dd> Like <samp class="option">reset_sar</samp>, but instead of stretching the video content to fill the new output aspect ratio, the content is instead padded or cropped as necessary. Mutually exclusive with <samp class="option">fit_mode</samp>. Disabled by default. </dd> <dt><samp class="option">pad_crop_ratio</samp></dt> <dd> Specifies a ratio (between <code class="code">0.0</code> and <code class="code">1.0</code>) between padding and cropping when the input aspect ratio does not match the output aspect ratio and <samp class="option">normalize_sar</samp> is in effect. The default of <code class="code">0.0</code> always pads the content with black borders, while a value of <code class="code">1.0</code> always crops off parts of the content. Intermediate values are possible, leading to a mix of the two approaches. </dd> <dt><samp class="option">fit_mode</samp></dt> <dd> Specify the content fit strategy according to a list of predefined modes. Determines how the input image is to be placed inside the destination crop rectangle (as defined by <code class="code">pos_x/y</code> and <code class="code">pos_w/h</code>). The names and their implementations are taken from the CSS ’object-fit’ property. Note that this option is mutually exclusive with <samp class="option">normalize_sar</samp>. Defaults to <code

class=”code”>fill</code>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>fill</samp>’</dt> <dd> Stretch the input to the output rectangle, ignoring aspect ratio mismatches. Note that unless <samp class=”option”>reset_sar</samp> is also enabled, the output will still have the correct pixel aspect ratio tagged. </dd> <dt>‘<samp class=”samp”>contain</samp>’</dt> <dd> Scale the input to fit inside the output, preserving aspect ratio by padding. Equivalent to <samp class=”option”>normalize_sar</samp> with <samp class=”option”>pad_crop_ratio</samp> set to <code class=”code”>0.0</code>. </dd> <dt>‘<samp class=”samp”>cover</samp>’</dt> <dd> Scale the input to fill the output, preserving aspect ratio by cropping. Equivalent to <samp class=”option”>normalize_sar</samp> with <samp class=”option”>pad_crop_ratio</samp> set to <code class=”code”>1.0</code>. </dd> <dt>‘<samp class=”samp”>none, place</samp>’</dt> <dd> Don’t scale the input. The input will be placed inside the output rectangle at its natural size; which may result in additional padding or cropping. </dd> <dt>‘<samp class=”samp”>scale_down</samp>’</dt> <dd> Scale the input down as much as needed to fit inside the output. Equivalent to either <code class=”code”>contain</code> or <code class=”code”>none</code>, depending on whether the input is larger than the output or not. </dd> </dl>

</dd> <dt><samp class=”option”>fit_sense</samp></dt> <dd> When <samp class=”option”>fit_mode</samp> is in use, this option controls how the fit strategy is applied against the specified output resolution. Mutually exclusive with <samp class=”option”>force_original_aspect_ratio</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>target</samp>’</dt> <dd> The computed output resolution is taken as the exact size of the output frame. This is the default behavior. </dd> <dt>‘<samp class=”samp”>constraint</samp>’</dt> <dd> The computed output resolution is a size reference against which the fit mode is applied, enlarging or decreasing the true frame size as needed to fit the content. </dd> </dl>

</dd> <dt><samp class=”option”>fillcolor</samp></dt> <dd> Set the color used to fill the output area not covered by the output image, for example as a result of <samp class=”option”>normalize_sar</samp>. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Defaults to <code class=”code”>black@0</code>. </dd> <dt><samp class=”option”>corner_rounding</samp></dt> <dd> Render frames with rounded corners. The value, given as a float ranging from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>, indicates the relative degree of rounding, from fully square to fully circular. In other words, it gives the radius divided by half the smaller side length. Defaults to <code class=”code”>0.0</code>. </dd>

<dt><samp class=”option”>lut</samp></dt> <dd> Specifies a custom LUT (in Adobe .cube format) to apply to the colors as part of color conversion. The exact interpretation depends on the value of <samp class=”option”>lut_type</samp>. </dd> <dt><samp class=”option”>lut_type</samp></dt> <dd> Controls the interpretation of color values fed to and from the LUT specified as <samp class=”option”>lut</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Chooses the interpretation of the LUT automatically from tagged metadata, and otherwise falls back to ‘<samp class=”samp”>native</samp>’. (Default) </dd> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Applied to raw image contents in its native RGB colorspace (non-linear light), before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>normalized</samp>’</dt> <dd> Applied to the normalized RGB image contents, in linear light, before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>conversion</samp>’</dt> <dd> Fully replaces the conversion from the image color space to the output color space. If such a LUT is present, it has the highest priority, and overrides any ICC profiles, as well as options related to tone mapping and output colorimetry (<samp class=”option”>color_primaries</samp>, <samp class=”option”>color_trc</samp>). </dd> </dl>

</dd> <dt><samp class=”option”>extra_opts</samp></dt> <dd> Pass extra libplacebo internal configuration options. These can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to configure a custom filter kernel ("EWA LanczosSharp") and use it to double the input image resolution: <pre class=”example-preformatted”>-vf "libplacebo=w=iw*2:h=ih*2:extra_opts='upscaler=custom\:upscaler_preset=ewa_
</pre>

</dd> <dt><samp class=”option”>shader_cache</samp></dt> <dd> File path of a cache directory that libplacebo will use to store and load cached shader objects. This cache is not cleaned up automatically. If the path does not end in a directory separator, the generated filenames will be effectively prefixed by the last path component. All directories must already exist.   <pre class=”example-preformatted”>-vf "libplacebo=shader_cache=/tmp/pl-shader-" </pre>

</dd> <dt><samp class=”option”>inherit_device</samp></dt> <dd> Take the Vulkan device from the hardware frames arriving on the input, instead of creating a private one from the filter’s own hardware device. This is useful when the input frames originate on another API (for example CUDA frames uploaded to Vulkan by <code class=”code”>hwupload</code>) and the filter’s device is not itself a Vulkan device: libplacebo would otherwise spin up a separate Vulkan device

that the input frames do not belong to, and the frames could not be mapped. When enabled, the filter requires Vulkan hardware frames on the input and will fail negotiation otherwise. Disabled by default.   <pre class=”example-preformatted”>-init_hw_device vulkan=vk:0 -init_hw_device cuda@vk -filter_hw_device cuda0 ... -vf "hwupload=derive_device=vulkan,libplacebo=inherit_device=1" </pre>

</dd> <dt><samp class=”option”>colorspace</samp></dt> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dt><samp class=”option”>range</samp></dt> <dt><samp class=”option”>chroma_location</samp><
<dd> Configure the colorspace that output frames will be delivered in. For any value other than the default <code class=”code”>auto</code>, conversion will be performed. With <code class=”code”>auto</code>, <samp class=”option”>color_primaries</samp> and <samp class=”option”>color_trc</samp> keep the values of the input frames, while <samp class=”option”>colorspace</samp>, <samp class=”option”>range</samp> and <samp class=”option”>chroma_location</samp> use the values negotiated with the filtergraph, which are those of the input unless a following filter requires others. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> Choose the desired output alpha mode, when the output format has an alpha channel. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>apply_filmgrain</samp></dt> <dd> Apply film grain (e.g. AV1 or H.274) if present in source frames, and strip it from the output. Enabled by default. </dd> <dt><samp class=”option”>apply_dolbyvision</samp></dt> <dd> Apply Dolby Vision RPU metadata if present in source frames, and strip it from the output. Enabled by default. Note that Dolby Vision will always output BT.2020+PQ, overriding the usual input frame metadata. These will also be picked as the values of <code class=”code”>auto</code> for the respective frame output options. </dd> </dl>

In addition to the expression constants documented for the scale filter, the <samp class=”option”>crop_w</samp>, <samp class=”option”>crop_h</samp>, <samp class=”option”>crop_x</samp>, <samp class=”option”>crop_y</samp>, <samp class=”option”>pos_w</samp>, <samp class=”option”>pos_h</samp>, <samp class=”option”>pos_x</samp> and <samp class=”option”>pos_y</samp> options can also contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_idx, idx</samp></dt> <dd> The (0-based) numeric index of the currently active input stream. </dd> <dt><samp class=”option”>crop_w, cw</samp></dt> <dt><samp class=”option”>crop_h, ch</samp></dt> <dd> The computed values of <samp class=”option”>crop_w</samp> and <samp class=”option”>crop_h</samp>. </dd> <dt><samp class=”option”>pos_w, pw</samp></dt> <dt><samp class=”option”>pos_h, ph</samp></dt> <dd> The computed values of <samp class=”option”>pos_w</samp>

and <samp class=”option”>pos_h</samp>. </dd> <dt><samp class=”option”>in_t, t</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class=”option”>out_t, ot</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The input frame number, starting with 0. </dd> </dl>

 <h4 class=”subsubsection”>30.148.1.2 Scaling# TOC</h4> The options in this section control how libplacebo performs upscaling and (if necessary) downscaling. Note that libplacebo will always internally operate on 4:4:4 content, so any sub-sampled chroma formats such as <code class=”code”>yuv420p</code> will necessarily be upsampled and downsampled as part of the rendering process. That means scaling might be in effect even if the source and destination resolution are the same. <dl class=”table”> <dt><samp class=”option”>upscaler</samp></dt> <dt><samp class=”option”>downscaler</samp></dt> <dd> Configure the filter kernel used for upscaling and downscaling. The respective defaults are <code class=”code”>spline36</code> and <code class=”code”>mitchell</code>. For a full list of possible values, pass <code class=”code”>help</code> to these options. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Forces the use of built-in GPU texture sampling (typically bilinear). Extremely fast but poor quality, especially when downscaling. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear interpolation. Can generally be done for free on GPUs, except when doing so would lead to aliasing. Fast and low quality. </dd> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Nearest-neighbour interpolation. Sharp but highly aliasing. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Algorithm that looks visually similar to nearest-neighbour interpolation but tries to preserve pixel aspect ratio. Good for pixel art, since it results in minimal distortion of the artistic appearance. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Standard sinc-sinc interpolation kernel. </dd> <dt>‘<samp class=”samp”>spline36</samp>’</dt> <dd> Cubic spline approximation of lanczos. No difference in performance, but has very slightly less ringing. </dd> <dt>‘<samp class=”samp”>ewa_lanczos</samp>’</dt> <dd> Elliptically weighted average version of lanczos, based on a jinc-jinc kernel. This is also popularly referred to as just "Jinc scaling". Slow but very high quality. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> Gaussian kernel. Has certain ideal mathematical properties, but subjectively very blurry. </dd> <dt>‘<samp

class=”samp”>mitchell</samp>’</dt> <dd> Cubic BC spline with parameters recommended by Mitchell and Netravali. Very little ringing. </dd> </dl>

</dd> <dt><samp class=”option”>frame_mixer</samp></dt> <dd> Controls the kernel used for mixing frames temporally. The default value is <code class=”code”>none</code>, which disables frame mixing. For a full list of possible values, pass <code class=”code”>help</code> to this option. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables frame mixing, giving a result equivalent to "nearest neighbour" semantics. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Oversamples the input video to create a "Smooth Motion"-type effect: if an output frame would exactly fall on the transition between two video frames, it is blended according to the relative overlap. This is the recommended option whenever preserving the original subjective appearance is desired. </dd> <dt>‘<samp class=”samp”>mitchell_clamp</samp>’</dt> <dd> Larger filter kernel that smoothly interpolates multiple frames in a manner designed to eliminate ringing and other artefacts as much as possible. This is the recommended option wherever maximum visual smoothness is desired. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linear blend/fade between frames. Especially useful for constructing e.g. slideshows. </dd> </dl>

</dd> <dt><samp class=”option”>antiringing</samp></dt> <dd> Enables anti-ringing (for non-EWA filters). The value (between <code class=”code”>0.0</code> and <code class=”code”>1.0</code>) configures the strength of the anti-ringing algorithm. May increase aliasing if set too high. Disabled by default. </dd> <dt><samp class=”option”>sigmoid</samp></dt> <dd> Enable sigmoidal compression during upscaling. Reduces ringing slightly. Enabled by default. </dd> </dl>

 <h4 class=”subsubsection”>30.148.1.3 Deinterlacing# TOC</h4> Deinterlacing is automatically supported when frames are tagged as interlaced, however frames are not deinterlaced unless a deinterlacing algorithm is chosen. <dl class=”table”> <dt><samp class=”option”>deinterlace</samp></dt> <dd> The the deinterlacing algorithm to use. <dl class=”table”> <dt>‘<samp class=”samp”>weave</samp>’</dt> <dd> No deinterlacing, weave fields together into a single frame. This is the default. </dd> <dt>‘<samp class=”samp”>bob</samp>’</dt> <dd> Naive bob deinterlacing, simply repeat each field line twice. </dd> <dt>‘<samp class=”samp”>yadif</samp>’</dt>

<dd> Yet another deinterlacing filter. See the yadif filter for more details. </dd> <dt>‘<samp class="samp">bwdif</samp>’</dt> <dd> Bob weaver deinterlacing filter. See the bwdif filter for more details. </dd> </dl>

</dd> <dt><samp class="option">skip_spatial_check</samp></dt> <dd> Skip the spatial deinterlacing check when using <code class="code">yadif</code> deinterlacing. </dd> <dt><samp class="option">send_fields</samp></dt> <dd> Output a frame for each field, rather than for each frame. Note that this will always double the tagged output frame rate, even if the input does not contain any interlaced frames. Disabled by default. </dd> </dl>

 <h4 class="subsubsection">30.148.1.4 Debanding# TOC</h4> Libplacebo comes with a built-in debanding filter that is good at counteracting many common sources of banding and blocking. Turning this on is highly recommended whenever quality is desired. <dl class="table"> <dt><samp class="option">deband</samp></dt> <dd> Enable (fast) debanding algorithm. Disabled by default. </dd> <dt><samp class="option">deband_iterations</samp></dt> <dd> Number of deband iterations of the debanding algorithm. Each iteration is performed with progressively increased radius (and diminished threshold). Recommended values are in the range <code class="code">1</code> to <code class="code">4</code>. Defaults to <code class="code">1</code>. </dd> <dt><samp class="option">deband_threshold</samp></dt> <dd> Debanding filter strength. Higher numbers lead to more aggressive debanding. Defaults to <code class="code">4.0</code>. </dd> <dt><samp class="option">deband_radius</samp></dt> <dd> Debanding filter radius. A higher radius is better for slow gradients, while a lower radius is better for steep gradients. Defaults to <code class="code">16.0</code>. </dd> <dt><samp class="option">deband_grain</samp></dt> <dd> Amount of extra output grain to add. Helps hide imperfections. Defaults to <code class="code">6.0</code>. </dd> </dl>

 <h4 class="subsubsection">30.148.1.5 Color adjustment# TOC</h4> A collection of subjective color controls. Not very rigorous, so the exact effect will vary somewhat depending on the input primaries and colorspace. <dl class="table"> <dt><samp class="option">brightness</samp></dt> <dd> Brightness boost, between <code class="code">-1.0</code> and <code class="code">1.0</code>. Defaults to <code class="code">0.0</code>. </dd> <dt><samp class="option">contrast</samp></dt>

<dd> Contrast gain, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Saturation gain, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">hue</samp></dt> <dd> Hue shift in radians, between <code class="code">-3.14</code> and <code class="code">3.14</code>. Defaults to <code class="code">0.0</code>. This will rotate the UV subvector, defaulting to BT.709 coefficients for RGB inputs. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Gamma adjustment, between <code class="code">0.0</code> and <code class="code">16.0</code>. Defaults to <code class="code">1.0</code>. </dd> <dt><samp class="option">temperature</samp></dt> <dd> Color temperature adjustment. Lower values make the output warmer/redder, down to <code class="code">1667</code>, while higher values make the output cooler/bluer, up to <code class="code">25000</code>. Defaults to <code class="code">6500</code> (neutral white). </dd> <dt><samp class="option">cones</samp></dt> <dd> Cone model to use for color blindness simulation. Accepts any combination of <code class="code">l</code>, <code class="code">m</code> and <code class="code">s</code>. Here are some examples: <dl class="table"> <dt>‘<samp class="samp">m</samp>’</dt> <dd> Deuteranomaly / deuteranopia (affecting 3%-4% of the population) </dd> <dt>‘<samp class="samp">l</samp>’</dt> <dd> Protanomaly / protanopia (affecting 1%-2% of the population) </dd> <dt>‘<samp class="samp">l+m</samp>’</dt> <dd> Monochromacy (very rare) </dd> <dt>‘<samp class="samp">l+m+s</samp>’</dt> <dd> Achromatopsy (complete loss of daytime vision, extremely rare) </dd> </dl>

</dd> <dt><samp class="option">cone-strength</samp></dt> <dd> Gain factor for the cones specified by <code class="code">cones</code>, between <code class="code">0.0</code> and <code class="code">10.0</code>. A value of <code class="code">1.0</code> results in no change to color vision. A value of <code class="code">0.0</code> (the default) simulates complete loss of those cones. Values above <code class="code">1.0</code> result in exaggerating the differences between cones, which may help compensate for reduced color vision. </dd> </dl>

 <h4 class="subsubsection">30.148.1.6 Peak detection# TOC</h4> To help deal with sources that only have static HDR10 metadata (or no tagging whatsoever), libplacebo uses its own internal frame analysis compute shader to analyze source frames and adapt the tone mapping function in realtime. If this is too slow, or if exactly reproducible frame-perfect results

are needed, it’s recommended to turn this feature off. <dl class="table"> <dt><samp class="option">peak_detect</samp></dt> <dd> Enable HDR peak detection. Ignores static MaxCLL/MaxFALL values in favor of dynamic detection from the input. Note that the detected values do not get written back to the output frames, they merely guide the internal tone mapping process. Enabled by default. </dd> <dt><samp class="option">smoothing_period</samp></dt> <dd> Peak detection smoothing period, between <code class="code">0.0</code> and <code class="code">1000.0</code>. Higher values result in peak detection becoming less responsive to changes in the input. Defaults to <code class="code">20.0</code>. </dd> <dt><samp class="option">scene_threshold_low</samp></dt> <dt><samp class="option">scene_threshold_high</
<dd> Lower and upper thresholds for scene change detection. Expressed in a logarithmic scale between <code class="code">0.0</code> and <code class="code">100.0</code>. Default to <code class="code">1.0</code> and <code class="code">3.0</code>, respectively. Setting either to a negative value disables this functionality. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Which percentile of the frame brightness histogram to use as the source peak for tone-mapping. Defaults to <code class="code">99.995</code>, a fairly conservative value. Setting this to <code class="code">100.0</code> disables frame histogram measurement and instead uses the true peak brightness for tone-mapping. </dd> </dl>

 <h4 class="subsubsection">30.148.1.7 Tone mapping# TOC</h4>
The options in this section control how libplacebo performs tone-mapping and gamut-mapping when dealing with mismatches between wide-gamut or HDR content. In general, libplacebo relies on accurate source tagging and mastering display gamut information to produce the best results. <dl class="table"> <dt><samp class="option">gamut_mode</samp></dt> <dd> How to handle out-of-gamut colors that can occur as a result of colorimetric gamut mapping. <dl class="table"> <dt>‘<samp class="samp">clip</samp>’</dt> <dd> Do nothing, simply clip out-of-range colors to the RGB volume. Low quality but extremely fast. </dd> <dt>‘<samp class="samp">perceptual</samp>’</dt>
<dd> Perceptually soft-clip colors to the gamut volume. This is the default. </dd> <dt>‘<samp class="samp">relative</samp>’</dt>
<dd> Relative colorimetric hard-clip. Similar to <code class="code">perceptual</code> but without the soft knee. </dd> <dt>‘<samp class="samp">saturation</samp>’</dt
<dd> Saturation mapping, maps primaries directly to primaries in RGB space. Not recommended except for artificial computer graphics for which a bright, saturated display is desired. </dd> <dt>‘<samp class="samp">absolute</samp>’</dt> <dd> Absolute colori-

metric hard-clip. Performs no adjustment of the white point. </dd> <dt>‘<samp class=”samp”>desaturate</samp>’</dt> <dd> Hard-desaturates out-of-gamut colors towards white, while preserving the luminance. Has a tendency to distort the visual appearance of bright objects. </dd> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dd> Linearly reduces content brightness to preserves saturated details, followed by clipping the remaining out-of-gamut colors. </dd> <dt>‘<samp class=”samp”>warn</samp>’</dt> <dd> Highlight out-of-gamut pixels (by inverting/marking them). </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly reduces chromaticity of the entire image to make it fit within the target color volume. Be careful when using this on BT.2020 sources without proper mastering metadata, as doing so will lead to excessive desaturation. </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping</samp></dt> <dd> Tonemapping algorithm to use. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection based on internal heuristics. This is the default. </dd> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Performs no tone-mapping, just clips out-of-range colors. Retains perfect color accuracy for in-range colors but completely destroys out-of-range information. Does not perform any black point adaptation. Not configurable. </dd> <dt>‘<samp class=”samp”>st2094-40</samp>’</dt> <dd> EETF from SMPTE ST 2094-40 Annex B, which applies the Bezier curves from HDR10+ dynamic metadata based on Bezier curves to perform tone-mapping. The OOTF used is adjusted based on the ratio between the targeted and actual display peak luminances. </dd> <dt>‘<samp class=”samp”>st2094-10</samp>’</dt> <dd> EETF from SMPTE ST 2094-10 Annex B.2, which takes into account the input signal average luminance in addition to the maximum/minimum. The configurable contrast parameter influences the slope of the linear output segment, defaulting to <code class=”code”>1.0</code> for no increase/decrease in contrast. Note that this does not currently include the subjective gain/offset/gamma controls defined in Annex B.3. </dd> <dt>‘<samp class=”samp”>bt.2390</samp>’</dt> <dd> EETF from the ITU-R Report BT.2390, a hermite spline roll-off with linear segment. The knee point offset is configurable. Note that this parameter defaults to <code class=”code”>1.0</code>, rather than the value of <code class=”code”>0.5</code> from the ITU-R spec. </dd> <dt>‘<samp class=”samp”>bt.2446a</samp>’</dt> <dd> EETF from ITU-R Report BT.2446, method A. Designed for well-mastered HDR sources. Can be used for both forward and inverse tone mapping. Not configurable. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Simple spline consisting of two polynomials, joined by a single pivot point. The parameter gives the pivot point (in PQ space), defaulting to <code class=”code”>0.30</code>. Can be used for both

forward and inverse tone mapping. </dd> <dt>‘<samp class=”samp”>reinhard</samp>’</dt> <dd> Simple non-linear, global tone mapping algorithm. The parameter specifies the local contrast coefficient at the display peak. Essentially, a parameter of <code class=”code”>0.5</code> implies that the reference white will be about half as bright as when clipping. Defaults to <code class=”code”>0.5</code>, which results in the simplest formulation of this function. </dd> <dt>‘<samp class=”samp”>mobius</samp>’</dt> <dd> Generalization of the reinhard tone mapping algorithm to support an additional linear slope near black. The tone mapping parameter indicates the trade-off between the linear section and the non-linear section. Essentially, for a given parameter <var class=”var”>x</var>, every color value below <var class=”var”>x</var> will be mapped linearly, while higher values get non-linearly tone-mapped. Values near <code class=”code”>1.0</code> make this curve behave like <code class=”code”>clip</code>, while values near <code class=”code”>0.0</code> make this curve behave like <code class=”code”>reinhard</code>. The default value is <code class=”code”>0.3</code>, which provides a good balance between colorimetric accuracy and preserving out-of-gamut details. </dd> <dt>‘<samp class=”samp”>hable</samp>’</dt> <dd> Piecewise, filmic tone-mapping algorithm developed by John Hable for use in Uncharted 2, inspired by a similar tone-mapping algorithm used by Kodak. Popularized by its use in video games with HDR rendering. Preserves both dark and bright details very well, but comes with the drawback of changing the average brightness quite significantly. This is sort of similar to <code class=”code”>reinhard</code> with parameter <code class=”code”>0.24</code>. </dd> <dt>‘<samp class=”samp”>gamma</samp>’</dt> <dd> Fits a gamma (power) function to transfer between the source and target color spaces, effectively resulting in a perceptual hard-knee joining two roughly linear sections. This preserves details at all scales fairly accurately, but can result in an image with a muted or dull appearance. The parameter is used as the cutoff point, defaulting to <code class=”code”>0.5</code>. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly stretches the input range to the output range, in PQ space. This will preserve all details accurately, but results in a significantly different average brightness. Can be used for inverse tone-mapping in addition to regular tone-mapping. The parameter can be used as an additional linear gain coefficient (defaulting to <code class=”code”>1.0</code>). </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping_param</samp></dt> <dd> For tunable tone mapping functions, this parameter can be used to fine-tune the curve behavior. Refer to the documentation of <code class=”code”>tonemapping</code>. The default value of <code class=”code”>0.0</code> is replaced by the curve’s preferred default setting. </dd> <dt><samp class=”option”>inverse_tonemapping</samp></dt>

<dd> If enabled, this filter will also attempt stretching SDR signals to fill HDR output color volumes. Disabled by default. </dd> <dt><samp class=”option”>tonemapping_lut_size</samp></dt> <dd> Size of the tone-mapping LUT, between <code class=”code”>2</code> and <code class=”code”>1024</code>. Defaults to <code class=”code”>256</code>. Note that this figure is squared when combined with <code class=”code”>peak_detect</code>. </dd> <dt><samp class=”option”>contrast_recovery</samp></dt> <dd> Contrast recovery strength. If set to a value above <code class=”code”>0.0</code>, the source image will be divided into high-frequency and low-frequency components, and a portion of the high-frequency image is added back onto the tone-mapped output. May cause excessive ringing artifacts for some HDR sources, but can improve the subjective sharpness and detail left over in the image after tone-mapping. Defaults to <code class=”code”>0.30</code>. </dd> <dt><samp class=”option”>contrast_smoothness</samp></dt> <dd> Contrast recovery lowpass kernel size. Defaults to <code class=”code”>3.5</code>. Increasing or decreasing this will affect the visual appearance substantially. Has no effect when <code class=”code”>contrast_recovery</code> is disabled. </dd> </dl>

 <h4 class=”subsubsection”>30.148.1.8 Dithering# TOC</h4> By default, libplacebo will dither whenever necessary, which includes rendering to any integer format below 16-bit precision. It’s recommended to always leave this on, since not doing so may result in visible banding in the output, even if the <code class=”code”>debanding</code> filter is enabled. If maximum performance is needed, use <code class=”code”>ordered_fixed</code> instead of disabling dithering. <dl class=”table”> <dt><samp class=”option”>dithering</samp></dt> <dd> Dithering method to use. Accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables dithering completely. May result in visible banding. </dd> <dt>‘<samp class=”samp”>blue</samp>’</dt> <dd> Dither with pseudo-blue noise. This is the default. </dd> <dt>‘<samp class=”samp”>ordered</samp>’</dt> <dd> Tunable ordered dither pattern. </dd> <dt>‘<samp class=”samp”>ordered_fixed</samp>’</dt> <dd> Faster ordered dither with a fixed size of <code class=”code”>6</code>. Texture-less. </dd> <dt>‘<samp class=”samp”>white</samp>’</dt> <dd> Dither with white noise. Texture-less. </dd> </dl>

</dd> <dt><samp class=”option”>dither_lut_size</samp></dt> <dd> Dither LUT size, as log base2 between <code class=”code”>1</code> and <code class=”code”>8</code>. Defaults to <code class=”code”>6</code>, corresponding to a LUT size of <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>dither_temporal</samp></dt>

<dd> Enables temporal dithering. Disabled by default. </dd>
</dl>

 <h4 class="subsubsection">30.148.1.9 Custom shaders# TOC</h4>
libplacebo supports a number of custom shaders based on the mpv .hook GLSL syntax. A collection of such shaders can be found here: https://github.com/mpv-player/mpv/wiki/User-Scripts#user-shaders A full description of the mpv shader format is beyond the scope of this section, but a summary can be found here: https://mpv.io/manual/master/#options-glsl-shader <dl class="table"> <dt><samp class="option">custom_shader_path</samp></dt>
<dd> Specifies a path to a custom shader file to load at runtime. </dd> <dt><samp class="option">custom_shader_bin</samp></dt>
<dd> Specifies a complete custom shader as a raw string. </dd>
</dl>

 <h4 class="subsubsection">30.148.1.10 Debugging / performance# TOC</h4> All of the options in this section default off. They may be of assistance when attempting to squeeze the maximum performance at the cost of quality. <dl class="table"> <dt><samp class="option">skip_aa</samp></dt>
<dd> Disable anti-aliasing when downscaling. </dd> <dt><samp class="option">disable_linear</samp></dt> <dd> Disable linear light scaling. </dd> <dt><samp class="option">disable_builtin</samp></dt>
<dd> Disable built-in GPU sampling (forces LUT). </dd>
<dt><samp class="option">disable_fbos</samp></dt> <dd> Forcibly disable FBOs, resulting in loss of almost all functionality, but offering the maximum possible speed. </dd> </dl>

 <h4 class="subsection">30.148.2 Commands# TOC</h4> This filter supports almost all of the above options as commands. The options negotiated with the filtergraph, <samp class="option">format</samp>, <samp class="option">colorspace</samp>, <samp class="option">range</samp>, <samp class="option">chroma_location</samp> and <samp class="option">alpha_mode</samp>, are fixed once the filtergraph is configured and can not be changed with commands. <h4 class="subsection">30.148.3

Examples# TOC</h4> <ul class=”itemize mark-bullet”> Tone-map input to standard gamut BT.709 output: <pre class=”example-preformatted”>libplacebo=colorspace=bt709:color_primaries=bt709 </pre>

Rescale input to fit into standard 1080p, with high quality scaling: <pre class=”example-preformatted”>libplacebo=w=1920:h=1080:force_original_aspe </pre>

Interpolate low FPS / VFR input to smoothed constant 60 fps output: <pre class=”example-preformatted”>libplacebo=fps=60:frame_mixer=mitchell_clam </pre>

Convert input to standard sRGB JPEG: <pre class=”example-preformatted”>libplacebo=format=yuv420p:colorspace=bt470bg:color_primaries=bt709 2-1:range=pc </pre>

Use higher quality debanding settings: <pre class=”example-preformatted”>libplacebo=deband=true:deband_iterations=3:deband_radius=8:deband_thr </pre>

Run this filter on the CPU, on systems with Mesa installed (and with the most expensive options disabled): <pre class=”example-preformatted”>ffmpeg ... -init_hw_device vulkan:llvmpipe ... -vf libplacebo=upscaler=none:downscaler=none:peak_detect=false </pre>

Suppress CPU-based AV1/H.274 film grain application in the decoder, in favor of doing it with this filter. Note that this is only a gain if the frames are either already on the GPU, or if you’re using libplacebo for other purposes, since otherwise the VRAM roundtrip will more than offset any expected speedup. <pre class=”example-preformatted”>ffmpeg -export_side_data +film_grain ... -vf libplacebo=apply_filmgrain=true </pre>

Interop with VAAPI hwdec to avoid round-tripping through RAM: <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -hwaccel vaapi -hwaccel_output_format vaapi ... -vf libplacebo </pre>

 <h3 class=”section”>30.149 libvmaf# TOC</h3>

Calculate the VMAF (Video Multi-Method Assessment Fusion) score for a reference/distorted pair of input videos. The first input is the distorted video, and the second input is the reference video. The

obtained VMAF score is printed through the logging system. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class="code">./configure --enable-libvmaf</code>. The filter has following options: <dl class="table"> <dt><samp class="option">model</samp></dt> <dd> A ‘|‘ delimited list of vmaf models. Each model can be configured with a number of parameters. Default value: <code class="code">"version=vmaf_v0.6.1"</code> </dd> <dt><samp class="option">feature</samp></dt> <dd> A ‘|‘ delimited list of features. Each feature can be configured with a number of parameters. </dd> <dt><samp class="option">log_path</samp></dt> <dd> Set the file path to be used to store log files. </dd> <dt><samp class="option">log_fmt</samp></dt> <dd> Set the format of the log file (xml, json, csv, or sub). </dd> <dt><samp class="option">pool</samp></dt> <dd> Set the pool method to be used for computing vmaf. Options are <code class="code">min</code>, <code class="code">harmonic_mean</code> or <code class="code">mean</code> (default). </dd> <dt><samp class="option">n_threads</samp></dt> <dd> Set number of threads to be used when initializing libvmaf. Default value: <code class="code">0</code>, no threads. </dd> <dt><samp class="option">n_subsample</samp></d <dd> Set frame subsampling interval to be used. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">30.149.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> In the examples below, a distorted video <samp class="file">distorted.mpg</samp> is compared with a reference file <samp class="file">reference.mpg</samp>.

Basic usage: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf=log_path=output.xml -f null - </pre>

Example with multiple models: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='model=version=vmaf_v0.6.1\\:name=vmaf|version=vmaf_v0.6.1neg\\:name=vmaf_neg' -f null - </pre>

Example with multiple additional features: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='feature=name=psnr|name=ciede' -f null - </pre>

Example with options and different containers: <pre class="example-preformatted">ffmpeg -i distorted.mpg -i reference.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-

STARTPTS[ref];[main][ref]libvmaf=log_fmt=json:log_path=output.json"
-f null - </pre>

 <h3 class="section">30.150 libvmaf_cuda# TOC</h3>

This is the CUDA variant of the libvmaf filter. It only accepts CUDA frames. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class="code">./configure --enable-nonfree --enable-ffnvcodec --enable-libvmaf</code>. <h4 class="subsection">30.150.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Basic usage showing CUVID hardware decoding and CUDA scaling with scale_cuda: <pre class="example-preformatted">ffmpeg \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i dis.obu \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i ref.obu \ -filter_complex " [0:v]scale_cuda=format=yuv420p[dis]; \ [1:v]scale_cuda=format=yuv420p[ref]; \ [dis][ref]libvmaf_cuda=log_fmt=json:log_path=output.json " \ -f null - </pre>

 <h3 class="section">30.151 limitdiff# TOC</h3> Apply limited difference filter using second and optionally third video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set the threshold to use when allowing certain differences between video streams. Any absolute difference value lower or exact than this threshold will pick pixel components from first video stream. </dd> <dt><samp class="option">elasticity</samp></dt> <dd> Set the elasticity of soft thresholding when processing video streams. This value multiplied with first one sets second threshold. Any absolute difference value greater or exact than second threshold will pick pixel components from second video stream. For values between those two threshold linear interpolation between first and second video stream will be used. </dd> <dt><samp class="option">reference</samp></dt> <dd> Enable the reference (third) video stream processing. By default is disabled. If set, this video stream will be used for calculating absolute difference with first video stream. </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available.

</dd> </dl>

 <h4 class="subsection">30.151.1 Commands# TOC</h4>

This filter supports the all above options as commands except option ‘<samp class="samp">reference</samp>’. <h3 class="section">30.152 limiter# TOC</h3>

Limits the pixel components values to the specified range [min, max]. The filter accepts the following options: <dl class="table"> <dt><samp class="option">min</samp></dt> <dd> Lower bound. Defaults to the lowest representable value for the input. Set to <var class="var">auto</var> to use the minimum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class="option">max</samp></dt> <dd> Upper bound. Defaults to the highest representable value for the input. Set to <var class="var">auto</var> to use the maximum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class="subsection">30.152.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.153 loop# TOC</h3>

Loop video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal size in number of frames. Default is 0. </dd> <dt><samp class="option">start</samp></dt> <dd> Set first frame of loop. Default is 0. </dd> <dt><samp class="option">time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class="var">start</var> is set to <code class="code">-1</code>. </dd> </dl>

 <h4 class="subsection">30.153.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Loop single first frame infinitely: <pre class="example-preformatted">loop=loop=-1:size=1:start=0 </pre>

Loop single first frame 10 times: <pre class="example-preformatted">loop=loop=10:size=1:start=0 </pre>

Loop 10 first frames 5 times: <pre class="example-preformatted">loop=loop=5:size=10:start=0 </pre>

 <h3 class="section">30.154 lut1d# TOC</h3>

Apply a 1D LUT to an input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> Set the 1D LUT file name. Currently supported formats: <dl class="table"> <dt>‘<samp class="samp">cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class="samp">csp</samp>’</dt> <dd> cineSpace </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> Interpolate values using the linear interpolation. </dd> <dt>‘<samp class="samp">cosine</samp>’</dt> <dd> Interpolate values using the cosine interpolation. </dd> <dt>‘<samp class="samp">cubic</samp>’</dt> <dd> Interpolate values using the cubic interpolation. </dd> <dt>‘<samp class="samp">spline</samp>’</dt> <dd> Interpolate values using the spline interpolation. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.154.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.155 lut3d<a class="anchor

hidden-xs" href="#lut3d" aria-hidden="true"># TOC</h3>

Apply a 3D LUT to an input video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">file</samp></dt> <dd> Set the 3D LUT file name. Currently supported formats: <dl class="table"> <dt>‘<samp class="samp">3dl</samp>’</dt> <dd> AfterEffects </dd> <dt>‘<samp class="samp">cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class="samp">dat</samp>’</dt> <dd> DaVinci </dd> <dt>‘<samp class="samp">m3d</samp>’</dt> <dd> Pandora </dd> <dt>‘<samp class="samp">csp</samp>’</dt> <dd> cineSpace </dd> </dl> </dd> <dt><samp class="option">interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class="samp">trilinear</samp>’</dt> <dd> Interpolate values using the 8 points defining a cube. </dd> <dt>‘<samp class="samp">tetrahedral</samp>’</dt> <dd> Interpolate values using a tetrahedron. </dd> <dt>‘<samp class="samp">pyramid</samp>’</dt> <dd> Interpolate values using a pyramid. </dd> <dt>‘<samp class="samp">prism</samp>’</dt> <dd> Interpolate values using a prism. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.155.1 Commands# TOC</h4>

This filter supports the <code class="code">interp</code> option as commands. <h3 class="section">30.156 lumakey# TOC</h3>

Turn certain luma values into transparency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set the luma which will be used as base for transparency. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set the range of luma values to be keyed out. Default value is <code class="code">0.01</code>. </dd> <dt><samp class="option">softness</samp></dt> <dd> Set the range of softness. Default value is <code class="code">0</code>. Use this to control gradual transition from zero to full transparency. </dd> </dl>

 <h4 class="subsection">30.156.1 Com-

mands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.157 lut, lutrgb, lutyuv# TOC</h3>

Compute a look-up table for binding each pixel component input value to an output value, and apply it to the input video. <var class="var">lutyuv</var> applies a lookup table to a YUV input video, <var class="var">lutrgb</var> to an RGB input video. These filters accept the following parameters: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class="option">r</samp></dt> <dd> set red component expression </dd> <dt><samp class="option">g</samp></dt> <dd> set green component expression </dd> <dt><samp class="option">b</samp></dt> <dd> set blue component expression </dd> <dt><samp class="option">a</samp></dt> <dd> alpha component expression </dd> <dt><samp class="option">y</samp></dt> <dd> set Y/luma component expression </dd> <dt><samp class="option">u</samp></dt> <dd> set U/Cb component expression </dd> <dt><samp class="option">v</samp></dt> <dd> set V/Cr component expression </dd> </dl>

Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class="var">c*</var> options depends on the format in input. The <var class="var">lut</var> filter requires either YUV or RGB pixel formats in input, <var class="var">lutrgb</var> requires RGB pixel formats in input, and <var class="var">lutyuv</var> requires YUV. The expressions can contain the following constants and functions: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp

class=”option”>clipval</samp></dt> <dd> The input value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>maxval</samp></dt> <dd> The maximum value for the pixel component. </dd> <dt><samp class=”option”>minval</samp></dt> <dd> The minimum value for the pixel component. </dd> <dt><samp class=”option”>negval</samp></dt> <dd> The negated value for the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range; it corresponds to the expression "maxval-clipval+minval". </dd> <dt><samp class=”option”>clip(val)</samp></dt> <dd> The computed value in <var class=”var”>val</var>, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>gammaval(gamma)</samp></dt> <dd> The computed gamma correction value of the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. It corresponds to the expression "pow((clipval-minval)/(maxval-minval)\,<var class=”var”>gamma</var>)*(maxval-minval)+minval" </dd> </dl>

All expressions default to "clipval". <h4 class=”subsection”>30.157.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>30.157.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Negate input video: <pre class=”example-preformatted”>lutrgb="r=maxval+minval-val:g=maxval+minval-val:b=maxval+minval-val" lutyuv="y=maxval+minval-val:u=maxval+minval-val:v=maxval+minval-val" </pre>

The above is the same as: <pre class=”example-preformatted”>lutrgb="r=negval:g=negval:b=negval" lutyuv="y=negval:u=negval:v=negval" </pre>

Negate luma: <pre class=”example-preformatted”>lutyuv=y=negval </pre>

Remove chroma components, turning the video into a graytone image: <pre class=”example-preformatted”>lutyuv="u=128:v=128" </pre>

Apply a luma burning effect: <pre class=”example-preformatted”>lutyuv="y=2*val" </pre>

Remove green and blue components: <pre class=”example-preformatted”>lutrgb="g=0:b=0" </pre>

Set a constant alpha channel value on input: <pre class=”example-preformatted”>format=rgba,lutrgb=a="maxval-minval/2" </pre>

Correct luma gamma by a factor of 0.5: <pre class=”example-preformatted”>lutyuv=y=gammaval(0.5) </pre>

Discard least significant bits of luma: <pre class=”example-preformatted”>lutyuv=y='bitand(val, 128+64+32)' </pre>

Technicolor like effect: <pre class=”example-preformatted”>lutyuv=u='(val-maxval/2)*2+maxval/2':v='(val-maxval/2)*2+maxval/2' </pre>

 <h3 class=”section”>30.158 lut2, tlut2# TOC</h3>

The <code class=”code”>lut2</code> filter takes two input streams and outputs one stream. The <code class=”code”>tlut2</code> (time lut2) filter takes two consecutive frames from one single stream. This filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class=”option”>c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class=”option”>c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class=”option”>c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class=”option”>d</samp></dt> <dd> set output bit depth, only available for <code class=”code”>lut2</code> filter. By default is 0, which means bit depth is automatically picked from first input format. </dd> </dl>

The <code class=”code”>lut2</code> filter also supports the framesync options. Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in inputs. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dd> The first input value for the pixel component. </dd> <dt><samp class=”option”>y</samp></dt> <dd> The second input

value for the pixel component. </dd> <dt><samp class="option">bdx</samp></dt> <dd> The first input video bit depth. </dd> <dt><samp class="option">bdy</samp></dt> <dd> The second input video bit depth. </dd> </dl>

All expressions default to "x". <h4 class="subsection">30.158.1 Commands# TOC</h4>

This filter supports the all above options as commands except option <code class="code">d</code>. <h4 class="subsection">30.158.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Highlight differences between two RGB video streams: <pre class="example-preformatted">lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1)' </pre>

Highlight differences between two YUV video streams: <pre class="example-preformatted">lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1)' </pre>

Show max difference between two video streams: <pre class="example-preformatted">lut2='if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1)))' </pre>

 <h3 class="section">30.159 maskedclamp# TOC</h3>

Clamp the first input stream with the second input and third input stream. Returns the value of first stream to be between second input stream - <code class="code">undershoot</code> and third input stream + <code class="code">overshoot</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">undershoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">overshoot</samp></dt> <dd> Default value is <code class="code">0</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">30.159.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.160 maskedmax# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is greater than first one or from third input stream otherwise. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">30.160.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.161 maskedmerge# TOC</h3>

Merge the first input stream with the second input stream using per pixel weights in the third input stream. A value of 0 in the third stream pixel component means that pixel component from first stream is returned unchanged, while maximum value (eg. 255 for 8-bit videos) means that pixel component from second stream is returned unchanged. Intermediate values define the amount of merging between both input stream’s pixel components. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">30.161.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.162 maskedmin# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is less than first one or from third input stream otherwise. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class=”subsection”>30.162.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.163 maskedthreshold# TOC</h3>
Pick pixels comparing absolute difference of two video streams with fixed threshold. If absolute difference between pixel component of first and second video stream is equal or lower than user supplied threshold than pixel component from first video stream is picked, otherwise pixel component from second video stream is picked. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold used when picking pixels from absolute difference from two input video streams. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from second stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of filter operation. Can be <code class=”code”>abs</code> or <code class=”code”>diff</code>. Default is <code class=”code”>abs</code>. </dd> </dl>

 <h4 class=”subsection”>30.163.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h3 class="section">30.164 maskfun# TOC</h3> Create mask from input video. For example it is useful to create motion masks after <code class="code">tblend</code> filter. This filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dd> Set low threshold. Any pixel component lower or exact than this value will be set to 0. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high threshold. Any pixel component higher than this value will be set to max value allowed for current pixel format. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all available planes are filtered. </dd> <dt><samp class="option">fill</samp></dt> <dd> Fill all frame pixels with this value. </dd> <dt><samp class="option">sum</samp></dt> <dd> Set max average pixel value for frame. If sum of all pixel components is higher that this average, output frame will be completely filled with value set by <var class="var">fill</var> option. Typically useful for scene changes when used in combination with <code class="code">tblend</code> filter. </dd> </dl>

 <h4 class="subsection">30.164.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.165 mcdeint# TOC</h3>

Apply motion-compensation deinterlacing. It needs one field per frame as input and must thus be used together with yadif=1/3 or equivalent. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set the deinterlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">medium</samp>’</d <dt>‘<samp class="samp">slow</samp>’</dt> <dd> use iterative motion estimation </dd> <dt>‘<samp class="samp">extra_slow</samp>’</d <dd> like ‘<samp class="samp">slow</samp>’, but use multiple reference frames. </dd> </dl> Default value is ‘<samp class="samp">fast</samp>’. </dd> <dt><samp class="option">parity</samp></dt> <dd> Set the picture field parity assumed for the input video. It must be one of the following

values: <dl class="table"> <dt>‘<samp class="samp">0, tff</samp>’</dt> <dd> assume top field first </dd> <dt>‘<samp class="samp">1, bff</samp>’</dt> <dd> assume bottom field first </dd> </dl>

Default value is ‘<samp class="samp">bff</samp>’. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set per-block quantization parameter (QP) used by the internal encoder. Higher values should result in a smoother motion vector field but less optimal individual vectors. Default value is 1. </dd> </dl>

 <h3 class="section">30.166 median# TOC</h3>

Pick median pixel from certain rectangle defined by radius. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set horizontal radius size. Default value is <code class="code">1</code>. Allowed range is integer from 1 to 127. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> <dt><samp class="option">radiusV</samp></dt> <dd> Set vertical radius size. Default value is <code class="code">0</code>. Allowed range is integer from 0 to 127. If it is 0, value will be picked from horizontal <code class="code">radius</code> option. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">30.166.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">30.167 mergeplanes# TOC</h3>

Merge color channel components from several video streams. The filter accepts up to 4 input streams, and merge selected input planes to the output video. This filter accepts the following options: <dl

class=”table”> <dt><samp class=”option”>mapping</samp></dt> <dd> Set input to output plane mapping. Default is <code class=”code”>0</code>. The mappings is specified as a bitmap. It should be specified as a hexadecimal number in the form 0xAa[Bb[Cc[Dd]]]. ’Aa’ describes the mapping for the first plane of the output stream. ’A’ sets the number of the input stream to use (from 0 to 3), and ’a’ the plane number of the corresponding input to use (from 0 to 3). The rest of the mappings is similar, ’Bb’ describes the mapping for the output stream second plane, ’Cc’ describes the mapping for the output stream third plane and ’Dd’ describes the mapping for the output stream fourth plane. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Set output pixel format. Default is <code class=”code”>yuva444p</code>. </dd> <dt><samp class=”option”>map0s</samp></dt> <dt><samp class=”option”>map1s</samp></dt> <dt><samp class=”option”>map2s</samp></dt> <dt><samp class=”option”>map3s</samp></dt> <dd> Set input to output stream mapping for output Nth plane. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>map0p</samp></dt> <dt><samp class=”option”>map1p</samp></dt> <dt><samp class=”option”>map2p</samp></dt> <dt><samp class=”option”>map3p</samp></dt> <dd> Set input to output plane mapping for output Nth plane. Default is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>30.167.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Merge three gray video streams of same width and height into single video stream: <pre class=”example-preformatted”>[a0][a1][a2]mergeplanes=0x001020:yuv444p
</pre>

Merge 1st yuv444p stream and 2nd gray video stream into yuva444p video stream: <pre class=”example-preformatted”>[a0][a1]mergeplanes=0x00010210:yuva444p </pre>

Swap Y and A plane in yuva444p stream:
<pre class=”example-preformatted”>format=yuva444p,mergeplanes=0x03010200:yuva444p
</pre>

Swap U and V plane in yuv420p stream:
<pre class=”example-preformatted”>format=yuv420p,mergeplanes=0x000201:yuv420p
</pre>

Cast a rgb24 clip to yuv444p: <pre class=”example-preformatted”>format=rgb24,mergeplanes=0x000102:yuv444p
</pre>

 <h3 class="section">30.168 mestimate# TOC</h3>

Estimate and export motion vectors using block matching algorithms. Motion vectors are stored in frame side data to be used by other filters. This filter accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> Specify the motion estimation method. Accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class="samp">tss</samp>’</dt> <dd> Three step search algorithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class="samp">ds</samp>’</dt> <dd> Diamond search algorithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt> <dd> Hexagon-based search algorithm. </dd> <dt>‘<samp class="samp">epzs</samp>’</dt> <dd> Enhanced predictive zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt> <dd> Uneven multi-hexagon search algorithm. </dd> </dl> Default value is ‘<samp class="samp">esa</samp>’. </dd> <dt><samp class="option">mb_size</samp></dt> <dd> Macroblock size. Default <code class="code">16</code>. </dd> <dt><samp class="option">search_param</samp></dt> <dd> Search parameter. Default <code class="code">7</code>. </dd> </dl>

 <h3 class="section">30.169 mestimate_d3d12# TOC</h3>

Estimate and export motion vectors using D3D12 hardware-accelerated motion estimation. This filter uses GPU hardware motion estimation capabilities available in DirectX 12 Video APIs to achieve significant performance improvements over the software-based <code class="code">mestimate</code> filter. Motion vectors are stored in frame side data to be used by other filters. This filter requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter uses quarter-pixel precision for motion vector estimation. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mb_size</samp></dt> <dd> Macroblock size. Only <code class="code">8</code> and <code class="code">16</code> are

supported. Default <code class=”code”>16</code>. </dd> </dl>

 <h4 class=”subsection”>30.169.1 Examples# TOC</h4>

Estimate motion vectors using D3D12 hardware acceleration with 16x16 blocks, and visualize them: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf mestimate_d3d12=mb_size=16,hwdownload,format=nv12,codecview=mv=pf \ -c:v libx264 -preset fast -b:v 5M output.mp4 </pre>

 <h3 class=”section”>30.170 midequalizer# TOC</h3>

Apply Midway Image Equalization effect using two video streams. Midway Image Equalization adjusts a pair of images to have the same histogram, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a pair of stereo cameras. This filter has two inputs and one output, which must be of same pixel format, but may be of different sizes. The output of filter is first input adjusted with midway histogram of both inputs. This filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. Default is <code class=”code”>15</code>, which is all available planes. </dd> </dl>

 <h3 class=”section”>30.171 minterpolate# TOC</h3>

Convert the video to specified frame rate using motion interpolation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frame rate. This can be rational e.g. <code class=”code”>60000/1001</code>. Frames are dropped if <var class=”var”>fps</var> is lower than source fps. Default <code class=”code”>60</code>. </dd> <dt><samp class=”option”>mi_mode</samp></dt> <dd> Motion interpolation mode. Following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>dup</samp>’</dt> <dd> Duplicate previous or next frame for interpolating new ones. </dd> <dt>‘<samp class=”samp”>blend</samp>’</dt> <dd> Blend source frames. Interpolated frame is mean of previous and next frames. </dd> <dt>‘<samp class=”samp”>mci</samp>’</dt> <dd> Motion compensated interpolation. Following options are effective when this mode is se-

lected: <dl class=”table”> <dt>‘<samp class=”samp”>mc_mode</samp>’</dt>
<dd> Motion compensation mode. Following values are accepted:
<dl class=”table”> <dt>‘<samp class=”samp”>obmc</samp>’</dt>
<dd> Overlapped block motion compensation. </dd> <dt>‘<samp
class=”samp”>aobmc</samp>’</dt> <dd> Adaptive overlapped
block motion compensation. Window weighting coefficients are controlled
adaptively according to the reliabilities of the neighboring motion vectors to reduce oversmoothing. </dd> </dl> Default mode is ‘<samp
class=”samp”>obmc</samp>’. </dd> <dt>‘<samp
class=”samp”>me_mode</samp>’</dt> <dd> Motion estimation mode. Following values are accepted: <dl class=”table”>
<dt>‘<samp class=”samp”>bidir</samp>’</dt> <dd> Bidirectional
motion estimation. Motion vectors are estimated for each source frame in
both forward and backward directions. </dd> <dt>‘<samp
class=”samp”>bilat</samp>’</dt> <dd> Bilateral motion
estimation. Motion vectors are estimated directly for interpolated frame.
</dd> </dl> Default mode is ‘<samp class=”samp”>bilat</samp>’.
 </dd> <dt>‘<samp class=”samp”>me</samp>’</dt>
<dd> The algorithm to be used for motion estimation. Following values are accepted: <dl class=”table”> <dt>‘<samp
class=”samp”>esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class=”samp”>tss</samp>’</dt>
<dd> Three step search algorithm. </dd> <dt>‘<samp
class=”samp”>tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class=”samp”>ntss</samp>’</dt>
<dd> New three step search algorithm. </dd> <dt>‘<samp
class=”samp”>fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class=”samp”>ds</samp>’</dt>
<dd> Diamond search algorithm. </dd> <dt>‘<samp
class=”samp”>hexbs</samp>’</dt> <dd> Hexagon-based
search algorithm. </dd> <dt>‘<samp class=”samp”>epzs</samp>’</dt>
<dd> Enhanced predictive zonal search algorithm. </dd>
<dt>‘<samp class=”samp”>umh</samp>’</dt> <dd> Uneven
multi-hexagon search algorithm. </dd> </dl> Default algorithm is ‘<samp class=”samp”>epzs</samp>’.
</dd> <dt>‘<samp class=”samp”>mb_size</samp>’</dt>
<dd> Macroblock size. Default <code class=”code”>16</code>.
</dd> <dt>‘<samp class=”samp”>search_param</samp>’</dt>
<dd> Motion estimation search parameter. Default <code class=”code”>32</code>.
 </dd> <dt>‘<samp class=”samp”>vsbmc</samp>’</dt>
<dd> Enable variable-size block motion compensation. Motion estimation
is applied with smaller block sizes at object boundaries in order to make them
less blurry. Default is <code class=”code”>0</code> (disabled). </dd>
</dl> </dd> </dl>

</dd> <dt><samp class=”option”>scd</samp></dt> <dd> Scene

change detection method. Scene change leads motion vectors to be in random direction. Scene change detection replace interpolated frames by duplicate ones. May not be needed for other modes. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable scene change detection. </dd> <dt>‘<samp class="samp">fdiff</samp>’</dt> <dd> Frame difference. Corresponding pixel values are compared and if it satisfies <var class="var">scd_threshold</var> scene change is detected. </dd> </dl> Default method is ‘<samp class="samp">fdiff</samp>’. </dd> <dt><samp class="option">scd_threshold</samp></ <dd> Scene change detection threshold. Default is <code class="code">10.</code>. </dd> </dl>

 <h3 class="section">30.172 mix# TOC</h3>

Mix several video input streams into one video stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input video stream as sequence. Each weight is separated by space. If number of weights is smaller than number of <var class="var">frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> <dt><samp class="option">duration</samp></dt> <dd> Specify how end of stream is determined. <dl class="table"> <dt>‘<samp class="samp">longest</samp>’</dt> <dd> The duration of the longest input. (default) </dd> <dt>‘<samp class="samp">shortest</samp>’</dt> <dd> The duration of the shortest input. </dd> <dt>‘<samp class="samp">first</samp>’</dt> <dd> The duration of the first input. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.172.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">30.173 monochrome# TOC</h3> Convert video to gray using custom color filter. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">cb</samp></dt> <dd> Set the chroma blue spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">cr</samp></dt> <dd> Set the chroma red spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the color filter size. Allowed range is from .1 to 10. Default value is 1. </dd> <dt><samp class="option">high</samp></dt> <dd> Set the highlights strength. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">30.173.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.174 morpho# TOC</h3>

This filter allows to apply main morphological grayscale transforms, erode and dilate with arbitrary structures set in second input stream. Unlike naive implementation and much slower performance in erosion and dilation filters, when speed is critical <code class="code">morpho</code> filter should be used instead. A description of accepted options follows, <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set morphological transform to apply, can be: <dl class="table"> <dt>‘<samp class="samp">erode</samp>’</dt> <dt>‘<samp class="samp">dilate</samp>’</d <dt>‘<samp class="samp">open</samp>’</dt> <dt>‘<samp class="samp">close</samp>’</dt> <dt>‘<samp class="samp">gradient</samp>’< <dt>‘<samp class="samp">tophat</samp>’</dt> <dt>‘<samp class="samp">blackhat</samp>’</dt> </dl> Default is <code class="code">erode</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all planes except alpha are filtered. </dd> <dt><samp class="option">structure</samp></dt> <dd> Set which structure video frames will be processed from second input stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">morpho</code> filter also supports the

framesync options. <h4 class=”subsection”>30.174.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>30.175 mpdecimate# TOC</h3>

Drop frames that do not differ greatly from the previous frame in order to reduce frame rate. The main use of this filter is for very-low-bitrate encoding (e.g. streaming over dialup modem), but it could in theory be used for fixing movies that were inverse-telecined incorrectly. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode. If the value is 0, the filter drops similar frames. If the value is 1, the filter keeps only frames that are one representative of sufficiently long duplicate runs. Default value is 0. </dd> <dt><samp class=”option”>max</samp></dt> <dd> For mode 0: Set the maximum number of consecutive frames which can be dropped (if positive), or the minimum interval between dropped frames (if negative). If the value is 0, the frame is dropped disregarding the number of previous sequentially dropped frames. Default value is 0. </dd> <dt><samp class=”option”>min</samp></dt> <dd> For mode 1: Set the minimum number of frame duplicates to mark the frame as non-unique and keep the first occurrence of it. Default value is 1. </dd> <dt><samp class=”option”>keep</samp></dt> <dd> For mode 0: Set the maximum number of consecutive similar frames to ignore before to start dropping them. If the value is 0, the frame is dropped disregarding the number of previous sequentially similar frames. Default value is 0. </dd> <dt><samp class=”option”>hi</samp></dt> <dt><samp class=”option”>lo</samp></dt> <dt><samp class=”option”>frac</samp></dt> <dd> Set the dropping threshold values. Values for <samp class=”option”>hi</samp> and <samp class=”option”>lo</samp> are for 8x8 pixel blocks and represent actual pixel value differences, so a threshold of 64 corresponds to 1 unit of difference for each pixel, or the same spread out differently over the block. A frame is a candidate for dropping if no 8x8 blocks differ by more than a threshold of <samp class=”option”>hi</samp>, and if no more than <samp class=”option”>frac</samp> blocks (1 meaning the whole image) differ by more than a threshold of <samp class=”option”>lo</samp>. Default value for <samp class=”option”>hi</samp> is 64*12, default value for <samp class=”option”>lo</samp> is 64*5, and default value for <samp class=”option”>frac</samp> is 0.33. </dd> </dl>

 <h3 class="section">30.176 msad# TOC</h3>

Obtain the MSAD (Mean Sum of Absolute Differences) between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max MSAD is printed through the logging system. The filter stores the calculated MSAD of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi msad -f null - </pre>

 <h3 class="section">30.177 multiply# TOC</h3> Multiply first video stream pixels values with second video stream pixels values. The filter accepts the following options: <dl class="table"> <dt><samp class="option">scale</samp></dt> <dd> Set the scale applied to second video stream. By default is <code class="code">1</code>. Allowed range is from <code class="code">0</code> to <code class="code">9</code>. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set the offset applied to second video stream. By default is <code class="code">0.5</code>. Allowed range is from <code class="code">-1</code> to <code class="code">1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Specify planes from input video stream that will be processed. By default all planes are processed. </dd> </dl>

 <h4 class="subsection">30.177.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">30.178 negate# TOC</h3>

Negate (invert) the input video. It accepts the following option: <dl class="table"> <dt><samp class="option">components</samp></dt> <dd> Set components to negate. Available values for compo-

nents are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

</dd> <dt><samp class="option">negate_alpha</samp></dt> <dd> With value 1, it negates the alpha component, if present. Default value is 0. </dd> </dl>

 <h4 class="subsection">30.178.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">30.179 nlmeans# TOC</h3>

Denoise frames using Non-Local Means algorithm. Each pixel is adjusted by looking for other pixels with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>x<samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp>x<samp class="option">r</samp> around the pixel. Note that the research area defines centers for patches, which means some patches will be made of pixels outside that research area. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength. Default is 1.0. Must be in range [1.0, 30.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">pc</samp></dt> <dd> Same as <samp class="option">p</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic. </dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">rc</samp></dt> <dd> Same as <samp class="option">r</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic. </dd> </dl>

 <h3 class="section">30.180 nnedi# TOC</h3>

Deinterlace video using neural network edge directed interpola-

tion. This filter accepts the following options: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dd> Mandatory option, without binary file filter can not work. Currently file can be found here: https://github.com/dubhater/vapoursynth-nnedi3/blob/master/src/nnedi3_weights.bin </dd> <dt><samp class="option">deint</samp></dt> <dd> Set which frames to deinterlace, by default it is <code class="code">all</code>. Can be <code class="code">all</code> or <code class="code">interlaced</code>. </dd> <dt><samp class="option">field</samp></dt> <dd> Set mode of operation. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">af</samp>’</dt> <dd> Use frame flags, both fields. </dd> <dt>‘<samp class="samp">a</samp>’</dt> <dd> Use frame flags, single field. </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> Use top field only. </dd> <dt>‘<samp class="samp">b</samp>’</dt> <dd> Use bottom field only. </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> Use both fields, top first. </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> Use both fields, bottom first. </dd> </dl>

</dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process, by default filter process all frames. </dd> <dt><samp class="option">nsize</samp></dt> <dd> Set size of local neighborhood around each pixel, used by the predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">s8x6</samp>’</dt> <dt>‘<samp class="samp">s16x6</samp>’</dt> <dt>‘<samp class="samp">s32x6</samp>’</d <dt>‘<samp class="samp">s48x6</samp>’</dt> <dt>‘<samp class="samp">s8x4</samp>’</dt> <dt>‘<samp class="samp">s16x4</samp>’</dt <dt>‘<samp class="samp">s32x4</samp>’</dt> </dl>

</dd> <dt><samp class="option">nns</samp></dt> <dd> Set the number of neurons in predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">n16</samp>’</dt> <dt>‘<samp class="samp">n32</samp>’</dt> <dt>‘<samp class="samp">n64</samp>’</dt> <dt>‘<samp class="samp">n128</samp>’</dt> <dt>‘<samp class="samp">n256</samp>’</dt> </dl>

</dd> <dt><samp class="option">qual</samp></dt> <dd> Controls the number of different neural network predictions that are blended together to compute the final output value. Can be <code class="code">fast</code>, default or <code class="code">slow</code>. </dd> <dt><samp class="option">etype</samp></dt> <dd> Set which set of weights to use in the predictor. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">a, abs</samp>’</dt> <dd> weights trained to minimize absolute error </dd> <dt>‘<samp class="samp">s, mse</samp>’</dt> <dd> weights trained

to minimize squared error </dd> </dl>

</dd> <dt><samp class="option">pscrn</samp></dt> <dd> Controls whether or not the prescreener neural network is used to decide which pixels should be processed by the predictor neural network and which can be handled by simple cubic interpolation. The prescreener is trained to know whether cubic interpolation will be sufficient for a pixel or whether it should be predicted by the predictor nn. The computational complexity of the prescreener nn is much less than that of the predictor nn. Since most pixels can be handled by cubic interpolation, using the prescreener generally results in much faster processing. The prescreener is pretty accurate, so the difference between using it and not using it is almost always unnoticeable. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">original</samp>’</
<dt>‘<samp class="samp">new</samp>’</dt> <dt>‘<samp class="samp">new2</samp>’</dt> <dt>‘<samp class="samp">new3</samp>’</dt
</dl>

Default is <code class="code">new</code>. </dd> </dl>

 <h4 class="subsection">30.180.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">weights</var> option. <h3 class="section">30.181 noformat# TOC</h3>

Force libavfilter not to use any of the specified pixel formats for the input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as pix_fmts=yuv420p|monow|rgb24". </dd> </dl>

 <h4 class="subsection">30.181.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Force libavfilter to use a format different from <var class="var">yuv420p</var> for the input to the vflip filter: <pre class="example-preformatted">noformat=pix_fmts=yuv420p,vflip
</pre>

Convert the input video to any of the formats not contained in the list: <pre class="example-preformatted">noformat=yuv420p|yuv444p|yuv410p

</pre>

 <h3 class="section">30.182 noise# TOC</h3>

Add noise on video input frame. The filter accepts the following options: <dl class="table"> <dt><samp class="option">all_seed</samp></dt> <dt><samp class="option">c0_seed</samp></dt> <dt><samp class="option">c1_seed</samp></dt> <dt><samp class="option">c2_seed</samp></dt> <dt><samp class="option">c3_seed</samp></dt> <dd> Set noise seed for specific pixel component or all pixel components in case of <var class="var">all_seed</var>. Default value is <code class="code">123457</code>. </dd> <dt><samp class="option">all_strength, alls</samp></dt> <dt><samp class="option">c0_strength, c0s</samp></dt> <dt><samp class="option">c1_strength, c1s</samp></dt> <dt><samp class="option">c2_strength, c2s</samp></dt> <dt><samp class="option">c3_strength, c3s</samp></dt> <dd> Set noise strength for specific pixel component or all pixel components in case <var class="var">all_strength</var>. Default value is <code class="code">0</code>. Allowed range is [0, 100]. </dd> <dt><samp class="option">all_flags, allf</samp></dt> <dt><samp class="option">c0_flags, c0f</samp></dt> <dt><samp class="option">c1_flags, c1f</samp></dt> <dt><samp class="option">c2_flags, c2f</samp></dt> <dt><samp class="option">c3_flags, c3f</samp></dt> <dd> Set pixel component flags or set flags for all components if <var class="var">all_flags</var>. Available values for component flags are: <dl class="table"> <dt>‘<samp class="samp">a</samp>’</dt> <dd> averaged temporal noise (smoother) </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> mix random noise with a (semi)regular pattern </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> temporal noise (noise pattern changes between frames) </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> uniform noise (gaussian otherwise) </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.182.1 Examples# TOC</h4>

Add temporal and uniform noise to input video: <pre class="example-preformatted">noise=alls=20:allf=t+u </pre>

 <h3 class="section">30.183 normalize# TOC</h3>

Normalize RGB video (aka histogram stretching, contrast stretching).

See: https://en.wikipedia.org/wiki/Normalization_(image_processing) For each channel of each frame, the filter computes the input range and maps it linearly to the user-specified output range. The output range defaults to the full dynamic range from pure black to pure white. Temporal smoothing can be used on the input range to reduce flickering (rapid changes in brightness) caused when small dark or bright objects enter or leave the scene. This is similar to the auto-exposure (automatic gain control) on a video camera, and, like a video camera, it may cause a period of over- or under-exposure of the video. The R,G,B channels can be normalized independently, which may cause some color shifting, or linked together as a single channel, which prevents color shifting. Linked normalization preserves hue. Independent normalization does not, so it can be used to remove some color casts. Independent and linked normalization can be combined in any ratio. The normalize filter accepts the following options: <dl class="table"> <dt><samp class="option">blackpt</samp></dt> <dt><samp class="option">whitept</samp></dt> <dd> Colors which define the output range. The minimum input value is mapped to the <var class="var">blackpt</var>. The maximum input value is mapped to the <var class="var">whitept</var>. The defaults are black and white respectively. Specifying white for <var class="var">blackpt</var> and black for <var class="var">whitept</var> will give color-inverted, normalized video. Shades of grey can be used to reduce the dynamic range (contrast). Specifying saturated colors here can create some interesting effects. </dd> <dt><samp class="option">smoothing</samp></dt> <dd> The number of previous frames to use for temporal smoothing. The input range of each channel is smoothed using a rolling average over the current frame and the <var class="var">smoothing</var> previous frames. The default is 0 (no temporal smoothing). </dd> <dt><samp class="option">independence</samp></dt> <dd> Controls the ratio of independent (color shifting) channel normalization to linked (color preserving) normalization. 0.0 is fully linked, 1.0 is fully independent. Defaults to 1.0 (fully independent). </dd> <dt><samp class="option">strength</samp></dt> <dd> Overall strength of the filter. 1.0 is full strength. 0.0 is a rather expensive no-op. Defaults to 1.0 (full strength). </dd> </dl>

 <h4 class="subsection">30.183.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">smoothing</var> option. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">30.183.2 Examples<a class="anchor hidden-xs" href="#Examples-

122" aria-hidden="true"># TOC</h4>

Stretch video contrast to use the full dynamic range, with no temporal smoothing; may flicker depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=
</pre>

As above, but with 50 frames of temporal smoothing; flicker should be reduced, depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50
</pre>

As above, but with hue-preserving linked channel normalization: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:
</pre>

As above, but with half strength: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50:independence=0:stre
</pre>

Map the darkest input color to red, the brightest input color to cyan: <pre class="example-preformatted">normalize=blackpt=red:whitept=cyan
</pre>

 <h3 class="section">30.184 null#
TOC</h3>

Pass the video source unchanged to the output.
<h3 class="section">30.185 ocio# TOC</h3>
OpenColorIO library filter This filter allows you to do color management using the OpenColorIO library. See https://opencolorio.org/ for more details. To Enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libopencolorio</code>.
It accepts the following options: <dl class="table"> <dt><samp class="option">config</samp></dt> <dd> By default the filter will use the OCIO config defined by the OCIO environment variable, but this parameter allows you to explicitly specify its location. If you are getting started, you can use config=ocio://studio-config-v1.0.0_aces-v1.3_ocio-v2.1 which specifies one of the built in defaults. </dd> <dt><samp class="option">input</samp></dt> <dd> Set the input colorspace. </dd> <dt><samp class="option">output</samp></dt> <dd> Set the output colorspace. </dd> <dt><samp class="option">display</samp></dt>
<dd> Set the display colorspace, used in combination with view. </dd> <dt><samp class="option">view</samp></dt> <dd> Set the view colorspace, used in combination with display. </dd> <dt><samp

class=”option”>inverse</samp></dt> <dd> When used in combination with display and view, this inverts the transform, so going from a display/view to the "input colorspace". </dd> <dt><samp class=”option”>filetransform</samp></dt> <dd> Allows you to specify an external file-transform to use instead of the OCIO config file. This is useful for applying a single transform without needing a full OCIO config file. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Allow you to specify the output pix_fmt of the OCIO filter. This *has* to be a RGB colorspace, so you are limited to rgb24, rgba, rgb48, rgba64, gbrp10, gbrap10, gbrp12, gbrap12, gbrpf32 and gbrapf32; for most encoding we would recommend rgb48. Half-float (gbrpf16, gbrapf16) is accepted as input but is not a valid output format, because swscale has no half-float write support and its legacy half-float read path clamps values to [0,1]. When this option is not set, the output format defaults to the input format, except that half-float input is promoted to the matching float format. Where negotiation cannot yet tell whether the input will be half-float, the output falls back to float, so a float output may be selected even when the input turns out to be an integer format. </dd> <dt><samp class=”option”>context_params</samp></dt> <dd> Allow you to specify additional context parameters for the OCIO filter. This is a list of key=value pairs, separated by colons. </dd> </dl>

 <h4 class=”subsection”>30.185.1 Examples# TOC</h4>

Map from ACEScg to ACEScct, this assumes the OCIO file is defined with the OCIO environment variable. <pre class=”example-preformatted”>input=ACEScg:output=ACEScct:format=rgb48 </pre>

Map from ACEScg to a sRGB display using the "ACES 1.0 - SDR Video" view transform, this assumes the OCIO file is defined with the OCIO environment variable. Note you will need to wrap the argument in quotes to ensure that the spaces are interpreted correctly. <pre class=”example-preformatted”>input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

As above but using the OCIO file studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio rather than the OCIO environment variable. <pre class=”example-preformatted”>config=studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio:input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

If you are converting to YCrCb you still will want to set the color matrix for the conversion. This is a good example of combining the two. <div

class=”example”> <pre class=”example-preformatted”>ffmpeg -y -i SOURCE-FRAMES.%05d.exr -c:v libx265 -vf "ocio=input=ACEScg:output=ACEScct:format=rgb48,scale=in_col OUTPUTFILE.mov </pre>

 <h3 class=”section”>30.186 ocr# TOC</h3> Optical Character Recognition This filter uses Tesseract for optical character recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libtesseract</code>. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>datapath</samp></dt> <dd> Set datapath to tesseract data. Default is to use whatever was set at installation. </dd> <dt><samp class=”option”>language</samp></dt> <dd> Set language, default is "eng". </dd> <dt><samp class=”option”>whitelist</samp></dt> <dd> Set character whitelist. </dd> <dt><samp class=”option”>blacklist</samp></dt> <dd> Set character blacklist. </dd> </dl>

The filter exports recognized text as the frame metadata <code class=”code”>lavfi.ocr.text</code>. The filter exports confidence of recognized words as the frame metadata <code class=”code”>lavfi.ocr.confidence</code>. <h3 class=”section”>30.187 ocv# TOC</h3>

Apply a video transform using libopencv. To enable this filter, install the libopencv library and headers and configure FFmpeg with <code class=”code”>--enable-libopencv</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the libopencv filter to apply. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> The parameters to pass to the libopencv filter. If not specified, the default values are assumed. </dd> </dl>

Refer to the official libopencv documentation for more precise information: <a class=”url” href=”http://docs.opencv.org/master/modules/imgproc/doc/filtering.html”>http://docs.open Several libopencv filters are supported; see the following subsections. <h4 class=”subsection”>30.187.1 dilate# TOC</h4>

Dilate an image by using a specific structuring element. It corresponds to the libopencv function <code class=”code”>cvDilate</code>. It accepts the parameters: <var class=”var”>struct_el</var>|<var class=”var”>nb_iterations</var>. <var class=”var”>struct_el</var> represents a structuring element, and has the syntax: <var class=”var”>cols</var>x<var

class=”var”>rows</var>+<var class=”var”>anchor_x</var>x<var class=”var”>anchor_y</var>/<var class=”var”>shape</var> <var class=”var”>cols</var> and <var class=”var”>rows</var> represent the number of columns and rows of the structuring element, <var class=”var”>anchor_x</var> and <var class=”var”>anchor_y</var> the anchor point, and <var class=”var”>shape</var> the shape for the structuring element. <var class=”var”>shape</var> must be "rect", "cross", "ellipse", or "custom". If the value for <var class=”var”>shape</var> is "custom", it must be followed by a string of the form "=<var class=”var”>filename</var>". The file with name <var class=”var”>filename</var> is assumed to represent a binary image, with each printable character corresponding to a bright pixel. When a custom <var class=”var”>shape</var> is used, <var class=”var”>cols</var> and <var class=”var”>rows</var> are ignored, the number or columns and rows of the read file are assumed instead. The default value for <var class=”var”>struct_el</var> is "3x3+0x0/rect". <var class=”var”>nb_iterations</var> specifies the number of times the transform is applied to the image, and defaults to 1. Some examples: <pre class=”example-preformatted”># Use the default values ocv=dilate

Dilate using a structuring element with a 5x5 cross, iterating two times ocv=filter_name=dilate:filter_params=5x5+2x2/cross|2

Read the shape from the file diamond.shape, iterating two times. # The file diamond.shape may contain a pattern of characters like this # * # *** # ***** # *** # * # The specified columns and rows are ignored # but the anchor point coordinates are not ocv=dilate:0x0+2x2/custom=diamond.shape|2 </pre>

 <h4 class=”subsection”>30.187.2 erode# TOC</h4>

Erode an image by using a specific structuring element. It corresponds to the libopencv function <code class=”code”>cvErode</code>. It accepts the parameters: <var class=”var”>struct_el</var>:<var class=”var”>nb_iterations</var>, with the same syntax and semantics as the dilate filter. <h4 class=”subsection”>30.187.3 smooth# TOC</h4>

Smooth the input video. The filter takes the following parameters: <var class=”var”>type</var>|<var class=”var”>param1</var>|<var class=”var”>param2</var>|<var class=”var”>param3</var>|<var class=”var”>param4</var>.

 <var class="var">type</var> is the type of smooth filter to apply, and must be one of the following values: "blur", "blur_no_scale", "median", "gaussian", or "bilateral". The default value is "gaussian". The meaning of <var class="var">param1</var>, <var class="var">param2</var>, <var class="var">param3</var>, and <var class="var">param4</var> depends on the smooth type. <var class="var">param1</var> and <var class="var">param2</var> accept integer positive values or 0. <var class="var">param3</var> and <var class="var">param4</var> accept floating point values. The default value for <var class="var">param1</var> is 3. The default value for the other parameters is 0. These parameters correspond to the parameters assigned to the libopencv function <code class="code">cvSmooth</code>. <h3 class="section">30.188 oscilloscope# TOC</h3>

2D Video Oscilloscope. Useful to measure spatial impulse, step responses, chroma delays, etc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set scope center x position. </dd> <dt><samp class="option">y</samp></dt> <dd> Set scope center y position. </dd> <dt><samp class="option">s</samp></dt> <dd> Set scope size, relative to frame diagonal. </dd> <dt><samp class="option">t</samp></dt> <dd> Set scope tilt/rotation. </dd> <dt><samp class="option">o</samp></dt> <dd> Set trace opacity. </dd> <dt><samp class="option">tx</samp></dt> <dd> Set trace center x position. </dd> <dt><samp class="option">ty</samp></dt> <dd> Set trace center y position. </dd> <dt><samp class="option">tw</samp></dt> <dd> Set trace width, relative to width of frame. </dd> <dt><samp class="option">th</samp></dt> <dd> Set trace height, relative to height of frame. </dd> <dt><samp class="option">c</samp></dt> <dd> Set which components to trace. By default it traces first three components. </dd> <dt><samp class="option">g</samp></dt> <dd> Draw trace grid. By default is enabled. </dd> <dt><samp class="option">st</samp></dt> <dd> Draw some statistics. By default is enabled. </dd> <dt><samp class="option">sc</samp></dt> <dd> Draw scope. By default is enabled. </dd> </dl>

 <h4 class="subsection">30.188.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the correspond-

ing option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">30.188.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Inspect full first row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=0:s=1 </pre>

Inspect full last row of video frame. <pre class="example-preformatted">oscilloscope=x=0.5:y=1:s=1 </pre>

Inspect full 5th line of video frame of height 1080. <pre class="example-preformatted">oscilloscope=x=0.5:y=5/1080:s=1 </pre>

Inspect full last column of video frame. <pre class="example-preformatted">oscilloscope=x=1:y=0.5:s=1:t=1 </pre>

 <h3 class="section">30.189 overlay# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: A description of the accepted options follows. <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set the expression for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. In case the expression is invalid, it is set to a huge value (meaning that the overlay will not be displayed within the output visible area). </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for <samp class="option">x</samp>, and <samp class="option">y</samp> are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">frame</samp>’.

 </dd> <dt><samp class="option">shortest</samp></dt>
<dd> See framesync.
</dd> <dt><samp class="option">format</samp></dt> <dd> Set
the format for the output video. It accepts the following values:
<dl class="table"> <dt>‘<samp class="samp">yuv420</samp>’</dt>
<dd> force YUV 4:2:0 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv420p10</samp>’</dt> <dd> force YUV 4:2:0
10-bit planar output </dd> <dt>‘<samp class="samp">yuv422</samp>’</dt>
<dd> force YUV 4:2:2 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv422p10</samp>’</dt> <dd> force YUV 4:2:2
10-bit planar output </dd> <dt>‘<samp class="samp">yuv444</samp>’</dt>
<dd> force YUV 4:4:4 8-bit planar output </dd> <dt>‘<samp
class="samp">yuv444p10</samp>’</dt> <dd> force YUV 4:4:4
10-bit planar output </dd> <dt>‘<samp class="samp">rgb</samp>’</dt>
<dd> force RGB 8-bit packed output </dd> <dt>‘<samp
class="samp">gbrp</samp>’</dt> <dd> force RGB 8-bit planar
output </dd> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> automatically pick format </dd> </dl>

Default value is ‘<samp class="samp">yuv420</samp>’.
 </dd> <dt><samp class="option">repeatlast</samp></dt>
<dd> See framesync.
</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set for-
mat of alpha of the overlaid video, it can be <var class="var">straight</var>
or <var class="var">premultiplied</var>, or <var class="var">auto</var>
to choose the alpha mode automatically. Default is <var class="var">auto</var>.
</dd> </dl>

The <samp class="option">x</samp>, and <samp class="option">y</samp>
expressions can contain the following parameters. <dl class="table">
<dt><samp class="option">main_w, W</samp></dt> <dt><samp
class="option">main_h, H</samp></dt> <dd> The main input
width and height. </dd> <dt><samp class="option">overlay_w,
w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt>
<dd> The overlay input width and height. </dd> <dt><samp
class="option">x</samp></dt> <dt><samp class="option">y</samp></dt>
<dd> The computed values for <var class="var">x</var> and <var
class="var">y</var>. They are evaluated for each new frame.
</dd> <dt><samp class="option">hsub</samp></dt> <dt><samp
class="option">vsub</samp></dt> <dd> horizontal and vertical
chroma subsample values of the output format. For example for the
pixel format "yuv422p" <var class="var">hsub</var> is 2
and <var class="var">vsub</var> is 1. </dd> <dt><samp
class="option">n</samp></dt> <dd> the number of input frame, start-
ing from 0 </dd> <dt><samp class="option">pos</samp></dt>
<dd> the position in the file of the input frame, NAN if unknown; depre-
cated, do not use </dd> <dt><samp class="option">t</samp></dt>

<dd> The timestamp, expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

This filter also supports the framesync options. Note that the <var class="var">n</var>, <var class="var">t</var> variables are available only when evaluation is done per frame, and will evaluate to NAN when <samp class="option">eval</samp> is set to ‘<samp class="samp">init</samp>’. Be aware that frames are taken from each input video in timestamp order, hence, if their initial timestamps differ, it is a good idea to pass the two inputs through a <var class="var">setpts=PTS-STARTPTS</var> filter to have them begin in the same zero timestamp, as the example for the <var class="var">movie</var> filter does. You can chain together more overlays but you should test the efficiency of such approach. <h4 class="subsection">30.189.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Modify the x and y of the overlay input. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class="subsection">30.189.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the overlay at 10 pixels from the bottom right corner of the main video: <pre class="example-preformatted">overlay=main_w-overlay_w-10:main_h-overlay_h-10 </pre>

Using named options the example above becomes: <pre class="example-preformatted">overlay=x=main_w-overlay_w-10:y=main_h-overlay_h-10 </pre>

Insert a transparent PNG logo in the bottom left corner of the input, using the <code class="command">ffmpeg</code> tool with the <code class="code">-filter_complex</code> option: <pre class="example-preformatted">ffmpeg -i input -i logo -filter_complex 'overlay=10:main_h-overlay_h-10' output </pre>

Insert 2 different transparent PNG logos (second logo on bottom right corner) using the <code class="command">ffmpeg</code> tool: <pre class="example-preformatted">ffmpeg -i input -i

logo1 -i logo2 -filter_complex 'overlay=x=10:y=H-h-10,overlay=x=W-w-10:y=H-h-10' output </pre>

Add a transparent color layer on top of the main video; <code class="code">WxH</code> must specify the size of the main input to the overlay filter: <pre class="example-preformatted">color=color=red@.3:size=WxH [over]; [in][over] overlay [out] </pre>

Play an original video and a filtered version (here with the deshake filter) side by side using the <code class="command">ffplay</code> tool: <pre class="example-preformatted">ffplay input.avi -vf 'split[a][b]; [a]pad=iw*2:ih[src]; [b]deshake[filt]; [src][filt]overlay=w' </pre>

The above command is the same as: <pre class="example-preformatted">ffplay input.avi -vf 'split[b], pad=iw*2[src], [b]deshake, [src]overlay=w' </pre>

Make a sliding overlay appearing from the left to the right top part of the screen starting since time 2: <pre class="example-preformatted">overlay=x='if(gte(t,2), -w+(t-2)*20, NAN)':y=0 </pre>

Compose output by putting two input videos side to side: <pre class="example-preformatted">ffmpeg -i left.avi -i right.avi -filter_complex " nullsrc=size=200x100 [background]; [0:v] setpts=PTS-STARTPTS, scale=100x100 [left]; [1:v] setpts=PTS-STARTPTS, scale=100x100 [right]; [background][left] overlay=shortest=1 [background+left]; [background+left][right] overlay=shortest=1:x=100 [left+right] " </pre>

Mask 10-20 seconds of a video by applying the delogo filter to a section <pre class="example-preformatted">ffmpeg -i test.avi -codec:v:0 wmv2 -ar 11025 -b:v 9000k -vf '[in]split[split_main][split_delogo];[split_delogo]trim=start=360:end=
masked.avi </pre>

Chain several overlays in cascade: <pre class="example-preformatted">nullsrc=s=200x200 [bg]; testsrc=s=100x100, split=4 [in0][in1][in2][in3]; [in0] lutrgb=r=0, [bg] overlay=0:0 [mid0]; [in1] lutrgb=g=0, [mid0] overlay=100:0 [mid1]; [in2] lutrgb=b=0, [mid1] overlay=0:100 [mid2]; [in3] null, [mid2] overlay=100:100 [out0] </pre>

 <h3 class="section">30.190 owdenoise# TOC</h3>

Apply Overcomplete Wavelet denoiser. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>depth</samp></dt> <dd> Set depth. Larger depth values will denoise lower frequency components more, but slow down filtering. Must be an int in the range 8-16, default is <code class=”code”>8</code>. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma strength. Must be a double value in the range 0-1000, default is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>chroma_strength, cs</samp></dt> <dd> Set chroma strength. Must be a double value in the range 0-1000, default is <code class=”code”>1.0</code>. </dd> </dl>

 <h3 class=”section”>30.191 pad# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class=”var”>x</var>, <var class=”var”>y</var> coordinates. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. The default value of <var class=”var”>color</var> is "black". </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Specify when to evaluate <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var> and <var class=”var”>y</var> expression. It accepts the following values: <dl class=”table”>

<dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offsets as specified by the <var class="var">x</var> and <var class="var">y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">30.191.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add paddings with the color "violet" to the input video. The output video size is 640x480, and the top-left corner of the input video is placed at column 0, row 40 <pre class="example-preformatted">pad=640:480:0:40:violet </pre>

The example above is equivalent to the following command: <pre class=”example-preformatted”>pad=width=640:height=480:x=0:y=40:color=violet </pre>

Pad the input to get an output with dimensions increased by 3/2, and put the input video at the center of the padded area: <pre class=”example-preformatted”>pad="3/2*iw:3/2*ih:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a squared output with size equal to the maximum value between the input width and height, and put the input video at the center of the padded area: <pre class=”example-preformatted”>pad="max(iw\,ih):ow:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a final w/h ratio of 16:9: <pre class=”example-preformatted”>pad="ih*16/9:ih:(ow-iw)/2:(oh-ih)/2" </pre>

In case of anamorphic video, in order to set the output display aspect correctly, it is necessary to use <var class=”var”>sar</var> in the expression, according to the relation: <pre class=”example-preformatted”>(ih * X / ih) * sar = output_dar X = output_dar / sar </pre>

Thus the previous example needs to be modified to: <pre class=”example-preformatted”>pad="ih*16/9/sar:ih:(ow-iw)/2:(oh-ih)/2" </pre>

Double the output size and put the input video in the bottom-right corner of the output padded area: <pre class=”example-preformatted”>pad="2*iw:2*ih:ow-iw:oh-ih" </pre>

 <h3 class=”section”>30.192 palettegen# TOC</h3>

Generate one palette for a whole video stream. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>max_colors</samp></dt> <dd> Set the maximum number of colors to quantize in the palette. Note: the palette will still contain 256 colors; the unused palette entries will be black. </dd> <dt><samp class=”option”>reserve_transparent</samp></dt> <dd> Create a palette of 255 colors maximum and reserve the last one for transparency. Reserving the transparency color is useful for GIF optimization. If not set, the maximum of colors in the palette will be 256. You probably want to disable this option for a standalone image. Set by default. </dd> <dt><samp class=”option”>transparency_color</samp></dt> <dd> Set the color

that will be used as background for transparency. </dd> <dt><samp class="option">stats_mode</samp></dt> <dd> Set statistics mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Compute full frame histograms. </dd> <dt>‘<samp class="samp">diff</samp>’</dt> <dd> Compute histograms only for the part that differs from previous frame. This might be relevant to give more importance to the moving part of your input if the background is static. </dd> <dt>‘<samp class="samp">single</samp>’</dt> <dd> Compute new histogram for each frame. </dd> </dl>

Default value is <var class="var">full</var>. </dd> </dl>

The filter also exports the frame metadata <code class="code">lavfi.color_quant_ratio</code> (<code class="code">nb_color_in / nb_color_out</code>) which you can use to evaluate the degree of color quantization of the palette. This information is also visible at <var class="var">info</var> logging level. <h4 class="subsection">30.192.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a representative palette of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -vf palettegen palette.png </pre>

 <h3 class="section">30.193 paletteuse# TOC</h3>

Use a palette to downsample an input video stream. The filter takes two inputs: one video stream and a palette. The palette must be a 256 pixels image. It accepts the following options: <dl class="table"> <dt><samp class="option">dither</samp></dt> <dd> Select dithering mode. Available algorithms are: <dl class="table"> <dt>‘<samp class="samp">bayer</samp>’</dt> <dd> Ordered 8x8 bayer dithering (deterministic) </dd> <dt>‘<samp class="samp">heckbert</samp>’</dt> <dd> Dithering as defined by Paul Heckbert in 1982 (simple error diffusion). Note: this dithering is sometimes considered "wrong" and is included as a reference. </dd> <dt>‘<samp class="samp">floyd_steinberg</samp>’</dt> <dd> Floyd and Steingberg dithering (error diffusion) </dd> <dt>‘<samp class="samp">sierra2</samp>’</dt> <dd> Frankie Sierra dithering v2 (error diffusion) </dd> <dt>‘<samp class="samp">sierra2_4a</samp>’</dt> <dd> Frankie Sierra dithering v2 "Lite" (error diffusion) </dd>

<dt>‘<samp class="samp">sierra3</samp>’</dt> <dd> Frankie Sierra dithering v3 (error diffusion) </dd> <dt>‘<samp class="samp">burkes</samp>’</dt> <dd> Burkes dithering (error diffusion) </dd> <dt>‘<samp class="samp">atkinson</samp>’</dt> <dd> Atkinson dithering by Bill Atkinson at Apple Computer (error diffusion) </dd> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable dithering. </dd> </dl>

Default is <var class="var">sierra2_4a</var>. </dd> <dt><samp class="option">bayer_scale</samp></dt> <dd> When <var class="var">bayer</var> dithering is selected, this option defines the scale of the pattern (how much the crosshatch pattern is visible). A low value means more visible pattern for less banding, and higher value means less visible pattern at the cost of more banding. The option must be an integer value in the range [0,5]. Default is <var class="var">2</var>. </dd> <dt><samp class="option">diff_mode</samp></dt> <dd> If set, define the zone to process <dl class="table"> <dt>‘<samp class="samp">rectangle</samp>’</dt> <dd> Only the changing rectangle will be reprocessed. This is similar to GIF cropping/offsetting compression mechanism. This option can be useful for speed if only a part of the image is changing, and has use cases such as limiting the scope of the error diffusal <samp class="option">dither</samp> to the rectangle that bounds the moving scene (it leads to more deterministic output if the scene doesn’t change much, and as a result less moving noise and better GIF compression). </dd> </dl>

Default is <var class="var">none</var>. </dd> <dt><samp class="option">new</samp></dt> <dd> Take new palette for each output frame. </dd> <dt><samp class="option">alpha_threshold</samp></dt> <dd> Sets the alpha threshold for transparency. Alpha values above this threshold will be treated as completely opaque, and values below this threshold will be treated as completely transparent. The option must be an integer value in the range [0,255]. Default is <var class="var">128</var>. </dd> </dl>

 <h4 class="subsection">30.193.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use a palette (generated for example with palettegen) to encode a GIF using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -i palette.png -lavfi paletteuse output.gif </pre>

 <h3 class="section">30.194 perspective<a class="anchor hidden-xs" href="#perspective" aria-

hidden=”true”># TOC</h3>

Correct perspective of video not recorded perpendicular to the screen. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dt><samp class=”option”>x1</samp></dt> <dt><samp class=”option”>y1</samp></dt> <dt><samp class=”option”>x2</samp></dt> <dt><samp class=”option”>y2</samp></dt> <dt><samp class=”option”>x3</samp></dt> <dt><samp class=”option”>y3</samp></dt> <dd> Set coordinates expression for top left, top right, bottom left and bottom right corners. Default values are <code class=”code”>0:0:W:0:0:H:W:H</code> with which perspective will remain unchanged. If the <code class=”code”>sense</code> option is set to <code class=”code”>source</code>, then the specified points will be sent to the corners of the destination. If the <code class=”code”>sense</code> option is set to <code class=”code”>destination</code>, then the corners of the source will be sent to the specified coordinates. The expressions can use the following variables: <dl class=”table”> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> the width and height of video frame. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> </dl>

</dd> <dt><samp class=”option”>interpolation</samp></dt> <dd> Set interpolation for perspective correction. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>cubic</samp>’</dt> </dl>

Default value is ‘<samp class=”samp”>linear</samp>’. </dd> <dt><samp class=”option”>sense</samp></dt> <dd> Set interpretation of coordinate options. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>0, source</samp>’</dt> <dd> Send point in the source specified by the given coordinates to the corners of the destination. </dd> <dt>‘<samp class=”samp”>1, destination</samp>’</dt> <dd> Send the corners of the source to the point in the destination specified by the given coordinates. Default value is ‘<samp class=”samp”>source</samp>’. </dd> </dl>

</dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for coordinates <samp class=”option”>x0,y0,...x3,y3</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.
</dd> </dl>

 <h3 class="section">30.195 phase<span class="pull-
right">#
TOC</h3>

Delay interlaced video by one field time so that the field order changes.
 The intended use is to fix PAL movies that have been captured
with the opposite field order to the film-to-video transfer. A
description of the accepted parameters follows. <dl class="table">
<dt><samp class="option">mode</samp></dt> <dd> Set phase
mode. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">t</samp>’</dt> <dd> Capture
field order top-first, transfer bottom-first. Filter will delay the bottom field.
 </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> Capture field order bottom-first, transfer top-first. Filter will delay
the top field. </dd> <dt>‘<samp class="samp">p</samp>’</dt>
<dd> Capture and transfer with the same field order. This mode only
exists for the documentation of the other options to refer to, but if you actually
select it, the filter will faithfully do nothing. </dd> <dt>‘<samp
class="samp">a</samp>’</dt> <dd> Capture field or-
der determined automatically by field flags, transfer opposite. Filter
selects among ‘<samp class="samp">t</samp>’ and
‘<samp class="samp">b</samp>’ modes on a frame by
frame basis using field flags. If no field information is available, then
this works just like ‘<samp class="samp">u</samp>’.
 </dd> <dt>‘<samp class="samp">u</samp>’</dt>
<dd> Capture unknown or varying, transfer opposite. Filter selects
among ‘<samp class="samp">t</samp>’ and ‘<samp
class="samp">b</samp>’ on a frame by frame basis by analyzing the
images and selecting the alternative that produces best match between the fields.
 </dd> <dt>‘<samp class="samp">T</samp>’</dt>
<dd> Capture top-first, transfer unknown or varying. Filter selects
among ‘<samp class="samp">t</samp>’ and ‘<samp
class="samp">p</samp>’ using image analysis. </dd>
<dt>‘<samp class="samp">B</samp>’</dt> <dd> Capture
bottom-first, transfer unknown or varying. Filter selects among ‘<samp
class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’
using image analysis. </dd> <dt>‘<samp class="samp">A</samp>’</dt>
<dd> Capture determined by field flags, transfer unknown or varying.
Filter selects among ‘<samp class="samp">t</samp>’,
‘<samp class="samp">b</samp>’ and ‘<samp
class="samp">p</samp>’ using field flags and image analysis. If
no field information is available, then this works just like ‘<samp
class="samp">U</samp>’. This is the default mode.
</dd> <dt>‘<samp class="samp">U</samp>’</dt>

<dd> Both capture and transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’, ‘<samp class="samp">b</samp>’ and ‘<samp class="samp">p</samp>’ using image analysis only. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.195.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.196 photosensitivity# TOC</h3> Reduce various flashes in video, so to help users with epilepsy. It accepts the following options: <dl class="table"> <dt><samp class="option">frames, f</samp></dt> <dd> Set how many frames to use when filtering. Default is 30. </dd> <dt><samp class="option">threshold, t</samp></dt> <dd> Set detection threshold factor. Default is 1. Lower is stricter. </dd> <dt><samp class="option">skip</samp></dt> <dd> Set how many pixels to skip when sampling frames. Default is 1. Allowed range is from 1 to 1024. </dd> <dt><samp class="option">bypass</samp></dt> <dd> Leave frames unchanged. Default is disabled. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the blending factor for frames that exceed the detection threshold. A value of 0 duplicates the last accepted frame, and a value of 1 blends up to the detection threshold. Default is 1. Allowed range is from 0 to 1. </dd> </dl>

 <h3 class="section">30.197 pixdesctest# TOC</h3>

Pixel format descriptor test filter, mainly useful for internal testing. The output video should be equal to the input video. For example: <pre class="example-preformatted">format=monow, pixdesctest </pre>

can be used to test the monowhite pixel format descriptor definition. <h3 class="section">30.198 pixelize# TOC</h3> Apply pixelization to video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set block

dimensions that will be used for pixelization. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set the mode of pixelization used. Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dt>‘<samp class=”samp”>min</samp>’</dt> <dt>‘<samp class=”samp”>max</samp>’</dt> </dl> Default value is <code class=”code”>avg</code>. </dd> <dt><samp class=”option”>planes, p</samp></dt> <dd> Set what planes to filter. Default is to filter all planes. </dd> </dl>

 <h4 class=”subsection”>30.198.1 Commands# TOC</h4>

This filter supports all options as commands. <h3 class=”section”>30.199 pixscope# TOC</h3>

Display sample values of color channels. Mainly useful for checking color and levels. Minimum supported resolution is 640x480. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set scope X position, relative offset on X axis. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set scope Y position, relative offset on Y axis. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set scope width. </dd> <dt><samp class=”option”>h</samp></dt> <dd> Set scope height. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Set window opacity. This window also holds statistics about pixel area. </dd> <dt><samp class=”option”>wx</samp></dt> <dd> Set window X position, relative offset on X axis. </dd> <dt><samp class=”option”>wy</samp></dt> <dd> Set window Y position, relative offset on Y axis. </dd> </dl>

 <h4 class=”subsection”>30.199.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>30.200 pp7# TOC</h3> Apply Postprocessing filter 7. It is variant of the spp filter, similar to spp = 6 with 7 point DCT, where only the center sample is used after IDCT.

 The filter accepts the following options: <dl class="table"> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0 to 63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Set medium thresholding (good results, default). </dd> </dl> </dd> </dl>

 <h3 class="section">30.201 premultiply# TOC</h3> Apply alpha premultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">30.202 premultiply_dynamic# TOC</h3> Dynamically premultiply or unpremultiply the input video stream as needed, to match the requirements of the downstream filter graph. This is roughly equivalent to either <code class="code">premultiply:inplace=yes</code> or <code class="code">unpremultiply:inplace=yes</code>, or otherwise a noop. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> </dl>

 <h3 class="section">30.203 prewitt# TOC</h3> Apply prewitt operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be

copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class=”subsection”>30.203.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.204 pseudocolor# TOC</h3>

Alter frame colors in video with pseudocolors. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>c0</samp></dt> <dd> set pixel first component expression </dd> <dt><samp class=”option”>c1</samp></dt> <dd> set pixel second component expression </dd> <dt><samp class=”option”>c2</samp></dt> <dd> set pixel third component expression </dd> <dt><samp class=”option”>c3</samp></dt> <dd> set pixel fourth component expression, corresponds to the alpha component </dd> <dt><samp class=”option”>index, i</samp></dt> <dd> set component to use as base for altering colors </dd> <dt><samp class=”option”>preset, p</samp></dt> <dd> Pick one of built-in LUTs. By default is set to none. Available LUTs: <dl class=”table”> <dt>‘<samp class=”samp”>magma</samp>’</dt> <dt>‘<samp class=”samp”>inferno</samp>’<
<dt>‘<samp class=”samp”>plasma</samp>’</dt> <dt>‘<samp class=”samp”>viridis</samp>’</dt> <dt>‘<samp class=”samp”>turbo</samp>’</
<dt>‘<samp class=”samp”>cividis</samp>’</dt> <dt>‘<samp class=”samp”>range1</samp>’</dt> <dt>‘<samp class=”samp”>range2</samp>’<
<dt>‘<samp class=”samp”>shadows</samp>’</dt> <dt>‘<samp class=”samp”>highlights</samp>’</dt> <dt>‘<samp class=”samp”>solar</samp>’</dt> <dt>‘<samp class=”samp”>nominal</samp>’</
<dt>‘<samp class=”samp”>preferred</samp>’</dt> <dt>‘<samp class=”samp”>total</samp>’</dt> <dt>‘<samp class=”samp”>spectral</samp>’</
<dt>‘<samp class=”samp”>cool</samp>’</dt> <dt>‘<samp class=”samp”>heat</samp>’</dt> <dt>‘<samp class=”samp”>fiery</samp>’</dt>
<dt>‘<samp class=”samp”>blues</samp>’</dt> <dt>‘<samp class=”samp”>green</samp>’</dt> <dt>‘<samp class=”samp”>helix</samp>’</dt
</dl>

</dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity of output colors. Allowed range is from 0 to 1. Default value is set to

1. </dd> </dl>

Each of the expression options specifies the expression to use for computing the lookup table for the corresponding pixel component values. The expressions can contain the following constants and functions: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class="option">ymin, umin, vmin, amin</samp></dt> <dd> The minimum allowed component value. </dd> <dt><samp class="option">ymax, umax, vmax, amax</samp></dt> <dd> The maximum allowed component value. </dd> </dl>

All expressions default to "val". <h4 class="subsection">30.204.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">30.204.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Change too high luma values to gradient: <pre class="example-preformatted">pseudocolor="'if(between(val,ymax,amax) ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(umax,umin,(val-ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(vmin,vmax,(val-ymax)/(amax-ymax)),-1):-1'" </pre>

 <h3 class="section">30.205 psnr# TOC</h3>

Obtain the average, maximum and minimum PSNR (Peak Signal to Noise Ratio) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the PSNR. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average PSNR is printed through the logging system. The filter stores the accumulated MSE (mean squared error) of each frame, and at the end of the processing it is averaged across all frames equally, and the following formula is applied to obtain the PSNR: <pre class="example-preformatted">PSNR = 10*log10(MAX^2/MSE) </pre>

Where MAX is the average of the maximum values of each component of the image. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the PSNR of each individual frame. When filename equals "-" the data is sent to standard output. </dd> <dt><samp class="option">stats_version</samp></dt> <dd> Specifies which version of the stats file format to use. Details of each format are written below. Default value is 1. </dd> <dt><samp class="option">output_max</samp></dt> <dd> Determines whether the max value is output to the stats log. Default value is 0. Requires stats_version >= 2. If this is set and stats_version < 2, the filter will return an error. </dd> </dl>

This filter also supports the framesync options. The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. If a <var class="var">stats_version</var> greater than 1 is specified, a header line precedes the list of per-frame-pair stats, with key value pairs following the frame format with the following parameters: <dl class="table"> <dt><samp class="option">psnr_log_version</samp></dt> <dd> The version of the log file format. Will match <var class="var">stats_version</var>. </dd> <dt><samp class="option">fields</samp></dt> <dd> A comma separated list of the per-frame-pair parameters included in the log. </dd> </dl>

A description of each shown per-frame-pair parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">mse_avg</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames, averaged over all the image components. </dd> <dt><samp class="option">mse_y, mse_u, mse_v, mse_r, mse_g, mse_b, mse_a</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">psnr_y, psnr_u, psnr_v, psnr_r, psnr_g, psnr_b, psnr_a</samp></dt> <dd> Peak Signal to Noise ratio of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">max_avg, max_y, max_u, max_v</samp></dt> <dd> Maximum allowed value for each channel, and average over all channels. </dd> </dl>

 <h4 class="subsection">30.205.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main];

[main][ref] psnr="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The PSNR of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]psnr" -f null - </pre>

 <h3 class="section">30.206 pullup# TOC</h3>

Pulldown reversal (inverse telecine) filter, capable of handling mixed hard-telecine, 24000/1001 fps progressive, and 30000/1001 fps progressive content. The pullup filter is designed to take advantage of future context in making its decisions. This filter is stateless in the sense that it does not lock onto a pattern to follow, but it instead looks forward to the following fields in order to identify matches and rebuild progressive frames. To produce content with an even framerate, insert the fps filter after pullup, use <code class="code">fps=24000/1001</code> if the input frame rate is 29.97fps, <code class="code">fps=24</code> for 30fps and the (rare) telecined 25fps input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">jl</samp></dt> <dt><samp class="option">jr</samp></dt> <dt><samp class="option">jt</samp></dt> <dt><samp class="option">jb</samp></dt> <dd> These options set the amount of "junk" to ignore at the left, right, top, and bottom of the image, respectively. Left and right are in units of 8 pixels, while top and bottom are in units of 2 lines. The default is 8 pixels on each side. </dd> <dt><samp class="option">sb</samp></dt> <dd> Set the strict breaks. Setting this option to 1 will reduce the chances of filter generating an occasional mismatched frame, but it may also cause an excessive number of frames to be dropped during high motion sequences. Conversely, setting it to -1 will make filter match fields more easily. This may help processing of video where there is slight blurring between the fields, but may also cause there to be interlaced frames in the output. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">mp</samp></dt> <dd> Set the metric plane to use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">l</samp>’</dt> <dd> Use luma plane. </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> Use chroma blue plane. </dd> <dt>‘<samp class="samp">v</samp>’</dt> <dd> Use chroma red plane. </dd> </dl>

This option may be set to use chroma plane instead of the default luma

plane for doing filter’s computations. This may improve accuracy on very clean source material, but more likely will decrease accuracy, especially if there is chroma noise (rainbow effect) or any grayscale video. The main purpose of setting <samp class="option">mp</samp> to a chroma plane is to reduce CPU load and make pullup usable in realtime on slow machines. </dd> </dl>

For best results (without duplicated frames in the output file) it is necessary to change the output frame rate. For example, to inverse telecine NTSC input: <pre class="example-preformatted">ffmpeg -i input -vf pullup -r 24000/1001 ... </pre>

 <h3 class="section">30.207 qp# TOC</h3>

Change video quantization parameters (QP). The filter accepts the following option: <dl class="table"> <dt><samp class="option">qp</samp></dt> <dd> Set expression for quantization parameter. </dd> </dl>

The expression is evaluated through the eval API and can contain, among others, the following constants: <dl class="table"> <dt><var class="var">known</var></dt> <dd> 1 if index is not 129, 0 otherwise. </dd> <dt><var class="var">qp</var></dt> <dd> Sequential index starting from -129 to 128. </dd> </dl>

 <h4 class="subsection">30.207.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Some equation like: <pre class="example-preformatted">qp=2+2*sin(PI*qp) </pre>

 <h3 class="section">30.208 qrencode# TOC</h3> Generate a QR code using the libqrencode library (see <a class="url" href="https://fukuchi.org/works/qrencode/">https://fukuchi.o and overlay it on top of the current frame. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and overlaid into the video output according to the specified options. In case no text is specified, no QR code is overlaied. This filter accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp

class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify an expression for positioning the padded QR code top-left corner. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice. By default <var class=”var”>x</var> and <var class=”var”>y</var> are set set to <var class=”var”>0</var>, meaning that the QR code is placed in the top left corner of the input. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is

"white". For the syntax of the color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">30.208.1 qrencode Expressions# TOC</h4>

The expressions set by the options contain the following constants and functions. <dl class="table"> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">duration</samp></dt> <dd> the current frame’s duration, in seconds </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">main_h, H</samp></dt> <dd> the input height </dd> <dt><samp class="option">main_w, W</samp></dt> <dd> the input width </dd> <dt><samp class="option">n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class="option">pict_type</samp></dt> <dd> a number representing the picture type </dd> <dt><samp class="option">qr_w, w</samp></dt> <dd> the width of the encoded QR code </dd> <dt><samp class="option">rendered_qr_w, q</samp></dt> <dt><samp class="option">rendered_padded_qr_w, Q</samp></dt> <dd> the width of the rendered QR code, without and without padding. These parameters allow the <var class="var">q</var> and <var class="var">Q</var> expressions to refer to each other, so you can for example specify <code class="code">q=3/4*Q</code>. </dd> <dt><samp class="option">rand(min, max)</samp></dt> <dd> return a random number included between <var class="var">min</var> and <var class="var">max</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> the input sample aspect ratio </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> the x and y offset coordinates where the text is drawn. These parameters allow the <var class="var">x</var> and <var class="var">y</var> expressions to refer to each other, so you can for example specify <code class="code">y=x/dar</code>. </dd> </dl>

<a

name=”qrencode-Text-expansion”> <h4 class=”subsection”>30.208.2 qrencode Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim. If <samp class=”option”>expansion</samp> is set to <code class=”code”>normal</code> (which is the default), the following expansion mechanism is used. The backslash character ‘<samp class=”samp”>\</samp>’, followed by any character, always expands to the second character. Sequences of the form <code class=”code”>%{...}</code> are expanded. The text between the braces is a function name, possibly followed by arguments separated by ’:’. If the arguments contain special characters or delimiters (’:’ or ’}’), they should be escaped. Note that they probably must also be escaped as the value for the <samp class=”option”>text</samp> option in the filter argument string and as the filter argument in the filtergraph description, and possibly also for the shell, that makes up to four levels of escaping; using a text file with the <samp class=”option”>textfile</samp> option avoids these problems. The following functions are available: <dl class=”table”> <dt><code class=”command”>n, frame_num</code></dt> <dd> return the frame number </dd> <dt><code class=”command”>pts</code></dt> <dd> Return the presentation timestamp of the current frame. It can take up to two arguments. The first argument is the format of the timestamp; it defaults to <code class=”code”>flt</code> for seconds as a decimal number with microsecond accuracy; <code class=”code”>hms</code> stands for a formatted <var class=”var”>[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy. <code class=”code”>gmtime</code> stands for the timestamp of the frame formatted as UTC time; <code class=”code”>localtime</code> stands for the timestamp of the frame formatted as local time zone time. If the format is set to <code class=”code”>hms24hh</code>, the time is formatted in 24h format (00-23). The second argument is an offset added to the timestamp. If the format is set to <code class=”code”>localtime</code> or <code class=”code”>gmtime</code>, a third argument may be supplied: a <code class=”code”>strftime</code> C function format string. By default, <var class=”var”>YYYY-MM-DD HH:MM:SS</var> format will be used. </dd> <dt><code class=”command”>expr, e</code></dt> <dd> Evaluate the expression’s value and output as a double. It must take one argument specifying the expression to be evaluated, accepting the constants and functions defined in qrencode_expressions. </dd> <dt><code class=”command”>expr_formatted, ef</code></dt> <dd> Evaluate the expression’s value and output as a formatted string. The first argument is the expression to

be evaluated, just as for the <var class=”var”>expr</var> function. The second argument specifies the output format. Allowed values are ‘<samp class=”samp”>x</samp>’, ‘<samp class=”samp”>X</samp>’, ‘<samp class=”samp”>d</samp>’ and ‘<samp class=”samp”>u</samp>’. They are treated exactly as in the <code class=”code”>printf</code> function. The third parameter is optional and sets the number of positions taken by the output. It can be used to add padding with zeros from the left. </dd> <dt><code class=”command”>gmtime</code></dt> <dd> The time at which the filter is running, expressed in UTC. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>localtime</code></dt> <dd> The time at which the filter is running, expressed in the local time zone. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>metadata</code></dt> <dd> Frame metadata. Takes one or two arguments. The first argument is mandatory and specifies the metadata key. The second argument is optional and specifies a default value, used when the metadata key is not found or empty. Available metadata can be identified by inspecting entries starting with TAG included within each frame section printed by running <code class=”code”>ffprobe -show_frames</code>. String metadata generated in filters leading to the qrencode filter are also available. </dd> <dt><code class=”command”>rand(min, max)</code></dt> <dd> return a random number included between <var class=”var”>min</var> and <var class=”var”>max</var> </dd> </dl>

 <h4 class=”subsection”>30.208.3 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Generate a QR code encoding the specified text with the default size, overalaid in the top left corner of the input video, with the default size: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:bc=pink@0.5:fc=blue </pre>

Place the QR code in the bottom right corner of the input video: <pre class=”example-preformatted”>qrencode=text=www.ffmpeg.org:x=W-

Q:y=H-Q </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Make the QR code a fraction of the input video width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=W/5 </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencode=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencode=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencode=text=%{pts} </pre>

 <h3 class="section">30.209 quirc# TOC</h3>

Identify and decode a QR code using the libquirc library (see https://github.com/dlbeer/quirc/), and print the identified QR codes positions and payload as metadata. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libquirc</code>. For each found QR code in the input video, some metadata entries are added with the prefix <var class="var">lavfi.quirc.N</var>, where <var class="var">N</var> is the index, starting from 0, associated to the QR code. A description of each metadata value follows: <dl class="table"> <dt><samp class="option">lavfi.quirc.count</samp></dt> <dd> the number of found QR codes, it is not set in case none was found </dd> <dt><samp class="option">lavfi.quirc.N.corner.M.x</samp></dt> <dt><samp class="option">lavfi.quirc.N.coreer.M.y</samp></dt> <dd> the x/y positions of the four corners of the square containing the QR code, where <var class="var">M</var> is the index of the corner starting from 0 </dd> <dt><samp class="option">lavfi.quirc.N.payload</samp></dt> <dd> the payload of the QR code </dd> </dl>

 <h3 class="section">30.210 random# TOC</h3>

Flush video frames from internal cache of frames into a random order. No frame is discarded. Inspired by frei0r nervous filter. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> Set size in number of frames of internal cache, in range from <code class="code">2</code> to <code class="code">512</code>. Default is <code class="code">30</code>. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed for random number generator, must be an integer included between <code class="code">0</code> and <code class="code">UINT32_MAX</code>. If not specified, or if explicitly set to less than <code class="code">0</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

 <h3 class="section">30.211 readeia608# TOC</h3>

Read closed captioning (EIA-608) information from the top lines of a video frame. This filter adds frame metadata for <code class="code">lavfi.readeia608.X.cc</code> and <code class="code">lavfi.readeia608.X.line</code>, where <code class="code">X</code> is the number of the identified line with EIA-608 data (starting from 0). A description of each metadata value follows: <dl class="table"> <dt><samp class="option">lavfi.readeia608.X.cc</samp></dt> <dd> The two bytes stored as EIA-608 data (printed in hexadecimal). </dd> <dt><samp class="option">lavfi.readeia608.X.line</samp></dt> <dd> The number of the line on which the EIA-608 data was identified and read. </dd> </dl>

This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_min</samp></dt> <dd> Set the line to start scanning for EIA-608 data. Default is <code class="code">0</code>. </dd> <dt><samp class="option">scan_max</samp></dt> <dd> Set the line to end scanning for EIA-608 data. Default is <code class="code">29</code>. </dd> <dt><samp class="option">spw</samp></dt> <dd> Set the ratio of width reserved for sync code detection. Default is <code class="code">0.27</code>. Allowed range is <code class="code">[0.1 - 0.7]</code>. </dd> <dt><samp class="option">chp</samp></dt> <dd> Enable checking the parity bit. In the event of a parity error, the filter will output <code class="code">0x00</code> for that character. Default is false. </dd> <dt><samp class="option">lp</samp></dt> <dd> Lowpass lines prior to further processing. Default is enabled. </dd> </dl>

 <h4 class="subsection">30.211.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">30.211.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output a csv with presentation time and the first two lines of identified EIA-608 captioning data. <pre class="example-preformatted">ffprobe -f lavfi -i movie=captioned_video.mov,readeia608 -show_entries frame=pts_time:frame_tags=lavfi.readeia608.0.cc,lavf -of csv </pre>

 <h3 class="section">30.212 readvitc# TOC</h3>

Read vertical interval timecode (VITC) information from the top lines of a video frame. The filter adds frame metadata key <code class="code">lavfi.readvitc.tc_str</code> with the timecode value, if a valid timecode has been detected. Further metadata key <code class="code">lavfi.readvitc.found</code> is set to 0/1 depending on whether timecode data has been found or not. This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_max</samp></dt> <dd> Set the maximum number of lines to scan for VITC data. If the value is set to <code class="code">-1</code> the full video frame is scanned. Default is <code class="code">45</code>. </dd> <dt><samp class="option">thr_b</samp></dt> <dd> Set the luma threshold for black. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.2</code>. The value must be equal or less than <code class="code">thr_w</code>. </dd> <dt><samp class="option">thr_w</samp></dt> <dd> Set the luma threshold for white. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.6</code>. The value must be equal or greater than <code class="code">thr_b</code>. </dd> </dl>

 <h4 class="subsection">30.212.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect and draw VITC data onto the video frame; if no valid VITC is detected, draw <code class="code">--:--:--

:--</code> as a placeholder: <pre class="example-preformatted">ffmpeg -i input.avi -filter:v 'readvitc,drawtext=fontfile=FreeMono.ttf:text=%{metadata\\:lavfi
-\\\\\\:--\\\\\\:--\\\\\\:--}:x=(w-tw)/2:y=400-ascent' </pre>

 <h3 class="section">30.213 remap# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 16bit depth, single channel.
<dl class="table"> <dt><samp class="option">format</samp></dt>
<dd> Specify pixel format of output from this filter. Can be <code class="code">color</code> or <code class="code">gray</code>. Default is <code class="code">color</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">30.214 removegrain# TOC</h3>

The removegrain filter is a spatial denoiser for progressive video. <dl class="table"> <dt><samp class="option">m0</samp></dt>
<dd> Set mode for the first plane. </dd> <dt><samp class="option">m1</samp></dt> <dd> Set mode for the second plane. </dd> <dt><samp class="option">m2</samp></dt>
<dd> Set mode for the third plane. </dd> <dt><samp class="option">m3</samp></dt> <dd> Set mode for the fourth plane. </dd> </dl>

Range of mode is from 0 to 24. Description of each mode follows:

<dl class="table">

<dt><var class="var">0</var></dt>

<dd> Leave input plane unchanged. Default.

</dd>

<dt><var class=”var”>1</var></dt>

<dd> Clips the pixel with the minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>2</var></dt>

<dd> Clips the pixel with the second minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>3</var></dt>

<dd> Clips the pixel with the third minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class=”var”>4</var></dt>

<dd> Clips the pixel with the fourth minimum and maximum of the 8 neighbour pixels.

This is equivalent to a median filter.

</dd>

<dt><var class=”var”>5</var></dt>

<dd> Line-sensitive clipping giving the minimal change.

</dd>

<dt><var class=”var”>6</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class=”var”>7</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">8</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">9</var></dt>

<dd> Line-sensitive clipping on a line where the neighbours pixels are the closest.

</dd>

<dt><var class="var">10</var></dt>

<dd> Replaces the target pixel with the closest neighbour.

</dd>

<dt><var class="var">11</var></dt>

<dd> [1 2 1] horizontal and vertical kernel blur.

</dd>

<dt><var class="var">12</var></dt>

<dd> Same as mode 11.

</dd>

<dt><var class="var">13</var></dt>

<dd> Bob mode, interpolates top field from the line where the neighbours pixels are the closest.

</dd>

<dt><var class="var">14</var></dt>

<dd> Bob mode, interpolates bottom field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class=”var”>15</var></dt>

<dd> Bob mode, interpolates top field. Same as 13 but with a more complicated

interpolation formula.

</dd>

<dt><var class=”var”>16</var></dt>

<dd> Bob mode, interpolates bottom field. Same as 14 but with a more complicated

interpolation formula.

</dd>

<dt><var class=”var”>17</var></dt>

<dd> Clips the pixel with the minimum and maximum of respectively the maximum and

minimum of each pair of opposite neighbour pixels.

</dd>

<dt><var class=”var”>18</var></dt>

<dd> Line-sensitive clipping using opposite neighbours whose greatest distance from

the current pixel is minimal.

</dd>

<dt><var class=”var”>19</var></dt>

<dd> Replaces the pixel with the average of its 8 neighbours.

</dd>

<dt><var class="var">20</var></dt>

<dd> Averages the 9 pixels ([1 1 1] horizontal and vertical blur).

</dd>

<dt><var class="var">21</var></dt>

<dd> Clips pixels using the averages of opposite neighbour.

</dd>

<dt><var class="var">22</var></dt>

<dd> Same as mode 21 but simpler and faster.

</dd>

<dt><var class="var">23</var></dt>

<dd> Small edge and halo removal, but reputed useless.

</dd>

<dt><var class="var">24</var></dt>

<dd> Similar as 23.

</dd>

</dl>

 <h3 class="section">30.215 removelogo# TOC</h3>

Suppress a TV station logo, using an image file to determine which pixels comprise the logo. It works by filling in the pixels that comprise the logo with neighboring pixels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filter bitmap file, which can be any image format supported by libavformat. The width and height of the image file must match those of the video stream being processed. </dd> </dl>

Pixels in the provided bitmap image with a value of zero are not considered part of the logo, non-zero pixels are considered part of the logo. If you use

white (255) for the logo and black (0) for the rest, you will be safe. For making the filter bitmap, it is recommended to take a screen capture of a black frame with the logo visible, and then using a threshold filter followed by the erode filter once or twice. If needed, little splotches can be fixed manually. Remember that if logo pixels are not covered, the filter quality will be much reduced. Marking too many pixels as part of the logo does not hurt as much, but it will increase the amount of blurring needed to cover over the image and will destroy more information than necessary, and extra pixels will slow things down on a large logo. <h3 class="section">30.216 repeatfields# TOC</h3>

This filter uses the repeat_field flag from the Video ES headers and hard repeats fields based on its value. <h3 class="section">30.217 reverse# TOC</h3>

Reverse a video clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">30.217.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">trim=end=5,reverse </pre>

 <h3 class="section">30.218 rgbashift# TOC</h3> Shift R/G/B/A pixels horizontally and/or vertically. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rh</samp></dt> <dd> Set amount to shift red horizontally. </dd> <dt><samp class="option">rv</samp></dt> <dd> Set amount to shift red vertically. </dd> <dt><samp class="option">gh</samp></dt> <dd> Set amount to shift green horizontally. </dd> <dt><samp class="option">gv</samp></dt> <dd> Set amount to shift green vertically. </dd> <dt><samp class="option">bh</samp></dt> <dd> Set amount to shift blue horizontally. </dd> <dt><samp class="option">bv</samp></dt> <dd> Set amount to shift blue vertically. </dd> <dt><samp class="option">ah</samp></dt> <dd> Set amount to shift alpha horizontally. </dd> <dt><samp class="option">av</samp></dt> <dd> Set amount to shift alpha

vertically. </dd> <dt><samp class="option">edge</samp></dt> <dd> Set edge mode, can be <var class="var">smear</var>, default, or <var class="var">warp</var>. </dd> </dl>

 <h4 class="subsection">30.218.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.219 roberts# TOC</h3> Apply roberts cross operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">30.219.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.220 rotate# TOC</h3>

Rotate video by an arbitrary angle expressed in radians. The filter accepts the following options: A description of the optional parameters follows. <dl class="table"> <dt><samp class="option">angle, a</samp></dt> <dd> Set an expression for the angle by which to rotate the input video clockwise, expressed as a number of radians. A negative value will result in a counter-clockwise rotation. By default it is set to "0". This expression is evaluated for each frame. </dd> <dt><samp class="option">out_w, ow</samp></dt> <dd> Set the output width expression, default value is "iw". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">out_h, oh</samp></dt> <dd> Set the output height expression, default value is "ih". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">bilinear</samp></dt> <dd> Enable bilinear interpo-

lation if set to 1, a value of 0 disables it. Default value is 1. </dd> <dt><samp class=”option”>fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the rotated image. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> </dl>

The expressions for the angle and the output size can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> sequential number of the input frame, starting from 0. It is always NAN before the first frame is filtered. </dd> <dt><samp class=”option”>t</samp></dt> <dd> time in seconds of the input frame, it is set to 0 when the filter is configured. It is always NAN before the first frame is filtered. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_w, iw</samp></dt> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> the input video width and height </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> the output width and height, that is the size of the padded area as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions </dd> <dt><samp class=”option”>rotw(a)</samp></dt> <dt><samp class=”option”>roth(a)</samp></dt> <dd> the minimal width/height required for completely containing the input video rotated by <var class=”var”>a</var> radians. These are only available when computing the <samp class=”option”>out_w</samp> and <samp class=”option”>out_h</samp> expressions. </dd> </dl>

 <h4 class=”subsection”>30.220.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Rotate the input by PI/6 radians clockwise: <pre class=”example-preformatted”>rotate=PI/6 </pre>

Rotate the input by PI/6 radians counter-clockwise: <pre class=”example-preformatted”>rotate=-PI/6 </pre>

Rotate the input by 45 degrees clockwise: <pre class=”example-preformatted”>rotate=45*PI/180 </pre>

Apply a constant rotation with period T, starting from an angle of PI/3: <pre class=”example-preformatted”>rotate=PI/3+2*PI*t/T </pre>

Make the input video rotation oscillating with a period of T seconds and an amplitude of A radians: <pre class=”example-preformatted”>rotate=A*sin(2*PI/T*t) </pre>

Rotate the video, output size is chosen so that the whole rotating input video is always completely contained in the output: <pre class=”example-preformatted”>rotate='2*PI*t:ow=hypot(iw,ih):oh=ow' </pre>

Rotate the video, reduce the output size so that no background is ever shown: <pre class=”example-preformatted”>rotate=2*PI*t:ow='min(iw,ih)/sqrt(2)':oh=ow:c=none </pre>

 <h4 class=”subsection”>30.220.2 Commands# TOC</h4>

The filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>a, angle</samp></dt> <dd> Set the angle expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>30.221 sab# TOC</h3>

Apply Shape Adaptive Blur. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dd> Set luma blur filter strength, must be a value in range 0.1-4.0, default value is 1.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>luma_pre_filter_radius, lpfr</samp></dt> <dd> Set luma pre-filter radius, must be a value in the 0.1-2.0 range, default value is 1.0. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma maximum difference between pixels to still be considered, must be a value in the 0.1-100.0 range, default value is 1.0. </dd> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dd> Set chroma blur filter strength, must be a value in range -0.9-4.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>chroma_pre_filter_radius, cpfr</samp></dt> <dd> Set chroma pre-filter radius, must be a value in the -0.9-2.0 range. </dd> <dt><samp class=”option”>chroma_strength,

cs</samp></dt> <dd> Set chroma maximum difference between pixels to still be considered, must be a value in the -0.9-100.0 range. </dd> </dl>

Each chroma option value, if not explicitly specified, is set to the corresponding luma option value. <h3 class="section">30.222 scale# TOC</h3>

Scale (resize) the input video, using the libswscale library. The scale filter forces the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the scale filter will convert the input to the requested format. <h4 class="subsection">30.222.1 Options# TOC</h4> The filter accepts the following options, any of the options supported by the libswscale scaler, as well as any of the framesync options. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#scaler_005foptions">the ffmpeg-scaler manual for the complete list of scaler options. <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class="var">width</var> or <var class="var">w</var> value is 0, the input width is used for the output. If the <var class="var">height</var> or <var class="var">h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the scale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class="option">eval</samp></dt> <dd> Specify when to evaluate <var class="var">width</var> and <var class="var">height</var> expression. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.

</dd> <dt><samp class="option">interl</samp></dt> <dd> Set the interlacing mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">1</samp>’</dt> <dd> Force interlaced aware scaling. </dd> <dt>‘<samp class="samp">0</samp>’</dt> <dd> Do not apply interlaced scaling. </dd> <dt>‘<samp class="samp">-1</samp>’</dt> <dd> Select interlaced aware scaling depending on whether the source frames are flagged as interlaced or not. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">flags</samp></dt> <dd> Set libswscale scaling flags. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#sws_005fflags">the ffmpeg-scaler manual for the complete list of values. If not explicitly specified the filter applies the default flags.

</dd> <dt><samp class="option">param0, param1</samp></dt> <dd> Set libswscale input parameters for scaling algorithms that need them. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#sws_005fparams">the ffmpeg-scaler manual for the complete documentation. If not explicitly specified the filter applies empty parameters. </dd> <dt><samp class="option">intent</samp></dt> <dd> Set the ICC rendering intent to use when transforming between different color spaces. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">perceptual</samp>’</dt> <dd> Use a perceptually guided tone and gamut mapping curve. The exact details of the mapping used may change at any time and should not be relied on as stable. This intent is recommended for final viewing of image/video content in typical viewing settings. </dd> <dt>‘<samp class="samp">relative_colorimetric</samp>’</dt> <dd> Statically clip out-of-gamut colors using a colorimetric clipping curve which attempts to find the colorimetrically least dissimilar in-gamut color. This intent performs white point adaptation and black point adaptation. This is the default. This intent is recommended wherever faithful color reproduction is of the utmost importance, even at the cost of clipping. </dd> <dt>‘<samp class="samp">absolute_colorimetric</samp>’</dt> <dd> Hard clip out-of-gamut colors with no attempt at white or black point reproduction. This intent will reproduce in-gamut colors 1:1 on the output display as they would appear on the reference display, assuming the output display is appropriately calibrated. </dd> <dt>‘<samp class="samp">saturation</samp>’</dt> <dd> Performs saturation mapping - that is, stretches the input color volume directly onto the output color volume, in non-linear fashion that preserves the original signal appearance as much as possible. This intent is recommended for signal content evaluation, as it will not lead to any clipping. It is roughly analogous to not performing

any color mapping, although it still takes into account the mastering display primaries and any differences in encoding TRC. </dd> </dl>

</dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">in_color_matrix</samp></dt> <dt><samp class="option">out_color_matrix</samp><
<dd> Set in/output YCbCr color space type. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the color space type depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> Format conforming to International Telecommunication Union (ITU) Recommendation BT.709. </dd> <dt>‘<samp class="samp">fcc</samp>’</dt> <dd> Set color space conforming to the United States Federal Communications Commission (FCC) Code of Federal Regulations (CFR) Title 47 (2003) 73.682 (a). </dd> <dt>‘<samp class="samp">bt601</samp>’</dt> <dt>‘<samp class="samp">bt470</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> Set color space conforming to: <ul class="itemize mark-bullet"> ITU Radiocommunication Sector (ITU-R) Recommendation BT.601

ITU-R Rec. BT.470-6 (1998) Systems B, B1, and G

Society of Motion Picture and Television Engineers (SMPTE) ST 170:2004

</dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> Set color space conforming to SMPTE ST 240:1999. </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> Set color space conforming to ITU-R BT.2020 non-constant luminance system. </dd> </dl>

</dd> <dt><samp class="option">in_range</samp></dt> <dt><samp class="option">out_range</samp></dt> <dd> Set in/output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt>

<dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> <dt><samp class="option">in_chroma_loc</samp></dt> <dt><samp class="option">out_chroma_loc</samp></dt> <dd> Set in/output chroma sample location. If <samp class="option">in_chroma_loc</samp> is not specified, the chroma location tagged on the input frame is used, and untagged input is assumed to be sited like the output, so it is tagged but not converted. If <samp class="option">out_chroma_loc</samp> is not specified, the output is converted to the chroma location negotiated with the rest of the filtergraph, or keeps the input location if none was negotiated. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, unknown</samp>’</dt> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt> <dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dt><samp class="option">out_primaries</samp></dt> <dd> Set in/output RGB primaries. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dt>‘<samp class="samp">ebu3213</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_transfer</samp></dt> <dt><samp class="option">out_transfer</samp></dt> <dd> Set in/output transfer response curve (TRC). This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt>

<dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dt>‘<samp class="samp">srgb</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dt>‘<samp class="samp">xvycc</samp>’</dt> <dt>‘<samp class="samp">bt1361e</samp>’</dt> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dd> Enable decreasing or increasing output video width or height if necessary to keep the original aspect ratio. Possible values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dd> Scale the video as specified and disable this feature. </dd> <dt>‘<samp class="samp">decrease</samp>’</dt> <dd> The output video dimensions will automatically be decreased if needed. </dd> <dt>‘<samp class="samp">increase</samp>’</dt> <dd> The output video dimensions will automatically be increased if needed. </dd> </dl>

One useful instance of this option is that when you know a specific device’s maximum allowed resolution, you can use this to limit the output video to that, while retaining the aspect ratio. For example, device A allows 1280x720 playback, and your video is 1920x800. Using this option (set it to decrease) and specifying 1280x720 to the command line makes the output 1280x533. Please note that this is a different thing than specifying -1 for <samp class="option">w</samp> or <samp class="option">h</samp>, you still need to specify the output resolution for this option to work. </dd> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Ensures that both the output dimensions, width and height, are divisible by the given integer when used together with <samp class="option">force_original_aspect_ratio</samp>. This works similar to using <code class="code">-n</code> in the <samp class="option">w</samp> and <samp class="option">h</samp> options. This option respects the value set for <samp class="option">force_original_aspect_ratio</samp>, increasing or decreasing the resolution accordingly. The video’s aspect ratio may be slightly modified. This option can be handy if you need to have a video fit within or exceed a defined resolution using <samp class="option">force_original_aspect_ratio</samp> but also have encoder restrictions on width or height divisibility. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Enabling this option leads to the output SAR being reset to 1. Additionally, if the user requests proportional scaling either through the width or height expressions, e.g. <code class="code">w=-4:h=360</code> or <code class="code">w=iw/2:h=-1</code> or by enabling <code class="code">force_original_aspect_ratio</code>,

then the input DAR is taken into account and the output is scaled to produce square pixels. Default is false. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt> <dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt> <dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt> <dd> The same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt> <dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt> <dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt> <dt><var class="var">ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">n</var></dt> <dd> The (sequential) number of the input frame, starting from 0. Only available with <code class="code">eval=frame</code>. </dd> <dt><var class="var">t</var></dt> <dd> The presentation timestamp of the input frame, expressed as a number of seconds. Only available with <code class="code">eval=frame</code>. </dd> <dt><var class="var">pos</var></dt> <dd> The position (byte offset) of the frame in the input stream, or NaN if this information is unavailable and/or meaningless (for example in case of synthetic video). Only available with <code class="code">eval=frame</code>. Deprecated, do not use. </dd> <dt><var class="var">ref_w, rw</var></dt> <dt><var class="var">ref_h, rh</var></dt> <dt><var class="var">ref_a</var></dt> <dt><var class="var">ref_dar, rdar</var></dt> <dt><var class="var">ref_n</var></dt> <dt><var class="var">ref_t</var></dt> <dt><var class="var">ref_pos</var></dt> <dd> Eqvuialent to the above, but for a second reference input. If any of these variables are present, this filter accepts two inputs. </dd> </dl>

 <h4 class="subsection">30.222.2 Examples<a class="anchor hidden-xs" href="#Examples-

137" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Scale the input video to a size of 200x100 <pre class="example-preformatted">scale=w=200:h=100 </pre>

This is equivalent to: <pre class="example-preformatted">scale=200:100 </pre>

or: <pre class="example-preformatted">scale=200x100 </pre>

Specify a size abbreviation for the output size: <pre class="example-preformatted">scale=qcif </pre>

which can also be written as: <pre class="example-preformatted">scale=size=qcif </pre>

Scale the input to 2x: <pre class="example-preformatted">scale=w=2*iw:h=2*ih </pre>

The above is the same as: <pre class="example-preformatted">scale=2*in_w:2*in_h </pre>

Scale the input to 2x with forced interlaced scaling: <pre class="example-preformatted">scale=2*iw:2*ih:interl=1 </pre>

Scale the input to half size: <pre class="example-preformatted">scale=w=iw/2:h=ih/2 </pre>

Increase the width, and set the height to the same size: <pre class="example-preformatted">scale=3/2*iw:ow </pre>

Seek Greek harmony: <pre class="example-preformatted">scale=iw:1/PHI*iw scale=ih*PHI:ih </pre>

Increase the height, and set the width to 3/2 of the height: <pre class="example-preformatted">scale=w=3/2*oh:h=3/5*ih </pre>

Increase the size, making the size a multiple of the chroma subsample values: <pre class="example-preformatted">scale="trunc(3/2*iw/hsub)*hsub:t </pre>

Increase the width to a maximum of 500 pixels, keeping the same aspect ratio as the input: <pre class="example-preformatted">scale=w='min(500\, iw*3/2):h=-1' </pre>

Make pixels square by combining scale and setsar: <pre class="example-preformatted">scale='trunc(ih*dar):ih',setsar=1/1

</pre>

Make pixels square using reset_sar, making sure the resulting resolution is even (required by some codecs): <pre class="example-preformatted">scale='-2:ih-mod(ih,2):reset_sar=1' </pre>

Scale to target exactly, however reset SAR to 1: <pre class="example-preformatted">scale='400:300:reset_sar=1' </pre>

Scale to even dimensions that fit within 400x300, preserving input SAR: <pre class="example-preformatted">scale='400:300:force_original_aspect_rat
</pre>

Scale to produce square pixels with even dimensions that fit within 400x300: <pre class="example-preformatted">scale='400:300:force_original_aspect_
</pre>

Scale a subtitle stream (sub) to match the main video (main) in size before overlaying. ("scale2ref") <pre class="example-preformatted">'[main]split[a][b]; [ref][a]scale=rw:rh[c]; [b][c]overlay' </pre>

Scale a logo to 1/10th the height of a video, while preserving its display aspect ratio. <pre class="example-preformatted">[logo-in][video-in]scale=w=oh*dar:h=rh/10[logo-out] </pre>

 <h4 class="subsection">30.222.3 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">30.223 scharr# TOC</h3> Apply scharr operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp

class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class=”subsection”>30.223.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.224 scroll# TOC</h3> Scroll input video horizontally and/or vertically by constant speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>hpos</samp></dt> <dd> Set the initial horizontal scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> <dt><samp class=”option”>vpos</samp></dt> <dd> Set the initial vertical scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> </dl>

 <h4 class=”subsection”>30.224.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. </dd> </dl>

 <h3 class=”section”>30.225 scdet# TOC</h3>

Detect video scene change. This filter sets frame metadata with mafd between frame, the scene score, and forward the frame to the next filter, so they can use these metadata to detect scene change or others. In addition, this filter logs a message and sets frame metadata when it detects a scene change by <samp class=”option”>threshold</samp>. <code class=”code”>lavfi.scd.mafd</code> metadata keys are set with mafd for every frame. <code class=”code”>lavfi.scd.score</code>

metadata keys are set with scene change score for every frame to detect scene change. <code class="code">lavfi.scd.time</code> metadata keys are set with current filtered frame time which detect scene change with <samp class="option">threshold</samp>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold, t</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change. Good values are in the <code class="code">[8.0, 14.0]</code> range. The range for <samp class="option">threshold</samp> is <code class="code">[0., 100.]</code>. Default value is <code class="code">10.</code>. </dd> <dt><samp class="option">sc_pass, s</samp></dt> <dd> Set the flag to pass scene change frames to the next filter. Default value is <code class="code">0</code> You can enable it if you want to get snapshot of scene change frames only. </dd> </dl>

 <h3 class="section">30.226 selectivecolor#
TOC</h3>

Adjust cyan, magenta, yellow and black (CMYK) to certain ranges of colors (such as "reds", "yellows", "greens", "cyans", ...). The adjustment range is defined by the "purity" of the color (that is, how saturated it already is). This filter is similar to the Adobe Photoshop Selective Color tool. The filter accepts the following options: <dl class="table"> <dt><samp class="option">correction_method</samp></dt> <dd> Select color correction method. Available values are: <dl class="table">
<dt>‘<samp class="samp">absolute</samp>’</dt> <dd> Specified adjustments are applied "as-is" (added/subtracted to original pixel component value). </dd> <dt>‘<samp class="samp">relative</samp>’</dt>
<dd> Specified adjustments are relative to the original component value. </dd> </dl> Default is <code class="code">absolute</code>. </dd> <dt><samp class="option">reds</samp></dt> <dd> Adjustments for red pixels (pixels where the red component is the maximum) </dd>
<dt><samp class="option">yellows</samp></dt> <dd> Adjustments for yellow pixels (pixels where the blue component is the minimum) </dd>
<dt><samp class="option">greens</samp></dt> <dd> Adjustments for green pixels (pixels where the green component is the maximum) </dd> <dt><samp class="option">cyans</samp></dt> <dd> Adjustments for cyan pixels (pixels where the red component is the minimum) </dd>
<dt><samp class="option">blues</samp></dt> <dd> Adjustments for blue pixels (pixels where the blue component is the maximum) </dd> <dt><samp class="option">magentas</samp></dt>
<dd> Adjustments for magenta pixels (pixels where the green component is the minimum) </dd> <dt><samp class="option">whites</samp></dt>
<dd> Adjustments for white pixels (pixels where all components are greater

than 128) </dd> <dt><samp class="option">neutrals</samp></dt> <dd> Adjustments for all pixels except pure black and pure white </dd> <dt><samp class="option">blacks</samp></dt> <dd> Adjustments for black pixels (pixels where all components are lesser than 128) </dd> <dt><samp class="option">psfile</samp></dt> <dd> Specify a Photoshop selective color file (<code class="code">.asv</code>) to import the settings from. </dd> </dl>

All the adjustment settings (<samp class="option">reds</samp>, <samp class="option">yellows</samp>, ...) accept up to 4 space separated floating point adjustment values in the [-1,1] range, respectively to adjust the amount of cyan, magenta, yellow and black for the pixels of its range. <h4 class="subsection">30.226.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Increase cyan by 50% and reduce yellow by 33% in every green areas, and increase magenta by 27% in blue areas: <pre class="example-preformatted">selectivecolor=greens=.5 0 -.33 0:blues=0 .27 </pre>

Use a Photoshop selective color preset: <pre class="example-preformatted">selectivecolor=psfile=MySelectiveColorPresets/Misty.asv </pre>

 <h3 class="section">30.227 separatefields# TOC</h3>

The <code class="code">separatefields</code> takes a frame-based video input and splits each frame into its components fields, producing a new half height clip with twice the frame rate and twice the frame count. This filter use field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class="code">separatefields</code> filter. <h3 class="section">30.228 setdar, setsar# TOC</h3>

The <code class="code">setdar</code> filter sets the Display Aspect Ratio for the filter output video. This is done by changing the specified Sample (aka Pixel) Aspect Ratio, according to the following equation: <pre class="example-preformatted"><var class="var">DAR</var> = <var class="var">HORIZONTAL_RESOLUTION</var>

/ <var class=”var”>VERTICAL_RESOLUTION</var> * <var class=”var”>SAR</var> </pre>

Keep in mind that the <code class=”code”>setdar</code> filter does not modify the pixel dimensions of the video frame. Also, the display aspect ratio set by this filter may be changed by later filters in the filterchain, e.g. in case of scaling or if another "setdar" or a "setsar" filter is applied. The <code class=”code”>setsar</code> filter sets the Sample (aka Pixel) Aspect Ratio for the filter output video. Note that as a consequence of the application of this filter, the output display aspect ratio will change according to the equation above. Keep in mind that the sample aspect ratio set by the <code class=”code”>setsar</code> filter may be changed by later filters in the filterchain, e.g. if another "setsar" or a "setdar" filter is applied. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>r, ratio, dar (<code class=”code”>setdar</code> only), sar (<code class=”code”>setsar</code> only)</samp></dt> <dd> Set the aspect ratio used by the filter. The parameter can be a floating point number string, or an expression. If the parameter is not specified, the value "0" is assumed, meaning that the same input value is used. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Set the maximum integer value to use for expressing numerator and denominator when reducing the expressed aspect ratio to a rational. Default value is <code class=”code”>100</code>. </dd> </dl>

The parameter <var class=”var”>sar</var> is an expression containing the following constants: <dl class=”table”> <dt><samp class=”option”>w, h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Same as <var class=”var”>w</var> / <var class=”var”>h</var>. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio. It is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub, vsub</samp></dt> <dd> Horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

 <h4 class=”subsection”>30.228.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> To change the display aspect ratio to 16:9, specify one of the following: <pre class=”example-preformatted”>setdar=dar=1.77777 setdar=dar=16/9 </pre>

To change the sample aspect ratio to 10:11, specify: <pre class=”example-preformatted”>setsar=sar=10/11 </pre>

To set a display aspect ratio of 16:9, and specify a maximum integer value of 1000 in the aspect ratio reduction, use the command: <pre class=”example-preformatted”>setdar=ratio=16/9:max=1000 </pre>

 <h3 class=”section”>30.229 setfield# TOC</h3>

Force field for the output video frame. The <code class=”code”>setfield</code> filter marks the interlace type field for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters (e.g. <code class=”code”>fieldorder</code> or <code class=”code”>yadif</code>). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same field property. </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl> </dd> </dl>

 <h3 class=”section”>30.230 setparams# TOC</h3>

Force frame parameter for the output video frame. The <code class=”code”>setparams</code> filter marks interlace and color range for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by filters/encoders. <dl class=”table”> <dt><samp class=”option”>field_mode</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same field property (default). </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl>

</dd> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property (default). </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Mark the frame as unspecified color range. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Mark the frame as limited range. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Mark the frame as full range. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> </dl>

</dd> <dt><samp class="option">color_trc</samp></dt> <dd> Set the color transfer. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color trc property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">log100</samp>’</dt> <dt>‘<samp class="samp">log316</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dt>‘<samp class="samp">bt1361e</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">colorspace</samp></dt> <dd> Set the colorspace. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same col-

orspace property (default). </dd> <dt>‘<samp class="samp">gbr</samp>’</dt>
<dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp
class="samp">unknown</samp>’</dt> <dt>‘<samp
class="samp">fcc</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</d
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dt>‘<samp class="samp">ycgco</samp>’</dt> <dt>‘<samp
class="samp">bt2020nc</samp>’</dt> <dt>‘<samp
class="samp">bt2020c</samp>’</dt> <dt>‘<samp class="samp">smpte2085</samp>&rsq
<dt>‘<samp class="samp">chroma-derived-nc</samp>’</dt>
<dt>‘<samp class="samp">chroma-derived-c</samp>’</dt>
<dt>‘<samp class="samp">ictcp</samp>’</dt> </dl>

</dd> <dt><samp class="option">chroma_location</samp></dt>
<dd> Set the chroma sample location. Available values are: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Keep the same chroma location (default). </dd>
<dt>‘<samp class="samp">unspecified, unknown</samp>’</dt>
<dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp
class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</
<dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp
class="samp">bottomleft</samp>’</dt> <dt>‘<samp
class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> Set
the alpha moda. Available values are: <dl class="table"> <dt>‘<samp
class="samp">auto</samp>’</dt> <dd> Keep the same alpha
mode (default). </dd> <dt>‘<samp class="samp">unspecified,
unknown</samp>’</dt> <dt>‘<samp class="samp">premultiplied</samp>’</dt>
<dt>‘<samp class="samp">straight</samp>’</dt> </dl>
</dd> </dl>

 <h3 class="section">30.231 shear<span class="pull-
right">#
TOC</h3>
Apply shear transform to input video. This filter sup-
ports the following options: <dl class="table"> <dt><samp
class="option">shx</samp></dt> <dd> Shear factor in X-direction.
Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp
class="option">shy</samp></dt> <dd> Shear factor in Y-direction.
Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp
class="option">fillcolor, c</samp></dt> <dd> Set the color used
to fill the output area not covered by the transformed video. For the
general syntax of this option, check the <a data-manual="ffmpeg-utils"
href="./ffmpeg-utils.html#color-syntax">"Color" section in the
ffmpeg-utils manual. If the special value "none" is se-
lected then no background is printed (useful for example if the background

is never shown). Default value is "black". </dd> <dt><samp class="option">interp</samp></dt> <dd> Set interpolation type. Can be <code class="code">bilinear</code> or <code class="code">nearest</code>. Default is <code class="code">bilinear</code>. </dd> </dl>

 <h4 class="subsection">30.231.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.232 showinfo# TOC</h3>

Show a line containing various information for each input video frame. The input video is not modified. This filter supports the following options: <dl class="table"> <dt><samp class="option">checksum</samp></dt> <dd> Calculate checksums of each plane. By default enabled. </dd> <dt><samp class="option">udu_sei_as_ascii</samp></dt> <dd> Try to print user data unregistered SEI as ascii character when possible, in hex format otherwise. </dd> </dl>

The shown line contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var>. The following values are shown in the output: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of time base units. The time base unit depends on the filter input pad. </dd> <dt><samp class="option">pts_time</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of seconds. </dd> <dt><samp class="option">fmt</samp></dt> <dd> The pixel format name. </dd> <dt><samp class="option">sar</samp></dt> <dd> The sample aspect ratio of the input frame, expressed in the form <var class="var">num</var>/<var class="var">den</var>. </dd> <dt><samp class="option">s</samp></dt> <dd> The size of the input frame. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">i</samp></dt> <dd> The type of interlaced mode ("P" for "progressive", "T" for top field first, "B" for bottom field first). </dd> <dt><samp class="option">iskey</samp></dt> <dd> This

is 1 if the frame is a key frame, 0 otherwise. </dd> <dt><samp class=”option”>type</samp></dt> <dd> The picture type of the input frame ("I" for an I-frame, "P" for a P-frame, "B" for a B-frame, or "?" for an unknown type). Also refer to the documentation of the <code class=”code”>AVPictureType</code> enum and of the <code class=”code”>av_get_picture_type_char</code> function defined in <samp class=”file”>libavutil/avutil.h</samp>. </dd> <dt><samp class=”option”>checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of all the planes of the input frame. </dd> <dt><samp class=”option”>plane_checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of each plane of the input frame, expressed in the form "[<var class=”var”>c0</var> <var class=”var”>c1</var> <var class=”var”>c2</var> <var class=”var”>c3</var>]". </dd> <dt><samp class=”option”>mean</samp></dt> <dd> The mean value of pixels in each plane of the input frame, expressed in the form "[<var class=”var”>mean0</var> <var class=”var”>mean1</var> <var class=”var”>mean2</var> <var class=”var”>mean3</var>]". </dd> <dt><samp class=”option”>stdev</samp></dt> <dd> The standard deviation of pixel values in each plane of the input frame, expressed in the form "[<var class=”var”>stdev0</var> <var class=”var”>stdev1</var> <var class=”var”>stdev2</var> <var class=”var”>stdev3</var>]". </dd> </dl>

 <h3 class=”section”>30.233 showpalette# TOC</h3>

Displays the 256 colors palette of each frame. This filter is only relevant for <var class=”var”>pal8</var> pixel format frames. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>s</samp></dt> <dd> Set the size of the box used to represent one palette color entry. Default is <code class=”code”>30</code> (for a <code class=”code”>30x30</code> pixel box). </dd> </dl>

 <h3 class=”section”>30.234 shuffleframes# TOC</h3>

Reorder and/or duplicate and/or drop video frames. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mapping</samp></dt> <dd> Set the destination indexes of input frames. This is space or ’|’ separated list of indexes that maps input frames to output frames. Number of indexes also sets maximal value that each index may have. ’-1’ index have special meaning and that is to drop frame. </dd> </dl>

The first frame has the index 0. The default is to keep the input unchanged. <h4 class="subsection">30.234.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap second and third frame of every three frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=0 2 1" OUTPUT </pre>

Swap 10th and 1st frame of every ten frames of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf "shuffleframes=9 1 2 3 4 5 6 7 8 0" OUTPUT </pre>

 <h3 class="section">30.235 shufflepixels# TOC</h3>

Reorder pixels in video frames. This filter accepts the following options: <dl class="table"> <dt><samp class="option">direction, d</samp></dt> <dd> Set shuffle direction. Can be forward or inverse direction. Default direction is forward. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set shuffle mode. Can be horizontal, vertical or block mode. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set shuffle block_size. In case of horizontal shuffle mode only width part of size is used, and in case of vertical shuffle mode only height part of size is used. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set random seed used with shuffling pixels. Mainly useful to set to be able to reverse filtering process to get original input. For example, to reverse forward shuffle you need to use same parameters and exact same seed and to set direction to inverse. </dd> </dl>

 <h3 class="section">30.236 shuffleplanes# TOC</h3>

Reorder and/or duplicate video planes. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map0</samp></dt> <dd> The index of the input plane to be used as the first output plane. </dd> <dt><samp class="option">map1</samp></dt> <dd> The index of the input plane to be used as the second output plane. </dd> <dt><samp class="option">map2</samp></dt> <dd> The index of the input plane to be used as the third output plane. </dd> <dt><samp class="option">map3</samp></dt>

<dd> The index of the input plane to be used as the fourth output plane. </dd> </dl>

The first plane has the index 0. The default is to keep the input unchanged. <h4 class="subsection">30.236.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap the second and third planes of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf shuffleplanes=0:2:1:3 OUTPUT </pre>

<h3 class="section">30.237 signalstats# TOC</h3>

Evaluate various visual metrics that assist in determining issues associated

with the digitization of analog video media.

By default the filter will log these metadata values:

<dl class="table">

<dt><samp class="option">YMIN</samp></dt>

<dd> Display the minimal Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YLOW</samp></dt>

<dd> Display the Y value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YAVG</samp></dt>

<dd> Display the average Y value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class="option">YHIGH</samp></dt>

<dd> Display the Y value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YMAX</samp></dt>

<dd> Display the maximum Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UMIN</samp></dt>

<dd> Display the minimal U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">ULOW</samp></dt>

<dd> Display the U value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UAVG</samp></dt>

<dd> Display the average U value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>UHIGH</samp></dt>

<dd> Display the U value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>UMAX</samp></dt>

<dd> Display the maximum U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VMIN</samp></dt>

<dd> Display the minimal V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VLOW</samp></dt>

<dd> Display the V value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VAVG</samp></dt>

<dd> Display the average V value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>VHIGH</samp></dt>

<dd> Display the V value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VMAX</samp></dt>

<dd> Display the maximum V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>SATMIN</samp></dt>

<dd> Display the minimal saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATLOW</samp></dt>

<dd> Display the saturation value at the 10% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATAVG</samp></dt>

<dd> Display the average saturation value within the input frame. Expressed in range

of [0-~181.02].

</dd>

<dt><samp class=”option”>SATHIGH</samp></dt>

<dd> Display the saturation value at the 90% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATMAX</samp></dt>

<dd> Display the maximum saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>HUEMED</samp></dt>

<dd> Display the median value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>HUEAVG</samp></dt>

<dd> Display the average value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>YDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the Y

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class=”option”>UDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the U

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class="option">VDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the V

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class="option">YBITDEPTH</samp></dt>

<dd> Display bit depth of Y plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class="option">UBITDEPTH</samp></dt>

<dd> Display bit depth of U plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class="option">VBITDEPTH</samp></dt>

<dd> Display bit depth of V plane in current frame.

Expressed in range of [0-16].

</dd>

</dl>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">stat</samp></dt> <dt><samp class="option">out</samp></dt> <dd> <samp class="option">stat</samp> specify an additional form of image analysis. <samp class="option">out</samp> output video with the specified type of pixel highlighted. Both options accept the following values: <dl class="table"> <dt>‘<samp class="samp">tout</samp>’</dt> <dd> Identify <var class="var">temporal outliers</var> pixels. A <var class="var">temporal outlier</var> is a pixel unlike the neighboring pixels of the same field. Examples of temporal outliers include the results of video dropouts,

head clogs, or tape tracking issues. </dd> <dt>‘<samp class=”samp”>vrep</samp>’</dt> <dd> Identify <var class=”var”>vertical line repetition</var>. Vertical line repetition includes similar rows of pixels within a frame. In born-digital video vertical line repetition is common, but this pattern is uncommon in video digitized from an analog source. When it occurs in video that results from the digitization of an analog source it can indicate concealment from a dropout compensator. </dd> <dt>‘<samp class=”samp”>brng</samp>’</dt> <dd> Identify pixels that fall outside of legal broadcast range. </dd> </dl>

</dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Set the highlight color for the <samp class=”option”>out</samp> option. The default color is yellow. </dd> </dl>

 <h4 class=”subsection”>30.237.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Output data of various video metrics: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats="stat=tout+vrep+brng" -show_frames </pre>

Output specific data about the minimum and maximum values of the Y plane per frame: <pre class=”example-preformatted”>ffprobe -f lavfi movie=example.mov,signalstats -show_entries frame_tags=lavfi.signalstats.YMAX,lavfi.signalstats.YMIN </pre>

Playback video while highlighting pixels that are outside of broadcast range in red. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats="out=brng:color=red" </pre>

Playback video with signalstats metadata drawn over the frame. <pre class=”example-preformatted”>ffplay example.mov -vf signalstats=stat=brng+vrep+tout,drawtext=fontfile=FreeSerif.ttf:textfile=signalstat_drawtext.txt </pre>

The contents of signalstat_drawtext.txt used in the command are: <pre class=”example-preformatted”>time %{pts:hms} Y (%{metadata:lavfi.signalstats.YMIN}-%{metadata:lavfi.signalstats.YMAX})
U (%{metadata:lavfi.signalstats.UMIN}-%{metadata:lavfi.signalstats.UMAX})
V (%{metadata:lavfi.signalstats.VMIN}-%{metadata:lavfi.signalstats.VMAX})
saturation maximum: %{metadata:lavfi.signalstats.SATMAX}

</pre>

 <h3

class=”section”>30.238 signature# TOC</h3>

Calculates the MPEG-7 Video Signature. The filter can handle more than one input. In this case the matching between the inputs can be calculated additionally. The filter always passes through the first input. The signature of each stream can be written into a file. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>detectmode</samp></dt> <dd> Enable or disable the matching process. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>off</samp>’</dt> <dd> Disable the calculation of a matching (default). </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Calculate the matching for the whole video and output whether the whole video matches or only parts. </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt> <dd> Calculate only until a matching is found or the video ends. Should be faster in some cases. </dd> </dl>

</dd> <dt><samp class=”option”>nb_inputs</samp></dt> <dd> Set the number of inputs. The option value must be a non negative integer. Default value is 1. </dd> <dt><samp class=”option”>filename</samp></dt> <dd> Set the path to which the output is written. If there is more than one input, the path must be a prototype, i.e. must contain %d or %0nd (where n is a positive integer), that will be replaced with the input number. If no filename is specified, no output will be written. This is the default. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Choose the output format. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>binary</samp>’</dt> <dd> Use the specified binary representation (default). </dd> <dt>‘<samp class=”samp”>xml</samp>’</dt> <dd> Use the specified xml representation. </dd> </dl>

</dd> <dt><samp class=”option”>th_d</samp></dt> <dd> Set threshold to detect one word as similar. The option value must be an integer greater than zero. The default value is 9000. </dd> <dt><samp class=”option”>th_dc</samp></dt> <dd> Set threshold to detect all words as similar. The option value must be an integer greater than zero. The default value is 60000. </dd> <dt><samp class=”option”>th_xh</samp></dt> <dd> Set threshold to detect frames as similar. The option value must be an integer greater than zero. The default value is 116. </dd> <dt><samp class=”option”>th_di</samp></dt> <dd> Set the minimum length of a sequence in frames to recognize it as matching sequence. The option value must be a non negative integer value. The default value is 0. </dd> <dt><samp class=”option”>th_it</samp></dt> <dd> Set the minimum relation, that matching frames to all frames must have. The

option value must be a double value between 0 and 1. The default value is 0.5. </dd> </dl>

 <h4 class="subsection">30.238.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> To calculate the signature of an input video and store it in signature.bin: <pre class="example-preformatted">ffmpeg -i input.mkv -vf signature=filename=signature.bin -map 0:v -f null - </pre>

To detect whether two videos match and store the signatures in XML format in signature0.xml and signature1.xml: <pre class="example-preformatted">ffmpeg -i input1.mkv -i input2.mkv -filter_complex "[0:v][1:v] signature=nb_inputs=2:detectmode=full:format=xml:filename=signature%d.x -map :v -f null - </pre>

 <h3 class="section">30.239 siti# TOC</h3>

Calculate Spatial Information (SI) and Temporal Information (TI) scores for a video, as defined in ITU-T Rec. P.910 (11/21): Subjective video quality assessment methods for multimedia applications. Available PDF at https://www.itu.int/rec/T-REC-P.910-202111-S/en. Note that this is a legacy implementation that corresponds to a superseded recommendation. Refer to ITU-T Rec. P.910 (07/22) for the latest version: https://www.itu.int/rec/T-REC-P.910-202207-I/en It accepts the following option: <dl class="table"> <dt><samp class="option">print_summary</samp></dt> <dd> If set to 1, Summary statistics will be printed to the console. Default 0. </dd> </dl>

 <h4 class="subsection">30.239.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> To calculate SI/TI metrics and print summary: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf siti=print_summary=1 -f null - </pre>

 <h3 class="section">30.240 smartblur<a class="anchor

hidden-xs" href="#smartblur" aria-hidden="true"># TOC</h3>

Blur the input video without impacting the outlines. It accepts the following options: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dd> Set the luma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is 1.0. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set the luma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is 1.0. </dd> <dt><samp class="option">luma_threshold, lt</samp></dt> <dd> Set the luma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is 0. </dd> <dt><samp class="option">chroma_radius, cr</samp></dt> <dd> Set the chroma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set the chroma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">chroma_threshold, ct</samp></dt> <dd> Set the chroma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set the alpha radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">alpha_strength, as</samp></dt> <dd> Set the alpha strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">alpha_threshold, at</samp></dt> <dd> Set the alpha threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30]

will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> </dl>

If a chroma or alpha option is not explicitly set, the corresponding luma value is set. <h3 class="section">30.241 sobel# TOC</h3> Apply sobel operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">30.241.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">30.242 spp# TOC</h3>

Apply a simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">6</code> - all) shifts and average the results. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-6. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">6</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding (default). </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> </dl>

</dd> <dt><samp class="option">use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class="code">1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class="code">0</code> (not enabled). </dd> </dl>

 <h4 class="subsection">30.242.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">quality, level</samp></dt> <dd> Set quality level. The value <code class="code">max</code> can be used to set the maximum level, currently <code class="code">6</code>. </dd> </dl>

 <h3 class="section">30.243 sr# TOC</h3>

Scale the input by applying one of the super-resolution methods based on convolutional neural networks. Supported models: <ul class="itemize mark-bullet"> Super-Resolution Convolutional Neural Network model (SRCNN). See https://arxiv.org/abs/1501.00092.

Efficient Sub-Pixel Convolutional Neural Network model (ESPCN). See https://arxiv.org/abs/1609.05158.

Training scripts as well as scripts for model file (.pb) saving can be found at <a class="url" href="https://github.com/XueweiMeng/sr/tree/sr_dnn_native">https://github.com/Xueweil Original repository is at <a class="url" href="https://github.com/HighVoltageRocknRoll/sr.git">https://gith The filter accepts the following options: <dl class="table"> <dt><samp class="option">dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class="url" href="https://www.tensorflow.org/install/lang_c">https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class="code">--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO backend can load files for only its format. </dd> <dt><samp class="option">scale_factor</samp></dt> <dd> Set scale factor for

SRCNN model. Allowed values are <code class=”code”>2</code>, <code class=”code”>3</code> and <code class=”code”>4</code>. Default value is <code class=”code”>2</code>. Scale factor is necessary for SRCNN model, because it accepts input upscaled using bicubic upscaling with proper scale factor. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>30.244 sr_amf# TOC</h3>

Upscale (size increasing) for the input video using AMD Advanced Media Framework library for hardware acceleration. Use advanced algorithms for upscaling with higher output quality. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>algorithm</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> <dt><var class=”var”>sr1-0</var></dt> <dd> Video SR1.0 This is a default value </dd> <dt><var class=”var”>point</var></dt> <dd> Point AMF returns an unwritten surface for this algorithm when the format is <code class=”code”>nv12</code> or <code class=”code”>p010</code>, so inputs in those formats are converted to a packed RGB format automatically. This is a driver bug rather than a documented restriction; the conversion can be dropped once a driver that scales <code class=”code”>nv12</code> and <code class=”code”>p010</code> correctly with <code class=”code”>point</code> is in wide use. </dd> <dt><var class=”var”>sr1-1</var></dt> <dd> Video SR1.1 This algorithm only supports packed RGB formats and requires a DirectX 11 or DirectX 12 device, so it is available on Windows only. Inputs in other formats such as <code class=”code”>nv12</code> or <code class=”code”>p010</code> are converted to a packed RGB format automatically. Note that the VideoSR algorithms (<code class=”code”>sr1-0</code> and <code class=”code”>sr1-1</code>) do not preserve the alpha channel (it is left zeroed); drop it after the download, e.g. with <code class=”code”>hwdownload,format=rgba,format=rgb24</code>, if an opaque result is required. </dd> </dl>

</dd> <dt><samp class=”option”>sharpness</samp></dt> <dd> Control

hq scaler sharpening. The value is a float in the range of [0.0, 2.0] </dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. The <code class="code">point</code> and <code class="code">sr1-1</code> algorithms emit the packed RGB format the scaler receives, so for them this option must name one of <code class="code">rgba</code>, <code class="code">bgra</code>, <code class="code">x2bgr10le</code> or <code class="code">rgbaf16le</code>. A named format also decides what software input is converted to, and what a hardware surface in another format is converted to on the GPU. Without it, <code class="code">x2bgr10le</code> is selected for <code class="code">p010</code> input and <code class="code">rgba</code> for other non-RGB input, and the result is tagged as full range RGB. libswscale cannot convert other software formats to <code class="code">rgbaf16le</code>; use software input already in that format or a hardware input for GPU conversion. </dd> <dt><samp class="option">keep-ratio</samp></dt> <dd> Force the scaler to keep the aspect ratio of the input image when the output size has a different aspect ratio. Default value is false. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specifies whether the output image outside the region of interest, which does not fill the entire output surface should be filled with a solid color. </dd> </dl>

 <h4 class="subsection">30.244.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Upscale to 720p, keeping aspect ratio, and read the result back into system memory. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class="example-preformatted">sr_amf=-2:720,hwdownload,format=nv12
</pre>

Upscale to 4K with algorithm video SR1.1. <pre class="example-preformatted">sr_amf=4096:2160:algorithm=sr1-1
</pre>

 <h3 class="section">30.245 ssim#
TOC</h3>

Obtain the SSIM (Structural SImilarity Metric) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the SSIM. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have

the same number of frames, which are compared one by one. The filter stores the calculated SSIM of each frame. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the SSIM of each individual frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. A description of each shown parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">Y, U, V, R, G, B</samp></dt> <dd> SSIM of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">All</samp></dt> <dd> SSIM of the compared frames for the whole frame. </dd> <dt><samp class="option">dB</samp></dt> <dd> Same as above but in dB representation. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">30.245.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] ssim="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The SSIM of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with both psnr and ssim at same time: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi "ssim;[0:v][1:v]psnr" -f null - </pre>

Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]ssim" -f null - </pre>

 <h3 class="section">30.246 stereo3d# TOC</h3>

Convert between different stereoscopic image formats. The filters accept the following options: <dl class="table"> <dt><samp

class=”option”>in</samp></dt> <dd> Set stereoscopic image format of input. Available values for input image formats are: <dl class=”table”> <dt>‘<samp class=”samp”>sbsl</samp>’</dt> <dd> side by side parallel (left eye left, right eye right) </dd> <dt>‘<samp class=”samp”>sbsr</samp>’</dt> <dd> side by side crosseye (right eye left, left eye right) </dd> <dt>‘<samp class=”samp”>sbs2l</samp>’</dt> <dd> side by side parallel with half width resolution (left eye left, right eye right) </dd> <dt>‘<samp class=”samp”>sbs2r</samp>’</dt> <dd> side by side crosseye with half width resolution (right eye left, left eye right) </dd> <dt>‘<samp class=”samp”>abl</samp>’</dt> <dt>‘<samp class=”samp”>tbl</samp>’</dt> <dd> above-below (left eye above, right eye below) </dd> <dt>‘<samp class=”samp”>abr</samp>’</dt> <dt>‘<samp class=”samp”>tbr</samp>’</dt> <dd> above-below (right eye above, left eye below) </dd> <dt>‘<samp class=”samp”>ab2l</samp>’</dt> <dt>‘<samp class=”samp”>tb2l</samp>’</dt> <dd> above-below with half height resolution (left eye above, right eye below) </dd> <dt>‘<samp class=”samp”>ab2r</samp>’</dt> <dt>‘<samp class=”samp”>tb2r</samp>’</dt> <dd> above-below with half height resolution (right eye above, left eye below) </dd> <dt>‘<samp class=”samp”>al</samp>’</dt> <dd> alternating frames (left eye first, right eye second) </dd> <dt>‘<samp class=”samp”>ar</samp>’</dt> <dd> alternating frames (right eye first, left eye second) </dd> <dt>‘<samp class=”samp”>irl</samp>’</dt> <dd> interleaved rows (left eye has top row, right eye starts on next row) </dd> <dt>‘<samp class=”samp”>irr</samp>’</dt> <dd> interleaved rows (right eye has top row, left eye starts on next row) </dd> <dt>‘<samp class=”samp”>icl</samp>’</dt> <dd> interleaved columns, left eye first </dd> <dt>‘<samp class=”samp”>icr</samp>’</dt> <dd> interleaved columns, right eye first Default value is ‘<samp class=”samp”>sbsl</samp>’. </dd> </dl>

</dd>

<dt><samp class=”option”>out</samp></dt>

<dd> Set stereoscopic image format of output.

<dl class=”table”>

<dt>‘<samp class=”samp”>sbsl</samp>’</dt>

<dd> side by side parallel (left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbsr</samp>’</dt>
<dd> side by side crosseye (right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">sbs2l</samp>’</dt>
<dd> side by side parallel with half width resolution
(left eye left, right eye right)

</dd>
<dt>‘<samp class="samp">sbs2r</samp>’</dt>
<dd> side by side crosseye with half width resolution
(right eye left, left eye right)

</dd>
<dt>‘<samp class="samp">abl</samp>’</dt>
<dt>‘<samp class="samp">tbl</samp>’</dt>
<dd> above-below (left eye above, right eye below)

</dd>
<dt>‘<samp class="samp">abr</samp>’</dt>
<dt>‘<samp class="samp">tbr</samp>’</dt>
<dd> above-below (right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">ab2l</samp>’</dt>
<dt>‘<samp class="samp">tb2l</samp>’</dt>
<dd> above-below with half height resolution
(left eye above, right eye below)

</dd>

<dt>‘<samp class="samp">ab2r</samp>’</dt>
<dt>‘<samp class="samp">tb2r</samp>’</dt>
<dd> above-below with half height resolution
(right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">al</samp>’</dt>
<dd> alternating frames (left eye first, right eye second)

</dd>
<dt>‘<samp class="samp">ar</samp>’</dt>
<dd> alternating frames (right eye first, left eye second)

</dd>
<dt>‘<samp class="samp">irl</samp>’</dt>
<dd> interleaved rows (left eye has top row, right eye starts on next row)

</dd>
<dt>‘<samp class="samp">irr</samp>’</dt>
<dd> interleaved rows (right eye has top row, left eye starts on next row)

</dd>
<dt>‘<samp class="samp">arbg</samp>’</dt>
<dd> anaglyph red/blue gray
(red filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class="samp">argg</samp>’</dt>
<dd> anaglyph red/green gray
(red filter on left eye, green filter on right eye)

</dd>

<dt>‘<samp class="samp">arcg</samp>’</dt>

<dd> anaglyph red/cyan gray

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arch</samp>’</dt>

<dd> anaglyph red/cyan half colored

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcc</samp>’</dt>

<dd> anaglyph red/cyan color

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcd</samp>’</dt>

<dd> anaglyph red/cyan color optimized with the least squares projection of dubois

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">agmg</samp>’</dt>

<dd> anaglyph green/magenta gray

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmh</samp>’</dt>

<dd> anaglyph green/magenta half colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmc</samp>’</dt>

<dd> anaglyph green/magenta colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmd</samp>’</dt>

<dd> anaglyph green/magenta color optimized with the least squares projection of dubois

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">aybg</samp>’</dt>

<dd> anaglyph yellow/blue gray

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybh</samp>’</dt>

<dd> anaglyph yellow/blue half colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybc</samp>’</dt>

<dd> anaglyph yellow/blue colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybd</samp>’</dt>

<dd> anaglyph yellow/blue color optimized with the least squares projection of dubois

(yellow filter on left eye, blue filter on right eye)

</dd>
<dt>‘<samp class=”samp”>ml</samp>’</dt>
<dd> mono output (left eye only)

</dd>
<dt>‘<samp class=”samp”>mr</samp>’</dt>
<dd> mono output (right eye only)

</dd>
<dt>‘<samp class=”samp”>chl</samp>’</dt>
<dd> checkerboard, left eye first

</dd>
<dt>‘<samp class=”samp”>chr</samp>’</dt>
<dd> checkerboard, right eye first

</dd>
<dt>‘<samp class=”samp”>icl</samp>’</dt>
<dd> interleaved columns, left eye first

</dd>
<dt>‘<samp class=”samp”>icr</samp>’</dt>
<dd> interleaved columns, right eye first

</dd>
<dt>‘<samp class=”samp”>hdmi</samp>’</dt>
<dd> HDMI frame pack
</dd>
</dl>

Default value is ‘<samp class="samp">arcd</samp>’.
</dd> </dl>

 <h4 class="subsection">30.246.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert input video from side by side parallel to anaglyph yellow/blue dubois: <pre class="example-preformatted">stereo3d=sbsl:aybd </pre>

Convert input video from above below (left eye above, right eye below) to side by side crosseye. <pre class="example-preformatted">stereo3d=abl:sbsr </pre>

 <h3 class="section">30.247 streamselect, astreamselect# TOC</h3> Select video or audio streams. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 2. </dd> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">30.247.1 Commands# TOC</h4>

The <code class="code">streamselect</code> and <code class="code">astreamselect</code> filter supports the following commands: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">30.247.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select first 5 seconds 1st stream and rest of time 2nd stream: <pre class="example-preformatted">sendcmd='5.0 streamselect map 1',streamselect=inputs=2:map=0
</pre>

Same as above, but for audio: <pre class="example-preformatted">asendcmd='5.0 astreamselect map 1',astreamselect=inputs=2:map=0 </pre>

 <h3 class=”section”>30.248 subtitles# TOC</h3>

Draw subtitles on top of input video using the libass library. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libass</code>. This filter also requires a build with libavcodec and libavformat to convert the passed subtitles file to ASS (Advanced Substation Alpha) subtitles format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filename, f</samp></dt> <dd> Set the filename of the subtitle file to read. It must be specified. </dd> <dt><samp class=”option”>original_size</samp></dt> <dd> Specify the size of the original video, the video for which the ASS file was composed. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Due to a misdesign in ASS aspect ratio arithmetic, this is necessary to correctly scale the fonts if the aspect ratio has been changed. </dd> <dt><samp class=”option”>fontsdir</samp></dt> <dd> Set a directory path containing fonts that can be used by the filter. These fonts will be used in addition to whatever the font provider uses. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Process alpha channel, by default alpha channel is untouched. </dd> <dt><samp class=”option”>charenc</samp></dt> <dd> Set subtitles input character encoding. <code class=”code”>subtitles</code> filter only. Only useful if not UTF-8. </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Set subtitles stream index. <code class=”code”>subtitles</code> filter only. </dd> <dt><samp class=”option”>force_style</samp></dt> <dd> Override default style or script info parameters of the subtitles. It accepts a string containing ASS style format <code class=”code”>KEY=VALUE</code> couples separated by ",". </dd> <dt><samp class=”option”>wrap_unicode</samp></dt> <dd> Break lines according to the Unicode Line Breaking Algorithm. Availability requires at least libass release 0.17.0 (or LIBASS_VERSION 0x01600010), and libass must have been built with libunibreak. The option is enabled by default except for native ASS. </dd> <dt><samp class=”option”>shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew,

Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd>
</dl>

The default is <code class="code">auto</code>. </dd> </dl>

If the first key is not specified, it is assumed that the first value specifies the <samp class="option">filename</samp>. For example, to render the file <samp class="file">sub.srt</samp> on top of the input video, use the command: <pre class="example-preformatted">subtitles=sub.srt </pre>

which is equivalent to: <pre class="example-preformatted">subtitles=filename=sub.srt </pre>

To render the default subtitles stream from file <samp class="file">video.mkv</samp>, use: <pre class="example-preformatted">subtitles=video.mkv
</pre>

To render the second subtitles stream from that file, use: <pre class="example-preformatted">subtitles=video.mkv:si=1
</pre>

To make the subtitles stream from <samp class="file">sub.srt</samp> appear in 80% transparent blue <code class="code">DejaVu Serif</code>, use: <pre class="example-preformatted">subtitles=sub.srt:force_style='Fontn
Serif,PrimaryColour=&HCCFF0000' </pre>

 <h3 class="section">30.249 super2xsai# TOC</h3>

Scale the input by 2x and smooth using the Super2xSaI (Scale and Interpolate) pixel art scaling algorithm. Useful for enlarging pixel art images without reducing sharpness. <h3 class="section">30.250 swaprect# TOC</h3>

Swap two rectangular objects in video. This filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dd> Set object width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set object height. </dd> <dt><samp class="option">x1</samp></dt> <dd> Set 1st rect x coordinate. </dd> <dt><samp class="option">y1</samp></dt> <dd> Set 1st rect y coordinate. </dd> <dt><samp class="option">x2</samp></dt> <dd> Set 2nd rect x coordinate. </dd> <dt><samp class="option">y2</samp></dt> <dd> Set 2nd rect y coordinate. All expressions are evaluated once for each frame. </dd> </dl>

The all options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>w</var> / <var class=”var”>h</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> </dl>

 <h4 class=”subsection”>30.250.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.251 swapuv# TOC</h3> Swap U & V plane. <h3 class=”section”>30.252 tblend# TOC</h3> Blend successive video frames. See blend <h3 class=”section”>30.253 telecine# TOC</h3>

Apply telecine process to the video. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> </dl>

 <pre class=”example-preformatted”>Some typical patterns:

NTSC output (30i): 27.5p: 32222 24p: 23 (classic) 24p: 2332 (preferred) 20p: 33 18p: 334 16p: 3444

PAL output (25i): 27.5p: 12222 24p: 222222222223 ("Euro pulldown") 16.67p: 33 16p: 33333334 </pre>

 <h3 class=”section”>30.254 thistogram# TOC</h3>

Compute and draw a color distribution histogram for the input video across time. Unlike histogram video filter which only shows histogram of single input frame at certain time, this filter shows also past histograms of number of frames defined by <code class=”code”>width</code> option. The computed histogram is a representation of the color component distribution in an image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dd> Set width of single color component output. Default value is <code class=”code”>0</code>. Value of <code class=”code”>0</code> means width will be picked from input video. This also set number of passed histograms to keep. Allowed range is [0, 8192]. </dd> <dt><samp class=”option”>display_mode, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class=”samp”>parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dd> Presents information identical to that in the <code class=”code”>parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class=”code”>stack</code>. </dd> <dt><samp class=”option”>levels_mode, m</samp></dt> <dd> Set mode. Can be either <code class=”code”>linear</code>, or <code class=”code”>logarithmic</code>. Default is <code class=”code”>linear</code>. </dd> <dt><samp class=”option”>components, c</samp></dt> <dd> Set what color components to display. Default is <code class=”code”>7</code>. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. Default is <code class=”code”>0.9</code>. </dd> <dt><samp class=”option”>envelope, e</samp></dt> <dd> Show envelope. Default is disabled. </dd> <dt><samp class=”option”>ecolor, ec</samp></dt> <dd> Set envelope color. Default is <code class=”code”>gold</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set

slide mode. Available values for slide is: <dl class="table">
<dt>‘<samp class="samp">frame</samp>’</dt> <dd> Draw
new frame when right border is reached. </dd> <dt>‘<samp
class="samp">replace</samp>’</dt> <dd> Replace old columns
with new ones. </dd> <dt>‘<samp class="samp">scroll</samp>’</dt>
<dd> Scroll from right to left. </dd> <dt>‘<samp
class="samp">rscroll</samp>’</dt> <dd> Scroll from left to
right. </dd> <dt>‘<samp class="samp">picture</samp>’</dt>
<dd> Draw single picture. </dd> </dl>

Default is <code class="code">replace</code>. </dd> </dl>

 <h3 class="section">30.255 threshold<span
class="pull-right"><a class="anchor hidden-xs" href="#threshold" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-threshold"
aria-hidden="true">TOC</h3>

Apply threshold effect to video stream. This filter needs
four video streams to perform thresholding. First stream is stream we are
filtering. Second stream is holding threshold values, third stream is holding
min values, and last, fourth stream is holding max values. The
filter accepts the following option: <dl class="table"> <dt><samp
class="option">planes</samp></dt> <dd> Set which planes will be
processed, unprocessed planes will be copied. By default value 0xf, all planes
will be processed. </dd> </dl>

For example if first stream pixel’s component value is less
then threshold value of pixel component from 2nd threshold stream, third
stream value will picked, otherwise fourth stream pixel component value
will be picked. Using color source filter one can perform
various types of thresholding:
<h4 class="subsection">30.255.1 Commands<a
class="anchor hidden-xs" href="#Commands-147" aria-hidden="true">#
TOC</h4>

This filter supports the all options as commands.
  <h4 class="subsection">30.255.2 Ex-
amples<a class="anchor hidden-xs" href="#Examples-
149" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-149" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Binary threshold, using gray color as
threshold: <pre class="example-preformatted">ffmpeg
-i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=black -f lavfi -i color=white
-lavfi threshold output.avi </pre>

Inverted binary threshold, using gray color as threshold: <div
class="example"> <pre class="example-preformatted">ffmpeg -i 320x240.avi
-f lavfi -i color=gray -f lavfi -i color=white -f lavfi -i color=black -lavfi threshold

output.avi </pre>

Truncate binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=gray -lavfi threshold output.avi </pre>

Threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=white -i 320x240.avi -lavfi threshold output.avi </pre>

Inverted threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=white -lavfi threshold output.avi </pre>

 <h3 class="section">30.256 thumbnail# TOC</h3> Select the most representative frame in a given sequence of consecutive frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> <dt><samp class="option">log</samp></dt> <dd> Set the log level to display picked frame stats. Default is <code class="code">info</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">30.256.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract one picture each 50 frames: <pre class="example-preformatted">thumbnail=50 </pre>

Complete example of a thumbnail creation with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf thumbnail,scale=300:200 -frames:v 1 out.png </pre>

 <h3 class="section">30.257 tile# TOC</h3>

Tile several successive frames together. The untile filter can do the reverse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size in the form <code class="code">COLUMNSxROWS</code>. Range is up to UINT_MAX cells. Default is <code class="code">6x5</code>. </dd> <dt><samp class="option">nb_frames</samp></dt> <dd> Set the maximum number of frames to render in the given area. It must be less than or equal to <var class="var">w</var>x<var class="var">h</var>. The default value is <code class="code">0</code>, meaning all the area will be used. </dd> <dt><samp class="option">margin</samp></dt> <dd> Set the outer border margin in pixels. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">padding</samp></dt> <dd> Set the inner border thickness (i.e. the number of pixels between frames). For more advanced padding options (such as having different values for the edges), refer to the pad video filter. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the unused area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set the number of frames to overlap when tiling several successive frames together. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> <dt><samp class="option">init_padding</samp></dt> <dd> Set the number of frames to initially be empty before displaying first output frame. This controls how soon will one get first output frame. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">30.257.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce 8x8 PNG tiles of all keyframes (<samp class="option">-skip_frame nokey</samp>) in a movie: <pre class="example-preformatted">ffmpeg -skip_frame nokey -i file.avi -vf 'scale=128:72,tile=8x8' -an -fps_mode passthrough keyframes%03d.png </pre> The <samp class="option">-fps_mode passthrough</samp> is necessary to prevent <code class="command">ffmpeg</code> from duplicating each output frame to accommodate the originally detected frame rate. Display <code class="code">5</code> pictures in an area of <code class="code">3x2</code> frames, with <code

class=”code”>7</code> pixels between them, and <code class=”code”>2</code> pixels of initial margin, using mixed flat and named options: <pre class=”example-preformatted”>tile=3x2:nb_frames=5:padding=7:margin=2 </pre>

 <h3 class=”section”>30.258 tiltandshift# TOC</h3> Apply tilt-and-shift effect. What happens when you invert time and space? Normally a video is composed of several frames that represent a different instant of time and shows a scene that evolves in the space captured by the frame. This filter is the antipode of that concept, taking inspiration from tilt and shift photography. A filtered frame contains the whole timeline of events composing the sequence, and this is obtained by placing a slice of pixels from each frame into a single one. However, since there are no infinite-width frames, this is done up the width of the input frame, and a video is recomposed by shifting away one column for each subsequent frame. In order to map space to time, the filter tilts each input frame as well, so that motion is preserved. This is accomplished by progressively selecting a different column from each input frame. The end result is a sort of inverted parallax, so that far away objects move much faster that the ones in the front. The ideal conditions for this video effect are when there is either very little motion and the background is static, or when there is a lot of motion and a very wide depth of field (e.g. wide panorama, while moving on a train). The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tilt</samp></dt> <dd> Tilt video while shifting (default). When unset, video will be sliding a static image, composed of the first column of each frame. </dd> <dt><samp class=”option”>start</samp></dt> <dd> What to do at the start of filtering (see below). </dd> <dt><samp class=”option”>end</samp></dt> <dd> What to do at the end of filtering (see below). </dd> <dt><samp class=”option”>hold</samp></dt> <dd> How many columns should pass through before start of filtering. </dd> <dt><samp class=”option”>pad</samp></dt> <dd> How many columns should be inserted before end of filtering. </dd> </dl>

Normally the filter shifts and tilts from the very first frame, and stops when the last one is received. However, before filtering starts, normal video may be preserved, so that the effect is slowly shifted in its place. Similarly, the last video frame may be reconstructed at the end. Alternatively it is possible to just start and end with black. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Filtering starts immediately and ends when the last frame is received. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> The first frames or the very last frame are kept intact during processing.

</dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Black is padded at the beginning or at the end of filtering. </dd> </dl>

 <h3 class="section">30.259 tinterlace# TOC</h3>

Perform various types of temporal field interlacing. Frames are counted starting from 1, so the first input frame is considered odd. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Specify the mode of the interlacing. This option can also be specified as a value alone. See below for a list of values for this option. Available values are: <dl class="table"> <dt>‘<samp class="samp">merge, 0</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at half frame rate. <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">drop_even, 1</samp>’</dt> <dd> Only output odd frames, even frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 11111 33333 11111 33333 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">drop_odd, 2</samp>’</dt> <dd> Only output even frames, odd frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 22222 44444 22222 44444 22222 44444 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">pad, 3</samp>’</dt> <dd> Expand each frame to full height, but pad alternate lines with black,

generating a frame with double height at the same input frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_top, 4</samp>’</dt> <dd> Interleave the upper field from odd frames with the lower field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<-

Output: 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">interleave_bottom, 5</samp>’</dt> <dd> Interleave the lower field from odd frames with the upper field from even frames, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444

Output: 22222 44444 11111 33333 22222 44444 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">interlacex2, 6</samp>’</dt> <dd> Double frame rate with unchanged height. Frames are inserted each containing the second temporal field from the previous input frame and the first temporal field from the next input frame. This mode relies on the top_field_first flag. Useful for interlaced video displays with no field synchronisation.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 </pre>

</dd> <dt>‘<samp class="samp">mergex2, 7</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at same frame rate.

<pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 </pre>

</dd> </dl>

Numeric values are deprecated but are accepted for backward compatibility reasons. Default mode is <code class=”code”>merge</code>. </dd> <dt><samp class=”option”>flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class=”var”>flags</var> is: <dl class=”table”> <dt><samp class=”option”>low_pass_filter, vlpf</samp></dt> <dd> Enable linear vertical low-pass filtering in the filter. Vertical low-pass filtering is required when creating an interlaced destination from a progressive source which contains high-frequency vertical detail. Filtering will reduce interlace ’twitter’ and Moire patterning. </dd> <dt><samp class=”option”>complex_filter, cvlpf</samp></dt> <dd> Enable complex vertical low-pass filtering. This will slightly less reduce interlace ’twitter’ and Moire patterning but better retain detail and subjective sharpness impression. </dd> <dt><samp class=”option”>bypass_il</samp></dt> <dd> Bypass already interlaced frames, only adjust the frame rate. </dd> </dl>

Vertical low-pass filtering and bypassing already interlaced frames can only be enabled for <samp class=”option”>mode</samp> <var class=”var”>interleave_top</var> and <var class=”var”>interleave_bottom</var>. </dd> </dl>

 <h3 class=”section”>30.260 tmedian# TOC</h3> Pick median pixels from several successive input video frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set radius of median filter. Default is 1. Allowed range is from 1 to 127. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>15</code>, by which all planes are processed. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Set median percentile. Default value is <code class=”code”>0.5</code>. Default value of <code class=”code”>0.5</code> will pick always median values, while <code class=”code”>0</code> will pick minimum values, and <code class=”code”>1</code> maximum values. </dd> </dl>

 <h4 class="subsection">30.260.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">radius</code>. <h3 class="section">30.261 tmidequalizer# TOC</h3>

Apply Temporal Midway Video Equalization effect. Midway Video Equalization adjusts a sequence of video frames to have the same histograms, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a video frames sequence. This filter accepts the following option: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set filtering radius. Default is <code class="code">5</code>. Allowed range is from 1 to 127. </dd> <dt><samp class="option">sigma</samp></dt> <dd> Set filtering sigma. Default is <code class="code">0.5</code>. This controls strength of filtering. Setting this option to 0 effectively does nothing. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> </dl>

 <h3 class="section">30.262 tmix# TOC</h3>

Mix successive video frames. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">frames</samp></dt> <dd> The number of successive frames to mix. If unspecified, it defaults to 3. </dd> <dt><samp class="option">weights</samp></dt> <dd> Specify weight of each input video frame. Each weight is separated by space. If number of weights is smaller than number of <var class="var">frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">30.262.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Average 7 successive frames: <pre class="example-preformatted">tmix=frames=7:weights="1 1 1 1 1 1 1" </pre>

Apply simple temporal convolution: <pre class="example-preformatted">tmix=frames=3:weights="-1 3 -1" </pre>

Similar as above but only showing temporal differences: <pre class="example-preformatted">tmix=frames=3:weights="-1 2 -1":scale=1 </pre>

 <h4 class="subsection">30.262.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">30.263 tonemap# TOC</h3> Tone map colors from different dynamic ranges. This filter expects data in single precision floating point, as it needs to operate on (and can output) out-of-range values. Another filter, such as zscale, is needed to convert the resulting frame to a usable format. The tonemapping algorithms implemented only work on linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,tonemap=clip,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h4 class="subsection">30.263.1 Options# TOC</h4> The filter accepts the following options. <dl class="table"> <dt><samp class="option">tonemap</samp></dt> <dd> Set the tone map algorithm to use. Possible values are: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Do not apply any tone map, only desaturate overbright pixels. </dd> <dt><var class="var">clip</var></dt> <dd> Hard-clip any out-of-range values. Use it for perfect color accuracy for in-range values, while distorting out-of-range values. </dd> <dt><var class="var">linear</var></dt>

<dd> Stretch the entire reference gamut to a linear multiple of the display. </dd> <dt><var class="var">gamma</var></dt> <dd> Fit a logarithmic transfer between the tone curves. </dd> <dt><var class="var">reinhard</var></dt> <dd> Preserve overall image brightness with a simple curve, using nonlinear contrast, which results in flattening details and degrading color accuracy. </dd> <dt><var class="var">hable</var></dt> <dd> Preserve both dark and bright details better than <var class="var">reinhard</var>, at the cost of slightly darkening everything. Use it when detail preservation is more important than color and brightness accuracy. </dd> <dt><var class="var">mobius</var></dt> <dd> Smoothly map out-of-range values, while retaining contrast and colors for in-range material as much as possible. Use it when color accuracy is more important than detail preservation. </dd> </dl>

Default is none. </dd> <dt><samp class="option">param</samp></dt> <dd> Tune the tone mapping algorithm. This affects the following algorithms: <dl class="table"> <dt><var class="var">none</var></dt> <dd> Ignored. </dd> <dt><var class="var">linear</var></dt> <dd> Specifies the scale factor to use while stretching. Default to 1.0. </dd> <dt><var class="var">gamma</var></dt> <dd> Specifies the exponent of the function. Default to 1.8. </dd> <dt><var class="var">clip</var></dt> <dd> Specify an extra linear coefficient to multiply into the signal before clipping. Default to 1.0. </dd> <dt><var class="var">reinhard</var></dt> <dd> Specify the local contrast coefficient at the display peak. Default to 0.5, which means that in-gamut values will be about half as bright as when clipping. </dd> <dt><var class="var">hable</var></dt> <dd> Ignored. </dd> <dt><var class="var">mobius</var></dt> <dd> Specify the transition point from linear to mobius transform. Every value below this point is guaranteed to be mapped 1:1. The higher the value, the more accurate the result will be, at the cost of losing bright details. Default to 0.3, which due to the steep initial slope still preserves in-range colors fairly accurately. </dd> </dl>

</dd> <dt><samp class="option">desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by (smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default of 2.0 is somewhat conservative and will mostly just apply to skies or directly sunlit surfaces. A setting of 0.0 disables this option. This option works only if the input frame has a supported color tag. </dd> <dt><samp class="option">peak</samp></dt> <dd> Override signal/nominal/reference peak with this value. Useful when the embedded peak information in display metadata is not reliable or when tone mapping from a

lower range to a higher range. </dd> </dl>

 <h3 class="section">30.264 tpad# TOC</h3>

Temporarily pad video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Specify number of delay frames before input video stream. Default is 0. </dd> <dt><samp class="option">stop</samp></dt> <dd> Specify number of padding frames after input video stream. Set to -1 to pad indefinitely. Default is 0. </dd> <dt><samp class="option">start_mode</samp></dt> <dd> Set kind of frames added to beginning of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of first frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">stop_mode</samp></dt> <dd> Set kind of frames added to end of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of last frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">start_duration, stop_duration</samp></dt> <dd> Specify the duration of the start/stop delay. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. These options override <var class="var">start</var> and <var class="var">stop</var>. Default is 0. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> </dl>

 <h3 class="section">30.265 transpose# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">0, 4, cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip (default), that is: <pre class="example-preformatted">L.R L.l . . -> . . l.r R.r </pre>

</dd> <dt>‘<samp class="samp">1, 5, clock</samp>’</dt>
<dd> Rotate by 90 degrees clockwise, that is:
<pre class="example-preformatted">L.R l.L . . -> . . l.r r.R </pre>

</dd> <dt>‘<samp class="samp">2, 6, cclock</samp>’</dt>
<dd> Rotate by 90 degrees counterclockwise, that is: <div
class="example"> <pre class="example-preformatted">L.R R.r . . -> . .
l.r L.l </pre>

</dd> <dt>‘<samp class="samp">3, 7, clock_flip</samp>’</dt>
<dd> Rotate by 90 degrees clockwise and vertically flip, that is: <div
class="example"> <pre class="example-preformatted">L.R r.R . . -> . .
l.r l.L </pre> </dd> </dl>

For values between 4-7, the transposition is only done if the input video geometry is portrait and not landscape. These values are deprecated, the <code class="code">passthrough</code> option should be used instead. Numerical values are deprecated, and should be dropped in favor of symbolic constants. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">none</samp>’</dt> <dd> Always
apply transposition. </dd> <dt>‘<samp class="samp">portrait</samp>’</dt>
<dd> Preserve portrait geometry (when <var class="var">height</var>
>= <var class="var">width</var>). </dd> <dt>‘<samp
class="samp">landscape</samp>’</dt> <dd> Preserve
landscape geometry (when <var class="var">width</var> >= <var
class="var">height</var>). </dd> </dl>

Default value is <code class="code">none</code>. </dd> </dl>

For example to rotate by 90 degrees clockwise and preserve portrait layout:
 <pre class="example-preformatted">transpose=dir=1:passthrough=portrait
</pre>

The command above can also be specified as:
<pre class="example-preformatted">transpose=1:portrait </pre>

 <h3 class="section">30.266 trim<span class="pull-
right">#
TOC</h3>
Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table">
<dt><samp class="option">start</samp></dt> <dd> Specify the
time of the start of the kept section, i.e. the frame with the timestamp <var class="var">start</var> will be the first frame in the output. </dd> <dt><samp class="option">end</samp></dt>
<dd> Specify the time of the first frame that will be dropped,

i.e. the frame immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last frame in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> This is the same as <var class=”var”>start</var>, except this option sets the start timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> This is the same as <var class=”var”>end</var>, except this option sets the end timestamp in timebase units instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> The number of the first frame that should be passed to the output. </dd> <dt><samp class=”option”>end_frame</samp></dt> <dd> The number of the first frame that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the frame timestamp, while the _frame variants simply count the frames that pass through the filter. Also note that this filter does not modify the timestamps. If you wish for the output timestamps to start at zero, insert a setpts filter after the trim filter. If multiple start or end options are set, this filter tries to be greedy and keep all the frames that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple trim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class=”itemize mark-bullet”> Drop everything except the second minute of input: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=60:120 </pre>

Keep only the first second: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf trim=duration=1 </pre>

 <h3 class=”section”>30.267 unpremultiply# TOC</h3> Apply alpha unpremultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt>

<dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. If the format has 1 or 2 components, then luma is bit 0. If the format has 3 or 4 components: for RGB formats bit 0 is green, bit 1 is blue and bit 2 is red; for YUV formats bit 0 is luma, bit 1 is chroma-U and bit 2 is chroma-V. If present, the alpha channel is always the last bit. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">30.268 unsharp# TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_amount, la</samp></dt> <dd> Set the luma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 1.0. </dd> <dt><samp class="option">chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> <dt><samp class="option">alpha_msize_x, ax</samp></dt> <dd> Set the alpha matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_msize_y, ay</samp></dt> <dd> Set the alpha matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">alpha_amount, aa</samp></dt> <dd> Set the alpha effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class="subsection">30.268.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply strong luma sharpen effect: <pre class="example-preformatted">unsharp=luma_msize_x=7:luma_msize_y=7:luma_
</pre>

Apply a strong blur of both luma and chroma parameters: <pre class="example-preformatted">unsharp=7:7:-2:7:7:-2
</pre>

 <h3 class="section">30.269 untile# TOC</h3>

Decompose a video made of tiled images into the individual images. The frame rate of the output video is the frame rate of the input video multiplied by the number of tiles. This filter does the reverse of tile. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size (i.e. the number of lines and columns). For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">30.269.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Produce a 1-second video from a still image file made of 25 frames stacked vertically, like an analogic film reel: <pre class="example-preformatted">ffmpeg -r 1 -i image.jpg -vf untile=1x25 movie.mkv </pre>

 <h3 class="section">30.270 uspp# TOC</h3>

Apply ultra slow/simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">8</code> - all) shifts and average the results. The way this differs from the behavior of spp is that uspp actually encodes & decodes each case with libavcodec Snow, whereas spp uses a simplified intra only 8x8 DCT similar to MJPEG. This filter

is not available in ffmpeg versions between 5.0 and 6.0. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-8. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">8</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">codec</samp></dt> <dd> Use specified codec instead of snow. </dd> </dl>

 <h3 class="section">30.271 v360# TOC</h3>

Convert 360 videos between various formats. The filter accepts the following options: <dl class="table"> <dt><samp class="option">input</samp></dt> <dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. Available formats: <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">c3x2</samp>’</dt> <dt>‘<samp class="samp">c6x1</samp>’</dt> <dt>‘<samp class="samp">c1x6</samp>’</dt> <dd> Cubemap with 3x2/6x1/1x6 layout. Format specific options: <dl class="table"> <dt><samp class="option">in_pad</samp></dt> <dt><samp class="option">out_pad</samp></dt> <dd> Set padding proportion for the input/output cubemap. Values in decimals. Example values: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> No padding. </dd> <dt>‘<samp class="samp">0.01</samp>’</dt> <dd> 1% of face is padding. For example, with 1920x1280 resolution face size would be 640x640 and padding would be 3 pixels from each side. (640 * 0.01 = 6 pixels) </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. Maximum value is ‘<samp class="samp">0.1</samp>’. </dd> <dt><samp class="option">fin_pad</samp></dt> <dt><samp class="option">fout_pad</samp></dt> <dd> Set fixed padding for the input/output cubemap. Values in pixels. Default value is ‘<samp class="samp">0</samp>’. If greater than zero it overrides other padding options. </dd> <dt><samp class="option">in_forder</samp></dt> <dt><samp class="option">out_forder</samp></dt> <dd> Set order of faces

for the input/output cubemap. Choose one direction for each position.
Designation of directions: <dl class="table"> <dt>‘<samp
class="samp">r</samp>’</dt> <dd> right </dd>
<dt>‘<samp class="samp">l</samp>’</dt> <dd> left
</dd> <dt>‘<samp class="samp">u</samp>’</dt>
<dd> up </dd> <dt>‘<samp class="samp">d</samp>’</dt>
<dd> down </dd> <dt>‘<samp class="samp">f</samp>’</dt>
<dd> forward </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> back </dd> </dl>

Default value is ‘<samp class="samp">rludfb</samp>’.
 </dd> <dt><samp class="option">in_frot</samp></dt> <dt><samp
class="option">out_frot</samp></dt> <dd> Set rotation of faces for
the input/output cubemap. Choose one angle for each position.
Designation of angles: <dl class="table"> <dt>‘<samp
class="samp">0</samp>’</dt> <dd> 0 degrees clockwise
</dd> <dt>‘<samp class="samp">1</samp>’</dt>
<dd> 90 degrees clockwise </dd> <dt>‘<samp class="samp">2</samp>’</dt>
<dd> 180 degrees clockwise </dd> <dt>‘<samp class="samp">3</samp>’</dt>
<dd> 270 degrees clockwise </dd> </dl>

Default value is ‘<samp class="samp">000000</samp>’.
</dd> </dl>

</dd> <dt>‘<samp class="samp">eac</samp>’</dt>
<dd> Equi-Angular Cubemap. </dd> <dt>‘<samp
class="samp">flat</samp>’</dt> <dt>‘<samp class="samp">gnomonic</samp>’<
<dt>‘<samp class="samp">rectilinear</samp>’</dt>
<dd> Regular video. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt>
<dd> Dual fisheye. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>

<dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>barrel</samp>’</dt> <dt>‘<samp class=”samp”>fb</samp>’</dt> <dt>‘<samp class=”samp”>barrelsplit</samp>’</dt> <dd> Facebook’s 360 formats. </dd> <dt>‘<samp class=”samp”>sg</samp>’</dt> <dd> Stereographic format. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>mercator</samp>’</dt> <dd> Mercator format. </dd> <dt>‘<samp class=”samp”>ball</samp>’</dt> <dd> Ball format, gives significant distortion toward the back. </dd> <dt>‘<samp class=”samp”>hammer</samp>’</dt> <dd> Hammer-Aitoff map projection format. </dd> <dt>‘<samp class=”samp”>sinusoidal</samp>’</dt> <dd> Sinusoidal map projection format. </dd> <dt>‘<samp class=”samp”>fisheye</samp>’</dt> <dd> Fisheye projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt> <dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt> <dt><samp class=”option”>id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>pannini</samp>’</dt> <dd> Pannini projection. Format specific options: <dl class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dd> Set output pannini parameter. </dd> <dt><samp class=”option”>ih_fov</samp></dt> <dd> Set input pannini parameter. </dd> </dl>

</dd> <dt>‘<samp class=”samp”>cylindrical</samp>’</dt> <dd> Cylindrical projection. Format specific options:

<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">perspective</samp>’</dt>
<dd> Perspective projection. (output only)
Format specific options: <dl class="table"> <dt><samp class="option">v_fov</samp></dt> <dd> Set perspective parameter. </dd> </dl>

</dd> <dt>‘<samp class="samp">tetrahedron</samp>’</dt>
<dd> Tetrahedron projection. </dd> <dt>‘<samp class="samp">tsp</samp>’</dt> <dd> Truncated square pyramid projection. </dd> <dt>‘<samp class="samp">he</samp>’</dt>
<dt>‘<samp class="samp">hequirect</samp>’</dt> <dd> Half equirectangular projection. </dd> <dt>‘<samp class="samp">equisolid</samp>’</dt
<dd> Equisolid format. Format specific options: <dl class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">og</samp>’</dt>
<dd> Orthographic format. Format specific options:
<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">octahedron</samp>’</dt>
<dd> Octahedron projection. </dd> <dt>‘<samp
class="samp">cylindricalea</samp>’</dt> <dd> Cylindrical
Equal Area projection. </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Set
interpolation method.
 Note: more complex interpolation
methods require much more memory to run. Available methods:
 <dl class="table"> <dt>‘<samp class="samp">near</samp>’</dt>
<dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Nearest
neighbour. </dd> <dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">linear</samp>’</dt> <dd> Bilinear
interpolation. </dd> <dt>‘<samp class="samp">lagrange9</samp>’</dt>
<dd> Lagrange9 interpolation. </dd> <dt>‘<samp
class="samp">cube</samp>’</dt> <dt>‘<samp class="samp">cubic</samp>’</dt
<dd> Bicubic interpolation. </dd> <dt>‘<samp class="samp">lanc</samp>’</dt
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Lanczos
interpolation. </dd> <dt>‘<samp class="samp">sp16</samp>’</dt>
<dt>‘<samp class="samp">spline16</samp>’</dt> <dd> Spline16
interpolation. </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">gaussian</samp>’</dt> <dd> Gaussian
interpolation. </dd> <dt>‘<samp class="samp">mitchell</samp>’</dt>
<dd> Mitchell interpolation. </dd> </dl>

Default value is ‘<samp class="samp">line</samp>’.
 </dd> <dt><samp class="option">w</samp></dt> <dt><samp
class="option">h</samp></dt> <dd> Set the output video resolution. Default resolution depends on formats.
</dd> <dt><samp class="option">in_stereo</samp></dt> <dt><samp
class="option">out_stereo</samp></dt> <dd> Set the input/output
stereo format. <dl class="table"> <dt>‘<samp class="samp">2d</samp>’</dt>
<dd> 2D mono </dd> <dt>‘<samp class="samp">sbs</samp>’</dt>
<dd> Side by side </dd> <dt>‘<samp class="samp">tb</samp>’</dt>
<dd> Top bottom </dd> </dl>

Default value is ‘<samp class="samp">2d</samp>’
for input and output format. </dd> <dt><samp class="option">yaw</samp></dt>
<dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt>
<dd> Set rotation for the output video. Values in degrees.
</dd> <dt><samp class="option">rorder</samp></dt> <dd> Set
rotation order for the output video. Choose one item for each position. <dl class="table"> <dt>‘<samp class="samp">y,
Y</samp>’</dt> <dd> yaw </dd> <dt>‘<samp
class="samp">p, P</samp>’</dt> <dd> pitch </dd>
<dt>‘<samp class="samp">r, R</samp>’</dt> <dd> roll
</dd> </dl>

Default value is ‘<samp class="samp">ypr</samp>’.

 </dd> <dt><samp class="option">h_flip</samp></dt> <dt><samp class="option">v_flip</samp></dt> <dt><samp class="option">d_flip</samp></dt> <dd> Flip the output video horizontally(swaps left-right)/vertically(swaps up-down)/in-depth(swaps back-forward). Boolean values. </dd> <dt><samp class="option">ih_flip</samp></dt> <dt><samp class="option">iv_flip</samp></dt> <dd> Set if input video is flipped horizontally/vertically. Boolean values. </dd> <dt><samp class="option">in_trans</samp></dt> <dd> Set if input video is transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">out_trans</samp></dt> <dd> Set if output video needs to be transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">h_offset</samp></dt> <dt><samp class="option">v_offset</samp></dt> <dd> Set output horizontal/vertical off-axis offset. Default is set to 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">alpha_mask</samp></dt> <dd> Build mask in alpha plane for all unmapped pixels by marking them fully transparent. Boolean value, by default disabled. </dd> <dt><samp class="option">reset_rot</samp></dt> <dd> Reset rotation of output video. Boolean value, by default disabled. </dd> </dl>

 <h4 class="subsection">30.271.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert equirectangular video to cubemap with 3x2 layout and 1% padding using bicubic interpolation: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=e:c3x2:cubic:out_pad=0.01 output.mkv </pre> Extract back view of Equi-Angular Cubemap: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=eac:flat:yaw=180 output.mkv </pre> Convert transposed and horizontally flipped Equi-Angular Cubemap in side-by-side stereo format to equirectangular top-bottom stereo format: <pre class="example-preformatted">v360=eac:equirect:in_stereo=sbs:in_trans=1:ih_flip=1:out_stereo=tb </pre>

 <h4 class="subsection">30.271.2 Commands# TOC</h4>

This filter supports subset of above options as commands. <h3 class="section">30.272 vaguedenoiser# TOC</h3>

Apply a wavelet based denoiser. It transforms each frame from the video input into the wavelet domain, using Cohen-Daubechies-Feauveau 9/7. Then it applies some filtering to the obtained coefficients. It does an inverse wavelet transform after. Due to wavelet properties, it should give a nice smoothed result, and reduced noise, without blurring picture features. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> The filtering strength. The higher, the more filtered the video will be. Hard thresholding can use a higher threshold than soft thresholding before the video looks overfiltered. Default value is 2. </dd> <dt><samp class="option">method</samp></dt> <dd> The filtering method the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> All values under the threshold will be zeroed. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> All values under the threshold will be zeroed. All values above will be reduced by the threshold. </dd> <dt>‘<samp class="samp">garrote</samp>’</dt> <dd> Scales or nullifies coefficients - intermediary between (more) soft and (less) hard thresholding. </dd> </dl>

Default is garrote. </dd> <dt><samp class="option">nsteps</samp></dt> <dd> Number of times, the wavelet will decompose the picture. Picture can’t be decomposed beyond a particular point (typically, 8 for a 640x480 frame - as 2^9 = 512 > 480). Valid values are integers between 1 and 32. Default value is 6. </dd> <dt><samp class="option">percent</samp></dt> <dd> Partial of full denoising (limited coefficients shrinking), from 0 to 100. Default value is 85. </dd> <dt><samp class="option">planes</samp></dt> <dd> A list of the planes to process. By default all planes are processed. </dd> <dt><samp class="option">type</samp></dt> <dd> The threshold type the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">universal</samp>’</dt> <dd> Threshold used is same for all decompositions. </dd> <dt>‘<samp class="samp">bayes</samp>’</dt> <dd> Threshold used depends also on each decomposition coefficients. </dd> </dl>

Default is universal. </dd> </dl>

 <h3 class="section">30.273 varblur# TOC</h3> Apply variable blur filter by using 2nd video stream to set blur radius. The 2nd stream must have the same dimensions. This filter accepts the following options: <dl class="table"> <dt><samp class="option">min_r</samp></dt>

<dd> Set min allowed radius. Allowed range is from 0 to 254. Default is 0. </dd> <dt><samp class="option">max_r</samp></dt> <dd> Set max allowed radius. Allowed range is from 1 to 255. Default is 8. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. By default, all are used. </dd> </dl>

The <code class="code">varblur</code> filter also supports the framesync options. <h4 class="subsection">30.273.1 Commands# TOC</h4>

This filter supports all the above options as commands. <h3 class="section">30.274 vectorscope# TOC</h3>

Display 2 color component values in the two dimensional graph (which is called a vectorscope). This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set vectorscope mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">gray</samp>’</dt> <dt>‘<samp class="samp">tint</samp>’</dt> <dd> Gray values are displayed on graph, higher brightness means more pixels have same component color value on location in graph. This is the default mode. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Gray values are displayed on graph. Surrounding pixels values which are not present in video frame are drawn in gradient of 2 color components which are set by option <code class="code">x</code> and <code class="code">y</code>. The 3rd color component is static. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Actual color components values present in video frame are displayed on graph. </dd> <dt>‘<samp class="samp">color3</samp>’</dt> <dd> Similar as color2 but higher frequency of same values <code class="code">x</code> and <code class="code">y</code> on graph increases value of another color component, which is luminance by default values of <code class="code">x</code> and <code class="code">y</code>. </dd> <dt>‘<samp class="samp">color4</samp>’</dt> <dd> Actual colors present in video frame are displayed on graph. If two different colors map to same position on graph then color with higher value of component not present in graph is picked. </dd> <dt>‘<samp class="samp">color5</samp>’</dt> <dd> Gray values are displayed on graph. Similar to <code class="code">color</code> but with 3rd color component picked from radial gradient. </dd> </dl>

</dd> <dt><samp class="option">x</samp></dt> <dd> Set which color component will be represented on X-axis. Default is <code class="code">1</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set which color component will be represented on Y-axis. Default is <code class="code">2</code>. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity, used by modes: gray, color, color3 and color5 for increasing brightness of color component which represents frequency of (X, Y) location in graph. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, even darkest single pixel will be clearly highlighted. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold maximum and minimum values presented in graph over time. This way you can still spot out of range values without constantly looking at vectorscope. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set what kind of graticule to draw. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">green</samp>’</dt <dt>‘<samp class="samp">color</samp>’</dt> <dt>‘<samp class="samp">invert</samp>’</dt> </dl>

</dd> <dt><samp class="option">opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class="option">flags, f</samp></dt> <dd> Set graticule flags. <dl class="table"> <dt>‘<samp class="samp">white</samp>’</dt> <dd> Draw graticule for white point. </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Draw graticule for black point. </dd> <dt>‘<samp class="samp">name</samp>’</dt> <dd> Draw color points short names. </dd> </dl>

</dd> <dt><samp class="option">bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">lthreshold, l</samp></dt> <dd> Set low threshold for color component not represented on X or Y axis. Values lower than this value will be ignored. Default is 0. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and low threshold value of 0.1 actual threshold is 0.1 * 255 = 25. </dd> <dt><samp class="option">hthreshold, h</samp></dt> <dd> Set high threshold for color component not represented on X or Y axis. Values higher than this value will be ignored. Default is 1. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and high threshold value of 0.9 actual threshold is 0.9 * 255 = 230. </dd>

<dt><samp class="option">colorspace, c</samp></dt> <dd> Set what kind of colorspace to use when drawing graticule. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">601</samp>’</dt> <dt>‘<samp class="samp">709</samp>’</dt> </dl> Default is auto. </dd> <dt><samp class="option">tint0, t0</samp></dt> <dt><samp class="option">tint1, t1</samp></dt> <dd> Set color tint for gray/tint vectorscope mode. By default both options are zero. This means no tint, and output will remain gray. </dd> </dl>

 <h3 class="section">30.275 vidstabdetect# TOC</h3>

Analyze video stabilization/deshaking. Perform pass 1 of 2, see vidstabtransform for pass 2. This filter generates a file with relative translation and rotation transform information about subsequent frames, which is then used by the vidstabtransform filter. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">result</samp></dt> <dd> Set the path to the file used to write the transforms information. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">shakiness</samp></dt> <dd> Set how shaky the video is and how quick the camera is. It accepts an integer in the range 1-10, a value of 1 means little shakiness, a value of 10 means strong shakiness. Default value is 5. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set the accuracy of the detection process. It must be a value in the range 1-15. A value of 1 means low accuracy, a value of 15 means high accuracy. Default value is 15. </dd> <dt><samp class="option">stepsize</samp></dt> <dd> Set stepsize of the search process. The region around minimum is scanned with 1 pixel resolution. Default value is 6. </dd> <dt><samp class="option">mincontrast</samp></dt> <dd> Set minimum contrast. Below this value a local measurement field is discarded. Must be a floating point value in the range 0-1. Default value is 0.3. </dd> <dt><samp class="option">tripod</samp></dt> <dd> Set reference frame number for tripod mode. If enabled, the motion of the frames is compared to a reference frame in the filtered stream, identified by the specified number. The idea is to compensate all movements in a more-or-less static scene and keep the camera view absolutely still. If set to 0, it is disabled. The frames are counted starting from 1. </dd> <dt><samp class="option">show</samp></dt> <dd> Show fields and transforms in the resulting frames. It accepts an integer in the range 0-2.

Default value is 0, which disables any visualization. </dd> <dt><samp class="option">fileformat</samp></dt> <dd> Format for the transforms data file to be written. Acceptable values are <dl class="table"> <dt>‘<samp class="samp">ascii</samp>’</dt> <dd> Human-readable plain text </dd> <dt>‘<samp class="samp">binary</samp>’</dt> <dd> Binary format, roughly 40% smaller than <code class="code">ascii</code>. (default) </dd> </dl>

</dd> </dl>

 <h4 class="subsection">30.275.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use default values: <pre class="example-preformatted">vidstabdetect </pre>

Analyze strongly shaky movie and put the results in file <samp class="file">mytransforms.trf</samp>: <pre class="example-preformatted">vidstabdetect=shakiness=10:accuracy=15:result="mytransforms.trf&qu
</pre>

Visualize the result of internal transformations in the resulting video: <pre class="example-preformatted">vidstabdetect=show=1 </pre>

Analyze a video with medium shakiness using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input -vf vidstabdetect=shakiness=5:show=1 dummy.avi </pre>

 <h3 class="section">30.276 vidstabtransform# TOC</h3>

Video stabilization/deshaking: pass 2 of 2, see vidstabdetect for pass 1. Read a file with transform information for each frame and apply/compensate them. Together with the vidstabdetect filter this can be used to deshake videos. See also http://public.hronopik.de/vid.stab. It is important to also use the unsharp filter, see below. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. <h4 class="subsection">30.276.1 Options<a class="anchor hidden-xs" href="#Options-

42" aria-hidden="true"># TOC</h4>

<dl class="table"> <dt><samp class="option">input</samp></dt> <dd> Set path to the file used to read the transforms. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set the number of frames (value*2 + 1) used for lowpass filtering the camera movements. Default value is 10. For example a number of 10 means that 21 frames are used (10 in the past and 10 in the future) to smoothen the motion in the video. A larger value leads to a smoother video, but limits the acceleration of the camera (pan/tilt movements). 0 is a special case where a static camera is simulated. </dd> <dt><samp class="option">optalgo</samp></dt> <dd> Set the camera path optimization algorithm. Accepted values are: <dl class="table"> <dt>‘<samp class="samp">gauss</samp>’</dt> <dd> gaussian kernel low-pass filter on camera motion (default) </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> averaging on transformations </dd> </dl>

</dd> <dt><samp class="option">maxshift</samp></dt> <dd> Set maximal number of pixels to translate frames. Default value is -1, meaning no limit. </dd> <dt><samp class="option">maxangle</samp></dt> <dd> Set maximal angle in radians (degree*PI/180) to rotate frames. Default value is -1, meaning no limit. </dd> <dt><samp class="option">crop</samp></dt> <dd> Specify how to deal with borders that may be visible due to movement compensation. Available values are: <dl class="table"> <dt>‘<samp class="samp">keep</samp>’</dt> <dd> keep image information from previous frame (default) </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> fill the border black </dd> </dl>

</dd> <dt><samp class="option">invert</samp></dt> <dd> Invert transforms if set to 1. Default value is 0. </dd> <dt><samp class="option">relative</samp></dt> <dd> Consider transforms as relative to previous frame if set to 1, absolute if set to 0. Default value is 0. </dd> <dt><samp class="option">zoom</samp></dt> <dd> Set percentage to zoom. A positive value will result in a zoom-in effect, a negative value in a zoom-out effect. Default value is 0 (no zoom). </dd> <dt><samp class="option">optzoom</samp></dt> <dd> Set optimal zooming to avoid borders. Accepted values are: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> disabled </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> optimal static zoom value is determined (only very strong movements will lead to visible borders) (default) </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> optimal adaptive zoom

value is determined (no borders will be visible), see <samp class="option">zoomspeed</samp>
</dd> </dl>

Note that the value given at zoom is added to the one calculated here. </dd> <dt><samp class="option">zoomspeed</samp></dt>
<dd> Set percent to zoom maximally each frame (enabled when <samp class="option">optzoom</samp> is set to 2). Range is from 0 to 5, default value is 0.25. </dd> <dt><samp class="option">interpol</samp></dt>
<dd> Specify type of interpolation. Available values are:
<dl class="table"> <dt>‘<samp class="samp">no</samp>’</dt>
<dd> no interpolation </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> linear only horizontal </dd> <dt>‘<samp class="samp">bilinear</samp>’</
<dd> linear in both directions (default) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> cubic in both directions (slow) </dd> </dl>

</dd> <dt><samp class="option">tripod</samp></dt> <dd> Enable virtual tripod mode if set to 1, which is equivalent to <code class="code">relative=0:smoothing=0</code>. Default value is 0. Use also <code class="code">tripod</code> option of vidstabdetect.
</dd> <dt><samp class="option">debug</samp></dt> <dd> Increase log verbosity if set to 1. Also the detected global motions are written to the temporary file <samp class="file">global_motions.trf</samp>. Default value is 0. </dd> </dl>

 <h4 class="subsection">30.276.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> for a typical stabilization with default values:
<pre class="example-preformatted">ffmpeg -i inp.mpeg -vf vidstabtransform,unsharp=5:5:0.8:3:3:0.4 inp_stabilized.mpeg </pre>

Note the use of the unsharp filter which is always recommended. Zoom in a bit more and load transform data from a given file: <pre class="example-preformatted">vidstabtransform=zoom=5:input="mytransforms.trf"
</pre>

Smoothen the video even more: <pre class="example-preformatted">vidstabtransform=smoothing=30 </pre>

 <h3 class="section">30.277 vflip#
TOC</h3>

Flip the input video vertically. For example, to vertically

flip a video with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i in.avi -vf "vflip" out.avi </pre>

 <h3 class=”section”>30.278 vfrdet# TOC</h3>

Detect variable frame rate video. This filter tries to detect if the input is variable or constant frame rate. At end it will output number of frames detected as having variable delta pts, and ones with constant delta pts. If there was frames with variable delta, than it will also show min, max and average delta encountered. <h3 class=”section”>30.279 vibrance# TOC</h3>

Boost or alter saturation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>intensity</samp></dt> <dd> Set strength of boost if positive value or strength of alter if negative value. Default is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>rbal</samp></dt> <dd> Set the red balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>gbal</samp></dt> <dd> Set the green balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>bbal</samp></dt> <dd> Set the blue balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>rlum</samp></dt> <dd> Set the red luma coefficient. </dd> <dt><samp class=”option”>glum</samp></dt> <dd> Set the green luma coefficient. </dd> <dt><samp class=”option”>blum</samp></dt> <dd> Set the blue luma coefficient. </dd> <dt><samp class=”option”>alternate</samp></dt> <dd> If <code class=”code”>intensity</code> is negative and this is set to 1, colors will change, otherwise colors will be less saturated, more towards gray. </dd> </dl>

 <h4 class=”subsection”>30.279.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>30.280 vif# TOC</h3>

Obtain the average VIF (Visual Information Fidelity) between two input

videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average VIF score is printed through the logging system. The filter stores the calculated VIF score of each frame. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi vif -f null - </pre>

 <h3 class=”section”>30.281 vignette# TOC</h3>

Make or reverse a natural vignetting effect. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>angle, a</samp></dt> <dd> Set lens angle expression as a number of radians. The value is clipped in the <code class=”code”>[0,PI/2]</code> range. Default value: <code class=”code”>"PI/5"</code> </dd> <dt><samp class=”option”>x0</samp></dt> <dt><samp class=”option”>y0</samp></dt> <dd> Set center coordinates expressions. Respectively <code class=”code”>"w/2"</code> and <code class=”code”>"h/2"</code> by default. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set forward/backward mode. Available modes are: <dl class=”table”> <dt>‘<samp class=”samp”>forward</samp>’</dt> <dd> The larger the distance from the central point, the darker the image becomes. </dd> <dt>‘<samp class=”samp”>backward</samp>’</dt> <dd> The larger the distance from the central point, the brighter the image becomes. This can be used to reverse a vignette effect, though there is no automatic detection to extract the lens <samp class=”option”>angle</samp> and other settings (yet). It can also be used to create a burning effect. </dd> </dl>

Default value is ‘<samp class=”samp”>forward</samp>’. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set evaluation mode for the expressions (<samp class=”option”>angle</samp>, <samp class=”option”>x0</samp>, <samp class=”option”>y0</samp>). It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> Evaluate expressions only once during the filter initialization. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. This is way slower than the ‘<samp class=”samp”>init</samp>’ mode since it requires all the scalers to

be re-computed, but it allows advanced dynamic expressions. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> <dt><samp class="option">dither</samp></dt> <dd> Set dithering to reduce the circular banding effects. Default is <code class="code">1</code> (enabled). </dd> <dt><samp class="option">aspect</samp></dt> <dd> Set vignette aspect. This setting allows one to adjust the shape of the vignette. Setting this value to the SAR of the input will make a rectangular vignetting following the dimensions of the video. Default is <code class="code">1/1</code>. </dd> </dl>

 <h4 class="subsection">30.281.1 Expressions# TOC</h4>

The <samp class="option">alpha</samp>, <samp class="option">x0</samp> and <samp class="option">y0</samp> expressions can contain the following parameters. <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> input width and height </dd> <dt><samp class="option">n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class="option">pts</samp></dt> <dd> the PTS (Presentation TimeStamp) time of the filtered video frame, expressed in <var class="var">TB</var> units, NAN if undefined </dd> <dt><samp class="option">r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class="option">t</samp></dt> <dd> the PTS (Presentation TimeStamp) of the filtered video frame, expressed in seconds, NAN if undefined </dd> <dt><samp class="option">tb</samp></dt> <dd> time base of the input video </dd> </dl>

 <h4 class="subsection">30.281.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply simple strong vignetting effect: <pre class="example-preformatted">vignette=PI/4 </pre>

Make a flickering vignetting: <pre class="example-preformatted">vignette='PI/4+random(1)*PI/50':eval=frame </pre>

 <h3 class="section">30.282 vmafmotion<a class="anchor hidden-xs" href="#vmafmotion"

aria-hidden=”true”># TOC</h3>

Obtain the average VMAF motion score of a video. It is one of the component metrics of VMAF. The obtained average motion score is printed through the logging system. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stats_file</samp></dt> <dd> If specified, the filter will use the named file to save the motion score of each frame with respect to the previous frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

Example: <pre class=”example-preformatted”>ffmpeg -i ref.mpg -vf vmafmotion -f null - </pre>

 <h3 class=”section”>30.283 vpp_amf# TOC</h3>

Scale (resize) and convert colorspace, transfer characteristics or color primaries for the input video, using AMD Advanced Media Framework library for hardware acceleration. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>scale_type</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear This is the default. </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used.

</dd> <dt><samp class=”option”>force_original_aspect_ratio</samp></dt> <dt><samp class=”option”>force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class=”option”>reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class=”option”>in_color_range</samp></dt> <dd> Override input color range. </dd> <dt><samp class=”option”>out_color_range</samp><dd> Specify output color range. The accepted values for in_trc and out_trc are: <dl class=”table”> <dt>‘<samp class=”samp”>studio</samp>’</dt> <dd> Studio (or re-

stricted, or MPEG) color range. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Full (or JPEG) color range. </dd> </dl>

</dd> <dt><samp class="option">color_profile</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">in_trc</samp></dt> <dd> Override input transfer characteristics. </dd> <dt><samp class="option">out_trc</samp></dt> <dd> Specify output transfer characteristics. The accepted values for in_trc and out_trc are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dd> Constant gamma of 2.2 </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dd> Constant gamma of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> LOG </dd> <dt>‘<samp class="samp">log-sqrt</samp>’</dt> <dd> LOG_SQRT </dd> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4 </dd> <dt>‘<samp class="samp">bt1361-ecg</samp>’</dt> <dd> BT1361_ECG </dd> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for 10-bits content </dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dd> BT.2020 for 12-bits content </dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dd> SMPTE2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dd> SMPTE428 </dd> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> <dd> ARIB_STD_B67 </dd> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dd> Override input color primaries. </dd> <dt><samp class="option">out_primaries</samp></dt> <dd> Specify output color primaries. The accepted values for in_primaries and out_primaries are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp

class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>film</samp>’</dt> <dd> film </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class=”samp”>smpte428</samp>’</dt> <dd> SMPTE-428 </dd> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class=”samp”>jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>30.283.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class=”example-preformatted”>vpp_amf=-2:720:format=yuv420p,hwdownload,format=yuv420p </pre>

Upscale to 4K and change color profile to bt2020. <pre class=”example-preformatted”>vpp_amf=4096:2160:color_profile=bt2020 </pre>

Override input primaries and input transfer characteristics, change both to bt709.

 <pre class=”example-preformatted”>vpp_amf=color_profile=bt2020:in_trc=smpte2 </pre>

 <h3 class=”section”>30.284 vstack# TOC</h3> Stack input videos vertically. All streams must be of same pixel format and of same width. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class=”section”>30.285 w3fdif<a class=”anchor hidden-xs” href=”#w3fdif” aria-

hidden=”true”># TOC</h3>

Deinterlace the input video ("w3fdif" stands for "Weston 3 Field Deinterlacing Filter"). Based on the process described by Martin Weston for BBC R&D, and implemented based on the de-interlace algorithm written by Jim Easterbrook for BBC R&D, the Weston 3 field deinterlacing filter uses filter coefficients calculated by BBC R&D. This filter uses field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class=”code”>w3fdif</code> filter. There are two sets of filter coefficients, so called "simple" and "complex". Which set of filter coefficients is used can be set by passing an optional parameter: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set the interlacing filter coefficients. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Simple filter coefficient set. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> More-complex filter coefficient set. </dd> </dl> Default value is ‘<samp class=”samp”>complex</samp>’. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames, </dd> <dt>‘<samp class=”samp”>interlaced</samp>&rsquo <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

Default value is ‘<samp class=”samp”>all</samp>’. </dd> </dl>

 <h4 class="subsection">30.285.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">30.286 waveform# TOC</h3> Video waveform monitor. The waveform monitor plots color component intensity. By default luma only. Each column of the waveform corresponds to a column of pixels in the source video. It accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Can be either <code class="code">row</code>, or <code class="code">column</code>. Default is <code class="code">column</code>. In row mode, the graph on the left side represents color component value 0 and the right side represents value = 255. In column mode, the top side represents color component value = 0 and bottom side represents value = 255. </dd> <dt><samp class="option">intensity, i</samp></dt> <dd> Set intensity. Smaller values are useful to find out how many values of the same luminance are distributed across input rows/columns. Default value is <code class="code">0.04</code>. Allowed range is [0, 1]. </dd> <dt><samp class="option">mirror, r</samp></dt> <dd> Set mirroring mode. <code class="code">0</code> means unmirrored, <code class="code">1</code> means mirrored. In mirrored mode, higher values will be represented on the left side for <code class="code">row</code> mode and at the top for <code class="code">column</code> mode. Default is <code class="code">1</code> (mirrored). </dd> <dt><samp class="option">display, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. This display mode makes it easier to spot relative differences or similarities in overlapping areas of the color components that are supposed to be identical, such as neutral whites, grays, or blacks. </dd> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">row</code> mode or one below the other in <code class="code">column</code> mode. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">column</code> mode or one below the other in <code class="code">row</code> mode. Using this display mode makes it easy to spot color casts in the highlights and shadows of an image, by comparing the contours of the top and the bottom

graphs of each waveform. Since whites, grays, and blacks are characterized by exactly equal amounts of red, green, and blue, neutral areas of the picture should display three waveforms of roughly equal width/height. If not, the correction is easy to perform by making level adjustments the three waveforms. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">components, c</samp></dt> <dd> Set which color components to display. Default is 1, which means only luma or red color component if input is in RGB colorspace. If is set for example to 7 it will display all 3 (if) available color components. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, minimum and maximum values presented in graph will be easily visible even with small <code class="code">step</code> value. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold minimum and maximum values presented in graph across time. This way you can still spot out of range values without constantly looking at waveforms. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">filter, f</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">lowpass</samp>’</dt> <dd> No filtering, this is default. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Luma and chroma combined together. </dd> <dt>‘<samp class="samp">aflat</samp>’</dt> <dd> Similar as above, but shows difference between blue and red chroma. </dd> <dt>‘<samp class="samp">xflat</samp>’</dt> <dd> Similar as above, but use different colors. </dd> <dt>‘<samp class="samp">yflat</samp>’</dt> <dd> Similar as above, but again with different colors. </dd> <dt>‘<samp class="samp">chroma</samp>’</dt> <dd> Displays only chroma. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Displays actual color value on waveform. </dd> <dt>‘<samp class="samp">acolor</samp>’</dt> <dd> Similar as above, but with luma showing frequency of chroma values. </dd> </dl>

</dd> <dt><samp class="option">graticule, g</samp></dt> <dd> Set which graticule to display. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Do not display graticule. </dd> <dt>‘<samp class="samp">green</samp>’</dt> <dd> Display green graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">orange</samp>’</dt> <dd> Display orange graticule showing legal broadcast ranges. </dd> <dt>‘<samp class="samp">invert</samp>’</dt> <dd> Display invert graticule showing legal broadcast ranges. </dd>

</dl>

</dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class=”option”>flags, fl</samp></dt> <dd> Set graticule flags. <dl class=”table”> <dt>‘<samp class=”samp”>numbers</samp>’</dt> <dd> Draw numbers above lines. By default enabled. </dd> <dt>‘<samp class=”samp”>dots</samp>’</dt> <dd> Draw dots instead of lines. </dd> </dl>

</dd> <dt><samp class=”option”>scale, s</samp></dt> <dd> Set scale used for displaying graticule. <dl class=”table”> <dt>‘<samp class=”samp”>digital</samp>’</dt> <dt>‘<samp class=”samp”>millivolts</samp>&rsquo <dt>‘<samp class=”samp”>ire</samp>’</dt> </dl> Default is digital. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class=”option”>tint0, t0</samp></dt> <dt><samp class=”option”>tint1, t1</samp></dt> <dd> Set tint for output. Only used with lowpass filter and when display is not overlay and input pixel formats are not RGB. </dd> <dt><samp class=”option”>fitmode, fm</samp></dt> <dd> Set sample aspect ratio of video output frames. Can be used to configure waveform so it is not stretched too much in one of directions. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Set sample aspect ration to 1/1. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Set sample aspect ratio to match input size of video </dd> </dl> Default is ‘<samp class=”samp”>none</samp>’. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set input formats for filter to pick from. Can be ‘<samp class=”samp”>all</samp>’, for selecting from all available formats, or ‘<samp class=”samp”>first</samp>’, for selecting first available format. Default is ‘<samp class=”samp”>first</samp>’. </dd> </dl>

 <h3 class=”section”>30.287 weave, doubleweave# TOC</h3>

The <code class=”code”>weave</code> takes a field-based video input and join each two sequential fields into single frame, producing a new double height clip with half the frame rate and half the frame count. The <code class=”code”>doubleweave</code> works same as <code class=”code”>weave</code> but without halving frame rate and frame count. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd> Set first field. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> Set the frame as

top-field-first. </dd> <dt>‘<samp class="samp">bottom, b</samp>’</dt> <dd> Set the frame as bottom-field-first. </dd> </dl> </dd> </dl>

 <h4 class="subsection">30.287.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interlace video using select and separatefields filter: <pre class="example-preformatted">separatefields,select=eq(mod(n,4),0)+eq(
</pre>

 <h3 class="section">30.288 xbr# TOC</h3> Apply the xBR high-quality magnification filter which is designed for pixel art. It follows a set of edge-detection rules, see https://forums.libretro.com/t/xbr-algorithm-tutorial/123. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xBR</code>, <code class="code">3</code> for <code class="code">3xBR</code> and <code class="code">4</code> for <code class="code">4xBR</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">30.289 xcorrelate# TOC</h3> Apply normalized cross-correlation between first and second input video stream. Second input video stream dimensions must be lower than first input video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">secondary</samp></dt> <dd> Set which secondary video frames will be processed from second input video stream, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">xcorrelate</code> filter also supports the framesync options. <h3 class="section">30.290 xfade# TOC</h3>

Apply cross fade from one input video stream to another input video stream.

The cross fade is applied for specified duration.

Both inputs must be constant frame-rate and have the same resolution, pixel format,

frame rate and timebase.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">transition</samp></dt>

<dd> Set one of available transition effects:

<dl class="table">

<dt>‘<samp class="samp">custom</samp>’</dt>

<dt>‘<samp class="samp">fade</samp>’</dt>

<dt>‘<samp class="samp">wipeleft</samp>’</dt>

<dt>‘<samp class="samp">wiperight</samp>’</dt>

<dt>‘<samp class="samp">wipeup</samp>’</dt>

<dt>‘<samp class="samp">wipedown</samp>’</dt>

<dt>‘<samp class="samp">slideleft</samp>’</dt>

<dt>‘<samp class="samp">slideright</samp>’</dt>

<dt>‘<samp class="samp">slideup</samp>’</dt>

<dt>‘<samp class="samp">slidedown</samp>’</dt>

<dt>‘<samp class="samp">circlecrop</samp>’</dt>

<dt>‘<samp class="samp">rectcrop</samp>’</dt>

<dt>‘<samp class="samp">distance</samp>’</dt>

<dt>‘<samp class="samp">fadeblack</samp>’</dt>

<dt>‘<samp class="samp">fadewhite</samp>’</dt>

<dt>‘<samp class="samp">radial</samp>’</dt>

<dt>‘<samp class="samp">smoothleft</samp>’</dt>

<dt>‘<samp class="samp">smoothright</samp>’</dt>

<dt>‘<samp class="samp">smoothup</samp>’</dt>
<dt>‘<samp class="samp">smoothdown</samp>’</dt>
<dt>‘<samp class="samp">circleopen</samp>’</dt>
<dt>‘<samp class="samp">circleclose</samp>’</dt>
<dt>‘<samp class="samp">vertopen</samp>’</dt>
<dt>‘<samp class="samp">vertclose</samp>’</dt>
<dt>‘<samp class="samp">horzopen</samp>’</dt>
<dt>‘<samp class="samp">horzclose</samp>’</dt>
<dt>‘<samp class="samp">dissolve</samp>’</dt>
<dt>‘<samp class="samp">pixelize</samp>’</dt>
<dt>‘<samp class="samp">diagtl</samp>’</dt>
<dt>‘<samp class="samp">diagtr</samp>’</dt>
<dt>‘<samp class="samp">diagbl</samp>’</dt>
<dt>‘<samp class="samp">diagbr</samp>’</dt>
<dt>‘<samp class="samp">hlslice</samp>’</dt>
<dt>‘<samp class="samp">hrslice</samp>’</dt>
<dt>‘<samp class="samp">vuslice</samp>’</dt>
<dt>‘<samp class="samp">vdslice</samp>’</dt>
<dt>‘<samp class="samp">hblur</samp>’</dt>
<dt>‘<samp class="samp">fadegrays</samp>’</dt>
<dt>‘<samp class="samp">wipetl</samp>’</dt>
<dt>‘<samp class="samp">wipetr</samp>’</dt>
<dt>‘<samp class="samp">wipebl</samp>’</dt>
<dt>‘<samp class="samp">wipebr</samp>’</dt>
<dt>‘<samp class="samp">squeezeh</samp>’</dt>
<dt>‘<samp class="samp">squeezev</samp>’</dt>
<dt>‘<samp class="samp">zoomin</samp>’</dt>
<dt>‘<samp class="samp">fadefast</samp>’</dt>
<dt>‘<samp class="samp">fadeslow</samp>’</dt>
<dt>‘<samp class="samp">hlwind</samp>’</dt>
<dt>‘<samp class="samp">hrwind</samp>’</dt>

<dt>‘<samp class="samp">vuwind</samp>’</dt>
<dt>‘<samp class="samp">vdwind</samp>’</dt>
<dt>‘<samp class="samp">coverleft</samp>’</dt>
<dt>‘<samp class="samp">coverright</samp>’</dt>
<dt>‘<samp class="samp">coverup</samp>’</dt>
<dt>‘<samp class="samp">coverdown</samp>’</dt>
<dt>‘<samp class="samp">revealleft</samp>’</dt>
<dt>‘<samp class="samp">revealright</samp>’</dt>
<dt>‘<samp class="samp">revealup</samp>’</dt>
<dt>‘<samp class="samp">revealdown</samp>’</dt>
</dl>
Default transition effect is fade.

</dd>
<dt><samp class="option">duration</samp></dt>
<dd> Set cross fade duration in seconds.
Range is 0 to 60 seconds.
Default duration is 1 second.

</dd>
<dt><samp class="option">offset</samp></dt>
<dd> Set cross fade start relative to first input stream in seconds.
Default offset is 0.

</dd>
<dt><samp class="option">expr</samp></dt>
<dd> Set expression for custom transition effect.

The expressions can use the following variables and functions:

<dl class="table">

<dt><samp class="option">X</samp></dt>
<dt><samp class="option">Y</samp></dt>
<dd> The coordinates of the current sample.

</dd>
<dt><samp class="option">W</samp></dt>
<dt><samp class="option">H</samp></dt>
<dd> The width and height of the image.

</dd>
<dt><samp class="option">P</samp></dt>
<dd> Progress of transition effect.

</dd>
<dt><samp class="option">PLANE</samp></dt>
<dd> Currently processed plane.

</dd>
<dt><samp class="option">A</samp></dt>
<dd> Return value of first input at current location and plane.

</dd>
<dt><samp class="option">B</samp></dt>
<dd> Return value of second input at current location and plane.

</dd>
<dt><samp class="option">a0(x, y)</samp></dt>
<dt><samp class="option">a1(x, y)</samp></dt>
<dt><samp class="option">a2(x, y)</samp></dt>
<dt><samp class="option">a3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of first input.

</dd>

<dt><samp class="option">b0(x, y)</samp></dt>

<dt><samp class="option">b1(x, y)</samp></dt>

<dt><samp class="option">b2(x, y)</samp></dt>

<dt><samp class="option">b3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of second input.

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">30.290.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input video to another input video, with fade transition and duration of transition of 2 seconds starting at offset of 5 seconds: <pre class="example-preformatted">ffmpeg -i first.mp4 -i second.mp4 -filter_complex xfade=transition=fade:duration=2:offset=5 output.mp4 </pre>

 <h3 class="section">30.291 xmedian# TOC</h3> Pick median pixels from several input videos. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 3. Allowed range is from 3 to 255. If number of inputs is even number, than result will be mean value between two median values. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">15</code>, by which all planes are processed. </dd> <dt><samp class="option">percentile</samp></dt>

<dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">30.291.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">inputs</code>. <h3 class="section">30.292 xpsnr# TOC</h3>

Obtain the average (across all input frames) and minimum (across all color plane averages) eXtended Perceptually weighted peak Signal-to-Noise Ratio (XPSNR) between two input videos. The XPSNR is a low-complexity psychovisually motivated distortion measurement algorithm for assessing the difference between two video streams or images. This is especially useful for objectively quantifying the distortions caused by video and image codecs, as an alternative to a formal subjective test. The logarithmic XPSNR output values are in a similar range as those of traditional psnr assessments but better reflect human impressions of visual coding quality. More details on the XPSNR measure, which essentially represents a blockwise weighted variant of the PSNR measure, can be found in the following freely available papers: <ul class="itemize mark-bullet"> C. R. Helmrich, M. Siekmann, S. Becker, S. Bosse, D. Marpe, and T. Wiegand, "XPSNR: A Low-Complexity Extension of the Perceptually Weighted Peak Signal-to-Noise Ratio for High-Resolution Video Quality Assessment," in Proc. IEEE Int. Conf. Acoustics, Speech, Sig. Process. (ICASSP), virt./online, May 2020. www.ecodis.de/xpsnr.htm

C. R. Helmrich, S. Bosse, H. Schwarz, D. Marpe, and T. Wiegand, "A Study of the Extended Perceptually Weighted Peak Signal-to-Noise Ratio (XPSNR) for Video Compression with Different Resolutions and Bit Depths," ITU Journal: ICT Discoveries, vol. 3, no. 1, pp. 65 - 72, May 2020. http://handle.itu.int/11.1002/pub/8153d78b-en

When publishing the results of XPSNR assessments obtained using, e.g., this FFmpeg filter, a reference to the above papers as a means of documentation is strongly encouraged. The filter requires two input videos. The first input is considered a (usually not distorted) reference source and is passed unchanged to the output, whereas the second input is a (distorted) test sig-

nal. Except for the bit depth, these two video inputs must have the same pixel format. In addition, for best performance, both compared input videos should be in YCbCr color format. The obtained overall XPSNR values mentioned above are printed through the logging system. In case of input with multiple color planes, we suggest reporting of the minimum XPSNR average. The following parameter, which behaves like the one for the psnr filter, is accepted: <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified, the filter will use the named file to save the XPSNR value of each individual frame and color plane. When the file name equals "-", that data is sent to standard output. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">30.292.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> XPSNR analysis of two 1080p HD videos, ref_source.yuv and test_video.yuv, both at 24 frames per second, with color format 4:2:0, bit depth 8, and output of a logfile named "xpsnr.log": <pre class="example-preformatted">ffmpeg -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i ref_source.yuv -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i test_video.yuv -lavfi xpsnr="stats_file=xpsnr.log" -f null - </pre>

XPSNR analysis of two 2160p UHD videos, ref_source.yuv with bit depth 8 and test_video.yuv with bit depth 10, both at 60 frames per second with color format 4:2:0, no logfile output: <pre class="example-preformatted">ffmpeg -s 3840x2160 -framerate 60 -pix_fmt yuv420p -i ref_source.yuv -s 3840x2160 -framerate 60 -pix_fmt yuv420p10le -i test_video.yuv -lavfi xpsnr="stats_file=-" -f null - </pre>

 <h3 class="section">30.293 xstack# TOC</h3> Stack video inputs into custom layout. All streams must be of same pixel format. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">layout</samp></dt> <dd> Specify layout of inputs. This option requires the desired layout configuration to be explicitly set by the user. This sets position of each video input in output. Each input is separated by ’|’. The first number represents the column, and the second number represents the row. Numbers start at 0 and are separated by ’_’. Optionally one can use wX and hX, where

X is video input from which to take width or height. Multiple values can be used when separated by ’+’. In such case values are summed together. Note that if inputs are of different sizes gaps may appear, as not all of the output video frame will be filled. Similarly, videos can overlap each other if their position doesn’t leave enough space for the full frame of adjoining videos. For 2 inputs, a default layout of <code class="code">0_0|w0_0</code> (equivalent to <code class="code">grid=2x1</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">grid</samp></dt> <dd> Specify a fixed size grid of inputs. This option is used to create a fixed size grid of the input streams. Set the grid size in the form <code class="code">COLUMNSxROWS</code>. There must be <code class="code">ROWS * COLUMNS</code> input streams and they will be arranged as a grid with <code class="code">ROWS</code> rows and <code class="code">COLUMNS</code> columns. When using this option, each input stream within a row must have the same height and all the rows must have the same width. If <code class="code">grid</code> is set, then <code class="code">inputs</code> option is ignored and is implicitly set to <code class="code">ROWS * COLUMNS</code>. For 2 inputs, a default grid of <code class="code">2x1</code> (equivalent to <code class="code">layout=0_0|w0_0</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class="code">grid</code> or <code class="code">layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">fill</samp></dt> <dd> If set to valid color, all unused pixels will be filled with that color. By default fill is set to none, so it is disabled. </dd> </dl>

 <h4 class="subsection">30.293.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Display 4 inputs into 2x2 grid.

Layout: <pre class="example-preformatted">input1(0, 0) | input3(w0, 0) input2(0, h0) | input4(w0, h0) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|w0_0|w0_h
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 4 inputs into 1x4 grid.

Layout: <pre class="example-preformatted">input1(0,

0) input2(0, h0) input3(0, h0+h1) input4(0, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|0_h0+h1|0_
</pre>

Note that if inputs are of different widths, unused space will appear.
Display 9 inputs into 3x3 grid.

Layout: <pre class="example-preformatted">input1(0, 0) | input4(w0, 0) | input7(w0+w3, 0) input2(0, h0) | input5(w0, h0) | input8(w0+w3, h0) input3(0, h0+h1) | input6(w0, h0+h1) | input9(w0+w3, h0+h1) </pre>

 <pre class="example-preformatted">xstack=inputs=9:layout=0_0|0_h0|0_h0+h1|w
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 16 inputs into 4x4 grid.

Layout: <pre class="example-preformatted">input1(0, 0) | input5(w0, 0) | input9 (w0+w4, 0) | input13(w0+w4+w8, 0) input2(0, h0) | input6(w0, h0) | input10(w0+w4, h0) | input14(w0+w4+w8, h0) input3(0, h0+h1) | input7(w0, h0+h1) | input11(w0+w4, h0+h1) | input15(w0+w4+w8, h0+h1) input4(0, h0+h1+h2)| input8(w0, h0+h1+h2)| input12(w0+w4, h0+h1+h2)| input16(w0+w4+w8, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=16:layout=0_0|0_h0|0_h0+h1|
w0+w4_h0|w0+w4_h0+h1|w0+w4_h0+h1+h2|w0+w4+w8_0|w0+w4+w8_h0|w0+w4+w8_h0+h1|w0+w4-
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.

 <h3 class="section">30.294 yadif# TOC</h3>

Deinterlace the input video ("yadif" means "yet another deinterlacing filter"). It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt>

<dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">30.295 yaepblur# TOC</h3>

Apply blur filter while preserving edges ("yaepblur" means "yet another edge preserving blur filter"). The algorithm is described in "J. S. Lee, Digital image enhancement and noise filtering by use of local statistics, IEEE Trans. Pattern Anal. Mach. Intell. PAMI-2, 1980." It accepts the following parameters: <dl class="table"> <dt><samp class="option">radius, r</samp></dt> <dd> Set the window radius. Default value is 3. </dd> <dt><samp class="option">planes, p</samp></dt> <dd> Set which planes to filter. Default is only the first plane. </dd> <dt><samp class="option">sigma, s</samp></dt> <dd> Set blur strength. Default value is 128. </dd> </dl>

 <h4 class="subsection">30.295.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">30.296 zoompan<a class="anchor hidden-xs" href="#zoompan"

aria-hidden=”true”># TOC</h3>

Apply Zoom & Pan effect. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>zoom, z</samp></dt> <dd> Set the zoom expression. Range is 1-10. Default is 1. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the x and y expression. Default is 0. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Set the duration expression in number of frames. This sets for how many number of frames effect will last for single input image. Default is 90. </dd> <dt><samp class=”option”>s</samp></dt> <dd> Set the output image size, default is ’hd720’. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set the output frame rate, default is ’25’. </dd> </dl>

Each expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> Input width. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> Input height. </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dd> Output width. </dd> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> Output height. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> <dt><samp class=”option”>in_time, it</samp></dt> <dd> The input timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>out_time, time, ot</samp></dt> <dd> The output timestamp expressed in seconds. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Last calculated ’x’ and ’y’ position from ’x’ and ’y’ expression for current input frame. </dd> <dt><samp class=”option”>px</samp></dt> <dt><samp class=”option”>py</samp></dt> <dd> ’x’ and ’y’ of last output frame of previous input frame or 0 when there was not yet such frame (first input frame). </dd> <dt><samp class=”option”>zoom</samp></dt> <dd> Last calculated zoom from ’z’ expression for current input frame. </dd> <dt><samp class=”option”>pzoom</samp></dt> <dd> Last calculated zoom of last output frame of previous input frame. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> Number of output frames for current input frame. Calculated from ’d’ expression for each input frame. </dd> <dt><samp class=”option”>pduration</samp></dt> <dd> number of output frames created for previous input frame </dd> <dt><samp class=”option”>a</samp></dt> <dd> Rational number: input width / input height </dd> <dt><samp class=”option”>sar</samp></dt>

<dd> sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt>
<dd> display aspect ratio </dd> </dl>

 <h4 class=”subsection”>30.296.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Zoom in up to 1.5x and pan at same time to some spot near center of picture: <pre class=”example-preformatted”>zoompan=z='min(zoom+0.0015,1.5)':d=700:x='if(gte(zoom,1.5),x,x+1/a)':y=
</pre>

Zoom in up to 1.5x and pan always at center of picture: <pre class=”example-preformatted”>zoompan=z='min(zoom+0.0015,1.5)':d=700:x='iw/2
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Same as above but without pausing: <pre class=”example-preformatted”>zoompan=z='min(max(zoom,pzoom)+0.0015,1.5)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Zoom in 2x into center of picture only for the first second of the input video: <pre class=”example-preformatted”>zoompan=z='if(between(in_time,0,1),2,1)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

 <h3 class=”section”>30.297 zscale# TOC</h3>
Scale (resize) the input video, using the z.lib library: https://github.com/sekrit-twc/zimg. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libzimg</code>. The zscale filter forces the output display aspect ratio to be the same as the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the zscale filter will convert the input to the requested format. <h4 class=”subsection”>30.297.1 Options# TOC</h4> The filter accepts the following options. <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt>
<dd> Set the output video dimension expression. Default value is the input dimension. If the <var class=”var”>width</var> or <var class=”var”>w</var> value is 0, the input width is used for the output. If the

<var class=”var”>height</var> or <var class=”var”>h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the zscale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>dither, d</samp></dt> <dd> Set the dither type. Possible values are: <dl class=”table”> <dt><var class=”var”>none</var></dt> <dt><var class=”var”>ordered</var></dt> <dt><var class=”var”>random</var></dt> <dt><var class=”var”>error_diffusion</var></dt> </dl>

Default is none. </dd> <dt><samp class=”option”>filter, f</samp></dt> <dd> Set the resize filter type. Possible values are: <dl class=”table”> <dt><var class=”var”>point</var></dt> <dt><var class=”var”>bilinear</var></dt> <dt><var class=”var”>bicubic</var></dt> <dt><var class=”var”>spline16</var></dt> <dt><var class=”var”>spline36</var></dt> <dt><var class=”var”>spline64</var></dt> <dt><var class=”var”>lanczos</var></dt> </dl>

Default is bilinear. </dd> <dt><samp class=”option”>range, r</samp></dt> <dd> Set the color range. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>limited</var></dt> <dt><var class=”var”>full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>primaries, p</samp></dt> <dd> Set the color primaries. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>170m</var></dt> <dt><var class=”var”>240m</var></dt> <dt><var class=”var”>2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>transfer, t</samp></dt> <dd> Set the transfer characteristics. Possible values are: <dl class=”table”> <dt><var class=”var”>input</var></dt> <dt><var class=”var”>709</var></dt> <dt><var class=”var”>unspecified</var></dt> <dt><var class=”var”>601</var></dt> <dt><var class=”var”>linear</var></dt> <dt><var class=”var”>2020_10</var></dt> <dt><var class=”var”>2020_12</var></dt> <dt><var class=”var”>smpte2084</var></dt> <dt><var class=”var”>iec61966-2-1</var></dt> <dt><var class=”var”>arib-std-b67</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">rangein, rin</samp></dt> <dd> Set the input color range. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">limited</var></dt> <dt><var class="var">full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primariesin, pin</samp></dt> <dd> Set the input color primaries. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">240m</var></dt> <dt><var class="var">2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transferin, tin</samp></dt> <dd> Set the input transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">601</var></dt> <dt><var class="var">linear</var></dt> <dt><var class="var">2020_10</var></dt> <dt><var class="var">2020_12</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrixin, min</samp></dt> <dd> Set the input colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt> <dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt> <dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt> </dl>

</dd> <dt><samp class="option">chromal, c</samp></dt> <dd> Set the output chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt> <dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt> <dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt> </dl>

</dd> <dt><samp class="option">chromalin, cin</samp></dt> <dd> Set the input chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt>

<dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt>
<dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt>
</dl>

</dd> <dt><samp class="option">npl</samp></dt> <dd> Set the nominal peak luminance. </dd> <dt><samp class="option">param_a</samp></dt>
<dd> Parameter A for scaling filters. Parameter "b" for bicubic, and the number of filter taps for lanczos. </dd> <dt><samp class="option">param_b</samp></dt> <dd> Parameter B for scaling filters. Parameter "c" for bicubic. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt>
<dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd>
<dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt>
<dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt>
<dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt>
<dd> The same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt>
<dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt>
<dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt>
<dt><var class="var">ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">30.297.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h2 class="chapter">31 CUDA Video Filters# TOC</h2>

Below is a description of the currently available Nvidia CUDA video filters. Prerequisites: <ul class="itemize mark-bullet"> Install Nvidia CUDA Toolkit

Note: If FFmpeg detects the Nvidia CUDA Toolkit during configuration, it will enable CUDA filters automatically without requiring any additional flags. If you want to explicitly enable them, use the following options: <ul class="itemize mark-bullet"> Configure FFmpeg with <code class="code">--enable-cuda-nvcc --enable-nonfree</code>. Configure FFmpeg with <code class="code">--enable-cuda-llvm</code>. Additional requirement: <code class="code">llvm</code> lib must be installed.

Running CUDA filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device cuda[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</samp
<dd> Initialise a new hardware device of type <var class="var">cuda</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of initializing second CUDA device on the system and running scale_cuda and bilateral_cuda filters. <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -init_hw_device cuda:1 -filter_complex \ "[0:v]scale_cuda=format=yuv444p[scaled_video];[sca
\ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

Since CUDA filters operate exclusively on GPU memory, frame data must sometimes be uploaded (hwupload) to hardware surfaces associated with the appropriate CUDA device before processing, and downloaded (hwdownload) back to normal memory afterward, if required. Whether hwupload or hwdownload is necessary depends on the specific workflow: <ul class="itemize mark-bullet"> If the input frames are already in GPU memory (e.g., when using <code class="code">-hwaccel cuda</code> or <code class="code">-hwaccel_output_format cuda</code>), explicit use of <a class="ref"

href=”#hwupload”>hwupload is not needed, as the data is already in the appropriate memory space. If the input frames are in CPU memory (e.g., software-decoded frames or frames processed by CPU-based filters), it is necessary to use hwupload to transfer the data to GPU memory for CUDA processing. If the output of the CUDA filters needs to be further processed by software-based filters or saved in a format not supported by GPU-based encoders, hwdownload is required to transfer the data back to CPU memory. Note that hwupload uploads data to a surface with the same layout as the software frame, so it may be necessary to add a format filter immediately before hwupload to ensure the input is in the correct format. Similarly, hwdownload may not support all output formats, so an additional format filter may need to be inserted immediately after hwdownload in the filter graph to ensure compatibility. <h3 class=”section”>31.1 bilateral_cuda# TOC</h3> CUDA accelerated bilateral filter, an edge preserving filter. This filter is mathematically accurate thanks to the use of GPU acceleration. For best output quality, use one to one chroma subsampling, i.e. yuv444p format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight, also called sigma space. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate color range weight, also called sigma color. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>window_size</samp></dt> <dd> Set window size of the bilateral function to determine the number of neighbours to loop on. If the number entered is even, one will be added automatically. Allowed range is 1 to 255. Default is 1. </dd> </dl> <h4 class=”subsection”>31.1.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the bilateral filter on a video.

 <pre class=”example-preformatted”>./ffmpeg -v verbose \ -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]scale_cuda=format=yuv444p[scaled_video]; [scaled_video]bilateral_cuda=window_size=9:sigmaS=3.0:sigmaR=50.0" \ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

 <h3 class="section">31.2 bwdif_cuda# TOC</h3>

Deinterlace the input video using the bwdif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">31.3 chromakey_cuda# TOC</h3> CUDA accelerated YUV colorspace color/chroma keying. This filter works like normal chromakey filter but operates on CUDA frames. for more details and parameters see chromakey. <h4 class="subsection">31.3.1 Examples<span class="pull-

right”>#
TOC</h4>

<ul class=”itemize mark-bullet”> Make all the green pixels in the input video transparent and use it as an overlay for another video:

 <pre class=”example-preformatted”>./ffmpeg \
-hwaccel cuda -hwaccel_output_format cuda -i input_green.mp4 \ -hwaccel cuda -hwaccel_output_format cuda -i base_video.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]chromakey_cuda=0x25302D:0.1:0.12:1[overlay_video]; \ [1:v]scale_cuda=format=yuv420p[base]; \ [base][overlay_video]overlay_cuda" \ -an -sn -c:v h264_nvenc -cq 20 output.mp4 </pre>

Process two software sources, explicitly uploading the frames:

 <pre class=”example-preformatted”>./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda \ -f lavfi -i color=size=800x600:color=white,format=yuv42 \ -f lavfi -i yuvtestsrc=size=200x200,format=yuv420p \ -filter_complex \ " \ [0]hwupload[under]; \ [1]hwupload,chromakey_cuda=green:0.1:0.12[over]; \ [under][over]overlay_cuda" \ -c:v hevc_nvenc -cq 18 -preset slow output.mp4 </pre>

 <h3 class=”section”>31.4 colorspace_cuda# TOC</h3>

CUDA accelerated implementation of the colorspace filter. It is by no means feature complete compared to the software colorspace filter, and at the current time only supports color range conversion between jpeg/full and mpeg/limited range. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> </dl>

 <h3 class=”section”>31.5 overlay_cuda# TOC</h3>

Overlay one video on top of another. This is the CUDA variant of the overlay filter. It only accepts CUDA frames. The underlying input pixel formats have to match. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. They can contain the following parameters: <dl class=”table”> <dt><samp class=”option”>main_w, W</samp></dt> <dt><samp class=”option”>main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class=”option”>overlay_w, w</samp></dt> <dt><samp class=”option”>overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The ordinal index of the main input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> The byte offset position in the file of the main input frame, NAN if unknown. Deprecated, do not use. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp of the main input frame, expressed in seconds, NAN if unknown. </dd> </dl>

Default value is "0" for both expressions. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for <samp class=”option”>x</samp> and <samp class=”option”>y</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>init</samp></dt> <dd> Evaluate expressions once during filter initialization or when a command is processed. </dd> <dt><samp class=”option”>frame</samp></dt> <dd> Evaluate expressions for each incoming frame </dd> </dl>

Default value is <samp class=”option”>frame</samp>. </dd> <dt><samp class=”option”>eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options.

 <h3 class=”section”>31.6 pad_cuda<a

class=”anchor hidden-xs” href=”#pad_005fcuda” aria-hidden=”true”>#
TOC</h3>
Add paddings to an input video stream using CUDA. This filter is the CUDA-accelerated version of the pad filter. It accepts the same options and expressions and provides the same core functionality. For a detailed description of available options, please see the documentation for the pad filter. <h4 class=”subsection”>31.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Add a 200-pixel black border to all sides of a video frame: <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=iw+400:h=ih+400:x=200:y=200" -c:v h264_nvenc out.mp4 </pre>

Pad the input video to a 16:9 aspect ratio, filling with the color "blue": <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=ih*16/9/sar:h=ih:x=(ow-iw)/2:y=(oh-ih)/2:color=blue" -c:v h264_nvenc out.mp4 </pre>

 <h3 class=”section”>31.7 scale_cuda#
TOC</h3>

Scale (resize) and convert (pixel format) the input video, using accelerated CUDA kernels. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>interp_algo</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>nearest</var></dt> <dd> Nearest neighbour Used by default if input parameters match the desired output. </dd> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic This is the default. </dd> <dt><var class=”var”>lanczos</var></dt> <dd> High-quality separable Lanczos (windowed sinc). The filter radius can be set with <samp class=”option”>param</samp>; the default is 3. </dd> </dl>

</dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. Converting between <code class="code">yuv420p</code>, <code class="code">yuv422p</code>, <code class="code">yuv444p</code>, <code class="code">yuv420p10</code>, <code class="code">yuv422p10</code>, <code class="code">yuv444p10</code>, <code class="code">nv12</code>, <code class="code">nv16</code>, <code class="code">p010</code> or <code class="code">p210</code> and <code class="code">bgra</code>, <code class="code">rgba</code>, <code class="code">bgr0</code> or <code class="code">rgb0</code> is supported, also together with resizing. Resizing is then done on the RGB side, after converting from YUV or before converting to YUV, and always uses the generic filters selected with <samp class="option">interp_algo</samp>. Resizing the YUV side in a separate instance instead needs less intermediate memory and may be faster, but filters chroma differently. YUV to RGB conversion uses the YCbCr matrix and range of the input, RGB to YUV conversion those selected for the output (see <samp class="option">out_color_matrix</samp> and <samp class="option">out_range</samp>). An unspecified matrix is treated as BT.601 and an unspecified range as limited. The conversion samples chroma independently of <samp class="option">interp_algo</samp>: YUV to RGB conversion uses the nearest chroma sample, RGB to YUV conversion averages, which centers the chroma samples. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> If set to 0, every frame is processed, even if no conversion is necessary. This mode can be useful to use the filter as a buffer for a downstream frame-consumer that exhausts the limited decoder frame pool. If set to 1, frames are passed through as-is if they match the desired output parameters. This is the default behaviour. </dd> <dt><samp class="option">use_filters</samp></dt> <dd> If set to 1, filter with a generic weight LUT instead of using fixed-function shader kernels. May be faster or slower depending on the hardware. A value of <code class="code">auto</code> (the default) enables this automatically when required for correct anti-aliasing when downscaling, and for high-quality Lanczos scaling. Set this to 0 to use the legacy fixed 4-tap Lanczos kernel. Resizing during YUV to RGB or RGB to YUV conversion always uses the generic filters. </dd> <dt><samp class="option">param</samp></dt> <dd> Algorithm-specific parameter. For <var class="var">bicubic</var>, this affects the filter curve. For <var class="var">lanczos</var>, this sets the filter radius from 1 to 10 (default 3) when using the generic filter path; it has no effect with <samp class="option">use_filters=0</samp>. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class="option">out_range</samp></dt>

<dd> Set the output YCbCr sample range. One of <code class=”code”>auto</code> (the default), <code class=”code”>jpeg</code>/<code class=”code”>full</code>/<code class=”code”>pc</code>, or <code class=”code”>mpeg</code>/<code class=”code”>limited</code>/<code class=”code”>tv</code>. </dd> <dt><samp class=”option”>out_color_matrix</samp></dt> <dd> Set the output YCbCr matrix. One of <code class=”code”>auto</code> (the default), <code class=”code”>bt601</code>, <code class=”code”>smpte170m</code>, <code class=”code”>bt470</code>, <code class=”code”>bt470bg</code>, <code class=”code”>bt709</code>, <code class=”code”>fcc</code>, <code class=”code”>smpte240m</code>, <code class=”code”>ycgco</code>, <code class=”code”>bt2020</code> or <code class=”code”>bt2020nc</code>. With <code class=”code”>auto</code>, <samp class=”option”>out_range</samp> and <samp class=”option”>out_color_matrix</samp> are left to filter-graph negotiation. They follow a single range and matrix negotiated for the input, as for decoded video, and otherwise prefer BT.601 limited range. For RGB to YUV conversion, <code class=”code”>auto</code> thus means BT.601 limited range, unless the filter graph requires otherwise; converting full range YUV to RGB and back keeps the full range. Outside RGB to YUV conversion, these options describe the existing samples without remapping them; an unspecified input description may adopt the negotiated output description, while conflicting specified descriptions are rejected. Both options can only be used with YUV output; RGB output is full range. </dd> </dl>

 <h4 class=”subsection”>31.7.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. <pre class=”example-preformatted”>scale_cuda=-2:720:format=yuv420p </pre>

Upscale to 4K using nearest neighbour algorithm. <pre class=”example-preformatted”>scale_cuda=4096:2160:interp_algo=nearest </pre>

Don’t do any conversion or scaling, but copy all input frames into newly allocated ones. This can be useful to deal with a filter and encode chain that otherwise exhausts the decoders frame pool. <pre class=”example-preformatted”>scale_cuda=passthrough=0 </pre>

Convert BT.709 input to RGB for a filter that requires it, then back to BT.709 NV12 with limited range. <pre class=”example-preformatted”>scale_cuda=format=bgra,scale_cuda=format=nv12:out_color_matrix=bt70 </pre>

Scale to 720p while converting to RGB.

<pre class="example-preformatted">scale_cuda=-2:720:format=bgra
</pre>

 <h3 class="section">31.8 thumbnail_cuda# TOC</h3>

Select the most representative frame in a given sequence of consecutive frames using CUDA. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">31.8.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Thumbnails are extracted from every <var class="var">n</var>=150-frame batch, selecting one per batch. Chosen frames are then scaled with scale_cuda.
 <pre class="example-preformatted">./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i ./input.mp4 -vf "thumbnail_cuda=150,scale_cuda=1920:
./output/out%03d.png </pre>

 <h3 class="section">31.9 transpose_cuda# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt>

<dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class="samp">reversal</samp>’</dt> <dd> Rotate by 180 degrees. </dd> <dt>‘<samp class="samp">hflip</samp>’</dt> <dd> Flip horizontally. </dd> <dt>‘<samp class="samp">vflip</samp>’</dt> <dd> Flip vertically. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">31.10 yadif_cuda# TOC</h3>

Deinterlace the input video using the yadif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt> <dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1,

auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h2 class="chapter">32 OpenCL Video Filters# TOC</h2>

Below is a description of the currently available OpenCL video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-opencl</code>. Running OpenCL filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device opencl[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</samp> <dd> Initialise a new hardware device of type <var class="var">opencl</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device on the second platform and running avgblur_opencl filter with default parameters on it. <pre class="example-preformatted">-init_hw_device opencl=gpu:1.0 -filter_hw_device gpu -i INPUT -vf "hwupload, avgblur_opencl, hwdownload" OUTPUT </pre>

Since OpenCL filters are not able to access frame data in normal memory, all frame data needs to be uploaded(hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded(hwdownload) back to normal memory. Note that hwupload will upload to a surface with the same layout as the software frame, so it

may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it may be necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class=”section”>32.1 avgblur_opencl# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 1024]</code> and default value is <code class=”code”>0</code>. If zero, <code class=”code”>sizeX</code> value will be used. </dd> </dl>

 <h4 class=”subsection”>32.1.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply average blur filter with horizontal and vertical size of 3, setting each pixel of the output to the average value of the 7x7 region centered on it in the input. For pixels on the edges of the image, the region does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, avgblur_opencl=3, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.2 boxblur_opencl# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dt><samp class="option">chroma_radius, cr</samp></dt> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class="code">min(w,h)/2</code> for the luma and alpha planes, and of <code class="code">min(cw,ch)/2</code> for the chroma planes. Default value for <samp class="option">luma_radius</samp> is "2". If not specified, <samp class="option">chroma_radius</samp> and <samp class="option">alpha_radius</samp> default to the corresponding value set for <samp class="option">luma_radius</samp>. The expressions can contain the following constants: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class="option">cw</samp></dt> <dt><samp class="option">ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class="option">luma_power, lp</samp></dt> <dt><samp class="option">chroma_power, cp</samp></dt> <dt><samp class="option">alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class="option">luma_power</samp> is 2. If not specified, <samp class="option">chroma_power</samp> and <samp class="option">alpha_power</samp> default to the corresponding value set for <samp class="option">luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class="subsection">32.2.1 Examples# TOC</h4>

Apply boxblur filter, setting each pixel of the output to the average value of box-radiuses <var class="var">luma_radius</var>, <var class="var">chroma_radius</var>, <var class="var">alpha_radius</var> for each plane respectively. The filter will apply <var class="var">luma_power</var>, <var class="var">chroma_power</var>, <var class="var">alpha_power</var> times onto the corresponding plane. For pixels on the edges of the image, the radius does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <ul class="itemize mark-bullet"> Apply a boxblur filter with the luma, chroma, and

alpha radius set to 2 and luma, chroma, and alpha power set to 3. The filter will run 3 times with box-radius set to 2 for every plane of the image. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=luma_radius=2:luma_power=3, hwdownload" OUTPUT -i INPUT -vf "hwupload, boxblur_opencl=2:3, hwdownload" OUTPUT </pre>

Apply a boxblur filter with luma radius set to 2, luma_power to 1, chroma_radius to 4, chroma_power to 5, alpha_radius to 3 and alpha_power to 7.

For the luma plane, a 2x2 box radius will be run once. For the chroma plane, a 4x4 box radius will be run 5 times. For the alpha plane, a 3x3 box radius will be run 7 times. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=2:1:4:5:3:7, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.3 colorkey_opencl# TOC</h3>
RGB colorspace color keying. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt>
<dd> The color which will be replaced with transparency. </dd>
<dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage.
0.0 makes pixels either fully transparent, or not transparent at all.
Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h4 class=”subsection”>32.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Make every semi-green pixel in the input transparent with some slight blending:
<pre class=”example-preformatted”>-i INPUT -vf "hwupload, colorkey_opencl=green:0.3:0.1, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.4 convolution_opencl# TOC</h3

Apply convolution of 3x3, 5x5, 7x7 matrix. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>0m</samp></dt> <dt><samp class=”option”>1m</samp></dt> <dt><samp class=”option”>2m</samp></dt> <dt><samp class=”option”>3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed numbers. Default value for each plane is <code class=”code”>0 0 0 0 1 0 0 0 0</code>. </dd> <dt><samp class=”option”>0rdiv</samp></dt> <dt><samp class=”option”>1rdiv</samp></dt> <dt><samp class=”option”>2rdiv</samp></dt> <dt><samp class=”option”>3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be sum of all matrix elements. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>0bias</samp></dt> <dt><samp class=”option”>1bias</samp></dt> <dt><samp class=”option”>2bias</samp></dt> <dt><samp class=”option”>3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. The option value must be a float number greater or equal to <code class=”code”>0.0</code>. Default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>32.4.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sharpen: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0, hwdownload" OUTPUT </pre>

Apply blur: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9, hwdownload" OUTPUT </pre>

Apply edge enhance: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply edge detect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1

1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0, hwdownload" OUTPUT </pre>

Apply emboss: <pre class="example-preformatted">-i INPUT -vf "hwupload, convolution_opencl=-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2, hwdownload" OUTPUT </pre>

 <h3 class="section">32.5 erosion_opencl# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">32.5.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply erosion filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local minimum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local minimum is more then threshold of the corresponding plane, output pixel will be set to input pixel - threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, erosion_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">32.6 deshake_opencl# TOC</h3> Feature-point based video stabilization filter. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>tripod</samp></dt> <dd> Simulates a tripod by preventing any camera movement whatsoever from the original frame. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>debug</samp></dt> <dd> Whether or not additional debug info should be displayed, both in the processed output and in the console. Note that in order to see console debug output you will also need to pass <code class=”code”>-v verbose</code> to ffmpeg. Viewing point matches in the output video is only supported for RGB input. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>adaptive_crop</samp></dt> <dd> Whether or not to do a tiny bit of cropping at the borders to cut down on the amount of mirrored pixels. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>refine_features</samp></dt> <dd> Whether or not feature points should be refined at a sub-pixel level. This can be turned off for a slight performance gain at the cost of precision. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>smooth_strength</samp></dt> <dd> The strength of the smoothing applied to the camera path from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. <code class=”code”>1.0</code> is the maximum smoothing strength while values less than that result in less smoothing. <code class=”code”>0.0</code> causes the filter to adaptively choose a smoothing strength on a per-frame basis. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>smooth_window_multiplier</samp>
<dd> Controls the size of the smoothing window (the number of frames buffered to determine motion information from). The size of the smoothing window is determined by multiplying the framerate of the video by this number. Acceptable values range from <code class=”code”>0.1</code> to <code class=”code”>10.0</code>. Larger values increase the amount of motion data available for determining how to smooth the camera path, potentially improving smoothness, but also increase latency and memory usage. Defaults to <code class=”code”>2.0</code>. </dd> </dl>

 <h4 class=”subsection”>32.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Stabilize a video with a fixed, medium smoothing strength: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, deshake_opencl=smooth_strength=0.5, hwdownload" OUTPUT </pre>

Stabilize a video with debugging (both in console and in rendered video): <pre class=”example-preformatted”>-i INPUT -filter_complex "[0:v]format=rgba, hwupload, deshake_opencl=debug=1,

hwdownload, format=rgba, format=yuv420p" -v verbose OUTPUT </pre>

 <h3 class="section">32.7 dilation_opencl# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class="code">[0, 65535]</code> and default value is <code class="code">65535</code>. If <code class="code">0</code>, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class="code">[0, 255]</code> and default value is <code class="code">255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class="code">x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class="subsection">32.7.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply dilation filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local maximum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local maximum is more then threshold of the corresponding plane, output pixel will be set to input pixel + threshold of corresponding plane. <pre class="example-preformatted">-i INPUT -vf "hwupload, dilation_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class="section">32.8 nlmeans_opencl# TOC</h3>

Non-local Means denoise filter through OpenCL, this filter accepts same options as nlmeans. <h3 class="section">32.9 overlay_opencl<a class="anchor hidden-xs"

href=”#overlay_005fopencl” aria-hidden=”true”># TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires same memory layout for all the inputs. So, format conversion may be needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set the x coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>32.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs are yuv420p format. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre> The inputs have same memory layout for color channels , the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.10 pad_opencl# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class=”var”>x</var>, <var class=”var”>y</var> coordinates. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt>

<dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offsets as specified by the <var class="var">x</var> and <var class="var">y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> </dl>

 <h3 class="section">32.11 prewitt_opencl# TOC</h3>

Apply the Prewitt operator (<a class="url" href="https://en.wikipedia.org/wiki/Prewitt_operator">http to input video stream. The filter accepts the following option:

 <dl class="table"> <dt><samp class="option">planes</samp></dt>
<dd> Set which planes to filter. Default value is <code class="code">0xf</code>,
by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt>
<dd> Set value which will be multiplied with filtered result. Range is
<code class="code">[0.0, 65535]</code> and default value is <code
class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt>
<dd> Set value which will be added to filtered result. Range is
<code class="code">[-65535, 65535]</code> and default value is <code
class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">32.11.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply the Prewitt operator with scale
set to 2 and delta set to 10. <pre class="example-
preformatted">-i INPUT -vf "hwupload, prewitt_opencl=scale=2:delta=10,
hwdownload" OUTPUT </pre>

 <h3 class="section">32.12 program_opencl# TOC</h3>

Filter video using an OpenCL program. <dl class="table">
<dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt>
<dd> Kernel name in program. </dd> <dt><samp class="option">inputs</samp></dt>
<dd> Number of inputs to the filter. Defaults to 1. </dd>
<dt><samp class="option">size, s</samp></dt> <dd> Size of output
frames. Defaults to the same as the first input. </dd> </dl>

The <code class="code">program_opencl</code> filter also supports the
framesync options. The
program source file must contain a kernel function with the given name, which
will be run once for each plane of the output. Each run on a plane gets enqueued
as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a
pixel in the destination image. The kernel function needs to take the
following arguments: <ul class="itemize mark-bullet"> Destination
image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it.
Frame index, <var class="var">unsigned int</var>.

This is a counter starting from zero and increasing by one for each
frame. Source images, <var class="var">__read_only

image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to.

Example programs: <ul class=”itemize mark-bullet”> Copy the input to the output (output must be the same size as the input). <pre class=”verbatim”>___kernel void copy(___write_only image2d_t destination, unsigned int index, ___read_only image2d_t source) { const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE;

int2 location = (int2)(get_global_id(0), get_global_id(1));

float4 value = read_imagef(source, sampler, location);

write_imagef(destination, location, value); } </pre> Apply a simple transformation, rotating the input by an amount increasing with the index counter. Pixel values are linearly interpolated by the sampler, and the output need not have the same dimensions as the input. <pre class=”verbatim”>___kernel void rotate_image(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float angle = (float)index / 100.0f;

float2 dst_dim = convert_float2(get_image_dim(dst)); float2 src_dim = convert_float2(get_image_dim(src));

float2 dst_cen = dst_dim / 2.0f; float2 src_cen = src_dim / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1));

float2 dst_pos = convert_float2(dst_loc) - dst_cen; float2 src_pos = { cos(angle) * dst_pos.x - sin(angle) * dst_pos.y, sin(angle) * dst_pos.x + cos(angle) * dst_pos.y }; src_pos = src_pos * src_dim / dst_dim;

float2 src_loc = src_pos + src_cen;

if (src_loc.x < 0.0f || src_loc.y < 0.0f || src_loc.x > src_dim.x || src_loc.y > src_dim.y) write_imagef(dst, dst_loc, 0.5f); else write_imagef(dst, dst_loc, read_imagef(src, sampler, src_loc)); } </pre> Blend two inputs together, with the amount of each input used varying with the index counter. <pre class=”verbatim”>___kernel void blend_images(___write_only image2d_t dst, unsigned int index, ___read_only image2d_t src1, ___read_only image2d_t src2) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float blend = (cos((float)index / 50.0f) + 1.0f) / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1)); int2 src1_loc = dst_loc * get_image_dim(src1) / get_image_dim(dst); int2 src2_loc = dst_loc * get_image_dim(src2) / get_image_dim(dst);

float4 val1 = read_imagef(src1, sampler, src1_loc); float4 val2 = read_imagef(src2, sampler, src2_loc);

write_imagef(dst, dst_loc, val1 * blend + val2 * (1.0f - blend)); } </pre>

 <h3 class="section">32.13 remap_opencl# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 32bit float pixel format, single channel. <dl class="table"> <dt><samp class="option">interp</samp></dt> <dd> Specify interpolation used for remapping of pixels. Allowed values are <code class="code">near</code> and <code class="code">linear</code>. Default value is <code class="code">linear</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">32.14 roberts_opencl# TOC</h3> Apply the Roberts cross operator (<a class="url" href="https://en.wikipedia.org/wiki/Roberts_cross">https://en.w to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">0xf</code>, by which all planes are processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class="code">[0.0, 65535]</code> and default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class="code">[-65535, 65535]</code> and default value is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">32.14.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the Roberts cross operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, roberts_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.15 sobel_opencl# TOC</h3>

Apply the Sobel operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Sobel_operator”>https://
to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>32.15.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sobel operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, sobel_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.16 tonemap_opencl# TOC</h3>

Perform HDR(PQ/HLG) to SDR conversion with tone-mapping. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tonemap</samp></dt> <dd> Specify the tone-mapping operator to be used. Same as tonemap option in tonemap. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Tune the tone mapping algorithm. same as param option in tonemap. </dd> <dt><samp class=”option”>desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by

(smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default value is 0.5, and the algorithm here is a little different from the cpu version tonemap currently. A setting of 0.0 disables this option. </dd> <dt><samp class="option">threshold</samp></dt> <dd> The tonemapping algorithm parameters is fine-tuned per each scene. And a threshold is used to detect whether the scene has changed or not. If the distance between the current frame average brightness and the current running average exceeds a threshold value, we would re-calculate scene average and peak brightness. The default value is 0.2. </dd> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Currently supported formats are: <dl class="table"> <dt><var class="var">p010</var></dt> <dt><var class="var">nv12</var></dt> </dl>

</dd> <dt><samp class="option">range, r</samp></dt> <dd> Set the output color range. Possible values are: <dl class="table"> <dt><var class="var">tv/mpeg</var></dt> <dt><var class="var">pc/jpeg</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Possible values are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is bt709. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> </dl>

 <h4 class="subsection">32.16.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(PQ/HLG) video to bt2020-transfer-characteristic p010 format using linear operator. <pre class="example-preformatted">-i INPUT -vf "format=p010,hwupload,tonemap_opencl=t=bt2020:tonemap=linear:format=p010,hwdownload,format OUTPUT </pre>

 <h3 class="section">32.17 unsharp_opencl<a class="anchor hidden-xs"

href=”#unsharp_005fopencl” aria-hidden=”true”># TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>luma_amount, la</samp></dt> <dd> Set the luma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>1.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> <dt><samp class=”option”>chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. Range is <code class=”code”>[1, 23]</code> and default value is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. Range is <code class=”code”>[-10, 10]</code> and default value is <code class=”code”>0.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class=”subsection”>32.17.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply strong luma sharpen effect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=luma_msize_x=7:luma_msize_y=7:luma_amount=2.5, hwdownload" OUTPUT </pre>

Apply a strong blur of both luma and chroma parameters: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=7:7:-2:7:7:-2, hwdownload" OUTPUT </pre>

 <h3 class=”section”>32.18 xfade_opencl# TOC</h3>

Cross fade two videos with custom transition effect by using OpenCL.

 It accepts the following options: <dl class="table"> <dt><samp class="option">transition</samp></dt> <dd> Set one of possible transition effects. <dl class="table"> <dt><samp class="option">custom</samp></dt> <dd> Select custom transition effect, the actual transition description will be picked from source and kernel options. </dd> <dt><samp class="option">fade</samp></dt> <dt><samp class="option">wipeleft</samp></dt> <dt><samp class="option">wiperight</samp></dt> <dt><samp class="option">wipeup</samp></dt> <dt><samp class="option">wipedown</samp></dt> <dt><samp class="option">slideleft</samp></dt> <dt><samp class="option">slideright</samp></dt> <dt><samp class="option">slideup</samp></dt> <dt><samp class="option">slidedown</samp></dt> <dd> Default transition is fade. </dd> </dl>

</dd> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file for custom transition. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Set name of kernel to use for custom transition from program source file. </dd> <dt><samp class="option">duration</samp></dt> <dd> Set duration of video transition. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set time of start of transition relative to first video. </dd> </dl>

The program source file must contain a kernel function with the given name, which will be run once for each plane of the output. Each run on a plane gets enqueued as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a pixel in the destination image. The kernel function needs to take the following arguments: <ul class="itemize mark-bullet"> Destination image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it. First Source image, <var class="var">__read_only image2d_t</var>. Second Source image, <var class="var">__read_only image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to. Transition progress, <var class="var">float</var>. This value is always between 0 and 1 inclusive.

Example programs: <ul class="itemize mark-bullet"> Apply dots curtain transition effect: <pre class="verbatim">__kernel void blend_images(__write_only image2d_t dst, __read_only image2d_t src1, __read_only image2d_t src2, float progress) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR); int2 p = (int2)(get_global_id(0), get_global_id(1)); float2 rp = (float2)(get_global_id(0), get_global_id(1)); float2 dim = (float2)(get_image_dim(src1).x, get_image_dim(src1).y); rp = rp / dim;

float2 dots = (float2)(20.0, 20.0); float2 center = (float2)(0,0); float2 unused;

float4 val1 = read_imagef(src1, sampler, p); float4 val2 = read_imagef(src2, sampler, p); bool next = distance(fract(rp * dots, &unused), (float2)(0.5, 0.5)) < (progress / distance(rp, center));

write_imagef(dst, p, next ? val1 : val2); } </pre>

 <h2 class="chapter">33 VAAPI Video Filters# TOC</h2>

VAAPI Video filters are usually used with VAAPI decoder and VAAPI encoder. Below is a description of VAAPI video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vaapi</code>. To use vaapi filters, you need to setup the vaapi device correctly. For more information, please read <a class="url" href="https://trac.ffmpeg.org/wiki/Hardware/VAAPI">https://trac.ffmpeg.org/wiki/Hardware/
  <h3 class="section">33.1 overlay_vaapi# TOC</h3>

Overlay one video on the top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set expressions for the width and height the overlaid video on the main video. Default values are ’overlay_iw’ for ’w’ and ’overlay_ih*w/overlay_iw’ for ’h’. The expressions can contain the following parameters: <dl class="table"> <dt><samp class="option">main_w, W</samp></dt> <dt><samp class="option">main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class="option">overlay_iw</samp></dt> <dt><samp class="option">overlay_ih</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class="option">overlay_w, w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt> <dd> The overlay output width and height. </dd> <dt><samp class="option">overlay_x, x</samp></dt> <dt><samp class="option">overlay_y, y</samp></dt> <dd> Position of the overlay layer inside of main </dd> </dl>

</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set transparency of overlaid video. Allowed range is 0.0 to 1.0. Higher value

means lower transparency. Default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">33.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs for this filter are yuv420p format. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_vaapi" OUTPUT </pre> Overlay an image LOGO at the offset (200, 100) from the top-left corner of the INPUT video. The inputs have same memory layout for color channels, the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_vaapi=x=200:y=100:w=400:h=300:alpha=1.0, hwdownload, format=nv12" OUTPUT </pre>

 <h3 class="section">33.2 tonemap_vaapi# TOC</h3>

Perform HDR-to-SDR or HDR-to-HDR tone-mapping. It currently only accepts HDR10 as input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Default is nv12 for HDR-to-SDR tone-mapping and p010 for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping.

</dd> <dt><samp class=”option”>display</samp></dt> <dd> Set the output mastering display colour volume. It is given by a ’|’-separated list of two values, two values are space separated. It set display primaries x & y in G, B, R order, then white point x & y, the nominal minimum & maximum display luminances. HDR-to-HDR tone-mapping will be performed when this option is set. </dd> <dt><samp class=”option”>light</samp></dt> <dd> Set the output content light level information. It accepts 2 space-separated values, the first input is the maximum light level and the second input is the maximum average light level. It is ignored for HDR-to-SDR tone-mapping, and optional for HDR-to-HDR tone-mapping. </dd> </dl>

 <h4 class=”subsection”>33.2.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Convert HDR(HDR10) video to bt2020-transfer-characteristic p010 format <pre class=”example-preformatted”>tonemap_vaapi=format=p010:t=bt2020-10 </pre> Convert HDR video to HDR video <pre class=”example-preformatted”>tonemap_vaapi=display=7500\ 3000|34000\ 16000|13250\ 34500|15635\ 16450|500\ 10000000 </pre>

 <h3 class=”section”>33.3 hstack_vaapi# TOC</h3> Stack input videos horizontally. This is the VA-API variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class=”section”>33.4 vstack_vaapi# TOC</h3> Stack input videos vertically. This is the VA-API variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping

the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">33.5 xstack_vaapi# TOC</h3> Stack video inputs into custom layout. This is the VA-API variant of the xstack filter, each input stream may have different size, this filter will scale down/up each input stream to the given output size, or the size of the first input stream. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_vaapi=inputs=4:layout=0_0_1920x1080|0_1
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h3 class="section">33.6 pad_vaapi# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an

expression for the size of the output image with the paddings added. If the value for <var class=”var”>width</var> or <var class=”var”>height</var> is 0, the corresponding input size is used for the output. The <var class=”var”>width</var> expression can reference the value set by the <var class=”var”>height</var> expression, and vice versa. The default value of <var class=”var”>width</var> and <var class=”var”>height</var> is 0. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice versa. The default value of <var class=”var”>x</var> and <var class=”var”>y</var> is 0. If <var class=”var”>x</var> or <var class=”var”>y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var>, and <var class=”var”>y</var> options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> </dl>

 <h3 class="section">33.7 drawbox_vaapi# TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class="code">fill</code> will create a filled box. Default value is <code class="code">3</code>. See below for the list of accepted constants. </dd> <dt><samp class="option">replace</samp></dt> <dd> With value <code class="code">1</code>, the pixels of the painted box will overwrite the video’s color and alpha pixels. Default is <code class="code">0</code>, which composites the box onto the input video. </dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var class="var">t</var> are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_h, ih</samp></dt> <dt><samp class="option">in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. </dd> </dl>

 <h4 class="subsection">33.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h2 class="chapter">34 VideoToolbox Video Filters# TOC</h2>

Below is a description of the currently available VideoToolbox video filters. VideoToolbox filter depends on VideoToolbox framework, and is auto detected when building ffmpeg for an Apple platform such as macOS. Add <code class="code">--enable-videotoolbox</code> to configure if autodetect is disabled. <h3 class="section">34.1 scale_vt# TOC</h3>

Scale and convert the color parameters using VTPixelTransferSession. The filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. </dd> <dt><samp class="option">color_matrix</samp></dt> <dd> Set the output colorspace matrix. </dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the output color primaries. </dd> <dt><samp class="option">color_transfer</samp></dt> <dd> Set the output transfer characteristics. </dd> </dl>

 <h4 class="subsection">34.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Perform HDR to SDR conversion, and scale to half size of input <pre class="example-

preformatted”>ffmpeg -hwaccel videotoolbox \ -hwaccel_output_format videotoolbox_vld \ -i hdr.mov \ -c:v hevc_videotoolbox \ -profile:v main \ -b:v 3M \ -vf scale_vt=w=iw/2:h=ih/2:color_matrix=bt709:color_primaries=bt709:color_transfer=bt709 \ -c:a copy \ -tag:v hvc1 \ sdr.mp4 </pre>

 <h3 class=”section”>34.2 transpose_vt# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class=”table”> <dt>‘<samp class=”samp”>cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class=”samp”>clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class=”samp”>cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class=”samp”>clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class=”samp”>hflip</samp>’</dt> <dd> Flip the input video horizontally. </dd> <dt>‘<samp class=”samp”>vflip</samp>’</dt> <dd> Flip the input video vertically. </dd> </dl>

</dd> <dt><samp class=”option”>passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class=”samp”>portrait</samp>’</d <dd> Preserve portrait geometry (when <var class=”var”>height</var> >= <var class=”var”>width</var>). </dd> <dt>‘<samp class=”samp”>landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class=”var”>width</var> >= <var class=”var”>height</var>). </dd> </dl>

</dd> </dl>

 <h2 class=”chapter”>35 Vulkan Video Filters# TOC</h2>

Below is a description of the currently available Vulkan video fil-

ters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vulkan</code>, and have a SPIR-V compiler (<code class="code">glslc</code> or <code class="code">glslangValidator</code>) available at build time. Running Vulkan filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device vulkan[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">vulkan</var> called <var class="var">name</var>, using the given device parameters and options in <var class="var">key=value</var>. The following options are supported: <dl class="table"> <dt><samp class="option">debug</samp></dt> <dd> Switches validation layers on if set to 1. </dd> <dt><samp class="option">linear_images</samp></dt> <dd> Allocates linear images. Does not apply to decoding. </dd> <dt><samp class="option">disable_multiplane</samp></dt> <dd> Disables multiplane images. Does not apply to decoding. </dd> <dt><samp class="option">avoid_host_import</samp></dt> <dd> Avoids the use of dynamic host memory imports, in favor of a regular memcpy() into a previously mapped buffer. </dd> </dl>

</dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device and running nlmeans_vulkan filter with default parameters on it. <pre class="example-preformatted">-init_hw_device vulkan=vk:0 -filter_hw_device vk -i INPUT -vf "hwupload,nlmeans_vulkan,hwdownload" OUTPUT
</pre>

As Vulkan filters are not able to access frame data in normal memory, all frame data needs to be uploaded (hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded (hwdownload) back to normal memory. Note that hwupload will upload to a frame with the same layout as the software frame, so it may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it is usually necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class="section">35.1

avgblur_vulkan# TOC</h3>

Apply an average blur filter, implemented on the GPU using Vulkan. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class=”code”>[1, 32]</code> and default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> </dl>

 <h3 class=”section”>35.2 blend_vulkan# TOC</h3>

Blend two Vulkan frames into each other. The <code class=”code”>blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>c0_mode</samp></dt> <dt><samp class=”option”>c1_mode</samp></dt> <dt><samp class=”option”>c2_mode</samp></dt> <dt><samp class=”option”>c3_mode</samp></dt> <dt><samp class=”option”>all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class=”var”>all_mode</var>. Default value is <code class=”code”>normal</code>. Available values for component modes are: <dl class=”table”> <dt>‘<samp class=”samp”>normal</samp>’</dt> <dt>‘<samp class=”samp”>multiply</samp>’
</dl>

</dd> </dl>

 <h3 class=”section”>35.3 bwdif_vulkan# TOC</h3>

Deinterlacer using bwdif, the "Bob Weaver Deinterlacing Filter" algorithm, implemented on the GPU using Vulkan. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0,

send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">35.4 chromaber_vulkan# TOC</h3>

Apply an effect that emulates chromatic aberration. Works best with RGB inputs, but provides a similar effect with YCbCr inputs too. <dl class="table"> <dt><samp class="option">dist_x</samp></dt> <dd> Horizontal displacement multiplier. Each chroma pixel’s position will be multiplied by this amount, starting from the center of the image. Default is <code class="code">0</code>. </dd> <dt><samp class="option">dist_y</samp></dt> <dd> Similarly, this sets the vertical displacement multiplier. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">35.5 color_vulkan# TOC</h3>

Video source that creates a Vulkan frame of a solid color. Useful for benchmarking, or overlaying. It accepts the following parameters: <dl class="table"> <dt><samp class="option">color</samp></dt>

<dd> The color to use. Either a name, or a hexadecimal value. The default value is <code class="code">black</code>. </dd> <dt><samp class="option">size</samp></dt> <dd> The size of the output frame. Default value is <code class="code">1920x1080</code>. </dd> <dt><samp class="option">rate</samp></dt> <dd> The framerate to output at. Default value is <code class="code">60</code> frames per second. </dd> <dt><samp class="option">duration</samp></dt> <dd> The video duration. Default value is <code class="code">-0.000001</code>. </dd> <dt><samp class="option">sar</samp></dt> <dd> The video signal aspect ratio. Default value is <code class="code">1/1</code>. </dd> <dt><samp class="option">format</samp></dt> <dd> The pixel format of the output Vulkan frames. Default value is <code class="code">yuv444p</code>. </dd> <dt><samp class="option">out_range</samp></dt> <dd> Set the output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> </dl>

 <h3 class="section">35.6 vflip_vulkan# TOC</h3>

Flips an image vertically. <h3 class="section">35.7 hflip_vulkan# TOC</h3>

Flips an image horizontally. <h3 class="section">35.8 flip_vulkan# TOC</h3>

Flips an image along both the vertical and horizontal axis. <h3 class="section">35.9 fruc_vulkan# TOC</h3>

Frame rate up-conversion using the NVIDIA Vulkan optical flow extension (<code class="code">VK_NV_optical_flow</code>), implemented on the

GPU using Vulkan. For each pair of consecutive input frames the filter computes a forward and backward optical flow field on the device’s optical flow engine, and uses those fields to synthesise motion-compensated intermediate frames at the requested output frame rate. The frame timing logic mirrors the <code class="code">framerate</code> filter. The majority of YUV and RGB pixel formats are supported. The exceptions are subsampled packed YUV formats such as <code class="code">yuyv422</code> and <code class="code">uyvy422</code>, formats with more than 16 bits integer data per component, such as <code class="code">rgb96</code> and <code class="code">gray32</code>, and Bayer formats. <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> A string describing the desired output frame rate, evaluated as an expression. The following constants are available: <code class="code">source_fps</code>. The default value is <code class="code">60</code>. </dd> <dt><samp class="option">perf</samp></dt> <dd> Optical flow performance level, trading motion-estimation quality for speed. This is the dominant cost at high resolutions. Possible values are: <dl class="table"> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> Highest quality, slowest. This is the default. </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Balanced quality and speed. </dd> <dt>‘<samp class="samp">fast</samp>’</dt> <dd> Lowest quality, fastest. </dd> </dl>

</dd> <dt><samp class="option">grid</samp></dt> <dd> Optical flow output grid size in pixels. A coarser grid computes fewer flow vectors, making it faster and less memory hungry at the cost of flow detail. Possible values are <code class="code">auto</code> (the default; selects the finest grid the device supports, usually 1x1), <code class="code">1</code>, <code class="code">2</code>, <code class="code">4</code> and <code class="code">8</code>. Only grid sizes the device advertises are accepted. </dd> </dl>

 <h3 class="section">35.10 gblur_vulkan# TOC</h3>

Apply Gaussian blur filter on Vulkan frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class="code">sigma</code>. Default is <code class="code">-1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the kernel size along the

horizontal axis. Default is <code class="code">19</code>. </dd> <dt><samp class="option">sizeV</samp></dt> <dd> Set the kernel size along the vertical axis. Default is <code class="code">0</code>, which sets to use the same value as <var class="var">size</var>. </dd> </dl>

 <h3 class="section">35.11 nlmeans_vulkan# TOC</h3>

Denoise frames using Non-Local Means algorithm, implemented on the GPU using Vulkan. Supports more pixel formats than nlmeans or nlmeans_opencl, including alpha channel support. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength for all components. Default is 1.0. Must be in range [0.0, 100.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size for all planes. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">t</samp></dt> <dd> Set parallelism. Default is 8. Must be a number in the range [1, 64]. Larger values will use more VRAM but may not result in greater speed. The optimal value is hardware and input dependent. </dd> <dt><samp class="option">s0</samp></dt> <dt><samp class="option">s1</samp></dt> <dt><samp class="option">s2</samp></dt> <dt><samp class="option">s3</samp></dt> <dd> Set denoising strength for a specific component. Default is <var class="var">1.0</var>, equal to <samp class="option">s</samp>. Must be in range [0.0, 100.0]. 0.0 disables denoising in that component. </dd> <dt><samp class="option">p0</samp></dt> <dt><samp class="option">p1</samp></dt> <dt><samp class="option">p2</samp></dt> <dt><samp class="option">p3</samp></dt> <dd> Set patch size for a specific component. Default is <var class="var">7</var>, equal to <samp class="option">p</samp>. Must be odd number in range [0, 99]. </dd> </dl>

 <h3 class="section">35.12 overlay_vulkan# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires all inputs to use the same pixel format. So, format conversion may be needed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set the x coordinate of the over-

laid video on the main video. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">35.13 transpose_vulkan# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">35.14 v360_vulkan# TOC</h3>

Convert 360 videos between various formats. Supported formats (projections) are subset of v360 filter, that are most popular 360-degree video formats. The filter accepts subset of v360 options: <dl class="table"> <dt><samp class="option">input</samp></dt>

<dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">eac</samp>’</dt> <dd> Equi-Angular Cube-map. (Output only) FIXME: current output doesn’t contain padding. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Regular video. </dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt> <dd> Dual fisheye. </dd> <dt>‘<samp class="samp">sg</samp>’</dt> <dd> Stereographic format. </dd> <dt>‘<samp class="samp">fisheye</samp>’</dt> <dd> Fisheye projection. </dd> <dt>‘<samp class="samp">gopromax</samp>’</dt> <dd> GoPro Max .360 format. (Input only) A .360 video file contains two video streams (front and rear); with this input projection the filter takes two inputs and combines them into a single stream. See the <samp class="option">overlap</samp> option. </dd> </dl>

</dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video resolution. See v360 options. </dd> <dt><samp class="option">yaw</samp></dt> <dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt> <dt><samp class="option">rorder</samp></dt> <dd> Set rotation for the output video. See v360 options. </dd> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dd> Set output horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt> <dd> Set input horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number of overlapped pixels on input .360 video. Only used with <code class="code">input=gopromax</code>. The GoPro Max .360 video contains overlapped area (See https://gopro.com/news/max-tech-specs-stitching-resolution). The filter blends overlapped images in these two areas. The number of pixels of overlapped areas depends on the video resolution. In most cases, the value is automatically calculated based on input video size, but it may be changed on GoPro Max2 or future models. The option is to adjust for unknown video resolutions. </dd> </dl>

 <h4 class="subsection">35.14.1 Examples# TOC</h4> Special

notes exist to use with <code class=”code”>input=gopromax</code>. The GoPro Max .360 video file contains two video streams, and the stream number must be specified. The stream number (e.g., <code class=”code”>[0:5]</code>) depends on the model and shooting mode. Please check actual stream number with <code class=”code”>ffprobe</code> command. <ul class=”itemize mark-bullet”> Convert GoPro Max (8bit color depth) .360 to Equirectangular projection. GoPro Max uses obsolete yuvj420p pixel format, thus convert the format to yuv420p before this filter. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]format=yuv420p,hwupload[vf],[0:5]format=yuv420p,hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=e, hwdownload,format=yuv420p" OUTPUT.mkv </pre>

Convert GoPro Max2 (10bit color depth) .360 to Dual fisheye projection. GoPro Max2 uses yuv420p10le pixel format. <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]hwupload[vf],[0:4]hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=dfisheye, hwdownload,format=yuv420p10le" OUTPUT.mkv </pre>

 <h2 class=”chapter”>36 QSV Video Filters# TOC</h2>

Below is a description of the currently available QSV video filters. To enable compilation of these filters you need to configure FFmpeg with <code class=”code”>--enable-libmfx</code> or <code class=”code”>--enable-libvpl</code>. To use QSV filters, you need to setup the QSV device correctly. For more information, please read <a class=”url” href=”https://trac.ffmpeg.org/wiki/Hardware/QuickSync”>https://trac.ffmpeg.org/wiki/Hardware/QuickSy
  <h3 class=”section”>36.1 hstack_qsv# TOC</h3> Stack input videos horizontally. This is the QSV variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">36.2 vstack_qsv# TOC</h3> Stack input videos vertically. This is the QSV variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">36.3 xstack_qsv# TOC</h3> Stack video inputs into custom layout. This is the QSV variant of the xstack filter. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class="example-preformatted">xstack_qsv=inputs=4:layout=0_0_1920x1080|0_h0_1920x1080|w0_0_1920
</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h2 class="chapter">37 Video Sources# TOC</h2>

Below is a description of the currently available video sources. <h3 class="section">37.1 buffer# TOC</h3>

Buffer video frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">video_size</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">width</samp></dt> <dd> The input video width. </dd> <dt><samp class="option">height</samp></dt> <dd> The input video height. </dd> <dt><samp class="option">pix_fmt</samp></dt> <dd> A string representing the pixel format of the buffered video frames. It may be a number corresponding to a pixel format, or a pixel format name. </dd> <dt><samp class="option">time_base</samp></dt> <dd> Specify the timebase assumed by the timestamps of the buffered frames. </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Specify the frame rate expected for the video stream. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> A string representing the color space of the buffered video frames. It may be a number corresponding to a color space, or a color space name. </dd> <dt><samp class="option">range</samp></dt> <dd> A string representing the color range of the buffered video frames. It may be a number corresponding to a color range, or a color range name. </dd> <dt><samp class="option">alpha_mode</samp></dt> <dd> A string representing the alpha mode of the buffered video frames. It may be a number corresponding to an alpha mode, or an alpha mode name. </dd> <dt><samp class="option">chroma_location</samp></dt> <dd> A string representing the chroma sample location of the buffered video frames. It may be a number corresponding to a chroma sample location, or a chroma sample location name. When it is not set, the filtergraph chooses the chroma sample location of the output. Frames without a chroma sample location are tagged with the one negotiated for the output. </dd> <dt><samp class="option">pixel_aspect, sar</samp></dt> <dd> The sample (pixel) aspect ratio of the input video. </dd> <dt><samp class="option">hw_frames_ctx</samp></dt> <dd> When using a hardware pixel format, this should be a reference to an AVHWFramesContext describing input frames. </dd> </dl>

For example: <pre class="example-preformatted">buffer=width=320:height=240:pix_fmt=yuv410p:time_base=1/24:sar=1
</pre>

will instruct the source to accept video frames with size 320x240 and with format "yuv410p", assuming 1/24 as the timestamps timebase and square pixels (1:1 sample aspect ratio). Since the pixel format with name "yuv410p" corresponds to the number 6 (check the enum AVPixelFormat definition in <samp class="file">libavutil/pixfmt.h</samp>), this example corresponds to: <pre class="example-preformatted">buffer=size=320x240:pixfmt=6:time_base=1/24:pixel_aspect=1/1
</pre>

Alternatively, the options can be specified as a flat string, but this syntax is deprecated: <var class="var">width</var>:<var class="var">height</var>:<var class="var">pix_fmt</var>:<var class="var">time_base.num</var>:<var class="var">time_base.den</var>:<var class="var">pixel_aspect.num</var>:<var class="var">pixel_aspect.den</var> <h3 class="section">37.2 cellauto# TOC</h3>

Create a pattern generated by an elementary cellular automaton. The initial state of the cellular automaton can be defined through the <samp class="option">filename</samp> and <samp class="option">pattern</samp> options. If such options are not specified an initial state is created randomly. At each new frame a new row in the video is filled with the result of the cellular automaton next generation. The behavior when the whole frame is filled is defined by the <samp class="option">scroll</samp> option. This source accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified file. In the file, each non-whitespace character is considered an alive cell, a newline will terminate the row, and further characters in the file will be ignored. </dd> <dt><samp class="option">pattern, p</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified string. Each non-whitespace character in the string is considered an alive cell, a newline will terminate the row, and further characters in the string will be ignored. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set the video rate, that is the number of frames generated per second. Default is 25. </dd> <dt><samp class="option">random_fill_ratio, ratio</samp></dt> <dd> Set the random fill ratio for the initial cellular automaton row. It is a floating point number value ranging from 0 to 1, defaults to 1/PHI. This option is ignored when a file or a pattern is specified. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> Set the seed for filling randomly the initial row, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">rule</samp></dt> <dd> Set the cellular automaton rule, it is a number ranging from 0 to

255. Default value is 110. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the size of the output video. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. If <samp class="option">filename</samp> or <samp class="option">pattern</samp> is specified, the size is set by default to the width of the specified initial state row, and the height is set to <var class="var">width</var> * PHI. If <samp class="option">size</samp> is set, it must contain the width of the specified pattern string, and the specified pattern will be centered in the larger row. If a filename or a pattern string is not specified, the size value defaults to "320x518" (used for a randomly generated initial state). </dd> <dt><samp class="option">scroll</samp></dt> <dd> If set to 1, scroll the output upward when all the rows in the output have been already filled. If set to 0, the new generated row will be written over the top row just after the bottom row is filled. Defaults to 1. </dd> <dt><samp class="option">start_full, full</samp></dt> <dd> If set to 1, completely fill the output with generated rows before outputting the first frame. This is the default behavior, for disabling set the value to 0. </dd> <dt><samp class="option">stitch</samp></dt> <dd> If set to 1, stitch the left and right row edges together. This is the default behavior, for disabling set the value to 0. </dd> </dl>

 <h4 class="subsection">37.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read the initial state from <samp class="file">pattern</samp>, and specify an output of size 200x400. <pre class="example-preformatted">cellauto=f=pattern:s=200x400 </pre>

Generate a random initial row with a width of 200 cells, with a fill ratio of 2/3: <pre class="example-preformatted">cellauto=ratio=2/3:s=200x200 </pre>

Create a pattern generated by rule 18 starting by a single alive cell centered on an initial row with width 100: <pre class="example-preformatted">cellauto=p=@:s=100x400:full=0:rule=18 </pre>

Specify a more elaborated initial pattern: <pre class="example-preformatted">cellauto=p='@@ @ @@':s=100x400:full=0:rule=18 </pre>

<a name="coreimagesrc-

1”> <h3 class=”section”>37.3 coreimagesrc# TOC</h3> Video source generated on GPU using Apple’s CoreImage API on OSX. This video source is a specialized version of the coreimage video filter. Use a core image generator at the beginning of the applied filterchain to generate the content. The coreimagesrc video source accepts the following options: <dl class=”table”> <dt><samp class=”option”>list_generators</samp></dt> <dd> List all available generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class=”example-preformatted”>list_generators=true </pre>

</dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the size of the sourced video. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. The default value is <code class=”code”>320x240</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> </dl>

Additionally, all options of the coreimage video filter are accepted. A complete filterchain can be used for further processing of the generated input without CPU-HOST transfer. See coreimage documentation and examples for details. <h4 class=”subsection”>37.3.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class=”example-preformatted”>ffmpeg -f lavfi -i coreimagesrc=s=100x100:filter=CIQRCodeGenerator@inputMessage=https\\\\\://FFmpeg.org/@inputCorrecti

-frames:v 1 QRCode.png </pre> This example is equivalent to the QRCode example of coreimage without the need for a nullsrc video source.

 <h3 class=”section”>37.4 ddagrab# TOC</h3>

Captures the Windows Desktop via Desktop Duplication API. The filter exclusively returns D3D11 Hardware Frames, for on-gpu encoding or processing. So an explicit hwdownload is needed for any kind of software processing. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>output_idx</samp></dt> <dd> DXGI Output Index to capture. Usually corresponds to the index Windows has given the screen minus one, so it’s starting at 0. Defaults to output 0. </dd> <dt><samp class=”option”>draw_mouse</samp></dt> <dd> Whether to draw the mouse cursor. Defaults to true. Only affects hardware cursors. If a game or application renders its own cursor, it’ll always be captured. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Maximum framerate at which the desktop will be captured - the interval between successive frames will not be smaller than the inverse of the framerate. When <var class=”var”>dup_frames</var> is true (the default) and the desktop is not being updated often enough, the filter will duplicate a previous frame. Note that there is no background buffering going on, so when the filter is not polled often enough then the actual inter-frame interval may be significantly larger. Defaults to 30 FPS. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Specify the size of the captured video. Defaults to the full size of the screen. Cropped from the bottom/right if smaller than screen size. </dd> <dt><samp class=”option”>offset_x</samp></dt> <dd> Horizontal offset of the captured video. </dd> <dt><samp class=”option”>offset_y</samp></dt> <dd> Vertical offset of the captured video. </dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired filter output format. Defaults to 8 Bit BGRA. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Passes all supported output formats to DDA and returns what DDA decides to use. </dd> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8 Bit formats always work, and DDA will convert to them if necessary. </dd> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dd> Filter initialization will fail if 10 bit format is requested but unavailable. </dd> </dl>

</dd> <dt><samp class="option">dup_frames</samp></dt> <dd> When this option is set to true (the default), the filter will duplicate frames when the desktop has not been updated in order to maintain approximately constant target framerate. When this option is set to false, the filter will wait for the desktop to be updated (inter-frame intervals may vary significantly in this case). </dd> </dl>

 <h4 class="subsection">37.4.1 Examples# TOC</h4>

Capture primary screen and encode using nvenc: <pre class="example-preformatted">ffmpeg -f lavfi -i ddagrab -c:v h264_nvenc -cq 18 output.mp4 </pre>

You can also skip the lavfi device and directly use the filter. Also demonstrates downloading the frame and encoding with libx264. Explicit output format specification is required in this case: <pre class="example-preformatted">ffmpeg -filter_complex ddagrab=output_idx=1:framerate=60,hwdownload,fo -c:v libx264 -crf 18 output.mp4 </pre>

If you want to capture only a subsection of the desktop, this can be achieved by specifying a smaller size and its offsets into the screen: <pre class="example-preformatted">ddagrab=video_size=800x600:offset_x=100:offset_y </pre>

 <h3 class="section">37.5 gfxcapture# TOC</h3>

Capture windows or monitors using Windows.Graphics.Capture API.

This source provides low overhead capture of application windows or entire monitors.

The filter outputs hardware frames in <code class="code">d3d11</code> format; use <code class="code">hwdownload,format=</code>

if system memory frames are required.

The window to be captured can be selected via regular expressions on its title,

class name or backing executable name, by explicit native handles, or by monitor

index or explicit native handle. A window must match all provided expressions to be

selected. The first matching window will be picked, in whatever order Windows

returns them.

Explicit handles (<samp class="option">hwnd</samp>, <samp class="option">hmonitor</samp>) override pattern or index

based selection. If neither handles nor a monitor index are given, the first

window matching the provided regular expressions is captured.

This source does NOT hold a stable FPS. It returns frames at whatever rate the compositor

provides them, only capped by the <samp class="option">max_framerate</samp>.

If you need a stable rate, you need to add an fps filter to drop/duplicate frames as needed.

Note that a static source may not deliver any frames at all, which also stalls

the fps filter; see <samp class="option">dup_frames</samp>.

If the capture source disappears mid-capture (window closed, monitor disconnected), the filter will return EOF.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">window_title</samp></dt>

<dd> ECMAScript regular expression matched against the window title. Supports a

PCRE style <code class="code">(?i)</code> prefix for case-insensitive matching.

</dd>

<dt><samp class="option">window_class</samp></dt>

<dd> As <samp class="option">window_title</samp>, but matched against the window class name.

</dd>

<dt><samp class=”option”>window_exe</samp></dt>

<dd> As <samp class=”option”>window_title</samp>, but matched against the executable file name of the

window’s process.

</dd>

<dt><samp class=”option”>monitor_idx</samp></dt>

<dd> Zero-based index of the monitor to capture.

Can also be set to <code class=”code”>window</code> to capture the monitor the selected window

is displayed on at filter initialization time.

</dd>

<dt><samp class=”option”>hwnd</samp></dt>

<dd> Explicit native window handle (HWND).

</dd>

<dt><samp class=”option”>hmonitor</samp></dt>

<dd> Explicit native monitor handle (HMONITOR).

</dd>

<dt><samp class=”option”>capture_cursor</samp></dt>

<dd> Capture the mouse cursor. Enabled by default.

</dd>

<dt><samp class=”option”>capture_border</samp></dt>

<dd> Capture the full area of the window, including its window decorarions/border.

Disabled by default.

</dd>

<dt><samp class="option">display_border</samp></dt>

<dd> Draw a yellow highlight border around the captured window. Disabled by default.

</dd>

<dt><samp class="option">max_framerate</samp></dt>

<dd> Maximum capture frame rate. Accepts a video rate (e.g. <code class="code">30</code>, <code class="code">60/1</code>,

<code class="code">24000/1001</code>). The default is <code class="code">60</code> FPS.

The actual rate is the rate at which the compositor renders the window/monitor,

capped by this option.

</dd>

<dt><samp class="option">width</samp></dt>

<dd> Force the output canvas width. If zero (default) the initial captured source

width is used. If a negative number is provided, the width will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">height</samp></dt>

<dd> Force the output canvas height. If zero (default) the initial captured source

height is used. If a negative number is provided, the height will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">crop_left</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_top</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_right</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_bottom</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">premultiplied</samp></dt>

<dd> If set to 1, return frames with premultiplied alpha. Default is 0 (straight

alpha).

</dd>

<dt><samp class="option">dup_frames</samp></dt>

<dd> If enabled (the default), the last frame is output again whenever the source

delivered no new frame within one <samp class="option">max_framerate</samp> interval, and black

frames are output until the source delivered its first frame. This keeps the

output flowing while the captured content is static. The output rate is still

not constant, add an <code class="code">fps</code> filter if needed.

</dd>

<dt><samp class=”option”>resize_mode</samp></dt>

<dd> Defines how the captured content is fitted into the output canvas size.

Possible values:

<dl class=”table”>

<dt>‘<samp class=”samp”>crop</samp>’</dt>

<dd> Crop (or pad with black) to the canvas size. (default)

</dd>

<dt>‘<samp class=”samp”>scale</samp>’</dt>

<dd> Scale the source to fill the canvas, potentially altering aspect ratio.

</dd>

<dt>‘<samp class=”samp”>scale_aspect</samp>’</dt>

<dd> Scale the source to fit inside the canvas while preserving aspect ratio.

Remaining area is filled with black.

</dd>

</dl>

</dd> <dt><samp class=”option”>scale_mode</samp></dt> <dd> Scaling algorithm used when resizing is required. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Nearest neighbour (pixelated) scaling. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear filtering. (default) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> Bicubic filtering. Potentially more blurry, but fewer scaling artifacts depending on contents. </dd> </dl>

</dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired output pixel format inside the D3D11 hardware frames. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dd> 8 bit BGRA output (default) </dd> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dd> 10 bit BGR output </dd> <dt>‘<samp class=”samp”>rgbaf16</samp>’</dt> <dt>‘<samp class=”samp”>16bit</samp>’</dt> <dd> 16bit float RGBA output </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>37.5.1 Examples# TOC</h4> <ul class=”itemize

mark-bullet”> Capture a window by title (case-insensitive) at a maximum of 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_title='(?i)My Application':max_framerate=60,hwdownload,format=bgra,format=yuv420p -c:v libx264 -crf 15 capture.mp4 </pre>

Capture monitor 1 at native refresh, 10bit color depth, scale to 1920x1080 preserving aspect: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=monitor_idx=1:width=1920:height=1080:resize_mode=sc -c:v hevc_nvenc -cq 15 capture.mp4 </pre>

Capture a window by executable name, draw border, force point scaling, fixed 60 fps: <pre class=”example-preformatted”>ffmpeg -filter_complex gfxcapture=window_exe='^firefox.exe$':display_border=1:scale_mod -rc qvbr -qvbr_quality_level 15 -c:v h264_amf capture.mp4 </pre>

 <h3 class=”section”>37.6 gradients# TOC</h3> Generate several gradients. <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>c0, c1, c2, c3, c4, c5, c6, c7</samp></dt> <dd> Set 8 colors. Default values for colors is to pick random one. </dd> <dt><samp class=”option”>x0, y0, x1, y1</samp></dt> <dd> Set gradient line source and destination points. If negative or out of range, random ones are picked. </dd> <dt><samp class=”option”>nb_colors, n</samp></dt> <dd> Set number of colors to use at once. Allowed range is from 2 to 8. Default value is 2. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed for picking gradient line points. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set speed of gradients rotation. </dd> <dt><samp class=”option”>type, t</samp></dt> <dd> Set type of gradients. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp class=”samp”>radial</samp>’</d

<dt>‘<samp class="samp">circular</samp>’</dt> <dt>‘<samp class="samp">spiral</samp>’</dt> <dt>‘<samp class="samp">square</samp>’</
</dl>

Default type is <var class="var">linear</var>. </dd> </dl>

 <h4 class="subsection">37.6.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">37.7 mandelbrot# TOC</h3>

Generate a Mandelbrot set fractal, and progressively zoom towards the point specified with <var class="var">start_x</var> and <var class="var">start_y</var>. This source accepts the following options: <dl class="table"> <dt><samp class="option">end_pts</samp></dt> <dd> Set the terminal pts value. Default value is 400. </dd> <dt><samp class="option">end_scale</samp></dt> <dd> Set the terminal scale value. Must be a floating point value. Default value is 0.3. </dd> <dt><samp class="option">inner</samp></dt> <dd> Set the inner coloring mode, that is the algorithm used to draw the Mandelbrot fractal internal region. It shall assume one of the following values: <dl class="table"> <dt><samp class="option">black</samp></dt> <dd> Set black mode. </dd> <dt><samp class="option">convergence</samp></dt> <dd> Show time until convergence. </dd> <dt><samp class="option">mincol</samp></dt> <dd> Set color based on point closest to the origin of the iterations. </dd> <dt><samp class="option">period</samp></dt> <dd> Set period mode. </dd> </dl>

Default value is <var class="var">mincol</var>. </dd> <dt><samp class="option">bailout</samp></dt> <dd> Set the bailout value. Default value is 10.0. </dd> <dt><samp class="option">maxiter</samp></dt> <dd> Set the maximum of iterations performed by the rendering algorithm. Default value is 7189. </dd> <dt><samp class="option">outer</samp></dt> <dd> Set outer coloring mode. It shall assume one of following values: <dl class="table"> <dt><samp class="option">iteration_count</samp></dt> <dd> Set iteration count mode. </dd> <dt><samp class="option">normalized_iteration_count</samp></dt> <dd> set normalized iteration count mode. </dd> </dl> Default value is <var class="var">normalized_iteration_count</var>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value

is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">start_scale</samp></dt> <dd> Set the initial scale value. Default value is 3.0. </dd> <dt><samp class="option">start_x</samp></dt> <dd> Set the initial x position. Must be a floating point value between -100 and 100. Default value is -0.74364388703715870475219150611477 4. </dd> <dt><samp class="option">start_y</samp></dt> <dd> Set the initial y position. Must be a floating point value between -100 and 100. Default value is -0.13182590420531197049313205638513 9. </dd> </dl>

 <h3 class="section">37.8 mptestsrc# TOC</h3>

Generate various test patterns, as generated by the MPlayer test filter. The size of the generated video is fixed, and is 512x512. This source is useful in particular for testing encoding features. This source accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class="option">test, t</samp></dt> <dd> Set the number or the name of the test to perform. Supported tests are: <dl class="table"> <dt><samp class="option">dc_luma</samp></dt> <dt><samp class="option">dc_chroma</samp></dt> <dt><samp class="option">freq_luma</samp></dt> <dt><samp class="option">freq_chroma</samp></dt> <dt><samp class="option">amp_luma</samp></dt> <dt><samp class="option">amp_chroma</samp></dt> <dt><samp class="option">cbp</samp></dt> <dt><samp class="option">mv</samp></dt> <dt><samp class="option">ring1</samp></dt> <dt><samp class="option">ring2</samp></dt> <dt><samp class="option">all</samp></dt> <dt><samp class="option">max_frames, m</samp></dt> <dd> Set the maximum number of frames generated for each test, default value is 30. </dd> </dl>

Default value is "all", which will cycle through the list of all

tests. </dd> </dl>

Some examples: <pre class=”example-preformatted”>mptestsrc=t=dc_luma </pre>

will generate a "dc_luma" test pattern. <h3 class=”section”>37.9 frei0r_src# TOC</h3>

Provide a frei0r source. To enable compilation of this filter you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. This source accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>size</samp></dt> <dd> The size of the video to generate. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> The framerate of the generated video. It may be a string of the form <var class=”var”>num</var>/<var class=”var”>den</var> or a frame rate abbreviation. </dd> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name to the frei0r source to load. For more information regarding frei0r and how to set the parameters, read the frei0r section in the video filters documentation. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r source. </dd> </dl>

For example, to generate a frei0r partik0l source with size 200x200 and frame rate 10 which is overlaid on the overlay filter main input: <pre class=”example-preformatted”>frei0r_src=size=200x200:framerate=10:filter_name=
[overlay]; [in][overlay] overlay </pre>

 <h3 class=”section”>37.10 life# TOC</h3>

Generate a life pattern.

This source is based on a generalization of John Conway’s life game.

The sourced input represents a life grid, each pixel represents a cell

which can be in one of two possible states, alive or dead. Every cell

interacts with its eight neighbours, which are the cells that are

horizontally, vertically, or diagonally adjacent.

At each interaction the grid evolves according to the adopted rule,

which specifies the number of neighbor alive cells which will make a

cell stay alive or born. The <samp class="option">rule</samp> option allows one to specify

the rule to adopt.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">filename, f</samp></dt>

<dd> Set the file from which to read the initial grid state. In the file,

each non-whitespace character is considered an alive cell, and newline

is used to delimit the end of each row.

If this option is not specified, the initial grid is generated

randomly.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the video rate, that is the number of frames generated per second.

Default is 25.

</dd>

<dt><samp class="option">random_fill_ratio, ratio</samp></dt>

<dd> Set the random fill ratio for the initial random grid. It is a

floating point number value ranging from 0 to 1, defaults to 1/PHI.

It is ignored when a file is specified.

</dd>

<dt><samp class=”option”>random_seed, seed</samp></dt>

<dd> Set the seed for filling the initial random grid, must be an integer

included between 0 and UINT32_MAX. If not specified, or if explicitly

set to -1, the filter will try to use a good random seed on a best

effort basis.

</dd>

<dt><samp class=”option”>rule</samp></dt>

<dd> Set the life rule.

A rule can be specified with a code of the kind "S<var class=”var”>NS</var>/B<var class=”var”>NB</var>",

where <var class=”var”>NS</var> and <var class=”var”>NB</var> are sequences of numbers in the range 0-8,

<var class=”var”>NS</var> specifies the number of alive neighbor cells which make a

live cell stay alive, and <var class=”var”>NB</var> the number of alive neighbor cells

which make a dead cell to become alive (i.e. to "born").

"s" and "b" can be used in place of "S" and "B", respectively.

Alternatively a rule can be specified by an 18-bits integer. The 9

high order bits are used to encode the next cell state if it is alive

for each number of neighbor alive cells, the low order bits specify

the rule for "borning" new cells. Higher order bits encode for an

higher number of neighbor cells.

For example the number 6153 = <code class=”code”>(12<<9)+9</code> specifies a stay alive

rule of 12 and a born rule of 9, which corresponds to "S23/B03".

Default value is "S23/B3", which is the original Conway’s game of life

rule, and will keep a cell alive if it has 2 or 3 neighbor alive

cells, and will born a new cell if there are three alive cells around

a dead cell.

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Set the size of the output video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

If <samp class="option">filename</samp> is specified, the size is set by default to the

same size of the input file. If <samp class="option">size</samp> is set, it must contain

the size specified in the input file, and the initial grid defined in

that file is centered in the larger resulting area.

If a filename is not specified, the size value defaults to "320x240"

(used for a randomly generated initial grid).

</dd>

<dt><samp class="option">stitch</samp></dt>

<dd> If set to 1, stitch the left and right grid edges together, and the

top and bottom edges also. Defaults to 1.

</dd>

<dt><samp class="option">mold</samp></dt>

<dd> Set cell mold speed. If set, a dead cell will go from <samp class="option">death_color</samp> to

<samp class="option">mold_color</samp> with a step of <samp class="option">mold</samp>. <samp class="option">mold</samp> can have a

value from 0 to 255.

</dd>

<dt><samp class="option">life_color</samp></dt>

<dd> Set the color of living (or new born) cells.

</dd>

<dt><samp class="option">death_color</samp></dt>

<dd> Set the color of dead cells. If <samp class="option">mold</samp> is set, this is the first color

used to represent a dead cell.

</dd>

<dt><samp class="option">mold_color</samp></dt>

<dd> Set mold color, for definitely dead and moldy cells.

For the syntax of these 3 color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the

ffmpeg-utils manual.

</dd>

</dl>

 <h4 class="subsection">37.10.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read a grid from <samp class="file">pattern</samp>, and center it on a grid of size 300x300 pixels: <pre class="example-preformatted">life=f=pattern:s=300x300 </pre>

Generate a random grid of size 200x200, with a fill ratio of 2/3: <pre class="example-preformatted">life=ratio=2/3:s=200x200 </pre>

Specify a custom rule for evolving a randomly generated grid: <pre class="example-preformatted">life=rule=S14/B34 </pre>

Full example with slow death effect (mold) using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi life=s=300x200:mold=10:r=60:ratio=0.1:death_color=#C83232:life_color=#00f
</pre>

 <h3 class="section">37.11 perlin#
TOC</h3>
Generate Perlin noise. Perlin noise is a kind of noise with local continuity in space. This can be used to generate patterns with continuity in space and time, e.g. to simulate smoke, fluids, or terrain. In case more than one octave is specified through the <samp class="option">octaves</samp> option, Perlin noise is generated as a sum of components, each one with doubled frequency. In this case the <samp class="option">persistence</samp> option specify the ratio of the amplitude with respect to the previous component. More octave components enable to specify more high frequency details in the generated noise (e.g. small size variations due to boulders in a generated terrain). <h4 class="subsection">37.11.1 Options# TOC</h4> <dl class="table">
<dt><samp class="option">size, s</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">320x240</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate expected for the video stream, expressed as a number of frames per second. Default value is <code class="code">25</code>. </dd>
<dt><samp class="option">octaves</samp></dt> <dd> Specify the total number of components making up the noise, each one with doubled frequency. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">persistence</samp></dt>
<dd> Set the ratio used to compute the amplitude of the next octave component with respect to the previous component amplitude. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">xscale</samp></dt> <dt><samp class="option">yscale</samp></dt>
<dd> Define a scale factor used to multiple the x, y coordinates. This can be useful to define an effect with a pattern stretched along the x or y axis. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">tscale</samp></dt> <dd> Define a scale factor used to multiple the time coordinate. This can be useful to change the time variation speed. Default value is <code class="code">1</code>. </dd>
<dt><samp class="option">random_mode</samp></dt> <dd> Set random mode used to compute initial pattern. Supported values are:

<dl class="table"> <dt><samp class="option">random</samp></dt> <dd> Compute and use random seed. </dd> <dt><samp class="option">ken</samp></dt> <dd> Use the predefined initial pattern defined by Ken Perlin in the original article, can be useful to compare the output with other sources. </dd> <dt><samp class="option">seed</samp></dt> <dd> Use the value specified by <samp class="option">random_seed</samp> option. </dd> </dl>

Default value is <code class="code">random</code>. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> When <samp class="option">random_mode</samp> is set to <var class="var">random_seed</var>, use this value to compute the initial pattern. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">37.11.2 Examples# TOC</h4> <ul class="itemize mark-bullet"> Generate single component: <pre class="example-preformatted">perlin </pre>

Use Perlin noise with 7 components, each one with a halved contribution to total amplitude: <pre class="example-preformatted">perlin=octaves=7:persistence=0.5 </pre>

Chain Perlin noise with the lutyuv to generate a black&white effect: <pre class="example-preformatted">perlin=octaves=3:tscale=0.3,lutyuv=y='if(lt(val\,128)\,255\,0)' </pre>

Stretch noise along the y axis, and convert gray level to red-only signal: <pre class="example-preformatted">perlin=octaves=7:tscale=0.4:yscale=0.3,lut </pre>

 <h3 class="section">37.12 qrencodesrc# TOC</h3>

Generate a QR code using the libqrencode library (see https://fukuchi.org/works/qrencode/). To enable the compilation of this source, you need to configure FFmpeg with <code class="code">--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and put in the video output according to the specified output size options. In case no text is specified, the QR code is not generated, but an empty colored output is returned instead. This source accepts the following options: <dl class="table"> <dt><samp class="option">qrcode_width, q</samp></dt> <dt><samp

class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated only once, when initializing the source. See the qrencode Expressions section for details. Note that some of the constants are missing for the source (for example the <var class=”var”>x</var> or <var class=”var”>t</var> or ¸<var class=”var”>n</var>), since they only makes sense when evaluating the expression for each frame rather than at initialization time. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is "white". For the syntax of the color options,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </p></dd> </dl>

 <h4 class="subsection">37.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:bc=pink:fc=blue </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencodesrc=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencodesrc=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencodesrc=text=%{pts} </pre>

 <h3 class="section">37.13

allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc# TOC</h3>

The <code class="code">allrgb</code> source returns frames of size 4096x4096 of all rgb colors.

The <code class="code">allyuv</code> source returns frames of size 4096x4096 of all yuv colors.

The <code class="code">color</code> source provides an uniformly colored input.

The <code class="code">colorchart</code> source provides a colors checker chart.

The <code class="code">colorspectrum</code> source provides a color spectrum input.

The <code class="code">haldclutsrc</code> source provides an identity Hald CLUT. See also

haldclut filter.

The <code class="code">nullsrc</code> source returns unprocessed video frames. It is

mainly useful to be employed in analysis / debugging tools, or as the

source for filters which ignore the input data.

The <code class="code">pal75bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 75% color levels.

The <code class=”code”>pal100bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 100% color levels.

The <code class=”code”>rgbtestsrc</code> source generates an RGB test pattern useful for

detecting RGB vs BGR issues. You should see a red, green and blue

stripe from top to bottom.

The <code class=”code”>smptebars</code> source generates a color bars pattern, based on

the SMPTE Engineering Guideline EG 1-1990.

The <code class=”code”>smptehdbars</code> source generates a color bars pattern, based on

the SMPTE RP 219-2002.

The <code class=”code”>testsrc</code> source generates a test video pattern, showing a

color pattern, a scrolling gradient and a timestamp. This is mainly

intended for testing purposes.

The <code class=”code”>testsrc2</code> source is similar to testsrc, but supports more

pixel formats instead of just <code class=”code”>rgb24</code>. This allows using it as an

input for other tests without requiring a format conversion.

The <code class=”code”>yuvtestsrc</code> source generates an YUV test pattern. You should

see a y, cb and cr stripe from top to bottom.

The sources accept the following parameters:

<dl class="table">

<dt><samp class="option">level</samp></dt>

<dd> Specify the level of the Hald CLUT, only available in the <code class="code">haldclutsrc</code>

source. A level of <code class="code">N</code> generates a picture of <code class="code">N*N*N</code> by <code class="code">N*N*N</code>

pixels to be used as identity matrix for 3D lookup tables. Each component is

coded on a <code class="code">1/(N*N)</code> scale.

</dd>

<dt><samp class="option">color, c</samp></dt>

<dd> Specify the color of the source, only available in the <code class="code">color</code>

source. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the size of the sourced video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

The default value is <code class="code">320x240</code>.

This option is not available with the <code class="code">allrgb</code>, <code class="code">allyuv</code>, and

<code class="code">haldclutsrc</code> filters.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Specify the frame rate of the sourced video, as the number of frames

generated per second. It has to be a string in the format

<var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point

number or a valid video frame rate abbreviation. The default value is

"25".

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Set the duration of the sourced video. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If not specified, or the expressed duration is negative, the video is

supposed to be generated forever.

Since the frame rate is used as time base, all frames including the last one

will have their full duration. If the specified duration is not a multiple

of the frame duration, it will be rounded up.

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> Set the sample aspect ratio of the sourced video.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Specify the alpha (opacity) of the background, only available in the

<code class="code">testsrc2</code> source. The value must be between 0 (fully transparent) and

255 (fully opaque, the default).

</dd>

<dt><samp class="option">decimals, n</samp></dt>

<dd> Set the number of decimals to show in the timestamp, only available in the

<code class="code">testsrc</code> source.

The displayed timestamp value will correspond to the original

timestamp value multiplied by the power of 10 of the specified

value. Default value is 0.

</dd>

<dt><samp class="option">type</samp></dt>

<dd> Set the type of the color spectrum, only available in the

<code class="code">colorspectrum</code> source. Can be one of the following:

<dl class="table">

<dt>‘<samp class="samp">black</samp>’</dt>

<dt>‘<samp class="samp">white</samp>’</dt>

<dt>‘<samp class="samp">all</samp>’</dt>

</dl>

</dd> <dt><samp class="option">patch_size</samp></dt> <dd> Set patch size of single color patch, only available in the <code class="code">colorchart</code> source. Default is <code class="code">64x64</code>. </dd> <dt><samp class="option">preset</samp></dt> <dd> Set colorchecker colors preset, only available in the <code class="code">colorchart</code> source. Available values are: <dl class="table"> <dt>‘<samp class="samp">reference</samp>’</dt> <dt>‘<samp class="samp">skintones</samp>’</dt> </dl>

Default value is <code class="code">reference</code>. </dd> </dl>

 <h4 class="subsection">37.13.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a video with a duration of 5.3 seconds, with size 176x144 and a frame rate of 10 frames per second: <pre class="example-preformatted">testsrc=duration=5.3:size=qcif:rate=10 </pre>

The following graph description will generate a red source with an opacity of 0.2, with size "qcif" and a frame rate of 10 frames per second: <pre class="example-preformatted">color=c=red@0.2:s=qcif:r=10 </pre>

If the input content is to be ignored, <code class="code">nullsrc</code> can be used. The following command generates noise in the luma plane by employing the <code class="code">geq</code> filter: <pre class="example-preformatted">nullsrc=s=256x256, geq=random(1)*255:128:128 </pre>

 <h4 class="subsection">37.13.2 Commands# TOC</h4>

The <code class="code">color</code> source supports the following commands: <dl class="table"> <dt><samp class="option">c, color</samp></dt> <dd> Set the color of the created image. Accepts the same syntax of the corresponding <samp class="option">color</samp> option. </dd> </dl>

 <h3 class="section">37.14 openclsrc# TOC</h3>

Generate video using an OpenCL program. <dl class="table"> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Kernel name in program. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Size of frames to generate. This must be set. </dd> <dt><samp class="option">format</samp></dt> <dd> Pixel format to use for the generated frames. This must be set. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Number of frames generated every second. Default value is ’25’. </dd> </dl>

For details of how the program loading works, see the program_opencl filter. Example

programs: <ul class=”itemize mark-bullet”> Generate a colour ramp by setting pixel values from the position of the pixel in the output image. (Note that this will work with all pixel formats, but the generated output will not be the same.) <pre class=”verbatim”>___kernel void ramp(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 val; val.xy = val.zw = convert_float2(loc) / convert_float2(get_image_dim(dst));

write_imagef(dst, loc, val); } </pre> Generate a Sierpinski carpet pattern, panning by a single pixel each frame. <pre class=”verbatim”>___kernel void sierpinski_carpet(___write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 value = 0.0f; int x = loc.x + index; int y = loc.y + index; while (x > 0 || y > 0) { if (x % 3 == 1 & y % 3 == 1) { value = 1.0f; break; } x /= 3; y /= 3; }

write_imagef(dst, loc, value); } </pre>

 <h3 class=”section”>37.15 sierpinski# TOC</h3>

Generate a Sierpinski carpet/triangle fractal, and randomly pan around. This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed which is used for random panning. </dd> <dt><samp class=”option”>jump</samp></dt> <dd> Set max jump for single pan destination. Allowed range is from 1 to 10000. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set fractal type, can be default <code class=”code”>carpet</code> or <code class=”code”>triangle</code>. </dd> </dl>

 <h3 class=”section”>37.16 zoneplate# TOC</h3> Generate a zoneplate test video pattern. This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option,

check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "320x240". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set precision in bits for look-up table for sine calculations. Default value is 10. Allowed range is from 4 to 16. </dd> <dt><samp class=”option”>xo</samp></dt> <dd> Set horizontal axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>yo</samp></dt> <dd> Set vertical axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>to</samp></dt> <dd> Set time axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>k0</samp></dt> <dd> Set 0-order, constant added to signal phase. Default value is 0. </dd> <dt><samp class=”option”>kx</samp></dt> <dd> Set 1-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky</samp></dt> <dd> Set 1-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt</samp></dt> <dd> Set 1-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>kxt, kyt, kxy</samp></dt> <dd> Set phase factor multipliers for combination of spatial and temporal axis. Default value is 0. </dd> <dt><samp class=”option”>kx2</samp></dt> <dd> Set 2-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class=”option”>ky2</samp></dt> <dd> Set 2-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class=”option”>kt2</samp></dt> <dd> Set 2-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class=”option”>ku</samp></dt> <dd> Set the constant added to final phase to produce chroma-blue component of signal. Default value is 0. </dd> <dt><samp class=”option”>kv</samp></dt> <dd> Set the constant added to final phase to produce chroma-red component of signal. Default value is 0. </dd> </dl>

 <h4 class=”subsection”>37.16.1 Commands# <a class=”anchor hidden-xs”href=”#toc-

Commands-159” aria-hidden=”true”>TOC</h4>

This source supports the some above options as commands. <h4 class=”subsection”>37.16.2 Examples#
TOC</h4>

<ul class=”itemize mark-bullet”> Generate horizontal color sine sweep:
 <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:kx2=256:s=wvg
426:kt=11 </pre>

Generate vertical color sine sweep: <pre class=”example-preformatted”>zoneplate=ku=512:kv=0:kt2=0:ky2=156:s=wvga:yo=-240:kt=11 </pre>

Generate circular zone-plate: <pre class=”example-preformatted”>zoneplate=ku=512:kv=100:kt2=0:ky2=256:kx2=556:s=wvga:yo=0:kt=11 </pre>

 <h2 class=”chapter”>38 Video Sinks# TOC</h2>

Below is a description of the currently available video sinks. <h3 class=”section”>38.1 buffersink# TOC</h3>

Buffer video frames, and make them available to the end of the filter graph. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVBufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class=”code”>avfilter_init_filter</code> for initialization. <h3 class=”section”>38.2 nullsink# TOC</h3>

Null video sink: do absolutely nothing with the input video. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class=”chapter”>39 Multimedia Filters# TOC</h2>

Below is a description of the currently available multimedia filters. <h3 class="section">39.1 a3dscope# TOC</h3>

Convert input audio to 3d scope video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">fov</samp></dt> <dd> Set the camera field of view. Default is 90 degrees. Allowed range is from 40 to 150. </dd> <dt><samp class="option">roll</samp></dt> <dd> Set the camera roll. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set the camera pitch. </dd> <dt><samp class="option">yaw</samp></dt> <dd> Set the camera yaw. </dd> <dt><samp class="option">xzoom</samp></dt> <dd> Set the camera zoom on X-axis. </dd> <dt><samp class="option">yzoom</samp></dt> <dd> Set the camera zoom on Y-axis. </dd> <dt><samp class="option">zzoom</samp></dt> <dd> Set the camera zoom on Z-axis. </dd> <dt><samp class="option">xpos</samp></dt> <dd> Set the camera position on X-axis. </dd> <dt><samp class="option">ypos</samp></dt> <dd> Set the camera position on Y-axis. </dd> <dt><samp class="option">zpos</samp></dt> <dd> Set the camera position on Z-axis. </dd> <dt><samp class="option">length</samp></dt> <dd> Set the length of displayed audio waves in number of frames. </dd> </dl>

 <h4 class="subsection">39.1.1 Commands# TOC</h4>

Filter supports the some above options as commands. <h3 class="section">39.2 abitscope# TOC</h3>

Convert input audio to a video output, displaying the audio bit scope. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default

value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">1024x256</code>. </dd> <dt><samp class="option">colors</samp></dt> <dd> Specify list of colors separated by space or by ’|’ which will be used to draw channels. Unrecognized or missing colors will be replaced by white color. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set output mode. Can be <code class="code">bars</code> or <code class="code">trace</code>. Default is <code class="code">bars</code>. </dd> </dl>

 <h3 class="section">39.3 adrawgraph# TOC</h3> Draw a graph using input audio metadata. See drawgraph <h3 class="section">39.4 agraphmonitor# TOC</h3>

See graphmonitor. <h3 class="section">39.5 ahistogram# TOC</h3>

Convert input audio to a video output, displaying the volume histogram.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">dmode</samp></dt>

<dd> Specify how histogram is calculated.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">single</samp>’</dt>

<dd> Use single histogram for all channels.

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> Use separate histogram for each channel.

</dd>

</dl>

Default is <code class="code">single</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set frame rate, expressed as number of frames per second. Default

value is "25".

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">hd720</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set display scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

<dt>‘<samp class="samp">sqrt</samp>’</dt>

<dd> square root

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> reverse logarithmic
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">acount</samp></dt>
<dd> Set how much frames to accumulate in histogram.

Default is 1. Setting this to -1 accumulates all frames.

</dd>
<dt><samp class="option">rheight</samp></dt>
<dd> Set histogram ratio of window height.

</dd>
<dt><samp class="option">slide</samp></dt>
<dd> Set sonogram sliding.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">replace</samp>’</dt>
<dd> replace old rows with new ones.
</dd>
<dt>‘<samp class="samp">scroll</samp>’</dt>
<dd> scroll from top to bottom.
</dd>
</dl>
Default is <code class="code">replace</code>.

</dd>
<dt><samp class="option">hmode</samp></dt>
<dd> Set histogram mode.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">abs</samp>’</dt>
<dd> Use absolute values of samples.
</dd>
<dt>‘<samp class="samp">sign</samp>’</dt>

<dd> Use untouched values of samples.

</dd>

</dl>

Default is <code class="code">abs</code>.

</dd>

</dl>

 <h3 class="section">39.6 aphasemeter# TOC</h3>

Measures phase of input audio, which is exported as metadata <code class="code">lavfi.aphasemeter.phase</code>, representing mean phase of current audio frame. A video output can also be produced and is enabled by default. The audio is passed through as first output. Audio will be rematrixed to stereo if it has a different channel layout. Phase value is in range <code class="code">[-1, 1]</code> where <code class="code">-1</code> means left and right channels are completely out of phase and <code class="code">1</code> means channels are in phase. The filter accepts the following options, all related to its video output: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class="code">25</code>. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">800x400</code>. </dd> <dt><samp class="option">rc</samp></dt> <dt><samp class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt> <dd> Specify the red, green, blue contrast. Default values are <code class="code">2</code>, <code class="code">7</code> and <code class="code">1</code>. Allowed range is <code class="code">[0, 255]</code>. </dd> <dt><samp class="option">mpc</samp></dt> <dd> Set color which will be used for drawing median phase. If color is <code class="code">none</code> which is default, no median phase value will be drawn. </dd> <dt><samp class="option">video</samp></dt> <dd> Enable video output. Default is enabled. </dd> </dl>

 <h4 class="subsection">39.6.1 phasing detection# TOC</h4>

The filter also detects out of phase and mono sequences in stereo

streams. It logs the sequence start, end and duration when it lasts longer or as long as the minimum set. The filter accepts the following options for this detection: <dl class="table"> <dt><samp class="option">phasing</samp></dt> <dd> Enable mono and out of phase detection. Default is disabled. </dd> <dt><samp class="option">tolerance, t</samp></dt> <dd> Set phase tolerance for mono detection, in amplitude ratio. Default is <code class="code">0</code>. Allowed range is <code class="code">[0, 1]</code>. </dd> <dt><samp class="option">angle, a</samp></dt> <dd> Set angle threshold for out of phase detection, in degree. Default is <code class="code">170</code>. Allowed range is <code class="code">[90, 180]</code>. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set mono or out of phase duration until notification, expressed in seconds. Default is <code class="code">2</code>. </dd> </dl>

 <h4 class="subsection">39.6.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example with <code class="command">ffmpeg</code> to detect 1 second of mono with 0.001 phase tolerance: <pre class="example-preformatted">ffmpeg -i stereo.wav -af aphasemeter=video=0:phasing=1:duration=1:tolerance=0.001 -f null - </pre>

 <h3 class="section">39.7 avectorscope# TOC</h3>

Convert input audio to a video output, representing the audio vector scope. The filter is used to measure the difference between channels of stereo audio stream. A monaural signal, consisting of identical left and right signal, results in straight vertical line. Any stereo separation is visible as a deviation from this line, creating a Lissajous figure. If the straight (or deviation from it) but horizontal line appears this indicates that the left and right channels are out of phase. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set the vectorscope mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">lissajous</samp>’</dt> <dd> Lissajous rotated by 45 degrees. </dd> <dt>‘<samp class="samp">lissajous_xy</samp>’</dt> <dd> Same as above but not rotated. </dd> <dt>‘<samp class="samp">polar</samp>’</dt> <dd> Shape resembling half of circle. </dd> </dl>

Default value is ‘<samp class="samp">lissajous</samp>’.
 </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set
the video size for the output. For the syntax of this option, check
the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-
syntax">"Video size" section in the ffmpeg-utils manual.
Default value is <code class="code">400x400</code>. </dd>
<dt><samp class="option">rate, r</samp></dt> <dd> Set the
output frame rate. Default value is <code class="code">25</code>.
 </dd> <dt><samp class="option">rc</samp></dt> <dt><samp
class="option">gc</samp></dt> <dt><samp class="option">bc</samp></dt>
<dt><samp class="option">ac</samp></dt> <dd> Specify the red,
green, blue and alpha contrast. Default values are <code class="code">40</code>,
<code class="code">160</code>, <code class="code">80</code> and
<code class="code">255</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">rf</samp></dt>
<dt><samp class="option">gf</samp></dt> <dt><samp class="option">bf</samp></dt>
<dt><samp class="option">af</samp></dt> <dd> Specify the red,
green, blue and alpha fade. Default values are <code class="code">15</code>,
<code class="code">10</code>, <code class="code">5</code> and
<code class="code">5</code>. Allowed range is <code class="code">[0,
255]</code>. </dd> <dt><samp class="option">zoom</samp></dt>
<dd> Set the zoom factor. Default value is <code class="code">1</code>.
Allowed range is <code class="code">[0, 10]</code>. Values lower than
<var class="var">1</var> will auto adjust zoom factor to maximal possible
value. </dd> <dt><samp class="option">draw</samp></dt>
<dd> Set the vectorscope drawing mode. Available values are:
<dl class="table"> <dt>‘<samp class="samp">dot</samp>’</dt>
<dd> Draw dot for each sample. </dd> <dt>‘<samp
class="samp">line</samp>’</dt> <dd> Draw line between pre-
vious and current sample. </dd> <dt>‘<samp class="samp">aaline</samp>’</dt>
<dd> Draw anti-aliased line between previous and current sample.
</dd> </dl>

Default value is ‘<samp class="samp">dot</samp>’.
 </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
amplitude scale of audio samples. Available values are: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root. </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic. </dd> </dl>

</dd> <dt><samp class="option">swap</samp></dt> <dd> Swap
left channel axis with right channel axis. </dd> <dt><samp
class="option">mirror</samp></dt> <dd> Mirror axis. <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> No mirror. </dd> <dt>‘<samp class="samp">x</samp>’</dt>

<dd> Mirror only x axis. </dd> <dt>‘<samp class="samp">y</samp>’</dt>
<dd> Mirror only y axis. </dd> <dt>‘<samp class="samp">xy</samp>’</dt>
<dd> Mirror both axis. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">39.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] avectorscope=zoom=1.3:rc=2:gc=200:bc=10:rf=1:gf=8:bf=7 [out0]' </pre>

 <h4 class="subsection">39.7.2 Commands# TOC</h4>

This filter supports the all above options as commands except options <code class="code">size</code> and <code class="code">rate</code>.
  <h3 class="section">39.8 bench, abench# TOC</h3>

Benchmark part of a filtergraph. The filter accepts the following options: <dl class="table"> <dt><samp class="option">action</samp></dt>
<dd> Start or stop a timer. Available values are: <dl class="table"> <dt>‘<samp class="samp">start</samp>’</dt>
<dd> Get the current time, set it as frame metadata (using the key <code class="code">lavfi.bench.start_time</code>), and forward the frame to the next filter. </dd> <dt>‘<samp class="samp">stop</samp>’</dt>
<dd> Get the current time and fetch the <code class="code">lavfi.bench.start_time</code> metadata from the input frame metadata to get the time difference. Time difference, average, maximum and minimum time (respectively <code class="code">t</code>, <code class="code">avg</code>, <code class="code">max</code> and <code class="code">min</code>) are then printed. The timestamps are expressed in seconds. </dd> </dl> </dd>
</dl>

 <h4 class="subsection">39.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Benchmark selectivecolor

filter: <pre class=”example-preformatted”>bench=start,selectivecolor=reds=-.2 .12 -.49,bench=stop </pre>

 <h3 class=”section”>39.9 concat# TOC</h3>

Concatenate audio and video streams, joining them together one after the other. The filter works on segments of synchronized video and audio streams. All segments must have the same number of streams of each type, and that will also be the number of streams at output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of segments. Default is 2. </dd> <dt><samp class=”option”>v</samp></dt> <dd> Set the number of output video streams, that is also the number of video streams in each segment. Default is 1. </dd> <dt><samp class=”option”>a</samp></dt> <dd> Set the number of output audio streams, that is also the number of audio streams in each segment. Default is 0. </dd> <dt><samp class=”option”>unsafe</samp></dt> <dd> Activate unsafe mode: do not fail if segments have a different format. </dd> </dl>

The filter has <var class=”var”>v</var>+<var class=”var”>a</var> outputs: first <var class=”var”>v</var> video outputs, then <var class=”var”>a</var> audio outputs. There are <var class=”var”>n</var>x(<var class=”var”>v</var>+<var class=”var”>a</var>) inputs: first the inputs for the first segment, in the same order as the outputs, then the inputs for the second segment, etc. Related streams do not always have exactly the same duration, for various reasons including codec frame size or sloppy authoring. For that reason, related synchronized streams (e.g. a video and its audio track) should be concatenated at once. The concat filter will use the duration of the longest stream in each segment (except the last one), and if necessary pad shorter audio streams with silence. For this filter to work correctly, all segments must start at timestamp 0. All corresponding streams must have the same parameters in all segments; the filtering system will automatically select a common pixel format for video streams, and a common sample format, sample rate and channel layout for audio streams, but other settings, such as resolution, must be converted explicitly by the user. Different frame rates are acceptable but will result in variable frame rate at output; be sure to configure the output file to handle it. <h4 class=”subsection”>39.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Concatenate an opening, an episode and an ending, all in bilingual version (video in stream 0, audio in streams 1 and 2):

 <pre class="example-preformatted">ffmpeg -i opening.mkv -i episode.mkv -i ending.mkv -filter_complex \ '[0:0] [0:1] [0:2] [1:0] [1:1] [1:2] [2:0] [2:1] [2:2] concat=n=3:v=1:a=2 [v] [a1] [a2]' \ -map '[v]' -map '[a1]' -map '[a2]' output.mkv </pre>

Concatenate two parts, handling audio and video separately, using the (a)movie sources, and adjusting the resolution: <pre class="example-preformatted">movie=part1.mp4, scale=512:288 [v1] ; amovie=part1.mp4 [a1] ; movie=part2.mp4, scale=512:288 [v2] ; amovie=part2.mp4 [a2] ; [v1] [v2] concat [outv] ; [a1] [a2] concat=v=0:a=1 [outa] </pre> Note that a desync will happen at the stitch if the audio and video streams do not have exactly the same duration in the first file.

 <h4 class="subsection">39.9.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">next</samp></dt> <dd> Close the current segment and step to the next one </dd> </dl>

 <h3 class="section">39.10 ebur128# TOC</h3>

EBU R128 scanner filter. This filter takes an audio stream and analyzes its loudness level. By default, it logs a message at a frequency of 10Hz with the Momentary loudness (identified by <code class="code">M</code>), Short-term loudness (<code class="code">S</code>), Integrated loudness (<code class="code">I</code>) and Loudness Range (<code class="code">LRA</code>). The filter can only analyze streams which have sample format is double-precision floating point. The input stream will be converted to this specification, if needed. Users may need to insert aformat and/or aresample filters after this filter to obtain the original parameters. The filter also has a video output (see the <var class="var">video</var> option) with a real time graph to observe the loudness evolution. The graphic contains the logged message mentioned above, so it is not printed anymore when this option is set, unless the verbose logging is set. The main graphing area contains the short-term loudness (3 seconds of analysis), and the gauge on the right is for the momentary loudness (400 milliseconds), but can optionally be configured to instead display short-term loudness (see <var class="var">gauge</var>). The green area marks a +/- 1LU target range around the target loudness (-23LUFS by default, unless modified through <var class="var">target</var>). More information about the Loudness Recommendation EBU R128 on <a class="url"

href=”http://tech.ebu.ch/loudness”>http://tech.ebu.ch/loudness. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>video</samp></dt> <dd> Activate the video output. The audio stream is passed unchanged whether this option is set or no. The video stream will be the first output stream if activated. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set the video size. This option is for video only. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default and minimum resolution is <code class=”code”>640x480</code>. </dd> <dt><samp class=”option”>meter</samp></dt> <dd> Set the EBU scale meter. Default is <code class=”code”>9</code>. Common values are <code class=”code”>9</code> and <code class=”code”>18</code>, respectively for EBU scale meter +9 and EBU scale meter +18. Any other integer value between this range is allowed. </dd> <dt><samp class=”option”>metadata</samp></dt> <dd> Set metadata injection. If set to <code class=”code”>1</code>, the audio input will be segmented into 100ms output frames, each of them containing various loudness information in metadata. All the metadata keys are prefixed with <code class=”code”>lavfi.r128.</code>. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>framelog</samp></dt> <dd> Force the frame logging level. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>quiet</samp>’</dt> <dd> logging disabled </dd> <dt>‘<samp class=”samp”>info</samp>’</dt> <dd> information logging level </dd> <dt>‘<samp class=”samp”>verbose</samp>’</dt> <dd> verbose logging level </dd> </dl>

By default, the logging level is set to <var class=”var”>info</var>. If the <samp class=”option”>video</samp> or the <samp class=”option”>metadata</samp> options are set, it switches to <var class=”var”>verbose</var>. </dd> <dt><samp class=”option”>peak</samp></dt> <dd> Set peak mode(s). Available modes can be cumulated (the option is a <code class=”code”>flag</code> type). Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disable any peak mode (default). </dd> <dt>‘<samp class=”samp”>sample</samp>’</dt> <dd> Enable sample-peak mode. Simple peak mode looking for the higher sample value. It logs a message for sample-peak (identified by <code class=”code”>SPK</code>). </dd> <dt>‘<samp class=”samp”>true</samp>’</dt> <dd> Enable true-peak mode. If enabled, the peak lookup is done on an over-sampled version of the input stream for better peak accuracy. It logs a message for true-peak. (identified by <code class=”code”>TPK</code>) and true-peak per frame (identified by <code class=”code”>FTPK</code>). This mode requires a build with <code

class=”code”>libswresample</code>. </dd> </dl>

</dd> <dt><samp class=”option”>dualmono</samp></dt> <dd> Treat mono input files as "dual mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. </dd> <dt><samp class=”option”>panlaw</samp></dt> <dd> Set a specific pan law to be used for the measurement of dual mono files. This parameter is optional, and has a default value of -3.01dB. </dd> <dt><samp class=”option”>target</samp></dt> <dd> Set a specific target level (in LUFS) used as relative zero in the visualization. This parameter is optional and has a default value of -23LUFS as specified by EBU R128. However, material published online may prefer a level of -16LUFS (e.g. for use with podcasts or video platforms). </dd> <dt><samp class=”option”>gauge</samp></dt> <dd> Set the value displayed by the gauge. Valid values are <code class=”code”>momentary</code> and s <code class=”code”>shortterm</code>. By default the momentary value will be used, but in certain scenarios it may be more useful to observe the short term value instead (e.g. live mixing). </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Sets the display scale for the loudness. Valid parameters are <code class=”code”>absolute</code> (in LUFS) or <code class=”code”>relative</code> (LU) relative to the target. This only affects the video output, not the summary or continuous log output. </dd> <dt><samp class=”option”>integrated</samp></dt> <dd> Read-only exported value for measured integrated loudness, in LUFS. </dd> <dt><samp class=”option”>range</samp></dt> <dd> Read-only exported value for measured loudness range, in LU. </dd> <dt><samp class=”option”>lra_low</samp></dt> <dd> Read-only exported value for measured LRA low, in LUFS. </dd> <dt><samp class=”option”>lra_high</samp></dt> <dd> Read-only exported value for measured LRA high, in LUFS. </dd> <dt><samp class=”option”>sample_peak</samp></dt> <dd> Read-only exported value for measured sample peak, in dBFS. </dd> <dt><samp class=”option”>true_peak</samp></dt> <dd> Read-only exported value for measured true peak, in dBFS. </dd> </dl>

 <h4 class=”subsection”>39.10.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Real-time graph using <code class=”command”>ffplay</code>, with a EBU scale meter +18: <pre class=”example-preformatted”>ffplay -f lavfi -i "amovie=input.mp3,ebur128=video=1:meter=18 [out0][out1]" </pre>

Run an analysis with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -nostats -i input.mp3 -filter_complex ebur128 -f null - </pre>

 <h3 class="section">39.11 interleave, ainterleave# TOC</h3>

Temporally interleave frames from several inputs. <code class="code">interleave</code> works with video inputs, <code class="code">ainterleave</code> with audio. These filters read frames from several inputs and send the oldest queued frame to the output. Input streams must have well defined, monotonically increasing frame timestamp values. In order to submit one frame to output, these filters need to enqueue at least one frame for each input, so they cannot work in case one input is not yet terminated and will not receive incoming frames. For example consider the case when one input is a <code class="code">select</code> filter which always drops input frames. The <code class="code">interleave</code> filter will keep reading from that input, but it will never be able to send new frames to output until the input sends an end-of-stream signal. Also, depending on inputs synchronization, the filters will drop frames in case one input receives more frames than the other ones, and the queue is already filled. These filters accept the following options: <dl class="table"> <dt><samp class="option">nb_inputs, n</samp></dt> <dd> Set the number of different inputs, it is 2 by default. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class="option">first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">39.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interleave frames belonging to different streams using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i bambi.avi -i pr0n.mkv -filter_complex "[0:v][1:v] interleave" out.avi </pre>

Add flickering blur effect: <pre

class=”example-preformatted”>select='if(gt(random(0), 0.2), 1, 2)':n=2 [tmp], boxblur=2:2, [tmp] interleave </pre>

 <h3 class=”section”>39.12 latency, alatency# TOC</h3> Measure filtering latency. Report previous filter filtering latency, delay in number of audio samples for audio filters or number of video frames for video filters. On end of input stream, filter will report min and max measured latency for previous running filter in filtergraph. <h3 class=”section”>39.13 metadata, ametadata# TOC</h3>

Manipulate frame metadata. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>select</samp>’</dt> <dd> If both <code class=”code”>value</code> and <code class=”code”>key</code> is set, select frames which have such metadata. If only <code class=”code”>key</code> is set, select every frame that has such key in metadata. </dd> <dt>‘<samp class=”samp”>add</samp>’</dt> <dd> Add new metadata <code class=”code”>key</code> and <code class=”code”>value</code>. If key is already available do nothing. </dd> <dt>‘<samp class=”samp”>modify</samp>’</dt> <dd> Modify value of already present key. </dd> <dt>‘<samp class=”samp”>delete</samp>’</dt> <dd> If <code class=”code”>value</code> is set, delete only keys that have such value. Otherwise, delete key. If <code class=”code”>key</code> is not set, delete all metadata values in the frame. </dd> <dt>‘<samp class=”samp”>print</samp>’</dt> <dd> Print key and its value if metadata was found. If <code class=”code”>key</code> is not set print all metadata values available in frame. </dd> </dl>

</dd> <dt><samp class=”option”>key</samp></dt> <dd> Set key used with all modes. Must be set for all modes except <code class=”code”>print</code> and <code class=”code”>delete</code>. </dd> <dt><samp class=”option”>value</samp></dt> <dd> Set metadata value which will be used. This option is mandatory for <code class=”code”>modify</code> and <code class=”code”>add</code> mode. </dd> <dt><samp class=”option”>function</samp></dt> <dd> Which function to use when comparing metadata value and <code class=”code”>value</code>. Can be one of following: <dl class=”table”> <dt>‘<samp class=”samp”>same_str</samp>’</dt>

<dd> Values are interpreted as strings, returns true if metadata value is same as <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>starts_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value starts with the <code class=”code”>value</code> option string. </dd> <dt>‘<samp class=”samp”>less</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is less than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>equal</samp>’</dt> <dd> Values are interpreted as floats, returns true if <code class=”code”>value</code> is equal with metadata value. </dd> <dt>‘<samp class=”samp”>greater</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is greater than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>expr</samp>’</dt> <dd> Values are interpreted as floats, returns true if expression from option <code class=”code”>expr</code> evaluates to true. </dd> <dt>‘<samp class=”samp”>ends_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value ends with the <code class=”code”>value</code> option string. </dd> </dl>

</dd> <dt><samp class=”option”>expr</samp></dt> <dd> Set expression which is used when <code class=”code”>function</code> is set to <code class=”code”>expr</code>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>VALUE1, FRAMEVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> from metadata key. </dd> <dt><samp class=”option”>VALUE2, USERVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> as supplied by user in <code class=”code”>value</code> option. </dd> </dl>

</dd> <dt><samp class=”option”>file</samp></dt> <dd> If specified in <code class=”code”>print</code> mode, output is written to the named file. Instead of plain filename any writable url can be specified. Filename “-” is a shorthand for standard output. If <code class=”code”>file</code> option is not set, output is written to the log with AV_LOG_INFO loglevel. </dd> <dt><samp class=”option”>direct</samp></dt> <dd> Reduces buffering in print mode when output is written to a URL set using <var class=”var”>file</var>. </dd> </dl>

 <h4 class=”subsection”>39.13.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print all metadata values for frames with key <code class=”code”>lavfi.signalstats.YDIF</code> with val-

ues between 0 and 1. <pre class=”example-preformatted”>signalstats,metadata=print:key=lavfi.signalstats.YDIF:value=0:function=expr:expr='between
</pre> Print silencedetect output to file <samp class=”file”>metadata.txt</samp>.
 <pre class=”example-preformatted”>silencedetect,ametadata=mode=print:file=meta
</pre> Direct all metadata to a pipe with file descriptor 4.
 <pre class=”example-preformatted”>metadata=mode=print:file='pipe\:4'
</pre>

 <h3 class=”section”>39.14 perms, aperms# TOC</h3>

Set read/write permissions for the output frames. These filters are mainly aimed at developers to test direct path in the following filter in the filtergraph. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Select the permissions mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Do nothing. This is the default. </dd> <dt>‘<samp class=”samp”>ro</samp>’</dt> <dd> Set all the output frames read-only. </dd> <dt>‘<samp class=”samp”>rw</samp>’</dt> <dd> Set all the output frames directly writable. </dd> <dt>‘<samp class=”samp”>toggle</samp>’</dt> <dd> Make the frame read-only if writable, and writable if read-only. </dd> <dt>‘<samp class=”samp”>random</samp>’</dt> <dd> Set each output frame read-only or writable randomly. </dd> </dl>

</dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set the seed for the <var class=”var”>random</var> mode, must be an integer included between <code class=”code”>0</code> and <code class=”code”>UINT32_MAX</code>. If not specified, or if explicitly set to <code class=”code”>-1</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

Note: in case of auto-inserted filter between the permission filter and the following one, the permission might not be received as expected in that following filter. Inserting a format or aformat filter before the perms/aperms filter can avoid this problem. <h3 class=”section”>39.15 realtime, arealtime# TOC</h3>

Slow down filtering to match real time approximately. These filters will pause the filtering for a variable amount of time to match the output rate with the input timestamps. They are similar to the <samp

class=”option”>re</samp> option to <code class=”code”>ffmpeg</code>. They accept the following options: <dl class=”table”> <dt><samp class=”option”>limit</samp></dt> <dd> Time limit for the pauses. Any pause longer than that will be considered a timestamp discontinuity and reset the timer. Default is 2 seconds. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Speed factor for processing. The value must be a float larger than zero. Values larger than 1.0 will result in faster than realtime processing, smaller will slow processing down. The <var class=”var”>limit</var> is automatically adapted accordingly. Default is 1.0. A processing speed faster than what is possible without these filters cannot be achieved. </dd> </dl>

 <h4 class=”subsection”>39.15.1 Commands# TOC</h4>

Both filters supports the all above options as commands. <h3 class=”section”>39.16 segment, asegment# TOC</h3

Split single input stream into multiple streams. This filter does opposite of concat filters. <code class=”code”>segment</code> works on video frames, <code class=”code”>asegment</code> on audio samples. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>timestamps</samp></dt> <dd> Timestamps of output segments separated by ’|’. The first segment will run from the beginning of the input stream. The last segment will run until the end of the input stream </dd> <dt><samp class=”option”>frames, samples</samp></dt> <dd> Exact frame/sample count to split the segments. </dd> </dl>

In all cases, prefixing an each segment with ’+’ will make it relative to the previous segment. <h4 class=”subsection”>39.16.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Split input audio stream into three output audio streams, starting at start of input audio stream and storing that in 1st output audio stream, then following at 60th second and storing than in 2nd output audio stream, and last after 150th second of input audio stream store in 3rd output audio stream: <pre class=”example-preformatted”>asegment=timestamps="60|150" </pre>

<h3 class=”section”>39.17 select, aselect<a
class=”anchor hidden-xs” href=”#select_002c-aselect” aria-hidden=”true”>#
TOC</h3>

Select frames to pass in output. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>expr, e</samp></dt> <dd> Set expression, which is evaluated for each input frame. If the expression is evaluated to zero, the frame is discarded. If the evaluation result is negative or NaN, the frame is sent to the first output; otherwise it is sent to the output with index <code class=”code”>ceil(val)-1</code>, assuming that the input index starts from 0. For example a value of <code class=”code”>1.2</code> corresponds to the output with index <code class=”code”>ceil(1.2)-1 = 2-1 = 1</code>, that is the second output. </dd> <dt><samp class=”option”>outputs, n</samp></dt> <dd> Set the number of outputs. The output to which to send the selected frame is based on the result of the evaluation. Default value is 1. </dd> </dl>

The expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the filtered frame, starting from 0. </dd> <dt><samp class=”option”>selected_n</samp></dt> <dd> The (sequential) number of the selected frame, starting from 0. </dd> <dt><samp class=”option”>prev_selected_n</samp></dt> <dd> The sequential number of the last selected frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the filtered frame, expressed in <var class=”var”>TB</var> units. It’s NAN if undefined. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The PTS of the filtered frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_pts</samp></dt> <dd> The PTS of the previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_pts</samp></dt> <dd> The PTS of the last previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_t</samp></dt> <dd> The PTS of the last previously selected frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> The first PTS in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>start_t</samp></dt> <dd> The first PTS, in seconds, in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>pict_type (video only)</samp></dt> <dd> The type of the filtered frame. It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>I</samp></dt> <dt><samp

class=”option”>P</samp></dt> <dt><samp class=”option”>B</samp></dt>
<dt><samp class=”option”>S</samp></dt> <dt><samp class=”option”>SI</samp></dt>
<dt><samp class=”option”>SP</samp></dt> <dt><samp class=”option”>BI</samp></dt>
</dl>

</dd> <dt><samp class=”option”>interlace_type (video
only)</samp></dt> <dd> The frame interlace type. It can assume one of the following values: <dl class=”table”> <dt><samp
class=”option”>PROGRESSIVE</samp></dt> <dd> The frame is progressive (not interlaced). </dd> <dt><samp class=”option”>TOPFIRST</samp></dt>
<dd> The frame is top-field-first. </dd> <dt><samp class=”option”>BOTTOMFIRST</samp><
<dd> The frame is bottom-field-first. </dd> </dl>

</dd> <dt><samp class=”option”>consumed_sample_n (audio
only)</samp></dt> <dd> the number of selected samples before
the current frame </dd> <dt><samp class=”option”>samples_n
(audio only)</samp></dt> <dd> the
number of samples in the current frame </dd> <dt><samp
class=”option”>sample_rate (audio only)</samp></dt>
<dd> the input sample rate </dd> <dt><samp class=”option”>key</samp></dt>
<dd> This is 1 if the filtered frame is a key-frame, 0 otherwise.
</dd> <dt><samp class=”option”>pos</samp></dt> <dd> the
position in the file of the filtered frame, -1 if the information is not
available (e.g. for synthetic video); deprecated, do not use
</dd> <dt><samp class=”option”>scene (video
only)</samp></dt> <dd> value between 0 and 1 to indicate
a new scene; a low value reflects a low probability for the current frame
to introduce a new scene, while a higher value means the current frame is
more likely to be one (see the example below) </dd> <dt><samp
class=”option”>concatdec_select</samp></dt> <dd> The concat
demuxer can select only part of a concat input file by setting an inpoint
and an outpoint, but the output packets may not be entirely contained
in the selected interval. By using this variable, it is possible to skip
frames generated by the concat demuxer which are not exactly contained
in the selected interval. This works by comparing the frame
pts against the <var class=”var”>lavf.concat.start_time</var> and the
<var class=”var”>lavf.concat.duration</var> packet metadata values
which are also present in the decoded frames. The <var
class=”var”>concatdec_select</var> variable is -1 if the frame pts is at least
start_time and either the duration metadata is missing or the frame pts is
less than start_time + duration, 0 otherwise, and NaN if the start_time
metadata is missing. That basically means that an input frame
is selected if its pts is within the interval set by the concat demuxer.
 </dd> <dt><samp class=”option”>iw (video
only)</samp></dt> <dd> Represents the width of the input video frame. </dd> <dt><samp class=”option”>ih <em
class=”emph”>(video only)</samp></dt> <dd> Represents the

height of the input video frame. </dd> <dt><samp class="option">view (video only)</samp></dt> <dd> View ID for multi-view video. </dd> </dl>

The default value of the select expression is "1". <h4 class="subsection">39.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select all frames in input: <pre class="example-preformatted">select </pre>

The example above is the same as: <pre class="example-preformatted">select=1 </pre>

Skip all frames: <pre class="example-preformatted">select=0 </pre>

Select only I-frames: <pre class="example-preformatted">select='eq(pict_type\,I)' </pre>

Select one frame every 100: <pre class="example-preformatted">select='not(mod(n\,100))' </pre>

Select only frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20) </pre>

Select only I-frames contained in the 10-20 time interval: <pre class="example-preformatted">select=between(t\,10\,20)*eq(pict_type\,I) </pre>

Select frames with a minimum distance of 10 seconds: <pre class="example-preformatted">select='isnan(prev_selected_t)+gte(t-prev_selected_t\,10)' </pre>

Use aselect to select only audio frames with samples number > 100: <pre class="example-preformatted">aselect='gt(samples_n\,100)' </pre>

Create a mosaic of the first scenes: <pre class="example-preformatted">ffmpeg -i video.avi -vf select='gt(scene\,0.4)',scale=160:120,tile -frames:v 1 preview.png </pre>

Comparing <var class="var">scene</var> against a value between 0.3 and 0.5 is generally a sane choice. Send even and odd frames to separate outputs, and compose them: <pre class="example-preformatted">select=n=2:e='mod(n, 2)+1' [odd][even]; [odd] pad=h=2*ih [tmp]; [tmp][even] overlay=y=h </pre>

Select useful frames from an ffconcat file which is using inpoints and outpoints but where the source files are not intra frame only. <pre class="example-preformatted">ffmpeg -copyts -segment_time_metadata 1 -i input.ffconcat -fps_mode passthrough -vf select=concatdec_select -af aselect=concatdec_select output.avi </pre>

 <h3 class="section">39.18 sendcmd, asendcmd# TOC</h3>

Send commands to filters in the filtergraph. These filters read commands to be sent to other filters in the filtergraph. <code class="code">sendcmd</code> must be inserted between two video filters, <code class="code">asendcmd</code> must be inserted between two audio filters, but apart from that they act the same way. The specification of commands can be provided in the filter arguments with the <var class="var">commands</var> option, or in a file specified by the <var class="var">filename</var> option. These filters accept the following options: <dl class="table"> <dt><samp class="option">commands, c</samp></dt> <dd> Set the commands to be read and sent to the other filters. </dd> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the commands to be read and sent to the other filters. </dd> </dl>

 <h4 class="subsection">39.18.1 Commands syntax# TOC</h4>

A commands description consists of a sequence of interval specifications, comprising a list of commands to be executed when a particular event related to that interval occurs. The occurring event is typically the current frame time entering or leaving a given time interval. An interval is specified by the following syntax: <pre class="example-preformatted"><var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var>; </pre>

The time interval is specified by the <var class="var">START</var> and <var class="var">END</var> times. <var class="var">END</var> is optional and defaults to the maximum time. The current frame time is considered within the specified interval if it is included in the interval [<var class="var">START</var>, <var class="var">END</var>), that is when the time is greater or equal to <var class="var">START</var> and is lesser than <var class="var">END</var>. <var

class=”var”>COMMANDS</var> consists of a sequence of one or more command specifications, separated by ",", relating to that interval. The syntax of a command specification is given by: <pre class=”example-preformatted”>[<var class=”var”>FLAGS</var>] <var class=”var”>TARGET</var> <var class=”var”>COMMAND</var> <var class=”var”>ARG</var> </pre>

<var class=”var”>FLAGS</var> is optional and specifies the type of events relating to the time interval which enable sending the specified command, and must be a non-null sequence of identifier flags separated by "+" or "|" and enclosed between "[" and "]". The following flags are recognized: <dl class=”table”> <dt><samp class=”option”>enter</samp></dt> <dd> The command is sent when the current frame timestamp enters the specified interval. In other words, the command is sent when the previous frame timestamp was not in the given interval, and the current is. </dd> <dt><samp class=”option”>leave</samp></dt> <dd> The command is sent when the current frame timestamp leaves the specified interval. In other words, the command is sent when the previous frame timestamp was in the given interval, and the current is not. </dd> <dt><samp class=”option”>expr</samp></dt> <dd> The command <var class=”var”>ARG</var> is interpreted as expression and result of expression is passed as <var class=”var”>ARG</var>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>POS</samp></dt> <dd> Original position in the file of the frame, or undefined if undefined for the current frame. Deprecated, do not use. </dd> <dt><samp class=”option”>PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class=”option”>N</samp></dt> <dd> The count of the input frame for video or audio, starting from 0. </dd> <dt><samp class=”option”>T</samp></dt> <dd> The time in seconds of the current frame. </dd> <dt><samp class=”option”>TS</samp></dt> <dd> The start time in seconds of the current command interval. </dd> <dt><samp class=”option”>TE</samp></dt> <dd> The end time in seconds of the current command interval. </dd> <dt><samp class=”option”>TI</samp></dt> <dd> The interpolated time of the current command interval, TI = (T - TS) / (TE - TS). </dd> <dt><samp class=”option”>W</samp></dt> <dd> The video frame width. </dd> <dt><samp class=”option”>H</samp></dt> <dd> The video frame height. </dd> </dl>

</dd> </dl>

If <var class=”var”>FLAGS</var> is not specified, a default value of <code class=”code”>[enter]</code> is assumed. <var class=”var”>TARGET</var> specifies the target of the command, usu-

ally the name of the filter class or a specific filter instance name. <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional list of argument for the given <var class="var">COMMAND</var>. Between one interval specification and another, whitespaces, or sequences of characters starting with <code class="code">#</code> until the end of line, are ignored and can be used to annotate comments. A simplified BNF description of the commands specification syntax follows: <pre class="example-preformatted"><var class="var">COMMAND_FLAG</var> ::= "enter" | "leave" <var class="var">COMMAND_FLAGS</var> ::= <var class="var">COMMAND_FLAG</var> [(+|"|")<var class="var">COMMAND_FLAG</var>] <var class="var">COMMAND</var> ::= ["[" <var class="var">COMMAND_FLAGS</var> "]"] <var class="var">TARGET</var> <var class="var">COMMAND</var> [<var class="var">ARG</var>] <var class="var">COMMANDS</var> ::= <var class="var">COMMAND</var> [,<var class="var">COMMANDS</var>] <var class="var">INTERVAL</var> ::= <var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var> <var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[;<var class="var">INTERVALS</var>] </pre>

 <h4 class="subsection">39.18.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Specify audio tempo change at second 4: <pre class="example-preformatted">asendcmd=c='4.0 atempo tempo 1.5',atempo </pre>

Target a specific filter instance: <pre class="example-preformatted">asendcmd=c='4.0 atempo@my tempo 1.5',atempo@my </pre>

Specify a list of drawtext and hue commands in a file. <pre class="example-preformatted"># show text in the interval 5-10 5.0-10.0 [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=hello world', [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=';

desaturate the image in the interval 15-20 15.0-20.0 [enter] hue s 0, [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=nocolor', [leave] hue s 1, [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=color';

apply an exponential saturation fade-out effect, starting from time 25 25 [enter] hue s exp(25-t) </pre>

A filtergraph allowing to read and process the above command list stored in a file <samp class="file">test.cmd</samp>, can be

specified with: <pre class="example-preformatted">sendcmd=f=test.cmd,drawtext=fontfile=FreeSerif.ttf:text='',hue
</pre>

<h3 class="section">39.19 setpts, asetpts<a
class="anchor hidden-xs" href="#setpts_002c-asetpts" aria-hidden="true">#
TOC</h3>

Change the PTS (presentation timestamp) of the input frames.
<code class="code">setpts</code> works on video frames, <code
class="code">asetpts</code> on audio frames. This filter
accepts the following options: <dl class="table"> <dt><samp
class="option">expr</samp></dt> <dd> The expression which is evaluated for each frame to construct its timestamp. </dd> <dt><samp
class="option">strip_fps (video only)</samp></dt>
<dd> Boolean option which determines if the original framerate and frame duration metadata is unset. If set to true, be advised that a sane frame rate should be explicitly specified if output is sent to a constant frame rate muxer. Default is <code class="code">false</code>. </dd> </dl>

The expression is evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp
class="option">FRAME_RATE, FR</samp></dt> <dd> frame rate, only defined for constant frame-rate video </dd> <dt><samp
class="option">PTS</samp></dt> <dd> The presentation timestamp in input </dd> <dt><samp class="option">N</samp></dt>
<dd> The count of the input frame for video or the number of consumed samples, not including the current frame for audio, starting from 0.
</dd> <dt><samp class="option">NB_CONSUMED_SAMPLES</samp></dt>
<dd> The number of consumed samples, not including the current frame (only audio) </dd> <dt><samp class="option">NB_SAMPLES, S</samp></dt> <dd> The number of samples in the current frame (only audio) </dd> <dt><samp class="option">SAMPLE_RATE, SR</samp></dt> <dd> The audio sample rate. </dd>
<dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first frame. </dd> <dt><samp class="option">STARTT</samp></dt>
<dd> the time in seconds of the first frame </dd> <dt><samp
class="option">INTERLACED</samp></dt> <dd> State whether the current frame is interlaced. </dd> <dt><samp class="option">T</samp></dt>
<dd> the time in seconds of the current frame </dd> <dt><samp
class="option">POS</samp></dt> <dd> original position in the file of the frame, or undefined if undefined for the current frame; deprecated, do not use
 </dd> <dt><samp class="option">PREV_INPTS</samp></dt>
<dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INT</samp></dt>
<dd> previous input time in seconds </dd> <dt><samp
class="option">PREV_OUTPTS</samp></dt> <dd> The previous

output PTS. </dd> <dt><samp class="option">PREV_OUTT</samp></dt> <dd> previous output time in seconds </dd> <dt><samp class="option">RTCTIME</samp></dt> <dd> The wallclock (RTC) time in microseconds. This is deprecated, use time(0) instead. </dd> <dt><samp class="option">RTCSTART</samp></dt> <dd> The wallclock (RTC) time at the start of the movie in microseconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class="option">T_CHANGE</samp></dt> <dd> Time of the first frame after command was applied or time of the first frame if no commands. </dd> </dl>

 <h4 class="subsection">39.19.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Start counting PTS from zero <pre class="example-preformatted">setpts=PTS-STARTPTS </pre>

Apply fast motion effect: <pre class="example-preformatted">setpts=0.5*PTS </pre>

Apply slow motion effect: <pre class="example-preformatted">setpts=2.0*PTS </pre>

Set fixed rate of 25 frames per second: <pre class="example-preformatted">setpts=N/(25*TB) </pre>

Apply a random jitter effect of +/-100 TB units: <pre class="example-preformatted">setpts=PTS+randomi(0, -100\,100) </pre>

Set fixed rate 25 fps with some jitter: <pre class="example-preformatted">setpts='1/(25*TB) * (N + 0.05 * sin(N*2*PI/25))' </pre>

Apply an offset of 10 seconds to the input PTS: <pre class="example-preformatted">setpts=PTS+10/TB </pre>

Generate timestamps from a "live source" and rebase onto the current timebase: <pre class="example-preformatted">setpts='(RTCTIME - RTCSTART) / (TB * 1000000)' </pre>

Generate timestamps by counting samples: <pre class="example-preformatted">asetpts=N/SR/TB </pre>

 <h4 class="subsection">39.19.2 Commands# TOC</h4>

Both filters support all above options as commands. <h3 class="section">39.20 setrange# TOC</h3>

Force color range for the output video frame. The <code class="code">setrange</code> filter marks the color range property for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters. The filter accepts the following options: <dl class="table"> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property. </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Set the color range as unspecified. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Set the color range as limited. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Set the color range as full. </dd> </dl> </dd> </dl>

 <h3 class="section">39.21 settb, asettb# TOC</h3>

Set the timebase to use for the output frames timestamps. It is mainly useful for testing timebase configuration. It accepts the following parameters: <dl class="table"> <dt><samp class="option">expr, tb</samp></dt> <dd> The expression which is evaluated into the output timebase. </dd> </dl>

The value for <samp class="option">tb</samp> is an arithmetic expression representing a rational. The expression can contain the constants "AVTB" (the default timebase), "intb" (the input timebase) and "sr" (the sample rate, audio only). Default value is "intb". <h4 class="subsection">39.21.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the timebase to 1/25: <pre class="example-preformatted">settb=expr=1/25 </pre>

Set the timebase to 1/10: <pre class="example-preformatted">settb=expr=0.1 </pre>

Set the timebase to 1001/1000: <pre class="example-preformatted">settb=1+0.001 </pre>

Set the timebase to 2*intb: <pre class="example-preformatted">settb=2*intb </pre>

Set the default timebase value: <pre class="example-preformatted">settb=AVTB </pre>

<h3 class="section">39.22 showcqt# TOC</h3>

Convert input audio to a video output representing frequency spectrum

logarithmically using Brown-Puckette constant Q transform algorithm with

direct frequency domain coefficient calculation (but the transform itself

is not really constant Q, instead the Q factor is actually variable/clamped),

with musical tone scale, from E0 to D#10.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. It must be even. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">1920x1080</code>.

</dd>

<dt><samp class="option">fps, rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">bar_h</samp></dt>

<dd> Set the bargraph height. It must be even. Default value is <code class="code">-1</code> which

computes the bargraph height automatically.

</dd>

<dt><samp class="option">axis_h</samp></dt>

<dd> Set the axis height. It must be even. Default value is <code class="code">-1</code> which computes

the axis height automatically.

</dd>

<dt><samp class="option">sono_h</samp></dt>

<dd> Set the sonogram height. It must be even. Default value is <code class="code">-1</code> which

computes the sonogram height automatically.

</dd>

<dt><samp class="option">fullhd</samp></dt>

<dd> Set the fullhd resolution. This option is deprecated, use <var class="var">size</var>, <var class="var">s</var>

instead. Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">sono_v, volume</samp></dt>

<dd> Specify the sonogram volume expression. It can contain variables:

<dl class="table">

<dt><samp class="option">bar_v</samp></dt>

<dd> the <var class="var">bar_v</var> evaluated expression

</dd>

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option
</dd>
</dl>
and functions:
<dl class="table">
<dt><samp class="option">a_weighting(f)</samp></dt>
<dd> A-weighting of equal loudness
</dd>
<dt><samp class="option">b_weighting(f)</samp></dt>
<dd> B-weighting of equal loudness
</dd>
<dt><samp class="option">c_weighting(f)</samp></dt>
<dd> C-weighting of equal loudness.
</dd>
</dl>
Default value is <code class="code">16</code>.

</dd>
<dt><samp class="option">bar_v, volume2</samp></dt>
<dd> Specify the bargraph volume expression. It can contain variables:
<dl class="table">
<dt><samp class="option">sono_v</samp></dt>
<dd> the <var class="var">sono_v</var> evaluated expression
</dd>
<dt><samp class="option">frequency, freq, f</samp></dt>
<dd> the frequency where it is evaluated
</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">a_weighting(f)</samp></dt>

<dd> A-weighting of equal loudness

</dd>

<dt><samp class="option">b_weighting(f)</samp></dt>

<dd> B-weighting of equal loudness

</dd>

<dt><samp class="option">c_weighting(f)</samp></dt>

<dd> C-weighting of equal loudness.

</dd>

</dl>

Default value is <code class="code">sono_v</code>.

</dd>

<dt><samp class="option">sono_g, gamma</samp></dt>

<dd> Specify the sonogram gamma. Lower gamma makes the spectrum more contrast,

higher gamma makes the spectrum having more range. Default value is <code class="code">3</code>.

Acceptable range is <code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_g, gamma2</samp></dt>

<dd> Specify the bargraph gamma. Default value is <code class="code">1</code>. Acceptable range is

<code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_t</samp></dt>

<dd> Specify the bargraph transparency level. Lower value makes the bargraph sharper.

Default value is <code class="code">1</code>. Acceptable range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> Specify the transform timeclamp. At low frequency, there is trade-off between

accuracy in time domain and frequency domain. If timeclamp is lower,

event in time domain is represented more accurately (such as fast bass drum),

otherwise event in frequency domain is represented more accurately

(such as bass guitar). Acceptable range is <code class="code">[0.002, 1]</code>. Default value is <code class="code">0.17</code>.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set attack time in seconds. The default is <code class="code">0</code> (disabled). Otherwise, it

limits future samples by applying asymmetric windowing in time domain, useful

when low latency is required. Accepted range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">basefreq</samp></dt>

<dd> Specify the transform base frequency. Default value is <code class="code">20.01523126408007475</code>,

which is frequency 50 cents below E0. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">endfreq</samp></dt>

<dd> Specify the transform end frequency. Default value is <code class="code">20495.59681441799654</code>,

which is frequency 50 cents above D#10. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">coeffclamp</samp></dt>

<dd> This option is deprecated and ignored.

</dd>

<dt><samp class="option">tlength</samp></dt>

<dd> Specify the transform length in time domain. Use this option to control accuracy

trade-off between time domain and frequency domain at every frequency sample.

It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option.

</dd>

</dl>

Default value is <code class="code">384*tc/(384+tc*f)</code>.

</dd>

<dt><samp class="option">count</samp></dt>

<dd> Specify the transform count for every video frame. Default value is <code class="code">6</code>.

Acceptable range is <code class="code">[1, 30]</code>.

</dd>

<dt><samp class="option">fcount</samp></dt>

<dd> Specify the transform count for every single pixel. Default value is <code class="code">0</code>,

which makes it computed automatically. Acceptable range is <code class="code">[0, 10]</code>.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> Specify font file for use with freetype to draw the axis. If not specified,

use embedded font. Note that drawing with font file or embedded font is not

implemented with custom <var class="var">basefreq</var> and <var class="var">endfreq</var>, use <var class="var">axisfile</var>

option instead.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> Specify fontconfig pattern. This has lower priority than <var class="var">fontfile</var>. The

<code class="code">:</code> in the pattern may be replaced by <code class="code">|</code> to avoid unnecessary

escaping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> Specify font color expression. This is arithmetic expression that should return

integer value 0xRRGGBB. It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">midi(f)</samp></dt>

<dd> midi number of frequency f, some midi numbers: E0(16), C1(24), C2(36), A4(69)

</dd>

<dt><samp class="option">r(x), g(x), b(x)</samp></dt>

<dd> red, green, and blue value of intensity x.

</dd>

</dl>

Default value is <code class="code">st(0, (midi(f)-59.5)/12);

st(1, if(between(ld(0),0,1), 0.5-0.5*cos(2*PI*ld(0)), 0));

r(1-ld(1)) + b(ld(1))</code>.

</dd>

<dt><samp class="option">axisfile</samp></dt>

<dd> Specify image file to draw the axis. This option override <var class="var">fontfile</var> and

<var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">axis, text</samp></dt>

<dd> Enable/disable drawing text to the axis. If it is set to <code class="code">0</code>, drawing to

the axis is disabled, ignoring <var class="var">fontfile</var> and <var class="var">axisfile</var> option.

Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">csp</samp></dt>

<dd> Set colorspace. The accepted values are:

<dl class="table">
<dt>‘<samp class="samp">unspecified</samp>’</dt>
<dd> Unspecified (default)

</dd>
<dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709

</dd>
<dt>‘<samp class="samp">fcc</samp>’</dt>
<dd> FCC

</dd>
<dt>‘<samp class="samp">bt470bg</samp>’</dt>
<dd> BT.470BG or BT.601-6 625

</dd>
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M or BT.601-6 525

</dd>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dd> SMPTE-240M

</dd>
<dt>‘<samp class="samp">bt2020ncl</samp>’</dt>
<dd> BT.2020 with non-constant luminance

</dd>
</dl>

</dd> <dt><samp class="option">cscheme</samp></dt> <dd> Set spectrogram color scheme. This is list of floating point values with format <code class="code">left_r|left_g|left_b|right_r|right_g|right_b</code>. The default is <code class="code">1|0.5|0|0|0.5|1</code>. </dd> </dl>

 <h4 class="subsection">39.22.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Playing audio while showing the spectrum: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with frame rate 30 fps: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=fps=30:count=5 [out0]' </pre>

Playing at 1280x720: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=s=1280x720:count=4 [out0]' </pre>

Disable sonogram display: <pre class="example-preformatted">sono_h=0 </pre>

A1 and its harmonics: A1, A2, (near)E3, A3: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6
asplit[a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with more accuracy in frequency domain: <pre class="example-preformatted">ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6*PI*55*t)+0.1*sin(8*PI*55*t), asplit[a][out1]; [a] showcqt=timeclamp=0.5 [out0]' </pre>

Custom volume: <pre class="example-preformatted">bar_v=10:sono_v=bar_v*a_weighting(f) </pre>

Custom gamma, now spectrum is linear to the amplitude. <pre class="example-preformatted">bar_g=2:sono_g=2 </pre>

Custom tlength equation: <pre class="example-preformatted">tc=0.33:tlength='st(0,0.17); 384*tc / (384 / ld(0) + tc*f /(1-ld(0))) + 384*tc / (tc*f / ld(0) + 384 /(1-ld(0)))' </pre>

Custom fontcolor and fontfile, C-note is colored green, others are colored blue: <pre class="example-preformatted">fontcolor='if(mod(floor(midi(f)+0.5),12), 0x0000FF, g(1))':fontfile=myfont.ttf </pre>

Custom font using fontconfig: <pre class="example-preformatted">font='Courier New,Monospace,mono|bold'

</pre>

Custom frequency range with custom axis using image file: <pre class="example-preformatted">axisfile=myaxis.png:basefreq=40:endfreq=10000
</pre>

 <h3 class="section">39.23 showcwt# TOC</h3>

Convert input audio to video output representing frequency spectrum

using Continuous Wavelet Transform and Morlet wavelet.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">640x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set the frequency scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">bark</samp></dt>

<dt><samp class="option">mel</samp></dt>

<dt><samp class="option">erbs</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

<dt><samp class="option">fm</samp></dt>

</dl>

Default value is <code class="code">linear</code>.

</dd>

<dt><samp class="option">iscale</samp></dt>

<dd> Set the intensity scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

</dl>

Default value is <code class="code">log</code>.

</dd>

<dt><samp class="option">min</samp></dt>

<dd> Set the minimum frequency that will be used in output.

Default is <code class="code">20</code> Hz.

</dd>

<dt><samp class="option">max</samp></dt>

<dd> Set the maximum frequency that will be used in output.

Default is <code class="code">20000</code> Hz. The real frequency upper limit

depends on input audio’s sample rate and such will be enforced
on this value when it is set to value greater than Nyquist frequency.

</dd>
<dt><samp class="option">imin</samp></dt>
<dd> Set the minimum intensity that will be used in output.

</dd>
<dt><samp class="option">imax</samp></dt>
<dd> Set the maximum intensity that will be used in output.

</dd>
<dt><samp class="option">logb</samp></dt>
<dd> Set the logarithmic basis for brightness strength when
mapping calculated magnitude values to pixel values.
Allowed range is from <code class="code">0</code> to <code class="code">1</code>.
Default value is <code class="code">0.0001</code>.

</dd>
<dt><samp class="option">deviation</samp></dt>
<dd> Set the frequency deviation.
Lower values than <code class="code">1</code> are more frequency oriented,
while higher values than <code class="code">1</code> are more time oriented.
Allowed range is from <code class="code">0</code> to <code class="code">10</code>.
Default value is <code class="code">1</code>.

</dd>
<dt><samp class="option">pps</samp></dt>
<dd> Set the number of pixel output per each second in one row.
Allowed range is from <code class="code">1</code> to <code class="code">1024</code>.

Default value is <code class=”code”>64</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set the output visual mode. Allowed values are:

<dl class=”table”>

<dt><samp class=”option”>magnitude</samp></dt>

<dd> Show magnitude.

</dd>

<dt><samp class=”option”>phase</samp></dt>

<dd> Show only phase.

</dd>

<dt><samp class=”option”>magphase</samp></dt>

<dd> Show combination of magnitude and phase.

Magnitude is mapped to brightness and phase to color.

</dd>

<dt><samp class=”option”>channel</samp></dt>

<dd> Show unique color per channel magnitude.

</dd>

<dt><samp class=”option”>stereo</samp></dt>

<dd> Show unique color per stereo difference.

</dd>

</dl>

Default value is <code class=”code”>magnitude</code>. </dd>
<dt><samp class=”option”>slide</samp></dt> <dd> Set the output slide method. Allowed values are: <dl class=”table”> <dt><samp class=”option”>replace</samp></dt> <dt><samp class=”option”>scroll</samp></dt>
<dt><samp class=”option”>frame</samp></dt> </dl>

</dd> <dt><samp class=”option”>direction</samp></dt> <dd> Set the direction method for output slide method. Allowed values are:
 <dl class=”table”> <dt><samp class=”option”>lr</samp></dt>
<dd> Direction from left to right. </dd> <dt><samp class=”option”>rl</samp></dt>

<dd> Direction from right to left. </dd> <dt><samp class="option">ud</samp></dt> <dd> Direction from up to down. </dd> <dt><samp class="option">du</samp></dt> <dd> Direction from down to up. </dd> </dl>

</dd> <dt><samp class="option">bar</samp></dt> <dd> Set the ratio of bargraph display to display size. Default is 0. </dd> <dt><samp class="option">rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">39.24 showfreqs# TOC</h3>

Convert input audio to video output representing the audio power spectrum.

Audio amplitude is on Y-axis while frequency is on X-axis.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify size of video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default is <code class="code">1024x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set video rate. Default is <code class="code">25</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Set display mode.

This set how each frequency bin will be represented.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">bar</samp>’</dt>
<dt>‘<samp class="samp">dot</samp>’</dt>
</dl>
Default is <code class="code">bar</code>.

</dd>
<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root scale.

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">fscale</samp></dt>
<dd> Set frequency scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.

</dd>
<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> Reverse logarithmic scale.
</dd>
</dl>
Default is <code class="code">lin</code>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from 16 to 65536.

Default is <code class="code">2048</code>

</dd>
<dt><samp class="option">win_func</samp></dt>
<dd> Set windowing function.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt>

<dt>‘<samp class="samp">hanning</samp>’</dt>

<dt>‘<samp class="samp">hamming</samp>’</dt>

<dt>‘<samp class="samp">blackman</samp>’</dt>

<dt>‘<samp class="samp">welch</samp>’</dt>

<dt>‘<samp class="samp">flattop</samp>’</dt>

<dt>‘<samp class="samp">bharris</samp>’</dt>

<dt>‘<samp class="samp">bnuttall</samp>’</dt>

<dt>‘<samp class="samp">bhann</samp>’</dt>

<dt>‘<samp class="samp">sine</samp>’</dt>

<dt>‘<samp class="samp">nuttall</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hanning</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. In range <code class="code">[0, 1]</code>. Default is <code class="code">1</code>,

which means optimal overlap for selected window function will be picked.

</dd>

<dt><samp class="option">averaging</samp></dt>

<dd> Set time averaging. Setting this to 0 will display current maximal peaks.

Default is <code class="code">1</code>, which means time averaging is disabled.

</dd>

<dt><samp class="option">colors</samp></dt>

<dd> Specify list of colors separated by space or by ’|’ which will be used to

draw channel frequencies. Unrecognized or missing colors will be replaced

by white color.

</dd>

<dt><samp class="option">cmode</samp></dt>

<dd> Set channel display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dt>‘<samp class="samp">separate</samp>’</dt>

</dl>

Default is <code class="code">combined</code>.

</dd>

<dt><samp class="option">minamp</samp></dt>

<dd> Set minimum amplitude used in <code class="code">log</code> amplitude scaler.

</dd>

<dt><samp class=”option”>data</samp></dt>

<dd> Set data display mode.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>magnitude</samp>’</dt>

<dt>‘<samp class=”samp”>phase</samp>’</dt>

<dt>‘<samp class=”samp”>delay</samp>’</dt>

</dl>

Default is <code class=”code”>magnitude</code>.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set channels to use when processing audio. By default all are processed.

</dd>

</dl>

 <h3 class=”section”>39.25 showspatial# TOC</h3>

Convert stereo input audio to a video output, representing the spatial relationship between two channels. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>512x512</code>. </dd> <dt><samp class=”option”>win_size</samp></dt> <dd> Set window size. Allowed range is from <var class=”var”>1024</var> to <var class=”var”>65536</var>. Default size is <var class=”var”>4096</var>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann</samp>’</dt> <dt>‘<samp class=”samp”>hanning</samp>’<
<dt>‘<samp class=”samp”>hamming</samp>’</dt>

<dt>‘<samp class=”samp”>blackman</samp>’</dt>
<dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp
class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’<
<dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp
class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt
<dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp
class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</
<dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp
class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</
<dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp
class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>&rsquo
<dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl>

Default value is <code class=”code”>hann</code>. </dd>
<dt><samp class=”option”>rate, r</samp></dt> <dd> Set output
framerate. </dd> </dl>

<a name=”showspectrum-
1”> <h3 class=”section”>39.26 showspectrum<span class=”pull-
right”>#
TOC</h3>

Convert input audio to a video output, representing the audio frequency spectrum. The filter accepts the following options:
<dl class=”table”> <dt><samp class=”option”>size, s</samp></dt>
<dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>640x512</code>.
</dd> <dt><samp class=”option”>slide</samp></dt> <dd> Specify how the spectrum should slide along the window. It accepts the following values: <dl class=”table”> <dt>‘<samp
class=”samp”>replace</samp>’</dt> <dd> the samples start again on the left when they reach the right </dd> <dt>‘<samp
class=”samp”>scroll</samp>’</dt> <dd> the samples scroll from right to left </dd> <dt>‘<samp class=”samp”>fullframe</samp>’</dt>
<dd> frames are only produced when the samples reach the right
</dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> the samples scroll from left to right </dd> <dt>‘<samp
class=”samp”>lreplace</samp>’</dt> <dd> the samples start again on the right when they reach the left </dd> </dl>

Default value is <code class=”code”>replace</code>. </dd>
<dt><samp class=”option”>mode</samp></dt> <dd> Specify display mode. It accepts the following values: <dl class=”table”>
<dt>‘<samp class=”samp”>combined</samp>’</dt>
<dd> all channels are displayed in the same row </dd> <dt>‘<samp
class=”samp”>separate</samp>’</dt> <dd> all channels are

displayed in separate rows </dd> </dl>

Default value is ‘<samp class="samp">combined</samp>’.
 </dd> <dt><samp class="option">color</samp></dt> <dd> Specify
display color mode. It accepts the following values: <dl
class="table"> <dt>‘<samp class="samp">channel</samp>’</dt>
<dd> each channel is displayed in a separate color </dd>
<dt>‘<samp class="samp">intensity</samp>’</dt> <dd> each
channel is displayed using the same color scheme </dd> <dt>‘<samp
class="samp">rainbow</samp>’</dt> <dd> each channel is dis-
played using the rainbow color scheme </dd> <dt>‘<samp
class="samp">moreland</samp>’</dt> <dd> each channel is
displayed using the moreland color scheme </dd> <dt>‘<samp
class="samp">nebulae</samp>’</dt> <dd> each channel is
displayed using the nebulae color scheme </dd> <dt>‘<samp
class="samp">fire</samp>’</dt> <dd> each channel is dis-
played using the fire color scheme </dd> <dt>‘<samp
class="samp">fiery</samp>’</dt> <dd> each channel is
displayed using the fiery color scheme </dd> <dt>‘<samp
class="samp">fruit</samp>’</dt> <dd> each channel is
displayed using the fruit color scheme </dd> <dt>‘<samp
class="samp">cool</samp>’</dt> <dd> each channel is dis-
played using the cool color scheme </dd> <dt>‘<samp
class="samp">magma</samp>’</dt> <dd> each channel is
displayed using the magma color scheme </dd> <dt>‘<samp
class="samp">green</samp>’</dt> <dd> each channel is
displayed using the green color scheme </dd> <dt>‘<samp
class="samp">viridis</samp>’</dt> <dd> each channel is
displayed using the viridis color scheme </dd> <dt>‘<samp
class="samp">plasma</samp>’</dt> <dd> each channel is
displayed using the plasma color scheme </dd> <dt>‘<samp
class="samp">cividis</samp>’</dt> <dd> each channel is
displayed using the cividis color scheme </dd> <dt>‘<samp
class="samp">terrain</samp>’</dt> <dd> each channel is
displayed using the terrain color scheme </dd> </dl>

Default value is ‘<samp class="samp">channel</samp>’.
 </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
scale used for calculating intensity color values. It accepts
the following values: <dl class="table"> <dt>‘<samp
class="samp">lin</samp>’</dt> <dd> linear </dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> square
root, default </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic </dd> <dt>‘<samp class="samp">4thrt</samp>’</dt>
<dd> 4th root </dd> <dt>‘<samp class="samp">5thrt</samp>’</dt>
<dd> 5th root </dd> </dl>

Default value is ‘<samp class="samp">sqrt</samp>’.
 </dd> <dt><samp class="option">fscale</samp></dt> <dd> Specify
frequency scale. It accepts the following values: <dl
class="table"> <dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic </dd> </dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values
provide alternative color scheme. <code class="code">0</code> is no
saturation at all. Saturation must be in [-10.0, 10.0] range. Default
value is <code class="code">1</code>. </dd> <dt><samp
class="option">win_func</samp></dt> <dd> Set window func-
tion. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp
class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp
class="samp">hamming</samp>’</dt> <dt>‘<samp
class="samp">blackman</samp>’</dt> <dt>‘<samp
class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp
class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp
class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp
class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp
class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp
class="samp">kaiser</samp>’</dt> </dl>

Default value is <code class="code">hann</code>. </dd>
<dt><samp class="option">orientation</samp></dt> <dd> Set orien-
tation of time vs frequency axis. Can be <code class="code">vertical</code>
or <code class="code">horizontal</code>. Default is <code class="code">vertical</code>.
 </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set
ratio of overlap window. Default value is <code class="code">0</code>.
When value is <code class="code">1</code> overlap is set to recommended
size for specific window function currently used. </dd> <dt><samp
class="option">gain</samp></dt> <dd> Set scale gain for calculating
intensity color values. Default value is <code class="code">1</code>.
</dd> <dt><samp class="option">data</samp></dt> <dd> Set
which data to display. Can be <code class="code">magnitude</code>,
default or <code class="code">phase</code>, or unwrapped phase: <code
class="code">uphase</code>. </dd> <dt><samp class="option">rotation</samp></dt>
<dd> Set color rotation, must be in [-1.0, 1.0] range. Default

value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set upper frame rate limit. Default is <code class=”code”>auto</code>, unlimited. </dd> <dt><samp class=”option”>legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is disabled. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

The usage is very similar to the showwaves filter; see the examples in that section. <h4 class=”subsection”>39.26.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Large window with logarithmic color scaling: <pre class=”example-preformatted”>showspectrum=s=1280x480:scale=log </pre>

Complete example for a colored and sliding spectrum per channel using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] showspectrum=mode=separate:color=intensity:slide=1:scale=cbrt [out0]' </pre>

 <h3 class=”section”>39.27 showspectrumpic# TOC</h3>

Convert input audio to a single video frame, representing the audio frequency

spectrum.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">4096x2048</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Specify display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dd> all channels are displayed in the same row

</dd>

<dt>‘<samp class="samp">separate</samp>’</dt>

<dd> all channels are displayed in separate rows

</dd>

</dl>

Default value is ‘<samp class="samp">combined</samp>’.

</dd>

<dt><samp class="option">color</samp></dt>

<dd> Specify display color mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">channel</samp>’</dt>

<dd> each channel is displayed in a separate color

</dd>

<dt>‘<samp class="samp">intensity</samp>’</dt>
<dd> each channel is displayed using the same color scheme
</dd>
<dt>‘<samp class="samp">rainbow</samp>’</dt>
<dd> each channel is displayed using the rainbow color scheme
</dd>
<dt>‘<samp class="samp">moreland</samp>’</dt>
<dd> each channel is displayed using the moreland color scheme
</dd>
<dt>‘<samp class="samp">nebulae</samp>’</dt>
<dd> each channel is displayed using the nebulae color scheme
</dd>
<dt>‘<samp class="samp">fire</samp>’</dt>
<dd> each channel is displayed using the fire color scheme
</dd>
<dt>‘<samp class="samp">fiery</samp>’</dt>
<dd> each channel is displayed using the fiery color scheme
</dd>
<dt>‘<samp class="samp">fruit</samp>’</dt>
<dd> each channel is displayed using the fruit color scheme
</dd>
<dt>‘<samp class="samp">cool</samp>’</dt>
<dd> each channel is displayed using the cool color scheme
</dd>
<dt>‘<samp class="samp">magma</samp>’</dt>
<dd> each channel is displayed using the magma color scheme
</dd>
<dt>‘<samp class="samp">green</samp>’</dt>
<dd> each channel is displayed using the green color scheme
</dd>
<dt>‘<samp class="samp">viridis</samp>’</dt>

<dd> each channel is displayed using the viridis color scheme
</dd>
<dt>‘<samp class="samp">plasma</samp>’</dt>
<dd> each channel is displayed using the plasma color scheme
</dd>
<dt>‘<samp class="samp">cividis</samp>’</dt>
<dd> each channel is displayed using the cividis color scheme
</dd>
<dt>‘<samp class="samp">terrain</samp>’</dt>
<dd> each channel is displayed using the terrain color scheme
</dd>
</dl>
Default value is ‘<samp class="samp">intensity</samp>’.

</dd>
<dt><samp class="option">scale</samp></dt>
<dd> Specify scale used for calculating intensity color values.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> square root, default
</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic

</dd>

<dt>‘<samp class="samp">4thrt</samp>’</dt>

<dd> 4th root

</dd>

<dt>‘<samp class="samp">5thrt</samp>’</dt>

<dd> 5th root

</dd>

</dl>

Default value is ‘<samp class="samp">log</samp>’.

</dd>

<dt><samp class="option">fscale</samp></dt>

<dd> Specify frequency scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">lin</samp>’</dt>

<dd> linear

</dd>

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

</dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values provide alternative color scheme. <code class="code">0</code> is no saturation at all. Saturation must be in [-10.0, 10.0] range. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp

class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</dt> <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’</dt> <dt>‘<samp class=”samp”>sine</samp>’</dt> <dt>‘<samp class=”samp”>nuttall</samp>’</dt> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dt>‘<samp class=”samp”>tukey</samp>’</dt> <dt>‘<samp class=”samp”>dolph</samp>’</dt> <dt>‘<samp class=”samp”>cauchy</samp>’</dt> <dt>‘<samp class=”samp”>parzen</samp>’</dt> <dt>‘<samp class=”samp”>poisson</samp>’</dt> <dt>‘<samp class=”samp”>bohman</samp>’</dt> <dt>‘<samp class=”samp”>kaiser</samp>’</dt> </dl> Default value is <code class=”code”>hann</code>. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of time vs frequency axis. Can be <code class=”code”>vertical</code> or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set scale gain for calculating intensity color values. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>legend</samp></dt> <dd> Draw time and frequency axes and legends. Default is enabled. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

 <h4 class=”subsection”>39.27.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Extract an audio spectrogram of a whole audio track in a 1024x1024 picture using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i audio.flac -lavfi showspectrumpic=s=1024x1024 spectrogram.png </pre>

 <h3 class="section">39.28 showvolume# TOC</h3>

Convert input audio volume to a video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set video rate. </dd> <dt><samp class="option">b</samp></dt> <dd> Set border width, allowed range is [0, 5]. Default is 1. </dd> <dt><samp class="option">w</samp></dt> <dd> Set channel width, allowed range is [80, 8192]. Default is 400. </dd> <dt><samp class="option">h</samp></dt> <dd> Set channel height, allowed range is [1, 900]. Default is 20. </dd> <dt><samp class="option">f</samp></dt> <dd> Set fade, allowed range is [0, 1]. Default is 0.95. </dd> <dt><samp class="option">c</samp></dt> <dd> Set volume color expression. The expression can use the following variables: <dl class="table"> <dt><samp class="option">VOLUME</samp></dt> <dd> Current max volume of channel in dB. </dd> <dt><samp class="option">PEAK</samp></dt> <dd> Current peak. </dd> <dt><samp class="option">CHANNEL</samp></dt> <dd> Current channel number, starting from 0. </dd> </dl>

</dd> <dt><samp class="option">t</samp></dt> <dd> If set, displays channel names. Default is enabled. </dd> <dt><samp class="option">v</samp></dt> <dd> If set, displays volume values. Default is enabled. </dd> <dt><samp class="option">o</samp></dt> <dd> Set orientation, can be horizontal: <code class="code">h</code> or vertical: <code class="code">v</code>, default is <code class="code">h</code>. </dd> <dt><samp class="option">s</samp></dt> <dd> Set step size, allowed range is [0, 5]. Default is 0, which means step is disabled. </dd> <dt><samp class="option">p</samp></dt> <dd> Set background opacity, allowed range is [0, 1]. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set metering mode, can be peak: <code class="code">p</code> or rms: <code class="code">r</code>, default is <code class="code">p</code>. </dd> <dt><samp class="option">ds</samp></dt> <dd> Set display scale, can be linear: <code class="code">lin</code> or log: <code class="code">log</code>, default is <code class="code">lin</code>. </dd> <dt><samp class="option">dm</samp></dt> <dd> In second. If set to > 0., display a line for the max level in the previous seconds. default is disabled: <code class="code">0.</code> </dd> <dt><samp class="option">dmc</samp></dt> <dd> The color of the max line. Use when <code class="code">dm</code> option is set to > 0. default is: <code class="code">orange</code> </dd> </dl>

 <h3 class="section">39.29 showwaves<span

class=”pull-right”># TOC</h3>

Convert input audio to a video output, representing the samples waves. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>600x240</code>. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set display mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>point</samp>’</dt> <dd> Draw a point for each sample. </dd> <dt>‘<samp class=”samp”>line</samp>’</dt> <dd> Draw a vertical line for each sample. </dd> <dt>‘<samp class=”samp”>p2p</samp>’</dt> <dd> Draw a point for each sample and a line between them. </dd> <dt>‘<samp class=”samp”>cline</samp>’</dt> <dd> Draw a centered vertical line for each sample. </dd> </dl>

Default value is <code class=”code”>point</code>. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Set the number of samples which are printed on the same column. A larger value will decrease the frame rate. Must be a positive integer. This option can be set only if the value for <var class=”var”>rate</var> is not explicitly specified. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the (approximate) output frame rate. This is done by setting the option <var class=”var”>n</var>. Default value is "25". </dd> <dt><samp class=”option”>split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class=”option”>draw</samp></dt> <dd> Set the draw mode. This is mostly useful to set for high <var class=”var”>n</var>. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> </dl>

 <h4 class="subsection">39.29.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output the input file audio and the corresponding video representation at the same time: <pre class="example-preformatted">amovie=a.mp3,asplit[out0],showwaves[out1]
</pre>

Create a synthetic signal and show it with showwaves, forcing a frame rate of 30 frames per second: <pre class="example-preformatted">aevalsrc=sin(1*2*PI*t)*sin(880*2*PI*t):cos(2*PI*200*t),asplit[out0],showwaves=r=30[out1]
</pre>

 <h3 class="section">39.30 showwavespic# TOC</h3>

Convert input audio to a single video frame, representing the samples waves. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">600x240</code>. </dd> <dt><samp class="option">split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class="option">colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class="table"> <dt>‘<samp class="samp">lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class="option">draw</samp></dt> <dd> Set the draw mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd>

<dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> <dt><samp class="option">filter</samp></dt> <dd> Set the filter mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">average</samp>’</dt> <dd> Use average samples values for each drawn sample. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Use peak samples values for each drawn sample. </dd> </dl>

Default value is <code class="code">average</code>. </dd> </dl>

 <h4 class="subsection">39.30.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract a channel split representation of the wave form of a whole audio track in a 1024x800 picture using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i audio.flac -lavfi showwavespic=split_channels=1:s=1024x800 waveform.png </pre>

 <h3 class="section">39.31 sidedata, asidedata# TOC</h3

Delete frame side data, or select frames based on it. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">select</samp>’</dt> <dd> Select every frame with side data of <code class="code">type</code>. </dd> <dt>‘<samp class="samp">delete</samp>’</dt> <dd> Delete side data of <code class="code">type</code>. If <code class="code">type</code> is not set, delete all side data in the frame. </dd> </dl>

</dd> <dt><samp class="option">type</samp></dt> <dd> Set side data type used with all modes. Must be set for <code class="code">select</code> mode. Possible values are: <dl class="table"> <dt>‘<samp class="samp">PANSCAN</samp>’</dt> <dt>‘<samp class="samp">A53_CC</samp>’</dt> <dt>‘<samp class="samp">STEREO3D</samp>’</dt> <dt>‘<samp class="samp">MATRIXENCODING</samp>’</dt> <dt>‘<samp class="samp">DOWNMIX_INFO</samp>’</dt> <dt>‘<samp

class=”samp”>REPLAYGAIN</samp>’</dt> <dt>‘<samp
class=”samp”>DISPLAYMATRIX</samp>’</dt> <dt>‘<samp
class=”samp”>AFD</samp>’</dt> <dt>‘<samp class=”samp”>MOTION_VECTORS</sa
<dt>‘<samp class=”samp”>SKIP_SAMPLES</samp>’</dt>
<dt>‘<samp class=”samp”>AUDIO_SERVICE_TYPE</samp>’</dt>
<dt>‘<samp class=”samp”>MASTERING_DISPLAY_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>GOP_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>SPHERICAL</samp>’</dt>
<dt>‘<samp class=”samp”>CONTENT_LIGHT_LEVEL</samp>’</dt>
<dt>‘<samp class=”samp”>ICC_PROFILE</samp>’</dt>
<dt>‘<samp class=”samp”>S12M_TIMECODE</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_PLUS</samp>’</dt>
<dt>‘<samp class=”samp”>REGIONS_OF_INTEREST</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_ENC_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>SEI_UNREGISTERED</samp>’</dt>
<dt>‘<samp class=”samp”>FILM_GRAIN_PARAMS</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BOUNDING_BOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DETECTION_BBOXES</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_RPU_BUFFER</samp>’</dt>
<dt>‘<samp class=”samp”>DOVI_METADATA</samp>’</dt>
<dt>‘<samp class=”samp”>DYNAMIC_HDR_VIVID</samp>’</dt>
<dt>‘<samp class=”samp”>AMBIENT_VIEWING_ENVIRONMENT</samp>’</dt>
<dt>‘<samp class=”samp”>VIDEO_HINT</samp>’</dt>
<dt>‘<samp class=”samp”>GAIN_MAP_PARAMS</samp>’</dt>
</dl>

</dd> </dl>

 <h3 class=”section”>39.32 spectrumsynth# TOC</h3>

Synthesize audio from 2 input video spectrums, first input stream represents magnitude across time and second represents phase across time. The filter will transform from frequency domain as displayed in videos back to time domain as presented in audio output.
This filter is primarily created for reversing processed showspectrum filter outputs, but can synthesize sound from other spectrograms too. But in such case results are going to be poor if the phase data is not available, because in such cases phase data need to be recreated, usually it’s just recreated from random noise. For best results use gray only output (<code class=”code”>channel</code> color mode in showspectrum filter) and <code class=”code”>log</code> scale for magnitude video and <code class=”code”>lin</code> scale for phase video. To produce phase, for 2nd video, use <code class=”code”>data</code> option. Inputs

videos should generally use <code class=”code”>fullframe</code> slide mode as that saves resources needed for decoding video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Specify sample rate of output audio, the sample rate of audio from which spectrum was generated may differ. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set number of channels represented in input video spectrums. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set scale which was used when generating magnitude input spectrum. Can be <code class=”code”>lin</code> or <code class=”code”>log</code>. Default is <code class=”code”>log</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide which was used when generating inputs spectrums. Can be <code class=”code”>replace</code>, <code class=”code”>scroll</code>, <code class=”code”>fullframe</code> or <code class=”code”>rscroll</code>. Default is <code class=”code”>fullframe</code>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function used for resynthesis. </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set window overlap. In range <code class=”code”>[0, 1]</code>. Default is <code class=”code”>1</code>, which means optimal overlap for selected window function will be picked. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of input videos. Can be <code class=”code”>vertical</code> or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>. </dd> </dl>

 <h4 class=”subsection”>39.32.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> First create magnitude and phase videos from audio, assuming audio is stereo with 44100 sample rate, then resynthesize videos back to audio with spectrumsynth: <pre class=”example-preformatted”>ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=log:overlap=0.875:color=channel:slide=fullframe:data=magnitude -an -c:v rawvideo magnitude.nut ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=lin:overlap=0.875:color=channel:slide=fullframe:data=phase -an -c:v rawvideo phase.nut ffmpeg -i magnitude.nut -i phase.nut -lavfi spectrumsynth=channels=2:sample_rate=44100:win_func=hann:overlap=0.875:slide=fullframe output.flac </pre>

 <h3 class=”section”>39.33 split, asplit# TOC</h3>

Split input into several identical outputs. <code class=”code”>asplit</code>

works with audio input, <code class="code">split</code> with video.
The filter accepts a single parameter which specifies the number of outputs. If unspecified, it defaults to 2.
<h4 class="subsection">39.33.1 Examples#
TOC</h4>

<ul class="itemize mark-bullet"> Create two separate outputs from the same input: <pre class="example-preformatted">[in] split [out0][out1] </pre>

To create 3 or more outputs, you need to specify the number of outputs, like in: <pre class="example-preformatted">[in] asplit=3 [out0][out1][out2] </pre>

Create two separate outputs from the same input, one cropped and one padded: <pre class="example-preformatted">[in] split [splitout1][splitout2]; [splitout1] crop=100:100:0:0 [cropout]; [splitout2] pad=200:200:100:100 [padout]; </pre>

Create 5 copies of the input audio with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i INPUT -filter_complex asplit=5 OUTPUT </pre>

 <h3 class="section">39.34 zmq, azmq# TOC</h3>

Receive commands sent through a libzmq client, and forward them to filters in the filtergraph. <code class="code">zmq</code> and <code class="code">azmq</code> work as a pass-through filters. <code class="code">zmq</code> must be inserted between two video filters, <code class="code">azmq</code> between two audio filters. Both are capable to send messages to any filter type. To enable these filters you need to install the libzmq library and headers and configure FFmpeg with <code class="code">--enable-libzmq</code>. For more information about libzmq see: http://www.zeromq.org/
The <code class="code">zmq</code> and <code class="code">azmq</code> filters work as a libzmq server, which receives messages sent through a network interface defined by the <samp class="option">bind_address</samp> (or the abbreviation "<samp class="option">b</samp>") option. Default value of this option is <samp class="file">tcp://localhost:5555</samp>. You may want to alter this value to your needs, but do not forget to escape any ’:’ signs (see filtergraph escaping). The received message must be in the form: <pre class="example-preformatted"><var class="var">TARGET</var> <var class="var">COMMAND</var>

[<var class="var">ARG</var>] </pre>

<var class="var">TARGET</var> specifies the target of the command, usually the name of the filter class or a specific filter instance name. The default filter instance name uses the pattern ‘<samp class="samp">Parsed_<filter_name>_<index></samp>’, but you can override this by using the ‘<samp class="samp">filter_name@id</samp>’ syntax (see Filtergraph syntax). <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional argument list for the given <var class="var">COMMAND</var>. Upon reception, the message is processed and the corresponding command is injected into the filtergraph. Depending on the result, the filter will send a reply to the client, adopting the format: <pre class="example-preformatted"><var class="var">ERROR_CODE</var> <var class="var">ERROR_REASON</var> <var class="var">MESSAGE</var> </pre>

<var class="var">MESSAGE</var> is optional. <h4 class="subsection">39.34.1 Examples# TOC</h4>

Look at <samp class="file">tools/zmqsend</samp> for an example of a zmq client which can be used to send commands processed by these filters. Consider the following filtergraph generated by <code class="command">ffplay</code>. In this example the last overlay filter has an instance name. All other filters will have default instance names.   <pre class="example-preformatted">ffplay -dumpgraph 1 -f lavfi " color=s=100x100:c=red [l]; color=s=100x100:c=blue [r]; nullsrc=s=200x100, zmq [bg]; [bg][l] overlay [bg+l]; [bg+l][r] overlay@my=x=100 " </pre>

To change the color of the left side of the video, the following command can be used: <pre class="example-preformatted">echo Parsed_color_0 c yellow | tools/zmqsend </pre>

To change the right side: <pre class="example-preformatted">echo Parsed_color_1 c pink | tools/zmqsend </pre>

To change the position of the right side: <pre class="example-preformatted">echo overlay@my x 150 | tools/zmqsend </pre>

 <h2 class="chapter">40 Multimedia Sources# TOC</h2>

Below is a description of the currently available multimedia sources. <h3 class="section">40.1 amovie# TOC</h3>

This is the same as movie source, except it selects an audio stream by default. <h3 class="section">40.2 avsynctest# TOC</h3> Generate an Audio/Video Sync Test. Generated stream periodically shows flash video frame and emits beep in audio. Useful to inspect A/V sync issues. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set output video size. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">framerate, fr</samp></dt> <dd> Set output video frame rate. Default value is <code class="code">30</code>. </dd> <dt><samp class="option">samplerate, sr</samp></dt> <dd> Set output audio sample rate. Default value is <code class="code">44100</code>. </dd> <dt><samp class="option">amplitude, a</samp></dt> <dd> Set output audio beep amplitude. Default value is <code class="code">0.7</code>. </dd> <dt><samp class="option">period, p</samp></dt> <dd> Set output audio beep period in seconds. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">delay, dl</samp></dt> <dd> Set output video flash delay in number of frames. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">cycle, c</samp></dt> <dd> Enable cycling of video delays, by default is disabled. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set stream output duration. By default duration is unlimited. </dd> <dt><samp class="option">fg, bg, ag</samp></dt> <dd> Set foreground/background/additional color. </dd> </dl>

 <h4 class="subsection">40.2.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">40.3 movie# TOC</h3>

Read audio and/or video stream(s) from a movie container. It accepts the following parameters: <dl class="table"> <dt><samp

class=”option”>filename</samp></dt> <dd> The name of the resource to read (not necessarily a file; it can also be a device or a stream accessed through some protocol). </dd> <dt><samp class=”option”>format_name, f</samp></dt> <dd> Specifies the format assumed for the movie to read, and can be either the name of a container or an input device. If not specified, the format is guessed from <var class=”var”>movie_name</var> or by probing. </dd> <dt><samp class=”option”>seek_point, sp</samp></dt> <dd> Specifies the seek point in seconds. The frames will be output starting from this seek point. The parameter is evaluated with <code class=”code”>av_strtod</code>, so the numerical value may be suffixed by an IS postfix. The default value is "0". </dd> <dt><samp class=”option”>streams, s</samp></dt> <dd> Specifies the streams to read. Several streams can be specified, separated by "+". The source will then have as many outputs, in the same order. The syntax is explained in the <a data-manual=”ffmpeg” href=”./ffmpeg.html#Stream-specifiers”>"Stream specifiers" section in the ffmpeg manual. Two special names, "dv" and "da" specify respectively the default (best suited) video and audio stream. Default is "dv", or "da" if the filter is called as "amovie". </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Specifies the index of the video stream to read. If the value is -1, the most suitable video stream will be automatically selected. The default value is "-1". Deprecated. If the filter is called "amovie", it will select audio instead of video. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> Specifies how many times to read the stream in sequence. If the value is 0, the stream will be looped infinitely. Default value is "1". Note that when the movie is looped the source timestamps are not changed, so it will generate non monotonically increasing timestamps. </dd> <dt><samp class=”option”>discontinuity</samp></dt> <dd> Specifies the time difference between frames above which the point is considered a timestamp discontinuity which is removed by adjusting the later timestamps. </dd> <dt><samp class=”option”>dec_threads</samp></dt> <dd> Specifies the number of threads for decoding </dd> <dt><samp class=”option”>format_opts</samp></dt> <dd> Specify format options for the opened file. Format options can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to add protocol_whitelist and protocol_blacklist options: <pre class=”example-preformatted”>ffplay -f lavfi "movie=filename='1.sdp':format_opts='protocol_whitelist=file,rtp,udp\:protocol_blacklist=http'&quo
</pre> </dd> </dl>

It allows overlaying a second video on top of the main input of a filtergraph, as shown in this graph: <pre class=”example-preformatted”>input ----------> deltapts0 --> overlay

--> output ^ | movie --> scale--> deltapts1 -------+ </pre> <h4 class="subsection">40.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Skip 3.2 seconds from the start of the AVI file in.avi, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=in.avi:seek_point=3.2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read from a video4linux2 device, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=/dev/video0:f=video4linux2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read the first video stream and the audio stream with id 0x81 from dvd.vob; the video is connected to the pad named "video" and the audio is connected to the pad named "audio": <pre class="example-preformatted">movie=dvd.vob:s=v:0+#0x81 [video] [audio] </pre>

 <h4 class="subsection">40.3.2 Commands# TOC</h4>

Both movie and amovie support the following commands: <dl class="table"> <dt><samp class="option">seek</samp></dt> <dd> Perform seek using "av_seek_frame". The syntax is: seek <var class="var">stream_index</var>|<var class="var">timestamp</var>|<var class="var">flags</var> <ul class="itemize mark-bullet"> <var class="var">stream_index</var>: If stream_index is -1, a default stream is selected, and <var class="var">timestamp</var> is automatically converted from AV_TIME_BASE units to the stream specific time_base. <var class="var">timestamp</var>: Timestamp in AVStream.time_base units or, if no stream is specified, in AV_TIME_BASE units. <var class="var">flags</var>: Flags which select direction and seeking mode.

</dd> <dt><samp class="option">get_duration</samp></dt> <dd> Get movie duration in AV_TIME_BASE units. </dd> </dl>

 <h2 class="chapter">41 External libraries# TOC</h2>

FFmpeg can be hooked up with a number of external libraries to add support for more formats. None of them are used by default, their use has to be explicitly requested by passing the appropriate flags to <code class=”command”>./configure</code>. <h3 class=”section”>41.1 Alliance for Open Media (AOM)# TOC</h3>

FFmpeg can make use of the AOM library for AV1 decoding and encoding. Go to http://aomedia.org/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libaom</code> to configure to enable it. <h3 class=”section”>41.2 ARM ASTC Encoder# TOC</h3>

FFmpeg can make use of the Arm ASTC Encoder library for ASTC image compression and decompression in the <code class=”code”>libastcenc</code> encoder and decoder. Go to https://github.com/ARM-software/astc-encoder to obtain the sources. Version 5.4 or newer is required, because the wrappers use the four-argument <code class=”code”>astcenc_context_alloc()</code> interface introduced in that release. The upstream build does not install the public header, and the library file name depends on the configured SIMD and linkage variants, so install the pieces manually in a layout that configure can detect: the header as <code class=”code”>astcenc/astcenc.h</code> and a library that can be linked with <code class=”code”>-lastcenc</code>. Keep the library’s own file name, which is the SONAME recorded in the binary, and provide the plain name as an alias: <pre class=”example-preformatted”>cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DASTCENC_SHAREDLIB=ON cmake --build build install -Dm644 Source/astcenc.h /usr/local/include/astcenc/astcenc.h install -Dm755 build/Source/libastcenc-native-shared.so \ /usr/local/lib/libastcenc-native-shared.so ln -sf libastcenc-native-shared.so /usr/local/lib/libastcenc.so </pre>

Use the platform’s shared library suffix, and adjust the built library name for other variants (for example <samp class=”file”>libastcenc-neon-shared.so</samp> or a static <samp class=”file”>libastcenc.a</samp>), or pass <code class=”code”>--extra-cflags</code> and <code class=”code”>--extra-ldflags</code> to configure if the files are installed elsewhere. Installing only under the plain name breaks linking at run time, and the installation directory has to be in the runtime loader’s search

path. On big-endian systems, also pass <code class="code">-DASTCENC_ISA_NONE=ON</code> and <code class="code">-DASTCENC_BIG_ENDIAN=ON</code> to CMake. For this variant, install <samp class="file">libastcenc-none-shared.so</samp> under its own name and provide <samp class="file">libastcenc.so</samp> as an alias, as in the example above. The library is licensed under Apache-2.0, which requires version 3 of the (L)GPL, so pass <code class="code">--enable-version3</code> and <code class="code">--enable-libastcenc</code> to configure to enable it. <h3 class="section">41.3 AMD AMF/VCE# TOC</h3>

FFmpeg can use the AMD Advanced Media Framework library for accelerated H.264 and HEVC(only windows) encoding on hardware with Video Coding Engine (VCE). To enable support you must obtain the AMF framework header files(version 1.4.9+) from https://github.com/GPUOpen-LibrariesAndSDKs/AMF.git. Create an <code class="code">AMF/</code> directory in the system include path. Copy the contents of <code class="code">AMF/amf/public/include/</code> into that directory. Then configure FFmpeg with <code class="code">--enable-amf</code>. Initialization of amf encoder occurs in this order: 1) trying to initialize through dx11(only windows) 2) trying to initialize through dx9(only windows) 3) trying to initialize through vulkan To use h.264(AMD VCE) encoder on linux amdgru-pro version 19.20+ and amf-amdgpu-pro package(amdgru-pro contains, but does not install automatically) are required. This driver can be installed using amdgpu-pro-install script in official amd driver archive. <h3 class="section">41.4 AviSynth# TOC</h3>

FFmpeg can read AviSynth scripts as input. To enable support, pass <code class="code">--enable-avisynth</code> to configure after installing the headers provided by AviSynth+. AviSynth+ can be configured to install only the headers by either passing <code class="code">-DHEADERS_ONLY:bool=on</code> to the normal CMake-based build system, or by using the supplied <code class="code">GNUmakefile</code>. For Windows, supported AviSynth variants are AviSynth 2.6 RC1 or higher for 32-bit builds and Avi
r1718 or higher for 32-bit and 64-bit builds. For Linux, macOS, and BSD, the only supported AviSynth variant is AviSynth+, start-

ing with version 3.5.   In 2016, AviSynth+ added support for building with GCC. However, due to the eccentricities of Windows’ calling conventions, 32-bit GCC builds of AviSynth+ are not compatible with typical 32-bit builds of FFmpeg. By default, FFmpeg assumes compatibility with 32-bit MSVC builds of AviSynth+ since that is the most widely-used and entrenched build configuration. Users can override this and enable support for 32-bit GCC builds of AviSynth+ by passing <code class=”code”>-DAVSC_WIN32_GCC32</code> to <code class=”code”>--extra-cflags</code> when configuring FFmpeg. 64-bit builds of FFmpeg are not affected, and can use either MSVC or GCC builds of AviSynth+ without any special flags.   AviSynth(+) is loaded dynamically. Distributors can build FFmpeg with <code class=”code”>--enable-avisynth</code>, and the binaries will work regardless of the end user having AviSynth installed. If/when an end user would like to use AviSynth scripts, then they can install AviSynth(+) and FFmpeg will be able to find and use it to open scripts. <h3 class=”section”>41.5 Chromaprint# TOC</h3>

FFmpeg can make use of the Chromaprint library for generating audio fingerprints. Pass <code class=”code”>--enable-chromaprint</code> to configure to enable it. See https://acoustid.org/chromapri
  <h3 class=”section”>41.6 codec2# TOC</h3>

FFmpeg can make use of the codec2 library for codec2 decoding and encoding. There is currently no native decoder, so libcodec2 must be used for decoding. Go to http://freedv.org/, download "Codec 2 source archive". Build and install using CMake. Debian users can install the libcodec2-dev package instead. Once libcodec2 is installed you can pass <code class=”code”>--enable-libcodec2</code> to configure to enable it. The easiest way to use codec2 is with .c2 files, since they contain the mode information required for decoding. To encode such a file, use a .c2 file extension and give the libcodec2 encoder the -mode option: <code class=”code”>ffmpeg -i input.wav -mode 700C output.c2</code>. Playback is as simple as <code class=”code”>ffplay output.c2</code>. For a list of supported modes, run <code class=”code”>ffmpeg -h encoder=libcodec2</code>. Raw codec2 files are also supported. To make sense of them the mode in use needs to be specified as a format option: <code class=”code”>ffmpeg -f codec2raw -mode 1300 -i input.raw output.wav</code>. <h3 class=”section”>41.7 dav1d# <a class=”anchor hidden-xs”href=”#toc-dav1d”

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the dav1d library for AV1 video decoding.
Go to https://code.videolan.org/videol
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libdav1d</code> to configure to enable it.
 <h3 class=”section”>41.8 davs2<span class=”pull-
right”>#
TOC</h3>

FFmpeg can make use of the davs2 library for AVS2-P2/IEEE1857.4 video
decoding. Go to https://github.com/pku
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libdavs2</code> to configure to enable it. <div
class=”info”> libdavs2 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>41.9
uavs3d<a class=”anchor hidden-xs” href=”#uavs3d”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-uavs3d”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the uavs3d library for AVS3-P2/IEEE1857.10
video decoding. Go to https://github.co
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libuavs3d</code> to configure to enable it. <a
name=”Game-Music-Emu”> <h3 class=”section”>41.10 Game Music
Emu<a class=”anchor hidden-xs” href=”#Game-
Music-Emu” aria-hidden=”true”># <a class=”anchor hidden-
xs”href=”#toc-Game-Music-Emu” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the Game Music Emu library to read audio
from supported video game music file formats. Pass <code class=”code”>-
-enable-libgme</code> to configure to enable it. See <a class=”url”
href=”https://bitbucket.org/mpyne/game-music-emu/overview”>https://bitbucket.org/mpyne/game-
music-emu/overview.
<h3 class=”section”>41.11 Intel QuickSync Video<span class=”pull-
right”><a class=”anchor hidden-xs” href=”#Intel-QuickSync-Video”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Intel-
QuickSync-Video” aria-hidden=”true”>TOC</h3>

FFmpeg can use Intel QuickSync Video (QSV) for accelerated decoding
and encoding of multiple codecs. To use QSV, FFmpeg must be linked
against the <code class=”code”>libmfx</code> dispatcher, which loads
the actual decoding libraries. The dispatcher is open source
and can be downloaded from <a class=”url” href=”https://github.com/lu-
zero/mfx_dispatch.git”>https://github.com/lu-zero/mfx_dispatch.git.

FFmpeg needs to be configured with the <code class=”code”>--enable-libmfx</code> option and <code class=”code”>pkg-config</code> needs to be able to locate the dispatcher’s <code class=”code”>.pc</code> files. <h3 class=”section”>41.12 Kvazaar# TOC</h3>

FFmpeg can make use of the Kvazaar library for HEVC encoding. Go to <a class=”url” href=”https://github.com/ultravideo/kvazaar”>https://github.com/ultravideo/kvaz and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libkvazaar</code> to configure to enable it. <h3 class=”section”>41.13 LAME# TOC</h3>

FFmpeg can make use of the LAME library for MP3 encoding. Go to http://lame.sourceforge.net/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libmp3lame</code> to configure to enable it. <h3 class=”section”>41.14 LCEVCdec# TOC</h3>

FFmpeg can make use of the liblcevc_dec library for LCEVC enhancement layer decoding on supported bitstreams. Go to https://github.com/v-novaltd/LCEVCdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblcevc-dec</code> to configure to enable it.   LCEVCdec is under the BSD-3-Clause-Clear License. <h3 class=”section”>41.15 libilbc# TOC</h3>

iLBC is a narrowband speech codec that has been made freely available by Google as part of the WebRTC project. libilbc is a packaging friendly copy of the iLBC codec. FFmpeg can make use of the libilbc library for iLBC decoding and encoding. Go to https://github.com/TimothyGu/libilbc and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libilbc</code> to configure to enable it. <h3 class=”section”>41.16 libjxl# TOC</h3>

JPEG XL is an image format intended to fully replace legacy JPEG for an extended period of life. See https://jpegxl.info/ for more information, and see <a class="url" href="https://github.com/libjxl/libjxl">https://github.com/libj for the library source. You can pass <code class="code">--enable-libjxl</code> to configure in order enable the libjxl wrapper. <h3 class="section">41.17 libvpx# TOC</h3>

FFmpeg can make use of the libvpx library for VP8/VP9 decoding and encoding. Go to http://www.webmproject.org and follow the instructions for installing the library. Then pass <code class="code">--enable-libvpx</code> to configure to enable it. <h3 class="section">41.18 ModPlug# TOC</h3>

FFmpeg can make use of this library, originating in Modplug-XMMS, to read from MOD-like music files. See https:/ Pass <code class="code">--enable-libmodplug</code> to configure to enable it. <h3 class="section">41.19 ONNX Runtime# TOC</h3>

FFmpeg can make use of the ONNX Runtime library as a backend for DNN based filters. Go to https://onnxruntime.ai/ and follow the instructions for installing the library. Pre-built packages are available for various platforms. Configure FFmpeg with ONNX Runtime support: <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib" </pre>

On ELF-based systems (Linux, BSD) you may also want to embed the library search path so that the installed <samp class="file">ffmpeg</samp> binary finds the shared library at run time without setting <code class="env">LD_LIBRARY_PATH</code>:   <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib -Wl,-rpath,/path/to/onnxruntime/lib" </pre>

The <code class="code">-Wl,-rpath</code> flag is ELF-specific and should be omitted on other platforms (Windows, macOS). For MinGW GCC builds, add the <code class="code">-D_stdcall=___stdcall</code> flag for compatibility with ONNX Runtime headers. <a name="OpenCORE_002c-

VisualOn_002c-and-Fraunhofer-libraries”> <h3 class=”section”>41.20 OpenCORE, VisualOn, and Fraunhofer libraries# TOC</h3>

Spun off Google Android sources, OpenCore, VisualOn and Fraunhofer libraries provide encoders for a number of audio codecs.   OpenCORE and VisualOn libraries are under the Apache License 2.0 (see http://www.apache.org/licenses/LICENSE-2.0 for details), which is incompatible to the LGPL version 2.1 and GPL version 2. You have to upgrade FFmpeg’s license to LGPL version 3 (or if you have enabled GPL components, GPL version 3) by passing <code class=”code”>--enable-version3</code> to configure in order to use it. The license of the Fraunhofer AAC library is incompatible with the GPL. Therefore, for GPL builds, you have to pass <code class=”code”>--enable-nonfree</code> to configure in order to use it. To the best of our knowledge, it is compatible with the LGPL. <h4 class=”subsection”>41.20.1 OpenCORE AMR# TOC</h4>

FFmpeg can make use of the OpenCORE libraries for AMR-NB decoding/encoding and AMR-WB decoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the libraries. Then pass <code class=”code”>--enable-libopencore-amrnb</code> and/or <code class=”code”>--enable-libopencore-amrwb</code> to configure to enable them. <h4 class=”subsection”>41.20.2 VisualOn AMR-WB encoder library# TOC</h4>

FFmpeg can make use of the VisualOn AMR-WBenc library for AMR-WB encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libvo-amrwbenc</code> to configure to enable it. <h4 class=”subsection”>41.20.3 Fraunhofer AAC library# <a class=”anchor hidden-xs”href=”#toc-Fraunhofer-AAC-library” aria-

hidden=”true”>TOC</h4>

FFmpeg can make use of the Fraunhofer AAC library for AAC decoding & encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-aac</code> to configure to enable it. <h4 class=”subsection”>41.20.4 Fraunhofer xHE-AAC library# TOC</h4>

FFmpeg can make use of the Fraunhofer xHE-AAC library for xHE-AAC encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-xhe-aac</code> to configure to enable it. <h4 class=”subsection”>41.20.5 Fraunhofer MPEG-H 3D Audio decoder library# TOC</h4>

FFmpeg can make use of the Fraunhofer MPEG-H decoder library for MPEG-H 3DA decoding. Go to https://github.com/Fraunhofer-IIS/mpeghdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libmpeghdec --enable-nonfree</code> to configure to enable it. <h4 class=”subsection”>41.20.6 LC3 library# TOC</h4>

FFmpeg can make use of the Google LC3 library for LC3 decoding & encoding. Go to <a class=”url” href=”https://github.com/google/liblc3/”>https://github.com/goo and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblc3</code> to configure to enable it. <h3 class=”section”>41.21 OpenH264# TOC</h3>

FFmpeg can make use of the OpenH264 library for H.264 decoding and encoding. Go to http://www.openh264.org/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libopenh264</code> to configure to enable it.

For decoding, this library is much more limited than the built-in decoder in libavcodec; currently, this library lacks support for decoding B-frames and some other main/high profile features. (It currently only supports constrained baseline profile and CABAC.) Using it is mostly useful for testing and for taking advantage of Cisco’s patent portfolio license (http://www.openh264.org/BINARY_LICENSE.
  <h3 class="section">41.22 OpenJPEG# TOC</h3>

FFmpeg can use the OpenJPEG libraries for decoding/encoding J2K videos. Go to http://www.openjpeg.org/ to get the libraries and follow the installation instructions. To enable using OpenJPEG in FFmpeg, pass <code class="code">--enable-libopenjpeg</code> to <samp class="file">./configure</samp>. <h3 class="section">41.23 rav1e# TOC</h3>

FFmpeg can make use of rav1e (Rust AV1 Encoder) via its C bindings to encode videos. Go to https://github.com/xiph/rav1e/<
and follow the instructions to build the C library. To enable using rav1e in FFmpeg, pass <code class="code">--enable-librav1e</code> to <samp class="file">./configure</samp>. <h3 class="section">41.24 SVT-AV1# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for AV1 library for AV1 encoding. Go to https://gitlab.com/AOMediaCodec/SVT-AV1/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtav1</code> to configure to enable it. <h3 class="section">41.25 SVT-JPEG-XS# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for JPEG-XS library for JPEG-XS decoding and encoding. Go to https://github.com/OpenVisualCloud/SVT-JPEG-XS and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtjpegxs</code> to configure to enable it. <h3 class="section">41.26 TwoLAME# <a class="anchor hidden-xs"href="#toc-TwoLAME"

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the TwoLAME library for MP2 encoding.
Go to http://www.twolame.org/
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libtwolame</code> to configure to enable it. <a
name=”VapourSynth”> <h3 class=”section”>41.27 VapourSynth<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#VapourSynth”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
VapourSynth” aria-hidden=”true”>TOC</h3>

FFmpeg can read VapourSynth scripts as input. To enable support, pass
<code class=”code”>--enable-vapoursynth</code> to configure. Vapoursynth
is detected via <code class=”code”>pkg-config</code>. Versions 42 or greater
supported. See http://www.vapoursynth.com/.
 Due to security concerns, Vapoursynth scripts will not be
autodetected so the input format has to be forced. For ff* CLI tools,
add <code class=”code”>-f vapoursynth</code> before the input <code
class=”code”>-i yourscript.vpy</code>.
<h3 class=”section”>41.28 x264<a class=”anchor
hidden-xs” href=”#x264” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-x264” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the x264 library for H.264 encoding.
Go to http://www.videolan.org/d
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libx264</code> to configure to enable it.
 x264 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>41.29
x265<a class=”anchor hidden-xs” href=”#x265”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-x265”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the x265 library for HEVC encoding.
Go to http://x265.org/developers.html
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libx265</code> to configure to enable it.
 x265 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>41.30
xavs<a class=”anchor hidden-xs” href=”#xavs”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-xavs”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the xavs library for AVS encoding. Go to http://xavs.sf.net/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libxavs</code> to configure to enable it. <h3 class="section">41.31 xavs2# TOC</h3>

FFmpeg can make use of the xavs2 library for AVS2-P2/IEEE1857.4 video encoding. Go to <a class="url" href="https://github.com/pkuvcl/xavs2">https://github.com/pku and follow the instructions for installing the library. Then pass <code class="code">--enable-libxavs2</code> to configure to enable it.   libxavs2 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class="section">41.32 eXtra-fast Essential Video Encoder (XEVE)# TOC</h3>

FFmpeg can make use of the XEVE library for EVC video encoding. Go to https://github.com/mpeg5/xeve and follow the instructions for installing the XEVE library. Then pass <code class="code">--enable-libxeve</code> to configure to enable it. <h3 class="section">41.33 eXtra-fast Essential Video Decoder (XEVD)# TOC</h3>

FFmpeg can make use of the XEVD library for EVC video decoding. Go to https://github.com/mpeg5/xevd and follow the instructions for installing the XEVD library. Then pass <code class="code">--enable-libxevd</code> to configure to enable it. <h3 class="section">41.34 ZVBI# TOC</h3>

ZVBI is a VBI decoding library which can be used by FFmpeg to decode DVB teletext pages and DVB teletext subtitles. Go to http://sourceforge.net/projects/zapping/ and follow the instructions for installing the library. Then pass <code

class=”code”>--enable-libzvbi</code> to configure to enable it. <h2 class=”chapter”>42 Supported File Formats, Codecs or Features# TOC</h2>

You can use the <code class=”code”>-formats</code> and <code class=”code”>-codecs</code> options to have an exhaustive list. <h3 class=”section”>42.1 File Formats# TOC</h3>

FFmpeg supports the following file formats through the <code class=”code”>libavformat</code>

library:

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>3dostr</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>4xm</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>4X Technologies format, used in some games.</td></tr>

<tr><td width=”40%”>8088flex TMV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Enhanced Audio format, used in audiobooks.</td></tr>

<tr><td width=”40%”>AA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Format 2, 3, and 4, used in audiobooks.</td></tr>

<tr><td width=”40%”>ACT Voice</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>contains G.729 audio</td></tr>

<tr><td width=”40%”>Adobe Filmstrip</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Audio IFF (AIFF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>American Laser Games MM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Multimedia format used in games

like Mad Dog McCree.</td></tr>

<tr><td width="40%">3GPP AMR</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Amazing Studio Packed Animation File</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in game Heart Of Darkness.</td></tr>

<tr><td width="40%">Apple HTTP Live Streaming</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Artworx Data Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only format used in some Interplay games.</td></tr>

<tr><td width="40%">ADP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">AFC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">ADS/SS2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS2.</td></tr>

<tr><td width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">.astc</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Raw ASTC texture container.</td></tr>

<tr><td width="40%">ASF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Advanced / Active Streaming Format.</td></tr>

<tr><td width="40%">AST</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">AviSynth</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AVR</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on Mac.</td></tr>

<tr><td width="40%">AVS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by the Creature Shock game.</td></tr>

<tr><td width=”40%”>Beam Software SIFF</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio and video format used in some games by Beam Software.</td></tr>

<tr><td width=”40%”>Bethesda Softworks VID</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some games from Bethesda Softworks.</td></tr>

<tr><td width=”40%”>Binary text</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Bink</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Multimedia format used by many games.</td></tr>

<tr><td width=”40%”>Bink Audio</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio only multimedia format used by some games.</td></tr>

<tr><td width=”40%”>Bitmap Brothers JV</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Z and Z95 games.</td></tr>

<tr><td width=”40%”>BRP</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Argonaut Games format.</td></tr>

<tr><td width=”40%”>Brute Force & Ignorance</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width=”40%”>BFSTM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used on the Nintendo WiiU (based on BRSTM).</td></tr>

<tr><td width=”40%”>BRSTM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used on the Nintendo Wii.</td></tr>

<tr><td width=”40%”>BW64</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Broadcast Wave 64bit.</td></tr>

<tr><td width=”40%”>BWF</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>codec2 (raw)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Must be given -mode format option to decode correctly.</td></tr>

<tr><td width=”40%”>codec2 (.c2 files)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Contains header with version and mode info, simplifying playback.</td></tr>

<tr><td width=”40%”>CRI ADX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Audio-only format used in console video games.</td></tr>

<tr><td width="40%">CRI AIX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">Discworld II BMV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Delphine Software International CIN</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by Delphine Software games.</td></tr>

<tr><td width="40%">Digital Speech Standard (DSS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video format used by CD+G karaoke disks</td></tr>

<tr><td width="40%">Phantom Cine</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Commodore CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video format</td></tr>

<tr><td width="40%">Core Audio Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Apple Core Audio Format</td></tr>

<tr><td width="40%">CRC testing format</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">Creative Voice</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Created for the Sound Blaster Pro.</td></tr>

<tr><td width="40%">CRYO APC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in some games by CRYO Interactive Entertainment.</td></tr>

<tr><td width="40%">D-Cinema audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DCSTR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>DFA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in Chronomaster game</td></tr>

<tr><td width=”40%”>DirectDraw Surface</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DSD Stream File (DSF)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DV video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DXA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in the non-Windows version of the Feeble Files

game and different game cutscenes repacked for use with ScummVM.</td></tr>

<tr><td width=”40%”>Electronic Arts cdata</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Electronic Arts Multimedia</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA games; files have extensions like WVE and UV2.</td></tr>

<tr><td width=”40%”>Ensoniq Paris Audio File</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>FFM (FFserver live feed)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash (SWF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash 9 (AVM2)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Only embedded audio is decoded.</td></tr>

<tr><td width=”40%”>FLI/FLC/FLX animation</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>.fli/.flc files</td></tr>

<tr><td width=”40%”>Flash Video (FLV)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Macromedia Flash video files</td></tr>

<tr><td width=”40%”>framecrc testing format</td><td width=”10%”>X</td><td width=”10%”></td></tr>

<tr><td width=”40%”>FunCom ISS</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used in various games from FunCom like The Longest Journey.</td></tr>

<tr><td width=”40%”>G.723.1</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width="40%">G.726</td><td width="10%"></td><td width="10%">X</td><td width="40%">Both left- and right-justified.</td></tr>

<tr><td width="40%">G.729 BIT</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.729 raw</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GENH</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format for various games.</td></tr>

<tr><td width="40%">GIF Animation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">General eXchange Format SMPTE 360M, used by Thomson Grass Valley

playout servers.</td></tr>

<tr><td width="40%">HNM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Only version 4 supported, used in some games from Cryo Interactive</td></tr>

<tr><td width="40%">iCEDraw File</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ICO</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft Windows ICO</td></tr>

<tr><td width="40%">id Quake II CIN video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">IEC61937 encapsulation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Interchange File Format</td></tr>

<tr><td width="40%">IFV</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format used by some old CCTV DVRs.</td></tr>

<tr><td width="40%">iLBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay MVE</td><td width="10%"></td><td width="10%">X</td><td width="40%">Format used in various Interplay computer games.</td></tr>

<tr><td width="40%">Iterated Systems ClearVideo</td><td width="10%"></td><td width="10%">X</td><td width="40%">I-frames only</td></tr>

<tr><td width="40%">IV8</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format generated by IndigoVision 8000 video server.</td></tr>

<tr><td width="40%">IVF (On2)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">A format used by libvpx</td></tr>

<tr><td width="40%">Internet Video Recording</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">IRCAM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">JPEG Multi-Picture Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Includes JPEG Ultra HDR (ISO 21496-1 gain map) images.</td></tr>

<tr><td width="40%">LAF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Limitless Audio Format</td></tr>

<tr><td width="40%">LATM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LMLM4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Linux Media Labs MPEG-4 PCI boards</td></tr>

<tr><td width="40%">LOAS</td><td width="10%"></td><td width="10%">X</td><td width="40%">contains LATM multiplexed AAC audio</td></tr>

<tr><td width="40%">LRC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LVF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LXF</td><td width="10%"></td><td width="10%">X</td><td width="40%">VR native stream format, used by Leitch/Harris’ video servers.</td></tr>

<tr><td width="40%">Magic Lantern Video (MLV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska audio</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">FFmpeg metadata</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Metadata in text format.</td></tr>

<tr><td width=”40%”>MAXIS XA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Sim City 3000; file extension .xa.</td></tr>

<tr><td width=”40%”>MCA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some games from Capcom; file extension .mca.</td></tr>

<tr><td width=”40%”>MD Studio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Metal Gear Solid: The Twin Snakes</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Megalux Frame</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used by Megalux Ultimate Paint</td></tr>

<tr><td width=”40%”>MobiClip MODS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MobiClip MOFLEX</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Mobotix .mxg</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Monkey’s Audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Motion Pixels MVI</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MOV/QuickTime/MP4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>3GP, 3GP2, PSP, iPod variants supported</td></tr>

<tr><td width=”40%”>MP2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MP3</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-1 System</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>muxed audio and video, VCD format supported</td></tr>

<tr><td width=”40%”>MPEG-PS (program stream)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>also known as <code class=”code”>VOB</code> file, SVCD and DVD format supported</td></tr>

<tr><td width=”40%”>MPEG-TS (transport stream)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>also known as DVB Transport Stream</td></tr>

<tr><td width="40%">MPEG-4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">MPEG-4 is a variant of QuickTime.</td></tr>

<tr><td width="40%">MSF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">Mirillis FIC video</td><td width="10%"></td><td width="10%">X</td><td width="40%">No cursor rendering.</td></tr>

<tr><td width="40%">MIDI Sample Dump Standard</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MIME multipart JPEG</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">MSN TCP webcam</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by MSN Messenger webcam streams.</td></tr>

<tr><td width="40%">MTV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Material eXchange Format (MXF)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 377M, used by D-Cinema, broadcast industry.</td></tr>

<tr><td width="40%">Material eXchange Format (MXF), D-10 Mapping</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 386M, D-10/IMX Mapping.</td></tr>

<tr><td width="40%">NC camera feed</td><td width="10%"></td><td width="10%">X</td><td width="40%">NC (AVIP NC4600) camera streams</td></tr>

<tr><td width="40%">NIST SPeech HEader REsources</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Computerized Speech Lab NSP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NTT TwinVQ (VQF)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Nippon Telegraph and Telephone Corporation TwinVQ.</td></tr>

<tr><td width="40%">Nullsoft Streaming Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NUT</td><td width="10%">X</td><td width="10%">X</td><td width="40%">NUT Open Container Format</td></tr>

<tr><td width="40%">Ogg</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Playstation Portable PMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Portable Voice Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">TechnoTrend PVA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by TechnoTrend DVB PCI boards.</td></tr>

<tr><td width="40%">QCP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw ADTS (AAC)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-NB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-WB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APAC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Bonk</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw Chinese AVS video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Dirac</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DNxHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS-HD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw E-AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw EVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw FLAC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw GSM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.263</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.264</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw HEVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Ingenient MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw MLP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw NULL</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw id RoQ</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw OBU</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw OSQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw SBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TAK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TrueHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw VC-1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 16.8 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 24.0 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RDT</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">REDCODE R3D</td><td width="10%"></td><td width="10%">X</td><td width="40%">File format used by RED Digital cameras, contains JPEG 2000 frames and PCM audio.</td></tr>

<tr><td width="40%">RealMedia</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Redirector</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RedSpark</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TeXture Dictionary</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Resolume DXV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoding is only supported for the DXT1 (Normal Quality, No Alpha) texture format.</td></tr>

<tr><td width="40%">RF64</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RL2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Entertainment Software Partners.</td></tr>

<tr><td width="40%">RPL/ARMovie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Lego Mindstorms RSO</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RSD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RTMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Output is performed by publishing stream to RTMP server</td></tr>

<tr><td width="40%">RTP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RTSP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sample Dump eXchange</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SAP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SBG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDNS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SER</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Pictures SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sega FILM/CPK</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in many Sega Saturn console games.</td></tr>

<tr><td width="40%">Silicon Graphics Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra SOL</td><td width="10%"></td><td width="10%">X</td><td width="40%">.sol files used in Sierra Online games.</td></tr>

<tr><td width="40%">Sierra VMD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra CD-ROM games.</td></tr>

<tr><td width="40%">Smacker</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">SMJPEG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in certain Loki game ports.</td></tr>

<tr><td width="40%">SMPTE 337M encapsulation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in some LucasArts games.</td></tr>

<tr><td width="40%">Sony OpenMG (OMA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used in Sony Sonic Stage and Sony Vegas.</td></tr>

<tr><td width="40%">Sony PlayStation STR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sony Wave64 (W64)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SoX native format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUN AU format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUP raw PGS subtitles</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Konami PS2 games.</td></tr>

<tr><td width="40%">TDSC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Text files</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">THP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used on the Nintendo GameCube.</td></tr>

<tr><td width="40%">Tiertex Limited SEQ</td><td width="10%"></td><td width="10%">X</td><td width="40%">Tiertex .seq files used in the DOS CD-ROM version of the game Flashback.</td></tr>

<tr><td width="40%">True Audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in many Sony PS2 games.</td></tr>

<tr><td width="40%">VC-1 test bitstream</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vidvox Hap</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vivo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VPK</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Sony PS games.</td></tr>

<tr><td width="40%">Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">WebM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Televison (WTV)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Wing Commander III movie</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Origin’s Wing Commander III computer game.</td></tr>

<tr><td width="40%">Westwood Studios audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Westwood Studios VQA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Wideband Single-bit Data (WSD)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WVE</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">KTX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Khronos Texture 1.0 container for ASTC.</td></tr>

<tr><td width="40%">Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">XMV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft video container used in Xbox games.</td></tr>

<tr><td width="40%">XVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">xWMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft audio container used by XAudio 2.</td></tr>

<tr><td width="40%">eXtended BINary text (XBIN)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">YUV4MPEG pipe</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP</td><td width="10%"></td><td width="10%">X</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <h3 class=”section”>42.2 Image Formats# TOC</h3>

FFmpeg can read and write images for each frame of a video sequence. The following image formats are supported: <table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr> <tr><td width=”40%”>.Y.U.V</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>one raw file per component</td></tr> <tr><td width=”40%”>Alias PIX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Alias/Wavefront PIX image format</td></tr> <tr><td width=”40%”>animated GIF</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>APNG</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Animated Portable Network Graphics</td></tr> <tr><td width=”40%”>ASTC</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Adaptive Scalable Texture Compression, through external library libastcenc</td></tr> <tr><td width=”40%”>BMP</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Microsoft BMP image</td></tr> <tr><td width=”40%”>BRender PIX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Argonaut BRender 3D engine image format.</td></tr> <tr><td width=”40%”>CRI</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Cintel RAW</td></tr> <tr><td width=”40%”>DPX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Digital Picture Exchange</td></tr> <tr><td width=”40%”>EXR</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>OpenEXR</td></tr> <tr><td width=”40%”>FITS</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Flexible Image Transport System</td></tr> <tr><td width=”40%”>HDR</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Radiance HDR RGBE Image format</td></tr> <tr><td width=”40%”>IMG</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>GEM Raster image</td></tr> <tr><td width=”40%”>JPEG</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Progressive JPEG is not supported.</td></tr> <tr><td width=”40%”>JPEG 2000</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JPEG-LS</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JPEG-XS</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Supported through external library libsvtjpegxs</td></tr> <tr><td width=”40%”>LJPEG</td><td width=”10%”>X</td><td width=”10%”></td><td width=”40%”>Lossless JPEG</td></tr> <tr><td width=”40%”>Media 100</td><td width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>MSP</td><td

width="10%"></td><td width="10%">X</td><td width="40%">Microsoft Paint image</td></tr> <tr><td width="40%">PAM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PAM is a PNM extension with alpha support.</td></tr> <tr><td width="40%">PBM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable BitMap image</td></tr> <tr><td width="40%">PCD</td><td width="10%"></td><td width="10%">X</td><td width="40%">PhotoCD</td></tr> <tr><td width="40%">PCX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PC Paintbrush</td></tr> <tr><td width="40%">PFM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable FloatMap image</td></tr> <tr><td width="40%">PGM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable GrayMap image</td></tr> <tr><td width="40%">PGMYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PGM with U and V components in YUV 4:2:0</td></tr> <tr><td width="40%">PGX</td><td width="10%"></td><td width="10%">X</td><td width="40%">PGX file decoder</td></tr> <tr><td width="40%">PHM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable HalfFloatMap image</td></tr> <tr><td width="40%">PIC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Pictor/PC Paint</td></tr> <tr><td width="40%">PNG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable Network Graphics image</td></tr> <tr><td width="40%">PPM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable PixelMap image</td></tr> <tr><td width="40%">PSD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Photoshop</td></tr> <tr><td width="40%">PTX</td><td width="10%"></td><td width="10%">X</td><td width="40%">V.Flash PTX format</td></tr> <tr><td width="40%">QOI</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Quite OK Image format</td></tr> <tr><td width="40%">RW2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Panasonic raw image, RawFormat 8 only</td></tr> <tr><td width="40%">SGI</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SGI RGB image format</td></tr> <tr><td width="40%">Sun Rasterfile</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sun RAS image format</td></tr> <tr><td width="40%">TIFF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">YUV, JPEG and some extension is not supported yet.</td></tr> <tr><td width="40%">Truevision Targa</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Targa (.TGA) image format</td></tr> <tr><td width="40%">VBN</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Vizrt Binary Image format</td></tr> <tr><td width="40%">WBMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Wireless Application Protocol Bitmap image format</td></tr> <tr><td width="40%">WebP</td><td width="10%">E</td><td width="10%">X</td><td width="40%">WebP

image format, encoding supported through external library libwebp</td></tr>
<tr><td width="40%">XBM</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X BitMap image format</td></tr>
<tr><td width="40%">XFace</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X-Face image format</td></tr>
<tr><td width="40%">XPM</td><td width="10%"></td><td width="10%">X</td><td
width="40%">X PixMap image format</td></tr> <tr><td width="40%">XWD</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">X
Window Dump image format</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code>
means that support is provided through an external library. <a
name="Video-Codecs"> <h3 class="section">42.3 Video Codecs<span
class="pull-right"><a class="anchor hidden-xs" href="#Video-Codecs"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-Video-
Codecs" aria-hidden="true">TOC</h3>

<table class="multitable">

<tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td
width="10%">Decoding</td><td width="40%">Comments</td></tr>

<tr><td width="40%">4X Movie</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in certain computer
games.</td></tr>

<tr><td width="40%">8088flex TMV</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">A64 multicolor</td><td width="10%">X</td><td
width="10%"></td><td width="40%">Creates video suitable to be played
on a commodore 64 (multicolor mode).</td></tr>

<tr><td width="40%">Amazing Studio PAF Video</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">American Laser Games MM</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in games like Mad Dog Mc-
Cree.</td></tr>

<tr><td width="40%">Amuse Graphics Movie</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">AMV Video</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Used in Chinese MP3 play-
ers.</td></tr>

<tr><td width="40%">ANSI/ASCII art</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">Apple Intermediate Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple MJPEG-B</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple Pixlet</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple ProRes</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: apch,apcn,apcs,apco,ap4h,ap4x</td></tr>

<tr><td width="40%">Apple QuickDraw</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: qdrw</td></tr>

<tr><td width="40%">APV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Argonaut Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Argonaut games.</td></tr>

<tr><td width="40%">Asus v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ASV1</td></tr>

<tr><td width="40%">Asus v2</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ASV2</td></tr>

<tr><td width="40%">ATI VCR1</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR1</td></tr>

<tr><td width="40%">ATI VCR2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR2</td></tr>

<tr><td width="40%">Auravision Aura</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Auravision Aura 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk Animator Flic video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk RLE</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: AASC</td></tr>

<tr><td width="40%">AV1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libaom, libdav1d, librav1e and libsvtav1</td></tr>

<tr><td width="40%">Avid 1:1 10-bit RGB Packer</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: AVrp</td></tr>

<tr><td width="40%">AVS (Audio Video Standard) video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used by the Creature Shock game.</td></tr>

<tr><td width="40%">AVS2-P2/IEEE1857.4</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libxavs2 and libdavs2</td></tr>

<tr><td width="40%">AVS3-P2/IEEE1857.10</td><td width="10%"></td><td width="10%">E</td><td width="40%">Supported through external library libuavs3d</td></tr>

<tr><td width="40%">AYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft uncompressed packed 4:4:4:4</td></tr>

<tr><td width="40%">Beam Software VB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bethesda VID video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Bink Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">BitJazz SheerVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bitmap Brothers JV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">y41p Brooktree uncompressed 4:1:1 12-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Brooktree ProSumer Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: BT20</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">C93 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Cyberia game.</td></tr>

<tr><td width="40%">CamStudio</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: CSCD</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video codec for CD+G karaoke disks</td></tr>

<tr><td width="40%">CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video codec</td></tr>

<tr><td width="40%">Chinese AVS video</td><td width="10%">E</td><td width="10%">X</td><td width="40%">AVS1-P2, JiZhun profile, encoding

through external library libxavs</td></tr>

<tr><td width="40%">Delphine Software International CIN video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">Discworld II BMV Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CineForm HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CDToons</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in various Broderbund games.</td></tr>

<tr><td width="40%">Cinepak</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Cirrus Logic AccuPak</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: CLJR</td></tr>

<tr><td width="40%">CPiA Video Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Creative YUV (CYUV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DFA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Chronomaster game.</td></tr>

<tr><td width="40%">Dirac</td><td width="10%">E</td><td width="10%">X</td><td width="40%">supported though the native vc2 (Dirac Pro) encoder</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DNxHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">aka SMPTE VC3</td></tr>

<tr><td width="40%">Duck TrueMotion 1.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: DUCK</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TM20</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0 RT</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TR20</td></tr>

<tr><td width="40%">DV (Digital Video)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Dxtory capture format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Feeble Files/ScummVM DXA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec originally used in Feeble Files game.</td></tr>

<tr><td width="40%">Electronic Arts CMV video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in NHL 95 game.</td></tr>

<tr><td width="40%">Electronic Arts Madcow video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGQ video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TQI video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 124</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 130</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">EVC / MPEG-5 Part 1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">encoding and decoding supported through external libraries libxeve and libxevd</td></tr>

<tr><td width="40%">FFmpeg video codec #1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">lossless codec (fourcc: FFV1)</td></tr>

<tr><td width="40%">Flash Screen Video v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: FSV1</td></tr>

<tr><td width="40%">Flash Screen Video v2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash Video (FLV)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sorenson H.263 used in Flash</td></tr>

<tr><td width="40%">FM Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Forward Uncompressed</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Fraps</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Go2Meeting</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M2, G2M3</td></tr>

<tr><td width="40%">Go2Webinar</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M4</td></tr>

<tr><td width="40%">Gremlin Digital Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263 / H.263-1996</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263+ / H.263-1998 / H.263 version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libx264 and OpenH264,

decoding supports data partitioning</td></tr>

<tr><td width="40%">HEVC</td><td width="10%">X</td><td width="10%">X</td><td width="40%">encoding supported through external library libx265 and libkvazaar</td></tr>

<tr><td width="40%">HNM version 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV FFmpeg variant</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IBM Ultimotion</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ULTI</td></tr>

<tr><td width="40%">id Cinematic video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Quake II.</td></tr>

<tr><td width="40%">id RoQ video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2, other computer games.</td></tr>

<tr><td width="40%">IFF ILBM</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF interleaved bitmap</td></tr>

<tr><td width="40%">IFF ByteRun1</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF run length encoded bitmap</td></tr>

<tr><td width="40%">Infinity IMM4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM6</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel H.263</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Interplay MVE video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Interplay .MVE files.</td></tr>

<tr><td width="40%">J2K</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Karl Morton’s video codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Worms games.</td></tr>

<tr><td width="40%">Kega Game Video (KGV1)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Kega emulator screen capture codec.</td></tr>

<tr><td width="40%">Lagarith</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCEVC / MPEG-5 LCEVC / MPEG-5 Part 2</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library liblcevc-dec</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) MSZH</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) ZLIB</td><td width="10%">E</td><td width="10%">E</td></tr>

<tr><td width="40%">LEAD MCMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LOCO</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LucasArts SANM/Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts games / SMUSH animations.</td></tr>

<tr><td width="40%">lossless MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MagicYUV Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mandsoft Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft ATC Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Screen 3.</td></tr>

<tr><td width="40%">Microsoft Expression Encoder Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Titanium Screen 2.</td></tr>

<tr><td width="40%">Microsoft RLE</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft Screen 1</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V7 Screen.</td></tr>

<tr><td width="40%">Microsoft Screen 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V9 Screen.</td></tr>

<tr><td width="40%">Microsoft Video 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mimic</td><td width="10%"></td><td

width=”10%”>X</td><td width=”40%”>Used in MSN Messenger Webcam streams.</td></tr>

<tr><td width=”40%”>Miro VideoXL</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VIXL</td></tr>

<tr><td width=”40%”>MJPEG (Motion JPEG)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Mobotix MxPEG video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Motion Pixels video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-1 video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-2 video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>libxvidcore can be used alternatively for encoding.</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 3</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Newtek SpeedHQ</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Nintendo Gamecube THP video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>NotchLC</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>NuppelVideo/RTjpeg</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Video encoding used in NuppelVideo files.</td></tr>

<tr><td width=”40%”>On2 VP3</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>still experimental</td></tr>

<tr><td width=”40%”>On2 VP4</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP40</td></tr>

<tr><td width=”40%”>On2 VP5</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP50</td></tr>

<tr><td width=”40%”>On2 VP6</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP60,VP61,VP62</td></tr>

<tr><td width=”40%”>On2 VP7</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP70,VP71</td></tr>

<tr><td width=”40%”>VP8</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>fourcc: VP80, encoding supported through external library libvpx</td></tr>

<tr><td width=”40%”>VP9</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>encoding supported through external library libvpx</td></tr>

<tr><td width=”40%”>Pinnacle TARGA CineWave YUV16</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: Y216</td></tr>

<tr><td width=”40%”>Q-team QPEG</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourccs: QPEG, Q1.0, Q1.1</td></tr>

<tr><td width=”40%”>QuickTime 8BPS video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>QuickTime Animation (RLE) video</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ’rle ’</td></tr>

<tr><td width=”40%”>QuickTime Graphics (SMC)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ’smc ’</td></tr>

<tr><td width=”40%”>QuickTime video (RPZA)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: rpza</td></tr>

<tr><td width=”40%”>R10K AJA Kona 10-bit RGB Codec</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>R210 Quicktime Uncompressed RGB 10-bit</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Raw Video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 1.0</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 2.0</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 3.0</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>still far from ideal</td></tr>

<tr><td width="40%">RealVideo 4.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 6.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TXD (TeXture Dictionary)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Texture dictionaries used by the Renderware Engine.</td></tr>

<tr><td width="40%">RivaTuner Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’RTV1’</td></tr>

<tr><td width="40%">RL2 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">used in some games by Entertainment Software Partners</td></tr>

<tr><td width="40%">ScreenPressor</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screenpresso</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screen Recorder Gold Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 1 (MVC1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 2 (MVC2)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics RLE 8-bit video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smacker video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in Smacker.</td></tr>

<tr><td width="40%">SMPTE VC-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Snow</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental wavelet codec (fourcc: SNOW)</td></tr>

<tr><td width="40%">Sony PlayStation MDEC (Motion DECoder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: SVQ1</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 3</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SVQ3</td></tr>

<tr><td width="40%">Sunplus JPEG (SP5X)</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SP5X</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSCC</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSC2</td></tr>

<tr><td width="40%">Theora</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libtheora</td></tr>

<tr><td width="40%">Tiertex Limited SEQ video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in DOS CD-ROM FlashBack game.</td></tr>

<tr><td width="40%">Ut Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v210 QuickTime uncompressed 4:2:2 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v308 QuickTime uncompressed 4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v408 QuickTime uncompressed 4:4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v410 QuickTime uncompressed 4:4:4 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VBLE Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">vMix Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’VMX1’</td></tr>

<tr><td width="40%">VMware Screen Codec / VMware Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in videos captured by VMware.</td></tr>

<tr><td width="40%">Westwood Studios VQA (Vector Quantized Animation) video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Image</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>Windows Media Video 7</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Video 8</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Video 9</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>not completely working</td></tr>

<tr><td width=”40%”>Wing Commander III / Xan</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Wing Commander III .MVE files.</td></tr>

<tr><td width=”40%”>Wing Commander IV / Xan</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Wing Commander IV.</td></tr>

<tr><td width=”40%”>Winnov WNV1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>WMV7</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>YAMAHA SMAF</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Psygnosis YOP Video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>yuv4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>libquicktime uncompressed packed 4:2:0</td></tr>

<tr><td width=”40%”>ZeroCodec Lossless Video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ZLIB</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>part of LCL, encoder experimental</td></tr>

<tr><td width=”40%”>Zip Motion Blocks Video</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Encoder works only in PAL8.</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <code class=”code”>E</code> means that support is provided through an external library. <h3 class=”section”>42.4 Audio Codecs<a class=”anchor hidden-xs” href=”#Audio-Codecs”

aria-hidden="true"># TOC</h3>

<table class="multitable">

<tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td width="10%">Decoding</td><td width="40%">Comments</td></tr>

<tr><td width="40%">8SVX exponential</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">8SVX fibonacci</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AAC</td><td width="10%">EX</td><td width="10%">X</td><td width="40%">encoding supported through internal encoder and external library libfdk-aac</td></tr>

<tr><td width="40%">AAC+</td><td width="10%">E</td><td width="10%">IX</td><td width="40%">encoding supported through external library libfdk-aac</td></tr>

<tr><td width="40%">AC-3</td><td width="10%">IX</td><td width="10%">IX</td></tr>

<tr><td width="40%">ACELP.KELVIN</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM 4X Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Yamaha AICA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM AmuseGraphics Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Argonaut Games</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM CDROM XA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Creative Technology</td><td width="10%"></td><td width="10%">X</td><td width="40%">16 -> 4, 8 -> 4, 8 -> 3, 8 -> 2</td></tr>

<tr><td width="40%">ADPCM Electronic Arts</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various EA titles.</td></tr>

<tr><td width="40%">ADPCM Electronic Arts Maxis CDROM XS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sim City 3000.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R3</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts XAS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.722</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.726</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Acorn Replay</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA AMV</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in AMV files</td></tr>

<tr><td width=”40%”>ADPCM IMA Citrix</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Cunning Developments</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Electronic Arts EACS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Electronic Arts SEAD</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Funcom</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA High Voltage Software ALP</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Mobiclip MOFLEX</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA QuickTime</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Simon & Schuster Interactive</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Ubisoft APM</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width="40%">ADPCM IMA Loki SDL MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Westwood</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM ISS IMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in FunCom games.</td></tr>

<tr><td width="40%">ADPCM IMA Dialogic</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK3</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Radical</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Xbox</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Microsoft</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM MS IMA</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube AFC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube DTK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo THP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Playstation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM QT IMA</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Rhetorex</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sanyo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM SEGA CRI ADX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Sega Dreamcast games.</td></tr>

<tr><td width="40%">ADPCM Shockwave Flash</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 2-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 2.6-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 4-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ADPCM Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Westwood Studios IMA</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Westwood Studios games like Command and Conquer.</td></tr>

<tr><td width="40%">ADPCM Yamaha</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Zork</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AMR-NB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopencore-amrnb</td></tr>

<tr><td width="40%">AMR-WB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libvo-amrwbenc</td></tr>

<tr><td width="40%">Amazing Studio PAF Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple lossless audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">QuickTime fourcc ’alac’</td></tr>

<tr><td width="40%">aptX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">aptX HD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">ATRAC1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3+</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC9</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Bink and Smacker files in many games.</td></tr>

<tr><td width="40%">Bonk audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">codec2</td><td width="10%">E</td><td width="10%">E</td><td width="40%">en/decoding supported through external library libcodec2</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Delphine Software International CIN audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Speech Standard - Standard Play mode (DSS SP)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Discworld II BMV Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">COOK</td><td width="10%"></td><td width="10%">X</td><td width="40%">All versions except 5.1 are supported.</td></tr>

<tr><td width="40%">DCA (DTS Coherent Acoustics)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">supported extensions: XCh, XXCH, X96, XBR, XLL, LBR (partially)</td></tr>

<tr><td width="40%">Dolby E</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Cuberoot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in few games.</td></tr>

<tr><td width="40%">DPCM Gremlin</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">DPCM Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Interplay</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various Interplay computer games.</td></tr>

<tr><td width="40%">DPCM Squareroot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various games.</td></tr>

<tr><td width="40%">DPCM Sierra Online</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra Online game audio files.</td></tr>

<tr><td width="40%">DPCM Sol</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Origin’s Wing Commander IV AVI files.</td></tr>

<tr><td width="40%">DPCM Xilam DERF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSP Group TrueSpeech</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DST (Direct Stream Transfer)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DV audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Enhanced AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">EVRC (Enhanced Variable Rate Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">FLAC (Free Lossless Audio Codec)</td><td width="10%">X</td><td width="10%">IX</td></tr>

<tr><td width="40%">FTR Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.729</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GSM</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">GSM Microsoft variant</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">IAC (Indeo Audio Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">iLBC (Internet Low Bitrate Codec)</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">encoding and decoding supported through external library libilbc</td></tr>

<tr><td width="40%">IMC (Intel Music Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LC3</td><td width="10%">E</td><td width="10%">E</td><td width="40%">supported through external library liblc3</td></tr>

<tr><td width="40%">MACE (Macintosh Audio Compression/Expansion) 6:1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Marian’s A-pac audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MI-SC4 (Micronas SC-4 Audio)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MLP (Meridian Lossless Packing)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in DVD-Audio discs.</td></tr>

<tr><td width="40%">Monkey’s Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MP1 (MPEG audio layer 1)</td><td width="10%"></td><td width="10%">IX</td></tr>

<tr><td width="40%">MP2 (MPEG audio layer 2)</td><td width="10%">IX</td><td width="10%">IX</td><td width="40%">encoding supported also through external library TwoLAME</td></tr>

<tr><td width="40%">MP3 (MPEG audio layer 3)</td><td width="10%">E</td><td width="10%">IX</td><td width="40%">encoding supported through external library LAME, ADU MP3 and MP3onMP4 also supported</td></tr>

<tr><td width="40%">MPEG-4 Audio Lossless Coding (ALS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-H 3D Audio</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library libmpeghdec</td></tr>

<tr><td width="40%">MobiClip FastAudio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV7</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Nellymoser Asao</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">On2 AVC (Audio for Video Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Opus</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopus</td></tr>

<tr><td width="40%">OSQ (Original Sound Quality)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM D-Cinema audio signed 24-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian for DVD-Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian in MPEG-TS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QCELP / PureVoice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">There are still some distortions.</td></tr>

<tr><td width="40%">RealAudio 1.0 (14.4K)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Real 14400 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 2.0 (28.8K)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Real 28800 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 3.0 (dnet)</td><td width="10%">IX</td><td width="10%">X</td><td width="40%">Real low bitrate AC-3 codec</td></tr>

<tr><td width="40%">RealAudio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealAudio SIPR / ACELP.NET</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SBC (low-complexity subband codec)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Smacker audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SMPTE 302M AES3 audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sonic</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Sonic lossless</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Speex</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">supported through external library libspeex</td></tr>

<tr><td width="40%">TAK (Tom’s lossless Audio Kompressor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">True Audio (TTA)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">TrueHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in HD-DVD and Blu-Ray discs.</td></tr>

<tr><td width="40%">TwinVQ (VQF flavor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ViewQuest VQC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Vorbis</td><td width="10%">E</td><td width="10%">X</td><td width="40%">A native but very primitive encoder exists.</td></tr>

<tr><td width=”40%”>Voxware MetaSound</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Waveform Archiver</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>WavPack</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Westwood Audio (SND1)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio 1</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio 2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Lossless</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Pro</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Voice</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Xbox Media Audio 1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Xbox Media Audio 2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <code class=”code”>E</code> means that support is provided through an external library. <code class=”code”>I</code> means that an integer-only version is available, too (ensures high performance on systems without hardware floating point support). <h3 class=”section”>42.5 Subtitle Formats# TOC</h3>

<table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Muxing</td><td width=”10%”>Demuxing</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td></tr>
<tr><td width=”40%”>3GPP Timed Text</td><td width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>AQTitle</td><td width=”10%”></td><td

width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>DVB</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>DVB teletext</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>E</td></tr>
<tr><td width=”40%”>DVD</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>JACOsub</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MicroDVD</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MPL2</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MPsub (MPlayer)</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>PGS</td><td width=”10%”></td><td width=”10%”></td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>PJS
(Phoenix)</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RealText</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SAMI</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Spruce format
(STL)</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SSA/ASS</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubRip (SRT)</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer v1</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>TED Talks cap-
tions</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>TTML</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td></tr> <tr><td width=”40%”>VobSub (IDX+SUB)</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>VPlayer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>WebVTT</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>XSUB</td><td
width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature is supported. <code class="code">E</code> means that support is provided through an external library. <h3 class="section">42.6 Network Protocols# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Support</td></tr> <tr><td width="40%">AMQP</td><td width="10%">E</td></tr> <tr><td width="40%">file</td><td width="10%">X</td></tr> <tr><td width="40%">FTP</td><td width="10%">X</td></tr> <tr><td width="40%">Gopher</td><td width="10%">X</td></tr> <tr><td width="40%">Gophers</td><td width="10%">X</td></tr> <tr><td width="40%">HLS</td><td width="10%">X</td></tr> <tr><td width="40%">HTTP</td><td width="10%">X</td></tr> <tr><td width="40%">HTTPS</td><td width="10%">X</td></tr> <tr><td width="40%">Icecast</td><td width="10%">X</td></tr> <tr><td width="40%">MMSH</td><td width="10%">X</td></tr> <tr><td width="40%">MMST</td><td width="10%">X</td></tr> <tr><td width="40%">pipe</td><td width="10%">X</td></tr> <tr><td width="40%">Pro-MPEG FEC</td><td width="10%">X</td></tr> <tr><td width="40%">RTMP</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPE</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPS</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPT</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPTE</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPTS</td><td width="10%">X</td></tr> <tr><td width="40%">RTP</td><td width="10%">X</td></tr> <tr><td width="40%">SAMBA</td><td width="10%">E</td></tr> <tr><td width="40%">SCTP</td><td width="10%">X</td></tr> <tr><td width="40%">SFTP</td><td width="10%">E</td></tr> <tr><td width="40%">TCP</td><td width="10%">X</td></tr> <tr><td width="40%">TLS</td><td width="10%">X</td></tr> <tr><td width="40%">UDP</td><td width="10%">X</td></tr> <tr><td width="40%">ZMQ</td><td width="10%">E</td></tr> </tbody> </table>

<code class="code">X</code> means that the protocol is supported. <code class="code">E</code> means that support is provided through an external library.

 <h3 class="section">42.7 Input/Output Devices# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td
width="10%">Input</td><td width="10%">Output</td></tr> <tr><td
width="40%">ALSA</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">BKTR</td><td width="10%">X</td><td
width="10%"></td></tr> <tr><td width="40%">caca</td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">DV1394</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">Lavfi
virtual device</td><td width="10%">X</td><td width="10%"></td></tr>
<tr><td width="40%">Linux framebuffer</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">JACK</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">LIBCDIO</td><td
width="10%">X</td></tr> <tr><td width="40%">LIBDC1394</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">OpenAL</td><td
width="10%">X</td></tr> <tr><td width="40%">OSS</td><td
width="10%">X</td><td width="10%">X</td></tr> <tr><td
width="40%">PulseAudio</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">SDL</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">Video4Linux2</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">VfW capture</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">X11
grabbing</td><td width="10%">X</td><td width="10%"></td></tr>
<tr><td width="40%">Win32 grabbing</td><td width="10%">X</td><td
width="10%"></td></tr> </tbody> </table>

<code class="code">X</code> means that input/output is supported. <h3 class="section">42.8 Timecode# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Codec/format</td><td
width="10%">Read</td><td width="10%">Write</td></tr> <tr><td
width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">DV</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">GXF</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">MOV</td><td
width="10%">X</td><td width="10%">X</td></tr> <tr><td
width="40%">MPEG1/2</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">MXF</td><td width="10%">X</td><td
width="10%">X</td></tr> </tbody> </table>

 <h2 class="chapter">43 See Also# TOC</h2>

ffplay, ffmpeg, ffprobe, <a class="url" href="ffmpeg-

utils.html”>ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class=”chapter”>44 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class=”command”>git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class=”file”>MAINTAINERS</samp> in the source code tree.

<p style=”font-size: small;”> This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffplay.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset=”utf-8”> <title> ffplay Documentation </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”> <link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body> <h1> ffplay Documentation </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1 Synopsis 2 Description 3 Options <ul class=”toc-numbered-mark”> 3.1 Stream specifiers 3.2 Generic options 3.3

AVOptions 3.4 Main options 3.5 Advanced options 3.6 While playing 4 See Also 5 Authors

 <h2 class="chapter">1 Synopsis# TOC</h2>

ffplay [<var class="var">options</var>] [<samp class="file">input_url</samp>] <h2 class="chapter">2 Description# TOC</h2>

FFplay is a very simple and portable media player using the FFmpeg libraries and the SDL library. It is mostly used as a testbed for the various FFmpeg APIs. <h2 class="chapter">3 Options# TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or ’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’ as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class="example-preformatted">ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">3.1 Stream specifiers#
TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class="code">a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class=”code”>-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class=”code”>-codec copy</code>

or <code class=”code”>-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class=”code”>-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class=”var”>stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class=”var”>stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class=”option”><var class=”var”>stream_type</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> <var class=”var”>stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class=”var”>additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class="var">additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class="option">g:<var class="var">group_specifier</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class="var">group_specifier</var>.

if <var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class="var">additional_stream_specifier</var>.

<var class="var">group_specifier</var> may be one of the following:

<dl class="table">

<dt><samp class="option"><var class="var">group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class="var">additional_stream_specifier</var>.

</dd>

<dt><samp class="option">#<var class="var">stream_id</var> or i:<var class="var">stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class="option">m:<var class="var">key</var>[:<var class="var">value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class="var">key</var> having the specified value. If

<var class="var">value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class="var">key</var> or <var class="var">value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class="option">disp:<var class="var">dispositions</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class="var">dispositions</var> is a list of

one or more dispositions (as printed by the <samp class="option">-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class="option">u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class="command">ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">3.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show

license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options, including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed information about the demuxer named <var class="var">demuxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class="option">muxer=<var class="var">muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class="var">muxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class="option">filter=<var class="var">filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class="var">filter_name</var>. Use the <samp class="option">-filters</samp> option to get a list of all filters. </dd> <dt><samp class="option">bsf=<var class="var">bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class="var">bitstream_filter_name</var>. Use the <samp class="option">-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class="option">protocol=<var class="var">protocol_name</var></samp></dt> <dd> Print detailed information about the protocol named <var class="var">protocol_name</var>. Use the <samp class="option">-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class="option">-version</samp></dt> <dd> Show version. </dd> <dt><samp class="option">-buildconf</samp></dt> <dd> Show the build configuration, one option per line. </dd> <dt><samp class="option">-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class="option">-demuxers</samp></dt> <dd> Show available demuxers. </dd> <dt><samp class="option">-muxers</samp></dt> <dd> Show avail-

able muxers. </dd> <dt><samp class="option">-devices</samp></dt>
<dd> Show available devices. </dd> <dt><samp class="option">-
codecs</samp></dt> <dd> Show all codecs known to libavcodec.
 Note that the term ’codec’ is used throughout
this documentation as a shortcut for what is more correctly called a
media bitstream format. </dd> <dt><samp class="option">-
decoders</samp></dt> <dd> Show available decoders. </dd>
<dt><samp class="option">-encoders</samp></dt> <dd> Show
all available encoders. </dd> <dt><samp class="option">-
bsfs</samp></dt> <dd> Show available bitstream filters. </dd>
<dt><samp class="option">-protocols</samp></dt> <dd> Show avail-
able protocols. </dd> <dt><samp class="option">-filters</samp></dt>
<dd> Show available libavfilter filters. </dd> <dt><samp
class="option">-pix_fmts</samp></dt> <dd> Show available pixel for-
mats. </dd> <dt><samp class="option">-sample_fmts</samp></dt>
<dd> Show available sample formats. </dd> <dt><samp
class="option">-layouts</samp></dt> <dd> Show channel names
and standard channel layouts. </dd> <dt><samp class="option">-
dispositions</samp></dt> <dd> Show stream dispositions.
</dd> <dt><samp class="option">-colors</samp></dt> <dd> Show
recognized color names. </dd> <dt><samp class="option">-sources
<var class="var">device</var>[,<var class="var">opt1</var>=<var
class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt>
<dd> Show autodetected sources of the input device. Some devices may
provide system-dependent source names that cannot be autodetected.
The returned list cannot be assumed to be always complete. <div
class="example"> <pre class="example-preformatted">ffmpeg -sources
pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var
class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var
class="var">val2</var>]...]</samp></dt> <dd> Show autodetected
sinks of the output device. Some devices may provide system-dependent sink
names that cannot be autodetected. The returned list cannot be assumed
to be always complete. <pre class="example-
preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var
class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var
class="var">loglevel</var></samp></dt> <dd> Set logging level and
flags used by the library. The optional <var class="var">flags</var>
prefix can consist of the following values: <dl class="table">
<dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates
that repeated log output should not be compressed to the first line and
the "Last message repeated n times" line will be omitted.
</dd> <dt>‘<samp class="samp">level</samp>’</dt>
<dd> Indicates that log output should add a <code class="code">[level]</code>

prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var class="var">flags</var> or changing <var class="var">loglevel</var>. When setting both <var class="var">flags</var> and <var class="var">loglevel</var>, a ’+’ separator is expected between the last <var class="var">flags</var> value and before <var class="var">loglevel</var>. <var class="var">loglevel</var> is a string or a number containing one of the following values: <dl class="table"> <dt>‘<samp class="samp">quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class="samp">panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class="samp">fatal, 8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class="samp">error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class="samp">warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class="samp">info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class="samp">verbose, 40</samp>’</dt> <dd> Same as <code class="code">info</code>, except more verbose. </dd> <dt>‘<samp class="samp">debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class="samp">trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class="code">level</code> prefix, and set <var class="var">loglevel</var> to <code class="code">verbose</code>: <pre class="example-preformatted">ffmpeg -loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code class="code">level</code> prefix flag or <var class="var">loglevel</var>: <pre class="example-preformatted">ffmpeg [...] -loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class="env">AV_LOG_FORCE_NOCOLOR</code>,

or can be forced setting the environment variable <code class=”env”>AV_LOG_FORCE_COLOR</code>. </dd> <dt><samp class=”option”>-report</samp></dt> <dd> Dump full command line and log output to a file named <code class=”code”><var class=”var”>program</var>-<var class=”var”>YYYYMMDD</var>-<var class=”var”>HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class=”code”>-loglevel debug</code>. Setting the environment variable <code class=”env”>FFREPORT</code> to any value has the same effect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual). The following options are recognized: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> set the file name to use for the report; <code class=”code”>%p</code> is expanded to the name of the program, <code class=”code”>%t</code> is expanded to a timestamp, <code class=”code”>%%</code> is expanded to a plain <code class=”code”>%</code> </dd> <dt><samp class=”option”>level</samp></dt> <dd> set the log verbosity level using a numerical value (see <code class=”code”>-loglevel</code>). </dd> </dl>

For example, to output a report to a file named <samp class=”file”>ffreport.log</samp> using a log level of <code class=”code”>32</code> (alias for log level <code class=”code”>info</code>):   <pre class=”example-preformatted”>FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class=”option”>-hide_banner</samp></dt> <dd> Suppress printing banner. All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class=”option”>-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class=”table”> <dt>‘<samp class=”samp”>x86</samp>’</dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>mmx</samp>’</dt> <dt>‘<samp class=”samp”>mmxext</samp>’</dt> <dt>‘<samp class=”samp”>sse</samp>’</d <dt>‘<samp class=”samp”>sse2</samp>’</dt> <dt>‘<samp class=”samp”>sse2slow</samp>’</dt> <dt>‘<samp class=”samp”>sse3</samp>’</ <dt>‘<samp class=”samp”>sse3slow</samp>’</dt> <dt>‘<samp class=”samp”>ssse3</samp>’</dt> <dt>‘<samp class=”samp”>atom</samp>’</dt

<dt>‘<samp class="samp">sse4.1</samp>’</dt> <dt>‘<samp
class="samp">sse4.2</samp>’</dt> <dt>‘<samp class="samp">avx</samp>’</dt>
<dt>‘<samp class="samp">avx2</samp>’</dt> <dt>‘<samp
class="samp">xop</samp>’</dt> <dt>‘<samp class="samp">fma3</samp>’</dt>
<dt>‘<samp class="samp">fma4</samp>’</dt> <dt>‘<samp
class="samp">3dnow</samp>’</dt> <dt>‘<samp class="samp">3dnowext</samp>&rsquo
<dt>‘<samp class="samp">bmi1</samp>’</dt> <dt>‘<samp
class="samp">bmi2</samp>’</dt> <dt>‘<samp class="samp">cmov</samp>’</dt
</dl> </dd> <dt>‘<samp class="samp">ARM</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv5te</samp>’</dt>
<dt>‘<samp class="samp">armv6</samp>’</dt> <dt>‘<samp
class="samp">armv6t2</samp>’</dt> <dt>‘<samp class="samp">vfp</samp>’</d
<dt>‘<samp class="samp">vfpv3</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> <dt>‘<samp class="samp">setend</samp>’</d
</dl> </dd> <dt>‘<samp class="samp">AArch64</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv8</samp>’</dt>
<dt>‘<samp class="samp">vfp</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> </dl> </dd> <dt>‘<samp
class="samp">PowerPC</samp>’</dt> <dd><dl class="table">
<dt>‘<samp class="samp">altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class="samp">Specific Processors</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">pentium2</samp>’</dt>
<dt>‘<samp class="samp">pentium3</samp>’</dt>
<dt>‘<samp class="samp">pentium4</samp>’</dt>
<dt>‘<samp class="samp">k6</samp>’</dt> <dt>‘<samp
class="samp">k62</samp>’</dt> <dt>‘<samp class="samp">athlon</samp>’</dt
<dt>‘<samp class="samp">athlonxp</samp>’</dt> <dt>‘<samp
class="samp">k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class="option">-cpucount <var class="var">count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class="example-preformatted">ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class="option">-max_alloc <var class="var">bytes</var></samp></dt>
<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is INT_MAX. </dd> </dl>

 <h3 class="section">3.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and

libavcodec libraries. To see the list of available AVOptions, use the <samp class=”option”>-help</samp> option. They are separated into two categories: <dl class=”table”> <dt><samp class=”option”>generic</samp></dt> <dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class=”option”>private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class=”option”>id3v2_version</samp> private option of the MP3 muxer: <pre class=”example-preformatted”>ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class=”example-preformatted”>ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class=”option”>-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class=”option”>-option 0</samp>/<samp class=”option”>-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class=”section”>3.4 Main options# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-x <var class=”var”>width</var></samp></dt> <dd> Force displayed width. </dd> <dt><samp class=”option”>-y <var class=”var”>height</var></samp></dt> <dd> Force displayed height. </dd> <dt><samp class=”option”>-fs</samp></dt> <dd> Start in fullscreen mode. </dd> <dt><samp class=”option”>-an</samp></dt> <dd> Disable audio. </dd> <dt><samp class=”option”>-vn</samp></dt> <dd> Disable video. </dd> <dt><samp class=”option”>-sn</samp></dt> <dd> Disable subtitles. </dd> <dt><samp class=”option”>-ss <var class=”var”>pos</var></samp></dt> <dd> Seek to <var class=”var”>pos</var>. Note that in most formats it is not possible to seek exactly, so <code class=”command”>ffplay</code> will seek to the nearest seek point to <var class=”var”>pos</var>. <var class=”var”>pos</var> must be a time duration specification, see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-

syntax”>the Time duration section in the ffmpeg-utils(1) manual. </dd> <dt><samp class=”option”>-t <var class=”var”>duration</var></samp></dt> <dd> Play <var class=”var”>duration</var> seconds of audio/video. <var class=”var”>duration</var> must be a time duration specification, see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. </dd> <dt><samp class=”option”>-bytes</samp></dt> <dd> Seek by bytes. </dd> <dt><samp class=”option”>-seek_interval</samp></dt> <dd> Set custom interval, in seconds, for seeking using left/right keys. Default is 10 seconds. </dd> <dt><samp class=”option”>-nodisp</samp></dt> <dd> Disable graphical display. </dd> <dt><samp class=”option”>-noborder</samp></dt> <dd> Borderless window. </dd> <dt><samp class=”option”>-alwaysontop</samp></dt> <dd> Window always on top. Available on: X11 with SDL >= 2.0.5, Windows SDL >= 2.0.6. </dd> <dt><samp class=”option”>-volume</samp></dt> <dd> Set the startup volume. 0 means silence, 100 means no volume reduction or amplification. Negative values are treated as 0, values above 100 are treated as 100. </dd> <dt><samp class=”option”>-f <var class=”var”>fmt</var></samp></dt> <dd> Force format. </dd> <dt><samp class=”option”>-window_title <var class=”var”>title</var></samp></dt> <dd> Set window title (default is the input filename). </dd> <dt><samp class=”option”>-left <var class=”var”>title</var></samp></dt> <dd> Set the x position for the left of the window (default is a centered window). </dd> <dt><samp class=”option”>-top <var class=”var”>title</var></samp></dt> <dd> Set the y position for the top of the window (default is a centered window). </dd> <dt><samp class=”option”>-loop <var class=”var”>number</var></samp></dt> <dd> Loops movie playback <number> times. 0 means forever. </dd> <dt><samp class=”option”>-showmode <var class=”var”>mode</var></samp></dt> <dd> Set the show mode to use. Available values for <var class=”var”>mode</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>0, video</samp>’</dt> <dd> show video </dd> <dt>‘<samp class=”samp”>1, waves</samp>’</dt> <dd> show audio waves </dd> <dt>‘<samp class=”samp”>2, rdft</samp>’</dt> <dd> show audio frequency band using RDFT ((Inverse) Real Discrete Fourier Transform) </dd> <dt>‘<samp class=”samp”>3, phase</samp>’</dt> <dd> like rdft, with stereo content whose left channel leads in phase shown in blue and whose right channel leads shown in yellow </dd> </dl>

Default value is "video", if video is not present or cannot be played "rdft" is automatically selected. You can interactively cycle through the available show modes by pressing the key <kbd class=”key”>w</kbd>. </dd> <dt><samp class=”option”>-vf <var class=”var”>filtergraph</var></samp></dt> <dd> Create the

filtergraph specified by <var class="var">filtergraph</var> and use it to filter the video stream. <var class="var">filtergraph</var> is a description of the filtergraph to apply to the stream, and must have a single video input and a single video output. In the filtergraph, the input is associated to the label <code class="code">in</code>, and the output to the label <code class="code">out</code>. See the ffmpeg-filters manual for more information about the filtergraph syntax. You can specify this parameter multiple times and cycle through the specified filtergraphs along with the show modes by pressing the key <kbd class="key">w</kbd>. </dd> <dt><samp class="option">-af <var class="var">filtergraph</var></samp></dt> <dd> <var class="var">filtergraph</var> is a description of the filtergraph to apply to the input audio. Use the option "-filters" to show all the available filters (including sources and sinks). </dd> <dt><samp class="option">-i <var class="var">input_url</var></samp></dt> <dd> Read <var class="var">input_url</var>. </dd> </dl>

<h3 class="section">3.5 Advanced options# TOC</h3>

<dl class="table">

<dt><samp class="option">-stats</samp></dt>

<dd> Print several playback statistics, in particular show the stream

duration, the codec parameters, the current position in the stream and

the audio/video synchronisation drift. It is shown by default, unless the

log level is lower than <code class="code">info</code>. Its display can be forced by manually

specifying this option. To disable it, you need to specify <code class="code">-nostats</code>.

</dd>

<dt><samp class="option">-fast</samp></dt>

<dd> Non-spec-compliant optimizations.

</dd>

<dt><samp class="option">-genpts</samp></dt>

<dd> Generate pts.

</dd>

<dt><samp class="option">-sync <var class="var">type</var></samp></dt>

<dd> Set the master clock to audio (<code class="code">type=audio</code>), video

(<code class="code">type=video</code>) or external (<code class="code">type=ext</code>). Default is audio. The

master clock is used to control audio-video synchronization. Most media

players use audio as master clock, but in some cases (streaming or high

quality broadcast) it is necessary to change that. This option is mainly

used for debugging purposes.

</dd>

<dt><samp class="option">-ast <var class="var">audio_stream_specifier</var></samp></dt>

<dd> Select the desired audio stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" audio stream is selected in the program of the

already selected video stream.

</dd>

<dt><samp class="option">-vst <var class="var">video_stream_specifier</var></samp></dt>

<dd> Select the desired video stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" video stream is selected.

</dd>

<dt><samp class="option">-sst <var class="var">subtitle_stream_specifier</var></samp></dt>

<dd> Select the desired subtitle stream using the given stream specifier. The stream

specifiers are described in the Stream specifiers chapter. If this option

is not specified, the "best" subtitle stream is selected in the program of the

already selected video or audio stream.

</dd>

<dt><samp class="option">-autoexit</samp></dt>

<dd> Exit when video is done playing.

</dd>

<dt><samp class="option">-exitonkeydown</samp></dt>

<dd> Exit if any key is pressed.

</dd>

<dt><samp class="option">-exitonmousedown</samp></dt>

<dd> Exit if any mouse button is pressed.

</dd>

<dt><samp class="option">-codec:<var class="var">media_specifier</var>
<var class="var">codec_name</var></samp></dt>

<dd> Force a specific decoder implementation for the stream identified by

<var class="var">media_specifier</var>, which can assume the values <code class="code">a</code> (audio),

<code class="code">v</code> (video), and <code class="code">s</code> subtitle.

</dd>

<dt><samp class="option">-acodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific audio decoder.

</dd>

<dt><samp class="option">-vcodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific video decoder.

</dd>

<dt><samp class="option">-scodec <var class="var">codec_name</var></samp></dt>

<dd> Force a specific subtitle decoder.

</dd>

<dt><samp class="option">-autorotate</samp></dt>

<dd> Automatically rotate the video according to file metadata. Enabled by

default, use <samp class="option">-noautorotate</samp> to disable it.

</dd>

<dt><samp class="option">-framedrop</samp></dt>

<dd> Drop video frames if video is out of sync. Enabled by default if the master

clock is not set to video. Use this option to enable frame dropping for all

master clock sources, use <samp class="option">-noframedrop</samp> to disable it.

</dd>

<dt><samp class="option">-infbuf</samp></dt>

<dd> Do not limit the input buffer size, read as much data as possible from the

input as soon as possible. Enabled by default for realtime streams, where data

may be dropped if not read in time. Use this option to enable infinite buffers

for all inputs, use <samp class="option">-noinfbuf</samp> to disable it.

</dd>

<dt><samp class="option">-filter_threads <var class="var">nb_threads</var></samp></dt>

<dd> Defines how many threads are used to process a filter pipeline. Each pipeline

will produce a thread pool with this many threads available for parallel

processing. The default is 0 which means that the thread count will be

determined by the number of available CPUs.

</dd>

<dt><samp class="option">-enable_vulkan</samp></dt>

<dd> Use vulkan renderer rather than SDL builtin renderer. Depends on libplacebo.

</dd>

<dt><samp class="option">-vulkan_params</samp></dt>

<dd>

Vulkan configuration using a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by

":".

</dd>

<dt><samp class="option">-hwaccel</samp></dt>

<dd> Use HW accelerated decoding. Enable this option will enable vulkan renderer

automatically.

</dd>

<dt><samp class="option">-video_bg <var class="var">pattern</var></samp></dt>

<dd> Set the video background pattern used for transparent videos. For solid color

patterns you can use the color name or color code as specified in the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">Color section in the ffmpeg-utils(1) manual.

You can also use the special <code class="code">tiles</code> keyword for a checker board style

pattern. This is also the default. The <code class="code">none</code> pattern does rendering

without alpha blending effectively ignoring alpha channel.

</dd>

</dl>

 <h3 class="section">3.6 While playing# TOC</h3>

<dl class="table">

<dt><kbd class=”key”>q, ESC</kbd></dt>
<dd> Quit.

</dd>
<dt><kbd class=”key”>f</kbd></dt>
<dd> Toggle full screen.

</dd>
<dt><kbd class=”key”>p, SPC</kbd></dt>
<dd> Pause.

</dd>
<dt><kbd class=”key”>m</kbd></dt>
<dd> Toggle mute.

</dd>
<dt><kbd class=”key”>9, 0</kbd></dt>
<dt><kbd class=”key”>/, *</kbd></dt>
<dd> Decrease and increase volume respectively.

</dd>
<dt><kbd class=”key”>a</kbd></dt>
<dd> Cycle audio channel in the current program.

</dd>
<dt><kbd class=”key”>v</kbd></dt>
<dd> Cycle video channel.

</dd>
<dt><kbd class=”key”>t</kbd></dt>
<dd> Cycle subtitle channel in the current program.

</dd>

<dt><kbd class=”key”>c</kbd></dt>

<dd> Cycle program.

</dd>

<dt><kbd class=”key”>w</kbd></dt>

<dd> Cycle video filters or show modes.

</dd>

<dt><kbd class=”key”>s</kbd></dt>

<dd> Step to the next frame.

Pause if the stream is not already paused, step to the next video

frame, and pause.

</dd>

<dt><kbd class=”key”>left/right</kbd></dt>

<dd> Seek backward/forward by 10 seconds or a custom interval if -seek_interval is set.

</dd>

<dt><kbd class=”key”>down/up</kbd></dt>

<dd> Seek backward/forward 1 minute.

</dd>

<dt><kbd class=”key”>page down/page up</kbd></dt>

<dd> Seek to previous/next chapter or backward/forward 10 minutes if no chapters.

The chapter list is shown briefly, marking the chapter being played at its

left edge. It needs libass and the SDL renderer, so it is unavailable

in builds without libass and with the Vulkan renderer.

</dd>

<dt><kbd class=”key”>l</kbd></dt>

<dd> Keep the chapter list on screen. While it is kept, typing filters the list by

artist and title, down/up move its selection and escape closes it.

</dd>

<dt><kbd class=”key”>o</kbd></dt>

<dd> Show the files in the directory of the input. Enter or a click plays the

selected file or opens the selected directory, escape closes the list. Typing

filters the list by name.

</dd>

<dt><kbd class=”key”>enter</kbd></dt>

<dd> Seek to the chapter selected in the chapter list while it is shown.

</dd>

<dt><kbd class=”key”>mouse wheel</kbd></dt>

<dd> Move the chapter list selection while it is shown.

</dd>

<dt><kbd class=”key”>mouse over the (i) symbol</kbd></dt>

<dd> Show the metadata of that chapter list entry.

</dd>

<dt><kbd class=”key”>left mouse click</kbd></dt>

<dd> Seek to the clicked chapter list entry, or sort the list by the clicked column

button. Clicking the same button again flips the order, and ties are resolved

by the columns clicked before. The shuffle button orders the list by a seed that its left half lowers and its right half raises.

</dd>

<dt><kbd class="key">right mouse click</kbd></dt>

<dd> Seek to percentage in file corresponding to fraction of width.

</dd>

<dt><kbd class="key">left mouse double-click</kbd></dt>

<dd> Toggle full screen.

</dd>

</dl>

 <h2 class="chapter">4 See Also# TOC</h2>

ffmpeg-all, ffmpeg, ffprobe, ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class="chapter">5 Authors#
TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg.
 Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffprobe-all.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> ffprobe Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> ffprobe Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark">

1 Synopsis

2 Description

3 Options

<ul class="toc-numbered-mark">

3.1 Stream specifiers

3.2 Generic options

3.3 AVOptions

3.4 Main options

4 Writers

<ul class="toc-numbered-mark">

4.1 default

4.2 compact, csv

4.3 flat

4.4 ini

4.5 json

4.6 xml

5 Timecode

6 Syntax

<ul class="toc-numbered-mark">

6.1 Quoting and escaping

<ul class="toc-numbered-mark">

6.1.1 Examples

6.2 Date

6.3 Time duration

<ul class="toc-numbered-mark">

6.3.1 Examples

6.4 Video size

6.5 Video rate

6.6 Ratio

6.7 Color

6.8 Channel Layout

7 Expression Evaluation

8 Codec Options

9 Decoders

10 Video Decoders

<ul class="toc-numbered-mark">

10.1 av1

<ul class="toc-numbered-mark">
10.1.1 Options

10.2 hevc
<ul class="toc-numbered-mark">
10.2.1 Options

10.3 rawvideo
<ul class="toc-numbered-mark">
10.3.1 Options

10.4 libastcenc
<ul class="toc-numbered-mark">
10.4.1 Options

10.5 libdav1d
<ul class="toc-numbered-mark">
10.5.1 Options

10.6 libdavs2
10.7 libuavs3d
<ul class="toc-numbered-mark">
10.7.1 Options

10.8 libxevd
<ul class="toc-numbered-mark">
10.8.1 Options

10.9 QSV Decoders
<ul class="toc-numbered-mark">

10.9.1 Common Options

10.9.2 HEVC Options

10.10 v210

<ul class="toc-numbered-mark">

10.10.1 Options

11 Audio Decoders

<ul class="toc-numbered-mark">

11.1 ac3

<ul class="toc-numbered-mark">

11.1.1 AC-3 Decoder Options

11.2 dca

<ul class="toc-numbered-mark">

11.2.1 DCA Decoder Options

11.3 flac

<ul class="toc-numbered-mark">

11.3.1 FLAC Decoder options

11.4 ffwavesynth

11.5 libgsm

11.6 libilbc

<ul class="toc-numbered-mark">

11.6.1 Options

11.7 libmpeghdec

11.8 libopencore-amrnb

11.9 libopencore-amrwb

11.10 libopus

11.11 pcm_dvda

12 Subtitles Decoders

<ul class="toc-numbered-mark">

12.1 libaribb24

<ul class="toc-numbered-mark">

12.1.1 libaribb24 Decoder Options

12.2 libaribcaption

<ul class="toc-numbered-mark">

12.2.1 libaribcaption Decoder Options

12.2.2 libaribcaption decoder usage examples

12.3 dvbsub

<ul class="toc-numbered-mark">

12.3.1 Options

12.4 dvdsub

<ul class="toc-numbered-mark">

12.4.1 Options

12.5 libzvbi-teletext

<ul class="toc-numbered-mark">

12.5.1 Options

13 Bitstream Filters

<ul class="toc-numbered-mark">

13.1 aac_adtstoasc

13.2 av1_metadata

13.3 chomp

13.4 dca_core

13.5 dovi_rpu

13.6 dovi_split

13.7 dump_extra

13.8 dv_error_marker

13.9 eac3_core

13.10 eia608_to_smpte436m

13.11 extract_extradata

13.12 filter_units

13.13 hapqa_extract

13.14 h264_metadata

13.15 h264_mp4toannexb

13.16 h264_redundant_pps

13.17 hevc_metadata

13.18 hevc_mp4toannexb

13.19 imxdump

13.20 jpegapp

13.21 mjpeg2jpeg

13.22 mjpegadump

13.23 mov2textsub

13.24 mpeg2_metadata

13.25 mpeg4_unpack_bframes

13.26 noise

<ul class="toc-numbered-mark">

13.26.1 Examples

13.27 null

13.28 pcm_rechunk

13.29 pgs_frame_merge

13.30 prores_metadata

13.31 remove_extra

13.32 setts

13.33 showinfo

13.34 side_data

13.35 smpte436m_to_eia608

13.36 text2movsub

13.37 trace_headers

13.38 trim

13.39 truehd_core

13.40 vp9_metadata

13.41 vp9_superframe

13.42 vp9_superframe_split

13.43 vp9_raw_reorder

14 Format Options

<ul class="toc-numbered-mark">

14.1 Format stream specifiers

15 Demuxers

<ul class="toc-numbered-mark">

15.1 aa

15.2 aac

15.3 apng

15.4 asf

15.5 astc

15.6 concat

<ul class="toc-numbered-mark">

15.6.1 Syntax

15.6.2 Options

15.6.3 Examples

15.7 dash

<ul class="toc-numbered-mark">

15.7.1 Options

15.8 dvdvideo

<ul class="toc-numbered-mark">

15.8.1 Background

15.8.2 Options

15.8.3 Examples

15.9 ea

<ul class="toc-numbered-mark">

15.9.1 Options

15.10 imf

15.11 flv, live_flv, kux

15.12 gif

15.13 hls

15.14 image2

<ul class="toc-numbered-mark">

15.14.1 Examples

15.15 jpeg_mpf

15.16 ktx

15.17 libgme

15.18 libmodplug

15.19 libopenmpt

15.20 mcc

<ul class="toc-numbered-mark">

15.20.1 Examples

15.21 mov/mp4/3gp

<ul class="toc-numbered-mark">

15.21.1 Options

15.21.2 Audible AAX

15.22 mpegts

15.23 mpjpeg

15.24 rawvideo

15.25 rcwt

<ul class="toc-numbered-mark">

15.25.1 Examples

15.26 sbg

15.27 tedcaptions

15.28 vapoursynth

15.29 w64

15.30 wav

15.31 webp

16 Metadata

17 Protocol Options

18 Protocols

<ul class="toc-numbered-mark">

18.1 amqp

18.2 async

18.3 bluray

18.4 cache

18.5 concat

18.6 concatf

18.7 crypto

18.8 data

18.9 fd

18.10 file

18.11 ftp

18.12 gopher

18.13 gophers

18.14 handle

18.15 http

<ul class="toc-numbered-mark">

18.15.1 HTTP Cookies

18.16 Icecast

18.17 ipfs

18.18 mmst

18.19 mmsh

18.20 md5

18.21 pipe

18.22 prompeg

18.23 rist

18.24 rtmp

18.25 rtmpe

18.26 rtmps

18.27 rtmpt

18.28 rtmpte

18.29 rtmpts

18.30 libcurl

18.31 libsmbclient

18.32 libssh

18.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte
18.34 rtp
18.35 rtsp
<ul class="toc-numbered-mark">
18.35.1 Muxer
18.35.2 Demuxer
18.35.3 Examples

18.36 sap
<ul class="toc-numbered-mark">
18.36.1 Muxer
18.36.2 Demuxer

18.37 sctp
18.38 shared
18.39 srt
18.40 srtp
18.41 subfile
18.42 tee
18.43 tcp
18.44 tls
18.45 dtls
18.46 udp
<ul class="toc-numbered-mark">
18.46.1 Examples

18.47 unix
18.48 zmq

19 Device Options

20 Input Devices

<ul class="toc-numbered-mark">

20.1 alsa

<ul class="toc-numbered-mark">

20.1.1 Options

20.2 android_camera

<ul class="toc-numbered-mark">

20.2.1 Options

20.3 avfoundation

<ul class="toc-numbered-mark">

20.3.1 Options

20.3.2 Examples

20.4 decklink

<ul class="toc-numbered-mark">

20.4.1 Options

20.4.2 Examples

20.5 dshow

<ul class="toc-numbered-mark">

20.5.1 Options

20.5.2 Examples

20.6 fbdev

<ul class="toc-numbered-mark">

20.6.1 Options

20.7 gdigrab
<ul class="toc-numbered-mark">
20.7.1 Options

20.8 iec61883
<ul class="toc-numbered-mark">
20.8.1 Options
20.8.2 Examples

20.9 jack
<ul class="toc-numbered-mark">
20.9.1 Options

20.10 kmsgrab
<ul class="toc-numbered-mark">
20.10.1 Options
20.10.2 Examples

20.11 lavfi
<ul class="toc-numbered-mark">
20.11.1 Options
20.11.2 Examples

20.12 libcdio
<ul class="toc-numbered-mark">
20.12.1 Options

20.13 libdc1394

<ul class="toc-numbered-mark">

20.13.1 Options

20.14 openal

<ul class="toc-numbered-mark">

20.14.1 Options

20.14.2 Examples

20.15 oss

<ul class="toc-numbered-mark">

20.15.1 Options

20.16 pulse

<ul class="toc-numbered-mark">

20.16.1 Options

20.16.2 Examples

20.17 sndio

<ul class="toc-numbered-mark">

20.17.1 Options

20.18 video4linux2, v4l2

<ul class="toc-numbered-mark">

20.18.1 Options

20.19 vfwcap

<ul class="toc-numbered-mark">

20.19.1 Options

20.20 x11grab

<ul class="toc-numbered-mark">

20.20.1 Options

21 Resampler Options

22 Scaler Options

23 Filtering Introduction

24 graph2dot

25 Filtergraph description

<ul class="toc-numbered-mark">

25.1 Filtergraph syntax

25.2 Notes on filtergraph escaping

26 Timeline editing

27 Changing options at runtime with a command

28 Options for filters with several inputs (framesync)

29 Audio Filters

<ul class="toc-numbered-mark">

29.1 aap

29.2 acompressor

<ul class="toc-numbered-mark">

29.2.1 Commands

29.3 acontrast
29.4 acopy
29.5 acrossfade
<ul class="toc-numbered-mark">
29.5.1 Examples

29.6 acrossover
<ul class="toc-numbered-mark">
29.6.1 Examples

29.7 acrusher
<ul class="toc-numbered-mark">
29.7.1 Commands

29.8 acue
29.9 adeclick
29.10 adeclip
29.11 adecorrelate
29.12 adelay
<ul class="toc-numbered-mark">
29.12.1 Examples

29.13 adenorm
<ul class="toc-numbered-mark">
29.13.1 Commands

29.14 aderivative, aintegral

29.15 adrc

<ul class="toc-numbered-mark">

29.15.1 Commands

29.15.2 Examples

29.16 adynamicequalizer

<ul class="toc-numbered-mark">

29.16.1 Commands

29.17 adynamicsmooth

<ul class="toc-numbered-mark">

29.17.1 Commands

29.18 aecho

<ul class="toc-numbered-mark">

29.18.1 Examples

29.19 aemphasis

<ul class="toc-numbered-mark">

29.19.1 Commands

29.20 aeval

<ul class="toc-numbered-mark">

29.20.1 Examples

29.21 aexciter

<ul class="toc-numbered-mark">

29.21.1 Commands

29.22 afade

<ul class="toc-numbered-mark">

29.22.1 Commands

29.22.2 Examples

29.23 afftdn

<ul class="toc-numbered-mark">

29.23.1 Commands

29.23.2 Examples

29.24 afftfilt

<ul class="toc-numbered-mark">

29.24.1 Examples

29.25 afir

<ul class="toc-numbered-mark">

29.25.1 Examples

29.26 aformat

29.27 afreqshift

<ul class="toc-numbered-mark">

29.27.1 Commands

29.28 afwtdn

<ul class="toc-numbered-mark">

29.28.1 Commands

29.29 agate

<ul class="toc-numbered-mark">

29.29.1 Commands

29.30 aiir

<ul class="toc-numbered-mark">

29.30.1 Examples

29.31 alimiter

29.32 allpass

<ul class="toc-numbered-mark">

29.32.1 Commands

29.33 aloop

29.34 amerge

<ul class="toc-numbered-mark">

29.34.1 Examples

29.35 amix

<ul class="toc-numbered-mark">

29.35.1 Examples

29.35.2 Commands

29.36 amultiply

29.37 anequalizer

<ul class="toc-numbered-mark">

29.37.1 Examples

29.37.2 Commands

29.38 anlmdn

<ul class="toc-numbered-mark">

29.38.1 Commands

29.39 anlmf, anlms

<ul class="toc-numbered-mark">

29.39.1 Examples

29.39.2 Commands

29.40 anull

29.41 apad

<ul class="toc-numbered-mark">

29.41.1 Examples

29.42 aphaser

29.43 aphaseshift

<ul class="toc-numbered-mark">

29.43.1 Commands

29.44 apsnr

29.45 apsyclip

<ul class="toc-numbered-mark">

29.45.1 Commands

29.46 apulsator

29.47 aresample

<ul class="toc-numbered-mark">

29.47.1 Examples

29.48 areverse

<ul class="toc-numbered-mark">

29.48.1 Examples

29.49 arls

29.50 arnndn

<ul class="toc-numbered-mark">

29.50.1 Commands

29.51 asdr

29.52 asetnsamples

29.53 asetrate

29.54 ashowinfo

29.55 asisdr

29.56 asoftclip

<ul class="toc-numbered-mark">

29.56.1 Commands

29.57 aspectralstats

29.58 asr

29.59 astats

29.60 asubboost

<ul class="toc-numbered-mark">

29.60.1 Commands

29.61 asubcut

<ul class="toc-numbered-mark">

29.61.1 Commands

29.62 asupercut

<ul class="toc-numbered-mark">

29.62.1 Commands

29.63 asuperpass

<ul class="toc-numbered-mark">

29.63.1 Commands

29.64 asuperstop

<ul class="toc-numbered-mark">

29.64.1 Commands

29.65 atempo

<ul class="toc-numbered-mark">

29.65.1 Examples

29.65.2 Commands

29.66 atilt

<ul class="toc-numbered-mark">

29.66.1 Commands

29.67 atrim

29.68 axcorrelate

<ul class="toc-numbered-mark">

29.68.1 Examples

29.69 bandpass

<ul class="toc-numbered-mark">

29.69.1 Commands

29.70 bandreject

<ul class="toc-numbered-mark">

29.70.1 Commands

29.71 bass, lowshelf

<ul class="toc-numbered-mark">

29.71.1 Commands

29.72 biquad

<ul class="toc-numbered-mark">

29.72.1 Commands

29.73 bs2b

29.74 channelmap

<ul class="toc-numbered-mark">

29.74.1 Examples

29.75 channelsplit

<ul class="toc-numbered-mark">

29.75.1 Examples

29.76 chorus

<ul class="toc-numbered-mark">

29.76.1 Examples

29.77 compand

<ul class="toc-numbered-mark">

29.77.1 Examples

29.78 compensationdelay

<ul class="toc-numbered-mark">

29.78.1 Commands

29.79 crossfeed

<ul class="toc-numbered-mark">

29.79.1 Commands

29.80 crystalizer

<ul class="toc-numbered-mark">

29.80.1 Commands

29.81 dcshift

29.82 deesser

29.83 dialoguenhance

<ul class="toc-numbered-mark">

29.83.1 Commands

29.84 drmeter

29.85 dynaudnorm

<ul class="toc-numbered-mark">

29.85.1 Commands

29.86 earwax

29.87 equalizer

<ul class="toc-numbered-mark">

29.87.1 Examples

29.87.2 Commands

29.88 extrastereo

<ul class="toc-numbered-mark">

29.88.1 Commands

29.89 firequalizer

<ul class="toc-numbered-mark">

29.89.1 Examples

29.90 flanger

29.91 haas

29.92 hdcd

29.93 headphone

<ul class="toc-numbered-mark">

29.93.1 Examples

29.94 highpass

<ul class="toc-numbered-mark">

29.94.1 Commands

29.95 join

29.96 ladspa

<ul class="toc-numbered-mark">

29.96.1 Examples

29.96.2 Commands

29.97 loudnorm

29.98 lowpass

<ul class="toc-numbered-mark">

29.98.1 Examples

29.98.2 Commands

29.99 lv2

<ul class="toc-numbered-mark">

29.99.1 Examples

29.99.2 Commands

29.100 mcompand

29.101 pan

<ul class="toc-numbered-mark">

29.101.1 Mixing examples

29.101.2 Remapping examples

29.102 replaygain

29.103 resample

29.104 rubberband

<ul class="toc-numbered-mark">

29.104.1 Commands

29.105 sidechaincompress

<ul class="toc-numbered-mark">

29.105.1 Commands

29.105.2 Examples

29.106 sidechaingate

<ul class="toc-numbered-mark">

29.106.1 Commands

29.107 silencedetect

<ul class="toc-numbered-mark">

29.107.1 Examples

29.108 silenceremove

<ul class="toc-numbered-mark">

29.108.1 Examples

29.108.2 Commands

29.109 sofalizer

<ul class="toc-numbered-mark">

29.109.1 Examples

29.110 speechnorm

<ul class="toc-numbered-mark">

29.110.1 Commands

29.110.2 Examples

29.111 stereotools

<ul class="toc-numbered-mark">

29.111.1 Commands

29.111.2 Examples

29.112 stereowiden

<ul class="toc-numbered-mark">

29.112.1 Commands

29.113 superequalizer

29.114 surround

29.115 tiltshelf

<ul class="toc-numbered-mark">

29.115.1 Commands

29.116 treble, highshelf

<ul class="toc-numbered-mark">

29.116.1 Commands

29.117 tremolo

29.118 vibrato

29.119 virtualbass

29.120 volume

<ul class="toc-numbered-mark">

29.120.1 Commands

29.120.2 Examples

29.121 volumedetect

<ul class="toc-numbered-mark">

29.121.1 Examples

29.122 whisper

<ul class="toc-numbered-mark">

29.122.1 Examples

30 Audio Sources

<ul class="toc-numbered-mark">

30.1 abuffer

<ul class="toc-numbered-mark">

30.1.1 Examples

30.2 aevalsrc

<ul class="toc-numbered-mark">

30.2.1 Examples

30.3 afdelaysrc

30.4 afireqsrc

30.5 afirsrc

30.6 anullsrc
<ul class="toc-numbered-mark">
30.6.1 Examples

30.7 flite
<ul class="toc-numbered-mark">
30.7.1 Examples

30.8 anoisesrc
<ul class="toc-numbered-mark">
30.8.1 Examples

30.9 hilbert
30.10 sinc
30.11 sine
<ul class="toc-numbered-mark">
30.11.1 Examples

31 Audio Sinks
<ul class="toc-numbered-mark">
31.1 abuffersink
31.2 anullsink

32 Video Filters
<ul class="toc-numbered-mark">
32.1 addroi
<ul class="toc-numbered-mark">
32.1.1 Examples

32.2 alphaextract

32.3 alphamerge

32.4 amplify

<ul class="toc-numbered-mark">

32.4.1 Commands

32.5 ass

<ul class="toc-numbered-mark">

32.5.1 Commands

32.6 atadenoise

<ul class="toc-numbered-mark">

32.6.1 Commands

32.7 avgblur

<ul class="toc-numbered-mark">

32.7.1 Commands

32.8 backgroundkey

<ul class="toc-numbered-mark">

32.8.1 Commands

32.9 bbox

<ul class="toc-numbered-mark">

32.9.1 Commands

32.10 bilateral

<ul class="toc-numbered-mark">

32.10.1 Commands

32.11 bitplanenoise

32.12 blackdetect, blackdetect_vulkan

32.13 blackframe

32.14 blend

<ul class="toc-numbered-mark">

32.14.1 Examples

32.14.2 Commands

32.15 blockdetect

<ul class="toc-numbered-mark">

32.15.1 Examples

32.16 blurdetect

<ul class="toc-numbered-mark">

32.16.1 Examples

32.17 bm3d

<ul class="toc-numbered-mark">

32.17.1 Examples

32.18 boxblur

<ul class="toc-numbered-mark">

32.18.1 Examples

32.19 bwdif

32.20 ccrepack

32.21 cas

<ul class="toc-numbered-mark">

32.21.1 Commands

32.22 chromahold

<ul class="toc-numbered-mark">

32.22.1 Commands

32.23 chromakey

<ul class="toc-numbered-mark">

32.23.1 Commands

32.23.2 Examples

32.24 chromanr

<ul class="toc-numbered-mark">

32.24.1 Commands

32.25 chromashift

<ul class="toc-numbered-mark">

32.25.1 Commands

32.26 ciescope

32.27 codecview

<ul class="toc-numbered-mark">

32.27.1 Examples

32.28 colorbalance

<ul class="toc-numbered-mark">

32.28.1 Examples

32.28.2 Commands

32.29 colorcontrast

<ul class="toc-numbered-mark">

32.29.1 Commands

32.30 colorcorrect

<ul class="toc-numbered-mark">

32.30.1 Commands

32.31 colorchannelmixer

<ul class="toc-numbered-mark">

32.31.1 Examples

32.31.2 Commands

32.32 colordetect

32.33 colorize

<ul class="toc-numbered-mark">

32.33.1 Commands

32.34 colorkey

<ul class="toc-numbered-mark">

32.34.1 Examples

32.34.2 Commands

32.35 colorhold

<ul class="toc-numbered-mark">

32.35.1 Commands

32.36 colorlevels

<ul class="toc-numbered-mark">

32.36.1 Examples

32.36.2 Commands

32.37 colormap

32.38 colormatrix

32.39 colorspace

32.40 colortemperature

<ul class="toc-numbered-mark">

32.40.1 Commands

32.41 convolution

<ul class="toc-numbered-mark">

32.41.1 Commands

32.41.2 Examples

32.42 convolve

32.43 copy

32.44 coreimage

<ul class="toc-numbered-mark">

32.44.1 Examples

32.45 corr

32.46 cover_rect

<ul class="toc-numbered-mark">

32.46.1 Examples

32.47 crop

<ul class="toc-numbered-mark">

32.47.1 Examples

32.47.2 Commands

32.48 cropdetect

<ul class="toc-numbered-mark">

32.48.1 Examples

32.48.2 Commands

32.49 cue

32.50 curves

<ul class="toc-numbered-mark">

32.50.1 Commands

32.50.2 Examples

32.51 datascope

<ul class="toc-numbered-mark">

32.51.1 Commands

32.52 dblur

<ul class="toc-numbered-mark">

32.52.1 Commands

32.53 dctdnoiz

<ul class="toc-numbered-mark">

32.53.1 Examples

32.54 deband

<ul class="toc-numbered-mark">

32.54.1 Commands

32.55 deblock

<ul class="toc-numbered-mark">

32.55.1 Examples

32.55.2 Commands

32.56 decimate

32.57 deconvolve

32.58 dedot

32.59 deflate

<ul class="toc-numbered-mark">

32.59.1 Commands

32.60 deflicker

32.61 deinterlace_d3d12

<ul class="toc-numbered-mark">

32.61.1 Examples

32.62 dejudder

32.63 delogo

<ul class="toc-numbered-mark">

32.63.1 Examples

32.64 derain

32.65 deshake

32.66 despill

<ul class="toc-numbered-mark">

32.66.1 Commands

32.67 detelecine

32.68 dilation

<ul class="toc-numbered-mark">

32.68.1 Commands

32.69 displace

<ul class="toc-numbered-mark">

32.69.1 Examples

32.70 dnn_classify

32.71 dnn_detect

<ul class="toc-numbered-mark">

32.71.1 Examples

32.72 dnn_processing

<ul class="toc-numbered-mark">

32.72.1 Examples

32.73 drawbox

<ul class="toc-numbered-mark">

32.73.1 Examples

32.73.2 Commands

32.74 drawgraph

32.75 drawgrid

<ul class="toc-numbered-mark">

32.75.1 Examples

32.75.2 Commands

32.76 drawtext

<ul class="toc-numbered-mark">

32.76.1 Syntax

32.76.2 Text expansion

32.76.3 Commands

32.76.4 Examples

32.77 drawvg

<ul class="toc-numbered-mark">

32.77.1 Parameters

32.77.2 Pixel Formats

32.77.3 Examples

32.78 edgedetect

<ul class="toc-numbered-mark">

32.78.1 Examples

32.79 elbg

32.80 entropy

32.81 epx

32.82 eq

<ul class="toc-numbered-mark">

32.82.1 Commands

32.83 erosion

<ul class="toc-numbered-mark">

32.83.1 Commands

32.84 estdif

<ul class="toc-numbered-mark">

32.84.1 Commands

32.85 exposure

<ul class="toc-numbered-mark">

32.85.1 Commands

32.86 extractplanes

<ul class="toc-numbered-mark">

32.86.1 Examples

32.87 fade

<ul class="toc-numbered-mark">

32.87.1 Examples

32.88 feedback

<ul class="toc-numbered-mark">

32.88.1 Examples

32.89 fftdnoiz

32.90 fftfilt

<ul class="toc-numbered-mark">

32.90.1 Examples

32.91 field

32.92 fieldhint

32.93 fieldmatch

<ul class="toc-numbered-mark">

32.93.1 p/c/n/u/b meaning

<ul class="toc-numbered-mark">

32.93.1.1 p/c/n

32.93.1.2 u/b

32.93.2 Examples

32.94 fieldorder

32.95 fillborders

<ul class="toc-numbered-mark">

32.95.1 Commands

32.96 find_rect

<ul class="toc-numbered-mark">

32.96.1 Examples

32.97 floodfill

32.98 format

<ul class="toc-numbered-mark">

32.98.1 Examples

32.99 fps

<ul class="toc-numbered-mark">

32.99.1 Examples

32.100 framepack

32.101 framerate

32.102 frc_amf

32.103 vqe_amf

32.104 framestep

32.105 freezedetect

32.106 freezeframes

32.107 frei0r

<ul class="toc-numbered-mark">

32.107.1 Examples

32.107.2 Commands

32.108 fspp

32.109 fsync

32.110 gainmap

<ul class="toc-numbered-mark">

32.110.1 Examples

32.111 gblur

<ul class="toc-numbered-mark">

32.111.1 Commands

32.112 geq

<ul class="toc-numbered-mark">

32.112.1 Examples

32.113 gradfun

<ul class="toc-numbered-mark">

32.113.1 Examples

32.114 graphmonitor

32.115 grayworld

32.116 greyedge

<ul class="toc-numbered-mark">

32.116.1 Examples

32.117 guided

<ul class="toc-numbered-mark">

32.117.1 Commands

32.117.2 Examples

32.118 haldclut

<ul class="toc-numbered-mark">

32.118.1 Commands

32.118.2 Workflow examples

<ul class="toc-numbered-mark">

32.118.2.1 Hald CLUT video stream

32.118.2.2 Hald CLUT with preview

32.119 hflip

32.120 histeq

32.121 histogram

<ul class="toc-numbered-mark">

32.121.1 Examples

32.122 hqdn3d

<ul class="toc-numbered-mark">

32.122.1 Commands

32.123 hwdownload

32.124 hwmap

32.125 hwupload

32.126 hwupload_cuda

32.127 hqx

32.128 hstack

32.129 hsvhold

32.130 hsvkey

32.131 hue

<ul class="toc-numbered-mark">

32.131.1 Examples

32.131.2 Commands

32.132 huesaturation

32.133 hysteresis

32.134 iccdetect

32.135 iccgen

32.136 identity

32.137 idet

<ul class="toc-numbered-mark">

32.137.1 Examples

32.138 il

<ul class="toc-numbered-mark">

32.138.1 Commands

32.139 inflate

<ul class="toc-numbered-mark">

32.139.1 Commands

32.140 interlace, interlace_vulkan

32.141 kerndeint

<ul class="toc-numbered-mark">

32.141.1 Examples

32.142 kirsch

<ul class="toc-numbered-mark">

32.142.1 Commands

32.143 lagfun

<ul class="toc-numbered-mark">

32.143.1 Commands

32.144 latticepal

32.145 lenscorrection

<ul class="toc-numbered-mark">

32.145.1 Options

32.145.2 Commands

32.146 lensfun

<ul class="toc-numbered-mark">

32.146.1 Examples

32.147 lcevc

32.148 libplacebo

<ul class="toc-numbered-mark">

32.148.1 Options

<ul class="toc-numbered-mark">

32.148.1.1 Output mode

32.148.1.2 Scaling

32.148.1.3 Deinterlacing

32.148.1.4 Debanding

32.148.1.5 Color adjustment

32.148.1.6 Peak detection

32.148.1.7 Tone mapping

32.148.1.8 Dithering

32.148.1.9 Custom shaders

32.148.1.10 Debugging / performance

32.148.2 Commands

32.148.3 Examples

32.149 libvmaf

<ul class="toc-numbered-mark">

32.149.1 Examples

32.150 libvmaf_cuda

<ul class="toc-numbered-mark">

32.150.1 Examples

32.151 limitdiff

<ul class="toc-numbered-mark">

32.151.1 Commands

32.152 limiter

<ul class="toc-numbered-mark">

32.152.1 Commands

32.153 loop

<ul class="toc-numbered-mark">

32.153.1 Examples

32.154 lut1d

<ul class="toc-numbered-mark">

32.154.1 Commands

32.155 lut3d

<ul class="toc-numbered-mark">

32.155.1 Commands

32.156 lumakey

<ul class="toc-numbered-mark">

32.156.1 Commands

32.157 lut, lutrgb, lutyuv

<ul class="toc-numbered-mark">

32.157.1 Commands

32.157.2 Examples

32.158 lut2, tlut2

<ul class="toc-numbered-mark">

32.158.1 Commands

32.158.2 Examples

32.159 maskedclamp

<ul class="toc-numbered-mark">

32.159.1 Commands

32.160 maskedmax

<ul class="toc-numbered-mark">

32.160.1 Commands

32.161 maskedmerge

<ul class="toc-numbered-mark">

32.161.1 Commands

32.162 maskedmin

<ul class="toc-numbered-mark">

32.162.1 Commands

32.163 maskedthreshold

<ul class="toc-numbered-mark">

32.163.1 Commands

32.164 maskfun

<ul class="toc-numbered-mark">

32.164.1 Commands

32.165 mcdeint

32.166 median

<ul class="toc-numbered-mark">

32.166.1 Commands

32.167 mergeplanes

<ul class="toc-numbered-mark">

32.167.1 Examples

32.168 mestimate

32.169 mestimate_d3d12

<ul class="toc-numbered-mark">

32.169.1 Examples

32.170 midequalizer

32.171 minterpolate

32.172 mix

<ul class="toc-numbered-mark">

32.172.1 Commands

32.173 monochrome

<ul class="toc-numbered-mark">

32.173.1 Commands

32.174 morpho

<ul class="toc-numbered-mark">

32.174.1 Commands

32.175 mpdecimate

32.176 msad

32.177 multiply

<ul class="toc-numbered-mark">

32.177.1 Commands

32.178 negate

<ul class="toc-numbered-mark">

32.178.1 Commands

32.179 nlmeans

32.180 nnedi

<ul class="toc-numbered-mark">

32.180.1 Commands

32.181 noformat

<ul class="toc-numbered-mark">

32.181.1 Examples

32.182 noise

<ul class="toc-numbered-mark">

32.182.1 Examples

32.183 normalize

<ul class="toc-numbered-mark">

32.183.1 Commands

32.183.2 Examples

32.184 null

32.185 ocio

<ul class="toc-numbered-mark">

32.185.1 Examples

32.186 ocr

32.187 ocv

<ul class="toc-numbered-mark">

32.187.1 dilate

32.187.2 erode

32.187.3 smooth

32.188 oscilloscope

<ul class="toc-numbered-mark">

32.188.1 Commands

32.188.2 Examples

32.189 overlay

<ul class="toc-numbered-mark">

32.189.1 Commands

32.189.2 Examples

32.190 owdenoise

32.191 pad

<ul class="toc-numbered-mark">

32.191.1 Examples

32.192 palettegen

<ul class="toc-numbered-mark">

32.192.1 Examples

32.193 paletteuse

<ul class="toc-numbered-mark">

32.193.1 Examples

32.194 perspective

32.195 phase

<ul class="toc-numbered-mark">

32.195.1 Commands

32.196 photosensitivity

32.197 pixdesctest

32.198 pixelize

<ul class="toc-numbered-mark">

32.198.1 Commands

32.199 pixscope

<ul class="toc-numbered-mark">

32.199.1 Commands

32.200 pp7

32.201 premultiply

32.202 premultiply_dynamic

32.203 prewitt

<ul class="toc-numbered-mark">

32.203.1 Commands

32.204 pseudocolor

<ul class="toc-numbered-mark">

32.204.1 Commands

32.204.2 Examples

32.205 psnr

<ul class="toc-numbered-mark">

32.205.1 Examples

32.206 pullup

32.207 qp

<ul class="toc-numbered-mark">

32.207.1 Examples

32.208 qrencode

<ul class="toc-numbered-mark">

32.208.1 qrencode Expressions

32.208.2 qrencode Text expansion

32.208.3 Examples

32.209 quirc

32.210 random

32.211 readeia608

<ul class="toc-numbered-mark">

32.211.1 Commands

32.211.2 Examples

32.212 readvitc

<ul class="toc-numbered-mark">

32.212.1 Examples

32.213 remap

32.214 removegrain

32.215 removelogo

32.216 repeatfields

32.217 reverse

<ul class="toc-numbered-mark">

32.217.1 Examples

32.218 rgbashift

<ul class="toc-numbered-mark">

32.218.1 Commands

32.219 roberts

<ul class="toc-numbered-mark">

32.219.1 Commands

32.220 rotate

<ul class="toc-numbered-mark">

32.220.1 Examples

32.220.2 Commands

32.221 sab

32.222 scale

<ul class="toc-numbered-mark">

32.222.1 Options

32.222.2 Examples

32.222.3 Commands

32.223 scharr

<ul class="toc-numbered-mark">

32.223.1 Commands

32.224 scroll

<ul class="toc-numbered-mark">

32.224.1 Commands

32.225 scdet

32.226 selectivecolor

<ul class="toc-numbered-mark">

32.226.1 Examples

32.227 separatefields

32.228 setdar, setsar

<ul class="toc-numbered-mark">

32.228.1 Examples

32.229 setfield

32.230 setparams

32.231 shear

<ul class="toc-numbered-mark">

32.231.1 Commands

32.232 showinfo

32.233 showpalette

32.234 shuffleframes

<ul class="toc-numbered-mark">

32.234.1 Examples

32.235 shufflepixels

32.236 shuffleplanes

<ul class="toc-numbered-mark">

32.236.1 Examples

32.237 signalstats

<ul class="toc-numbered-mark">

32.237.1 Examples

32.238 signature

<ul class="toc-numbered-mark">

32.238.1 Examples

32.239 siti

<ul class="toc-numbered-mark">

32.239.1 Examples

32.240 smartblur

32.241 sobel

<ul class="toc-numbered-mark">

32.241.1 Commands

32.242 spp

<ul class="toc-numbered-mark">

32.242.1 Commands

32.243 sr

32.244 sr_amf

<ul class="toc-numbered-mark">

32.244.1 Examples

32.245 ssim

<ul class="toc-numbered-mark">

32.245.1 Examples

32.246 stereo3d

<ul class="toc-numbered-mark">

32.246.1 Examples

32.247 streamselect, astreamselect

<ul class="toc-numbered-mark">

32.247.1 Commands

32.247.2 Examples

32.248 subtitles

32.249 super2xsai

32.250 swaprect

<ul class="toc-numbered-mark">

32.250.1 Commands

32.251 swapuv

32.252 tblend

32.253 telecine

32.254 thistogram

32.255 threshold

<ul class="toc-numbered-mark">

32.255.1 Commands

32.255.2 Examples

32.256 thumbnail

<ul class="toc-numbered-mark">

32.256.1 Examples

32.257 tile

<ul class="toc-numbered-mark">

32.257.1 Examples

32.258 tiltandshift

32.259 tinterlace

32.260 tmedian

<ul class="toc-numbered-mark">

32.260.1 Commands

32.261 tmidequalizer

32.262 tmix

<ul class="toc-numbered-mark">

32.262.1 Examples

32.262.2 Commands

32.263 tonemap

<ul class="toc-numbered-mark">

32.263.1 Options

32.264 tpad

32.265 transpose

32.266 trim

32.267 unpremultiply

32.268 unsharp

<ul class="toc-numbered-mark">

32.268.1 Examples

32.269 untile

<ul class="toc-numbered-mark">

32.269.1 Examples

32.270 uspp

32.271 v360

<ul class="toc-numbered-mark">

32.271.1 Examples

32.271.2 Commands

32.272 vaguedenoiser

32.273 varblur

<ul class="toc-numbered-mark">

32.273.1 Commands

32.274 vectorscope

32.275 vidstabdetect

<ul class="toc-numbered-mark">

32.275.1 Examples

32.276 vidstabtransform

<ul class="toc-numbered-mark">

32.276.1 Options

32.276.2 Examples

32.277 vflip

32.278 vfrdet

32.279 vibrance

<ul class="toc-numbered-mark">

32.279.1 Commands

32.280 vif

32.281 vignette

<ul class="toc-numbered-mark">

32.281.1 Expressions

32.281.2 Examples

32.282 vmafmotion

32.283 vpp_amf

<ul class="toc-numbered-mark">

32.283.1 Examples

32.284 vstack

32.285 w3fdif

<ul class="toc-numbered-mark">

32.285.1 Commands

32.286 waveform

32.287 weave, doubleweave

<ul class="toc-numbered-mark">

32.287.1 Examples

32.288 xbr

32.289 xcorrelate

32.290 xfade

<ul class="toc-numbered-mark">

32.290.1 Examples

32.291 xmedian

<ul class="toc-numbered-mark">

32.291.1 Commands

32.292 xpsnr

<ul class="toc-numbered-mark">

32.292.1 Examples

32.293 xstack

<ul class="toc-numbered-mark">

32.293.1 Examples

32.294 yadif

32.295 yaepblur

<ul class="toc-numbered-mark">

32.295.1 Commands

32.296 zoompan

<ul class="toc-numbered-mark">

32.296.1 Examples

32.297 zscale

<ul class="toc-numbered-mark">

32.297.1 Options

32.297.2 Commands

33 CUDA Video Filters

<ul class="toc-numbered-mark">

33.1 bilateral_cuda

<ul class="toc-numbered-mark">

33.1.1 Examples

33.2 bwdif_cuda

33.3 chromakey_cuda

<ul class="toc-numbered-mark">

33.3.1 Examples

33.4 colorspace_cuda

33.5 overlay_cuda

33.6 pad_cuda

<ul class="toc-numbered-mark">

33.6.1 Examples

33.7 scale_cuda

<ul class="toc-numbered-mark">

33.7.1 Examples

33.8 thumbnail_cuda

<ul class="toc-numbered-mark">

33.8.1 Example

33.9 transpose_cuda

33.10 yadif_cuda

34 OpenCL Video Filters

<ul class="toc-numbered-mark">

34.1 avgblur_opencl

<ul class="toc-numbered-mark">

34.1.1 Example

34.2 boxblur_opencl

<ul class="toc-numbered-mark">

34.2.1 Examples

34.3 colorkey_opencl

<ul class="toc-numbered-mark">

34.3.1 Examples

34.4 convolution_opencl

<ul class="toc-numbered-mark">

34.4.1 Examples

34.5 erosion_opencl

<ul class="toc-numbered-mark">

34.5.1 Example

34.6 deshake_opencl

<ul class="toc-numbered-mark">

34.6.1 Examples

34.7 dilation_opencl

<ul class="toc-numbered-mark">

34.7.1 Example

34.8 nlmeans_opencl

34.9 overlay_opencl

<ul class="toc-numbered-mark">

34.9.1 Examples

34.10 pad_opencl

34.11 prewitt_opencl

<ul class="toc-numbered-mark">

34.11.1 Example

34.12 program_opencl

34.13 remap_opencl

34.14 roberts_opencl

<ul class="toc-numbered-mark">

34.14.1 Example

34.15 sobel_opencl

<ul class="toc-numbered-mark">

34.15.1 Example

34.16 tonemap_opencl

<ul class="toc-numbered-mark">

34.16.1 Example

34.17 unsharp_opencl

<ul class="toc-numbered-mark">

34.17.1 Examples

34.18 xfade_opencl

35 VAAPI Video Filters

<ul class="toc-numbered-mark">

35.1 overlay_vaapi

<ul class="toc-numbered-mark">

35.1.1 Examples

35.2 tonemap_vaapi

<ul class="toc-numbered-mark">

35.2.1 Example

35.3 hstack_vaapi

35.4 vstack_vaapi

35.5 xstack_vaapi

35.6 pad_vaapi

35.7 drawbox_vaapi

<ul class="toc-numbered-mark">

35.7.1 Examples

36 VideoToolbox Video Filters

<ul class="toc-numbered-mark">

36.1 scale_vt

<ul class="toc-numbered-mark">

36.1.1 Examples

36.2 transpose_vt

37 Vulkan Video Filters

<ul class="toc-numbered-mark">

37.1 avgblur_vulkan

37.2 blend_vulkan

37.3 bwdif_vulkan

37.4 chromaber_vulkan

37.5 color_vulkan

37.6 vflip_vulkan

37.7 hflip_vulkan

37.8 flip_vulkan

37.9 fruc_vulkan

37.10 gblur_vulkan

37.11 nlmeans_vulkan

37.12 overlay_vulkan

37.13 transpose_vulkan

37.14 v360_vulkan

<ul class="toc-numbered-mark">

37.14.1 Examples

38 QSV Video Filters

<ul class="toc-numbered-mark">

38.1 hstack_qsv

38.2 vstack_qsv

38.3 xstack_qsv

39 Video Sources

<ul class="toc-numbered-mark">

39.1 buffer

39.2 cellauto

<ul class="toc-numbered-mark">

39.2.1 Examples

39.3 coreimagesrc

<ul class="toc-numbered-mark">

39.3.1 Examples

39.4 ddagrab

<ul class="toc-numbered-mark">

39.4.1 Examples

39.5 gfxcapture

<ul class="toc-numbered-mark">

39.5.1 Examples

39.6 gradients

<ul class="toc-numbered-mark">

39.6.1 Commands

39.7 mandelbrot

39.8 mptestsrc

39.9 frei0r_src

39.10 life

<ul class="toc-numbered-mark">

39.10.1 Examples

39.11 perlin

<ul class="toc-numbered-mark">

39.11.1 Options

39.11.2 Examples

39.12 qrencodesrc

<ul class="toc-numbered-mark">

39.12.1 Examples

39.13 allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc

<ul class="toc-numbered-mark">

39.13.1 Examples

39.13.2 Commands

39.14 openclsrc

39.15 sierpinski

39.16 zoneplate

<ul class="toc-numbered-mark">

39.16.1 Commands

39.16.2 Examples

40 Video Sinks

<ul class="toc-numbered-mark">

40.1 buffersink

40.2 nullsink

41 Multimedia Filters

<ul class="toc-numbered-mark">

41.1 a3dscope

<ul class="toc-numbered-mark">

41.1.1 Commands

41.2 abitscope

41.3 adrawgraph

41.4 agraphmonitor

41.5 ahistogram

41.6 aphasemeter

<ul class="toc-numbered-mark">

41.6.1 phasing detection

41.6.2 Examples

41.7 avectorscope

<ul class="toc-numbered-mark">

41.7.1 Examples

41.7.2 Commands

41.8 bench, abench

<ul class="toc-numbered-mark">

41.8.1 Examples

41.9 concat

<ul class="toc-numbered-mark">

41.9.1 Examples

41.9.2 Commands

41.10 ebur128

<ul class="toc-numbered-mark">

41.10.1 Examples

41.11 interleave, ainterleave

<ul class="toc-numbered-mark">

41.11.1 Examples

41.12 latency, alatency

41.13 metadata, ametadata

<ul class="toc-numbered-mark">

41.13.1 Examples

41.14 perms, aperms

41.15 realtime, arealtime

<ul class="toc-numbered-mark">

41.15.1 Commands

41.16 segment, asegment

<ul class="toc-numbered-mark">

41.16.1 Examples

41.17 select, aselect

<ul class="toc-numbered-mark">

41.17.1 Examples

41.18 sendcmd, asendcmd

<ul class="toc-numbered-mark">

41.18.1 Commands syntax

41.18.2 Examples

41.19 setpts, asetpts

<ul class="toc-numbered-mark">

41.19.1 Examples

41.19.2 Commands

41.20 setrange

41.21 settb, asettb

<ul class="toc-numbered-mark">

41.21.1 Examples

41.22 showcqt

<ul class="toc-numbered-mark">

41.22.1 Examples

41.23 showcwt

41.24 showfreqs

41.25 showspatial

41.26 showspectrum

<ul class="toc-numbered-mark">

41.26.1 Examples

41.27 showspectrumpic

<ul class="toc-numbered-mark">

41.27.1 Examples

41.28 showvolume

41.29 showwaves

<ul class="toc-numbered-mark">

41.29.1 Examples

41.30 showwavespic

<ul class="toc-numbered-mark">

41.30.1 Examples

41.31 sidedata, asidedata

41.32 spectrumsynth

<ul class="toc-numbered-mark">

41.32.1 Examples

41.33 split, asplit

<ul class="toc-numbered-mark">

41.33.1 Examples

41.34 zmq, azmq

<ul class="toc-numbered-mark">

41.34.1 Examples

42 Multimedia Sources

<ul class="toc-numbered-mark">

42.1 amovie

42.2 avsynctest

<ul class="toc-numbered-mark">

42.2.1 Commands

42.3 movie

<ul class="toc-numbered-mark">

42.3.1 Examples

42.3.2 Commands

43 External libraries

<ul class="toc-numbered-mark">

43.1 Alliance for Open Media (AOM)

43.2 ARM ASTC Encoder

43.3 AMD AMF/VCE

43.4 AviSynth

43.5 Chromaprint

43.6 codec2

43.7 dav1d

43.8 davs2

43.9 uavs3d

43.10 Game Music Emu

43.11 Intel QuickSync Video

43.12 Kvazaar

43.13 LAME

43.14 LCEVCdec

43.15 libilbc

43.16 libjxl

43.17 libvpx

43.18 ModPlug

43.19 ONNX Runtime

43.20 OpenCORE, VisualOn, and Fraunhofer libraries

<ul class="toc-numbered-mark">

43.20.1 OpenCORE AMR

43.20.2 VisualOn AMR-WB encoder library

43.20.3 Fraunhofer AAC library

43.20.4 Fraunhofer xHE-AAC library

43.20.5 Fraunhofer MPEG-H 3D Audio decoder library

43.20.6 LC3 library

43.21 OpenH264

43.22 OpenJPEG

43.23 rav1e

43.24 SVT-AV1

43.25 SVT-JPEG-XS

43.26 TwoLAME

43.27 VapourSynth

43.28 x264

43.29 x265

43.30 xavs

43.31 xavs2

43.32 eXtra-fast Essential Video Encoder (XEVE)

43.33 eXtra-fast Essential Video Decoder (XEVD)

43.34 ZVBI

44 Supported File Formats, Codecs or Features

<ul class="toc-numbered-mark">

44.1 File Formats

44.2 Image Formats

44.3 Video Codecs

44.4 Audio Codecs

44.5 Subtitle Formats

44.6 Network Protocols

44.7 Input/Output Devices

44.8 Timecode

45 See Also

46 Authors

 <h2 class="chapter">1 Synopsis# TOC</h2>

ffprobe [<var class="var">options</var>] <samp class="file">input_url</samp> <h2 class="chapter">2 Description# TOC</h2>

ffprobe gathers information from multimedia streams and prints it in human- and machine-readable fashion. For example it can be used to check the format of the container used by a multimedia stream and the format and type of each media stream contained in it. If a url is specified in input, ffprobe will try to open and probe the url content. If the url cannot be opened or recognized as a multimedia file, a positive exit code is returned. If no output is specified as output with <samp class="option">o</samp> ffprobe will write to stdout. ffprobe may be employed both as a standalone application or in combination with a textual filter, which may perform more sophisticated processing, e.g. statistical processing or plotting. Options are used to list some of the formats supported by ffprobe or for specifying which information to display, and for setting how ffprobe will show it. ffprobe output is designed to be easily parsable by a textual filter, and consists of one or more sections of a form defined by the selected writer, which is specified by the <samp class="option">output_format</samp> option. Sections may contain other nested sections, and are identified by a name (which may be shared by other sections), and an unique name. See the output of <samp class="option">sections</samp>. Metadata tags stored in the container or in the streams are recognized and printed in the corresponding "FORMAT", "STREAM", "STREAM_GROUP_STREAM" or "PROGRAM_STREAM" section.

 <h2 class="chapter">3 Options# TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or ’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’ as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class="example-preformatted">ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">3.1 Stream specifiers# TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class=”code”>a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class=”code”>-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class=”code”>-codec copy</code>

or <code class=”code”>-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class=”code”>-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class=”var”>stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class=”var”>stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class=”option”><var class=”var”>stream_type</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> <var class=”var”>stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class="var">additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class="var">additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class="option">g:<var class="var">group_specifier</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class="var">group_specifier</var>.

if <var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class="var">additional_stream_specifier</var>.

<var class="var">group_specifier</var> may be one of the following:

<dl class="table">

<dt><samp class="option"><var class="var">group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class=”var”>additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class=”var”>additional_stream_specifier</var>.

</dd>

<dt><samp class=”option”>#<var class=”var”>stream_id</var> or i:<var class=”var”>stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class=”option”>m:<var class=”var”>key</var>[:<var class=”var”>value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class=”var”>key</var> having the specified value. If

<var class=”var”>value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class=”var”>key</var> or <var class=”var”>value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class=”option”>disp:<var class=”var”>dispositions</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class=”var”>dispositions</var> is a list of

one or more dispositions (as printed by the <samp class=”option”>-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class=”option”>u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class=”command”>ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">3.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options, including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed information about the demuxer named <var class="var">demuxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class="option">muxer=<var class="var">muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class="var">muxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class="option">filter=<var class="var">filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class="var">filter_name</var>. Use the <samp class="option">-filters</samp> option to get a list of all filters. </dd> <dt><samp class="option">bsf=<var class="var">bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class="var">bitstream_filter_name</var>. Use the <samp class="option">-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class="option">protocol=<var class="var">protocol_name</var></samp></dt> <dd> Print detailed

information about the protocol named <var class="var">protocol_name</var>. Use the <samp class="option">-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class="option">-version</samp></dt> <dd> Show version. </dd> <dt><samp class="option">-buildconf</samp></dt> <dd> Show the build configuration, one option per line. </dd> <dt><samp class="option">-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class="option">-demuxers</samp></dt> <dd> Show available demuxers. </dd> <dt><samp class="option">-muxers</samp></dt> <dd> Show available muxers. </dd> <dt><samp class="option">-devices</samp></dt> <dd> Show available devices. </dd> <dt><samp class="option">-codecs</samp></dt> <dd> Show all codecs known to libavcodec. Note that the term ’codec’ is used throughout this documentation as a shortcut for what is more correctly called a media bitstream format. </dd> <dt><samp class="option">-decoders</samp></dt> <dd> Show available decoders. </dd> <dt><samp class="option">-encoders</samp></dt> <dd> Show all available encoders. </dd> <dt><samp class="option">-bsfs</samp></dt> <dd> Show available bitstream filters. </dd> <dt><samp class="option">-protocols</samp></dt> <dd> Show available protocols. </dd> <dt><samp class="option">-filters</samp></dt> <dd> Show available libavfilter filters. </dd> <dt><samp class="option">-pix_fmts</samp></dt> <dd> Show available pixel formats. </dd> <dt><samp class="option">-sample_fmts</samp></dt> <dd> Show available sample formats. </dd> <dt><samp class="option">-layouts</samp></dt> <dd> Show channel names and standard channel layouts. </dd> <dt><samp class="option">-dispositions</samp></dt> <dd> Show stream dispositions. </dd> <dt><samp class="option">-colors</samp></dt> <dd> Show recognized color names. </dd> <dt><samp class="option">-sources <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sources of the input device. Some devices may provide system-dependent source names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sources pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt> <dd> Show autodetected sinks of the output device. Some devices may provide system-dependent sink names that cannot be autodetected. The returned list cannot be assumed to be always complete. <pre class="example-preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var class="var">loglevel</var></samp></dt> <dd> Set logging level and flags used by the library. The optional <var class="var">flags</var> prefix can consist of the following values: <dl class="table"> <dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates that repeated log output should not be compressed to the first line and the "Last message repeated n times" line will be omitted. </dd> <dt>‘<samp class="samp">level</samp>’</dt> <dd> Indicates that log output should add a <code class="code">[level]</code> prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var class="var">flags</var> or changing <var class="var">loglevel</var>. When setting both <var class="var">flags</var> and <var class="var">loglevel</var>, a ’+’ separator is expected between the last <var class="var">flags</var> value and before <var class="var">loglevel</var>. <var class="var">loglevel</var> is a string or a number containing one of the following values: <dl class="table"> <dt>‘<samp class="samp">quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class="samp">panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class="samp">fatal, 8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class="samp">error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class="samp">warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class="samp">info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class="samp">verbose, 40</samp>’</dt> <dd> Same as <code class="code">info</code>, except more verbose. </dd> <dt>‘<samp class="samp">debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class="samp">trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class="code">level</code>

prefix, and set <var class="var">loglevel</var> to <code class="code">verbose</code>:
 <pre class="example-preformatted">ffmpeg
-loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code class="code">level</code> prefix flag or <var class="var">loglevel</var>:
 <pre class="example-preformatted">ffmpeg [...]
-loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class="env">AV_LOG_FORCE_NOCOLOR</code>, or can be forced setting the environment variable <code class="env">AV_LOG_FORCE_COLOR</code>.
 </dd> <dt><samp class="option">-report</samp></dt> <dd> Dump full command line and log output to a file named <code class="code"><var class="var">program</var>-<var class="var">YYYYMMDD</var>-<var class="var">HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class="code">-loglevel debug</code>. Setting the environment variable <code class="env">FFREPORT</code> to any value has the same effect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual).
 The following options are recognized: <dl class="table">
<dt><samp class="option">file</samp></dt> <dd> set the file name to use for the report; <code class="code">%p</code> is expanded to the name of the program, <code class="code">%t</code> is expanded to a timestamp, <code class="code">%%</code> is expanded to a plain <code class="code">%</code> </dd> <dt><samp class="option">level</samp></dt> <dd> set the log verbosity level using a numerical value (see <code class="code">-loglevel</code>).
</dd> </dl>

For example, to output a report to a file named <samp class="file">ffreport.log</samp> using a log level of <code class="code">32</code> (alias for log level <code class="code">info</code>):   <pre class="example-preformatted">FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class="option">-hide_banner</samp></dt> <dd> Suppress printing banner.
All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class="option">-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it

unless you know what you’re doing.
<pre class="example-preformatted">ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class="table"> <dt>‘<samp class="samp">x86</samp>’</dt> <dd><dl class="table">
<dt>‘<samp class="samp">mmx</samp>’</dt> <dt>‘<samp class="samp">mmxext</samp>’</dt> <dt>‘<samp class="samp">sse</samp>’</dt>
<dt>‘<samp class="samp">sse2</samp>’</dt> <dt>‘<samp class="samp">sse2slow</samp>’</dt> <dt>‘<samp class="samp">sse3</samp>’</dt>
<dt>‘<samp class="samp">sse3slow</samp>’</dt> <dt>‘<samp class="samp">ssse3</samp>’</dt> <dt>‘<samp class="samp">atom</samp>’</dt>
<dt>‘<samp class="samp">sse4.1</samp>’</dt> <dt>‘<samp class="samp">sse4.2</samp>’</dt> <dt>‘<samp class="samp">avx</samp>’</dt>
<dt>‘<samp class="samp">avx2</samp>’</dt> <dt>‘<samp class="samp">xop</samp>’</dt> <dt>‘<samp class="samp">fma3</samp>’</dt>
<dt>‘<samp class="samp">fma4</samp>’</dt> <dt>‘<samp class="samp">3dnow</samp>’</dt> <dt>‘<samp class="samp">3dnowext</samp>’</dt>
<dt>‘<samp class="samp">bmi1</samp>’</dt> <dt>‘<samp class="samp">bmi2</samp>’</dt> <dt>‘<samp class="samp">cmov</samp>’</dt>
</dl> </dd> <dt>‘<samp class="samp">ARM</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv5te</samp>’</dt>
<dt>‘<samp class="samp">armv6</samp>’</dt> <dt>‘<samp class="samp">armv6t2</samp>’</dt> <dt>‘<samp class="samp">vfp</samp>’</dt>
<dt>‘<samp class="samp">vfpv3</samp>’</dt> <dt>‘<samp class="samp">neon</samp>’</dt> <dt>‘<samp class="samp">setend</samp>’</dt>
</dl> </dd> <dt>‘<samp class="samp">AArch64</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv8</samp>’</dt>
<dt>‘<samp class="samp">vfp</samp>’</dt> <dt>‘<samp class="samp">neon</samp>’</dt> </dl> </dd> <dt>‘<samp class="samp">PowerPC</samp>’</dt> <dd><dl class="table">
<dt>‘<samp class="samp">altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class="samp">Specific Processors</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">pentium2</samp>’</dt>
<dt>‘<samp class="samp">pentium3</samp>’</dt>
<dt>‘<samp class="samp">pentium4</samp>’</dt>
<dt>‘<samp class="samp">k6</samp>’</dt> <dt>‘<samp class="samp">k62</samp>’</dt> <dt>‘<samp class="samp">athlon</samp>’</dt>
<dt>‘<samp class="samp">athlonxp</samp>’</dt> <dt>‘<samp class="samp">k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class="option">-cpucount <var class="var">count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class="example-preformatted">ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class="option">-max_alloc <var class="var">bytes</var></samp></dt>

<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is INT_MAX. </dd> </dl>

 <h3 class=”section”>3.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and libavcodec libraries. To see the list of available AVOptions, use the <samp class=”option”>-help</samp> option. They are separated into two categories: <dl class=”table”> <dt><samp class=”option”>generic</samp></dt> <dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class=”option”>private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class=”option”>id3v2_version</samp> private option of the MP3 muxer: <pre class=”example-preformatted”>ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class=”example-preformatted”>ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class=”option”>-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class=”option”>-option 0</samp>/<samp class=”option”>-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class=”section”>3.4 Main options# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-f <var class=”var”>format</var></samp></dt> <dd> Force format to use. </dd> <dt><samp class=”option”>-unit</samp></dt> <dd> Show the unit of the displayed values.

</dd> <dt><samp class="option">-prefix</samp></dt> <dd> Use SI prefixes for the displayed values. Unless the "-byte_binary_prefix" option is used all the prefixes are decimal. </dd> <dt><samp class="option">-byte_binary_prefix</samp></dt> <dd> Force the use of binary prefixes for byte values. </dd> <dt><samp class="option">-sexagesimal</samp></dt> <dd> Use sexagesimal format HH:MM:SS.MICROSECONDS for time values. </dd> <dt><samp class="option">-pretty</samp></dt> <dd> Prettify the format of the displayed values, it corresponds to the options "-unit -prefix -byte_binary_prefix -sexagesimal". </dd> <dt><samp class="option">-output_format, -of, -print_format <var class="var">writer_name</var>[=<var class="var">writer_options</var>]</samp></dt> <dd> Set the output printing format. <var class="var">writer_name</var> specifies the name of the writer, and <var class="var">writer_options</var> specifies the options to be passed to the writer. For example for printing the output in JSON format, specify: <pre class="example-preformatted">-output_format json </pre>

For more details on the available output printing formats, see the Writers section below. </dd> <dt><samp class="option">-sections</samp></dt> <dd> Print sections structure and section information, and exit. The output is not meant to be parsed by a machine. </dd> <dt><samp class="option">-select_streams <var class="var">stream_specifier</var></samp></dt> <dd> Select only the streams specified by <var class="var">stream_specifier</var>. This option affects only the options related to streams (e.g. <code class="code">show_streams</code>, <code class="code">show_packets</code>, etc.). For example to show only audio streams, you can use the command: <pre class="example-preformatted">ffprobe -show_streams -select_streams a INPUT </pre>

To show only video packets belonging to the video stream with index 1: <pre class="example-preformatted">ffprobe -show_packets -select_streams v:1 INPUT </pre>

</dd> <dt><samp class="option">-show_data</samp></dt> <dd> Show payload data, as a hexadecimal and ASCII dump (other formats can be selected using <samp class="option">-data_dump_format</samp>). Coupled with <samp class="option">-show_packets</samp>, it will dump the packets’ data. Coupled with <samp class="option">-show_streams</samp>, it will dump the codec extradata. The dump is printed as the "data" field. It may contain newlines. </dd> <dt><samp class="option">-show_data_hash <var class="var">algorithm</var></samp></dt> <dd> Show a hash of payload data, for packets with <samp class="option">-show_packets</samp> and for codec extradata with <samp class="option">-show_streams</samp>. </dd> <dt><samp class="option">-

data_dump_format <var class=”var”>format</var></samp></dt> <dd> Select a format used for the data dumps enabled with the <samp class=”option”>-show_data</samp> option. The default is <code class=”code”>xxd</code> which is a hexdump format compatible with the well-known <code class=”command”>xxd</code> program. <code class=”code”>base64</code> is also supported. </dd> <dt><samp class=”option”>-show_error</samp></dt> <dd> Show information about the error found when trying to probe the input. The error information is printed within a section with name "ERROR". </dd> <dt><samp class=”option”>-show_format</samp></dt> <dd> Show information about the container format of the input multimedia stream. All the container format information is printed within a section with name "FORMAT". </dd> <dt><samp class=”option”>-show_entries <var class=”var”>section_entries</var></samp></dt> <dd> Set list of entries to show. Entries are specified according to the following syntax. <var class=”var”>section_entries</var> contains a list of section entries separated by <code class=”code”>:</code>. Each section entry is composed by a section name (or unique name), optionally followed by a list of entries local to that section, separated by <code class=”code”>,</code>. If section name is specified but is followed by no <code class=”code”>=</code>, all entries are printed to output, together with all the contained sections. Otherwise only the entries specified in the local section entries list are printed. In particular, if <code class=”code”>=</code> is specified but the list of local entries is empty, then no entries will be shown for that section. Note that the order of specification of the local section entries is not honored in the output, and the usual display order will be retained. The formal syntax is given by: <pre class=”example-preformatted”><var class=”var”>LOCAL_SECTION_ENTRIES</var> ::= <var class=”var”>SECTION_ENTRY_NAME</var>[,<var class=”var”>LOCAL_SECTION_ENTRIES<
<var class=”var”>SECTION_ENTRY</var> ::= <var class=”var”>SECTION_NAME</var>[=[<var class=”var”>LOCAL_SECTION_ENTRIES</var>]] <var class=”var”>SECTION_ENTRIES</var> ::= <var class=”var”>SECTION_ENTRY</var>[:<var class=”var”>SECTION_ENTRIES</var>] </pre>

For example, to show only the index and type of each stream, and the PTS time, duration time, and stream index of the packets, you can specify the argument: <pre class=”example-preformatted”>packet=pts_time,duration_time,stream_index : stream=index,codec_type </pre>

To show all the entries in the section "format", but only the codec type in the section "stream", specify the argument: <pre class=”example-preformatted”>format : stream=codec_type </pre>

To show all the tags in the stream and format sections: <pre class=”example-preformatted”>stream_tags : format_tags </pre>

To show only the <code class=”code”>title</code> tag (if available) in the stream sections: <pre class=”example-preformatted”>stream_tags=title </pre>

Selecting <code class=”code”>stream</code> or <code class=”code”>streams</code> does not by itself enable program or

stream group output. To also show streams within programs or groups, use

<code class=”code”>-show_programs</code>, <code class=”code”>-show_stream_groups</code>, or select their unique

section name, for example <code class=”code”>-show_entries program_stream=index</code> or

<code class=”code”>-show_entries stream_group_stream=index</code>. The shared names still select

fields within programs and stream groups when their output is enabled.

</dd>

<dt><samp class=”option”>-show_packets</samp></dt>

<dd> Show information about each packet contained in the input multimedia

stream.

The information for each single packet is printed within a dedicated

section with name "PACKET".

</dd>

<dt><samp class=”option”>-show_frames</samp></dt>

<dd> Show information about each frame and subtitle contained in the input

multimedia stream.

The information for each single frame is printed within a dedicated

section with name "FRAME" or "SUBTITLE".

</dd>

<dt><samp class=”option”>-show_log <var class=”var”>loglevel</var></samp></dt>

<dd> Show logging information from the decoder about each frame according to

the value set in <var class=”var”>loglevel</var>, (see <code class=”code”>-loglevel</code>). This option requires <code class=”code”>-show_frames</code>.

The information for each log message is printed within a dedicated

section with name "LOG".

</dd>

<dt><samp class=”option”>-show_streams</samp></dt>

<dd> Show information about each media stream contained in the input

multimedia stream.

Each media stream information is printed within a dedicated section

with name "STREAM".

</dd>

<dt><samp class=”option”>-show_programs</samp></dt>

<dd> Show information about programs and their streams contained in the input

multimedia stream.

Each media stream information is printed within a dedicated section

with name "PROGRAM_STREAM".

</dd>

<dt><samp class=”option”>-show_stream_groups</samp></dt>

<dd> Show information about stream groups and their streams contained in the

input multimedia stream.

Each media stream information is printed within a dedicated section

with name "STREAM_GROUP_STREAM".

</dd>

<dt><samp class="option">-show_chapters</samp></dt>

<dd> Show information about chapters stored in the format.

Each chapter is printed within a dedicated section with name "CHAPTER".

</dd>

<dt><samp class="option">-count_frames</samp></dt>

<dd> Count the number of frames per stream and report it in the

corresponding stream section.

</dd>

<dt><samp class="option">-count_packets</samp></dt>

<dd> Count the number of packets per stream and report it in the

corresponding stream section.

</dd>

<dt><samp class="option">-read_intervals <var class="var">read_intervals</var></samp></dt>

<dd>

Read only the specified intervals. <var class="var">read_intervals</var> must be a

sequence of interval specifications separated by ",".

<code class="command">ffprobe</code> will seek to the interval starting point, and will

continue reading from that.

Each interval is specified by two optional parts, separated by "%".

The first part specifies the interval start position. It is
interpreted as an absolute position, or as a relative offset from the
current position if it is preceded by the "+" character. If this first
part is not specified, no seeking will be performed when reading this
interval.

The second part specifies the interval end position. It is interpreted
as an absolute position, or as a relative offset from the current
position if it is preceded by the "+" character. If the offset
specification starts with "#", it is interpreted as the number of
packets to read (not including the flushing packets) from the interval
start. If no second part is specified, the program will read until the
end of the input.

Note that seeking is not accurate, thus the actual interval start
point may be different from the specified position. Also, when an
interval duration is specified, the absolute end time will be computed
by adding the duration to the interval start point found by seeking
the file, rather than to the specified start value.

The formal syntax is given by:

<pre class="example-preformatted"><var class="var">INTERVAL</var>
::= [<var class="var">START</var>|+<var class="var">START_OFFSET</var>][%[<var
class="var">END</var>|+<var class="var">END_OFFSET</var>]]
<var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[,<var
class="var">INTERVALS</var>]
</pre>
A few examples follow. <ul class="itemize mark-bullet"> Seek
to time 10, read packets until 20 seconds after the found seek point, then seek

to position <code class=”code”>01:30</code> (1 minute and thirty seconds) and read packets until position <code class=”code”>01:45</code>. <pre class=”example-preformatted”>10%+20,01:30%01:45 </pre>

Read only 42 packets after seeking to position <code class=”code”>01:23</code>: <pre class=”example-preformatted”>01:23%+#42 </pre>

Read only the first 20 seconds from the start: <pre class=”example-preformatted”>%+20 </pre>

Read from the start until position <code class=”code”>02:30</code>: <pre class=”example-preformatted”>%02:30 </pre>

</dd> <dt><samp class=”option”>-show_private_data, -private</samp></dt> <dd> Show private data, that is data depending on the format of the particular shown element. This option is enabled by default, but you may need to disable it for specific uses, for example when creating XSD-compliant XML output. </dd> <dt><samp class=”option”>-show_program_version</samp></dt> <dd> Show information related to program version. Version information is printed within a section with name "PROGRAM_VERSION". </dd> <dt><samp class=”option”>-show_library_versions</samp></dt> <dd> Show information related to library versions. Version information for each library is printed within a section with name "LIBRARY_VERSION". </dd> <dt><samp class=”option”>-show_versions</samp></dt> <dd> Show information related to program and library versions. This is the equivalent of setting both <samp class=”option”>-show_program_version</samp> and <samp class=”option”>-show_library_versions</samp> options. </dd> <dt><samp class=”option”>-show_pixel_formats</samp></dt> <dd> Show information about all pixel formats supported by FFmpeg. Pixel format information for each format is printed within a section with name "PIXEL_FORMAT". </dd> <dt><samp class=”option”>-show_optional_fields <var class=”var”>value</var></samp></dt> <dd> Some writers viz. JSON and XML, omit the printing of fields with invalid or non-applicable values, while other writers always print them. This option enables one to control this behaviour. Valid values are <code class=”code”>always</code>/<code class=”code”>1</code>, <code class=”code”>never</code>/<code class=”code”>0</code> and <code class=”code”>auto</code>/<code class=”code”>-1</code>. Default is <var class=”var”>auto</var>. </dd> <dt><samp class=”option”>-analyze_frames</samp></dt> <dd> Analyze frames and/or their side data up to the provided read interval, providing additional information that may be useful at a stream level. Must be paired with the <samp class=”option”>-show_streams</samp> option or it will have no effect.

 Currently, the additional fields provided by this option when enabled are the <code class="code">closed_captions</code> and <code class="code">film_grain</code> fields. For example, to analyze the first 20 seconds and populate these fields: <pre class="example-preformatted">ffprobe -show_streams -analyze_frames -read_intervals "%+20" INPUT </pre>

</dd> <dt><samp class="option">-bitexact</samp></dt> <dd> Force bitexact output, useful to produce output which is not dependent on the specific build. </dd> <dt><samp class="option">-i <var class="var">input_url</var></samp></dt> <dd> Read <var class="var">input_url</var>. </dd> <dt><samp class="option">-o <var class="var">output_url</var></samp></dt> <dd> Write output to <var class="var">output_url</var>. If not specified, the output is sent to stdout. </dd> <dt><samp class="option">-c:<var class="var">media_specifier</var> <var class="var">codec_name</var></samp></dt> <dt><samp class="option">-codec:<var class="var">media_specifier</var> <var class="var">codec_name</var></samp></dt> <dd> Force a specific decoder implementation for the stream identified by <var class="var">media_specifier</var>, which can assume the values <code class="code">a</code> (audio), <code class="code">v</code> (video), <code class="code">s</code> (subtitle), and <code class="code">d</code> (data). </dd> </dl>

 <h2 class="chapter">4 Writers# TOC</h2>

A writer defines the output format adopted by <code class="command">ffprobe</code>, and will be used for printing all the parts of the output. A writer may accept one or more arguments, which specify the options to adopt. The options are specified as a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ":". All writers support the following options: <dl class="table"> <dt><samp class="option">string_validation, sv</samp></dt> <dd> Set string validation mode. The following values are accepted. <dl class="table"> <dt>‘<samp class="samp">fail</samp>’</dt> <dd> The writer will fail immediately in case an invalid string (UTF-8) sequence or code point is found in the input. This is especially useful to validate input metadata. </dd> <dt>‘<samp class="samp">ignore</samp>’</dt> <dd> Any validation error will be ignored. This will result in possibly broken output, especially with the json or xml writer. </dd> <dt>‘<samp class="samp">replace</samp>’</dt> <dd> The writer will substitute invalid UTF-8 sequences or code points with the string specified with the <samp class="option">string_validation_replacement</samp>. </dd> </dl>

Default value is ‘<samp class="samp">replace</samp>’. </dd> <dt><samp class="option">string_validation_replacement, svr</samp></dt> <dd> Set replacement string to use in case <samp class="option">string_validation</samp> is set to ‘<samp class="samp">replace</samp>’. In case the option is not specified, the writer will assume the empty string, that is it will remove the invalid sequences from the input strings. </dd> </dl>

A description of the currently available writers follows. <h3 class="section">4.1 default# TOC</h3> Default format. Print each section in the form: <pre class="example-preformatted">[SECTION] key1=val1 ... keyN=valN [/SECTION] </pre>

Metadata tags are printed as a line in the corresponding FORMAT, STREAM, STREAM_GROUP_STREAM or PROGRAM_STREAM section, and are prefixed by the string "TAG:". A description of the accepted options follows. <dl class="table"> <dt><samp class="option">nokey, nk</samp></dt> <dd> If set to 1 specify not to print the key of each field. Default value is 0. </dd> <dt><samp class="option">noprint_wrappers, nw</samp></dt> <dd> If set to 1 specify not to print the section header and footer. Default value is 0. </dd> </dl>

 <h3 class="section">4.2 compact, csv# TOC</h3> Compact and CSV format. The <code class="code">csv</code> writer is equivalent to <code class="code">compact</code>, but supports different defaults. Each section is printed on a single line. If no option is specified, the output has the form: <pre class="example-preformatted">section|key1=val1| ... |keyN=valN </pre>

Metadata tags are printed in the corresponding "format" or "stream" section. A metadata tag key, if printed, is prefixed by the string "tag:". The description of the accepted options follows. <dl class="table"> <dt><samp class="option">item_sep, s</samp></dt> <dd> Specify the character to use for separating fields in the output line. It must be a single printable character, it is "|" by default ("," for the <code class="code">csv</code> writer). </dd> <dt><samp class="option">nokey, nk</samp></dt> <dd> If set to 1 specify not to print the key of each field. Its default value is 0 (1 for the <code class="code">csv</code> writer). </dd> <dt><samp class="option">escape, e</samp></dt> <dd> Set

the escape mode to use, default to "c" ("csv" for the <code class="code">csv</code> writer). It can assume one of the following values: <dl class="table"> <dt><samp class="option">c</samp></dt> <dd> Perform C-like escaping. Strings containing a newline (‘<samp class="samp">\n</samp>’), carriage return (‘<samp class="samp">\r</samp>’), a tab (‘<samp class="samp">\t</samp>’), a form feed (‘<samp class="samp">\f</samp>’), the escaping character (‘<samp class="samp">\</samp>’) or the item separator character <var class="var">SEP</var> are escaped using C-like fashioned escaping, so that a newline is converted to the sequence ‘<samp class="samp">\n</samp>’, a carriage return to ‘<samp class="samp">\r</samp>’, ‘<samp class="samp">\</samp>’ to ‘<samp class="samp">\\</samp>’ and the separator <var class="var">SEP</var> is converted to ‘<samp class="samp">\<var class="var">SEP</var></samp>’. </dd> <dt><samp class="option">csv</samp></dt> <dd> Perform CSV-like escaping, as described in RFC4180. Strings containing a newline (‘<samp class="samp">\n</samp>’), a carriage return (‘<samp class="samp">\r</samp>’), a double quote (‘<samp class="samp">"</samp>’), or <var class="var">SEP</var> are enclosed in double-quotes. </dd> <dt><samp class="option">none</samp></dt> <dd> Perform no escaping. </dd> </dl>

</dd> <dt><samp class="option">print_section, p</samp></dt> <dd> Print the section name at the beginning of each line if the value is <code class="code">1</code>, disable it with value set to <code class="code">0</code>. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">4.3 flat# TOC</h3> Flat format. A free-form output where each line contains an explicit key=value, such as "streams.stream.3.tags.foo=bar". The output is shell escaped, so it can be directly embedded in sh scripts as long as the separator character is an alphanumeric character or an underscore (see <var class="var">sep_char</var> option). The description of the accepted options follows. <dl class="table"> <dt><samp class="option">sep_char, s</samp></dt> <dd> Separator character used to separate the chapter, the section name, IDs and potential tags in the printed field key. Default value is ‘<samp class="samp">.</samp>’. </dd> <dt><samp class="option">hierarchical, h</samp></dt> <dd> Specify if the section name specification should be hierarchical. If set to 1, and if there is more than one section in the current chapter, the section name will be prefixed

by the name of the chapter. A value of 0 will disable this behavior.
Default value is 1. </dd> </dl>

 <h3 class="section">4.4 ini<span class="pull-
right">#
TOC</h3>
INI format output. Print output in an INI based format.
The following conventions are adopted: <ul class="itemize
mark-bullet"> all key and values are UTF-8 ‘<samp
class="samp">.</samp>’ is the subgroup separator newline,
‘<samp class="samp">\t</samp>’, ‘<samp class="samp">\f</samp>’,
‘<samp class="samp">\b</samp>’ and the following characters
are escaped ‘<samp class="samp">\</samp>’ is the
escape character ‘<samp class="samp">#</samp>’
is the comment indicator ‘<samp class="samp">=</samp>’
is the key/value separator ‘<samp class="samp">:</samp>’
is not used but usually parsed as key/value separator

This writer accepts options as a list of <var class="var">key</var>=<var
class="var">value</var> pairs, separated by ‘<samp class="samp">:</samp>’.
 The description of the accepted options follows. <dl
class="table"> <dt><samp class="option">hierarchical, h</samp></dt>
<dd> Specify if the section name specification should be hierarchical. If set
to 1, and if there is more than one section in the current chapter, the section
name will be prefixed by the name of the chapter. A value of 0 will disable this
behavior. Default value is 1. </dd> </dl>

 <h3 class="section">4.5 json<span class="pull-
right">#
TOC</h3>
JSON based format. Each section is printed using JSON no-
tation. The description of the accepted options follows.
<dl class="table"> <dt><samp class="option">compact, c</samp></dt>
<dd> If set to 1 enable compact output, that is each section will be printed
on a single line. Default value is 0. </dd> </dl>

For more information about JSON, see <a class="url" href="http://www.json.org/">http://www.json.org
  <h3 class="section">4.6 xml<span class="pull-
right">#
TOC</h3>
XML based format. The XML output is described in the
XML schema description file <samp class="file">ffprobe.xsd</samp> installed
in the FFmpeg datadir. An updated version of the schema can be re-
trieved at the url <a class="url" href="http://www.ffmpeg.org/schema/ffprobe.xsd">http://www.ffmpeg.org/
which redirects to the latest schema committed into the FFmpeg development
source code tree. Note that the output issued will be compli-
ant to the <samp class="file">ffprobe.xsd</samp> schema only when no
special global output options (<samp class="option">unit</samp>, <samp

class=”option”>prefix</samp>, <samp class=”option”>byte_binary_prefix</samp>, <samp class=”option”>sexagesimal</samp> etc.) are specified. The description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>fully_qualified, q</samp></dt> <dd> If set to 1 specify if the output should be fully qualified. Default value is 0. This is required for generating an XML file which can be validated through an XSD file. </dd> <dt><samp class=”option”>xsd_strict, x</samp></dt> <dd> If set to 1 perform more checks for ensuring that the output is XSD compliant. Default value is 0. This option automatically sets <samp class=”option”>fully_qualified</samp> to 1. </dd> </dl>

For more information about the XML format, see https://www.w3.org/XML/. <h2 class=”chapter”>5 Timecode# TOC</h2>

<code class=”command”>ffprobe</code> supports Timecode extraction: <ul class=”itemize mark-bullet”> MPEG1/2 timecode is extracted from the GOP, and is available in the video stream details (<samp class=”option”>-show_streams</samp>, see <var class=”var”>timecode</var>).

MOV timecode is extracted from tmcd track, so is available in the tmcd stream metadata (<samp class=”option”>-show_streams</samp>, see <var class=”var”>TAG:timecode</var>).

DV, GXF and AVI timecodes are available in format metadata (<samp class=”option”>-show_format</samp>, see <var class=”var”>TAG:timecode</var>).

 <h2 class=”chapter”>6 Syntax# TOC</h2>

This section documents the syntax and formats employed by the FFmpeg libraries and tools. <h3 class=”section”>6.1 Quoting and escaping# TOC</h3>

FFmpeg adopts the following quoting and escaping mechanism, unless explicitly specified. The following rules are applied: <ul class=”itemize mark-bullet”> ‘<samp class=”samp”>'</samp>’ and ‘<samp class=”samp”>\</samp>’ are special characters (respectively used for quoting and escaping). In addition to them, there might be other special characters depending on the specific syntax where the escaping and quoting are employed.

A special character is escaped by prefixing it with a ‘<samp class="samp">\</samp>’.

All characters enclosed between ‘<samp class="samp">''</samp>’ are included literally in the parsed string. The quote character ‘<samp class="samp">'</samp>’ itself cannot be quoted, so you may need to close the quote and escape it.

Leading and trailing whitespaces, unless escaped or quoted, are removed from the parsed string.

Note that you may need to add a second level of escaping when using the command line or a script, which depends on the syntax of the adopted shell language. The function <code class="code">av_get_token</code> defined in <samp class="file">libavutil/avstring.h</samp> can be used to parse a token quoted or escaped according to the rules defined above. The tool <samp class="file">tools/ffescape</samp> in the FFmpeg source tree can be used to automatically quote or escape a string in a script. <h4 class="subsection">6.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Escape the string <code class="code">Crime d'Amour</code> containing the <code class="code">'</code> special character: <pre class="example-preformatted">Crime d\'Amour </pre>

The string above contains a quote, so the <code class="code">'</code> needs to be escaped when quoting it: <pre class="example-preformatted">'Crime d'\''Amour' </pre>

Include leading or trailing whitespaces using quoting: <pre class="example-preformatted">' this string starts and ends with whitespaces ' </pre>

Escaping and quoting can be mixed together: <pre class="example-preformatted">' The string '\'string\'' is a string ' </pre>

To include a literal ‘<samp class="samp">\</samp>’ you can use either escaping or quoting: <pre class="example-preformatted">'c:\foo' can be written as c:\\foo </pre>

 <h3 class="section">6.2 Date# TOC</h3>

The accepted syntax is: <pre class=”example-preformatted”>[(YYYY-MM-DD|YYYYMMDD)[T|t|]](HH:MM:SS[.m...]]])|(HHMMSS[.m...]]]))[Z]
now </pre>

If the value is "now" it takes the current time. Time is local time unless Z is appended, in which case it is interpreted as UTC. If the year-month-day part is not specified it takes the current year-month-day. <h3 class=”section”>6.3 Time duration# TOC</h3>

There are two accepted syntaxes for expressing time duration.   <pre class=”example-preformatted”>[-][<var class=”var”>HH</var>:]<var class=”var”>MM</var>:<var class=”var”>SS</var>[.<var class=”var”>m</var>...] </pre>

<var class=”var”>HH</var> expresses the number of hours, <var class=”var”>MM</var> the number of minutes for a maximum of 2 digits, and <var class=”var”>SS</var> the number of seconds for a maximum of 2 digits. The <var class=”var”>m</var> at the end expresses decimal value for <var class=”var”>SS</var>. or   <pre class=”example-preformatted”>[-]<var class=”var”>S</var>+[.<var class=”var”>m</var>...][s|ms|us]
</pre>

<var class=”var”>S</var> expresses the number of seconds, with the optional decimal part <var class=”var”>m</var>. The optional literal suffixes ‘<samp class=”samp”>s</samp>’, ‘<samp class=”samp”>ms</samp>’ or ‘<samp class=”samp”>us</samp>’ indicate to interpret the value as seconds, milliseconds or microseconds, respectively. In both expressions, the optional ‘<samp class=”samp”>-</samp>’ indicates negative duration. <h4 class=”subsection”>6.3.1 Examples# TOC</h4>

The following examples are all valid time duration: <dl class=”table”> <dt>‘<samp class=”samp”>55</samp>’</dt>
<dd> 55 seconds </dd> <dt>‘<samp class=”samp”>0.2</samp>’</dt>
<dd> 0.2 seconds </dd> <dt>‘<samp class=”samp”>200ms</samp>’</dt>
<dd> 200 milliseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>200000us</samp>’</dt> <dd> 200000 microseconds, that’s 0.2s </dd> <dt>‘<samp class=”samp”>12:03:45</samp>’</dt>
<dd> 12 hours, 03 minutes and 45 seconds </dd> <dt>‘<samp class=”samp”>23.189</samp>’</dt> <dd> 23.189 seconds

</dd> </dl>

<h3 class="section">6.4 Video size# TOC</h3>

Specify the size of the sourced video, it may be a string of the form

<var class="var">width</var>x<var class="var">height</var>, or the name of a size abbreviation.

The following abbreviations are recognized:

<dl class="table">

<dt>‘<samp class="samp">ntsc</samp>’</dt>

<dd> 720x480

</dd>

<dt>‘<samp class="samp">pal</samp>’</dt>

<dd> 720x576

</dd>

<dt>‘<samp class="samp">qntsc</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class="samp">qpal</samp>’</dt>

<dd> 352x288

</dd>

<dt>‘<samp class="samp">sntsc</samp>’</dt>

<dd> 640x480

</dd>

<dt>‘<samp class="samp">spal</samp>’</dt>

<dd> 768x576

</dd>

<dt>‘<samp class="samp">film</samp>’</dt>

<dd> 352x240

</dd>

<dt>‘<samp class="samp">ntsc-film</samp>’</dt>
<dd> 352x240
</dd>
<dt>‘<samp class="samp">sqcif</samp>’</dt>
<dd> 128x96
</dd>
<dt>‘<samp class="samp">qcif</samp>’</dt>
<dd> 176x144
</dd>
<dt>‘<samp class="samp">cif</samp>’</dt>
<dd> 352x288
</dd>
<dt>‘<samp class="samp">4cif</samp>’</dt>
<dd> 704x576
</dd>
<dt>‘<samp class="samp">16cif</samp>’</dt>
<dd> 1408x1152
</dd>
<dt>‘<samp class="samp">qqvga</samp>’</dt>
<dd> 160x120
</dd>
<dt>‘<samp class="samp">qvga</samp>’</dt>
<dd> 320x240
</dd>
<dt>‘<samp class="samp">vga</samp>’</dt>
<dd> 640x480
</dd>
<dt>‘<samp class="samp">svga</samp>’</dt>
<dd> 800x600
</dd>
<dt>‘<samp class="samp">xga</samp>’</dt>

<dd> 1024x768
</dd>
<dt>‘<samp class="samp">uxga</samp>’</dt>
<dd> 1600x1200
</dd>
<dt>‘<samp class="samp">qxga</samp>’</dt>
<dd> 2048x1536
</dd>
<dt>‘<samp class="samp">sxga</samp>’</dt>
<dd> 1280x1024
</dd>
<dt>‘<samp class="samp">qsxga</samp>’</dt>
<dd> 2560x2048
</dd>
<dt>‘<samp class="samp">hsxga</samp>’</dt>
<dd> 5120x4096
</dd>
<dt>‘<samp class="samp">wvga</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">wxga</samp>’</dt>
<dd> 1366x768
</dd>
<dt>‘<samp class="samp">wsxga</samp>’</dt>
<dd> 1600x1024
</dd>
<dt>‘<samp class="samp">wuxga</samp>’</dt>
<dd> 1920x1200
</dd>
<dt>‘<samp class="samp">woxga</samp>’</dt>
<dd> 2560x1600

</dd>
<dt>‘<samp class="samp">wqsxga</samp>’</dt>
<dd> 3200x2048
</dd>
<dt>‘<samp class="samp">wquxga</samp>’</dt>
<dd> 3840x2400
</dd>
<dt>‘<samp class="samp">whsxga</samp>’</dt>
<dd> 6400x4096
</dd>
<dt>‘<samp class="samp">whuxga</samp>’</dt>
<dd> 7680x4800
</dd>
<dt>‘<samp class="samp">cga</samp>’</dt>
<dd> 320x200
</dd>
<dt>‘<samp class="samp">ega</samp>’</dt>
<dd> 640x350
</dd>
<dt>‘<samp class="samp">hd480</samp>’</dt>
<dd> 852x480
</dd>
<dt>‘<samp class="samp">hd720</samp>’</dt>
<dd> 1280x720
</dd>
<dt>‘<samp class="samp">hd1080</samp>’</dt>
<dd> 1920x1080
</dd>
<dt>‘<samp class="samp">2k</samp>’</dt>
<dd> 2048x1080
</dd>

<dt>‘<samp class=”samp”>2kflat</samp>’</dt>
<dd> 1998x1080
</dd>
<dt>‘<samp class=”samp”>2kscope</samp>’</dt>
<dd> 2048x858
</dd>
<dt>‘<samp class=”samp”>4k</samp>’</dt>
<dd> 4096x2160
</dd>
<dt>‘<samp class=”samp”>4kflat</samp>’</dt>
<dd> 3996x2160
</dd>
<dt>‘<samp class=”samp”>4kscope</samp>’</dt>
<dd> 4096x1716
</dd>
<dt>‘<samp class=”samp”>nhd</samp>’</dt>
<dd> 640x360
</dd>
<dt>‘<samp class=”samp”>hqvga</samp>’</dt>
<dd> 240x160
</dd>
<dt>‘<samp class=”samp”>wqvga</samp>’</dt>
<dd> 400x240
</dd>
<dt>‘<samp class=”samp”>fwqvga</samp>’</dt>
<dd> 432x240
</dd>
<dt>‘<samp class=”samp”>hvga</samp>’</dt>
<dd> 480x320
</dd>
<dt>‘<samp class=”samp”>qhd</samp>’</dt>

<dd> 960x540

</dd>

<dt>‘<samp class="samp">2kdci</samp>’</dt>

<dd> 2048x1080

</dd>

<dt>‘<samp class="samp">4kdci</samp>’</dt>

<dd> 4096x2160

</dd>

<dt>‘<samp class="samp">uhd2160</samp>’</dt>

<dd> 3840x2160

</dd>

<dt>‘<samp class="samp">uhd4320</samp>’</dt>

<dd> 7680x4320

</dd>

</dl>

<h3 class="section">6.5 Video rate# TOC</h3>

Specify the frame rate of a video, expressed as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a float number or a valid video frame rate abbreviation. The following abbreviations are recognized: <dl class="table"> <dt>‘<samp class="samp">ntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">pal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">qntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">qpal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">sntsc</samp>’</dt>
<dd> 30000/1001 </dd> <dt>‘<samp class="samp">spal</samp>’</dt>
<dd> 25/1 </dd> <dt>‘<samp class="samp">film</samp>’</dt>
<dd> 24/1 </dd> <dt>‘<samp class="samp">ntsc-film</samp>’</dt> <dd> 24000/1001 </dd> </dl>

 <h3 class="section">6.6 Ratio# TOC</h3>

A ratio can be expressed as an expression, or in the form <var class=”var”>numerator</var>:<var class=”var”>denominator</var>. Note that a ratio with infinite (1/0) or negative value is considered valid, so you should check on the returned value if you want to exclude those values. The undefined value can be expressed using the "0:0" string. <h3 class=”section”>6.7 Color# TOC</h3>

It can be the name of a color as defined below (case insensitive match) or a

<code class=”code”>[0x|#]RRGGBB[AA]</code> sequence, possibly followed by @ and a string

representing the alpha component.

The alpha component may be a string composed by "0x" followed by an

hexadecimal number or a decimal number between 0.0 and 1.0, which

represents the opacity value (‘<samp class=”samp”>0x00</samp>’ or ‘<samp class=”samp”>0.0</samp>’ means completely

transparent, ‘<samp class=”samp”>0xff</samp>’ or ‘<samp class=”samp”>1.0</samp>’ completely opaque). If the alpha

component is not specified then ‘<samp class=”samp”>0xff</samp>’ is assumed.

The string ‘<samp class=”samp”>random</samp>’ will result in a random color.

The following names of colors are recognized:

<dl class=”table”>

<dt>‘<samp class=”samp”>AliceBlue</samp>’</dt>

<dd> 0xF0F8FF

</dd>

<dt>‘<samp class=”samp”>AntiqueWhite</samp>’</dt>

<dd> 0xFAEBD7

</dd>

<dt>‘<samp class=”samp”>Aqua</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class=”samp”>Aquamarine</samp>’</dt>
<dd> 0x7FFFD4
</dd>
<dt>‘<samp class=”samp”>Azure</samp>’</dt>
<dd> 0xF0FFFF
</dd>
<dt>‘<samp class=”samp”>Beige</samp>’</dt>
<dd> 0xF5F5DC
</dd>
<dt>‘<samp class=”samp”>Bisque</samp>’</dt>
<dd> 0xFFE4C4
</dd>
<dt>‘<samp class=”samp”>Black</samp>’</dt>
<dd> 0x000000
</dd>
<dt>‘<samp class=”samp”>BlanchedAlmond</samp>’</dt>
<dd> 0xFFEBCD
</dd>
<dt>‘<samp class=”samp”>Blue</samp>’</dt>
<dd> 0x0000FF
</dd>
<dt>‘<samp class=”samp”>BlueViolet</samp>’</dt>
<dd> 0x8A2BE2
</dd>
<dt>‘<samp class=”samp”>Brown</samp>’</dt>
<dd> 0xA52A2A
</dd>
<dt>‘<samp class=”samp”>BurlyWood</samp>’</dt>

<dd> 0xDEB887
</dd>
<dt>‘<samp class=”samp”>CadetBlue</samp>’</dt>
<dd> 0x5F9EA0
</dd>
<dt>‘<samp class=”samp”>Chartreuse</samp>’</dt>
<dd> 0x7FFF00
</dd>
<dt>‘<samp class=”samp”>Chocolate</samp>’</dt>
<dd> 0xD2691E
</dd>
<dt>‘<samp class=”samp”>Coral</samp>’</dt>
<dd> 0xFF7F50
</dd>
<dt>‘<samp class=”samp”>CornflowerBlue</samp>’</dt>
<dd> 0x6495ED
</dd>
<dt>‘<samp class=”samp”>Cornsilk</samp>’</dt>
<dd> 0xFFF8DC
</dd>
<dt>‘<samp class=”samp”>Crimson</samp>’</dt>
<dd> 0xDC143C
</dd>
<dt>‘<samp class=”samp”>Cyan</samp>’</dt>
<dd> 0x00FFFF
</dd>
<dt>‘<samp class=”samp”>DarkBlue</samp>’</dt>
<dd> 0x00008B
</dd>
<dt>‘<samp class=”samp”>DarkCyan</samp>’</dt>
<dd> 0x008B8B

</dd>
<dt>‘<samp class=”samp”>DarkGoldenRod</samp>’</dt>
<dd> 0xB8860B
</dd>
<dt>‘<samp class=”samp”>DarkGray</samp>’</dt>
<dd> 0xA9A9A9
</dd>
<dt>‘<samp class=”samp”>DarkGreen</samp>’</dt>
<dd> 0x006400
</dd>
<dt>‘<samp class=”samp”>DarkKhaki</samp>’</dt>
<dd> 0xBDB76B
</dd>
<dt>‘<samp class=”samp”>DarkMagenta</samp>’</dt>
<dd> 0x8B008B
</dd>
<dt>‘<samp class=”samp”>DarkOliveGreen</samp>’</dt>
<dd> 0x556B2F
</dd>
<dt>‘<samp class=”samp”>Darkorange</samp>’</dt>
<dd> 0xFF8C00
</dd>
<dt>‘<samp class=”samp”>DarkOrchid</samp>’</dt>
<dd> 0x9932CC
</dd>
<dt>‘<samp class=”samp”>DarkRed</samp>’</dt>
<dd> 0x8B0000
</dd>
<dt>‘<samp class=”samp”>DarkSalmon</samp>’</dt>
<dd> 0xE9967A
</dd>

<dt>‘<samp class=”samp”>DarkSeaGreen</samp>’</dt>
<dd> 0x8FBC8F
</dd>
<dt>‘<samp class=”samp”>DarkSlateBlue</samp>’</dt>
<dd> 0x483D8B
</dd>
<dt>‘<samp class=”samp”>DarkSlateGray</samp>’</dt>
<dd> 0x2F4F4F
</dd>
<dt>‘<samp class=”samp”>DarkTurquoise</samp>’</dt>
<dd> 0x00CED1
</dd>
<dt>‘<samp class=”samp”>DarkViolet</samp>’</dt>
<dd> 0x9400D3
</dd>
<dt>‘<samp class=”samp”>DeepPink</samp>’</dt>
<dd> 0xFF1493
</dd>
<dt>‘<samp class=”samp”>DeepSkyBlue</samp>’</dt>
<dd> 0x00BFFF
</dd>
<dt>‘<samp class=”samp”>DimGray</samp>’</dt>
<dd> 0x696969
</dd>
<dt>‘<samp class=”samp”>DodgerBlue</samp>’</dt>
<dd> 0x1E90FF
</dd>
<dt>‘<samp class=”samp”>FireBrick</samp>’</dt>
<dd> 0xB22222
</dd>
<dt>‘<samp class=”samp”>FloralWhite</samp>’</dt>

<dd> 0xFFFAF0
</dd>
<dt>‘<samp class=”samp”>ForestGreen</samp>’</dt>
<dd> 0x228B22
</dd>
<dt>‘<samp class=”samp”>Fuchsia</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class=”samp”>Gainsboro</samp>’</dt>
<dd> 0xDCDCDC
</dd>
<dt>‘<samp class=”samp”>GhostWhite</samp>’</dt>
<dd> 0xF8F8FF
</dd>
<dt>‘<samp class=”samp”>Gold</samp>’</dt>
<dd> 0xFFD700
</dd>
<dt>‘<samp class=”samp”>GoldenRod</samp>’</dt>
<dd> 0xDAA520
</dd>
<dt>‘<samp class=”samp”>Gray</samp>’</dt>
<dd> 0x808080
</dd>
<dt>‘<samp class=”samp”>Green</samp>’</dt>
<dd> 0x008000
</dd>
<dt>‘<samp class=”samp”>GreenYellow</samp>’</dt>
<dd> 0xADFF2F
</dd>
<dt>‘<samp class=”samp”>HoneyDew</samp>’</dt>
<dd> 0xF0FFF0

</dd>
<dt>‘<samp class="samp">HotPink</samp>’</dt>
<dd> 0xFF69B4
</dd>
<dt>‘<samp class="samp">IndianRed</samp>’</dt>
<dd> 0xCD5C5C
</dd>
<dt>‘<samp class="samp">Indigo</samp>’</dt>
<dd> 0x4B0082
</dd>
<dt>‘<samp class="samp">Ivory</samp>’</dt>
<dd> 0xFFFFF0
</dd>
<dt>‘<samp class="samp">Khaki</samp>’</dt>
<dd> 0xF0E68C
</dd>
<dt>‘<samp class="samp">Lavender</samp>’</dt>
<dd> 0xE6E6FA
</dd>
<dt>‘<samp class="samp">LavenderBlush</samp>’</dt>
<dd> 0xFFF0F5
</dd>
<dt>‘<samp class="samp">LawnGreen</samp>’</dt>
<dd> 0x7CFC00
</dd>
<dt>‘<samp class="samp">LemonChiffon</samp>’</dt>
<dd> 0xFFFACD
</dd>
<dt>‘<samp class="samp">LightBlue</samp>’</dt>
<dd> 0xADD8E6
</dd>

<dt>‘<samp class=”samp”>LightCoral</samp>’</dt>
<dd> 0xF08080
</dd>
<dt>‘<samp class=”samp”>LightCyan</samp>’</dt>
<dd> 0xE0FFFF
</dd>
<dt>‘<samp class=”samp”>LightGoldenRodYellow</samp>’</dt>
<dd> 0xFAFAD2
</dd>
<dt>‘<samp class=”samp”>LightGreen</samp>’</dt>
<dd> 0x90EE90
</dd>
<dt>‘<samp class=”samp”>LightGrey</samp>’</dt>
<dd> 0xD3D3D3
</dd>
<dt>‘<samp class=”samp”>LightPink</samp>’</dt>
<dd> 0xFFB6C1
</dd>
<dt>‘<samp class=”samp”>LightSalmon</samp>’</dt>
<dd> 0xFFA07A
</dd>
<dt>‘<samp class=”samp”>LightSeaGreen</samp>’</dt>
<dd> 0x20B2AA
</dd>
<dt>‘<samp class=”samp”>LightSkyBlue</samp>’</dt>
<dd> 0x87CEFA
</dd>
<dt>‘<samp class=”samp”>LightSlateGray</samp>’</dt>
<dd> 0x778899
</dd>
<dt>‘<samp class=”samp”>LightSteelBlue</samp>’</dt>

<dd> 0xB0C4DE
</dd>
<dt>‘<samp class=”samp”>LightYellow</samp>’</dt>
<dd> 0xFFFFE0
</dd>
<dt>‘<samp class=”samp”>Lime</samp>’</dt>
<dd> 0x00FF00
</dd>
<dt>‘<samp class=”samp”>LimeGreen</samp>’</dt>
<dd> 0x32CD32
</dd>
<dt>‘<samp class=”samp”>Linen</samp>’</dt>
<dd> 0xFAF0E6
</dd>
<dt>‘<samp class=”samp”>Magenta</samp>’</dt>
<dd> 0xFF00FF
</dd>
<dt>‘<samp class=”samp”>Maroon</samp>’</dt>
<dd> 0x800000
</dd>
<dt>‘<samp class=”samp”>MediumAquaMarine</samp>’</dt>
<dd> 0x66CDAA
</dd>
<dt>‘<samp class=”samp”>MediumBlue</samp>’</dt>
<dd> 0x0000CD
</dd>
<dt>‘<samp class=”samp”>MediumOrchid</samp>’</dt>
<dd> 0xBA55D3
</dd>
<dt>‘<samp class=”samp”>MediumPurple</samp>’</dt>
<dd> 0x9370D8

</dd>
<dt>‘<samp class=”samp”>MediumSeaGreen</samp>’</dt>
<dd> 0x3CB371
</dd>
<dt>‘<samp class=”samp”>MediumSlateBlue</samp>’</dt>
<dd> 0x7B68EE
</dd>
<dt>‘<samp class=”samp”>MediumSpringGreen</samp>’</dt>
<dd> 0x00FA9A
</dd>
<dt>‘<samp class=”samp”>MediumTurquoise</samp>’</dt>
<dd> 0x48D1CC
</dd>
<dt>‘<samp class=”samp”>MediumVioletRed</samp>’</dt>
<dd> 0xC71585
</dd>
<dt>‘<samp class=”samp”>MidnightBlue</samp>’</dt>
<dd> 0x191970
</dd>
<dt>‘<samp class=”samp”>MintCream</samp>’</dt>
<dd> 0xF5FFFA
</dd>
<dt>‘<samp class=”samp”>MistyRose</samp>’</dt>
<dd> 0xFFE4E1
</dd>
<dt>‘<samp class=”samp”>Moccasin</samp>’</dt>
<dd> 0xFFE4B5
</dd>
<dt>‘<samp class=”samp”>NavajoWhite</samp>’</dt>
<dd> 0xFFDEAD
</dd>

<dt>‘<samp class=”samp”>Navy</samp>’</dt>
<dd> 0x000080
</dd>
<dt>‘<samp class=”samp”>OldLace</samp>’</dt>
<dd> 0xFDF5E6
</dd>
<dt>‘<samp class=”samp”>Olive</samp>’</dt>
<dd> 0x808000
</dd>
<dt>‘<samp class=”samp”>OliveDrab</samp>’</dt>
<dd> 0x6B8E23
</dd>
<dt>‘<samp class=”samp”>Orange</samp>’</dt>
<dd> 0xFFA500
</dd>
<dt>‘<samp class=”samp”>OrangeRed</samp>’</dt>
<dd> 0xFF4500
</dd>
<dt>‘<samp class=”samp”>Orchid</samp>’</dt>
<dd> 0xDA70D6
</dd>
<dt>‘<samp class=”samp”>PaleGoldenRod</samp>’</dt>
<dd> 0xEEE8AA
</dd>
<dt>‘<samp class=”samp”>PaleGreen</samp>’</dt>
<dd> 0x98FB98
</dd>
<dt>‘<samp class=”samp”>PaleTurquoise</samp>’</dt>
<dd> 0xAFEEEE
</dd>
<dt>‘<samp class=”samp”>PaleVioletRed</samp>’</dt>

<dd> 0xD87093
</dd>
<dt>‘<samp class=”samp”>PapayaWhip</samp>’</dt>
<dd> 0xFFEFD5
</dd>
<dt>‘<samp class=”samp”>PeachPuff</samp>’</dt>
<dd> 0xFFDAB9
</dd>
<dt>‘<samp class=”samp”>Peru</samp>’</dt>
<dd> 0xCD853F
</dd>
<dt>‘<samp class=”samp”>Pink</samp>’</dt>
<dd> 0xFFC0CB
</dd>
<dt>‘<samp class=”samp”>Plum</samp>’</dt>
<dd> 0xDDA0DD
</dd>
<dt>‘<samp class=”samp”>PowderBlue</samp>’</dt>
<dd> 0xB0E0E6
</dd>
<dt>‘<samp class=”samp”>Purple</samp>’</dt>
<dd> 0x800080
</dd>
<dt>‘<samp class=”samp”>Red</samp>’</dt>
<dd> 0xFF0000
</dd>
<dt>‘<samp class=”samp”>RosyBrown</samp>’</dt>
<dd> 0xBC8F8F
</dd>
<dt>‘<samp class=”samp”>RoyalBlue</samp>’</dt>
<dd> 0x4169E1

</dd>
<dt>‘<samp class=”samp”>SaddleBrown</samp>’</dt>
<dd> 0x8B4513
</dd>
<dt>‘<samp class=”samp”>Salmon</samp>’</dt>
<dd> 0xFA8072
</dd>
<dt>‘<samp class=”samp”>SandyBrown</samp>’</dt>
<dd> 0xF4A460
</dd>
<dt>‘<samp class=”samp”>SeaGreen</samp>’</dt>
<dd> 0x2E8B57
</dd>
<dt>‘<samp class=”samp”>SeaShell</samp>’</dt>
<dd> 0xFFF5EE
</dd>
<dt>‘<samp class=”samp”>Sienna</samp>’</dt>
<dd> 0xA0522D
</dd>
<dt>‘<samp class=”samp”>Silver</samp>’</dt>
<dd> 0xC0C0C0
</dd>
<dt>‘<samp class=”samp”>SkyBlue</samp>’</dt>
<dd> 0x87CEEB
</dd>
<dt>‘<samp class=”samp”>SlateBlue</samp>’</dt>
<dd> 0x6A5ACD
</dd>
<dt>‘<samp class=”samp”>SlateGray</samp>’</dt>
<dd> 0x708090
</dd>

<dt>‘<samp class=”samp”>Snow</samp>’</dt>
<dd> 0xFFFAFA
</dd>
<dt>‘<samp class=”samp”>SpringGreen</samp>’</dt>
<dd> 0x00FF7F
</dd>
<dt>‘<samp class=”samp”>SteelBlue</samp>’</dt>
<dd> 0x4682B4
</dd>
<dt>‘<samp class=”samp”>Tan</samp>’</dt>
<dd> 0xD2B48C
</dd>
<dt>‘<samp class=”samp”>Teal</samp>’</dt>
<dd> 0x008080
</dd>
<dt>‘<samp class=”samp”>Thistle</samp>’</dt>
<dd> 0xD8BFD8
</dd>
<dt>‘<samp class=”samp”>Tomato</samp>’</dt>
<dd> 0xFF6347
</dd>
<dt>‘<samp class=”samp”>Turquoise</samp>’</dt>
<dd> 0x40E0D0
</dd>
<dt>‘<samp class=”samp”>Violet</samp>’</dt>
<dd> 0xEE82EE
</dd>
<dt>‘<samp class=”samp”>Wheat</samp>’</dt>
<dd> 0xF5DEB3
</dd>
<dt>‘<samp class=”samp”>White</samp>’</dt>

<dd> 0xFFFFFF

</dd>

<dt>‘<samp class="samp">WhiteSmoke</samp>’</dt>

<dd> 0xF5F5F5

</dd>

<dt>‘<samp class="samp">Yellow</samp>’</dt>

<dd> 0xFFFF00

</dd>

<dt>‘<samp class="samp">YellowGreen</samp>’</dt>

<dd> 0x9ACD32

</dd>

</dl>

 <h3 class="section">6.8 Channel Layout# TOC</h3>

A channel layout specifies the spatial disposition of the channels in a multi-channel audio stream. To specify a channel layout, FFmpeg makes use of a special syntax. Individual channels are identified by an id, as given by the table below: <dl class="table"> <dt>‘<samp class="samp">FL</samp>’</dt> <dd> front left </dd> <dt>‘<samp class="samp">FR</samp>’</dt> <dd> front right </dd> <dt>‘<samp class="samp">FC</samp>’</dt> <dd> front center </dd> <dt>‘<samp class="samp">LFE</samp>’</dt> <dd> low frequency </dd> <dt>‘<samp class="samp">BL</samp>’</dt> <dd> back left </dd> <dt>‘<samp class="samp">BR</samp>’</dt> <dd> back right </dd> <dt>‘<samp class="samp">FLC</samp>’</dt> <dd> front left-of-center </dd> <dt>‘<samp class="samp">FRC</samp>’</dt> <dd> front right-of-center </dd> <dt>‘<samp class="samp">BC</samp>’</dt> <dd> back center </dd> <dt>‘<samp class="samp">SL</samp>’</dt> <dd> side left </dd> <dt>‘<samp class="samp">SR</samp>’</dt> <dd> side right </dd> <dt>‘<samp class="samp">TC</samp>’</dt> <dd> top center </dd> <dt>‘<samp class="samp">TFL</samp>’</dt> <dd> top front left </dd> <dt>‘<samp class="samp">TFC</samp>’</dt> <dd> top front center </dd> <dt>‘<samp class="samp">TFR</samp>’</dt> <dd> top front right </dd> <dt>‘<samp class="samp">TBL</samp>’</dt> <dd> top back left </dd> <dt>‘<samp class="samp">TBC</samp>’</dt> <dd> top back center </dd> <dt>‘<samp class="samp">TBR</samp>’</dt>

<dd> top back right </dd> <dt>‘<samp class="samp">DL</samp>’</dt>
<dd> downmix left </dd> <dt>‘<samp class="samp">DR</samp>’</dt>
<dd> downmix right </dd> <dt>‘<samp class="samp">WL</samp>’</dt>
<dd> wide left </dd> <dt>‘<samp class="samp">WR</samp>’</dt>
<dd> wide right </dd> <dt>‘<samp class="samp">SDL</samp>’</dt>
<dd> surround direct left </dd> <dt>‘<samp class="samp">SDR</samp>’</dt>
<dd> surround direct right </dd> <dt>‘<samp class="samp">LFE2</samp>’</dt
<dd> low frequency 2 </dd> </dl>

Standard channel layout compositions can be specified by using the

following identifiers:

<dl class="table">

<dt>‘<samp class="samp">mono</samp>’</dt>

<dd> FC

</dd>

<dt>‘<samp class="samp">stereo</samp>’</dt>

<dd> FL+FR

</dd>

<dt>‘<samp class="samp">2.1</samp>’</dt>

<dd> FL+FR+LFE

</dd>

<dt>‘<samp class="samp">3.0</samp>’</dt>

<dd> FL+FR+FC

</dd>

<dt>‘<samp class="samp">3.0(back)</samp>’</dt>

<dd> FL+FR+BC

</dd>

<dt>‘<samp class="samp">4.0</samp>’</dt>

<dd> FL+FR+FC+BC

</dd>

<dt>‘<samp class="samp">quad</samp>’</dt>

<dd> FL+FR+BL+BR

</dd>

<dt>‘<samp class="samp">quad(side)</samp>’</dt>

<dd> FL+FR+SL+SR
</dd>
<dt>‘<samp class="samp">3.1</samp>’</dt>
<dd> FL+FR+FC+LFE
</dd>
<dt>‘<samp class="samp">5.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR
</dd>
<dt>‘<samp class="samp">5.0(side)</samp>’</dt>
<dd> FL+FR+FC+SL+SR
</dd>
<dt>‘<samp class="samp">4.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC
</dd>
<dt>‘<samp class="samp">5.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR
</dd>
<dt>‘<samp class="samp">5.1(side)</samp>’</dt>
<dd> FL+FR+FC+LFE+SL+SR
</dd>
<dt>‘<samp class="samp">6.0</samp>’</dt>
<dd> FL+FR+FC+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.0(front)</samp>’</dt>
<dd> FL+FR+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">3.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+TFL+TFR
</dd>
<dt>‘<samp class="samp">hexagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC

</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BC+SL+SR
</dd>
<dt>‘<samp class="samp">6.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+BC
</dd>
<dt>‘<samp class="samp">6.1(front)</samp>’</dt>
<dd> FL+FR+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.0</samp>’</dt>
<dd> FL+FR+FC+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.0(front)</samp>’</dt>
<dd> FL+FR+FC+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">7.1</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR
</dd>
<dt>‘<samp class="samp">7.1(wide)</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC
</dd>
<dt>‘<samp class="samp">7.1(wide-side)</samp>’</dt>
<dd> FL+FR+FC+LFE+FLC+FRC+SL+SR
</dd>
<dt>‘<samp class="samp">5.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR
</dd>
<dt>‘<samp class="samp">octagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR
</dd>

<dt>‘<samp class="samp">cube</samp>’</dt>
<dd> FL+FR+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">5.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.1.2</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR
</dd>
<dt>‘<samp class="samp">7.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">7.2.3</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+SL+SR+TFL+TFR+TBC+LFE2
</dd>
<dt>‘<samp class="samp">9.1.4</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR
</dd>
<dt>‘<samp class="samp">9.1.6</samp>’</dt>
<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+SL+SR+TFL+TFR+TBL+TBR+TSL+TSR
</dd>
<dt>‘<samp class="samp">hexadecagonal</samp>’</dt>
<dd> FL+FR+FC+BL+BR+BC+SL+SR+WL+WR+TBL+TBR+TBC+TFC+TFL+TFR
</dd>
<dt>‘<samp class="samp">binaural</samp>’</dt>
<dd> BIL+BIR
</dd>
<dt>‘<samp class="samp">downmix</samp>’</dt>
<dd> DL+DR
</dd>
<dt>‘<samp class="samp">22.2</samp>’</dt>

<dd> FL+FR+FC+LFE+BL+BR+FLC+FRC+BC+SL+SR+TC+TFL+TFC+TFR+TBL+TBC+TBR

</dd>

</dl>

A custom channel layout can be specified as a sequence of terms, separated by ’+’. Each term can be: <ul class="itemize mark-bullet"> the name of a single channel (e.g. ‘<samp class="samp">FL</samp>’, ‘<samp class="samp">FR</samp>’, ‘<samp class="samp">FC</samp>’, ‘<samp class="samp">LFE</samp>’, etc.), each optionally containing a custom name after a ’@’, (e.g. ‘<samp class="samp">FL@Left</samp>’, ‘<samp class="samp">FR@Right</samp>’, ‘<samp class="samp">FC@Center</samp>’, ‘<samp class="samp">LFE@Low_Frequency</samp>’, etc.)

A standard channel layout can be specified by the following: <ul class="itemize mark-bullet"> the name of a single channel (e.g. ‘<samp class="samp">FL</samp>’, ‘<samp class="samp">FR</samp>’, ‘<samp class="samp">FC</samp>’, ‘<samp class="samp">LFE</samp>’, etc.)

the name of a standard channel layout (e.g. ‘<samp class="samp">mono</samp>’, ‘<samp class="samp">stereo</samp>’, ‘<samp class="samp">4.0</samp>’, ‘<samp class="samp">quad</samp>’, ‘<samp class="samp">5.0</samp>’, etc.)

a number of channels, in decimal, followed by ’c’, yielding the default channel layout for that number of channels (see the function <code class="code">av_channel_layout_default</code>). Note that not all channel counts have a default layout.

a number of channels, in decimal, followed by ’C’, yielding an unknown channel layout with the specified number of channels. Note that not all channel layout specification strings support unknown channel layouts.

a channel layout mask, in hexadecimal starting with "0x" (see the <code class="code">AV_CH_*</code> macros in <samp class="file">libavutil/channel_layout.h</samp>.

Before libavutil version 53 the trailing character "c" to specify a number of channels was optional, but now it is required, while a channel layout mask can also be specified as a decimal number (if and only if not followed by "c" or "C"). See also the function <code class="code">av_channel_layout_from_string</code> defined in <samp class="file">libavutil/channel_layout.h</samp>. <h2 class="chapter">7 Expression Evaluation# <a

class=”anchor hidden-xs”href=”#toc-Expression-Evaluation” aria-hidden=”true”>TOC</h2>

When evaluating an arithmetic expression, FFmpeg uses an internal

formula evaluator, implemented through the <samp class=”file”>libavutil/eval.h</samp>

interface.

An expression may contain unary, binary operators, constants, and

functions.

Two expressions <var class=”var”>expr1</var> and <var class=”var”>expr2</var>
can be combined to form

another expression "<var class=”var”>expr1</var>;<var class=”var”>expr2</var>".

<var class=”var”>expr1</var> and <var class=”var”>expr2</var> are eval-
uated in turn, and the new

expression evaluates to the value of <var class=”var”>expr2</var>.

The following binary operators are available: <code class=”code”>+</code>,
<code class=”code”>-</code>,

<code class=”code”>*</code>, <code class=”code”>/</code>, <code
class=”code”>^</code>.

The following unary operators are available: <code class=”code”>+</code>,
<code class=”code”>-</code>.

Some internal variables can be used to store and load intermediary

results. They can be accessed using the <code class=”code”>ld</code> and
<code class=”code”>st</code>

functions with an index argument varying from 0 to 9 to specify which

internal variable to access.

The following functions are available:

<dl class=”table”>

<dt><samp class=”option”>abs(x)</samp></dt>

<dd> Compute absolute value of <var class=”var”>x</var>.

</dd>

<dt><samp class="option">acos(x)</samp></dt>

<dd> Compute arccosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">asin(x)</samp></dt>

<dd> Compute arcsine of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan(x)</samp></dt>

<dd> Compute arctangent of <var class="var">x</var>.

</dd>

<dt><samp class="option">atan2(y, x)</samp></dt>

<dd> Compute principal value of the arc tangent of <var class="var">y</var>/<var class="var">x</var>.

</dd>

<dt><samp class="option">between(x, min, max)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is greater than or equal to <var class="var">min</var> and lesser than or

equal to <var class="var">max</var>, 0 otherwise.

</dd>

<dt><samp class="option">bitand(x, y)</samp></dt>

<dt><samp class="option">bitor(x, y)</samp></dt>

<dd> Compute bitwise and/or operation on <var class="var">x</var> and <var class="var">y</var>.

The results of the evaluation of <var class="var">x</var> and <var class="var">y</var> are converted to

integers before executing the bitwise operation.

Note that both the conversion to integer and the conversion back to

floating point can lose precision. Beware of unexpected results for

large numbers (usually 2^53 and larger).

</dd>

<dt><samp class="option">ceil(expr)</samp></dt>

<dd> Round the value of expression <var class="var">expr</var> upwards to the nearest

integer. For example, "ceil(1.5)" is "2.0".

</dd>

<dt><samp class="option">clip(x, min, max)</samp></dt>

<dd> Return the value of <var class="var">x</var> clipped between <var class="var">min</var> and <var class="var">max</var>.

</dd>

<dt><samp class="option">cos(x)</samp></dt>

<dd> Compute cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">cosh(x)</samp></dt>

<dd> Compute hyperbolic cosine of <var class="var">x</var>.

</dd>

<dt><samp class="option">eq(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> and <var class="var">y</var> are equivalent, 0 otherwise.

</dd>

<dt><samp class="option">exp(x)</samp></dt>

<dd> Compute exponential of <var class=”var”>x</var> (with base <code class=”code”>e</code>, the Euler’s number).

</dd>

<dt><samp class=”option”>floor(expr)</samp></dt>

<dd> Round the value of expression <var class=”var”>expr</var> downwards to the nearest

integer. For example, "floor(-1.5)" is "-2.0".

</dd>

<dt><samp class=”option”>gauss(x)</samp></dt>

<dd> Compute Gauss function of <var class=”var”>x</var>, corresponding to

<code class=”code”>exp(-x*x/2) / sqrt(2*PI)</code>.

</dd>

<dt><samp class=”option”>gcd(x, y)</samp></dt>

<dd> Return the greatest common divisor of <var class=”var”>x</var> and <var class=”var”>y</var>. If both <var class=”var”>x</var> and

<var class=”var”>y</var> are 0 or either or both are less than zero then behavior is undefined.

</dd>

<dt><samp class=”option”>gt(x, y)</samp></dt>

<dd> Return 1 if <var class=”var”>x</var> is greater than <var class=”var”>y</var>, 0 otherwise.

</dd>

<dt><samp class=”option”>gte(x, y)</samp></dt>

<dd> Return 1 if <var class=”var”>x</var> is greater than or equal to <var class=”var”>y</var>, 0 otherwise.

</dd>

<dt><samp class=”option”>hypot(x, y)</samp></dt>

<dd> This function is similar to the C function with the same name; it returns

"sqrt(<var class=”var”>x</var>*<var class=”var”>x</var> + <var class=”var”>y</var>*<var class=”var”>y</var>)", the length of the hypotenuse of a

right triangle with sides of length <var class=”var”>x</var> and <var class=”var”>y</var>, or the distance of the

point (<var class=”var”>x</var>, <var class=”var”>y</var>) from the origin.

</dd>

<dt><samp class=”option”>if(x, y)</samp></dt>

<dd> Evaluate <var class=”var”>x</var>, and if the result is non-zero return the result of

the evaluation of <var class=”var”>y</var>, return 0 otherwise.

</dd>

<dt><samp class=”option”>if(x, y, z)</samp></dt>

<dd> Evaluate <var class=”var”>x</var>, and if the result is non-zero return the evaluation

result of <var class=”var”>y</var>, otherwise the evaluation result of <var class=”var”>z</var>.

</dd>

<dt><samp class=”option”>ifnot(x, y)</samp></dt>

<dd> Evaluate <var class=”var”>x</var>, and if the result is zero return the result of the

evaluation of <var class=”var”>y</var>, return 0 otherwise.

</dd>

<dt><samp class=”option”>ifnot(x, y, z)</samp></dt>

<dd> Evaluate <var class=”var”>x</var>, and if the result is zero return the evaluation

result of <var class="var">y</var>, otherwise the evaluation result of <var class="var">z</var>.

</dd>

<dt><samp class="option">isinf(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is +/-INFINITY, 0.0 otherwise.

</dd>

<dt><samp class="option">isnan(x)</samp></dt>

<dd> Return 1.0 if <var class="var">x</var> is NAN, 0.0 otherwise.

</dd>

<dt><samp class="option">ld(idx)</samp></dt>

<dd> Load the value of the internal variable with index <var class="var">idx</var>, which was

previously stored with st(<var class="var">idx</var>, <var class="var">expr</var>).

The function returns the loaded value.

</dd>

<dt><samp class="option">lerp(x, y, z)</samp></dt>

<dd> Return linear interpolation between <var class="var">x</var> and <var class="var">y</var> by amount of <var class="var">z</var>.

</dd>

<dt><samp class="option">log(x)</samp></dt>

<dd> Compute natural logarithm of <var class="var">x</var>.

</dd>

<dt><samp class="option">lt(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">lte(x, y)</samp></dt>

<dd> Return 1 if <var class="var">x</var> is lesser than or equal to <var class="var">y</var>, 0 otherwise.

</dd>

<dt><samp class="option">max(x, y)</samp></dt>

<dd> Return the maximum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">min(x, y)</samp></dt>

<dd> Return the minimum between <var class="var">x</var> and <var class="var">y</var>.

</dd>

<dt><samp class="option">mod(x, y)</samp></dt>

<dd> Compute the remainder of division of <var class="var">x</var> by <var class="var">y</var>.

</dd>

<dt><samp class="option">not(expr)</samp></dt>

<dd> Return 1.0 if <var class="var">expr</var> is zero, 0.0 otherwise.

</dd>

<dt><samp class="option">pow(x, y)</samp></dt>

<dd> Compute the power of <var class="var">x</var> elevated <var class="var">y</var>, it is equivalent to

"(<var class="var">x</var>)^(<var class="var">y</var>)".

</dd>

<dt><samp class="option">print(t)</samp></dt>

<dt><samp class="option">print(t, l)</samp></dt>

<dd> Print the value of expression <var class="var">t</var> with loglevel <var class="var">l</var>. If <var class="var">l</var> is not

specified then a default log level is used.

Return the value of the expression printed.

</dd>

<dt><samp class="option">random(idx)</samp></dt>

<dd> Return a pseudo random value between 0.0 and 1.0. <var class="var">idx</var> is the

index of the internal variable used to save the seed/state, which can be

previously stored with <code class="code">st(idx)</code>.

To initialize the seed, you need to store the seed value as a 64-bit

unsigned integer in the internal variable with index <var class="var">idx</var>.

For example, to store the seed with value <code class="code">42</code> in the internal

variable with index <code class="code">0</code> and print a few random values:

<pre class="example-preformatted">st(0,42); print(random(0)); print(random(0)); print(random(0))

</pre>

</dd> <dt><samp class="option">randomi(idx, min, max)</samp></dt> <dd> Return a pseudo random value in the interval between <var class="var">min</var> and <var class="var">max</var>. <var class="var">idx</var> is the index of the internal variable which will be used to save the seed/state, which can be previously stored with <code class="code">st(idx)</code>. To initialize the seed, you need to store the seed value as a 64-bit unsigned integer in the internal variable with index <var class="var">idx</var>. </dd> <dt><samp class="option">root(expr, max)</samp></dt> <dd> Find an input value for which the function represented by <var class="var">expr</var> with argument <var class="var">ld(0)</var> is 0

in the interval 0..<var class="var">max</var>. The expression in <var class="var">expr</var> must denote a continuous function or the result is undefined. <var class="var">ld(0)</var> is used to represent the function input value, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class="code">ld(0)</code>. When the expression evaluates to 0 then the corresponding input value will be returned. </dd> <dt><samp class="option">round(expr)</samp></dt> <dd> Round the value of expression <var class="var">expr</var> to the nearest integer. For example, "round(1.5)" is "2.0". </dd> <dt><samp class="option">sgn(x)</samp></dt> <dd> Compute sign of <var class="var">x</var>. </dd> <dt><samp class="option">sin(x)</samp></dt> <dd> Compute sine of <var class="var">x</var>. </dd> <dt><samp class="option">sinh(x)</samp></dt> <dd> Compute hyperbolic sine of <var class="var">x</var>. </dd> <dt><samp class="option">sqrt(expr)</samp></dt> <dd> Compute the square root of <var class="var">expr</var>. This is equivalent to "(<var class="var">expr</var>)^.5". </dd> <dt><samp class="option">squish(x)</samp></dt> <dd> Compute expression <code class="code">1/(1 + exp(4*x))</code>. </dd> <dt><samp class="option">st(idx, expr)</samp></dt> <dd> Store the value of the expression <var class="var">expr</var> in an internal variable. <var class="var">idx</var> specifies the index of the variable where to store the value, and it is a value ranging from 0 to 9. The function returns the value stored in the internal variable. The stored value can be retrieved with <code class="code">ld(var)</code>. Note: variables are currently not shared between expressions. </dd> <dt><samp class="option">tan(x)</samp></dt> <dd> Compute tangent of <var class="var">x</var>. </dd> <dt><samp class="option">tanh(x)</samp></dt> <dd> Compute hyperbolic tangent of <var class="var">x</var>. </dd> <dt><samp class="option">taylor(expr, x)</samp></dt> <dt><samp class="option">taylor(expr, x, idx)</samp></dt> <dd> Evaluate a Taylor series at <var class="var">x</var>, given an expression representing the <code class="code">ld(idx)</code>-th derivative of a function at 0. When the series does not converge the result is undefined. <var class="var">ld(idx)</var> is used to represent the derivative order in <var class="var">expr</var>, which means that the given expression will be evaluated multiple times with various input values that the expression can access through <code class="code">ld(idx)</code>. If <var class="var">idx</var> is not specified then 0 is assumed. Note, when you have the derivatives at y instead of 0, <code class="code">taylor(expr, x-y)</code> can be used. </dd> <dt><samp class="option">time(0)</samp></dt> <dd> Return the current (wallclock) time in seconds. </dd> <dt><samp class="option">trunc(expr)</samp></dt> <dd> Round the value of expression <var class="var">expr</var> towards zero to the nearest integer. For example, "trunc(-1.5)" is "-1.0".

 </dd> <dt><samp class=”option”>while(cond, expr)</samp></dt> <dd> Evaluate expression <var class=”var”>expr</var> while the expression <var class=”var”>cond</var> is non-zero, and returns the value of the last <var class=”var”>expr</var> evaluation, or NAN if <var class=”var”>cond</var> was always false. </dd> </dl>

The following constants are available: <dl class=”table”> <dt><samp class=”option”>PI</samp></dt> <dd> area of the unit disc, approximately 3.14 </dd> <dt><samp class=”option”>E</samp></dt> <dd> exp(1) (Euler’s number), approximately 2.718 </dd> <dt><samp class=”option”>PHI</samp></dt> <dd> golden ratio (1+sqrt(5))/2, approximately 1.618 </dd> </dl>

Assuming that an expression is considered "true" if it has a non-zero value, note that: <code class=”code”>*</code> works like AND <code class=”code”>+</code> works like OR For example the construct: <pre class=”example-preformatted”>if (A AND B) then C </pre> is equivalent to: <pre class=”example-preformatted”>if(A*B, C) </pre>

In your C code, you can extend the list of unary and binary functions, and define recognized constants, so that they are available for your expressions. The evaluator also recognizes the International System unit prefixes. If ’i’ is appended after the prefix, binary prefixes are used, which are based on powers of 1024 instead of powers of 1000. The ’B’ postfix multiplies the value by 8, and can be appended after a unit prefix or used alone. This allows using for example ’KB’, ’MiB’, ’G’ and ’B’ as number postfix. The list of available International System prefixes follows, with indication of the corresponding powers of 10 and of 2. <dl class=”table”> <dt><samp class=”option”>y</samp></dt> <dd> 10^-24 / 2^-80 </dd> <dt><samp class=”option”>z</samp></dt> <dd> 10^-21 / 2^-70 </dd> <dt><samp class=”option”>a</samp></dt> <dd> 10^-18 / 2^-60 </dd> <dt><samp class=”option”>f</samp></dt> <dd> 10^-15 / 2^-50 </dd> <dt><samp class=”option”>p</samp></dt> <dd> 10^-12 / 2^-40 </dd> <dt><samp class=”option”>n</samp></dt> <dd> 10^-9 / 2^-30 </dd> <dt><samp class=”option”>u</samp></dt> <dd> 10^-6 / 2^-20 </dd> <dt><samp class=”option”>m</samp></dt> <dd> 10^-3 / 2^-10 </dd> <dt><samp class=”option”>c</samp></dt> <dd> 10^-2 </dd> <dt><samp class=”option”>d</samp></dt> <dd> 10^-1 </dd> <dt><samp class=”option”>h</samp></dt> <dd> 10^2 </dd> <dt><samp class=”option”>k</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>K</samp></dt> <dd> 10^3 / 2^10 </dd> <dt><samp class=”option”>M</samp></dt> <dd> 10^6 / 2^20 </dd> <dt><samp class=”option”>G</samp></dt> <dd> 10^9 / 2^30 </dd> <dt><samp class=”option”>T</samp></dt>

<dd> 10^12 / 2^40 </dd> <dt><samp class="option">P</samp></dt>
<dd> 10^15 / 2^50 </dd> <dt><samp class="option">E</samp></dt>
<dd> 10^18 / 2^60 </dd> <dt><samp class="option">Z</samp></dt>
<dd> 10^21 / 2^70 </dd> <dt><samp class="option">Y</samp></dt>
<dd> 10^24 / 2^80 </dd> </dl>

 <h2 class="chapter">8 Codec Options#
TOC</h2>

libavcodec provides some generic global options, which can be set on all the encoders and decoders. In addition, each codec may support so-called private options, which are specific for a given codec. Sometimes, a global option may only affect a specific kind of codec, and may be nonsensical or ignored by another, so you need to be aware of the meaning of the specified options. Also some options are meant only for decoding or encoding.
Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVCodecContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follow: <dl class="table">
<dt><samp class="option">b <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set bitrate in bits/s. Default value is 200K. </dd> <dt><samp class="option">ab <var class="var">integer</var> (encoding,audio)</samp><
<dd> Set audio bitrate (in bits/s). Default value is 128K.
</dd> <dt><samp class="option">bt <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set video bitrate tolerance (in bits/s). In 1-pass mode, bitrate tolerance specifies how far ratecontrol is willing to deviate from the target average bitrate value. This is not related to min/max bitrate. Lowering tolerance too much has an adverse effect on quality. </dd>
<dt><samp class="option">flags <var class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Set generic flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">mv4</samp>’</dt>
<dd> Use four motion vector by macroblock (mpeg4). </dd>
<dt>‘<samp class="samp">qpel</samp>’</dt> <dd> Use 1/4 pel motion compensation. </dd> <dt>‘<samp class="samp">loop</samp>’</dt>
<dd> Use loop filter. </dd> <dt>‘<samp class="samp">qscale</samp>’</dt>
<dd> Use fixed qscale. </dd> <dt>‘<samp class="samp">pass1</samp>’</dt>
<dd> Use internal 2pass ratecontrol in first pass mode. </dd>
<dt>‘<samp class="samp">pass2</samp>’</dt> <dd> Use internal 2pass ratecontrol in second pass mode. </dd> <dt>‘<samp class="samp">gray</samp>’</dt> <dd> Only decode/encode grayscale. </dd> <dt>‘<samp class="samp">psnr</samp>’</dt>

<dd> Set error[?] variables during encoding. </dd> <dt>‘<samp class=”samp”>truncated</samp>’</dt> <dd> Input bitstream might be randomly truncated. </dd> <dt>‘<samp class=”samp”>drop_changed</samp>’</dt> <dd> Don’t output frames whose parameters differ from first decoded frame in stream. Error AVERROR_INPUT_CHANGED is returned when a frame is dropped. </dd> <dt>‘<samp class=”samp”>ildct</samp>’</dt> <dd> Use interlaced DCT. </dd> <dt>‘<samp class=”samp”>low_delay</samp>’<
<dd> Force low delay. </dd> <dt>‘<samp class=”samp”>global_header</samp>’<
<dd> Place global headers in extradata instead of every keyframe. </dd> <dt>‘<samp class=”samp”>bitexact</samp>’</dt> <dd> Only write platform-, build- and time-independent data. (except (I)DCT). This ensures that file and data checksums are reproducible and match between platforms. Its primary use is for regression testing. </dd> <dt>‘<samp class=”samp”>aic</samp>’</dt> <dd> Apply H263 advanced intra coding / mpeg4 ac prediction. </dd> <dt>‘<samp class=”samp”>ilme</samp>’</dt> <dd> Apply interlaced motion estimation. </dd> <dt>‘<samp class=”samp”>cgop</samp>’</dt> <dd> Use closed gop. </dd> <dt>‘<samp class=”samp”>output_corrupt</samp>’</dt> <dd> Output even potentially corrupted frames. </dd> </dl>

</dd>

<dt><samp class=”option”>time_base <var class=”var”>rational number</var></samp></dt>

<dd> Set codec time base.

It is the fundamental unit of time (in seconds) in terms of which

frame timestamps are represented. For fixed-fps content, timebase

should be <code class=”code”>1 / frame_rate</code> and timestamp increments should be

identically 1.

</dd>

<dt><samp class=”option”>g <var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set the group of picture (GOP) size. Default value is 12.

</dd>

<dt><samp class="option">ar <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set audio sampling rate (in Hz).

</dd>

<dt><samp class="option">ac <var class="var">integer</var> (decoding/encoding,audio)</samp></dt>

<dd> Set number of audio channels.

</dd>

<dt><samp class="option">cutoff <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set cutoff bandwidth. (Supported only by selected encoders, see

their respective documentation sections.)

</dd>

<dt><samp class="option">frame_size <var class="var">integer</var> (encoding,audio)</samp></dt>

<dd> Set audio frame size.

Each submitted frame except the last must contain exactly frame_size

samples per channel. May be 0 when the codec has

CODEC_CAP_VARIABLE_FRAME_SIZE set, in that case the frame size is not

restricted. It is set by some decoders to indicate constant frame

size.

</dd>

<dt><samp class="option">frame_number <var class="var">integer</var></samp></dt>

<dd> Set the frame number.

</dd>

<dt><samp class=”option”>delay <var class=”var”>integer</var></samp></dt>

<dt><samp class=”option”>qcomp <var class=”var”>float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale compression (VBR). It is used as a constant

in the ratecontrol equation. Recommended range for default rc_eq:

0.0-1.0.

</dd>

<dt><samp class=”option”>qblur <var class=”var”>float</var> (encoding,video)</samp></dt>

<dd> Set video quantizer scale blur (VBR).

</dd>

<dt><samp class=”option”>qmin <var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set min video quantizer scale (VBR). Must be included between -1 and

69, default value is 2.

</dd>

<dt><samp class=”option”>qmax <var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set max video quantizer scale (VBR). Must be included between -1 and

1024, default value is 31.

</dd>

<dt><samp class=”option”>qdiff <var class=”var”>integer</var> (encoding,video)</samp></dt>

<dd> Set max difference between the quantizer scale (VBR).

</dd>

<dt><samp class="option">bf <var class="var">integer</var> (encoding,video)</samp></dt>

<dd> Set max number of B frames between non-B-frames.

Must be an integer between -1 and 16. 0 means that B-frames are

disabled. If a value of -1 is used, it will choose an automatic value

depending on the encoder.

Default value is 0.

</dd>

<dt><samp class="option">b_qfactor <var class="var">float</var> (encoding,video)</samp></dt>

<dd> Set qp factor between P and B frames.

</dd>

<dt><samp class="option">codec_tag <var class="var">integer</var></samp></dt>

<dt><samp class="option">bug <var class="var">flags</var> (decoding,video)</samp></dt>

<dd> Workaround not auto detected encoder bugs.

Possible values:

<dl class="table">

<dt>‘<samp class="samp">autodetect</samp>’</dt>

<dt>‘<samp class="samp">xvid_ilace</samp>’</dt>

<dd> Xvid interlacing bug (autodetected if fourcc==XVIX)

</dd>

<dt>‘<samp class="samp">ump4</samp>’</dt>

<dd> (autodetected if fourcc==UMP4)

</dd>

<dt>‘<samp class="samp">no_padding</samp>’</dt>

<dd> padding bug (autodetected)

</dd>

<dt>‘<samp class="samp">amv</samp>’</dt>

<dt>‘<samp class="samp">qpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">std_qpel</samp>’</dt>

<dd> old standard qpel (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">qpel_chroma2</samp>’</dt>

<dt>‘<samp class="samp">direct_blocksize</samp>’</dt>

<dd> direct-qpel-blocksize bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">edge</samp>’</dt>

<dd> edge padding bug (autodetected per fourcc/version)

</dd>

<dt>‘<samp class="samp">hpel_chroma</samp>’</dt>

<dt>‘<samp class="samp">dc_clip</samp>’</dt>

<dt>‘<samp class="samp">ms</samp>’</dt>

<dd> Workaround various bugs in microsoft broken decoders.

</dd>

<dt>‘<samp class="samp">trunc</samp>’</dt>

<dd> trancated frames

</dd>

<dt>‘<samp class="samp">h264_dp_nnz</samp>’</dt>

<dd> H.264 nC derivation for partitioned slices, as done by JM rather than
by

9.2.1 (autodetected)

</dd>

</dl>

</dd> <dt><samp class="option">strict <var class="var">integer</var>
(decoding/encoding,audio,video)</samp></dt>
<dd> Specify how strictly to follow the standards. Possible values: <dl class="table"> <dt>‘<samp class="samp">very</samp>’</dt>
<dd> strictly conform to an older more strict version of the spec or reference
software </dd> <dt>‘<samp class="samp">strict</samp>’</dt>

<dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class="samp">normal</samp>’</dt>
<dt>‘<samp class="samp">unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class="samp">experimental</samp>’</dt>
<dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> <dt><samp class="option">b_qoffset <var class="var">float</var> (encoding,video)</samp></dt> <dd> Set QP offset between P and B frames. </dd> <dt><samp class="option">err_detect <var class="var">flags</var> (decoding,audio,video)</samp></dt>
<dd> Set error detection flags. Possible values: <dl class="table"> <dt>‘<samp class="samp">crccheck</samp>’</dt>
<dd> verify embedded CRCs </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt> <dd> detect bitstream specification deviations </dd> <dt>‘<samp class="samp">buffer</samp>’</dt>
<dd> detect improper bitstream length </dd> <dt>‘<samp class="samp">explode</samp>’</dt> <dd> abort decoding on minor error detection </dd> <dt>‘<samp class="samp">ignore_err</samp>’</dt>
<dd> ignore decoding errors, and continue decoding. This is useful if you want to analyze the content of a video and thus want everything to be decoded no matter what. This option will not result in a video that is pleasing to watch in case of errors. </dd> <dt>‘<samp class="samp">careful</samp>’</dt> <dd> consider things that violate the spec and have not been seen in the wild as errors </dd>
<dt>‘<samp class="samp">compliant</samp>’</dt>
<dd> consider all spec non compliancies as errors </dd>
<dt>‘<samp class="samp">aggressive</samp>’</dt>
<dd> consider things that a sane encoder should not do as an error </dd> </dl>

</dd> <dt><samp class="option">has_b_frames <var class="var">integer</var></samp></dt>
<dt><samp class="option">block_align <var class="var">integer</var></samp></dt>
<dt><samp class="option">rc_override_count <var class="var">integer</var></samp></dt>
<dt><samp class="option">maxrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set max bitrate tolerance (in bits/s). Requires bufsize to be set. </dd>
<dt><samp class="option">minrate <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set min bitrate tolerance (in bits/s). Most useful in setting up a CBR encode. It is of little use elsewise. </dd> <dt><samp class="option">bufsize <var class="var">integer</var> (encoding,audio,video)</samp></dt>
<dd> Set ratecontrol buffer size (in bits). </dd> <dt><samp class="option">i_qfactor <var class="var">float</var> (encoding,video)</sam
<dd> Set QP factor between P and I frames. </dd> <dt><samp

class=”option”>i_qoffset <var class=”var”>float</var> (encoding,video)</sam
<dd> Set QP offset between P and I frames. </dd> <dt><samp
class=”option”>dct <var class=”var”>integer</var> (encoding,video)</samp>
<dd> Set DCT algorithm. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> autoselect a good one (default) </dd> <dt>‘<samp
class=”samp”>fastint</samp>’</dt> <dd> fast integer
</dd> <dt>‘<samp class=”samp”>int</samp>’</dt>
<dd> accurate integer </dd> <dt>‘<samp class=”samp”>mmx</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>faan</samp>’</dt> <dd> floating point AAN
DCT </dd> </dl>

</dd> <dt><samp class=”option”>lumi_mask <var class=”var”>float</var>
(encoding,video)</samp></dt> <dd> Compress
bright areas stronger than medium ones. </dd> <dt><samp
class=”option”>tcplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set temporal complexity masking. </dd> <dt><samp
class=”option”>scplx_mask <var class=”var”>float</var> (encoding,video)</s
<dd> Set spatial complexity masking. </dd> <dt><samp
class=”option”>p_mask <var class=”var”>float</var> (encoding,video)</samp
<dd> Set inter masking. </dd> <dt><samp class=”option”>dark_mask
<var class=”var”>float</var> (encoding,video)</samp></dt>
<dd> Compress dark areas stronger than medium ones.
</dd> <dt><samp class=”option”>idct <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Select IDCT implementation. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dt>‘<samp class=”samp”>int</samp>’</dt> <dt>‘<samp
class=”samp”>simple</samp>’</dt> <dt>‘<samp class=”samp”>simplemmx</samp>&rsq
<dt>‘<samp class=”samp”>simpleauto</samp>’</dt>
<dd> Automatically pick a IDCT compatible with the simple one
 </dd> <dt>‘<samp class=”samp”>arm</samp>’</dt>
<dt>‘<samp class=”samp”>altivec</samp>’</dt> <dt>‘<samp
class=”samp”>sh4</samp>’</dt> <dt>‘<samp class=”samp”>simplearm</samp>’<
<dt>‘<samp class=”samp”>simplearmv5te</samp>’</dt>
<dt>‘<samp class=”samp”>simplearmv6</samp>’</dt>
<dt>‘<samp class=”samp”>simpleneon</samp>’</dt>
<dt>‘<samp class=”samp”>xvid</samp>’</dt> <dt>‘<samp
class=”samp”>faani</samp>’</dt> <dd> floating point AAN
IDCT </dd> </dl>

</dd> <dt><samp class=”option”>slice_count <var class=”var”>integer</var></samp></dt>
<dt><samp class=”option”>ec <var class=”var”>flags</var> (<em
class=”emph”>decoding,video)</samp></dt> <dd> Set error
concealment strategy. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>guess_mvs</samp>’</dt>

<dd> iterative motion vector (MV) search (slow) </dd> <dt>‘<samp
class="samp">deblock</samp>’</dt> <dd> use strong deblock
filter for damaged MBs </dd> <dt>‘<samp class="samp">favor_inter</samp>’</dt>
<dd> favor predicting from the previous frame instead of the current
</dd> </dl>

</dd> <dt><samp class="option">bits_per_coded_sample <var class="var">integer</var></samp></d
<dt><samp class="option">aspect <var class="var">rational number</var>
(encoding,video)</samp></dt> <dd> Set
sample aspect ratio. </dd> <dt><samp class="option">sar <var
class="var">rational number</var> (encoding,video)</samp></dt>
<dd> Set sample aspect ratio. Alias to <var class="var">aspect</var>.
 </dd> <dt><samp class="option">debug <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>
<dd> Print specific debug info. Possible values: <dl
class="table"> <dt>‘<samp class="samp">pict</samp>’</dt>
<dd> picture info </dd> <dt>‘<samp class="samp">rc</samp>’</dt>
<dd> rate control </dd> <dt>‘<samp class="samp">bitstream</samp>’</dt>
<dt>‘<samp class="samp">mb_type</samp>’</dt> <dd> macroblock
(MB) type </dd> <dt>‘<samp class="samp">qp</samp>’</dt>
<dd> per-block quantization parameter (QP) </dd> <dt>‘<samp
class="samp">dct_coeff</samp>’</dt> <dt>‘<samp
class="samp">green_metadata</samp>’</dt> <dd> display
complexity metadata for the upcoming frame, GoP or for a given duration.
 </dd> <dt>‘<samp class="samp">skip</samp>’</dt>
<dt>‘<samp class="samp">startcode</samp>’</dt> <dt>‘<samp
class="samp">er</samp>’</dt> <dd> error recognition
</dd> <dt>‘<samp class="samp">mmco</samp>’</dt>
<dd> memory management control operations (H.264) </dd>
<dt>‘<samp class="samp">bugs</samp>’</dt> <dt>‘<samp
class="samp">buffers</samp>’</dt> <dd> picture buffer alloca-
tions </dd> <dt>‘<samp class="samp">thread_ops</samp>’</dt>
<dd> threading operations </dd> <dt>‘<samp class="samp">nomc</samp>’</dt
<dd> skip motion compensation </dd> </dl>

</dd> <dt><samp class="option">cmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
full pel me compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp

class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>subcmp <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
sub pel me compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class=”samp”>dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class=”samp”>psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class=”samp”>bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class=”samp”>rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class=”samp”>zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class=”samp”>vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class=”samp”>vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class=”samp”>nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class=”samp”>w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class=”samp”>w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class=”samp”>dctmax</samp>’</dt>
<dt>‘<samp class=”samp”>chroma</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>mbcmp <var class=”var”>integer</var>
(encoding,video)</samp></dt> <dd> Set
macroblock compare function. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class=”samp”>sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class=”samp”>satd</samp>’</dt>

<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">ildctcmp <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
interlaced dct compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">dia_size <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set

diamond type & size for motion estimation. <dl class="table">
<dt>‘<samp class="samp">(1024, INT_MAX)</samp>’</dt>
<dd> full motion estimation(slowest) </dd> <dt>‘<samp
class="samp">(768, 1024]</samp>’</dt> <dd> umh mo-
tion estimation </dd> <dt>‘<samp class="samp">(512,
768]</samp>’</dt> <dd> hex motion estimation </dd>
<dt>‘<samp class="samp">(256, 512]</samp>’</dt>
<dd> l2s diamond motion estimation </dd> <dt>‘<samp
class="samp">[2,256]</samp>’</dt> <dd> var diamond
motion estimation </dd> <dt>‘<samp class="samp">(-1,
2)</samp>’</dt> <dd> small diamond motion estimation
</dd> <dt>‘<samp class="samp">-1</samp>’</dt>
<dd> funny diamond motion estimation </dd> <dt>‘<samp
class="samp">(INT_MIN, -1)</samp>’</dt> <dd> sab dia-
mond motion estimation </dd> </dl>

</dd> <dt><samp class="option">last_pred <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
amount of motion predictors from the previous frame. </dd>
<dt><samp class="option">precmp <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set pre
motion estimation compare function. Possible values: <dl
class="table"> <dt>‘<samp class="samp">sad</samp>’</dt>
<dd> sum of absolute differences, fast (default) </dd> <dt>‘<samp
class="samp">sse</samp>’</dt> <dd> sum of squared errors
</dd> <dt>‘<samp class="samp">satd</samp>’</dt>
<dd> sum of absolute Hadamard transformed differences </dd>
<dt>‘<samp class="samp">dct</samp>’</dt> <dd> sum
of absolute DCT transformed differences </dd> <dt>‘<samp
class="samp">psnr</samp>’</dt> <dd> sum of squared
quantization errors (avoid, low quality) </dd> <dt>‘<samp
class="samp">bit</samp>’</dt> <dd> number of bits needed
for the block </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> rate distortion optimal, slow </dd> <dt>‘<samp
class="samp">zero</samp>’</dt> <dd> 0 </dd>
<dt>‘<samp class="samp">vsad</samp>’</dt> <dd> sum
of absolute vertical differences </dd> <dt>‘<samp class="samp">vsse</samp>’</dt>
<dd> sum of squared vertical differences </dd> <dt>‘<samp
class="samp">nsse</samp>’</dt> <dd> noise preserving sum of
squared differences </dd> <dt>‘<samp class="samp">w53</samp>’</dt>
<dd> 5/3 wavelet, only used in snow </dd> <dt>‘<samp
class="samp">w97</samp>’</dt> <dd> 9/7 wavelet, only used
in snow </dd> <dt>‘<samp class="samp">dctmax</samp>’</dt>
<dt>‘<samp class="samp">chroma</samp>’</dt> </dl>

</dd> <dt><samp class="option">pre_dia_size <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set

diamond type & size for motion estimation pre-pass. </dd>
<dt><samp class="option">subq <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set sub pel
motion estimation quality. </dd> <dt><samp class="option">me_range
<var class="var">integer</var> (encoding,video)</samp></dt>
<dd> Set limit motion vectors range (1023 for DivX player). </dd>
<dt><samp class="option">global_quality <var class="var">integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class="option">slice_flags <var class="var">integer</var></samp></dt>
<dt><samp class="option">mbd <var class="var">integer</var> (<em
class="emph">encoding,video)</samp></dt> <dd> Set mac-
roblock decision algorithm (high quality mode). Possible values:
<dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt>
<dd> use mbcmp (default) </dd> <dt>‘<samp class="samp">bits</samp>’</dt>
<dd> use fewest bits </dd> <dt>‘<samp class="samp">rd</samp>’</dt>
<dd> use best rate distortion </dd> </dl>

</dd> <dt><samp class="option">rc_init_occupancy <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
number of bits which should be loaded into the rc buffer before de-
coding starts. </dd> <dt><samp class="option">flags2 <var
class="var">flags</var> (decoding/encoding,audio,video,subtitles)</samp></
<dd> Possible values: <dl class="table"> <dt>‘<samp
class="samp">fast</samp>’</dt> <dd> Allow non spec compli-
ant speedup tricks. </dd> <dt>‘<samp class="samp">noout</samp>’</dt>
<dd> Skip bitstream encoding. </dd> <dt>‘<samp
class="samp">ignorecrop</samp>’</dt> <dd> Ignore cropping
information from sps. </dd> <dt>‘<samp class="samp">local_header</samp>’</dt>
<dd> Place global headers at every keyframe instead of in extradata.
</dd> <dt>‘<samp class="samp">chunks</samp>’</dt>
<dd> Frame data might be split into multiple chunks. </dd>
<dt>‘<samp class="samp">showall</samp>’</dt> <dd> Show
all frames before the first keyframe. </dd> <dt>‘<samp
class="samp">export_mvs</samp>’</dt> <dd> Export motion
vectors into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>)
for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>.
</dd> <dt>‘<samp class="samp">skip_manual</samp>’</dt>
<dd> Do not skip samples and export skip information as frame side data.
</dd> <dt>‘<samp class="samp">ass_ro_flush_noop</samp>’</dt>
<dd> Do not reset ASS ReadOrder field on flush. </dd>
<dt>‘<samp class="samp">icc_profiles</samp>’</dt>
<dd> Generate/parse embedded ICC profiles from/to colorimetry tags.
</dd> <dt>‘<samp class="samp">fixed_frame_size</samp>’</dt>
<dd> Force audio encoders to use a fixed frame size. </dd> </dl>

</dd> <dt><samp class="option">export_side_data <var class="var">flags</var>
(decoding/encoding,audio,video,subtitles)</samp></dt>

<dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">mvs</samp>’</dt> <dd> Export motion vectors into frame side-data (see <code class="code">AV_FRAME_DATA_MOTION_VECTORS</code>) for codecs that support it. See also <samp class="file">doc/examples/export_mvs.c</samp>. </dd> <dt>‘<samp class="samp">prft</samp>’</dt> <dd> Export encoder Producer Reference Time into packet side-data (see <code class="code">AV_PKT_DATA_PRFT</code>) for codecs that support it. </dd> <dt>‘<samp class="samp">venc_params</samp>’</dt> <dd> Export video encoding parameters through frame side data (see <code class="code">AV_FRAME_DATA_VIDEO_ENC_PARAMS</code>) for codecs that support it. At present, those are H.264 and VP9. </dd> <dt>‘<samp class="samp">film_grain</samp>’</dt> <dd> Export film grain parameters through frame side data (see <code class="code">AV_FRAME_DATA_FILM_GRAIN_PARAMS</code>). Supported at present by AV1 decoders. </dd> <dt>‘<samp class="samp">enhancements</samp>’</dt> <dd> Export picture enhancement metadata through frame side data, e.g. LCEVC (see <code class="code">AV_FRAME_DATA_LCEVC</code>). </dd> </dl>

</dd> <dt><samp class="option">threads <var class="var">integer</var> (decoding/encoding,video)</samp></dt> <dd> Set the number of threads to be used, in case the selected codec implementation supports multi-threading. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, 0</samp>’</dt> <dd> automatically select the number of threads to set </dd> </dl>

Default value is ‘<samp class="samp">auto</samp>’. </dd> <dt><samp class="option">dc <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set intra_dc_precision. </dd> <dt><samp class="option">nssew <var class="var">integer</var> (encoding,video)</samp></dt> <dd> Set nsse weight. </dd> <dt><samp class="option">skip_top <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the top which are skipped. </dd> <dt><samp class="option">skip_bottom <var class="var">integer</var> (decoding,video)</samp></dt> <dd> Set number of macroblock rows at the bottom which are skipped. </dd> <dt><samp class="option">profile <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set encoder codec profile. Default value is ‘<samp class="samp">unknown</samp>’. Encoder specific profiles are documented in the relevant encoder documentation. </dd> <dt><samp class="option">level <var class="var">integer</var> (encoding,audio,video)</samp></dt> <dd> Set the encoder level. This level depends on the specific codec, and might correspond to the profile level. It is set by default to ‘<samp class="samp">unknown</samp>’. Possible values:

<dl class="table"> <dt>‘<samp class="samp">unknown</samp>’</dt>
</dl>

</dd> <dt><samp class="option">lowres <var class="var">integer</var>
(decoding,audio,video)</samp></dt> <dd> Decode
at 1= 1/2, 2=1/4, 3=1/8 resolutions. </dd> <dt><samp
class="option">mblmin <var class="var">integer</var> (encoding,video)</sam
<dd> Set min macroblock lagrange factor (VBR). </dd>
<dt><samp class="option">mblmax <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
max macroblock lagrange factor (VBR). </dd> <dt><samp
class="option">skip_loop_filter <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dt><samp class="option">skip_idct
<var class="var">integer</var> (decoding,video)</samp></dt>
<dt><samp class="option">skip_pred <var class="var">integer</var>
(decoding,video)</samp></dt> <dt><samp
class="option">skip_frame <var class="var">integer</var> (<em
class="emph">decoding,video)</samp></dt> <dd> Make
decoder discard processing depending on the frame type selected by the
option value. <samp class="option">skip_loop_filter</samp>
skips frame loop filtering, <samp class="option">skip_idct</samp> skips
frame IDCT/dequantization, <samp class="option">skip_pred</samp>
skips prediction (intra prediction and motion compensation), <samp
class="option">skip_frame</samp> skips decoding. With both
<samp class="option">skip_pred</samp> and <samp class="option">skip_idct</samp>
set to ‘<samp class="samp">all</samp>’ the decoder may skip
all pixel operations and output frames with undefined pixels but valid metadata,
such as timestamps and frame types. Possible values: <dl
class="table"> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> Discard no frame. </dd> <dt>‘<samp class="samp">default</samp>’</dt
<dd> Discard useless frames like 0-sized frames. </dd>
<dt>‘<samp class="samp">noref</samp>’</dt> <dd> Discard
all non-reference frames. </dd> <dt>‘<samp class="samp">bidir</samp>’</dt>
<dd> Discard all bidirectional frames. </dd> <dt>‘<samp
class="samp">nokey</samp>’</dt> <dd> Discard all frames ex-
cepts keyframes. </dd> <dt>‘<samp class="samp">nointra</samp>’</dt>
<dd> Discard all frames except I frames. </dd> <dt>‘<samp
class="samp">all</samp>’</dt> <dd> Discard all frames.
</dd> </dl>

Default value is ‘<samp class="samp">default</samp>’.
 </dd> <dt><samp class="option">bidir_refine <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Refine
the two motion vectors used in bidirectional macroblocks. </dd>
<dt><samp class="option">keyint_min <var class="var">integer</var>
(encoding,video)</samp></dt> <dd> Set
minimum interval between IDR-frames. </dd> <dt><samp

class=”option”>refs <var class=”var”>integer</var> (encoding,video)</samp>
<dd> Set reference frames to consider for motion compensation.
</dd> <dt><samp class=”option”>trellis <var class=”var”>integer</var>
(encoding,audio,video)</samp></dt> <dd> Set
rate-distortion optimal quantization. </dd> <dt><samp class=”option”>mv0_threshold
<var class=”var”>integer</var> (encoding,video)</samp></dt>
<dt><samp class=”option”>compression_level <var class=”var”>integer</var>
(encoding,audio,video)</samp></dt> <dt><samp
class=”option”>bits_per_raw_sample <var class=”var”>integer</var></samp></dt>
<dt><samp class=”option”>channel_layout <var class=”var”>integer</var>
(decoding/encoding,audio)</samp></dt>
<dd> See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#channel-
layout-syntax”>the Channel Layout section in the ffmpeg-utils(1) manual
for the required syntax. </dd> <dt><samp class=”option”>rc_max_vbv_use
<var class=”var”>float</var> (encoding,video)</samp></dt>
<dt><samp class=”option”>rc_min_vbv_use <var class=”var”>float</var>
(encoding,video)</samp></dt> <dt><samp
class=”option”>color_primaries <var class=”var”>integer</var> (<em
class=”emph”>decoding/encoding,video)</samp></dt> <dd> Possible
values: <dl class=”table”> <dt>‘<samp class=”samp”>bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class=”samp”>bt470m</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class=”samp”>film</samp>’</dt>
<dd> Film </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt>
<dd> BT.2020 </dd> <dt>‘<samp class=”samp”>smpte428</samp>’</dt>
<dt>‘<samp class=”samp”>smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class=”samp”>smpte431</samp>’</d
<dd> SMPTE 431-2 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt>
<dd> SMPTE 432-1 </dd> <dt>‘<samp class=”samp”>jedec-
p22</samp>’</dt> <dd> JEDEC P22 </dd> </dl>

</dd> <dt><samp class=”option”>color_trc <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd>
<dt>‘<samp class=”samp”>gamma22</samp>’</dt>
<dd> BT.470 M </dd> <dt>‘<samp class=”samp”>gamma28</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt>
<dd> Linear </dd> <dt>‘<samp class=”samp”>log</samp>’</dt>
<dt>‘<samp class=”samp”>log100</samp>’</dt> <dd> Log
</dd> <dt>‘<samp class=”samp”>log_sqrt</samp>’</dt>
<dt>‘<samp class=”samp”>log316</samp>’</dt> <dd> Log

square root </dd> <dt>‘<samp class=”samp”>iec61966_2_4</samp>’</dt>
<dt>‘<samp class=”samp”>iec61966-2-4</samp>’</dt>
<dd> IEC 61966-2-4 </dd> <dt>‘<samp class=”samp”>bt1361</samp>’</dt>
<dt>‘<samp class=”samp”>bt1361e</samp>’</dt> <dd> BT.1361
</dd> <dt>‘<samp class=”samp”>iec61966_2_1</samp>’</dt>
<dt>‘<samp class=”samp”>iec61966-2-1</samp>’</dt>
<dd> IEC 61966-2-1 </dd> <dt>‘<samp class=”samp”>bt2020_10</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020_10bit</samp>’</dt>
<dd> BT.2020 - 10 bit </dd> <dt>‘<samp class=”samp”>bt2020_12</samp>’</d
<dt>‘<samp class=”samp”>bt2020_12bit</samp>’</dt>
<dd> BT.2020 - 12 bit </dd> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt
<dd> SMPTE ST 2084 </dd> <dt>‘<samp class=”samp”>smpte428</samp>’</dt
<dt>‘<samp class=”samp”>smpte428_1</samp>’</dt>
<dd> SMPTE ST 428-1 </dd> <dt>‘<samp class=”samp”>arib-
std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>
</dl>

</dd> <dt><samp class=”option”>colorspace <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>rgb</samp>’</dt> <dd> RGB </dd>
<dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709
</dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt>
<dd> FCC </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt>
<dd> BT.470 BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt>
<dd> SMPTE 170 M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt
<dd> SMPTE 240 M </dd> <dt>‘<samp class=”samp”>ycocg</samp>’</dt>
<dd> YCOCG </dd> <dt>‘<samp class=”samp”>bt2020nc</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020_ncl</samp>’</dt>
<dd> BT.2020 NCL </dd> <dt>‘<samp class=”samp”>bt2020c</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020_cl</samp>’</dt>
<dd> BT.2020 CL </dd> <dt>‘<samp class=”samp”>smpte2085</samp>’</dt>
<dd> SMPTE 2085 </dd> <dt>‘<samp class=”samp”>chroma-
derived-nc</samp>’</dt> <dd> Chroma-derived NCL </dd>
<dt>‘<samp class=”samp”>chroma-derived-c</samp>’</dt>
<dd> Chroma-derived CL </dd> <dt>‘<samp class=”samp”>ictcp</samp>’</dt>
<dd> ICtCp </dd> </dl>

</dd> <dt><samp class=”option”>color_range <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> If used as input parameter, it serves as a hint to the de-
coder, which color_range the input has. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt>
<dt>‘<samp class=”samp”>mpeg</samp>’</dt> <dt>‘<samp
class=”samp”>limited</samp>’</dt> <dd> MPEG (219*2^(n-
8)) </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt>
<dt>‘<samp class=”samp”>jpeg</samp>’</dt> <dt>‘<samp

class=”samp”>full</samp>’</dt> <dd> JPEG (2^n-1) </dd>
</dl>

</dd> <dt><samp class=”option”>chroma_sample_location <var
class=”var”>integer</var> (decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>left</samp>’</dt> <dt>‘<samp class=”samp”>center</samp>’</dt>
<dt>‘<samp class=”samp”>topleft</samp>’</dt> <dt>‘<samp
class=”samp”>top</samp>’</dt> <dt>‘<samp class=”samp”>bottomleft</samp>’<
<dt>‘<samp class=”samp”>bottom</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>alpha_mode <var class=”var”>integer</var>
(decoding/encoding,video)</samp></dt>
<dd> Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>premultiplied</samp>’</dt> <dt>‘<samp
class=”samp”>straight</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>log_level_offset <var class=”var”>integer</var></samp></dt>
<dd> Set the log level offset. </dd> <dt><samp class=”option”>slices
<var class=”var”>integer</var> (encoding,video)</samp></dt>
<dd> Number of slices, used in parallelized encoding. </dd>
<dt><samp class=”option”>thread_type <var class=”var”>flags</var>
(decoding/encoding,video)</samp></dt>
<dd> Select which multithreading methods to use. Use
of ‘<samp class=”samp”>frame</samp>’ will increase decod-
ing delay by one frame per thread, so clients which cannot provide future
frames should not use it. Possible values: <dl class=”table”>
<dt>‘<samp class=”samp”>slice</samp>’</dt> <dd> Decode
more than one part of a single frame at once. Multithreading
using slices works only when the video was encoded with slices.
</dd> <dt>‘<samp class=”samp”>frame</samp>’</dt>
<dd> Decode more than one frame at once. </dd> </dl>

Default value is ‘<samp class=”samp”>slice+frame</samp>’.
 </dd> <dt><samp class=”option”>audio_service_type <var
class=”var”>integer</var> (encoding,audio)</samp></dt>
<dd> Set audio service type. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>ma</samp>’</dt>
<dd> Main Audio Service </dd> <dt>‘<samp class=”samp”>ef</samp>’</dt>
<dd> Effects </dd> <dt>‘<samp class=”samp”>vi</samp>’</dt>
<dd> Visually Impaired </dd> <dt>‘<samp class=”samp”>hi</samp>’</dt>
<dd> Hearing Impaired </dd> <dt>‘<samp class=”samp”>di</samp>’</dt>
<dd> Dialogue </dd> <dt>‘<samp class=”samp”>co</samp>’</dt>
<dd> Commentary </dd> <dt>‘<samp class=”samp”>em</samp>’</dt>
<dd> Emergency </dd> <dt>‘<samp class=”samp”>vo</samp>’</dt>
<dd> Voice Over </dd> <dt>‘<samp class=”samp”>ka</samp>’</dt>
<dd> Karaoke </dd> </dl>

</dd> <dt><samp class="option">request_sample_fmt <var class="var">sample_fmt</var> (decoding,audio)</samp></dt> <dd> Set sample format audio decoders should prefer. Default value is <code class="code">none</code>. </dd> <dt><samp class="option">pkt_timebase <var class="var">rational number</var></samp></dt> <dt><samp class="option">sub_charenc <var class="var">encoding</var> (decoding,subtitles)</samp></dt> <dd> Set the input subtitles character encoding. </dd> <dt><samp class="option">field_order <var class="var">field_order</var> (video)</samp></dt> <dd> Set/override the field order of the video. Possible values: <dl class="table"> <dt>‘<samp class="samp">progressive</samp>’</dt> <dd> Progressive video </dd> <dt>‘<samp class="samp">tt</samp>’</dt> <dd> Interlaced video, top field coded and displayed first </dd> <dt>‘<samp class="samp">bb</samp>’</dt> <dd> Interlaced video, bottom field coded and displayed first </dd> <dt>‘<samp class="samp">tb</samp>’</dt> <dd> Interlaced video, top coded first, bottom displayed first </dd> <dt>‘<samp class="samp">bt</samp>’</dt> <dd> Interlaced video, bottom coded first, top displayed first </dd> </dl>

</dd> <dt><samp class="option">skip_alpha <var class="var">bool</var> (decoding,video)</samp></dt> <dd> Set to 1 to disable processing alpha (transparency). This works like the ‘<samp class="samp">gray</samp>’ flag in the <samp class="option">flags</samp> option which skips chroma information instead of alpha. Default is 0. </dd> <dt><samp class="option">codec_whitelist <var class="var">list</var> (input)</samp></dt> <dd> "," separated list of allowed decoders. By default all are allowed. </dd> <dt><samp class="option">dump_separator <var class="var">string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class="example-preformatted">ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class="option">max_pixels <var class="var">integer</var> (decoding/encoding,video)</samp></dt> <dd> Maximum number of pixels per image. This value can be used to avoid out of memory failures due to large images. </dd> <dt><samp class="option">apply_cropping <var class="var">bool</var> (decoding,video)</samp></dt> <dd> Enable cropping if cropping parameters are multiples of the required alignment for the left and top parameters. If the alignment is not met the cropping will be partially applied to maintain alignment. Default is 1 (enabled). Note: The required alignment depends on if <code

class=”code”>AV_CODEC_FLAG_UNALIGNED</code> is set and the CPU. <code class=”code”>AV_CODEC_FLAG_UNALIGNED</code> cannot be changed from the command line. Also hardware decoders will not apply left/top Cropping. </dd> <dt><samp class=”option”>lcevc-dec-params <var class=”var”>string</var> (decoding,video)</samp></dt> <dd> A :-separated list of key=value parameters passed to liblcevc-dec to initialize it. </dd> </dl>

 <h2 class=”chapter”>9 Decoders# TOC</h2>

Decoders are configured elements in FFmpeg which allow the decoding of multimedia streams. When you configure your FFmpeg build, all the supported native decoders are enabled by default. Decoders requiring an external library must be enabled manually via the corresponding <code class=”code”>--enable-lib</code> option. You can list all available decoders using the configure option <code class=”code”>--list-decoders</code>. You can disable all the decoders with the configure option <code class=”code”>--disable-decoders</code> and selectively enable / disable single decoders with the options <code class=”code”>--enable-decoder=<var class=”var”>DECODER</var></code> / <code class=”code”>--disable-decoder=<var class=”var”>DECODER</var></code>. The option <code class=”code”>-decoders</code> of the ff* tools will display the list of enabled decoders.

 <h2 class=”chapter”>10 Video Decoders# TOC</h2>

A description of some of the currently available video decoders follows. <h3 class=”section”>10.1 av1# TOC</h3>

AOMedia Video 1 (AV1) decoder. <h4 class=”subsection”>10.1.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>operating_point</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Default is 0. </dd> </dl>

 <h3 class=”section”>10.2 hevc# TOC</h3>

HEVC (AKA ITU-T H.265 or ISO/IEC 23008-2) decoder. The decoder supports MV-HEVC multiview streams with at most two views. Views to be output are selected by supplying a list of view IDs to the decoder (the <samp class="option">view_ids</samp> option). This option may be set either statically before decoder init, or from the <code class="code">get_format()</code> callback - useful for the case when the view count or IDs change dynamically during decoding. Only the base layer is decoded by default. Note that if you are using the <code class="code">ffmpeg</code> CLI tool, you should be using view specifiers as documented in its manual, rather than the options documented here. <h4 class="subsection">10.2.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">view_ids (MV-HEVC)</samp></dt> <dd> Specify a list of view IDs that should be output. This option can also be set to a single ’-1’, which will cause all views defined in the VPS to be decoded and output. </dd> <dt><samp class="option">view_ids_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view IDs available in the active VPS. The array is empty for single-layer video. The value of this option is guaranteed to be accurate when read from the <code class="code">get_format()</code> callback. It may also be set at other times (e.g. after opening the decoder), but the value is informational only and may be incorrect (e.g. when the stream contains multiple distinct VPS NALUs). </dd> <dt><samp class="option">view_pos_available (MV-HEVC)</samp></dt> <dd> This option may be read by the caller to retrieve an array of view positions (left, right, or unspecified) available in the active VPS, as <code class="code">AVStereo3DView</code> values. When the array is available, its elements apply to the corresponding elements of <samp class="option">view_ids_available</samp>, i.e. <code class="code">view_pos_available[i]</code> contains the position of view with ID <code class="code">view_ids_available[i]</code>. Same validity restrictions as for <samp class="option">view_ids_available</samp> apply to this option. </dd> </dl>

 <h3 class="section">10.3 rawvideo# TOC</h3>

Raw video decoder. This decoder decodes rawvideo streams. <h4 class="subsection">10.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">top <var class="var">top_field_first</var></samp></dt> <dd> Specify the assumed field type of the input video. <dl class="table"> <dt><samp class="option">-1</samp></dt> <dd> the video is assumed to be progressive (default) </dd> <dt><samp class="option">0</samp></dt> <dd> bottom-field-first is assumed </dd> <dt><samp class="option">1</samp></dt> <dd> top-field-first is assumed </dd> </dl>

</dd> </dl>

 <h3 class="section">10.4 libastcenc# TOC</h3>

ARM ASTC Encoder wrapper for decoding ASTC textures. This decoder requires the Arm ASTC Encoder (astc-encoder) library version 5.4 or newer, installed as <code class="code">astcenc/astcenc.h</code> next to a library that <code class="code">-lastcenc</code> finds. Pass <code class="code">--enable-version3</code> together with <code class="code">--enable-libastcenc</code> to configure. The <samp class="file">.astc</samp> header does not record the intended decoding profile. Individual blocks identify their endpoint encodings, but this does not establish the sRGB versus linear interpretation. The decoder therefore uses the container metadata or <samp class="option">dec_profile</samp>: a KTX sRGB texture is decoded as <code class="code">ldr-srgb</code>, while a KTX texture in the linear format is sampled with HDR precision (<code class="code">hdr-ldr-a</code>, half-float output) because the linear GL internal format says nothing about LDR. A raw <code class="code">.astc</code> file carries no profile at all, so it falls back to <code class="code">ldr-srgb</code>; use <samp class="option">dec_profile</samp> for a raw HDR image: <pre class="example-preformatted">ffmpeg -dec_profile hdr-ldr-a -i input.astc output.png </pre>

 <h4 class="subsection">10.4.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">dec_profile <var class="var">int</var></samp></dt> <dd> Color profile to decode with, overriding the container metadata. It must match the profile the texture was encoded with: <code class="code">astcenc</code> substitutes a fixed magenta error color for blocks whose endpoints do not fit the selected profile. The values are <code class="code">ldr</code>, <code class="code">ldr-srgb</code>, <code class="code">hdr-ldr-a</code> and <code class="code">hdr</code>. Without this option the profile published by the container is used; a raw <samp class="file">.astc</samp> stream, which publishes none, falls back to

<code class="code">ldr-srgb</code>. </dd> </dl>

 <h3 class="section">10.5 libdav1d# TOC</h3>

dav1d AV1 decoder. libdav1d allows libavcodec to decode the AOMedia Video 1 (AV1) codec. Requires the presence of the libdav1d headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libdav1d</code>. <h4 class="subsection">10.5.1 Options# TOC</h4>

The following options are supported by the libdav1d wrapper. <dl class="table"> <dt><samp class="option">max_frame_delay</samp></dt> <dd> Set max amount of frames the decoder may buffer internally. The default value is 0 (autodetect). </dd> <dt><samp class="option">filmgrain</samp></dt> <dd> Apply film grain to the decoded video if present in the bitstream. Defaults to the internal default of the library. This option is deprecated and will be removed in the future. See the global option <code class="code">export_side_data</code> to export Film Grain parameters instead of applying it. </dd> <dt><samp class="option">oppoint</samp></dt> <dd> Select an operating point of a scalable AV1 bitstream (0 - 31). Defaults to the internal default of the library. </dd> <dt><samp class="option">alllayers</samp></dt> <dd> Output all spatial layers of a scalable AV1 bitstream. The default value is false. </dd> </dl>

 <h3 class="section">10.6 libdavs2# TOC</h3>

AVS2-P2/IEEE1857.4 video decoder wrapper. This decoder allows libavcodec to decode AVS2 streams with davs2 library.

 <h3 class="section">10.7 libuavs3d# TOC</h3>

AVS3-P2/IEEE1857.10 video decoder. libuavs3d allows libavcodec to decode AVS3 streams. Requires the presence of the libuavs3d headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libuavs3d</code>. <h4 class="subsection">10.7.1 Options<span

class=”pull-right”># TOC</h4>

The following option is supported by the libuavs3d wrapper. <dl class=”table”> <dt><samp class=”option”>frame_threads</samp></dt> <dd> Set amount of frame threads to use during decoding. The default value is 0 (autodetect). </dd> </dl>

 <h3 class=”section”>10.8 libxevd# TOC</h3>

eXtra-fast Essential Video Decoder (XEVD) MPEG-5 EVC decoder wrapper. This decoder requires the presence of the libxevd headers and library during configuration. You need to explicitly configure the build with <samp class=”option”>--enable-libxevd</samp>. The xevd project website is at <a class=”url” href=”https://github.com/mpeg5/xevd”>https://github.com/mpeg5/xevd <h4 class=”subsection”>10.8.1 Options# TOC</h4>

The following options are supported by the libxevd wrapper. The xevd-equivalent options or values are listed in parentheses for easy migration. To get a more accurate and extensive documentation of the libxevd options, invoke the command <code class=”code”>xevd_app --help</code> or consult the libxevd documentation. <dl class=”table”> <dt><samp class=”option”>threads (threads)</samp></dt> <dd> Force to use a specific number of threads </dd> </dl>

 <h3 class=”section”>10.9 QSV Decoders# TOC</h3>

The family of Intel QuickSync Video decoders (VC1, MPEG-2, H.264, HEVC, JPEG/MJPEG, VP8, VP9, AV1, VVC). <h4 class=”subsection”>10.9.1 Common Options# TOC</h4>

The following options are supported by all qsv decoders. <dl class=”table”> <dt><samp class=”option”><var class=”var”>async_depth</var></samp></dt> <dd> Internal parallelization depth, the higher the value the higher the latency. </dd> <dt><samp class=”option”><var class=”var”>gpu_copy</var></samp></dt> <dd> A GPU-accelerated copy between video and system memory

<dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt>
<dt>‘<samp class="samp">on</samp>’</dt> <dt>‘<samp
class="samp">off</samp>’</dt> </dl>

</dd> </dl>

 <h4 class="subsection">10.9.2 HEVC Options# TOC</h4> Extra options for hevc_qsv. <dl class="table"> <dt><samp class="option"><var class="var">load_plugin</var></samp></dt> <dd> A user plugin to load in an internal session <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">hevc_sw</samp>’<
<dt>‘<samp class="samp">hevc_hw</samp>’</dt> </dl>

</dd> <dt><samp class="option"><var class="var">load_plugins</var></samp></dt>
<dd> A :-separate list of hexadecimal plugin UIDs to load in an internal session </dd> </dl>

 <h3 class="section">10.10 v210#
TOC</h3>

Uncompressed 4:2:2 10-bit decoder.
<h4 class="subsection">10.10.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">custom_stride</samp></dt>
<dd> Set the line size of the v210 data in bytes. The default value is 0 (autodetect). You can use the special -1 value for a strideless v210 as seen in BOXX files. </dd> </dl>

 <h2 class="chapter">11 Audio Decoders# TOC</h2>

A description of some of the currently available audio decoders follows. <h3 class="section">11.1 ac3#
TOC</h3>

AC-3 audio decoder. This decoder implements part of ATSC A/52:2010 and ETSI TS 102 366, as well as the undocumented RealAudio 3 (a.k.a. dnet).
<h4 class="subsection">11.1.1 AC-3 Decoder Options<a class="anchor hidden-xs" href="#AC_002d3-Decoder-Options"

aria-hidden="true"># TOC</h4>

<dl class="table"> <dt><samp class="option">-drc_scale <var class="var">value</var></samp></dt> <dd> Dynamic Range Scale Factor. The factor to apply to dynamic range values from the AC-3 stream. This factor is applied exponentially. The default value is 1. There are 3 notable scale factor ranges: <dl class="table"> <dt><samp class="option">drc_scale == 0</samp></dt> <dd> DRC disabled. Produces full range audio. </dd> <dt><samp class="option">0 < drc_scale <= 1</samp></dt> <dd> DRC enabled. Applies a fraction of the stream DRC value. Audio reproduction is between full range and full compression. </dd> <dt><samp class="option">drc_scale > 1</samp></dt> <dd> DRC enabled. Applies drc_scale asymmetrically. Loud sounds are fully compressed. Soft sounds are enhanced. </dd> </dl>

</dd> </dl>

 <h3 class="section">11.2 dca# TOC</h3>

DCA (DTS Coherent Acoustics) audio decoder, including the DTS-HD extensions. <h4 class="subsection">11.2.1 DCA Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-downmix <var class="var">channel_layout</var></samp <dd> Request a smaller channel layout from the decoder: ‘<samp class="samp">stereo</samp>’, ‘<samp class="samp">5.0(side)</samp>’, ‘<samp class="samp">5.1(side)</samp>’ or ‘<samp class="samp">7.1</samp>’. A DTS:X stream in the standard presentation decodes to 7.1.4 by default. Its four height channels are carried in an extension at the end of each lossless (XLL) frame, and the encoder also folds them into the 7.1 bed, so the decoder removes them from the bed when it outputs them. Requesting ‘<samp class="samp">7.1</samp>’ outputs the bed as coded instead, with the heights folded in, and so does every smaller layout. A ‘<samp class="samp">7.1</samp>’ request has no effect on other streams. </dd> </dl>

 <h3 class="section">11.3 flac# TOC</h3>

FLAC audio decoder. This decoder aims to implement the complete FLAC specification from Xiph. <a name="FLAC-

Decoder-options”> <h4 class=”subsection”>11.3.1 FLAC Decoder options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>-use_buggy_lpc</samp></dt> <dd> The lavc FLAC encoder used to produce buggy streams with high lpc values (like the default value). This option makes it possible to decode such streams correctly by using lavc’s old buggy lpc logic for decoding. </dd> </dl>

 <h3 class=”section”>11.4 ffwavesynth# TOC</h3>

Internal wave synthesizer. This decoder generates wave patterns according to predefined sequences. Its use is purely internal and the format of the data it accepts is not publicly documented. <h3 class=”section”>11.5 libgsm# TOC</h3>

libgsm decoder wrapper. libgsm allows libavcodec to decode the GSM full rate audio codec. Requires the presence of the libgsm headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libgsm</code>. This decoder supports both the ordinary GSM and the Microsoft variant. <h3 class=”section”>11.6 libilbc# TOC</h3>

libilbc decoder wrapper. libilbc allows libavcodec to decode the Internet Low Bitrate Codec (iLBC) audio codec. Requires the presence of the libilbc headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libilbc</code>. <h4 class=”subsection”>11.6.1 Options# TOC</h4>

The following option is supported by the libilbc wrapper. <dl class=”table”> <dt><samp class=”option”>enhance</samp></dt> <dd> Enable the enhancement of the decoded audio when set to 1. The default value is 0 (disabled). </dd> </dl>

 <h3 class=”section”>11.7 libmpeghdec<span

class=”pull-right”># TOC</h3>

libmpeghdec decoder wrapper. libmpeghdec allows libmpeghdec to decode the MPEG-H 3D audio codec. Requires the presence of the libmpeghdec headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libmpeghdec --enable-nonfree</code>. <h3 class=”section”>11.8 libopencore-amrnb# TOC</h3>

libopencore-amrnb decoder wrapper. libopencore-amrnb allows libavcodec to decode the Adaptive Multi-Rate Narrowband audio codec. Using it requires the presence of the libopencore-amrnb headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopencore-amrnb</code>. An FFmpeg native decoder for AMR-NB exists, so users can decode AMR-NB without this library. <h3 class=”section”>11.9 libopencore-amrwb# TOC</h

libopencore-amrwb decoder wrapper. libopencore-amrwb allows libavcodec to decode the Adaptive Multi-Rate Wideband audio codec. Using it requires the presence of the libopencore-amrwb headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopencore-amrwb</code>. An FFmpeg native decoder for AMR-WB exists, so users can decode AMR-WB without this library. <h3 class=”section”>11.10 libopus# TOC</h3>

libopus decoder wrapper. libopus allows libavcodec to decode the Opus Interactive Audio Codec. Requires the presence of the libopus headers and library during configuration. You need to explicitly configure the build with <code class=”code”>--enable-libopus</code>. An FFmpeg native decoder for Opus exists, so users can decode Opus without this library. <h3 class=”section”>11.11 pcm_dvda# TOC</h3>

PCM DVD-Audio decoder. Decodes the LPCM variant stored in DVD-Audio AOB files. Unlike DVD-Video LPCM it supports sample rates

up to 192 kHz and allows splitting the channels into two groups with different quantization.

 <h2 class="chapter">12 Subtitles Decoders# TOC</h2>

 <h3 class="section">12.1 libaribb24# TOC</h3>

ARIB STD-B24 caption decoder. Implements profiles A and C of the ARIB STD-B24 standard. <h4 class="subsection">12.1.1 libaribb24 Decoder Options# TOC</h4>

<dl class="table"> <dt><samp class="option">-aribb24-base-path <var class="var">path</var></samp></dt> <dd> Sets the base path for the libaribb24 library. This is utilized for reading of configuration files (for custom unicode conversions), and for dumping of non-text symbols as images under that location. Unset by default. </dd> <dt><samp class="option">-aribb24-skip-ruby-text <var class="var">boolean</var></samp></dt> <dd> Tells the decoder wrapper to skip text blocks that contain half-height ruby text. Enabled by default. </dd> </dl>

 <h3 class="section">12.2 libaribcaption# TOC</h3>

Yet another ARIB STD-B24 caption decoder using external libaribcaption library. Implements profiles A and C of the Japanese ARIB STD-B24 standard, Brazilian ABNT NBR 15606-1, and Philippines version of ISDB-T. Requires the presence of the libaribcaption headers and library (https://github.com/xqq/libaribcaption) during configuration. You need to explicitly configure the build with <code class="code">--enable-libaribcaption</code>. If both libaribb24 and libaribcaption are enabled, libaribcaption decoder precedes. <h4 class="subsection">12.2.1 libaribcaption Decoder Options<a class="anchor hidden-xs" href="#libaribcaption-

Decoder-Options" aria-hidden="true"># TOC</h4>

<dl class="table"> <dt><samp class="option">-sub_type <var class="var">subtitle_type</var></samp>
<dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt>‘<samp class="samp">bitmap</samp>’</dt>
<dd> Graphical image. </dd> <dt>‘<samp class="samp">ass</samp>’</dt>
<dd> ASS formatted text. </dd> <dt>‘<samp class="samp">text</samp>’</dt>
<dd> Simple text based output without formatting. </dd> </dl>

The default is ass as same as libaribb24 decoder. Some present players (e.g., mpv) expect ASS format for ARIB caption. </dd> <dt><samp class="option">-caption_encoding <var class="var">encoding_scheme</var></samp></dt>
<dd> Specifies the encoding scheme of input subtitle text. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Automatically detect text encoding (default). </dd>
<dt>‘<samp class="samp">jis</samp>’</dt> <dd> 8bit-char JIS encoding defined in ARIB STD B24. This encoding used in Japan for ISDB captions. </dd> <dt>‘<samp class="samp">utf8</samp>’</dt>
<dd> UTF-8 encoding defined in ARIB STD B24. This encoding is used in Philippines for ISDB-T captions. </dd> <dt>‘<samp class="samp">latin</samp>’</dt> <dd> Latin character encoding defined in ABNT NBR 15606-1. This encoding is used in South America for SBTVD / ISDB-Tb captions. </dd> </dl>

</dd> <dt><samp class="option">-font <var class="var">font_name[,font_name2,...]</var></samp></
<dd> Specify comma-separated list of font family names to be used for bitmap or ass type subtitle rendering. Only first font name is used for ass type subtitle. If not specified, use internally defined default font family. </dd> <dt><samp class="option">-ass_single_rect <var class="var">boolean</var></samp></dt> <dd> ARIB STD-B24 specifies that some captions may be displayed at different positions at a time (multi-rectangle subtitle). Since some players (e.g., old mpv) can’t handle multiple ASS rectangles in a single AVSubtitle, or multiple ASS rectangles of indeterminate duration with the same start timestamp, this option can change the behavior so that all the texts are displayed in a single ASS rectangle. The default is <var class="var">false</var>. If your player cannot handle AVSubtitles with multiple ASS rectangles properly, set this option to <var class="var">true</var> or define <code class="env">ASS_SINGLE_RECT=1</code> to change default behavior at compilation. </dd> <dt><samp class="option">-force_outline_text <var class="var">boolean</var></samp></dt>
<dd> Specify whether always render outline text for all characters regardless of the indication by character style. The default is <var class="var">false</var>. </dd> <dt><samp

class=”option”>-outline_width <var class=”var”>number</var> (0.0 - 3.0)</samp></dt> <dd> Specify width for outline text, in dots (relative). The default is <var class=”var”>1.5</var>. </dd> <dt><samp class=”option”>-ignore_background <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore background color rendering. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-ignore_ruby <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to ignore rendering for ruby-like (furigana) characters. The default is <var class=”var”>false</var>. </dd> <dt><samp class=”option”>-replace_drcs <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to render replaced DRCS characters as Unicode characters. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_ascii <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) fullwidth alphanumerics with halfwidth alphanumerics. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_japanese <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace some MSZ (Middle Size; half width) fullwidth japanese special characters with halfwidth ones. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-replace_msz_glyph <var class=”var”>boolean</var></samp></dt> <dd> Specify whether to replace MSZ (Middle Size; half width) characters with halfwidth glyphs if the fonts supports it. This option works under FreeType or DirectWrite renderer with Adobe-Japan1 compliant fonts. e.g., IBM Plex Sans JP, Morisawa BIZ UDGothic, Morisawa BIZ UDMincho, Yu Gothic, Yu Mincho, and Meiryo. The default is <var class=”var”>true</var>. </dd> <dt><samp class=”option”>-canvas_size <var class=”var”>image_size</var></samp></dt> <dd> Specify the resolution of the canvas to render subtitles to; usually, this should be frame size of input video. This only applies when <code class=”code”>-subtitle_type</code> is set to <var class=”var”>bitmap</var>. The libaribcaption decoder assumes input frame size for bitmap rendering as below: <ol class=”enumerate”> PROFILE_A : 1440 x 1080 with SAR (PAR) 4:3 PROFILE_C : 320 x 180 with SAR (PAR) 1:1

If actual frame size of input video does not match above assumption, the rendered captions may be distorted. To make the captions undistorted, add <code class=”code”>-canvas_size</code> option to specify actual input video size. Note that the <code class=”code”>-canvas_size</code> option is not required for video with different size but same aspect ratio. In such cases, the caption will be stretched or shrunk to actual video size if <code class=”code”>-canvas_size</code> option is not specified. If <code class=”code”>-canvas_size</code> option is specified with different size, the

caption will be stretched or shrunk as specified size with calculated SAR.
</dd> </dl>

 <h4 class="subsection">12.2.2 libaribcaption decoder usage examples# TOC</h4>

Display MPEG-TS file with ARIB subtitle by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap MPEG.TS </pre>

Display MPEG-TS file with input frame size 1920x1080 by <code class="code">ffplay</code> tool: <pre class="example-preformatted">ffplay -sub_type bitmap -canvas_size 1920x1080 MPEG.TS </pre>

Embed ARIB subtitle in transcoded video: <pre class="example-preformatted">ffmpeg -sub_type bitmap -i src.m2t -filter_complex "[0:v][0:s]overlay" -vcodec h264 dest.mp4 </pre>

 <h3 class="section">12.3 dvbsub# TOC</h3>

 <h4 class="subsection">12.3.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">compute_clut</samp></dt> <dd><dl class="table"> <dt><samp class="option">-2</samp></dt> <dd> Compute clut once if no matching CLUT is in the stream. </dd> <dt><samp class="option">-1</samp></dt> <dd> Compute clut if no matching CLUT is in the stream. </dd> <dt><samp class="option">0</samp></dt> <dd> Never compute CLUT </dd> <dt><samp class="option">1</samp></dt> <dd> Always compute CLUT and override the one provided in the stream. </dd> </dl> </dd> <dt><samp class="option">dvb_substream</samp></dt> <dd> Selects the dvb substream, or all substreams if -1 which is default. </dd> </dl>

 <h3 class="section">12.4 dvdsub# TOC</h3>

This codec decodes the bitmap subtitles used in DVDs; the same subtitles can also be found in VobSub file pairs and in some Matroska files. <h4 class="subsection">12.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">palette</samp></dt> <dd> Specify the global palette used by the bitmaps. When stored in VobSub, the palette is normally specified in the index file; in Matroska, the palette is stored in the codec extra-data in the same format as in VobSub. In DVDs, the palette is stored in the IFO file, and therefore not available when reading from dumped VOB files. The format for this option is a string containing 16 24-bits hexadecimal numbers (without 0x prefix) separated by commas, for example <code class="code">0d00ee, ee450d, 101010, eaeaea, 0ce60b, ec14ed, ebff0b, 0d617a, 7b7b7b, d1d1d1, 7b2a0e, 0d950c, 0f007b, cf0dec, cfa80c, 7c127b</code>. </dd> <dt><samp class="option">ifo_palette</samp></dt> <dd> Specify the IFO file from which the global palette is obtained. (experimental) </dd> <dt><samp class="option">forced_subs_only</samp></dt> <dd> Only decode subtitle entries marked as forced. Some titles have forced and non-forced subtitles in the same track. Setting this flag to <code class="code">1</code> will only keep the forced subtitles. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">12.5 libzvbi-teletext# TOC</h3>

Libzvbi allows libavcodec to decode DVB teletext pages and DVB teletext subtitles. Requires the presence of the libzvbi headers and library during configuration. You need to explicitly configure the build with <code class="code">--enable-libzvbi</code>. <h4 class="subsection">12.5.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">txt_page</samp></dt> <dd> List of teletext page numbers to decode. Pages that do not match the specified list are dropped. You may use the special <code class="code">*</code> string to match all pages, or <code class="code">subtitle</code> to match all subtitle pages. Default value is *. </dd> <dt><samp class="option">txt_default_region</samp></dt> <dd> Set default character set used for decoding, a value between 0 and 87 (see ETS 300 706, Section 15, Table 32). Default value is -1, which does not override the libzvbi default. This option is needed for some legacy level 1.0 transmissions which cannot signal the proper charset.

</dd> <dt><samp class="option">txt_chop_top</samp></dt> <dd> Discards the top teletext line. Default value is 1. </dd> <dt><samp class="option">txt_format</samp></dt> <dd> Specifies the format of the decoded subtitles. <dl class="table"> <dt><samp class="option">bitmap</samp></dt> <dd> The default format, you should use this for teletext pages, because certain graphics and colors cannot be expressed in simple text or even ASS. </dd> <dt><samp class="option">text</samp></dt> <dd> Simple text based output without formatting. </dd> <dt><samp class="option">ass</samp></dt> <dd> Formatted ASS output, subtitle pages and teletext pages are returned in different styles, subtitle pages are stripped down to text, but an effort is made to keep the text alignment and the formatting. </dd> </dl> </dd> <dt><samp class="option">txt_left</samp></dt> <dd> X offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_top</samp></dt> <dd> Y offset of generated bitmaps, default is 0. </dd> <dt><samp class="option">txt_chop_spaces</samp></dt> <dd> Chops leading and trailing spaces and removes empty lines from the generated text. This option is useful for teletext based subtitles where empty spaces may be present at the start or at the end of the lines or empty lines may be present between the subtitle lines because of double-sized teletext characters. Default value is 1. </dd> <dt><samp class="option">txt_duration</samp></dt> <dd> Sets the display duration of the decoded teletext pages or subtitles in milliseconds. Default value is -1 which means infinity or until the next subtitle event comes. </dd> <dt><samp class="option">txt_transparent</samp></dt> <dd> Force transparent background of the generated teletext bitmaps. Default value is 0 which means an opaque background. </dd> <dt><samp class="option">txt_opacity</samp></dt> <dd> Sets the opacity (0-255) of the teletext background. If <samp class="option">txt_transparent</samp> is not set, it only affects characters between a start box and an end box, typically subtitles. Default value is 0 if <samp class="option">txt_transparent</samp> is set, 255 otherwise. </dd> </dl>

 <h2 class="chapter">13 Bitstream Filters# TOC</h2>

When you configure your FFmpeg build, all the supported bitstream filters are enabled by default. You can list all available ones using the configure option <code class="code">--list-bsfs</code>. You can disable all the bitstream filters using the configure option <code class="code">--disable-bsfs</code>, and selectively enable any bitstream filter using the option <code class="code">--enable-bsf=BSF</code>, or you can disable a particular bitstream filter using the option <code class="code">--disable-bsf=BSF</code>. The option <code class="code">-bsfs</code>

of the ff* tools will display the list of all the supported bitstream filters included in your build. The ff* tools have a -bsf option applied per stream, taking a comma-separated list of filters, whose parameters follow the filter name after a ’=’.   <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v filter1[=opt1=str1:opt2=str2][,filter2] OUTPUT </pre>

Below is a description of the currently available bitstream filters, with their parameters, if any. <h3 class="section">13.1 aac_adtstoasc# TOC</h3>

Convert MPEG-2/4 AAC ADTS to an MPEG-4 Audio Specific Configuration bitstream. This filter creates an MPEG-4 AudioSpecificConfig from an MPEG-2/4 ADTS header and removes the ADTS header. This filter is required for example when copying an AAC stream from a raw ADTS AAC or an MPEG-TS container to MP4A-LATM, to an FLV file, or to MOV/MP4 files and related formats such as 3GP or M4A. Please note that it is auto-inserted for MP4A-LATM and MOV/MP4 and related formats. <h3 class="section">13.2 av1_metadata# TOC</h3>

Modify metadata embedded in an AV1 stream. <dl class="table"> <dt><samp class="option">td</samp></dt> <dd> Insert or remove temporal delimiter OBUs in all temporal units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dd> Insert a TD at the beginning of every TU which does not already have one. </dd> <dt>‘<samp class="samp">remove</samp>’</dt> <dd> Remove the TD from the beginning of every TU which has one. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the color description fields in the stream (see AV1 section 6.4.2). </dd> <dt><samp class="option">color_range</samp></dt> <dd> Set the color range in the stream (see AV1 section 6.4.2; note that this cannot be set for streams using BT.709 primaries, sRGB transfer characteristic and identity (RGB) matrix coefficients). <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dd> Limited range. </dd> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> Full range. </dd> </dl>

</dd> <dt><samp class="option">chroma_sample_position</samp></dt> <dd> Set the chroma sample location in the stream (see AV1 section

6.4.2). This can only be set for 4:2:0 streams. <dl class="table">
<dt>‘<samp class="samp">vertical</samp>’</dt> <dd> Left
position (matching the default in MPEG-2 and H.264). </dd>
<dt>‘<samp class="samp">colocated</samp>’</dt>
<dd> Top-left position. </dd> </dl>

</dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set
the tick rate (time_scale / num_units_in_display_tick)
in the timing info in the sequence header. </dd> <dt><samp
class="option">num_ticks_per_picture</samp></dt> <dd> Set the
number of ticks in each picture, to indicate that the stream has a fixed fram-
erate. Ignored if <samp class="option">tick_rate</samp> is not also set.
 </dd> <dt><samp class="option">delete_padding</samp></dt>
<dd> Deletes Padding OBUs. </dd> </dl>

 <h3 class="section">13.3 chomp<span class="pull-
right">#
TOC</h3>

Remove zero padding at the end of a packet.
<h3 class="section">13.4 dca_core<a class="anchor
hidden-xs" href="#dca_005fcore" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-dca_005fcore" aria-hidden="true">TOC</h3>

Extract the core from a DCA/DTS stream, dropping extensions such as
DTS-HD. <h3 class="section">13.5
dovi_rpu<a class="anchor hidden-xs" href="#dovi_005frpu"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
dovi_005frpu" aria-hidden="true">TOC</h3>

Manipulate Dolby Vision metadata in a HEVC/AV1 bitstream, optionally
enabling metadata compression. <dl class="table"> <dt><samp
class="option">strip</samp></dt> <dd> If enabled, strip all Dolby
Vision metadata (configuration record + RPU data blocks) from the stream.
</dd> <dt><samp class="option">compression</samp></dt>
<dd> Which compression level to enable. <dl class="table">
<dt>‘<samp class="samp">none</samp>’</dt> <dd> No
metadata compression. </dd> <dt>‘<samp class="samp">limited</samp>’</dt>
<dd> Limited metadata compression scheme. Should be compatible with
most devices. This is the default. </dd> <dt>‘<samp
class="samp">extended</samp>’</dt> <dd> Extended meta-
data compression. Devices are not required to support this. Note that this level
currently behaves the same as ‘<samp class="samp">limited</samp>’
in libavcodec. </dd> </dl> </dd> </dl>

 <h3 class="section">13.6 dovi_split<span
class="pull-right"><a class="anchor hidden-xs" href="#dovi_005fsplit"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
dovi_005fsplit" aria-hidden="true">TOC</h3>

Split a Dolby Vision Profile 7 multi-layer HEVC bitstream. Profile 7 carries the enhancement-layer HEVC bitstream interleaved inside the base-layer access units, wrapped in user-unspecified NAL units of type 63 (UNSPEC63), and the RPU metadata as a sibling user-unspecified NAL of type 62 (UNSPEC62). <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Which Dolby Vision components to keep in the output bitstream. <dl class="table"> <dt>‘<samp class="samp">bl</samp>’</dt> <dd> Base layer only: drop every UNSPEC63 (EL) and every UNSPEC62 (RPU). The output is a plain HEVC stream with no Dolby Vision markers. This is the default. </dd> <dt>‘<samp class="samp">bl_rpu</samp>’</dt> <dd> Base layer with the RPU NAL kept. </dd> <dt>‘<samp class="samp">el</samp>’</dt> <dd> Enhancement layer only: for every UNSPEC63 NAL, strip the two-byte outer NAL header and emit the inner payload. The result is a standalone HEVC bitstream. UNSPEC62 (RPU) is dropped. </dd> <dt>‘<samp class="samp">el_rpu</samp>’</dt> <dd> Enhancement layer with the RPU NAL kept verbatim. Same as ‘<samp class="samp">el</samp>’, but the UNSPEC62 RPU NALs are also emitted alongside the unwrapped EL NALs. </dd> </dl> </dd> </dl>

 <h3 class="section">13.7 dump_extra# TOC</h3>

Add extradata to the beginning of the filtered packets except when said packets already exactly begin with the extradata that is intended to be added. <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> The additional argument specifies which packets should be filtered. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">k</samp>’</dt> <dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> add extradata to all key packets </dd> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">all</samp>’</dt> <dd> add extradata to all packets </dd> </dl> </dd> </dl>

If not specified it is assumed ‘<samp class="samp">k</samp>’. For example the following <code class="command">ffmpeg</code> command forces a global header (thus disabling individual packet headers) in the H.264 packets generated by the <code class="code">libx264</code> encoder, but corrects them by adding the header stored in extradata to the key packets: <pre class="example-preformatted">ffmpeg -i INPUT -map 0 -flags:v +global_header -c:v libx264 -bsf:v dump_extra out.ts </pre>

 <h3 class="section">13.8 dv_error_marker<a class="anchor hidden-

xs" href="#dv_005ferror_005fmarker" aria-hidden="true"># TOC</h3>

Blocks in DV which are marked as damaged are replaced by blocks of the specified color. <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color to replace damaged blocks by </dd> <dt><samp class="option">sta</samp></dt> <dd> A 16 bit mask which specifies which of the 16 possible error status values are to be replaced by colored blocks. 0xFFFE is the default which replaces all non 0 error status values. <dl class="table"> <dt>‘<samp class="samp">ok</samp>’</dt> <dd> No error, no concealment </dd> <dt>‘<samp class="samp">err</samp>’</dt> <dd> Error, No concealment </dd> <dt>‘<samp class="samp">res</samp>’</dt> <dd> Reserved </dd> <dt>‘<samp class="samp">notok</samp>’</dt> <dd> Error or concealment </dd> <dt>‘<samp class="samp">notres</samp>’</dt> <dd> Not reserved </dd> <dt>‘<samp class="samp">Aa, Ba, Ca, Ab, Bb, Cb, A, B, C, a, b, erri, erru</samp>’</dt> <dd> The specific error status code </dd> </dl> see page 44-46 or section 5.5 of <a class="url" href="http://web.archive.org/web/20060927044735/http://www.smpte.org/ </dd> </dl>

 <h3 class="section">13.9 eac3_core# TOC</h3>

Extract the core from a E-AC-3 stream, dropping extra channels. <h3 class="section">13.10 eia608_to_smpte436m# TOC</h3>

Convert from a <code class="code">EIA_608</code> stream to a <code class="code">SMPTE_436M_ANC</code> data stream, wrapping the closed captions in CTA-708 CDP VANC packets. <dl class="table"> <dt><samp class="option">line_number</samp></dt> <dd> Choose which line number the generated VANC packets should go on. You generally want either line 9 (the default) or 11. </dd> <dt><samp class="option">wrapping_type</samp></dt> <dd> Choose the SMPTE 436M wrapping type, defaults to ‘<samp class="samp">vanc_frame</samp>’. It accepts the values: <dl class="table"> <dt>‘<samp class="samp">vanc_frame</samp>’</dt> <dd> VANC frame (interlaced or segmented progressive frame) </dd> <dt>‘<samp class="samp">vanc_field_1</samp>’</dt> <dt>‘<samp class="samp">vanc_field_2</samp>’</dt> <dt>‘<samp

class=”samp”>vanc_progressive_frame</samp>’</dt> </dl>
</dd> <dt><samp class=”option”>sample_coding</samp></dt>
<dd> Choose the SMPTE 436M sample coding, defaults to ‘<samp
class=”samp”>8bit_luma</samp>’. It accepts the values: <dl
class=”table”> <dt>‘<samp class=”samp”>8bit_luma</samp>’</dt>
<dd> 8-bit component luma samples </dd> <dt>‘<samp
class=”samp”>8bit_color_diff</samp>’</dt> <dd> 8-bit
component color difference samples </dd> <dt>‘<samp
class=”samp”>8bit_luma_and_color_diff</samp>’</dt> <dd> 8-
bit component luma and color difference samples </dd> <dt>‘<samp
class=”samp”>10bit_luma</samp>’</dt> <dd> 10-bit compo-
nent luma samples </dd> <dt>‘<samp class=”samp”>10bit_color_diff</samp>’</dt>
<dd> 10-bit component color difference samples </dd> <dt>‘<samp
class=”samp”>10bit_luma_and_color_diff</samp>’</dt> <dd> 10-
bit component luma and color difference samples </dd> <dt>‘<samp
class=”samp”>8bit_luma_parity_error</samp>’</dt> <dd> 8-
bit component luma samples with parity error </dd> <dt>‘<samp
class=”samp”>8bit_color_diff_parity_error</samp>’</dt> <dd> 8-
bit component color difference samples with parity error </dd>
<dt>‘<samp class=”samp”>8bit_luma_and_color_diff_parity_error</samp>’</dt>
<dd> 8-bit component luma and color difference samples with parity error
</dd> </dl> </dd> <dt><samp class=”option”>initial_cdp_sequence_cntr</samp></dt>
<dd> The initial value of the CDP’s 16-bit unsigned inte-
ger <code class=”code”>cdp_hdr_sequence_cntr</code> and <code
class=”code”>cdp_ftr_sequence_cntr</code> fields. Defaults to 0.
</dd> <dt><samp class=”option”>cdp_frame_rate</samp></dt>
<dd> Set the CDP’s <code class=”code”>cdp_frame_rate</code>
field. This doesn’t actually change the timing of the data stream, it just
changes the values inserted in that field in the generated CDP packets. Defaults
to ‘<samp class=”samp”>30000/1001</samp>’. </dd>
</dl>

 <h3 class=”section”>13.11
extract_extradata<a class=”anchor hidden-
xs” href=”#extract_005fextradata” aria-hidden=”true”># <a
class=”anchor hidden-xs”href=”#toc-extract_005fextradata” aria-hidden=”true”>TOC</h3>

Extract the in-band extradata. Certain codecs allow the
long-term headers (e.g. MPEG-2 sequence headers, or H.264/HEVC
(VPS/)SPS/PPS) to be transmitted either "in-band" (i.e. as a part
of the bitstream containing the coded frames) or "out of band"
(e.g. on the container level). This latter form is called "extradata"
in FFmpeg terminology. This bitstream filter detects the in-band
headers and makes them available as extradata. <dl class=”table”>
<dt><samp class=”option”>remove</samp></dt> <dd> When this
option is enabled, the long-term headers are removed from the bitstream after
extraction. </dd> </dl>

 <h3 class="section">13.12 filter_units# TOC</h3>

Remove units with types in or not in a given set from the stream. <dl class="table"> <dt><samp class="option">pass_types</samp></dt> <dd> List of unit types or ranges of unit types to pass through while removing all others. This is specified as a ’|’-separated list of unit type values or ranges of values with ’-’. </dd> <dt><samp class="option">remove_types</samp></dt> <dd> Identical to <samp class="option">pass_types</samp>, except the units in the given set removed and all others passed through. </dd> </dl>

The types used by pass_types and remove_types correspond to NAL unit types (nal_unit_type) in H.264, HEVC and H.266 (see Table 7-1 in the H.264 and HEVC specifications or Table 5 in the H.266 specification), to marker values for JPEG (without 0xFF prefix) and to start codes without start code prefix (i.e. the byte following the 0x000001) for MPEG-2. For VP8 and VP9, every unit has type zero. Extradata is unchanged by this transformation, but note that if the stream contains inline parameter sets then the output may be unusable if they are removed. For example, to remove all non-VCL NAL units from an H.264 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=pass_types=1-5' OUTPUT </pre>

To remove all AUDs, SEI and filler from an H.265 stream: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=35|38-40' OUTPUT </pre>

To remove all user data from a MPEG-2 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=178' OUTPUT </pre>

To remove all SEI from a H264 stream, including Closed Captions: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=6' OUTPUT </pre>

To remove all prefix and suffix SEI from a HEVC stream, including Closed Captions and dynamic HDR: <pre class="example-preformatted">ffmpeg -i INPUT -c:v copy -bsf:v 'filter_units=remove_types=39|40' OUTPUT </pre>

 <h3 class="section">13.13 hapqa_extract# TOC</h3>

Extract Rgb or Alpha part of an HAPQA file, without recompression, in order to create an HAPQ or an HAPAlphaOnly file. <dl class="table"> <dt><samp class="option">texture</samp></dt> <dd> Specifies the texture to keep. <dl class="table"> <dt><samp class="option">color</samp></dt> <dt><samp class="option">alpha</samp></dt> </dl>

</dd> </dl>

Convert HAPQA to HAPQ <pre class="example-preformatted">ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=color -tag:v HapY -metadata:s:v:0 encoder="HAPQ" hapq_file.mov </pre>

Convert HAPQA to HAPAlphaOnly <pre class="example-preformatted">ffmpeg -i hapqa_inputfile.mov -c copy -bsf:v hapqa_extract=texture=alpha -tag:v HapA -metadata:s:v:0 encoder="HAPAlpha Only" hapalphaonly_file.mov </pre>

 <h3 class="section">13.14 h264_metadata# TOC</h3>

Modify metadata embedded in an H.264 stream. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">pass</samp>’</dt> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</
</dl>

Default is pass. </dd> <dt><samp class="option">sample_aspect_ratio</samp></dt> <dd> Set the sample aspect ratio of the stream in the VUI parameters. See H.264 table E-1. </dd> <dt><samp class="option">overscan_appropriate_flag</samp></dt> <dd> Set whether the stream is suitable for display using overscan or not (see H.264 section E.2.1). </dd> <dt><samp class="option">video_format</samp></dt> <dt><samp class="option">video_full_range_flag</samp> <dd> Set the video format in the stream (see H.264 section E.2.1 and table E-2). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.264 section E.2.1 and tables E-3, E-4 and E-5). </dd> <dt><samp class="option">chroma_sample_loc_type</samp></dt> <dd> Set the chroma sample location in the stream (see H.264 section E.2.1 and figure E-1). </dd> <dt><samp class="option">tick_rate</samp></dt> <dd> Set the tick rate (time_scale / num_units_in_tick) in the VUI parameters. This is the smallest time unit representable in the stream, and in many cases represents the field rate of the stream (double the frame rate). </dd> <dt><samp class="option">fixed_frame_rate_flag</samp></dt>

<dd> Set whether the stream has fixed framerate - typically this indicates that the framerate is exactly half the tick rate, but the exact meaning is dependent on interlacing and the picture structure (see H.264 section E.2.1 and table E-6). </dd> <dt><samp class="option">zero_new_constraint_set_flags</samp></dt> <dd> Zero constraint_set4_flag and constraint_set5_flag in the SPS. These bits were reserved in a previous version of the H.264 spec, and thus some hardware decoders require these to be zero. The result of zeroing this is still a valid bitstream. </dd> <dt><samp class="option">crop_left</samp></dt> <dt><samp class="option">crop_right</samp></dt> <dt><samp class="option">crop_top</samp></dt> <dt><samp class="option">crop_bottom</samp></dt> <dd> Set the frame cropping offsets in the SPS. These values will replace the current ones if the stream is already cropped. These fields are set in pixels. Note that some sizes may not be representable if the chroma is subsampled or the stream is interlaced (see H.264 section 7.4.2.1.1). </dd> <dt><samp class="option">sei_user_data</samp></dt> <dd> Insert a string as SEI unregistered user data. The argument must be of the form UUID+string, where the UUID is as hex digits possibly separated by hyphens, and the string can be anything. For example, ‘<samp class="samp">086f3693-b7b3-4f2c-9653-21492feee5b8+hello</samp>’ will insert the string “hello” associated with the given UUID. </dd> <dt><samp class="option">delete_filler</samp></dt> <dd> Deletes both filler NAL units and filler SEI messages. </dd> <dt><samp class="option">display_orientation</samp></dt> <dd> Insert, extract or remove Display orientation SEI messages. See H.264 section D.1.27 and D.2.27 for syntax and semantics. <dl class="table"> <dt>‘<samp class="samp">pass</samp>’</dt> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp class="samp">remove</samp>’</dt> <dt>‘<samp class="samp">extract</samp>’</dt> </dl>

Default is pass. Insert mode works in conjunction with <code class="code">rotate</code> and <code class="code">flip</code> options. Any pre-existing Display orientation messages will be removed in insert or remove mode. Extract mode attaches the display matrix to the packet as side data. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Set rotation in display orientation SEI (anticlockwise angle in degrees). Range is -360 to +360. Default is NaN. </dd> <dt><samp class="option">flip</samp></dt> <dd> Set flip in display orientation SEI. <dl class="table"> <dt>‘<samp class="samp">horizontal</samp>’</dt> <dt>‘<samp class="samp">vertical</samp>’</dt> </dl>

Default is unset. </dd> <dt><samp class="option">level</samp></dt> <dd> Set the level in the SPS. Refer to H.264 section A.3 and tables A-1 to A-5. The argument must be the name of a level (for example, ‘<samp class="samp">4.2</samp>’), a level_idc value (for

example, ‘<samp class="samp">42</samp>’), or the special name ‘<samp class="samp">auto</samp>’ indicating that the filter should attempt to guess the level from the input stream properties. </dd> </dl>

 <h3 class="section">13.15 h264_mp4toannexb# TOC</h3

Convert an H.264 bitstream from length prefixed mode to start code prefixed mode (as defined in the Annex B of the ITU-T H.264 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an H.264 stream to mpegts format with <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.mp4 -codec copy -bsf:v h264_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class="code">mpegts</code>) and raw H.264 (muxer <code class="code">h264</code>) output formats. <h3 class="section">13.16 h264_redundant_pps# TOC</h3>

This applies a specific fixup to some Blu-ray BDMV H264 streams which contain redundant PPSs. The PPSs modify irrelevant parameters of the stream, confusing other transformations which require the correct extradata. The encoder used on these impacted streams adds extra PPSs throughout the stream, varying the initial QP and whether weighted prediction was enabled. This causes issues after copying the stream into a global header container, as the starting PPS is not suitable for the rest of the stream. One side effect, for example, is seeking will return garbled output until a new PPS appears. This BSF removes the extra PPSs and rewrites the slice headers such that the stream uses a single leading PPS in the global header, which resolves the issue. <h3 class="section">13.17 hevc_metadata# TOC</h3>

Modify metadata embedded in an HEVC stream. <dl class="table"> <dt><samp class="option">aud</samp></dt> <dd> Insert or remove AUD NAL units in all access units of the stream. <dl class="table"> <dt>‘<samp class="samp">insert</samp>’</dt> <dt>‘<samp

class=”samp”>remove</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>sample_aspect_ratio</samp></dt>
<dd> Set the sample aspect ratio in the stream in the VUI parameters.
 </dd> <dt><samp class=”option”>video_format</samp></dt>
<dt><samp class=”option”>video_full_range_flag</samp></dt> <dd> Set
the video format in the stream (see H.265 section E.3.1 and table E.2).
</dd> <dt><samp class=”option”>colour_primaries</samp></dt>
<dt><samp class=”option”>transfer_characteristics</samp></dt>
<dt><samp class=”option”>matrix_coefficients</samp></dt> <dd> Set
the colour description in the stream (see H.265 section E.3.1 and tables E.3, E.4
and E.5). </dd> <dt><samp class=”option”>chroma_sample_loc_type</samp></dt>
<dd> Set the chroma sample location in the stream (see H.265 section E.3.1
and figure E.1). </dd> <dt><samp class=”option”>tick_rate</samp></dt>
<dd> Set the tick rate in the VPS and VUI parameters (time_scale /
num_units_in_tick). Combined with <samp class=”option”>num_ticks_poc_diff_one</samp>,
this can set a constant framerate in the stream. Note that it is likely to be over-
ridden by container parameters when the stream is in a container. </dd>
<dt><samp class=”option”>num_ticks_poc_diff_one</samp></dt>
<dd> Set poc_proportional_to_timing_flag in VPS and VUI and use
this value to set num_ticks_poc_diff_one_minus1 (see H.265 sections 7.4.3.1
and E.3.1). Ignored if <samp class=”option”>tick_rate</samp> is not also
set. </dd> <dt><samp class=”option”>crop_left</samp></dt>
<dt><samp class=”option”>crop_right</samp></dt> <dt><samp
class=”option”>crop_top</samp></dt> <dt><samp class=”option”>crop_bottom</samp></dt>
<dd> Set the conformance window cropping offsets in the SPS. These
values will replace the current ones if the stream is already cropped.
These fields are set in pixels. Note that some sizes may not be rep-
resentable if the chroma is subsampled (H.265 section 7.4.3.2.1).
</dd> <dt><samp class=”option”>width</samp></dt> <dt><samp
class=”option”>height</samp></dt> <dd> Set width and height af-
ter crop. </dd> <dt><samp class=”option”>level</samp></dt>
<dd> Set the level in the VPS and SPS. See H.265 section A.4 and
tables A.6 and A.7. The argument must be the name of a level
(for example, ‘<samp class=”samp”>5.1</samp>’), a <em
class=”emph”>general_level_idc value (for example, ‘<samp
class=”samp”>153</samp>’ for level 5.1), or the special name
‘<samp class=”samp”>auto</samp>’ indicating that the filter
should attempt to guess the level from the input stream properties.
</dd> </dl>

 <h3 class=”section”>13.18
hevc_mp4toannexb<a class=”anchor hidden-
xs” href=”#hevc_005fmp4toannexb” aria-hidden=”true”># <a
class=”anchor hidden-xshref=”#toc-hevc_005fmp4toannexb” aria-hidden=”true”>TOC</h3

Convert an HEVC/H.265 bitstream from length prefixed mode to start code

prefixed mode (as defined in the Annex B of the ITU-T H.265 specification). This is required by some streaming formats, typically the MPEG-2 transport stream format (muxer <code class="code">mpegts</code>). For example to remux an MP4 file containing an HEVC stream to mpegts format with <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.mp4 -codec copy -bsf:v hevc_mp4toannexb OUTPUT.ts </pre>

Please note that this filter is auto-inserted for MPEG-TS (muxer <code class="code">mpegts</code>) and raw HEVC/H.265 (muxer <code class="code">h265</code> or <code class="code">hevc</code>) output formats. <h3 class="section">13.19 imxdump# TOC</h3>

Modifies the bitstream to fit in MOV and to be usable by the Final Cut Pro decoder. This filter only applies to the mpeg2video codec, and is likely not needed for Final Cut Pro 7 and newer with the appropriate <samp class="option">-tag:v</samp>. For example, to remux 30 MB/sec NTSC IMX to MOV: <pre class="example-preformatted">ffmpeg -i input.mxf -c copy -bsf:v imxdump -tag:v mx3n output.mov </pre>

 <h3 class="section">13.20 jpegapp# TOC</h3>

Inject JPEG APP0..APP15 segments from a length-prefixed sidecar into each MJPEG packet, after SOI. The sidecar is a sequence of records, one per encoded JPEG, in encode order: <pre class="example-preformatted">u32be length u8 payload[length] </pre>

<var class="var">length</var> does not include the 4-byte prefix and must not exceed 16 MiB. A zero <var class="var">length</var> leaves that packet unchanged. A non-empty payload must be a concatenation of complete APP segments (<code class="code">FF En</code> plus the 16-bit length that includes those two length bytes). If the payload starts with APP0 and the packet has an APP0 marker in the two bytes immediately following SOI, that APP0 segment is dropped. APP0 segments preceded by fill bytes or another segment are retained. An APP1-only payload keeps the encoder APP0: SOI, APP1, APP0, DQT, ... The filter requires <samp class="option">sidecar</samp>. Extra unused sidecar bytes are ignored; a packet without a matching record is an error. Records follow packets in arrival order. Flushing the filter does not rewind the sidecar, so a reused

context needs a sidecar positioned for the packets it will see next. A sidecar read or validation error prevents further record consumption by that filter context. To resume, create a new context with a sidecar matching the remaining packets. Flushing does not clear this error.   <pre class="example-preformatted">ffmpeg -i in.mp4 -c:v mjpeg -bsf:v jpegapp=sidecar=00000004DEADBEEF frame_%d.jpg </pre>

 <h3 class="section">13.21 mjpeg2jpeg# TOC</h3>

Convert MJPEG/AVI1 packets to full JPEG/JFIF packets. MJPEG is a video codec wherein each video frame is essentially a JPEG image. The individual frames can be extracted without loss, e.g. by   <pre class="example-preformatted">ffmpeg -i ../some_mjpeg.avi -c:v copy frames_%d.jpg </pre>

Unfortunately, these chunks are incomplete JPEG images, because they lack the DHT segment required for decoding. Quoting from <a class="url" href="http://www.digitalpreservation.gov/formats/fdd/fdd000063.shtml">http://www.digitalpreservation.go
 Avery Lee, writing in the rec.video.desktop newsgroup in 2001, commented that "MJPEG, or at least the MJPEG in AVIs having the MJPG fourcc, is restricted JPEG with a fixed – and *omitted* – Huffman table. The JPEG must be YCbCr colorspace, it must be 4:2:2, and it must use basic Huffman encoding, not arithmetic or progressive. . . . You can indeed extract the MJPEG frames and decode them with a regular JPEG decoder, but you have to prepend the DHT segment to them, or else the decoder won’t have any idea how to decompress the data. The exact table necessary is given in the OpenDML spec." This bitstream filter patches the header of frames extracted from an MJPEG stream (carrying the AVI1 header ID and lacking a DHT segment) to produce fully qualified JPEG images.   <pre class="example-preformatted">ffmpeg -i mjpeg-movie.avi -c:v copy -bsf:v mjpeg2jpeg frame_%d.jpg exiftran -i -9 frame*.jpg ffmpeg -i frame_%d.jpg -c:v copy rotated.avi </pre>

 <h3 class="section">13.22 mjpegadump# TOC</h3>

Add an MJPEG A header to the bitstream, to enable decoding by Quicktime. <h3 class="section">13.23 mov2textsub# TOC</h3>

Extract a representable text file from MOV subtitles, stripping the metadata header from each subtitle packet. See also the text2movsub filter. <h3 class="section">13.24 mpeg2_metadata# TOC</h3>

Modify metadata embedded in an MPEG-2 stream. <dl class="table"> <dt><samp class="option">display_aspect_ratio</samp></dt> <dd> Set the display aspect ratio in the stream. The following fixed values are supported: <dl class="table"> <dt><samp class="option">4/3</samp></dt> <dt><samp class="option">16/9</samp></dt> <dt><samp class="option">221/100</samp></dt> </dl> Any other value will result in square pixels being signalled instead (see H.262 section 6.3.3 and table 6-3). </dd> <dt><samp class="option">frame_rate</samp></dt> <dd> Set the frame rate in the stream. This is constructed from a table of known values combined with a small multiplier and divisor - if the supplied value is not exactly representable, the nearest representable value will be used instead (see H.262 section 6.3.3 and table 6-4). </dd> <dt><samp class="option">video_format</samp></dt> <dd> Set the video format in the stream (see H.262 section 6.3.6 and table 6-6). </dd> <dt><samp class="option">colour_primaries</samp></dt> <dt><samp class="option">transfer_characteristics</samp></dt> <dt><samp class="option">matrix_coefficients</samp></dt> <dd> Set the colour description in the stream (see H.262 section 6.3.6 and tables 6-7, 6-8 and 6-9). </dd> </dl>

 <h3 class="section">13.25 mpeg4_unpack_bframes# TOC</h3>

Unpack DivX-style packed B-frames. DivX-style packed B-frames are not valid MPEG-4 and were only a workaround for the broken Video for Windows subsystem. They use more space, can cause minor AV sync issues, require more CPU power to decode (unless the player has some decoded picture queue to compensate the 2,0,2,0 frame per packet style) and cause trouble if copied into a standard container like mp4 or mpeg-ps/ts, because MPEG-4 decoders may not be able to decode them, since they are not valid MPEG-4. For example to fix an AVI file containing an MPEG-4 stream with DivX-style packed B-frames using <code class="command">ffmpeg</code>, you can use the command: <pre class="example-preformatted">ffmpeg -i INPUT.avi -codec copy -bsf:v mpeg4_unpack_bframes OUTPUT.avi </pre>

 <h3 class="section">13.26 noise<span class="pull-

right”>#
TOC</h3>

Damages the contents of packets or simply drops them without damaging the container. Can be used for fuzzing or testing error resilience/concealment. Parameters: <dl class=”table”> <dt><samp class=”option”>amount</samp></dt> <dd> Accepts an expression whose evaluation per-packet determines how often bytes in that packet will be modified. A value below 0 will result in a variable frequency. Default is 0 which results in no modification. However, if neither amount nor drop is specified, amount will be set to <var class=”var”>-1</var>. See below for accepted variables. </dd> <dt><samp class=”option”>drop</samp></dt> <dd> Accepts an expression evaluated per-packet whose value determines whether that packet is dropped. Evaluation to a positive value results in the packet being dropped. Evaluation to a negative value results in a variable chance of it being dropped, roughly inverse in proportion to the magnitude of the value. Default is 0 which results in no drops. See below for accepted variables. </dd> <dt><samp class=”option”>dropamount</samp></dt> <dd> Accepts a non-negative integer, which assigns a variable chance of it being dropped, roughly inverse in proportion to the value. Default is 0 which results in no drops. This option is kept for backwards compatibility and is equivalent to setting drop to a negative value with the same magnitude i.e. <code class=”code”>dropamount=4</code> is the same as <code class=”code”>drop=-4</code>. Ignored if drop is also specified. </dd> </dl>

Both <code class=”code”>amount</code> and <code class=”code”>drop</code> accept expressions containing the following variables: <dl class=”table”> <dt>‘<samp class=”samp”>n</samp>’</dt> <dd> The index of the packet, starting from zero. </dd> <dt>‘<samp class=”samp”>tb</samp>’</dt> <dd> The timebase for packet timestamps. </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Packet presentation timestamp. </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Packet decoding timestamp. </dd> <dt>‘<samp class=”samp”>nopts</samp>’</dt> <dd> Constant representing AV_NOPTS_VALUE. </dd> <dt>‘<samp class=”samp”>startpts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE PTS seen in the stream. </dd> <dt>‘<samp class=”samp”>startdts</samp>’</dt> <dd> First non-AV_NOPTS_VALUE DTS seen in the stream. </dd> <dt>‘<samp class=”samp”>duration</samp>’</dt> <dt>‘<samp class=”samp”>d</samp>’</dt <dd> Packet duration, in timebase units. </dd> <dt>‘<samp class=”samp”>pos</samp>’</dt> <dd> Packet position in input; may be -1 when unknown or not set. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Packet size, in bytes. </dd> <dt>‘<samp class=”samp”>key</samp>’</dt>

<dd> Whether packet is marked as a keyframe. </dd> <dt>‘<samp class="samp">state</samp>’</dt> <dd> A pseudo random integer, primarily derived from the content of packet payload. </dd> </dl>

 <h4 class="subsection">13.26.1 Examples# TOC</h4> Apply modification to every byte but don’t drop any packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf noise=1 output.mkv </pre>

Drop every video packet not marked as a keyframe after timestamp 30s but do not modify any of the remaining packets. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v noise=drop='gt(pts*tb\,30)*not(key)' output.mkv </pre>

Drop one second of audio every 10 seconds and add some random noise to the rest. <pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:a noise=amount=-1:drop='between(mod(pts*tb\,10)\,9\,10)' output.mkv </pre>

 <h3 class="section">13.27 null# TOC</h3> This bitstream filter passes the packets through unchanged. <h3 class="section">13.28 pcm_rechunk# TOC</h3>

Repacketize PCM audio to a fixed number of samples per packet or a fixed packet rate per second. This is similar to the <a data-manual="ffmpeg-filters" href="./ffmpeg-filters.html#asetnsamples">asetnsamples audio filter but works on audio packets instead of audio frames. <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of samples per each output audio packet. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio packet with silence, so that it will contain the same number of samples (or roughly the same number of samples, see <samp class="option">frame_rate</samp>) as the previous ones. Default value is 1. </dd> <dt><samp class="option">frame_rate, r</samp></dt> <dd> This option makes the filter output a fixed number of packets per second instead of a fixed number of samples per packet. If the audio sample rate is not divisible by the frame rate then the number of samples will not be constant but will vary slightly so that

each packet will start as close to the frame boundary as possible. Using this option has precedence over <samp class=”option”>nb_out_samples</samp>. </dd> </dl>

You can generate the well known 1602-1601-1602-1601-1602 pattern of 48kHz audio for NTSC frame rate using the <samp class=”option”>frame_rate</samp> option. <pre class=”example-preformatted”>ffmpeg -f lavfi -i sine=r=48000:d=1 -c pcm_s16le -bsf pcm_rechunk=r=30000/1001 -f framecrc - </pre>

 <h3 class=”section”>13.29 pgs_frame_merge# TOC</h3>

Merge a sequence of PGS Subtitle segments ending with an "end of display set" segment into a single packet. This is required by some containers that support PGS subtitles (muxer <code class=”code”>matroska</code>). <h3 class=”section”>13.30 prores_metadata# TOC</h3>

Modify color property metadata embedded in prores stream. <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT601 625 </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> BT601 525 </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>smpte431</samp>’</dt> <dd> DCI P3 </dd> <dt>‘<samp class=”samp”>smpte432</samp>’</dt> <dd> P3 D65 </dd> </dl>

</dd> <dt><samp class=”option”>transfer_characteristics</samp></dt> <dd> Set the color transfer. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same transfer characteristics property (default). </dd> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT 601, BT 709, BT 2020 </dd> <dt>‘<samp class=”samp”>smpte2084</samp>’</dt> <dd> SMPTE ST 2084 </dd> <dt>‘<samp class=”samp”>arib-std-b67</samp>’</dt> <dd> ARIB STD-B67 </dd>

</dl>

</dd> <dt><samp class="option">matrix_coefficients</samp></dt>
<dd> Set the matrix coefficient. Available values are: <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> Keep the same colorspace property (default). </dd>
<dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>&rsqu
<dd> BT 601 </dd> <dt>‘<samp class="samp">bt2020nc</samp>’</dt>
</dl> </dd> </dl>

Set Rec709 colorspace for each frame of the file
<pre class="example-preformatted">ffmpeg -i INPUT -c copy -bsf:v
prores_metadata=color_primaries=bt709:color_trc=bt709:colorspace=bt709
output.mov </pre>

Set Hybrid Log-Gamma parameters for each frame of the file <div
class="example"> <pre class="example-preformatted">ffmpeg -i INPUT -c
copy -bsf:v prores_metadata=color_primaries=bt2020:color_trc=arib-std-
b67:colorspace=bt2020nc output.mov </pre>

 <h3 class="section">13.31 re-
move_extra<a class="anchor hidden-xs" href="#remove_005fextra"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
remove_005fextra" aria-hidden="true">TOC</h3>

Remove extradata from packets. It accepts the following param-
eter: <dl class="table"> <dt><samp class="option">freq</samp></dt>
<dd> Set which frame types to remove extradata from. <dl
class="table"> <dt>‘<samp class="samp">k</samp>’</dt>
<dd> Remove extradata from non-keyframes only. </dd>
<dt>‘<samp class="samp">keyframe</samp>’</dt> <dd> Remove
extradata from keyframes only. </dd> <dt>‘<samp
class="samp">e, all</samp>’</dt> <dd> Remove extradata
from all frames. </dd> </dl> </dd> </dl>

 <h3 class="section">13.32 setts<span class="pull-
right">#
TOC</h3>
Set PTS and DTS in packets. It accepts the following parame-
ters: <dl class="table"> <dt><samp class="option">ts</samp></dt>
<dt><samp class="option">pts</samp></dt> <dt><samp class="option">dts</samp></dt>
<dd> Set expressions for PTS, DTS or both. </dd> <dt><samp
class="option">duration</samp></dt> <dd> Set expression for dura-
tion. </dd> <dt><samp class="option">time_base</samp></dt>
<dd> Set output time base. </dd> <dt><samp class="option">prescale</samp></dt>
<dd> Set whether to convert time fields to user-set output time base before
evaluation of expressions. Defaults to 0. </dd> </dl>

The expressions are evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The count of the input packet. Starting from 0. </dd> <dt><samp class="option">TS</samp></dt> <dd> The demux timestamp in input in case of <code class="code">ts</code> or <code class="code">dts</code> option or presentation timestamp in case of <code class="code">pts</code> option. </dd> <dt><samp class="option">POS</samp></dt> <dd> The original position in the file of the packet, or undefined if undefined for the current packet </dd> <dt><samp class="option">DTS</samp></dt> <dd> The demux timestamp in input. </dd> <dt><samp class="option">PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class="option">DURATION</samp></dt> <dd> The duration in input. </dd> <dt><samp class="option">STARTDTS</samp></dt> <dd> The DTS of the first packet. </dd> <dt><samp class="option">STARTPTS</samp></dt> <dd> The PTS of the first packet. </dd> <dt><samp class="option">PREV_INDTS</samp></dt> <dd> The previous input DTS. </dd> <dt><samp class="option">PREV_INPTS</samp></dt> <dd> The previous input PTS. </dd> <dt><samp class="option">PREV_INDURATION</samp></dt> <dd> The previous input duration. </dd> <dt><samp class="option">PREV_OUTDTS</samp></dt> <dd> The previous output DTS. </dd> <dt><samp class="option">PREV_OUTPTS</samp></dt> <dd> The previous output PTS. </dd> <dt><samp class="option">PREV_OUTDURATION</samp></dt> <dd> The previous output duration. </dd> <dt><samp class="option">NEXT_DTS</samp></dt> <dd> The next input DTS. </dd> <dt><samp class="option">NEXT_PTS</samp></dt> <dd> The next input PTS. </dd> <dt><samp class="option">NEXT_DURATION</samp></dt> <dd> The next input duration. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase in which time fields are denominated. The user-set output timebase if prescale is enabled, else the input timebase. </dd> <dt><samp class="option">TB_OUT</samp></dt> <dd> The output timebase. </dd> <dt><samp class="option">SR</samp></dt> <dd> The sample rate of stream packet belongs. </dd> <dt><samp class="option">NOPTS</samp></dt> <dd> The AV_NOPTS_VALUE constant. </dd> </dl>

For example, to set PTS equal to DTS (not recommended if B-frames are involved): <pre class="example-preformatted">ffmpeg -i INPUT -c:a copy -bsf:a setts=pts=DTS out.mkv </pre>

 <h3 class="section">13.33 showinfo# TOC</h3> Log basic packet information. Mainly useful for testing, debugging, and development. <h3 class="section">13.34 side_data<span

class=”pull-right”># TOC</h3> Remove side data from packets and from the stream, or select packets by their side data. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>remove</samp></dt> <dd> Set the side data types to remove, joined with ‘<samp class=”samp”>+</samp>’, from the packets and from the side data of the stream. Each type is given by its handle; <code class=”command”>ffmpeg -h bsf=side_data</code> lists them. </dd> <dt><samp class=”option”>select</samp></dt> <dd> Set the side data types of the packets to keep, joined with ‘<samp class=”samp”>+</samp>’. A packet is kept if it carries any of them, and dropped otherwise. The selection looks at the side data before <samp class=”option”>remove</samp> applies. Dropping a packet breaks the decoding of the packets that depend on it, so this is meant for analysis or for streams where each packet decodes on its own. </dd> </dl>

For example, to drop the display matrix and the replay gain of a stream: <pre class=”example-preformatted”>ffmpeg -i INPUT -c copy -bsf:v side_data=remove=displaymatrix+replaygain output.mkv </pre>

 <h3 class=”section”>13.35 smpte436m_to_eia608# TOC</h3>

Convert from a <code class=”code”>SMPTE_436M_ANC</code> data stream to a <code class=”code”>EIA_608</code> stream, extracting the closed captions from CTA-708 CDP VANC packets, and ignoring all other data. <h3 class=”section”>13.36 text2movsub# TOC</h3>

Convert text subtitles to MOV subtitles (as used by the <code class=”code”>mov_text</code> codec) with metadata headers. See also the mov2textsub filter. <h3 class=”section”>13.37 trace_headers# TOC</h3>

Log trace output containing all syntax elements in the coded stream headers (everything above the level of individual coded blocks). This can be useful

for debugging low-level stream issues. Supports AV1, H.264, H.265, (M)JPEG, MPEG-2 and VP9, but depending on the build only a subset of these may be available. <h3 class="section">13.38 trim# TOC</h3>

Drop packets outside a given range, similar to the trim and atrim filters, but operating on packets instead of decoded frames. The range is described by a <samp class="option">start</samp> and an <samp class="option">end</samp> bound. At least one of them must be set. Each bound is fully described by three independent choices: <ul class="itemize mark-bullet"> The discriminant it is compared against: the packet’s presentation timestamp, its decoding timestamp, or its index. The two bounds need not use the same one, for instance a <code class="code">dts</code> start with a <code class="code">pts</code> end. The unit its value is given in: the stream time base, milliseconds, or packets. What it is counted from: zero, or its reference packet when <samp class="option">start_rel</samp> / <samp class="option">end_rel</samp> is set. For <samp class="option">start</samp> the reference packet is the first packet carrying that discriminant. For <samp class="option">end</samp> it is the first exported packet.

The <samp class="option">start_type</samp> and <samp class="option">end_type</samp> options name a combination of the three. The <code class="code">dur_ts</code> and <code class="code">dur_t_msec</code> end types are the always relative forms of <code class="code">pts</code> and <code class="code">msec_pt</code>. A zero duration therefore produces an empty stream. Every packet is judged on its own discriminant, and on nothing else. A packet not carrying the discriminant its bound uses cannot be placed, and is dropped. <samp class="option">start_rel</samp> makes <samp class="option">start</samp> an offset from a reference packet instead of an absolute value. The reference is the first packet carrying the discriminant that bound uses. Its value is added to the offset once, and the bound stays there for the rest of the stream. <samp class="option">end_rel</samp> anchors on the first exported packet instead, at the position <samp class="option">start</samp> trimmed it to. The two bounds therefore move together. Only a discriminant that cannot come back into the range ends the stream, namely the packet index or the decoding timestamp. End of stream is signaled once one of those passes the <samp class="option">end</samp> bound. A presentation timestamp past the end only drops its packet, so that later packets still inside the range are not lost. A bound given in a time unit can fall inside a packet rather than between two. Such a packet is trimmed. Set <samp class="option">trim_packets</samp> to false to keep every packet exactly as it came in: exported whole if its own value is inside the range, dropped otherwise. Audio is trimmed in sample space, through skip samples

side data. The timestamps and duration are left untouched, and a decoder discards the signaled samples on output. This needs a sample rate and a time base. Audio missing either is rejected rather than trimmed on the timeline. Every other media type is trimmed by shifting the packet timestamps and shortening its duration. Trimming inside an audio packet needs the number of samples it decodes to. Where the codec does not give that number, the packet is handled as though <samp class="option">trim_packets</samp> were false. A skip larger than the packet carrying it continues into the packets after it. Dropping such a packet carries the remainder over to the next one. A skip that packet already carries replaces the remainder, as a decoder would, and the larger of it and this filter’s own trim wins. By default a packet outside the range is dropped, whether or not later packets depend on it. Streams of independent frames, such as ADTS or intra only video, can be cut anywhere. Cutting an inter coded stream inside a group of pictures leaves the first packets in range undecodable. Set <samp class="option">preroll</samp> to export the packets those first packets depend on as well. They are flagged for a decoder to decode and then drop, so the frames it outputs still begin at the <samp class="option">start</samp> bound. Only a decoder acts on that flag. Use <samp class="option">preroll</samp> where the stream is decoded. A muxer writes the extra packets like any other. A remux then covers more than the requested range. <samp class="option">preroll</samp> covers the start of the range only. On a stream with B-frames an <samp class="option">end</samp> bound on the presentation timestamp can drop a packet the last frames in the range need. Those frames then decode incorrectly. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Time or index marking the start of the accepted range. Packets before it are dropped. A packet straddling it is trimmed at its start, or dropped when <samp class="option">trim_packets</samp> is false. </dd> <dt><samp class="option">start_type</samp></dt> <dd> How to interpret <samp class="option">start</samp>: <dl class="table"> <dt>‘<samp class="samp">pts</samp>’</dt> <dd> Presentation timestamp, in stream time base (default). </dd> <dt>‘<samp class="samp">dts</samp>’</dt> <dd> Decoding timestamp, in stream time base. </dd> <dt>‘<samp class="samp">pkt_index</samp>’</dt> <dd> Packet index, counting from zero. </dd> <dt>‘<samp class="samp">msec_pt</samp>’</dt> <dd> Milliseconds, matched against the presentation timestamp. </dd> <dt>‘<samp class="samp">msec_dt</samp>’</dt> <dd> Milliseconds, matched against the decoding timestamp. </dd> </dl>

</dd> <dt><samp class="option">start_rel</samp></dt> <dd> Count <samp class="option">start</samp> from the reference packet rather than from zero. Has no effect on ‘<samp class="samp">pkt_index</samp>’, whose first packet is already zero. Default false. </dd> <dt><samp

class=”option”>end</samp></dt> <dd> Time or index marking the end of the accepted range. Signals end of stream when reached. A packet straddling it is trimmed at its end, or exported untouched when <samp class=”option”>trim_packets</samp> is false. </dd> <dt><samp class=”option”>end_type</samp></dt> <dd> How to interpret <samp class=”option”>end</samp>. Same values as <samp class=”option”>start_type</samp>, plus: <dl class=”table”> <dt>‘<samp class=”samp”>dur_ts</samp>’</dt> <dd> Duration in stream time base. Same as ‘<samp class=”samp”>pts</samp>’ with <samp class=”option”>end_rel</samp> set. </dd> <dt>‘<samp class=”samp”>dur_t_msec</samp>’</dt> <dd> Duration in milliseconds. Same as ‘<samp class=”samp”>msec_pt</samp>’ with <samp class=”option”>end_rel</samp> set. </dd> </dl>

</dd> <dt><samp class=”option”>end_rel</samp></dt> <dd> Count <samp class=”option”>end</samp> from the first exported packet rather than from zero. The reference is that packet’s value on the discriminant <samp class=”option”>end_type</samp> names. On a timestamp it is taken at the position the packet is trimmed to. For audio that is the time of its first retained sample. The skip that packet carries can push it past the packet itself. With ‘<samp class=”samp”>pkt_index</samp>’ it exports a fixed number of packets, which can be combined with a timestamp-based <samp class=”option”>start</samp>. Always set for the duration types. Default false. </dd> <dt><samp class=”option”>trim_packets</samp></dt> <dd> Trim packets straddling a boundary. When false, packets are exported exactly as they came in, with no timestamp adjustment and no skip samples side data. A packet is then exported untouched if its value falls inside the range, and dropped otherwise. Default true. </dd> <dt><samp class=”option”>preroll</samp></dt> <dd> Export the packets a decoder needs to decode the first packet in range, rather than dropping them for falling before the <samp class=”option”>start</samp> bound. The filter holds every packet from the last keyframe on. It exports them ahead of the first packet in range, carrying <code class=”code”>AV_PKT_FLAG_DISCARD</code>. Video only. Default false. </dd> <dt><samp class=”option”>preroll_size</samp></dt> <dd> Maximum number of packets held for <samp class=”option”>preroll</samp>. A group of pictures longer than this is dropped whole, with a warning naming the count to raise it past. Default 300, which covers the keyframe intervals encoders commonly default to. </dd> </dl>

For example, to keep only the 10 first packets of an ADTS stream: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=end=10:end_type=pkt_index OUTPUT.aac </pre>

To keep 5 seconds starting 1.5 seconds into the stream: <div

class=”example”> <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

The same, expressed relative to the first packet so it works regardless of the stream’s starting timestamp: <pre class=”example-preformatted”>ffmpeg -i INPUT.aac -c copy -bsf:a trim=start=1500:start_type=msec_pt:start_rel=1:end=5000:end_type=dur_t_msec OUTPUT.aac </pre>

To start on a keyframeless point of an inter coded video stream, re-encoding it so that the preroll is decoded rather than written out. The filter must run before the decoder, so it is given before the input: <pre class=”example-preformatted”>ffmpeg -bsf:v trim=start=17000:start_type=msec_pt:end=30000:end_type=dur_t_msec:preroll=1 -i INPUT.mp4 OUTPUT.mp4 </pre>

 <h3 class=”section”>13.39 truehd_core# TOC</h3>

Extract the core from a TrueHD stream, dropping ATMOS data. <h3 class=”section”>13.40 vp9_metadata# TOC</h3>

Modify metadata embedded in a VP9 stream. <dl class=”table”> <dt><samp class=”option”>color_space</samp></dt> <dd> Set the color space value in the frame header. Note that any frame set to RGB will be implicitly set to PC range and that RGB is incompatible with profiles 0 and 2. <dl class=”table”> <dt>‘<samp class=”samp”>unknown</samp>’</dt> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dt>‘<samp class=”samp”>bt709</samp>’</d
<dt>‘<samp class=”samp”>smpte170</samp>’</dt>
<dt>‘<samp class=”samp”>smpte240</samp>’</dt>
<dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dt>‘<samp class=”samp”>rgb</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>color_range</samp></dt> <dd> Set the color range value in the frame header. Note that any value imposed by the color space will take precedence over this value. <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dt>‘<samp class=”samp”>pc</samp>’</dt> </dl> </dd> </dl>

 <h3 class=”section”>13.41 vp9_superframe<a class=”anchor hidden-xs” href=”#vp9_005fsuperframe”

aria-hidden=”true”># TOC</h3>

Merge VP9 invisible (alt-ref) frames back into VP9 superframes. This fixes merging of split/segmented VP9 streams where the alt-ref frame was split from its visible counterpart. <h3 class=”section”>13.42 vp9_superframe_split# TOC</h3>

Split VP9 superframes into single frames. <h3 class=”section”>13.43 vp9_raw_reorder# TOC</h3>

Given a VP9 stream with correct timestamps but possibly out of order, insert additional show-existing-frame packets to correct the ordering. <h2 class=”chapter”>14 Format Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the muxers and demuxers. In addition each muxer or demuxer may support so-called private options, which are specific for that component. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class=”code”>AVFormatContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class=”table”> <dt><samp class=”option”>avioflags <var class=”var”>flags</var> (input/output)</samp></dt> <dd> Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>direct</samp>’</dt> <dd> Reduce buffering. </dd> </dl>

</dd> <dt><samp class=”option”>probesize <var class=”var”>integer</var> (input)</samp></dt> <dd> Set probing size in bytes, i.e. the size of the data to analyze to get stream information. A higher value will enable detecting more information in case it is dispersed into the stream, but will increase latency. Must be an integer not lesser than 32. It is 5000000 by default. </dd> <dt><samp class=”option”>max_probe_packets <var class=”var”>integer</var> (input)</samp></dt> <dd> Set the maximum number of buffered packets when probing a codec. Default is 2500 packets. </dd> <dt><samp class=”option”>packetsize <var class=”var”>integer</var> (output)</samp></dt>

<dd> Set packet size. </dd> <dt><samp class="option">fflags
<var class="var">flags</var></samp></dt> <dd> Set format flags.
Some are implemented for a limited number of formats. Possible
values for input files: <dl class="table"> <dt>‘<samp
class="samp">discardcorrupt</samp>’</dt> <dd> Discard cor-
rupted packets. </dd> <dt>‘<samp class="samp">fastseek</samp>’</dt>
<dd> Enable fast, but inaccurate seeks for some formats. </dd>
<dt>‘<samp class="samp">genpts</samp>’</dt> <dd> Generate
missing PTS if DTS is present. </dd> <dt>‘<samp
class="samp">igndts</samp>’</dt> <dd> Ignore DTS if
PTS is also set. In case the PTS is set, the DTS value is set to NOPTS. This is
ignored when the <code class="code">nofillin</code> flag is set. </dd>
<dt>‘<samp class="samp">ignidx</samp>’</dt> <dd> Ignore
index. </dd> <dt>‘<samp class="samp">nobuffer</samp>’</dt>
<dd> Reduce the latency introduced by buffering during initial input
streams analysis. </dd> <dt>‘<samp class="samp">nofillin</samp>’</dt>
<dd> Do not fill in missing values in packet fields that can be exactly calcu-
lated. </dd> <dt>‘<samp class="samp">noparse</samp>’</dt>
<dd> Disable AVParsers, this needs <code class="code">+nofillin</code>
too. </dd> <dt>‘<samp class="samp">sortdts</samp>’</dt>
<dd> Try to interleave output packets by DTS. At present, available only
for AVIs with an index. </dd> </dl>

Possible values for output files: <dl class="table"> <dt>‘<samp
class="samp">autobsf</samp>’</dt> <dd> Automatically ap-
ply bitstream filters as required by the output format. Enabled by default.
</dd> <dt>‘<samp class="samp">bitexact</samp>’</dt>
<dd> Only write platform-, build- and time-independent data. This
ensures that file and data checksums are reproducible and match be-
tween platforms. Its primary use is for regression testing. </dd>
<dt>‘<samp class="samp">flush_packets</samp>’</dt>
<dd> Write out packets immediately. </dd> <dt>‘<samp
class="samp">shortest</samp>’</dt> <dd> Stop muxing at the
end of the shortest stream. It may be needed to increase max_interleave_delta
to avoid flushing the longer streams before EOF. </dd> </dl>

</dd> <dt><samp class="option">seek2any <var class="var">integer</var>
(input)</samp></dt> <dd> Allow seeking
to non-keyframes on demuxer level when supported if set to 1. Default
is 0. </dd> <dt><samp class="option">analyzeduration <var
class="var">integer</var> (input)</samp></dt>
<dd> Specify how many microseconds are analyzed to probe the input.
A higher value will enable detecting more accurate information, but will
increase latency. It defaults to 5,000,000 microseconds = 5 seconds.
</dd> <dt><samp class="option">cryptokey <var class="var">hexadecimal
string</var> (input)</samp></dt> <dd> Set
decryption key. </dd> <dt><samp class="option">indexmem <var

class=”var”>integer</var> (input)</samp></dt>
<dd> Set max memory used for timestamp index (per stream).
</dd> <dt><samp class=”option”>rtbufsize <var class=”var”>integer</var>
(input)</samp></dt> <dd> Set max
memory used for buffering real-time frames. </dd> <dt><samp
class=”option”>fdebug <var class=”var”>flags</var> (input/output)</samp><
<dd> Print specific debug info. Possible values: <dl
class=”table”> <dt>‘<samp class=”samp”>ts</samp>’</dt>
<dd> Print timestamp information. </dd> <dt>‘<samp
class=”samp”>id3v2</samp>’</dt> <dd> Print ID3v2 frame
data when demuxing files containing ID3v2 tags. </dd> </dl>

</dd> <dt><samp class=”option”>max_delay <var class=”var”>integer</var>
(input/output)</samp></dt> <dd> Set
maximum muxing or demuxing delay in microseconds. </dd>
<dt><samp class=”option”>fpsprobesize <var class=”var”>integer</var>
(input)</samp></dt> <dd> Set number of
frames used to probe fps. </dd> <dt><samp class=”option”>audio_preload
<var class=”var”>integer</var> (output)</samp></dt>
<dd> Set microseconds by which audio packets should be interleaved
earlier. </dd> <dt><samp class=”option”>chunk_duration <var
class=”var”>integer</var> (output)</samp></dt>
<dd> Set microseconds for each chunk. </dd> <dt><samp
class=”option”>chunk_size <var class=”var”>integer</var> (output)</samp>
<dd> Set size in bytes for each chunk. </dd> <dt><samp
class=”option”>err_detect, f_err_detect <var class=”var”>flags</var>
(input)</samp></dt> <dd> Set error
detection flags. <code class=”code”>f_err_detect</code> is deprecated and
should be used only via the <code class=”command”>ffmpeg</code> tool.
 Possible values: <dl class=”table”> <dt>‘<samp
class=”samp”>crccheck</samp>’</dt> <dd> Verify embedded
CRCs. </dd> <dt>‘<samp class=”samp”>bitstream</samp>’</dt>
<dd> Detect bitstream specification deviations. </dd> <dt>‘<samp
class=”samp”>buffer</samp>’</dt> <dd> Detect improper bit-
stream length. </dd> <dt>‘<samp class=”samp”>explode</samp>’</dt>
<dd> Abort decoding on minor error detection. </dd> <dt>‘<samp
class=”samp”>careful</samp>’</dt> <dd> Consider things that
violate the spec and have not been seen in the wild as errors. </dd>
<dt>‘<samp class=”samp”>compliant</samp>’</dt>
<dd> Consider all spec non compliancies as errors. </dd>
<dt>‘<samp class=”samp”>aggressive</samp>’</dt>
<dd> Consider things that a sane encoder should not do as an error.
</dd> </dl>

</dd> <dt><samp class=”option”>max_interleave_delta <var class=”var”>integer</var>
(output)</samp></dt> <dd> Set maximum
buffering duration for interleaving. The duration is expressed in microsec-

onds, and defaults to 10000000 (10 seconds). To ensure all the streams are interleaved correctly, libavformat will wait until it has at least one packet for each stream before actually writing any packets to the output file. When some streams are "sparse" (i.e. there are large gaps between successive packets), this can result in excessive buffering. This field specifies the maximum difference between the timestamps of the first and the last packet in the muxing queue, above which libavformat will output a packet regardless of whether it has queued a packet for all the streams. If set to 0, libavformat will continue buffering packets until it has a packet for each stream, regardless of the maximum timestamp difference between the buffered packets. </dd> <dt><samp class="option">use_wallclock_as_timestamps <var class="var">integer</var> (input)</samp></dt> <dd> Use wallclock as timestamps if set to 1. Default is 0. </dd> <dt><samp class="option">avoid_negative_ts <var class="var">integer</var> (output)</samp></dt> <dd> Possible values: <dl class="table"> <dt>‘<samp class="samp">make_non_negative</samp>’</dt> <dd> Shift timestamps to make them non-negative. Also note that this affects only leading negative timestamps, and not non-monotonic negative timestamps. </dd> <dt>‘<samp class="samp">make_zero</samp>’</dt> <dd> Shift timestamps so that the first timestamp is 0. </dd> <dt>‘<samp class="samp">auto (default)</samp>’</dt> <dd> Enables shifting when required by the target format. </dd> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Disables shifting of timestamp. </dd> </dl>

When shifting is enabled, all output timestamps are shifted by the same amount. Audio, video, and subtitles desynching and relative timestamp differences are preserved compared to how they would have been without shifting. </dd> <dt><samp class="option">skip_initial_bytes <var class="var">integer</var> (input)</samp></dt> <dd> Set number of bytes to skip before reading header and frames if set to 1. Default is 0. </dd> <dt><samp class="option">correct_ts_overflow <var class="var">integer</var> (input)</samp></dt> <dd> Correct single timestamp overflows if set to 1. Default is 1. </dd> <dt><samp class="option">flush_packets <var class="var">integer</var> (output)</samp></dt> <dd> Flush the underlying I/O stream after each packet. Default is -1 (auto), which means that the underlying protocol will decide, 1 enables it, and has the effect of reducing the latency, 0 disables it and may increase IO throughput in some cases. </dd> <dt><samp class="option">output_ts_offset <var class="var">offset</var> (output)</samp></dt> <dd> Set the output time offset. <var class="var">offset</var> must be a time duration specification, see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the

ffmpeg-utils(1) manual. The offset is added by the muxer to the output timestamps. Specifying a positive offset means that the corresponding streams are delayed bt the time duration specified in <var class="var">offset</var>. Default value is <code class="code">0</code> (meaning that no offset is applied). </dd> <dt><samp class="option">format_whitelist <var class="var">list</var> (input)</samp></dt> <dd> "," separated list of allowed demuxers. By default all are allowed. </dd> <dt><samp class="option">dump_separator <var class="var">string</var> (input)</samp></dt> <dd> Separator used to separate the fields printed on the command line about the Stream parameters. For example, to separate the fields with newlines and indentation: <pre class="example-preformatted">ffprobe -dump_separator " " -i ~/videos/matrixbench_mpeg2.mpg </pre>

</dd> <dt><samp class="option">max_streams <var class="var">integer</var> (input)</samp></dt> <dd> Specifies the maximum number of streams. This can be used to reject files that would require too many resources due to a large number of streams. </dd> <dt><samp class="option">skip_estimate_duration_from_pts <var class="var">bool</var> (input)</samp></dt> <dd> Skip estimation of input duration if it requires an additional probing for PTS at end of file. At present, applicable for MPEG-PS and MPEG-TS. </dd> <dt><samp class="option">duration_probesize <var class="var">integer</var> (input)</samp></dt> <dd> Set probing size, in bytes, for input duration estimation when it actually requires an additional probing for PTS at end of file (at present: MPEG-PS and MPEG-TS). It is aimed at users interested in better durations probing for itself, or indirectly because using the concat demuxer, for example. The typical use case is an MPEG-TS CBR with a high bitrate, high video buffering and ending cleaning with similar PTS for video and audio: in such a scenario, the large physical gap between the last video packet and the last audio packet makes it necessary to read many bytes in order to get the video stream duration. Another use case is where the default probing behaviour only reaches a single video frame which is not the last one of the stream due to frame reordering, so the duration is not accurate. Setting this option has a performance impact even for small files because the probing size is fixed. Default behaviour is a general purpose trade-off, largely adaptive, but the probing size will not be extended to get streams durations at all costs. Must be an integer not lesser than 1, or 0 for default behaviour. </dd> <dt><samp class="option">strict, f_strict <var class="var">integer</var> (input/output)</samp></dt> <dd> Specify how strictly to follow the standards. <code class="code">f_strict</code> is deprecated and should be used only via the <code class="command">ffmpeg</code> tool. Possible values: <dl class="table"> <dt>‘<samp

class=”samp”>very</samp>’</dt> <dd> strictly conform to an older more strict version of the spec or reference software </dd> <dt>‘<samp class=”samp”>strict</samp>’</dt> <dd> strictly conform to all the things in the spec no matter what consequences </dd> <dt>‘<samp class=”samp”>normal</samp>’</dt> <dt>‘<samp class=”samp”>unofficial</samp>’</dt> <dd> allow unofficial extensions </dd> <dt>‘<samp class=”samp”>experimental</samp>’</dt> <dd> allow non standardized experimental things, experimental (unfinished/work in progress/not well tested) decoders and encoders. Note: experimental decoders can pose a security risk, do not use this for decoding untrusted input. </dd> </dl>

</dd> </dl>

 <h3 class=”section”>14.1 Format stream specifiers# TOC</h3>

Format stream specifiers allow selection of one or more streams that match specific properties. The exact semantics of stream specifiers is defined by the <code class=”code”>avformat_match_stream_specifier()</code> function declared in the <samp class=”file”>libavformat/avformat.h</samp> header and documented in the <a data-manual=”ffmpeg” href=”./ffmpeg.html#Stream-specifiers”>Stream specifiers section in the ffmpeg(1) manual. <h2 class=”chapter”>15 Demuxers# TOC</h2>

Demuxers are configured elements in FFmpeg that can read the multimedia streams from a particular type of file. When you configure your FFmpeg build, all the supported demuxers are enabled by default. You can list all available ones using the configure option <code class=”code”>--list-demuxers</code>. You can disable all the demuxers using the configure option <code class=”code”>--disable-demuxers</code>, and selectively enable a single demuxer with the option <code class=”code”>--enable-demuxer=<var class=”var”>DEMUXER</var></code>, or disable it with the option <code class=”code”>--disable-demuxer=<var class=”var”>DEMUXER</var></code>. The option <code class=”code”>-demuxers</code> of the ff* tools will display the list of enabled demuxers. Use <code class=”code”>-formats</code> to view a combined list of enabled demuxers and muxers. The description of some of the currently available demuxers follows. <h3 class=”section”>15.1 aa# TOC</h3>

Audible Format 2, 3, and 4 demuxer. This demuxer is used to demux Audible Format 2, 3, and 4 (.aa) files. <h3 class="section">15.2 aac# TOC</h3>

Raw Audio Data Transport Stream AAC demuxer. This demuxer is used to demux an ADTS input containing a single AAC stream alongwith any ID3v1/2 or APE tags in it. <h3 class="section">15.3 apng# TOC</h3>

Animated Portable Network Graphics demuxer. This demuxer is used to demux APNG files. All headers, but the PNG signature, up to (but not including) the first fcTL chunk are transmitted as extradata. Frames are then split as being all the chunks between two fcTL ones, or between the last fcTL and IEND chunks. <dl class="table"> <dt><samp class="option">-ignore_loop <var class="var">bool</var></samp></dt> <dd> Ignore the loop variable in the file if set. Default is enabled. </dd> <dt><samp class="option">-max_fps <var class="var">int</var></samp></dt> <dd> Maximum framerate in frames per second. Default of 0 imposes no limit. </dd> <dt><samp class="option">-default_fps <var class="var">int</var></samp></dt> <dd> Default framerate in frames per second when none is specified in the file (0 meaning as fast as possible). Default is 15. </dd> </dl>

 <h3 class="section">15.4 asf# TOC</h3>

Advanced Systems Format demuxer. This demuxer is used to demux ASF files and MMS network streams. <dl class="table"> <dt><samp class="option">-no_resync_search <var class="var">bool</var></samp></dt> <dd> Do not try to resynchronize by looking for a certain optional start code. </dd> </dl>

 <h3 class="section">15.5 astc# TOC</h3>

ASTC (Adaptive Scalable Texture Compression) demuxer. This demuxer reads the 16-byte <samp class="file">.astc</samp> header, hands it to the decoder as extradata, and emits the compressed image as a single packet. <h3 class="section">15.6 concat# TOC</h3>

Virtual concatenation script demuxer. This demuxer reads a list of files and other directives from a text file and demuxes them one after the other, as if all their packets had been muxed together. The timestamps in the files are adjusted so that the first file starts at 0 and each next file starts where the previous one finishes. Note that it is done globally and may cause gaps if all streams do not have exactly the same length. All files must have the same streams (same codecs, same time base, etc.). The duration of each file is used to adjust the timestamps of the next file: if the duration is incorrect (because it was computed using the bit-rate or because the file is truncated, for example), it can cause artifacts. The <code class="code">duration</code> directive can be used to override the duration stored in each file. <h4 class="subsection">15.6.1 Syntax# TOC</h4>

The script is a text file in extended-ASCII, with one directive per line.

Empty lines, leading spaces and lines starting with ’#’ are ignored. The

following directive is recognized:

<dl class="table">

<dt><samp class="option"><code class="code">file <var class="var">path</var></code></samp></dt

<dd> Path to a file to read; special characters and spaces must be escaped with

backslash or single quotes.

All subsequent file-related directives apply to that file.

</dd>

<dt><samp class="option"><code class="code">ffconcat version 1.0</code></samp></dt>

<dd> Identify the script type and version.

To make FFmpeg recognize the format automatically, this directive must

appear exactly as is (no extra space or byte-order-mark) on the very first

line of the script.

</dd>
<dt><samp class="option"><code class="code">duration <var class="var">dur</var></code></samp>
<dd> Duration of the file. This information can be specified from the file;
specifying it here may be more efficient or help if the information from the
file is not available or accurate.

If the duration is set for all files, then it is possible to seek in the
whole concatenated video.

</dd>
<dt><samp class="option"><code class="code">inpoint <var class="var">timestamp</var></code></sa
<dd> In point of the file. When the demuxer opens the file it instantly seeks to the
specified timestamp. Seeking is done so that all streams can be presented
successfully at In point.

This directive works best with intra frame codecs, because for non-intra frame
ones you will usually get extra packets before the actual In point and the
decoded content will most likely contain frames before In point too.

For each file, packets before the file In point will have timestamps less than
the calculated start timestamp of the file (negative in case of the first
file), and the duration of the files (if not specified by the <code class="code">duration</code>
directive) will be reduced based on their specified In point.

Because of potential packets before the specified In point, packet timestamps
may overlap between two concatenated files.

</dd>

<dt><samp class="option"><code class="code">outpoint <var class="var">timestamp</var></code></

<dd> Out point of the file. When the demuxer reaches the specified decoding

timestamp in any of the streams, it handles it as an end of file condition and

skips the current and all the remaining packets from all streams.

Out point is exclusive, which means that the demuxer will not output packets

with a decoding timestamp greater or equal to Out point.

This directive works best with intra frame codecs and formats where all streams

are tightly interleaved. For non-intra frame codecs you will usually get

additional packets with presentation timestamp after Out point therefore the

decoded content will most likely contain frames after Out point too. If your

streams are not tightly interleaved you may not get all the packets from all

streams before Out point and you may only will be able to decode the earliest

stream until Out point.

The duration of the files (if not specified by the <code class="code">duration</code>

directive) will be reduced based on their specified Out point.

</dd>

<dt><samp class="option"><code class="code">file_packet_metadata <var class="var">key=value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

This directive is deprecated, use <code class="code">file_packet_meta</code> instead.

</dd>

<dt><samp class="option"><code class="code">file_packet_meta <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Metadata of the packets of the file. The specified metadata will be set for

each file packet. You can specify this directive multiple times to add multiple

metadata entries.

</dd>

<dt><samp class="option"><code class="code">option <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Option to access, open and probe the file.

Can be present multiple times.

</dd>

<dt><samp class="option"><code class="code">stream</code></samp></dt>

<dd> Introduce a stream in the virtual file.

All subsequent stream-related directives apply to the last introduced

stream.

Some streams properties must be set in order to allow identifying the

matching streams in the subfiles.

If no streams are defined in the script, the streams from the first file are

copied.

</dd>

<dt><samp class="option"><code class="code">exact_stream_id <var class="var">id</var></code></samp></dt>

<dd> Set the id of the stream.

If this directive is given, the string with the corresponding id in the

subfiles will be used.

This is especially useful for MPEG-PS (VOB) files, where the order of the

streams is not reliable.

</dd>

<dt><samp class="option"><code class="code">stream_meta <var class="var">key</var> <var class="var">value</var></code></samp></dt>

<dd> Metadata for the stream.

Can be present multiple times.

</dd>

<dt><samp class="option"><code class="code">stream_codec <var class="var">value</var></code></samp></dt>

<dd> Codec for the stream.

</dd>

<dt><samp class="option"><code class="code">stream_extradata <var class="var">hex_string</var></code></samp></dt>

<dd> Extradata for the string, encoded in hexadecimal.

</dd>

<dt><samp class="option"><code class="code">chapter <var class="var">id</var> <var class="var">start</var> <var class="var">end</var></code></samp></dt>

<dd> Add a chapter. <var class="var">id</var> is an unique identifier, possibly small and

consecutive.

</dd>

</dl>

 <h4 class="subsection">15.6.2 Options# TOC</h4>

This demuxer accepts the following option: <dl class="table"> <dt><samp class="option">safe</samp></dt> <dd> If set to 1, reject unsafe file paths and directives. A file path is considered safe if it does not contain a protocol specification and is relative and all components only contain characters from the portable character set (letters, digits, period, underscore and hyphen) and have no period at the beginning of a component.

 If set to 0, any file name is accepted. The default is 1. </dd> <dt><samp class="option">auto_convert</samp></dt> <dd> If set to 1, try to perform automatic conversions on packet data to make the streams concatenable. The default is 1. Currently, the only conversion is adding the h264_mp4toannexb bitstream filter to H.264 streams in MP4 format. This is necessary in particular if there are resolution changes. </dd> <dt><samp class="option">segment_time_metadata</samp></dt> <dd> If set to 1, every packet will contain the <var class="var">lavf.concat.start_time</var> and the <var class="var">lavf.concat.duration</var> packet metadata values which are the start_time and the duration of the respective file segments in the concatenated output expressed in microseconds. The duration metadata is only set if it is known based on the concat file. The default is 0. </dd> <dt><samp class="option">chapter_per_file</samp></dt> <dd> If set to 1, add a chapter for each file, named after the <code class="code">title</code> metadata of the file or, failing that, its file name, and carrying the rest of the file’s metadata. The files are opened in order when the script is read to learn their durations, stopping after the first file whose duration is unknown, which gets the last chapter; when all durations are known, this also makes the total duration known and the output seekable. The default is 0. </dd> </dl>

 <h4 class="subsection">15.6.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use absolute filenames and include some comments: <pre class="example-preformatted"># my first filename file /mnt/share/file-1.wav # my second filename including whitespace file '/mnt/share/file 2.wav' # my third filename including whitespace plus single quote file '/mnt/share/file 3'\''.wav' </pre>

Allow for input format auto-probing, use safe filenames and set the duration of the first file: <pre class="example-preformatted">ffconcat version 1.0

file file-1.wav duration 20.0

file subdir/file-2.wav </pre>

 <h3 class="section">15.7 dash#
TOC</h3>

Dynamic Adaptive Streaming over HTTP demuxer. This demuxer presents all AVStreams found in the manifest. By setting the discard flags on AVStreams the caller can decide which streams to actually receive. Each stream mirrors the <code class="code">id</code>

and <code class=”code”>bandwidth</code> properties from the <code class=”code”><Representation></code> as metadata keys named "id" and "variant_bitrate" respectively. <h4 class=”subsection”>15.7.1 Options# TOC</h4>

This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>cenc_decryption_key</samp></dt> <dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>cenc_decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> </dl>

 <h3 class=”section”>15.8 dvdvideo# TOC</h3>

DVD-Video demuxer, powered by libdvdnav and libdvdread. Can directly ingest DVD titles, specifically sequential PGCs, into a conversion pipeline. Menu assets, such as background video or audio, can also be demuxed given the menu’s coordinates (at best effort). Block devices (DVD drives), ISO files, and directory structures are accepted. Activate with <code class=”code”>-f dvdvideo</code> in front of one of these inputs. This demuxer does NOT have decryption code of any kind. You are on your own working with encrypted DVDs, and should not expect support on the matter. Underlying playback is handled by libdvdnav, and structure parsing by libdvdread. FFmpeg must be built with GPL library support available as well as the configure switches <code class=”code”>--enable-libdvdnav</code> and <code class=”code”>--enable-libdvdread</code>. You will need to provide either the desired "title number" or exact PGC/PG coordinates. Many open-source DVD players and tools can aid in providing this information. If not specified, the demuxer will default to title 1 which works for many discs. However, due to the flexibility of the format, it is recommended to check manually. There are many discs that are authored strangely or with invalid headers. If the input is a real DVD drive, please note that there are some drives which may silently fail on reading bad sectors from the disc, returning random bits instead which is effectively corrupt data. This is especially prominent on aging or rotting discs. A second pass and integrity checks would be needed to detect the corruption. This is not an FFmpeg issue. <h4 class=”subsection”>15.8.1 Background#

TOC</h4>

DVD-Video is not a directly accessible, linear container format in the traditional sense. Instead, it allows for complex and programmatic playback of carefully muxed MPEG-PS streams that are stored in headerless VOB files. To the end-user, these streams are known simply as "titles", but the actual logical playback sequence is defined by one or more "PGCs", or Program Group Chains, within the title. The PGC is in turn comprised of multiple "PGs", or Programs", which are the actual video segments (and for a typical video feature, sequentially ordered). The PGC structure, along with stream layout and metadata, are stored in IFO files that need to be parsed. PGCs can be thought of as playlists in easier terms. An actual DVD player relies on user GUI interaction via menus and an internal VM to drive the direction of demuxing. Generally, the user would either navigate (via menus) or automatically be redirected to the PGC of their choice. During this process and the subsequent playback, the DVD player’s internal VM also maintains a state and executes instructions that can create jumps to different sectors during playback. This is why libdvdnav is involved, as a linear read of the MPEG-PS blobs on the disc (VOBs) is not enough to produce the right sequence in many cases. There are many other DVD structures (a long subject) that will not be discussed here. NAV packets, in particular, are handled by this demuxer to build accurate timing but not emitted as a stream. For a good high-level understanding, refer to: https://code.videolan.org/videolan/libdvdnav/-/blob/master/doc/dvd_structures <h4 class=”subsection”>15.8.2 Options# TOC</h4>

This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>title <var class=”var”>int</var></samp></dt> <dd> The title number to play. Must be set if <samp class=”option”>pgc</samp> and <samp class=”option”>pg</samp> are not set. Not applicable to menus. Default is 0 (auto), which currently only selects the first available title (title 1) and notifies the user about the implications. </dd> <dt><samp class=”option”>chapter_start <var class=”var”>int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to start at. Not applicable to menus. Default is 1. </dd> <dt><samp class=”option”>chapter_end <var class=”var”>int</var></samp></dt> <dd> The chapter, or PTT (part-of-title), number to end at. Not applicable to menus. Default is 0, which is a special value to signal end at the last possible chapter. </dd> <dt><samp class=”option”>angle <var class=”var”>int</var></samp></dt> <dd> The video angle number, referring to what is essentially an additional video stream that is composed from alternate frames interleaved in the VOBs. Not applicable to menus. Default is 1. </dd> <dt><samp class=”option”>region

<var class=”var”>int</var></samp></dt> <dd> The region code to use for playback. Some discs may use this to default playback at a particular angle in different regions. This option will not affect the region code of a real DVD drive, if used as an input. Not applicable to menus. Default is 0, "world". </dd> <dt><samp class=”option”>menu <var class=”var”>bool</var></samp></dt> <dd> Demux menu assets instead of navigating a title. Requires exact coordinates of the menu (<samp class=”option”>menu_lu</samp>, <samp class=”option”>menu_vts</samp>, <samp class=”option”>pgc</samp>, <samp class=”option”>pg</samp>). Default is false. </dd> <dt><samp class=”option”>menu_lu <var class=”var”>int</var></samp></dt> <dd> The menu language to demux. In DVD, menus are grouped by language. Default is 1, the first language unit. </dd> <dt><samp class=”option”>menu_vts <var class=”var”>int</var></samp></dt> <dd> The VTS where the menu lives, or 0 if it is a VMG menu (root-level). Default is 1, menu of the first VTS. </dd> <dt><samp class=”option”>pgc <var class=”var”>int</var></samp></dt> <dd> The entry PGC to start playback, in conjunction with <samp class=”option”>pg</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Must be explicitly set for menus. Default is 0, automatically resolve from value of <samp class=”option”>title</samp>. </dd> <dt><samp class=”option”>pg <var class=”var”>int</var></samp></dt> <dd> The entry PG to start playback, in conjunction with <samp class=”option”>pgc</samp>. Alternative to setting <samp class=”option”>title</samp>. Chapter markers are not supported at this time. Default is 1, the first PG of the PGC. </dd> <dt><samp class=”option”>preindex <var class=”var”>bool</var></samp></dt> <dd> Enable this to have accurate chapter (PTT) markers and duration measurement, which requires a slow second pass read in order to index the chapter marker timestamps from NAV packets. This is non-ideal extra work for real optical drives. It is recommended and faster to use this option with a backup of the DVD structure stored on a hard drive. Not compatible with <samp class=”option”>pgc</samp> and <samp class=”option”>pg</samp>. Default is 0, false. </dd> <dt><samp class=”option”>trim <var class=”var”>bool</var></samp></dt> <dd> Skip padding cells (i.e. cells shorter than 1 second) from the beginning. There exist many discs with filler segments at the beginning of the PGC, often with junk data intended for controlling a real DVD player’s buffering speed and with no other material data value. Not applicable to menus. Default is 1, true. </dd> </dl>

 <h4 class=”subsection”>15.8.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Open title 3 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -title 3 -i <path to DVD> ... </pre>

Open chapters 3-6 from title 1 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -chapter_start 3 -chapter_end 6 -title 1 -i <path to DVD> ... </pre>

Open only chapter 5 from title 1 from a given DVD structure: <pre class="example-preformatted">ffmpeg -f dvdvideo -chapter_start 5 -chapter_end 5 -title 1 -i <path to DVD> ... </pre>

Demux menu with language 1 from VTS 1, PGC 1, starting at PG 1: <pre class="example-preformatted">ffmpeg -f dvdvideo -menu 1 -menu_lu 1 -menu_vts 1 -pgc 1 -pg 1 -i <path to DVD> ... </pre>

 <h3 class="section">15.9 ea# TOC</h3>

Electronic Arts Multimedia format demuxer. This format is used by various Electronic Arts games. <h4 class="subsection">15.9.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">merge_alpha <var class="var">bool</var></samp></dt> <dd> Normally the VP6 alpha channel (if exists) is returned as a secondary video stream, by setting this option you can make the demuxer return a single video stream which contains the alpha channel in addition to the ordinary video. </dd> </dl>

 <h3 class="section">15.10 imf# TOC</h3>

Interoperable Master Format demuxer. This demuxer presents audio and video streams found in an IMF Composition, as specified in SMPTE ST 2067-2.   <pre class="example-preformatted">ffmpeg [-assetmaps <path of ASSETMAP1>,<path of ASSETMAP2>,...] -i <path of CPL> ... </pre>

If <code class="code">-assetmaps</code> is not specified, the demuxer looks for a file called <samp class="file">ASSETMAP.xml</samp> in the same directory as the CPL. <h3 class="section">15.11 flv, live_flv, kux<span class="pull-

right”># TOC</h3>

Adobe Flash Video Format demuxer. This demuxer is used to demux FLV files and RTMP network streams. In case of live network streams, if you force format, you may use live_flv option instead of flv to survive timestamp discontinuities. KUX is a flv variant used on the Youku platform.   <pre class=”example-preformatted”>ffmpeg -f flv -i myfile.flv ... ffmpeg -f live_flv -i rtmp://<any.server>/anything/key .... </pre>

<dl class=”table”> <dt><samp class=”option”>-flv_metadata <var class=”var”>bool</var></samp></dt> <dd> Allocate the streams according to the onMetaData array content. </dd> <dt><samp class=”option”>-flv_ignore_prevtag <var class=”var”>bool</var></samp></dt> <dd> Ignore the size of previous tag value. </dd> <dt><samp class=”option”>-flv_full_metadata <var class=”var”>bool</var></samp></dt> <dd> Output all context of the onMetadata. </dd> </dl>

 <h3 class=”section”>15.12 gif# TOC</h3>

Animated GIF demuxer. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>min_delay</samp></dt> <dd> Set the minimum valid delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 2. </dd> <dt><samp class=”option”>max_gif_delay</samp></dt> <dd> Set the maximum valid delay between frames in hundredth of seconds. Range is 0 to 65535. Default value is 65535 (nearly eleven minutes), the maximum value allowed by the specification. </dd> <dt><samp class=”option”>default_delay</samp></dt> <dd> Set the default delay between frames in hundredths of seconds. Range is 0 to 6000. Default value is 10. </dd> <dt><samp class=”option”>ignore_loop</samp></dt> <dd> GIF files can contain information to loop a certain number of times (or infinitely). If <samp class=”option”>ignore_loop</samp> is set to 1, then the loop setting from the input will be ignored and looping will not occur. If set to 0, then looping will occur and will cycle the number of times according to the GIF. Default value is 1. </dd> </dl>

For example, with the overlay filter, place an infinitely looping GIF over another video: <pre class=”example-preformatted”>ffmpeg -i input.mp4 -ignore_loop 0 -i input.gif -filter_complex overlay=shortest=1 out.mkv </pre>

Note that in the above example the shortest option for overlay filter is used to end the output video at the length of the shortest input file, which in this case is <samp class=”file”>input.mp4</samp> as the GIF in this example

loops infinitely. <h3 class=”section”>15.13 hls# TOC</h3>

HLS demuxer Apple HTTP Live Streaming demuxer. This demuxer presents all AVStreams from all variant streams. The id field is set to the bitrate variant index number. By setting the discard flags on AVStreams (by pressing ’a’ or ’v’ in ffplay), the caller can decide which variant streams to actually receive. The total bitrate of the variant that the stream belongs to is available in a metadata key named "variant_bitrate". It is unset when the stream is shared by variants with different bitrates. The value is always available in the corresponding program’s metadata. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>live_start_index</samp></dt> <dd> segment index to start live streams at (negative values are from the end). </dd> <dt><samp class=”option”>prefer_x_start</samp></dt> <dd> prefer to use #EXT-X-START if it’s in playlist instead of live_start_index. </dd> <dt><samp class=”option”>allowed_extensions</samp></dt> <dd> ’,’ separated list of file extensions that hls is allowed to access. </dd> <dt><samp class=”option”>extension_picky</samp></dt> <dd> This blocks disallowed extensions from probing It also requires all available segments to have matching extensions to the format except mpegts, which is always allowed. It is recommended to set the whitelists correctly instead of depending on extensions Enabled by default. </dd> <dt><samp class=”option”>max_reload</samp></dt> <dd> Maximum number of times a insufficient list is attempted to be reloaded. Default value is 1000. </dd> <dt><samp class=”option”>m3u8_hold_counters</samp></dt> <dd> The maximum number of times to load m3u8 when it refreshes without new segments. Default value is 1000. </dd> <dt><samp class=”option”>http_persistent</samp></dt> <dd> Use persistent HTTP connections. Applicable only for HTTP streams. Enabled by default. </dd> <dt><samp class=”option”>http_multiple</samp></dt> <dd> Use multiple HTTP connections for downloading HTTP segments. Enabled by default for HTTP/1.1 servers. </dd> <dt><samp class=”option”>http_seekable</samp></dt> <dd> Use HTTP partial requests for downloading HTTP segments. 0 = disable, 1 = enable, -1 = auto, Default is auto. </dd> <dt><samp class=”option”>seg_format_options</samp></dt> <dd> Set options for the demuxer of media segments using a list of key=value pairs separated by <code class=”code”>:</code>. </dd> <dt><samp class=”option”>seg_max_retry</samp></dt> <dd> Maximum number of times to reload a segment on error, useful when segment skip on network error is not desired. Default value is 0. </dd> </dl>

 <h3 class=”section”>15.14 image2<span

class=”pull-right”># TOC</h3>

Image file demuxer. This demuxer reads from a list of image files specified by a pattern. The syntax and meaning of the pattern is specified by the option <var class=”var”>pattern_type</var>. The pattern may contain a suffix which is used to automatically determine the format of the images contained in the files. The size, the pixel format, and the format of each image must be the same for all the files in the sequence. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate for the video stream. It defaults to 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the input. Default value is 0. </dd> <dt><samp class=”option”>pattern_type</samp></dt> <dd> Select the pattern type used to interpret the provided filename. <var class=”var”>pattern_type</var> accepts one of the following values. <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Disable pattern matching, therefore the video will only contain the specified image. You should use this option if you do not want to create sequences from multiple images and your filenames may contain special pattern characters. </dd> <dt><samp class=”option”>sequence</samp></dt> <dd> Select a sequence pattern type, used to specify a sequence of files indexed by sequential numbers. A sequence pattern may contain the string "%d" or "%0<var class=”var”>N</var>d", which specifies the position of the characters representing a sequential number in each filename matched by the pattern. If the form "%d0<var class=”var”>N</var>d" is used, the string representing the number in each filename is 0-padded and <var class=”var”>N</var> is the total number of 0-padded digits representing the number. The literal character ’%’ can be specified in the pattern with the string "%%". If the sequence pattern contains "%d" or "%0<var class=”var”>N</var>d", the first filename of the file list specified by the pattern must contain a number inclusively contained between <var class=”var”>start_number</var> and <var class=”var”>start_number</var>+<var class=”var”>start_number_range</var>-1, and all the following numbers must be sequential. For example the pattern "img-%03d.bmp" will match a sequence of filenames of the form <samp class=”file”>img-001.bmp</samp>, <samp class=”file”>img-002.bmp</samp>, ..., <samp class=”file”>img-010.bmp</samp>, etc.; the pattern "i%%m%%g-%d.jpg" will match a sequence of filenames of the form <samp class=”file”>i%m%g-1.jpg</samp>, <samp class=”file”>i%m%g-2.jpg</samp>, ..., <samp class=”file”>i%m%g-10.jpg</samp>, etc. Note that the pattern must not necessarily contain "%d" or "%0<var

class=”var”>N</var>d", for example to convert a single image file <samp class=”file”>img.jpeg</samp> you can employ the command: <pre class=”example-preformatted”>ffmpeg -i img.jpeg img.png </pre>

</dd> <dt><samp class=”option”>glob</samp></dt> <dd> Select a glob wildcard pattern type. The pattern is interpreted like a <code class=”code”>glob()</code> pattern. This is only selectable if libavformat was compiled with globbing support. </dd> </dl>

Default value is <var class=”var”>sequence</var>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Set the pixel format of the images to read. If not specified the pixel format is guessed from the first image file in the sequence. </dd> <dt><samp class=”option”>start_number</samp></dt> <dd> Set the index of the file matched by the image file pattern to start to read from. Default value is 0. </dd> <dt><samp class=”option”>start_number_range</samp></dt> <dd> Set the index interval range to check when looking for the first image file in the sequence, starting from <var class=”var”>start_number</var>. Default value is 5. </dd> <dt><samp class=”option”>ts_from_file</samp></dt> <dd> If set to 1, will set frame timestamp to modification time of image file. Note that monotonity of timestamps is not provided: images go in the same order as without this option. Default value is 0. If set to 2, will set frame timestamp to the modification time of the image file in nanosecond precision. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size of the images to read. If not specified the video size is guessed from the first image file in the sequence. </dd> <dt><samp class=”option”>export_path_metadata</samp></dt> <dd> If set to 1, will add two extra fields to the metadata found in input, making them also available for other filters (see <var class=”var”>drawtext</var> filter for examples). Default value is 0. The extra fields are described below: <dl class=”table”> <dt><samp class=”option”>lavf.image2dec.source_path</samp></dt> <dd> Corresponds to the full path to the input file being read. </dd> <dt><samp class=”option”>lavf.image2dec.source_basename</samp></dt> <dd> Corresponds to the name of the file being read. </dd> </dl>

</dd> </dl>

 <h4 class=”subsection”>15.14.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Use <code class=”command”>ffmpeg</code> for creating a video from the images in the file sequence <samp class=”file”>img-001.jpeg</samp>, <samp class=”file”>img-002.jpeg</samp>, ..., assuming an input frame rate of 10 frames per second: <pre class=”example-preformatted”>ffmpeg -framerate 10 -i 'img-%03d.jpeg'

out.mkv </pre>

As above, but start by reading from a file with index 100 in the sequence: <pre class=”example-preformatted”>ffmpeg -framerate 10 -start_number 100 -i 'img-%03d.jpeg' out.mkv </pre>

Read images matching the "*.png" glob pattern , that is all the files terminating with the ".png" suffix: <pre class=”example-preformatted”>ffmpeg -framerate 10 -pattern_type glob -i "*.png" out.mkv </pre>

 <h3 class=”section”>15.15 jpeg_mpf# TOC</h3>

JPEG Multi-Picture Format demuxer. Exposes each JPEG image stored in a CIPA DC-007 MP file as a separate stream, in the order of the MP Entries, with the First Individual Image flagged as the default stream. If the file contains an ISO 21496-1 gain map, as is the case for JPEG Ultra HDR, the primary image and the gain map are linked by a ‘<samp class=”samp”>gain_map</samp>’ stream group, and the gain map parameters are exported as frame side data by the decoder. Images of any other type (e.g. large thumbnails or multi-view images) are exposed as additional streams. This demuxer accepts the following options, which behave like the options of the same name of the ‘<samp class=”samp”>image2</samp>’ demuxer: <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate of the video streams. The default is 25. </dd> <dt><samp class=”option”>loop</samp></dt> <dd> If set to 1, loop over the images in the file. The default is 0. This requires seekable input. </dd> </dl>

 <h3 class=”section”>15.16 ktx# TOC</h3>

Khronos Texture 1.0 demuxer for ASTC textures. This demuxer reads the KTX 1.0 header and maps the ASTC GL internal format to the block footprint the decoder needs, so only files using the ASTC formats are accepted. A <code class=”code”>KTXorientation</code> key is preserved as stream display metadata, and the texture is emitted as a single packet. <h3 class=”section”>15.17 libgme# TOC</h3>

The Game Music Emu library is a collection of video game music file emulators. See <a class=”url” href=”https://bitbucket.org/mpyne/game-

music-emu/overview”>https://bitbucket.org/mpyne/game-music-emu/overview for more information. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>track_index</samp></dt> <dd> Set the index of which track to demux. The demuxer can only export one track. Track indexes start at 0. Default is to pick the first track. Number of tracks is exported as <var class=”var”>tracks</var> metadata entry. </dd> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sampling rate of the exported track. Range is 1000 to 999999. Default is 44100. </dd> <dt><samp class=”option”>max_size (bytes)</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Default is 50 MiB. </dd> </dl>

 <h3 class=”section”>15.18 libmodplug# TOC</h3>

ModPlug based module demuxer See https://github.com/Konstanty/libmodplug It will export one 2-channel 16-bit 44.1 kHz audio stream. Optionally, a <code class=”code”>pal8</code> 16-color video stream can be exported with or without printed metadata. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>noise_reduction</samp></dt> <dd> Apply a simple low-pass filter. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class=”option”>reverb_depth</samp></dt> <dd> Set amount of reverb. Range 0-100. Default is 0. </dd> <dt><samp class=”option”>reverb_delay</samp></dt> <dd> Set delay in ms, clamped to 40-250 ms. Default is 0. </dd> <dt><samp class=”option”>bass_amount</samp></dt> <dd> Apply bass expansion a.k.a. XBass or megabass. Range is 0 (quiet) to 100 (loud). Default is 0. </dd> <dt><samp class=”option”>bass_range</samp></dt> <dd> Set cutoff i.e. upper-bound for bass frequencies. Range is 10-100 Hz. Default is 0. </dd> <dt><samp class=”option”>surround_depth</samp></dt> <dd> Apply a Dolby Pro-Logic surround effect. Range is 0 (quiet) to 100 (heavy). Default is 0. </dd> <dt><samp class=”option”>surround_delay</samp></dt> <dd> Set surround delay in ms, clamped to 5-40 ms. Default is 0. </dd> <dt><samp class=”option”>max_size</samp></dt> <dd> The demuxer buffers the entire file into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of files that can be read. Range is 0 to 100 MiB. 0 removes buffer size limit (not recommended). Default is 5 MiB. </dd> <dt><samp class=”option”>video_stream_expr</samp></dt> <dd> String which is evaluated using the eval API to assign colors to the generated video stream. Variables which can be used are <code class=”code”>x</code>,

<code class="code">y</code>, <code class="code">w</code>, <code class="code">h</code>, <code class="code">t</code>, <code class="code">speed</code>, <code class="code">tempo</code>, <code class="code">order</code>, <code class="code">pattern</code> and <code class="code">row</code>. </dd> <dt><samp class="option">video_stream</samp></dt> <dd> Generate video stream. Can be 1 (on) or 0 (off). Default is 0. </dd> <dt><samp class="option">video_stream_w</samp></dt> <dd> Set video frame width in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class="option">video_stream_h</samp></dt> <dd> Set video frame height in ’chars’ where one char indicates 8 pixels. Range is 20-512. Default is 30. </dd> <dt><samp class="option">video_stream_ptxt</samp></dt> <dd> Print metadata on video stream. Includes <code class="code">speed</code>, <code class="code">tempo</code>, <code class="code">order</code>, <code class="code">pattern</code>, <code class="code">row</code> and <code class="code">ts</code> (time in ms). Can be 1 (on) or 0 (off). Default is 1. </dd> </dl>

 <h3 class="section">15.19 libopenmpt# TOC</h3>

libopenmpt based module demuxer See https://lib.openmpt.org/libopenmpt/ for more information. Some files have multiple subsongs (tracks) this can be set with the <samp class="option">subsong</samp> option. It accepts the following options: <dl class="table"> <dt><samp class="option">subsong</samp></dt> <dd> Set the subsong index. This can be either ’all’, ’auto’, or the index of the subsong. Subsong indexes start at 0. The default is ’auto’. The default value is to let libopenmpt choose. </dd> <dt><samp class="option">layout</samp></dt> <dd> Set the channel layout. Valid values are 1, 2, and 4 channel layouts. The default value is STEREO. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate for libopenmpt to output. Range is from 1000 to INT_MAX. The value default is 48000. </dd> </dl>

 <h3 class="section">15.20 mcc# TOC</h3>

Demuxer for MacCaption MCC files, it supports MCC versions 1.0 and 2.0. MCC files store VANC data, which can include closed captions (EIA-608 and CEA-708), ancillary time code, pan-scan data, etc. By default, for backward compatibility, the MCC demuxer extracts just the EIA-608 and

CEA-708 closed captions and returns a <code class=”code”>EIA_608</code> stream, ignoring all other VANC data. You can change it to return all VANC data in a <code class=”code”>SMPTE_436M_ANC</code> data stream by setting <samp class=”option”>-eia608_extract 0</samp> <h4 class=”subsection”>15.20.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Convert a MCC file to Scenarist (SCC) format: <pre class=”example-preformatted”>ffmpeg -i CC.mcc -c:s copy CC.scc </pre> Note that the SCC format only supports EIA-608, so this will discard all other data such as CEA-708 extensions. Merge a MCC file into a MXF file: <pre class=”example-preformatted”>ffmpeg -i video_and_audio.mxf -eia608_extract 0 -i CC.mcc -c copy -map 0 -map 1 out.mxf </pre> This retains all VANC data and inserts it into the output MXF file as a <code class=”code”>SMPTE_436M_ANC</code> data stream.

 <h3 class=”section”>15.21 mov/mp4/3gp# TOC</h3>

Demuxer for Quicktime File Format & ISO/IEC Base Media File Format (ISO/IEC 14496-12 or MPEG-4 Part 12, ISO/IEC 15444-12 or JPEG 2000 Part 12). Registered extensions: mov, mp4, m4a, 3gp, 3g2, mj2, psp, m4b, ism, ismv, isma, f4v <h4 class=”subsection”>15.21.1 Options# TOC</h4>

This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>enable_drefs</samp></dt> <dd> Enable loading of external tracks, disabled by default. Enabling this can theoretically leak information in some use cases. </dd> <dt><samp class=”option”>use_absolute_path</samp></dt> <dd> Allows loading of external tracks via absolute paths, disabled by default. Enabling this poses a security risk. It should only be enabled if the source is known to be non-malicious. </dd> <dt><samp class=”option”>seek_streams_individually</samp></dt> <dd> When seeking, identify the closest point in each stream individually and demux packets in that stream from identified point. This can lead to a different sequence of packets compared to demuxing linearly from the beginning. Default is true. </dd> <dt><samp class=”option”>ignore_editlist</samp></dt> <dd> Ignore any edit list atoms. The demuxer, by default, modifies the stream index to reflect the

timeline described by the edit list. Default is false. </dd> <dt><samp class=”option”>advanced_editlist</samp></dt> <dd> Modify the stream index to reflect the timeline described by the edit list. <code class=”code”>ignore_editlist</code> must be set to false for this option to be effective. If both <code class=”code”>ignore_editlist</code> and this option are set to false, then only the start of the stream index is modified to reflect initial dwell time or starting timestamp described by the edit list. Default is true. </dd> <dt><samp class=”option”>ignore_chapters</samp></dt> <dd> Don’t parse chapters. This includes GoPro ’HiLight’ tags/moments. Note that chapters are only parsed when input is seekable. Default is false. </dd> <dt><samp class=”option”>use_mfra_for</samp></dt> <dd> For seekable fragmented input, set fragment’s starting timestamp from media fragment random access box, if present. Following options are available: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Auto-detect whether to set mfra timestamps as PTS or DTS (default) </dd> <dt>‘<samp class=”samp”>dts</samp>’</dt> <dd> Set mfra timestamps as DTS </dd> <dt>‘<samp class=”samp”>pts</samp>’</dt> <dd> Set mfra timestamps as PTS </dd> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Don’t use mfra box to set timestamps </dd> </dl>

</dd> <dt><samp class=”option”>use_tfdt</samp></dt> <dd> For fragmented input, set fragment’s starting timestamp to <code class=”code”>baseMediaDecodeTime</code> from the <code class=”code”>tfdt</code> box. Default is enabled, which will prefer to use the <code class=”code”>tfdt</code> box to set DTS. Disable to use the <code class=”code”>earliest_presentation_time</code> from the <code class=”code”>sidx</code> box. In either case, the timestamp from the <code class=”code”>mfra</code> box will be used if it’s available and <code class=”code”>use_mfra_for</code> is set to pts or dts. </dd> <dt><samp class=”option”>export_all</samp></dt> <dd> Export unrecognized boxes within the <var class=”var”>udta</var> box as metadata entries. The first four characters of the box type are set as the key. Default is false. </dd> <dt><samp class=”option”>export_xmp</samp></dt> <dd> Export entire contents of <var class=”var”>XMP_</var> box and <var class=”var”>uuid</var> box as a string with key <code class=”code”>xmp</code>. Note that if <code class=”code”>export_all</code> is set and this option isn’t, the contents of <var class=”var”>XMP_</var> box are still exported but with key <code class=”code”>XMP_</code>. Default is false. </dd> <dt><samp class=”option”>activation_bytes</samp></dt> <dd> 4-byte key required to decrypt Audible AAX and AAX+ files. See Audible AAX subsection below. </dd> <dt><samp class=”option”>audible_fixed_key</samp></dt> <dd> Fixed key used for handling Audible AAX/AAX+ files. It has been pre-set so should not be necessary to specify. </dd> <dt><samp class=”option”>decryption_key</samp></dt>

<dd> Default 16-byte key, in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>decryption_keys</samp></dt> <dd> Dictionary of 16-byte key ID => 16-byte key, both in hex, to decrypt files encrypted using ISO Common Encryption (CENC/AES-128 CTR; ISO/IEC 23001-7). </dd> <dt><samp class=”option”>max_stts_delta</samp></dt> <dd> Very high sample deltas written in a trak’s stts box may occasionally be intended but usually they are written in error or used to store a negative value for dts correction when treated as signed 32-bit integers. This option lets the user set an upper limit, beyond which the delta is clamped to 1. Values greater than the limit if negative when cast to int32 are used to adjust onward dts. Unit is the track time scale. Range is 0 to UINT_MAX. Default is <code class=”code”>UINT_MAX - 48000*10</code> which allows up to a 10 second dts correction for 48 kHz audio streams while accommodating 99.9% of <code class=”code”>uint32</code> range. </dd> <dt><samp class=”option”>interleaved_read</samp></dt> <dd> Interleave packets from multiple tracks at demuxer level. For badly interleaved files, this prevents playback issues caused by large gaps between packets in different tracks, as MOV/MP4 do not have packet placement requirements. However, this can cause excessive seeking on very badly interleaved files, due to seeking between tracks, so disabling it may prevent I/O issues, at the expense of playback. </dd> </dl>

 <h4 class=”subsection”>15.21.2 Audible AAX# TOC</h4>

Audible AAX files are encrypted M4B files, and they can be decrypted by specifying a 4 byte activation secret. <pre class=”example-preformatted”>ffmpeg -activation_bytes 1CEB00DA -i test.aax -vn -c:a copy output.mp4 </pre>

 <h3 class=”section”>15.22 mpegts# TOC</h3>

MPEG-2 transport stream demuxer. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>resync_size</samp></dt> <dd> Set size limit for looking up a new synchronization. Default value is 65536. </dd> <dt><samp class=”option”>skip_unknown_pmt</samp></dt> <dd> Skip PMTs for programs not defined in the PAT. Default value is 0. </dd> <dt><samp class=”option”>fix_teletext_pts</samp></dt> <dd> Override teletext packet PTS and DTS values with the timestamps calculated from the PCR of the first program which the teletext stream is part of and is not discarded. Default value is 1, set this option

to 0 if you want your teletext packet PTS and DTS values untouched. </dd> <dt><samp class="option">ts_packetsize</samp></dt> <dd> Output option carrying the raw packet size in bytes. Show the detected raw packet size, cannot be set by the user. </dd> <dt><samp class="option">scan_all_pmts</samp></dt> <dd> Scan and combine all PMTs. The value is an integer with value from -1 to 1 (-1 means automatic setting, 1 means enabled, 0 means disabled). Default value is -1. </dd> <dt><samp class="option">merge_pmt_versions</samp></dt> <dd> Reuse existing streams when a PMT’s version is updated and elementary streams move to different PIDs. Default value is 0. </dd> <dt><samp class="option">max_packet_size</samp></dt> <dd> Set maximum size, in bytes, of packet emitted by the demuxer. Payloads above this size are split across multiple packets. Range is 184 to INT_MAX/2. Default is 204800 bytes. </dd> </dl>

 <h3 class="section">15.23 mpjpeg# TOC</h3>

MJPEG encapsulated in multi-part MIME demuxer. This demuxer allows reading of MJPEG, where each frame is represented as a part of multipart/x-mixed-replace stream. <dl class="table"> <dt><samp class="option">strict_mime_boundary</samp></dt> <dd> Default implementation applies a relaxed standard to multi-part MIME boundary detection, to prevent regression with numerous existing endpoints not generating a proper MIME MJPEG stream. Turning this option on by setting it to 1 will result in a stricter check of the boundary value. </dd> </dl>

 <h3 class="section">15.24 rawvideo# TOC</h3>

Raw video demuxer. This demuxer allows one to read raw video data. Since there is no header specifying the assumed video parameters, the user must specify them in order to be able to decode the data correctly. This demuxer accepts the following options: <dl class="table"> <dt><samp class="option">framerate</samp></dt> <dd> Set input video frame rate. Default value is 25. </dd> <dt><samp class="option">pixel_format</samp></dt> <dd> Set the input video pixel format. Default value is <code class="code">yuv420p</code>. </dd> <dt><samp class="option">stride</samp></dt> <dd> Set frame line size in bytes, only required if there are extra padding. For multiplane formats, <samp class="option">stride</samp> is a list of line size for each plane. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the input video size. This value must be specified explicitly. </dd> </dl>

For example to read a rawvideo file <samp class="file">input.raw</samp> with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">rgb24</code>, a video size of <code class="code">320x240</code>, and a frame rate of 10 images per second, use the command: <pre class="example-preformatted">ffplay -f rawvideo -pixel_format rgb24 -video_size 320x240 -framerate 10 input.raw </pre>

Read a rawvideo with <code class="command">ffplay</code>, assuming a pixel format of <code class="code">yuv420p</code>, a video size of <code class="code">1080x1920</code>, has 8 bytes of padding in each line of Y plane, and 4 bytes of padding with UV plane, <pre class="example-preformatted">ffplay -f rawvideo -pixel_format yuv420p -video_size 1080x1920 -stride 1088,544,544 input.raw </pre>

 <h3 class="section">15.25 rcwt# TOC</h3>

RCWT (Raw Captions With Time) is a format native to ccextractor, a commonly used open source tool for processing 608/708 Closed Captions (CC) sources. For more information on the format, see <a data-manual="ffmpeg-formats" href="./ffmpeg-formats.html#rcwtenc">(ffmpeg-formats)rcwtenc. This demuxer implements the specification as of March 2024, which has been stable and unchanged since April 2014. <h4 class="subsection">15.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Render CC to ASS using the built-in decoder: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin CC.ass </pre> Note that if your output appears to be empty, you may have to manually set the decoder’s <samp class="option">data_field</samp> option to pick the desired CC substream. Convert an RCWT backup to Scenarist (SCC) format: <pre class="example-preformatted">ffmpeg -i CC.rcwt.bin -c:s copy CC.scc </pre> Note that the SCC format does not support all of the possible CC extensions that can be stored in RCWT (such as EIA-708).

 <h3 class="section">15.26 sbg# TOC</h3>

SBaGen script demuxer. This demuxer reads the script language used by SBaGen http://uazu.net/sbagen/ to generate binaural beats sessions. A SBG script looks like that: <div

class=”example”> <pre class=”example-preformatted”>-SE a: 300-2.5/3 440+4.5/0 b: 300-2.5/0 440+4.5/3 off: - NOW == a +0:07:00 == b +0:14:00 == a +0:21:00 == b +0:30:00 off </pre>

A SBG script can mix absolute and relative timestamps. If the script uses either only absolute timestamps (including the script start time) or only relative ones, then its layout is fixed, and the conversion is straightforward. On the other hand, if the script mixes both kind of timestamps, then the <var class=”var”>NOW</var> reference for relative timestamps will be taken from the current time of day at the time the script is read, and the script layout will be frozen according to that reference. That means that if the script is directly played, the actual times will match the absolute timestamps up to the sound controller’s clock accuracy, but if the user somehow pauses the playback or seeks, all times will be shifted accordingly. <h3 class=”section”>15.27 tedcaptions# TOC</h3>

JSON captions used for TED Talks. TED does not provide links to the captions, but they can be guessed from the page. The file <samp class=”file”>tools/bookmarklets.html</samp> from the FFmpeg source tree contains a bookmarklet to expose them. This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>start_time</samp></dt> <dd> Set the start time of the TED talk, in milliseconds. The default is 15000 (15s). It is used to sync the captions with the downloadable videos, because they include a 15s intro. </dd> </dl>

Example: convert the captions to a format most players understand: <pre class=”example-preformatted”>ffmpeg -i http://www.ted.com/talks/subtitles/id/1/lang/en talk1-en.srt </pre>

 <h3 class=”section”>15.28 vapoursynth# TOC</h3>

Vapoursynth wrapper. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class=”code”>-f vapoursynth</code> before the input <code class=”code”>-i yourscript.vpy</code>. This demuxer accepts the following option: <dl class=”table”> <dt><samp class=”option”>max_script_size</samp></dt> <dd> The demuxer buffers the entire script into memory. Adjust this value to set the maximum buffer size, which in turn, acts as a ceiling for the size of scripts that can be read. Default is 1 MiB. </dd> </dl>

 <h3 class=”section”>15.29 w64<span class=”pull-

right”>#
TOC</h3>

Sony Wave64 Audio demuxer. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>max_size</samp></dt> <dd> See the same option for the wav demuxer. </dd> </dl>

 <h3 class=”section”>15.30 wav# TOC</h3>

RIFF Wave Audio demuxer. This demuxer accepts the following options: <dl class=”table”> <dt><samp class=”option”>ignore_length <var class=”var”>bool</var></samp></dt> <dd> Ignore the size of the <code class=”code”>data</code> chunk and keep reading until the end of the file if set. May be useful to read broken or partial files where the header was not properly updated, but will misinterpret files with non-audio chunks after the <code class=”code”>data</code> chunk. Default is disabled. </dd> <dt><samp class=”option”>max_size</samp></dt> <dd> Specify the maximum packet size in bytes for the demuxed packets. By default this is set to 0, which means that a sensible value is chosen based on the input format. </dd> </dl>

 <h3 class=”section”>15.31 webp#
TOC</h3>

Animated WebP demuxer. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>-min_delay <var class=”var”>int</var></samp></dt> <dd> Set the minimum valid delay between frames in milliseconds. Range is 0 to 60000. Default value is 10. </dd> <dt><samp class=”option”>-max_webp_delay <var class=”var”>int</var></samp></dt> <dd> Set the maximum valid delay between frames in milliseconds. Range is 0 to 16777215. Default value is 16777215 (over four hours), the maximum value allowed by the specification. </dd> <dt><samp class=”option”>-default_delay <var class=”var”>int</var></samp></dt> <dd> Set the default delay between frames in milliseconds. Range is 0 to 60000. Default value is 100. </dd> <dt><samp class=”option”>-ignore_loop <var class=”var”>bool</var></samp></dt> <dd> WebP files can contain information to loop a certain number of times (or infinitely). If <samp class=”option”>ignore_loop</samp> is set to true, then the loop setting from the input will be ignored and looping will not occur. If set to false, then looping will occur and will cycle the number of times according to the WebP. Default value is true. </dd> <dt><samp class=”option”>usebgcolor <var class=”var”>bool</var></samp></dt> <dd> WebP files contain

a background color hint in the ANIM chunk, but the WebP specification says that viewer applications are not required to use it. If <samp class=”option”>usebgcolor</samp> is set to true, then the background color hint will be used, otherwise transparent black will be used for the background. Default value is false. </dd> </dl>

 <h2 class=”chapter”>16 Metadata# TOC</h2>

FFmpeg is able to dump metadata from media files into a simple UTF-8-encoded INI-like text file and then load it back using the metadata muxer/demuxer. The file format is as follows: <ol class=”enumerate”> A file consists of a header and a number of metadata tags divided into sections, each on its own line.

 The header is a ‘<samp class=”samp”>;FFMETADATA</samp>’ string, followed by a version number (now 1).

 Metadata tags are of the form ‘<samp class=”samp”>key=value</samp>’

 Immediately after header follows global metadata

 After global metadata there may be sections with per-stream/per-chapter metadata.

 A section starts with the section name in uppercase (i.e. STREAM or CHAPTER) in brackets (‘<samp class=”samp”>[</samp>’, ‘<samp class=”samp”>]</samp>’) and ends with next section or end of file.

 At the beginning of a chapter section there may be an optional timebase to be used for start/end values. It must be in form ‘<samp class=”samp”>TIMEBASE=<var class=”var”>num</var>/<var class=”var”>den</var></samp>’ where <var class=”var”>num</var> and <var class=”var”>den</var> are integers. If the timebase is missing then start/end times are assumed to be in nanoseconds.

Next a chapter section must contain chapter start and end times in form ‘<samp class=”samp”>START=<var class=”var”>num</var></samp>’, ‘<samp class=”samp”>END=<var class=”var”>num</var></samp>’, where <var class=”var”>num</var> is a positive integer. Empty lines and lines starting with ‘<samp class=”samp”>;</samp>’ or ‘<samp class=”samp”>#</samp>’ are ignored.

 Metadata keys or values containing special characters (‘<samp class=”samp”>=</samp>’, ‘<samp class=”samp”>;</samp>’, ‘<samp class=”samp”>#</samp>’, ‘<samp class=”samp”>\</samp>’ and a newline) must be escaped with a backslash ‘<samp class=”samp”>\</samp>’.

 Note that whitespace in metadata (e.g. ‘<samp class="samp">foo = bar</samp>’) is considered to be a part of the tag (in the example above key is ‘<samp class="samp">foo </samp>’, value is ‘<samp class="samp"> bar</samp>’).

A ffmetadata file might look like this: <pre class="example-preformatted">;FFMETADATA1 title=bike\\shed ;this is a comment artist=FFmpeg troll team

[CHAPTER] TIMEBASE=1/1000 START=0 #chapter ends at 0:01:00 END=60000 title=chapter \#1 [STREAM] title=multi\ line </pre>

By using the ffmetadata muxer and demuxer it is possible to extract metadata from an input file to an ffmetadata file, and then transcode the file into an output file with the edited ffmetadata file. Extracting an ffmetadata file with <samp class="file">ffmpeg</samp> goes as follows: <pre class="example-preformatted">ffmpeg -i INPUT -f ffmetadata FFMETADATAFILE </pre>

Reinserting edited metadata information from the FFMETADATAFILE file can be done as: <pre class="example-preformatted">ffmpeg -i INPUT -i FFMETADATAFILE -map_metadata 1 -codec copy OUTPUT </pre>

 <h2 class="chapter">17 Protocol Options# TOC</h2>

The libavformat library provides some generic global options, which can be set on all the protocols. In addition each protocol may support so-called private options, which are specific for that component. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, or by setting the value explicitly in the <code class="code">AVFormatContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. The list of supported options follows: <dl class="table"> <dt><samp class="option">protocol_whitelist <var class="var">list</var> (input)</samp></dt> <dd> Set a ","-separated list of allowed protocols. "ALL" matches all protocols. Protocols prefixed by "-" are disabled. All protocols are allowed by default but protocols used by an another protocol (nested protocols) are restricted to a per protocol subset. </dd> </dl>

 <h2 class="chapter">18 Protocols# TOC</h2>

Protocols are configured elements in FFmpeg that enable access to resources that require specific protocols. When you configure your FFmpeg build, all the supported protocols are enabled by default. You can list all available ones using the configure option "–list-protocols". You can disable all the protocols using the configure option "–disable-protocols", and selectively enable a protocol using the option "–enable-protocol=<var class="var">PROTOCOL</var>", or you can disable a particular protocol using the option "–disable-protocol=<var class="var">PROTOCOL</var>". The option "-protocols" of the ff* tools will display the list of supported protocols. All protocols accept the following options: <dl class="table"> <dt><samp class="option">rw_timeout</samp></dt> <dd> Maximum time to wait for (network) read/write operations to complete, in microseconds. </dd> </dl>

A description of the currently available protocols follows. <h3 class="section">18.1 amqp# TOC</h3>

Advanced Message Queueing Protocol (AMQP) version 0-9-1 is a broker based publish-subscribe communication protocol. FFmpeg must be compiled with –enable-librabbitmq to support AMQP. A separate AMQP broker must also be run. An example open-source AMQP broker is RabbitMQ. After starting the broker, an FFmpeg client may stream data to the broker using the command: <pre class="example-preformatted">ffmpeg -re -i input -f mpegts amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

Where hostname and port (default is 5672) is the address of the broker. The client may also set a user/password for authentication. The default for both fields is "guest". Name of virtual host on broker can be set with vhost. The default value is "/". Multiple subscribers may stream from the broker using the command: <pre class="example-preformatted">ffplay amqp://[[user]:[password]@]hostname[:port][/vhost] </pre>

In RabbitMQ all data published to the broker flows through a specific exchange, and each subscribing client has an assigned queue/buffer. When a packet arrives at an exchange, it may be copied to a client’s queue depending on the exchange and routing_key fields. The following options are supported: <dl class="table"> <dt><samp class="option">exchange</samp></dt> <dd> Sets the exchange to use on the broker. RabbitMQ has several predefined exchanges: "amq.direct" is the default exchange, where the publisher and subscriber must have a matching routing_key; "amq.fanout" is the same as a broadcast operation (i.e. the data is forwarded to all queues on the

fanout exchange independent of the routing_key); and "amq.topic" is similar to "amq.direct", but allows for more complex pattern matching (refer to the RabbitMQ documentation). </dd> <dt><samp class="option">routing_key</samp></dt> <dd> Sets the routing key. The default value is "amqp". The routing key is used on the "amq.direct" and "amq.topic" exchanges to decide whether packets are written to the queue of a subscriber. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Maximum size of each packet sent/received to the broker. Default is 131072. Minimum is 4096 and max is any large value (representable by an int). When receiving packets, this sets an internal buffer size in FFmpeg. It should be equal to or greater than the size of the published packets to the broker. Otherwise the received message may be truncated causing decoding errors. </dd> <dt><samp class="option">connection_timeout</samp></dt> <dd> The timeout in seconds during the initial connection to the broker. The default value is rw_timeout, or 5 seconds if rw_timeout is not set. </dd> <dt><samp class="option">delivery_mode <var class="var">mode</var></samp></dt> <dd> Sets the delivery mode of each message sent to broker. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">persistent</samp>’</dt> <dd> Delivery mode set to "persistent" (2). This is the default value. Messages may be written to the broker’s disk depending on its setup. </dd> <dt>‘<samp class="samp">non-persistent</samp>’</dt> <dd> Delivery mode set to "non-persistent" (1). Messages will stay in broker’s memory unless the broker is under memory pressure. </dd> </dl>

</dd> </dl>

 <h3 class="section">18.2 async# TOC</h3>

Asynchronous data filling wrapper for input stream. Fill data in a background thread, to decouple I/O operation from demux thread.   <pre class="example-preformatted">async:<var class="var">URL</var> async:http://host/resource async:cache:http://host/resource </pre>

 <h3 class="section">18.3 bluray# TOC</h3>

Read BluRay playlist. The accepted options are: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> BluRay angle </dd> <dt><samp class="option">chapter</samp></dt> <dd> Start chapter (1...N) </dd> <dt><samp class="option">playlist</samp></dt> <dd> Playlist to read (BDMV/PLAYLIST/?????.mpls) </dd>

</dl>

Examples: Read longest playlist from BluRay mounted to /mnt/bluray: <pre class="example-preformatted">bluray:/mnt/bluray </pre>

Read angle 2 of playlist 4 from BluRay mounted to /mnt/bluray, start from chapter 2: <pre class="example-preformatted">-playlist 4 -angle 2 -chapter 2 bluray:/mnt/bluray </pre>

 <h3 class="section">18.4 cache# TOC</h3>

Caching wrapper for input stream. Cache the input stream to temporary file. It brings seeking capability to live streams. The accepted options are: <dl class="table"> <dt><samp class="option">read_ahead_limit</samp></dt> <dd> Amount in bytes that may be read ahead when seeking isn’t supported. Range is -1 to INT_MAX. -1 for unlimited. Default is 65536. </dd> </dl>

URL Syntax is <pre class="example-preformatted">cache:<var class="var">URL</var> </pre>

 <h3 class="section">18.5 concat# TOC</h3>

Physical concatenation protocol. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class="example-preformatted">concat:<var class="var">URL1</var>|<var class="var">URL2</var>|...|<var class="var">URLN</var> </pre>

where <var class="var">URL1</var>, <var class="var">URL2</var>, ..., <var class="var">URLN</var> are the urls of the resource to be concatenated, each one possibly specifying a distinct protocol. For example to read a sequence of files <samp class="file">split1.mpeg</samp>, <samp class="file">split2.mpeg</samp>, <samp class="file">split3.mpeg</samp> with <code class="command">ffplay</code> use the command: <pre class="example-preformatted">ffplay concat:split1.mpeg\|split2.mpeg\|split3.mpeg </pre>

Note that you may need to escape the character "|" which is special for many shells. <h3 class="section">18.6 concatf# TOC</h3>

Physical concatenation protocol using a line break delimited list of resources. Read and seek from many resources in sequence as if they were a unique resource. A URL accepted by this protocol has the syntax: <pre class=”example-preformatted”>concatf:<var class=”var”>URL</var> </pre>

where <var class=”var”>URL</var> is the url containing a line break delimited list of resources to be concatenated, each one possibly specifying a distinct protocol. Special characters must be escaped with backslash or single quotes. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#quoting_005fand_005fescaping”>the "Quoting and escaping" section in the ffmpeg-utils(1) manual. For example to read a sequence of files <samp class=”file”>split1.mpeg</samp>, <samp class=”file”>split2.mpeg</samp>, <samp class=”file”>split3.mpeg</samp> listed in separate lines within a file <samp class=”file”>split.txt</samp> with <code class=”command”>ffplay</code> use the command: <pre class=”example-preformatted”>ffplay concatf:split.txt </pre> Where <samp class=”file”>split.txt</samp> contains the lines: <pre class=”example-preformatted”>split1.mpeg split2.mpeg split3.mpeg </pre>

 <h3 class=”section”>18.7 crypto#
TOC</h3>

AES-encrypted stream reading protocol. The accepted options are: <dl class=”table”> <dt><samp class=”option”>key</samp></dt>
<dd> Set the AES decryption key binary block from given hexadecimal representation. </dd> <dt><samp class=”option”>iv</samp></dt>
<dd> Set the AES decryption initialization vector binary block from given hexadecimal representation. </dd> </dl>

Accepted URL formats: <pre class=”example-preformatted”>crypto:<var class=”var”>URL</var> crypto+<var class=”var”>URL</var>
</pre>

 <h3 class=”section”>18.8 data#
TOC</h3>

Data in-line in the URI. See <a class=”url” href=”http://en.wikipedia.org/wiki/Data_URI_scheme”>http
 For example, to convert a GIF file given inline with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i "
smiley.png </pre>

 <h3 class=”section”>18.9 fd#
TOC</h3>

File descriptor access protocol. The accepted syntax is: <pre class="example-preformatted">fd: -fd <var class="var">file_descriptor</var> </pre>

If <samp class="option">fd</samp> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. Unlike the pipe protocol, fd protocol has seek support if it corresponding to a regular file. fd protocol doesn’t support pass file descriptor via URL for security. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

 <h3 class="section">18.10 file# TOC</h3>

File access protocol. Read from or write to a file. A file URL can have the form: <pre class="example-preformatted">file:<var class="var">filename</var> </pre>

where <var class="var">filename</var> is the path of the file to read. An URL that does not have a protocol prefix will be assumed to be a file URL. Depending on the build, an URL that looks like a Windows path with the drive letter at the beginning will also be assumed to be a file URL (usually not the case in builds for unix-like systems). For example to read from a file <samp class="file">input.mpeg</samp> with <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -i file:input.mpeg output.mpeg </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable for files on slow medium. </dd> <dt><samp class="option">follow</samp></dt> <dd> If set to 1, the protocol will retry reading at the end of the file, allowing reading files that still are being written. In order for this to terminate, you either need to use the rw_timeout option, or use the interrupt callback (for API

users). Setting this option also ignores the file size reported by the file system. </dd> <dt><samp class=”option”>seekable</samp></dt> <dd> Controls if seekability is advertised on the file. 0 means non-seekable, -1 means auto (seekable for normal files, non-seekable for named pipes). Many demuxers handle seekable and non-seekable resources differently, overriding this might speed up opening certain files at the cost of losing some features (e.g. accurate seeking). </dd> <dt><samp class=”option”>pkt_size</samp></dt> <dd> Set the maximum packet size used for file I/O. A smaller value may reduce memory usage. A higher value may increase throughput especially with networked filesystems. For reading, if explicitly set, it overrides the default internal buffer size (32 KB) and limits the maximum amount of data read per operation. For writing, this sets the size of each write operation. The default is 256 KB for regular files, 32 KB otherwise. </dd> </dl>

 <h3 class=”section”>18.11 ftp# TOC</h3>

FTP (File Transfer Protocol). Read from or write to remote resources using FTP protocol. Following syntax is required. <pre class=”example-preformatted”>ftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg </pre>

This protocol accepts the following options. <dl class=”table”> <dt><samp class=”option”>timeout</samp></dt> <dd> Set timeout in microseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class=”option”>ftp-user</samp></dt> <dd> Set a user to be used for authenticating to the FTP server. This is overridden by the user in the FTP URL. </dd> <dt><samp class=”option”>ftp-password</samp></dt> <dd> Set a password to be used for authenticating to the FTP server. This is overridden by the password in the FTP URL, or by <samp class=”option”>ftp-anonymous-password</samp> if no user is set. </dd> <dt><samp class=”option”>ftp-anonymous-password</samp></dt> <dd> Password used when login as anonymous user. Typically an e-mail address should be used. </dd> <dt><samp class=”option”>ftp-write-seekable</samp></dt> <dd> Control seekability of connection during encoding. If set to 1 the resource is supposed to be seekable, if set to 0 it is assumed not to be seekable. Default value is 0. </dd> </dl>

NOTE: Protocol can be used as output, but it is recommended to not do it, unless special care is taken (tests, customized server configuration etc.). Different FTP servers behave in different way during seek operation. ff* tools may produce incomplete content due to server limitations. <h3 class=”section”>18.12 gopher<span

class=”pull-right”># TOC</h3>

Gopher protocol. <h3 class=”section”>18.13 gophers# TOC</h3>

Gophers protocol. The Gopher protocol with TLS encapsulation. <h3 class=”section”>18.14 handle# TOC</h3>

Windows handle access protocol. The accepted syntax is: <pre class=”example-preformatted”>handle:-handle <var class=”var”>handle</var> </pre>

If <samp class=”option”>handle</samp> is not specified, by default the output handle will be used for writing, input handle for reading. This protocol is only available on Windows. It has seek support if the handle corresponds to a regular file. It doesn’t support passing the handle via URL. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class=”code”>INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class=”option”>handle</samp></dt> <dd> Set Windows handle. The supplied handle must already be valid in the calling process. When launching <code class=”command”>ffmpeg</code>, arrange for the child process to inherit the handle before passing its numeric value with <samp class=”option”>handle</samp>. Handles opened with <code class=”code”>FILE_FLAG_OVERLAPPED</code> are not supported. Closing the protocol does not close the caller’s original handle. </dd> </dl>

 <h3 class=”section”>18.15 http# TOC</h3>

HTTP (Hyper Text Transfer Protocol).

This protocol accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>seekable</samp></dt>
<dd> Control seekability of connection. If set to 1 the resource is
supposed to be seekable, if set to 0 it is assumed not to be seekable,
if set to -1 it will try to autodetect if it is seekable. Default
value is -1.

</dd>
<dt><samp class=”option”>chunked_post</samp></dt>
<dd> If set to 1 use chunked Transfer-Encoding for posts, default is 1.

</dd>
<dt><samp class=”option”>http_proxy</samp></dt>
<dd> set HTTP proxy to tunnel through e.g. http://example.com:1234

</dd>
<dt><samp class=”option”>headers</samp></dt>
<dd> Set custom HTTP headers, can override built in default headers. The
value must be a string encoding the headers.

</dd>
<dt><samp class=”option”>content_type</samp></dt>
<dd> Set a specific content type for the POST messages or for listen mode.

</dd>
<dt><samp class=”option”>user_agent</samp></dt>
<dd> Override the User-Agent header. If not specified the protocol will use
a
string describing the libavformat build. ("Lavf/<version>")

</dd>
<dt><samp class=”option”>referer</samp></dt>

<dd> Set the Referer header. Include ’Referer: URL’ header in HTTP request.

</dd>

<dt><samp class="option">multiple_requests</samp></dt>

<dd> Force persistent connections if set to 1, or disable if 0. Default is -1,

which means auto (implies keep-alive when using -request_size or

-initial_request_size).

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> Limit the size of requests made. This is useful for some pathological servers

that throttle unbounded range requests, as well as when expecting to seek

frequently. Disabled (set to 0) by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher. Doing so allows FFmpeg

to reuse a single HTTP connection wherever possible.

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also enable

the <samp class="option">multiple_requests</samp> option, as well as setting

<samp class="option">short_seek_size</samp> to the same value or higher.

</dd>

<dt><samp class="option">post_data</samp></dt>

<dd> Set custom HTTP post data.

</dd>

<dt><samp class="option">mime_type</samp></dt>

<dd> Export the MIME type.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> Exports the HTTP response version number. Usually "1.0" or "1.1".

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Set the cookies to be sent in future requests. The format of each cookie is the

same as the value of a Set-Cookie HTTP response field. Multiple cookies can be

delimited by a newline character. A cookie without a Domain attribute is only

sent to the host of the URL it is set for.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class=”option”>icy_metadata_headers</samp> and <samp class=”option”>icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class=”option”>icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class=”option”>icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class=”option”>icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class=”option”>metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

<dt><samp class=”option”>auth_type</samp></dt>

<dd>

Set HTTP authentication type. No option for Digest, since this method requires

getting nonce parameters from the server first and can’t be used straight away like

Basic.

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> Choose the HTTP authentication type automatically. This is the default.

</dd>

<dt><samp class=”option”>basic</samp></dt>

<dd>

Choose the HTTP basic authentication.

Basic authentication sends a Base64-encoded string that contains a user name and password

for the client. Base64 is not a form of encryption and should be considered the same as

sending the user name and password in clear text (Base64 is a reversible encoding).

If a resource needs to be protected, strongly consider using an authentication scheme

other than basic authentication. HTTPS/TLS should be used with basic authentication.

Without these additional security enhancements, basic authentication should not be used

to protect sensitive or valuable information.

</dd>

</dl>

</dd> <dt><samp class=”option”>send_expect_100</samp></dt> <dd> Send an Expect: 100-continue header for POST. If set to 1 it will send, if set to 0 it won’t, if set to -1 it will try to send if it is applicable. Default value is -1. </dd> <dt><samp class=”option”>location</samp></dt> <dd> An exported dictionary containing the content location. Only useful with the C API. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set initial byte offset. </dd> <dt><samp class=”option”>end_offset</samp></dt> <dd> Try to limit the request to bytes preceding this offset. </dd> <dt><samp class=”option”>method</samp></dt> <dd> When used as a client option it sets the HTTP method for the request. When used as a server option it sets the HTTP method that is going to be expected from the client(s). If the expected and the received HTTP method do not

match the client will be given a Bad Request response. When unset the HTTP method is not checked for now. This will be replaced by autodetection in the future. </dd> <dt><samp class="option">reconnect</samp></dt> <dd> Reconnect automatically when disconnected before EOF is hit. </dd> <dt><samp class="option">reconnect_at_eof</samp></dt> <dd> If set then eof is treated like an error and causes reconnection, this is useful for live / endless streams. This does not apply to the deliberately established end of a resource whose complete size is unknown, i.e. when a ranged request past the end is answered with <code class="code">416</code>. Such an end is reported as a clean EOF and is not retried, unless <samp class="option">reconnect_on_http_error</samp> lists 416 by number. </dd> <dt><samp class="option">reconnect_on_network_error</samp></dt> <dd> Reconnect automatically in case of TCP/TLS errors during connect. </dd> <dt><samp class="option">reconnect_on_http_error</samp></dt> <dd> A comma separated list of HTTP status codes to reconnect on. The list can include specific status codes (e.g. ’503’) or the strings ’4xx’ / ’5xx’. A <code class="code">416</code> that ends a read or a seek past the end of a resource whose complete size is unknown is reported as EOF. It is retried first only if 416 is listed by number, for example to read a resource that is still being written. ’4xx’ does not cover it. </dd> <dt><samp class="option">reconnect_streamed</samp></dt> <dd> If set then even streamed/non seekable streams will be reconnected on errors. </dd> <dt><samp class="option">reconnect_delay_max</samp></dt> <dd> Set the maximum delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">reconnect_max_retries</samp></dt> <dd> Set the maximum number of times to retry a connection. Default unset. </dd> <dt><samp class="option">reconnect_delay_total_max</samp></dt> <dd> Set the maximum total delay in seconds after which to give up reconnecting. </dd> <dt><samp class="option">respect_retry_after</samp></dt> <dd> If enabled, and a Retry-After header is encountered, its requested reconnection delay will be honored, rather than using exponential backoff. Useful for 429 and 503 errors. Default enabled. </dd> <dt><samp class="option">listen</samp></dt> <dd> If set to 1 enables experimental HTTP server. This can be used to send data when used as an output option, or read data from a client with HTTP POST when used as an input option. If set to 2 enables experimental multi-client HTTP server. This is not yet implemented in ffmpeg.c and thus must not be used as a command line option. <pre class="example-preformatted"># Server side (sending): ffmpeg -i somefile.ogg -c copy -listen 1 -f ogg http://<var class="var">server</var>:<var class="var">port</var>

Client side (receiving): ffmpeg -i http://<var class="var">server</var>:<var class="var">port</var> -c copy somefile.ogg

Client can also be done with wget: wget http://<var class="var">server</var>:<var class="var">port</var> -O somefile.ogg

Server side (receiving): ffmpeg -listen 1 -i http://<var class=”var”>server</var>:<var class=”var”>port</var> -c copy somefile.ogg

Client side (sending): ffmpeg -i somefile.ogg -chunked_post 0 -c copy -f ogg http://<var class=”var”>server</var>:<var class=”var”>port</var>

Client can also be done with wget: wget --post-file=somefile.ogg http://<var class=”var”>server</var>:<var class=”var”>port</var> </pre>

</dd> <dt><samp class=”option”>resource</samp></dt> <dd> The resource requested by a client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>reply_code</samp></dt> <dd> The HTTP code returned to the client, when the experimental HTTP server is in use. </dd> <dt><samp class=”option”>short_seek_size</samp></dt> <dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and new HTTP request. This is useful, for example, to make sure the same connection is used for reading large video packets with small audio packets in between. </dd> </dl>

 <h4 class=”subsection”>18.15.1 HTTP Cookies# TOC</h4>

Some HTTP requests will be denied unless cookie values are passed in with the request. The <samp class=”option”>cookies</samp> option allows these cookies to be specified. At the very least, each cookie must specify a value. HTTP requests that match both the domain and path will automatically include the cookie value in the HTTP Cookie header field. Multiple cookies can be delimited by a newline. The required syntax to play a stream specifying a cookie is: <pre class=”example-preformatted”>ffplay -cookies "nlqptid=nltid=tsn; path=/; domain=somedomain.com;" http://somedomain.com/somestream.m3u8 </pre>

 <h3 class=”section”>18.16 Icecast# TOC</h3>

Icecast protocol (stream to Icecast servers) This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>ice_genre</samp></dt> <dd> Set the stream genre. </dd> <dt><samp class=”option”>ice_name</samp></dt> <dd> Set the stream name. </dd> <dt><samp class=”option”>ice_description</samp></dt> <dd> Set the stream description. </dd> <dt><samp class=”option”>ice_url</samp></dt> <dd> Set the stream website URL. </dd> <dt><samp class=”option”>ice_public</samp></dt> <dd> Set if the stream should be public. The default is 0 (not public). </dd> <dt><samp

class=”option”>user_agent</samp></dt> <dd> Override the User-Agent header. If not specified a string of the form "Lavf/<version>" will be used. </dd> <dt><samp class=”option”>password</samp></dt> <dd> Set the Icecast mountpoint password. </dd> <dt><samp class=”option”>content_type</samp></dt> <dd> Set the stream content type. This must be set if it is different from audio/mpeg. </dd> <dt><samp class=”option”>legacy_icecast</samp></dt> <dd> This enables support for Icecast versions < 2.4.0, that do not support the HTTP PUT method but the SOURCE method. </dd> <dt><samp class=”option”>tls</samp></dt> <dd> Establish a TLS (HTTPS) connection to Icecast. </dd> </dl>

 <pre class=”example-preformatted”>icecast://[<var class=”var”>username</var>[:<var class=”var”>password</var>]@]<var class=”var”>server</var>:<var class=”var”>port</var>/<var class=”var”>mountpoint</var> </pre>

 <h3 class=”section”>18.17 ipfs# TOC</h3>

InterPlanetary File System (IPFS) protocol support. One can access files stored on the IPFS network through so-called gateways. These are http(s) endpoints. This protocol wraps the IPFS native protocols (ipfs:// and ipns://) to be sent to such a gateway. Users can (and should) host their own node which means this protocol will use one’s local gateway to access files on the IPFS network. This protocol accepts the following options: <dl class=”table”> <dt><samp class=”option”>gateway</samp></dt> <dd> Defines the gateway to use. When not set, the protocol will first try locating the local gateway by looking at <code class=”code”>$IPFS_GATEWAY</code>, <code class=”code”>$IPFS_PATH</code> and <code class=”code”>$HOME/.ipfs/</code>, in that order. </dd> </dl>

One can use this protocol in 2 ways. Using IPFS: <pre class=”example-preformatted”>ffplay ipfs://<hash> </pre>

Or the IPNS protocol (IPNS is mutable IPFS): <pre class=”example-preformatted”>ffplay ipns://<hash> </pre>

 <h3 class=”section”>18.18 mmst# TOC</h3>

MMS (Microsoft Media Server) protocol over TCP. <h3 class=”section”>18.19 mmsh# TOC</h3>

MMS (Microsoft Media Server) protocol over HTTP. The required syntax is: <pre class="example-preformatted">mmsh://<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">playpath</var>] </pre>

 <h3 class="section">18.20 md5#
TOC</h3>

MD5 output protocol. Computes the MD5 hash of the data to be written, and on close writes this to the designated output or stdout if none is specified. It can be used to test muxers without writing an actual file. Some examples follow. <pre class="example-preformatted"># Write the MD5 hash of the encoded AVI file to the file output.avi.md5. ffmpeg -i input.flv -f avi -y md5:output.avi.md5

Write the MD5 hash of the encoded AVI file to stdout. ffmpeg -i input.flv -f avi -y md5: </pre>

Note that some formats (typically MOV) require the output protocol to be seekable, so they will fail with the MD5 output protocol. <h3 class="section">18.21 pipe# TOC</h3>

UNIX pipe access protocol. Read and write from UNIX pipes. The accepted syntax is: <pre class="example-preformatted">pipe:[<var class="var">number</var>] </pre>

If <samp class="option">fd</samp> isn’t specified, <var class="var">number</var> is the number corresponding to the file descriptor of the pipe (e.g. 0 for stdin, 1 for stdout, 2 for stderr). If <var class="var">number</var> is not specified, by default the stdout file descriptor will be used for writing, stdin for reading. For example to read from stdin with <code class="command">ffmpeg</code>: <pre class="example-preformatted">cat test.wav | ffmpeg -i pipe:0 # ...this is the same as... cat test.wav | ffmpeg -i pipe: </pre>

For writing to stdout with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i test.wav -f avi pipe:1 | cat > test.avi # ...this is the same as... ffmpeg -i test.wav -f avi pipe: | cat > test.avi </pre>

This protocol accepts the following options: <dl class="table"> <dt><samp class="option">blocksize</samp></dt> <dd> Set I/O operation maximum block size, in bytes. Default value is <code class="code">INT_MAX</code>, which results in not limiting the requested block size. Setting this value reasonably low improves user termination request

reaction time, which is valuable if data transmission is slow. </dd> <dt><samp class="option">fd</samp></dt> <dd> Set file descriptor. </dd> </dl>

Note that some formats (typically MOV), require the output protocol to be seekable, so they will fail with the pipe output protocol. <h3 class="section">18.22 prompeg# TOC</h3>

Pro-MPEG Code of Practice #3 Release 2 FEC protocol. The Pro-MPEG CoP#3 FEC is a 2D parity-check forward error correction mechanism for MPEG-2 Transport Streams sent over RTP. This protocol must be used in conjunction with the <code class="code">rtp_mpegts</code> muxer and the <code class="code">rtp</code> protocol. The required syntax is: <pre class="example-preformatted">-f rtp_mpegts -fec prompeg=<var class="var">option</var>=<var class="var">val</var>... rtp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

The destination UDP ports are <code class="code">port + 2</code> for the column FEC stream and <code class="code">port + 4</code> for the row FEC stream. This protocol accepts the following options: <dl class="table"> <dt><samp class="option">l=<var class="var">n</var></samp></dt> <dd> The number of columns (4-20, LxD <= 100) </dd> <dt><samp class="option">d=<var class="var">n</var></samp></dt> <dd> The number of rows (4-20, LxD <= 100) </dd> </dl>

Example usage: <pre class="example-preformatted">-f rtp_mpegts -fec prompeg=l=8:d=4 rtp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">18.23 rist# TOC</h3>

Reliable Internet Streaming Transport protocol The accepted options are: <dl class="table"> <dt><samp class="option">rist_profile</samp></dt> <dd> Supported values: <dl class="table"> <dt>‘<samp class="samp">simple</samp>’</dt> <dt>‘<samp class="samp">main</samp>’</d <dd> This one is default. </dd> <dt>‘<samp class="samp">advanced</samp>’</ </dl>

</dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set internal RIST buffer size in milliseconds for retransmission of data. Default value is 0 which means the librist default (1 sec). Maximum value is 30 seconds. </dd> <dt><samp class="option">fifo_size</samp></dt>

<dd> Size of the librist receiver output fifo in number of packets. This must be a power of 2. Defaults to 8192 (vs the librist default of 1024). </dd> <dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt> <dd> Survive in case of librist fifo buffer overrun. Default value is 0. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set maximum packet size for sending data. 1316 by default. </dd> <dt><samp class="option">log_level</samp></dt> <dd> Set loglevel for RIST logging messages. You only need to set this if you explicitly want to enable debug level messages or packet loss simulation, otherwise the regular loglevel is respected. </dd> <dt><samp class="option">secret</samp></dt> <dd> Set override of encryption secret, by default is unset. </dd> <dt><samp class="option">encryption</samp></dt> <dd> Set encryption type, by default is disabled. Acceptable values are 128 and 256. </dd> </dl>

 <h3 class="section">18.24 rtmp# TOC</h3>

Real-Time Messaging Protocol. The Real-Time Messaging Protocol (RTMP) is used for streaming multimedia content across a TCP/IP network. The required syntax is: <pre class="example-preformatted">rtmp://[<var class="var">username</var>:<var class="var">password</var>@]<var class="var">server</var>[:<var class="var">port</var>][/<var class="var">app</var>][/<var class="var">instance</var>][/<var class="var">playpath</var>] </pre>

The accepted parameters are: <dl class="table"> <dt><samp class="option">username</samp></dt> <dd> An optional username (mostly for publishing). </dd> <dt><samp class="option">password</samp></dt> <dd> An optional password (mostly for publishing). </dd> <dt><samp class="option">server</samp></dt> <dd> The address of the RTMP server. </dd> <dt><samp class="option">port</samp></dt> <dd> The number of the TCP port to use (by default is 1935). </dd> <dt><samp class="option">app</samp></dt> <dd> It is the name of the application to access. It usually corresponds to the path where the application is installed on the RTMP server (e.g. <samp class="file">/ondemand/</samp>, <samp class="file">/flash/live/</samp>, etc.). You can override the value parsed from the URI through the <code class="code">rtmp_app</code> option, too. </dd> <dt><samp class="option">playpath</samp></dt> <dd> It is the path or name of the resource to play with reference to the application specified in <var class="var">app</var>, may be prefixed by "mp4:". You can override the value parsed from the URI through the <code class="code">rtmp_playpath</code> option, too. </dd> <dt><samp class="option">listen</samp></dt> <dd> Act as a

server, listening for an incoming connection. </dd> <dt><samp class=”option”>timeout</samp></dt> <dd> Maximum time to wait for the incoming connection. Implies listen. </dd> </dl>

Additionally, the following parameters can be set via command line options

(or in code via <code class=”code”>AVOption</code>s):

<dl class=”table”>

<dt><samp class=”option”>rtmp_app</samp></dt>

<dd> Name of application to connect on the RTMP server. This option

overrides the parameter specified in the URI.

</dd>

<dt><samp class=”option”>rtmp_buffer</samp></dt>

<dd> Set the client buffer time in milliseconds. The default is 3000.

</dd>

<dt><samp class=”option”>rtmp_conn</samp></dt>

<dd> Extra arbitrary AMF connection parameters, parsed from a string,

e.g. like <code class=”code”>B:1 S:authMe O:1 NN:code:1.23 NS:flag:ok O:0</code>.

Each value is prefixed by a single character denoting the type,

B for Boolean, N for number, S for string, O for object, or Z for null,

followed by a colon. For Booleans the data must be either 0 or 1 for

FALSE or TRUE, respectively. Likewise for Objects the data must be 0 or

1 to end or begin an object, respectively. Data items in subobjects may

be named, by prefixing the type with ’N’ and specifying the name before

the value (i.e. <code class=”code”>NB:myFlag:1</code>). This option may be used multiple

times to construct arbitrary AMF sequences.

</dd>

<dt><samp class=”option”>rtmp_enhanced_codecs</samp></dt>

<dd> Specify the list of codecs the client advertises to support in an

enhanced RTMP stream. This option should be set to a comma separated

list of fourcc values, like <code class=”code”>hvc1,av01,vp09</code> for multiple codecs

or <code class=”code”>hvc1</code> for only one codec. The specified list will be presented

in the "fourCcLive" property of the Connect Command Message.

</dd>

<dt><samp class=”option”>rtmp_flashver</samp></dt>

<dd> Version of the Flash plugin used to run the SWF player. The default

is LNX 9,0,124,2. (When publishing, the default is FMLE/3.0 (compatible;

<libavformat version>).)

</dd>

<dt><samp class=”option”>rtmp_flush_interval</samp></dt>

<dd> Number of packets flushed in the same request (RTMPT only). The default

is 10.

</dd>

<dt><samp class=”option”>rtmp_live</samp></dt>

<dd> Specify that the media is a live stream. No resuming or seeking in

live streams is possible. The default value is <code class=”code”>any</code>, which means the

subscriber first tries to play the live stream specified in the

playpath. If a live stream of that name is not found, it plays the

recorded stream. The other possible values are <code class=”code”>live</code> and

<code class=”code”>recorded</code>.

</dd>

<dt><samp class=”option”>rtmp_pageurl</samp></dt>

<dd> URL of the web page in which the media was embedded. By default no

value will be sent.

</dd>

<dt><samp class=”option”>rtmp_playpath</samp></dt>

<dd> Stream identifier to play or to publish. This option overrides the

parameter specified in the URI.

</dd>

<dt><samp class=”option”>rtmp_subscribe</samp></dt>

<dd> Name of live stream to subscribe to. By default no value will be sent.

It is only sent if the option is specified or if rtmp_live

is set to live.

</dd>

<dt><samp class=”option”>rtmp_swfhash</samp></dt>

<dd> SHA256 hash of the decompressed SWF file (32 bytes).

</dd>

<dt><samp class=”option”>rtmp_swfsize</samp></dt>

<dd> Size of the decompressed SWF file, required for SWFVerification.

</dd>

<dt><samp class=”option”>rtmp_swfurl</samp></dt>

<dd> URL of the SWF player for the media. By default no value will be sent.

</dd>

<dt><samp class=”option”>rtmp_swfverify</samp></dt>

<dd> URL to player swf file, compute hash/size automatically.

</dd>

<dt><samp class="option">rtmp_tcurl</samp></dt>

<dd> URL of the target stream. Defaults to proto://host[:port]/app.

</dd>

<dt><samp class="option">tcp_nodelay=<var class="var">1|0</var></samp></dt>

<dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0.

Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY.

</dd>

<dt><samp class="option">tcp_keepalive=<var class="var">1|0</var></samp></dt>

<dd> Enable the TCP keepalive mechanism to detect dead peers and help maintain long-lived idle connections. Default value is 0.

Only the basic keepalive option (SO_KEEPALIVE) can be enabled or disabled. Platform-specific tuning parameters such as TCP_KEEPIDLE, TCP_KEEPINTVL, or TCP_KEEPCNT are not configurable and will use the operating system’s default values.

</dd>

</dl>

For example to read with <code class="command">ffplay</code> a multimedia resource named "sample" from the application "vod" from an RTMP server "myserver": <pre class="example-preformatted">ffplay rtmp://myserver/vod/sample </pre>

To publish to a password protected server, passing the playpath and app names separately: <pre class="example-

preformatted”>ffmpeg -re -i <input> -f flv -rtmp_playpath some/long/path -rtmp_app long/app/name rtmp://username:password@myserver/ </pre>

 <h3 class=”section”>18.25 rtmpe# TOC</h3>

Encrypted Real-Time Messaging Protocol. The Encrypted Real-Time Messaging Protocol (RTMPE) is used for streaming multimedia content within standard cryptographic primitives, consisting of Diffie-Hellman key exchange and HMACSHA256, generating a pair of RC4 keys. <h3 class=”section”>18.26 rtmps# TOC</h3>

Real-Time Messaging Protocol over a secure SSL connection. The Real-Time Messaging Protocol (RTMPS) is used for streaming multimedia content across an encrypted connection. <h3 class=”section”>18.27 rtmpt# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTP. The Real-Time Messaging Protocol tunneled through HTTP (RTMPT) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class=”section”>18.28 rtmpte# TOC</h3>

Encrypted Real-Time Messaging Protocol tunneled through HTTP. The Encrypted Real-Time Messaging Protocol tunneled through HTTP (RTMPTE) is used for streaming multimedia content within HTTP requests to traverse firewalls. <h3 class=”section”>18.29 rtmpts# TOC</h3>

Real-Time Messaging Protocol tunneled through HTTPS. The Real-Time Messaging Protocol tunneled through HTTPS (RTMPTS) is used for streaming multimedia content within HTTPS requests to traverse firewalls. <h3 class=”section”>18.30 libcurl# TOC</h3>

HTTP and HTTPS access through the libcurl library, available as an alternative to the native <code class=”code”>http</code> protocol

when FFmpeg is built with <code class=”code”>--enable-libcurl</code>. It runs transfers on a dedicated event-loop thread shared per <code class=”code”>AVFormatContext</code>, so all of a demuxer’s connections (for example HLS segments) reuse libcurl’s connection cache and HTTP/2 and HTTP/3 multiplexing. The native <code class=”code”>http</code> protocol stays the default. There are two ways to use libcurl instead: <ul class=”itemize mark-bullet”> Prefix the URL with <code class=”code”>libcurl:</code> to force it explicitly, for example <code class=”code”>libcurl:https://example.com/video.mp4</code>.

Set the <samp class=”option”>prefer_libcurl</samp> option to route <code class=”code”>http://</code> and <code class=”code”>https://</code> URLs through libcurl transparently.

<dl class=”table”> <dt><samp class=”option”>prefer_libcurl</samp></dt> <dd> When set to <code class=”code”>1</code>, open <code class=”code”>http(s)</code> URLs with the libcurl protocol instead of the native one. This is a generic IO option rather than a libcurl-specific one, so it also applies on builds without libcurl (where it has no effect). Default is <code class=”code”>0</code>. </dd> </dl>

The libcurl protocol accepts the following options.

<dl class=”table”>

<dt><samp class=”option”>user_agent</samp></dt>

<dd> Override the <code class=”code”>User-Agent</code> header. Defaults to <code class=”code”>Lavf/<version></code>.

</dd>

<dt><samp class=”option”>referer</samp></dt>

<dd> Set the <code class=”code”>Referer</code> header.

</dd>

<dt><samp class=”option”>headers</samp></dt>

<dd> Set custom HTTP headers, overriding internal defaults. Multiple headers are

separated by newlines.

</dd>

<dt><samp class="option">http_proxy</samp></dt>

<dd> Set the HTTP proxy to tunnel through.

</dd>

<dt><samp class="option">cookies</samp></dt>

<dd> Cookies to be sent in future requests, given as newline-delimited

<code class="code">Set-Cookie</code>-style values.

</dd>

<dt><samp class="option">location</samp></dt>

<dd> The actual location of the data after following redirects. This is an output

option, read with <code class="command">av_opt_get</code>; demuxers use it to resolve relative URLs.

</dd>

<dt><samp class="option">offset</samp></dt>

<dd> Initial byte offset of the request. Default is <code class="code">0</code>.

</dd>

<dt><samp class="option">end_offset</samp></dt>

<dd> If greater than <code class="code">0</code>, limit the request to the bytes preceding this offset.

Together with <samp class="option">offset</samp> this requests a sub-range of the resource, as used

for HLS byte-range segments.

</dd>

<dt><samp class="option">seekable</samp></dt>

<dd> Control seekability of the connection. If set to <code class="code">0</code> the stream is treated

as non-seekable, <code class="code">1</code> forces seekable, and <code class="code">-1</code> (the default) lets it be

detected from the server response.

</dd>

<dt><samp class=”option”>tls_verify</samp></dt>

<dd> Verify the peer certificate and host name. Enabled by default.

</dd>

<dt><samp class=”option”>ca_file</samp></dt>

<dd> Path to a certificate authority bundle that overrides the system store.

</dd>

<dt><samp class=”option”>cert_file</samp></dt>

<dt><samp class=”option”>key_file</samp></dt>

<dd> Client certificate and private key files for mutual TLS.

</dd>

<dt><samp class=”option”>connect_timeout</samp></dt>

<dd> Connection timeout in seconds. <code class=”code”>0</code> (the default) uses libcurl’s own timeout.

</dd>

<dt><samp class=”option”>max_redirects</samp></dt>

<dd> Maximum number of redirects to follow. Default is <code class=”code”>16</code>.

</dd>

<dt><samp class=”option”>multiple_requests</samp></dt>

<dd> Reuse the connection across requests (HTTP keep-alive). Enabled by default;

setting it to <code class=”code”>0</code> uses a fresh connection per transfer.

</dd>

<dt><samp class="option">http_version</samp></dt>

<dd> HTTP version to use: <code class="code">auto</code> (default), <code class="code">1.0</code>, <code class="code">1.1</code>, <code class="code">2</code>,

<code class="code">2tls</code>, <code class="code">2-prior-knowledge</code>, <code class="code">3</code> or <code class="code">3only</code>.

</dd>

<dt><samp class="option">buffer_size</samp></dt>

<dd> Size in bytes of the receive buffer. Default is 4 MiB.

</dd>

<dt><samp class="option">request_size</samp></dt>

<dd> If greater than <code class="code">0</code>, split a seekable transfer into ranged requests of at

most this many bytes each. Default is <code class="code">0</code> (a single request).

</dd>

<dt><samp class="option">initial_request_size</samp></dt>

<dd> Limit the size of initial requests. Similar to <code class="code">request_size</code>, but only

used during initial format parsing. Useful for formats like MXF or MOV that

require frequent seeks during header parsing. Lasts until the demuxer makes a

read request larger than this size (without a seek in between), after which

the implementation will continue using requests as usual. Disabled (set to 0)

by default.

Note that if enabling this option, it’s strongly recommended to also set

<samp class="option">short_seek_size</samp> to the same value or higher, otherwise the HTTP

connection may be closed and recreated for each subsequent initial request.

</dd>

<dt><samp class="option">max_retries</samp></dt>

<dd> Maximum number of retries after a recoverable error on a seekable transfer.

Default is <code class="code">5</code>.

</dd>

<dt><samp class="option">short_seek_size</samp></dt>

<dd> Set the threshold, in bytes, for when a readahead should be preferred over a seek and

new HTTP request. This is useful, for example, to make sure the same connection

is used for reading large video packets with small audio packets in between.

</dd>

<dt><samp class="option">icy</samp></dt>

<dd> If set to 1 request ICY (SHOUTcast) metadata from the server. If the server

supports this, the metadata has to be retrieved by the application by reading

the <samp class="option">icy_metadata_headers</samp> and <samp class="option">icy_metadata_packet</samp> options.

The default is 1.

</dd>

<dt><samp class="option">icy_status</samp></dt>

<dd> If set to 1 accept the legacy Shoutcast v1 <code class="code">ICY 200 OK</code> status line in

place of an HTTP one. Any reply whose status line starts with <code class="code">ICY 200</code>

is then treated as a successful HTTP 200.

The default is 1.

</dd>

<dt><samp class=”option”>icy_metadata_headers</samp></dt>

<dd> If the server supports ICY metadata, this contains the ICY-specific HTTP reply

headers, separated by newline characters.

</dd>

<dt><samp class=”option”>icy_metadata_packet</samp></dt>

<dd> If the server supports ICY metadata, and <samp class=”option”>icy</samp> was set to 1, this

contains the last non-empty metadata packet sent by the server. It should be

polled in regular intervals by applications interested in mid-stream metadata

updates.

</dd>

<dt><samp class=”option”>metadata</samp></dt>

<dd> Set an exported dictionary containing Icecast metadata from the bit-stream, if present.

Only useful with the C API.

</dd>

</dl>

A stream carrying in-band ICY metadata is always treated as non-seekable, since its byte offsets do not address the payload. <samp class=”option”>seekable</samp>, <samp class=”option”>request_size</samp> and <samp class=”option”>max_retries</samp> therefore have no effect on such a stream. For more information see: https://curl.se/libcurl/. <h3 class=”section”>18.31 libsmbclient# TOC</h3>

libsmbclient permits one to manipulate CIFS/SMB network resources. Following syntax is required.   <pre class=”example-preformatted”>smb://[[domain:]user[:password@]]server[/share[/path[/file]]] </pre>

This protocol accepts the following options. <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Set timeout in milliseconds of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">workgroup</samp></dt> <dd> Set the workgroup used for making connections. By default workgroup is not specified. </dd> </dl>

For more information see: http://www.samba.org/. <h3 class="section">18.32 libssh# TOC</h3>

Secure File Transfer Protocol via libssh Read from or write to remote resources using SFTP protocol. Following syntax is required.   <pre class="example-preformatted">sftp://[user[:password]@]server[:port]/path/to/remote/resource.mpeg </pre>

This protocol accepts the following options. <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Set timeout of socket I/O operations used by the underlying low level operation. By default it is set to -1, which means that the timeout is not specified. </dd> <dt><samp class="option">truncate</samp></dt> <dd> Truncate existing files on write, if set to 1. A value of 0 prevents truncating. Default value is 1. </dd> <dt><samp class="option">private_key</samp></dt> <dd> Specify the path of the file containing private key to use during authorization. By default libssh searches for keys in the <samp class="file">~/.ssh/</samp> directory. </dd> <dt><samp class="option">verify</samp></dt> <dd> Verify the server host key against the known_hosts file before authenticating, if set to 1. Servers that are not in the file, or whose key differs from the one recorded there, are refused. Default value is 1. </dd> </dl>

Example: Play a file stored on remote server.   <pre class="example-preformatted">ffplay sftp://user:password@server_address:22/home/user/resource.mpeg </pre>

 <h3 class="section">18.33 librtmp rtmp, rtmpe, rtmps, rtmpt, rtmpte# <a class="anchor hidden-xs"href="#toc-librtmp-rtmp_002c-rtmpe_002c-rtmps_002c-rtmpt_002c-rtmpte" aria-

hidden=”true”>TOC</h3>

Real-Time Messaging Protocol and its variants supported through librtmp. Requires the presence of the librtmp headers and library during configuration. You need to explicitly configure the build with "–enable-librtmp". If enabled this will replace the native RTMP protocol. This protocol provides most client functions and a few server functions needed to support RTMP, RTMP tunneled in HTTP (RTMPT), encrypted RTMP (RTMPE), RTMP over SSL/TLS (RTMPS) and tunneled variants of these encrypted types (RTMPTE, RTMPTS). The required syntax is: <pre class=”example-preformatted”><var class=”var”>rtmp_proto</var>://<var class=”var”>server</var>[:<var class=”var”>port</var>][/<var class=”var”>app</var>][/<var class=”var”>playpath</var>] <var class=”var”>options</var> </pre>

where <var class=”var”>rtmp_proto</var> is one of the strings "rtmp", "rtmpt", "rtmpe", "rtmps", "rtmpte", "rtmpts" corresponding to each RTMP variant, and <var class=”var”>server</var>, <var class=”var”>port</var>, <var class=”var”>app</var> and <var class=”var”>playpath</var> have the same meaning as specified for the RTMP native protocol. <var class=”var”>options</var> contains a list of space-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. See the librtmp manual page (man 3 librtmp) for more information. For example, to stream a file in real-time to an RTMP server using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -re -i myfile -f flv rtmp://myserver/live/mystream </pre>

To play the same stream using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay "rtmp://myserver/live/mystream live=1" </pre>

 <h3 class=”section”>18.34 rtp# TOC</h3>

Real-time Transport Protocol. The required syntax for an RTP URL is: <pre class=”example-preformatted”>rtp://<var class=”var”>hostname</var>[:<var class=”var”>port</var>][?<var class=”var”>options</var>] </pre>

<var class=”var”>port</var> specifies the RTP port to use. <var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”>

<dt><samp class=”option”>ttl=<var class=”var”>n</var></samp></dt>
<dd> Set the TTL (Time-To-Live) value (for multicast only). </dd>
<dt><samp class=”option”>rtcpport=<var class=”var”>n</var></samp></dt>
<dd> Set the remote RTCP port to <var class=”var”>n</var>.
 </dd> <dt><samp class=”option”>localport, local_rtpport, localrtpport=<var class=”var”>n</var></samp></dt> <dd> Set the local RTP port to <var class=”var”>n</var>. Using the localport option name is deprecated and should not be used.
</dd> <dt><samp class=”option”>local_rtcpport, localrtcpport=<var class=”var”>n</var>'</samp></dt> <dd> Set the local RTCP port to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>pkt_size=<var class=”var”>n</var></samp></dt> <dd> Set max packet size (in bytes) to <var class=”var”>n</var>. </dd> <dt><samp class=”option”>buffer_size=<var class=”var”>size</var></samp></dt>
<dd> Set the maximum UDP socket buffer size in bytes. </dd>
<dt><samp class=”option”>connect=0|1</samp></dt> <dd> Do a <code class=”code”>connect()</code> on the UDP socket (if set to 1) or not (if set to 0). </dd> <dt><samp class=”option”>sources=<var class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt>
<dd> List allowed source IP addresses. </dd> <dt><samp class=”option”>block=<var class=”var”>ip</var>[,<var class=”var”>ip</var>]</samp></dt>
<dd> List disallowed (blocked) source IP addresses. </dd>
<dt><samp class=”option”>write_to_source=0|1</samp></dt> <dd> Send packets to the source address of the latest received packet (if set to 1) or to a default remote address (if set to 0). </dd> <dt><samp class=”option”>localaddr=<var class=”var”>addr</var></samp></dt>
<dd> Local IP address of a network interface used for sending packets or joining multicast groups. </dd> <dt><samp class=”option”>timeout=<var class=”var”>n</var></samp></dt> <dd> Set timeout (in microseconds) of socket I/O operations to <var class=”var”>n</var>. </dd>
</dl>

Important notes: <ol class=”enumerate”> If <samp class=”option”>rtcpport</samp> is not set the RTCP port will be set to the RTP port value plus 1.

 If <samp class=”option”>localrtpport</samp> (the local RTP port) is not set any available port will be used for the local RTP and RTCP ports.

 If <samp class=”option”>localrtcpport</samp> (the local RTCP port) is not set it will be set to the local RTP port value plus 1.

 <h3 class=”section”>18.35 rtsp#
TOC</h3>

Real-Time Streaming Protocol. RTSP is not technically

a protocol handler in libavformat, it is a demuxer and muxer. The demuxer supports both normal RTSP (with data transferred over RTP; this is used by e.g. Apple and Microsoft) and Real-RTSP (with data transferred over RDT). The muxer can be used to send a stream using RTSP ANNOUNCE to a server supporting it (currently Darwin Streaming Server and Mischa Spiegelmock’s RTSP server). The required syntax for a RTSP url is: <pre class="example-preformatted">rtsp://<var class="var">hostname</var>[:<var class="var">port</var>]/<var class="var">path</var> </pre>

Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line, or set in code via <code class="code">AVOption</code>s or in <code class="code">avformat_open_input</code>. <h4 class="subsection">18.35.1 Muxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">latm</samp>’</dt> <dd> Use MP4A-LATM packetization instead of MPEG4-GENERIC for AAC. </dd> <dt>‘<samp class="samp">rfc2190</samp>’</dt> <dd> Use RFC 2190 packetization instead of RFC 4629 for H.263. </dd> <dt>‘<samp class="samp">skip_rtcp</samp>’</dt> <dd> Don’t send RTCP sender reports. </dd> <dt>‘<samp class="samp">h264_mode0</samp>’</dt> <dd> Use mode 0 for H.264 in RTP. </dd> <dt>‘<samp class="samp">send_bye</samp>’</dt> <dd> Send RTCP BYE packets when finishing. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’.

</dd> <dt><samp class="option">min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class="option">max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class="option">buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Set max send packet size (in bytes). Default value is 1472.

</dd> </dl>

 <h4 class="subsection">18.35.2 Demuxer# TOC</h4> The following options are supported. <dl class="table"> <dt><samp class="option">initial_pause</samp></dt> <dd> Do not start playing the stream immediately if set to 1. Default value is 0. </dd> <dt><samp class="option">rtsp_transport</samp></dt> <dd> Set RTSP transport protocols. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">udp</samp>’</dt> <dd> Use UDP as lower transport protocol. </dd> <dt>‘<samp class="samp">tcp</samp>’</dt> <dd> Use TCP (interleaving within the RTSP control channel) as lower transport protocol. </dd> <dt>‘<samp class="samp">udp_multicast</samp>’</dt> <dd> Use UDP multicast as lower transport protocol. </dd> <dt>‘<samp class="samp">http</samp>’</dt> <dd> Use HTTP tunneling as lower transport protocol, which is useful for passing proxies. </dd> <dt>‘<samp class="samp">https</samp>’</dt> <dd> Use HTTPs tunneling as lower transport protocol, which is useful for passing proxies and widely used for security consideration. </dd> </dl>

Multiple lower transport protocols may be specified, in that case they are tried one at a time (if the setup of one fails, the next one is tried). For the muxer, only the ‘<samp class="samp">tcp</samp>’ and ‘<samp class="samp">udp</samp>’ options are supported. </dd> <dt><samp class="option">rtsp_flags</samp></dt> <dd> Set RTSP flags. The following values are accepted: <dl class="table"> <dt>‘<samp class="samp">filter_src</samp>’</dt> <dd> Accept packets only from negotiated peer address and port. </dd> <dt>‘<samp class="samp">listen</samp>’</dt> <dd> Act as a server, listening for an incoming connection. </dd> <dt>‘<samp class="samp">prefer_tcp</samp>’</dt> <dd> Try TCP for RTP transport first, if TCP is available as RTSP RTP transport. </dd> <dt>‘<samp class="samp">satip_raw</samp>’</dt> <dd> Export raw MPEG-TS stream instead of demuxing. The flag will simply write out the raw stream, with the original PAT/PMT/PIDs intact. </dd> </dl>

Default value is ‘<samp class="samp">none</samp>’. </dd> <dt><samp class="option">allowed_media_types</samp></dt> <dd> Set media types to accept from the server. The following flags are accepted: <dl class="table"> <dt>‘<samp class="samp">video</samp>’</dt> <dt>‘<samp class="samp">audio</samp>’</d <dt>‘<samp class="samp">data</samp>’</dt> <dt>‘<samp class="samp">subtitle</samp>’</dt> </dl>

By default it accepts all media types. </dd> <dt><samp class=”option”>min_port</samp></dt> <dd> Set minimum local UDP port. Default value is 5000. </dd> <dt><samp class=”option”>max_port</samp></dt> <dd> Set maximum local UDP port. Default value is 65000. </dd> <dt><samp class=”option”>listen_timeout</samp></dt> <dd> Set maximum timeout (in seconds) to establish an initial connection. Setting <samp class=”option”>listen_timeout</samp> > 0 sets <samp class=”option”>rtsp_flags</samp> to ‘<samp class=”samp”>listen</samp>’. Default is -1 which means an infinite timeout when ‘<samp class=”samp”>listen</samp>’ mode is set. </dd> <dt><samp class=”option”>reorder_queue_size</samp></dt> <dd> Set number of packets to buffer for handling of reordered packets. </dd> <dt><samp class=”option”>timeout</samp></dt> <dd> Set socket TCP I/O timeout in microseconds. </dd> <dt><samp class=”option”>user_agent</samp></dt> <dd> Override User-Agent header. If not specified, it defaults to the libavformat identifier string. </dd> <dt><samp class=”option”>buffer_size</samp></dt> <dd> Set the maximum socket buffer size in bytes. </dd> </dl>

When receiving data over UDP, the demuxer tries to reorder received packets (since they may arrive out of order, or packets may get lost totally). This can be disabled by setting the maximum demuxing delay to zero (via the <code class=”code”>max_delay</code> field of AVFormatContext). When watching multi-bitrate Real-RTSP streams with <code class=”command”>ffplay</code>, the streams to display can be chosen with <code class=”code”>-vst</code> <var class=”var”>n</var> and <code class=”code”>-ast</code> <var class=”var”>n</var> for video and audio respectively, and can be switched on the fly by pressing <code class=”code”>v</code> and <code class=”code”>a</code>. <h4 class=”subsection”>18.35.3 Examples# TOC</h4>

The following examples all make use of the <code class=”command”>ffplay</code> and <code class=”command”>ffmpeg</code> tools. <ul class=”itemize mark-bullet”> Watch a stream over UDP, with a max reordering delay of 0.5 seconds: <pre class=”example-preformatted”>ffplay -max_delay 500000 -rtsp_transport udp rtsp://server/video.mp4 </pre>

Watch a stream tunneled over HTTP: <pre class=”example-preformatted”>ffplay -rtsp_transport http rtsp://server/video.mp4 </pre>

Send a stream in realtime to a RTSP server, for others to watch: <pre class=”example-preformatted”>ffmpeg -re -i <var class=”var”>input</var> -f rtsp -muxdelay 0.1 rtsp://server/live.sdp

</pre>

Receive a stream in realtime: <pre class="example-preformatted">ffmpeg -rtsp_flags listen -i rtsp://ownaddress/live.sdp <var class="var">output</var> </pre>

 <h3 class="section">18.36 sap# TOC</h3>

Session Announcement Protocol (RFC 2974). This is not technically a protocol handler in libavformat, it is a muxer and demuxer. It is used for signalling of RTP streams, by announcing the SDP for the streams regularly on a separate port. <h4 class="subsection">18.36.1 Muxer# TOC</h4>

The syntax for a SAP url given to the muxer is: <pre class="example-preformatted">sap://<var class="var">destination</var>[:<var class="var">port</var>][?<var class="var">options</var>] </pre>

The RTP packets are sent to <var class="var">destination</var> on port <var class="var">port</var>, or to port 5004 if no port is specified. <var class="var">options</var> is a <code class="code">&</code>-separated list. The following options are supported: <dl class="table"> <dt><samp class="option">announce_addr=<var class="var">address</var></samp></dt> <dd> Specify the destination IP address for sending the announcements to. If omitted, the announcements are sent to the commonly used SAP announcement multicast address 224.2.127.254 (sap.mcast.net), or ff0e::2:7ffe if <var class="var">destination</var> is an IPv6 address. </dd> <dt><samp class="option">announce_port=<var class="var">port</var></samp></dt> <dd> Specify the port to send the announcements on, defaults to 9875 if not specified. </dd> <dt><samp class="option">ttl=<var class="var">ttl</var></samp></dt> <dd> Specify the time to live value for the announcements and RTP packets, defaults to 255. </dd> <dt><samp class="option">same_port=<var class="var">0|1</var></samp></dt> <dd> If set to 1, send all RTP streams on the same port pair. If zero (the default), all streams are sent on unique ports, with each stream on a port 2 numbers higher than the previous. VLC/Live555 requires this to be set to 1, to be able to receive the stream. The RTP stack in libavformat for receiving requires all streams to be sent on unique ports. </dd> </dl>

Example command lines follow. To broadcast a stream on the local subnet, for watching in VLC: <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255?same_port=1 </pre>

Similarly, for watching in <code class="command">ffplay</code>:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://224.0.0.255 </pre>

And for watching in <code class="command">ffplay</code>, over IPv6:   <pre class="example-preformatted">ffmpeg -re -i <var class="var">input</var> -f sap sap://[ff0e::1:2:3:4] </pre>

 <h4 class="subsection">18.36.2 Demuxer# TOC</h4>

The syntax for a SAP url given to the demuxer is: <pre class="example-preformatted">sap://[<var class="var">address</var>][:<var class="var">port</var>] </pre>

<var class="var">address</var> is the multicast address to listen for announcements on, if omitted, the default 224.2.127.254 (sap.mcast.net) is used. <var class="var">port</var> is the port that is listened on, 9875 if omitted. The demuxers listens for announcements on the given address and port. Once an announcement is received, it tries to receive that particular stream. Example command lines follow. To play back the first stream announced on the normal SAP multicast address: <pre class="example-preformatted">ffplay sap:// </pre>

To play back the first stream announced on one the default IPv6 SAP multicast address: <pre class="example-preformatted">ffplay sap://[ff0e::2:7ffe] </pre>

 <h3 class="section">18.37 sctp# TOC</h3>

Stream Control Transmission Protocol. The accepted URL syntax is: <pre class="example-preformatted">sctp://<var class="var">host</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">listen</samp></dt> <dd> If set to any value, listen for an incoming connection. Outgoing connection is done by default. </dd> <dt><samp class="option">max_streams</samp></dt>

<dd> Set the maximum number of streams. By default no limit is set. </dd> </dl>

 <h3 class="section">18.38 shared# TOC</h3>

Thread-safe, persistent, cross-process cache wrapper for input streams. Caches the input stream to a file in the specified directory. This is similar to cache, but with a persistent cache on disk that may be shared between multiple processes or demuxers, even concurrently. The filename of the cache is obtained by hashing the input URL, so there is some risk of collision if the same URL is used for different content (e.g. as a result of extra POST data being used to select the stream ID). To avoid this, make sure to only use this protocol for URLs that uniquely identify the content; otherwise you may get a garbled mix of sources. This includes content selected by request options such as headers, cookies or credentials: data cached for one user is returned to every other user of the same cache directory. The accepted options are: <dl class="table"> <dt><samp class="option">cache_dir</samp></dt> <dd> Path to the directory where cache files are stored. This option is required. Anyone who can write to this directory or to <samp class="option">metadata_dir</samp> can alter the data read by all users of the cache, or make them crash, so neither may be writable by untrusted users. </dd> <dt><samp class="option">metadata_dir</samp></dt> <dd> Path to the directory where metadata files are stored. Optional. If not specified, metadata is stored in the same directory as the cache file. </dd> <dt><samp class="option">block_shift</samp></dt> <dd> Shift factor (log2) of the block size used for internal reads/writes. Defaults to 15, i.e. 32KB blocks. If this does not match the value that was specified when an existing cache file was created, the previously specified value will be used instead. </dd> <dt><samp class="option">cache_read_only</samp></dt> <dd> If true, use the shared cache for reads but don’t write any new blocks to it. Default is false. Note that even with this option enabled, the cache file will be initialized if it does not already exist. </dd> <dt><samp class="option">cache_verify</samp></dt> <dd> If true, verify any data read from the cache against the underlying input stream, and error out on any mismatches. Note that this will make the cache layer effectively useless. This is purely a debug option. </dd> <dt><samp class="option">cache_timeout</samp></dt> <dd> If set to a nonzero value, specifies the maximum time (in microseconds) to wait for data to become available, if another process is currently trying to fetch and cache the same block at the same time. If this timeout elapses, it’s assumed that the other process may have gotten stuck or died in the meantime. If set to zero, no waiting is done and all processes will immediately race to try and fetch the same missing blocks themselves. Defaults to 10000 (10 ms). </dd>

<dt><samp class=”option”>ignore_errors</samp></dt> <dd> If true, failures of the underlying input stream are not treated as fatal, and playback continues using only the data already present in the cache file. Default is false. Any attempt to read into uncached data will result in an unrecoverable IO error. </dd> <dt><samp class=”option”>retry_errors</samp></dt> <dd> If true (the default), transient read errors from the underlying input stream are ignored and retried again. If false, any blocks that previously failed being read from will be treated as permanently inaccessible. </dd> <dt><samp class=”option”>retry_corrupt</samp></dt> <dd> If true (the default), blocks whose contents fail the CRC integrity check are re-fetched from the underlying input stream, overwriting the corrupt cached data. If false, cache corruption is treated as a fatal read error. </dd> <dt><samp class=”option”>cache_size_max</samp></dt> <dd> Maximum amount of data to cache, in bytes, rounded down to a multiple of the chosen block size. Accepts the usual size suffixes, e.g. <code class=”code”>10G</code> for 10*10^9 bytes, or <code class=”code”>10Gi</code> for 10*2^30 bytes. Defaults to 0, meaning no limit. Once this limit is reached, the protocol switches to read-only mode (see <samp class=”option”>cache_read_only</samp>). Already cached data is never evicted, so once this limit is reached, no new data will ever be cached until the cache file is deleted. Note that this option is best-effort only, and may over-write up to N-1 blocks, where N is the number of concurrent processes using the same cache file. Note also that the underlying file system may reserve more space than written, if the underlying allocation granularity is larger than the block size. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Initial byte offset to start reading from. This option is not passed on to the underlying protocol; it instead gets a regular seek on the first cache miss. Defaults to 0. </dd> </dl>

URL Syntax is <pre class=”example-preformatted”>shared:<var class=”var”>URL</var> </pre>

 <h3 class=”section”>18.39 srt# TOC</h3>

Haivision Secure Reliable Transport Protocol via libsrt. The supported syntax for a SRT URL is: <pre class=”example-preformatted”>srt://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form

<var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via
<code class="code">AVOption</code>s).

The list of supported options follows.

<dl class="table">
<dt><samp class="option">connect_timeout=<var class="var">milliseconds</var></samp></dt>
<dd> Connection timeout; SRT cannot connect for RTT > 1500 msec
(2 handshake exchanges) with the default connect timeout of
3 seconds. This option applies to the caller and rendezvous
connection modes. The connect timeout is 10 times the value
set for the rendezvous mode (which can be used as a
workaround for this connection problem with earlier versions).

</dd>
<dt><samp class="option">ffs=<var class="var">bytes</var></samp></dt>
<dd> Flight Flag Size (Window Size), in bytes. FFS is actually an
internal parameter and you should set it to not less than
<samp class="option">recv_buffer_size</samp> and <samp class="option">mss</samp>.
The default value
is relatively large, therefore unless you set a very large receiver buffer,
you do not need to change this option. Default value is 25600.

</dd>
<dt><samp class="option">inputbw=<var class="var">bytes/seconds</var></samp></dt>
<dd> Sender nominal input rate, in bytes per seconds. Used along with
<samp class="option">oheadbw</samp>, when <samp class="option">maxbw</samp>
is set to relative (0), to
calculate maximum sending rate when recovery packets are sent
along with the main media stream:

<samp class="option">inputbw</samp> * (100 + <samp class="option">oheadbw</samp>) / 100

if <samp class="option">inputbw</samp> is not set while <samp class="option">maxbw</samp> is set to

relative (0), the actual input rate is evaluated inside

the library. Default value is 0.

</dd>

<dt><samp class="option">iptos=<var class="var">tos</var></samp></dt>

<dd> IP Type of Service. Applies to sender only. Default value is 0xB8.

</dd>

<dt><samp class="option">ipttl=<var class="var">ttl</var></samp></dt>

<dd> IP Time To Live. Applies to sender only. Default value is 64.

</dd>

<dt><samp class="option">latency=<var class="var">microseconds</var></samp></dt>

<dd> Timestamp-based Packet Delivery Delay.

Used to absorb bursts of missed packet retransmissions.

This flag sets both <samp class="option">rcvlatency</samp> and <samp class="option">peerlatency</samp>

to the same value. Note that prior to version 1.3.0

this is the only flag to set the latency, however

this is effectively equivalent to setting <samp class="option">peerlatency</samp>,

when side is sender and <samp class="option">rcvlatency</samp>

when side is receiver, and the bidirectional stream

sending is not supported.

</dd>

<dt><samp class="option">listen_timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set socket listen timeout.

</dd>

<dt><samp class="option">maxbw=<var class="var">bytes/seconds</var></samp></dt>

<dd> Maximum sending bandwidth, in bytes per seconds.

-1 infinite (CSRTCC limit is 30mbps)

0 relative to input rate (see <samp class="option">inputbw</samp>)

>0 absolute limit value

Default value is 0 (relative)

</dd>

<dt><samp class="option">mode=<var class="var">caller|listener|rendezvous</var></samp></dt>

<dd> Connection mode.

<samp class="option">caller</samp> opens client connection.

<samp class="option">listener</samp> starts server to listen for incoming connections.

<samp class="option">rendezvous</samp> use Rendez-Vous connection mode.

Default value is caller.

</dd>

<dt><samp class="option">mss=<var class="var">bytes</var></samp></dt>

<dd> Maximum Segment Size, in bytes. Used for buffer allocation

and rate calculation using a packet counter assuming fully

filled packets. The smallest MSS between the peers is

used. This is 1500 by default in the overall internet.

This is the maximum size of the UDP packet and can be

only decreased, unless you have some unusual dedicated

network settings. Default value is 1500.

</dd>

<dt><samp class="option">nakreport=<var class="var">1|0</var></samp></dt>

<dd> If set to 1, Receiver will send ‘UMSG_LOSSREPORT‘ messages

periodically until a lost packet is retransmitted or
intentionally dropped. Default value is 1.

</dd>
<dt><samp class=”option”>oheadbw=<var class=”var”>percents</var></samp></dt>
<dd> Recovery bandwidth overhead above input rate, in percents.
See <samp class=”option”>inputbw</samp>. Default value is 25%.

</dd>
<dt><samp class=”option”>passphrase=<var class=”var”>string</var></samp></dt>
<dd> HaiCrypt Encryption/Decryption Passphrase string, length
from 10 to 79 characters. The passphrase is the shared
secret between the sender and the receiver. It is used
to generate the Key Encrypting Key using PBKDF2
(Password-Based Key Derivation Function). It is used
only if <samp class=”option”>pbkeylen</samp> is non-zero. It is used on
the receiver only if the received data is encrypted.
The configured passphrase cannot be recovered (write-only).

</dd>
<dt><samp class=”option”>enforced_encryption=<var class=”var”>1|0</var></samp></dt>
<dd> If true, both connection parties must have the same password
set (including empty, that is, with no encryption). If the
password doesn’t match or only one side is unencrypted,
the connection is rejected. Default is true.

</dd>
<dt><samp class=”option”>kmrefreshrate=<var class=”var”>packets</var></samp></dt>
<dd> The number of packets to be transmitted after which the
encryption key is switched to a new key. Default is -1.
-1 means auto (0x1000000 in srt library). The range for

this option is integers in the 0 - <code class=”code”>INT_MAX</code>.

</dd>

<dt><samp class=”option”>kmpreannounce=<var class=”var”>packets</var></samp></dt>

<dd> The interval between when a new encryption key is sent and

when switchover occurs. This value also applies to the

subsequent interval between when switchover occurs and

when the old encryption key is decommissioned. Default is -1.

-1 means auto (0x1000 in srt library). The range for

this option is integers in the 0 - <code class=”code”>INT_MAX</code>.

</dd>

<dt><samp class=”option”>snddropdelay=<var class=”var”>microseconds</var></samp></dt>

<dd> The sender’s extra delay before dropping packets. This delay
is

added to the default drop delay time interval value.

Special value -1: Do not drop packets on the sender at all.

</dd>

<dt><samp class=”option”>payload_size=<var class=”var”>bytes</var></samp></dt>

<dd> Sets the maximum declared size of a packet transferred

during the single call to the sending function in Live

mode. Use 0 if this value isn’t used (which is default in

file mode).

Default is -1 (automatic), which typically means MPEG-TS;

if you are going to use SRT

to send any different kind of payload, such as, for example,

wrapping a live stream in very small frames, then you can

use a bigger maximum frame size, though not greater than

1456 bytes.

</dd>

<dt><samp class="option">pkt_size=<var class="var">bytes</var></samp></dt>

<dd> Alias for ‘<samp class="samp">payload_size</samp>’.

</dd>

<dt><samp class="option">peerlatency=<var class="var">microseconds</var></samp></dt>

<dd> The latency value (as described in <samp class="option">rcvlatency</samp>)
that is

set by the sender side as a minimum value for the receiver.

</dd>

<dt><samp class="option">pbkeylen=<var class="var">bytes</var></samp></dt>

<dd> Sender encryption key length, in bytes.

Only can be set to 0, 16, 24 and 32.

Enable sender encryption if not 0.

Not required on receiver (set to 0),

key size obtained from sender in HaiCrypt handshake.

Default value is 0.

</dd>

<dt><samp class="option">rcvlatency=<var class="var">microseconds</var></samp></dt>

<dd> The time that should elapse since the moment when the

packet was sent and the moment when it’s delivered to

the receiver application in the receiving function.

This time should be a buffer time large enough to cover

the time spent for sending, unexpectedly extended RTT

time, and the time needed to retransmit the lost UDP

packet. The effective latency value will be the maximum

of this options’ value and the value of <samp class="option">peerlatency</samp>

set by the peer side. Before version 1.3.0 this option

is only available as <samp class="option">latency</samp>.

</dd>

<dt><samp class="option">recv_buffer_size=<var class="var">bytes</var></samp></dt>

<dd> Set UDP receive buffer size, expressed in bytes.

</dd>

<dt><samp class="option">send_buffer_size=<var class="var">bytes</var></samp></dt>

<dd> Set UDP send buffer size, expressed in bytes.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeouts for read, write and connect operations. Note
that the

SRT library has internal timeouts which can be controlled separately, the

value set here is only a cap on those.

</dd>

<dt><samp class="option">tlpktdrop=<var class="var">1|0</var></samp></dt>

<dd> Too-late Packet Drop. When enabled on receiver, it skips

missing packets that have not been delivered in time and

delivers the following packets to the application when

their time-to-play has come. It also sends a fake ACK to

the sender. When enabled on sender and enabled on the

receiving peer, the sender drops the older packets that

have no chance of being delivered in time. It was

automatically enabled in the sender if the receiver

supports it.

</dd>

<dt><samp class="option">sndbuf=<var class="var">bytes</var></samp></dt>

<dd> Set send buffer size, expressed in bytes.

</dd>
<dt><samp class=”option”>rcvbuf=<var class=”var”>bytes</var></samp></dt>
<dd> Set receive buffer size, expressed in bytes.

Receive buffer must not be greater than <samp class=”option”>ffs</samp>.

</dd>
<dt><samp class=”option”>lossmaxttl=<var class=”var”>packets</var></samp></dt>
<dd> The value up to which the Reorder Tolerance may grow. When
Reorder Tolerance is > 0, then packet loss report is delayed
until that number of packets come in. Reorder Tolerance
increases every time a "belated" packet has come, but it
wasn’t due to retransmission (that is, when UDP packets tend
to come out of order), with the difference between the latest
sequence and this packet’s sequence, and not more than the
value of this option. By default it’s 0, which means that this
mechanism is turned off, and the loss report is always sent
immediately upon experiencing a "gap" in sequences.

</dd>
<dt><samp class=”option”>minversion</samp></dt>
<dd> The minimum SRT version that is required from the peer. A connection
to a peer that does not satisfy the minimum version requirement
will be rejected.

The version format in hex is 0xXXYYZZ for x.y.z in human readable
form.

</dd>
<dt><samp class="option">streamid=<var class="var">string</var></samp></dt>
<dd> A string limited to 512 characters that can be set on the socket prior
to connecting. This stream ID will be able to be retrieved by the
listener side from the socket that is returned from srt_accept and
was connected by a socket with that set stream ID. SRT does not enforce
any special interpretation of the contents of this string.
This option doesn't make sense in Rendezvous connection; the result
might be that simply one side will override the value from the other
side and it's the matter of luck which one would win

</dd>
<dt><samp class="option">srt_streamid=<var class="var">string</var></samp></dt>
<dd> Alias for ‘<samp class="samp">streamid</samp>’
to avoid conflict with ffmpeg command line option.

</dd>
<dt><samp class="option">smoother=<var class="var">live|file</var></samp></dt>
<dd> The type of Smoother used for the transmission for that socket, which
is responsible for the transmission and congestion control. The Smoother
type must be exactly the same on both connecting parties, otherwise
the connection is rejected.

</dd>
<dt><samp class="option">messageapi=<var class="var">1|0</var></samp></dt>
<dd> When set, this socket uses the Message API, otherwise it uses Buffer
API. Note that in live mode (see <samp class="option">transtype</samp>)
there's only
message API available. In File mode you can chose to use one of two modes:

Stream API (default, when this option is false). In this mode you may

send as many data as you wish with one sending instruction, or even use
dedicated functions that read directly from a file. The internal facility
will take care of any speed and congestion control. When receiving, you
can also receive as many data as desired, the data not extracted will be
waiting for the next call. There is no boundary between data portions in
the Stream mode.

Message API. In this mode your single sending instruction passes exactly
one piece of data that has boundaries (a message). Contrary to Live mode,
this message may span across multiple UDP packets and the only size
limitation is that it shall fit as a whole in the sending buffer. The
receiver shall use as large buffer as necessary to receive the message,
otherwise the message will not be given up. When the message is not
complete (not all packets received or there was a packet loss) it will
not be given up.

</dd>
<dt><samp class="option">transtype=<var class="var">live|file</var></samp></dt>
<dd> Sets the transmission type for the socket, in particular, setting this
option sets multiple other parameters to their default values as required
for a particular transmission type.

live: Set options as for live transmission. In this mode, you should
send by one sending instruction only so many data that fit in one UDP packet,
and limited to the value defined first in <samp class="option">payload_size</samp>
(1316 is
default in this mode). There is no speed control in this mode, only the
bandwidth control, if configured, in order to not exceed the bandwidth with
the overhead transmission (retransmitted and control packets).

file: Set options as for non-live transmission. See <samp class="option">messageapi</samp>

for further explanations

</dd>

<dt><samp class="option">linger=<var class="var">seconds</var></samp></dt>

<dd> The number of seconds that the socket waits for unsent data when closing.

Default is -1. -1 means auto (off with 0 seconds in live mode, on with 180

seconds in file mode). The range for this option is integers in the

0 - <code class="code">INT_MAX</code>.

</dd>

<dt><samp class="option">tsbpd=<var class="var">1|0</var></samp></dt>

<dd> When true, use Timestamp-based Packet Delivery mode. The default behavior

depends on the transmission type: enabled in live mode, disabled in file

mode.

</dd>

<dt><samp class="option">ipv6only=<var class="var">1|0</var></samp></dt>

<dd> Accept IPv4 or not while using the IPv6 wildcard address. This should be set

when listening on a IPv6 wildcard address.

</dd>

</dl>

For more information see: <a class="url" href="https://github.com/Haivision/srt">https://github.com/H
  <h3 class="section">18.40 srtp# TOC</h3>

Secure Real-time Transport Protocol. The accepted options are:
<dl class="table"> <dt><samp class="option">srtp_in_suite</samp></dt>
<dt><samp class="option">srtp_out_suite</samp></dt> <dd> Select input and output encoding suites. Supported values: <dl

class=”table”> <dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_80</samp>’</d
<dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_80</samp>’</dt>
<dt>‘<samp class=”samp”>AES_CM_128_HMAC_SHA1_32</samp>’</dt>
<dt>‘<samp class=”samp”>SRTP_AES128_CM_HMAC_SHA1_32</samp>’</dt>
</dl>

</dd> <dt><samp class=”option”>srtp_in_params</samp></dt>
<dt><samp class=”option”>srtp_out_params</samp></dt> <dd> Set input and output encoding parameters, which are expressed by a base64-encoded representation of a binary block. The first 16 bytes of this binary block are used as master key, the following 14 bytes are used as master salt. </dd>
</dl>

 <h3 class=”section”>18.41 subfile#
TOC</h3>

Virtually extract a segment of a file or another stream. The underlying stream must be seekable. Accepted options: <dl class=”table”> <dt><samp class=”option”>start</samp></dt>
<dd> Start offset of the extracted segment, in bytes. </dd>
<dt><samp class=”option”>end</samp></dt> <dd> End offset of the extracted segment, in bytes. If set to 0, extract till end of file. </dd>
</dl>

Examples: Extract a chapter from a DVD VOB file (start and end sectors obtained externally and multiplied by 2048): <pre class=”example-preformatted”>subfile,,start,153391104,end,268142592,,:/media/dvd/
</pre>

Play an AVI file directly from a TAR archive:
<pre class=”example-preformatted”>subfile,,start,183241728,end,366490624,,:archive.tar
</pre>

Play a MPEG-TS file from start offset till end:
<pre class=”example-preformatted”>subfile,,start,32815239,end,0,,:video.ts
</pre>

 <h3 class=”section”>18.42 tee#
TOC</h3>

Writes the output to multiple protocols. The individual outputs are separated by | <pre class=”example-preformatted”>tee:file://path/to/local/this.avi|file://path/to/local/that.avi
</pre>

 <h3 class=”section”>18.43 tcp#
TOC</h3>

Transmission Control Protocol. The required syntax for a TCP url is: <pre class=”example-preformatted”>tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>listen=<var class=”var”>2|1|0</var></samp></dt> <dd> Listen for an incoming connection. 0 disables listen, 1 enables listen in single client mode, 2 enables listen in multi-client mode. Default value is 0. </dd> <dt><samp class=”option”>local_addr=<var class=”var”>addr</var></samp></dt> <dd> Local IP address of a network interface used for tcp socket connect. </dd> <dt><samp class=”option”>local_port=<var class=”var”>port</var></samp></dt> <dd> Local port used for tcp socket connect. </dd> <dt><samp class=”option”>timeout=<var class=”var”>microseconds</var></samp></dt> <dd> Set raise error timeout, expressed in microseconds. This option is only relevant in read mode: if no data arrived in more than this time interval, raise error. </dd> <dt><samp class=”option”>listen_timeout=<var class=”var”>milliseconds</var></samp></dt> <dd> Set listen timeout, expressed in milliseconds. </dd> <dt><samp class=”option”>recv_buffer_size=<var class=”var”>bytes</var></samp></dt> <dd> Set receive buffer size, expressed bytes. </dd> <dt><samp class=”option”>send_buffer_size=<var class=”var”>bytes</var></samp></dt> <dd> Set send buffer size, expressed bytes. </dd> <dt><samp class=”option”>tcp_nodelay=<var class=”var”>1|0</var></samp></dt> <dd> Set TCP_NODELAY to disable Nagle’s algorithm. Default value is 0. Remark: Writing to the socket is currently not optimized to minimize system calls and reduces the efficiency / effect of TCP_NODELAY. </dd> <dt><samp class=”option”>tcp_mss=<var class=”var”>bytes</var></samp></dt> <dd> Set maximum segment size for outgoing TCP packets, expressed in bytes. </dd> </dl>

The following example shows how to setup a listening TCP connection with <code class=”command”>ffmpeg</code>, which is then accessed with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffmpeg -i <var class=”var”>input</var> -f <var class=”var”>format</var> tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var>?listen ffplay tcp://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class="section">18.44 tls# TOC</h3>

Transport Layer Security (TLS) / Secure Sockets Layer (SSL) The required syntax for a TLS/SSL url is: <pre class="example-preformatted">tls://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form <var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class="code">AVOption</code>s). The list of supported options follows. <dl class="table"> <dt><samp class="option">ca_file, cafile=<var class="var">filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class="option">tls_verify, verify=<var class="var">1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. (With other backends, the host name is validated as well.) This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class="option">cert_file, cert=<var class="var">filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class="option">key_file, key=<var class="var">filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class="option">listen=<var class="var">1|0</var></samp></dt> <dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd> <dt><samp class="option">http_proxy</samp></dt> <dd> The HTTP proxy to tunnel through, e.g. <code class="code">http://example.com:1234</code>. The proxy must support the CONNECT method. </dd> </dl>

Example command lines: To create a TLS/SSL server that serves an input stream.   <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> tls://<var class="var">hostname</var>:<var

class=”var”>port</var>?listen&cert=<var class=”var”>server.crt</var>&key=<var class=”var”>server.key</var> </pre>

To play back a stream from the TLS/SSL server using <code class=”command”>ffplay</code>:   <pre class=”example-preformatted”>ffplay tls://<var class=”var”>hostname</var>:<var class=”var”>port</var> </pre>

 <h3 class=”section”>18.45 dtls# TOC</h3>

Datagram Transport Layer Security (DTLS) The required syntax for a DTLS URL is: <pre class=”example-preformatted”>dtls://<var class=”var”>hostname</var>:<var class=”var”>port</var>[?<var class=”var”>options</var>] </pre>

<var class=”var”>options</var> contains a list of &-separated options of the form <var class=”var”>key</var>=<var class=”var”>val</var>. Standard percent-encoding (and using the plus sign for space) can be used to escape keys and values. Options can also can be specified via command line options (or in code via <code class=”code”>AVOption</code>s). DTLS shares most options with TLS, but operates over UDP instead of TCP. The list of supported options follows. <dl class=”table”> <dt><samp class=”option”>ca_file, cafile=<var class=”var”>filename</var></samp></dt> <dd> A file containing certificate authority (CA) root certificates to treat as trusted. If the linked TLS library contains a default this might not need to be specified for verification to work, but not all libraries and setups have defaults built in. The file must be in OpenSSL PEM format. </dd> <dt><samp class=”option”>tls_verify, verify=<var class=”var”>1|0</var></samp></dt> <dd> If enabled, try to verify the peer that we are communicating with. Note, if using OpenSSL, this currently only makes sure that the peer certificate is signed by one of the root certificates in the CA database, but it does not validate that the certificate actually matches the host name we are trying to connect to. This is enabled by default. Verifying the peer requires a CA database, which in some cases has to be provided by the caller. </dd> <dt><samp class=”option”>cert_file, cert=<var class=”var”>filename</var></samp></dt> <dd> A file containing a certificate to use in the handshake with the peer. (When operating as server, in listen mode, this is more often required by the peer, while client certificates only are mandated in certain setups.) </dd> <dt><samp class=”option”>key_file, key=<var class=”var”>filename</var></samp></dt> <dd> A file containing the private key for the certificate. </dd> <dt><samp class=”option”>cert_pem=<var class=”var”>string</var></samp></dt> <dd> Certificate PEM string </dd> <dt><samp class=”option”>key_pem=<var class=”var”>string</var></samp></dt> <dd> Private key PEM string

 </dd> <dt><samp class="option">listen=<var class="var">1|0</var></samp></dt>
<dd> If enabled, listen for connections on the provided port, and assume the server role in the handshake instead of the client role. </dd>
<dt><samp class="option">mtu=<var class="var">size</var></samp></dt>
<dd> Set the Maximum Transmission Unit (MTU) for DTLS packets.
 </dd> <dt><samp class="option">use_srtp=<var class="var">1|0</var></samp></dt>
<dd> Enable the use_srtp DTLS extension. This is used in WebRTC applications to establish SRTP encryption keys through the DTLS handshake. Default is disabled. </dd> <dt><samp class="option">external_sock=<var class="var">1|0</var></samp></dt>
<dd> Use an external socket instead of creating a new one. This option only makes sense to pass when interacting with the code via API, enabling this from CLI will cause immediate failure. Default is disabled. </dd>
</dl>

Example command lines: To create a DTLS server:
 <pre class="example-preformatted">ffmpeg -listen 1 -i dtls://<var class="var">hostname</var>:<var class="var">port</var> <var class="var">output</var> </pre>

To create a DTLS client and send data to server: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> dtls://<var class="var">hostname</var>:<var class="var">port</var> </pre>

 <h3 class="section">18.46 udp# TOC</h3>

User Datagram Protocol. The required syntax for an UDP URL is: <pre class="example-preformatted">udp://<var class="var">hostname</var>:<var class="var">port</var>[?<var class="var">options</var>] </pre>

<var class="var">options</var> contains a list of &-separated options of the form

<var class="var">key</var>=<var class="var">val</var>. Standard percent-encoding (and using the plus sign for

space) can be used to escape keys and values.

Options can also can be specified via command line options (or in code via

<code class="code">AVOption</code>s).

In case threading is enabled on the system, a circular buffer is used

to store the incoming data, which allows one to reduce loss of data due to

UDP socket buffer overruns. The <var class=”var”>fifo_size</var> and

<var class=”var”>overrun_nonfatal</var> options are related to this buffer.

The list of supported options follows.

<dl class=”table”>

<dt><samp class=”option”>buffer_size=<var class=”var”>size</var></samp></dt>

<dd> Set the UDP maximum socket buffer size in bytes. This is used to
set either

the receive or send buffer size, depending on what the socket is used for.

Default is 32 KB for output, 384 KB for input. See also <var class=”var”>fifo_size</var>.

</dd>

<dt><samp class=”option”>bitrate=<var class=”var”>bitrate</var></samp></dt>

<dd> If set to nonzero, the output will have the specified constant bitrate
if the

input has enough packets to sustain it.

</dd>

<dt><samp class=”option”>burst_bits=<var class=”var”>bits</var></samp></dt>

<dd> When using <var class=”var”>bitrate</var> this specifies the max-
imum number of bits in

packet bursts.

</dd>

<dt><samp class=”option”>localport=<var class=”var”>port</var></samp></dt>

<dd> Override the local UDP port to bind with.

</dd>

<dt><samp class=”option”>localaddr=<var class=”var”>addr</var></samp></dt>

<dd> Local IP address of a network interface used for sending packets or joining

multicast groups.

</dd>

<dt><samp class=”option”>pkt_size=<var class=”var”>size</var></samp></dt>

<dd> Set the size in bytes of UDP packets.

</dd>

<dt><samp class=”option”>reuse=<var class=”var”>1|0</var></samp></dt>

<dd> Explicitly allow or disallow reusing UDP sockets.

</dd>

<dt><samp class=”option”>ttl=<var class=”var”>ttl</var></samp></dt>

<dd> Set the time to live value (for multicast only).

</dd>

<dt><samp class=”option”>dscp=<var class=”var”>dscp</var></samp></dt>

<dd> Set the 6-bit DSCP field for outgoing packets.

</dd>

<dt><samp class=”option”>connect=<var class=”var”>1|0</var></samp></dt>

<dd> Initialize the UDP socket with <code class=”code”>connect()</code>. In this case, the

destination address can’t be changed with ff_udp_set_remote_url later.

If the destination address isn’t known at the start, this option can

be specified in ff_udp_set_remote_url, too.

This allows finding out the source address for the packets with getsockname,

and makes writes return with AVERROR(ECONNREFUSED) if "destination

unreachable" is received.

For receiving, this gives the benefit of only receiving packets from

the specified peer address/port.

</dd>

<dt><samp class="option">sources=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Only receive packets sent from the specified addresses. In case of multicast,

also subscribe to multicast traffic coming from these addresses only.

</dd>

<dt><samp class="option">block=<var class="var">address</var>[,<var class="var">address</var>]</samp></dt>

<dd> Ignore packets sent from the specified addresses. In case of multicast, also

exclude the source addresses in the multicast subscription.

</dd>

<dt><samp class="option">fifo_size=<var class="var">units</var></samp></dt>

<dd> Set the UDP receiving circular buffer size, expressed as a number of

packets with size of 188 bytes. If not specified defaults to 7*4096.

Set to 0 to disable the receiving circular buffer and its receive thread.

</dd>

<dt><samp class="option">overrun_nonfatal=<var class="var">1|0</var></samp></dt>

<dd> Survive in case of UDP receiving circular buffer overrun. Default

value is 0.

</dd>

<dt><samp class="option">timeout=<var class="var">microseconds</var></samp></dt>

<dd> Set raise error timeout, expressed in microseconds.

This option is only relevant in read mode: if no data arrived in more

than this time interval, raise error.

</dd>

<dt><samp class="option">broadcast=<var class="var">1|0</var></samp></dt>

<dd> Explicitly allow or disallow UDP broadcasting.

Note that broadcasting may not work properly on networks having

a broadcast storm protection.

</dd>

</dl>

 <h4 class="subsection">18.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> to stream over UDP to a remote endpoint: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f <var class="var">format</var> udp://<var class="var">hostname</var>:<var class="var">port</var> </pre>

Use <code class="command">ffmpeg</code> to stream in mpegts format over UDP using 188 sized UDP packets, using a large input buffer: <pre class="example-preformatted">ffmpeg -i <var class="var">input</var> -f mpegts udp://<var class="var">hostname</var>:<var class="var">port</var>?pkt_size=188&buffer_size=65535 </pre>

Use <code class="command">ffmpeg</code> to receive over UDP from a remote endpoint: <pre class="example-preformatted">ffmpeg -i udp://[<var class="var">multicast-address</var>]:<var class="var">port</var> ... </pre>

 <h3 class="section">18.47 unix# TOC</h3>

Unix local socket The required syntax for a Unix socket URL is: <pre class="example-preformatted">unix://<var class="var">filepath</var> </pre>

The following parameters can be set via command line options (or in code via <code class="code">AVOption</code>s): <dl class="table"> <dt><samp class="option">timeout</samp></dt> <dd> Timeout in ms. </dd> <dt><samp class="option">listen</samp></dt>

<dd> Create the Unix socket in listening mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Select socket type. <dl class="table"> <dt>‘<samp class="samp">stream Corresponds to SOCK_STREAM (reliable stream-oriented)</samp>’</dt> <dt>‘<samp class="samp">datagram Corresponds to SOCK_DGRAM (unreliable packet-oriented)</samp>’</dt> <dt>‘<samp class="samp">seqpacket Corresponds to SOCK_SEQPACKET (reliable packet-oriented)</samp>’</dt> </dl> </dd> <dt><samp class="option">pkt_size</samp></dt> <dd> Maximum packet size for packet-oriented sockets (SOCK_DGRAM and SOCK_SEQPACKET). If greater than zero, this value is used as <code class="code">max_packet_size</code>. Ignored for SOCK_STREAM. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">18.48 zmq# TOC</h3>

ZeroMQ asynchronous messaging using the libzmq library. This library supports unicast streaming to multiple clients without relying on an external server. The required syntax for streaming or connecting to a stream is: <pre class="example-preformatted">zmq:tcp://ip-address:port </pre>

Example: Create a localhost stream on port 5555: <pre class="example-preformatted">ffmpeg -re -i input -f mpegts zmq:tcp://127.0.0.1:5555 </pre>

Multiple clients may connect to the stream using: <pre class="example-preformatted">ffplay zmq:tcp://127.0.0.1:5555 </pre>

Streaming to multiple clients is implemented using a ZeroMQ Pub-Sub pattern. The server side binds to a port and publishes data. Clients connect to the server (via IP address/port) and subscribe to the stream. The order in which the server and client start generally does not matter. ffmpeg must be compiled with the –enable-libzmq option to support this protocol. Options can be set on the <code class="command">ffmpeg</code>/<code class="command">ffplay</code> command line. The following options are supported: <dl class="table"> <dt><samp class="option">pkt_size</samp></dt> <dd> Forces the maximum packet size for sending/receiving data. The default value is 131,072 bytes. On the server side, this sets the maximum size of sent packets via ZeroMQ. On the clients, it sets an internal buffer size for receiving packets. Note that pkt_size on the clients should be equal to or greater than pkt_size on the server. Otherwise the received message may be truncated causing decoding errors. </dd> </dl>

 <h2 class="chapter">19 Device Op-

tions# TOC</h2>

The libavdevice library provides the same interface as libavformat. Namely, an input device is considered like a demuxer, and an output device like a muxer, and the interface and generic device options are the same provided by libavformat (see the ffmpeg-formats manual). In addition each input or output device may support so-called private options, which are specific for that component. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, or by setting the value explicitly in the device <code class=”code”>AVFormatContext</code> options or using the <samp class=”file”>libavutil/opt.h</samp> API for programmatic use.

 <h2 class=”chapter”>20 Input Devices# TOC</h2>

Input devices are configured elements in FFmpeg which enable accessing the data coming from a multimedia device attached to your system. When you configure your FFmpeg build, all the supported input devices are enabled by default. You can list all available ones using the configure option "–list-indevs". You can disable all the input devices using the configure option "–disable-indevs", and selectively enable an input device using the option "–enable-indev=<var class=”var”>INDEV</var>", or you can disable a particular input device using the option "–disable-indev=<var class=”var”>INDEV</var>". The option "-devices" of the ff* tools will display the list of supported input devices. A description of the currently available input devices follows. <h3 class=”section”>20.1 alsa# TOC</h3>

ALSA (Advanced Linux Sound Architecture) input device. To enable this input device during configuration you need libasound installed on your system. This device allows capturing from an ALSA device. The name of the device to capture has to be an ALSA card identifier. An ALSA identifier has the syntax: <pre class=”example-preformatted”>hw:<var class=”var”>CARD</var>[,<var class=”var”>DEV</var>[,<var class=”var”>SUBDEV</var>]] </pre>

where the <var class=”var”>DEV</var> and <var class=”var”>SUBDEV</var> components are optional. The three arguments (in order: <var class=”var”>CARD</var>,<var class=”var”>DEV</var>,<var

class=”var”>SUBDEV</var>) specify card number or identifier, device number and subdevice number (-1 means any). To see the list of cards currently recognized by your system check the files <samp class=”file”>/proc/asound/cards</samp> and <samp class=”file”>/proc/asound/devices</samp>. For example to capture with <code class=”command”>ffmpeg</code> from an ALSA device with card id 0, you may run the command: <pre class=”example-preformatted”>ffmpeg -f alsa -i hw:0 alsaout.wav </pre>

For more information see: http://www.alsa-project.org/alsa-doc/alsa-lib/pcm.html <h4 class=”subsection”>20.1.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class=”option”>ch_layout</samp></dt> <dd> Set the channel layout. When unset, the layout defaults to stereo, or to the minimum channel count supported by the device, and is filled in from the ALSA channel map on a best-effort basis. </dd> </dl>

 <h3 class=”section”>20.2 android_camera# TOC</h3>

Android camera input device. This input devices uses the Android Camera2 NDK API which is available on devices with API level 24+. The availability of android_camera is autodetected during configuration. This device allows capturing from all cameras on an Android device, which are integrated into the Camera2 NDK API. The available cameras are enumerated internally and can be selected with the <var class=”var”>camera_index</var> parameter. The input file string is discarded. Generally the back facing camera has index 0 while the front facing camera has index 1. <h4 class=”subsection”>20.2.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size given as a string such as 640x480 or hd720. Falls back to the first available configuration reported by Android if requested video size is not available or by default. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the video framerate. Falls back to the first available configuration reported by Android

if requested framerate is not available or by default (-1). </dd> <dt><samp class="option">camera_index</samp></dt> <dd> Set the index of the camera to use. Default is 0. </dd> <dt><samp class="option">input_queue_size</samp></dt> <dd> Set the maximum number of frames to buffer. Default is 5. </dd> </dl>

 <h3 class="section">20.3 avfoundation# TOC</h3>

AVFoundation input device. AVFoundation is the currently recommended framework by Apple for streamgrabbing on OSX >= 10.7 as well as on iOS. The input filename has to be given in the following syntax: <pre class="example-preformatted">-i "[[VIDEO]:[AUDIO]]" </pre> The first entry selects the video input while the latter selects the audio input. The stream has to be specified by the device name or the device index as shown by the device list. Alternatively, the video and/or audio input device can be chosen by index using the <samp class="option"> -video_device_index <INDEX> </samp> and/or <samp class="option"> -audio_device_index <INDEX> </samp> , overriding any device name or index given in the input filename. All available devices can be enumerated by using <samp class="option">-list_devices true</samp>, listing all device names and corresponding indices. There are two device name aliases: <dl class="table"> <dt><code class="code">default</code></dt> <dd> Select the AVFoundation default device of the corresponding type. </dd> <dt><code class="code">none</code></dt> <dd> Do not record the corresponding media type. This is equivalent to specifying an empty device name or index. </dd> </dl>

 <h4 class="subsection">20.3.1 Options# TOC</h4>

AVFoundation supports the following options: <dl class="table"> <dt><samp class="option">-list_devices <TRUE|FALSE></samp></dt> <dd> If set to true, a list of all available input devices is given showing their index, name, unique ID, and USB serial number (when available). </dd> <dt><samp class="option">-video_device_index <INDEX></samp></dt> <dd> Specify the video device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-audio_device_index <INDEX></samp></dt> <dd> Specify the audio device by its index. Overrides anything given in the input filename. </dd> <dt><samp class="option">-video_device_id <ID></samp></dt> <dd> Specify the video device by a prefixed identifier, as shown by <samp class="option">-list_devices</samp>:

<code class=”code”>uid:<UID></code> selects by AVFoundation unique ID, <code class=”code”>serial:<SERIAL></code> by USB serial number. The unique ID of a UVC device may not be stable across reboots. Only UVC devices expose a serial number. Overrides anything given in the input filename. </dd> <dt><samp class=”option”>-audio_device_id <ID></samp></dt> <dd> Specify the audio device by a prefixed identifier. See <samp class=”option”>-video_device_id</samp>; for audio, the serial is only available for class-compliant USB devices. </dd> <dt><samp class=”option”>-pixel_format <FORMAT></samp></dt> <dd> Request the video device to use a specific pixel format. If the specified format is not supported, a list of available formats is given and the first one in this list is used instead. Available pixel formats are: <code class=”code”>monob, rgb555be, rgb555le, rgb565be, rgb565le, rgb24, bgr24, 0rgb, bgr0, 0bgr, rgb0, bgr48be, uyvy422, yuva444p, yuva444p16le, yuv444p, yuv422p16, yuv422p10, yuv444p10, yuv420p, nv12, yuyv422, gray</code> </dd> <dt><samp class=”option”>-framerate</samp></dt> <dd> Set the grabbing frame rate. Default is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>-video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class=”option”>-capture_cursor</samp></dt> <dd> Capture the mouse pointer. Default is 0. </dd> <dt><samp class=”option”>-capture_mouse_clicks</samp></dt> <dd> Capture the screen mouse clicks. Default is 0. </dd> <dt><samp class=”option”>-capture_raw_data</samp></dt> <dd> Capture the raw device data. Default is 0. Using this option may result in receiving the underlying data delivered to the AVFoundation framework. E.g. for muxed devices that sends raw DV data to the framework (like tape-based camcorders), setting this option to false results in extracted video frames captured in the designated pixel format only. Setting this option to true results in receiving the raw DV stream untouched. </dd> </dl>

 <h4 class=”subsection”>20.3.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print the list of AVFoundation supported devices and exit: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -list_devices true -i "" </pre>

Record video from video device 0 and audio from audio device 0 into out.avi: <pre class=”example-preformatted”>$ ffmpeg -f avfoundation -i "0:0" out.avi </pre>

Record video from video device 2 and audio from audio device 1 into out.avi: <pre class=”example-preformatted”>$

ffmpeg -f avfoundation -video_device_index 2 -i ":1" out.avi </pre>

Record video from the system default video device using the pixel format bgr0 and do not record any audio into out.avi: <pre class="example-preformatted">$ ffmpeg -f avfoundation -pixel_format bgr0 -i "default:none" out.avi </pre>

Record raw DV data from a suitable input device and write the output into out.dv: <pre class="example-preformatted">$ ffmpeg -f avfoundation -capture_raw_data true -i "zr100:none" out.dv </pre>

 <h3 class="section">20.4 decklink# TOC</h3>

The decklink input device provides capture capabilities for Blackmagic DeckLink devices. To enable this input device, you need the Blackmagic DeckLink SDK and you need to configure with the appropriate <code class="code">--extra-cflags</code>, <code class="code">--extra-cxxflags</code> and <code class="code">--extra-ldflags</code>. The SDK include directory must be available to both the C and C++ compilers. On Windows, you need to run the IDL files through <code class="command">widl</code>. DeckLink is very picky about the formats it supports. Pixel format of the input can be set with <samp class="option">raw_format</samp>. Framerate and video size must be determined for your device with <code class="command">-list_formats 1</code>. Audio sample rate is always 48 kHz and the number of channels can be 2, 8 or 16. Note that all audio channels are bundled in one single audio track. <h4 class="subsection">20.4.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. This option is deprecated, please use the <code class="code">-sources</code> option of ffmpeg to list the available input devices. </dd> <dt><samp class="option">list_formats</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of supported formats and exit. Defaults to <samp class="option">false</samp>. </dd> <dt><samp class="option">format_code <FourCC></samp></dt> <dd> This sets the input video format to the format given by the FourCC. To see the supported values of your device(s) use <samp

class=”option”>list_formats</samp>. Note that there is a FourCC <samp class=”option”>'pal '</samp> that can also be used as <samp class=”option”>pal</samp> (3 letters). Default behavior is autodetection of the input video format, if the hardware supports it. </dd> <dt><samp class=”option”>raw_format</samp></dt> <dd> Set the pixel format of the captured video. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> This is the default which means 8-bit YUV 422 or 8-bit ARGB if format autodetection is used, 8-bit YUV 422 otherwise. </dd> <dt>‘<samp class=”samp”>uyvy422</samp>’</dt> <dd> 8-bit YUV 422. </dd> <dt>‘<samp class=”samp”>yuv422p10</samp>’</dt> <dd> 10-bit YUV 422. </dd> <dt>‘<samp class=”samp”>argb</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8-bit RGB. </dd> <dt>‘<samp class=”samp”>rgb10</samp>’</dt> <dd> 10-bit RGB. </dd> </dl>

</dd> <dt><samp class=”option”>teletext_lines</samp></dt> <dd> If set to nonzero, an additional teletext stream will be captured from the vertical ancillary data. Both SD PAL (576i) and HD (1080i or 1080p) sources are supported. In case of HD sources, OP47 packets are decoded. This option is a bitmask of the SD PAL VBI lines captured, specifically lines 6 to 22, and lines 318 to 335. Line 6 is the LSB in the mask. Selected lines which do not contain teletext information will be ignored. You can use the special <samp class=”option”>all</samp> constant to select all possible lines, or <samp class=”option”>standard</samp> to skip lines 6, 318 and 319, which are not compatible with all receivers. For SD sources, ffmpeg needs to be compiled with <code class=”code”>--enable-libzvbi</code>. For HD sources, on older (pre-4K) DeckLink card models you have to capture in 10 bit mode. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Defines number of audio channels to capture. Must be ‘<samp class=”samp”>2</samp>’, ‘<samp class=”samp”>8</samp>’ or ‘<samp class=”samp”>16</samp>’. Defaults to ‘<samp class=”samp”>2</samp>’. </dd> <dt><samp class=”option”>duplex_mode</samp></dt> <dd> Sets the decklink device duplex/profile mode. Must be ‘<samp class=”samp”>unset</samp>’, ‘<samp class=”samp”>half</samp>’, ‘<samp class=”samp”>full</samp>’, ‘<samp class=”samp”>one_sub_device_full</sa ‘<samp class=”samp”>one_sub_device_half</samp>’, ‘<samp class=”samp”>two_sub_device_full</samp>’, ‘<samp class=”samp”>four_sub_device_half</samp>’ Defaults to ‘<samp class=”samp”>unset</samp>’. Note: DeckLink SDK 11.0 have replaced the duplex property by a profile property. For the DeckLink Duo 2 and DeckLink Quad 2, a profile is shared between any 2 sub-devices that utilize the same connectors. For the DeckLink 8K Pro, a profile is shared between all 4 sub-devices. So DeckLink 8K Pro support four profiles. Valid profile modes for DeckLink 8K Pro(with DeckLink SDK >= 11.0):

‘<samp class=”samp”>one_sub_device_full</samp>’, ‘<samp class=”samp”>one_sub_device_half</samp>’, ‘<samp class=”samp”>two_sub_device_full</samp>’, ‘<samp class=”samp”>four_sub_device_half</samp>’ Valid profile modes for DeckLink Quad 2 and DeckLink Duo 2: ‘<samp class=”samp”>half</samp>’, ‘<samp class=”samp”>full</samp>’ </dd> <dt><samp class=”option”>timecode_format</samp>< <dd> Timecode type to include in the frame and video stream metadata. Must be ‘<samp class=”samp”>none</samp>’, ‘<samp class=”samp”>rp188vitc</samp>’, ‘<samp class=”samp”>rp188vitc2</samp>’, ‘<samp class=”samp”>rp188ltc</samp>’, ‘<samp class=”samp”>rp188hfr</samp>’, ‘<samp class=”samp”>rp188any</samp>’, ‘<samp class=”samp”>vitc</samp>’, ‘<samp class=”samp”>vitc2</samp>’, or ‘<samp class=”samp”>serial</samp>’. Defaults to ‘<samp class=”samp”>none</samp>’ (not included). In order to properly support 50/60 fps timecodes, the ordering of the queried timecode types for ‘<samp class=”samp”>rp188any</samp>’ is HFR, VITC1, VITC2 and LTC for >30 fps content. Note that this is slightly different to the ordering used by the DeckLink API, which is HFR, VITC1, LTC, VITC2. </dd> <dt><samp class=”option”>video_input</samp></dt> <dd> Sets the video input source. Must be ‘<samp class=”samp”>unset</samp>’, ‘<samp class=”samp”>sdi</samp>’, ‘<samp class=”samp”>hdmi</samp>’, ‘<samp class=”samp”>optical_sdi</samp>’, ‘<samp class=”samp”>component</samp>’, ‘<samp class=”samp”>composite</samp>’ or ‘<samp class=”samp”>s_video</samp>’. Defaults to ‘<samp class=”samp”>unset</samp>’. </dd> <dt><samp class=”option”>audio_input</samp></dt> <dd> Sets the audio input source. Must be ‘<samp class=”samp”>unset</samp>’, ‘<samp class=”samp”>embedded</samp>’, ‘<samp class=”samp”>aes_ebu</samp>’, ‘<samp class=”samp”>analog</samp>’, ‘<samp class=”samp”>analog_xlr</samp>’, ‘<samp class=”samp”>analog_rca</samp>’ or ‘<samp class=”samp”>microphone</samp>’. Defaults to ‘<samp class=”samp”>unset</samp>’. </dd> <dt><samp class=”option”>video_pts</samp></dt> <dd> Sets the video packet timestamp source. Must be ‘<samp class=”samp”>video</samp>’, ‘<samp class=”samp”>audio</samp>’, ‘<samp class=”samp”>reference</samp>’, ‘<samp class=”samp”>wallclock</samp>’ or ‘<samp class=”samp”>abs_wallclock</samp>’. Defaults to ‘<samp class=”samp”>video</samp>’. </dd> <dt><samp class=”option”>audio_pts</samp></dt> <dd> Sets the audio packet timestamp source. Must be ‘<samp class=”samp”>video</samp>’, ‘<samp class=”samp”>audio</samp>’, ‘<samp class=”samp”>reference</samp>’, ‘<samp class=”samp”>wallclock</samp>’ or ‘<samp class=”samp”>abs_wallclock</samp>’. Defaults to ‘<samp class=”samp”>audio</samp>’. </dd>

<dt><samp class=”option”>draw_bars</samp></dt> <dd> If set to ‘<samp class=”samp”>true</samp>’, color bars are drawn in the event of a signal loss. Defaults to ‘<samp class=”samp”>true</samp>’. This option is deprecated, please use the <code class=”code”>signal_loss_action</code> option. </dd> <dt><samp class=”option”>signal_loss_action</samp></dt> <dd> Sets the action to take in the event of a signal loss. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>1, none</samp></dt> <dd> Do nothing on signal loss. This usually results in black frames. </dd> <dt><samp class=”option”>2, bars</samp></dt> <dd> Draw color bars on signal loss. Only supported for 8-bit input signals. </dd> <dt><samp class=”option”>3, repeat</samp></dt> <dd> Repeat the last video frame on signal loss. </dd> </dl>

Defaults to ‘<samp class=”samp”>bars</samp>’. </dd> <dt><samp class=”option”>queue_size</samp></dt> <dd> Sets maximum input buffer size in bytes. If the buffering reaches this value, incoming frames will be dropped. Defaults to ‘<samp class=”samp”>1073741824</samp>’. </dd> <dt><samp class=”option”>audio_depth</samp></dt> <dd> Sets the audio sample bit depth. Must be ‘<samp class=”samp”>16</samp>’ or ‘<samp class=”samp”>32</samp>’. Defaults to ‘<samp class=”samp”>16</samp>’. </dd> <dt><samp class=”option”>decklink_copyts</samp></
<dd> If set to <samp class=”option”>true</samp>, timestamps are forwarded as they are without removing the initial offset. Defaults to <samp class=”option”>false</samp>. </dd> <dt><samp class=”option”>timestamp_align</samp></dt> <dd> Capture start time alignment in seconds. If set to nonzero, input frames are dropped till the system timestamp aligns with configured value. Alignment difference of up to one frame duration is tolerated. This is useful for maintaining input synchronization across N different hardware devices deployed for ’N-way’ redundancy. The system time of different hardware devices should be synchronized with protocols such as NTP or PTP, before using this option. Note that this method is not foolproof. In some border cases input synchronization may not happen due to thread scheduling jitters in the OS. Either sync could go wrong by 1 frame or in a rarer case <samp class=”option”>timestamp_align</samp> seconds. Defaults to ‘<samp class=”samp”>0</samp>’. </dd> <dt><samp class=”option”>wait_for_tc (bool)</samp></dt>
<dd> Drop frames till a frame with timecode is received. Sometimes serial timecode isn’t received with the first input frame. If that happens, the stored stream timecode will be inaccurate. If this option is set to <samp class=”option”>true</samp>, input frames are dropped till a frame with timecode is received. Option <var class=”var”>timecode_format</var> must be specified. Defaults to <samp class=”option”>false</samp>. </dd>

<dt><samp class="option">enable_klv(bool)</samp></dt>
<dd> If set to <samp class="option">true</samp>, extracts KLV data from VANC and outputs KLV packets. KLV VANC packets are joined based on MID and PSC fields and aggregated into one KLV packet. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">20.4.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List input devices: <pre class="example-preformatted">ffmpeg -sources decklink </pre>

List supported formats: <pre class="example-preformatted">ffmpeg -f decklink -list_formats 1 -i 'Intensity Pro' </pre>

Capture video clip at 1080i50: <pre class="example-preformatted">ffmpeg -format_code Hi50 -f decklink -i 'Intensity Pro' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 10 bit: <pre class="example-preformatted">ffmpeg -raw_format yuv422p10 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

Capture video clip at 1080i50 with 16 audio channels: <pre class="example-preformatted">ffmpeg -channels 16 -format_code Hi50 -f decklink -i 'UltraStudio Mini Recorder' -c:a copy -c:v copy output.avi </pre>

 <h3 class="section">20.5 dshow# TOC</h3>

Windows DirectShow input device. DirectShow support is enabled when FFmpeg is built with the mingw-w64 project. Currently only audio and video devices are supported. Multiple devices may be opened as separate inputs, but they may also be opened on the same input, which should improve synchronism between them. The input name should be in the format: <pre class="example-preformatted"><var class="var">TYPE</var>=<var class="var">NAME</var>[:<var class="var">TYPE</var>=<var class="var">NAME</var>] </pre>

where <var class="var">TYPE</var> can be either <var class="var">audio</var> or <var class="var">video</var>, and <var class="var">NAME</var> is the device’s name or alternative name.. <h4 class="subsection">20.5.1 Options<span class="pull-

right”>#
TOC</h4>

If no options are specified, the device’s defaults are used.

If the device does not support the requested options, it will

fail to open.

<dl class=”table”>

<dt><samp class=”option”>video_size</samp></dt>

<dd> Set the video size in the captured video.

</dd>

<dt><samp class=”option”>framerate</samp></dt>

<dd> Set the frame rate in the captured video.

</dd>

<dt><samp class=”option”>sample_rate</samp></dt>

<dd> Set the sample rate (in Hz) of the captured audio.

</dd>

<dt><samp class=”option”>sample_size</samp></dt>

<dd> Set the sample size (in bits) of the captured audio.

</dd>

<dt><samp class=”option”>channels</samp></dt>

<dd> Set the number of channels in the captured audio.

</dd>

<dt><samp class=”option”>list_devices</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of devices
and exit.

</dd>

<dt><samp class=”option”>list_options</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, print a list of selected device’s options

and exit.

</dd>

<dt><samp class=”option”>video_device_number</samp></dt>

<dd> Set video device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class=”option”>audio_device_number</samp></dt>

<dd> Set audio device number for devices with the same name (starts at 0,

defaults to 0).

</dd>

<dt><samp class=”option”>pixel_format</samp></dt>

<dd> Select pixel format to be used by DirectShow. This may only be set when

the video codec is not set or set to rawvideo.

</dd>

<dt><samp class=”option”>audio_buffer_size</samp></dt>

<dd> Set audio device buffer size in milliseconds (which can directly

impact latency, depending on the device).

Defaults to using the audio device’s

default buffer size (typically some multiple of 500ms).

Setting this value too low can degrade performance.

See also

us/library/windows/desktop/dd377582(v=vs.85).aspx

</dd>

<dt><samp class=”option”>video_pin_name</samp></dt>

<dd> Select video capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>audio_pin_name</samp></dt>

<dd> Select audio capture pin to use by name or alternative name.

</dd>

<dt><samp class=”option”>crossbar_video_input_pin_number</samp></dt>

<dd> Select video input pin number for crossbar device. This will be

routed to the crossbar device’s Video Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>crossbar_audio_input_pin_number</samp></dt>

<dd> Select audio input pin number for crossbar device. This will be

routed to the crossbar device’s Audio Decoder output pin.

Note that changing this value can affect future invocations

(sets a new default) until system reboot occurs.

</dd>

<dt><samp class=”option”>show_video_device_dialog</samp></dt>

<dd> If set to <samp class=”option”>true</samp>, before capture starts,
popup a display dialog

to the end user, allowing them to change video filter properties

and configurations manually.

Note that for crossbar devices, adjusting values in this dialog

may be needed at times to toggle between PAL (25 fps) and NTSC (29.97)

input frame rates, sizes, interlacing, etc. Changing these values can

enable different scan rates/frame rates and avoiding green bars at

the bottom, flickering scan lines, etc.

Note that with some devices, changing these properties can also affect future

invocations (sets new defaults) until system reboot occurs.

</dd>

<dt><samp class="option">show_audio_device_dialog</samp></dt>

<dd> If set to <samp class="option">true</samp>, before capture starts, popup a display dialog

to the end user, allowing them to change audio filter properties

and configurations manually.

</dd>

<dt><samp class="option">show_video_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class="option">true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens a video device.

</dd>

<dt><samp class="option">show_audio_crossbar_connection_dialog</samp></dt>

<dd> If set to <samp class="option">true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify crossbar pin routings, when it opens an audio device.

</dd>

<dt><samp class="option">show_analog_tv_tuner_dialog</samp></dt>

<dd> If set to <samp class="option">true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV channels and frequencies.

</dd>

<dt><samp class="option">show_analog_tv_tuner_audio_dialog</samp></dt>

<dd> If set to <samp class="option">true</samp>, before capture starts, popup a display

dialog to the end user, allowing them to manually

modify TV audio (like mono vs. stereo, Language A,B or C).

</dd>

<dt><samp class="option">audio_device_load</samp></dt>

<dd> Load an audio capture filter device from file instead of searching

it by name. It may load additional parameters too, if the filter

supports the serialization of its properties to.

To use this an audio capture source has to be specified, but it can

be anything even fake one.

</dd>

<dt><samp class="option">audio_device_save</samp></dt>

<dd> Save the currently used audio capture filter device and its

parameters (if the filter supports it) to a file.

If a file with the same name exists it will be overwritten.

</dd>

<dt><samp class="option">video_device_load</samp></dt>

<dd> Load a video capture filter device from file instead of searching

it by name. It may load additional parameters too, if the filter

supports the serialization of its properties to.

To use this a video capture source has to be specified, but it can

be anything even fake one.

</dd>

<dt><samp class="option">video_device_save</samp></dt>

<dd> Save the currently used video capture filter device and its

parameters (if the filter supports it) to a file.

If a file with the same name exists it will be overwritten.

</dd>

<dt><samp class="option">use_video_device_timestamps</samp></dt>

<dd> If set to <samp class="option">false</samp>, the timestamp for video frames will be

derived from the wallclock instead of the timestamp provided by

the capture device. This allows working around devices that

provide unreliable timestamps.

</dd>

</dl>

 <h4 class="subsection">20.5.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Print the list of DirectShow supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f dshow -i dummy </pre>

Open video device <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -i video="Camera" </pre>

Open second video device with name <var class="var">Camera</var>: <pre class="example-preformatted">$ ffmpeg -f dshow -video_device_number 1 -i video="Camera" </pre>

Open video device <var class="var">Camera</var> and audio device <var class="var">Microphone</var>: <pre

class=”example-preformatted”>$ ffmpeg -f dshow -i video="Camera":audio="Microphone&q
</pre>

Print the list of supported options in selected device and exit: <pre class=”example-preformatted”>$ ffmpeg -list_options true -f dshow -i video="Camera" </pre>

Specify pin names to capture by name or alternative name, specify alternative device name: <pre class=”example-preformatted”>$ ffmpeg -f dshow -audio_pin_name "Audio Out" -video_pin_name 2 -i video=video="@device_pnp_\\?\pci#ven_1a0a&dev_6200&subsys_62
8f56-11d0-a3b9-00a0c9223196}\{ca465100-deb0-4d59-818f-8c477184adf6}":audio="Microphone&qu
</pre>

Configure a crossbar device, specifying crossbar pins, allow user to adjust video capture properties at startup: <pre class=”example-preformatted”>$ ffmpeg -f dshow -show_video_device_dialog true -crossbar_video_input_pin_number 0 -crossbar_audio_input_pin_number 3 -i video="AVerMedia BDA Analog Capture":audio="AVerMedia BDA Analog Capture" </pre>

 <h3 class=”section”>20.6 fbdev# TOC</h3>

Linux framebuffer input device. The Linux framebuffer is a graphic hardware-independent abstraction layer to show graphics on a computer monitor, typically on the console. It is accessed through a file device node, usually <samp class=”file”>/dev/fb0</samp>. For more detailed information read the file Documentation/fb/framebuffer.txt included in the Linux source tree. See also http://linux-fbdev.sourceforge.net/, and fbset(1). To record from the framebuffer device <samp class=”file”>/dev/fb0</samp> with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 10 -i /dev/fb0 out.avi </pre>

You can take a single screenshot image with the command: <pre class=”example-preformatted”>ffmpeg -f fbdev -framerate 1 -i /dev/fb0 -frames:v 1 screenshot.jpeg </pre>

 <h4 class=”subsection”>20.6.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>framerate</samp></dt>

<dd> Set the frame rate. Default is 25. </dd> </dl>

 <h3 class="section">20.7 gdigrab# TOC</h3>

Win32 GDI-based screen capture device. This device allows you to capture a region of the display on Windows. Amongst options for the input filenames are such elements as: <pre class="example-preformatted">desktop </pre> or <pre class="example-preformatted">title=<var class="var">window_title</var> </pre> or <pre class="example-preformatted">hwnd=<var class="var">window_hwnd</var> </pre>

The first option will capture the entire desktop, or a fixed region of the desktop. The second and third options will instead capture the contents of a single window, regardless of its position on the screen. For example, to grab the entire desktop using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i desktop out.mpg </pre>

Grab a 640x480 region at position <code class="code">10,20</code>: <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -offset_x 10 -offset_y 20 -video_size vga -i desktop out.mpg </pre>

Grab the contents of the window named "Calculator" <pre class="example-preformatted">ffmpeg -f gdigrab -framerate 6 -i title=Calculator out.mpg </pre>

 <h4 class="subsection">20.7.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. Use the value <code class="code">0</code> to not draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. Note that <var class="var">show_region</var> is incompatible with grabbing the contents of

a single window. For example: <pre class=”example-preformatted”>ffmpeg -f gdigrab -show_region 1 -framerate 6 -video_size cif -offset_x 10 -offset_y 20 -i desktop out.mpg </pre>

</dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video frame size. The default is to capture the full screen if <samp class=”file”>desktop</samp> is selected, or the full window size if <samp class=”file”>title=<var class=”var”>window_title</var></samp> is selected. </dd> <dt><samp class=”option”>offset_x</samp></dt> <dd> When capturing a region with <var class=”var”>video_size</var>, set the distance from the left edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned to the left of your primary monitor, you will need to use a negative <var class=”var”>offset_x</var> value to move the region to that monitor. </dd> <dt><samp class=”option”>offset_y</samp></dt> <dd> When capturing a region with <var class=”var”>video_size</var>, set the distance from the top edge of the screen or desktop. Note that the offset calculation is from the top left corner of the primary monitor on Windows. If you have a monitor positioned above your primary monitor, you will need to use a negative <var class=”var”>offset_y</var> value to move the region to that monitor. </dd> </dl>

 <h3 class=”section”>20.8 iec61883# TOC</h3>

FireWire DV/HDV input device using libiec61883. To enable this input device, you need libiec61883, libraw1394 and libavc1394 installed on your system. Use the configure option <code class=”code”>--enable-libiec61883</code> to compile with the device enabled. The iec61883 capture device supports capturing from a video device connected via IEEE1394 (FireWire), using libiec61883 and the new Linux FireWire stack (juju). This is the default DV/HDV input method in Linux Kernel 2.6.37 and later, since the old FireWire stack was removed. Specify the FireWire port to be used as input file, or "auto" to choose the first port connected. <h4 class=”subsection”>20.8.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>dvtype</samp></dt> <dd> Override autodetection of DV/HDV. This should only be used if auto detection does not work, or if usage of a different device type should be prohibited. Treating a DV device as HDV (or vice versa) will not work and result in undefined behavior. The values <samp

class=”option”>auto</samp>, <samp class=”option”>dv</samp> and <samp class=”option”>hdv</samp> are supported. </dd> <dt><samp class=”option”>dvbuffer</samp></dt> <dd> Set maximum size of buffer for incoming data, in frames. For DV, this is an exact value. For HDV, it is not frame exact, since HDV does not have a fixed frame size. </dd> <dt><samp class=”option”>dvguid</samp></dt> <dd> Select the capture device by specifying its GUID. Capturing will only be performed from the specified device and fails if no device with the given GUID is found. This is useful to select the input if multiple devices are connected at the same time. Look at /sys/bus/firewire/devices to find out the GUIDs. </dd> </dl>

 <h4 class=”subsection”>20.8.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Grab and show the input of a FireWire DV/HDV device. <pre class=”example-preformatted”>ffplay -f iec61883 -i auto </pre>

Grab and record the input of a FireWire DV/HDV device, using a packet buffer of 100000 packets if the source is HDV. <pre class=”example-preformatted”>ffmpeg -f iec61883 -i auto -dvbuffer 100000 out.mpg </pre>

 <h3 class=”section”>20.9 jack# TOC</h3>

JACK input device. To enable this input device during configuration you need libjack installed on your system. A JACK input device creates one or more JACK writable clients, one for each audio channel, with name <var class=”var”>client_name</var>:input_<var class=”var”>N</var>, where <var class=”var”>client_name</var> is the name provided by the application, and <var class=”var”>N</var> is a number which identifies the channel. Each writable client will send the acquired data to the FFmpeg input device. Once you have created one or more JACK readable clients, you need to connect them to one or more JACK writable clients. To connect or disconnect JACK clients you can use the <code class=”command”>jack_connect</code> and <code class=”command”>jack_disconnect</code> programs, or do it through a graphical interface, for example with <code class=”command”>qjackctl</code>. To list the JACK clients and their properties you can invoke the command <code class=”command”>jack_lsp</code>. Follows an example which shows how to capture a JACK readable client with <code class=”command”>ffmpeg</code>. <pre

class=”example-preformatted”># Create a JACK writable client with name "ffmpeg". $ ffmpeg -f jack -i ffmpeg -y out.wav

Start the sample jack_metro readable client. $ jack_metro -b 120 -d 0.2 -f 4000

List the current JACK clients. $ jack_lsp -c system:capture_1 system:capture_2 system:playback_1 system:playback_2 ffmpeg:input_1 metro:120_bpm

Connect metro to the ffmpeg writable client. $ jack_connect metro:120_bpm ffmpeg:input_1 </pre>

For more information read: http://jackaudio.org/ <h4 class=”subsection”>20.9.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class=”section”>20.10 kmsgrab# TOC</h3>

KMS video input device. Captures the KMS scanout framebuffer associated with a specified CRTC or plane as a DRM object that can be passed to other hardware functions. Requires either DRM master or CAP_SYS_ADMIN to run. If you don’t understand what all of that means, you probably don’t want this. Look at <samp class=”option”>x11grab</samp> instead. <h4 class=”subsection”>20.10.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>device</samp></dt> <dd> DRM device to capture on. Defaults to <samp class=”option”>/dev/dri/card0</samp>. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Pixel format of the framebuffer. This can be autodetected if you are running Linux 5.7 or later, but needs to be provided for earlier versions. Defaults to <samp class=”option”>bgr0</samp>, which is the most common format used by the Linux console and Xorg X server. </dd> <dt><samp class=”option”>format_modifier</samp></dt> <dd> Format modifier to signal on output frames. This is necessary to import correctly into some APIs. It can be autodetected if you are running Linux 5.7 or later, but will need to be provided explicitly when needed in earlier versions. See the libdrm documentation for possible values. </dd> <dt><samp

class=”option”>crtc_id</samp></dt> <dd> KMS CRTC ID to define the capture source. The first active plane on the given CRTC will be used. </dd> <dt><samp class=”option”>plane_id</samp></dt> <dd> KMS plane ID to define the capture source. Defaults to the first active plane found if neither <samp class=”option”>crtc_id</samp> nor <samp class=”option”>plane_id</samp> are specified. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Framerate to capture at. This is not synchronised to any page flipping or framebuffer changes - it just defines the interval at which the framebuffer is sampled. Sampling faster than the framebuffer update rate will generate independent frames with the same content. Defaults to <code class=”code”>30</code>. </dd> </dl>

 <h4 class=”subsection”>20.10.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Capture from the first active plane, download the result to normal frames and encode. This will only work if the framebuffer is both linear and mappable - if not, the result may be scrambled or fail to download. <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwdownload,format=bgr0' output.mp4 </pre>

Capture from CRTC ID 42 at 60fps, map the result to VAAPI, convert to NV12 and encode as H.264. <pre class=”example-preformatted”>ffmpeg -crtc_id 42 -framerate 60 -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,scale_vaapi=w=1920:h=1080:format=nv12' -c:v h264_vaapi output.mp4 </pre>

To capture only part of a plane the output can be cropped - this can be used to capture a single window, as long as it has a known absolute position and size. For example, to capture and encode the middle quarter of a 1920x1080 plane: <pre class=”example-preformatted”>ffmpeg -f kmsgrab -i - -vf 'hwmap=derive_device=vaapi,crop=960:540:480:270,scale_vaapi=960:540:nv12' -c:v h264_vaapi output.mp4 </pre>

 <h3 class=”section”>20.11 lavfi# TOC</h3>

Libavfilter input virtual device. This input device reads data from the open output pads of a libavfilter filtergraph. For each filtergraph open output, the input device will create a corresponding stream which is mapped to the generated output. The filtergraph is specified through the option <samp class=”option”>graph</samp>. <a

name=”Options-28”> <h4 class=”subsection”>20.11.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>graph</samp></dt> <dd> Specify the filtergraph to use as input. Each video open output must be labelled by a unique string of the form "out<var class=”var”>N</var>", where <var class=”var”>N</var> is a number starting from 0 corresponding to the mapped input stream generated by the device. The first unlabelled output is automatically assigned to the "out0" label, but all the others need to be specified explicitly. The suffix "+subcc" can be appended to the output label to create an extra stream with the closed captions packets attached to that output (experimental; only for EIA-608 / CEA-708 for now). The subcc streams are created after all the normal streams, in the order of the corresponding stream. For example, if there is "out19+subcc", "out7+subcc" and up to "out42", the stream #43 is subcc for stream #7 and stream #44 is subcc for stream #19. If not specified defaults to the filename specified for the input device. </dd> <dt><samp class=”option”>graph_file</samp></dt> <dd> Set the filename of the filtergraph to be read and sent to the other filters. Syntax of the filtergraph is the same as the one specified by the option <var class=”var”>graph</var>. </dd> <dt><samp class=”option”>dumpgraph</samp></dt> <dd> Dump graph to stderr. </dd> </dl>

 <h4 class=”subsection”>20.11.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Create a color video stream and play it back with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi -graph "color=c=pink [out0]" dummy </pre>

As the previous example, but use filename for specifying the graph description, and omit the "out0" label: <pre class=”example-preformatted”>ffplay -f lavfi color=c=pink </pre>

Create three different video test filtered sources and play them: <pre class=”example-preformatted”>ffplay -f lavfi -graph "testsrc [out0]; testsrc,hflip [out1]; testsrc,negate [out2]" test3 </pre>

Read an audio stream from a file using the amovie source and play it back with <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi "amovie=test.wav" </pre>

Read an audio stream and a video stream and play it back with <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi "movie=test.avi[out0];amovie=test.wav[out1]" </pre>

Dump decoded frames to images and Closed Captions to an RCWT backup: <pre class="example-preformatted">ffmpeg -f lavfi -i "movie=test.ts[out0+subcc]" -map v frame%08d.png -map s -c copy -f rcwt subcc.bin </pre>

 <h3 class="section">20.12 libcdio# TOC</h3>

Audio-CD input device based on libcdio. To enable this input device during configuration you need libcdio installed on your system. It requires the configure option <code class="code">--enable-libcdio</code>. This device allows playing and grabbing from an Audio-CD. For example to copy with <code class="command">ffmpeg</code> the entire Audio-CD in <samp class="file">/dev/sr0</samp>, you may run the command: <pre class="example-preformatted">ffmpeg -f libcdio -i /dev/sr0 cd.wav </pre>

 <h4 class="subsection">20.12.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">speed</samp></dt> <dd> Set drive reading speed. Default value is 0. The speed is specified CD-ROM speed units. The speed is set through the libcdio <code class="code">cdio_cddap_speed_set</code> function. On many CD-ROM drives, specifying a value too large will result in using the fastest speed. </dd> <dt><samp class="option">paranoia_mode</samp></dt> <dd> Set paranoia recovery mode flags. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dt>‘<samp class="samp">verify</samp>’</dt> <dt>‘<samp class="samp">overlap</samp>’</dt> <dt>‘<samp class="samp">neverskip</samp>’</dt> <dt>‘<samp class="samp">full</samp>’</dt> </dl>

Default value is ‘<samp class="samp">disable</samp>’. For more information about the available recovery modes, consult the paranoia project documentation. </dd> </dl>

 <h3 class="section">20.13 libdc1394# <a class="anchor hidden-xs"href="#toc-libdc1394"

aria-hidden=”true”>TOC</h3>

IIDC1394 input device, based on libdc1394 and libraw1394. Requires the configure option <code class=”code”>--enable-libdc1394</code>. <h4 class=”subsection”>20.13.1 Options# TOC</h4> <dl class=”table”> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the frame rate. Default is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Select the pixel format. Default is <code class=”code”>uyvy422</code>. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video size given as a string such as <code class=”code”>640x480</code> or <code class=”code”>hd720</code>. Default is <code class=”code”>qvga</code>. </dd> </dl>

 <h3 class=”section”>20.14 openal# TOC</h3>

The OpenAL input device provides audio capture on all systems with a working OpenAL 1.1 implementation. To enable this input device during configuration, you need OpenAL headers and libraries installed on your system, and need to configure FFmpeg with <code class=”code”>--enable-openal</code>. OpenAL headers and libraries should be provided as part of your OpenAL implementation, or as an additional download (an SDK). Depending on your installation you may need to specify additional flags via the <code class=”code”>--extra-cflags</code> and <code class=”code”>--extra-ldflags</code> for allowing the build system to locate the OpenAL headers and libraries. An incomplete list of OpenAL implementations follows: <dl class=”table”> <dt>Creative</dt> <dd> The official Windows implementation, providing hardware acceleration with supported devices and software fallback. See http://openal.org/. </dd> <dt>OpenAL Soft</dt> <dd> Portable, open source (LGPL) software implementation. Includes backends for the most common sound APIs on the Windows, Linux, Solaris, and BSD operating systems. See <a class=”url” href=”http://kcat.strangesoft.net/openal.html”>http://kcat.strangesoft.net/openal.html< </dd> <dt>Apple</dt> <dd> OpenAL is part of Core Audio, the official Mac OS X Audio interface. See http://developer.apple.com/technologies/mac/audio-and-video.html </dd> </dl>

This device allows one to capture from an audio input device handled through OpenAL. You need to specify the name of the device to capture in the provided filename. If the empty string is provided, the device will automatically select the default device. You can get the list of the supported devices by using the option <var class="var">list_devices</var>. <h4 class="subsection">20.14.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels in the captured audio. Only the values <samp class="option">1</samp> (monaural) and <samp class="option">2</samp> (stereo) are currently supported. Defaults to <samp class="option">2</samp>. </dd> <dt><samp class="option">sample_size</samp></dt> <dd> Set the sample size (in bits) of the captured audio. Only the values <samp class="option">8</samp> and <samp class="option">16</samp> are currently supported. Defaults to <samp class="option">16</samp>. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate (in Hz) of the captured audio. Defaults to <samp class="option">44.1k</samp>. </dd> <dt><samp class="option">list_devices</samp></dt> <dd> If set to <samp class="option">true</samp>, print a list of devices and exit. Defaults to <samp class="option">false</samp>. </dd> </dl>

 <h4 class="subsection">20.14.2 Examples# TOC</h4>

Print the list of OpenAL supported devices and exit: <pre class="example-preformatted">$ ffmpeg -list_devices true -f openal -i dummy out.ogg </pre>

Capture from the OpenAL device <samp class="file">DR-BT101 via PulseAudio</samp>: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out.ogg </pre>

Capture from the default device (note the empty string ” as filename): <pre class="example-preformatted">$ ffmpeg -f openal -i '' out.ogg </pre>

Capture from two devices simultaneously, writing to two different files, within the same <code class="command">ffmpeg</code> command: <pre class="example-preformatted">$ ffmpeg -f openal -i 'DR-BT101 via PulseAudio' out1.ogg -f openal -i 'ALSA Default' out2.ogg </pre> Note: not all OpenAL implementations support

multiple simultaneous capture - try the latest OpenAL Soft if the above does not work. <h3 class="section">20.15 oss# TOC</h3>

Open Sound System input device. The filename to provide to the input device is the device node representing the OSS input device, and is usually set to <samp class="file">/dev/dsp</samp>. For example to grab from <samp class="file">/dev/dsp</samp> using <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -f oss -i /dev/dsp /tmp/oss.wav </pre>

For more information about OSS see: <a class="url" href="http://manuals.opensound.com/usersguide/ds
  <h4 class="subsection">20.15.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">20.16 pulse# TOC</h3>

PulseAudio input device. To enable this output device you need to configure FFmpeg with <code class="code">--enable-libpulse</code>. The filename to provide to the input device is a source device or the string "default" To list the PulseAudio source devices and their properties you can invoke the command <code class="command">pactl list sources</code>. More information about PulseAudio can be found on http://www.pulseaudio.org. <h4 class="subsection">20.16.1 Options# TOC</h4> <dl class="table"> <dt><samp class="option">server</samp></dt> <dd> Connect to a specific PulseAudio server, specified by an IP address. Default server is used when not provided. </dd> <dt><samp class="option">name</samp></dt> <dd> Specify the application name PulseAudio will use when showing active clients, by default it is the <code class="code">LIBAVFORMAT_IDENT</code> string. </dd> <dt><samp class="option">stream_name</samp></dt> <dd> Specify the stream name PulseAudio will use when showing active

streams, by default it is "record". </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> Specify the samplerate in Hz, by default 48kHz is used. </dd> <dt><samp class="option">channels</samp></dt> <dd> Specify the channels in use, by default 2 (stereo) is set. </dd> <dt><samp class="option">frame_size</samp></dt> <dd> This option does nothing and is deprecated. </dd> <dt><samp class="option">fragment_size</samp></dt> <dd> Specify the size in bytes of the minimal buffering fragment in PulseAudio, it will affect the audio latency. By default it is set to 50 ms amount of data. </dd> <dt><samp class="option">wallclock</samp></dt> <dd> Set the initial PTS using the current time. Default is 1. </dd> </dl>

 <h4 class="subsection">20.16.2 Examples# TOC</h4> Record a stream from default device: <pre class="example-preformatted">ffmpeg -f pulse -i default /tmp/pulse.wav </pre>

 <h3 class="section">20.17 sndio# TOC</h3>

sndio input device. To enable this input device during configuration you need libsndio installed on your system. The filename to provide to the input device is the device node representing the sndio input device, and is usually set to <samp class="file">/dev/audio0</samp>. For example to grab from <samp class="file">/dev/audio0</samp> using <code class="command">ffmpeg</code> use the command: <pre class="example-preformatted">ffmpeg -f sndio -i /dev/audio0 /tmp/oss.wav </pre>

 <h4 class="subsection">20.17.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">sample_rate</samp></dt> <dd> Set the sample rate in Hz. Default is 48000. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the number of channels. Default is 2. </dd> </dl>

 <h3 class="section">20.18 video4linux2, v4l2# TOC</h3>

Video4Linux2 input video device. "v4l2" can be

used as alias for "video4linux2". If FFmpeg is built with v4l-utils support (by using the <code class="code">--enable-libv4l2</code> configure option), it is possible to use it with the <code class="code">-use_libv4l2</code> input device option. The name of the device to grab is a file device node, usually Linux systems tend to automatically create such nodes when the device (e.g. an USB webcam) is plugged into the system, and has a name of the kind <samp class="file">/dev/video<var class="var">N</var></samp>, where <var class="var">N</var> is a number associated to the device. Video4Linux2 devices usually support a limited set of <var class="var">width</var>x<var class="var">height</var> sizes and frame rates. You can check which are supported using <code class="command">-list_formats all</code> for Video4Linux2 devices. Some devices, like TV cards, support one or more standards. It is possible to list all the supported standards using <code class="command">-list_standards all</code>. The time base for the timestamps is 1 microsecond. Depending on the kernel version and configuration, the timestamps may be derived from the real time clock (origin at the Unix Epoch) or the monotonic clock (origin usually at boot time, unaffected by NTP or manual changes to the clock). The <samp class="option">-timestamps abs</samp> or <samp class="option">-ts abs</samp> option can be used to force conversion into the real time clock. Some usage examples of the video4linux2 device with <code class="command">ffmpeg</code> and <code class="command">ffplay</code>: <ul class="itemize mark-bullet"> List supported formats for a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -list_formats all /dev/video0 </pre>

Grab and show the input of a video4linux2 device: <pre class="example-preformatted">ffplay -f video4linux2 -framerate 30 -video_size hd720 /dev/video0 </pre>

Grab and record the input of a video4linux2 device, leave the frame rate and size as previously set: <pre class="example-preformatted">ffmpeg -f video4linux2 -input_format mjpeg -i /dev/video0 out.mpeg </pre>

For more information about Video4Linux, check http://linuxtv.org/. <h4 class="subsection">20.18.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">standard</samp></dt> <dd> Set the standard. Must be the name of a supported standard. To get a list of the supported standards, use the <samp class="option">list_standards</samp> option. </dd> <dt><samp class="option">channel</samp></dt> <dd> Set the input channel number. Default to -1, which means using the previously selected channel. </dd> <dt><samp

class=”option”>video_size</samp></dt> <dd> Set the video frame size. The argument must be a string in the form <var class=”var”>WIDTH</var>x<var class=”var”>HEIGHT</var> or a valid size abbreviation. </dd> <dt><samp class=”option”>pixel_format</samp></dt> <dd> Select the pixel format (only valid for raw video input). </dd> <dt><samp class=”option”>input_format</samp></dt> <dd> Set the preferred pixel format (for raw video) or a codec name. This option allows one to select the input format, when several are available. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the preferred video frame rate. </dd> <dt><samp class=”option”>list_formats</samp></dt> <dd> List available formats (supported pixel formats, codecs, and frame sizes) and exit. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Show all available (compressed and non-compressed) formats. </dd> <dt>‘<samp class=”samp”>raw</samp>’</dt> <dd> Show only raw video (non-compressed) formats. </dd> <dt>‘<samp class=”samp”>compressed</samp>’</dt> <dd> Show only compressed formats. </dd> </dl>

</dd> <dt><samp class=”option”>list_standards</samp></dt> <dd> List supported standards and exit. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Show all supported standards. </dd> </dl>

</dd> <dt><samp class=”option”>timestamps, ts</samp></dt> <dd> Set type of timestamps for grabbed frames. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>default</samp>’</dt> <dd> Use timestamps from the kernel. </dd> <dt>‘<samp class=”samp”>abs</samp>’</dt> <dd> Use absolute timestamps (wall clock). </dd> <dt>‘<samp class=”samp”>mono2abs</samp>’</dt> <dd> Force conversion from monotonic to absolute timestamps. </dd> </dl>

Default value is <code class=”code”>default</code>. </dd> <dt><samp class=”option”>use_libv4l2</samp></dt> <dd> Use libv4l2 (v4l-utils) conversion functions. Default is 0. </dd> </dl>

 <h3 class=”section”>20.19 vfwcap# TOC</h3>

VfW (Video for Windows) capture input device. The filename passed as input is the capture driver number, ranging from 0 to 9. You may use "list" as filename to print a list of drivers. Any other filename will be interpreted as device number 0. <h4 class=”subsection”>20.19.1 Options<a class=”anchor hidden-xs” href=”#Options-36” aria-

hidden=”true”># TOC</h4>

<dl class=”table”> <dt><samp class=”option”>video_size</samp></dt> <dd> Set the video frame size. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class=”code”>ntsc</code>, corresponding to a frame rate of <code class=”code”>30000/1001</code>. </dd> </dl>

 <h3 class=”section”>20.20 x11grab# TOC</h3>

X11 video input device. To enable this input device during configuration you need libxcb installed on your system. It will be automatically detected during configuration. This device allows one to capture a region of an X11 display. The filename passed as input has the syntax: <pre class=”example-preformatted”>[<var class=”var”>hostname</var>]:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var>[+<var class=”var”>x_offset</var>,<var class=”var”>y_offset</var>] </pre>

<var class=”var”>hostname</var>:<var class=”var”>display_number</var>.<var class=”var”>screen_number</var> specifies the X11 display name of the screen to grab from. <var class=”var”>hostname</var> can be omitted, and defaults to "localhost". The environment variable <code class=”env”>DISPLAY</code> contains the default display name. <var class=”var”>x_offset</var> and <var class=”var”>y_offset</var> specify the offsets of the grabbed area with respect to the top-left border of the X11 screen. They default to 0. Check the X11 documentation (e.g. <code class=”command”>man X</code>) for more detailed information. Use the <code class=”command”>xdpyinfo</code> program for getting basic information about the properties of your X11 display (e.g. grep for "name" or "dimensions"). For example to grab from <samp class=”file”>:0.0</samp> using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

Grab at position <code class=”code”>10,20</code>: <pre class=”example-preformatted”>ffmpeg -f x11grab -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

 <h4 class=”subsection”>20.20.1 Options# TOC</h4>

<dl class="table"> <dt><samp class="option">select_region</samp></dt> <dd> Specify whether to select the grabbing area graphically using the pointer. A value of <code class="code">1</code> prompts the user to select the grabbing area graphically by clicking and dragging. A single click with no dragging will select the whole screen. A region with zero width or height will also select the whole screen. This option overwrites the <var class="var">video_size</var>, <var class="var">grab_x</var>, and <var class="var">grab_y</var> options. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">draw_mouse</samp></dt> <dd> Specify whether to draw the mouse pointer. A value of <code class="code">0</code> specifies not to draw the pointer. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">follow_mouse</samp></dt> <dd> Make the grabbed area follow the mouse. The argument can be <code class="code">centered</code> or a number of pixels <var class="var">PIXELS</var>. When it is specified with "centered", the grabbing region follows the mouse pointer and keeps the pointer at the center of region; otherwise, the region follows only when the mouse pointer reaches within <var class="var">PIXELS</var> (greater than zero) to the edge of region. For example: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse centered -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

To follow only when the mouse pointer reaches within 100 pixels to edge: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse 100 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">framerate</samp></dt> <dd> Set the grabbing frame rate. Default value is <code class="code">ntsc</code>, corresponding to a frame rate of <code class="code">30000/1001</code>. </dd> <dt><samp class="option">show_region</samp></dt> <dd> Show grabbed region on screen. If <var class="var">show_region</var> is specified with <code class="code">1</code>, then the grabbing region will be indicated on screen. With this option, it is easy to know what is being grabbed if only a portion of the screen is grabbed. </dd> <dt><samp class="option">region_border</samp></dt> <dd> Set the region border thickness if <samp class="option">-show_region 1</samp> is used. Range is 1 to 128 and default is 3 (XCB-based x11grab only). For example: <pre class="example-preformatted">ffmpeg -f x11grab -show_region 1 -framerate 25 -video_size cif -i :0.0+10,20 out.mpg </pre>

With <var class="var">follow_mouse</var>: <pre class="example-preformatted">ffmpeg -f x11grab -follow_mouse centered -show_region 1 -framerate 25 -video_size cif -i :0.0 out.mpg </pre>

</dd> <dt><samp class="option">window_id</samp></dt> <dd> Grab this window, instead of the whole screen. Default value is 0, which maps to the whole screen (root window). The id of a window can be found using the <code class="command">xwininfo</code> program, possibly with options -tree and -root. If the window is later enlarged, the new area is not recorded. Video ends when the window is closed, unmapped (i.e., iconified) or shrunk beyond the video size (which defaults to the initial window size). This option disables options <samp class="option">follow_mouse</samp> and <samp class="option">select_region</samp>. </dd> <dt><samp class="option">video_size</samp></dt> <dd> Set the video frame size. Default is the full desktop or window. </dd> <dt><samp class="option">grab_x</samp></dt> <dt><samp class="option">grab_y</samp></dt> <dd> Set the grabbing region coordinates. They are expressed as offset from the top left corner of the X11 window and correspond to the <var class="var">x_offset</var> and <var class="var">y_offset</var> parameters in the device name. The default value for both options is 0. </dd> </dl>

 <h2 class="chapter">21 Resampler Options# TOC</h2>

The audio resampler supports the following named options. Options may be set by specifying -<var class="var">option</var> <var class="var">value</var> in the FFmpeg tools, <var class="var">option</var>=<var class="var">value</var> for the aresample filter, by setting the value explicitly in the <code class="code">SwrContext</code> options or using the <samp class="file">libavutil/opt.h</samp> API for programmatic use. <dl class="table"> <dt><samp class="option">uchl, used_chlayout</samp></dt> <dd> Set used input channel layout. Default is unset. This option is only used for special remapping. </dd> <dt><samp class="option">isr, in_sample_rate</samp></dt> <dd> Set the input sample rate. Default value is 0. </dd> <dt><samp class="option">osr, out_sample_rate</samp></dt> <dd> Set the output sample rate. Default value is 0. </dd> <dt><samp class="option">isf, in_sample_fmt</samp></dt> <dd> Specify the input sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">osf, out_sample_fmt</samp></dt> <dd> Specify the output sample format. It is set by default to <code class="code">none</code>. </dd> <dt><samp class="option">tsf, internal_sample_fmt</samp></dt> <dd> Set the internal sample format. Default value is <code class="code">none</code>. This will automatically be chosen when it is not explicitly set. </dd> <dt><samp class="option">ichl, in_chlayout</samp></dt> <dt><samp class="option">ochl, out_chlayout</samp></dt> <dd> Set the in-

put/output channel layout. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> <dt><samp class="option">clev, center_mix_level</samp></dt> <dd> Set the center mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">slev, surround_mix_level</samp></dt> <dd> Set the surround mix level. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">lfe_mix_level</samp></dt> <dd> Set LFE mix into non LFE level. It is used when there is a LFE input but no LFE output. It is a value expressed in deciBel, and must be in the interval [-32,32]. </dd> <dt><samp class="option">rmvol, rematrix_volume</samp></dt> <dd> Set rematrix volume. Default value is 1.0. </dd> <dt><samp class="option">rematrix_maxval</samp></dt> <dd> Set maximum output value for rematrixing. This can be used to prevent clipping vs. preventing volume reduction. A value of 1.0 prevents clipping. </dd> <dt><samp class="option">flags, swr_flags</samp></dt> <dd> Set flags used by the converter. Default value is 0. It supports the following individual flags: <dl class="table"> <dt><samp class="option">res</samp></dt> <dd> force resampling, this flag forces resampling to be used even when the input and output sample rates match. </dd> </dl>

</dd> <dt><samp class="option">dither_scale</samp></dt> <dd> Set the dither scale. Default value is 1. </dd> <dt><samp class="option">dither_method</samp></dt>
<dd> Set dither method. Default value is 0. Supported values:
<dl class="table"> <dt>‘<samp class="samp">rectangular</samp>’</dt>
<dd> select rectangular dither </dd> <dt>‘<samp class="samp">triangular</samp>’</dt> <dd> select triangular dither </dd> <dt>‘<samp class="samp">triangular_hp</samp>’</dt>
<dd> select triangular dither with high pass </dd> <dt>‘<samp class="samp">lipshitz</samp>’</dt> <dd> select Lipshitz noise shaping dither. </dd> <dt>‘<samp class="samp">shibata</samp>’</dt>
<dd> select Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">low_shibata</samp>’</dt> <dd> select low Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">high_shibata</samp>’</
<dd> select high Shibata noise shaping dither. </dd> <dt>‘<samp class="samp">f_weighted</samp>’</dt> <dd> select f-weighted noise shaping dither </dd> <dt>‘<samp class="samp">modified_e_weighted</samp>
<dd> select modified-e-weighted noise shaping dither </dd>
<dt>‘<samp class="samp">improved_e_weighted</samp>’</dt>
<dd> select improved-e-weighted noise shaping dither </dd> </dl>

</dd> <dt><samp class="option">resampler</samp></dt> <dd> Set resampling engine. Default value is swr. Supported values:
<dl class="table"> <dt>‘<samp class="samp">swr</samp>’</dt>

<dd> select the native SW Resampler; filter options precision and cheby are not applicable in this case. </dd> <dt>‘<samp class="samp">soxr</samp>’</dt> <dd> select the SoX Resampler (where available); compensation, and filter options filter_size, phase_shift, exact_rational, filter_type & kaiser_beta, are not applicable in this case. </dd> </dl>

</dd>

<dt><samp class="option">filter_size</samp></dt>

<dd> For swr only, set resampling filter size, default value is 32.

</dd>

<dt><samp class="option">phase_shift</samp></dt>

<dd> For swr only, set resampling phase shift, default value is 10, and must be in

the interval [0,30].

</dd>

<dt><samp class="option">linear_interp</samp></dt>

<dd> Use linear interpolation when enabled (the default). Disable it if you want

to preserve speed instead of quality when exact_rational fails.

</dd>

<dt><samp class="option">exact_rational</samp></dt>

<dd> For swr only, when enabled, try to use exact phase_count based on input and

output sample rate. However, if it is larger than <code class="code">1 << phase_shift</code>,

the phase_count will be <code class="code">1 << phase_shift</code> as fallback. Default is enabled.

</dd>

<dt><samp class="option">cutoff</samp></dt>

<dd> Set cutoff frequency (swr: 6dB point; soxr: 0dB point) ratio; must be a float

value between 0 and 1. Default value is 0.97 with swr, and 0.91 with soxr

(which, with a sample-rate of 44100, preserves the entire audio band to 20kHz).

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> For soxr only, the precision in bits to which the resampled signal will be

calculated. The default value of 20 (which, with suitable dithering, is

appropriate for a destination bit-depth of 16) gives SoX’s ’High Quality’; a

value of 28 gives SoX’s ’Very High Quality’.

</dd>

<dt><samp class="option">cheby</samp></dt>

<dd> For soxr only, selects passband rolloff none (Chebyshev) & higher-precision

approximation for ’irrational’ ratios. Default value is 0.

</dd>

<dt><samp class="option">async</samp></dt>

<dd> For swr only, simple 1 parameter audio sync to timestamps using stretching,

squeezing, filling and trimming. Setting this to 1 will enable filling and

trimming, larger values represent the maximum amount in samples that the data

may be stretched or squeezed for each second.

Default value is 0, thus no compensation is applied to make the samples match

the audio timestamps.

</dd>

<dt><samp class="option">first_pts</samp></dt>

<dd> For swr only, assume the first pts should be this value. The time unit is 1 / sample rate.

This allows for padding/trimming at the start of stream. By default, no

assumption is made about the first frame’s expected pts, so no padding or

trimming is done. For example, this could be set to 0 to pad the beginning with

silence if an audio stream starts after the video stream or to trim any samples

with a negative pts due to encoder delay.

</dd>

<dt><samp class="option">min_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger stretching/squeezing/filling or trimming of the

data to make it match the timestamps. The default is that

stretching/squeezing/filling and trimming is disabled

(<samp class="option">min_comp</samp> = <code class="code">FLT_MAX</code>).

</dd>

<dt><samp class="option">min_hard_comp</samp></dt>

<dd> For swr only, set the minimum difference between timestamps and audio data (in

seconds) to trigger adding/dropping samples to make it match the

timestamps. This option effectively is a threshold to select between

hard (trim/fill) and soft (squeeze/stretch) compensation. Note that

all compensation is by default disabled through <samp class="option">min_comp</samp>.

The default is 0.1.

</dd>

<dt><samp class="option">comp_duration</samp></dt>

<dd> For swr only, set duration (in seconds) over which data is stretched/squeezed

to make it match the timestamps. Must be a non-negative double float value,

default value is 1.0.

</dd>

<dt><samp class=”option”>max_soft_comp</samp></dt>

<dd> For swr only, set maximum factor by which data is stretched/squeezed to make it

match the timestamps. Must be a non-negative double float value, default value

is 0.

</dd>

<dt><samp class=”option”>matrix_encoding</samp></dt>

<dd> Select matrixed stereo encoding.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>none</samp>’</dt>

<dd> select none

</dd>

<dt>‘<samp class=”samp”>dolby</samp>’</dt>

<dd> select Dolby

</dd>

<dt>‘<samp class=”samp”>dplii</samp>’</dt>

<dd> select Dolby Pro Logic II

</dd>

</dl>

Default value is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>filter_type</samp></dt> <dd> For swr only, select resampling filter type. This only affects resampling operations. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>cubic</samp>’</dt> <dd> select cubic </dd> <dt>‘<samp class=”samp”>blackman_nuttall</samp>’</dt> <dd> select Blackman Nuttall windowed sinc </dd> <dt>‘<samp

class=”samp”>kaiser</samp>’</dt> <dd> select Kaiser windowed sinc </dd> </dl>

</dd> <dt><samp class=”option”>kaiser_beta</samp></dt> <dd> For swr only, set Kaiser window beta value. Must be a double float value in the interval [2,16], default value is 9. </dd> <dt><samp class=”option”>output_sample_bits</samp></dt> <dd> For swr only, set number of used output sample bits for dithering. Must be an integer in the interval [0,64], default value is 0, which means it’s not used. </dd> </dl>

 <h2 class=”chapter”>22 Scaler Options# TOC</h2>

The video scaler supports the following named options. Options may be set by specifying -<var class=”var”>option</var> <var class=”var”>value</var> in the FFmpeg tools, with a few API-only exceptions noted below. For programmatic use, they can be set explicitly in the <code class=”code”>SwsContext</code> options or through the <samp class=”file”>libavutil/opt.h</samp> API. <dl class=”table”> <dd> </dd> <dt><samp class=”option”>scaler, scaler_sub</samp></dt> <dd> Choose the scaling algorithm to use. Default value is ‘<samp class=”samp”>auto</samp>’ for both. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Aumotic choice. For ‘<samp class=”samp”>scaler_sub</samp>’, this means the same algorithm as ‘<samp class=”samp”>scaler</samp>’. For ‘<samp class=”samp”>scaler</samp>’, this defaults to the scaler flag selected by ‘<samp class=”samp”>sws_flags</samp>’. </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Bilinear filter. (AKA triangle filter) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> 2-tap cubic BC-spline (AKA Mitchell-Netravali spline). The B and C parameters can be configured by setting <code class=”code”>param0</code> and <code class=”code”>param1</code>, defaulting to 0.0 and 0.6 respectively. </dd> <dt>‘<samp class=”samp”>point, neighbor</samp>’</dt> <dd> Point sampling (AKA nearest neighbor). </dd> <dt>‘<samp class=”samp”>area</samp>’</dt> <dd> Area averaging. Equivalent to ‘<samp class=”samp”>bilinear</samp>’ for upscaling. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> 2-tap Gaussian filter approximation. The sharpness parameter can be configured by setting <code class=”code”>param0</code>, defaulting to 3.0. </dd> <dt>‘<samp class=”samp”>sinc</samp>’</dt> <dd> Unwindowed sinc filter. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Lanczos resam-

pling (sinc windowed sinc). The number of filter taps can be configured by setting <code class=”code”>param0</code>, defaulting to 3. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Unwindowed natural bicubic spline. </dd> </dl>

</dd> <dt><samp class=”option”>sws_flags</samp></dt> <dd> Set the scaler flags. This is also used to set the scaling algorithm, though this usage is deprecated in favor of setting ‘<samp class=”samp”>scaler</samp>’. Only a single algorithm may be selected. Default value is ‘<samp class=”samp”>bicubic</samp>’. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>fast_bilinear</samp>’</dt> <dd> Select fast bilinear scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bilinear</samp>’</dt> <dd> Select bilinear scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bicubic</samp>’</dt> <dd> Select bicubic scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>experimental</samp>’</dt> <dd> Select experimental scaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>neighbor</samp>’</dt> <dd> Select nearest neighbor rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>area</samp>’</dt> <dd> Select averaging area rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>bicublin</samp>’</dt> <dd> Select bicubic scaling algorithm for the luma component, bilinear for chroma components. (Deprecated) </dd> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dd> Select Gaussian rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>sinc</samp>’</dt> <dd> Select sinc rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Select Lanczos rescaling algorithm. The default width (alpha) is 3 and can be changed by setting <code class=”code”>param0</code>. (Deprecated) </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Select natural bicubic spline rescaling algorithm. (Deprecated) </dd> <dt>‘<samp class=”samp”>print_info</samp>’</dt> <dd> Enable printing/debug logging. </dd> <dt>‘<samp class=”samp”>accurate_rnd</samp>’</dt> <dd> Enable accurate rounding. </dd> <dt>‘<samp class=”samp”>full_chroma_int</samp>’</dt> <dd> Enable full chroma interpolation. </dd> <dt>‘<samp class=”samp”>full_chroma_inp</samp>’</dt> <dd> Select full chroma input. </dd> <dt>‘<samp class=”samp”>bitexact</samp>’</dt> <dd> Enable bitexact output. </dd> <dt>‘<samp class=”samp”>unstable</samp>’</dt> <dd> Allow the use of experimental new code. May subtly affect the output or even produce wrong results. For testing only. </dd> </dl>

</dd> <dt><samp class=”option”>srcw <var class=”var”>(API only)</var></samp></dt>

<dd> Set source width. </dd> <dt><samp class="option">srch
<var class="var">(API only)</var></samp></dt> <dd> Set
source height. </dd> <dt><samp class="option">dstw <var
class="var">(API only)</var></samp></dt> <dd> Set desti-
nation width. </dd> <dt><samp class="option">dsth <var
class="var">(API only)</var></samp></dt> <dd> Set destina-
tion height. </dd> <dt><samp class="option">src_format <var
class="var">(API only)</var></samp></dt> <dd> Set source pixel
format (must be expressed as an integer). </dd> <dt><samp
class="option">dst_format <var class="var">(API only)</var></samp></dt>
<dd> Set destination pixel format (must be expressed as an in-
teger). </dd> <dt><samp class="option">src_range <var
class="var">(boolean)</var></samp></dt> <dd> If value is set
to <code class="code">1</code>, indicates source is full range. De-
fault value is <code class="code">0</code>, which indicates source is
limited range. </dd> <dt><samp class="option">dst_range
<var class="var">(boolean)</var></samp></dt> <dd> If value
is set to <code class="code">1</code>, enable full range for destina-
tion. Default value is <code class="code">0</code>, which enables
limited range. </dd> <dt><samp class="option">gamma <var
class="var">(boolean)</var></samp></dt> <dd> If value is set
to <code class="code">1</code>, enable gamma correct scaling. De-
fault value is <code class="code">0</code>. <a class="anchor"
id="sws_005fparams"></dd> <dt><samp class="option">param0,
param1</samp></dt> <dd> Set scaling algorithm parameters. The
specified values are specific of some scaling algorithms and ignored by oth-
ers. The specified values are floating point number values. </dd>
<dt><samp class="option">sws_dither</samp></dt> <dd> Set
the dithering algorithm. Accepts one of the following values. Default
value is ‘<samp class="samp">auto</samp>’. <dl
class="table"> <dt>‘<samp class="samp">auto</samp>’</dt>
<dd> automatic choice </dd> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> no dithering </dd> <dt>‘<samp class="samp">bayer</samp>’</dt>
<dd> bayer dither </dd> <dt>‘<samp class="samp">ed</samp>’</dt>
<dd> error diffusion dither </dd> <dt>‘<samp class="samp">a_dither</samp>’<
<dd> arithmetic dither, based using addition </dd> <dt>‘<samp
class="samp">x_dither</samp>’</dt> <dd> arithmetic dither,
based using xor (more random/less apparent patterning that a_dither).
</dd> </dl>

</dd> <dt><samp class="option">alphablend</samp></dt> <dd> Set
the alpha blending to use when the input has alpha but the output does not. De-
fault value is ‘<samp class="samp">none</samp>’. <dl
class="table"> <dt>‘<samp class="samp">uniform_color</samp>’</dt>
<dd> Blend onto a uniform background color </dd> <dt>‘<samp
class="samp">checkerboard</samp>’</dt> <dd> Blend onto a

checkerboard </dd> <dt>‘<samp class="samp">none</samp>’</dt>
<dd> No blending </dd> </dl>

</dd> <dt><samp class="option">sws_backends</samp></dt> <dd> Set the allowed swscale backends. This is a flags option, so multiple backends may be combined. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic selection. Equal to either ‘<samp class="samp">stable</samp>’ or ‘<samp class="samp">all</samp>’ depending on whether or not the ‘<samp class="samp">unstable</samp>’ flag is set. This is the default value. </dd> <dt>‘<samp class="samp">stable</samp>’</dt> <dd> All stable backends. </dd> <dt>‘<samp class="samp">unstable</samp>’</dt> <dd> All unstable backends. </dd> <dt>‘<samp class="samp">all</samp>’</dt> <dd> All available backends. </dd> <dt>‘<samp class="samp">legacy</samp>’</dt> <dd> Legacy swscale code. </dd> <dt>‘<samp class="samp">c</samp>’</dt> <dd> Template-based reference code. </dd> <dt>‘<samp class="samp">memcpy</samp>’</dt> <dd> Fast path using libc <code class="code">memcpy</code>. </dd> <dt>‘<samp class="samp">x86</samp>’</dt> <dd> x86 SIMD kernels. </dd> <dt>‘<samp class="samp">aarch64</samp>’</d <dd> AArch64 NEON kernels. </dd> <dt>‘<samp class="samp">spirv</samp>’</dt> <dd> Vulkan SPIR-V backend. </dd> </dl>

</dd> </dl>

 <h2 class="chapter">23 Filtering Introduction# TOC</h2>

Filtering in FFmpeg is enabled through the libavfilter library. In libavfilter, a filter can have multiple inputs and multiple outputs. To illustrate the sorts of things that are possible, we consider the following filtergraph. <pre class="verbatim"> [main] input --> split ----------> overlay --> output | ^ |[tmp] [flip]| +-----> crop --> vflip -------+ </pre> This filtergraph splits the input stream in two streams, then sends one stream through the crop filter and the vflip filter, before merging it back with the other stream by overlaying it on top. You can use the following command to achieve this: <pre class="example-preformatted">ffmpeg -i INPUT -vf "split [main][tmp]; [tmp] crop=iw:ih/2:0:0, vflip [flip]; [main][flip] overlay=0:H/2" OUTPUT </pre>

The result will be that the top half of the video is mirrored onto the bottom half of the output video. Filters in the same linear chain are

separated by commas, and distinct linear chains of filters are separated by semicolons. In our example, <var class="var">crop,vflip</var> are in one linear chain, <var class="var">split</var> and <var class="var">overlay</var> are separately in another. The points where the linear chains join are labelled by names enclosed in square brackets. In the example, the split filter generates two outputs that are associated to the labels <var class="var">[main]</var> and <var class="var">[tmp]</var>. The stream sent to the second output of <var class="var">split</var>, labelled as <var class="var">[tmp]</var>, is processed through the <var class="var">crop</var> filter, which crops away the lower half part of the video, and then vertically flipped. The <var class="var">overlay</var> filter takes in input the first unchanged output of the split filter (which was labelled as <var class="var">[main]</var>), and overlay on its lower half the output generated by the <var class="var">crop,vflip</var> filterchain. Some filters take in input a list of parameters: they are specified after the filter name and an equal sign, and are separated from each other by a colon. There exist so-called <var class="var">source filters</var> that do not have an audio/video input, and <var class="var">sink filters</var> that will not have audio/video output.

 <h2 class="chapter">24 graph2dot# TOC</h2>

The <samp class="file">graph2dot</samp> program included in the FFmpeg <samp class="file">tools</samp> directory can be used to parse a filtergraph description and issue a corresponding textual representation in the dot language. Invoke the command: <pre class="example-preformatted">graph2dot -h </pre>

to see how to use <samp class="file">graph2dot</samp>. You can then pass the dot description to the <samp class="file">dot</samp> program (from the graphviz suite of programs) and obtain a graphical representation of the filtergraph. For example the sequence of commands: <pre class="example-preformatted">echo <var class="var">GRAPH_DESCRIPTION</var> | \ tools/graph2dot -o graph.tmp && \ dot -Tpng graph.tmp -o graph.png && \ display graph.png </pre>

can be used to create and display an image representing the graph described by the <var class="var">GRAPH_DESCRIPTION</var> string. Note that this string must be a complete self-contained graph, with its inputs and outputs explicitly defined. For example if your command line is of the form: <pre class="example-preformatted">ffmpeg -i infile -vf scale=640:360 outfile </pre> your <var class="var">GRAPH_DESCRIPTION</var> string will need to be of the form: <pre class="example-

preformatted”>nullsrc,scale=640:360,nullsink </pre> you may also need to set the <var class=”var”>nullsrc</var> parameters and add a <var class=”var”>format</var> filter in order to simulate a specific input file.

 <h2 class=”chapter”>25 Filtergraph description# TOC</h2>

A filtergraph is a directed graph of connected filters. It can contain cycles, and there can be multiple links between a pair of filters. Each link has one input pad on one side connecting it to one filter from which it takes its input, and one output pad on the other side connecting it to one filter accepting its output. Each filter in a filtergraph is an instance of a filter class registered in the application, which defines the features and the number of input and output pads of the filter. A filter with no input pads is called a "source", and a filter with no output pads is called a "sink". <h3 class=”section”>25.1 Filtergraph syntax# TOC</h3>

A filtergraph has a textual representation, which is recognized by the <samp class=”option”>-filter</samp>/<samp class=”option”>-vf</samp>/<samp class=”option”>-af</samp> and <samp class=”option”>-filter_complex</samp> options in <code class=”command”>ffmpeg</code> and <samp class=”option”>-vf</samp>/<samp class=”option”>-af</samp> in <code class=”command”>ffplay</code>, and by the <code class=”code”>avfilter_graph_parse_ptr()</co function defined in <samp class=”file”>libavfilter/avfilter.h</samp>. A filterchain consists of a sequence of connected filters, each one connected to the previous one in the sequence. A filterchain is represented by a list of ","-separated filter descriptions. A filtergraph consists of a sequence of filterchains. A sequence of filterchains is represented by a list of ";"-separated filterchain descriptions. A filter is represented by a string of the form: [<var class=”var”>in_link_1</var>]...[<var class=”var”>in_link_N</var>]<var class=”var”>filter_name</var>@<var class=”var”>id</var>=<var class=”var”>arguments</var>[<var class=”var”>out_link_1</var>]...[<var class=”var”>out_link_M</var>] <var class=”var”>filter_name</var> is the name of the filter class of which the described filter is an instance of, and has to be the name of one of the filter classes registered in the program optionally followed by "@<var class=”var”>id</var>". The name of the filter class is optionally followed by a string "=<var class=”var”>arguments</var>". <var class=”var”>arguments</var>

is a string which contains the parameters used to initialize the filter instance. It may have one of two forms: <ul class="itemize mark-bullet"> A ’:’-separated list of <var class="var">key=value</var> pairs.

A ’:’-separated list of <var class="var">value</var>. In this case, the keys are assumed to be the option names in the order they are declared. E.g. the <code class="code">fade</code> filter declares three options in this order – <samp class="option">type</samp>, <samp class="option">start_frame</samp> and <samp class="option">nb_frames</samp>. Then the parameter list <var class="var">in:0:30</var> means that the value <var class="var">in</var> is assigned to the option <samp class="option">type</samp>, <var class="var">0</var> to <samp class="option">start_frame</samp> and <var class="var">30</var> to <samp class="option">nb_frames</samp>.

A ’:’-separated list of mixed direct <var class="var">value</var> and long <var class="var">key=value</var> pairs. The direct <var class="var">value</var> must precede the <var class="var">key=value</var> pairs, and follow the same constraints order of the previous point. The following <var class="var">key=value</var> pairs can be set in any preferred order.

If the option value itself is a list of items (e.g. the <code class="code">format</code> filter takes a list of pixel formats), the items in the list are usually separated by ‘<samp class="samp">|</samp>’. The list of arguments can be quoted using the character ‘<samp class="samp">'</samp>’ as initial and ending mark, and the character ‘<samp class="samp">\</samp>’ for escaping the characters within the quoted text; otherwise the argument string is considered terminated when the next special character (belonging to the set ‘<samp class="samp">[]=;,</samp>’) is encountered. A special syntax implemented in the <code class="command">ffmpeg</code> CLI tool allows loading option values from files. This is done be prepending a slash ’/’ to the option name, then the supplied value is interpreted as a path from which the actual value is loaded. E.g. <pre class="example-preformatted">ffmpeg -i <INPUT> -vf drawtext=/text=/tmp/some_text <OUTPUT> </pre> will load the text to be drawn from <samp class="file">/tmp/some_text</samp>. API users wishing to implement a similar feature should use the <code class="code">avfilter_graph_segment_*()</code> functions together with custom IO code. The name and arguments of the filter are optionally preceded and followed by a list of link labels. A link label allows one to name a link and associate it to a filter output or input pad. The preceding labels <var class="var">in_link_1</var> ... <var class="var">in_link_N</var>, are associated to the filter input pads, the following labels <var class="var">out_link_1</var> ... <var class="var">out_link_M</var>, are associated to the output pads.

When two link labels with the same name are found in the filtergraph, a link between the corresponding input and output pad is created. If an output pad is not labelled, it is linked by default to the first unlabelled input pad of the next filter in the filterchain. For example in the filterchain <pre class="example-preformatted">nullsrc, split[L1], [L2]overlay, nullsink </pre> the split filter instance has two output pads, and the overlay filter instance two input pads. The first output pad of split is labelled "L1", the first input pad of overlay is labelled "L2", and the second output pad of split is linked to the second input pad of overlay, which are both unlabelled. In a filter description, if the input label of the first filter is not specified, "in" is assumed; if the output label of the last filter is not specified, "out" is assumed. In a complete filterchain all the unlabelled filter input and output pads must be connected. A filtergraph is considered valid if all the filter input and output pads of all the filterchains are connected. Leading and trailing whitespaces (space, tabs, or line feeds) separating tokens in the filtergraph specification are ignored. This means that the filtergraph can be expressed using empty lines and spaces to improve readability. For example, the filtergraph: <pre class="example-preformatted">testsrc,split[L1],hflip[L2];[L1][L2] hstack </pre>

can be represented as: <pre class="example-preformatted">testsrc, split [L1], hflip [L2];

[L1][L2] hstack </pre>

Libavfilter will automatically insert scale filters where format conversion is required. It is possible to specify swscale flags for those automatically inserted scalers by prepending <code class="code">sws_flags=<var class="var">flags</var>;</code> to the filtergraph description. Here is a BNF description of the filtergraph syntax: <pre class="example-preformatted"><var class="var">NAME</var> ::= sequence of alphanumeric characters and '_' <var class="var">FILTER_NAME</var> ::= <var class="var">NAME</var>["@"<var class="var">NAME</var>] <var class="var">LINKLABEL</var> ::= "[" <var class="var">NAME</var> "]" <var class="var">LINKLABELS</var> ::= <var class="var">LINKLABEL</var> [<var class="var">LINKLABELS</var>] <var class="var">FILTER_ARGUMENTS</var> ::= sequence of chars (possibly quoted) <var class="var">FILTER</var> ::= [<var class="var">LINKLABELS</var>] <var class="var">FILTER_NAME</var> ["=" <var class="var">FILTER_ARGUMENTS</var>] [<var class="var">LINKLABELS</var>] <var class="var">FILTERCHAIN</var> ::= <var class="var">FILTER</var> [,<var class="var">FILTERCHAIN</var>] <var class="var">FILTERGRAPH</var> ::= [sws_flags=<var class="var">flags</var>;] <var class="var">FILTERCHAIN</var> [;<var class="var">FILTERGRAPH</var>] </pre>

 <h3 class=”section”>25.2 Notes on filtergraph escaping# TOC</h3>

Filtergraph description composition entails several levels of escaping. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#quoting_005fand_005fescaping”>the "Quoting and escaping" section in the ffmpeg-utils(1) manual for more information about the employed escaping procedure. A first level escaping affects the content of each filter option value, which may contain the special character <code class=”code”>:</code> used to separate values, or one of the escaping characters <code class=”code”>\'</code>. A second level escaping affects the whole filter description, which may contain the escaping characters <code class=”code”>\'</code> or the special characters <code class=”code”>[],;</code> used by the filtergraph description. Finally, when you specify a filtergraph on a shell commandline, you need to perform a third level escaping for the shell special characters contained within it. For example, consider the following string to be embedded in the drawtext filter description <samp class=”option”>text</samp> value: <pre class=”example-preformatted”>this is a 'string': may contain one, or more, special characters </pre>

This string contains the <code class=”code”>'</code> special escaping character, and the <code class=”code”>:</code> special character, so it needs to be escaped in this way: <pre class=”example-preformatted”>text=this is a \'string\'\: may contain one, or more, special characters </pre>

A second level of escaping is required when embedding the filter description in a filtergraph description, in order to escape all the filtergraph special characters. Thus the example above becomes: <pre class=”example-preformatted”>drawtext=text=this is a \\\'string\\\'\\: may contain one\, or more\, special characters </pre> (note that in addition to the <code class=”code”>\'</code> escaping special characters, also <code class=”code”>,</code> needs to be escaped). Finally an additional level of escaping is needed when writing the filtergraph description in a shell command, which depends on the escaping rules of the adopted shell. For example, assuming that <code class=”code”>\</code> is special and needs to be escaped with another <code class=”code”>\</code>, the previous string will finally result in: <pre class=”example-preformatted”>-vf "drawtext=text=this is a \\\\\\'string\\\\\\'\\\\: may contain one\\, or more\\, special characters" </pre>

In order to avoid cumbersome escaping when using a commandline tool accepting a filter specification as input, it is advisable to avoid direct inclu-

sion of the filter or options specification in the shell. For example, in case of the drawtext filter, you might prefer to use the <samp class="option">textfile</samp> option in place of <samp class="option">text</samp> to specify the text to render. <h2 class="chapter">26 Timeline editing# TOC</h2>

Some filters support a generic <samp class="option">enable</samp> option. For the filters supporting timeline editing, this option can be set to an expression which is evaluated before sending a frame to the filter. If the evaluation is non-zero, the filter will be enabled, otherwise the frame will be sent unchanged to the next filter in the filtergraph. The expression accepts the following values: <dl class="table"> <dt>‘<samp class="samp">t</samp>’</dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt>‘<samp class="samp">n</samp>’</dt> <dd> sequential number of the input frame, starting from 0 </dd> <dt>‘<samp class="samp">pos</samp>’</dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt>‘<samp class="samp">w</samp>’</dt> <dt>‘<samp class="samp">h</samp>’</dt> <dd> width and height of the input frame if video </dd> </dl>

Additionally, these filters support an <samp class="option">enable</samp> command that can be used to re-define the expression. Like any other filtering option, the <samp class="option">enable</samp> option follows the same rules. For example, to enable a blur filter (smartblur) from 10 seconds to 3 minutes, and a curves filter starting at 3 seconds: <pre class="example-preformatted">smartblur = enable='between(t,10,3*60)', curves = enable='gte(t,3)' : preset=cross_process </pre>

See <code class="code">ffmpeg -filters</code> to view which filters have timeline support.

 <h2 class="chapter">27 Changing options at runtime with a command# TOC</h2>

Some options can be changed during the operation of the filter using a command. These options are marked ’T’ on the output of <code class="command">ffmpeg</code> <samp class="option">-h

filter=<name of filter></samp>. The name of the command is the name of the option and the argument is the new value. <h2 class="chapter">28 Options for filters with several inputs (framesync)# TOC</h2>

Some filters with several inputs support a common set of options. These options can only be set by name, not with the short notation. <dl class="table"> <dt><samp class="option">eof_action</samp></dt> <dd> The action to take when EOF is encountered on the secondary input; it accepts one of the following values: <dl class="table"> <dt><samp class="option">repeat</samp></dt> <dd> Repeat the last frame (the default). </dd> <dt><samp class="option">endall</samp></dt> <dd> End both streams. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass the main input through. </dd> </dl>

</dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> If set to 1, force the filter to extend the last frame of secondary streams until the end of the primary stream. A value of 0 disables this behavior. Default value is 1. </dd> <dt><samp class="option">ts_sync_mode</samp></dt> <dd> How strictly to sync streams based on secondary input timestamps; it accepts one of the following values: <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Frame from secondary input with the nearest lower or equal timestamp to the primary input frame. </dd> <dt><samp class="option">nearest</samp></dt> <dd> Frame from secondary input with the absolute nearest timestamp to the primary input frame. </dd> </dl> </dd> </dl>

 <h2 class="chapter">29 Audio Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class="code">--disable-filters</code>. The configure output will show the audio filters included in your build. Below is a description of the currently available audio filters. <h3 class="section">29.1 aap# TOC</h3> Apply Affine Projection algorithm to the first audio stream using the second

audio stream. This adaptive filter is used to estimate unknown audio based on multiple input audio samples. Affine projection algorithm can make trade-offs between computation complexity with convergence speed. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class="option">projection</samp></dt> <dd> Set the projection order. </dd> <dt><samp class="option">mu</samp></dt> <dd> Set the filter mu. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class="option">out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>. </dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class="section">29.2 acompressor# TOC</h3>

A compressor is mainly used to reduce the dynamic range of a signal. Especially modern music is mostly compressed at a high ratio to improve the overall loudness. It’s done to get the highest attention of a listener, "fatten" the sound and bring more "power" to the track. If a signal is compressed too much it may sound dull or "dead" afterwards or it may start to "pump" (which could be a powerful effect but can also destroy a track completely). The right compression is the key to reach a professional sound and is the high art of mixing and mastering. Because of its complex settings it may take a long time to get the right feeling for this kind of effect. Compression is done by detecting the volume above a chosen level <code class="code">threshold</code> and dividing it by the factor set with <code class="code">ratio</code>.

So if you set the threshold to -12dB and your signal reaches -6dB a ratio of 2:1 will result in a signal at -9dB. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over the time. This is done by setting "Attack" and "Release". <code class="code">attack</code> determines how long the signal has to rise above the threshold before any reduction will occur and <code class="code">release</code> sets the time the signal has to fall below the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. The overall reduction of the signal can be made up afterwards with the <code class="code">makeup</code> setting. So compressing the peaks of a signal about 6dB and raising the makeup to this level results in a signal twice as loud than the source. To gain a softer entry in the compression the <code class="code">knee</code> flattens the hard edge at the threshold in the range of the chosen decibels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal of stream rises above this level it will affect the gain reduction. By default it is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. 1:2 means that if the level rose 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the <code class="code">average</code> level between all channels of input stream or the louder(<code class="code">maximum</code>) channel of input stream affects the reduction. Default is <code class="code">average</code>. </dd> <dt><samp class="option">detection</samp></dt> <dd> Should the exact signal be taken in case of <code class="code">peak</code> or an RMS one in case of <code class="code">rms</code>. Default is <code class="code">rms</code> which is mostly smoother. </dd>

<dt><samp class="option">mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class="subsection">29.2.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.3 acontrast# TOC</h3> Simple audio dynamic range compression/expansion filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set contrast. Default is 33. Allowed range is between 0 and 100. </dd> </dl>

 <h3 class="section">29.4 acopy# TOC</h3>

Copy the input audio source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">29.5 acrossfade# TOC</h3>

Apply cross fade from one input audio stream to another input audio stream. The cross fade is applied for specified duration near the end of first stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs, n</samp></dt> <dd> Specify the number of inputs to crossfade. When crossfading multiple inputs, each input will be concatenated and crossfaded in sequence, similar to the concat filter. Default is 2. </dd> <dt><samp class="option">nb_samples, ns</samp></dt> <dd> Specify the number of samples for which the cross fade effect has to last. At the end of the cross fade effect the first input audio will be completely silent. Default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the cross fade effect. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. By default the duration is determined by <var class="var">nb_samples</var>. If set this option is used instead of <var class="var">nb_samples</var>. </dd> <dt><samp class="option">overlap, o</samp></dt> <dd> Should first stream end overlap with second stream start. Default is enabled.

</dd> <dt><samp class="option">curve1</samp></dt> <dd> Set curve for cross fade transition for first stream. </dd> <dt><samp class="option">curve2</samp></dt> <dd> Set curve for cross fade transition for second stream. For description of available curve types see afade filter description. </dd> </dl>

 <h4 class="subsection">29.5.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input to another: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:c1=exp:c2=exp output.flac </pre>

Cross fade from one input to another but without overlapping: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -filter_complex acrossfade=d=10:o=0:c1=exp:c2=exp output.flac </pre>

Concatenate multiple inputs with cross fade between each: <pre class="example-preformatted">ffmpeg -i first.flac -i second.flac -i third.flac -filter_complex acrossfade=n=3 output.flac </pre>

 <h3 class="section">29.6 acrossover# TOC</h3> Split audio stream into several bands. This filter splits audio stream into two or more frequency ranges. Summing all streams back will give flat output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">split</samp></dt> <dd> Set split frequencies. Those must be positive and increasing. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order for each band split. This controls filter roll-off or steepness of filter transfer function. Available values are: <dl class="table"> <dt>‘<samp class="samp">2nd</samp>’</dt> <dd> 12 dB per octave. </dd> <dt>‘<samp class="samp">4th</samp>’</dt> <dd> 24 dB per octave. </dd> <dt>‘<samp class="samp">6th</samp>’</dt> <dd> 36 dB per octave. </dd> <dt>‘<samp class="samp">8th</samp>’</dt> <dd> 48 dB per octave. </dd> <dt>‘<samp class="samp">10th</samp>’</dt> <dd> 60 dB per octave. </dd> <dt>‘<samp class="samp">12th</samp>’</dt> <dd> 72 dB per octave. </dd> <dt>‘<samp class="samp">14th</samp>’</dt> <dd> 84 dB per octave. </dd> <dt>‘<samp class="samp">16th</samp>’</dt> <dd> 96 dB per octave. </dd> <dt>‘<samp class="samp">18th</samp>’</dt>

<dd> 108 dB per octave. </dd> <dt>‘<samp class="samp">20th</samp>’</dt> <dd> 120 dB per octave. </dd> </dl>

Default is <var class="var">4th</var>. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">gains</samp></dt> <dd> Set output gain for each band. Default value is 1 for all bands. </dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class="option">float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class="option">double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl>

Default value is <code class="code">auto</code>. </dd> </dl>

 <h4 class="subsection">29.6.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Split input audio stream into two bands (low and high) with split frequency of 1500 Hz, each band will be in separate stream: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but with higher filter order: <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500:order=8th[LOW][HIGH]' -map '[LOW]' low.wav -map '[HIGH]' high.wav </pre>

Same as above, but also with additional middle band (frequencies between 1500 and 8000): <pre class="example-preformatted">ffmpeg -i in.flac -filter_complex 'acrossover=split=1500 8000:order=8th[LOW][MID][HIGH]' -map '[LOW]' low.wav -map '[MID]' mid.wav -map '[HIGH]' high.wav </pre>

 <h3 class="section">29.7 acrusher# TOC</h3>

Reduce audio bit resolution. This filter is bit crusher with enhanced functionality. A bit crusher is used to audibly reduce number of bits an audio signal is sampled with. This doesn’t change the bit depth at all, it just produces the effect. Material reduced in bit depth sounds

more harsh and "digital". This filter is able to even round to continuous values instead of discrete bit depths. Additionally it has a D/C offset which results in different crushing of the lower and the upper half of the signal. An Anti-Aliasing setting is able to produce "softer" crushing sounds. Another feature of this filter is the logarithmic mode. This setting switches from linear distances between bits to logarithmic ones. The result is a much more "natural" sounding crusher which doesn’t gate low signals for example. The human ear has a logarithmic perception, so this kind of crushing is much more pleasant. Logarithmic crushing is also able to get anti-aliased. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set level in. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set level out. </dd> <dt><samp class="option">bits</samp></dt> <dd> Set bit reduction. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set mixing amount. </dd> <dt><samp class="option">mode</samp></dt> <dd> Can be linear: <code class="code">lin</code> or logarithmic: <code class="code">log</code>. </dd> <dt><samp class="option">dc</samp></dt> <dd> Set DC. </dd> <dt><samp class="option">aa</samp></dt> <dd> Set anti-aliasing. </dd> <dt><samp class="option">samples</samp></dt> <dd> Set sample reduction. </dd> <dt><samp class="option">lfo</samp></dt> <dd> Enable LFO. By default disabled. </dd> <dt><samp class="option">lforange</samp></dt> <dd> Set LFO range. </dd> <dt><samp class="option">lforate</samp></dt> <dd> Set LFO rate. </dd> </dl>

 <h4 class="subsection">29.7.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.8 acue# TOC</h3>

Delay audio filtering until a given wallclock timestamp. See the cue filter. <h3 class="section">29.9 adeclick# TOC</h3> Remove impulsive noise from input audio. Samples detected as impulsive noise are replaced by interpolated samples using autoregressive modelling. <dl class="table"> <dt><samp class="option">window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range

is from <code class=”code”>10</code> to <code class=”code”>100</code>. Default value is <code class=”code”>55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class=”option”>overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class=”code”>50</code> to <code class=”code”>95</code>. Default value is <code class=”code”>75</code> percent. Setting this to a very high value increases impulsive noise removal but makes whole process much slower. </dd> <dt><samp class=”option”>arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class=”code”>0</code> to <code class=”code”>25</code>. Default value is <code class=”code”>2</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class=”code”>1</code> to <code class=”code”>100</code>. Default value is <code class=”code”>2</code>. This controls the strength of impulsive noise which is going to be removed. The lower value, the more samples will be detected as impulsive noise. </dd> <dt><samp class=”option”>burst, b</samp></dt> <dd> Set burst fusion, in percentage of window size. Allowed range is <code class=”code”>0</code> to <code class=”code”>10</code>. Default value is <code class=”code”>2</code>. If any two samples detected as noise are spaced less than this value then any sample between those two samples will be also detected as noise. </dd> <dt><samp class=”option”>method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class=”option”>save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class=”code”>a</code>. </dd> </dl>

 <h3 class=”section”>29.10 adeclip# TOC</h3> Remove clipped samples from input audio. Samples detected as clipped are replaced by interpolated samples using autoregressive modelling. <dl class=”table”> <dt><samp class=”option”>window, w</samp></dt> <dd> Set window size, in milliseconds. Allowed range is from <code class=”code”>10</code> to <code class=”code”>100</code>. Default value is <code class=”code”>55</code> milliseconds. This sets size of window which will be processed at once. </dd> <dt><samp class=”option”>overlap, o</samp></dt> <dd> Set window overlap, in percentage of window size. Allowed range is from <code class=”code”>50</code> to <code class=”code”>95</code>. De-

fault value is <code class=”code”>75</code> percent. </dd> <dt><samp class=”option”>arorder, a</samp></dt> <dd> Set autoregression order, in percentage of window size. Allowed range is from <code class=”code”>0</code> to <code class=”code”>25</code>. Default value is <code class=”code”>8</code> percent. This option also controls quality of interpolated samples using neighbour good samples. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set threshold value. Allowed range is from <code class=”code”>1</code> to <code class=”code”>100</code>. Default value is <code class=”code”>10</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class=”option”>hsize, n</samp></dt> <dd> Set size of histogram used to detect clips. Allowed range is from <code class=”code”>100</code> to <code class=”code”>9999</code>. Default value is <code class=”code”>1000</code>. Higher values make clip detection less aggressive. </dd> <dt><samp class=”option”>method, m</samp></dt> <dd> Set overlap method. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>add, a</samp></dt> <dd> Select overlap-add method. Even not interpolated samples are slightly changed with this method. </dd> <dt><samp class=”option”>save, s</samp></dt> <dd> Select overlap-save method. Not interpolated samples remain unchanged. </dd> </dl>

Default value is <code class=”code”>a</code>. </dd> </dl>

 <h3 class=”section”>29.11 adecorrelate# TOC</h3> Apply decorrelation to input audio stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stages</samp></dt> <dd> Set decorrelation stages of filtering. Allowed range is from 1 to 16. Default value is 6. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set random seed used for setting delay in samples across channels. </dd> </dl>

 <h3 class=”section”>29.12 adelay# TOC</h3>

Delay one or more audio channels. Samples in delayed channel are filled with silence. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>delays</samp></dt> <dd> Set list of delays in milliseconds for each channel separated by ’|’. Unused delays will be silently ignored. If number of given delays is smaller than number of channels all remaining channels will not be delayed. If you want to delay exact number of samples, append ’S’ to number. If you want instead to delay in seconds, append ’s’ to number. </dd> <dt><samp class=”option”>all</samp></dt>

<dd> Use last set delay for all remaining channels. By default is disabled. This option if enabled changes how option <code class="code">delays</code> is interpreted. </dd> </dl>

 <h4 class="subsection">29.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Delay first channel by 1.5 seconds, the third channel by 0.5 seconds and leave the second channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=1500|0|500 </pre>

Delay second channel by 500 samples, the third channel by 700 samples and leave the first channel (and any other channels that may be present) unchanged. <pre class="example-preformatted">adelay=0|500S|700S </pre>

Delay all channels by same number of samples: <pre class="example-preformatted">adelay=delays=64S:all=1 </pre>

 <h3 class="section">29.13 adenorm# TOC</h3> Remedy denormals in audio by adding extremely low-level noise. This filter shall be placed before any filter that can produce denormals. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">level</samp></dt> <dd> Set level of added noise in dB. Default is <code class="code">-351</code>. Allowed range is from -451 to -90. </dd> <dt><samp class="option">type</samp></dt> <dd> Set type of added noise. <dl class="table"> <dt><samp class="option">dc</samp></dt> <dd> Add DC signal. </dd> <dt><samp class="option">ac</samp></dt> <dd> Add AC signal. </dd> <dt><samp class="option">square</samp></dt> <dd> Add square signal. </dd> <dt><samp class="option">pulse</samp></dt> <dd> Add pulse signal. </dd> </dl>

Default is <code class="code">dc</code>. </dd> </dl>

 <h4 class="subsection">29.13.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.14

aderivative, aintegral# TOC</h3>

Compute derivative/integral of audio stream. Applying both filters one after another produces original audio. <h3 class="section">29.15 adrc# TOC</h3>

Apply spectral dynamic range controller filter to input audio stream. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">transfer</samp></dt> <dd> Set the transfer expression. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> current channel number </dd> <dt><samp class="option">sn</samp></dt> <dd> current sample number </dd> <dt><samp class="option">nb_channels</samp></dt> <dd> number of channels </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds </dd> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">p</samp></dt> <dd> current frequency power value, in dB </dd> <dt><samp class="option">f</samp></dt> <dd> current frequency in Hz </dd> </dl>

Default value is <code class="code">p</code>. </dd> <dt><samp class="option">attack</samp></dt> <dd> Set the attack in milliseconds. Default is <code class="code">50</code> milliseconds. Allowed range is from 1 to 1000 milliseconds. </dd> <dt><samp class="option">release</samp></dt> <dd> Set the release in milliseconds. Default is <code class="code">100</code> milliseconds. Allowed range is from 5 to 2000 milliseconds. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set which channels to filter, by default <code class="code">all</code> channels in audio stream are filtered. </dd> </dl>

 <h4 class="subsection">29.15.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">29.15.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply spectral compression to all

frequencies with threshold of -50 dB and 1:6 ratio: <pre class=”example-preformatted”>adrc=transfer='if(gt(p,-50),-50+(p-(-50))/6,p)':attack=50:release=100 </pre>

Similar to above but with 1:2 ratio and filtering only front center channel: <pre class=”example-preformatted”>adrc=transfer='if(gt(p,-50),-50+(p-(-50))/2,p)':attack=50:release=100:channels=FC </pre>

Apply spectral noise gate to all frequencies with threshold of -85 dB and with short attack time and short release time: <pre class=”example-preformatted”>adrc=transfer='if(lte(p,-85),p-800,p)':attack=1:release=5 </pre>

Apply spectral expansion to all frequencies with threshold of -10 dB and 1:2 ratio: <pre class=”example-preformatted”>adrc=transfer='if(lt(p,-10),-10+(p-(-10))*2,p)':attack=50:release=100 </pre>

Apply limiter to max -60 dB to all frequencies, with attack of 2 ms and release of 10 ms: <pre class=”example-preformatted”>adrc=transfer='min(p,-60)':attack=2:release=10 </pre>

 <h3 class=”section”>29.16 adynamicequalizer# TOC</h3>

Apply dynamic equalization to input audio stream.

A description of the accepted options follows.

<dl class=”table”>

<dt><samp class=”option”>threshold</samp></dt>

<dd> Set the detection threshold used to trigger equalization.

Threshold detection is using detection filter.

Default value is 0. Allowed range is from 0 to 100.

</dd>

<dt><samp class=”option”>dfrequency</samp></dt>

<dd> Set the detection frequency in Hz used for detection filter used to trigger equalization.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">dqfactor</samp></dt>

<dd> Set the detection resonance factor for detection filter used to trigger equalization.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">tfrequency</samp></dt>

<dd> Set the target frequency of equalization filter.

Default value is 1000 Hz. Allowed range is between 2 and 1000000 Hz.

</dd>

<dt><samp class="option">tqfactor</samp></dt>

<dd> Set the target resonance factor for target equalization filter.

Default value is 1. Allowed range is from 0.001 to 1000.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to rise above

the detection threshold before equalization starts.

Default is 20. Allowed range is between 1 and 2000.

</dd>

<dt><samp class="option">release</samp></dt>

<dd> Set the amount of milliseconds the signal from detection has to fall below the

detection threshold before equalization ends.

Default is 200. Allowed range is between 1 and 2000.

</dd>
<dt><samp class="option">ratio</samp></dt>
<dd> Set the ratio by which the equalization gain is raised.
Default is 1. Allowed range is between 0 and 30.

</dd>
<dt><samp class="option">makeup</samp></dt>
<dd> Set the makeup offset by which the equalization gain is raised.
Default is 0. Allowed range is between 0 and 100.

</dd>
<dt><samp class="option">range</samp></dt>
<dd> Set the max allowed cut/boost amount. Default is 50.
Allowed range is from 1 to 200.

</dd>
<dt><samp class="option">mode</samp></dt>
<dd> Set the mode of filter operation, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">listen</samp>’</dt>
<dd> Output only isolated detection signal.
</dd>
<dt>‘<samp class="samp">cutbelow</samp>’</dt>
<dd> Cut frequencies below detection threshold.
</dd>
<dt>‘<samp class="samp">cutabove</samp>’</dt>
<dd> Cut frequencies above detection threshold.
</dd>
<dt>‘<samp class="samp">boostbelow</samp>’</dt>
<dd> Boost frequencies below detection threshold.

</dd>
<dt>‘<samp class="samp">boostabove</samp>’</dt>
<dd> Boost frequencies above detection threshold.
</dd>
</dl>
Default mode is ‘<samp class="samp">cutbelow</samp>’.

</dd>
<dt><samp class="option">dftype</samp></dt>
<dd> Set the type of detection filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bandpass</samp>’</dt>
<dt>‘<samp class="samp">lowpass</samp>’</dt>
<dt>‘<samp class="samp">highpass</samp>’</dt>
<dt>‘<samp class="samp">peak</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bandpass</samp>’.

</dd>
<dt><samp class="option">tftype</samp></dt>
<dd> Set the type of target filter, can be one of the following:

<dl class="table">
<dt>‘<samp class="samp">bell</samp>’</dt>
<dt>‘<samp class="samp">lowshelf</samp>’</dt>
<dt>‘<samp class="samp">highshelf</samp>’</dt>
</dl>
Default type is ‘<samp class="samp">bell</samp>’.

</dd>

<dt><samp class=”option”>auto</samp></dt>

<dd> Automatically gather threshold from detection filter. By default

is ‘<samp class=”samp”>disabled</samp>’.

This option is useful to detect threshold in certain time frame of

input audio stream, in such case option value is changed at runtime.

Available values are:

<dl class=”table”>

<dt>‘<samp class=”samp”>disabled</samp>’</dt>

<dd> Disable using automatically gathered threshold value.

</dd>

<dt>‘<samp class=”samp”>off</samp>’</dt>

<dd> Stop picking threshold value.

</dd>

<dt>‘<samp class=”samp”>on</samp>’</dt>

<dd> Start picking threshold value.

</dd>

<dt>‘<samp class=”samp”>adaptive</samp>’</dt>

<dd> Adaptively pick threshold value, by calculating sliding window entropy.

</dd>

</dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>29.16.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>29.17 adynamicsmooth# TOC</h3>

Apply dynamic smoothing to input audio stream. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>sensitivity</samp></dt> <dd> Set an amount of sensitivity to frequency fluctations. Default is 2. Allowed range is from 0 to 1e+06. </dd> <dt><samp class=”option”>basefreq</samp></dt> <dd> Set a base frequency for smoothing. Default value is 22050. Allowed range is from 2 to 1e+06. </dd> </dl>

 <h4 class=”subsection”>29.17.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>29.18 aecho# TOC</h3>

Apply echoing to the input audio. Echoes are reflected sound and can occur naturally amongst mountains (and sometimes large buildings) when talking or shouting; digital echo effects emulate this behaviour and are often used to help fill out the sound of a single instrument or vocal. The time difference between the original signal and the reflection is the <code class=”code”>delay</code>, and the loudness of the reflected signal is the <code class=”code”>decay</code>. Multiple echoes can have different delays and decays. A description of the accepted parameters follows. <dl class=”table”> <dt><samp class=”option”>in_gain</samp></dt> <dd> Set input gain of reflected signal. Default is <code class=”code”>0.6</code>. </dd> <dt><samp class=”option”>out_gain</samp></dt> <dd> Set output gain of reflected signal. Default is <code class=”code”>0.3</code>. </dd> <dt><samp class=”option”>delays</samp></dt> <dd> Set list of time intervals in milliseconds between original signal and reflections separated by ’|’. Allowed range for each <code class=”code”>delay</code> is <code class=”code”>(0 - 90000.0]</code>. Default is <code class=”code”>1000</code>. </dd> <dt><samp class=”option”>decays</samp></dt> <dd> Set list of loudness of reflected signals separated by ’|’. Allowed range for each <code class=”code”>decay</code> is <code class=”code”>(0 - 1.0]</code>. Default is <code class=”code”>0.5</code>. </dd> </dl>

 <h4 class="subsection">29.18.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make it sound as if there are twice as many instruments as are actually playing: <pre class="example-preformatted">aecho=0.8:0.88:60:0.4 </pre>

If delay is very short, then it sounds like a (metallic) robot playing music: <pre class="example-preformatted">aecho=0.8:0.88:6:0.4 </pre>

A longer delay will sound like an open air concert in the mountains: <pre class="example-preformatted">aecho=0.8:0.9:1000:0.3 </pre>

Same as above but with one more mountain: <pre class="example-preformatted">aecho=0.8:0.9:1000|1800:0.3|0.25 </pre>

 <h3 class="section">29.19 aemphasis# TOC</h3> Audio emphasis filter creates or restores material directly taken from LPs or emphased CDs with different filter curves. E.g. to store music on vinyl the signal has to be altered by a filter first to even out the disadvantages of this recording medium. Once the material is played back the inverse filter has to be applied to restore the distortion of the frequency response. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set filter mode. For restoring material use <code class="code">reproduction</code> mode, otherwise use <code class="code">production</code> mode. Default is <code class="code">reproduction</code> mode. </dd> <dt><samp class="option">type</samp></dt> <dd> Set filter type. Selects medium. Can be one of the following: <dl class="table"> <dt><samp class="option">col</samp></dt> <dd> select Columbia. </dd> <dt><samp class="option">emi</samp></dt> <dd> select EMI. </dd> <dt><samp class="option">bsi</samp></dt> <dd> select BSI (78RPM). </dd> <dt><samp class="option">riaa</samp></dt> <dd> select RIAA. </dd> <dt><samp class="option">cd</samp></dt> <dd> select Compact Disc (CD). </dd> <dt><samp class="option">50fm</samp></dt> <dd> select 50µs (FM). </dd> <dt><samp class="option">75fm</samp></dt> <dd> select 75µs (FM). </dd> <dt><samp class="option">50kf</samp></dt> <dd> select 50µs (FM-KF). </dd> <dt><samp class="option">75kf</samp></dt>

<dd> select 75µs (FM-KF). </dd> </dl> </dd> </dl>

 <h4 class="subsection">29.19.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.20 aeval# TOC</h3>

Modify an audio signal according to the specified expressions. This filter accepts one or more expressions (one for each channel), which are evaluated and used to modify a corresponding audio signal. It accepts the following parameters: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set output channel layout. If not specified, the channel layout is specified by the number of expressions. If set to ‘<samp class="samp">same</samp>’, it will use by default the same input channel layout. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">ch</samp></dt> <dd> channel number of the current expression </dd> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds </dd> <dt><samp class="option">nb_in_channels</samp></dt> <dt><samp class="option">nb_out_channels</samp><
<dd> input and output number of channels </dd> <dt><samp class="option">val(CH)</samp></dt> <dd> the value of input channel with number <var class="var">CH</var> </dd> </dl>

Note: this filter is slow. For faster processing you should use a dedicated filter. <h4 class="subsection">29.20.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Half volume: <pre class="example-preformatted">aeval=val(ch)/2:c=same </pre>

Invert phase of the second channel:

<pre class="example-preformatted">aeval=val(0)|-val(1) </pre>

 <h3 class="section">29.21 aexciter# TOC</h3>

An exciter is used to produce high sound that is not present in the original signal. This is done by creating harmonic distortions of the signal which are restricted in range and added to the original signal. An Exciter raises the upper end of an audio signal without simply raising the higher frequencies like an equalizer would do to create a more "crisp" or "brilliant" sound. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input level prior processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output level after processing of signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set the amount of harmonics added to original signal. Allowed range is from 0 to 64. Default value is 1. </dd> <dt><samp class="option">drive</samp></dt> <dd> Set the amount of newly created harmonics. Allowed range is from 0.1 to 10. Default value is 8.5. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set the octave of newly created harmonics. Allowed range is from -10 to 10. Default value is 0. </dd> <dt><samp class="option">freq</samp></dt> <dd> Set the lower frequency limit of producing harmonics in Hz. Allowed range is from 2000 to 12000 Hz. Default is 7500 Hz. </dd> <dt><samp class="option">ceil</samp></dt> <dd> Set the upper frequency limit of producing harmonics. Allowed range is from 9999 to 20000 Hz. If value is lower than 10000 Hz no limit is applied. </dd> <dt><samp class="option">listen</samp></dt> <dd> Mute the original signal and output only added harmonics. By default is disabled. </dd> </dl>

 <h4 class="subsection">29.21.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.22 afade# TOC</h3>

Apply fade-in/out effect to input audio.

A description of the accepted parameters follows.

<dl class="table">

<dt><samp class="option">type, t</samp></dt>

<dd> Specify the effect type, can be either <code class="code">in</code> for fade-in, or

<code class="code">out</code> for a fade-out effect. Default is <code class="code">in</code>.

</dd>

<dt><samp class="option">start_sample, ss</samp></dt>

<dd> Specify the number of the start sample for starting to apply the fade

effect. Default is 0.

</dd>

<dt><samp class="option">nb_samples, ns</samp></dt>

<dd> Specify the number of samples for which the fade effect has to last. At

the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence. Default is 44100.

</dd>

<dt><samp class="option">start_time, st</samp></dt>

<dd> Specify the start time of the fade effect. Default is 0.

The value must be specified as a time duration; see

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If set this option is used instead of <var class="var">start_sample</var>.

</dd>

<dt><samp class="option">duration, d</samp></dt>

<dd> Specify the duration of the fade effect. See

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

At the end of the fade-in effect the output audio will have the same

volume as the input audio, at the end of the fade-out transition

the output audio will be silence.

By default the duration is determined by <var class="var">nb_samples</var>.

If set this option is used instead of <var class="var">nb_samples</var>.

</dd>

<dt><samp class="option">curve</samp></dt>

<dd> Set curve for fade transition.

It accepts the following values:

<dl class="table">

<dt><samp class="option">tri</samp></dt>

<dd> select triangular, linear slope (default)

</dd>

<dt><samp class="option">qsin</samp></dt>

<dd> select quarter of sine wave

</dd>

<dt><samp class="option">hsin</samp></dt>

<dd> select half of sine wave

</dd>

<dt><samp class="option">esin</samp></dt>

<dd> select exponential sine wave

</dd>

<dt><samp class="option">log</samp></dt>

<dd> select logarithmic

</dd>
<dt><samp class="option">ipar</samp></dt>
<dd> select inverted parabola
</dd>
<dt><samp class="option">qua</samp></dt>
<dd> select quadratic
</dd>
<dt><samp class="option">cub</samp></dt>
<dd> select cubic
</dd>
<dt><samp class="option">squ</samp></dt>
<dd> select square root
</dd>
<dt><samp class="option">cbr</samp></dt>
<dd> select cubic root
</dd>
<dt><samp class="option">par</samp></dt>
<dd> select parabola
</dd>
<dt><samp class="option">exp</samp></dt>
<dd> select exponential
</dd>
<dt><samp class="option">iqsin</samp></dt>
<dd> select inverted quarter of sine wave
</dd>
<dt><samp class="option">ihsin</samp></dt>
<dd> select inverted half of sine wave
</dd>
<dt><samp class="option">dese</samp></dt>
<dd> select double-exponential seat
</dd>

<dt><samp class="option">desi</samp></dt>

<dd> select double-exponential sigmoid

</dd>

<dt><samp class="option">losi</samp></dt>

<dd> select logistic sigmoid

</dd>

<dt><samp class="option">sinc</samp></dt>

<dd> select sine cardinal function

</dd>

<dt><samp class="option">isinc</samp></dt>

<dd> select inverted sine cardinal function

</dd>

<dt><samp class="option">quat</samp></dt>

<dd> select quartic

</dd>

<dt><samp class="option">quatr</samp></dt>

<dd> select quartic root

</dd>

<dt><samp class="option">qsin2</samp></dt>

<dd> select squared quarter of sine wave

</dd>

<dt><samp class="option">hsin2</samp></dt>

<dd> select squared half of sine wave

</dd>

<dt><samp class="option">nofade</samp></dt>

<dd> no fade applied

</dd>

</dl>

</dd> <dt><samp class="option">silence</samp></dt> <dd> Set the initial gain for fade-in or final gain for fade-out. Default value is <code class="code">0.0</code>. </dd> <dt><samp class="option">unity</samp></dt>

<dd> Set the initial gain for fade-out or final gain for fade-in. Default value is <code class="code">1.0</code>. </dd> </dl>

 <h4 class="subsection">29.22.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">29.22.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in first 15 seconds of audio: <pre class="example-preformatted">afade=t=in:ss=0:d=15 </pre>

Fade out last 25 seconds of a 900 seconds audio: <pre class="example-preformatted">afade=t=out:st=875:d=25 </pre>

 <h3 class="section">29.23 afftdn# TOC</h3> Denoise audio samples with FFT. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">noise_reduction, nr</samp></dt> <dd> Set the noise reduction in dB, allowed range is 0.01 to 97. Default value is 12 dB. </dd> <dt><samp class="option">noise_floor, nf</samp></dt> <dd> Set the noise floor in dB, allowed range is -80 to -20. Default value is -50 dB. </dd> <dt><samp class="option">noise_type, nt</samp></dt> <dd> Set the noise type. It accepts the following values: <dl class="table"> <dt><samp class="option">white, w</samp></dt> <dd> Select white noise. </dd> <dt><samp class="option">vinyl, v</samp></dt> <dd> Select vinyl noise. </dd> <dt><samp class="option">shellac, s</samp></dt> <dd> Select shellac noise. </dd> <dt><samp class="option">custom, c</samp></dt> <dd> Select custom noise, defined in <code class="code">bn</code> option. Default value is white noise. </dd> </dl>

</dd> <dt><samp class="option">band_noise, bn</samp></dt> <dd> Set custom band noise profile for every one of 15 bands. Bands are separated by ’ ’ or ’|’. </dd> <dt><samp class="option">residual_floor, rf</samp></dt> <dd> Set the residual floor in dB, allowed range is -80 to -20. Default value is -38 dB. </dd> <dt><samp class="option">track_noise, tn</samp></dt> <dd> Enable noise floor tracking. By default is disabled. With this enabled, noise floor is automatically adjusted. </dd> <dt><samp

class=”option”>track_residual, tr</samp></dt> <dd> Enable residual tracking. By default is disabled. </dd> <dt><samp class=”option”>output_mode, om</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>input, i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class=”option”>output, o</samp></dt> <dd> Pass noise filtered out. </dd> <dt><samp class=”option”>noise, n</samp></dt> <dd> Pass only noise. Default value is <var class=”var”>output</var>. </dd> </dl>

</dd> <dt><samp class=”option”>adaptivity, ad</samp></dt> <dd> Set the adaptivity factor, used how fast to adapt gains adjustments per each frequency bin. Value <var class=”var”>0</var> enables instant adaptation, while higher values react much slower. Allowed range is from <var class=”var”>0</var> to <var class=”var”>1</var>. Default value is <var class=”var”>0.5</var>. </dd> <dt><samp class=”option”>floor_offset, fo</samp></dt> <dd> Set the noise floor offset factor. This option is used to adjust offset applied to measured noise floor. It is only effective when noise floor tracking is enabled. Allowed range is from <var class=”var”>-2.0</var> to <var class=”var”>2.0</var>. Default value is <var class=”var”>1.0</var>. </dd> <dt><samp class=”option”>noise_link, nl</samp></dt> <dd> Set the noise link used for multichannel audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>none</samp></dt> <dd> Use unchanged channel’s noise floor. </dd> <dt><samp class=”option”>min</samp></dt> <dd> Use measured min noise floor of all channels. </dd> <dt><samp class=”option”>max</samp></dt> <dd> Use measured max noise floor of all channels. </dd> <dt><samp class=”option”>average</samp></dt> <dd> Use measured average noise floor of all channels. Default value is <var class=”var”>min</var>. </dd> </dl>

</dd> <dt><samp class=”option”>band_multiplier, bm</samp></dt> <dd> Set the band multiplier factor, used how much to spread bands across frequency bins. Allowed range is from <var class=”var”>0.2</var> to <var class=”var”>5</var>. Default value is <var class=”var”>1.25</var>. </dd> <dt><samp class=”option”>sample_noise, sn</samp></dt> <dd> Toggle capturing and measurement of noise profile from input audio. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>start, begin</samp></dt> <dd> Start sample noise capture. </dd> <dt><samp class=”option”>stop, end</samp></dt> <dd> Stop sample noise capture and measure new noise band profile. Default value is <code class=”code”>none</code>. </dd> </dl>

</dd> <dt><samp class=”option”>gain_smooth, gs</samp></dt>

<dd> Set gain smooth spatial radius, used to smooth gains applied to each frequency bin. Useful to reduce random music noise artefacts. Higher values increases smoothing of gains. Allowed range is from <code class="code">0</code> to <code class="code">50</code>. Default value is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">29.23.1 Commands# TOC</h4>

This filter supports the some above mentioned options as commands. <h4 class="subsection">29.23.2 Examples#
TOC</h4>

<ul class="itemize mark-bullet"> Reduce white noise by 10dB, and use previously measured noise floor of -40dB: <pre class="example-preformatted">afftdn=nr=10:nf=-40 </pre>

Reduce white noise by 10dB, also set initial noise floor to -80dB and enable automatic tracking of noise floor so noise floor will gradually change during processing: <pre class="example-preformatted">afftdn=nr=10:nf=-80:tn=1 </pre>

Reduce noise by 20dB, using noise floor of -40dB and using commands to take noise profile of first 0.4 seconds of input audio: <pre class="example-preformatted">asendcmd=0.0 afftdn sn start,asendcmd=0.4 afftdn sn stop,afftdn=nr=20:nf=-40 </pre>

 <h3 class="section">29.24 afftfilt#
TOC</h3>
Apply arbitrary expressions to samples in frequency domain.
<dl class="table"> <dt><samp class="option">real</samp></dt>
<dd> Set frequency domain real expression for each separate channel separated by ’|’. Default is "re". If the number of input channels is greater than the number of expressions, the last specified expression is used for the remaining output channels. </dd> <dt><samp class="option">imag</samp></dt> <dd> Set frequency domain imaginary expression for each separate channel separated by ’|’. Default is "im". Each expression in <var class="var">real</var> and <var class="var">imag</var> can contain the following constants and functions: <dl class="table"> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">b</samp></dt> <dd> current frequency bin number </dd> <dt><samp class="option">nb</samp></dt>

<dd> number of available bins </dd> <dt><samp class="option">ch</samp></dt>
<dd> channel number of the current expression </dd> <dt><samp
class="option">chs</samp></dt> <dd> number of channels
</dd> <dt><samp class="option">pts</samp></dt> <dd> current
frame pts </dd> <dt><samp class="option">re</samp></dt>
<dd> current real part of frequency bin of current channel </dd>
<dt><samp class="option">im</samp></dt> <dd> current imaginary part of frequency bin of current channel </dd> <dt><samp
class="option">real(b, ch)</samp></dt> <dd> Return the value of
real part of frequency bin at location (<var class="var">bin</var>,<var
class="var">channel</var>) </dd> <dt><samp class="option">imag(b,
ch)</samp></dt> <dd> Return the value of imaginary part of frequency
bin at location (<var class="var">bin</var>,<var class="var">channel</var>)
</dd> </dl>

</dd> <dt><samp class="option">win_size</samp></dt> <dd> Set
window size. Allowed range is from 16 to 131072. Default is <code
class="code">4096</code> </dd> <dt><samp class="option">win_func</samp></dt>
<dd> Set window function. It accepts the following values:
<dl class="table"> <dt>‘<samp class="samp">rect</samp>’</dt>
<dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp
class="samp">hann, hanning</samp>’</dt> <dt>‘<samp
class="samp">hamming</samp>’</dt> <dt>‘<samp
class="samp">blackman</samp>’</dt> <dt>‘<samp
class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp
class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp
class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp
class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp
class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp
class="samp">kaiser</samp>’</dt> </dl> Default is <code
class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt>
<dd> Set window overlap. If set to 1, the recommended overlap for selected
window function will be picked. Default is <code class="code">0.75</code>.
</dd> </dl>

 <h4 class="subsection">29.24.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Leave almost only low frequencies in audio: <pre class="example-preformatted">afftfilt="'real=re

* (1-clip((b/nb)*b,0,1))':imag='im * (1-clip((b/nb)*b,0,1))'" </pre>

Apply robotize effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*sin(0)':imag='hypot(re,im)*cos(0)':win_size=512:overlap=
</pre>

Apply whisper effect: <pre class="example-preformatted">afftfilt="real='hypot(re,im)*cos((random(0)*2-1)*2*3.14)':imag='hypot(re,im)*sin((ran
1)*2*3.14)':win_size=128:overlap=0.8" </pre>

Apply phase shift: <pre class="example-preformatted">afftfilt="real=re*cos(1)-im*sin(1):imag=re*sin(1)+im*cos(1)"
</pre>

 <h3 class="section">29.25 afir# TOC</h3>

Apply an arbitrary Finite Impulse Response filter.

This filter is designed for applying long FIR filters,

up to 60 seconds long.

It can be used as component for digital crossover filters,

room equalization, cross talk cancellation, wavefield synthesis,

auralization, ambiophonics, ambisonics and spatialization.

This filter uses the streams higher than first one as FIR coefficients.

If the non-first stream holds a single channel, it will be used

for all input channels in the first stream, otherwise

the number of channels in the non-first stream must be same as

the number of channels in the first stream.

It accepts the following parameters:

<dl class="table">

<dt><samp class="option">dry</samp></dt>

<dd> Set dry gain. This sets input gain.

</dd>

<dt><samp class="option">wet</samp></dt>

<dd> Set wet gain. This sets final output gain.

</dd>

<dt><samp class="option">length</samp></dt>

<dd> Set Impulse Response filter length. Default is 1, which means whole IR is processed.

</dd>

<dt><samp class="option">gtype</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">irnorm</samp></dt>

<dd> Set norm to be applied to IR coefficients before filtering.

Allowed range is from <var class="var">-1</var> to <var class="var">2</var>.

IR coefficients are normalized with calculated vector norm set by this option.

For negative values, no norm is calculated, and IR coefficients are not modified at all.

Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">irlink</samp></dt>

<dd> For multichannel IR if this option is set to <var class="var">true</var>, all IR channels will be

normalized with maximal measured gain of all IR channels coefficients as set by <code class="code">irnorm</code> option.

When disabled, all IR coefficients in each IR channel will be normalized independently.

Default is <var class="var">true</var>.

</dd>

<dt><samp class="option">irgain</samp></dt>

<dd> Set gain to be applied to IR coefficients before filtering.

Allowed range is 0 to 1. This gain is applied after any gain applied with <var class="var">irnorm</var> option.

</dd>

<dt><samp class="option">irfmt</samp></dt>

<dd> Set format of IR stream. Can be <code class="code">mono</code> or <code class="code">input</code>.

Default is <code class="code">input</code>.

</dd>

<dt><samp class="option">maxir</samp></dt>

<dd> Set max allowed Impulse Response filter duration in seconds. Default is 30 seconds.

Allowed range is 0.1 to 60 seconds.

</dd>

<dt><samp class="option">response</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">channel</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">size</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">rate</samp></dt>

<dd> This option is deprecated, and does nothing.

</dd>

<dt><samp class="option">minp</samp></dt>

<dd> Set minimal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">1</var> to <var class="var">65536</var>.

Lower values decreases latency at cost of higher CPU usage.

</dd>

<dt><samp class="option">maxp</samp></dt>

<dd> Set maximal partition size used for convolution. Default is <var class="var">8192</var>.

Allowed range is from <var class="var">8</var> to <var class="var">65536</var>.

Lower values may increase CPU usage.

</dd>

<dt><samp class="option">nbirs</samp></dt>

<dd> Set number of input impulse responses streams which will be switchable at runtime.

Allowed range is from <var class="var">1</var> to <var class="var">32</var>. Default is <var class="var">1</var>.

</dd>

<dt><samp class="option">ir</samp></dt>

<dd> Set IR stream which will be used for convolution, starting from <var class="var">0</var>, should always be

lower than supplied value by <code class="code">nbirs</code> option. Default is <var class="var">0</var>.

This option can be changed at runtime via commands.

</dd>

<dt><samp class="option">precision</samp></dt>

<dd> Set which precision to use when processing samples.

<dl class="table">

<dt><samp class="option">auto</samp></dt>

<dd> Auto pick internal sample format depending on other filters.

</dd>

<dt><samp class="option">float</samp></dt>

<dd> Always use single-floating point precision sample format.

</dd>

<dt><samp class="option">double</samp></dt>

<dd> Always use double-floating point precision sample format.

</dd>

</dl>

Default value is auto. </dd> <dt><samp class="option">irload</samp></dt>
<dd> Set when to load IR stream. Can be <code class="code">init</code>
or <code class="code">access</code>. First one load and prepares all IRs
on initialization, second one once on first access of specific IR. Default is <code
class="code">init</code>. </dd> </dl>

 <h4 class="subsection">29.25.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply reverb to stream using
mono IR file as second input, complete command using ffmpeg: <div
class="example"> <pre class="example-preformatted">ffmpeg -i input.wav -i
middle_tunnel_1way_mono.wav -lavfi afir output.wav </pre>

Apply true stereo processing given input stereo stream, and
two stereo impulse responses for left and right channel, the impulse response files are files with names l_ir.wav and r_ir.wav, and setting
irnorm option value: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR|c3=FR[a];amovie=l_ir.wav[LIR];amovie=r_ir.wav[RIR
</pre>

Similar to above example, but with <code class="code">irgain</code> explicitly set to estimated value and with <code class="code">irnorm</code> disabled: <pre class="example-preformatted">"pan=4C|c0=FL|c1=FL|c2=FR 5dB:irnom=-1,pan=stereo|FL<c0+c2|FR<c1+c3" </pre>

 <h3 class="section">29.26 aformat# TOC</h3>

Set output format constraints for the input audio. The framework will negotiate the most appropriate format to minimize conversions. It accepts the following parameters: <dl class="table"> <dt><samp class="option">sample_fmts, f</samp></dt> <dd> A ’|’-separated list of requested sample formats. </dd> <dt><samp class="option">sample_rates, r</samp></dt> <dd> A ’|’-separated list of requested sample rates. </dd> <dt><samp class="option">channel_layouts, cl</samp></dt> <dd> A ’|’-separated list of requested channel layouts. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual for the required syntax. </dd> </dl>

If a parameter is omitted, all values are allowed. Force the output to either unsigned 8-bit or signed 16-bit stereo <pre class="example-preformatted">aformat=sample_fmts=u8|s16:channel_layouts=stereo </pre>

 <h3 class="section">29.27 afreqshift# TOC</h3> Apply frequency shift to input audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Specify frequency shift. Allowed range is -INT_MAX to INT_MAX. Default value is 0.0. </dd> <dt><samp class="option">level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">29.27.1 Commands# TOC</h4>

This filter supports the all above options as commands.

  <h3 class="section">29.28 afwtdn# TOC</h3> Reduce broadband noise from input samples using Wavelets. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma, allowed range is from 0 to 1. Default value is 0. This option controls strength of denoising applied to input samples. Most useful way to set this option is via decibels, eg. -45dB. </dd> <dt><samp class="option">levels</samp></dt> <dd> Set the number of wavelet levels of decomposition. Allowed range is from 1 to 12. Default value is 10. Setting this too low make denoising performance very poor. </dd> <dt><samp class="option">wavet</samp></dt> <dd> Set wavelet type for decomposition of input frame. They are sorted by number of coefficients, from lowest to highest. More coefficients means worse filtering speed, but overall better quality. Available wavelets are: <dl class="table"> <dt>‘<samp class="samp">sym2</samp>’</dt> <dt>‘<samp class="samp">sym4</samp>’</dt> <dt>‘<samp class="samp">rbior68</samp>’</dt> <dt>‘<samp class="samp">deb10</samp>’</dt> <dt>‘<samp class="samp">sym10</samp>’</dt> <dt>‘<samp class="samp">coif5</samp>’</dt> <dt>‘<samp class="samp">bl3</samp>’</dt> </dl>

</dd> <dt><samp class="option">percent</samp></dt> <dd> Set percent of full denoising. Allowed range is from 0 to 100 percent. Default value is 85 percent or partial denoising. </dd> <dt><samp class="option">profile</samp></dt> <dd> If enabled, first input frame will be used as noise profile. If first frame samples contain non-noise performance will be very poor. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> If enabled, input frames are analyzed for presence of noise. If noise is detected with high possibility then input frame profile will be used for processing following frames, until new noise frame is detected. </dd> <dt><samp class="option">samples</samp></dt> <dd> Set size of single frame in number of samples. Allowed range is from 512 to 65536. Default frame size is 8192 samples. </dd> <dt><samp class="option">softness</samp></dt> <dd> Set softness applied inside thresholding function. Allowed range is from 0 to 10. Default softness is 1. </dd> </dl>

 <h4 class="subsection">29.28.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.29 agate<span

class=”pull-right”># TOC</h3>

A gate is mainly used to reduce lower parts of a signal. This kind of signal processing reduces disturbing noise between useful signals. Gating is done by detecting the volume below a chosen level <var class=”var”>threshold</var> and dividing it by the factor set with <var class=”var”>ratio</var>. The bottom of the noise floor is set via <var class=”var”>range</var>. Because an exact manipulation of the signal would cause distortion of the waveform the reduction can be levelled over time. This is done by setting <var class=”var”>attack</var> and <var class=”var”>release</var>. <var class=”var”>attack</var> determines how long the signal has to fall below the threshold before any reduction will occur and <var class=”var”>release</var> sets the time the signal has to rise above the threshold to reduce the reduction again. Shorter signals than the chosen attack time will be left untouched. <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode of operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>. If set to <code class=”code”>upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class=”code”>downward</code> lower parts of signal will be reduced. </dd> <dt><samp class=”option”>range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class=”option”>ratio</samp></dt> <dd> Set a ratio by which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class=”option”>makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class=”option”>knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to

8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is <code class="code">rms</code>. Can be <code class="code">peak</code> or <code class="code">rms</code>. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is <code class="code">average</code>. Can be <code class="code">average</code> or <code class="code">maximum</code>. </dd> </dl>

 <h4 class="subsection">29.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.30 aiir# TOC</h3>

Apply an arbitrary Infinite Impulse Response filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">zeros, z</samp></dt> <dd> Set B/numerator/zeros/reflection coefficients. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set A/denominator/poles/ladder coefficients. </dd> <dt><samp class="option">gains, k</samp></dt> <dd> Set channels gains. </dd> <dt><samp class="option">dry_gain</samp></dt> <dd> Set input gain. </dd> <dt><samp class="option">wet_gain</samp></dt> <dd> Set output gain. </dd> <dt><samp class="option">format, f</samp></dt> <dd> Set coefficients format. <dl class="table"> <dt>‘<samp class="samp">ll</samp>’</dt> <dd> lattice-ladder function </dd> <dt>‘<samp class="samp">sf</samp>’</dt> <dd> analog transfer function </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> digital transfer function </dd> <dt>‘<samp class="samp">zp</samp>’</dt> <dd> Z-plane zeros/poles, cartesian (default) </dd> <dt>‘<samp class="samp">pr</samp>’</dt> <dd> Z-plane zeros/poles, polar radians </dd> <dt>‘<samp class="samp">pd</samp>’</dt> <dd> Z-plane zeros/poles, polar degrees </dd> <dt>‘<samp class="samp">sp</samp>’</dt> <dd> S-plane zeros/poles </dd> </dl>

</dd> <dt><samp class="option">process, r</samp></dt> <dd> Set type of processing. <dl class="table"> <dt>‘<samp class="samp">d</samp>’</dt> <dd> direct processing </dd> <dt>‘<samp class="samp">s</samp>’</dt> <dd> serial processing </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> parallel processing </dd> </dl>

</dd> <dt><samp class="option">precision, e</samp></dt> <dd> Set filtering precision. <dl class="table"> <dt>‘<samp class="samp">dbl</samp>’</dt> <dd> double-precision floating-point (default) </dd> <dt>‘<samp class="samp">flt</samp>’</dt> <dd> single-precision floating-point </dd> <dt>‘<samp class="samp">i32</samp>’</dt> <dd> 32-bit integers </dd> <dt>‘<samp class="samp">i16</samp>’</dt> <dd> 16-bit integers </dd> </dl>

</dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize filter coefficients, by default is enabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">mix</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">response</samp></dt> <dd> Show IR frequency response, magnitude(magenta), phase(green) and group delay(yellow) in additional video stream. By default it is disabled. </dd> <dt><samp class="option">channel</samp></dt> <dd> Set for which IR channel to display frequency response. By default is first channel displayed. This option is used only when <var class="var">response</var> is enabled. </dd> <dt><samp class="option">size</samp></dt> <dd> Set video stream size. This option is used only when <var class="var">response</var> is enabled. </dd> </dl>

Coefficients in <code class="code">tf</code> and <code class="code">sf</code> format are separated by spaces and are in ascending order. Coefficients in <code class="code">zp</code> format are separated by spaces and order of coefficients doesn’t matter. Coefficients in <code class="code">zp</code> format are complex numbers with <var class="var">i</var> imaginary unit. Different coefficients and gains can be provided for every channel, in such case use ’|’ to separate coefficients or gains. Last provided coefficients will be used for all remaining channels. <h4 class="subsection">29.30.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply 2 pole elliptic notch at around 5000Hz for 48000 Hz sample rate: <pre class="example-preformatted">aiir=k=1:z=7.957584807809675810E-1 -2.575128568908332300 3.674839853930788710 -2.57512875289799137 7.957586296317130880E-1:p=1 -2.86950072432325953 3.63022088054647218 -2.28075678147272232 6.361362326477423500E-1:f=tf:r=d </pre>

Same as above but in <code class="code">zp</code> format: <pre class="example-preformatted">aiir=k=0.7957584807809675 6:z=0.80918701+0.58773 0.80918701-0.58773007i 0.80884700+0.58784055i 0.80884700-0.58784055i:p=0.63892345+0.59951235i 0.63892345-0.59951235i 0.79582691+0.44198673i 0.79582691-0.44198673i:f=zp:r=s </pre>

Apply 3-rd order analog normalized Butterworth low-pass filter, using analog transfer function format: <pre class=”example-preformatted”>aiir=z=1.3057 0 0 0:p=1.3057 2.3892 2.1860 1:f=sf:r=d </pre>

 <h3 class=”section”>29.31 alimiter# TOC</h3>

The limiter prevents an input signal from rising over a desired threshold. This limiter uses lookahead technology to prevent your signal from distorting. It means that there is a small delay after the signal is processed. Keep in mind that the delay it produces is the attack time you set. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Don’t let signals above this level pass the limiter. Default is 1. </dd> <dt><samp class=”option”>attack</samp></dt> <dd> The limiter will reach its attenuation level in this amount of time in milliseconds. Default is 5 milliseconds. </dd> <dt><samp class=”option”>release</samp></dt> <dd> Come back from limiting to attenuation 1.0 in this amount of milliseconds. Default is 50 milliseconds. </dd> <dt><samp class=”option”>asc</samp></dt> <dd> When gain reduction is always needed ASC takes care of releasing to an average reduction level rather than reaching a reduction of 0 in the release time. </dd> <dt><samp class=”option”>asc_level</samp></dt> <dd> Select how much the release time is affected by ASC, 0 means nearly no changes in release time while 1 produces higher release times. </dd> <dt><samp class=”option”>level</samp></dt> <dd> Auto level output signal. Default is enabled. This normalizes audio back to 0dB if enabled. </dd> <dt><samp class=”option”>latency</samp></dt> <dd> Compensate the delay introduced by using the lookahead buffer set with attack parameter. Also flush the valid audio data in the lookahead buffer when the stream hits EOF. </dd> </dl>

Depending on picked setting it is recommended to upsample input 2x or 4x times with aresample before applying this filter. <h3 class=”section”>29.32 allpass# TOC</h3>

Apply a two-pole all-pass filter with central frequency (in Hz) <var class=”var”>frequency</var>, and filter-width <var class=”var”>width</var>. An all-pass filter changes the audio’s frequency to phase relation-

ship without changing its frequency to amplitude relationship. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set frequency in Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">order, o</samp></dt> <dd> Set the filter order, can be 1 or 2. Default is 2. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl> </dd> </dl>

 <h4 class="subsection">29.32.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change allpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt>

<dd> Change allpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change allpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change allpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">29.33 aloop# TOC</h3>

Loop audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set maximal number of samples. Default is 0. </dd> <dt><samp class="option">start</samp></dt> <dd> Set first sample of loop. Default is 0. </dd> <dt><samp class="option">time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class="var">start</var> is set to <code class="code">-1</code>. </dd> </dl>

 <h3 class="section">29.34 amerge# TOC</h3>

Merge two or more audio streams into a single multi-channel stream. All inputs must have the same sample rate, and format. If inputs do not have the same duration, the output will stop with the shortest. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set the number of inputs. Default is 2. </dd> <dt><samp class="option">layout_mode</samp></dt> <dd> This option controls how the output channel layout is determined and if the audio channels are reordered during merge. <dl class="table"> <dt><samp class="option">legacy</samp></dt> <dd> This is the mode how the filter behaved historically so it is the default. If the channel layouts of the inputs are known and disjoint, and therefore compatible, the channel layout of the output will be set accordingly and the channels will be reordered as necessary. If the channel layouts of the inputs are not disjoint, some of them are unknown, or they are using special channel layouts, such as ambisonics, the output will have all the channels of the first input then all the channels of the second input, in that order, and the channel layout of the output will be the default value corresponding to the total number of channels. For example, if the first input is in 2.1 (FL+FR+LF) and the second input is FC+BL+BR, then the output will be in 5.1, with

the channels in the following order: a1, a2, b1, a3, b2, b3 (a1 is the first channel of the first input, b1 is the first channel of the second input). On the other hand, if both input are in stereo, the output channels will be in the default order: a1, a2, b1, b2, and the channel layout will be arbitrarily set to 4.0, which may or may not be the expected value. </dd> <dt><samp class="option">reset</samp></dt> <dd> This mode ignores the input channel layouts and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. The output channel layout will only specify the total channel count. </dd> <dt><samp class="option">normal</samp></dt> <dd> This mode keeps channel name and designation information from the input channels and does no channel reordering. The output will have all the channels of the first input, then all the channels of the second input, in that order, and so on. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">29.34.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Merge two mono files into a stereo stream: <pre class="example-preformatted">amovie=left.wav [l] ; amovie=right.mp3 [r] ; [l] [r] amerge </pre>

Multiple merges assuming 1 video stream and 6 audio streams in <samp class="file">input.mkv</samp>: <pre class="example-preformatted">ffmpeg -i input.mkv -filter_complex "[0:1][0:2][0:3][0:4][0:5][0:6] amerge=inputs=6" -c:a pcm_s16le output.mkv </pre>

 <h3 class="section">29.35 amix# TOC</h3>

Mixes multiple audio inputs into a single output. Note that this filter only supports float samples (the <var class="var">amerge</var> and <var class="var">pan</var> audio filters support many formats). If the <var class="var">amix</var> input has integer samples then aresample will be automatically inserted to perform the conversion to float samples. It accepts the following parameters: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> The number of inputs. If unspecified, it defaults to 2. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp

class=”option”>first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> <dt><samp class=”option”>dropout_transition</samp></dt> <dd> The transition time, in seconds, for volume renormalization when an input stream ends. The default value is 2 seconds. </dd> <dt><samp class=”option”>weights</samp></dt> <dd> Specify weight of each input audio stream as a sequence of numbers separated by a space. If fewer weights are specified compared to number of inputs, the last weight is assigned to the remaining inputs. Default weight for each input is 1. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Always scale inputs instead of only doing summation of samples. Beware of heavy clipping if inputs are not normalized prior or after filtering by this filter if this option is disabled. By default is enabled. </dd> </dl>

 <h4 class=”subsection”>29.35.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> This will mix 3 input audio streams to a single output with the same duration as the first input and a dropout transition time of 3 seconds: <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex amix=inputs=3:duration=first:dropout_transition=3 OUTPUT </pre>

This will mix one vocal and one music input audio stream to a single output with the same duration as the longest input. The music will have quarter the weight as the vocals, and the inputs are not normalized: <pre class=”example-preformatted”>ffmpeg -i VOCALS -i MUSIC -filter_complex amix=inputs=2:duration=longest:dropout_transition=0:weights="1 0.25":normalize=0 OUTPUT </pre>

 <h4 class=”subsection”>29.35.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>weights</samp></dt> <dt><samp class=”option”>normalize</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class=”section”>29.36 amultiply# TOC</h3>

Multiply first audio stream with second audio stream and store result in output audio stream. Multiplication is done by multiplying each sample from first stream with sample at same position from second stream. With this element-wise multiplication one can create amplitude fades and amplitude modulations. <h3 class=”section”>29.37 anequalizer# TOC</h3>

High-order parametric multiband equalizer for each channel. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>params</samp></dt> <dd> This option string is in format: "c<var class=”var”>chn</var> f=<var class=”var”>cf</var> w=<var class=”var”>w</var> g=<var class=”var”>g</var> t=<var class=”var”>f</var> | ..." Each equalizer band is separated by ’|’. <dl class=”table”> <dt><samp class=”option”>chn</samp></dt> <dd> Set channel number to which equalization will be applied. If input doesn’t have that channel the entry is ignored. </dd> <dt><samp class=”option”>f</samp></dt> <dd> Set central frequency for band. If input doesn’t have that frequency the entry is ignored. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set band width in Hertz. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Set band gain in dB. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set filter type for band, optional, can be: <dl class=”table”> <dt>‘<samp class=”samp”>0</samp>’</dt> <dd> Butterworth, this is default. </dd> <dt>‘<samp class=”samp”>1</samp>’</dt> <dd> Chebyshev type 1. </dd> <dt>‘<samp class=”samp”>2</samp>’</dt> <dd> Chebyshev type 2. </dd> </dl> </dd> </dl>

</dd> <dt><samp class=”option”>curves</samp></dt> <dd> With this option activated frequency response of anequalizer is displayed in video stream. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set video stream size. Only useful if curves option is activated. </dd> <dt><samp class=”option”>mgain</samp></dt> <dd> Set max gain that will be displayed. Only useful if curves option is activated. Setting this to a reasonable value makes it possible to display gain which is derived from neighbour bands which are too close to each other and thus produce higher gain when both are activated. </dd> <dt><samp class=”option”>fscale</samp></dt> <dd> Set frequency scale used to draw frequency response in video output. Can be linear or logarithmic. Default is logarithmic. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set color for each channel curve which is going to be displayed in video stream. This is list of color names separated by space or by ’|’. Unrecognised or missing colors will be replaced by white color. </dd> </dl>

 <h4 class="subsection">29.37.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Lower gain by 10 of central frequency 200Hz and width 100 Hz for first 2 channels using Chebyshev type 1 filter: <pre class="example-preformatted">anequalizer=c0 f=200 w=100 g=-10 t=1|c1 f=200 w=100 g=-10 t=1 </pre>

 <h4 class="subsection">29.37.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">change</samp></dt> <dd> Alter existing filter parameters. Syntax for the commands is : "<var class="var">fN</var>|f=<var class="var">freq</var>|w=<var class="var">width</var>|g=<var class="var">gain</var>" <var class="var">fN</var> is existing filter number, starting from 0, if no such filter is available error is returned. <var class="var">freq</var> set new frequency parameter. <var class="var">width</var> set new width parameter in Hertz. <var class="var">gain</var> set new gain parameter in dB. Full filter invocation with asendcmd may look like this: asendcmd=c=’4.0 anequalizer change 0|f=200|w=50|g=1’,anequalizer=... </dd> </dl>

 <h3 class="section">29.38 anlmdn# TOC</h3>

Reduce broadband noise in audio samples using Non-Local Means algorithm. Each sample is adjusted by looking for other samples with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp> around the sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength, s</samp></dt> <dd> Set denoising strength. Allowed range is from 0.00001 to 10000. Default value is 0.00001. </dd> <dt><samp class="option">patch, p</samp></dt> <dd> Set patch radius duration. Allowed range is from 1 to 100 milliseconds. Default value is 2 milliseconds. </dd> <dt><samp class="option">research, r</samp></dt> <dd> Set research radius duration. Allowed range is from 2 to 300 milliseconds. Default value is 6 milliseconds. </dd> <dt><samp class="option">output, o</samp></dt> <dd> Set the output mode.

 It accepts the following values: <dl class="table">
<dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt>
<dd> Pass noise filtered out. </dd> <dt><samp class="option">n</samp></dt>
<dd> Pass only noise. Default value is <var class="var">o</var>.
</dd> </dl>

</dd> <dt><samp class="option">smooth, m</samp></dt> <dd> Set smooth factor. Default value is <var class="var">11</var>. Allowed range is from <var class="var">1</var> to <var class="var">1000</var>.
</dd> </dl>

 <h4 class="subsection">29.38.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">29.39 anlmf, anlms# TOC</h3>
Apply Normalized Least-Mean-(Squares|Fourth) algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by finding the filter coefficients that relate to producing the least mean square of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream).
A description of the accepted options follows. <dl class="table">
<dt><samp class="option">order</samp></dt> <dd> Set filter order.
 </dd> <dt><samp class="option">mu</samp></dt> <dd> Set filter mu. </dd> <dt><samp class="option">eps</samp></dt>
<dd> Set the filter eps. </dd> <dt><samp class="option">leakage</samp></dt>
<dd> Set the filter leakage. </dd> <dt><samp class="option">out_mode</samp></dt>
<dd> It accepts the following values: <dl class="table">
<dt><samp class="option">i</samp></dt> <dd> Pass the 1st input.
 </dd> <dt><samp class="option">d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class="option">o</samp></dt>
<dd> Pass difference between desired, 2nd input and error signal estimate.
 </dd> <dt><samp class="option">n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd>
<dt><samp class="option">e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class="var">o</var>.
</dd> </dl>

</dd> <dt><samp class="option">precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class="table">
<dt><samp class="option">auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp

class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>29.39.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> One of many usages of this filter is noise reduction, input audio is filtered with same samples that are delayed by fixed amount, one such example for stereo audio is: <pre class=”example-preformatted”>asplit[a][b],[a]adelay=32S|32S[a],[b][a]anlms=order=128:leakage=0.0005:mu=.
</pre>

 <h4 class=”subsection”>29.39.2 Commands# TOC</h4>

This filter supports the same commands as options, excluding option <code class=”code”>order</code>. <h3 class=”section”>29.40 anull# TOC</h3>

Pass the audio source unchanged to the output. <h3 class=”section”>29.41 apad# TOC</h3>

Pad the end of an audio stream with silence. This can be used together with <code class=”command”>ffmpeg</code> <samp class=”option”>-shortest</samp> to extend audio streams to the same length as the video stream. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>packet_size</samp></dt> <dd> Set silence packet size. Default value is 4096. </dd> <dt><samp class=”option”>pad_len</samp></dt> <dd> Set the number of samples of silence to add to the end. After the value is reached, the stream is terminated. This option is mutually exclusive with <samp class=”option”>whole_len</samp>. </dd> <dt><samp class=”option”>whole_len</samp></dt> <dd> Set the minimum total number of samples in the output audio stream. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class=”option”>pad_len</samp>. </dd> <dt><samp class=”option”>pad_dur</samp></dt> <dd> Specify the duration of samples of silence to add. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the

Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. </dd> <dt><samp class="option">whole_dur</samp></dt> <dd> Specify the minimum total duration in the output audio stream. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Used only if set to non-negative value. If the value is longer than the input audio length, silence is added to the end, until the value is reached. This option is mutually exclusive with <samp class="option">pad_dur</samp> </dd> </dl>

If neither the <samp class="option">pad_len</samp> nor the <samp class="option">whole_len</samp> nor <samp class="option">pad_dur</samp> nor <samp class="option">whole_dur</samp> option is set, the filter will add silence to the end of the input stream indefinitely. Note that for ffmpeg 4.4 and earlier a zero <samp class="option">pad_dur</samp> or <samp class="option">whole_dur</samp> also caused the filter to add silence indefinitely. <h4 class="subsection">29.41.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add 1024 samples of silence to the end of the input: <pre class="example-preformatted">apad=pad_len=1024 </pre>

Make sure the audio output will contain at least 10000 samples, pad the input with silence if required: <pre class="example-preformatted">apad=whole_len=10000 </pre>

Use <code class="command">ffmpeg</code> to pad the audio input with silence, so that the video stream will always result the shortest and will be converted until the end in the output file when using the <samp class="option">shortest</samp> option: <pre class="example-preformatted">ffmpeg -i VIDEO -i AUDIO -filter_complex "[1:0]apad" -shortest OUTPUT </pre>

 <h3 class="section">29.42 aphaser# TOC</h3> Add a phasing effect to the input audio. A phaser filter creates series of peaks and troughs in the frequency spectrum. The position of the peaks and troughs are modulated so that they vary over time, creating a sweeping effect. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.74 </dd> <dt><samp class="option">delay</samp></dt>

<dd> Set delay in milliseconds. Default is 3.0. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set decay. Default is 0.4. </dd> <dt><samp class="option">speed</samp></dt> <dd> Set modulation speed in Hz. Default is 0.5. </dd> <dt><samp class="option">type</samp></dt> <dd> Set modulation type. Default is triangular. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">triangular, t</samp>’</dt> <dt>‘<samp class="samp">sinusoidal, s</samp>’</dt> </dl> </dd> </dl>

 <h3 class="section">29.43 aphaseshift# TOC</h3> Apply phase shift to input audio samples. The filter accepts the following options: <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Specify phase shift. Allowed range is from -1.0 to 1.0. Default value is 0.0. </dd> <dt><samp class="option">level</samp></dt> <dd> Set output gain applied to final output. Allowed range is from 0.0 to 1.0. Default value is 1.0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order used for filtering. Allowed range is from 1 to 16. Default value is 8. </dd> </dl>

 <h4 class="subsection">29.43.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.44 apsnr# TOC</h3> Measure Audio Peak Signal-to-Noise Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">29.45 apsyclip# TOC</h3> Apply Psychoacoustic clipper to input audio stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">clip</samp></dt> <dd> Set the clipping start value.

Default value is 0dBFS or 1. </dd> <dt><samp class="option">diff</samp></dt> <dd> Output only difference samples, useful to hear introduced distortions. By default is disabled. </dd> <dt><samp class="option">adaptive</samp></dt> <dd> Set strength of adaptive distortion applied. Default value is 0.5. Allowed range is from 0 to 1. </dd> <dt><samp class="option">iterations</samp></dt> <dd> Set number of iterations of psychoacoustic clipper. Allowed range is from 1 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Auto level output signal. Default is disabled. This normalizes audio back to 0dBFS if enabled. </dd> </dl>

 <h4 class="subsection">29.45.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.46 apulsator# TOC</h3>

Audio pulsator is something between an autopanner and a tremolo. But it can produce funny stereo effects as well. Pulsator changes the volume of the left and right channel based on a LFO (low frequency oscillator) with different waveforms and shifted phases. This filter have the ability to define an offset between left and right channel. An offset of 0 means that both LFO shapes match each other. The left and right channel are altered equally - a conventional tremolo. An offset of 50% means that the shape of the right channel is exactly shifted in phase (or moved backwards about half of the frequency) - pulsator acts as an autopanner. At 1 both curves match again. Every setting in between moves the phase shift gapless between all stages and produces some "bypassing" sounds with sine and triangle waveforms. The more you set the offset near 1 (starting from the 0.5) the faster the signal passes from the left to the right speaker. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. By default it is 1. Range is [0.015625 - 64]. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set waveform shape the LFO will use. Can be one of: sine, triangle, square, sawup or sawdown. Default is sine. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set modulation. Define how much of original signal is affected by the LFO. </dd> <dt><samp class="option">offset_l</samp></dt> <dd> Set left channel offset. Default is 0. Allowed range is [0 - 1]. </dd> <dt><samp class="option">offset_r</samp></dt> <dd> Set

right channel offset. Default is 0.5. Allowed range is [0 - 1]. </dd> <dt><samp class="option">width</samp></dt> <dd> Set pulse width. Default is 1. Allowed range is [0 - 2]. </dd> <dt><samp class="option">timing</samp></dt> <dd> Set possible timing mode. Can be one of: bpm, ms or hz. Default is hz. </dd> <dt><samp class="option">bpm</samp></dt> <dd> Set bpm. Default is 120. Allowed range is [30 - 300]. Only used if timing is set to bpm. </dd> <dt><samp class="option">ms</samp></dt> <dd> Set ms. Default is 500. Allowed range is [10 - 2000]. Only used if timing is set to ms. </dd> <dt><samp class="option">hz</samp></dt> <dd> Set frequency in Hz. Default is 2. Allowed range is [0.01 - 100]. Only used if timing is set to hz. </dd> </dl>

 <h3 class="section">29.47 aresample# TOC</h3>

Resample the input audio to the specified parameters, using the libswresample library. If none are specified then the filter will automatically convert between its input and output. This filter is also able to stretch/squeeze the audio data to make it match the timestamps or to inject silence / cut out audio to make it match the timestamps, do a combination of both or do neither. The filter accepts the syntax [<var class="var">sample_rate</var>:]<var class="var">resampler_options</var>, where <var class="var">sample_rate</var> expresses a sample rate and <var class="var">resampler_options</var> is a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ":". See the <a data-manual="ffmpeg-resampler" href="./ffmpeg-resampler.html#Resampler-Options">"Resampler Options" section in the ffmpeg-resampler(1) manual for the complete list of supported options. <h4 class="subsection">29.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Resample the input audio to 44100Hz: <pre class="example-preformatted">aresample=44100 </pre>

Stretch/squeeze samples to the given timestamps, with a maximum of 1000 samples per second compensation: <pre class="example-preformatted">aresample=async=1000 </pre>

 <h3 class="section">29.48 areverse# <a class="anchor hidden-xs"href="#toc-areverse"

aria-hidden=”true”>TOC</h3>

Reverse an audio clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class=”subsection”>29.48.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Take the first 5 seconds of a clip, and reverse it. <pre class=”example-preformatted”>atrim=end=5,areverse </pre>

 <h3 class=”section”>29.49 arls# TOC</h3> Apply Recursive Least Squares algorithm to the first audio stream using the second audio stream. This adaptive filter is used to mimic a desired filter by recursively finding the filter coefficients that relate to producing the minimal weighted linear least squares cost function of the error signal (difference between the desired, 2nd input audio stream and the actual signal, the 1st input audio stream). A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>order</samp></dt> <dd> Set the filter order. </dd> <dt><samp class=”option”>lambda</samp></dt> <dd> Set the forgetting factor. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set the coefficient to initialize internal covariance matrix. </dd> <dt><samp class=”option”>out_mode</samp></dt> <dd> Set the filter output samples. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>i</samp></dt> <dd> Pass the 1st input. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Pass the 2nd input. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Pass difference between desired, 2nd input and error signal estimate. </dd> <dt><samp class=”option”>n</samp></dt> <dd> Pass difference between input, 1st input and error signal estimate. </dd> <dt><samp class=”option”>e</samp></dt> <dd> Pass error signal estimated samples. Default value is <var class=”var”>o</var>. </dd> </dl>

</dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set which precision to use when processing samples. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt> <dd> Auto pick internal sample format depending on other filters. </dd> <dt><samp class=”option”>float</samp></dt> <dd> Always use single-floating point precision sample format. </dd> <dt><samp class=”option”>double</samp></dt> <dd> Always use double-floating point precision sample format. </dd> </dl> </dd> </dl>

 <h3 class="section">29.50 arnndn# TOC</h3>

Reduce noise from speech using Recurrent Neural Networks. This filter accepts the following options: <dl class="table"> <dt><samp class="option">model, m</samp></dt> <dd> Set train model file to load. This option is always required. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set how much to mix filtered samples into final output. Allowed range is from -1 to 1. Default value is 1. Negative values are special, they set how much to keep filtered noise in the final filter output. Set this option to -1 to hear actual noise removed from input signal. </dd> </dl>

 <h4 class="subsection">29.50.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.51 asdr# TOC</h3> Measure Audio Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">29.52 asetnsamples# TOC</h3>

Set the number of samples per each output audio frame. The last output packet may contain a different number of samples, as the filter will flush all the remaining samples when the input audio signals its end. The filter accepts the following options: <dl class="table"> <dt><samp class="option">nb_out_samples, n</samp></dt> <dd> Set the number of frames per each output audio frame. The number is intended as the number of samples per each channel. Default value is 1024. </dd> <dt><samp class="option">pad, p</samp></dt> <dd> If set to 1, the filter will pad the last audio frame with zeroes, so that the last frame will contain the same number of samples as the previous ones. Default value is 1. </dd> </dl>

For example, to set the number of per-frame samples to 1234 and disable padding for the last frame, use: <pre class="example-preformatted">asetnsamples=n=1234:p=0 </pre>

 <h3 class="section">29.53 asetrate# TOC</h3>

Set the sample rate without altering the PCM data. This will result in a change of speed and pitch. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set the output sample rate. Default is 44100 Hz. </dd> </dl>

 <h3 class="section">29.54 ashowinfo# TOC</h3>

Show a line containing various information for each input audio frame. The input audio is not modified. The shown line contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var>. The following values are shown in the output: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The presentation timestamp of the input frame, in time base units; the time base depends on the filter input pad, and is usually 1/<var class="var">sample_rate</var>. </dd> <dt><samp class="option">pts_time</samp></dt> <dd> The presentation timestamp of the input frame in seconds. </dd> <dt><samp class="option">fmt</samp></dt> <dd> The sample format. </dd> <dt><samp class="option">chlayout</samp></dt> <dd> The channel layout. </dd> <dt><samp class="option">rate</samp></dt> <dd> The sample rate for the audio frame. </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> The number of samples (per channel) in the frame. </dd> <dt><samp class="option">checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of the audio data. For planar audio, the data is treated as if all the planes were concatenated. </dd> <dt><samp class="option">plane_checksums</samp></dt> <dd> A list of Adler-32 checksums for each data plane. </dd> </dl>

 <h3 class="section">29.55 asisdr# TOC</h3> Measure Audio Scaled-Invariant Signal-to-Distortion Ratio. This filter takes two audio streams for input, and outputs first audio stream. Results are in dB per channel at end of either input. <h3 class="section">29.56 asoftclip# <a class="anchor hidden-xs"href="#toc-asoftclip"

aria-hidden=”true”>TOC</h3> Apply audio soft clipping. Soft clipping is a type of distortion effect where the amplitude of a signal is saturated along a smooth curve, rather than the abrupt shape of hard-clipping. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>type</samp></dt> <dd> Set type of soft-clipping. It accepts the following values: <dl class=”table”> <dt><samp class=”option”>hard</samp></dt> <dt><samp class=”option”>tanh</samp></dt> <dt><samp class=”option”>atan</samp></dt> <dt><samp class=”option”>cubic</samp></dt> <dt><samp class=”option”>exp</samp></dt> <dt><samp class=”option”>alg</samp></dt> <dt><samp class=”option”>quintic</samp></dt> <dt><samp class=”option”>sin</samp></dt> <dt><samp class=”option”>erf</samp></dt> </dl>

</dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold from where to start clipping. Default value is 0dB or 1. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set gain applied to output. Default value is 0dB or 1. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Set additional parameter which controls sigmoid function. </dd> <dt><samp class=”option”>oversample</samp></dt> <dd> Set oversampling factor. </dd> </dl>

 <h4 class=”subsection”>29.56.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>29.57 aspectralstats# TOC</h3>

Display frequency domain statistical information about the audio channels. Statistics are calculated and stored as metadata for each audio channel and for each audio frame. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>win_size</samp></dt> <dd> Set the window length in samples. Default value is 2048. Allowed range is from 32 to 65536. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann, hanning</samp>’</dt> <dt>‘<samp class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</ <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’<

<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp class="samp">kaiser</samp>’</dt> </dl> Default is <code class="code">hann</code>. </dd> <dt><samp class="option">overlap</samp></dt>
<dd> Set window overlap. Allowed range is from <code class="code">0</code> to <code class="code">1</code>. Default value is <code class="code">0.5</code>.
 </dd> <dt><samp class="option">measure</samp></dt>
<dd> Select the parameters which are measured. The metadata keys can be used as flags, default is <samp class="option">all</samp> which measures everything. <samp class="option">none</samp> disables all measurement. </dd> </dl>

A list of each metadata key follows: <dl class="table"> <dt><samp class="option">mean</samp></dt> <dt><samp class="option">variance</samp></dt>
<dt><samp class="option">centroid</samp></dt> <dt><samp class="option">spread</samp></dt>
<dt><samp class="option">skewness</samp></dt> <dt><samp class="option">kurtosis</samp></dt>
<dt><samp class="option">entropy</samp></dt> <dt><samp class="option">flatness</samp></dt>
<dt><samp class="option">crest</samp></dt> <dt><samp class="option">flux</samp></dt>
<dt><samp class="option">slope</samp></dt> <dt><samp class="option">decrease</samp></dt>
<dt><samp class="option">rolloff</samp></dt> </dl>

 <h3 class="section">29.58 asr#
TOC</h3>
Automatic Speech Recognition This filter uses PocketSphinx for speech recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-pocketsphinx</code>.
 It accepts the following options: <dl class="table">
<dt><samp class="option">rate</samp></dt> <dd> Set sampling rate of input audio. Defaults is <code class="code">16000</code>. This need to match speech models, otherwise one will get poor results.
</dd> <dt><samp class="option">hmm</samp></dt> <dd> Set dictionary containing acoustic model files. </dd> <dt><samp class="option">dict</samp></dt> <dd> Set pronunciation dictionary.
 </dd> <dt><samp class="option">lm</samp></dt> <dd> Set language model file. </dd> <dt><samp class="option">lmctl</samp></dt>
<dd> Set language model set. </dd> <dt><samp class="option">lmname</samp></dt>
<dd> Set which language model to use. </dd> <dt><samp class="option">logfn</samp></dt> <dd> Set output for log messages.
</dd> </dl>

The filter exports recognized speech as the frame metadata <code

class=”code”>lavfi.asr.text</code>. <h3 class=”section”>29.59 astats# TOC</h3>

Display time domain statistical information about the audio channels. Statistics are calculated and displayed for each audio channel and, where applicable, an overall figure is also given. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>length</samp></dt> <dd> Short window length in seconds, used for peak and through RMS measurement. Default is <code class=”code”>0.05</code> (50 milliseconds). Allowed range is <code class=”code”>[0 - 10]</code>. </dd> <dt><samp class=”option”>metadata</samp></dt> <dd> Set metadata injection. All the metadata keys are prefixed with <code class=”code”>lavfi.astats.X</code>, where <code class=”code”>X</code> is channel number starting from 1 or string <code class=”code”>Overall</code>. Default is disabled. Available keys for each channel are: <var class=”var”>Bit_depth</var> <var class=”var”>Crest_factor</var> <var class=”var”>DC_offset</var> <var class=”var”>Dynamic_range</var> <var class=”var”>Entropy</var> <var class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var> <var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var> <var class=”var”>Min_difference</var> <var class=”var”>Min_level</var> <var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var> <var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var> <var class=”var”>Number_of_denormals</var> <var class=”var”>Peak_count</var> <var class=”var”>Abs_Peak_count</var> <var class=”var”>Peak_level</var> <var class=”var”>RMS_difference</var> <var class=”var”>RMS_peak</var> <var class=”var”>RMS_trough</var> <var class=”var”>Zero_crossings</var> <var class=”var”>Zero_crossings_rate</var> and for <code class=”code”>Overall</code>: <var class=”var”>Bit_depth</var> <var class=”var”>DC_offset</var> <var class=”var”>Entropy</var> <var class=”var”>Flat_factor</var> <var class=”var”>Max_difference</var> <var class=”var”>Max_level</var> <var class=”var”>Mean_difference</var> <var class=”var”>Min_difference</var> <var class=”var”>Min_level</var> <var class=”var”>Noise_floor</var> <var class=”var”>Noise_floor_count</var> <var class=”var”>Number_of_Infs</var> <var class=”var”>Number_of_NaNs</var> <var class=”var”>Number_of_denormals</var> <var class=”var”>Number_of_samples</var> <var class=”var”>Peak_count</var> <var class=”var”>Abs_Peak_count</var> <var class=”var”>Peak_level</var> <var class=”var”>RMS_difference</var> <var class=”var”>RMS_level</var> <var class=”var”>RMS_peak</var> <var class=”var”>RMS_trough</var> For example, a full key looks like <code class=”code”>lavfi.astats.1.DC_offset</code> or <code class=”code”>lavfi.astats.Overall.Peak_count</code>. Read below for the description of the keys. </dd> <dt><samp class=”option”>reset</samp></dt> <dd> Set the number of frames

over which cumulative stats are calculated before being reset. Default is disabled.
 </dd> <dt><samp class=”option”>measure_perchannel</samp></dt>
<dd> Select the parameters which are measured per channel. The metadata
keys can be used as flags, default is <samp class=”option”>all</samp> which
measures everything. <samp class=”option”>none</samp> disables all per
channel measurement. </dd> <dt><samp class=”option”>measure_overall</samp></dt>
<dd> Select the parameters which are measured overall. The metadata
keys can be used as flags, default is <samp class=”option”>all</samp>
which measures everything. <samp class=”option”>none</samp> disables all
overall measurement. </dd> </dl>

A description of the measure keys follow:

<dl class=”table”>

<dt><samp class=”option”>none</samp></dt>

<dd> no measures

</dd>

<dt><samp class=”option”>all</samp></dt>

<dd> all measures

</dd>

<dt><samp class=”option”>Bit_depth</samp></dt>

<dd> overall bit depth of audio, i.e. number of bits used for each sample

</dd>

<dt><samp class=”option”>Crest_factor</samp></dt>

<dd> standard ratio of peak to RMS level (note: not in dB)

</dd>

<dt><samp class=”option”>DC_offset</samp></dt>

<dd> mean amplitude displacement from zero

</dd>

<dt><samp class=”option”>Dynamic_range</samp></dt>

<dd> measured dynamic range of audio in dB

</dd>

<dt><samp class="option">Entropy</samp></dt>

<dd> entropy measured across whole audio, entropy of value near 1.0 is typically measured for white noise

</dd>

<dt><samp class="option">Flat_factor</samp></dt>

<dd> flatness (i.e. consecutive samples with the same value) of the signal at its peak levels

(i.e. either <var class="var">Min_level</var> or <var class="var">Max_level</var>)

</dd>

<dt><samp class="option">Max_difference</samp></dt>

<dd> maximal difference between two consecutive samples

</dd>

<dt><samp class="option">Max_level</samp></dt>

<dd> maximal sample level

</dd>

<dt><samp class="option">Mean_difference</samp></dt>

<dd> mean difference between two consecutive samples, i.e. the average of each difference between two consecutive samples

</dd>

<dt><samp class="option">Min_difference</samp></dt>

<dd> minimal difference between two consecutive samples

</dd>

<dt><samp class="option">Min_level</samp></dt>

<dd> minimal sample level

</dd>

<dt><samp class="option">Noise_floor</samp></dt>

<dd> minimum local peak measured in dBFS over a short window

</dd>

<dt><samp class="option">Noise_floor_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained

<var class="var">Noise floor</var>

</dd>

<dt><samp class="option">Number_of_Infs</samp></dt>

<dd> number of samples with an infinite value

</dd>

<dt><samp class="option">Number_of_NaNs</samp></dt>

<dd> number of samples with a NaN (not a number) value

</dd>

<dt><samp class="option">Number_of_denormals</samp></dt>

<dd> number of samples with a subnormal value

</dd>

<dt><samp class="option">Number_of_samples</samp></dt>

<dd> number of samples

</dd>

<dt><samp class="option">Peak_count</samp></dt>

<dd> number of occasions (not the number of samples) that the signal attained either

<var class=”var”>Min_level</var> or <var class=”var”>Max_level</var>

</dd>

<dt><samp class=”option”>Abs_Peak_count</samp></dt>

<dd> number of occasions that the absolute samples taken from the signal attained

max absolute value of <var class=”var”>Min_level</var> and <var class=”var”>Max_level</var>

</dd>

<dt><samp class=”option”>Peak_level</samp></dt>

<dd> standard peak level measured in dBFS

</dd>

<dt><samp class=”option”>RMS_difference</samp></dt>

<dd> Root Mean Square difference between two consecutive samples

</dd>

<dt><samp class=”option”>RMS_level</samp></dt>

<dd> standard RMS level measured in dBFS

</dd>

<dt><samp class=”option”>RMS_peak</samp></dt>

<dt><samp class=”option”>RMS_trough</samp></dt>

<dd> peak and through values for RMS level measured over a short window,

measured in dBFS.

</dd>

<dt><samp class=”option”>Zero crossings</samp></dt>

<dd> number of points where the waveform crosses the zero level axis

</dd>

<dt><samp class="option">Zero crossings rate</samp></dt>

<dd> rate of Zero crossings and number of audio samples

</dd>

</dl>

 <h3 class="section">29.60 asubboost# TOC</h3> Boost subwoofer frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">dry</samp></dt> <dd> Set dry gain, how much of original signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet gain, how much of filtered signal is kept. Allowed range is from 0 to 1. Default value is 1.0. </dd> <dt><samp class="option">boost</samp></dt> <dd> Set max boost factor. Allowed range is from 1 to 12. Default value is 2. </dd> <dt><samp class="option">decay</samp></dt> <dd> Set delay line decay gain value. Allowed range is from 0 to 1. Default value is 0.0. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Set delay line feedback gain value. Allowed range is from 0 to 1. Default value is 0.9. </dd> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 50 to 900. Default value is 100. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope amount for cutoff frequency. Allowed range is 0.0001 to 1. Default value is 0.5. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay. Allowed range is from 1 to 100. Default value is 20. </dd> <dt><samp class="option">channels</samp></dt> <dd> Set the channels to process. Default value is all available. </dd> </dl>

 <h4 class="subsection">29.60.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.61 asubcut# TOC</h3> Cut subwoofer frequencies. This filter allows to set custom, steeper roll off than highpass filter, and thus is able to more attenuate frequency content in stop-band. The filter accepts the following options: <dl class="table">

<dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 2 to 200. Default value is 20. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">29.61.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.62 asupercut# TOC</h3> Cut super frequencies. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set cutoff frequency in Hertz. Allowed range is 20000 to 192000. Default value is 20000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 3 to 20. Default value is 10. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 1. Default value is 1. </dd> </dl>

 <h4 class="subsection">29.62.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.63 asuperpass# TOC</h3> Apply high order Butterworth band-pass filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">29.63.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.64 asuperstop# TOC</h3> Apply high order Butterworth band-stop filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">centerf</samp></dt> <dd> Set center frequency in Hertz. Allowed range is 2 to 999999. Default value is 1000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set filter order. Available values are from 4 to 20. Default value is 4. </dd> <dt><samp class="option">qfactor</samp></dt> <dd> Set Q-factor. Allowed range is from 0.01 to 100. Default value is 1. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input gain level. Allowed range is from 0 to 2. Default value is 1. </dd> </dl>

 <h4 class="subsection">29.64.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.65 atempo# TOC</h3>

Adjust audio tempo. The filter accepts exactly one parameter, the audio tempo. If not specified then the filter will assume nominal 1.0 tempo. Tempo must be in the [0.5, 100.0] range. Note that tempo greater than 2 will skip some samples rather than blend them in. If for any reason this is a concern it is always possible to daisy-chain several instances of atempo to achieve the desired product tempo. <h4 class="subsection">29.65.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Slow down audio to 80% tempo: <pre class="example-preformatted">atempo=0.8 </pre>

To speed up audio to 300% tempo: <pre class="example-preformatted">atempo=3 </pre>

To speed up audio to 300% tempo by daisy-chaining two atempo instances: <pre class="example-preformatted">atempo=sqrt(3),atempo=sqrt(3)
</pre>

 <h4 class="subsection">29.65.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class="var">tempo</var>" </dd> </dl>

 <h3 class="section">29.66 atilt# TOC</h3> Apply spectral tilt filter to audio stream. This filter apply any spectral roll-off slope over any specified frequency band. The filter accepts the following options: <dl class="table"> <dt><samp class="option">freq</samp></dt> <dd> Set central frequency of tilt in Hz. Default is 10000 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set slope direction of tilt. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of tilt. Default is 1000. Allowed range is from 100 to 10000. </dd> <dt><samp class="option">order</samp></dt> <dd> Set order of tilt filter. </dd> <dt><samp class="option">level</samp></dt> <dd> Set input volume level. Allowed range is from 0 to 4. Default is 1. </dd> </dl>

 <h4 class="subsection">29.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.67 atrim# TOC</h3>

Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Timestamp (in seconds) of the start of the section to keep. I.e. the audio sample with the timestamp <var class="var">start</var> will be the first sample in the output. </dd> <dt><samp class="option">end</samp></dt> <dd> Specify time of the first audio sample that will be dropped,

i.e. the audio sample immediately preceding the one with the timestamp <var class=”var”>end</var> will be the last sample in the output. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> Same as <var class=”var”>start</var>, except this option sets the start timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>end_pts</samp></dt> <dd> Same as <var class=”var”>end</var>, except this option sets the end timestamp in samples instead of seconds. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class=”option”>start_sample</samp></dt> <dd> The number of the first sample that should be output. </dd> <dt><samp class=”option”>end_sample</samp></dt> <dd> The number of the first sample that should be dropped. </dd> </dl>

<samp class=”option”>start</samp>, <samp class=”option”>end</samp>, and <samp class=”option”>duration</samp> are expressed as time duration specifications; see <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual. Note that the first two sets of the start/end options and the <samp class=”option”>duration</samp> option look at the frame timestamp, while the _sample options simply count the samples that pass through the filter. So start/end_pts and start/end_sample will give different results when the timestamps are wrong, inexact or do not start at zero. Also note that this filter does not modify the timestamps. If you wish to have the output timestamps start at zero, insert the asetpts filter after the atrim filter. If multiple start or end options are set, this filter tries to be greedy and keep all samples that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple atrim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class=”itemize mark-bullet”> Drop everything except the second minute of input: <pre class=”example-preformatted”>ffmpeg -i INPUT -af atrim=60:120 </pre>

Keep only the first 1000 samples: <pre class=”example-preformatted”>ffmpeg -i INPUT -af atrim=end_sample=1000 </pre>

 <h3 class=”section”>29.68 axcorrelate# TOC</h3> Calculate normalized windowed cross-correlation between two input audio streams. Resulted samples are always between -1 and 1 inclusive. If result is 1 it means two input samples are highly correlated in that selected

segment. Result 0 means they are not correlated at all. If result is -1 it means two input samples are out of phase, which means they cancel each other. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size</samp></dt> <dd> Set size of segment over which cross-correlation is calculated. Default is 256. Allowed range is from 2 to 131072. </dd> <dt><samp class="option">algo</samp></dt> <dd> Set algorithm for cross-correlation. Can be <code class="code">slow</code> or <code class="code">fast</code> or <code class="code">best</code>. Default is <code class="code">best</code>. Fast algorithm assumes mean values over any given segment are always zero and thus need much less calculations to make. This is generally not true, but is valid for typical audio streams. </dd> </dl>

 <h4 class="subsection">29.68.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Calculate correlation between channels in stereo audio stream: <pre class="example-preformatted">ffmpeg -i stereo.wav -af channelsplit,axcorrelate=size=1024:algo=fast correlation.wav </pre>

 <h3 class="section">29.69 bandpass# TOC</h3>

Apply a two-pole Butterworth band-pass filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width width. The <var class="var">csg</var> option selects a constant skirt gain (peak gain = Q) instead of the default: constant 0dB peak gain. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">csg</samp></dt> <dd> Constant skirt gain if set to 1. Defaults to 0. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify

the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.69.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change bandpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change bandpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bandpass width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandpass mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">29.70 bandreject# TOC</h3>

Apply a two-pole Butterworth band-reject filter with central frequency <var class="var">frequency</var>, and (3dB-point) band-width <var class="var">width</var>. The filter roll off at 6dB per octave (20dB per decade). The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency. Default is <code class="code">3000</code>. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero

values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.70.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change bandreject frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change bandreject width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change bandreject width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change bandreject mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">29.71 bass, lowshelf# TOC</h3>

Boost or cut the bass (lower) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">100</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. De-

fault is 2. </dd> <dt><samp class="option">mix, m</samp></dt>
<dd> How much to use filtered signal in output. Default is 1. Range
is between 0 and 1. </dd> <dt><samp class="option">channels,
c</samp></dt> <dd> Specify which channels to filter, by default all
available are filtered. </dd> <dt><samp class="option">normalize,
n</samp></dt> <dd> Normalize biquad coefficients, by default is
disabled. Enabling it will normalize magnitude response at DC to 0dB.
 </dd> <dt><samp class="option">transform, a</samp></dt>
<dd> Set transform type of IIR filter. <dl class="table">
<dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt>
<dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt>
<dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt>
<dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set
precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt>
<dd> Pick automatic sample format depending on surround fil-
ters. </dd> <dt><samp class="option">s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough
value (higher than impulse response length truncated when reaches near zero
values) filtering will become linear phase otherwise if not big enough it will just
produce nasty artifacts. Note that filter delay will be exactly this
many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.71.1 Com-
mands<a class="anchor hidden-xs" href="#Commands-
31" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Commands-31" aria-hidden="true">TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">frequency, f</samp></dt> <dd> Change
bass frequency. Syntax for the command is : "<var class="var">frequency</var>"
 </dd> <dt><samp class="option">width_type, t</samp></dt>
<dd> Change bass width_type. Syntax for the command is : "<var
class="var">width_type</var>" </dd> <dt><samp class="option">width,
w</samp></dt> <dd> Change bass width. Syntax for the command is
: "<var class="var">width</var>" </dd> <dt><samp
class="option">gain, g</samp></dt> <dd> Change bass gain. Syn-
tax for the command is : "<var class="var">gain</var>"
</dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change
bass mix. Syntax for the command is : "<var class="var">mix</var>"
</dd> </dl>

 <h3 class="section">29.72 biquad# TOC</h3>

Apply a biquad IIR filter with the given coefficients. Where <var class="var">b0</var>, <var class="var">b1</var>, <var class="var">b2</var> and <var class="var">a0</var>, <var class="var">a1</var>, <var class="var">a2</var> are the numerator and denominator coefficients respectively. and <var class="var">channels</var>, <var class="var">c</var> specify which channels to filter, by default all available are filtered. <h4 class="subsection">29.72.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">a0</samp></dt> <dt><samp class="option">a1</samp></dt> <dt><samp class="option">a2</samp></dt> <dt><samp class="option">b0</samp></dt> <dt><samp class="option">b1</samp></dt> <dt><samp class="option">b2</samp></dt> <dd> Change biquad parameter. Syntax for the command is : "<var class="var">value</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero

values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h3 class="section">29.73 bs2b# TOC</h3>
Bauer stereo to binaural transformation, which improves headphone listening of stereo audio records. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libbs2b</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">profile</samp></dt> <dd> Pre-defined crossfeed level. <dl class="table"> <dt><samp class="option">default</samp></dt> <dd> Default level (fcut=700, feed=50). </dd> <dt><samp class="option">cmoy</samp></dt> <dd> Chu Moy circuit (fcut=700, feed=60). </dd> <dt><samp class="option">jmeier</samp></dt> <dd> Jan Meier circuit (fcut=650, feed=95). </dd> </dl>

</dd> <dt><samp class="option">fcut</samp></dt> <dd> Cut frequency (in Hz). </dd> <dt><samp class="option">feed</samp></dt> <dd> Feed level (in Hz). </dd> </dl>

 <h3 class="section">29.74 channelmap# TOC</h3>

Remap input channels to new locations. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Map channels from input to output. The argument is a ’|’-separated list of mappings, each in the <code class="code"><var class="var">in_channel</var>-<var class="var">out_channel</var></code> or <code class="code"><var class="var">in_channel</var></code> form. <var class="var">in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the input channel layout. <var class="var">out_channel</var> is the name of the output channel or its index in the output channel layout. If <var class="var">out_channel</var> is not given then it is implicitly an index, starting with zero and increasing by one for each mapping. Mixing different types of mappings is not allowed and will result in a parse error. </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the output stream. If not specified, then filter will guess it based on the <var class="var">out_channel</var> names or the number of mappings. Guessed layouts will not necessarily contain channels in the order of the mappings. </dd> </dl>

If no mapping is present, the filter will implicitly map input channels to

output channels, preserving indices. <h4 class="subsection">29.74.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a 5.1+downmix input MOV file, <pre class="example-preformatted">ffmpeg -i in.mov -filter 'channelmap=map=DL-FL|DR-FR' out.wav </pre> will create an output WAV file tagged as stereo from the downmix channels of the input. To fix a 5.1 WAV improperly encoded in AAC’s native channel order <pre class="example-preformatted">ffmpeg -i in.wav -filter 'channelmap=1|2|0|5|3|4:5.1' out.wav </pre>

 <h3 class="section">29.75 channelsplit# TOC</h3>

Split each channel from an input audio stream into a separate output stream. It accepts the following parameters: <dl class="table"> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the input stream. The default is "stereo". </dd> <dt><samp class="option">channels</samp></dt> <dd> A channel layout describing the channels to be extracted as separate output streams or "all" to extract each input channel as a separate stream. The default is "all". Choosing channels not present in channel layout in the input will result in an error. </dd> </dl>

 <h4 class="subsection">29.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> For example, assuming a stereo input MP3 file, <pre class="example-preformatted">ffmpeg -i in.mp3 -filter_complex channelsplit out.mkv </pre> will create an output Matroska file with two audio streams, one containing only the left channel and the other the right channel. Split a 5.1 WAV file into per-channel files: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1[FL][FR][FC][LFE][SL][SR]' -map '[FL]' front_left.wav -map '[FR]' front_right.wav -map '[FC]' front_center.wav -map '[LFE]' lfe.wav -map '[SL]' side_left.wav -map '[SR]' side_right.wav </pre>

Extract only LFE from a 5.1 WAV file: <pre class="example-preformatted">ffmpeg -i in.wav -filter_complex 'channelsplit=channel_layout=5.1:channels=LFE[LFE]' -map '[LFE]' lfe.wav </pre>

 <h3 class="section">29.76 chorus# TOC</h3> Add a chorus effect to the audio. Can make a single vocal sound like a chorus, but can also be applied to instrumentation. Chorus resembles an echo effect with a short delay, but whereas with echo the delay is constant, with chorus, it is varied using sinusoidal or triangular modulation. The modulation depth defines the range the modulated delay is played before or after the delay. Hence the delayed sound will sound slower or faster, that is the delayed sound tuned around the original one, like in a chorus where some vocals are slightly off key. It accepts the following parameters: <dl class="table"> <dt><samp class="option">in_gain</samp></dt> <dd> Set input gain. Default is 0.4. </dd> <dt><samp class="option">out_gain</samp></dt> <dd> Set output gain. Default is 0.4. </dd> <dt><samp class="option">delays</samp></dt> <dd> Set delays. A typical delay is around 40ms to 60ms. </dd> <dt><samp class="option">decays</samp></dt> <dd> Set decays. </dd> <dt><samp class="option">speeds</samp></dt> <dd> Set speeds. </dd> <dt><samp class="option">depths</samp></dt> <dd> Set depths. </dd> </dl>

 <h4 class="subsection">29.76.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A single delay: <pre class="example-preformatted">chorus=0.7:0.9:55:0.4:0.25:2 </pre>

Two delays: <pre class="example-preformatted">chorus=0.6:0.9:50|60:0.4|0.32:0.25|0.4:2|1.3 </pre>

Fuller sounding chorus with three delays: <pre class="example-preformatted">chorus=0.5:0.9:50|60|40:0.4|0.32|0.3:0.25|0.4|0.3:2|2.3|1.3 </pre>

 <h3 class="section">29.77 compand# TOC</h3> Compress or expand the audio’s dynamic range. It accepts the following parameters: <dl class="table"> <dt><samp class="option">attacks</samp></dt> <dt><samp class="option">decays</samp></dt> <dd> A list of times in seconds for each channel over which the instantaneous level of the input signal is averaged to determine its volume. <var class="var">attacks</var> refers to increase of volume and <var class="var">decays</var> refers to decrease of volume. For most situations, the attack time (response to the audio getting

louder) should be shorter than the decay time, because the human ear is more sensitive to sudden loud audio than sudden soft audio. A typical value for attack is 0.3 seconds and a typical value for decay is 0.8 seconds. If specified number of attacks & decays is lower than number of channels, the last set attack/decay will be used for all remaining channels. </dd> <dt><samp class="option">points</samp></dt> <dd> A list of points for the transfer function, specified in dB relative to the maximum possible signal amplitude. Each key points list must be defined using the following syntax: <code class="code">x0/y0|x1/y1|x2/y2|....</code> or <code class="code">x0/y0 x1/y1 x2/y2 ....</code> The input values must be in strictly increasing order but the transfer function does not have to be monotonically rising. The point <code class="code">0/0</code> is assumed but may be overridden (by <code class="code">0/out-dBn</code>). Typical values for the transfer function are <code class="code">-70/-70|-60/-20|1/0</code>. </dd> <dt><samp class="option">soft-knee</samp></dt> <dd> Set the curve radius in dB for all joints. It defaults to 0.01. </dd> <dt><samp class="option">gain</samp></dt> <dd> Set the additional gain in dB to be applied at all points on the transfer function. This allows for easy adjustment of the overall gain. It defaults to 0. </dd> <dt><samp class="option">volume</samp></dt> <dd> Set an initial volume, in dB, to be assumed for each channel when filtering starts. This permits the user to supply a nominal level initially, so that, for example, a very large gain is not applied to initial signal levels before the companding has begun to operate. A typical value for audio which is initially quiet is -90 dB. It defaults to 0. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set a delay, in seconds. The input audio is analyzed immediately, but audio is delayed before being fed to the volume adjuster. Specifying a delay approximately equal to the attack/decay times allows the filter to effectively operate in predictive rather than reactive mode. It defaults to 0. </dd> </dl>

 <h4 class="subsection">29.77.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make music with both quiet and loud passages suitable for listening to in a noisy environment: <pre class="example-preformatted">compand=.3|.3:1|1:-90/-60|-60/-40|-40/-30|-20/-20:6:0:-90:0.2 </pre>

Another example for audio with whisper and explosion parts: <pre class="example-preformatted">compand=0|0:1|1:-90/-900|-70/-70|-30/-9|0/-3:6:0:0:0 </pre>

A noise gate for when the noise is at a lower level than the signal: <pre class="example-preformatted">compand=.1|.1:.2|.2:-900/-900|-50.1/-900|-50/-50:.01:0:-90:.1 </pre>

Here is another noise gate, this time for when the noise is at a higher level than the signal (making it, in some ways, similar to squelch): <pre class=”example-preformatted”>compand=.1|.1:.1|.1:-45.1/-45.1|-45/-900|0/-900:.01:45:-90:.1 </pre>

2:1 compression starting at -6dB: <pre class=”example-preformatted”>compand=points=-80/-80|-6/-6|0/-3.8|20/3.5 </pre>

2:1 compression starting at -9dB: <pre class=”example-preformatted”>compand=points=-80/-80|-9/-9|0/-5.3|20/2.9 </pre>

2:1 compression starting at -12dB: <pre class=”example-preformatted”>compand=points=-80/-80|-12/-12|0/-6.8|20/1.9 </pre>

2:1 compression starting at -18dB: <pre class=”example-preformatted”>compand=points=-80/-80|-18/-18|0/-9.8|20/0.7 </pre>

3:1 compression starting at -15dB: <pre class=”example-preformatted”>compand=points=-80/-80|-15/-15|0/-10.8|20/-5.2 </pre>

Compressor/Gate: <pre class=”example-preformatted”>compand=points=-80/-105|-62/-80|-15.4/-15.4|0/-12|20/-7.6 </pre>

Expander: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-169|-54/-80|-49.5/-64.6|-41.1/-41.1|-25.8/-15|-10.8/-4.5|0/0|20/8.3 </pre>

Hard limiter at -6dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-6/-6|20/-6 </pre>

Hard limiter at -12dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12/-12|20/-12 </pre>

Hard noise gate at -35 dB: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-115|-35.1/-80|-35/-35|20/20 </pre>

Soft limiter: <pre class=”example-preformatted”>compand=attacks=0:points=-80/-80|-12.4/-12.4|-6/-8|0/-6.8|20/-2.8 </pre>

 <h3 class=”section”>29.78 compensationdelay# TOC</h3>

Compensation Delay Line is a metric based delay to compensate differing positions of microphones or speakers. For example, you have recorded guitar with two microphones placed in different locations. Because the front of sound wave has fixed speed in normal conditions, the phasing of microphones can vary and depends on their location and interposition. The best sound mix can be achieved when these microphones are in phase (synchronized). Note that a distance of ~30 cm between microphones makes one microphone capture the signal in antiphase to the other microphone. That makes the final mix sound moody. This filter helps to solve phasing problems by adding different delays to each microphone track and make them synchronized. The best result can be reached when you take one track as base and synchronize other tracks one by one with it. Remember that synchronization/delay tolerance depends on sample rate, too. Higher sample rates will give more tolerance. The filter accepts the following parameters: <dl class="table"> <dt><samp class="option">mm</samp></dt> <dd> Set millimeters distance. This is compensation distance for fine tuning. Default is 0. </dd> <dt><samp class="option">cm</samp></dt> <dd> Set cm distance. This is compensation distance for tightening distance setup. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set meters distance. This is compensation distance for hard distance setup. Default is 0. </dd> <dt><samp class="option">dry</samp></dt> <dd> Set dry amount. Amount of unprocessed (dry) signal. Default is 0. </dd> <dt><samp class="option">wet</samp></dt> <dd> Set wet amount. Amount of processed (wet) signal. Default is 1. </dd> <dt><samp class="option">temp</samp></dt> <dd> Set temperature in degrees Celsius. This is the temperature of the environment. Default is 20. </dd> </dl>

 <h4 class="subsection">29.78.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.79 crossfeed# TOC</h3> Apply headphone crossfeed filter. Crossfeed is the process of blending the left and right channels of stereo audio recording. It is mainly used to reduce extreme stereo separation of low frequencies. The intent is to produce more speaker like sound to the listener. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Set strength of crossfeed. Default is 0.2. Allowed range is from 0 to 1. This sets gain of low shelf filter for side part of stereo image. Default is -6dB. Max allowed is -30db when strength

is set to 1. </dd> <dt><samp class="option">range</samp></dt> <dd> Set soundstage wideness. Default is 0.5. Allowed range is from 0 to 1. This sets cut off frequency of low shelf filter. Default is cut off near 1550 Hz. With range set to 1 cut off frequency is set to 2100 Hz. </dd> <dt><samp class="option">slope</samp></dt> <dd> Set curve slope of low shelf filter. Default is 0.5. Allowed range is from 0.01 to 1. </dd> <dt><samp class="option">level_in</samp></dt> <dd> Set input gain. Default is 0.9. </dd> <dt><samp class="option">level_out</samp></dt> <dd> Set output gain. Default is 1. </dd> <dt><samp class="option">block_size</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.79.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.80 crystalizer# TOC</h3> Simple algorithm for audio noise sharpening. This filter linearly increases differences between each audio sample. The filter accepts the following options: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Sets the intensity of effect (default: 2.0). Must be in range between -10.0 to 0 (unchanged sound) to 10.0 (maximum effect). To inverse filtering use negative value. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">29.80.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.81 dcshift# TOC</h3> Apply a DC shift to the audio. This can be useful to remove a DC offset (caused perhaps by a hardware problem in the recording chain) from the audio.

The effect of a DC offset is reduced headroom and hence volume. The astats filter can be used to determine if a signal has a DC offset. <dl class="table"> <dt><samp class="option">shift</samp></dt> <dd> Set the DC shift, allowed range is [-1, 1]. It indicates the amount to shift the audio. </dd> <dt><samp class="option">limitergain</samp></dt> <dd> Optional. It should have a value much less than 1 (e.g. 0.05 or 0.02) and is used to prevent clipping. </dd> </dl>

 <h3 class="section">29.82 deesser# TOC</h3>

Apply de-essing to the audio samples. <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Set intensity for triggering de-essing. Allowed range is from 0 to 1. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set amount of ducking on treble part of sound. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">f</samp></dt> <dd> How much of original frequency content to keep when de-essing. Allowed range is from 0 to 1. Default is 0.5. </dd> <dt><samp class="option">s</samp></dt> <dd> Set the output mode. It accepts the following values: <dl class="table"> <dt><samp class="option">i</samp></dt> <dd> Pass input unchanged. </dd> <dt><samp class="option">o</samp></dt> <dd> Pass ess filtered out. </dd> <dt><samp class="option">e</samp></dt> <dd> Pass only ess. Default value is <var class="var">o</var>. </dd> </dl>

</dd> </dl>

 <h3 class="section">29.83 dialoguenhance# TOC</h3> Enhance dialogue in stereo audio. This filter accepts stereo input and produce surround (3.0) channels output. The newly produced front center channel have enhanced speech dialogue originally available in both stereo channels. This filter outputs front left and front right channels same as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">original</samp></dt> <dd> Set the original center factor to keep in front center channel output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">enhance</samp></dt> <dd> Set the dialogue enhance factor to put in front center channel output. Allowed range is from 0 to 3. Default value is 1. </dd> <dt><samp class="option">voice</samp></dt> <dd> Set the voice detection

factor. Allowed range is from 2 to 32. Default value is 2. </dd> </dl>

 <h4 class="subsection">29.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.84 drmeter# TOC</h3> Measure audio dynamic range. DR values of 14 and higher is found in very dynamic material. DR of 8 to 13 is found in transition material. And anything less that 8 have very poor dynamics and is very compressed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">length</samp></dt> <dd> Set window length in seconds used to split audio into segments of equal length. Default is 3 seconds. </dd> </dl>

<h3 class="section">29.85 dynaudnorm# TOC</h3>

Dynamic Audio Normalizer.

This filter applies a certain amount of gain to the input audio in order

to bring its peak magnitude to a target level (e.g. 0 dBFS). However, in

contrast to more "simple" normalization algorithms, the Dynamic Audio

Normalizer *dynamically* re-adjusts the gain factor to the input audio.

This allows for applying extra gain to the "quiet" sections of the audio

while avoiding distortions or clipping the "loud" sections. In other words:

The Dynamic Audio Normalizer will "even out" the volume of quiet and loud

sections, in the sense that the volume of each section is brought to the

same target level. Note, however, that the Dynamic Audio Normalizer achieves

this goal *without* applying "dynamic range compressing". It will retain 100%

of the dynamic range *within* each section of the audio file.

<dl class="table">

<dt><samp class="option">framelen, f</samp></dt>

<dd> Set the frame length in milliseconds. In range from 10 to 8000 milliseconds.

Default is 500 milliseconds.

The Dynamic Audio Normalizer processes the input audio in small chunks,

referred to as frames. This is required, because a peak magnitude has no

meaning for just a single sample value. Instead, we need to determine the

peak magnitude for a contiguous sequence of sample values. While a "standard"

normalizer would simply use the peak magnitude of the complete file, the

Dynamic Audio Normalizer determines the peak magnitude individually for each

frame. The length of a frame is specified in milliseconds. By default, the

Dynamic Audio Normalizer uses a frame length of 500 milliseconds, which has

been found to give good results with most files.

Note that the exact frame length, in number of samples, will be determined

automatically, based on the sampling rate of the individual input audio file.

</dd>

<dt><samp class="option">gausssize, g</samp></dt>

<dd> Set the Gaussian filter window size. In range from 3 to 301, must be odd

number. Default is 31.

Probably the most important parameter of the Dynamic Audio Normalizer is the

<code class="code">window size</code> of the Gaussian smoothing filter. The filter’s window size

is specified in frames, centered around the current frame. For the sake of

simplicity, this must be an odd number. Consequently, the default value of 31

takes into account the current frame, as well as the 15 preceding frames and

the 15 subsequent frames. Using a larger window results in a stronger

smoothing effect and thus in less gain variation, i.e. slower gain

adaptation. Conversely, using a smaller window results in a weaker smoothing

effect and thus in more gain variation, i.e. faster gain adaptation.

In other words, the more you increase this value, the more the Dynamic Audio

Normalizer will behave like a "traditional" normalization filter. On the

contrary, the more you decrease this value, the more the Dynamic Audio

Normalizer will behave like a dynamic range compressor.

</dd>

<dt><samp class="option">peak, p</samp></dt>

<dd> Set the target peak value. This specifies the highest permissible magnitude

level for the normalized audio input. This filter will try to approach the

target peak magnitude as closely as possible, but at the same time it also

makes sure that the normalized signal will never exceed the peak magnitude.

A frame’s maximum local gain factor is imposed directly by the target peak

magnitude. The default value is 0.95 and thus leaves a headroom of 5%*.

It is not recommended to go above this value.

</dd>

<dt><samp class="option">maxgain, m</samp></dt>

<dd> Set the maximum gain factor. In range from 1.0 to 100.0. Default is 10.0.

The Dynamic Audio Normalizer determines the maximum possible (local) gain

factor for each input frame, i.e. the maximum gain factor that does not

result in clipping or distortion. The maximum gain factor is determined by

the frame’s highest magnitude sample. However, the Dynamic Audio Normalizer

additionally bounds the frame’s maximum gain factor by a predetermined

(global) maximum gain factor. This is done in order to avoid excessive gain

factors in "silent" or almost silent frames. By default, the maximum gain

factor is 10.0, For most inputs the default value should be sufficient and

it usually is not recommended to increase this value. Though, for input

with an extremely low overall volume level, it may be necessary to allow even

higher gain factors. Note, however, that the Dynamic Audio Normalizer does

not simply apply a "hard" threshold (i.e. cut off values above the threshold).

Instead, a "sigmoid" threshold function will be applied. This way, the

gain factors will smoothly approach the threshold value, but never exceed that

value.

</dd>

<dt><samp class="option">targetrms, r</samp></dt>

<dd> Set the target RMS. In range from 0.0 to 1.0. Default is 0.0 - disabled.

By default, the Dynamic Audio Normalizer performs "peak" normalization.

This means that the maximum local gain factor for each frame is defined

(only) by the frame’s highest magnitude sample. This way, the samples can

be amplified as much as possible without exceeding the maximum signal

level, i.e. without clipping. Optionally, however, the Dynamic Audio

Normalizer can also take into account the frame’s root mean square,

abbreviated RMS. In electrical engineering, the RMS is commonly used to

determine the power of a time-varying signal. It is therefore considered

that the RMS is a better approximation of the "perceived loudness" than

just looking at the signal’s peak magnitude. Consequently, by adjusting all

frames to a constant RMS value, a uniform "perceived loudness" can be

established. If a target RMS value has been specified, a frame’s local gain

factor is defined as the factor that would result in exactly that RMS value.

Note, however, that the maximum local gain factor is still restricted by the

frame’s highest magnitude sample, in order to prevent clipping.

</dd>

<dt><samp class="option">coupling, n</samp></dt>

<dd> Enable channels coupling. By default is enabled.

By default, the Dynamic Audio Normalizer will amplify all channels by the same

amount. This means the same gain factor will be applied to all channels, i.e.

the maximum possible gain factor is determined by the "loudest" channel.

However, in some recordings, it may happen that the volume of the different

channels is uneven, e.g. one channel may be "quieter" than the other one(s).

In this case, this option can be used to disable the channel coupling. This way,

the gain factor will be determined independently for each channel, depending

only on the individual channel’s highest magnitude sample. This allows for

harmonizing the volume of the different channels.

</dd>

<dt><samp class="option">correctdc, c</samp></dt>

<dd> Enable DC bias correction. By default is disabled.

An audio signal (in the time domain) is a sequence of sample values.

In the Dynamic Audio Normalizer these sample values are represented in the

-1.0 to 1.0 range, regardless of the original input format. Normally, the

audio signal, or "waveform", should be centered around the zero point.

That means if we calculate the mean value of all samples in a file, or in a

single frame, then the result should be 0.0 or at least very close to that

value. If, however, there is a significant deviation of the mean value from

0.0, in either positive or negative direction, this is referred to as a

DC bias or DC offset. Since a DC bias is clearly undesirable, the Dynamic

Audio Normalizer provides optional DC bias correction.

With DC bias correction enabled, the Dynamic Audio Normalizer will determine

the mean value, or "DC correction" offset, of each input frame and subtract

that value from all of the frame’s sample values which ensures those samples

are centered around 0.0 again. Also, in order to avoid "gaps" at the frame

boundaries, the DC correction offset values will be interpolated smoothly

between neighbouring frames.

</dd>

<dt><samp class="option">altboundary, b</samp></dt>

<dd> Enable alternative boundary mode. By default is disabled.

The Dynamic Audio Normalizer takes into account a certain neighbourhood

around each frame. This includes the preceding frames as well as the

subsequent frames. However, for the "boundary" frames, located at the very

beginning and at the very end of the audio file, not all neighbouring

frames are available. In particular, for the first few frames in the audio

file, the preceding frames are not known. And, similarly, for the last few

frames in the audio file, the subsequent frames are not known. Thus, the

question arises which gain factors should be assumed for the missing frames

in the "boundary" region. The Dynamic Audio Normalizer implements two modes

to deal with this situation. The default boundary mode assumes a gain factor

of exactly 1.0 for the missing frames, resulting in a smooth "fade in" and

"fade out" at the beginning and at the end of the input, respectively.

</dd>
<dt><samp class=”option”>compress, s</samp></dt>
<dd> Set the compress factor. In range from 0.0 to 30.0. Default is 0.0.
By default, the Dynamic Audio Normalizer does not apply "traditional"
compression. This means that signal peaks will not be pruned and thus the
full dynamic range will be retained within each local neighbourhood. However,
in some cases it may be desirable to combine the Dynamic Audio Normalizer’s
normalization algorithm with a more "traditional" compression.
For this purpose, the Dynamic Audio Normalizer provides an optional compression
(thresholding) function. If (and only if) the compression feature is enabled,
all input frames will be processed by a soft knee thresholding function prior
to the actual normalization process. Put simply, the thresholding function is
going to prune all samples whose magnitude exceeds a certain threshold value.
However, the Dynamic Audio Normalizer does not simply apply a fixed threshold
value. Instead, the threshold value will be adjusted for each individual
frame.
In general, smaller parameters result in stronger compression, and vice versa.
Values below 3.0 are not recommended, because audible distortion may appear.

</dd>
<dt><samp class=”option”>threshold, t</samp></dt>
<dd> Set the target threshold value. This specifies the lowest permissible
magnitude level for the audio input which will be normalized.
If input frame volume is above this value frame will be normalized.
Otherwise frame may not be normalized at all. The default value is set
to 0, which means all input frames will be normalized.
This option is mostly useful if digital noise is not wanted to be amplified.

</dd>

<dt><samp class=”option”>channels, h</samp></dt>

<dd> Specify which channels to filter, by default all available channels are filtered.

</dd>

<dt><samp class=”option”>overlap, o</samp></dt>

<dd> Specify overlap for frames. If set to 0 (default) no frame overlapping is done.

Using >0 and <1 values will make less conservative gain adjustments, like

when framelen option is set to smaller value, if framelen option value is

compensated for non-zero overlap then gain adjustments will be smoother across time

compared to zero overlap case.

</dd>

<dt><samp class=”option”>curve, v</samp></dt>

<dd> Specify the peak mapping curve expression which is going to be used when calculating

gain applied to frames. The max output frame gain will still be limited by other

options mentioned previously for this filter.

The expression can contain the following constants:

<dl class=”table”>

<dt><samp class=”option”>ch</samp></dt>

<dd> current channel number

</dd>

<dt><samp class=”option”>sn</samp></dt>

<dd> current sample number

</dd>

<dt><samp class="option">nb_channels</samp></dt>

<dd> number of channels

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds

</dd>

<dt><samp class="option">sr</samp></dt>

<dd> sample rate

</dd>

<dt><samp class="option">p</samp></dt>

<dd> current frame peak value

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">29.85.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">29.86 earwax# TOC</h3>

Make audio easier to listen to on headphones. This filter adds ‘cues’ to 44.1kHz stereo (i.e. audio CD format) audio so that when listened to on headphones the stereo image is moved from inside your head (standard for headphones) to outside and in front of the listener (standard for speakers). Ported from SoX.
 <h3 class="section">29.87 equalizer# TOC</h3>

Apply a two-pole peaking equalisation (EQ) filter. With this filter, the signal-level at and around a selected frequency can be increased or decreased, whilst (unlike bandpass and bandreject filters) that at all other frequencies is unchanged. In order to produce complex equalisation curves, this filter can be given several times, each with a different central frequency. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency in Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Set the required gain or attenuation in dB. Beware of clipping when using a positive gain. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just

produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.87.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Attenuate 10 dB at 1000 Hz, with a bandwidth of 200 Hz: <pre class="example-preformatted">equalizer=f=1000:t=h:width=200:g=-10 </pre>

Apply 2 dB gain at 1000 Hz with Q 1 and attenuate 5 dB at 100 Hz with Q 2: <pre class="example-preformatted">equalizer=f=1000:t=q:w=1:g=2,equalizer=f=100:t=q:w=2:g=-5 </pre>

 <h4 class="subsection">29.87.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change equalizer frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change equalizer width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change equalizer width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change equalizer gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change equalizer mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">29.88 extrastereo# TOC</h3>

Linearly increases the difference between left and right channels which adds some sort of "live" effect to playback. The filter accepts the following options: <dl class="table"> <dt><samp class="option">m</samp></dt> <dd> Sets the difference coefficient (default: 2.5). 0.0 means mono sound (average of both channels), with 1.0 sound will be unchanged, with -1.0 left and right channels will be swapped. </dd> <dt><samp class="option">c</samp></dt> <dd> Enable clipping. By default is enabled. </dd> </dl>

 <h4 class="subsection">29.88.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.89 firequalizer# TOC</h3> Apply FIR Equalization using arbitrary frequency response. The filter accepts the following option: <dl class="table"> <dt><samp class="option">gain</samp></dt> <dd> Set gain curve equation (in dB). The expression can contain variables: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> the evaluated frequency </dd> <dt><samp class="option">sr</samp></dt> <dd> sample rate </dd> <dt><samp class="option">ch</samp></dt> <dd> channel number, set to 0 when multichannels evaluation is disabled </dd> <dt><samp class="option">chid</samp></dt> <dd> channel id, see libavutil/channel_layout.h, set to the first channel id when multichannels evaluation is disabled </dd> <dt><samp class="option">chs</samp></dt> <dd> number of channels </dd> <dt><samp class="option">chlayout</samp></dt> <dd> channel_layout, see libavutil/channel_layout.h </dd> </dl> and functions: <dl class="table"> <dt><samp class="option">gain_interpolate(f)</samp></dt> <dd> interpolate gain on frequency f based on gain_entry </dd> <dt><samp class="option">cubic_interpolate(f)</samp></dt> <dd> same as gain_interpolate, but smoother </dd> </dl> This option is also available as command. Default is <code class="code">gain_interpolate(f)</code>. </dd> <dt><samp class="option">gain_entry</samp></dt> <dd> Set gain entry for gain_interpolate function. The expression can contain functions: <dl class="table"> <dt><samp class="option">entry(f, g)</samp></dt> <dd> store gain entry at frequency f with value g </dd> </dl> This option is also available as command. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set filter delay in seconds. Higher value means more accurate. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set filter accuracy in Hz. Lower value means more accurate. Default is <code class="code">5</code>. </dd> <dt><samp class="option">wfunc</samp></dt> <dd> Set window function. Acceptable values are: <dl class="table"> <dt><samp class="option">rectangular</samp></dt> <dd> rectangular window, useful when gain curve is already smooth </dd> <dt><samp class="option">hann</samp></dt> <dd> hann window (default) </dd> <dt><samp class="option">hamming</samp></dt> <dd> hamming window </dd> <dt><samp class="option">blackman</samp></dt>

<dd> blackman window </dd> <dt><samp class="option">nuttall3</samp></dt>
<dd> 3-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">mnuttall3</samp></dt> <dd> minimum
3-terms discontinuous nuttall window </dd> <dt><samp class="option">nuttall</samp></dt>
<dd> 4-terms continuous 1st derivative nuttall window </dd>
<dt><samp class="option">bnuttall</samp></dt> <dd> minimum
4-terms discontinuous nuttall (blackman-nuttall) window </dd>
<dt><samp class="option">bharris</samp></dt> <dd> blackman-
harris window </dd> <dt><samp class="option">tukey</samp></dt>
<dd> tukey window </dd> </dl>

</dd> <dt><samp class="option">fixed</samp></dt> <dd> If
enabled, use fixed number of audio samples. This improves speed when
filtering with large delay. Default is disabled. </dd> <dt><samp
class="option">multi</samp></dt> <dd> Enable multichannels
evaluation on gain. Default is disabled. </dd> <dt><samp
class="option">zero_phase</samp></dt> <dd> Enable zero phase
mode by subtracting timestamp to compensate delay. Default is disabled.
</dd> <dt><samp class="option">scale</samp></dt> <dd> Set scale
used by gain. Acceptable values are: <dl class="table"> <dt><samp
class="option">linlin</samp></dt> <dd> linear frequency, linear
gain </dd> <dt><samp class="option">linlog</samp></dt>
<dd> linear frequency, logarithmic (in dB) gain (default) </dd>
<dt><samp class="option">loglin</samp></dt> <dd> logarithmic (in
octave scale where 20 Hz is 0) frequency, linear gain </dd> <dt><samp
class="option">loglog</samp></dt> <dd> logarithmic frequency,
logarithmic gain </dd> </dl>

</dd> <dt><samp class="option">dumpfile</samp></dt> <dd> Set
file for dumping, suitable for gnuplot. </dd> <dt><samp
class="option">dumpscale</samp></dt> <dd> Set scale for dump-
file. Acceptable values are same with scale option. Default is lin-
log. </dd> <dt><samp class="option">fft2</samp></dt>
<dd> Enable 2-channel convolution using complex FFT. This im-
proves speed significantly. Default is disabled. </dd> <dt><samp
class="option">min_phase</samp></dt> <dd> Enable minimum
phase impulse response. Default is disabled. </dd> </dl>

 <h4 class="subsection">29.89.1 Exam-
ples<a class="anchor hidden-xs" href="#Examples-
43" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-43" aria-hidden="true">TOC</h4> <ul class="itemize
mark-bullet"> lowpass at 1000 Hz: <pre
class="example-preformatted">firequalizer=gain='if(lt(f,1000), 0, -INF)'
</pre> lowpass at 1000 Hz with gain_entry: <div
class="example"> <pre class="example-preformatted">firequalizer=gain_entry='entry(1000,0);
entry(1001, -INF)' </pre> custom equalization: <div

class=”example”> <pre class=”example-preformatted”>firequalizer=gain_entry='entry(100,0); entry(400, -4); entry(1000, -6); entry(2000, 0)' </pre> higher delay with zero phase to compensate delay: <pre class=”example-preformatted”>firequalizer=delay=0.1:fixed=on:zero_phase=on </pre> lowpass on left channel, highpass on right channel: <pre class=”example-preformatted”>firequalizer=gain='if(eq(chid,1), gain_interpolate(f), if(eq(chid,2), gain_interpolate(1e6+f), 0))' :gain_entry='entry(1000, 0); entry(1001,-INF); entry(1e6+1000,0)':multi=on </pre>

 <h3 class=”section”>29.90 flanger# TOC</h3> Apply a flanging effect to the audio. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay</samp></dt> <dd> Set base delay in milliseconds. Range from 0 to 30. Default value is 0. </dd> <dt><samp class=”option”>depth</samp></dt> <dd> Set added sweep delay in milliseconds. Range from 0 to 10. Default value is 2. </dd> <dt><samp class=”option”>regen</samp></dt> <dd> Set percentage regeneration (delayed signal feedback). Range from -95 to 95. Default value is 0. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set percentage of delayed signal mixed with original. Range from 0 to 100. Default value is 71. </dd> <dt><samp class=”option”>speed</samp></dt> <dd> Set sweeps per second (Hz). Range from 0.1 to 10. Default value is 0.5. </dd> <dt><samp class=”option”>shape</samp></dt> <dd> Set swept wave shape, can be <var class=”var”>triangular</var> or <var class=”var”>sinusoidal</var>. Default value is <var class=”var”>sinusoidal</var>. </dd> <dt><samp class=”option”>phase</samp></dt> <dd> Set swept wave percentage-shift for multi channel. Range from 0 to 100. Default value is 25. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Set delay-line interpolation, <var class=”var”>linear</var> or <var class=”var”>quadratic</var>. Default is <var class=”var”>linear</var>. </dd> </dl>

 <h3 class=”section”>29.91 haas# TOC</h3> Apply Haas effect to audio. Note that this makes most sense to apply on mono signals. With this filter applied to mono signals it give some directionality and stretches its stereo image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level. By default is <var class=”var”>1</var>, or 0dB </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level. By default is <var class=”var”>1</var>, or 0dB. </dd> <dt><samp class=”option”>side_gain</samp></dt> <dd> Set gain applied to side part of signal. By default is <var class=”var”>1</var>. </dd>

<dt><samp class="option">middle_source</samp></dt> <dd> Set kind of middle source. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">left</samp>’</dt> <dd> Pick left channel. </dd> <dt>‘<samp class="samp">right</samp>’</dt> <dd> Pick right channel. </dd> <dt>‘<samp class="samp">mid</samp>’</dt> <dd> Pick middle part signal of stereo image. </dd> <dt>‘<samp class="samp">side</samp>’</dt> <dd> Pick side part signal of stereo image. </dd> </dl>

</dd> <dt><samp class="option">middle_phase</samp></dt> <dd> Change middle phase. By default is disabled. </dd> <dt><samp class="option">left_delay</samp></dt> <dd> Set left channel delay. By default is <var class="var">2.05</var> milliseconds. </dd> <dt><samp class="option">left_balance</samp></dt> <dd> Set left channel balance. By default is <var class="var">-1</var>. </dd> <dt><samp class="option">left_gain</samp></dt> <dd> Set left channel gain. By default is <var class="var">1</var>. </dd> <dt><samp class="option">left_phase</samp></dt> <dd> Change left phase. By default is disabled. </dd> <dt><samp class="option">right_delay</samp></dt> <dd> Set right channel delay. By defaults is <var class="var">2.12</var> milliseconds. </dd> <dt><samp class="option">right_balance</samp></dt> <dd> Set right channel balance. By default is <var class="var">1</var>. </dd> <dt><samp class="option">right_gain</samp></dt> <dd> Set right channel gain. By default is <var class="var">1</var>. </dd> <dt><samp class="option">right_phase</samp></dt> <dd> Change right phase. By default is enabled. </dd> </dl>

 <h3 class="section">29.92 hdcd# TOC</h3>

Decodes High Definition Compatible Digital (HDCD) data. A 16-bit PCM stream with embedded HDCD codes is expanded into a 20-bit PCM stream. The filter supports the Peak Extend and Low-level Gain Adjustment features of HDCD, and detects the Transient Filter flag.   <pre class="example-preformatted">ffmpeg -i HDCD16.flac -af hdcd OUT24.flac </pre>

When using the filter with wav, note the default encoding for wav is 16-bit, so the resulting 20-bit stream will be truncated back to 16-bit. Use something like <code class="command">-acodec pcm_s24le</code> after the filter to get 24-bit PCM output. <pre class="example-preformatted">ffmpeg -i HDCD16.wav -af hdcd OUT16.wav ffmpeg -i HDCD16.wav -af hdcd -c:a pcm_s24le OUT24.wav </pre>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">disable_autoconvert</samp></dt> <dd> Disable any automatic format conversion or resampling in the filter graph.

 </dd> <dt><samp class=”option”>process_stereo</samp></dt> <dd> Process the stereo channels together. If target_gain does not match between channels, consider it invalid and use the last valid target_gain. </dd> <dt><samp class=”option”>cdt_ms</samp></dt> <dd> Set the code detect timer period in ms. </dd> <dt><samp class=”option”>force_pe</samp></dt> <dd> Always extend peaks above -3dBFS even if PE isn’t signaled. </dd> <dt><samp class=”option”>analyze_mode</samp></dt> <dd> Replace audio with a solid tone and adjust the amplitude to signal some specific aspect of the decoding process. The output file can be loaded in an audio editor alongside the original to aid analysis. <code class=”code”>analyze_mode=pe:force_pe=true</code> can be used to see all samples above the PE level. Modes are: <dl class=”table”> <dt>‘<samp class=”samp”>0, off</samp>’</dt> <dd> Disabled </dd> <dt>‘<samp class=”samp”>1, lle</samp>’</dt> <dd> Gain adjustment level at each sample </dd> <dt>‘<samp class=”samp”>2, pe</samp>’</dt> <dd> Samples where peak extend occurs </dd> <dt>‘<samp class=”samp”>3, cdt</samp>’</dt> <dd> Samples where the code detect timer is active </dd> <dt>‘<samp class=”samp”>4, tgm</samp>’</dt> <dd> Samples where the target gain does not match between channels </dd> </dl> </dd> </dl>

 <h3 class=”section”>29.93 headphone# TOC</h3>

Apply head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones. The HRIRs are provided via additional streams, for each channel one stereo input stream is needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>map</samp></dt> <dd> Set mapping of input streams for convolution. The argument is a ’|’-separated list of channel names in order as they are given as additional stream inputs for filter. This also specify number of input streams. Number of input streams must be not less than number of channels in first stream plus one. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>lfe</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set size of frame in num-

ber of samples which will be processed at once. Default value is <var class=”var”>1024</var>. Allowed range is from 1024 to 96000. </dd> <dt><samp class=”option”>hrir</samp></dt> <dd> Set format of hrir stream. Default value is <var class=”var”>stereo</var>. Alternative value is <var class=”var”>multich</var>. If value is set to <var class=”var”>stereo</var>, number of additional streams should be greater or equal to number of input channels in first input stream. Also each additional stream should have stereo number of channels. If value is set to <var class=”var”>multich</var>, number of additional streams should be exactly one. Also number of input channels of additional stream should be equal or greater than twice number of channels of first input stream. </dd> </dl>

 <h4 class=”subsection”>29.93.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Full example using wav files as coefficients with amovie filters for 7.1 downmix, each amovie filter use stereo file with IR coefficients as input. The files give coefficients for each position of virtual loudspeaker: <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=azi_270_ele_0_DFC.wav[sr];amovie=azi_90_ele_0_DFC.wav[sl];amovie=azi_225_ele_0_D
output.wav </pre>

Full example using wav files as coefficients with amovie filters for 7.1 downmix, but now in <var class=”var”>multich</var> <var class=”var”>hrir</var> format. <pre class=”example-preformatted”>ffmpeg -i input.wav -filter_complex "amovie=minp.wav[hrirs];[0:a][hrirs]
output.wav </pre>

 <h3 class=”section”>29.94 highpass# TOC</h3>

Apply a high-pass filter with 3dB point frequency. The filter can be either single-pole, or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 3000. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt>

<dd> octave </dd> <dt><samp class="option">s</samp></dt>
<dd> slope </dd> <dt><samp class="option">k</samp></dt>
<dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.94.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change highpass frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change highpass width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd>

<dt><samp class=”option”>width, w</samp></dt> <dd> Change highpass width. Syntax for the command is : "<var class=”var”>width</var>" </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change highpass mix. Syntax for the command is : "<var class=”var”>mix</var>" </dd> </dl>

 <h3 class=”section”>29.95 join# TOC</h3>

Join multiple input streams into one multi-channel stream. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> The number of input streams. It defaults to 2. </dd> <dt><samp class=”option”>channel_layout</samp></dt> <dd> The desired output channel layout. It defaults to stereo. </dd> <dt><samp class=”option”>map</samp></dt> <dd> Map channels from inputs to output. The argument is a ’|’-separated list of mappings, each in the <code class=”code”><var class=”var”>input_idx</var>.<var class=”var”>in_channel</var>-<var class=”var”>out_channel</var></code> form. <var class=”var”>input_idx</var> is the 0-based index of the input stream. <var class=”var”>in_channel</var> can be either the name of the input channel (e.g. FL for front left) or its index in the specified input stream. <var class=”var”>out_channel</var> is the name of the output channel. </dd> </dl>

The filter will attempt to guess the mappings when they are not specified explicitly. It does so by first trying to find an unused matching input channel and if that fails it picks the first unused input channel. Join 3 inputs (with properly set channel layouts): <pre class=”example-preformatted”>ffmpeg -i INPUT1 -i INPUT2 -i INPUT3 -filter_complex join=inputs=3 OUTPUT </pre>

Build a 5.1 output from 6 single-channel streams: <pre class=”example-preformatted”>ffmpeg -i fl -i fr -i fc -i sl -i sr -i lfe -filter_complex 'join=inputs=6:channel_layout=5.1:map=0.0-FL|1.0-FR|2.0-FC|3.0-SL|4.0-SR|5.0-LFE' out </pre>

 <h3 class=”section”>29.96 ladspa# TOC</h3>

Load a LADSPA (Linux Audio Developer’s Simple Plugin API) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-ladspa</code>. <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> Specifies the name of LADSPA plugin library to load. If the environment variable <code class=”env”>LADSPA_PATH</code> is defined, the LADSPA plugin is searched in each one of the directories specified by the colon separated list in <code class=”env”>LADSPA_PATH</code>,

otherwise in the standard LADSPA paths, which are in this order: <samp class="file">HOME/.ladspa/lib/</samp>, <samp class="file">/usr/local/lib/ladspa/</samp>, <samp class="file">/usr/lib/ladspa/</samp>. </dd> <dt><samp class="option">plugin, p</samp></dt> <dd> Specifies the plugin within the library. Some libraries contain only one plugin, but others contain many of them. If this is not set filter will list all available plugins within the specified library. </dd> <dt><samp class="option">controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). Controls need to be defined using the following syntax: c0=<var class="var">value0</var>|c1=<var class="var">value1</var>|c2=<var class="var">value2</var>|..., where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. Alternatively they can be also defined using the following syntax: <var class="var">value0</var>|<var class="var">value1</var>|<var class="var">value2</var>|..., where <var class="var">valuei</var> is the value set on the <var class="var">i</var>-th control. If <samp class="option">controls</samp> is set to <code class="code">help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> <dt><samp class="option">latency, l</samp></dt> <dd> Enable latency compensation, by default is disabled. Only used if plugin have inputs. </dd> </dl>

 <h4 class="subsection">29.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all available plugins within amp (LADSPA example plugin) library: <pre class="example-preformatted">ladspa=file=amp </pre>

List all available controls and their valid ranges for <code class="code">vcf_notch</code> plugin from <code class="code">VCF</code>

library: <pre class=”example-preformatted”>ladspa=f=vcf:p=vcf_notch:c=help
</pre>

Simulate low quality audio equipment using <code class=”code”>Computer
Music Toolkit</code> (CMT) plugin library: <pre
class=”example-preformatted”>ladspa=file=cmt:plugin=lofi:controls=c0=22|c1=12|c2=12
</pre>

Add reverberation to the audio using TAP-plugins (Tom’s
Audio Processing plugins): <pre class=”example-
preformatted”>ladspa=file=tap_reverb:tap_reverb </pre>

Generate white noise, with 0.2 amplitude:
<pre class=”example-preformatted”>ladspa=file=cmt:noise_source_white:c=c0=.2
</pre>

Generate 20 bpm clicks using plugin <code class=”code”>C*
Click - Metronome</code> from the <code class=”code”>C* Audio Plugin
Suite</code> (CAPS) library: <pre class=”example-
preformatted”>ladspa=file=caps:Click:c=c1=20' </pre>

Apply <code class=”code”>C* Eq10X2 - Stereo 10-band
equaliser</code> effect: <pre class=”example-
preformatted”>ladspa=caps:Eq10X2:c=c0=-48|c9=-24|c3=12|c4=2 </pre>

Increase volume by 20dB using fast lookahead limiter from
Steve Harris <code class=”code”>SWH Plugins</code> collection: <div
class=”example”> <pre class=”example-preformatted”>ladspa=fast_lookahead_limiter_1913:fastLookahead
</pre>

Attenuate low frequencies using Multiband EQ from Steve Harris
<code class=”code”>SWH Plugins</code> collection:
<pre class=”example-preformatted”>ladspa=mbeq_1197:mbeq:-24|-24|-
24|0|0|0|0|0|0|0|0|0|0|0 </pre>

Reduce stereo image using <code class=”code”>Narrower</code>
from the <code class=”code”>C* Audio Plugin Suite</code> (CAPS) library:
 <pre class=”example-preformatted”>ladspa=caps:Narrower
</pre>

Another white noise, now using <code class=”code”>C* Au-
dio Plugin Suite</code> (CAPS) library: <pre
class=”example-preformatted”>ladspa=caps:White:.2 </pre>

Some fractal noise, using <code class=”code”>C* Audio Plugin
Suite</code> (CAPS) library: <pre class=”example-
preformatted”>ladspa=caps:Fractal:c=c1=1 </pre>

Dynamic volume normalization using <code class=”code”>VLevel</code>
plugin: <pre class=”example-preformatted”>ladspa=vlevel-
ladspa:vlevel_mono </pre>

 <h4 class="subsection">29.96.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">cN</samp></dt> <dd> Modify the <var class="var">N</var>-th control value. If the specified value is not valid, it is ignored and prior one is kept. </dd> </dl>

 <h3 class="section">29.97 loudnorm# TOC</h3>

EBU R128 loudness normalization. Includes both dynamic and linear normalization modes. Support for both single pass (livestreams, files) and double pass (files) modes. This algorithm can target IL, LRA, and maximum true peak. In dynamic mode, to accurately detect true peaks, the audio stream will be upsampled to 192 kHz. Use the <code class="code">-ar</code> option or <code class="code">aresample</code> filter to explicitly set an output sample rate. The filter accepts the following options: <dl class="table"> <dt><samp class="option">I, i</samp></dt> <dd> Set integrated loudness target. Range is -70.0 - -5.0. Default value is -24.0. </dd> <dt><samp class="option">LRA, lra</samp></dt> <dd> Set loudness range target. Range is 1.0 - 50.0. Default value is 7.0. </dd> <dt><samp class="option">TP, tp</samp></dt> <dd> Set maximum true peak. Range is -9.0 - +0.0. Default value is -2.0. </dd> <dt><samp class="option">measured_I, measured_i</samp></dt> <dd> Measured IL of input file. Range is -99.0 - +0.0. </dd> <dt><samp class="option">measured_LRA, measured_lra</samp></dt> <dd> Measured LRA of input file. Range is 0.0 - 99.0. </dd> <dt><samp class="option">measured_TP, measured_tp</samp></dt> <dd> Measured true peak of input file. Range is -99.0 - +99.0. </dd> <dt><samp class="option">measured_thresh</samp></dt> <dd> Measured threshold of input file. Range is -99.0 - +0.0. </dd> <dt><samp class="option">offset</samp></dt> <dd> Set offset gain. Gain is applied before the true-peak limiter. Range is -99.0 - +99.0. Default is +0.0. </dd> <dt><samp class="option">linear</samp></dt> <dd> Normalize by linearly scaling the source audio. <code class="code">measured_I</code>, <code class="code">measured_LRA</code>, <code class="code">measured_TP</code>, and <code class="code">measured_thresh</code> must all be specified. Target LRA shouldn’t be lower than source LRA and the change in integrated loudness shouldn’t result in a true peak which exceeds the target TP. If any of these conditions aren’t met, normalization mode will revert to <var class="var">dynamic</var>. Options are <code class="code">true</code> or <code class="code">false</code>.

Default is <code class=”code”>true</code>. </dd> <dt><samp class=”option”>dual_mono</samp></dt> <dd> Treat mono input files as "dual-mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. Options are true or false. Default is false. </dd> <dt><samp class=”option”>print_format</samp></dt> <dd> Set print format for stats. Options are summary, json, or none. Default value is none. </dd> <dt><samp class=”option”>stats_file</samp></dt> <dd> Write stats to specified file. Format is controlled by <samp class=”option”>print_format</samp>, which must be set. Specify <code class=”code”>-</code> to write to standard output. Default is unset. </dd> </dl>

 <h3 class=”section”>29.98 lowpass# TOC</h3>

Apply a low-pass filter with 3dB point frequency. The filter can be either single-pole or double-pole (the default). The filter roll off at 6dB per pole per octave (20dB per pole per decade). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>frequency, f</samp></dt> <dd> Set frequency in Hz. Default is 500. </dd> <dt><samp class=”option”>poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class=”option”>width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Hz </dd> <dt><samp class=”option”>q</samp></dt> <dd> Q-Factor </dd> <dt><samp class=”option”>o</samp></dt> <dd> octave </dd> <dt><samp class=”option”>s</samp></dt> <dd> slope </dd> <dt><samp class=”option”>k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class=”option”>width, w</samp></dt> <dd> Specify the band-width of a filter in width_type units. Applies only to double-pole filter. The default is 0.707q and gives a Butterworth response. </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class=”option”>channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class=”option”>normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class=”option”>transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class=”table”> <dt><samp

class=”option”>di</samp></dt> <dt><samp class=”option”>dii</samp></dt>
<dt><samp class=”option”>tdi</samp></dt> <dt><samp class=”option”>tdii</samp></dt>
<dt><samp class=”option”>latt</samp></dt> <dt><samp class=”option”>svf</samp></dt>
<dt><samp class=”option”>zdf</samp></dt> </dl>

</dd> <dt><samp class=”option”>precision, r</samp></dt> <dd> Set
precision of filtering. <dl class=”table”> <dt><samp class=”option”>auto</samp></dt>
<dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class=”option”>s16</samp></dt>
<dd> Always use signed 16-bit. </dd> <dt><samp class=”option”>s32</samp></dt>
<dd> Always use signed 32-bit. </dd> <dt><samp class=”option”>f32</samp></dt>
<dd> Always use float 32-bit. </dd> <dt><samp class=”option”>f64</samp></dt>
<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class=”option”>block_size, b</samp></dt> <dd> Set
block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class=”subsection”>29.98.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Lowpass only LFE channel, it LFE is not present it does nothing: <pre class=”example-preformatted”>lowpass=c=LFE
</pre>

 <h4 class=”subsection”>29.98.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>frequency, f</samp></dt> <dd> Change
lowpass frequency. Syntax for the command is : "<var class=”var”>frequency</var>"
 </dd> <dt><samp class=”option”>width_type, t</samp></dt>
<dd> Change lowpass width_type. Syntax for the command is : "<var class=”var”>width_type</var>" </dd>
<dt><samp class=”option”>width, w</samp></dt> <dd> Change lowpass width. Syntax for the command is : "<var class=”var”>width</var>"
 </dd> <dt><samp class=”option”>mix, m</samp></dt> <dd> Change
lowpass mix. Syntax for the command is : "<var class=”var”>mix</var>"
</dd> </dl>

 <h3 class=”section”>29.99 lv2#
TOC</h3>

Load a LV2 (LADSPA Version 2) plugin. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-lv2</code>. <dl class=”table”> <dt><samp class=”option”>plugin, p</samp></dt> <dd> Specifies the plugin URI. You may need to escape ’:’. </dd> <dt><samp class=”option”>controls, c</samp></dt> <dd> Set the ’|’ separated list of controls which are zero or more floating point values that determine the behavior of the loaded plugin (for example delay, threshold or gain). If <samp class=”option”>controls</samp> is set to <code class=”code”>help</code>, all available controls and their valid ranges are printed. </dd> <dt><samp class=”option”>sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. Only used if plugin have zero inputs. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default is 1024. Only used if plugin have zero inputs. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. Only used if plugin have zero inputs. </dd> </dl>

 <h4 class=”subsection”>29.99.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply bass enhancer plugin from Calf: <pre class=”example-preformatted”>lv2=p=http\\\\://calf.sourceforge.net/plugins/
</pre>

Apply vinyl plugin from Calf: <pre class=”example-preformatted”>lv2=p=http\\\\://calf.sourceforge.net/plugins/Vinyl:c=drone=0.2|aging=0.5
</pre>

Apply bit crusher plugin from ArtyFX: <pre class=”example-preformatted”>lv2=p=http\\\\://www.openavproductions.com/artyfx#bitta:c=crush=0.3
</pre>

 <h4 class=”subsection”>29.99.2 Commands# TOC</h4>

This filter supports all options that are exported by plugin as commands. <h3 class=”section”>29.100 mcom-

pand# TOC</h3> Multiband Compress or expand the audio’s dynamic range. The input audio is divided into bands using 4th order Linkwitz-Riley IIRs. This is akin to the crossover of a loudspeaker, and results in flat frequency response when absent compander action. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>args</samp></dt> <dd> This option syntax is: attack,decay,[attack,decay..] soft-knee points crossover_frequency [delay [initial_volume [gain]]] | attack,decay ... For explanation of each item refer to compand filter documentation. </dd> </dl>

 <h3 class=”section”>29.101 pan# TOC</h3>

Mix channels with specific gain levels. The filter accepts the output channel layout followed by a set of channels definitions. This filter is also designed to efficiently remap the channels of an audio stream. The filter accepts parameters of the form: "<var class=”var”>l</var>|<var class=”var”>outdef</var>|<var class=”var”>outdef</var>|..." <dl class=”table”> <dt><samp class=”option”>l</samp></dt> <dd> output channel layout or number of channels </dd> <dt><samp class=”option”>outdef</samp></dt> <dd> output channel specification, of the form: "<var class=”var”>out_name</var>=[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>[(+-)[<var class=”var”>gain</var>*]<var class=”var”>in_name</var>...]" </dd> <dt><samp class=”option”>out_name</samp></dt> <dd> output channel to define, either a channel name (FL, FR, etc.) or a channel number (c0, c1, etc.) </dd> <dt><samp class=”option”>gain</samp></dt> <dd> multiplicative coefficient for the channel, 1 leaving the volume unchanged </dd> <dt><samp class=”option”>in_name</samp></dt> <dd> input channel to use, see out_name for details; it is not possible to mix named and numbered input channels </dd> </dl>

If the ‘=’ in a channel specification is replaced by ‘<’, then the gains for that specification will be renormalized so that the total is 1, thus avoiding clipping noise. <h4 class=”subsection”>29.101.1 Mixing examples# TOC</h4>

For example, if you want to down-mix from stereo to mono, but with a bigger factor for the left channel: <pre

class=”example-preformatted”>pan=1c|c0=0.9*c0+0.1*c1 </pre>

A customized down-mix to stereo that works automatically for 3-, 4-, 5- and 7-channels surround: <pre class=”example-preformatted”>pan=stereo| FL < FL + 0.5*FC + 0.6*BL + 0.6*SL | FR < FR + 0.5*FC + 0.6*BR + 0.6*SR </pre>

Note that <code class=”command”>ffmpeg</code> integrates a default down-mix (and up-mix) system that should be preferred (see "-ac" option) unless you have very specific needs. <h4 class=”subsection”>29.101.2 Remapping examples# TOC</h4>

The channel remapping will be effective if, and only if: <ul class=”itemize mark-bullet”> gain coefficients are zeroes or ones, only one input per channel output,

If all these conditions are satisfied, the filter will notify the user ("Pure channel mapping detected"), and use an optimized and lossless method to do the remapping. For example, if you have a 5.1 source and want a stereo audio stream by dropping the extra channels: <pre class=”example-preformatted”>pan="stereo| c0=FL | c1=FR" </pre>

Given the same source, you can also switch front left and front right channels and keep the input channel layout: <pre class=”example-preformatted”>pan="5.1| c0=c1 | c1=c0 | c2=c2 | c3=c3 | c4=c4 | c5=c5" </pre>

If the input is a stereo audio stream, you can mute the front left channel (and still keep the stereo channel layout) with: <pre class=”example-preformatted”>pan="stereo|c1=c1" </pre>

Still with a stereo audio stream input, you can copy the right channel in both front left and right: <pre class=”example-preformatted”>pan="stereo| c0=FR | c1=FR" </pre>

 <h3 class=”section”>29.102 replaygain# TOC</h3>

ReplayGain scanner filter. This filter takes an audio stream as an input and outputs it unchanged. At end of filtering it displays <code class=”code”>track_gain</code> and <code class=”code”>track_peak</code>. The filter accepts the following exported read-only options: <dl class=”table”> <dt><samp class=”option”>track_gain</samp></dt> <dd> Exported track gain in dB at end of stream. </dd>

<dt><samp class="option">track_peak</samp></dt> <dd> Exported track peak at end of stream. </dd> </dl>

 <h3 class="section">29.103 resample# TOC</h3>

Convert the audio sample format, sample rate and channel layout. It is not meant to be used directly. <h3 class="section">29.104 rubberband# TOC</h3> Apply time-stretching and pitch-shifting with librubberband. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-librubberband</code>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">tempo</samp></dt> <dd> Set tempo scale factor. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set pitch scale factor. </dd> <dt><samp class="option">transients</samp></dt> <dd> Set transients detector. Possible values are: <dl class="table"> <dt><var class="var">crisp</var></dt> <dt><var class="var">mixed</var></dt> <dt><var class="var">smooth</var></dt> </dl>

</dd> <dt><samp class="option">detector</samp></dt> <dd> Set detector. Possible values are: <dl class="table"> <dt><var class="var">compound</var></dt> <dt><var class="var">percussive</var></dt> <dt><var class="var">soft</var></dt> </dl>

</dd> <dt><samp class="option">phase</samp></dt> <dd> Set phase. Possible values are: <dl class="table"> <dt><var class="var">laminar</var></dt> <dt><var class="var">independent</var></dt> </dl>

</dd> <dt><samp class="option">window</samp></dt> <dd> Set processing window size. Possible values are: <dl class="table"> <dt><var class="var">standard</var></dt> <dt><var class="var">short</var></dt> <dt><var class="var">long</var></dt> </dl>

</dd> <dt><samp class="option">smoothing</samp></dt> <dd> Set smoothing. Possible values are: <dl class="table"> <dt><var class="var">off</var></dt> <dt><var class="var">on</var></dt> </dl>

</dd> <dt><samp class="option">formant</samp></dt> <dd> Enable formant preservation when shift pitching. Possible values are: <dl class="table"> <dt><var class="var">shifted</var></dt> <dt><var class="var">preserved</var></dt> </dl>

</dd> <dt><samp class="option">pitchq</samp></dt> <dd> Set pitch quality. Possible values are: <dl class="table"> <dt><var

class=”var”>quality</var></dt> <dt><var class=”var”>speed</var></dt>
<dt><var class=”var”>consistency</var></dt> </dl>

</dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set channels. Possible values are: <dl class=”table”> <dt><var class=”var”>apart</var></dt> <dt><var class=”var”>together</var></dt>
</dl> </dd> </dl>

 <h4 class=”subsection”>29.104.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>tempo</samp></dt> <dd> Change filter tempo scale factor. Syntax for the command is : "<var class=”var”>tempo</var>" </dd> <dt><samp class=”option”>pitch</samp></dt>
<dd> Change filter pitch scale factor. Syntax for the command is : "<var class=”var”>pitch</var>" </dd> </dl>

 <h3 class=”section”>29.105 sidechaincompress# TOC</h3>

This filter acts like normal compressor but has the ability to compress detected signal using second input signal. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filtered signal then can be filtered with other filters in later stages of processing. See pan and amerge filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set mode of compressor operation. Can be <code class=”code”>upward</code> or <code class=”code”>downward</code>. Default is <code class=”code”>downward</code>.
 </dd> <dt><samp class=”option”>threshold</samp></dt>
<dd> If a signal of second stream raises above this level it will affect the gain reduction of first stream. By default is 0.125. Range is between 0.00097563 and 1. </dd> <dt><samp class=”option”>ratio</samp></dt>
<dd> Set a ratio about which the signal is reduced. 1:2 means that if the level raised 4dB above the threshold, it will be only 2dB above after the reduction. Default is 2. Range is between 1 and 20. </dd>
<dt><samp class=”option”>attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction starts. Default is 20. Range is between 0.01 and 2000. </dd>
<dt><samp class=”option”>release</samp></dt> <dd> Amount

of milliseconds the signal has to fall below the threshold before reduction is decreased again. Default is 250. Range is between 0.01 and 9000. </dd> <dt><samp class=”option”>makeup</samp></dt> <dd> Set the amount by how much signal will be amplified after processing. Default is 1. Range is from 1 to 64. </dd> <dt><samp class=”option”>knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.82843. Range is between 1 and 8. </dd> <dt><samp class=”option”>link</samp></dt> <dd> Choose if the <code class=”code”>average</code> level between all channels of side-chain stream or the louder(<code class=”code”>maximum</code>) channel of side-chain stream affects the reduction. Default is <code class=”code”>average</code>. </dd> <dt><samp class=”option”>detection</samp></dt> <dd> Should the exact signal be taken in case of <code class=”code”>peak</code> or an RMS one in case of <code class=”code”>rms</code>. Default is <code class=”code”>rms</code> which is mainly smoother. </dd> <dt><samp class=”option”>level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is between 0.015625 and 64. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> How much to use compressed signal in output. Default is 1. Range is between 0 and 1. </dd> </dl>

 <h4 class=”subsection”>29.105.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>29.105.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Full ffmpeg example taking 2 audio inputs, 1st input to be compressed depending on the signal of 2nd input and later compressed signal to be merged with 2nd input: <pre class=”example-preformatted”>ffmpeg -i main.flac -i sidechain.flac -filter_complex "[1:a]asplit=2[sc][mix];[0:a][sc]sidechaincompress[compr];[compr][mix]amerge" </pre>

 <h3 class=”section”>29.106 sidechaingate# TOC</h3>

A sidechain gate acts like a normal (wideband) gate but has the ability to filter the detected signal before sending it to the gain reduction stage. Normally a gate uses the full range signal to detect a level above the threshold.

For example: If you cut all lower frequencies from your sidechain signal the gate will decrease the volume of your track only if not enough highs appear. With this technique you are able to reduce the resonation of a natural drum or remove "rumbling" of muted strokes from a heavily distorted guitar. It needs two input streams and returns one output stream. First input stream will be processed depending on second stream signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_in</samp></dt> <dd> Set input level before filtering. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the mode of operation. Can be <code class="code">upward</code> or <code class="code">downward</code>. Default is <code class="code">downward</code>. If set to <code class="code">upward</code> mode, higher parts of signal will be amplified, expanding dynamic range in upward direction. Otherwise, in case of <code class="code">downward</code> lower parts of signal will be reduced. </dd> <dt><samp class="option">range</samp></dt> <dd> Set the level of gain reduction when the signal is below the threshold. Default is 0.06125. Allowed range is from 0 to 1. Setting this to 0 disables reduction and then filter behaves like expander. </dd> <dt><samp class="option">threshold</samp></dt> <dd> If a signal rises above this level the gain reduction is released. Default is 0.125. Allowed range is from 0 to 1. </dd> <dt><samp class="option">ratio</samp></dt> <dd> Set a ratio about which the signal is reduced. Default is 2. Allowed range is from 1 to 9000. </dd> <dt><samp class="option">attack</samp></dt> <dd> Amount of milliseconds the signal has to rise above the threshold before gain reduction stops. Default is 20 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">release</samp></dt> <dd> Amount of milliseconds the signal has to fall below the threshold before the reduction is increased again. Default is 250 milliseconds. Allowed range is from 0.01 to 9000. </dd> <dt><samp class="option">makeup</samp></dt> <dd> Set amount of amplification of signal after processing. Default is 1. Allowed range is from 1 to 64. </dd> <dt><samp class="option">knee</samp></dt> <dd> Curve the sharp knee around the threshold to enter gain reduction more softly. Default is 2.828427125. Allowed range is from 1 to 8. </dd> <dt><samp class="option">detection</samp></dt> <dd> Choose if exact signal should be taken for detection or an RMS like one. Default is rms. Can be peak or rms. </dd> <dt><samp class="option">link</samp></dt> <dd> Choose if the average level between all channels or the louder channel affects the reduction. Default is average. Can be average or maximum. </dd> <dt><samp class="option">level_sc</samp></dt> <dd> Set sidechain gain. Default is 1. Range is from 0.015625 to 64. </dd> </dl>

 <h4 class="subsection">29.106.1 Com-

mands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">29.107 silencedetect# TOC</h3>

Detect silence in an audio stream. This filter logs a message when it detects that the input audio volume is less or equal to a noise tolerance value for a duration greater or equal to the minimum detected noise duration. The printed times and duration are expressed in seconds. The <code class="code">lavfi.silence_start</code> or <code class="code">lavfi.silence_start.X</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the silence. The <code class="code">lavfi.silence_duration</code> or <code class="code">lavfi.silence_duration.X</code> and <code class="code">lavfi.silence_end</code> or <code class="code">lavfi.silence_end.X</code> metadata keys are set on the first frame after the silence. If <samp class="option">mono</samp> is enabled, and each channel is evaluated separately, the <code class="code">.X</code> suffixed keys are used, and <code class="code">X</code> corresponds to the channel number. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or amplitude ratio. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set silence duration until notification (default is 2 seconds). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. </dd> <dt><samp class="option">mono, m</samp></dt> <dd> Process each channel separately, instead of combined. By default is disabled. </dd> </dl>

 <h4 class="subsection">29.107.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect 5 seconds of silence with -50dB noise tolerance: <pre class="example-preformatted">silencedetect=n=-50dB:d=5 </pre>

Complete example with <code class="command">ffmpeg</code> to detect silence with 0.0001 noise tolerance in <samp class="file">silence.mp3</samp>:

 <pre class=”example-preformatted”>ffmpeg -i silence.mp3 -af silencedetect=noise=0.0001 -f null - </pre>

 <h3 class=”section”>29.108 silenceremove# TOC</h3>

Remove silence from the beginning, middle or end of the audio.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>start_periods</samp></dt>

<dd> This value is used to indicate if audio should be trimmed at beginning of

the audio. A value of zero indicates no silence should be trimmed from the

beginning. When specifying a non-zero value, it trims audio up until it

finds non-silence. Normally, when trimming silence from beginning of audio

the <var class=”var”>start_periods</var> will be <code class=”code”>1</code> but it can be increased to higher

values to trim all audio up to specific count of non-silence periods.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_duration</samp></dt>

<dd> Specify the amount of time that non-silence must be detected before it stops

trimming audio. By increasing the duration, bursts of noises can be treated

as silence and trimmed off. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_threshold</samp></dt>

<dd> This indicates what sample value should be treated as silence. For digital

audio, a value of <code class=”code”>0</code> may be fine but for audio recorded from analog,

you may wish to increase the value to account for background noise.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>start_silence</samp></dt>

<dd> Specify max duration of silence at beginning that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>start_mode</samp></dt>

<dd> Specify mode of detection of silence end at start of multi-channel audio.

Can be <var class=”var”>any</var> or <var class=”var”>all</var>. Default is <var class=”var”>any</var>.

With <var class=”var”>any</var>, any sample from any channel that is detected as non-silence

will trigger end of silence trimming at start of audio stream.

With <var class=”var”>all</var>, only if every sample from every channel is detected as non-silence

will trigger end of silence trimming at start of audio stream, limited usage.

</dd>

<dt><samp class=”option”>stop_periods</samp></dt>

<dd> Set the count for trimming silence from the end of audio. When specifying a

positive value, it trims audio after it finds specified silence period.

To remove silence from the middle of a file, specify a <var class=”var”>stop_periods</var>

that is negative. This value is then treated as a positive value and is

used to indicate the effect should restart processing as specified by

<var class=”var”>stop_periods</var>, making it suitable for removing periods of silence

in the middle of the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_duration</samp></dt>

<dd> Specify a duration of silence that must exist before audio is not copied any

more. By specifying a higher duration, silence that is wanted can be left in

the audio.

Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_threshold</samp></dt>

<dd> This is the same as <samp class=”option”>start_threshold</samp> but for trimming silence from

the end of audio.

Can be specified in dB (in case "dB" is appended to the specified value)

or amplitude ratio. Default value is <code class=”code”>0</code>.

</dd>

<dt><samp class=”option”>stop_silence</samp></dt>

<dd> Specify max duration of silence at end that will be kept after

trimming. Default is 0, which is equal to trimming all samples detected

as silence.

</dd>

<dt><samp class=”option”>stop_mode</samp></dt>

<dd> Specify mode of detection of silence start after start of multi-channel audio.

Can be <var class="var">any</var> or <var class="var">all</var>. Default is <var class="var">all</var>.

With <var class="var">any</var>, any sample from any channel that is detected as silence

will trigger start of silence trimming after start of audio stream, limited usage.

With <var class="var">all</var>, only if every sample from every channel is detected as silence

will trigger start of silence trimming after start of audio stream.

</dd>

<dt><samp class="option">detection</samp></dt>

<dd> Set how is silence detected.

<dl class="table">

<dt><samp class="option">avg</samp></dt>

<dd> Mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">rms</samp></dt>

<dd> Root squared mean of absolute values of samples in moving window.

</dd>

<dt><samp class="option">peak</samp></dt>

<dd> Maximum of absolute values of samples in moving window.

</dd>

<dt><samp class="option">median</samp></dt>

<dd> Median of absolute values of samples in moving window.

</dd>

<dt><samp class="option">ptp</samp></dt>

<dd> Absolute of max peak to min peak difference of samples in moving window.

</dd>

<dt><samp class="option">dev</samp></dt>

<dd> Standard deviation of values of samples in moving window.

</dd>

</dl>

Default value is <code class="code">rms</code>.

</dd>

<dt><samp class="option">window</samp></dt>

<dd> Set duration in number of seconds used to calculate size of window in number

of samples for detecting silence. Using <code class="code">0</code> will effectively disable

any windowing and use only single sample per channel for silence detection.

In that case it may be needed to also set <samp class="option">start_silence</samp> and/or

<samp class="option">stop_silence</samp> to nonzero values with also <samp class="option">start_duration</samp> and/or

<samp class="option">stop_duration</samp> to nonzero values.

Default value is <code class="code">0.02</code>. Allowed range is from <code class="code">0</code> to <code class="code">10</code>.

</dd>

<dt><samp class="option">timestamp</samp></dt>

<dd> Set processing mode of every audio frame output timestamp.

<dl class="table">

<dt><samp class="option">write</samp></dt>

<dd> Full timestamps rewrite, keep only the start time for the first output frame.

</dd>

<dt><samp class="option">copy</samp></dt>

<dd> Non-dropped frames are left with same timestamp as input audio frame.

</dd>

</dl>

Defaults value is <code class=”code”>write</code>.

</dd>

</dl>

 <h4 class=”subsection”>29.108.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> The following example shows how this filter can be used to start a recording that does not contain the delay at the start which usually occurs between pressing the record button and the start of the performance: <pre class=”example-preformatted”>silenceremove=start_periods=1:start_duration=5:start_threshold=0.02 </pre>

Trim all silence encountered from beginning to end where there is more than 1 second of silence in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-1:stop_duration=1:stop_threshold=-90dB </pre>

Trim all digital silence samples, using peak detection, from beginning to end where there is more than 0 samples of digital silence in audio and digital silence is detected in all channels at same positions in stream: <pre class=”example-preformatted”>silenceremove=window=0:detection=peak:stop_mode=all:start_mode=all:stop_periods=-1:stop_threshold=0 </pre>

Trim every 2nd encountered silence period from beginning to end where there is more than 1 second of silence per silence period in audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB </pre>

Similar as above, but keep maximum of 0.5 seconds of silence from each trimmed period: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5 </pre>

Similar as above, but keep maximum of 1.5 seconds of silence from start of audio: <pre class=”example-preformatted”>silenceremove=stop_periods=-2:stop_duration=1:stop_threshold=-90dB:stop_silence=0.5:start_periods=1:start_duration=1:start_silence=1.5:stop_threshold=-90dB </pre>

 <h4 class=”subsection”>29.108.2 Commands# TOC</h4>

This filter supports some above options as commands. <h3 class=”section”>29.109 sofalizer# TOC</h3>

SOFAlizer uses head-related transfer functions (HRTFs) to create virtual loudspeakers around the user for binaural listening via headphones (audio formats up to 9 channels supported). The HRTFs are stored in SOFA files (see http://www.sofacoustics.org/ for a database). SOFAlizer is developed at the Acoustics Research Institute (ARI) of the Austrian Academy of Sciences. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libmysofa</code>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sofa</samp></dt> <dd> Set the SOFA file used for rendering. </dd> <dt><samp class=”option”>gain</samp></dt> <dd> Set gain applied to audio. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>rotation</samp></dt> <dd> Set rotation of virtual loudspeakers in deg. Default is 0. </dd> <dt><samp class=”option”>elevation</samp></dt> <dd> Set elevation of virtual speakers in deg. Default is 0. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> Set distance in meters between loudspeakers and the listener with near-field HRTFs. Default is 1. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set processing type. Can be <var class=”var”>time</var> or <var class=”var”>freq</var>. <var class=”var”>time</var> is processing audio in time domain which is slow. <var class=”var”>freq</var> is processing audio in frequency domain which is fast. Default is <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>speakers</samp></dt> <dd> Set custom positions of virtual loudspeakers. Syntax for this option is: <CH> <AZIM> <ELEV>[|<CH> <AZIM> <ELEV>|...]. Each virtual loudspeaker is described with short channel name following with azimuth and elevation in degrees. Each virtual loudspeaker description is separated by ’|’. For example to override front left and front right channel positions use: ’speakers=FL 45 15|FR 345 15’. Descriptions with unrecognised channel names are ignored. </dd> <dt><samp class=”option”>lfegain</samp></dt> <dd> Set custom gain for LFE channels. Value is in dB. Default is 0. </dd> <dt><samp class=”option”>framesize</samp></dt> <dd> Set custom frame size in number of samples. Default is 1024. Allowed range is from 1024 to 96000. Only used if option ‘<samp class=”samp”>type</samp>’ is set to <var class=”var”>freq</var>. </dd> <dt><samp class=”option”>normalize</samp></dt> <dd> Should all IRs be normalized upon importing SOFA file. By default is enabled. </dd> <dt><samp class=”option”>interpolate</samp></dt> <dd> Should

nearest IRs be interpolated with neighbor IRs if exact position does not match. By default is disabled. </dd> <dt><samp class="option">minphase</samp></dt> <dd> Minphase all IRs upon loading of SOFA file. By default is disabled. </dd> <dt><samp class="option">anglestep</samp></dt> <dd> Set neighbor search angle step. Only used if option <var class="var">interpolate</var> is enabled. </dd> <dt><samp class="option">radstep</samp></dt> <dd> Set neighbor search radius step. Only used if option <var class="var">interpolate</var> is enabled. </dd> </dl>

 <h4 class="subsection">29.109.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Using ClubFritz6 sofa file: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz6.sofa:type=freq:r
</pre>

Using ClubFritz12 sofa file and bigger radius with small rotation: <pre class="example-preformatted">sofalizer=sofa=/path/to/ClubFritz12.sofa:type=
</pre>

Similar as above but with custom speaker positions for front left, front right, back left and back right and also with custom gain: <pre class="example-preformatted">"sofalizer=sofa=/path/to/ClubFritz6.sofa:type
45|FR 315|BL 135|BR 225:gain=28" </pre>

 <h3 class="section">29.110 speechnorm# TOC</h3> Speech Normalizer. This filter expands or compresses each half-cycle of audio samples (local set of samples all above or all below zero and between two nearest zero crossings) depending on threshold value, so audio reaches target peak value under conditions controlled by below options. The filter accepts the following options: <dl class="table"> <dt><samp class="option">peak, p</samp></dt> <dd> Set the expansion target peak value. This specifies the highest allowed absolute amplitude level for the normalized audio input. Default value is 0.95. Allowed range is from 0.0 to 1.0. </dd> <dt><samp class="option">expansion, e</samp></dt> <dd> Set the maximum expansion factor. Allowed range is from 1.0 to 50.0. Default value is 2.0. This option controls maximum local half-cycle of samples expansion. The maximum expansion would be such that local peak value reaches target peak value but never to surpass it and that ratio between new and previous peak value does not surpass this option value. </dd> <dt><samp class="option">compression, c</samp></dt> <dd> Set the maximum compression factor. Allowed range is from 1.0 to 50.0. Default

value is 2.0. This option controls maximum local half-cycle of samples compression. This option is used only if <samp class=”option”>threshold</samp> option is set to value greater than 0.0, then in such cases when local peak is lower or same as value set by <samp class=”option”>threshold</samp> all samples belonging to that peak’s half-cycle will be compressed by current compression factor. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set the threshold value. Default value is 0.0. Allowed range is from 0.0 to 1.0. This option specifies which half-cycles of samples will be compressed and which will be expanded. Any half-cycle samples with their local peak value below or same as this option value will be compressed by current compression factor, otherwise, if greater than threshold value they will be expanded with expansion factor so that it could reach peak target value but never surpass it. </dd> <dt><samp class=”option”>raise, r</samp></dt> <dd> Set the expansion raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast expansion factor is raised per each new half-cycle until it reaches <samp class=”option”>expansion</samp> value. Setting this options too high may lead to distortions. </dd> <dt><samp class=”option”>fall, f</samp></dt> <dd> Set the compression raising amount per each half-cycle of samples. Default value is 0.001. Allowed range is from 0.0 to 1.0. This controls how fast compression factor is raised per each new half-cycle until it reaches <samp class=”option”>compression</samp> value. </dd> <dt><samp class=”option”>channels, h</samp></dt> <dd> Specify which channels to filter, by default all available channels are filtered. </dd> <dt><samp class=”option”>invert, i</samp></dt> <dd> Enable inverted filtering, by default is disabled. This inverts interpretation of <samp class=”option”>threshold</samp> option. When enabled any half-cycle of samples with their local peak value below or same as <samp class=”option”>threshold</samp> option will be expanded otherwise it will be compressed. </dd> <dt><samp class=”option”>link, l</samp></dt> <dd> Link channels when calculating gain applied to each filtered channel sample, by default is disabled. When disabled each filtered channel gain calculation is independent, otherwise when this option is enabled the minimum of all possible gains for each filtered channel is used. </dd> <dt><samp class=”option”>rms, m</samp></dt> <dd> Set the expansion target RMS value. This specifies the highest allowed RMS level for the normalized audio input. Default value is 0.0, thus disabled. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class=”subsection”>29.110.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>29.110.2 Examples<a class=”anchor hidden-xs” href=”#Examples-

52” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Weak and slow amplification: <pre class=”example-preformatted”>speechnorm=e=3:r=0.00001:l=1 </pre>

Moderate and slow amplification: <pre class=”example-preformatted”>speechnorm=e=6.25:r=0.00001:l=1 </pre>

Strong and fast amplification: <pre class=”example-preformatted”>speechnorm=e=12.5:r=0.0001:l=1 </pre>

Very strong and fast amplification: <pre class=”example-preformatted”>speechnorm=e=25:r=0.0001:l=1 </pre>

Extreme and fast amplification: <pre class=”example-preformatted”>speechnorm=e=50:r=0.0001:l=1 </pre>

 <h3 class=”section”>29.111 stereotools# TOC</h3>

This filter has some handy utilities to manage stereo signals, for converting M/S stereo recordings to L/R signal while having control over the parameters or spreading the stereo image of master track. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>level_in</samp></dt> <dd> Set input level before filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>level_out</samp></dt> <dd> Set output level after filtering for both channels. Defaults is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class=”option”>balance_in</samp></dt> <dd> Set input balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>balance_out</samp></dt> <dd> Set output balance between both channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class=”option”>softclip</samp></dt> <dd> Enable softclipping. Results in analog distortion instead of harsh digital 0dB clipping. Disabled by default. </dd> <dt><samp class=”option”>mutel</samp></dt> <dd> Mute the left channel. Disabled by default. </dd> <dt><samp class=”option”>muter</samp></dt> <dd> Mute the right channel. Disabled by default. </dd> <dt><samp class=”option”>phasel</samp></dt> <dd> Change the phase of the left channel. Disabled by default. </dd> <dt><samp class=”option”>phaser</samp></dt> <dd> Change

the phase of the right channel. Disabled by default. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set stereo mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">lr>lr</samp>’</dt> <dd> Left/Right to Left/Right, this is default. </dd> <dt>‘<samp class="samp">lr>ms</samp>’</dt> <dd> Left/Right to Mid/Side. </dd> <dt>‘<samp class="samp">ms>lr</samp>’</dt> <dd> Mid/Side to Left/Right. </dd> <dt>‘<samp class="samp">lr>ll</samp>’</dt> <dd> Left/Right to Left/Left. </dd> <dt>‘<samp class="samp">lr>rr</samp>’</dt> <dd> Left/Right to Right/Right. </dd> <dt>‘<samp class="samp">lr>l+r</samp>’</dt> <dd> Left/Right to Left + Right. </dd> <dt>‘<samp class="samp">lr>rl</samp>’</dt> <dd> Left/Right to Right/Left. </dd> <dt>‘<samp class="samp">ms>ll</samp>’</dt> <dd> Mid/Side to Left/Left. </dd> <dt>‘<samp class="samp">ms>rr</samp>’</dt> <dd> Mid/Side to Right/Right. </dd> <dt>‘<samp class="samp">ms>rl</samp>’</dt> <dd> Mid/Side to Right/Left. </dd> <dt>‘<samp class="samp">lr>l-r</samp>’</dt> <dd> Left/Right to Left - Right. </dd> </dl>

</dd> <dt><samp class="option">slev</samp></dt> <dd> Set level of side signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">sbal</samp></dt> <dd> Set balance of side signal. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">mlev</samp></dt> <dd> Set level of the middle signal. Default is 1. Allowed range is from 0.015625 to 64. </dd> <dt><samp class="option">mpan</samp></dt> <dd> Set middle signal pan. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">base</samp></dt> <dd> Set stereo base between mono and inversed channels. Default is 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">delay</samp></dt> <dd> Set delay in milliseconds how much to delay left from right channel and vice versa. Default is 0. Allowed range is from -20 to 20. </dd> <dt><samp class="option">sclevel</samp></dt> <dd> Set S/C level. Default is 1. Allowed range is from 1 to 100. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set the stereo phase in degrees. Default is 0. Allowed range is from 0 to 360. </dd> <dt><samp class="option">bmode_in, bmode_out</samp></dt> <dd> Set balance mode for balance_in/balance_out option. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">balance</samp>’</dt> <dd> Classic balance mode. Attenuate one channel at time. Gain is raised up to 1. </dd> <dt>‘<samp class="samp">amplitude</samp>’</dt> <dd> Similar as classic mode above but gain is raised up to 2. </dd> <dt>‘<samp class="samp">power</samp>’</dt>

<dd> Equal power distribution, from -6dB to +6dB range. </dd>
</dl> </dd> </dl>

 <h4 class="subsection">29.111.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h4 class="subsection">29.111.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply karaoke like effect: <pre class="example-preformatted">stereotools=mlev=0.015625
</pre>

Convert M/S signal to L/R: <pre class="example-preformatted">"stereotools=mode=ms>lr"
</pre>

 <h3 class="section">29.112 stereowiden# TOC</h3>

This filter enhance the stereo effect by suppressing signal common to both channels and by delaying the signal of left into right and vice versa, thereby widening the stereo effect. The filter accepts the following options:
 <dl class="table"> <dt><samp class="option">delay</samp></dt>
<dd> Time in milliseconds of the delay of left signal into right and vice versa. Default is 20 milliseconds. </dd> <dt><samp class="option">feedback</samp></dt> <dd> Amount of gain in delayed signal into right and vice versa. Gives a delay effect of left signal in right output and vice versa which gives widening effect. Default is 0.3. </dd> <dt><samp class="option">crossfeed</samp></dt>
<dd> Cross feed of left into right with inverted phase. This helps in suppressing the mono. If the value is 1 it will cancel all the signal common to both channels. Default is 0.3. </dd> <dt><samp class="option">drymix</samp></dt> <dd> Set level of input signal of original channel. Default is 0.8. </dd> </dl>

 <h4 class="subsection">29.112.1 Commands# TOC</h4>

This filter supports the all above options except <code class="code">delay</code>
as commands. <a

name=”superequalizer”> <h3 class=”section”>29.113 superequalizer# TOC</h3> Apply 18 band equalizer. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>1b</samp></dt> <dd> Set 65Hz band gain. </dd> <dt><samp class=”option”>2b</samp></dt> <dd> Set 92Hz band gain. </dd> <dt><samp class=”option”>3b</samp></dt> <dd> Set 131Hz band gain. </dd> <dt><samp class=”option”>4b</samp></dt> <dd> Set 185Hz band gain. </dd> <dt><samp class=”option”>5b</samp></dt> <dd> Set 262Hz band gain. </dd> <dt><samp class=”option”>6b</samp></dt> <dd> Set 370Hz band gain. </dd> <dt><samp class=”option”>7b</samp></dt> <dd> Set 523Hz band gain. </dd> <dt><samp class=”option”>8b</samp></dt> <dd> Set 740Hz band gain. </dd> <dt><samp class=”option”>9b</samp></dt> <dd> Set 1047Hz band gain. </dd> <dt><samp class=”option”>10b</samp></dt> <dd> Set 1480Hz band gain. </dd> <dt><samp class=”option”>11b</samp></dt> <dd> Set 2093Hz band gain. </dd> <dt><samp class=”option”>12b</samp></dt> <dd> Set 2960Hz band gain. </dd> <dt><samp class=”option”>13b</samp></dt> <dd> Set 4186Hz band gain. </dd> <dt><samp class=”option”>14b</samp></dt> <dd> Set 5920Hz band gain. </dd> <dt><samp class=”option”>15b</samp></dt> <dd> Set 8372Hz band gain. </dd> <dt><samp class=”option”>16b</samp></dt> <dd> Set 11840Hz band gain. </dd> <dt><samp class=”option”>17b</samp></dt> <dd> Set 16744Hz band gain. </dd> <dt><samp class=”option”>18b</samp></dt> <dd> Set 20000Hz band gain. </dd> </dl>

<h3 class=”section”>29.114 surround# TOC</h3>

Apply audio surround upmix filter.

This filter allows to produce multichannel output from audio stream.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>chl_out</samp></dt>

<dd> Set output channel layout. By default, this is <var class=”var”>5.1</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">chl_in</samp></dt>

<dd> Set input channel layout. By default, this is <var class="var">stereo</var>.

See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#channel-layout-syntax">the Channel Layout section in the ffmpeg-utils(1) manual

for the required syntax.

</dd>

<dt><samp class="option">level_in</samp></dt>

<dd> Set input volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">level_out</samp></dt>

<dd> Set output volume level. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe</samp></dt>

<dd> Enable LFE channel output if output channel layout has it. By default, this is enabled.

</dd>

<dt><samp class="option">lfe_low</samp></dt>

<dd> Set LFE low cut off frequency. By default, this is <var class="var">128</var> Hz.

</dd>

<dt><samp class="option">lfe_high</samp></dt>

<dd> Set LFE high cut off frequency. By default, this is <var class="var">256</var> Hz.

</dd>

<dt><samp class="option">lfe_mode</samp></dt>

<dd> Set LFE mode, can be <var class="var">add</var> or <var class="var">sub</var>. Default is <var class="var">add</var>.

In <var class="var">add</var> mode, LFE channel is created from input audio and added to output.

In <var class="var">sub</var> mode, LFE channel is created from input audio and added to output but

also all non-LFE output channels are subtracted with output LFE channel.

</dd>

<dt><samp class="option">smooth</samp></dt>

<dd> Set temporal smoothness strength, used to gradually change factors when transforming

stereo sound in time. Allowed range is from <var class="var">0.0</var> to <var class="var">1.0</var>.

Useful to improve output quality with <var class="var">focus</var> option values greater than <var class="var">0.0</var>.

Default is <var class="var">0.0</var>. Only values inside this range and without edges are effective.

</dd>

<dt><samp class="option">angle</samp></dt>

<dd> Set angle of stereo surround transform, Allowed range is from <var class="var">0</var> to <var class="var">360</var>.

Default is <var class="var">90</var>.

</dd>

<dt><samp class="option">focus</samp></dt>

<dd> Set focus of stereo surround transform, Allowed range is from <var class="var">-1</var> to <var class="var">1</var>.

Default is <var class="var">0</var>.

</dd>

<dt><samp class="option">fc_in</samp></dt>

<dd> Set front center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fc_out</samp></dt>

<dd> Set front center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_in</samp></dt>

<dd> Set front left input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fl_out</samp></dt>

<dd> Set front left output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_in</samp></dt>

<dd> Set front right input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">fr_out</samp></dt>

<dd> Set front right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">sl_in</samp></dt>

<dd> Set side left input volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">sl_out</samp></dt>
<dd> Set side left output volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">sr_in</samp></dt>
<dd> Set side right input volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">sr_out</samp></dt>
<dd> Set side right output volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">bl_in</samp></dt>
<dd> Set back left input volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">bl_out</samp></dt>
<dd> Set back left output volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">br_in</samp></dt>
<dd> Set back right input volume. By default, this is <var class="var">1</var>.

</dd>
<dt><samp class="option">br_out</samp></dt>
<dd> Set back right output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_in</samp></dt>

<dd> Set back center input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">bc_out</samp></dt>

<dd> Set back center output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_in</samp></dt>

<dd> Set LFE input volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">lfe_out</samp></dt>

<dd> Set LFE output volume. By default, this is <var class="var">1</var>.

</dd>

<dt><samp class="option">allx</samp></dt>

<dd> Set spread usage of stereo image across X axis for all channels.

Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.

By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>

<dt><samp class="option">ally</samp></dt>

<dd> Set spread usage of stereo image across Y axis for all channels.

Allowed range is from <var class="var">-1</var> to <var class="var">15</var>.

By default this value is negative <var class="var">-1</var>, and thus unused.

</dd>

<dt><samp class="option">fcx, flx, frx, blx, brx, slx, srx, bcx</samp></dt>

<dd> Set spread usage of stereo image across X axis for each channel.

Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.

By default this value is <var class="var">0.5</var>.

</dd>

<dt><samp class="option">fcy, fly, fry, bly, bry, sly, sry, bcy</samp></dt>

<dd> Set spread usage of stereo image across Y axis for each channel.

Allowed range is from <var class="var">0.06</var> to <var class="var">15</var>.

By default this value is <var class="var">0.5</var>.

</dd>

<dt><samp class="option">win_size</samp></dt>

<dd> Set window size. Allowed range is from <var class="var">1024</var>
to <var class="var">65536</var>. Default size is <var class="var">4096</var>.

</dd>

<dt><samp class="option">win_func</samp></dt>

<dd> Set window function.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">rect</samp>’</dt>

<dt>‘<samp class="samp">bartlett</samp>’</dt>

<dt>‘<samp class="samp">hann, hanning</samp>’</dt>

<dt>‘<samp class="samp">hamming</samp>’</dt>

<dt>‘<samp class="samp">blackman</samp>’</dt>

<dt>‘<samp class="samp">welch</samp>’</dt>

<dt>‘<samp class="samp">flattop</samp>’</dt>

<dt>‘<samp class="samp">bharris</samp>’</dt>

<dt>‘<samp class="samp">bnuttall</samp>’</dt>

<dt>‘<samp class="samp">bhann</samp>’</dt>

<dt>‘<samp class="samp">sine</samp>’</dt>

<dt>‘<samp class="samp">nuttall</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hann</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. If set to 1, the recommended overlap for selected window function will be picked. Default is <code class="code">0.5</code>.

</dd>

</dl>

 <h3 class="section">29.115 tiltshelf# TOC</h3>

Boost or cut the lower frequencies and cut or boost higher frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at 0 Hz. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">3000</code> Hz. </dd>

<dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt> <dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.115.1 Commands# TOC</h4>

This filter supports some options as commands. <h3 class="section">29.116

treble, highshelf# TOC</h3>

Boost or cut treble (upper) frequencies of the audio using a two-pole shelving filter with a response similar to that of a standard hi-fi’s tone-controls. This is also known as shelving equalisation (EQ). The filter accepts the following options: <dl class="table"> <dt><samp class="option">gain, g</samp></dt> <dd> Give the gain at whichever is the lower of ~22 kHz and the Nyquist frequency. Its useful range is about -20 (for a large cut) to +20 (for a large boost). Beware of clipping when using a positive gain. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the filter’s central frequency and so can be used to extend or reduce the frequency range to be boosted or cut. The default value is <code class="code">3000</code> Hz. </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Set method to specify band-width of filter. <dl class="table"> <dt><samp class="option">h</samp></dt> <dd> Hz </dd> <dt><samp class="option">q</samp></dt> <dd> Q-Factor </dd> <dt><samp class="option">o</samp></dt> <dd> octave </dd> <dt><samp class="option">s</samp></dt> <dd> slope </dd> <dt><samp class="option">k</samp></dt> <dd> kHz </dd> </dl>

</dd> <dt><samp class="option">width, w</samp></dt> <dd> Determine how steep is the filter’s shelf transition. </dd> <dt><samp class="option">poles, p</samp></dt> <dd> Set number of poles. Default is 2. </dd> <dt><samp class="option">mix, m</samp></dt> <dd> How much to use filtered signal in output. Default is 1. Range is between 0 and 1. </dd> <dt><samp class="option">channels, c</samp></dt> <dd> Specify which channels to filter, by default all available are filtered. </dd> <dt><samp class="option">normalize, n</samp></dt> <dd> Normalize biquad coefficients, by default is disabled. Enabling it will normalize magnitude response at DC to 0dB. </dd> <dt><samp class="option">transform, a</samp></dt> <dd> Set transform type of IIR filter. <dl class="table"> <dt><samp class="option">di</samp></dt> <dt><samp class="option">dii</samp></dt> <dt><samp class="option">tdi</samp></dt> <dt><samp class="option">tdii</samp></dt> <dt><samp class="option">latt</samp></dt> <dt><samp class="option">svf</samp></dt> <dt><samp class="option">zdf</samp></dt> </dl>

</dd> <dt><samp class="option">precision, r</samp></dt> <dd> Set precision of filtering. <dl class="table"> <dt><samp class="option">auto</samp></dt> <dd> Pick automatic sample format depending on surround filters. </dd> <dt><samp class="option">s16</samp></dt> <dd> Always use signed 16-bit. </dd> <dt><samp class="option">s32</samp></dt> <dd> Always use signed 32-bit. </dd> <dt><samp class="option">f32</samp></dt> <dd> Always use float 32-bit. </dd> <dt><samp class="option">f64</samp></dt>

<dd> Always use float 64-bit. </dd> </dl>

</dd> <dt><samp class="option">block_size, b</samp></dt> <dd> Set block size used for reverse IIR processing. If this value is set to high enough value (higher than impulse response length truncated when reaches near zero values) filtering will become linear phase otherwise if not big enough it will just produce nasty artifacts. Note that filter delay will be exactly this many samples when set to non-zero value. </dd> </dl>

 <h4 class="subsection">29.116.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Change treble frequency. Syntax for the command is : "<var class="var">frequency</var>" </dd> <dt><samp class="option">width_type, t</samp></dt> <dd> Change treble width_type. Syntax for the command is : "<var class="var">width_type</var>" </dd> <dt><samp class="option">width, w</samp></dt> <dd> Change treble width. Syntax for the command is : "<var class="var">width</var>" </dd> <dt><samp class="option">gain, g</samp></dt> <dd> Change treble gain. Syntax for the command is : "<var class="var">gain</var>" </dd> <dt><samp class="option">mix, m</samp></dt> <dd> Change treble mix. Syntax for the command is : "<var class="var">mix</var>" </dd> </dl>

 <h3 class="section">29.117 tremolo# TOC</h3>

Sinusoidal amplitude modulation. The filter accepts the following options: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Modulation frequencies in the subharmonic range (20 Hz or lower) will result in a tremolo effect. This filter may also be used as a ring modulator by specifying a modulation frequency higher than 20 Hz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">29.118 vibrato# TOC</h3>

Sinusoidal phase modulation. The filter accepts the following op-

tions: <dl class="table"> <dt><samp class="option">f</samp></dt> <dd> Modulation frequency in Hertz. Range is 0.1 - 20000.0. Default value is 5.0 Hz. </dd> <dt><samp class="option">d</samp></dt> <dd> Depth of modulation as a percentage. Range is 0.0 - 1.0. Default value is 0.5. </dd> </dl>

 <h3 class="section">29.119 virtualbass# TOC</h3>

Apply audio Virtual Bass filter. This filter accepts stereo input and produce stereo with LFE (2.1) channels output. The newly produced LFE channel have enhanced virtual bass originally obtained from both stereo channels. This filter outputs front left and front right channels unchanged as available in stereo input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">cutoff</samp></dt> <dd> Set the virtual bass cutoff frequency. Default value is 250 Hz. Allowed range is from 100 to 500 Hz. </dd> <dt><samp class="option">strength</samp></dt> <dd> Set the virtual bass strength. Allowed range is from 0.5 to 3. Default value is 3. </dd> </dl>

 <h3 class="section">29.120 volume# TOC</h3>

Adjust the input audio volume. It accepts the following parameters: <dl class="table"> <dt><samp class="option">volume</samp></dt> <dd> Set audio volume expression. Output values are clipped to the maximum value. The output audio volume is given by the relation: <pre class="example-preformatted"><var class="var">output_volume</var> = <var class="var">volume</var> * <var class="var">input_volume</var> </pre>

The default value for <var class="var">volume</var> is "1.0". </dd> <dt><samp class="option">precision</samp></dt> <dd> This parameter represents the mathematical precision. It determines which input sample formats will be allowed, which affects the precision of the volume scaling. <dl class="table"> <dt><samp class="option">fixed</samp></dt> <dd> 8-bit fixed-point; this limits input sample format to U8, S16, and S32. </dd> <dt><samp class="option">float</samp></dt> <dd> 32-bit floating-point; this limits input sample format to FLT. (default) </dd> <dt><samp class="option">double</samp></dt> <dd> 64-bit floating-point; this limits input sample format to DBL. </dd> </dl>

</dd> <dt><samp class="option">replaygain</samp></dt> <dd> Choose

the behaviour on encountering ReplayGain side data in input frames.
<dl class="table"> <dt><samp class="option">drop</samp></dt>
<dd> Remove ReplayGain side data, ignoring its contents (the default). </dd> <dt><samp class="option">ignore</samp></dt>
<dd> Ignore ReplayGain side data, but leave it in the frame. </dd>
<dt><samp class="option">track</samp></dt> <dd> Prefer the track gain, if present. </dd> <dt><samp class="option">album</samp></dt>
<dd> Prefer the album gain, if present. </dd> </dl>

</dd> <dt><samp class="option">replaygain_preamp</samp></dt>
<dd> Pre-amplification gain in dB to apply to the selected replaygain gain.
 Default value for <var class="var">replaygain_preamp</var> is 0.0. </dd> <dt><samp class="option">replaygain_noclip</samp></dt>
<dd> Prevent clipping by limiting the gain applied. Default value for <var class="var">replaygain_noclip</var> is 1. </dd>
<dt><samp class="option">eval</samp></dt> <dd> Set when the volume expression is evaluated. It accepts the following values:
<dl class="table"> <dt>‘<samp class="samp">once</samp>’</dt>
<dd> only evaluate expression once during the filter initialization, or when the ‘<samp class="samp">volume</samp>’ command is sent
 </dd> <dt>‘<samp class="samp">frame</samp>’</dt>
<dd> evaluate expression for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">once</samp>’.
</dd> </dl>

The volume expression can contain the following parameters. <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> frame number (starting at zero) </dd> <dt><samp class="option">nb_channels</samp></dt>
<dd> number of channels </dd> <dt><samp class="option">nb_consumed_samples</samp></
<dd> number of samples consumed by the filter </dd> <dt><samp class="option">nb_samples</samp></dt> <dd> number of samples in the current frame </dd> <dt><samp class="option">pos</samp></dt>
<dd> original frame position in the file; deprecated, do not use </dd>
<dt><samp class="option">pts</samp></dt> <dd> frame PTS
</dd> <dt><samp class="option">sample_rate</samp></dt>
<dd> sample rate </dd> <dt><samp class="option">startpts</samp></dt>
<dd> PTS at start of stream </dd> <dt><samp class="option">startt</samp></dt>
<dd> time at start of stream </dd> <dt><samp class="option">t</samp></dt>
<dd> frame time </dd> <dt><samp class="option">tb</samp></dt>
<dd> timestamp timebase </dd> <dt><samp class="option">volume</samp></dt>
<dd> last set volume value </dd> </dl>

Note that when <samp class="option">eval</samp> is set to ‘<samp class="samp">once</samp>’ only the <var class="var">sample_rate</var> and <var class="var">tb</var> variables are available, all other variables will evaluate to NAN.
<h4 class="subsection">29.120.1 Commands<a

class=”anchor hidden-xs” href=”#Commands-53” aria-hidden=”true”>#
TOC</h4>

This filter supports the following commands: <dl class=”table”>
<dt><samp class=”option”>volume</samp></dt> <dd> Modify the volume expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class=”subsection”>29.120.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Halve the input audio volume: <pre class=”example-preformatted”>volume=volume=0.5
volume=volume=1/2 volume=volume=-6.0206dB </pre>

In all the above example the named key for <samp class=”option”>volume</samp> can be omitted, for example like in: <pre class=”example-preformatted”>volume=0.5 </pre>

Increase input audio power by 6 decibels using fixed-point precision:
 <pre class=”example-preformatted”>volume=volume=6dB:precision=fixed
</pre>

Fade volume after time 10 with an annihilation period of 5 seconds:
 <pre class=”example-preformatted”>volume='if(lt(t,10),1,max(1-(t-10)/5,0))':eval=frame </pre>

 <h3 class=”section”>29.121 volumedetect# TOC</h3>

Detect the volume of the input video. The filter has no parameters. It supports only 16-bit signed integer samples, so the input will be converted when needed. Statistics about the volume will be printed in the log when the input stream end is reached. In particular it will show the mean volume (root mean square), maximum volume (on a per-sample basis), and the beginning of a histogram of the registered volume values (from the maximum value to a cumulated 1/1000 of the samples).
All volumes are in decibels relative to the maximum PCM value.
 <h4 class=”subsection”>29.121.1 Examples# TOC</h4>

Here is an excerpt of the output: <pre class=”example-preformatted”>[Parsed_volumedetect_0 0xa23120]

mean_volume: -27 dB [Parsed_volumedetect_0 0xa23120] max_volume: -4 dB [Parsed_volumedetect_0 0xa23120] histogram_4db: 6 [Parsed_volumedetect_0 0xa23120] histogram_5db: 62 [Parsed_volumedetect_0 0xa23120] histogram_6db: 286 [Parsed_volumedetect_0 0xa23120] histogram_7db: 1042 [Parsed_volumedetect_0 0xa23120] histogram_8db: 2551 [Parsed_volumedetect_0 0xa23120] histogram_9db: 4609 [Parsed_volumedetect_0 0xa23120] histogram_10db: 8409 </pre>

It means that: <ul class="itemize mark-bullet"> The mean square energy is approximately -27 dB, or 10^-2.7. The largest sample is at -4 dB, or more precisely between -4 dB and -5 dB. There are 6 samples at -4 dB, 62 at -5 dB, 286 at -6 dB, etc.

In other words, raising the volume by +4 dB does not cause any clipping, raising it by +5 dB causes clipping for 6 samples, etc. <h3 class="section">29.122 whisper# TOC</h3>

It runs automatic speech recognition using the OpenAI’s Whisper model. It requires the whisper.cpp library (https://github.com/ggml-org/whisper.cpp) as a prerequisite. After installing the library it can be enabled using: <code class="code">./configure --enable-whisper</code>. The filter has following options: <dl class="table"> <dt><samp class="option">model</samp></dt> <dd> The file path of the downloaded whisper.cpp model (mandatory). </dd> <dt><samp class="option">language</samp></dt> <dd> The language to use for transcription. Besides a language code, the following special values enable auto-detection: <dl class="table"> <dt>‘<samp class="samp">eval</samp>’</dt> <dd> Re-run language detection on every queued chunk. This costs an extra encoder pass per chunk and the detected language may change mid-file. </dd> <dt>‘<samp class="samp">lock</samp>’</dt> <dd> Detect the language on the first chunk that produces transcribed text, then reuse it for the remaining chunks. </dd> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Currently an alias for ‘<samp class="samp">eval</samp>’. This may change in a future version, so use ‘<samp class="samp">eval</samp>’ or ‘<samp class="samp">lock</samp>’ if the behavior matters. </dd> </dl>

Default value: <code class="code">"auto"</code> </dd> <dt><samp class="option">translate</samp></dt> <dd> If enabled, translate the transcription from the source language to English. A multilingual model is required to enable this option. Default value: <code class="code">"false"</code> </dd> <dt><samp

class=”option”>queue</samp></dt> <dd> The maximum size that will be queued into the filter before processing the audio with whisper. Using a small value the audio stream will be processed more often, but the transcription quality will be lower and the required processing power will be higher. Using a large value (e.g. 10-20s) will produce more accurate results using less CPU (as using the whisper-cli tool), but the transcription latency will be higher, thus not useful to process real-time streams. Consider using the vad_model option associated with a large queue value. Default value: <code class=”code”>"3"</code> </dd> <dt><samp class=”option”>use_gpu</samp></dt> <dd> If the GPU support should be enabled. Default value: <code class=”code”>"true"</code> </dd> <dt><samp class=”option”>gpu_device</samp></dt> <dd> The GPU device index to use. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>destination</samp></dt> <dd> If set, the transcription output will be sent to the specified file or URL (use one of the FFmpeg AVIO protocols); otherwise, the output will be logged as info messages. The output will also be set in the "lavfi.whisper.text" frame metadata. If the destination is a file and it already exists, it will be overwritten. </dd> <dt><samp class=”option”>format</samp></dt> <dd> The destination format string; it could be "text" (only the transcribed text will be sent to the destination), "srt" (subtitle format) or "json". JSON segments include a <code class=”code”>language</code> field containing the language code reported by whisper.cpp for the transcription chunk, or <code class=”code”>unknown</code> if none is available. All segments from the same chunk have the same language. Default value: <code class=”code”>"text"</code> </dd> <dt><samp class=”option”>max_len</samp></dt> <dd> Maximum segment length in characters. When set to a value greater than 0, transcription segments will be split by word to not exceed this length. This is useful for generating subtitles with shorter lines. Default value: <code class=”code”>"0"</code> </dd> <dt><samp class=”option”>vad_model</samp></dt> <dd> Path to the VAD model file. If set, the filter will load an additional voice activity detection module (https://github.com/snakers4/silero-vad) that will be used to fragment the audio queue; use this option setting a valid path obtained from the whisper.cpp repository (e.g. "../whisper.cpp/models/ggml-silero-v5.1.2.bin") and increase the queue parameter to a higher value (e.g. 20). </dd> <dt><samp class=”option”>vad_threshold</samp></dt> <dd> The VAD threshold to use. Default value: <code class=”code”>"0.5"</code> </dd> <dt><samp class=”option”>vad_min_speech_duration</samp></dt> <dd> The minimum VAD speaking duration. Default value: <code class=”code”>"0.1"</code> </dd> <dt><samp class=”option”>vad_min_silence_duration</samp></dt> <dd> The minimum VAD silence duration. Default value: <code class=”code”>"0.5"</code>

 </dd> </dl>

 <h4 class="subsection">29.122.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Run a transcription with srt file generation: <pre class="example-preformatted">ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=output.srt\ :format=srt" -f null - </pre>

Run a transcription and send the output in JSON format to an HTTP service: <pre class="example-preformatted">ffmpeg -i input.mp4 -vn -af "whisper=model=../whisper.cpp/models/ggml-base.en.bin\ :language=en\ :queue=3\ :destination=http\\://localhost\\:3000\ :format=json' -f null - </pre>

Transcribe the microphone input using the VAD option: <pre class="example-preformatted">ffmpeg -loglevel warning -f pulse -i default \ -af 'highpass=f=200,lowpass=f=3000,whisper=model=../whisper.cpp/models/ggml-medium.bin\ :language=en\ :queue=10\ :destination=-\ :format=json\ :vad_model=../whisper.cpp/models/ggml-silero-v5.1.2.bin' -f null - </pre>

 <h2 class="chapter">30 Audio Sources# TOC</h2>

Below is a description of the currently available audio sources. <h3 class="section">30.1 abuffer# TOC</h3>

Buffer audio frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">time_base</samp></dt> <dd> The timebase which will be used for timestamps of submitted frames. It must be either a floating-point number or in <var class="var">numerator</var>/<var class="var">denominator</var> form. </dd> <dt><samp class="option">sample_rate</samp></dt> <dd> The sample rate of the incoming audio buffers. </dd> <dt><samp class="option">sample_fmt</samp></dt> <dd> The sample format of the incoming audio buffers. Either a sample format name or its corresponding integer representation from the enum AVSample-

Format in <samp class="file">libavutil/samplefmt.h</samp> </dd> <dt><samp class="option">channel_layout</samp></dt> <dd> The channel layout of the incoming audio buffers. Either a channel layout name from channel_layout_map in <samp class="file">libavutil/channel_layout.c</samp> or its corresponding integer representation from the AV_CH_LAYOUT_* macros in <samp class="file">libavutil/channel_layout.h</samp> </dd> <dt><samp class="option">channels</samp></dt> <dd> The number of channels of the incoming audio buffers. If both <var class="var">channels</var> and <var class="var">channel_layout</var> are specified, then they must be consistent. </dd> </dl>

 <h4 class="subsection">30.1.1 Examples# TOC</h4>

 <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=s16p
</pre>

will instruct the source to accept planar 16bit signed stereo at 44100Hz. Since the sample format with name "s16p" corresponds to the number 6 and the "stereo" channel layout corresponds to the value 0x3, this is equivalent to: <pre class="example-preformatted">abuffer=sample_rate=44100:sample_fmt=6:channel_layout=0x3
</pre>

 <h3 class="section">30.2 aevalsrc# TOC</h3>

Generate an audio signal specified by an expression. This source accepts in input one or more expressions (one for each channel), which are evaluated and used to generate a corresponding audio signal. This source accepts the following options: <dl class="table"> <dt><samp class="option">exprs</samp></dt> <dd> Set the ’|’-separated expressions list for each separate channel. In case the <samp class="option">channel_layout</samp> option is not specified, the selected channel layout depends on the number of provided expressions. Otherwise the last specified expression is applied to the remaining output channels. </dd> <dt><samp class="option">channel_layout, c</samp></dt> <dd> Set the channel layout. The number of channels in the specified layout must be equal to the number of specified expressions. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the minimum duration of the sourced audio. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the resulting duration may be greater than the specified duration, as the generated

audio is always cut at the end of a complete frame. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per channel per each output frame, default to 1024. </dd> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Specify the sample rate, default to 44100. </dd> </dl>

Each expression in <var class="var">exprs</var> can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> number of the evaluated sample, starting from 0 </dd> <dt><samp class="option">t</samp></dt> <dd> time of the evaluated sample expressed in seconds, starting from 0 </dd> <dt><samp class="option">s</samp></dt> <dd> sample rate </dd> </dl>

 <h4 class="subsection">30.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate silence: <pre class="example-preformatted">aevalsrc=0 </pre>

Generate a sin signal with frequency of 440 Hz, set sample rate to 8000 Hz: <pre class="example-preformatted">aevalsrc="sin(440*2*PI*t):s=8000" </pre>

Generate a two channels signal, specify the channel layout (Front Center + Back Center) explicitly: <pre class="example-preformatted">aevalsrc="sin(420*2*PI*t)|cos(430*2*PI*t):c=FC|BC" </pre>

Generate white noise: <pre class="example-preformatted">aevalsrc="-2+random(0)" </pre>

Generate an amplitude modulated signal: <pre class="example-preformatted">aevalsrc="sin(10*2*PI*t)*sin(880*2*PI*t)" </pre>

Generate 2.5 Hz binaural beats on a 360 Hz carrier: <pre class="example-preformatted">aevalsrc="0.1*sin(2*PI*(360-2.5/2)*t) | 0.1*sin(2*PI*(360+2.5/2)*t)" </pre>

 <h3 class="section">30.3 afdelaysrc# TOC</h3>

Generate a fractional delay FIR coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>delay, d</samp></dt> <dd> Set the fractional delay. Default is 0. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Set the sample rate, default is 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per each frame. Default is 1024. </dd> <dt><samp class=”option”>taps, t</samp></dt> <dd> Set the number of filter coefficients in output audio stream. Default value is 0. </dd> <dt><samp class=”option”>channel_layout, c</samp></dt> <dd> Specifies the channel layout, and can be a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". </dd> </dl>

 <h3 class=”section”>30.4 afireqsrc# TOC</h3>

Generate a FIR equalizer coefficients. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>preset, p</samp></dt> <dd> Set equalizer preset. Default preset is <code class=”code”>flat</code>. Available presets are: <dl class=”table”> <dt>‘<samp class=”samp”>custom</samp>’</dt> <dt>‘<samp class=”samp”>flat</samp>’</dt> <dt>‘<samp class=”samp”>acoustic</samp>’</dt> <dt>‘<samp class=”samp”>bass</samp>’</dt> <dt>‘<samp class=”samp”>beats</samp>’</dt> <dt>‘<samp class=”samp”>classic</samp>’</dt> <dt>‘<samp class=”samp”>clear</samp>’</dt> <dt>‘<samp class=”samp”>deep bass</samp>’</dt> <dt>‘<samp class=”samp”>dubstep</samp>’</dt> <dt>‘<samp class=”samp”>electronic</samp>’</dt> <dt>‘<samp class=”samp”>hard-style</samp>’</dt> <dt>‘<samp class=”samp”>hip-hop</samp>’</dt> <dt>‘<samp class=”samp”>jazz</samp>’</dt> <dt>‘<samp class=”samp”>metal</samp>’</dt> <dt>‘<samp class=”samp”>movie</samp>’</dt> <dt>‘<samp class=”samp”>pop</samp>’</dt> <dt>‘<samp class=”samp”>r&b</samp>’</dt> <dt>‘<samp class=”samp”>rock</samp>’</dt> <dt>‘<samp class=”samp”>vocal booster</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gains, g</samp></dt> <dd> Set custom gains for each band. Only used if the preset option is set to <code class=”code”>custom</code>. Gains are separated by white spaces and each gain is set in dBFS. Default is <code class=”code”>0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0</code>. </dd> <dt><samp class="option">bands, b</samp></dt> <dd> Set the custom bands from where custom equalizer gains are set. This must be in strictly increasing order. Only used if the preset option is set to <code class="code">custom</code>. Bands are separated by white spaces and each band represent frequency in Hz. Default is <code class="code">25 40 63 100 160 250 400 630 1000 1600 2500 4000 6300 10000 16000 24000</code>. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is <code class="code">4096</code>. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate of output audio stream, default is <code class="code">44100</code>. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame in output audio stream. Default is <code class="code">1024</code>. </dd> <dt><samp class="option">interp, i</samp></dt> <dd> Set interpolation method for FIR equalizer coefficients. Can be <code class="code">linear</code> or <code class="code">cubic</code>. </dd> <dt><samp class="option">phase, h</samp></dt> <dd> Set phase type of FIR filter. Can be <code class="code">linear</code> or <code class="code">min</code>: minimum-phase. Default is minimum-phase filter. </dd> </dl>

 <h3 class="section">30.5 afirsrc# TOC</h3>

Generate a FIR coefficients using frequency sampling method. The resulting stream can be used with afir filter for filtering the audio signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">taps, t</samp></dt> <dd> Set number of filter coefficients in output audio stream. Default value is 1025. </dd> <dt><samp class="option">frequency, f</samp></dt> <dd> Set frequency points from where magnitude and phase are set. This must be in non decreasing order, and first element must be 0, while last element must be 1. Elements are separated by white spaces. </dd> <dt><samp class="option">magnitude, m</samp></dt> <dd> Set magnitude value for every frequency point set by <samp class="option">frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class="option">phase, p</samp></dt> <dd> Set phase value for every frequency point set by <samp class="option">frequency</samp>. Number of values must be same as number of frequency points. Values are separated by white spaces. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp

class=”option”>win_func, w</samp></dt> <dd> Set window function. Default is blackman. </dd> </dl>

 <h3 class=”section”>30.6 anullsrc# TOC</h3>

The null audio source, return unprocessed audio frames. It is mainly useful as a template and to be employed in analysis / debugging tools, or as the source for filters which ignore the input data (for example the sox synth filter). This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>channel_layout, cl</samp></dt> <dd> Specifies the channel layout, and can be either an integer or a string representing a channel layout. The default value of <var class=”var”>channel_layout</var> is "stereo". Check the channel_layout_map definition in <samp class=”file”>libavutil/channel_layout.c</samp> for the mapping between strings and channel layout values. </dd> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specifies the sample rate, and defaults to 44100. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the number of samples per requested frames. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced audio. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the audio is supposed to be generated forever. </dd> </dl>

 <h4 class=”subsection”>30.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the sample rate to 48000 Hz and the channel layout to AV_CH_LAYOUT_MONO. <pre class=”example-preformatted”>anullsrc=r=48000:cl=4 </pre>

Do the same operation with a more obvious syntax: <pre class=”example-preformatted”>anullsrc=r=48000:cl=mono </pre>

All the parameters need to be explicitly defined. <h3 class=”section”>30.7 flite# TOC</h3>

Synthesize a voice utterance using the libflite library. To enable compilation of this filter you need to configure FFmpeg with <code

class=”code”>--enable-libflite</code>. Note that versions of the flite library prior to 2.0 are not thread-safe. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>list_voices</samp></dt> <dd> If set to 1, list the names of the available voices and exit immediately. Default value is 0. </dd> <dt><samp class=”option”>nb_samples, n</samp></dt> <dd> Set the maximum number of samples per frame. Default value is 512. </dd> <dt><samp class=”option”>textfile</samp></dt> <dd> Set the filename containing the text to speak. </dd> <dt><samp class=”option”>text</samp></dt> <dd> Set the text to speak. </dd> <dt><samp class=”option”>voice, v</samp></dt> <dd> Set the voice to use for the speech synthesis. Default value is <code class=”code”>kal</code>. See also the <var class=”var”>list_voices</var> option. </dd> </dl>

 <h4 class=”subsection”>30.7.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Read from file <samp class=”file”>speech.txt</samp>, and synthesize the text using the standard flite voice: <pre class=”example-preformatted”>flite=textfile=speech.txt </pre>

Read the specified text selecting the <code class=”code”>slt</code> voice: <pre class=”example-preformatted”>flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Input text to ffmpeg: <pre class=”example-preformatted”>ffmpeg -f lavfi -i flite=text='So fare thee well, poor devil of a Sub-Sub, whose commentator I am':voice=slt </pre>

Make <samp class=”file”>ffplay</samp> speak the specified text, using <code class=”code”>flite</code> and the <code class=”code”>lavfi</code> device: <pre class=”example-preformatted”>ffplay -f lavfi flite=text='No more be grieved for which that thou hast done.' </pre>

For more information about libflite, check: <a class=”url” href=”http://www.festvox.org/flite/”>http://w
  <h3 class=”section”>30.8 anoisesrc# TOC</h3>

Generate a noise audio signal. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate, r</samp></dt> <dd> Specify the sample rate. Default value is 48000 Hz. </dd> <dt><samp class=”option”>amplitude, a</samp></dt>

<dd> Specify the amplitude (0.0 - 1.0) of the generated audio stream. Default value is 1.0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. Not specifying this option results in noise with an infinite length. </dd> <dt><samp class="option">color, colour, c</samp></dt> <dd> Specify the color of noise. Available noise colors are white, pink, brown, blue, violet and velvet. Default color is white. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Specify a value used to seed the PRNG. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set the number of samples per each output frame, default is 1024. </dd> <dt><samp class="option">density</samp></dt> <dd> Set the density (0.0 - 1.0) for the velvet noise generator, default is 0.05. </dd> </dl>

 <h4 class="subsection">30.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate 60 seconds of pink noise, with a 44.1 kHz sampling rate and an amplitude of 0.5: <pre class="example-preformatted">anoisesrc=d=60:c=pink:r=44100:a=0.5 </pre>

 <h3 class="section">30.9 hilbert# TOC</h3>

Generate odd-tap Hilbert transform FIR coefficients. The resulting stream can be used with afir filter for phase-shifting the signal by 90 degrees. This is used in many matrix coding schemes and for analytic signal generation. The process is often written as a multiplication by i (or j), the imaginary unit. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, s</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">taps, t</samp></dt> <dd> Set length of FIR filter, default is 22051. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. </dd> <dt><samp class="option">win_func, w</samp></dt> <dd> Set window function to be used when generating FIR coefficients. </dd> </dl>

 <h3 class="section">30.10 sinc# TOC</h3>

Generate a sinc kaiser-windowed low-pass, high-pass, band-pass, or band-reject FIR coefficients. The resulting stream can be used with afir filter for filtering the audio

signal. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Set sample rate, default is 44100. </dd> <dt><samp class="option">nb_samples, n</samp></dt> <dd> Set number of samples per each frame. Default is 1024. </dd> <dt><samp class="option">hp</samp></dt> <dd> Set high-pass frequency. Default is 0. </dd> <dt><samp class="option">lp</samp></dt> <dd> Set low-pass frequency. Default is 0. If high-pass frequency is lower than low-pass frequency and low-pass frequency is higher than 0 then filter will create band-pass filter coefficients, otherwise band-reject filter coefficients. </dd> <dt><samp class="option">phase</samp></dt> <dd> Set filter phase response. Default is 50. Allowed range is from 0 to 100. </dd> <dt><samp class="option">beta</samp></dt> <dd> Set Kaiser window beta. </dd> <dt><samp class="option">att</samp></dt> <dd> Set stop-band attenuation. Default is 120dB, allowed range is from 40 to 180 dB. </dd> <dt><samp class="option">round</samp></dt> <dd> Enable rounding, by default is disabled. </dd> <dt><samp class="option">hptaps</samp></dt> <dd> Set number of taps for high-pass filter. </dd> <dt><samp class="option">lptaps</samp></dt> <dd> Set number of taps for low-pass filter. </dd> </dl>

 <h3 class="section">30.11 sine# TOC</h3>

Generate an audio signal made of a sine wave with amplitude 1/8. The audio signal is bit-exact. The filter accepts the following options: <dl class="table"> <dt><samp class="option">frequency, f</samp></dt> <dd> Set the carrier frequency. Default is 440 Hz. </dd> <dt><samp class="option">beep_factor, b</samp></dt> <dd> Enable a periodic beep every second with frequency <var class="var">beep_factor</var> times the carrier frequency. Default is 0, meaning the beep is disabled. </dd> <dt><samp class="option">sample_rate, r</samp></dt> <dd> Specify the sample rate, default is 44100. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Specify the duration of the generated audio stream. </dd> <dt><samp class="option">samples_per_frame</samp></dt> <dd> Set the number of samples per output frame. The expression can contain the following constants: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> The (sequential) number of the output audio frame, starting from 0. </dd> <dt><samp class="option">pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the output audio frame, expressed in <var class="var">TB</var> units. </dd> <dt><samp class="option">t</samp></dt> <dd> The PTS of the output audio frame, expressed in seconds. </dd> <dt><samp class="option">TB</samp></dt> <dd> The timebase of the output audio frames. </dd> </dl>

Default is <code class="code">1024</code>. </dd> </dl>

 <h4 class="subsection">30.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a simple 440 Hz sine wave: <pre class="example-preformatted">sine
</pre>

Generate a 220 Hz sine wave with a 880 Hz beep each second, for 5 seconds: <pre class="example-preformatted">sine=220:4:d=5 sine=f=220:b=4:d=5 sine=frequency=220:beep_factor=4:duration=5
</pre>

Generate a 1 kHz sine wave following <code class="code">1602,1601,1602,1601,1602</code> NTSC pattern: <pre class="example-preformatted">sine=1000:samples_per_frame=
1602-not(not(eq(ld(0),1)+eq(ld(0),3)))' </pre>

 <h2 class="chapter">31 Audio Sinks# TOC</h2>

Below is a description of the currently available audio sinks. <h3 class="section">31.1 abuffersink# TOC</h3>

Buffer audio frames, and make them available to the end of filter chain. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVABufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class="code">avfilter_init_filter</code> for initialization. <h3 class="section">31.2 anullsink# TOC</h3>

Null audio sink; do absolutely nothing with the input audio. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class="chapter">32 Video Filters# TOC</h2>

When you configure your FFmpeg build, you can disable any of the existing filters using <code class="code">--disable-filters</code>. The configure output will show the video filters included in your build. Below is a description of the currently available video filters. <h3 class="section">32.1 addroi# TOC</h3>

Mark a region of interest in a video frame. The frame data is passed through unchanged, but metadata is attached to the frame indicating regions of interest which can affect the behaviour of later encoding. Multiple regions can be marked by applying the filter multiple times. <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Region distance in pixels from the left edge of the frame. </dd> <dt><samp class="option">y</samp></dt> <dd> Region distance in pixels from the top edge of the frame. </dd> <dt><samp class="option">w</samp></dt> <dd> Region width in pixels. </dd> <dt><samp class="option">h</samp></dt> <dd> Region height in pixels. The parameters <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> are expressions, and may contain the following variables: <dl class="table"> <dt><samp class="option">iw</samp></dt> <dd> Width of the input frame. </dd> <dt><samp class="option">ih</samp></dt> <dd> Height of the input frame. </dd> </dl>

</dd> <dt><samp class="option">qoffset</samp></dt> <dd> Quantisation offset to apply within the region. This must be a real value in the range -1 to +1. A value of zero indicates no quality change. A negative value asks for better quality (less quantisation), while a positive value asks for worse quality (greater quantisation). The range is calibrated so that the extreme values indicate the largest possible offset - if the rest of the frame is encoded with the worst possible quality, an offset of -1 indicates that this region should be encoded with the best possible quality anyway. Intermediate values are then interpolated in some codec-dependent way. For example, in 10-bit H.264 the quantisation parameter varies between -12 and 51. A typical qoffset value of -1/10 therefore indicates that this region should be encoded with a QP around one-tenth of the full range better than the rest of the frame. So, if most of the frame were to be encoded with a QP of around 30, this region would get a QP of around 24 (an offset of approximately -1/10 * (51 - -12) = -6.3). An extreme value of -1 would indicate that this region should be encoded with the best possible quality regardless of the treatment of the rest of the frame - that is, should be encoded at a QP of -12. </dd> <dt><samp class="option">clear</samp></dt> <dd> If set to true, remove any existing regions of interest marked on the frame before adding the new one. </dd> </dl>

 <h4 class="subsection">32.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Mark the centre quarter of the frame as interesting. <pre class="example-preformatted">addroi=iw/4:ih/4:iw/2:ih/2:-1/10 </pre> Mark the 100-pixel-wide region on the left edge of the frame as very uninteresting (to be encoded at much lower quality than the rest of the frame). <pre class="example-preformatted">addroi=0:0:100:ih:+1/5 </pre>

 <h3 class="section">32.2 alphaextract# TOC</h3>

Extract the alpha component from the input as a grayscale video. This is especially useful with the <var class="var">alphamerge</var> filter. <h3 class="section">32.3 alphamerge# TOC</h3>

Add or replace the alpha component of the primary input with the grayscale value of a second input. This is intended for use with <var class="var">alphaextract</var> to allow the transmission or storage of frame sequences that have alpha in a format that doesn’t support an alpha channel. For example, to reconstruct full frames from a normal YUV-encoded video and a separate video created with <var class="var">alphaextract</var>, you might use: <pre class="example-preformatted">movie=in_alpha.mkv [alpha]; [in][alpha] alphamerge [out] </pre>

 <h3 class="section">32.4 amplify# TOC</h3>

Amplify differences between current pixel and pixels of adjacent frames in same pixel location. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set frame radius. Default is 2. Allowed range is from 1 to 63. For example radius of 3 will instruct filter to calculate average of 7 frames. </dd> <dt><samp class="option">factor</samp></dt> <dd> Set factor to amplify difference. Default is 2. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold for difference amplification. Any difference greater or equal to this

value will not alter source pixel. Default is 10. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">tolerance</samp></dt> <dd> Set tolerance for difference amplification. Any difference lower to this value will not alter source pixel. Default is 0. Allowed range is from 0 to 65535. </dd> <dt><samp class="option">low</samp></dt> <dd> Set lower limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will decrease source pixel value. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high limit for changing source pixel. Default is 65535. Allowed range is from 0 to 65535. This option controls maximum possible value that will increase source pixel value. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">32.4.1 Commands# TOC</h4>

This filter supports the following commands that corresponds to option of same name: <dl class="table"> <dt><samp class="option">factor</samp></dt> <dt><samp class="option">threshold</samp></dt> <dt><samp class="option">tolerance</samp></dt> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dt><samp class="option">planes</samp></dt> </dl>

 <h3 class="section">32.5 ass# TOC</h3>

Same as the subtitles filter, except that it doesn’t require libavcodec and libavformat to work. On the other hand, it is limited to ASS (Advanced Substation Alpha) subtitles files. This filter accepts <samp class="option">filename</samp>/<samp class="option">f</samp>, <samp class="option">original_size</samp>, <samp class="option">fontsdir</samp>, and <samp class="option">alpha</samp> from the subtitles filter, plus the following options: <dl class="table"> <dt><samp class="option">script</samp></dt> <dd> The ASS script to render, given inline. Exactly one of <samp class="option">filename</samp> and <samp class="option">script</samp> must be set. </dd> <dt><samp class="option">shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class="samp">simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class="samp">complex</samp>’</dt>

<dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew, Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd> </dl>

The default is <code class="code">auto</code>. </dd> </dl>

 <h4 class="subsection">32.5.1 Commands# TOC</h4>

This filter supports the <samp class="option">script</samp> option as commands; the new script replaces the one being rendered. <h3 class="section">32.6 atadenoise# TOC</h3> Apply an Adaptive Temporal Averaging Denoiser to the video input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">0a</samp></dt> <dd> Set threshold A for 1st plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">0b</samp></dt> <dd> Set threshold B for 1st plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">1a</samp></dt> <dd> Set threshold A for 2nd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">1b</samp></dt> <dd> Set threshold B for 2nd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">2a</samp></dt> <dd> Set threshold A for 3rd plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">2b</samp></dt> <dd> Set threshold B for 3rd plane. Default is 0.04. Valid range is 0 to 5. </dd> <dt><samp class="option">3a</samp></dt> <dd> Set threshold A for 4th plane. Default is 0.02. Valid range is 0 to 0.3. </dd> <dt><samp class="option">3b</samp></dt> <dd> Set threshold B for 4th plane. Default is 0.04. Valid range is 0 to 5. Threshold A is designed to react on abrupt changes in the input signal and threshold B is designed to react on continuous changes in the input signal. </dd> <dt><samp class="option">s</samp></dt> <dd> Set number of frames filter will use for averaging. Default is 9. Must be odd number in range [5, 129]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set what planes of frame filter will use for averaging. Default is all. </dd> <dt><samp class="option">a</samp></dt> <dd> Set what variant of algorithm filter will use for averaging. Default is <code class="code">p</code> parallel. Alternatively can be set to <code class="code">s</code> serial. Parallel can be faster then serial, while other way around is never true. Parallel will abort

early on first change being greater then thresholds, while serial will continue processing other side of frames if they are equal or below thresholds. </dd> <dt><samp class=”option”>0s</samp></dt> <dt><samp class=”option”>1s</samp></dt> <dt><samp class=”option”>2s</samp></dt> <dt><samp class=”option”>3s</samp></dt> <dd> Set sigma for 1st plane, 2nd plane, 3rd plane or 4th plane. Default is 32767. Valid range is from 0 to 32767. This options controls weight for each pixel in radius defined by size. Default value means every pixel have same weight. Setting this option to 0 effectively disables filtering. </dd> </dl>

 <h4 class=”subsection”>32.6.1 Commands# TOC</h4> This filter supports same commands as options except option <code class=”code”>s</code>. The command accepts the same syntax of the corresponding option. <h3 class=”section”>32.7 avgblur# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sizeX</samp></dt> <dd> Set horizontal radius size. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sizeY</samp></dt> <dd> Set vertical radius size, if zero it will be same as <code class=”code”>sizeX</code>. Default is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>32.7.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>32.8 backgroundkey# TOC</h3>

Turns a static background into transparency. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Threshold for scene change detection. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the background. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set the blend amount for pixels that are not similar. </dd> </dl>

 <h4 class="subsection">32.8.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.9 bbox# TOC</h3>

Compute the bounding box for the non-black pixels in the input frame luma plane. This filter computes the bounding box containing all the pixels with a luma value greater than the minimum allowed value. The parameters describing the bounding box are printed on the filter log. The filter accepts the following option: <dl class="table"> <dt><samp class="option">min_val</samp></dt> <dd> Set the minimal luma value. Default is <code class="code">16</code>. </dd> </dl>

 <h4 class="subsection">32.9.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.10 bilateral# TOC</h3> Apply bilateral filter, spatial smoothing while preserving edges. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight. Allowed range is 0 to 512. Default is 0.1. </dd> <dt><samp class="option">sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate range weight. Allowed range is 0 to 1. Default is 0.1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">32.10.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.11 bitplanenoise# TOC</h3>

Show and measure bit plane noise. The filter accepts the following options: <dl class="table"> <dt><samp class="option">bitplane</samp></dt> <dd> Set which plane to analyze. Default is <code class="code">1</code>. </dd> <dt><samp class="option">filter</samp></dt> <dd> Filter out noisy pixels from <code class="code">bitplane</code> set above. Default is disabled. </dd> </dl>

 <h3 class="section">32.12 blackdetect, blackdetect_vulkan# TOC</h3>

Detect video intervals that are (almost) completely black. Can be useful to detect chapter transitions, commercials, or invalid recordings. The filter outputs its detection analysis to both the log as well as frame metadata. If a black segment of at least the specified minimum duration is found, a line with the start and end timestamps as well as duration is printed to the log with level <code class="code">info</code>. In addition, a log line with level <code class="code">debug</code> is printed per frame showing the black amount detected for that frame. The filter also attaches metadata to the first frame of a black segment with key <code class="code">lavfi.black_start</code> and to the first frame after the black segment ends with key <code class="code">lavfi.black_end</code>. The value is the frame’s timestamp. This metadata is added regardless of the minimum duration specified. The filter accepts the following options: <dl class="table"> <dt><samp class="option">black_min_duration, d</samp></dt> <dd> Set the minimum detected black duration expressed in seconds. It must be a non-negative floating point number. Default value is 2.0. </dd> <dt><samp class="option">picture_black_ratio_th, pic_th</samp></dt> <dd> Set the threshold for considering a picture "black". Express the minimum value for the ratio: <pre class="example-preformatted"><var class="var">nb_black_pixels</var> / <var class="var">nb_pixels</var> </pre>

for which a picture is considered black. Default value is 0.98. </dd> <dt><samp class="option">pixel_black_th, pix_th</samp></dt> <dd> Set the threshold for considering a pixel "black". The threshold expresses the maximum pixel luma value for which a pixel is considered "black". The provided value is scaled according to the following equation: <pre class="example-preformatted"><var class="var">absolute_threshold</var> = <var class="var">luma_minimum_value</var> + <var class="var">pixel_black_th</var> * <var class="var">luma_range_size</var> </pre>

<var class="var">luma_range_size</var> and <var class="var">luma_minimum_value</var> depend on the input video format, the range is [0-255] for YUV full-range

formats and [16-235] for YUV non full-range formats. Default value is 0.10. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If true, check the alpha channel instead of the luma channel. Detects frames which are (almost) transparent, instead of frames which are almost black. Default value is disabled. </dd> </dl>

The following example sets the maximum pixel threshold to the minimum value, and detects only black intervals of 2 or more seconds: <pre class="example-preformatted">blackdetect=d=2:pix_th=0.00
</pre>

 <h3 class="section">32.13 blackframe# TOC</h3>

Detect frames that are (almost) completely black. Can be useful to detect chapter transitions or commercials. Output lines consist of the frame number of the detected frame, the percentage of blackness, the position in the file if known or -1 and the timestamp in seconds. In order to display the output lines, you need to set the loglevel at least to the AV_LOG_INFO value. This filter exports frame metadata <code class="code">lavfi.blackframe.pblack</code>. The value represents the percentage of pixels in the picture that are below the threshold value. It accepts the following parameters: <dl class="table"> <dt><samp class="option">amount</samp></dt> <dd> The percentage of the pixels that have to be below the threshold; it defaults to <code class="code">98</code>. </dd> <dt><samp class="option">threshold, thresh</samp></dt> <dd> The threshold below which a pixel value is considered black; it defaults to <code class="code">32</code>. </dd> </dl>

 <h3 class="section">32.14 blend# TOC</h3>

Blend two video frames into each other. The <code class="code">blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output terminates when the longest input terminates. The <code class="code">tblend</code> (time blend) filter takes two consecutive frames from one single stream, and outputs the result obtained by blending the new frame on top of the old frame. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">c0_mode</samp></dt> <dt><samp class="option">c1_mode</samp></dt> <dt><samp class="option">c2_mode</samp></dt> <dt><samp class="option">c3_mode</samp></dt>

<dt><samp class="option">all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class="var">all_mode</var>. Default value is <code class="code">normal</code>. Available values for component modes are: <dl class="table"> <dt>‘<samp class="samp">addition</samp>’</dt> <dt>‘<samp class="samp">and</samp>’</dt> <dt>‘<samp class="samp">average</samp>’</dt> <dt>‘<samp class="samp">bleach</samp>’</dt> <dt>‘<samp class="samp">burn</samp>’</dt> <dt>‘<samp class="samp">darken</samp>’</dt> <dt>‘<samp class="samp">difference</samp>’</dt> <dt>‘<samp class="samp">divide</samp>’</dt> <dt>‘<samp class="samp">dodge</samp>’</dt> <dt>‘<samp class="samp">exclusion</samp>’</dt> <dt>‘<samp class="samp">extremity</samp>’</dt> <dt>‘<samp class="samp">freeze</samp>’</dt> <dt>‘<samp class="samp">geometric</samp>’</dt> <dt>‘<samp class="samp">glow</samp>’</dt> <dt>‘<samp class="samp">grainextract</samp>’</dt> <dt>‘<samp class="samp">grainmerge</samp>’</dt> <dt>‘<samp class="samp">hardlight</samp>’</dt> <dt>‘<samp class="samp">hardmix</samp>’</dt> <dt>‘<samp class="samp">hardoverlay</samp>’</dt> <dt>‘<samp class="samp">harmonic</samp>’</dt> <dt>‘<samp class="samp">heat</samp>’</dt> <dt>‘<samp class="samp">interpolate</samp>’</dt> <dt>‘<samp class="samp">lighten</samp>’</dt> <dt>‘<samp class="samp">linearlight</samp>’</dt> <dt>‘<samp class="samp">multiply</samp>’</dt> <dt>‘<samp class="samp">multiply128</samp>’</dt> <dt>‘<samp class="samp">negation</samp>’</dt> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">or</samp>’</dt> <dt>‘<samp class="samp">overlay</samp>’</dt> <dt>‘<samp class="samp">phoenix</samp>’</dt> <dt>‘<samp class="samp">pinlight</samp>’</dt> <dt>‘<samp class="samp">reflect</samp>’</dt> <dt>‘<samp class="samp">screen</samp>’</dt> <dt>‘<samp class="samp">softdifference</samp>’</dt> <dt>‘<samp class="samp">softlight</samp>’</dt> <dt>‘<samp class="samp">stain</samp>’</dt> <dt>‘<samp class="samp">subtract</samp>’</dt> <dt>‘<samp class="samp">vividlight</samp>’</dt> <dt>‘<samp class="samp">xor</samp>’</dt> </dl>

</dd> <dt><samp class="option">c0_opacity</samp></dt> <dt><samp class="option">c1_opacity</samp></dt> <dt><samp class="option">c2_opacity</samp></dt> <dt><samp class="option">c3_opacity</samp></dt> <dt><samp class="option">all_opacity</samp></dt> <dd> Set blend opacity for specific pixel component or all pixel components in case of <var class="var">all_opacity</var>. Only used in combination with pixel component blend modes. </dd> <dt><samp class="option">c0_expr</samp></dt> <dt><samp class="option">c1_expr</samp></dt> <dt><samp class="option">c2_expr</samp></dt> <dt><samp class="option">c3_expr</samp></dt> <dt><samp class="option">all_expr</samp></dt> <dd> Set blend expression for specific pixel component or all pixel components in case of <var class="var">all_expr</var>. Note that

related mode options will be ignored if those are set. The expressions can use the following variables: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class="code">0</code>. </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> the coordinates of the current sample </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> the width and height of currently filtered plane </dd> <dt><samp class="option">SW</samp></dt> <dt><samp class="option">SH</samp></dt> <dd> Width and height scale for the plane being filtered. It is the ratio between the dimensions of the current plane to the luma plane, e.g. for a <code class="code">yuv420p</code> frame, the values are <code class="code">1,1</code> for the luma plane and <code class="code">0.5,0.5</code> for the chroma planes. </dd> <dt><samp class="option">T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class="option">TOP, A</samp></dt> <dd> Value of pixel component at current location for first video frame (top layer). </dd> <dt><samp class="option">BOTTOM, B</samp></dt> <dd> Value of pixel component at current location for second video frame (bottom layer). </dd> </dl> </dd> </dl>

The <code class="code">blend</code> filter also supports the framesync options. <h4 class="subsection">32.14.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply transition from bottom layer to top layer in first 10 seconds: <pre class="example-preformatted">blend=all_expr='A*(if(gte(T,10),1,T/10))+B*(1-(if(gte(T,10),1,T/10)))' </pre>

Apply linear horizontal transition from top layer to bottom layer: <pre class="example-preformatted">blend=all_expr='A*(X/W)+B*(1-X/W)' </pre>

Apply 1x1 checkerboard effect: <pre class="example-preformatted">blend=all_expr='if(eq(mod(X,2),mod(Y,2)),A,B)' </pre>

Apply uncover left effect: <pre class="example-preformatted">blend=all_expr='if(gte(N*SW+X,W),A,B)' </pre>

Apply uncover down effect: <pre class="example-preformatted">blend=all_expr='if(gte(Y-N*SH,0),A,B)' </pre>

Apply uncover up-left effect: <pre class=”example-preformatted”>blend=all_expr='if(gte(T*SH*40+Y,H)*gte((T*40*SW+X)*W/H,W),A,B)' </pre>

Split diagonally video and shows top and bottom layer on each side: <pre class=”example-preformatted”>blend=all_expr='if(gt(X,Y*(W/H)),A,B)' </pre>

Display differences between the current and the previous frame: <pre class=”example-preformatted”>tblend=all_mode=grainextract </pre>

 <h4 class=”subsection”>32.14.2 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>32.15 blockdetect# TOC</h3>

Determines blockiness of frames without altering the input frames. Based on Remco Muijs and Ihor Kirenko: "A no-reference blocking artifact measure for adaptive video processing." 2005 13th European signal processing conference. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>period_min</samp></dt> <dt><samp class=”option”>period_max</samp></dt> <dd> Set minimum and maximum values for determining pixel grids (periods). Default values are [3,24]. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>32.15.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Determine blockiness for the first plane and search for periods within [8,32]: <pre class=”example-preformatted”>blockdetect=period_min=8:period_max=32:planes=1 </pre>

 <h3 class=”section”>32.16 blurdetect# TOC</h3>

Determines blurriness of frames without altering the input frames. Based on Marziliano, Pina, et al. "A no-reference perceptual blur metric." Allows for a block-based abbreviation. The

filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">20/255</code>, and default value for <var class="var">high</var> is <code class="code">50/255</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Define the radius to search around an edge pixel for local maxima. </dd> <dt><samp class="option">block_pct</samp></dt> <dd> Determine blurriness only for the most significant blocks, given in percentage. </dd> <dt><samp class="option">block_width</samp></dt> <dd> Determine blurriness for blocks of width <var class="var">block_width</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class="var">block_height</var>. </dd> <dt><samp class="option">block_height</samp></dt> <dd> Determine blurriness for blocks of height <var class="var">block_height</var>. If set to any value smaller 1, no blocks are used and the whole image is processed as one no matter of <var class="var">block_width</var>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class="subsection">32.16.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Determine blur for 80% of most significant 32x32 blocks: <pre class="example-preformatted">blurdetect=block_width=32:block_height=32:block_pct=80 </pre>

 <h3 class="section">32.17 bm3d# TOC</h3>

Denoise frames using Block-Matching 3D algorithm. The filter accepts the following options. <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set denoising strength. Default value is 1. Allowed range is from 0 to 999.9. The denoising algorithm is very sensitive to sigma, so adjust it according to the source. </dd> <dt><samp class="option">block</samp></dt> <dd> Set local patch size. This sets dimensions in 2D. </dd> <dt><samp class="option">bstep</samp></dt> <dd> Set sliding step for pro-

cessing blocks. Default value is 4. Allowed range is from 1 to 64. Smaller values allows processing more reference blocks and is slower. </dd> <dt><samp class="option">group</samp></dt> <dd> Set maximal number of similar blocks for 3rd dimension. Default value is 1. When set to 1, no block matching is done. Larger values allows more blocks in single group. Allowed range is from 1 to 256. </dd> <dt><samp class="option">range</samp></dt> <dd> Set radius for search block matching. Default is 9. Allowed range is from 1 to INT32_MAX. </dd> <dt><samp class="option">mstep</samp></dt> <dd> Set step between two search locations for block matching. Default is 1. Allowed range is from 1 to 64. Smaller is slower. </dd> <dt><samp class="option">thmse</samp></dt> <dd> Set threshold of mean square error for block matching. Valid range is 0 to INT32_MAX. </dd> <dt><samp class="option">hdthr</samp></dt> <dd> Set thresholding parameter for hard thresholding in 3D transformed domain. Larger values results in stronger hard-thresholding filtering in frequency domain. </dd> <dt><samp class="option">estim</samp></dt> <dd> Set filtering estimation mode. Can be <code class="code">basic</code> or <code class="code">final</code>. Default is <code class="code">basic</code>. </dd> <dt><samp class="option">ref</samp></dt> <dd> If enabled, filter will use 2nd stream for block matching. Default is disabled for <code class="code">basic</code> value of <var class="var">estim</var> option, and always enabled if value of <var class="var">estim</var> is <code class="code">final</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter. Default is all available except alpha. </dd> </dl>

 <h4 class="subsection">32.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Basic filtering with bm3d: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=ba
</pre>

Same as above, but filtering only luma: <pre class="example-preformatted">bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic:planes=1
</pre>

Same as above, but with both estimation modes:
<pre class="example-preformatted">split[a][b],[a]bm3d=sigma=3:block=4:bstep=2:group=1:estim=basic[a],[b
</pre>

Same as above, but prefilter with nlmeans
filter instead: <pre class="example-preformatted">split[a][b],[a]nlmeans=s=3:r=7:p=
</pre>

 <h3 class="section">32.18 boxblur<span

class=”pull-right”># TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class=”code”>min(w,h)/2</code> for the luma and alpha planes, and of <code class=”code”>min(cw,ch)/2</code> for the chroma planes. Default value for <samp class=”option”>luma_radius</samp> is "2". If not specified, <samp class=”option”>chroma_radius</samp> and <samp class=”option”>alpha_radius</samp> default to the corresponding value set for <samp class=”option”>luma_radius</samp>. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class=”option”>cw</samp></dt> <dt><samp class=”option”>ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class=”option”>luma_power</samp> is 2. If not specified, <samp class=”option”>chroma_power</samp> and <samp class=”option”>alpha_power</samp> default to the corresponding value set for <samp class=”option”>luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class=”subsection”>32.18.1 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-68” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Apply a boxblur filter with the luma, chroma, and alpha radii set to 2: <pre class=”example-preformatted”>boxblur=luma_radius=2:luma_power=1
boxblur=2:1 </pre>

Set the luma radius to 2, and alpha and chroma radius to 0: <pre class=”example-preformatted”>boxblur=2:1:cr=0:ar=0
</pre>

Set the luma and chroma radii to a fraction of the video dimension:
 <pre class=”example-preformatted”>boxblur=luma_radius=min(h\,w)/10:luma_po
</pre>

 <h3 class=”section”>32.19 bwdif# TOC</h3>

Deinterlace the input video ("bwdif" stands for "Bob Weaver Deinterlacing Filter"). Motion adaptive deinterlacing based on yadif with the use of w3fdif and cubic interpolation algorithms. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>send_field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h3 class=”section”>32.20 ccrepack# TOC</h3>

Repack CEA-708 closed captioning side data This filter fixes various issues seen with commercial encoders related to upstream malformed CEA-708 payloads, specifically incorrect number of tuples (wrong cc_count for the target FPS), and incorrect ordering of tuples (i.e. the CEA-608 tuples are not at the first entries in the payload). <h3 class=”section”>32.21 cas# TOC</h3>

Apply Contrast Adaptive Sharpen filter to video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>strength</samp></dt> <dd> Set the sharpening strength. Default value is 0. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default value is to filter all planes except alpha plane. </dd> </dl>

 <h4 class=”subsection”>32.21.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>32.22 chromahold# TOC</h3> Remove all color information for all colors except for certain one. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt> <dd> The color which will not be replaced with neutral chroma. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more preserved color. </dd> <dt><samp class=”option”>yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">32.22.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.23 chromakey# TOC</h3> YUV colorspace color/chroma keying. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will be replaced with transparency. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> <dt><samp class="option">yuv</samp></dt> <dd> Signals that the color passed is already in YUV instead of RGB. Literal colors like "green" or "red" don’t make sense with this enabled anymore. This can be used to pass exact YUV values as hexadecimal numbers. </dd> </dl>

 <h4 class="subsection">32.23.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">32.23.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf chromakey=green out.png </pre>

Overlay a greenscreen-video on top of a static black background. <pre class="example-preformatted">ffmpeg -f lavfi -i color=c=black:s=1280x720 -i video.mp4 -shortest -filter_complex

"[1:v]chromakey=0x70de77:0.1:0.2[ckout];[0:v][ckout]overlay[out]"
-map "[out]" output.mkv </pre>

 <h3 class="section">32.24 chromanr# TOC</h3> Reduce chrominance noise. The filter accepts the following options: <dl class="table"> <dt><samp class="option">thres</samp></dt> <dd> Set threshold for averaging chrominance values. Sum of absolute difference of Y, U and V pixel components of current pixel and neighbour pixels lower than this threshold will be used in averaging. Luma component is left unchanged and is copied to output. Default value is 30. Allowed range is from 1 to 200. </dd> <dt><samp class="option">sizew</samp></dt> <dd> Set horizontal radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">sizeh</samp></dt> <dd> Set vertical radius of rectangle used for averaging. Allowed range is from 1 to 100. Default value is 5. </dd> <dt><samp class="option">stepw</samp></dt> <dd> Set horizontal step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">steph</samp></dt> <dd> Set vertical step when averaging. Default value is 1. Allowed range is from 1 to 50. Mostly useful to speed-up filtering. </dd> <dt><samp class="option">threy</samp></dt> <dd> Set Y threshold for averaging chrominance values. Set finer control for max allowed difference between Y components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threu</samp></dt> <dd> Set U threshold for averaging chrominance values. Set finer control for max allowed difference between U components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">threv</samp></dt> <dd> Set V threshold for averaging chrominance values. Set finer control for max allowed difference between V components of current pixel and neighbour pixels. Default value is 200. Allowed range is from 1 to 200. </dd> <dt><samp class="option">distance</samp></dt> <dd> Set distance type used in calculations. <dl class="table"> <dt>‘<samp class="samp">manhattan</samp>’</dt> <dd> Absolute difference. </dd> <dt>‘<samp class="samp">euclidean</samp>’</dt> <dd> Difference squared. </dd> </dl>

Default distance type is manhattan. </dd> </dl>

 <h4 class="subsection">32.24.1 Commands# TOC</h4> This

filter supports same commands as options. The command accepts the same syntax of the corresponding option. <h3 class=”section”>32.25 chromashift# TOC</h3> Shift chroma pixels horizontally and/or vertically. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cbh</samp></dt> <dd> Set amount to shift chroma-blue horizontally. </dd> <dt><samp class=”option”>cbv</samp></dt> <dd> Set amount to shift chroma-blue vertically. </dd> <dt><samp class=”option”>crh</samp></dt> <dd> Set amount to shift chroma-red horizontally. </dd> <dt><samp class=”option”>crv</samp></dt> <dd> Set amount to shift chroma-red vertically. </dd> <dt><samp class=”option”>edge</samp></dt> <dd> Set edge mode, can be <var class=”var”>smear</var>, default, or <var class=”var”>warp</var>. </dd> </dl>

 <h4 class=”subsection”>32.25.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.26 ciescope# TOC</h3>

Display CIE color diagram with pixels overlaid onto it. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>system</samp></dt> <dd> Set color system. <dl class=”table”> <dt>‘<samp class=”samp”>ntsc, 470m</samp>’</dt> <dt>‘<samp class=”samp”>ebu, 470bg</samp>’</dt> <dt>‘<samp class=”samp”>smpte</samp>’</dt> <dt>‘<samp class=”samp”>240m</samp>’</dt> <dt>‘<samp class=”samp”>apple</samp>’</dt> <dt>‘<samp class=”samp”>widergb</samp>’< <dt>‘<samp class=”samp”>cie1931</samp>’</dt> <dt>‘<samp class=”samp”>rec709, hdtv</samp>’</dt> <dt>‘<samp class=”samp”>uhdtv, rec2020</samp>’</dt> <dt>‘<samp class=”samp”>dcip3</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>cie</samp></dt> <dd> Set CIE system. <dl class=”table”> <dt>‘<samp class=”samp”>xyy</samp>’</dt> <dt>‘<samp class=”samp”>ucs</samp>’</dt> <dt>‘<samp class=”samp”>luv</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>gamuts</samp></dt> <dd> Set

what gamuts to draw. See <code class=”code”>system</code> option for available values. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set ciescope size, by default set to 512. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity used to map input pixel values to CIE diagram. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Set contrast used to draw tongue colors that are out of active color system gamut. </dd> <dt><samp class=”option”>corrgamma</samp></dt> <dd> Correct gamma displayed on scope, by default enabled. </dd> <dt><samp class=”option”>showwhite</samp></dt> <dd> Show white point on CIE diagram, by default disabled. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Set input gamma. Used only with XYZ input color space. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> Fill with CIE colors. By default is enabled. </dd> </dl>

 <h3 class=”section”>32.27 codecview# TOC</h3>

Visualize information exported by some codecs. Some codecs can export information through frames using side-data or other means. For example, some MPEG based codecs export motion vectors through the <var class=”var”>export_mvs</var> flag in the codec <samp class=”option”>flags2</samp> option. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>block</samp></dt> <dd> Display block partition structure using the luma plane. </dd> <dt><samp class=”option”>mv</samp></dt> <dd> Set motion vectors to visualize. Available flags for <var class=”var”>mv</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>pf</samp>’</dt> <dd> forward predicted MVs of P-frames </dd> <dt>‘<samp class=”samp”>bf</samp>’</dt> <dd> forward predicted MVs of B-frames </dd> <dt>‘<samp class=”samp”>bb</samp>’</dt> <dd> backward predicted MVs of B-frames </dd> </dl>

</dd> <dt><samp class=”option”>qp</samp></dt> <dd> Display quantization parameters using the chroma planes. </dd> <dt><samp class=”option”>mv_type, mvt</samp></dt> <dd> Set motion vectors type to visualize. Includes MVs from all frames unless specified by <var class=”var”>frame_type</var> option. Available flags for <var class=”var”>mv_type</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>fp</samp>’</dt> <dd> forward predicted MVs </dd> <dt>‘<samp class=”samp”>bp</samp>’</dt> <dd> backward predicted MVs </dd> </dl>

</dd> <dt><samp class=”option”>frame_type, ft</samp></dt>

<dd> Set frame type to visualize motion vectors of. Available flags for <var class="var">frame_type</var> are: <dl class="table"> <dt>‘<samp class="samp">if</samp>’</dt> <dd> intra-coded frames (I-frames) </dd> <dt>‘<samp class="samp">pf</samp>’</dt> <dd> predicted frames (P-frames) </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> bi-directionally predicted frames (B-frames) </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.27.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Visualize forward predicted MVs of all frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv_type=fp </pre>

Visualize multi-directionals MVs of P and B-Frames using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -flags2 +export_mvs input.mp4 -vf codecview=mv=pf+bf+bb </pre>

 <h3 class="section">32.28 colorbalance# TOC</h3> Modify intensity of primary colors (red, green and blue) of input frames. The filter allows an input frame to be adjusted in the shadows, midtones or highlights regions for the red-cyan, green-magenta or blue-yellow balance. A positive adjustment value shifts the balance towards the primary color, a negative value towards the complementary color. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rs</samp></dt> <dt><samp class="option">gs</samp></dt> <dt><samp class="option">bs</samp></dt> <dd> Adjust red, green and blue shadows (darkest pixels). </dd> <dt><samp class="option">rm</samp></dt> <dt><samp class="option">gm</samp></dt> <dt><samp class="option">bm</samp></dt> <dd> Adjust red, green and blue midtones (medium pixels). </dd> <dt><samp class="option">rh</samp></dt> <dt><samp class="option">gh</samp></dt> <dt><samp class="option">bh</samp></dt> <dd> Adjust red, green and blue highlights (brightest pixels). Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">pl</samp></dt> <dd> Preserve lightness when changing color balance. Default is disabled. </dd> </dl>

 <h4 class="subsection">32.28.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Add red color cast to shadows: <pre class="example-preformatted">colorbalance=rs=.3 </pre>

 <h4 class="subsection">32.28.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.29 colorcontrast# TOC</h3>

Adjust color contrast between RGB components. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rc</samp></dt> <dd> Set the red-cyan contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">gm</samp></dt> <dd> Set the green-magenta contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">by</samp></dt> <dd> Set the blue-yellow contrast. Defaults is 0.0. Allowed range is from -1.0 to 1.0. </dd> <dt><samp class="option">rcw</samp></dt> <dt><samp class="option">gmw</samp></dt> <dt><samp class="option">byw</samp></dt> <dd> Set the weight of each <code class="code">rc</code>, <code class="code">gm</code>, <code class="code">by</code> option value. Default value is 0.0. Allowed range is from 0.0 to 1.0. If all weights are 0.0 filtering is disabled. </dd> <dt><samp class="option">pl</samp></dt> <dd> Set the amount of preserving lightness. Default value is 0.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">32.29.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.30 colorcorrect# TOC</h3>

Adjust color white balance selectively for blacks and whites. This filter operates in YUV colorspace. The filter accepts the following options:

 <dl class="table"> <dt><samp class="option">rl</samp></dt>
<dd> Set the red shadow spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">bl</samp></dt>
<dd> Set the blue shadow spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">rh</samp></dt>
<dd> Set the red highlight spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">bh</samp></dt>
<dd> Set the blue highlight spot. Allowed range is from -1.0 to 1.0. Default
value is 0. </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the amount of saturation. Allowed range is from -3.0 to 3.0. De-
fault value is 1. </dd> <dt><samp class="option">analyze</samp></dt>
<dd> If set to anything other than <code class="code">manual</code>
it will analyze every frame and use derived parameters for filtering output frame.
 Possible values are: <dl class="table"> <dt>‘<samp
class="samp">manual</samp>’</dt> <dt>‘<samp class="samp">average</samp>’
<dt>‘<samp class="samp">minmax</samp>’</dt> <dt>‘<samp
class="samp">median</samp>’</dt> </dl> Default value is
<code class="code">manual</code>. </dd> </dl>

 <h4 class="subsection">32.30.1 Com-
mands<a class="anchor hidden-xs" href="#Commands-
69" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Commands-69" aria-hidden="true">TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">32.31
colorchannelmixer<a class="anchor hidden-xs"
href="#colorchannelmixer" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-colorchannelmixer" aria-hidden="true">TOC</h3>

Adjust video input frames by re-mixing color channels.
This filter modifies a color channel by adding the values associated
to the other channels of the same pixels. For example if the value to
modify is red, the output value will be:
<pre class="example-preformatted"><var class="var">red</var>=<var
class="var">red</var>*<var class="var">rr</var> + <var class="var">blue</var>*<var
class="var">rb</var> + <var class="var">green</var>*<var class="var">rg</var>
+ <var class="var">alpha</var>*<var class="var">ra</var> </pre>

The filter accepts the following options: <dl class="table">
<dt><samp class="option">rr</samp></dt> <dt><samp class="option">rg</samp></dt>
<dt><samp class="option">rb</samp></dt> <dt><samp class="option">ra</samp></dt>
<dd> Adjust contribution of input red, green, blue and alpha chan-
nels for output red channel. Default is <code class="code">1</code>
for <var class="var">rr</var>, and <code class="code">0</code>
for <var class="var">rg</var>, <var class="var">rb</var> and <var
class="var">ra</var>. </dd> <dt><samp class="option">gr</samp></dt>
<dt><samp class="option">gg</samp></dt> <dt><samp class="option">gb</samp></dt>

<dt><samp class="option">ga</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output green channel. Default is <code class="code">1</code> for <var class="var">gg</var>, and <code class="code">0</code> for <var class="var">gr</var>, <var class="var">gb</var> and <var class="var">ga</var>. </dd> <dt><samp class="option">br</samp></dt> <dt><samp class="option">bg</samp></dt> <dt><samp class="option">bb</samp></dt> <dt><samp class="option">ba</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output blue channel. Default is <code class="code">1</code> for <var class="var">bb</var>, and <code class="code">0</code> for <var class="var">br</var>, <var class="var">bg</var> and <var class="var">ba</var>. </dd> <dt><samp class="option">ar</samp></dt> <dt><samp class="option">ag</samp></dt> <dt><samp class="option">ab</samp></dt> <dt><samp class="option">aa</samp></dt> <dd> Adjust contribution of input red, green, blue and alpha channels for output alpha channel. Default is <code class="code">1</code> for <var class="var">aa</var>, and <code class="code">0</code> for <var class="var">ar</var>, <var class="var">ag</var> and <var class="var">ab</var>. Allowed ranges for options are <code class="code">[-2.0, 2.0]</code>. </dd> <dt><samp class="option">pc</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl>

</dd> <dt><samp class="option">pa</samp></dt> <dd> Set the preserve color amount when changing colors. Allowed range is from <code class="code">[0.0, 1.0]</code>. Default is <code class="code">0.0</code>, thus disabled. </dd> </dl>

 <h4 class="subsection">32.31.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert source to grayscale: <pre class="example-preformatted">colorchannelmixer=.3:.4:.3:0:.3:.4:.3:0:.3:.4:.3 </pre> Simulate sepia tones: <pre class="example-preformatted">colorchannelmixer=.393:.769:.189:0:.349:.686:.168:0:.272:.534:.131

</pre>

 <h4 class="subsection">32.31.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.32 colordetect# TOC</h3> Analyze the video frames to determine the effective value range and alpha mode. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set of properties to detect. Unavailable properties, such as alpha mode for an input image without an alpha channel, will be ignored automatically. Accepts a combination of the following flags: <dl class="table"> <dt>‘<samp class="samp">color_range</samp>’</dt> <dd> Detect if the source contains luma pixels outside the limited (MPEG) range, which indicates that this is a full range YUV source. </dd> <dt>‘<samp class="samp">alpha_mode</samp>’</dt> <dd> Detect if the source contains color values above the alpha channel, which indicates that the alpha channel is independent (straight), rather than premultiplied. Also detects if the alpha plane is fully opaque or not. </dd> <dt>‘<samp class="samp">all</samp>’</dt> <dd> Enable detection of all of the above properties. This is the default. </dd> </dl>

</dd> <dt><samp class="option">threshold</samp></dt> <dd> Allow pixels to exceed the expected bounds by this fraction of the full scale value range before treating it as evidence of (respectively) full range or straight alpha. Value range is from 0 to 0.05. Defaults to 0.001 (=0.1%). </dd> </dl>

 <h3 class="section">32.33 colorize# TOC</h3> Overlay a solid color on the video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the color hue. Allowed range is from 0 to 360. Default value is 0. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the color saturation. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">lightness</samp></dt> <dd> Set the color lightness. Allowed range is from 0 to 1. Default value is 0.5. </dd> <dt><samp class="option">mix</samp></dt> <dd> Set the mix of source lightness. By default is set to 1.0. Allowed range is from 0.0 to 1.0. </dd> </dl>

 <h4 class="subsection">32.33.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.34 colorkey# TOC</h3> RGB colorspace color keying. This filter operates on 8-bit RGB format frames by setting the alpha component of each pixel which falls within the similarity radius of the key color to 0. The alpha value for pixels outside the similarity radius depends on the value of the blend option. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> Set the color for which alpha will be set to 0 (full transparency). See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default is <code class="code">black</code>. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set the radius from the key color within which other colors also have full transparency. The computed distance is related to the unit fractional distance in 3D space between the RGB values of the key color and the pixel’s color. Range is 0.01 to 1.0. 0.01 matches within a very small radius around the exact key color, while 1.0 matches everything. Default is <code class="code">0.01</code>. </dd> <dt><samp class="option">blend</samp></dt> <dd> Set how the alpha value for pixels that fall outside the similarity radius is computed. 0.0 makes pixels either fully transparent or fully opaque. Higher values result in semi-transparent pixels, with greater transparency the more similar the pixel color is to the key color. Range is 0.0 to 1.0. Default is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">32.34.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every green pixel in the input image transparent: <pre class="example-preformatted">ffmpeg -i input.png -vf colorkey=green out.png </pre>

Overlay a greenscreen-video on top of a static background image. <pre class="example-preformatted">ffmpeg -i background.png -i video.mp4 -filter_complex "[1:v]colorkey=0x3BBD1E:0.3:0.2[ckout];[0:v][ckout]overlay[out]" -map "[out]" output.flv </pre>

 <h4 class="subsection">32.34.2 Com-

mands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.35 colorhold# TOC</h3> Remove all color information for all RGB colors except for certain one. The filter accepts the following options: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color which will not be replaced with neutral gray. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Similarity percentage with the above color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. 0.0 makes pixels fully gray. Higher values result in more preserved color. </dd> </dl>

 <h4 class="subsection">32.35.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.36 colorlevels# TOC</h3>

Adjust video input frames using levels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rimin</samp></dt> <dt><samp class="option">gimin</samp></dt> <dt><samp class="option">bimin</samp></dt> <dt><samp class="option">aimin</samp></dt> <dd> Adjust red, green, blue and alpha input black point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">rimax</samp></dt> <dt><samp class="option">gimax</samp></dt> <dt><samp class="option">bimax</samp></dt> <dt><samp class="option">aimax</samp></dt> <dd> Adjust red, green, blue and alpha input white point. Allowed ranges for options are <code class="code">[-1.0, 1.0]</code>. Defaults are <code class="code">1</code>. Input levels are used to lighten highlights (bright tones), darken shadows (dark tones), change the balance of bright and dark tones. </dd> <dt><samp class="option">romin</samp></dt> <dt><samp class="option">gomin</samp></dt> <dt><samp class="option">bomin</samp></dt>

<dt><samp class="option">aomin</samp></dt> <dd> Adjust red, green, blue and alpha output black point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">0</code>. </dd> <dt><samp class="option">romax</samp></dt> <dt><samp class="option">gomax</samp></dt> <dt><samp class="option">bomax</samp></dt> <dt><samp class="option">aomax</samp></dt> <dd> Adjust red, green, blue and alpha output white point. Allowed ranges for options are <code class="code">[0, 1.0]</code>. Defaults are <code class="code">1</code>. Output levels allows manual selection of a constrained output level range. </dd> <dt><samp class="option">preserve</samp></dt> <dd> Set preserve color mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Disable color preserving, this is default. </dd> <dt>‘<samp class="samp">lum</samp>’</dt> <dd> Preserve luminance. </dd> <dt>‘<samp class="samp">max</samp>’</dt> <dd> Preserve max value of RGB triplet. </dd> <dt>‘<samp class="samp">avg</samp>’</dt> <dd> Preserve average value of RGB triplet. </dd> <dt>‘<samp class="samp">sum</samp>’</dt> <dd> Preserve sum value of RGB triplet. </dd> <dt>‘<samp class="samp">nrm</samp>’</dt> <dd> Preserve normalized value of RGB triplet. </dd> <dt>‘<samp class="samp">pwr</samp>’</dt> <dd> Preserve power value of RGB triplet. </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.36.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make video output darker: <pre class="example-preformatted">colorlevels=rimin=0.058:gimin=0.058:bimin=0.058 </pre>

Increase contrast: <pre class="example-preformatted">colorlevels=rimin=0.039:gimin=0.039:bimin=0.039:rimax=0.96:gimax=0.96:bimax=0.96 </pre>

Make video output lighter: <pre class="example-preformatted">colorlevels=rimax=0.902:gimax=0.902:bimax=0.902 </pre>

Increase brightness: <pre class="example-preformatted">colorlevels=romin=0.5:gomin=0.5:bomin=0.5 </pre>

 <h4 class="subsection">32.36.2 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class=”section”>32.37 colormap# TOC</h3> Apply custom color maps to video stream. This filter needs three input video streams. First stream is video stream that is going to be filtered out. Second and third video stream specify color patches for source color to target color mapping.
 The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>patch_size</samp></dt> <dd> Set the source and target video stream patch size in pixels. </dd> <dt><samp class=”option”>nb_patches</samp></dt> <dd> Set the max number of used patches from source and target video stream. Default value is number of patches available in additional video streams. Max allowed number of patches is <code class=”code”>64</code>. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set the adjustments used for target colors. Can be <code class=”code”>relative</code> or <code class=”code”>absolute</code>. Defaults is <code class=”code”>absolute</code>.
 </dd> <dt><samp class=”option”>kernel</samp></dt> <dd> Set the kernel used to measure color differences between mapped colors.
The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>euclidean</samp>’</dt> <dt>‘<samp class=”samp”>weuclidean</samp>’</dt> </dl>

Default is <code class=”code”>euclidean</code>. </dd> </dl>

 <h3 class=”section”>32.38 colormatrix# TOC</h3>

Convert color matrix. The filter accepts the following options:
 <dl class=”table”> <dt><samp class=”option”>src</samp></dt> <dt><samp class=”option”>dst</samp></dt> <dd> Specify the source and destination color matrix. Both values must be specified.
The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dd> BT.601 </dd> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dd> BT.470 </dd> <dt>‘<samp class=”samp”>bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class=”samp”>bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl> </dd> </dl>

For example to convert from BT.601 to SMPTE-240M, use the command:
 <pre class=”example-preformatted”>colormatrix=bt601:smpte240m

</pre>

 <h3 class="section">32.39 colorspace# TOC</h3>

Convert colorspace, transfer characteristics or color primaries. Input video needs to have an even size. The filter accepts the following options: <dl class="table"> <dd></dd> <dt><samp class="option">all</samp></dt> <dd> Specify all color properties at once. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class="samp">bt601-6-525</samp>’</dt> <dd> BT.601-6 525 </dd> <dt>‘<samp class="samp">bt601-6-625</samp>’</dt> <dd> BT.601-6 625 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">space</samp></dt> <dd> Specify output colorspace. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">fcc</samp>’</dt> <dd> FCC </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">ycgco</samp>’</dt> <dd> YCgCo </dd> <dt>‘<samp class="samp">bt2020ncl</samp>’</dt> <dd> BT.2020 with non-constant luminance </dd> </dl>

</dd> <dt><samp class="option">trc</samp></dt> <dd> Specify output transfer characteristics. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG </dd> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dd> Constant gamma of 2.2 </dd> <dt>‘<samp class="samp">gamma28</samp>’</dt> <dd> Constant gamma of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M, BT.601-6 625 or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">srgb</samp>’</dt> <dd> SRGB </dd> <dt>‘<samp class="samp">iec61966-2-

1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4 </dd> <dt>‘<samp class="samp">xvycc</samp>’</dt> <dd> xvycc </dd> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for 10-bits content </dd> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dd> BT.2020 for 12-bits content </dd> </dl>

</dd> <dt><samp class="option">primaries</samp></dt> <dd> Specify output color primaries. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> film </dd> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl>

</dd> <dt><samp class="option">range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class="samp">mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class="samp">jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> <dt><samp class="option">format</samp></dt> <dd> Specify output color format. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">yuv420p</samp>’</dt> <dd> YUV 4:2:0 planar 8-bits </dd> <dt>‘<samp class="samp">yuv420p10</samp>’</dt> <dd> YUV 4:2:0 planar 10-bits </dd> <dt>‘<samp class="samp">yuv420p12</samp>’</dt> <dd> YUV 4:2:0 planar 12-bits </dd> <dt>‘<samp class="samp">yuv422p</samp>’</dt> <dd> YUV 4:2:2 planar 8-bits </dd> <dt>‘<samp class="samp">yuv422p10</samp>’</dt> <dd> YUV 4:2:2 planar 10-bits </dd> <dt>‘<samp class="samp">yuv422p12</samp>’</dt> <dd> YUV 4:2:2 planar 12-bits </dd> <dt>‘<samp class="samp">yuv444p</samp>’</dt> <dd> YUV 4:4:4 planar 8-bits </dd> <dt>‘<samp class="samp">yuv444p10</samp>’</dt> <dd> YUV 4:4:4 pla-

nar 10-bits </dd> <dt>‘<samp class="samp">yuv444p12</samp>’</dt>
<dd> YUV 4:4:4 planar 12-bits </dd> </dl>

</dd> <dt><samp class="option">fast</samp></dt> <dd> Do a fast conversion, which skips gamma/primary correction. This will take significantly less CPU, but will be mathematically incorrect. To get output compatible with that produced by the colormatrix filter, use fast=1. </dd> <dt><samp class="option">dither</samp></dt> <dd> Specify dithering mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No dithering </dd> <dt>‘<samp class="samp">fsb</samp>’</dt> <dd> Floyd-Steinberg dithering </dd> </dl>

</dd> <dt><samp class="option">wpadapt</samp></dt> <dd> Whitepoint adaptation mode. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">bradford</samp>’</dt> <dd> Bradford whitepoint adaptation </dd> <dt>‘<samp class="samp">vonkries</samp>’</dt> <dd> von Kries white-point adaptation </dd> <dt>‘<samp class="samp">identity</samp>’</dt> <dd> identity whitepoint adaptation (i.e. no whitepoint adaptation) </dd> </dl>

</dd> <dt><samp class="option">clipgamut</samp></dt> <dd> Controls how to clip out-of-gamut colors that arise as a result of colorspace conversion. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No clipping of out of gamut colors. </dd> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> Clips the RGB values to the [0, 1] range when building the gamma transfer LUTs. </dd> </dl>

</dd> <dt><samp class="option">iall</samp></dt> <dd> Override all input properties at once. Same accepted values as all. </dd> <dt><samp class="option">ispace</samp></dt> <dd> Override input colorspace. Same accepted values as space. </dd> <dt><samp class="option">iprimaries</samp></dt> <dd> Override input color primaries. Same accepted values as primaries. </dd> <dt><samp class="option">itrc</samp></dt> <dd> Override input transfer characteristics. Same accepted values as trc. </dd> <dt><samp class="option">irange</samp></dt> <dd> Override input color range. Same accepted values as range. </dd> </dl>

The filter converts the transfer characteristics, color space and color primaries to the specified user values. The output value, if not specified, is set to a default value based on the "all" property. If that property is also not specified, the filter will log an error. The output color range and for-

mat default to the same value as the input color range and format. The input transfer characteristics, color space, color primaries and color range should be set on the input data. If any of these are missing, the filter will log an error and no conversion will take place. For example to convert the input to SMPTE-240M, use the command: <pre class="example-preformatted">colorspace=smpte240m </pre>

 <h3 class="section">32.40 colortemperature# TOC</h3>
Adjust color temperature in video to simulate variations in ambient color temperature. The filter accepts the following options: <dl class="table"> <dt><samp class="option">temperature</samp></dt>
<dd> Set the temperature in Kelvin. Allowed range is from 1000 to 40000. Default value is 6500 K. </dd> <dt><samp class="option">mix</samp></dt>
<dd> Set mixing with filtered output. Allowed range is from 0 to 1. Default value is 1. </dd> <dt><samp class="option">pl</samp></dt>
<dd> Set the amount of preserving lightness. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">32.40.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">32.41 convolution# TOC</h3>

Apply convolution of 3x3, 5x5, 7x7 or horizontal/vertical up to 49 elements. The filter accepts the following options: <dl class="table"> <dt><samp class="option">0m</samp></dt> <dt><samp class="option">1m</samp></dt> <dt><samp class="option">2m</samp></dt>
<dt><samp class="option">3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed integers in <var class="var">square</var> mode, and from 1 to 49 odd number of signed integers in <var class="var">row</var> mode.
</dd> <dt><samp class="option">0rdiv</samp></dt> <dt><samp class="option">1rdiv</samp></dt> <dt><samp class="option">2rdiv</samp></dt>
<dt><samp class="option">3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be 1/sum of all matrix elements. </dd> <dt><samp class="option">0bias</samp></dt>
<dt><samp class="option">1bias</samp></dt> <dt><samp class="option">2bias</samp></dt>
<dt><samp class="option">3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful

for making the overall image brighter or darker. Default is 0.0. </dd> <dt><samp class="option">0mode</samp></dt> <dt><samp class="option">1mode</samp></dt> <dt><samp class="option">2mode</samp></dt> <dt><samp class="option">3mode</samp></dt> <dd> Set matrix mode for each plane. Can be <var class="var">square</var>, <var class="var">row</var> or <var class="var">column</var>. Default is <var class="var">square</var>. </dd> </dl>

 <h4 class="subsection">32.41.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">32.41.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply sharpen: <pre class="example-preformatted">convolution="0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0" </pre>

Apply blur: <pre class="example-preformatted">convolution="1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9" </pre>

Apply edge enhance: <pre class="example-preformatted">convolution="0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128" </pre>

Apply edge detect: <pre class="example-preformatted">convolution="0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128" </pre>

Apply laplacian edge detector which includes diagonals: <pre class="example-preformatted">convolution="1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0" </pre>

Apply emboss: <pre class="example-preformatted">convolution="-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2" </pre>

 <h3 class="section">32.42 convolve# TOC</h3>

Apply 2D convolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options:

<dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> </dl>

The <code class="code">convolve</code> filter also supports the framesync options. <h3 class="section">32.43 copy# TOC</h3>

Copy the input video source unchanged to the output. This is mainly useful for testing purposes. <h3 class="section">32.44 coreimage# TOC</h3> Video filtering on GPU using Apple’s CoreImage API on OSX. Hardware acceleration is based on an OpenGL context. Usually, this means it is processed by video hardware. However, software-based OpenGL implementations exist which means there is no guarantee for hardware processing. It depends on the respective OSX. There are many filters and image generators provided by Apple that come with a large variety of options. The filter has to be referenced by its name along with its options. The coreimage filter accepts the following options: <dl class="table"> <dt><samp class="option">list_filters</samp></dt> <dd> List all available filters and generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class="example-preformatted">list_filters=true </pre>

</dd> <dt><samp class="option">filter</samp></dt> <dd> Specify all filters by their respective name and options. Use <var class="var">list_filters</var> to determine all valid filter names and options. Numerical options are specified by a float value and are automatically clamped to their respective value range. Vector and color options have to be specified by a list of space separated float values. Character escaping has to be done. A special option name <code class="code">default</code> is available to use default options for a filter. It is required to specify either <code class="code">default</code> or at least one of the filter options. All omitted options are used with their default values. The syntax of the filter string is as follows: <pre class="example-preformatted">filter=<NAME>@<OPTION>=<VALUE>[@<OPTION>=<VALUE&
</pre>

</dd> <dt><samp class="option">output_rect</samp></dt> <dd> Specify

a rectangle where the output of the filter chain is copied into the input image. It is given by a list of space separated float values: <pre class="example-preformatted">output_rect=x\ y\ width\ height </pre> If not given, the output rectangle equals the dimensions of the input image. The output rectangle is automatically cropped at the borders of the input image. Negative values are valid for each component. <pre class="example-preformatted">output_rect=25\ 25\ 100\ 100 </pre> </dd> </dl>

Several filters can be chained for successive processing without GPU-HOST transfers allowing for fast processing of complex filter chains. Currently, only filters with zero (generators) or exactly one (filters) input image and one output image are supported. Also, transition filters are not yet usable as intended. Some filters generate output images with additional padding depending on the respective filter kernel. The padding is automatically removed to ensure the filter output has the same size as the input image. For image generators, the size of the output image is determined by the previous output image of the filter chain or the input image of the whole filterchain, respectively. The generators do not use the pixel information of this image to generate their output. However, the generated output is blended onto this image, resulting in partial or complete coverage of the output image. The coreimagesrc video source can be used for generating input images which are directly fed into the filter chain. By using it, providing input images by another video source or an input video is not required. <h4 class="subsection">32.44.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> List all filters available: <pre class="example-preformatted">coreimage=list_filters=true </pre>

Use the CIBoxBlur filter with default options to blur an image: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default </pre>

Use a filter chain with CISepiaTone at default values and CIVignetteEffect with its center at 100x100 and a radius of 50 pixels: <pre class="example-preformatted">coreimage=filter=CIBoxBlur@default#CIVignetteEf
100@inputRadius=50 </pre>

Use nullsrc and CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class="example-preformatted">ffmpeg -f lavfi -i nullsrc=s=100x100,coreimage=filter=CIQRCodeGenerator@inputMessage=ht
-frames:v 1 QRCode.png </pre>

 <h3 class="section">32.45 corr# TOC</h3>

Obtain the correlation between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max correlation is printed through the logging system. The filter stores the calculated correlation of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi corr -f null - </pre>

 <h3 class="section">32.46 cover_rect# TOC</h3>

Cover a rectangular object It accepts the following options: <dl class="table"> <dt><samp class="option">cover</samp></dt> <dd> Filepath of the optional cover image, needs to be in yuv420. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set covering mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> cover it by the supplied image </dd> <dt>‘<samp class="samp">blur</samp>’</dt> <dd> cover it by interpolating the surrounding pixels </dd> </dl>

Default value is <var class="var">blur</var>. </dd> </dl>

 <h4 class="subsection">32.46.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

 <h3 class="section">32.47 crop# TOC</h3>

Crop the input video to given dimensions. It accepts the following parameters: <dl class="table"> <dt><samp class="option">w, out_w</samp></dt> <dd> The width of the output video. It defaults to <code class="code">iw</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">w</samp>’ or ‘<samp class="samp">out_w</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">h, out_h</samp></dt> <dd> The height of the output video. It defaults to <code class="code">ih</code>. This expression is evaluated during the filter configuration, when the ‘<samp class="samp">h</samp>’ or ‘<samp class="samp">out_h</samp>’ command is sent, or when the input frame size changes. </dd> <dt><samp class="option">x</samp></dt> <dd> The horizontal position, in the input video, of the left edge of the output video. It defaults to <code class="code">(in_w-out_w)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">y</samp></dt> <dd> The vertical position, in the input video, of the top edge of the output video. It defaults to <code class="code">(in_h-out_h)/2</code>. This expression is evaluated per-frame. </dd> <dt><samp class="option">keep_aspect</samp></dt> <dd> If set to 1 will force the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. It defaults to 0. </dd> <dt><samp class="option">exact</samp></dt> <dd> Enable exact cropping. If enabled, subsampled videos will be cropped at exact width/height/x/y as specified and will not be rounded to nearest smaller value. It defaults to 0. </dd> </dl>

The <var class="var">out_w</var>, <var class="var">out_h</var>, <var class="var">x</var>, <var class="var">y</var> parameters are expressions containing the following constants: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The computed values for <var class="var">x</var> and <var class="var">y</var>. They are evaluated for each new frame. </dd> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output (cropped) width and height. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample

aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">iw</var> / <var class="var">ih</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class="option">pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class="option">t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

The expression for <var class="var">out_w</var> may depend on the value of <var class="var">out_h</var>, and the expression for <var class="var">out_h</var> may depend on <var class="var">out_w</var>, but they cannot depend on <var class="var">x</var> and <var class="var">y</var>, as <var class="var">x</var> and <var class="var">y</var> are evaluated after <var class="var">out_w</var> and <var class="var">out_h</var>. The <var class="var">x</var> and <var class="var">y</var> parameters specify the expressions for the position of the top-left corner of the output (non-cropped) area. They are evaluated for each frame. If the evaluated value is not valid, it is approximated to the nearest valid value. The expression for <var class="var">x</var> may depend on <var class="var">y</var>, and the expression for <var class="var">y</var> may depend on <var class="var">x</var>. <h4 class="subsection">32.47.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Crop area with size 100x100 at position (12,34). <pre class="example-preformatted">crop=100:100:12:34 </pre>

Using named options, the example above becomes: <pre class="example-preformatted">crop=w=100:h=100:x=12:y=34 </pre>

Crop the central input area with size 100x100: <pre class="example-preformatted">crop=100:100 </pre>

Crop the central input area with size 2/3 of the input video: <pre class="example-preformatted">crop=2/3*in_w:2/3*in_h </pre>

Crop the input video central square: <pre

class=”example-preformatted”>crop=out_w=in_h crop=in_h </pre>

Delimit the rectangle with the top-left corner placed at position 100:100 and the right-bottom corner corresponding to the right-bottom corner of the input image. <pre class=”example-preformatted”>crop=in_w-100:in_h-100:100:100 </pre>

Crop 10 pixels from the left and right borders, and 20 pixels from the top and bottom borders <pre class=”example-preformatted”>crop=in_w-2*10:in_h-2*20 </pre>

Keep only the bottom right quarter of the input image: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:in_w/2:in_h/2 </pre>

Crop height for getting Greek harmony: <pre class=”example-preformatted”>crop=in_w:1/PHI*in_w </pre>

Apply trembling effect: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(n/10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(n/7) </pre>

Apply erratic camera effect depending on timestamp: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:(in_w-out_w)/2+((in_w-out_w)/2)*sin(t*10):(in_h-out_h)/2 +((in_h-out_h)/2)*sin(t*13) </pre>

Set x depending on the value of y: <pre class=”example-preformatted”>crop=in_w/2:in_h/2:y:10+10*sin(n/10) </pre>

 <h4 class=”subsection”>32.47.2 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>w, out_w</samp></dt> <dt><samp class=”option”>h, out_h</samp></dt> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set width/height of the output video and the horizontal/vertical position in the input video. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>32.48 cropdetect# TOC</h3>

Auto-detect the crop size. It calculates the necessary

cropping parameters and prints the recommended parameters via the logging system. The detected dimensions correspond to the non-black or video area of the input video according to <var class=”var”>mode</var>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Depending on <var class=”var”>mode</var> crop detection is based on either the mere black value of surrounding pixels or a combination of motion vectors and edge pixels. <dl class=”table”> <dt>‘<samp class=”samp”>black</samp>’</dt> <dd> Detect black pixels surrounding the playing video. For fine control use option <var class=”var”>limit</var>. </dd> <dt>‘<samp class=”samp”>mvedges</samp>’</dt> <dd> Detect the playing video by the motion vectors inside the video and scanning for edge pixels typically forming the border of a playing video. </dd> </dl>

</dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set higher black value threshold, which can be optionally specified from nothing (0) to everything (255 for 8-bit based formats). An intensity value greater to the set value is considered non-black. It defaults to 24. You can also specify a value between 0.0 and 1.0 which will be scaled depending on the bitdepth of the pixel format. </dd> <dt><samp class=”option”>round</samp></dt> <dd> The value which the width/height should be divisible by. It defaults to 16. The offset is automatically adjusted to center the video. Use 2 to get only even dimensions (needed for 4:2:2 video). 16 is best when encoding to most video codecs. </dd> <dt><samp class=”option”>skip</samp></dt> <dd> Set the number of initial frames for which evaluation is skipped. Default is 2. Range is 0 to INT_MAX. </dd> <dt><samp class=”option”>reset_count, reset</samp></dt> <dd> Set the counter that determines after how many frames cropdetect will reset the previously detected largest video area and start over to detect the current optimal crop area. Default value is 0. This can be useful when channel logos distort the video area. 0 indicates ’never reset’, and returns the largest area encountered during playback. </dd> <dt><samp class=”option”>mv_threshold</samp></dt> <dd> Set motion in pixel units as threshold for motion detection. It defaults to 8. </dd> <dt><samp class=”option”>low</samp></dt> <dt><samp class=”option”>high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class=”var”>low</var> and <var class=”var”>high</var> threshold values must be chosen in the range [0,1], and <var class=”var”>low</var> should be lesser or equal to <var class=”var”>high</var>. Default value for <var class=”var”>low</var> is <code class=”code”>5/255</code>, and default value for <var class=”var”>high</var> is <code class=”code”>15/255</code>.

</dd> </dl>

 <h4 class="subsection">32.48.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Find video area surrounded by black borders: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf cropdetect,metadata=mode=print -f null - </pre>

Find an embedded video area, generate motion vectors beforehand: <pre class="example-preformatted">ffmpeg -i file.mp4 -vf mestimate,cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

Find an embedded video area, use motion vectors from decoder: <pre class="example-preformatted">ffmpeg -flags2 +export_mvs -i file.mp4 -vf cropdetect=mode=mvedges,metadata=mode=print -f null - </pre>

 <h4 class="subsection">32.48.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">limit</samp></dt> <dd> The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">32.49 cue# TOC</h3>

Delay video filtering until a given wallclock timestamp. The filter first passes on <samp class="option">preroll</samp> amount of frames, then it buffers at most <samp class="option">buffer</samp> amount of frames and waits for the cue. After reaching the cue it forwards the buffered frames and also any subsequent frames coming in its input. The filter can be used synchronize the output of multiple ffmpeg processes for realtime output devices like decklink. By putting the delay in the filtering chain and pre-buffering frames the process can pass on data to output almost immediately after the target wallclock timestamp is reached. Perfect frame accuracy cannot be guaranteed, but the result is good enough for some use cases. <dl class="table"> <dt><samp class="option">cue</samp></dt> <dd> The cue timestamp expressed in a UNIX timestamp in microseconds. Default is 0. </dd> <dt><samp class="option">preroll</samp></dt> <dd> The duration of content

to pass on as preroll expressed in seconds. Default is 0. </dd> <dt><samp class="option">buffer</samp></dt> <dd> The maximum duration of content to buffer before waiting for the cue expressed in seconds. Default is 0. </dd> </dl>

 <h3 class="section">32.50 curves# TOC</h3>

Apply color adjustments using curves. This filter is similar to the Adobe Photoshop and GIMP curves tools. Each component (red, green and blue) has its values defined by <var class="var">N</var> key points tied from each other using a smooth curve. The x-axis represents the pixel values from the input frame, and the y-axis the new pixel values to be set for the output frame. By default, a component curve is defined by the two points <var class="var">(0;0)</var> and <var class="var">(1;1)</var>. This creates a straight line where each original pixel value is "adjusted" to its own value, which means no change to the image. The filter allows you to redefine these two points and add some more. A new curve will be defined to pass smoothly through all these new coordinates. The new defined points need to be strictly increasing over the x-axis, and their <var class="var">x</var> and <var class="var">y</var> values must be in the <var class="var">[0;1]</var> interval. The curve is formed by using a natural or monotonic cubic spline interpolation, depending on the <var class="var">interp</var> option (default: <code class="code">natural</code>). The <code class="code">natural</code> spline produces a smoother curve in general while the monotonic (<code class="code">pchip</code>) spline guarantees the transitions between the specified points to be monotonic. If the computed curves happened to go outside the vector spaces, the values will be clipped accordingly. The filter accepts the following options: <dl class="table"> <dt><samp class="option">preset</samp></dt> <dd> Select one of the available color presets. This option can be used in addition to the <samp class="option">r</samp>, <samp class="option">g</samp>, <samp class="option">b</samp> parameters; in this case, the later options takes priority on the preset values. Available presets are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">color_negative</samp>&rs
<dt>‘<samp class="samp">cross_process</samp>’</dt>
<dt>‘<samp class="samp">darker</samp>’</dt> <dt>‘<samp class="samp">increase_contrast</samp>’</dt> <dt>‘<samp class="samp">lighter</samp>’</dt> <dt>‘<samp class="samp">linear_contrast</samp>&
<dt>‘<samp class="samp">medium_contrast</samp>’</dt>
<dt>‘<samp class="samp">negative</samp>’</dt> <dt>‘<samp class="samp">strong_contrast</samp>’</dt> <dt>‘<samp class="samp">vintage</samp>’</dt> </dl> Default is <code

class=”code”>none</code>. </dd> <dt><samp class=”option”>master, m</samp></dt> <dd> Set the master key points. These points will define a second pass mapping. It is sometimes called a "luminance" or "value" mapping. It can be used with <samp class=”option”>r</samp>, <samp class=”option”>g</samp>, <samp class=”option”>b</samp> or <samp class=”option”>all</samp> since it acts like a post-processing LUT. </dd> <dt><samp class=”option”>red, r</samp></dt> <dd> Set the key points for the red component. </dd> <dt><samp class=”option”>green, g</samp></dt> <dd> Set the key points for the green component. </dd> <dt><samp class=”option”>blue, b</samp></dt> <dd> Set the key points for the blue component. </dd> <dt><samp class=”option”>all</samp></dt> <dd> Set the key points for all components (not including master). Can be used in addition to the other key points component options. In this case, the unset component(s) will fallback on this <samp class=”option”>all</samp> setting. </dd> <dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop curves file (<code class=”code”>.acv</code>) to import the settings from. </dd> <dt><samp class=”option”>plot</samp></dt> <dd> Save Gnuplot script of the curves in specified file. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the kind of interpolation. Available algorithms are: <dl class=”table”> <dt>‘<samp class=”samp”>natural</samp>’</dt> <dd> Natural cubic spline using a piece-wise cubic polynomial that is twice continuously differentiable. </dd> <dt>‘<samp class=”samp”>pchip</samp>’</dt> <dd> Monotonic cubic spline using a piecewise cubic Hermite interpolating polynomial (PCHIP). </dd> </dl>

</dd> </dl>

To avoid some filtergraph syntax conflicts, each key points list need to be defined using the following syntax: <code class=”code”>x0/y0 x1/y1 x2/y2 ...</code>. <h4 class=”subsection”>32.50.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>32.50.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Increase slightly the middle level of blue: <pre class=”example-preformatted”>curves=blue='0/0 0.5/0.58 1/1' </pre>

Vintage effect: <pre class=”example-

preformatted”>curves=r='0/0.11 .42/.51 1/0.95':g='0/0 0.50/0.48 1/1':b='0/0.22 .49/.44 1/0.8' </pre> Here we obtain the following coordinates for each components: <dl class=”table”> <dt><var class=”var”>red</var></dt> <dd> <code class=”code”>(0;0.11) (0.42;0.51) (1;0.95)</code> </dd> <dt><var class=”var”>green</var></dt> <dd> <code class=”code”>(0;0) (0.50;0.48) (1;1)</code> </dd> <dt><var class=”var”>blue</var></dt> <dd> <code class=”code”>(0;0.22) (0.49;0.44) (1;0.80)</code> </dd> </dl>

The previous example can also be achieved with the associated built-in preset: <pre class=”example-preformatted”>curves=preset=vintage </pre>

Or simply: <pre class=”example-preformatted”>curves=vintage </pre>

Use a Photoshop preset and redefine the points of the green component: <pre class=”example-preformatted”>curves=psfile='MyCurvesPresets/purple.a 0.45/0.53 1/1' </pre>

Check out the curves of the <code class=”code”>cross_process</code> profile using <code class=”command”>ffmpeg</code> and <code class=”command”>gnuplot</code>: <pre class=”example-preformatted”>ffmpeg -f lavfi -i color -vf curves=cross_process:plot=/tmp/curves.plt -frames:v 1 -f null - gnuplot -p /tmp/curves.plt </pre>

 <h3 class=”section”>32.51 datascope# TOC</h3>

Video data analysis filter. This filter shows hexadecimal pixel values of part of video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set output video size. </dd> <dt><samp class=”option”>x</samp></dt> <dd> Set x offset from where to pick pixels. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set y offset from where to pick pixels. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set scope mode, can be one of the following: <dl class=”table”> <dt>‘<samp class=”samp”>mono</samp>’</dt> <dd> Draw hexadecimal pixel values with white color on black background. </dd> <dt>‘<samp class=”samp”>color</samp>’</dt> <dd> Draw hexadecimal pixel values with input video pixel color on black background. </dd> <dt>‘<samp class=”samp”>color2</samp>’</dt> <dd> Draw hexadecimal pixel values on color background picked from input video, the text color is picked in such way so its always visible. </dd> </dl>

</dd> <dt><samp class="option">axis</samp></dt> <dd> Draw rows and columns numbers on left and top of video. </dd> <dt><samp class="option">opacity</samp></dt> <dd> Set background opacity. </dd> <dt><samp class="option">format</samp></dt> <dd> Set display number format. Can be <code class="code">hex</code>, or <code class="code">dec</code>. Default is <code class="code">hex</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set pixel components to display. By default all pixel components are displayed. </dd> </dl>

 <h4 class="subsection">32.51.1 Commands# TOC</h4>

This filter supports same commands as options excluding <code class="code">size</code> option. <h3 class="section">32.52 dblur# TOC</h3> Apply Directional blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">angle</samp></dt> <dd> Set angle of directional blur. Default is <code class="code">45</code>. </dd> <dt><samp class="option">radius</samp></dt> <dd> Set radius of directional blur. Default is <code class="code">5</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> </dl>

 <h4 class="subsection">32.52.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.53 dctdnoiz# TOC</h3>

Denoise frames using 2D DCT (frequency domain filtering). This filter is not designed for real time. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma, s</samp></dt> <dd> Set the noise sigma constant. This <var class="var">sigma</var> defines a hard threshold of <code class="code">3 * sigma</code>; every DCT coefficient (absolute value) below this threshold with be dropped. If you need a more advanced filtering, see <samp class="option">expr</samp>.

Default is <code class="code">0</code>. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number overlapping pixels for each block. Since the filter can be slow, you may want to reduce this value, at the cost of a less effective filter and the risk of various artefacts. If the overlapping value doesn’t permit processing the whole input width or height, a warning will be displayed and according borders won’t be denoised. Default value is <var class="var">blocksize</var>-1, which is the best possible setting. </dd> <dt><samp class="option">expr, e</samp></dt> <dd> Set the coefficient factor expression. For each coefficient of a DCT block, this expression will be evaluated as a multiplier value for the coefficient. If this is option is set, the <samp class="option">sigma</samp> option will be ignored. The absolute value of the coefficient can be accessed through the <var class="var">c</var> variable. </dd> <dt><samp class="option">n</samp></dt> <dd> Set the <var class="var">blocksize</var> using the number of bits. <code class="code">1<<<var class="var">n</var></code> defines the <var class="var">blocksize</var>, which is the width and height of the processed blocks. The default value is <var class="var">3</var> (8x8) and can be raised to <var class="var">4</var> for a <var class="var">blocksize</var> of 16x16. Note that changing this setting has huge consequences on the speed processing. Also, a larger block size does not necessarily means a better de-noising. </dd> </dl>

 <h4 class="subsection">32.53.1 Examples# TOC</h4>

Apply a denoise with a <samp class="option">sigma</samp> of <code class="code">4.5</code>: <pre class="example-preformatted">dctdnoiz=4.5 </pre>

The same operation can be achieved using the expression system: <pre class="example-preformatted">dctdnoiz=e='gte(c, 4.5*3)' </pre>

Violent denoise using a block size of <code class="code">16x16</code>: <pre class="example-preformatted">dctdnoiz=15:n=4 </pre>

 <h3 class="section">32.54 deband# TOC</h3>

Remove banding artifacts from input video. It works by replacing banded pixels with average value of referenced pixels. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>1thr</samp></dt> <dt><samp class=”option”>2thr</samp></dt>
<dt><samp class=”option”>3thr</samp></dt> <dt><samp class=”option”>4thr</samp></dt>
<dd> Set banding detection threshold for each plane. Default is 0.02. Valid range is 0.00003 to 0.5. If difference between current pixel and reference pixel is less than threshold, it will be considered as banded. </dd>
<dt><samp class=”option”>range, r</samp></dt> <dd> Banding detection range in pixels. Default is 16. If positive, random number in range 0 to set value will be used. If negative, exact absolute value will be used. The range defines square of four pixels around current pixel. </dd>
<dt><samp class=”option”>direction, d</samp></dt> <dd> Set direction in radians from which four pixel will be compared. If positive, random direction from 0 to set direction will be picked. If negative, exact of absolute value will be picked. For example direction 0, -PI or -2*PI radians will pick only pixels on same row and -PI/2 will pick only pixels on same column. </dd> <dt><samp class=”option”>blur, b</samp></dt> <dd> If enabled, current pixel is compared with average value of all four surrounding pixels. The default is enabled. If disabled current pixel is compared with all four surrounding pixels. The pixel is considered banded if only all four differences with surrounding pixels are less than threshold. </dd>
<dt><samp class=”option”>coupling, c</samp></dt> <dd> If enabled, current pixel is changed if and only if all pixel components are banded, e.g. banding detection threshold is triggered for all color components. The default is disabled. </dd> </dl>

 <h4 class=”subsection”>32.54.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.55 deblock# TOC</h3>

Remove blocking artifacts from input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set filter type, can be <var class=”var”>weak</var> or <var class=”var”>strong</var>. Default is <var class=”var”>strong</var>. This controls what kind of deblocking is applied. </dd> <dt><samp class=”option”>block</samp></dt>
<dd> Set size of block, allowed range is from 4 to 512. Default is <var class=”var”>8</var>. </dd> <dt><samp class=”option”>alpha</samp></dt>
<dt><samp class=”option”>beta</samp></dt> <dt><samp class=”option”>gamma</samp></dt>
<dt><samp class=”option”>delta</samp></dt> <dd> Set blocking detection thresholds. Allowed range is 0 to 1. Defaults are: <var class=”var”>0.098</var> for <var class=”var”>alpha</var> and <var

class=”var”>0.05</var> for the rest. Using higher threshold gives more deblocking strength. Setting <var class=”var”>alpha</var> controls threshold detection at exact edge of block. Remaining options controls threshold detection near the edge. Each one for below/above or left/right. Setting any of those to <var class=”var”>0</var> disables deblocking. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is to filter all available planes. </dd> </dl>

 <h4 class=”subsection”>32.55.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Deblock using weak filter and block size of 4 pixels. <pre class=”example-preformatted”>deblock=filter=weak:block=4 </pre>

Deblock using strong filter, block size of 4 pixels and custom thresholds for deblocking more edges. <pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta=0.05
</pre>

Similar as above, but filter only first plane.
<pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.07:gamma=0.06:delta
</pre>

Similar as above, but filter only second and third plane. <pre class=”example-preformatted”>deblock=filter=strong:block=4:alpha=0.12:beta=0.0
</pre>

 <h4 class=”subsection”>32.55.2 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class=”section”>32.56 decimate# TOC</h3>

Drop duplicated frames at regular intervals. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cycle</samp></dt> <dd> Set the number of frames from which one will be dropped. Setting this to <var class=”var”>N</var> means one frame in every batch of <var class=”var”>N</var> frames will be dropped. Default is <code class=”code”>5</code>. </dd> <dt><samp class=”option”>dupthresh</samp></dt> <dd> Set the threshold for duplicate detection. If the difference metric for a

frame is less than or equal to this value, then it is declared as duplicate. Default is <code class="code">1.1</code> </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set scene change threshold. Default is <code class="code">15</code>. </dd> <dt><samp class="option">blockx</samp></dt> <dt><samp class="option">blocky</samp></dt> <dd> Set the size of the x and y-axis blocks used during metric calculations. Larger blocks give better noise suppression, but also give worse detection of small movements. Must be a power of two. Default is <code class="code">32</code>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark main input as a pre-processed input and activate clean source input stream. This allows the input to be pre-processed with various filters to help the metrics calculation while keeping the frame selection lossless. When set to <code class="code">1</code>, the first stream is for the pre-processed input, and the second stream is the clean source from where the kept frames are chosen. Default is <code class="code">0</code>. </dd> <dt><samp class="option">chroma</samp></dt> <dd> Set whether or not chroma is considered in the metric calculations. Default is <code class="code">1</code>. </dd> <dt><samp class="option">mixed</samp></dt> <dd> Set whether or not the input only partially contains content to be decimated. Default is <code class="code">false</code>. If enabled video output stream will be in variable frame rate. </dd> </dl>

 <h3 class="section">32.57 deconvolve# TOC</h3>

Apply 2D deconvolution of video stream in frequency domain using second stream as impulse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. </dd> <dt><samp class="option">impulse</samp></dt> <dd> Set which impulse video frames will be processed, can be <var class="var">first</var> or <var class="var">all</var>. Default is <var class="var">all</var>. </dd> <dt><samp class="option">noise</samp></dt> <dd> Set noise when doing divisions. Default is <var class="var">0.0000001</var>. Useful when width and height are not same and not power of 2 or if stream prior to convolving had noise. </dd> </dl>

The <code class="code">deconvolve</code> filter also supports the framesync options. <h3 class="section">32.58 dedot#
TOC</h3>

Reduce cross-luminance (dot-crawl) and cross-color (rainbows) from

video. It accepts the following options: <dl class="table"> <dt><samp class="option">m</samp></dt> <dd> Set mode of operation. Can be combination of <var class="var">dotcrawl</var> for cross-luminance reduction and/or <var class="var">rainbows</var> for cross-color reduction. </dd> <dt><samp class="option">lt</samp></dt> <dd> Set spatial luma threshold. Lower values increases reduction of cross-luminance. </dd> <dt><samp class="option">tl</samp></dt> <dd> Set tolerance for temporal luma. Higher values increases reduction of cross-luminance. </dd> <dt><samp class="option">tc</samp></dt> <dd> Set tolerance for chroma temporal variation. Higher values increases reduction of cross-color. </dd> <dt><samp class="option">ct</samp></dt> <dd> Set temporal chroma threshold. Lower values increases reduction of cross-color. </dd> </dl>

 <h3 class="section">32.59 deflate# TOC</h3>

Apply deflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values lower than the pixel. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class="subsection">32.59.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.60 deflicker# TOC</h3>

Remove temporal frame luminance variations. It accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Set moving-average filter size in frames. Default is 5. Allowed range is 2 - 129. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set averaging mode to smooth temporal luminance variations. Available values are: <dl class="table"> <dt>‘<samp class="samp">am</samp>’</dt> <dd> Arithmetic mean </dd> <dt>‘<samp class="samp">gm</samp>’</dt> <dd> Geometric mean </dd> <dt>‘<samp class="samp">hm</samp>’</dt>

<dd> Harmonic mean </dd> <dt>‘<samp class="samp">qm</samp>’</dt>
<dd> Quadratic mean </dd> <dt>‘<samp class="samp">cm</samp>’</dt>
<dd> Cubic mean </dd> <dt>‘<samp class="samp">pm</samp>’</dt>
<dd> Power mean </dd> <dt>‘<samp class="samp">median</samp>’</dt>
<dd> Median </dd> </dl>

</dd> <dt><samp class="option">bypass</samp></dt> <dd> Do not actually modify frame. Useful when one only wants metadata. </dd> </dl>

 <h3 class="section">32.61 deinterlace_d3d12# TOC</h3>

Deinterlace the input video using D3D12 hardware-accelerated video processing. This filter uses the DirectX 12 Video Processor to perform deinterlacing on GPU, supporting both simple bob and driver-defined advanced (motion-adaptive) algorithms. It requires the input to be in <code class="code">d3d12</code> hardware pixel format. The filter automatically queries the hardware for the required number of temporal reference frames (if needed) and manages a frame queue to supply them. It accepts the following options: <dl class="table"> <dt><samp class="option">method</samp></dt> <dd> The deinterlacing method. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">default</samp>’</dt> <dd> Select the best deinterlacing method available on the hardware. If the driver supports <code class="code">custom</code>, that method is used; otherwise falls back to <code class="code">bob</code>. This is the default. </dd> <dt>‘<samp class="samp">bob</samp>’</dt> <dd> Bob deinterlacing. Each field is independently scaled to full frame height. Simple and fast, but may produce visible bob artifacts on moving content. </dd> <dt>‘<samp class="samp">custom</samp>’</dt> <dd> Driver-defined advanced deinterlacing. The exact algorithm is hardware-specific and typically employs motion-adaptive techniques with temporal reference frames for higher quality. </dd> </dl>

The default value is <code class="code">default</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Specify the interlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Send one frame for each frame. The output frame rate equals the input frame rate. This is the default. </dd> <dt>‘<samp class="samp">field</samp>’</dt> <dd> Send one frame for each field. The output frame rate is double the input frame rate. </dd> </dl>

The default value is <code class="code">frame</code>. </dd>

<dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. It accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames. This is the default. </dd> <dt>‘<samp class=”samp”>interlaced</samp>’</dt> <dd> Only deinterlace frames marked as interlaced; progressive frames are passed through unchanged. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> </dl>

 <h4 class=”subsection”>32.61.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Deinterlace an interlaced video using software decoding and the best available hardware deinterlacing method: <pre class=”example-preformatted”>ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=method=default,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

Deinterlace at field rate (double the frame rate) using bob method with hardware decoding (full d3d12 pipeline): <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.ts \ -vf "deinterlace_d3d12=method=bob:mode=field" \ -c:v h264_d3d12va output.mp4 </pre>

Deinterlace only interlaced frames, passing progressive frames through: <pre class=”example-preformatted”>ffmpeg -init_hw_device d3d12va=d3d12 -filter_hw_device d3d12 -i input.ts \ -vf "format=nv12,hwupload,deinterlace_d3d12=deint=interlaced,hwdownload,format=nv12" \ -c:v libx264 -crf 18 output.mp4 </pre>

 <h3 class=”section”>32.62 dejudder# TOC</h3>

Remove judder produced by partially interlaced telecined content. Judder can be introduced, for instance, by pullup filter. If the original source was partially telecined content then the output of <code class=”code”>pullup,dejudder</code> will have a variable frame rate. May change the recorded frame rate of the container. Aside from that change, this filter will not affect constant frame rate video. The option available in this filter is: <dl class=”table”> <dt><samp class=”option”>cycle</samp></dt> <dd> Specify the

length of the window over which the judder repeats. Accepts any integer greater than 1. Useful values are: <dl class="table"> <dt>‘<samp class="samp">4</samp>’</dt> <dd> If the original was telecined from 24 to 30 fps (Film to NTSC). </dd> <dt>‘<samp class="samp">5</samp>’</dt> <dd> If the original was telecined from 25 to 30 fps (PAL to NTSC). </dd> <dt>‘<samp class="samp">20</samp>’</dt> <dd> If a mixture of the two. </dd> </dl>

The default is ‘<samp class="samp">4</samp>’. </dd> </dl>

 <h3 class="section">32.63 delogo# TOC</h3>

Suppress a TV station logo by a simple interpolation of the surrounding pixels. Just set a rectangle covering the logo and watch it disappear (and sometimes something even uglier appear - your mileage may vary). It accepts the following parameters: <dl class="table"> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the top left corner coordinates of the logo. They must be specified. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Specify the width and height of the logo to clear. They must be specified. </dd> <dt><samp class="option">show</samp></dt> <dd> When set to 1, a green rectangle is drawn on the screen to simplify finding the right <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var>, and <var class="var">h</var> parameters. The default value is 0. The rectangle is drawn on the outermost pixels which will be (partly) replaced with interpolated values. The values of the next pixels immediately outside this rectangle in each direction will be used to compute the interpolated pixel values inside the rectangle. </dd> </dl>

 <h4 class="subsection">32.63.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set a rectangle covering the area with top left corner coordinates 0,0 and size 100x77: <pre class="example-preformatted">delogo=x=0:y=0:w=100:h=77 </pre>

 <h3 class="section">32.64 derain# <a class="anchor

hidden-xs”href=”#toc-derain” aria-hidden=”true”>TOC</h3>

Remove the rain in the input image/video by applying the derain methods based on convolutional neural networks. Supported models: <ul class=”itemize mark-bullet”> Recurrent Squeeze-and-Excitation Context Aggregation Net (RESCAN). See http://openaccess.thecvf.com/content_ECCV_2018/pap and-Excitation_Context_ECCV_2018_paper.pdf.

Training as well as model generation scripts are provided in the repository at <a class=”url” href=”https://github.com/XueweiMeng/derain_filter.git”>https://github.com/XueweiMeng/d The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>filter_type</samp></dt> <dd> Specify which filter to use. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>derain</samp>’</dt> <dd> Derain filter. To conduct derain filter, you need to use a derain model. </dd> <dt>‘<samp class=”samp”>dehaze</samp>’</dt> <dd> Dehaze filter. To conduct dehaze filter, you need to use a dehaze model. </dd> </dl> Default value is ‘<samp class=”samp”>derain</samp>’. </dd> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow can load files for only its format. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class=”section”>32.65 deshake# TOC</h3>

Attempt to fix small changes in horizontal and/or vertical shift. This filter helps remove camera shake from hand-holding a camera, bumping a tripod, moving on a vehicle, etc. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Specify a rectan-

gular area where to limit the search for motion vectors. If desired the search for motion vectors can be limited to a rectangular area of the frame defined by its top left corner, width and height. These parameters have the same meaning as the drawbox filter which can be used to visualise the position of the bounding box. This is useful when simultaneous movement of subjects within the frame might be confused for camera motion by the motion vector search. If any or all of <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> are set to -1 then the full frame is used. This allows later options to be set without specifying the bounding box for the motion vector search. Default - search the whole frame. </dd> <dt><samp class="option">rx</samp></dt> <dt><samp class="option">ry</samp></dt> <dd> Specify the maximum extent of movement in x and y directions in the range 0-64 pixels. Default 16. </dd> <dt><samp class="option">edge</samp></dt> <dd> Specify how to generate pixels to fill blanks at the edge of the frame. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank, 0</samp>’</dt> <dd> Fill zeroes at blank locations </dd> <dt>‘<samp class="samp">original, 1</samp>’</dt> <dd> Original image at blank locations </dd> <dt>‘<samp class="samp">clamp, 2</samp>’</dt> <dd> Extruded edge value at blank locations </dd> <dt>‘<samp class="samp">mirror, 3</samp>’</dt> <dd> Mirrored edge at blank locations </dd> </dl> Default value is ‘<samp class="samp">mirror</samp>’. </dd> <dt><samp class="option">blocksize</samp></dt> <dd> Specify the blocksize to use for motion search. Range 4-128 pixels, default 8. </dd> <dt><samp class="option">contrast</samp></dt> <dd> Specify the contrast threshold for blocks. Only blocks with more than the specified contrast (difference between darkest and lightest pixels) will be considered. Range 1-255, default 125. </dd> <dt><samp class="option">search</samp></dt> <dd> Specify the search strategy. Available values are: <dl class="table"> <dt>‘<samp class="samp">exhaustive, 0</samp>’</dt> <dd> Set exhaustive search </dd> <dt>‘<samp class="samp">less, 1</samp>’</dt> <dd> Set less exhaustive search. </dd> </dl> Default value is ‘<samp class="samp">exhaustive</samp>’. </dd> <dt><samp class="option">filename</samp></dt> <dd> If set then a detailed log of the motion search is written to the specified file. </dd> </dl>

 <h3 class="section">32.66 despill# TOC</h3>

Remove unwanted contamination of foreground colors, caused by reflected color of greenscreen or bluescreen. This filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>type</samp></dt> <dd> Set what type of despill to use. </dd> <dt><samp class=”option”>mix</samp></dt> <dd> Set how spillmap will be generated. </dd> <dt><samp class=”option”>expand</samp></dt> <dd> Set how much to get rid of still remaining spill. </dd> <dt><samp class=”option”>red</samp></dt> <dd> Controls amount of red in spill area. </dd> <dt><samp class=”option”>green</samp></dt> <dd> Controls amount of green in spill area. Should be -1 for greenscreen. </dd> <dt><samp class=”option”>blue</samp></dt> <dd> Controls amount of blue in spill area. Should be -1 for bluescreen. </dd> <dt><samp class=”option”>brightness</samp></dt> <dd> Controls brightness of spill area, preserving colors. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Modify alpha from generated spillmap. </dd> </dl>

 <h4 class=”subsection”>32.66.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.67 detelecine# TOC</h3>

Apply an exact inverse of the telecine operation. It requires a predefined pattern specified using the pattern option which must be the same as that passed to the telecine filter. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> <dt><samp class=”option”>start_frame</samp></dt> <dd> A number representing position of the first frame with respect to the telecine pattern. This is to be used if the stream is cut. The default value is <code class=”code”>0</code>. </dd> </dl>

 <h3 class=”section”>32.68 dilation# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel

by the local(3x3) maximum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class="subsection">32.68.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.69 displace# TOC</h3>

Displace pixels as indicated by second and third input stream. It takes three input streams and outputs one stream, the first input is the source, and second and third input are displacement maps. The second input specifies how much to displace pixels along the x-axis, while the third input specifies how much to displace pixels along the y-axis. If one of displacement map streams terminates, last frame from that displacement map will be used. Note that once generated, displacements maps can be reused over and over again. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">edge</samp></dt> <dd> Set displace behavior for pixels that are out of range. Available values are: <dl class="table"> <dt>‘<samp class="samp">blank</samp>’</dt> <dd> Missing pixels are replaced by black pixels. </dd> <dt>‘<samp class="samp">smear</samp>’</dt> <dd> Adjacent pixels will spread out to replace missing pixels. </dd> <dt>‘<samp class="samp">wrap</samp>’</dt> <dd> Out of range pixels are wrapped so they point to pixels of other side. </dd> <dt>‘<samp class="samp">mirror</samp>’</dt> <dd> Out of range pixels will be replaced with mirrored pixels. </dd> </dl> Default is ‘<samp class="samp">smear</samp>’. </dd> </dl>

 <h4 class="subsection">32.69.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add ripple effect to rgb input

of video size hd720: <pre class=”example-preformatted”>ffmpeg -i INPUT -f lavfi -i nullsrc=s=hd720,lutrgb=128:128:128 -f lavfi -i nullsrc=s=hd720,geq='r=128+30*sin(2*PI*X/400+T):g=128+30*sin(2*PI*X/400+T):b=128+30*sin -lavfi '[0][1][2]displace' OUTPUT </pre>

Add wave effect to rgb input of video size hd720: <pre class=”example-preformatted”>ffmpeg -i INPUT -f lavfi -i nullsrc=hd720,geq='r=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):g=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T)):b=128+80*(sin(sqrt((X-W/2)*(X-W/2)+(Y-H/2)*(Y-H/2))/220*2*PI+T))' -lavfi '[1]split[x][y],[0][x][y]displace' OUTPUT </pre>

 <h3 class=”section”>32.70 dnn_classify# TOC</h3>

Do classification with deep neural networks based on bounding boxes. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts only openvino now, tensorflow backends will be added. </dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0, and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend For tensorflow backend, you can set its configs with <samp class=”option”>sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs for your system. </dd> </dl>

 <h3 class=”section”>32.71 dnn_detect# TOC</h3>

Do object detection with deep neural networks. The fil-

ter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt> <dd> OpenVINO backend. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. </dd> </dl>

</dd> <dt><samp class=”option”>model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set the input name of the dnn network. </dd> <dt><samp class=”option”>output</samp></dt> <dd> Set the output name of the dnn network. The TensorFlow and ONNX Runtime backends accept multiple output names separated by ‘<samp class=”samp”>&</samp>’, e.g. <samp class=”option”>output=yolo_13&yolo_26</samp>. </dd> <dt><samp class=”option”>confidence</samp></dt> <dd> Set the confidence threshold (default: 0.5). </dd> <dt><samp class=”option”>labels</samp></dt> <dd> Set path to label file specifying the mapping between label id and name. Each label name is written in one line, tailing spaces and empty lines are skipped. The first line is the name of label id 0 (usually it is ’background’), and the second line is the name of label id 1, etc. The label id is considered as name if the label file is not provided. </dd> <dt><samp class=”option”>model_type</samp></dt> <dd> Set the detection model output format. The following values are accepted: <dl class=”table”> <dt>‘<samp class=”samp”>ssd</samp>’</dt> <dd> Single-stage detector (default). </dd> <dt>‘<samp class=”samp”>yolo</samp>’</dt> <dd> YOLO v1/v2. </dd> <dt>‘<samp class=”samp”>yolov3</samp>’</dt> <dd> YOLOv3. </dd> <dt>‘<samp class=”samp”>yolov4</samp>’</dt> <dd> YOLOv4. </dd> </dl> </dd> <dt><samp class=”option”>anchors</samp></dt> <dd> Anchor box dimensions, separated by ‘<samp class=”samp”>&</samp>’. Required for YOLO-family models. The list is consumed in output order, so the anchors of the first output come first: with <samp class=”option”>output=yolo_13&yolo_26</samp> the leading pairs belong to <code class=”code”>yolo_13</code>. </dd> <dt><samp class=”option”>nb_classes</samp></dt> <dd> Number of detection classes. Required for YOLO-family models. </dd> <dt><samp class=”option”>backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. </dd> </dl>

 <h4 class=”subsection”>32.71.1 Examples<a class=”anchor hidden-xs” href=”#Examples-

86” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Run SSD object detection with an ONNX model (single output, DirectML on Windows): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=300:300,format=rgb24,dnn_detect=:model=ssd.onnx:model_type=ssd:confidence=0.5:device=dml" -f null - </pre>

Run YOLOv3 object detection with an ONNX model (two outputs, CPU): <pre class=”example-preformatted”>ffmpeg -i input.mp4 -vf "scale=416:416,format=rgb24,dnn_detect=dnn_backend=onnx :model=yolov3.onnx:output=yolo_13&yolo_26:model_type=yolov3:nb_classes=80 :anchors=116&90&156&198&373&326&30&61&62&45&59&119&116&90&156&198&373&326" -f null - </pre>

 <h3 class=”section”>32.72 dnn_processing# TOC</h3>

Do image processing with deep neural networks. It works together with another filter which converts the pixel format of the Frame to what the dnn network requires. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class=”url” href=”https://www.tensorflow.org/install/lang_c”>https://www and configure FFmpeg with <code class=”code”>--enable-libtensorflow</code> </dd> <dt>‘<samp class=”samp”>openvino</samp>’</dt> <dd> OpenVINO backend. To enable this backend you need to build and install the OpenVINO for C library (see https://github.com/openvinotoolkit/openvino/blob/master/build-instruction.md) and configure FFmpeg with <code class=”code”>--enable-libopenvino</code> (–extra-cflags=-I... –extra-ldflags=-L... might be needed if the header files and libraries are not installed into system path) </dd> <dt>‘<samp class=”samp”>torch</samp>’</dt> <dd> Libtorch backend. To enable this backend you need to build and install Libtorch for C++ library. Please download cxx11 ABI version (see https://pytorch.org/get-started/locally) and configure FFmpeg with <code class=”code”>--enable-libtorch --extra-cflags=-I/libtorch_root/libtorch/include --extra-cflags=-I/libtorch_root/libtorch/include/torch/csrc/api/include --extra-ldflags=-L/libtorch_root/libtorch/lib/</code> When FFm-

peg is built with CUDA support and the input frames use <code class=”code”>AV_PIX_FMT_CUDA</code> with an underlying pixel format of <code class=”code”>rgb0</code>, <code class=”code”>bgr0</code>, <code class=”code”>rgb24</code>, or <code class=”code”>bgr24</code>, the Torch backend maps GPU frame buffers directly into LibTorch CUDA tensors using zero-copy memory mapping. This avoids host-device PCIe transfers and can significantly reduce inference latency. The zero-copy path is selected automatically when the input format qualifies; no additional options are required. When the input format is not <code class=”code”>AV_PIX_FMT_CUDA</code>, the backend uses the standard CPU tensor path. </dd> <dt>‘<samp class=”samp”>onnx</samp>’</dt> <dd> ONNX Runtime backend. To enable this backend you need to install the ONNX Runtime library (see https://onnxruntime.ai/) and configure FFmpeg with <code class=”code”>--enable-libonnxruntime</code>. The current ONNX Runtime backend expects 4-D input and output tensors with NCHW layout and 32-bit floating-point element type (ONNX <code class=”code”>FLOAT</code>); models with integer or other element types (e.g. <code class=”code”>UINT8</code>) are not supported and will be rejected at load time. Models using NHWC layout or other ranks are not yet supported. Only single-input models are supported; the backend binds exactly one input tensor when running the model. The <samp class=”option”>input</samp> and <samp class=”option”>output</samp> options are optional for the ONNX Runtime backend; when they are omitted the backend resolves the tensor names from the session. Multiple output names may be supplied separated by ‘<samp class=”samp”>&</samp>’ (e.g. <samp class=”option”>output=out_a&out_b</samp>); however for <code class=”code”>dnn_processing</code> only the first output tensor is used for frame post-processing. The ONNX Runtime backend runs inference synchronously using a single inference request. The shared <samp class=”option”>async</samp> and <samp class=”option”>nireq</samp> options therefore have no effect for <code class=”code”>dnn_backend=onnx</code>; inference always runs synchronously regardless of their values. When FFmpeg is built with Direct3D 12 support and this build’s ONNX Runtime provides a working DirectML execution provider, setting <samp class=”option”>device=dml</samp> together with <code class=”code”>AV_PIX_FMT_D3D12</code> NV12 hardware frames as input (e.g. via <code class=”code”>-hwaccel d3d12va -hwaccel_output_format d3d12</code>) keeps frames in VRAM for the whole filter, from the decoder’s output through DNN inference and back out, without a host round-trip. Software-frame input retains the host I/O path. D3D12 input requires successful DirectML negotiation; unsupported hardware-input configurations fail during configuration. At runtime, <samp class=”file”>DirectML.dll</samp> and <samp class=”file”>d3dcompiler_47.dll</samp>

must be loadable from the application directory or System32. Currently, only single-channel (luma-only), <code class="code">FLOAT</code>/NCHW models with a <code class="code">[1,1,H,W]</code> output tensor and even input/output frame dimensions are supported; the chroma plane is copied or bilinearly upscaled on the GPU without going through the model. The output frames are D3D12 hardware frames, so they must either be consumed by a D3D12 encoder or downloaded explicitly with <code class="code">hwdownload,format=nv12</code>. </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO, Libtorch, and ONNX Runtime backends can load files for only their respective formats. </dd> <dt><samp class="option">input</samp></dt> <dd> Set the input name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class="option">output</samp></dt> <dd> Set the output name of the dnn network. Required for the TensorFlow backend; optional for the ONNX Runtime backend. </dd> <dt><samp class="option">backend_configs</samp></dt> <dd> Set the configs to be passed into backend. To use async execution, set async (default: set). Roll back to sync execution if the backend does not support async. For tensorflow backend, you can set its configs with <samp class="option">sess_config</samp> options, please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. </dd> <dt><samp class="option">async</samp></dt> <dd> Use DNN async inference (default: 1). When enabled, frames are submitted to the backend without blocking the caller. Supported by the Torch, OpenVINO, and TensorFlow backends; the ONNX Runtime backend ignores this option. </dd> <dt><samp class="option">nireq</samp></dt> <dd> Number of concurrent inference requests (default: 0, meaning auto). When set to 0, the backend automatically determines an appropriate number of requests. Higher values increase throughput by allowing multiple frames to be in flight simultaneously. </dd> <dt><samp class="option">batch_size</samp></dt> <dd> Number of frames to batch into a single inference request (default: 1). Values greater than 1 cause the backend to accumulate frames and concatenate them along the batch dimension before running the model. This can significantly increase throughput on GPUs at the cost of slightly higher per-frame latency. Currently supported by the OpenVINO and Torch backends. </dd> <dt><samp class="option">device</samp></dt> <dd> Set the device to run the model. For the ONNX Runtime backend this selects the execution provider: <code class="code">cpu</code> (default), <code class="code">cuda</code> (NVIDIA GPU), <code class="code">dml</code> (DirectML, Windows only) or <code class="code">vitisai</code> (AMD Ryzen AI NPU). </dd> <dt><samp class="option">device_id</samp></dt>

<dd> Set the device index used by GPU execution providers (e.g. <code class=”code”>cuda</code> or <code class=”code”>dml</code>) for the ONNX Runtime backend. Default is 0. </dd> <dt><samp class=”option”>threads_per_operation</samp></dt> <dd> ONNX Runtime backend only. Set the number of CPU threads used per ONNX Runtime operator when running with <code class=”code”>device=cpu</code>. Default is 0 (let ONNX Runtime choose automatically). Has no effect for GPU/NPU providers. </dd> </dl>

 <h4 class=”subsection”>32.72.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Remove rain in rgb24 frame with can.pb (see derain filter): <pre class=”example-preformatted”>./ffmpeg -i rain.jpg -vf format=rgb24,dnn_processing=dnn_backend=tensorflow:model=can.pb:input=x:output=y derain.jpg </pre>

Handle the Y channel with srcnn.pb (see sr filter) for frame with yuv420p (planar YUV formats supported): <pre class=”example-preformatted”>./ffmpeg -i 480p.jpg -vf format=yuv420p,scale=w=iw*2:h=ih*2,dnn_processing=dnn_backend=tensorflow:model=srcn -y srcnn.jpg </pre>

Handle the Y channel with espcn.pb (see sr filter), which changes frame size, for format yuv420p (planar YUV formats supported), please use tools/python/tf_sess_config.py to get the configs of TensorFlow backend for your system. <pre class=”example-preformatted”>./ffmpeg -i 480p.jpg -vf format=yuv420p,dnn_processing=dnn_backend= -y tmp.espcn.jpg </pre>

Perform zero-copy GPU inference with the Libtorch backend. Because there is currently no GPU-side filter to convert from YUV to RGB, <code class=”code”>hwupload</code> is used to upload CPU RGB frames into device memory. Once a GPU-side YUV-to-RGB filter becomes available, the <code class=”code”>hwupload</code> step will no longer be necessary and all frames can stay in device memory throughout the pipeline: <pre class=”example-preformatted”>./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda -i input.mp4 \ -vf "format=rgb24,hwupload,dnn_processing=dnn_backend=torch:model=my_model.pt,hwdownload,form \ -y output.mp4 </pre>

Run a single-channel (luma-only) super-resolution model on D3D12 hardware frames with the ONNX Runtime backend, keeping frames in VRAM from decode through inference (Windows only, requires a build with Direct3D 12 support and <code class=”code”>--enable-libonnxruntime</code> against a DirectML-capable ONNX Runtime).

The D3D12 output frames are encoded directly by a D3D12 hardware encoder: <pre class=”example-preformatted”>./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml \ -c:v h264_d3d12va -bf 0 -y output.mp4 </pre>

Same as above, but download the result to software frames after inference so that a software encoder can be used: <pre class=”example-preformatted”>./ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf dnn_processing=dnn_backend=onnx:model=sr_model.onnx:device=dml,hwdownloa \ -y output.mp4 </pre>

Process rgb24 frames with a TorchScript model using the Libtorch backend: <pre class=”example-preformatted”>./ffmpeg -i input.jpg -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_model.pt output.jpg </pre>

Process rgb24 frames with the Libtorch backend using batch inference (four frames per request): <pre class=”example-preformatted”>./ffmpeg -i input.mp4 -vf format=rgb24,dnn_processing=dnn_backend=torch:model=my_mo -y output.mp4 </pre>

 <h3 class=”section”>32.73 drawbox# TOC</h3>

Draw a colored box on the input image. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The expressions which specify the top left corner coordinates of the box. It defaults to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are interpreted as the input width and height. It defaults to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the box to write. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the box edge color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the box edge. A value of <code class=”code”>fill</code> will create a filled box. Default value is <code class=”code”>3</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With value <code class=”code”>1</code>, the pixels of the painted

box will overwrite the video’s color and alpha pixels. Default is <code class=”code”>0</code>, which composites the box onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> and <var class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class=”option”>box_source</samp></dt> <dd> Box source can be set as side_data_detection_bboxes if you want to use box data in detection bboxes of side data. If <var class=”var”>box_source</var> is set, the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>width</var> and <var class=”var”>height</var> will be ignored and still use box data in detection bboxes of side data. So please do not use this parameter if you were not sure about the box source. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The thickness of the drawn box. These constants allow the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var>, <var class=”var”>h</var> and <var class=”var”>t</var> expressions to refer to each other, so you may for example specify <code class=”code”>y=x/dar</code> or <code class=”code”>h=w/dar</code>. </dd> </dl>

 <h4 class=”subsection”>32.73.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Draw a black box around the edge of the input image: <pre class=”example-preformatted”>drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class=”example-preformatted”>drawbox=10:20:200:60:red@0.5

</pre>

The previous example can be specified as:
<pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5
</pre>

Fill the box with pink color: <pre
class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill
</pre>

Draw a 2-pixel red 2.40:1 mask:
<pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-
t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h4 class="subsection">32.73.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.74 drawgraph# TOC</h3>
Draw a graph using input video metadata. It accepts the following parameters: <dl class="table"> <dt><samp class="option">m1</samp></dt>
<dd> Set 1st frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class="option">fg1</samp></dt>
<dd> Set 1st foreground color expression. </dd> <dt><samp class="option">m2</samp></dt> <dd> Set 2nd frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class="option">fg2</samp></dt> <dd> Set 2nd foreground color expression. </dd> <dt><samp class="option">m3</samp></dt>
<dd> Set 3rd frame metadata key from which metadata values will be used to draw a graph. </dd> <dt><samp class="option">fg3</samp></dt>
<dd> Set 3rd foreground color expression. </dd> <dt><samp class="option">m4</samp></dt> <dd> Set 4th frame metadata key from which metadata values will be used to draw a graph. </dd>
<dt><samp class="option">fg4</samp></dt> <dd> Set 4th foreground color expression. </dd> <dt><samp class="option">min</samp></dt>
<dd> Set minimal value of metadata value. </dd> <dt><samp class="option">max</samp></dt> <dd> Set maximal value of metadata value. </dd> <dt><samp class="option">bg</samp></dt>
<dd> Set graph background color. Default is white. </dd>
<dt><samp class="option">mode</samp></dt> <dd> Set graph mode. Available values for mode is: <dl class="table">
<dt>‘<samp class="samp">bar</samp>’</dt> <dt>‘<samp

class=”samp”>dot</samp>’</dt> <dt>‘<samp class=”samp”>line</samp>’</dt>
</dl>

Default is <code class=”code”>line</code>. </dd> <dt><samp
class=”option”>slide</samp></dt> <dd> Set slide mode.
Available values for slide is: <dl class=”table”> <dt>‘<samp
class=”samp”>frame</samp>’</dt> <dd> Draw new frame
when right border is reached. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt
<dd> Replace old columns with new ones. </dd> <dt>‘<samp
class=”samp”>scroll</samp>’</dt> <dd> Scroll from right to
left. </dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt>
<dd> Scroll from left to right. </dd> <dt>‘<samp
class=”samp”>picture</samp>’</dt> <dd> Draw single picture.
</dd> </dl>

Default is <code class=”code”>frame</code>. </dd>
<dt><samp class=”option”>size</samp></dt> <dd> Set size of
graph video. For the syntax of this option, check the <a data-manual=”ffmpeg-
utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size"
section in the ffmpeg-utils manual. The default value is <code
class=”code”>900x256</code>. </dd> <dt><samp class=”option”>rate,
r</samp></dt> <dd> Set the output frame rate. Default value is <code
class=”code”>25</code>. The foreground color expressions
can use the following variables: <dl class=”table”> <dt><samp
class=”option”>MIN</samp></dt> <dd> Minimal value of metadata
value. </dd> <dt><samp class=”option”>MAX</samp></dt>
<dd> Maximal value of metadata value. </dd> <dt><samp
class=”option”>VAL</samp></dt> <dd> Current metadata key value.
</dd> </dl>

The color is defined as 0xAABBGGRR. </dd> </dl>

Example using metadata from signalstats
filter: <pre class=”example-preformatted”>signalstats,drawgraph=lavfi.signalst
</pre>

Example using metadata from ebur128
filter: <pre class=”example-preformatted”>ebur128=metadata=1,adrawgraph=
120:max=5 </pre>

 <h3 class=”section”>32.75 drawgrid<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#drawgrid” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-drawgrid”
aria-hidden=”true”>TOC</h3>

Draw a grid on the input image. It accepts the following parame-
ters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt>
<dt><samp class=”option”>y</samp></dt> <dd> The expressions
which specify the coordinates of some point of grid intersection (meant

to configure offset). Both default to 0. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the grid cell, if 0 they are interpreted as the input width and height, respectively, minus <code class=”code”>thickness</code>, so image gets framed. Default to 0. </dd> <dt><samp class=”option”>color, c</samp></dt> <dd> Specify the color of the grid. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value <code class=”code”>invert</code> is used, the grid color is the same as the video with inverted luma. </dd> <dt><samp class=”option”>thickness, t</samp></dt> <dd> The expression which sets the thickness of the grid line. Default value is <code class=”code”>1</code>. See below for the list of accepted constants. </dd> <dt><samp class=”option”>replace</samp></dt> <dd> Applicable if the input has alpha. With <code class=”code”>1</code> the pixels of the painted grid will overwrite the video’s color and alpha pixels. Default is <code class=”code”>0</code>, which composites the grid onto the input, leaving the video’s alpha intact. </dd> </dl>

The parameters for <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var> and <var class=”var”>h</var> and <var class=”var”>t</var> are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> The input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> The input grid cell width and height. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y coordinates of some point of grid intersection (meant to configure offset). </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The width and height of the drawn cell. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The thickness of the drawn cell. These constants allow the <var class=”var”>x</var>, <var class=”var”>y</var>, <var class=”var”>w</var>, <var class=”var”>h</var> and <var class=”var”>t</var> expressions to refer to each other, so you may for example specify <code class=”code”>y=x/dar</code> or <code class=”code”>h=w/dar</code>. </dd> </dl>

 <h4 class="subsection">32.75.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a grid with cell 100x100 pixels, thickness 2 pixels, with color red and an opacity of 50%: <pre class="example-preformatted">drawgrid=width=100:height=100:thickness=2:color=red@0.5 </pre>

Draw a white 3x3 grid with an opacity of 50%: <pre class="example-preformatted">drawgrid=w=iw/3:h=ih/3:t=2:c=white@0.5 </pre>

 <h4 class="subsection">32.75.2 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.76 drawtext# TOC</h3>

Draw a text string or text from a specified file on top of a video, using the libfreetype library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libfreetype</code> and <code class="code">--enable-libharfbuzz</code>. To enable default font fallback and the <var class="var">font</var> option you need to configure FFmpeg with <code class="code">--enable-libfontconfig</code>. To enable the <var class="var">text_shaping</var> option, you need to configure FFmpeg with <code class="code">--enable-libfribidi</code>. <h4 class="subsection">32.76.1 Syntax# TOC</h4>

It accepts the following parameters:

<dl class="table">

<dt><samp class="option">box</samp></dt>

<dd> Used to draw a box around text using the background color.

The value must be either 1 (enable) or 0 (disable).

The default value of <var class=”var”>box</var> is 0.

</dd>

<dt><samp class=”option”>boxborderw</samp></dt>

<dd> Set the width of the border to be drawn around the box using <var class=”var”>boxcolor</var>.

The value must be specified using one of the following formats:

<ul class=”itemize mark-bullet”>

<code class=”code”>boxborderw=10</code> set the width of all the borders to 10

<code class=”code”>boxborderw=10|20</code> set the width of the top and bottom borders to 10

and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30</code> set the width of the top border to 10, the width

of the bottom border to 30 and the width of the left and right borders to 20

<code class=”code”>boxborderw=10|20|30|40</code> set the borders width to 10 (top), 20 (right),

30 (bottom), 40 (left)

The default value of <var class=”var”>boxborderw</var> is "0".

</dd>

<dt><samp class=”option”>boxcolor</samp></dt>

<dd> The color to be used for drawing box around text. For the syntax of this

option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

The default value of <var class=”var”>boxcolor</var> is "white".

</dd>

<dt><samp class=”option”>line_spacing</samp></dt>

<dd> Set the line spacing in pixels. The default value of <var class="var">line_spacing</var> is 0.

</dd>

<dt><samp class="option">text_align</samp></dt>

<dd> Set the vertical and horizontal alignment of the text with respect to the box boundaries.

The value is combination of flags, one for the vertical alignment (T=top,

M=middle, B=bottom) and one for the horizontal alignment (L=left, C=center, R=right).

Please note that tab characters are only supported with the left horizontal alignment.

</dd>

<dt><samp class="option">y_align</samp></dt>

<dd> Specify what the <var class="var">y</var> value is referred to. Possible values are:

<ul class="itemize mark-bullet">

<code class="code">text</code> the top of the highest glyph of the first text line is placed at <var class="var">y</var>

<code class="code">baseline</code> the baseline of the first text line is placed at <var class="var">y</var>

<code class="code">font</code> the baseline of the first text line is placed at <var class="var">y</var> plus the

ascent (in pixels) defined in the font metrics

The default value of <var class="var">y_align</var> is "text" for backward compatibility.

</dd>

<dt><samp class="option">borderw</samp></dt>

<dd> Set the width of the border to be drawn around the text using <var class="var">bordercolor</var>.

The default value of <var class="var">borderw</var> is 0.

</dd>

<dt><samp class="option">bordercolor</samp></dt>

<dd> Set the color to be used for drawing border around text. For the syntax of this

option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">bordercolor</var> is "black".

</dd>

<dt><samp class="option">expansion</samp></dt>

<dd> Select how the <var class="var">text</var> is expanded. Can be either <code class="code">none</code>,

<code class="code">strftime</code> (deprecated) or <code class="code">normal</code> (default). See the

Text expansion section below for details.

</dd>

<dt><samp class="option">basetime</samp></dt>

<dd> Set a start time for the count. Value is in microseconds. Only applied

in the deprecated <code class="code">strftime</code> expansion mode. To emulate in normal expansion

mode use the <code class="code">pts</code> function, supplying the start time (in seconds)

as the second argument.

</dd>

<dt><samp class="option">fix_bounds</samp></dt>

<dd> If true, check and fix text coords to avoid clipping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> The color to be used for drawing fonts. For the syntax of this option, check

the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual.

The default value of <var class="var">fontcolor</var> is "black".

</dd>

<dt><samp class="option">fontcolor_expr</samp></dt>

<dd> String which is expanded the same way as <var class="var">text</var> to obtain dynamic

<var class="var">fontcolor</var> value. By default this option has empty value and is not

processed. When this option is set, it overrides <var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> The font family to be used for drawing text. By default Sans.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> The font file to be used for drawing text. The path must be included.

This parameter is mandatory if the fontconfig support is disabled.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Draw the text applying alpha blending. The value can

be a number between 0.0 and 1.0.

The expression accepts the same variables <var class="var">x, y</var> as well.

The default value is 1.

Please see <var class=”var”>fontcolor_expr</var>.

</dd>

<dt><samp class=”option”>fontsize</samp></dt>

<dd> The font size to be used for drawing text.

The default value of <var class=”var”>fontsize</var> is 16.

</dd>

<dt><samp class=”option”>text_shaping</samp></dt>

<dd> If set to 1, attempt to shape the text (for example, reverse the order of

right-to-left text and join Arabic characters) before drawing it.

Otherwise, just draw the text exactly as given.

By default 1 (if supported).

</dd>

<dt><samp class=”option”>ft_load_flags</samp></dt>

<dd> The flags to be used for loading the fonts.

The flags map the corresponding flags supported by libfreetype, and are

a combination of the following values:

<dl class=”table”>

<dt><var class=”var”>default</var></dt>

<dt><var class=”var”>no_scale</var></dt>

<dt><var class=”var”>no_hinting</var></dt>

<dt><var class=”var”>render</var></dt>

<dt><var class=”var”>no_bitmap</var></dt>

<dt><var class=”var”>vertical_layout</var></dt>

<dt><var class=”var”>force_autohint</var></dt>

<dt><var class=”var”>crop_bitmap</var></dt>

<dt><var class=”var”>pedantic</var></dt>

<dt><var class=”var”>ignore_global_advance_width</var></dt>

<dt><var class=”var”>no_recurse</var></dt>

<dt><var class=”var”>ignore_transform</var></dt>

<dt><var class=”var”>monochrome</var></dt>

<dt><var class=”var”>linear_design</var></dt>

<dt><var class=”var”>no_autohint</var></dt>

</dl>

Default value is "default".

For more information consult the documentation for the FT_LOAD_*

libfreetype flags.

</dd>

<dt><samp class=”option”>shadowcolor</samp></dt>

<dd> The color to be used for drawing a shadow behind the drawn text. For the

syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the

ffmpeg-utils manual.

The default value of <var class=”var”>shadowcolor</var> is "black".

</dd>

<dt><samp class=”option”>boxw</samp></dt>

<dd> Set the width of the box to be drawn around text.

The default value of <var class=”var”>boxw</var> is computed automatically to match the text width

</dd>

<dt><samp class=”option”>boxh</samp></dt>

<dd> Set the height of the box to be drawn around text.

The default value of <var class=”var”>boxh</var> is computed automatically to match the text height

</dd>

<dt><samp class=”option”>shadowx</samp></dt>

<dt><samp class=”option”>shadowy</samp></dt>

<dd> The x and y offsets for the text shadow position with respect to the

position of the text. They can be either positive or negative

values. The default value for both is "0".

</dd>

<dt><samp class=”option”>start_number</samp></dt>

<dd> The starting frame number for the n/frame_num variable. The default value

is "0".

</dd>

<dt><samp class=”option”>tabsize</samp></dt>

<dd> The size in number of spaces to use for rendering the tab.

Default value is 4.

</dd>

<dt><samp class=”option”>timecode</samp></dt>

<dd> Set the initial timecode representation in "hh:mm:ss[:;.]ff"

format. It can be used with or without text parameter. <var class=”var”>timecode_rate</var>

option must be specified.

</dd>

<dt><samp class=”option”>timecode_rate, rate, r</samp></dt>

<dd> Set the timecode frame rate (timecode only). Value will be rounded to nearest

integer. Minimum value is "1".

Drop-frame timecode is supported for frame rates 30 & 60.

</dd>

<dt><samp class="option">tc24hmax</samp></dt>

<dd> If set to 1, the output of the timecode option will wrap around at 24 hours.

Default is 0 (disabled).

</dd>

<dt><samp class="option">text</samp></dt>

<dd> The text string to be drawn. The text must be a sequence of UTF-8

encoded characters.

This parameter is mandatory if no file is specified with the parameter

<var class="var">textfile</var>.

</dd>

<dt><samp class="option">textfile</samp></dt>

<dd> A text file containing text to be drawn. The text must be a sequence

of UTF-8 encoded characters.

This parameter is mandatory if no text string is specified with the

parameter <var class="var">text</var>.

If both <var class="var">text</var> and <var class="var">textfile</var> are specified, an error is thrown.

</dd>

<dt><samp class="option">text_source</samp></dt>

<dd> Text source should be set as side_data_detection_bboxes if you want to use text data in

detection bboxes of side data.

If text source is set, <var class="var">text</var> and <var class="var">textfile</var> will be ignored and still use

text data in detection bboxes of side data. So please do not use this parameter

if you are not sure about the text source.

</dd>

<dt><samp class="option">reload</samp></dt>

<dd> The <var class="var">textfile</var> will be reloaded at specified frame interval.

Be sure to update <var class="var">textfile</var> atomically, or it may be read partially,

or even fail.

Range is 0 to INT_MAX. Default is 0.

</dd>

<dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt>

<dd> The expressions which specify the offsets where text will be drawn

within the video frame. They are relative to the top/left border of the

output image.

The default value of <var class="var">x</var> and <var class="var">y</var> is "0".

See below for the list of accepted constants and functions.

</dd>

</dl>

The parameters for <var class="var">x</var> and <var class="var">y</var> are expressions containing the

following constants and functions:

<dl class="table">

<dt><samp class=”option”>dar</samp></dt>

<dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>

</dd>

<dt><samp class=”option”>hsub</samp></dt>

<dt><samp class=”option”>vsub</samp></dt>

<dd> horizontal and vertical chroma subsample values. For example for the

pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1.

</dd>

<dt><samp class=”option”>line_h, lh</samp></dt>

<dd> the height of each text line

</dd>

<dt><samp class=”option”>main_h, h, H</samp></dt>

<dd> the input height

</dd>

<dt><samp class=”option”>main_w, w, W</samp></dt>

<dd> the input width

</dd>

<dt><samp class=”option”>max_glyph_a, ascent</samp></dt>

<dd> the maximum distance from the baseline to the highest/upper grid

coordinate used to place a glyph outline point, for all the rendered

glyphs.

It is a positive value, due to the grid’s orientation with the Y axis

upwards.

</dd>
<dt><samp class="option">max_glyph_d, descent</samp></dt>
<dd> the maximum distance from the baseline to the lowest grid coordinate used to place a glyph outline point, for all the rendered glyphs.
This is a negative value, due to the grid’s orientation, with the Y axis upwards.

</dd>
<dt><samp class="option">max_glyph_h</samp></dt>
<dd> maximum glyph height, that is the maximum height for all the glyphs contained in the rendered text, it is equivalent to <var class="var">ascent</var>
-
<var class="var">descent</var>.

</dd>
<dt><samp class="option">max_glyph_w</samp></dt>
<dd> maximum glyph width, that is the maximum width for all the glyphs contained in the rendered text

</dd>
<dt><samp class="option">font_a</samp></dt>
<dd> the ascent size defined in the font metrics

</dd>
<dt><samp class="option">font_d</samp></dt>
<dd> the descent size defined in the font metrics

</dd>
<dt><samp class="option">top_a</samp></dt>
<dd> the maximum ascender of the glyphs of the first text line

</dd>

<dt><samp class="option">bottom_d</samp></dt>

<dd> the maximum descender of the glyphs of the last text line

</dd>

<dt><samp class="option">n</samp></dt>

<dd> the number of input frame, starting from 0

</dd>

<dt><samp class="option">rand(min, max)</samp></dt>

<dd> return a random number included between <var class="var">min</var> and <var class="var">max</var>

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> The input sample aspect ratio.

</dd>

<dt><samp class="option">t</samp></dt>

<dd> timestamp expressed in seconds, NAN if the input timestamp is unknown

</dd>

<dt><samp class="option">text_h, th</samp></dt>

<dd> the height of the rendered text

</dd>

<dt><samp class="option">text_w, tw</samp></dt>

<dd> the width of the rendered text

</dd>

<dt><samp class=”option”>x</samp></dt>

<dt><samp class=”option”>y</samp></dt>

<dd> the x and y offset coordinates where the text is drawn.

These parameters allow the <var class=”var”>x</var> and <var class=”var”>y</var> expressions to refer

to each other, so you can for example specify <code class=”code”>y=x/dar</code>.

</dd>

<dt><samp class=”option”>pict_type</samp></dt>

<dd> A one character description of the current frame’s picture type.

</dd>

<dt><samp class=”option”>pkt_pos</samp></dt>

<dd> The current packet’s position in the input file or stream

(in bytes, from the start of the input). A value of -1 indicates

this info is not available.

</dd>

<dt><samp class=”option”>duration</samp></dt>

<dd> The current packet’s duration, in seconds.

</dd>

<dt><samp class=”option”>pkt_size</samp></dt>

<dd> The current packet’s size (in bytes).

</dd>

</dl>

 <h4 class=”subsection”>32.76.2 Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>strftime</code>, the filter recognizes

sequences accepted by the <code class=”code”>strftime</code> C function in the provided

text and expands them accordingly. Check the documentation of

<code class=”code”>strftime</code>. This feature is deprecated in favor of <code class=”code”>normal</code>

expansion with the <code class=”code”>gmtime</code> or <code class=”code”>localtime</code> expansion

functions.

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim.

If <samp class=”option”>expansion</samp> is set to <code class=”code”>normal</code> (which is the default),

the following expansion mechanism is used.

The backslash character ‘<samp class=”samp”>\</samp>’, followed by any character, always expands to

the second character.

Sequences of the form <code class=”code”>%{...}</code> are expanded. The text between the

braces is a function name, possibly followed by arguments separated by ’:’.

If the arguments contain special characters or delimiters (’:’ or ’}’),

they should be escaped.

Note that they probably must also be escaped as the value for the <samp class=”option”>text</samp>

option in the filter argument string and as the filter argument in the

filtergraph description, and possibly also for the shell, that makes up to four

levels of escaping; using a text file with the <samp class="option">textfile</samp> option avoids

these problems.

The following functions are available:

<dl class="table">

<dt><code class="command">expr, e</code></dt>

<dd> The expression evaluation result.

It must take one argument specifying the expression to be evaluated,

which accepts the same constants and functions as the <var class="var">x</var> and

<var class="var">y</var> values. Note that not all constants should be used, for

example the text size is not known when evaluating the expression, so

the constants <var class="var">text_w</var> and <var class="var">text_h</var> will have an undefined

value.

</dd>

<dt><code class="command">expr_int_format, eif</code></dt>

<dd> Evaluate the expression’s value and output as formatted integer.

The first argument is the expression to be evaluated, just as for the <var class="var">expr</var> function.

The second argument specifies the output format. Allowed values are ‘<samp class="samp">x</samp>’,

‘<samp class="samp">X</samp>’, ‘<samp class="samp">d</samp>’ and ‘<samp class="samp">u</samp>’. They are treated exactly as in the

<code class="code">printf</code> function.

The third parameter is optional and sets the number of positions taken by the output.

It can be used to add padding with zeros from the left.

</dd>

<dt><code class=”command”>gmtime</code></dt>

<dd> The time at which the filter is running, expressed in UTC.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>localtime</code></dt>

<dd> The time at which the filter is running, expressed in the local time zone.

It can accept an argument: a <code class=”code”>strftime</code> C function format string.

The format string is extended to support the variable <var class=”var”>%[1-6]N</var>

which prints fractions of the second with optionally specified number of digits.

</dd>

<dt><code class=”command”>metadata</code></dt>

<dd> Frame metadata. Takes one or two arguments.

The first argument is mandatory and specifies the metadata key.

The second argument is optional and specifies a default value, used when the

metadata key is not found or empty.

Available metadata can be identified by inspecting entries

starting with TAG included within each frame section

printed by running <code class=”code”>ffprobe -show_frames</code>.

String metadata generated in filters leading to

the drawtext filter are also available.

</dd>

<dt><code class=”command”>n, frame_num</code></dt>

<dd> The frame number, starting from 0.

</dd>

<dt><code class=”command”>pict_type</code></dt>

<dd> A one character description of the current picture type.

</dd>

<dt><code class=”command”>pts</code></dt>

<dd> The timestamp of the current frame.

It can take up to three arguments.

The first argument is the format of the timestamp; it defaults to <code class=”code”>flt</code>

for seconds as a decimal number with microsecond accuracy; <code class=”code”>hms</code> stands

for a formatted <var class=”var”>[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy.

<code class=”code”>gmtime</code> stands for the timestamp of the frame formatted as UTC time;

<code class=”code”>localtime</code> stands for the timestamp of the frame formatted as

local time zone time.

The second argument is an offset added to the timestamp.

If the format is set to <code class="code">hms</code>, a third argument <code class="code">24HH</code> may be

supplied to present the hour part of the formatted timestamp in 24h format

(00-23).

If the format is set to <code class="code">localtime</code> or <code class="code">gmtime</code>, a third

argument may be supplied: a <code class="code">strftime</code> C function format string.

By default, <var class="var">YYYY-MM-DD HH:MM:SS</var> format will be used.

</dd>

</dl>

 <h4 class="subsection">32.76.3 Commands# TOC</h4>

This filter supports altering parameters via commands: <dl class="table"> <dt><samp class="option">reinit</samp></dt> <dd> Alter existing filter parameters. Syntax for the argument is the same as for filter invocation, e.g.   <pre class="example-preformatted">fontsize=56:fontcolor=green:text='Hello World' </pre>

Full filter invocation with sendcmd would look like this: <pre class="example-preformatted">sendcmd=c='56.0 drawtext reinit fontsize=56\:fontcolor=green\:text=Hello\\ World' </pre>

If the entire argument can’t be parsed or applied as valid values then the filter will continue with its existing parameters. </dd> </dl>

The following options are also supported as commands: <ul class="itemize mark-bullet"> x y alpha fontsize fontcolor boxcolor bordercolor shadowcolor box boxw boxh boxborderw line_spacing text_align shadowx shadowy borderw

 <h4 class="subsection">32.76.4 Examples<a class="anchor hidden-xs" href="#Examples-

90” aria-hidden=”true”># TOC</h4>

<ul class=”itemize mark-bullet”> Draw "Test Text" with font FreeSerif, using the default values for the optional parameters.

 <pre class=”example-preformatted”>drawtext="fontfile=/usr/share/fonts/true
text='Test Text'" </pre>

Draw ’Test Text’ with font FreeSerif of size 24 at position x=100 and y=50 (counting from the top-left corner of the screen), text is yellow with a red box around it. Both the text and the box have an opacity of 20%.

 <pre class=”example-preformatted”>drawtext="fontfile=/usr/share/fonts/true
text='Test Text':\ x=100: y=50: fontsize=24: fontcolor=yellow@0.2: box=1: boxcolor=red@0.2" </pre>

Note that the double quotes are not necessary if spaces are not used within the parameter list. Show the text at the center of the video frame: <pre class=”example-preformatted”>drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=(w-text_w)/2:y=(h-text_h)/2" </pre>

Show the text at a random position, switching to a new position every 30 seconds: <pre class=”example-preformatted”>drawtext="fontsize=30:fontfile=FreeSerif.ttf:text='hello world':x=if(eq(mod(t\,30)\,0)\,rand(0\,(w-text_w))\,x):y=if(eq(mod(t\,30)\,0)\,rand(0\,(h-text_h))\,y)" </pre>

Show a text line sliding from right to left in the last row of the video frame. The file <samp class=”file”>LONG_LINE</samp> is assumed to contain a single line with no newlines. <pre class=”example-preformatted”>drawtext="fontsize=15:fontfile=FreeSerif.ttf:text=LONG_LINE:y=h-line_h:x=-50*t" </pre>

Show the content of file <samp class=”file”>CREDITS</samp> off the bottom of the frame and scroll up. <pre class=”example-preformatted”>drawtext="fontsize=20:fontfile=FreeSerif.ttf:textfile=CREDITS:y=h-20*t" </pre>

Draw a single green letter "g", at the center of the input video. The glyph baseline is placed at half screen height. <pre class=”example-preformatted”>drawtext="fontsize=60:fontfile=FreeSerif.ttf:fo
max_glyph_w)/2:y=h/2-ascent" </pre>

Show text for 1 second every 3 seconds: <pre class=”example-preformatted”>drawtext="fontfile=FreeSerif.ttf:fontcolor=white:x=100:y=x/dar:enable
</pre>

Use fontconfig to set the font. Note that the colons need to be escaped.

 <pre class=”example-preformatted”>drawtext='fontfile=Linux Libertine O-40\\:style=Semibold:text=FFmpeg' </pre>

Draw "Test Text" with font size dependent on height of the video. <pre class=”example-preformatted”>drawtext="text='Test Text': fontsize=h/30: x=(w-text_w)/2: y=(h-text_h*2)" </pre>

Print the date of a real-time encoding (see documentation for the <code class=”code”>strftime</code> C function): <pre class=”example-preformatted”>drawtext='fontfile=FreeSans.ttf:text=%{localtime\:%a %b %d %Y}' </pre>

Show text fading in and out (appearing/disappearing): <pre class=”example-preformatted”>#!/bin/sh DS=1.0 # display start DE=10.0 # display end FID=1.5 # fade in duration FOD=5 # fade out duration ffplay -f lavfi "color,drawtext=text=TEST:fontsize=50:fontfile=FreeSerif.ttf:fontcolor_
clip(255*(1*between(t\\, $DS + $FID\\, $DE - $FOD) + ((t - $DS)/$FID)*between(t\\, $DS\\, $DS + $FID) + (-(t - $DE)/$FOD)*between(t\\, $DE - $FOD\\, $DE))\\, 0\\, 255) \\\\: x\\\\: 2 }" </pre>

Horizontally align multiple separate texts. Note that <samp class=”option”>max_glyph_a</samp> and the <samp class=”option”>fontsize</samp> value are included in the <samp class=”option”>y</samp> offset. <pre class=”example-preformatted”>drawtext=fontfile=FreeSans.ttf:text=DOG:fontsize=
max_glyph_a, drawtext=fontfile=FreeSans.ttf:text=cow:fontsize=24:x=80:y=20+24-max_glyph_a </pre>

Plot special <var class=”var”>lavf.image2dec.source_basename</var> metadata onto each frame if such metadata exists. Otherwise, plot the string "NA". Note that image2 demuxer must have option <samp class=”option”>-export_path_metadata 1</samp> for the special metadata fields to be available for filters. <pre class=”example-preformatted”>drawtext="fontsize=20:fontcolor=white:fontfile=FreeSans.ttf:text='%{metadata\:lavf.i
</pre>

For more information about libfreetype, check: http://www.freetype.org/. For more information about fontconfig, check: <a class=”url” href=”http://freedesktop.org/software/fontc
user.html”>http://freedesktop.org/software/fontconfig/fontconfig-user.html. For more information about libfribidi, check: http://fribidi.org/. For more information about libharfbuzz, check: https://git
  <h3 class=”section”>32.77 drawvg# TOC</h3>

Draw vector graphics on top of video frames, by executing a script written in a custom language called VGS (Vector Graphics Script). The documentation for the language can be found in <a data-manual="drawvg-reference" href="./drawvg-reference.html#Top">drawvg - Language Reference. A version of this reference with rendered examples can be found at the author’s site. Graphics are rendered using the cario 2D graphics library. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-cairo</code>. <h4 class="subsection">32.77.1 Parameters# TOC</h4>

Either <code class="code">script</code> or <code class="code">file</code> must be set. <dl class="table"> <dt><samp class="option">s, script</samp></dt> <dd> Script source to draw the graphics. </dd> <dt><samp class="option">file</samp></dt> <dd> Path of the file to load the script source. </dd> </dl>

 <h4 class="subsection">32.77.2 Pixel Formats# TOC</h4>

Since Cairo only supports RGB images, if the input video is something else (like YUV 4:2:0), before executing the script the video is converted to a format compatible with Cairo. Then, you have to use either the format filter, or the <code class="code">-pix_fmt</code> option, to convert it to the expected format in the output. <h4 class="subsection">32.77.3 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw the outline of an ellipse.

 <pre class="example-preformatted">ffmpeg -i input.webm \ -vf 'drawvg=ellipse (w/2) (h/2) (w/3) (h/3) stroke' \ -pix_fmt yuv420p \ output.webm </pre>

 Draw a square rotating in the middle of the frame.

The script for drawvg is in a file <code class="code">draw.vgs</code>:   <pre class="example-preformatted">translate (w/2) (h/2) rotate t rect -100 -100 200 200 setcolor red@0.5 fill </pre>

Then: <pre class="example-preformatted">ffmpeg -i input.webm -vf 'drawvg=file=draw.vgs,format=yuv420p' output.webm

</pre>

 <h3 class="section">32.78 edgedetect# TOC</h3>

Detect and draw edges. The filter uses the Canny Edge Detection algorithm. The filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dt><samp class="option">high</samp></dt> <dd> Set low and high threshold values used by the Canny thresholding algorithm. The high threshold selects the "strong" edge pixels, which are then connected through 8-connectivity with the "weak" edge pixels selected by the low threshold. <var class="var">low</var> and <var class="var">high</var> threshold values must be chosen in the range [0,1], and <var class="var">low</var> should be lesser or equal to <var class="var">high</var>. Default value for <var class="var">low</var> is <code class="code">20/255</code>, and default value for <var class="var">high</var> is <code class="code">50/255</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Define the drawing mode. <dl class="table"> <dt>‘<samp class="samp">wires</samp>’</dt> <dd> Draw white/gray wires on black background. </dd> <dt>‘<samp class="samp">colormix</samp>’</dt> <dd> Mix the colors to create a paint/cartoon effect. </dd> <dt>‘<samp class="samp">canny</samp>’</dt> <dd> Apply Canny edge detector on all selected planes. </dd> </dl> Default value is <var class="var">wires</var>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Select planes for filtering. By default all available planes are filtered. </dd> </dl>

 <h4 class="subsection">32.78.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Standard edge detection with custom values for the hysteresis thresholding: <pre class="example-preformatted">edgedetect=low=0.1:high=0.4 </pre>

Painting effect without thresholding: <pre class="example-preformatted">edgedetect=mode=colormix:high=0 </pre>

 <h3 class="section">32.79 elbg# TOC</h3>

Apply a posterize effect using the ELBG (Enhanced LBG) algorithm. For each input image, the filter will compute the optimal mapping from the input to the output given the codebook length, that is the number of distinct output colors. This filter accepts the following options. <dl class="table"> <dt><samp class="option">codebook_length, l</samp></dt> <dd> Set codebook length. The value must be a positive integer, and represents the number of distinct output colors. Default value is 256. </dd> <dt><samp class="option">nb_steps, n</samp></dt> <dd> Set the maximum number of iterations to apply for computing the optimal mapping. The higher the value the better the result and the higher the computation time. Default value is 1. </dd> <dt><samp class="option">seed, s</samp></dt> <dd> Set a random seed, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">pal8</samp></dt> <dd> Set pal8 output pixel format. This option does not work with codebook length greater than 256. Default is disabled. </dd> <dt><samp class="option">use_alpha</samp></dt> <dd> Include alpha values in the quantization calculation. Allows creating palettized output images (e.g. PNG8) with multiple alpha smooth blending. </dd> </dl>

 <h3 class="section">32.80 entropy# TOC</h3>

Measure graylevel entropy in histogram of color channels of video frames. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Can be either <var class="var">normal</var> or <var class="var">diff</var>. Default is <var class="var">normal</var>. <var class="var">diff</var> mode measures entropy of histogram delta values, absolute differences between neighbour histogram values. </dd> </dl>

 <h3 class="section">32.81 epx# TOC</h3> Apply the EPX magnification filter which is designed for pixel art. It accepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">2xEPX</code>, <code class="code">3</code> for <code class="code">3xEPX</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">32.82 eq# TOC</h3> Set brightness, contrast, saturation and approximate gamma adjustment.

 The filter accepts the following options: <dl class="table">
<dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. The value must be a float value in range <code class="code">-1000.0</code> to <code class="code">1000.0</code>. The default value is "1". </dd> <dt><samp class="option">brightness</samp></dt>
<dd> Set the brightness expression. The value must be a float value in range <code class="code">-1.0</code> to <code class="code">1.0</code>. The default value is "0". </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the saturation expression. The value must be a float in range <code class="code">0.0</code> to <code class="code">3.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma</samp></dt>
<dd> Set the gamma expression. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_r</samp></dt>
<dd> Set the gamma expression for red. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_g</samp></dt>
<dd> Set the gamma expression for green. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_b</samp></dt>
<dd> Set the gamma expression for blue. The value must be a float in range <code class="code">0.1</code> to <code class="code">10.0</code>. The default value is "1". </dd> <dt><samp class="option">gamma_weight</samp></dt>
<dd> Set the gamma weight expression. It can be used to reduce the effect of a high gamma value on bright image areas, e.g. keep them from getting overamplified and just plain white. The value must be a float in range <code class="code">0.0</code> to <code class="code">1.0</code>. A value of <code class="code">0.0</code> turns the gamma correction all the way down while <code class="code">1.0</code> leaves it at its full strength. Default is "1". </dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for brightness, contrast, saturation and gamma expressions are evaluated. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt>
<dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.
</dd> </dl>

The expressions accept the following parameters: <dl class="table">
<dt><samp class="option">n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class="option">pos</samp></dt>
<dd> byte position of the corresponding packet in the input file, NAN if unspecified; deprecated, do not use </dd> <dt><samp class="option">r</samp></dt> <dd> frame rate of the input video,

NAN if the input frame rate is unknown </dd> <dt><samp class="option">t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> </dl>

 <h4 class="subsection">32.82.1 Commands# TOC</h4> The filter supports the following commands: <dl class="table"> <dt><samp class="option">contrast</samp></dt> <dd> Set the contrast expression. </dd> <dt><samp class="option">brightness</samp></dt> <dd> Set the brightness expression. </dd> <dt><samp class="option">saturation</samp></dt> <dd> Set the saturation expression. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Set the gamma expression. </dd> <dt><samp class="option">gamma_r</samp></dt> <dd> Set the gamma_r expression. </dd> <dt><samp class="option">gamma_g</samp></dt> <dd> Set gamma_g expression. </dd> <dt><samp class="option">gamma_b</samp></dt> <dd> Set gamma_b expression. </dd> <dt><samp class="option">gamma_weight</samp></dt> <dd> Set gamma_weight expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class="section">32.83 erosion# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class="table"> <dt><samp class="option">threshold0</samp></dt> <dt><samp class="option">threshold1</samp></dt> <dt><samp class="option">threshold2</samp></dt> <dt><samp class="option">threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> <dt><samp class="option">coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Default is 255 i.e. all eight pixels are used. Flags to local 3x3 coordinates maps like this: 1 2 3 4 5 6 7 8 </dd> </dl>

 <h4 class="subsection">32.83.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.84 estdif# <a class="anchor hidden-xs"href="#toc-estdif"

aria-hidden=”true”>TOC</h3>

Deinterlace the input video ("estdif" stands for "Edge Slope Tracing Deinterlacing Filter"). Spatial only filter that uses edge slope tracing algorithm to interpolate missing lines. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class=”option”>interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class=”code”>all</code>. </dd> <dt><samp class=”option”>rslope</samp></dt> <dd> Specify the search radius for edge slope tracing. Default value is 1. Allowed range is from 1 to 15. </dd> <dt><samp class=”option”>redge</samp></dt> <dd> Specify the search radius for best edge matching. Default value is 2. Allowed range is from 0 to 15. </dd> <dt><samp class=”option”>ecost</samp></dt> <dd> Specify the edge cost for edge matching. Default value is 2. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>mcost</samp></dt> <dd> Specify the middle cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>dcost</samp></dt> <dd> Specify the distance cost for edge matching. Default value is 1. Allowed range is from 0 to 50. </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Specify the interpolation used. Default is 4-point interpolation. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>2p</samp></dt> <dd> Two-point in-

terpolation. </dd> <dt><samp class="option">4p</samp></dt>
<dd> Four-point interpolation. </dd> <dt><samp class="option">6p</samp></dt>
<dd> Six-point interpolation. </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.84.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class="section">32.85 exposure# TOC</h3> Adjust exposure of the video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">exposure</samp></dt>
<dd> Set the exposure correction in EV. Allowed range is from -3.0 to 3.0 EV Default value is 0 EV. </dd> <dt><samp class="option">black</samp></dt> <dd> Set the black level correction. Allowed range is from -1.0 to 1.0. Default value is 0. </dd>
</dl>

 <h4 class="subsection">32.85.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">32.86 extractplanes# TOC</h3>

Extract color channel components from input video stream into separate grayscale video streams. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt>
<dd> Set plane(s) to extract. Available values for planes are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt>
<dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt>
<dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt>
</dl>

Choosing planes not available in the input will result in an error. That means you cannot select <code class="code">r</code>, <code class="code">g</code>, <code class="code">b</code> planes with <code class="code">y</code>, <code class="code">u</code>, <code class="code">v</code> planes at same time. </dd> </dl>

 <h4 class="subsection">32.86.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract luma, u and v color channel component from input video frame into 3 grayscale outputs: <pre class="example-preformatted">ffmpeg -i video.avi -filter_complex 'extractplanes=y+u+v[y][u][v]' -map '[y]' y.avi -map '[u]' u.avi -map '[v]' v.avi </pre>

 <h3 class="section">32.87 fade# TOC</h3>

Apply a fade-in/out effect to the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">type, t</samp></dt> <dd> The effect type can be either "in" for a fade-in, or "out" for a fade-out effect. Default is <code class="code">in</code>. </dd> <dt><samp class="option">start_frame, s</samp></dt> <dd> Specify the number of the frame to start applying the fade effect at. Default is 0. </dd> <dt><samp class="option">nb_frames, n</samp></dt> <dd> The number of frames that the fade effect lasts. At the end of the fade-in effect, the output video will have the same intensity as the input video. At the end of the fade-out transition, the output video will be filled with the selected <samp class="option">color</samp>. Default is 25. </dd> <dt><samp class="option">alpha</samp></dt> <dd> If set to 1, fade only alpha channel, if one exists on the input. Default value is 0. </dd> <dt><samp class="option">start_time, st</samp></dt> <dd> Specify the timestamp (in seconds) of the frame to start to apply the fade effect. If both start_frame and start_time are specified, the fade will start at whichever comes last. Default is 0. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> The number of seconds for which the fade effect has to last. At the end of the fade-in effect the output video will have the same intensity as the input video, at the end of the fade-out transition the output video will be filled with the selected <samp class="option">color</samp>. If both duration and nb_frames are specified, duration is used. Default is 0 (nb_frames is used by default). </dd> <dt><samp class="option">color, c</samp></dt> <dd> Specify the color of the fade. Default is "black". </dd> </dl>

 <h4 class="subsection">32.87.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Fade in the first 30 frames of video:

 <pre class=”example-preformatted”>fade=in:0:30 </pre>

The command above is equivalent to: <pre class=”example-preformatted”>fade=t=in:s=0:n=30 </pre>

Fade out the last 45 frames of a 200-frame video: <pre class=”example-preformatted”>fade=out:155:45 fade=type=out:start_frame=155:nb_frames=45 </pre>

Fade in the first 25 frames and fade out the last 25 frames of a 1000-frame video: <pre class=”example-preformatted”>fade=in:0:25, fade=out:975:25 </pre>

Make the first 5 frames yellow, then fade in from frame 5-24: <pre class=”example-preformatted”>fade=in:5:20:color=yellow </pre>

Fade in alpha over first 25 frames of video: <pre class=”example-preformatted”>fade=in:0:25:alpha=1 </pre>

Make the first 5.5 seconds black, then fade in for 0.5 seconds: <pre class=”example-preformatted”>fade=t=in:st=5.5:d=0.5 </pre>

 <h3 class=”section”>32.88 feedback# TOC</h3> Apply feedback video filter. This filter pass cropped input frames to 2nd output. From there it can be filtered with other video filters. After filter receives frame from 2nd input, that frame is combined on top of original frame from 1st input and passed to 1st output. The typical usage is filter only part of frame. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the top left crop position. </dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the crop size. </dd> </dl>

 <h4 class=”subsection”>32.88.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Blur only top left rectangular part of video frame size 100x100 with gblur filter. <pre class=”example-preformatted”>[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]gblur=8[bl </pre>

Draw black box on top left part of video frame of size 100x100 with drawbox filter. <pre class="example-preformatted">[in][blurin]feedback=x=0:y=0:w=100:h=100[out][blurout];[blurout]drawbox=x=0:y=0:w=100:
</pre>

Pixelize rectangular part of video frame of size 100x100 with pixelize filter. <pre class="example-preformatted">[in][blurin]feedback=x=320:y=240:w=100
</pre>

 <h3 class="section">32.89 fftdnoiz# TOC</h3> Denoise frames using 3D FFT (frequency domain filtering). The filter accepts the following options: <dl class="table"> <dt><samp class="option">sigma</samp></dt> <dd> Set the noise sigma constant. This sets denoising strength. Default value is 1. Allowed range is from 0 to 30. Using very high sigma with low overlap may give blocking artifacts. </dd> <dt><samp class="option">amount</samp></dt> <dd> Set amount of denoising. By default all detected noise is reduced. Default value is 1. Allowed range is from 0 to 1. </dd> <dt><samp class="option">block</samp></dt> <dd> Set size of block in pixels, Default is 32, can be 8 to 256. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set block overlap. Default is 0.5. Allowed range is from 0.2 to 0.8. </dd> <dt><samp class="option">method</samp></dt> <dd> Set denoising method. Default is <code class="code">wiener</code>, can also be <code class="code">hard</code>. </dd> <dt><samp class="option">prev</samp></dt> <dd> Set number of previous frames to use for denoising. By default is set to 0. </dd> <dt><samp class="option">next</samp></dt> <dd> Set number of next frames to to use for denoising. By default is set to 0. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes which will be filtered, by default are all available filtered except alpha. </dd> </dl>

 <h3 class="section">32.90 fftfilt#
TOC</h3>
Apply arbitrary expressions to samples in frequency domain
<dl class="table"> <dt><samp class="option">dc_Y</samp></dt>
<dd> Adjust the dc value (gain) of the luma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code class="code">0</code>. </dd> <dt><samp class="option">dc_U</samp></dt>
<dd> Adjust the dc value (gain) of the 1st chroma plane of the image. The filter accepts an integer value in range <code class="code">0</code> to <code class="code">1000</code>. The default value is set to <code

class=”code”>0</code>. </dd> <dt><samp class=”option”>dc_V</samp></dt> <dd> Adjust the dc value (gain) of the 2nd chroma plane of the image. The filter accepts an integer value in range <code class=”code”>0</code> to <code class=”code”>1000</code>. The default value is set to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>weight_Y</samp></dt> <dd> Set the frequency domain weight expression for the luma plane. </dd> <dt><samp class=”option”>weight_U</samp></dt> <dd> Set the frequency domain weight expression for the 1st chroma plane. </dd> <dt><samp class=”option”>weight_V</samp></dt> <dd> Set the frequency domain weight expression for the 2nd chroma plane. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’. The filter accepts the following variables: </dd> <dt><samp class=”option”>X</samp></dt> <dt><samp class=”option”>Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class=”option”>W</samp></dt> <dt><samp class=”option”>H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class=”option”>N</samp></dt> <dd> The number of input frame, starting from 0. </dd> <dt><samp class=”option”>WS</samp></dt> <dt><samp class=”option”>HS</samp></dt> <dd> The size of FFT array for horizontal and vertical processing. </dd> </dl>

 <h4 class=”subsection”>32.90.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> High-pass: <pre class=”example-preformatted”>fftfilt=dc_Y=128:weight_Y='squish(1-(Y+X)/100)' </pre>

Low-pass: <pre class=”example-preformatted”>fftfilt=dc_Y=0:weight_Y='squish((Y+X)/100-1)' </pre>

Sharpen: <pre class=”example-preformatted”>fftfilt=dc_Y=0:weight_Y=
(Y+X)/100)' </pre>

Blur: <pre class=”example-preformatted”>fftfilt=dc_Y=0:weight_Y='exp
4 * ((Y+X)/(W+H)))' </pre>

 <h3 class="section">32.91 field#
TOC</h3>

Extract a single field from an interlaced image using stride arithmetic to avoid wasting CPU time. The output frames are marked as non-interlaced. The filter accepts the following options: <dl class="table"> <dt><samp class="option">type</samp></dt> <dd> Specify whether to extract the top (if the value is <code class="code">0</code> or <code class="code">top</code>) or the bottom field (if the value is <code class="code">1</code> or <code class="code">bottom</code>). </dd> </dl>

 <h3 class="section">32.92 fieldhint# TOC</h3>

Create new frames by copying the top and bottom fields from surrounding frames supplied as numbers by the hint file. <dl class="table"> <dt><samp class="option">hint</samp></dt> <dd> Set file containing hints: absolute/relative frame numbers. There must be one line for each frame in a clip. Each line must contain two numbers separated by the comma, optionally followed by <code class="code">-</code> or <code class="code">+</code>. Numbers supplied on each line of file can not be out of [N-1,N+1] where N is current frame number for <code class="code">absolute</code> mode or out of [-1, 1] range for <code class="code">relative</code> mode. First number tells from which frame to pick up top field and second number tells from which frame to pick up bottom field. If optionally followed by <code class="code">+</code> output frame will be marked as interlaced, else if followed by <code class="code">-</code> output frame will be marked as progressive, else it will be marked same as input frame. If optionally followed by <code class="code">t</code> output frame will use only top field, or in case of <code class="code">b</code> it will use only bottom field. If line starts with <code class="code">#</code> or <code class="code">;</code> that line is skipped. </dd> <dt><samp class="option">mode</samp></dt> <dd> Can be item <code class="code">absolute</code> or <code class="code">relative</code> or <code class="code">pattern</code>. Default is <code class="code">absolute</code>. The <code class="code">pattern</code> mode is same as <code class="code">relative</code> mode, except at last entry of file if there are more frames to process than <code class="code">hint</code> file is seek back to start. </dd> </dl>

Example of first several lines of <code class="code">hint</code> file for <code class="code">relative</code> mode: <pre class="example-preformatted">0,0 - # first frame 1,0 - # second frame, use third's frame top field and second's frame bottom field 1,0 - # third frame,

use fourth's frame top field and third's frame bottom field 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - 0,0 - 1,0 - 1,0 - 1,0 - 0,0 - </pre>

 <h3 class="section">32.93 fieldmatch# TOC</h3>

Field matching filter for inverse telecine. It is meant to reconstruct the progressive frames from a telecined stream. The filter does not drop duplicated frames, so to achieve a complete inverse telecine <code class="code">fieldmatch</code> needs to be followed by a decimation filter such as decimate in the filtergraph. The separation of the field matching and the decimation is notably motivated by the possibility of inserting a de-interlacing filter fallback between the two. If the source has mixed telecined and real interlaced content, <code class="code">fieldmatch</code> will not be able to match fields for the interlaced parts. But these remaining combed frames will be marked as interlaced, and thus can be de-interlaced by a later filter such as yadif before decimation. In addition to the various configuration options, <code class="code">fieldmatch</code> can take an optional second stream, activated through the <samp class="option">ppsrc</samp> option. If enabled, the frames reconstruction will be based on the fields and frames from this second stream. This allows the first input to be pre-processed in order to help the various algorithms of the filter, while keeping the output lossless (assuming the fields are matched properly). Typically, a field-aware denoiser, or brightness/contrast adjustments can help. Note that this filter uses the same algorithms as TIVTC/TFM (AviSynth project) and VIVTC/VFM (VapourSynth project). The later is a light clone of TFM from which <code class="code">fieldmatch</code> is based on. While the semantic and usage are very close, some behaviour and options names can differ. The decimate filter currently only works for constant frame rate input. If your input has mixed telecined (30fps) and progressive content with a lower framerate like 24fps use the following filterchain to produce the necessary cfr stream: <code class="code">dejudder,fps=30000/1001,fieldmatch,decimate</code>. The filter accepts the following options: <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> Specify the assumed field order of the input stream. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Auto detect parity (use FFmpeg’s internal parity value). </dd> <dt>‘<samp class="samp">bff</samp>’</dt> <dd> Assume bottom field first. </dd> <dt>‘<samp class="samp">tff</samp>’</dt> <dd> Assume top field first. </dd> </dl>

Note that it is sometimes recommended not to trust the parity announced by the stream. Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set the matching mode or strategy to use. <samp class="option">pc</samp> mode is the safest in the sense that it won’t risk creating jerkiness due to duplicate frames when possible, but if there are bad edits or blended fields it will end up outputting combed frames when a good match might actually exist. On the other hand, <samp class="option">pcn_ub</samp> mode is the most risky in terms of creating jerkiness, but will almost always find a good frame if there is one. The other values are all somewhere in between <samp class="option">pc</samp> and <samp class="option">pcn_ub</samp> in terms of risking jerkiness and creating duplicate frames versus finding good matches in sections with bad edits, orphaned fields, blended fields, etc. More details about p/c/n/u/b are available in p/c/n/u/b meaning section. Available values are: <dl class="table"> <dt>‘<samp class="samp">pc</samp>’</dt> <dd> 2-way matching (p/c) </dd> <dt>‘<samp class="samp">pc_n</samp>’</dt> <dd> 2-way matching, and trying 3rd match if still combed (p/c + n) </dd> <dt>‘<samp class="samp">pc_u</samp>’</dt> <dd> 2-way matching, and trying 3rd match (same order) if still combed (p/c + u) </dd> <dt>‘<samp class="samp">pc_n_ub</samp>’</dt> <dd> 2-way matching, trying 3rd match if still combed, and trying 4th/5th matches if still combed (p/c + n + u/b) </dd> <dt>‘<samp class="samp">pcn</samp>’</dt> <dd> 3-way matching (p/c/n) </dd> <dt>‘<samp class="samp">pcn_ub</samp>’</dt> <dd> 3-way matching, and trying 4th/5th matches if all 3 of the original matches are detected as combed (p/c/n + u/b) </dd> </dl>

The parenthesis at the end indicate the matches that would be used for that mode assuming <samp class="option">order</samp>=<var class="var">tff</var> (and <samp class="option">field</samp> on <var class="var">auto</var> or <var class="var">top</var>). In terms of speed <samp class="option">pc</samp> mode is by far the fastest and <samp class="option">pcn_ub</samp> is the slowest. Default value is <var class="var">pc_n</var>. </dd> <dt><samp class="option">ppsrc</samp></dt> <dd> Mark the main input stream as a pre-processed input, and enable the secondary input stream as the clean source to pick the fields from. See the filter introduction for more details. It is similar to the <samp class="option">clip2</samp> feature from VFM/TFM. Default value is <code class="code">0</code> (disabled). </dd> <dt><samp class="option">field</samp></dt> <dd> Set the field to match from. It is recommended to set this to the same value as <samp class="option">order</samp> unless you experience matching failures with that setting. In certain circumstances changing the field that is used to match from can have a large impact on matching perfor-

mance. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Automatic (same value as <samp class="option">order</samp>). </dd> <dt>‘<samp class="samp">bottom</samp>’</dt> <dd> Match from the bottom field. </dd> <dt>‘<samp class="samp">top</samp>’</dt> <dd> Match from the top field. </dd> </dl>

Default value is <var class="var">auto</var>. </dd> <dt><samp class="option">mchroma</samp></dt> <dd> Set whether or not chroma is included during the match comparisons. In most cases it is recommended to leave this enabled. You should set this to <code class="code">0</code> only if your clip has bad chroma problems such as heavy rainbowing or other artifacts. Setting this to <code class="code">0</code> could also be used to speed things up at the cost of some accuracy. Default value is <code class="code">1</code>. </dd> <dt><samp class="option">y0</samp></dt> <dt><samp class="option">y1</samp></dt> <dd> These define an exclusion band which excludes the lines between <samp class="option">y0</samp> and <samp class="option">y1</samp> from being included in the field matching decision. An exclusion band can be used to ignore subtitles, a logo, or other things that may interfere with the matching. <samp class="option">y0</samp> sets the starting scan line and <samp class="option">y1</samp> sets the ending line; all lines in between <samp class="option">y0</samp> and <samp class="option">y1</samp> (including <samp class="option">y0</samp> and <samp class="option">y1</samp>) will be ignored. Setting <samp class="option">y0</samp> and <samp class="option">y1</samp> to the same value will disable the feature. <samp class="option">y0</samp> and <samp class="option">y1</samp> defaults to <code class="code">0</code>. </dd> <dt><samp class="option">scthresh</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change on the luma plane. Good values are in the <code class="code">[8.0, 14.0]</code> range. Scene change detection is only relevant in case <samp class="option">combmatch</samp>=<var class="var">sc</var>. The range for <samp class="option">scthresh</samp> is <code class="code">[0.0, 100.0]</code>. Default value is <code class="code">12.0</code>. </dd> <dt><samp class="option">combmatch</samp></dt> <dd> When <samp class="option">combatch</samp> is not <var class="var">none</var>, <code class="code">fieldmatch</code> will take into account the combed scores of matches when deciding what match to use as the final match. Available values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No final matching based on combed scores. </dd> <dt>‘<samp class="samp">sc</samp>’</dt> <dd> Combed scores are only used when a scene change is detected. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Use combed scores all the time. </dd> </dl>

Default is <var class=”var”>sc</var>. </dd> <dt><samp class=”option”>combdbg</samp></dt> <dd> Force <code class=”code”>fieldmatch</code> to calculate the combed metrics for certain matches and print them. This setting is known as <samp class=”option”>micout</samp> in TFM/VFM vocabulary. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No forced calculation. </dd> <dt>‘<samp class=”samp”>pcn</samp>’</dt> <dd> Force p/c/n calculations. </dd> <dt>‘<samp class=”samp”>pcnub</samp>’</dt> <dd> Force p/c/n/u/b calculations. </dd> </dl>

Default value is <var class=”var”>none</var>. </dd> <dt><samp class=”option”>cthresh</samp></dt> <dd> This is the area combing threshold used for combed frame detection. This essentially controls how "strong" or "visible" combing must be to be detected. Larger values mean combing must be more visible and smaller values mean combing can be less visible or strong and still be detected. Valid settings are from <code class=”code”>-1</code> (every pixel will be detected as combed) to <code class=”code”>255</code> (no pixel will be detected as combed). This is basically a pixel difference value. A good range is <code class=”code”>[8, 12]</code>. Default value is <code class=”code”>9</code>. </dd> <dt><samp class=”option”>chroma</samp></dt> <dd> Sets whether or not chroma is considered in the combed frame decision. Only disable this if your source has chroma problems (rainbowing, etc.) that are causing problems for the combed frame detection with chroma enabled. Actually, using <samp class=”option”>chroma</samp>=<var class=”var”>0</var> is usually more reliable, except for the case where there is chroma only combing in the source. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>blockx</samp></dt> <dt><samp class=”option”>blocky</samp></dt> <dd> Respectively set the x-axis and y-axis size of the window used during combed frame detection. This has to do with the size of the area in which <samp class=”option”>combpel</samp> pixels are required to be detected as combed for a frame to be declared combed. See the <samp class=”option”>combpel</samp> parameter description for more info. Possible values are any number that is a power of 2 starting at 4 and going up to 512. Default value is <code class=”code”>16</code>. </dd> <dt><samp class=”option”>combpel</samp></dt> <dd> The number of combed pixels inside any of the <samp class=”option”>blocky</samp> by <samp class=”option”>blockx</samp> size blocks on the frame for the frame to be detected as combed. While <samp class=”option”>cthresh</samp> controls how "visible" the combing must be, this setting controls "how much" combing there must be in any localized area (a window defined by the <samp class=”option”>blockx</samp> and <samp class=”option”>blocky</samp> settings) on the frame. Minimum value is

<code class="code">0</code> and maximum is <code class="code">blocky x blockx</code> (at which point no frames will ever be detected as combed). This setting is known as <samp class="option">MI</samp> in TFM/VFM vocabulary. Default value is <code class="code">80</code>. </dd> </dl>

 <h4 class="subsection">32.93.1 p/c/n/u/b meaning# TOC</h4>

 <h4 class="subsubsection">32.93.1.1 p/c/n# TOC</h4>

We assume the following telecined stream: <pre class="example-preformatted">Top fields: 1 2 2 3 4 Bottom fields: 1 2 3 4 4 </pre>

The numbers correspond to the progressive frame the fields relate to. Here, the first two frames are progressive, the 3rd and 4th are combed, and so on. When <code class="code">fieldmatch</code> is configured to run a matching from bottom (<samp class="option">field</samp>=<var class="var">bottom</var>) this is how this input stream get transformed: <pre class="example-preformatted">Input stream: T 1 2 2 3 4 B 1 2 3 4 4 <-- matching reference

Matches: c c n n c

Output stream: T 1 2 3 4 4 B 1 2 3 4 4 </pre>

As a result of the field matching, we can see that some frames get duplicated. To perform a complete inverse telecine, you need to rely on a decimation filter after this operation. See for instance the decimate filter. The same operation now matching from top fields (<samp class="option">field</samp>=<var class="var">top</var>) looks like this: <pre class="example-preformatted">Input stream: T 1 2 2 3 4 <-- matching reference B 1 2 3 4 4

Matches: c c p p c

Output stream: T 1 2 2 3 4 B 1 2 2 3 4 </pre>

In these examples, we can see what <var class="var">p</var>, <var class="var">c</var> and <var class="var">n</var> mean; basically, they refer to the frame and field of the opposite parity: <ul class="itemize mark-bullet"> <var class="var">p</var> matches the field of the

opposite parity in the previous frame <var class="var">c</var> matches the field of the opposite parity in the current frame <var class="var">n</var> matches the field of the opposite parity in the next frame

 <h4 class="subsubsection">32.93.1.2 u/b# TOC</h4>

The <var class="var">u</var> and <var class="var">b</var> matching are a bit special in the sense that they match from the opposite parity flag. In the following examples, we assume that we are currently matching the 2nd frame (Top:2, bottom:2). According to the match, a ’x’ is placed above and below each matched fields. With bottom matching (<samp class="option">field</samp>=<var class="var">bottom</var>): <pre class="example-preformatted">Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 1 2 2 2 2 2 2 1 3 </pre>

With top matching (<samp class="option">field</samp>=<var class="var">top</var>): <pre class="example-preformatted">Match: c p n b u

x x x x x Top 1 2 2 1 2 2 1 2 2 1 2 2 1 2 2 Bottom 1 2 3 1 2 3 1 2 3 1 2 3 1 2 3 x x x x x

Output frames: 2 2 2 1 2 2 1 3 2 2 </pre>

 <h4 class="subsection">32.93.2 Examples# TOC</h4>

Simple IVTC of a top field first telecined stream: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=none, decimate </pre>

Advanced IVTC, with fallback on yadif for still combed frames: <pre class="example-preformatted">fieldmatch=order=tff:combmatch=full, yadif=deint=interlaced, decimate </pre>

 <h3 class="section">32.94 fieldorder# TOC</h3>

Transform the field order of the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">order</samp></dt> <dd> The output field order. Valid values are <var class="var">tff</var> for top field first or <var class="var">bff</var> for bottom field first. </dd> </dl>

The default value is ‘<samp class="samp">tff</samp>’. The transformation is done by shifting the picture content up or down by one line, and filling the remaining line with appropriate picture content. This method is consistent with most broadcast field order converters. If the input video is not flagged as being interlaced, or it is already flagged as being of the required output field order, then this filter does not alter the incoming video. It is very useful when converting to or from PAL DV material, which is bottom field first. For example: <pre class="example-preformatted">ffmpeg -i in.vob -vf "fieldorder=bff" out.dv </pre>

 <h3 class="section">32.95 fillborders# TOC</h3>

Fill borders of the input video, without changing video stream dimensions. Sometimes video can have garbage at the four edges and you may not want to crop video input to keep size multiple of some number. This filter accepts the following options: <dl class="table"> <dt><samp class="option">left</samp></dt> <dd> Number of pixels to fill from left border. </dd> <dt><samp class="option">right</samp></dt> <dd> Number of pixels to fill from right border. </dd> <dt><samp class="option">top</samp></dt> <dd> Number of pixels to fill from top border. </dd> <dt><samp class="option">bottom</samp></dt> <dd> Number of pixels to fill from bottom border. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set fill mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">smear</samp>’</dt> <dd> fill pixels using outermost pixels </dd> <dt>‘<samp class="samp">mirror</samp>’</dt> <dd> fill pixels using mirroring (half sample symmetric) </dd> <dt>‘<samp class="samp">fixed</samp>’</dt> <dd> fill pixels with constant value </dd> <dt>‘<samp class="samp">reflect</samp>’</dt> <dd> fill pixels using reflecting (whole sample symmetric) </dd> <dt>‘<samp class="samp">wrap</samp>’</dt> <dd> fill pixels using wrapping </dd> <dt>‘<samp class="samp">fade</samp>’</dt> <dd> fade pixels to constant value </dd> <dt>‘<samp class="samp">margins</samp>’</dt> <dd> fill pixels at top and bottom with weighted averages pixels near borders </dd> </dl>

Default is <var class=”var”>smear</var>. </dd> <dt><samp class=”option”>color</samp></dt> <dd> Set color for pixels in fixed or fade mode. Default is <var class=”var”>black</var>. </dd> </dl>

 <h4 class=”subsection”>32.95.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>32.96 find_rect# TOC</h3>

Find a rectangular object in the input video. The object to search for must be specified as a gray8 image specified with the <samp class=”option”>object</samp> option. For each possible match, a score is computed. If the score reaches the specified threshold, the object is considered found. If the input video contains multiple instances of the object, the filter will find only one of them. When an object is found, the following metadata entries are set in the matching frame: <dl class=”table”> <dt><samp class=”option”>lavfi.rect.w</samp></dt> <dd> width of object </dd> <dt><samp class=”option”>lavfi.rect.h</samp></dt> <dd> height of object </dd> <dt><samp class=”option”>lavfi.rect.x</samp></dt> <dd> x position of object </dd> <dt><samp class=”option”>lavfi.rect.y</samp></dt> <dd> y position of object </dd> <dt><samp class=”option”>lavfi.rect.score</samp></dt> <dd> match score of the found object </dd> </dl>

It accepts the following options: <dl class=”table”> <dt><samp class=”option”>object</samp></dt> <dd> Filepath of the object image, needs to be in gray8. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Detection threshold, expressed as a decimal number in the range 0-1. A threshold value of 0.01 means only exact matches, a threshold of 0.99 means almost everything matches. Default value is 0.5. </dd> <dt><samp class=”option”>mipmaps</samp></dt> <dd> Number of mipmaps, default is 3. </dd> <dt><samp class=”option”>xmin, ymin, xmax, ymax</samp></dt> <dd> Specifies the rectangle in which to search. </dd> <dt><samp class=”option”>discard</samp></dt> <dd> Discard frames where object is not detected. Default is disabled. </dd> </dl>

 <h4 class=”subsection”>32.96.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cover a rectangular object by the supplied image of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i file.ts -vf find_rect=newref.pgm,cover_rect=cover.jpg:mode=cover new.mkv </pre>

Find the position of an object in each frame using <code class="command">ffprobe</code> and write it to a log file: <pre class="example-preformatted">ffprobe -f lavfi movie=test.mp4,find_rect=object=object.pgm:threshold=0.3 \ -show_entries frame=pkt_pts_time:frame_tags=lavfi.rect.x,lavfi.rect.y \ -of csv -o find_rect.csv </pre>

 <h3 class="section">32.97 floodfill# TOC</h3>

Flood area with values of same pixel components with another values. It accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set pixel x coordinate. </dd> <dt><samp class="option">y</samp></dt> <dd> Set pixel y coordinate. </dd> <dt><samp class="option">s0</samp></dt> <dd> Set source #0 component value. </dd> <dt><samp class="option">s1</samp></dt> <dd> Set source #1 component value. </dd> <dt><samp class="option">s2</samp></dt> <dd> Set source #2 component value. </dd> <dt><samp class="option">s3</samp></dt> <dd> Set source #3 component value. </dd> <dt><samp class="option">d0</samp></dt> <dd> Set destination #0 component value. </dd> <dt><samp class="option">d1</samp></dt> <dd> Set destination #1 component value. </dd> <dt><samp class="option">d2</samp></dt> <dd> Set destination #2 component value. </dd> <dt><samp class="option">d3</samp></dt> <dd> Set destination #3 component value. </dd> </dl>

 <h3 class="section">32.98 format# TOC</h3>

Convert the input video to one of the specified pixel formats. Libavfilter will try to pick one that is suitable as input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as "pix_fmts=yuv420p|monow|rgb24". </dd> <dt><samp class="option">color_spaces</samp></dt>

<dd> A ’|’-separated list of color space names, such as "color_spaces=bt709|bt470bg|bt2020nc". </dd> <dt><samp class="option">color_ranges</samp></dt> <dd> A ’|’-separated list of color range names, such as "color_ranges=tv|pc". </dd> <dt><samp class="option">alpha_modes</samp></dt> <dd> A ’|’-separated list of color range names, such as "alpha_modes=straight|premultiplied". </dd> <dt><samp class="option">chroma_locations</samp></dt> <dd> A ’|’-separated list of chroma sample location names, such as "chroma_locations=left|topleft". </dd> </dl>

 <h4 class="subsection">32.98.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert the input video to the <var class="var">yuv420p</var> format <pre class="example-preformatted">format=pix_fmts=yuv420p </pre>

Convert the input video to any of the formats in the list <pre class="example-preformatted">format=pix_fmts=yuv420p|yuv444p|yuv410p </pre>

 <h3 class="section">32.99 fps# TOC</h3>

Convert the video to specified constant frame rate by duplicating or dropping frames as necessary. It accepts the following parameters: <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> The desired output frame rate. It accepts expressions containing the following constants: <dl class="table"> <dt>‘<samp class="samp">source_fps</samp>’</dt> <dd> The input’s frame rate </dd> <dt>‘<samp class="samp">ntsc</samp>’</dt> <dd> NTSC frame rate of <code class="code">30000/1001</code> </dd> <dt>‘<samp class="samp">pal</samp>’</dt> <dd> PAL frame rate of <code class="code">25.0</code> </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> Film frame rate of <code class="code">24.0</code> </dd> <dt>‘<samp class="samp">ntsc_film</samp>’</dt> <dd> NTSC-film frame rate of <code class="code">24000/1001</code> </dd> </dl>

The default is <code class="code">25</code>. </dd> <dt><samp class="option">start_time</samp></dt> <dd> Assume the first PTS should be the given value, in seconds. This allows for padding/trimming at the start of stream. By default, no assumption is made

about the first frame’s expected PTS, so no padding or trimming is done. For example, this could be set to 0 to pad the beginning with duplicates of the first frame if a video stream starts after the audio stream or to trim any frames with a negative PTS. </dd> <dt><samp class="option">round</samp></dt> <dd> Timestamp (PTS) rounding method. Possible values are: <dl class="table"> <dt><samp class="option">zero</samp></dt> <dd> round towards 0 </dd> <dt><samp class="option">inf</samp></dt> <dd> round away from 0 </dd> <dt><samp class="option">down</samp></dt> <dd> round towards -infinity </dd> <dt><samp class="option">up</samp></dt> <dd> round towards +infinity </dd> <dt><samp class="option">near</samp></dt> <dd> round to nearest </dd> </dl> The default is <code class="code">near</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> Action performed when reading the last frame. Possible values are: <dl class="table"> <dt><samp class="option">round</samp></dt> <dd> Use same timestamp rounding method as used for other frames. </dd> <dt><samp class="option">pass</samp></dt> <dd> Pass through last frame if input duration has not been reached yet. </dd> </dl> The default is <code class="code">round</code>. If timestamp rounding would otherwise discard every frame, one frame is retained when its rounded timestamp equals both the first output timestamp and the rounded end of input. This exception extends the output beyond the rounded end of input by one output frame period. Frames whose entire interval is trimmed by <samp class="option">start_time</samp> remain discarded. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class="var">fps</var>[:<var class="var">start_time</var>[:<var class="var">round</var>]]. See also the setpts filter. <h4 class="subsection">32.99.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> A typical usage in order to set the fps to 25: <pre class="example-preformatted">fps=fps=25 </pre>

Sets the fps to 24, using abbreviation and rounding method to round to nearest: <pre class="example-preformatted">fps=fps=film:round=near </pre>

 <h3 class="section">32.100 framepack# TOC</h3>

Pack two different video streams into a stereoscopic video, setting

proper metadata on supported codecs. The two views should have the same size and framerate and processing will stop when the shorter video ends. Please note that you may conveniently adjust view properties with the scale and fps filters. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>format</samp></dt> <dd> The desired packing format. Supported values are: <dl class=”table”> <dt><samp class=”option”>sbs</samp></dt> <dd> The views are next to each other (default). </dd> <dt><samp class=”option”>tab</samp></dt> <dd> The views are on top of each other. </dd> <dt><samp class=”option”>lines</samp></dt> <dd> The views are packed by line. </dd> <dt><samp class=”option”>columns</samp></dt> <dd> The views are packed by column. </dd> <dt><samp class=”option”>frameseq</samp></dt> <dd> The views are temporally interleaved. </dd> </dl>

</dd> </dl>

Some examples: <pre class=”example-preformatted”># Convert left and right views into a frame-sequential video ffmpeg -i LEFT -i RIGHT -filter_complex framepack=frameseq OUTPUT

Convert views into a side-by-side video with the same output resolution as the input ffmpeg -i LEFT -i RIGHT -filter_complex [0:v]scale=w=iw/2[left],[1:v]scale=w=iw/2[right],[left][right]fra
OUTPUT </pre>

 <h3 class=”section”>32.101 framerate# TOC</h3>

Change the frame rate by interpolating new video output frames from the source frames. This filter is not designed to function correctly with interlaced media. If you wish to change the frame rate of interlaced media then you are required to deinterlace before this filter and re-interlace after this filter. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frames per second. This option can also be specified as a value alone. The default is <code class=”code”>50</code>. </dd> <dt><samp class=”option”>interp_start</samp></dt> <dd> Specify the start of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the default is <code class=”code”>15</code>. </dd> <dt><samp class=”option”>interp_end</samp></dt> <dd> Specify the end of a range where the output frame will be created as a linear interpolation of two frames. The range is [<code class=”code”>0</code>-<code class=”code”>255</code>], the de-

fault is <code class="code">240</code>. </dd> <dt><samp class="option">scene</samp></dt> <dd> Specify the level at which a scene change is detected as a value between 0 and 100 to indicate a new scene; a low value reflects a low probability for the current frame to introduce a new scene, while a higher value means the current frame is more likely to be one. The default is <code class="code">8.2</code>. </dd> <dt><samp class="option">flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class="var">flags</var> is: <dl class="table"> <dt><samp class="option">scene_change_detect, scd</samp></dt> <dd> Enable scene change detection using the value of the option <var class="var">scene</var>. This flag is enabled by default. </dd> </dl> </dd> </dl>

 <h3 class="section">32.102 frc_amf# TOC</h3>

Double the frame rate, using Frame Rate Converter (FRC) provided by AMD Advanced Media Framework library for hardware acceleration. The filter accepts the following options: <dl class="table"> <dt><samp class="option">engine_type</samp></dt> <dd> Specify the engine used to run shaders. <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11. </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 (Default value). </dd> </dl>

</dd> <dt><samp class="option">enable</samp></dt> <dd> Boolean value: enable/disable FRC. Dynamic value, can be altered at a runtime without re-initializing the filter. (Default value: enabled). </dd> <dt><samp class="option">fallback_mode</samp></dt> <dd> Fallback behavior in case of low interpolation confidence. <dl class="table"> <dt>‘<samp class="samp">duplicate</samp>’</dt> <dd> Duplicate frame. </dd> <dt>‘<samp class="samp">blend</samp>’</dt> <dd> Blend two frames together (Default value). </dd> </dl>

</dd> <dt><samp class="option">indicator</samp></dt> <dd> Boolean value: show FRC indicator square in the top left corner of the video (Default value: disabled). </dd> <dt><samp class="option">profile</samp></dt> <dd> Level of hierarchical motion search. <dl class="table"> <dt>‘<samp class="samp">low</samp>’</dt> <dd> Less levels of hierarchical motion search. Only recommended for extremely low resolutions. </dd> <dt>‘<samp class="samp">high</samp>’</dt> <dd> Recommended for any resolution up to 1440p. (Default value) </dd> <dt>‘<samp class="samp">super</samp>’</dt> <dd> More levels of hierarchical motion search. Recommended for resolutions 1440p or higher. </dd> </dl>

</dd> <dt><samp class="option">mv_search_mode</samp></dt> <dd> Performance mode of the motion search. <dl class="table"> <dt>‘<samp class="samp">native</samp>’</dt> <dd> Conduct motion search on the full resolution of source images. </dd> <dt>‘<samp class="samp">performance</samp>’</dt> <dd> Conduct motion search on the down scaled source images. Recommended for APU or low end GPU for better performance. </dd> </dl>

</dd> <dt><samp class="option">use_future_frame</samp></dt> <dd> Boolean value: enable dependency on future frame, improves quality for the cost of latency (Default value: enabled). </dd> </dl>

 <h3 class="section">32.103 vqe_amf# TOC</h3>

AMD AMF VQ Enhancer filter. This filter applies AMD AMF VQ Enhancement to the input video. This filter accepts the following option: <dl class="table"> <dt><samp class="option">attenuation</samp></dt> <dd> Set VQ Enhancer strength. Allowed range is from <code class="code">0.02</code> to <code class="code">0.4</code>. Default is <code class="code">0.1</code>. </dd> <dt><samp class="option">engine_type</samp></dt> <dd> Set AMF memory type used by the filter. Possible values: <dl class="table"> <dt>‘<samp class="samp">dx11</samp>’</dt> <dd> DirectX 11 </dd> <dt>‘<samp class="samp">dx12</samp>’</dt> <dd> DirectX 12 </dd> <dt>‘<samp class="samp">vulkan</samp>’</dt> <dd> Vulkan </dd> <dt>‘<samp class="samp">opencl</samp>’</dt> <dd> OpenCL </dd> </dl>

</dd> </dl>

Example: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf vqe_amf=attenuation=0.4 -c:v hevc_amf output.mp4 </pre>

 <h3 class="section">32.104 framestep# TOC</h3>

Select one frame every N-th frame. This filter accepts the following option: <dl class="table"> <dt><samp class="option">step</samp></dt> <dd> Select frame after every <code class="code">step</code> frames. Allowed values are positive integers higher than 0. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">32.105 freezedetect# TOC</h3>

Detect frozen video. This filter logs a message and sets frame metadata when it detects that the input video has no significant change in content during a specified duration. Video freeze detection calculates the mean average absolute difference of all the components of video frames and compares it to a noise floor. The printed times and duration are expressed in seconds. The <code class="code">lavfi.freezedetect.freeze_start</code> metadata key is set on the first frame whose timestamp equals or exceeds the detection duration and it contains the timestamp of the first frame of the freeze. The <code class="code">lavfi.freezedetect.freeze_duration</code> and <code class="code">lavfi.freezedetect.freeze_end</code> metadata keys are set on the first frame after the freeze. The filter accepts the following options: <dl class="table"> <dt><samp class="option">noise, n</samp></dt> <dd> Set noise tolerance. Can be specified in dB (in case "dB" is appended to the specified value) or as a difference ratio between 0 and 1. Default is -60dB, or 0.001. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set freeze duration until notification (default is 2 seconds). </dd> </dl>

 <h3 class="section">32.106 freezeframes# TOC</h3>

Freeze video frames. This filter freezes video frames using frame from 2nd input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">first</samp></dt> <dd> Set number of first frame from which to start freeze. </dd> <dt><samp class="option">last</samp></dt> <dd> Set number of last frame from which to end freeze. </dd> <dt><samp class="option">replace</samp></dt> <dd> Set number of frame from 2nd input which will be used instead of replaced frames. </dd> </dl>

 <h3 class="section">32.107 frei0r# TOC</h3>

Apply a frei0r effect to the input video. To enable the compilation of this filter, you need to install the frei0r header and configure FFmpeg with <code class="code">--enable-frei0r</code>. It accepts the following parameters: <dl class="table"> <dt><samp class="option">filter_name</samp></dt> <dd> The name of the frei0r effect to load. If the environment variable <code

class=”env”>FREI0R_PATH</code> is defined, the frei0r effect is searched for in each of the directories specified by the colon-separated list in <code class=”env”>FREI0R_PATH</code>. Otherwise, the standard frei0r paths are searched, in this order: <samp class=”file”>HOME/.frei0r-1/lib/</samp>, <samp class=”file”>/usr/local/lib/frei0r-1/</samp>, <samp class=”file”>/usr/lib/frei0r-1/</samp>. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r effect. </dd> </dl>

A frei0r effect parameter can be a boolean (its value is either "y" or "n"), a double, a color (specified as <var class=”var”>R</var>/<var class=”var”>G</var>/<var class=”var”>B</var>, where <var class=”var”>R</var>, <var class=”var”>G</var>, and <var class=”var”>B</var> are floating point numbers between 0.0 and 1.0, inclusive) or a color description as specified in the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual, a position (specified as <var class=”var”>X</var>/<var class=”var”>Y</var>, where <var class=”var”>X</var> and <var class=”var”>Y</var> are floating point numbers) and/or a string. The number and types of parameters depend on the loaded effect. If an effect parameter is not specified, the default value is set. <h4 class=”subsection”>32.107.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the distort0r effect, setting the first two double parameters: <pre class=”example-preformatted”>frei0r=filter_name=distort0r:filter_params=0.5|0.01 </pre>

Apply the colordistance effect, taking a color as the first parameter: <pre class=”example-preformatted”>frei0r=colordistance:0.2/0.3/0.4 frei0r=colordistance:violet frei0r=colordistance:0x112233 </pre>

Apply the perspective effect, specifying the top left and top right image positions: <pre class=”example-preformatted”>frei0r=perspective:0.2/0.2|0.8/0.2 </pre>

For more information, see http://frei0r.dyne.org <h4 class=”subsection”>32.107.2 Commands# TOC</h4>

This filter supports the <samp class=”option”>filter_params</samp> option as commands. <h3 class=”section”>32.108 fspp#

TOC</h3>

Apply fast and simple postprocessing. It is a faster version of spp. It splits (I)DCT into horizontal/vertical passes. Unlike the simple post- processing filter, one of them is performed once per block, not per pixel. This allows for much higher speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 4-5. Default value is <code class=”code”>4</code>. </dd> <dt><samp class=”option”>qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0-63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class=”option”>strength</samp></dt> <dd> Set filter strength. It accepts an integer in range -15 to 32. Lower values mean more details but also more artifacts, while higher values make the image smoother but also blurrier. Default value is <code class=”code”>0</code> − PSNR optimal. </dd> <dt><samp class=”option”>use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class=”code”>1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class=”code”>0</code> (not enabled). </dd> </dl>

 <h3 class=”section”>32.109 fsync# TOC</h3>

Synchronize video frames with an external mapping from a file. For each input PTS given in the map file it either drops or creates as many frames as necessary to recreate the sequence of output frames given in the map file. This filter is useful to recreate the output frames of a framerate conversion by the fps filter, recorded into a map file using the ffmpeg option <code class=”code”>-stats_mux_pre</code>, and do further processing to the corresponding frames e.g. quality comparison. Each line of the map file must contain three items per input frame, the input PTS (decimal), the output PTS (decimal) and the output TIMEBASE (decimal/decimal), separated by a space. This file format corresponds to the output of <code class=”code”>-stats_mux_pre_fmt="{ptsi} {pts} {tb}"</code>. The filter assumes the map file is sorted by increasing input PTS. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file, f</samp></dt> <dd> The filename of the map file to be used. </dd> </dl>

Example: <pre class=”example-preformatted”># Convert a video to 25 fps and record a MAP_FILE file with the default format of this filter ffmpeg -i INPUT -vf fps=fps=25 -stats_mux_pre MAP_FILE

-stats_mux_pre_fmt "{ptsi} {pts} {tb}" OUTPUT

Sort MAP_FILE by increasing input PTS sort -n MAP_FILE

Use INPUT, OUTPUT and the MAP_FILE from above to compare the corresponding frames in INPUT and OUTPUT via SSIM ffmpeg -i INPUT -i OUTPUT -filter_complex '[0:v]fsync=file=MAP_FILE[ref];[1:v][ref]ssim' -f null - </pre>

 <h3 class="section">32.110 gainmap# TOC</h3>

Compute an HDR gain map from two renditions of the same image, as defined by ISO 21496-1. The first input is the base rendition (typically SDR), and the second input is the alternate rendition (typically HDR). Both must have the same dimensions. Outputs a gain map, as either ‘<samp class="samp">gbrpf32</samp>’ or ‘<samp class="samp">grayf32</samp>’ depending on the chosen <samp class="option">mode</samp>. <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Controls what quantity the gain map is computed against. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">rgb</samp>’</dt> <dd> One gain map channel per component. </dd> <dt>‘<samp class="samp">luma</samp>’</dt> <dd> A single gain map channel, computed from the luma. </dd> <dt>‘<samp class="samp">maxrgb</samp>’</dt> <dd> A single gain map channel, computed from ‘<samp class="samp">max(R, G, B)</samp>’. </dd> </dl>

Default is ‘<samp class="samp">rgb</samp>’. </dd> <dt><samp class="option">colorspace</samp></dt> <dd> Chooses which colorspace to compute the gain map in. The accepted values are: <dl class="table"> <dt>‘<samp class="samp">base</samp>’</dt> <dd> The base rendition’s colorspace. </dd> <dt>‘<samp class="samp">alternate</samp>’</dt> <dd> The alternate rendition’s colorspace. </dd> </dl>

Default is ‘<samp class="samp">base</samp>’. </dd> <dt><samp class="option">min</samp></dt> <dt><samp class="option">max</samp></dt> <dd> Bounds on the encoded gain (log2). If unspecified, these will be measured from the frame. </dd> <dt><samp class="option">gamma</samp></dt> <dd> Encoding gamma of the resulting gain map. If unspecified, this will be optimized for the frame in question. Setting this to <code class="code">1.0</code> will produce a linear gain map, which may be more efficient in some applications. </dd> <dt><samp class="option">base_offset</samp></dt> <dt><samp class="option">alt_offset</samp></dt> <dd> Offset to prevent numer-

ical instability near zero. Both default to <code class=”code”>1/64</code>. </dd> <dt><samp class=”option”>base_nits</samp></dt> <dt><samp class=”option”>alt_nits</samp></dt> <dd> Override the reference luminance of the input, in nits. If unspecified, uses the values from the input metadata. </dd> </dl>

The <code class=”code”>gainmap</code> filter also supports the framesync options. <h4 class=”subsection”>32.110.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Compute a gain map between an SDR and an HDR version of the same image: <pre class=”example-preformatted”>ffmpeg -i $sdr -i $hdr -filter_complex "[0:v][1:v]gainmap" $out </pre>

 <h3 class=”section”>32.111 gblur# TOC</h3>

Apply Gaussian blur filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>steps</samp></dt> <dd> Set number of steps for Gaussian approximation. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class=”code”>sigma</code>. Default is <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>32.111.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>32.112 geq# TOC</h3>

Apply generic equation to each pixel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>lum_expr, lum</samp></dt> <dd> Set the luma expression. </dd> <dt><samp class=”option”>cb_expr, cb</samp></dt>

<dd> Set the chrominance blue expression. </dd> <dt><samp class="option">cr_expr, cr</samp></dt> <dd> Set the chrominance red expression. </dd> <dt><samp class="option">alpha_expr, a</samp></dt> <dd> Set the alpha expression. </dd> <dt><samp class="option">red_expr, r</samp></dt> <dd> Set the red expression. </dd> <dt><samp class="option">green_expr, g</samp></dt> <dd> Set the green expression. </dd> <dt><samp class="option">blue_expr, b</samp></dt> <dd> Set the blue expression. </dd> </dl>

The colorspace is selected according to the specified options. If one of the <samp class="option">lum_expr</samp>, <samp class="option">cb_expr</samp>, or <samp class="option">cr_expr</samp> options is specified, the filter will automatically select a YCbCr colorspace. If one of the <samp class="option">red_expr</samp>, <samp class="option">green_expr</samp>, or <samp class="option">blue_expr</samp> options is specified, it will select an RGB colorspace. If one of the chrominance expression is not defined, it falls back on the other one. If no alpha expression is specified it will evaluate to opaque value. If none of chrominance expressions are specified, they will evaluate to the luma expression. The expressions can use the following variables and functions: <dl class="table"> <dt><samp class="option">N</samp></dt> <dd> The sequential number of the filtered frame, starting from <code class="code">0</code>. </dd> <dt><samp class="option">X</samp></dt> <dt><samp class="option">Y</samp></dt> <dd> The coordinates of the current sample. </dd> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> The width and height of the image. </dd> <dt><samp class="option">SW</samp></dt> <dt><samp class="option">SH</samp></dt> <dd> Width and height scale depending on the currently filtered plane. It is the ratio between the corresponding luma plane number of pixels and the current plane ones. E.g. for YUV4:2:0 the values are <code class="code">1,1</code> for the luma plane, and <code class="code">0.5,0.5</code> for chroma planes. </dd> <dt><samp class="option">T</samp></dt> <dd> Time of the current frame, expressed in seconds. </dd> <dt><samp class="option">p(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the current plane. </dd> <dt><samp class="option">lum(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the luma plane. </dd> <dt><samp class="option">cb(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the blue-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">cr(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of

the red-difference chroma plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">r(x, y)</samp></dt> <dt><samp class="option">g(x, y)</samp></dt> <dt><samp class="option">b(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the red/green/blue component. Return 0 if there is no such component. </dd> <dt><samp class="option">alpha(x, y)</samp></dt> <dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the alpha plane. Return 0 if there is no such plane. </dd> <dt><samp class="option">psum(x,y), lumsum(x, y), cbsum(x,y), crsum(x,y), rsum(x,y), gsum(x,y), bsum(x,y), alphasum(x,y)</samp></dt> <dd> Sum of sample values in the rectangle from (0,0) to (x,y), this allows obtaining sums of samples within a rectangle. See the functions without the sum postfix. </dd> <dt><samp class="option">interpolation</samp></dt> <dd> Set one of interpolation methods: <dl class="table"> <dt><samp class="option">nearest, n</samp></dt> <dt><samp class="option">bilinear, b</samp></dt> </dl> Default is bilinear. </dd> </dl>

For functions, if <var class="var">x</var> and <var class="var">y</var> are outside the area, the value will be automatically clipped to the closer edge. Please note that this filter can use multiple threads in which case each slice will have its own expression state. If you want to use only a single expression state because your expressions depend on previous state then you should limit the number of filter threads to 1. <h4 class="subsection">32.112.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Flip the image horizontally: <pre class="example-preformatted">geq=p(W-X\,Y) </pre>

Generate a bidimensional sine wave, with angle <code class="code">PI/3</code> and a wavelength of 100 pixels: <pre class="example-preformatted">geq=128 + 100*sin(2*(PI/100)*(cos(PI/3)*(X-50*T) + sin(PI/3)*Y)):128:128 </pre>

Generate a fancy enigmatic moving light: <pre class="example-preformatted">nullsrc=s=256x256,geq=random(1)/hypot(X-cos(N*0.07)*W/2-W/2\,Y-sin(N*0.09)*H/2-H/2)^2*1000000*sin(N*0.02):128:128 </pre>

Generate a quick emboss effect: <pre class="example-preformatted">format=gray,geq=lum_expr='(p(X,Y)+(256-p(X-4,Y-4)))/2' </pre>

Modify RGB components depending on pixel position: <div

class=”example”> <pre class=”example-preformatted”>geq=r='X/W*r(X,Y)':g='(1-X/W)*g(X,Y)':b='(H-Y)/H*b(X,Y)' </pre>

Create a radial gradient that is the same size as the input (also see the vignette filter): <pre class=”example-preformatted”>geq=lum=255*gauss((X/W-0.5)*3)*gauss((Y/H-0.5)*3)/gauss(0)/gauss(0),format=gray </pre>

 <h3 class=”section”>32.113 gradfun# TOC</h3>

Fix the banding artifacts that are sometimes introduced into nearly flat regions by truncation to 8-bit color depth. Interpolate the gradients that should go where the bands are, and dither them. It is designed for playback only. Do not use it prior to lossy compression, because compression tends to lose the dither and bring back the bands. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>strength</samp></dt> <dd> The maximum amount by which the filter will change any one pixel. This is also the threshold for detecting nearly flat regions. Acceptable values range from .51 to 64; the default value is 1.2. Out-of-range values will be clipped to the valid range. </dd> <dt><samp class=”option”>radius</samp></dt> <dd> The neighborhood to fit the gradient to. A larger radius makes for smoother gradients, but also prevents the filter from modifying the pixels near detailed regions. Acceptable values are 8-32; the default value is 16. Out-of-range values will be clipped to the valid range. </dd> </dl>

Alternatively, the options can be specified as a flat string: <var class=”var”>strength</var>[:<var class=”var”>radius</var>] <h4 class=”subsection”>32.113.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the filter with a <code class=”code”>3.5</code> strength and radius of <code class=”code”>8</code>: <pre class=”example-preformatted”>gradfun=3.5:8 </pre>

Specify radius, omitting the strength (which will fall-back to the default value): <pre class=”example-preformatted”>gradfun=radius=8 </pre>

 <h3 class=”section”>32.114 graphmonitor<span class=”pull-

right”>#
TOC</h3>
Show various filtergraph stats. With this filter one can debug
complete filtergraph. Especially issues with links filling with queued frames.
 The filter accepts the following options: <dl class=”table”>
<dt><samp class=”option”>size, s</samp></dt> <dd> Set video
output size. Default is <var class=”var”>hd720</var>. </dd>
<dt><samp class=”option”>opacity, o</samp></dt> <dd> Set video
opacity. Default is <var class=”var”>0.9</var>. Allowed range is from
<var class=”var”>0</var> to <var class=”var”>1</var>. </dd>
<dt><samp class=”option”>mode, m</samp></dt> <dd> Set output
mode flags. Available values for flags are: <dl class=”table”>
<dt>‘<samp class=”samp”>full</samp>’</dt> <dd> No
any filtering. Default. </dd> <dt>‘<samp class=”samp”>compact</samp>’</dt>
<dd> Show only filters with queued frames. </dd> <dt>‘<samp
class=”samp”>nozero</samp>’</dt> <dd> Show only filters
with non-zero stats. </dd> <dt>‘<samp class=”samp”>noeof</samp>’</dt>
<dd> Show only filters with non-eof stat. </dd> <dt>‘<samp
class=”samp”>nodisabled</samp>’</dt> <dd> Show only filters
that are enabled in timeline. </dd> </dl>

</dd> <dt><samp class=”option”>flags, f</samp></dt> <dd> Set
flags which enable which stats are shown in video. Available
values for flags are: <dl class=”table”> <dt>‘<samp
class=”samp”>none</samp>’</dt> <dd> All flags turned off.
 </dd> <dt>‘<samp class=”samp”>all</samp>’</dt>
<dd> All flags turned on. </dd> <dt>‘<samp class=”samp”>queue</samp>’</dt
<dd> Display number of queued frames in each link. </dd>
<dt>‘<samp class=”samp”>frame_count_in</samp>’</dt>
<dd> Display number of frames taken from filter. </dd>
<dt>‘<samp class=”samp”>frame_count_out</samp>’</dt>
<dd> Display number of frames given out from filter. </dd>
<dt>‘<samp class=”samp”>frame_count_delta</samp>’</dt>
<dd> Display delta number of frames between above two values.
</dd> <dt>‘<samp class=”samp”>pts</samp>’</dt>
<dd> Display current filtered frame pts. </dd> <dt>‘<samp
class=”samp”>pts_delta</samp>’</dt> <dd> Display pts delta
between current and previous frame. </dd> <dt>‘<samp
class=”samp”>time</samp>’</dt> <dd> Display current filtered
frame time. </dd> <dt>‘<samp class=”samp”>time_delta</samp>’</dt>
<dd> Display time delta between current and previous frame.
</dd> <dt>‘<samp class=”samp”>timebase</samp>’</dt>
<dd> Display time base for filter link. </dd> <dt>‘<samp
class=”samp”>format</samp>’</dt> <dd> Display used format
for filter link. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt>
<dd> Display video size or number of audio channels in case of audio used

by filter link. </dd> <dt>‘<samp class="samp">rate</samp>’</dt> <dd> Display video frame rate or sample rate in case of audio used by filter link. </dd> <dt>‘<samp class="samp">eof</samp>’</dt> <dd> Display link output status. </dd> <dt>‘<samp class="samp">sample_count_in</samp>’</dt> <dd> Display number of samples taken from filter. </dd> <dt>‘<samp class="samp">sample_count_out</samp>’</dt> <dd> Display number of samples given out from filter. </dd> <dt>‘<samp class="samp">sample_count_delta</samp>’</dt> <dd> Display delta number of samples between above two values. </dd> <dt>‘<samp class="samp">disabled</samp>’</dt> <dd> Show the timeline filter status. </dd> </dl>

</dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set upper limit for video rate of output stream, Default value is <var class="var">25</var>. This guarantee that output video frame rate will not be higher than this value. </dd> </dl>

 <h3 class="section">32.115 grayworld# TOC</h3> A color constancy filter that applies color correction based on the grayworld assumption See: <a class="url" href="https://www.researchgate.net/publication/275213614_A_New_Color_Correction_Met
 The algorithm uses linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,grayworld,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h3 class="section">32.116 greyedge# TOC</h3> A color constancy variation filter which estimates scene illumination via grey edge algorithm and corrects the scene colors accordingly. See: <a class="url" href="https://staff.science.uva.nl/th.gevers/pub/GeversTIP07.pdf">https://staff.science.uva.nl/th.gevers/pu
 The filter accepts the following options: <dl class="table"> <dt><samp class="option">difford</samp></dt> <dd> The order of differentiation to be applied on the scene. Must be chosen in the range [0,2] and default value is 1. </dd> <dt><samp class="option">minknorm</samp></dt> <dd> The Minkowski parameter to be used for calculating the Minkowski distance. Must be chosen in the range [0,20] and default value is 1. Set to 0 for getting max value instead of calculating Minkowski distance. </dd> <dt><samp class="option">sigma</samp></dt> <dd> The standard deviation of Gaussian blur to be applied on the scene. Must

be chosen in the range [0,1024.0] and default value = 1. floor(<var class=”var”>sigma</var> * break_off_sigma(3)) can’t be equal to 0 if <var class=”var”>difford</var> is greater than 0. </dd> </dl>

 <h4 class=”subsection”>32.116.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Grey Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=5:sigma=2 </pre>

Max Edge: <pre class=”example-preformatted”>greyedge=difford=1:minknorm=0:sigma=2 </pre>

 <h3 class=”section”>32.117 guided# TOC</h3> Apply guided filter for edge-preserving smoothing, dehazing and so on. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set the box radius in pixels. Allowed range is 1 to 20. Default is 3. </dd> <dt><samp class=”option”>eps</samp></dt> <dd> Set regularization parameter (with square). Allowed range is 0 to 1. Default is 0.01. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set filter mode. Can be <code class=”code”>basic</code> or <code class=”code”>fast</code>. Default is <code class=”code”>basic</code>. </dd> <dt><samp class=”option”>sub</samp></dt> <dd> Set subsampling ratio for <code class=”code”>fast</code> mode. Range is 2 to 64. Default is 4. No subsampling occurs in <code class=”code”>basic</code> mode. </dd> <dt><samp class=”option”>guidance</samp></dt> <dd> Set guidance mode. Can be <code class=”code”>off</code> or <code class=”code”>on</code>. Default is <code class=”code”>off</code>. If <code class=”code”>off</code>, single input is required. If <code class=”code”>on</code>, two inputs of the same resolution and pixel format are required. The second input serves as the guidance. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set planes to filter. Default is first only. </dd> </dl>

 <h4 class=”subsection”>32.117.1 Commands# TOC</h4> This filter supports the all above options as commands. <h4 class=”subsection”>32.117.2 Examples<a class=”anchor hidden-xs” href=”#Examples-

106” aria-hidden=”true”># TOC</h4> <ul class=”itemize mark-bullet”> Edge-preserving smoothing with guided filter: <pre class=”example-preformatted”>ffmpeg -i in.png -vf guided out.png </pre>

Dehazing, structure-transferring filtering, detail enhancement with guided filter. For the generation of guidance image, refer to paper "Guided Image Filtering". See: <a class=”url” href=”http://kaiminghe.com/publications/pami12guidedfilter.pdf <pre class=”example-preformatted”>ffmpeg -i in.png -i guidance.png -filter_complex guided=guidance=on out.png </pre>

 <h3 class=”section”>32.118 haldclut# TOC</h3>

Apply a Hald CLUT to a video stream. First input is the video stream to process, and second one is the Hald CLUT. The Hald CLUT input can be a simple picture or a complete video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>clut</samp></dt> <dd> Set which CLUT video frames will be processed from second input stream, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> Force termination when the shortest input terminates. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> Continue applying the last CLUT after the end of the stream. A value of <code class=”code”>0</code> disable the filter after the last frame of the CLUT is reached. Default is <code class=”code”>1</code>. </dd> </dl>

<code class=”code”>haldclut</code> also has the same interpolation options as lut3d (both filters share the same internals). This filter also supports the framesync options. More information about the Hald CLUT can be found on Eskil Steenberg’s website (Hald CLUT author) at <a class=”url” href=”http://www.quelsolaar.com/technology/clut.html”>http://www.quelsolaar.com/technology/clut.html< <h4 class=”subsection”>32.118.1 Commands# TOC</h4>

This filter supports the <code class=”code”>interp</code> option as commands.

 <h4 class="subsection">32.118.2 Workflow examples# TOC</h4>

 <h4 class="subsubsection">32.118.2.1 Hald CLUT video stream# TOC</h4>

Generate an identity Hald CLUT stream altered with various effects: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf "hue=H=2*PI*t:s=sin(2*PI*t)+1, curves=cross_process" -t 10 -c:v ffv1 clut.nut </pre>

Note: make sure you use a lossless codec. Then use it with <code class="code">haldclut</code> to apply it on some random stream: <pre class="example-preformatted">ffmpeg -f lavfi -i mandelbrot -i clut.nut -filter_complex '[0][1] haldclut' -t 20 mandelclut.mkv </pre>

The Hald CLUT will be applied to the 10 first seconds (duration of <samp class="file">clut.nut</samp>), then the latest picture of that CLUT stream will be applied to the remaining frames of the <code class="code">mandelbrot</code> stream. <h4 class="subsubsection">32.118.2.2 Hald CLUT with preview# TOC</h4>

A Hald CLUT is supposed to be a squared image of <code class="code">Level*Level*Level</code> by <code class="code">Level*Level*Level</code> pixels. For a given Hald CLUT, FFmpeg will select the biggest possible square starting at the top left of the picture. The remaining padding pixels (bottom or right) will be ignored. This area can be used to add a preview of the Hald CLUT. Typically, the following generated Hald CLUT will be supported by the <code class="code">haldclut</code> filter: <pre class="example-preformatted">ffmpeg -f lavfi -i haldclutsrc=8 -vf " pad=iw+320 [padded_clut]; smptebars=s=320x256, split [a][b]; [padded_clut][a] overlay=W-320:h, curves=color_negative [main]; [main][b] overlay=W-320" -frames:v 1 clut.png </pre>

It contains the original and a preview of the effect of the CLUT: SMPTE color bars are displayed on the right-top, and below the same color bars processed by the color changes. Then, the effect of this Hald CLUT

can be visualized with: <pre class="example-preformatted">ffplay input.mkv -vf "movie=clut.png, [in] haldclut" </pre>

 <h3 class="section">32.119 hflip# TOC</h3>

Flip the input video horizontally. For example, to horizontally flip the input video with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf "hflip" out.avi </pre>

 <h3 class="section">32.120 histeq# TOC</h3> This filter applies a global color histogram equalization on a per-frame basis. It can be used to correct video that has a compressed range of pixel intensities. The filter redistributes the pixel intensities to equalize their distribution across the intensity range. It may be viewed as an "automatically adjusting contrast filter". This filter is useful only for correcting degraded or poorly captured source video. The filter accepts the following options: <dl class="table"> <dt><samp class="option">strength</samp></dt> <dd> Determine the amount of equalization to be applied. As the strength is reduced, the distribution of pixel intensities more-and-more approaches that of the input frame. The value must be a float number in the range [0,1] and defaults to 0.200. </dd> <dt><samp class="option">intensity</samp></dt> <dd> Set the maximum intensity that can generated and scale the output values appropriately. The strength should be set as desired and then the intensity can be limited if needed to avoid washing-out. The value must be a float number in the range [0,1] and defaults to 0.210. </dd> <dt><samp class="option">antibanding</samp></dt> <dd> Set the antibanding level. If enabled the filter will randomly vary the luminance of output pixels by a small amount to avoid banding of the histogram. Possible values are <code class="code">none</code>, <code class="code">weak</code> or <code class="code">strong</code>. It defaults to <code class="code">none</code>. </dd> </dl>

 <h3 class="section">32.121 histogram# TOC</h3>

Compute and draw a color distribution histogram for the input video. The computed histogram is a representation of the color component distribution in an image. Standard histogram displays the color components distribution in an image. Displays color graph for

each color component. Shows distribution of the Y, U, V, A or R, G, B components, depending on input format, in the current frame. Below each graph a color component scale meter is shown. The filter accepts the following options: <dl class="table"> <dt><samp class="option">level_height</samp></dt> <dd> Set height of level. Default value is <code class="code">200</code>. Allowed range is [50, 2048]. </dd> <dt><samp class="option">scale_height</samp></dt> <dd> Set height of color scale. Default value is <code class="code">12</code>. Allowed range is [0, 40]. </dd> <dt><samp class="option">display_mode</samp></dt> <dd> Set display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class="samp">overlay</samp>’</dt> <dd> Presents information identical to that in the <code class="code">parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">levels_mode</samp></dt> <dd> Set mode. Can be either <code class="code">linear</code>, or <code class="code">logarithmic</code>. Default is <code class="code">linear</code>. </dd> <dt><samp class="option">components</samp></dt> <dd> Set what color components to display. Default is <code class="code">7</code>. </dd> <dt><samp class="option">fgopacity</samp></dt> <dd> Set foreground opacity. Default is <code class="code">0.7</code>. </dd> <dt><samp class="option">bgopacity</samp></dt> <dd> Set background opacity. Default is <code class="code">0.5</code>. </dd> <dt><samp class="option">colors_mode</samp></dt> <dd> Set colors mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">whiteonblack</samp>’</dt> <dt>‘<samp class="samp">blackonwhite</samp>’</dt> <dt>‘<samp class="samp">whiteongray</samp>’</dt> <dt>‘<samp class="samp">blackongray</samp>’</dt> <dt>‘<samp class="samp">coloronblack</samp>’</dt> <dt>‘<samp class="samp">coloronwhite</samp>’</dt> <dt>‘<samp class="samp">colorongray</samp>’</dt> <dt>‘<samp class="samp">blackoncolor</samp>’</dt> <dt>‘<samp class="samp">whiteoncolor</samp>’</dt> <dt>‘<samp class="samp">grayoncolor</samp>’</dt> </dl> Default is <code class="code">whiteonblack</code>. </dd> </dl>

 <h4 class="subsection">32.121.1 Examples# <a class="anchor hidden-xs"href="#toc-

Examples-107" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Calculate and draw histogram: <pre class="example-preformatted">ffplay -i input -vf histogram </pre>

 <h3 class="section">32.122 hqdn3d# TOC</h3>

This is a high precision/quality 3d denoise filter. It aims to reduce image noise, producing smooth images and making still images really still. It should enhance compressibility. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">luma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial luma strength. It defaults to 4.0. </dd> <dt><samp class="option">chroma_spatial</samp></dt> <dd> A non-negative floating point number which specifies spatial chroma strength. It defaults to 3.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">luma_tmp</samp></dt> <dd> A floating point number which specifies luma temporal strength. It defaults to 6.0*<var class="var">luma_spatial</var>/4.0. </dd> <dt><samp class="option">chroma_tmp</samp></dt> <dd> A floating point number which specifies chroma temporal strength. It defaults to <var class="var">luma_tmp</var>*<var class="var">chroma_spatial</var>/<var class="var">luma_spatial</var>. </dd> </dl>

 <h4 class="subsection">32.122.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.123 hwdownload# TOC</h3>

Download hardware frames to system memory. The input must be in hardware frames, and the output a non-hardware format. Not all formats will be supported on the output - it may be necessary to insert an additional <samp class="option">format</samp> filter immediately following in the graph to get the output in a supported format. <h3 class="section">32.124 hwmap<a class="anchor hidden-xs" href="#hwmap" aria-

hidden=”true”># TOC</h3>

Map hardware frames to system memory or to another device. This filter has several different modes of operation; which one is used depends on the input and output formats: <ul class=”itemize mark-bullet”> Hardware frame input, normal frame output

Map the input frames to system memory and pass them to the output. If the original hardware frame is later required (for example, after overlaying something else on part of it), the <samp class=”option”>hwmap</samp> filter can be used again in the next mode to retrieve it. Normal frame input, hardware frame output

If the input is actually a software-mapped hardware frame, then unmap it - that is, return the original hardware frame. Otherwise, a device must be provided. Create new hardware surfaces on that device for the output, then map them back to the software format at the input and give those frames to the preceding filter. This will then act like the <samp class=”option”>hwupload</samp> filter, but may be able to avoid an additional copy when the input is already in a compatible format. Hardware frame input and output

A device must be supplied for the output, either directly or with the <samp class=”option”>derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). If the input frames were originally created on the output device, then unmap to retrieve the original frames. Otherwise, map the frames to the output device - create new hardware frames on the output corresponding to the frames on the input.

The following additional parameters are accepted: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the frame mapping mode. Some combination of: <dl class=”table”> <dt><var class=”var”>read</var></dt> <dd> The mapped frame should be readable. </dd> <dt><var class=”var”>write</var></dt> <dd> The mapped frame should be writeable. </dd> <dt><var class=”var”>overwrite</var></dt> <dd> The mapping will always overwrite the entire frame. This may improve performance in some cases, as the original contents of the frame need not be loaded. </dd> <dt><var class=”var”>direct</var></dt> <dd> The mapping must not involve any copying. Indirect mappings to copies of frames are created in some cases where either direct mapping is not possible or it would have unexpected properties. Setting this flag ensures that the mapping is direct and will fail if that is not possible. </dd> </dl> Defaults to <var class=”var”>read+write</var>

if not specified. </dd> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> <dt><samp class="option">reverse</samp></dt> <dd> In a hardware to hardware mapping, map in reverse - create frames in the sink and map them back to the source. This may be necessary in some cases where a mapping in one direction is required but only the opposite direction is supported by the devices being used. This option is dangerous - it may break the preceding filter in undefined ways if there are any additional constraints on that filter’s output. Do not use it without fully understanding the implications of its use. </dd> </dl>

 <h3 class="section">32.125 hwupload# TOC</h3>

Upload system memory frames to hardware surfaces. The device to upload to must be supplied when the filter is initialised. If using ffmpeg, select the appropriate device with the <samp class="option">-filter_hw_device</samp> option or with the <samp class="option">derive_device</samp> option. The input and output devices must be of different types and compatible - the exact meaning of this is system-dependent, but typically it means that they must refer to the same underlying hardware context (for example, refer to the same graphics card). The following additional parameters are accepted: <dl class="table"> <dt><samp class="option">derive_device <var class="var">type</var></samp></dt> <dd> Rather than using the device supplied at initialisation, instead derive a new device of type <var class="var">type</var> from the device the input frames exist on. </dd> </dl>

 <h3 class="section">32.126 hwupload_cuda# TOC</h3>

Upload system memory frames to a CUDA device. It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">device</samp></dt> <dd> The number of the CUDA device to use </dd> </dl>

 <h3 class="section">32.127 hqx# TOC</h3>

Apply a high-quality magnification filter designed for pixel art. This filter was originally created by Maxim Stepin. It ac-

cepts the following option: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the scaling dimension: <code class="code">2</code> for <code class="code">hq2x</code>, <code class="code">3</code> for <code class="code">hq3x</code> and <code class="code">4</code> for <code class="code">hq4x</code>. Default is <code class="code">3</code>. </dd> </dl>

 <h3 class="section">32.128 hstack# TOC</h3> Stack input videos horizontally. All streams must be of same pixel format and of same height. Note that this filter is faster than using overlay and pad filter to create same output. The filter accepts the following option: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class="option">shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class="section">32.129 hsvhold# TOC</h3> Turns a certain HSV range into gray values. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to be gray or not. The filter accepts the following options: <dl class="table"> <dt><samp class="option">hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class="option">sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class="option">blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully gray, or not gray at all. Higher values result in more gray pixels, with a higher gray pixel the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class="section">32.130 hsvkey<span

class=”pull-right”># TOC</h3> Turns a certain HSV range into transparency. This filter measures color difference between set HSV color in options and ones measured in video stream. Depending on options, output colors can be changed to transparent by adding alpha channel. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>hue</samp></dt> <dd> Set the hue value which will be used in color difference calculation. Allowed range is from -360 to 360. Default value is 0. </dd> <dt><samp class=”option”>sat</samp></dt> <dd> Set the saturation value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class=”option”>val</samp></dt> <dd> Set the value which will be used in color difference calculation. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class=”option”>similarity</samp></dt> <dd> Set similarity percentage with the key color. Allowed range is from 0 to 1. Default value is 0.01. 0.00001 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage. Allowed range is from 0 to 1. Default value is 0. 0.0 makes pixels either fully transparent, or not transparent at all. Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h3 class=”section”>32.131 hue# TOC</h3>

Modify the hue and/or the saturation of the input. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>h</samp></dt> <dd> Specify the hue angle as a number of degrees. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>s</samp></dt> <dd> Specify the saturation in the [-10,10] range. It accepts an expression and defaults to "1". </dd> <dt><samp class=”option”>H</samp></dt> <dd> Specify the hue angle as a number of radians. It accepts an expression, and defaults to "0". </dd> <dt><samp class=”option”>b</samp></dt> <dd> Specify the brightness in the [-10,10] range. It accepts an expression and defaults to "0". </dd> </dl>

<samp class=”option”>h</samp> and <samp class=”option”>H</samp> are mutually exclusive, and can’t be specified at the same time. The <samp class=”option”>b</samp>, <samp class=”option”>h</samp>, <samp class=”option”>H</samp> and <samp class=”option”>s</samp> option values are expressions containing the following constants: <dl

class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> frame count of the input frame starting from 0 </dd> <dt><samp class=”option”>pts</samp></dt> <dd> presentation timestamp of the input frame expressed in time base units </dd> <dt><samp class=”option”>r</samp></dt> <dd> frame rate of the input video, NAN if the input frame rate is unknown </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class=”option”>tb</samp></dt> <dd> time base of the input video </dd> </dl>

 <h4 class=”subsection”>32.131.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Set the hue to 90 degrees and the saturation to 1.0: <pre class=”example-preformatted”>hue=h=90:s=1 </pre>

Same command but expressing the hue in radians: <pre class=”example-preformatted”>hue=H=PI/2:s=1 </pre>

Rotate hue and make the saturation swing between 0 and 2 over a period of 1 second: <pre class=”example-preformatted”>hue="H=2*PI*t: s=sin(2*PI*t)+1" </pre>

Apply a 3 seconds saturation fade-in effect starting at 0: <pre class=”example-preformatted”>hue="s=min(t/3\,1)" </pre>

The general fade-in expression can be written as: <pre class=”example-preformatted”>hue="s=min(0\, max((t-START)/DURATION\, 1))" </pre>

Apply a 3 seconds saturation fade-out effect starting at 5 seconds: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (8-t)/3))" </pre>

The general fade-out expression can be written as: <pre class=”example-preformatted”>hue="s=max(0\, min(1\, (START+DURATION-t)/DURATION))" </pre>

 <h4 class=”subsection”>32.131.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table">
<dt><samp class="option">b</samp></dt> <dt><samp class="option">s</samp></dt>
<dt><samp class="option">h</samp></dt> <dt><samp class="option">H</samp></dt>
<dd> Modify the hue and/or the saturation and/or brightness of the input
video. The command accepts the same syntax of the corresponding option.
 If the specified expression is not valid, it is kept at its current value.
</dd> </dl>

 <h3 class="section">32.132 huesatura-
tion<a class="anchor hidden-xs" href="#huesaturation"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
huesaturation" aria-hidden="true">TOC</h3> Apply
hue-saturation-intensity adjustments to input video stream. This
filter operates in RGB colorspace. This filter accepts the following op-
tions: <dl class="table"> <dt><samp class="option">hue</samp></dt>
<dd> Set the hue shift in degrees to apply. Default is 0. Allowed range is
from -180 to 180. </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set the saturation shift. Default is 0. Allowed range is from -1
to 1. </dd> <dt><samp class="option">intensity</samp></dt>
<dd> Set the intensity shift. Default is 0. Allowed range is from -1
to 1. </dd> <dt><samp class="option">colors</samp></dt>
<dd> Set which primary and complementary colors are going to be
adjusted. This options is set by providing one or multiple values. This can
select multiple colors at once. By default all colors are selected. <dl
class="table"> <dt>‘<samp class="samp">r</samp>’</dt>
<dd> Adjust reds. </dd> <dt>‘<samp class="samp">y</samp>’</dt>
<dd> Adjust yellows. </dd> <dt>‘<samp class="samp">g</samp>’</dt>
<dd> Adjust greens. </dd> <dt>‘<samp class="samp">c</samp>’</dt>
<dd> Adjust cyans. </dd> <dt>‘<samp class="samp">b</samp>’</dt>
<dd> Adjust blues. </dd> <dt>‘<samp class="samp">m</samp>’</dt>
<dd> Adjust magentas. </dd> <dt>‘<samp class="samp">a</samp>’</dt>
<dd> Adjust all colors. </dd> </dl>

</dd> <dt><samp class="option">strength</samp></dt> <dd> Set
strength of filtering. Allowed range is from 0 to 100. Default value is
1. </dd> <dt><samp class="option">rw, gw, bw</samp></dt>
<dd> Set weight for each RGB component. Allowed range is from
0 to 1. By default is set to 0.333, 0.334, 0.333. Those options are
used in saturation and lightess processing. </dd> <dt><samp
class="option">lightness</samp></dt> <dd> Set preserving lightness,
by default is disabled. Adjusting hues can change lightness from original RGB
triplet, with this option enabled lightness is kept at same value. </dd>
</dl>

 <h3 class="section">32.133 hysteresis<span
class="pull-right"><a class="anchor hidden-xs" href="#hysteresis" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-hysteresis"

aria-hidden=”true”>TOC</h3>

Grow first stream into second stream by connecting components. This makes it possible to build more robust edge masks. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> Set threshold which is used in filtering. If pixel component value is higher than this value filter algorithm for connecting components is activated. By default value is 0. </dd> </dl>

The <code class=”code”>hysteresis</code> filter also supports the framesync options. <h3 class=”section”>32.134 iccdetect# TOC</h3>

Detect the colorspace from an embedded ICC profile (if present), and update the frame’s tags accordingly. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>force</samp></dt> <dd> If true, the frame’s existing colorspace tags will always be overridden by values detected from an ICC profile. Otherwise, they will only be assigned if they contain <code class=”code”>unknown</code>. Enabled by default. </dd> </dl>

 <h3 class=”section”>32.135 iccgen# TOC</h3>

Generate ICC profiles and attach them to frames. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dd> Configure the colorspace that the ICC profile will be generated for. The default value of <code class=”code”>auto</code> infers the value from the input frame’s metadata, defaulting to BT.709/sRGB as appropriate. See the setparams filter for a list of possible values, but note that <code class=”code”>unknown</code> are not valid values for this filter. </dd> <dt><samp class=”option”>force</samp></dt> <dd> If true, an ICC profile will be generated even if it would overwrite an already existing ICC profile. Disabled by default. </dd> </dl>

 <h3 class=”section”>32.136 identity# <a class=”anchor hidden-xs”href=”#toc-identity”

aria-hidden=”true”>TOC</h3>

Obtain the identity score between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max identity score is printed through the logging system. The filter stores the calculated identity scores of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi identity -f null - </pre>

 <h3 class=”section”>32.137 idet# TOC</h3>

Detect video interlacing type. This filter tries to detect if the input frames are interlaced, progressive, top or bottom field first. It will also try to detect fields that are repeated between adjacent frames (a sign of telecine). Single frame detection considers only immediately adjacent frames when classifying each frame. Multiple frame detection incorporates the classification history of previous frames. The filter will log these metadata values: <dl class=”table”> <dt><samp class=”option”>single.current_frame</samp></dt> <dd> Detected type of current frame using single-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class=”option”>single.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using single-frame detection. </dd> <dt><samp class=”option”>multiple.tff</samp></dt> <dd> Cumulative number of frames detected as top field first using multiple-frame detection. </dd> <dt><samp class=”option”>single.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using single-frame detection. </dd> <dt><samp class=”option”>multiple.current_frame</samp></dt> <dd> Detected type of current frame using multiple-frame detection. One of: “tff” (top field first), “bff” (bottom field first), “progressive”, or “undetermined” </dd> <dt><samp class=”option”>multiple.bff</samp></dt> <dd> Cumulative number of frames detected as bottom field first using multiple-frame detection. </dd> <dt><samp class=”option”>single.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using single-frame detection. </dd> <dt><samp class=”option”>multiple.progressive</samp></dt> <dd> Cumulative number of frames detected as progressive using multiple-frame detection. </dd> <dt><samp class=”option”>single.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using single-

frame detection. </dd> <dt><samp class=”option”>multiple.undetermined</samp></dt> <dd> Cumulative number of frames that could not be classified using multiple-frame detection. </dd> <dt><samp class=”option”>repeated.current_frame</samp></dt> <dd> Which field in the current frame is repeated from the last. One of “neither”, “top”, or “bottom”. </dd> <dt><samp class=”option”>repeated.neither</samp></dt> <dd> Cumulative number of frames with no repeated field. </dd> <dt><samp class=”option”>repeated.top</samp></dt> <dd> Cumulative number of frames with the top field repeated from the previous frame’s top field. </dd> <dt><samp class=”option”>repeated.bottom</samp></dt> <dd> Cumulative number of frames with the bottom field repeated from the previous frame’s bottom field. </dd> </dl>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>intl_thres</samp></dt> <dd> Set interlacing threshold. </dd> <dt><samp class=”option”>prog_thres</samp></dt> <dd> Set progressive threshold. </dd> <dt><samp class=”option”>rep_thres</samp></dt> <dd> Threshold for repeated field detection. </dd> <dt><samp class=”option”>half_life</samp></dt> <dd> Number of frames after which a given frame’s contribution to the statistics is halved (i.e., it contributes only 0.5 to its classification). The default of 0 means that all frames seen are given full weight of 1.0 forever. </dd> <dt><samp class=”option”>analyze_interlaced_flag</samp></dt> <dd> When this is not 0 then idet will use the specified number of frames to determine if the interlaced flag is accurate, it will not count undetermined frames. If the flag is found to be accurate it will be used without any further computations, if it is found to be inaccurate it will be cleared without any further computations. This allows inserting the idet filter as a low computational method to clean up the interlaced flag </dd> </dl>

 <h4 class=”subsection”>32.137.1 Examples# TOC</h4>

Inspect the field order of the first 360 frames in a video, in verbose detail: <pre class=”example-preformatted”>ffmpeg -i INPUT -filter:v idet,metadata=mode=print -frames:v 360 -an -f null - </pre> The idet filter will add analysis metadata to each frame, which will then be discarded. At the end, the filter will also print a final report with statistics. <h3 class=”section”>32.138 il# TOC</h3>

Deinterleave or interleave fields. This filter allows one to process interlaced images fields without deinterlacing them. Deinterleaving splits the input frame into 2 fields (so called half pictures). Odd lines

are moved to the top half of the output image, even lines to the bottom half. You can process (filter) them independently and then re-interleave them. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>luma_mode, l</samp></dt> <dt><samp class=”option”>chroma_mode, c</samp></dt> <dt><samp class=”option”>alpha_mode, a</samp></dt> <dd> Available values for <var class=”var”>luma_mode</var>, <var class=”var”>chroma_mode</var> and <var class=”var”>alpha_mode</var> are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Do nothing. </dd> <dt>‘<samp class=”samp”>deinterleave, d</samp>’</dt> <dd> Deinterleave fields, placing one above the other. </dd> <dt>‘<samp class=”samp”>interleave, i</samp>’</dt> <dd> Interleave fields. Reverse the effect of deinterleaving. </dd> </dl> Default value is <code class=”code”>none</code>. </dd> <dt><samp class=”option”>luma_swap, ls</samp></dt> <dt><samp class=”option”>chroma_swap, cs</samp></dt> <dt><samp class=”option”>alpha_swap, as</samp></dt> <dd> Swap luma/chroma/alpha fields. Exchange even & odd lines. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>32.138.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.139 inflate# TOC</h3>

Apply inflate effect to the video. This filter replaces the pixel by the local(3x3) average by taking into account only values higher than the pixel. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane, default is 65535. If 0, plane will remain unchanged. </dd> </dl>

 <h4 class=”subsection”>32.139.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.140 interlace, interlace_vulkan<a class=”anchor hidden-

xs" href="#interlace_002c-interlace_005fvulkan" aria-hidden="true"># TOC</h3>

Simple interlacing filter from progressive contents. This interleaves upper (or lower) lines from odd frames with lower (or upper) lines from even frames, halving the frame rate and preserving image height.   <pre class="example-preformatted"> Original Original New Frame Frame 'j' Frame 'j+1' (tff) ========== ========== ========== Line 0 ----------> Frame 'j' Line 0 Line 1 Line 1 ----> Frame 'j+1' Line 1 Line 2 ----------> Frame 'j' Line 2 Line 3 Line 3 ----> Frame 'j+1' Line 3 ... ... New Frame + 1 will be generated by Frame 'j+2' and Frame 'j+3' and so on </pre>

It accepts the following optional parameters: <dl class="table"> <dt><samp class="option">scan</samp></dt> <dd> This determines whether the interlaced frame is taken from the even (tff - default) or odd (bff) lines of the progressive frame. </dd> <dt><samp class="option">lowpass</samp></dt> <dd> Vertical lowpass filter to avoid twitter interlacing and reduce moire patterns. <dl class="table"> <dt>‘<samp class="samp">0, off</samp>’</dt> <dd> Disable vertical lowpass filter </dd> <dt>‘<samp class="samp">1, linear</samp>’</dt> <dd> Enable linear filter (default) </dd> <dt>‘<samp class="samp">2, complex</samp>’</dt> <dd> Enable complex filter. This will slightly less reduce twitter and moire but better retain detail and subjective sharpness impression. </dd> </dl> </dd> </dl>

 <h3 class="section">32.141 kerndeint# TOC</h3>

Deinterlace input video by applying Donald Graft’s adaptive kernel deinterling. Work on interlaced parts of a video to produce progressive frames. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">thresh</samp></dt> <dd> Set the threshold which affects the filter’s tolerance when determining if a pixel line must be processed. It must be an integer in the range [0,255] and defaults to 10. A value of 0 will result in applying the process on every pixels. </dd> <dt><samp class="option">map</samp></dt> <dd> Paint pixels exceeding the threshold value to white if set to 1. Default is 0. </dd> <dt><samp class="option">order</samp></dt> <dd> Set the fields order. Swap fields if set to 1, leave fields alone if 0. Default is 0. </dd> <dt><samp class="option">sharp</samp></dt> <dd> Enable additional sharpening if set to 1. Default is 0. </dd> <dt><samp class="option">twoway</samp></dt> <dd> Enable twoway sharpening if set to 1. Default is 0. </dd> </dl>

 <h4 class="subsection">32.141.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply default values:
<pre class="example-preformatted">kerndeint=thresh=10:map=0:order=0:sharp=0:twoway=0
</pre>

Enable additional sharpening: <pre class="example-preformatted">kerndeint=sharp=1 </pre>

Paint processed pixels in white: <pre class="example-preformatted">kerndeint=map=1 </pre>

 <h3 class="section">32.142 kirsch# TOC</h3> Apply kirsch operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">32.142.1 Commands# TOC</h4>

This filter supports the all above options as commands.
  <h3 class="section">32.143 lagfun# TOC</h3>

Slowly update darker pixels. This filter makes short flashes of light appear longer. This filter accepts the following options: <dl class="table"> <dt><samp class="option">decay</samp></dt> <dd> Set factor for decaying. Default is .95. Allowed range is from 0 to 1. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">32.143.1 Commands<a class="anchor hidden-xs" href="#Commands-

104" aria-hidden="true"># TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.144 latticepal# TOC</h3>

Convert packed RGB/BGR input to PAL8 with a per-frame palette whose colors are placed on a face-centered cubic (FCC) lattice spanning the RGB cube. Each frame gets its own palette, built only from the lattice colors actually used by that frame. If a frame uses more than 256 lattice colors, they are reduced by iteratively dropping the color whose removal has the least impact (its pixel count times the squared distance to the nearest remaining color) and mapping its pixels to that nearest remaining color. The alpha channel, if present, is ignored unless the <samp class="option">alpha</samp> option is set. The filter accepts the following options: <dl class="table"> <dt><samp class="option">density</samp></dt> <dd> Set the lattice density as the number of lattice steps spanning one color axis (0 to 255). Higher values give a finer lattice and thus more possible colors, making it more likely that the per-frame color reduction has to drop some of them. At a density of <code class="code">7</code> with 256 palette entries no reduction ever happens since the whole lattice then holds exactly 256 points. Allowed range is <code class="code">1</code> to <code class="code">255</code>. Default is <code class="code">20</code>. </dd> <dt><samp class="option">max_colors</samp></dt> <dd> Set the maximum number of palette entries to use. Frames using more lattice colors than this are reduced accordingly. Allowed range is <code class="code">2</code> to <code class="code">256</code>. Default is <code class="code">256</code>. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Quantize the alpha channel too, placing the palette colors on a 4 dimensional D4 lattice over RGBA instead of the FCC lattice over RGB. Inputs without an alpha channel are treated as opaque. With this option the density is limited to <code class="code">64</code>. Default is <code class="code">0</code> (alpha is ignored and the palette is fully opaque). </dd> <dt><samp class="option">refine</samp></dt> <dd> Redistribute the error introduced by dropping colors, which is otherwise uncompensated and can show as discolored patches. Frames that needed no reduction are not changed by this option. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No refinement. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Run a Floyd-Steinberg error diffusion pass over the whole frame against the final palette. Most accurate on average, but redithers everything, including regions whose colors survived. </dd> <dt>‘<samp class="samp">residual</samp>’</dt>

<dd> Diffuse only the residual between each pixel’s first pass quantized color and its final palette color. Pixels of surviving colors that receive no residual are unchanged, keeping the character of the selected dither mode outside the reduced regions. </dd> <dt>‘<samp class="samp">batched</samp>’</dt> <dd> Interleave color removal and rediffusion: drop half of the excess colors, rediffuse the whole frame against the survivors, recount the colors from the actual assignments and repeat. Colors that the dithering can reproduce by mixing become free to drop while extreme colors keep their pixels and are protected; this gives the best quality at the cost of about log2 of the number of dropped colors rediffusion passes. </dd> </dl>

Default is <code class="code">none</code>. </dd> <dt><samp class="option">dither</samp></dt> <dd> Select the dithering mode. Available algorithms are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No dithering. </dd> <dt>‘<samp class="samp">bayer</samp>’</dt> <dd> Ordered 8x8 dithering using the classic swscale dither matrix with its per-channel decorrelation, offsets spanning one lattice period. </dd> <dt>‘<samp class="samp">blue_noise</samp>’</dt> <dd> Void-and-cluster blue noise dithering using three 64x64 masks generated at initialization, one per color channel. </dd> <dt>‘<samp class="samp">floyd_steinberg</samp>’</dt> <dd> Floyd-Steinberg error diffusion. </dd> </dl>

Default is <code class="code">floyd_steinberg</code>. </dd> </dl>

 <h3 class="section">32.145 lenscorrection# TOC</h3>

Correct radial lens distortion This filter can be used to correct for radial distortion as can result from the use of wide angle lenses, and thereby re-rectify the image. To find the right parameters one can use tools available for example as part of opencv or simply trial-and-error. To use opencv use the calibration sample (under samples/cpp) from the opencv sources and extract the k1 and k2 coefficients from the resulting matrix. Note that effectively the same filter is available in the open-source tools Krita and Digikam from the KDE project. In contrast to the vignette filter, which can also be used to compensate lens errors, this filter corrects the distortion of the image, whereas vignette corrects the brightness distribution, so you may want to use both filters together in certain cases, though you will have to take care of ordering, i.e. whether vignetting should be applied before or after lens correction. <h4 class="subsection">32.145.1 Options<span

class=”pull-right”># TOC</h4>

The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>cx</samp></dt> <dd> Relative x-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image width. Default is 0.5. </dd> <dt><samp class=”option”>cy</samp></dt> <dd> Relative y-coordinate of the focal point of the image, and thereby the center of the distortion. This value has a range [0,1] and is expressed as fractions of the image height. Default is 0.5. </dd> <dt><samp class=”option”>k1</samp></dt> <dd> Coefficient of the quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>k2</samp></dt> <dd> Coefficient of the double quadratic correction term. This value has a range [-1,1]. 0 means no correction. Default is 0. </dd> <dt><samp class=”option”>i</samp></dt> <dd> Set interpolation type. Can be <code class=”code”>nearest</code> or <code class=”code”>bilinear</code>. Default is <code class=”code”>nearest</code>. </dd> <dt><samp class=”option”>fc</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Default color is <code class=”code”>black@0</code>. </dd> </dl>

The formula that generates the correction is: <var class=”var”>r_src</var> = <var class=”var”>r_tgt</var> * (1 + <var class=”var”>k1</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^2 + <var class=”var”>k2</var> * (<var class=”var”>r_tgt</var> / <var class=”var”>r_0</var>)^4) where <var class=”var”>r_0</var> is halve of the image diagonal and <var class=”var”>r_src</var> and <var class=”var”>r_tgt</var> are the distances from the focal point in the source and target images, respectively. <h4 class=”subsection”>32.145.2 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.146 lensfun# TOC</h3>

Apply lens correction via the lensfun library (<a class=”url” href=”http://lensfun.sourceforge.net/”>http:/

The <code class=”code”>lensfun</code> filter requires the camera make, camera model, and lens model

to apply the lens correction. The filter will load the lensfun database and

query it to find the corresponding camera and lens entries in the database. As

long as these entries can be found with the given options, the filter can

perform corrections on frames. Note that incomplete strings will result in the

filter choosing the best match with the given options, and the filter will

output the chosen camera and lens models (logged with level "info"). You must

provide the make, camera model, and lens model as they are required.

To obtain a list of available makes and models, leave out one or both of <code class=”code”>make</code> and

<code class=”code”>model</code> options. The filter will send the full list to the log with level <code class=”code”>INFO</code>.

The first column is the make and the second column is the model.

To obtain a list of available lenses, set any values for make and model and leave out the

<code class=”code”>lens_model</code> option. The filter will send the full list of lenses in the log with level

<code class=”code”>INFO</code>. The ffmpeg tool will exit after the list is printed.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>make</samp></dt>

<dd> The make of the camera (for example, "Canon"). This option is required.

</dd>

<dt><samp class=”option”>model</samp></dt>

<dd> The model of the camera (for example, "Canon EOS 100D"). This option is

required.

</dd>

<dt><samp class=”option”>lens_model</samp></dt>

<dd> The model of the lens (for example, "Canon EF-S 18-55mm f/3.5-5.6 IS STM"). This

option is required.

</dd>

<dt><samp class=”option”>db_path</samp></dt>

<dd> The full path to the lens database folder. If not set, the filter will attempt to

load the database from the install path when the library was built. Default is unset.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> The type of correction to apply. The following values are valid options:

<dl class=”table”>

<dt>‘<samp class=”samp”>vignetting</samp>’</dt>

<dd> Enables fixing lens vignetting.

</dd>

<dt>‘<samp class=”samp”>geometry</samp>’</dt>

<dd> Enables fixing lens geometry. This is the default.

</dd>

<dt>‘<samp class=”samp”>subpixel</samp>’</dt>

<dd> Enables fixing chromatic aberrations.

</dd>

<dt>‘<samp class="samp">vig_geo</samp>’</dt>

<dd> Enables fixing lens vignetting and lens geometry.

</dd>

<dt>‘<samp class="samp">vig_subpixel</samp>’</dt>

<dd> Enables fixing lens vignetting and chromatic aberrations.

</dd>

<dt>‘<samp class="samp">distortion</samp>’</dt>

<dd> Enables fixing both lens geometry and chromatic aberrations.

</dd>

<dt>‘<samp class="samp">all</samp>’</dt>

<dd> Enables all possible corrections.

</dd>

</dl>

</dd>

<dt><samp class="option">focal_length</samp></dt>

<dd> The focal length of the image/video (zoom; expected constant for video). For

example, a 18–55mm lens has focal length range of [18–55], so a value in that

range should be chosen when using that lens. Default 18.

</dd>

<dt><samp class="option">aperture</samp></dt>

<dd> The aperture of the image/video (expected constant for video). Note that

aperture is only used for vignetting correction. Default 3.5.

</dd>

<dt><samp class="option">focus_distance</samp></dt>

<dd> The focus distance of the image/video (expected constant for video). Note that

focus distance is only used for vignetting and only slightly affects the

vignetting correction process. If unknown, leave it at the default value (which

is 1000).

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> The scale factor which is applied after transformation. After correction the

video is no longer necessarily rectangular. This parameter controls how much of

the resulting image is visible. The value 0 means that a value will be chosen

automatically such that there is little or no unmapped area in the output

image. 1.0 means that no additional scaling is done. Lower values may result

in more of the corrected image being visible, while higher values may avoid

unmapped areas in the output.

</dd>

<dt><samp class="option">target_geometry</samp></dt>

<dd> The target geometry of the output image/video. The following values are valid

options:

<dl class="table">

<dt>‘<samp class="samp">rectilinear (default)</samp>’</dt>

<dt>‘<samp class="samp">fisheye</samp>’</dt>

<dt>‘<samp class="samp">panoramic</samp>’</dt>

<dt>‘<samp class="samp">equirectangular</samp>’</dt>

<dt>‘<samp class="samp">fisheye_orthographic</samp>’</dt>

<dt>‘<samp class="samp">fisheye_stereographic</samp>’</dt>

<dt>‘<samp class="samp">fisheye_equisolid</samp>’</dt>

<dt>‘<samp class="samp">fisheye_thoby</samp>’</dt>

</dl>

</dd>

<dt><samp class="option">reverse</samp></dt>

<dd> Apply the reverse of image correction (instead of correcting distortion, apply

it).

</dd>

<dt><samp class="option">interpolation</samp></dt>

<dd> The type of interpolation used when correcting distortion. The following values

are valid options:

<dl class="table">

<dt>‘<samp class="samp">nearest</samp>’</dt>

<dt>‘<samp class="samp">linear (default)</samp>’</dt>

<dt>‘<samp class="samp">lanczos</samp>’</dt>

</dl>

</dd>

</dl>

 <h4 class="subsection">32.146.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply lens correction with make "Canon", camera model "Canon EOS 100D", and lens model "Canon EF-S 18-55mm f/3.5-5.6 IS STM" with focal length of "18" and aperture of "8.0".

 <pre class="example-preformatted">ffmpeg -i input.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon

EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8 -c:v h264
-b:v 8000k output.mov </pre>

Apply the same as before, but only for the first 5 seconds of video.

 <pre class="example-preformatted">ffmpeg -i in-
put.mov -vf lensfun=make=Canon:model="Canon EOS 100D":lens_model="Canon
EF-S 18-55mm f/3.5-5.6 IS STM":focal_length=18:aperture=8:enable='lte(t\,5)'
-c:v h264 -b:v 8000k output.mov </pre>

 <h3 class="section">32.147 lcevc<span class="pull-
right">#
TOC</h3>

Low Complexity Enhancement Video Codec filter based on liblcevc_dec
(https://github.com/v-
novaltd/LCEVCdec). <h3
class="section">32.148 libplacebo<a class="anchor
hidden-xs" href="#libplacebo" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-libplacebo" aria-hidden="true">TOC</h3>

Flexible GPU-accelerated processing filter based on libplacebo (<a
class="url" href="https://code.videolan.org/videolan/libplacebo">https://code.videolan.org/videolan/libplac
  <h4 class="subsection">32.148.1 Op-
tions<a class="anchor hidden-xs" href="#Options-
39" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Options-39" aria-hidden="true">TOC</h4>

The options for this filter are divided into the following sections: <a
name="Output-mode"> <h4 class="subsubsection">32.148.1.1 Output
mode<a class="anchor hidden-xs" href="#Output-
mode" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Output-mode" aria-hidden="true">TOC</h4> These
options control the overall output mode. By default, libplacebo will try
to preserve the source colorimetry and size as best as it can, but it will
apply any embedded film grain, dolby vision metadata or anamorphic
SAR present in source frames. <dl class="table"> <dt><samp
class="option">inputs</samp></dt> <dd> Set the number of in-
puts. This can be used, alongside the <code class="code">idx</code>
variable, to allow placing/blending multiple inputs inside the output
frame. This effectively enables functionality similar to <a class="ref"
href="#hstack">hstack, overlay,
etc. </dd> <dt><samp class="option">w</samp></dt>
<dt><samp class="option">h</samp></dt> <dd> Set the output
video dimension expression. Default values are <code class="code">iw</code>
and <code class="code">ih</code>. Allows for the same ex-
pressions as the scale filter.

</dd> <dt><samp class="option">crop_x</samp></dt> <dt><samp class="option">crop_y</samp></dt> <dd> Set the input crop x/y expressions, default values are <code class="code">(iw-cw)/2</code> and <code class="code">(ih-ch)/2</code>. </dd> <dt><samp class="option">crop_w</samp></dt> <dt><samp class="option">crop_h</samp></dt> <dd> Set the input crop width/height expressions, default values are <code class="code">iw</code> and <code class="code">ih</code>. </dd> <dt><samp class="option">pos_x</samp></dt> <dt><samp class="option">pos_y</samp></dt> <dd> Set the output placement x/y expressions, default values are <code class="code">(ow-pw)/2</code> and <code class="code">(oh-ph)/2</code>. </dd> <dt><samp class="option">pos_w</samp></dt> <dt><samp class="option">pos_h</samp></dt> <dd> Set the output placement width/height expressions, default values are <code class="code">ow</code> and <code class="code">oh</code>. </dd> <dt><samp class="option">rotate</samp></dt> <dd> Rotate the input frame clockwise by the specified angle. <dl class="table"> <dt>‘<samp class="samp">0, 360</samp>’</dt> <dt>‘<samp class="samp">90</samp>’</dt> <dt>‘<samp class="samp">180</samp>’</dt> <dt>‘<samp class="samp">270</samp>’</dt> </dl>

</dd> <dt><samp class="option">fps</samp></dt> <dd> Set the output frame rate. This can be rational, e.g. <code class="code">60000/1001</code>. If set to the special string <code class="code">none</code> (the default), input timestamps will instead be passed through to the output unmodified. Otherwise, the input video frames will be interpolated as necessary to rescale the video to the specified target framerate, in a manner as determined by the <samp class="option">frame_mixer</samp> option. </dd> <dt><samp class="option">format</samp></dt> <dd> Set the output format override. If unset (the default), frames will be output in the same format as the respective input frames. Otherwise, format conversion will be performed. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. Note that <samp class="option">force_divisible_by</samp> also works with <code class="code">fit_sense=constraint</code>. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> If enabled, output frames will always have a pixel aspect ratio of 1:1. If disabled (the default), any aspect ratio mismatches, including those from e.g. anamorphic video sources, are forwarded to the output pixel aspect ratio. </dd> <dt><samp class="option">normalize_sar</samp></dt> <dd> Like <samp class="option">reset_sar</samp>, but instead of stretching the video content to fill the new output aspect ratio, the content is instead padded or cropped as necessary. Mutually exclusive with <samp class="option">fit_mode</samp>. Disabled by default.

 </dd> <dt><samp class="option">pad_crop_ratio</samp></dt> <dd> Specifies a ratio (between <code class="code">0.0</code> and <code class="code">1.0</code>) between padding and cropping when the input aspect ratio does not match the output aspect ratio and <samp class="option">normalize_sar</samp> is in effect. The default of <code class="code">0.0</code> always pads the content with black borders, while a value of <code class="code">1.0</code> always crops off parts of the content. Intermediate values are possible, leading to a mix of the two approaches. </dd> <dt><samp class="option">fit_mode</samp></dt> <dd> Specify the content fit strategy according to a list of predefined modes. Determines how the input image is to be placed inside the destination crop rectangle (as defined by <code class="code">pos_x/y</code> and <code class="code">pos_w/h</code>). The names and their implementations are taken from the CSS ’object-fit’ property. Note that this option is mutually exclusive with <samp class="option">normalize_sar</samp>. Defaults to <code class="code">fill</code>. Valid values are: <dl class="table"> <dt>‘<samp class="samp">fill</samp>’</dt> <dd> Stretch the input to the output rectangle, ignoring aspect ratio mismatches. Note that unless <samp class="option">reset_sar</samp> is also enabled, the output will still have the correct pixel aspect ratio tagged. </dd> <dt>‘<samp class="samp">contain</samp>’</dt> <dd> Scale the input to fit inside the output, preserving aspect ratio by padding. Equivalent to <samp class="option">normalize_sar</samp> with <samp class="option">pad_crop_ratio</samp> set to <code class="code">0.0</code>. </dd> <dt>‘<samp class="samp">cover</samp>’</dt> <dd> Scale the input to fill the output, preserving aspect ratio by cropping. Equivalent to <samp class="option">normalize_sar</samp> with <samp class="option">pad_crop_ratio</samp> set to <code class="code">1.0</code>. </dd> <dt>‘<samp class="samp">none, place</samp>’</dt> <dd> Don’t scale the input. The input will be placed inside the output rectangle at its natural size; which may result in additional padding or cropping. </dd> <dt>‘<samp class="samp">scale_down</samp>’</dt> <dd> Scale the input down as much as needed to fit inside the output. Equivalent to either <code class="code">contain</code> or <code class="code">none</code>, depending on whether the input is larger than the output or not. </dd> </dl>

</dd> <dt><samp class="option">fit_sense</samp></dt> <dd> When <samp class="option">fit_mode</samp> is in use, this option controls how the fit strategy is applied against the specified output resolution. Mutually exclusive with <samp class="option">force_original_aspect_ratio</samp>. Valid values are: <dl class="table"> <dt>‘<samp class="samp">target</samp>’</dt> <dd> The computed output resolution is taken as the exact size of the output frame. This is the default behavior. </dd> <dt>‘<samp

class=”samp”>constraint</samp>’</dt> <dd> The computed output resolution is a size reference against which the fit mode is applied, enlarging or decreasing the true frame size as needed to fit the content. </dd> </dl>

</dd> <dt><samp class=”option”>fillcolor</samp></dt> <dd> Set the color used to fill the output area not covered by the output image, for example as a result of <samp class=”option”>normalize_sar</samp>. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. Defaults to <code class=”code”>black@0</code>. </dd> <dt><samp class=”option”>corner_rounding</samp></dt> <dd> Render frames with rounded corners. The value, given as a float ranging from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>, indicates the relative degree of rounding, from fully square to fully circular. In other words, it gives the radius divided by half the smaller side length. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>lut</samp></dt> <dd> Specifies a custom LUT (in Adobe .cube format) to apply to the colors as part of color conversion. The exact interpretation depends on the value of <samp class=”option”>lut_type</samp>. </dd> <dt><samp class=”option”>lut_type</samp></dt> <dd> Controls the interpretation of color values fed to and from the LUT specified as <samp class=”option”>lut</samp>. Valid values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Chooses the interpretation of the LUT automatically from tagged metadata, and otherwise falls back to ‘<samp class=”samp”>native</samp>’. (Default) </dd> <dt>‘<samp class=”samp”>native</samp>’</dt> <dd> Applied to raw image contents in its native RGB colorspace (nonlinear light), before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>normalized</samp>’</dt> <dd> Applied to the normalized RGB image contents, in linear light, before conversion to the output color space. </dd> <dt>‘<samp class=”samp”>conversion</samp>’</dt> <dd> Fully replaces the conversion from the image color space to the output color space. If such a LUT is present, it has the highest priority, and overrides any ICC profiles, as well as options related to tone mapping and output colorimetry (<samp class=”option”>color_primaries</samp>, <samp class=”option”>color_trc</samp>). </dd> </dl>

</dd> <dt><samp class=”option”>extra_opts</samp></dt> <dd> Pass extra libplacebo internal configuration options. These can be specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs separated by ’:’. The following example shows how to configure a custom filter kernel ("EWA LanczosSharp") and use it to double the input image resolution: <pre class=”example-preformatted”>-vf "libplacebo=w=iw*2:h=ih*2:extra_opts='upscaler=custom\:upscaler_preset=ewa_

</pre>

</dd> <dt><samp class=”option”>shader_cache</samp></dt> <dd> File path of a cache directory that libplacebo will use to store and load cached shader objects. This cache is not cleaned up automatically. If the path does not end in a directory separator, the generated filenames will be effectively prefixed by the last path component. All directories must already exist.   <pre class=”example-preformatted”>-vf "libplacebo=shader_cache=/tmp/pl-shader-" </pre>

</dd> <dt><samp class=”option”>inherit_device</samp></dt> <dd> Take the Vulkan device from the hardware frames arriving on the input, instead of creating a private one from the filter’s own hardware device. This is useful when the input frames originate on another API (for example CUDA frames uploaded to Vulkan by <code class=”code”>hwupload</code>) and the filter’s device is not itself a Vulkan device: libplacebo would otherwise spin up a separate Vulkan device that the input frames do not belong to, and the frames could not be mapped. When enabled, the filter requires Vulkan hardware frames on the input and will fail negotiation otherwise. Disabled by default.   <pre class=”example-preformatted”>-init_hw_device vulkan=vk:0 -init_hw_device cuda@vk -filter_hw_device cuda0 ... -vf "hwupload=derive_device=vulkan,libplacebo=inherit_device=1" </pre>

</dd> <dt><samp class=”option”>colorspace</samp></dt> <dt><samp class=”option”>color_primaries</samp></dt> <dt><samp class=”option”>color_trc</samp></dt> <dt><samp class=”option”>range</samp></dt> <dt><samp class=”option”>chroma_location</samp><
<dd> Configure the colorspace that output frames will be delivered in. For any value other than the default <code class=”code”>auto</code>, conversion will be performed. With <code class=”code”>auto</code>, <samp class=”option”>color_primaries</samp> and <samp class=”option”>color_trc</samp> keep the values of the input frames, while <samp class=”option”>colorspace</samp>, <samp class=”option”>range</samp> and <samp class=”option”>chroma_location</samp> use the values negotiated with the filtergraph, which are those of the input unless a following filter requires others. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> Choose the desired output alpha mode, when the output format has an alpha channel. See the setparams filter for a list of possible values. </dd> <dt><samp class=”option”>apply_filmgrain</samp></dt> <dd> Apply film grain (e.g. AV1 or H.274) if present in source frames, and strip it from the output. Enabled by default. </dd> <dt><samp class=”option”>apply_dolbyvision</samp></dt> <dd> Apply Dolby Vision RPU metadata if present in source frames, and strip it from the output. Enabled by default. Note that Dolby Vision will always output BT.2020+PQ, overriding the usual input frame metadata. These will also be picked as the

values of <code class="code">auto</code> for the respective frame output options. </dd> </dl>

In addition to the expression constants documented for the scale filter, the <samp class="option">crop_w</samp>, <samp class="option">crop_h</samp>, <samp class="option">crop_x</samp>, <samp class="option">crop_y</samp>, <samp class="option">pos_w</samp>, <samp class="option">pos_h</samp>, <samp class="option">pos_x</samp> and <samp class="option">pos_y</samp> options can also contain the following constants: <dl class="table"> <dt><samp class="option">in_idx, idx</samp></dt> <dd> The (0-based) numeric index of the currently active input stream. </dd> <dt><samp class="option">crop_w, cw</samp></dt> <dt><samp class="option">crop_h, ch</samp></dt> <dd> The computed values of <samp class="option">crop_w</samp> and <samp class="option">crop_h</samp>. </dd> <dt><samp class="option">pos_w, pw</samp></dt> <dt><samp class="option">pos_h, ph</samp></dt> <dd> The computed values of <samp class="option">pos_w</samp> and <samp class="option">pos_h</samp>. </dd> <dt><samp class="option">in_t, t</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class="option">out_t, ot</samp></dt> <dd> The input frame timestamp, in seconds. NAN if input timestamp is unknown. </dd> <dt><samp class="option">n</samp></dt> <dd> The input frame number, starting with 0. </dd> </dl>

 <h4 class="subsubsection">32.148.1.2 Scaling# TOC</h4> The options in this section control how libplacebo performs upscaling and (if necessary) downscaling. Note that libplacebo will always internally operate on 4:4:4 content, so any sub-sampled chroma formats such as <code class="code">yuv420p</code> will necessarily be upsampled and downsampled as part of the rendering process. That means scaling might be in effect even if the source and destination resolution are the same. <dl class="table"> <dt><samp class="option">upscaler</samp></dt> <dt><samp class="option">downscaler</samp></dt> <dd> Configure the filter kernel used for upscaling and downscaling. The respective defaults are <code class="code">spline36</code> and <code class="code">mitchell</code>. For a full list of possible values, pass <code class="code">help</code> to these options. The most important values are: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Forces the use of built-in GPU texture sampling (typically bilinear). Extremely fast but poor quality, especially when downscaling. </dd> <dt>‘<samp class="samp">bilinear</samp>’</dt> <dd> Bilinear interpolation. Can generally be done for free on GPUs, except when doing so would lead to aliasing. Fast and low quality. </dd> <dt>‘<samp

class=”samp”>nearest</samp>’</dt> <dd> Nearest-neighbour interpolation. Sharp but highly aliasing. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Algorithm that looks visually similar to nearest-neighbour interpolation but tries to preserve pixel aspect ratio. Good for pixel art, since it results in minimal distortion of the artistic appearance. </dd> <dt>‘<samp class=”samp”>lanczos</samp>’</dt> <dd> Standard sinc-sinc interpolation kernel. </dd> <dt>‘<samp class=”samp”>spline36</samp>’</dt> <dd> Cubic spline approximation of lanczos. No difference in performance, but has very slightly less ringing. </dd> <dt>‘<samp class=”samp”>ewa_lanczos</samp>’</dt> <dd> Elliptically weighted average version of lanczos, based on a jinc-jinc kernel. This is also popularly referred to as just "Jinc scaling". Slow but very high quality. </dd> <dt>‘<samp class=”samp”>gaussian</samp>’</dt> <dd> Gaussian kernel. Has certain ideal mathematical properties, but subjectively very blurry. </dd> <dt>‘<samp class=”samp”>mitchell</samp>’</dt> <dd> Cubic BC spline with parameters recommended by Mitchell and Netravali. Very little ringing. </dd> </dl>

</dd> <dt><samp class=”option”>frame_mixer</samp></dt> <dd> Controls the kernel used for mixing frames temporally. The default value is <code class=”code”>none</code>, which disables frame mixing. For a full list of possible values, pass <code class=”code”>help</code> to this option. The most important values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disables frame mixing, giving a result equivalent to "nearest neighbour" semantics. </dd> <dt>‘<samp class=”samp”>oversample</samp>’</dt> <dd> Oversamples the input video to create a "Smooth Motion"-type effect: if an output frame would exactly fall on the transition between two video frames, it is blended according to the relative overlap. This is the recommended option whenever preserving the original subjective appearance is desired. </dd> <dt>‘<samp class=”samp”>mitchell_clamp</samp>’</dt> <dd> Larger filter kernel that smoothly interpolates multiple frames in a manner designed to eliminate ringing and other artefacts as much as possible. This is the recommended option wherever maximum visual smoothness is desired. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linear blend/fade between frames. Especially useful for constructing e.g. slideshows. </dd> </dl>

</dd> <dt><samp class=”option”>antiringing</samp></dt> <dd> Enables anti-ringing (for non-EWA filters). The value (between <code class=”code”>0.0</code> and <code class=”code”>1.0</code>) configures the strength of the anti-ringing algorithm. May increase aliasing if set too high. Disabled by default. </dd> <dt><samp class=”option”>sigmoid</samp></dt> <dd> Enable sigmoidal compression during upscaling. Reduces ringing

slightly. Enabled by default. </dd> </dl>

 <h4 class="subsubsection">32.148.1.3 Deinterlacing# TOC</h4> Deinterlacing is automatically supported when frames are tagged as interlaced, however frames are not deinterlaced unless a deinterlacing algorithm is chosen. <dl class="table"> <dt><samp class="option">deinterlace</samp></dt> <dd> The the deinterlacing algorithm to use. <dl class="table"> <dt>‘<samp class="samp">weave</samp>’</dt> <dd> No deinterlacing, weave fields together into a single frame. This is the default. </dd> <dt>‘<samp class="samp">bob</samp>’</dt> <dd> Naive bob deinterlacing, simply repeat each field line twice. </dd> <dt>‘<samp class="samp">yadif</samp>’</dt> <dd> Yet another deinterlacing filter. See the yadif filter for more details. </dd> <dt>‘<samp class="samp">bwdif</samp>’</dt> <dd> Bob weaver deinterlacing filter. See the bwdif filter for more details. </dd> </dl>

</dd> <dt><samp class="option">skip_spatial_check</samp></dt> <dd> Skip the spatial deinterlacing check when using <code class="code">yadif</code> deinterlacing. </dd> <dt><samp class="option">send_fields</samp></dt> <dd> Output a frame for each field, rather than for each frame. Note that this will always double the tagged output frame rate, even if the input does not contain any interlaced frames. Disabled by default. </dd> </dl>

 <h4 class="subsubsection">32.148.1.4 Debanding# TOC</h4> Libplacebo comes with a built-in debanding filter that is good at counteracting many common sources of banding and blocking. Turning this on is highly recommended whenever quality is desired. <dl class="table"> <dt><samp class="option">deband</samp></dt> <dd> Enable (fast) debanding algorithm. Disabled by default. </dd> <dt><samp class="option">deband_iterations</samp></dt> <dd> Number of deband iterations of the debanding algorithm. Each iteration is performed with progressively increased radius (and diminished threshold). Recommended values are in the range <code class="code">1</code> to <code class="code">4</code>. Defaults to <code class="code">1</code>. </dd> <dt><samp class="option">deband_threshold</samp></dt> <dd> Debanding filter strength. Higher numbers lead to more aggressive debanding. Defaults to <code class="code">4.0</code>. </dd> <dt><samp class="option">deband_radius</samp></dt> <dd> Debanding filter radius. A higher radius is better for slow

gradients, while a lower radius is better for steep gradients. Defaults to <code class=”code”>16.0</code>. </dd> <dt><samp class=”option”>deband_grain</samp></dt> <dd> Amount of extra output grain to add. Helps hide imperfections. Defaults to <code class=”code”>6.0</code>. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.5 Color adjustment# TOC</h4> A collection of subjective color controls. Not very rigorous, so the exact effect will vary somewhat depending on the input primaries and colorspace. <dl class=”table”> <dt><samp class=”option”>brightness</samp></dt> <dd> Brightness boost, between <code class=”code”>-1.0</code> and <code class=”code”>1.0</code>. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>contrast</samp></dt> <dd> Contrast gain, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Saturation gain, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>hue</samp></dt> <dd> Hue shift in radians, between <code class=”code”>-3.14</code> and <code class=”code”>3.14</code>. Defaults to <code class=”code”>0.0</code>. This will rotate the UV subvector, defaulting to BT.709 coefficients for RGB inputs. </dd> <dt><samp class=”option”>gamma</samp></dt> <dd> Gamma adjustment, between <code class=”code”>0.0</code> and <code class=”code”>16.0</code>. Defaults to <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>temperature</samp></dt> <dd> Color temperature adjustment. Lower values make the output warmer/redder, down to <code class=”code”>1667</code>, while higher values make the output cooler/bluer, up to <code class=”code”>25000</code>. Defaults to <code class=”code”>6500</code> (neutral white). </dd> <dt><samp class=”option”>cones</samp></dt> <dd> Cone model to use for color blindness simulation. Accepts any combination of <code class=”code”>l</code>, <code class=”code”>m</code> and <code class=”code”>s</code>. Here are some examples: <dl class=”table”> <dt>‘<samp class=”samp”>m</samp>’</dt> <dd> Deuteranomaly / deuteranopia (affecting 3%-4% of the population) </dd> <dt>‘<samp class=”samp”>l</samp>’</dt> <dd> Protanomaly / protanopia (affecting 1%-2% of the population) </dd> <dt>‘<samp class=”samp”>l+m</samp>’</dt> <dd> Monochromacy (very rare) </dd> <dt>‘<samp class=”samp”>l+m+s</samp>’</dt> <dd> Achromatopsy (complete loss of daytime vision, extremely rare) </dd> </dl>

</dd> <dt><samp class=”option”>cone-strength</samp></dt> <dd> Gain

factor for the cones specified by <code class=”code”>cones</code>, between <code class=”code”>0.0</code> and <code class=”code”>10.0</code>. A value of <code class=”code”>1.0</code> results in no change to color vision. A value of <code class=”code”>0.0</code> (the default) simulates complete loss of those cones. Values above <code class=”code”>1.0</code> result in exaggerating the differences between cones, which may help compensate for reduced color vision. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.6 Peak detection# TOC</h4> To help deal with sources that only have static HDR10 metadata (or no tagging whatsoever), libplacebo uses its own internal frame analysis compute shader to analyze source frames and adapt the tone mapping function in realtime. If this is too slow, or if exactly reproducible frame-perfect results are needed, it’s recommended to turn this feature off. <dl class=”table”> <dt><samp class=”option”>peak_detect</samp></dt> <dd> Enable HDR peak detection. Ignores static MaxCLL/MaxFALL values in favor of dynamic detection from the input. Note that the detected values do not get written back to the output frames, they merely guide the internal tone mapping process. Enabled by default. </dd> <dt><samp class=”option”>smoothing_period</samp></dt> <dd> Peak detection smoothing period, between <code class=”code”>0.0</code> and <code class=”code”>1000.0</code>. Higher values result in peak detection becoming less responsive to changes in the input. Defaults to <code class=”code”>20.0</code>. </dd> <dt><samp class=”option”>scene_threshold_low</samp></dt> <dt><samp class=”option”>scene_threshold_high</
<dd> Lower and upper thresholds for scene change detection. Expressed in a logarithmic scale between <code class=”code”>0.0</code> and <code class=”code”>100.0</code>. Default to <code class=”code”>1.0</code> and <code class=”code”>3.0</code>, respectively. Setting either to a negative value disables this functionality. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Which percentile of the frame brightness histogram to use as the source peak for tone-mapping. Defaults to <code class=”code”>99.995</code>, a fairly conservative value. Setting this to <code class=”code”>100.0</code> disables frame histogram measurement and instead uses the true peak brightness for tone-mapping. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.7 Tone mapping# TOC</h4> The options in this section control how libplacebo performs tone-mapping and gamut-mapping when dealing with mismatches between wide-gamut or HDR content. In general, libplacebo relies on accurate source tagging and

mastering display gamut information to produce the best results. <dl class=”table”> <dt><samp class=”option”>gamut_mode</samp></dt> <dd> How to handle out-of-gamut colors that can occur as a result of colorimetric gamut mapping. <dl class=”table”> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Do nothing, simply clip out-of-range colors to the RGB volume. Low quality but extremely fast. </dd> <dt>‘<samp class=”samp”>perceptual</samp>’</dt> <dd> Perceptually soft-clip colors to the gamut volume. This is the default. </dd> <dt>‘<samp class=”samp”>relative</samp>’</dt> <dd> Relative colorimetric hard-clip. Similar to <code class=”code”>perceptual</code> but without the soft knee. </dd> <dt>‘<samp class=”samp”>saturation</samp>’</dt> <dd> Saturation mapping, maps primaries directly to primaries in RGB space. Not recommended except for artificial computer graphics for which a bright, saturated display is desired. </dd> <dt>‘<samp class=”samp”>absolute</samp>’</dt> <dd> Absolute colorimetric hard-clip. Performs no adjustment of the white point. </dd> <dt>‘<samp class=”samp”>desaturate</samp>’</dt> <dd> Hard-desaturates out-of-gamut colors towards white, while preserving the luminance. Has a tendency to distort the visual appearance of bright objects. </dd> <dt>‘<samp class=”samp”>darken</samp>’</dt> <dd> Linearly reduces content brightness to preserves saturated details, followed by clipping the remaining out-of-gamut colors. </dd> <dt>‘<samp class=”samp”>warn</samp>’</dt> <dd> Highlight out-of-gamut pixels (by inverting/marking them). </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly reduces chromaticity of the entire image to make it fit within the target color volume. Be careful when using this on BT.2020 sources without proper mastering metadata, as doing so will lead to excessive desaturation. </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping</samp></dt> <dd> Tonemapping algorithm to use. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Automatic selection based on internal heuristics. This is the default. </dd> <dt>‘<samp class=”samp”>clip</samp>’</dt> <dd> Performs no tone-mapping, just clips out-of-range colors. Retains perfect color accuracy for in-range colors but completely destroys out-of-range information. Does not perform any black point adaptation. Not configurable. </dd> <dt>‘<samp class=”samp”>st2094-40</samp>’</dt> <dd> EETF from SMPTE ST 2094-40 Annex B, which applies the Bezier curves from HDR10+ dynamic metadata based on Bezier curves to perform tone-mapping. The OOTF used is adjusted based on the ratio between the targeted and actual display peak luminances. </dd> <dt>‘<samp class=”samp”>st2094-10</samp>’</dt> <dd> EETF from SMPTE ST 2094-10 Annex B.2, which takes into account the input signal average luminance in addition to the maximum/minimum. The configurable contrast parameter influences the slope of the linear out-

put segment, defaulting to <code class=”code”>1.0</code> for no increase/decrease in contrast. Note that this does not currently include the subjective gain/offset/gamma controls defined in Annex B.3. </dd> <dt>‘<samp class=”samp”>bt.2390</samp>’</dt> <dd> EETF from the ITU-R Report BT.2390, a hermite spline roll-off with linear segment. The knee point offset is configurable. Note that this parameter defaults to <code class=”code”>1.0</code>, rather than the value of <code class=”code”>0.5</code> from the ITU-R spec. </dd> <dt>‘<samp class=”samp”>bt.2446a</samp>’</dt> <dd> EETF from ITU-R Report BT.2446, method A. Designed for well-mastered HDR sources. Can be used for both forward and inverse tone mapping. Not configurable. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Simple spline consisting of two polynomials, joined by a single pivot point. The parameter gives the pivot point (in PQ space), defaulting to <code class=”code”>0.30</code>. Can be used for both forward and inverse tone mapping. </dd> <dt>‘<samp class=”samp”>reinhard</samp>’</dt> <dd> Simple non-linear, global tone mapping algorithm. The parameter specifies the local contrast coefficient at the display peak. Essentially, a parameter of <code class=”code”>0.5</code> implies that the reference white will be about half as bright as when clipping. Defaults to <code class=”code”>0.5</code>, which results in the simplest formulation of this function. </dd> <dt>‘<samp class=”samp”>mobius</samp>’</dt> <dd> Generalization of the reinhard tone mapping algorithm to support an additional linear slope near black. The tone mapping parameter indicates the trade-off between the linear section and the non-linear section. Essentially, for a given parameter <var class=”var”>x</var>, every color value below <var class=”var”>x</var> will be mapped linearly, while higher values get non-linearly tone-mapped. Values near <code class=”code”>1.0</code> make this curve behave like <code class=”code”>clip</code>, while values near <code class=”code”>0.0</code> make this curve behave like <code class=”code”>reinhard</code>. The default value is <code class=”code”>0.3</code>, which provides a good balance between colorimetric accuracy and preserving out-of-gamut details. </dd> <dt>‘<samp class=”samp”>hable</samp>’</dt> <dd> Piecewise, filmic tone-mapping algorithm developed by John Hable for use in Uncharted 2, inspired by a similar tone-mapping algorithm used by Kodak. Popularized by its use in video games with HDR rendering. Preserves both dark and bright details very well, but comes with the drawback of changing the average brightness quite significantly. This is sort of similar to <code class=”code”>reinhard</code> with parameter <code class=”code”>0.24</code>. </dd> <dt>‘<samp class=”samp”>gamma</samp>’</dt> <dd> Fits a gamma (power) function to transfer between the source and target color spaces, effectively resulting in a perceptual hard-knee joining two roughly linear sections. This preserves details at all scales fairly accurately, but can result

in an image with a muted or dull appearance. The parameter is used as the cutoff point, defaulting to <code class=”code”>0.5</code>. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Linearly stretches the input range to the output range, in PQ space. This will preserve all details accurately, but results in a significantly different average brightness. Can be used for inverse tone-mapping in addition to regular tone-mapping. The parameter can be used as an additional linear gain coefficient (defaulting to <code class=”code”>1.0</code>). </dd> </dl>

</dd> <dt><samp class=”option”>tonemapping_param</samp></dt> <dd> For tunable tone mapping functions, this parameter can be used to fine-tune the curve behavior. Refer to the documentation of <code class=”code”>tonemapping</code>. The default value of <code class=”code”>0.0</code> is replaced by the curve’s preferred default setting. </dd> <dt><samp class=”option”>inverse_tonemapping</samp></dt> <dd> If enabled, this filter will also attempt stretching SDR signals to fill HDR output color volumes. Disabled by default. </dd> <dt><samp class=”option”>tonemapping_lut_size</samp></dt> <dd> Size of the tone-mapping LUT, between <code class=”code”>2</code> and <code class=”code”>1024</code>. Defaults to <code class=”code”>256</code>. Note that this figure is squared when combined with <code class=”code”>peak_detect</code>. </dd> <dt><samp class=”option”>contrast_recovery</samp></dt> <dd> Contrast recovery strength. If set to a value above <code class=”code”>0.0</code>, the source image will be divided into high-frequency and low-frequency components, and a portion of the high-frequency image is added back onto the tone-mapped output. May cause excessive ringing artifacts for some HDR sources, but can improve the subjective sharpness and detail left over in the image after tone-mapping. Defaults to <code class=”code”>0.30</code>. </dd> <dt><samp class=”option”>contrast_smoothness</samp></dt> <dd> Contrast recovery lowpass kernel size. Defaults to <code class=”code”>3.5</code>. Increasing or decreasing this will affect the visual appearance substantially. Has no effect when <code class=”code”>contrast_recovery</code> is disabled. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.8 Dithering# TOC</h4> By default, libplacebo will dither whenever necessary, which includes rendering to any integer format below 16-bit precision. It’s recommended to always leave this on, since not doing so may result in visible banding in the output, even if the <code class=”code”>debanding</code> filter is enabled. If maximum performance is needed, use <code class=”code”>ordered_fixed</code> instead of disabling dithering. <dl class=”table”> <dt><samp class=”option”>dithering</samp></dt> <dd> Dithering method to use. Accepts the following values: <dl class=”table”> <dt>‘<samp

class=”samp”>none</samp>’</dt> <dd> Disables dithering completely. May result in visible banding. </dd> <dt>‘<samp class=”samp”>blue</samp>’</dt> <dd> Dither with pseudo-blue noise. This is the default. </dd> <dt>‘<samp class=”samp”>ordered</samp>’</dt> <dd> Tunable ordered dither pattern. </dd> <dt>‘<samp class=”samp”>ordered_fixed</samp>’</dt> <dd> Faster ordered dither with a fixed size of <code class=”code”>6</code>. Texture-less. </dd> <dt>‘<samp class=”samp”>white</samp>’</dt> <dd> Dither with white noise. Texture-less. </dd> </dl>

</dd> <dt><samp class=”option”>dither_lut_size</samp></dt> <dd> Dither LUT size, as log base2 between <code class=”code”>1</code> and <code class=”code”>8</code>. Defaults to <code class=”code”>6</code>, corresponding to a LUT size of <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>dither_temporal</samp></dt> <dd> Enables temporal dithering. Disabled by default. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.9 Custom shaders# TOC</h4> libplacebo supports a number of custom shaders based on the mpv .hook GLSL syntax. A collection of such shaders can be found here: https://github.com/mpv-player/mpv/wiki/User-Scripts#user-shaders A full description of the mpv shader format is beyond the scope of this section, but a summary can be found here: https://mpv.io/manual/master/#options-glsl-shader <dl class=”table”> <dt><samp class=”option”>custom_shader_path</samp></dt> <dd> Specifies a path to a custom shader file to load at runtime. </dd> <dt><samp class=”option”>custom_shader_bin</samp></dt> <dd> Specifies a complete custom shader as a raw string. </dd> </dl>

 <h4 class=”subsubsection”>32.148.1.10 Debugging / performance# TOC</h4> All of the options in this section default off. They may be of assistance when attempting to squeeze the maximum performance at the cost of quality. <dl class=”table”> <dt><samp class=”option”>skip_aa</samp></dt> <dd> Disable anti-aliasing when downscaling. </dd> <dt><samp class=”option”>disable_linear</samp></dt> <dd> Disable linear light scaling. </dd> <dt><samp class=”option”>disable_builtin</samp></dt>

<dd> Disable built-in GPU sampling (forces LUT). </dd> <dt><samp class="option">disable_fbos</samp></dt> <dd> Forcibly disable FBOs, resulting in loss of almost all functionality, but offering the maximum possible speed. </dd> </dl>

 <h4 class="subsection">32.148.2 Commands# TOC</h4> This filter supports almost all of the above options as commands. The options negotiated with the filtergraph, <samp class="option">format</samp>, <samp class="option">colorspace</samp>, <samp class="option">range</samp>, <samp class="option">chroma_location</samp> and <samp class="option">alpha_mode</samp>, are fixed once the filtergraph is configured and can not be changed with commands. <h4 class="subsection">32.148.3 Examples# TOC</h4> <ul class="itemize mark-bullet"> Tone-map input to standard gamut BT.709 output: <pre class="example-preformatted">libplacebo=colorspace=bt709:color_primaries=bt709
</pre>

Rescale input to fit into standard 1080p, with high quality scaling: <pre class="example-preformatted">libplacebo=w=1920:h=1080:force_original_asp
</pre>

Interpolate low FPS / VFR input to smoothed constant 60 fps output: <pre class="example-preformatted">libplacebo=fps=60:frame_mixer=mitchell_clam
</pre>

Convert input to standard sRGB JPEG: <pre class="example-preformatted">libplacebo=format=yuv420p:colorspace=bt470bg:color_primaries=bt709
2-1:range=pc </pre>

Use higher quality debanding settings: <pre class="example-preformatted">libplacebo=deband=true:deband_iterations=3:deband_radius=8:deband_th
</pre>

Run this filter on the CPU, on systems with Mesa installed (and with the most expensive options disabled): <pre class="example-preformatted">ffmpeg ... -init_hw_device vulkan:llvmpipe ... -vf libplacebo=upscaler=none:downscaler=none:peak_detect=false
</pre>

Suppress CPU-based AV1/H.274 film grain application in the decoder, in favor of doing it with this filter. Note that this is only a gain if the frames are either already on the GPU, or if you’re using libplacebo for other purposes, since otherwise the VRAM roundtrip will more than offset any expected speedup. <pre

class=”example-preformatted”>ffmpeg -export_side_data +film_grain ... -vf libplacebo=apply_filmgrain=true </pre>

Interop with VAAPI hwdec to avoid round-tripping through RAM: <pre class=”example-preformatted”>ffmpeg -init_hw_device vulkan -hwaccel vaapi -hwaccel_output_format vaapi ... -vf libplacebo </pre>

 <h3 class=”section”>32.149 libvmaf# TOC</h3>

Calculate the VMAF (Video Multi-Method Assessment Fusion) score for a reference/distorted pair of input videos. The first input is the distorted video, and the second input is the reference video. The obtained VMAF score is printed through the logging system. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-libvmaf</code>. The filter has following options: <dl class=”table”> <dt><samp class=”option”>model</samp></dt> <dd> A ‘|‘ delimited list of vmaf models. Each model can be configured with a number of parameters. Default value: <code class=”code”>"version=vmaf_v0.6.1"</code> </dd> <dt><samp class=”option”>feature</samp></dt> <dd> A ‘|‘ delimited list of features. Each feature can be configured with a number of parameters. </dd> <dt><samp class=”option”>log_path</samp></dt> <dd> Set the file path to be used to store log files. </dd> <dt><samp class=”option”>log_fmt</samp></dt> <dd> Set the format of the log file (xml, json, csv, or sub). </dd> <dt><samp class=”option”>pool</samp></dt> <dd> Set the pool method to be used for computing vmaf. Options are <code class=”code”>min</code>, <code class=”code”>harmonic_mean</code> or <code class=”code”>mean</code> (default). </dd> <dt><samp class=”option”>n_threads</samp></dt> <dd> Set number of threads to be used when initializing libvmaf. Default value: <code class=”code”>0</code>, no threads. </dd> <dt><samp class=”option”>n_subsample</samp></d <dd> Set frame subsampling interval to be used. </dd> </dl>

This filter also supports the framesync options. <h4 class=”subsection”>32.149.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> In the examples below, a distorted video <samp class=”file”>distorted.mpg</samp> is compared with a reference file <samp class=”file”>reference.mpg</samp>.

Basic usage: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf=log_path=output.xml -f null - </pre>

Example with multiple models: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='model=version=vmaf_v0.6.1\\:name=vmaf|version=vmaf_v0.6.1neg\\:name=vmaf_neg' -f null - </pre>

Example with multiple additional features: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mpg -lavfi libvmaf='feature=name=psnr|name=ciede' -f null - </pre>

Example with options and different containers: <pre class=”example-preformatted”>ffmpeg -i distorted.mpg -i reference.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]libvmaf=log_fmt=json:log_path=output.json" -f null - </pre>

 <h3 class=”section”>32.150 libvmaf_cuda# TOC</h3>

This is the CUDA variant of the libvmaf filter. It only accepts CUDA frames. It requires Netflix’s vmaf library (libvmaf) as a pre-requisite. After installing the library it can be enabled using: <code class=”code”>./configure --enable-nonfree --enable-ffnvcodec --enable-libvmaf</code>. <h4 class=”subsection”>32.150.1 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Basic usage showing CUVID hardware decoding and CUDA scaling with scale_cuda: <pre class=”example-preformatted”>ffmpeg \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i dis.obu \ -hwaccel cuda -hwaccel_output_format cuda -codec:v av1_cuvid -i ref.obu \ -filter_complex " [0:v]scale_cuda=format=yuv420p[dis]; \ [1:v]scale_cuda=format=yuv420p[ref]; \ [dis][ref]libvmaf_cuda=log_fmt=json:log_path=output.json " \ -f null - </pre>

 <h3 class=”section”>32.151 limitdiff# TOC</h3> Apply limited difference filter using second and optionally third video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set the

threshold to use when allowing certain differences between video streams. Any absolute difference value lower or exact than this threshold will pick pixel components from first video stream. </dd> <dt><samp class=”option”>elasticity</samp></dt> <dd> Set the elasticity of soft thresholding when processing video streams. This value multiplied with first one sets second threshold. Any absolute difference value greater or exact than second threshold will pick pixel components from second video stream. For values between those two threshold linear interpolation between first and second video stream will be used. </dd> <dt><samp class=”option”>reference</samp></dt> <dd> Enable the reference (third) video stream processing. By default is disabled. If set, this video stream will be used for calculating absolute difference with first video stream. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class=”subsection”>32.151.1 Commands# TOC</h4>

This filter supports the all above options as commands except option ‘<samp class=”samp”>reference</samp>’. <h3 class=”section”>32.152 limiter# TOC</h3>

Limits the pixel components values to the specified range [min, max]. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>min</samp></dt> <dd> Lower bound. Defaults to the lowest representable value for the input. Set to <var class=”var”>auto</var> to use the minimum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class=”option”>max</samp></dt> <dd> Upper bound. Defaults to the highest representable value for the input. Set to <var class=”var”>auto</var> to use the maximum signal level for the input colorspace (only affects TV-range YUV signals). </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify which planes will be processed. Defaults to all available. </dd> </dl>

 <h4 class=”subsection”>32.152.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.153 loop<span

class=”pull-right”># TOC</h3>

Loop video frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>loop</samp></dt> <dd> Set the number of loops. Setting this value to -1 will result in infinite loops. Default is 0. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set maximal size in number of frames. Default is 0. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set first frame of loop. Default is 0. </dd> <dt><samp class=”option”>time</samp></dt> <dd> Set the time of loop start in seconds. Only used if option named <var class=”var”>start</var> is set to <code class=”code”>-1</code>. </dd> </dl>

 <h4 class=”subsection”>32.153.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Loop single first frame infinitely: <pre class=”example-preformatted”>loop=loop=-1:size=1:start=0 </pre>

Loop single first frame 10 times: <pre class=”example-preformatted”>loop=loop=10:size=1:start=0 </pre>

Loop 10 first frames 5 times: <pre class=”example-preformatted”>loop=loop=5:size=10:start=0 </pre>

 <h3 class=”section”>32.154 lut1d# TOC</h3>

Apply a 1D LUT to an input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> Set the 1D LUT file name. Currently supported formats: <dl class=”table”> <dt>‘<samp class=”samp”>cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class=”samp”>csp</samp>’</dt> <dd> cineSpace </dd> </dl>

</dd> <dt><samp class=”option”>interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class=”samp”>linear</samp>’</dt> <dd> Interpolate values using the linear interpolation. </dd> <dt>‘<samp

class=”samp”>cosine</samp>’</dt> <dd> Interpolate values using the cosine interpolation. </dd> <dt>‘<samp class=”samp”>cubic</samp>’</dt> <dd> Interpolate values using the cubic interpolation. </dd> <dt>‘<samp class=”samp”>spline</samp>’</dt> <dd> Interpolate values using the spline interpolation. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>32.154.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.155 lut3d# TOC</h3>

Apply a 3D LUT to an input video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> Set the 3D LUT file name. Currently supported formats: <dl class=”table”> <dt>‘<samp class=”samp”>3dl</samp>’</dt> <dd> AfterEffects </dd> <dt>‘<samp class=”samp”>cube</samp>’</dt> <dd> Iridas </dd> <dt>‘<samp class=”samp”>dat</samp>’</dt> <dd> DaVinci </dd> <dt>‘<samp class=”samp”>m3d</samp>’</dt> <dd> Pandora </dd> <dt>‘<samp class=”samp”>csp</samp>’</dt> <dd> cineSpace </dd> </dl> </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Select interpolation mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>nearest</samp>’</dt> <dd> Use values from the nearest defined point. </dd> <dt>‘<samp class=”samp”>trilinear</samp>’</dt> <dd> Interpolate values using the 8 points defining a cube. </dd> <dt>‘<samp class=”samp”>tetrahedral</samp>’</dt> <dd> Interpolate values using a tetrahedron. </dd> <dt>‘<samp class=”samp”>pyramid</samp>’</dt> <dd> Interpolate values using a pyramid. </dd> <dt>‘<samp class=”samp”>prism</samp>’</dt> <dd> Interpolate values using a prism. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>32.155.1 Commands# TOC</h4>

This filter supports the <code class=”code”>interp</code> option as commands. <h3 class=”section”>32.156 lumakey<a class=”anchor hidden-xs” href=”#lumakey” aria-

hidden=”true”># TOC</h3>

Turn certain luma values into transparency. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold</samp></dt> <dd> Set the luma which will be used as base for transparency. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>tolerance</samp></dt> <dd> Set the range of luma values to be keyed out. Default value is <code class=”code”>0.01</code>. </dd> <dt><samp class=”option”>softness</samp></dt> <dd> Set the range of softness. Default value is <code class=”code”>0</code>. Use this to control gradual transition from zero to full transparency. </dd> </dl>

 <h4 class=”subsection”>32.156.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class=”section”>32.157 lut, lutrgb, lutyuv# TOC</h3>

Compute a look-up table for binding each pixel component input value to an output value, and apply it to the input video. <var class=”var”>lutyuv</var> applies a lookup table to a YUV input video, <var class=”var”>lutrgb</var> to an RGB input video. These filters accept the following parameters: <dl class=”table”> <dt><samp class=”option”>c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class=”option”>c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class=”option”>c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class=”option”>c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class=”option”>r</samp></dt> <dd> set red component expression </dd> <dt><samp class=”option”>g</samp></dt> <dd> set green component expression </dd> <dt><samp class=”option”>b</samp></dt> <dd> set blue component expression </dd> <dt><samp class=”option”>a</samp></dt> <dd> alpha component expression </dd> <dt><samp class=”option”>y</samp></dt> <dd> set Y/luma component expression </dd> <dt><samp

class=”option”>u</samp></dt> <dd> set U/Cb component expression </dd> <dt><samp class=”option”>v</samp></dt> <dd> set V/Cr component expression </dd> </dl>

Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in input. The <var class=”var”>lut</var> filter requires either YUV or RGB pixel formats in input, <var class=”var”>lutrgb</var> requires RGB pixel formats in input, and <var class=”var”>lutyuv</var> requires YUV. The expressions can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class=”option”>clipval</samp></dt> <dd> The input value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>maxval</samp></dt> <dd> The maximum value for the pixel component. </dd> <dt><samp class=”option”>minval</samp></dt> <dd> The minimum value for the pixel component. </dd> <dt><samp class=”option”>negval</samp></dt> <dd> The negated value for the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range; it corresponds to the expression "maxval-clipval+minval". </dd> <dt><samp class=”option”>clip(val)</samp></dt> <dd> The computed value in <var class=”var”>val</var>, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. </dd> <dt><samp class=”option”>gammaval(gamma)</samp></dt> <dd> The computed gamma correction value of the pixel component value, clipped to the <var class=”var”>minval</var>-<var class=”var”>maxval</var> range. It corresponds to the expression "pow((clipval-minval)/(maxval-minval)\,<var class=”var”>gamma</var>)*(maxval-minval)+minval" </dd> </dl>

All expressions default to "clipval". <h4 class=”subsection”>32.157.1 Commands# TOC</h4>

This filter supports same commands as options. <h4 class=”subsection”>32.157.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Negate input video: <pre class=”example-preformatted”>lutrgb="r=maxval+minval-val:g=maxval+minval-val:b=maxval+minval-val" lutyuv="y=maxval+minval-

val:u=maxval+minval-val:v=maxval+minval-val" </pre>

The above is the same as: <pre class="example-preformatted">lutrgb="r=negval:g=negval:b=negval" lutyuv="y=negval:u=negval:v=negval" </pre>

Negate luma: <pre class="example-preformatted">lutyuv=y=negval </pre>

Remove chroma components, turning the video into a graytone image: <pre class="example-preformatted">lutyuv="u=128:v=128" </pre>

Apply a luma burning effect: <pre class="example-preformatted">lutyuv="y=2*val" </pre>

Remove green and blue components: <pre class="example-preformatted">lutrgb="g=0:b=0" </pre>

Set a constant alpha channel value on input: <pre class="example-preformatted">format=rgba,lutrgb=a="maxval-minval/2" </pre>

Correct luma gamma by a factor of 0.5: <pre class="example-preformatted">lutyuv=y=gammaval(0.5) </pre>

Discard least significant bits of luma: <pre class="example-preformatted">lutyuv=y='bitand(val, 128+64+32)' </pre>

Technicolor like effect: <pre class="example-preformatted">lutyuv=u='(val-maxval/2)*2+maxval/2':v='(val-maxval/2)*2+maxval/2' </pre>

 <h3 class="section">32.158 lut2, tlut2# TOC</h3>

The <code class="code">lut2</code> filter takes two input streams and outputs one stream. The <code class="code">tlut2</code> (time lut2) filter takes two consecutive frames from one single stream. This filter accepts the following parameters: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set first pixel component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set second pixel component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set third pixel component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set fourth pixel component expression, corresponds to the alpha component </dd> <dt><samp class="option">d</samp></dt> <dd> set output bit depth, only available for <code class="code">lut2</code>

filter. By default is 0, which means bit depth is automatically picked from first input format. </dd> </dl>

The <code class=”code”>lut2</code> filter also supports the framesync options. Each of them specifies the expression to use for computing the lookup table for the corresponding pixel component values. The exact component associated to each of the <var class=”var”>c*</var> options depends on the format in inputs. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dd> The first input value for the pixel component. </dd> <dt><samp class=”option”>y</samp></dt> <dd> The second input value for the pixel component. </dd> <dt><samp class=”option”>bdx</samp></dt> <dd> The first input video bit depth. </dd> <dt><samp class=”option”>bdy</samp></dt> <dd> The second input video bit depth. </dd> </dl>

All expressions default to "x". <h4 class=”subsection”>32.158.1 Commands# TOC</h4>

This filter supports the all above options as commands except option <code class=”code”>d</code>. <h4 class=”subsection”>32.158.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Highlight differences between two RGB video streams: <pre class=”example-preformatted”>lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,0,pow(2,bdx)-1)' </pre>

Highlight differences between two YUV video streams: <pre class=”example-preformatted”>lut2='ifnot(x-y,0,pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1):ifnot(x-y,pow(2,bdx-1),pow(2,bdx)-1)' </pre>

Show max difference between two video streams: <pre class=”example-preformatted”>lut2='if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1))):if(lt(x,y),0,if(gt(x,y),pow(2,bdx)-1,pow(2,bdx-1)))' </pre>

 <h3 class=”section”>32.159 maskedclamp# TOC</h3>

Clamp the first input stream with the second input and third input stream. Returns the value of first stream to be between second input stream - <code class=”code”>undershoot</code> and third input stream + <code class=”code”>overshoot</code>. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>undershoot</samp></dt> <dd> Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>overshoot</samp></dt> <dd> Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class=”subsection”>32.159.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.160 maskedmax# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is greater than first one or from third input stream otherwise. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class=”subsection”>32.160.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.161 maskedmerge# TOC</h3>

Merge the first input stream with the second input stream using per pixel weights in the third input stream. A value of 0 in the third stream pixel component means that pixel component from first stream is returned unchanged, while maximum value (eg. 255 for 8-bit videos) means

that pixel component from second stream is returned unchanged. Intermediate values define the amount of merging between both input stream’s pixel components. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">32.161.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.162 maskedmin# TOC</h3>

Merge the second and third input stream into output stream using absolute differences between second input stream and first input stream and absolute difference between third input stream and first input stream. The picked value will be from second input stream if second absolute difference is less than first one or from third input stream otherwise. This filter accepts the following options: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed as bitmap, unprocessed planes will be copied from first stream. By default value 0xf, all planes will be processed. </dd> </dl>

 <h4 class="subsection">32.162.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.163 maskedthreshold# TOC</h3>
Pick pixels comparing absolute difference of two video streams with fixed threshold. If absolute difference between pixel component of first and second video stream is equal or lower than user supplied threshold than pixel component from first video stream is picked, otherwise pixel component from second video stream is picked. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> Set threshold used when picking pixels from absolute difference from two input video streams. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set

which planes will be processed as bitmap, unprocessed planes will be copied from second stream. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set mode of filter operation. Can be <code class="code">abs</code> or <code class="code">diff</code>. Default is <code class="code">abs</code>. </dd> </dl>

 <h4 class="subsection">32.163.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.164 maskfun# TOC</h3> Create mask from input video. For example it is useful to create motion masks after <code class="code">tblend</code> filter. This filter accepts the following options: <dl class="table"> <dt><samp class="option">low</samp></dt> <dd> Set low threshold. Any pixel component lower or exact than this value will be set to 0. </dd> <dt><samp class="option">high</samp></dt> <dd> Set high threshold. Any pixel component higher than this value will be set to max value allowed for current pixel format. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set planes to filter, by default all available planes are filtered. </dd> <dt><samp class="option">fill</samp></dt> <dd> Fill all frame pixels with this value. </dd> <dt><samp class="option">sum</samp></dt> <dd> Set max average pixel value for frame. If sum of all pixel components is higher that this average, output frame will be completely filled with value set by <var class="var">fill</var> option. Typically useful for scene changes when used in combination with <code class="code">tblend</code> filter. </dd> </dl>

 <h4 class="subsection">32.164.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.165 mcdeint# TOC</h3>

Apply motion-compensation deinterlacing. It needs one field per frame as input and must thus be used together with yadif=1/3

or equivalent. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set the deinterlacing mode. It accepts one of the following values: <dl class="table"> <dt>‘<samp class="samp">fast</samp>’</dt> <dt>‘<samp class="samp">medium</samp>’</dt> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> use iterative motion estimation </dd> <dt>‘<samp class="samp">extra_slow</samp>’</dt> <dd> like ‘<samp class="samp">slow</samp>’, but use multiple reference frames. </dd> </dl> Default value is ‘<samp class="samp">fast</samp>’. </dd> <dt><samp class="option">parity</samp></dt> <dd> Set the picture field parity assumed for the input video. It must be one of the following values: <dl class="table"> <dt>‘<samp class="samp">0, tff</samp>’</dt> <dd> assume top field first </dd> <dt>‘<samp class="samp">1, bff</samp>’</dt> <dd> assume bottom field first </dd> </dl>

Default value is ‘<samp class="samp">bff</samp>’. </dd> <dt><samp class="option">qp</samp></dt> <dd> Set per-block quantization parameter (QP) used by the internal encoder. Higher values should result in a smoother motion vector field but less optimal individual vectors. Default value is 1. </dd> </dl>

 <h3 class="section">32.166 median# TOC</h3>

Pick median pixel from certain rectangle defined by radius. This filter accepts the following options: <dl class="table"> <dt><samp class="option">radius</samp></dt> <dd> Set horizontal radius size. Default value is <code class="code">1</code>. Allowed range is integer from 1 to 127. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. Default is <code class="code">15</code>, which is all available planes. </dd> <dt><samp class="option">radiusV</samp></dt> <dd> Set vertical radius size. Default value is <code class="code">0</code>. Allowed range is integer from 0 to 127. If it is 0, value will be picked from horizontal <code class="code">radius</code> option. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">32.166.1 Commands# <a class="anchor hidden-xs"href="#toc-

Commands-120" aria-hidden="true">TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h3 class="section">32.167 mergeplanes# TOC</h3>

Merge color channel components from several video streams. The filter accepts up to 4 input streams, and merge selected input planes to the output video. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mapping</samp></dt> <dd> Set input to output plane mapping. Default is <code class="code">0</code>. The mappings is specified as a bitmap. It should be specified as a hexadecimal number in the form 0xAa[Bb[Cc[Dd]]]. ’Aa’ describes the mapping for the first plane of the output stream. ’A’ sets the number of the input stream to use (from 0 to 3), and ’a’ the plane number of the corresponding input to use (from 0 to 3). The rest of the mappings is similar, ’Bb’ describes the mapping for the output stream second plane, ’Cc’ describes the mapping for the output stream third plane and ’Dd’ describes the mapping for the output stream fourth plane. </dd> <dt><samp class="option">format</samp></dt> <dd> Set output pixel format. Default is <code class="code">yuva444p</code>. </dd> <dt><samp class="option">map0s</samp></dt> <dt><samp class="option">map1s</samp></dt> <dt><samp class="option">map2s</samp></dt> <dt><samp class="option">map3s</samp></dt> <dd> Set input to output stream mapping for output Nth plane. Default is <code class="code">0</code>. </dd> <dt><samp class="option">map0p</samp></dt> <dt><samp class="option">map1p</samp></dt> <dt><samp class="option">map2p</samp></dt> <dt><samp class="option">map3p</samp></dt> <dd> Set input to output plane mapping for output Nth plane. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">32.167.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Merge three gray video streams of same width and height into single video stream: <pre class="example-preformatted">[a0][a1][a2]mergeplanes=0x001020:yuv444p
</pre>

Merge 1st yuv444p stream and 2nd gray video stream into yuva444p video stream: <pre class="example-preformatted">[a0][a1]mergeplanes=0x00010210:yuva444p </pre>

Swap Y and A plane in yuva444p stream:
<pre class="example-preformatted">format=yuva444p,mergeplanes=0x03010200:yuva444p
</pre>

Swap U and V plane in yuv420p stream:
<pre class="example-preformatted">format=yuv420p,mergeplanes=0x000201:yuv420p
</pre>

Cast a rgb24 clip to yuv444p: <pre
class="example-preformatted">format=rgb24,mergeplanes=0x000102:yuv444p
</pre>

 <h3 class="section">32.168 mestimate<span
class="pull-right"><a class="anchor hidden-xs" href="#mestimate" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-mestimate"
aria-hidden="true">TOC</h3>

Estimate and export motion vectors using block matching algorithms.
Motion vectors are stored in frame side data to be used by other filters.
 This filter accepts the following options: <dl class="table">
<dt><samp class="option">method</samp></dt> <dd> Specify the
motion estimation method. Accepts one of the following values: <dl
class="table"> <dt>‘<samp class="samp">esa</samp>’</dt>
<dd> Exhaustive search algorithm. </dd> <dt>‘<samp
class="samp">tss</samp>’</dt> <dd> Three step search algo-
rithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt>
<dd> Two dimensional logarithmic search algorithm. </dd>
<dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New
three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt>
<dd> Four step search algorithm. </dd> <dt>‘<samp
class="samp">ds</samp>’</dt> <dd> Diamond search algo-
rithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt>
<dd> Hexagon-based search algorithm. </dd> <dt>‘<samp
class="samp">epzs</samp>’</dt> <dd> Enhanced predictive
zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt>
<dd> Uneven multi-hexagon search algorithm. </dd> </dl>
Default value is ‘<samp class="samp">esa</samp>’.
 </dd> <dt><samp class="option">mb_size</samp></dt>
<dd> Macroblock size. Default <code class="code">16</code>.
 </dd> <dt><samp class="option">search_param</samp></dt>
<dd> Search parameter. Default <code class="code">7</code>.
</dd> </dl>

 <h3 class="section">32.169
mestimate_d3d12<a class="anchor hidden-
xs" href="#mestimate_005fd3d12" aria-hidden="true"># <a
class="anchor hidden-xs"href="#toc-mestimate_005fd3d12" aria-hidden="true">TOC</h3>

Estimate and export motion vectors using D3D12 hardware-accelerated mo-

tion estimation. This filter uses GPU hardware motion estimation capabilities available in DirectX 12 Video APIs to achieve significant performance improvements over the software-based <code class=”code”>mestimate</code> filter. Motion vectors are stored in frame side data to be used by other filters. This filter requires the input to be in <code class=”code”>d3d12</code> hardware pixel format. The filter uses quarter-pixel precision for motion vector estimation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mb_size</samp></dt> <dd> Macroblock size. Only <code class=”code”>8</code> and <code class=”code”>16</code> are supported. Default <code class=”code”>16</code>. </dd> </dl>

 <h4 class=”subsection”>32.169.1 Examples# TOC</h4>

Estimate motion vectors using D3D12 hardware acceleration with 16x16 blocks, and visualize them: <pre class=”example-preformatted”>ffmpeg -hwaccel d3d12va -hwaccel_output_format d3d12 -i input.mp4 \ -vf mestimate_d3d12=mb_size=16,hwdownload,format=nv12,codecview=mv=pf \ -c:v libx264 -preset fast -b:v 5M output.mp4 </pre>

 <h3 class=”section”>32.170 midequalizer# TOC</h3>

Apply Midway Image Equalization effect using two video streams. Midway Image Equalization adjusts a pair of images to have the same histogram, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a pair of stereo cameras. This filter has two inputs and one output, which must be of same pixel format, but may be of different sizes. The output of filter is first input adjusted with midway histogram of both inputs. This filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. Default is <code class=”code”>15</code>, which is all available planes. </dd> </dl>

 <h3 class=”section”>32.171 minterpolate# TOC</h3>

Convert the video to specified frame rate using motion interpolation. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>fps</samp></dt> <dd> Specify the output frame rate. This can be rational e.g. <code class=”code”>60000/1001</code>.

Frames are dropped if <var class="var">fps</var> is lower than source fps. Default <code class="code">60</code>. </dd> <dt><samp class="option">mi_mode</samp></dt> <dd> Motion interpolation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">dup</samp>’</dt> <dd> Duplicate previous or next frame for interpolating new ones. </dd> <dt>‘<samp class="samp">blend</samp>’</dt> <dd> Blend source frames. Interpolated frame is mean of previous and next frames. </dd> <dt>‘<samp class="samp">mci</samp>’</dt> <dd> Motion compensated interpolation. Following options are effective when this mode is selected: <dl class="table"> <dt>‘<samp class="samp">mc_mode</samp>’</dt> <dd> Motion compensation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">obmc</samp>’</dt> <dd> Overlapped block motion compensation. </dd> <dt>‘<samp class="samp">aobmc</samp>’</dt> <dd> Adaptive overlapped block motion compensation. Window weighting coefficients are controlled adaptively according to the reliabilities of the neighboring motion vectors to reduce oversmoothing. </dd> </dl> Default mode is ‘<samp class="samp">obmc</samp>’. </dd> <dt>‘<samp class="samp">me_mode</samp>’</dt> <dd> Motion estimation mode. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">bidir</samp>’</dt> <dd> Bidirectional motion estimation. Motion vectors are estimated for each source frame in both forward and backward directions. </dd> <dt>‘<samp class="samp">bilat</samp>’</dt> <dd> Bilateral motion estimation. Motion vectors are estimated directly for interpolated frame. </dd> </dl> Default mode is ‘<samp class="samp">bilat</samp>’. </dd> <dt>‘<samp class="samp">me</samp>’</dt> <dd> The algorithm to be used for motion estimation. Following values are accepted: <dl class="table"> <dt>‘<samp class="samp">esa</samp>’</dt> <dd> Exhaustive search algorithm. </dd> <dt>‘<samp class="samp">tss</samp>’</dt> <dd> Three step search algorithm. </dd> <dt>‘<samp class="samp">tdls</samp>’</dt> <dd> Two dimensional logarithmic search algorithm. </dd> <dt>‘<samp class="samp">ntss</samp>’</dt> <dd> New three step search algorithm. </dd> <dt>‘<samp class="samp">fss</samp>’</dt> <dd> Four step search algorithm. </dd> <dt>‘<samp class="samp">ds</samp>’</dt> <dd> Diamond search algorithm. </dd> <dt>‘<samp class="samp">hexbs</samp>’</dt> <dd> Hexagon-based search algorithm. </dd> <dt>‘<samp class="samp">epzs</samp>’</dt> <dd> Enhanced predictive zonal search algorithm. </dd> <dt>‘<samp class="samp">umh</samp>’</dt> <dd> Uneven multi-hexagon search algorithm. </dd> </dl> Default algorithm is ‘<samp class="samp">epzs</samp>’. </dd> <dt>‘<samp class="samp">mb_size</samp>’</dt>

<dd> Macroblock size. Default <code class="code">16</code>.
</dd> <dt>‘<samp class="samp">search_param</samp>’</dt>
<dd> Motion estimation search parameter. Default <code class="code">32</code>.
 </dd> <dt>‘<samp class="samp">vsbmc</samp>’</dt>
<dd> Enable variable-size block motion compensation. Motion estimation
is applied with smaller block sizes at object boundaries in order to make them
less blurry. Default is <code class="code">0</code> (disabled). </dd>
</dl> </dd> </dl>

</dd> <dt><samp class="option">scd</samp></dt> <dd> Scene
change detection method. Scene change leads motion vectors to be in
random direction. Scene change detection replace interpolated frames
by duplicate ones. May not be needed for other modes. Following
values are accepted: <dl class="table"> <dt>‘<samp
class="samp">none</samp>’</dt> <dd> Disable scene change
detection. </dd> <dt>‘<samp class="samp">fdiff</samp>’</dt>
<dd> Frame difference. Corresponding pixel values are compared and
if it satisfies <var class="var">scd_threshold</var> scene change is
detected. </dd> </dl> Default method is ‘<samp
class="samp">fdiff</samp>’. </dd> <dt><samp class="option">scd_threshold</samp></
<dd> Scene change detection threshold. Default is <code class="code">10.</code>.
</dd> </dl>

 <h3 class="section">32.172 mix<span class="pull-
right">#
TOC</h3>

Mix several video input streams into one video stream. A
description of the accepted options follows. <dl class="table">
<dt><samp class="option">inputs</samp></dt> <dd> The number
of inputs. If unspecified, it defaults to 2. </dd> <dt><samp
class="option">weights</samp></dt> <dd> Specify weight of each
input video stream as sequence. Each weight is separated by space. If
number of weights is smaller than number of <var class="var">frames</var>
last specified weight will be used for all remaining unset weights.
</dd> <dt><samp class="option">scale</samp></dt> <dd> Specify
scale, if it is set it will be multiplied with sum of each weight multiplied
with pixel values to give final destination pixel value. By default <var
class="var">scale</var> is auto scaled to sum of weights. </dd>
<dt><samp class="option">planes</samp></dt> <dd> Set which
planes to filter. Default is all. Allowed range is from 0 to 15. </dd>
<dt><samp class="option">duration</samp></dt> <dd> Specify how
end of stream is determined. <dl class="table"> <dt>‘<samp
class="samp">longest</samp>’</dt> <dd> The duration of the
longest input. (default) </dd> <dt>‘<samp class="samp">shortest</samp>’</dt>
<dd> The duration of the shortest input. </dd> <dt>‘<samp
class="samp">first</samp>’</dt> <dd> The duration of the

first input. </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.172.1 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">32.173 monochrome# TOC</h3> Convert video to gray using custom color filter. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">cb</samp></dt> <dd> Set the chroma blue spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">cr</samp></dt> <dd> Set the chroma red spot. Allowed range is from -1 to 1. Default value is 0. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the color filter size. Allowed range is from .1 to 10. Default value is 1. </dd> <dt><samp class="option">high</samp></dt> <dd> Set the highlights strength. Allowed range is from 0 to 1. Default value is 0. </dd> </dl>

 <h4 class="subsection">32.173.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.174 morpho# TOC</h3>

This filter allows to apply main morphological grayscale transforms, erode and dilate with arbitrary structures set in second input stream. Unlike naive implementation and much slower performance in erosion and dilation filters, when speed is critical <code class="code">morpho</code> filter should be used instead. A description of accepted options follows, <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set morphological transform to apply, can be: <dl class="table"> <dt>‘<samp class="samp">erode</samp>’</dt> <dt>‘<samp class="samp">dilate</samp>’</d <dt>‘<samp class="samp">open</samp>’</dt> <dt>‘<samp

class=”samp”>close</samp>’</dt> <dt>‘<samp class=”samp”>gradient</samp>’<
<dt>‘<samp class=”samp”>tophat</samp>’</dt> <dt>‘<samp
class=”samp”>blackhat</samp>’</dt> </dl> Default is <code
class=”code”>erode</code>. </dd> <dt><samp class=”option”>planes</samp></dt>
<dd> Set planes to filter, by default all planes except alpha are filtered. </dd> <dt><samp class=”option”>structure</samp></dt>
<dd> Set which structure video frames will be processed from second input stream, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> </dl>

The <code class=”code”>morpho</code> filter also supports the framesync options. <h4 class=”subsection”>32.174.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class=”section”>32.175 mpdecimate# TOC</h3>

Drop frames that do not differ greatly from the previous frame in order to reduce frame rate. The main use of this filter is for very-low-bitrate encoding (e.g. streaming over dialup modem), but it could in theory be used for fixing movies that were inverse-telecined incorrectly. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Set the mode. If the value is 0, the filter drops similar frames. If the value is 1, the filter keeps only frames that are one representative of sufficiently long duplicate runs. Default value is 0. </dd> <dt><samp class=”option”>max</samp></dt> <dd> For mode 0: Set the maximum number of consecutive frames which can be dropped (if positive), or the minimum interval between dropped frames (if negative). If the value is 0, the frame is dropped disregarding the number of previous sequentially dropped frames. Default value is 0. </dd> <dt><samp class=”option”>min</samp></dt> <dd> For mode 1: Set the minimum number of frame duplicates to mark the frame as non-unique and keep the first occurrence of it. Default value is 1. </dd> <dt><samp class=”option”>keep</samp></dt> <dd> For mode 0: Set the maximum number of consecutive similar frames to ignore before to start dropping them. If the value is 0, the frame is dropped disregarding the number of previous sequentially similar frames. Default value is 0. </dd> <dt><samp class=”option”>hi</samp></dt> <dt><samp class=”option”>lo</samp></dt> <dt><samp class=”option”>frac</samp></dt>
<dd> Set the dropping threshold values. Values for <samp

class=”option”>hi</samp> and <samp class=”option”>lo</samp> are for 8x8 pixel blocks and represent actual pixel value differences, so a threshold of 64 corresponds to 1 unit of difference for each pixel, or the same spread out differently over the block. A frame is a candidate for dropping if no 8x8 blocks differ by more than a threshold of <samp class=”option”>hi</samp>, and if no more than <samp class=”option”>frac</samp> blocks (1 meaning the whole image) differ by more than a threshold of <samp class=”option”>lo</samp>. Default value for <samp class=”option”>hi</samp> is 64*12, default value for <samp class=”option”>lo</samp> is 64*5, and default value for <samp class=”option”>frac</samp> is 0.33. </dd> </dl>

 <h3 class=”section”>32.176 msad# TOC</h3>

Obtain the MSAD (Mean Sum of Absolute Differences) between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained per component, average, min and max MSAD is printed through the logging system. The filter stores the calculated MSAD of each frame in frame metadata. This filter also supports the framesync options. In the below example the input file <samp class=”file”>main.mpg</samp> being processed is compared with the reference file <samp class=”file”>ref.mpg</samp>.   <pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mpg -lavfi msad -f null - </pre>

 <h3 class=”section”>32.177 multiply# TOC</h3> Multiply first video stream pixels values with second video stream pixels values. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>scale</samp></dt> <dd> Set the scale applied to second video stream. By default is <code class=”code”>1</code>. Allowed range is from <code class=”code”>0</code> to <code class=”code”>9</code>. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set the offset applied to second video stream. By default is <code class=”code”>0.5</code>. Allowed range is from <code class=”code”>-1</code> to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Specify planes from input video stream that will be processed. By default all planes are processed. </dd> </dl>

 <h4 class=”subsection”>32.177.1 Commands<a class=”anchor hidden-xs” href=”#Commands-

124" aria-hidden="true"># TOC</h4>

This filter supports same commands as options. <h3 class="section">32.178 negate# TOC</h3>

Negate (invert) the input video. It accepts the following option: <dl class="table"> <dt><samp class="option">components</samp></dt> <dd> Set components to negate. Available values for components are: <dl class="table"> <dt>‘<samp class="samp">y</samp>’</dt> <dt>‘<samp class="samp">u</samp>’</dt> <dt>‘<samp class="samp">v</samp>’</dt> <dt>‘<samp class="samp">a</samp>’</dt> <dt>‘<samp class="samp">r</samp>’</dt> <dt>‘<samp class="samp">g</samp>’</dt> <dt>‘<samp class="samp">b</samp>’</dt> </dl>

</dd> <dt><samp class="option">negate_alpha</samp></dt> <dd> With value 1, it negates the alpha component, if present. Default value is 0. </dd> </dl>

 <h4 class="subsection">32.178.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">32.179 nlmeans# TOC</h3>

Denoise frames using Non-Local Means algorithm. Each pixel is adjusted by looking for other pixels with similar contexts. This context similarity is defined by comparing their surrounding patches of size <samp class="option">p</samp>x<samp class="option">p</samp>. Patches are searched in an area of <samp class="option">r</samp>x<samp class="option">r</samp> around the pixel. Note that the research area defines centers for patches, which means some patches will be made of pixels outside that research area. The filter accepts the following options. <dl class="table"> <dt><samp class="option">s</samp></dt> <dd> Set denoising strength. Default is 1.0. Must be in range [1.0, 30.0]. </dd> <dt><samp class="option">p</samp></dt> <dd> Set patch size. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">pc</samp></dt> <dd> Same as <samp class="option">p</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic.

</dd> <dt><samp class="option">r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class="option">rc</samp></dt> <dd> Same as <samp class="option">r</samp> but for chroma planes. The default value is <var class="var">0</var> and means automatic. </dd> </dl>

 <h3 class="section">32.180 nnedi# TOC</h3>

Deinterlace video using neural network edge directed interpolation. This filter accepts the following options: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dd> Mandatory option, without binary file filter can not work. Currently file can be found here: https://github.com/dubhater/vapoursynth-nnedi3/blob/master/src/nnedi3_weights.bin </dd> <dt><samp class="option">deint</samp></dt> <dd> Set which frames to deinterlace, by default it is <code class="code">all</code>. Can be <code class="code">all</code> or <code class="code">interlaced</code>. </dd> <dt><samp class="option">field</samp></dt> <dd> Set mode of operation. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">af</samp>’</dt> <dd> Use frame flags, both fields. </dd> <dt>‘<samp class="samp">a</samp>’</dt> <dd> Use frame flags, single field. </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> Use top field only. </dd> <dt>‘<samp class="samp">b</samp>’</dt> <dd> Use bottom field only. </dd> <dt>‘<samp class="samp">tf</samp>’</dt> <dd> Use both fields, top first. </dd> <dt>‘<samp class="samp">bf</samp>’</dt> <dd> Use both fields, bottom first. </dd> </dl>

</dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process, by default filter process all frames. </dd> <dt><samp class="option">nsize</samp></dt> <dd> Set size of local neighborhood around each pixel, used by the predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">s8x6</samp>’</dt> <dt>‘<samp class="samp">s16x6</samp>’</dt> <dt>‘<samp class="samp">s32x6</samp>’</d <dt>‘<samp class="samp">s48x6</samp>’</dt> <dt>‘<samp class="samp">s8x4</samp>’</dt> <dt>‘<samp class="samp">s16x4</samp>’</dt <dt>‘<samp class="samp">s32x4</samp>’</dt> </dl>

</dd> <dt><samp class="option">nns</samp></dt> <dd> Set the number of neurons in predictor neural network. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">n16</samp>’</dt> <dt>‘<samp class="samp">n32</samp>’</dt> <dt>‘<samp class="samp">n64</samp>’</dt> <dt>‘<samp class="samp">n128</samp>’</dt>

<dt>‘<samp class="samp">n256</samp>’</dt> </dl>

</dd> <dt><samp class="option">qual</samp></dt> <dd> Controls the number of different neural network predictions that are blended together to compute the final output value. Can be <code class="code">fast</code>, default or <code class="code">slow</code>. </dd> <dt><samp class="option">etype</samp></dt> <dd> Set which set of weights to use in the predictor. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">a, abs</samp>’</dt> <dd> weights trained to minimize absolute error </dd> <dt>‘<samp class="samp">s, mse</samp>’</dt> <dd> weights trained to minimize squared error </dd> </dl>

</dd> <dt><samp class="option">pscrn</samp></dt> <dd> Controls whether or not the prescreener neural network is used to decide which pixels should be processed by the predictor neural network and which can be handled by simple cubic interpolation. The prescreener is trained to know whether cubic interpolation will be sufficient for a pixel or whether it should be predicted by the predictor nn. The computational complexity of the prescreener nn is much less than that of the predictor nn. Since most pixels can be handled by cubic interpolation, using the prescreener generally results in much faster processing. The prescreener is pretty accurate, so the difference between using it and not using it is almost always unnoticeable. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dt>‘<samp class="samp">original</samp>’</
<dt>‘<samp class="samp">new</samp>’</dt> <dt>‘<samp class="samp">new2</samp>’</dt> <dt>‘<samp class="samp">new3</samp>’</dt
</dl>

Default is <code class="code">new</code>. </dd> </dl>

 <h4 class="subsection">32.180.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">weights</var> option. <h3 class="section">32.181 noformat# TOC</h3>

Force libavfilter not to use any of the specified pixel formats for the input to the next filter. It accepts the following parameters: <dl class="table"> <dt><samp class="option">pix_fmts</samp></dt> <dd> A ’|’-separated list of pixel format names, such as pix_fmts=yuv420p|monow|rgb24". </dd> </dl>

 <h4 class="subsection">32.181.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Force libavfilter to use a format different from <var class="var">yuv420p</var> for the input to the vflip filter: <pre class="example-preformatted">noformat=pix_fmts=yuv420p,vflip
</pre>

Convert the input video to any of the formats not contained in the list: <pre class="example-preformatted">noformat=yuv420p|yuv444p|yuv410p
</pre>

 <h3 class="section">32.182 noise# TOC</h3>

Add noise on video input frame. The filter accepts the following options: <dl class="table"> <dt><samp class="option">all_seed</samp></dt> <dt><samp class="option">c0_seed</samp></dt> <dt><samp class="option">c1_seed</samp></dt> <dt><samp class="option">c2_seed</samp></dt> <dt><samp class="option">c3_seed</samp></dt> <dd> Set noise seed for specific pixel component or all pixel components in case of <var class="var">all_seed</var>. Default value is <code class="code">123457</code>. </dd> <dt><samp class="option">all_strength, alls</samp></dt> <dt><samp class="option">c0_strength, c0s</samp></dt> <dt><samp class="option">c1_strength, c1s</samp></dt> <dt><samp class="option">c2_strength, c2s</samp></dt> <dt><samp class="option">c3_strength, c3s</samp></dt> <dd> Set noise strength for specific pixel component or all pixel components in case <var class="var">all_strength</var>. Default value is <code class="code">0</code>. Allowed range is [0, 100]. </dd> <dt><samp class="option">all_flags, allf</samp></dt> <dt><samp class="option">c0_flags, c0f</samp></dt> <dt><samp class="option">c1_flags, c1f</samp></dt> <dt><samp class="option">c2_flags, c2f</samp></dt> <dt><samp class="option">c3_flags, c3f</samp></dt> <dd> Set pixel component flags or set flags for all components if <var class="var">all_flags</var>. Available values for component flags are: <dl class="table"> <dt>‘<samp class="samp">a</samp>’</dt> <dd> averaged temporal noise (smoother) </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> mix random noise with a (semi)regular pattern </dd> <dt>‘<samp class="samp">t</samp>’</dt> <dd> temporal noise (noise pattern changes between frames) </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> uniform noise (gaussian otherwise) </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.182.1 Examples<a class="anchor hidden-xs" href="#Examples-

121" aria-hidden="true"># TOC</h4>

Add temporal and uniform noise to input video: <pre class="example-preformatted">noise=alls=20:allf=t+u </pre>

 <h3 class="section">32.183 normalize# TOC</h3>

Normalize RGB video (aka histogram stretching, contrast stretching). See: https://en.wikipedia.org/wiki/Normalization_(image_processing) For each channel of each frame, the filter computes the input range and maps it linearly to the user-specified output range. The output range defaults to the full dynamic range from pure black to pure white. Temporal smoothing can be used on the input range to reduce flickering (rapid changes in brightness) caused when small dark or bright objects enter or leave the scene. This is similar to the auto-exposure (automatic gain control) on a video camera, and, like a video camera, it may cause a period of over- or under-exposure of the video. The R,G,B channels can be normalized independently, which may cause some color shifting, or linked together as a single channel, which prevents color shifting. Linked normalization preserves hue. Independent normalization does not, so it can be used to remove some color casts. Independent and linked normalization can be combined in any ratio. The normalize filter accepts the following options: <dl class="table"> <dt><samp class="option">blackpt</samp></dt> <dt><samp class="option">whitept</samp></dt> <dd> Colors which define the output range. The minimum input value is mapped to the <var class="var">blackpt</var>. The maximum input value is mapped to the <var class="var">whitept</var>. The defaults are black and white respectively. Specifying white for <var class="var">blackpt</var> and black for <var class="var">whitept</var> will give color-inverted, normalized video. Shades of grey can be used to reduce the dynamic range (contrast). Specifying saturated colors here can create some interesting effects. </dd> <dt><samp class="option">smoothing</samp></dt> <dd> The number of previous frames to use for temporal smoothing. The input range of each channel is smoothed using a rolling average over the current frame and the <var class="var">smoothing</var> previous frames. The default is 0 (no temporal smoothing). </dd> <dt><samp class="option">independence</samp></dt> <dd> Controls the ratio of independent (color shifting) channel normalization to linked (color preserving) normalization. 0.0 is fully linked, 1.0 is fully independent. Defaults to 1.0 (fully independent). </dd> <dt><samp class="option">strength</samp></dt> <dd> Overall strength of the filter. 1.0 is full strength. 0.0 is a rather expensive no-op. Defaults to 1.0 (full strength). </dd> </dl>

 <h4 class="subsection">32.183.1 Commands# TOC</h4> This filter supports same commands as options, excluding <var class="var">smoothing</var> option. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class="subsection">32.183.2 Examples# TOC</h4>

Stretch video contrast to use the full dynamic range, with no temporal smoothing; may flicker depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=
</pre>

As above, but with 50 frames of temporal smoothing; flicker should be reduced, depending on the source content: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50
</pre>

As above, but with hue-preserving linked channel normalization: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:
</pre>

As above, but with half strength: <pre class="example-preformatted">normalize=blackpt=black:whitept=white:smoothing=50:independence=0:stre
</pre>

Map the darkest input color to red, the brightest input color to cyan: <pre class="example-preformatted">normalize=blackpt=red:whitept=cyan
</pre>

 <h3 class="section">32.184 null# TOC</h3>

Pass the video source unchanged to the output. <h3 class="section">32.185 ocio# TOC</h3> OpenColorIO library filter This filter allows you to do color management using the OpenColorIO library. See https://opencolorio.org/ for more details. To Enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libopencolorio</code>. It accepts the following options: <dl class="table"> <dt><samp class="option">config</samp></dt> <dd> By default the filter will

use the OCIO config defined by the OCIO environment variable, but this parameter allows you to explicitly specify its location. If you are getting started, you can use config=ocio://studio-config-v1.0.0_aces-v1.3_ocio-v2.1 which specifies one of the built in defaults. </dd> <dt><samp class="option">input</samp></dt> <dd> Set the input colorspace. </dd> <dt><samp class="option">output</samp></dt> <dd> Set the output colorspace. </dd> <dt><samp class="option">display</samp></dt> <dd> Set the display colorspace, used in combination with view. </dd> <dt><samp class="option">view</samp></dt> <dd> Set the view colorspace, used in combination with display. </dd> <dt><samp class="option">inverse</samp></dt> <dd> When used in combination with display and view, this inverts the transform, so going from a display/view to the "input colorspace". </dd> <dt><samp class="option">filetransform</samp></dt> <dd> Allows you to specify an external file-transform to use instead of the OCIO config file. This is useful for applying a single transform without needing a full OCIO config file. </dd> <dt><samp class="option">format</samp></dt> <dd> Allow you to specify the output pix_fmt of the OCIO filter. This *has* to be a RGB colorspace, so you are limited to rgb24, rgba, rgb48, rgba64, gbrp10, gbrap10, gbrp12, gbrap12, gbrpf32 and gbrapf32; for most encoding we would recommend rgb48. Half-float (gbrpf16, gbrapf16) is accepted as input but is not a valid output format, because swscale has no half-float write support and its legacy half-float read path clamps values to [0,1]. When this option is not set, the output format defaults to the input format, except that half-float input is promoted to the matching float format. Where negotiation cannot yet tell whether the input will be half-float, the output falls back to float, so a float output may be selected even when the input turns out to be an integer format. </dd> <dt><samp class="option">context_params</samp></dt> <dd> Allow you to specify additional context parameters for the OCIO filter. This is a list of key=value pairs, separated by colons. </dd> </dl>

 <h4 class="subsection">32.185.1 Examples# TOC</h4>

Map from ACEScg to ACEScct, this assumes the OCIO file is defined with the OCIO environment variable. <pre class="example-preformatted">input=ACEScg:output=ACEScct:format=rgb48 </pre>

Map from ACEScg to a sRGB display using the "ACES 1.0 - SDR Video" view transform, this assumes the OCIO file is defined with the OCIO environment variable. Note you will need to wrap the argument in quotes to ensure that the spaces are interpreted correctly. <pre class="example-

preformatted”>input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

As above but using the OCIO file studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio rather than the OCIO environment variable. <pre class=”example-preformatted”>config=studio-config-v1.0.0_aces-v1.3_ocio-v2.1_ns.ocio:input=ACEScg:display=sRGB - Display:view=ACES 1.0 - SDR Video:format=rgb48 </pre>

If you are converting to YCrCb you still will want to set the color matrix for the conversion. This is a good example of combining the two. <pre class=”example-preformatted”>ffmpeg -y -i SOURCE-FRAMES.%05d.exr -c:v libx265 -vf "ocio=input=ACEScg:output=ACEScct:format=rgb48,scale=in_col
OUTPUTFILE.mov </pre>

 <h3 class=”section”>32.186 ocr# TOC</h3> Optical Character Recognition This filter uses Tesseract for optical character recognition. To enable compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libtesseract</code>. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>datapath</samp></dt> <dd> Set datapath to tesseract data. Default is to use whatever was set at installation. </dd> <dt><samp class=”option”>language</samp></dt> <dd> Set language, default is "eng". </dd> <dt><samp class=”option”>whitelist</samp></dt> <dd> Set character whitelist. </dd> <dt><samp class=”option”>blacklist</samp></dt> <dd> Set character blacklist. </dd> </dl>

The filter exports recognized text as the frame metadata <code class=”code”>lavfi.ocr.text</code>. The filter exports confidence of recognized words as the frame metadata <code class=”code”>lavfi.ocr.confidence</code>. <h3 class=”section”>32.187 ocv# TOC</h3>

Apply a video transform using libopencv. To enable this filter, install the libopencv library and headers and configure FFmpeg with <code class=”code”>--enable-libopencv</code>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filter_name</samp></dt> <dd> The name of the libopencv filter to apply. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> The parameters to pass to the libopencv filter. If not specified, the default values are assumed. </dd> </dl>

Refer to the official libopencv documentation for more precise information: <a class=”url” href=”http://docs.opencv.org/master/modules/imgproc/doc/filtering.html”>http://docs.open
 Several libopencv filters are supported; see the following subsections.

  <h4 class="subsection">32.187.1 dilate# TOC</h4>

Dilate an image by using a specific structuring element. It corresponds to the libopencv function <code class="code">cvDilate</code>. It accepts the parameters: <var class="var">struct_el</var>|<var class="var">nb_iterations</var>. <var class="var">struct_el</var> represents a structuring element, and has the syntax: <var class="var">cols</var>x<var class="var">rows</var>+<var class="var">anchor_x</var>x<var class="var">anchor_y</var>/<var class="var">shape</var> <var class="var">cols</var> and <var class="var">rows</var> represent the number of columns and rows of the structuring element, <var class="var">anchor_x</var> and <var class="var">anchor_y</var> the anchor point, and <var class="var">shape</var> the shape for the structuring element. <var class="var">shape</var> must be "rect", "cross", "ellipse", or "custom". If the value for <var class="var">shape</var> is "custom", it must be followed by a string of the form "=<var class="var">filename</var>". The file with name <var class="var">filename</var> is assumed to represent a binary image, with each printable character corresponding to a bright pixel. When a custom <var class="var">shape</var> is used, <var class="var">cols</var> and <var class="var">rows</var> are ignored, the number or columns and rows of the read file are assumed instead. The default value for <var class="var">struct_el</var> is "3x3+0x0/rect". <var class="var">nb_iterations</var> specifies the number of times the transform is applied to the image, and defaults to 1. Some examples: <pre class="example-preformatted"># Use the default values ocv=dilate

Dilate using a structuring element with a 5x5 cross, iterating two times ocv=filter_name=dilate:filter_params=5x5+2x2/cross|2

Read the shape from the file diamond.shape, iterating two times. # The file diamond.shape may contain a pattern of characters like this # * # *** # ***** # *** # * # The specified columns and rows are ignored # but the anchor point coordinates are not ocv=dilate:0x0+2x2/custom=diamond.shape|2 </pre>

 <h4 class="subsection">32.187.2 erode# TOC</h4>

Erode an image by using a specific structuring element. It corresponds to the libopencv function <code class="code">cvErode</code>. It accepts the parameters: <var class="var">struct_el</var>:<var

class=”var”>nb_iterations</var>, with the same syntax and semantics as the dilate filter. <h4 class=”subsection”>32.187.3 smooth# TOC</h4>

Smooth the input video. The filter takes the following parameters: <var class=”var”>type</var>|<var class=”var”>param1</var>|<var class=”var”>param2</var>|<var class=”var”>param3</var>|<var class=”var”>param4</var>. <var class=”var”>type</var> is the type of smooth filter to apply, and must be one of the following values: "blur", "blur_no_scale", "median", "gaussian", or "bilateral". The default value is "gaussian". The meaning of <var class=”var”>param1</var>, <var class=”var”>param2</var>, <var class=”var”>param3</var>, and <var class=”var”>param4</var> depends on the smooth type. <var class=”var”>param1</var> and <var class=”var”>param2</var> accept integer positive values or 0. <var class=”var”>param3</var> and <var class=”var”>param4</var> accept floating point values. The default value for <var class=”var”>param1</var> is 3. The default value for the other parameters is 0. These parameters correspond to the parameters assigned to the libopencv function <code class=”code”>cvSmooth</code>. <h3 class=”section”>32.188 oscilloscope# TOC</h3>

2D Video Oscilloscope. Useful to measure spatial impulse, step responses, chroma delays, etc. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set scope center x position. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set scope center y position. </dd> <dt><samp class=”option”>s</samp></dt> <dd> Set scope size, relative to frame diagonal. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set scope tilt/rotation. </dd> <dt><samp class=”option”>o</samp></dt> <dd> Set trace opacity. </dd> <dt><samp class=”option”>tx</samp></dt> <dd> Set trace center x position. </dd> <dt><samp class=”option”>ty</samp></dt> <dd> Set trace center y position. </dd> <dt><samp class=”option”>tw</samp></dt> <dd> Set trace width, relative to width of frame. </dd> <dt><samp class=”option”>th</samp></dt> <dd> Set trace height, relative to height of frame. </dd> <dt><samp class=”option”>c</samp></dt> <dd> Set which components to trace. By default it traces first three components. </dd> <dt><samp class=”option”>g</samp></dt> <dd> Draw trace grid. By default is enabled. </dd> <dt><samp

class=”option”>st</samp></dt> <dd> Draw some statistics. By default is enabled. </dd> <dt><samp class=”option”>sc</samp></dt> <dd> Draw scope. By default is enabled. </dd> </dl>

 <h4 class=”subsection”>32.188.1 Commands# TOC</h4> This filter supports same commands as options. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. <h4 class=”subsection”>32.188.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Inspect full first row of video frame. <pre class=”example-preformatted”>oscilloscope=x=0.5:y=0:s=1 </pre>

Inspect full last row of video frame. <pre class=”example-preformatted”>oscilloscope=x=0.5:y=1:s=1 </pre>

Inspect full 5th line of video frame of height 1080. <pre class=”example-preformatted”>oscilloscope=x=0.5:y=5/1080:s=1 </pre>

Inspect full last column of video frame. <pre class=”example-preformatted”>oscilloscope=x=1:y=0.5:s=1:t=1 </pre>

 <h3 class=”section”>32.189 overlay# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the expression for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. In case the expression is invalid, it is set to a huge value (meaning that the overlay will not be displayed within the output visible area). </dd> <dt><samp class=”option”>eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp

class=”option”>eval</samp></dt> <dd> Set when the expressions for <samp class=”option”>x</samp>, and <samp class=”option”>y</samp> are evaluated. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class=”samp”>frame</samp>’. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Set the format for the output video. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>yuv420</samp>’</dt> <dd> force YUV 4:2:0 8-bit planar output </dd> <dt>‘<samp class=”samp”>yuv420p10</samp>’</dt> <dd> force YUV 4:2:0 10-bit planar output </dd> <dt>‘<samp class=”samp”>yuv422</samp>’</dt> <dd> force YUV 4:2:2 8-bit planar output </dd> <dt>‘<samp class=”samp”>yuv422p10</samp>’</dt> <dd> force YUV 4:2:2 10-bit planar output </dd> <dt>‘<samp class=”samp”>yuv444</samp>’</dt> <dd> force YUV 4:4:4 8-bit planar output </dd> <dt>‘<samp class=”samp”>yuv444p10</samp>’</dt> <dd> force YUV 4:4:4 10-bit planar output </dd> <dt>‘<samp class=”samp”>rgb</samp>’</dt> <dd> force RGB 8-bit packed output </dd> <dt>‘<samp class=”samp”>gbrp</samp>’</dt> <dd> force RGB 8-bit planar output </dd> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> automatically pick format </dd> </dl>

Default value is ‘<samp class=”samp”>yuv420</samp>’. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>alpha</samp></dt> <dd> Set format of alpha of the overlaid video, it can be <var class=”var”>straight</var> or <var class=”var”>premultiplied</var>, or <var class=”var”>auto</var> to choose the alpha mode automatically. Default is <var class=”var”>auto</var>. </dd> </dl>

The <samp class=”option”>x</samp>, and <samp class=”option”>y</samp> expressions can contain the following parameters. <dl class=”table”> <dt><samp class=”option”>main_w, W</samp></dt> <dt><samp class=”option”>main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class=”option”>overlay_w, w</samp></dt> <dt><samp class=”option”>overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var

class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values of the output format. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class=”option”>pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp, expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> </dl>

This filter also supports the framesync options. Note that the <var class=”var”>n</var>, <var class=”var”>t</var> variables are available only when evaluation is done per frame, and will evaluate to NAN when <samp class=”option”>eval</samp> is set to ‘<samp class=”samp”>init</samp>’. Be aware that frames are taken from each input video in timestamp order, hence, if their initial timestamps differ, it is a good idea to pass the two inputs through a <var class=”var”>setpts=PTS-STARTPTS</var> filter to have them begin in the same zero timestamp, as the example for the <var class=”var”>movie</var> filter does. You can chain together more overlays but you should test the efficiency of such approach. <h4 class=”subsection”>32.189.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Modify the x and y of the overlay input. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h4 class=”subsection”>32.189.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Draw the overlay at 10 pixels from the bottom right corner of the main video: <pre class=”example-preformatted”>overlay=main_w-overlay_w-10:main_h-overlay_h-10 </pre>

Using named options the example above becomes: <pre class=”example-preformatted”>overlay=x=main_w-overlay_w-10:y=main_h-overlay_h-10 </pre>

Insert a transparent PNG logo in the bottom left corner of the input, using the <code class="command">ffmpeg</code> tool with the <code class="code">-filter_complex</code> option: <pre class="example-preformatted">ffmpeg -i input -i logo -filter_complex 'overlay=10:main_h-overlay_h-10' output </pre>

Insert 2 different transparent PNG logos (second logo on bottom right corner) using the <code class="command">ffmpeg</code> tool: <pre class="example-preformatted">ffmpeg -i input -i logo1 -i logo2 -filter_complex 'overlay=x=10:y=H-h-10,overlay=x=W-w-10:y=H-h-10' output </pre>

Add a transparent color layer on top of the main video; <code class="code">WxH</code> must specify the size of the main input to the overlay filter: <pre class="example-preformatted">color=color=red@.3:size=WxH [over]; [in][over] overlay [out] </pre>

Play an original video and a filtered version (here with the deshake filter) side by side using the <code class="command">ffplay</code> tool: <pre class="example-preformatted">ffplay input.avi -vf 'split[a][b]; [a]pad=iw*2:ih[src]; [b]deshake[filt]; [src][filt]overlay=w' </pre>

The above command is the same as: <pre class="example-preformatted">ffplay input.avi -vf 'split[b], pad=iw*2[src], [b]deshake, [src]overlay=w' </pre>

Make a sliding overlay appearing from the left to the right top part of the screen starting since time 2: <pre class="example-preformatted">overlay=x='if(gte(t,2), -w+(t-2)*20, NAN)':y=0 </pre>

Compose output by putting two input videos side to side: <pre class="example-preformatted">ffmpeg -i left.avi -i right.avi -filter_complex " nullsrc=size=200x100 [background]; [0:v] setpts=PTS-STARTPTS, scale=100x100 [left]; [1:v] setpts=PTS-STARTPTS, scale=100x100 [right]; [background][left] overlay=shortest=1 [background+left]; [background+left][right] overlay=shortest=1:x=100 [left+right] " </pre>

Mask 10-20 seconds of a video by applying the delogo filter to a section <pre class="example-preformatted">ffmpeg -i test.avi -codec:v:0 wmv2 -ar 11025 -b:v 9000k -vf '[in]split[split_main][split_delogo];[split_delogo]trim=start=360:end=
masked.avi </pre>

Chain several overlays in cascade: <pre class="example-preformatted">nullsrc=s=200x200 [bg]; testsrc=s=100x100, split=4 [in0][in1][in2][in3]; [in0] lutrgb=r=0, [bg] overlay=0:0 [mid0]; [in1]

lutrgb=g=0, [mid0] overlay=100:0 [mid1]; [in2] lutrgb=b=0, [mid1] overlay=0:100 [mid2]; [in3] null, [mid2] overlay=100:100 [out0] </pre>

 <h3 class="section">32.190 owdenoise# TOC</h3>

Apply Overcomplete Wavelet denoiser. The filter accepts the following options: <dl class="table"> <dt><samp class="option">depth</samp></dt> <dd> Set depth. Larger depth values will denoise lower frequency components more, but slow down filtering. Must be an int in the range 8-16, default is <code class="code">8</code>. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set luma strength. Must be a double value in the range 0-1000, default is <code class="code">1.0</code>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set chroma strength. Must be a double value in the range 0-1000, default is <code class="code">1.0</code>. </dd> </dl>

 <h3 class="section">32.191 pad# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following parameters: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class="var">width</var> or <var class="var">height</var> is 0, the corresponding input size is used for the output. The <var class="var">width</var> expression can reference the value set by the <var class="var">height</var> expression, and vice versa. The default value of <var class="var">width</var> and <var class="var">height</var> is 0. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp

class=”option”>color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. The default value of <var class=”var”>color</var> is "black". </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Specify when to evaluate <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var> and <var class=”var”>y</var> expression. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class=”samp”>init</samp>’. </dd> <dt><samp class=”option”>aspect</samp></dt> <dd> Pad to aspect instead to a resolution. </dd> </dl>

The value for the <var class=”var”>width</var>, <var class=”var”>height</var>, <var class=”var”>x</var>, and <var class=”var”>y</var> options are expressions containing the following constants: <dl class=”table”> <dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt> <dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt> <dt><samp class=”option”>oh</samp></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">32.191.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Add paddings with the color "violet" to the input video. The output video size is 640x480, and the top-left corner of the input video is placed at column 0, row 40 <pre class="example-preformatted">pad=640:480:0:40:violet
</pre>

The example above is equivalent to the following command: <pre class="example-preformatted">pad=width=640:height=480:x=0:y=40:color=violet
</pre>

Pad the input to get an output with dimensions increased by 3/2, and put the input video at the center of the padded area: <pre class="example-preformatted">pad="3/2*iw:3/2*ih:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a squared output with size equal to the maximum value between the input width and height, and put the input video at the center of the padded area: <pre class="example-preformatted">pad="max(iw\,ih):ow:(ow-iw)/2:(oh-ih)/2" </pre>

Pad the input to get a final w/h ratio of 16:9: <pre class="example-preformatted">pad="ih*16/9:ih:(ow-iw)/2:(oh-ih)/2" </pre>

In case of anamorphic video, in order to set the output display aspect correctly, it is necessary to use <var class="var">sar</var> in the expression, according to the relation: <pre class="example-preformatted">(ih * X / ih) * sar = output_dar X = output_dar / sar
</pre>

Thus the previous example needs to be modified to: <pre class="example-preformatted">pad="ih*16/9/sar:ih:(ow-iw)/2:(oh-ih)/2" </pre>

Double the output size and put the input video in the bottom-right corner of the output padded area: <pre class="example-preformatted">pad="2*iw:2*ih:ow-iw:oh-ih"
</pre>

 <h3 class="section">32.192 palettegen# TOC</h3>

Generate one palette for a whole video stream. It accepts the following options: <dl class="table"> <dt><samp class="option">max_colors</samp></dt> <dd> Set the maximum number of colors to quantize in the palette. Note: the palette will still contain 256 colors; the unused palette entries will be black. </dd> <dt><samp class="option">reserve_transparent</samp></dt> <dd> Create a palette of 255 colors maximum and reserve the last one for transparency. Reserving the transparency color is useful for GIF optimization. If not set, the maximum of colors in the palette will be 256. You probably want to disable this option for a standalone image. Set by default. </dd> <dt><samp class="option">transparency_color</samp></dt> <dd> Set the color that will be used as background for transparency. </dd> <dt><samp class="option">stats_mode</samp></dt> <dd> Set statistics mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Compute full frame histograms. </dd> <dt>‘<samp class="samp">diff</samp>’</dt> <dd> Compute histograms only for the part that differs from previous frame. This might be relevant to give more importance to the moving part of your input if the background is static. </dd> <dt>‘<samp class="samp">single</samp>’</dt> <dd> Compute new histogram for each frame. </dd> </dl>

Default value is <var class="var">full</var>. </dd> </dl>

The filter also exports the frame metadata <code class="code">lavfi.color_quant_ratio</code> (<code class="code">nb_color_in / nb_color_out</code>) which you can use to evaluate the degree of color quantization of the palette. This information is also visible at <var class="var">info</var> logging level.
 <h4 class="subsection">32.192.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a representative palette of a given video using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -vf palettegen palette.png </pre>

 <h3 class="section">32.193 paletteuse# TOC</h3>

Use a palette to downsample an input video stream. The filter takes two inputs: one video stream and a palette. The palette must be a 256 pixels image. It accepts the following options: <dl class="table"> <dt><samp class="option">dither</samp></dt> <dd> Select dithering mode. Available algorithms are: <dl

class=”table”> <dt>‘<samp class=”samp”>bayer</samp>’</dt>
<dd> Ordered 8x8 bayer dithering (deterministic) </dd>
<dt>‘<samp class=”samp”>heckbert</samp>’</dt> <dd> Dithering
as defined by Paul Heckbert in 1982 (simple error diffusion). Note: this dither-
ing is sometimes considered "wrong" and is included as a reference.
</dd> <dt>‘<samp class=”samp”>floyd_steinberg</samp>’</dt>
<dd> Floyd and Steingberg dithering (error diffusion) </dd>
<dt>‘<samp class=”samp”>sierra2</samp>’</dt> <dd> Frankie
Sierra dithering v2 (error diffusion) </dd> <dt>‘<samp
class=”samp”>sierra2_4a</samp>’</dt> <dd> Frankie
Sierra dithering v2 "Lite" (error diffusion) </dd>
<dt>‘<samp class=”samp”>sierra3</samp>’</dt> <dd> Frankie
Sierra dithering v3 (error diffusion) </dd> <dt>‘<samp
class=”samp”>burkes</samp>’</dt> <dd> Burkes dithering (er-
ror diffusion) </dd> <dt>‘<samp class=”samp”>atkinson</samp>’</dt>
<dd> Atkinson dithering by Bill Atkinson at Apple Computer (error diffu-
sion) </dd> <dt>‘<samp class=”samp”>none</samp>’</dt>
<dd> Disable dithering. </dd> </dl>

Default is <var class=”var”>sierra2_4a</var>. </dd>
<dt><samp class=”option”>bayer_scale</samp></dt> <dd> When
<var class=”var”>bayer</var> dithering is selected, this option defines the
scale of the pattern (how much the crosshatch pattern is visible). A low value
means more visible pattern for less banding, and higher value means less
visible pattern at the cost of more banding. The option must be an
integer value in the range [0,5]. Default is <var class=”var”>2</var>.
</dd> <dt><samp class=”option”>diff_mode</samp></dt> <dd> If
set, define the zone to process <dl class=”table”> <dt>‘<samp
class=”samp”>rectangle</samp>’</dt> <dd> Only the chang-
ing rectangle will be reprocessed. This is similar to GIF cropping/offsetting
compression mechanism. This option can be useful for speed if only a part
of the image is changing, and has use cases such as limiting the scope of the
error diffusal <samp class=”option”>dither</samp> to the rectangle that
bounds the moving scene (it leads to more deterministic output if the scene
doesn’t change much, and as a result less moving noise and better GIF
compression). </dd> </dl>

Default is <var class=”var”>none</var>. </dd> <dt><samp
class=”option”>new</samp></dt> <dd> Take new palette for each out-
put frame. </dd> <dt><samp class=”option”>alpha_threshold</samp></dt>
<dd> Sets the alpha threshold for transparency. Alpha values above this
threshold will be treated as completely opaque, and values below this threshold
will be treated as completely transparent. The option must be an
integer value in the range [0,255]. Default is <var class=”var”>128</var>.
</dd> </dl>

 <h4 class=”subsection”>32.193.1 Exam-

ples# TOC</h4>

<ul class="itemize mark-bullet"> Use a palette (generated for example with palettegen) to encode a GIF using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input.mkv -i palette.png -lavfi paletteuse output.gif </pre>

 <h3 class="section">32.194 perspective# TOC</h3>

Correct perspective of video not recorded perpendicular to the screen. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">x0</samp></dt> <dt><samp class="option">y0</samp></dt> <dt><samp class="option">x1</samp></dt> <dt><samp class="option">y1</samp></dt> <dt><samp class="option">x2</samp></dt> <dt><samp class="option">y2</samp></dt> <dt><samp class="option">x3</samp></dt> <dt><samp class="option">y3</samp></dt> <dd> Set coordinates expression for top left, top right, bottom left and bottom right corners. Default values are <code class="code">0:0:W:0:0:H:W:H</code> with which perspective will remain unchanged. If the <code class="code">sense</code> option is set to <code class="code">source</code>, then the specified points will be sent to the corners of the destination. If the <code class="code">sense</code> option is set to <code class="code">destination</code>, then the corners of the source will be sent to the specified coordinates. The expressions can use the following variables: <dl class="table"> <dt><samp class="option">W</samp></dt> <dt><samp class="option">H</samp></dt> <dd> the width and height of video frame. </dd> <dt><samp class="option">in</samp></dt> <dd> Input frame count. </dd> <dt><samp class="option">on</samp></dt> <dd> Output frame count. </dd> </dl>

</dd> <dt><samp class="option">interpolation</samp></dt> <dd> Set interpolation for perspective correction. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">cubic</samp>’</dt> </dl>

Default value is ‘<samp class="samp">linear</samp>’. </dd> <dt><samp class="option">sense</samp></dt> <dd> Set interpretation of coordinate options. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">0, source</samp>’</dt> <dd> Send point in the source specified by the given coordinates to the corners of the destination. </dd> <dt>‘<samp class="samp">1, destination</samp>’</dt>

<dd> Send the corners of the source to the point in the destination specified by the given coordinates. Default value is ‘<samp class="samp">source</samp>’. </dd> </dl>

</dd> <dt><samp class="option">eval</samp></dt> <dd> Set when the expressions for coordinates <samp class="option">x0,y0,...x3,y3</samp> are evaluated. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> only evaluate expressions once during the filter initialization or when a command is processed </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> evaluate expressions for each incoming frame </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’. </dd> </dl>

 <h3 class="section">32.195 phase# TOC</h3>

Delay interlaced video by one field time so that the field order changes. The intended use is to fix PAL movies that have been captured with the opposite field order to the film-to-video transfer. A description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set phase mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">t</samp>’</dt> <dd> Capture field order top-first, transfer bottom-first. Filter will delay the bottom field. </dd> <dt>‘<samp class="samp">b</samp>’</dt> <dd> Capture field order bottom-first, transfer top-first. Filter will delay the top field. </dd> <dt>‘<samp class="samp">p</samp>’</dt> <dd> Capture and transfer with the same field order. This mode only exists for the documentation of the other options to refer to, but if you actually select it, the filter will faithfully do nothing. </dd> <dt>‘<samp class="samp">a</samp>’</dt> <dd> Capture field order determined automatically by field flags, transfer opposite. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp class="samp">b</samp>’ modes on a frame by frame basis using field flags. If no field information is available, then this works just like ‘<samp class="samp">u</samp>’. </dd> <dt>‘<samp class="samp">u</samp>’</dt> <dd> Capture unknown or varying, transfer opposite. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp class="samp">b</samp>’ on a frame by frame basis by analyzing the images and selecting the alternative that produces best match between the fields. </dd> <dt>‘<samp class="samp">T</samp>’</dt> <dd> Capture top-first, transfer unknown or varying. Filter selects among ‘<samp class="samp">t</samp>’ and ‘<samp

class=”samp”>p</samp>’ using image analysis. </dd> <dt>‘<samp class=”samp”>B</samp>’</dt> <dd> Capture bottom-first, transfer unknown or varying. Filter selects among ‘<samp class=”samp”>b</samp>’ and ‘<samp class=”samp”>p</samp>’ using image analysis. </dd> <dt>‘<samp class=”samp”>A</samp>’</dt> <dd> Capture determined by field flags, transfer unknown or varying. Filter selects among ‘<samp class=”samp”>t</samp>’, ‘<samp class=”samp”>b</samp>’ and ‘<samp class=”samp”>p</samp>’ using field flags and image analysis. If no field information is available, then this works just like ‘<samp class=”samp”>U</samp>’. This is the default mode. </dd> <dt>‘<samp class=”samp”>U</samp>’</dt> <dd> Both capture and transfer unknown or varying. Filter selects among ‘<samp class=”samp”>t</samp>’, ‘<samp class=”samp”>b</samp>’ and ‘<samp class=”samp”>p</samp>’ using image analysis only. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>32.195.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.196 photosensitivity# TOC</h3> Reduce various flashes in video, so to help users with epilepsy. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>frames, f</samp></dt> <dd> Set how many frames to use when filtering. Default is 30. </dd> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set detection threshold factor. Default is 1. Lower is stricter. </dd> <dt><samp class=”option”>skip</samp></dt> <dd> Set how many pixels to skip when sampling frames. Default is 1. Allowed range is from 1 to 1024. </dd> <dt><samp class=”option”>bypass</samp></dt> <dd> Leave frames unchanged. Default is disabled. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Set the blending factor for frames that exceed the detection threshold. A value of 0 duplicates the last accepted frame, and a value of 1 blends up to the detection threshold. Default is 1. Allowed range is from 0 to 1. </dd> </dl>

 <h3 class=”section”>32.197 pixdesctest# TOC</h3>

Pixel format descriptor test filter, mainly useful for internal test-

ing. The output video should be equal to the input video. For example: <pre class="example-preformatted">format=monow, pixdesctest </pre>

can be used to test the monowhite pixel format descriptor definition. <h3 class="section">32.198 pixelize# TOC</h3> Apply pixelization to video stream. The filter accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set block dimensions that will be used for pixelization. Default value is <code class="code">16</code>. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set the mode of pixelization used. Possible values are: <dl class="table"> <dt>‘<samp class="samp">avg</samp>’</dt> <dt>‘<samp class="samp">min</samp>’</dt> <dt>‘<samp class="samp">max</samp>’</dt> </dl> Default value is <code class="code">avg</code>. </dd> <dt><samp class="option">planes, p</samp></dt> <dd> Set what planes to filter. Default is to filter all planes. </dd> </dl>

 <h4 class="subsection">32.198.1 Commands# TOC</h4>

This filter supports all options as commands. <h3 class="section">32.199 pixscope# TOC</h3>

Display sample values of color channels. Mainly useful for checking color and levels. Minimum supported resolution is 640x480. The filters accept the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set scope X position, relative offset on X axis. </dd> <dt><samp class="option">y</samp></dt> <dd> Set scope Y position, relative offset on Y axis. </dd> <dt><samp class="option">w</samp></dt> <dd> Set scope width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set scope height. </dd> <dt><samp class="option">o</samp></dt> <dd> Set window opacity. This window also holds statistics about pixel area. </dd> <dt><samp class="option">wx</samp></dt> <dd> Set window X position, relative offset on X axis. </dd> <dt><samp class="option">wy</samp></dt> <dd> Set window Y position, relative offset on Y axis. </dd> </dl>

 <h4 class="subsection">32.199.1 Commands# TOC</h4>

This filter supports same commands as options. <h3 class="section">32.200 pp7# TOC</h3> Apply Postprocessing filter 7. It is variant of the spp filter, similar to spp = 6 with 7 point DCT, where only the center sample is used after IDCT. The filter accepts the following options: <dl class="table"> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. It accepts an integer in range 0 to 63. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> Set hard thresholding. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Set medium thresholding (good results, default). </dd> </dl> </dd> </dl>

 <h3 class="section">32.201 premultiply# TOC</h3> Apply alpha premultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">32.202 premultiply_dynamic# TOC</h3> Dynamically premultiply or unpremultiply the input video stream as needed, to match the requirements of the downstream filter graph. This is roughly equivalent to either <code class="code">premultiply:inplace=yes</code> or <code class="code">unpremultiply:inplace=yes</code>, or otherwise a noop.

 The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> </dl>

 <h3 class="section">32.203 prewitt# TOC</h3> Apply prewitt operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">32.203.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.204 pseudocolor# TOC</h3>

Alter frame colors in video with pseudocolors. This filter accepts the following options: <dl class="table"> <dt><samp class="option">c0</samp></dt> <dd> set pixel first component expression </dd> <dt><samp class="option">c1</samp></dt> <dd> set pixel second component expression </dd> <dt><samp class="option">c2</samp></dt> <dd> set pixel third component expression </dd> <dt><samp class="option">c3</samp></dt> <dd> set pixel fourth component expression, corresponds to the alpha component </dd> <dt><samp class="option">index, i</samp></dt> <dd> set component to use as base for altering colors </dd> <dt><samp class="option">preset, p</samp></dt> <dd> Pick one of built-in LUTs. By default is set to none. Available LUTs: <dl class="table"> <dt>‘<samp class="samp">magma</samp>’</dt> <dt>‘<samp class="samp">inferno</samp>’<
<dt>‘<samp class="samp">plasma</samp>’</dt> <dt>‘<samp class="samp">viridis</samp>’</dt> <dt>‘<samp class="samp">turbo</samp>’</
<dt>‘<samp class="samp">cividis</samp>’</dt> <dt>‘<samp class="samp">range1</samp>’</dt> <dt>‘<samp class="samp">range2</samp>’<
<dt>‘<samp class="samp">shadows</samp>’</dt> <dt>‘<samp

class=”samp”>highlights</samp>’</dt> <dt>‘<samp class=”samp”>solar</samp>’</dt> <dt>‘<samp class=”samp”>nominal</samp>’</dt> <dt>‘<samp class=”samp”>preferred</samp>’</dt> <dt>‘<samp class=”samp”>total</samp>’</dt> <dt>‘<samp class=”samp”>spectral</samp>’</dt> <dt>‘<samp class=”samp”>cool</samp>’</dt> <dt>‘<samp class=”samp”>heat</samp>’</dt> <dt>‘<samp class=”samp”>fiery</samp>’</dt> <dt>‘<samp class=”samp”>blues</samp>’</dt> <dt>‘<samp class=”samp”>green</samp>’</dt> <dt>‘<samp class=”samp”>helix</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity of output colors. Allowed range is from 0 to 1. Default value is set to 1. </dd> </dl>

Each of the expression options specifies the expression to use for computing the lookup table for the corresponding pixel component values. The expressions can contain the following constants and functions: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height. </dd> <dt><samp class=”option”>val</samp></dt> <dd> The input value for the pixel component. </dd> <dt><samp class=”option”>ymin, umin, vmin, amin</samp></dt> <dd> The minimum allowed component value. </dd> <dt><samp class=”option”>ymax, umax, vmax, amax</samp></dt> <dd> The maximum allowed component value. </dd> </dl>

All expressions default to "val". <h4 class=”subsection”>32.204.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class=”subsection”>32.204.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Change too high luma values to gradient: <pre class=”example-preformatted”>pseudocolor="'if(between(val,ymax,amax) ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(umax,umin,(val-ymax)/(amax-ymax)),-1):if(between(val,ymax,amax),lerp(vmin,vmax,(val-ymax)/(amax-ymax)),-1):-1'" </pre>

 <h3 class=”section”>32.205 psnr# TOC</h3>

Obtain the average, maximum and minimum PSNR (Peak Signal to Noise

Ratio) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the PSNR. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average PSNR is printed through the logging system. The filter stores the accumulated MSE (mean squared error) of each frame, and at the end of the processing it is averaged across all frames equally, and the following formula is applied to obtain the PSNR: <pre class="example-preformatted">PSNR = 10*log10(MAX^2/MSE) </pre>

Where MAX is the average of the maximum values of each component of the image. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the PSNR of each individual frame. When filename equals "-" the data is sent to standard output. </dd> <dt><samp class="option">stats_version</samp></dt> <dd> Specifies which version of the stats file format to use. Details of each format are written below. Default value is 1. </dd> <dt><samp class="option">output_max</samp></dt> <dd> Determines whether the max value is output to the stats log. Default value is 0. Requires stats_version >= 2. If this is set and stats_version < 2, the filter will return an error. </dd> </dl>

This filter also supports the framesync options. The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. If a <var class="var">stats_version</var> greater than 1 is specified, a header line precedes the list of per-frame-pair stats, with key value pairs following the frame format with the following parameters: <dl class="table"> <dt><samp class="option">psnr_log_version</samp></dt> <dd> The version of the log file format. Will match <var class="var">stats_version</var>. </dd> <dt><samp class="option">fields</samp></dt> <dd> A comma separated list of the per-frame-pair parameters included in the log. </dd> </dl>

A description of each shown per-frame-pair parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">mse_avg</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames, averaged over all the image components. </dd> <dt><samp class="option">mse_y, mse_u, mse_v, mse_r, mse_g, mse_b, mse_a</samp></dt> <dd> Mean Square Error pixel-by-pixel average difference of the compared frames

for the component specified by the suffix. </dd> <dt><samp class="option">psnr_y, psnr_u, psnr_v, psnr_r, psnr_g, psnr_b, psnr_a</samp></dt> <dd> Peak Signal to Noise ratio of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">max_avg, max_y, max_u, max_v</samp></dt> <dd> Maximum allowed value for each channel, and average over all channels. </dd> </dl>

 <h4 class="subsection">32.205.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] psnr="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The PSNR of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with different containers: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mkv -lavfi "[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-STARTPTS[ref];[main][ref]psnr" -f null - </pre>

 <h3 class="section">32.206 pullup# TOC</h3>

Pulldown reversal (inverse telecine) filter, capable of handling mixed hard-telecine, 24000/1001 fps progressive, and 30000/1001 fps progressive content. The pullup filter is designed to take advantage of future context in making its decisions. This filter is stateless in the sense that it does not lock onto a pattern to follow, but it instead looks forward to the following fields in order to identify matches and rebuild progressive frames. To produce content with an even framerate, insert the fps filter after pullup, use <code class="code">fps=24000/1001</code> if the input frame rate is 29.97fps, <code class="code">fps=24</code> for 30fps and the (rare) telecined 25fps input. The filter accepts the following options: <dl class="table"> <dt><samp class="option">jl</samp></dt> <dt><samp class="option">jr</samp></dt> <dt><samp class="option">jt</samp></dt> <dt><samp class="option">jb</samp></dt> <dd> These options set the amount of "junk" to ignore at the left, right, top, and bottom of the image, respectively. Left and right are in units of 8 pixels, while top and bottom are in units of 2 lines. The default is 8 pixels on each side. </dd> <dt><samp class="option">sb</samp></dt> <dd> Set the strict breaks. Setting this option to 1 will reduce the chances of filter generating an occasional mismatched frame, but it may also cause an excessive number of frames to be dropped during high motion

sequences. Conversely, setting it to -1 will make filter match fields more easily. This may help processing of video where there is slight blurring between the fields, but may also cause there to be interlaced frames in the output. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>mp</samp></dt> <dd> Set the metric plane to use. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>l</samp>’</dt> <dd> Use luma plane. </dd> <dt>‘<samp class=”samp”>u</samp>’</dt> <dd> Use chroma blue plane. </dd> <dt>‘<samp class=”samp”>v</samp>’</dt> <dd> Use chroma red plane. </dd> </dl>

This option may be set to use chroma plane instead of the default luma plane for doing filter’s computations. This may improve accuracy on very clean source material, but more likely will decrease accuracy, especially if there is chroma noise (rainbow effect) or any grayscale video. The main purpose of setting <samp class=”option”>mp</samp> to a chroma plane is to reduce CPU load and make pullup usable in realtime on slow machines. </dd> </dl>

For best results (without duplicated frames in the output file) it is necessary to change the output frame rate. For example, to inverse telecine NTSC input: <pre class=”example-preformatted”>ffmpeg -i input -vf pullup -r 24000/1001 ... </pre>

 <h3 class=”section”>32.207 qp# TOC</h3>

Change video quantization parameters (QP). The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>qp</samp></dt> <dd> Set expression for quantization parameter. </dd> </dl>

The expression is evaluated through the eval API and can contain, among others, the following constants: <dl class=”table”> <dt><var class=”var”>known</var></dt> <dd> 1 if index is not 129, 0 otherwise. </dd> <dt><var class=”var”>qp</var></dt> <dd> Sequential index starting from -129 to 128. </dd> </dl>

 <h4 class=”subsection”>32.207.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Some equation like: <pre class=”example-preformatted”>qp=2+2*sin(PI*qp) </pre>

 <h3 class=”section”>32.208 qrencode<span

class=”pull-right”># TOC</h3> Generate a QR code using the libqrencode library (see <a class=”url” href=”https://fukuchi.org/works/qrencode/”>https://fukuchi.o and overlay it on top of the current frame. To enable the compilation of this filter, you need to configure FFmpeg with <code class=”code”>--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and overlaid into the video output according to the specified options. In case no text is specified, no QR code is overlaied. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>qrcode_width, q</samp></dt> <dt><samp class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Specify an expression for positioning the padded QR code top-left corner. The <var class=”var”>x</var> expression can reference the value set by the <var class=”var”>y</var> expression, and vice. By default <var class=”var”>x</var> and <var class=”var”>y</var> are set set to <var class=”var”>0</var>, meaning that the QR code is placed in the top left corner of the input. These expressions are evaluated for each new frame. See the qrencode Expressions section for details. </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can be disabled to reduce the QR encoding size. </dd> <dt><samp class=”option”>level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class=”var”>L</var>. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>L</samp>’</dt> <dt>‘<samp class=”samp”>M</samp>’</dt> <dt>‘<samp class=”samp”>Q</samp>’</dt> <dt>‘<samp class=”samp”>H</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class=”code”>none</code>, or <code class=”code”>normal</code> (default). See the <a class=”ref”

href=”#qrencode_005ftext_005fexpansion”>qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>text</samp></dt> <dt><samp class=”option”>textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section below for details. </dd> <dt><samp class=”option”>background_color, bc</samp></dt> <dt><samp class=”option”>foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class=”var”>foreground_color</var> is "black", the default value of <var class=”var”>background_color</var> is "white". For the syntax of the color options, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class=”subsection”>32.208.1 qrencode Expressions# TOC</h4>

The expressions set by the options contain the following constants and functions. <dl class=”table”> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var> </dd> <dt><samp class=”option”>duration</samp></dt> <dd> the current frame’s duration, in seconds </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><samp class=”option”>main_h, H</samp></dt> <dd> the input height </dd> <dt><samp class=”option”>main_w, W</samp></dt> <dd> the input width </dd> <dt><samp class=”option”>n</samp></dt> <dd> the number of input frame, starting from 0 </dd> <dt><samp class=”option”>pict_type</samp></dt> <dd> a number representing the picture type </dd> <dt><samp class=”option”>qr_w, w</samp></dt> <dd> the width of the encoded QR code </dd> <dt><samp class=”option”>rendered_qr_w, q</samp></dt> <dt><samp class=”option”>rendered_padded_qr_w, Q</samp></dt> <dd> the width of the rendered QR code, without and without padding. These parameters allow the <var class=”var”>q</var> and <var class=”var”>Q</var> expressions to refer to each other, so you can for example specify <code class=”code”>q=3/4*Q</code>. </dd> <dt><samp class=”option”>rand(min, max)</samp></dt> <dd> return a ran-

dom number included between <var class=”var”>min</var> and <var class=”var”>max</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> the input sample aspect ratio </dd> <dt><samp class=”option”>t</samp></dt> <dd> timestamp expressed in seconds, NAN if the input timestamp is unknown </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> the x and y offset coordinates where the text is drawn. These parameters allow the <var class=”var”>x</var> and <var class=”var”>y</var> expressions to refer to each other, so you can for example specify <code class=”code”>y=x/dar</code>. </dd> </dl>

 <h4 class=”subsection”>32.208.2 qrencode Text expansion# TOC</h4>

If <samp class=”option”>expansion</samp> is set to <code class=”code”>none</code>, the text is printed verbatim. If <samp class=”option”>expansion</samp> is set to <code class=”code”>normal</code> (which is the default), the following expansion mechanism is used. The backslash character ‘<samp class=”samp”>\</samp>’, followed by any character, always expands to the second character. Sequences of the form <code class=”code”>%{...}</code> are expanded. The text between the braces is a function name, possibly followed by arguments separated by ’:’. If the arguments contain special characters or delimiters (’:’ or ’}’), they should be escaped. Note that they probably must also be escaped as the value for the <samp class=”option”>text</samp> option in the filter argument string and as the filter argument in the filtergraph description, and possibly also for the shell, that makes up to four levels of escaping; using a text file with the <samp class=”option”>textfile</samp> option avoids these problems. The following functions are available: <dl class=”table”> <dt><code class=”command”>n, frame_num</code></dt> <dd> return the frame number </dd> <dt><code class=”command”>pts</code></dt> <dd> Return the presentation timestamp of the current frame. It can take up to two arguments. The first argument is the format of the timestamp; it defaults to <code class=”code”>flt</code> for seconds as a decimal number with microsecond accuracy; <code class=”code”>hms</code> stands for a formatted <var class=”var”>[-]HH:MM:SS.mmm</var> timestamp with millisecond accuracy. <code class=”code”>gmtime</code> stands for the timestamp of the frame formatted as UTC time; <code class=”code”>localtime</code> stands for the timestamp of the frame formatted as local time zone time. If the format is set to <code class=”code”>hms24hh</code>, the time is formatted in 24h format (00-23). The second argument is an offset added to the timestamp. If the format is set to <code class=”code”>localtime</code>

or <code class=”code”>gmtime</code>, a third argument may be supplied: a <code class=”code”>strftime</code> C function format string. By default, <var class=”var”>YYYY-MM-DD HH:MM:SS</var> format will be used. </dd> <dt><code class=”command”>expr, e</code></dt> <dd> Evaluate the expression’s value and output as a double. It must take one argument specifying the expression to be evaluated, accepting the constants and functions defined in qrencode_expressions. </dd> <dt><code class=”command”>expr_formatted, ef</code></dt> <dd> Evaluate the expression’s value and output as a formatted string. The first argument is the expression to be evaluated, just as for the <var class=”var”>expr</var> function. The second argument specifies the output format. Allowed values are ‘<samp class=”samp”>x</samp>’, ‘<samp class=”samp”>X</samp>’, ‘<samp class=”samp”>d</samp>’ and ‘<samp class=”samp”>u</samp>’. They are treated exactly as in the <code class=”code”>printf</code> function. The third parameter is optional and sets the number of positions taken by the output. It can be used to add padding with zeros from the left. </dd> <dt><code class=”command”>gmtime</code></dt> <dd> The time at which the filter is running, expressed in UTC. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>localtime</code></dt> <dd> The time at which the filter is running, expressed in the local time zone. It can accept an argument: a <code class=”code”>strftime</code> C function format string. The format string is extended to support the variable <var class=”var”>%[1-6]N</var> which prints fractions of the second with optionally specified number of digits. </dd> <dt><code class=”command”>metadata</code></dt> <dd> Frame metadata. Takes one or two arguments. The first argument is mandatory and specifies the metadata key. The second argument is optional and specifies a default value, used when the metadata key is not found or empty. Available metadata can be identified by inspecting entries starting with TAG included within each frame section printed by running <code class=”code”>ffprobe -show_frames</code>. String metadata generated in filters leading to the qrencode filter are also available. </dd> <dt><code class=”command”>rand(min, max)</code></dt> <dd> return a random number included between <var class=”var”>min</var> and <var class=”var”>max</var> </dd> </dl>

 <h4 class=”subsection”>32.208.3 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-132" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size, overalaid in the top left corner of the input video, with the default size: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:bc=pink@0.5:fc=blue </pre>

Place the QR code in the bottom right corner of the input video: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:x=W-Q:y=H-Q </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding, making the QR code width 3/4 of the padded width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Make the QR code a fraction of the input video width: <pre class="example-preformatted">qrencode=text=www.ffmpeg.org:q=W/5 </pre>

Generate a QR code encoding the frame number: <pre class="example-preformatted">qrencode=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class="example-preformatted">qrencode=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class="example-preformatted">qrencode=text=%{pts} </pre>

 <h3 class="section">32.209 quirc# TOC</h3>

Identify and decode a QR code using the libquirc library (see https://github.com/dlbeer/quirc/), and print the identified QR codes positions and payload as metadata. To enable the compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libquirc</code>. For each found QR code in the input video, some metadata entries are added with the prefix <var

class=”var”>lavfi.quirc.N</var>, where <var class=”var”>N</var> is the index, starting from 0, associated to the QR code. A description of each metadata value follows: <dl class=”table”> <dt><samp class=”option”>lavfi.quirc.count</samp></dt> <dd> the number of found QR codes, it is not set in case none was found </dd> <dt><samp class=”option”>lavfi.quirc.N.corner.M.x</samp></dt> <dt><samp class=”option”>lavfi.quirc.N.coreer.M.y</samp></dt> <dd> the x/y positions of the four corners of the square containing the QR code, where <var class=”var”>M</var> is the index of the corner starting from 0 </dd> <dt><samp class=”option”>lavfi.quirc.N.payload</samp></dt> <dd> the payload of the QR code </dd> </dl>

 <h3 class=”section”>32.210 random# TOC</h3>

Flush video frames from internal cache of frames into a random order. No frame is discarded. Inspired by frei0r nervous filter. <dl class=”table”> <dt><samp class=”option”>frames</samp></dt> <dd> Set size in number of frames of internal cache, in range from <code class=”code”>2</code> to <code class=”code”>512</code>. Default is <code class=”code”>30</code>. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed for random number generator, must be an integer included between <code class=”code”>0</code> and <code class=”code”>UINT32_MAX</code>. If not specified, or if explicitly set to less than <code class=”code”>0</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

 <h3 class=”section”>32.211 readeia608# TOC</h3>

Read closed captioning (EIA-608) information from the top lines of a video frame. This filter adds frame metadata for <code class=”code”>lavfi.readeia608.X.cc</code> and <code class=”code”>lavfi.readeia608.X.line</code>, where <code class=”code”>X</code> is the number of the identified line with EIA-608 data (starting from 0). A description of each metadata value follows: <dl class=”table”> <dt><samp class=”option”>lavfi.readeia608.X.cc</samp></dt> <dd> The two bytes stored as EIA-608 data (printed in hexadecimal). </dd> <dt><samp class=”option”>lavfi.readeia608.X.line</samp></dt> <dd> The number of the line on which the EIA-608 data was identified and read. </dd> </dl>

This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>scan_min</samp></dt> <dd> Set the line to start scanning for EIA-608 data. Default is <code class=”code”>0</code>.

 </dd> <dt><samp class="option">scan_max</samp></dt> <dd> Set the line to end scanning for EIA-608 data. Default is <code class="code">29</code>. </dd> <dt><samp class="option">spw</samp></dt> <dd> Set the ratio of width reserved for sync code detection. Default is <code class="code">0.27</code>. Allowed range is <code class="code">[0.1 - 0.7]</code>. </dd> <dt><samp class="option">chp</samp></dt> <dd> Enable checking the parity bit. In the event of a parity error, the filter will output <code class="code">0x00</code> for that character. Default is false. </dd> <dt><samp class="option">lp</samp></dt> <dd> Lowpass lines prior to further processing. Default is enabled. </dd> </dl>

 <h4 class="subsection">32.211.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h4 class="subsection">32.211.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output a csv with presentation time and the first two lines of identified EIA-608 captioning data. <pre class="example-preformatted">ffprobe -f lavfi -i movie=captioned_video.mov,readeia608 -show_entries frame=pts_time:frame_tags=lavfi.readeia608.0.cc,lavf -of csv </pre>

 <h3 class="section">32.212 readvitc# TOC</h3>

Read vertical interval timecode (VITC) information from the top lines of a video frame. The filter adds frame metadata key <code class="code">lavfi.readvitc.tc_str</code> with the timecode value, if a valid timecode has been detected. Further metadata key <code class="code">lavfi.readvitc.found</code> is set to 0/1 depending on whether timecode data has been found or not. This filter accepts the following options: <dl class="table"> <dt><samp class="option">scan_max</samp></dt> <dd> Set the maximum number of lines to scan for VITC data. If the value is set to <code class="code">-1</code> the full video frame is scanned. Default is <code class="code">45</code>. </dd> <dt><samp class="option">thr_b</samp></dt> <dd> Set the luma threshold for black. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.2</code>. The value must be equal or less

than <code class="code">thr_w</code>. </dd> <dt><samp class="option">thr_w</samp></dt> <dd> Set the luma threshold for white. Accepts float numbers in the range [0.0,1.0], default value is <code class="code">0.6</code>. The value must be equal or greater than <code class="code">thr_b</code>. </dd> </dl>

 <h4 class="subsection">32.212.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Detect and draw VITC data onto the video frame; if no valid VITC is detected, draw <code class="code">--:--:--:--</code> as a placeholder: <pre class="example-preformatted">ffmpeg -i input.avi -filter:v 'readvitc,drawtext=fontfile=FreeMono.ttf:text=%{metadata\\:lavfi.readvitc.tc_str\\:--:--:--:--}:x=(w-tw)/2:y=400-ascent' </pre>

 <h3 class="section">32.213 remap# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 16bit depth, single channel. <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> Specify pixel format of output from this filter. Can be <code class="code">color</code> or <code class="code">gray</code>. Default is <code class="code">color</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">32.214 removegrain# TOC</h3>

The removegrain filter is a spatial denoiser for progressive video. <dl class="table"> <dt><samp class="option">m0</samp></dt> <dd> Set mode for the first plane. </dd> <dt><samp class="option">m1</samp></dt> <dd> Set mode for the second

plane. </dd> <dt><samp class="option">m2</samp></dt> <dd> Set mode for the third plane. </dd> <dt><samp class="option">m3</samp></dt> <dd> Set mode for the fourth plane. </dd> </dl>

Range of mode is from 0 to 24. Description of each mode follows:

<dl class="table">

<dt><var class="var">0</var></dt>

<dd> Leave input plane unchanged. Default.

</dd>

<dt><var class="var">1</var></dt>

<dd> Clips the pixel with the minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">2</var></dt>

<dd> Clips the pixel with the second minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">3</var></dt>

<dd> Clips the pixel with the third minimum and maximum of the 8 neighbour pixels.

</dd>

<dt><var class="var">4</var></dt>

<dd> Clips the pixel with the fourth minimum and maximum of the 8 neighbour pixels.

This is equivalent to a median filter.

</dd>

<dt><var class="var">5</var></dt>

<dd> Line-sensitive clipping giving the minimal change.

</dd>

<dt><var class="var">6</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">7</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">8</var></dt>

<dd> Line-sensitive clipping, intermediate.

</dd>

<dt><var class="var">9</var></dt>

<dd> Line-sensitive clipping on a line where the neighbours pixels are the closest.

</dd>

<dt><var class="var">10</var></dt>

<dd> Replaces the target pixel with the closest neighbour.

</dd>

<dt><var class="var">11</var></dt>

<dd> [1 2 1] horizontal and vertical kernel blur.

</dd>

<dt><var class="var">12</var></dt>

<dd> Same as mode 11.

</dd>

<dt><var class="var">13</var></dt>

<dd> Bob mode, interpolates top field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class="var">14</var></dt>

<dd> Bob mode, interpolates bottom field from the line where the neighbours

pixels are the closest.

</dd>

<dt><var class="var">15</var></dt>

<dd> Bob mode, interpolates top field. Same as 13 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">16</var></dt>

<dd> Bob mode, interpolates bottom field. Same as 14 but with a more complicated

interpolation formula.

</dd>

<dt><var class="var">17</var></dt>

<dd> Clips the pixel with the minimum and maximum of respectively the maximum and

minimum of each pair of opposite neighbour pixels.

</dd>

<dt><var class="var">18</var></dt>

<dd> Line-sensitive clipping using opposite neighbours whose greatest distance from

the current pixel is minimal.

</dd>

<dt><var class=”var”>19</var></dt>

<dd> Replaces the pixel with the average of its 8 neighbours.

</dd>

<dt><var class=”var”>20</var></dt>

<dd> Averages the 9 pixels ([1 1 1] horizontal and vertical blur).

</dd>

<dt><var class=”var”>21</var></dt>

<dd> Clips pixels using the averages of opposite neighbour.

</dd>

<dt><var class=”var”>22</var></dt>

<dd> Same as mode 21 but simpler and faster.

</dd>

<dt><var class=”var”>23</var></dt>

<dd> Small edge and halo removal, but reputed useless.

</dd>

<dt><var class=”var”>24</var></dt>

<dd> Similar as 23.

</dd>

</dl>

 <h3 class="section">32.215 removelogo# TOC</h3>

Suppress a TV station logo, using an image file to determine which pixels comprise the logo. It works by filling in the pixels that comprise the logo with neighboring pixels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filter bitmap file, which can be any image format supported by libavformat. The width and height of the image file must match those of the video stream being processed. </dd> </dl>

Pixels in the provided bitmap image with a value of zero are not considered part of the logo, non-zero pixels are considered part of the logo. If you use white (255) for the logo and black (0) for the rest, you will be safe. For making the filter bitmap, it is recommended to take a screen capture of a black frame with the logo visible, and then using a threshold filter followed by the erode filter once or twice. If needed, little splotches can be fixed manually. Remember that if logo pixels are not covered, the filter quality will be much reduced. Marking too many pixels as part of the logo does not hurt as much, but it will increase the amount of blurring needed to cover over the image and will destroy more information than necessary, and extra pixels will slow things down on a large logo. <h3 class="section">32.216 repeatfields# TOC</h3>

This filter uses the repeat_field flag from the Video ES headers and hard repeats fields based on its value. <h3 class="section">32.217 reverse# TOC</h3>

Reverse a video clip. Warning: This filter requires memory to buffer the entire clip, so trimming is suggested. <h4 class="subsection">32.217.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Take the first 5 seconds of a clip, and reverse it. <pre class="example-preformatted">trim=end=5,reverse </pre>

 <h3 class="section">32.218 rgbashift# <a class="anchor hidden-xs"href="#toc-rgbashift"

aria-hidden=”true”>TOC</h3> Shift R/G/B/A pixels horizontally and/or vertically. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>rh</samp></dt> <dd> Set amount to shift red horizontally. </dd> <dt><samp class=”option”>rv</samp></dt> <dd> Set amount to shift red vertically. </dd> <dt><samp class=”option”>gh</samp></dt> <dd> Set amount to shift green horizontally. </dd> <dt><samp class=”option”>gv</samp></dt> <dd> Set amount to shift green vertically. </dd> <dt><samp class=”option”>bh</samp></dt> <dd> Set amount to shift blue horizontally. </dd> <dt><samp class=”option”>bv</samp></dt> <dd> Set amount to shift blue vertically. </dd> <dt><samp class=”option”>ah</samp></dt> <dd> Set amount to shift alpha horizontally. </dd> <dt><samp class=”option”>av</samp></dt> <dd> Set amount to shift alpha vertically. </dd> <dt><samp class=”option”>edge</samp></dt> <dd> Set edge mode, can be <var class=”var”>smear</var>, default, or <var class=”var”>warp</var>. </dd> </dl>

 <h4 class=”subsection”>32.218.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.219 roberts# TOC</h3> Apply roberts cross operator to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class=”subsection”>32.219.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.220 rotate# TOC</h3>

Rotate video by an arbitrary angle expressed in radians. The filter accepts the following options: A description of the optional parameters follows. <dl class="table"> <dt><samp class="option">angle, a</samp></dt> <dd> Set an expression for the angle by which to rotate the input video clockwise, expressed as a number of radians. A negative value will result in a counter-clockwise rotation. By default it is set to "0". This expression is evaluated for each frame. </dd> <dt><samp class="option">out_w, ow</samp></dt> <dd> Set the output width expression, default value is "iw". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">out_h, oh</samp></dt> <dd> Set the output height expression, default value is "ih". This expression is evaluated just once during configuration. </dd> <dt><samp class="option">bilinear</samp></dt> <dd> Enable bilinear interpolation if set to 1, a value of 0 disables it. Default value is 1. </dd> <dt><samp class="option">fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the rotated image. For the general syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> </dl>

The expressions for the angle and the output size can contain the following constants and functions: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 0. It is always NAN before the first frame is filtered. </dd> <dt><samp class="option">t</samp></dt> <dd> time in seconds of the input frame, it is set to 0 when the filter is configured. It is always NAN before the first frame is filtered. </dd> <dt><samp class="option">hsub</samp></dt> <dt><samp class="option">vsub</samp></dt> <dd> horizontal and vertical chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><samp class="option">in_w, iw</samp></dt> <dt><samp class="option">in_h, ih</samp></dt> <dd> the input video width and height </dd> <dt><samp class="option">out_w, ow</samp></dt> <dt><samp class="option">out_h, oh</samp></dt> <dd> the output width and height, that is the size of the padded area as specified by the <var class="var">width</var> and <var class="var">height</var> expressions </dd> <dt><samp class="option">rotw(a)</samp></dt> <dt><samp class="option">roth(a)</samp></dt> <dd> the minimal width/height required for completely containing the input video rotated by <var class="var">a</var> radians. These are only available when computing the <samp class="option">out_w</samp> and <samp class="option">out_h</samp> expressions. </dd> </dl>

 <h4 class="subsection">32.220.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Rotate the input by PI/6 radians clockwise: <pre class="example-preformatted">rotate=PI/6
</pre>

Rotate the input by PI/6 radians counter-clockwise: <pre class="example-preformatted">rotate=-PI/6
</pre>

Rotate the input by 45 degrees clockwise:
<pre class="example-preformatted">rotate=45*PI/180 </pre>

Apply a constant rotation with period T, starting from an angle of PI/3: <pre class="example-preformatted">rotate=PI/3+2*PI*t/T
</pre>

Make the input video rotation oscillating with a period of T seconds and an amplitude of A radians: <pre class="example-preformatted">rotate=A*sin(2*PI/T*t) </pre>

Rotate the video, output size is chosen so that the whole rotating input video is always completely contained in the output:
<pre class="example-preformatted">rotate='2*PI*t:ow=hypot(iw,ih):oh=ow'
</pre>

Rotate the video, reduce the output size so that no background is ever shown: <pre class="example-preformatted">rotate=2*PI*t:ow='min(iw,ih)/sqrt(2)':oh=ow:c=none
</pre>

 <h4 class="subsection">32.220.2 Commands# TOC</h4>

The filter supports the following commands: <dl class="table">
<dt><samp class="option">a, angle</samp></dt> <dd> Set the angle expression. The command accepts the same syntax of the corresponding option.
 If the specified expression is not valid, it is kept at its current value.
</dd> </dl>

 <h3 class="section">32.221 sab#
TOC</h3>

Apply Shape Adaptive Blur. The filter accepts the following options: <dl class="table"> <dt><samp class="option">luma_radius,

lr</samp></dt> <dd> Set luma blur filter strength, must be a value in range 0.1-4.0, default value is 1.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>luma_pre_filter_radius, lpfr</samp></dt> <dd> Set luma pre-filter radius, must be a value in the 0.1-2.0 range, default value is 1.0. </dd> <dt><samp class=”option”>luma_strength, ls</samp></dt> <dd> Set luma maximum difference between pixels to still be considered, must be a value in the 0.1-100.0 range, default value is 1.0. </dd> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dd> Set chroma blur filter strength, must be a value in range -0.9-4.0. A greater value will result in a more blurred image, and in slower processing. </dd> <dt><samp class=”option”>chroma_pre_filter_radius, cpfr</samp></dt> <dd> Set chroma pre-filter radius, must be a value in the -0.9-2.0 range. </dd> <dt><samp class=”option”>chroma_strength, cs</samp></dt> <dd> Set chroma maximum difference between pixels to still be considered, must be a value in the -0.9-100.0 range. </dd> </dl>

Each chroma option value, if not explicitly specified, is set to the corresponding luma option value. <h3 class=”section”>32.222 scale# TOC</h3>

Scale (resize) the input video, using the libswscale library. The scale filter forces the output display aspect ratio to be the same of the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the scale filter will convert the input to the requested format. <h4 class=”subsection”>32.222.1 Options# TOC</h4> The filter accepts the following options, any of the options supported by the libswscale scaler, as well as any of the framesync options. See <a data-manual=”ffmpeg-scaler” href=”./ffmpeg-scaler.html#scaler_005foptions”>the ffmpeg-scaler manual for the complete list of scaler options. <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class=”var”>width</var> or <var class=”var”>w</var> value is 0, the input width is used for the output. If the <var class=”var”>height</var> or <var class=”var”>h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the scale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After

that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class="option">eval</samp></dt> <dd> Specify when to evaluate <var class="var">width</var> and <var class="var">height</var> expression. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">init</samp>’</dt> <dd> Only evaluate expressions once during the filter initialization or when a command is processed. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate expressions for each incoming frame. </dd> </dl>

Default value is ‘<samp class="samp">init</samp>’.

</dd> <dt><samp class="option">interl</samp></dt> <dd> Set the interlacing mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">1</samp>’</dt> <dd> Force interlaced aware scaling. </dd> <dt>‘<samp class="samp">0</samp>’</dt> <dd> Do not apply interlaced scaling. </dd> <dt>‘<samp class="samp">-1</samp>’</dt> <dd> Select interlaced aware scaling depending on whether the source frames are flagged as interlaced or not. </dd> </dl>

Default value is ‘<samp class="samp">0</samp>’. </dd> <dt><samp class="option">flags</samp></dt> <dd> Set libswscale scaling flags. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#sws_005fflags">the ffmpeg-scaler manual for the complete list of values. If not explicitly specified the filter applies the default flags.

</dd> <dt><samp class="option">param0, param1</samp></dt> <dd> Set libswscale input parameters for scaling algorithms that need them. See <a data-manual="ffmpeg-scaler" href="./ffmpeg-scaler.html#sws_005fparams">the ffmpeg-scaler manual for the complete documentation. If not explicitly specified the filter applies empty parameters. </dd> <dt><samp class="option">intent</samp></dt> <dd> Set the ICC rendering intent to use when transforming between different color spaces. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">perceptual</samp>’</dt> <dd> Use a perceptually guided tone and gamut mapping curve. The exact details of the mapping used may change at any time and should not be relied on as stable. This intent is recommended for final viewing of image/video content in typical viewing settings. </dd> <dt>‘<samp class="samp">relative_colorimetric</samp>’</dt> <dd> Statically clip out-of-gamut colors using a colorimetric clipping curve which attempts to find the colorimetrically least dissimilar in-gamut color. This intent performs

white point adaptation and black point adaptation. This is the default. This intent is recommended wherever faithful color reproduction is of the utmost importance, even at the cost of clipping. </dd> <dt>‘<samp class=”samp”>absolute_colorimetric</samp>’</dt> <dd> Hard clip out-of-gamut colors with no attempt at white or black point reproduction. This intent will reproduce in-gamut colors 1:1 on the output display as they would appear on the reference display, assuming the output display is appropriately calibrated. </dd> <dt>‘<samp class=”samp”>saturation</samp>’</dt> <dd> Performs saturation mapping - that is, stretches the input color volume directly onto the output color volume, in non-linear fashion that preserves the original signal appearance as much as possible. This intent is recommended for signal content evaluation, as it will not lead to any clipping. It is roughly analogous to not performing any color mapping, although it still takes into account the mastering display primaries and any differences in encoding TRC. </dd> </dl>

</dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>in_color_matrix</samp></dt> <dt><samp class=”option”>out_color_matrix</samp><
<dd> Set in/output YCbCr color space type. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the color space type depends on the pixel format. Possible values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class=”samp”>bt709</samp>’</dt> <dd> Format conforming to International Telecommunication Union (ITU) Recommendation BT.709. </dd> <dt>‘<samp class=”samp”>fcc</samp>’</dt> <dd> Set color space conforming to the United States Federal Communications Commission (FCC) Code of Federal Regulations (CFR) Title 47 (2003) 73.682 (a). </dd> <dt>‘<samp class=”samp”>bt601</samp>’</dt> <dt>‘<samp class=”samp”>bt470</samp>’</dt> <dt>‘<samp class=”samp”>smpte170m</samp>’</dt> <dd> Set color space conforming to: <ul class=”itemize mark-bullet”> ITU Radiocommunication Sector (ITU-R) Recommendation BT.601

ITU-R Rec. BT.470-6 (1998) Systems B, B1, and G

Society of Motion Picture and Television Engineers (SMPTE) ST 170:2004

</dd> <dt>‘<samp class=”samp”>smpte240m</samp>’</dt> <dd> Set color space conforming to SMPTE ST 240:1999.

</dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> Set color space conforming to ITU-R BT.2020 non-constant luminance system. </dd> </dl>

</dd> <dt><samp class="option">in_range</samp></dt> <dt><samp class="option">out_range</samp></dt> <dd> Set in/output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> <dt><samp class="option">in_chroma_loc</samp></dt> <dt><samp class="option">out_chroma_loc</samp></dt> <dd> Set in/output chroma sample location. If <samp class="option">in_chroma_loc</samp> is not specified, the chroma location tagged on the input frame is used, and untagged input is assumed to be sited like the output, so it is tagged but not converted. If <samp class="option">out_chroma_loc</samp> is not specified, the output is converted to the chroma location negotiated with the rest of the filtergraph, or keeps the input location if none was negotiated. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto, unknown</samp>’</dt> <dt>‘<samp class="samp">left</samp>’</dt> <dt>‘<samp class="samp">center</samp>’</dt> <dt>‘<samp class="samp">topleft</samp>’</dt> <dt>‘<samp class="samp">top</samp>’</dt> <dt>‘<samp class="samp">bottomleft</samp>’</dt> <dt>‘<samp class="samp">bottom</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dt><samp class="option">out_primaries</samp></dt> <dd> Set in/output RGB primaries. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt>

<dt>‘<samp class="samp">ebu3213</samp>’</dt> </dl>

</dd> <dt><samp class="option">in_transfer</samp></dt> <dt><samp class="option">out_transfer</samp></dt> <dd> Set in/output transfer response curve (TRC). This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Choose automatically. This is the default. </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">gamma22</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">gamma28</samp>&rsqu
<dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dt>‘<samp class="samp">smpte240m</samp>’</dt>
<dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-1</samp>’</dt> <dt>‘<samp class="samp">srgb</samp>’</dt> <dt>‘<samp class="samp">iec61966-2-4</samp>’</dt> <dt>‘<samp class="samp">xvycc</samp>’</dt> <dt>‘<samp class="samp">bt1361e</samp>’</dt> <dt>‘<samp class="samp">bt2020-10</samp>’</dt> <dt>‘<samp class="samp">bt2020-12</samp>’</dt> <dt>‘<samp class="samp">smpte2084</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dd> Enable decreasing or increasing output video width or height if necessary to keep the original aspect ratio. Possible values: <dl class="table"> <dt>‘<samp class="samp">disable</samp>’</dt> <dd> Scale the video as specified and disable this feature. </dd> <dt>‘<samp class="samp">decrease</samp>’</dt> <dd> The output video dimensions will automatically be decreased if needed. </dd> <dt>‘<samp class="samp">increase</samp>’</dt> <dd> The output video dimensions will automatically be increased if needed. </dd> </dl>

One useful instance of this option is that when you know a specific device’s maximum allowed resolution, you can use this to limit the output video to that, while retaining the aspect ratio. For example, device A allows 1280x720 playback, and your video is 1920x800. Using this option (set it to decrease) and specifying 1280x720 to the command line makes the output 1280x533. Please note that this is a different thing than specifying -1 for <samp class="option">w</samp> or <samp class="option">h</samp>, you still need to specify the output resolution for this option to work. </dd> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Ensures that both the output dimensions, width and height, are divisible by the given integer when used together with <samp

class=”option”>force_original_aspect_ratio</samp>. This works similar to using <code class=”code”>-n</code> in the <samp class=”option”>w</samp> and <samp class=”option”>h</samp> options. This option respects the value set for <samp class=”option”>force_original_aspect_ratio</samp>, increasing or decreasing the resolution accordingly. The video’s aspect ratio may be slightly modified. This option can be handy if you need to have a video fit within or exceed a defined resolution using <samp class=”option”>force_original_aspect_ratio</samp> but also have encoder restrictions on width or height divisibility. </dd> <dt><samp class=”option”>reset_sar</samp></dt> <dd> Enabling this option leads to the output SAR being reset to 1. Additionally, if the user requests proportional scaling either through the width or height expressions, e.g. <code class=”code”>w=-4:h=360</code> or <code class=”code”>w=iw/2:h=-1</code> or by enabling <code class=”code”>force_original_aspect_ratio</code>, then the input DAR is taken into account and the output is scaled to produce square pixels. Default is false. </dd> </dl>

The values of the <samp class=”option”>w</samp> and <samp class=”option”>h</samp> options are expressions containing the following constants: <dl class=”table”> <dt><var class=”var”>in_w</var></dt> <dt><var class=”var”>in_h</var></dt> <dd> The input width and height </dd> <dt><var class=”var”>iw</var></dt> <dt><var class=”var”>ih</var></dt> <dd> These are the same as <var class=”var”>in_w</var> and <var class=”var”>in_h</var>. </dd> <dt><var class=”var”>out_w</var></dt> <dt><var class=”var”>out_h</var></dt> <dd> The output (scaled) width and height </dd> <dt><var class=”var”>ow</var></dt> <dt><var class=”var”>oh</var></dt> <dd> These are the same as <var class=”var”>out_w</var> and <var class=”var”>out_h</var> </dd> <dt><var class=”var”>a</var></dt> <dd> The same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><var class=”var”>sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class=”var”>dar</var></dt> <dd> The input display aspect ratio. Calculated from <code class=”code”>(iw / ih) * sar</code>. </dd> <dt><var class=”var”>hsub</var></dt> <dt><var class=”var”>vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><var class=”var”>ohsub</var></dt> <dt><var class=”var”>ovsub</var></dt> <dd> horizontal and vertical output chroma subsample values. For example for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> <dt><var class=”var”>n</var></dt> <dd> The (sequential) number of the input frame, starting from 0. Only available with <code class=”code”>eval=frame</code>. </dd> <dt><var class=”var”>t</var></dt> <dd> The presentation timestamp of the input frame, expressed as a number of seconds.

Only available with <code class="code">eval=frame</code>. </dd> <dt><var class="var">pos</var></dt> <dd> The position (byte offset) of the frame in the input stream, or NaN if this information is unavailable and/or meaningless (for example in case of synthetic video). Only available with <code class="code">eval=frame</code>. Deprecated, do not use. </dd> <dt><var class="var">ref_w, rw</var></dt> <dt><var class="var">ref_h, rh</var></dt> <dt><var class="var">ref_a</var></dt> <dt><var class="var">ref_dar, rdar</var></dt> <dt><var class="var">ref_n</var></dt> <dt><var class="var">ref_t</var></dt> <dt><var class="var">ref_pos</var></dt> <dd> Eqvuialent to the above, but for a second reference input. If any of these variables are present, this filter accepts two inputs. </dd> </dl>

 <h4 class="subsection">32.222.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Scale the input video to a size of 200x100 <pre class="example-preformatted">scale=w=200:h=100 </pre>

This is equivalent to: <pre class="example-preformatted">scale=200:100 </pre>

or: <pre class="example-preformatted">scale=200x100 </pre>

Specify a size abbreviation for the output size: <pre class="example-preformatted">scale=qcif </pre>

which can also be written as: <pre class="example-preformatted">scale=size=qcif </pre>

Scale the input to 2x: <pre class="example-preformatted">scale=w=2*iw:h=2*ih </pre>

The above is the same as: <pre class="example-preformatted">scale=2*in_w:2*in_h </pre>

Scale the input to 2x with forced interlaced scaling: <pre class="example-preformatted">scale=2*iw:2*ih:interl=1 </pre>

Scale the input to half size: <pre class="example-preformatted">scale=w=iw/2:h=ih/2 </pre>

Increase the width, and set the height to the same size: <pre class="example-preformatted">scale=3/2*iw:ow </pre>

Seek Greek harmony: <pre class=”example-preformatted”>scale=iw:1/PHI*iw scale=ih*PHI:ih </pre>

Increase the height, and set the width to 3/2 of the height: <pre class=”example-preformatted”>scale=w=3/2*oh:h=3/5*ih </pre>

Increase the size, making the size a multiple of the chroma subsample values: <pre class=”example-preformatted”>scale="trunc(3/2*iw/hsub)*hsub:t </pre>

Increase the width to a maximum of 500 pixels, keeping the same aspect ratio as the input: <pre class=”example-preformatted”>scale=w='min(500\, iw*3/2):h=-1' </pre>

Make pixels square by combining scale and setsar: <pre class=”example-preformatted”>scale='trunc(ih*dar):ih',setsar=1/1 </pre>

Make pixels square using reset_sar, making sure the resulting resolution is even (required by some codecs): <pre class=”example-preformatted”>scale='-2:ih-mod(ih,2):reset_sar=1' </pre>

Scale to target exactly, however reset SAR to 1: <pre class=”example-preformatted”>scale='400:300:reset_sar=1' </pre>

Scale to even dimensions that fit within 400x300, preserving input SAR: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_rat </pre>

Scale to produce square pixels with even dimensions that fit within 400x300: <pre class=”example-preformatted”>scale='400:300:force_original_aspect_ </pre>

Scale a subtitle stream (sub) to match the main video (main) in size before overlaying. ("scale2ref") <pre class=”example-preformatted”>'[main]split[a][b]; [ref][a]scale=rw:rh[c]; [b][c]overlay' </pre>

Scale a logo to 1/10th the height of a video, while preserving its display aspect ratio. <pre class=”example-preformatted”>[logo-in][video-in]scale=w=oh*dar:h=rh/10[logo-out] </pre>

 <h4 class=”subsection”>32.222.3 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dt><samp

class=”option”>height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h3 class=”section”>32.223 scharr# TOC</h3> Apply scharr operator to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class=”subsection”>32.223.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.224 scroll# TOC</h3> Scroll input video horizontally and/or vertically by constant speed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt> <dd> Set the horizontal scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. Default is 0. Allowed range is from -1 to 1. Negative values changes scrolling direction. </dd> <dt><samp class=”option”>hpos</samp></dt> <dd> Set the initial horizontal scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> <dt><samp class=”option”>vpos</samp></dt> <dd> Set the initial vertical scrolling position. Default is 0. Allowed range is from 0 to 1. </dd> </dl>

 <h4 class=”subsection”>32.224.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>horizontal, h</samp></dt>

<dd> Set the horizontal scrolling speed. </dd> <dt><samp class=”option”>vertical, v</samp></dt> <dd> Set the vertical scrolling speed. </dd> </dl>

 <h3 class=”section”>32.225 scdet# TOC</h3>

Detect video scene change. This filter sets frame metadata with mafd between frame, the scene score, and forward the frame to the next filter, so they can use these metadata to detect scene change or others. In addition, this filter logs a message and sets frame metadata when it detects a scene change by <samp class=”option”>threshold</samp>. <code class=”code”>lavfi.scd.mafd</code> metadata keys are set with mafd for every frame. <code class=”code”>lavfi.scd.score</code> metadata keys are set with scene change score for every frame to detect scene change. <code class=”code”>lavfi.scd.time</code> metadata keys are set with current filtered frame time which detect scene change with <samp class=”option”>threshold</samp>. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold, t</samp></dt> <dd> Set the scene change detection threshold as a percentage of maximum change. Good values are in the <code class=”code”>[8.0, 14.0]</code> range. The range for <samp class=”option”>threshold</samp> is <code class=”code”>[0., 100.]</code>. Default value is <code class=”code”>10.</code>. </dd> <dt><samp class=”option”>sc_pass, s</samp></dt> <dd> Set the flag to pass scene change frames to the next filter. Default value is <code class=”code”>0</code> You can enable it if you want to get snapshot of scene change frames only. </dd> </dl>

 <h3 class=”section”>32.226 selectivecolor#
TOC</h3>

Adjust cyan, magenta, yellow and black (CMYK) to certain ranges of colors (such as "reds", "yellows", "greens", "cyans", ...). The adjustment range is defined by the "purity" of the color (that is, how saturated it already is). This filter is similar to the Adobe Photoshop Selective Color tool. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>correction_method</samp></dt> <dd> Select color correction method. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>absolute</samp>’</dt> <dd> Specified adjustments are applied "as-is" (added/subtracted to original pixel component value). </dd> <dt>‘<samp class=”samp”>relative</samp>’</dt> <dd> Specified adjustments are relative to the original component value.

</dd> </dl> Default is <code class=”code”>absolute</code>.
</dd> <dt><samp class=”option”>reds</samp></dt> <dd> Adjustments
for red pixels (pixels where the red component is the maximum) </dd>
<dt><samp class=”option”>yellows</samp></dt> <dd> Adjustments
for yellow pixels (pixels where the blue component is the minimum) </dd>
<dt><samp class=”option”>greens</samp></dt> <dd> Adjustments
for green pixels (pixels where the green component is the maximum)
</dd> <dt><samp class=”option”>cyans</samp></dt> <dd> Adjustments
for cyan pixels (pixels where the red component is the minimum) </dd>
<dt><samp class=”option”>blues</samp></dt> <dd> Adjustments
for blue pixels (pixels where the blue component is the maximum)
</dd> <dt><samp class=”option”>magentas</samp></dt>
<dd> Adjustments for magenta pixels (pixels where the green component is
the minimum) </dd> <dt><samp class=”option”>whites</samp></dt>
<dd> Adjustments for white pixels (pixels where all components are greater
than 128) </dd> <dt><samp class=”option”>neutrals</samp></dt>
<dd> Adjustments for all pixels except pure black and pure white
</dd> <dt><samp class=”option”>blacks</samp></dt> <dd> Adjustments
for black pixels (pixels where all components are lesser than 128) </dd>
<dt><samp class=”option”>psfile</samp></dt> <dd> Specify a Photoshop selective color file (<code class=”code”>.asv</code>) to import the
settings from. </dd> </dl>

All the adjustment settings (<samp class=”option”>reds</samp>,
<samp class=”option”>yellows</samp>, ...) accept up to 4 space separated
floating point adjustment values in the [-1,1] range, respectively to adjust the
amount of cyan, magenta, yellow and black for the pixels of its range.
 <h4 class=”subsection”>32.226.1 Examples<a class=”anchor hidden-xs” href=”#Examples-
138” aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Examples-138” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Increase cyan by 50% and reduce yellow
by 33% in every green areas, and increase magenta by 27% in blue areas: <div
class=”example”> <pre class=”example-preformatted”>selectivecolor=greens=.5
0 -.33 0:blues=0 .27 </pre>

Use a Photoshop selective color preset: <pre
class=”example-preformatted”>selectivecolor=psfile=MySelectiveColorPresets/Misty.asv
</pre>

<a name=”separatefields-
1”> <h3 class=”section”>32.227 separatefields<span class=”pull-
right”>#
TOC</h3>

The <code class=”code”>separatefields</code> takes a frame-based
video input and splits each frame into its components fields, producing

a new half height clip with twice the frame rate and twice the frame count. This filter use field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class="code">separatefields</code> filter. <h3 class="section">32.228 setdar, setsar# TOC</h3>

The <code class="code">setdar</code> filter sets the Display Aspect Ratio for the filter output video. This is done by changing the specified Sample (aka Pixel) Aspect Ratio, according to the following equation: <pre class="example-preformatted"><var class="var">DAR</var> = <var class="var">HORIZONTAL_RESOLUTION</var> / <var class="var">VERTICAL_RESOLUTION</var> * <var class="var">SAR</var> </pre>

Keep in mind that the <code class="code">setdar</code> filter does not modify the pixel dimensions of the video frame. Also, the display aspect ratio set by this filter may be changed by later filters in the filterchain, e.g. in case of scaling or if another "setdar" or a "setsar" filter is applied. The <code class="code">setsar</code> filter sets the Sample (aka Pixel) Aspect Ratio for the filter output video. Note that as a consequence of the application of this filter, the output display aspect ratio will change according to the equation above. Keep in mind that the sample aspect ratio set by the <code class="code">setsar</code> filter may be changed by later filters in the filterchain, e.g. if another "setsar" or a "setdar" filter is applied. It accepts the following parameters: <dl class="table"> <dt><samp class="option">r, ratio, dar (<code class="code">setdar</code> only), sar (<code class="code">setsar</code> only)</samp></dt> <dd> Set the aspect ratio used by the filter. The parameter can be a floating point number string, or an expression. If the parameter is not specified, the value "0" is assumed, meaning that the same input value is used. </dd> <dt><samp class="option">max</samp></dt> <dd> Set the maximum integer value to use for expressing numerator and denominator when reducing the expressed aspect ratio to a rational. Default value is <code class="code">100</code>. </dd> </dl>

The parameter <var class="var">sar</var> is an expression containing the following constants: <dl class="table"> <dt><samp class="option">w, h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">a</samp></dt> <dd> Same as <var class="var">w</var> / <var class="var">h</var>. </dd> <dt><samp class="option">sar</samp></dt> <dd> The input sample aspect ratio. </dd> <dt><samp class="option">dar</samp></dt>

<dd> The input display aspect ratio. It is the same as (<var class=”var”>w</var> / <var class=”var”>h</var>) * <var class=”var”>sar</var>. </dd> <dt><samp class=”option”>hsub, vsub</samp></dt> <dd> Horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p" <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

 <h4 class=”subsection”>32.228.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> To change the display aspect ratio to 16:9, specify one of the following: <pre class=”example-preformatted”>setdar=dar=1.77777 setdar=dar=16/9 </pre>

To change the sample aspect ratio to 10:11, specify: <pre class=”example-preformatted”>setsar=sar=10/11 </pre>

To set a display aspect ratio of 16:9, and specify a maximum integer value of 1000 in the aspect ratio reduction, use the command: <pre class=”example-preformatted”>setdar=ratio=16/9:max=1000 </pre>

 <h3 class=”section”>32.229 setfield# TOC</h3>

Force field for the output video frame. The <code class=”code”>setfield</code> filter marks the interlace type field for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters (e.g. <code class=”code”>fieldorder</code> or <code class=”code”>yadif</code>). The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same field property. </dd> <dt>‘<samp class=”samp”>bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class=”samp”>tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl> </dd> </dl>

 <h3 class=”section”>32.230 setparams<a class=”anchor

hidden-xs" href="#setparams" aria-hidden="true"># TOC</h3>

Force frame parameter for the output video frame. The <code class="code">setparams</code> filter marks interlace and color range for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by filters/encoders. <dl class="table"> <dt><samp class="option">field_mode</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same field property (default). </dd> <dt>‘<samp class="samp">bff</samp>’</dt> <dd> Mark the frame as bottom-field-first. </dd> <dt>‘<samp class="samp">tff</samp>’</dt> <dd> Mark the frame as top-field-first. </dd> <dt>‘<samp class="samp">prog</samp>’</dt> <dd> Mark the frame as progressive. </dd> </dl>

</dd> <dt><samp class="option">range</samp></dt> <dd> Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color range property (default). </dd> <dt>‘<samp class="samp">unspecified, unknown</samp>’</dt> <dd> Mark the frame as unspecified color range. </dd> <dt>‘<samp class="samp">limited, tv, mpeg</samp>’</dt> <dd> Mark the frame as limited range. </dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt> <dd> Mark the frame as full range. </dd> </dl>

</dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the color primaries. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color primaries property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dt>‘<samp class="samp">film</samp>’</dt> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> </dl>

</dd> <dt><samp class="option">color_trc</samp></dt> <dd> Set the color transfer. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> Keep the same color trc property (default). </dd> <dt>‘<samp class="samp">bt709</samp>’</dt> <dt>‘<samp class="samp">unknown</samp>’</dt> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dt>‘<samp class="samp">smpte170m</samp>&rs

<dt>‘<samp class=”samp”>smpte240m</samp>’</dt>
<dt>‘<samp class=”samp”>linear</samp>’</dt> <dt>‘<samp
class=”samp”>log100</samp>’</dt> <dt>‘<samp class=”samp”>log316</samp>’</
<dt>‘<samp class=”samp”>iec61966-2-4</samp>’</dt>
<dt>‘<samp class=”samp”>bt1361e</samp>’</dt> <dt>‘<samp
class=”samp”>iec61966-2-1</samp>’</dt> <dt>‘<samp
class=”samp”>bt2020-10</samp>’</dt> <dt>‘<samp
class=”samp”>bt2020-12</samp>’</dt> <dt>‘<samp
class=”samp”>smpte2084</samp>’</dt> <dt>‘<samp
class=”samp”>smpte428</samp>’</dt> <dt>‘<samp
class=”samp”>arib-std-b67</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>colorspace</samp></dt> <dd> Set
the colorspace. Available values are: <dl class=”table”> <dt>‘<samp
class=”samp”>auto</samp>’</dt> <dd> Keep the same col-
orspace property (default). </dd> <dt>‘<samp class=”samp”>gbr</samp>’</dt>
<dt>‘<samp class=”samp”>bt709</samp>’</dt> <dt>‘<samp
class=”samp”>unknown</samp>’</dt> <dt>‘<samp
class=”samp”>fcc</samp>’</dt> <dt>‘<samp class=”samp”>bt470bg</samp>’</d
<dt>‘<samp class=”samp”>smpte170m</samp>’</dt>
<dt>‘<samp class=”samp”>smpte240m</samp>’</dt>
<dt>‘<samp class=”samp”>ycgco</samp>’</dt> <dt>‘<samp
class=”samp”>bt2020nc</samp>’</dt> <dt>‘<samp
class=”samp”>bt2020c</samp>’</dt> <dt>‘<samp class=”samp”>smpte2085</samp>&rsq
<dt>‘<samp class=”samp”>chroma-derived-nc</samp>’</dt>
<dt>‘<samp class=”samp”>chroma-derived-c</samp>’</dt>
<dt>‘<samp class=”samp”>ictcp</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>chroma_location</samp></dt>
<dd> Set the chroma sample location. Available values are: <dl
class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt>
<dd> Keep the same chroma location (default). </dd>
<dt>‘<samp class=”samp”>unspecified, unknown</samp>’</dt>
<dt>‘<samp class=”samp”>left</samp>’</dt> <dt>‘<samp
class=”samp”>center</samp>’</dt> <dt>‘<samp class=”samp”>topleft</samp>’</
<dt>‘<samp class=”samp”>top</samp>’</dt> <dt>‘<samp
class=”samp”>bottomleft</samp>’</dt> <dt>‘<samp
class=”samp”>bottom</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> Set
the alpha moda. Available values are: <dl class=”table”> <dt>‘<samp
class=”samp”>auto</samp>’</dt> <dd> Keep the same alpha
mode (default). </dd> <dt>‘<samp class=”samp”>unspecified,
unknown</samp>’</dt> <dt>‘<samp class=”samp”>premultiplied</samp>’</dt>
<dt>‘<samp class=”samp”>straight</samp>’</dt> </dl>
</dd> </dl>

 <h3 class=”section”>32.231 shear<span class=”pull-

right”>#
TOC</h3>
Apply shear transform to input video. This filter supports the following options: <dl class=”table”> <dt><samp class=”option”>shx</samp></dt> <dd> Shear factor in X-direction. Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>shy</samp></dt> <dd> Shear factor in Y-direction. Default value is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>fillcolor, c</samp></dt> <dd> Set the color used to fill the output area not covered by the transformed video. For the general syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual. If the special value "none" is selected then no background is printed (useful for example if the background is never shown). Default value is "black". </dd> <dt><samp class=”option”>interp</samp></dt> <dd> Set interpolation type. Can be <code class=”code”>bilinear</code> or <code class=”code”>nearest</code>. Default is <code class=”code”>bilinear</code>. </dd> </dl>

 <h4 class=”subsection”>32.231.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class=”section”>32.232 showinfo# TOC</h3>

Show a line containing various information for each input video frame. The input video is not modified. This filter supports the following options: <dl class=”table”> <dt><samp class=”option”>checksum</samp></dt> <dd> Calculate checksums of each plane. By default enabled. </dd> <dt><samp class=”option”>udu_sei_as_ascii</samp></dt> <dd> Try to print user data unregistered SEI as ascii character when possible, in hex format otherwise. </dd> </dl>

The shown line contains a sequence of key/value pairs of the form <var class=”var”>key</var>:<var class=”var”>value</var>. The following values are shown in the output: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the input frame, starting from 0. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of time base units. The time base unit depends on the filter input pad. </dd> <dt><samp

class=”option”>pts_time</samp></dt> <dd> The Presentation TimeStamp of the input frame, expressed as a number of seconds. </dd> <dt><samp class=”option”>fmt</samp></dt> <dd> The pixel format name. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> The sample aspect ratio of the input frame, expressed in the form <var class=”var”>num</var>/<var class=”var”>den</var>. </dd> <dt><samp class=”option”>s</samp></dt> <dd> The size of the input frame. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>i</samp></dt> <dd> The type of interlaced mode ("P" for "progressive", "T" for top field first, "B" for bottom field first). </dd> <dt><samp class=”option”>iskey</samp></dt> <dd> This is 1 if the frame is a key frame, 0 otherwise. </dd> <dt><samp class=”option”>type</samp></dt> <dd> The picture type of the input frame ("I" for an I-frame, "P" for a P-frame, "B" for a B-frame, or "?" for an unknown type). Also refer to the documentation of the <code class=”code”>AVPictureType</code> enum and of the <code class=”code”>av_get_picture_type_char</code> function defined in <samp class=”file”>libavutil/avutil.h</samp>. </dd> <dt><samp class=”option”>checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of all the planes of the input frame. </dd> <dt><samp class=”option”>plane_checksum</samp></dt> <dd> The Adler-32 checksum (printed in hexadecimal) of each plane of the input frame, expressed in the form "[<var class=”var”>c0</var> <var class=”var”>c1</var> <var class=”var”>c2</var> <var class=”var”>c3</var>]". </dd> <dt><samp class=”option”>mean</samp></dt> <dd> The mean value of pixels in each plane of the input frame, expressed in the form "[<var class=”var”>mean0</var> <var class=”var”>mean1</var> <var class=”var”>mean2</var> <var class=”var”>mean3</var>]". </dd> <dt><samp class=”option”>stdev</samp></dt> <dd> The standard deviation of pixel values in each plane of the input frame, expressed in the form "[<var class=”var”>stdev0</var> <var class=”var”>stdev1</var> <var class=”var”>stdev2</var> <var class=”var”>stdev3</var>]". </dd> </dl>

 <h3 class=”section”>32.233 showpalette# TOC</h3>

Displays the 256 colors palette of each frame. This filter is only relevant for <var class=”var”>pal8</var> pixel format frames. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>s</samp></dt> <dd> Set the size of the box used to represent one palette color entry. Default is <code class=”code”>30</code>

(for a <code class=”code”>30x30</code> pixel box). </dd> </dl>

 <h3 class=”section”>32.234 shuffleframes# TOC</h3>

Reorder and/or duplicate and/or drop video frames. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>mapping</samp></dt> <dd> Set the destination indexes of input frames. This is space or ’|’ separated list of indexes that maps input frames to output frames. Number of indexes also sets maximal value that each index may have. ’-1’ index have special meaning and that is to drop frame. </dd> </dl>

The first frame has the index 0. The default is to keep the input unchanged. <h4 class=”subsection”>32.234.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Swap second and third frame of every three frames of the input: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf "shuffleframes=0 2 1" OUTPUT </pre>

Swap 10th and 1st frame of every ten frames of the input: <pre class=”example-preformatted”>ffmpeg -i INPUT -vf "shuffleframes=9 1 2 3 4 5 6 7 8 0" OUTPUT </pre>

 <h3 class=”section”>32.235 shufflepixels# TOC</h3>

Reorder pixels in video frames. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>direction, d</samp></dt> <dd> Set shuffle direction. Can be forward or inverse direction. Default direction is forward. </dd> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set shuffle mode. Can be horizontal, vertical or block mode. </dd> <dt><samp class=”option”>width, w</samp></dt> <dt><samp class=”option”>height, h</samp></dt> <dd> Set shuffle block_size. In case of horizontal shuffle mode only width part of size is used, and in case of vertical shuffle mode only height part of size is used. </dd> <dt><samp class=”option”>seed, s</samp></dt> <dd> Set random seed used with shuffling pixels. Mainly useful to set to be able to reverse filtering process to get original input. For example, to reverse forward shuffle you need to use same parameters and

exact same seed and to set direction to inverse. </dd> </dl>

 <h3 class="section">32.236 shuffleplanes# TOC</h3>

Reorder and/or duplicate video planes. It accepts the following parameters: <dl class="table"> <dt><samp class="option">map0</samp></dt> <dd> The index of the input plane to be used as the first output plane. </dd> <dt><samp class="option">map1</samp></dt> <dd> The index of the input plane to be used as the second output plane. </dd> <dt><samp class="option">map2</samp></dt> <dd> The index of the input plane to be used as the third output plane. </dd> <dt><samp class="option">map3</samp></dt> <dd> The index of the input plane to be used as the fourth output plane. </dd> </dl>

The first plane has the index 0. The default is to keep the input unchanged. <h4 class="subsection">32.236.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Swap the second and third planes of the input: <pre class="example-preformatted">ffmpeg -i INPUT -vf shuffleplanes=0:2:1:3 OUTPUT </pre>

<h3 class="section">32.237 signalstats# TOC</h3>

Evaluate various visual metrics that assist in determining issues associated

with the digitization of analog video media.

By default the filter will log these metadata values:

<dl class="table">

<dt><samp class="option">YMIN</samp></dt>

<dd> Display the minimal Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YLOW</samp></dt>

<dd> Display the Y value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YAVG</samp></dt>

<dd> Display the average Y value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class="option">YHIGH</samp></dt>

<dd> Display the Y value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">YMAX</samp></dt>

<dd> Display the maximum Y value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UMIN</samp></dt>

<dd> Display the minimal U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">ULOW</samp></dt>

<dd> Display the U value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UAVG</samp></dt>

<dd> Display the average U value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class="option">UHIGH</samp></dt>

<dd> Display the U value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">UMAX</samp></dt>

<dd> Display the maximum U value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VMIN</samp></dt>

<dd> Display the minimal V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class="option">VLOW</samp></dt>

<dd> Display the V value at the 10% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VAVG</samp></dt>

<dd> Display the average V value within the input frame. Expressed in range of

[0-255].

</dd>

<dt><samp class=”option”>VHIGH</samp></dt>

<dd> Display the V value at the 90% percentile within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>VMAX</samp></dt>

<dd> Display the maximum V value contained within the input frame. Expressed in

range of [0-255].

</dd>

<dt><samp class=”option”>SATMIN</samp></dt>

<dd> Display the minimal saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATLOW</samp></dt>

<dd> Display the saturation value at the 10% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATAVG</samp></dt>

<dd> Display the average saturation value within the input frame. Expressed in range

of [0-~181.02].

</dd>

<dt><samp class=”option”>SATHIGH</samp></dt>

<dd> Display the saturation value at the 90% percentile within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>SATMAX</samp></dt>

<dd> Display the maximum saturation value contained within the input frame.

Expressed in range of [0-~181.02].

</dd>

<dt><samp class=”option”>HUEMED</samp></dt>

<dd> Display the median value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>HUEAVG</samp></dt>

<dd> Display the average value for hue within the input frame. Expressed in range of

[0-360].

</dd>

<dt><samp class=”option”>YDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the Y

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class="option">UDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the U

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class="option">VDIF</samp></dt>

<dd> Display the average of sample value difference between all values of the V

plane in the current frame and corresponding values of the previous input frame.

Expressed in range of [0-255].

</dd>

<dt><samp class="option">YBITDEPTH</samp></dt>

<dd> Display bit depth of Y plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class="option">UBITDEPTH</samp></dt>

<dd> Display bit depth of U plane in current frame.

Expressed in range of [0-16].

</dd>

<dt><samp class="option">VBITDEPTH</samp></dt>

<dd> Display bit depth of V plane in current frame.

Expressed in range of [0-16].

</dd>

</dl>

The filter accepts the following options: <dl class="table"> <dt><samp class="option">stat</samp></dt> <dt><samp class="option">out</samp></dt> <dd> <samp class="option">stat</samp> specify an additional form of image analysis. <samp class="option">out</samp> output video with the specified type of pixel highlighted. Both options accept the following values: <dl class="table"> <dt>‘<samp class="samp">tout</samp>’</dt> <dd> Identify <var class="var">temporal outliers</var> pixels. A <var class="var">temporal outlier</var> is a pixel unlike the neighboring pixels of the same field. Examples of temporal outliers include the results of video dropouts, head clogs, or tape tracking issues. </dd> <dt>‘<samp class="samp">vrep</samp>’</dt> <dd> Identify <var class="var">vertical line repetition</var>. Vertical line repetition includes similar rows of pixels within a frame. In born-digital video vertical line repetition is common, but this pattern is uncommon in video digitized from an analog source. When it occurs in video that results from the digitization of an analog source it can indicate concealment from a dropout compensator. </dd> <dt>‘<samp class="samp">brng</samp>’</dt> <dd> Identify pixels that fall outside of legal broadcast range. </dd> </dl>

</dd> <dt><samp class="option">color, c</samp></dt> <dd> Set the highlight color for the <samp class="option">out</samp> option. The default color is yellow. </dd> </dl>

 <h4 class="subsection">32.237.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Output data of various video metrics: <pre class="example-preformatted">ffprobe -f lavfi movie=example.mov,signalstats="stat=tout+vrep+brng" -show_frames </pre>

Output specific data about the minimum and maximum values of the Y plane per frame: <pre class="example-preformatted">ffprobe -f lavfi movie=example.mov,signalstats -show_entries frame_tags=lavfi.signalstats.YMAX,lavfi.signalstats.YMIN </pre>

Playback video while highlighting pixels that are outside of broadcast range in red. <pre class="example-preformatted">ffplay example.mov -vf signalstats="out=brng:color=red" </pre>

Playback video with signalstats metadata drawn over the frame. <div

class=”example”> <pre class=”example-preformatted”>ffplay example.mov -
vf signalstats=stat=brng+vrep+tout,drawtext=fontfile=FreeSerif.ttf:textfile=signalstat_drawtext.txt
</pre>

The contents of signalstat_drawtext.txt used in the command are:
 <pre class=”example-preformatted”>time
%{pts:hms} Y (%{metadata:lavfi.signalstats.YMIN}-%{metadata:lavfi.signalstats.YMAX})
U (%{metadata:lavfi.signalstats.UMIN}-%{metadata:lavfi.signalstats.UMAX})
V (%{metadata:lavfi.signalstats.VMIN}-%{metadata:lavfi.signalstats.VMAX})
saturation maximum: %{metadata:lavfi.signalstats.SATMAX}

</pre>

 <h3
class=”section”>32.238 signature<a class=”anchor
hidden-xs” href=”#signature” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-signature” aria-hidden=”true”>TOC</h3>

Calculates the MPEG-7 Video Signature. The filter can handle more
than one input. In this case the matching between the inputs can be
calculated additionally. The filter always passes through the first input.
The signature of each stream can be written into a file. It
accepts the following options: <dl class=”table”> <dt><samp
class=”option”>detectmode</samp></dt> <dd> Enable or dis-
able the matching process. Available values are: <dl
class=”table”> <dt>‘<samp class=”samp”>off</samp>’</dt>
<dd> Disable the calculation of a matching (default). </dd>
<dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Calculate
the matching for the whole video and output whether the whole video matches or
only parts. </dd> <dt>‘<samp class=”samp”>fast</samp>’</dt>
<dd> Calculate only until a matching is found or the video ends. Should
be faster in some cases. </dd> </dl>

</dd> <dt><samp class=”option”>nb_inputs</samp></dt> <dd> Set
the number of inputs. The option value must be a non negative integer. Default
value is 1. </dd> <dt><samp class=”option”>filename</samp></dt>
<dd> Set the path to which the output is written. If there is more than one
input, the path must be a prototype, i.e. must contain %d or %0nd (where n is
a positive integer), that will be replaced with the input number. If no filename
is specified, no output will be written. This is the default. </dd>
<dt><samp class=”option”>format</samp></dt> <dd> Choose the
output format. Available values are: <dl class=”table”>
<dt>‘<samp class=”samp”>binary</samp>’</dt> <dd> Use
the specified binary representation (default). </dd> <dt>‘<samp
class=”samp”>xml</samp>’</dt> <dd> Use the specified xml
representation. </dd> </dl>

</dd> <dt><samp class=”option”>th_d</samp></dt> <dd> Set
threshold to detect one word as similar. The option value must be an

integer greater than zero. The default value is 9000. </dd> <dt><samp class="option">th_dc</samp></dt> <dd> Set threshold to detect all words as similar. The option value must be an integer greater than zero. The default value is 60000. </dd> <dt><samp class="option">th_xh</samp></dt> <dd> Set threshold to detect frames as similar. The option value must be an integer greater than zero. The default value is 116. </dd> <dt><samp class="option">th_di</samp></dt> <dd> Set the minimum length of a sequence in frames to recognize it as matching sequence. The option value must be a non negative integer value. The default value is 0. </dd> <dt><samp class="option">th_it</samp></dt> <dd> Set the minimum relation, that matching frames to all frames must have. The option value must be a double value between 0 and 1. The default value is 0.5. </dd> </dl>

 <h4 class="subsection">32.238.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> To calculate the signature of an input video and store it in signature.bin: <pre class="example-preformatted">ffmpeg -i input.mkv -vf signature=filename=signature.bin -map 0:v -f null - </pre>

To detect whether two videos match and store the signatures in XML format in signature0.xml and signature1.xml: <pre class="example-preformatted">ffmpeg -i input1.mkv -i input2.mkv -filter_complex "[0:v][1:v] signature=nb_inputs=2:detectmode=full:format=xml:filename=signature%d.x -map :v -f null - </pre>

 <h3 class="section">32.239 siti# TOC</h3>

Calculate Spatial Information (SI) and Temporal Information (TI) scores for a video, as defined in ITU-T Rec. P.910 (11/21): Subjective video quality assessment methods for multimedia applications. Available PDF at https://www.itu.int/rec/T-REC-P.910-202111-S/en. Note that this is a legacy implementation that corresponds to a superseded recommendation. Refer to ITU-T Rec. P.910 (07/22) for the latest version: https://www.itu.int/rec/T-REC-P.910-202207-I/en It accepts the following option: <dl class="table"> <dt><samp class="option">print_summary</samp></dt> <dd> If set to 1, Sum-

mary statistics will be printed to the console. Default 0. </dd> </dl>

 <h4 class="subsection">32.239.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> To calculate SI/TI metrics and print summary: <pre class="example-preformatted">ffmpeg -i input.mp4 -vf siti=print_summary=1 -f null - </pre>

 <h3 class="section">32.240 smartblur# TOC</h3>

Blur the input video without impacting the outlines. It accepts the following options: <dl class="table"> <dt><samp class="option">luma_radius, lr</samp></dt> <dd> Set the luma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is 1.0. </dd> <dt><samp class="option">luma_strength, ls</samp></dt> <dd> Set the luma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is 1.0. </dd> <dt><samp class="option">luma_threshold, lt</samp></dt> <dd> Set the luma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is 0. </dd> <dt><samp class="option">chroma_radius, cr</samp></dt> <dd> Set the chroma radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">chroma_strength, cs</samp></dt> <dd> Set the chroma strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">chroma_threshold, ct</samp></dt> <dd> Set the chroma threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> <dt><samp class="option">alpha_radius, ar</samp></dt> <dd> Set

the alpha radius. The option value must be a float number in the range [0.1,5.0] that specifies the variance of the gaussian filter used to blur the image (slower if larger). Default value is <samp class="option">luma_radius</samp>. </dd> <dt><samp class="option">alpha_strength, as</samp></dt> <dd> Set the alpha strength. The option value must be a float number in the range [-1.0,1.0] that configures the blurring. A value included in [0.0,1.0] will blur the image whereas a value included in [-1.0,0.0] will sharpen the image. Default value is <samp class="option">luma_strength</samp>. </dd> <dt><samp class="option">alpha_threshold, at</samp></dt> <dd> Set the alpha threshold used as a coefficient to determine whether a pixel should be blurred or not. The option value must be an integer in the range [-30,30]. A value of 0 will filter all the image, a value included in [0,30] will filter flat areas and a value included in [-30,0] will filter edges. Default value is <samp class="option">luma_threshold</samp>. </dd> </dl>

If a chroma or alpha option is not explicitly set, the corresponding luma value is set. <h3 class="section">32.241 sobel# TOC</h3> Apply sobel operator to input video stream. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set value which will be multiplied with filtered result. </dd> <dt><samp class="option">delta</samp></dt> <dd> Set value which will be added to filtered result. </dd> </dl>

 <h4 class="subsection">32.241.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.242 spp# TOC</h3>

Apply a simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">6</code> - all) shifts and average the results. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-6. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">6</code> means the higher

quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class=”option”>mode</samp></dt> <dd> Set thresholding mode. Available modes are: <dl class=”table”> <dt>‘<samp class=”samp”>hard</samp>’</dt> <dd> Set hard thresholding (default). </dd> <dt>‘<samp class=”samp”>soft</samp>’</dt> <dd> Set soft thresholding (better de-ringing effect, but likely blurrier). </dd> </dl>

</dd> <dt><samp class=”option”>use_bframe_qp</samp></dt> <dd> Enable the use of the QP from the B-Frames if set to <code class=”code”>1</code>. Using this option may cause flicker since the B-Frames have often larger QP. Default is <code class=”code”>0</code> (not enabled). </dd> </dl>

 <h4 class=”subsection”>32.242.1 Commands# TOC</h4>

This filter supports the following commands: <dl class=”table”> <dt><samp class=”option”>quality, level</samp></dt> <dd> Set quality level. The value <code class=”code”>max</code> can be used to set the maximum level, currently <code class=”code”>6</code>. </dd> </dl>

 <h3 class=”section”>32.243 sr# TOC</h3>

Scale the input by applying one of the super-resolution methods based on convolutional neural networks. Supported models: <ul class=”itemize mark-bullet”> Super-Resolution Convolutional Neural Network model (SRCNN). See https://arxiv.org/abs/1501.00092.

Efficient Sub-Pixel Convolutional Neural Network model (ESPCN). See https://arxiv.org/abs/1609.05158.

Training scripts as well as scripts for model file (.pb) saving can be found at <a class=”url” href=”https://github.com/XueweiMeng/sr/tree/sr_dnn_native”>https://github.com/Xuewei Original repository is at <a class=”url” href=”https://github.com/HighVoltageRocknRoll/sr.git”>https://gith The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>dnn_backend</samp></dt> <dd> Specify which DNN backend to use for model loading and execution. This option

accepts the following values: <dl class="table"> <dt>‘<samp class="samp">tensorflow</samp>’</dt> <dd> TensorFlow backend. To enable this backend you need to install the TensorFlow for C library (see <a class="url" href="https://www.tensorflow.org/install/lang_c">https://www.tensorflow.org/install/lang_c and configure FFmpeg with <code class="code">--enable-libtensorflow</code> </dd> </dl>

</dd> <dt><samp class="option">model</samp></dt> <dd> Set path to model file specifying network architecture and its parameters. Note that different backends use different file formats. TensorFlow, OpenVINO backend can load files for only its format. </dd> <dt><samp class="option">scale_factor</samp></dt> <dd> Set scale factor for SRCNN model. Allowed values are <code class="code">2</code>, <code class="code">3</code> and <code class="code">4</code>. Default value is <code class="code">2</code>. Scale factor is necessary for SRCNN model, because it accepts input upscaled using bicubic upscaling with proper scale factor. </dd> </dl>

To get full functionality (such as async execution), please use the dnn_processing filter. <h3 class="section">32.244 sr_amf# TOC</h3>

Upscale (size increasing) for the input video using AMD Advanced Media Framework library for hardware acceleration. Use advanced algorithms for upscaling with higher output quality. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class="option">algorithm</samp></dt> <dd> Sets the algorithm used for scaling: <dl class="table"> <dt><var class="var">bilinear</var></dt> <dd> Bilinear </dd> <dt><var class="var">bicubic</var></dt> <dd> Bicubic </dd> <dt><var class="var">sr1-0</var></dt> <dd> Video SR1.0 This is a default value </dd> <dt><var class="var">point</var></dt> <dd> Point AMF returns an unwritten surface for this algorithm when the format is <code class="code">nv12</code> or <code class="code">p010</code>, so inputs in those formats are converted to a packed RGB format automatically. This is a driver bug rather than a documented restriction; the conversion can be dropped once a driver that scales <code class="code">nv12</code> and <code class="code">p010</code> correctly with <code class="code">point</code> is in wide use.

</dd> <dt><var class="var">sr1-1</var></dt> <dd> Video SR1.1 This algorithm only supports packed RGB formats and requires a DirectX 11 or DirectX 12 device, so it is available on Windows only. Inputs in other formats such as <code class="code">nv12</code> or <code class="code">p010</code> are converted to a packed RGB format automatically. Note that the VideoSR algorithms (<code class="code">sr1-0</code> and <code class="code">sr1-1</code>) do not preserve the alpha channel (it is left zeroed); drop it after the download, e.g. with <code class="code">hwdownload,format=rgba,format=rgb24</code>, if an opaque result is required. </dd> </dl>

</dd> <dt><samp class="option">sharpness</samp></dt> <dd> Control hq scaler sharpening. The value is a float in the range of [0.0, 2.0] </dd> <dt><samp class="option">format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. The <code class="code">point</code> and <code class="code">sr1-1</code> algorithms emit the packed RGB format the scaler receives, so for them this option must name one of <code class="code">rgba</code>, <code class="code">bgra</code>, <code class="code">x2bgr10le</code> or <code class="code">rgbaf16le</code>. A named format also decides what software input is converted to, and what a hardware surface in another format is converted to on the GPU. Without it, <code class="code">x2bgr10le</code> is selected for <code class="code">p010</code> input and <code class="code">rgba</code> for other non-RGB input, and the result is tagged as full range RGB. libswscale cannot convert other software formats to <code class="code">rgbaf16le</code>; use software input already in that format or a hardware input for GPU conversion. </dd> <dt><samp class="option">keep-ratio</samp></dt> <dd> Force the scaler to keep the aspect ratio of the input image when the output size has a different aspect ratio. Default value is false. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specifies whether the output image outside the region of interest, which does not fill the entire output surface should be filled with a solid color. </dd> </dl>

 <h4 class="subsection">32.244.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Upscale to 720p, keeping aspect ratio, and read the result back into system memory. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class="example-preformatted">sr_amf=-2:720,hwdownload,format=nv12
</pre>

Upscale to 4K with algorithm video SR1.1. <pre class="example-preformatted">sr_amf=4096:2160:algorithm=sr1-1

</pre>

 <h3 class="section">32.245 ssim#
TOC</h3>

Obtain the SSIM (Structural SImilarity Metric) between two input videos. This filter takes in input two input videos, the first input is considered the "main" source and is passed unchanged to the output. The second input is used as a "reference" video for computing the SSIM. Both video inputs must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The filter stores the calculated SSIM of each frame. The description of the accepted parameters follows. <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified the filter will use the named file to save the SSIM of each individual frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

The file printed if <var class="var">stats_file</var> is selected, contains a sequence of key/value pairs of the form <var class="var">key</var>:<var class="var">value</var> for each compared couple of frames. A description of each shown parameter follows: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> sequential number of the input frame, starting from 1 </dd> <dt><samp class="option">Y, U, V, R, G, B</samp></dt> <dd> SSIM of the compared frames for the component specified by the suffix. </dd> <dt><samp class="option">All</samp></dt> <dd> SSIM of the compared frames for the whole frame. </dd> <dt><samp class="option">dB</samp></dt> <dd> Same as above but in dB representation. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">32.245.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> For example: <pre class="example-preformatted">movie=ref_movie.mpg, setpts=PTS-STARTPTS [main]; [main][ref] ssim="stats_file=stats.log" [out] </pre>

On this example the input file being processed is compared with the reference file <samp class="file">ref_movie.mpg</samp>. The SSIM of each individual frame is stored in <samp class="file">stats.log</samp>. Another example with both psnr and ssim at same time: <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi "ssim;[0:v][1:v]psnr" -f null - </pre>

Another example with different containers:
<pre class=”example-preformatted”>ffmpeg -i main.mpg -i ref.mkv -lavfi
"[0:v]settb=AVTB,setpts=PTS-STARTPTS[main];[1:v]settb=AVTB,setpts=PTS-
STARTPTS[ref];[main][ref]ssim" -f null - </pre>

 <h3 class=”section”>32.246 stereo3d<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#stereo3d” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-stereo3d”
aria-hidden=”true”>TOC</h3>

Convert between different stereoscopic image formats. The
filters accept the following options: <dl class=”table”> <dt><samp
class=”option”>in</samp></dt> <dd> Set stereoscopic image format
of input. Available values for input image formats are: <dl
class=”table”> <dt>‘<samp class=”samp”>sbsl</samp>’</dt>
<dd> side by side parallel (left eye left, right eye right)
</dd> <dt>‘<samp class=”samp”>sbsr</samp>’</dt>
<dd> side by side crosseye (right eye left, left eye right)
</dd> <dt>‘<samp class=”samp”>sbs2l</samp>’</dt>
<dd> side by side parallel with half width resolution (left eye left, right eye
right) </dd> <dt>‘<samp class=”samp”>sbs2r</samp>’</dt>
<dd> side by side crosseye with half width resolution (right eye left, left eye
right) </dd> <dt>‘<samp class=”samp”>abl</samp>’</dt>
<dt>‘<samp class=”samp”>tbl</samp>’</dt> <dd> above-
below (left eye above, right eye below) </dd> <dt>‘<samp
class=”samp”>abr</samp>’</dt> <dt>‘<samp class=”samp”>tbr</samp>’</dt>
<dd> above-below (right eye above, left eye below) </dd>
<dt>‘<samp class=”samp”>ab2l</samp>’</dt> <dt>‘<samp
class=”samp”>tb2l</samp>’</dt> <dd> above-below with
half height resolution (left eye above, right eye below) </dd>
<dt>‘<samp class=”samp”>ab2r</samp>’</dt> <dt>‘<samp
class=”samp”>tb2r</samp>’</dt> <dd> above-below with
half height resolution (right eye above, left eye below) </dd>
<dt>‘<samp class=”samp”>al</samp>’</dt> <dd> alternating
frames (left eye first, right eye second) </dd> <dt>‘<samp
class=”samp”>ar</samp>’</dt> <dd> alternating frames
(right eye first, left eye second) </dd> <dt>‘<samp
class=”samp”>irl</samp>’</dt> <dd> interleaved rows (left eye
has top row, right eye starts on next row) </dd> <dt>‘<samp
class=”samp”>irr</samp>’</dt> <dd> interleaved rows (right
eye has top row, left eye starts on next row) </dd> <dt>‘<samp
class=”samp”>icl</samp>’</dt> <dd> interleaved columns, left
eye first </dd> <dt>‘<samp class=”samp”>icr</samp>’</dt>
<dd> interleaved columns, right eye first Default value is
‘<samp class=”samp”>sbsl</samp>’. </dd> </dl>

</dd>

<dt><samp class=”option”>out</samp></dt>
<dd> Set stereoscopic image format of output.

<dl class=”table”>
<dt>‘<samp class=”samp”>sbsl</samp>’</dt>
<dd> side by side parallel (left eye left, right eye right)

</dd>
<dt>‘<samp class=”samp”>sbsr</samp>’</dt>
<dd> side by side crosseye (right eye left, left eye right)

</dd>
<dt>‘<samp class=”samp”>sbs2l</samp>’</dt>
<dd> side by side parallel with half width resolution
(left eye left, right eye right)

</dd>
<dt>‘<samp class=”samp”>sbs2r</samp>’</dt>
<dd> side by side crosseye with half width resolution
(right eye left, left eye right)

</dd>
<dt>‘<samp class=”samp”>abl</samp>’</dt>
<dt>‘<samp class=”samp”>tbl</samp>’</dt>
<dd> above-below (left eye above, right eye below)

</dd>
<dt>‘<samp class=”samp”>abr</samp>’</dt>
<dt>‘<samp class=”samp”>tbr</samp>’</dt>
<dd> above-below (right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">ab2l</samp>’</dt>
<dt>‘<samp class="samp">tb2l</samp>’</dt>
<dd> above-below with half height resolution
(left eye above, right eye below)

</dd>
<dt>‘<samp class="samp">ab2r</samp>’</dt>
<dt>‘<samp class="samp">tb2r</samp>’</dt>
<dd> above-below with half height resolution
(right eye above, left eye below)

</dd>
<dt>‘<samp class="samp">al</samp>’</dt>
<dd> alternating frames (left eye first, right eye second)

</dd>
<dt>‘<samp class="samp">ar</samp>’</dt>
<dd> alternating frames (right eye first, left eye second)

</dd>
<dt>‘<samp class="samp">irl</samp>’</dt>
<dd> interleaved rows (left eye has top row, right eye starts on next row)

</dd>
<dt>‘<samp class="samp">irr</samp>’</dt>
<dd> interleaved rows (right eye has top row, left eye starts on next row)

</dd>
<dt>‘<samp class="samp">arbg</samp>’</dt>
<dd> anaglyph red/blue gray

(red filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">argg</samp>’</dt>

<dd> anaglyph red/green gray

(red filter on left eye, green filter on right eye)

</dd>

<dt>‘<samp class="samp">arcg</samp>’</dt>

<dd> anaglyph red/cyan gray

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arch</samp>’</dt>

<dd> anaglyph red/cyan half colored

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcc</samp>’</dt>

<dd> anaglyph red/cyan color

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">arcd</samp>’</dt>

<dd> anaglyph red/cyan color optimized with the least squares projection of dubois

(red filter on left eye, cyan filter on right eye)

</dd>

<dt>‘<samp class="samp">agmg</samp>’</dt>

<dd> anaglyph green/magenta gray

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmh</samp>’</dt>

<dd> anaglyph green/magenta half colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmc</samp>’</dt>

<dd> anaglyph green/magenta colored

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">agmd</samp>’</dt>

<dd> anaglyph green/magenta color optimized with the least squares projection of dubois

(green filter on left eye, magenta filter on right eye)

</dd>

<dt>‘<samp class="samp">aybg</samp>’</dt>

<dd> anaglyph yellow/blue gray

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybh</samp>’</dt>

<dd> anaglyph yellow/blue half colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybc</samp>’</dt>

<dd> anaglyph yellow/blue colored

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">aybd</samp>’</dt>

<dd> anaglyph yellow/blue color optimized with the least squares projection of dubois

(yellow filter on left eye, blue filter on right eye)

</dd>

<dt>‘<samp class="samp">ml</samp>’</dt>

<dd> mono output (left eye only)

</dd>

<dt>‘<samp class="samp">mr</samp>’</dt>

<dd> mono output (right eye only)

</dd>

<dt>‘<samp class="samp">chl</samp>’</dt>

<dd> checkerboard, left eye first

</dd>

<dt>‘<samp class="samp">chr</samp>’</dt>

<dd> checkerboard, right eye first

</dd>

<dt>‘<samp class="samp">icl</samp>’</dt>

<dd> interleaved columns, left eye first

</dd>

<dt>‘<samp class="samp">icr</samp>’</dt>

<dd> interleaved columns, right eye first

</dd>

<dt>‘<samp class="samp">hdmi</samp>’</dt>

<dd> HDMI frame pack

</dd>

</dl>

Default value is ‘<samp class="samp">arcd</samp>’.
</dd> </dl>

 <h4 class="subsection">32.246.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert input video from side by side parallel to anaglyph yellow/blue dubois: <pre class="example-preformatted">stereo3d=sbsl:aybd </pre>

Convert input video from above below (left eye above, right eye below) to side by side crosseye. <pre class="example-preformatted">stereo3d=abl:sbsr </pre>

 <h3 class="section">32.247 streamselect, astreamselect# TOC</h3> Select video or audio streams. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 2. </dd> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">32.247.1 Commands# TOC</h4>

The <code class="code">streamselect</code> and <code class="code">astreamselect</code> filter supports the following commands: <dl class="table"> <dt><samp class="option">map</samp></dt> <dd> Set input indexes to remap to outputs. </dd> </dl>

 <h4 class="subsection">32.247.2 Examples<a class="anchor hidden-xs" href="#Examples-

148" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Select first 5 seconds 1st stream and rest of time 2nd stream: <pre class="example-preformatted">sendcmd='5.0 streamselect map 1',streamselect=inputs=2:map=0 </pre>

Same as above, but for audio: <pre class="example-preformatted">asendcmd='5.0 astreamselect map 1',astreamselect=inputs=2:map=0 </pre>

 <h3 class="section">32.248 subtitles# TOC</h3>

Draw subtitles on top of input video using the libass library. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libass</code>. This filter also requires a build with libavcodec and libavformat to convert the passed subtitles file to ASS (Advanced Substation Alpha) subtitles format. The filter accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the subtitle file to read. It must be specified. </dd> <dt><samp class="option">original_size</samp></dt> <dd> Specify the size of the original video, the video for which the ASS file was composed. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Due to a misdesign in ASS aspect ratio arithmetic, this is necessary to correctly scale the fonts if the aspect ratio has been changed. </dd> <dt><samp class="option">fontsdir</samp></dt> <dd> Set a directory path containing fonts that can be used by the filter. These fonts will be used in addition to whatever the font provider uses. </dd> <dt><samp class="option">alpha</samp></dt> <dd> Process alpha channel, by default alpha channel is untouched. </dd> <dt><samp class="option">charenc</samp></dt> <dd> Set subtitles input character encoding. <code class="code">subtitles</code> filter only. Only useful if not UTF-8. </dd> <dt><samp class="option">stream_index, si</samp></dt> <dd> Set subtitles stream index. <code class="code">subtitles</code> filter only. </dd> <dt><samp class="option">force_style</samp></dt> <dd> Override default style or script info parameters of the subtitles. It accepts a string containing ASS style format <code class="code">KEY=VALUE</code> couples separated by ",". </dd> <dt><samp class="option">wrap_unicode</samp></dt> <dd> Break lines according to the Unicode Line Breaking Algorithm. Availability requires at least libass release 0.17.0 (or LIBASS_VERSION 0x01600010), and libass must have been built with

libunibreak. The option is enabled by default except for native ASS. </dd> <dt><samp class="option">shaping</samp></dt> <dd> Set the shaping engine. Available values are: <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dd> The default libass shaping engine, which is the best available. </dd> <dt>‘<samp class="samp">simple</samp>’</dt> <dd> Fast, font-agnostic shaper that can do only substitutions. </dd> <dt>‘<samp class="samp">complex</samp>’</dt> <dd> Slower shaper using OpenType for substitutions and positioning. Required for correct rendering of complex scripts such as Arabic, Hebrew, Devanagari and Thai. Requires libass to be built with HarfBuzz. </dd> </dl>

The default is <code class="code">auto</code>. </dd> </dl>

If the first key is not specified, it is assumed that the first value specifies the <samp class="option">filename</samp>. For example, to render the file <samp class="file">sub.srt</samp> on top of the input video, use the command: <pre class="example-preformatted">subtitles=sub.srt </pre>

which is equivalent to: <pre class="example-preformatted">subtitles=filename=sub.srt </pre>

To render the default subtitles stream from file <samp class="file">video.mkv</samp>, use: <pre class="example-preformatted">subtitles=video.mkv </pre>

To render the second subtitles stream from that file, use: <pre class="example-preformatted">subtitles=video.mkv:si=1 </pre>

To make the subtitles stream from <samp class="file">sub.srt</samp> appear in 80% transparent blue <code class="code">DejaVu Serif</code>, use: <pre class="example-preformatted">subtitles=sub.srt:force_style='Fontn Serif,PrimaryColour=&HCCFF0000' </pre>

 <h3 class="section">32.249 super2xsai# TOC</h3>

Scale the input by 2x and smooth using the Super2xSaI (Scale and Interpolate) pixel art scaling algorithm. Useful for enlarging pixel art images without reducing sharpness. <h3 class="section">32.250 swaprect# TOC</h3>

Swap two rectangular objects in video. This filter ac-

cepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dd> Set object width. </dd> <dt><samp class="option">h</samp></dt> <dd> Set object height. </dd> <dt><samp class="option">x1</samp></dt> <dd> Set 1st rect x coordinate. </dd> <dt><samp class="option">y1</samp></dt> <dd> Set 1st rect y coordinate. </dd> <dt><samp class="option">x2</samp></dt> <dd> Set 2nd rect x coordinate. </dd> <dt><samp class="option">y2</samp></dt> <dd> Set 2nd rect y coordinate. All expressions are evaluated once for each frame. </dd> </dl>

The all options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">a</samp></dt> <dd> same as <var class="var">w</var> / <var class="var">h</var> </dd> <dt><samp class="option">sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class="var">w</var> / <var class="var">h</var>) * <var class="var">sar</var> </dd> <dt><samp class="option">n</samp></dt> <dd> The number of the input frame, starting from 0. </dd> <dt><samp class="option">t</samp></dt> <dd> The timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class="option">pos</samp></dt> <dd> the position in the file of the input frame, NAN if unknown; deprecated, do not use </dd> </dl>

 <h4 class="subsection">32.250.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.251 swapuv# TOC</h3> Swap U & V plane. <h3 class="section">32.252 tblend# TOC</h3> Blend successive video frames. See blend <h3 class="section">32.253 telecine# TOC</h3>

Apply telecine process to the video. This filter ac-

cepts the following options: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> top field first </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> bottom field first The default value is <code class=”code”>top</code>. </dd> </dl>

</dd> <dt><samp class=”option”>pattern</samp></dt> <dd> A string of numbers representing the pulldown pattern you wish to apply. The default value is <code class=”code”>23</code>. </dd> </dl>

 <pre class=”example-preformatted”>Some typical patterns:

NTSC output (30i): 27.5p: 32222 24p: 23 (classic) 24p: 2332 (preferred) 20p: 33 18p: 334 16p: 3444

PAL output (25i): 27.5p: 12222 24p: 222222222223 ("Euro pulldown") 16.67p: 33 16p: 33333334 </pre>

 <h3 class=”section”>32.254 thistogram# TOC</h3>

Compute and draw a color distribution histogram for the input video across time. Unlike histogram video filter which only shows histogram of single input frame at certain time, this filter shows also past histograms of number of frames defined by <code class=”code”>width</code> option. The computed histogram is a representation of the color component distribution in an image. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>width, w</samp></dt> <dd> Set width of single color component output. Default value is <code class=”code”>0</code>. Value of <code class=”code”>0</code> means width will be picked from input video. This also set number of passed histograms to keep. Allowed range is [0, 8192]. </dd> <dt><samp class=”option”>display_mode, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>stack</samp>’</dt> <dd> Per color component graphs are placed below each other. </dd> <dt>‘<samp class=”samp”>parade</samp>’</dt> <dd> Per color component graphs are placed side by side. </dd> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dd> Presents information identical to that in the <code class=”code”>parade</code>, except that the graphs representing color components are superimposed directly over one another. </dd> </dl> Default is <code class=”code”>stack</code>. </dd> <dt><samp class=”option”>levels_mode, m</samp></dt> <dd> Set mode. Can be either <code class=”code”>linear</code>, or

<code class=”code”>logarithmic</code>. Default is <code class=”code”>linear</code>. </dd> <dt><samp class=”option”>components, c</samp></dt> <dd> Set what color components to display. Default is <code class=”code”>7</code>. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. Default is <code class=”code”>0.9</code>. </dd> <dt><samp class=”option”>envelope, e</samp></dt> <dd> Show envelope. Default is disabled. </dd> <dt><samp class=”option”>ecolor, ec</samp></dt> <dd> Set envelope color. Default is <code class=”code”>gold</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide mode. Available values for slide is: <dl class=”table”> <dt>‘<samp class=”samp”>frame</samp>’</dt> <dd> Draw new frame when right border is reached. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt> <dd> Replace old columns with new ones. </dd> <dt>‘<samp class=”samp”>scroll</samp>’</dt> <dd> Scroll from right to left. </dd> <dt>‘<samp class=”samp”>rscroll</samp>’</dt> <dd> Scroll from left to right. </dd> <dt>‘<samp class=”samp”>picture</samp>’</dt> <dd> Draw single picture. </dd> </dl>

Default is <code class=”code”>replace</code>. </dd> </dl>

 <h3 class=”section”>32.255 threshold# TOC</h3>

Apply threshold effect to video stream. This filter needs four video streams to perform thresholding. First stream is stream we are filtering. Second stream is holding threshold values, third stream is holding min values, and last, fourth stream is holding max values. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. </dd> </dl>

For example if first stream pixel’s component value is less then threshold value of pixel component from 2nd threshold stream, third stream value will picked, otherwise fourth stream pixel component value will be picked. Using color source filter one can perform various types of thresholding: <h4 class=”subsection”>32.255.1 Commands# TOC</h4>

This filter supports the all options as commands. <h4 class=”subsection”>32.255.2 Examples<a class=”anchor hidden-xs” href=”#Examples-

149" aria-hidden="true"># TOC</h4>

<ul class="itemize mark-bullet"> Binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=black -f lavfi -i color=white -lavfi threshold output.avi </pre>

Inverted binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=white -f lavfi -i color=black -lavfi threshold output.avi </pre>

Truncate binary threshold, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=gray -lavfi threshold output.avi </pre>

Threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -f lavfi -i color=white -i 320x240.avi -lavfi threshold output.avi </pre>

Inverted threshold to zero, using gray color as threshold: <pre class="example-preformatted">ffmpeg -i 320x240.avi -f lavfi -i color=gray -i 320x240.avi -f lavfi -i color=white -lavfi threshold output.avi </pre>

 <h3 class="section">32.256 thumbnail# TOC</h3> Select the most representative frame in a given sequence of consecutive frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class="var">n</var> frames, the filter will pick one of them, and then handle the next batch of <var class="var">n</var> frames until the end. Default is <code class="code">100</code>. </dd> <dt><samp class="option">log</samp></dt> <dd> Set the log level to display picked frame stats. Default is <code class="code">info</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class="var">n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class="subsection">32.256.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract one picture each 50 frames: <pre class="example-preformatted">thumbnail=50

</pre>

Complete example of a thumbnail creation with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i in.avi -vf thumbnail,scale=300:200 -frames:v 1 out.png </pre>

 <h3 class="section">32.257 tile# TOC</h3>

Tile several successive frames together. The untile filter can do the reverse. The filter accepts the following options: <dl class="table"> <dt><samp class="option">layout</samp></dt> <dd> Set the grid size in the form <code class="code">COLUMNSxROWS</code>. Range is up to UINT_MAX cells. Default is <code class="code">6x5</code>. </dd> <dt><samp class="option">nb_frames</samp></dt> <dd> Set the maximum number of frames to render in the given area. It must be less than or equal to <var class="var">w</var>x<var class="var">h</var>. The default value is <code class="code">0</code>, meaning all the area will be used. </dd> <dt><samp class="option">margin</samp></dt> <dd> Set the outer border margin in pixels. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">padding</samp></dt> <dd> Set the inner border thickness (i.e. the number of pixels between frames). For more advanced padding options (such as having different values for the edges), refer to the pad video filter. Range is 0 to 1024. Default is <code class="code">0</code>. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the unused area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set the number of frames to overlap when tiling several successive frames together. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> <dt><samp class="option">init_padding</samp></dt> <dd> Set the number of frames to initially be empty before displaying first output frame. This controls how soon will one get first output frame. The value must be between <code class="code">0</code> and <var class="var">nb_frames - 1</var>. Default is <code class="code">0</code>. </dd> </dl>

 <h4 class="subsection">32.257.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Produce 8x8 PNG tiles of all keyframes (<samp class=”option”>-skip_frame nokey</samp>) in a movie: <pre class=”example-preformatted”>ffmpeg -skip_frame nokey -i file.avi -vf 'scale=128:72,tile=8x8' -an -fps_mode passthrough keyframes%03d.png </pre> The <samp class=”option”>-fps_mode passthrough</samp> is necessary to prevent <code class=”command”>ffmpeg</code> from duplicating each output frame to accommodate the originally detected frame rate. Display <code class=”code”>5</code> pictures in an area of <code class=”code”>3x2</code> frames, with <code class=”code”>7</code> pixels between them, and <code class=”code”>2</code> pixels of initial margin, using mixed flat and named options: <pre class=”example-preformatted”>tile=3x2:nb_frames=5:padding=7:margin=2 </pre>

 <h3 class=”section”>32.258 tiltandshift# TOC</h3> Apply tilt-and-shift effect. What happens when you invert time and space? Normally a video is composed of several frames that represent a different instant of time and shows a scene that evolves in the space captured by the frame. This filter is the antipode of that concept, taking inspiration from tilt and shift photography. A filtered frame contains the whole timeline of events composing the sequence, and this is obtained by placing a slice of pixels from each frame into a single one. However, since there are no infinite-width frames, this is done up the width of the input frame, and a video is recomposed by shifting away one column for each subsequent frame. In order to map space to time, the filter tilts each input frame as well, so that motion is preserved. This is accomplished by progressively selecting a different column from each input frame. The end result is a sort of inverted parallax, so that far away objects move much faster that the ones in the front. The ideal conditions for this video effect are when there is either very little motion and the background is static, or when there is a lot of motion and a very wide depth of field (e.g. wide panorama, while moving on a train). The filter accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tilt</samp></dt> <dd> Tilt video while shifting (default). When unset, video will be sliding a static image, composed of the first column of each frame. </dd> <dt><samp class=”option”>start</samp></dt> <dd> What to do at the start of filtering (see below). </dd> <dt><samp class=”option”>end</samp></dt> <dd> What to do at the end of filtering (see below). </dd> <dt><samp class=”option”>hold</samp></dt> <dd> How many columns should pass through before start of filtering. </dd> <dt><samp class=”option”>pad</samp></dt> <dd> How many columns should be inserted before end of filtering. </dd> </dl>

Normally the filter shifts and tilts from the very first frame, and stops when the last one is received. However, before filtering starts, normal video may be preserved, so that the effect is slowly shifted in its place. Similarly, the last video frame may be reconstructed at the end. Alternatively it is possible to just start and end with black. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Filtering starts immediately and ends when the last frame is received. </dd> <dt>‘<samp class="samp">frame</samp>’</dt> <dd> The first frames or the very last frame are kept intact during processing. </dd> <dt>‘<samp class="samp">black</samp>’</dt> <dd> Black is padded at the beginning or at the end of filtering. </dd> </dl>

 <h3 class="section">32.259 tinterlace# TOC</h3>

Perform various types of temporal field interlacing. Frames are counted starting from 1, so the first input frame is considered odd. The filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Specify the mode of the interlacing. This option can also be specified as a value alone. See below for a list of values for this option. Available values are: <dl class="table"> <dt>‘<samp class="samp">merge, 0</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at half frame rate. <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class="samp">drop_even, 1</samp>’</dt> <dd> Only output odd frames, even frames are dropped, generating a frame with unchanged height at half frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 11111 33333 11111 33333 11111 33333 </pre>

</dd> <dt>‘<samp class="samp">drop_odd, 2</samp>’</dt> <dd> Only output even frames, odd frames are dropped, generating a frame with unchanged height at half frame rate.

<pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 22222 44444 22222 44444 22222 44444 22222 44444 </pre>

</dd> <dt>‘<samp class=”samp”>pad, 3</samp>’</dt> <dd> Expand each frame to full height, but pad alternate lines with black, generating a frame with double height at the same input frame rate.   <pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 11111 ..... 33333 .... 22222 ..... 44444 </pre>

</dd> <dt>‘<samp class=”samp”>interleave_top, 4</samp>’</dt> <dd> Interleave the upper field from odd frames with the lower field from even frames, generating a frame with unchanged height at half frame rate.   <pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<-

Output: 11111 33333 22222 44444 11111 33333 22222 44444 </pre>

</dd> <dt>‘<samp class=”samp”>interleave_bottom, 5</samp>’</dt> <dd> Interleave the lower field from odd frames with the upper field from even frames, generating a frame with unchanged height at half frame rate.   <pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444 11111 22222<- 33333 44444<- 11111<- 22222 33333<- 44444

Output: 22222 44444 11111 33333 22222 44444 11111 33333 </pre>

</dd> <dt>‘<samp class=”samp”>interlacex2, 6</samp>’</dt> <dd> Double frame rate with unchanged height. Frames are inserted each containing the second temporal field from the previous input frame and the first temporal field from the next input frame. This mode relies on the top_field_first flag. Useful for interlaced video displays with no field synchronisation.   <pre class=”example-preformatted”> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 11111 22222 22222 33333 33333 44444 44444 11111 11111 22222 22222 33333 33333 44444 </pre>

</dd> <dt>‘<samp class="samp">mergex2, 7</samp>’</dt> <dd> Move odd frames into the upper field, even into the lower field, generating a double height frame at same frame rate.   <pre class="example-preformatted"> ------> time Input: Frame 1 Frame 2 Frame 3 Frame 4

11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444 11111 22222 33333 44444

Output: 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 11111 33333 33333 55555 22222 22222 44444 44444 </pre>

</dd> </dl>

Numeric values are deprecated but are accepted for backward compatibility reasons. Default mode is <code class="code">merge</code>. </dd> <dt><samp class="option">flags</samp></dt> <dd> Specify flags influencing the filter process. Available value for <var class="var">flags</var> is: <dl class="table"> <dt><samp class="option">low_pass_filter, vlpf</samp></dt> <dd> Enable linear vertical low-pass filtering in the filter. Vertical low-pass filtering is required when creating an interlaced destination from a progressive source which contains high-frequency vertical detail. Filtering will reduce interlace ’twitter’ and Moire patterning. </dd> <dt><samp class="option">complex_filter, cvlpf</samp></dt> <dd> Enable complex vertical low-pass filtering. This will slightly less reduce interlace ’twitter’ and Moire patterning but better retain detail and subjective sharpness impression. </dd> <dt><samp class="option">bypass_il</samp></dt> <dd> Bypass already interlaced frames, only adjust the frame rate. </dd> </dl>

Vertical low-pass filtering and bypassing already interlaced frames can only be enabled for <samp class="option">mode</samp> <var class="var">interleave_top</var> and <var class="var">interleave_bottom</var>. </dd> </dl>

 <h3 class="section">32.260 tmedian# TOC</h3> Pick median pixels from several successive input video frames. The filter accepts the following options: <dl class="table"> <dt><samp

class=”option”>radius</samp></dt> <dd> Set radius of median filter. Default is 1. Allowed range is from 1 to 127. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>15</code>, by which all planes are processed. </dd> <dt><samp class=”option”>percentile</samp></dt> <dd> Set median percentile. Default value is <code class=”code”>0.5</code>. Default value of <code class=”code”>0.5</code> will pick always median values, while <code class=”code”>0</code> will pick minimum values, and <code class=”code”>1</code> maximum values. </dd> </dl>

 <h4 class=”subsection”>32.260.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class=”code”>radius</code>. <h3 class=”section”>32.261 tmidequalizer# TOC</h3>

Apply Temporal Midway Video Equalization effect. Midway Video Equalization adjusts a sequence of video frames to have the same histograms, while maintaining their dynamics as much as possible. It’s useful for e.g. matching exposures from a video frames sequence. This filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>radius</samp></dt> <dd> Set filtering radius. Default is <code class=”code”>5</code>. Allowed range is from 1 to 127. </dd> <dt><samp class=”option”>sigma</samp></dt> <dd> Set filtering sigma. Default is <code class=”code”>0.5</code>. This controls strength of filtering. Setting this option to 0 effectively does nothing. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to process. Default is <code class=”code”>15</code>, which is all available planes. </dd> </dl>

 <h3 class=”section”>32.262 tmix# TOC</h3>

Mix successive video frames. A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>frames</samp></dt> <dd> The number of successive frames to mix. If unspecified, it defaults to 3. </dd> <dt><samp class=”option”>weights</samp></dt> <dd> Specify weight of each input video frame. Each weight is separated by space. If number of weights is smaller than number of <var class=”var”>frames</var> last specified weight will be used for all remaining unset weights. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Specify scale, if it is set it will be multiplied with sum of each weight

multiplied with pixel values to give final destination pixel value. By default <var class="var">scale</var> is auto scaled to sum of weights. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default is all. Allowed range is from 0 to 15. </dd> </dl>

 <h4 class="subsection">32.262.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Average 7 successive frames: <pre class="example-preformatted">tmix=frames=7:weights="1 1 1 1 1 1 1" </pre>

Apply simple temporal convolution: <pre class="example-preformatted">tmix=frames=3:weights="-1 3 -1" </pre>

Similar as above but only showing temporal differences: <pre class="example-preformatted">tmix=frames=3:weights="-1 2 -1":scale=1 </pre>

 <h4 class="subsection">32.262.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">weights</samp></dt> <dt><samp class="option">scale</samp></dt> <dt><samp class="option">planes</samp></dt> <dd> Syntax is same as option with same name. </dd> </dl>

 <h3 class="section">32.263 tonemap# TOC</h3> Tone map colors from different dynamic ranges. This filter expects data in single precision floating point, as it needs to operate on (and can output) out-of-range values. Another filter, such as zscale, is needed to convert the resulting frame to a usable format. The tonemapping algorithms implemented only work on linear light, so input data should be linearized beforehand (and possibly correctly tagged).   <pre class="example-preformatted">ffmpeg -i INPUT -vf zscale=transfer=linear,tonemap=clip,zscale=transfer=bt709,format=yuv420p OUTPUT </pre>

 <h4 class="subsection">32.263.1 Options<a class="anchor hidden-xs" href="#Options-

41” aria-hidden=”true”># TOC</h4> The filter accepts the following options. <dl class=”table”> <dt><samp class=”option”>tonemap</samp></dt> <dd> Set the tone map algorithm to use. Possible values are: <dl class=”table”> <dt><var class=”var”>none</var></dt> <dd> Do not apply any tone map, only desaturate overbright pixels. </dd> <dt><var class=”var”>clip</var></dt> <dd> Hard-clip any out-of-range values. Use it for perfect color accuracy for in-range values, while distorting out-of-range values. </dd> <dt><var class=”var”>linear</var></dt> <dd> Stretch the entire reference gamut to a linear multiple of the display. </dd> <dt><var class=”var”>gamma</var></dt> <dd> Fit a logarithmic transfer between the tone curves. </dd> <dt><var class=”var”>reinhard</var></dt> <dd> Preserve overall image brightness with a simple curve, using nonlinear contrast, which results in flattening details and degrading color accuracy. </dd> <dt><var class=”var”>hable</var></dt> <dd> Preserve both dark and bright details better than <var class=”var”>reinhard</var>, at the cost of slightly darkening everything. Use it when detail preservation is more important than color and brightness accuracy. </dd> <dt><var class=”var”>mobius</var></dt> <dd> Smoothly map out-of-range values, while retaining contrast and colors for in-range material as much as possible. Use it when color accuracy is more important than detail preservation. </dd> </dl>

Default is none. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Tune the tone mapping algorithm. This affects the following algorithms: <dl class=”table”> <dt><var class=”var”>none</var></dt> <dd> Ignored. </dd> <dt><var class=”var”>linear</var></dt> <dd> Specifies the scale factor to use while stretching. Default to 1.0. </dd> <dt><var class=”var”>gamma</var></dt> <dd> Specifies the exponent of the function. Default to 1.8. </dd> <dt><var class=”var”>clip</var></dt> <dd> Specify an extra linear coefficient to multiply into the signal before clipping. Default to 1.0. </dd> <dt><var class=”var”>reinhard</var></dt> <dd> Specify the local contrast coefficient at the display peak. Default to 0.5, which means that in-gamut values will be about half as bright as when clipping. </dd> <dt><var class=”var”>hable</var></dt> <dd> Ignored. </dd> <dt><var class=”var”>mobius</var></dt> <dd> Specify the transition point from linear to mobius transform. Every value below this point is guaranteed to be mapped 1:1. The higher the value, the more accurate the result will be, at the cost of losing bright details. Default to 0.3, which due to the steep initial slope still preserves in-range colors fairly accurately. </dd> </dl>

</dd> <dt><samp class=”option”>desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher

the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by (smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default of 2.0 is somewhat conservative and will mostly just apply to skies or directly sunlit surfaces. A setting of 0.0 disables this option. This option works only if the input frame has a supported color tag. </dd> <dt><samp class="option">peak</samp></dt> <dd> Override signal/nominal/reference peak with this value. Useful when the embedded peak information in display metadata is not reliable or when tone mapping from a lower range to a higher range. </dd> </dl>

 <h3 class="section">32.264 tpad# TOC</h3>

Temporarily pad video frames. The filter accepts the following options: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Specify number of delay frames before input video stream. Default is 0. </dd> <dt><samp class="option">stop</samp></dt> <dd> Specify number of padding frames after input video stream. Set to -1 to pad indefinitely. Default is 0. </dd> <dt><samp class="option">start_mode</samp></dt> <dd> Set kind of frames added to beginning of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of first frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">stop_mode</samp></dt> <dd> Set kind of frames added to end of stream. Can be either <var class="var">add</var> or <var class="var">clone</var>. With <var class="var">add</var> frames of solid-color are added. With <var class="var">clone</var> frames are clones of last frame. Default is <var class="var">add</var>. </dd> <dt><samp class="option">start_duration, stop_duration</samp></dt> <dd> Specify the duration of the start/stop delay. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. These options override <var class="var">start</var> and <var class="var">stop</var>. Default is 0. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. The default value of <var class="var">color</var> is "black". </dd> </dl>

 <h3 class="section">32.265 transpose<a class="anchor

hidden-xs" href="#transpose" aria-hidden="true"># TOC</h3>

Transpose rows with columns in the input video and optionally flip it. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">0, 4, cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip (default), that is: <pre class="example-preformatted">L.R L.l . . -> . . l.r R.r </pre>

</dd> <dt>‘<samp class="samp">1, 5, clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise, that is: <pre class="example-preformatted">L.R l.L . . -> . . l.r r.R </pre>

</dd> <dt>‘<samp class="samp">2, 6, cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise, that is: <pre class="example-preformatted">L.R R.r . . -> . . l.r L.l </pre>

</dd> <dt>‘<samp class="samp">3, 7, clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip, that is: <pre class="example-preformatted">L.R r.R . . -> . . l.r l.L </pre> </dd> </dl>

For values between 4-7, the transposition is only done if the input video geometry is portrait and not landscape. These values are deprecated, the <code class="code">passthrough</code> option should be used instead. Numerical values are deprecated, and should be dropped in favor of symbolic constants. </dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. </dd> <dt>‘<samp class="samp">portrait</samp>’</dt> <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

Default value is <code class="code">none</code>. </dd> </dl>

For example to rotate by 90 degrees clockwise and preserve portrait layout: <pre class="example-preformatted">transpose=dir=1:passthrough=portrait </pre>

The command above can also be specified as: <pre class="example-preformatted">transpose=1:portrait </pre>

 <h3 class="section">32.266 trim# TOC</h3> Trim the input so that the output contains one continuous subpart of the input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">start</samp></dt> <dd> Specify the time of the start of the kept section, i.e. the frame with the timestamp <var class="var">start</var> will be the first frame in the output. </dd> <dt><samp class="option">end</samp></dt> <dd> Specify the time of the first frame that will be dropped, i.e. the frame immediately preceding the one with the timestamp <var class="var">end</var> will be the last frame in the output. </dd> <dt><samp class="option">start_pts</samp></dt> <dd> This is the same as <var class="var">start</var>, except this option sets the start timestamp in timebase units instead of seconds. </dd> <dt><samp class="option">end_pts</samp></dt> <dd> This is the same as <var class="var">end</var>, except this option sets the end timestamp in timebase units instead of seconds. </dd> <dt><samp class="option">duration</samp></dt> <dd> The maximum duration of the output in seconds. </dd> <dt><samp class="option">start_frame</samp></dt> <dd> The number of the first frame that should be passed to the output. </dd> <dt><samp class="option">end_frame</samp></dt> <dd> The number of the first frame that should be dropped. </dd> </dl>

<samp class="option">start</samp>, <samp class="option">end</samp>, and <samp class="option">duration</samp> are expressed as time duration specifications; see <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. Note that the first two sets of the start/end options and the <samp class="option">duration</samp> option look at the frame timestamp, while the _frame variants simply count the frames that pass through the filter. Also note that this filter does not modify the timestamps. If you wish for the output timestamps to start at zero, insert a setpts filter after the trim filter. If multiple start or end options are set, this filter tries to be greedy and keep all the frames that match at least one of the specified constraints. To keep only the part that matches all the constraints at once, chain multiple trim filters. The defaults are such that all the input is kept. So it is possible to set e.g. just the end values to keep everything before the specified time. Examples: <ul class="itemize mark-bullet"> Drop everything except the second minute of input: <pre class="example-preformatted">ffmpeg -i INPUT -vf trim=60:120 </pre>

Keep only the first second: <pre class="example-preformatted">ffmpeg -i INPUT -vf trim=duration=1 </pre>

 <h3 class="section">32.267 unpremultiply# TOC</h3> Apply alpha unpremultiply effect to input video stream using first plane of second stream as alpha. Both streams must have same dimensions and same pixel format. The filter accepts the following option: <dl class="table"> <dt><samp class="option">planes</samp></dt> <dd> Set which planes will be processed, unprocessed planes will be copied. By default value 0xf, all planes will be processed. If the format has 1 or 2 components, then luma is bit 0. If the format has 3 or 4 components: for RGB formats bit 0 is green, bit 1 is blue and bit 2 is red; for YUV formats bit 0 is luma, bit 1 is chroma-U and bit 2 is chroma-V. If present, the alpha channel is always the last bit. </dd> <dt><samp class="option">inplace</samp></dt> <dd> Do not require 2nd input for processing, instead use alpha plane from input stream. </dd> </dl>

 <h3 class="section">32.268 unsharp# TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">luma_amount, la</samp></dt> <dd> Set the luma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 1.0. </dd> <dt><samp class="option">chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class="option">chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> <dt><samp class="option">alpha_msize_x, ax</samp></dt> <dd> Set the alpha matrix horizontal size. It must be an odd integer

between 3 and 23. The default value is 5. </dd> <dt><samp class=”option”>alpha_msize_y, ay</samp></dt> <dd> Set the alpha matrix vertical size. It must be an odd integer between 3 and 23. The default value is 5. </dd> <dt><samp class=”option”>alpha_amount, aa</samp></dt> <dd> Set the alpha effect strength. It must be a floating point number, reasonable values lay between -1.5 and 1.5. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. Default value is 0.0. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class=”subsection”>32.268.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply strong luma sharpen effect: <pre class=”example-preformatted”>unsharp=luma_msize_x=7:luma_msize_y=7:luma_
</pre>

Apply a strong blur of both luma and chroma parameters: <pre class=”example-preformatted”>unsharp=7:7:-2:7:7:-2
</pre>

 <h3 class=”section”>32.269 untile# TOC</h3>

Decompose a video made of tiled images into the individual images. The frame rate of the output video is the frame rate of the input video multiplied by the number of tiles. This filter does the reverse of tile. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>layout</samp></dt> <dd> Set the grid size (i.e. the number of lines and columns). For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class=”subsection”>32.269.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Produce a 1-second video from a still image file made of 25 frames stacked vertically, like an analogic film reel: <pre class=”example-preformatted”>ffmpeg -r 1 -i image.jpg -vf untile=1x25 movie.mkv </pre>

 <h3 class="section">32.270 uspp# TOC</h3>

Apply ultra slow/simple postprocessing filter that compresses and decompresses the image at several (or - in the case of <samp class="option">quality</samp> level <code class="code">8</code> - all) shifts and average the results. The way this differs from the behavior of spp is that uspp actually encodes & decodes each case with libavcodec Snow, whereas spp uses a simplified intra only 8x8 DCT similar to MJPEG. This filter is not available in ffmpeg versions between 5.0 and 6.0. The filter accepts the following options: <dl class="table"> <dt><samp class="option">quality</samp></dt> <dd> Set quality. This option defines the number of levels for averaging. It accepts an integer in the range 0-8. If set to <code class="code">0</code>, the filter will have no effect. A value of <code class="code">8</code> means the higher quality. For each increment of that value the speed drops by a factor of approximately 2. Default value is <code class="code">3</code>. </dd> <dt><samp class="option">qp</samp></dt> <dd> Force a constant quantization parameter. If not set, the filter will use the QP from the video stream (if available). </dd> <dt><samp class="option">codec</samp></dt> <dd> Use specified codec instead of snow. </dd> </dl>

 <h3 class="section">32.271 v360# TOC</h3>

Convert 360 videos between various formats. The filter accepts the following options: <dl class="table"> <dt><samp class="option">input</samp></dt> <dt><samp class="option">output</samp></dt> <dd> Set format of the input/output video. Available formats: <dl class="table"> <dt>‘<samp class="samp">e</samp>’</dt> <dt>‘<samp class="samp">equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class="samp">c3x2</samp>’</dt> <dt>‘<samp class="samp">c6x1</samp>’</dt> <dt>‘<samp class="samp">c1x6</samp>’</dt> <dd> Cubemap with 3x2/6x1/1x6 layout. Format specific options: <dl class="table"> <dt><samp class="option">in_pad</samp></dt> <dt><samp class="option">out_pad</samp></dt> <dd> Set padding proportion for the input/output cubemap. Values in decimals. Example values: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> No padding. </dd> <dt>‘<samp class="samp">0.01</samp>’</dt> <dd> 1% of face is padding. For example, with 1920x1280 resolution face size would be 640x640 and padding would be 3 pixels from each side. (640 * 0.01 = 6 pixels) </dd> </dl>

Default value is ‘<samp class=”samp”>0</samp>’.
Maximum value is ‘<samp class=”samp”>0.1</samp>’.
 </dd> <dt><samp class=”option”>fin_pad</samp></dt>
<dt><samp class=”option”>fout_pad</samp></dt> <dd> Set fixed
padding for the input/output cubemap. Values in pixels. Default
value is ‘<samp class=”samp”>0</samp>’.
If greater than zero it overrides other padding options. </dd>
<dt><samp class=”option”>in_forder</samp></dt> <dt><samp
class=”option”>out_forder</samp></dt> <dd> Set order of faces
for the input/output cubemap. Choose one direction for each position.
Designation of directions: <dl class=”table”> <dt>‘<samp
class=”samp”>r</samp>’</dt> <dd> right </dd>
<dt>‘<samp class=”samp”>l</samp>’</dt> <dd> left
</dd> <dt>‘<samp class=”samp”>u</samp>’</dt>
<dd> up </dd> <dt>‘<samp class=”samp”>d</samp>’</dt>
<dd> down </dd> <dt>‘<samp class=”samp”>f</samp>’</dt>
<dd> forward </dd> <dt>‘<samp class=”samp”>b</samp>’</dt>
<dd> back </dd> </dl>

Default value is ‘<samp class=”samp”>rludfb</samp>’.
 </dd> <dt><samp class=”option”>in_frot</samp></dt> <dt><samp
class=”option”>out_frot</samp></dt> <dd> Set rotation of faces for
the input/output cubemap. Choose one angle for each position.
Designation of angles: <dl class=”table”> <dt>‘<samp
class=”samp”>0</samp>’</dt> <dd> 0 degrees clockwise
</dd> <dt>‘<samp class=”samp”>1</samp>’</dt>
<dd> 90 degrees clockwise </dd> <dt>‘<samp class=”samp”>2</samp>’</dt>
<dd> 180 degrees clockwise </dd> <dt>‘<samp class=”samp”>3</samp>’</dt>
<dd> 270 degrees clockwise </dd> </dl>

Default value is ‘<samp class=”samp”>000000</samp>’.
</dd> </dl>

</dd> <dt>‘<samp class=”samp”>eac</samp>’</dt>
<dd> Equi-Angular Cubemap. </dd> <dt>‘<samp
class=”samp”>flat</samp>’</dt> <dt>‘<samp class=”samp”>gnomonic</samp>’<
<dt>‘<samp class=”samp”>rectilinear</samp>’</dt>
<dd> Regular video. Format specific options: <dl
class=”table”> <dt><samp class=”option”>h_fov</samp></dt> <dt><samp
class=”option”>v_fov</samp></dt> <dt><samp class=”option”>d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Val-
ues in degrees. If diagonal field of view is set it over-
rides horizontal and vertical field of view. </dd> <dt><samp
class=”option”>ih_fov</samp></dt> <dt><samp class=”option”>iv_fov</samp></dt>
<dt><samp class=”option”>id_fov</samp></dt> <dd> Set input hori-
zontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>

</dl>

</dd> <dt>‘<samp class="samp">dfisheye</samp>’</dt>
<dd> Dual fisheye. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">barrel</samp>’</dt>
<dt>‘<samp class="samp">fb</samp>’</dt> <dt>‘<samp
class="samp">barrelsplit</samp>’</dt> <dd> Facebook’s
360 formats. </dd> <dt>‘<samp class="samp">sg</samp>’</dt>
<dd> Stereographic format. Format specific options:
<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">mercator</samp>’</dt>
<dd> Mercator format. </dd> <dt>‘<samp class="samp">ball</samp>’</dt>
<dd> Ball format, gives significant distortion toward the back.
</dd> <dt>‘<samp class="samp">hammer</samp>’</dt>
<dd> Hammer-Aitoff map projection format. </dd> <dt>‘<samp
class="samp">sinusoidal</samp>’</dt> <dd> Sinusoidal map
projection format. </dd> <dt>‘<samp class="samp">fisheye</samp>’</dt>
<dd> Fisheye projection. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal

field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">pannini</samp>’</dt>
<dd> Pannini projection. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dd> Set
output pannini parameter. </dd> <dt><samp class="option">ih_fov</samp></dt>
<dd> Set input pannini parameter. </dd> </dl>

</dd> <dt>‘<samp class="samp">cylindrical</samp>’</dt>
<dd> Cylindrical projection. Format specific options:
<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">perspective</samp>’</dt>
<dd> Perspective projection. (output only)
Format specific options: <dl class="table"> <dt><samp
class="option">v_fov</samp></dt> <dd> Set perspective parameter. </dd> </dl>

</dd> <dt>‘<samp class="samp">tetrahedron</samp>’</dt>
<dd> Tetrahedron projection. </dd> <dt>‘<samp
class="samp">tsp</samp>’</dt> <dd> Truncated square pyramid projection. </dd> <dt>‘<samp class="samp">he</samp>’</dt>
<dt>‘<samp class="samp">hequirect</samp>’</dt> <dd> Half
equirectangular projection. </dd> <dt>‘<samp class="samp">equisolid</samp>’</dt
<dd> Equisolid format. Format specific options: <dl
class="table"> <dt><samp class="option">h_fov</samp></dt> <dt><samp
class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp
class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal
field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">og</samp>’</dt>
<dd> Orthographic format. Format specific options:

<dl class="table"> <dt><samp class="option">h_fov</samp></dt>
<dt><samp class="option">v_fov</samp></dt> <dt><samp class="option">d_fov</samp></dt>
<dd> Set output horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt>
<dt><samp class="option">id_fov</samp></dt> <dd> Set input horizontal/vertical/diagonal field of view. Values in degrees. If diagonal field of view is set it overrides horizontal and vertical field of view. </dd>
</dl>

</dd> <dt>‘<samp class="samp">octahedron</samp>’</dt>
<dd> Octahedron projection. </dd> <dt>‘<samp class="samp">cylindricalea</samp>’</dt> <dd> Cylindrical Equal Area projection. </dd> </dl>

</dd> <dt><samp class="option">interp</samp></dt> <dd> Set interpolation method.
 Note: more complex interpolation methods require much more memory to run. Available methods:
 <dl class="table"> <dt>‘<samp class="samp">near</samp>’</dt>
<dt>‘<samp class="samp">nearest</samp>’</dt> <dd> Nearest neighbour. </dd> <dt>‘<samp class="samp">line</samp>’</dt>
<dt>‘<samp class="samp">linear</samp>’</dt> <dd> Bilinear interpolation. </dd> <dt>‘<samp class="samp">lagrange9</samp>’</dt>
<dd> Lagrange9 interpolation. </dd> <dt>‘<samp class="samp">cube</samp>’</dt> <dt>‘<samp class="samp">cubic</samp>’</dt
<dd> Bicubic interpolation. </dd> <dt>‘<samp class="samp">lanc</samp>’</dt
<dt>‘<samp class="samp">lanczos</samp>’</dt> <dd> Lanczos interpolation. </dd> <dt>‘<samp class="samp">sp16</samp>’</dt>
<dt>‘<samp class="samp">spline16</samp>’</dt> <dd> Spline16 interpolation. </dd> <dt>‘<samp class="samp">gauss</samp>’</dt>
<dt>‘<samp class="samp">gaussian</samp>’</dt> <dd> Gaussian interpolation. </dd> <dt>‘<samp class="samp">mitchell</samp>’</dt>
<dd> Mitchell interpolation. </dd> </dl>

Default value is ‘<samp class="samp">line</samp>’.
 </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video resolution. Default resolution depends on formats.
</dd> <dt><samp class="option">in_stereo</samp></dt> <dt><samp class="option">out_stereo</samp></dt> <dd> Set the input/output stereo format. <dl class="table"> <dt>‘<samp class="samp">2d</samp>’</dt>
<dd> 2D mono </dd> <dt>‘<samp class="samp">sbs</samp>’</dt>
<dd> Side by side </dd> <dt>‘<samp class="samp">tb</samp>’</dt>
<dd> Top bottom </dd> </dl>

Default value is ‘<samp class="samp">2d</samp>’
for input and output format. </dd> <dt><samp class="option">yaw</samp></dt>

<dt><samp class="option">pitch</samp></dt> <dt><samp class="option">roll</samp></dt> <dd> Set rotation for the output video. Values in degrees. </dd> <dt><samp class="option">rorder</samp></dt> <dd> Set rotation order for the output video. Choose one item for each position. <dl class="table"> <dt>‘<samp class="samp">y, Y</samp>’</dt> <dd> yaw </dd> <dt>‘<samp class="samp">p, P</samp>’</dt> <dd> pitch </dd> <dt>‘<samp class="samp">r, R</samp>’</dt> <dd> roll </dd> </dl>

Default value is ‘<samp class="samp">ypr</samp>’. </dd> <dt><samp class="option">h_flip</samp></dt> <dt><samp class="option">v_flip</samp></dt> <dt><samp class="option">d_flip</samp></dt> <dd> Flip the output video horizontally(swaps left-right)/vertically(swaps up-down)/in-depth(swaps back-forward). Boolean values. </dd> <dt><samp class="option">ih_flip</samp></dt> <dt><samp class="option">iv_flip</samp></dt> <dd> Set if input video is flipped horizontally/vertically. Boolean values. </dd> <dt><samp class="option">in_trans</samp></dt> <dd> Set if input video is transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">out_trans</samp></dt> <dd> Set if output video needs to be transposed. Boolean value, by default disabled. </dd> <dt><samp class="option">h_offset</samp></dt> <dt><samp class="option">v_offset</samp></dt> <dd> Set output horizontal/vertical off-axis offset. Default is set to 0. Allowed range is from -1 to 1. </dd> <dt><samp class="option">alpha_mask</samp></dt> <dd> Build mask in alpha plane for all unmapped pixels by marking them fully transparent. Boolean value, by default disabled. </dd> <dt><samp class="option">reset_rot</samp></dt> <dd> Reset rotation of output video. Boolean value, by default disabled. </dd> </dl>

 <h4 class="subsection">32.271.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Convert equirectangular video to cubemap with 3x2 layout and 1% padding using bicubic interpolation: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=e:c3x2:cubic:out_pad=0.01 output.mkv </pre> Extract back view of Equi-Angular Cubemap: <pre class="example-preformatted">ffmpeg -i input.mkv -vf v360=eac:flat:yaw=180 output.mkv </pre> Convert transposed and horizontally flipped Equi-Angular Cubemap in side-by-side stereo format to equirectangular top-bottom stereo format: <pre class="example-preformatted">v360=eac:equirect:in_stereo=sbs:in_trans=1:ih_flip=1:out_stereo=tb </pre>

 <h4 class="subsection">32.271.2 Commands# TOC</h4>

This filter supports subset of above options as commands. <h3 class="section">32.272 vaguedenoiser# TOC</h3>

Apply a wavelet based denoiser. It transforms each frame from the video input into the wavelet domain, using Cohen-Daubechies-Feauveau 9/7. Then it applies some filtering to the obtained coefficients. It does an inverse wavelet transform after. Due to wavelet properties, it should give a nice smoothed result, and reduced noise, without blurring picture features. This filter accepts the following options: <dl class="table"> <dt><samp class="option">threshold</samp></dt> <dd> The filtering strength. The higher, the more filtered the video will be. Hard thresholding can use a higher threshold than soft thresholding before the video looks overfiltered. Default value is 2. </dd> <dt><samp class="option">method</samp></dt> <dd> The filtering method the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">hard</samp>’</dt> <dd> All values under the threshold will be zeroed. </dd> <dt>‘<samp class="samp">soft</samp>’</dt> <dd> All values under the threshold will be zeroed. All values above will be reduced by the threshold. </dd> <dt>‘<samp class="samp">garrote</samp>’</dt> <dd> Scales or nullifies coefficients - intermediary between (more) soft and (less) hard thresholding. </dd> </dl>

Default is garrote. </dd> <dt><samp class="option">nsteps</samp></dt> <dd> Number of times, the wavelet will decompose the picture. Picture can’t be decomposed beyond a particular point (typically, 8 for a 640x480 frame - as 2^9 = 512 > 480). Valid values are integers between 1 and 32. Default value is 6. </dd> <dt><samp class="option">percent</samp></dt> <dd> Partial of full denoising (limited coefficients shrinking), from 0 to 100. Default value is 85. </dd> <dt><samp class="option">planes</samp></dt> <dd> A list of the planes to process. By default all planes are processed. </dd> <dt><samp class="option">type</samp></dt> <dd> The threshold type the filter will use. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">universal</samp>’</dt> <dd> Threshold used is same for all decompositions. </dd> <dt>‘<samp class="samp">bayes</samp>’</dt> <dd> Threshold used

depends also on each decomposition coefficients. </dd> </dl>

Default is universal. </dd> </dl>

 <h3 class="section">32.273 varblur# TOC</h3> Apply variable blur filter by using 2nd video stream to set blur radius. The 2nd stream must have the same dimensions. This filter accepts the following options: <dl class="table"> <dt><samp class="option">min_r</samp></dt> <dd> Set min allowed radius. Allowed range is from 0 to 254. Default is 0. </dd> <dt><samp class="option">max_r</samp></dt> <dd> Set max allowed radius. Allowed range is from 1 to 255. Default is 8. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to process. By default, all are used. </dd> </dl>

The <code class="code">varblur</code> filter also supports the framesync options. <h4 class="subsection">32.273.1 Commands# TOC</h4>

This filter supports all the above options as commands. <h3 class="section">32.274 vectorscope# TOC</h3>

Display 2 color component values in the two dimensional graph (which is called a vectorscope). This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode, m</samp></dt> <dd> Set vectorscope mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">gray</samp>’</dt> <dt>‘<samp class="samp">tint</samp>’</dt> <dd> Gray values are displayed on graph, higher brightness means more pixels have same component color value on location in graph. This is the default mode. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Gray values are displayed on graph. Surrounding pixels values which are not present in video frame are drawn in gradient of 2 color components which are set by option <code class="code">x</code> and <code class="code">y</code>. The 3rd color component is static. </dd> <dt>‘<samp class="samp">color2</samp>’</dt> <dd> Actual color components values present in video frame are displayed on graph. </dd> <dt>‘<samp class="samp">color3</samp>’</dt> <dd> Similar as color2 but higher frequency of same values <code

class=”code”>x</code> and <code class=”code”>y</code> on graph increases value of another color component, which is luminance by default values of <code class=”code”>x</code> and <code class=”code”>y</code>. </dd> <dt>‘<samp class=”samp”>color4</samp>’</dt> <dd> Actual colors present in video frame are displayed on graph. If two different colors map to same position on graph then color with higher value of component not present in graph is picked. </dd> <dt>‘<samp class=”samp”>color5</samp>’</dt> <dd> Gray values are displayed on graph. Similar to <code class=”code”>color</code> but with 3rd color component picked from radial gradient. </dd> </dl>

</dd> <dt><samp class=”option”>x</samp></dt> <dd> Set which color component will be represented on X-axis. Default is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set which color component will be represented on Y-axis. Default is <code class=”code”>2</code>. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity, used by modes: gray, color, color3 and color5 for increasing brightness of color component which represents frequency of (X, Y) location in graph. </dd> <dt><samp class=”option”>envelope, e</samp></dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class=”samp”>instant</samp>’</dt> <dd> Instant envelope, even darkest single pixel will be clearly highlighted. </dd> <dt>‘<samp class=”samp”>peak</samp>’</dt> <dd> Hold maximum and minimum values presented in graph over time. This way you can still spot out of range values without constantly looking at vectorscope. </dd> <dt>‘<samp class=”samp”>peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class=”option”>graticule, g</samp></dt> <dd> Set what kind of graticule to draw. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dt>‘<samp class=”samp”>green</samp>’</dt> <dt>‘<samp class=”samp”>color</samp>’</dt> <dt>‘<samp class=”samp”>invert</samp>’</dt> </dl>

</dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class=”option”>flags, f</samp></dt> <dd> Set graticule flags. <dl class=”table”> <dt>‘<samp class=”samp”>white</samp>’</dt> <dd> Draw graticule for white point. </dd> <dt>‘<samp class=”samp”>black</samp>’</dt> <dd> Draw graticule for black point. </dd> <dt>‘<samp class=”samp”>name</samp>’</dt> <dd> Draw color points short names. </dd> </dl>

</dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class=”option”>lthreshold,

l</samp></dt> <dd> Set low threshold for color component not represented on X or Y axis. Values lower than this value will be ignored. Default is 0. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and low threshold value of 0.1 actual threshold is 0.1 * 255 = 25. </dd> <dt><samp class="option">hthreshold, h</samp></dt> <dd> Set high threshold for color component not represented on X or Y axis. Values higher than this value will be ignored. Default is 1. Note this value is multiplied with actual max possible value one pixel component can have. So for 8-bit input and high threshold value of 0.9 actual threshold is 0.9 * 255 = 230. </dd> <dt><samp class="option">colorspace, c</samp></dt> <dd> Set what kind of colorspace to use when drawing graticule. <dl class="table"> <dt>‘<samp class="samp">auto</samp>’</dt> <dt>‘<samp class="samp">601</samp>’</dt> <dt>‘<samp class="samp">709</samp>’</dt> </dl> Default is auto. </dd> <dt><samp class="option">tint0, t0</samp></dt> <dt><samp class="option">tint1, t1</samp></dt> <dd> Set color tint for gray/tint vectorscope mode. By default both options are zero. This means no tint, and output will remain gray. </dd> </dl>

 <h3 class="section">32.275 vidstabdetect# TOC</h3>

Analyze video stabilization/deshaking. Perform pass 1 of 2, see vidstabtransform for pass 2. This filter generates a file with relative translation and rotation transform information about subsequent frames, which is then used by the vidstabtransform filter. To enable compilation of this filter you need to configure FFmpeg with <code class="code">--enable-libvidstab</code>. This filter accepts the following options: <dl class="table"> <dt><samp class="option">result</samp></dt> <dd> Set the path to the file used to write the transforms information. Default value is <samp class="file">transforms.trf</samp>. </dd> <dt><samp class="option">shakiness</samp></dt> <dd> Set how shaky the video is and how quick the camera is. It accepts an integer in the range 1-10, a value of 1 means little shakiness, a value of 10 means strong shakiness. Default value is 5. </dd> <dt><samp class="option">accuracy</samp></dt> <dd> Set the accuracy of the detection process. It must be a value in the range 1-15. A value of 1 means low accuracy, a value of 15 means high accuracy. Default value is 15. </dd> <dt><samp class="option">stepsize</samp></dt> <dd> Set stepsize of the search process. The region around minimum is scanned with 1 pixel resolution. Default value is 6. </dd> <dt><samp class="option">mincontrast</samp></dt> <dd> Set minimum con-

trast. Below this value a local measurement field is discarded. Must be a floating point value in the range 0-1. Default value is 0.3. </dd> <dt><samp class="option">tripod</samp></dt> <dd> Set reference frame number for tripod mode. If enabled, the motion of the frames is compared to a reference frame in the filtered stream, identified by the specified number. The idea is to compensate all movements in a more-or-less static scene and keep the camera view absolutely still. If set to 0, it is disabled. The frames are counted starting from 1. </dd> <dt><samp class="option">show</samp></dt> <dd> Show fields and transforms in the resulting frames. It accepts an integer in the range 0-2. Default value is 0, which disables any visualization. </dd> <dt><samp class="option">fileformat</samp></dt> <dd> Format for the transforms data file to be written. Acceptable values are <dl class="table"> <dt>‘<samp class="samp">ascii</samp>’</dt> <dd> Human-readable plain text </dd> <dt>‘<samp class="samp">binary</samp>’</dt> <dd> Binary format, roughly 40% smaller than <code class="code">ascii</code>. (default) </dd> </dl>

</dd> </dl>

 <h4 class="subsection">32.275.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use default values: <pre class="example-preformatted">vidstabdetect </pre>

Analyze strongly shaky movie and put the results in file <samp class="file">mytransforms.trf</samp>: <pre class="example-preformatted">vidstabdetect=shakiness=10:accuracy=15:result="mytransforms.trf&qu
</pre>

Visualize the result of internal transformations in the resulting video: <pre class="example-preformatted">vidstabdetect=show=1
</pre>

Analyze a video with medium shakiness using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i input -vf vidstabdetect=shakiness=5:show=1 dummy.avi </pre>

 <h3 class="section">32.276 vidstabtransform# TOC</h3>

Video stabilization/deshaking: pass 2 of 2, see vidstabdetect for pass 1. Read

a file with transform information for each frame and apply/compensate them. Together with the vidstabdetect filter this can be used to deshake videos. See also http://public.hronopik.de/vid.stab. It is important to also use the unsharp filter, see below. To enable compilation of this filter you need to configure FFmpeg with <code class=”code”>--enable-libvidstab</code>. <h4 class=”subsection”>32.276.1 Options# TOC</h4>

<dl class=”table”> <dt><samp class=”option”>input</samp></dt> <dd> Set path to the file used to read the transforms. Default value is <samp class=”file”>transforms.trf</samp>. </dd> <dt><samp class=”option”>smoothing</samp></dt> <dd> Set the number of frames (value*2 + 1) used for lowpass filtering the camera movements. Default value is 10. For example a number of 10 means that 21 frames are used (10 in the past and 10 in the future) to smoothen the motion in the video. A larger value leads to a smoother video, but limits the acceleration of the camera (pan/tilt movements). 0 is a special case where a static camera is simulated. </dd> <dt><samp class=”option”>optalgo</samp></dt> <dd> Set the camera path optimization algorithm. Accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>gauss</samp>’</dt> <dd> gaussian kernel low-pass filter on camera motion (default) </dd> <dt>‘<samp class=”samp”>avg</samp>’</dt> <dd> averaging on transformations </dd> </dl>

</dd> <dt><samp class=”option”>maxshift</samp></dt> <dd> Set maximal number of pixels to translate frames. Default value is -1, meaning no limit. </dd> <dt><samp class=”option”>maxangle</samp></dt> <dd> Set maximal angle in radians (degree*PI/180) to rotate frames. Default value is -1, meaning no limit. </dd> <dt><samp class=”option”>crop</samp></dt> <dd> Specify how to deal with borders that may be visible due to movement compensation. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>keep</samp>’</dt> <dd> keep image information from previous frame (default) </dd> <dt>‘<samp class=”samp”>black</samp>’</dt> <dd> fill the border black </dd> </dl>

</dd> <dt><samp class=”option”>invert</samp></dt> <dd> Invert transforms if set to 1. Default value is 0. </dd> <dt><samp class=”option”>relative</samp></dt> <dd> Consider transforms as relative to previous frame if set to 1, absolute if set to 0. Default value is 0. </dd> <dt><samp class=”option”>zoom</samp></dt>

<dd> Set percentage to zoom. A positive value will result in a zoom-in effect, a negative value in a zoom-out effect. Default value is 0 (no zoom). </dd> <dt><samp class="option">optzoom</samp></dt> <dd> Set optimal zooming to avoid borders. Accepted values are: <dl class="table"> <dt>‘<samp class="samp">0</samp>’</dt> <dd> disabled </dd> <dt>‘<samp class="samp">1</samp>’</dt> <dd> optimal static zoom value is determined (only very strong movements will lead to visible borders) (default) </dd> <dt>‘<samp class="samp">2</samp>’</dt> <dd> optimal adaptive zoom value is determined (no borders will be visible), see <samp class="option">zoomspeed</samp> </dd> </dl>

Note that the value given at zoom is added to the one calculated here. </dd> <dt><samp class="option">zoomspeed</samp></dt> <dd> Set percent to zoom maximally each frame (enabled when <samp class="option">optzoom</samp> is set to 2). Range is from 0 to 5, default value is 0.25. </dd> <dt><samp class="option">interpol</samp></dt> <dd> Specify type of interpolation. Available values are: <dl class="table"> <dt>‘<samp class="samp">no</samp>’</dt> <dd> no interpolation </dd> <dt>‘<samp class="samp">linear</samp>’</dt> <dd> linear only horizontal </dd> <dt>‘<samp class="samp">bilinear</samp>’</ <dd> linear in both directions (default) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> cubic in both directions (slow) </dd> </dl>

</dd> <dt><samp class="option">tripod</samp></dt> <dd> Enable virtual tripod mode if set to 1, which is equivalent to <code class="code">relative=0:smoothing=0</code>. Default value is 0. Use also <code class="code">tripod</code> option of vidstabdetect. </dd> <dt><samp class="option">debug</samp></dt> <dd> Increase log verbosity if set to 1. Also the detected global motions are written to the temporary file <samp class="file">global_motions.trf</samp>. Default value is 0. </dd> </dl>

 <h4 class="subsection">32.276.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use <code class="command">ffmpeg</code> for a typical stabilization with default values: <pre class="example-preformatted">ffmpeg -i inp.mpeg -vf vidstabtransform,unsharp=5:5:0.8:3:3:0.4 inp_stabilized.mpeg </pre>

Note the use of the unsharp filter which is always recommended. Zoom in a bit more and load transform data from a given file: <pre class="example-preformatted">vidstabtransform=zoom=5:input="mytransforms.trf"

</pre>

Smoothen the video even more: <pre class=”example-preformatted”>vidstabtransform=smoothing=30 </pre>

 <h3 class=”section”>32.277 vflip# TOC</h3>

Flip the input video vertically. For example, to vertically flip a video with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i in.avi -vf "vflip" out.avi </pre>

 <h3 class=”section”>32.278 vfrdet# TOC</h3>

Detect variable frame rate video. This filter tries to detect if the input is variable or constant frame rate. At end it will output number of frames detected as having variable delta pts, and ones with constant delta pts. If there was frames with variable delta, than it will also show min, max and average delta encountered. <h3 class=”section”>32.279 vibrance# TOC</h3>

Boost or alter saturation. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>intensity</samp></dt> <dd> Set strength of boost if positive value or strength of alter if negative value. Default is 0. Allowed range is from -2 to 2. </dd> <dt><samp class=”option”>rbal</samp></dt> <dd> Set the red balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>gbal</samp></dt> <dd> Set the green balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>bbal</samp></dt> <dd> Set the blue balance. Default is 1. Allowed range is from -10 to 10. </dd> <dt><samp class=”option”>rlum</samp></dt> <dd> Set the red luma coefficient. </dd> <dt><samp class=”option”>glum</samp></dt> <dd> Set the green luma coefficient. </dd> <dt><samp class=”option”>blum</samp></dt> <dd> Set the blue luma coefficient. </dd> <dt><samp class=”option”>alternate</samp></dt> <dd> If <code class=”code”>intensity</code> is negative and this is set to 1, colors will change, otherwise colors will be less saturated, more towards gray. </dd> </dl>

 <h4 class="subsection">32.279.1 Commands# TOC</h4>

This filter supports the all above options as commands. <h3 class="section">32.280 vif# TOC</h3>

Obtain the average VIF (Visual Information Fidelity) between two input videos. This filter takes two input videos. Both input videos must have the same resolution and pixel format for this filter to work correctly. Also it assumes that both inputs have the same number of frames, which are compared one by one. The obtained average VIF score is printed through the logging system. The filter stores the calculated VIF score of each frame. This filter also supports the framesync options. In the below example the input file <samp class="file">main.mpg</samp> being processed is compared with the reference file <samp class="file">ref.mpg</samp>.   <pre class="example-preformatted">ffmpeg -i main.mpg -i ref.mpg -lavfi vif -f null - </pre>

 <h3 class="section">32.281 vignette# TOC</h3>

Make or reverse a natural vignetting effect. The filter accepts the following options: <dl class="table"> <dt><samp class="option">angle, a</samp></dt> <dd> Set lens angle expression as a number of radians. The value is clipped in the <code class="code">[0,PI/2]</code> range. Default value: <code class="code">"PI/5"</code> </dd> <dt><samp class="option">x0</samp></dt> <dt><samp class="option">y0</samp></dt> <dd> Set center coordinates expressions. Respectively <code class="code">"w/2"</code> and <code class="code">"h/2"</code> by default. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set forward/backward mode. Available modes are: <dl class="table"> <dt>‘<samp class="samp">forward</samp>’</dt> <dd> The larger the distance from the central point, the darker the image becomes. </dd> <dt>‘<samp class="samp">backward</samp>’</dt> <dd> The larger the distance from the central point, the brighter the image becomes. This can be used to reverse a vignette effect, though there is no automatic detection to extract the lens <samp class="option">angle</samp> and other settings (yet). It can also be used to create a burning effect. </dd> </dl>

Default value is ‘<samp class="samp">forward</samp>’.
 </dd> <dt><samp class="option">eval</samp></dt> <dd> Set
evaluation mode for the expressions (<samp class="option">angle</samp>,
<samp class="option">x0</samp>, <samp class="option">y0</samp>).
 It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">init</samp>’</dt> <dd> Evaluate
expressions only once during the filter initialization. </dd>
<dt>‘<samp class="samp">frame</samp>’</dt> <dd> Evaluate
expressions for each incoming frame. This is way slower than the ‘<samp
class="samp">init</samp>’ mode since it requires all the scalers to
be re-computed, but it allows advanced dynamic expressions. </dd>
</dl>

Default value is ‘<samp class="samp">init</samp>’.
 </dd> <dt><samp class="option">dither</samp></dt> <dd> Set
dithering to reduce the circular banding effects. Default is <code
class="code">1</code> (enabled). </dd> <dt><samp class="option">aspect</samp></dt>
<dd> Set vignette aspect. This setting allows one to adjust the shape of
the vignette. Setting this value to the SAR of the input will make a rectangular
vignetting following the dimensions of the video. Default is <code
class="code">1/1</code>. </dd> </dl>

 <h4 class="subsection">32.281.1 Expressions<a class="anchor hidden-xs" href="#Expressions"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Expressions" aria-hidden="true">TOC</h4>

The <samp class="option">alpha</samp>, <samp class="option">x0</samp>
and <samp class="option">y0</samp> expressions can contain the following
parameters. <dl class="table"> <dt><samp class="option">w</samp></dt>
<dt><samp class="option">h</samp></dt> <dd> input width
and height </dd> <dt><samp class="option">n</samp></dt>
<dd> the number of input frame, starting from 0 </dd>
<dt><samp class="option">pts</samp></dt> <dd> the PTS (Presentation TimeStamp) time of the filtered video frame, expressed in <var
class="var">TB</var> units, NAN if undefined </dd> <dt><samp
class="option">r</samp></dt> <dd> frame rate of the input video,
NAN if the input frame rate is unknown </dd> <dt><samp
class="option">t</samp></dt> <dd> the PTS (Presentation TimeStamp) of the filtered video frame, expressed in seconds, NAN if undefined
</dd> <dt><samp class="option">tb</samp></dt> <dd> time base
of the input video </dd> </dl>

 <h4 class="subsection">32.281.2 Examples<a class="anchor hidden-xs" href="#Examples-
158" aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-
Examples-158" aria-hidden="true">TOC</h4>

<ul class=”itemize mark-bullet”> Apply simple strong vignetting effect: <pre class=”example-preformatted”>vignette=PI/4 </pre>

Make a flickering vignetting: <pre class=”example-preformatted”>vignette='PI/4+random(1)*PI/50':eval=frame </pre>

 <h3 class=”section”>32.282 vmafmotion# TOC</h3>

Obtain the average VMAF motion score of a video. It is one of the component metrics of VMAF. The obtained average motion score is printed through the logging system. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>stats_file</samp></dt> <dd> If specified, the filter will use the named file to save the motion score of each frame with respect to the previous frame. When filename equals "-" the data is sent to standard output. </dd> </dl>

Example: <pre class=”example-preformatted”>ffmpeg -i ref.mpg -vf vmafmotion -f null - </pre>

 <h3 class=”section”>32.283 vpp_amf# TOC</h3>

Scale (resize) and convert colorspace, transfer characteristics or color primaries for the input video, using AMD Advanced Media Framework library for hardware acceleration. Setting the output width and height works in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>scale_type</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear This is the default. </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used.

</dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt>
<dt><samp class="option">force_divisible_by</samp></dt> <dd> Work
the same as the identical scale filter op-
tions. </dd> <dt><samp class="option">reset_sar</samp></dt>
<dd> Works the same as the identical scale
filter option. </dd> <dt><samp class="option">in_color_range</samp></dt>
<dd> Override input color range. </dd> <dt><samp class="option">out_color_range</samp><
<dd> Specify output color range. The accepted values for
in_trc and out_trc are: <dl class="table"> <dt>‘<samp
class="samp">studio</samp>’</dt> <dd> Studio (or re-
stricted, or MPEG) color range. </dd> <dt>‘<samp
class="samp">full</samp>’</dt> <dd> Full (or JPEG) color
range. </dd> </dl>

</dd> <dt><samp
class="option">color_profile</samp></dt> <dd> Specify all color prop-
erties at once. The accepted values are: <dl class="table">
<dt>‘<samp class="samp">bt601</samp>’</dt> <dd> BT.601
 </dd> <dt>‘<samp class="samp">bt709</samp>’</dt>
<dd> BT.709 </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt>
<dd> BT.2020 </dd> </dl>

</dd> <dt><samp class="option">in_trc</samp></dt> <dd> Override
input transfer characteristics. </dd> <dt><samp class="option">out_trc</samp></dt>
<dd> Specify output transfer characteristics. The accepted val-
ues for in_trc and out_trc are: <dl class="table"> <dt>‘<samp
class="samp">bt709</samp>’</dt> <dd> BT.709
</dd> <dt>‘<samp class="samp">gamma22</samp>’</dt>
<dd> Constant gamma of 2.2 </dd> <dt>‘<samp
class="samp">gamma28</samp>’</dt> <dd> Constant gamma
of 2.8 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt>
<dd> SMPTE-170M </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt
<dd> SMPTE-240M </dd> <dt>‘<samp class="samp">linear</samp>’</dt>
<dd> Linear </dd> <dt>‘<samp class="samp">log</samp>’</dt>
<dd> LOG </dd> <dt>‘<samp class="samp">log-
sqrt</samp>’</dt> <dd> LOG_SQRT </dd> <dt>‘<samp
class="samp">iec61966-2-4</samp>’</dt> <dd> iec61966-2-4
 </dd> <dt>‘<samp class="samp">bt1361-ecg</samp>’</dt>
<dd> BT1361_ECG </dd> <dt>‘<samp class="samp">iec61966-
2-1</samp>’</dt> <dd> iec61966-2-1 </dd> <dt>‘<samp
class="samp">bt2020-10</samp>’</dt> <dd> BT.2020 for
10-bits content </dd> <dt>‘<samp class="samp">bt2020-
12</samp>’</dt> <dd> BT.2020 for 12-bits content
</dd> <dt>‘<samp class="samp">smpte2084</samp>’</dt>
<dd> SMPTE2084 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt>
<dd> SMPTE428 </dd> <dt>‘<samp class="samp">arib-
std-b67</samp>’</dt> <dd> ARIB_STD_B67 </dd>

</dl>

</dd> <dt><samp class="option">in_primaries</samp></dt> <dd> Override input color primaries. </dd> <dt><samp class="option">out_primaries</samp></dt> <dd> Specify output color primaries. The accepted values for in_primaries and out_primaries are: <dl class="table"> <dt>‘<samp class="samp">bt709</samp>’</dt> <dd> BT.709 </dd> <dt>‘<samp class="samp">bt470m</samp>’</dt> <dd> BT.470M </dd> <dt>‘<samp class="samp">bt470bg</samp>’</dt> <dd> BT.470BG or BT.601-6 625 </dd> <dt>‘<samp class="samp">smpte170m</samp>’</dt> <dd> SMPTE-170M or BT.601-6 525 </dd> <dt>‘<samp class="samp">smpte240m</samp>’</dt> <dd> SMPTE-240M </dd> <dt>‘<samp class="samp">film</samp>’</dt> <dd> film </dd> <dt>‘<samp class="samp">bt2020</samp>’</dt> <dd> BT.2020 </dd> <dt>‘<samp class="samp">smpte428</samp>’</dt> <dd> SMPTE-428 </dd> <dt>‘<samp class="samp">smpte431</samp>’</dt> <dd> SMPTE-431 </dd> <dt>‘<samp class="samp">smpte432</samp>’</dt> <dd> SMPTE-432 </dd> <dt>‘<samp class="samp">jedec-p22</samp>’</dt> <dd> JEDEC P22 phosphors </dd> </dl> </dd> </dl>

 <h4 class="subsection">32.283.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Scale input to 720p, keeping aspect ratio and ensuring the output is yuv420p. The filter always outputs AMF surfaces, so a hwdownload is needed to feed a software encoder. <pre class="example-preformatted">vpp_amf=-2:720:format=yuv420p,hwdownload,format=yuv420p
</pre>

Upscale to 4K and change color profile to bt2020. <pre class="example-preformatted">vpp_amf=4096:2160:color_profile=bt2020
</pre>

Override input primaries and input transfer characteristics, change both to bt709.

 <pre class="example-preformatted">vpp_amf=color_profile=bt2020:in_trc=smpte2
</pre>

 <h3 class="section">32.284 vstack# TOC</h3> Stack input videos vertically. All streams must be of same pixel format and of same width. Note that this filter is faster

than using overlay and pad filter to create same output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> </dl>

 <h3 class=”section”>32.285 w3fdif# TOC</h3>

Deinterlace the input video ("w3fdif" stands for "Weston 3 Field Deinterlacing Filter"). Based on the process described by Martin Weston for BBC R&D, and implemented based on the de-interlace algorithm written by Jim Easterbrook for BBC R&D, the Weston 3 field deinterlacing filter uses filter coefficients calculated by BBC R&D. This filter uses field-dominance information in frame to decide which of each pair of fields to place first in the output. If it gets it wrong use setfield filter before <code class=”code”>w3fdif</code> filter. There are two sets of filter coefficients, so called "simple" and "complex". Which set of filter coefficients is used can be set by passing an optional parameter: <dl class=”table”> <dt><samp class=”option”>filter</samp></dt> <dd> Set the interlacing filter coefficients. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>simple</samp>’</dt> <dd> Simple filter coefficient set. </dd> <dt>‘<samp class=”samp”>complex</samp>’</dt> <dd> More-complex filter coefficient set. </dd> </dl> Default value is ‘<samp class=”samp”>complex</samp>’. </dd> <dt><samp class=”option”>mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class=”option”>field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class=”code”>field</code>. </dd> <dt><samp class=”option”>parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class=”table”> <dt><samp class=”option”>tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class=”option”>bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class=”option”>auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class=”code”>auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class=”option”>deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class=”table”> <dt>‘<samp class=”samp”>all</samp>’</dt> <dd> Deinterlace all frames, </dd> <dt>‘<samp class=”samp”>interlaced</samp>&rsquo <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

Default value is ‘<samp class=”samp”>all</samp>’. </dd> </dl>

 <h4 class=”subsection”>32.285.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>32.286 waveform# TOC</h3> Video waveform monitor. The waveform monitor plots color component intensity. By default luma only. Each column of the waveform corresponds to a column of pixels in the source video. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode, m</samp></dt> <dd> Can be either <code class=”code”>row</code>, or <code class=”code”>column</code>. Default is <code class=”code”>column</code>. In row mode, the graph on the left side represents color component value 0 and the right side represents value = 255. In column mode, the top side represents color component value = 0 and bottom side represents value = 255. </dd> <dt><samp class=”option”>intensity, i</samp></dt> <dd> Set intensity. Smaller values are useful to find out how many values of the same luminance are distributed across input rows/columns. Default value is <code class=”code”>0.04</code>. Allowed range is [0, 1]. </dd> <dt><samp class=”option”>mirror, r</samp></dt> <dd> Set mirroring mode. <code class=”code”>0</code> means unmirrored, <code class=”code”>1</code> means mirrored. In mirrored mode, higher values will be represented on the left side for <code class=”code”>row</code> mode and at the top for <code class=”code”>column</code> mode. Default is <code class=”code”>1</code> (mirrored). </dd> <dt><samp class=”option”>display, d</samp></dt> <dd> Set display mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>overlay</samp>’</dt> <dd> Presents information identical to that in the <code class=”code”>parade</code>, except that the graphs representing color components are superimposed directly over one another. This display mode makes it easier to spot relative differences or similarities in overlapping areas of the color components that

are supposed to be identical, such as neutral whites, grays, or blacks. </dd> <dt>‘<samp class="samp">stack</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">row</code> mode or one below the other in <code class="code">column</code> mode. </dd> <dt>‘<samp class="samp">parade</samp>’</dt> <dd> Display separate graph for the color components side by side in <code class="code">column</code> mode or one below the other in <code class="code">row</code> mode. Using this display mode makes it easy to spot color casts in the highlights and shadows of an image, by comparing the contours of the top and the bottom graphs of each waveform. Since whites, grays, and blacks are characterized by exactly equal amounts of red, green, and blue, neutral areas of the picture should display three waveforms of roughly equal width/height. If not, the correction is easy to perform by making level adjustments the three waveforms. </dd> </dl> Default is <code class="code">stack</code>. </dd> <dt><samp class="option">components, c</samp></dt> <dd> Set which color components to display. Default is 1, which means only luma or red color component if input is in RGB colorspace. If is set for example to 7 it will display all 3 (if) available color components. </dd> <dt><samp class="option">envelope, e</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No envelope, this is default. </dd> <dt>‘<samp class="samp">instant</samp>’</dt> <dd> Instant envelope, minimum and maximum values presented in graph will be easily visible even with small <code class="code">step</code> value. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Hold minimum and maximum values presented in graph across time. This way you can still spot out of range values without constantly looking at waveforms. </dd> <dt>‘<samp class="samp">peak+instant</samp>’</dt> <dd> Peak and instant envelope combined together. </dd> </dl>

</dd> <dt><samp class="option">filter, f</samp></dt> <dd><dl class="table"> <dt>‘<samp class="samp">lowpass</samp>’</dt> <dd> No filtering, this is default. </dd> <dt>‘<samp class="samp">flat</samp>’</dt> <dd> Luma and chroma combined together. </dd> <dt>‘<samp class="samp">aflat</samp>’</dt> <dd> Similar as above, but shows difference between blue and red chroma. </dd> <dt>‘<samp class="samp">xflat</samp>’</dt> <dd> Similar as above, but use different colors. </dd> <dt>‘<samp class="samp">yflat</samp>’</dt> <dd> Similar as above, but again with different colors. </dd> <dt>‘<samp class="samp">chroma</samp>’</dt> <dd> Displays only chroma. </dd> <dt>‘<samp class="samp">color</samp>’</dt> <dd> Displays actual color value on waveform. </dd> <dt>‘<samp class="samp">acolor</samp>’</dt> <dd> Similar as above, but with luma showing frequency of chroma values. </dd>

</dl>

</dd> <dt><samp class=”option”>graticule, g</samp></dt> <dd> Set which graticule to display. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Do not display graticule. </dd> <dt>‘<samp class=”samp”>green</samp>’</dt> <dd> Display green graticule showing legal broadcast ranges. </dd> <dt>‘<samp class=”samp”>orange</samp>’</dt> <dd> Display orange graticule showing legal broadcast ranges. </dd> <dt>‘<samp class=”samp”>invert</samp>’</dt> <dd> Display invert graticule showing legal broadcast ranges. </dd> </dl>

</dd> <dt><samp class=”option”>opacity, o</samp></dt> <dd> Set graticule opacity. </dd> <dt><samp class=”option”>flags, fl</samp></dt> <dd> Set graticule flags. <dl class=”table”> <dt>‘<samp class=”samp”>numbers</samp>’</dt> <dd> Draw numbers above lines. By default enabled. </dd> <dt>‘<samp class=”samp”>dots</samp>’</dt> <dd> Draw dots instead of lines. </dd> </dl>

</dd> <dt><samp class=”option”>scale, s</samp></dt> <dd> Set scale used for displaying graticule. <dl class=”table”> <dt>‘<samp class=”samp”>digital</samp>’</dt> <dt>‘<samp class=”samp”>millivolts</samp>&rsquo <dt>‘<samp class=”samp”>ire</samp>’</dt> </dl> Default is digital. </dd> <dt><samp class=”option”>bgopacity, b</samp></dt> <dd> Set background opacity. </dd> <dt><samp class=”option”>tint0, t0</samp></dt> <dt><samp class=”option”>tint1, t1</samp></dt> <dd> Set tint for output. Only used with lowpass filter and when display is not overlay and input pixel formats are not RGB. </dd> <dt><samp class=”option”>fitmode, fm</samp></dt> <dd> Set sample aspect ratio of video output frames. Can be used to configure waveform so it is not stretched too much in one of directions. <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Set sample aspect ration to 1/1. </dd> <dt>‘<samp class=”samp”>size</samp>’</dt> <dd> Set sample aspect ratio to match input size of video </dd> </dl> Default is ‘<samp class=”samp”>none</samp>’. </dd> <dt><samp class=”option”>input</samp></dt> <dd> Set input formats for filter to pick from. Can be ‘<samp class=”samp”>all</samp>’, for selecting from all available formats, or ‘<samp class=”samp”>first</samp>’, for selecting first available format. Default is ‘<samp class=”samp”>first</samp>’. </dd> </dl>

 <h3 class=”section”>32.287 weave, doubleweave# <a class=”anchor hidden-xs”href=”#toc-weave_002c-doubleweave” aria-

hidden=”true”>TOC</h3>

The <code class=”code”>weave</code> takes a field-based video input and join each two sequential fields into single frame, producing a new double height clip with half the frame rate and half the frame count. The <code class=”code”>doubleweave</code> works same as <code class=”code”>weave</code> but without halving frame rate and frame count. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>first_field</samp></dt> <dd> Set first field. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>top, t</samp>’</dt> <dd> Set the frame as top-field-first. </dd> <dt>‘<samp class=”samp”>bottom, b</samp>’</dt> <dd> Set the frame as bottom-field-first. </dd> </dl> </dd> </dl>

 <h4 class=”subsection”>32.287.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Interlace video using select and separatefields
filter: <pre class=”example-preformatted”>separatefields,select=eq(mod(n,4),0)+eq(
</pre>

 <h3 class=”section”>32.288 xbr#
TOC</h3>
Apply the xBR high-quality magnification filter which is designed for pixel art. It follows a set of edge-detection rules, see https://forums.libretro.com/t/xbr-algorithm-tutorial/123. It accepts the following option: <dl class=”table”> <dt><samp class=”option”>n</samp></dt>
<dd> Set the scaling dimension: <code class=”code”>2</code> for <code class=”code”>2xBR</code>, <code class=”code”>3</code> for <code class=”code”>3xBR</code> and <code class=”code”>4</code> for <code class=”code”>4xBR</code>. Default is <code class=”code”>3</code>. </dd> </dl>

 <h3 class=”section”>32.289 xcorrelate# TOC</h3> Apply normalized cross-correlation between first and second input video stream. Second input video stream dimensions must be lower than first input video stream. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt>
<dd> Set which planes to process. </dd> <dt><samp

class=”option”>secondary</samp></dt> <dd> Set which secondary video frames will be processed from second input video stream, can be <var class=”var”>first</var> or <var class=”var”>all</var>. Default is <var class=”var”>all</var>. </dd> </dl>

The <code class=”code”>xcorrelate</code> filter also supports the framesync options. <h3 class=”section”>32.290 xfade#
TOC</h3>

Apply cross fade from one input video stream to another input video stream.

The cross fade is applied for specified duration.

Both inputs must be constant frame-rate and have the same resolution, pixel format,

frame rate and timebase.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>transition</samp></dt>

<dd> Set one of available transition effects:

<dl class=”table”>

<dt>‘<samp class=”samp”>custom</samp>’</dt>

<dt>‘<samp class=”samp”>fade</samp>’</dt>

<dt>‘<samp class=”samp”>wipeleft</samp>’</dt>

<dt>‘<samp class=”samp”>wiperight</samp>’</dt>

<dt>‘<samp class=”samp”>wipeup</samp>’</dt>

<dt>‘<samp class=”samp”>wipedown</samp>’</dt>

<dt>‘<samp class=”samp”>slideleft</samp>’</dt>

<dt>‘<samp class=”samp”>slideright</samp>’</dt>

<dt>‘<samp class=”samp”>slideup</samp>’</dt>

<dt>‘<samp class=”samp”>slidedown</samp>’</dt>

<dt>‘<samp class="samp">circlecrop</samp>’</dt>
<dt>‘<samp class="samp">rectcrop</samp>’</dt>
<dt>‘<samp class="samp">distance</samp>’</dt>
<dt>‘<samp class="samp">fadeblack</samp>’</dt>
<dt>‘<samp class="samp">fadewhite</samp>’</dt>
<dt>‘<samp class="samp">radial</samp>’</dt>
<dt>‘<samp class="samp">smoothleft</samp>’</dt>
<dt>‘<samp class="samp">smoothright</samp>’</dt>
<dt>‘<samp class="samp">smoothup</samp>’</dt>
<dt>‘<samp class="samp">smoothdown</samp>’</dt>
<dt>‘<samp class="samp">circleopen</samp>’</dt>
<dt>‘<samp class="samp">circleclose</samp>’</dt>
<dt>‘<samp class="samp">vertopen</samp>’</dt>
<dt>‘<samp class="samp">vertclose</samp>’</dt>
<dt>‘<samp class="samp">horzopen</samp>’</dt>
<dt>‘<samp class="samp">horzclose</samp>’</dt>
<dt>‘<samp class="samp">dissolve</samp>’</dt>
<dt>‘<samp class="samp">pixelize</samp>’</dt>
<dt>‘<samp class="samp">diagtl</samp>’</dt>
<dt>‘<samp class="samp">diagtr</samp>’</dt>
<dt>‘<samp class="samp">diagbl</samp>’</dt>
<dt>‘<samp class="samp">diagbr</samp>’</dt>
<dt>‘<samp class="samp">hlslice</samp>’</dt>
<dt>‘<samp class="samp">hrslice</samp>’</dt>
<dt>‘<samp class="samp">vuslice</samp>’</dt>
<dt>‘<samp class="samp">vdslice</samp>’</dt>
<dt>‘<samp class="samp">hblur</samp>’</dt>
<dt>‘<samp class="samp">fadegrays</samp>’</dt>
<dt>‘<samp class="samp">wipetl</samp>’</dt>
<dt>‘<samp class="samp">wipetr</samp>’</dt>
<dt>‘<samp class="samp">wipebl</samp>’</dt>

<dt>‘<samp class="samp">wipebr</samp>’</dt>
<dt>‘<samp class="samp">squeezeh</samp>’</dt>
<dt>‘<samp class="samp">squeezev</samp>’</dt>
<dt>‘<samp class="samp">zoomin</samp>’</dt>
<dt>‘<samp class="samp">fadefast</samp>’</dt>
<dt>‘<samp class="samp">fadeslow</samp>’</dt>
<dt>‘<samp class="samp">hlwind</samp>’</dt>
<dt>‘<samp class="samp">hrwind</samp>’</dt>
<dt>‘<samp class="samp">vuwind</samp>’</dt>
<dt>‘<samp class="samp">vdwind</samp>’</dt>
<dt>‘<samp class="samp">coverleft</samp>’</dt>
<dt>‘<samp class="samp">coverright</samp>’</dt>
<dt>‘<samp class="samp">coverup</samp>’</dt>
<dt>‘<samp class="samp">coverdown</samp>’</dt>
<dt>‘<samp class="samp">revealleft</samp>’</dt>
<dt>‘<samp class="samp">revealright</samp>’</dt>
<dt>‘<samp class="samp">revealup</samp>’</dt>
<dt>‘<samp class="samp">revealdown</samp>’</dt>
</dl>
Default transition effect is fade.

</dd>
<dt><samp class="option">duration</samp></dt>
<dd> Set cross fade duration in seconds.
Range is 0 to 60 seconds.
Default duration is 1 second.

</dd>
<dt><samp class="option">offset</samp></dt>
<dd> Set cross fade start relative to first input stream in seconds.
Default offset is 0.

</dd>
<dt><samp class="option">expr</samp></dt>
<dd> Set expression for custom transition effect.

The expressions can use the following variables and functions:

<dl class="table">
<dt><samp class="option">X</samp></dt>
<dt><samp class="option">Y</samp></dt>
<dd> The coordinates of the current sample.

</dd>
<dt><samp class="option">W</samp></dt>
<dt><samp class="option">H</samp></dt>
<dd> The width and height of the image.

</dd>
<dt><samp class="option">P</samp></dt>
<dd> Progress of transition effect.

</dd>
<dt><samp class="option">PLANE</samp></dt>
<dd> Currently processed plane.

</dd>
<dt><samp class="option">A</samp></dt>
<dd> Return value of first input at current location and plane.

</dd>
<dt><samp class="option">B</samp></dt>

<dd> Return value of second input at current location and plane.

</dd>

<dt><samp class="option">a0(x, y)</samp></dt>

<dt><samp class="option">a1(x, y)</samp></dt>

<dt><samp class="option">a2(x, y)</samp></dt>

<dt><samp class="option">a3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of first input.

</dd>

<dt><samp class="option">b0(x, y)</samp></dt>

<dt><samp class="option">b1(x, y)</samp></dt>

<dt><samp class="option">b2(x, y)</samp></dt>

<dt><samp class="option">b3(x, y)</samp></dt>

<dd> Return the value of the pixel at location (<var class="var">x</var>,<var class="var">y</var>) of the

first/second/third/fourth component of second input.

</dd>

</dl>

</dd>

</dl>

 <h4 class="subsection">32.290.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Cross fade from one input video to another input video, with fade transition and duration of transition of 2 seconds starting at offset of 5 seconds: <pre class="example-preformatted">ffmpeg -i first.mp4 -i second.mp4 -filter_complex xfade=transition=fade:duration=2:offset=5 output.mp4 </pre>

 <h3 class="section">32.291 xmedian# TOC</h3> Pick median pixels from several input videos. The filter accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> Set number of inputs. Default is 3. Allowed range is from 3 to 255. If number of inputs is even number, than result will be mean value between two median values. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">15</code>, by which all planes are processed. </dd> <dt><samp class="option">percentile</samp></dt> <dd> Set median percentile. Default value is <code class="code">0.5</code>. Default value of <code class="code">0.5</code> will pick always median values, while <code class="code">0</code> will pick minimum values, and <code class="code">1</code> maximum values. </dd> </dl>

 <h4 class="subsection">32.291.1 Commands# TOC</h4>

This filter supports all above options as commands, excluding option <code class="code">inputs</code>. <h3 class="section">32.292 xpsnr# TOC</h3>

Obtain the average (across all input frames) and minimum (across all color plane averages) eXtended Perceptually weighted peak Signal-to-Noise Ratio (XPSNR) between two input videos. The XPSNR is a low-complexity psychovisually motivated distortion measurement algorithm for assessing the difference between two video streams or images. This is especially useful for objectively quantifying the distortions caused by video and image codecs, as an alternative to a formal subjective test. The logarithmic XPSNR output values are in a similar range as those of traditional psnr assessments but better reflect human impressions of visual coding quality. More details on the XPSNR measure, which essentially represents a blockwise weighted variant of the PSNR measure, can be found in the following freely available papers: <ul class="itemize mark-bullet"> C. R. Helmrich, M. Siekmann, S. Becker, S. Bosse, D. Marpe, and T. Wiegand, "XPSNR: A Low-Complexity Extension of the Perceptually Weighted Peak Signal-to-Noise Ratio for High-Resolution Video Quality Assessment," in Proc. IEEE Int. Conf. Acoustics, Speech, Sig. Process. (ICASSP), virt./online, May 2020. www.ecodis.de/xpsnr.htm

C. R. Helmrich, S. Bosse, H. Schwarz, D. Marpe, and T. Wiegand, "A Study of the Extended Perceptually Weighted Peak Signal-to-Noise Ratio (XPSNR) for Video Compression with Different Resolutions and Bit Depths," ITU Journal: ICT Discoveries, vol. 3, no. 1, pp. 65 - 72, May 2020. http://handle.itu.int/11.1002/pub/8153d78b-en

When publishing the results of XPSNR assessments obtained using, e.g., this FFmpeg filter, a reference to the above papers as a means of documentation is strongly encouraged. The filter requires two input videos. The first input is considered a (usually not distorted) reference source and is passed unchanged to the output, whereas the second input is a (distorted) test signal. Except for the bit depth, these two video inputs must have the same pixel format. In addition, for best performance, both compared input videos should be in YCbCr color format. The obtained overall XPSNR values mentioned above are printed through the logging system. In case of input with multiple color planes, we suggest reporting of the minimum XPSNR average. The following parameter, which behaves like the one for the psnr filter, is accepted: <dl class="table"> <dt><samp class="option">stats_file, f</samp></dt> <dd> If specified, the filter will use the named file to save the XPSNR value of each individual frame and color plane. When the file name equals "-", that data is sent to standard output. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">32.292.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> XPSNR analysis of two 1080p HD videos, ref_source.yuv and test_video.yuv, both at 24 frames per second, with color format 4:2:0, bit depth 8, and output of a logfile named "xpsnr.log": <pre class="example-preformatted">ffmpeg -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i ref_source.yuv -s 1920x1080 -framerate 24 -pix_fmt yuv420p -i test_video.yuv -lavfi xpsnr="stats_file=xpsnr.log" -f null - </pre>

XPSNR analysis of two 2160p UHD videos, ref_source.yuv with bit depth 8 and test_video.yuv with bit depth 10, both at 60 frames per second with color format 4:2:0, no logfile output: <pre class="example-preformatted">ffmpeg -s 3840x2160 -framerate 60 -pix_fmt yuv420p -i ref_source.yuv -s 3840x2160 -framerate 60 -pix_fmt yuv420p10le -i test_video.yuv -lavfi xpsnr="stats_file=-" -f null - </pre>

 <h3 class="section">32.293 xstack# <a class="anchor

hidden-xs”href=”#toc-xstack” aria-hidden=”true”>TOC</h3> Stack video inputs into custom layout. All streams must be of same pixel format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> Set number of input streams. Default is 2. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> Specify layout of inputs. This option requires the desired layout configuration to be explicitly set by the user. This sets position of each video input in output. Each input is separated by ’|’. The first number represents the column, and the second number represents the row. Numbers start at 0 and are separated by ’_’. Optionally one can use wX and hX, where X is video input from which to take width or height. Multiple values can be used when separated by ’+’. In such case values are summed together. Note that if inputs are of different sizes gaps may appear, as not all of the output video frame will be filled. Similarly, videos can overlap each other if their position doesn’t leave enough space for the full frame of adjoining videos. For 2 inputs, a default layout of <code class=”code”>0_0|w0_0</code> (equivalent to <code class=”code”>grid=2x1</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class=”code”>grid</code> or <code class=”code”>layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class=”option”>grid</samp></dt> <dd> Specify a fixed size grid of inputs. This option is used to create a fixed size grid of the input streams. Set the grid size in the form <code class=”code”>COLUMNSxROWS</code>. There must be <code class=”code”>ROWS * COLUMNS</code> input streams and they will be arranged as a grid with <code class=”code”>ROWS</code> rows and <code class=”code”>COLUMNS</code> columns. When using this option, each input stream within a row must have the same height and all the rows must have the same width. If <code class=”code”>grid</code> is set, then <code class=”code”>inputs</code> option is ignored and is implicitly set to <code class=”code”>ROWS * COLUMNS</code>. For 2 inputs, a default grid of <code class=”code”>2x1</code> (equivalent to <code class=”code”>layout=0_0|w0_0</code>) is set. In all other cases, a layout or a grid must be set by the user. Either <code class=”code”>grid</code> or <code class=”code”>layout</code> can be specified at a time. Specifying both will result in an error. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> If set to 1, force the output to terminate when the shortest input terminates. Default value is 0. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> If set to valid color, all unused pixels will be filled with that color. By default fill is set to none, so it is disabled. </dd> </dl>

 <h4 class=”subsection”>32.293.1 Examples# <a class=”anchor hidden-xs”href=”#toc-

Examples-163" aria-hidden="true">TOC</h4>

<ul class="itemize mark-bullet"> Display 4 inputs into 2x2 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input3(w0, 0) input2(0, h0) | input4(w0, h0) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|w0_0|w0_h
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 4 inputs into 1x4 grid.

Layout: <pre class="example-preformatted">input1(0,
0) input2(0, h0) input3(0, h0+h1) input4(0, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=4:layout=0_0|0_h0|0_h0+h1|0
</pre>

Note that if inputs are of different widths, unused space will appear.
Display 9 inputs into 3x3 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input4(w0, 0) | input7(w0+w3, 0) input2(0, h0) | input5(w0, h0) | in-
put8(w0+w3, h0) input3(0, h0+h1) | input6(w0, h0+h1) | input9(w0+w3,
h0+h1) </pre>

 <pre class="example-preformatted">xstack=inputs=9:layout=0_0|0_h0|0_h0+h1|w
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.
Display 16 inputs into 4x4 grid.

Layout: <pre class="example-preformatted">input1(0,
0) | input5(w0, 0) | input9 (w0+w4, 0) | input13(w0+w4+w8, 0) input2(0, h0)
| input6(w0, h0) | input10(w0+w4, h0) | input14(w0+w4+w8, h0) input3(0,
h0+h1) | input7(w0, h0+h1) | input11(w0+w4, h0+h1) | input15(w0+w4+w8,
h0+h1) input4(0, h0+h1+h2)| input8(w0, h0+h1+h2)| input12(w0+w4,
h0+h1+h2)| input16(w0+w4+w8, h0+h1+h2) </pre>

 <pre class="example-preformatted">xstack=inputs=16:layout=0_0|0_h0|0_h0+h1|0
w0+w4_h0|w0+w4_h0+h1|w0+w4_h0+h1+h2|w0+w4+w8_0|w0+w4+w8_h0|w0+w4+w8_h0+h1|w0+w4-
</pre>

Note that if inputs are of different sizes, gaps or overlaps may occur.

 <h3
class="section">32.294 yadif<a class="anchor
hidden-xs" href="#yadif" aria-hidden="true"># <a class="anchor
hidden-xs"href="#toc-yadif" aria-hidden="true">TOC</h3>

Deinterlace the input video ("yadif" means "yet another deinterlacing filter"). It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt> <dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">32.295 yaepblur# TOC</h3>

Apply blur filter while preserving edges ("yaepblur" means "yet another edge preserving blur filter"). The algorithm is described in "J. S. Lee, Digital image enhancement and noise filtering by use of local statistics, IEEE Trans. Pattern Anal. Mach. Intell. PAMI-2, 1980." It accepts the following parameters: <dl class="table"> <dt><samp class="option">radius, r</samp></dt>

<dd> Set the window radius. Default value is 3. </dd> <dt><samp class=”option”>planes, p</samp></dt> <dd> Set which planes to filter. Default is only the first plane. </dd> <dt><samp class=”option”>sigma, s</samp></dt> <dd> Set blur strength. Default value is 128. </dd> </dl>

 <h4 class=”subsection”>32.295.1 Commands# TOC</h4> This filter supports same commands as options. <h3 class=”section”>32.296 zoompan# TOC</h3>

Apply Zoom & Pan effect. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>zoom, z</samp></dt> <dd> Set the zoom expression. Range is 1-10. Default is 1. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set the x and y expression. Default is 0. </dd> <dt><samp class=”option”>d</samp></dt> <dd> Set the duration expression in number of frames. This sets for how many number of frames effect will last for single input image. Default is 90. </dd> <dt><samp class=”option”>s</samp></dt> <dd> Set the output image size, default is ’hd720’. </dd> <dt><samp class=”option”>fps</samp></dt> <dd> Set the output frame rate, default is ’25’. </dd> </dl>

Each expression can contain the following constants: <dl class=”table”> <dt><samp class=”option”>in_w, iw</samp></dt> <dd> Input width. </dd> <dt><samp class=”option”>in_h, ih</samp></dt> <dd> Input height. </dd> <dt><samp class=”option”>out_w, ow</samp></dt> <dd> Output width. </dd> <dt><samp class=”option”>out_h, oh</samp></dt> <dd> Output height. </dd> <dt><samp class=”option”>in</samp></dt> <dd> Input frame count. </dd> <dt><samp class=”option”>on</samp></dt> <dd> Output frame count. </dd> <dt><samp class=”option”>in_time, it</samp></dt> <dd> The input timestamp expressed in seconds. It’s NAN if the input timestamp is unknown. </dd> <dt><samp class=”option”>out_time, time, ot</samp></dt> <dd> The output timestamp expressed in seconds. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Last calculated ’x’ and ’y’ position from ’x’ and ’y’ expression for current input frame. </dd> <dt><samp class=”option”>px</samp></dt> <dt><samp class=”option”>py</samp></dt> <dd> ’x’

and ’y’ of last output frame of previous input frame or 0 when there was not yet such frame (first input frame). </dd> <dt><samp class="option">zoom</samp></dt> <dd> Last calculated zoom from ’z’ expression for current input frame. </dd> <dt><samp class="option">pzoom</samp></dt> <dd> Last calculated zoom of last output frame of previous input frame. </dd> <dt><samp class="option">duration</samp></dt> <dd> Number of output frames for current input frame. Calculated from ’d’ expression for each input frame. </dd> <dt><samp class="option">pduration</samp></dt> <dd> number of output frames created for previous input frame </dd> <dt><samp class="option">a</samp></dt> <dd> Rational number: input width / input height </dd> <dt><samp class="option">sar</samp></dt> <dd> sample aspect ratio </dd> <dt><samp class="option">dar</samp></dt> <dd> display aspect ratio </dd> </dl>

 <h4 class="subsection">32.296.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Zoom in up to 1.5x and pan at same time to some spot near center of picture: <pre class="example-preformatted">zoompan=z='min(zoom+0.0015,1.5)':d=700:x='if(gte(zoom,1.5),x,x+1/a)':y=
</pre>

Zoom in up to 1.5x and pan always at center of picture: <pre class="example-preformatted">zoompan=z='min(zoom+0.0015,1.5)':d=700:x='iw/2
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Same as above but without pausing: <pre class="example-preformatted">zoompan=z='min(max(zoom,pzoom)+0.0015,1.5)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

Zoom in 2x into center of picture only for the first second of the input video: <pre class="example-preformatted">zoompan=z='if(between(in_time,0,1),2,1)':d=1:x='iw/2-
(iw/zoom/2)':y='ih/2-(ih/zoom/2)' </pre>

 <h3 class="section">32.297 zscale# TOC</h3> Scale (resize) the input video, using the z.lib library: https://github.com/sekrit-twc/zimg. To enable compilation of this filter, you need to configure FFmpeg with <code class="code">--enable-libzimg</code>. The

zscale filter forces the output display aspect ratio to be the same as the input, by changing the output sample aspect ratio. If the input image format is different from the format requested by the next filter, the zscale filter will convert the input to the requested format. <h4 class="subsection">32.297.1 Options# TOC</h4> The filter accepts the following options. <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. If the <var class="var">width</var> or <var class="var">w</var> value is 0, the input width is used for the output. If the <var class="var">height</var> or <var class="var">h</var> value is 0, the input height is used for the output. If one and only one of the values is -n with n >= 1, the zscale filter will use a value that maintains the aspect ratio of the input image, calculated from the other specified dimension. After that it will, however, make sure that the calculated dimension is divisible by n and adjust the value if necessary. If both values are -n with n >= 1, the behavior will be identical to both values being set to 0 as previously detailed. See below for the list of accepted constants for use in the dimension expression. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the video size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">dither, d</samp></dt> <dd> Set the dither type. Possible values are: <dl class="table"> <dt><var class="var">none</var></dt> <dt><var class="var">ordered</var></dt> <dt><var class="var">random</var></dt> <dt><var class="var">error_diffusion</var></dt> </dl>

Default is none. </dd> <dt><samp class="option">filter, f</samp></dt> <dd> Set the resize filter type. Possible values are: <dl class="table"> <dt><var class="var">point</var></dt> <dt><var class="var">bilinear</var></dt> <dt><var class="var">bicubic</var></dt> <dt><var class="var">spline16</var></dt> <dt><var class="var">spline36</var></dt> <dt><var class="var">spline64</var></dt> <dt><var class="var">lanczos</var></dt> </dl>

Default is bilinear. </dd> <dt><samp class="option">range, r</samp></dt> <dd> Set the color range. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">limited</var></dt> <dt><var class="var">full</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the color primaries. Possible val-

ues are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt>
<dt><var class="var">170m</var></dt> <dt><var class="var">240m</var></dt>
<dt><var class="var">2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transfer,
t</samp></dt> <dd> Set the transfer characteristics.
Possible values are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt>
<dt><var class="var">601</var></dt> <dt><var class="var">linear</var></dt>
<dt><var class="var">2020_10</var></dt> <dt><var class="var">2020_12</var></dt>
<dt><var class="var">smpte2084</var></dt> <dt><var class="var">iec61966-
2-1</var></dt> <dt><var class="var">arib-std-b67</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrix,
m</samp></dt> <dd> Set the colorspace matrix. Possible
value are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt>
<dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt>
<dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt>
</dl>

Default is same as input. </dd> <dt><samp class="option">rangein,
rin</samp></dt> <dd> Set the input color range. Possible
values are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">limited</var></dt> <dt><var class="var">full</var></dt>
</dl>

Default is same as input. </dd> <dt><samp class="option">primariesin,
pin</samp></dt> <dd> Set the input color primaries.
Possible values are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt>
<dt><var class="var">170m</var></dt> <dt><var class="var">240m</var></dt>
<dt><var class="var">2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">transferin,
tin</samp></dt> <dd> Set the input transfer characteristics.
 Possible values are: <dl class="table"> <dt><var
class="var">input</var></dt> <dt><var class="var">709</var></dt>
<dt><var class="var">unspecified</var></dt> <dt><var class="var">601</var></dt>
<dt><var class="var">linear</var></dt> <dt><var class="var">2020_10</var></dt>
<dt><var class="var">2020_12</var></dt> </dl>

Default is same as input. </dd> <dt><samp class="option">matrixin,
min</samp></dt> <dd> Set the input colorspace matrix.
Possible value are: <dl class="table"> <dt><var class="var">input</var></dt>
<dt><var class="var">709</var></dt> <dt><var class="var">unspecified</var></dt>
<dt><var class="var">470bg</var></dt> <dt><var class="var">170m</var></dt>

<dt><var class="var">2020_ncl</var></dt> <dt><var class="var">2020_cl</var></dt>
</dl>

</dd> <dt><samp class="option">chromal, c</samp></dt> <dd> Set the output chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt> <dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt> <dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt>
</dl>

</dd> <dt><samp class="option">chromalin, cin</samp></dt> <dd> Set the input chroma location. Possible values are: <dl class="table"> <dt><var class="var">input</var></dt> <dt><var class="var">left</var></dt> <dt><var class="var">center</var></dt> <dt><var class="var">topleft</var></dt> <dt><var class="var">top</var></dt> <dt><var class="var">bottomleft</var></dt> <dt><var class="var">bottom</var></dt>
</dl>

</dd> <dt><samp class="option">npl</samp></dt> <dd> Set the nominal peak luminance. </dd> <dt><samp class="option">param_a</samp></dt> <dd> Parameter A for scaling filters. Parameter "b" for bicubic, and the number of filter taps for lanczos. </dd> <dt><samp class="option">param_b</samp></dt> <dd> Parameter B for scaling filters. Parameter "c" for bicubic. </dd> </dl>

The values of the <samp class="option">w</samp> and <samp class="option">h</samp> options are expressions containing the following constants: <dl class="table"> <dt><var class="var">in_w</var></dt> <dt><var class="var">in_h</var></dt> <dd> The input width and height </dd> <dt><var class="var">iw</var></dt> <dt><var class="var">ih</var></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><var class="var">out_w</var></dt> <dt><var class="var">out_h</var></dt> <dd> The output (scaled) width and height </dd> <dt><var class="var">ow</var></dt> <dt><var class="var">oh</var></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var> </dd> <dt><var class="var">a</var></dt> <dd> The same as <var class="var">iw</var> / <var class="var">ih</var> </dd> <dt><var class="var">sar</var></dt> <dd> input sample aspect ratio </dd> <dt><var class="var">dar</var></dt> <dd> The input display aspect ratio. Calculated from <code class="code">(iw / ih) * sar</code>. </dd> <dt><var class="var">hsub</var></dt> <dt><var class="var">vsub</var></dt> <dd> horizontal and vertical input chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> <dt><var class="var">ohsub</var></dt> <dt><var class="var">ovsub</var></dt> <dd> horizontal and ver-

tical output chroma subsample values. For example for the pixel format "yuv422p" <var class="var">hsub</var> is 2 and <var class="var">vsub</var> is 1. </dd> </dl>

 <h4 class="subsection">32.297.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Set the output video dimension expression. The command accepts the same syntax of the corresponding option. If the specified expression is not valid, it is kept at its current value. </dd> </dl>

 <h2 class="chapter">33 CUDA Video Filters# TOC</h2>

Below is a description of the currently available Nvidia CUDA video filters. Prerequisites: <ul class="itemize mark-bullet"> Install Nvidia CUDA Toolkit

Note: If FFmpeg detects the Nvidia CUDA Toolkit during configuration, it will enable CUDA filters automatically without requiring any additional flags. If you want to explicitly enable them, use the following options: <ul class="itemize mark-bullet"> Configure FFmpeg with <code class="code">--enable-cuda-nvcc --enable-nonfree</code>. Configure FFmpeg with <code class="code">--enable-cuda-llvm</code>. Additional requirement: <code class="code">llvm</code> lib must be installed.

Running CUDA filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device cuda[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</samp
<dd> Initialise a new hardware device of type <var class="var">cuda</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of initializing second CUDA device on the system and running scale_cuda

and bilateral_cuda filters. <pre class=”example-preformatted”>./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -init_hw_device cuda:1 -filter_complex \ "[0:v]scale_cuda=format=yuv444p[scaled_video];[sca
\ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

Since CUDA filters operate exclusively on GPU memory, frame data must sometimes be uploaded (hwupload) to hardware surfaces associated with the appropriate CUDA device before processing, and downloaded (hwdownload) back to normal memory afterward, if required. Whether hwupload or hwdownload is necessary depends on the specific workflow: <ul class=”itemize mark-bullet”> If the input frames are already in GPU memory (e.g., when using <code class=”code”>-hwaccel cuda</code> or <code class=”code”>-hwaccel_output_format cuda</code>), explicit use of hwupload is not needed, as the data is already in the appropriate memory space. If the input frames are in CPU memory (e.g., software-decoded frames or frames processed by CPU-based filters), it is necessary to use hwupload to transfer the data to GPU memory for CUDA processing. If the output of the CUDA filters needs to be further processed by software-based filters or saved in a format not supported by GPU-based encoders, hwdownload is required to transfer the data back to CPU memory. Note that hwupload uploads data to a surface with the same layout as the software frame, so it may be necessary to add a format filter immediately before hwupload to ensure the input is in the correct format. Similarly, hwdownload may not support all output formats, so an additional format filter may need to be inserted immediately after hwdownload in the filter graph to ensure compatibility. <h3 class=”section”>33.1 bilateral_cuda#
TOC</h3>
CUDA accelerated bilateral filter, an edge preserving filter. This filter is mathematically accurate thanks to the use of GPU acceleration. For best output quality, use one to one chroma subsampling, i.e. yuv444p format. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigmaS</samp></dt> <dd> Set sigma of gaussian function to calculate spatial weight, also called sigma space. Allowed range is 0.1 to 512. Default is 0.1. </dd> <dt><samp class=”option”>sigmaR</samp></dt> <dd> Set sigma of gaussian function to calculate color range weight, also called sigma color. Allowed range is 0.1 to 512. Default is 0.1. </dd>

<dt><samp class="option">window_size</samp></dt> <dd> Set window size of the bilateral function to determine the number of neighbours to loop on. If the number entered is even, one will be added automatically. Allowed range is 1 to 255. Default is 1. </dd> </dl> <h4 class="subsection">33.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply the bilateral filter on a video.

 <pre class="example-preformatted">./ffmpeg -v verbose \ -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]scale_cuda=format=yuv444p[scaled_video]; [scaled_video]bilateral_cuda=window_size=9:sigmaS=3.0:sigmaR=50.0" \ -an -sn -c:v h264_nvenc -cq 20 out.mp4 </pre>

 <h3 class="section">33.2 bwdif_cuda# TOC</h3>

Deinterlace the input video using the bwdif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the fol-

lowing values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h3 class="section">33.3 chromakey_cuda# TOC</h3> CUDA accelerated YUV colorspace color/chroma keying. This filter works like normal chromakey filter but operates on CUDA frames. for more details and parameters see chromakey. <h4 class="subsection">33.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make all the green pixels in the input video transparent and use it as an overlay for another video:

 <pre class="example-preformatted">./ffmpeg \ -hwaccel cuda -hwaccel_output_format cuda -i input_green.mp4 \ -hwaccel cuda -hwaccel_output_format cuda -i base_video.mp4 \ -init_hw_device cuda \ -filter_complex \ " \ [0:v]chromakey_cuda=0x25302D:0.1:0.12:1[overlay_video]; \ [1:v]scale_cuda=format=yuv420p[base]; \ [base][overlay_video]overlay_cuda" \ -an -sn -c:v h264_nvenc -cq 20 output.mp4 </pre>

Process two software sources, explicitly uploading the frames:

 <pre class="example-preformatted">./ffmpeg -init_hw_device cuda=cuda -filter_hw_device cuda \ -f lavfi -i color=size=800x600:color=white,format=yuv42 \ -f lavfi -i yuvtestsrc=size=200x200,format=yuv420p \ -filter_complex \ " \ [0]hwupload[under]; \ [1]hwupload,chromakey_cuda=green:0.1:0.12[over]; \ [under][over]overlay_cuda" \ -c:v hevc_nvenc -cq 18 -preset slow output.mp4 </pre>

 <h3 class="section">33.4 colorspace_cuda# TOC</h3>

CUDA accelerated implementation of the colorspace filter. It is by no means feature complete compared to the software colorspace filter, and at the current time only supports color range conversion between jpeg/full and mpeg/limited range. The filter

accepts the following options: <dl class=”table”> <dt><samp class=”option”>range</samp></dt> <dd> Specify output color range. The accepted values are: <dl class=”table”> <dt>‘<samp class=”samp”>tv</samp>’</dt> <dd> TV (restricted) range </dd> <dt>‘<samp class=”samp”>mpeg</samp>’</dt> <dd> MPEG (restricted) range </dd> <dt>‘<samp class=”samp”>pc</samp>’</dt> <dd> PC (full) range </dd> <dt>‘<samp class=”samp”>jpeg</samp>’</dt> <dd> JPEG (full) range </dd> </dl>

</dd> </dl>

 <h3 class=”section”>33.5 overlay_cuda# TOC</h3>

Overlay one video on top of another. This is the CUDA variant of the overlay filter. It only accepts CUDA frames. The underlying input pixel formats have to match. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. They can contain the following parameters: <dl class=”table”> <dt><samp class=”option”>main_w, W</samp></dt> <dt><samp class=”option”>main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class=”option”>overlay_w, w</samp></dt> <dt><samp class=”option”>overlay_h, h</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp class=”option”>y</samp></dt> <dd> The computed values for <var class=”var”>x</var> and <var class=”var”>y</var>. They are evaluated for each new frame. </dd> <dt><samp class=”option”>n</samp></dt> <dd> The ordinal index of the main input frame, starting from 0. </dd> <dt><samp class=”option”>pos</samp></dt> <dd> The byte offset position in the file of the main input frame, NAN if unknown. Deprecated, do not use. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The timestamp of the main input frame, expressed in seconds, NAN if unknown. </dd> </dl>

Default value is "0" for both expressions. </dd> <dt><samp class=”option”>eval</samp></dt> <dd> Set when the expressions for <samp class=”option”>x</samp> and <samp class=”option”>y</samp> are evaluated. It accepts the following values: <dl

class=”table”> <dt><samp class=”option”>init</samp></dt> <dd> Evaluate expressions once during filter initialization or when a command is processed. </dd> <dt><samp class=”option”>frame</samp></dt> <dd> Evaluate expressions for each incoming frame </dd> </dl>

Default value is <samp class=”option”>frame</samp>. </dd> <dt><samp class=”option”>eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class=”option”>repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options.

 <h3 class=”section”>33.6 pad_cuda# TOC</h3> Add paddings to an input video stream using CUDA. This filter is the CUDA-accelerated version of the pad filter. It accepts the same options and expressions and provides the same core functionality. For a detailed description of available options, please see the documentation for the pad filter. <h4 class=”subsection”>33.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Add a 200-pixel black border to all sides of a video frame: <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=iw+400:h=ih+400:x=200:y=200" -c:v h264_nvenc out.mp4 </pre>

Pad the input video to a 16:9 aspect ratio, filling with the color "blue": <pre class=”example-preformatted”>ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i input.mp4 -vf "pad_cuda=w=ih*16/9/sar:h=ih:x=(ow-iw)/2:y=(oh-ih)/2:color=blue" -c:v h264_nvenc out.mp4 </pre>

 <h3 class=”section”>33.7 scale_cuda# TOC</h3>

Scale (resize) and convert (pixel format) the input video, using accelerated CUDA kernels. Setting the output width and height works

in the same way as for the scale filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. Allows for the same expressions as the scale filter. </dd> <dt><samp class=”option”>interp_algo</samp></dt> <dd> Sets the algorithm used for scaling: <dl class=”table”> <dt><var class=”var”>nearest</var></dt> <dd> Nearest neighbour Used by default if input parameters match the desired output. </dd> <dt><var class=”var”>bilinear</var></dt> <dd> Bilinear </dd> <dt><var class=”var”>bicubic</var></dt> <dd> Bicubic This is the default. </dd> <dt><var class=”var”>lanczos</var></dt> <dd> High-quality separable Lanczos (windowed sinc). The filter radius can be set with <samp class=”option”>param</samp>; the default is 3. </dd> </dl>

</dd> <dt><samp class=”option”>format</samp></dt> <dd> Controls the output pixel format. By default, or if none is specified, the input pixel format is used. Converting between <code class=”code”>yuv420p</code>, <code class=”code”>yuv422p</code>, <code class=”code”>yuv444p</code>, <code class=”code”>yuv420p10</code>, <code class=”code”>yuv422p10</code>, <code class=”code”>yuv444p10</code>, <code class=”code”>nv12</code>, <code class=”code”>nv16</code>, <code class=”code”>p010</code> or <code class=”code”>p210</code> and <code class=”code”>bgra</code>, <code class=”code”>rgba</code>, <code class=”code”>bgr0</code> or <code class=”code”>rgb0</code> is supported, also together with resizing. Resizing is then done on the RGB side, after converting from YUV or before converting to YUV, and always uses the generic filters selected with <samp class=”option”>interp_algo</samp>. Resizing the YUV side in a separate instance instead needs less intermediate memory and may be faster, but filters chroma differently. YUV to RGB conversion uses the YCbCr matrix and range of the input, RGB to YUV conversion those selected for the output (see <samp class=”option”>out_color_matrix</samp> and <samp class=”option”>out_range</samp>). An unspecified matrix is treated as BT.601 and an unspecified range as limited. The conversion samples chroma independently of <samp class=”option”>interp_algo</samp>: YUV to RGB conversion uses the nearest chroma sample, RGB to YUV conversion averages, which centers the chroma samples. </dd> <dt><samp class=”option”>passthrough</samp></dt> <dd> If set to 0, every frame is processed, even if no conversion is necessary. This mode can be useful to use the filter as a buffer for a downstream frame-consumer that exhausts the limited decoder frame pool. If set to 1, frames are passed through as-is if they match the desired output parameters. This is the default behaviour. </dd> <dt><samp class=”option”>use_filters</samp></dt> <dd> If set to 1, filter with a generic weight LUT instead of using

fixed-function shader kernels. May be faster or slower depending on the hardware. A value of <code class="code">auto</code> (the default) enables this automatically when required for correct anti-aliasing when downscaling, and for high-quality Lanczos scaling. Set this to 0 to use the legacy fixed 4-tap Lanczos kernel. Resizing during YUV to RGB or RGB to YUV conversion always uses the generic filters. </dd> <dt><samp class="option">param</samp></dt> <dd> Algorithm-specific parameter. For <var class="var">bicubic</var>, this affects the filter curve. For <var class="var">lanczos</var>, this sets the filter radius from 1 to 10 (default 3) when using the generic filter path; it has no effect with <samp class="option">use_filters=0</samp>. </dd> <dt><samp class="option">force_original_aspect_ratio</samp></dt> <dt><samp class="option">force_divisible_by</samp></dt> <dd> Work the same as the identical scale filter options. </dd> <dt><samp class="option">reset_sar</samp></dt> <dd> Works the same as the identical scale filter option. </dd> <dt><samp class="option">out_range</samp></dt> <dd> Set the output YCbCr sample range. One of <code class="code">auto</code> (the default), <code class="code">jpeg</code>/<code class="code">full</code>/<code class="code">pc</code>, or <code class="code">mpeg</code>/<code class="code">limited</code>/<code class="code">tv</code>. </dd> <dt><samp class="option">out_color_matrix</samp></dt> <dd> Set the output YCbCr matrix. One of <code class="code">auto</code> (the default), <code class="code">bt601</code>, <code class="code">smpte170m</code>, <code class="code">bt470</code>, <code class="code">bt470bg</code>, <code class="code">bt709</code>, <code class="code">fcc</code>, <code class="code">smpte240m</code>, <code class="code">ycgco</code>, <code class="code">bt2020</code> or <code class="code">bt2020nc</code>. With <code class="code">auto</code>, <samp class="option">out_range</samp> and <samp class="option">out_color_matrix</samp> are left to filter-graph negotiation. They follow a single range and matrix negotiated for the input, as for decoded video, and otherwise prefer BT.601 limited range. For RGB to YUV conversion, <code class="code">auto</code> thus means BT.601 limited range, unless the filter graph requires otherwise; converting full range YUV to RGB and back keeps the full range. Outside RGB to YUV conversion, these options describe the existing samples without remapping them; an unspecified input description may adopt the negotiated output description, while conflicting specified descriptions are rejected. Both options can only be used with YUV output; RGB output is full range. </dd> </dl>

 <h4 class="subsection">33.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Scale input to 720p, keeping as-

pect ratio and ensuring the output is yuv420p. <pre class=”example-preformatted”>scale_cuda=-2:720:format=yuv420p </pre>

Upscale to 4K using nearest neighbour algorithm. <pre class=”example-preformatted”>scale_cuda=4096:2160:interp_algo=nearest </pre>

Don’t do any conversion or scaling, but copy all input frames into newly allocated ones. This can be useful to deal with a filter and encode chain that otherwise exhausts the decoders frame pool. <pre class=”example-preformatted”>scale_cuda=passthrough=0 </pre>

Convert BT.709 input to RGB for a filter that requires it, then back to BT.709 NV12 with limited range. <pre class=”example-preformatted”>scale_cuda=format=bgra,scale_cuda=format=nv12:out_color_matrix=bt70 </pre>

Scale to 720p while converting to RGB. <pre class=”example-preformatted”>scale_cuda=-2:720:format=bgra </pre>

 <h3 class=”section”>33.8 thumbnail_cuda# TOC</h3>

Select the most representative frame in a given sequence of consecutive frames using CUDA. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> Set the frames batch size to analyze; in a set of <var class=”var”>n</var> frames, the filter will pick one of them, and then handle the next batch of <var class=”var”>n</var> frames until the end. Default is <code class=”code”>100</code>. </dd> </dl>

Since the filter keeps track of the whole frames sequence, a bigger <var class=”var”>n</var> value will result in a higher memory usage, so a high value is not recommended. <h4 class=”subsection”>33.8.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Thumbnails are extracted from every <var class=”var”>n</var>=150-frame batch, selecting one per batch. Chosen frames are then scaled with scale_cuda. <pre class=”example-preformatted”>./ffmpeg -hwaccel cuda -hwaccel_output_format cuda -i ./input.mp4 -vf "thumbnail_cuda=150,scale_cuda=1920:1 ./output/out%03d.png </pre>

 <h3 class="section">33.9 transpose_cuda# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class="samp">reversal</samp>’</dt> <dd> Rotate by 180 degrees. </dd> <dt>‘<samp class="samp">hflip</samp>’</dt> <dd> Flip horizontally. </dd> <dt>‘<samp class="samp">vflip</samp>’</dt> <dd> Flip vertically. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h3 class="section">33.10 yadif_cuda# TOC</h3>

Deinterlace the input video using the yadif algorithm, but implemented in CUDA so that it can work as part of a GPU accelerated pipeline with nvdec and/or nvenc. It accepts the following parameters:

<dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the follow-

ing values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> <dt><samp class="option">2, send_frame_nospatial</samp></dt> <dd> Like <code class="code">send_frame</code>, but it skips the spatial interlacing check. </dd> <dt><samp class="option">3, send_field_nospatial</samp></dt> <dd> Like <code class="code">send_field</code>, but it skips the spatial interlacing check. </dd> </dl>

The default value is <code class="code">send_frame</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd> </dl>

 <h2 class="chapter">34 OpenCL Video Filters# TOC</h2>

Below is a description of the currently available OpenCL video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-opencl</code>. Running OpenCL filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device opencl[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam
<dd> Initialise a new hardware device of type <var class="var">opencl</var> called <var class="var">name</var>, using the given device parameters. </dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware

device called <var class="var">name</var> to all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device on the second platform and running avgblur_opencl filter with default parameters on it. <pre class="example-preformatted">-init_hw_device opencl=gpu:1.0 -filter_hw_device gpu -i INPUT -vf "hwupload, avgblur_opencl, hwdownload" OUTPUT </pre>

Since OpenCL filters are not able to access frame data in normal memory, all frame data needs to be uploaded(hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded(hwdownload) back to normal memory. Note that hwupload will upload to a surface with the same layout as the software frame, so it may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it may be necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class="section">34.1 avgblur_opencl# TOC</h3>

Apply average blur filter. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class="code">[1, 1024]</code> and default value is <code class="code">1</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">0xf</code>, by which all planes are processed. </dd> <dt><samp class="option">sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class="code">[1, 1024]</code> and default value is <code class="code">0</code>. If zero, <code class="code">sizeX</code> value will be used. </dd> </dl>

 <h4 class="subsection">34.1.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Apply average blur filter with horizontal and vertical size of 3, setting each pixel of the output to the average value

of the 7x7 region centered on it in the input. For pixels on the edges of the image, the region does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, avgblur_opencl=3, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.2 boxblur_opencl# TOC</h3>

Apply a boxblur algorithm to the input video. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> </dl>

A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>luma_radius, lr</samp></dt> <dt><samp class=”option”>chroma_radius, cr</samp></dt> <dt><samp class=”option”>alpha_radius, ar</samp></dt> <dd> Set an expression for the box radius in pixels used for blurring the corresponding input plane. The radius value must be a non-negative number, and must not be greater than the value of the expression <code class=”code”>min(w,h)/2</code> for the luma and alpha planes, and of <code class=”code”>min(cw,ch)/2</code> for the chroma planes. Default value for <samp class=”option”>luma_radius</samp> is "2". If not specified, <samp class=”option”>chroma_radius</samp> and <samp class=”option”>alpha_radius</samp> default to the corresponding value set for <samp class=”option”>luma_radius</samp>. The expressions can contain the following constants: <dl class=”table”> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> The input width and height in pixels. </dd> <dt><samp class=”option”>cw</samp></dt> <dt><samp class=”option”>ch</samp></dt> <dd> The input chroma image width and height in pixels. </dd> <dt><samp class=”option”>hsub</samp></dt> <dt><samp class=”option”>vsub</samp></dt> <dd> The horizontal and vertical chroma subsample values. For example, for the pixel format "yuv422p", <var class=”var”>hsub</var> is 2 and <var class=”var”>vsub</var> is 1. </dd> </dl>

</dd> <dt><samp class=”option”>luma_power, lp</samp></dt> <dt><samp class=”option”>chroma_power, cp</samp></dt> <dt><samp class=”option”>alpha_power, ap</samp></dt> <dd> Specify how many times the boxblur filter is applied to the corresponding plane. Default value for <samp class=”option”>luma_power</samp> is 2. If not specified, <samp class=”option”>chroma_power</samp> and <samp

class=”option”>alpha_power</samp> default to the corresponding value set for <samp class=”option”>luma_power</samp>. A value of 0 will disable the effect. </dd> </dl>

 <h4 class=”subsection”>34.2.1 Examples# TOC</h4>

Apply boxblur filter, setting each pixel of the output to the average value of box-radiuses <var class=”var”>luma_radius</var>, <var class=”var”>chroma_radius</var>, <var class=”var”>alpha_radius</var> for each plane respectively. The filter will apply <var class=”var”>luma_power</var>, <var class=”var”>chroma_power</var>, <var class=”var”>alpha_power</var> times onto the corresponding plane. For pixels on the edges of the image, the radius does not extend beyond the image boundaries, and so out-of-range coordinates are not used in the calculations. <ul class=”itemize mark-bullet”> Apply a boxblur filter with the luma, chroma, and alpha radius set to 2 and luma, chroma, and alpha power set to 3. The filter will run 3 times with box-radius set to 2 for every plane of the image. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=luma_radius=2:luma_power=3, hwdownload" OUTPUT -i INPUT -vf "hwupload, boxblur_opencl=2:3, hwdownload" OUTPUT </pre>

Apply a boxblur filter with luma radius set to 2, luma_power to 1, chroma_radius to 4, chroma_power to 5, alpha_radius to 3 and alpha_power to 7.

For the luma plane, a 2x2 box radius will be run once. For the chroma plane, a 4x4 box radius will be run 5 times. For the alpha plane, a 3x3 box radius will be run 7 times. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, boxblur_opencl=2:1:4:5:3:7, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.3 colorkey_opencl# TOC</h3>
RGB colorspace color keying. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>color</samp></dt>
<dd> The color which will be replaced with transparency. </dd>
<dt><samp class=”option”>similarity</samp></dt> <dd> Similarity percentage with the key color. 0.01 matches only the exact key color, while 1.0 matches everything. </dd> <dt><samp class=”option”>blend</samp></dt> <dd> Blend percentage.
0.0 makes pixels either fully transparent, or not transparent at all.

Higher values result in semi-transparent pixels, with a higher transparency the more similar the pixels color is to the key color. </dd> </dl>

 <h4 class="subsection">34.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Make every semi-green pixel in the input transparent with some slight blending: <pre class="example-preformatted">-i INPUT -vf "hwupload, colorkey_opencl=green:0.3:0.1, hwdownload" OUTPUT </pre>

 <h3 class="section">34.4 convolution_opencl# TOC</h3

Apply convolution of 3x3, 5x5, 7x7 matrix. The filter accepts the following options: <dl class="table"> <dt><samp class="option">0m</samp></dt> <dt><samp class="option">1m</samp></dt> <dt><samp class="option">2m</samp></dt> <dt><samp class="option">3m</samp></dt> <dd> Set matrix for each plane. Matrix is sequence of 9, 25 or 49 signed numbers. Default value for each plane is <code class="code">0 0 0 0 1 0 0 0 0</code>. </dd> <dt><samp class="option">0rdiv</samp></dt> <dt><samp class="option">1rdiv</samp></dt> <dt><samp class="option">2rdiv</samp></dt> <dt><samp class="option">3rdiv</samp></dt> <dd> Set multiplier for calculated value for each plane. If unset or 0, it will be sum of all matrix elements. The option value must be a float number greater or equal to <code class="code">0.0</code>. Default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">0bias</samp></dt> <dt><samp class="option">1bias</samp></dt> <dt><samp class="option">2bias</samp></dt> <dt><samp class="option">3bias</samp></dt> <dd> Set bias for each plane. This value is added to the result of the multiplication. Useful for making the overall image brighter or darker. The option value must be a float number greater or equal to <code class="code">0.0</code>. Default value is <code class="code">0.0</code>. </dd> </dl>

 <h4 class="subsection">34.4.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Apply sharpen: <pre class="example-preformatted">-i INPUT -vf "hwupload, convolution_opencl=0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0:0 -1 0 -1 5 -1 0 -1 0, hwdownload" OUTPUT </pre>

Apply blur: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1 1 1 1 1 1 1 1 1:1/9:1/9:1/9:1/9, hwdownload" OUTPUT </pre>

Apply edge enhance: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:0 0 0 -1 1 0 0 0 0:5:1:1:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply edge detect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:0 1 0 1 -4 1 0 1 0:5:5:5:1:0:128:128:128, hwdownload" OUTPUT </pre>

Apply laplacian edge detector which includes diagonals: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:1 1 1 1 -8 1 1 1 1:5:5:5:1:0:128:128:0, hwdownload" OUTPUT </pre>

Apply emboss: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, convolution_opencl=-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2:-2 -1 0 -1 1 1 0 1 2, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.5 erosion_opencl# TOC</h3>

Apply erosion effect to the video. This filter replaces the pixel by the local(3x3) minimum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class=”code”>[0, 65535]</code> and default value is <code class=”code”>65535</code>. If <code class=”code”>0</code>, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class=”code”>[0, 255]</code> and default value is <code class=”code”>255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class=”code”>x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>34.5.1 Example# <a class=”anchor hidden-xs”href=”#toc-Example-2”

aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Apply erosion filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local minimum between pixels: 1, 2, 3, 6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local minimum is more then threshold of the corresponding plane, output pixel will be set to input pixel - threshold of corresponding plane. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, erosion_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.6 deshake_opencl# TOC</h3> Feature-point based video stabilization filter. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>tripod</samp></dt> <dd> Simulates a tripod by preventing any camera movement whatsoever from the original frame. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>debug</samp></dt> <dd> Whether or not additional debug info should be displayed, both in the processed output and in the console. Note that in order to see console debug output you will also need to pass <code class=”code”>-v verbose</code> to ffmpeg. Viewing point matches in the output video is only supported for RGB input. Defaults to <code class=”code”>0</code>. </dd> <dt><samp class=”option”>adaptive_crop</samp></dt> <dd> Whether or not to do a tiny bit of cropping at the borders to cut down on the amount of mirrored pixels. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>refine_features</samp></dt> <dd> Whether or not feature points should be refined at a sub-pixel level. This can be turned off for a slight performance gain at the cost of precision. Defaults to <code class=”code”>1</code>. </dd> <dt><samp class=”option”>smooth_strength</samp></dt> <dd> The strength of the smoothing applied to the camera path from <code class=”code”>0.0</code> to <code class=”code”>1.0</code>. <code class=”code”>1.0</code> is the maximum smoothing strength while values less than that result in less smoothing. <code class=”code”>0.0</code> causes the filter to adaptively choose a smoothing strength on a per-frame basis. Defaults to <code class=”code”>0.0</code>. </dd> <dt><samp class=”option”>smooth_window_multiplier</samp> <dd> Controls the size of the smoothing window (the number of frames buffered to determine motion information from). The size of the smoothing window is determined by multiplying the framerate of the video by this number. Acceptable values range from <code class=”code”>0.1</code> to <code class=”code”>10.0</code>.

Larger values increase the amount of motion data available for determining how to smooth the camera path, potentially improving smoothness, but also increase latency and memory usage. Defaults to <code class=”code”>2.0</code>. </dd> </dl>

 <h4 class=”subsection”>34.6.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Stabilize a video with a fixed, medium smoothing strength: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, deshake_opencl=smooth_strength=0.5, hwdownload" OUTPUT </pre>

Stabilize a video with debugging (both in console and in rendered video): <pre class=”example-preformatted”>-i INPUT -filter_complex "[0:v]format=rgba, hwupload, deshake_opencl=debug=1, hwdownload, format=rgba, format=yuv420p" -v verbose OUTPUT </pre>

 <h3 class=”section”>34.7 dilation_opencl# TOC</h3>

Apply dilation effect to the video. This filter replaces the pixel by the local(3x3) maximum. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>threshold0</samp></dt> <dt><samp class=”option”>threshold1</samp></dt> <dt><samp class=”option”>threshold2</samp></dt> <dt><samp class=”option”>threshold3</samp></dt> <dd> Limit the maximum change for each plane. Range is <code class=”code”>[0, 65535]</code> and default value is <code class=”code”>65535</code>. If <code class=”code”>0</code>, plane will remain unchanged. </dd> <dt><samp class=”option”>coordinates</samp></dt> <dd> Flag which specifies the pixel to refer to. Range is <code class=”code”>[0, 255]</code> and default value is <code class=”code”>255</code>, i.e. all eight pixels are used. Flags to local 3x3 coordinates region centered on <code class=”code”>x</code>: 1 2 3 4 x 5 6 7 8 </dd> </dl>

 <h4 class=”subsection”>34.7.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply dilation filter with threshold0 set to 30, threshold1 set 40, threshold2 set to 50 and coordinates set to 231, setting each pixel of the output to the local maximum between pixels: 1, 2, 3,

6, 7, 8 of the 3x3 region centered on it in the input. If the difference between input pixel and local maximum is more then threshold of the corresponding plane, output pixel will be set to input pixel + threshold of corresponding plane. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, dilation_opencl=30:40:50:coordinates=231, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.8 nlmeans_opencl# TOC</h3>

Non-local Means denoise filter through OpenCL, this filter accepts same options as nlmeans. <h3 class=”section”>34.9 overlay_opencl# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires same memory layout for all the inputs. So, format conversion may be needed. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>x</samp></dt> <dd> Set the x coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>34.9.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs are yuv420p format. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre> The inputs have same memory layout for color channels , the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class=”example-preformatted”>-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_opencl, hwdownload" OUTPUT </pre>

 <h3 class="section">34.10 pad_opencl# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class="var">width</var> or <var class="var">height</var> is 0, the corresponding input size is used for the output. The <var class="var">width</var> expression can reference the value set by the <var class="var">height</var> expression, and vice versa. The default value of <var class="var">width</var> and <var class="var">height</var> is 0. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_w</samp></dt> <dt><samp class="option">in_h</samp></dt> <dd> The input video width and height. </dd> <dt><samp class="option">iw</samp></dt> <dt><samp class="option">ih</samp></dt> <dd> These are the same as <var class="var">in_w</var> and <var class="var">in_h</var>. </dd> <dt><samp class="option">out_w</samp></dt> <dt><samp class="option">out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the <var class="var">width</var> and <var class="var">height</var> expressions. </dd> <dt><samp class="option">ow</samp></dt> <dt><samp class="option">oh</samp></dt> <dd> These are the same as <var class="var">out_w</var> and <var class="var">out_h</var>. </dd> <dt><samp class="option">x</samp></dt> <dt><samp

class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var> expressions, or NAN if not yet specified. </dd> <dt><samp class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var> / <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt> <dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt> <dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var> / <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd> </dl>

 <h3 class=”section”>34.11 prewitt_opencl# TOC</h3>

Apply the Prewitt operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Prewitt_operator”>http to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>34.11.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the Prewitt operator with scale set to 2 and delta set to 10. <pre class=”example-preformatted”>-i INPUT -vf "hwupload, prewitt_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.12 program_opencl# TOC</h3>

Filter video using an OpenCL program. <dl class=”table”> <dt><samp class=”option”>source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class=”option”>kernel</samp></dt> <dd> Kernel name in program. </dd> <dt><samp class=”option”>inputs</samp></dt> <dd> Number of inputs to the filter. Defaults to 1. </dd>

<dt><samp class="option">size, s</samp></dt> <dd> Size of output frames. Defaults to the same as the first input. </dd> </dl>

The <code class="code">program_opencl</code> filter also supports the framesync options. The program source file must contain a kernel function with the given name, which will be run once for each plane of the output. Each run on a plane gets enqueued as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a pixel in the destination image. The kernel function needs to take the following arguments: <ul class="itemize mark-bullet"> Destination image, <var class="var">__write_only image2d_t</var>.

This image will become the output; the kernel should write all of it. Frame index, <var class="var">unsigned int</var>.

This is a counter starting from zero and increasing by one for each frame. Source images, <var class="var">__read_only image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to.

Example programs: <ul class="itemize mark-bullet"> Copy the input to the output (output must be the same size as the input). <pre class="verbatim">__kernel void copy(__write_only image2d_t destination, unsigned int index, __read_only image2d_t source) { const sampler_t sampler = CLK_NORMALIZED_COORDS_FALSE;

int2 location = (int2)(get_global_id(0), get_global_id(1));

float4 value = read_imagef(source, sampler, location);

write_imagef(destination, location, value); } </pre> Apply a simple transformation, rotating the input by an amount increasing with the index counter. Pixel values are linearly interpolated by the sampler, and the output need not have the same dimensions as the input. <pre class="verbatim">__kernel void rotate_image(__write_only image2d_t dst, unsigned int index, __read_only image2d_t src) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float angle = (float)index / 100.0f;

float2 dst_dim = convert_float2(get_image_dim(dst)); float2 src_dim = convert_float2(get_image_dim(src));

float2 dst_cen = dst_dim / 2.0f; float2 src_cen = src_dim / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1));

float2 dst_pos = convert_float2(dst_loc) - dst_cen; float2 src_pos = { cos(angle) * dst_pos.x - sin(angle) * dst_pos.y, sin(angle) * dst_pos.x + cos(angle) * dst_pos.y }; src_pos = src_pos * src_dim / dst_dim;

float2 src_loc = src_pos + src_cen;

if (src_loc.x < 0.0f || src_loc.y < 0.0f || src_loc.x > src_dim.x || src_loc.y > src_dim.y) write_imagef(dst, dst_loc, 0.5f); else write_imagef(dst, dst_loc, read_imagef(src, sampler, src_loc)); } </pre> Blend two inputs together, with the amount of each input used varying with the index counter. <pre class="verbatim">__kernel void blend_images(__write_only image2d_t dst, unsigned int index, __read_only image2d_t src1, __read_only image2d_t src2) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR);

float blend = (cos((float)index / 50.0f) + 1.0f) / 2.0f;

int2 dst_loc = (int2)(get_global_id(0), get_global_id(1)); int2 src1_loc = dst_loc * get_image_dim(src1) / get_image_dim(dst); int2 src2_loc = dst_loc * get_image_dim(src2) / get_image_dim(dst);

float4 val1 = read_imagef(src1, sampler, src1_loc); float4 val2 = read_imagef(src2, sampler, src2_loc);

write_imagef(dst, dst_loc, val1 * blend + val2 * (1.0f - blend)); } </pre>

 <h3 class="section">34.13 remap_opencl# TOC</h3>

Remap pixels using 2nd: Xmap and 3rd: Ymap input video stream. Destination pixel at position (X, Y) will be picked from source (x, y) position where x = Xmap(X, Y) and y = Ymap(X, Y). If mapping values are out of range, zero value for pixel will be used for destination pixel. Xmap and Ymap input video streams must be of same dimensions. Output video stream will have Xmap/Ymap video stream dimensions. Xmap and Ymap input video streams are 32bit float pixel format, single channel. <dl class="table"> <dt><samp class="option">interp</samp></dt> <dd> Specify interpolation used for remapping of pixels. Allowed values are <code class="code">near</code> and <code class="code">linear</code>. Default value is <code class="code">linear</code>. </dd> <dt><samp class="option">fill</samp></dt> <dd> Specify the color of the unmapped pixels. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. Default color is <code class="code">black</code>. </dd> </dl>

 <h3 class="section">34.14 roberts_opencl<span

class=”pull-right”># TOC</h3> Apply the Roberts cross operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Roberts_cross”>https://en.w to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>34.14.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply the Roberts cross operator with scale set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, roberts_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.15 sobel_opencl# TOC</h3>

Apply the Sobel operator (<a class=”url” href=”https://en.wikipedia.org/wiki/Sobel_operator”>https:// to input video stream. The filter accepts the following option: <dl class=”table”> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. Default value is <code class=”code”>0xf</code>, by which all planes are processed. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set value which will be multiplied with filtered result. Range is <code class=”code”>[0.0, 65535]</code> and default value is <code class=”code”>1.0</code>. </dd> <dt><samp class=”option”>delta</samp></dt> <dd> Set value which will be added to filtered result. Range is <code class=”code”>[-65535, 65535]</code> and default value is <code class=”code”>0.0</code>. </dd> </dl>

 <h4 class=”subsection”>34.15.1 Example# TOC</h4>

<ul class=”itemize mark-bullet”> Apply sobel operator with scale

set to 2 and delta set to 10 <pre class=”example-preformatted”>-i INPUT -vf "hwupload, sobel_opencl=scale=2:delta=10, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.16 tonemap_opencl# TOC</h3>

Perform HDR(PQ/HLG) to SDR conversion with tone-mapping. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>tonemap</samp></dt> <dd> Specify the tone-mapping operator to be used. Same as tonemap option in tonemap. </dd> <dt><samp class=”option”>param</samp></dt> <dd> Tune the tone mapping algorithm. same as param option in tonemap. </dd> <dt><samp class=”option”>desat</samp></dt> <dd> Apply desaturation for highlights that exceed this level of brightness. The higher the parameter, the more color information will be preserved. This setting helps prevent unnaturally blown-out colors for super-highlights, by (smoothly) turning into white instead. This makes images feel more natural, at the cost of reducing information about out-of-range colors. The default value is 0.5, and the algorithm here is a little different from the cpu version tonemap currently. A setting of 0.0 disables this option. </dd> <dt><samp class=”option”>threshold</samp></dt> <dd> The tonemapping algorithm parameters is fine-tuned per each scene. And a threshold is used to detect whether the scene has changed or not. If the distance between the current frame average brightness and the current running average exceeds a threshold value, we would re-calculate scene average and peak brightness. The default value is 0.2. </dd> <dt><samp class=”option”>format</samp></dt> <dd> Specify the output pixel format. Currently supported formats are: <dl class=”table”> <dt><var class=”var”>p010</var></dt> <dt><var class=”var”>nv12</var></dt> </dl>

</dd> <dt><samp class=”option”>range, r</samp></dt> <dd> Set the output color range. Possible values are: <dl class=”table”> <dt><var class=”var”>tv/mpeg</var></dt> <dt><var class=”var”>pc/jpeg</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>primaries, p</samp></dt> <dd> Set the output color primaries. Possible values are: <dl class=”table”> <dt><var class=”var”>bt709</var></dt> <dt><var class=”var”>bt2020</var></dt> </dl>

Default is same as input. </dd> <dt><samp class=”option”>transfer, t</samp></dt> <dd> Set the output transfer characteristics. Possible values are: <dl class=”table”> <dt><var class=”var”>bt709</var></dt>

<dt><var class="var">bt2020</var></dt> </dl>

Default is bt709. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Possible value are: <dl class="table"> <dt><var class="var">bt709</var></dt> <dt><var class="var">bt2020</var></dt> </dl>

Default is same as input. </dd> </dl>

 <h4 class="subsection">34.16.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(PQ/HLG) video to bt2020-transfer-characteristic p010 format using linear operator. <pre class="example-preformatted">-i INPUT -vf "format=p010,hwupload,tonemap_opencl=t=bt2020:tonemap=linear:format=p010,hwdownload,format OUTPUT </pre>

 <h3 class="section">34.17 unsharp_opencl# TOC</h3>

Sharpen or blur the input video. It accepts the following parameters: <dl class="table"> <dt><samp class="option">luma_msize_x, lx</samp></dt> <dd> Set the luma matrix horizontal size. Range is <code class="code">[1, 23]</code> and default value is <code class="code">5</code>. </dd> <dt><samp class="option">luma_msize_y, ly</samp></dt> <dd> Set the luma matrix vertical size. Range is <code class="code">[1, 23]</code> and default value is <code class="code">5</code>. </dd> <dt><samp class="option">luma_amount, la</samp></dt> <dd> Set the luma effect strength. Range is <code class="code">[-10, 10]</code> and default value is <code class="code">1.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> <dt><samp class="option">chroma_msize_x, cx</samp></dt> <dd> Set the chroma matrix horizontal size. Range is <code class="code">[1, 23]</code> and default value is <code class="code">5</code>. </dd> <dt><samp class="option">chroma_msize_y, cy</samp></dt> <dd> Set the chroma matrix vertical size. Range is <code class="code">[1, 23]</code> and default value is <code class="code">5</code>. </dd> <dt><samp class="option">chroma_amount, ca</samp></dt> <dd> Set the chroma effect strength. Range is <code class="code">[-10, 10]</code> and default value is <code class="code">0.0</code>. Negative values will blur the input video, while positive values will sharpen it, a value of zero will disable the effect. </dd> </dl>

All parameters are optional and default to the equivalent of the string ’5:5:1.0:5:5:0.0’. <h4 class=”subsection”>34.17.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Apply strong luma sharpen effect: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=luma_msize_x=7:luma_msize_y=7:luma_amount=2.5, hwdownload" OUTPUT </pre>

Apply a strong blur of both luma and chroma parameters: <pre class=”example-preformatted”>-i INPUT -vf "hwupload, unsharp_opencl=7:7:-2:7:7:-2, hwdownload" OUTPUT </pre>

 <h3 class=”section”>34.18 xfade_opencl# TOC</h3>

Cross fade two videos with custom transition effect by using OpenCL. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>transition</samp></dt> <dd> Set one of possible transition effects. <dl class=”table”> <dt><samp class=”option”>custom</samp></dt> <dd> Select custom transition effect, the actual transition description will be picked from source and kernel options. </dd> <dt><samp class=”option”>fade</samp></dt> <dt><samp class=”option”>wipeleft</samp></dt> <dt><samp class=”option”>wiperight</samp></dt> <dt><samp class=”option”>wipeup</samp></dt> <dt><samp class=”option”>wipedown</samp></dt> <dt><samp class=”option”>slideleft</samp></dt> <dt><samp class=”option”>slideright</samp></dt> <dt><samp class=”option”>slideup</samp></dt> <dt><samp class=”option”>slidedown</samp></dt> <dd> Default transition is fade. </dd> </dl>

</dd> <dt><samp class=”option”>source</samp></dt> <dd> OpenCL program source file for custom transition. </dd> <dt><samp class=”option”>kernel</samp></dt> <dd> Set name of kernel to use for custom transition from program source file. </dd> <dt><samp class=”option”>duration</samp></dt> <dd> Set duration of video transition. </dd> <dt><samp class=”option”>offset</samp></dt> <dd> Set time of start of transition relative to first video. </dd> </dl>

The program source file must contain a kernel function with the given name, which will be run once for each plane of the output. Each run on a plane gets enqueued as a separate 2D global NDRange with one work-item for each pixel to be generated. The global ID offset for each work-item is therefore the coordinates of a pixel in the destination image. The kernel function

needs to take the following arguments: <ul class="itemize mark-bullet"> Destination image, <var class="var">___write_only image2d_t</var>.

This image will become the output; the kernel should write all of it. First Source image, <var class="var">___read_only image2d_t</var>. Second Source image, <var class="var">___read_only image2d_t</var>.

These are the most recent images on each input. The kernel may read from them to generate the output, but they can’t be written to. Transition progress, <var class="var">float</var>. This value is always between 0 and 1 inclusive.

Example programs: <ul class="itemize mark-bullet"> Apply dots curtain transition effect: <pre class="verbatim">___kernel void blend_images(___write_only image2d_t dst, ___read_only image2d_t src1, ___read_only image2d_t src2, float progress) { const sampler_t sampler = (CLK_NORMALIZED_COORDS_FALSE | CLK_FILTER_LINEAR); int2 p = (int2)(get_global_id(0), get_global_id(1)); float2 rp = (float2)(get_global_id(0), get_global_id(1)); float2 dim = (float2)(get_image_dim(src1).x, get_image_dim(src1).y); rp = rp / dim;

float2 dots = (float2)(20.0, 20.0); float2 center = (float2)(0,0); float2 unused;

float4 val1 = read_imagef(src1, sampler, p); float4 val2 = read_imagef(src2, sampler, p); bool next = distance(fract(rp * dots, &unused), (float2)(0.5, 0.5)) < (progress / distance(rp, center));

write_imagef(dst, p, next ? val1 : val2); } </pre>

 <h2 class="chapter">35 VAAPI Video Filters# TOC</h2>

VAAPI Video filters are usually used with VAAPI decoder and VAAPI encoder. Below is a description of VAAPI video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vaapi</code>. To use vaapi filters, you need to setup the vaapi device correctly. For more information, please read <a class="url" href="https://trac.ffmpeg.org/wiki/Hardware/VAAPI">https://trac.ffmpeg.org/wiki/Hardware/
  <h3 class="section">35.1 overlay_vaapi# TOC</h3>

Overlay one video on the top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt>

<dt><samp class="option">y</samp></dt> <dd> Set expressions for the x and y coordinates of the overlaid video on the main video. Default value is "0" for both expressions. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set expressions for the width and height the overlaid video on the main video. Default values are ’overlay_iw’ for ’w’ and ’overlay_ih*w/overlay_iw’ for ’h’. The expressions can contain the following parameters: <dl class="table"> <dt><samp class="option">main_w, W</samp></dt> <dt><samp class="option">main_h, H</samp></dt> <dd> The main input width and height. </dd> <dt><samp class="option">overlay_iw</samp></dt> <dt><samp class="option">overlay_ih</samp></dt> <dd> The overlay input width and height. </dd> <dt><samp class="option">overlay_w, w</samp></dt> <dt><samp class="option">overlay_h, h</samp></dt> <dd> The overlay output width and height. </dd> <dt><samp class="option">overlay_x, x</samp></dt> <dt><samp class="option">overlay_y, y</samp></dt> <dd> Position of the overlay layer inside of main </dd> </dl>

</dd> <dt><samp class="option">alpha</samp></dt> <dd> Set transparency of overlaid video. Allowed range is 0.0 to 1.0. Higher value means lower transparency. Default value is <code class="code">1.0</code>. </dd> <dt><samp class="option">eof_action</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See framesync. </dd> <dt><samp class="option">repeatlast</samp></dt> <dd> See framesync. </dd> </dl>

This filter also supports the framesync options. <h4 class="subsection">35.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Overlay an image LOGO at the top-left corner of the INPUT video. Both inputs for this filter are yuv420p format. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuv420p, hwupload[b], [a][b]overlay_vaapi" OUTPUT </pre> Overlay an image LOGO at the offset (200, 100) from the top-left corner of the INPUT video. The inputs have same memory layout for color channels, the overlay has additional alpha plane, like INPUT is yuv420p, and the LOGO is yuva420p. <pre class="example-preformatted">-i INPUT -i LOGO -filter_complex "[0:v]hwupload[a], [1:v]format=yuva420p, hwupload[b], [a][b]overlay_vaapi=x=200:y=100:w=400:h=300:alpha=1.0, hwdownload,

format=nv12" OUTPUT </pre>

 <h3 class="section">35.2 tonemap_vaapi# TOC</h3>

Perform HDR-to-SDR or HDR-to-HDR tone-mapping. It currently only accepts HDR10 as input. It accepts the following parameters: <dl class="table"> <dt><samp class="option">format</samp></dt> <dd> Specify the output pixel format. Default is nv12 for HDR-to-SDR tone-mapping and p010 for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">primaries, p</samp></dt> <dd> Set the output color primaries. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">transfer, t</samp></dt> <dd> Set the output transfer characteristics. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">matrix, m</samp></dt> <dd> Set the output colorspace matrix. Default is bt709 for HDR-to-SDR tone-mapping and same as input for HDR-to-HDR tone-mapping. </dd> <dt><samp class="option">display</samp></dt> <dd> Set the output mastering display colour volume. It is given by a ’|’-separated list of two values, two values are space separated. It set display primaries x & y in G, B, R order, then white point x & y, the nominal minimum & maximum display luminances. HDR-to-HDR tone-mapping will be performed when this option is set. </dd> <dt><samp class="option">light</samp></dt> <dd> Set the output content light level information. It accepts 2 space-separated values, the first input is the maximum light level and the second input is the maximum average light level. It is ignored for HDR-to-SDR tone-mapping, and optional for HDR-to-HDR tone-mapping. </dd> </dl>

 <h4 class="subsection">35.2.1 Example# TOC</h4>

<ul class="itemize mark-bullet"> Convert HDR(HDR10) video to bt2020-transfer-characteristic p010 format <pre class="example-preformatted">tonemap_vaapi=format=p010:t=bt2020-10 </pre> Convert HDR video to HDR video <pre class="example-preformatted">tonemap_vaapi=display=7500\ 3000|34000\ 16000|13250\ 34500|15635\ 16450|500\ 10000000 </pre>

 <h3 class="section">35.3 hstack_vaapi<span

class=”pull-right”># TOC</h3> Stack input videos horizontally. This is the VA-API variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class=”option”>height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class=”section”>35.4 vstack_vaapi# TOC</h3> Stack input videos vertically. This is the VA-API variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class=”option”>width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class=”section”>35.5 xstack_vaapi# TOC</h3> Stack video inputs into custom layout. This is the VA-API variant of the xstack filter, each input stream may have different size, this filter will scale down/up each input stream to the given output size, or the size of the first input stream. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class=”example-preformatted”>xstack_vaapi=inputs=4:layout=0_0_1920x1080|0_1

</pre>

</dd> <dt><samp class="option">grid</samp></dt> <dd> See xstack. </dd> <dt><samp class="option">grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class="option">grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h3 class="section">35.6 pad_vaapi# TOC</h3>

Add paddings to the input image, and place the original input at the provided <var class="var">x</var>, <var class="var">y</var> coordinates. It accepts the following options: <dl class="table"> <dt><samp class="option">width, w</samp></dt> <dt><samp class="option">height, h</samp></dt> <dd> Specify an expression for the size of the output image with the paddings added. If the value for <var class="var">width</var> or <var class="var">height</var> is 0, the corresponding input size is used for the output. The <var class="var">width</var> expression can reference the value set by the <var class="var">height</var> expression, and vice versa. The default value of <var class="var">width</var> and <var class="var">height</var> is 0. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> Specify the offsets to place the input image at within the padded area, with respect to the top/left border of the output image. The <var class="var">x</var> expression can reference the value set by the <var class="var">y</var> expression, and vice versa. The default value of <var class="var">x</var> and <var class="var">y</var> is 0. If <var class="var">x</var> or <var class="var">y</var> evaluate to a negative number, they’ll be changed so the input image is centered on the padded area. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify the color of the padded area. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> <dt><samp class="option">aspect</samp></dt> <dd> Pad to an aspect instead to a resolution. </dd> </dl>

The value for the <var class="var">width</var>, <var class="var">height</var>, <var class="var">x</var>, and <var class="var">y</var> options are expressions containing the following constants: <dl class="table">

<dt><samp class=”option”>in_w</samp></dt> <dt><samp class=”option”>in_h</samp></dt>
<dd> The input video width and height. </dd> <dt><samp
class=”option”>iw</samp></dt> <dt><samp class=”option”>ih</samp></dt>
<dd> These are the same as <var class=”var”>in_w</var> and <var
class=”var”>in_h</var>. </dd> <dt><samp class=”option”>out_w</samp></dt>
<dt><samp class=”option”>out_h</samp></dt> <dd> The output width and height (the size of the padded area), as specified by the
<var class=”var”>width</var> and <var class=”var”>height</var> expressions. </dd> <dt><samp class=”option”>ow</samp></dt>
<dt><samp class=”option”>oh</samp></dt> <dd> These are the same
as <var class=”var”>out_w</var> and <var class=”var”>out_h</var>.
 </dd> <dt><samp class=”option”>x</samp></dt> <dt><samp
class=”option”>y</samp></dt> <dd> The x and y offsets as specified by the <var class=”var”>x</var> and <var class=”var”>y</var>
expressions, or NAN if not yet specified. </dd> <dt><samp
class=”option”>a</samp></dt> <dd> same as <var class=”var”>iw</var>
/ <var class=”var”>ih</var> </dd> <dt><samp class=”option”>sar</samp></dt>
<dd> input sample aspect ratio </dd> <dt><samp class=”option”>dar</samp></dt>
<dd> input display aspect ratio, it is the same as (<var class=”var”>iw</var>
/ <var class=”var”>ih</var>) * <var class=”var”>sar</var> </dd>
</dl>

 <h3 class=”section”>35.7 drawbox_vaapi<a class=”anchor hidden-xs” href=”#drawbox_005fvaapi”
aria-hidden=”true”># TOC</h3>

Draw a colored box on the input image. It accepts the following
parameters: <dl class=”table”> <dt><samp class=”option”>x</samp></dt>
<dt><samp class=”option”>y</samp></dt> <dd> The expressions
which specify the top left corner coordinates of the box. It defaults to
0. </dd> <dt><samp class=”option”>width, w</samp></dt>
<dt><samp class=”option”>height, h</samp></dt> <dd> The expressions which specify the width and height of the box; if 0 they are
interpreted as the input width and height. It defaults to 0. </dd>
<dt><samp class=”option”>color, c</samp></dt> <dd> Specify
the color of the box to write. For the general syntax of this option,
check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.
 </dd> <dt><samp class=”option”>thickness, t</samp></dt>
<dd> The expression which sets the thickness of the box edge. A value
of <code class=”code”>fill</code> will create a filled box. Default value is
<code class=”code”>3</code>. See below for the list of accepted
constants. </dd> <dt><samp class=”option”>replace</samp></dt>
<dd> With value <code class=”code”>1</code>, the pixels of the painted
box will overwrite the video’s color and alpha pixels. Default is <code
class=”code”>0</code>, which composites the box onto the input video.

</dd> </dl>

The parameters for <var class="var">x</var>, <var class="var">y</var>, <var class="var">w</var> and <var class="var">h</var> and <var class="var">t</var> are expressions containing the following constants: <dl class="table"> <dt><samp class="option">in_h, ih</samp></dt> <dt><samp class="option">in_w, iw</samp></dt> <dd> The input width and height. </dd> <dt><samp class="option">x</samp></dt> <dt><samp class="option">y</samp></dt> <dd> The x and y offset coordinates where the box is drawn. </dd> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> The width and height of the drawn box. </dd> <dt><samp class="option">t</samp></dt> <dd> The thickness of the drawn box. </dd> </dl>

 <h4 class="subsection">35.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Draw a black box around the edge of the input image: <pre class="example-preformatted">drawbox </pre>

Draw a box with color red and an opacity of 50%: <pre class="example-preformatted">drawbox=10:20:200:60:red@0.5 </pre>

The previous example can be specified as: <pre class="example-preformatted">drawbox=x=10:y=20:w=200:h=60:color=red@0.5 </pre>

Fill the box with pink color: <pre class="example-preformatted">drawbox=x=10:y=10:w=100:h=100:color=pink@0.5:t=fill </pre>

Draw a 2-pixel red 2.40:1 mask: <pre class="example-preformatted">drawbox=x=-t:y=0.5*(ih-iw/2.4)-t:w=iw+t*2:h=iw/2.4+t*2:t=2:c=red </pre>

 <h2 class="chapter">36 VideoToolbox Video Filters# TOC</h2>

Below is a description of the currently available VideoToolbox video filters. VideoToolbox filter depends on VideoToolbox framework, and is auto detected when building ffmpeg for an

Apple platform such as macOS. Add <code class="code">--enable-videotoolbox</code> to configure if autodetect is disabled. <h3 class="section">36.1 scale_vt# TOC</h3>

Scale and convert the color parameters using VTPixelTransferSession. The filter accepts the following options: <dl class="table"> <dt><samp class="option">w</samp></dt> <dt><samp class="option">h</samp></dt> <dd> Set the output video dimension expression. Default value is the input dimension. </dd> <dt><samp class="option">color_matrix</samp></dt> <dd> Set the output colorspace matrix. </dd> <dt><samp class="option">color_primaries</samp></dt> <dd> Set the output color primaries. </dd> <dt><samp class="option">color_transfer</samp></dt> <dd> Set the output transfer characteristics. </dd> </dl>

 <h4 class="subsection">36.1.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Perform HDR to SDR conversion, and scale to half size of input <pre class="example-preformatted">ffmpeg -hwaccel videotoolbox \ -hwaccel_output_format videotoolbox_vld \ -i hdr.mov \ -c:v hevc_videotoolbox \ -profile:v main \ -b:v 3M \ -vf scale_vt=w=iw/2:h=ih/2:color_matrix=bt709:color_primaries=bt709:color_transfer=bt709 \ -c:a copy \ -tag:v hvc1 \ sdr.mp4 </pre>

 <h3 class="section">36.2 transpose_vt# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt>

<dd> Rotate by 90 degrees clockwise and vertically flip. </dd> <dt>‘<samp class="samp">hflip</samp>’</dt> <dd> Flip the input video horizontally. </dd> <dt>‘<samp class="samp">vflip</samp>’</dt> <dd> Flip the input video vertically. </dd> </dl>

</dd> <dt><samp class="option">passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class="samp">portrait</samp>’</d <dd> Preserve portrait geometry (when <var class="var">height</var> >= <var class="var">width</var>). </dd> <dt>‘<samp class="samp">landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class="var">width</var> >= <var class="var">height</var>). </dd> </dl>

</dd> </dl>

 <h2 class="chapter">37 Vulkan Video Filters# TOC</h2>

Below is a description of the currently available Vulkan video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-vulkan</code>, and have a SPIR-V compiler (<code class="code">glslc</code> or <code class="code">glslangValidator</code>) available at build time. Running Vulkan filters requires you to initialize a hardware device and to pass that device to all filters in any filter graph. <dl class="table"> <dt><samp class="option">-init_hw_device vulkan[=<var class="var">name</var>][:<var class="var">device</var>[,<var class="var">key=value</var>...]]</sam <dd> Initialise a new hardware device of type <var class="var">vulkan</var> called <var class="var">name</var>, using the given device parameters and options in <var class="var">key=value</var>. The following options are supported: <dl class="table"> <dt><samp class="option">debug</samp></dt> <dd> Switches validation layers on if set to 1. </dd> <dt><samp class="option">linear_images</samp></dt> <dd> Allocates linear images. Does not apply to decoding. </dd> <dt><samp class="option">disable_multiplane</samp></dt> <dd> Disables multiplane images. Does not apply to decoding. </dd> <dt><samp class="option">avoid_host_import</samp></dt> <dd> Avoids the use of dynamic host memory imports, in favor of a regular memcpy() into a previously mapped buffer. </dd> </dl>

</dd> <dt><samp class="option">-filter_hw_device <var class="var">name</var></samp></dt> <dd> Pass the hardware device called <var class="var">name</var> to

all filters in any filter graph. </dd> </dl>

For more detailed information see https://www.ffmpeg.org/ffmpeg.html#Advanced-Video-options <ul class="itemize mark-bullet"> Example of choosing the first device and running nlmeans_vulkan filter with default parameters on it. <pre class="example-preformatted">-init_hw_device vulkan=vk:0 -filter_hw_device vk -i INPUT -vf "hwupload,nlmeans_vulkan,hwdownload" OUTPUT </pre>

As Vulkan filters are not able to access frame data in normal memory, all frame data needs to be uploaded (hwupload) to hardware surfaces connected to the appropriate device before being used and then downloaded (hwdownload) back to normal memory. Note that hwupload will upload to a frame with the same layout as the software frame, so it may be necessary to add a format filter immediately before to get the input into the right format and hwdownload does not support all formats on the output - it is usually necessary to insert an additional format filter immediately following in the graph to get the output in a supported format. <h3 class="section">37.1 avgblur_vulkan# TOC</h3>

Apply an average blur filter, implemented on the GPU using Vulkan. The filter accepts the following options: <dl class="table"> <dt><samp class="option">sizeX</samp></dt> <dd> Set horizontal radius size. Range is <code class="code">[1, 32]</code> and default value is <code class="code">3</code>. </dd> <dt><samp class="option">sizeY</samp></dt> <dd> Set vertical radius size. Range is <code class="code">[1, 32]</code> and default value is <code class="code">3</code>. </dd> <dt><samp class="option">planes</samp></dt> <dd> Set which planes to filter. Default value is <code class="code">0xf</code>, by which all planes are processed. </dd> </dl>

 <h3 class="section">37.2 blend_vulkan# TOC</h3>

Blend two Vulkan frames into each other. The <code class="code">blend</code> filter takes two input streams and outputs one stream, the first input is the "top" layer and second input is "bottom" layer. By default, the output termi-

nates when the longest input terminates. A description of the accepted options follows. <dl class="table"> <dt><samp class="option">c0_mode</samp></dt> <dt><samp class="option">c1_mode</samp></dt> <dt><samp class="option">c2_mode</samp></dt> <dt><samp class="option">c3_mode</samp></dt> <dt><samp class="option">all_mode</samp></dt> <dd> Set blend mode for specific pixel component or all pixel components in case of <var class="var">all_mode</var>. Default value is <code class="code">normal</code>. Available values for component modes are: <dl class="table"> <dt>‘<samp class="samp">normal</samp>’</dt> <dt>‘<samp class="samp">multiply</samp>’
</dl>

</dd> </dl>

 <h3 class="section">37.3 bwdif_vulkan# TOC</h3>

Deinterlacer using bwdif, the "Bob Weaver Deinterlacing Filter" algorithm, implemented on the GPU using Vulkan. It accepts the following parameters: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> The interlacing mode to adopt. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, send_frame</samp></dt> <dd> Output one frame for each frame. </dd> <dt><samp class="option">1, send_field</samp></dt> <dd> Output one frame for each field. </dd> </dl>

The default value is <code class="code">send_field</code>. </dd> <dt><samp class="option">parity</samp></dt> <dd> The picture field parity assumed for the input interlaced video. It accepts one of the following values: <dl class="table"> <dt><samp class="option">0, tff</samp></dt> <dd> Assume the top field is first. </dd> <dt><samp class="option">1, bff</samp></dt> <dd> Assume the bottom field is first. </dd> <dt><samp class="option">-1, auto</samp></dt> <dd> Enable automatic detection of field parity. </dd> </dl>

The default value is <code class="code">auto</code>. If the interlacing is unknown or the decoder does not export this information, top field first will be assumed. </dd> <dt><samp class="option">deint</samp></dt> <dd> Specify which frames to deinterlace. Accepts one of the following values: <dl class="table"> <dt><samp class="option">0, all</samp></dt> <dd> Deinterlace all frames. </dd> <dt><samp class="option">1, interlaced</samp></dt> <dd> Only deinterlace frames marked as interlaced. </dd> </dl>

The default value is <code class="code">all</code>. </dd>

</dl>

 <h3 class="section">37.4 chromaber_vulkan# TOC</h3>

Apply an effect that emulates chromatic aberration. Works best with RGB inputs, but provides a similar effect with YCbCr inputs too. <dl class="table"> <dt><samp class="option">dist_x</samp></dt> <dd> Horizontal displacement multiplier. Each chroma pixel’s position will be multiplied by this amount, starting from the center of the image. Default is <code class="code">0</code>. </dd> <dt><samp class="option">dist_y</samp></dt> <dd> Similarly, this sets the vertical displacement multiplier. Default is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">37.5 color_vulkan# TOC</h3>

Video source that creates a Vulkan frame of a solid color. Useful for benchmarking, or overlaying. It accepts the following parameters: <dl class="table"> <dt><samp class="option">color</samp></dt> <dd> The color to use. Either a name, or a hexadecimal value. The default value is <code class="code">black</code>. </dd> <dt><samp class="option">size</samp></dt> <dd> The size of the output frame. Default value is <code class="code">1920x1080</code>. </dd> <dt><samp class="option">rate</samp></dt> <dd> The framerate to output at. Default value is <code class="code">60</code> frames per second. </dd> <dt><samp class="option">duration</samp></dt> <dd> The video duration. Default value is <code class="code">-0.000001</code>. </dd> <dt><samp class="option">sar</samp></dt> <dd> The video signal aspect ratio. Default value is <code class="code">1/1</code>. </dd> <dt><samp class="option">format</samp></dt> <dd> The pixel format of the output Vulkan frames. Default value is <code class="code">yuv444p</code>. </dd> <dt><samp class="option">out_range</samp></dt> <dd> Set the output YCbCr sample range. This allows the autodetected value to be overridden as well as allows forcing a specific value used for the output and encoder. If not specified, the range depends on the pixel format. Possible values: <dl class="table"> <dt>‘<samp class="samp">auto/unknown</samp>’</dt> <dd> Choose automatically. </dd> <dt>‘<samp class="samp">jpeg/full/pc</samp>’</dt> <dd> Set full range (0-255 in case of 8-bit luma). </dd> <dt>‘<samp class="samp">mpeg/limited/tv</samp>’</dt> <dd> Set "MPEG" range (16-235 in case of 8-bit luma). </dd> </dl>

</dd> </dl>

 <h3 class="section">37.6 vflip_vulkan# TOC</h3>

Flips an image vertically. <h3 class="section">37.7 hflip_vulkan# TOC</h3>

Flips an image horizontally. <h3 class="section">37.8 flip_vulkan# TOC</h3>

Flips an image along both the vertical and horizontal axis. <h3 class="section">37.9 fruc_vulkan# TOC</h3>

Frame rate up-conversion using the NVIDIA Vulkan optical flow extension (<code class="code">VK_NV_optical_flow</code>), implemented on the GPU using Vulkan. For each pair of consecutive input frames the filter computes a forward and backward optical flow field on the device’s optical flow engine, and uses those fields to synthesise motion-compensated intermediate frames at the requested output frame rate. The frame timing logic mirrors the <code class="code">framerate</code> filter. The majority of YUV and RGB pixel formats are supported. The exceptions are subsampled packed YUV formats such as <code class="code">yuyv422</code> and <code class="code">uyvy422</code>, formats with more than 16 bits integer data per component, such as <code class="code">rgb96</code> and <code class="code">gray32</code>, and Bayer formats. <dl class="table"> <dt><samp class="option">fps</samp></dt> <dd> A string describing the desired output frame rate, evaluated as an expression. The following constants are available: <code class="code">source_fps</code>. The default value is <code class="code">60</code>. </dd> <dt><samp class="option">perf</samp></dt> <dd> Optical flow performance level, trading motion-estimation quality for speed. This is the dominant cost at high resolutions. Possible values are: <dl class="table"> <dt>‘<samp class="samp">slow</samp>’</dt> <dd> Highest quality, slowest. This is the default. </dd> <dt>‘<samp class="samp">medium</samp>’</dt> <dd> Balanced quality and speed. </dd> <dt>‘<samp class="samp">fast</samp>’</dt> <dd> Lowest quality, fastest. </dd> </dl>

</dd> <dt><samp class="option">grid</samp></dt> <dd> Optical

flow output grid size in pixels. A coarser grid computes fewer flow vectors, making it faster and less memory hungry at the cost of flow detail. Possible values are <code class=”code”>auto</code> (the default; selects the finest grid the device supports, usually 1x1), <code class=”code”>1</code>, <code class=”code”>2</code>, <code class=”code”>4</code> and <code class=”code”>8</code>. Only grid sizes the device advertises are accepted. </dd> </dl>

 <h3 class=”section”>37.10 gblur_vulkan# TOC</h3>

Apply Gaussian blur filter on Vulkan frames. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sigma</samp></dt> <dd> Set horizontal sigma, standard deviation of Gaussian blur. Default is <code class=”code”>0.5</code>. </dd> <dt><samp class=”option”>sigmaV</samp></dt> <dd> Set vertical sigma, if negative it will be same as <code class=”code”>sigma</code>. Default is <code class=”code”>-1</code>. </dd> <dt><samp class=”option”>planes</samp></dt> <dd> Set which planes to filter. By default all planes are filtered. </dd> <dt><samp class=”option”>size</samp></dt> <dd> Set the kernel size along the horizontal axis. Default is <code class=”code”>19</code>. </dd> <dt><samp class=”option”>sizeV</samp></dt> <dd> Set the kernel size along the vertical axis. Default is <code class=”code”>0</code>, which sets to use the same value as <var class=”var”>size</var>. </dd> </dl>

 <h3 class=”section”>37.11 nlmeans_vulkan# TOC</h3>

Denoise frames using Non-Local Means algorithm, implemented on the GPU using Vulkan. Supports more pixel formats than nlmeans or nlmeans_opencl, including alpha channel support. The filter accepts the following options. <dl class=”table”> <dt><samp class=”option”>s</samp></dt> <dd> Set denoising strength for all components. Default is 1.0. Must be in range [0.0, 100.0]. </dd> <dt><samp class=”option”>p</samp></dt> <dd> Set patch size for all planes. Default is 7. Must be odd number in range [0, 99]. </dd> <dt><samp class=”option”>r</samp></dt> <dd> Set research size. Default is 15. Must be odd number in range [0, 99]. </dd> <dt><samp class=”option”>t</samp></dt> <dd> Set parallelism. Default is 8. Must be a number in the range [1, 64]. Larger values will use more VRAM but may not result in greater speed. The optimal value is hardware and input dependent.

 </dd> <dt><samp class="option">s0</samp></dt> <dt><samp class="option">s1</samp></dt> <dt><samp class="option">s2</samp></dt> <dt><samp class="option">s3</samp></dt> <dd> Set denoising strength for a specific component. Default is <var class="var">1.0</var>, equal to <samp class="option">s</samp>. Must be in range [0.0, 100.0]. 0.0 disables denoising in that component. </dd> <dt><samp class="option">p0</samp></dt> <dt><samp class="option">p1</samp></dt> <dt><samp class="option">p2</samp></dt> <dt><samp class="option">p3</samp></dt> <dd> Set patch size for a specific component. Default is <var class="var">7</var>, equal to <samp class="option">p</samp>. Must be odd number in range [0, 99]. </dd> </dl>

 <h3 class="section">37.12 overlay_vulkan# TOC</h3>

Overlay one video on top of another. It takes two inputs and has one output. The first input is the "main" video on which the second input is overlaid. This filter requires all inputs to use the same pixel format. So, format conversion may be needed. The filter accepts the following options: <dl class="table"> <dt><samp class="option">x</samp></dt> <dd> Set the x coordinate of the overlaid video on the main video. Default value is <code class="code">0</code>. </dd> <dt><samp class="option">y</samp></dt> <dd> Set the y coordinate of the overlaid video on the main video. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">37.13 transpose_vulkan# TOC</h3>

Transpose rows with columns in the input video and optionally flip it. For more in depth examples see the transpose video filter, which shares mostly the same options. It accepts the following parameters: <dl class="table"> <dt><samp class="option">dir</samp></dt> <dd> Specify the transposition direction. Can assume the following values: <dl class="table"> <dt>‘<samp class="samp">cclock_flip</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise and vertically flip. (default) </dd> <dt>‘<samp class="samp">clock</samp>’</dt> <dd> Rotate by 90 degrees clockwise. </dd> <dt>‘<samp class="samp">cclock</samp>’</dt> <dd> Rotate by 90 degrees counterclockwise. </dd> <dt>‘<samp class="samp">clock_flip</samp>’</dt> <dd> Rotate by 90 degrees clockwise and vertically flip. </dd> </dl>

</dd> <dt><samp class=”option”>passthrough</samp></dt> <dd> Do not apply the transposition if the input geometry matches the one specified by the specified value. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Always apply transposition. (default) </dd> <dt>‘<samp class=”samp”>portrait</samp>’</d <dd> Preserve portrait geometry (when <var class=”var”>height</var> >= <var class=”var”>width</var>). </dd> <dt>‘<samp class=”samp”>landscape</samp>’</dt> <dd> Preserve landscape geometry (when <var class=”var”>width</var> >= <var class=”var”>height</var>). </dd> </dl>

</dd> </dl>

 <h3 class=”section”>37.14 v360_vulkan# TOC</h3>

Convert 360 videos between various formats. Supported formats (projections) are subset of v360 filter, that are most popular 360-degree video formats. The filter accepts subset of v360 options: <dl class=”table”> <dt><samp class=”option”>input</samp></dt> <dt><samp class=”option”>output</samp></dt> <dd> Set format of the input/output video. <dl class=”table”> <dt>‘<samp class=”samp”>e</samp>’</dt> <dt>‘<samp class=”samp”>equirect</samp>’</dt> <dd> Equirectangular projection. </dd> <dt>‘<samp class=”samp”>eac</samp>’</dt> <dd> Equi-Angular Cubemap. (Output only) FIXME: current output doesn’t contain padding. </dd> <dt>‘<samp class=”samp”>flat</samp>’</dt> <dd> Regular video. </dd> <dt>‘<samp class=”samp”>dfisheye</samp>’</dt> <dd> Dual fisheye. </dd> <dt>‘<samp class=”samp”>sg</samp>’</dt> <dd> Stereographic format. </dd> <dt>‘<samp class=”samp”>fisheye</samp>’</dt> <dd> Fisheye projection. </dd> <dt>‘<samp class=”samp”>gopromax</samp>’</dt> <dd> GoPro Max .360 format. (Input only) A .360 video file contains two video streams (front and rear); with this input projection the filter takes two inputs and combines them into a single stream. See the <samp class=”option”>overlap</samp> option. </dd> </dl>

</dd> <dt><samp class=”option”>w</samp></dt> <dt><samp class=”option”>h</samp></dt> <dd> Set the output video resolution. See v360 options. </dd> <dt><samp class=”option”>yaw</samp></dt> <dt><samp class=”option”>pitch</samp></dt> <dt><samp class=”option”>roll</samp></dt> <dt><samp class=”option”>rorder</samp></dt> <dd> Set rotation

for the output video. See v360 options. </dd> <dt><samp class="option">h_fov</samp></dt> <dt><samp class="option">v_fov</samp></dt> <dd> Set output horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">ih_fov</samp></dt> <dt><samp class="option">iv_fov</samp></dt> <dd> Set input horizontal/vertical field of view. See v360 options. </dd> <dt><samp class="option">overlap</samp></dt> <dd> Set number of overlapped pixels on input .360 video. Only used with <code class="code">input=gopromax</code>. The GoPro Max .360 video contains overlapped area (See https://gopro.com/news/max-tech-specs-stitching-resolution). The filter blends overlapped images in these two areas. The number of pixels of overlapped areas depends on the video resolution. In most cases, the value is automatically calculated based on input video size, but it may be changed on GoPro Max2 or future models. The option is to adjust for unknown video resolutions. </dd> </dl>

 <h4 class="subsection">37.14.1 Examples# TOC</h4> Special notes exist to use with <code class="code">input=gopromax</code>. The GoPro Max .360 video file contains two video streams, and the stream number must be specified. The stream number (e.g., <code class="code">[0:5]</code>) depends on the model and shooting mode. Please check actual stream number with <code class="code">ffprobe</code> command. <ul class="itemize mark-bullet"> Convert GoPro Max (8bit color depth) .360 to Equirectangular projection. GoPro Max uses obsolete yuvj420p pixel format, thus convert the format to yuv420p before this filter. <pre class="example-preformatted">ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]format=yuv420p,hwupload[vf],[0:5]format=yuv420p,hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=e, hwdownload,format=yuv420p" OUTPUT.mkv </pre>

Convert GoPro Max2 (10bit color depth) .360 to Dual fisheye projection. GoPro Max2 uses yuv420p10le pixel format. <pre class="example-preformatted">ffmpeg -init_hw_device vulkan -i INPUT.360 -filter_complex "[0:0]hwupload[vf],[0:4]hwupload[vr], [vf][vr]v360_vulkan=input=gopromax:output=dfisheye, hwdownload,format=yuv420p10le" OUTPUT.mkv </pre>

 <h2 class="chapter">38 QSV Video Filters# <a class="anchor hidden-

xs"href="#toc-QSV-Video-Filters" aria-hidden="true">TOC</h2>

Below is a description of the currently available QSV video filters. To enable compilation of these filters you need to configure FFmpeg with <code class="code">--enable-libmfx</code> or <code class="code">--enable-libvpl</code>. To use QSV filters, you need to setup the QSV device correctly. For more information, please read <a class="url" href="https://trac.ffmpeg.org/wiki/Hardware/QuickSync">https://trac.ffmpeg.org/wiki/Hardware/QuickSy
  <h3 class="section">38.1 hstack_qsv# TOC</h3> Stack input videos horizontally. This is the QSV variant of the hstack filter, each input stream may have different height, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See hstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See hstack. </dd> <dt><samp class="option">height</samp></dt> <dd> Set height of output. If set to 0, this filter will set height of output to height of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">38.2 vstack_qsv# TOC</h3> Stack input videos vertically. This is the QSV variant of the vstack filter, each input stream may have different width, this filter will scale down/up each input stream while keeping the original aspect. It accepts the following options: <dl class="table"> <dt><samp class="option">inputs</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">shortest</samp></dt> <dd> See vstack. </dd> <dt><samp class="option">width</samp></dt> <dd> Set width of output. If set to 0, this filter will set width of output to width of the first input stream. Default value is 0. </dd> </dl>

 <h3 class="section">38.3 xstack_qsv# TOC</h3> Stack video inputs into custom layout. This is the QSV variant of the xstack filter. It accepts the following options: <dl class="table"> <dt><samp

class=”option”>inputs</samp></dt> <dd> See xstack. </dd> <dt><samp class=”option”>shortest</samp></dt> <dd> See xstack. </dd> <dt><samp class=”option”>layout</samp></dt> <dd> See xstack. Moreover, this permits the user to supply output size for each input stream. <pre class=”example-preformatted”>xstack_qsv=inputs=4:layout=0_0_1920x1080|0_h0_1920x1080|w0_0_1920
</pre>

</dd> <dt><samp class=”option”>grid</samp></dt> <dd> See xstack. </dd> <dt><samp class=”option”>grid_tile_size</samp></dt> <dd> Set output size for each input stream when <samp class=”option”>grid</samp> is set. If this option is not set, this filter will set output size by default to the size of the first input stream. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>fill</samp></dt> <dd> See xstack. </dd> </dl>

 <h2 class=”chapter”>39 Video Sources# TOC</h2>

Below is a description of the currently available video sources. <h3 class=”section”>39.1 buffer# TOC</h3>

Buffer video frames, and make them available to the filter chain. This source is mainly intended for a programmatic use, in particular through the interface defined in <samp class=”file”>libavfilter/buffersrc.h</samp>. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>video_size</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>width</samp></dt> <dd> The input video width. </dd> <dt><samp class=”option”>height</samp></dt> <dd> The input video height. </dd> <dt><samp class=”option”>pix_fmt</samp></dt> <dd> A string representing the pixel format of the buffered video frames. It may be a number corresponding to a pixel format, or a pixel format name. </dd> <dt><samp class=”option”>time_base</samp></dt> <dd> Specify the timebase assumed by the timestamps of the buffered frames. </dd> <dt><samp class=”option”>frame_rate</samp></dt> <dd> Specify the frame rate expected for the video stream. </dd>

<dt><samp class=”option”>colorspace</samp></dt> <dd> A string representing the color space of the buffered video frames. It may be a number corresponding to a color space, or a color space name. </dd> <dt><samp class=”option”>range</samp></dt> <dd> A string representing the color range of the buffered video frames. It may be a number corresponding to a color range, or a color range name. </dd> <dt><samp class=”option”>alpha_mode</samp></dt> <dd> A string representing the alpha mode of the buffered video frames. It may be a number corresponding to an alpha mode, or an alpha mode name. </dd> <dt><samp class=”option”>chroma_location</samp></dt> <dd> A string representing the chroma sample location of the buffered video frames. It may be a number corresponding to a chroma sample location, or a chroma sample location name. When it is not set, the filtergraph chooses the chroma sample location of the output. Frames without a chroma sample location are tagged with the one negotiated for the output. </dd> <dt><samp class=”option”>pixel_aspect, sar</samp></dt> <dd> The sample (pixel) aspect ratio of the input video. </dd> <dt><samp class=”option”>hw_frames_ctx</samp></dt> <dd> When using a hardware pixel format, this should be a reference to an AVHWFramesContext describing input frames. </dd> </dl>

For example: <pre class=”example-preformatted”>buffer=width=320:height=240:pix_fmt=yuv410p:time_base=1/24:sar=1
</pre>

will instruct the source to accept video frames with size 320x240 and with format "yuv410p", assuming 1/24 as the timestamps timebase and square pixels (1:1 sample aspect ratio). Since the pixel format with name "yuv410p" corresponds to the number 6 (check the enum AVPixelFormat definition in <samp class=”file”>libavutil/pixfmt.h</samp>), this example corresponds to: <pre class=”example-preformatted”>buffer=size=320x240:pixfmt=6:time_base=1/24:pixel_aspect=1/1
</pre>

Alternatively, the options can be specified as a flat string, but this syntax is deprecated: <var class=”var”>width</var>:<var class=”var”>height</var>:<var class=”var”>pix_fmt</var>:<var class=”var”>time_base.num</var>:<var class=”var”>time_base.den</var>:<var class=”var”>pixel_aspect.num</var>:<var class=”var”>pixel_aspect.den</var> <h3 class=”section”>39.2 cellauto# TOC</h3>

Create a pattern generated by an elementary cellular automaton. The initial state of the cellular automaton can be defined through the <samp class=”option”>filename</samp> and <samp class=”option”>pattern</samp> options. If such options are not specified an initial state is created randomly. At each new frame a new row in the video is filled with the

result of the cellular automaton next generation. The behavior when the whole frame is filled is defined by the <samp class="option">scroll</samp> option. This source accepts the following options: <dl class="table"> <dt><samp class="option">filename, f</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified file. In the file, each non-whitespace character is considered an alive cell, a newline will terminate the row, and further characters in the file will be ignored. </dd> <dt><samp class="option">pattern, p</samp></dt> <dd> Read the initial cellular automaton state, i.e. the starting row, from the specified string. Each non-whitespace character in the string is considered an alive cell, a newline will terminate the row, and further characters in the string will be ignored. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set the video rate, that is the number of frames generated per second. Default is 25. </dd> <dt><samp class="option">random_fill_ratio, ratio</samp></dt> <dd> Set the random fill ratio for the initial cellular automaton row. It is a floating point number value ranging from 0 to 1, defaults to 1/PHI. This option is ignored when a file or a pattern is specified. </dd> <dt><samp class="option">random_seed, seed</samp></dt> <dd> Set the seed for filling randomly the initial row, must be an integer included between 0 and UINT32_MAX. If not specified, or if explicitly set to -1, the filter will try to use a good random seed on a best effort basis. </dd> <dt><samp class="option">rule</samp></dt> <dd> Set the cellular automaton rule, it is a number ranging from 0 to 255. Default value is 110. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set the size of the output video. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. If <samp class="option">filename</samp> or <samp class="option">pattern</samp> is specified, the size is set by default to the width of the specified initial state row, and the height is set to <var class="var">width</var> * PHI. If <samp class="option">size</samp> is set, it must contain the width of the specified pattern string, and the specified pattern will be centered in the larger row. If a filename or a pattern string is not specified, the size value defaults to "320x518" (used for a randomly generated initial state). </dd> <dt><samp class="option">scroll</samp></dt> <dd> If set to 1, scroll the output upward when all the rows in the output have been already filled. If set to 0, the new generated row will be written over the top row just after the bottom row is filled. Defaults to 1. </dd> <dt><samp class="option">start_full, full</samp></dt> <dd> If set to 1, completely fill the output with generated rows before outputting the first frame. This is the default behavior, for disabling set the value to 0. </dd> <dt><samp class="option">stitch</samp></dt> <dd> If set to 1, stitch the left and right row edges together. This is the default behavior, for disabling set the value to 0. </dd> </dl>

 <h4 class="subsection">39.2.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read the initial state from <samp class="file">pattern</samp>, and specify an output of size 200x400. <pre class="example-preformatted">cellauto=f=pattern:s=200x400 </pre>

Generate a random initial row with a width of 200 cells, with a fill ratio of 2/3: <pre class="example-preformatted">cellauto=ratio=2/3:s=200x200 </pre>

Create a pattern generated by rule 18 starting by a single alive cell centered on an initial row with width 100: <pre class="example-preformatted">cellauto=p=@:s=100x400:full=0:rule=18 </pre>

Specify a more elaborated initial pattern: <pre class="example-preformatted">cellauto=p='@@ @ @@':s=100x400:full=0:rule=18 </pre>

 <h3 class="section">39.3 coreimagesrc# TOC</h3> Video source generated on GPU using Apple’s CoreImage API on OSX. This video source is a specialized version of the coreimage video filter. Use a core image generator at the beginning of the applied filterchain to generate the content. The coreimagesrc video source accepts the following options: <dl class="table"> <dt><samp class="option">list_generators</samp></dt> <dd> List all available generators along with all their respective options as well as possible minimum and maximum values along with the default values. <pre class="example-preformatted">list_generators=true </pre>

</dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the size of the sourced video. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. The default value is <code class="code">320x240</code>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class="var">frame_rate_num</var>/<var class="var">frame_rate_den</var>, an integer number, a floating point

number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class="option">sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> </dl>

Additionally, all options of the coreimage video filter are accepted. A complete filterchain can be used for further processing of the generated input without CPU-HOST transfer. See coreimage documentation and examples for details. <h4 class="subsection">39.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Use CIQRCodeGenerator to create a QR code for the FFmpeg homepage, given as complete and escaped command-line for Apple’s standard bash shell: <pre class="example-preformatted">ffmpeg -f lavfi -i coreimagesrc=s=100x100:filter=CIQRCodeGenerator@inputMessage=https\\\\\://FFmpeg.org/@inputCorrecti -frames:v 1 QRCode.png </pre> This example is equivalent to the QRCode example of coreimage without the need for a nullsrc video source.

 <h3 class="section">39.4 ddagrab# TOC</h3>

Captures the Windows Desktop via Desktop Duplication API. The filter exclusively returns D3D11 Hardware Frames, for on-gpu encoding or processing. So an explicit hwdownload is needed for any kind of software processing. It accepts the following options: <dl class="table"> <dt><samp class="option">output_idx</samp></dt> <dd> DXGI Output Index to capture. Usually corresponds to the index Windows has given the screen minus one, so it’s starting at 0. Defaults to output 0. </dd> <dt><samp class="option">draw_mouse</samp></dt> <dd> Whether to draw the mouse cursor. Defaults to true. Only affects hardware cursors. If a game or application renders its own cursor, it’ll always be captured. </dd> <dt><samp class="option">framerate</samp></dt> <dd> Maximum framerate at which the desktop will be captured - the interval between successive frames will not be smaller than the inverse of the framerate. When <var

class=”var”>dup_frames</var> is true (the default) and the desktop is not being updated often enough, the filter will duplicate a previous frame. Note that there is no background buffering going on, so when the filter is not polled often enough then the actual inter-frame interval may be significantly larger. Defaults to 30 FPS. </dd> <dt><samp class=”option”>video_size</samp></dt> <dd> Specify the size of the captured video. Defaults to the full size of the screen. Cropped from the bottom/right if smaller than screen size. </dd> <dt><samp class=”option”>offset_x</samp></dt> <dd> Horizontal offset of the captured video. </dd> <dt><samp class=”option”>offset_y</samp></dt> <dd> Vertical offset of the captured video. </dd> <dt><samp class=”option”>output_fmt</samp></dt> <dd> Desired filter output format. Defaults to 8 Bit BGRA. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Passes all supported output formats to DDA and returns what DDA decides to use. </dd> <dt>‘<samp class=”samp”>8bit</samp>’</dt> <dt>‘<samp class=”samp”>bgra</samp>’</dt> <dd> 8 Bit formats always work, and DDA will convert to them if necessary. </dd> <dt>‘<samp class=”samp”>10bit</samp>’</dt> <dt>‘<samp class=”samp”>x2bgr10</samp>’</dt> <dd> Filter initialization will fail if 10 bit format is requested but unavailable. </dd> </dl>

</dd> <dt><samp class=”option”>dup_frames</samp></dt> <dd> When this option is set to true (the default), the filter will duplicate frames when the desktop has not been updated in order to maintain approximately constant target framerate. When this option is set to false, the filter will wait for the desktop to be updated (inter-frame intervals may vary significantly in this case). </dd> </dl>

 <h4 class=”subsection”>39.4.1 Examples# TOC</h4>

Capture primary screen and encode using nvenc: <pre class=”example-preformatted”>ffmpeg -f lavfi -i ddagrab -c:v h264_nvenc -cq 18 output.mp4 </pre>

You can also skip the lavfi device and directly use the filter. Also demonstrates downloading the frame and encoding with libx264. Explicit output format specification is required in this case: <pre class=”example-preformatted”>ffmpeg -filter_complex ddagrab=output_idx=1:framerate=60,hwdownload,fo -c:v libx264 -crf 18 output.mp4 </pre>

If you want to capture only a subsection of the desktop, this can be achieved by specifying a smaller size and its offsets into the screen: <div

class=”example”> <pre class=”example-preformatted”>ddagrab=video_size=800x600:offset_x=100:offset_y
</pre>

 <h3 class=”section”>39.5 gfxcapture# TOC</h3>

Capture windows or monitors using Windows.Graphics.Capture API.

This source provides low overhead capture of application windows or entire monitors.

The filter outputs hardware frames in <code class=”code”>d3d11</code> format; use <code class=”code”>hwdownload,format=</code>

if system memory frames are required.

The window to be captured can be selected via regular expressions on its title,

class name or backing executable name, by explicit native handles, or by monitor

index or explicit native handle. A window must match all provided expressions to be

selected. The first matching window will be picked, in whatever order Windows

returns them.

Explicit handles (<samp class=”option”>hwnd</samp>, <samp class=”option”>hmonitor</samp>) override pattern or index

based selection. If neither handles nor a monitor index are given, the first

window matching the provided regular expressions is captured.

This source does NOT hold a stable FPS. It returns frames at whatever rate the compositor

provides them, only capped by the <samp class=”option”>max_framerate</samp>.

If you need a stable rate, you need to add an fps filter to drop/duplicate frames as needed.

Note that a static source may not deliver any frames at all, which also stalls

the fps filter; see <samp class=”option”>dup_frames</samp>.

If the capture source disappears mid-capture (window closed, monitor disconnected), the filter will return EOF.

This source accepts the following options:

<dl class="table">

<dt><samp class="option">window_title</samp></dt>

<dd> ECMAScript regular expression matched against the window title. Supports a

PCRE style <code class="code">(?i)</code> prefix for case-insensitive matching.

</dd>

<dt><samp class="option">window_class</samp></dt>

<dd> As <samp class="option">window_title</samp>, but matched against the window class name.

</dd>

<dt><samp class="option">window_exe</samp></dt>

<dd> As <samp class="option">window_title</samp>, but matched against the executable file name of the

window’s process.

</dd>

<dt><samp class="option">monitor_idx</samp></dt>

<dd> Zero-based index of the monitor to capture.

Can also be set to <code class="code">window</code> to capture the monitor the selected window

is displayed on at filter initialization time.

</dd>

<dt><samp class="option">hwnd</samp></dt>

<dd> Explicit native window handle (HWND).

</dd>

<dt><samp class="option">hmonitor</samp></dt>

<dd> Explicit native monitor handle (HMONITOR).

</dd>

<dt><samp class="option">capture_cursor</samp></dt>

<dd> Capture the mouse cursor. Enabled by default.

</dd>

<dt><samp class="option">capture_border</samp></dt>

<dd> Capture the full area of the window, including its window decorarions/border.

Disabled by default.

</dd>

<dt><samp class="option">display_border</samp></dt>

<dd> Draw a yellow highlight border around the captured window. Disabled by default.

</dd>

<dt><samp class="option">max_framerate</samp></dt>

<dd> Maximum capture frame rate. Accepts a video rate (e.g. <code class="code">30</code>, <code class="code">60/1</code>,

<code class="code">24000/1001</code>). The default is <code class="code">60</code> FPS.

The actual rate is the rate at which the compositor renders the window/monitor,

capped by this option.

</dd>

<dt><samp class="option">width</samp></dt>

<dd> Force the output canvas width. If zero (default) the initial captured source

width is used. If a negative number is provided, the width will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">height</samp></dt>

<dd> Force the output canvas height. If zero (default) the initial captured source

height is used. If a negative number is provided, the height will be rounded

down to the next multiple of that number.

See <samp class="option">resize_mode</samp>.

</dd>

<dt><samp class="option">crop_left</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_top</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_right</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class="option">crop_bottom</samp></dt>

<dd> Crop this many pixels from the left side of the captured frames.

</dd>

<dt><samp class=”option”>premultiplied</samp></dt>

<dd> If set to 1, return frames with premultiplied alpha. Default is 0 (straight

alpha).

</dd>

<dt><samp class=”option”>dup_frames</samp></dt>

<dd> If enabled (the default), the last frame is output again whenever the source

delivered no new frame within one <samp class=”option”>max_framerate</samp> interval, and black

frames are output until the source delivered its first frame. This keeps the

output flowing while the captured content is static. The output rate is still

not constant, add an <code class=”code”>fps</code> filter if needed.

</dd>

<dt><samp class=”option”>resize_mode</samp></dt>

<dd> Defines how the captured content is fitted into the output canvas size.

Possible values:

<dl class=”table”>

<dt>‘<samp class=”samp”>crop</samp>’</dt>

<dd> Crop (or pad with black) to the canvas size. (default)

</dd>

<dt>‘<samp class=”samp”>scale</samp>’</dt>

<dd> Scale the source to fill the canvas, potentially altering aspect ratio.

</dd>

<dt>‘<samp class=”samp”>scale_aspect</samp>’</dt>

<dd> Scale the source to fit inside the canvas while preserving aspect ratio.

Remaining area is filled with black.

</dd>

</dl>

</dd> <dt><samp class="option">scale_mode</samp></dt> <dd> Scaling algorithm used when resizing is required. Possible values: <dl class="table"> <dt>‘<samp class="samp">point</samp>’</dt> <dd> Nearest neighbour (pixelated) scaling. </dd> <dt>‘<samp class="samp">bilinear</samp>’</dt> <dd> Bilinear filtering. (default) </dd> <dt>‘<samp class="samp">bicubic</samp>’</dt> <dd> Bicubic filtering. Potentially more blurry, but fewer scaling artifacts depending on contents. </dd> </dl>

</dd> <dt><samp class="option">output_fmt</samp></dt> <dd> Desired output pixel format inside the D3D11 hardware frames. Possible values: <dl class="table"> <dt>‘<samp class="samp">bgra</samp>’</dt> <dt>‘<samp class="samp">8bit</samp>’</dt> <dd> 8 bit BGRA output (default) </dd> <dt>‘<samp class="samp">x2bgr10</samp>’</dt> <dt>‘<samp class="samp">10bit</samp>’</dt> <dd> 10 bit BGR output </dd> <dt>‘<samp class="samp">rgbaf16</samp>’</dt> <dt>‘<samp class="samp">16bit</samp>’</dt> <dd> 16bit float RGBA output </dd> </dl>

</dd> </dl>

 <h4 class="subsection">39.5.1 Examples# TOC</h4> <ul class="itemize mark-bullet"> Capture a window by title (case-insensitive) at a maximum of 60 fps: <pre class="example-preformatted">ffmpeg -filter_complex gfxcapture=window_title='(?i)My Application':max_framerate=60,hwdownload,format=bgra,format=yuv420p -c:v libx264 -crf 15 capture.mp4 </pre>

Capture monitor 1 at native refresh, 10bit color depth, scale to 1920x1080 preserving aspect: <pre class="example-preformatted">ffmpeg -filter_complex gfxcapture=monitor_idx=1:width=1920:height=1080:resize_mode=sc -c:v hevc_nvenc -cq 15 capture.mp4 </pre>

Capture a window by executable name, draw border, force point scaling, fixed 60 fps: <pre class="example-preformatted">ffmpeg -filter_complex gfxcapture=window_exe='^firefox.exe$':display_border=1:scale_mod -rc qvbr -qvbr_quality_level 15 -c:v h264_amf capture.mp4 </pre>

 <h3 class="section">39.6 gradients# TOC</h3> Generate several gradients. <dl class="table"> <dt><samp class="option">size,

s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">c0, c1, c2, c3, c4, c5, c6, c7</samp></dt> <dd> Set 8 colors. Default values for colors is to pick random one. </dd> <dt><samp class="option">x0, y0, x1, y1</samp></dt> <dd> Set gradient line source and destination points. If negative or out of range, random ones are picked. </dd> <dt><samp class="option">nb_colors, n</samp></dt> <dd> Set number of colors to use at once. Allowed range is from 2 to 8. Default value is 2. </dd> <dt><samp class="option">seed</samp></dt> <dd> Set seed for picking gradient line points. </dd> <dt><samp class="option">duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#time-duration-syntax">the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class="option">speed</samp></dt> <dd> Set speed of gradients rotation. </dd> <dt><samp class="option">type, t</samp></dt> <dd> Set type of gradients. Available values are: <dl class="table"> <dt>‘<samp class="samp">linear</samp>’</dt> <dt>‘<samp class="samp">radial</samp>’</dt> <dt>‘<samp class="samp">circular</samp>’</dt> <dt>‘<samp class="samp">spiral</samp>’</dt> <dt>‘<samp class="samp">square</samp>’</dt> </dl>

Default type is <var class="var">linear</var>. </dd> </dl>

 <h4 class="subsection">39.6.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class="section">39.7 mandelbrot# TOC</h3>

Generate a Mandelbrot set fractal, and progressively zoom towards the point specified with <var class="var">start_x</var> and <var class="var">start_y</var>. This source accepts the following options: <dl class="table"> <dt><samp class="option">end_pts</samp></dt> <dd> Set the terminal pts value. Default value is 400. </dd> <dt><samp class="option">end_scale</samp></dt> <dd> Set the

terminal scale value. Must be a floating point value. Default value is 0.3. </dd> <dt><samp class="option">inner</samp></dt> <dd> Set the inner coloring mode, that is the algorithm used to draw the Mandelbrot fractal internal region. It shall assume one of the following values: <dl class="table"> <dt><samp class="option">black</samp></dt> <dd> Set black mode. </dd> <dt><samp class="option">convergence</samp></dt> <dd> Show time until convergence. </dd> <dt><samp class="option">mincol</samp></dt> <dd> Set color based on point closest to the origin of the iterations. </dd> <dt><samp class="option">period</samp></dt> <dd> Set period mode. </dd> </dl>

Default value is <var class="var">mincol</var>. </dd> <dt><samp class="option">bailout</samp></dt> <dd> Set the bailout value. Default value is 10.0. </dd> <dt><samp class="option">maxiter</samp></dt> <dd> Set the maximum of iterations performed by the rendering algorithm. Default value is 7189. </dd> <dt><samp class="option">outer</samp></dt> <dd> Set outer coloring mode. It shall assume one of following values: <dl class="table"> <dt><samp class="option">iteration_count</samp></dt> <dd> Set iteration count mode. </dd> <dt><samp class="option">normalized_iteration_count</samp></dt> <dd> set normalized iteration count mode. </dd> </dl> Default value is <var class="var">normalized_iteration_count</var>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class="option">start_scale</samp></dt> <dd> Set the initial scale value. Default value is 3.0. </dd> <dt><samp class="option">start_x</samp></dt> <dd> Set the initial x position. Must be a floating point value between -100 and 100. Default value is -0.743643887037158704752191506114774. </dd> <dt><samp class="option">start_y</samp></dt> <dd> Set the initial y position. Must be a floating point value between -100 and 100. Default value is -0.131825904205311970493132056385139. </dd> </dl>

 <h3 class="section">39.8 mptestsrc# TOC</h3>

Generate various test patterns, as generated by the MPlayer test filter. The size of the generated video is fixed, and is 512x512. This source is useful in particular for testing encoding features. This source accepts the following options: <dl class="table"> <dt><samp

class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>test, t</samp></dt> <dd> Set the number or the name of the test to perform. Supported tests are: <dl class=”table”> <dt><samp class=”option”>dc_luma</samp></dt> <dt><samp class=”option”>dc_chroma</samp></dt> <dt><samp class=”option”>freq_luma</samp></dt> <dt><samp class=”option”>freq_chroma</samp></dt> <dt><samp class=”option”>amp_luma</samp></dt> <dt><samp class=”option”>amp_chroma</samp></dt> <dt><samp class=”option”>cbp</samp></dt> <dt><samp class=”option”>mv</samp></dt> <dt><samp class=”option”>ring1</samp></dt> <dt><samp class=”option”>ring2</samp></dt> <dt><samp class=”option”>all</samp></dt> <dt><samp class=”option”>max_frames, m</samp></dt> <dd> Set the maximum number of frames generated for each test, default value is 30. </dd> </dl>

Default value is "all", which will cycle through the list of all tests. </dd> </dl>

Some examples: <pre class=”example-preformatted”>mptestsrc=t=dc_luma </pre>

will generate a "dc_luma" test pattern. <h3 class=”section”>39.9 frei0r_src# TOC</h3>

Provide a frei0r source. To enable compilation of this filter you need to install the frei0r header and configure FFmpeg with <code class=”code”>--enable-frei0r</code>. This source accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>size</samp></dt> <dd> The size of the video to generate. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. </dd> <dt><samp class=”option”>framerate</samp></dt> <dd> The framerate of the generated video. It may be a string of the form <var class=”var”>num</var>/<var class=”var”>den</var> or a frame rate abbreviation. </dd> <dt><samp class=”option”>filter_name</samp></dt> <dd> The

name to the frei0r source to load. For more information regarding frei0r and how to set the parameters, read the frei0r section in the video filters documentation. </dd> <dt><samp class=”option”>filter_params</samp></dt> <dd> A ’|’-separated list of parameters to pass to the frei0r source. </dd> </dl>

For example, to generate a frei0r partik0l source with size 200x200 and frame rate 10 which is overlaid on the overlay filter main input: <pre class=”example-preformatted”>frei0r_src=size=200x200:framerate=10:filter_name=
[overlay]; [in][overlay] overlay </pre>

 <h3 class=”section”>39.10 life# TOC</h3>

Generate a life pattern.

This source is based on a generalization of John Conway’s life game.

The sourced input represents a life grid, each pixel represents a cell

which can be in one of two possible states, alive or dead. Every cell

interacts with its eight neighbours, which are the cells that are

horizontally, vertically, or diagonally adjacent.

At each interaction the grid evolves according to the adopted rule,

which specifies the number of neighbor alive cells which will make a

cell stay alive or born. The <samp class=”option”>rule</samp> option allows one to specify

the rule to adopt.

This source accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>filename, f</samp></dt>

<dd> Set the file from which to read the initial grid state. In the file,

each non-whitespace character is considered an alive cell, and newline

is used to delimit the end of each row.

If this option is not specified, the initial grid is generated

randomly.

</dd>

<dt><samp class=”option”>rate, r</samp></dt>

<dd> Set the video rate, that is the number of frames generated per second.

Default is 25.

</dd>

<dt><samp class=”option”>random_fill_ratio, ratio</samp></dt>

<dd> Set the random fill ratio for the initial random grid. It is a

floating point number value ranging from 0 to 1, defaults to 1/PHI.

It is ignored when a file is specified.

</dd>

<dt><samp class=”option”>random_seed, seed</samp></dt>

<dd> Set the seed for filling the initial random grid, must be an integer

included between 0 and UINT32_MAX. If not specified, or if explicitly

set to -1, the filter will try to use a good random seed on a best

effort basis.

</dd>

<dt><samp class=”option”>rule</samp></dt>

<dd> Set the life rule.

A rule can be specified with a code of the kind "S<var class=”var”>NS</var>/B<var class=”var”>NB</var>",

where <var class=”var”>NS</var> and <var class=”var”>NB</var> are sequences of numbers in the range 0-8,

<var class="var">NS</var> specifies the number of alive neighbor cells which make a

live cell stay alive, and <var class="var">NB</var> the number of alive neighbor cells

which make a dead cell to become alive (i.e. to "born").

"s" and "b" can be used in place of "S" and "B", respectively.

Alternatively a rule can be specified by an 18-bits integer. The 9

high order bits are used to encode the next cell state if it is alive

for each number of neighbor alive cells, the low order bits specify

the rule for "borning" new cells. Higher order bits encode for an

higher number of neighbor cells.

For example the number 6153 = <code class="code">(12<<9)+9</code> specifies a stay alive

rule of 12 and a born rule of 9, which corresponds to "S23/B03".

Default value is "S23/B3", which is the original Conway’s game of life

rule, and will keep a cell alive if it has 2 or 3 neighbor alive

cells, and will born a new cell if there are three alive cells around

a dead cell.

</dd>

<dt><samp class="option">size, s</samp></dt>

<dd> Set the size of the output video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

If <samp class="option">filename</samp> is specified, the size is set by default to the

same size of the input file. If <samp class="option">size</samp> is set, it must contain

the size specified in the input file, and the initial grid defined in

that file is centered in the larger resulting area.

If a filename is not specified, the size value defaults to "320x240"

(used for a randomly generated initial grid).

</dd>

<dt><samp class="option">stitch</samp></dt>

<dd> If set to 1, stitch the left and right grid edges together, and the

top and bottom edges also. Defaults to 1.

</dd>

<dt><samp class="option">mold</samp></dt>

<dd> Set cell mold speed. If set, a dead cell will go from <samp class="option">death_color</samp> to

<samp class="option">mold_color</samp> with a step of <samp class="option">mold</samp>. <samp class="option">mold</samp> can have a

value from 0 to 255.

</dd>

<dt><samp class="option">life_color</samp></dt>

<dd> Set the color of living (or new born) cells.

</dd>

<dt><samp class="option">death_color</samp></dt>

<dd> Set the color of dead cells. If <samp class="option">mold</samp> is set, this is the first color

used to represent a dead cell.

</dd>

<dt><samp class="option">mold_color</samp></dt>

<dd> Set mold color, for definitely dead and moldy cells.

For the syntax of these 3 color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the

ffmpeg-utils manual.

</dd>

</dl>

 <h4 class="subsection">39.10.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Read a grid from <samp class="file">pattern</samp>, and center it on a grid of size 300x300 pixels: <pre class="example-preformatted">life=f=pattern:s=300x300 </pre>

Generate a random grid of size 200x200, with a fill ratio of 2/3: <pre class="example-preformatted">life=ratio=2/3:s=200x200 </pre>

Specify a custom rule for evolving a randomly generated grid: <pre class="example-preformatted">life=rule=S14/B34 </pre>

Full example with slow death effect (mold) using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi life=s=300x200:mold=10:r=60:ratio=0.1:death_color=#C83232:life_color=#00f
</pre>

 <h3 class="section">39.11 perlin# TOC</h3> Generate Perlin noise. Perlin noise is a kind of noise with local continuity in space. This can be used to generate patterns with continuity in space and time, e.g. to simulate smoke, fluids, or terrain. In case more than one octave is specified through the <samp class="option">octaves</samp> option, Perlin noise is generated as a sum of components, each one with doubled frequency. In this case the <samp class="option">persistence</samp> option specify the ratio of the amplitude with respect to the previous component. More octave components enable to specify more high frequency details in the generated noise (e.g. small size variations due to boulders in a generated terrain). <h4 class="subsection">39.11.1 Options<span

class=”pull-right”># TOC</h4> <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Specify the size (width and height) of the buffered video frames. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>320x240</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate expected for the video stream, expressed as a number of frames per second. Default value is <code class=”code”>25</code>. </dd> <dt><samp class=”option”>octaves</samp></dt> <dd> Specify the total number of components making up the noise, each one with doubled frequency. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>persistence</samp></dt> <dd> Set the ratio used to compute the amplitude of the next octave component with respect to the previous component amplitude. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>xscale</samp></dt> <dt><samp class=”option”>yscale</samp></dt> <dd> Define a scale factor used to multiple the x, y coordinates. This can be useful to define an effect with a pattern stretched along the x or y axis. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>tscale</samp></dt> <dd> Define a scale factor used to multiple the time coordinate. This can be useful to change the time variation speed. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>random_mode</samp></dt> <dd> Set random mode used to compute initial pattern. Supported values are: <dl class=”table”> <dt><samp class=”option”>random</samp></dt> <dd> Compute and use random seed. </dd> <dt><samp class=”option”>ken</samp></dt> <dd> Use the predefined initial pattern defined by Ken Perlin in the original article, can be useful to compare the output with other sources. </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Use the value specified by <samp class=”option”>random_seed</samp> option. </dd> </dl>

Default value is <code class=”code”>random</code>. </dd> <dt><samp class=”option”>random_seed, seed</samp></dt> <dd> When <samp class=”option”>random_mode</samp> is set to <var class=”var”>random_seed</var>, use this value to compute the initial pattern. Default value is <code class=”code”>0</code>. </dd> </dl>

 <h4 class=”subsection”>39.11.2 Examples# TOC</h4> <ul class=”itemize mark-bullet”> Generate single component: <pre class=”example-preformatted”>perlin </pre>

Use Perlin noise with 7 components, each one with a halved contribution to total amplitude: <pre class=”example-preformatted”>perlin=octaves=7:persistence=0.5 </pre>

Chain Perlin noise with the lutyuv to generate a black&white effect: <pre class=”example-preformatted”>perlin=octaves=3:tscale=0.3,lutyuv=y='if(lt(val\,128)\,255\,0)' </pre>

Stretch noise along the y axis, and convert gray level to red-only signal: <pre class=”example-preformatted”>perlin=octaves=7:tscale=0.4:yscale=0.3,lut </pre>

 <h3 class=”section”>39.12 qrencodesrc# TOC</h3>

Generate a QR code using the libqrencode library (see https://fukuchi.org/works/qrencode/). To enable the compilation of this source, you need to configure FFmpeg with <code class=”code”>--enable-libqrencode</code>. The QR code is generated from the provided text or text pattern. The corresponding QR code is scaled and put in the video output according to the specified output size options. In case no text is specified, the QR code is not generated, but an empty colored output is returned instead. This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>qrcode_width, q</samp></dt> <dt><samp class=”option”>padded_qrcode_width, Q</samp></dt> <dd> Specify an expression for the width of the rendered QR code, with and without padding. The <var class=”var”>qrcode_width</var> expression can reference the value set by the <var class=”var”>padded_qrcode_width</var> expression, and vice versa. By default <var class=”var”>padded_qrcode_width</var> is set to <var class=”var”>qrcode_width</var>, meaning that there is no padding. These expressions are evaluated only once, when initializing the source. See the qrencode Expressions section for details. Note that some of the constants are missing for the source (for example the <var class=”var”>x</var> or <var class=”var”>t</var> or ¸<var class=”var”>n</var>), since they only makes sense when evaluating the expression for each frame rather than at initialization time. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Specify the frame rate of the sourced video, as the number of frames generated per second. It has to be a string in the format <var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point number or a valid video frame rate abbreviation. The default value is "25". </dd> <dt><samp class=”option”>case_sensitive, cs</samp></dt> <dd> Instruct libqrencode to use case sensitive encoding. This is enabled by default. This can

be disabled to reduce the QR encoding size. </dd> <dt><samp class="option">level, l</samp></dt> <dd> Specify the QR encoding error correction level. With an higher correction level, the encoding size will increase but the code will be more robust to corruption. Lower level is <var class="var">L</var>. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">L</samp>’</dt> <dt>‘<samp class="samp">M</samp>’</dt> <dt>‘<samp class="samp">Q</samp>’</dt> <dt>‘<samp class="samp">H</samp>’</dt> </dl>

</dd> <dt><samp class="option">expansion</samp></dt> <dd> Select how the input text is expanded. Can be either <code class="code">none</code>, or <code class="code">normal</code> (default). See the qrencode Text expansion section for details. </dd> <dt><samp class="option">text</samp></dt> <dt><samp class="option">textfile</samp></dt> <dd> Define the text to be rendered. In case neither is specified, no QR is encoded (just an empty colored frame). In case expansion is enabled, the text is treated as a text template, using the qrencode expansion mechanism. See the qrencode Text expansion section for details. </dd> <dt><samp class="option">background_color, bc</samp></dt> <dt><samp class="option">foreground_color, fc</samp></dt> <dd> Set the QR code and background color. The default value of <var class="var">foreground_color</var> is "black", the default value of <var class="var">background_color</var> is "white". For the syntax of the color options, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#color-syntax">"Color" section in the ffmpeg-utils manual. </dd> </dl>

 <h4 class="subsection">39.12.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate a QR code encoding the specified text with the default size: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org </pre>

Same as below, but select blue on pink colors: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:bc=pink:fc=blue </pre>

Generate a QR code with width of 200 pixels and padding, making the padded width 4/3 of the QR code width: <pre class="example-preformatted">qrencodesrc=text=www.ffmpeg.org:q=200:Q=4/3*q </pre>

Generate a QR code with padded width of 200 pixels and padding,

making the QR code width 3/4 of the padded width: <pre class=”example-preformatted”>qrencodesrc=text=www.ffmpeg.org:Q=200:q=3/4*Q </pre>

Generate a QR code encoding the frame number: <pre class=”example-preformatted”>qrencodesrc=text=%{n} </pre>

Generate a QR code encoding the GMT timestamp: <pre class=”example-preformatted”>qrencodesrc=text=%{gmtime} </pre>

Generate a QR code encoding the timestamp expressed as a float: <pre class=”example-preformatted”>qrencodesrc=text=%{pts} </pre>

 <h3 class=”section”>39.13 allrgb, allyuv, color, colorchart, colorspectrum, haldclutsrc, nullsrc, pal75bars, pal100bars, rgbtestsrc, smptebars, smptehdbars, testsrc, testsrc2, yuvtestsrc# TOC</h3>

The <code class=”code”>allrgb</code> source returns frames of size 4096x4096 of all rgb colors.

The <code class=”code”>allyuv</code> source returns frames of size 4096x4096 of all yuv colors.

The <code class=”code”>color</code> source provides an uniformly colored input.

The <code class=”code”>colorchart</code> source provides a colors checker chart.

The <code class=”code”>colorspectrum</code> source provides a color spectrum input.

The <code class=”code”>haldclutsrc</code> source provides an identity Hald CLUT. See also

haldclut filter.

The <code class=”code”>nullsrc</code> source returns unprocessed video frames. It is

mainly useful to be employed in analysis / debugging tools, or as the

source for filters which ignore the input data.

The <code class=”code”>pal75bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 75% color levels.

The <code class=”code”>pal100bars</code> source generates a color bars pattern, based on

EBU PAL recommendations with 100% color levels.

The <code class=”code”>rgbtestsrc</code> source generates an RGB test pattern useful for

detecting RGB vs BGR issues. You should see a red, green and blue

stripe from top to bottom.

The <code class=”code”>smptebars</code> source generates a color bars pattern, based on

the SMPTE Engineering Guideline EG 1-1990.

The <code class=”code”>smptehdbars</code> source generates a color bars pattern, based on

the SMPTE RP 219-2002.

The <code class=”code”>testsrc</code> source generates a test video pattern, showing a

color pattern, a scrolling gradient and a timestamp. This is mainly

intended for testing purposes.

The <code class=”code”>testsrc2</code> source is similar to testsrc, but supports more

pixel formats instead of just <code class=”code”>rgb24</code>. This allows using it as an

input for other tests without requiring a format conversion.

The <code class=”code”>yuvtestsrc</code> source generates an YUV test pattern. You should

see a y, cb and cr stripe from top to bottom.

The sources accept the following parameters:

<dl class=”table”>

<dt><samp class=”option”>level</samp></dt>

<dd> Specify the level of the Hald CLUT, only available in the <code class=”code”>haldclutsrc</code>

source. A level of <code class=”code”>N</code> generates a picture of <code class=”code”>N*N*N</code> by <code class=”code”>N*N*N</code>

pixels to be used as identity matrix for 3D lookup tables. Each component is

coded on a <code class=”code”>1/(N*N)</code> scale.

</dd>

<dt><samp class=”option”>color, c</samp></dt>

<dd> Specify the color of the source, only available in the <code class=”code”>color</code>

source. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#color-syntax”>"Color" section in the ffmpeg-utils manual.

</dd>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Specify the size of the sourced video. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

The default value is <code class=”code”>320x240</code>.

This option is not available with the <code class=”code”>allrgb</code>, <code class=”code”>allyuv</code>, and

<code class=”code”>haldclutsrc</code> filters.

</dd>

<dt><samp class=”option”>rate, r</samp></dt>

<dd> Specify the frame rate of the sourced video, as the number of frames

generated per second. It has to be a string in the format

<var class=”var”>frame_rate_num</var>/<var class=”var”>frame_rate_den</var>, an integer number, a floating point

number or a valid video frame rate abbreviation. The default value is

"25".

</dd>

<dt><samp class=”option”>duration, d</samp></dt>

<dd> Set the duration of the sourced video. See

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual

for the accepted syntax.

If not specified, or the expressed duration is negative, the video is

supposed to be generated forever.

Since the frame rate is used as time base, all frames including the last one

will have their full duration. If the specified duration is not a multiple

of the frame duration, it will be rounded up.

</dd>

<dt><samp class="option">sar</samp></dt>

<dd> Set the sample aspect ratio of the sourced video.

</dd>

<dt><samp class="option">alpha</samp></dt>

<dd> Specify the alpha (opacity) of the background, only available in the

<code class="code">testsrc2</code> source. The value must be between 0 (fully transparent) and

255 (fully opaque, the default).

</dd>

<dt><samp class="option">decimals, n</samp></dt>

<dd> Set the number of decimals to show in the timestamp, only available in the

<code class="code">testsrc</code> source.

The displayed timestamp value will correspond to the original

timestamp value multiplied by the power of 10 of the specified

value. Default value is 0.

</dd>

<dt><samp class="option">type</samp></dt>

<dd> Set the type of the color spectrum, only available in the

<code class=”code”>colorspectrum</code> source. Can be one of the following:

<dl class=”table”>

<dt>‘<samp class=”samp”>black</samp>’</dt>

<dt>‘<samp class=”samp”>white</samp>’</dt>

<dt>‘<samp class=”samp”>all</samp>’</dt>

</dl>

</dd> <dt><samp class=”option”>patch_size</samp></dt> <dd> Set patch size of single color patch, only available in the <code class=”code”>colorchart</code> source. Default is <code class=”code”>64x64</code>. </dd> <dt><samp class=”option”>preset</samp></dt> <dd> Set colorchecker colors preset, only available in the <code class=”code”>colorchart</code> source. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>reference</samp>’</dt> <dt>‘<samp class=”samp”>skintones</samp>’</dt> </dl>

Default value is <code class=”code”>reference</code>. </dd> </dl>

 <h4 class=”subsection”>39.13.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Generate a video with a duration of 5.3 seconds, with size 176x144 and a frame rate of 10 frames per second: <pre class=”example-preformatted”>testsrc=duration=5.3:size=qcif:rate=10 </pre>

The following graph description will generate a red source with an opacity of 0.2, with size "qcif" and a frame rate of 10 frames per second: <pre class=”example-preformatted”>color=c=red@0.2:s=qcif:r=10 </pre>

If the input content is to be ignored, <code class=”code”>nullsrc</code> can be used. The following command generates noise in the luma plane by employing the <code class=”code”>geq</code> filter: <pre class=”example-preformatted”>nullsrc=s=256x256, geq=random(1)*255:128:128 </pre>

 <h4 class=”subsection”>39.13.2 Commands<a class=”anchor hidden-xs” href=”#Commands-

158" aria-hidden="true"># TOC</h4>

The <code class="code">color</code> source supports the following commands: <dl class="table"> <dt><samp class="option">c, color</samp></dt> <dd> Set the color of the created image. Accepts the same syntax of the corresponding <samp class="option">color</samp> option. </dd> </dl>

 <h3 class="section">39.14 openclsrc# TOC</h3>

Generate video using an OpenCL program. <dl class="table"> <dt><samp class="option">source</samp></dt> <dd> OpenCL program source file. </dd> <dt><samp class="option">kernel</samp></dt> <dd> Kernel name in program. </dd> <dt><samp class="option">size, s</samp></dt> <dd> Size of frames to generate. This must be set. </dd> <dt><samp class="option">format</samp></dt> <dd> Pixel format to use for the generated frames. This must be set. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Number of frames generated every second. Default value is ’25’. </dd> </dl>

For details of how the program loading works, see the program_opencl filter. Example programs: <ul class="itemize mark-bullet"> Generate a colour ramp by setting pixel values from the position of the pixel in the output image. (Note that this will work with all pixel formats, but the generated output will not be the same.) <pre class="verbatim">__kernel void ramp(__write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 val; val.xy = val.zw = convert_float2(loc) / convert_float2(get_image_dim(dst));

write_imagef(dst, loc, val); } </pre> Generate a Sierpinski carpet pattern, panning by a single pixel each frame. <pre class="verbatim">__kernel void sierpinski_carpet(__write_only image2d_t dst, unsigned int index) { int2 loc = (int2)(get_global_id(0), get_global_id(1));

float4 value = 0.0f; int x = loc.x + index; int y = loc.y + index; while (x > 0 || y > 0) { if (x % 3 == 1 && y % 3 == 1) { value = 1.0f; break; } x /= 3; y /= 3; }

write_imagef(dst, loc, value); } </pre>

 <h3 class="section">39.15 sierpinski# <a class="anchor hidden-xs"href="#toc-sierpinski"

aria-hidden=”true”>TOC</h3>

Generate a Sierpinski carpet/triangle fractal, and randomly pan around. This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "640x480". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>seed</samp></dt> <dd> Set seed which is used for random panning. </dd> <dt><samp class=”option”>jump</samp></dt> <dd> Set max jump for single pan destination. Allowed range is from 1 to 10000. </dd> <dt><samp class=”option”>type</samp></dt> <dd> Set fractal type, can be default <code class=”code”>carpet</code> or <code class=”code”>triangle</code>. </dd> </dl>

 <h3 class=”section”>39.16 zoneplate# TOC</h3> Generate a zoneplate test video pattern. This source accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set frame size. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is "320x240". </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set the duration of the sourced video. See <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#time-duration-syntax”>the Time duration section in the ffmpeg-utils(1) manual for the accepted syntax. If not specified, or the expressed duration is negative, the video is supposed to be generated forever. </dd> <dt><samp class=”option”>sar</samp></dt> <dd> Set the sample aspect ratio of the sourced video. </dd> <dt><samp class=”option”>precision</samp></dt> <dd> Set precision in bits for look-up table for sine calculations. Default value is 10. Allowed range is from 4 to 16. </dd> <dt><samp class=”option”>xo</samp></dt> <dd> Set horizontal axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>yo</samp></dt> <dd> Set vertical axis offset for output signal. Default value is 0. </dd> <dt><samp class=”option”>to</samp></dt> <dd> Set time axis offset for output signal. Default value is 0. </dd>

<dt><samp class="option">k0</samp></dt> <dd> Set 0-order, constant added to signal phase. Default value is 0. </dd> <dt><samp class="option">kx</samp></dt> <dd> Set 1-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class="option">ky</samp></dt> <dd> Set 1-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class="option">kt</samp></dt> <dd> Set 1-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class="option">kxt, kyt, kxy</samp></dt> <dd> Set phase factor multipliers for combination of spatial and temporal axis. Default value is 0. </dd> <dt><samp class="option">kx2</samp></dt> <dd> Set 2-order, phase factor multiplier for horizontal axis. Default value is 0. </dd> <dt><samp class="option">ky2</samp></dt> <dd> Set 2-order, phase factor multiplier for vertical axis. Default value is 0. </dd> <dt><samp class="option">kt2</samp></dt> <dd> Set 2-order, phase factor multiplier for time axis. Default value is 0. </dd> <dt><samp class="option">ku</samp></dt> <dd> Set the constant added to final phase to produce chroma-blue component of signal. Default value is 0. </dd> <dt><samp class="option">kv</samp></dt> <dd> Set the constant added to final phase to produce chroma-red component of signal. Default value is 0. </dd> </dl>

 <h4 class="subsection">39.16.1 Commands# TOC</h4>

This source supports the some above options as commands. <h4 class="subsection">39.16.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Generate horizontal color sine sweep: <pre class="example-preformatted">zoneplate=ku=512:kv=0:kt2=0:kx2=256:s=wvg
426:kt=11 </pre>

Generate vertical color sine sweep: <pre class="example-preformatted">zoneplate=ku=512:kv=0:kt2=0:ky2=156:s=wvga:yo=-240:kt=11 </pre>

Generate circular zone-plate: <pre class="example-preformatted">zoneplate=ku=512:kv=100:kt2=0:ky2=256:kx2=556:s=wvga:yo=0:kt=11 </pre>

 <h2 class="chapter">40 Video Sinks# TOC</h2>

Below is a description of the currently available video sinks. <h3 class="section">40.1 buffersink# TOC</h3>

Buffer video frames, and make them available to the end of the filter graph. This sink is mainly intended for programmatic use, in particular through the interface defined in <samp class="file">libavfilter/buffersink.h</samp> or the options system. It accepts a pointer to an AVBufferSinkContext structure, which defines the incoming buffers’ formats, to be passed as the opaque parameter to <code class="code">avfilter_init_filter</code> for initialization. <h3 class="section">40.2 nullsink# TOC</h3>

Null video sink: do absolutely nothing with the input video. It is mainly useful as a template and for use in analysis / debugging tools.

 <h2 class="chapter">41 Multimedia Filters# TOC</h2>

Below is a description of the currently available multimedia filters. <h3 class="section">41.1 a3dscope# TOC</h3>

Convert input audio to 3d scope video output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">hd720</code>. </dd> <dt><samp class="option">fov</samp></dt> <dd> Set the camera field of view. Default is 90 degrees. Allowed range is from 40 to 150. </dd> <dt><samp class="option">roll</samp></dt> <dd> Set the camera roll. </dd> <dt><samp class="option">pitch</samp></dt> <dd> Set the camera pitch. </dd> <dt><samp class="option">yaw</samp></dt> <dd> Set the camera yaw. </dd> <dt><samp class="option">xzoom</samp></dt> <dd> Set the camera zoom on X-axis. </dd> <dt><samp class="option">yzoom</samp></dt> <dd> Set the camera zoom on

Y-axis. </dd> <dt><samp class="option">zzoom</samp></dt> <dd> Set the camera zoom on Z-axis. </dd> <dt><samp class="option">xpos</samp></dt> <dd> Set the camera position on X-axis. </dd> <dt><samp class="option">ypos</samp></dt> <dd> Set the camera position on Y-axis. </dd> <dt><samp class="option">zpos</samp></dt> <dd> Set the camera position on Z-axis. </dd> <dt><samp class="option">length</samp></dt> <dd> Set the length of displayed audio waves in number of frames. </dd> </dl>

 <h4 class="subsection">41.1.1 Commands# TOC</h4>

Filter supports the some above options as commands. <h3 class="section">41.2 abitscope# TOC</h3>

Convert input audio to a video output, displaying the audio bit scope. The filter accepts the following options: <dl class="table"> <dt><samp class="option">rate, r</samp></dt> <dd> Set frame rate, expressed as number of frames per second. Default value is "25". </dd> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">1024x256</code>. </dd> <dt><samp class="option">colors</samp></dt> <dd> Specify list of colors separated by space or by ’|’ which will be used to draw channels. Unrecognized or missing colors will be replaced by white color. </dd> <dt><samp class="option">mode, m</samp></dt> <dd> Set output mode. Can be <code class="code">bars</code> or <code class="code">trace</code>. Default is <code class="code">bars</code>. </dd> </dl>

 <h3 class="section">41.3 adrawgraph# TOC</h3> Draw a graph using input audio metadata. See drawgraph <h3 class="section">41.4 agraphmonitor# TOC</h3>

See graphmonitor. <h3 class=”section”>41.5 ahistogram# TOC</h3>

Convert input audio to a video output, displaying the volume histogram.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>dmode</samp></dt>

<dd> Specify how histogram is calculated.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>single</samp>’</dt>

<dd> Use single histogram for all channels.

</dd>

<dt>‘<samp class=”samp”>separate</samp>’</dt>

<dd> Use separate histogram for each channel.

</dd>

</dl>

Default is <code class=”code”>single</code>.

</dd>

<dt><samp class=”option”>rate, r</samp></dt>

<dd> Set frame rate, expressed as number of frames per second. Default value is "25".

</dd>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

Default value is <code class=”code”>hd720</code>.

</dd>

<dt><samp class=”option”>scale</samp></dt>

<dd> Set display scale.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>log</samp>’</dt>

<dd> logarithmic

</dd>

<dt>‘<samp class=”samp”>sqrt</samp>’</dt>

<dd> square root

</dd>

<dt>‘<samp class=”samp”>cbrt</samp>’</dt>

<dd> cubic root

</dd>

<dt>‘<samp class=”samp”>lin</samp>’</dt>

<dd> linear

</dd>

<dt>‘<samp class=”samp”>rlog</samp>’</dt>

<dd> reverse logarithmic

</dd>

</dl>

Default is <code class=”code”>log</code>.

</dd>

<dt><samp class="option">ascale</samp></dt>
<dd> Set amplitude scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> logarithmic
</dd>
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> linear
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">acount</samp></dt>
<dd> Set how much frames to accumulate in histogram.
Default is 1. Setting this to -1 accumulates all frames.

</dd>
<dt><samp class="option">rheight</samp></dt>
<dd> Set histogram ratio of window height.

</dd>
<dt><samp class="option">slide</samp></dt>
<dd> Set sonogram sliding.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">replace</samp>’</dt>
<dd> replace old rows with new ones.

</dd>

<dt>‘<samp class="samp">scroll</samp>’</dt>

<dd> scroll from top to bottom.

</dd>

</dl>

Default is <code class="code">replace</code>.

</dd>

<dt><samp class="option">hmode</samp></dt>

<dd> Set histogram mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">abs</samp>’</dt>

<dd> Use absolute values of samples.

</dd>

<dt>‘<samp class="samp">sign</samp>’</dt>

<dd> Use untouched values of samples.

</dd>

</dl>

Default is <code class="code">abs</code>.

</dd>

</dl>

 <h3 class="section">41.6 aphasemeter# TOC</h3>

Measures phase of input audio, which is exported as metadata <code class="code">lavfi.aphasemeter.phase</code>, representing mean phase of current audio frame. A video output can also be produced and is enabled by default. The audio is passed through as first output. Audio will be rematrixed to stereo if it has a different channel layout. Phase value is in range <code class="code">[-1, 1]</code> where <code class="code">-1</code> means left and right channels are completely

out of phase and <code class=”code”>1</code> means channels are in phase. The filter accepts the following options, all related to its video output: <dl class=”table”> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class=”code”>25</code>. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>800x400</code>. </dd> <dt><samp class=”option”>rc</samp></dt> <dt><samp class=”option”>gc</samp></dt> <dt><samp class=”option”>bc</samp></dt> <dd> Specify the red, green, blue contrast. Default values are <code class=”code”>2</code>, <code class=”code”>7</code> and <code class=”code”>1</code>. Allowed range is <code class=”code”>[0, 255]</code>. </dd> <dt><samp class=”option”>mpc</samp></dt> <dd> Set color which will be used for drawing median phase. If color is <code class=”code”>none</code> which is default, no median phase value will be drawn. </dd> <dt><samp class=”option”>video</samp></dt> <dd> Enable video output. Default is enabled. </dd> </dl>

 <h4 class=”subsection”>41.6.1 phasing detection# TOC</h4>

The filter also detects out of phase and mono sequences in stereo streams. It logs the sequence start, end and duration when it lasts longer or as long as the minimum set. The filter accepts the following options for this detection: <dl class=”table”> <dt><samp class=”option”>phasing</samp></dt> <dd> Enable mono and out of phase detection. Default is disabled. </dd> <dt><samp class=”option”>tolerance, t</samp></dt> <dd> Set phase tolerance for mono detection, in amplitude ratio. Default is <code class=”code”>0</code>. Allowed range is <code class=”code”>[0, 1]</code>. </dd> <dt><samp class=”option”>angle, a</samp></dt> <dd> Set angle threshold for out of phase detection, in degree. Default is <code class=”code”>170</code>. Allowed range is <code class=”code”>[90, 180]</code>. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set mono or out of phase duration until notification, expressed in seconds. Default is <code class=”code”>2</code>. </dd> </dl>

 <h4 class=”subsection”>41.6.2 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Complete example with <code

class=”command”>ffmpeg</code> to detect 1 second of mono with 0.001 phase tolerance: <pre class=”example-preformatted”>ffmpeg -i stereo.wav -af aphasemeter=video=0:phasing=1:duration=1:tolerance=0.001 -f null - </pre>

 <h3 class=”section”>41.7 avectorscope# TOC</h3>

Convert input audio to a video output, representing the audio vector scope. The filter is used to measure the difference between channels of stereo audio stream. A monaural signal, consisting of identical left and right signal, results in straight vertical line. Any stereo separation is visible as a deviation from this line, creating a Lissajous figure. If the straight (or deviation from it) but horizontal line appears this indicates that the left and right channels are out of phase. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>mode, m</samp></dt> <dd> Set the vectorscope mode. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lissajous</samp>’</dt> <dd> Lissajous rotated by 45 degrees. </dd> <dt>‘<samp class=”samp”>lissajous_xy</samp>’</dt> <dd> Same as above but not rotated. </dd> <dt>‘<samp class=”samp”>polar</samp>’</dt> <dd> Shape resembling half of circle. </dd> </dl>

Default value is ‘<samp class=”samp”>lissajous</samp>’. </dd> <dt><samp class=”option”>size, s</samp></dt> <dd> Set the video size for the output. For the syntax of this option, check the <a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual. Default value is <code class=”code”>400x400</code>. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the output frame rate. Default value is <code class=”code”>25</code>. </dd> <dt><samp class=”option”>rc</samp></dt> <dt><samp class=”option”>gc</samp></dt> <dt><samp class=”option”>bc</samp></dt> <dt><samp class=”option”>ac</samp></dt> <dd> Specify the red, green, blue and alpha contrast. Default values are <code class=”code”>40</code>, <code class=”code”>160</code>, <code class=”code”>80</code> and <code class=”code”>255</code>. Allowed range is <code class=”code”>[0, 255]</code>. </dd> <dt><samp class=”option”>rf</samp></dt> <dt><samp class=”option”>gf</samp></dt> <dt><samp class=”option”>bf</samp></dt> <dt><samp class=”option”>af</samp></dt> <dd> Specify the red, green, blue and alpha fade. Default values are <code class=”code”>15</code>, <code class=”code”>10</code>, <code class=”code”>5</code> and <code class=”code”>5</code>. Allowed range is <code class=”code”>[0, 255]</code>. </dd> <dt><samp class=”option”>zoom</samp></dt>

<dd> Set the zoom factor. Default value is <code class="code">1</code>. Allowed range is <code class="code">[0, 10]</code>. Values lower than <var class="var">1</var> will auto adjust zoom factor to maximal possible value. </dd> <dt><samp class="option">draw</samp></dt> <dd> Set the vectorscope drawing mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">dot</samp>’</dt> <dd> Draw dot for each sample. </dd> <dt>‘<samp class="samp">line</samp>’</dt> <dd> Draw line between previous and current sample. </dd> <dt>‘<samp class="samp">aaline</samp>’</dt> <dd> Draw anti-aliased line between previous and current sample. </dd> </dl>

Default value is ‘<samp class="samp">dot</samp>’. </dd> <dt><samp class="option">scale</samp></dt> <dd> Specify amplitude scale of audio samples. Available values are: <dl class="table"> <dt>‘<samp class="samp">lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt> <dd> Cubic root. </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> Logarithmic. </dd> </dl>

</dd> <dt><samp class="option">swap</samp></dt> <dd> Swap left channel axis with right channel axis. </dd> <dt><samp class="option">mirror</samp></dt> <dd> Mirror axis. <dl class="table"> <dt>‘<samp class="samp">none</samp>’</dt> <dd> No mirror. </dd> <dt>‘<samp class="samp">x</samp>’</dt> <dd> Mirror only x axis. </dd> <dt>‘<samp class="samp">y</samp>’</dt> <dd> Mirror only y axis. </dd> <dt>‘<samp class="samp">xy</samp>’</dt> <dd> Mirror both axis. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">41.7.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Complete example using <code class="command">ffplay</code>: <pre class="example-preformatted">ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] avectorscope=zoom=1.3:rc=2:gc=200:bc=10:rf=1:gf=8:bf=7 [out0]' </pre>

 <h4 class="subsection">41.7.2 Commands# TOC</h4>

This filter supports the all above options as commands except options

<code class="code">size</code> and <code class="code">rate</code>. <h3 class="section">41.8 bench, abench# TOC</h3>

Benchmark part of a filtergraph. The filter accepts the following options: <dl class="table"> <dt><samp class="option">action</samp></dt> <dd> Start or stop a timer. Available values are: <dl class="table"> <dt>‘<samp class="samp">start</samp>’</dt> <dd> Get the current time, set it as frame metadata (using the key <code class="code">lavfi.bench.start_time</code>), and forward the frame to the next filter. </dd> <dt>‘<samp class="samp">stop</samp>’</dt> <dd> Get the current time and fetch the <code class="code">lavfi.bench.start_time</code> metadata from the input frame metadata to get the time difference. Time difference, average, maximum and minimum time (respectively <code class="code">t</code>, <code class="code">avg</code>, <code class="code">max</code> and <code class="code">min</code>) are then printed. The timestamps are expressed in seconds. </dd> </dl> </dd> </dl>

 <h4 class="subsection">41.8.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Benchmark selectivecolor filter: <pre class="example-preformatted">bench=start,selectivecolor=reds=-.2 .12 -.49,bench=stop </pre>

 <h3 class="section">41.9 concat# TOC</h3>

Concatenate audio and video streams, joining them together one after the other. The filter works on segments of synchronized video and audio streams. All segments must have the same number of streams of each type, and that will also be the number of streams at output. The filter accepts the following options: <dl class="table"> <dt><samp class="option">n</samp></dt> <dd> Set the number of segments. Default is 2. </dd> <dt><samp class="option">v</samp></dt> <dd> Set the number of output video streams, that is also the number of video streams in each segment. Default is 1. </dd> <dt><samp class="option">a</samp></dt> <dd> Set the number of output audio streams, that is also the number of audio streams in each segment. Default is 0. </dd> <dt><samp class="option">unsafe</samp></dt> <dd> Activate unsafe mode: do not fail if segments have a different format.

 </dd> </dl>

The filter has <var class="var">v</var>+<var class="var">a</var> outputs: first <var class="var">v</var> video outputs, then <var class="var">a</var> audio outputs. There are <var class="var">n</var>x(<var class="var">v</var>+<var class="var">a</var>) inputs: first the inputs for the first segment, in the same order as the outputs, then the inputs for the second segment, etc. Related streams do not always have exactly the same duration, for various reasons including codec frame size or sloppy authoring. For that reason, related synchronized streams (e.g. a video and its audio track) should be concatenated at once. The concat filter will use the duration of the longest stream in each segment (except the last one), and if necessary pad shorter audio streams with silence. For this filter to work correctly, all segments must start at timestamp 0. All corresponding streams must have the same parameters in all segments; the filtering system will automatically select a common pixel format for video streams, and a common sample format, sample rate and channel layout for audio streams, but other settings, such as resolution, must be converted explicitly by the user. Different frame rates are acceptable but will result in variable frame rate at output; be sure to configure the output file to handle it. <h4 class="subsection">41.9.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Concatenate an opening, an episode and an ending, all in bilingual version (video in stream 0, audio in streams 1 and 2): <pre class="example-preformatted">ffmpeg -i opening.mkv -i episode.mkv -i ending.mkv -filter_complex \ '[0:0] [0:1] [0:2] [1:0] [1:1] [1:2] [2:0] [2:1] [2:2] concat=n=3:v=1:a=2 [v] [a1] [a2]' \ -map '[v]' -map '[a1]' -map '[a2]' output.mkv </pre>

Concatenate two parts, handling audio and video separately, using the (a)movie sources, and adjusting the resolution: <pre class="example-preformatted">movie=part1.mp4, scale=512:288 [v1] ; amovie=part1.mp4 [a1] ; movie=part2.mp4, scale=512:288 [v2] ; amovie=part2.mp4 [a2] ; [v1] [v2] concat [outv] ; [a1] [a2] concat=v=0:a=1 [outa] </pre> Note that a desync will happen at the stitch if the audio and video streams do not have exactly the same duration in the first file.

 <h4 class="subsection">41.9.2 Commands# TOC</h4>

This filter supports the following commands: <dl class="table"> <dt><samp class="option">next</samp></dt> <dd> Close the current

segment and step to the next one </dd> </dl>

 <h3 class="section">41.10 ebur128# TOC</h3>

EBU R128 scanner filter. This filter takes an audio stream and analyzes its loudness level. By default, it logs a message at a frequency of 10Hz with the Momentary loudness (identified by <code class="code">M</code>), Short-term loudness (<code class="code">S</code>), Integrated loudness (<code class="code">I</code>) and Loudness Range (<code class="code">LRA</code>). The filter can only analyze streams which have sample format is double-precision floating point. The input stream will be converted to this specification, if needed. Users may need to insert aformat and/or aresample filters after this filter to obtain the original parameters. The filter also has a video output (see the <var class="var">video</var> option) with a real time graph to observe the loudness evolution. The graphic contains the logged message mentioned above, so it is not printed anymore when this option is set, unless the verbose logging is set. The main graphing area contains the short-term loudness (3 seconds of analysis), and the gauge on the right is for the momentary loudness (400 milliseconds), but can optionally be configured to instead display short-term loudness (see <var class="var">gauge</var>). The green area marks a +/- 1LU target range around the target loudness (-23LUFS by default, unless modified through <var class="var">target</var>). More information about the Loudness Recommendation EBU R128 on http://tech.ebu.ch/loudness. The filter accepts the following options: <dl class="table"> <dt><samp class="option">video</samp></dt> <dd> Activate the video output. The audio stream is passed unchanged whether this option is set or no. The video stream will be the first output stream if activated. Default is <code class="code">0</code>. </dd> <dt><samp class="option">size</samp></dt> <dd> Set the video size. This option is for video only. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default and minimum resolution is <code class="code">640x480</code>. </dd> <dt><samp class="option">meter</samp></dt> <dd> Set the EBU scale meter. Default is <code class="code">9</code>. Common values are <code class="code">9</code> and <code class="code">18</code>, respectively for EBU scale meter +9 and EBU scale meter +18. Any other integer value between this range is allowed. </dd> <dt><samp class="option">metadata</samp></dt> <dd> Set metadata injection. If set to <code class="code">1</code>, the audio input will be segmented into 100ms output frames, each of them containing various loudness information in metadata. All the metadata keys are prefixed

with <code class=”code”>lavfi.r128.</code>. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>framelog</samp></dt> <dd> Force the frame logging level. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>quiet</samp>’</dt> <dd> logging disabled </dd> <dt>‘<samp class=”samp”>info</samp>’</dt> <dd> information logging level </dd> <dt>‘<samp class=”samp”>verbose</samp>’</dt> <dd> verbose logging level </dd> </dl>

By default, the logging level is set to <var class=”var”>info</var>. If the <samp class=”option”>video</samp> or the <samp class=”option”>metadata</samp> options are set, it switches to <var class=”var”>verbose</var>. </dd> <dt><samp class=”option”>peak</samp></dt> <dd> Set peak mode(s). Available modes can be cumulated (the option is a <code class=”code”>flag</code> type). Possible values are: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Disable any peak mode (default). </dd> <dt>‘<samp class=”samp”>sample</samp>’</dt> <dd> Enable sample-peak mode. Simple peak mode looking for the higher sample value. It logs a message for sample-peak (identified by <code class=”code”>SPK</code>). </dd> <dt>‘<samp class=”samp”>true</samp>’</dt> <dd> Enable true-peak mode. If enabled, the peak lookup is done on an over-sampled version of the input stream for better peak accuracy. It logs a message for true-peak. (identified by <code class=”code”>TPK</code>) and true-peak per frame (identified by <code class=”code”>FTPK</code>). This mode requires a build with <code class=”code”>libswresample</code>. </dd> </dl>

</dd> <dt><samp class=”option”>dualmono</samp></dt> <dd> Treat mono input files as "dual mono". If a mono file is intended for playback on a stereo system, its EBU R128 measurement will be perceptually incorrect. If set to <code class=”code”>true</code>, this option will compensate for this effect. Multi-channel input files are not affected by this option. </dd> <dt><samp class=”option”>panlaw</samp></dt> <dd> Set a specific pan law to be used for the measurement of dual mono files. This parameter is optional, and has a default value of -3.01dB. </dd> <dt><samp class=”option”>target</samp></dt> <dd> Set a specific target level (in LUFS) used as relative zero in the visualization. This parameter is optional and has a default value of -23LUFS as specified by EBU R128. However, material published online may prefer a level of -16LUFS (e.g. for use with podcasts or video platforms). </dd> <dt><samp class=”option”>gauge</samp></dt> <dd> Set the value displayed by the gauge. Valid values are <code class=”code”>momentary</code> and s <code class=”code”>shortterm</code>. By default the momentary value will be used, but in certain scenarios it may be more useful to observe the short term value instead (e.g. live mixing). </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Sets the display scale for the

loudness. Valid parameters are <code class="code">absolute</code> (in LUFS) or <code class="code">relative</code> (LU) relative to the target. This only affects the video output, not the summary or continuous log output. </dd> <dt><samp class="option">integrated</samp></dt> <dd> Read-only exported value for measured integrated loudness, in LUFS. </dd> <dt><samp class="option">range</samp></dt> <dd> Read-only exported value for measured loudness range, in LU. </dd> <dt><samp class="option">lra_low</samp></dt> <dd> Read-only exported value for measured LRA low, in LUFS. </dd> <dt><samp class="option">lra_high</samp></dt> <dd> Read-only exported value for measured LRA high, in LUFS. </dd> <dt><samp class="option">sample_peak</samp></dt> <dd> Read-only exported value for measured sample peak, in dBFS. </dd> <dt><samp class="option">true_peak</samp></dt> <dd> Read-only exported value for measured true peak, in dBFS. </dd> </dl>

 <h4 class="subsection">41.10.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Real-time graph using <code class="command">ffplay</code>, with a EBU scale meter +18: <pre class="example-preformatted">ffplay -f lavfi -i "amovie=input.mp3,ebur128=video=1:meter=18 [out0][out1]" </pre>

Run an analysis with <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -nostats -i input.mp3 -filter_complex ebur128 -f null - </pre>

 <h3 class="section">41.11 interleave, ainterleave# TOC</h3>

Temporally interleave frames from several inputs. <code class="code">interleave</code> works with video inputs, <code class="code">ainterleave</code> with audio. These filters read frames from several inputs and send the oldest queued frame to the output. Input streams must have well defined, monotonically increasing frame timestamp values. In order to submit one frame to output, these filters need to enqueue at least one frame for each input, so they cannot work in case one input is not yet terminated and will not receive incoming frames. For example consider the case when one input is a <code class="code">select</code> filter which always drops input frames. The <code class="code">interleave</code> filter will keep reading from that input, but it will never be able to send

new frames to output until the input sends an end-of-stream signal. Also, depending on inputs synchronization, the filters will drop frames in case one input receives more frames than the other ones, and the queue is already filled. These filters accept the following options: <dl class="table"> <dt><samp class="option">nb_inputs, n</samp></dt> <dd> Set the number of different inputs, it is 2 by default. </dd> <dt><samp class="option">duration</samp></dt> <dd> How to determine the end-of-stream. <dl class="table"> <dt><samp class="option">longest</samp></dt> <dd> The duration of the longest input. (default) </dd> <dt><samp class="option">shortest</samp></dt> <dd> The duration of the shortest input. </dd> <dt><samp class="option">first</samp></dt> <dd> The duration of the first input. </dd> </dl>

</dd> </dl>

 <h4 class="subsection">41.11.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Interleave frames belonging to different streams using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i bambi.avi -i pr0n.mkv -filter_complex "[0:v][1:v] interleave" out.avi </pre>

Add flickering blur effect: <pre class="example-preformatted">select='if(gt(random(0), 0.2), 1, 2)':n=2 [tmp], boxblur=2:2, [tmp] interleave </pre>

 <h3 class="section">41.12 latency, alatency# TOC</h3>

Measure filtering latency. Report previous filter filtering latency, delay in number of audio samples for audio filters or number of video frames for video filters. On end of input stream, filter will report min and max measured latency for previous running filter in filtergraph.

 <h3 class="section">41.13 metadata, ametadata# TOC</h3>

Manipulate frame metadata. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class="table"> <dt>‘<samp

class=”samp”>select</samp>’</dt> <dd> If both <code class=”code”>value</code> and <code class=”code”>key</code> is set, select frames which have such metadata. If only <code class=”code”>key</code> is set, select every frame that has such key in metadata. </dd> <dt>‘<samp class=”samp”>add</samp>’</dt> <dd> Add new metadata <code class=”code”>key</code> and <code class=”code”>value</code>. If key is already available do nothing. </dd> <dt>‘<samp class=”samp”>modify</samp>’</dt> <dd> Modify value of already present key. </dd> <dt>‘<samp class=”samp”>delete</samp>’</dt> <dd> If <code class=”code”>value</code> is set, delete only keys that have such value. Otherwise, delete key. If <code class=”code”>key</code> is not set, delete all metadata values in the frame. </dd> <dt>‘<samp class=”samp”>print</samp>’</dt> <dd> Print key and its value if metadata was found. If <code class=”code”>key</code> is not set print all metadata values available in frame. </dd> </dl>

</dd> <dt><samp class=”option”>key</samp></dt> <dd> Set key used with all modes. Must be set for all modes except <code class=”code”>print</code> and <code class=”code”>delete</code>. </dd> <dt><samp class=”option”>value</samp></dt> <dd> Set metadata value which will be used. This option is mandatory for <code class=”code”>modify</code> and <code class=”code”>add</code> mode. </dd> <dt><samp class=”option”>function</samp></dt> <dd> Which function to use when comparing metadata value and <code class=”code”>value</code>. Can be one of following: <dl class=”table”> <dt>‘<samp class=”samp”>same_str</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value is same as <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>starts_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value starts with the <code class=”code”>value</code> option string. </dd> <dt>‘<samp class=”samp”>less</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is less than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>equal</samp>’</dt> <dd> Values are interpreted as floats, returns true if <code class=”code”>value</code> is equal with metadata value. </dd> <dt>‘<samp class=”samp”>greater</samp>’</dt> <dd> Values are interpreted as floats, returns true if metadata value is greater than <code class=”code”>value</code>. </dd> <dt>‘<samp class=”samp”>expr</samp>’</dt> <dd> Values are interpreted as floats, returns true if expression from option <code class=”code”>expr</code> evaluates to true. </dd> <dt>‘<samp class=”samp”>ends_with</samp>’</dt> <dd> Values are interpreted as strings, returns true if metadata value ends with the <code class=”code”>value</code> option string. </dd> </dl>

</dd> <dt><samp class=”option”>expr</samp></dt> <dd> Set

expression which is used when <code class=”code”>function</code> is set to <code class=”code”>expr</code>. The expression is evaluated through the eval API and can contain the following constants: <dl class=”table”> <dt><samp class=”option”>VALUE1, FRAMEVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> from metadata key. </dd> <dt><samp class=”option”>VALUE2, USERVAL</samp></dt> <dd> Float representation of <code class=”code”>value</code> as supplied by user in <code class=”code”>value</code> option. </dd> </dl>

</dd> <dt><samp class=”option”>file</samp></dt> <dd> If specified in <code class=”code”>print</code> mode, output is written to the named file. Instead of plain filename any writable url can be specified. Filename “-” is a shorthand for standard output. If <code class=”code”>file</code> option is not set, output is written to the log with AV_LOG_INFO loglevel. </dd> <dt><samp class=”option”>direct</samp></dt> <dd> Reduces buffering in print mode when output is written to a URL set using <var class=”var”>file</var>. </dd> </dl>

 <h4 class=”subsection”>41.13.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Print all metadata values for frames with key <code class=”code”>lavfi.signalstats.YDIF</code> with values between 0 and 1. <pre class=”example-preformatted”>signalstats,metadata=print:key=lavfi.signalstats.YDIF:value=0:function=expr:expr='between
</pre> Print silencedetect output to file <samp class=”file”>metadata.txt</samp>.
 <pre class=”example-preformatted”>silencedetect,ametadata=mode=print:file=meta
</pre> Direct all metadata to a pipe with file descriptor 4.
 <pre class=”example-preformatted”>metadata=mode=print:file='pipe\:4'
</pre>

 <h3 class=”section”>41.14 perms, aperms# TOC</h3>

Set read/write permissions for the output frames. These filters are mainly aimed at developers to test direct path in the following filter in the filtergraph. The filters accept the following options: <dl class=”table”> <dt><samp class=”option”>mode</samp></dt> <dd> Select the permissions mode. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>none</samp>’</dt> <dd> Do nothing. This is the default. </dd> <dt>‘<samp class=”samp”>ro</samp>’</dt> <dd> Set all the output frames

read-only. </dd> <dt>‘<samp class="samp">rw</samp>’</dt> <dd> Set all the output frames directly writable. </dd> <dt>‘<samp class="samp">toggle</samp>’</dt> <dd> Make the frame read-only if writable, and writable if read-only. </dd> <dt>‘<samp class="samp">random</samp>’</dt> <dd> Set each output frame read-only or writable randomly. </dd> </dl>

</dd> <dt><samp class="option">seed</samp></dt> <dd> Set the seed for the <var class="var">random</var> mode, must be an integer included between <code class="code">0</code> and <code class="code">UINT32_MAX</code>. If not specified, or if explicitly set to <code class="code">-1</code>, the filter will try to use a good random seed on a best effort basis. </dd> </dl>

Note: in case of auto-inserted filter between the permission filter and the following one, the permission might not be received as expected in that following filter. Inserting a format or aformat filter before the perms/aperms filter can avoid this problem. <h3 class="section">41.15 realtime, arealtime# TOC</h3>

Slow down filtering to match real time approximately. These filters will pause the filtering for a variable amount of time to match the output rate with the input timestamps. They are similar to the <samp class="option">re</samp> option to <code class="code">ffmpeg</code>. They accept the following options: <dl class="table"> <dt><samp class="option">limit</samp></dt> <dd> Time limit for the pauses. Any pause longer than that will be considered a timestamp discontinuity and reset the timer. Default is 2 seconds. </dd> <dt><samp class="option">speed</samp></dt> <dd> Speed factor for processing. The value must be a float larger than zero. Values larger than 1.0 will result in faster than realtime processing, smaller will slow processing down. The <var class="var">limit</var> is automatically adapted accordingly. Default is 1.0. A processing speed faster than what is possible without these filters cannot be achieved. </dd> </dl>

 <h4 class="subsection">41.15.1 Commands# TOC</h4>

Both filters supports the all above options as commands. <h3 class="section">41.16 segment, asegment# <a

class=”anchor hidden-xs”href=”#toc-segment_002c-asegment” aria-hidden=”true”>TOC</h3

Split single input stream into multiple streams. This filter does opposite of concat filters. <code class=”code”>segment</code> works on video frames, <code class=”code”>asegment</code> on audio samples. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>timestamps</samp></dt> <dd> Timestamps of output segments separated by ’|’. The first segment will run from the beginning of the input stream. The last segment will run until the end of the input stream </dd> <dt><samp class=”option”>frames, samples</samp></dt> <dd> Exact frame/sample count to split the segments. </dd> </dl>

In all cases, prefixing an each segment with ’+’ will make it relative to the previous segment. <h4 class=”subsection”>41.16.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Split input audio stream into three output audio streams, starting at start of input audio stream and storing that in 1st output audio stream, then following at 60th second and storing than in 2nd output audio stream, and last after 150th second of input audio stream store in 3rd output audio stream: <pre class=”example-preformatted”>asegment=timestamps="60|150" </pre>

 <h3 class=”section”>41.17 select, aselect# TOC</h3>

Select frames to pass in output. This filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>expr, e</samp></dt> <dd> Set expression, which is evaluated for each input frame. If the expression is evaluated to zero, the frame is discarded. If the evaluation result is negative or NaN, the frame is sent to the first output; otherwise it is sent to the output with index <code class=”code”>ceil(val)-1</code>, assuming that the input index starts from 0. For example a value of <code class=”code”>1.2</code> corresponds to the output with index <code class=”code”>ceil(1.2)-1 = 2-1 = 1</code>, that is the second output. </dd> <dt><samp class=”option”>outputs, n</samp></dt> <dd> Set the number of outputs. The output to which to send the selected frame is based on the result of the evaluation. Default value is 1. </dd> </dl>

The expression can contain the following constants: <dl

class=”table”> <dt><samp class=”option”>n</samp></dt> <dd> The (sequential) number of the filtered frame, starting from 0. </dd> <dt><samp class=”option”>selected_n</samp></dt> <dd> The (sequential) number of the selected frame, starting from 0. </dd> <dt><samp class=”option”>prev_selected_n</samp></dt> <dd> The sequential number of the last selected frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class=”option”>pts</samp></dt> <dd> The PTS (Presentation TimeStamp) of the filtered frame, expressed in <var class=”var”>TB</var> units. It’s NAN if undefined. </dd> <dt><samp class=”option”>t</samp></dt> <dd> The PTS of the filtered frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_pts</samp></dt> <dd> The PTS of the previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_pts</samp></dt> <dd> The PTS of the last previously filtered frame. It’s NAN if undefined. </dd> <dt><samp class=”option”>prev_selected_t</samp></dt> <dd> The PTS of the last previously selected frame, expressed in seconds. It’s NAN if undefined. </dd> <dt><samp class=”option”>start_pts</samp></dt> <dd> The first PTS in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>start_t</samp></dt> <dd> The first PTS, in seconds, in the stream which is not NAN. It remains NAN if not found. </dd> <dt><samp class=”option”>pict_type (video only)</samp></dt> <dd> The type of the filtered frame. It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>I</samp></dt> <dt><samp class=”option”>P</samp></dt> <dt><samp class=”option”>B</samp></dt> <dt><samp class=”option”>S</samp></dt> <dt><samp class=”option”>SI</samp></dt> <dt><samp class=”option”>SP</samp></dt> <dt><samp class=”option”>BI</samp></dt> </dl>

</dd> <dt><samp class=”option”>interlace_type (video only)</samp></dt> <dd> The frame interlace type. It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>PROGRESSIVE</samp></dt> <dd> The frame is progressive (not interlaced). </dd> <dt><samp class=”option”>TOPFIRST</samp></dt> <dd> The frame is top-field-first. </dd> <dt><samp class=”option”>BOTTOMFIRST</samp><
<dd> The frame is bottom-field-first. </dd> </dl>

</dd> <dt><samp class=”option”>consumed_sample_n (audio only)</samp></dt> <dd> the number of selected samples before the current frame </dd> <dt><samp class=”option”>samples_n (audio only)</samp></dt> <dd> the number of samples in the current frame </dd> <dt><samp class=”option”>sample_rate (audio only)</samp></dt> <dd> the input sample rate </dd> <dt><samp class=”option”>key</samp></dt>

<dd> This is 1 if the filtered frame is a key-frame, 0 otherwise. </dd> <dt><samp class="option">pos</samp></dt> <dd> the position in the file of the filtered frame, -1 if the information is not available (e.g. for synthetic video); deprecated, do not use </dd> <dt><samp class="option">scene (video only)</samp></dt> <dd> value between 0 and 1 to indicate a new scene; a low value reflects a low probability for the current frame to introduce a new scene, while a higher value means the current frame is more likely to be one (see the example below) </dd> <dt><samp class="option">concatdec_select</samp></dt> <dd> The concat demuxer can select only part of a concat input file by setting an inpoint and an outpoint, but the output packets may not be entirely contained in the selected interval. By using this variable, it is possible to skip frames generated by the concat demuxer which are not exactly contained in the selected interval. This works by comparing the frame pts against the <var class="var">lavf.concat.start_time</var> and the <var class="var">lavf.concat.duration</var> packet metadata values which are also present in the decoded frames. The <var class="var">concatdec_select</var> variable is -1 if the frame pts is at least start_time and either the duration metadata is missing or the frame pts is less than start_time + duration, 0 otherwise, and NaN if the start_time metadata is missing. That basically means that an input frame is selected if its pts is within the interval set by the concat demuxer. </dd> <dt><samp class="option">iw (video only)</samp></dt> <dd> Represents the width of the input video frame. </dd> <dt><samp class="option">ih (video only)</samp></dt> <dd> Represents the height of the input video frame. </dd> <dt><samp class="option">view (video only)</samp></dt> <dd> View ID for multi-view video. </dd> </dl>

The default value of the select expression is "1". <h4 class="subsection">41.17.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Select all frames in input: <pre class="example-preformatted">select </pre>

The example above is the same as: <pre class="example-preformatted">select=1 </pre>

Skip all frames: <pre class="example-preformatted">select=0 </pre>

Select only I-frames: <pre class="example-preformatted">select='eq(pict_type\,I)' </pre>

Select one frame every 100: <pre class=”example-preformatted”>select='not(mod(n\,100))' </pre>

Select only frames contained in the 10-20 time interval: <pre class=”example-preformatted”>select=between(t\,10\,20) </pre>

Select only I-frames contained in the 10-20 time interval: <pre class=”example-preformatted”>select=between(t\,10\,20)*eq(pict_type\,I) </pre>

Select frames with a minimum distance of 10 seconds: <pre class=”example-preformatted”>select='isnan(prev_selected_t)+gte(t-prev_selected_t\,10)' </pre>

Use aselect to select only audio frames with samples number > 100: <pre class=”example-preformatted”>aselect='gt(samples_n\,100)' </pre>

Create a mosaic of the first scenes: <pre class=”example-preformatted”>ffmpeg -i video.avi -vf select='gt(scene\,0.4)',scale=160:120,tile -frames:v 1 preview.png </pre>

Comparing <var class=”var”>scene</var> against a value between 0.3 and 0.5 is generally a sane choice. Send even and odd frames to separate outputs, and compose them: <pre class=”example-preformatted”>select=n=2:e='mod(n, 2)+1' [odd][even]; [odd] pad=h=2*ih [tmp]; [tmp][even] overlay=y=h </pre>

Select useful frames from an ffconcat file which is using inpoints and outpoints but where the source files are not intra frame only. <pre class=”example-preformatted”>ffmpeg -copyts -segment_time_metadata 1 -i input.ffconcat -fps_mode passthrough -vf select=concatdec_select -af aselect=concatdec_select output.avi </pre>

 <h3 class=”section”>41.18 sendcmd, asendcmd# TOC</h3>

Send commands to filters in the filtergraph. These filters read commands to be sent to other filters in the filtergraph. <code class=”code”>sendcmd</code> must be inserted between two video filters, <code class=”code”>asendcmd</code> must be inserted between two audio filters, but apart from that they act the same way. The specification of commands can be provided in the filter arguments with the <var class=”var”>commands</var> option, or in a file specified by the <var class=”var”>filename</var> option.

These filters accept the following options: <dl class="table"> <dt><samp class="option">commands, c</samp></dt> <dd> Set the commands to be read and sent to the other filters. </dd> <dt><samp class="option">filename, f</samp></dt> <dd> Set the filename of the commands to be read and sent to the other filters. </dd> </dl>

 <h4 class="subsection">41.18.1 Commands syntax# TOC</h4>

A commands description consists of a sequence of interval specifications, comprising a list of commands to be executed when a particular event related to that interval occurs. The occurring event is typically the current frame time entering or leaving a given time interval. An interval is specified by the following syntax: <pre class="example-preformatted"><var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var>; </pre>

The time interval is specified by the <var class="var">START</var> and <var class="var">END</var> times. <var class="var">END</var> is optional and defaults to the maximum time. The current frame time is considered within the specified interval if it is included in the interval [<var class="var">START</var>, <var class="var">END</var>), that is when the time is greater or equal to <var class="var">START</var> and is lesser than <var class="var">END</var>. <var class="var">COMMANDS</var> consists of a sequence of one or more command specifications, separated by ",", relating to that interval. The syntax of a command specification is given by: <pre class="example-preformatted">[<var class="var">FLAGS</var>] <var class="var">TARGET</var> <var class="var">COMMAND</var> <var class="var">ARG</var> </pre>

<var class="var">FLAGS</var> is optional and specifies the type of events relating to the time interval which enable sending the specified command, and must be a non-null sequence of identifier flags separated by "+" or "|" and enclosed between "[" and "]". The following flags are recognized: <dl class="table"> <dt><samp class="option">enter</samp></dt> <dd> The command is sent when the current frame timestamp enters the specified interval. In other words, the command is sent when the previous frame timestamp was not in the given interval, and the current is. </dd> <dt><samp class="option">leave</samp></dt> <dd> The command is sent when the current frame timestamp leaves the specified interval. In other words, the command is sent when the previous frame timestamp was in the given interval, and the current is not. </dd> <dt><samp class="option">expr</samp></dt> <dd> The

command <var class="var">ARG</var> is interpreted as expression and result of expression is passed as <var class="var">ARG</var>. The expression is evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp class="option">POS</samp></dt> <dd> Original position in the file of the frame, or undefined if undefined for the current frame. Deprecated, do not use. </dd> <dt><samp class="option">PTS</samp></dt> <dd> The presentation timestamp in input. </dd> <dt><samp class="option">N</samp></dt> <dd> The count of the input frame for video or audio, starting from 0. </dd> <dt><samp class="option">T</samp></dt> <dd> The time in seconds of the current frame. </dd> <dt><samp class="option">TS</samp></dt> <dd> The start time in seconds of the current command interval. </dd> <dt><samp class="option">TE</samp></dt> <dd> The end time in seconds of the current command interval. </dd> <dt><samp class="option">TI</samp></dt> <dd> The interpolated time of the current command interval, TI = (T - TS) / (TE - TS). </dd> <dt><samp class="option">W</samp></dt> <dd> The video frame width. </dd> <dt><samp class="option">H</samp></dt> <dd> The video frame height. </dd> </dl>

</dd> </dl>

If <var class="var">FLAGS</var> is not specified, a default value of <code class="code">[enter]</code> is assumed. <var class="var">TARGET</var> specifies the target of the command, usually the name of the filter class or a specific filter instance name. <var class="var">COMMAND</var> specifies the name of the command for the target filter. <var class="var">ARG</var> is optional and specifies the optional list of argument for the given <var class="var">COMMAND</var>. Between one interval specification and another, whitespaces, or sequences of characters starting with <code class="code">#</code> until the end of line, are ignored and can be used to annotate comments. A simplified BNF description of the commands specification syntax follows: <pre class="example-preformatted"><var class="var">COMMAND_FLAG</var> ::= "enter" | "leave"
<var class="var">COMMAND_FLAGS</var> ::= <var class="var">COMMAND_FLAG</var> [(+|"|")<var class="var">COMMAND_FLAG</var>] <var class="var">COMMAND</var> ::= ["[" <var class="var">COMMAND_FLAGS</var> "]"] <var class="var">TARGET</var> <var class="var">COMMAND</var> [<var class="var">ARG</var>] <var class="var">COMMANDS</var> ::= <var class="var">COMMAND</var> [,<var class="var">COMMANDS</var>]
<var class="var">INTERVAL</var> ::= <var class="var">START</var>[-<var class="var">END</var>] <var class="var">COMMANDS</var> <var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[;<var class="var">INTERVALS</var>] </pre>

 <h4 class="subsection">41.18.2 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Specify audio tempo change at second 4: <pre class="example-preformatted">asendcmd=c='4.0 atempo tempo 1.5',atempo </pre>

Target a specific filter instance: <pre class="example-preformatted">asendcmd=c='4.0 atempo@my tempo 1.5',atempo@my </pre>

Specify a list of drawtext and hue commands in a file. <pre class="example-preformatted"># show text in the interval 5-10 5.0-10.0 [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=hello world', [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=';

desaturate the image in the interval 15-20 15.0-20.0 [enter] hue s 0, [enter] drawtext reinit 'fontfile=FreeSerif.ttf:text=nocolor', [leave] hue s 1, [leave] drawtext reinit 'fontfile=FreeSerif.ttf:text=color';

apply an exponential saturation fade-out effect, starting from time 25 25 [enter] hue s exp(25-t) </pre>

A filtergraph allowing to read and process the above command list stored in a file <samp class="file">test.cmd</samp>, can be specified with: <pre class="example-preformatted">sendcmd=f=test.cmd,drawtext=fontfile=FreeSerif.ttf:text='',hue </pre>

 <h3 class="section">41.19 setpts, asetpts# TOC</h3>

Change the PTS (presentation timestamp) of the input frames. <code class="code">setpts</code> works on video frames, <code class="code">asetpts</code> on audio frames. This filter accepts the following options: <dl class="table"> <dt><samp class="option">expr</samp></dt> <dd> The expression which is evaluated for each frame to construct its timestamp. </dd> <dt><samp class="option">strip_fps (video only)</samp></dt> <dd> Boolean option which determines if the original framerate and frame duration metadata is unset. If set to true, be advised that a sane frame rate should be explicitly specified if output is sent to a constant frame rate muxer. Default is <code class="code">false</code>. </dd> </dl>

The expression is evaluated through the eval API and can contain the following constants: <dl class="table"> <dt><samp

class=”option”>FRAME_RATE, FR</samp></dt> <dd> frame rate, only defined for constant frame-rate video </dd> <dt><samp class=”option”>PTS</samp></dt> <dd> The presentation timestamp in input </dd> <dt><samp class=”option”>N</samp></dt> <dd> The count of the input frame for video or the number of consumed samples, not including the current frame for audio, starting from 0. </dd> <dt><samp class=”option”>NB_CONSUMED_SAMPLES</samp></dt> <dd> The number of consumed samples, not including the current frame (only audio) </dd> <dt><samp class=”option”>NB_SAMPLES, S</samp></dt> <dd> The number of samples in the current frame (only audio) </dd> <dt><samp class=”option”>SAMPLE_RATE, SR</samp></dt> <dd> The audio sample rate. </dd> <dt><samp class=”option”>STARTPTS</samp></dt> <dd> The PTS of the first frame. </dd> <dt><samp class=”option”>STARTT</samp></dt> <dd> the time in seconds of the first frame </dd> <dt><samp class=”option”>INTERLACED</samp></dt> <dd> State whether the current frame is interlaced. </dd> <dt><samp class=”option”>T</samp></dt> <dd> the time in seconds of the current frame </dd> <dt><samp class=”option”>POS</samp></dt> <dd> original position in the file of the frame, or undefined if undefined for the current frame; deprecated, do not use </dd> <dt><samp class=”option”>PREV_INPTS</samp></dt> <dd> The previous input PTS. </dd> <dt><samp class=”option”>PREV_INT</samp></dt> <dd> previous input time in seconds </dd> <dt><samp class=”option”>PREV_OUTPTS</samp></dt> <dd> The previous output PTS. </dd> <dt><samp class=”option”>PREV_OUTT</samp></dt> <dd> previous output time in seconds </dd> <dt><samp class=”option”>RTCTIME</samp></dt> <dd> The wallclock (RTC) time in microseconds. This is deprecated, use time(0) instead. </dd> <dt><samp class=”option”>RTCSTART</samp></dt> <dd> The wallclock (RTC) time at the start of the movie in microseconds. </dd> <dt><samp class=”option”>TB</samp></dt> <dd> The timebase of the input timestamps. </dd> <dt><samp class=”option”>T_CHANGE</samp></dt> <dd> Time of the first frame after command was applied or time of the first frame if no commands. </dd> </dl>

 <h4 class=”subsection”>41.19.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Start counting PTS from zero <pre class=”example-preformatted”>setpts=PTS-STARTPTS </pre>

Apply fast motion effect: <pre class=”example-preformatted”>setpts=0.5*PTS </pre>

Apply slow motion effect: <pre class=”example-preformatted”>setpts=2.0*PTS </pre>

Set fixed rate of 25 frames per second: <pre class=”example-preformatted”>setpts=N/(25*TB) </pre>

Apply a random jitter effect of +/-100 TB units: <pre class=”example-preformatted”>setpts=PTS+randomi(0, -100\,100) </pre>

Set fixed rate 25 fps with some jitter: <pre class=”example-preformatted”>setpts='1/(25*TB) * (N + 0.05 * sin(N*2*PI/25))' </pre>

Apply an offset of 10 seconds to the input PTS: <pre class=”example-preformatted”>setpts=PTS+10/TB </pre>

Generate timestamps from a "live source" and rebase onto the current timebase: <pre class=”example-preformatted”>setpts='(RTCTIME - RTCSTART) / (TB * 1000000)' </pre>

Generate timestamps by counting samples: <pre class=”example-preformatted”>asetpts=N/SR/TB </pre>

 <h4 class=”subsection”>41.19.2 Commands# TOC</h4>

Both filters support all above options as commands. <h3 class=”section”>41.20 setrange# TOC</h3>

Force color range for the output video frame. The <code class=”code”>setrange</code> filter marks the color range property for the output frames. It does not change the input frame, but only sets the corresponding property, which affects how the frame is treated by following filters. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>range</samp></dt> <dd> Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>auto</samp>’</dt> <dd> Keep the same color range property. </dd> <dt>‘<samp class=”samp”>unspecified, unknown</samp>’</dt> <dd> Set the color range as unspecified. </dd> <dt>‘<samp class=”samp”>limited, tv, mpeg</samp>’</dt> <dd> Set the color range as limited.

</dd> <dt>‘<samp class="samp">full, pc, jpeg</samp>’</dt>
<dd> Set the color range as full. </dd> </dl> </dd> </dl>

 <h3 class="section">41.21 settb, asettb# TOC</h3>

Set the timebase to use for the output frames timestamps. It is mainly useful for testing timebase configuration. It accepts the following parameters: <dl class="table"> <dt><samp class="option">expr, tb</samp></dt> <dd> The expression which is evaluated into the output timebase. </dd> </dl>

The value for <samp class="option">tb</samp> is an arithmetic expression representing a rational. The expression can contain the constants "AVTB" (the default timebase), "intb" (the input timebase) and "sr" (the sample rate, audio only). Default value is "intb". <h4 class="subsection">41.21.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Set the timebase to 1/25: <pre class="example-preformatted">settb=expr=1/25 </pre>

Set the timebase to 1/10: <pre class="example-preformatted">settb=expr=0.1 </pre>

Set the timebase to 1001/1000: <pre class="example-preformatted">settb=1+0.001 </pre>

Set the timebase to 2*intb: <pre class="example-preformatted">settb=2*intb </pre>

Set the default timebase value: <pre class="example-preformatted">settb=AVTB </pre>

<h3 class="section">41.22 showcqt# TOC</h3>

Convert input audio to a video output representing frequency spectrum

logarithmically using Brown-Puckette constant Q transform algorithm with

direct frequency domain coefficient calculation (but the transform itself

is not really constant Q, instead the Q factor is actually variable/clamped),

with musical tone scale, from E0 to D#10.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify the video size for the output. It must be even. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">1920x1080</code>.

</dd>

<dt><samp class="option">fps, rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">bar_h</samp></dt>

<dd> Set the bargraph height. It must be even. Default value is <code class="code">-1</code> which

computes the bargraph height automatically.

</dd>

<dt><samp class="option">axis_h</samp></dt>

<dd> Set the axis height. It must be even. Default value is <code class="code">-1</code> which computes

the axis height automatically.

</dd>

<dt><samp class="option">sono_h</samp></dt>

<dd> Set the sonogram height. It must be even. Default value is <code class="code">-1</code> which

computes the sonogram height automatically.

</dd>

<dt><samp class="option">fullhd</samp></dt>

<dd> Set the fullhd resolution. This option is deprecated, use <var class="var">size</var>, <var class="var">s</var>

instead. Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">sono_v, volume</samp></dt>

<dd> Specify the sonogram volume expression. It can contain variables:

<dl class="table">

<dt><samp class="option">bar_v</samp></dt>

<dd> the <var class="var">bar_v</var> evaluated expression

</dd>

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">a_weighting(f)</samp></dt>

<dd> A-weighting of equal loudness

</dd>

<dt><samp class="option">b_weighting(f)</samp></dt>

<dd> B-weighting of equal loudness

</dd>

<dt><samp class="option">c_weighting(f)</samp></dt>

<dd> C-weighting of equal loudness.
</dd>
</dl>
Default value is <code class="code">16</code>.

</dd>
<dt><samp class="option">bar_v, volume2</samp></dt>
<dd> Specify the bargraph volume expression. It can contain variables:
<dl class="table">
<dt><samp class="option">sono_v</samp></dt>
<dd> the <var class="var">sono_v</var> evaluated expression
</dd>
<dt><samp class="option">frequency, freq, f</samp></dt>
<dd> the frequency where it is evaluated
</dd>
<dt><samp class="option">timeclamp, tc</samp></dt>
<dd> the value of <var class="var">timeclamp</var> option
</dd>
</dl>
and functions:
<dl class="table">
<dt><samp class="option">a_weighting(f)</samp></dt>
<dd> A-weighting of equal loudness
</dd>
<dt><samp class="option">b_weighting(f)</samp></dt>
<dd> B-weighting of equal loudness
</dd>
<dt><samp class="option">c_weighting(f)</samp></dt>
<dd> C-weighting of equal loudness.
</dd>
</dl>

Default value is <code class="code">sono_v</code>.

</dd>

<dt><samp class="option">sono_g, gamma</samp></dt>

<dd> Specify the sonogram gamma. Lower gamma makes the spectrum more contrast,

higher gamma makes the spectrum having more range. Default value is <code class="code">3</code>.

Acceptable range is <code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_g, gamma2</samp></dt>

<dd> Specify the bargraph gamma. Default value is <code class="code">1</code>. Acceptable range is

<code class="code">[1, 7]</code>.

</dd>

<dt><samp class="option">bar_t</samp></dt>

<dd> Specify the bargraph transparency level. Lower value makes the bargraph sharper.

Default value is <code class="code">1</code>. Acceptable range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> Specify the transform timeclamp. At low frequency, there is trade-off between

accuracy in time domain and frequency domain. If timeclamp is lower,

event in time domain is represented more accurately (such as fast bass drum),

otherwise event in frequency domain is represented more accurately

(such as bass guitar). Acceptable range is <code class="code">[0.002, 1]</code>. Default value is <code class="code">0.17</code>.

</dd>

<dt><samp class="option">attack</samp></dt>

<dd> Set attack time in seconds. The default is <code class="code">0</code> (disabled). Otherwise, it

limits future samples by applying asymmetric windowing in time domain, useful

when low latency is required. Accepted range is <code class="code">[0, 1]</code>.

</dd>

<dt><samp class="option">basefreq</samp></dt>

<dd> Specify the transform base frequency. Default value is <code class="code">20.01523126408007475</code>,

which is frequency 50 cents below E0. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">endfreq</samp></dt>

<dd> Specify the transform end frequency. Default value is <code class="code">20495.59681441799654</code>,

which is frequency 50 cents above D#10. Acceptable range is <code class="code">[10, 100000]</code>.

</dd>

<dt><samp class="option">coeffclamp</samp></dt>

<dd> This option is deprecated and ignored.

</dd>

<dt><samp class="option">tlength</samp></dt>

<dd> Specify the transform length in time domain. Use this option to control accuracy

trade-off between time domain and frequency domain at every frequency sample.

It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option.

</dd>

</dl>

Default value is <code class="code">384*tc/(384+tc*f)</code>.

</dd>

<dt><samp class="option">count</samp></dt>

<dd> Specify the transform count for every video frame. Default value is <code class="code">6</code>.

Acceptable range is <code class="code">[1, 30]</code>.

</dd>

<dt><samp class="option">fcount</samp></dt>

<dd> Specify the transform count for every single pixel. Default value is <code class="code">0</code>,

which makes it computed automatically. Acceptable range is <code class="code">[0, 10]</code>.

</dd>

<dt><samp class="option">fontfile</samp></dt>

<dd> Specify font file for use with freetype to draw the axis. If not specified,

use embedded font. Note that drawing with font file or embedded font is not

implemented with custom <var class="var">basefreq</var> and <var class="var">endfreq</var>, use <var class="var">axisfile</var>

option instead.

</dd>

<dt><samp class="option">font</samp></dt>

<dd> Specify fontconfig pattern. This has lower priority than <var class="var">fontfile</var>. The

<code class="code">:</code> in the pattern may be replaced by <code class="code">|</code> to avoid unnecessary

escaping.

</dd>

<dt><samp class="option">fontcolor</samp></dt>

<dd> Specify font color expression. This is arithmetic expression that should return

integer value 0xRRGGBB. It can contain variables:

<dl class="table">

<dt><samp class="option">frequency, freq, f</samp></dt>

<dd> the frequency where it is evaluated

</dd>

<dt><samp class="option">timeclamp, tc</samp></dt>

<dd> the value of <var class="var">timeclamp</var> option

</dd>

</dl>

and functions:

<dl class="table">

<dt><samp class="option">midi(f)</samp></dt>

<dd> midi number of frequency f, some midi numbers: E0(16), C1(24), C2(36), A4(69)

</dd>

<dt><samp class="option">r(x), g(x), b(x)</samp></dt>

<dd> red, green, and blue value of intensity x.

</dd>

</dl>

Default value is <code class="code">st(0, (midi(f)-59.5)/12);

st(1, if(between(ld(0),0,1), 0.5-0.5*cos(2*PI*ld(0)), 0));

r(1-ld(1)) + b(ld(1))</code>.

</dd>

<dt><samp class="option">axisfile</samp></dt>

<dd> Specify image file to draw the axis. This option override <var class="var">fontfile</var> and

<var class="var">fontcolor</var> option.

</dd>

<dt><samp class="option">axis, text</samp></dt>

<dd> Enable/disable drawing text to the axis. If it is set to <code class="code">0</code>, drawing to

the axis is disabled, ignoring <var class="var">fontfile</var> and <var class="var">axisfile</var> option.

Default value is <code class="code">1</code>.

</dd>

<dt><samp class="option">csp</samp></dt>

<dd> Set colorspace. The accepted values are:

<dl class="table">

<dt>‘<samp class="samp">unspecified</samp>’</dt>

<dd> Unspecified (default)

</dd>

<dt>‘<samp class="samp">bt709</samp>’</dt>

<dd> BT.709

</dd>

<dt>‘<samp class="samp">fcc</samp>’</dt>

<dd> FCC

</dd>

<dt>‘<samp class="samp">bt470bg</samp>’</dt>

<dd> BT.470BG or BT.601-6 625

</dd>

<dt>‘<samp class="samp">smpte170m</samp>’</dt>

<dd> SMPTE-170M or BT.601-6 525

</dd>

<dt>‘<samp class="samp">smpte240m</samp>’</dt>

<dd> SMPTE-240M

</dd>

<dt>‘<samp class="samp">bt2020ncl</samp>’</dt>

<dd> BT.2020 with non-constant luminance

</dd>

</dl>

</dd> <dt><samp class="option">cscheme</samp></dt> <dd> Set spectrogram color scheme. This is list of floating point values with format <code class="code">left_r|left_g|left_b|right_r|right_g|right_b</code>. The default is <code class="code">1|0.5|0|0|0.5|1</code>. </dd> </dl>

 <h4 class="subsection">41.22.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Playing audio while showing the spectrum: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with frame rate 30 fps: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=fps=30:count=5 [out0]' </pre>

Playing at 1280x720: <pre class="example-preformatted">ffplay -f lavfi 'amovie=a.mp3, asplit [a][out1]; [a] showcqt=s=1280x720:count=4 [out0]' </pre>

Disable sonogram display: <pre class="example-preformatted">sono_h=0 </pre>

A1 and its harmonics: A1, A2, (near)E3, A3:
<pre class=”example-preformatted”>ffplay -f lavfi 'aevalsrc=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6
asplit[a][out1]; [a] showcqt [out0]' </pre>

Same as above, but with more accuracy in frequency domain: <div
class=”example”> <pre class=”example-preformatted”>ffplay -f lavfi 'aeval-
src=0.1*sin(2*PI*55*t)+0.1*sin(4*PI*55*t)+0.1*sin(6*PI*55*t)+0.1*sin(8*PI*55*t),
asplit[a][out1]; [a] showcqt=timeclamp=0.5 [out0]' </pre>

Custom volume: <pre class=”example-
preformatted”>bar_v=10:sono_v=bar_v*a_weighting(f) </pre>

Custom gamma, now spectrum is linear to the amplitude. <div
class=”example”> <pre class=”example-preformatted”>bar_g=2:sono_g=2
</pre>

Custom tlength equation: <pre class=”example-
preformatted”>tc=0.33:tlength='st(0,0.17); 384*tc / (384 / ld(0) + tc*f
/(1-ld(0))) + 384*tc / (tc*f / ld(0) + 384 /(1-ld(0)))' </pre>

Custom fontcolor and fontfile, C-note is colored green, oth-
ers are colored blue: <pre class=”example-
preformatted”>fontcolor='if(mod(floor(midi(f)+0.5),12), 0x0000FF, g(1))':fontfile=myfont.ttf
</pre>

Custom font using fontconfig: <pre
class=”example-preformatted”>font='Courier New,Monospace,mono|bold'
</pre>

Custom frequency range with custom axis using image file: <div
class=”example”> <pre class=”example-preformatted”>axisfile=myaxis.png:basefreq=40:endfreq=10000
</pre>

 <h3 class=”section”>41.23 showcwt<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#showcwt” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-showcwt”
aria-hidden=”true”>TOC</h3>

Convert input audio to video output representing frequency spectrum

using Continuous Wavelet Transform and Morlet wavelet.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option,

check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default value is <code class="code">640x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set the output frame rate. Default value is <code class="code">25</code>.

</dd>

<dt><samp class="option">scale</samp></dt>

<dd> Set the frequency scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class="option">log</samp></dt>

<dt><samp class="option">bark</samp></dt>

<dt><samp class="option">mel</samp></dt>

<dt><samp class="option">erbs</samp></dt>

<dt><samp class="option">sqrt</samp></dt>

<dt><samp class="option">cbrt</samp></dt>

<dt><samp class="option">qdrt</samp></dt>

<dt><samp class="option">fm</samp></dt>

</dl>

Default value is <code class="code">linear</code>.

</dd>

<dt><samp class="option">iscale</samp></dt>

<dd> Set the intensity scale used. Allowed values are:

<dl class="table">

<dt><samp class="option">linear</samp></dt>

<dt><samp class=”option”>log</samp></dt>

<dt><samp class=”option”>sqrt</samp></dt>

<dt><samp class=”option”>cbrt</samp></dt>

<dt><samp class=”option”>qdrt</samp></dt>

</dl>

Default value is <code class=”code”>log</code>.

</dd>

<dt><samp class=”option”>min</samp></dt>

<dd> Set the minimum frequency that will be used in output.

Default is <code class=”code”>20</code> Hz.

</dd>

<dt><samp class=”option”>max</samp></dt>

<dd> Set the maximum frequency that will be used in output.

Default is <code class=”code”>20000</code> Hz. The real frequency upper limit

depends on input audio’s sample rate and such will be enforced

on this value when it is set to value greater than Nyquist frequency.

</dd>

<dt><samp class=”option”>imin</samp></dt>

<dd> Set the minimum intensity that will be used in output.

</dd>

<dt><samp class=”option”>imax</samp></dt>

<dd> Set the maximum intensity that will be used in output.

</dd>

<dt><samp class=”option”>logb</samp></dt>

<dd> Set the logarithmic basis for brightness strength when

mapping calculated magnitude values to pixel values.

Allowed range is from <code class=”code”>0</code> to <code class=”code”>1</code>.

Default value is <code class=”code”>0.0001</code>.

</dd>

<dt><samp class=”option”>deviation</samp></dt>

<dd> Set the frequency deviation.

Lower values than <code class=”code”>1</code> are more frequency oriented,

while higher values than <code class=”code”>1</code> are more time oriented.

Allowed range is from <code class=”code”>0</code> to <code class=”code”>10</code>.

Default value is <code class=”code”>1</code>.

</dd>

<dt><samp class=”option”>pps</samp></dt>

<dd> Set the number of pixel output per each second in one row.

Allowed range is from <code class=”code”>1</code> to <code class=”code”>1024</code>.

Default value is <code class=”code”>64</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Set the output visual mode. Allowed values are:

<dl class=”table”>

<dt><samp class=”option”>magnitude</samp></dt>

<dd> Show magnitude.

</dd>

<dt><samp class=”option”>phase</samp></dt>

<dd> Show only phase.

</dd>

<dt><samp class=”option”>magphase</samp></dt>

<dd> Show combination of magnitude and phase.

Magnitude is mapped to brightness and phase to color.

</dd>

<dt><samp class="option">channel</samp></dt>

<dd> Show unique color per channel magnitude.

</dd>

<dt><samp class="option">stereo</samp></dt>

<dd> Show unique color per stereo difference.

</dd>

</dl>

Default value is <code class="code">magnitude</code>. </dd> <dt><samp class="option">slide</samp></dt> <dd> Set the output slide method. Allowed values are: <dl class="table"> <dt><samp class="option">replace</samp></dt> <dt><samp class="option">scroll</samp></dt> <dt><samp class="option">frame</samp></dt> </dl>

</dd> <dt><samp class="option">direction</samp></dt> <dd> Set the direction method for output slide method. Allowed values are: <dl class="table"> <dt><samp class="option">lr</samp></dt> <dd> Direction from left to right. </dd> <dt><samp class="option">rl</samp></dt> <dd> Direction from right to left. </dd> <dt><samp class="option">ud</samp></dt> <dd> Direction from up to down. </dd> <dt><samp class="option">du</samp></dt> <dd> Direction from down to up. </dd> </dl>

</dd> <dt><samp class="option">bar</samp></dt> <dd> Set the ratio of bargraph display to display size. Default is 0. </dd> <dt><samp class="option">rotation</samp></dt> <dd> Set color rotation, must be in [-1.0, 1.0] range. Default value is <code class="code">0</code>. </dd> </dl>

 <h3 class="section">41.24 showfreqs# TOC</h3>

Convert input audio to video output representing the audio power spectrum.

Audio amplitude is on Y-axis while frequency is on X-axis.

The filter accepts the following options:

<dl class="table">

<dt><samp class="option">size, s</samp></dt>

<dd> Specify size of video. For the syntax of this option, check the

<a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual.

Default is <code class="code">1024x512</code>.

</dd>

<dt><samp class="option">rate, r</samp></dt>

<dd> Set video rate. Default is <code class="code">25</code>.

</dd>

<dt><samp class="option">mode</samp></dt>

<dd> Set display mode.

This set how each frequency bin will be represented.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">line</samp>’</dt>

<dt>‘<samp class="samp">bar</samp>’</dt>

<dt>‘<samp class="samp">dot</samp>’</dt>

</dl>

Default is <code class="code">bar</code>.

</dd>

<dt><samp class="option">ascale</samp></dt>

<dd> Set amplitude scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">lin</samp>’</dt>

<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">sqrt</samp>’</dt>
<dd> Square root scale.

</dd>
<dt>‘<samp class="samp">cbrt</samp>’</dt>
<dd> Cubic root scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.
</dd>
</dl>
Default is <code class="code">log</code>.

</dd>
<dt><samp class="option">fscale</samp></dt>
<dd> Set frequency scale.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">lin</samp>’</dt>
<dd> Linear scale.

</dd>
<dt>‘<samp class="samp">log</samp>’</dt>
<dd> Logarithmic scale.

</dd>

<dt>‘<samp class="samp">rlog</samp>’</dt>
<dd> Reverse logarithmic scale.
</dd>
</dl>
Default is <code class="code">lin</code>.

</dd>
<dt><samp class="option">win_size</samp></dt>
<dd> Set window size. Allowed range is from 16 to 65536.

Default is <code class="code">2048</code>

</dd>
<dt><samp class="option">win_func</samp></dt>
<dd> Set windowing function.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt>
<dt>‘<samp class="samp">bartlett</samp>’</dt>
<dt>‘<samp class="samp">hanning</samp>’</dt>
<dt>‘<samp class="samp">hamming</samp>’</dt>
<dt>‘<samp class="samp">blackman</samp>’</dt>
<dt>‘<samp class="samp">welch</samp>’</dt>
<dt>‘<samp class="samp">flattop</samp>’</dt>
<dt>‘<samp class="samp">bharris</samp>’</dt>
<dt>‘<samp class="samp">bnuttall</samp>’</dt>
<dt>‘<samp class="samp">bhann</samp>’</dt>
<dt>‘<samp class="samp">sine</samp>’</dt>
<dt>‘<samp class="samp">nuttall</samp>’</dt>
<dt>‘<samp class="samp">lanczos</samp>’</dt>

<dt>‘<samp class="samp">gauss</samp>’</dt>

<dt>‘<samp class="samp">tukey</samp>’</dt>

<dt>‘<samp class="samp">dolph</samp>’</dt>

<dt>‘<samp class="samp">cauchy</samp>’</dt>

<dt>‘<samp class="samp">parzen</samp>’</dt>

<dt>‘<samp class="samp">poisson</samp>’</dt>

<dt>‘<samp class="samp">bohman</samp>’</dt>

<dt>‘<samp class="samp">kaiser</samp>’</dt>

</dl>

Default is <code class="code">hanning</code>.

</dd>

<dt><samp class="option">overlap</samp></dt>

<dd> Set window overlap. In range <code class="code">[0, 1]</code>. Default is <code class="code">1</code>,

which means optimal overlap for selected window function will be picked.

</dd>

<dt><samp class="option">averaging</samp></dt>

<dd> Set time averaging. Setting this to 0 will display current maximal peaks.

Default is <code class="code">1</code>, which means time averaging is disabled.

</dd>

<dt><samp class="option">colors</samp></dt>

<dd> Specify list of colors separated by space or by ’|’ which will be used to

draw channel frequencies. Unrecognized or missing colors will be replaced

by white color.

</dd>

<dt><samp class="option">cmode</samp></dt>

<dd> Set channel display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">combined</samp>’</dt>

<dt>‘<samp class="samp">separate</samp>’</dt>

</dl>

Default is <code class="code">combined</code>.

</dd>

<dt><samp class="option">minamp</samp></dt>

<dd> Set minimum amplitude used in <code class="code">log</code> amplitude scaler.

</dd>

<dt><samp class="option">data</samp></dt>

<dd> Set data display mode.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">magnitude</samp>’</dt>

<dt>‘<samp class="samp">phase</samp>’</dt>

<dt>‘<samp class="samp">delay</samp>’</dt>

</dl>

Default is <code class="code">magnitude</code>.

</dd>

<dt><samp class="option">channels</samp></dt>

<dd> Set channels to use when processing audio. By default all are processed.

</dd>

</dl>

 <h3 class="section">41.25 showspatial# TOC</h3>

Convert stereo input audio to a video output, representing the spatial relationship between two channels. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">512x512</code>. </dd> <dt><samp class="option">win_size</samp></dt> <dd> Set window size. Allowed range is from <var class="var">1024</var> to <var class="var">65536</var>. Default size is <var class="var">4096</var>. </dd> <dt><samp class="option">win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</dt> <dt>‘<samp class="samp">hanning</samp>’<
<dt>‘<samp class="samp">hamming</samp>’</dt>
<dt>‘<samp class="samp">blackman</samp>’</dt>
<dt>‘<samp class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</dt> <dt>‘<samp class="samp">bharris</samp>’<
<dt>‘<samp class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’</dt> <dt>‘<samp class="samp">sine</samp>’</dt
<dt>‘<samp class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’</dt> <dt>‘<samp class="samp">gauss</samp>’</
<dt>‘<samp class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</dt> <dt>‘<samp class="samp">cauchy</samp>’</
<dt>‘<samp class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’</dt> <dt>‘<samp class="samp">bohman</samp>&rsquo
<dt>‘<samp class="samp">kaiser</samp>’</dt> </dl>

Default value is <code class="code">hann</code>. </dd> <dt><samp class="option">rate, r</samp></dt> <dd> Set output framerate. </dd> </dl>

 <h3 class="section">41.26 showspectrum# TOC</h3>

Convert input audio to a video output, representing the audio frequency spectrum. The filter accepts the following options:

<dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">640x512</code>. </dd> <dt><samp class="option">slide</samp></dt> <dd> Specify how the spectrum should slide along the window. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">replace</samp>’</dt> <dd> the samples start again on the left when they reach the right </dd> <dt>‘<samp class="samp">scroll</samp>’</dt> <dd> the samples scroll from right to left </dd> <dt>‘<samp class="samp">fullframe</samp>’</dt> <dd> frames are only produced when the samples reach the right </dd> <dt>‘<samp class="samp">rscroll</samp>’</dt> <dd> the samples scroll from left to right </dd> <dt>‘<samp class="samp">lreplace</samp>’</dt> <dd> the samples start again on the right when they reach the left </dd> </dl>

Default value is <code class="code">replace</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Specify display mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">combined</samp>’</dt> <dd> all channels are displayed in the same row </dd> <dt>‘<samp class="samp">separate</samp>’</dt> <dd> all channels are displayed in separate rows </dd> </dl>

Default value is ‘<samp class="samp">combined</samp>’. </dd> <dt><samp class="option">color</samp></dt> <dd> Specify display color mode. It accepts the following values: <dl class="table"> <dt>‘<samp class="samp">channel</samp>’</dt> <dd> each channel is displayed in a separate color </dd> <dt>‘<samp class="samp">intensity</samp>’</dt> <dd> each channel is displayed using the same color scheme </dd> <dt>‘<samp class="samp">rainbow</samp>’</dt> <dd> each channel is displayed using the rainbow color scheme </dd> <dt>‘<samp class="samp">moreland</samp>’</dt> <dd> each channel is displayed using the moreland color scheme </dd> <dt>‘<samp class="samp">nebulae</samp>’</dt> <dd> each channel is displayed using the nebulae color scheme </dd> <dt>‘<samp class="samp">fire</samp>’</dt> <dd> each channel is displayed using the fire color scheme </dd> <dt>‘<samp class="samp">fiery</samp>’</dt> <dd> each channel is displayed using the fiery color scheme </dd> <dt>‘<samp class="samp">fruit</samp>’</dt> <dd> each channel is displayed using the fruit color scheme </dd> <dt>‘<samp class="samp">cool</samp>’</dt> <dd> each channel is displayed using the cool color scheme </dd> <dt>‘<samp

class=”samp”>magma</samp>’</dt> <dd> each channel is displayed using the magma color scheme </dd> <dt>‘<samp class=”samp”>green</samp>’</dt> <dd> each channel is displayed using the green color scheme </dd> <dt>‘<samp class=”samp”>viridis</samp>’</dt> <dd> each channel is displayed using the viridis color scheme </dd> <dt>‘<samp class=”samp”>plasma</samp>’</dt> <dd> each channel is displayed using the plasma color scheme </dd> <dt>‘<samp class=”samp”>cividis</samp>’</dt> <dd> each channel is displayed using the cividis color scheme </dd> <dt>‘<samp class=”samp”>terrain</samp>’</dt> <dd> each channel is displayed using the terrain color scheme </dd> </dl>

Default value is ‘<samp class=”samp”>channel</samp>’. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Specify scale used for calculating intensity color values. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> linear </dd> <dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> square root, default </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt> <dd> cubic root </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> logarithmic </dd> <dt>‘<samp class=”samp”>4thrt</samp>’</dt> <dd> 4th root </dd> <dt>‘<samp class=”samp”>5thrt</samp>’</dt> <dd> 5th root </dd> </dl>

Default value is ‘<samp class=”samp”>sqrt</samp>’. </dd> <dt><samp class=”option”>fscale</samp></dt> <dd> Specify frequency scale. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> linear </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> logarithmic </dd> </dl>

Default value is ‘<samp class=”samp”>lin</samp>’. </dd> <dt><samp class=”option”>saturation</samp></dt> <dd> Set saturation modifier for displayed colors. Negative values provide alternative color scheme. <code class=”code”>0</code> is no saturation at all. Saturation must be in [-10.0, 10.0] range. Default value is <code class=”code”>1</code>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function. It accepts the following values: <dl class=”table”> <dt>‘<samp class=”samp”>rect</samp>’</dt> <dt>‘<samp class=”samp”>bartlett</samp>’</dt> <dt>‘<samp class=”samp”>hann</samp>’</ <dt>‘<samp class=”samp”>hanning</samp>’</dt> <dt>‘<samp class=”samp”>hamming</samp>’</dt> <dt>‘<samp class=”samp”>blackman</samp>’</dt> <dt>‘<samp class=”samp”>welch</samp>’</dt> <dt>‘<samp class=”samp”>flattop</samp>’</ <dt>‘<samp class=”samp”>bharris</samp>’</dt> <dt>‘<samp

class=”samp”>bnuttall</samp>’</dt> <dt>‘<samp class=”samp”>bhann</samp>’<
<dt>‘<samp class=”samp”>sine</samp>’</dt> <dt>‘<samp
class=”samp”>nuttall</samp>’</dt> <dt>‘<samp class=”samp”>lanczos</samp>’<
<dt>‘<samp class=”samp”>gauss</samp>’</dt> <dt>‘<samp
class=”samp”>tukey</samp>’</dt> <dt>‘<samp class=”samp”>dolph</samp>’</d
<dt>‘<samp class=”samp”>cauchy</samp>’</dt> <dt>‘<samp
class=”samp”>parzen</samp>’</dt> <dt>‘<samp class=”samp”>poisson</samp>’<
<dt>‘<samp class=”samp”>bohman</samp>’</dt> <dt>‘<samp
class=”samp”>kaiser</samp>’</dt> </dl>

Default value is <code class=”code”>hann</code>. </dd>
<dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of time vs frequency axis. Can be <code class=”code”>vertical</code>
or <code class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>.
 </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set
ratio of overlap window. Default value is <code class=”code”>0</code>.
When value is <code class=”code”>1</code> overlap is set to recommended
size for specific window function currently used. </dd> <dt><samp
class=”option”>gain</samp></dt> <dd> Set scale gain for calculating
intensity color values. Default value is <code class=”code”>1</code>.
</dd> <dt><samp class=”option”>data</samp></dt> <dd> Set
which data to display. Can be <code class=”code”>magnitude</code>,
default or <code class=”code”>phase</code>, or unwrapped phase: <code
class=”code”>uphase</code>. </dd> <dt><samp class=”option”>rotation</samp></dt>
<dd> Set color rotation, must be in [-1.0, 1.0] range. Default
value is <code class=”code”>0</code>. </dd> <dt><samp
class=”option”>start</samp></dt> <dd> Set start frequency from
which to display spectrogram. Default is <code class=”code”>0</code>.
 </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set
stop frequency to which to display spectrogram. Default is <code
class=”code”>0</code>. </dd> <dt><samp class=”option”>fps</samp></dt>
<dd> Set upper frame rate limit. Default is <code class=”code”>auto</code>,
unlimited. </dd> <dt><samp class=”option”>legend</samp></dt>
<dd> Draw time and frequency axes and legends. Default is disabled. </dd> <dt><samp class=”option”>drange</samp></dt>
<dd> Set dynamic range used to calculate intensity color values. Default
is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp
class=”option”>limit</samp></dt> <dd> Set upper limit of input
audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100
to 100. </dd> <dt><samp class=”option”>opacity</samp></dt>
<dd> Set opacity strength when using pixel format output with alpha
component. </dd> </dl>

The usage is very similar to the showwaves filter; see the examples in that section. <h4
class=”subsection”>41.26.1 Examples<a class=”anchor
hidden-xs” href=”#Examples-201” aria-hidden=”true”># <a class=”anchor

hidden-xs”href=”#toc-Examples-201” aria-hidden=”true”>TOC</h4>

<ul class=”itemize mark-bullet”> Large window with logarithmic color scaling: <pre class=”example-preformatted”>showspectrum=s=1280x480:scale=log
</pre>

Complete example for a colored and sliding spectrum per channel using <code class=”command”>ffplay</code>: <pre class=”example-preformatted”>ffplay -f lavfi 'amovie=input.mp3, asplit [a][out1]; [a] showspectrum=mode=separate:color=intensity:slide=1:scale=cbrt [out0]' </pre>

 <h3 class=”section”>41.27 showspectrumpic# TOC</h3>

Convert input audio to a single video frame, representing the audio frequency

spectrum.

The filter accepts the following options:

<dl class=”table”>

<dt><samp class=”option”>size, s</samp></dt>

<dd> Specify the video size for the output. For the syntax of this option, check the

<a data-manual=”ffmpeg-utils” href=”./ffmpeg-utils.html#video-size-syntax”>"Video size" section in the ffmpeg-utils manual.

Default value is <code class=”code”>4096x2048</code>.

</dd>

<dt><samp class=”option”>mode</samp></dt>

<dd> Specify display mode.

It accepts the following values:

<dl class=”table”>

<dt>‘<samp class=”samp”>combined</samp>’</dt>

<dd> all channels are displayed in the same row

</dd>
<dt>‘<samp class="samp">separate</samp>’</dt>
<dd> all channels are displayed in separate rows
</dd>
</dl>
Default value is ‘<samp class="samp">combined</samp>’.

</dd>
<dt><samp class="option">color</samp></dt>
<dd> Specify display color mode.

It accepts the following values:
<dl class="table">
<dt>‘<samp class="samp">channel</samp>’</dt>
<dd> each channel is displayed in a separate color
</dd>
<dt>‘<samp class="samp">intensity</samp>’</dt>
<dd> each channel is displayed using the same color scheme
</dd>
<dt>‘<samp class="samp">rainbow</samp>’</dt>
<dd> each channel is displayed using the rainbow color scheme
</dd>
<dt>‘<samp class="samp">moreland</samp>’</dt>
<dd> each channel is displayed using the moreland color scheme
</dd>
<dt>‘<samp class="samp">nebulae</samp>’</dt>
<dd> each channel is displayed using the nebulae color scheme
</dd>
<dt>‘<samp class="samp">fire</samp>’</dt>
<dd> each channel is displayed using the fire color scheme
</dd>

<dt>‘<samp class="samp">fiery</samp>’</dt>
<dd> each channel is displayed using the fiery color scheme
</dd>
<dt>‘<samp class="samp">fruit</samp>’</dt>
<dd> each channel is displayed using the fruit color scheme
</dd>
<dt>‘<samp class="samp">cool</samp>’</dt>
<dd> each channel is displayed using the cool color scheme
</dd>
<dt>‘<samp class="samp">magma</samp>’</dt>
<dd> each channel is displayed using the magma color scheme
</dd>
<dt>‘<samp class="samp">green</samp>’</dt>
<dd> each channel is displayed using the green color scheme
</dd>
<dt>‘<samp class="samp">viridis</samp>’</dt>
<dd> each channel is displayed using the viridis color scheme
</dd>
<dt>‘<samp class="samp">plasma</samp>’</dt>
<dd> each channel is displayed using the plasma color scheme
</dd>
<dt>‘<samp class="samp">cividis</samp>’</dt>
<dd> each channel is displayed using the cividis color scheme
</dd>
<dt>‘<samp class="samp">terrain</samp>’</dt>
<dd> each channel is displayed using the terrain color scheme
</dd>
</dl>
Default value is ‘<samp class="samp">intensity</samp>’.

</dd>

<dt><samp class=”option”>scale</samp></dt>
<dd> Specify scale used for calculating intensity color values.

It accepts the following values:
<dl class=”table”>
<dt>‘<samp class=”samp”>lin</samp>’</dt>
<dd> linear
</dd>
<dt>‘<samp class=”samp”>sqrt</samp>’</dt>
<dd> square root, default
</dd>
<dt>‘<samp class=”samp”>cbrt</samp>’</dt>
<dd> cubic root
</dd>
<dt>‘<samp class=”samp”>log</samp>’</dt>
<dd> logarithmic
</dd>
<dt>‘<samp class=”samp”>4thrt</samp>’</dt>
<dd> 4th root
</dd>
<dt>‘<samp class=”samp”>5thrt</samp>’</dt>
<dd> 5th root
</dd>
</dl>
Default value is ‘<samp class=”samp”>log</samp>’.

</dd>
<dt><samp class=”option”>fscale</samp></dt>
<dd> Specify frequency scale.

It accepts the following values:

<dl class="table">

<dt>‘<samp class="samp">lin</samp>’</dt>

<dd> linear

</dd>

<dt>‘<samp class="samp">log</samp>’</dt>

<dd> logarithmic

</dd>

</dl>

Default value is ‘<samp class="samp">lin</samp>’.
 </dd> <dt><samp class="option">saturation</samp></dt>
<dd> Set saturation modifier for displayed colors. Negative values
provide alternative color scheme. <code class="code">0</code> is no
saturation at all. Saturation must be in [-10.0, 10.0] range. Default
value is <code class="code">1</code>. </dd> <dt><samp
class="option">win_func</samp></dt> <dd> Set window func-
tion. It accepts the following values: <dl class="table">
<dt>‘<samp class="samp">rect</samp>’</dt> <dt>‘<samp
class="samp">bartlett</samp>’</dt> <dt>‘<samp class="samp">hann</samp>’</
<dt>‘<samp class="samp">hanning</samp>’</dt> <dt>‘<samp
class="samp">hamming</samp>’</dt> <dt>‘<samp
class="samp">blackman</samp>’</dt> <dt>‘<samp
class="samp">welch</samp>’</dt> <dt>‘<samp class="samp">flattop</samp>’</
<dt>‘<samp class="samp">bharris</samp>’</dt> <dt>‘<samp
class="samp">bnuttall</samp>’</dt> <dt>‘<samp class="samp">bhann</samp>’<
<dt>‘<samp class="samp">sine</samp>’</dt> <dt>‘<samp
class="samp">nuttall</samp>’</dt> <dt>‘<samp class="samp">lanczos</samp>’<
<dt>‘<samp class="samp">gauss</samp>’</dt> <dt>‘<samp
class="samp">tukey</samp>’</dt> <dt>‘<samp class="samp">dolph</samp>’</d
<dt>‘<samp class="samp">cauchy</samp>’</dt> <dt>‘<samp
class="samp">parzen</samp>’</dt> <dt>‘<samp class="samp">poisson</samp>’<
<dt>‘<samp class="samp">bohman</samp>’</dt> <dt>‘<samp
class="samp">kaiser</samp>’</dt> </dl> Default value
is <code class="code">hann</code>. </dd> <dt><samp
class="option">orientation</samp></dt> <dd> Set orientation of
time vs frequency axis. Can be <code class="code">vertical</code> or <code
class="code">horizontal</code>. Default is <code class="code">vertical</code>.
 </dd> <dt><samp class="option">gain</samp></dt> <dd> Set
scale gain for calculating intensity color values. Default value is <code
class="code">1</code>. </dd> <dt><samp class="option">legend</samp></dt>
<dd> Draw time and frequency axes and legends. Default is en-
abled. </dd> <dt><samp class="option">rotation</samp></dt>
<dd> Set color rotation, must be in [-1.0, 1.0] range. Default

value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>start</samp></dt> <dd> Set start frequency from which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>stop</samp></dt> <dd> Set stop frequency to which to display spectrogram. Default is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>drange</samp></dt> <dd> Set dynamic range used to calculate intensity color values. Default is 120 dBFS. Allowed range is from 10 to 200. </dd> <dt><samp class=”option”>limit</samp></dt> <dd> Set upper limit of input audio samples volume in dBFS. Default is 0 dBFS. Allowed range is from -100 to 100. </dd> <dt><samp class=”option”>opacity</samp></dt> <dd> Set opacity strength when using pixel format output with alpha component. </dd> </dl>

 <h4 class=”subsection”>41.27.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Extract an audio spectrogram of a whole audio track in a 1024x1024 picture using <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i audio.flac -lavfi showspectrumpic=s=1024x1024 spectrogram.png </pre>

 <h3 class=”section”>41.28 showvolume# TOC</h3>

Convert input audio volume to a video output. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set video rate. </dd> <dt><samp class=”option”>b</samp></dt> <dd> Set border width, allowed range is [0, 5]. Default is 1. </dd> <dt><samp class=”option”>w</samp></dt> <dd> Set channel width, allowed range is [80, 8192]. Default is 400. </dd> <dt><samp class=”option”>h</samp></dt> <dd> Set channel height, allowed range is [1, 900]. Default is 20. </dd> <dt><samp class=”option”>f</samp></dt> <dd> Set fade, allowed range is [0, 1]. Default is 0.95. </dd> <dt><samp class=”option”>c</samp></dt> <dd> Set volume color expression. The expression can use the following variables: <dl class=”table”> <dt><samp class=”option”>VOLUME</samp></dt> <dd> Current max volume of channel in dB. </dd> <dt><samp class=”option”>PEAK</samp></dt> <dd> Current peak. </dd> <dt><samp class=”option”>CHANNEL</samp></dt> <dd> Current channel number, starting from 0. </dd> </dl>

</dd> <dt><samp class="option">t</samp></dt> <dd> If set, displays channel names. Default is enabled. </dd> <dt><samp class="option">v</samp></dt> <dd> If set, displays volume values. Default is enabled. </dd> <dt><samp class="option">o</samp></dt> <dd> Set orientation, can be horizontal: <code class="code">h</code> or vertical: <code class="code">v</code>, default is <code class="code">h</code>. </dd> <dt><samp class="option">s</samp></dt> <dd> Set step size, allowed range is [0, 5]. Default is 0, which means step is disabled. </dd> <dt><samp class="option">p</samp></dt> <dd> Set background opacity, allowed range is [0, 1]. Default is 0. </dd> <dt><samp class="option">m</samp></dt> <dd> Set metering mode, can be peak: <code class="code">p</code> or rms: <code class="code">r</code>, default is <code class="code">p</code>. </dd> <dt><samp class="option">ds</samp></dt> <dd> Set display scale, can be linear: <code class="code">lin</code> or log: <code class="code">log</code>, default is <code class="code">lin</code>. </dd> <dt><samp class="option">dm</samp></dt> <dd> In second. If set to > 0., display a line for the max level in the previous seconds. default is disabled: <code class="code">0.</code> </dd> <dt><samp class="option">dmc</samp></dt> <dd> The color of the max line. Use when <code class="code">dm</code> option is set to > 0. default is: <code class="code">orange</code> </dd> </dl>

 <h3 class="section">41.29 showwaves# TOC</h3>

Convert input audio to a video output, representing the samples waves. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">600x240</code>. </dd> <dt><samp class="option">mode</samp></dt> <dd> Set display mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">point</samp>’</dt> <dd> Draw a point for each sample. </dd> <dt>‘<samp class="samp">line</samp>’</dt> <dd> Draw a vertical line for each sample. </dd> <dt>‘<samp class="samp">p2p</samp>’</dt> <dd> Draw a point for each sample and a line between them. </dd> <dt>‘<samp class="samp">cline</samp>’</dt> <dd> Draw a centered vertical line for each sample. </dd> </dl>

Default value is <code class="code">point</code>. </dd> <dt><samp class="option">n</samp></dt> <dd> Set the number

of samples which are printed on the same column. A larger value will decrease the frame rate. Must be a positive integer. This option can be set only if the value for <var class=”var”>rate</var> is not explicitly specified. </dd> <dt><samp class=”option”>rate, r</samp></dt> <dd> Set the (approximate) output frame rate. This is done by setting the option <var class=”var”>n</var>. Default value is "25". </dd> <dt><samp class=”option”>split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class=”option”>colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class=”samp”>log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class=”samp”>sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class=”samp”>cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class=”option”>draw</samp></dt> <dd> Set the draw mode. This is mostly useful to set for high <var class=”var”>n</var>. Available values are: <dl class=”table”> <dt>‘<samp class=”samp”>scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd> <dt>‘<samp class=”samp”>full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class=”code”>scale</code>. </dd> </dl>

 <h4 class=”subsection”>41.29.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Output the input file audio and the corresponding video representation at the same time: <pre class=”example-preformatted”>amovie=a.mp3,asplit[out0],showwaves[out1] </pre>

Create a synthetic signal and show it with showwaves, forcing a frame rate of 30 frames per second: <pre class=”example-preformatted”>aevalsrc=sin(1*2*PI*t)*sin(880*2*PI*t):cos(2*PI*200*t),asplit[out0],showwaves=r=30[out1] </pre>

 <h3 class=”section”>41.30 showwavespic# TOC</h3>

Convert input audio to a single video frame, representing the samples waves. The filter accepts the following options: <dl class="table"> <dt><samp class="option">size, s</samp></dt> <dd> Specify the video size for the output. For the syntax of this option, check the <a data-manual="ffmpeg-utils" href="./ffmpeg-utils.html#video-size-syntax">"Video size" section in the ffmpeg-utils manual. Default value is <code class="code">600x240</code>. </dd> <dt><samp class="option">split_channels</samp></dt> <dd> Set if channels should be drawn separately or overlap. Default value is 0. </dd> <dt><samp class="option">colors</samp></dt> <dd> Set colors separated by ’|’ which are going to be used for drawing of each channel. </dd> <dt><samp class="option">scale</samp></dt> <dd> Set amplitude scale. Available values are: <dl class="table"> <dt>‘<samp class="samp">lin</samp>’</dt> <dd> Linear. </dd> <dt>‘<samp class="samp">log</samp>’</dt> <dd> Logarithmic. </dd> <dt>‘<samp class="samp">sqrt</samp>’</dt> <dd> Square root. </dd> <dt>‘<samp class="samp">cbrt</samp>’</dt> <dd> Cubic root. </dd> </dl>

Default is linear. </dd> <dt><samp class="option">draw</samp></dt> <dd> Set the draw mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">scale</samp>’</dt> <dd> Scale pixel values for each drawn sample. </dd> <dt>‘<samp class="samp">full</samp>’</dt> <dd> Draw every sample directly. </dd> </dl>

Default value is <code class="code">scale</code>. </dd> <dt><samp class="option">filter</samp></dt> <dd> Set the filter mode. Available values are: <dl class="table"> <dt>‘<samp class="samp">average</samp>’</dt> <dd> Use average samples values for each drawn sample. </dd> <dt>‘<samp class="samp">peak</samp>’</dt> <dd> Use peak samples values for each drawn sample. </dd> </dl>

Default value is <code class="code">average</code>. </dd> </dl>

 <h4 class="subsection">41.30.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Extract a channel split representation of the wave form of a whole audio track in a 1024x800 picture using <code class="command">ffmpeg</code>: <pre class="example-preformatted">ffmpeg -i audio.flac -lavfi showwavespic=split_channels=1:s=1024x800 waveform.png </pre>

 <h3 class="section">41.31 sidedata, asidedata# TOC</h3>

Delete frame side data, or select frames based on it. This filter accepts the following options: <dl class="table"> <dt><samp class="option">mode</samp></dt> <dd> Set mode of operation of the filter. Can be one of the following: <dl class="table"> <dt>‘<samp class="samp">select</samp>’</dt> <dd> Select every frame with side data of <code class="code">type</code>. </dd> <dt>‘<samp class="samp">delete</samp>’</dt> <dd> Delete side data of <code class="code">type</code>. If <code class="code">type</code> is not set, delete all side data in the frame. </dd> </dl>

</dd> <dt><samp class="option">type</samp></dt> <dd> Set side data type used with all modes. Must be set for <code class="code">select</code> mode. Possible values are: <dl class="table"> <dt>‘<samp class="samp">PANSCAN</samp>’</dt> <dt>‘<samp class="samp">A53_CC</samp>’</dt> <dt>‘<samp class="samp">STEREO3D</samp>’</dt> <dt>‘<samp class="samp">MATRIXENCODING</samp>’</dt> <dt>‘<samp class="samp">DOWNMIX_INFO</samp>’</dt> <dt>‘<samp class="samp">REPLAYGAIN</samp>’</dt> <dt>‘<samp class="samp">DISPLAYMATRIX</samp>’</dt> <dt>‘<samp class="samp">AFD</samp>’</dt> <dt>‘<samp class="samp">MOTION_VECTORS</sa
<dt>‘<samp class="samp">SKIP_SAMPLES</samp>’</dt>
<dt>‘<samp class="samp">AUDIO_SERVICE_TYPE</samp>’</dt>
<dt>‘<samp class="samp">MASTERING_DISPLAY_METADATA</samp>’</dt>
<dt>‘<samp class="samp">GOP_TIMECODE</samp>’</dt>
<dt>‘<samp class="samp">SPHERICAL</samp>’</dt>
<dt>‘<samp class="samp">CONTENT_LIGHT_LEVEL</samp>’</dt>
<dt>‘<samp class="samp">ICC_PROFILE</samp>’</dt>
<dt>‘<samp class="samp">S12M_TIMECODE</samp>’</dt>
<dt>‘<samp class="samp">DYNAMIC_HDR_PLUS</samp>’</dt>
<dt>‘<samp class="samp">REGIONS_OF_INTEREST</samp>’</dt>
<dt>‘<samp class="samp">VIDEO_ENC_PARAMS</samp>’</dt>
<dt>‘<samp class="samp">SEI_UNREGISTERED</samp>’</dt>
<dt>‘<samp class="samp">FILM_GRAIN_PARAMS</samp>’</dt>
<dt>‘<samp class="samp">DETECTION_BOUNDING_BOXES</samp>’</dt>
<dt>‘<samp class="samp">DETECTION_BBOXES</samp>’</dt>
<dt>‘<samp class="samp">DOVI_RPU_BUFFER</samp>’</dt>
<dt>‘<samp class="samp">DOVI_METADATA</samp>’</dt>
<dt>‘<samp class="samp">DYNAMIC_HDR_VIVID</samp>’</dt>
<dt>‘<samp class="samp">AMBIENT_VIEWING_ENVIRONMENT</samp>’</dt>
<dt>‘<samp class="samp">VIDEO_HINT</samp>’</dt>

<dt>‘<samp class=”samp”>GAIN_MAP_PARAMS</samp>’</dt> </dl>

</dd> </dl>

 <h3 class=”section”>41.32 spectrumsynth# TOC</h3>

Synthesize audio from 2 input video spectrums, first input stream represents magnitude across time and second represents phase across time. The filter will transform from frequency domain as displayed in videos back to time domain as presented in audio output. This filter is primarily created for reversing processed showspectrum filter outputs, but can synthesize sound from other spectrograms too. But in such case results are going to be poor if the phase data is not available, because in such cases phase data need to be recreated, usually it’s just recreated from random noise. For best results use gray only output (<code class=”code”>channel</code> color mode in showspectrum filter) and <code class=”code”>log</code> scale for magnitude video and <code class=”code”>lin</code> scale for phase video. To produce phase, for 2nd video, use <code class=”code”>data</code> option. Inputs videos should generally use <code class=”code”>fullframe</code> slide mode as that saves resources needed for decoding video. The filter accepts the following options: <dl class=”table”> <dt><samp class=”option”>sample_rate</samp></dt> <dd> Specify sample rate of output audio, the sample rate of audio from which spectrum was generated may differ. </dd> <dt><samp class=”option”>channels</samp></dt> <dd> Set number of channels represented in input video spectrums. </dd> <dt><samp class=”option”>scale</samp></dt> <dd> Set scale which was used when generating magnitude input spectrum. Can be <code class=”code”>lin</code> or <code class=”code”>log</code>. Default is <code class=”code”>log</code>. </dd> <dt><samp class=”option”>slide</samp></dt> <dd> Set slide which was used when generating inputs spectrums. Can be <code class=”code”>replace</code>, <code class=”code”>scroll</code>, <code class=”code”>fullframe</code> or <code class=”code”>rscroll</code>. Default is <code class=”code”>fullframe</code>. </dd> <dt><samp class=”option”>win_func</samp></dt> <dd> Set window function used for resynthesis. </dd> <dt><samp class=”option”>overlap</samp></dt> <dd> Set window overlap. In range <code class=”code”>[0, 1]</code>. Default is <code class=”code”>1</code>, which means optimal overlap for selected window function will be picked. </dd> <dt><samp class=”option”>orientation</samp></dt> <dd> Set orientation of input videos. Can be <code class=”code”>vertical</code> or <code

class=”code”>horizontal</code>. Default is <code class=”code”>vertical</code>.
</dd> </dl>

 <h4 class=”subsection”>41.32.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> First create magnitude and phase videos from audio, assuming audio is stereo with 44100 sample rate, then resynthesize videos back to audio with spectrumsynth: <pre class=”example-preformatted”>ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=log:overlap=0.875:color=channel:slide=fullframe:data=magnitude -an -c:v rawvideo magnitude.nut ffmpeg -i input.flac -lavfi showspectrum=mode=separate:scale=lin:overlap=0.875:color=channel:slide=fullframe:data=phase -an -c:v rawvideo phase.nut ffmpeg -i magnitude.nut -i phase.nut -lavfi spectrumsynth=channels=2:sample_rate=44100:win_func=hann:overlap=0.875:slide=fullframe output.flac </pre>

 <h3 class=”section”>41.33 split, asplit# TOC</h3>

Split input into several identical outputs. <code class=”code”>asplit</code> works with audio input, <code class=”code”>split</code> with video. The filter accepts a single parameter which specifies the number of outputs. If unspecified, it defaults to 2. <h4 class=”subsection”>41.33.1 Examples# TOC</h4>

<ul class=”itemize mark-bullet”> Create two separate outputs from the same input: <pre class=”example-preformatted”>[in] split [out0][out1] </pre>

To create 3 or more outputs, you need to specify the number of outputs, like in: <pre class=”example-preformatted”>[in] asplit=3 [out0][out1][out2] </pre>

Create two separate outputs from the same input, one cropped and one padded: <pre class=”example-preformatted”>[in] split [splitout1][splitout2]; [splitout1] crop=100:100:0:0 [cropout]; [splitout2] pad=200:200:100:100 [padout]; </pre>

Create 5 copies of the input audio with <code class=”command”>ffmpeg</code>: <pre class=”example-preformatted”>ffmpeg -i INPUT -filter_complex asplit=5 OUTPUT </pre>

 <h3 class=”section”>41.34 zmq,

azmq# TOC</h3>

Receive commands sent through a libzmq client, and forward them to filters in the filtergraph. <code class=”code”>zmq</code> and <code class=”code”>azmq</code> work as a pass-through filters. <code class=”code”>zmq</code> must be inserted between two video filters, <code class=”code”>azmq</code> between two audio filters. Both are capable to send messages to any filter type. To enable these filters you need to install the libzmq library and headers and configure FFmpeg with <code class=”code”>--enable-libzmq</code>. For more information about libzmq see: http://www.zeromq.org/ The <code class=”code”>zmq</code> and <code class=”code”>azmq</code> filters work as a libzmq server, which receives messages sent through a network interface defined by the <samp class=”option”>bind_address</samp> (or the abbreviation "<samp class=”option”>b</samp>") option. Default value of this option is <samp class=”file”>tcp://localhost:5555</samp>. You may want to alter this value to your needs, but do not forget to escape any ’:’ signs (see filtergraph escaping). The received message must be in the form: <pre class=”example-preformatted”><var class=”var”>TARGET</var> <var class=”var”>COMMAND</var> [<var class=”var”>ARG</var>] </pre>

<var class=”var”>TARGET</var> specifies the target of the command, usually the name of the filter class or a specific filter instance name. The default filter instance name uses the pattern ‘<samp class=”samp”>Parsed_<filter_name>_<index></samp>’, but you can override this by using the ‘<samp class=”samp”>filter_name@id</samp>’ syntax (see Filtergraph syntax). <var class=”var”>COMMAND</var> specifies the name of the command for the target filter. <var class=”var”>ARG</var> is optional and specifies the optional argument list for the given <var class=”var”>COMMAND</var>. Upon reception, the message is processed and the corresponding command is injected into the filtergraph. Depending on the result, the filter will send a reply to the client, adopting the format: <pre class=”example-preformatted”><var class=”var”>ERROR_CODE</var> <var class=”var”>ERROR_REASON</var> <var class=”var”>MESSAGE</var> </pre>

<var class=”var”>MESSAGE</var> is optional. <h4 class=”subsection”>41.34.1 Examples# TOC</h4>

Look at <samp class=”file”>tools/zmqsend</samp> for an example of a zmq client which can be used to send commands processed by these filters. Consider the following filtergraph generated by <code class=”command”>ffplay</code>. In this example the last overlay filter has an instance name. All other filters will have default instance names.   <pre class=”example-preformatted”>ffplay -dumpgraph 1 -f lavfi " color=s=100x100:c=red [l]; color=s=100x100:c=blue [r]; nullsrc=s=200x100, zmq [bg]; [bg][l] overlay [bg+l]; [bg+l][r] overlay@my=x=100 " </pre>

To change the color of the left side of the video, the following command can be used: <pre class=”example-preformatted”>echo Parsed_color_0 c yellow | tools/zmqsend </pre>

To change the right side: <pre class=”example-preformatted”>echo Parsed_color_1 c pink | tools/zmqsend </pre>

To change the position of the right side: <pre class=”example-preformatted”>echo overlay@my x 150 | tools/zmqsend </pre>

 <h2 class=”chapter”>42 Multimedia Sources# TOC</h2>

Below is a description of the currently available multimedia sources. <h3 class=”section”>42.1 amovie# TOC</h3>

This is the same as movie source, except it selects an audio stream by default. <h3 class=”section”>42.2 avsynctest# TOC</h3>
Generate an Audio/Video Sync Test. Generated stream periodically shows flash video frame and emits beep in audio. Useful to inspect A/V sync issues. It accepts the following options: <dl class=”table”> <dt><samp class=”option”>size, s</samp></dt> <dd> Set output video size. Default value is <code class=”code”>hd720</code>. </dd> <dt><samp class=”option”>framerate, fr</samp></dt> <dd> Set output video frame rate. Default value is <code class=”code”>30</code>. </dd> <dt><samp class=”option”>samplerate, sr</samp></dt> <dd> Set output audio sample rate. Default value is <code class=”code”>44100</code>. </dd> <dt><samp class=”option”>amplitude, a</samp></dt> <dd> Set output audio beep amplitude. Default value is <code

class=”code”>0.7</code>. </dd> <dt><samp class=”option”>period, p</samp></dt> <dd> Set output audio beep period in seconds. Default value is <code class=”code”>3</code>. </dd> <dt><samp class=”option”>delay, dl</samp></dt> <dd> Set output video flash delay in number of frames. Default value is <code class=”code”>0</code>. </dd> <dt><samp class=”option”>cycle, c</samp></dt> <dd> Enable cycling of video delays, by default is disabled. </dd> <dt><samp class=”option”>duration, d</samp></dt> <dd> Set stream output duration. By default duration is unlimited. </dd> <dt><samp class=”option”>fg, bg, ag</samp></dt> <dd> Set foreground/background/additional color. </dd> </dl>

 <h4 class=”subsection”>42.2.1 Commands# TOC</h4>

This source supports the some above options as commands. <h3 class=”section”>42.3 movie# TOC</h3>

Read audio and/or video stream(s) from a movie container. It accepts the following parameters: <dl class=”table”> <dt><samp class=”option”>filename</samp></dt> <dd> The name of the resource to read (not necessarily a file; it can also be a device or a stream accessed through some protocol). </dd> <dt><samp class=”option”>format_name, f</samp></dt> <dd> Specifies the format assumed for the movie to read, and can be either the name of a container or an input device. If not specified, the format is guessed from <var class=”var”>movie_name</var> or by probing. </dd> <dt><samp class=”option”>seek_point, sp</samp></dt> <dd> Specifies the seek point in seconds. The frames will be output starting from this seek point. The parameter is evaluated with <code class=”code”>av_strtod</code>, so the numerical value may be suffixed by an IS postfix. The default value is "0". </dd> <dt><samp class=”option”>streams, s</samp></dt> <dd> Specifies the streams to read. Several streams can be specified, separated by "+". The source will then have as many outputs, in the same order. The syntax is explained in the <a data-manual=”ffmpeg” href=”./ffmpeg.html#Stream-specifiers”>"Stream specifiers" section in the ffmpeg manual. Two special names, "dv" and "da" specify respectively the default (best suited) video and audio stream. Default is "dv", or "da" if the filter is called as "amovie". </dd> <dt><samp class=”option”>stream_index, si</samp></dt> <dd> Specifies the index of the video stream to read. If the value is -1, the most suitable video stream will be automatically selected. The

default value is "-1". Deprecated. If the filter is called "amovie", it will select audio instead of video. </dd> <dt><samp class="option">loop</samp></dt> <dd> Specifies how many times to read the stream in sequence. If the value is 0, the stream will be looped infinitely. Default value is "1". Note that when the movie is looped the source timestamps are not changed, so it will generate non monotonically increasing timestamps. </dd> <dt><samp class="option">discontinuity</samp></dt> <dd> Specifies the time difference between frames above which the point is considered a timestamp discontinuity which is removed by adjusting the later timestamps. </dd> <dt><samp class="option">dec_threads</samp></dt> <dd> Specifies the number of threads for decoding </dd> <dt><samp class="option">format_opts</samp></dt> <dd> Specify format options for the opened file. Format options can be specified as a list of <var class="var">key</var>=<var class="var">value</var> pairs separated by ’:’. The following example shows how to add protocol_whitelist and protocol_blacklist options: <pre class="example-preformatted">ffplay -f lavfi "movie=filename='1.sdp':format_opts='protocol_whitelist=file,rtp,udp\:protocol_blacklist=http'&quo
</pre> </dd> </dl>

It allows overlaying a second video on top of the main input of a filtergraph, as shown in this graph: <pre class="example-preformatted">input ----------> deltapts0 --> overlay --> output ^ | movie --> scale--> deltapts1 -------+ </pre> <h4 class="subsection">42.3.1 Examples# TOC</h4>

<ul class="itemize mark-bullet"> Skip 3.2 seconds from the start of the AVI file in.avi, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=in.avi:seek_point=3.2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read from a video4linux2 device, and overlay it on top of the input labelled "in": <pre class="example-preformatted">movie=/dev/video0:f=video4linux2, scale=180:-1, setpts=PTS-STARTPTS [over]; [in] setpts=PTS-STARTPTS [main]; [main][over] overlay=16:16 [out] </pre>

Read the first video stream and the audio stream with id 0x81 from dvd.vob; the video is connected to the pad named "video" and the audio is connected to the pad named "audio": <pre class="example-preformatted">movie=dvd.vob:s=v:0+#0x81 [video] [audio] </pre>

 <h4 class="subsection">42.3.2 Commands# TOC</h4>

Both movie and amovie support the following commands: <dl class="table"> <dt><samp class="option">seek</samp></dt> <dd> Perform seek using "av_seek_frame". The syntax is: seek <var class="var">stream_index</var>|<var class="var">timestamp</var>|<var class="var">flags</var> <ul class="itemize mark-bullet"> <var class="var">stream_index</var>: If stream_index is -1, a default stream is selected, and <var class="var">timestamp</var> is automatically converted from AV_TIME_BASE units to the stream specific time_base. <var class="var">timestamp</var>: Timestamp in AVStream.time_base units or, if no stream is specified, in AV_TIME_BASE units. <var class="var">flags</var>: Flags which select direction and seeking mode.

</dd> <dt><samp class="option">get_duration</samp></dt> <dd> Get movie duration in AV_TIME_BASE units. </dd> </dl>

 <h2 class="chapter">43 External libraries# TOC</h2>

FFmpeg can be hooked up with a number of external libraries to add support for more formats. None of them are used by default, their use has to be explicitly requested by passing the appropriate flags to <code class="command">./configure</code>. <h3 class="section">43.1 Alliance for Open Media (AOM)# TOC</h3>

FFmpeg can make use of the AOM library for AV1 decoding and encoding. Go to http://aomedia.org/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libaom</code> to configure to enable it. <h3 class="section">43.2 ARM ASTC Encoder# TOC</h3>

FFmpeg can make use of the Arm ASTC Encoder library for ASTC image compression and decompression in the <code class="code">libastcenc</code> encoder and decoder. Go to https://github.com/ARM-software/astc-encoder

to obtain the sources. Version 5.4 or newer is required, because the wrappers use the four-argument <code class=”code”>astcenc_context_alloc()</code> interface introduced in that release. The upstream build does not install the public header, and the library file name depends on the configured SIMD and linkage variants, so install the pieces manually in a layout that configure can detect: the header as <code class=”code”>astcenc/astcenc.h</code> and a library that can be linked with <code class=”code”>-lastcenc</code>. Keep the library’s own file name, which is the SONAME recorded in the binary, and provide the plain name as an alias: <pre class=”example-preformatted”>cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DASTCENC_SHAREDLIB=ON cmake --build build install -Dm644 Source/astcenc.h /usr/local/include/astcenc/astcenc.h install -Dm755 build/Source/libastcenc-native-shared.so \ /usr/local/lib/libastcenc-native-shared.so ln -sf libastcenc-native-shared.so /usr/local/lib/libastcenc.so </pre>

Use the platform’s shared library suffix, and adjust the built library name for other variants (for example <samp class=”file”>libastcenc-neon-shared.so</samp> or a static <samp class=”file”>libastcenc.a</samp>), or pass <code class=”code”>--extra-cflags</code> and <code class=”code”>--extra-ldflags</code> to configure if the files are installed elsewhere. Installing only under the plain name breaks linking at run time, and the installation directory has to be in the runtime loader’s search path. On big-endian systems, also pass <code class=”code”>-DASTCENC_ISA_NONE=ON</code> and <code class=”code”>-DASTCENC_BIG_ENDIAN=ON</code> to CMake. For this variant, install <samp class=”file”>libastcenc-none-shared.so</samp> under its own name and provide <samp class=”file”>libastcenc.so</samp> as an alias, as in the example above. The library is licensed under Apache-2.0, which requires version 3 of the (L)GPL, so pass <code class=”code”>--enable-version3</code> and <code class=”code”>--enable-libastcenc</code> to configure to enable it. <h3 class=”section”>43.3 AMD AMF/VCE# TOC</h3>

FFmpeg can use the AMD Advanced Media Framework library for accelerated H.264 and HEVC(only windows) encoding on hardware with Video Coding Engine (VCE). To enable support you must obtain the AMF framework header files(version 1.4.9+) from https://github.com/GPUOpen-LibrariesAndSDKs/AMF.git. Create an <code class=”code”>AMF/</code> directory in the system include path. Copy the contents of <code class=”code”>AMF/amf/public/include/</code> into that directory. Then configure FFmpeg with <code class=”code”>--enable-amf</code>.

 Initialization of amf encoder occurs in this order: 1) trying to initialize through dx11(only windows) 2) trying to initialize through dx9(only windows) 3) trying to initialize through vulkan To use h.264(AMD VCE) encoder on linux amdgru-pro version 19.20+ and amf-amdgpu-pro package(amdgru-pro contains, but does not install automatically) are required. This driver can be installed using amdgpu-pro-install script in official amd driver archive. <h3 class="section">43.4 AviSynth# TOC</h3>

FFmpeg can read AviSynth scripts as input. To enable support, pass <code class="code">--enable-avisynth</code> to configure after installing the headers provided by AviSynth+. AviSynth+ can be configured to install only the headers by either passing <code class="code">-DHEADERS_ONLY:bool=on</code> to the normal CMake-based build system, or by using the supplied <code class="code">GNUmakefile</code>. For Windows, supported AviSynth variants are AviSynth 2.6 RC1 or higher for 32-bit builds and Avi r1718 or higher for 32-bit and 64-bit builds. For Linux, macOS, and BSD, the only supported AviSynth variant is AviSynth+, starting with version 3.5.   In 2016, AviSynth+ added support for building with GCC. However, due to the eccentricities of Windows’ calling conventions, 32-bit GCC builds of AviSynth+ are not compatible with typical 32-bit builds of FFmpeg. By default, FFmpeg assumes compatibility with 32-bit MSVC builds of AviSynth+ since that is the most widely-used and entrenched build configuration. Users can override this and enable support for 32-bit GCC builds of AviSynth+ by passing <code class="code">-DAVSC_WIN32_GCC32</code> to <code class="code">--extra-cflags</code> when configuring FFmpeg. 64-bit builds of FFmpeg are not affected, and can use either MSVC or GCC builds of AviSynth+ without any special flags.   AviSynth(+) is loaded dynamically. Distributors can build FFmpeg with <code class="code">--enable-avisynth</code>, and the binaries will work regardless of the end user having AviSynth installed. If/when an end user would like to use AviSynth scripts, then they can install AviSynth(+) and FFmpeg will be able to find and use it to open scripts. <h3 class="section">43.5 Chromaprint# TOC</h3>

FFmpeg can make use of the Chromaprint library for generating audio fingerprints. Pass <code class="code">--enable-chromaprint</code> to config-

ure to enable it. See https://acoustid.org/chromapri
  <h3 class="section">43.6 codec2<span
class="pull-right"><a class="anchor hidden-xs" href="#codec2" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-codec2"
aria-hidden="true">TOC</h3>

FFmpeg can make use of the codec2 library for codec2 decoding and encoding. There is currently no native decoder, so libcodec2 must be used for decoding.
 Go to http://freedv.org/,
download "Codec 2 source archive". Build and install using CMake.
Debian users can install the libcodec2-dev package instead. Once libcodec2
is installed you can pass <code class="code">--enable-libcodec2</code> to
configure to enable it. The easiest way to use codec2 is with .c2 files,
since they contain the mode information required for decoding. To encode such
a file, use a .c2 file extension and give the libcodec2 encoder the -mode option:
<code class="code">ffmpeg -i input.wav -mode 700C output.c2</code>.
Playback is as simple as <code class="code">ffplay output.c2</code>.
For a list of supported modes, run <code class="code">ffmpeg -h encoder=libcodec2</code>. Raw codec2 files are also supported. To make
sense of them the mode in use needs to be specified as a format option:
<code class="code">ffmpeg -f codec2raw -mode 1300 -i input.raw output.wav</code>. <h3 class="section">43.7
dav1d<a class="anchor hidden-xs" href="#dav1d"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-dav1d"
aria-hidden="true">TOC</h3>

FFmpeg can make use of the dav1d library for AV1 video decoding.
Go to https://code.videolan.org/videol
and follow the instructions for installing the library. Then pass <code
class="code">--enable-libdav1d</code> to configure to enable it.
 <h3 class="section">43.8 davs2<span class="pull-
right">#
TOC</h3>

FFmpeg can make use of the davs2 library for AVS2-P2/IEEE1857.4 video
decoding. Go to https://github.com/pku
and follow the instructions for installing the library. Then pass <code
class="code">--enable-libdavs2</code> to configure to enable it. <div
class="info"> libdavs2 is under the GNU Public License Version 2 or
later (see <a class="url" href="http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html">http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class="section">43.9
uavs3d<a class="anchor hidden-xs" href="#uavs3d"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-uavs3d"
aria-hidden="true">TOC</h3>

FFmpeg can make use of the uavs3d library for AVS3-P2/IEEE1857.10

video decoding. Go to https://github.co
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libuavs3d</code> to configure to enable it. <a
name=”Game-Music-Emu”> <h3 class=”section”>43.10 Game Music
Emu<a class=”anchor hidden-xs” href=”#Game-
Music-Emu” aria-hidden=”true”># <a class=”anchor hidden-
xs”href=”#toc-Game-Music-Emu” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the Game Music Emu library to read audio
from supported video game music file formats. Pass <code class=”code”>-
-enable-libgme</code> to configure to enable it. See <a class=”url”
href=”https://bitbucket.org/mpyne/game-music-emu/overview”>https://bitbucket.org/mpyne/game-
music-emu/overview.
<h3 class=”section”>43.11 Intel QuickSync Video<span class=”pull-
right”><a class=”anchor hidden-xs” href=”#Intel-QuickSync-Video”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Intel-
QuickSync-Video” aria-hidden=”true”>TOC</h3>

FFmpeg can use Intel QuickSync Video (QSV) for accelerated decoding
and encoding of multiple codecs. To use QSV, FFmpeg must be linked
against the <code class=”code”>libmfx</code> dispatcher, which loads
the actual decoding libraries. The dispatcher is open source
and can be downloaded from <a class=”url” href=”https://github.com/lu-
zero/mfx_dispatch.git”>https://github.com/lu-zero/mfx_dispatch.git.
FFmpeg needs to be configured with the <code class=”code”>--enable-
libmfx</code> option and <code class=”code”>pkg-config</code> needs to
be able to locate the dispatcher’s <code class=”code”>.pc</code> files.
  <h3 class=”section”>43.12 Kvazaar<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Kvazaar” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Kvazaar”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the Kvazaar library for HEVC encoding.
Go to https://github.com/ultravideo/kva
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libkvazaar</code> to configure to enable it.
 <h3 class=”section”>43.13 LAME<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#LAME” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-LAME”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the LAME library for MP3 encoding.
Go to http://lame.sourceforge.net/
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libmp3lame</code> to configure to enable it.
 <h3 class=”section”>43.14 LCEVCdec<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#LCEVCdec” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-LCEVCdec”

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the liblcevc_dec library for LCEVC enhancement layer decoding on supported bitstreams. Go to https://github.com/v-novaltd/LCEVCdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblcevc-dec</code> to configure to enable it.   LCEVCdec is under the BSD-3-Clause-Clear License. <h3 class=”section”>43.15 libilbc# TOC</h3>

iLBC is a narrowband speech codec that has been made freely available by Google as part of the WebRTC project. libilbc is a packaging friendly copy of the iLBC codec. FFmpeg can make use of the libilbc library for iLBC decoding and encoding. Go to https://github.com/TimothyGu/libilbc and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libilbc</code> to configure to enable it. <h3 class=”section”>43.16 libjxl# TOC</h3>

JPEG XL is an image format intended to fully replace legacy JPEG for an extended period of life. See https://jpegxl.info/ for more information, and see https://github.com/libj for the library source. You can pass <code class=”code”>--enable-libjxl</code> to configure in order enable the libjxl wrapper. <h3 class=”section”>43.17 libvpx# TOC</h3>

FFmpeg can make use of the libvpx library for VP8/VP9 decoding and encoding. Go to http://www.webmproject.org and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libvpx</code> to configure to enable it. <h3 class=”section”>43.18 ModPlug# TOC</h3>

FFmpeg can make use of this library, originating in Modplug-XMMS, to read from MOD-like music files. See https:/ Pass <code class=”code”>--enable-libmodplug</code> to configure to enable it. <h3 class=”section”>43.19 ONNX Runtime# <a class=”anchor

hidden-xs”href=”#toc-ONNX-Runtime” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the ONNX Runtime library as a backend for DNN based filters. Go to https://onnxruntime.ai/ and follow the instructions for installing the library. Pre-built packages are available for various platforms. Configure FFmpeg with ONNX Runtime support: <pre class=”example-preformatted”>./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib" </pre>

On ELF-based systems (Linux, BSD) you may also want to embed the library search path so that the installed <samp class=”file”>ffmpeg</samp> binary finds the shared library at run time without setting <code class=”env”>LD_LIBRARY_PATH</code>:   <pre class=”example-preformatted”>./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib -Wl,-rpath,/path/to/onnxruntime/lib" </pre>

The <code class=”code”>-Wl,-rpath</code> flag is ELF-specific and should be omitted on other platforms (Windows, macOS). For MinGW GCC builds, add the <code class=”code”>-D_stdcall=___stdcall</code> flag for compatibility with ONNX Runtime headers. <h3 class=”section”>43.20 OpenCORE, VisualOn, and Fraunhofer libraries# TOC</h3>

Spun off Google Android sources, OpenCore, VisualOn and Fraunhofer libraries provide encoders for a number of audio codecs.   OpenCORE and VisualOn libraries are under the Apache License 2.0 (see http://www.apache.org/licenses/LICENSE-2.0 for details), which is incompatible to the LGPL version 2.1 and GPL version 2. You have to upgrade FFmpeg’s license to LGPL version 3 (or if you have enabled GPL components, GPL version 3) by passing <code class=”code”>--enable-version3</code> to configure in order to use it. The license of the Fraunhofer AAC library is incompatible with the GPL. Therefore, for GPL builds, you have to pass <code class=”code”>--enable-nonfree</code> to configure in order to use it. To the best of our knowledge, it is compatible with the LGPL. <h4 class=”subsection”>43.20.1 OpenCORE AMR# TOC</h4>

FFmpeg can make use of the OpenCORE libraries for AMR-NB decoding/encoding and AMR-WB decoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the libraries. Then pass <code class="code">--enable-libopencore-amrnb</code> and/or <code class="code">--enable-libopencore-amrwb</code> to configure to enable them. <h4 class="subsection">43.20.2 VisualOn AMR-WB encoder library# TOC</h4>

FFmpeg can make use of the VisualOn AMR-WBenc library for AMR-WB encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libvo-amrwbenc</code> to configure to enable it. <h4 class="subsection">43.20.3 Fraunhofer AAC library# TOC</h4>

FFmpeg can make use of the Fraunhofer AAC library for AAC decoding & encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libfdk-aac</code> to configure to enable it. <h4 class="subsection">43.20.4 Fraunhofer xHE-AAC library# TOC</h4>

FFmpeg can make use of the Fraunhofer xHE-AAC library for xHE-AAC encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libfdk-xhe-aac</code> to configure to enable it. <h4 class="subsection">43.20.5 Fraunhofer MPEG-H 3D Audio decoder library# TOC</h4>

FFmpeg can make use of the Fraunhofer MPEG-H decoder li-

brary for MPEG-H 3DA decoding. Go to https://github.com/Fraunhofer-IIS/mpeghdec and follow the instructions for installing the library. Then pass <code class="code">--enable-libmpeghdec --enable-nonfree</code> to configure to enable it. <h4 class="subsection">43.20.6 LC3 library# TOC</h4>

FFmpeg can make use of the Google LC3 library for LC3 decoding & encoding. Go to https://github.com/go and follow the instructions for installing the library. Then pass <code class="code">--enable-liblc3</code> to configure to enable it. <h3 class="section">43.21 OpenH264# TOC</h3>

FFmpeg can make use of the OpenH264 library for H.264 decoding and encoding. Go to http://www.openh264.org/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libopenh264</code> to configure to enable it. For decoding, this library is much more limited than the built-in decoder in libavcodec; currently, this library lacks support for decoding B-frames and some other main/high profile features. (It currently only supports constrained baseline profile and CABAC.) Using it is mostly useful for testing and for taking advantage of Cisco’s patent portfolio license (http://www.openh264.org/BINARY_LICENSE. <h3 class="section">43.22 OpenJPEG# TOC</h3>

FFmpeg can use the OpenJPEG libraries for decoding/encoding J2K videos. Go to http://www.openjpeg.org/ to get the libraries and follow the installation instructions. To enable using OpenJPEG in FFmpeg, pass <code class="code">--enable-libopenjpeg</code> to <samp class="file">./configure</samp>. <h3 class="section">43.23 rav1e# TOC</h3>

FFmpeg can make use of rav1e (Rust AV1 Encoder) via its C bindings to encode videos. Go to https://github.com/xiph/rav1e/< and follow the instructions to build the C library. To enable using rav1e in FFmpeg, pass <code class="code">--enable-librav1e</code> to <samp class="file">./configure</samp>. <h3 class="section">43.24 SVT-AV1<a class="anchor

hidden-xs" href="#SVT_002dAV1" aria-hidden="true"># TOC</h3>

FFmpeg can make use of the Scalable Video Technology for AV1 library for AV1 encoding. Go to https://gitlab.com/AOMediaCodec/SVT-AV1/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtav1</code> to configure to enable it. <h3 class="section">43.25 SVT-JPEG-XS# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for JPEG-XS library for JPEG-XS decoding and encoding. Go to https://github.com/OpenVisualCloud/SVT-JPEG-XS and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtjpegxs</code> to configure to enable it. <h3 class="section">43.26 TwoLAME# TOC</h3>

FFmpeg can make use of the TwoLAME library for MP2 encoding. Go to http://www.twolame.org/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libtwolame</code> to configure to enable it. <h3 class="section">43.27 VapourSynth# TOC</h3>

FFmpeg can read VapourSynth scripts as input. To enable support, pass <code class="code">--enable-vapoursynth</code> to configure. Vapoursynth is detected via <code class="code">pkg-config</code>. Versions 42 or greater supported. See http://www.vapoursynth.com/. Due to security concerns, Vapoursynth scripts will not be autodetected so the input format has to be forced. For ff* CLI tools, add <code class="code">-f vapoursynth</code> before the input <code class="code">-i yourscript.vpy</code>. <h3 class="section">43.28 x264# TOC</h3>

FFmpeg can make use of the x264 library for H.264 encoding. Go to <a class="url" href="http://www.videolan.org/developers/x264.html">http://www.videolan.org/d and follow the instructions for installing the library. Then pass <code

class=”code”>--enable-libx264</code> to configure to enable it.   x264 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>43.29 x265# TOC</h3>

FFmpeg can make use of the x265 library for HEVC encoding. Go to http://x265.org/developers.html and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libx265</code> to configure to enable it.   x265 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>43.30 xavs# TOC</h3>

FFmpeg can make use of the xavs library for AVS encoding. Go to http://xavs.sf.net/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libxavs</code> to configure to enable it. <h3 class=”section”>43.31 xavs2# TOC</h3>

FFmpeg can make use of the xavs2 library for AVS2-P2/IEEE1857.4 video encoding. Go to <a class=”url” href=”https://github.com/pkuvcl/xavs2”>https://github.com/pku and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libxavs2</code> to configure to enable it.   libxavs2 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>43.32 eXtra-fast Essential Video Encoder (XEVE)# TOC</h3>

FFmpeg can make use of the XEVE library for EVC video encoding.

Go to https://github.com/mpeg5/xeve and follow the instructions for installing the XEVE library. Then pass <code class="code">--enable-libxeve</code> to configure to enable it. <h3 class="section">43.33 eXtra-fast Essential Video Decoder (XEVD)# TOC</h3>

FFmpeg can make use of the XEVD library for EVC video decoding. Go to https://github.com/mpeg5/xevd and follow the instructions for installing the XEVD library. Then pass <code class="code">--enable-libxevd</code> to configure to enable it. <h3 class="section">43.34 ZVBI# TOC</h3>

ZVBI is a VBI decoding library which can be used by FFmpeg to decode DVB teletext pages and DVB teletext subtitles. Go to http://sourceforge.net/projects/zapping/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libzvbi</code> to configure to enable it. <h2 class="chapter">44 Supported File Formats, Codecs or Features# TOC</h2>

You can use the <code class="code">-formats</code> and <code class="code">-codecs</code> options to have an exhaustive list. <h3 class="section">44.1 File Formats# TOC</h3>

FFmpeg supports the following file formats through the <code class="code">libavformat</code>

library:

<table class="multitable">

<tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td width="10%">Decoding</td><td width="40%">Comments</td></tr>

<tr><td width="40%">3dostr</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">4xm</td><td width="10%"></td><td width="10%">X</td><td width="40%">4X Technologies format, used in some games.</td></tr>

<tr><td width="40%">8088flex TMV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AAX</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audible Enhanced Audio format, used in audiobooks.</td></tr>

<tr><td width="40%">AA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audible Format 2, 3, and 4, used in audiobooks.</td></tr>

<tr><td width="40%">ACT Voice</td><td width="10%"></td><td width="10%">X</td><td width="40%">contains G.729 audio</td></tr>

<tr><td width="40%">Adobe Filmstrip</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Audio IFF (AIFF)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">American Laser Games MM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in games like Mad Dog McCree.</td></tr>

<tr><td width="40%">3GPP AMR</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Amazing Studio Packed Animation File</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in game Heart Of Darkness.</td></tr>

<tr><td width="40%">Apple HTTP Live Streaming</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Artworx Data Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only format used in some Interplay games.</td></tr>

<tr><td width="40%">ADP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">AFC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">ADS/SS2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS2.</td></tr>

<tr><td width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">.astc</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Raw ASTC texture container.</td></tr>

<tr><td width="40%">ASF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Advanced / Active Streaming Format.</td></tr>

<tr><td width="40%">AST</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">AviSynth</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AVR</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on Mac.</td></tr>

<tr><td width="40%">AVS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by the Creature Shock game.</td></tr>

<tr><td width="40%">Beam Software SIFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Beam Software.</td></tr>

<tr><td width="40%">Bethesda Softworks VID</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Binary text</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only multimedia format used by some games.</td></tr>

<tr><td width="40%">Bitmap Brothers JV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Z and Z95 games.</td></tr>

<tr><td width="40%">BRP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Argonaut Games format.</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">BFSTM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo WiiU (based on BRSTM).</td></tr>

<tr><td width="40%">BRSTM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">BW64</td><td width="10%"></td><td width="10%">X</td><td width="40%">Broadcast Wave 64bit.</td></tr>

<tr><td width="40%">BWF</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">codec2 (raw)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Must be given -mode format option to decode correctly.</td></tr>

<tr><td width="40%">codec2 (.c2 files)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Contains header with version and mode info, simplifying playback.</td></tr>

<tr><td width="40%">CRI ADX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">CRI AIX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">Discworld II BMV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Delphine Software International CIN</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by Delphine Software games.</td></tr>

<tr><td width="40%">Digital Speech Standard (DSS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video format used by CD+G karaoke disks</td></tr>

<tr><td width="40%">Phantom Cine</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Commodore CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video format</td></tr>

<tr><td width="40%">Core Audio Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Apple Core Audio Format</td></tr>

<tr><td width="40%">CRC testing format</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">Creative Voice</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Created for the Sound Blaster Pro.</td></tr>

<tr><td width="40%">CRYO APC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in some games by CRYO Interactive Entertainment.</td></tr>

<tr><td width="40%">D-Cinema audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DCSTR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DFA</td><td width="10%"></td><td width="10%">X</td><td width="40%">This format is used in Chronomaster game</td></tr>

<tr><td width="40%">DirectDraw Surface</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD Stream File (DSF)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DV video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">DXA</td><td width="10%"></td><td width="10%">X</td><td width="40%">This format is used in the non-Windows version of the Feeble Files

game and different game cutscenes repacked for use with ScummVM.</td></tr>

<tr><td width="40%">Electronic Arts cdata</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts Multimedia</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various EA games; files have extensions like WVE and UV2.</td></tr>

<tr><td width="40%">Ensoniq Paris Audio File</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">FFM (FFserver live feed)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash (SWF)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash 9 (AVM2)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Only embedded audio is decoded.</td></tr>

<tr><td width="40%">FLI/FLC/FLX animation</td><td width="10%"></td><td width="10%">X</td><td width="40%">.fli/.flc files</td></tr>

<tr><td width="40%">Flash Video (FLV)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Macromedia Flash video files</td></tr>

<tr><td width="40%">framecrc testing format</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">FunCom ISS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in various games from FunCom like The Longest Journey.</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.726</td><td width="10%"></td><td width="10%">X</td><td width="40%">Both left- and right-justified.</td></tr>

<tr><td width="40%">G.729 BIT</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.729 raw</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GENH</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format for various games.</td></tr>

<tr><td width="40%">GIF Animation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">General eXchange Format SMPTE 360M, used by Thomson Grass Valley

playout servers.</td></tr>

<tr><td width="40%">HNM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Only version 4 supported, used in some games from Cryo Interactive</td></tr>

<tr><td width="40%">iCEDraw File</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ICO</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft Windows ICO</td></tr>

<tr><td width="40%">id Quake II CIN video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">IEC61937 encapsulation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Interchange File Format</td></tr>

<tr><td width="40%">IFV</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format used by some old CCTV DVRs.</td></tr>

<tr><td width="40%">iLBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay MVE</td><td width="10%"></td><td width="10%">X</td><td width="40%">Format used in various Interplay computer games.</td></tr>

<tr><td width="40%">Iterated Systems ClearVideo</td><td width="10%"></td><td width="10%">X</td><td width="40%">I-frames only</td></tr>

<tr><td width="40%">IV8</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format generated by IndigoVision 8000 video server.</td></tr>

<tr><td width="40%">IVF (On2)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">A format used by libvpx</td></tr>

<tr><td width="40%">Internet Video Recording</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">IRCAM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">JPEG Multi-Picture Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Includes JPEG Ultra HDR (ISO 21496-1 gain map) images.</td></tr>

<tr><td width="40%">LAF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Limitless Audio Format</td></tr>

<tr><td width="40%">LATM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LMLM4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Linux Media Labs MPEG-4 PCI boards</td></tr>

<tr><td width="40%">LOAS</td><td width="10%"></td><td width="10%">X</td><td width="40%">contains LATM multiplexed AAC audio</td></tr>

<tr><td width="40%">LRC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LVF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LXF</td><td width="10%"></td><td width="10%">X</td><td width="40%">VR native stream format, used by Leitch/Harris’ video servers.</td></tr>

<tr><td width="40%">Magic Lantern Video (MLV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska audio</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">FFmpeg metadata</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Metadata in text format.</td></tr>

<tr><td width="40%">MAXIS XA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sim City 3000; file extension .xa.</td></tr>

<tr><td width="40%">MCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Capcom; file extension .mca.</td></tr>

<tr><td width="40%">MD Studio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Metal Gear Solid: The Twin Snakes</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Megalux Frame</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Megalux Ultimate Paint</td></tr>

<tr><td width="40%">MobiClip MODS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MobiClip MOFLEX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mobotix .mxg</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>Monkey’s Audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Motion Pixels MVI</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MOV/QuickTime/MP4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>3GP, 3GP2, PSP, iPod variants supported</td></tr>

<tr><td width=”40%”>MP2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MP3</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-1 System</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>muxed audio and video, VCD format supported</td></tr>

<tr><td width=”40%”>MPEG-PS (program stream)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>also known as <code class=”code”>VOB</code> file, SVCD and DVD format supported</td></tr>

<tr><td width=”40%”>MPEG-TS (transport stream)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>also known as DVB Transport Stream</td></tr>

<tr><td width=”40%”>MPEG-4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>MPEG-4 is a variant of QuickTime.</td></tr>

<tr><td width=”40%”>MSF</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used on the PS3.</td></tr>

<tr><td width=”40%”>Mirillis FIC video</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>No cursor rendering.</td></tr>

<tr><td width=”40%”>MIDI Sample Dump Standard</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MIME multipart JPEG</td><td width=”10%”>X</td><td width=”10%”></td></tr>

<tr><td width=”40%”>MSN TCP webcam</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used by MSN Messenger webcam streams.</td></tr>

<tr><td width=”40%”>MTV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Musepack</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Material eXchange Format (MXF)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 377M, used by D-Cinema, broadcast industry.</td></tr>

<tr><td width="40%">Material eXchange Format (MXF), D-10 Mapping</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 386M, D-10/IMX Mapping.</td></tr>

<tr><td width="40%">NC camera feed</td><td width="10%"></td><td width="10%">X</td><td width="40%">NC (AVIP NC4600) camera streams</td></tr>

<tr><td width="40%">NIST SPeech HEader REsources</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Computerized Speech Lab NSP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NTT TwinVQ (VQF)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Nippon Telegraph and Telephone Corporation TwinVQ.</td></tr>

<tr><td width="40%">Nullsoft Streaming Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NUT</td><td width="10%">X</td><td width="10%">X</td><td width="40%">NUT Open Container Format</td></tr>

<tr><td width="40%">Ogg</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Playstation Portable PMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Portable Voice Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">TechnoTrend PVA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by TechnoTrend DVB PCI boards.</td></tr>

<tr><td width="40%">QCP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw ADTS (AAC)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-NB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-WB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APAC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Bonk</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw Chinese AVS video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Dirac</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DNxHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS-HD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw E-AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw EVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw FLAC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw GSM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.263</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.264</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw HEVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Ingenient MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw MLP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw NULL</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw id RoQ</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw OBU</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw OSQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw SBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TAK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TrueHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw VC-1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 16.8 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 24.0 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RDT</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">REDCODE R3D</td><td width="10%"></td><td width="10%">X</td><td width="40%">File format used by RED Digital cameras, contains JPEG 2000 frames and PCM audio.</td></tr>

<tr><td width="40%">RealMedia</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Redirector</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RedSpark</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TeXture Dictionary</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Resolume DXV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoding is only supported for the DXT1 (Normal Quality, No Alpha) texture format.</td></tr>

<tr><td width="40%">RF64</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RL2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Entertainment Software Partners.</td></tr>

<tr><td width="40%">RPL/ARMovie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Lego Mindstorms RSO</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RSD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RTMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Output is performed by publishing stream to RTMP server</td></tr>

<tr><td width="40%">RTP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RTSP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sample Dump eXchange</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SAP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SBG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDNS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SER</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Pictures SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sega FILM/CPK</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in many Sega Saturn console games.</td></tr>

<tr><td width="40%">Silicon Graphics Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra SOL</td><td width="10%"></td><td width="10%">X</td><td width="40%">.sol files used in Sierra Online games.</td></tr>

<tr><td width="40%">Sierra VMD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra CD-ROM games.</td></tr>

<tr><td width="40%">Smacker</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">SMJPEG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in certain Loki game ports.</td></tr>

<tr><td width="40%">SMPTE 337M encapsulation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in some LucasArts games.</td></tr>

<tr><td width="40%">Sony OpenMG (OMA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used in Sony Sonic Stage and Sony Vegas.</td></tr>

<tr><td width="40%">Sony PlayStation STR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sony Wave64 (W64)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SoX native format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUN AU format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUP raw PGS subtitles</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Konami PS2 games.</td></tr>

<tr><td width="40%">TDSC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Text files</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">THP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used on the Nintendo GameCube.</td></tr>

<tr><td width="40%">Tiertex Limited SEQ</td><td width="10%"></td><td width="10%">X</td><td width="40%">Tiertex .seq files used in the DOS

CD-ROM version of the game Flashback.</td></tr>

<tr><td width="40%">True Audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in many Sony PS2 games.</td></tr>

<tr><td width="40%">VC-1 test bitstream</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vidvox Hap</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vivo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VPK</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Sony PS games.</td></tr>

<tr><td width="40%">Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">WebM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Televison (WTV)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Wing Commander III movie</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Origin’s Wing Commander III computer game.</td></tr>

<tr><td width="40%">Westwood Studios audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Westwood Studios VQA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Wideband Single-bit Data (WSD)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WVE</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">KTX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Khronos Texture 1.0 container for ASTC.</td></tr>

<tr><td width="40%">Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">XMV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft video container used in Xbox games.</td></tr>

<tr><td width="40%">XVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">xWMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft audio container used by XAudio 2.</td></tr>

<tr><td width="40%">eXtended BINary text (XBIN)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">YUV4MPEG pipe</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP</td><td width="10%"></td><td width="10%">X</td></tr>

</tbody>

</table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <h3 class="section">44.2 Image Formats# TOC</h3>

FFmpeg can read and write images for each frame of a video sequence. The following image formats are supported: <table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td width="10%">Decoding</td><td width="40%">Comments</td></tr> <tr><td width="40%">.Y.U.V</td><td width="10%">X</td><td width="10%">X</td><td width="40%">one raw file per component</td></tr> <tr><td width="40%">Alias PIX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Alias/Wavefront PIX image format</td></tr> <tr><td width="40%">animated GIF</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Animated Portable Network Graphics</td></tr> <tr><td width="40%">ASTC</td><td width="10%">E</td><td width="10%">E</td><td

width=”40%”>Adaptive Scalable Texture Compression, through external library libastcenc</td></tr> <tr><td width=”40%”>BMP</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Microsoft BMP image</td></tr> <tr><td width=”40%”>BRender PIX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Argonaut BRender 3D engine image format.</td></tr> <tr><td width=”40%”>CRI</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Cintel RAW</td></tr> <tr><td width=”40%”>DPX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Digital Picture Exchange</td></tr>
<tr><td width=”40%”>EXR</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>OpenEXR</td></tr> <tr><td width=”40%”>FITS</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Flexible Image Transport System</td></tr> <tr><td width=”40%”>HDR</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Radiance HDR RGBE Image format</td></tr> <tr><td width=”40%”>IMG</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>GEM Raster image</td></tr> <tr><td width=”40%”>JPEG</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Progressive JPEG is not supported.</td></tr> <tr><td width=”40%”>JPEG 2000</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>JPEG-LS</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JPEG-XS</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Supported through external library libsvtjpegxs</td></tr> <tr><td width=”40%”>LJPEG</td><td width=”10%”>X</td><td width=”10%”></td><td width=”40%”>Lossless JPEG</td></tr> <tr><td width=”40%”>Media 100</td><td width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>MSP</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Microsoft Paint image</td></tr> <tr><td width=”40%”>PAM</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>PAM is a PNM extension with alpha support.</td></tr> <tr><td width=”40%”>PBM</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Portable BitMap image</td></tr> <tr><td width=”40%”>PCD</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>PhotoCD</td></tr>
<tr><td width=”40%”>PCX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>PC Paintbrush</td></tr>
<tr><td width=”40%”>PFM</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Portable FloatMap image</td></tr>
<tr><td width=”40%”>PGM</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Portable GrayMap image</td></tr>
<tr><td width=”40%”>PGMYUV</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>PGM with U and V components in YUV 4:2:0</td></tr> <tr><td width=”40%”>PGX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>PGX file decoder</td></tr> <tr><td width=”40%”>PHM</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Portable HalfFloatMap im-

age</td></tr> <tr><td width="40%">PIC</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Pictor/PC Paint</td></tr>
<tr><td width="40%">PNG</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Portable Network Graphics im-
age</td></tr> <tr><td width="40%">PPM</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Portable PixelMap image</td></tr>
<tr><td width="40%">PSD</td><td width="10%"></td><td width="10%">X</td><td
width="40%">Photoshop</td></tr> <tr><td width="40%">PTX</td><td
width="10%"></td><td width="10%">X</td><td width="40%">V.Flash
PTX format</td></tr> <tr><td width="40%">QOI</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Quite OK Image format</td></tr>
<tr><td width="40%">RW2</td><td width="10%"></td><td width="10%">X</td><td
width="40%">Panasonic raw image, RawFormat 8 only</td></tr> <tr><td
width="40%">SGI</td><td width="10%">X</td><td width="10%">X</td><td
width="40%">SGI RGB image format</td></tr> <tr><td width="40%">Sun
Rasterfile</td><td width="10%">X</td><td width="10%">X</td><td
width="40%">Sun RAS image format</td></tr> <tr><td width="40%">TIFF</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">YUV,
JPEG and some extension is not supported yet.</td></tr> <tr><td
width="40%">Truevision Targa</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Targa (.TGA) image for-
mat</td></tr> <tr><td width="40%">VBN</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Vizrt Binary Image format</td></tr>
<tr><td width="40%">WBMP</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Wireless Application Protocol
Bitmap image format</td></tr> <tr><td width="40%">WebP</td><td
width="10%">E</td><td width="10%">X</td><td width="40%">WebP
image format, encoding supported through external library libwebp</td></tr>
<tr><td width="40%">XBM</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X BitMap image format</td></tr>
<tr><td width="40%">XFace</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X-Face image format</td></tr>
<tr><td width="40%">XPM</td><td width="10%"></td><td width="10%">X</td><td
width="40%">X PixMap image format</td></tr> <tr><td width="40%">XWD</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">X
Window Dump image format</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code> means that support is provided through an external library. <h3 class="section">44.3 Video Codecs# TOC</h3>

<table class="multitable">

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>4X Movie</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in certain computer games.</td></tr>

<tr><td width=”40%”>8088flex TMV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>A64 multicolor</td><td width=”10%”>X</td><td width=”10%”></td><td width=”40%”>Creates video suitable to be played on a commodore 64 (multicolor mode).</td></tr>

<tr><td width=”40%”>Amazing Studio PAF Video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>American Laser Games MM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in games like Mad Dog McCree.</td></tr>

<tr><td width=”40%”>Amuse Graphics Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AMV Video</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in Chinese MP3 players.</td></tr>

<tr><td width=”40%”>ANSI/ASCII art</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple Intermediate Codec</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple MJPEG-B</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple Pixlet</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Apple ProRes</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: apch,apcn,apcs,apco,ap4h,ap4x</td></tr>

<tr><td width=”40%”>Apple QuickDraw</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: qdrw</td></tr>

<tr><td width=”40%”>APV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Argonaut Video</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some Argonaut games.</td></tr>

<tr><td width=”40%”>Asus v1</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ASV1</td></tr>

<tr><td width="40%">Asus v2</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ASV2</td></tr>

<tr><td width="40%">ATI VCR1</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR1</td></tr>

<tr><td width="40%">ATI VCR2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR2</td></tr>

<tr><td width="40%">Auravision Aura</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Auravision Aura 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk Animator Flic video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk RLE</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: AASC</td></tr>

<tr><td width="40%">AV1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libaom, libdav1d, librav1e and libsvtav1</td></tr>

<tr><td width="40%">Avid 1:1 10-bit RGB Packer</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: AVrp</td></tr>

<tr><td width="40%">AVS (Audio Video Standard) video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used by the Creature Shock game.</td></tr>

<tr><td width="40%">AVS2-P2/IEEE1857.4</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libxavs2 and libdavs2</td></tr>

<tr><td width="40%">AVS3-P2/IEEE1857.10</td><td width="10%"></td><td width="10%">E</td><td width="40%">Supported through external library libuavs3d</td></tr>

<tr><td width="40%">AYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft uncompressed packed 4:4:4:4</td></tr>

<tr><td width="40%">Beam Software VB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bethesda VID video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Bink Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">BitJazz SheerVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bitmap Brothers JV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">y41p Brooktree uncompressed 4:1:1 12-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Brooktree ProSumer Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: BT20</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">C93 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Cyberia game.</td></tr>

<tr><td width="40%">CamStudio</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: CSCD</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video codec for CD+G karaoke disks</td></tr>

<tr><td width="40%">CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video codec</td></tr>

<tr><td width="40%">Chinese AVS video</td><td width="10%">E</td><td width="10%">X</td><td width="40%">AVS1-P2, JiZhun profile, encoding through external library libxavs</td></tr>

<tr><td width="40%">Delphine Software International CIN video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">Discworld II BMV Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CineForm HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CDToons</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in various Broderbund games.</td></tr>

<tr><td width="40%">Cinepak</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Cirrus Logic AccuPak</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: CLJR</td></tr>

<tr><td width="40%">CPiA Video Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Creative YUV (CYUV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DFA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Chronomaster game.</td></tr>

<tr><td width="40%">Dirac</td><td width="10%">E</td><td width="10%">X</td><td width="40%">supported though the native vc2 (Dirac Pro) encoder</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DNxHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">aka SMPTE VC3</td></tr>

<tr><td width="40%">Duck TrueMotion 1.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: DUCK</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TM20</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0 RT</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TR20</td></tr>

<tr><td width="40%">DV (Digital Video)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Dxtory capture format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Feeble Files/ScummVM DXA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec originally used in Feeble Files game.</td></tr>

<tr><td width="40%">Electronic Arts CMV video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in NHL 95 game.</td></tr>

<tr><td width="40%">Electronic Arts Madcow video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGQ video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TQI video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 124</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 130</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">EVC / MPEG-5 Part 1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">encoding and decoding supported through external libraries libxeve and libxevd</td></tr>

<tr><td width="40%">FFmpeg video codec #1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">lossless codec (fourcc: FFV1)</td></tr>

<tr><td width="40%">Flash Screen Video v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: FSV1</td></tr>

<tr><td width="40%">Flash Screen Video v2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash Video (FLV)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sorenson H.263 used in Flash</td></tr>

<tr><td width="40%">FM Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Forward Uncompressed</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Fraps</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Go2Meeting</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M2, G2M3</td></tr>

<tr><td width="40%">Go2Webinar</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M4</td></tr>

<tr><td width="40%">Gremlin Digital Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263 / H.263-1996</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263+ / H.263-1998 / H.263 version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libx264 and OpenH264,

decoding supports data partitioning</td></tr>

<tr><td width="40%">HEVC</td><td width="10%">X</td><td width="10%">X</td><td width="40%">encoding supported through external library libx265 and libkvazaar</td></tr>

<tr><td width="40%">HNM version 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV FFmpeg variant</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IBM Ultimotion</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ULTI</td></tr>

<tr><td width="40%">id Cinematic video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Quake II.</td></tr>

<tr><td width="40%">id RoQ video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2, other computer games.</td></tr>

<tr><td width="40%">IFF ILBM</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF interleaved bitmap</td></tr>

<tr><td width="40%">IFF ByteRun1</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF run length encoded bitmap</td></tr>

<tr><td width="40%">Infinity IMM4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM6</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel H.263</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Interplay MVE video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Interplay .MVE files.</td></tr>

<tr><td width="40%">J2K</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Karl Morton’s video codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Worms games.</td></tr>

<tr><td width="40%">Kega Game Video (KGV1)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Kega emulator screen capture codec.</td></tr>

<tr><td width="40%">Lagarith</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCEVC / MPEG-5 LCEVC / MPEG-5 Part 2</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library liblcevc-dec</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) MSZH</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) ZLIB</td><td width="10%">E</td><td width="10%">E</td></tr>

<tr><td width="40%">LEAD MCMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LOCO</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LucasArts SANM/Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts games / SMUSH animations.</td></tr>

<tr><td width="40%">lossless MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MagicYUV Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mandsoft Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft ATC Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Screen 3.</td></tr>

<tr><td width="40%">Microsoft Expression Encoder Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Titanium Screen 2.</td></tr>

<tr><td width="40%">Microsoft RLE</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft Screen 1</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V7 Screen.</td></tr>

<tr><td width="40%">Microsoft Screen 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V9 Screen.</td></tr>

<tr><td width="40%">Microsoft Video 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mimic</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in MSN Messenger Webcam streams.</td></tr>

<tr><td width="40%">Miro VideoXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VIXL</td></tr>

<tr><td width="40%">MJPEG (Motion JPEG)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mobotix MxPEG video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Motion Pixels video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-1 video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-2 video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2</td><td width="10%">X</td><td width="10%">X</td><td width="40%">libxvidcore can be used alternatively for encoding.</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-4 part 2 Microsoft variant version 3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Newtek SpeedHQ</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Nintendo Gamecube THP video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NotchLC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo/RTjpeg</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in NuppelVideo files.</td></tr>

<tr><td width="40%">On2 VP3</td><td width="10%"></td><td width="10%">X</td><td width="40%">still experimental</td></tr>

<tr><td width="40%">On2 VP4</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP40</td></tr>

<tr><td width="40%">On2 VP5</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP50</td></tr>

<tr><td width="40%">On2 VP6</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP60,VP61,VP62</td></tr>

<tr><td width="40%">On2 VP7</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VP70,VP71</td></tr>

<tr><td width="40%">VP8</td><td width="10%">E</td><td width="10%">X</td><td width="40%">fourcc: VP80, encoding supported through external library libvpx</td></tr>

<tr><td width="40%">VP9</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libvpx</td></tr>

<tr><td width="40%">Pinnacle TARGA CineWave YUV16</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: Y216</td></tr>

<tr><td width="40%">Q-team QPEG</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourccs: QPEG, Q1.0, Q1.1</td></tr>

<tr><td width="40%">QuickTime 8BPS video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QuickTime Animation (RLE) video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ’rle ’</td></tr>

<tr><td width="40%">QuickTime Graphics (SMC)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ’smc ’</td></tr>

<tr><td width="40%">QuickTime video (RPZA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: rpza</td></tr>

<tr><td width="40%">R10K AJA Kona 10-bit RGB Codec</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">R210 Quicktime Uncompressed RGB 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Raw Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 1.0</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 2.0</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 3.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">still far from ideal</td></tr>

<tr><td width="40%">RealVideo 4.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 6.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TXD (TeXture Dictionary)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Texture dictionaries used by the Renderware Engine.</td></tr>

<tr><td width="40%">RivaTuner Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’RTV1’</td></tr>

<tr><td width="40%">RL2 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">used in some games by Entertainment Software Partners</td></tr>

<tr><td width="40%">ScreenPressor</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screenpresso</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screen Recorder Gold Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 1 (MVC1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 2 (MVC2)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics RLE 8-bit video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smacker video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in Smacker.</td></tr>

<tr><td width="40%">SMPTE VC-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Snow</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental wavelet codec (fourcc: SNOW)</td></tr>

<tr><td width="40%">Sony PlayStation MDEC (Motion DECoder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: SVQ1</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 3</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SVQ3</td></tr>

<tr><td width="40%">Sunplus JPEG (SP5X)</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SP5X</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSCC</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSC2</td></tr>

<tr><td width="40%">Theora</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libtheora</td></tr>

<tr><td width="40%">Tiertex Limited SEQ video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in DOS CD-ROM FlashBack game.</td></tr>

<tr><td width="40%">Ut Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v210 QuickTime uncompressed 4:2:2 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v308 QuickTime uncompressed 4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v408 QuickTime uncompressed 4:4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v410 QuickTime uncompressed 4:4:4 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VBLE Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">vMix Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’VMX1’</td></tr>

<tr><td width="40%">VMware Screen Codec / VMware Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in videos captured by VMware.</td></tr>

<tr><td width="40%">Westwood Studios VQA (Vector Quantized Animation) video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Image</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 8</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 9</td><td width="10%"></td><td width="10%">X</td><td width="40%">not completely working</td></tr>

<tr><td width="40%">Wing Commander III / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander III .MVE files.</td></tr>

<tr><td width="40%">Wing Commander IV / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander IV.</td></tr>

<tr><td width="40%">Winnov WNV1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WMV7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">YAMAHA SMAF</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width=”40%”>Psygnosis YOP Video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>yuv4</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>libquicktime uncompressed packed 4:2:0</td></tr>

<tr><td width=”40%”>ZeroCodec Lossless Video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ZLIB</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>part of LCL, encoder experimental</td></tr>

<tr><td width=”40%”>Zip Motion Blocks Video</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Encoder works only in PAL8.</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <code class=”code”>E</code> means that support is provided through an external library. <h3 class=”section”>44.4 Audio Codecs# TOC</h3>

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>8SVX exponential</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>8SVX fibonacci</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAC</td><td width=”10%”>EX</td><td width=”10%”>X</td><td width=”40%”>encoding supported through internal encoder and external library libfdk-aac</td></tr>

<tr><td width=”40%”>AAC+</td><td width=”10%”>E</td><td width=”10%”>IX</td><td width=”40%”>encoding supported through external library libfdk-aac</td></tr>

<tr><td width=”40%”>AC-3</td><td width=”10%”>IX</td><td width=”10%”>IX</td></tr>

<tr><td width=”40%”>ACELP.KELVIN</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM 4X Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Yamaha AICA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM AmuseGraphics Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Argonaut Games</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM CDROM XA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Creative Technology</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>16 -> 4, 8 -> 4, 8 -> 3, 8 -> 2</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA titles.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts Maxis CDROM XS</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Sim City 3000.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts R3</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts XAS</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.722</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM G.726</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA Acorn Replay</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM IMA AMV</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in AMV files</td></tr>

<tr><td width=”40%”>ADPCM IMA Citrix</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width="40%">ADPCM IMA Cunning Developments</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Electronic Arts EACS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Electronic Arts SEAD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Funcom</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA High Voltage Software ALP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Mobiclip MOFLEX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA QuickTime</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Simon & Schuster Interactive</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Ubisoft APM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Loki SDL MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Westwood</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM ISS IMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in FunCom games.</td></tr>

<tr><td width="40%">ADPCM IMA Dialogic</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK3</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Radical</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>ADPCM IMA Xbox</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Microsoft</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM MS IMA</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Nintendo Gamecube AFC</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Nintendo Gamecube DTK</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Nintendo THP</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Playstation</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM QT IMA</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Rhetorex</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sanyo</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM SEGA CRI ADX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in Sega Dreamcast games.</td></tr>

<tr><td width=”40%”>ADPCM Shockwave Flash</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 2-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 2.6-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Sound Blaster Pro 4-bit</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM VIMA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in LucasArts SMUSH animations.</td></tr>

<tr><td width=”40%”>ADPCM Konami XMD</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width="40%">ADPCM Westwood Studios IMA</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Westwood Studios games like Command and Conquer.</td></tr>

<tr><td width="40%">ADPCM Yamaha</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Zork</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AMR-NB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopencore-amrnb</td></tr>

<tr><td width="40%">AMR-WB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libvo-amrwbenc</td></tr>

<tr><td width="40%">Amazing Studio PAF Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple lossless audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">QuickTime fourcc ’alac’</td></tr>

<tr><td width="40%">aptX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">aptX HD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">ATRAC1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3+</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC9</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Bink and Smacker files in many games.</td></tr>

<tr><td width="40%">Bonk audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">codec2</td><td width="10%">E</td><td width="10%">E</td><td width="40%">en/decoding supported through external library libcodec2</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Delphine Software International CIN audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Speech Standard - Standard Play mode (DSS SP)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Discworld II BMV Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">COOK</td><td width="10%"></td><td width="10%">X</td><td width="40%">All versions except 5.1 are supported.</td></tr>

<tr><td width="40%">DCA (DTS Coherent Acoustics)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">supported extensions: XCh, XXCH, X96, XBR, XLL, LBR (partially)</td></tr>

<tr><td width="40%">Dolby E</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Cuberoot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in few games.</td></tr>

<tr><td width="40%">DPCM Gremlin</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">DPCM Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Interplay</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various Interplay computer games.</td></tr>

<tr><td width="40%">DPCM Squareroot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various games.</td></tr>

<tr><td width="40%">DPCM Sierra Online</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra Online game audio files.</td></tr>

<tr><td width="40%">DPCM Sol</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Origin’s Wing Commander IV AVI files.</td></tr>

<tr><td width="40%">DPCM Xilam DERF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSP Group TrueSpeech</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DST (Direct Stream Transfer)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DV audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Enhanced AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">EVRC (Enhanced Variable Rate Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">FLAC (Free Lossless Audio Codec)</td><td width="10%">X</td><td width="10%">IX</td></tr>

<tr><td width="40%">FTR Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.729</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GSM</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">GSM Microsoft variant</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width=”40%”>IAC (Indeo Audio Coder)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>iLBC (Internet Low Bitrate Codec)</td><td width=”10%”>E</td><td width=”10%”>EX</td><td width=”40%”>encoding and decoding supported through external library libilbc</td></tr>

<tr><td width=”40%”>IMC (Intel Music Coder)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Interplay ACM</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>LC3</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>supported through external library liblc3</td></tr>

<tr><td width=”40%”>MACE (Macintosh Audio Compression/Expansion) 6:1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Marian’s A-pac audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MI-SC4 (Micronas SC-4 Audio)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MLP (Meridian Lossless Packing)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Used in DVD-Audio discs.</td></tr>

<tr><td width=”40%”>Monkey’s Audio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MP1 (MPEG audio layer 1)</td><td width=”10%”></td><td width=”10%”>IX</td></tr>

<tr><td width=”40%”>MP2 (MPEG audio layer 2)</td><td width=”10%”>IX</td><td width=”10%”>IX</td><td width=”40%”>encoding supported also through external library TwoLAME</td></tr>

<tr><td width=”40%”>MP3 (MPEG audio layer 3)</td><td width=”10%”>E</td><td width=”10%”>IX</td><td width=”40%”>encoding supported through external library LAME, ADU MP3 and MP3onMP4 also supported</td></tr>

<tr><td width=”40%”>MPEG-4 Audio Lossless Coding (ALS)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-H 3D Audio</td><td width=”10%”></td><td width=”10%”>E</td><td width=”40%”>decoding supported through external library libmpeghdec</td></tr>

<tr><td width=”40%”>MobiClip FastAudio</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width="40%">Musepack SV7</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Nellymoser Asao</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">On2 AVC (Audio for Video Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Opus</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopus</td></tr>

<tr><td width="40%">OSQ (Original Sound Quality)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM D-Cinema audio signed 24-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian for DVD-Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian in MPEG-TS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QCELP / PureVoice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">There are still some distortions.</td></tr>

<tr><td width="40%">RealAudio 1.0 (14.4K)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Real 14400 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 2.0 (28.8K)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Real 28800 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 3.0 (dnet)</td><td width="10%">IX</td><td width="10%">X</td><td width="40%">Real low bitrate AC-3 codec</td></tr>

<tr><td width="40%">RealAudio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealAudio SIPR / ACELP.NET</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SBC (low-complexity subband codec)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Smacker audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SMPTE 302M AES3 audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sonic</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Sonic lossless</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Speex</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">supported through external library libspeex</td></tr>

<tr><td width="40%">TAK (Tom’s lossless Audio Kompressor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">True Audio (TTA)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">TrueHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in HD-DVD and Blu-Ray discs.</td></tr>

<tr><td width="40%">TwinVQ (VQF flavor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ViewQuest VQC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Vorbis</td><td width="10%">E</td><td width="10%">X</td><td width="40%">A native but very primitive encoder exists.</td></tr>

<tr><td width="40%">Voxware MetaSound</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Westwood Audio (SND1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio 1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Pro</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Audio Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Xbox Media Audio 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Xbox Media Audio 2</td><td width="10%"></td><td width="10%">X</td></tr>

</tbody>

</table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code> means that support is provided through an external library. <code class="code">I</code> means that an integer-only version is available, too (ensures high performance on systems without hardware floating point support). <h3 class="section">44.5 Subtitle Formats# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Muxing</td><td width="10%">Demuxing</td><td width="10%">Encoding</td><td width="10%">Decoding</td></tr>
<tr><td width="40%">3GPP Timed Text</td><td width="10%"></td><td width="10%"></td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">AQTitle</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">DVB</td><td width="10%">X</td><td width="10%">X</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">DVB teletext</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">E</td></tr>
<tr><td width="40%">DVD</td><td width="10%">X</td><td width="10%">X</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">JACOsub</td><td width="10%">X</td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MicroDVD</td><td width="10%">X</td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MPL2</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">MPsub (MPlayer)</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">PGS</td><td width="10%"></td><td width="10%"></td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">PJS (Phoenix)</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">RealText</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">SAMI</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">Spruce format (STL)</td><td width="10%"></td><td width="10%">X</td><td width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">SSA/ASS</td><td

width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubRip (SRT)</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer v1</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>TED Talks cap-
tions</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>TTML</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td></tr> <tr><td width=”40%”>VobSub (IDX+SUB)</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>VPlayer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>WebVTT</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>XSUB</td><td
width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> </tbody> </table>

<code class=”code”>X</code> means that the feature is sup-
ported. <code class=”code”>E</code> means that support
is provided through an external library. <a name=”Network-
Protocols”> <h3 class=”section”>44.6 Network Protocols<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Network-Protocols”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
Network-Protocols” aria-hidden=”true”>TOC</h3>

<table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td
width=”10%”>Support</td></tr> <tr><td width=”40%”>AMQP</td><td
width=”10%”>E</td></tr> <tr><td width=”40%”>file</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>FTP</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Gopher</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Gophers</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>HLS</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>HTTP</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>HTTPS</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Icecast</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>MMSH</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>MMST</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>pipe</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Pro-MPEG FEC</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>RTMP</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPE</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPS</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPT</td><td

width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPTE</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTMPTS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SAMBA</td><td width=”10%”>E</td></tr> <tr><td width=”40%”>SCTP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SFTP</td><td width=”10%”>E</td></tr> <tr><td width=”40%”>TCP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>TLS</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>UDP</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>ZMQ</td><td width=”10%”>E</td></tr> </tbody> </table>

<code class=”code”>X</code> means that the protocol is supported. <code class=”code”>E</code> means that support is provided through an external library.

 <h3 class=”section”>44.7 Input/Output Devices# TOC</h3>

<table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Input</td><td width=”10%”>Output</td></tr> <tr><td width=”40%”>ALSA</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>BKTR</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>caca</td><td width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>DV1394</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>Lavfi virtual device</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>Linux framebuffer</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JACK</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>LIBCDIO</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>LIBDC1394</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>OpenAL</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>OSS</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>PulseAudio</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SDL</td><td width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>Video4Linux2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>VfW capture</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>X11 grabbing</td><td width=”10%”>X</td><td width=”10%”></td></tr> <tr><td width=”40%”>Win32 grabbing</td><td width=”10%”>X</td><td width=”10%”></td></tr> </tbody> </table>

<code class=”code”>X</code> means that input/output is supported. <h3 class=”section”>44.8 Time-

code# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Codec/format</td><td width="10%">Read</td><td width="10%">Write</td></tr> <tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">DV</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MOV</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MPEG1/2</td><td width="10%">X</td><td width="10%">X</td></tr> <tr><td width="40%">MXF</td><td width="10%">X</td><td width="10%">X</td></tr> </tbody> </table>

 <h2 class="chapter">45 See Also# TOC</h2>

ffprobe, ffmpeg, ffplay, ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class="chapter">46 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\ffprobe.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> ffprobe Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> ffprobe Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Synopsis 2 Description 3 Options <ul class="toc-numbered-mark"> 3.1 Stream specifiers 3.2 Generic options 3.3 AVOptions 3.4 Main options 4 Writers <ul class="toc-numbered-mark"> 4.1 default 4.2 compact, csv 4.3 flat 4.4 ini 4.5 json 4.6 xml 5 Timecode 6 See Also 7 Authors

 <h2 class="chapter">1 Synopsis# TOC</h2>

ffprobe [<var class="var">options</var>] <samp class="file">input_url</samp> <h2 class="chapter">2 Description# TOC</h2>

ffprobe gathers information from multimedia streams and prints it in human- and machine-readable fashion. For example it can be used

to check the format of the container used by a multimedia stream and the format and type of each media stream contained in it. If a url is specified in input, ffprobe will try to open and probe the url content. If the url cannot be opened or recognized as a multimedia file, a positive exit code is returned. If no output is specified as output with <samp class="option">o</samp> ffprobe will write to stdout. ffprobe may be employed both as a standalone application or in combination with a textual filter, which may perform more sophisticated processing, e.g. statistical processing or plotting. Options are used to list some of the formats supported by ffprobe or for specifying which information to display, and for setting how ffprobe will show it. ffprobe output is designed to be easily parsable by a textual filter, and consists of one or more sections of a form defined by the selected writer, which is specified by the <samp class="option">output_format</samp> option. Sections may contain other nested sections, and are identified by a name (which may be shared by other sections), and an unique name. See the output of <samp class="option">sections</samp>. Metadata tags stored in the container or in the streams are recognized and printed in the corresponding "FORMAT", "STREAM", "STREAM_GROUP_STREAM" or "PROGRAM_STREAM" section.

 <h2 class="chapter">3 Options# TOC</h2>

All the numerical options, if not specified otherwise, accept a string

representing a number as input, which may be followed by one of the SI

unit prefixes, for example: ’K’, ’M’, or ’G’.

If ’i’ is appended to the SI unit prefix, the complete prefix will be

interpreted as a unit prefix for binary multiples, which are based on

powers of 1024 instead of powers of 1000. Appending ’B’ to the SI unit

prefix multiplies the value by 8. This allows using, for example:

’KB’, ’MiB’, ’G’ and ’B’ as number suffixes.

Options which do not take arguments are boolean options, and set the

corresponding value to true. They can be set to false by prefixing

the option name with "no". For example using "-nofoo"

will set the boolean option with name "foo" to false.

Options that take arguments support a special syntax where the argument given on

the command line is interpreted as a path to the file from which the actual

argument value is loaded. To use this feature, add a forward slash ’/’

immediately before the option name (after the leading dash). E.g.

<pre class="example-preformatted">ffmpeg -i INPUT -/filter:v filter.script OUTPUT

</pre>

will load a filtergraph description from the file named <samp class="file">filter.script</samp>.

<h3 class="section">3.1 Stream specifiers#
TOC</h3>

Some options are applied per-stream, e.g. bitrate or codec. Stream specifiers

are used to precisely specify which stream(s) a given option belongs to.

A stream specifier is a string generally appended to the option name and

separated from it by a colon. E.g. <code class="code">-codec:a:1 ac3</code> contains the

<code class="code">a:1</code> stream specifier, which matches the second audio stream. Therefore, it

would select the ac3 codec for the second audio stream.

A stream specifier can match several streams, so that the option is applied to all

of them. E.g. the stream specifier in <code class=”code”>-b:a 128k</code> matches all audio

streams.

An empty stream specifier matches all streams. For example, <code class=”code”>-codec copy</code>

or <code class=”code”>-codec: copy</code> would copy all the streams without reencoding.

Possible forms of stream specifiers are:

<dl class=”table”>

<dt><samp class=”option”><var class=”var”>stream_index</var></samp></dt>

<dd> Matches the stream with this index. E.g. <code class=”code”>-threads:1 4</code> would set the

thread count for the second stream to 4. If <var class=”var”>stream_index</var> is used as an

additional stream specifier (see below), then it selects stream number

<var class=”var”>stream_index</var> from the matching streams. Stream numbering is based on the

order of the streams as detected by libavformat except when a stream group

specifier or program ID is also specified. In this case it is based on the

ordering of the streams in the group or program.

</dd>

<dt><samp class=”option”><var class=”var”>stream_type</var>[:<var class=”var”>additional_stream_specifier</var>]</samp></dt>

<dd> <var class=”var”>stream_type</var> is one of following: ’v’ or ’V’ for video, ’a’ for audio, ’s’

for subtitle, ’d’ for data, and ’t’ for attachments. ’v’ matches all video

streams, ’V’ only matches video streams which are not attached pictures, video

thumbnails or cover arts. If <var class=”var”>additional_stream_specifier</var> is used, then

it matches streams which both have this type and match the

<var class="var">additional_stream_specifier</var>. Otherwise, it matches all streams of the

specified type.

</dd>

<dt><samp class="option">g:<var class="var">group_specifier</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the group with the specifier <var class="var">group_specifier</var>.

if <var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the group and match the <var class="var">additional_stream_specifier</var>.

<var class="var">group_specifier</var> may be one of the following:

<dl class="table">

<dt><samp class="option"><var class="var">group_index</var></samp></dt>

<dd> Match the stream with this group index.

</dd>

<dt><samp class="option">#<var class="var">group_id</var> or i:<var class="var">group_id</var></samp></dt>

<dd> Match the stream with this group id.

</dd>

</dl>

</dd>

<dt><samp class="option">p:<var class="var">program_id</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams which are in the program with the id <var class="var">program_id</var>. If

<var class="var">additional_stream_specifier</var> is used, then it matches streams which both

are part of the program and match the <var class="var">additional_stream_specifier</var>.

</dd>

<dt><samp class="option">#<var class="var">stream_id</var> or i:<var class="var">stream_id</var></samp></dt>

<dd> Match the stream by stream id (e.g. PID in MPEG-TS container).

</dd>

<dt><samp class="option">m:<var class="var">key</var>[:<var class="var">value</var>]</samp></d

<dd> Matches streams with the metadata tag <var class="var">key</var> having the specified value. If

<var class="var">value</var> is not given, matches streams that contain the given tag with any

value. The colon character ’:’ in <var class="var">key</var> or <var class="var">value</var> needs to be

backslash-escaped.

</dd>

<dt><samp class="option">disp:<var class="var">dispositions</var>[:<var class="var">additional_stream_specifier</var>]</samp></dt>

<dd> Matches streams with the given disposition(s). <var class="var">dispositions</var> is a list of

one or more dispositions (as printed by the <samp class="option">-dispositions</samp> option)

joined with ’+’.

</dd>

<dt><samp class="option">u</samp></dt>

<dd> Matches streams with usable configuration, the codec must be defined and the

essential information such as video dimension or audio sample rate must be present.

Note that in <code class="command">ffmpeg</code>, matching by metadata will only work properly for

input files.

</dd>

</dl>

 <h3 class="section">3.2 Generic options# TOC</h3>

These options are shared amongst the ff* tools. <dl class="table"> <dt><samp class="option">-L, -license</samp></dt> <dd> Show

license. </dd> <dt><samp class="option">-h, -?, -help, --help [<var class="var">arg</var>]</samp></dt> <dd> Show help. An optional parameter may be specified to print help about a specific item. If no argument is specified, only basic (non advanced) tool options are shown. Possible values of <var class="var">arg</var> are: <dl class="table"> <dt><samp class="option">long</samp></dt> <dd> Print advanced tool options in addition to the basic tool options. </dd> <dt><samp class="option">full</samp></dt> <dd> Print complete list of options, including shared and private options for encoders, decoders, demuxers, muxers, filters, etc. </dd> <dt><samp class="option">decoder=<var class="var">decoder_name</var></samp></dt> <dd> Print detailed information about the decoder named <var class="var">decoder_name</var>. Use the <samp class="option">-decoders</samp> option to get a list of all decoders. </dd> <dt><samp class="option">encoder=<var class="var">encoder_name</var></samp></dt> <dd> Print detailed information about the encoder named <var class="var">encoder_name</var>. Use the <samp class="option">-encoders</samp> option to get a list of all encoders. </dd> <dt><samp class="option">demuxer=<var class="var">demuxer_name</var></samp></dt> <dd> Print detailed information about the demuxer named <var class="var">demuxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all demuxers and muxers. </dd> <dt><samp class="option">muxer=<var class="var">muxer_name</var></samp></dt> <dd> Print detailed information about the muxer named <var class="var">muxer_name</var>. Use the <samp class="option">-formats</samp> option to get a list of all muxers and demuxers. </dd> <dt><samp class="option">filter=<var class="var">filter_name</var></samp></dt> <dd> Print detailed information about the filter named <var class="var">filter_name</var>. Use the <samp class="option">-filters</samp> option to get a list of all filters. </dd> <dt><samp class="option">bsf=<var class="var">bitstream_filter_name</var></samp></dt> <dd> Print detailed information about the bitstream filter named <var class="var">bitstream_filter_name</var>. Use the <samp class="option">-bsfs</samp> option to get a list of all bitstream filters. </dd> <dt><samp class="option">protocol=<var class="var">protocol_name</var></samp></dt> <dd> Print detailed information about the protocol named <var class="var">protocol_name</var>. Use the <samp class="option">-protocols</samp> option to get a list of all protocols. </dd> </dl>

</dd> <dt><samp class="option">-version</samp></dt> <dd> Show version. </dd> <dt><samp class="option">-buildconf</samp></dt> <dd> Show the build configuration, one option per line. </dd> <dt><samp class="option">-formats</samp></dt> <dd> Show available formats (including devices). </dd> <dt><samp class="option">-demuxers</samp></dt> <dd> Show available demuxers. </dd> <dt><samp class="option">-muxers</samp></dt> <dd> Show avail-

able muxers. </dd> <dt><samp class="option">-devices</samp></dt>
<dd> Show available devices. </dd> <dt><samp class="option">-
codecs</samp></dt> <dd> Show all codecs known to libavcodec.
 Note that the term ’codec’ is used throughout
this documentation as a shortcut for what is more correctly called a
media bitstream format. </dd> <dt><samp class="option">-
decoders</samp></dt> <dd> Show available decoders. </dd>
<dt><samp class="option">-encoders</samp></dt> <dd> Show
all available encoders. </dd> <dt><samp class="option">-
bsfs</samp></dt> <dd> Show available bitstream filters. </dd>
<dt><samp class="option">-protocols</samp></dt> <dd> Show avail-
able protocols. </dd> <dt><samp class="option">-filters</samp></dt>
<dd> Show available libavfilter filters. </dd> <dt><samp
class="option">-pix_fmts</samp></dt> <dd> Show available pixel for-
mats. </dd> <dt><samp class="option">-sample_fmts</samp></dt>
<dd> Show available sample formats. </dd> <dt><samp
class="option">-layouts</samp></dt> <dd> Show channel names
and standard channel layouts. </dd> <dt><samp class="option">-
dispositions</samp></dt> <dd> Show stream dispositions.
</dd> <dt><samp class="option">-colors</samp></dt> <dd> Show
recognized color names. </dd> <dt><samp class="option">-sources
<var class="var">device</var>[,<var class="var">opt1</var>=<var
class="var">val1</var>[,<var class="var">opt2</var>=<var class="var">val2</var>]...]</samp></dt>
<dd> Show autodetected sources of the input device. Some devices may
provide system-dependent source names that cannot be autodetected.
The returned list cannot be assumed to be always complete. <div
class="example"> <pre class="example-preformatted">ffmpeg -sources
pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-sinks <var class="var">device</var>[,<var
class="var">opt1</var>=<var class="var">val1</var>[,<var class="var">opt2</var>=<var
class="var">val2</var>]...]</samp></dt> <dd> Show autodetected
sinks of the output device. Some devices may provide system-dependent sink
names that cannot be autodetected. The returned list cannot be assumed
to be always complete. <pre class="example-
preformatted">ffmpeg -sinks pulse,server=192.168.0.4 </pre>

</dd> <dt><samp class="option">-loglevel [<var class="var">flags</var>+]<var
class="var">loglevel</var> | -v [<var class="var">flags</var>+]<var
class="var">loglevel</var></samp></dt> <dd> Set logging level and
flags used by the library. The optional <var class="var">flags</var>
prefix can consist of the following values: <dl class="table">
<dt>‘<samp class="samp">repeat</samp>’</dt> <dd> Indicates
that repeated log output should not be compressed to the first line and
the "Last message repeated n times" line will be omitted.
</dd> <dt>‘<samp class="samp">level</samp>’</dt>
<dd> Indicates that log output should add a <code class="code">[level]</code>

prefix to each message line. This can be used as an alternative to log coloring, e.g. when dumping the log to file. </dd> <dt>‘<samp class="samp">time</samp>’</dt> <dd> Indicates that log lines should be prefixed with time information. </dd> <dt>‘<samp class="samp">datetime</samp>’</dt> <dd> Indicates that log lines should be prefixed with date and time information. </dd> </dl> Flags can also be used alone by adding a ’+’/’-’ prefix to set/reset a single flag without affecting other <var class="var">flags</var> or changing <var class="var">loglevel</var>. When setting both <var class="var">flags</var> and <var class="var">loglevel</var>, a ’+’ separator is expected between the last <var class="var">flags</var> value and before <var class="var">loglevel</var>. <var class="var">loglevel</var> is a string or a number containing one of the following values: <dl class="table"> <dt>‘<samp class="samp">quiet, -8</samp>’</dt> <dd> Show nothing at all; be silent. </dd> <dt>‘<samp class="samp">panic, 0</samp>’</dt> <dd> Only show fatal errors which could lead the process to crash, such as an assertion failure. This is not currently used for anything. </dd> <dt>‘<samp class="samp">fatal, 8</samp>’</dt> <dd> Only show fatal errors. These are errors after which the process absolutely cannot continue. </dd> <dt>‘<samp class="samp">error, 16</samp>’</dt> <dd> Show all errors, including ones which can be recovered from. </dd> <dt>‘<samp class="samp">warning, 24</samp>’</dt> <dd> Show all warnings and errors. Any message related to possibly incorrect or unexpected events will be shown. </dd> <dt>‘<samp class="samp">info, 32</samp>’</dt> <dd> Show informative messages during processing. This is in addition to warnings and errors. This is the default value. </dd> <dt>‘<samp class="samp">verbose, 40</samp>’</dt> <dd> Same as <code class="code">info</code>, except more verbose. </dd> <dt>‘<samp class="samp">debug, 48</samp>’</dt> <dd> Show everything, including debugging information. </dd> <dt>‘<samp class="samp">trace, 56</samp>’</dt> </dl>

For example to enable repeated log output, add the <code class="code">level</code> prefix, and set <var class="var">loglevel</var> to <code class="code">verbose</code>: <pre class="example-preformatted">ffmpeg -loglevel repeat+level+verbose -i input output </pre> Another example that enables repeated log output without affecting current state of <code class="code">level</code> prefix flag or <var class="var">loglevel</var>: <pre class="example-preformatted">ffmpeg [...] -loglevel +repeat </pre>

By default the program logs to stderr. If coloring is supported by the terminal, colors are used to mark errors and warnings. Log coloring can be disabled setting the environment variable <code class="env">AV_LOG_FORCE_NOCOLOR</code>,

or can be forced setting the environment variable <code class=”env”>AV_LOG_FORCE_COLOR</code>. </dd> <dt><samp class=”option”>-report</samp></dt> <dd> Dump full command line and log output to a file named <code class=”code”><var class=”var”>program</var>-<var class=”var”>YYYYMMDD</var>-<var class=”var”>HHMMSS</var>.log</code> in the current directory. This file can be useful for bug reports. It also implies <code class=”code”>-loglevel debug</code>. Setting the environment variable <code class=”env”>FFREPORT</code> to any value has the same effect. If the value is a ’:’-separated key=value sequence, these options will affect the report; option values must be escaped if they contain special characters or the options delimiter ’:’ (see the “Quoting and escaping” section in the ffmpeg-utils manual). The following options are recognized: <dl class=”table”> <dt><samp class=”option”>file</samp></dt> <dd> set the file name to use for the report; <code class=”code”>%p</code> is expanded to the name of the program, <code class=”code”>%t</code> is expanded to a timestamp, <code class=”code”>%%</code> is expanded to a plain <code class=”code”>%</code> </dd> <dt><samp class=”option”>level</samp></dt> <dd> set the log verbosity level using a numerical value (see <code class=”code”>-loglevel</code>). </dd> </dl>

For example, to output a report to a file named <samp class=”file”>ffreport.log</samp> using a log level of <code class=”code”>32</code> (alias for log level <code class=”code”>info</code>):   <pre class=”example-preformatted”>FFREPORT=file=ffreport.log:level=32 ffmpeg -i input output </pre>

Errors in parsing the environment variable are not fatal, and will not appear in the report. </dd> <dt><samp class=”option”>-hide_banner</samp></dt> <dd> Suppress printing banner. All FFmpeg tools will normally show a copyright notice, build options and library versions. This option can be used to suppress printing this information. </dd> <dt><samp class=”option”>-cpuflags flags (global)</samp></dt> <dd> Allows setting and clearing cpu flags. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class=”example-preformatted”>ffmpeg -cpuflags -sse+mmx ... ffmpeg -cpuflags mmx ... ffmpeg -cpuflags 0 ... </pre> Possible flags for this option are: <dl class=”table”> <dt>‘<samp class=”samp”>x86</samp>’</dt> <dd><dl class=”table”> <dt>‘<samp class=”samp”>mmx</samp>’</dt> <dt>‘<samp class=”samp”>mmxext</samp>’</dt> <dt>‘<samp class=”samp”>sse</samp>’</d <dt>‘<samp class=”samp”>sse2</samp>’</dt> <dt>‘<samp class=”samp”>sse2slow</samp>’</dt> <dt>‘<samp class=”samp”>sse3</samp>’</ <dt>‘<samp class=”samp”>sse3slow</samp>’</dt> <dt>‘<samp class=”samp”>ssse3</samp>’</dt> <dt>‘<samp class=”samp”>atom</samp>’</dt

<dt>‘<samp class="samp">sse4.1</samp>’</dt> <dt>‘<samp
class="samp">sse4.2</samp>’</dt> <dt>‘<samp class="samp">avx</samp>’</dt>
<dt>‘<samp class="samp">avx2</samp>’</dt> <dt>‘<samp
class="samp">xop</samp>’</dt> <dt>‘<samp class="samp">fma3</samp>’</dt>
<dt>‘<samp class="samp">fma4</samp>’</dt> <dt>‘<samp
class="samp">3dnow</samp>’</dt> <dt>‘<samp class="samp">3dnowext</samp>&rsquo
<dt>‘<samp class="samp">bmi1</samp>’</dt> <dt>‘<samp
class="samp">bmi2</samp>’</dt> <dt>‘<samp class="samp">cmov</samp>’</dt
</dl> </dd> <dt>‘<samp class="samp">ARM</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv5te</samp>’</dt>
<dt>‘<samp class="samp">armv6</samp>’</dt> <dt>‘<samp
class="samp">armv6t2</samp>’</dt> <dt>‘<samp class="samp">vfp</samp>’</d
<dt>‘<samp class="samp">vfpv3</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> <dt>‘<samp class="samp">setend</samp>’</d
</dl> </dd> <dt>‘<samp class="samp">AArch64</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">armv8</samp>’</dt>
<dt>‘<samp class="samp">vfp</samp>’</dt> <dt>‘<samp
class="samp">neon</samp>’</dt> </dl> </dd> <dt>‘<samp
class="samp">PowerPC</samp>’</dt> <dd><dl class="table">
<dt>‘<samp class="samp">altivec</samp>’</dt> </dl>
</dd> <dt>‘<samp class="samp">Specific Processors</samp>’</dt>
<dd><dl class="table"> <dt>‘<samp class="samp">pentium2</samp>’</dt>
<dt>‘<samp class="samp">pentium3</samp>’</dt>
<dt>‘<samp class="samp">pentium4</samp>’</dt>
<dt>‘<samp class="samp">k6</samp>’</dt> <dt>‘<samp
class="samp">k62</samp>’</dt> <dt>‘<samp class="samp">athlon</samp>’</dt
<dt>‘<samp class="samp">athlonxp</samp>’</dt> <dt>‘<samp
class="samp">k8</samp>’</dt> </dl> </dd> </dl>

</dd> <dt><samp class="option">-cpucount <var class="var">count</var>
(global)</samp></dt> <dd> Override detection of CPU count. This option is intended for testing. Do not use it unless you know what you’re doing. <pre class="example-preformatted">ffmpeg -cpucount 2 </pre>

</dd> <dt><samp class="option">-max_alloc <var class="var">bytes</var></samp></dt>
<dd> Set the maximum size limit for allocating a block on the heap by ffmpeg’s family of malloc functions. Exercise extreme caution when using this option. Don’t use if you do not understand the full consequence of doing so. Default is INT_MAX. </dd> </dl>

 <h3 class="section">3.3 AVOptions# TOC</h3>

These options are provided directly by the libavformat, libavdevice and

libavcodec libraries. To see the list of available AVOptions, use the <samp class=”option”>-help</samp> option. They are separated into two categories: <dl class=”table”> <dt><samp class=”option”>generic</samp></dt> <dd> These options can be set for any container, codec or device. Generic options are listed under AVFormatContext options for containers/devices and under AVCodecContext options for codecs. </dd> <dt><samp class=”option”>private</samp></dt> <dd> These options are specific to the given container, device or codec. Private options are listed under their corresponding containers/devices/codecs. </dd> </dl>

For example to write an ID3v2.3 header instead of a default ID3v2.4 to an MP3 file, use the <samp class=”option”>id3v2_version</samp> private option of the MP3 muxer: <pre class=”example-preformatted”>ffmpeg -i input.flac -id3v2_version 3 out.mp3 </pre>

All codec AVOptions are per-stream, and thus a stream specifier should be attached to them: <pre class=”example-preformatted”>ffmpeg -i multichannel.mxf -map 0:v:0 -map 0:a:0 -map 0:a:0 -c:a:0 ac3 -b:a:0 640k -ac:a:1 2 -c:a:1 aac -b:2 128k out.mp4 </pre>

In the above example, a multichannel audio stream is mapped twice for output. The first instance is encoded with codec ac3 and bitrate 640k. The second instance is downmixed to 2 channels and encoded with codec aac. A bitrate of 128k is specified for it using absolute index of the output stream. Note: the <samp class=”option”>-nooption</samp> syntax cannot be used for boolean AVOptions, use <samp class=”option”>-option 0</samp>/<samp class=”option”>-option 1</samp>. Note: the old undocumented way of specifying per-stream AVOptions by prepending v/a/s to the options name is now obsolete and will be removed soon. <h3 class=”section”>3.4 Main options# TOC</h3>

<dl class=”table”> <dt><samp class=”option”>-f <var class=”var”>format</var></samp></dt> <dd> Force format to use. </dd> <dt><samp class=”option”>-unit</samp></dt> <dd> Show the unit of the displayed values. </dd> <dt><samp class=”option”>-prefix</samp></dt> <dd> Use SI prefixes for the displayed values. Unless the "-byte_binary_prefix" option is used all the prefixes are decimal. </dd> <dt><samp class=”option”>-byte_binary_prefix</samp></dt> <dd> Force the use of binary prefixes for byte values. </dd> <dt><samp class=”option”>-sexagesimal</samp></dt> <dd> Use sexagesimal format HH:MM:SS.MICROSECONDS for time values. </dd> <dt><samp class=”option”>-pretty</samp></dt> <dd> Prettify the format of the displayed values, it corresponds to the options "-unit -prefix -byte_binary_prefix -sexagesimal". </dd> <dt><samp class=”option”>-output_format, -of, -print_format <var

class=”var”>writer_name</var>[=<var class=”var”>writer_options</var>]</samp></dt> <dd> Set the output printing format. <var class=”var”>writer_name</var> specifies the name of the writer, and <var class=”var”>writer_options</var> specifies the options to be passed to the writer. For example for printing the output in JSON format, specify: <pre class=”example-preformatted”>-output_format json </pre>

For more details on the available output printing formats, see the Writers section below. </dd> <dt><samp class=”option”>-sections</samp></dt> <dd> Print sections structure and section information, and exit. The output is not meant to be parsed by a machine. </dd> <dt><samp class=”option”>-select_streams <var class=”var”>stream_specifier</var></samp></dt> <dd> Select only the streams specified by <var class=”var”>stream_specifier</var>. This option affects only the options related to streams (e.g. <code class=”code”>show_streams</code>, <code class=”code”>show_packets</code>, etc.). For example to show only audio streams, you can use the command: <pre class=”example-preformatted”>ffprobe -show_streams -select_streams a INPUT </pre>

To show only video packets belonging to the video stream with index 1: <pre class=”example-preformatted”>ffprobe -show_packets -select_streams v:1 INPUT </pre>

</dd> <dt><samp class=”option”>-show_data</samp></dt> <dd> Show payload data, as a hexadecimal and ASCII dump (other formats can be selected using <samp class=”option”>-data_dump_format</samp>). Coupled with <samp class=”option”>-show_packets</samp>, it will dump the packets’ data. Coupled with <samp class=”option”>-show_streams</samp>, it will dump the codec extradata. The dump is printed as the "data" field. It may contain newlines. </dd> <dt><samp class=”option”>-show_data_hash <var class=”var”>algorithm</var></samp></dt> <dd> Show a hash of payload data, for packets with <samp class=”option”>-show_packets</samp> and for codec extradata with <samp class=”option”>-show_streams</samp>. </dd> <dt><samp class=”option”>-data_dump_format <var class=”var”>format</var></samp></dt> <dd> Select a format used for the data dumps enabled with the <samp class=”option”>-show_data</samp> option. The default is <code class=”code”>xxd</code> which is a hexdump format compatible with the well-known <code class=”command”>xxd</code> program. <code class=”code”>base64</code> is also supported. </dd> <dt><samp class=”option”>-show_error</samp></dt> <dd> Show information about the error found when trying to probe the input. The error information is printed within a section with name "ERROR". </dd> <dt><samp class=”option”>-show_format</samp></dt> <dd> Show information about the

container format of the input multimedia stream. All the container format information is printed within a section with name "FORMAT". </dd> <dt><samp class="option">-show_entries <var class="var">section_entries</var></samp></dt> <dd> Set list of entries to show. Entries are specified according to the following syntax. <var class="var">section_entries</var> contains a list of section entries separated by <code class="code">:</code>. Each section entry is composed by a section name (or unique name), optionally followed by a list of entries local to that section, separated by <code class="code">,</code>. If section name is specified but is followed by no <code class="code">=</code>, all entries are printed to output, together with all the contained sections. Otherwise only the entries specified in the local section entries list are printed. In particular, if <code class="code">=</code> is specified but the list of local entries is empty, then no entries will be shown for that section. Note that the order of specification of the local section entries is not honored in the output, and the usual display order will be retained. The formal syntax is given by: <pre class="example-preformatted"><var class="var">LOCAL_SECTION_ENTRIES</var> ::= <var class="var">SECTION_ENTRY_NAME</var>[,<var class="var">LOCAL_SECTION_ENTRIES</
<var class="var">SECTION_ENTRY</var> ::= <var class="var">SECTION_NAME</var>[=[<var class="var">LOCAL_SECTION_ENTRIES</var>]] <var class="var">SECTION_ENTRIES</var> ::= <var class="var">SECTION_ENTRY</var>[:<var class="var">SECTION_ENTRIES</var>] </pre>

For example, to show only the index and type of each stream, and the PTS time, duration time, and stream index of the packets, you can specify the argument: <pre class="example-preformatted">packet=pts_time,duration_time,stream_index : stream=index,codec_type </pre>

To show all the entries in the section "format", but only the codec type in the section "stream", specify the argument: <pre class="example-preformatted">format : stream=codec_type </pre>

To show all the tags in the stream and format sections: <pre class="example-preformatted">stream_tags : format_tags </pre>

To show only the <code class="code">title</code> tag (if available) in the stream sections: <pre class="example-preformatted">stream_tags=title </pre>

Selecting <code class="code">stream</code> or <code class="code">streams</code> does not by itself enable program or

stream group output. To also show streams within programs or groups, use

<code class="code">-show_programs</code>, <code class="code">-show_stream_groups</code>, or select their unique

section name, for example <code class="code">-show_entries program_stream=index</code> or

<code class="code">-show_entries stream_group_stream=index</code>. The shared names still select

fields within programs and stream groups when their output is enabled.

</dd>

<dt><samp class="option">-show_packets</samp></dt>

<dd> Show information about each packet contained in the input multimedia

stream.

The information for each single packet is printed within a dedicated

section with name "PACKET".

</dd>

<dt><samp class="option">-show_frames</samp></dt>

<dd> Show information about each frame and subtitle contained in the input

multimedia stream.

The information for each single frame is printed within a dedicated

section with name "FRAME" or "SUBTITLE".

</dd>

<dt><samp class="option">-show_log <var class="var">loglevel</var></samp></dt>

<dd> Show logging information from the decoder about each frame according to

the value set in <var class="var">loglevel</var>, (see <code class="code">-loglevel</code>). This option requires <code class="code">-show_frames</code>.

The information for each log message is printed within a dedicated

section with name "LOG".

</dd>

<dt><samp class=”option”>-show_streams</samp></dt>

<dd> Show information about each media stream contained in the input

multimedia stream.

Each media stream information is printed within a dedicated section

with name "STREAM".

</dd>

<dt><samp class=”option”>-show_programs</samp></dt>

<dd> Show information about programs and their streams contained in the input

multimedia stream.

Each media stream information is printed within a dedicated section

with name "PROGRAM_STREAM".

</dd>

<dt><samp class=”option”>-show_stream_groups</samp></dt>

<dd> Show information about stream groups and their streams contained in the

input multimedia stream.

Each media stream information is printed within a dedicated section

with name "STREAM_GROUP_STREAM".

</dd>

<dt><samp class=”option”>-show_chapters</samp></dt>

<dd> Show information about chapters stored in the format.

Each chapter is printed within a dedicated section with name "CHAPTER".

</dd>

<dt><samp class="option">-count_frames</samp></dt>

<dd> Count the number of frames per stream and report it in the

corresponding stream section.

</dd>

<dt><samp class="option">-count_packets</samp></dt>

<dd> Count the number of packets per stream and report it in the

corresponding stream section.

</dd>

<dt><samp class="option">-read_intervals <var class="var">read_intervals</var></samp></dt>

<dd>

Read only the specified intervals. <var class="var">read_intervals</var> must be a

sequence of interval specifications separated by ",".

<code class="command">ffprobe</code> will seek to the interval starting point, and will

continue reading from that.

Each interval is specified by two optional parts, separated by "%".

The first part specifies the interval start position. It is

interpreted as an absolute position, or as a relative offset from the

current position if it is preceded by the "+" character. If this first

part is not specified, no seeking will be performed when reading this

interval.

The second part specifies the interval end position. It is interpreted
as an absolute position, or as a relative offset from the current
position if it is preceded by the "+" character. If the offset
specification starts with "#", it is interpreted as the number of
packets to read (not including the flushing packets) from the interval
start. If no second part is specified, the program will read until the
end of the input.

Note that seeking is not accurate, thus the actual interval start
point may be different from the specified position. Also, when an
interval duration is specified, the absolute end time will be computed
by adding the duration to the interval start point found by seeking
the file, rather than to the specified start value.

The formal syntax is given by:

<pre class="example-preformatted"><var class="var">INTERVAL</var>
::= [<var class="var">START</var>|+<var class="var">START_OFFSET</var>][%[<var
class="var">END</var>|+<var class="var">END_OFFSET</var>]]
<var class="var">INTERVALS</var> ::= <var class="var">INTERVAL</var>[,<var
class="var">INTERVALS</var>]
</pre>
A few examples follow. <ul class="itemize mark-bullet"> Seek
to time 10, read packets until 20 seconds after the found seek point, then seek
to position <code class="code">01:30</code> (1 minute and thirty seconds)
and read packets until position <code class="code">01:45</code>. <div
class="example"> <pre class="example-preformatted">10%+20,01:30%01:45
</pre>
Read only 42 packets after seeking to position <code class="code">01:23</code>:
 <pre class="example-preformatted">01:23%+#42
</pre>
Read only the first 20 seconds from the start:
<pre class="example-preformatted">%+20 </pre>

Read from the start until position <code class="code">02:30</code>:
 <pre class="example-preformatted">%02:30
</pre>

</dd> <dt><samp class="option">-show_private_data, -private</samp></dt>
<dd> Show private data, that is data depending on the format of the particular shown element. This option is enabled by default, but you may need to disable it for specific uses, for example when creating XSD-compliant XML output. </dd> <dt><samp class="option">-show_program_version</samp></dt> <dd> Show information related to program version. Version information is printed within a section with name "PROGRAM_VERSION". </dd> <dt><samp class="option">-show_library_versions</samp></dt> <dd> Show information related to library versions. Version information for each library is printed within a section with name "LIBRARY_VERSION". </dd> <dt><samp class="option">-show_versions</samp></dt> <dd> Show information related to program and library versions. This is the equivalent of setting both <samp class="option">-show_program_version</samp> and <samp class="option">-show_library_versions</samp> options. </dd> <dt><samp class="option">-show_pixel_formats</samp></dt> <dd> Show information about all pixel formats supported by FFmpeg. Pixel format information for each format is printed within a section with name "PIXEL_FORMAT". </dd> <dt><samp class="option">-show_optional_fields <var class="var">value</var></samp></dt> <dd> Some writers viz. JSON and XML, omit the printing of fields with invalid or non-applicable values, while other writers always print them. This option enables one to control this behaviour. Valid values are <code class="code">always</code>/<code class="code">1</code>, <code class="code">never</code>/<code class="code">0</code> and <code class="code">auto</code>/<code class="code">-1</code>. Default is <var class="var">auto</var>. </dd> <dt><samp class="option">-analyze_frames</samp></dt> <dd> Analyze frames and/or their side data up to the provided read interval, providing additional information that may be useful at a stream level. Must be paired with the <samp class="option">-show_streams</samp> option or it will have no effect. Currently, the additional fields provided by this option when enabled are the <code class="code">closed_captions</code> and <code class="code">film_grain</code> fields. For example, to analyze the first 20 seconds and populate these fields: <pre class="example-preformatted">ffprobe -show_streams -analyze_frames -read_intervals "%+20" INPUT </pre>

</dd> <dt><samp class="option">-bitexact</samp></dt> <dd> Force bitexact output, useful to produce output which is not dependent on the specific build. </dd> <dt><samp class="option">-i <var class="var">input_url</var></samp></dt> <dd> Read <var

class=”var”>input_url</var>. </dd> <dt><samp class=”option”>-o <var class=”var”>output_url</var></samp></dt> <dd> Write output to <var class=”var”>output_url</var>. If not specified, the output is sent to stdout. </dd> <dt><samp class=”option”>-c:<var class=”var”>media_specifier</var> <var class=”var”>codec_name</var></samp></dt> <dt><samp class=”option”>-codec:<var class=”var”>media_specifier</var> <var class=”var”>codec_name</var></samp></dt> <dd> Force a specific decoder implementation for the stream identified by <var class=”var”>media_specifier</var>, which can assume the values <code class=”code”>a</code> (audio), <code class=”code”>v</code> (video), <code class=”code”>s</code> (subtitle), and <code class=”code”>d</code> (data). </dd> </dl>

 <h2 class=”chapter”>4 Writers# TOC</h2>

A writer defines the output format adopted by <code class=”command”>ffprobe</code>, and will be used for printing all the parts of the output. A writer may accept one or more arguments, which specify the options to adopt. The options are specified as a list of <var class=”var”>key</var>=<var class=”var”>value</var> pairs, separated by ":". All writers support the following options: <dl class=”table”> <dt><samp class=”option”>string_validation, sv</samp></dt> <dd> Set string validation mode. The following values are accepted. <dl class=”table”> <dt>‘<samp class=”samp”>fail</samp>’</dt> <dd> The writer will fail immediately in case an invalid string (UTF-8) sequence or code point is found in the input. This is especially useful to validate input metadata. </dd> <dt>‘<samp class=”samp”>ignore</samp>’</dt> <dd> Any validation error will be ignored. This will result in possibly broken output, especially with the json or xml writer. </dd> <dt>‘<samp class=”samp”>replace</samp>’</dt> <dd> The writer will substitute invalid UTF-8 sequences or code points with the string specified with the <samp class=”option”>string_validation_replacement</samp>. </dd> </dl>

Default value is ‘<samp class=”samp”>replace</samp>’. </dd> <dt><samp class=”option”>string_validation_replacement, svr</samp></dt> <dd> Set replacement string to use in case <samp class=”option”>string_validation</samp> is set to ‘<samp class=”samp”>replace</samp>’. In case the option is not specified, the writer will assume the empty string, that is it will remove the invalid sequences from the input strings. </dd> </dl>

A description of the currently available writers follows. <h3 class=”section”>4.1 default#

TOC</h3> Default format. Print each section in the form: <pre class=”example-preformatted”>[SECTION] key1=val1 ... keyN=valN [/SECTION] </pre>

Metadata tags are printed as a line in the corresponding FORMAT, STREAM, STREAM_GROUP_STREAM or PROGRAM_STREAM section, and are prefixed by the string "TAG:". A description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>nokey, nk</samp></dt> <dd> If set to 1 specify not to print the key of each field. Default value is 0. </dd> <dt><samp class=”option”>noprint_wrappers, nw</samp></dt> <dd> If set to 1 specify not to print the section header and footer. Default value is 0. </dd> </dl>

 <h3 class=”section”>4.2 compact, csv# TOC</h3> Compact and CSV format. The <code class=”code”>csv</code> writer is equivalent to <code class=”code”>compact</code>, but supports different defaults. Each section is printed on a single line. If no option is specified, the output has the form: <pre class=”example-preformatted”>section|key1=val1| ... |keyN=valN </pre>

Metadata tags are printed in the corresponding "format" or "stream" section. A metadata tag key, if printed, is prefixed by the string "tag:". The description of the accepted options follows. <dl class=”table”> <dt><samp class=”option”>item_sep, s</samp></dt> <dd> Specify the character to use for separating fields in the output line. It must be a single printable character, it is "|" by default ("," for the <code class=”code”>csv</code> writer). </dd> <dt><samp class=”option”>nokey, nk</samp></dt> <dd> If set to 1 specify not to print the key of each field. Its default value is 0 (1 for the <code class=”code”>csv</code> writer). </dd> <dt><samp class=”option”>escape, e</samp></dt> <dd> Set the escape mode to use, default to "c" ("csv" for the <code class=”code”>csv</code> writer). It can assume one of the following values: <dl class=”table”> <dt><samp class=”option”>c</samp></dt> <dd> Perform C-like escaping. Strings containing a newline (‘<samp class=”samp”>\n</samp>’), carriage return (‘<samp class=”samp”>\r</samp>’), a tab (‘<samp class=”samp”>\t</samp>’), a form feed (‘<samp class=”samp”>\f</samp>’), the escaping character (‘<samp class=”samp”>\</samp>’) or the item separator character <var class=”var”>SEP</var> are escaped using

C-like fashioned escaping, so that a newline is converted to the sequence ‘<samp class="samp">\n</samp>’, a carriage return to ‘<samp class="samp">\r</samp>’, ‘<samp class="samp">\</samp>’ to ‘<samp class="samp">\\</samp>’ and the separator <var class="var">SEP</var> is converted to ‘<samp class="samp">\<var class="var">SEP</var></samp>’. </dd> <dt><samp class="option">csv</samp></dt> <dd> Perform CSV-like escaping, as described in RFC4180. Strings containing a newline (‘<samp class="samp">\n</samp>’), a carriage return (‘<samp class="samp">\r</samp>’), a double quote (‘<samp class="samp">"</samp>’), or <var class="var">SEP</var> are enclosed in double-quotes. </dd> <dt><samp class="option">none</samp></dt> <dd> Perform no escaping. </dd> </dl>

</dd> <dt><samp class="option">print_section, p</samp></dt> <dd> Print the section name at the beginning of each line if the value is <code class="code">1</code>, disable it with value set to <code class="code">0</code>. Default value is <code class="code">1</code>. </dd> </dl>

 <h3 class="section">4.3 flat# TOC</h3> Flat format. A free-form output where each line contains an explicit key=value, such as "streams.stream.3.tags.foo=bar". The output is shell escaped, so it can be directly embedded in sh scripts as long as the separator character is an alphanumeric character or an underscore (see <var class="var">sep_char</var> option). The description of the accepted options follows. <dl class="table"> <dt><samp class="option">sep_char, s</samp></dt> <dd> Separator character used to separate the chapter, the section name, IDs and potential tags in the printed field key. Default value is ‘<samp class="samp">.</samp>’. </dd> <dt><samp class="option">hierarchical, h</samp></dt> <dd> Specify if the section name specification should be hierarchical. If set to 1, and if there is more than one section in the current chapter, the section name will be prefixed by the name of the chapter. A value of 0 will disable this behavior. Default value is 1. </dd> </dl>

 <h3 class="section">4.4 ini# TOC</h3> INI format output. Print output in an INI based format. The following conventions are adopted: <ul class="itemize mark-bullet"> all key and values are UTF-8 ‘<samp class="samp">.</samp>’ is the subgroup separator newline,

‘<samp class="samp">\t</samp>’, ‘<samp class="samp">\f</samp>’, ‘<samp class="samp">\b</samp>’ and the following characters are escaped ‘<samp class="samp">\</samp>’ is the escape character ‘<samp class="samp">#</samp>’ is the comment indicator ‘<samp class="samp">=</samp>’ is the key/value separator ‘<samp class="samp">:</samp>’ is not used but usually parsed as key/value separator

This writer accepts options as a list of <var class="var">key</var>=<var class="var">value</var> pairs, separated by ‘<samp class="samp">:</samp>’. The description of the accepted options follows. <dl class="table"> <dt><samp class="option">hierarchical, h</samp></dt> <dd> Specify if the section name specification should be hierarchical. If set to 1, and if there is more than one section in the current chapter, the section name will be prefixed by the name of the chapter. A value of 0 will disable this behavior. Default value is 1. </dd> </dl>

 <h3 class="section">4.5 json# TOC</h3> JSON based format. Each section is printed using JSON notation. The description of the accepted options follows. <dl class="table"> <dt><samp class="option">compact, c</samp></dt> <dd> If set to 1 enable compact output, that is each section will be printed on a single line. Default value is 0. </dd> </dl>

For more information about JSON, see <a class="url" href="http://www.json.org/">http://www.json.org <h3 class="section">4.6 xml# TOC</h3> XML based format. The XML output is described in the XML schema description file <samp class="file">ffprobe.xsd</samp> installed in the FFmpeg datadir. An updated version of the schema can be retrieved at the url <a class="url" href="http://www.ffmpeg.org/schema/ffprobe.xsd">http://www.ffmpeg.org/ which redirects to the latest schema committed into the FFmpeg development source code tree. Note that the output issued will be compliant to the <samp class="file">ffprobe.xsd</samp> schema only when no special global output options (<samp class="option">unit</samp>, <samp class="option">prefix</samp>, <samp class="option">byte_binary_prefix</samp>, <samp class="option">sexagesimal</samp> etc.) are specified. The description of the accepted options follows. <dl class="table"> <dt><samp class="option">fully_qualified, q</samp></dt> <dd> If set to 1 specify if the output should be fully qualified. Default value is 0. This is required for generating an XML file which can be validated through an XSD file. </dd> <dt><samp class="option">xsd_strict, x</samp></dt> <dd> If set to 1 perform more checks for ensuring that the output is XSD compliant. Default value is 0. This option automatically sets <samp

class=”option”>fully_qualified</samp> to 1. </dd> </dl>

For more information about the XML format, see https://www.w3.org/XML/. <h2 class=”chapter”>5 Timecode# TOC</h2>

<code class=”command”>ffprobe</code> supports Timecode extraction: <ul class=”itemize mark-bullet”> MPEG1/2 timecode is extracted from the GOP, and is available in the video stream details (<samp class=”option”>-show_streams</samp>, see <var class=”var”>timecode</var>).

MOV timecode is extracted from tmcd track, so is available in the tmcd stream metadata (<samp class=”option”>-show_streams</samp>, see <var class=”var”>TAG:timecode</var>).

DV, GXF and AVI timecodes are available in format metadata (<samp class=”option”>-show_format</samp>, see <var class=”var”>TAG:timecode</var>).

 <h2 class=”chapter”>6 See Also# TOC</h2>

ffprobe-all, ffmpeg, ffplay, ffmpeg-utils, ffmpeg-scaler, ffmpeg-resampler, ffmpeg-codecs, ffmpeg-bitstream-filters, ffmpeg-formats, ffmpeg-devices, ffmpeg-protocols, ffmpeg-filters

 <h2 class=”chapter”>7 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class=”command”>git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg.

 Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\general.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> General Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> General Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 External libraries <ul class="toc-numbered-mark"> 1.1 Alliance for Open Media (AOM) 1.2 ARM ASTC Encoder 1.3 AMD AMF/VCE 1.4 AviSynth 1.5 Chromaprint 1.6 codec2 1.7 dav1d 1.8 davs2 1.9 uavs3d 1.10 Game Music Emu 1.11 Intel QuickSync Video 1.12 Kvazaar 1.13 LAME 1.14 LCEVCdec 1.15 libilbc 1.16 libjxl 1.17 libvpx 1.18 ModPlug 1.19 ONNX Runtime 1.20 OpenCORE, VisualOn, and Fraunhofer libraries <ul class="toc-

numbered-mark"> 1.20.1 OpenCORE AMR 1.20.2 VisualOn AMR-WB encoder library 1.20.3 Fraunhofer AAC library 1.20.4 Fraunhofer xHE-AAC library 1.20.5 Fraunhofer MPEG-H 3D Audio decoder library 1.20.6 LC3 library 1.21 OpenH264 1.22 OpenJPEG 1.23 rav1e 1.24 SVT-AV1 1.25 SVT-JPEG-XS 1.26 TwoLAME 1.27 VapourSynth 1.28 x264 1.29 x265 1.30 xavs 1.31 xavs2 1.32 eXtra-fast Essential Video Encoder (XEVE) 1.33 eXtra-fast Essential Video Decoder (XEVD) 1.34 ZVBI 2 Supported File Formats, Codecs or Features <ul class="toc-numbered-mark"> 2.1 File Formats 2.2 Image Formats 2.3 Video Codecs 2.4 Audio Codecs 2.5 Subtitle Formats 2.6 Network Protocols 2.7 Input/Output Devices 2.8 Timecode

 <h2 class="chapter">1 External libraries# <a class="anchor hidden-xs"href="#toc-

External-libraries" aria-hidden="true">TOC</h2>

FFmpeg can be hooked up with a number of external libraries to add support for more formats. None of them are used by default, their use has to be explicitly requested by passing the appropriate flags to <code class="command">./configure</code>. <h3 class="section">1.1 Alliance for Open Media (AOM)# TOC</h3>

FFmpeg can make use of the AOM library for AV1 decoding and encoding. Go to http://aomedia.org/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libaom</code> to configure to enable it. <h3 class="section">1.2 ARM ASTC Encoder# TOC</h3>

FFmpeg can make use of the Arm ASTC Encoder library for ASTC image compression and decompression in the <code class="code">libastcenc</code> encoder and decoder. Go to https://github.com/ARM-software/astc-encoder to obtain the sources. Version 5.4 or newer is required, because the wrappers use the four-argument <code class="code">astcenc_context_alloc()</code> interface introduced in that release. The upstream build does not install the public header, and the library file name depends on the configured SIMD and linkage variants, so install the pieces manually in a layout that configure can detect: the header as <code class="code">astcenc/astcenc.h</code> and a library that can be linked with <code class="code">-lastcenc</code>. Keep the library’s own file name, which is the SONAME recorded in the binary, and provide the plain name as an alias: <pre class="example-preformatted">cmake -S . -B build -DCMAKE_BUILD_TYPE=Release -DASTCENC_SHAREDLIB=ON cmake --build build install -Dm644 Source/astcenc.h /usr/local/include/astcenc/astcenc.h install -Dm755 build/Source/libastcenc-native-shared.so \ /usr/local/lib/libastcenc-native-shared.so ln -sf libastcenc-native-shared.so /usr/local/lib/libastcenc.so </pre>

Use the platform’s shared library suffix, and adjust the built library name for other variants (for example <samp class="file">libastcenc-neon-shared.so</samp> or a static <samp class="file">libastcenc.a</samp>), or pass <code class="code">--extra-cflags</code> and <code class="code">--extra-ldflags</code> to configure if the files are installed elsewhere. Installing only under the plain name breaks linking at run time, and the

installation directory has to be in the runtime loader’s search path. On big-endian systems, also pass <code class="code">-DASTCENC_ISA_NONE=ON</code> and <code class="code">-DASTCENC_BIG_ENDIAN=ON</code> to CMake. For this variant, install <samp class="file">libastcenc-none-shared.so</samp> under its own name and provide <samp class="file">libastcenc.so</samp> as an alias, as in the example above. The library is licensed under Apache-2.0, which requires version 3 of the (L)GPL, so pass <code class="code">--enable-version3</code> and <code class="code">--enable-libastcenc</code> to configure to enable it. <h3 class="section">1.3 AMD AMF/VCE# TOC</h3>

FFmpeg can use the AMD Advanced Media Framework library for accelerated H.264 and HEVC(only windows) encoding on hardware with Video Coding Engine (VCE). To enable support you must obtain the AMF framework header files(version 1.4.9+) from https://github.com/GPUOpen-LibrariesAndSDKs/AMF.git. Create an <code class="code">AMF/</code> directory in the system include path. Copy the contents of <code class="code">AMF/amf/public/include/</code> into that directory. Then configure FFmpeg with <code class="code">--enable-amf</code>. Initialization of amf encoder occurs in this order: 1) trying to initialize through dx11(only windows) 2) trying to initialize through dx9(only windows) 3) trying to initialize through vulkan To use h.264(AMD VCE) encoder on linux amdgru-pro version 19.20+ and amf-amdgpu-pro package(amdgru-pro contains, but does not install automatically) are required. This driver can be installed using amdgpu-pro-install script in official amd driver archive. <h3 class="section">1.4 AviSynth# TOC</h3>

FFmpeg can read AviSynth scripts as input. To enable support, pass <code class="code">--enable-avisynth</code> to configure after installing the headers provided by AviSynth+. AviSynth+ can be configured to install only the headers by either passing <code class="code">-DHEADERS_ONLY:bool=on</code> to the normal CMake-based build system, or by using the supplied <code class="code">GNUmakefile</code>. For Windows, supported AviSynth variants are AviSynth 2.6 RC1 or higher for 32-bit builds and Avi
r1718 or higher for 32-bit and 64-bit builds. For Linux, macOS, and BSD, the only supported AviSynth variant is <a class="url"

href=”https://github.com/AviSynth/AviSynthPlus”>AviSynth+, starting with version 3.5.   In 2016, AviSynth+ added support for building with GCC. However, due to the eccentricities of Windows’ calling conventions, 32-bit GCC builds of AviSynth+ are not compatible with typical 32-bit builds of FFmpeg. By default, FFmpeg assumes compatibility with 32-bit MSVC builds of AviSynth+ since that is the most widely-used and entrenched build configuration. Users can override this and enable support for 32-bit GCC builds of AviSynth+ by passing <code class=”code”>-DAVSC_WIN32_GCC32</code> to <code class=”code”>--extra-cflags</code> when configuring FFmpeg. 64-bit builds of FFmpeg are not affected, and can use either MSVC or GCC builds of AviSynth+ without any special flags.   AviSynth(+) is loaded dynamically. Distributors can build FFmpeg with <code class=”code”>--enable-avisynth</code>, and the binaries will work regardless of the end user having AviSynth installed. If/when an end user would like to use AviSynth scripts, then they can install AviSynth(+) and FFmpeg will be able to find and use it to open scripts. <h3 class=”section”>1.5 Chromaprint# TOC</h3>

FFmpeg can make use of the Chromaprint library for generating audio fingerprints. Pass <code class=”code”>--enable-chromaprint</code> to configure to enable it. See https://acoustid.org/chromapri
  <h3 class=”section”>1.6 codec2# TOC</h3>

FFmpeg can make use of the codec2 library for codec2 decoding and encoding. There is currently no native decoder, so libcodec2 must be used for decoding. Go to http://freedv.org/, download "Codec 2 source archive". Build and install using CMake. Debian users can install the libcodec2-dev package instead. Once libcodec2 is installed you can pass <code class=”code”>--enable-libcodec2</code> to configure to enable it. The easiest way to use codec2 is with .c2 files, since they contain the mode information required for decoding. To encode such a file, use a .c2 file extension and give the libcodec2 encoder the -mode option: <code class=”code”>ffmpeg -i input.wav -mode 700C output.c2</code>. Playback is as simple as <code class=”code”>ffplay output.c2</code>. For a list of supported modes, run <code class=”code”>ffmpeg -h encoder=libcodec2</code>. Raw codec2 files are also supported. To make sense of them the mode in use needs to be specified as a format option: <code class=”code”>ffmpeg -f codec2raw -mode 1300 -i input.raw output.wav</code>. <h3 class=”section”>1.7 dav1d<a class=”anchor hidden-xs” href=”#dav1d”

aria-hidden=”true”># TOC</h3>

FFmpeg can make use of the dav1d library for AV1 video decoding. Go to https://code.videolan.org/videol and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libdav1d</code> to configure to enable it. <h3 class=”section”>1.8 davs2# TOC</h3>

FFmpeg can make use of the davs2 library for AVS2-P2/IEEE1857.4 video decoding. Go to https://github.com/pku and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libdavs2</code> to configure to enable it.   libdavs2 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class=”section”>1.9 uavs3d# TOC</h3>

FFmpeg can make use of the uavs3d library for AVS3-P2/IEEE1857.10 video decoding. Go to https://github.co and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libuavs3d</code> to configure to enable it. <h3 class=”section”>1.10 Game Music Emu# TOC</h3>

FFmpeg can make use of the Game Music Emu library to read audio from supported video game music file formats. Pass <code class=”code”>--enable-libgme</code> to configure to enable it. See https://bitbucket.org/mpyne/game-music-emu/overview. <h3 class=”section”>1.11 Intel QuickSync Video# TOC</h3>

FFmpeg can use Intel QuickSync Video (QSV) for accelerated decoding and encoding of multiple codecs. To use QSV, FFmpeg must be linked against the <code class=”code”>libmfx</code> dispatcher, which loads the actual decoding libraries. The dispatcher is open source and can be downloaded from <a class=”url” href=”https://github.com/lu-

zero/mfx_dispatch.git”>https://github.com/lu-zero/mfx_dispatch.git. FFmpeg needs to be configured with the <code class=”code”>--enable-libmfx</code> option and <code class=”code”>pkg-config</code> needs to be able to locate the dispatcher’s <code class=”code”>.pc</code> files. <h3 class=”section”>1.12 Kvazaar# TOC</h3>

FFmpeg can make use of the Kvazaar library for HEVC encoding. Go to <a class=”url” href=”https://github.com/ultravideo/kvazaar”>https://github.com/ultravideo/kvaz and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libkvazaar</code> to configure to enable it. <h3 class=”section”>1.13 LAME# TOC</h3>

FFmpeg can make use of the LAME library for MP3 encoding. Go to http://lame.sourceforge.net/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libmp3lame</code> to configure to enable it. <h3 class=”section”>1.14 LCEVCdec# TOC</h3>

FFmpeg can make use of the liblcevc_dec library for LCEVC enhancement layer decoding on supported bitstreams. Go to https://github.com/v-novaltd/LCEVCdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblcevc-dec</code> to configure to enable it.   LCEVCdec is under the BSD-3-Clause-Clear License. <h3 class=”section”>1.15 libilbc# TOC</h3>

iLBC is a narrowband speech codec that has been made freely available by Google as part of the WebRTC project. libilbc is a packaging friendly copy of the iLBC codec. FFmpeg can make use of the libilbc library for iLBC decoding and encoding. Go to https://github.com/TimothyGu/libilbc and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libilbc</code> to configure to enable it. <h3 class=”section”>1.16 libjxl# TOC</h3>

JPEG XL is an image format intended to fully replace legacy JPEG for an extended period of life. See https://jpegxl.info/ for more information, and see <a class="url" href="https://github.com/libjxl/libjxl">https://github.com/libj for the library source. You can pass <code class="code">--enable-libjxl</code> to configure in order enable the libjxl wrapper. <h3 class="section">1.17 libvpx# TOC</h3>

FFmpeg can make use of the libvpx library for VP8/VP9 decoding and encoding. Go to http://www.webmproject.org and follow the instructions for installing the library. Then pass <code class="code">--enable-libvpx</code> to configure to enable it. <h3 class="section">1.18 ModPlug# TOC</h3>

FFmpeg can make use of this library, originating in Modplug-XMMS, to read from MOD-like music files. See https:/ Pass <code class="code">--enable-libmodplug</code> to configure to enable it. <h3 class="section">1.19 ONNX Runtime# TOC</h3>

FFmpeg can make use of the ONNX Runtime library as a backend for DNN based filters. Go to https://onnxruntime.ai/ and follow the instructions for installing the library. Pre-built packages are available for various platforms. Configure FFmpeg with ONNX Runtime support: <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib" </pre>

On ELF-based systems (Linux, BSD) you may also want to embed the library search path so that the installed <samp class="file">ffmpeg</samp> binary finds the shared library at run time without setting <code class="env">LD_LIBRARY_PATH</code>:   <pre class="example-preformatted">./configure \ --enable-libonnxruntime \ --extra-cflags="-I/path/to/onnxruntime/include" \ --extra-ldflags="-L/path/to/onnxruntime/lib -Wl,-rpath,/path/to/onnxruntime/lib" </pre>

The <code class="code">-Wl,-rpath</code> flag is ELF-specific and should be omitted on other platforms (Windows, macOS). For MinGW GCC builds, add the <code class="code">-D_stdcall=___stdcall</code> flag for compatibility with ONNX Runtime headers. <a name="OpenCORE_002c-

VisualOn_002c-and-Fraunhofer-libraries”> <h3 class=”section”>1.20 OpenCORE, VisualOn, and Fraunhofer libraries# TOC</h3>

Spun off Google Android sources, OpenCore, VisualOn and Fraunhofer libraries provide encoders for a number of audio codecs.   OpenCORE and VisualOn libraries are under the Apache License 2.0 (see http://www.apache.org/licenses/LICENSE-2.0 for details), which is incompatible to the LGPL version 2.1 and GPL version 2. You have to upgrade FFmpeg’s license to LGPL version 3 (or if you have enabled GPL components, GPL version 3) by passing <code class=”code”>--enable-version3</code> to configure in order to use it. The license of the Fraunhofer AAC library is incompatible with the GPL. Therefore, for GPL builds, you have to pass <code class=”code”>--enable-nonfree</code> to configure in order to use it. To the best of our knowledge, it is compatible with the LGPL. <h4 class=”subsection”>1.20.1 OpenCORE AMR# TOC</h4>

FFmpeg can make use of the OpenCORE libraries for AMR-NB decoding/encoding and AMR-WB decoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the libraries. Then pass <code class=”code”>--enable-libopencore-amrnb</code> and/or <code class=”code”>--enable-libopencore-amrwb</code> to configure to enable them. <h4 class=”subsection”>1.20.2 VisualOn AMR-WB encoder library# TOC</h4>

FFmpeg can make use of the VisualOn AMR-WBenc library for AMR-WB encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libvo-amrwbenc</code> to configure to enable it. <h4 class=”subsection”>1.20.3 Fraunhofer AAC library# <a class=”anchor hidden-xs”href=”#toc-Fraunhofer-AAC-library” aria-

hidden=”true”>TOC</h4>

FFmpeg can make use of the Fraunhofer AAC library for AAC decoding & encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-aac</code> to configure to enable it. <h4 class=”subsection”>1.20.4 Fraunhofer xHE-AAC library# TOC</h4>

FFmpeg can make use of the Fraunhofer xHE-AAC library for xHE-AAC encoding. Go to http://sourceforge.net/projects/opencore-amr/ and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libfdk-xhe-aac</code> to configure to enable it. <h4 class=”subsection”>1.20.5 Fraunhofer MPEG-H 3D Audio decoder library# TOC</h4>

FFmpeg can make use of the Fraunhofer MPEG-H decoder library for MPEG-H 3DA decoding. Go to https://github.com/Fraunhofer-IIS/mpeghdec and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libmpeghdec --enable-nonfree</code> to configure to enable it. <h4 class=”subsection”>1.20.6 LC3 library# TOC</h4>

FFmpeg can make use of the Google LC3 library for LC3 decoding & encoding. Go to <a class=”url” href=”https://github.com/google/liblc3/”>https://github.com/go
and follow the instructions for installing the library. Then pass <code class=”code”>--enable-liblc3</code> to configure to enable it. <h3 class=”section”>1.21 OpenH264# TOC</h3>

FFmpeg can make use of the OpenH264 library for H.264 decoding and encoding. Go to http://www.openh264.org/
and follow the instructions for installing the library. Then pass <code class=”code”>--enable-libopenh264</code> to configure to enable it.

For decoding, this library is much more limited than the built-in decoder in libavcodec; currently, this library lacks support for decoding B-frames and some other main/high profile features. (It currently only supports constrained baseline profile and CABAC.) Using it is mostly useful for testing and for taking advantage of Cisco’s patent portfolio license (http://www.openh264.org/BINARY_LICENSE.
  <h3 class="section">1.22 OpenJPEG# TOC</h3>

FFmpeg can use the OpenJPEG libraries for decoding/encoding J2K videos. Go to http://www.openjpeg.org/ to get the libraries and follow the installation instructions. To enable using OpenJPEG in FFmpeg, pass <code class="code">--enable-libopenjpeg</code> to <samp class="file">./configure</samp>. <h3 class="section">1.23 rav1e# TOC</h3>

FFmpeg can make use of rav1e (Rust AV1 Encoder) via its C bindings to encode videos. Go to https://github.com/xiph/rav1e/<
and follow the instructions to build the C library. To enable using rav1e in FFmpeg, pass <code class="code">--enable-librav1e</code> to <samp class="file">./configure</samp>. <h3 class="section">1.24 SVT-AV1# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for AV1 library for AV1 encoding. Go to https://gitlab.com/AOMediaCodec/SVT-AV1/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtav1</code> to configure to enable it. <h3 class="section">1.25 SVT-JPEG-XS# TOC</h3>

FFmpeg can make use of the Scalable Video Technology for JPEG-XS library for JPEG-XS decoding and encoding. Go to https://github.com/OpenVisualCloud/SVT-JPEG-XS and follow the instructions for installing the library. Then pass <code class="code">--enable-libsvtjpegxs</code> to configure to enable it. <h3 class="section">1.26 TwoLAME# <a class="anchor hidden-xs"href="#toc-TwoLAME"

aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the TwoLAME library for MP2 encoding.
Go to http://www.twolame.org/
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libtwolame</code> to configure to enable it. <a
name=”VapourSynth”> <h3 class=”section”>1.27 VapourSynth<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#VapourSynth”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-
VapourSynth” aria-hidden=”true”>TOC</h3>

FFmpeg can read VapourSynth scripts as input. To enable support, pass
<code class=”code”>--enable-vapoursynth</code> to configure. Vapoursynth
is detected via <code class=”code”>pkg-config</code>. Versions 42 or greater
supported. See http://www.vapoursynth.com/.
 Due to security concerns, Vapoursynth scripts will not be
autodetected so the input format has to be forced. For ff* CLI tools,
add <code class=”code”>-f vapoursynth</code> before the input <code
class=”code”>-i yourscript.vpy</code>.
<h3 class=”section”>1.28 x264<a class=”anchor
hidden-xs” href=”#x264” aria-hidden=”true”># <a class=”anchor
hidden-xs”href=”#toc-x264” aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the x264 library for H.264 encoding.
Go to http://www.videolan.org/d
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libx264</code> to configure to enable it.
 x264 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>1.29
x265<a class=”anchor hidden-xs” href=”#x265”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-x265”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the x265 library for HEVC encoding.
Go to http://x265.org/developers.html
and follow the instructions for installing the library. Then pass <code
class=”code”>--enable-libx265</code> to configure to enable it.
 x265 is under the GNU Public License Version 2 or
later (see <a class=”url” href=”http://www.gnu.org/licenses/old-licenses/gpl-
2.0.html”>http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for
details), you must upgrade FFmpeg’s license to GPL in order to
use it. <h3 class=”section”>1.30
xavs<a class=”anchor hidden-xs” href=”#xavs”
aria-hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-xavs”
aria-hidden=”true”>TOC</h3>

FFmpeg can make use of the xavs library for AVS encoding. Go to http://xavs.sf.net/ and follow the instructions for installing the library. Then pass <code class="code">--enable-libxavs</code> to configure to enable it. <h3 class="section">1.31 xavs2# TOC</h3>

FFmpeg can make use of the xavs2 library for AVS2-P2/IEEE1857.4 video encoding. Go to <a class="url" href="https://github.com/pkuvcl/xavs2">https://github.com/pku and follow the instructions for installing the library. Then pass <code class="code">--enable-libxavs2</code> to configure to enable it.   libxavs2 is under the GNU Public License Version 2 or later (see http://www.gnu.org/licenses/old-licenses/gpl-2.0.html for details), you must upgrade FFmpeg’s license to GPL in order to use it. <h3 class="section">1.32 eXtra-fast Essential Video Encoder (XEVE)# TOC</h3>

FFmpeg can make use of the XEVE library for EVC video encoding. Go to https://github.com/mpeg5/xeve and follow the instructions for installing the XEVE library. Then pass <code class="code">--enable-libxeve</code> to configure to enable it. <h3 class="section">1.33 eXtra-fast Essential Video Decoder (XEVD)# TOC</h3>

FFmpeg can make use of the XEVD library for EVC video decoding. Go to https://github.com/mpeg5/xevd and follow the instructions for installing the XEVD library. Then pass <code class="code">--enable-libxevd</code> to configure to enable it. <h3 class="section">1.34 ZVBI# TOC</h3>

ZVBI is a VBI decoding library which can be used by FFmpeg to decode DVB teletext pages and DVB teletext subtitles. Go to http://sourceforge.net/projects/zapping/ and follow the instructions for installing the library. Then pass <code

class=”code”>--enable-libzvbi</code> to configure to enable it. <h2 class=”chapter”>2 Supported File Formats, Codecs or Features# TOC</h2>

You can use the <code class=”code”>-formats</code> and <code class=”code”>-codecs</code> options to have an exhaustive list. <h3 class=”section”>2.1 File Formats# TOC</h3>

FFmpeg supports the following file formats through the <code class=”code”>libavformat</code>

library:

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>3dostr</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>4xm</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>4X Technologies format, used in some games.</td></tr>

<tr><td width=”40%”>8088flex TMV</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Enhanced Audio format, used in audiobooks.</td></tr>

<tr><td width=”40%”>AA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audible Format 2, 3, and 4, used in audiobooks.</td></tr>

<tr><td width=”40%”>ACT Voice</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>contains G.729 audio</td></tr>

<tr><td width=”40%”>Adobe Filmstrip</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Audio IFF (AIFF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>American Laser Games MM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Multimedia format used in games

like Mad Dog McCree.</td></tr>

<tr><td width="40%">3GPP AMR</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Amazing Studio Packed Animation File</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in game Heart Of Darkness.</td></tr>

<tr><td width="40%">Apple HTTP Live Streaming</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Artworx Data Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio only format used in some Interplay games.</td></tr>

<tr><td width="40%">ADP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">AFC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Gamecube.</td></tr>

<tr><td width="40%">ADS/SS2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS2.</td></tr>

<tr><td width="40%">APNG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">.astc</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Raw ASTC texture container.</td></tr>

<tr><td width="40%">ASF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Advanced / Active Streaming Format.</td></tr>

<tr><td width="40%">AST</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used on the Nintendo Wii.</td></tr>

<tr><td width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">AviSynth</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AVR</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on Mac.</td></tr>

<tr><td width="40%">AVS</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by the Creature Shock game.</td></tr>

<tr><td width=”40%”>Beam Software SIFF</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio and video format used in some games by Beam Software.</td></tr>

<tr><td width=”40%”>Bethesda Softworks VID</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in some games from Bethesda Softworks.</td></tr>

<tr><td width=”40%”>Binary text</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Bink</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Multimedia format used by many games.</td></tr>

<tr><td width=”40%”>Bink Audio</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio only multimedia format used by some games.</td></tr>

<tr><td width=”40%”>Bitmap Brothers JV</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Z and Z95 games.</td></tr>

<tr><td width=”40%”>BRP</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Argonaut Games format.</td></tr>

<tr><td width=”40%”>Brute Force & Ignorance</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width=”40%”>BFSTM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used on the Nintendo WiiU (based on BRSTM).</td></tr>

<tr><td width=”40%”>BRSTM</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used on the Nintendo Wii.</td></tr>

<tr><td width=”40%”>BW64</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Broadcast Wave 64bit.</td></tr>

<tr><td width=”40%”>BWF</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>codec2 (raw)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Must be given -mode format option to decode correctly.</td></tr>

<tr><td width=”40%”>codec2 (.c2 files)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Contains header with version and mode info, simplifying playback.</td></tr>

<tr><td width=”40%”>CRI ADX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Audio-only format used in console video games.</td></tr>

<tr><td width="40%">CRI AIX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio-only format used in console video games.</td></tr>

<tr><td width="40%">Discworld II BMV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Delphine Software International CIN</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by Delphine Software games.</td></tr>

<tr><td width="40%">Digital Speech Standard (DSS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video format used by CD+G karaoke disks</td></tr>

<tr><td width="40%">Phantom Cine</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Commodore CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video format</td></tr>

<tr><td width="40%">Core Audio Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Apple Core Audio Format</td></tr>

<tr><td width="40%">CRC testing format</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">Creative Voice</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Created for the Sound Blaster Pro.</td></tr>

<tr><td width="40%">CRYO APC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in some games by CRYO Interactive Entertainment.</td></tr>

<tr><td width="40%">D-Cinema audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DCSTR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width=”40%”>DFA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in Chronomaster game</td></tr>

<tr><td width=”40%”>DirectDraw Surface</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DSD Stream File (DSF)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DV video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>DXA</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>This format is used in the non-Windows version of the Feeble Files

game and different game cutscenes repacked for use with ScummVM.</td></tr>

<tr><td width=”40%”>Electronic Arts cdata</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Electronic Arts Multimedia</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA games; files have extensions like WVE and UV2.</td></tr>

<tr><td width=”40%”>Ensoniq Paris Audio File</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>FFM (FFserver live feed)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash (SWF)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Flash 9 (AVM2)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Only embedded audio is decoded.</td></tr>

<tr><td width=”40%”>FLI/FLC/FLX animation</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>.fli/.flc files</td></tr>

<tr><td width=”40%”>Flash Video (FLV)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Macromedia Flash video files</td></tr>

<tr><td width=”40%”>framecrc testing format</td><td width=”10%”>X</td><td width=”10%”></td></tr>

<tr><td width=”40%”>FunCom ISS</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Audio format used in various games from FunCom like The Longest Journey.</td></tr>

<tr><td width=”40%”>G.723.1</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width="40%">G.726</td><td width="10%"></td><td width="10%">X</td><td width="40%">Both left- and right-justified.</td></tr>

<tr><td width="40%">G.729 BIT</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.729 raw</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GENH</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format for various games.</td></tr>

<tr><td width="40%">GIF Animation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">GXF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">General eXchange Format SMPTE 360M, used by Thomson Grass Valley

playout servers.</td></tr>

<tr><td width="40%">HNM</td><td width="10%"></td><td width="10%">X</td><td width="40%">Only version 4 supported, used in some games from Cryo Interactive</td></tr>

<tr><td width="40%">iCEDraw File</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ICO</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft Windows ICO</td></tr>

<tr><td width="40%">id Quake II CIN video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">IEC61937 encapsulation</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IFF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Interchange File Format</td></tr>

<tr><td width="40%">IFV</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format used by some old CCTV DVRs.</td></tr>

<tr><td width="40%">iLBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay MVE</td><td width="10%"></td><td width="10%">X</td><td width="40%">Format used in various Interplay computer games.</td></tr>

<tr><td width="40%">Iterated Systems ClearVideo</td><td width="10%"></td><td width="10%">X</td><td width="40%">I-frames only</td></tr>

<tr><td width="40%">IV8</td><td width="10%"></td><td width="10%">X</td><td width="40%">A format generated by IndigoVision 8000 video server.</td></tr>

<tr><td width="40%">IVF (On2)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">A format used by libvpx</td></tr>

<tr><td width="40%">Internet Video Recording</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">IRCAM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">JPEG Multi-Picture Format</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Includes JPEG Ultra HDR (ISO 21496-1 gain map) images.</td></tr>

<tr><td width="40%">LAF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Limitless Audio Format</td></tr>

<tr><td width="40%">LATM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LMLM4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Linux Media Labs MPEG-4 PCI boards</td></tr>

<tr><td width="40%">LOAS</td><td width="10%"></td><td width="10%">X</td><td width="40%">contains LATM multiplexed AAC audio</td></tr>

<tr><td width="40%">LRC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">LVF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LXF</td><td width="10%"></td><td width="10%">X</td><td width="40%">VR native stream format, used by Leitch/Harris’ video servers.</td></tr>

<tr><td width="40%">Magic Lantern Video (MLV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Matroska audio</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">FFmpeg metadata</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Metadata in text format.</td></tr>

<tr><td width="40%">MAXIS XA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sim City 3000; file extension .xa.</td></tr>

<tr><td width="40%">MCA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Capcom; file extension .mca.</td></tr>

<tr><td width="40%">MD Studio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Metal Gear Solid: The Twin Snakes</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Megalux Frame</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by Megalux Ultimate Paint</td></tr>

<tr><td width="40%">MobiClip MODS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MobiClip MOFLEX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mobotix .mxg</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Monkey’s Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Motion Pixels MVI</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MOV/QuickTime/MP4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">3GP, 3GP2, PSP, iPod variants supported</td></tr>

<tr><td width="40%">MP2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MP3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-1 System</td><td width="10%">X</td><td width="10%">X</td><td width="40%">muxed audio and video, VCD format supported</td></tr>

<tr><td width="40%">MPEG-PS (program stream)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">also known as <code class="code">VOB</code> file, SVCD and DVD format supported</td></tr>

<tr><td width="40%">MPEG-TS (transport stream)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">also known as DVB Transport Stream</td></tr>

<tr><td width="40%">MPEG-4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">MPEG-4 is a variant of QuickTime.</td></tr>

<tr><td width="40%">MSF</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">Mirillis FIC video</td><td width="10%"></td><td width="10%">X</td><td width="40%">No cursor rendering.</td></tr>

<tr><td width="40%">MIDI Sample Dump Standard</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MIME multipart JPEG</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">MSN TCP webcam</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by MSN Messenger webcam streams.</td></tr>

<tr><td width="40%">MTV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Material eXchange Format (MXF)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 377M, used by D-Cinema, broadcast industry.</td></tr>

<tr><td width="40%">Material eXchange Format (MXF), D-10 Mapping</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SMPTE 386M, D-10/IMX Mapping.</td></tr>

<tr><td width="40%">NC camera feed</td><td width="10%"></td><td width="10%">X</td><td width="40%">NC (AVIP NC4600) camera streams</td></tr>

<tr><td width="40%">NIST SPeech HEader REsources</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Computerized Speech Lab NSP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NTT TwinVQ (VQF)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Nippon Telegraph and Telephone Corporation TwinVQ.</td></tr>

<tr><td width="40%">Nullsoft Streaming Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NuppelVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">NUT</td><td width="10%">X</td><td width="10%">X</td><td width="40%">NUT Open Container Format</td></tr>

<tr><td width="40%">Ogg</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Playstation Portable PMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Portable Voice Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">TechnoTrend PVA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used by TechnoTrend DVB PCI boards.</td></tr>

<tr><td width="40%">QCP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw ADTS (AAC)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-NB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw AMR-WB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APAC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw APV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw aptX HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Bonk</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw Chinese AVS video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Dirac</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DNxHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw DTS-HD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw E-AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw EVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw FLAC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw GSM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.263</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw H.264</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw HEVC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Ingenient MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw MLP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw MPEG-4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw NULL</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw id RoQ</td><td width="10%">X</td><td width="10%"></td></tr>

<tr><td width="40%">raw OBU</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw OSQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw SBC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TAK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw TrueHD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw VC-1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM signed 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 8 bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 16 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 24 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM unsigned 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 16.8 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM 24.0 floating point little-endian</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 32 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">raw PCM floating-point 64 bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RDT</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">REDCODE R3D</td><td width="10%"></td><td width="10%">X</td><td width="40%">File format used by RED Digital cameras, contains JPEG 2000 frames and PCM audio.</td></tr>

<tr><td width="40%">RealMedia</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Redirector</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RedSpark</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TeXture Dictionary</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Resolume DXV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoding is only supported for the DXT1 (Normal Quality, No Alpha) texture format.</td></tr>

<tr><td width="40%">RF64</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RL2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio and video format used in some games by Entertainment Software Partners.</td></tr>

<tr><td width="40%">RPL/ARMovie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Lego Mindstorms RSO</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RSD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RTMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Output is performed by publishing stream to RTMP server</td></tr>

<tr><td width="40%">RTP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">RTSP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sample Dump eXchange</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SAP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SBG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDNS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SDP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SER</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Pictures SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sega FILM/CPK</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in many Sega Saturn console games.</td></tr>

<tr><td width="40%">Silicon Graphics Movie</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra SOL</td><td width="10%"></td><td width="10%">X</td><td width="40%">.sol files used in Sierra Online games.</td></tr>

<tr><td width="40%">Sierra VMD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra CD-ROM games.</td></tr>

<tr><td width="40%">Smacker</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used by many games.</td></tr>

<tr><td width="40%">SMJPEG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in certain Loki game ports.</td></tr>

<tr><td width="40%">SMPTE 337M encapsulation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in some LucasArts games.</td></tr>

<tr><td width="40%">Sony OpenMG (OMA)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Audio format used in Sony Sonic Stage and Sony Vegas.</td></tr>

<tr><td width="40%">Sony PlayStation STR</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sony Wave64 (W64)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SoX native format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUN AU format</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SUP raw PGS subtitles</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">SVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Konami PS2 games.</td></tr>

<tr><td width="40%">TDSC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Text files</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">THP</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used on the Nintendo GameCube.</td></tr>

<tr><td width="40%">Tiertex Limited SEQ</td><td width="10%"></td><td width="10%">X</td><td width="40%">Tiertex .seq files used in the DOS CD-ROM version of the game Flashback.</td></tr>

<tr><td width="40%">True Audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in many Sony PS2 games.</td></tr>

<tr><td width="40%">VC-1 test bitstream</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vidvox Hap</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Vivo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VPK</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used in Sony PS games.</td></tr>

<tr><td width="40%">Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Waveform Archiver</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WavPack</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">WebM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Televison (WTV)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Wing Commander III movie</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Origin’s Wing Commander III computer game.</td></tr>

<tr><td width="40%">Westwood Studios audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Westwood Studios VQA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Multimedia format used in Westwood Studios games.</td></tr>

<tr><td width="40%">Wideband Single-bit Data (WSD)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WVE</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">KTX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Khronos Texture 1.0 container for ASTC.</td></tr>

<tr><td width="40%">Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">XMV</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft video container used in Xbox games.</td></tr>

<tr><td width="40%">XVAG</td><td width="10%"></td><td width="10%">X</td><td width="40%">Audio format used on the PS3.</td></tr>

<tr><td width="40%">xWMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Microsoft audio container used by XAudio 2.</td></tr>

<tr><td width="40%">eXtended BINary text (XBIN)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">YUV4MPEG pipe</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP</td><td width="10%"></td><td width="10%">X</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <h3 class=”section”>2.2 Image Formats# TOC</h3>

FFmpeg can read and write images for each frame of a video sequence. The following image formats are supported: <table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr> <tr><td width=”40%”>.Y.U.V</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>one raw file per component</td></tr> <tr><td width=”40%”>Alias PIX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Alias/Wavefront PIX image format</td></tr> <tr><td width=”40%”>animated GIF</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>APNG</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Animated Portable Network Graphics</td></tr> <tr><td width=”40%”>ASTC</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Adaptive Scalable Texture Compression, through external library libastcenc</td></tr> <tr><td width=”40%”>BMP</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Microsoft BMP image</td></tr> <tr><td width=”40%”>BRender PIX</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Argonaut BRender 3D engine image format.</td></tr> <tr><td width=”40%”>CRI</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Cintel RAW</td></tr> <tr><td width=”40%”>DPX</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Digital Picture Exchange</td></tr> <tr><td width=”40%”>EXR</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>OpenEXR</td></tr> <tr><td width=”40%”>FITS</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Flexible Image Transport System</td></tr> <tr><td width=”40%”>HDR</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Radiance HDR RGBE Image format</td></tr> <tr><td width=”40%”>IMG</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>GEM Raster image</td></tr> <tr><td width=”40%”>JPEG</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>Progressive JPEG is not supported.</td></tr> <tr><td width=”40%”>JPEG 2000</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JPEG-LS</td><td width=”10%”>X</td><td width=”10%”>X</td></tr> <tr><td width=”40%”>JPEG-XS</td><td width=”10%”>E</td><td width=”10%”>E</td><td width=”40%”>Supported through external library libsvtjpegxs</td></tr> <tr><td width=”40%”>LJPEG</td><td width=”10%”>X</td><td width=”10%”></td><td width=”40%”>Lossless JPEG</td></tr> <tr><td width=”40%”>Media 100</td><td width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>MSP</td><td

width="10%"></td><td width="10%">X</td><td width="40%">Microsoft Paint image</td></tr> <tr><td width="40%">PAM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PAM is a PNM extension with alpha support.</td></tr> <tr><td width="40%">PBM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable BitMap image</td></tr> <tr><td width="40%">PCD</td><td width="10%"></td><td width="10%">X</td><td width="40%">PhotoCD</td></tr> <tr><td width="40%">PCX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PC Paintbrush</td></tr> <tr><td width="40%">PFM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable FloatMap image</td></tr> <tr><td width="40%">PGM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable GrayMap image</td></tr> <tr><td width="40%">PGMYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">PGM with U and V components in YUV 4:2:0</td></tr> <tr><td width="40%">PGX</td><td width="10%"></td><td width="10%">X</td><td width="40%">PGX file decoder</td></tr> <tr><td width="40%">PHM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable HalfFloatMap image</td></tr> <tr><td width="40%">PIC</td><td width="10%"></td><td width="10%">X</td><td width="40%">Pictor/PC Paint</td></tr> <tr><td width="40%">PNG</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable Network Graphics image</td></tr> <tr><td width="40%">PPM</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Portable PixelMap image</td></tr> <tr><td width="40%">PSD</td><td width="10%"></td><td width="10%">X</td><td width="40%">Photoshop</td></tr> <tr><td width="40%">PTX</td><td width="10%"></td><td width="10%">X</td><td width="40%">V.Flash PTX format</td></tr> <tr><td width="40%">QOI</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Quite OK Image format</td></tr> <tr><td width="40%">RW2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Panasonic raw image, RawFormat 8 only</td></tr> <tr><td width="40%">SGI</td><td width="10%">X</td><td width="10%">X</td><td width="40%">SGI RGB image format</td></tr> <tr><td width="40%">Sun Rasterfile</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sun RAS image format</td></tr> <tr><td width="40%">TIFF</td><td width="10%">X</td><td width="10%">X</td><td width="40%">YUV, JPEG and some extension is not supported yet.</td></tr> <tr><td width="40%">Truevision Targa</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Targa (.TGA) image format</td></tr> <tr><td width="40%">VBN</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Vizrt Binary Image format</td></tr> <tr><td width="40%">WBMP</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Wireless Application Protocol Bitmap image format</td></tr> <tr><td width="40%">WebP</td><td width="10%">E</td><td width="10%">X</td><td width="40%">WebP

image format, encoding supported through external library libwebp</td></tr>
<tr><td width="40%">XBM</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X BitMap image format</td></tr>
<tr><td width="40%">XFace</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">X-Face image format</td></tr>
<tr><td width="40%">XPM</td><td width="10%"></td><td width="10%">X</td><td
width="40%">X PixMap image format</td></tr> <tr><td width="40%">XWD</td><td
width="10%">X</td><td width="10%">X</td><td width="40%">X
Window Dump image format</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code>
means that support is provided through an external library. <a
name="Video-Codecs"> <h3 class="section">2.3 Video Codecs<span
class="pull-right"><a class="anchor hidden-xs" href="#Video-Codecs"
aria-hidden="true"># <a class="anchor hidden-xs"href="#toc-Video-
Codecs" aria-hidden="true">TOC</h3>

<table class="multitable">

<tbody><tr><td width="40%">Name</td><td width="10%">Encoding</td><td
width="10%">Decoding</td><td width="40%">Comments</td></tr>

<tr><td width="40%">4X Movie</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in certain computer
games.</td></tr>

<tr><td width="40%">8088flex TMV</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">A64 multicolor</td><td width="10%">X</td><td
width="10%"></td><td width="40%">Creates video suitable to be played
on a commodore 64 (multicolor mode).</td></tr>

<tr><td width="40%">Amazing Studio PAF Video</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">American Laser Games MM</td><td width="10%"></td><td
width="10%">X</td><td width="40%">Used in games like Mad Dog Mc-
Cree.</td></tr>

<tr><td width="40%">Amuse Graphics Movie</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">AMV Video</td><td width="10%">X</td><td
width="10%">X</td><td width="40%">Used in Chinese MP3 play-
ers.</td></tr>

<tr><td width="40%">ANSI/ASCII art</td><td width="10%"></td><td
width="10%">X</td></tr>

<tr><td width="40%">Apple Intermediate Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple MJPEG-B</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple Pixlet</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple ProRes</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: apch,apcn,apcs,apco,ap4h,ap4x</td></tr>

<tr><td width="40%">Apple QuickDraw</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: qdrw</td></tr>

<tr><td width="40%">APV</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Argonaut Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Argonaut games.</td></tr>

<tr><td width="40%">Asus v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ASV1</td></tr>

<tr><td width="40%">Asus v2</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: ASV2</td></tr>

<tr><td width="40%">ATI VCR1</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR1</td></tr>

<tr><td width="40%">ATI VCR2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: VCR2</td></tr>

<tr><td width="40%">Auravision Aura</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Auravision Aura 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk Animator Flic video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Autodesk RLE</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: AASC</td></tr>

<tr><td width="40%">AV1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libaom, libdav1d, librav1e and libsvtav1</td></tr>

<tr><td width="40%">Avid 1:1 10-bit RGB Packer</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: AVrp</td></tr>

<tr><td width="40%">AVS (Audio Video Standard) video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used by the Creature Shock game.</td></tr>

<tr><td width="40%">AVS2-P2/IEEE1857.4</td><td width="10%">E</td><td width="10%">E</td><td width="40%">Supported through external libraries libxavs2 and libdavs2</td></tr>

<tr><td width="40%">AVS3-P2/IEEE1857.10</td><td width="10%"></td><td width="10%">E</td><td width="40%">Supported through external library libuavs3d</td></tr>

<tr><td width="40%">AYUV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Microsoft uncompressed packed 4:4:4:4</td></tr>

<tr><td width="40%">Beam Software VB</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bethesda VID video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some games from Bethesda Softworks.</td></tr>

<tr><td width="40%">Bink Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">BitJazz SheerVideo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bitmap Brothers JV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">y41p Brooktree uncompressed 4:1:1 12-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Brooktree ProSumer Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: BT20</td></tr>

<tr><td width="40%">Brute Force & Ignorance</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Flash Traffic: City of Angels.</td></tr>

<tr><td width="40%">C93 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Cyberia game.</td></tr>

<tr><td width="40%">CamStudio</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: CSCD</td></tr>

<tr><td width="40%">CD+G</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video codec for CD+G karaoke disks</td></tr>

<tr><td width="40%">CDXL</td><td width="10%"></td><td width="10%">X</td><td width="40%">Amiga CD video codec</td></tr>

<tr><td width="40%">Chinese AVS video</td><td width="10%">E</td><td width="10%">X</td><td width="40%">AVS1-P2, JiZhun profile, encoding

through external library libxavs</td></tr>

<tr><td width="40%">Delphine Software International CIN video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">Discworld II BMV Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CineForm HD</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQ</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus HQX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Canopus Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">CDToons</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in various Broderbund games.</td></tr>

<tr><td width="40%">Cinepak</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Cirrus Logic AccuPak</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: CLJR</td></tr>

<tr><td width="40%">CPiA Video Format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Creative YUV (CYUV)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DFA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Chronomaster game.</td></tr>

<tr><td width="40%">Dirac</td><td width="10%">E</td><td width="10%">X</td><td width="40%">supported though the native vc2 (Dirac Pro) encoder</td></tr>

<tr><td width="40%">Deluxe Paint Animation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DNxHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">aka SMPTE VC3</td></tr>

<tr><td width="40%">Duck TrueMotion 1.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: DUCK</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TM20</td></tr>

<tr><td width="40%">Duck TrueMotion 2.0 RT</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TR20</td></tr>

<tr><td width="40%">DV (Digital Video)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Dxtory capture format</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Feeble Files/ScummVM DXA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec originally used in Feeble Files game.</td></tr>

<tr><td width="40%">Electronic Arts CMV video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in NHL 95 game.</td></tr>

<tr><td width="40%">Electronic Arts Madcow video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGV video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TGQ video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Electronic Arts TQI video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 124</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Escape 130</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">EVC / MPEG-5 Part 1</td><td width="10%">E</td><td width="10%">E</td><td width="40%">encoding and decoding supported through external libraries libxeve and libxevd</td></tr>

<tr><td width="40%">FFmpeg video codec #1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">lossless codec (fourcc: FFV1)</td></tr>

<tr><td width="40%">Flash Screen Video v1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: FSV1</td></tr>

<tr><td width="40%">Flash Screen Video v2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Flash Video (FLV)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Sorenson H.263 used in Flash</td></tr>

<tr><td width="40%">FM Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Forward Uncompressed</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Fraps</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Go2Meeting</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M2, G2M3</td></tr>

<tr><td width="40%">Go2Webinar</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: G2M4</td></tr>

<tr><td width="40%">Gremlin Digital Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">H.261</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263 / H.263-1996</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.263+ / H.263-1998 / H.263 version 2</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">H.264 / AVC / MPEG-4 AVC / MPEG-4 part 10</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libx264 and OpenH264,

decoding supports data partitioning</td></tr>

<tr><td width="40%">HEVC</td><td width="10%">X</td><td width="10%">X</td><td width="40%">encoding supported through external library libx265 and libkvazaar</td></tr>

<tr><td width="40%">HNM version 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">HuffYUV FFmpeg variant</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">IBM Ultimotion</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ULTI</td></tr>

<tr><td width="40%">id Cinematic video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Quake II.</td></tr>

<tr><td width="40%">id RoQ video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2, other computer games.</td></tr>

<tr><td width="40%">IFF ILBM</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF interleaved bitmap</td></tr>

<tr><td width="40%">IFF ByteRun1</td><td width="10%"></td><td width="10%">X</td><td width="40%">IFF run length encoded bitmap</td></tr>

<tr><td width="40%">Infinity IMM4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Infinity IMM6</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel H.263</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 4</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Intel Indeo 5</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay C93</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in the game Cyberia from Interplay.</td></tr>

<tr><td width="40%">Interplay MVE video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Interplay .MVE files.</td></tr>

<tr><td width="40%">J2K</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Karl Morton’s video codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Worms games.</td></tr>

<tr><td width="40%">Kega Game Video (KGV1)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Kega emulator screen capture codec.</td></tr>

<tr><td width="40%">Lagarith</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCEVC / MPEG-5 LCEVC / MPEG-5 Part 2</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library liblcevc-dec</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) MSZH</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LCL (LossLess Codec Library) ZLIB</td><td width="10%">E</td><td width="10%">E</td></tr>

<tr><td width="40%">LEAD MCMP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LOCO</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LucasArts SANM/Smush</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts games / SMUSH animations.</td></tr>

<tr><td width="40%">lossless MJPEG</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">MagicYUV Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Mandsoft Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft ATC Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Screen 3.</td></tr>

<tr><td width="40%">Microsoft Expression Encoder Screen</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Microsoft Titanium Screen 2.</td></tr>

<tr><td width="40%">Microsoft RLE</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Microsoft Screen 1</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V7 Screen.</td></tr>

<tr><td width="40%">Microsoft Screen 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">Also known as Windows Media Video V9 Screen.</td></tr>

<tr><td width="40%">Microsoft Video 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Mimic</td><td width="10%"></td><td

width=”10%”>X</td><td width=”40%”>Used in MSN Messenger Webcam streams.</td></tr>

<tr><td width=”40%”>Miro VideoXL</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VIXL</td></tr>

<tr><td width=”40%”>MJPEG (Motion JPEG)</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Mobotix MxPEG video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Motion Pixels video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-1 video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-2 video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>libxvidcore can be used alternatively for encoding.</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>MPEG-4 part 2 Microsoft variant version 3</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Newtek SpeedHQ</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Nintendo Gamecube THP video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>NotchLC</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>NuppelVideo/RTjpeg</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Video encoding used in NuppelVideo files.</td></tr>

<tr><td width=”40%”>On2 VP3</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>still experimental</td></tr>

<tr><td width=”40%”>On2 VP4</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP40</td></tr>

<tr><td width=”40%”>On2 VP5</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP50</td></tr>

<tr><td width=”40%”>On2 VP6</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP60,VP61,VP62</td></tr>

<tr><td width=”40%”>On2 VP7</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: VP70,VP71</td></tr>

<tr><td width=”40%”>VP8</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>fourcc: VP80, encoding supported through external library libvpx</td></tr>

<tr><td width=”40%”>VP9</td><td width=”10%”>E</td><td width=”10%”>X</td><td width=”40%”>encoding supported through external library libvpx</td></tr>

<tr><td width=”40%”>Pinnacle TARGA CineWave YUV16</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourcc: Y216</td></tr>

<tr><td width=”40%”>Q-team QPEG</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>fourccs: QPEG, Q1.0, Q1.1</td></tr>

<tr><td width=”40%”>QuickTime 8BPS video</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>QuickTime Animation (RLE) video</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ’rle ’</td></tr>

<tr><td width=”40%”>QuickTime Graphics (SMC)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: ’smc ’</td></tr>

<tr><td width=”40%”>QuickTime video (RPZA)</td><td width=”10%”>X</td><td width=”10%”>X</td><td width=”40%”>fourcc: rpza</td></tr>

<tr><td width=”40%”>R10K AJA Kona 10-bit RGB Codec</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>R210 Quicktime Uncompressed RGB 10-bit</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Raw Video</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 1.0</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 2.0</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>RealVideo 3.0</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>still far from ideal</td></tr>

<tr><td width="40%">RealVideo 4.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealVideo 6.0</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Renderware TXD (TeXture Dictionary)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Texture dictionaries used by the Renderware Engine.</td></tr>

<tr><td width="40%">RivaTuner Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’RTV1’</td></tr>

<tr><td width="40%">RL2 video</td><td width="10%"></td><td width="10%">X</td><td width="40%">used in some games by Entertainment Software Partners</td></tr>

<tr><td width="40%">ScreenPressor</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screenpresso</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Screen Recorder Gold Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 1 (MVC1)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics Motion Video Compressor 2 (MVC2)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Silicon Graphics RLE 8-bit video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Smacker video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Video encoding used in Smacker.</td></tr>

<tr><td width="40%">SMPTE VC-1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Snow</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental wavelet codec (fourcc: SNOW)</td></tr>

<tr><td width="40%">Sony PlayStation MDEC (Motion DECoder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 1</td><td width="10%">X</td><td width="10%">X</td><td width="40%">fourcc: SVQ1</td></tr>

<tr><td width="40%">Sorenson Vector Quantizer 3</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SVQ3</td></tr>

<tr><td width="40%">Sunplus JPEG (SP5X)</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: SP5X</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSCC</td></tr>

<tr><td width="40%">TechSmith Screen Capture Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: TSC2</td></tr>

<tr><td width="40%">Theora</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libtheora</td></tr>

<tr><td width="40%">Tiertex Limited SEQ video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in DOS CD-ROM FlashBack game.</td></tr>

<tr><td width="40%">Ut Video</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v210 QuickTime uncompressed 4:2:2 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v308 QuickTime uncompressed 4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v408 QuickTime uncompressed 4:4:4:4</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">v410 QuickTime uncompressed 4:4:4 10-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">VBLE Lossless Codec</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">vMix Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">fourcc: ’VMX1’</td></tr>

<tr><td width="40%">VMware Screen Codec / VMware Video</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in videos captured by VMware.</td></tr>

<tr><td width="40%">Westwood Studios VQA (Vector Quantized Animation) video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Image</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 8</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Windows Media Video 9</td><td width="10%"></td><td width="10%">X</td><td width="40%">not completely working</td></tr>

<tr><td width="40%">Wing Commander III / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander III .MVE files.</td></tr>

<tr><td width="40%">Wing Commander IV / Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Wing Commander IV.</td></tr>

<tr><td width="40%">Winnov WNV1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">WMV7</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">YAMAHA SMAF</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Psygnosis YOP Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">yuv4</td><td width="10%">X</td><td width="10%">X</td><td width="40%">libquicktime uncompressed packed 4:2:0</td></tr>

<tr><td width="40%">ZeroCodec Lossless Video</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ZLIB</td><td width="10%">X</td><td width="10%">X</td><td width="40%">part of LCL, encoder experimental</td></tr>

<tr><td width="40%">Zip Motion Blocks Video</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Encoder works only in PAL8.</td></tr>

</tbody>

</table>

<code class="code">X</code> means that the feature in that column (encoding / decoding) is supported. <code class="code">E</code> means that support is provided through an external library. <h3 class="section">2.4 Audio Codecs<a class="anchor hidden-xs" href="#Audio-Codecs"

aria-hidden=”true”># TOC</h3>

<table class=”multitable”>

<tbody><tr><td width=”40%”>Name</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td><td width=”40%”>Comments</td></tr>

<tr><td width=”40%”>8SVX exponential</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>8SVX fibonacci</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>AAC</td><td width=”10%”>EX</td><td width=”10%”>X</td><td width=”40%”>encoding supported through internal encoder and external library libfdk-aac</td></tr>

<tr><td width=”40%”>AAC+</td><td width=”10%”>E</td><td width=”10%”>IX</td><td width=”40%”>encoding supported through external library libfdk-aac</td></tr>

<tr><td width=”40%”>AC-3</td><td width=”10%”>IX</td><td width=”10%”>IX</td></tr>

<tr><td width=”40%”>ACELP.KELVIN</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM 4X Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Yamaha AICA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM AmuseGraphics Movie</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Argonaut Games</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM CDROM XA</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>ADPCM Creative Technology</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>16 -> 4, 8 -> 4, 8 -> 3, 8 -> 2</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in various EA titles.</td></tr>

<tr><td width=”40%”>ADPCM Electronic Arts Maxis CDROM XS</td><td width=”10%”></td><td width=”10%”>X</td><td width=”40%”>Used in Sim City 3000.</td></tr>

<tr><td width="40%">ADPCM Electronic Arts R1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Electronic Arts R2</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Electronic Arts R3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Electronic Arts XAS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM G.722</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM G.726</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Acorn Replay</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA AMV</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in AMV files</td></tr>

<tr><td width="40%">ADPCM IMA Citrix</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Cunning Developments</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Electronic Arts EACS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Electronic Arts SEAD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Funcom</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA High Voltage Software ALP</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Mobiclip MOFLEX</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA QuickTime</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Simon & Schuster Interactive</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Ubisoft APM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Loki SDL MJPEG</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA WAV</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Westwood</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM ISS IMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in FunCom games.</td></tr>

<tr><td width="40%">ADPCM IMA Dialogic</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK3</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Duck DK4</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in some Sega Saturn console games.</td></tr>

<tr><td width="40%">ADPCM IMA Radical</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM IMA Xbox</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Microsoft</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM MS IMA</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube AFC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo Gamecube DTK</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Nintendo THP</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Playstation</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM QT IMA</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Rhetorex</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sanyo</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM SEGA CRI ADX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Sega Dreamcast games.</td></tr>

<tr><td width="40%">ADPCM Shockwave Flash</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 2-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 2.6-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Sound Blaster Pro 4-bit</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ADPCM Konami XMD</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Westwood Studios IMA</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Westwood Studios games like Command and Conquer.</td></tr>

<tr><td width="40%">ADPCM Yamaha</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">ADPCM Zork</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">AMR-NB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopencore-amrnb</td></tr>

<tr><td width="40%">AMR-WB</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libvo-amrwbenc</td></tr>

<tr><td width="40%">Amazing Studio PAF Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Apple lossless audio</td><td width="10%">X</td><td width="10%">X</td><td width="40%">QuickTime fourcc ’alac’</td></tr>

<tr><td width="40%">aptX</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">aptX HD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">ATRAC1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC3+</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">ATRAC9</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Bink Audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Bink and Smacker files in many games.</td></tr>

<tr><td width="40%">Bonk audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">codec2</td><td width="10%">E</td><td width="10%">E</td><td width="40%">en/decoding supported through external library libcodec2</td></tr>

<tr><td width="40%">CRI HCA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Delphine Software International CIN audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Codec used in Delphine Software International games.</td></tr>

<tr><td width="40%">DFPWM</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Digital Speech Standard - Standard Play mode (DSS SP)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Discworld II BMV Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">COOK</td><td width="10%"></td><td width="10%">X</td><td width="40%">All versions except 5.1 are supported.</td></tr>

<tr><td width="40%">DCA (DTS Coherent Acoustics)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">supported extensions: XCh, XXCH, X96, XBR, XLL, LBR (partially)</td></tr>

<tr><td width="40%">Dolby E</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Cuberoot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in few games.</td></tr>

<tr><td width="40%">DPCM Gremlin</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM id RoQ</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Quake III, Jedi Knight 2 and other computer games.</td></tr>

<tr><td width="40%">DPCM Marble WADY</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Interplay</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various Interplay computer games.</td></tr>

<tr><td width="40%">DPCM Squareroot-Delta-Exact</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in various games.</td></tr>

<tr><td width="40%">DPCM Sierra Online</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra Online game audio files.</td></tr>

<tr><td width="40%">DPCM Sol</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DPCM Xan</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Origin’s Wing Commander IV AVI files.</td></tr>

<tr><td width="40%">DPCM Xilam DERF</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), least significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSD (Direct Stream Digital), most significant bit first, planar</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DSP Group TrueSpeech</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DST (Direct Stream Transfer)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">DV audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Enhanced AC-3</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">EVRC (Enhanced Variable Rate Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">FLAC (Free Lossless Audio Codec)</td><td width="10%">X</td><td width="10%">IX</td></tr>

<tr><td width="40%">FTR Voice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.723.1</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">G.728</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">G.729</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">GSM</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">GSM Microsoft variant</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libgsm</td></tr>

<tr><td width="40%">IAC (Indeo Audio Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">iLBC (Internet Low Bitrate Codec)</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">encoding and decoding supported through external library libilbc</td></tr>

<tr><td width="40%">IMC (Intel Music Coder)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Interplay ACM</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">LC3</td><td width="10%">E</td><td width="10%">E</td><td width="40%">supported through external library liblc3</td></tr>

<tr><td width="40%">MACE (Macintosh Audio Compression/Expansion) 6:1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Marian’s A-pac audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MI-SC4 (Micronas SC-4 Audio)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MLP (Meridian Lossless Packing)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in DVD-Audio discs.</td></tr>

<tr><td width="40%">Monkey’s Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MP1 (MPEG audio layer 1)</td><td width="10%"></td><td width="10%">IX</td></tr>

<tr><td width="40%">MP2 (MPEG audio layer 2)</td><td width="10%">IX</td><td width="10%">IX</td><td width="40%">encoding supported also through external library TwoLAME</td></tr>

<tr><td width="40%">MP3 (MPEG audio layer 3)</td><td width="10%">E</td><td width="10%">IX</td><td width="40%">encoding supported through external library LAME, ADU MP3 and MP3onMP4 also supported</td></tr>

<tr><td width="40%">MPEG-4 Audio Lossless Coding (ALS)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">MPEG-H 3D Audio</td><td width="10%"></td><td width="10%">E</td><td width="40%">decoding supported through external library libmpeghdec</td></tr>

<tr><td width="40%">MobiClip FastAudio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV7</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Musepack SV8</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Nellymoser Asao</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">On2 AVC (Audio for Video Codec)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Opus</td><td width="10%">E</td><td width="10%">X</td><td width="40%">encoding supported through external library libopus</td></tr>

<tr><td width="40%">OSQ (Original Sound Quality)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM A-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM mu-law</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM Archimedes VIDC</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian planar</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 32-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM 64-bit floating point little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM D-Cinema audio signed 24-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian for DVD-Audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM signed 16/20/24-bit big-endian in MPEG-TS</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 8-bit</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 16-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 24-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit big-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM unsigned 32-bit little-endian</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">PCM SGA</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QCELP / PureVoice</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 1</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">QDesign Music Codec 2</td><td width="10%"></td><td width="10%">X</td><td width="40%">There are still some distortions.</td></tr>

<tr><td width="40%">RealAudio 1.0 (14.4K)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Real 14400 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 2.0 (28.8K)</td><td width="10%"></td><td width="10%">X</td><td width="40%">Real 28800 bit/s codec</td></tr>

<tr><td width="40%">RealAudio 3.0 (dnet)</td><td width="10%">IX</td><td width="10%">X</td><td width="40%">Real low bitrate AC-3 codec</td></tr>

<tr><td width="40%">RealAudio Lossless</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RealAudio SIPR / ACELP.NET</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">RK Audio (RKA)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SBC (low-complexity subband codec)</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in Bluetooth A2DP</td></tr>

<tr><td width="40%">Shorten</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Sierra VMD audio</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in Sierra VMD files.</td></tr>

<tr><td width="40%">Smacker audio</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">SMPTE 302M AES3 audio</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">Sonic</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Sonic lossless</td><td width="10%">X</td><td width="10%">X</td><td width="40%">experimental codec</td></tr>

<tr><td width="40%">Speex</td><td width="10%">E</td><td width="10%">EX</td><td width="40%">supported through external library libspeex</td></tr>

<tr><td width="40%">TAK (Tom’s lossless Audio Kompressor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">True Audio (TTA)</td><td width="10%">X</td><td width="10%">X</td></tr>

<tr><td width="40%">TrueHD</td><td width="10%">X</td><td width="10%">X</td><td width="40%">Used in HD-DVD and Blu-Ray discs.</td></tr>

<tr><td width="40%">TwinVQ (VQF flavor)</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">VIMA</td><td width="10%"></td><td width="10%">X</td><td width="40%">Used in LucasArts SMUSH animations.</td></tr>

<tr><td width="40%">ViewQuest VQC</td><td width="10%"></td><td width="10%">X</td></tr>

<tr><td width="40%">Vorbis</td><td width="10%">E</td><td width="10%">X</td><td width="40%">A native but very primitive encoder exists.</td></tr>

<tr><td width=”40%”>Voxware MetaSound</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Waveform Archiver</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>WavPack</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Westwood Audio (SND1)</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio 1</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio 2</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Lossless</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Pro</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Windows Media Audio Voice</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Xbox Media Audio 1</td><td width=”10%”></td><td width=”10%”>X</td></tr>

<tr><td width=”40%”>Xbox Media Audio 2</td><td width=”10%”></td><td width=”10%”>X</td></tr>

</tbody>

</table>

<code class=”code”>X</code> means that the feature in that column (encoding / decoding) is supported. <code class=”code”>E</code> means that support is provided through an external library. <code class=”code”>I</code> means that an integer-only version is available, too (ensures high performance on systems without hardware floating point support). <h3 class=”section”>2.5 Subtitle Formats# TOC</h3>

<table class=”multitable”> <tbody><tr><td width=”40%”>Name</td><td width=”10%”>Muxing</td><td width=”10%”>Demuxing</td><td width=”10%”>Encoding</td><td width=”10%”>Decoding</td></tr>
<tr><td width=”40%”>3GPP Timed Text</td><td width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>AQTitle</td><td width=”10%”></td><td

width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>DVB</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>DVB teletext</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>E</td></tr>
<tr><td width=”40%”>DVD</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>JACOsub</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MicroDVD</td><td width=”10%”>X</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MPL2</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>MPsub (MPlayer)</td><td width=”10%”></td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td></tr>
<tr><td width=”40%”>PGS</td><td width=”10%”></td><td width=”10%”></td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>PJS
(Phoenix)</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>RealText</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SAMI</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>Spruce format
(STL)</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>SSA/ASS</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubRip (SRT)</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer v1</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>SubViewer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>TED Talks cap-
tions</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td><td width=”10%”>X</td></tr> <tr><td width=”40%”>TTML</td><td
width=”10%”>X</td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”></td></tr> <tr><td width=”40%”>VobSub (IDX+SUB)</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>VPlayer</td><td
width=”10%”></td><td width=”10%”>X</td><td width=”10%”></td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>WebVTT</td><td
width=”10%”>X</td><td width=”10%”>X</td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> <tr><td width=”40%”>XSUB</td><td
width=”10%”></td><td width=”10%”></td><td width=”10%”>X</td><td
width=”10%”>X</td></tr> </tbody> </table>

<code class="code">X</code> means that the feature is supported. <code class="code">E</code> means that support is provided through an external library. <h3 class="section">2.6 Network Protocols# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td width="10%">Support</td></tr> <tr><td width="40%">AMQP</td><td width="10%">E</td></tr> <tr><td width="40%">file</td><td width="10%">X</td></tr> <tr><td width="40%">FTP</td><td width="10%">X</td></tr> <tr><td width="40%">Gopher</td><td width="10%">X</td></tr> <tr><td width="40%">Gophers</td><td width="10%">X</td></tr> <tr><td width="40%">HLS</td><td width="10%">X</td></tr> <tr><td width="40%">HTTP</td><td width="10%">X</td></tr> <tr><td width="40%">HTTPS</td><td width="10%">X</td></tr> <tr><td width="40%">Icecast</td><td width="10%">X</td></tr> <tr><td width="40%">MMSH</td><td width="10%">X</td></tr> <tr><td width="40%">MMST</td><td width="10%">X</td></tr> <tr><td width="40%">pipe</td><td width="10%">X</td></tr> <tr><td width="40%">Pro-MPEG FEC</td><td width="10%">X</td></tr> <tr><td width="40%">RTMP</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPE</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPS</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPT</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPTE</td><td width="10%">X</td></tr> <tr><td width="40%">RTMPTS</td><td width="10%">X</td></tr> <tr><td width="40%">RTP</td><td width="10%">X</td></tr> <tr><td width="40%">SAMBA</td><td width="10%">E</td></tr> <tr><td width="40%">SCTP</td><td width="10%">X</td></tr> <tr><td width="40%">SFTP</td><td width="10%">E</td></tr> <tr><td width="40%">TCP</td><td width="10%">X</td></tr> <tr><td width="40%">TLS</td><td width="10%">X</td></tr> <tr><td width="40%">UDP</td><td width="10%">X</td></tr> <tr><td width="40%">ZMQ</td><td width="10%">E</td></tr> </tbody> </table>

<code class="code">X</code> means that the protocol is supported. <code class="code">E</code> means that support is provided through an external library.

 <h3 class="section">2.7 Input/Output Devices# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Name</td><td
width="10%">Input</td><td width="10%">Output</td></tr> <tr><td
width="40%">ALSA</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">BKTR</td><td width="10%">X</td><td
width="10%"></td></tr> <tr><td width="40%">caca</td><td
width="10%"></td><td width="10%">X</td></tr> <tr><td width="40%">DV1394</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">Lavfi
virtual device</td><td width="10%">X</td><td width="10%"></td></tr>
<tr><td width="40%">Linux framebuffer</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">JACK</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">LIBCDIO</td><td
width="10%">X</td></tr> <tr><td width="40%">LIBDC1394</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">OpenAL</td><td
width="10%">X</td></tr> <tr><td width="40%">OSS</td><td
width="10%">X</td><td width="10%">X</td></tr> <tr><td
width="40%">PulseAudio</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">SDL</td><td width="10%"></td><td width="10%">X</td></tr>
<tr><td width="40%">Video4Linux2</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">VfW capture</td><td
width="10%">X</td><td width="10%"></td></tr> <tr><td width="40%">X11
grabbing</td><td width="10%">X</td><td width="10%"></td></tr>
<tr><td width="40%">Win32 grabbing</td><td width="10%">X</td><td
width="10%"></td></tr> </tbody> </table>

<code class="code">X</code> means that input/output is supported.
  <h3 class="section">2.8 Timecode<span
class="pull-right"><a class="anchor hidden-xs" href="#Timecode" aria-
hidden="true"># <a class="anchor hidden-xs"href="#toc-Timecode"
aria-hidden="true">TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">Codec/format</td><td
width="10%">Read</td><td width="10%">Write</td></tr> <tr><td
width="40%">AVI</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">DV</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">GXF</td><td width="10%">X</td><td
width="10%">X</td></tr> <tr><td width="40%">MOV</td><td
width="10%">X</td><td width="10%">X</td></tr> <tr><td
width="40%">MPEG1/2</td><td width="10%">X</td><td width="10%">X</td></tr>
<tr><td width="40%">MXF</td><td width="10%">X</td><td
width="10%">X</td></tr> </tbody> </table>

 This document was generated using <a
class="uref" href="https://www.gnu.org/software/texinfo/">makeinfo.
  </body> </html>

===== .\ffmpeg\doc\git-howto.html ===== <!DOCTYPE html PUBLIC "-
//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/

--> <head> <meta charset="utf-8"> <title> Using Git to develop FFmpeg </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Using Git to develop FFmpeg </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Introduction 2 Basics Usage <ul class="toc-numbered-mark"> 2.1 Get Git 2.2 Cloning the source tree 2.3 Updating the source tree to the latest revision 2.4 Rebasing your local branches 2.5 Adding/removing files/directories 2.6 Showing modifications 2.7 Inspecting the changelog 2.8 Checking source tree status 2.9 Committing 2.10 Writing a commit message 2.11 Preparing a patchset 2.12 Sending patches for review 2.13 Renaming/moving/copying files or contents of files 3 Git configuration <ul class="toc-numbered-mark"> 3.1 Personal Git installation 3.2 Repository configuration 4 FFmpeg specific <ul class="toc-numbered-mark"> 4.1 Reverting broken commits 4.2 Pushing

changes to remote trees 4.3 Finding a specific svn revision 5 gpg key generation 6 Pre-push checklist 7 Server Issues

 <h2 class="chapter">1 Introduction# TOC</h2>

This document aims in giving some quick references on a set of useful Git commands. You should always use the extensive and detailed documentation provided directly by Git: <pre class="example-preformatted">git --help man git </pre>

shows you the available subcommands,   <pre class="example-preformatted">git <command> --help man git-<command> </pre>

shows information about the subcommand <command>. Additional information could be found on the Git Reference website. For more information about the Git project, visit the Git website. Consult these resources whenever you have problems, they are quite exhaustive. What follows now is a basic introduction to Git and some FFmpeg-specific guidelines to ease the contribution to the project. <h2 class="chapter">2 Basics Usage# TOC</h2>

 <h3 class="section">2.1 Get Git# TOC</h3>

You can get Git from http://git-scm.com/ Most distribution and operating system provide a package for it.

 <h3 class="section">2.2 Cloning the source tree# TOC</h3>

 <pre class=”example-preformatted”>git clone https://git.ffmpeg.org/ffmpeg.git <target> </pre>

This will put the FFmpeg sources into the directory <var class=”var”><target></var>.   <pre class=”example-preformatted”>git clone git@source.ffmpeg.org:ffmpeg <target> </pre>

This will put the FFmpeg sources into the directory <var class=”var”><target></var> and let you push back your changes to the remote repository.   <pre class=”example-preformatted”>git clone git@ffmpeg.org:ffmpeg-web <target> </pre>

This will put the source of the FFmpeg website into the directory <var class=”var”><target></var> and let you push back your changes to the remote repository. If you don’t have write-access to the ffmpeg-web repository, you can create patches after making a read-only ffmpeg-web clone: <pre class=”example-preformatted”>git clone git://ffmpeg.org/ffmpeg-web <target> </pre>

Make sure that you do not have Windows line endings in your checkouts, otherwise you may experience spurious compilation failures. One way to achieve this is to run   <pre class=”example-preformatted”>git config --global core.autocrlf false </pre>

 <h3 class=”section”>2.3 Updating the source tree to the latest revision# TOC</h3>

 <pre class=”example-preformatted”>git pull (--rebase) </pre>

pulls in the latest changes from the tracked branch. The tracked branch can be remote. By default the master branch tracks the branch master in the remote origin.   <code class=”command”>--rebase</code> (see below) is recommended. <h3 class=”section”>2.4 Rebasing your local branches# TOC</h3>

 <pre class=”example-preformatted”>git pull --rebase </pre>

fetches the changes from the main repository and replays your local com-

mits over it. This is required to keep all your local changes at the top of FFmpeg’s master tree. The master tree will reject pushes with merge commits.

 <h3 class="section">2.5 Adding/removing files/directories# TOC</h3>

 <pre class="example-preformatted">git add [-A] <filename/dirname> git rm [-r] <filename/dirname> </pre>

Git needs to get notified of all changes you make to your working directory that makes files appear or disappear. Line moves across files are automatically tracked.

 <h3 class="section">2.6 Showing modifications# TOC</h3>

 <pre class="example-preformatted">git diff <filename(s)> </pre>

will show all local modifications in your working directory as unified diff.

 <h3 class="section">2.7 Inspecting the changelog# TOC</h3>

 <pre class="example-preformatted">git log <filename(s)> </pre>

You may also use the graphical tools like <code class="command">gitview</code> or <code class="command">gitk</code> or the web interface available at https://git.ffmpeg.org/ffmpeg.git. <h3 class="section">2.8 Checking source tree status# TOC</h3>

 <pre class="example-preformatted">git status </pre>

detects all the changes you made and lists what actions will be taken in case of a commit (additions, modifications, deletions, etc.).

 <h3 class="section">2.9 Committing# TOC</h3>

 <pre class="example-preformatted">git diff --check </pre>

to double check your changes before committing them to avoid trouble later on. All experienced developers do this on each and every commit, no matter how small. Every one of them has been saved from looking like a fool by this many times. It’s very easy for stray debug output or cosmetic modifications to slip in, please avoid problems through this extra level of scrutiny. For cosmetics-only commits you should get (almost) empty output from   <pre class="example-preformatted">git diff -w -b <filename(s)> </pre>

Also check the output of   <pre class="example-preformatted">git status </pre>

to make sure you don’t have untracked files or deletions.   <pre class="example-preformatted">git add [-i|-p|-A] <filenames/dirnames> </pre>

Make sure you have told Git your name, email address and GPG key   <pre class="example-preformatted">git config --global user.name "My Name" git config --global user.email my@email.invalid git config --global user.signingkey ABCDEF0123245 </pre>

Enable signing all commits or use -S   <pre class="example-preformatted">git config --global commit.gpgsign true </pre>

Use <samp class="option">--global</samp> to set the global configuration for all your Git checkouts. Git will select the changes to the files for commit. Optionally you can use the interactive or the patch mode to select hunk by hunk what should be added to the commit.

 <pre class="example-preformatted">git commit </pre>

Git will commit the selected changes to your current local branch. You will be prompted for a log message in an editor, which is either set in your personal configuration file through   <pre class="example-preformatted">git config --global core.editor </pre>

or set by one of the following environment variables: <var class="var">GIT_EDITOR</var>, <var class="var">VISUAL</var> or <var class="var">EDITOR</var>. <h3 class="section">2.10 Writing a commit message<a class="anchor

hidden-xs" href="#Writing-a-commit-message" aria-hidden="true"># TOC</h3>

Log messages should be concise but descriptive. The first line must contain the context, a colon and a very short summary of what the commit does. Details can be added, if necessary, separated by an empty line. These details should not exceed 60-72 characters per line, except when containing code. Example of a good commit message: <pre class="example-preformatted">avcodec/cbs: add a helper to read extradata within packet side data

Using ff_cbs_read() on the raw buffer will not parse it as extradata, resulting in parsing errors for example when handling ISOBMFF avcC. This helper works around that. </pre>

 <pre class="example-preformatted">ptr might be NULL </pre>

If the summary on the first line is not enough, in the body of the message, explain why you made a change, what you did will be obvious from the changes themselves most of the time. Saying just "bug fix" or "10l" is bad. Remember that people of varying skill levels look at and educate themselves while reading through your code. Don’t include filenames in log messages except in the context, Git provides that information. If the commit fixes a registered issue, state it in a separate line of the body: <code class="code">Fix Trac ticket #42.</code> The first line will be used to name the patch by <code class="command">git format-patch</code>. Common mistakes for the first line, as seen in <code class="command">git log --oneline</code> include: missing context at the beginning; description of what the code did before the patch; line too long or wrapped to the second line. <h3 class="section">2.11 Preparing a patchset# TOC</h3>

 <pre class="example-preformatted">git format-patch <commit> [-o directory] </pre>

will generate a set of patches for each commit between <var class="var"><commit></var> and current <var class="var">HEAD</var>. E.g.   <pre class="example-preformatted">git format-patch origin/master </pre>

will generate patches for all commits on current branch which are not present in upstream. A useful shortcut is also   <pre class="example-preformatted">git format-patch -n </pre>

which will generate patches from last <var class="var">n</var> commits. By default the patches are created in the current directory.

 <h3 class="section">2.12 Sending patches for review# TOC</h3>

 <pre class="example-preformatted">git send-email <commit list|directory> </pre>

will send the patches created by <code class="command">git format-patch</code> or directly generates them. All the email fields can be configured in the global/local configuration or overridden by command line. Note that this tool must often be installed separately (e.g. <var class="var">git-email</var> package on Debian-based distros).

 <h3 class="section">2.13 Renaming/moving/copying files or contents of files# TOC</h3>

Git automatically tracks such changes, making those normal commits.   <pre class="example-preformatted">mv/cp path/file otherpath/otherfile git add [-A] . git commit </pre>

 <h2 class="chapter">3 Git configuration# TOC</h2>

In order to simplify a few workflows, it is advisable to configure both your personal Git installation and your local FFmpeg repository. <h3 class="section">3.1 Personal Git installation# TOC</h3>

Add the following to your <samp class="file">~/.gitconfig</samp> to help <code class="command">git send-email</code> and <code class="command">git format-patch</code> detect renames: <pre class="example-preformatted">[diff] renames = copy </pre>

 <h3 class="section">3.2 Repository configuration# TOC</h3>

In order to have <code class="command">git send-email</code> automatically send patches to the ffmpeg-devel mailing list, add the following stanza to <samp class="file">/path/to/ffmpeg/repository/.git/config</samp>:   <pre class="example-preformatted">[sendemail] to = ffmpeg-devel@ffmpeg.org </pre>

 <h2 class="chapter">4 FFmpeg specific# TOC</h2>

 <h3 class="section">4.1 Reverting broken commits# TOC</h3>

 <pre class="example-preformatted">git reset <commit> </pre>

<code class="command">git reset</code> will uncommit the changes till <var class="var"><commit></var> rewriting the current branch history.   <pre class="example-preformatted">git commit --amend </pre>

allows one to amend the last commit details quickly.   <pre class="example-preformatted">git rebase -i origin/master </pre>

will replay local commits over the main repository allowing to edit, merge or remove some of them in the process.   <code class="command">git reset</code>, <code class="command">git commit --amend</code> and <code class="command">git rebase</code> rewrite history, so you should use them ONLY on your local or topic branches. The main repository will reject those changes.   <pre class="example-preformatted">git revert <commit> </pre>

<code class="command">git revert</code> will generate a revert commit. This will not make the faulty commit disappear from the history. <h3 class="section">4.2 Pushing changes to remote trees# TOC</h3>

 <pre class="example-preformatted">git push origin master --dry-run </pre>

Will simulate a push of the local master branch to the default remote

(<var class=”var”>origin</var>). And list which branches and ranges or commits would have been pushed. Git will prevent you from pushing changes if the local and remote trees are out of sync. Refer to Updating the source tree to the latest revision.   <pre class=”example-preformatted”>git remote add <name> <url> </pre>

Will add additional remote with a name reference, it is useful if you want to push your local branch for review on a remote host.   <pre class=”example-preformatted”>git push <remote> <refspec> </pre>

Will push the changes to the <var class=”var”><remote></var> repository. Omitting <var class=”var”><refspec></var> makes <code class=”command”>git push</code> update all the remote branches matching the local ones. <h3 class=”section”>4.3 Finding a specific svn revision# TOC</h3>

Since version 1.7.1 Git supports ‘<samp class=”samp”>:/foo</samp>’ syntax for specifying commits based on a regular expression. see man gitrevisions   <pre class=”example-preformatted”>git show :/'as revision 23456' </pre>

will show the svn changeset ‘<samp class=”samp”>r23456</samp>’. With older Git versions searching in the <code class=”command”>git log</code> output is the easiest option (especially if a pager with search capabilities is used). This commit can be checked out with   <pre class=”example-preformatted”>git checkout -b svn_23456 :/'as revision 23456' </pre>

or for Git < 1.7.1 with   <pre class=”example-preformatted”>git checkout -b svn_23456 $SHA1 </pre>

where <var class=”var”>$SHA1</var> is the commit hash from the <code class=”command”>git log</code> output.

 <h2 class=”chapter”>5 gpg key generation# TOC</h2>

If you have no gpg key yet, we recommend that you create a ed25519 based key as it is small, fast and secure. Especially it results in small signatures in git.   <pre class=”example-preformatted”>gpg --default-new-key-algo "ed25519/cert,sign+cv25519/encr" --quick-generate-key "human@server.com" </pre>

When generating a key, make sure the email specified matches the email used in git as some sites like github consider mismatches a reason to declare such commits unverified. After generating a key you can add it to the MAINTAINER file and upload it to a keyserver. <h2 class="chapter">6 Pre-push checklist# TOC</h2>

Once you have a set of commits that you feel are ready for pushing, work through the following checklist to doublecheck everything is in proper order. This list tries to be exhaustive. In case you are just pushing a typo in a comment, some of the steps may be unnecessary. Apply your common sense, but if in doubt, err on the side of caution. First, make sure that the commits and branches you are going to push match what you want pushed and that nothing is missing, extraneous or wrong. You can see what will be pushed by running the git push command with <samp class="option">--dry-run</samp> first. And then inspecting the commits listed with <code class="command">git log -p 1234567..987654</code>. The <code class="command">git status</code> command may help in finding local changes that have been forgotten to be added. Next let the code pass through a full run of our test suite. <ul class="itemize mark-bullet">
<code class="command">make distclean</code> <code class="command">/path/to/ffmpeg/configure</code> <code class="command">make fate</code> if fate fails due to missing samples run <code class="command">make fate-rsync</code> and retry

Make sure all your changes have been checked before pushing them, the test suite only checks against regressions and that only to some extend. It does obviously not check newly added features/code to be working unless you have added a test for that (which is recommended). Also note that every single commit should pass the test suite, not just the result of a series of patches. Once everything passed, push the changes to your public ffmpeg clone and post a merge request to ffmpeg-devel. You can also push them directly but this is not recommended. <h2 class="chapter">7 Server Issues# TOC</h2>

Contact the project admins at root@ffmpeg.org if you have technical problems with the Git server.   This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\libavcodec.html ===== <!DOCTYPE html PUBLIC

”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> Libavcodec Documentation
</title> <meta name=”viewport” content=”width=device-width,initial-
scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> Libavcodec Documentation </h1>

 <h2 class=”contents-
heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1
Description 2
See Also 3 Au-
thors

 <h2 class=”chapter”>1 Description<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Description” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Description”
aria-hidden=”true”>TOC</h2>

The libavcodec library provides a generic encoding/decoding framework
and contains multiple decoders and encoders for audio, video and subtitle
streams, and several bitstream filters. The shared architecture
provides various services ranging from bit stream I/O to DSP optimizations,
and makes it suitable for implementing robust and fast codecs as well as for
experimentation.

 <h2 class=”chapter”>2 See Also<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#See-Also” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-See-Also”
aria-hidden=”true”>TOC</h2>

ffmpeg, <a class=”url”
href=”ffplay.html”>ffplay, ffprobe,
ffmpeg-codecs, <a class=”url”
href=”ffmpeg-bitstream-filters.html”>bitstream-filters, <a class=”url”
href=”libavutil.html”>libavutil

 <h2 class=”chapter”>3 Authors<span class=”pull-
right”>#
TOC</h2>

The FFmpeg developers. For details about the authorship,
see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by
typing the command <code class=”command”>git log</code> in the FFm-
peg source directory, or browsing the online repository at <a class=”url”

href=”https://git.ffmpeg.org/ffmpeg”>https://git.ffmpeg.org/ffmpeg.
 Maintainers for the specific components are listed in the file <samp
class=”file”>MAINTAINERS</samp> in the source code tree.

<p style=”font-size: small;”> This document was generated using <a
class=”uref” href=”https://www.gnu.org/software/texinfo/”>makeinfo.
  </body> </html>

===== .\ffmpeg\doc\libavdevice.html ===== <!DOCTYPE html PUBLIC
”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> Libavdevice Documentation
</title> <meta name=”viewport” content=”width=device-width,initial-
scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> Libavdevice Documentation </h1>

 <h2 class=”contents-
heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1
Description 2
See Also 3 Au-
thors

 <h2 class=”chapter”>1 Description<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Description” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Description”
aria-hidden=”true”>TOC</h2>

The libavdevice library provides a generic framework for grabbing from
and rendering to many common multimedia input/output devices, and supports
several input and output devices, including Video4Linux2, VfW, DShow, and
ALSA.

 <h2 class=”chapter”>2 See Also<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#See-Also” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-See-Also”
aria-hidden=”true”>TOC</h2>

ffmpeg, <a class=”url”
href=”ffplay.html”>ffplay, ffprobe,
ffmpeg-devices, <a
class=”url” href=”libavutil.html”>libavutil, libavcodec,
libavformat

 <h2 class="chapter">3 Authors#
TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg.
 Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.
  </body> </html>

===== .\ffmpeg\doc\libavfilter.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset="utf-8"> <title> Libavfilter Documentation
</title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css">
<link rel="stylesheet" type="text/css" href="style.min.css"> </head>
<body> <h1> Libavfilter Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 See Also 3 Authors

 <h2 class="chapter">1 Description# TOC</h2>

The libavfilter library provides a generic audio/video filtering framework containing several filters, sources and sinks.

 <h2 class="chapter">2 See Also# TOC</h2>

ffmpeg, <a class="url"

href=”ffplay.html”>ffplay, ffprobe,
ffmpeg-filters, <a class=”url”
href=”libavutil.html”>libavutil, libswscale,
libswresample, <a class=”url”
href=”libavcodec.html”>libavcodec, libavformat,
libavdevice

 <h2 class=”chapter”>3 Authors<span class=”pull-
right”>#
TOC</h2>

The FFmpeg developers. For details about the authorship,
see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by
typing the command <code class=”command”>git log</code> in the FFm-
peg source directory, or browsing the online repository at <a class=”url”
href=”https://git.ffmpeg.org/ffmpeg”>https://git.ffmpeg.org/ffmpeg.
 Maintainers for the specific components are listed in the file <samp
class=”file”>MAINTAINERS</samp> in the source code tree.

<p style=”font-size: small;”> This document was generated using <a
class=”uref” href=”https://www.gnu.org/software/texinfo/”>makeinfo.
  </body> </html>

===== .\ffmpeg\doc\libavformat.html ===== <!DOCTYPE html PUB-
LIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”>
<html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/
--> <head> <meta charset=”utf-8”> <title> Libavformat Documentation
</title> <meta name=”viewport” content=”width=device-width,initial-
scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”>
<link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head>
<body> <h1> Libavformat Documentation </h1>

 <h2 class=”contents-
heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1
Description 2
See Also 3 Au-
thors

 <h2 class=”chapter”>1 Description<span
class=”pull-right”><a class=”anchor hidden-xs” href=”#Description” aria-
hidden=”true”># <a class=”anchor hidden-xs”href=”#toc-Description”
aria-hidden=”true”>TOC</h2>

The libavformat library provides a generic framework for multiplexing and
demultiplexing (muxing and demuxing) audio, video and subtitle streams. It

encompasses multiple muxers and demuxers for multimedia container formats. It also supports several input and output protocols to access a media resource.

 <h2 class="chapter">2 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, ffmpeg-formats, ffmpeg-protocols, libavutil, libavcodec

 <h2 class="chapter">3 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\libavutil.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> Libavutil Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Libavutil Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2

See Also 3 Authors

 <h2 class="chapter">1 Description# TOC</h2>

The libavutil library is a utility library to aid portable multimedia programming. It contains safe portable string functions, random number generators, data structures, additional mathematics functions, cryptography and multimedia related functionality (like enumerations for pixel and sample formats). It is not a library for code needed by both libavcodec and libavformat. The goals for this library is to be: <dl class="table"> <dt>Modular</dt> <dd> It should have few interdependencies and the possibility of disabling individual parts during <code class="command">./configure</code>. </dd> <dt>Small</dt> <dd> Both sources and objects should be small. </dd> <dt>Efficient</dt> <dd> It should have low CPU and memory usage. </dd> <dt>Useful</dt> <dd> It should avoid useless features that almost no one needs. </dd> </dl>

 <h2 class="chapter">2 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, ffmpeg-utils

 <h2 class="chapter">3 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\libswresample.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> Libswresample Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Libswresample Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 See Also 3 Authors

 <h2 class="chapter">1 Description# TOC</h2>

The libswresample library performs highly optimized audio resampling, rematrixing and sample format conversion operations. Specifically, this library performs the following conversions: <ul class="itemize mark-bullet"> Resampling: is the process of changing the audio rate, for example from a high sample rate of 44100Hz to 8000Hz. Audio conversion from high to low sample rate is a lossy process. Several resampling options and algorithms are available.

Format conversion: is the process of converting the type of samples, for example from 16-bit signed samples to unsigned 8-bit or float samples. It also handles packing conversion, when passing from packed layout (all samples belonging to distinct channels interleaved in the same buffer), to planar layout (all samples belonging to the same channel stored in a dedicated buffer or "plane").

Rematrixing: is the process of changing the channel layout, for example from stereo to mono. When the input channels cannot be mapped to the output streams, the process is lossy, since it involves different gain factors and mixing.

Various other audio conversions (e.g. stretching and padding) are enabled through dedicated options.

 <h2 class="chapter">2 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, ffmpeg-resampler, libavutil

 <h2 class="chapter">3 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\libswscale.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> Libswscale Documentation </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> Libswscale Documentation </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 See Also 3 Authors

 <h2 class="chapter">1 Description# TOC</h2>

The libswscale library performs highly optimized image scaling and colorspace and pixel format conversion operations. Specifically, this library performs the following conversions: <ul class="itemize mark-bullet"> Rescaling: is the process of changing the video size. Several rescaling options and algorithms are available. This is usually a lossy process.

Pixel format conversion: is the process of converting the image format and colorspace of the image, for example from planar YUV420P to RGB24 packed. It also handles packing conversion, that is converts from packed layout (all pixels belonging to distinct planes interleaved in the same buffer), to planar layout (all samples belonging to the same plane stored in a dedicated buffer or "plane").

This is usually a lossy process in case the source and destination colorspaces differ.

 <h2 class="chapter">2 See Also# TOC</h2>

ffmpeg, ffplay, ffprobe, ffmpeg-scaler, libavutil

 <h2 class="chapter">3 Authors# TOC</h2>

The FFmpeg developers. For details about the authorship, see the Git history of the project (https://git.ffmpeg.org/ffmpeg), e.g. by typing the command <code class="command">git log</code> in the FFmpeg source directory, or browsing the online repository at https://git.ffmpeg.org/ffmpeg. Maintainers for the specific components are listed in the file <samp class="file">MAINTAINERS</samp> in the source code tree.

 This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\mailing-list-faq.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> FFmpeg Mailing List FAQ </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css">

<link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> FFmpeg Mailing List FAQ </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 General Questions <ul class="toc-numbered-mark"> 1.1 What is a mailing list? 1.2 What type of questions can I ask? 1.3 How do I ask a question or send a message to a mailing list? 2 Subscribing / Unsubscribing <ul class="toc-numbered-mark"> 2.1 How do I subscribe? 2.2 How do I unsubscribe? 3 Moderation Queue <ul class="toc-numbered-mark"> 3.1 Why is my message awaiting moderator approval? 3.2 How long does it take for my message in the moderation queue to be approved? 3.3 How do I delete my message in the moderation queue? 4 Archives <ul class="toc-numbered-mark"> 4.1 Where are the archives? 4.2 How do I reply to a message in the archives? 4.3 How do I search the archives? 5 Other <ul class="toc-numbered-mark"> 5.1 Is there an alternative to the

mailing list? 5.2 What is top-posting? 5.3 What is the message size limit? 5.4 Where can I upload sample files? 5.5 Will I receive spam if I send and/or subscribe to a mailing list? 5.6 How do I filter mailing list messages? 5.7 How do I disable mail delivery without unsubscribing? 5.8 Why is the mailing list munging my address? 6 Rules and Etiquette <ul class="toc-numbered-mark"> 6.1 What are the rules and the proper etiquette? 7 Help <ul class="toc-numbered-mark"> 7.1 Why am I not receiving any messages? 7.2 Why are my sent messages not showing up? 7.3 Why do I keep getting unsubscribed from ffmpeg-devel? 7.4 Who do I contact if I have a problem with the mailing list?

 <h2 class="chapter">1 General Questions# TOC</h2>

 <h3 class="section">1.1 What is a mailing list?# TOC</h3>

A mailing list is not much different than emailing someone, but the main difference is that your message is received by everyone who subscribes to the list. It is somewhat like a forum but in email form. See the ffmpeg-user archives for examples. <h3 class="section">1.2 What type of questions can I ask?# TOC</h3>

<ul class="itemize mark-bullet"> ffmpeg-user: For questions involving unscripted usage or compilation of the FFmpeg command-line tools (<code class="command">ffmpeg</code>, <code class="command">ffprobe</code>, <code class="command">ffplay</code>).

libav-user: For questions involving the FFmpeg libav* libraries (libavcodec, libavformat, libavfilter, etc).

ffmpeg-devel: For discussions involving the development of FFmpeg and for submitting patches. User questions should be asked at ffmpeg-user or libav-user.

To report a bug see https://ffmpeg.org/bugre
 We cannot provide help for scripts and/or third-party tools. <h3 class="section">1.3 How do I ask a question or send a message to a mailing list?# TOC</h3>

First you must subscribe. Then all you have to do is send an email: <ul class="itemize mark-bullet"> Email ffmpeg-user@ffmpeg.org to send a message to the ffmpeg-user mailing list.

Email libav-user@ffmpeg.org to send a message to the libav-user mailing list.

Email ffmpeg-devel@ffmpeg.org to send a message to the ffmpeg-devel mailing list.

 <h2 class="chapter">2 Subscribing / Unsubscribing<a class="anchor

hidden-xs" href="#Subscribing-_002f-Unsubscribing" aria-hidden="true"># TOC</h2>

 <h3 class="section">2.1 How do I subscribe?# TOC</h3>

Email ffmpeg-user-request@ffmpeg.org with the subject subscribe. Or visit the ffmpeg-user mailing list info page and refer to the Subscribing to ffmpeg-user section. The process is the same for the other mailing lists. <h3 class="section">2.2 How do I unsubscribe?# TOC</h3>

Email ffmpeg-user-request@ffmpeg.org with subject unsubscribe. Or visit the ffmpeg-user mailing list info page, scroll to bottom of page, enter your email address in the box, and click the Unsubscribe or edit options button. The process is the same for the other mailing lists. Please avoid asking a mailing list admin to unsubscribe you unless you are absolutely unable to do so by yourself. See Who do I contact if I have a problem with the mailing list? Note that it is possible to temporarily halt message delivery (vacation mode). See How do I disable mail delivery without unsubscribing? <h2 class="chapter">3 Moderation Queue# TOC</h2> <h3 class="section">3.1 Why is my message awaiting moderator approval?# TOC</h3>

Some messages are automatically held in the moderation

queue and must be manually approved by a mailing list admin: These are: <ul class="itemize mark-bullet"> Messages that exceed the message size limit.

Messages from users whose accounts have been set with the moderation flag (very rarely occurs, but may if a user repeatedly ignores the rules or is abusive towards others).

 <h3 class="section">3.2 How long does it take for my message in the moderation queue to be approved?# TOC</h3>

The queue is not checked on a regular basis. You can ask on the <code class="t">#ffmpeg-devel</code> IRC channel on Libera Chat for someone to approve your message. <h3 class="section">3.3 How do I delete my message in the moderation queue?# TOC</h3>

You should have received an email with the subject Your message to <mailing list name> awaits moderator approval. A link is in the message that will allow you to delete your message unless a mailing list admin already approved or rejected it. <h2 class="chapter">4 Archives# TOC</h2>

 <h3 class="section">4.1 Where are the archives?# TOC</h3>

See the Archives section on the FFmpeg Contact page for links to all FFmpeg mailing list archives. Note that the archives are split by month. Discussions that span several months will be split

into separate months in the archives. <h3 class="section">4.2 How do I reply to a message in the archives?# TOC</h3>

Click the email link at the top of the message just under the subject title. The link will provide the proper headers to keep the message within the thread. Note that you must be subscribed to send a message to the ffmpeg-user or libav-user mailing lists. <h3 class="section">4.3 How do I search the archives?# TOC</h3>

Perform a site search using your favorite search engine. Example: <code class="t">site:lists.ffmpeg.org/pipermail/ffmpeg-user/ "search term"</code> <h2 class="chapter">5 Other# TOC</h2>

 <h3 class="section">5.1 Is there an alternative to the mailing list?# TOC</h3>

You can ask for help in the official <code class="t">#ffmpeg</code> IRC channel on Libera Chat. There are also numerous third-party help sites such as Super User and r/ffmpeg on reddit. <h3 class="section">5.2 What is top-posting?# TOC</h3>

See https://en.wikipedia.org/wiki/Posting_style#Top-posting. Instead, use trimmed interleaved/inline replies (example). <a

name=”What-is-the-message-size-limit_003f-1”> <h3 class=”section”>5.3 What is the message size limit?# TOC</h3>

The message size limit is 1000 kilobytes. Please provide links to larger files instead of attaching them. <h3 class=”section”>5.4 Where can I upload sample files?# TOC</h3>

Anywhere that is not too annoying for us to use. Google Drive and Dropbox are acceptable if you need a file host, and 0x0.st is good for files under 256 MiB. Small, short samples are preferred if possible. <h3 class=”section”>5.5 Will I receive spam if I send and/or subscribe to a mailing list?# TOC</h3>

Highly unlikely. <ul class=”itemize mark-bullet”> The list of subscribed users is not public.

Email addresses in the archives are obfuscated.

Several unique test email accounts were utilized and none have yet received any spam.

However, you may see a spam in the mailing lists on rare occasions: <ul class=”itemize mark-bullet”> Spam in the moderation queue may be accidentally approved due to human error.

There have been a few messages from subscribed users who had their own email addresses hacked and spam messages from (or appearing to be from) the hacked account were sent to their contacts (a mailing list being a contact in these cases).

If you are subscribed to the bug tracker mailing list (ffmpeg-trac) you may see the occasional spam as a false bug report, but we take measures to try to prevent this.

 <h3 class=”section”>5.6 How do I filter mailing list messages?<a class=”anchor hidden-xs” href=”#How-do-I-filter-mailing-list-messages_003f”

aria-hidden=”true”># TOC</h3>

Use the List-Id. For example, the ffmpeg-user mailing list is <code class=”t”>ffmpeg-user.ffmpeg.org</code>. You can view the List-Id in the raw message or headers. You can then filter the mailing list messages to their own folder. <h3 class=”section”>5.7 How do I disable mail delivery without unsubscribing?# TOC</h3>

Sometimes you may want to temporarily stop receiving all mailing list messages. This "vacation mode" is simple to do: <ol class=”enumerate”> Go to the ffmpeg-user mailing list info page

 Enter your email address in the box at very bottom of the page and click the Unsubscribe or edit options box.

 Enter your password and click the Log in button.

 Look for the Mail delivery option. Here you can disable/enable mail delivery. If you check Set globally it will apply your choice to all other FFmpeg mailing lists you are subscribed to.

Alternatively, from your subscribed address, send a message to ffmpeg-user-request@ffmpeg.org with the subject set delivery off. To re-enable mail delivery send a message to ffmpeg-user-request@ffmpeg.org with the subject set delivery on. <h3 class=”section”>5.8 Why is the mailing list munging my address?# TOC</h3>

This is due to subscribers that use an email service with a DMARC reject policy which adds difficulties to mailing list operators. The mailing list must re-write (munge) the From:

header for such users; otherwise their email service will reject and bounce the message resulting in automatic unsubscribing from the mailing list. When sending a message these users will see via <mailing list name> added to their name and the From: address munged to the address of the particular mailing list. If you want to avoid this then please use a different email service. Note that ffmpeg-devel does not apply any munging as it causes issues with patch authorship. As a result users with an email service with a DMARC reject policy may be automatically unsubscribed due to rejected and bounced messages. <h2 class="chapter">6 Rules and Etiquette# TOC</h2>

 <h3 class="section">6.1 What are the rules and the proper etiquette?# TOC</h3>

There may seem to be many things to remember, but we want to help and following these guidelines will allow you to get answers more quickly and help avoid getting ignored. <ul class="itemize mark-bullet"> Always show your actual, unscripted <code class="command">ffmpeg</code> command and the complete, uncut console output from your command.

Use the most simple and minimal command that still shows the issue you are encountering.

Provide all necessary information so others can attempt to duplicate your issue. This includes the actual command, complete uncut console output, and any inputs that are required to duplicate the issue.

Use the latest <code class="command">ffmpeg</code> build you can get. See the FFmpeg Download page for links to recent builds for Linux, macOS, and Windows. Or compile from the current git master branch.

Avoid top-posting. Also see What is top-posting?

Avoid hijacking threads. Thread hijacking is replying to a message and changing the subject line to something unrelated to the original thread. Most email clients will still show the renamed message under the original thread. This can be confusing and these types of messages are often ignored.

Do not send screenshots. Copy and paste console text instead of making screenshots of the text.

Avoid sending email disclaimers and legalese if possible as this is a public list.

Avoid using the <code class="code">-loglevel debug</code>, <code class="code">-loglevel quiet</code>, and <code class="command">-hide_banner</code> options unless requested to do so.

If you attach files avoid compressing small files. Uncompressed is preferred.

Please do not send HTML-only messages. The mailing list will ignore the HTML component of your message. Most mail clients will automatically include a text component: this is what the mailing list will use.

Configuring your mail client to break lines after 70 or so characters is recommended.

Avoid sending the same message to multiple mailing lists.

Please follow our Code of Conduct.

 <h2 class="chapter">7 Help# TOC</h2>

 <h3 class="section">7.1 Why am I not receiving any messages?# TOC</h3>

Some email providers have blacklists or spam filters that block or mark the mailing list messages as false positives. Unfortunately, the user is often not aware of this and is often out of their control. When possible we attempt to notify the provider to be removed from the blacklists or filters. <h3 class="section">7.2 Why are my sent messages not showing up?# TOC</h3>

Excluding messages that are held in the moderation queue there are a few other reasons why your messages may fail to appear: <ul class="itemize mark-bullet"> HTML-only messages are ignored by the mailing lists. Most mail clients automatically include a text component

alongside HTML email: this is what the mailing list will use. If it does not then consider your client to be broken, because sending a text component along with the HTML component to form a multi-part message is recommended by email standards.

Check your spam folder.

 <h3 class="section">7.3 Why do I keep getting unsubscribed from ffmpeg-devel?# TOC</h3>

Users with an email service that has a DMARC reject or quarantine policy may be automatically unsubscribed from the ffmpeg-devel mailing list due to the mailing list messages being continuously rejected and bounced back. Consider using a different email service. <h3 class="section">7.4 Who do I contact if I have a problem with the mailing list?# TOC</h3>

Send a message to ffmpeg-user-owner@ffmpeg.org.   This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\nut.html ===== <!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd"> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset="utf-8"> <title> NUT </title> <meta name="viewport" content="width=device-width,initial-scale=1.0"> <link rel="stylesheet" type="text/css" href="bootstrap.min.css"> <link rel="stylesheet" type="text/css" href="style.min.css"> </head> <body> <h1> NUT </h1>

 <h2 class="contents-heading">Table of Contents</h2>

<ul class="toc-numbered-mark"> 1 Description 2 Modes <ul class="toc-numbered-mark"> 2.1 BROADCAST 2.2 PIPE 3 Container-specific codec tags <ul class="toc-numbered-mark"> 3.1 Generic raw YUVA formats 3.2 Raw Audio 3.3 Subtitles 3.4 Raw Data 3.5 Codecs

 <h2 class="chapter">1 Description# TOC</h2> NUT is a low overhead generic container format. It stores audio, video, subtitle and user-defined streams in a simple, yet efficient, way. It was created by a group of FFmpeg and MPlayer developers in 2003 and was finalized in 2008. The official nut specification is at https://git.ffmpeg.org/gitweb/nut.git/blob/refs/heads/master:/docs/nut.txt In case of any differences between this text and the official specification, the official specification shall prevail. <h2 class="chapter">2 Modes# TOC</h2> NUT has some variants signaled by using the flags field in its main header. <table class="multitable"> <tbody><tr><td width="40%">BROADCAST</td><td width="40%">Extend the syncpoint to report the sender wallclock</td></tr> <tr><td width="40%">PIPE</td><td width="40%">Omit completely the syncpoint</td></tr> </tbody> </table>

 <h3 class="section">2.1 BROADCAST# TOC</h3>

The BROADCAST variant provides a secondary time reference to facilitate detecting endpoint latency and network delays. It assumes all the endpoint clocks are synchronized. To be used in real-time scenarios. <h3 class="section">2.2 PIPE# TOC</h3>

The PIPE variant assumes NUT is used as non-seekable intermediate container, by not using syncpoint removes unneeded overhead and reduces the overall memory usage. <h2 class="chapter">3 Container-specific codec tags# TOC</h2>

 <h3 class="section">3.1 Generic raw YUVA formats# TOC</h3>

Since many exotic planar YUVA pixel formats are not considered by the AVI/QuickTime FourCC lists, the following scheme is adopted for representing them. The first two bytes can contain the values: Y1 = only Y Y2 = Y+A Y3 = YUV Y4 = YUVA The third byte represents the width and height chroma subsampling values for the UV planes, that is the amount to shift the luma width/height right to find the chroma width/height. The fourth byte is the number of bits used (8, 16, ...). If the order of bytes is inverted, that means that each component has to be read big-endian. <h3 class="section">3.2 Raw Audio# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">ALAW</td><td width="40%">A-LAW</td></tr> <tr><td width="40%">ULAW</td><td width="40%">MU-LAW</td></tr> <tr><td width="40%">P<type><interleaving><bits&g width="40%">little-endian PCM</td></tr> <tr><td width="40%"><bits><interleaving><t width="40%">big-endian PCM</td></tr> </tbody> </table>

<type> is S for signed integer, U for unsigned integer, F for IEEE float <interleaving> is D for default, P is for planar. <bits> is 8/16/24/32   <pre class="example-preformatted">PFD[32] would for example be signed 32 bit little-endian IEEE float </pre>

 <h3 class="section">3.3 Subtitles# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">UTF8</td><td width="40%">Raw UTF-8</td></tr> <tr><td width="40%">SSA[0]</td><td width="40%">SubStation Alpha</td></tr> <tr><td width="40%">DVDS</td><td width="40%">DVD subtitles</td></tr> <tr><td width="40%">DVBS</td><td width="40%">DVB subtitles</td></tr> </tbody> </table>

 <h3 class="section">3.4 Raw Data# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">UTF8</td><td width="40%">Raw UTF-8</td></tr> </tbody> </table>

 <h3 class="section">3.5 Codecs# TOC</h3>

<table class="multitable"> <tbody><tr><td width="40%">3IV1</td><td width="40%">non-compliant MPEG-4 generated by old 3ivx</td></tr> <tr><td width="40%">ASV1</td><td width="40%">Asus Video</td></tr> <tr><td width="40%">ASV2</td><td width="40%">Asus Video 2</td></tr> <tr><td width="40%">CVID</td><td width="40%">Cinepak</td></tr> <tr><td width="40%">CYUV</td><td width="40%">Creative YUV</td></tr> <tr><td width="40%">DIVX</td><td width="40%">non-compliant MPEG-4 generated by old DivX</td></tr> <tr><td width="40%">DUCK</td><td width="40%">Truemotion 1</td></tr> <tr><td width="40%">FFV1</td><td width="40%">FFmpeg video 1</td></tr> <tr><td width="40%">FFVH</td><td width="40%">FFmpeg Huffyuv</td></tr> <tr><td width="40%">H261</td><td width="40%">ITU H.261</td></tr> <tr><td width="40%">H262</td><td width="40%">ITU H.262</td></tr> <tr><td width="40%">H263</td><td width="40%">ITU H.263</td></tr> <tr><td width="40%">H264</td><td width="40%">ITU H.264</td></tr> <tr><td width="40%">HFYU</td><td width="40%">Huffyuv</td></tr> <tr><td width="40%">I263</td><td width="40%">Intel H.263</td></tr> <tr><td width="40%">IV31</td><td width="40%">Indeo 3.1</td></tr> <tr><td width="40%">IV32</td><td width="40%">Indeo 3.2</td></tr> <tr><td width="40%">IV50</td><td width="40%">Indeo 5.0</td></tr> <tr><td width="40%">LJPG</td><td width="40%">ITU JPEG (lossless)</td></tr> <tr><td width="40%">MJLS</td><td width="40%">ITU JPEG-LS</td></tr> <tr><td width="40%">MJPG</td><td width="40%">ITU JPEG</td></tr> <tr><td width="40%">MPG4</td><td width="40%">MS MPEG-4v1 (not ISO MPEG-4)</td></tr> <tr><td width="40%">MP42</td><td width="40%">MS MPEG-4v2</td></tr> <tr><td width="40%">MP43</td><td width="40%">MS MPEG-4v3</td></tr> <tr><td width="40%">MP4V</td><td width="40%">ISO MPEG-4 Part 2 Video (from old encoders)</td></tr> <tr><td width="40%">mpg1</td><td width="40%">ISO MPEG-1 Video</td></tr> <tr><td width="40%">mpg2</td><td width="40%">ISO MPEG-2 Video</td></tr> <tr><td width="40%">MRLE</td><td width="40%">MS RLE</td></tr> <tr><td width="40%">MSVC</td><td width="40%">MS Video 1</td></tr> <tr><td width="40%">RT21</td><td width="40%">Indeo 2.1</td></tr> <tr><td width="40%">RV10</td><td width="40%">RealVideo 1.0</td></tr> <tr><td width="40%">RV20</td><td width="40%">RealVideo 2.0</td></tr> <tr><td width="40%">RV30</td><td

width=”40%”>RealVideo 3.0</td></tr> <tr><td width=”40%”>RV40</td><td width=”40%”>RealVideo 4.0</td></tr> <tr><td width=”40%”>SNOW</td><td width=”40%”>FFmpeg Snow</td></tr> <tr><td width=”40%”>SVQ1</td><td width=”40%”>Sorenson Video 1</td></tr> <tr><td width=”40%”>SVQ3</td><td width=”40%”>Sorenson Video 3</td></tr> <tr><td width=”40%”>theo</td><td width=”40%”>Xiph Theora</td></tr> <tr><td width=”40%”>TM20</td><td width=”40%”>Truemotion 2.0</td></tr> <tr><td width=”40%”>UMP4</td><td width=”40%”>non-compliant MPEG-4 generated by UB Video MPEG-4</td></tr> <tr><td width=”40%”>VCR1</td><td width=”40%”>ATI VCR1</td></tr> <tr><td width=”40%”>VP30</td><td width=”40%”>VP 3.0</td></tr> <tr><td width=”40%”>VP31</td><td width=”40%”>VP 3.1</td></tr> <tr><td width=”40%”>VP50</td><td width=”40%”>VP 5.0</td></tr> <tr><td width=”40%”>VP60</td><td width=”40%”>VP 6.0</td></tr> <tr><td width=”40%”>VP61</td><td width=”40%”>VP 6.1</td></tr> <tr><td width=”40%”>VP62</td><td width=”40%”>VP 6.2</td></tr> <tr><td width=”40%”>VP70</td><td width=”40%”>VP 7.0</td></tr> <tr><td width=”40%”>WMV1</td><td width=”40%”>MS WMV7</td></tr> <tr><td width=”40%”>WMV2</td><td width=”40%”>MS WMV8</td></tr> <tr><td width=”40%”>WMV3</td><td width=”40%”>MS WMV9</td></tr> <tr><td width=”40%”>WV1F</td><td width=”40%”>non-compliant MPEG-4 generated by ?</td></tr> <tr><td width=”40%”>WVC1</td><td width=”40%”>VC-1</td></tr> <tr><td width=”40%”>XVID</td><td width=”40%”>non-compliant MPEG-4 generated by old Xvid</td></tr> <tr><td width=”40%”>XVIX</td><td width=”40%”>non-compliant MPEG-4 generated by old Xvid with interlacing bug</td></tr> </tbody> </table> <p style=”font-size: small;”> This document was generated using makeinfo. </p> </body> </html>

===== .\ffmpeg\doc\platform.html ===== <!DOCTYPE html PUBLIC ”-//W3C//DTD HTML 4.01 Transitional//EN” ”http://www.w3.org/TR/html4/loose.dtd”> <html> <!-- Created by GNU Texinfo 7.2, https://www.gnu.org/software/texinfo/ --> <head> <meta charset=”utf-8”> <title> Platform Specific Information </title> <meta name=”viewport” content=”width=device-width,initial-scale=1.0”> <link rel=”stylesheet” type=”text/css” href=”bootstrap.min.css”> <link rel=”stylesheet” type=”text/css” href=”style.min.css”> </head> <body> <h1> Platform Specific Information </h1>

 <h2 class=”contents-heading”>Table of Contents</h2>

<ul class=”toc-numbered-mark”> 1 Unix-like <ul class=”toc-numbered-mark”> 1.1 Advanced

linking configuration 1.2 BSD 1.3 (Open)Solaris 1.4 Darwin (Mac OS X, iPhone) 2 DOS 3 OS/2 4 Windows <ul class="toc-numbered-mark"> 4.1 Native Windows compilation using MinGW or MinGW-w64 <ul class="toc-numbered-mark"> 4.1.1 Native Windows compilation using MSYS2 4.2 Microsoft Visual C++ or Intel C++ Compiler for Windows <ul class="toc-numbered-mark"> 4.2.1 Linking to FFmpeg with Microsoft Visual C++ 4.3 Cross compilation for Windows with Linux 4.4 Compilation under Cygwin 4.5 Crosscompilation for Windows under Cygwin 4.6 ARM64EC

 <h2 class="chapter">1 Unix-like# TOC</h2>

 <h3 class="section">1.1 Advanced linking configuration# TOC</h3>

If you compiled FFmpeg libraries statically and you want to use them to build your own shared library, you may need to force PIC support (with <code class="code">--enable-pic</code> during FFmpeg configure) and add the following option to your project LDFLAGS: <pre class="example-preformatted">-Wl,-Bsymbolic </pre>

If your target platform requires position independent binaries, you should

pass the correct linking flag (e.g. <code class=”code”>-pie</code>) to <code class=”code”>--extra-ldexeflags</code>. <h3 class=”section”>1.2 BSD# TOC</h3>

BSD make will not build FFmpeg, you need to install and use GNU Make (<code class=”command”>gmake</code>). <h3 class=”section”>1.3 (Open)Solaris# TOC</h3>

GNU Make is required to build FFmpeg, so you have to invoke (<code class=”command”>gmake</code>), standard Solaris Make will not work. <h3 class=”section”>1.4 Darwin (Mac OS X, iPhone)# TOC</h3>

The toolchain provided with Xcode is sufficient to build the basic unaccelerated code. macOS on x86 requires <code class=”command”>nasm</code> to build most of the optimized assembly functions. Gentoo Prefix, Homebrew or MacPorts can easily provide it.

 <h2 class=”chapter”>2 DOS# TOC</h2>

Using a cross-compiler is preferred for various reasons. http://www.delorie.com/howto/djgpp/linux-x-djgpp.html

 <h2 class=”chapter”>3 OS/2# TOC</h2>

For information about compiling FFmpeg on OS/2 see http://www.edm2.com/index.php/FFmpeg.

 <h2 class=”chapter”>4 Windows# TOC</h2>

<h3 class="section">4.1 Native Windows compilation using MinGW or MinGW-w64# TOC</h3>

FFmpeg can be built to run natively on Windows using the MinGW-w64 toolchain. Install the latest versions of MSYS2 and MinGW-w64 from https://www.msys2.org/ and/or https://www.mingw-w64.org/. You can find detailed installation instructions in the download section and the FAQ. Notes: <ul class="itemize mark-bullet"> Building for the MSYS environment is discouraged, use one of the MinGW-w64 environments provided by MSYS2 instead. UCRT64 provides a GCC based toolchain, CLANG64 a LLVM based one and CLANGARM64 targets Windows on ARM64.

Building using MSYS2 can be sped up by disabling implicit rules in the Makefile by calling <code class="code">make -r</code> instead of plain <code class="code">make</code>. This speed up is close to non-existent for normal one-off builds and is only noticeable when running make for a second time (for example during <code class="code">make install</code>).

In order to compile FFplay, you must have the MinGW development library of SDL and <code class="code">pkg-config</code> installed.

By using <code class="code">./configure --enable-shared</code> when configuring FFmpeg, you can build the FFmpeg libraries (e.g. libavutil, libavcodec, libavformat) as DLLs.

 <h4 class="subsection">4.1.1 Native Windows compilation using MSYS2# TOC</h4>

The MSYS2 MinGW-w64 environments provide ready to use toolchains and dependencies through <code class="command">pacman</code>. Launch the shell of the environment you want to build in using its launcher, for example <samp class="file">ucrt64.exe</samp> or the corresponding start menu shortcut for UCRT64. Then install the toolchain and the FFmpeg dependencies. The <code class="command">pacboy</code> wrapper from the <code class="code">pactoys</code> package fills in the package name prefix of the current environment, so the same command works in every environment.

  <pre class=”example-preformatted”># normal msys2 packages pacman -S make pkgconf diffutils pactoys

mingw-w64 packages and toolchains for the current environment pacboy -S cc nasm SDL2 </pre>

 <h3 class=”section”>4.2 Microsoft Visual C++ or Intel C++ Compiler for Windows# TOC</h3>

FFmpeg can be built with MSVC 2019 16.8 or later, the first version able to compile C11. You will need the following prerequisites: <ul class=”itemize mark-bullet”> MSYS2 NASM (Also available via MSYS2’s package manager.)

To set up a proper environment in MSYS2, you need to run <code class=”code”>msys2_shell.cmd -use-full-path</code> from the Visual Studio or Intel Compiler command prompt. Place <code class=”code”>nasm.exe</code> somewhere in your <code class=”code”>PATH</code>. Next, make sure any other headers and libs you want to use, such as zlib, are located in a spot that the compiler can see. Do so by modifying the <code class=”code”>LIB</code> and <code class=”code”>INCLUDE</code> environment variables to include the Windows-style paths to these directories. Alternatively, you can try to use the <code class=”code”>--extra-cflags</code>/<code class=”code”>--extra-ldflags</code> configure options. Finally, run: <pre class=”example-preformatted”>For MSVC: ./configure --toolchain=msvc

For ICL: ./configure --toolchain=icl

make make install </pre>

If you wish to compile shared libraries, add <code class=”code”>--enable-shared</code> to your configure options. Note that due to the way MSVC and ICL handle DLL imports and exports, you cannot compile static and shared libraries at the same time, and enabling shared libraries will automatically disable the static ones. Notes: <ul class=”itemize mark-bullet”> If you wish to build with zlib support, you will have to grab a compatible zlib binary from somewhere, with an MSVC import lib, or if you wish to link statically, you can follow the instructions below to build a compatible <code class=”code”>zlib.lib</code> with MSVC. Regardless of which method you use, you must still fol-

low step 3, or compilation will fail. <ol class="enumerate"> Grab the zlib sources. Edit <code class="code">win32/Makefile.msc</code> so that it uses -MT instead of -MD, since this is how FFmpeg is built as well. Edit <code class="code">zconf.h</code> and remove its inclusion of <code class="code">unistd.h</code>. This gets erroneously included when building FFmpeg. Run <code class="code">nmake -f win32/Makefile.msc</code>. Move <code class="code">zlib.lib</code>, <code class="code">zconf.h</code>, and <code class="code">zlib.h</code> to somewhere MSVC can see.

 <h4 class="subsection">4.2.1 Linking to FFmpeg with Microsoft Visual C++# TOC</h4>

If you plan to link with MSVC-built static libraries, you will need to make sure you have <code class="code">Runtime Library</code> set to <code class="code">Multi-threaded (/MT)</code> in your project’s settings. If you plan on using import libraries created by dlltool, you must set <code class="code">References</code> to <code class="code">No (/OPT:NOREF)</code> under the linker optimization settings, otherwise the resulting binaries will fail during runtime. This is not required when using import libraries generated by <code class="code">lib.exe</code>. This issue is reported upstream at <a class="url" href="http://sourceware.org/bugzilla/show_bug.cgi?id=12633">http://sourceware.org/bugzilla/s
 To create import libraries that work with the <code class="code">/OPT:REF</code> option (which is enabled by default in Release mode), follow these steps: <ol class="enumerate"> Open the Visual Studio Command Prompt.

Alternatively, in a normal command line prompt, call <samp class="file">vcvarsall.bat</samp> which sets up the environment variables for the Visual C++ tools. Enter the <samp class="file">bin</samp> directory where the created LIB and DLL files are stored.

 Generate new import libraries with <code class="command">lib.exe</code>:

 <pre class="example-preformatted">lib /machine:i386 /def:..\lib\foo-version.def /out:foo.lib </pre>

Replace <code class="code">foo-version</code> and <code class="code">foo</code> with the respective library names.

 <h3 class="section">4.3

Cross compilation for Windows with Linux# TOC</h3>

You must use the MinGW-w64 cross compilation tools, which most Linux distributions provide as packages. Then configure FFmpeg with the following options: <pre class=”example-preformatted”>./configure --arch=x86_64 --target-os=mingw32 --cross-prefix=x86_64-w64-mingw32- </pre> (you can change the cross-prefix according to the prefix chosen for the MinGW-w64 tools). Then you can easily test FFmpeg with Wine. <h3 class=”section”>4.4 Compilation under Cygwin# TOC</h3>

Install your Cygwin with all the "Base" packages, plus the following "Devel" ones: <pre class=”example-preformatted”>binutils, gcc-core, make, git, texinfo </pre>

In order to run FATE you will also need the following "Utils" packages: <pre class=”example-preformatted”>diffutils </pre>

If you want to build FFmpeg with additional libraries, download Cygwin "Devel" packages for Ogg and Vorbis from any Cygwin packages repository: <pre class=”example-preformatted”>libogg-devel, libvorbis-devel </pre>

 <h3 class=”section”>4.5 Crosscompilation for Windows under Cygwin# TOC</h3>

With Cygwin you can create Windows binaries that do not need the cygwin1.dll. Just install your Cygwin as explained before, plus the MinGW-w64 cross toolchain "Devel" packages: <pre class=”example-preformatted”>mingw64-x86_64-gcc-core, mingw64-x86_64-binutils </pre>

and configure with: <pre class=”example-preformatted”>./configure --arch=x86_64 --target-os=mingw32 --cross-prefix=x86_64-w64-mingw32- </pre>

 <h3 class="section">4.6 ARM64EC# TOC</h3>

FFmpeg does not intend to support the Windows ARM64EC build configuration; patches for changing the individual libraries for the purposes of ARM64EC will not be accepted. It may still be possible to build FFmpeg in this build configuration; such a build may seem to work to some extent. Such a build may have some amounts of ABI inconsistencies though - which we are not willing to fix. (Changing aarch64 assembly code to strictly support ARM64EC would require ifdeffing/changing essentially all such assembly code, and would be a huge maintenance burden. Additionally, changing the library ABI surfaces to fix ABI inconsistencies could require extensive and intrusive changes.)   This document was generated using makeinfo.   </body> </html>

===== .\ffmpeg\doc\style.min.css ===== /*! The MIT License (MIT)

Copyright (c) 2014 Barbara Lepage <db0company@gmail.com>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE. */body{background-color:#313131;color:#e6e6e6;text-align:justify}body, h1, h2, h3, h4, h5, h6{font-family:"Lucida Grande","Lucida Sans Unicode","Lucida Sans","Helvetica Neue",Helvetica,Verdana,Tahoma,sans-serif}a{color:#4cae4c}a strong{color:#e6e6e6}a:hover{color:#7fc77f}a:hover strong{color:#4cae4c}main{width:100% ! important;min-height:600px;margin:auto}h1, h2, h3, h4{font-weight:bold;text-align:left}h1, h2, h3{color:#bebebe}h1 strong, h2 strong, h3 strong{color:#e6e6e6}h4, h5, h6{color:#3c8b3c}h1{border-

bottom:4px #bebebe solid;padding:20px 2%}h3{border-bottom:2px #bebebe solid;padding:15px 1%}h4{border-bottom:1px solid #e6e6e6;padding:10px 0;margin:20px 0;color:#e6e6e6}.list-group .list-group-item{background-color:#3e3e3e;border-color:black}.list-group.list-group-big .list-group-item{padding:25px}.list-group a.list-group-item{color:#7fc77f}.list-group a.list-group-item:hover{background-color:#313131;color:#4cae4c}.well{background-color:#242424;border-color:black;color:#bebebe}.well strong{color:#e6e6e6}.well code{background-color:#313131}.well hr{border-color:#3c8b3c}.well h3{margin:5px 0 15px 0;border:0;padding:0}.well a{color:#4cae4c}.well a.btn{color:white}.well small{display:block;padding:0 10px;font-style:italic}.well.example{padding-top:40px;margin-bottom:130px}.well.example pre{margin:50px;margin-bottom:30px;font-size:1.5em}.well.example .btn{margin-right:50px;margin-bottom:20px}.well.well-with-icon{min-height:136px}.well.well-with-icon .pull-right,.well.well-with-icon .pull-left{background-color:#4cae4c;color:#e6e6e6;padding:10px;borde radius:5px;margin:5px}.well.well-with-icon .pull-right{margin-left:20px}.well.well-with-icon .pull-left{margin-right:20px}a.well{display:block}a.well:hover{text-decoration:none;opacity:0.8}.info, .warning{margin:10px;padding:10px;background-color:#3e3e3e;color:#e6e6e6}.info code, .warning code{background-color:#313131}.info{border-left:10px #4cae4c solid}.warning{border-left:10px #ae4c4c solid}.with-icon{padding:30px}.with-icon .pull-left{padding-right:30px}.with-icon .pull-right{padding-left:30px}dd{margin-left:20px}code{background-color:#242424;color:#7fc77f;display:inline-block;margin:5px}.table{margin:20px 0;border-radius:4px}.table th,.table td,.table tr{border:1px solid #171717}.table tr th{background-color:#3e3e3e;border-bottom:2px solid #e6e6e6}.table tr:nth-child(odd){background-color:#242424}#sidebar-wrapper, .navbar{background-color:#171717;overflow-x:hidden}#sidebar-wrapper .sidebar-brand img,#sidebar-wrapper .navbar-brand img, .navbar .sidebar-brand img, .navbar .navbar-brand img{opacity:0.6;margin-right:8px}#sidebar-wrapper .sidebar-brand:hover,#sidebar-wrapper .navbar-brand:hover, .navbar .sidebar-brand:hover, .navbar .navbar-brand:hover{color:#fff}#sidebar-wrapper .sidebar-brand:hover img,#sidebar-wrapper .navbar-brand:hover img, .navbar .sidebar-brand:hover img, .navbar .navbar-brand:hover img{opacity:1}#sidebar-wrapper .sidebar-nav li ul, .navbar .sidebar-nav li ul{list-style-type:none;padding:0}#sidebar-wrapper .sidebar-nav li ul li, .navbar .sidebar-nav li ul li{line-height:20px}#sidebar-wrapper .sidebar-nav li ul li a, .navbar .sidebar-nav li ul li a{padding-left:20px}.content-header{height:auto;background-color:#242424}.content-header h1{color:#e6e6e6;display:block;margin:0;margi bottom:20px;line-height:normal;border-bottom:none}#download h4, #index h4{margin-top:180px}#download h4.first, #index h4.first{margin-top:20px}#download h4.first small, #index h4.first small{color:inherit;font-size:1em}#download .btn-download-wrapper, #index .btn-download-wrapper{text-align:center;margin:160px auto}#download .btn-download-wrapper .btn, #index .btn-download-wrapper .btn{font-size:3em;padding:3%;display:inline-block;margin-bottom:5px}#download .btn-download-wrapper small, #index .btn-download-wrapper small{display:block;font-size:0.4em}#download h2.description, #index h2.description{color:#e6e6e6;font-size:2em;font-weight:bold;margin:120px 50px;line-height:2em}#download h2.description .label, #index h2.description .label{font-size:0.5em}#download .btn-download-

wrapper{margin:40px auto}#download .os-selector{text-align:center;color:#e6e6e6;margin:30px 0}#download .os-selector a.btn-build{color:#e6e6e6;display:block;padding:20px;border-radius:2px}#download .os-selector .btn-build[href="#build-linux"]{background-color:#e43}#download .os-selector .btn-build[href="#build-linux"]:hover{color:#e43;background-color:#e6e6e6}#download .os-selector .btn-build[href="#build-windows"]{background-color:#06a}#download .os-selector .btn-build[href="#build-windows"]:hover{color:#06a;background-color:#e6e6e6}#download .os-selector .btn-build[href="#build-mac"]{background-color:darkgrey}#download .os-selector .btn-build[href="#build-mac"]:hover{color:darkgrey;background-color:#e6e6e6}#download .os-selector .tab-content{margin-top:20px}#download .os-selector #build-linux h3{color:#e43}#download .os-selector #build-windows h3{color:#06a}#download .os-selector #build-mac h3{color:darkgrey}footer{background-color:#242424;border-top:1px #101010 solid;padding:20px 0%}footer a{display:block}footer img[alt="FFmpeg"]{width:50%;display:block;margin:auto}.example.user-good pre{color:#bfbfbf;border:1px solid #357735;border-left:10px solid #357735;background-color:#242424}.example.user-bad pre{color:#bfbfbf;border:1px solid #ae4c4c;border-left:10px solid #ae4c4c;background-color:#242424}

===== .\static\index.html ===== <!DOCTYPE html> <html lang="id"><head><meta charset="UTF-8"><title>Claude Chat</title> <style> body{margin:0;display:flex;height:100vh;font-family:sans-serif} #side{width:260px;background:#f3f3f3;overflow:auto;padding:10px;box-sizing:border-box} #side>button{width:100%;padding:8px;margin-bottom:8px;cursor:pointer} .conv{display:flex;align-items:center;gap:4px;padding:8px;cursor:pointer;border-radius:6px} .conv:hover{background:#ddd}.conv.active{background:#cfe2ff} .conv span{flex:1;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;font-size:14px} .conv .del{border:none;background:none;cursor:pointer;opacity:.5}.conv .del:hover{opacity:1} #main{flex:1;display:flex;flex-direction:column;min-width:0} #msgs{flex:1;overflow:auto;padding:16px} .m{margin:8px 0;padding:10px;border-radius:8px;white-space:pre-wrap;word-wrap:break-word} .user{background:#e3f0ff}.assistant{background:#f6
.cost{font-size:11px;color:#888;margin-top:4px} #bar{display:flex;gap:6px;padding:10px;border-top:1px solid #ddd;align-items:center} #inp{flex:1;padding:8px} #bar label{cursor:pointer;font-size:20px} button:disabled{opacity:.5;cursor:wait} </style></head><body> <button onclick="newChat()">+ Chat Baru</button>   <select id="model"></select> <input id="inp" placeholder="Ketik pesan..." onkeydown="if(event.key==='Enter')send()"> <label title="Upload audio/video untuk ditranskrip"> <input type="file" id="file" accept="audio/*,video/*" hidden onchange="upload()"></label> <button id="btn" onclick="send()">Kirim</button>   <script> let cid=null, sibuk=false; const $=id=>document.getElementById(id); const fmt=c=>c&&c.usd!=null¿$${c.usd.toFixed(5)} (Rp${c.idr.toLocaleString('id')})`:'';

function add(role,text,meta){ const d=document.createElement('div');d.className='m
'+role;d.textContent=text; if(meta){const s=document.createElement('div');s.className='cost';s.textContent
$('msgs').appendChild(d);$('msgs').scrollTop=1e9; return d; } function setSibuk(v){sibuk=v;$('btn').disabled=v;$('inp').disabled=v;if(!v)$('inp').focus();}

async function loadModels(){ try{ const res=await fetch('/models');const

```
ms=await res.json(); if(!res.ok)throw new Error(ms.detail||'Gagal ambil model');
$('model').innerHTML=ms.map(m=>`<option>${m}</option>`).join('');
}catch(e){add('assistant',' '+e.message);} }

async function loadList(){ try{ const res=await fetch('/conversations');const
cs=await res.json(); $('list').innerHTML=''; cs.forEach(c=>{ const d=document.createElement('div');
d.className='conv'+(c.id===cid?' active':''); d.title=`Total biaya:
$${c.total_usd}`; const t=document.createElement('span');t.textContent=c.title||'(tanpa
judul)'; const x=document.createElement('button');x.className='del';x.textContent=' ';x.title='Hapus';
x.onclick=async ev=>{ ev.stopPropagation(); if(!confirm('Hapus chat
ini?'))return; await fetch('/conversations/'+c.id,{method:'DELETE'});
if(c.id===cid)newChat(); loadList(); }; d.onclick=()=>openConv(c.id);
d.append(t,x);$('list').appendChild(d); }); }catch(e){console.error(e);} }

async function openConv(id){ if(sibuk)return; cid=id;$('msgs').innerHTML='';
try{ const res=await fetch('/conversations/'+id);const ms=await res.json();
ms.forEach(m=>{ const meta=m.role==='assistant'&&m.input_tokens!=null
¿${m.input_tokens} in / ${m.output_tokens} out`+(m.cost_usd!=null¿
· $${m.cost_usd.toFixed(5)}`:'') :null; add(m.role,m.content,meta); });
}catch(e){add('assistant',' Gagal membuka chat: '+e.message);} loadList(); }

function newChat(){ if(sibuk)return; cid=null;$('msgs').innerHTML='';$('inp').focus();loadList();
}

async function send(text){ if(sibuk)return; text=text||$('inp').value.trim();if(!text)return;
$('inp').value='';add('user',text); const tunggu=add('assistant',' Claude
sedang mengetik...'); setSibuk(true); try{ const res=await fetch('/chat',{method:'POST',headers:{'Content-
Type':'application/json'}, body:JSON.stringify({conversation_id:cid,message:text,model:$('model').value||null
const r=await res.json(); tunggu.remove(); if(!res.ok){add('assistant','
'+(r.detail||'Error'));return;} cid=r.conversation_id; add('assistant',r.reply,`${r.usage.input_tokens}
in / ${r.usage.output_tokens} out · ${fmt(r.cost)}`); loadList(); }catch(e){
tunggu.remove(); add('assistant',' Tidak bisa terhubung ke server. Pastikan
jendela server masih terbuka.'); }finally{setSibuk(false);} }

async function upload(){ const f=$('file').files[0];if(!f||sibuk)return; add('user',`
Mentranskripsi ${f.name}... (pertama kali akan download model ±460MB,
harap sabar)`); setSibuk(true); let r=null; try{ const fd=new Form-
Data();fd.append('file',f); const res=await fetch('/transcribe',{method:'POST',body:fd});r=await
res.json(); if(!res.ok){add('assistant',' '+(r.detail||'Error'));r=null;} else
add('assistant',` Transkrip (${r.language}):\n${r.text}`); }catch(e){add('assistant','
Gagal transkripsi: '+e.message);} finally{$('file').value='';setSibuk(false);}
if(r&&r.text&&confirm('Kirim transkrip ke Claude untuk diringkas?'))
send('Ringkas transkrip berikut:\n\n'+r.text); }

loadModels();loadList(); </script></body></html>
```